

Semiconductor IP Design

Theory, Production Code, and the Practice of a Design House

A graduate curriculum for the design, modelling and verification of semiconductor IP blocks, written from first principles and grounded throughout in source code that is shipping in production silicon.

Corpus. 55,409 files, **19,632,596 lines** of real design and modelling code were collected and indexed mechanically. Every quantity quoted in this book is computed from that index; none is written from memory.

How to read. Green boxes recall undergraduate material; purple boxes carry graduate-level detail and the reasoning a practising engineer needs; red boxes mark places where real projects have failed.

Contents

A1. Signals, Systems and Linearity	5
A2. Transforms and Spectral Representation	8
A3. Sampling, Reconstruction and Quantisation	10
A4. Probability, Random Processes and Noise	12
A5. Linear Algebra and Numerical Methods for Hardware	14
B1. The MOSFET — What a Digital Designer Must Know	16
B2. CMOS Logic, Delay and Power	18
B3. Timing, Clocking and Static Timing Analysis	20
B4. Interconnect, Transmission Lines and Signal Integrity	22
B5. Analogue Building Blocks	25
C1. Arithmetic Units — The Core of Every Datapath	27
C2. Control: State Machines, Pipelines and Flow Control	30
C3. Clock Domain Crossing, Reset and Low Power	32
D1. Instruction Set Architecture	35
D2. Pipelining, Hazards and Speculation	37
D3. Memory Hierarchy and Coherence	39
E1. Digital Filters as Hardware	41
E2. Transforms in Hardware: FFT and Friends	44
F1. The Wireline Link, End to End	46
F2. Error Control Coding	49
G1. Electromagnetics for the Digital Engineer	52
G2. Radar: Signal Processing as an IP Block Family	54
H1. Verification: Theory and Organisation	57
H2. The Implementation Flow	60
H3. Hardware Security	62
W1. What a One-Person Design House Actually Sells	64
W2. The Solo Process, Stage by Stage	67
W3. Automation: Running Verification Around the Clock	70
Volume II — Production Source Code	72

AES 골든 모델 (C)	73
AES·SHA·MD5 C 참조 구현	82
AES 코어 (Verilog)	117
SHA-256 코어 (Verilog)	138
OpenTitan AES -- 양산 보안 RTL	148
OpenTitan HMAC-SHA256	194
OpenTitan KMAC (Keccak/SHA-3)	225
OpenTitan CSRNG (SP800-90A)	260
OpenTitan 엔트로피원 (SP800-90B)	287
OpenTitan OTBN -- 빅넘버 코프로세서	327
SPI 호스트	367
I2C 컨트롤러	402
UART	436
USB 디바이스	458
RISC-V 타이머	500
RISC-V 디버그 모듈	509
TL-UL 버스 구조	531
SRAM 컨트롤러(스크램블)	581
OTP 컨트롤러	597
키 관리자	600
수명주기 컨트롤러	634
플래시 컨트롤러	675
OpenTitan 기본 셀 라이브러리	676
Caliptra SHA-512	725
Caliptra SHA-256	760
Caliptra HMAC	795
Caliptra ECC (P-384)	829
Caliptra AES	865
PULP 기본 셀 (FIFO·CDC·아비터)	911
AXI4 상호연결	971
APB	1020

RISC-V 디버그	1035
CV32E40P RISC-V 코어	1070
CV32E40P RVFI 트레이스	1099
PicoRV32 -- 초소형 RISC-V	1137
SERV -- 비트직렬 RISC-V	1181
형식검증된 AXI/Wishbone 브리지	1222
PCIe 코어	1255
RVFI UVM 에이전트	1308
step-compare 테스트벤치	1327
Spike ISS -- 참조모델 본체	1356
Berkeley SoftFloat	1394
RISC-V 형식 명세 (Sail)	1440

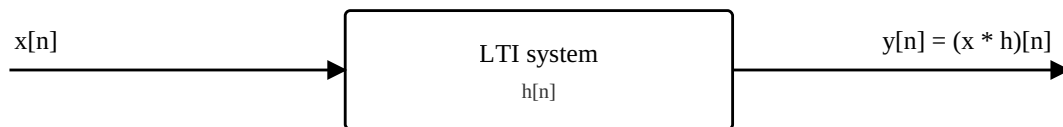
A1. Signals, Systems and Linearity

Every digital IP block is a *system*: it maps input sequences to output sequences. Before discussing any particular block we fix the vocabulary that makes such maps analysable, and — more importantly for a hardware engineer — we identify exactly which property of a system makes it **cheap** to build and which makes it **expensive**.

A1.1 Definitions

Table 1. System properties and their hardware consequences

Property	Definition	Hardware consequence
Memoryless	$y[n]$ depends only on $x[n]$	Pure combinational logic; no registers, no state to reset or scan
Causal	$y[n]$ depends only on $x[k]$, $k \leq n$	Required for real-time streaming. Non-causal filters are realised by adding latency , not by seeing the future
Linear	Superposition holds	Enables decomposition into impulse responses; permits frequency-domain design
Time-invariant	Delay in $\rightarrow$ same delay out	Coefficients are constants $\rightarrow$ can be hard-wired or stored once
BIBO stable	Bounded input $\rightarrow$ bounded output	$\sum h[n] < \infty$. In fixed point, determines whether accumulators can overflow
Passive lossless	/ Energy not increased	Guarantees no growth in word length



Linearity + time invariance $\Rightarrow$ one impulse response is the whole system

$Y(z) = H(z) \cdot X(z)$ — convolution becomes multiplication

Figure 1. An LTI system is completely characterised by its impulse response. This single fact underpins all filter design.

석사 수준 · The practical importance of linearity is not elegance but **separability of verification**. If a block is LTI you can characterise it with a single impulse and predict its response to every input; a random-stimulus campaign is then a redundancy check rather than the primary evidence. If it is *not* LTI — a decision-feedback equaliser, a μ -law compander, a saturating accumulator, any adaptive loop — superposition fails and you must verify over the operating region, which is exponentially more work. **The first architectural question about any new block is therefore "where exactly is the nonlinearity?"** Isolating it into the smallest possible sub-block is a verification cost decision, not an aesthetic one.

A1.2 Convolution and its cost

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n-k]$$

Table 2. Realisations of the same convolution

Method	Operations per output	Latency	When preferred
Direct convolution	N MACs	0 (plus pipeline)	$N \lesssim 64$
Overlap-add FFT	$\sim(2 \log_2 L + 1)$ per sample	One block (L samples)	Long filters; latency tolerable
Overlap-save FFT	Same order	One block	Streaming; avoids add stage
Frequency-domain adaptive	FFT + per-bin update	One block	Long adaptive filters (echo cancellation)
Distributed arithmetic	LUT reads only	W cycles	Fixed coefficients, LUT-rich fabric
Multiplierless (CSD)	Shifts and adds	0	Fixed coefficients, ASIC area critical

석사 수준 · The crossover between direct and FFT-based convolution is not a fixed number of taps; it depends on the **cost ratio of a multiplier to a memory word** in the target technology, and on whether block latency is acceptable. In an ASIC at an advanced node, multipliers are comparatively cheap and SRAM is comparatively expensive, which pushes the crossover *up*. On an FPGA with hard DSP slices and abundant block RAM it moves the other way. **Quoting a fixed crossover (“use FFT above 128 taps”) without naming the technology is a common and costly error in architecture documents.**

A1.3 Correlation, matched filtering and the origin of SNR gain

Cross-correlation is convolution with a time-reversed conjugated kernel. When the kernel is the transmitted pulse itself, the operation is a **matched filter**, and it maximises output SNR for a signal in additive white Gaussian noise.

$$SNR_{out,max} = 2E_s / N_0$$

석사 수준 · The result is worth internalising because it recurs in radar, in spread spectrum, in synchronisation preambles and in every correlator IP. **The gain comes from integrating signal coherently while noise adds incoherently:** over M samples, signal amplitude grows as M while noise amplitude grows as $\sqrt{M}$, hence power SNR improves by M ($10 \log_{10} M$ dB). Two engineering consequences follow. First, coherent integration requires **phase stability over the integration window**; if the oscillator drifts, gain saturates — this is the practical limit on radar integration time and on preamble length. Second, the correlator's dynamic range must accommodate the M -fold growth, which is a fixed-point word-growth problem, not a signal-processing one.

A1.4 Group delay, phase linearity and why FIR dominates

$$\tau_g(\omega) = -d\phi(\omega)/d\omega$$

학부 복습 · A filter has **linear phase** when τ_g is constant: all frequency components are delayed equally, so pulse shapes are preserved and only shifted. An FIR filter with symmetric (or antisymmetric) coefficients has exactly linear phase. An IIR filter cannot, except trivially.

석사 수준 · This is the decisive argument for FIR in data transmission. Non-constant group delay **spreads a pulse asymmetrically**, which is indistinguishable from channel dispersion and therefore directly closes the eye. In audio the ear is comparatively insensitive to phase and IIR is preferred for its efficiency; in a wireline receiver an IIR equaliser would introduce the very distortion it is meant to remove. **The same mathematics leads to opposite engineering choices in two domains, and knowing why is the difference between applying a rule and understanding it.** Note also the hardware corollary: symmetric FIR coefficients permit the *folded* structure that halves the multiplier count, so linear phase is not only free, it is cheaper.

A1.5 Stability in the Z domain

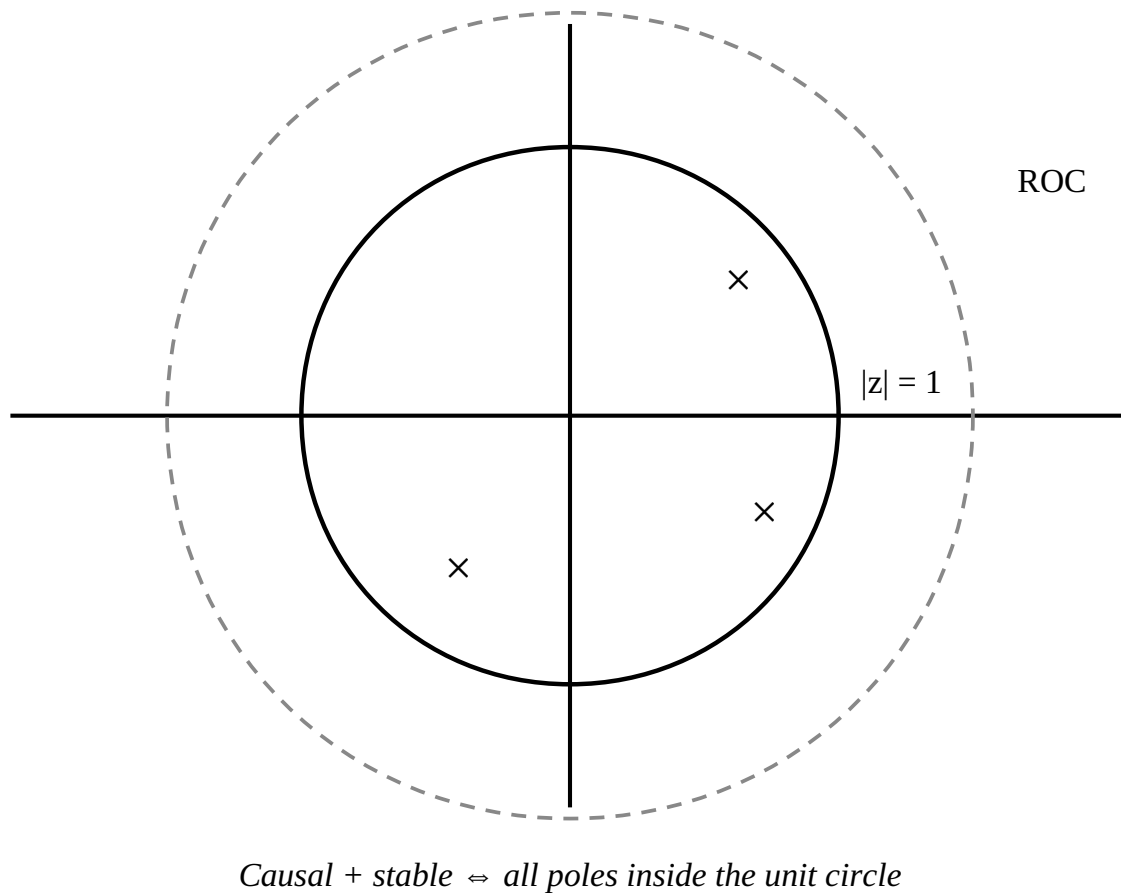


Figure 2. Pole-zero plot and region of convergence. For a causal system the ROC is outside the outermost pole; stability additionally requires that the ROC contain the unit circle.

Table 3. Pole location and behaviour

Pole radius $ p $	Behaviour	Fixed-point implication
$ p < 1$	Decaying; stable	Accumulator bounded by $1/(1- p)$
$ p = 1$	Marginally stable (oscillator, integrator)	Accumulator grows without bound — needs explicit leakage or reset
$ p > 1$	Divergent	Unusable
$ p \rightarrow 1^-$	Very narrow band; long memory	Coefficient quantisation can push the pole outside — the classic failure mode of high-order direct-form IIR

석사 수준 · Quantifying the last row: for a direct-form IIR of order N , the sensitivity of pole p_i to a denominator coefficient a_k is $\partial p_i / \partial a_k = -p_i^{N-k} / \prod_{j \neq i} (p_i - p_j)$. The denominator is a product of *pole separations*: **clustered poles make the sensitivity explode**. Cascaded second-order sections avoid this because each section contains only one conjugate pair, so no small separations appear in its own denominator. This is the quantitative reason behind the universal practice of implementing high-order IIR filters as cascaded biquads, and it is worth being able to state it rather than merely follow it.

A2. Transforms and Spectral Representation

Table 4. The transform family

Transform	Domain	Kernel	Principal use in IP work
CTFT	continuous time $\rightarrow \omega$	$e^{-j\omega t}$	Analogue filter and antenna analysis
Laplace	continuous time $\rightarrow s$	e^{-st}	PLL loop dynamics, control, transmission lines
DTFT	discrete time $\rightarrow \omega$	$e^{-j\omega n}$	Theoretical filter response (continuous in ω)
DFT / FFT	finite sequence $\rightarrow$ bins	$e^{-j2\pi kn/N}$	Everything computable
Z	discrete time $\rightarrow z$	z^{-n}	Poles, zeros, stability, filter structures
Hilbert	real $\rightarrow$ analytic	$-j \operatorname{sgn}(\omega)$	I/Q generation, envelope, SSB
DCT	finite $\rightarrow$ cosine basis	$\cos$	Image/video coding (energy compaction)
Wavelet	time–scale	scaled basis	Transient detection, JPEG2000

A2.1 The DFT is not the DTFT

The DFT computes samples of the DTFT of a **windowed, periodically extended** sequence. Three artefacts follow, and all three appear as bugs in practice.

Table 5. DFT artefacts

Artefact	Cause	Symptom	Mitigation
Spectral leakage	Implicit rectangular window	A pure tone smears across bins	Apply a window (Hann, Blackman–Harris)
Scalloping loss	Tone falls between bin centres	Up to 3.9 dB amplitude error (rectangular)	Window, or interpolate the peak
Picket fence	Only N samples of a continuum	Frequency resolution limited to f_s/N	Zero-pad (interpolates, does not add resolution)
Circular convolution	Periodic extension	Wrap-around in fast convolution	Zero-pad to $N \geq L+M-1$

Zero-padding is routinely misdescribed as improving resolution. It does not. **Resolution is set by the observation time** $T = N/f_s$; two tones closer than $\approx 1/T$ cannot be separated no matter how many zeros are appended. Padding interpolates the displayed spectrum, which makes peaks easier to *locate* but not easier to *resolve*. In a specification, resolution and interpolation must be stated separately.

Table 6. Common windows

Window	Main-lobe width (bins)	First sidelobe	Use
Rectangular	2	−13 dB	Transient capture; maximum resolution
Hann	4	−31 dB	General purpose
Hamming	4	−41 dB	Slightly better near sidelobe
Blackman–Harris (4-term)	8	−92 dB	Spur hunting in converter test
Kaiser (β adjustable)	tunable	tunable	Design-to-spec
Flat-top	~9	−90 dB	Amplitude accuracy (calibration)

A2.2 Parseval, power spectral density, and dB conventions

$$\sum_n |x[n]|^2 = (1/N) \sum_k |X[k]|^2$$

Table 7. Decibel conventions that are routinely confused

Unit	Reference	Note
dB	ratio	10 log for power, 20 log for amplitude
dBm	1 mW	Absolute power
dBc	carrier	Spur levels, phase noise
dBFS	converter full scale	Digital level; not a physical power
dBm/Hz	1 mW per Hz	Spectral density; thermal floor -174 dBm/Hz @ 290 K
dBc/Hz	carrier per Hz	Phase noise $L(\Delta f)$

석사 수준 · A recurring measurement error is comparing a **single-bin** FFT magnitude with a **spectral density**. The FFT bin contains power integrated over the equivalent noise bandwidth of the window, which for a Hann window is 1.5 bins, not 1. Reporting “noise floor at -110 dBFS” without stating N , f_s and the window is not a measurement, because the number changes when any of the three changes. **Processing gain $10 \log_{10}(N/2)$ is often mistaken for converter performance.**

A3. Sampling, Reconstruction and Quantisation

A3.1 The sampling theorem and its practical corollaries

$$X_s(f) = f_s \sum_k X(f - k f_s)$$

Table 8. Sampling regimes

Regime	Condition	Where used	Hardware cost
Nyquist	$f_s > 2B$, base-band	Most baseband DSP	Anti-alias filter must be steep
Oversampling (OSR > 1)	$f_s \gg 2B$	$\Sigma\Delta$ converters, relaxed AAF	Quantisation noise spread over wider band: +3 dB SNR per doubling
Bandpass / under-sampling	$2f_H/n \leq f_s \leq 2f_L/(n-1)$	RF direct sampling receivers	Sampler analogue bandwidth must exceed f_H , not f_s
Time-interleaved	M converters at f_s/M	>10 GS/s ADCs (112G SerDes uses 32–64 way)	Offset / gain / timing skew mismatch creates spurs at kf_s/M

석사 수준 · **Time-interleaving is where the design house earns its fee.** An M -way interleaved ADC has three mismatch mechanisms, each producing a distinct spur family: offset mismatch gives tones at kf_s/M independent of input; gain mismatch gives sidebands at $kf_s/M \pm f_{in}$; timing skew gives the same sideband locations but with amplitude proportional to f_{in} . **The frequency dependence is the diagnostic:** sweep the input and see which spur grows. Digital background calibration of all three is a standard deliverable in a modern ADC-DSP receiver, and it is a *digital* block — which means a fabless digital IP vendor can sell it even though the converter itself is analogue. This is a concrete example of finding a sellable digital block adjacent to an analogue one you cannot build.

A3.2 Quantisation noise

$$SQNR_{dB} = 6.02 B + 1.76 + 10 \log_{10}(OSR)$$

Table 9. Where the ideal formula breaks

Effect	Mechanism	Consequence	Design response
Aperture jitter	Sampling instant uncertainty σ_t	$SNR = -20 \log_{10}(2\pi f_{in} \sigma_t)$	Dominates above ~1 GHz input; clock design becomes the ADC spec
Thermal noise	KT/C of sampling capacitor	Floors SNR regardless of bits	Increase C (costs power and bandwidth)
DNL / INL	Element mismatch	Harmonic distortion, missing codes	Calibration, dynamic element matching
Comparator meta-stability	Finite regeneration time	Rare large errors	More regeneration time or error detection
Correlated quantisation	Low-amplitude periodic input	Quantisation noise becomes tonal, not white	Dither — deliberately added noise to decorrelate

석사 수준 · The dither entry deserves emphasis because it is counter-intuitive: **adding noise improves the result.** Quantisation error is only white and signal-independent when the signal moves over many LSBs; for a small or slowly varying signal

the error becomes periodic and produces discrete tones, which are far more objectionable than an equivalent amount of broad-band noise (in audio they are audible; in a radar they look like targets). Subtractive dither — add a known pseudo-random sequence before quantising and subtract it after — removes the tones without the SNR penalty. **A dither generator is a small, precisely specifiable digital block and therefore exactly the kind of thing a one-person IP house can build and sell as part of a converter-adjacent package.**

A3.3 Reconstruction

Table 10. Reconstruction effects

Effect	Expression	Compensation
Zero-order hold droop	$\text{sinc}(f/f_s)$	−3.92 dB at Nyquist; pre-compensate with an inverse-sinc FIR
Image replicas	at $kf_s \pm f$	Reconstruction (anti-imaging) filter
Interpolation	up-sample then low-pass	Moves images away so the analogue filter can be gentle
Mismatch-shaping DAC	—	Pushes element mismatch error out of band; a digital technique for an analogue defect

A4. Probability, Random Processes and Noise

A4.1 Distributions that occur in hardware

Table 11. Distributions and where they arise

Distribution	Arises from	Appears in
Gaussian	Sum of many small independent effects (CLT)	Thermal noise, random jitter, process variation
Rayleigh	Magnitude of complex Gaussian	Fading envelope, noise magnitude
Rician	Complex Gaussian + deterministic component	Fading with line of sight
Poisson	Rare independent events	Shot noise, soft-error (SER) counts
Uniform	Quantisation residue (ideal)	Quantisation noise model
Exponential	Waiting time between Poisson events	Time between soft errors
Log-normal	Product of many factors	Shadowing, some reliability lifetimes
Weibull	Weakest-link failure	Oxide breakdown, reliability
Binomial	n Bernoulli trials	Bit errors in a block, yield
Chi-square	Sum of squared Gaussians	Energy detection, CFAR statistics

A4.2 Physical noise mechanisms

Table 12. Noise sources in silicon

Mechanism	Spectrum	Expression	Design lever
Thermal (Johnson)	White	$v_n^2 = 4kTR\Delta f$	Lower R; lower T; narrower band
Channel thermal (MOS)	White	$i_n^2 = 4kTyg_m\Delta f$	Higher g_m improves noise <i>figure</i> though noise power rises
Flicker (1/f)	1/f	$\propto K/(WLC_{ox}f)$	Larger devices ; chopping; auto-zeroing
Shot	White	$i_n^2 = 2qI\Delta f$	Present where carriers cross a barrier (junctions, photodiodes)
kT/C	Sampled	$v^2 = kT/C$	Independent of resistance ; only C helps
Substrate / supply coupling	Structured	—	Guard rings, separate supplies, differential design
Phase noise	$1/f^3$, $1/f^2$, flat	Leeson model	Higher Q, higher power, lower flicker

석사 수준 · **Why flicker noise sets the architecture of precision analogue.** $1/f$ noise is not reducible by filtering, because it lives at the frequencies where the signal of interest often lives (DC to kHz). The standard responses — chopping and auto-zeroing — both work by *moving the signal to a higher frequency where only white noise remains*, and both require digital support: a chopping clock, demodulation, and residual-ripple filtering. In a modern mixed-signal IP the digital half of that scheme is a well-defined block with a clean interface, and it is again a piece that a digital design house can own.

A4.3 Jitter and phase noise — the same thing in two languages

$$\sigma_t^2 = (1/(2\pi f_0)^2) \int 2 L(f) df$$

Table 13. Jitter taxonomy used in link specifications

Type	Symbol	Statistics	Cause	How it is separated
Random jitter	RJ	Gaussian, unbounded	Thermal, flicker	Fit tails of the bathtub curve
Deterministic jitter	DJ	Bounded	—	Peak-to-peak; decomposed below
Periodic	PJ	Sinusoidal	Supply coupling, spurs	FFT of the jitter sequence
Data-dependent	DDJ / ISI	Pattern-correlated	Channel loss	Compare per-pattern edge times
Duty-cycle distortion	DCD	Two-valued	Rise/fall asymmetry, threshold offset	Even/odd UI comparison
Bounded uncorrelated	BUJ	Bounded, random	Crosstalk	Statistical separation
Total jitter at BER	TJ(BER)	RJ⊗DJ convolution	—	$TJ = DJ + 2 \cdot Q(BER) \cdot RJ_{rms}$

석사 수준 · The last row is the one that appears in every SerDes specification, and the factor $Q(BER)$ is why: at $BER\ 10^{-12}$, $Q = 7.03$, so $TJ = DJ + 14.07\ RJ_{rms}$. **Random jitter is multiplied by fourteen.** This is why a small improvement in oscillator or phase noise is worth a large amount of design effort, and why jitter separation — distinguishing RJ from DJ in a measurement — is a required capability rather than a nicety. A modelling engineer asked to produce a link budget must implement this convolution correctly; adding RMS values in quadrature, which is the intuitive move, is wrong because DJ is bounded and does not add that way.

A4.4 Random processes

Table 14. Concepts and their use

Concept	Statement	Use in IP design
Stationarity (WSS)	Mean constant, autocorrelation depends on lag only	Precondition for spectral analysis
Ergodicity	Time average = ensemble average	Justifies measuring one long record instead of many devices
Wiener–Khinchin	PSD = FT of autocorrelation	Links time and spectrum
Markov chain	Memoryless state evolution	Bus arbitration analysis, FIFO occupancy, error models
Gauss–Markov	Correlated Gaussian	Coloured noise models
Cyclostationarity	Statistics periodic in time	All digitally modulated signals ; exploited for blind synchronisation
Point process	Random event times	Soft errors, packet arrivals

석사 수준 · Cyclostationarity is the theoretical basis for **blind** timing and carrier recovery: a modulated signal has periodically varying statistics at the symbol rate, so a suitable nonlinearity produces a spectral line at that rate even though the signal itself has no such line. Gardner and Mueller–Müller timing detectors are practical exploitations of this. Knowing the principle rather than the recipe is what lets an engineer design a detector for a modulation format that has no textbook detector — a situation that occurs constantly in proprietary links.

A5. Linear Algebra and Numerical Methods for Hardware

A5.1 Decompositions and their hardware character

Table 15. Matrix decompositions as IP blocks

Decomposition	Form	Cost	Streaming?	Numerical character
LU	$A = LU$	$n^3/3$	No (pivoting needs lookahead)	Needs pivoting for stability
Cholesky	$A = LL^*$	$n^3/6$	Yes	No pivoting needed for SPD; half the work of LU
QR (Givens)	$A = QR$	$2mn^2$	Yes	Backward stable; rotations are well conditioned
QR (Householder)	same	fewer ops	Less so	Best stability
SVD	$A = U\Sigma V^*$	iterative	No	Most robust ; reveals rank and conditioning
Eigen (symmetric)	$A = Q\Lambda Q^*$	iterative	No	Jacobi parallelises well
Schur / Hessenberg	—	n^3	No	Control applications

석사 수준 · **The streaming column is the commercial column.** Cholesky and Givens QR touch each input element once and can therefore be built with an `hls::stream`-style interface and no external memory; SVD and LU must revisit the array and therefore need an AXI master and a buffer. That single architectural difference changes the IP's integration story, its area, and its price. When a customer asks for “a matrix inverse block”, the first engineering question is whether the matrix is symmetric positive definite — because if it is, Cholesky gives a streaming solution at half the arithmetic, and if it is not, you are quoting a much larger block.

A5.2 Conditioning

$$\kappa(A) = \sigma_{\max}/\sigma_{\min} \quad \text{relative error} \lesssim \kappa(A) \cdot \epsilon_{\text{machine}}$$

석사 수준 · The rule of thumb that follows is directly usable: **you lose roughly $\log_{10}\kappa$ decimal digits**. In fixed point, substitute $\epsilon = 2^{-F}$ for F fractional bits and the statement becomes a word-length requirement: $F \gtrsim \log_2\kappa + (\text{bits of accuracy required})$. This is how a modelling engineer converts a mathematical property of the customer's data into a hardware parameter. It also explains why **iterative refinement** works: solve in low precision, compute the residual in high precision, correct, repeat. Each iteration roughly squares the accuracy, so three refinements from a rough start can reach near-full precision — but only if the condition number is modest, because κ enters the convergence factor. **Measured behaviour: for a well-conditioned (isotropic) system the scheme reached 24.2 bits of accuracy; at correlation 0.99 it reached only 12.4 bits.** An IP that advertises mixed-precision refinement must therefore state a conditioning limit, or it will fail at a customer whose data is correlated.

A5.3 Iterative solvers and their hardware mapping

Table 16. Iterative methods

Method	Update	Parallelism	Convergence
Jacobi	uses only previous iterate	Fully parallel	Slow; needs diagonal dominance
Gauss–Seidel	uses updated values immediately	Sequential within a sweep	Roughly twice Jacobi's rate
SOR	over-relaxed GS	Sequential	Tunable ω

Method	Update	Parallelism	Convergence
Conjugate gradient	Krylov subspace	Parallel (dot products)	n steps exactly; $\sqrt{k}$ in practice
Richardson	$x \leftarrow x + \alpha(b - Ax)$	Fully parallel	Simplest; α must be tuned

석사 수준 · There is a physical realisation worth knowing because it is repeatedly rediscovered and mis-described. An RC network that solves $C \, d\mathbf{v}/dt = -(\mathbf{A}\mathbf{v} - \mathbf{b})$ settles to $\mathbf{A}^{-1}\mathbf{b}$, and is often presented as an “analogue Gauss–Seidel solver”. **It is not:** because all nodes evolve simultaneously from the same previous state, the continuous-time limit corresponds to **Jacobi / Richardson**, not Gauss–Seidel, which is defined by sequential in-sweep updates. The settling time is governed by $1/\lambda_{\min}$, i.e. by the condition number again. Under a normalisation that fixes $\lambda_{\max} = 1$, the measured settling-time-to-condition ratio was constant at 13.2 — the analogue solver buys constant factors, not asymptotic ones. **Being able to state this precisely prevents a design house from accepting a proposal whose speedup claim rests on a misidentification.**

B1. The MOSFET — What a Digital Designer Must Know

A digital IP engineer does not size transistors, but every timing number, every power figure and several verification hazards trace back to device behaviour. This chapter keeps only what changes decisions.

B1.1 Regions of operation

Table 17. MOSFET regions

Region	Condition	Current	Where it matters
Cut-off	$V_{GS} < V_{TH}$	$I_{sub} \propto e^{(V_{GS}-V_{TH})/nV_T}$	Subthreshold leakage — the dominant static power term below 65 nm
Triode / linear	$V_{DS} < V_{GS}-V_{TH}$	$\mu C_{ox}(W/L)[(V_{GS}-V_{TH})V_{DS} - V_{DS}^2/2]$	Switch resistance R_{on} ; pass gates; the ON device of an inverter at end of transition
Saturation	$V_{DS} > V_{GS}-V_{TH}$	$\frac{1}{2}\mu C_{ox}(W/L)(V_{GS}-V_{TH})^2(1+\lambda V_{DS})$	Analogue gain stages; the beginning of a digital transition
Velocity saturation	Short channel, high field	$\propto (V_{GS}-V_{TH})^1$ — linear, not square	Modern nodes live here. Textbook square-law intuition overestimates drive

석사 수준 · The last row is the single most important correction for anyone who learned the square law. In a short-channel device carriers reach their saturation velocity, so drive current becomes roughly proportional to overdrive rather than its square. Two practical consequences: **(i)** increasing supply voltage buys less speed than square-law intuition suggests, while dynamic power still rises as V^2 , so voltage scaling is a poor speed lever; **(ii)** stacking devices (as in a NAND with many inputs) costs more than the naive series-resistance estimate, which is why wide fan-in gates are avoided and logic is restructured into trees. When a synthesis tool reports that a 4-input gate is slower than two 2-input gates in series, this is why.

B1.2 Threshold voltage and its enemies

Table 18. V_{TH} modulation effects

Effect	Mechanism	Design consequence
Body effect	Source–bulk bias raises V_{TH}	Stacked devices are slower than expected
DIBL	Drain field lowers the barrier	V_{TH} falls with $V_{DS} \rightarrow$ more leakage
Random dopant fluctuation	Discrete dopant atoms	$\sigma_{V_{TH}} \propto 1/\sqrt{WL}$ — small devices mismatch more
Temperature	–1 to –2 mV/K	Hot chip is <i>faster</i> at low V_{DD} (temperature inversion)
NBTI / PBTI ageing	Trap generation under stress	V_{TH} drifts over years — timing must include an ageing margin
Multi- V_{TH} offering	Process option	LVT for critical paths, HVT elsewhere: the primary leakage-vs-speed lever

석사 수준 · **Temperature inversion is a verification trap.** At high supply voltage a chip is slowest when hot, so signing off at the high-temperature corner is conservative. At low supply voltage — which is where modern low-power modes operate — the V_{TH} reduction outweighs mobility degradation and the chip is slowest when **cold**. Teams that carried forward a "hot is

worst case" habit have taped out parts that fail at $-40\text{ }^{\circ}\text{C}$. The corner list is not a formality; it is a statement about which physics dominates in the operating region you are selling into.

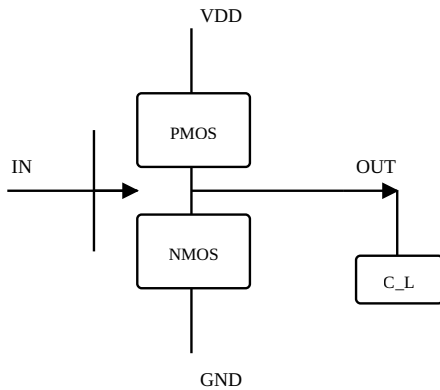
B1.3 Scaling and what it stopped giving

Table 19. Dennard scaling and its breakdown

Quantity	Ideal scaling (factor $s > 1$)	What actually happened
Dimensions	$1/s$	Continued (with FinFET, GAA)
Supply V_{DD}	$1/s$	Stalled near 0.7–0.9 V — V_{TH} cannot follow without leakage explosion
Delay	$1/s$	Slower than ideal; interconnect does not scale
Power density	constant	Rose → dark silicon, thermal limits
Leakage	negligible	Comparable to dynamic below 65 nm
Interconnect RC	—	Worsens : resistance rises as cross-section shrinks

석사 수준 · The end of Dennard scaling is the reason the industry turned to **specialised accelerators**, and therefore the reason the IP business grew. When a single general-purpose core could no longer be made faster within a power budget, the way to get performance became putting the right fixed-function block next to the processor. **Every FEC, crypto, codec and NPU IP sold today exists because of this graph.** Understanding it lets a design house argue its value proposition in the customer's own terms: not "our block is fast", but "your general-purpose alternative costs x times the energy per operation".

B2. CMOS Logic, Delay and Power



$$t_{pd} \approx 0.69 R_{on} C_L$$

$R_{on} \propto 1/(W/L) \Rightarrow$ bigger device, faster, more input cap

FO4 = delay driving four identical inverters

— the technology-independent speed unit

Figure 3. The inverter, and the delay model every timing number ultimately rests on.

B2.1 Delay models

Table 20. Delay models by fidelity

Model	Form	Use
RC (Elmore)	$t_{pd} = 0.69 R_{on} C_L$	Hand estimates, intuition
Logical effort	$d = gh + p$	Optimal gate sizing and stage count without simulation
NLDM (lookup)	table over input slew $\times$ output load	Standard-cell STA
CCS / ECSM	current-source model	Advanced nodes, accurate slew propagation
SPICE	device equations	Characterisation, critical custom paths

석사 수준 · Logical effort is the one hand method worth memorising. With g the logical effort of a gate (1 for an inverter, 4/3 for NAND2, 5/3 for NOR2), h the electrical effort (fanout), and p parasitic delay, the path delay is minimised when **every stage carries equal effort** $f = (GH)^{1/N}$, and the optimal per-stage effort is ≈ 3.6 (≈ 4 in practice, whence "fanout-of-4"). This yields the optimal number of stages $N \approx \log_4(GH)$ directly. In an IP context it answers questions such as "how many buffer stages should the clock tree have?" or "is this 12-input decoder better as one stage or three?" before a single tool is run — which matters because those questions arise during architecture, when no netlist exists.

석사 수준 · FO4 as a unit. Quoting delay in FO4 units makes a design comparable across processes: a 30-FO4 pipeline stage is 30 FO4 whether the node is 130 nm or 5 nm. A measured data point for calibration: in the open SkyWater 130 nm PDK, an inverter chain with $W_p = 2.0 \mu\text{m}$, $W_n = 1.0 \mu\text{m}$, $L = 0.15 \mu\text{m}$ and $C_L = 6 \text{ fF}$ gives **FO4 $\approx 48 \text{ ps}$** (ngspice; stage delays 47.5 and 44.5 ps, rise time 67.9 ps). That implies a practical ceiling near **1 GHz** for a ~ 20 -FO4 pipeline stage in that process — which is the quantitative reason a 56 Gbd analogue decision loop is impossible there, and why an open-PDK project must choose a digital, sub-rate architecture.

B2.2 Power

$$P = \alpha C V_{DD}^2 f + I_{SC} V_{DD} + I_{leak} V_{DD}$$

Table 21. Power components and levers

Component	Driver	Lever	Cost of the lever
Dynamic (switching)	$\alpha CV^2 f$	Clock gating; reduce α by data gating; lower V_{DD}	Gating adds skew and verification states
Short-circuit	Both devices on during transition	Sharp input slews	Bigger drivers $\rightarrow$ more C
Subthreshold leakage	e^{-V_{TH}/nV_T}	HVT cells, power gating, body bias	Slower; wake-up latency; state retention
Gate leakage	Tunnelling through oxide	High- κ dielectric	Process choice, not design
Glitch power	Unbalanced arrival times	Balance logic depth ; pipeline	Area, latency

석사 수준 · Glitch power is an IP-level design decision that rarely appears in textbooks. In an unbalanced arithmetic tree, a node can toggle several times before settling, and each toggle costs full switching energy. Ripple-carry adders and unbalanced multiplier trees are notorious. Balancing the tree — or inserting a pipeline register that stops glitch propagation — can reduce datapath power by tens of percent at no functional cost. Because glitching is invisible in RTL simulation (it appears only in gate-level simulation with timing), it is systematically underestimated during architecture. **A design house that reports power measured from a back-annotated gate-level run, with the customer's own data pattern, differentiates itself from one that reports a synthesis estimate.**

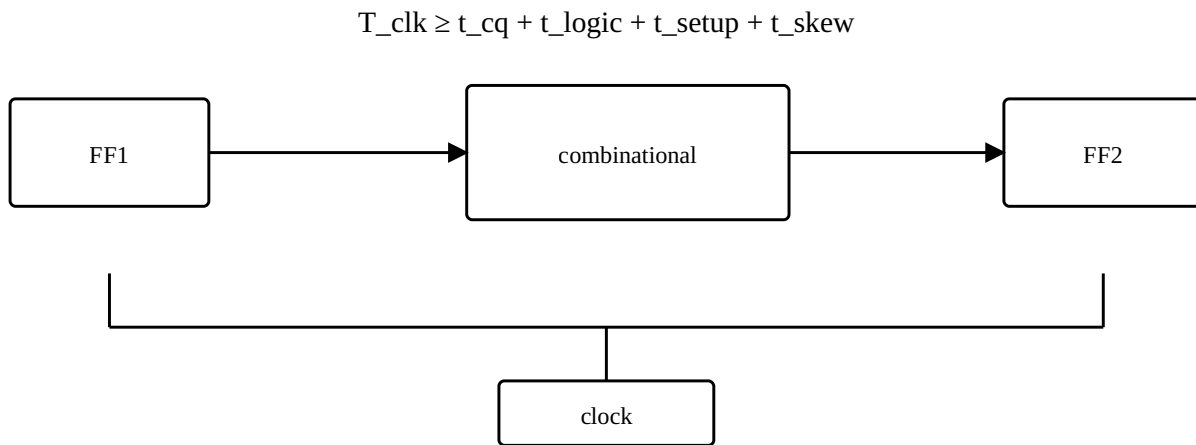
B2.3 Logic families

Table 22. Logic styles

Style	Principle	Advantage	Why it is rarely used in IP
Static CMOS	Complementary pull-up/down	Robust, no clock, easy to verify	— the default
Transmission gate	Pass logic	Fewer devices for MUX/XOR	Threshold loss, charge sharing
Domino / dynamic	Precharge then evaluate	Fast, small	Noise-sensitive, needs custom timing ; not synthesisable
Pass-transistor (CPL)	—	Compact XOR	Level restoration needed
Current-mode (CML/ECL)	Differential current steering	Very high speed, constant current	Static power; used only in SerDes front ends
Adiabatic	Slow charge recovery	Low energy	Needs special clocks; niche

석사 수준 · A synthesisable IP is static CMOS by definition — everything else requires custom timing verification that a customer cannot repeat in their own flow. This is a commercial constraint, not a technical preference. The exception is the SerDes front end, where CML is unavoidable above a few GHz; that is precisely the part that is sold as a hard macro rather than RTL, and it is why the PMA/PCS boundary in a PHY falls where it does.

B3. Timing, Clocking and Static Timing Analysis



$$\text{Hold: } t_{cq} + t_{logic,min} \geq t_{hold} + t_{skew}$$

Setup fails $\Rightarrow$ slow the clock. Hold fails $\Rightarrow$ the chip is dead.

Figure 4. The two timing constraints. Setup limits frequency; hold limits correctness at any frequency.

B3.1 The two inequalities

$$T_{clk} \geq t_{cq} + t_{logic,max} + t_{setup} + t_{skew} + t_{jitter}$$

$$t_{cq,min} + t_{logic,min} \geq t_{hold} + t_{skew}$$

The asymmetry between the two is the single most important operational fact in digital timing. A setup violation is fixed by lowering the clock frequency, so a chip that fails setup still works, just slower. A hold violation has no frequency term: the chip is **broken at every frequency** and the only remedy is a mask change. Consequently hold is fixed exhaustively before tape-out, by buffer insertion, while setup is traded against performance targets. In IP delivery this means the constraint file (SDC) you ship must make the customer's hold analysis possible — omitting clock definitions or false-path declarations can cause a customer to close hold on a path that does not exist, or worse, to miss one that does.

B3.2 Clock distribution

Table 23. Clock network structures

Structure	Skew	Power	Use
H-tree	Low by symmetry	High	Regular arrays, large blocks
Buffered tree (CTS)	Tool-balanced	Moderate	Standard ASIC flow
Mesh / grid	Very low	Very high	High-performance CPUs
Spine + local trees	Moderate	Moderate	Common compromise
Useful skew	Deliberately non-zero	—	Borrow time from a fast stage to a slow one

석사 수준 · **Useful skew (clock borrowing)** is worth knowing because it turns a timing failure into a scheduling problem. If stage A has slack and stage B does not, delaying B's launch clock moves margin from A to B. The tool does this automatically, but the consequence for an IP vendor is subtle: **a block whose internal paths are already borrowing has little tolerance for the customer's clock tree**. Reporting timing with an assumed skew budget, and stating that budget explicitly in the datasheet, prevents the integration failure where the block meets timing in the vendor's flow and misses it in the customer's.

B3.3 What STA cannot see

Table 24. Limits of static timing analysis

Issue	Why STA misses it	What catches it
False paths	STA is structural, not functional	Designer-declared exceptions — and a wrong declaration hides a real path
Multicycle paths	Same	Declaration plus functional verification
Asynchronous cross-ings	No single clock relation	CDC static analysis , not STA
Metastability	Probabilistic, not deterministic	MTBF calculation; synchroniser depth
Glitches	STA reports arrival, not toggle count	Gate-level power simulation
Crosstalk-induced delay	Depends on aggressor timing	SI-aware STA (signal-integrity mode)
IR drop	Voltage assumed constant	Power/IR co-analysis
On-chip variation	Single delay per cell	OCV / AOCV / POCV derating

석사 수준 · **Exception declarations are the most dangerous artefact in a timing constraint file**. Each `set_false_path` removes a path from analysis; if the declaration is wrong, that path is simply never checked and the failure appears only in silicon. A mature IP deliverable therefore ships its SDC with a written justification for every exception, and ideally with an assertion or formal property that proves the path is genuinely unexercisable. **This is a place where a modelling engineer contributes directly: the argument that a path cannot be sensitised is a functional argument, and the person who wrote the reference model is usually the one who can make it.**

B3.4 Metastability quantified

$$MTBF = e^{t_r/\tau} / (T_0 f_{clk} f_{data})$$

석사 수준 · The exponential in t_r (resolution time allowed) is why adding one synchroniser stage improves MTBF by orders of magnitude rather than by a factor of two. With typical 28 nm flip-flop parameters ($\tau \approx 20$ ps), one extra stage at 1 GHz multiplies MTBF by $e^{1000/20} = e^{50}$ — an astronomically large factor. The corollary is that **two stages is almost always enough and three is almost always superstition**, except at very high clock rates where t_r per stage is small. Being able to compute this rather than follow a rule is what lets an IP vendor justify a latency budget to a customer who wants the crossing to be faster.

B4. Interconnect, Transmission Lines and Signal Integrity

Below roughly 250 nm, wires stopped being ideal connections and became circuit elements. Above a few GHz they stop being lumped elements and become transmission lines. Both transitions change what a designer must do.

B4.1 On-chip interconnect as distributed RC

$$t_{\text{delay}} \approx 0.38 R_w C_w L^2 \quad (\text{distributed RC line of length } L)$$

Table 25. Interconnect scaling and its remedies

Problem	Cause	Remedy	Cost
Delay grows as L^2	Distributed RC	Repeater insertion → delay becomes linear in L	Power and area; repeaters can be 30% of a large SoC's dynamic power
Resistance rises with scaling	Smaller cross-section, surface scattering	Thicker upper metal layers for global routes	Fewer routing tracks
Coupling capacitance dominates	Tall narrow wires, close spacing	Increase spacing; shield with ground lines	Routing area
Electromigration	Current density in narrow wires	Width rules, current limits	Area; constrains power grid design
IR drop	Resistance of the power grid	Grid density, decoupling capacitance	Metal resources

석사 수준 · Optimal repeater insertion follows from minimising the total delay of a line broken into k segments each driven by a repeater of size h . The classic result is that the optimum makes the repeater's intrinsic delay comparable to the wire segment delay, giving $k_{\text{opt}} = L\sqrt{(R_w C_w)/(2R_0 C_0)}$. The practical consequence for an IP vendor is that **a block's timing cannot be guaranteed independent of its physical size**: a 2 mm-wide block has internal wire delays that a synthesis-only estimate will miss entirely. This is why serious IP deliverables include a floorplan hint or a hardened layout, and why "it met timing in synthesis" is not a claim a customer will accept for a large block.

B4.2 When a wire becomes a transmission line

학부 복습 · A connection must be treated as a transmission line when its electrical length is a significant fraction of the signal's wavelength — in practice when the propagation delay exceeds roughly one tenth of the signal rise time. Since t_r shrinks with every process generation, the threshold length keeps falling.

$$Z_0 = \sqrt{L/C} \quad v_p = 1/\sqrt{LC} = c/\sqrt{\epsilon_r} \quad \Gamma = (Z_L - Z_0)/(Z_L + Z_0)$$

Table 26. Transmission-line effects in a link

Effect	Physical cause	Frequency behaviour	Consequence
Conductor (skin) loss	Current crowds into a skin depth $\delta \propto 1/\sqrt{f}$	$\propto \sqrt{f}$	Dominant loss below ~20 GHz on PCB
Dielectric loss	$\tan\delta$ of the substrate	$\propto f$	Dominant at high frequency; chooses the laminate

Effect	Physical cause	Frequency behaviour	Consequence
Reflections	Impedance discontinuities (vias, connectors)	Resonant dips	Return loss; creates ISI that FFE cannot fully fix
Crosstalk (NEXT/FEXT)	Mutual L and C between traces	Rises with f	Limits lane density
Fibre weave effect	Inhomogeneous ϵ_r of woven glass	—	Skew between differential pair halves
Mode conversion	Asymmetry in a differential pair	—	Differential energy becomes common mode → EMI

석사 수준 · **Why skin effect makes channel loss go as $\sqrt{f}$ and why that shapes the equaliser.** Skin depth $\delta = \sqrt{2\rho/\omega\mu}$ shrinks as $1/\sqrt{f}$, so the effective conductor cross-section shrinks and resistance rises as $\sqrt{f}$. The resulting channel magnitude response is approximately $\exp(-k\sqrt{f})$, which in the time domain produces a **long, slowly decaying pulse tail** rather than a sharp one. That specific tail shape is why decision-feedback equalisation works so well on wireline channels: the post-cursor ISI is long but the symbols causing it have already been decided, so they can be subtracted exactly without noise enhancement. **The physics of the conductor selects the equaliser architecture.** An engineer who knows only that "DFE cancels post-cursor ISI" cannot explain why a different channel — say a multipath radio channel with pre-cursor energy — needs a different structure.

B4.3 S-parameters

Table 27. Scattering parameters for a two-port

Parameter	Meaning	Specification name
S_{11}	Input reflection	Return loss (dB, negative is good)
S_{21}	Forward transmission	Insertion loss
S_{12}	Reverse transmission	Isolation
S_{22}	Output reflection	Output return loss
S_{dd}, S_{cc}	Differential / common mode	Mixed-mode S-parameters
S_{cd}, S_{dc}	Mode conversion	Asymmetry metric — an EMI predictor

석사 수준 · S-parameters are used because at high frequency you cannot measure voltage and current at a port without perturbing it, but you can measure incident and reflected power waves. For a modelling engineer the practical point is the **conversion to a time-domain model**: a channel is delivered as a Touchstone (.s4p) file, and a link simulation needs its pulse response. Doing that conversion correctly requires enforcing **causality and passivity** on measured data that generally violates both because of measurement noise and band limitation. Non-causal channel models produce link simulations that are optimistic in a way that is very hard to notice. **A channel-model conditioning tool is a small, genuinely sellable piece of software**, and it is squarely modelling-team work.

B4.4 Power delivery and simultaneous switching noise

$$V_{noise} = L_{package} \cdot dI/dt + R_{grid} \cdot I$$

Table 28. Power integrity mechanisms

Mechanism	Where	Mitigation
Static IR drop	On-chip grid resistance	Grid density, more straps
dI/dt noise (SSN)	Package and board inductance	Decoupling capacitance at three levels: on-die, package, board
Resonance	L_{pkg} with on-die C	Anti-resonance peak typically 50–200 MHz ; add damping
Ground bounce	Shared return inductance	More ground pins, differential IO
Supply-induced jitter	V_{DD} modulates delay	Regulate locally; use supply-insensitive delay cells in clock paths

석사 수준 · Supply-induced jitter is the mechanism by which a *digital* design decision damages an *analogue* specification. A large synchronous block that starts and stops — a clock-gated accelerator, a burst-mode FEC decoder — produces a current step at the burst rate, which modulates the supply, which modulates the delay of the PLL's delay cells, which appears as a spur in the phase noise, which appears as periodic jitter (PJ) in the link budget. **The FEC block is the root cause of a jitter specification failure**, and nothing in either block's own verification would reveal it. This class of cross-domain coupling is why chip-level power-aware simulation exists, and why an IP datasheet should state the block's *current profile*, not merely its average power.

B5. Analogue Building Blocks

A digital IP house does not design these, but it must model them, specify them, and understand which digital block can compensate which analogue imperfection — because that compensation is often the sellable part.

B5.1 Amplifiers

Table 29. Amplifier topologies

Topology	Gain	Bandwidth	Where used
Common source	$-g_m r_o$	Moderate	Basic gain stage
Cascode	$-g_m (g_m r_o) r_o$	High (reduced Miller)	High-gain stages
Differential pair	—	—	Rejects common-mode noise ; the basis of nearly everything
Folded cascode	High	Good	Wide input range op-amps
Two-stage Miller	Very high	Compensated	General-purpose op-amp
CTLE	peaked	—	Source degeneration creates a zero that boosts high frequency
TIA	$-R_f$	—	Photodiode front end

석사 수준 · **Why a CTLE amplifies noise and an FFE does not have to.** A CTLE is a linear analogue filter placed *before* the point where noise is added by the rest of the receiver, but *after* the channel has already attenuated the signal and after thermal noise from the channel and the front end. Boosting high frequency therefore boosts the noise already present in that band by the same factor. A digital FFE placed after the ADC has exactly the same problem. **ADFE** escapes it because it subtracts a *decided* symbol — a noiseless quantity — rather than amplifying a received one. This is the whole reason receivers combine a modest CTLE with a DFE instead of using a large CTLE alone, and it is a piece of reasoning that a modelling engineer must be able to reproduce when asked to justify an architecture.

B5.2 Comparators and references

Table 30. Comparator and reference issues

Block	Key imperfection	Digital compensation
Strong-arm latch	Input-referred offset (mismatch)	Offset calibration DAC trimmed digitally
	Metastability when inputs are close	Allow regeneration time; detect and flag
	Kickback onto the input	Sample-and-hold isolation
Bandgap reference	Curvature over temperature	Piecewise digital trim
	Start-up failure mode	Start-up circuit plus watchdog
Current DAC	Element mismatch	Dynamic element matching (digital)

석사 수준 · Each row's right-hand column is a digital block with a clean specification, and that is the commercial observation: **the calibration and trim logic around an analogue block is frequently larger than the analogue block itself, and it is synthesisable.** A design house that cannot build a comparator can still own the offset-calibration engine, the DEM scrambler, and the background interleaving-mismatch corrector. These are sold as part of converter IP, and they are where the modelling effort concentrates because their correctness is statistical rather than logical.

B5.3 PLLs and clock generation

$$H(s) = K_{PD}K_{VCO}F(s) / (s + K_{PD}K_{VCO}F(s)/N)$$

Table 31. PLL noise transfer

Noise source	Transfer to output	Design implication
Reference noise	Low-pass ($\times N$ in band)	In-band noise is multiplied by $20 \log_{10} N$
VCO noise	High-pass	Wide loop suppresses VCO noise
Divider noise	Low-pass	—
Charge-pump noise	Low-pass	Dominates in-band floor
$\Sigma\Delta$ quantisation (fractional-N)	High-pass shaped	Loop must be narrow enough to filter it

석사 수준 · The opposing shapes create the **optimal loop bandwidth**: wide enough to suppress VCO phase noise, narrow enough to suppress reference and fractional- N quantisation noise. The optimum is where the two contributions cross. This is a genuinely quantitative design decision that a system modeller can and should compute, and it is directly connected to the link jitter budget of Chapter A4 — the integrated phase noise becomes the RJ term that is multiplied by 14 at 10^{-12} BER. **Chapters A4, B4 and B5 are three views of the same budget**, and one of the marks of a senior engineer is holding them together.

B5.4 Data converters

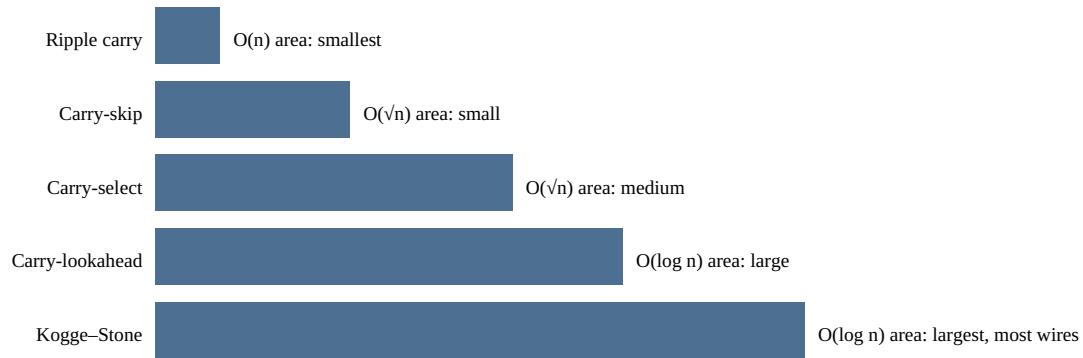
Table 32. ADC architectures

Architecture	Speed	Resolution	Principle	Digital support needed
Flash	Highest	≤ 6 bit	$2^N - 1$ comparators	Thermometer $\rightarrow$ binary, bubble correction
SAR	Moderate–high	8–14 bit	Binary search with a DAC	Redundancy and digital error correction
Pipeline	High	10–14 bit	Stages with residue amplification	Inter-stage gain calibration
$\Sigma\Delta$	Low–moderate	16–24 bit	Oversample and noise-shape	Decimation filter (CIC + FIR)
Time-interleaved SAR	>10 GS/s	6–8 bit	M SARs in parallel	Offset/gain/skew background calibration — the 112G receiver case
Folding/interpolating	High	8–10 bit	Reduce comparator count	—

석사 수준 · **$\Sigma\Delta$ noise shaping** is worth stating exactly because it is the clearest example of trading bandwidth for resolution. An L th-order modulator with oversampling ratio OSR gives SNR gain $\approx (6L+3)$ dB per doubling of OSR, versus 3 dB for plain oversampling. The price is that the modulator output is a low-resolution, very fast stream that must be decimated — and **that decimation filter is a substantial digital IP block** whose passband ripple and stopband attenuation determine the converter's final specification. A converter vendor's datasheet number is in part a statement about a digital filter. This is the second time in this chapter that the saleable digital block sits next to an analogue one.

C1. Arithmetic Units — The Core of Every Datapath

C1.1 Addition



Same function. Five points on the area–delay curve.

Figure 5. Adder architectures span an area–delay curve of roughly an order of magnitude in each axis.

Table 33. Adder architectures

Architecture	Delay	Area	Notes
Ripple carry	$O(n)$	$O(n)$	Baseline; fine for $n \leq 8$ at low frequency
Carry-skip	$O(\sqrt{n})$	$O(n)$	Cheap improvement
Carry-select	$O(\sqrt{n})$	$\sim 2\times$	Duplicate blocks, select on carry
Carry-lookahead	$O(\log n)$	$O(n \log n)$	Classic
Kogge–Stone	$O(\log n)$	Large, heavy wiring	Minimum depth; wire-limited in practice
Brent–Kung	$O(\log n)$	Fewer wires	Depth $2 \log n$; better area
Carry-save	$O(1)$ per add	$O(n)$	Does not propagate carry — the key to fast multipliers and accumulators

석사 수준 · **Carry-save is the single most useful idea in this table.** A carry-save adder takes three operands and produces two (sum and carry vectors) in constant time, because no carry propagates. A chain of them accumulates many terms at constant delay per term, and only the *final* conversion to a normal binary number requires a carry-propagate adder. This is why multiplier trees, FIR accumulators, and MAC units are built as "compress many partial products with carry-save, then one fast adder at the end". **In an accumulation loop — the feedback path that limits every recursive filter — keeping the state in carry-save form removes the carry-propagate delay from the loop entirely.** That single transformation has rescued more timing-critical datapaths than any other.

C1.2 Multiplication

Table 34. Multiplier construction

Stage	Technique	Effect
Partial product generation	Booth radix-4	Halves the number of partial products; needs 2's-complement handling
	Booth radix-8	Fewer still, but needs a 3× multiple (extra adder)
Reduction	Wallace tree	$O(\log n)$ depth, irregular layout
	Dadda tree	Slightly fewer adders than Wallace
	Array	Regular layout, $O(n)$ depth
Final add	Fast carry-propagate adder	Kogge–Stone or similar
Whole unit	Pipelining	Cut the tree with registers to raise $f_{\max}$

석사 수준 · **Constant multiplication is a different problem** and is worth separating in any specification. When one operand is a compile-time constant — filter coefficients, twiddle factors, scaling constants — the multiplier collapses into shifts and adds. Canonical signed digit (CSD) representation minimises the number of non-zero digits, and **multiple constant multiplication (MCM)** algorithms share sub-expressions across a whole coefficient set. For a 32-tap FIR with fixed coefficients this routinely gives a 3–5× area reduction over generic multipliers. The commercial point: **an IP that is parameterised for run-time coefficients cannot use this, so "programmable coefficients" is an expensive feature** that should be priced and specified as such, not thrown in because it seems flexible.

C1.3 Division, square root, and reciprocal

Table 35. Division approaches

Method	Convergence	Hardware	Use
Restoring / non-restoring	1 bit per cycle	Adder + shifter	Small area, high latency
SRT (radix-4/8)	2–3 bits per cycle	Lookup for quotient digit	Standard in FPU's
Newton–Raphson	Quadratic (doubles digits)	Multipliers	Reuses the multiplier; good when one exists
Goldschmidt	Quadratic	Multipliers	Better pipelining than Newton
Reciprocal LUT + refine	—	Small ROM + multiplier	The usual answer in DSP

The most important division fact for an IP designer is architectural, not numerical: never put a divider in a feedback loop. Its latency enters the loop directly and sets the initiation interval. The standard transformation is to compute a reciprocal *once* outside the loop and multiply inside it. The Cholesky decomposition is a textbook example — the diagonal reciprocal is formed once per column and the column is then scaled by multiplication, which is why the inner loop can be pipelined at $II = 1$. This is also a place where the reference model and the RTL can legitimately differ: the model may divide, the RTL multiplies by a reciprocal, and the two agree only to within the reciprocal's rounding. **The specification must state which one is normative.**

C1.4 Fixed-point and floating-point units

Table 36. Number format decisions

Format	Range	Hardware cost	Where it belongs
Fixed point $Q_{m.n}$	Limited, uniform	Adder/multiplier only	Most DSP and comms datapaths
Block floating point	Shared exponent per block	Small extra	FFT, where dynamic range grows predictably

Format	Range	Hardware cost	Where it belongs
IEEE 754 binary32	Wide	Align, normalise, round — expensive	Scientific, some solvers
binary16 / bfloat16	Wide, low precision	Cheaper	Neural inference
Integer with saturation	Limited, clamped	Compare and clamp	Where wrap would be catastrophic (LLRs, audio)
Posit / logarithmic	Tapered	Unusual	Research; rarely in shipped IP

석사 수준 · **Block floating point deserves more attention than it gets.** In an FFT the signal magnitude grows by up to a factor of two per stage. Full floating point is overkill; fixed point requires either wide words or per-stage scaling that loses SNR. Block floating point keeps one exponent per stage (or per block) and normalises once, recovering most of floating point's dynamic range at a small fraction of the cost. **For a modelling engineer this format is a trap**, because a naive floating-point reference model will not match the hardware's block-normalised behaviour bit for bit — the reference must model the normalisation decisions too. Specifying "bit-accurate reference model" without specifying the scaling policy produces months of false mismatches.

C2. Control: State Machines, Pipelines and Flow Control

C2.1 FSM encoding

Table 37. State encodings

Encoding	Re-gisters	Next-state logic	When
Binary	$\log_2 N$	Complex	Many states, area critical
Gray	$\log_2 N$	Complex	Adjacent states differ by one bit — low power, safer across CDC
One-hot	N	Simple (one AND per transition)	FPGA default ; fast; easy to check
Johnson	$N/2$	Simple	Counters, ring sequencers
Hamming-protected	$>\log_2 N$	—	Safety/security : illegal states detectable

석사 수준 · **Security-grade FSM encoding** is a concrete example of a requirement that comes from a threat model rather than from function. If an attacker can flip a state bit with a laser or a voltage glitch, a binary-encoded FSM may land in a valid but wrong state — for instance skipping rounds of a cipher. Countermeasures are: sparse encodings with large Hamming distance so that a single flip lands in an illegal state; a default branch that raises an alarm rather than doing nothing; and **duplicated FSMs with complementary encodings** whose outputs must remain complementary. The last technique requires a synthesis barrier so the tool does not optimise the inverters away and merge the two copies — if it does, the redundancy disappears while *functional verification still passes*. That silent failure mode is the reason such designs contain explicit buffer primitives whose only purpose is to stop the optimiser.

C2.2 Pipelining and its hazards

Table 38. Pipeline transformations

Transformation	What it does	Cost	Limit
Pipelining	Insert registers between logic stages	Latency, area	Cannot cut a feedback loop
Retiming	Move existing registers across logic	None (if legal)	Total registers on each loop is invariant
Loop un-rolling	Process k iterations per cycle	$k \times$ area	The loop's critical path is divided by k only in the feedforward part
Speculation	Compute all outcomes, select afterwards	Exponential in the number of speculated bits	Removes the decision from the loop
C-slow	Interleave k independent streams	Registers $\times k$	Needs k independent problems
Algebraic transform	Restructure the recurrence itself	Design effort	The only way past a fundamental loop bound

석사 수준 · **The iteration bound** is the hard limit that these transformations negotiate with. For a recursive system, the minimum achievable iteration period is $T_\infty = \max_{\text{loops}} (t_{\text{loop}}/w_{\text{loop}})$, where t_{loop} is the total computation delay around a loop

and w_{loop} the number of registers in it. Retiming and pipelining cannot beat it; only changing the recurrence can. Recognising this quantity turns a vague "we need more speed" into a specific question: *which loop, and can its algebra be rewritten?* Look-ahead transformation (computing two steps of a recurrence in one) doubles w and thereby halves T_{∞} , at the cost of more arithmetic. **This is the abstract form of the unrolled DFE, radix-4 Viterbi ACS, and reformulated Berlekamp-Massey — three blocks that look unrelated and are the same transformation.**

C2.3 Flow control and backpressure

Table 39. Handshake styles

Style	Signals	Property	Hazard
Valid only	valid	Producer never stalls	Consumer must always accept — needs guaranteed rate
Valid/Ready	valid, ready	Full backpressure	Combinational loop if ready depends on valid
Credit-based	credits returned	Decouples latency from throughput	Credit accounting bugs
Req/Ack (4-phase)	(4- handshake pairs)	Async-safe	Slow (two round trips)

The valid/ready combinational loop is one of the most common integration failures in modular RTL. If module A's ready depends combinationaly on its input valid, and module B's valid depends combinationaly on its input ready, connecting them creates a zero-delay cycle. Each module is individually correct and passes its own tests. The standard remedies are a *skid buffer* (also called a spill register), which breaks the path at the cost of one register stage and one entry of buffering, or a rule that ready is registered. **An IP that documents which of its interfaces are combinationaly coupled — and ships a skid buffer — saves its customers a week each.** This is exactly the kind of detail that distinguishes a product from a piece of code.

C2.4 Arbitration

Table 40. Arbiter types

Type	Fairness	Latency	Use
Fixed priority	None (starvation possible)	Lowest	Strict QoS classes
Round robin	Fair	Low	General purpose
Weighted round robin	Proportional	Low	Mixed traffic classes
Matrix / LRU	Fair, true least-recently-used	Higher	Caches
Lottery	Statistical	Low	Rare
Age-based	Strict FIFO fairness	Higher	Avoids starvation strictly

석사 수준 · Arbitration is where **fairness and forward progress** become formal properties rather than intentions. A round-robin arbiter is fair by construction; a fixed-priority arbiter is not, and a low-priority requester can starve forever. Whether starvation matters is a system question — it is acceptable for a debug port, fatal for a DMA that feeds a real-time codec. Because starvation is a *liveness* property ("eventually granted") rather than a safety property, it is natural to express and prove with formal tools, and arbiters are one of the few places where formal verification is both easy and genuinely valuable. **Shipping a formally proven arbiter with its properties is a differentiator that costs a design house little.**

C3. Clock Domain Crossing, Reset and Low Power

C3.1 The CDC problem stated precisely

학부 복습 · A signal crossing between unrelated clock domains violates setup or hold at the receiving flip-flop with non-zero probability. The flip-flop may enter a metastable state whose resolution time is unbounded (though exponentially unlikely to be long). **This cannot be simulated:** a logic simulator has no metastable value and will always resolve to 0 or 1.

Table 41. CDC techniques by payload type

Payload	Technique	Why	Failure if misapplied
1-bit level	Two-flop synchroniser	Allows a full cycle to resolve	—
1-bit pulse	Toggle synchroniser (level convert, then edge detect)	A pulse shorter than the destination period would be lost	Lost events
Multi-bit, unrelated	Synchronise a request; use a data hold register	Bits arriving in different cycles corrupt the value	Corrupted data words
Multi-bit counter	Gray code	Only one bit changes per increment	Wrong pointer → FIFO corruption
Stream	Asynchronous FIFO (Gray pointers, dual-port RAM)	Decouples rate and phase	—
Bus transaction	Handshake (req/ack) or credit	Explicit acknowledgement	—
Reset	Asynchronous assert, synchronous de-assert	De-assertion must not violate recovery/removal	Part of the design leaves reset a cycle late

석사 수준 · **Why Gray coding a FIFO pointer is safe even though the pointer is multi-bit.** The synchroniser may sample a Gray pointer mid-transition, but since only one bit changes per increment, the sampled value is either the old or the new pointer — never a third value. An off-by-one pointer makes the FIFO report itself slightly *fuller* or *emptier* than it is, which is conservative: the FIFO may stall a cycle early but never overflows or underflows. **The design is safe because its error mode is biased in the harmless direction.** This principle — arrange for uncertainty to fail conservatively — generalises far beyond FIFOs and is one of the most transferable ideas in digital design.

C3.2 Reset architecture

Table 42. Reset strategies

Strategy	Advantage	Problem
Synchronous	No recovery/removal issue; clean STA	Needs a running clock — fails at power-up
Asynchronous	Works without a clock	De-assertion can violate recovery
Async assert, sync de-assert	Both advantages	Requires a reset synchroniser per domain — the standard solution
Reset-less registers	Area and routing saving	Must be provably initialised by data flow; a verification obligation

Reset de-assertion order is a specification item that is almost always omitted. If two blocks leave reset in different cycles and one samples the other's outputs, the receiving block may latch garbage. Within a block the designer controls this; across

an IP boundary the *customer* controls it, and they cannot get it right unless the datasheet states the requirement. A good IP datasheet specifies: the reset polarity, whether it is synchronous or asynchronous, the minimum assertion width in cycles of each clock, and any required ordering between multiple resets. **Omitting this produces intermittent bring-up failures that are blamed on the IP.**

C3.3 Low power design

Table 43. Power reduction techniques by level

Level	Technique	Saving	Verification burden
RTL	Clock gating	Large (clock tree is 20–40% of dynamic power)	Low — tools insert it automatically
RTL	Operand isolation / data gating	Moderate	Low
RTL	Reduce switching activity (encoding, bus inversion)	Moderate	Low
Architecture	Parallelism + voltage scaling	Large — power $\propto V^2$	Moderate
Architecture	Memory hierarchy (avoid far accesses)	Large	Moderate
Physical	Multi- V_{TH} cells	Leakage	None
System	Power gating	Leakage $\rightarrow \sim 0$	High: isolation, retention, wake sequencing
System	DVFS	Large	High: timing at every operating point
System	Multiple power domains	Large	Highest: UPF, isolation cells

석사 수준 · **Parallelism as a power technique** is the least intuitive entry and the most powerful. Two copies of a block running at half the clock frequency deliver the same throughput; because they now have twice the time budget, the supply voltage can be lowered until the slower path just meets timing. Dynamic power scales as V^2f , so halving f and reducing V by, say, 30% gives roughly $0.5 \times 0.49 = 0.25$ of the original power, at the cost of about twice the area. **This trade — area for energy — is the fundamental reason accelerators are more efficient than processors**, and it is the argument an IP vendor uses when selling a block against a software implementation. State it with the exponents and it is persuasive; state it as "our block is more efficient" and it is not.

C3.4 Design for test

Table 44. DFT structures

Structure	Tests what	Cost	IP implication
Scan chains	All flip-flops and combinational logic	Mux in every FF (~5–10% area, some delay)	IP must be scan-insertable: no gated clocks without test bypass, no latches, no async loops
MBIST	Embedded memories	Controller per memory group	Memory interfaces must be accessible
BSD (JTAG 1149.1)	Board-level interconnect	Boundary cells	—
LBIST	Logic, in-field	PRPG + MISR	Automotive/safety

Structure	Tests what	Cost	IP implication
ATPG coverage	—	—	Stuck-at >99%, transition >90% typical targets
Test compression	—	Decompressor/compactor	Reduces tester time and cost

Scan is a security hole. A scan chain can read and write every flip-flop in the design, including key registers. Production chips must therefore disable scan after manufacturing test, typically by blowing a fuse or advancing a life-cycle state. A security IP that does not document its scan-isolation requirement will be integrated insecurely. **Conversely, an IP that cannot be scanned at all is unsellable**, because the customer cannot achieve their test coverage target. The resolution — scannable, but with key registers excluded from the chain or cleared on scan entry — must be designed in, and it is a requirement that comes from neither function nor timing.

D1. Instruction Set Architecture

An ISA is a contract between hardware and software. For a modelling engineer it is the *specification document* in the most literal sense: the reference model **is** the ISA, written executably.

Table 45. ISA design axes

Axis	Options	Consequence for hardware
Operand model	Stack / accumulator / register-memory / load-store	Load-store simplifies pipelining; every RISC uses it
Instruction length	Fixed / variable / compressed	Fixed simplifies fetch; compressed (RVC, Thumb) saves 25–30% code size at the cost of alignment logic
Addressing modes	Few / many	Many modes need an address generation unit; few modes need more instructions
Condition handling	Flags / compare-and-branch	Flags create a serialising dependency ; RISC-V deliberately has none
Delay slots	Present (classic MIPS) / absent	Delay slots expose the pipeline in the ISA — a design mistake later reversed
Register count	8 / 16 / 32 / more	More registers reduce spills but widen every instruction field
Exceptions	Precise / imprecise	Precise exceptions constrain the whole microarchitecture

석사 수준 · Delay slots are the canonical example of leaking microarchitecture into architecture. MIPS exposed its single branch-delay slot so that the compiler could fill it, saving a stall in a five-stage pipeline. When later implementations had deeper pipelines and branch prediction, the delay slot became useless baggage that every implementation still had to honour. **The lesson for IP design generalises: a specification that encodes today's implementation constrains tomorrow's.** When writing a block's interface, ask which parts describe *what* and which describe *how*, and expose only the former.

석사 수준 · Precise exceptions mean that when an exception is taken, all instructions before it have completed and none after it has had any effect. This is what makes an ISS a valid reference: the model can execute one instruction at a time and still describe a machine that executes many concurrently. Achieving it in an out-of-order machine requires a reorder buffer and speculative state, which is a large fraction of such a core's complexity. **The entire step-and-compare verification methodology rests on precise exceptions**; without them there would be no well-defined point at which model and RTL should agree.

D1.2 RISC-V as a case study

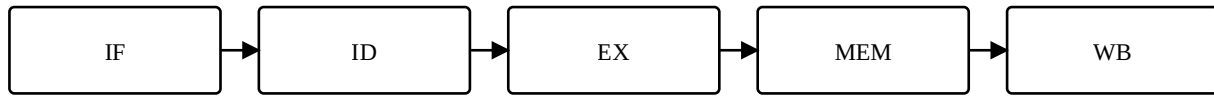
Table 46. RISC-V structure

Element	Content	Modelling note
Base integer	RV32I / RV64I — ~40 instructions	Small enough to model exhaustively
M	Multiply/divide	Division by zero is defined , not trapping
A	Atomics (LR/SC, AMO)	Requires a memory consistency model
F/D/Q	Floating point	IEEE 754; needs SoftFloat, not host arithmetic
C	Compressed 16-bit	Alignment: instructions may straddle a word
V	Vector	Vector-length agnostic — a model must track <code>v1</code> , <code>vtype</code>
B, Zk, Zc...	Bit manipulation, crypto, code size	—

Element	Content	Modelling note
Privileged	M/S/U modes, CSRs, PMP, virtual memory	Where most model bugs live: exception priority and CSR side effects

Exception priority is the most common source of reference-model defects in a CPU project. One instruction can simultaneously satisfy the conditions for several exceptions — misaligned address, page fault, access fault, breakpoint. The specification gives a strict priority order. A model that implements a different order is correct on almost every test and wrong on a narrow class of inputs that constrained-random stimulus may take weeks to find. The remedy is not more random testing but **directed tests derived from the priority table itself:** enumerate the table, construct one case per row, and check. This is a half-day of work that reliably finds bugs a month of random testing misses.

D2. Pipelining, Hazards and Speculation



Throughput 1 instr/cycle; latency 5 cycles

hazards: structural · data · control

Forwarding removes most data hazards; load-use needs one stall

Control hazards cost the branch penalty × misprediction rate

Figure 6. The classic five-stage pipeline and its three hazard classes.

Table 47. Hazards and remedies

Hazard	Cause	Remedy	Residual cost
Structural	Two instructions need the same resource	Duplicate the resource, or stall	Area
Data RAW	Read after write	Forwarding from EX/MEM	Load-use still costs one cycle
Data WAR / WAW	Anti- and output dependencies	Register renaming	Only in out-of-order designs
Control	Branch outcome unknown at fetch	Prediction + speculative execution	Misprediction penalty × rate
Memory ordering	Loads and stores to the same address	Store buffer with forwarding, memory disambiguation	Complexity

Table 48. Branch prediction

Predictor	Mechanism	Typical accuracy
Static (backward taken)	Loop heuristic	~65%
Bimodal	2-bit saturating counter per PC	~85%
gshare	Global history XOR PC indexes the table	~93%
Tournament	Choose between local and global predictors	~95%
TAGE	Multiple history lengths with tags	>97%
Perceptron	Linear classifier over history	>97%
Return address stack	Dedicated stack for call/return	>99% for returns

석사 수준 · Why a misprediction is expensive and getting more so. The penalty is the pipeline depth from fetch to resolve. In a 5-stage machine that is 2–3 cycles; in a 15-stage out-of-order machine it is 15–20, and the machine may have to discard dozens of in-flight instructions. With a 5% miss rate and a 15-cycle penalty, branches alone cost 0.75 cycles per branch — often more than every other stall combined. This is why predictor research continues to be worthwhile and why deep pipelines

have diminishing returns. **For an accelerator IP designer the lesson is the opposite and liberating: a fixed-function datapath has no branches, so none of this cost exists.** Quantifying that difference — "your software loop spends 0.75 cycles per iteration on branch misprediction alone" — is one of the strongest arguments an accelerator vendor has.

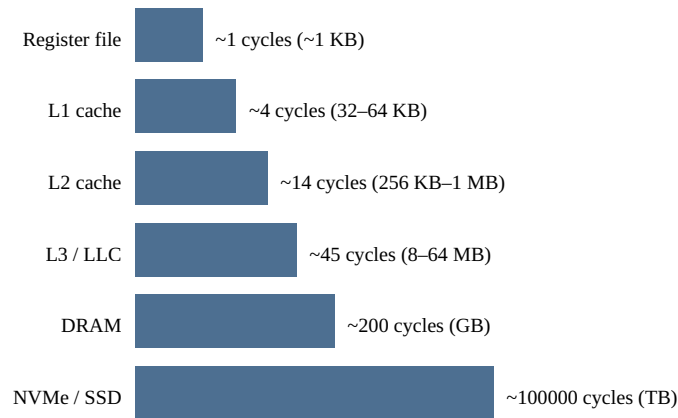
D2.2 Out-of-order execution

Table 49. Out-of-order machinery

Structure	Purpose	Scaling limit
Register renaming (RAT)	Removes WAR/WAW	Physical register file size
Reservation stations / issue queue	Hold instructions until operands ready	Wakeup-select loop is the critical path
Reorder buffer (ROB)	In-order commit → precise exceptions	Entries scale with memory latency
Load/store queue	Memory disambiguation	Associative search cost
Scoreboard	Simpler alternative (no renaming)	Stalls on WAR/WAW

석사 수준 · The **wakeup-select loop** — broadcasting a result tag, waking dependent instructions, selecting which to issue — must complete in one cycle for back-to-back dependent execution. It is a broadcast across the whole issue queue, so its delay grows with queue size and with issue width. This single loop, not transistor speed, is why issue widths have plateaued around 4–8 and why further performance has come from more cores rather than wider ones. **It is the same phenomenon as the DFE feedback loop and the Viterbi ACS loop: a recurrence whose latency cannot be pipelined away.** Recognising the pattern across domains is exactly the transferable skill this book aims to build.

D3. Memory Hierarchy and Coherence



Each level is roughly an order of magnitude slower and larger

Figure 7. The memory hierarchy. Each level trades capacity against latency by roughly an order of magnitude.

Table 50. Cache design parameters

Parameter	Effect	Trade
Block size	Exploits spatial locality	Too large wastes bandwidth and causes false sharing
Associativity	Reduces conflict misses	More comparators, longer hit time; 4–8 way is the usual optimum
Capacity	Reduces capacity misses	Area, leakage, hit latency
Write policy	Write-through vs write-back	Write-back reduces bandwidth; needs dirty bits and coherence
Allocation	Write-allocate vs not	Depends on write locality
Replacement	LRU / pseudo-LRU / random	True LRU is expensive beyond 4-way; pseudo-LRU is within 1%
Inclusion	Inclusive / exclusive / NINE	Affects snoop filtering

학부 복습 · Misses classify as **compulsory** (first reference), **capacity** (working set exceeds the cache), **conflict** (limited associativity), and in multiprocessors **coherence** (invalidated by another core). The classification is useful because each class has a different remedy: prefetching for compulsory, larger cache for capacity, higher associativity or better indexing for conflict, and data layout changes for coherence.

Table 51. Coherence protocols

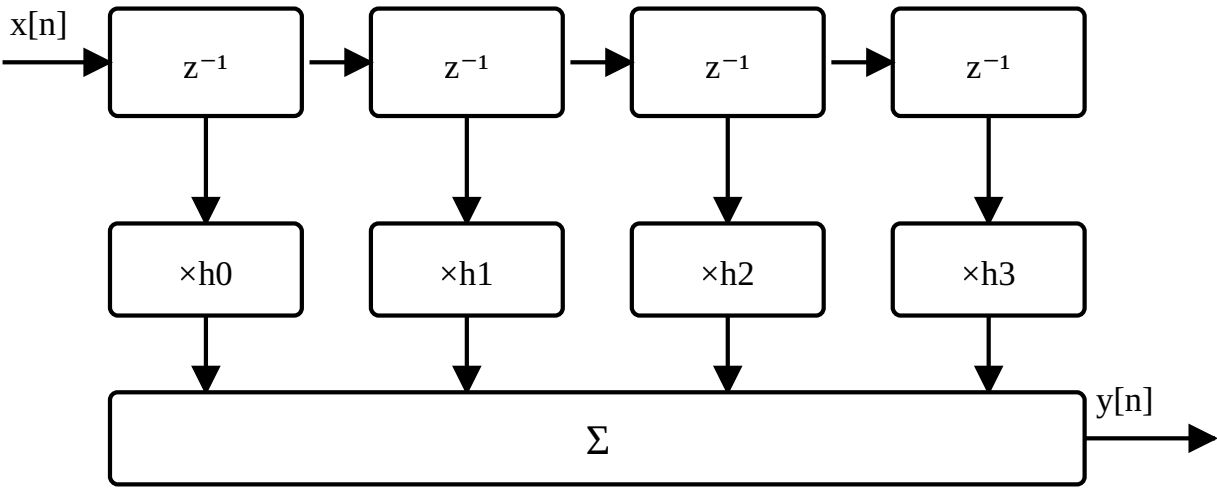
Protocol	States	Note
MSI	Modified, Shared, Invalid	Minimal
MESI	+ Exclusive	Avoids a bus transaction on read–modify
MOESI	+ Owned	Allows dirty sharing; saves writebacks
Directory-based	—	Scales beyond a bus ; required for many-core
Snoop filter	—	Reduces useless snoops in large systems

석사 수준 · **Memory consistency is not coherence.** Coherence concerns a single address — all cores must see one order of writes to it. Consistency concerns the relative order of operations to *different* addresses. A machine can be perfectly coherent and still reorder a store to A before a store to B as observed by another core. The consistency model (sequential, TSO, release, RVWMO) specifies what reorderings are allowed and which fences prevent them. **For a modelling engineer this is the hardest specification to capture**, because the reference model is inherently sequential while the hardware is not: a simple step-and-compare against an ISS *cannot* verify a weak memory model. The correct approach is **litmus tests** — short multi-threaded programs with enumerated permitted outcomes — checked against the architecture's formal memory model. Knowing that a different verification technique is required, and why, is the point.

Table 52. DRAM characteristics that leak into system design

Property	Meaning	Design response
Row buffer	A row must be activated before access	Exploit row locality; schedule for open rows
$t_{\text{RCD}}, t_{\text{RP}}, t_{\text{RAS}}$	Activate/precharge timings	Memory controller scheduling
Refresh	Every ~32–64 ms	Blocks access; matters for real-time guarantees
Bank/rank parallelism	Independent banks	Interleave addresses to exploit it
Burst length	Minimum transfer granularity	A 4-byte read still fetches 32–64 bytes
Rowhammer	Repeated activation disturbs neighbours	A security problem solved by refresh management — a DRAM controller feature

E1. Digital Filters as Hardware



Direct form: adder tree depth grows with N

Transposed form: critical path is one mult + one add, independent of N

Figure 8. Direct-form FIR. The tap count sets the multiplier count; the *form* sets the critical path.

Table 53. FIR structures and their costs

Structure	Multipliers	Critical path	Choose when
Direct form	N	1 mult + $\log N$ adds	N small
Transposed direct form	N	1 mult + 1 add (constant)	Almost always, for speed. Cost: high fanout on $x[n]$
Symmetric (folded)	$\lceil N/2 \rceil$	1 add + 1 mult + 1 add	Linear phase — free 2× saving
Polyphase	N/M effective	—	Combined with rate change
Distributed arithmetic	0 (LUTs)	W cycles	Fixed coefficients, LUT fabric
CSD / multiplierless	0	adds only	Fixed coefficients, ASIC area
Frequency domain	—	—	$N \gtrsim 100$ and block latency acceptable

석사 수준 · **The fanout cost of the transposed form is real and is often overlooked in architecture documents.** The input sample must reach all N multipliers in one cycle. For $N = 128$ at 500 MHz this is a significant buffer tree whose delay and power must be budgeted. The usual remedy is to *partition* the filter into segments with a pipelined input distribution network — effectively reintroducing a small amount of direct-form structure. **Real designs are hybrids, and a specification that names only "transposed FIR" is incomplete.**

E1.2 Filter design methods

Table 54. FIR design methods

Method	Optimality	Control	Use
Windowing	Suboptimal	Simple	Quick designs
Frequency sampling	Suboptimal	Direct in frequency	—
Parks–McClellan (Remez)	Minimax optimal	Ripple in each band	Standard for equiripple specs
Least squares	L2 optimal	Weighted error	When energy matters more than peak
Convex optimisation	Flexible	Arbitrary constraints	Handles quantised coefficients directly

석사 수준 · Designing in floating point and rounding afterwards is a mistake that costs stopband attenuation. Rounding a Parks–McClellan design to B bits perturbs each coefficient by up to half an LSB, and the resulting stopband degradation can be 10–20 dB. The correct procedure is to include the quantisation in the design: either use an integer-programming or convex formulation that searches over representable coefficients, or re-optimize the remaining coefficients after fixing some. **For an IP vendor this is a concrete deliverable: ship the coefficient generator, not just the coefficients**, so the customer can re-tune for their own band plan without repeating the analysis.

E1.3 Multirate structures

Table 55. Rate conversion building blocks

Block	Function	Key property
Decimator ($\downarrow M$)	Filter then discard $M-1$ of every M	Filter first , or aliasing is irreversible
Interpolator ($\uparrow L$)	Insert $L-1$ zeros then filter	Zeros create images; the filter removes them
Polyphase decomposition	Split the filter into M sub-filters	Avoids computing samples that are discarded — $M\times$ saving
CIC	Integrator–comb cascade	No multipliers ; passband droop must be compensated
Half-band	Every other coefficient is zero	$\sim 2\times$ saving for 2:1 rates
Farrow	Polynomial interpolation with variable delay	Arbitrary, time-varying resampling ratio — used in timing recovery

석사 수준 · The CIC word-growth question is a classic fixed-point trap. An N -stage CIC with rate change R and differential delay M has DC gain $(RM)^N$, so the integrators need $W_{\text{out}} = W_{\text{in}} + N \log_2(RM)$ bits. The textbook trick is to use **two's-complement wrap-around** in the integrators: intermediate values overflow, but because the comb stages later subtract, the overflow cancels exactly, provided the *final* result fits. This works because wrap-around arithmetic is a ring homomorphism. **Replace wrap with saturation — an apparently safer choice — and the cancellation is destroyed and the filter breaks.** A specification that says only "use saturating arithmetic throughout" would silently kill a correct CIC. This is why Part A's number-system chapter insists that overflow behaviour is a per-block decision.

E1.4 Adaptive filters

Table 56. Adaptive algorithms in hardware terms

Algorithm	Update	Multipliers per tap per update	Convergence
LMS	$\mathbf{w} \leftarrow \mathbf{w} + \mu \mathbf{e} \mathbf{x}^*$	2	Slow; sensitive to eigenvalue spread
NLMS	μ normalised by $\ \mathbf{x}\ ^2$	2 + norm	Robust to input level
Sign-error LMS	uses $\text{sgn}(\mathbf{e})$	1	Slower

Algorithm	Update	Multipliers per tap per update	Convergence
Sign-data LMS	uses $\text{sgn}(x)$	1	Slower
Sign-sign LMS	both signs	0 — adds only	Slowest; used in every high-speed SerDes
RLS	inverse correlation update	$O(N)$ per tap	Fast; expensive
Frequency-domain LMS	block FFT	—	Fast for long filters

석사 수준 · **Why sign-sign LMS dominates at 56 GBd.** Two independent facts combine. First, the channel changes on a timescale of milliseconds while symbols arrive every 18 ps — **adaptation can be nine orders of magnitude slower than the datapath.** Second, multipliers cannot run at the symbol rate at all. Taking only signs reduces the update to increment/decrement, which can be accumulated over thousands of symbols and applied occasionally. The resulting architecture has a *fast datapath* and a *slow adaptation loop* in a different clock domain. **For a modelling engineer this separation must be explicit in the model,** because a model that updates taps every symbol will not match RTL that updates them every 4096 symbols, even though both "implement sign-sign LMS". The update schedule is part of the specification.

E2. Transforms in Hardware: FFT and Friends

Table 57. FFT algorithm families

Algorithm	Complex multiplies	Structure	Note
Radix-2 DIT/DIF	$(N/2)\log_2 N$	Regular butterflies	Simplest control
Radix-4	fewer	4-point butterflies	~25% fewer multiplies
Split-radix	fewest for radix-2 ^k	Irregular	Control complexity
Winograd / PFA	Minimum multiplies	Length must factor into coprimes	Niche
Pipelined SDF	—	One butterfly per stage + delay line	Streaming; the standard for OFDM
MDC	—	Multiple data paths	Higher throughput
SSR / parallel	—	Several samples per clock	When the data rate exceeds the clock rate

석사 수준 · **The resource that actually limits an FFT is memory and the twiddle factors, not the butterflies.** A pipelined N -point SDF FFT needs $\log_2 N$ butterflies but a total of $N-1$ words of delay line, and a twiddle ROM whose size grows with N . For $N = 4096$ that is thousands of words of storage against a dozen arithmetic units. Two standard optimisations follow: exploit twiddle symmetry to store only one eighth of a period and derive the rest by sign and swap, or generate twiddles with a CORDIC instead of a ROM. **Which one wins depends on whether the target has cheap memory (FPGA) or cheap logic (ASIC)** — another instance of the rule that architecture choices are technology-dependent and must be re-derived, not memorised.

Bit-reversed output order is an interface decision, not an implementation detail. A DIF FFT naturally produces its output in bit-reversed order. Reordering costs a full N -word buffer and its bandwidth. In many systems the consumer — a channel equaliser, a magnitude detector — does not care about order, so the reordering is pure waste. Competent designs pass the bit-reversed order through and let the next block absorb it. **This means the reference model and the RTL will disagree about output order unless the specification says which convention is normative**, and a large fraction of "FFT mismatch" bug reports are exactly this.

Table 58. Numerical behaviour of fixed-point FFT

Issue	Cause	Handling
Magnitude growth	Each radix-2 stage can double magnitude	Scale by $\frac{1}{2}$ per stage (lose SNR), grow the word, or block floating point
Twiddle quantisation	Finite coefficient precision	Adds a noise floor; usually 2–4 bits more than data width
Rounding vs truncation	Bias accumulates over $\log N$ stages	Convergent rounding avoids DC bias
Overflow detection	Input-dependent	Saturate and flag, or scale conservatively

Table 59. Other transforms used in IP

Transform	Property	Application
DCT-II	Energy compaction for correlated signals	JPEG, H.264/HEVC (integer approximations)
Integer DCT	Exactly invertible, multiplier-free	Video codecs — avoids encoder/decoder drift
Hadamard	± 1 only	Secondary transform, spreading

Transform	Property	Application
Number-theoretic (NTT)	Exact modular arithmetic	Post-quantum crypto (ML-KEM/ML-DSA) , homomorphic encryption
Wavelet (DWT)	Time–frequency localisation	JPEG2000, denoising

석사 수준 · **The integer DCT in video codecs is a beautiful example of specification-driven design.** An exact DCT requires irrational coefficients; any two implementations would round differently, and since the decoder's output feeds back into the encoder's prediction loop, the tiny differences would accumulate into visible *drift*. H.264 solved this by **defining the transform to be a specific integer matrix**, so every conforming implementation produces bit-identical results. The lesson generalises directly to IP work: **when an algorithm sits in a feedback loop that spans two independently implemented systems, the specification must be bit-exact, not mathematical.** The same reasoning applies to a DFE's tap update, to an adaptive equaliser shared between link partners, and to any distributed control loop.

F1. The Wireline Link, End to End

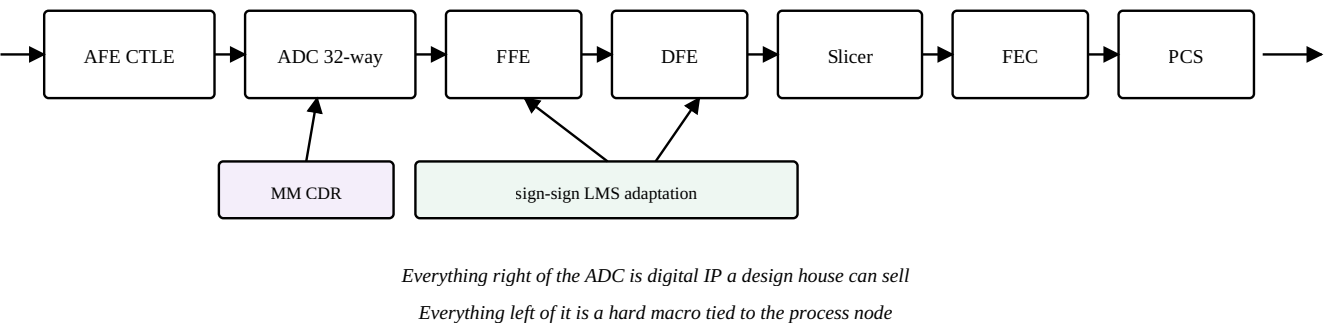


Figure 9. A modern ADC-DSP receiver. The PMA/PCS boundary is also the boundary of what can be sold as soft IP.

F1.1 Why the architecture looks like this

The structure above is not arbitrary; each block exists because a specific physical limit made the previous approach fail. Following that chain is the fastest way to understand the field.

Table 60. The forcing chain

Limit encountered	Response	New problem created
Channel loss rises as $\sqrt{f}$ (skin effect)	Equalise	Analogue boost amplifies noise as well as signal
CTLE alone cannot open the eye at 30 dB loss	Add DFE (subtracts decided symbols, no noise gain)	DFE feedback must close within 1 UI
1 UI = 17.86 ps at 56 GBd; latch + summer + wire + setup > 23 ps	Speculate (unroll) and parallelise (sub-rate)	Area grows; still not enough alone
Baud rate cannot be raised further — the channel will not pass it	PAM4 : two bits per symbol at the same baud rate	9.54 dB SNR penalty
PAM4 raw BER is $\sim 10^{-4}$, far from 10^{-15}	Forward error correction (RS-FEC KP4)	Latency, and FEC becomes mandatory in the standard
Analogue equalisation cannot be adapted precisely enough	Sample first with a fast ADC, equalise digitally	Time-interleaved ADC mismatch; huge digital block
200 Gb/s per lane: even KP4 is not enough	Concatenated FEC (outer RS-KP4 + inner Hamming)	More latency; more complex decoder

석사 수준 · **This table is the single most useful thing to memorise about high-speed links**, because it lets you derive the architecture rather than recall it, and it identifies where a newcomer can contribute. Notice that the last four rows are all *digital* responses to *analogue* limits. That migration — from analogue equalisation to digital — is why a digital IP house can participate in SerDes at all, and it is still in progress.

F1.2 Timing budget arithmetic

Table 61. DFE first-tap loop budget at 56 GBd (1 UI = 17.86 ps)

Element	Delay	Comment
Slicer (latch) clock-to-Q	12 ps	Regeneration plus output drive
Summer	8 ps	Analogue summing node settling
Wiring	6 ps	Layout-dependent; often underestimated
Setup at next slicer	5 ps	—
Total loop	31.0 ps	—
Available	17.86 ps	1 UI
Margin	-13.14 ps	Infeasible
Latch + wire + setup alone	23 ps	Still exceeds 1 UI

The last row is decisive and is often missed. Unrolling removes the *summer* from the loop by pre-computing both hypotheses, but the latch, the wiring and the setup time remain — and they already exceed one UI. **Therefore no amount of speculation alone reaches 56 GBd; sub-rate parallelism is mandatory.** Stating this as a measured budget rather than an opinion is what turns an architecture argument into an engineering conclusion, and it is exactly the kind of analysis a modelling engineer is expected to produce before any RTL exists.

F1.3 Equaliser taxonomy

Table 62. Equalisers compared

Type	Location	Removes	Noise effect	Constraint
TX FFE (de-emphas-is)	Transmitter	Pre- and post-cursor	None (pre-distortion)	Reduces transmitted amplitude
CTLE	RX ana- logue	Broad high-frequency loss	Amplifies noise	Limited number of poles/zeros
RX FFE	RX digital	Pre- and post-cursor	Amplifies noise	Needs ADC; tap count $\times$ rate
DFE	RX digital	Post-cursor only	None (uses de- cisions)	1 UI loop; error propagation
MLSE / Viterbi	RX digital	Optimal sequence detec- tion	None	Complexity grows as M^L
Nonlinear (Volterra, NN)	RX digital	Nonlinear channel ef- fects	—	Training, complexity; an active research area

석사 수준 · **Error propagation** is the price of the DFE's noise-free subtraction. A wrong decision is fed back and corrupts the next few symbols, so errors arrive in bursts rather than independently. Two consequences follow, and both are design decisions rather than analysis details. First, the raw BER is worse than an independent-error model predicts, so a link budget built on Q-function arithmetic alone is optimistic. Second — and more usefully — **bursty errors are exactly what a symbol-oriented code like Reed–Solomon handles well**, because several wrong bits inside one symbol still count as one symbol error. The choice of RS over a binary code at the FEC layer is therefore partly a consequence of the DFE at the equaliser layer. **Layers that look independent in a block diagram are coupled through their error statistics.**

F1.4 Clock and data recovery

Table 63. CDR phase detectors

Detector	Samples needed	Characteristic	Used with
Alexander (bang-bang)	Data + edge (2×)	Binary output; nonlinear	Classic NRZ links
Hogge	Data + edge	Linear output	Lower jitter, more complex
Gardner	2× oversampling	Modulation-independent	Wireless, some wireline
Mueller–Müller	Baud-rate only	Uses ISI structure; needs equalised signal	ADC-DSP receivers — no 2× sampling needed
Oversampling / majority	3–8×	Simple, robust, low rate	Low-speed links, FPGA

석사 수준 · **Why Mueller–Müller won in ADC-based receivers.** At 56 Gb/s, sampling at 2× the baud rate means 112 GS/s — roughly doubling the ADC's power and area, which are already the dominant cost. MM extracts a timing error from baud-rate samples alone by exploiting the asymmetry of the equalised pulse response: it drives the sampling phase to the point where the first pre-cursor and first post-cursor are equal. The subtlety is that **MM's lock point depends on the equaliser**, so CDR and equaliser adaptation interact; a modelling engineer who simulates them independently will not see the interaction, and joint convergence failures are a classic late-stage surprise. **Model the loops together or state explicitly that you have not.**

F2. Error Control Coding

F2.1 The coding landscape

Table 64. Codes by structure and application

Family	Alphabet	Decoding	Strength	Typical use
Parity / SECDED	binary	syndrome	1-bit correct, 2-bit detect	Memory ECC , register files
Hamming / extended	binary	syndrome	—	Inner code in concatenation
BCH	binary	BM + Chien	t bit errors	Flash, optical
Reed–Solomon	$GF(2^m)$	syndrome $\rightarrow$ BM $\rightarrow$ Chien $\rightarrow$ Forney	t symbol errors — burst tolerant	Ethernet KP4 , storage, DVB
Convolutional	binary	Viterbi (ML)	Random errors	Legacy wireless, satellite
Turbo	binary	iterative BCJR	Near capacity	3G/4G
LDPC	binary	belief propagation (min-sum)	Near capacity	5G NR data , Wi-Fi, 802.3ca, NAND
Polar	binary	SC / SC-List	Provably capacity-achieving	5G control
Fountain (LT, Raptor)	binary	BP	Rateless	Broadcast, storage
Concatenated	mixed	inner then outer	Burst + random	802.3dj 224G , deep space

석사 수준 · Choosing a code is choosing an error model. A binary code assumes errors are independent bit flips; a symbol code assumes they cluster. Getting this wrong is expensive in both directions — a strong binary code performs poorly on bursts, and an RS code wastes redundancy on isolated bit errors. The correct procedure is to **measure the error statistics of the actual slicer output**, including DFE error propagation and any burst noise, and choose accordingly. In concatenated schemes the division of labour is explicit: the inner code cleans up random errors and the outer code mops up the bursts that the inner decoder's own failures produce. **The inner decoder's failure mode is the outer decoder's input distribution** — a subtlety that must be in the system model.

F2.2 Reed–Solomon in detail

Table 65. RS decoder pipeline

Stage	Computation	Parallelism	Hardware character
Syndrome	$S_i = r(\alpha^{tcr+i}), i=0\dots 2t-1$	Fully parallel ($2t$ independent evaluations)	$2t$ constant-multiplier accumulators
Key equation (BM)	Find $\sigma(x)$	Sequential — $2t$ dependent iterations	This is the critical path
Chien search	Roots of $\sigma(x)$	Parallel over positions	n/P cycles
Forney	$e_j = \omega/\sigma'$ evaluated at roots	Parallel	Needs GF inversion (ROM or Itoh–Tsujii)
Correction	XOR into the buffer	Parallel	Requires the codeword buffered

석사 수준 · **The Berlekamp–Massey iteration is the same kind of obstacle as the DFE loop and the OOO wakeup loop.** Each iteration depends on the previous discrepancy, so it cannot be pipelined away. The standard remedies mirror those in the other two cases: *inversionless* BM removes the GF inversion from the loop (replacing it with an extra multiplication), and *re-formulated* BM restructures the recurrence so that fewer operations sit inside it. A design house that recognises this pattern can move an engineer from a SerDes project to an FEC project and expect them to be productive quickly, because the underlying problem is identical.

Table 66. RS decoder behaviour that must appear in the datasheet

Condition	Behaviour	Why it must be documented
$\leq t$ symbol errors	Always corrected	The guarantee being sold
$> t$ errors	Detected or mis-corrected	Not all are detected. Claiming otherwise is false
Mis-correction probability	Roughly $n^t / (t! 2^{m(n-k-t)})$	Customers at 10^{-15} FLR need this number
Erasures decoding	Corrects up to $n-k$ erasures	If the receiver can flag unreliable symbols, capacity doubles
Latency	Fixed, = pipeline depth	Real-time systems need the number

“All errors beyond t are detected” is a false statement that appears in datasheets. A received word with more than t errors may land within distance t of a *different* valid codeword, in which case the decoder confidently produces the wrong answer. A measured illustration on a small code, RS(15,9) with $t = 3$: injecting exactly four symbol errors 120 times gave **117 detected failures and 3 mis-corrections** — never a correct recovery, but not always a detection either. An honest specification states the mis-correction probability; a verification plan must distinguish the three outcomes (corrected / detected / mis-corrected) rather than counting pass and fail.

F2.3 LDPC

Table 67. LDPC decoding choices

Choice	Options	Effect
Message form	Probability vs log-likelihood ratio	LLR turns products into sums — essential in hardware
Check-node update	Sum-product (tanh) vs min-sum	Min-sum loses ~0.5 dB, removes transcendental functions
Correction	Normalised or offset min-sum	Recovers most of the loss for one constant
Schedule	Flooding vs layered	Layered halves the iteration count; creates pipeline hazards
Parallelism	Fully parallel / row / block	Quasi-cyclic structure enables block parallelism
Early termination	Syndrome check per iteration	Large average power saving ; worst-case latency unchanged

LDPC's fixed-point rule is the opposite of the CIC's. An LLR's sign carries the hard decision. If an LLR wraps in two's complement, a strong belief in “0” becomes a strong belief in “1”, and the decoder does not merely lose precision — it is actively misled, and iteration amplifies the damage. Measured on a 5G NR base-graph decoder, reducing the message width by three bits gave BLER 0.250 with saturation and **1.000 — complete failure — with wrap-around**. Meanwhile the CIC filter of Chapter E1 *requires* wrap-around to function. **There is no global “correct” overflow policy; it is a per-block property that must be stated in the specification and enforced by tests.**

석사 수준 · **Layered scheduling and pipeline hazards.** In a layered decoder, layer j may need a variable node that layer i is still updating, forcing a stall whose length depends on the pipeline depth D and on the layer ordering. It is tempting to optimise the ordering against a closed-form stall count, but that formula is only an **upper bound**: it assumes every potential conflict costs a full stall. A measured comparison on 5G NR base graph 1 at $D = 4$ gave the formula 108 stall cycles while a cycle-accurate model and the RTL both gave **80** — the difference between a 70.1% and a 62.0% efficiency loss. **Optimising**

a schedule against the wrong objective function finds the wrong schedule. Build the cycle-accurate model first; it is a day of work and it changes the answer.

G1. Electromagnetics for the Digital Engineer

A digital IP engineer meets electromagnetics at three boundaries: the package and board that carry the block's signals, the channel a SerDes must equalise, and the antenna of a radio or radar system whose baseband processing is the block itself. This chapter covers what is needed at those boundaries.

G1.1 From Maxwell to circuits

Table 68. Regimes and the model that applies

Regime	Condition	Model	Consequence
Lumped	size $\ll \lambda$	KCL/KVL, R L C	Voltage is well defined everywhere
Quasi-static	size $\leq \lambda/10$	Lumped + parasitics	Inductance begins to matter
Distributed	size $\geq \lambda/10$	Transmission line	Position matters; reflections occur
Full wave	structure comparable to λ	Maxwell solvers	Radiation, resonance, modes

석사 수준 · **The threshold keeps moving toward the designer.** The criterion is not clock frequency but edge rate: significant spectral content extends to roughly $0.35/t_r$. A 50 ps edge has content to 7 GHz regardless of whether the clock is 100 MHz, and at 7 GHz a 2 cm trace is a transmission line. **Slow clocks do not protect you from fast edges.** This is why deliberately slowing output slew rates is a legitimate design technique — and why an IP's output driver strength is a specification item with EMI consequences, not just a timing one.

G1.2 Transmission lines

$$Z_0 = \sqrt{(R + j\omega L)/(G + j\omega C)} \rightarrow \sqrt{L/C} \text{ (lossless)}$$

Table 69. Line types

Type	Z_0 range	Where
Microstrip	50–100 Ω	Outer PCB layer; radiates more
Stripline	50–100 Ω	Inner layer; better shielded, slower (higher ϵ_{eff})
Coplanar waveguide	tunable	On-chip RF; ground on the same layer
Differential pair	85–100 Ω differential	The standard for high-speed serial
On-chip global wire	—	Usually RC-dominated, not a true TL

Table 70. Termination strategies

Scheme	Placement	Power	Note
Series (source)	At driver	Low	Doubles the driver's output swing requirement
Parallel	At receiver	High (static)	Best match
Thevenin	At receiver	High	Sets a DC bias
AC-coupled	In line	Low	Requires DC-balanced coding (8b/10b, 64b/66b)
On-die termination (ODT)	Inside the chip	Moderate	Calibrated against an external precision resistor

석사 수준 · **AC coupling and line coding are linked requirements.** A series capacitor blocks DC, so a long run of identical symbols causes the signal to droop toward the decision threshold — baseline wander. The line code must therefore bound the run length and keep the running disparity small. 8b/10b does this with an explicit disparity rule at 25% overhead; 64b/66b uses a scrambler and accepts a small probability of a long run, at 3% overhead. **The coding choice is thus determined by an analogue component.** An engineer who knows only the coding layer cannot explain why 64b/66b needs a scrambler with a particular polynomial, nor why certain data patterns are used as stress tests.

G1.3 Antennas and propagation

$$P_r = P_t G_t G_r (\lambda/4\pi d)^2 \quad (\text{Friis})$$

Table 71. Antenna and propagation quantities

Quantity	Definition	Design relevance
Gain G	Directivity × efficiency	Trades coverage for range
Effective aperture	$A_e = G\lambda^2/4\pi$	Higher frequency → smaller aperture for the same gain
Beamwidth	$\approx 70^\circ/\sqrt{D}$	Array size sets angular resolution
Path loss exponent	2 (free space) to 4+ (urban)	Link budget
Multipath / delay spread	—	Sets the equaliser or OFDM CP length
Doppler shift	$f_d = 2v/\lambda$	Coherence time; tracking loop bandwidth
Fading	Rayleigh / Rician	Diversity, MIMO

석사 수준 · **Delay spread converts directly into a hardware parameter.** If the channel's impulse response lasts $\tau_{\max}$, a single-carrier receiver needs an equaliser spanning $\tau_{\max}/T_s$ symbols, and an OFDM receiver needs a cyclic prefix at least that long. The first costs taps (area, adaptation time); the second costs throughput (the CP is overhead). **This is the quantitative form of the choice between single-carrier and OFDM**, and it explains why wireline links — where delay spread is long but the channel is static — use equalisers, while wireless links with time-varying multipath use OFDM. The same physics, opposite conclusions, because the *rate of change* differs.

G1.4 Noise figure and receiver chains

$$F_{\text{total}} = F_1 + (F_2 - 1)/G_1 + (F_3 - 1)/(G_1 G_2) + \dots$$

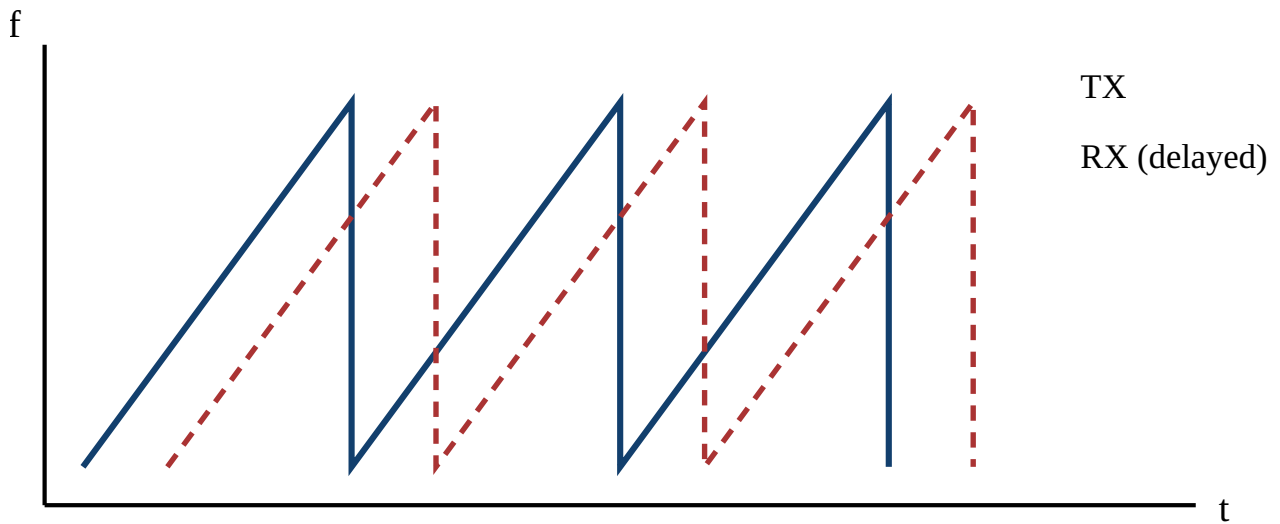
석사 수준 · **Friis's formula justifies the entire architecture of a receiver front end.** Because later stages are divided by the cumulative gain ahead of them, the first amplifier dominates the noise figure — hence the low-noise amplifier, placed as close to the antenna as possible, often before any filter whose insertion loss would add directly to F_1 . The counterweight is linearity: gain early in the chain means large signals later, so the cascaded third-order intercept point degrades as $1/\text{IIP3}_{\text{total}} = 1/\text{IIP3}_1 + G_1/\text{IIP3}_2 + \dots$ **Noise pushes gain forward; linearity pushes it backward.** Every receiver's gain distribution is a solution to that tension, and the digital back end inherits the result as its required dynamic range — which is to say, as its ADC bit count.

G2. Radar: Signal Processing as an IP Block Family

Automotive and industrial radar is one of the largest current markets for signal-processing IP, and its chain is a clean illustration of how physical parameters become hardware parameters.

$$P_r = P_t G^2 \lambda^2 \sigma / ((4\pi)^3 R^4) \quad (\text{radar equation})$$

학부 복습 · The fourth-power range dependence is the defining fact of radar: doubling the detection range requires sixteen times the received power. Since transmit power and antenna gain are bounded, the remaining lever is **processing gain** — coherent integration — which is entirely a digital matter and therefore entirely IP.



$$\text{Beat frequency } f_b = 2 \cdot R \cdot S / c \quad (S = \text{chirp slope})$$

Range from frequency; velocity from phase change across chirps

Figure 10. FMCW radar. Range appears as a beat frequency; velocity appears as phase progression across successive chirps.

G2.1 FMCW parameter derivation

Table 72. From requirement to hardware parameter

Requirement	Relation	Hardware consequence
Range resolution ΔR	$\Delta R = c/2B$	Sweep bandwidth — 4 GHz gives 3.75 cm
Maximum range $R_{\max}$	$f_{b,\max} = 2R_{\max}S/c$	ADC sample rate and IF bandwidth
Velocity resolution Δv	$\Delta v = \lambda/(2N_c T_c)$	Number of chirps per frame → frame memory
Maximum velocity	$v_{\max} = \lambda/(4T_c)$	Chirp repetition period
Angular resolution	$\approx \lambda/(N_a d \cos \theta)$	Number of antennas → number of RX chains
Processing gain	$10\log(N_s N_c)$	FFT sizes → compute and memory

석사 수준 · **The range–Doppler map is a two-dimensional FFT** and its dimensions come straight from the table: a fast-time FFT across samples within one chirp yields range bins; a slow-time FFT across chirps at each range bin yields Doppler bins. A typical automotive frame — 256 samples $\times$ 128 chirps $\times$ 8 virtual antennas — requires a corner-turn between the two FFTs, and **that transpose, not the FFTs, dominates the memory bandwidth**. Designing the memory organisation (ping-pong banks, tiled transpose, or on-the-fly second-stage accumulation) is the real architectural work. This is a recurring theme: in data-intensive blocks the arithmetic is rarely the bottleneck.

G2.2 Detection

Table 73. CFAR detectors

Type	Noise estimate	Strength	Weakness
CA-CFAR	Mean of surrounding cells	Simple, efficient in homogeneous noise	Masks targets near clutter edges
GO-CFAR	Greater of the two halves	Handles clutter edges	Loses closely spaced targets
SO-CFAR	Smaller of the two halves	Resolves close targets	High false alarm at edges
OS-CFAR	k -th order statistic	Robust to interfering targets	Needs a sorter — more hardware
Adaptive / ML	learned	Best in complex scenes	Verification difficulty

석사 수준 · **CFAR sets a threshold, not a decision**. The multiplier α is chosen so that the probability of false alarm is fixed regardless of the noise level — that is what "constant false alarm rate" means. For CA-CFAR with N reference cells, $\alpha = N(P_{FA}^{-1/N} - 1)$. Two hardware consequences: the block needs a divider or a reciprocal table, and it needs the N reference cells buffered, which for a 2-D CFAR means a sliding window over a two-dimensional array. **OS-CFAR's sorter is the most hardware-hungry element in a typical radar back end**, and choosing between a full sorting network and an approximate selection is a genuine PPA decision that a design house can differentiate on.

G2.3 Angle estimation and MIMO

Table 74. Angle-of-arrival methods

Method	Resolution	Computation	Robustness
FFT beamforming	Rayleigh limit	One FFT	Very robust
Capon / MVDR	Super-resolution	Matrix inverse per snapshot	Needs good covariance estimate
MUSIC	Super-resolution	Eigendecomposition	Needs the source count
ESPRIT	Super-resolution	SVD + rotation	Needs array structure
Compressive sensing	High	Iterative solver	Sensitive to model mismatch

석사 수준 · **This is where Part A's matrix decompositions become products**. MUSIC needs the eigendecomposition of an $N \times N$ covariance matrix per snapshot; Capon needs its inverse. For an 8–16 element array that is a small, dense, Hermitian, **positive-definite** problem — exactly the case where Cholesky applies and streams. A design house that owns a parameterised streaming Cholesky or Jacobi eigenvalue block can sell it into radar, into MIMO communications, and into adaptive beamforming, because all three reduce to the same kernel. **Recognising that one kernel serves three markets is how a small IP portfolio becomes viable**.

석사 수준 · **MIMO radar's virtual array** deserves a precise statement because it is the main reason automotive radar improved so quickly. With N_t transmitters and N_r receivers, transmitting orthogonal waveforms (time-, frequency-, or code-division) and separating them at the receiver yields $N_t \times N_r$ distinct transmit–receive paths, which behave like a physical array of

that many elements. Three transmitters and four receivers give a twelve-element virtual array — a threefold improvement in angular resolution at the cost of **digital** separation logic rather than antennas and RF chains. **Once again the solution to a physical limit is a digital block.**

H1. Verification: Theory and Organisation

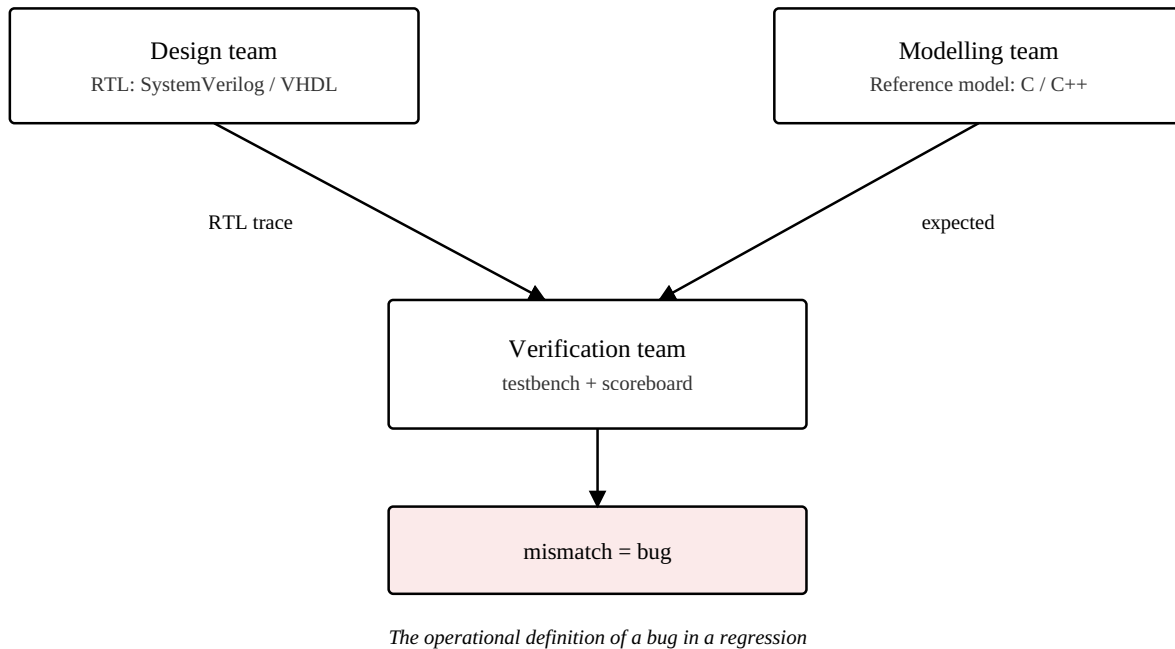


Figure 11. The three-team structure. It exists to make “correct” an operational, testable notion.

Verification consumes more effort than design in every serious IP project — commonly cited at 60–70% of the schedule. Understanding why it is organised as it is prevents a great deal of wasted work.

H1.1 Why two independent implementations

학부 복습 · A specification written in prose is ambiguous; a single implementation is self-consistent by construction and therefore cannot reveal its own misreadings. Two implementations built independently from the same specification will agree only where both read it the same way. **Disagreement localises ambiguity.** This is the same principle as dissimilar redundancy in avionics, applied to development rather than to operation.

석사 수준 · **The argument fails if the two implementations are not independent.** If the modeller and the designer sit together, share an interpretation, and one consults the other's code when unsure, the common-mode error rate rises and the regression turns green on a shared misunderstanding. Organisations counter this in three ways: physical or organisational separation; deriving one implementation from a **formal, executable specification** (the RISC-V Sail model is the canonical example, generating both prose and a C emulator from one source); and an **external oracle** — standard test vectors from NIST, IEEE or a conformance suite. **The third is the strongest and the cheapest, and it is the reason standards-based blocks are the best first product for a small IP house.**

H1.2 The verification stack

Table 75. Verification levels

Level	Checks	Catches	Cannot catch
Lint	Coding rules	Latches, width mismatches, incomplete sensitivity	Anything functional
CDC / RDC static	Crossing structure	Missing synchronisers, reconvergence	Functional protocol errors
Unit simulation	Block behaviour vs model	Most functional bugs	Integration, timing
Formal property	Exhaustive over state space	Deadlock, unreachable states, protocol violations	Deep datapath equivalence (state explosion)
Formal equivalence (LEC)	RTL vs netlist	Synthesis and ECO errors	Specification errors
Integration / SoC sim	Blocks together	Interface mismatches, arbitration	Long-running system behaviour
Gate-level with timing	Netlist + SDF	X-propagation, timing-dependent bugs	Slow; limited coverage
Emulation / FPGA prototype	Real software on real RTL	Software-visible bugs, long sequences	Analogue, timing corners
Post-silicon	Actual device	Everything else	— but fixes are expensive

X-propagation deserves a specific warning because it is asymmetric between RTL and gates. In RTL simulation, an if (x) with an unknown condition may take a branch and produce a definite value, hiding the uncertainty — this is *X-optimism*. In gate-level simulation the same unknown spreads — *X-pessimism*. A design that relies on registers being initialised by data flow rather than by reset can therefore pass RTL simulation and fail in gates, or vice versa. The remedies are X-propagation-aware simulation modes and, better, **explicit initialisation of any register whose value is read before it is written**. For an IP vendor this is also a deliverable question: if your block requires *n* cycles of reset before its outputs are valid, say so.

H1.3 Coverage and closure

Table 76. Coverage metrics

Metric	Measures	Automat-ic?	Weakness
Line / block	Statements executed	Yes	Says nothing about values
Branch / condition	Decisions taken	Yes	—
Expression (MC/DC)	Each condition independently affects the outcome	Yes	Required for DO-254 / ISO 26262
Toggle	Every bit changed both ways	Yes	Weak functional signal
FSM state / transition	States and arcs visited	Yes	—
Functional (coverage groups)	Scenarios the engineer defined	No	Blind to scenarios nobody thought of
Assertion	Properties actually exercised	Partly	Vacuous passes must be excluded
Mutation / fault injection	Whether the testbench would notice a bug	Yes	The only metric that tests the testbench

석사 수준 · Mutation coverage is the metric that answers the question the others avoid: would this testbench detect a bug if there were one? The procedure is to inject a small, semantically meaningful change — invert a comparison, drop a term, stick a bit — and confirm the regression turns red. A mutant that survives identifies a check that does not exist or a stimulus that never reaches the code. This is cheap to automate and routinely reveals that a large fraction of a mature suite is

inert. **The same technique applies to the reference model itself**, which is otherwise unverified: break the model deliberately and confirm the model's own self-checks fail. A verified reference model is a differentiator that very few IP vendors can claim.

H1.4 The scoreboard problem

Table 77. Comparison strategies

Situation	Strategy	Pitfall
In-order, fixed latency	FIFO compare	—
In-order, variable latency	FIFO + timeout	Timeout too tight → false failures
Out-of-order, tagged	Per-tag queues	A single queue produces both false failures and false passes
Multiple legal orders	Compare against a set of allowed outcomes	Requires the model to enumerate them
Don't-care fields	Mask before comparing	Masking too much hides real bugs
Weak memory model	Litmus tests , not trace comparison	Trace comparison is simply invalid here

석사 수준 · The AXI case is worth stating concretely because it is so common. AXI guarantees ordering only *within* an ID; transactions with different IDs may complete in any order. A scoreboard with one queue will report a failure when the DUT legitimately reorders across IDs, and — more dangerously — may accept a genuine intra-ID reordering as long as the totals match. **The correct structure is one queue per ID**, and knowing this is a direct consequence of having read the protocol specification rather than inferring behaviour from waveforms. It is a frequent and productive point of contact between the modelling and verification teams.

H1.5 Regression engineering

Table 78. Regression discipline

Practice	Reason
Record the seed with every run	Without it a failure is not reproducible
Classify failures automatically	Triage time dominates otherwise
Track failure <i>signatures</i> , not counts	Distinguishes new from known
Never close an intermittent failure as 'flaky'	Intermittent failures are usually real races
Keep a golden log of the last known-good	Enables bisection
Measure regression runtime	A suite nobody can run nightly is not a suite
Separate infrastructure failures	Licence and disk errors must not be counted as bugs

The prohibition on dismissing intermittent failures is the most frequently violated rule in this book. An intermittent failure means the outcome depends on something the test does not control — a race, an uninitialised value, a timing-dependent path. All three exist in the silicon too. **Closing such a ticket does not remove the bug; it removes the evidence.** The correct disposition is to keep it open with the failing seed recorded, even if it cannot be root-caused immediately.

H2. The Implementation Flow

Table 79. From RTL to GDSII

Step	Input → output	Key decisions	Common failure
Elaboration	RTL → generic netlist	Parameter resolution	Unintended latch inference
Logic synthesis	generic → standard cells	Constraints, effort, V_{TH} mix	Over-constrained clock → area and power explosion
DFT insertion	+ scan chains	Chain count and length	Scan shift timing
Floorplan	—	Block placement, pin assignment, power grid	The decision with the largest downstream impact
Placement	cells → locations	Congestion vs timing	Routing congestion
CTS	+ clock tree	Skew target, useful skew	Clock power, OCV
Routing	—	Layer assignment, shielding	DRC violations, crosstalk
Sign-off STA	—	Corners, derating, SI	Missing a corner
Physical verification	DRC, LVS, antenna	—	—
Extraction + final timing	RC → delays	—	—

석사 수준 · Floorplanning is where an IP block's shape becomes a commercial issue. A block delivered as RTL will be placed by the customer, whose aspect ratio, pin locations and macro placement may be nothing like the vendor's. If the block's internal timing depends on a particular arrangement — a wide datapath that must stay compact, a memory that must sit next to its controller — then the RTL alone is not a sufficient deliverable. Serious IP therefore ships a **floorplan guideline**, sometimes a hardened macro, and always a statement of the assumed aspect ratio and utilisation. **A block that met timing in the vendor's flow and misses in the customer's is the single most damaging support event a small IP house can have**, because it is expensive to debug remotely and it destroys confidence.

Table 80. Sign-off corners

Axis	Values	Why
Process	SS, TT, FF, SF, FS	Global and local variation
Voltage	min, nominal, max	IR drop and regulation tolerance
Temperature	−40, 25, 125 °C	Both extremes — temperature inversion makes cold the worst case at low V
RC extraction	Cmin, Cmax, RCmin, RCmax	Metal thickness variation
Mode	functional, scan, BIST, low power	Different constraints apply
Ageing	fresh, end-of-life	NBTI/PBTI drift

석사 수준 · The corner count multiplies. Five process points × three voltages × three temperatures × four RC × four modes is 720 combinations; full sign-off on all of them is not affordable. The industry response is **statistical or parametric OCV** (POCV), which models variation as a distribution rather than enumerating corners, plus careful selection of dominant corners. For an IP vendor the practical obligation is to state **which corners were signed off** in the PPA report. "500 MHz" without a corner is not a specification, and a customer who assumes typical-typical will be disappointed at slow-slow.

Table 81. High-level synthesis in the flow

Aspect	Reality
Input	C/C++/SystemC with vendor pragmas — not portable across tools
Strength	Design-space exploration: one source, many area/throughput points
Weakness	Generated RTL is hard to read and debug; timing closure less predictable
Where it wins	Datapath-dominated blocks with regular loops (filters, transforms, crypto)
Where it loses	Control-dominated logic, precise cycle-level interfaces
Verification role	The same C source doubles as the reference model , because a general compiler ignores the pragmas — convenient, but it shares every algorithmic assumption with the design

The last row is a genuine methodological trap. Using the HLS source as the golden model gives *zero* independence: the two implementations are literally the same text. It verifies the HLS tool, not the design. **If the HLS C is the design, the reference model must be written separately** — ideally in a different language, by a different person, from the specification.

H3. Hardware Security

Table 82. Threat model, mechanism, countermeasure

Threat	Mechanism	Countermeasure	Verifiable by simulation?
Timing side channel	Execution time depends on secret	Constant-time algorithms; no secret-dependent branches or table indices	Partly — can check control flow
Power / EM (SPA, DPA, CPA)	Switching activity correlates with data	Masking (Boolean or arithmetic), threshold implementations, DOM	No — requires TVLA on silicon or power simulation
Fault injection (laser, glitch, EM)	Flip a bit to skip a round or corrupt a check	Redundant/complementary FSMs, computation duplication, output checks	Partly — by explicit fault simulation
Probing / FIB	Physically read an internal wire	Shields, sensors, routing obfuscation	No
Scan / debug access	Read state through test infrastructure	Lock scan after test; life-cycle state control	Yes
Rowhammer memory disturb	Repeated activation flips neighbours	Refresh management, tracking	Partly
Supply chain	Trojan insertion, counterfeiting	Split manufacturing, logic locking, PUF identity	No

석사 수준 · **The fourth column is the key organisational insight for a modelling engineer.** A functional regression, no matter how complete, is blind to four of the seven rows. Saying “the AES block is verified” is therefore ambiguous, and in a security review the ambiguity will be challenged. The mature formulation is: *functional equivalence to FIPS-197 is verified by NIST vectors and constrained-random comparison against an independent model; side-channel resistance is assessed separately by TVLA on the test chip; fault resistance is assessed by gate-level fault injection.* **Three claims, three methods, three owners.**

Table 83. Masking: the principle and its cost

Concept	Statement	Consequence
Boolean masking	x represented as $(x \oplus m, m)$	Each share is independent of $x \rightarrow$ first-order leakage removed
Linear operations	Apply to each share separately	Nearly free
Nonlinear operations	Shares must interact	The expensive part — this is why the AES S-box is rebuilt in arithmetic
Glitches	Transient values can momentarily depend on x	Registers must separate share-combining stages
DOM	Domain separation + fresh randomness + registers	Provably glitch-resistant; needs a continuous randomness supply
Order d masking	$d+1$ shares resist d -th order analysis	Cost grows roughly as $(d+1)^2$

석사 수준 · **Randomness becomes a system-level resource.** A DOM-protected S-box consumes fresh random bits on every evaluation. An AES core performing sixteen S-box lookups per round for fourteen rounds therefore needs a sustained random-bit rate that a slow true-RNG cannot supply, so the design includes a fast PRNG reseeded from the TRNG. That PRNG's quality, its reseed policy, and its behaviour when starved are now **functional** requirements of the cipher block, and they are

verifiable. **A security countermeasure has turned into an ordinary functional specification with a throughput requirement** — which is precisely the kind of boundary a modelling engineer should be able to identify and take ownership of.

Table 84. Standards a security IP is measured against

Standard	Scope
FIPS 140-3	Cryptographic module validation
Common Criteria (ISO 15408)	Evaluation assurance levels
ISO/IEC 17825	Side-channel test methods
NIST CAVP	Algorithm test vectors — the practical oracle
SP 800-90A/B/C	RNG construction, entropy, and validation
FIPS 203/204/205	Post-quantum: ML-KEM, ML-DSA, SLH-DSA
OCP Caliptra	Datacentre root of trust specification
ISO 26262 / IEC 61508	Functional safety (distinct from security)

W1. What a One-Person Design House Actually Sells

The central proposition of this volume is that a customer does not buy RTL. They buy **transferred risk**. A competent customer could usually write the block themselves; what they cannot cheaply produce is the evidence that it is correct, the documentation that makes it integrable, and a commitment to fix it when it is not. **Sellability is therefore a property of the evidence package, not of the code.** Everything in this volume follows from that.

W1.1 The deliverable set

Table 85. Minimum viable IP package

#	Deliverable	Content	What its absence signals to a buyer
1	Product brief (2–4 pp)	Function, performance, resources, interfaces, standards conformance	Not a product
2	User guide	Parameters and legal combinations, ports with timing diagrams, register map, integration procedure, reset and clocking requirements	Integration will consume the vendor's time, not the customer's
3	Synthesisable RTL	Parameterised, lint-clean, no tool-specific constructs	—
4	Reference model (C/C++ or Python)	Bit-accurate; usable in the customer's own environment	The customer cannot verify integration independently
5	Testbench and vectors	Self-checking, runnable on an open simulator if possible; standard vectors included	No acceptance test is possible
6	Verification report	Method, coverage achieved, tests passed, known limitations	The most common omission and the most revealing
7	Constraints (SDC) and synthesis scripts	Clock definitions, exceptions with justification, a reproduction recipe	Customer cannot reproduce the timing claim
8	PPA report	Area, frequency, power — with process, library, corner, utilisation stated	The numbers are unusable
9	Example design	Minimal system on a named FPGA board or simulation target	Evaluation takes weeks instead of a day
10	Release notes and versioning	Change history, compatibility policy	No basis for a long-term commitment
11	Licence and third-party notices	Scope of use, open-source bill of materials	Blocked in legal review
12	IP-XACT metadata (recommended)	Machine-readable ports, parameters, register map	Manual integration

Stating known limitations increases the probability of a sale. This is counter-intuitive and consistently true. The reviewer is an engineer who knows that every block has boundaries; a report without them reads as either incomplete verification or concealment. A limitation stated precisely — "throughput is guaranteed only when the sink asserts `ready` at least once every four cycles" — is information the integrator needs, and it demonstrates that the vendor has characterised their own block.

W1.2 Choosing the first product

Table 86. Suitability screen for a solo developer

Criterion	Good	Bad	Reason
Source of truth	Public standard with test vectors	Proprietary or informal	A solo developer cannot arbitrate correctness
Demand	Mandated by a standard	Nice to have	The customer must <i>need</i> it, not merely like it
Size	One function, 3–5 kLOC	Subsystem	Verification volume scales superlinearly
Domain	Digital, process-independent	Analogue/mixed-signal	PDK, tools and measurement equipment are out of reach
Interface	One standard bus	Custom protocol	Reuse and integration
Lifetime	Consumer/datacentre	Automotive/aerospace	10–20 year support obligations
Novelty	Well-understood algorithm	Novel algorithm	Proving novelty costs more than implementing it

석사 수준 · The last row is the one most often got wrong by technical founders. An IP business sells power, performance, area, interfaces, verification and support — not algorithmic novelty. A well-known algorithm has an external oracle, comparable prior implementations to benchmark against, and a customer who already knows they need it. A novel algorithm has none of these, and the first question every customer asks ("how do I know it works?") has no cheap answer. **Existing implementations are an asset, not competition: they give you a baseline to quantify against.**

W1.3 Licensing and contracts

Table 87. Licence structures

Model	Mechanics	Suits
Per-design licence	Fixed fee, one tape-out	Most common; predictable for both sides
Multi-use / site	Fee for unlimited designs in a period	Larger customers
Royalty per unit	Fee per shipped device	High-volume; requires audit rights
Hybrid	Reduced up-front plus royalty	Industry norm for significant IP
Source vs obfuscated	RTL source, encrypted RTL, or netlist	Source access materially increases price
Maintenance	15–20% of licence per year	The recurring revenue that makes a solo practice viable

Table 88. Contract clauses that matter most to a solo vendor

Clause	Position to take	Why
Scope of use	Enumerate designs, derivatives, resale explicitly	The most frequent source of dispute
Acceptance criteria	Name the exact tests that constitute acceptance	Otherwise payment can be withheld indefinitely
Warranty period	Bounded (e.g. 12 months) defect correction	Unbounded is uninsurable
Limitation of liability	Cap at fees received	A consequential-damage claim on a tape-out would end a solo business
IP indemnity	'To the best of knowledge', capped	Unlimited patent indemnity is not survivable
Third-party components	Disclose the open-source bill of materials	Legal review will ask
Escrow	Offer it	Removes the 'what if you disappear' objection cheaply

Clause	Position to take	Why
Support response	Define business-hours response, not resolution	Resolution cannot be promised

석사 수준 · **The indemnity clause deserves specific preparation.** An unqualified promise that the IP infringes no patent exposes a one-person business to a liability unrelated to its size. The defensible position combines three things: a knowledge qualifier, a liability cap tied to fees received, and — crucially — **evidence that a search was actually performed.** A documented patent search, even a modest one, changes the character of any later dispute and is worth the few days it takes. Record the databases searched, the classification codes, the date, and the conclusions, exactly as you would record a verification campaign.

W2. The Solo Process, Stage by Stage

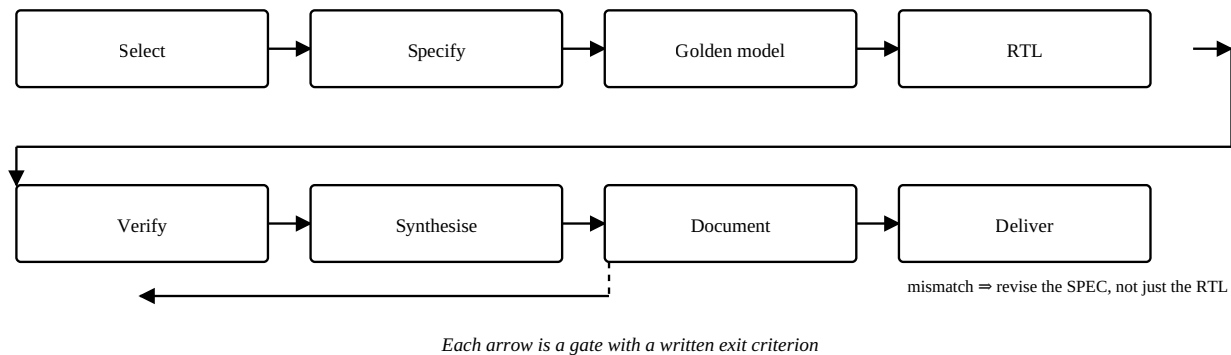


Figure 12. The development flow. What makes it a process rather than a sequence of activities is that each arrow has a written exit criterion.

Table 89. Stage gates

Stage	Output	Exit criterion (all must be true)
Select	Prior-art note	A named standard mandates it • prior implementations listed by name and size • the specification text is obtainable • a patent search has been performed and recorded
Specify	Specification document	Every parameter has a range and legal-combination rules • undefined behaviour is marked as such • word lengths, rounding, overflow policy and reset values are written down • reset and clocking requirements stated
Golden model	Reference model + self-tests	Standard vectors pass • deliberate mutation of the model turns the tests red • the model is deterministic • a second independent implementation or an external oracle agrees
RTL	Synthesisable RTL	Lint clean • no inferred latches • CDC static analysis clean • synthesises without warnings at the target frequency
Verify	Testbench, coverage, regression logs	Zero mismatches against the model • functional coverage goals met • no unexplained intermittent failures • mutation testing performed
Synthesise	PPA report	Timing closed at the stated corner • conditions fully documented
Document	The twelve deliverables	No section empty • the example design runs from a clean checkout
Deliver	Release	Acceptance tests defined and demonstrated

석사 수준 • **The gate that solo developers skip is the third one: verifying the reference model.** Because the model defines correctness, there is a natural tendency to treat it as correct by definition. It is not, and an error there propagates into the RTL (which is written to match it) and into the testbench (which checks against it), so the regression is green and the silicon is wrong. The three cheap defences are standard vectors, a second independent implementation, and mutation testing of the model's own checks. **In a one-person operation, where independence between designer and modeller cannot be organisational, these substitutes are the only thing standing in for it.**

W2.1 Specification: the parts that are always missing

Table 90. Specification checklist for numerical blocks

Item	Question to answer explicitly
Word lengths	Every signal's Q format, including internal accumulators
Rounding	Truncate, round-half-up, or round-half-even — at every reduction point
Overflow	Saturate or wrap , per signal — they are not interchangeable
Reset values	What is every state element after reset
Accumulation order	Fixed or unspecified — it matters under saturation
Latency	Exact cycle count, and whether it is data-dependent
Throughput	Initiation interval and any conditions on backpressure
Error behaviour	What happens on illegal input — and is it flagged
Undefined regions	Marked as such, with the expectation that DV masks them

A measured illustration of why reset values belong in the specification. In a feedback structure (DFE, IIR, accumulator), a mismatch in the initial register value causes the first outputs to differ, and the feedback carries the difference forward. In one measured comparison, 103 of 200 vectors mismatched; after aligning the model's and the RTL's reset value the count fell to 10, and the remaining ten had a separate cause. **Half the apparent failures were one unwritten specification line.** Debugging that without the specification costs days; writing it costs a sentence.

W2.2 Building the golden model

Table 91. Reference model construction rules

Rule	Rationale
Do not use host arithmetic for fixed or floating point	Results become machine-dependent; use explicit integer or SoftFloat
Hide state access behind accessors	Allows the representation to change without touching every operation
One file (or function) per operation where possible	Extensions are additive; the Spike ISS is the model for this
Make it fast	It will run millions of times in regression
Make it deterministic	Seeded randomness only; no wall-clock, no hash order
Emit a trace	The DV team cannot compare what it cannot see
Annotate with specification section numbers	Three years later, this is the only record of why
Write tests that bite	Deliberately break the model and confirm the tests fail

석사 수준 · **A concrete pattern for “tests that bite”.** For an error-correcting decoder, a test that injects zero errors passes even if the decoder does nothing at all. The useful test injects *exactly* t errors and requires full correction, injects $t+1$ and requires that the original is *not* recovered, and separately counts detected failures versus mis-corrections. Then it breaks the decoder on purpose — forcing the syndrome to zero — and asserts that the first test now fails. A suite that does not contain that last step has never been tested itself.

W2.3 Parameterisation

Table 92. Parameter design rules

Rule	Mechanism	Effect
Expose orthogonal axes only	Design, not syntax	Prevents combinatorial explosion
Document legal combinations	A table in the user guide	Customer cannot guess
Enforce them at elaboration	<code>\$error inside generate; static_assert</code> in C++	Illegal configurations fail to build rather than misbehave
Declare the verified set	A table of tested combinations	Honest scope statement; exhaustive testing is impossible
Safe defaults	Most conservative values	Wrong usage degrades, not breaks

석사 수준 · **Elaboration-time enforcement is the cheapest quality feature available.** An illegal parameter combination that merely produces wrong behaviour will be discovered by the customer, late, and reported as a bug in your IP. The same combination rejected with a clear message at elaboration costs the customer five minutes. The difference in support burden over a product's life is large, and it requires perhaps twenty lines of code.

W3. Automation: Running Verification Around the Clock

A solo developer cannot out-work a verification team, but can out-schedule one: machines run while people sleep. The objective is a loop that, unattended, generates stimulus, compares against the golden model, classifies failures, and leaves a prioritised queue for the morning.

W3.1 The continuous verification loop

Table 93. Loop stages

Stage	Action	Requirement
1. Generate	Random parameters and stimulus from declared legal ranges	Seeds recorded
2. Run model	Reference model produces expected output	Fast and deterministic
3. Run RTL	Open-source simulator (Verilator/Icarus) or vendor tool	Scriptable, headless
4. Compare	Scoreboard with the correct ordering semantics	See H1.4
5. Classify	Group failures by signature	Automated triage
6. Minimise	Shrink a failing case to a minimal reproducer	The single highest-value automation step
7. Report	Ranked queue with reproduction commands	—
8. Coverage	Accumulate and identify gaps	Feeds back into stage 1

석사 수준 · Stage 6 is where automation earns its keep. A randomly generated failing case may be thousands of transactions long; a human cannot read it. Delta-debugging — repeatedly removing parts of the stimulus and re-running to see whether the failure persists — typically reduces such a case to a handful of transactions in a few minutes of machine time. The engineer then arrives to a minimal reproducer rather than a log file. **This is the difference between an overnight run that produces work and one that produces guilt.**

W3.2 Where AI helps and where it must not be trusted

Table 94. Machine assistance in an IP flow

Task	Suitability	Condition
Writing testbench boilerplate	High	Reviewed; it is code like any other
Generating directed tests from a specification table	High	The table is the source of truth, not the model
Triage and clustering of failures	High	Statistical, low risk
Test case minimisation	High	Mechanical; verifiable by re-running
Documentation drafting	Moderate	Every number must be regenerated from data
Suggesting RTL fixes	Moderate	Must be re-verified from scratch
Writing the reference model	Low	It would share the same misreading as generated RTL — independence is lost
Judging whether coverage is sufficient	Low	Requires domain judgement
Deciding that an intermittent failure is benign	None	This is the judgement that must never be delegated

The independence argument constrains automation more than capability does. Verification works because two implementations were derived separately. If the same tool writes the model and the RTL from the same prompt, the two share every misunderstanding and the regression proves only internal consistency. Automation is therefore safest on the *mechanical* parts of the loop — generation, execution, comparison, minimisation, reporting — and least safe on the parts that define what correct means. **Automate the pipeline; keep the oracle human or standards-derived.**

W3.3 Infrastructure a solo practice needs

Table 95. Toolchain

Function	Open-source option	Note
RTL simulation	Verilator (cycle-based), Icarus Verilog (event)	Verilator is fast enough for overnight regressions
Testbench	cocotb (Python) or plain SystemVerilog	cocotb lets the golden model and testbench share a language
Formal	SymbiYosys / Yosys	Sufficient for arbiters, FIFOs, protocol properties
Synthesis (estimation)	Yosys + an open PDK	Relative area/timing trends, not sign-off numbers
Lint	Verilator --lint-only, Verible	—
Coverage	Verilator coverage, simulator built-ins	—
Waveforms	GTKWave, Surfer	—
CI	Any runner	The regression must run on a clean checkout
Documentation	Generated from the same source as the RTL	See W2.3

석사 수준 · **On open-source synthesis results: report trends, not absolutes.** Yosys with an open PDK will not reproduce a commercial tool's area or frequency, and quoting its numbers as a specification invites embarrassment. What it *does* reliably show is *relative* behaviour — that configuration B is 40% larger than A, that removing a pipeline stage raises the critical path by 30%. Those comparisons are legitimate, useful during architecture, and free. **State the tool and PDK next to every number and the report stays honest.**

Volume II — Production Source Code

This volume reproduces **156,966 lines** of source from 43 blocks that are shipping in silicon or sold as commercial IP. Nothing is paraphrased: every line is read from disk and printed with its real line number, so a citation of the form `file:line` anywhere in this book can be followed directly.

Each block is preceded by four notes — **What** it is, **Why** it exists (what fails without it), the **EECS concepts** it rests on, and a **practice note** for engineers who must work with it. An **API dictionary** then lists every module, class, function, parameter, port, macro and typedef that the block declares, with the arguments each one takes, because names alone are ambiguous.

The API dictionary is generated by pattern extraction, not by a full language front end. It is therefore complete enough to navigate by and may miss unusual declarations; the verbatim source that follows each table is authoritative.

AES 골든 모델 (C)

What. FIPS-197 AES 를 573줄 C 로 구현한 것. S-box 표, xtime, 네 라운드 함수, 키 확장, ECB/CBC/CTR 모드가 전부 들어 있다.

Why it exists. 정답을 정의하기 위해서다. RTL 이 맞는지 판단하려면 독립적으로 계산된 기대값이 있어야 한다. 이 파일이 그 기대값을 만든다. 하드웨어 개념(클럭·포트·상태기계)이 하나도 없는 것이 의도된 것이다 — 순수한 의미만 남긴다.

학부 복습 · **EECS concepts.** 유한체 $GF(2^8)$ 산술(대수학) · 블록암호 구조(SPN, 치환-치환망) · 혼돈(confusion)과 확산(diffusion)의 Shannon 원리.

석사 수준 · **Practice note.** 이 코드의 xtime 이 분기 대신 곱셈을 쓰는 이유를 보라 — 실행 시간이 데이터에 의존하지 않게 하려는 것이다. 소프트웨어 암호에서 타이밍 부채널 대책의 가장 기초 형태이고, 모델이라도 이 습관을 지킨다.

3 files • 979 lines

API — aes.c

Table 96. kokke_tiny-AES-c/aes.c — 함수 (23개)

함수	반환형	받는 파라미터
getSBoxValue	static uint8_t	uint8_t num
KeyExpansion	static void	uint8_t* RoundKey, const uint8_t* Key
AES_init_ctx	void	struct AES_ctx* ctx, const uint8_t* key
AES_init_ctx_iv	void	struct AES_ctx* ctx, const uint8_t* key, const uint8_t* iv
AES_ctx_set_iv	void	struct AES_ctx* ctx, const uint8_t* iv
AddRoundKey	static void	uint8_t round, state_t* state, const uint8_t* RoundKey
SubBytes	static void	state_t* state
ShiftRows	static void	state_t* state
xtime	static uint8_t	uint8_t x
MixColumns	static void	state_t* state
Multiply	static uint8_t	uint8_t x, uint8_t y
getSBoxInvert	static uint8_t	uint8_t num
InvMixColumns	static void	state_t* state
InvSubBytes	static void	state_t* state
InvShiftRows	static void	state_t* state
Cipher	static void	state_t* state, const uint8_t* RoundKey
InvCipher	static void	state_t* state, const uint8_t* RoundKey
AES_ECB_encrypt	void	const struct AES_ctx* ctx, uint8_t* buf

함수	반환형	받는 파라미터
AES_ECB_decrypt	void	const struct AES_ctx* ctx, uint8_t* buf
XorWithIv	static void	uint8_t* buf, const uint8_t* lv
AES_CBC_encrypt_buffer	void	struct AES_ctx *ctx, uint8_t* buf, size_t length
AES_CBC_decrypt_buffer	void	struct AES_ctx* ctx, uint8_t* buf, size_t length
AES_CTR_xcrypt_buffer	void	struct AES_ctx* ctx, uint8_t* buf, size_t length

Table 97. kokke_tiny-AES-c/aes.c — #define / `define 매크로 (11개)

매크로	인자	정의
Nb		4
Nk		8
Nr		14
Nk		6
Nr		12
Nk		4 // The number of 32 bit words in a key.
Nr		10 // The number of rounds in AES Cipher.
MULTIPLY_AS_A_FUNCTION		0
getSBoxValue	(num)	(sbox[(num)])
Multiply	(x, y)	\
getSBoxInvert	(num)	(rsbox[(num)])

API — aes.h

Table 98. kokke_tiny-AES-c/aes.h — 클래스/구조체 (1개)

이름	템플릿 인자	종류	상속
AES_ctx		struct	

Table 99. kokke_tiny-AES-c/aes.h — #define / `define 매크로 (12개)

매크로	인자	정의
_AES_H_		#include <stdint.h>
CBC		1
ECB		1
CTR		1
AES128		1
AES_BLOCKLEN		16 // Block length in bytes - AES is 128b block only
AES_KEYLEN		32
AES_keyExpSize		240
AES_KEYLEN		24

매크로	인자	정의
AES_keyExpSize		208
AES_KEYLEN		16 // Key length in bytes
AES_keyExpSize		176

API — test.c

Table 100. kokke_tiny-AES-c/test.c — 함수 (10개)

함수	반환형	받는 파라미터
main	int	void
phex	static void	uint8_t* str
test_encrypt_ecb_verbose	static void	void
test_encrypt_ecb	static int	void
test_decrypt_cbc	static int	void
test_encrypt_cbc	static int	void
test_encrypt_ctr	static int	void
test_decrypt_ctr	static int	void
test_xcrypt_ctr	static int	const char* xcrypt
test_decrypt_ecb	static int	void

Table 101. kokke_tiny-AES-c/test.c — #define / `define 매크로 (3개)

매크로	인자	정의
CBC		1
CTR		1
ECB		1

1. kokke_tiny-AES-c/aes.c (572 lines)

```

1 /*
2
3 This is an implementation of the AES algorithm, specifically
ECB, CTR and CBC mode.
4 Block size can be chosen in aes.h - available choices are
AES128, AES192, AES256.
5
6 The implementation is verified against the test vectors in:
7 National Institute of Standards and Technology Special Pub-
lication 800-38A 2001 ED
8
9 ECB-AES128
10 -----
11
12 plain-text:
13 6bc1bee22e409f96e93d7e117393172a
14 ae2d8a571e03ac9c9eb76fac45af8e51
15 30c81c46a35ce411e5fbc1191a0a52ef
16 f69f2445df4f9b17ad2b417be66c3710
17
18 key:
19 2b7e151628aed2a6abf7158809cf4f3c
20
21 resulting cipher
22 3ad77bb40d7a3660a89ecaf32466ef97
23 f5d3d58503b9699de785895a96fdbaa
24 43b1cd7f598ece23881b00e3ed030688
25 7b0c785e27e8ad3f8223207104725dd4
26
27
28 NOTE: String length must be evenly divisible by 16byte
(str_len % 16 == 0)

```

```

29 You should pad the end of the string with zeros if
this is not the case.
30 For AES192/256 the key size is proportionally larger.
31
32 */
33
34
35 /
*****
36 /* In-
cludes:
*/
37 /
*****
38 #include <string.h> // CBC mode, for memset
39 #include "aes.h"
40
41 /
*****
42 /*
Defines:
*/
43 /
*****
44 // The number of columns comprising a state in AES. This is a
constant in AES. Value=4
45 #define Nb 4
46
47 #if defined(AES256) && (AES256 == 1)
48 #define Nk 8
49 #define Nr 14

```

```

50 #elif defined(AES192) && (AES192 == 1)
51     #define Nk 6
52     #define Nr 12
53 #else
54     #define Nk 4          // The number of 32 bit words in a
key.
55     #define Nr 10        // The number of rounds in AES Cipher.
56 #endif
57
58 // jcallan@github points out that declaring Multiply as a
function
59 // reduces code size considerably with the Keil ARM compiler.
60 // See this link for more information: https://github.com/
kokke/tiny-AES-C/pull/3
61 #ifndef MULTIPLY_AS_A_FUNCTION
62     #define MULTIPLY_AS_A_FUNCTION 0
63 #endif
64
65
66
67
68 /
*****
69 /* Private vari-
ables:
70 /
*****
71 // state - array holding the intermediate results during de-
cryption.
72 typedef uint8_t state_t[4][4];
73
74
75
76 // The lookup-tables are marked const so they can be placed in
read-only storage instead of RAM
77 // The numbers below can be computed dynamically trading ROM
for RAM -
78 // This can be useful in (embedded) bootloader applications,
where ROM is often limited.
79 static const uint8_t sbox[256] = {
80 //0    1    2    3    4    5    6    7    8    9
A      B      C      D      E      F
81 0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01,
0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
82 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4,
0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
83 0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5,
0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
84 0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12,
0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
85 0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b,
0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
86 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb,
0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
87 0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9,
0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
88 0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6,
0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
89 0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7,
0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
90 0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee,
0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
91 0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3,
0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
92 0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56,
0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
93 0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd,
0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
94 0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35,
0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
95 0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e,
0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
96 0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99,
0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16 };
97
98 #if (defined(CBC) && CBC == 1) || (defined(ECB) && ECB == 1)
99 static const uint8_t rsbox[256] = {
100 0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40,
0xa3, 0x9e, 0x81, 0xf3, 0xd7, 0xfb,
101 0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e,
0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb,
102 0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c,
0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e,
103 0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b,
0xa2, 0x49, 0x6d, 0x8b, 0xd1, 0x25,
104 0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4,
0x5c, 0xcc, 0x5d, 0x65, 0xb6, 0x92,
105 0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15,
0x46, 0x57, 0xa7, 0x8d, 0x9d, 0x84,
106 0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4,
0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06,
107 0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf,
0xbd, 0x03, 0x01, 0x13, 0x8a, 0x6b,
108 0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2,
0xcf, 0xce, 0xf0, 0xb4, 0xe6, 0x73,
109 0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9,
0x37, 0xe8, 0x1c, 0x75, 0xdf, 0x6e,

```

```

110 0x47, 0xf1, 0x1a, 0x71, 0xd1, 0x29, 0xc5, 0x89, 0x6f, 0xb7,
0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b,
111 0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb,
0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4,
112 0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12,
0x10, 0x59, 0x27, 0x80, 0xec, 0x5f,
113 0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5,
0x7a, 0x9f, 0x93, 0xc9, 0x9c, 0xef,
114 0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb,
0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61,
115 0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69,
0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d };
116 #endif
117
118 // The round constant word array, Rcon[i], contains the values
given by
119 // x to the power (i-1) being powers of x (x is denoted as
{02}) in the field GF(2^8)
120 static const uint8_t Rcon[11] = {
121 0x8d, 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1b,
0x36 };
122
123 /*
124 * Jordan Goulder points out in PR #12 (https://github.com/
kokke/tiny-AES-C/pull/12),
125 * that you can remove most of the elements in the Rcon array,
because they are unused.
126 *
127 * From Wikipedia's article on the Rijndael key schedule @ ht-
tps://en.wikipedia.org/wiki/Rijndael_key_schedule#Rcon
128 *
129 * "Only the first some of these constants are actually used -
up to rcon[10] for AES-128 (as 11 round keys are needed),
130 * up to rcon[8] for AES-192, up to rcon[7] for AES-256.
rcon[0] is not used in AES algorithm."
131 */
132
133
134 /
*****
135 /* Private func-
tions:
136 /
*****
137 /*
138 static uint8_t getSBoxValue(uint8_t num)
139 {
140     return sbox[num];
141 }
142 */
143 #define getSBoxValue(num) (sbox[(num)])
144
145 // This function produces Nb(Nr+1) round keys. The round keys
are used in each round to decrypt the states.
146 static void KeyExpansion(uint8_t* RoundKey, const uint8_t*
Key)
147 {
148     unsigned i, j, k;
149     uint8_t tempa[4]; // Used for the column/row operations
150
151     // The first round key is the key itself.
152     for (i = 0; i < Nk; ++i)
153     {
154         RoundKey[(i * 4) + 0] = Key[(i * 4) + 0];
155         RoundKey[(i * 4) + 1] = Key[(i * 4) + 1];
156         RoundKey[(i * 4) + 2] = Key[(i * 4) + 2];
157         RoundKey[(i * 4) + 3] = Key[(i * 4) + 3];
158     }
159
160     // All other round keys are found from the previous round
keys.
161     for (i = Nk; i < Nb * (Nr + 1); ++i)
162     {
163         {
164             k = (i - 1) * 4;
165             tempa[0] = RoundKey[k + 0];
166             tempa[1] = RoundKey[k + 1];
167             tempa[2] = RoundKey[k + 2];
168             tempa[3] = RoundKey[k + 3];
169         }
170
171         if (i % Nk == 0)
172         {
173             // This function shifts the 4 bytes in a word to the
left once.
174             // [a0,a1,a2,a3] becomes [a1,a2,a3,a0]
175
176             // Function RotWord()
177             {
178                 const uint8_t u8tmp = tempa[0];
179                 tempa[0] = tempa[1];
180                 tempa[1] = tempa[2];
181                 tempa[2] = tempa[3];
182                 tempa[3] = u8tmp;
183             }
184

```

```

185
186 // SubWord() is a function that takes a four-byte input
word and
187 // applies the S-box to each of the four bytes to pro-
duce an output word.
188
189 // Function Subword()
190 {
191     tempa[0] = getSBoxValue(tempa[0]);
192     tempa[1] = getSBoxValue(tempa[1]);
193     tempa[2] = getSBoxValue(tempa[2]);
194     tempa[3] = getSBoxValue(tempa[3]);
195 }
196
197 tempa[0] = tempa[0] ^ Rcon[i/Nk];
198 }
199 #if defined(AES256) && (AES256 == 1)
200 if (i % Nk == 4)
201 {
202     // Function Subword()
203     {
204         tempa[0] = getSBoxValue(tempa[0]);
205         tempa[1] = getSBoxValue(tempa[1]);
206         tempa[2] = getSBoxValue(tempa[2]);
207         tempa[3] = getSBoxValue(tempa[3]);
208     }
209 }
210 #endif
211 j = i * 4; k=(i - Nk) * 4;
212 RoundKey[j + 0] = RoundKey[k + 0] ^ tempa[0];
213 RoundKey[j + 1] = RoundKey[k + 1] ^ tempa[1];
214 RoundKey[j + 2] = RoundKey[k + 2] ^ tempa[2];
215 RoundKey[j + 3] = RoundKey[k + 3] ^ tempa[3];
216 }
217 }
218
219 void AES_init_ctx(struct AES_ctx* ctx, const uint8_t* key)
220 {
221     KeyExpansion(ctx->RoundKey, key);
222 }
223 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
1))
224 void AES_init_ctx_iv(struct AES_ctx* ctx, const uint8_t* key,
const uint8_t* iv)
225 {
226     KeyExpansion(ctx->RoundKey, key);
227     memcpy(ctx->iv, iv, AES_BLOCKLEN);
228 }
229 void AES_ctx_set_iv(struct AES_ctx* ctx, const uint8_t* iv)
230 {
231     memcpy(ctx->iv, iv, AES_BLOCKLEN);
232 }
233 #endif
234
235 // This function adds the round key to state.
236 // The round key is added to the state by an XOR function.
237 static void AddRoundKey(uint8_t round, state_t* state, const
uint8_t* RoundKey)
238 {
239     uint8_t i, j;
240     for (i = 0; i < 4; ++i)
241     {
242         for (j = 0; j < 4; ++j)
243         {
244             (*state)[i][j] ^= RoundKey[(round * Nb * 4) + (i * Nb) +
j];
245         }
246     }
247 }
248
249 // The SubBytes Function Substitutes the values in the
250 // state matrix with values in an S-box.
251 static void SubBytes(state_t* state)
252 {
253     uint8_t i, j;
254     for (i = 0; i < 4; ++i)
255     {
256         for (j = 0; j < 4; ++j)
257         {
258             (*state)[j][i] = getSBoxValue((*state)[j][i]);
259         }
260     }
261 }
262
263 // The ShiftRows() function shifts the rows in the state to
the left.
264 // Each row is shifted with different offset.
265 // Offset = Row number. So the first row is not shifted.
266 static void ShiftRows(state_t* state)
267 {
268     uint8_t temp;
269
270     // Rotate first row 1 columns to left
271     temp = (*state)[0][1];
272     (*state)[0][1] = (*state)[1][1];
273     (*state)[1][1] = (*state)[2][1];
274     (*state)[2][1] = (*state)[3][1];
275     (*state)[3][1] = temp;
276
277     // Rotate second row 2 columns to left

```

```

278     temp = (*state)[0][2];
279     (*state)[0][2] = (*state)[2][2];
280     (*state)[2][2] = temp;
281
282     temp = (*state)[1][2];
283     (*state)[1][2] = (*state)[3][2];
284     (*state)[3][2] = temp;
285
286     // Rotate third row 3 columns to left
287     temp = (*state)[0][3];
288     (*state)[0][3] = (*state)[3][3];
289     (*state)[3][3] = (*state)[2][3];
290     (*state)[2][3] = (*state)[1][3];
291     (*state)[1][3] = temp;
292 }
293
294 static uint8_t xtime(uint8_t x)
295 {
296     return ((x<<1) ^ (((x>>7) & 1) * 0x1b));
297 }
298
299 // MixColumns function mixes the columns of the state matrix
300 static void MixColumns(state_t* state)
301 {
302     uint8_t i;
303     uint8_t Tmp, Tm, t;
304     for (i = 0; i < 4; ++i)
305     {
306         t = (*state)[i][0];
307         Tmp = (*state)[i][0] ^ (*state)[i][1] ^ (*state)[i][2] ^
(*state)[i][3];
308         Tm = (*state)[i][0] ^ (*state)[i][1]; Tm = xtime(Tm);
309         (*state)[i][0] ^= Tm ^ Tmp;
310         Tm = (*state)[i][1] ^ (*state)[i][2]; Tm = xtime(Tm);
311         (*state)[i][1] ^= Tm ^ Tmp;
312         Tm = (*state)[i][2] ^ (*state)[i][3]; Tm = xtime(Tm);
313         (*state)[i][2] ^= Tm ^ Tmp;
314         Tm = (*state)[i][3] ^ t; Tm = xtime(Tm);
315         (*state)[i][3] ^= Tm ^ Tmp;
316     }
317 }
318
319 // Multiply is used to multiply numbers in the field GF(2^8)
320 // Note: The last call to xtime() is unneeded, but often ends
up generating a smaller binary
321 // The compiler seems to be able to vectorize the opera-
tion better this way.
322 // See https://github.com/kokke/tiny-AES-c/pull/34
323 #if MULTIPLY_AS_A_FUNCTION
324 static uint8_t Multiply(uint8_t x, uint8_t y)
325 {
326     return (((y & 1) * x) ^
((y>>1 & 1) * xtime(x)) ^
((y>>2 & 1) * xtime(xtime(x))) ^
((y>>3 & 1) * xtime(xtime(xtime(x)))) ^
((y>>4 & 1) * xtime(xtime(xtime(xtime(x)))))); /* this
last call to xtime() can be omitted */
327 }
328 #else
329 #define Multiply(x, y) \
330 ( ((y & 1) * x) ^ \
331 ((y>>1 & 1) * xtime(x)) ^ \
332 ((y>>2 & 1) * xtime(xtime(x))) ^ \
333 ((y>>3 & 1) * xtime(xtime(xtime(x)))) ^ \
334 ((y>>4 & 1) * xtime(xtime(xtime(xtime(x)))))) \
335 )
336 #endif
337
338 #if (defined(CBC) && (CBC == 1)) || (defined(ECB) && (ECB == 1)
339 /*
340 static uint8_t getSBoxInvert(uint8_t num)
341 {
342     return rsbox[num];
343 }
344 */
345 #define getSBoxInvert(num) (rsbox[(num)])
346
347 // MixColumns function mixes the columns of the state matrix.
348 // The method used to multiply may be difficult to understand
for the inexperienced.
349 // Please use the references to gain more information.
350 static void InvMixColumns(state_t* state)
351 {
352     int i;
353     uint8_t a, b, c, d;
354     for (i = 0; i < 4; ++i)
355     {
356         a = (*state)[i][0];
357         b = (*state)[i][1];
358         c = (*state)[i][2];
359         d = (*state)[i][3];
360
361         (*state)[i][0] = Multiply(a, 0x0e) ^ Multiply(b, 0x0b) ^
Multiply(c, 0x0d) ^ Multiply(d, 0x09);
362         (*state)[i][1] = Multiply(a, 0x09) ^ Multiply(b, 0x0e) ^
Multiply(c, 0x0b) ^ Multiply(d, 0x0d);
363         (*state)[i][2] = Multiply(a, 0x0d) ^ Multiply(b, 0x09) ^
Multiply(c, 0x0e) ^ Multiply(d, 0x0b);
364         (*state)[i][3] = Multiply(a, 0x0b) ^ Multiply(b, 0x0d) ^
Multiply(c, 0x09) ^ Multiply(d, 0x0e);
365     }

```

```

366 }
367
368 // The SubBytes Function Substitutes the values in the
369 // state matrix with values in an S-box.
370 static void InvSubBytes(state_t* state)
371 {
372     uint8_t i, j;
373     for (i = 0; i < 4; ++i)
374     {
375         for (j = 0; j < 4; ++j)
376         {
377             (*state)[j][i] = getSBoxInvert((*state)[j][i]);
378         }
379     }
380 }
381
382 static void InvShiftRows(state_t* state)
383 {
384     uint8_t temp;
385
386     // Rotate first row 1 columns to right
387     temp = (*state)[3][1];
388     (*state)[3][1] = (*state)[2][1];
389     (*state)[2][1] = (*state)[1][1];
390     (*state)[1][1] = (*state)[0][1];
391     (*state)[0][1] = temp;
392
393     // Rotate second row 2 columns to right
394     temp = (*state)[0][2];
395     (*state)[0][2] = (*state)[2][2];
396     (*state)[2][2] = temp;
397
398     temp = (*state)[1][2];
399     (*state)[1][2] = (*state)[3][2];
400     (*state)[3][2] = temp;
401
402     // Rotate third row 3 columns to right
403     temp = (*state)[0][3];
404     (*state)[0][3] = (*state)[1][3];
405     (*state)[1][3] = (*state)[2][3];
406     (*state)[2][3] = (*state)[3][3];
407     (*state)[3][3] = temp;
408 }
409
410 #endif // #if (defined(CBC) && CBC == 1) || (defined(ECB) &&
411 ECB == 1)
412 // Cipher is the main function that encrypts the PlainText.
413 static void Cipher(state_t* state, const uint8_t* RoundKey)
414 {
415     uint8_t round = 0;
416
417     // Add the First round key to the state before starting the
418     rounds.
419     AddRoundKey(0, state, RoundKey);
420
421     // There will be Nr rounds.
422     // The first Nr-1 rounds are identical.
423     // These Nr rounds are executed in the loop below.
424     // Last one without MixColumns()
425     for (round = 1; ; ++round)
426     {
427         SubBytes(state);
428         ShiftRows(state);
429         if (round == Nr) {
430             break;
431         }
432         MixColumns(state);
433         AddRoundKey(round, state, RoundKey);
434     }
435     // Add round key to last round
436     AddRoundKey(Nr, state, RoundKey);
437 }
438
439 #if (defined(CBC) && CBC == 1) || (defined(ECB) && ECB == 1)
440 static void InvCipher(state_t* state, const uint8_t* RoundKey)
441 {
442     uint8_t round = 0;
443
444     // Add the First round key to the state before starting the
445     rounds.
446     AddRoundKey(Nr, state, RoundKey);
447
448     // There will be Nr rounds.
449     // The first Nr-1 rounds are identical.
450     // These Nr rounds are executed in the loop below.
451     // Last one without InvMixColumn()
452     for (round = (Nr - 1); ; --round)
453     {
454         InvShiftRows(state);
455         InvSubBytes(state);
456         AddRoundKey(round, state, RoundKey);
457         if (round == 0) {
458             break;
459         }
460         InvMixColumns(state);
461     }
462 }
463 #endif // #if (defined(CBC) && CBC == 1) || (defined(ECB) &&
464 ECB == 1)

```

```

463 /
464 /
465 /* Public func-
466 tions: */
467 /
468 /
469 /
470 #if defined(ECB) && (ECB == 1)
471 void AES_ECB_encrypt(const struct AES_ctx* ctx, uint8_t* buf)
472 {
473     // The next function call encrypts the PlainText with the
474     Key using AES algorithm.
475     Cipher((state_t*)buf, ctx->RoundKey);
476 }
477 void AES_ECB_decrypt(const struct AES_ctx* ctx, uint8_t* buf)
478 {
479     // The next function call decrypts the PlainText with the
480     Key using AES algorithm.
481     InvCipher((state_t*)buf, ctx->RoundKey);
482 }
483 #endif // #if defined(ECB) && (ECB == 1)
484
485 #if defined(CBC) && (CBC == 1)
486 static void XorWithIv(uint8_t* buf, const uint8_t* Iv)
487 {
488     uint8_t i;
489     for (i = 0; i < AES_BLOCKLEN; ++i) // The block in AES is
490     always 128bit no matter the key size
491     {
492         buf[i] ^= Iv[i];
493     }
494 }
495
496 void AES_CBC_encrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
497 size_t length)
498 {
499     size_t i;
500     uint8_t* Iv = ctx->Iv;
501     for (i = 0; i < length; i += AES_BLOCKLEN)
502     {
503         XorWithIv(buf, Iv);
504         Cipher((state_t*)buf, ctx->RoundKey);
505         Iv = buf;
506         buf += AES_BLOCKLEN;
507     }
508     /* store Iv in ctx for next call */
509     memcpy(ctx->Iv, Iv, AES_BLOCKLEN);
510 }
511
512 void AES_CBC_decrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
513 size_t length)
514 {
515     size_t i;
516     uint8_t storeNextIv[AES_BLOCKLEN];
517     for (i = 0; i < length; i += AES_BLOCKLEN)
518     {
519         memcpy(storeNextIv, buf, AES_BLOCKLEN);
520         InvCipher((state_t*)buf, ctx->RoundKey);
521         XorWithIv(buf, ctx->Iv);
522         memcpy(ctx->Iv, storeNextIv, AES_BLOCKLEN);
523         buf += AES_BLOCKLEN;
524     }
525 }
526
527 #endif // #if defined(CBC) && (CBC == 1)
528
529 #if defined(CTR) && (CTR == 1)
530 /* Symmetrical operation: same function for encrypting as for
531 decrypting. Note any IV/nonce should never be reused with the same
532 key */
533 void AES_CTR_xcrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
534 size_t length)
535 {
536     uint8_t buffer[AES_BLOCKLEN];
537
538     size_t i;
539     int bi;
540     for (i = 0, bi = AES_BLOCKLEN; i < length; ++i, ++bi)
541     {
542         if (bi == AES_BLOCKLEN) /* we need to regen xor compliment
543         in buffer */

```

```

547 {
548     memcpy(buffer, ctx->Iv, AES_BLOCKLEN);
549     Cipher((state_t*)buffer, ctx->RoundKey);
550
551     /* Increment Iv and handle overflow */
552     for (bi = (AES_BLOCKLEN - 1); bi >= 0; --bi)
553     {
554         /* inc will overflow */
555         if (ctx->Iv[bi] == 255)
556         {
557             ctx->Iv[bi] = 0;
558             continue;
559         }
560         ctx->Iv[bi] += 1;
561         break;
562     }
563     bi = 0;
564 }
565
566 buf[i] = (buf[i] ^ buffer[bi]);
567 }
568 }
569 }
570
571 #endif // #if defined(CTR) && (CTR == 1)
572

```

2. kokke_tiny-AES-c/aes.h (91 lines)

```

1 #ifndef _AES_H
2 #define _AES_H
3
4 #include <stdint.h>
5 #include <stddef.h>
6
7 // #define the macros below to 1/0 to enable/disable the mode
8 // of operation.
9 // CBC enables AES encryption in CBC-mode of operation.
10 // CTR enables encryption in counter-mode.
11 // ECB enables the basic ECB 16-byte block algorithm. All can
12 // be enabled simultaneously.
13 // The #ifndef-guard allows it to be configured before #in-
14 // clude'ing or at compile time.
15 #ifndef CBC
16 #define CBC 1
17 #endif
18
19 #ifndef ECB
20 #define ECB 1
21 #endif
22
23 #ifndef CTR
24 #define CTR 1
25 #endif
26
27 #define AES128 1
28 // #define AES192 1
29 // #define AES256 1
30
31 #define AES_BLOCKLEN 16 // Block length in bytes - AES is 128b
32 // block only
33
34 #if defined(AES256) && (AES256 == 1)
35 #define AES_KEYLEN 32
36 #define AES_keyExpSize 240
37 #elif defined(AES192) && (AES192 == 1)
38 #define AES_KEYLEN 24
39 #define AES_keyExpSize 208
40 #else
41 #define AES_KEYLEN 16 // Key length in bytes
42 #define AES_keyExpSize 176
43 #endif
44
45 struct AES_ctx
46 {
47     uint8_t RoundKey[AES_keyExpSize];
48 }
49
50 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
51 1))
52     uint8_t Iv[AES_BLOCKLEN];
53 #endif
54
55 void AES_init_ctx(struct AES_ctx* ctx, const uint8_t* key);
56
57 #if (defined(CBC) && (CBC == 1)) || (defined(CTR) && (CTR ==
58 1))
59     void AES_init_ctx_iv(struct AES_ctx* ctx, const uint8_t* key,
60     const uint8_t* iv);
61     void AES_ctx_set_iv(struct AES_ctx* ctx, const uint8_t* iv);
62 #endif
63
64 #if defined(ECB) && (ECB == 1)
65     // buffer size is exactly AES_BLOCKLEN bytes;
66     // you need only AES_init_ctx as IV is not used in ECB
67     // NB: ECB is considered insecure for most uses
68     void AES_ECB_encrypt(const struct AES_ctx* ctx, uint8_t* buf);
69     void AES_ECB_decrypt(const struct AES_ctx* ctx, uint8_t* buf);

```

```

64
65 #endif // #if defined(ECB) && (ECB == 1)
66
67
68 #if defined(CBC) && (CBC == 1)
69     // buffer size MUST be mutile of AES_BLOCKLEN;
70     // Suggest https://en.wikipedia.org/wiki/Padding_(cryptography)#PKCS7 for padding scheme
71     // NOTES: you need to set IV in ctx via AES_init_ctx_iv() or
72     AES_ctx_set_iv()
73     // no IV should ever be reused with the same key
74
75 void AES_CBC_encrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
76 size_t length);
77
78 void AES_CBC_decrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
79 size_t length);
80
81 #endif // #if defined(CBC) && (CBC == 1)
82
83 #if defined(CTR) && (CTR == 1)
84
85 // Same function for encrypting as for decrypting.
86 // IV is incremented for every block, and used after encryp-
87 // tion as XOR-compliment for output
88 // Suggesting https://en.wikipedia.org/wiki/Padding_(cryptography)#PKCS7 for padding scheme
89 // NOTES: you need to set IV in ctx with AES_init_ctx_iv() or
90 AES_ctx_set_iv()
91 // no IV should ever be reused with the same key
92 void AES_CTR_xcrypt_buffer(struct AES_ctx* ctx, uint8_t* buf,
93 size_t length);
94
95 #endif // #if defined(CTR) && (CTR == 1)
96
97 #endif // _AES_H

```

3. kokke_tiny-AES-c/test.c (316 lines)

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <stdint.h>
4
5 // Enable ECB, CTR and CBC mode. Note this can be done before
6 // including aes.h or at compile-time.
7 // E.g. with GCC by using the -D flag: gcc -c aes.c -DCBC=0 -
8 DCTR=1 -DECBC=1
9
10 #define CBC 1
11 #define CTR 1
12 #define ECB 1
13
14 #include "aes.h"
15
16 static void phex(uint8_t* str);
17 static int test_encrypt_cbc(void);
18 static int test_decrypt_cbc(void);
19 static int test_encrypt_ctr(void);
20 static int test_decrypt_ctr(void);
21 static int test_encrypt_ecb(void);
22 static int test_decrypt_ecb(void);
23 static void test_encrypt_ecb_verbose(void);
24
25 int main(void)
26 {
27     int exit;
28
29     #if defined(AES256)
30     printf("\nTesting AES256\n\n");
31     #elif defined(AES192)
32     printf("\nTesting AES192\n\n");
33     #elif defined(AES128)
34     printf("\nTesting AES128\n\n");
35     #else
36     printf("You need to specify a symbol between AES128,
37 AES192 or AES256. Exiting");
38     return 0;
39 #endif
40
41     exit = test_encrypt_cbc() + test_decrypt_cbc() +
42     test_encrypt_ctr() + test_decrypt_ctr() +
43     test_encrypt_ecb() + test_decrypt_ecb();
44     test_encrypt_ecb_verbose();
45     return exit;
46 }
47
48 // prints string as hex
49 static void phex(uint8_t* str)
50 {
51
52     #if defined(AES256)
53     uint8_t len = 32;
54     #elif defined(AES192)
55     uint8_t len = 24;
56     #elif defined(AES128)

```



```

57     uint8_t len = 16;
58 #endif
59
60     unsigned char i;
61     for (i = 0; i < len; ++i)
62         printf("%.2x", str[i]);
63     printf("\n");
64 }
65
66 static void test_encrypt_ecb_verbose(void)
67 {
68     // Example of more verbose verification
69
70     uint8_t i;
71
72     // 128bit key
73
74     uint8_t key[16] = { (uint8_t) 0x2b, (uint8_t) 0x7e,
75 (uint8_t) 0x15, (uint8_t) 0x16, (uint8_t) 0x28, (uint8_t) 0xae,
76 (uint8_t) 0xd2, (uint8_t) 0xa6, (uint8_t) 0xab, (uint8_t) 0xf7,
77 (uint8_t) 0x15, (uint8_t) 0x88, (uint8_t) 0x09, (uint8_t) 0xcf,
78 (uint8_t) 0x4f, (uint8_t) 0x3c };
79 // 512bit text
80
81 uint8_t plain_text[64] = { (uint8_t) 0x6b, (uint8_t) 0xc1,
82 (uint8_t) 0xbe, (uint8_t) 0xe2, (uint8_t) 0x2e, (uint8_t) 0x40,
83 (uint8_t) 0x9f, (uint8_t) 0x96, (uint8_t) 0xe9, (uint8_t) 0x3d,
84 (uint8_t) 0x7e, (uint8_t) 0x11, (uint8_t) 0x73, (uint8_t) 0x93,
85 (uint8_t) 0x17, (uint8_t) 0x2a,
86
87 (uint8_t) 0xae, (uint8_t) 0x2d,
88 (uint8_t) 0x8a, (uint8_t) 0x57, (uint8_t) 0x1e, (uint8_t) 0x03,
89 (uint8_t) 0xac, (uint8_t) 0x9c, (uint8_t) 0x9e, (uint8_t) 0xb7,
90 (uint8_t) 0x6f, (uint8_t) 0xac, (uint8_t) 0x45, (uint8_t) 0xaf,
91 (uint8_t) 0x8e, (uint8_t) 0x51,
92
93 (uint8_t) 0x30, (uint8_t) 0xc8,
94 (uint8_t) 0x1c, (uint8_t) 0x46, (uint8_t) 0xa3, (uint8_t) 0x5c,
95 (uint8_t) 0xe4, (uint8_t) 0x11, (uint8_t) 0xe5, (uint8_t) 0xfb,
96 (uint8_t) 0xc1, (uint8_t) 0x19, (uint8_t) 0x1a, (uint8_t) 0x0a,
97 (uint8_t) 0x52, (uint8_t) 0xef,
98
99 (uint8_t) 0xf6, (uint8_t) 0x9f,
100 (uint8_t) 0x24, (uint8_t) 0x45, (uint8_t) 0xdf, (uint8_t) 0x4f,
101 (uint8_t) 0x9b, (uint8_t) 0x17, (uint8_t) 0xad, (uint8_t) 0x2b,
102 (uint8_t) 0x41, (uint8_t) 0x7b, (uint8_t) 0xe6, (uint8_t) 0x6c,
103 (uint8_t) 0x37, (uint8_t) 0x10 };
104
105 // print text to encrypt, key and IV
106 printf("ECB encrypt verbose:\n\n");
107 printf("plain text:\n");
108 for (i = (uint8_t) 0; i < (uint8_t) 4; ++i)
109 {
110     phex(plain_text + i * (uint8_t) 16);
111 }
112 printf("\n");
113
114 printf("key:\n");
115 phex(key);
116 printf("\n");
117
118 // print the resulting cipher as 4 x 16 byte strings
119 printf("ciphertext:\n");
120
121 struct AES_ctx ctx;
122 AES_init_ctx(&ctx, key);
123 for (i = 0; i < 4; ++i)
124 {
125     AES_ECB_encrypt(&ctx, plain_text + (i * 16));
126     phex(plain_text + (i * 16));
127 }
128 printf("\n");
129 }
130
131 static int test_encrypt_ecb(void)
132 {
133 #if defined(AES256)
134     uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
135 0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
136 0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
137 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
138     uint8_t out[] = { 0xf3, 0xee, 0xd1, 0xbd, 0xb5, 0xd2,
139 0xa0, 0x3c, 0x06, 0x4b, 0x5a, 0x7e, 0x3d, 0xb1, 0x81, 0xf8 };
140 #elif defined(AES192)
141     uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
142 0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5,
143 0x62, 0xf8, 0xea, 0xd2, 0x52, 0x2c,
144 0x6b, 0x7b };
145     uint8_t out[] = { 0xbd, 0x33, 0x4f, 0x1d, 0x6e, 0x45,
146 0xf2, 0x5f, 0xf7, 0x12, 0xa2, 0x14, 0x57, 0x1f, 0xa5, 0xcc };
147 #elif defined(AES128)
148     uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
149 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
150     uint8_t out[] = { 0x3a, 0xd7, 0x7b, 0xb4, 0x0d, 0x7a,
151 0x36, 0x60, 0xa8, 0x9e, 0xca, 0xf3, 0x24, 0x66, 0xef, 0x97 };
152 #endif
153
154     uint8_t in[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
155 0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a };

```

```

124     struct AES_ctx ctx;
125
126     AES_init_ctx(&ctx, key);
127     AES_ECB_encrypt(&ctx, in);
128
129     printf("ECB encrypt: ");
130
131     if (0 == memcmp((char*) out, (char*) in, 16)) {
132         printf("SUCCESS!\n");
133         return(0);
134     } else {
135         printf("FAILURE!\n");
136         return(1);
137     }
138 }
139
140 static int test_decrypt_cbc(void)
141 {
142 #if defined(AES256)
143     uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
144 0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
145 0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
146 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
147     uint8_t in[] = { 0xf5, 0x8c, 0x4c, 0x04, 0xd6, 0xe5,
148 0xf1, 0xba, 0x77, 0x9e, 0xab, 0xf6, 0x5f, 0x7b, 0xfb, 0xd6,
149 0x9c, 0xfc, 0x4e, 0x96, 0x7e, 0xdb,
150 0x80, 0x8d, 0x67, 0x9f, 0x77, 0x7b, 0xc6, 0x70, 0x2c, 0x7d,
151 0x39, 0xf2, 0x33, 0x69, 0xa9, 0xd9,
152 0xba, 0xcf, 0xa5, 0x30, 0xe2, 0x63, 0x04, 0x23, 0x14, 0x61,
153 0xb2, 0xeb, 0x05, 0xe2, 0xc3, 0x9b,
154 0xe9, 0xfc, 0xda, 0x6c, 0x19, 0x07, 0x8c, 0x6a, 0x9d, 0x1b };
155 #elif defined(AES192)
156     uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
157 0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5, 0x62,
158 0xf8, 0xea, 0xd2, 0x52, 0x2c, 0x6b, 0x7b };
159     uint8_t in[] = { 0x4f, 0x02, 0x1d, 0xb2, 0x43, 0xbc,
160 0x63, 0x3d, 0x71, 0x78, 0x18, 0x3a, 0x9f, 0xa0, 0x71, 0xe8,
161 0xb4, 0xd9, 0xad, 0xa9, 0xad, 0x7d,
162 0xed, 0xf4, 0xe5, 0xe7, 0x38, 0x76, 0x3f, 0x69, 0x14, 0x5a,
163 0x57, 0x1b, 0x24, 0x20, 0x12, 0xfb,
164 0x7a, 0xe0, 0x7f, 0xa9, 0xba, 0xac, 0x3d, 0xf1, 0x02, 0xe0,
165 0x08, 0xb0, 0xe2, 0x79, 0x88, 0x59,
166 0x88, 0x81, 0xd9, 0x20, 0xa9, 0xe6, 0x4f, 0x56, 0x15, 0xcd };
167 #elif defined(AES128)
168     uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
169 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
170     uint8_t in[] = { 0x76, 0x49, 0xab, 0xac, 0x81, 0x19,
171 0xb2, 0x46, 0xce, 0xe9, 0x8e, 0x9b, 0x12, 0xe9, 0x19, 0x7d,
172 0x50, 0x86, 0xcb, 0xb9, 0x50, 0x72,
173 0x19, 0xee, 0x95, 0xdb, 0x11, 0x3a, 0x91, 0x76, 0x78, 0xb2,
174 0x73, 0xbe, 0xd6, 0xb8, 0xe3, 0xc1,
175 0x74, 0x3b, 0x71, 0x16, 0xe6, 0x9e, 0x22, 0x22, 0x95, 0x16,
176 0x3f, 0xf1, 0xca, 0xa1, 0x68, 0x1f,
177 0xac, 0x09, 0x12, 0x0e, 0xca, 0x30, 0x75, 0x86, 0xe1, 0xa7 };
178 #endif
179     uint8_t iv[] = { 0x00, 0x01, 0x02, 0x03, 0x04, 0x05,
180 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f };
181     uint8_t out[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
182 0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a,
183 0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03,
184 0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51,
185 0x30, 0xc8, 0x10, 0x1c, 0x46, 0xa3, 0x5c,
186 0xe4, 0x11, 0xe5, 0xfb, 0xc1, 0x19, 0x1a, 0x0a, 0x52, 0xef,
187 0xf6, 0x9f, 0x24, 0x45, 0xdf, 0x4f,
188 0x9b, 0x17, 0xad, 0x2b, 0x41, 0x7b, 0xe6, 0x6c, 0x37, 0x10 };
189 // uint8_t buffer[64];
190 struct AES_ctx ctx;
191
192 AES_init_ctx_iv(&ctx, key, iv);
193 AES_CBC_decrypt_buffer(&ctx, in, 64);
194
195     printf("CBC decrypt: ");
196
197     if (0 == memcmp((char*) out, (char*) in, 64)) {
198         printf("SUCCESS!\n");
199         return(0);
200     } else {
201         printf("FAILURE!\n");
202         return(1);
203     }
204 }
205
206 static int test_encrypt_cbc(void)
207 {
208 #if defined(AES256)
209     uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
210 0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
211 0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
212 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
213     uint8_t out[] = { 0xf5, 0x8c, 0x4c, 0x04, 0xd6, 0xe5,
214 0xf1, 0xba, 0x77, 0x9e, 0xab, 0xf6, 0x5f, 0x7b, 0xfb, 0xd6,
215 0x9c, 0xfc, 0x4e, 0x96, 0x7e, 0xdb,
216 0x80, 0x8d, 0x67, 0x9f, 0x77, 0x7b, 0xc6, 0x70, 0x2c, 0x7d,
217 0x39, 0xf2, 0x33, 0x69, 0xa9, 0xd9,
218 0xba, 0xcf, 0xa5, 0x30, 0xe2, 0x63, 0x04, 0x23, 0x14, 0x61,
219 0xb2, 0xeb, 0x05, 0xe2, 0xc3, 0x9b,
220 0xe9, 0xfc, 0xda, 0x6c, 0x19, 0x07, 0x8c, 0x6a, 0x9d, 0x1b };
221 #elif defined(AES192)
222     uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
223 0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5, 0x62,

```



```

0xf8, 0xea, 0xd2, 0x52, 0x2c, 0x6b, 0x7b };
196     uint8_t out[] = { 0x4f, 0x02, 0x1d, 0xb2, 0x43, 0xbc,
0x63, 0x3d, 0x71, 0x78, 0x18, 0x3a, 0x9f, 0xa0, 0x71, 0xe8,
197     0xb4, 0xd9, 0xad, 0xa9, 0xad, 0x7d,
0xed, 0xf4, 0xe5, 0xe7, 0x38, 0x76, 0x3f, 0x69, 0x14, 0x5a,
198     0x57, 0x1b, 0x24, 0x20, 0x12, 0xfb,
0x7a, 0xe0, 0x7f, 0xa9, 0xba, 0xac, 0x3d, 0xf1, 0x02, 0xe0,
199     0x08, 0xb0, 0xe2, 0x79, 0x88, 0x59,
0x88, 0x81, 0xd9, 0x20, 0xa9, 0xe6, 0x4f, 0x56, 0x15, 0xcd };
200     #elif defined(AES128)
201     uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
202     uint8_t out[] = { 0x76, 0x49, 0xab, 0xac, 0x81, 0x19,
0xb2, 0x46, 0xce, 0xe9, 0x8e, 0x9b, 0x12, 0xe9, 0x19, 0x7d,
203     0x50, 0x86, 0xcb, 0x9b, 0x50, 0x72,
0x19, 0xee, 0x95, 0xdb, 0x11, 0x3a, 0x91, 0x76, 0x78, 0xb2,
204     0x73, 0xbe, 0xd6, 0xb8, 0xe3, 0xc1,
0x74, 0x3b, 0x71, 0x16, 0xe6, 0x9e, 0x22, 0x22, 0x95, 0x16,
205     0x3f, 0xf1, 0xca, 0xa1, 0x68, 0x1f,
0xac, 0x09, 0x12, 0x0e, 0xca, 0x30, 0x75, 0x86, 0xe1, 0xa7 };
206     #endif
207     uint8_t iv[] = { 0x00, 0x01, 0x02, 0x03, 0x04, 0x05,
0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c, 0x0d, 0x0e, 0x0f };
208     uint8_t in[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a,
209     0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03,
0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51,
210     0x30, 0xc8, 0x1c, 0x46, 0xa3, 0x5c,
0xe4, 0x11, 0xe5, 0xfb, 0xc1, 0x19, 0x1a, 0x0a, 0x52, 0xef,
211     0xf6, 0x9f, 0x24, 0x45, 0xdf, 0x4f,
0x9b, 0x17, 0xad, 0x2b, 0x41, 0x7b, 0xe6, 0x6c, 0x37, 0x10 };
212     struct AES_ctx ctx;
213
214     AES_init_ctx_iv(&ctx, key, iv);
215     AES_CBC_encrypt_buffer(&ctx, in, 64);
216
217     printf("CBC encrypt: ");
218
219     if (0 == memcmp((char*) out, (char*) in, 64)) {
220         printf("SUCCESS!\n");
221         return(0);
222     } else {
223         printf("FAILURE!\n");
224         return(1);
225     }
226 }
227
228 static int test_xcrypt_ctr(const char* xcrypt);
229 static int test_encrypt_ctr(void)
230 {
231     return test_xcrypt_ctr("encrypt");
232 }
233
234 static int test_decrypt_ctr(void)
235 {
236     return test_xcrypt_ctr("decrypt");
237 }
238
239 static int test_xcrypt_ctr(const char* xcrypt)
240 {
241     #if defined(AES256)
242     uint8_t key[32] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
243     0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
244     uint8_t in[64] = { 0x60, 0x1e, 0xc3, 0x13, 0x77, 0x57,
0x89, 0xa5, 0xb7, 0xa7, 0xf5, 0x04, 0xbb, 0xf3, 0xd2, 0x28,
245     0xf4, 0x43, 0xe3, 0xca, 0x4d, 0x62,
0xb5, 0x9a, 0xca, 0x84, 0xe9, 0x90, 0xca, 0xca, 0xf5, 0xc5,
246     0x2b, 0x09, 0x30, 0xda, 0xa2, 0x3d,
0xe9, 0x4c, 0xe8, 0x70, 0x17, 0xba, 0x2d, 0x84, 0x98, 0x8d,
247     0xdf, 0xc9, 0xc5, 0x8d, 0xb6, 0x7a,
0xad, 0xa6, 0x13, 0xc2, 0xdd, 0x08, 0x45, 0x79, 0x41, 0xa6 };
248     #elif defined(AES192)
249     uint8_t key[24] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5,
250     0x62, 0xf8, 0xea, 0xd2, 0x52, 0x2c,
0x6b, 0x7b };
251     uint8_t in[64] = { 0x1a, 0xbc, 0x93, 0x24, 0x17, 0x52,
0x1c, 0xa2, 0x4f, 0x2b, 0x04, 0x59, 0xfe, 0x7e, 0x6e, 0x0b,
252     0x09, 0x03, 0x39, 0xec, 0x0a, 0xa6,
0xfa, 0xef, 0xd5, 0xcc, 0xc2, 0xc6, 0xf4, 0xce, 0x8e, 0x94,
253     0x1e, 0x36, 0xb2, 0x6b, 0xd1, 0xeb,
0xc6, 0x70, 0xd1, 0xbd, 0x1d, 0x66, 0x56, 0x20, 0xab, 0xf7,

```

```

254     0x4f, 0x78, 0xa7, 0xf6, 0xd2, 0x98,
0x09, 0x58, 0x5a, 0x97, 0xda, 0xec, 0x58, 0xc6, 0xb0, 0x50 };
255     #elif defined(AES128)
256     uint8_t key[16] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
257     uint8_t in[64] = { 0x87, 0x4d, 0x61, 0x91, 0xb6, 0x20,
0xe3, 0x26, 0x1b, 0xef, 0x68, 0x64, 0x99, 0x0d, 0xb6, 0xce,
258     0x98, 0x06, 0xf6, 0x6b, 0x79, 0x70,
0xfd, 0xff, 0x86, 0x17, 0x18, 0x7b, 0xb9, 0xff, 0xfd, 0xff,
259     0x5a, 0xe4, 0xdf, 0x3e, 0xdb, 0xd5,
0xd3, 0x5e, 0x5b, 0x4f, 0x09, 0x02, 0x0d, 0xb0, 0x3e, 0xab,
260     0x1e, 0x03, 0x1d, 0xda, 0x2f, 0xbe,
0x03, 0xd1, 0x79, 0x21, 0x70, 0x70, 0xa0, 0xf3, 0x00, 0x9c, 0xee };
261     #endif
262     uint8_t iv[16] = { 0xf0, 0xf1, 0xf2, 0xf3, 0xf4, 0xf5,
0xf6, 0xf7, 0xf8, 0xf9, 0xfa, 0xfb, 0xfc, 0xfd, 0xfe, 0xff };
263     uint8_t out[64] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a,
264     0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03,
0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51,
265     0x30, 0xc8, 0x1c, 0x46, 0xa3, 0x5c,
0xe4, 0x11, 0xe5, 0xfb, 0xc1, 0x19, 0x1a, 0x0a, 0x52, 0xef,
266     0xf6, 0x9f, 0x24, 0x45, 0xdf, 0x4f,
0x9b, 0x17, 0xad, 0x2b, 0x41, 0x7b, 0xe6, 0x6c, 0x37, 0x10 };
267     struct AES_ctx ctx;
268
269     AES_init_ctx_iv(&ctx, key, iv);
270     AES_CTR_xcrypt_buffer(&ctx, in, 64);
271
272     printf("CTR %s: ", xcrypt);
273
274     if (0 == memcmp((char *) out, (char *) in, 64)) {
275         printf("SUCCESS!\n");
276         return(0);
277     } else {
278         printf("FAILURE!\n");
279         return(1);
280     }
281 }
282
283 static int test_decrypt_ecb(void)
284 {
285     #if defined(AES256)
286     uint8_t key[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca,
0x71, 0xbe, 0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
287     0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61,
0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };
288     uint8_t in[] = { 0xf3, 0xee, 0xd1, 0xbd, 0xb5, 0xd2,
0xa0, 0x3c, 0x06, 0x4b, 0x5a, 0x7e, 0x3d, 0xb1, 0x81, 0xf8 };
289     #elif defined(AES192)
290     uint8_t key[] = { 0x8e, 0x73, 0xb0, 0xf7, 0xda, 0x0e,
0x64, 0x52, 0xc8, 0x10, 0xf3, 0x2b, 0x80, 0x90, 0x79, 0xe5,
291     0x62, 0xf8, 0xea, 0xd2, 0x52, 0x2c,
0x6b, 0x7b };
292     uint8_t in[] = { 0xbd, 0x33, 0x4f, 0x1d, 0x6e, 0x45,
0xf2, 0x5f, 0xf7, 0x12, 0xa2, 0x14, 0x57, 0x1f, 0xa5, 0xcc };
293     #elif defined(AES128)
294     uint8_t key[] = { 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae,
0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c };
295     uint8_t in[] = { 0x3a, 0xd7, 0x7b, 0xb4, 0x0d, 0x7a,
0x36, 0x60, 0xa8, 0x9e, 0xca, 0xf3, 0x24, 0x66, 0xef, 0x97 };
296     #endif
297
298     uint8_t out[] = { 0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40,
0x9f, 0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a };
299     struct AES_ctx ctx;
300
301     AES_init_ctx(&ctx, key);
302     AES_ECB_decrypt(&ctx, in);
303
304     printf("ECB decrypt: ");
305
306     if (0 == memcmp((char *) out, (char *) in, 16)) {
307         printf("SUCCESS!\n");
308         return(0);
309     } else {
310         printf("FAILURE!\n");
311         return(1);
312     }
313 }
314 }
315
316

```

AES-SHA-MD5 C 참조 구현

What. Brad Conte 의 교육용 암호 참조 구현 모음. 각 알고리즘이 한 파일에 독립적으로 들어 있다.

Why it exists. 두 번째 독립 구현 으로 쓴다. 골든 모델 하나만 있으면 그 모델의 오해를 검증할 수 없다. 서로 다른 저자가 같은 표준을 읽고 쓴 두 구현이 일치하면 표준 해석이 맞았을 확률이 크게 올라간다.

학부 복습 · **EECS concepts.** 해시 함수의 Merkle–Damgård 구조 · 압축 함수 · 패딩 규칙 · 충돌 저항성의 정의.

석사 수준 · **Practice note.** 교차 검증(cross-validation)을 설계 초기에 해 두면, 나중에 RTL 과 불일치가 났을 때 ‘모델 이 틀렸을 가능성’을 빠르게 지울 수 있다. 3장의 불일치 삼분법에서 2번을 싸게 배제하는 방법이다.

30 files • 3,673 lines

API — aes.c

Table 102. B-Con_crypto-algorithms/aes.c — 함수 (26개)

함수	반환형	받는 파라미터
xor_buf	void	const BYTE in[], BYTE out[], size_t len
aes_encrypt_cbc	int	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_encrypt_cbc_mac	int	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_decrypt_cbc	int	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
increment_iv	void	BYTE iv[], int counter_size
aes_encrypt_ctr	void	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_decrypt_ctr	void	const BYTE in[], size_t in_len, BYTE out[], const WORD key[], int keysize, const BYTE iv[]
aes_encrypt_ccm	int	const BYTE payload[], WORD payload_len, const BYTE assoc[], unsigned short assoc_len, const BYTE nonce[], unsigned short nonce_len, BYTE out[], WORD *...
aes_decrypt_ccm	int	const BYTE ciphertext[], WORD ciphertext_len, const BYTE assoc[], unsigned short assoc_len, const BYTE nonce[], unsigned short nonce_len, BYTE plainte...
ccm_prepare_first_ctr_blk	void	BYTE counter[], const BYTE nonce[], int nonce_len, int payload_len_store_size
ccm_prepare_first_format_blk	void	BYTE buf[], int assoc_len, int payload_len, int payload_len_store_size, int mac_len, const BYTE nonce[], int nonce_len
ccm_format_assoc_data	void	BYTE buf[], int *end_of_buf, const BYTE assoc[], int assoc_len
ccm_format_payload_data	void	BYTE buf[], int *end_of_buf, const BYTE payload[], int payload_len
SubWord	WORD	WORD word
aes_key_setup	void	const BYTE key[], WORD w[], int keysize
AddRoundKey	void	BYTE state[][4], const WORD w[]

함수	반환형	받는 파라미터
SubBytes	void	BYTE state[][4]
InvSubBytes	void	BYTE state[][4]
ShiftRows	void	BYTE state[][4]
InvShiftRows	void	BYTE state[][4]
MixColumns	void	BYTE state[][4]
InvMixColumns	void	BYTE state[][4]
aes_encrypt	void	const BYTE in[], BYTE out[], const WORD key[], int keysize
aes_decrypt	void	const BYTE in[], BYTE out[], const WORD key[], int keysize
print_state	void	BYTE state[][4]
print_rnd_key	void	WORD key[]

Table 103. B-Con_crypto-algorithms/aes.c — #define / `define 매크로 (6개)

매크로	인자	정의
KE_ROTWORD	(x)	$((x) < 8) \mid ((x) > 24)$
TRUE		1
FALSE		0
AES_128_ROUNDS		10
AES_192_ROUNDS		12
AES_256_ROUNDS		14

API — aes.h

Table 104. B-Con_crypto-algorithms/aes.h — typedef / using (2개)

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 105. B-Con_crypto-algorithms/aes.h — #define / `define 매크로 (2개)

매크로	인자	정의
AES_H		/***** HEADER FILES *****/
AES_BLOCK_SIZE		16 // AES operates on 16 bytes at a time

API — aes_test.c

Table 106. B-Con_crypto-algorithms/aes_test.c — 함수 (7개)

함수	반환형	받는 파라미터
print_hex	void	BYTE str[], int len
aes_ecb_test	int	
aes_cbc_test	int	

함수	반환형	받는 파라미터
aes_ctr_test	int	
aes_ccm_test	int	
aes_test	int	
main	int	int argc, char *argv[]

API — arcfour.c

Table 107. B-Con_crypto-algorithms/arcfour.c — 함수 (2개)

함수	반환형	받는 파라미터
arcfour_key_setup	void	BYTE state[], const BYTE key[], int len
arcfour_generate_stream	void	BYTE state[], BYTE out[], size_t len

API — arcfour.h

Table 108. B-Con_crypto-algorithms/arcfour.h — typedef / using (1개)

새 이름	원래 형
BYTE	unsigned char

Table 109. B-Con_crypto-algorithms/arcfour.h — #define / `define 매크로 (1개)

매크로	인자	정의
ARCFOUR_H		/***** HEADER FILES *****/

API — arcfour_test.c

Table 110. B-Con_crypto-algorithms/arcfour_test.c — 함수 (2개)

함수	반환형	받는 파라미터
rc4_test	int	
main	int	

API — base64.c

Table 111. B-Con_crypto-algorithms/base64.c — 함수 (3개)

함수	반환형	받는 파라미터
revchar	BYTE	char ch
base64_encode	size_t	const BYTE in[], BYTE out[], size_t len, int newline_flag
base64_decode	size_t	const BYTE in[], BYTE out[], size_t len

Table 112. B-Con_crypto-algorithms/base64.c — #define / `define 매크로 (1개)

매크로	인자	정의
NEWLINE_INVL		76

API — base64.h**Table 113. B-Con_crypto-algorithms/base64.h — typedef / using (1개)**

새 이름	원래 형
BYTE	unsigned char

Table 114. B-Con_crypto-algorithms/base64.h — #define / `define 매크로 (1개)

매크로	인자	정의
BASE64_H		/***** HEADER FILES *****/

API — base64_test.c**Table 115. B-Con_crypto-algorithms/base64_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
base64_test	int	
main	int	

API — blowfish.c**Table 116. B-Con_crypto-algorithms/blowfish.c — 함수 (3개)**

함수	반환형	받는 파라미터
blowfish_encrypt	void	const BYTE in[], BYTE out[], const BLOWFISH_KEY *keystruct
blowfish_decrypt	void	const BYTE in[], BYTE out[], const BLOWFISH_KEY *keystruct
blowfish_key_setup	void	const BYTE user_key[], BLOWFISH_KEY *keystruct, size_t len

Table 117. B-Con_crypto-algorithms/blowfish.c — #define / `define 매크로 (3개)

매크로	인자	정의
F	(x,t)	t = keystruct->s[0][(x) >> 24]; \
swap	(r,l,t)	t = l; l = r; r = t;
ITERATION	(l,r,t,pval)	l ^= keystruct->p[pval]; F(l,t); r ^= t; swap(r,l,t);

API — blowfish.h**Table 118. B-Con_crypto-algorithms/blowfish.h — typedef / using (2개)**

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 119. B-Con_crypto-algorithms/blowfish.h — #define / `define 매크로 (2개)

매크로	인자	정의
BLOWFISH_H		/***** HEADER FILES *****/
BLOWFISH_BLOCK_SIZE		8 // Blowfish operates on 8 bytes at a time

API — blowfish_test.c**Table 120. B-Con_crypto-algorithms/blowfish_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
blowfish_test	int	
main	int	

API — des.c**Table 121. B-Con_crypto-algorithms/des.c — 함수 (7개)**

함수	반환형	받는 파라미터
IP	void	WORD state[], const BYTE in[]
InvIP	void	WORD state[], BYTE in[]
f	WORD	WORD state, const BYTE key[]
des_key_setup	void	const BYTE key[], BYTE schedule[][6], DES_MODE mode
des_crypt	void	const BYTE in[], BYTE out[], const BYTE key[][6]
three_des_key_setup	void	const BYTE key[], BYTE schedule[][16][6], DES_MODE mode
three_des_crypt	void	const BYTE in[], BYTE out[], const BYTE key[][16][6]

Table 122. B-Con_crypto-algorithms/des.c — #define / `define 매크로 (4개)

매크로	인자	정의
BITNUM	(a,b,c)	$((a[(b)/8] \gg (7 - (b\%8))) \& 0x01) \ll (c))$
BITNUMINTR	(a,b,c)	$((a \gg (31 - (b))) \& 0x00000001) \ll (c))$
BITNUMINTL	(a,b,c)	$((a \ll (b)) \& 0x80000000) \gg (c))$
SBOXBIT	(a)	$((a \& 0x20) ((a \& 0x1f) \gg 1) ((a \& 0x01) \ll 4))$

API — des.h**Table 123. B-Con_crypto-algorithms/des.h — typedef / using (2개)**

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 124. B-Con_crypto-algorithms/des.h — #define / `define 매크로 (2개)

매크로	인자	정의
DESH		/***** HEADER FILES *****/
DES_BLOCK_SIZE		8 // DES operates on 8 bytes at a time

API — des_test.c**Table 125. B-Con_crypto-algorithms/des_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
des_test	int	
main	int	

API — md2.c

Table 126. B-Con_crypto-algorithms/md2.c — 함수 (4개)

함수	반환형	받는 파라미터
md2_transform	void	MD2_CTX *ctx, BYTE data[]
md2_init	void	MD2_CTX *ctx
md2_update	void	MD2_CTX *ctx, const BYTE data[], size_t len
md2_final	void	MD2_CTX *ctx, BYTE hash[]

API — md2.h

Table 127. B-Con_crypto-algorithms/md2.h — typedef / using (1개)

새 이름	원래 형
BYTE	unsigned char

Table 128. B-Con_crypto-algorithms/md2.h — #define / `define 매크로 (2개)

매크로	인자	정의
MD2_H		/***** HEADER FILES *****/
MD2_BLOCK_SIZE		16

API — md2_test.c

Table 129. B-Con_crypto-algorithms/md2_test.c — 함수 (2개)

함수	반환형	받는 파라미터
md2_test	int	
main	int	

API — md5.c

Table 130. B-Con_crypto-algorithms/md5.c — 함수 (4개)

함수	반환형	받는 파라미터
md5_transform	void	MD5_CTX *ctx, const BYTE data[]
md5_init	void	MD5_CTX *ctx
md5_update	void	MD5_CTX *ctx, const BYTE data[], size_t len
md5_final	void	MD5_CTX *ctx, BYTE hash[]

Table 131. B-Con_crypto-algorithms/md5.c — #define / `define 매크로 (9개)

매크로	인자	정의
ROTLEFT	(a,b)	((a << b) (a >> (32-b)))

매크로	인자	정의
F	(x,y,z)	$((x \& y) \mid (\sim x \& z))$
G	(x,y,z)	$((x \& z) \mid (y \& \sim z))$
H	(x,y,z)	$(x \wedge y \wedge z)$
I	(x,y,z)	$(y \wedge (x \mid \sim z))$
FF	(a,b,c,d,m,s,t)	{ a += F(b,c,d) + m + t; \
GG	(a,b,c,d,m,s,t)	{ a += G(b,c,d) + m + t; \
HH	(a,b,c,d,m,s,t)	{ a += H(b,c,d) + m + t; \
II	(a,b,c,d,m,s,t)	{ a += I(b,c,d) + m + t; \

API — md5.h

Table 132. B-Con_crypto-algorithms/md5.h — typedef / using (2개)

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 133. B-Con_crypto-algorithms/md5.h — #define / `define 매크로 (2개)

매크로	인자	정의
MD5_H		/***** HEADER FILES *****/
MD5_BLOCK_SIZE		16 // MD5 outputs a 16 byte digest

API — md5_test.c

Table 134. B-Con_crypto-algorithms/md5_test.c — 함수 (2개)

함수	반환형	받는 파라미터
md5_test	int	
main	int	

API — rot-13.c

Table 135. B-Con_crypto-algorithms/rot-13.c — 함수 (1개)

함수	반환형	받는 파라미터
rot13	void	char str[]

API — rot-13.h

Table 136. B-Con_crypto-algorithms/rot-13.h — #define / `define 매크로 (1개)

매크로	인자	정의
ROT13_H		/***** HEADER FILES *****/

API — rot-13_test.c**Table 137. B-Con_crypto-algorithms/rot-13_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
rot13_test	int	
main	int	

API — sha1.c**Table 138. B-Con_crypto-algorithms/sha1.c — 함수 (4개)**

함수	반환형	받는 파라미터
sha1_transform	void	SHA1_CTX *ctx, const BYTE data[]
sha1_init	void	SHA1_CTX *ctx
sha1_update	void	SHA1_CTX *ctx, const BYTE data[], size_t len
sha1_final	void	SHA1_CTX *ctx, BYTE hash[]

Table 139. B-Con_crypto-algorithms/sha1.c — #define / `define 매크로 (1개)

매크로	인자	정의
ROTLLEFT	(a, b)	((a << b) (a >> (32 - b)))

API — sha1.h**Table 140. B-Con_crypto-algorithms/sha1.h — typedef / using (2개)**

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 141. B-Con_crypto-algorithms/sha1.h — #define / `define 매크로 (2개)

매크로	인자	정의
SHA1_H		/***** HEADER FILES *****/
SHA1_BLOCK_SIZE		20 // SHA1 outputs a 20 byte digest

API — sha1_test.c**Table 142. B-Con_crypto-algorithms/sha1_test.c — 함수 (2개)**

함수	반환형	받는 파라미터
sha1_test	int	
main	int	

API — sha256.c**Table 143. B-Con_crypto-algorithms/sha256.c — 함수 (4개)**

함수	반환형	받는 파라미터
sha256_transform	void	SHA256_CTX *ctx, const BYTE data[]

함수	반환형	받는 파라미터
sha256_init	void	SHA256_CTX *ctx
sha256_update	void	SHA256_CTX *ctx, const BYTE data[], size_t len
sha256_final	void	SHA256_CTX *ctx, BYTE hash[]

Table 144. B-Con_crypto-algorithms/sha256.c — #define / `define 매크로 (8개)

매크로	인자	정의
ROTL	(a,b)	$((a) \ll (b)) \mid ((a) \gg (32-(b)))$
ROTR	(a,b)	$((a) \gg (b)) \mid ((a) \ll (32-(b)))$
CH	(x,y,z)	$((x) \& (y)) \wedge \neg(x) \& (z)$
MAJ	(x,y,z)	$((x) \& (y)) \wedge ((x) \& (z)) \wedge ((y) \& (z))$
EP0	(x)	$(\text{ROTRIGHT}(x,2) \wedge \text{ROTRIGHT}(x,13) \wedge \text{ROTRIGHT}(x,22))$
EP1	(x)	$(\text{ROTRIGHT}(x,6) \wedge \text{ROTRIGHT}(x,11) \wedge \text{ROTRIGHT}(x,25))$
SIG0	(x)	$(\text{ROTRIGHT}(x,7) \wedge \text{ROTRIGHT}(x,18) \wedge ((x) \gg 3))$
SIG1	(x)	$(\text{ROTRIGHT}(x,17) \wedge \text{ROTRIGHT}(x,19) \wedge ((x) \gg 10))$

API — sha256.h

Table 145. B-Con_crypto-algorithms/sha256.h — typedef / using (2개)

새 이름	원래 형
BYTE	unsigned char
WORD	unsigned int

Table 146. B-Con_crypto-algorithms/sha256.h — #define / `define 매크로 (2개)

매크로	인자	정의
SHA256_H		/***** HEADER FILES *****/
SHA256_BLOCK_SIZE		32 // SHA256 outputs a 32 byte digest

API — sha256_test.c

Table 147. B-Con_crypto-algorithms/sha256_test.c — 함수 (2개)

함수	반환형	받는 파라미터
sha256_test	int	
main	int	

1. B-Con_crypto-algorithms/aes.c (1,095 lines)

```

1 /
*****
2 * Filename:   aes.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   This code is the implementation of the AES al-
gorithm and
7             the CTR, CBC, and CCM modes of operation it can
be used in.

```

```

8             AES is, specified by the NIST in in publication
FIPS PUB 197,
9             available at:
10            * http://csrc.nist.gov/publications/fips/
fips197/fips-197.pdf .
11
12            The CBC and CTR modes of operation are specified by
13            NIST SP 800-38 A, available at:
14            * http://csrc.nist.gov/publications/nistpubs/
800-38a/sp800-38a.pdf .
15            The CCM mode of operation is specified by NIST
SP800-38 C, available at:
16            * http://csrc.nist.gov/publications/nistpubs/
800-38C/SP800-38C_updated-July20_2007.pdf

```

```

*****
*/
17
18 /***** HEADER FILES
*****
19 #include <stdlib.h>
20 #include <memory.h>
21 #include "aes.h"
22
23 #include <stdio.h>
24
25 /***** MACROS
*****
26 // The least significant byte of the word is rotated to the
end.
27 #define KE_ROTWORD(x) (((x) << 8) | ((x) >> 24))
28
29 #define TRUE 1
30 #define FALSE 0
31
32 /***** DATA TYPES
*****
33 #define AES_128_ROUNDS 10
34 #define AES_192_ROUNDS 12
35 #define AES_256_ROUNDS 14
36
37 /***** FUNCTION DECLARATIONS
*****
38 void ccm_prepare_first_ctr_blk(BYTE counter[], const BYTE
nonce[], int nonce_len, int payload_len_store_size);
39 void ccm_prepare_first_format_blk(BYTE buf[], int assoc_len,
int payload_len, int payload_len_store_size, int mac_len, const BYTE
nonce[], int nonce_len);
40 void ccm_format_assoc_data(BYTE buf[], int *end_of_buf, const
BYTE assoc[], int assoc_len);
41 void ccm_format_payload_data(BYTE buf[], int *end_of_buf,
const BYTE payload[], int payload_len);
42
43 /***** VARIABLES
*****
44 // This is the specified AES SBox. To look up a substitution
value, put the first
45 // nibble in the first index (row) and the second nibble in
the second index (column).
46 static const BYTE aes_sbox[16][16] = {
47     {0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F,
0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76},
48     {0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD,
0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0},
49     {0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC,
0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15},
50     {0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A,
0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75},
51     {0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B,
0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84},
52     {0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB,
0xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF},
53     {0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D,
0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8},
54     {0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC,
0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2},
55     {0xCD, 0x0C, 0x13, 0xEC, 0x5F,
0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73},
56     {0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE,
0xB8, 0x14, 0xDE, 0x5E, 0xB0, 0xDB},
57     {0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C,
0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0x79},
58     {0xE7, 0xC8, 0x37, 0x6D, 0x80, 0xD5, 0x4E, 0xA9, 0x6C,
0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08},
59     {0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD,
0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A},
60     {0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E,
0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E},
61     {0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E,
0x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF},
62     {0x8C, 0xA1, 0x89, 0x0D, 0xBF,
0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16}
63 };
64
65 static const BYTE aes_invbox[16][16] = {
66     {0x52, 0x09, 0x6A, 0xD5, 0x30, 0x36, 0xA5, 0x38, 0xBF,
0x40, 0xA3, 0x9E, 0x81, 0xF3, 0xD7, 0xFB},
67     {0x7C, 0xE3, 0x39, 0x82, 0x9B, 0x2F, 0xFF, 0x87, 0x34, 0x8E,
0x43, 0x44, 0xC4, 0xDE, 0xE9, 0xCB},
68     {0x54, 0x7B, 0x94, 0x32, 0xA6, 0xC2, 0x23, 0x3D, 0xEE, 0x4C,
0x95, 0x0B, 0x42, 0xFA, 0xC3, 0x4E},
69     {0x08, 0x2E, 0xA1, 0x66, 0x28, 0xD9, 0x24, 0xB2, 0x76, 0x5B,
0xA2, 0x49, 0x6D, 0x88, 0xD1, 0x25},
70
71     {0x72, 0xF8, 0xF6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xD4, 0xA4, 0x5C, 0xCC, 0x5D,
0x65, 0xB6, 0x92},
72     {0x6C, 0x70, 0x48, 0x50, 0xFD, 0xED, 0xB9, 0xDA, 0x5E,
0x15, 0x46, 0x57, 0xA7, 0x8D, 0x9D, 0x84},
73     {0x90, 0xD8, 0xAB, 0x00, 0x8C, 0xBC, 0xD3, 0x0A,
0xF7, 0xE4, 0x58, 0x05, 0xB8, 0xB3, 0x45, 0x06},
74     {0xD0, 0x2C, 0x1E, 0x8F, 0xCA, 0x3F, 0x0F, 0x02, 0xC1, 0xAF,
0xBD, 0x03, 0x01, 0x13, 0x8A, 0x6B},
75     {0x3A, 0x91, 0x11, 0x41, 0x4F, 0x67, 0xDC, 0xEA,
0x97, 0xF2, 0xCF, 0xCE, 0xF0, 0xB4, 0xE6, 0x73},
76     {0x96, 0xAC, 0x74, 0x22, 0xE7, 0xAD,
0x35, 0x85, 0xE2, 0xF9, 0x37, 0xE8, 0x1C, 0x75, 0xDF, 0x6E},

```

```

76     {0x47, 0xF1, 0x1A, 0x71, 0x1D, 0x29, 0xC5, 0x89, 0x6F,
0xB7, 0x62, 0x0E, 0xAA, 0x18, 0xBE, 0x1B},
77     {0xFC, 0x56, 0x3E, 0x4B, 0xC6, 0xD2, 0x79, 0x20, 0x9A, 0xDB,
0xC0, 0xFE, 0x78, 0xCD, 0x5A, 0xF4},
78     {0x1F, 0xDD,
0xA8, 0x33, 0x88, 0x07, 0xC7, 0x31, 0xB1, 0x12, 0x10, 0x59, 0x27, 0x80, 0xEC,
0x5F},
79     {0x60, 0x51, 0x7F, 0xA9, 0x19, 0xB5, 0x4A, 0x0D, 0x2D,
0xE5, 0x7A, 0x9F, 0x93, 0xC9, 0x9C, 0xEF},
80     {0xA0, 0xE0, 0x3B, 0x4D, 0xAE, 0x2A, 0xF5, 0xB0, 0xC8, 0xEB,
0xBB, 0x3C, 0x83, 0x53, 0x99, 0x61},
81     {0x17, 0x2B, 0x04, 0x7E, 0xBA,
0x77, 0xD6, 0x26, 0xE1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0C, 0x7D}
82 };
83
84 // This table stores pre-calculated values for all possible
GF(2^8) calculations. This
85 // table is only used by the (Inv)MixColumns steps.
86 // USAGE: The second index (column) is the coefficient of mul-
tiplication. Only 7 different
87 // coefficients are used: 0x01, 0x02, 0x03, 0x09, 0x0b, 0x0d,
0x0e, but multiplication by
88 // 1 is negligible leaving only 6 coefficients. Each column of
the table is devoted to one
89 // of these coefficients, in the ascending order of value,
from values 0x00 to 0xFF.
90 static const BYTE gf_mul[256][6] = {
91     {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, {0x02, 0x03, 0x09, 0x0b,
0x0d, 0x0e},
92     {0x04, 0x06, 0x12, 0x16, 0x1a, 0x1c}, {0x08, 0x05, 0x1b, 0x1d,
0x17, 0x12},
93     {0x00, 0x0c, 0x24, 0x2c, 0x34, 0x38}, {0x0a, 0x0f, 0x2d,
0x27, 0x39, 0x36},
94     {0x0c, 0x0a, 0x36, 0x3a, 0x2e, 0x24}, {0x0e, 0x09, 0x3f,
0x31, 0x23, 0x2a},
95     {0x10, 0x18, 0x48, 0x58, 0x68, 0x70}, {0x12, 0x1b,
0x41, 0x53, 0x65, 0x7e},
96     {0x14, 0x1e, 0x5a, 0x4e, 0x72, 0x6c}, {0x16, 0x1d,
0x53, 0x45, 0x7f, 0x62},
97     {0x18, 0x14, 0x6c, 0x74, 0x5c, 0x48}, {0x1a, 0x17, 0x65, 0x7f,
0x51, 0x46},
98     {0x1c, 0x12, 0x7e, 0x62, 0x46, 0x54}, {0x1e,
0x11, 0x77, 0x69, 0x4b, 0x5a},
99     {0x20, 0x30, 0x90, 0xb0, 0xd0, 0xe0}, {0x22, 0x33, 0x99, 0xbb,
0xdd, 0xee},
100    {0x24, 0x36, 0x82, 0xa6, 0xca, 0xfc}, {0x26, 0x35, 0x8b, 0xad,
0xc7, 0xf2},
101    {0x28, 0x3c, 0xb4, 0x9c, 0xe4, 0xd8}, {0x2a, 0x3f, 0xbd,
0x97, 0xe9, 0xd6},
102    {0x2c, 0x3a, 0xa6, 0x8a, 0xfe, 0xc4}, {0x2e, 0x39, 0xaf,
0x81, 0xf3, 0xca},
103    {0x30, 0x28, 0xd8, 0xe8, 0xb8, 0x90}, {0x32, 0x2b,
0xd1, 0xe3, 0xb5, 0x9e},
104    {0x34, 0x2e, 0xca, 0xfe, 0xa2, 0x8c}, {0x36, 0x2d,
0xc3, 0xf5, 0xaf, 0x82},
105    {0x38, 0x24, 0xfc, 0xc4, 0x8c, 0xa8}, {0x3a, 0x27, 0xf5, 0xcf,
0x81, 0xa6},
106    {0x3c, 0x22, 0xee, 0xd2, 0x96, 0xb4}, {0x3e,
0x21, 0xe7, 0xd9, 0x9b, 0xba},
107    {0x40, 0x60, 0x3b, 0x7b, 0xb0, 0xdb}, {0x42, 0x63, 0x32, 0x70, 0xb6, 0xd5},
108    {0x44, 0x66, 0x29, 0x6d, 0xa1, 0xc7}, {0x46, 0x65, 0x20, 0x66, 0xac, 0xc9},
109    {0x48, 0x6c, 0x1f, 0x57, 0x8f, 0xe3}, {0x4a, 0x6f, 0x16, 0x5c,
0x82, 0xed},
110    {0x4c, 0x6a, 0x0d, 0x41, 0x95, 0xff}, {0x4e, 0x69, 0x04, 0x4a,
0x98, 0xf1},
111    {0x50, 0x78, 0x73, 0x23, 0xd3, 0xab}, {0x52, 0x7b, 0x7a,
0x28, 0xde, 0xa5},
112    {0x54, 0x7e, 0x61, 0x35, 0xc9, 0xb7}, {0x56, 0x7d, 0x68, 0x3e,
0xc4, 0xb9},
113    {0x58, 0x74, 0x57, 0x0f, 0xe7, 0x93}, {0x5a, 0x77, 0x5e,
0x04, 0xea, 0x9d},
114    {0x5c, 0x72, 0x45, 0x19, 0xfd, 0x8f}, {0x5e, 0x71, 0x4c,
0x12, 0xf0, 0x81},
115    {0x60, 0x50, 0xab, 0xcb, 0x6b, 0x3b}, {0x62, 0x53, 0xa2, 0xc0, 0x66, 0x35},
116    {0x64, 0x56, 0xb9, 0xdd, 0x71, 0x27}, {0x66, 0x55, 0xb0, 0xd6, 0x7c, 0x29},
117    {0x68, 0x5c, 0x8f, 0xe7, 0x5f, 0x03}, {0x6a, 0x5f, 0x86, 0xec,
0x52, 0x0d},
118    {0x6c, 0x5a, 0x9d, 0xf1, 0x45, 0x1f}, {0x6e, 0x59, 0x94, 0xfa,
0x48, 0x11},
119    {0x70, 0x48, 0xe3, 0x93, 0x03, 0x4b}, {0x72, 0x4b, 0xea,
0x98, 0x0e, 0x45},
120    {0x74, 0x4e, 0xf1, 0x85, 0x19, 0x57}, {0x76, 0x4d, 0xf8, 0x8e,
0x14, 0x59},
121    {0x78, 0x44, 0xc7, 0xbf, 0x37, 0x73}, {0x7a, 0x47, 0xce,
0xb4, 0x3a, 0x7d},
122    {0x7c, 0x42, 0xd5, 0xa9, 0x2d, 0x6f}, {0x7e, 0x41, 0xdc,
0xa2, 0x20, 0x61},
123    {0x80, 0xc0, 0x76, 0xf6, 0x6d, 0xad}, {0x82, 0xc3, 0x7f, 0xfd,
0x60, 0xa3},
124    {0x84, 0xc6, 0x64, 0xe0, 0x77, 0xb1}, {0x86, 0xc5, 0x6d, 0xeb,
0x7a, 0xbf},
125    {0x88, 0xcc, 0x52, 0xda, 0x59, 0x95}, {0x8a, 0xcf, 0x5b,
0xd1, 0x54, 0x9b},
126    {0x8c, 0xca, 0x40, 0xcc, 0x43, 0x89}, {0x8e,
0xc9, 0x49, 0xc7, 0x4e, 0x87},
127    {0x90, 0xd8, 0x3e, 0xae, 0x05, 0xdd}, {0x92, 0xdb,

```

```

0x37, 0xa5, 0x08, 0xd3},
128 {0x94, 0xde, 0x2c, 0xb8, 0x1f, 0xc1}, {0x96, 0xdd,
0x25, 0xb3, 0x12, 0xc4},
129 {0x98, 0xd4, 0x1a, 0x82, 0x31, 0xe5}, {0x9a,
0xd7, 0x13, 0x89, 0x3c, 0xeb},
130 {0x9c, 0xd2, 0x08, 0x94, 0x2b, 0xf9}, {0x9e, 0xd1, 0x01, 0x9f,
0x26, 0xf7},
131 {0xa0, 0xf0, 0xe6, 0x46, 0xbd, 0x4d}, {0xa2, 0xf3, 0xef, 0x4d,
0xb0, 0x43},
132 {0xa4, 0xf6, 0xf4, 0x50, 0xa7, 0x51}, {0xa6, 0xf5, 0xfd, 0x5b,
0xaa, 0x5f},
133 {0xa8, 0xfc, 0xc2, 0x6a, 0x89, 0x75}, {0xaa, 0xff, 0xcb,
0x61, 0x84, 0x7b},
134 {0xac, 0xfa, 0xd0, 0x7c, 0x93, 0x69}, {0xae,
0xf9, 0xd9, 0x77, 0x9e, 0x67},
135 {0xb0, 0xe8, 0xae, 0x1e, 0xd5, 0x3d}, {0xb2, 0xeb,
0xa7, 0x15, 0xd8, 0x33},
136 {0xb4, 0xee, 0xbc, 0x08, 0xcf, 0x21}, {0xb6, 0xed,
0xb5, 0x03, 0xc2, 0x2f},
137 {0xb8, 0xe4, 0x8a, 0x32, 0xe1, 0x05}, {0xba,
0xe7, 0x83, 0x39, 0xec, 0x0b},
138 {0xbc, 0xe2, 0x98, 0x24, 0xfb, 0x19}, {0xbe, 0xe1, 0x91, 0x2f,
0xf6, 0x17},
139 {0xc0, 0xa0, 0x4d, 0x8d, 0xd6, 0x76},
{0xc2, 0xa3, 0x44, 0x86, 0xdb, 0x78},
140 {0xc4, 0xa6, 0x5f, 0x9b, 0xcc, 0x6a},
{0xc6, 0xa5, 0x56, 0x90, 0xc1, 0x64},
141 {0xc8, 0xac, 0x69, 0xa1, 0xe2, 0x4e}, {0xca, 0xaf, 0x60, 0xaa,
0xef, 0x40},
142 {0xcc, 0xaa, 0x7b, 0xb7, 0xf8, 0x52}, {0xce, 0xa9, 0x72, 0xbc,
0xf5, 0x5c},
143 {0xd0, 0xb8, 0x05, 0xd5, 0xbe, 0x06}, {0xd2, 0xbb, 0x0c, 0xde,
0xb3, 0x08},
144 {0xd4, 0xbe, 0x17, 0xc3, 0xa4, 0x1a}, {0xd6, 0xbd, 0x1e,
0xc8, 0xa9, 0x14},
145 {0xd8, 0xb4, 0x21, 0xf9, 0x8a, 0x3e}, {0xda,
0xb7, 0x28, 0xf2, 0x87, 0x30},
146 {0xdc, 0xb2, 0x33, 0xef, 0x90, 0x22}, {0xde, 0xb1, 0x3a,
0xe4, 0x9d, 0x2c},
147 {0xe0, 0x90, 0xdd, 0x3d, 0x06, 0x96},
{0xe2, 0x93, 0xd4, 0x36, 0x0b, 0x98},
148 {0xe4, 0x96, 0xcf, 0x2b, 0x1c, 0x8a},
{0xe6, 0x95, 0xc6, 0x20, 0x11, 0x84},
149 {0xe8, 0x9c, 0xf9, 0x11, 0x32, 0xae}, {0xea, 0x9f, 0xf0, 0x1a,
0x3f, 0xa0},
150 {0xec, 0x9a, 0xeb, 0x07, 0x28, 0xb2}, {0xee, 0x99, 0xe2, 0x0c,
0x25, 0xbc},
151 {0xf0, 0x88, 0x95, 0x65, 0x6e, 0xe6}, {0xf2, 0x8b, 0x9c, 0x6e,
0x63, 0xe8},
152 {0xf4, 0x8e, 0x87, 0x73, 0xf4, 0xfa}, {0xf6, 0x8d, 0x8e,
0x78, 0x79, 0xf4},
153 {0xf8, 0x84, 0xb1, 0x49, 0x5a, 0xde}, {0xfa,
0x87, 0xb8, 0x42, 0x57, 0xd0},
154 {0xfc, 0x82, 0xa3, 0x5f, 0x40, 0xc2}, {0xfe, 0x81, 0xaa,
0x54, 0x4d, 0xcc},
155 {0x1b, 0x9b, 0xec, 0xf7, 0xda, 0x41}, {0x19, 0x98, 0xe5, 0xfc,
0xd7, 0x4f},
156 {0x1f, 0x9d, 0xfe, 0xe1, 0xc0, 0x5d}, {0x1d, 0x9e, 0xf7, 0xea,
0xcd, 0x53},
157 {0x13, 0x97, 0xc8, 0xdb, 0xee, 0x79},
{0x11, 0x94, 0xc1, 0xd0, 0xe3, 0x77},
158 {0x17, 0x91, 0xda, 0xcd, 0xf4, 0x65},
{0x15, 0x92, 0xd3, 0xc6, 0xf9, 0x6b},
159 {0x0b, 0x83, 0xa4, 0xaf, 0xb2, 0x31}, {0x09, 0x80, 0xad,
0xa4, 0xbf, 0x3f},
160 {0x0f, 0x85, 0xb6, 0xb9, 0xa8, 0x2d}, {0x0d, 0x86, 0xbf,
0xb2, 0xa5, 0x23},
161 {0x03, 0x8f, 0x80, 0x83, 0x86, 0x09}, {0x01, 0x8c,
0x89, 0x88, 0x8b, 0x07},
162 {0x07, 0x89, 0x92, 0x95, 0x9c, 0x15}, {0x05, 0x8a, 0x9b, 0x9e,
0x91, 0x1b},
163 {0x3b, 0xab, 0x7c, 0x47, 0x0a, 0xa1}, {0x39, 0xa8, 0x75, 0x4c,
0x07, 0xaf},
164 {0x3f, 0xad, 0x6e, 0x51, 0x10, 0xbd}, {0x3d, 0xae, 0x67, 0x5a,
0x1d, 0xb3},
165 {0x33, 0xa7, 0x58, 0x6b, 0x3e, 0x99},
{0x31, 0xa4, 0x51, 0x60, 0x33, 0x97},
166 {0x37, 0xa1, 0x4a, 0x7d, 0x24, 0x85},
{0x35, 0xa2, 0x43, 0x76, 0x29, 0x8b},
167 {0x2b, 0xb3, 0x34, 0x1f, 0x62, 0xd1}, {0x29, 0xb0, 0x3d,
0x14, 0x6f, 0xdf},
168 {0x2f, 0xb5, 0x26, 0x09, 0x78, 0xcd}, {0x2d, 0xb6, 0x2f,
0x02, 0x75, 0xc3},
169 {0x23, 0xbf, 0x10, 0x33, 0x56, 0xe9}, {0x21, 0xbc,
0x19, 0x38, 0x5b, 0xe7},
170 {0x27, 0xb9, 0x02, 0x25, 0x4c, 0xf5}, {0x25, 0xba, 0x0b, 0x2e,
0x41, 0xfb},
171 {0x5b, 0xfb, 0xd7, 0x8c, 0x61, 0x9a}, {0x59, 0xf8, 0xde,
0x87, 0x6c, 0x94},
172 {0x5f, 0xfd, 0xc5, 0x9a, 0x7b, 0x86}, {0x5d, 0xfe, 0xcc,
0x91, 0x76, 0x88},
173 {0x53, 0xf7, 0xf3, 0xa0, 0x55, 0xa2}, {0x51, 0xf4, 0xfa, 0xab,
0x58, 0xac},
174 {0x57, 0xf1, 0xe1, 0xb6, 0x4f, 0xbe}, {0x55, 0xf2, 0xe8, 0xbd,
0x42, 0xb0},
175 {0x4b, 0xe3, 0x9f, 0xd4, 0x09, 0xea}, {0x49, 0xe0, 0x96, 0xdf,
0x04, 0xe4},
176 {0x4f, 0xe5, 0x8d, 0xc2, 0x13, 0xf6}, {0x4d,
0xe6, 0x84, 0xc9, 0x1e, 0xf8},
177 {0x43, 0xef, 0xbb, 0xf8, 0x3d, 0xd2}, {0x41, 0xec,
0xb2, 0xf3, 0x30, 0xdc},

```

```

178 {0x47, 0xe9, 0xa9, 0xee, 0x27, 0xce}, {0x45, 0xea,
0xa0, 0xe5, 0x2a, 0xc0},
179 {0x7b, 0xcb, 0x47, 0x3c, 0xb1, 0x7a}, {0x79, 0xc8, 0x4e,
0x37, 0xbc, 0x74},
180 {0x7f, 0xcd, 0x55, 0x2a, 0xab, 0x66}, {0x7d, 0xce, 0x5c,
0x21, 0xa6, 0x68},
181 {0x73, 0xc7, 0x63, 0x10, 0x85, 0x42}, {0x71, 0xc4, 0x6a, 0x1b,
0x88, 0x4c},
182 {0x77, 0xc1, 0x71, 0x06, 0x9f, 0x5e}, {0x75, 0xc2, 0x78, 0x0d,
0x92, 0x50},
183 {0x6b, 0xd3, 0x0f, 0x64, 0xd9, 0x0a}, {0x69, 0xd0, 0x06, 0x6f,
0xd4, 0x04},
184 {0x6f, 0xd5, 0x1d, 0x72, 0xc3, 0x16}, {0x6d,
0xd6, 0x14, 0x79, 0xce, 0x18},
185 {0x63, 0xdf, 0x2b, 0x48, 0xed, 0x32}, {0x61, 0xdc,
0x22, 0x43, 0xe0, 0x3c},
186 {0x67, 0xd9, 0x39, 0x5e, 0xf7, 0x2e}, {0x65, 0xda,
0x30, 0x55, 0xfa, 0x20},
187 {0x9b, 0x5b, 0x9a, 0x01, 0xb7, 0xec}, {0x99, 0x58, 0x93, 0x0a,
0xba, 0xe2},
188 {0x9f, 0x5d, 0x88, 0x17, 0xad, 0xf0}, {0x9d, 0x5e, 0x81, 0x1c,
0xa0, 0xfe},
189 {0x93, 0x57, 0xbe, 0x2d, 0x83, 0xd4},
{0x91, 0x54, 0xb7, 0x26, 0x8e, 0xda},
190 {0x97, 0x51, 0xac, 0x3b, 0x99, 0xc8},
{0x95, 0x52, 0xa5, 0x30, 0x94, 0xc6},
191 {0x8b, 0x43, 0xd2, 0x59, 0xdf, 0x9c}, {0x89, 0x40, 0xdb,
0x52, 0xd2, 0x92},
192 {0x8f, 0x45, 0xc0, 0x4f, 0xc5, 0x80}, {0x8d,
0x46, 0xc9, 0x44, 0xc8, 0x8e},
193 {0x83, 0x4f, 0xf6, 0x75, 0xeb, 0xa4}, {0x81, 0x4c, 0xff, 0x7e,
0xe6, 0xaa},
194 {0x87, 0x49, 0xe4, 0x63, 0xf1, 0xb8}, {0x85, 0x4a, 0xed,
0xe8, 0xfc, 0xb6},
195 {0xbb, 0x6b, 0x0a, 0xb1, 0x67, 0x0c}, {0xb9, 0x68, 0x03, 0xba,
0x6a, 0x02},
196 {0xbf, 0x6d, 0x18, 0xa7, 0x7d, 0x10}, {0xbd, 0x6e, 0x11, 0xac,
0x70, 0x1e},
197 {0xb3, 0x67, 0x2e, 0x9d, 0x53, 0x34},
{0xb1, 0x64, 0x27, 0x96, 0x5e, 0x3a},
198 {0xb7, 0x61, 0x3c, 0x8b, 0x49, 0x28},
{0xb5, 0x62, 0x35, 0x80, 0x44, 0x26},
199 {0xab, 0x73, 0x42, 0xe9, 0x0f, 0x7c}, {0xa9, 0x70, 0x4b,
0xe2, 0x02, 0x72},
200 {0xaf, 0x75, 0x50, 0xff, 0x15, 0x60}, {0xad,
0x76, 0x59, 0xf4, 0x18, 0x6e},
201 {0xa3, 0x7f, 0x66, 0xc5, 0x3b, 0x44}, {0xa1, 0x7c, 0x6f, 0xce,
0x36, 0x4a},
202 {0xa7, 0x79, 0x74, 0xd3, 0x21, 0x58}, {0xa5, 0x7a, 0x7d,
0xd8, 0x2c, 0x56},
203 {0xdb, 0x3b, 0xa1, 0x7a, 0x0c, 0x37},
{0xd9, 0x38, 0xa8, 0x71, 0x01, 0x39},
204 {0xdf, 0x3d, 0xb3, 0x6c, 0x16, 0x2b}, {0xdd, 0x3e, 0xba,
0x67, 0x1b, 0x25},
205 {0xd3, 0x37, 0x85, 0x56, 0x38, 0x0f}, {0xd1, 0x34, 0x8c, 0x5d,
0x35, 0x01},
206 {0xd7, 0x31, 0x97, 0x40, 0x22, 0x13}, {0xd5, 0x32, 0x9e, 0x4b,
0x2f, 0x1d},
207 {0xcb, 0x23, 0xe9, 0x22, 0x64, 0x47},
{0xc9, 0x20, 0xe0, 0x29, 0x69, 0x49},
208 {0xcf, 0x25, 0xfb, 0x34, 0x7e, 0x5b}, {0xcd, 0x26, 0xf2, 0x3f,
0x73, 0x55},
209 {0xc3, 0x2f, 0xcd, 0x0e, 0x50, 0x7f}, {0xc1, 0x2c,
0xc4, 0x05, 0x5d, 0x71},
210 {0xc7, 0x29, 0xdf, 0x18, 0x4a, 0x63}, {0xc5, 0x2a,
0xd6, 0x13, 0x47, 0x6d},
211 {0xfb, 0x0b, 0x31, 0xca, 0xdc, 0xd7},
{0xf9, 0x08, 0x38, 0xc1, 0xd1, 0xd9},
212 {0xff, 0x0d, 0x23, 0xdc, 0xc6, 0xcb}, {0xfd, 0x0e, 0x2a,
0xd7, 0xcb, 0xc5},
213 {0xf3, 0x07, 0x15, 0xe6, 0xe8, 0xef}, {0xf1, 0x04, 0x1c, 0xed,
0xe5, 0xe1},
214 {0xf7, 0x01, 0x07, 0xf0, 0xf2, 0xf3}, {0xf5, 0x02, 0x0e, 0xfb,
0xff, 0xfd},
215 {0xeb, 0x13, 0x79, 0x92, 0xb4, 0xa7},
{0xe9, 0x10, 0x70, 0x99, 0xb9, 0xa9},
216 {0xef, 0x15, 0x6b, 0x84, 0xae, 0xbb}, {0xed, 0x16, 0x62, 0x8f,
0xa3, 0xb5},
217 {0xe3, 0x1f, 0x5d, 0xbe, 0x80, 0x9f}, {0xe1, 0x1c,
0x54, 0xb5, 0x8d, 0x91},
218 {0xe7, 0x19, 0x4f, 0xa8, 0x9a, 0x83}, {0xe5, 0x1a,
0x46, 0xa3, 0x97, 0x8d}
219 };
220
221 /***** FUNCTION DEFINITIONS *****/
222 // XORs the in and out buffers, storing the result in out.
Length is in bytes.
223 void xor_buf(const BYTE in[], BYTE out[], size_t len)
224 {
225     size_t idx;
226     for (idx = 0; idx < len; idx++)
227         out[idx] ^= in[idx];
228 }
229
230 /*****
231 * AES - CBC
232 *****/
233
234 int aes_encrypt_cbc(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])

```

```

235 {
236     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
237     int blocks, idx;
238
239     if (in_len % AES_BLOCK_SIZE != 0)
240         return(FALSE);
241
242     blocks = in_len / AES_BLOCK_SIZE;
243
244     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
245
246     for (idx = 0; idx < blocks; idx++) {
247         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
248         xor_buf(iv_buf, buf_in, AES_BLOCK_SIZE);
249         aes_encrypt(buf_in, buf_out, key, keysize);
250         memcpy(&out[idx * AES_BLOCK_SIZE], buf_out,
AES_BLOCK_SIZE);
251         memcpy(iv_buf, buf_out, AES_BLOCK_SIZE);
252     }
253
254     return(TRUE);
255 }
256
257 int aes_encrypt_cbc_mac(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
258 {
259     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
260     int blocks, idx;
261
262     if (in_len % AES_BLOCK_SIZE != 0)
263         return(FALSE);
264
265     blocks = in_len / AES_BLOCK_SIZE;
266
267     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
268
269     for (idx = 0; idx < blocks; idx++) {
270         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
271         xor_buf(iv_buf, buf_in, AES_BLOCK_SIZE);
272         aes_encrypt(buf_in, buf_out, key, keysize);
273         memcpy(iv_buf, buf_out, AES_BLOCK_SIZE);
274         // Do not output all encrypted blocks.
275     }
276
277     memcpy(out, buf_out, AES_BLOCK_SIZE); // Only output
the last block.
278
279     return(TRUE);
280 }
281
282 int aes_decrypt_cbc(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
283 {
284     BYTE buf_in[AES_BLOCK_SIZE], buf_out[AES_BLOCK_SIZE],
iv_buf[AES_BLOCK_SIZE];
285     int blocks, idx;
286
287     if (in_len % AES_BLOCK_SIZE != 0)
288         return(FALSE);
289
290     blocks = in_len / AES_BLOCK_SIZE;
291
292     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
293
294     for (idx = 0; idx < blocks; idx++) {
295         memcpy(buf_in, &in[idx * AES_BLOCK_SIZE],
AES_BLOCK_SIZE);
296         aes_decrypt(buf_in, buf_out, key, keysize);
297         xor_buf(iv_buf, buf_out, AES_BLOCK_SIZE);
298         memcpy(&out[idx * AES_BLOCK_SIZE], buf_out,
AES_BLOCK_SIZE);
299         memcpy(iv_buf, buf_in, AES_BLOCK_SIZE);
300     }
301
302     return(TRUE);
303 }
304
305 /*****
306 * AES - CTR
307 *****/
308 void increment_iv(BYTE iv[], int counter_size)
309 {
310     int idx;
311
312     // Use counter_size bytes at the end of the IV as the
big-endian integer to increment.
313     for (idx = AES_BLOCK_SIZE - 1; idx >= AES_BLOCK_SIZE -
counter_size; idx--) {
314         iv[idx]++;
315         if (iv[idx] != 0 || idx == AES_BLOCK_SIZE -
counter_size)
316             break;
317     }
318 }
319
320 // Performs the encryption in-place, the input and output buf-
fers may be the same.

```

```

321 // Input may be an arbitrary length (in bytes).
322 void aes_encrypt_ctr(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
323 {
324     size_t idx = 0, last_block_length;
325     BYTE iv_buf[AES_BLOCK_SIZE], out_buf[AES_BLOCK_SIZE];
326
327     if (in != out)
328         memcpy(out, in, in_len);
329
330     memcpy(iv_buf, iv, AES_BLOCK_SIZE);
331     last_block_length = in_len - AES_BLOCK_SIZE;
332
333     if (in_len > AES_BLOCK_SIZE) {
334         for (idx = 0; idx < last_block_length; idx +=
AES_BLOCK_SIZE) {
335             aes_encrypt(iv_buf, out_buf, key, key-
size);
336             xor_buf(out_buf, &out[idx], AES_B-
LOCK_SIZE);
337             increment_iv(iv_buf, AES_BLOCK_SIZE);
338         }
339     }
340
341     aes_encrypt(iv_buf, out_buf, key, keysize);
342
343     xor_buf(out_buf, &out[idx], in_len - idx); // Use the
Most Significant bytes.
344 }
345
346 void aes_decrypt_ctr(const BYTE in[], size_t in_len, BYTE
out[], const WORD key[], int keysize, const BYTE iv[])
347 {
348     // CTR encryption is its own inverse function.
349     aes_encrypt_ctr(in, in_len, out, key, keysize, iv);
350 }
351 /*****
352 * AES - CCM
353 *****/
354 // out_len = payload_len + assoc_len
355 int aes_encrypt_ccm(const BYTE payload[], WORD payload_len,
const BYTE assoc[], unsigned short assoc_len,
const BYTE nonce[], unsigned short
nonce_len, BYTE out[], WORD *out_len,
WORD mac_len, const BYTE key_str[], int
keysize)
356 {
357     BYTE temp_iv[AES_BLOCK_SIZE], counter[AES_BLOCK_SIZE],
mac[16], *buf;
358     int end_of_buf, payload_len_store_size;
359     WORD key[60];
360
361     if (mac_len != 4 && mac_len != 6 && mac_len != 8 &&
mac_len != 10 &&
362         mac_len != 12 && mac_len != 14 && mac_len != 16)
363         return(FALSE);
364
365     if (nonce_len < 7 || nonce_len > 13)
366         return(FALSE);
367
368     if (assoc_len > 32768 /* = 2^15 */)
369         return(FALSE);
370
371     buf = (BYTE*)malloc(payload_len + assoc_len + 48 /
*Round both payload and associated data up a block size and add an
extra block.*);
372     if (! buf)
373         return(FALSE);
374
375     // Prepare the key for usage.
376     aes_key_setup(key_str, key, keysize);
377
378     // Format the first block of the formatted data.
379     payload_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
380     ccm_prepare_first_format_blk(buf, assoc_len, pay-
load_len, payload_len_store_size, mac_len, nonce, nonce_len);
381     end_of_buf = AES_BLOCK_SIZE;
382
383     // Format the Associated Data, aka, assoc[.
384     ccm_format_assoc_data(buf, &end_of_buf, assoc, as-
soc_len);
385
386     // Format the Payload, aka payload[.
387     ccm_format_payload_data(buf, &end_of_buf, payload, pay-
load_len);
388
389     // Create the first counter block.
390     ccm_prepare_first_ctr_blk(counter, nonce, nonce_len,
payload_len_store_size);
391
392     // Perform the CBC operation with an IV of zeros on the
formatted buffer to calculate the MAC.
393     memset(temp_iv, 0, AES_BLOCK_SIZE);
394
395     aes_encrypt_cbc_mac(buf, end_of_buf, mac, key, keysize,
temp_iv);
396
397     // Copy the Payload and MAC to the output buffer.

```

```

399     memcpy(out, payload, payload_len);
400     memcpy(&out[payload_len], mac, mac_len);
401
402     // Encrypt the Payload with CTR mode with a counter
starting at 1.
403     memcpy(temp_iv, counter, AES_BLOCK_SIZE);
404     increment_iv(temp_iv, AES_BLOCK_SIZE - 1 -
mac_len); // Last argument is the byte size of the counting por-
tion of the counter block. /*BUG?*/
405     aes_encrypt_ctr(out, payload_len, out, key, keysize,
temp_iv);
406
407     // Encrypt the MAC with CTR mode with a counter start-
ing at 0.
408     aes_encrypt_ctr(&out[payload_len], mac_len, &out[pay-
load_len], key, keysize, counter);
409
410     free(buf);
411     *out_len = payload_len + mac_len;
412
413     return(TRUE);
414 }
415
416 // plaintext_len = ciphertext_len - mac_len
417 // Needs a flag for whether the MAC matches.
418 int aes_decrypt_ccm(const BYTE ciphertext[], WORD cipher-
text_len, const BYTE assoc[], unsigned short assoc_len,
419 const BYTE nonce[], unsigned short
nonce_len, BYTE plaintext[], WORD *plaintext_len,
420 WORD mac_len, int *mac_auth, const BYTE
key_str[], int keysize)
421 {
422     BYTE temp_iv[AES_BLOCK_SIZE], counter[AES_BLOCK_SIZE],
mac[16], mac_buf[16], *buf;
423     int end_of_buf, plaintext_len_store_size;
424     WORD key[60];
425
426     if (ciphertext_len <= mac_len)
427         return(FALSE);
428
429     buf = (BYTE*)malloc(assoc_len + ciphertext_len /
*ciphertext_len = plaintext_len + mac_len*/ + 48);
430     if (! buf)
431         return(FALSE);
432
433     // Prepare the key for usage.
434     aes_key_setup(key_str, key, keysize);
435
436     // Copy the plaintext and MAC to the output buffers.
437     *plaintext_len = ciphertext_len - mac_len;
438     plaintext_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
439     memcpy(plaintext, ciphertext, *plaintext_len);
440     memcpy(mac, &ciphertext[*plaintext_len], mac_len);
441
442     // Prepare the first counter block for use in decryp-
tion.
443     ccm_prepare_first_ctr_blk(counter, nonce, nonce_len,
plaintext_len_store_size);
444
445     // Decrypt the Payload with CTR mode with a counter
starting at 1.
446     memcpy(temp_iv, counter, AES_BLOCK_SIZE);
447     increment_iv(temp_iv, AES_BLOCK_SIZE - 1 -
mac_len); // (AES_BLOCK_SIZE - 1 - mac_len) is the byte size of
the counting portion of the counter block.
448     aes_decrypt_ctr(plaintext, *plaintext_len, plaintext,
key, keysize, temp_iv);
449
450     // Setting mac_auth to NULL disables the authentication
check.
451     if (mac_auth != NULL) {
452
453         // Decrypt the MAC with CTR mode with a counter
starting at 0.
454         aes_decrypt_ctr(mac, mac_len, mac, key, key-
size, counter);
455
456         // Format the first block of the formatted
data.
457         plaintext_len_store_size = AES_BLOCK_SIZE - 1 -
nonce_len;
458         ccm_prepare_first_format_blk(buf, assoc_len,
*plaintext_len, plaintext_len_store_size, mac_len, nonce,
nonce_len);
459         end_of_buf = AES_BLOCK_SIZE;
460
461         // Format the Associated Data into the authen-
tication buffer.
462         ccm_format_assoc_data(buf, &end_of_buf, assoc,
assoc_len);
463
464         // Format the Payload into the authentication
buffer.
465         ccm_format_payload_data(buf, &end_of_buf,
plaintext, *plaintext_len);
466
467         // Perform the CBC operation with an IV of zer-
os on the formatted buffer to calculate the MAC.

```

```

467     memset(temp_iv, 0, AES_BLOCK_SIZE);
468     aes_encrypt_cbc_mac(buf, end_of_buf, mac_buf,
key, keysize, temp_iv);
469
470     // Compare the calculated MAC against the MAC
embedded in the ciphertext to see if they are the same.
471     if (! memcmp(mac, mac_buf, mac_len)) {
472         *mac_auth = TRUE;
473     }
474     else {
475         *mac_auth = FALSE;
476         memset(plaintext, 0, *plaintext_len);
477     }
478 }
479
480 free(buf);
481
482 return(TRUE);
483 }
484
485 // Creates the first counter block. First byte is flags, then
the nonce, then the incremented part.
486 void ccm_prepare_first_ctr_blk(BYTE counter[], const BYTE
nonce[], int nonce_len, int payload_len_store_size)
487 {
488     memset(counter, 0, AES_BLOCK_SIZE);
489     counter[0] = (payload_len_store_size - 1) & 0x07;
490     memcpy(&counter[1], nonce, nonce_len);
491 }
492
493 void ccm_prepare_first_format_blk(BYTE buf[], int assoc_len,
int payload_len, int payload_len_store_size, int mac_len, const BYTE
nonce[], int nonce_len)
494 {
495     // Set the flags for the first byte of the first block.
496     buf[0] = (((mac_len - 2) / 2) & 0x07) << 3 | ((pay-
load_len_store_size - 1) & 0x07);
497     if (assoc_len > 0)
498         buf[0] += 0x40;
499     // Format the rest of the first block, storing the
nonce and the size of the payload.
500     memcpy(&buf[1], nonce, nonce_len);
501     memset(&buf[1 + nonce_len], 0, AES_BLOCK_SIZE - 1 -
nonce_len);
502     buf[15] = payload_len & 0x000000FF;
503     buf[14] = (payload_len >> 8) & 0x000000FF;
504 }
505
506 void ccm_format_assoc_data(BYTE buf[], int *end_of_buf, const
BYTE assoc[], int assoc_len)
507 {
508     int pad;
509
510     buf[*end_of_buf + 1] = assoc_len & 0x00FF;
511     buf[*end_of_buf] = (assoc_len >> 8) & 0x00FF;
512     *end_of_buf += 2;
513     memcpy(&buf[*end_of_buf], assoc, assoc_len);
514     *end_of_buf += assoc_len;
515     pad = AES_BLOCK_SIZE - (*end_of_buf % AES_B-
LOCK_SIZE); /*BUG?*/
516     memset(&buf[*end_of_buf], 0, pad);
517     *end_of_buf += pad;
518 }
519
520 void ccm_format_payload_data(BYTE buf[], int *end_of_buf,
const BYTE payload[], int payload_len)
521 {
522     int pad;
523
524     memcpy(&buf[*end_of_buf], payload, payload_len);
525     *end_of_buf += payload_len;
526     pad = *end_of_buf % AES_BLOCK_SIZE;
527     if (pad != 0)
528         pad = AES_BLOCK_SIZE - pad;
529     memset(&buf[*end_of_buf], 0, pad);
530     *end_of_buf += pad;
531 }
532
533 /*****
534 * AES
535 *****/
536 // KEY EXPANSION
537 // Substitutes a word using the AES S-Box.
538 WORD SubWord(WORD word)
539 {
540     unsigned int result;
541
542     result = (int)aes_sbox[(word >> 4) & 0x0000000F][word &
0x0000000F];
543     result += (int)aes_sbox[(word >> 12) & 0x0000000F]
[(word >> 8) & 0x0000000F] << 8;
544     result += (int)aes_sbox[(word >> 20) & 0x0000000F]
[(word >> 16) & 0x0000000F] << 16;
545     result += (int)aes_sbox[(word >> 28) & 0x0000000F]
[(word >> 24) & 0x0000000F] << 24;
546     return(result);
547 }

```



```

551 // Performs the action of generating the keys that will be
552 used in every round of
553 // encryption. "key" is the user-supplied input key, "w" is
the output key schedule,
554 // "keysize" is the length in bits of "key", must be 128, 192,
or 256.
555 void aes_key_setup(const BYTE key[], WORD w[], int keysize)
556 {
557     int Nb=Nr,Nk,idx;
558     WORD
temp,Rcon[]={0x01000000,0x02000000,0x04000000,0x08000000,0x10000000,
0x20000000,
559 0x40000000,0x80000000,0x1b000000,0x36000000,0x6c000000,0xd8000000,
560 0xab000000,0x4d000000,0x9a000000};
561
562     switch (keysize) {
563         case 128: Nr = 10; Nk = 4; break;
564         case 192: Nr = 12; Nk = 6; break;
565         case 256: Nr = 14; Nk = 8; break;
566         default: return;
567     }
568
569     for (idx=0; idx < Nk; ++idx) {
570         w[idx] = ((key[4 * idx]) << 24) | ((key[4 * idx +
1]) << 16) |
571                                     ((key[4 * idx + 2]) << 8) |
((key[4 * idx + 3]));
572     }
573
574     for (idx = Nk; idx < Nb * (Nr+1); ++idx) {
575         temp = w[idx - 1];
576         if ((idx % Nk) == 0)
577             temp = SubWord(KE_ROTWORD(temp)) ^
Rcon[(idx-1)/Nk];
578         else if (Nk > 6 && (idx % Nk) == 4)
579             temp = SubWord(temp);
580         w[idx] = w[idx-Nk] ^ temp;
581     }
582 }
583
584 ////////////
585 // ADD ROUND KEY
586 ////////////
587
588 // Performs the AddRoundKey step. Each round has its own pre-generated 16-byte key in the
589 // form of 4 integers (the "w" array). Each integer is XOR'd by one column of the state.
590 // Also performs the job of InvAddRoundKey(); since the function is a simple XOR process,
591 // it is its own inverse.
592 void AddRoundKey(BYTE state[][4], const WORD w[])
593 {
594     BYTE subkey[4];
595
596     // memcpy(subkey,&w[idx],4); // Not accurate for big
endian machines
597     // Subkey 1
598     subkey[0] = w[0] >> 24;
599     subkey[1] = w[0] >> 16;
600     subkey[2] = w[0] >> 8;
601     subkey[3] = w[0];
602     state[0][0] ^= subkey[0];
603     state[1][0] ^= subkey[1];
604     state[2][0] ^= subkey[2];
605     state[3][0] ^= subkey[3];
606     // Subkey 2
607     subkey[0] = w[1] >> 24;
608     subkey[1] = w[1] >> 16;
609     subkey[2] = w[1] >> 8;
610     subkey[3] = w[1];
611     state[0][1] ^= subkey[0];
612     state[1][1] ^= subkey[1];
613     state[2][1] ^= subkey[2];
614     state[3][1] ^= subkey[3];
615     // Subkey 3
616     subkey[0] = w[2] >> 24;
617     subkey[1] = w[2] >> 16;
618     subkey[2] = w[2] >> 8;
619     subkey[3] = w[2];
620     state[0][2] ^= subkey[0];
621     state[1][2] ^= subkey[1];
622     state[2][2] ^= subkey[2];
623     state[3][2] ^= subkey[3];
624     // Subkey 4
625     subkey[0] = w[3] >> 24;
626     subkey[1] = w[3] >> 16;
627     subkey[2] = w[3] >> 8;
628     subkey[3] = w[3];
629     state[0][3] ^= subkey[0];
630     state[1][3] ^= subkey[1];
631     state[2][3] ^= subkey[2];
632     state[3][3] ^= subkey[3];
633 }
634
635 ////////////
636 // (Inv)SubBytes

```

```

637 //////////////////////////////////////////////////
638
639 // Performs the SubBytes step. All bytes in the state are sub-
stituted with a
640 // pre-calculated value from a lookup table.
641 void SubBytes(BYTE state[][4])
642 {
643     state[0][0] = aes_sbox[state[0][0] >> 4][state[0][0] &
0x0F];
644     state[0][1] = aes_sbox[state[0][1] >> 4][state[0][1] &
0x0F];
645     state[0][2] = aes_sbox[state[0][2] >> 4][state[0][2] &
0x0F];
646     state[0][3] = aes_sbox[state[0][3] >> 4][state[0][3] &
0x0F];
647     state[1][0] = aes_sbox[state[1][0] >> 4][state[1][0] &
0x0F];
648     state[1][1] = aes_sbox[state[1][1] >> 4][state[1][1] &
0x0F];
649     state[1][2] = aes_sbox[state[1][2] >> 4][state[1][2] &
0x0F];
650     state[1][3] = aes_sbox[state[1][3] >> 4][state[1][3] &
0x0F];
651     state[2][0] = aes_sbox[state[2][0] >> 4][state[2][0] &
0x0F];
652     state[2][1] = aes_sbox[state[2][1] >> 4][state[2][1] &
0x0F];
653     state[2][2] = aes_sbox[state[2][2] >> 4][state[2][2] &
0x0F];
654     state[2][3] = aes_sbox[state[2][3] >> 4][state[2][3] &
0x0F];
655     state[3][0] = aes_sbox[state[3][0] >> 4][state[3][0] &
0x0F];
656     state[3][1] = aes_sbox[state[3][1] >> 4][state[3][1] &
0x0F];
657     state[3][2] = aes_sbox[state[3][2] >> 4][state[3][2] &
0x0F];
658     state[3][3] = aes_sbox[state[3][3] >> 4][state[3][3] &
0x0F];
659 }
660
661 void InvSubBytes(BYTE state[][4])
662 {
663     state[0][0] = aes_inv_sbox[state[0][0] >> 4][state[0][0] &
0x0F];
664     state[0][1] = aes_inv_sbox[state[0][1] >> 4][state[0][1] &
0x0F];
665     state[0][2] = aes_inv_sbox[state[0][2] >> 4][state[0][2] &
0x0F];
666     state[0][3] = aes_inv_sbox[state[0][3] >> 4][state[0][3] &
0x0F];
667     state[1][0] = aes_inv_sbox[state[1][0] >> 4][state[1][0] &
0x0F];
668     state[1][1] = aes_inv_sbox[state[1][1] >> 4][state[1][1] &
0x0F];
669     state[1][2] = aes_inv_sbox[state[1][2] >> 4][state[1][2] &
0x0F];
670     state[1][3] = aes_inv_sbox[state[1][3] >> 4][state[1][3] &
0x0F];
671     state[2][0] = aes_inv_sbox[state[2][0] >> 4][state[2][0] &
0x0F];
672     state[2][1] = aes_inv_sbox[state[2][1] >> 4][state[2][1] &
0x0F];
673     state[2][2] = aes_inv_sbox[state[2][2] >> 4][state[2][2] &
0x0F];
674     state[2][3] = aes_inv_sbox[state[2][3] >> 4][state[2][3] &
0x0F];
675     state[3][0] = aes_inv_sbox[state[3][0] >> 4][state[3][0] &
0x0F];
676     state[3][1] = aes_inv_sbox[state[3][1] >> 4][state[3][1] &
0x0F];
677     state[3][2] = aes_inv_sbox[state[3][2] >> 4][state[3][2] &
0x0F];
678     state[3][3] = aes_inv_sbox[state[3][3] >> 4][state[3][3] &
0x0F];
679 }
680
681 ////////////////////////////////////
682 // (Inv)ShiftRows
683 ////////////////////////////////////
684
685 // Performs the ShiftRows step. All rows are shifted cyl-
indrically to the left.
686 void ShiftRows(BYTE state[][4])
687 {

```

```

688     int t;
689
690     // Shift left by 1
691     t = state[1][0];
692     state[1][0] = state[1][1];
693     state[1][1] = state[1][2];
694     state[1][2] = state[1][3];
695     state[1][3] = t;
696     // Shift left by 2
697     t = state[2][0];
698     state[2][0] = state[2][2];
699     state[2][2] = t;
700     t = state[2][1];
701     state[2][1] = state[2][3];
702     state[2][3] = t;
703     // Shift left by 3
704     t = state[3][0];
705     state[3][0] = state[3][3];
706     state[3][3] = state[3][2];
707     state[3][2] = state[3][1];
708     state[3][1] = t;
709 }
710
711 // All rows are shifted cylindrically to the right.
712 void InvShiftRows(BYTE state[][4])
713 {
714     int t;
715
716     // Shift right by 1
717     t = state[1][3];
718     state[1][3] = state[1][2];
719     state[1][2] = state[1][1];
720     state[1][1] = state[1][0];
721     state[1][0] = t;
722     // Shift right by 2
723     t = state[2][3];
724     state[2][3] = state[2][1];
725     state[2][1] = t;
726     t = state[2][2];
727     state[2][2] = state[2][0];
728     state[2][0] = t;
729     // Shift right by 3
730     t = state[3][3];
731     state[3][3] = state[3][0];
732     state[3][0] = state[3][1];
733     state[3][1] = state[3][2];
734     state[3][2] = t;
735 }
736
737 ///////////////////////////////////////////////////
738 // (Inv)MixColumns
739 ///////////////////////////////////////////////////
740
741 // Performs the MixColumns step. The state is multiplied by it-
self using matrix
742 // multiplication in a Galios Field 2^8. All multiplication is
pre-computed in a table.
743 // Addition is equivalent to XOR. (Must always make a copy of
the column as the original
744 // values will be destroyed.)
745 void MixColumns(BYTE state[][4])
746 {
747     BYTE col[4];
748
749     // Column 1
750     col[0] = state[0][0];
751     col[1] = state[1][0];
752     col[2] = state[2][0];
753     col[3] = state[3][0];
754     state[0][0] = gf_mul[col[0]][0];
755     state[0][0] ^= gf_mul[col[1]][1];
756     state[0][0] ^= col[2];
757     state[0][0] ^= col[3];
758     state[1][0] = col[0];
759     state[1][0] ^= gf_mul[col[1]][0];
760     state[1][0] ^= gf_mul[col[2]][1];
761     state[1][0] ^= col[3];
762     state[2][0] = col[0];
763     state[2][0] ^= col[1];
764     state[2][0] ^= gf_mul[col[2]][0];
765     state[2][0] ^= gf_mul[col[3]][1];
766     state[3][0] = gf_mul[col[0]][1];
767     state[3][0] ^= col[1];
768     state[3][0] ^= col[2];
769     state[3][0] ^= gf_mul[col[3]][0];
770     // Column 2
771     col[0] = state[0][1];
772     col[1] = state[1][1];
773     col[2] = state[2][1];
774     col[3] = state[3][1];
775     state[0][1] = gf_mul[col[0]][0];
776     state[0][1] ^= gf_mul[col[1]][1];
777     state[0][1] ^= col[2];
778     state[0][1] ^= col[3];
779     state[1][1] = col[0];
780     state[1][1] ^= gf_mul[col[1]][0];
781     state[1][1] ^= gf_mul[col[2]][1];
782     state[1][1] ^= col[3];
783     state[2][1] = col[0];
784     state[2][1] ^= col[1];

```

```

785     state[2][1] ^= gf_mul[col[2]][0];
786     state[2][1] ^= gf_mul[col[3]][1];
787     state[3][1] = gf_mul[col[0]][1];
788     state[3][1] ^= col[1];
789     state[3][1] ^= col[2];
790     state[3][1] ^= gf_mul[col[3]][0];
791     // Column 3
792     col[0] = state[0][2];
793     col[1] = state[1][2];
794     col[2] = state[2][2];
795     col[3] = state[3][2];
796     state[0][2] = gf_mul[col[0]][0];
797     state[0][2] ^= gf_mul[col[1]][1];
798     state[0][2] ^= col[2];
799     state[0][2] ^= col[3];
800     state[1][2] = col[0];
801     state[1][2] ^= gf_mul[col[1]][0];
802     state[1][2] ^= gf_mul[col[2]][1];
803     state[1][2] ^= col[3];
804     state[2][2] = col[0];
805     state[2][2] ^= col[1];
806     state[2][2] ^= gf_mul[col[2]][0];
807     state[2][2] ^= gf_mul[col[3]][1];
808     state[3][2] = gf_mul[col[0]][1];
809     state[3][2] ^= col[1];
810     state[3][2] ^= col[2];
811     state[3][2] ^= gf_mul[col[3]][0];
812     // Column 4
813     col[0] = state[0][3];
814     col[1] = state[1][3];
815     col[2] = state[2][3];
816     col[3] = state[3][3];
817     state[0][3] = gf_mul[col[0]][0];
818     state[0][3] ^= gf_mul[col[1]][1];
819     state[0][3] ^= col[2];
820     state[0][3] ^= col[3];
821     state[1][3] = col[0];
822     state[1][3] ^= gf_mul[col[1]][0];
823     state[1][3] ^= gf_mul[col[2]][1];
824     state[1][3] ^= col[3];
825     state[2][3] = col[0];
826     state[2][3] ^= col[1];
827     state[2][3] ^= gf_mul[col[2]][0];
828     state[2][3] ^= gf_mul[col[3]][1];
829     state[3][3] = gf_mul[col[0]][1];
830     state[3][3] ^= col[1];
831     state[3][3] ^= col[2];
832     state[3][3] ^= gf_mul[col[3]][0];
833 }
834
835 void InvMixColumns(BYTE state[][4])
836 {
837     BYTE col[4];
838
839     // Column 1
840     col[0] = state[0][0];
841     col[1] = state[1][0];
842     col[2] = state[2][0];
843     col[3] = state[3][0];
844     state[0][0] = gf_mul[col[0]][5];
845     state[0][0] ^= gf_mul[col[1]][3];
846     state[0][0] ^= gf_mul[col[2]][4];
847     state[0][0] ^= gf_mul[col[3]][2];
848     state[1][0] = gf_mul[col[0]][2];
849     state[1][0] ^= gf_mul[col[1]][5];
850     state[1][0] ^= gf_mul[col[2]][3];
851     state[1][0] ^= gf_mul[col[3]][4];
852     state[2][0] = gf_mul[col[0]][4];
853     state[2][0] ^= gf_mul[col[1]][2];
854     state[2][0] ^= gf_mul[col[2]][5];
855     state[2][0] ^= gf_mul[col[3]][3];
856     state[3][0] = gf_mul[col[0]][3];
857     state[3][0] ^= gf_mul[col[1]][4];
858     state[3][0] ^= gf_mul[col[2]][2];
859     state[3][0] ^= gf_mul[col[3]][5];
860     // Column 2
861     col[0] = state[0][1];
862     col[1] = state[1][1];
863     col[2] = state[2][1];
864     col[3] = state[3][1];
865     state[0][1] = gf_mul[col[0]][5];
866     state[0][1] ^= gf_mul[col[1]][3];
867     state[0][1] ^= gf_mul[col[2]][4];
868     state[0][1] ^= gf_mul[col[3]][2];
869     state[1][1] = gf_mul[col[0]][2];
870     state[1][1] ^= gf_mul[col[1]][5];
871     state[1][1] ^= gf_mul[col[2]][3];
872     state[1][1] ^= gf_mul[col[3]][4];
873     state[2][1] = gf_mul[col[0]][4];
874     state[2][1] ^= gf_mul[col[1]][2];
875     state[2][1] ^= gf_mul[col[2]][5];
876     state[2][1] ^= gf_mul[col[3]][3];
877     state[3][1] = gf_mul[col[0]][3];
878     state[3][1] ^= gf_mul[col[1]][4];
879     state[3][1] ^= gf_mul[col[2]][2];
880     state[3][1] ^= gf_mul[col[3]][5];
881     // Column 3
882     col[0] = state[0][2];
883     col[1] = state[1][2];
884     col[2] = state[2][2];
885     col[3] = state[3][2];

```



```

886     state[0][2] = gf_mul[col[0]][5];
887     state[0][2] ^= gf_mul[col[1]][3];
888     state[0][2] ^= gf_mul[col[2]][4];
889     state[0][2] ^= gf_mul[col[3]][2];
890     state[1][2] = gf_mul[col[0]][2];
891     state[1][2] ^= gf_mul[col[1]][5];
892     state[1][2] ^= gf_mul[col[2]][3];
893     state[1][2] ^= gf_mul[col[3]][4];
894     state[2][2] = gf_mul[col[0]][4];
895     state[2][2] ^= gf_mul[col[1]][2];
896     state[2][2] ^= gf_mul[col[2]][5];
897     state[2][2] ^= gf_mul[col[3]][3];
898     state[3][2] = gf_mul[col[0]][3];
899     state[3][2] ^= gf_mul[col[1]][4];
900     state[3][2] ^= gf_mul[col[2]][2];
901     state[3][2] ^= gf_mul[col[3]][5];
902     // Column 4
903     col[0] = state[0][3];
904     col[1] = state[1][3];
905     col[2] = state[2][3];
906     col[3] = state[3][3];
907     state[0][3] = gf_mul[col[0]][5];
908     state[0][3] ^= gf_mul[col[1]][3];
909     state[0][3] ^= gf_mul[col[2]][4];
910     state[0][3] ^= gf_mul[col[3]][2];
911     state[1][3] = gf_mul[col[0]][2];
912     state[1][3] ^= gf_mul[col[1]][5];
913     state[1][3] ^= gf_mul[col[2]][3];
914     state[1][3] ^= gf_mul[col[3]][4];
915     state[2][3] = gf_mul[col[0]][4];
916     state[2][3] ^= gf_mul[col[1]][2];
917     state[2][3] ^= gf_mul[col[2]][5];
918     state[2][3] ^= gf_mul[col[3]][3];
919     state[3][3] = gf_mul[col[0]][3];
920     state[3][3] ^= gf_mul[col[1]][4];
921     state[3][3] ^= gf_mul[col[2]][2];
922     state[3][3] ^= gf_mul[col[3]][5];
923 }
924
925 // (En/De)Crypt
926 // (En/De)Crypt
927 // (En/De)Crypt
928
929 void aes_encrypt(const BYTE in[], BYTE out[], const WORD
key[], int keysize)
930 {
931     BYTE state[4][4];
932
933     // Copy input array (should be 16 bytes long) to a mat-
rix (sequential bytes are ordered
934     // by row, not col) called "state" for processing.
935     // *** Implementation note: The official AES documenta-
tion references the state by
936     // column, then row. Accessing an element in C requires
row then column. Thus, all state
937     // references in AES must have the column and row in-
dexes reversed for C implementation.
938     state[0][0] = in[0];
939     state[1][0] = in[1];
940     state[2][0] = in[2];
941     state[3][0] = in[3];
942     state[0][1] = in[4];
943     state[1][1] = in[5];
944     state[2][1] = in[6];
945     state[3][1] = in[7];
946     state[0][2] = in[8];
947     state[1][2] = in[9];
948     state[2][2] = in[10];
949     state[3][2] = in[11];
950     state[0][3] = in[12];
951     state[1][3] = in[13];
952     state[2][3] = in[14];
953     state[3][3] = in[15];
954
955     // Perform the necessary number of rounds. The round
key is added first.
956     // The last round does not perform the MixColumns step.
957     AddRoundKey(state, &key[0]);
958     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[4]);
959     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[8]);
960     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[12]);
961     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[16]);
962     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[20]);
963     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[24]);
964     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[28]);
965     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[32]);
966     SubBytes(state); ShiftRows(state); MixColumns(state);
AddRoundKey(state, &key[36]);
967     if (keysize != 128) {
968         SubBytes(state); ShiftRows(state); Mix-
Columns(state); AddRoundKey(state, &key[40]);
969         SubBytes(state); ShiftRows(state); Mix-

```

```

Columns(state); AddRoundKey(state, &key[44]);
970     if (keysize != 192) {
971         SubBytes(state); ShiftRows(state); Mix-
Columns(state); AddRoundKey(state, &key[48]);
972         SubBytes(state); ShiftRows(state); Mix-
Columns(state); AddRoundKey(state, &key[52]);
973         SubBytes(state); ShiftRows(state); Ad-
dRoundKey(state, &key[56]);
974     }
975     else {
976         SubBytes(state); ShiftRows(state); Ad-
dRoundKey(state, &key[48]);
977     }
978     }
979     else {
980         SubBytes(state); ShiftRows(state); Ad-
dRoundKey(state, &key[40]);
981     }
982
983     // Copy the state to the output array.
984     out[0] = state[0][0];
985     out[1] = state[1][0];
986     out[2] = state[2][0];
987     out[3] = state[3][0];
988     out[4] = state[0][1];
989     out[5] = state[1][1];
990     out[6] = state[2][1];
991     out[7] = state[3][1];
992     out[8] = state[0][2];
993     out[9] = state[1][2];
994     out[10] = state[2][2];
995     out[11] = state[3][2];
996     out[12] = state[0][3];
997     out[13] = state[1][3];
998     out[14] = state[2][3];
999     out[15] = state[3][3];
1000 }
1001
1002 void aes_decrypt(const BYTE in[], BYTE out[], const WORD
key[], int keysize)
1003 {
1004     BYTE state[4][4];
1005
1006     // Copy the input to the state.
1007     state[0][0] = in[0];
1008     state[1][0] = in[1];
1009     state[2][0] = in[2];
1010     state[3][0] = in[3];
1011     state[0][1] = in[4];
1012     state[1][1] = in[5];
1013     state[2][1] = in[6];
1014     state[3][1] = in[7];
1015     state[0][2] = in[8];
1016     state[1][2] = in[9];
1017     state[2][2] = in[10];
1018     state[3][2] = in[11];
1019     state[0][3] = in[12];
1020     state[1][3] = in[13];
1021     state[2][3] = in[14];
1022     state[3][3] = in[15];
1023
1024     // Perform the necessary number of rounds. The round
key is added first.
1025     // The last round does not perform the MixColumns step.
1026     if (keysize > 128) {
1027         if (keysize > 192) {
1028             AddRoundKey(state, &key[56]);
1029
1030             InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[52]);
1031             InvMixColumns(state);
1032         }
1033         else {
1034             AddRoundKey(state, &key[48]);
1035
1036             InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[44]);
1037             InvMixColumns(state);
1038         }
1039     }
1040     else {
1041         AddRoundKey(state, &key[40]);
1042
1043         InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[36]);
1044         InvMixColumns(state);
1045     }
1046     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[32]);
1047     InvMixColumns(state);
1048     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[28]);
1049     InvMixColumns(state);
1050     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[24]);
1051     InvMixColumns(state);
1052     InvShiftRows(state); InvSubBytes(state); AddRoundKey(state, &key[20]);

```

```

InvMixColumns(state);
1046 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[16]);
InvMixColumns(state);
1047 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[12]);
InvMixColumns(state);
1048 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[8]);I
nvMixColumns(state);
1049 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[4]);I
nvMixColumns(state);
1050 InvShiftRows(state);InvSubBytes(state);AddRoundKey(state,&key[0]);
1051
1052 // Copy the state to the output array.
1053 out[0] = state[0][0];
1054 out[1] = state[1][0];
1055 out[2] = state[2][0];
1056 out[3] = state[3][0];
1057 out[4] = state[0][1];
1058 out[5] = state[1][1];
1059 out[6] = state[2][1];
1060 out[7] = state[3][1];
1061 out[8] = state[0][2];
1062 out[9] = state[1][2];
1063 out[10] = state[2][2];
1064 out[11] = state[3][2];
1065 out[12] = state[0][3];
1066 out[13] = state[1][3];
1067 out[14] = state[2][3];
1068 out[15] = state[3][3];
1069 }
1070
1071 /*****
1072 ** AES DEBUGGING FUNCTIONS
1073 *****/
1074 /*
1075 // This prints the "state" grid as a linear hex string.
1076 void print_state(BYTE state[][4])
1077 {
1078     int idx,idx2;
1079
1080     for (idx=0; idx < 4; idx++)
1081         for (idx2=0; idx2 < 4; idx2++)
1082             printf("%02x",state[idx2][idx]);
1083     printf("\n");
1084 }
1085
1086 // This prints the key (4 consecutive ints) used for a given
1087 // round as a linear hex string.
1088 void print_rnd_key(WORD key[])
1089 {
1090     int idx;
1091
1092     for (idx=0; idx < 4; idx++)
1093         printf("%08x",key[idx]);
1094     printf("\n");
1095 }
1096 */

```

2. B-Con_crypto-algorithms/aes.h (123 lines)

```

1 /
2 *
3 * Filename: aes.h
4 * Author: Brad Conte (brad AT bradconte.com)
5 * Copyright:
6 * Disclaimer: This code is presented "as is" without any guar-
7 * antees.
8 * Details: Defines the API for the corresponding AES imple-
9 * mentation.
10 *
11 *****/
12
13 #ifndef AES_H
14 #define AES_H
15
16 /***** HEADER FILES *****/
17 #include <stdint.h>
18
19 /***** MACROS *****/
20 #define AES_BLOCK_SIZE 16 // AES operates on 16
21 // bytes at a time
22
23 /***** DATA TYPES *****/
24 typedef unsigned char BYTE; // 8-bit byte
25 typedef unsigned int WORD; // 32-bit word, change
26 // to "long" for 16-bit machines
27
28 /***** FUNCTION DECLARATIONS *****/
29
30 // AES

```

```

25 // Key setup must be done before any AES en/de-cryption func-
26 // tions can be used.
27 void aes_key_setup(const BYTE key[], // The key, must be
28 // 128, 192, or 256 bits
29 // Output key
30 // schedule to be used later
31 int keysz); // Bit length of
32 // the key, 128, 192, or 256
33 void aes_encrypt(const BYTE in[], // 16 bytes of
34 // plaintext
35 // 16 bytes of
36 // ciphertext
37 // const WORD key[], // From the key
38 // setup
39 // int keysz); // Bit length of
40 // the key, 128, 192, or 256
41 void aes_decrypt(const BYTE in[], // 16 bytes of
42 // ciphertext
43 // 16 bytes of
44 // plaintext
45 // const WORD key[], // From the key
46 // setup
47 // int keysz); // Bit length of
48 // the key, 128, 192, or 256
49 void aes_encrypt_cbc(const BYTE in[], // Plaintext
50 // size_t in_len, // Must be a
51 // multiple of AES_BLOCK_SIZE
52 // 16 bytes of
53 // ciphertext,
54 // const WORD key[], // From the key
55 // setup
56 // int keysz, // Bit length of
57 // the key, 128, 192, or 256
58 // const BYTE iv[], // IV, must be
59 // AES_BLOCK_SIZE bytes long
60 // Only output the CBC-MAC of the input.
61 // Plaintext
62 // size_t in_len, // Must be a
63 // multiple of AES_BLOCK_SIZE
64 // 16 bytes of
65 // Output MAC
66 // const WORD key[], // From the key
67 // setup
68 // int keysz, // Bit length of
69 // the key, 128, 192, or 256
70 // const BYTE iv[], // IV, must be
71 // AES_BLOCK_SIZE bytes long
72 // void increment_iv(BYTE iv[], // Must be a
73 // multiple of AES_BLOCK_SIZE
74 // int counter_size); // Bytes of the
75 // IV used for counting (low end)
76 void aes_encrypt_ctr(const BYTE in[], // Plaintext
77 // size_t in_len, // Any byte
78 // length
79 // 16 bytes of
80 // ciphertext,
81 // const WORD key[], // From the key
82 // setup
83 // int keysz, // Bit length of
84 // the key, 128, 192, or 256
85 // const BYTE iv[], // IV, must be
86 // AES_BLOCK_SIZE bytes long
87 // void aes_decrypt_ctr(const BYTE in[], // Ciphertext
88 // size_t in_len, // Any byte
89 // length
90 // 16 bytes of
91 // plaintext,
92 // const WORD key[], // From the key
93 // setup
94 // int keysz, // Bit length of
95 // the key, 128, 192, or 256
96 // const BYTE iv[], // IV, must be
97 // AES_BLOCK_SIZE bytes long
98 // Returns True if the input parameters do not violate any
99 // constraint.
100 // int aes_encrypt_ccm(const BYTE plaintext[], //

```

```

IN - Plaintext.
84 WORD plaintext_len, //
IN - Plaintext length.
85 const BYTE associated_data[], //
IN - Associated Data included in authentication, but not encryption.
86 unsigned short associated_data_len, //
IN - Associated Data length in bytes.
87 const BYTE nonce[], //
IN - The Nonce to be used for encryption.
88 unsigned short nonce_len, //
IN - Nonce length in bytes.
89 BYTE ciphertext[], //
OUT - Ciphertext, a concatenation of the plaintext and the MAC.
90 WORD *ciphertext_len, //
OUT - The length of the ciphertext, always plaintext_len + mac_len.
91 WORD mac_len, //
IN - The desired length of the MAC, must be 4, 6, 8, 10, 12, 14, or 16.
92 const BYTE key[], //
IN - The AES key for encryption.
93 int keysize; //
IN - The length of the key in bits. Valid values are 128, 192, 256.
94 // Returns True if the input parameters do not violate any constraint.
95 // Use mac_auth to ensure decryption/validation was preformed correctly.
96 // If authentication does not succeed, the plaintext is zeroed out. To overwrite
97 // this, call with mac_auth = NULL. The proper procedure is to decrypt with
98 // authentication enabled (mac_auth != NULL) and make a second call to that
99 // ignores authentication explicitly if the first call failes.
100 int aes_decrypt_ccm(const BYTE ciphertext[], //
IN - Ciphertext, the concatenation of encrypted plaintext and MAC.
101 WORD ciphertext_len, //
IN - Ciphertext length in bytes.
102 const BYTE assoc[], //
IN - The Associated Data, required for authentication.
103 unsigned short assoc_len, //
IN - Associated Data length in bytes.
104 const BYTE nonce[], //
IN - The Nonce to use for decryption, same one as for encryption.
105 unsigned short nonce_len, //
IN - Nonce length in bytes.
106 BYTE plaintext[], //
OUT - The plaintext that was decrypted. Will need to be large enough to hold ciphertext_len - mac_len.
107 WORD *plaintext_len, //
OUT - Length in bytes of the output plaintext, always ciphertext_len - mac_len.
108 WORD mac_len, //
IN - The length of the MAC that was calculated.
109 int *mac_auth, //
OUT - TRUE if authentication succeeded, FALSE if it did not. NULL pointer will ignore the authentication.
110 const BYTE key[], //
IN - The AES key for decryption.
111 int keysize; //
IN - The length of the key in BITS. Valid values are 128, 192, 256.
112
113 // Test functions
114 // Test functions
115 int aes_test();
116 int aes_ecb_test();
117 int aes_cbc_test();
118 int aes_ctr_test();
119 int aes_ccm_test();
120
121 #endif // AES_H

```

3. B-Con_crypto-algorithms/aes_test.c (285 lines)

```

1 /
*****
2 * Filename:   aes_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guarantees.
6 * Details:   Performs known-answer tests on the corresponding AES
7             implementation. These tests do not encompass the
8             full range of available test vectors and are not sufficient
9             for FIPS-140 certification. However, if the
10            tests pass it is very, very likely that the code is correct
11            and was compiled properly. This code also serves as
12            example usage of the functions.

```

```

13
*****
14
15 /***** HEADER FILES *****/
16 #include <stdio.h>
17 #include <memory.h>
18 #include "aes.h"
19
20 /***** FUNCTION DEFINITIONS *****/
21 void print_hex(BYTE str[], int len)
22 {
23     int idx;
24     for(idx = 0; idx < len; idx++)
25         printf("%02x", str[idx]);
26 }
27
28 int aes_ecb_test()
29 {
30     WORD key_schedule[60], idx;
31     BYTE enc_buf[128];
32     BYTE plaintext[2][16] = {
33         {0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f,
34          0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a},
35         {0xae, 0x2d, 0x8a, 0x57, 0x1e, 0x03, 0xac, 0x9c, 0x9e,
36          0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51}
37     };
38     BYTE ciphertext[2][16] = {
39         {0xf3, 0xee, 0xd1, 0xbd, 0xb5, 0xd2, 0xa0, 0x3c,
40          0x06, 0x4b, 0x5a, 0x7e, 0x3d, 0xb1, 0x81, 0xf8},
41         {0x59, 0x1c, 0xcb, 0x10, 0xd4, 0x10, 0xed, 0x26, 0xdc,
42          0x5b, 0xa7, 0x4a, 0x31, 0x36, 0x28, 0x70}
43     };
44     int pass = 1;
45     // Raw ECB mode.
46     //printf(" ECB mode:\n");
47     aes_key_setup(key[0], key_schedule, 256);
48     //printf(" Key : ");
49     //print_hex(key[0], 32);
50
51     for(idx = 0; idx < 2; idx++) {
52         aes_encrypt(plaintext[idx], enc_buf, key_schedule, 256);
53         //printf("\nPlaintext : ");
54         //print_hex(plaintext[idx], 16);
55         //printf("\n-n-encrypted to: ");
56         //print_hex(enc_buf, 16);
57         pass = pass && !memcmp(enc_buf, ciphertext[idx], 16);
58     }
59     aes_decrypt(ciphertext[idx], enc_buf, key_schedule, 256);
60     //printf("\nCiphertext : ");
61     //print_hex(ciphertext[idx], 16);
62     //printf("\n-n-decrypted to: ");
63     //print_hex(enc_buf, 16);
64
65     pass = pass && !memcmp(enc_buf, plaintext[idx], 16);
66
67     //printf("\n\n");
68 }
69
70 return(pass);
71 }
72
73 int aes_cbc_test()
74 {
75     WORD key_schedule[60];
76     BYTE enc_buf[128];
77     BYTE plaintext[1][32] = {
78         {0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f,
79          0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a, 0xae, 0x2d, 0x8a,
80          0x57, 0x1e, 0x03, 0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51}
81     };
82     BYTE ciphertext[1][32] = {
83         {0xf5, 0x8c, 0x4c, 0x04, 0xd6, 0xe5, 0xf1, 0xba,
84          0x77, 0x9e, 0xab, 0xfb, 0x5f, 0x7b, 0xfb, 0xd6, 0x9c, 0xfc, 0x4e, 0x96, 0x7e,
85          0xdb, 0x80, 0x8d, 0x67, 0x9f, 0x77, 0x7b, 0xc6, 0x70, 0x2c, 0x7d}
86     };
87     BYTE iv[1][16] = {
88         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c,
89          0xd0, 0xe0, 0xf0}
90     };
91     BYTE key[1][32] = {
92         {0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca, 0x71, 0xbe, 0x2b,
93          0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81, 0x1f, 0x35, 0x2c, 0x07, 0x3b,
94          0x61, 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4}
95     };
96     int pass = 1;

```

```

91 //printf(" CBC mode:\n");
92 aes_key_setup(key[0], key_schedule, 256);
93
94 //printf("Key : ");
95 //print_hex(key[0], 32);
96 //printf("\nIV : ");
97 //print_hex(iv[0], 16);
98
99 aes_encrypt_cbc(plaintext[0], 32, enc_buf, key_schedule, 256, iv[0]);
100 //printf("\nPlaintext : ");
101 //print_hex(plaintext[0], 32);
102 //printf("\n-n-encrypted to: ");
103 //print_hex(enc_buf, 32);
104 //printf("\nCiphertext : ");
105 //print_hex(ciphertext[0], 32);
106 pass = pass && !memcmp(enc_buf, ciphertext[0], 32);
107
108 aes_decrypt_cbc(ciphertext[0], 32, enc_buf, key_schedule, 256, iv[0]);
109 //printf("\nCiphertext : ");
110 //print_hex(ciphertext[0], 32);
111 //printf("\n-decrypt to: ");
112 //print_hex(enc_buf, 32);
113 //printf("\nPlaintext : ");
114 //print_hex(plaintext[0], 32);
115 pass = pass && !memcmp(enc_buf, plaintext[0], 32);
116
117 //printf("\n\n");
118 return(pass);
119 }
120
121 int aes_ctr_test()
122 {
123     WORD key_schedule[60];
124     BYTE enc_buf[128];
125     BYTE plaintext[1][32] = {
126         {0x6b, 0xc1, 0xbe, 0xe2, 0x2e, 0x40, 0x9f,
0x96, 0xe9, 0x3d, 0x7e, 0x11, 0x73, 0x93, 0x17, 0x2a, 0xae, 0x2d, 0x8a,
0x57, 0x1e, 0x03, 0xac, 0x9c, 0x9e, 0xb7, 0x6f, 0xac, 0x45, 0xaf, 0x8e, 0x51}
127     };
128     BYTE ciphertext[1][32] = {
129         {0x60, 0x1e,
0xc3, 0x13, 0x77, 0x57, 0x89, 0xa5, 0xb7, 0xa7, 0xf5, 0x04, 0xbb,
0xf3, 0xd2, 0x28, 0xf4, 0x43, 0xe3, 0xca, 0x4d, 0x62, 0xb5, 0x9a, 0xca,
0x84, 0xe9, 0x90, 0xca, 0xca, 0xf5, 0xc5}
130     };
131     BYTE iv[1][16] = {
132         {0xf0, 0xf1, 0xf2, 0xf3, 0xf4, 0xf5, 0xf6, 0xf7, 0xf8, 0xf9, 0xfa, 0xfb, 0xfc,
0xfd, 0xfe, 0xff},
133     };
134     BYTE key[1][32] = {
135         {0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca, 0x71, 0xbe, 0x2b,
0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81, 0x1f, 0x35, 0x2c, 0x07, 0x3b,
0x61, 0x08, 0xd7, 0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4}
136     };
137     int pass = 1;
138
139     //printf(" CTR mode:\n");
140     aes_key_setup(key[0], key_schedule, 256);
141
142     //printf("Key : ");
143     //print_hex(key[0], 32);
144     //printf("\nIV : ");
145     //print_hex(iv[0], 16);
146
147     aes_encrypt_ctr(plaintext[0], 32, enc_buf, key_schedule, 256, iv[0]);
148     //printf("\nPlaintext : ");
149     //print_hex(plaintext[0], 32);
150     //printf("\n-n-encrypted to: ");
151     //print_hex(enc_buf, 32);
152     pass = pass && !memcmp(enc_buf, ciphertext[0], 32);
153
154     aes_decrypt_ctr(ciphertext[0], 32, enc_buf, key_schedule, 256, iv[0]);
155     //printf("\nCiphertext : ");
156     //print_hex(ciphertext[0], 32);
157     //printf("\n-decrypt to: ");
158     //print_hex(enc_buf, 32);
159     pass = pass && !memcmp(enc_buf, plaintext[0], 32);
160
161     //printf("\n\n");
162     return(pass);
163 }
164
165 int aes_ccm_test()
166 {
167     int mac_auth;
168     WORD enc_buf_len;
169     BYTE enc_buf[128];
170     BYTE plaintext[3][32] = {
171         {0x20, 0x21, 0x22, 0x23},
172         {0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28, 0x29, 0x2a, 0x2b, 0x2c,
0x2d, 0x2e, 0x2f},
173         {0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28, 0x29, 0x2a, 0x2b, 0x2c,
0x2d, 0x2e, 0x2f, 0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37}
174     };

```

```

175     BYTE assoc[3][32] = {
176         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
0x08, 0x09, 0x0a, 0x0b, 0x0c,
0x0d, 0x0e, 0x0f},
177         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c,
0x0d, 0x0e, 0x0f},
178         {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0a, 0x0b, 0x0c,
0x0d, 0x0e, 0x0f, 0x10, 0x11, 0x12, 0x13}
179     };
180     BYTE ciphertext[3][32 + 16] = {
181         {0x71, 0x62, 0x01, 0x5b, 0x4d, 0xac, 0x25, 0x5d},
182         {0xd2, 0xa1, 0xf0, 0xe0, 0x51, 0xea, 0x5f,
0x62, 0x08, 0x1a, 0x77, 0x92, 0x07, 0x3d, 0x59, 0x3d, 0x1f, 0xc6, 0x4f, 0xbf,
0xac, 0xcd},
183         {0xe3, 0xb2, 0x01, 0xa9, 0xf5, 0xb7, 0x1a, 0x7a, 0x9b,
0x1c, 0xea, 0xec, 0xcd, 0x97, 0xe7, 0x0b, 0x61, 0x76, 0xaa,
0xd9, 0xa4, 0x42, 0x8a, 0xa5, 0x48, 0x43, 0x92, 0xfb, 0xc1, 0xb0, 0x99, 0x51}
184     };
185     BYTE iv[3][16] = {
186         {0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16},
187         {0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17},
188         {0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1a, 0x1b}
189     };
190     BYTE key[1][32] = {
191         {0x40, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46, 0x47, 0x48, 0x49, 0x4a, 0x4b, 0x4c,
0x4d, 0x4e, 0x4f}
192     };
193     int pass = 1;
194
195     //printf(" CCM mode:\n");
196     //printf("Key : ");
197     //print_hex(key[0], 16);
198
199     //print_hex(plaintext[0], 4);
200     //print_hex(assoc[0], 8);
201     //print_hex(ciphertext[0], 8);
202     //print_hex(iv[0], 7);
203     //print_hex(key[0], 16);
204
205     aes_encrypt_ccm(plaintext[0], 4, assoc[0], 8, iv[0], 7,
enc_buf, &enc_buf_len, 4, key[0], 128);
206     //printf("\nNONCE : ");
207     //print_hex(iv[0], 7);
208     //printf("\nAssoc. Data : ");
209     //print_hex(assoc[0], 8);
210     //printf("\nPayload : ");
211     //print_hex(plaintext[0], 4);
212     //printf("\n-n-encrypted to: ");
213     //print_hex(enc_buf, enc_buf_len);
214     pass = pass && !memcmp(enc_buf, ciphertext[0],
enc_buf_len);
215
216     aes_decrypt_ccm(ciphertext[0], 8, assoc[0], 8, iv[0],
7, enc_buf, &enc_buf_len, 4, &mac_auth, key[0], 128);
217     //printf("\n-Ciphertext : ");
218     //print_hex(ciphertext[0], 8);
219     //printf("\n-decrypt to: ");
220     //print_hex(enc_buf, enc_buf_len);
221     //printf("\nAuthenticated: %d ", mac_auth);
222     pass = pass && !memcmp(enc_buf, plaintext[0],
enc_buf_len) && mac_auth;
223
224
225     aes_encrypt_ccm(plaintext[1], 16, assoc[1], 16, iv[1],
8, enc_buf, &enc_buf_len, 6, key[0], 128);
226     //printf("\n\nNONCE : ");
227     //print_hex(iv[1], 8);
228     //printf("\nAssoc. Data : ");
229     //print_hex(assoc[1], 16);
230     //printf("\nPayload : ");
231     //print_hex(plaintext[1], 16);
232     //printf("\n-n-encrypted to: ");
233     //print_hex(enc_buf, enc_buf_len);
234     pass = pass && !memcmp(enc_buf, ciphertext[1],
enc_buf_len);
235
236     aes_decrypt_ccm(ciphertext[1], 22, assoc[1], 16, iv[1],
8, enc_buf, &enc_buf_len, 6, &mac_auth, key[0], 128);
237     //printf("\n-Ciphertext : ");
238     //print_hex(ciphertext[1], 22);
239     //printf("\n-decrypt to: ");
240     //print_hex(enc_buf, enc_buf_len);
241     //printf("\nAuthenticated: %d ", mac_auth);
242     pass = pass && !memcmp(enc_buf, plaintext[1],
enc_buf_len) && mac_auth;
243
244
245     aes_encrypt_ccm(plaintext[2], 24, assoc[2], 20, iv[2],
12, enc_buf, &enc_buf_len, 8, key[0], 128);
246     //printf("\n\nNONCE : ");
247     //print_hex(iv[2], 12);
248     //printf("\nAssoc. Data : ");
249     //print_hex(assoc[2], 20);
250     //printf("\nPayload : ");
251     //print_hex(plaintext[2], 24);
252     //printf("\n-n-encrypted to: ");
253     //print_hex(enc_buf, enc_buf_len);

```

```

254     pass = pass && !memcmp(enc_buf, ciphertext[2],
enc_buf_len);
255
256     aes_decrypt_ccm(ciphertext[2], 32, assoc[2], 20, iv[2],
12, enc_buf, &enc_buf_len, 8, &mac_auth, key[0], 128);
257     //printf("\n-Ciphertext : ");
258     //print_hex(ciphertext[2], 32);
259     //printf("\n-decrypted to: ");
260     //print_hex(enc_buf, enc_buf_len);
261     //printf("\nAuthenticated: %d ", mac_auth);
262     pass = pass && !memcmp(enc_buf, plaintext[2],
enc_buf_len) && mac_auth;
263
264     //printf("\n\n");
265     return(pass);
266 }
267
268 int aes_test()
269 {
270     int pass = 1;
271
272     pass = pass && aes_ecb_test();
273     pass = pass && aes_cbc_test();
274     pass = pass && aes_ctr_test();
275     pass = pass && aes_ccm_test();
276
277     return(pass);
278 }
279
280 int main(int argc, char *argv[])
281 {
282     printf("AES Tests: %s\n", aes_test() ? "SUCCEEDED" :
"FAILED");
283
284     return(0);
285 }

```

4. B-Con_crypto-algorithms/arcfour.c (47 lines)

```

1 /
*****
2 * Filename:   arcfour.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the ARCFOUR encryption al-
gorithm.
7             Algorithm specification can be found here:
8             * http://en.wikipedia.org/wiki/RC4
9
*****
*/
10
11 /***** HEADER FILES
*****
12 #include <stdlib.h>
13 #include "arcfour.h"
14
15 /***** FUNCTION DEFINITIONS
*****
16 void arcfour_key_setup(BYTE state[], const BYTE key[], int
len)
17 {
18     int i, j;
19     BYTE t;
20
21     for (i = 0; i < 256; ++i)
22         state[i] = i;
23     for (i = 0, j = 0; i < 256; ++i) {
24         j = (j + state[i] + key[i % len]) % 256;
25         t = state[i];
26         state[i] = state[j];
27         state[j] = t;
28     }
29 }
30
31 // This does not hold state between calls. It always generates
the
32 // stream starting from the first output byte.
33 void arcfour_generate_stream(BYTE state[], BYTE out[], size_t
len)
34 {
35     int i, j;
36     size_t idx;
37     BYTE t;
38
39     for (idx = 0, i = 0, j = 0; idx < len; ++idx) {
40         i = (i + 1) % 256;
41         j = (j + state[i]) % 256;
42         t = state[i];
43         state[i] = state[j];
44         state[j] = t;
45         out[idx] = state[(state[i] + state[j]) % 256];
46     }
47 }

```

5. B-Con_crypto-algorithms/arcfour.h (30 lines)

```

1 /
*****
2 * Filename:   arcfour.h
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Defines the API for the corresponding ARCFOUR
implementation.
7
*****
*/
8
9 #ifndef ARCFOUR_H
10 #define ARCFOUR_H
11
12 /***** HEADER FILES
*****
13 #include <stddef.h>
14
15 /***** DATA TYPES
*****
16 typedef unsigned char BYTE;           // 8-bit byte
17
18 /***** FUNCTION DECLARATIONS
*****
19 // Input: state - the state used to generate the keystream
20 //          key - Key to use to initialize the state
21 //          len - length of key in bytes (valid len is 1 to
256)
22 void arcfour_key_setup(BYTE state[], const BYTE key[], int
len);
23
24 // Pseudo-Random Generator Algorithm
25 // Input: state - the state used to generate the keystream
26 //          out - Must be allocated to be of at least "len"
length
27 //          len - number of bytes to generate
28 void arcfour_generate_stream(BYTE state[], BYTE out[], size_t
len);
29
30 #endif // ARCFOUR_H

```

6. B-Con_crypto-algorithms/arcfour_test.c (47 lines)

```

1 /
*****
2 * Filename:   arcfour_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Performs known-answer tests on the corresponding AR-
CFOUR
7             implementation. These tests do not encompass
the full
8             range of available test vectors, however, if
the tests
9
10             pass it is very, very likely that the code is
correct
11             and was compiled properly. This code also
serves as
12             example usage of the functions.
13
*****
14 /***** HEADER FILES
*****
15 #include <stdio.h>
16 #include <memory.h>
17 #include "arcfour.h"
18
19 /***** FUNCTION DEFINITIONS
*****
20 int rc4_test()
21 {
22     BYTE state[256];
23     BYTE key[3][10] = {{ "Key", "Wiki", "Secret" }};
24     BYTE stream[3][10] = {{ 0xEB, 0x9F,
0x77, 0x81, 0xB7, 0x34, 0xCA, 0x72, 0xA7, 0x19 },
25                             { 0x60, 0x44, 0xdb, 0x6d, 0x41, 0xb7 },
26                             { 0x04, 0xd4, 0x6b, 0x05, 0x3c,
0xa8, 0x7b, 0x59 }};
27     int stream_len[3] = {10, 6, 8};
28     BYTE buf[1024];
29     int idx;
30     int pass = 1;
31
32     // Only test the output stream. Note that the state can be
reused.
33     for (idx = 0; idx < 3; idx++) {

```



```

34         arcfour_key_setup(state, key[idx],
strlen(key[idx]));
35         arcfour_generate_stream(state, buf,
stream_len[idx]);
36         pass = pass && !memcmp(stream[idx], buf,
stream_len[idx]);
37     }
38 }
39     return(pass);
40 }
41
42 int main()
43 {
44     printf("ARCFOUR tests: %s\n", rc4_test() ? "SUC-
CEDED" : "FAILED");
45
46     return(0);
47 }

```

7. B-Con_crypto-algorithms/base64.c (135 lines)

```

1 /
*****
2 * Filename:   base64.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the Base64 encoding algorithm.
7
*****
8
9 /***** HEADER FILES *****/
10 #include <stdlib.h>
11 #include "base64.h"
12
13 /***** MACROS *****/
14 #define NEWLINE_INVL 76
15
16 /***** VARIABLES *****/
17 // Note: To change the charset to a URL encoding, replace the
'+ and '/' with '*' and '-'
18 static const BYTE char-
set[]={ "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"};
19
20 /***** FUNCTION DEFINITIONS *****/
21 BYTE revchar(char ch)
22 {
23     if (ch >= 'A' && ch <= 'Z')
24         ch -= 'A';
25     else if (ch >= 'a' && ch <= 'z')
26         ch = ch - 'a' + 26;
27     else if (ch >= '0' && ch <= '9')
28         ch = ch - '0' + 52;
29     else if (ch == '+')
30         ch = 62;
31     else if (ch == '/')
32         ch = 63;
33
34     return(ch);
35 }
36
37 size_t base64_encode(const BYTE in[], BYTE out[], size_t len,
int newline_flag)
38 {
39     size_t idx, idx2, blks, blk_ceiling, left_over,
newline_count = 0;
40
41     blks = (len / 3);
42     left_over = len % 3;
43
44     if (out == NULL) {
45         idx2 = blks * 4;
46         if (left_over)
47             idx2 += 4;
48         if (newline_flag)
49             idx2 += len / 57; // (NEWLINE_INVL /
4) * 3 = 57. One newline per 57 input bytes.
50     }
51     else {
52         // Since 3 input bytes = 4 output bytes, de-
termine out how many even sets of
53         // 3 bytes the input has.
54         blk_ceiling = blks * 3;
55         for (idx = 0, idx2 = 0; idx < blk_ceiling; idx
+= 3, idx2 += 4) {
56             out[idx2] = charset[in[idx] >> 2];
57             out[idx2 + 1] = charset[(in[idx] &
0x03) << 4] | (in[idx + 1] >> 4)];
58
59             out[idx2 + 2] = charset[(((in[idx + 1] &
0x0f) << 2) | (in[idx + 2] >> 6))];
60             out[idx2 + 3] = charset[in[idx + 2] &

```

```

0x3f];
60             // The official standard requires a
newline every 76 characters.
61             // (Eg, first newline is character 77
of the output.)
62             if (((idx2 - newline_count + 4) %
NEWLINE_INVL == 0) && newline_flag) {
63                 out[idx2 + 4] = '\n';
64                 idx2++;
65                 newline_count++;
66             }
67         }
68
69         if (left_over == 1) {
70             out[idx2] = charset[in[idx] >> 2];
71             out[idx2 + 1] = charset[(in[idx] &
0x03) << 4];
72             out[idx2 + 2] = '=';
73             out[idx2 + 3] = '=';
74             idx2 += 4;
75         }
76         else if (left_over == 2) {
77             out[idx2] = charset[in[idx] >> 2];
78             out[idx2 + 1] = charset[(in[idx] &
0x03) << 4] | (in[idx + 1] >> 4)];
79             out[idx2 + 2] = charset[(in[idx + 1] &
0x0f) << 2];
80             out[idx2 + 3] = '=';
81             idx2 += 4;
82         }
83     }
84
85     return(idx2);
86 }
87
88 size_t base64_decode(const BYTE in[], BYTE out[], size_t len)
89 {
90     BYTE ch;
91     size_t idx, idx2, blks, blk_ceiling, left_over;
92
93     if (in[len - 1] == '=')
94         len--;
95     if (in[len - 1] == '=')
96         len--;
97
98     blks = len / 4;
99     left_over = len % 4;
100
101     if (out == NULL) {
102         if (len >= 77 && in[NEWLINE_INVL] == '\n') //
Verify that newlines where used.
103             len -= len / (NEWLINE_INVL + 1);
104         blks = len / 4;
105         left_over = len % 4;
106
107         idx = blks * 3;
108         if (left_over == 2)
109             idx++;
110         else if (left_over == 3)
111             idx += 2;
112     }
113     else {
114         blk_ceiling = blks * 4;
115
116         for (idx = 0, idx2 = 0; idx2 < blk_ceiling; idx
+= 3, idx2 += 4) {
117             if (in[idx2] == '\n')
118                 idx2++;
119
120             out[idx] = (revchar(in[idx2]) << 2) |
((revchar(in[idx2 + 1]) & 0x30) >> 4);
121             out[idx + 1] = (revchar(in[idx2 + 1])
<< 4) | (revchar(in[idx2 + 2]) >> 2);
122             out[idx + 2] = (revchar(in[idx2 + 2])
<< 6) | revchar(in[idx2 + 3]);
123         }
124
125         if (left_over == 2) {
126             out[idx] = (revchar(in[idx2]) << 2) |
((revchar(in[idx2 + 1]) & 0x30) >> 4);
127             idx++;
128         }
129         else if (left_over == 3) {
130             out[idx] = (revchar(in[idx2]) << 2) |
((revchar(in[idx2 + 1]) & 0x30) >> 4);
131             out[idx + 1] = (revchar(in[idx2 + 1])
<< 4) | (revchar(in[idx2 + 2]) >> 2);
132             idx += 2;
133         }
134     }
135
136     return(idx);
137 }

```

8. B-Con_crypto-algorithms/base64.h (27 lines)

```

1 /
*****
2 * Filename:   base64.h
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Defines the API for the corresponding Base64 im-
plementation.
7
*****
*/
8
9 #ifndef BASE64_H
10 #define BASE64_H
11
12 /***** HEADER FILES *****/
13 #include <stddef.h>
14
15 /***** DATA TYPES *****/
16 typedef unsigned char BYTE;          // 8-bit byte
17
18 /***** FUNCTION DECLARATIONS *****/
19 // Returns the size of the output. If called with out = NULL,
will just return
20 // the size of what the output would have been (without a ter-
minating NULL).
21 size_t base64_encode(const BYTE in[], BYTE out[], size_t len,
int newline_flag);
22
23 // Returns the size of the output. If called with out = NULL,
will just return
24 // the size of what the output would have been (without a ter-
minating NULL).
25 size_t base64_decode(const BYTE in[], BYTE out[], size_t len);
26
27 #endif // BASE64_H

```

9. B-Con_crypto-algorithms/base64_test.c (54 lines)

```

1 /
*****
2 * Filename:   blowfish_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6
7 * Details:   Performs known-answer tests on the corresponding
Base64
8               implementation. These tests do not encompass
the full
9               range of available test vectors, however, if
the tests
10               pass it is very, very likely that the code is
correct
11               and was compiled properly. This code also
serves as
12               example usage of the functions.
13
*****
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <memory.h>
17 #include "base64.h"
18
19 /***** FUNCTION DEFINITIONS *****/
20 int base64_test()
21 {
22     BYTE text[3][1024] = {{"fo"},
23                             {"foobar"},
24                             {"Man is distinguished, not only
by his reason, but by this singular passion from other animals,
which is a lust of the mind, that by a perseverance of delight in
the continued and indefatigable generation of knowledge, exceeds the
short vehemence of any carnal pleasure."}};
25     BYTE code[3][1024] = {"Zm8="},
26                           {"Zm9vYmFy"},
27                           {"TWFuIGlzIGRpc3Rpbm-
d1aXNoZWQsIG5vdCBvbm5lIGJ5IGhpcyByZWZzb24sIGJldCBieS-
B0aGlzInIHNpbmd1bGFyIHBhc3Npb24gZnJvbSBvZGhlc1Bhbm1tYWxzLCB3aGljaCB
pcyBhIGxlc3Qg-
b2YgYndGhlIGljbmlkIHRoYXQgYnkqYSBvZWZjZjZlcmFuY2UgY2YgZGVzawdodCBp-
biB-
0aGUGY29udGluYndwIGFuZCBpbmRlZmF0aWdhYmxlIGd1bmVvYXNpb24gY2Yga25v
d2x1ZG-
d1LCBleGlnZWZrIHRoZSBzaG9yZCB2ZWhlbWVuY2UgY2YgY2Y5IGhncm5hbnBwGv

```

```

hc3VyZS4="}}};
28     BYTE buf[1024];
29     size_t buf_len;
30     int pass = 1;
31     int idx;
32
33     for (idx = 0; idx < 3; idx++) {
34         buf_len = base64_encode(text[idx], buf,
strlen(text[idx]), 1);
35         pass = pass && ((buf_len == strlen(code[idx]))
&&
36                     (buf_len ==
base64_encode(text[idx], NULL, strlen(text[idx]), 1)));
37         pass = pass && !strcmp(code[idx], buf);
38
39         memset(buf, 0, sizeof(buf));
40         buf_len = base64_decode(code[idx], buf,
strlen(code[idx]));
41         pass = pass && ((buf_len == strlen(text[idx]))
&&
42                     (buf_len ==
base64_decode(code[idx], NULL, strlen(code[idx]), 1)));
43         pass = pass && !strcmp(text[idx], buf);
44     }
45     return(pass);
46 }
47
48 int main()
49 {
51     printf("Base64 tests: %s\n", base64_test() ? "PASSED" :
"FAILED");
52
53     return 0;
54 }

```

10. B-Con_crypto-algorithms/blowfish.c (269 lines)

```

1 /
*****
2 * Filename:   blowfish.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the Blowfish encryption al-
gorithm.
7               Modes of operation (such as CBC) are not in-
cluded.
8               Algorithm specification can be found here:
9               * http://www.schneier.com/blowfish.html
10
*****
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "blowfish.h"
16
17 /***** MACROS *****/
18 #define F(x,t) t = keystream->s[0][((x) >> 24); \
19               t += keystream->s[1][((x) >> 16) & 0xff]; \
20               t ^= keystream->s[2][((x) >> 8) & 0xff]; \
21               t += keystream->s[3][((x) & 0xff)];
22 #define swap(r,l,t) t = l; l = r; r = t;
23
24 #define ITERATION(l,r,t,pval) l ^= keystream->p[pval]; F(l,t);
r ^= t; swap(r,l,t);
25
26 /***** VARIABLES *****/
27 static const WORD p_perm[18] = {
28     0x243F6A88, 0x85A308D3, 0x13198A2E,
0x03707344, 0xA4093822, 0x299F31D0, 0x082EFA98,
29     0xEC4E6C89, 0x452821E6, 0x38D01377, 0xBE5466CF, 0x34E90C6C, 0x-
C0AC29B7, 0xC97C50DD,
30     0x3F84D5B5, 0xB5470917, 0x9216D5D9, 0x8979FB1B
31 };
32
33 static const WORD s_perm[4][256] = { {
34     0xD1310BA6, 0x98DFB5AC, 0x2FFD72DB, 0XD01ADF7, 0xB8E1AFED,
0x6A267E96, 0xBA7C9045, 0xF12C7F99,
35     0x24A19947, 0xB3916CF7, 0x0801F2E2, 0x858EFC16, 0x636920D8, 0x71574E69, 0x-
XA458FEA3, 0xF4933D7E,
36     0x0D95748F, 0x728EB658, 0x718BCD58, 0x82154AEE, 0x7B54A41D, 0x-
C25A59B5, 0x9C30D539, 0x2AF26013,
37     0xC5D1B023, 0x286085F0, 0xCA417918, 0xB8DB38EF, 0x8E79D-
CB0, 0x603A180E, 0x6C9E0E8B, 0xB01E8A3E,
38     0xD71577C1, 0xBD314B27, 0x78FAF2FA,
0x556605C6, 0xE65525F3, 0x0AA5AB94, 0x57489862, 0x63E81440,
39     0x55CA396A, 0x2AAB1086, 0xB4CC5C34, 0x1141E8CE, 0xA15486AF,
0x7C7E993, 0xB3EE1411, 0x636FBC2A,
40     0x2BA9C55D, 0x741831F6, 0xCE5C3E16, 0x9B87931E, 0xAF-
D6BA33, 0x6C24CF5C, 0x7A325381, 0x28958677,

```

```

40 0x3B8F4898, 0x6B4BB9AF, 0xC4BFE81B, 0x66282193, 0x61D809CC, 0xF-
B21A991, 0x487CAC60, 0x5DEC8032,
41 0xEF845D5D,
0xE98575B1, 0xDC262302, 0xEB651B88, 0x23893E81, 0xD396ACCS, 0x0F6D6FF3, 0
x83F44239,
42 0x2E0B4482, 0xA4842004, 0x69C8F04A, 0x9E1F9B5E,
0x21C66842, 0xF6E96C9A, 0x670C9C61, 0xAB0388F0,
43 0x6A51A0D2, 0xD8542F68, 0x960FA728, 0xAB5133A3, 0x6EEF0B6C,
0x137A3BE4, 0xBA3BF050, 0x7EFF2A98,
44 0xA1F1651D, 0x39AF0176, 0x66CA593E,
0x82430E88, 0x8CEE8619, 0x456F9FB4, 0x7D84A5C3, 0x3B8B5EBE,
45 0xE06F75D8, 0x85C12073, 0x401A449F,
0x56C16AA6, 0x4ED3AA62, 0x363F7706, 0x1BFEDF72, 0x429B023D,
46 0x37D0D724, 0xD00A1248, 0xDB0FEAD3, 0x49F1C09B,
0x075372C9, 0x80991B78, 0x25D479D8, 0xF6E8DEF7,
47 0xE3FE501A, 0xB86794C3B, 0x976CE0BD, 0x04C006BA, 0xC1A94F-
B6, 0x409F60C4, 0x5E5C9EC2, 0x196A2463,
48 0x68FB6FAF, 0x3E6C53B5, 0x1339B2EB, 0x3B52EC6F, 0x6DFC511F,
0x9B30952C, 0xCC814544, 0xAF5EBD09,
49 0xBEE3D004, 0xDE334AFD,
0x660F2807, 0x192E4BB3, 0x0C0BA857, 0x45C8740F, 0xD20B5F39, 0xB9D3FBD8,
50 0x5579C0BD, 0x1A60320A, 0xD6A100C6, 0x402C7279, 0x679F25FE, 0xF-
B1FA3CC, 0x8EA5E9F8, 0xDB3222F8,
51 0x3C7516DF, 0xFD616B15, 0x2F501EC8, 0xAD0552AB, 0x323DB5FA, 0xF-
D238760, 0x53317B48, 0x3E00DF82,
52 0x9E5C57B8, 0xCA6F8CA0, 0x1A87562E, 0xDF1769DB,
0xD542A8F6, 0x287EFFC3, 0xACA6732C6, 0x8C4F5573,
53 0x695B27B0, 0xBBCA58C8, 0xE1FFA35D,
0xB8F011A0, 0x10FA3D98, 0xFD2183B8, 0x4AFCB56C, 0x2DD1D35B,
54 0x9A53C479, 0xB6F84565, 0xD28E49BC, 0x4BF8790, 0xE1DDF2DA, 0x-
A4CB7E33, 0x62FB1341, 0xCEEA4C6E8,
55 0xEF20CADA, 0x36774C01, 0xD07E9EFE, 0x2BF11FB4, 0x95DBDA4D, 0x-
AE909198, 0xEAA8DE71, 0x68993D5A0,
56 0xD08ED1D0, 0xAF7275E0, 0xE83C5B2F, 0xE87594B7, 0x8FF6E2FB,
0xF2122B64, 0x8888B812, 0x900DF01C,
57 0x4FAD5EA0, 0x688FC31C, 0xD1CFF191, 0xB3A8C1AD,
0x2F2F2218, 0xBE0E1777, 0xEA752DFE, 0xB8021FA1,
58 0xE5A0CC0F,
0xB56F74E8, 0x18ACF3D6, 0xCE89E299, 0xB4844FE0, 0xFD13E0B7, 0x7C443B81, 0
xD2ADAD89,
59 0x165-
FA266, 0x80957705, 0x93CC7314, 0x211A1477, 0xE6AD2065, 0x77B5FA86, 0xC7544
2F5, 0xFB9D35CF,
60 0xEBCDAF0C, 0x7B3E89A0, 0xD6411BD3, 0xAE1E7E49, 0x00250E2D,
0x2071B35E, 0x226800BB, 0x57B8E0AF,
61 0x24643698, 0xF009B91E, 0x5563911D, 0x59DFA6AA,
0x78C14389, 0xD95A537F, 0x207D5BA2, 0x02E5B9C5,
62 0x83260376, 0x6295CFA9, 0x11C81968, 0x4E734A41, 0xB3472DCA,
0x7B14A94A, 0x1B510052, 0x9A532915,
63 0xD60F573F, 0xBC9B-
C6E4, 0x2B60A476, 0x81E67400, 0x08BA6FB5, 0x571BE91F, 0xF296EC6B, 0x2A0D-
D915,
64 0xB6636521, 0xE7B79FB6, 0xFF34052E, 0xC5855664, 0x53B02D5D, 0x-
A998FA1, 0x08BA4799, 0x6E85076A
65 }, {
66 0x4B7A70E9, 0xB5B32944, 0xDB75092E, 0x-
C4192623, 0xAD6EA6B0, 0x49A7DF7D, 0x9CEEE60B8, 0x8FEDB266,
67 0xECAA8C71, 0x699A17FF, 0x5664526C, 0x-
C2B19EE1, 0x193602A5, 0x75094C29, 0xA0591340, 0xE4183A3E,
68 0x3F54989A,
0x58429D65, 0x6B8FE4D6, 0x99F73FD6, 0xA1D29C07, 0xEFE830F5, 0x4D2D38E6, 0
xF0255DC1,
69 0x4CDD2086, 0x8470EB26, 0x6382E9C6, 0x021ECC5E, 0x09686B3F,
0x3BEAEFC9, 0x3C971814, 0x6B6A70A1,
70 0x687F3584, 0x52A0E286, 0xB79C5305, 0xAA500737, 0x3E07841C,
0x7FDEAESC, 0xE87D44EC, 0x5716F2B8,
71 0xB03ADA37, 0xF0500C0D, 0xF01C1F04, 0x0200B3FF, 0xAE0CF51A,
0x3CB574B2, 0x25837A58, 0xDC0921BD,
72
0xD19113F9, 0x7CA92FF6, 0x94324773, 0x22F54701, 0x3AE5E581, 0x37C2DADC,
0x8B57634, 0x9AF3DDA7,
73 0xA9446146, 0x0FD0030E, 0xECC8C73E, 0x-
A4751E41, 0xE238CD99, 0x3BEA0E2F, 0x3280BBA1, 0x183EB331,
74 0x4E548B38, 0x4F60B908, 0x6F42D0D3, 0xF60A04BF,
0x2CB81290, 0x24977C79, 0x5679B072, 0xBCAF89AF,
75 0xDE9A771F, 0xD9930810, 0xB38BAE12, 0xDCCF3F2E, 0x5512721F,
0x2E6B7124, 0x501ADD06, 0x9F84CD87,
76 0x7A584718, 0x7408DA17, 0xBC9F9ABC, 0xE94B7D8C, 0xEC7AEC3A, 0xD-
B851DFA, 0x63094366, 0xC464C3D2,
77
0xEF1C1847, 0x3215D908, 0xDD433B37, 0x24C2BA16, 0x12A14D43, 0x2A65C451, 0
x50940002, 0x133AE4DD,
78 0x71DDF89E, 0x10314E55, 0x81AC77D6, 0x5F11199B,
0x043556F1, 0xD7A3C76B, 0x3C11183B, 0x5924A509,
79 0xF28FE6ED, 0x9771FBFA, 0x9EAB8F2C, 0x1E153C6E,
0x86E34570, 0xEAE96FB1, 0x806E5E0A, 0x5A3E2AB3,
80 0x771FE71C, 0x4E3D06FA, 0x2965DCB9, 0x99E71D0F,
0x803E89D6, 0x5266C825, 0x2E4CC978, 0x9AC10B36A,
81 0xC6150EBA,
0x94E2EA78, 0xA5FC3C53, 0x1E0A2DF4, 0xF2F74EA7, 0x361D2B3D, 0x1939260F,
0x19C27960,
82 0x5223A708, 0xF71312B6, 0xEBADF6E6,
0xEAC31F66, 0xE3BC4595, 0xA67BC883, 0xB17F37D1, 0x018CFF28,
83 0xC332DDEF, 0xBE6C5A85, 0x65582185, 0x68AB9802, 0xEECEAE50F, 0xD-
B2F953B, 0x2AEF7DAD, 0x586E2F84,
84
0x1521B628, 0x29076170, 0xECDD4775, 0x619F1510, 0x13CCA830, 0xEB61BD96, 0
x0334FE1E, 0xAA0363CF,
85 0xB573C90, 0x4C70A239, 0xD59E9E0B, 0xCBAADE14, 0xECC8B6BC,
0x60622CA7, 0x9CAB5CAB, 0xB2F3846E,
86 0x648B1EAF, 0x19BDF0CA, 0x-

```

```

A02369B9, 0x655ABB50, 0x40685A32, 0x3C2AB4B3, 0x319EE9D5, 0xC021B8F7,
87 0x9B540B19, 0x875FA099, 0x95F7997E, 0x623D7DA8, 0xF837889A,
0x97FE32D7, 0x11ED935F, 0x16681281,
88
0x0E358829, 0xC7E61FD6, 0x96DEDAF1, 0x7858BA99, 0x57F584A5, 0x1B227263, 0
x98B3C3FF, 0x1AC24696,
89 0xCDB30AEB, 0x532E3054, 0x8FD948E4, 0x6DBC3128, 0x58EBF2EF,
0x34C6FFEA, 0xFE28ED61, 0xEE7C3C73,
90
0x5D4A14D9, 0xE864B7E3, 0x42105D14, 0x203E13E0, 0x45EEE2B6, 0xA3AAABEA,
0xDB6C4F15, 0xFACB4FD0,
91 0xC742F442, 0xEF6ABBB5, 0x654F3B1D, 0x41CD2105, 0xD81E799E,
0x86854DC7, 0xE44B476A, 0x3D816250,
92 0x-
CF62A1F2, 0x58BD2646, 0xFC8883A0, 0xC1C7B6A3, 0x7F1524C3, 0x69CB7492, 0x47
848A0B, 0x5692B285,
93 0x095BBF00, 0xAD19489D, 0x1462B174, 0x23820E00, 0x58428D2A,
0x0C55F5EA, 0x1DADF43E, 0x233F7061,
94 0x3372F092, 0x8D937E41, 0xD65FECF1, 0x6C223BDB,
0x7CDE3759, 0xCBE7460, 0x4085F2A7, 0xCE77326E,
95 0xA6078084, 0x19F8509E, 0xCE8EFD855, 0x61D99735, 0xA969A7AA, 0x-
C50C6C2, 0x5A04ABFC, 0x800BCADC,
96 0x9E447A2E, 0x-
C3453484, 0xFDD56705, 0x0E1E9EC9, 0xDB73DBD3, 0x105588CD, 0x675F-
DA79, 0xE3674340,
97 0xC5C43465, 0x713E38D8, 0x3D28F89E, 0xF16D-
FF20, 0x153E21E7, 0x8FB03D4A, 0xE6E39F28, 0xDB83ADF7
98 }, {
99 0xE93D5A68, 0x948140F7, 0xF64C261C,
0x94692934, 0x411520F7, 0x7602D4F7, 0xBCF46B2E, 0xD4A20068,
100 0xD4082471, 0x3320F46A, 0x43B7D4B7, 0x500061AF, 0x1E39F62E,
0x97244546, 0x14214F74, 0xBF8B8840,
101 0x4D95FC1D, 0x96B591AF, 0x70F4DD3, 0x66A02F45, 0xBFBC09EC,
0x03BD9785, 0x7FAC6DD0, 0x31CB8504,
102 0x96E827B3, 0x55FD3941, 0xDA2547E6, 0xABCA0A9A,
0x28507825, 0x530429F4, 0x0A2C86DA, 0xEB966DFB,
103 0x68DC1462, 0xD7486900, 0x680EC0A4, 0x27A18DEE, 0x4F3F-
FEA2, 0xE887AD8C, 0xB58CE006, 0x7AF4D6B6,
104 0xAACE1E7C, 0xD3375FEC, 0x-
CE78A399, 0x406B2A42, 0x20FE9E35, 0xD9F385B9, 0xEE39D7AB, 0x3B124EEB,
105 0x1DC9-
FAF7, 0x4B6D1856, 0x26A36631, 0xEAE397B2, 0x3A6EFA74, 0xD5B4332, 0x6841E7
F7, 0xCA7820FB,
106 0xF80AF54E, 0xD8FEB397, 0x454056AC, 0xBA489527, 0x55533A3A,
0x20838D87, 0xFE6BA9B7, 0xD096954B,
107 0x55A867BC, 0x-
A1159A58, 0xCCA92963, 0x99E1DB33, 0xA62A4A56, 0x3F3125F9, 0x5EF47E1C,
0x9029317C,
108 0xFDF8E802, 0x04272F70, 0x80BB155C,
0x05282CE3, 0x95C11548, 0xE4C66D22, 0x48C1133F, 0xC70F86DC,
109 0x07F9C9EE, 0x41041F0F, 0x404779A4, 0x5D886E17, 0x325F51EB,
0xD59BC0D1, 0xF2BCC18F, 0x41113564,
110 0x257B7834, 0x0620A9C60, 0xDFF8E8A3, 0x1F636C1B,
0x0E12B4C2, 0x02E1329E, 0xAF664FD1, 0xCAD18115,
111 0x6B2395E0, 0x333E92E1, 0x3B240B62, 0xEEBE922, 0x85B2A20E,
0xE6BA0D99, 0xDE720C8C, 0x2DA2F728,
112 0xD0127845, 0x95B794FD, 0x647D0862, 0xE7CCF5F0, 0x5449A36F,
0xB77D48FA, 0xC39D0FD7, 0xF33E8D1E,
113 0x0A476341, 0x992EFF74, 0x3A6F6EAB, 0xF4F8F-
D37, 0xA812DC60, 0xA1EBDDF8, 0x991BE14C, 0xDBE66B0D,
114 0xC67B5510, 0x6D672C37, 0x2765D43B, 0xD-
CD0E804, 0xF1290DC7, 0xCC00FFA3, 0xB5390F92, 0x690FED0B,
115 0x667B9FFB, 0xCEDB7D9C, 0xA091CF0B,
0xD9155EA3, 0xBB132F88, 0x515BADA4, 0x7B9479BF, 0x763BD6EB,
116 0x37392EB3, 0xCC115979, 0x8026E297, 0xF42E312D,
0x6842ADA7, 0xC66A2B3B, 0x12754CCC, 0x782EF11C,
117
0x6A124237, 0xB79251E7, 0x06A1BBE6, 0x4BF6B350, 0x1A6B1018, 0x11CAEDFA,
0x3D25BDD8, 0xE2E1C3C9,
118 0x44421659, 0x0A121386, 0xD90CEC6E, 0xD5ABEA2A, 0x64AF674E,
0xDA86A85F, 0xBEBFE988, 0x64E4C3FE,
119 0x9DB8C8057, 0xF0F7C086, 0x60787BF8, 0x6003604D, 0xD1F-
D8346, 0xF6381FB0, 0x7745EA04, 0xD736FCCC,
120 0x83426B33, 0xF01EAB71, 0xB0804187, 0x3C005E5F, 0x77A057BE, 0xB-
DEBAE24, 0x55464299, 0xBF582E61,
121 0x4E58F48F, 0xF2DDF-
DA2, 0xF474E-
F38, 0x8789BDC2, 0x5366F9C3, 0xC8B38E74, 0xB475F255, 0x46FCD9B9,
122 0x7AEB2661, 0x8B1DDF84, 0x846A0E79, 0x915F95E2, 0x466E598E,
0x20B45770, 0x8CD55591, 0xC902DE4C,
123 0xB90BACE1, 0xBB8205D0, 0x11A86248, 0x7574A99E,
0xB77F19B6, 0xE0A9DC09, 0x662D09A1, 0xC4324633,
124 0xE85A1F02, 0x09F0BE8C, 0x4A99A025, 0x1D6EFE10, 0x1AB93D1D,
0x0BA5A4DF, 0xA186F20F, 0x2868F169,
125 0xDCB7DA83, 0x573906FE, 0xA1E2CE9B,
0x4FCD7F52, 0x50115E01, 0xA70683FA, 0x00DE2B5C4, 0x0DE6D027,
126
0x9AF88C27, 0x773F8641, 0xC3604C06, 0x61A806B5, 0xF0177A28, 0xC0F586E0, 0
x06058AA, 0x39DC7D62,
127
0x11E69ED7, 0x2338EA63, 0x53C2DD94, 0xC2C21634, 0xBBCEEE56, 0x90BCB6DE,
0xEBFC7DA1, 0xC5E91D76,
128 0x6F05E409, 0x4B7C0188, 0x39720A3D, 0x7C927C24, 0x86E3725F,
0x724D9DB9, 0x1AC15BB4, 0xD39EB8FC,
129 0xED545578, 0x08FCA5B5, 0xD83D7CD3, 0x4DAD0FC4, 0x1E50EFE5E,
0xB161E6F8, 0xA28514D9, 0x6C51133C,
130 0x6FD5C7E7, 0x56E14EC4, 0x362ABFCE, 0xD-
DC6C837, 0xD79A3234, 0x92638212, 0x670EFA8E, 0x46600E0
131 }, {
132 0x3A39CE37, 0xD3FAF5CF, 0xABC27737, 0x5AC52D1B, 0x5CB0679E, 0x4-
FA33742, 0xD3822740, 0x99BC9BBE,

```



```

133 0xD5118E9D, 0xBF0F7315, 0xD62D1C7E, 0xC700C47B, 0xB78C1B6B,
0x21A19045, 0xB26EB1BE, 0x6A366EB4,
134 0x5748AB2F, 0xCBC946E7, 0xC6A376D2, 0x6549C2C8, 0x530FF8EE,
0x468DDE7D, 0xD5730A1D, 0x4CD04DC6,
135 0x2939BBD8, 0x-
A9BA4650, 0xAC9526E8, 0xBE5EE304, 0xA1FAD5F0, 0xA6A2D519, 0x63E-
F8CE2, 0x9A86EE22,
136 0xC089C2B8, 0x43242EF6, 0xA51E03AA, 0x9CF2D0A4, 0x83C061BA,
0x9BE96A4D, 0x8FE51550, 0xBA645BD6,
137
0x2826A2F9, 0xA73A3AE1, 0x4BA99586, 0xEF5562E9, 0xC72FEFD3, 0xF752F7DA,
0x3F046F69, 0x77FA0A59,
138 0x80E4A915, 0x87B08601, 0x9B09E6AD, 0x3B3EE593, 0xE990FD5A,
0x9E34D797, 0x2CF0B7D9, 0x022B8B51,
139 0x96D5AC3A, 0x017DA67D, 0xD1CF3ED6, 0x7C7D2D28, 0x1F9F25CF, 0x-
ADF2B89B, 0x5AD6B472, 0x5A88F54C,
140 0xE029AC71, 0xE019A5E6, 0x47B0ACFD, 0xED93FA9B, 0xE8D3C48D,
0x283B57CC, 0xF8D56629, 0x79132E28,
141 0x785F0191, 0xED756055, 0xF7960E44, 0xE3D35E8C, 0x15056D-
D4, 0x88F46DBA, 0x03A16125, 0x0564F0BD,
142 0xC3EB9E15, 0x3C9057A2, 0x97271AEC, 0xA93A072A, 0x1B3F6D9B,
0x1E6321F5, 0xF59C66FB, 0x26DCF319,
143 0x7533D928, 0xB155FDF5, 0x03563482, 0x8ABA3CBB,
0x28517711, 0xC2AD9F8, 0xABCC5167, 0xCCAD925F,
144 0x4DE81751, 0x3830DC8E,
0x379D5862, 0x9320F991, 0xEA7A90C2, 0xFB3E7BCE, 0x5121CE64, 0x774FBE32,
145 0xA8B6E37E, 0x-
C3293D46, 0x48DE5369, 0x6413E680, 0xA2AE0810, 0xDD6DB224, 0x69852DFD,
0x09072166,
146 0xB39A460A, 0x6445C0DD, 0x586CDECF, 0x1C20C8AE, 0x5BBEF7DD,
0x1B58DD0, 0xCCD2017F, 0x6BB4E3BB,
147 0xDDA26A7E,
0x3A59FF45, 0x3E350A44, 0xBCB4CDD5, 0x72EACEA8, 0xFA6484BB, 0x8D6612AE,
0xBF3C6F47,
148 0xD29BE463, 0x542F5D9E, 0xAEC2771B, 0xF64E6370, 0x740E08D0,
0xE75B1357, 0xF8721671, 0xAF537D5D,
149 0x4040CB08, 0x4EB4E2CC, 0x34D2466A,
0x0115AF84, 0xE1B00428, 0x95983A1D, 0x06B89FB4, 0xCE6EA048,
150 0x6F3F3B82, 0x3520AB82, 0x011A1D4B,
0x277227F8, 0x611560B1, 0xE7933FDC, 0xBB3A792B, 0x344525BD,
151 0xA08839E1, 0x51CE794B, 0x2F32C9B7, 0xA01FBAC9, 0xE01CC87E, 0xB-
CC7D1F6, 0xCF0111C3, 0xA1E8AAC7,
152 0x1A908749, 0xD44FB09A, 0xD0DADECB, 0xD50ADA38, 0x0339C32A, 0x-
C6913667, 0x8DF9317C, 0xE0B12B4F,
153 0xF79E59B7, 0x43F5BB3A, 0xF2D519FF, 0x27D9459C, 0xBF97222C,
0x15E6FC2A, 0x0F91FC71, 0x9B941525,
154 0xFAE59361, 0xCEB69CEB, 0xC2A86459, 0x12BAA8D1, 0xB6C1075E,
0xE3056A0C, 0x10D25065, 0xC803A442,
155 0xE0EC6E0E, 0x1698DB3B, 0x4C98A0BE,
0x3278E964, 0x9F1F9532, 0xE0D392DF, 0xD3A0342B, 0x8971F21E,
156 0x1B0A7441, 0x4BA3348C, 0xC5BE7120, 0xC37632D8, 0xDF359F8D,
0x9B992F2E, 0xE60B6F47, 0x0FE3F11D,
157 0xE54CDA54, 0x1EDAD891, 0xCE6279CF, 0xCD3E7E6F,
0x1618B166, 0xFD2C1D05, 0x848FD2C5, 0xF6FB2299,
158
0xF523F357, 0xA6327623, 0x93A83531, 0x56CCDC0D, 0xACF08162, 0x5A75EBB5, 0
x6E163697, 0x88D273CC,
159 0xDE966292, 0x81B949D0, 0x4C50901B, 0x71C65614, 0xE6C6C7BD,
0x327A140A, 0x45E1D006, 0xC3F27B9A,
160 0xC9AA53FD,
0x62A80F00, 0xBB25BF62, 0x35BDD2F6, 0x71126905, 0xB2040222, 0xB6CBCF7C,
0xCD769C2B,
161 0x53113EC0, 0x1640E3D3, 0x38ABBD60, 0x2547ADF0, 0xBA38209C,
0xF746CE76, 0x77AFA1C5, 0x20756060,
162 0x85CBFE4E, 0x8AE88DD8, 0x7AAAF9B0, 0x4CF9AA7E, 0x1948C25C,
0x02F8B8AC, 0x01C36AE4, 0xD6EBE1F9,
163 0x90D4F869, 0xA65CDEA0, 0x3F09252D, 0xC208E69F,
0xB74E6132, 0xCE77E25B, 0x578FDFE3, 0x3AC372E6
164 } };
165
166 /***** FUNCTION DEFINITIONS *****/
167 void blowfish_encrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystuct)
168 {
169     WORD l, r, t; //, i;
170
171     l = (in[0] << 24) | (in[1] << 16) | (in[2] << 8) | (in[3]);
172     r = (in[4] << 24) | (in[5] << 16) | (in[6] << 8) | (in[7]);
173
174     ITERATION(l, r, t, 0);
175     ITERATION(l, r, t, 1);
176     ITERATION(l, r, t, 2);
177     ITERATION(l, r, t, 3);
178     ITERATION(l, r, t, 4);
179     ITERATION(l, r, t, 5);
180     ITERATION(l, r, t, 6);
181     ITERATION(l, r, t, 7);
182     ITERATION(l, r, t, 8);
183     ITERATION(l, r, t, 9);
184     ITERATION(l, r, t, 10);
185     ITERATION(l, r, t, 11);
186     ITERATION(l, r, t, 12);
187     ITERATION(l, r, t, 13);
188     ITERATION(l, r, t, 14);
189     l ^= keystuct->p[15]; F(l, t); r ^= t; //Last iteration has
no swap()
190     r ^= keystuct->p[16];
191     l ^= keystuct->p[17];
192
193     out[0] = l >> 24;

```

```

194     out[1] = l >> 16;
195     out[2] = l >> 8;
196     out[3] = l;
197     out[4] = r >> 24;
198     out[5] = r >> 16;
199     out[6] = r >> 8;
200     out[7] = r;
201 }
202
203 void blowfish_decrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystuct)
204 {
205     WORD l, r, t; //, i;
206
207     l = (in[0] << 24) | (in[1] << 16) | (in[2] << 8) | (in[3]);
208     r = (in[4] << 24) | (in[5] << 16) | (in[6] << 8) | (in[7]);
209
210     ITERATION(l, r, t, 17);
211     ITERATION(l, r, t, 16);
212     ITERATION(l, r, t, 15);
213     ITERATION(l, r, t, 14);
214     ITERATION(l, r, t, 13);
215     ITERATION(l, r, t, 12);
216     ITERATION(l, r, t, 11);
217     ITERATION(l, r, t, 10);
218     ITERATION(l, r, t, 9);
219     ITERATION(l, r, t, 8);
220     ITERATION(l, r, t, 7);
221     ITERATION(l, r, t, 6);
222     ITERATION(l, r, t, 5);
223     ITERATION(l, r, t, 4);
224     ITERATION(l, r, t, 3);
225     l ^= keystuct->p[2]; F(l, t); r ^= t; //Last iteration has
no swap()
226     r ^= keystuct->p[1];
227     l ^= keystuct->p[0];
228
229     out[0] = l >> 24;
230     out[1] = l >> 16;
231     out[2] = l >> 8;
232     out[3] = l;
233     out[4] = r >> 24;
234     out[5] = r >> 16;
235     out[6] = r >> 8;
236     out[7] = r;
237 }
238
239 void blowfish_key_setup(const BYTE user_key[], BLOWFISH_KEY
*keystuct, size_t len)
240 {
241     BYTE block[8];
242     int idx, idx2;
243
244     // Copy over the constant init array vals (so the originals
aren't destroyed).
245     memcpy(keystuct->p, p_perm, sizeof(WORD) * 18);
246     memcpy(keystuct->s, s_perm, sizeof(WORD) * 1024);
247
248     // Combine the key with the P box. Assume key is standard
448 bits (56 bytes) or less.
249     for (idx = 0, idx2 = 0; idx < 18; ++idx, idx2 += 4)
250         keystuct->p[idx] ^= (user_key[idx2 % len] << 24) |
(user_key[(idx2+1) % len] << 16)
| (user_key[(idx2+2) % len] << 8) |
(user_key[(idx2+3) % len]);
251
252     // Re-calculate the P box.
253     memset(block, 0, 8);
254     for (idx = 0; idx < 18; idx += 2) {
255         blowfish_encrypt(block, block, keystuct);
256         keystuct->p[idx] = (block[0] << 24) | (block[1] << 16)
| (block[2] << 8) | block[3];
257         keystuct->p[idx+1] = (block[4] << 24) | (block[5] << 16)
| (block[6] << 8) | block[7];
258     }
259     // Recalculate the S-boxes.
260     for (idx = 0; idx < 4; ++idx) {
261         for (idx2 = 0; idx2 < 256; idx2 += 2) {
262             blowfish_encrypt(block, block, keystuct);
263             keystuct->s[idx][idx2] = (block[0] << 24) |
(block[1] << 16) |
(block[2] << 8) | block[3];
264             keystuct->s[idx][idx2+1] = (block[4] << 24) |
(block[5] << 16) |
(block[6] << 8) |
block[7];
265         }
266     }
267 }
268
269 }

```

11. B-Con_crypto-algorithms/blowfish.h (32 lines)

```

1 /
*****
*
2 * Filename:    blowfish.h
3 * Author:      Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-

```

```

antees.
6 * Details:    Defines the API for the corresponding Blowfish
implementation.
7
8 /*****
9 #ifndef BLOWFISH_H
10 #define BLOWFISH_H
11
12 /***** HEADER FILES
13 #include <stddef.h>
14
15 /***** MACROS
16 #define BLOWFISH_BLOCK_SIZE 8          // Blowfish operates
on 8 bytes at a time
17
18 /***** DATA TYPES
19 typedef unsigned char BYTE;           // 8-bit byte
20
21 typedef unsigned int WORD;           // 32-bit word, change to
"long" for 16-bit machines
22
23 typedef struct {
24     WORD p[18];
25     WORD s[4][256];
26 } BLOWFISH_KEY;
27
28 /***** FUNCTION DECLARATIONS
29 void blowfish_key_setup(const BYTE user_key[], BLOWFISH_KEY
*keystruct, size_t len);
30 void blowfish_encrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystruct);
31 void blowfish_decrypt(const BYTE in[], BYTE out[], const BLOW-
FISH_KEY *keystruct);
32 #endif // BLOWFISH_H

```

12. B-Con_crypto-algorithms/blowfish_test.c (68 lines)

```

1 /
2 * Filename:    blowfish_test.c
3 * Author:      Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-
antees.
6 * Details:     Performs known-answer tests on the corresponding
Blowfish
7                implementation. These tests do not encompass
the full
8                range of available test vectors, however, if
the tests
9                pass it is very, very likely that the code is
correct
10               and was compiled properly. This code also
serves as
11               example usage of the functions.
12
13 /*****
14 /***** HEADER FILES
15 #include <stdio.h>
16 #include <memory.h>
17 #include "blowfish.h"
18
19 /***** FUNCTION DEFINITIONS
20 int blowfish_test()
21 {
22     BYTE key1[8] =
{0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
23     BYTE key2[8] = {0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff};
24     BYTE key3[24] =
{0xf0, 0xe1, 0xd2, 0xc3, 0xb4, 0xa5, 0x96, 0x87,
25      0x78, 0x69, 0x5a, 0x4b, 0x3c, 0x2d, 0x1e,
0x0f,
26      0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77};
27     BYTE p1[BLOWFISH_BLOCK_SIZE] =
{0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
28     BYTE p2[BLOWFISH_BLOCK_SIZE] = {0xff, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff};
29     BYTE p3[BLOWFISH_BLOCK_SIZE] = {0xfe, 0xdc, 0xba,
0x98, 0x76, 0x54, 0x32, 0x10};
30
31     BYTE c1[BLOWFISH_BLOCK_SIZE] = {0x4e,
0xf9, 0x97, 0x45, 0x61, 0x98, 0xdd, 0x78};
32     BYTE c2[BLOWFISH_BLOCK_SIZE] = {0x51, 0x86, 0x6f,
0xd5, 0xb8, 0x5e, 0xcb, 0x8a};

```

```

33     BYTE c3[BLOWFISH_BLOCK_SIZE] = {0x05, 0x04, 0x4b,
0x62, 0xfa, 0x52, 0xd0, 0x80};
34
35     BYTE enc_buf[BLOWFISH_BLOCK_SIZE];
36     BLOWFISH_KEY key;
37     int pass = 1;
38
39     // Test vector 1.
40     blowfish_key_setup(key1, &key, BLOWFISH_BLOCK_SIZE);
41     blowfish_encrypt(p1, enc_buf, &key);
42     pass = pass && !memcmp(c1, enc_buf, BLOW-
FISH_BLOCK_SIZE);
43     blowfish_decrypt(c1, enc_buf, &key);
44     pass = pass && !memcmp(p1, enc_buf, BLOW-
FISH_BLOCK_SIZE);
45
46     // Test vector 2.
47     blowfish_key_setup(key2, &key, BLOWFISH_BLOCK_SIZE);
48     blowfish_encrypt(p2, enc_buf, &key);
49     pass = pass && !memcmp(c2, enc_buf, BLOW-
FISH_BLOCK_SIZE);
50     blowfish_decrypt(c2, enc_buf, &key);
51     pass = pass && !memcmp(p2, enc_buf, BLOW-
FISH_BLOCK_SIZE);
52
53     // Test vector 3.
54     blowfish_key_setup(key3, &key, 24);
55     blowfish_encrypt(p3, enc_buf, &key);
56     pass = pass && !memcmp(c3, enc_buf, BLOW-
FISH_BLOCK_SIZE);
57     blowfish_decrypt(c3, enc_buf, &key);
58     pass = pass && !memcmp(p3, enc_buf, BLOW-
FISH_BLOCK_SIZE);
59
60     return(pass);
61 }
62
63 int main()
64 {
65     printf("Blowfish tests: %s\n", blowfish_test() ? "SUC-
CEDED" : "FAILED");
66
67     return(0);
68 }

```

13. B-Con_crypto-algorithms/des.c (269 lines)

```

1 /
2 * Filename:    des.c
3 * Author:      Brad Conte (brad AT radconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-
antees.
6 * Details:     Implementation of the DES encryption algorithm.
Modes of operation (such as CBC) are not in-
cluded.
7                The formal NIST algorithm specification can be
found here:
8                * http://csrc.nist.gov/publications/fips/
fips46-3/fips46-3.pdf
9
10 /*****
11 /***** HEADER FILES
12 #include <stdlib.h>
13 #include <memory.h>
14 #include "des.h"
15
16 /***** MACROS
17
18 // Obtain bit "b" from the left and shift it "c" places from
the right
19 #define BITNUM(a,b,c) (((a[(b)/8] >> (7 - (b%8))) & 0x01) <<
(c))
20 #define BITNUMINTR(a,b,c) (((a >> (31 - (b))) & 0x00000001)
<< (c))
21 #define BITNUMINTL(a,b,c) (((a << (b)) & 0x80000000) >> (c))
22
23 // This macro converts a 6 bit block with the S-Box row
defined as the first and last
24 // bits to a 6 bit block with the row defined by the first two
bits.
25 #define SBOXBIT(a) (((a) & 0x20) | (((a) & 0x1f) >> 1) | (((a) &
0x01) << 4))
26
27 /***** VARIABLES
28 static const BYTE sbox1[64] = {
29     14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12,
5, 9, 0, 7,
30     8, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11,
9, 5, 3, 8,
31     4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7,
3, 10, 5, 0,

```

```

32      15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14,
10, 0, 6, 13
33 };
34 static const BYTE sbox2[64] = {
35     15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13,
12, 0, 5, 10,
36     3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10,
6, 9, 11, 5,
37     0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6,
9, 3, 2, 15,
38     13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12,
0, 5, 14, 9
39 };
40 static const BYTE sbox3[64] = {
41     10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7,
11, 4, 2, 8,
42     13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14,
12, 11, 15, 1,
43     13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12,
5, 10, 14, 7,
44     1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3,
11, 5, 2, 12
45 };
46 static const BYTE sbox4[64] = {
47     7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5,
11, 12, 4, 15,
48     13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12,
1, 10, 14, 9,
49     10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14,
5, 2, 8, 4,
50     3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11,
12, 7, 2, 14
51 };
52 static const BYTE sbox5[64] = {
53     2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15,
13, 0, 14, 9,
54     14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10,
3, 9, 8, 6,
55     4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5,
6, 3, 0, 14,
56     11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9,
10, 4, 5, 3
57 };
58 static const BYTE sbox6[64] = {
59     12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4,
14, 7, 5, 11,
60     10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14,
0, 11, 3, 8,
61     9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10,
1, 13, 11, 6,
62     4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7,
6, 0, 8, 13
63 };
64 static const BYTE sbox7[64] = {
65     4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7,
5, 10, 6, 1,
66     13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12,
2, 15, 8, 6,
67     1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8,
0, 5, 9, 2,
68     6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15,
14, 2, 3, 12
69 };
70 static const BYTE sbox8[64] = {
71     13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14,
5, 0, 12, 7,
72     1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11,
0, 14, 9, 2,
73     7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13,
15, 3, 5, 8,
74     2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0,
3, 5, 6, 11
75 };
76
77 /***** FUNCTION DEFINITIONS *****/
78 // Initial (Inv)Permutation step
79 void IP(WORD state[], const BYTE in[])
80 {
81     state[0] = BITNUM(in, 57, 31) | BITNUM(in, 49, 30) | BIT-
NUM(in, 41, 29) | BITNUM(in, 33, 28) |
82     BITNUM(in, 25, 27) | BITNUM(in,
17, 26) | BITNUM(in, 9, 25) | BITNUM(in, 1, 24) |
83     BITNUM(in, 59, 23) | BITNUM(in,
51, 22) | BITNUM(in, 43, 21) | BITNUM(in, 35, 20) |
84     BITNUM(in, 27, 19) | BITNUM(in,
19, 18) | BITNUM(in, 11, 17) | BITNUM(in, 3, 16) |
85     BITNUM(in, 61, 15) | BITNUM(in,
53, 14) | BITNUM(in, 45, 13) | BITNUM(in, 37, 12) |
86     BITNUM(in, 29, 11) | BITNUM(in,
21, 10) | BITNUM(in, 13, 9) | BITNUM(in, 5, 8) |
87     BITNUM(in, 63, 7) | BITNUM(in,
55, 6) | BITNUM(in, 47, 5) | BITNUM(in, 39, 4) |
88     BITNUM(in, 31, 3) | BITNUM(in,
23, 2) | BITNUM(in, 15, 1) | BITNUM(in, 7, 0);
89
90     state[1] = BITNUM(in, 56, 31) | BITNUM(in, 48, 30) | BIT-
NUM(in, 40, 29) | BITNUM(in, 32, 28) |
91     BITNUM(in, 24, 27) | BITNUM(in,
16, 26) | BITNUM(in, 8, 25) | BITNUM(in, 0, 24) |
92     BITNUM(in, 58, 23) | BITNUM(in,

```

```

50, 22) | BITNUM(in, 42, 21) | BITNUM(in, 34, 20) |
93     BITNUM(in, 26, 19) | BITNUM(in,
18, 18) | BITNUM(in, 10, 17) | BITNUM(in, 2, 16) |
94     BITNUM(in, 60, 15) | BITNUM(in,
52, 14) | BITNUM(in, 44, 13) | BITNUM(in, 36, 12) |
95     BITNUM(in, 28, 11) | BITNUM(in,
20, 10) | BITNUM(in, 12, 9) | BITNUM(in, 4, 8) |
96     BITNUM(in, 62, 7) | BITNUM(in,
54, 6) | BITNUM(in, 46, 5) | BITNUM(in, 38, 4) |
97     BITNUM(in, 30, 3) | BITNUM(in,
22, 2) | BITNUM(in, 14, 1) | BITNUM(in, 6, 0);
98 }
99
100 void InvIP(WORD state[], BYTE in[])
101 {
102     in[0] = BITNUMINTR(state[1], 7, 7) | BITNUMINTR(state[0],
7, 6) | BITNUMINTR(state[1], 15, 5) |
103     BITNUMINTR(state[0], 15, 4) | BITNUMIN-
TR(state[1], 23, 3) | BITNUMINTR(state[0], 23, 2) |
104     BITNUMINTR(state[1], 31, 1) | BITNUMIN-
TR(state[0], 31, 0);
105
106     in[1] = BITNUMINTR(state[1], 6, 7) | BITNUMINTR(state[0],
6, 6) | BITNUMINTR(state[1], 14, 5) |
107     BITNUMINTR(state[0], 14, 4) | BITNUMIN-
TR(state[1], 22, 3) | BITNUMINTR(state[0], 22, 2) |
108     BITNUMINTR(state[1], 30, 1) | BITNUMIN-
TR(state[0], 30, 0);
109
110     in[2] = BITNUMINTR(state[1], 5, 7) | BITNUMINTR(state[0],
5, 6) | BITNUMINTR(state[1], 13, 5) |
111     BITNUMINTR(state[0], 13, 4) | BITNUMIN-
TR(state[1], 21, 3) | BITNUMINTR(state[0], 21, 2) |
112     BITNUMINTR(state[1], 29, 1) | BITNUMIN-
TR(state[0], 29, 0);
113
114     in[3] = BITNUMINTR(state[1], 4, 7) | BITNUMINTR(state[0],
4, 6) | BITNUMINTR(state[1], 12, 5) |
115     BITNUMINTR(state[0], 12, 4) | BITNUMIN-
TR(state[1], 20, 3) | BITNUMINTR(state[0], 20, 2) |
116     BITNUMINTR(state[1], 28, 1) | BITNUMIN-
TR(state[0], 28, 0);
117
118     in[4] = BITNUMINTR(state[1], 3, 7) | BITNUMINTR(state[0],
3, 6) | BITNUMINTR(state[1], 11, 5) |
119     BITNUMINTR(state[0], 11, 4) | BITNUMIN-
TR(state[1], 19, 3) | BITNUMINTR(state[0], 19, 2) |
120     BITNUMINTR(state[1], 27, 1) | BITNUMIN-
TR(state[0], 27, 0);
121
122     in[5] = BITNUMINTR(state[1], 2, 7) | BITNUMINTR(state[0],
2, 6) | BITNUMINTR(state[1], 10, 5) |
123     BITNUMINTR(state[0], 10, 4) | BITNUMIN-
TR(state[1], 18, 3) | BITNUMINTR(state[0], 18, 2) |
124     BITNUMINTR(state[1], 26, 1) | BITNUMIN-
TR(state[0], 26, 0);
125
126     in[6] = BITNUMINTR(state[1], 1, 7) | BITNUMINTR(state[0],
1, 6) | BITNUMINTR(state[1], 9, 5) |
127     BITNUMINTR(state[0], 9, 4) | BITNUMIN-
TR(state[1], 17, 3) | BITNUMINTR(state[0], 17, 2) |
128     BITNUMINTR(state[1], 25, 1) | BITNUMIN-
TR(state[0], 25, 0);
129
130     in[7] = BITNUMINTR(state[1], 0, 7) | BITNUMINTR(state[0],
0, 6) | BITNUMINTR(state[1], 8, 5) |
131     BITNUMINTR(state[0], 8, 4) | BITNUMIN-
TR(state[1], 16, 3) | BITNUMINTR(state[0], 16, 2) |
132     BITNUMINTR(state[1], 24, 1) | BITNUMIN-
TR(state[0], 24, 0);
133 }
134
135 WORD f(WORD state, const BYTE key[])
136 {
137     BYTE lrgstate[6]; //, i;
138     WORD t1, t2;
139
140     // Expansion Permutation
141     t1 = BITNUMINTL(state, 31, 0) | ((state & 0xf0000000) >>
1) | BITNUMINTL(state, 4, 5) |
142     BITNUMINTL(state, 3, 6) | ((state & 0xf0000000)
>> 3) | BITNUMINTL(state, 8, 11) |
143     BITNUMINTL(state, 7, 12) | ((state &
0x00f00000) >> 5) | BITNUMINTL(state, 12, 17) |
144     BITNUMINTL(state, 11, 18) | ((state &
0x000f0000) >> 7) | BITNUMINTL(state, 16, 23);
145
146     t2 = BITNUMINTL(state, 15, 0) | ((state & 0x0000f000) <<
15) | BITNUMINTL(state, 20, 5) |
147     BITNUMINTL(state, 19, 6) | ((state &
0x00000f00) << 13) | BITNUMINTL(state, 24, 11) |
148     BITNUMINTL(state, 23, 12) | ((state &
0x000000f0) << 11) | BITNUMINTL(state, 28, 17) |
149     BITNUMINTL(state, 27, 18) | ((state &
0x0000000f) << 9) | BITNUMINTL(state, 0, 23);
150
151     lrgstate[0] = (t1 >> 24) & 0x000000ff;
152     lrgstate[1] = (t1 >> 16) & 0x000000ff;
153     lrgstate[2] = (t1 >> 8) & 0x000000ff;
154     lrgstate[3] = (t2 >> 24) & 0x000000ff;

```

```

155     lrgstate[4] = (t2 >> 16) & 0x000000ff;
156     lrgstate[5] = (t2 >> 8) & 0x000000ff;
157
158     // Key XOR
159     lrgstate[0] ^= key[0];
160     lrgstate[1] ^= key[1];
161     lrgstate[2] ^= key[2];
162     lrgstate[3] ^= key[3];
163     lrgstate[4] ^= key[4];
164     lrgstate[5] ^= key[5];
165
166     // S-Box Permutation
167     state = (sbox1[SBOXBIT(lrgstate[0] >> 2)] << 28) |
168             (sbox2[SBOXBIT(((lrgstate[0] & 0x03)
169 << 4) | (lrgstate[1] >> 4))] << 24) |
170             (sbox3[SBOXBIT(((lrgstate[1] & 0x0f)
171 << 2) | (lrgstate[2] >> 6))] << 20) |
172             (sbox4[SBOXBIT(lrgstate[2] & 0x3f)]
173 << 16) |
174             (sbox5[SBOXBIT(lrgstate[3] >> 2)] <<
175 12) |
176             (sbox6[SBOXBIT(((lrgstate[3] & 0x03)
177 << 4) | (lrgstate[4] >> 4))] << 8) |
178             (sbox7[SBOXBIT(((lrgstate[4] & 0x0f)
179 << 2) | (lrgstate[5] >> 6))] << 4) |
180             sbox8[SBOXBIT(lrgstate[5] &
181 0x3f)]);
182
183     // P-Box Permutation
184     state = BITNUMINTL(state,15,0) | BITNUMINTL(state,6,1)
185 | BITNUMINTL(state,19,2) |
186     BITNUMINTL(state,20,3) | BITNU-
187 MINTL(state,28,4) | BITNUMINTL(state,11,5) |
188     BITNUMINTL(state,27,6) | BITNU-
189 MINTL(state,16,7) | BITNUMINTL(state,0,8) |
190     BITNUMINTL(state,14,9) | BITNU-
191 MINTL(state,22,10) | BITNUMINTL(state,25,11) |
192     BITNUMINTL(state,4,12) | BITNU-
193 MINTL(state,17,13) | BITNUMINTL(state,30,14) |
194     BITNUMINTL(state,9,15) | BITNU-
195 MINTL(state,1,16) | BITNUMINTL(state,7,17) |
196     BITNUMINTL(state,23,18) | BITNU-
197 MINTL(state,13,19) | BITNUMINTL(state,31,20) |
198     BITNUMINTL(state,26,21) | BITNU-
199 MINTL(state,2,22) | BITNUMINTL(state,8,23) |
200     BITNUMINTL(state,18,24) | BITNU-
201 MINTL(state,12,25) | BITNUMINTL(state,29,26) |
202     BITNUMINTL(state,5,27) | BITNU-
203 MINTL(state,21,28) | BITNUMINTL(state,10,29) |
204     BITNUMINTL(state,3,30) | BITNU-
205 MINTL(state,24,31);
206
207     // Return the final state value
208     return(state);
209 }
210
211 void des_key_setup(const BYTE key[], BYTE schedule[][6],
212 DES_MODE mode)
213 {
214     WORD i, j, to_gen, C, D;
215     const WORD key_rnd_shift[16] =
216 {1,1,2,2,2,2,2,2,1,2,2,2,2,2,1};
217     const WORD key_perm_c[28] =
218 {56,48,40,32,24,16,8,0,57,49,41,33,25,17,
219 9,1,58,50,42,34,26,18,10,2,59,51,43,35};
220     const WORD key_perm_d[28] =
221 {62,54,46,38,30,22,14,6,61,53,45,37,29,21,
222 13,5,60,52,44,36,28,20,12,4,27,19,11,3};
223     const WORD key_compression[48] =
224 {13,16,10,23,0,4,2,27,14,5,20,9,
225 22,18,11,3,25,7,15,6,26,19,12,1,
226 40,51,30,36,46,54,29,39,50,44,32,47,
227 43,48,38,55,33,52,45,41,49,35,28,31};
228
229     // Permuted Choice #1 (copy the key in, ignoring par-
230 ity bits).
231     for (i = 0, j = 31, C = 0; i < 28; ++i, --j)
232         C |= BITNUM(key, key_perm_c[i], j);
233     for (i = 0, j = 31, D = 0; i < 28; ++i, --j)
234         D |= BITNUM(key, key_perm_d[i], j);
235
236     // Generate the 16 subkeys.
237     for (i = 0; i < 16; ++i) {
238         C = ((C << key_rnd_shift[i]) | (C >> (28-
239 key_rnd_shift[i]))) & 0xffffffff;
240         D = ((D << key_rnd_shift[i]) | (D >> (28-
241 key_rnd_shift[i]))) & 0xffffffff;
242
243         // Decryption subkeys are reverse order of en-
244 cryptation subkeys so
245
246         // generate them in reverse if the key schedule
247 is for decryption useage.
248         if (mode == DES_DECRYPT)
249             to_gen = 15 - i;
250         else /*(if mode == DES_ENCRYPT)*/

```

```

222         to_gen = i;
223         // Initialize the array
224         for (j = 0; j < 6; ++j)
225             schedule[to_gen][j] = 0;
226         for (j = 0; j < 24; ++j)
227             schedule[to_gen][j/8] |= BITNUMIN-
228 TR(C, key_compression[j], 7 - (j%8));
229         for (j < 48; ++j)
230             schedule[to_gen][j/8] |= BITNUMIN-
231 TR(D, key_compression[j] - 28, 7 - (j%8));
232     }
233 }
234 void des_crypt(const BYTE in[], BYTE out[], const BYTE key[]
235 [6])
236 {
237     WORD state[2], idx, t;
238     IP(state, in);
239     for (idx=0; idx < 15; ++idx) {
240         t = state[1];
241         state[1] = f(state[1], key[idx]) ^ state[0];
242         state[0] = t;
243     }
244
245     // Perform the final loop manually as it doesn't switch
246 sides
247     state[0] = f(state[1], key[15]) ^ state[0];
248     InvIP(state, out);
249 }
250 void three_des_key_setup(const BYTE key[], BYTE schedule[][16]
251 [6], DES_MODE mode)
252 {
253     if (mode == DES_ENCRYPT) {
254         des_key_setup(&key[0], schedule[0], mode);
255         des_key_setup(&key[8], schedule[1], !mode);
256         des_key_setup(&key[16], schedule[2], mode);
257     }
258     else /*if (mode == DES_DECRYPT)*/ {
259         des_key_setup(&key[16], schedule[0], mode);
260         des_key_setup(&key[8], schedule[1], !mode);
261         des_key_setup(&key[0], schedule[2], mode);
262     }
263 }
264 void three_des_crypt(const BYTE in[], BYTE out[], const BYTE
265 key[][16][6])
266 {
267     des_crypt(in, out, key[0]);
268     des_crypt(out, out, key[1]);
269     des_crypt(out, out, key[2]);
270 }

```

14. B-Con_crypto-algorithms/des.h (37 lines)

```

1 /
2 *****
3
4 * Filename:   des.h
5 * Author:    Brad Conte (brad AT bradconte.com)
6 * Copyright:
7 * Disclaimer: This code is presented "as is" without any guar-
8 antees.
9 * Details:   Defines the API for the corresponding DES imple-
10 mentation.
11
12 Note that encryption and decryption are defined
13 by how
14 the key setup is performed, the actual en/de-
15 cryptation is
16 performed by the same function.
17
18 *****
19 */
20
21 #ifndef DES_H
22 #define DESH
23
24 /***** HEADER FILES *****/
25 #include <stddef.h>
26
27 /***** MACROS *****/
28 #define DES_BLOCK_SIZE 8 // DES operates on 8
29 bytes at a time
30
31 /***** DATA TYPES *****/
32 typedef unsigned char BYTE; // 8-bit byte
33
34 typedef unsigned int WORD; // 32-bit word, change to
35 "long" for 16-bit machines
36
37 typedef enum {
38     DES_ENCRYPT,
39     DES_DECRYPT
40 } DES_MODE;

```

```

29
30 /***** FUNCTION DECLARATIONS *****/
31 void des_key_setup(const BYTE key[], BYTE schedule[][6],
DES_MODE mode);
32 void des_crypt(const BYTE in[], BYTE out[], const BYTE key[]
[6]);
33
34 void three_des_key_setup(const BYTE key[], BYTE schedule[][16]
[6], DES_MODE mode);
35 void three_des_crypt(const BYTE in[], BYTE out[], const BYTE
key[][16][6]);
36
37 #endif // DES_H

```

15. B-Con_crypto-algorithms/des_test.c (83 lines)

```

1 /
*****
2 * Filename:   des_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Performs known-answer tests on the corresponding DES
implementation. These tests do not encompass
the full
8             range of available test vectors, however, if
the tests
9             pass it is very, very likely that the code is
correct
10            and was compiled properly. This code also
serves as
11            example usage of the functions.
12
*****
*/
13
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <memory.h>
17 #include "des.h"
18
19 /***** FUNCTION DEFINITIONS *****/
20 int des_test()
21 {
22     BYTE pt1[DES_BLOCK_SIZE] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xE7};
23     BYTE pt2[DES_BLOCK_SIZE] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF};
24     BYTE pt3[DES_BLOCK_SIZE] =
{0x54,0x68,0x65,0x20,0x71,0x75,0x66,0x63};
25     BYTE ct1[DES_BLOCK_SIZE] = {0xc9,0x57,0x44,0x25,0x6a,
0x5e,0xd3,0x1d};
26     BYTE ct2[DES_BLOCK_SIZE] = {0x85,0xe8,0x13,0x54,0x0f,
0x0a,0xb4,0x05};
27     BYTE ct3[DES_BLOCK_SIZE] = {0xc9,0x57,0x44,0x25,0x6a,
0x5e,0xd3,0x1d};
28     BYTE ct4[DES_BLOCK_SIZE] = {0xA8,0x26,0xFD,0x8C,
0xE5,0x3B,0x85,0x5F};
29     BYTE key1[DES_BLOCK_SIZE] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF};
30     BYTE key2[DES_BLOCK_SIZE] = {0x13,0x34,0x57,0x79,0x9B,
0xBC,0xDF,0xF1};
31     BYTE three_key1[DES_BLOCK_SIZE * 3] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,
0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,
0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF};
34     BYTE three_key2[DES_BLOCK_SIZE * 3] =
{0x01,0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,
0x23,0x45,0x67,0x89,0xAB,0xCD,0xEF,0x01,
0x45,0x67,0x89,0xAB,0xCD,0xEF,0x01,0x23};
36
37     BYTE schedule[16][6];
38     BYTE three_schedule[3][16][6];
39     BYTE buf[DES_BLOCK_SIZE];
40     int pass = 1;
41
42     des_key_setup(key1, schedule, DES_ENCRYPT);
43     des_crypt(pt1, buf, schedule);
44     pass = pass && !memcmp(ct1, buf, DES_BLOCK_SIZE);
45
46     des_key_setup(key1, schedule, DES_DECRYPT);
47     des_crypt(ct1, buf, schedule);
48     pass = pass && !memcmp(pt1, buf, DES_BLOCK_SIZE);
49
50     des_key_setup(key2, schedule, DES_ENCRYPT);
51     des_crypt(pt2, buf, schedule);
52     pass = pass && !memcmp(ct2, buf, DES_BLOCK_SIZE);
53
54     des_key_setup(key2, schedule, DES_DECRYPT);
55

```

```

56     des_crypt(ct2, buf, schedule);
57     pass = pass && !memcmp(pt2, buf, DES_BLOCK_SIZE);
58
59     three_des_key_setup(three_key1, three_schedule, DES_EN-
CRYPT);
60     three_des_crypt(pt1, buf, three_schedule);
61     pass = pass && !memcmp(ct3, buf, DES_BLOCK_SIZE);
62
63     three_des_key_setup(three_key1, three_schedule, DES_DE-
CRYPT);
64     three_des_crypt(ct3, buf, three_schedule);
65     pass = pass && !memcmp(pt1, buf, DES_BLOCK_SIZE);
66
67     three_des_key_setup(three_key2, three_schedule, DES_EN-
CRYPT);
68     three_des_crypt(pt3, buf, three_schedule);
69     pass = pass && !memcmp(ct4, buf, DES_BLOCK_SIZE);
70
71     three_des_key_setup(three_key2, three_schedule, DES_DE-
CRYPT);
72     three_des_crypt(ct4, buf, three_schedule);
73     pass = pass && !memcmp(pt3, buf, DES_BLOCK_SIZE);
74
75     return(pass);
76 }
77
78 int main()
79 {
80     printf("DES test: %s\n", des_test() ? "SUCCEEDED" :
"FAILED");
81
82     return(0);
83 }

```

16. B-Con_crypto-algorithms/md2.c (104 lines)

```

1 /
*****
2 * Filename:   md2.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the MD2 hashing algorithm.
Algorithm specification can
be found here:
8             * http://tools.ietf.org/
html/rfc1319 .
9             Input is little endian byte order.
10
*****
*/
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "md2.h"
16
17 /***** VARIABLES *****/
18 static const BYTE s[256] = {
19     41, 46, 67, 201, 162, 216, 124, 1, 61, 54, 84, 161,
236, 240, 6,
20     19, 98, 167, 5, 243, 192, 199, 115, 140, 152, 147, 43,
217, 188,
21     76, 130, 202, 30, 155, 87, 60, 253, 212, 224, 22, 103,
66, 111, 24,
22     138, 23, 229, 18, 190, 78, 196, 214, 218, 158, 222, 73,
160, 251,
23     245, 142, 187, 47, 238, 122, 169, 104, 121, 145, 21,
178, 7, 63,
24     148, 194, 16, 137, 11, 34, 95, 33, 128, 127, 93, 154,
90, 144, 50,
25     39, 53, 62, 204, 231, 191, 247, 151, 3, 255, 25, 48,
179, 72, 165,
26     181, 209, 215, 94, 146, 42, 172, 86, 170, 198, 79, 184,
56, 210,
27     150, 164, 125, 182, 118, 252, 107, 226, 156, 116, 4,
241, 69, 157,
28     112, 89, 100, 113, 135, 32, 134, 91, 207, 101, 230, 45,
168, 2, 27,
29     96, 37, 173, 174, 176, 185, 246, 28, 70, 97, 105, 52,
64, 126, 15,
30     85, 71, 163, 35, 221, 81, 175, 58, 195, 92, 249, 206,
186, 197,
31     234, 38, 44, 83, 13, 110, 133, 40, 132, 9, 211, 223,
205, 244, 65,
32     129, 77, 82, 106, 220, 55, 200, 108, 193, 171, 250, 36,
225, 123,
33     8, 12, 189, 177, 74, 120, 136, 149, 139, 227, 99, 232,
109, 233,
34     203, 213, 254, 59, 0, 29, 57, 242, 239, 183, 14, 102,
88, 208, 228,
35     166, 119, 114, 248, 235, 117, 75, 10, 49, 68, 80, 180,

```



```

143, 237,
36      31, 26, 219, 153, 141, 51, 159, 17, 131, 20
37  };
38
39  /***** FUNCTION DEFINITIONS *****/
40  void md2_transform(MD2_CTX *ctx, BYTE data[])
41  {
42      int j,k,t;
43
44      //memcpy(&ctx->state[16], data);
45      for (j=0; j < 16; ++j) {
46          ctx->state[j + 16] = data[j];
47          ctx->state[j + 32] = (ctx->state[j+16] ^ ctx-
>state[j]);
48      }
49
50      t = 0;
51      for (j = 0; j < 18; ++j) {
52          for (k = 0; k < 48; ++k) {
53              ctx->state[k] ^= s[t];
54              t = ctx->state[k];
55          }
56          t = (t+j) & 0xFF;
57      }
58
59      t = ctx->checksum[15];
60      for (j=0; j < 16; ++j) {
61          ctx->checksum[j] ^= s[data[j] ^ t];
62          t = ctx->checksum[j];
63      }
64  }
65
66  void md2_init(MD2_CTX *ctx)
67  {
68      int i;
69
70      for (i=0; i < 48; ++i)
71          ctx->state[i] = 0;
72      for (i=0; i < 16; ++i)
73          ctx->checksum[i] = 0;
74      ctx->len = 0;
75  }
76
77  void md2_update(MD2_CTX *ctx, const BYTE data[], size_t len)
78  {
79      size_t i;
80
81      for (i = 0; i < len; ++i) {
82          ctx->data[ctx->len] = data[i];
83          ctx->len++;
84          if (ctx->len == MD2_BLOCK_SIZE) {
85              md2_transform(ctx, ctx->data);
86              ctx->len = 0;
87          }
88      }
89  }
90
91  void md2_final(MD2_CTX *ctx, BYTE hash[])
92  {
93      int to_pad;
94
95      to_pad = MD2_BLOCK_SIZE - ctx->len;
96
97      while (ctx->len < MD2_BLOCK_SIZE)
98          ctx->data[ctx->len++] = to_pad;
99
100     md2_transform(ctx, ctx->data);
101     md2_transform(ctx, ctx->checksum);
102
103     memcpy(hash, ctx->state, MD2_BLOCK_SIZE);
104 }

```

17. B-Con_crypto-algorithms/md2.h (33 lines)

```

1  /
2  * Filename:  md2.h
3  * Author:    Brad Conte (brad AT bradconte.com)
4  * Copyright:
5  * Disclaimer: This code is presented "as is" without any guar-
antees.
6  * Details:   Defines the API for the corresponding MD2 imple-
mentation.
7
8  /*****
9  #ifndef MD2_H
10 #define MD2_H
11
12 /***** HEADER FILES *****/
13 #include <stddef.h>
14
15 /***** MACROS *****/
16 #define MD2_BLOCK_SIZE 16
17

```

```

18 /***** DATA TYPES *****/
19 typedef unsigned char BYTE;          // 8-bit byte
20
21 typedef struct {
22     BYTE data[16];
23     BYTE state[48];
24     BYTE checksum[16];
25     int len;
26 } MD2_CTX;
27
28 /***** FUNCTION DECLARATIONS *****/
29 void md2_init(MD2_CTX *ctx);
30 void md2_update(MD2_CTX *ctx, const BYTE data[], size_t len);
31 void md2_final(MD2_CTX *ctx, BYTE hash[]); // size of hash
must be MD2_BLOCK_SIZE
32
33 #endif // MD2_H

```

18. B-Con_crypto-algorithms/md2_test.c (58 lines)

```

1  /
2  * Filename:  md2_test.c
3  * Author:    Brad Conte (brad AT bradconte.com)
4  * Copyright:
5  * Disclaimer: This code is presented "as is" without any guar-
antees.
6
7  * Details:   Performs known-answer tests on the corresponding MD2
the full
8                  implementation. These tests do not encompass
the tests
9                  range of available test vectors, however, if
correct
10                   pass it is very, very likely that the code is
serves as
11                   and was compiled properly. This code also
12                   example usage of the functions.
13
14 /*****
15 #include <stdio.h>
16 #include <string.h>
17 #include <memory.h>
18 #include "md2.h"
19
20 /***** FUNCTION DEFINITIONS *****/
21 int md2_test()
22 {
23     BYTE text1[] = {"abc"};
24     BYTE text2[] = {"abcdefghijklmnopqrstuvwxyz"};
25     BYTE text3_1[] = {"ABCDEFGHIJKLMNOPQRSTUVWXYZabcde"};
26     BYTE text3_2[] = {"fghijklmnopqrstuvwxyz0123456789"};
27     BYTE hash1[MD2_BLOCK_SIZE] = {0xda, 0x85, 0x3b, 0x0d, 0x3f,
0xb8, 0xd9, 0x9b, 0x30, 0x28, 0x3a, 0x69, 0xe6, 0xde, 0xd6, 0xbb};
28     BYTE hash2[MD2_BLOCK_SIZE] = {0x4e, 0x8d, 0xdf,
0xf3, 0x65, 0x02, 0x92, 0xab, 0x5a, 0x41, 0x08, 0xc3, 0xaa, 0x47, 0x94, 0x0b};
29     BYTE hash3[MD2_BLOCK_SIZE] = {0xda, 0x33, 0xde,
0xf2, 0xa4, 0x2d, 0xf1, 0x39, 0x75, 0x35, 0x28, 0x46, 0xc3, 0x03, 0x38, 0xcd};
30     BYTE buf[16];
31     MD2_CTX ctx;
32     int pass = 1;
33
34     md2_init(&ctx);
35     md2_update(&ctx, text1, strlen(text1));
36     md2_final(&ctx, buf);
37     pass = pass && !memcmp(hash1, buf, MD2_BLOCK_SIZE);
38
39     // Note that the MD2 object can be re-used.
40     md2_init(&ctx);
41     md2_update(&ctx, text2, strlen(text2));
42     md2_final(&ctx, buf);
43     pass = pass && !memcmp(hash2, buf, MD2_BLOCK_SIZE);
44
45     // Note that the data is added in two chunks.
46     md2_init(&ctx);
47     md2_update(&ctx, text3_1, strlen(text3_1));
48     md2_update(&ctx, text3_2, strlen(text3_2));
49     md2_final(&ctx, buf);
50     pass = pass && !memcmp(hash3, buf, MD2_BLOCK_SIZE);
51
52     return(pass);
53 }
54
55 int main()
56 {
57     printf("MD2 tests: %s\n", md2_test() ? "SUCCEEDED" :
"FAILED");
58 }

```

19. B-Con_crypto-algorithms/md5.c (189 lines)

```

1 /
*****
2 * Filename:   md5.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:   Implementation of the MD5 hashing algorithm.
7             Algorithm specification can
be found here:
8             * http://tools.ietf.org/
html/rfc1321
9             This implementation uses
little endian byte order.
10
*****
*/
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "md5.h"
16
17 /***** MACROS *****/
18 #define ROTLEFT(a,b) ((a << b) | (a >> (32-b)))
19
20 #define F(x,y,z) ((x & y) | (~x & z))
21 #define G(x,y,z) ((x & z) | (y & ~z))
22 #define H(x,y,z) (x ^ y ^ z)
23 #define I(x,y,z) (y ^ (x | ~z))
24
25 #define FF(a,b,c,d,m,s,t) { a += F(b,c,d) + m + t; \
26                             a = b + ROTLEFT(a,s); }
27 #define GG(a,b,c,d,m,s,t) { a += G(b,c,d) + m + t; \
28                             a = b + ROTLEFT(a,s); }
29 #define HH(a,b,c,d,m,s,t) { a += H(b,c,d) + m + t; \
30                             a = b + ROTLEFT(a,s); }
31 #define II(a,b,c,d,m,s,t) { a += I(b,c,d) + m + t; \
32                             a = b + ROTLEFT(a,s); }
33
34 /***** FUNCTION DEFINITIONS *****/
35 void md5_transform(MD5_CTX *ctx, const BYTE data[])
36 {
37     WORD a, b, c, d, m[16], i, j;
38
39     // MD5 specifies big endian byte order, but this imple-
40     // mentation assumes a little
41     // endian byte order CPU. Reverse all the bytes upon
42     // input, and re-reverse them
43     // on output (in md5_final()).
44     for (i = 0, j = 0; i < 16; ++i, j += 4)
45     {
46         m[i] = (data[j] + (data[j + 1] << 8) + (data[j +
47         2] << 16) + (data[j + 3] << 24));
48
49         a = ctx->state[0];
50         b = ctx->state[1];
51         c = ctx->state[2];
52         d = ctx->state[3];
53
54         FF(a,b,c,d,m[0], 7,0xd76aa478);
55         FF(d,a,b,c,m[1], 12,0xe8c7b756);
56         FF(c,d,a,b,m[2], 17,0x242070db);
57         FF(b,c,d,a,m[3], 22,0xc1bdcee5);
58         FF(a,b,c,d,m[4], 7,0xf57c0faf);
59         FF(d,a,b,c,m[5], 12,0x4787c62a);
60         FF(c,d,a,b,m[6], 17,0xa8304613);
61         FF(b,c,d,a,m[7], 22,0xfd469501);
62         FF(a,b,c,d,m[8], 7,0x698098d8);
63         FF(d,a,b,c,m[9], 12,0x8b44f7af);
64         FF(c,d,a,b,m[10],17,0xffff5bb1);
65         FF(b,c,d,a,m[11],22,0x895cd7be);
66         FF(a,b,c,d,m[12], 7,0x6b901122);
67         FF(d,a,b,c,m[13],12,0xfd987193);
68         FF(c,d,a,b,m[14],17,0xa679438e);
69         FF(b,c,d,a,m[15],22,0x49b40821);
70
71         GG(a,b,c,d,m[1], 5,0xf61e2562);
72         GG(d,a,b,c,m[6], 9,0xc940b340);
73         GG(c,d,a,b,m[11],14,0x265e5a51);
74         GG(b,c,d,a,m[0], 20,0xe9b6c7aa);
75         GG(a,b,c,d,m[5], 5,0xd62f105d);
76         GG(d,a,b,c,m[10], 9,0x02441453);
77         GG(c,d,a,b,m[15],14,0xd8a1e681);
78         GG(b,c,d,a,m[4], 20,0xe7d3fbc8);
79         GG(a,b,c,d,m[9], 5,0x21e1cde6);
80         GG(d,a,b,c,m[14], 9,0xc33707d6);
81         GG(c,d,a,b,m[3], 14,0xf4d50d87);
82         GG(b,c,d,a,m[8], 20,0xa55a14ed);
83         GG(a,b,c,d,m[13], 5,0xa9e3e905);
84         GG(d,a,b,c,m[2], 9,0xfcfa3f8);
85         GG(c,d,a,b,m[7], 14,0x676f02d9);
86         GG(b,c,d,a,m[12],20,0x8d2a4c8a);

```

```

83
84     HH(a,b,c,d,m[5], 4,0xffff3942);
85     HH(d,a,b,c,m[8], 11,0x8771f681);
86     HH(c,d,a,b,m[11],16,0x6d9d6122);
87     HH(b,c,d,a,m[14],23,0xfde5380c);
88     HH(a,b,c,d,m[1], 4,0xa4bee44);
89     HH(d,a,b,c,m[4], 11,0x4bdecfa9);
90     HH(c,d,a,b,m[7], 16,0xf6bb4b60);
91     HH(b,c,d,a,m[10],23,0xbee5fbc7);
92     HH(a,b,c,d,m[13], 4,0x289b7ec6);
93     HH(d,a,b,c,m[0], 11,0xeaa127fa);
94     HH(c,d,a,b,m[3], 16,0xd4ef3085);
95     HH(b,c,d,a,m[6], 23,0x04881d05);
96     HH(a,b,c,d,m[9], 4,0xd9d4d039);
97     HH(d,a,b,c,m[12],11,0xe6db99e5);
98     HH(c,d,a,b,m[15],16,0x1fa27cf8);
99     HH(b,c,d,a,m[2], 23,0xc4ac5665);
100
101     II(a,b,c,d,m[0], 6,0xf4292244);
102     II(d,a,b,c,m[7], 10,0x432aff97);
103     II(c,d,a,b,m[14],15,0xab9423a7);
104     II(b,c,d,a,m[5], 21,0xfc93a039);
105     II(a,b,c,d,m[12], 6,0x655b59c3);
106     II(d,a,b,c,m[3], 10,0x8f6ccc92);
107     II(c,d,a,b,m[10],15,0xffefff47d);
108     II(b,c,d,a,m[1], 21,0x85845dd1);
109     II(a,b,c,d,m[8], 6,0x6fa87e4f);
110     II(d,a,b,c,m[15],10,0xfe2ce6e0);
111     II(c,d,a,b,m[6], 15,0xa3014314);
112     II(b,c,d,a,m[13],21,0x4e0811a1);
113     II(a,b,c,d,m[4], 6,0xf7537e82);
114     II(d,a,b,c,m[11],10,0xbd3af235);
115     II(c,d,a,b,m[2], 15,0x2ad7d2bb);
116     II(b,c,d,a,m[9], 21,0xeb86d391);
117
118     ctx->state[0] += a;
119     ctx->state[1] += b;
120     ctx->state[2] += c;
121     ctx->state[3] += d;
122 }
123
124 void md5_init(MD5_CTX *ctx)
125 {
126     ctx->datalen = 0;
127     ctx->bitlen = 0;
128     ctx->state[0] = 0x67452301;
129     ctx->state[1] = 0xefcdab89;
130     ctx->state[2] = 0x98badcf5;
131     ctx->state[3] = 0x10325476;
132 }
133
134 void md5_update(MD5_CTX *ctx, const BYTE data[], size_t len)
135 {
136     size_t i;
137
138     for (i = 0; i < len; ++i) {
139         ctx->data[ctx->datalen] = data[i];
140         ctx->datalen++;
141         if (ctx->datalen == 64) {
142             md5_transform(ctx, ctx->data);
143             ctx->bitlen += 512;
144             ctx->datalen = 0;
145         }
146     }
147 }
148
149 void md5_final(MD5_CTX *ctx, BYTE hash[])
150 {
151     size_t i;
152
153     i = ctx->datalen;
154
155     // Pad whatever data is left in the buffer.
156     if (ctx->datalen < 56) {
157         ctx->data[i++] = 0x80;
158         while (i < 56)
159             ctx->data[i++] = 0x00;
160     }
161     else if (ctx->datalen == 56) {
162         ctx->data[i++] = 0x80;
163         while (i < 64)
164             ctx->data[i++] = 0x00;
165         md5_transform(ctx, ctx->data);
166         memset(ctx->data, 0, 56);
167     }
168
169     // Append to the padding the total message's length in
170     // bits and transform.
171     ctx->bitlen += ctx->datalen * 8;
172     ctx->data[56] = ctx->bitlen;
173     ctx->data[57] = ctx->bitlen >> 8;
174     ctx->data[58] = ctx->bitlen >> 16;
175     ctx->data[59] = ctx->bitlen >> 24;
176     ctx->data[60] = ctx->bitlen >> 32;
177     ctx->data[61] = ctx->bitlen >> 40;
178     ctx->data[62] = ctx->bitlen >> 48;
179     ctx->data[63] = ctx->bitlen >> 56;
180     md5_transform(ctx, ctx->data);
181
182     // Since this implementation uses little endian byte
183     // ordering and MD uses big endian,

```

```

182 // reverse all the bytes when copying the final state
to the output hash.
183 for (i = 0; i < 4; ++i) {
184     hash[i] = (ctx->state[0] >> (i * 8)) &
0x000000ff;
185     hash[i + 4] = (ctx->state[1] >> (i * 8)) &
0x000000ff;
186     hash[i + 8] = (ctx->state[2] >> (i * 8)) &
0x000000ff;
187     hash[i + 12] = (ctx->state[3] >> (i * 8)) &
0x000000ff;
188 }
189 }

```

20. B-Con_crypto-algorithms/md5.h (34 lines)

```

1 /
*****
2 * Filename:    md5.h
3 * Author:      Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-
antees.
6 * Details:     Defines the API for the corresponding MD5 imple-
mentation.
7
*****
*/
8
9 #ifndef MD5_H
10 #define MD5_H
11
12 /***** HEADER FILES *****/
13 #include <stdint.h>
14
15 /***** MACROS *****/
16 #define MD5_BLOCK_SIZE 16 // MD5 outputs a 16
byte digest
17
18 /***** DATA TYPES *****/
19 typedef unsigned char BYTE; // 8-bit byte
20
21 typedef unsigned int WORD; // 32-bit word, change to
"long" for 16-bit machines
22
23 typedef struct {
24     BYTE data[64];
25     WORD datalen;
26     unsigned long long bitlen;
27     WORD state[4];
28 } MD5_CTX;
29
30 /***** FUNCTION DECLARATIONS *****/
31 void md5_init(MD5_CTX *ctx);
32 void md5_update(MD5_CTX *ctx, const BYTE data[], size_t len);
33 void md5_final(MD5_CTX *ctx, BYTE hash[]);
34 #endif // MD5_H

```

21. B-Con_crypto-algorithms/md5_test.c (60 lines)

```

1 /
*****
2 * Filename:    md5_test.c
3 * Author:      Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-
antees.
6 * Details:     Performs known-answer tests on the corresponding MD5
implementation. These tests do not encompass
the full
7
8
9
10
11
12
13
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <memory.h>
17 #include <string.h>
18 #include "md5.h"
19
20 /***** FUNCTION DEFINITIONS *****/
21 int md5_test()

```

```

22 {
23     BYTE text1[] = {"";
24     BYTE text2[] = {"abc";
25     BYTE text3_1[] = {"ABCDEFGHJKLMNOPQRSTUVWXYZabcde";
26     BYTE text3_2[] = {"fghijklmnopqrstuvwxyz0123456789";
27     BYTE hash1[MD5_BLOCK_SIZE] = {0xd4, 0x1d, 0x8c, 0xd9, 0x8f,
0x00, 0xb2, 0x04, 0xe9, 0x80, 0x09, 0x98, 0xec, 0xf8, 0x42, 0xe7};
28     BYTE hash2[MD5_BLOCK_SIZE] = {0x90, 0x01, 0x50, 0x98, 0x3c,
0xd2, 0x4f, 0xb0, 0xd6, 0x96, 0x3f, 0x7d, 0x28, 0xe1, 0x7f, 0x72};
29     BYTE hash3[MD5_BLOCK_SIZE] = {0xd1, 0x74, 0xab,
0x98, 0xd2, 0x77, 0xd9, 0xf5, 0xa5, 0x61, 0x1c, 0x2c, 0x9f, 0x41, 0x9d, 0x9f};
30     BYTE buf[16];
31     MD5_CTX ctx;
32     int pass = 1;
33
34     md5_init(&ctx);
35     md5_update(&ctx, text1, strlen(text1));
36     md5_final(&ctx, buf);
37     pass = pass && !memcmp(hash1, buf, MD5_BLOCK_SIZE);
38
39     // Note the MD5 object can be reused.
40     md5_init(&ctx);
41     md5_update(&ctx, text2, strlen(text2));
42     md5_final(&ctx, buf);
43     pass = pass && !memcmp(hash2, buf, MD5_BLOCK_SIZE);
44
45     // Note the data is being added in two chunks.
46     md5_init(&ctx);
47     md5_update(&ctx, text3_1, strlen(text3_1));
48     md5_update(&ctx, text3_2, strlen(text3_2));
49     md5_final(&ctx, buf);
50     pass = pass && !memcmp(hash3, buf, MD5_BLOCK_SIZE);
51
52     return(pass);
53 }
54
55 int main()
56 {
57     printf("MD5 tests: %s\n", md5_test() ? "SUCCEEDED" :
"FAILED");
58
59     return(0);
60 }

```

22. B-Con_crypto-algorithms/rot-13.c (35 lines)

```

1 /
*****
2 * Filename:    rot-13.c
3 * Author:      Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer:   This code is presented "as is" without any guar-
antees.
6 * Details:     Implementation of the ROT-13 encryption al-
gorithm.
7
8
9
10
11
12 /***** HEADER FILES *****/
13 #include <string.h>
14 #include "rot-13.h"
15
16 /***** FUNCTION DEFINITIONS *****/
17 void rot13(char str[])
18 {
19     int case_type, idx, len;
20
21     for (idx = 0, len = strlen(str); idx < len; idx++) {
22         // Only process alphabetic characters.
23
24         if (str[idx] < 'A' || (str[idx] > 'Z' && str[idx] < 'a') ||
str[idx] > 'z')
25             continue;
26         // Determine if the char is upper or lower case.
27         if (str[idx] >= 'a')
28             case_type = 'a';
29         else
30             case_type = 'A';
31         // Rotate the char's value, ensuring it doesn't acci-
dentally "fall off" the end.
32         str[idx] = (str[idx] + 13) % (case_type + 26);
33         if (str[idx] < 26)
34             str[idx] += case_type;
35     }

```

23. B-Con_crypto-algorithms/rot-13.h (20 lines)

```

1 /
*****

```



```

2 * Filename:   rot-13.h
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Defines the API for the corresponding ROT-13 im-
plementation.
7
*****
*/
8
9 #ifndef ROT13_H
10 #define ROT13_H
11
12 /***** HEADER FILES *****/
13 #include <stddef.h>
14
15 /***** FUNCTION DECLARATIONS *****/
16 // Performs IN PLACE rotation of the input. Assumes input is
NULL terminated.
17 // Preserves each character's case. Ignores non alphabetic
characters.
18 void rot13(char str[]);
19
20 #endif // ROT13_H

```

24. B-Con_crypto-algorithms/rot-13_test.c (44 lines)

```

1 /
*****
*
2 * Filename:   rot-13_test.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Performs known-answer tests on the corresponding
ROT-13
7                  implementation. These tests do not encompass
the full
8                  range of available test vectors, however, if
the tests
9
10                 pass it is very, very likely that the code is
correct
11                 and was compiled properly. This code also
serves as
12                 example usage of the functions.
*****
*/
13
14 /***** HEADER FILES *****/
15 #include <stdio.h>
16 #include <string.h>
17 #include "rot-13.h"
18
19 /***** FUNCTION DEFINITIONS *****/
20 int rot13_test()
21 {
22     char text[] = {"ABCDEFGHIJKLMNQRSTUUVWXYZabcde-
fghijklmnopqrstuvwxyz"};
23     char code[] = {"NOPQRSTUVWXYZABCDEFGHIJKLMnopqr-
stuvwxyzabcdefghijklmnop"};
24     char buf[1024];
25     int pass = 1;
26
27     // To encode, just apply ROT-13.
28     strcpy(buf, text);
29     rot13(buf);
30     pass = pass && !strcmp(code, buf);
31
32     // To decode, just re-apply ROT-13.
33     rot13(buf);
34     pass = pass && !strcmp(text, buf);
35
36     return(pass);
37 }
38
39 int main()
40 {
41     printf("ROT-13 tests: %s\n", rot13_test() ? "SUC-
CEDED" : "FAILED");
42
43     return(0);
44 }

```

25. B-Con_crypto-algorithms/sha1.c (149 lines)

```

1 /
*****
*
2 * Filename:   sha1.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:

```

```

5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Implementation of the SHA1 hashing algorithm.
Algorithm specification can be found here:
7                * http://csrc.nist.gov/publications/fips/
fips180-2/fips180-2withchangenotice.pdf
8                This implementation uses little endian byte or-
der.
9
10 *****
11
12 /***** HEADER FILES *****/
13 #include <stdlib.h>
14 #include <memory.h>
15 #include "sha1.h"
16
17 /***** MACROS *****/
18 #define ROTLEFT(a, b) ((a << b) | (a >> (32 - b)))
19
20 /***** FUNCTION DEFINITIONS *****/
21 void sha1_transform(SHA1_CTX *ctx, const BYTE data[])
22 {
23     WORD a, b, c, d, e, i, j, t, m[80];
24
25     for (i = 0, j = 0; i < 16; ++i, j += 4)
26         m[i] = (data[j] << 24) + (data[j + 1] << 16) +
(data[j + 2] << 8) + (data[j + 3]);
27     for ( ; i < 80; ++i) {
28         m[i] = (m[i - 3] ^ m[i - 8] ^ m[i - 14] ^ m[i -
16]);
29         m[i] = (m[i] << 1) | (m[i] >> 31);
30     }
31
32     a = ctx->state[0];
33     b = ctx->state[1];
34     c = ctx->state[2];
35     d = ctx->state[3];
36     e = ctx->state[4];
37
38     for (i = 0; i < 20; ++i) {
39         t = ROTLEFT(a, 5) + ((b & c) ^ (~b & d)) + e +
ctx->k[0] + m[i];
40         e = d;
41         d = c;
42         c = ROTLEFT(b, 30);
43         b = a;
44         a = t;
45     }
46     for ( ; i < 40; ++i) {
47         t = ROTLEFT(a, 5) + (b ^ c ^ d) + e + ctx->k[1] +
m[i];
48         e = d;
49         d = c;
50         c = ROTLEFT(b, 30);
51         b = a;
52         a = t;
53     }
54     for ( ; i < 60; ++i) {
55         t = ROTLEFT(a, 5) + ((b & c) ^ (b & d) ^ (c &
d)) + e + ctx->k[2] + m[i];
56         e = d;
57         d = c;
58         c = ROTLEFT(b, 30);
59         b = a;
60         a = t;
61     }
62     for ( ; i < 80; ++i) {
63         t = ROTLEFT(a, 5) + (b ^ c ^ d) + e + ctx->k[3] +
m[i];
64         e = d;
65         d = c;
66         c = ROTLEFT(b, 30);
67         b = a;
68         a = t;
69     }
70
71     ctx->state[0] += a;
72     ctx->state[1] += b;
73     ctx->state[2] += c;
74     ctx->state[3] += d;
75     ctx->state[4] += e;
76 }
77
78 void sha1_init(SHA1_CTX *ctx)
79 {
80     ctx->datalen = 0;
81     ctx->bitlen = 0;
82     ctx->state[0] = 0x67452301;
83     ctx->state[1] = 0xEFCDAB89;
84     ctx->state[2] = 0x98BADCFE;
85     ctx->state[3] = 0x10325476;
86     ctx->state[4] = 0xC3D2E1F0;
87     ctx->k[0] = 0x5A827999;
88     ctx->k[1] = 0x6ED9EBA1;

```

```

89     ctx->k[2] = 0x8f1bbcdc;
90     ctx->k[3] = 0xca62c1d6;
91 }
92
93 void sha1_update(SHA1_CTX *ctx, const BYTE data[], size_t len)
94 {
95     size_t i;
96
97     for (i = 0; i < len; ++i) {
98         ctx->data[ctx->datalen] = data[i];
99         ctx->datalen++;
100         if (ctx->datalen == 64) {
101             sha1_transform(ctx, ctx->data);
102             ctx->bitlen += 512;
103             ctx->datalen = 0;
104         }
105     }
106 }
107
108 void sha1_final(SHA1_CTX *ctx, BYTE hash[])
109 {
110     WORD i;
111
112     i = ctx->datalen;
113
114     // Pad whatever data is left in the buffer.
115     if (ctx->datalen < 56) {
116         ctx->data[i++] = 0x80;
117         while (i < 56)
118             ctx->data[i++] = 0x00;
119     }
120     else {
121         ctx->data[i++] = 0x80;
122         while (i < 64)
123             ctx->data[i++] = 0x00;
124         sha1_transform(ctx, ctx->data);
125         memset(ctx->data, 0, 56);
126     }
127
128     // Append to the padding the total message's length in
129     bits and transform.
130     ctx->bitlen += ctx->datalen * 8;
131     ctx->data[63] = ctx->bitlen;
132     ctx->data[62] = ctx->bitlen >> 8;
133     ctx->data[61] = ctx->bitlen >> 16;
134     ctx->data[60] = ctx->bitlen >> 24;
135     ctx->data[59] = ctx->bitlen >> 32;
136     ctx->data[58] = ctx->bitlen >> 40;
137     ctx->data[57] = ctx->bitlen >> 48;
138     ctx->data[56] = ctx->bitlen >> 56;
139     sha1_transform(ctx, ctx->data);
140
141     // Since this implementation uses little endian byte
142     ordering and MD uses big endian,
143     // reverse all the bytes when copying the final state
144     to the output hash.
145     for (i = 0; i < 4; ++i) {
146         hash[i] = (ctx->state[0] >> (24 - i * 8))
147         & 0x000000ff;
148         hash[i + 4] = (ctx->state[1] >> (24 - i * 8))
149         & 0x000000ff;
150         hash[i + 8] = (ctx->state[2] >> (24 - i * 8))
151         & 0x000000ff;
152         hash[i + 12] = (ctx->state[3] >> (24 - i * 8))
153         & 0x000000ff;
154         hash[i + 16] = (ctx->state[4] >> (24 - i * 8))
155         & 0x000000ff;
156     }
157 }

```

26. B-Con_crypto-algorithms/sha1.h (35 lines)

```

1 /
2 *
3 * Filename: sha1.h
4 * Author: Brad Conte (brad AT bradconte.com)
5 * Copyright:
6 * Disclaimer: This code is presented "as is" without any guar-
7 antees.
8 * Details: Defines the API for the corresponding SHA1 im-
9 plementation.
10
11 /*****
12  *
13  *
14  *
15  *
16  *
17  *
18  *
19  *
20  *
21  *
22  *
23  *
24  *
25  *
26  *
27  *
28  *
29  *
30  *
31  *
32  *
33  *
34  *
35  *
36  *
37  *
38  *
39  *
40  *
41  *
42  *
43  *
44  *
45  *
46  *
47  *
48  *
49  *
50  *
51  *
52  *
53  *
54  *
55  *
56  *
57  *
58  *
59  *
60  *
61  *
62  *
63  *
64  *
65  *
66  *
67  *
68  *
69  *
70  *
71  *
72  *
73  *
74  *
75  *
76  *
77  *
78  *
79  *
80  *
81  *
82  *
83  *
84  *
85  *
86  *
87  *
88  *
89  *
90  *
91  *
92  *
93  *
94  *
95  *
96  *
97  *
98  *
99  *
100  *
101  *
102  *
103  *
104  *
105  *
106  *
107  *
108  *
109  *
110  *
111  *
112  *
113  *
114  *
115  *
116  *
117  *
118  *
119  *
120  *
121  *
122  *
123  *
124  *
125  *
126  *
127  *
128  *
129  *
130  *
131  *
132  *
133  *
134  *
135  *
136  *
137  *
138  *
139  *
140  *
141  *
142  *
143  *
144  *
145  *
146  *
147  *
148  *
149  *
150  *
151  *
152  *
153  *
154  *
155  *
156  *
157  *
158  *
159  *
160  *
161  *
162  *
163  *
164  *
165  *
166  *
167  *
168  *
169  *
170  *
171  *
172  *
173  *
174  *
175  *
176  *
177  *
178  *
179  *
180  *
181  *
182  *
183  *
184  *
185  *
186  *
187  *
188  *
189  *
190  *
191  *
192  *
193  *
194  *
195  *
196  *
197  *
198  *
199  *
200  *
201  *
202  *
203  *
204  *
205  *
206  *
207  *
208  *
209  *
210  *
211  *
212  *
213  *
214  *
215  *
216  *
217  *
218  *
219  *
220  *
221  *
222  *
223  *
224  *
225  *
226  *
227  *
228  *
229  *
230  *
231  *
232  *
233  *
234  *
235  *
236  *
237  *
238  *
239  *
240  *
241  *
242  *
243  *
244  *
245  *
246  *
247  *
248  *
249  *
250  *
251  *
252  *
253  *
254  *
255  *
256  *
257  *
258  *
259  *
260  *
261  *
262  *
263  *
264  *
265  *
266  *
267  *
268  *
269  *
270  *
271  *
272  *
273  *
274  *
275  *
276  *
277  *
278  *
279  *
280  *
281  *
282  *
283  *
284  *
285  *
286  *
287  *
288  *
289  *
290  *
291  *
292  *
293  *
294  *
295  *
296  *
297  *
298  *
299  *
300  *
301  *
302  *
303  *
304  *
305  *
306  *
307  *
308  *
309  *
310  *
311  *
312  *
313  *
314  *
315  *
316  *
317  *
318  *
319  *
320  *
321  *
322  *
323  *
324  *
325  *
326  *
327  *
328  *
329  *
330  *
331  *
332  *
333  *
334  *
335  *
336  *
337  *
338  *
339  *
340  *
341  *
342  *
343  *
344  *
345  *
346  *
347  *
348  *
349  *
350  *
351  *
352  *
353  *
354  *
355  *
356  *
357  *
358  *
359  *
360  *
361  *
362  *
363  *
364  *
365  *
366  *
367  *
368  *
369  *
370  *
371  *
372  *
373  *
374  *
375  *
376  *
377  *
378  *
379  *
380  *
381  *
382  *
383  *
384  *
385  *
386  *
387  *
388  *
389  *
390  *
391  *
392  *
393  *
394  *
395  *
396  *
397  *
398  *
399  *
400  *
401  *
402  *
403  *
404  *
405  *
406  *
407  *
408  *
409  *
410  *
411  *
412  *
413  *
414  *
415  *
416  *
417  *
418  *
419  *
420  *
421  *
422  *
423  *
424  *
425  *
426  *
427  *
428  *
429  *
430  *
431  *
432  *
433  *
434  *
435  *
436  *
437  *
438  *
439  *
440  *
441  *
442  *
443  *
444  *
445  *
446  *
447  *
448  *
449  *
450  *
451  *
452  *
453  *
454  *
455  *
456  *
457  *
458  *
459  *
460  *
461  *
462  *
463  *
464  *
465  *
466  *
467  *
468  *
469  *
470  *
471  *
472  *
473  *
474  *
475  *
476  *
477  *
478  *
479  *
480  *
481  *
482  *
483  *
484  *
485  *
486  *
487  *
488  *
489  *
490  *
491  *
492  *
493  *
494  *
495  *
496  *
497  *
498  *
499  *
500  *
501  *
502  *
503  *
504  *
505  *
506  *
507  *
508  *
509  *
510  *
511  *
512  *
513  *
514  *
515  *
516  *
517  *
518  *
519  *
520  *
521  *
522  *
523  *
524  *
525  *
526  *
527  *
528  *
529  *
530  *
531  *
532  *
533  *
534  *
535  *
536  *
537  *
538  *
539  *
540  *
541  *
542  *
543  *
544  *
545  *
546  *
547  *
548  *
549  *
550  *
551  *
552  *
553  *
554  *
555  *
556  *
557  *
558  *
559  *
560  *
561  *
562  *
563  *
564  *
565  *
566  *
567  *
568  *
569  *
570  *
571  *
572  *
573  *
574  *
575  *
576  *
577  *
578  *
579  *
580  *
581  *
582  *
583  *
584  *
585  *
586  *
587  *
588  *
589  *
590  *
591  *
592  *
593  *
594  *
595  *
596  *
597  *
598  *
599  *
600  *
601  *
602  *
603  *
604  *
605  *
606  *
607  *
608  *
609  *
610  *
611  *
612  *
613  *
614  *
615  *
616  *
617  *
618  *
619  *
620  *
621  *
622  *
623  *
624  *
625  *
626  *
627  *
628  *
629  *
630  *
631  *
632  *
633  *
634  *
635  *
636  *
637  *
638  *
639  *
640  *
641  *
642  *
643  *
644  *
645  *
646  *
647  *
648  *
649  *
650  *
651  *
652  *
653  *
654  *
655  *
656  *
657  *
658  *
659  *
660  *
661  *
662  *
663  *
664  *
665  *
666  *
667  *
668  *
669  *
670  *
671  *
672  *
673  *
674  *
675  *
676  *
677  *
678  *
679  *
680  *
681  *
682  *
683  *
684  *
685  *
686  *
687  *
688  *
689  *
690  *
691  *
692  *
693  *
694  *
695  *
696  *
697  *
698  *
699  *
700  *
701  *
702  *
703  *
704  *
705  *
706  *
707  *
708  *
709  *
710  *
711  *
712  *
713  *
714  *
715  *
716  *
717  *
718  *
719  *
720  *
721  *
722  *
723  *
724  *
725  *
726  *
727  *
728  *
729  *
730  *
731  *
732  *
733  *
734  *
735  *
736  *
737  *
738  *
739  *
740  *
741  *
742  *
743  *
744  *
745  *
746  *
747  *
748  *
749  *
750  *
751  *
752  *
753  *
754  *
755  *
756  *
757  *
758  *
759  *
760  *
761  *
762  *
763  *
764  *
765  *
766  *
767  *
768  *
769  *
770  *
771  *
772  *
773  *
774  *
775  *
776  *
777  *
778  *
779  *
780  *
781  *
782  *
783  *
784  *
785  *
786  *
787  *
788  *
789  *
790  *
791  *
792  *
793  *
794  *
795  *
796  *
797  *
798  *
799  *
800  *
801  *
802  *
803  *
804  *
805  *
806  *
807  *
808  *
809  *
810  *
811  *
812  *
813  *
814  *
815  *
816  *
817  *
818  *
819  *
820  *
821  *
822  *
823  *
824  *
825  *
826  *
827  *
828  *
829  *
830  *
831  *
832  *
833  *
834  *
835  *
836  *
837  *
838  *
839  *
840  *
841  *
842  *
843  *
844  *
845  *
846  *
847  *
848  *
849  *
850  *
851  *
852  *
853  *
854  *
855  *
856  *
857  *
858  *
859  *
860  *
861  *
862  *
863  *
864  *
865  *
866  *
867  *
868  *
869  *
870  *
871  *
872  *
873  *
874  *
875  *
876  *
877  *
878  *
879  *
880  *
881  *
882  *
883  *
884  *
885  *
886  *
887  *
888  *
889  *
890  *
891  *
892  *
893  *
894  *
895  *
896  *
897  *
898  *
899  *
900  *
901  *
902  *
903  *
904  *
905  *
906  *
907  *
908  *
909  *
910  *
911  *
912  *
913  *
914  *
915  *
916  *
917  *
918  *
919  *
920  *
921  *
922  *
923  *
924  *
925  *
926  *
927  *
928  *
929  *
930  *
931  *
932  *
933  *
934  *
935  *
936  *
937  *
938  *
939  *
940  *
941  *
942  *
943  *
944  *
945  *
946  *
947  *
948  *
949  *
950  *
951  *
952  *
953  *
954  *
955  *
956  *
957  *
958  *
959  *
960  *
961  *
962  *
963  *
964  *
965  *
966  *
967  *
968  *
969  *
970  *
971  *
972  *
973  *
974  *
975  *
976  *
977  *
978  *
979  *
980  *
981  *
982  *
983  *
984  *
985  *
986  *
987  *
988  *
989  *
990  *
991  *
992  *
993  *
994  *
995  *
996  *
997  *
998  *
999  *
1000  *
1001  *
1002  *
1003  *
1004  *
1005  *
1006  *
1007  *
1008  *
1009  *
1010  *
1011  *
1012  *
1013  *
1014  *
1015  *
1016  *
1017  *
1018  *
1019  *
1020  *
1021  *
1022  *
1023  *
1024  *
1025  *
1026  *
1027  *
1028  *
1029  *
1030  *
1031  *
1032  *
1033  *
1034  *
1035  *
1036  *
1037  *
1038  *
1039  *
1040  *
1041  *
1042  *
1043  *
1044  *
1045  *
1046  *
1047  *
1048  *
1049  *
1050  *
1051  *
1052  *
1053  *
1054  *
1055  *
1056  *
1057  *
1058  *
1059  *
1060  *
1061  *
1062  *
1063  *
1064  *
1065  *
1066  *
1067  *
1068  *
1069  *
1070  *
1071  *
1072  *
1073  *
1074  *
1075  *
1076  *
1077  *
1078  *
1079  *
1080  *
1081  *
1082  *
1083  *
1084  *
1085  *
1086  *
1087  *
1088  *
1089  *
1090  *
1091  *
1092  *
1093  *
1094  *
1095  *
1096  *
1097  *
1098  *
1099  *
1100  *
1101  *
1102  *
1103  *
1104  *
1105  *
1106  *
1107  *
1108  *
1109  *
1110  *
1111  *
1112  *
1113  *
1114  *
1115  *
1116  *
1117  *
1118  *
1119  *
1120  *
1121  *
1122  *
1123  *
1124  *
1125  *
1126  *
1127  *
1128  *
1129  *
1130  *
1131  *
1132  *
1133  *
1134  *
1135  *
1136  *
1137  *
1138  *
1139  *
1140  *
1141  *
1142  *
1143  *
1144  *
1145  *
1146  *
1147  *
1148  *
1149  *
1150  *
1151  *
1152  *
1153  *
1154  *
1155  *
1156  *
1157  *
1158  *
1159  *
1160  *
1161  *
1162  *
1163  *
1164  *
1165  *
1166  *
1167  *
1168  *
1169  *
1170  *
1171  *
1172  *
1173  *
1174  *
1175  *
1176  *
1177  *
1178  *
1179  *
1180  *
1181  *
1182  *
1183  *
1184  *
1185  *
1186  *
1187  *
1188  *
1189  *
1190  *
1191  *
1192  *
1193  *
1194  *
1195  *
1196  *
1197  *
1198  *
1199  *
1200  *
1201  *
1202  *
1203  *
1204  *
1205  *
1206  *
1207  *
1208  *
1209  *
1210  *
1211  *
1212  *
1213  *
1214  *
1215  *
1216  *
1217  *
1218  *
1219  *
1220  *
1221  *
1222  *
1223  *
1224  *
1225  *
1226  *
1227  *
1228  *
1229  *
1230  *
1231  *
1232  *
1233  *
1234  *
1235  *
1236  *
1237  *
1238  *
1239  *
1240  *
1241  *
1242  *
1243  *
1244  *
1245  *
1246  *
1247  *
1248  *
1249  *
1250  *
1251  *
1252  *
1253  *
1254  *
1255  *
1256  *
1257  *
1258  *
1259  *
1260  *
1261  *
1262  *
1263  *
1264  *
1265  *
1266  *
1267  *
1268  *
1269  *
1270  *
1271  *
1272  *
1273  *
1274  *
1275  *
1276  *
1277  *
1278  *
1279  *
1280  *
1281  *
1282  *
1283  *
1284  *
1285  *
1286  *
1287  *
1288  *
1289  *
1290  *
1291  *
1292  *
1293  *
1294  *
1295  *
1296  *
1297  *
1298  *
1299  *
1300  *
1301  *
1302  *
1303  *
1304  *
1305  *
1306  *
1307  *
1308  *
1309  *
1310  *
1311  *
1312  *
1313  *
1314  *
1315  *
1316  *
1317  *
1318  *
1319  *
1320  *
1321  *
1322  *
1323  *
1324  *
1325  *
1326  *
1327  *
1328  *
1329  *
1330  *
1331  *
1332  *
1333  *
1334  *
1335  *
1336  *
1337  *
1338  *
1339  *
1340  *
1341  *
1342  *
1343  *
1344  *
1345  *
1346  *
1347  *
1348  *
1349  *
1350  *
1351  *
1352  *
1353  *
1354  *
1355  *
1356  *
1357  *
1358  *
1359  *
1360  *
1361  *
1362  *
1363  *
1364  *
1365  *
1366  *
1367  *
1368  *
1369  *
1370  *
1371  *
1372  *
1373  *
1374  *
1375  *
1376  *
1377  *
1378  *
1379  *
1380  *
1381  *
1382  *
1383  *
1384  *
1385  *
1386  *
1387  *
1388  *
1389  *
1390  *
1391  *
1392  *
1393  *
1394  *
1395  *
1396  *
1397  *
1398  *
1399  *
1400  *
1401  *
1402  *
1403  *
1404  *
1405  *
1406  *
1407  *
1408  *
1409  *
1410  *
1411  *
1412  *
1413  *
1414  *
1415  *
1416  *
1417  *
1418  *
1419  *
1420  *
1421  *
1422  *
1423  *
1424  *
1425  *
1426  *
1427  *
1428  *
1429  *
1430  *
1431  *
1432  *
1433  *
1434  *
1435  *
1436  *
1437  *
1438  *
1439  *
1440  *
1441  *
1442  *
1443  *
1444  *
1445  *
1446  *
1447  *
1448  *
1449  *
1450  *
1451  *
1452  *
1453  *
1454  *
1455  *
1456  *
1457  *
1458  *
1459  *
1460  *
1461  *
1462  *
1463  *
1464  *
1465  *
1466  *
1467  *
1468  *
1469  *
1470  *
1471  *
1472  *
1473  *
1474  *
1475  *
1476  *
1477  *
1478  *
1479  *
1480  *
1481  *
1482  *
1483  *
1484  *
1485  *
1486  *
1487  *
1488  *
1489  *
1490  *
1491  *
1492  *
1493  *
1494  *
1495  *
1496  *
1497  *
1498  *
1499  *
1500  *
1501  *
1502  *
1503  *
1504  *
1505  *
1506  *
1507  *
1508  *
1509  *
1510  *
1511  *
1512  *
1513  *
1514  *
1515  *
1516  *
1517  *
1518  *
1519  *
1520  *
1521  *
1522  *
1523  *
1524  *
1525  *
1526  *
1527  *
1528  *
1529  *
1530  *
1531  *
1532  *
1533  *
1534  *
1535  *
1536  *
1537  *
1538  *
1539  *
1540  *
1541  *
1542  *
1543  *
1544  *
1545  *
1546  *
1547  *
1548  *
1549  *
1550  *
1551  *
1552  *
1553  *
1554  *
1555  *
1556  *
1557  *
1558  *
1559  *
1560  *
1561  *
1562  *
1563  *
1564  *
1565  *
1566  *
1567  *
1568  *
1569  *
1570  *
1571  *
1572  *
1573  *
1574  *
1575  *
1576  *
1577  *
1578  *
1579  *
1580  *
1581  *
1582  *
1583  *
1584  *
1585  *
1586  *
1587  *
1588  *
1589  *
1590  *
1591  *
1592  *
1593  *
1594  *
1595  *
1596  *
1597  *
1598  *
1599  *
1600  *
1601  *
1602  *
1603  *
1604  *
1605  *
1606  *
1607  *
1608  *
1609  *
1610  *
1611  *
1612  *
1613  *
1614  *
1615  *
1616  *
1617  *
1618  *
1619  *
1620  *
1621  *
1622  *
1623  *
1624  *
1625  *
1626  *
1627  *
1628  *
1629  *
1630  *
1631  *
1632  *
1633  *
1634  *
1635  *
1636  *
1637  *
1638  *
1639  *
1640  *
1641  *
1642  *
1643  *
1644  *
1645  *
1646  *
1647  *
1648  *
1649  *
1650  *
1651  *
1652  *
1653  *
1654  *
1655  *
1656  *
1657  *
1658  *
1659  *
1660  *
1661  *
1662  *
1663  *
1664  *
1665  *
1666  *
1667  *
1668  *
1669  *
1670  *
1671  *
1672  *
1673  *
1674  *
1675  *
1676  *
1677  *
1678  *
1679  *
1680  *
1681  *
1682  *
1683  *
1684  *
1685  *
1686  *
1687  *
1688  *
1689  *
1690  *
1691  *
1692  *
1693  *
1694  *
1695  *
1696  *
1697  *
1698  *
1699  *
1700  *
1701  *
1702  *
1703  *
1704  *
1705  *
1706  *
1707  *
1708  *
1709  *
1710  *
1711  *
1712  *
1713  *
1714  *
1715  *
1716  *
1717  *
1718  *
1719  *
1720  *
1721  *
1722  *
1723  *
1724  *
1725  *
1726  *
1727  *
1728  *
1729  *
1730  *
1731  *
1732 
```

```

"FAILED");
56         return(0);
57     }
58 }

```

28. B-Con_crypto-algorithms/sha256.c (158 lines)

```

1 /
*****
2 * Filename:   sha256.c
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antees.
6 * Details:    Implementation of the SHA-256 hashing algorithm.
SHA256 is one of the three algorithms in the
7
8             specification. The others, SHA-384 and SHA-512,
are not
9             offered in this implementation.
10            Algorithm specification can be found here:
11            * http://csrc.nist.gov/publications/fips/
fips180-2/fips180-2withchangenotice.pdf
12            This implementation uses little endian byte or-
der.
13
*****
14
15 /***** HEADER FILES
*****
16 #include <stdlib.h>
17 #include <memory.h>
18 #include "sha256.h"
19
20 /***** MACROS
*****
21 #define ROTLEFT(a,b) (((a) << (b)) | ((a) >> (32-(b))))
22 #define ROTRIGHT(a,b) (((a) >> (b)) | ((a) << (32-(b))))
23
24 #define CH(x,y,z) (((x) & (y)) ^ (~(x) & (z)))
25 #define MAJ(x,y,z) (((x) & (y)) ^ ((x) & (z)) ^ ((y) & (z)))
26 #define EP0(x) (ROTRIGHT(x,2) ^ ROTRIGHT(x,13) ^ ROTRIGHT(x,
22))
27 #define EP1(x) (ROTRIGHT(x,6) ^ ROTRIGHT(x,11) ^ ROTRIGHT(x,
25))
28 #define SIG0(x) (ROTRIGHT(x,7) ^ ROTRIGHT(x,18) ^ ((x) >> 3))
29 #define SIG1(x) (ROTRIGHT(x,17) ^ ROTRIGHT(x,19) ^ ((x) >>
10))
30
31 /***** VARIABLES
*****
32 static const WORD k[64] = {
33     0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b,
0x59f111f1, 0x923f82a4, 0xab1c5ed5,
34     0xd807aa98, 0x12835b01, 0x243185be, 0x550c7d-
c3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
35     0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f,
0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
36     0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
37     0x27b70a85, 0x2e1b2138, 0x4d2c6dfc,
0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
38     0xa2bfe8a1, 0xa81a664b, 0x-
c24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
39     0x19a4c116, 0x1e377c08, 0x2748774c, 0x34b0b0-
cb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
40     0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x900beffa,
0xa4506ceb, 0xbef9a3f7, 0xc67178f2
41 };
42
43 /***** FUNCTION DEFINITIONS
*****
44 void sha256_transform(SHA256_CTX *ctx, const BYTE data[])
45 {
46     WORD a, b, c, d, e, f, g, h, i, j, t1, t2, m[64];
47
48     for (i = 0, j = 0; i < 16; ++i, j += 4)
49         m[i] = (data[j] << 24) | (data[j + 1] << 16) |
(data[j + 2] << 8) | (data[j + 3]);
50     for (; i < 64; ++i)
51         m[i] = SIG1(m[i - 2]) + m[i - 7] + SIG0(m[i -
15]) + m[i - 16];
52
53     a = ctx->state[0];
54     b = ctx->state[1];
55     c = ctx->state[2];
56     d = ctx->state[3];
57     e = ctx->state[4];
58     f = ctx->state[5];
59     g = ctx->state[6];
60     h = ctx->state[7];
61
62     for (i = 0; i < 64; ++i) {
63         t1 = h + EP1(e) + CH(e,f,g) + k[i] + m[i];
64         t2 = EP0(a) + MAJ(a,b,c);
65         g = f;
66         f = g;
67         g = e;

```

```

68         e = d + t1;
69         d = c;
70         c = b;
71         b = a;
72         a = t1 + t2;
73     }
74
75     ctx->state[0] += a;
76     ctx->state[1] += b;
77     ctx->state[2] += c;
78     ctx->state[3] += d;
79     ctx->state[4] += e;
80     ctx->state[5] += f;
81     ctx->state[6] += g;
82     ctx->state[7] += h;
83 }
84
85 void sha256_init(SHA256_CTX *ctx)
86 {
87     ctx->datalen = 0;
88     ctx->bitlen = 0;
89     ctx->state[0] = 0x6a09e667;
90     ctx->state[1] = 0xbb67ae85;
91     ctx->state[2] = 0x3c6ef372;
92     ctx->state[3] = 0xa54ff53a;
93     ctx->state[4] = 0x510e527f;
94     ctx->state[5] = 0x9b05688c;
95     ctx->state[6] = 0x1f83d9ab;
96     ctx->state[7] = 0x5be0cd19;
97 }
98
99 void sha256_update(SHA256_CTX *ctx, const BYTE data[], size_t
len)
100 {
101     WORD i;
102
103     for (i = 0; i < len; ++i) {
104         ctx->data[ctx->datalen] = data[i];
105         ctx->datalen++;
106         if (ctx->datalen == 64) {
107             sha256_transform(ctx, ctx->data);
108             ctx->bitlen += 512;
109             ctx->datalen = 0;
110         }
111     }
112 }
113
114 void sha256_final(SHA256_CTX *ctx, BYTE hash[])
115 {
116     WORD i;
117
118     i = ctx->datalen;
119
120     // Pad whatever data is left in the buffer.
121     if (ctx->datalen < 56) {
122         ctx->data[i++] = 0x80;
123         while (i < 56)
124             ctx->data[i++] = 0x00;
125     }
126     else {
127         ctx->data[i++] = 0x80;
128         while (i < 64)
129             ctx->data[i++] = 0x00;
130         sha256_transform(ctx, ctx->data);
131         memset(ctx->data, 0, 56);
132     }
133
134     // Append to the padding the total message's length in
bits and transform.
135     ctx->bitlen += ctx->datalen * 8;
136     ctx->data[63] = ctx->bitlen;
137     ctx->data[62] = ctx->bitlen >> 8;
138     ctx->data[61] = ctx->bitlen >> 16;
139     ctx->data[60] = ctx->bitlen >> 24;
140     ctx->data[59] = ctx->bitlen >> 32;
141     ctx->data[58] = ctx->bitlen >> 40;
142     ctx->data[57] = ctx->bitlen >> 48;
143     ctx->data[56] = ctx->bitlen >> 56;
144     sha256_transform(ctx, ctx->data);
145
146     // Since this implementation uses little endian byte
ordering and SHA uses big endian,
147     // reverse all the bytes when copying the final state
to the output hash.
148     for (i = 0; i < 4; ++i) {
149         hash[i] = (ctx->state[0] >> (24 - i * 8))
& 0x000000ff;
150         hash[i + 4] = (ctx->state[1] >> (24 - i * 8))
& 0x000000ff;
151         hash[i + 8] = (ctx->state[2] >> (24 - i * 8))
& 0x000000ff;
152         hash[i + 12] = (ctx->state[3] >> (24 - i * 8))
& 0x000000ff;
153         hash[i + 16] = (ctx->state[4] >> (24 - i * 8))
& 0x000000ff;
154         hash[i + 20] = (ctx->state[5] >> (24 - i * 8))
& 0x000000ff;
155         hash[i + 24] = (ctx->state[6] >> (24 - i * 8))
& 0x000000ff;
156         hash[i + 28] = (ctx->state[7] >> (24 - i * 8))
& 0x000000ff;

```

```
157         }
158     }
```

29. B-Con_crypto-algorithms/sha256.h (34 lines)

```

1 /
*****
2 * Filename:   sha256.h
3 * Author:    Brad Conte (brad AT bradconte.com)
4 * Copyright:
5 * Disclaimer: This code is presented "as is" without any guar-
antes.
6 * Details:   Defines the API for the corresponding SHA1 im-
plementation.
7
*****
*/
8
9 #ifndef SHA256_H
10 #define SHA256_H
11
12 /****** HEADER FILES
******/
13 #include <stddef.h>
14
15 /****** MACROS
******/
16 #define SHA256_BLOCK_SIZE 32          // SHA256 outputs a 32
byte digest
17
18 /****** DATA TYPES
******/
19 typedef unsigned char BYTE;          // 8-bit byte
20 typedef unsigned int WORD;           // 32-bit word, change to
"long" for 16-bit machines
21
22 typedef struct {
23     BYTE data[64];
24     WORD datalen;
25     unsigned long long bitlen;
26     WORD state[8];
27 } SHA256_CTX;
28
29 /****** FUNCTION DECLARATIONS
******/
30 void sha256_init(SHA256_CTX *ctx);
31 void sha256_update(SHA256_CTX *ctx, const BYTE data[], size_t
len);
32 void sha256_final(SHA256_CTX *ctx, BYTE hash[]);
33
34 #endif // SHA256_H

```

AES 코어 (Verilog)

What. Joachim Strömbergson 의 AES Verilog 구현. 암호화/복호화 코어, S-box, 키 메모리로 구성된다.

Why it exists. HLS C++ 도 아니고 양산 보안 RTL 도 아닌, ‘평범한 RTL’의 기준선 이다. OpenTitan 의 17,214줄과 비교하면 부채널·고장주입 대책이 차지하는 비중이 드러난다. 기능만 구현하면 AES 가 얼마나 작은지 보여 준다.

학부 복습 · EECS concepts. 레지스터 전송 수준(RTL) 추상화 · 유한상태기계 설계 · 데이터패스와 제어의 분리.

석사 수준 · **Practice note.** IP 를 평가할 때 “몇 줄인가”가 아니라 “어떤 위협 모델을 가정했는가”를 물어야 한다. 같은 AES 라도 위협 모델이 다르면 다른 제품이다.

7 files • 2,714 lines

API — aes.v

Table 148. secworks_aes/src/rtl/aes.v — module / interface / package (1개)

이름	종류
aes	module

Table 149. secworks_aes/src/rtl/aes.v — localparam (21개)

상수	형/폭	값
ADDR_NAME0		8'h00
ADDR_NAME1		8'h01
ADDR_VERSION		8'h02
ADDR_CTRL		8'h08
CTRL_INIT_BIT		0
CTRL_NEXT_BIT		1
ADDR_STATUS		8'h09
STATUS_READY_BIT		0
STATUS_VALID_BIT		1
ADDR_CONFIG		8'h0a
CTRL_ENCDEC_BIT		0
CTRL_KEYLEN_BIT		1
ADDR_KEY0		8'h10
ADDR_KEY7		8'h17
ADDR_BLOCK0		8'h20
ADDR_BLOCK3		8'h23
ADDR_RESULT0		8'h30
ADDR_RESULT3		8'h33

상수	형/폭	값
CORE_NAME0		32'h61657320
CORE_NAME1		32'h20202020
CORE_VERSION		32'h302e3630

Table 150. secworks_aes/src/rtl/aes.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cs	input	wire
we	input	wire
address	input	wire [7 : 0]
write_data	input	wire [31 : 0]
read_data	output	wire [31 : 0]

API — aes_core.v**Table 151. secworks_aes/src/rtl/aes_core.v — module / interface / package (1개)**

이름	종류
aes_core	module

Table 152. secworks_aes/src/rtl/aes_core.v — localparam (3개)

상수	형/폭	값
CTRL_IDLE		2'h0
CTRL_INIT		2'h1
CTRL_NEXT		2'h2

Table 153. secworks_aes/src/rtl/aes_core.v — 포트 (11개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
encdec	input	wire
init	input	wire
next	input	wire
ready	output	wire
key	input	wire [255 : 0]
keylen	input	wire
block	input	wire [127 : 0]
result	output	wire [127 : 0]

포트	방향	형/폭
result_valid	output	wire

API — aes_decipher_block.v**Table 154. secworks_aes/src/rtl/aes_decipher_block.v — module / interface / package (1개)**

이름	종류
aes_decipher_block	module

Table 155. secworks_aes/src/rtl/aes_decipher_block.v — localparam (13개)

상수	형/폭	값
AES_128_BIT_KEY		1'h0
AES_256_BIT_KEY		1'h1
AES128_ROUNDS		4'ha
AES256_ROUNDS		4'he
NO_UPDATE		3'h0
INIT_UPDATE		3'h1
SBOX_UPDATE		3'h2
MAIN_UPDATE		3'h3
FINAL_UPDATE		3'h4
CTRL_IDLE		2'h0
CTRL_INIT		2'h1
CTRL_SBOX		2'h2
CTRL_MAIN		2'h3

Table 156. secworks_aes/src/rtl/aes_decipher_block.v — 포트 (9개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
next	input	wire
keylen	input	wire
round	output	wire [3 : 0]
round_key	input	wire [127 : 0]
block	input	wire [127 : 0]
new_block	output	wire [127 : 0]
ready	output	wire

API — aes_encipher_block.v**Table 157. secworks_aes/src/rtl/aes_encipher_block.v — module / interface / package (1개)**

이름	종류
aes_encipher_block	module

Table 158. secworks_aes/src/rtl/aes_encipher_block.v — localparam (13개)

상수	형/폭	값
AES_128_BIT_KEY		1'h0
AES_256_BIT_KEY		1'h1
AES128_ROUNDS		4'ha
AES256_ROUNDS		4'he
NO_UPDATE		3'h0
INIT_UPDATE		3'h1
SBOX_UPDATE		3'h2
MAIN_UPDATE		3'h3
FINAL_UPDATE		3'h4
CTRL_IDLE		2'h0
CTRL_INIT		2'h1
CTRL_SBOX		2'h2
CTRL_MAIN		2'h3

Table 159. secworks_aes/src/rtl/aes_encipher_block.v — 포트 (11개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
next	input	wire
keylen	input	wire
round	output	wire [3 : 0]
round_key	input	wire [127 : 0]
sboxw	output	wire [31 : 0]
new_sboxw	input	wire [31 : 0]
block	input	wire [127 : 0]
new_block	output	wire [127 : 0]
ready	output	wire

API — aes_inv_sbox.v**Table 160. secworks_aes/src/rtl/aes_inv_sbox.v — module / interface / package (1개)**

이름	종류
aes_inv_sbox	module

Table 161. secworks_aes/src/rtl/aes_inv_sbox.v — 포트 (2개)

포트	방향	형/폭
sboxw	input	wire [31 : 0]
new_sboxw	output	wire [31 : 0]

API — aes_key_mem.v**Table 162. secworks_aes/src/rtl/aes_key_mem.v — module / interface / package (1개)**

이름	종류
aes_key_mem	module

Table 163. secworks_aes/src/rtl/aes_key_mem.v — localparam (8개)

상수	형/폭	값
AES_128_BIT_KEY		1'h0
AES_256_BIT_KEY		1'h1
AES_128_NUM_ROUNDS		10
AES_256_NUM_ROUNDS		14
CTRL_IDLE		3'h0
CTRL_INIT		3'h1
CTRL_GENERATE		3'h2
CTRL_DONE		3'h3

Table 164. secworks_aes/src/rtl/aes_key_mem.v — 포트 (10개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
key	input	wire [255 : 0]
keylen	input	wire
init	input	wire
round	input	wire [3 : 0]
round_key	output	wire [127 : 0]
ready	output	wire
sboxw	output	wire [31 : 0]
new_sboxw	input	wire [31 : 0]

API — aes_sbox.v**Table 165. secworks_aes/src/rtl/aes_sbox.v — module / interface / package (1개)**

이름	종류
aes_sbox	module

Table 166. secworks_aes/src/rtl/aes_sbox.v — 포트 (2개)

포트	방향	형/폭
sboxw	input	wire [31 : 0]
new_sboxw	output	wire [31 : 0]

1. secworks_aes/src/rtl/aes.v (273 lines)

```

1 //
=====
2 //
3 // aes.v
4 // -----
5 // Top level wrapper for the AES block cipher core.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013, 2014 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
14 // lowing conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
17 // copyright notice, this list of conditions and the following dis-
18 // claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright notice, this list of conditions and the following dis-
22 // claimer in the documentation and/or other materials provided with
23 // the distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
26 // TRIBUTORS
27 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
28 // BUT NOT
29 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
30 // FITNESS
31 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
32 // THE
33 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
34 // INDIRECT,
35 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
36 // (INCLUDING,
37 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
38 // VICES;
39 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
40 // HOWEVER
41 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
42 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
43 // WISE)
44 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
45 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
46 //
47 //
=====
48 //
49 `default_nettype none
50
51 module aes(
52     // Clock and reset.
53     input wire      clk,
54     input wire      reset_n,
55
56     // Control.
57     input wire      cs,
58     input wire      we,
59
60     // Data ports.
61     input wire [7 : 0] address,
62     input wire [31 : 0] write_data,
63     output wire [31 : 0] read_data
64 );
65
66 //-----
67 // Internal constant and parameter definitions.
68 //-----
69
70 localparam ADDR_NAME0      = 8'h00;
71 localparam ADDR_NAME1      = 8'h01;
72 localparam ADDR_VERSION    = 8'h02;
73
74 localparam ADDR_CTRL       = 8'h08;
75 localparam CTRL_INIT_BIT   = 0;

```

```

65 localparam CTRL_NEXT_BIT   = 1;
66
67 localparam ADDR_STATUS     = 8'h09;
68 localparam STATUS_READY_BIT = 0;
69 localparam STATUS_VALID_BIT = 1;
70
71 localparam ADDR_CONFIG     = 8'h0a;
72 localparam CTRL_ENCDEC_BIT = 0;
73 localparam CTRL_KEYLEN_BIT = 1;
74
75 localparam ADDR_KEY0       = 8'h10;
76 localparam ADDR_KEY7       = 8'h17;
77
78 localparam ADDR_BLOCK0     = 8'h20;
79 localparam ADDR_BLOCK3     = 8'h23;
80
81 localparam ADDR_RESULT0    = 8'h30;
82 localparam ADDR_RESULT3    = 8'h33;
83
84 localparam CORE_NAME0      = 32'h61657320; // "aes "
85 localparam CORE_NAME1      = 32'h20202020; // " "
86 localparam CORE_VERSION    = 32'h302e3630; // "0.60"
87
88 //-----
89 // Registers including update variables and write enable.
90 //-----
91
92 reg init_reg;
93 reg init_new;
94
95 reg next_reg;
96 reg next_new;
97
98 reg encdec_reg;
99 reg keylen_reg;
100 reg config_we;
101
102 reg [31 : 0] block_reg [0 : 3];
103 reg          block_we;
104
105 reg [31 : 0] key_reg [0 : 7];
106 reg          key_we;
107
108 reg [127 : 0] result_reg;
109 reg          valid_reg;
110 reg          ready_reg;
111
112 //-----
113 // Wires.
114 //-----
115
116 reg [31 : 0] tmp_read_data;
117
118 wire          core_encdec;
119 wire          core_init;
120 wire          core_next;
121 wire          core_ready;
122 wire [255 : 0] core_key;
123 wire          core_keylen;
124 wire [127 : 0] core_block;
125 wire [127 : 0] core_result;
126 wire          core_valid;
127
128 //-----
129 // Concurrent connectivity for ports etc.
130 //-----
131
132 assign read_data = tmp_read_data;
133
134 assign core_key = {key_reg[0], key_reg[1], key_reg[2],
135 key_reg[3],
136 key_reg[4], key_reg[5], key_reg[6],
137 key_reg[7]};
138
139 assign core_block = {block_reg[0], block_reg[1],
140 block_reg[2], block_reg[3]};
141
142 assign core_init = init_reg;
143 assign core_next = next_reg;
144 assign core_encdec = encdec_reg;
145 assign core_keylen = keylen_reg;
146
147 //-----

```

```

146 // core instantiation.
147 //-----
148 aes_core core(
149     .clk(clk),
150     .reset_n(reset_n),
151
152     .encdec(core_encdec),
153     .init(core_init),
154     .next(core_next),
155     .ready(core_ready),
156
157     .key(core_key),
158     .keylen(core_keylen),
159
160     .block(core_block),
161     .result(core_result),
162     .result_valid(core_valid)
163 );
164
165 //-----
166
167 // reg_update
168 // Update functionality for all registers in the core.
169 // All registers are positive edge triggered with asynchron-
170 // ous
171 // active low reset.
172 //-----
173 always @(posedge clk or negedge reset_n)
174     begin : reg_update
175         integer i;
176
177         if (!reset_n)
178             begin
179                 for (i = 0 ; i < 4 ; i = i + 1)
180                     block_reg[i] <= 32'h0;
181
182                 for (i = 0 ; i < 8 ; i = i + 1)
183                     key_reg[i] <= 32'h0;
184
185                 init_reg <= 1'b0;
186                 next_reg <= 1'b0;
187                 encdec_reg <= 1'b0;
188                 keylen_reg <= 1'b0;
189
190                 result_reg <= 128'h0;
191                 valid_reg <= 1'b0;
192                 ready_reg <= 1'b0;
193             end
194         else
195             begin
196                 ready_reg <= core_ready;
197                 valid_reg <= core_valid;
198                 result_reg <= core_result;
199                 init_reg <= init_new;
200                 next_reg <= next_new;
201
202                 if (config_we)
203                     begin
204                         encdec_reg <= write_data[CTRL_ENCDEC_BIT];
205                         keylen_reg <= write_data[CTRL_KEYLEN_BIT];
206                     end
207
208                 if (key_we)
209                     key_reg[address[2 : 0]] <= write_data;
210
211                 if (block_we)
212                     block_reg[address[1 : 0]] <= write_data;
213             end
214         end // reg_update
215
216 //-----
217 // api
218 //
219 // The interface command decoding logic.
220 //-----
221
222 always @*
223     begin : api
224         init_new = 1'b0;
225         next_new = 1'b0;
226         config_we = 1'b0;
227         key_we = 1'b0;
228         block_we = 1'b0;
229         tmp_read_data = 32'h0;
230
231         if (cs)
232             begin
233                 if (we)
234                     begin
235                         if (address == ADDR_CTRL)
236                             begin
237                                 init_new = write_data[CTRL_INIT_BIT];
238                                 next_new = write_data[CTRL_NEXT_BIT];
239                             end

```

```

240         if (address == ADDR_CONFIG)
241             config_we = 1'b1;
242
243         if ((address >= ADDR_KEY0) && (address <=
244             ADDR_KEY7))
245             key_we = 1'b1;
246
247         if ((address >= ADDR_BLOCK0) && (address <=
248             ADDR_BLOCK3))
249             block_we = 1'b1;
250         end // if (we)
251     else
252         begin
253             case (address)
254                 ADDR_NAME0: tmp_read_data = CORE_NAME0;
255                 ADDR_NAME1: tmp_read_data = CORE_NAME1;
256                 ADDR_VERSION: tmp_read_data = CORE_VERSION;
257                 ADDR_CTRL: tmp_read_data = {28'h0,
258                     keylen_reg, encdec_reg, next_reg, init_reg};
259                 ADDR_STATUS: tmp_read_data = {30'h0, val-
260                     id_reg, ready_reg};
261             default:
262                 begin
263                     end
264             endcase // case (address)
265
266         if ((address >= ADDR_RESULT0) && (address <=
267             ADDR_RESULT3))
268             tmp_read_data = result_reg[(3 - (address -
269             ADDR_RESULT0)) * 32 +: 32];
270         end
271     end // addr_decoder
272 endmodule // aes
273 //
274 //=====
275 // 272 // EOF aes.v
276 // 273 //
277 //=====
278
279 2. secworks_aes/src/rtl/aes_core.v (342 lines)
280
281 1 //
282 //=====
283 //
284 2 //
285 3 // aes_core.v
286 4 // -----
287 5 // The AES core. This core supports key size of 128, and 256
288 6 // bits.
289 7 // Most of the functionality is within the submodules.
290 8 //
291 9 // Author: Joachim Strombergson
292 10 // Copyright (c) 2013, 2014, Secworks Sweden AB
293 11 // All rights reserved.
294 12 //
295 13 // Redistribution and use in source and binary forms, with or
296 14 // without modification, are permitted provided that the fol-
297 15 // lowing
298 16 // conditions are met:
299 17 // 1. Redistributions of source code must retain the above
300 18 // copyright
301 19 // notice, this list of conditions and the following dis-
302 20 // claimer.
303 21 // 2. Redistributions in binary form must reproduce the above
304 22 // copyright
305 23 // notice, this list of conditions and the following dis-
306 24 // claimer in
307 25 // the documentation and/or other materials provided with
308 26 // the
309 27 // distribution.
310 28 //
311 29 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
312 30 // TRIBUTORS
313 31 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
314 32 // BUT NOT
315 33 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
316 34 // FITNESS
317 35 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
318 36 // THE
319 37 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
320 38 // INDIRECT,
321 39 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
322 40 // (INCLUDING,
323 41 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
324 42 // VICES;
325 43 // HOWEVER
326 44 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
327 45 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
328 46 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
329 47 // WISE)
330 48 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF

```

```

36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module aes_core(
43     input wire      clk,
44     input wire      reset_n,
45
46     input wire      encdec,
47     input wire      init,
48     input wire      next,
49     output wire     ready,
50
51     input wire [255 : 0] key,
52     input wire      keylen,
53
54     input wire [127 : 0] block,
55     output wire [127 : 0] result,
56     output wire      result_valid
57 );
58
59
60
61
62 //-----
63 // Internal constant and parameter definitions.
64 //-----
65 localparam CTRL_IDLE = 2'h0;
66 localparam CTRL_INIT = 2'h1;
67 localparam CTRL_NEXT = 2'h2;
68
69
70 //-----
71 // Registers including update variables and write enable.
72 //-----
73 reg [1 : 0] aes_core_ctrl_reg;
74 reg [1 : 0] aes_core_ctrl_new;
75 reg        aes_core_ctrl_we;
76
77 reg        result_valid_reg;
78 reg        result_valid_new;
79 reg        result_valid_we;
80
81 reg        ready_reg;
82 reg        ready_new;
83 reg        ready_we;
84
85
86 //-----
87 // Wires.
88 //-----
89 reg        init_state;
90
91 wire [127 : 0] round_key;
92 wire          key_ready;
93
94 reg        enc_next;
95 wire [3 : 0] enc_round_nr;
96 wire [127 : 0] enc_new_block;
97 wire          enc_ready;
98 wire [31 : 0] enc_sboxw;
99
100 reg        dec_next;
101 wire [3 : 0] dec_round_nr;
102 wire [127 : 0] dec_new_block;
103 wire          dec_ready;
104
105 reg [127 : 0] muxed_new_block;
106 reg [3 : 0] muxed_round_nr;
107 reg        muxed_ready;
108
109 wire [31 : 0] keymem_sboxw;
110
111 /* verilator lint_off UNOPTFLAT */
112 reg [31 : 0] muxed_sboxw;
113 wire [31 : 0] new_sboxw;
114 /* verilator lint_on UNOPTFLAT */
115
116
117 //-----
118 // Instantiations.
119 //-----
120 aes_encipher_block enc_block(
121     .clk(clk),
122     .reset_n(reset_n),
123
124     .next(enc_next),
125
126     .keylen(keylen),
127
128     .round(enc_round_nr),
129     .round_key(round_key),
130
131     .sboxw(enc_sboxw),
132     .new_sboxw(new_sboxw),
133
134     .block(block),
135     .new_block(enc_new_block),
136     .ready(enc_ready)
137 );
138
139 aes_decipher_block dec_block(
140     .clk(clk),
141     .reset_n(reset_n),
142
143     .next(dec_next),
144
145     .keylen(keylen),
146     .round(dec_round_nr),
147     .round_key(round_key),
148
149     .block(block),
150     .new_block(dec_new_block),
151     .ready(dec_ready)
152 );
153
154
155 aes_key_mem keymem(
156     .clk(clk),
157     .reset_n(reset_n),
158
159     .key(key),
160     .keylen(keylen),
161     .init(init),
162
163     .round(muxed_round_nr),
164     .round_key(round_key),
165     .ready(key_ready),
166
167     .sboxw(keymem_sboxw),
168     .new_sboxw(new_sboxw)
169 );
170
171
172 aes_sbox
173 sbox_inst(.sboxw(muxed_sboxw), .new_sboxw(new_sboxw));
174
175 //-----
176 // Concurrent connectivity for ports etc.
177 //-----
178 assign ready      = ready_reg;
179 assign result     = muxed_new_block;
180 assign result_valid = result_valid_reg;
181
182
183 //-----
184 // reg_update
185 //
186 // Update functionality for all registers in the core.
187 // All registers are positive edge triggered with asynchron-
188 // ous
189 // active low reset. All registers have write enable.
190 //-----
191 always @ (posedge clk or negedge reset_n)
192 begin: reg_update
193     if (!reset_n)
194     begin
195         result_valid_reg <= 1'b0;
196         ready_reg <= 1'b1;
197         aes_core_ctrl_reg <= CTRL_IDLE;
198     end
199     else
200     begin
201         if (result_valid_we)
202             result_valid_reg <= result_valid_new;
203
204         if (ready_we)
205             ready_reg <= ready_new;
206
207         if (aes_core_ctrl_we)
208             aes_core_ctrl_reg <= aes_core_ctrl_new;
209     end
210 end // reg_update
211
212 //-----
213 // sbox_mux
214 //
215 // Controls which of the encipher datapath or the key memory
216 // that gets access to the sbox.
217 //-----
218 always @*
219 begin : sbox_mux

```

```

220     if (init_state)
221     begin
222         muxed_sboxw = keymem_sboxw;
223     end
224     else
225     begin
226         muxed_sboxw = enc_sboxw;
227     end
228     end // sbox_mux
229
230
231 //-----
232 // encdec_mux
233 //
234 // Controls which of the datapaths that get the next signal,
have
235 // access to the memory as well as the block processing res-
ult.
236 //-----
237 always @*
238 begin : encdec_mux
239     enc_next = 1'b0;
240     dec_next = 1'b0;
241
242     if (encdec)
243     begin
244         // Encipher operations
245         enc_next = next;
246         muxed_round_nr = enc_round_nr;
247         muxed_new_block = enc_new_block;
248         muxed_ready = enc_ready;
249     end
250     else
251     begin
252         // Decipher operations
253         dec_next = next;
254         muxed_round_nr = dec_round_nr;
255         muxed_new_block = dec_new_block;
256         muxed_ready = dec_ready;
257     end
258     end // encdec_mux
259
260
261 //-----
262 // aes_core_ctrl
263 //
264 // Control FSM for aes core. Basically tracks if we are in
265 // key init, encipher or decipher modes and connects the
266 // different submodules to shared resources and interface
ports.
267 //-----
268 always @*
269 begin : aes_core_ctrl
270     init_state = 1'b0;
271     ready_new = 1'b0;
272     ready_we = 1'b0;
273     result_valid_new = 1'b0;
274     result_valid_we = 1'b0;
275     aes_core_ctrl_new = CTRL_IDLE;
276     aes_core_ctrl_we = 1'b0;
277
278     case (aes_core_ctrl_reg)
279     CTRL_IDLE:
280     begin
281         if (init)
282         begin
283             init_state = 1'b1;
284             ready_new = 1'b0;
285             ready_we = 1'b1;
286             result_valid_new = 1'b0;
287             result_valid_we = 1'b1;
288             aes_core_ctrl_new = CTRL_INIT;
289             aes_core_ctrl_we = 1'b1;
290         end
291         else if (next)
292         begin
293             init_state = 1'b0;
294             ready_new = 1'b0;
295             ready_we = 1'b1;
296             result_valid_new = 1'b0;
297             result_valid_we = 1'b1;
298             aes_core_ctrl_new = CTRL_NEXT;
299             aes_core_ctrl_we = 1'b1;
300         end
301     end
302
303     CTRL_INIT:
304     begin
305         init_state = 1'b1;
306
307         if (key_ready)
308         begin
309             ready_new = 1'b1;
310             ready_we = 1'b1;
311             aes_core_ctrl_new = CTRL_IDLE;
312             aes_core_ctrl_we = 1'b1;

```

```

313     end
314     end
315
316     CTRL_NEXT:
317     begin
318         init_state = 1'b0;
319
320         if (muxed_ready)
321         begin
322             ready_new = 1'b1;
323             ready_we = 1'b1;
324             result_valid_new = 1'b1;
325             result_valid_we = 1'b1;
326             aes_core_ctrl_new = CTRL_IDLE;
327             aes_core_ctrl_we = 1'b1;
328         end
329     end
330
331     default:
332     begin
333
334     end
335     endcase // case (aes_core_ctrl_reg)
336
337     end // aes_core_ctrl
338 endmodule // aes_core
339
340 //

```

```

=====
341 // EOF aes_core.v
342 //
=====

```

3. secworks_aes/src/rtl/aes_decipher_block.v (526 lines)

```

1 //
=====
2 //
3 // aes_decipher_block.v
4 // -----
5 // The AES decipher round. A pure combinational module that
implements
6 // the initial round, main round and final round logic for
7 // decipher operations.
8 //
9 //
10 // Author: Joachim Strombergson
11 // Copyright (c) 2013, 2014, Secworks Sweden AB
12 // All rights reserved.
13 //
14 // Redistribution and use in source and binary forms, with or
15 // without modification, are permitted provided that the fol-
lowing
16 // conditions are met:
17 //
18 // 1. Redistributions of source code must retain the above
copyright
19 // notice, this list of conditions and the following dis-
claimer.
20 //
21 // 2. Redistributions in binary form must reproduce the above
copyright
22 // notice, this list of conditions and the following dis-
claimer in
23 // the documentation and/or other materials provided with
the
24 // distribution.
25 //
26 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
27 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
28 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
29 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
30 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
31 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
32 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
33 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
34 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
35 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
36 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
37 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
38 //
39 //
=====
40
41 `default_nettype none
42
43 module aes_decipher_block(

```

```

44         input wire      clk,
45         input wire      reset_n,
46
47         input wire      next,
48
49         input wire      keylen,
50         output wire [3 : 0] round,
51         input wire [127 : 0] round_key,
52
53         input wire [127 : 0] block,
54         output wire [127 : 0] new_block,
55         output wire      ready
56     );
57
58 //-----
59 // Internal constant and parameter definitions.
60 //-----
61
62 localparam AES_128_BIT_KEY = 1'h0;
63 localparam AES_256_BIT_KEY = 1'h1;
64
65 localparam AES128_ROUNDS = 4'ha;
66 localparam AES256_ROUNDS = 4'he;
67
68 localparam NO_UPDATE = 3'h0;
69 localparam INIT_UPDATE = 3'h1;
70 localparam SBOX_UPDATE = 3'h2;
71 localparam MAIN_UPDATE = 3'h3;
72 localparam FINAL_UPDATE = 3'h4;
73
74 localparam CTRL_IDLE = 2'h0;
75 localparam CTRL_INIT = 2'h1;
76 localparam CTRL_SBOX = 2'h2;
77 localparam CTRL_MAIN = 2'h3;
78
79 //-----
80 // Gaolis multiplication functions for Inverse MixColumn.
81 //-----
82
83 function automatic [7 : 0] gm2(input [7 : 0] op);
84     begin
85         gm2 = {op[6 : 0], 1'b0} ^ {8'h1b & {8{op[7]}}};
86     end
87 endfunction // gm2
88
89 function automatic [7 : 0] gm3(input [7 : 0] op);
90     begin
91         gm3 = gm2(op) ^ op;
92     end
93 endfunction // gm3
94
95 function automatic [7 : 0] gm4(input [7 : 0] op);
96     begin
97         gm4 = gm2(gm2(op));
98     end
99 endfunction // gm4
100
101 function automatic [7 : 0] gm8(input [7 : 0] op);
102     begin
103         gm8 = gm2(gm4(op));
104     end
105 endfunction // gm8
106
107 function automatic [7 : 0] gm09(input [7 : 0] op);
108     begin
109         gm09 = gm8(op) ^ op;
110     end
111 endfunction // gm09
112
113 function automatic [7 : 0] gm11(input [7 : 0] op);
114     begin
115         gm11 = gm8(op) ^ gm2(op) ^ op;
116     end
117 endfunction // gm11
118
119 function automatic [7 : 0] gm13(input [7 : 0] op);
120     begin
121         gm13 = gm8(op) ^ gm4(op) ^ op;
122     end
123 endfunction // gm13
124
125 function automatic [7 : 0] gm14(input [7 : 0] op);
126     begin
127         gm14 = gm8(op) ^ gm4(op) ^ gm2(op);
128     end
129 endfunction // gm14
130
131 function automatic [31 : 0] inv_mixw(input [31 : 0] w);
132     reg [7 : 0] b0, b1, b2, b3;
133     reg [7 : 0] mb0, mb1, mb2, mb3;
134     begin
135         b0 = w[31 : 24];
136         b1 = w[23 : 16];
137         b2 = w[15 : 08];
138         b3 = w[07 : 00];
139
140         mb0 = gm14(b0) ^ gm11(b1) ^ gm13(b2) ^ gm09(b3);

```

```

141         mb1 = gm09(b0) ^ gm14(b1) ^ gm11(b2) ^ gm13(b3);
142         mb2 = gm13(b0) ^ gm09(b1) ^ gm14(b2) ^ gm11(b3);
143         mb3 = gm11(b0) ^ gm13(b1) ^ gm09(b2) ^ gm14(b3);
144
145         inv_mixw = {mb0, mb1, mb2, mb3};
146     end
147 endfunction // mixw
148
149 function automatic [127 : 0] inv_mixcolumns(input [127 : 0]
data);
150     reg [31 : 0] w0, w1, w2, w3;
151     reg [31 : 0] ws0, ws1, ws2, ws3;
152     begin
153         w0 = data[127 : 096];
154         w1 = data[095 : 064];
155         w2 = data[063 : 032];
156         w3 = data[031 : 000];
157
158         ws0 = inv_mixw(w0);
159         ws1 = inv_mixw(w1);
160         ws2 = inv_mixw(w2);
161         ws3 = inv_mixw(w3);
162
163         inv_mixcolumns = {ws0, ws1, ws2, ws3};
164     end
165 endfunction // inv_mixcolumns
166
167 function automatic [127 : 0] inv_shiftrows(input [127 : 0]
data);
168     reg [31 : 0] w0, w1, w2, w3;
169     reg [31 : 0] ws0, ws1, ws2, ws3;
170     begin
171         w0 = data[127 : 096];
172         w1 = data[095 : 064];
173         w2 = data[063 : 032];
174         w3 = data[031 : 000];
175
176         ws0 = {w0[31 : 24], w3[23 : 16], w2[15 : 08], w1[07 :
00]};
177         ws1 = {w1[31 : 24], w0[23 : 16], w3[15 : 08], w2[07 :
00]};
178         ws2 = {w2[31 : 24], w1[23 : 16], w0[15 : 08], w3[07 :
00]};
179         ws3 = {w3[31 : 24], w2[23 : 16], w1[15 : 08], w0[07 :
00]};
180
181         inv_shiftrows = {ws0, ws1, ws2, ws3};
182     end
183 endfunction // inv_shiftrows
184
185 function automatic [127 : 0] addroundkey(input [127 : 0]
data, input [127 : 0] rkey);
186     begin
187         addroundkey = data ^ rkey;
188     end
189 endfunction // addroundkey
190
191 //-----
192 // Registers including update variables and write enable.
193 //-----
194
195 reg [1 : 0] sword_ctr_reg;
196 reg [1 : 0] sword_ctr_new;
197 reg sword_ctr_we;
198 reg sword_ctr_inc;
199 reg sword_ctr_rst;
200
201 reg [3 : 0] round_ctr_reg;
202 reg [3 : 0] round_ctr_new;
203 reg round_ctr_we;
204 reg round_ctr_set;
205 reg round_ctr_dec;
206
207 reg [127 : 0] block_new;
208 reg [31 : 0] block_w0_reg;
209 reg [31 : 0] block_w1_reg;
210 reg [31 : 0] block_w2_reg;
211 reg [31 : 0] block_w3_reg;
212 reg block_w0_we;
213 reg block_w1_we;
214 reg block_w2_we;
215 reg block_w3_we;
216
217 reg ready_reg;
218 reg ready_new;
219 reg ready_we;
220
221 reg [1 : 0] dec_ctrl_reg;
222 reg [1 : 0] dec_ctrl_new;
223 reg dec_ctrl_we;
224
225 //-----
226 // Wires.
227 //-----
228
229 reg [31 : 0] tmp_sboxw;
230 wire [31 : 0] new_sboxw;

```

```

231 reg [2 : 0] update_type;
232
233 //-----
234 // Instantiations.
235 //-----
236
237 aes_inv_sbox inv_s-
box_inst(.sboxw(tmp_sboxw), .new_sboxw(new_sboxw));
238
239 //-----
240 // Concurrent connectivity for ports etc.
241 //-----
242
243 assign round = round_ctr_reg;
244 assign new_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
245 assign ready = ready_reg;
246
247 //-----
248 // reg_update
249 //
250 // Update functionality for all registers in the core.
251 // All registers are positive edge triggered with synchron-
ous
252 // active low reset. All registers have write enable.
253 //-----
254
255 always @(posedge clk or negedge reset_n)
256 begin: reg_update
257 if (!reset_n)
258 begin
259 block_w0_reg <= 32'h0;
260 block_w1_reg <= 32'h0;
261 block_w2_reg <= 32'h0;
262 block_w3_reg <= 32'h0;
263 sword_ctr_reg <= 2'h0;
264 round_ctr_reg <= 4'h0;
265 ready_reg <= 1'b1;
266 dec_ctrl_reg <= CTRL_IDLE;
267 end
268 else
269 begin
270 if (block_w0_we)
271 block_w0_reg <= block_new[127 : 096];
272
273 if (block_w1_we)
274 block_w1_reg <= block_new[095 : 064];
275
276 if (block_w2_we)
277 block_w2_reg <= block_new[063 : 032];
278
279 if (block_w3_we)
280 block_w3_reg <= block_new[031 : 000];
281
282 if (sword_ctr_we)
283 sword_ctr_reg <= sword_ctr_new;
284
285 if (round_ctr_we)
286 round_ctr_reg <= round_ctr_new;
287
288 if (ready_we)
289 ready_reg <= ready_new;
290
291 if (dec_ctrl_we)
292 dec_ctrl_reg <= dec_ctrl_new;
293 end
294 end // reg_update
295
296 //-----
297 // round_logic
298 //
299 // The logic needed to implement init, main and final
rounds.
300 //-----
301
302 always @*
303 begin : round_logic
304 reg [127 : 0] old_block, inv_shiftrows_block, inv_mix-
columns_block;
305 reg [127 : 0] addkey_block;
306
307 inv_shiftrows_block = 128'h0;
308 inv_mixcolumns_block = 128'h0;
309 addkey_block = 128'h0;
310 block_new = 128'h0;
311 tmp_sboxw = 32'h0;
312 block_w0_we = 1'b0;
313 block_w1_we = 1'b0;
314 block_w2_we = 1'b0;
315 block_w3_we = 1'b0;
316
317 old_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};

```

```

318
319 // Update based on update type.
320 case (update_type)
321 // InitRound
322 INIT_UPDATE:
323 begin
324 old_block = block;
325 addkey_block = addroundkey(old_block,
round_key);
326 inv_shiftrows_block = inv_shiftrows(addkey_block);
327 block_new = inv_shiftrows_block;
328 block_w0_we = 1'b1;
329 block_w1_we = 1'b1;
330 block_w2_we = 1'b1;
331 block_w3_we = 1'b1;
332 end
333
334 SBUX_UPDATE:
335 begin
336 block_new = {new_sboxw, new_sboxw, new_sboxw,
new_sboxw};
337
338 case (sword_ctr_reg)
339 2'h0:
340 begin
341 tmp_sboxw = block_w0_reg;
342 block_w0_we = 1'b1;
343 end
344
345 2'h1:
346 begin
347 tmp_sboxw = block_w1_reg;
348 block_w1_we = 1'b1;
349 end
350
351 2'h2:
352 begin
353 tmp_sboxw = block_w2_reg;
354 block_w2_we = 1'b1;
355 end
356
357 2'h3:
358 begin
359 tmp_sboxw = block_w3_reg;
360 block_w3_we = 1'b1;
361 end
362 endcase // case (sbox_mux_ctrl_reg)
363 end
364
365 MAIN_UPDATE:
366 begin
367 addkey_block = addroundkey(old_block,
round_key);
368 inv_mixcolumns_block = inv_mix-
columns(addkey_block);
369 inv_shiftrows_block =
inv_shiftrows(inv_mixcolumns_block);
370 block_new = inv_shiftrows_block;
371 block_w0_we = 1'b1;
372 block_w1_we = 1'b1;
373 block_w2_we = 1'b1;
374 block_w3_we = 1'b1;
375 end
376
377 FINAL_UPDATE:
378 begin
379 block_new = addroundkey(old_block, round_key);
380 block_w0_we = 1'b1;
381 block_w1_we = 1'b1;
382 block_w2_we = 1'b1;
383 block_w3_we = 1'b1;
384 end
385
386 default:
387 begin
388 end
389 endcase // case (update_type)
390 end // round_logic
391
392 //-----
393
394 // sword_ctr
395 //
396 // The subbytes word counter with reset and increase logic.
397 //-----
398
399 always @*
400 begin : sword_ctr
401 sword_ctr_new = 2'h0;
402 sword_ctr_we = 1'b0;
403
404 if (sword_ctr_rst)
405 begin
406 sword_ctr_new = 2'h0;
407 sword_ctr_we = 1'b1;
408 end
409 else if (sword_ctr_inc)
410 begin
411 sword_ctr_new = sword_ctr_reg + 1'b1;
412 sword_ctr_we = 1'b1;

```



```

412     end
413     end // sword_ctr
414
415     //-----
416     // round_ctr
417     //
418     // The round counter with reset and increase logic.
419     //-----
420
421     always @*
422     begin : round_ctr
423         round_ctr_new = 4'h0;
424         round_ctr_we = 1'b0;
425
426         if (round_ctr_set)
427         begin
428             if (keylen == AES_256_BIT_KEY)
429             begin
430                 round_ctr_new = AES256_ROUNDS;
431             end
432             else
433             begin
434                 round_ctr_new = AES128_ROUNDS;
435             end
436             round_ctr_we = 1'b1;
437         end
438         else if (round_ctr_dec)
439         begin
440             round_ctr_new = round_ctr_reg - 1'b1;
441             round_ctr_we = 1'b1;
442         end
443     end // round_ctr
444
445     //-----
446
447     // decipher_ctrl
448     //
449     // The FSM that controls the decipher operations.
450     //-----
451
452     always @*
453     begin: decipher_ctrl
454         sword_ctr_inc = 1'b0;
455         sword_ctr_rst = 1'b0;
456         round_ctr_dec = 1'b0;
457         round_ctr_set = 1'b0;
458         ready_new = 1'b0;
459         ready_we = 1'b0;
460         update_type = NO_UPDATE;
461         dec_ctrl_new = CTRL_IDLE;
462         dec_ctrl_we = 1'b0;
463
464         case(dec_ctrl_reg)
465             CTRL_IDLE:
466             begin
467                 if (next)
468                 begin
469                     round_ctr_set = 1'b1;
470                     ready_new = 1'b0;
471                     ready_we = 1'b1;
472                     dec_ctrl_new = CTRL_INIT;
473                     dec_ctrl_we = 1'b1;
474                 end
475             end
476
477             CTRL_INIT:
478             begin
479                 sword_ctr_rst = 1'b1;
480                 update_type = INIT_UPDATE;
481                 dec_ctrl_new = CTRL_SBOX;
482                 dec_ctrl_we = 1'b1;
483             end
484
485             CTRL_SBOX:
486             begin
487                 sword_ctr_inc = 1'b1;
488                 update_type = SBOX_UPDATE;
489                 if (sword_ctr_reg == 2'h3)
490                 begin
491                     round_ctr_dec = 1'b1;
492                     dec_ctrl_new = CTRL_MAIN;
493                     dec_ctrl_we = 1'b1;
494                 end
495             end
496
497             CTRL_MAIN:
498             begin
499                 sword_ctr_rst = 1'b1;
500                 if (round_ctr_reg > 0)
501                 begin
502                     update_type = MAIN_UPDATE;
503                     dec_ctrl_new = CTRL_SBOX;
504                     dec_ctrl_we = 1'b1;
505                 end
506                 else
507                 begin
508                     update_type = FINAL_UPDATE;
509                     ready_new = 1'b1;

```

```

509         ready_we = 1'b1;
510         dec_ctrl_new = CTRL_IDLE;
511         dec_ctrl_we = 1'b1;
512     end
513 end
514
515 default:
516     begin
517         // Empty. Just here to make the synthesis tool
518         happy.
519     end
520     endcase // case (dec_ctrl_reg)
521 end // decipher_ctrl
522 endmodule // aes_decipher_block
523
524 //
=====
525 // EOF aes_decipher_block.v
526 //
=====

```

4. secworks_aes/src/rtl/aes_encipher_block.v (487 lines)

```

1 //
=====
2 //
3 // aes_encipher_block.v
4 // -----
5 // The AES encipher round. A pure combinational module that
6 // implements
7 // the initial round, main round and final round logic for
8 // encipher operations.
9 //
10 // Author: Joachim Strombergson
11 // Copyright (c) 2013, 2014, Secworks Sweden AB
12 // All rights reserved.
13 //
14 // Redistribution and use in source and binary forms, with or
15 // without modification, are permitted provided that the fol-
16 // lowing
17 // conditions are met:
18 // 1. Redistributions of source code must retain the above
19 // copyright
20 // notice, this list of conditions and the following dis-
21 // claimer.
22 // 2. Redistributions in binary form must reproduce the above
23 // copyright
24 // notice, this list of conditions and the following dis-
25 // claimer in
26 // the documentation and/or other materials provided with
27 // the
28 // distribution.
29 //
30 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
31 // TRIBUTORS
32 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
33 // BUT NOT
34 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
35 // FITNESS
36 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
37 // THE
38 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
39 // INDIRECT,
40 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
41 // (INCLUDING,
42 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
43 // VICES;
44 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
45 // HOWEVER
46 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
47 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
48 // WISE)
49 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
50 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
51 //
52 //
=====
40 `default_nettype none
41
42 module aes_encipher_block(
43     input wire      clk,
44     input wire      reset_n,
45
46     input wire      next,
47
48     input wire      keylen,
49     output wire [3 : 0] round,
50     input wire [127 : 0] round_key,
51
52     output wire [31 : 0] sboxw,
53     input wire [31 : 0] new_sboxw,

```



```

55         input wire [127 : 0] block,
56         output wire [127 : 0] new_block,
57         output wire      ready
58     );
59
60
61 //-----
62
63 // Internal constant and parameter definitions.
64 //-----
65 localparam AES_128_BIT_KEY = 1'h0;
66 localparam AES_256_BIT_KEY = 1'h1;
67
68 localparam AES128_ROUNDS = 4'ha;
69 localparam AES256_ROUNDS = 4'he;
70
71 localparam NO_UPDATE = 3'h0;
72 localparam INIT_UPDATE = 3'h1;
73 localparam SBOX_UPDATE = 3'h2;
74 localparam MAIN_UPDATE = 3'h3;
75 localparam FINAL_UPDATE = 3'h4;
76
77 localparam CTRL_IDLE = 2'h0;
78 localparam CTRL_INIT = 2'h1;
79 localparam CTRL_SBOX = 2'h2;
80 localparam CTRL_MAIN = 2'h3;
81
82
83 //-----
84
85 // Round functions with sub functions.
86 //-----
87 function automatic [7 : 0] gm2(input [7 : 0] op);
88     begin
89         gm2 = {op[6 : 0], 1'b0} ^ (8'h1b & {8{op[7]}});
90     end
91 endfunction // gm2
92
93 function automatic [7 : 0] gm3(input [7 : 0] op);
94     begin
95         gm3 = gm2(op) ^ op;
96     end
97 endfunction // gm3
98
99 function automatic [31 : 0] mixw(input [31 : 0] w);
100     reg [7 : 0] b0, b1, b2, b3;
101     reg [7 : 0] mb0, mb1, mb2, mb3;
102     begin
103         b0 = w[31 : 24];
104         b1 = w[23 : 16];
105         b2 = w[15 : 08];
106         b3 = w[07 : 00];
107
108         mb0 = gm2(b0) ^ gm3(b1) ^ b2 ^ b3;
109         mb1 = b0 ^ gm2(b1) ^ gm3(b2) ^ b3;
110         mb2 = b0 ^ b1 ^ gm2(b2) ^ gm3(b3);
111         mb3 = gm3(b0) ^ b1 ^ b2 ^ gm2(b3);
112
113         mixw = {mb0, mb1, mb2, mb3};
114     end
115 endfunction // mixw
116
117 function automatic [127 : 0] mixcolumns(input [127 : 0]
data);
118     reg [31 : 0] w0, w1, w2, w3;
119     reg [31 : 0] ws0, ws1, ws2, ws3;
120     begin
121         w0 = data[127 : 096];
122         w1 = data[095 : 064];
123         w2 = data[063 : 032];
124         w3 = data[031 : 000];
125
126         ws0 = mixw(w0);
127         ws1 = mixw(w1);
128         ws2 = mixw(w2);
129         ws3 = mixw(w3);
130
131         mixcolumns = {ws0, ws1, ws2, ws3};
132     end
133 endfunction // mixcolumns
134
135 function automatic [127 : 0] shiftrows(input [127 : 0]
data);
136     reg [31 : 0] w0, w1, w2, w3;
137     reg [31 : 0] ws0, ws1, ws2, ws3;
138     begin
139         w0 = data[127 : 096];
140         w1 = data[095 : 064];
141         w2 = data[063 : 032];
142         w3 = data[031 : 000];
143
144         ws0 = {w0[31 : 24], w1[23 : 16], w2[15 : 08], w3[07 :
00]};
145         ws1 = {w1[31 : 24], w2[23 : 16], w3[15 : 08], w0[07 :
00]};
146         ws2 = {w2[31 : 24], w3[23 : 16], w0[15 : 08], w1[07 :
00]};
147         ws3 = {w3[31 : 24], w0[23 : 16], w1[15 : 08], w2[07 :
00]};
148
149         shiftrows = {ws0, ws1, ws2, ws3};
150     end
151 endfunction // shiftrows
152
153 function automatic [127 : 0] addroundkey(input [127 : 0]
data, input [127 : 0] rkey);
154     begin
155         addroundkey = data ^ rkey;
156     end
157 endfunction // addroundkey
158
159 //-----
160 // Registers including update variables and write enable.
161 //-----
162 reg [1 : 0] sword_ctr_reg;
163 reg [1 : 0] sword_ctr_new;
164 reg sword_ctr_we;
165 reg sword_ctr_inc;
166 reg sword_ctr_rst;
167
168 reg [3 : 0] round_ctr_reg;
169 reg [3 : 0] round_ctr_new;
170 reg round_ctr_we;
171 reg round_ctr_rst;
172 reg round_ctr_inc;
173
174 reg [127 : 0] block_new;
175 reg [31 : 0] block_w0_reg;
176 reg [31 : 0] block_w1_reg;
177 reg [31 : 0] block_w2_reg;
178 reg [31 : 0] block_w3_reg;
179 reg block_w0_we;
180 reg block_w1_we;
181 reg block_w2_we;
182 reg block_w3_we;
183
184 reg ready_reg;
185 reg ready_new;
186 reg ready_we;
187
188 reg [1 : 0] enc_ctrl_reg;
189 reg [1 : 0] enc_ctrl_new;
190 reg enc_ctrl_we;
191
192
193 //-----
194 // Wires.
195 //-----
196 reg [2 : 0] update_type;
197 reg [31 : 0] muxed_sboxw;
198
199 //-----
200
201 // Concurrent connectivity for ports etc.
202 //-----
203 assign round = round_ctr_reg;
204 assign sboxw = muxed_sboxw;
205 assign new_block = {block_w0_reg, block_w1_reg,
block_w2_reg, block_w3_reg};
206 assign ready = ready_reg;
207
208 //-----
209
210 // reg_update
211 //
212 // Update functionality for all registers in the core.
213 // All registers are positive edge triggered with asynchron-
ous
214 // active low reset. All registers have write enable.
215 //-----
216 always @ (posedge clk or negedge reset_n)
217     begin: reg_update
218         if (!reset_n)
219             begin
220                 block_w0_reg <= 32'h0;
221                 block_w1_reg <= 32'h0;
222                 block_w2_reg <= 32'h0;
223                 block_w3_reg <= 32'h0;
224                 sword_ctr_reg <= 2'h0;
225                 round_ctr_reg <= 4'h0;
226                 ready_reg <= 1'b1;
227                 enc_ctrl_reg <= CTRL_IDLE;
228             end
229         else
230             begin
231                 if (block_w0_we)
232                     block_w0_reg <= block_new[127 : 096];
233                 if (block_w1_we)
234                     block_w1_reg <= block_new[095 : 064];
235

```

```

236
237     if (block_w2_we)
238         block_w2_reg <= block_new[063 : 032];
239
240     if (block_w3_we)
241         block_w3_reg <= block_new[031 : 000];
242
243     if (sword_ctr_we)
244         sword_ctr_reg <= sword_ctr_new;
245
246     if (round_ctr_we)
247         round_ctr_reg <= round_ctr_new;
248
249     if (ready_we)
250         ready_reg <= ready_new;
251
252     if (enc_ctrl_we)
253         enc_ctrl_reg <= enc_ctrl_new;
254     end
255     end // reg_update
256
257 //-----
258 // round_logic
259 //
260 // The logic needed to implement init, main and final
261 rounds.
262 //-----
263 always @*
264     begin : round_logic
265         reg [127 : 0] old_block, shiftrows_block, mix-
266         columns_block;
267         reg [127 : 0] addkey_init_block, addkey_main_block, ad-
268         dkey_final_block;
269         block_new = 128'h0;
270         muxed_sboxw = 32'h0;
271         block_w0_we = 1'b0;
272         block_w1_we = 1'b0;
273         block_w2_we = 1'b0;
274         block_w3_we = 1'b0;
275         old_block = {block_w0_reg, block_w1_reg,
276         block_w2_reg, block_w3_reg};
277         shiftrows_block = shiftrows(old_block);
278         mixcolumns_block = mixcolumns(shiftrows_block);
279         addkey_init_block = addroundkey(block, round_key);
280         addkey_main_block = addroundkey(mixcolumns_block,
281         round_key);
282         addkey_final_block = addroundkey(shiftrows_block,
283         round_key);
284         case (update_type)
285             INIT_UPDATE:
286                 begin
287                     block_new = addkey_init_block;
288                     block_w0_we = 1'b1;
289                     block_w1_we = 1'b1;
290                     block_w2_we = 1'b1;
291                     block_w3_we = 1'b1;
292                 end
293             SBOX_UPDATE:
294                 begin
295                     block_new = {new_sboxw, new_sboxw, new_sboxw,
296                     new_sboxw};
297                     case (sword_ctr_reg)
298                         2'h0:
299                             begin
300                                 muxed_sboxw = block_w0_reg;
301                                 block_w0_we = 1'b1;
302                             end
303                         2'h1:
304                             begin
305                                 muxed_sboxw = block_w1_reg;
306                                 block_w1_we = 1'b1;
307                             end
308                         2'h2:
309                             begin
310                                 muxed_sboxw = block_w2_reg;
311                                 block_w2_we = 1'b1;
312                             end
313                         2'h3:
314                             begin
315                                 muxed_sboxw = block_w3_reg;
316                                 block_w3_we = 1'b1;
317                             end
318                     endcase // case (sbox_mux_ctrl_reg)
319                 end
320             MAIN_UPDATE:
321                 begin
322                     block_new = addkey_main_block;
323                     block_w0_we = 1'b1;
324                     block_w1_we = 1'b1;
325                     block_w2_we = 1'b1;
326                     block_w3_we = 1'b1;
327

```

```

328         block_w2_we = 1'b1;
329         block_w3_we = 1'b1;
330     end
331
332     FINAL_UPDATE:
333     begin
334         block_new = addkey_final_block;
335         block_w0_we = 1'b1;
336         block_w1_we = 1'b1;
337         block_w2_we = 1'b1;
338         block_w3_we = 1'b1;
339     end
340
341     default:
342     begin
343     end
344     endcase // case (update_type)
345     end // round_logic
346
347 //-----
348 // sword_ctr
349 //
350 // The subbytes word counter with reset and increase logic.
351 //-----
352
353 always @*
354     begin : sword_ctr
355         sword_ctr_new = 2'h0;
356         sword_ctr_we = 1'b0;
357
358         if (sword_ctr_rst)
359             begin
360                 sword_ctr_new = 2'h0;
361                 sword_ctr_we = 1'b1;
362             end
363         else if (sword_ctr_inc)
364             begin
365                 sword_ctr_new = sword_ctr_reg + 1'b1;
366                 sword_ctr_we = 1'b1;
367             end
368         end // sword_ctr
369
370 //-----
371 // round_ctr
372 //
373 // The round counter with reset and increase logic.
374 //-----
375
376 always @*
377     begin : round_ctr
378         round_ctr_new = 4'h0;
379         round_ctr_we = 1'b0;
380
381         if (round_ctr_rst)
382             begin
383                 round_ctr_new = 4'h0;
384                 round_ctr_we = 1'b1;
385             end
386         else if (round_ctr_inc)
387             begin
388                 round_ctr_new = round_ctr_reg + 1'b1;
389                 round_ctr_we = 1'b1;
390             end
391         end // round_ctr
392
393 //-----
394 // encipher_ctrl
395 //
396 // The FSM that controls the encipher operations.
397 //-----
398
399 always @*
400     begin: encipher_ctrl
401         reg [3 : 0] num_rounds;
402
403         // Default assignments.
404         sword_ctr_inc = 1'b0;
405         sword_ctr_rst = 1'b0;
406         round_ctr_inc = 1'b0;
407         round_ctr_rst = 1'b0;
408         ready_new = 1'b0;
409         ready_we = 1'b0;
410         update_type = NO_UPDATE;
411         enc_ctrl_new = CTRL_IDLE;
412         enc_ctrl_we = 1'b0;
413
414         if (keylen == AES_256_BIT_KEY)
415             begin
416                 num_rounds = AES256_ROUNDS;
417             end
418         else
419             begin
420                 num_rounds = AES128_ROUNDS;
421             end
422         end

```

```

423 case(enc_ctrl_reg)
424   CTRL_IDLE:
425     begin
426       if (next)
427         begin
428           round_ctr_rst = 1'b1;
429           ready_new     = 1'b0;
430           ready_we      = 1'b1;
431           enc_ctrl_new  = CTRL_INIT;
432           enc_ctrl_we   = 1'b1;
433         end
434       end
435     end
436   CTRL_INIT:
437     begin
438       round_ctr_inc = 1'b1;
439       sword_ctr_rst = 1'b1;
440       update_type   = INIT_UPDATE;
441       enc_ctrl_new  = CTRL_SBOX;
442       enc_ctrl_we   = 1'b1;
443     end
444   CTRL_SBOX:
445     begin
446       sword_ctr_inc = 1'b1;
447       update_type   = SBOX_UPDATE;
448       if (sword_ctr_reg == 2'h3)
449         begin
450           enc_ctrl_new = CTRL_MAIN;
451           enc_ctrl_we  = 1'b1;
452         end
453       end
454     end
455   CTRL_MAIN:
456     begin
457       sword_ctr_rst = 1'b1;
458       round_ctr_inc = 1'b1;
459       if (round_ctr_reg < num_rounds)
460         begin
461           update_type = MAIN_UPDATE;
462           enc_ctrl_new = CTRL_SBOX;
463           enc_ctrl_we  = 1'b1;
464         end
465       else
466         begin
467           update_type = FINAL_UPDATE;
468           ready_new   = 1'b1;
469           ready_we    = 1'b1;
470           enc_ctrl_new = CTRL_IDLE;
471           enc_ctrl_we  = 1'b1;
472         end
473       end
474     end
475   default:
476     begin
477       // Empty. Just here to make the synthesis tool
478       happy.
479     end
480   endcase // case (enc_ctrl_reg)
481 end // encipher_ctrl
482
483 endmodule // aes_encipher_block
484
485 //
=====
486 // EOF aes_encipher_block.v
487 //
=====

```

5. secworks_aes/src/rtl/aes_inv_sbox.v (325 lines)

```

1 //
=====
2 //
3 // aes_inv_sbox.v
4 // -----
5 // The inverse AES S-box. Basically a 256 Byte ROM.
6 //
7 //
8 // Copyright (c) 2013 Secworks Sweden AB
9 // All rights reserved.
10 //
11 // Redistribution and use in source and binary forms, with or
12 // without modification, are permitted provided that the fol-
13 // lowing conditions are met:
14 //
15 // 1. Redistributions of source code must retain the above
16 // copyright
17 // notice, this list of conditions and the following dis-
18 // claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright
22 // notice, this list of conditions and the following dis-
23 // claimer in
24 // the documentation and/or other materials provided with

```

```

the
21 // distribution.
22 //
23 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
24 // TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
26 // BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
28 // FITNESS
29 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
30 // THE
31 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
32 // INDIRECT,
33 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
34 // (INCLUDING,
35 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
36 // VICES;
37 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
38 // HOWEVER
39 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
40 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
41 // WISE)
42 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
43 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
44 //
=====
45 //
46 // `default_nettype none
47 //
48 module aes_inv_sbox(
49   input wire [31 : 0] sboxw,
50   output wire [31 : 0] new_sboxw
51 );
52
53 //-----
54 // The inverse sbox array.
55 //-----
56 wire [7 : 0] inv_sbox [0 : 255];
57
58 //-----
59 // Four parallel muxes.
60 //-----
61
62 assign new_sboxw[31 : 24] = inv_sbox[sboxw[31 : 24]];
63 assign new_sboxw[23 : 16] = inv_sbox[sboxw[23 : 16]];
64 assign new_sboxw[15 : 08] = inv_sbox[sboxw[15 : 08]];
65 assign new_sboxw[07 : 00] = inv_sbox[sboxw[07 : 00]];
66
67 //-----
68 // Creating the contents of the array.
69 //-----
70
71 assign inv_sbox[8'h00] = 8'h52;
72 assign inv_sbox[8'h01] = 8'h09;
73 assign inv_sbox[8'h02] = 8'h6a;
74 assign inv_sbox[8'h03] = 8'hdf;
75 assign inv_sbox[8'h04] = 8'h30;
76 assign inv_sbox[8'h05] = 8'h36;
77 assign inv_sbox[8'h06] = 8'ha5;
78 assign inv_sbox[8'h07] = 8'h8c;
79 assign inv_sbox[8'h08] = 8'h9b;
80 assign inv_sbox[8'h09] = 8'h40;
81 assign inv_sbox[8'h0a] = 8'ha3;
82 assign inv_sbox[8'h0b] = 8'h9e;
83 assign inv_sbox[8'h0c] = 8'h81;
84 assign inv_sbox[8'h0d] = 8'hf3;
85 assign inv_sbox[8'h0e] = 8'hd7;
86 assign inv_sbox[8'h0f] = 8'hfb;
87 assign inv_sbox[8'h10] = 8'h7c;
88 assign inv_sbox[8'h11] = 8'he3;
89 assign inv_sbox[8'h12] = 8'h39;
90 assign inv_sbox[8'h13] = 8'h82;
91 assign inv_sbox[8'h14] = 8'h9d;
92 assign inv_sbox[8'h15] = 8'h2f;
93 assign inv_sbox[8'h16] = 8'hff;
94 assign inv_sbox[8'h17] = 8'h87;
95 assign inv_sbox[8'h18] = 8'h34;
96 assign inv_sbox[8'h19] = 8'h8e;
97 assign inv_sbox[8'h1a] = 8'h43;
98 assign inv_sbox[8'h1b] = 8'h44;
99 assign inv_sbox[8'h1c] = 8'hc4;
100 assign inv_sbox[8'h1d] = 8'hde;
101 assign inv_sbox[8'h1e] = 8'he9;
102 assign inv_sbox[8'h1f] = 8'hcb;
103 assign inv_sbox[8'h20] = 8'h54;
104 assign inv_sbox[8'h21] = 8'h7b;
105 assign inv_sbox[8'h22] = 8'h94;
106 assign inv_sbox[8'h23] = 8'h32;
107 assign inv_sbox[8'h24] = 8'ha6;
108 assign inv_sbox[8'h25] = 8'hc2;
109 assign inv_sbox[8'h26] = 8'h23;
110 assign inv_sbox[8'h27] = 8'h3d;

```

```

104 assign inv_sbox[8'h28] = 8'hee;
105 assign inv_sbox[8'h29] = 8'h4c;
106 assign inv_sbox[8'h2a] = 8'h95;
107 assign inv_sbox[8'h2b] = 8'h0b;
108 assign inv_sbox[8'h2c] = 8'h42;
109 assign inv_sbox[8'h2d] = 8'hfa;
110 assign inv_sbox[8'h2e] = 8'hc3;
111 assign inv_sbox[8'h2f] = 8'h4e;
112 assign inv_sbox[8'h30] = 8'h08;
113 assign inv_sbox[8'h31] = 8'h2e;
114 assign inv_sbox[8'h32] = 8'ha1;
115 assign inv_sbox[8'h33] = 8'h66;
116 assign inv_sbox[8'h34] = 8'h28;
117 assign inv_sbox[8'h35] = 8'hd9;
118 assign inv_sbox[8'h36] = 8'h24;
119 assign inv_sbox[8'h37] = 8'hb2;
120 assign inv_sbox[8'h38] = 8'h76;
121 assign inv_sbox[8'h39] = 8'h5b;
122 assign inv_sbox[8'h3a] = 8'ha2;
123 assign inv_sbox[8'h3b] = 8'h49;
124 assign inv_sbox[8'h3c] = 8'h6d;
125 assign inv_sbox[8'h3d] = 8'h8b;
126 assign inv_sbox[8'h3e] = 8'hdl;
127 assign inv_sbox[8'h3f] = 8'h25;
128 assign inv_sbox[8'h40] = 8'h72;
129 assign inv_sbox[8'h41] = 8'hf8;
130 assign inv_sbox[8'h42] = 8'hf6;
131 assign inv_sbox[8'h43] = 8'h64;
132 assign inv_sbox[8'h44] = 8'h86;
133 assign inv_sbox[8'h45] = 8'h68;
134 assign inv_sbox[8'h46] = 8'h98;
135 assign inv_sbox[8'h47] = 8'h16;
136 assign inv_sbox[8'h48] = 8'hd4;
137 assign inv_sbox[8'h49] = 8'ha4;
138 assign inv_sbox[8'h4a] = 8'h5c;
139 assign inv_sbox[8'h4b] = 8'hcc;
140 assign inv_sbox[8'h4c] = 8'h5d;
141 assign inv_sbox[8'h4d] = 8'h65;
142 assign inv_sbox[8'h4e] = 8'hb6;
143 assign inv_sbox[8'h4f] = 8'h92;
144 assign inv_sbox[8'h50] = 8'h6c;
145 assign inv_sbox[8'h51] = 8'h70;
146 assign inv_sbox[8'h52] = 8'h48;
147 assign inv_sbox[8'h53] = 8'h50;
148 assign inv_sbox[8'h54] = 8'hfd;
149 assign inv_sbox[8'h55] = 8'hed;
150 assign inv_sbox[8'h56] = 8'hb9;
151 assign inv_sbox[8'h57] = 8'hda;
152 assign inv_sbox[8'h58] = 8'h5e;
153 assign inv_sbox[8'h59] = 8'h15;
154 assign inv_sbox[8'h5a] = 8'h46;
155 assign inv_sbox[8'h5b] = 8'h57;
156 assign inv_sbox[8'h5c] = 8'ha7;
157 assign inv_sbox[8'h5d] = 8'h8d;
158 assign inv_sbox[8'h5e] = 8'h9d;
159 assign inv_sbox[8'h5f] = 8'h84;
160 assign inv_sbox[8'h60] = 8'h90;
161 assign inv_sbox[8'h61] = 8'hd8;
162 assign inv_sbox[8'h62] = 8'hab;
163 assign inv_sbox[8'h63] = 8'h08;
164 assign inv_sbox[8'h64] = 8'h8c;
165 assign inv_sbox[8'h65] = 8'hbc;
166 assign inv_sbox[8'h66] = 8'hd3;
167 assign inv_sbox[8'h67] = 8'h0a;
168 assign inv_sbox[8'h68] = 8'hf7;
169 assign inv_sbox[8'h69] = 8'he4;
170 assign inv_sbox[8'h6a] = 8'h58;
171 assign inv_sbox[8'h6b] = 8'h05;
172 assign inv_sbox[8'h6c] = 8'hb8;
173 assign inv_sbox[8'h6d] = 8'hb3;
174 assign inv_sbox[8'h6e] = 8'h45;
175 assign inv_sbox[8'h6f] = 8'h06;
176 assign inv_sbox[8'h70] = 8'hd0;
177 assign inv_sbox[8'h71] = 8'h2c;
178 assign inv_sbox[8'h72] = 8'h1e;
179 assign inv_sbox[8'h73] = 8'h8f;
180 assign inv_sbox[8'h74] = 8'hca;
181 assign inv_sbox[8'h75] = 8'h3f;
182 assign inv_sbox[8'h76] = 8'h0f;
183 assign inv_sbox[8'h77] = 8'h02;
184 assign inv_sbox[8'h78] = 8'hcl;
185 assign inv_sbox[8'h79] = 8'haf;
186 assign inv_sbox[8'h7a] = 8'hbd;
187 assign inv_sbox[8'h7b] = 8'h03;
188 assign inv_sbox[8'h7c] = 8'h01;
189 assign inv_sbox[8'h7d] = 8'h13;
190 assign inv_sbox[8'h7e] = 8'h8a;
191 assign inv_sbox[8'h7f] = 8'h6b;
192 assign inv_sbox[8'h80] = 8'h3a;
193 assign inv_sbox[8'h81] = 8'h91;
194 assign inv_sbox[8'h82] = 8'h11;
195 assign inv_sbox[8'h83] = 8'h41;
196 assign inv_sbox[8'h84] = 8'h4f;
197 assign inv_sbox[8'h85] = 8'h67;
198 assign inv_sbox[8'h86] = 8'hdc;
199 assign inv_sbox[8'h87] = 8'hea;
200 assign inv_sbox[8'h88] = 8'h97;
201 assign inv_sbox[8'h89] = 8'hf2;
202 assign inv_sbox[8'h8a] = 8'hcf;
203 assign inv_sbox[8'h8b] = 8'hce;
204 assign inv_sbox[8'h8c] = 8'hf0;

```

```

205 assign inv_sbox[8'h8d] = 8'hb4;
206 assign inv_sbox[8'h8e] = 8'he6;
207 assign inv_sbox[8'h8f] = 8'h73;
208 assign inv_sbox[8'h90] = 8'h96;
209 assign inv_sbox[8'h91] = 8'hac;
210 assign inv_sbox[8'h92] = 8'h74;
211 assign inv_sbox[8'h93] = 8'h22;
212 assign inv_sbox[8'h94] = 8'he7;
213 assign inv_sbox[8'h95] = 8'had;
214 assign inv_sbox[8'h96] = 8'h35;
215 assign inv_sbox[8'h97] = 8'h85;
216 assign inv_sbox[8'h98] = 8'he2;
217 assign inv_sbox[8'h99] = 8'hf9;
218 assign inv_sbox[8'h9a] = 8'h37;
219 assign inv_sbox[8'h9b] = 8'he8;
220 assign inv_sbox[8'h9c] = 8'h1c;
221 assign inv_sbox[8'h9d] = 8'h75;
222 assign inv_sbox[8'h9e] = 8'hdf;
223 assign inv_sbox[8'h9f] = 8'h6e;
224 assign inv_sbox[8'ha0] = 8'h47;
225 assign inv_sbox[8'ha1] = 8'hf1;
226 assign inv_sbox[8'ha2] = 8'h1a;
227 assign inv_sbox[8'ha3] = 8'h71;
228 assign inv_sbox[8'ha4] = 8'h1d;
229 assign inv_sbox[8'ha5] = 8'h29;
230 assign inv_sbox[8'ha6] = 8'hc5;
231 assign inv_sbox[8'ha7] = 8'h89;
232 assign inv_sbox[8'ha8] = 8'h6f;
233 assign inv_sbox[8'ha9] = 8'hb7;
234 assign inv_sbox[8'haa] = 8'h62;
235 assign inv_sbox[8'hab] = 8'h0e;
236 assign inv_sbox[8'hac] = 8'haa;
237 assign inv_sbox[8'had] = 8'h18;
238 assign inv_sbox[8'hae] = 8'hbe;
239 assign inv_sbox[8'haf] = 8'h1b;
240 assign inv_sbox[8'hb0] = 8'hfc;
241 assign inv_sbox[8'hb1] = 8'h56;
242 assign inv_sbox[8'hb2] = 8'h3e;
243 assign inv_sbox[8'hb3] = 8'h4b;
244 assign inv_sbox[8'hb4] = 8'hc6;
245 assign inv_sbox[8'hb5] = 8'hd2;
246 assign inv_sbox[8'hb6] = 8'h79;
247 assign inv_sbox[8'hb7] = 8'h20;
248 assign inv_sbox[8'hb8] = 8'h9a;
249 assign inv_sbox[8'hb9] = 8'hdb;
250 assign inv_sbox[8'hba] = 8'hc0;
251 assign inv_sbox[8'hbb] = 8'hfe;
252 assign inv_sbox[8'hbc] = 8'h78;
253 assign inv_sbox[8'hbd] = 8'hcd;
254 assign inv_sbox[8'hbe] = 8'h5a;
255 assign inv_sbox[8'hbf] = 8'hf4;
256 assign inv_sbox[8'hc0] = 8'h1f;
257 assign inv_sbox[8'hc1] = 8'hdd;
258 assign inv_sbox[8'hc2] = 8'ha8;
259 assign inv_sbox[8'hc3] = 8'h33;
260 assign inv_sbox[8'hc4] = 8'h88;
261 assign inv_sbox[8'hc5] = 8'h07;
262 assign inv_sbox[8'hc6] = 8'hc7;
263 assign inv_sbox[8'hc7] = 8'h31;
264 assign inv_sbox[8'hc8] = 8'hb1;
265 assign inv_sbox[8'hc9] = 8'h12;
266 assign inv_sbox[8'hca] = 8'h10;
267 assign inv_sbox[8'hcb] = 8'h59;
268 assign inv_sbox[8'hcc] = 8'h27;
269 assign inv_sbox[8'hcd] = 8'h00;
270 assign inv_sbox[8'hce] = 8'hec;
271 assign inv_sbox[8'hcf] = 8'h5f;
272 assign inv_sbox[8'hd0] = 8'h60;
273 assign inv_sbox[8'hd1] = 8'h51;
274 assign inv_sbox[8'hd2] = 8'h7f;
275 assign inv_sbox[8'hd3] = 8'ha9;
276 assign inv_sbox[8'hd4] = 8'h19;
277 assign inv_sbox[8'hd5] = 8'hb5;
278 assign inv_sbox[8'hd6] = 8'h4a;
279 assign inv_sbox[8'hd7] = 8'h0d;
280 assign inv_sbox[8'hd8] = 8'h2d;
281 assign inv_sbox[8'hd9] = 8'he5;
282 assign inv_sbox[8'hda] = 8'h7a;
283 assign inv_sbox[8'hdb] = 8'h9f;
284 assign inv_sbox[8'hdc] = 8'h93;
285 assign inv_sbox[8'hdd] = 8'hc9;
286 assign inv_sbox[8'hde] = 8'h9c;
287 assign inv_sbox[8'hdf] = 8'hef;
288 assign inv_sbox[8'he0] = 8'ha0;
289 assign inv_sbox[8'he1] = 8'he0;
290 assign inv_sbox[8'he2] = 8'h3b;
291 assign inv_sbox[8'he3] = 8'h4d;
292 assign inv_sbox[8'he4] = 8'hae;
293 assign inv_sbox[8'he5] = 8'h2a;
294 assign inv_sbox[8'he6] = 8'hf5;
295 assign inv_sbox[8'he7] = 8'hb0;
296 assign inv_sbox[8'he8] = 8'hc8;
297 assign inv_sbox[8'he9] = 8'h6b;
298 assign inv_sbox[8'hea] = 8'hbb;
299 assign inv_sbox[8'heb] = 8'h3c;
300 assign inv_sbox[8'hec] = 8'h83;
301 assign inv_sbox[8'hed] = 8'h53;
302 assign inv_sbox[8'hee] = 8'h99;
303 assign inv_sbox[8'hdf] = 8'h61;
304 assign inv_sbox[8'hf0] = 8'h17;
305 assign inv_sbox[8'hf1] = 8'h2b;

```

```

306 assign inv_sbox[8'hf2] = 8'h04;
307 assign inv_sbox[8'hf3] = 8'h7e;
308 assign inv_sbox[8'hf4] = 8'hba;
309 assign inv_sbox[8'hf5] = 8'h77;
310 assign inv_sbox[8'hf6] = 8'hd6;
311 assign inv_sbox[8'hf7] = 8'h26;
312 assign inv_sbox[8'hf8] = 8'he1;
313 assign inv_sbox[8'hf9] = 8'h69;
314 assign inv_sbox[8'hfa] = 8'h14;
315 assign inv_sbox[8'hfb] = 8'h63;
316 assign inv_sbox[8'hfc] = 8'h55;
317 assign inv_sbox[8'hfd] = 8'h21;
318 assign inv_sbox[8'hfe] = 8'h0c;
319 assign inv_sbox[8'hff] = 8'h7d;
320
321 endmodule // aes_inv_sbox
322
323 //
=====
324 // EOF aes_inv_sbox.v
325 //
=====

```

6. secworks_aes/src/rtl/aes_key_mem.v (434 lines)

```

1 //
=====
2 //
3 // aes_key_mem.v
4 // -----
5 // The AES key memory including round key generator.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
copyright
17 // notice, this list of conditions and the following dis-
claimer.
18 //
19 // 2. Redistributions in binary form must reproduce the above
copyright
20 // notice, this list of conditions and the following dis-
claimer in
21 // the documentation and/or other materials provided with
the
22 // distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
26 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
27 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
28 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
29 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 `default_nettype none
40
41 module aes_key_mem(
42     input wire          clk,
43     input wire          reset_n,
44
45     input wire [255 : 0] key,
46     input wire          keylen,
47     input wire          init,
48
49     input wire [3 : 0] round,
50     output wire [127 : 0] round_key,
51     output wire          ready,
52
53     output wire [31 : 0] sboxw,
54

```

```

55     input wire [31 : 0] new_sboxw
56 );
57
58
59 //-----
60 // Parameters.
61 //-----
62
63 localparam AES_128_BIT_KEY = 1'h0;
64 localparam AES_256_BIT_KEY = 1'h1;
65
66 localparam AES_128_NUM_ROUNDS = 10;
67 localparam AES_256_NUM_ROUNDS = 14;
68
69 localparam CTRL_IDLE = 3'h0;
70 localparam CTRL_INIT = 3'h1;
71 localparam CTRL_GENERATE = 3'h2;
72 localparam CTRL_DONE = 3'h3;
73
74 //-----
75 // Registers.
76 //-----
77
78 reg [127 : 0] key_mem [0 : 14];
79 reg [127 : 0] key_mem_new;
80 reg          key_mem_we;
81
82 reg [127 : 0] prev_key0_reg;
83 reg [127 : 0] prev_key0_new;
84 reg          prev_key0_we;
85
86 reg [127 : 0] prev_key1_reg;
87 reg [127 : 0] prev_key1_new;
88 reg          prev_key1_we;
89
90 reg [3 : 0] round_ctr_reg;
91 reg [3 : 0] round_ctr_new;
92 reg          round_ctr_rst;
93 reg          round_ctr_inc;
94 reg          round_ctr_we;
95
96 reg [2 : 0] key_mem_ctrl_reg;
97 reg [2 : 0] key_mem_ctrl_new;
98 reg          key_mem_ctrl_we;
99
100 reg          ready_reg;
101 reg          ready_new;
102 reg          ready_we;
103
104 reg [7 : 0] rcon_reg;
105 reg [7 : 0] rcon_new;
106 reg          rcon_we;
107 reg          rcon_set;
108 reg          rcon_next;
109
110 //-----
111 // Wires.
112 //-----
113
114 reg [31 : 0] tmp_sboxw;
115 reg          round_key_update;
116 reg [127 : 0] tmp_round_key;
117
118 //-----
119 // Concurrent assignments for ports.
120 //-----
121
122 assign round_key = tmp_round_key;
123 assign ready     = ready_reg;
124 assign sboxw     = tmp_sboxw;
125
126 //-----
127 // reg_update
128 //
129 // Update functionality for all registers in the core.
130 // All registers are positive edge triggered with asynchron-
ous
131 // active low reset. All registers have write enable.
132 //-----
133
134 always @ (posedge clk or negedge reset_n)
135 begin: reg_update
136     integer i;
137
138     if (!reset_n)
139     begin
140         for (i = 0 ; i <= AES_256_NUM_ROUNDS ; i = i + 1)
141             key_mem [i] <= 128'h0;
142
143         ready_reg <= 1'b1;
144         rcon_reg  <= 8'h0;
145         round_ctr_reg <= 4'h0;
146     end
147 end

```

```

145     prev_key0_reg    <= 128'h0;
146     prev_key1_reg    <= 128'h0;
147     key_mem_ctrl_reg <= CTRL_IDLE;
148   end
149   else
150     begin
151       if (ready_we)
152         ready_reg <= ready_new;
153
154       if (rcon_we)
155         rcon_reg <= rcon_new;
156
157       if (round_ctr_we)
158         round_ctr_reg <= round_ctr_new;
159
160       if (key_mem_we)
161         key_mem[round_ctr_reg] <= key_mem_new;
162
163       if (prev_key0_we)
164         prev_key0_reg <= prev_key0_new;
165
166       if (prev_key1_we)
167         prev_key1_reg <= prev_key1_new;
168
169       if (key_mem_ctrl_we)
170         key_mem_ctrl_reg <= key_mem_ctrl_new;
171     end
172   end // reg_update
173
174   //-----
175   // key_mem_read
176   //
177   // Combinational read port for the key memory.
178   //-----
179
180   always @*
181   begin : key_mem_read
182     tmp_round_key = key_mem[round];
183   end // key_mem_read
184
185   //-----
186
187   // round_key_gen
188   //
189   // The round key generator logic for AES-128 and AES-256.
190   //-----
191
192   always @*
193   begin: round_key_gen
194     reg [31 : 0] w0, w1, w2, w3, w4, w5, w6, w7;
195     reg [31 : 0] k0, k1, k2, k3;
196     reg [31 : 0] rconw, rotstw, tw, trw;
197
198     // Default assignments.
199     key_mem_new = 128'h0;
200     key_mem_we = 1'b0;
201     prev_key0_new = 128'h0;
202     prev_key0_we = 1'b0;
203     prev_key1_new = 128'h0;
204     prev_key1_we = 1'b0;
205
206     k0 = 32'h0;
207     k1 = 32'h0;
208     k2 = 32'h0;
209     k3 = 32'h0;
210
211     rcon_set = 1'b1;
212     rcon_next = 1'b0;
213
214     // Extract words and calculate intermediate values.
215     // Perform rotation of sbox word etc.
216     w0 = prev_key0_reg[127 : 096];
217     w1 = prev_key0_reg[095 : 064];
218     w2 = prev_key0_reg[063 : 032];
219     w3 = prev_key0_reg[031 : 000];
220
221     w4 = prev_key1_reg[127 : 096];
222     w5 = prev_key1_reg[095 : 064];
223     w6 = prev_key1_reg[063 : 032];
224     w7 = prev_key1_reg[031 : 000];
225
226     rconw = {rcon_reg, 24'h0};
227     tmp_sboxw = w7;
228     rotstw = {new_sboxw[23 : 00], new_sboxw[31 : 24]};
229     trw = rotstw ^ rconw;
230     tw = new_sboxw;
231
232     // Generate the specific round keys.
233     if (round_key_update)
234     begin
235       rcon_set = 1'b0;
236       key_mem_we = 1'b1;
237       case (keylen)
238         AES_128_BIT_KEY:
239         begin
240           if (round_ctr_reg == 0)
241             begin
242               key_mem_new = key[255 : 128];

```

```

242     prev_key1_new = key[255 : 128];
243     prev_key1_we = 1'b1;
244     rcon_next = 1'b1;
245   end
246   else
247     begin
248       k0 = w4 ^ trw;
249       k1 = w5 ^ w4 ^ trw;
250       k2 = w6 ^ w5 ^ w4 ^ trw;
251       k3 = w7 ^ w6 ^ w5 ^ w4 ^ trw;
252
253       key_mem_new = {k0, k1, k2, k3};
254       prev_key1_new = {k0, k1, k2, k3};
255       prev_key1_we = 1'b1;
256       rcon_next = 1'b1;
257     end
258   end
259
260   AES_256_BIT_KEY:
261   begin
262     if (round_ctr_reg == 0)
263     begin
264       key_mem_new = key[255 : 128];
265       prev_key0_new = key[255 : 128];
266       prev_key0_we = 1'b1;
267     end
268     else if (round_ctr_reg == 1)
269     begin
270       key_mem_new = key[127 : 0];
271       prev_key1_new = key[127 : 0];
272       prev_key1_we = 1'b1;
273       rcon_next = 1'b1;
274     end
275     else
276     begin
277       if (round_ctr_reg[0] == 0)
278       begin
279         k0 = w0 ^ trw;
280         k1 = w1 ^ w0 ^ trw;
281         k2 = w2 ^ w1 ^ w0 ^ trw;
282         k3 = w3 ^ w2 ^ w1 ^ w0 ^ trw;
283       end
284       else
285       begin
286         k0 = w0 ^ tw;
287         k1 = w1 ^ w0 ^ tw;
288         k2 = w2 ^ w1 ^ w0 ^ tw;
289         k3 = w3 ^ w2 ^ w1 ^ w0 ^ tw;
290         rcon_next = 1'b1;
291       end
292     end
293
294     // Store the generated round keys.
295     key_mem_new = {k0, k1, k2, k3};
296     prev_key1_new = {k0, k1, k2, k3};
297     prev_key1_we = 1'b1;
298     prev_key0_new = prev_key1_reg;
299     prev_key0_we = 1'b1;
300   end
301
302   default:
303   begin
304     endcase // case (keylen)
305   end
306   end // round_key_gen
307
308   //-----
309
310   // rcon_logic
311   //
312   // Calculates the rcon value for the different key expansion
313   // iterations.
314   //-----
315
316   always @*
317   begin : rcon_logic
318     reg [7 : 0] tmp_rcon;
319     rcon_new = 8'h00;
320     rcon_we = 1'b0;
321
322     tmp_rcon = {rcon_reg[6 : 0], 1'b0} ^ (8'h1b &
323     {8{rcon_reg[7]}));
324
325     if (rcon_set)
326     begin
327       rcon_new = 8'h8d;
328       rcon_we = 1'b1;
329     end
330
331     if (rcon_next)
332     begin
333       rcon_new = tmp_rcon[7 : 0];
334       rcon_we = 1'b1;
335     end
336   end
337
338   //-----

```



```

339 // round_ctr
340 //
341 // The round counter logic with increase and reset.
342 //-----
343 always @*
344 begin : round_ctr
345   round_ctr_new = 4'h0;
346   round_ctr_we = 1'b0;
347
348   if (round_ctr_rst)
349   begin
350     round_ctr_new = 4'h0;
351     round_ctr_we = 1'b1;
352   end
353
354   else if (round_ctr_inc)
355   begin
356     round_ctr_new = round_ctr_reg + 1'b1;
357     round_ctr_we = 1'b1;
358   end
359 end
360
361 //-----
362
363 // key_mem_ctrl
364 //
365 //
366 // The FSM that controls the round key generation.
367 //-----
368 always @*
369 begin: key_mem_ctrl
370   reg [3 : 0] num_rounds;
371
372   // Default assignments.
373   ready_new = 1'b0;
374   ready_we = 1'b0;
375   round_key_update = 1'b0;
376   round_ctr_rst = 1'b0;
377   round_ctr_inc = 1'b0;
378   key_mem_ctrl_new = CTRL_IDLE;
379   key_mem_ctrl_we = 1'b0;
380
381   if (keylen == AES_128_BIT_KEY)
382     num_rounds = AES_128_NUM_ROUNDS;
383   else
384     num_rounds = AES_256_NUM_ROUNDS;
385
386   case(key_mem_ctrl_reg)
387     CTRL_IDLE:
388       begin
389         if (init)
390         begin
391           ready_new = 1'b0;
392           ready_we = 1'b1;
393           key_mem_ctrl_new = CTRL_INIT;
394           key_mem_ctrl_we = 1'b1;
395         end
396       end
397
398     CTRL_INIT:
399       begin
400         round_ctr_rst = 1'b1;
401         key_mem_ctrl_new = CTRL_GENERATE;
402         key_mem_ctrl_we = 1'b1;
403       end
404
405     CTRL_GENERATE:
406       begin
407         round_ctr_inc = 1'b1;
408         round_key_update = 1'b1;
409         if (round_ctr_reg == num_rounds)
410         begin
411           key_mem_ctrl_new = CTRL_DONE;
412           key_mem_ctrl_we = 1'b1;
413         end
414       end
415
416     CTRL_DONE:
417       begin
418         ready_new = 1'b1;
419         ready_we = 1'b1;
420         key_mem_ctrl_new = CTRL_IDLE;
421         key_mem_ctrl_we = 1'b1;
422       end
423
424     default:
425       begin
426       end
427   endcase // case (key_mem_ctrl_reg)
428
429   end // key_mem_ctrl
430 endmodule // aes_key_mem
431
432 //
=====
433 // EOF aes_key_mem.v
434 //

```

7. secworks_aes/src/rtl/aes_sbox.v (327 lines)

```

1 //
=====
2 //
3 // aes_sbox.v
4 // -----
5 // The AES S-box. Basically a 256 Byte ROM. This implementa-
tion
6 // contains four parallel S-boxes to handle a 32 bit word.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2014, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-
claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module aes_sbox(
43   input wire [31 : 0] sboxw,
44   output wire [31 : 0] new_sboxw
45 );
46
47
48 //-----
49 // The sbox array.
50 //-----
51 wire [7 : 0] sbox [0 : 255];
52
53
54 //-----
55 // Four parallel muxes.
56 //-----
57 assign new_sboxw[31 : 24] = sbox[sboxw[31 : 24]];
58 assign new_sboxw[23 : 16] = sbox[sboxw[23 : 16]];
59 assign new_sboxw[15 : 08] = sbox[sboxw[15 : 08]];
60 assign new_sboxw[07 : 00] = sbox[sboxw[07 : 00]];
61
62
63 //-----
64 // Creating the sbox array contents.
65 //-----
66 assign sbox[8'h00] = 8'h63;
67 assign sbox[8'h01] = 8'h7c;
68 assign sbox[8'h02] = 8'h77;
69 assign sbox[8'h03] = 8'h7b;

```

```

70 assign sbox[8'h04] = 8'hf2;
71 assign sbox[8'h05] = 8'h6b;
72 assign sbox[8'h06] = 8'h6f;
73 assign sbox[8'h07] = 8'hc5;
74 assign sbox[8'h08] = 8'h30;
75 assign sbox[8'h09] = 8'h01;
76 assign sbox[8'h0a] = 8'h67;
77 assign sbox[8'h0b] = 8'h2b;
78 assign sbox[8'h0c] = 8'hfe;
79 assign sbox[8'h0d] = 8'hd7;
80 assign sbox[8'h0e] = 8'hab;
81 assign sbox[8'h0f] = 8'h76;
82 assign sbox[8'h10] = 8'hca;
83 assign sbox[8'h11] = 8'h82;
84 assign sbox[8'h12] = 8'hc9;
85 assign sbox[8'h13] = 8'h7d;
86 assign sbox[8'h14] = 8'hfa;
87 assign sbox[8'h15] = 8'h59;
88 assign sbox[8'h16] = 8'h47;
89 assign sbox[8'h17] = 8'hf0;
90 assign sbox[8'h18] = 8'had;
91 assign sbox[8'h19] = 8'hd4;
92 assign sbox[8'h1a] = 8'ha2;
93 assign sbox[8'h1b] = 8'haf;
94 assign sbox[8'h1c] = 8'h9c;
95 assign sbox[8'h1d] = 8'ha4;
96 assign sbox[8'h1e] = 8'h72;
97 assign sbox[8'h1f] = 8'hc0;
98 assign sbox[8'h20] = 8'hb7;
99 assign sbox[8'h21] = 8'hfd;
100 assign sbox[8'h22] = 8'h93;
101 assign sbox[8'h23] = 8'h26;
102 assign sbox[8'h24] = 8'h36;
103 assign sbox[8'h25] = 8'h3f;
104 assign sbox[8'h26] = 8'hf7;
105 assign sbox[8'h27] = 8'hcc;
106 assign sbox[8'h28] = 8'h34;
107 assign sbox[8'h29] = 8'ha5;
108 assign sbox[8'h2a] = 8'he5;
109 assign sbox[8'h2b] = 8'hf1;
110 assign sbox[8'h2c] = 8'h71;
111 assign sbox[8'h2d] = 8'hd8;
112 assign sbox[8'h2e] = 8'h31;
113 assign sbox[8'h2f] = 8'h15;
114 assign sbox[8'h30] = 8'h04;
115 assign sbox[8'h31] = 8'hc7;
116 assign sbox[8'h32] = 8'h23;
117 assign sbox[8'h33] = 8'hc3;
118 assign sbox[8'h34] = 8'h18;
119 assign sbox[8'h35] = 8'h96;
120 assign sbox[8'h36] = 8'ho5;
121 assign sbox[8'h37] = 8'h9a;
122 assign sbox[8'h38] = 8'h07;
123 assign sbox[8'h39] = 8'h12;
124 assign sbox[8'h3a] = 8'h80;
125 assign sbox[8'h3b] = 8'he2;
126 assign sbox[8'h3c] = 8'heb;
127 assign sbox[8'h3d] = 8'h27;
128 assign sbox[8'h3e] = 8'hb2;
129 assign sbox[8'h3f] = 8'h75;
130 assign sbox[8'h40] = 8'h09;
131 assign sbox[8'h41] = 8'h83;
132 assign sbox[8'h42] = 8'h2c;
133 assign sbox[8'h43] = 8'h1a;
134 assign sbox[8'h44] = 8'h1b;
135 assign sbox[8'h45] = 8'h6e;
136 assign sbox[8'h46] = 8'h5a;
137 assign sbox[8'h47] = 8'ha0;
138 assign sbox[8'h48] = 8'h52;
139 assign sbox[8'h49] = 8'h3b;
140 assign sbox[8'h4a] = 8'hd6;
141 assign sbox[8'h4b] = 8'hb3;
142 assign sbox[8'h4c] = 8'h29;
143 assign sbox[8'h4d] = 8'he3;
144 assign sbox[8'h4e] = 8'h2f;
145 assign sbox[8'h4f] = 8'h84;
146 assign sbox[8'h50] = 8'h53;
147 assign sbox[8'h51] = 8'hd1;
148 assign sbox[8'h52] = 8'h00;
149 assign sbox[8'h53] = 8'hed;
150 assign sbox[8'h54] = 8'h20;
151 assign sbox[8'h55] = 8'hfc;
152 assign sbox[8'h56] = 8'hb1;
153 assign sbox[8'h57] = 8'h5b;
154 assign sbox[8'h58] = 8'h6a;
155 assign sbox[8'h59] = 8'hcb;
156 assign sbox[8'h5a] = 8'hbe;
157 assign sbox[8'h5b] = 8'h39;
158 assign sbox[8'h5c] = 8'h4a;
159 assign sbox[8'h5d] = 8'h4c;
160 assign sbox[8'h5e] = 8'h58;
161 assign sbox[8'h5f] = 8'hcf;
162 assign sbox[8'h60] = 8'hd0;
163 assign sbox[8'h61] = 8'hef;
164 assign sbox[8'h62] = 8'haa;
165 assign sbox[8'h63] = 8'hfb;
166 assign sbox[8'h64] = 8'h43;
167 assign sbox[8'h65] = 8'h4d;
168 assign sbox[8'h66] = 8'h33;
169 assign sbox[8'h67] = 8'h85;
170 assign sbox[8'h68] = 8'h45;

171 assign sbox[8'h69] = 8'hf9;
172 assign sbox[8'h6a] = 8'h02;
173 assign sbox[8'h6b] = 8'h7f;
174 assign sbox[8'h6c] = 8'h50;
175 assign sbox[8'h6d] = 8'h3c;
176 assign sbox[8'h6e] = 8'h9f;
177 assign sbox[8'h6f] = 8'ha8;
178 assign sbox[8'h70] = 8'h51;
179 assign sbox[8'h71] = 8'ha3;
180 assign sbox[8'h72] = 8'h40;
181 assign sbox[8'h73] = 8'h8f;
182 assign sbox[8'h74] = 8'h92;
183 assign sbox[8'h75] = 8'h9d;
184 assign sbox[8'h76] = 8'h38;
185 assign sbox[8'h77] = 8'hf5;
186 assign sbox[8'h78] = 8'hbc;
187 assign sbox[8'h79] = 8'hb6;
188 assign sbox[8'h7a] = 8'hda;
189 assign sbox[8'h7b] = 8'h21;
190 assign sbox[8'h7c] = 8'h10;
191 assign sbox[8'h7d] = 8'hff;
192 assign sbox[8'h7e] = 8'hf3;
193 assign sbox[8'h7f] = 8'hd2;
194 assign sbox[8'h80] = 8'hcd;
195 assign sbox[8'h81] = 8'h0c;
196 assign sbox[8'h82] = 8'h13;
197 assign sbox[8'h83] = 8'hec;
198 assign sbox[8'h84] = 8'h5f;
199 assign sbox[8'h85] = 8'h97;
200 assign sbox[8'h86] = 8'h44;
201 assign sbox[8'h87] = 8'h17;
202 assign sbox[8'h88] = 8'hc4;
203 assign sbox[8'h89] = 8'ha7;
204 assign sbox[8'h8a] = 8'h7e;
205 assign sbox[8'h8b] = 8'h3d;
206 assign sbox[8'h8c] = 8'h64;
207 assign sbox[8'h8d] = 8'h5d;
208 assign sbox[8'h8e] = 8'h19;
209 assign sbox[8'h8f] = 8'h73;
210 assign sbox[8'h90] = 8'h60;
211 assign sbox[8'h91] = 8'h81;
212 assign sbox[8'h92] = 8'h4f;
213 assign sbox[8'h93] = 8'hdc;
214 assign sbox[8'h94] = 8'h22;
215 assign sbox[8'h95] = 8'h2a;
216 assign sbox[8'h96] = 8'h90;
217 assign sbox[8'h97] = 8'h88;
218 assign sbox[8'h98] = 8'h46;
219 assign sbox[8'h99] = 8'hee;
220 assign sbox[8'h9a] = 8'hb8;
221 assign sbox[8'h9b] = 8'h14;
222 assign sbox[8'h9c] = 8'hde;
223 assign sbox[8'h9d] = 8'h5e;
224 assign sbox[8'h9e] = 8'h0b;
225 assign sbox[8'h9f] = 8'hdb;
226 assign sbox[8'ha0] = 8'he0;
227 assign sbox[8'ha1] = 8'h32;
228 assign sbox[8'ha2] = 8'h3a;
229 assign sbox[8'ha3] = 8'h0a;
230 assign sbox[8'ha4] = 8'h49;
231 assign sbox[8'ha5] = 8'h06;
232 assign sbox[8'ha6] = 8'h24;
233 assign sbox[8'ha7] = 8'h5c;
234 assign sbox[8'ha8] = 8'hc2;
235 assign sbox[8'ha9] = 8'hd3;
236 assign sbox[8'haa] = 8'hac;
237 assign sbox[8'hab] = 8'h62;
238 assign sbox[8'hac] = 8'h91;
239 assign sbox[8'had] = 8'h95;
240 assign sbox[8'hae] = 8'he4;
241 assign sbox[8'haf] = 8'h79;
242 assign sbox[8'hb0] = 8'he7;
243 assign sbox[8'hb1] = 8'hc8;
244 assign sbox[8'hb2] = 8'h37;
245 assign sbox[8'hb3] = 8'h6d;
246 assign sbox[8'hb4] = 8'h8d;
247 assign sbox[8'hb5] = 8'hd5;
248 assign sbox[8'hb6] = 8'h4e;
249 assign sbox[8'hb7] = 8'ha9;
250 assign sbox[8'hb8] = 8'h6c;
251 assign sbox[8'hb9] = 8'h56;
252 assign sbox[8'hba] = 8'hf4;
253 assign sbox[8'hbb] = 8'hea;
254 assign sbox[8'hbc] = 8'h65;
255 assign sbox[8'hbd] = 8'h7a;
256 assign sbox[8'hbe] = 8'hae;
257 assign sbox[8'hbf] = 8'h08;
258 assign sbox[8'hc0] = 8'hba;
259 assign sbox[8'hc1] = 8'h78;
260 assign sbox[8'hc2] = 8'h25;
261 assign sbox[8'hc3] = 8'h2e;
262 assign sbox[8'hc4] = 8'h1c;
263 assign sbox[8'hc5] = 8'ha6;
264 assign sbox[8'hc6] = 8'hb4;
265 assign sbox[8'hc7] = 8'hc6;
266 assign sbox[8'hc8] = 8'he8;
267 assign sbox[8'hc9] = 8'hdd;
268 assign sbox[8'hca] = 8'h74;
269 assign sbox[8'hcb] = 8'h1f;
270 assign sbox[8'hcc] = 8'h4b;
271 assign sbox[8'hcd] = 8'hbd;

```



```

272 assign sbox[8'hce] = 8'h8b;
273 assign sbox[8'hcf] = 8'h8a;
274 assign sbox[8'hd0] = 8'h70;
275 assign sbox[8'hd1] = 8'h3e;
276 assign sbox[8'hd2] = 8'hb5;
277 assign sbox[8'hd3] = 8'h66;
278 assign sbox[8'hd4] = 8'h48;
279 assign sbox[8'hd5] = 8'h03;
280 assign sbox[8'hd6] = 8'hf6;
281 assign sbox[8'hd7] = 8'h0e;
282 assign sbox[8'hd8] = 8'h61;
283 assign sbox[8'hd9] = 8'h35;
284 assign sbox[8'hda] = 8'h57;
285 assign sbox[8'hdb] = 8'hb9;
286 assign sbox[8'hdc] = 8'h86;
287 assign sbox[8'hdd] = 8'hc1;
288 assign sbox[8'hde] = 8'h1d;
289 assign sbox[8'hdf] = 8'h9e;
290 assign sbox[8'he0] = 8'he1;
291 assign sbox[8'he1] = 8'hf8;
292 assign sbox[8'he2] = 8'h98;
293 assign sbox[8'he3] = 8'h11;
294 assign sbox[8'he4] = 8'h69;
295 assign sbox[8'he5] = 8'hd9;
296 assign sbox[8'he6] = 8'h8e;
297 assign sbox[8'he7] = 8'h94;
298 assign sbox[8'he8] = 8'h9b;
299 assign sbox[8'he9] = 8'h1e;
300 assign sbox[8'hea] = 8'h87;
301 assign sbox[8'heb] = 8'he9;

```

```

302 assign sbox[8'hec] = 8'hce;
303 assign sbox[8'hec] = 8'h55;
304 assign sbox[8'hee] = 8'h28;
305 assign sbox[8'hef] = 8'hdf;
306 assign sbox[8'hf0] = 8'h8c;
307 assign sbox[8'hf1] = 8'ha1;
308 assign sbox[8'hf2] = 8'h89;
309 assign sbox[8'hf3] = 8'h0d;
310 assign sbox[8'hf4] = 8'hbf;
311 assign sbox[8'hf5] = 8'he6;
312 assign sbox[8'hf6] = 8'h42;
313 assign sbox[8'hf7] = 8'h68;
314 assign sbox[8'hf8] = 8'h41;
315 assign sbox[8'hf9] = 8'h99;
316 assign sbox[8'hfa] = 8'h2d;
317 assign sbox[8'hfb] = 8'h0f;
318 assign sbox[8'hfc] = 8'hb0;
319 assign sbox[8'hfd] = 8'h54;
320 assign sbox[8'hfe] = 8'hbb;
321 assign sbox[8'hff] = 8'h16;
322
323 endmodule // aes_sbox
324
325 //
=====
326 // EOF aes_sbox.v
327 //
=====

```

SHA-256 코어 (Verilog)

What. SHA-256 압축 함수와 메시지 스케줄(W 메모리)을 분리한 Verilog 구현.

Why it exists. 해시는 메시지 스케줄이 별도 블록 이 된다는 것을 보여 준다. 64라운드 각각이 $W[t]$ 를 필요로 하고, W 는 이전 W 들로부터 재귀적으로 만들어진다. 이 재귀를 미리 다 펼칠지(메모리 64워드) 그때그때 계산할지(레지스터 16워드 + 순환)가 면적-지연 맞교환의 전형이다.

학부 복습 · **EECS concepts.** 재귀 수열의 하드웨어 사상 · 순환 버퍼 · 면적/시간 맞교환.

석사 수준 · **Practice note.** sha256_w_mem.v 를 먼저 읽어라. ‘16워드 순환’ 기법이 여기 있고, 같은 기법이 LDPC 메시지 저장, FFT 지연선, FIR 지연선에 반복해서 나온다.

4 files • 1,194 lines

API — sha256.v

Table 167. secworks_sha256/src/rtl/sha256.v — module / interface / package (1개)

이름	종류
sha256	module

Table 168. secworks_sha256/src/rtl/sha256.v — localparam (19개)

상수	형/폭	값
ADDR_NAME0		8'h00
ADDR_NAME1		8'h01
ADDR_VERSION		8'h02
ADDR_CTRL		8'h08
CTRL_INIT_BIT		0
CTRL_NEXT_BIT		1
CTRL_MODE_BIT		2
ADDR_STATUS		8'h09
STATUS_READY_BIT		0
STATUS_VALID_BIT		1
ADDR_BLOCK0		8'h10
ADDR_BLOCK15		8'h1f
ADDR_DIGEST0		8'h20
ADDR_DIGEST7		8'h27
CORE_NAME0		32'h73686132
CORE_NAME1		32'h2d323536
CORE_VERSION		32'h312e3830

상수	형/폭	값
MODE_SHA_224		1'h0
MODE_SHA_256		1'h1

Table 169. secworks_sha256/src/rtl/sha256.v — 포트 (8개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cs	input	wire
we	input	wire
address	input	wire [7 : 0]
write_data	input	wire [31 : 0]
read_data	output	wire [31 : 0]
error	output	wire

API — sha256_core.v**Table 170. secworks_sha256/src/rtl/sha256_core.v — module / interface / package (1개)**

이름	종류
sha256_core	module

Table 171. secworks_sha256/src/rtl/sha256_core.v — localparam (20개)

상수	형/폭	값
SHA224_H0_0		32'hc1059ed8
SHA224_H0_1		32'h367cd507
SHA224_H0_2		32'h3070dd17
SHA224_H0_3		32'hf70e5939
SHA224_H0_4		32'hffc00b31
SHA224_H0_5		32'h68581511
SHA224_H0_6		32'h64f98fa7
SHA224_H0_7		32'hbfa4fa4
SHA256_H0_0		32'h6a09e667
SHA256_H0_1		32'hbb67ae85
SHA256_H0_2		32'h3c6ef372
SHA256_H0_3		32'ha54ff53a
SHA256_H0_4		32'h510e527f
SHA256_H0_5		32'h9b05688c
SHA256_H0_6		32'h1f83d9ab

상수	형/폭	값
SHA256_H0_7		32'h5be0cd19
SHA256_ROUNDS		63
CTRL_IDLE		0
CTRL_ROUNDS		1
CTRL_DONE		2

Table 172. secworks_sha256/src/rtl/sha256_core.v — 포트 (9개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
init	input	wire
next	input	wire
mode	input	wire
block	input	wire [511 : 0]
ready	output	wire
digest	output	wire [255 : 0]
digest_valid	output	wire

API — sha256_k_constants.v**Table 173. secworks_sha256/src/rtl/sha256_k_constants.v — module / interface / package (1개)**

이름	종류
sha256_k_constants	module

Table 174. secworks_sha256/src/rtl/sha256_k_constants.v — 포트 (2개)

포트	방향	형/폭
round	input	wire [5 : 0]
K	output	wire [31 : 0]

API — sha256_w_mem.v**Table 175. secworks_sha256/src/rtl/sha256_w_mem.v — module / interface / package (1개)**

이름	종류
sha256_w_mem	module

Table 176. secworks_sha256/src/rtl/sha256_w_mem.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire

포트	방향	형/폭
block	input	wire [511 : 0]
round	input	wire [5 : 0]
init	input	wire
next	input	wire
w	output	wire [31 : 0]

1. secworks_sha256/src/rtl/sha256.v (265 lines)

```

1 //
=====
2 //
3 // sha256.v
4 // -----
5 // Top level wrapper for the SHA-256 hash function providing
6 // a simple memory like interface with 32 bit data access.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, 201, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
15 // lowing conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
18 // copyright notice, this list of conditions and the following dis-
19 // claimer.
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright notice, this list of conditions and the following dis-
22 // claimer in the documentation and/or other materials provided with
23 // the distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
26 // TRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
27 // BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
28 // FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
29 // THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
30 // INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
31 // (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
32 // VICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
33 // HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
35 // WISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module sha256(
43     // Clock and reset.
44     input wire      clk,
45     input wire      reset_n,
46
47     // Control.
48     input wire      cs,
49     input wire      we,
50
51     // Data ports.
52     input wire [7 : 0] address,
53     input wire [31 : 0] write_data,
54     output wire [31 : 0] read_data,
55     output wire      error
56 );
57
58 //-----
59 // Internal constant and parameter definitions.

```

```

60 //-----
61 localparam ADDR_NAME0    = 8'h00;
62 localparam ADDR_NAME1    = 8'h01;
63 localparam ADDR_VERSION  = 8'h02;
64
65 localparam ADDR_CTRL     = 8'h08;
66 localparam CTRL_INIT_BIT = 0;
67 localparam CTRL_NEXT_BIT = 1;
68 localparam CTRL_MODE_BIT = 2;
69
70 localparam ADDR_STATUS   = 8'h09;
71 localparam STATUS_READY_BIT = 0;
72 localparam STATUS_VALID_BIT = 1;
73
74 localparam ADDR_BLOCK0   = 8'h10;
75 localparam ADDR_BLOCK15 = 8'h1f;
76
77 localparam ADDR_DIGEST0  = 8'h20;
78 localparam ADDR_DIGEST7  = 8'h27;
79
80 localparam CORE_NAME0    = 32'h73686132; // "sha2"
81 localparam CORE_NAME1    = 32'h2d323536; // "-256"
82 localparam CORE_VERSION  = 32'h312e3830; // "1.80"
83
84 localparam MODE_SHA_224  = 1'h0;
85 localparam MODE_SHA_256 = 1'h1;
86
87
88 //-----
89 // Registers including update variables and write enable.
90 //-----
91 reg init_reg;
92 reg init_new;
93
94 reg next_reg;
95 reg next_new;
96
97 reg mode_reg;
98 reg mode_new;
99 reg mode_we;
100
101 reg ready_reg;
102
103 reg [31 : 0] block_reg [0 : 15];
104 reg          block_we;
105
106 reg [255 : 0] digest_reg;
107
108 reg digest_valid_reg;
109
110
111 //-----
112 // Wires.
113 //-----
114 wire          core_ready;
115 wire [511 : 0] core_block;
116 wire [255 : 0] core_digest;
117 wire          core_digest_valid;
118
119 reg [31 : 0] tmp_read_data;
120 reg          tmp_error;
121
122
123 //-----
124 // Concurrent connectivity for ports etc.
125 //-----
126 assign core_block = {block_reg[00], block_reg[01],
127                     block_reg[02], block_reg[03],
128                     block_reg[04], block_reg[05],
129                     block_reg[06], block_reg[07],
130                     block_reg[08], block_reg[09],
131                     block_reg[10], block_reg[11],
132                     block_reg[12], block_reg[13],
133                     block_reg[14], block_reg[15]};
134
135 assign read_data = tmp_read_data;

```

```

132 assign error      = tmp_error;
133
134 //-----
135
136 // core instantiation.
137 //-----
138 sha256_core core(
139     .clk(clk),
140     .reset_n(reset_n),
141
142     .init(init_reg),
143     .next(next_reg),
144     .mode(mode_reg),
145
146     .block(core_block),
147
148     .ready(core_ready),
149
150     .digest(core_digest),
151     .digest_valid(core_digest_valid)
152 );
153
154 //-----
155
156 // reg_update
157 //
158 // Update functionality for all registers in the core.
159 // All registers are positive edge triggered with asynchron-
160 // ous
161 // active low reset. All registers have write enable.
162 //-----
163 always @ (posedge clk or negedge reset_n)
164 begin : reg_update
165     integer i;
166
167     if (!reset_n)
168     begin
169         for (i = 0 ; i < 16 ; i = i + 1)
170             block_reg[i] <= 32'h0;
171
172         init_reg      <= 0;
173         next_reg      <= 0;
174         ready_reg     <= 0;
175         mode_reg      <= MODE_SHA_256;
176         digest_reg    <= 256'h0;
177         digest_valid_reg <= 0;
178     end
179     else
180     begin
181         ready_reg      <= core_ready;
182         digest_valid_reg <= core_digest_valid;
183         init_reg       <= init_new;
184         next_reg       <= next_new;
185
186         if (mode_we)
187             mode_reg <= mode_new;
188
189         if (core_digest_valid)
190             digest_reg <= core_digest;
191
192         if (block_we)
193             block_reg[address[3 : 0]] <= write_data;
194     end
195 end // reg_update
196
197 //-----
198 // api_logic
199 //
200 // Implementation of the api logic. If cs is enabled will
201 // try to write to or read from the internal registers.
202 //-----
203 either
204 begin : api_logic
205     init_new      = 0;
206     next_new      = 0;
207     mode_new      = 0;
208     mode_we       = 0;
209     block_we      = 0;
210     tmp_read_data = 32'h0;
211     tmp_error     = 0;
212
213     if (cs)
214     begin
215         if (we)
216         begin
217             if (address == ADDR_CTRL)
218             begin
219                 init_new = write_data[CTRL_INIT_BIT];
220                 next_new = write_data[CTRL_NEXT_BIT];
221                 mode_new = write_data[CTRL_MODE_BIT];
222                 mode_we  = 1;
223             end
224

```

```

225         if ((address >= ADDR_BLOCK0) && (address <=
ADDR_BLOCK15))
226             block_we = 1;
227         end // if (we)
228
229     else
230     begin
231         if ((address >= ADDR_BLOCK0) && (address <=
ADDR_BLOCK15))
232             tmp_read_data = block_reg[address[3 : 0]];
233
234         if ((address >= ADDR_DIGEST0) && (address <=
ADDR_DIGEST7))
235             tmp_read_data = digest_reg[(7 - (address -
ADDR_DIGEST0)) * 32 +: 32];
236
237         case (address)
238             // Read operations.
239             ADDR_NAME0:
240                 tmp_read_data = CORE_NAME0;
241
242             ADDR_NAME1:
243                 tmp_read_data = CORE_NAME1;
244
245             ADDR_VERSION:
246                 tmp_read_data = CORE_VERSION;
247
248             ADDR_CTRL:
249                 tmp_read_data = {29'h0, mode_reg, next_reg,
init_reg};
250
251             ADDR_STATUS:
252                 tmp_read_data = {30'h0, digest_valid_reg,
ready_reg};
253
254             default:
255                 begin
256                     end
257             endcase // case (address)
258         end
259     end
260 end // addr_decoder
261 endmodule // sha256
262
263 //
=====
264 // EOF sha256.v
265 //
=====

```

2. secworks_sha256/src/rtl/sha256_core.v (554 lines)

```

1 //
=====
2 //
3 // sha256_core.v
4 // -----
5 // Verilog 2001 implementation of the SHA-256 hash function.
6 // This is the internal core with wide interfaces.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
15 // lowing
16 // conditions are met:
17 // 1. Redistributions of source code must retain the above
18 // copyright
19 // notice, this list of conditions and the following dis-
20 // claimer.
21 // 2. Redistributions in binary form must reproduce the above
22 // copyright
23 // notice, this list of conditions and the following dis-
24 // claimer in
25 // the documentation and/or other materials provided with
26 // the
27 // distribution.
28 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
29 // TRIBUTORS
30 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
31 // BUT NOT
32 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
33 // FITNESS
34 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
35 // THE
36 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
37 // INDIRECT,
38 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
39 // (INCLUDING,
40 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
41 // VICES;

```

```

32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module sha256_core(
43     input wire      clk,
44     input wire      reset_n,
45
46     input wire      init,
47     input wire      next,
48     input wire      mode,
49
50     input wire [511 : 0] block,
51
52     output wire      ready,
53     output wire [255 : 0] digest,
54     output wire      digest_valid
55 );
56
57
58 //-----
59 // Internal constant and parameter definitions.
60 //-----
61
62 localparam SHA224_H0_0 = 32'hc1059ed8;
63 localparam SHA224_H0_1 = 32'h367cd507;
64 localparam SHA224_H0_2 = 32'h3070dd17;
65 localparam SHA224_H0_3 = 32'hf70e5939;
66 localparam SHA224_H0_4 = 32'hffc00b31;
67 localparam SHA224_H0_5 = 32'h68581511;
68 localparam SHA224_H0_6 = 32'h64f98fa7;
69 localparam SHA224_H0_7 = 32'hbfa4fa4;
70
71 localparam SHA256_H0_0 = 32'h6a09e667;
72 localparam SHA256_H0_1 = 32'hbb67ae85;
73 localparam SHA256_H0_2 = 32'h3c6ef372;
74 localparam SHA256_H0_3 = 32'ha54fff53a;
75 localparam SHA256_H0_4 = 32'h510e527f;
76 localparam SHA256_H0_5 = 32'h9b05688c;
77 localparam SHA256_H0_6 = 32'h1f83d9ab;
78 localparam SHA256_H0_7 = 32'h5be0cd19;
79
80 localparam SHA256_ROUNDS = 63;
81
82 localparam CTRL_IDLE = 0;
83 localparam CTRL_ROUNDS = 1;
84 localparam CTRL_DONE = 2;
85
86 //-----
87 // Registers including update variables and write enable.
88 //-----
89
90 reg [31 : 0] a_reg;
91 reg [31 : 0] a_new;
92 reg [31 : 0] b_reg;
93 reg [31 : 0] b_new;
94 reg [31 : 0] c_reg;
95 reg [31 : 0] c_new;
96 reg [31 : 0] d_reg;
97 reg [31 : 0] d_new;
98 reg [31 : 0] e_reg;
99 reg [31 : 0] e_new;
100 reg [31 : 0] f_reg;
101 reg [31 : 0] f_new;
102 reg [31 : 0] g_reg;
103 reg [31 : 0] g_new;
104 reg [31 : 0] h_reg;
105 reg [31 : 0] h_new;
106 reg a_h_we;
107
108 reg [31 : 0] H0_reg;
109 reg [31 : 0] H0_new;
110 reg [31 : 0] H1_reg;
111 reg [31 : 0] H1_new;
112 reg [31 : 0] H2_reg;
113 reg [31 : 0] H2_new;
114 reg [31 : 0] H3_reg;
115 reg [31 : 0] H3_new;
116 reg [31 : 0] H4_reg;
117 reg [31 : 0] H4_new;
118 reg [31 : 0] H5_reg;
119 reg [31 : 0] H5_new;
120 reg [31 : 0] H6_reg;
121 reg [31 : 0] H6_new;
122 reg [31 : 0] H7_reg;
123 reg [31 : 0] H7_new;
124 reg H_we;

```

```

125 reg [5 : 0] t_ctr_reg;
126 reg [5 : 0] t_ctr_new;
127 reg t_ctr_we;
128 reg t_ctr_inc;
129 reg t_ctr_rst;
130
131 reg digest_valid_reg;
132 reg digest_valid_new;
133 reg digest_valid_we;
134
135 reg [1 : 0] sha256_ctrl_reg;
136 reg [1 : 0] sha256_ctrl_new;
137 reg sha256_ctrl_we;
138
139
140 //-----
141 // Wires.
142 //-----
143
144 reg digest_init;
145 reg digest_update;
146
147 reg state_init;
148 reg state_update;
149
150 reg first_block;
151
152 reg ready_flag;
153
154 reg [31 : 0] t1;
155 reg [31 : 0] t2;
156
157 wire [31 : 0] k_data;
158
159 reg w_init;
160 reg w_next;
161 reg [5 : 0] w_round;
162 wire [31 : 0] w_data;
163
164 //-----
165 // Module instantiations.
166 //-----
167
168 sha256_k_constants k_constants_inst(
169     .round(t_ctr_reg),
170     .K(k_data)
171 );
172
173 sha256_w_mem w_mem_inst(
174     .clk(clk),
175     .reset_n(reset_n),
176
177     .block(block),
178     .round(t_ctr_reg),
179
180     .init(w_init),
181     .next(w_next),
182     .w(w_data)
183 );
184
185 //-----
186 // Concurrent connectivity for ports etc.
187 //-----
188
189 assign ready = ready_flag;
190
191 assign digest = {H0_reg, H1_reg, H2_reg, H3_reg,
192                 H4_reg, H5_reg, H6_reg, H7_reg};
193
194 assign digest_valid = digest_valid_reg;
195
196 //-----
197 // reg_update
198 // Update functionality for all registers in the core.
199 // All registers are positive edge triggered with asynchron-
200 // active low reset. All registers have write enable.
201 //-----
202
203 always @ (posedge clk or negedge reset_n)
204     begin : reg_update
205         if (!reset_n)
206             begin
207                 a_reg      <= 32'h0;
208                 b_reg      <= 32'h0;
209                 c_reg      <= 32'h0;
210                 d_reg      <= 32'h0;
211                 e_reg      <= 32'h0;
212                 f_reg      <= 32'h0;
213                 g_reg      <= 32'h0;
214                 h_reg      <= 32'h0;
215                 H0_reg     <= 32'h0;
216                 H1_reg     <= 32'h0;

```

```

217     H2_reg      <= 32'h0;
218     H3_reg      <= 32'h0;
219     H4_reg      <= 32'h0;
220     H5_reg      <= 32'h0;
221     H6_reg      <= 32'h0;
222     H7_reg      <= 32'h0;
223     digest_valid_reg <= 0;
224     t_ctr_reg    <= 6'h0;
225     sha256_ctrl_reg <= CTRL_IDLE;
226 end
227 else
228 begin
229
230     if (a_h_we)
231     begin
232         a_reg <= a_new;
233         b_reg <= b_new;
234         c_reg <= c_new;
235         d_reg <= d_new;
236         e_reg <= e_new;
237         f_reg <= f_new;
238         g_reg <= g_new;
239         h_reg <= h_new;
240     end
241
242     if (H_we)
243     begin
244         H0_reg <= H0_new;
245         H1_reg <= H1_new;
246         H2_reg <= H2_new;
247         H3_reg <= H3_new;
248         H4_reg <= H4_new;
249         H5_reg <= H5_new;
250         H6_reg <= H6_new;
251         H7_reg <= H7_new;
252     end
253
254     if (t_ctr_we)
255     t_ctr_reg <= t_ctr_new;
256
257     if (digest_valid_we)
258     digest_valid_reg <= digest_valid_new;
259
260     if (sha256_ctrl_we)
261     sha256_ctrl_reg <= sha256_ctrl_new;
262 end
263 end // reg_update
264
265 //-----
266
267 // digest_logic
268 //
269 // The logic needed to init as well as update the digest.
270 //-----
271
272 always @*
273 begin : digest_logic
274     H0_new = 32'h0;
275     H1_new = 32'h0;
276     H2_new = 32'h0;
277     H3_new = 32'h0;
278     H4_new = 32'h0;
279     H5_new = 32'h0;
280     H6_new = 32'h0;
281     H7_new = 32'h0;
282     H_we = 0;
283
284     if (digest_init)
285     begin
286         H_we = 1;
287         if (mode)
288         begin
289             H0_new = SHA256_H0_0;
290             H1_new = SHA256_H0_1;
291             H2_new = SHA256_H0_2;
292             H3_new = SHA256_H0_3;
293             H4_new = SHA256_H0_4;
294             H5_new = SHA256_H0_5;
295             H6_new = SHA256_H0_6;
296             H7_new = SHA256_H0_7;
297         end
298     else
299     begin
300         H0_new = SHA224_H0_0;
301         H1_new = SHA224_H0_1;
302         H2_new = SHA224_H0_2;
303         H3_new = SHA224_H0_3;
304         H4_new = SHA224_H0_4;
305         H5_new = SHA224_H0_5;
306         H6_new = SHA224_H0_6;
307         H7_new = SHA224_H0_7;
308     end
309 end
310
311 if (digest_update)
312 begin
313     H0_new = H0_reg + a_reg;
314     H1_new = H1_reg + b_reg;
315     H2_new = H2_reg + c_reg;
316     H3_new = H3_reg + d_reg;

```

```

316     H4_new = H4_reg + e_reg;
317     H5_new = H5_reg + f_reg;
318     H6_new = H6_reg + g_reg;
319     H7_new = H7_reg + h_reg;
320     H_we = 1;
321 end
322 end // digest_logic
323
324 //-----
325
326 // t1_logic
327 //
328 // The logic for the T1 function.
329 //-----
330
331 always @*
332 begin : t1_logic
333     reg [31 : 0] sum1;
334     reg [31 : 0] ch;
335
336     sum1 = {e_reg[5 : 0], e_reg[31 : 6]} ^
337           {e_reg[10 : 0], e_reg[31 : 11]} ^
338           {e_reg[24 : 0], e_reg[31 : 25]};
339
340     ch = (e_reg & f_reg) ^ ((~e_reg) & g_reg);
341
342     t1 = h_reg + sum1 + ch + w_data + k_data;
343 end // t1_logic
344
345 //-----
346
347 // t2_logic
348 //
349 // The logic for the T2 function
350 //-----
351
352 always @*
353 begin : t2_logic
354     reg [31 : 0] sum0;
355     reg [31 : 0] maj;
356
357     sum0 = {a_reg[1 : 0], a_reg[31 : 2]} ^
358           {a_reg[12 : 0], a_reg[31 : 13]} ^
359           {a_reg[21 : 0], a_reg[31 : 22]};
360
361     maj = (a_reg & b_reg) ^ (a_reg & c_reg) ^ (b_reg &
362 c_reg);
363
364     t2 = sum0 + maj;
365 end // t2_logic
366
367 //-----
368
369 // state_logic
370 //
371 // The logic needed to init as well as update the state during
372 // round processing.
373 //-----
374
375 always @*
376 begin : state_logic
377     a_new = 32'h0;
378     b_new = 32'h0;
379     c_new = 32'h0;
380     d_new = 32'h0;
381     e_new = 32'h0;
382     f_new = 32'h0;
383     g_new = 32'h0;
384     h_new = 32'h0;
385     a_h_we = 0;
386
387     if (state_init)
388     begin
389         a_h_we = 1;
390         if (first_block)
391         begin
392             if (mode)
393             begin
394                 a_new = SHA256_H0_0;
395                 b_new = SHA256_H0_1;
396                 c_new = SHA256_H0_2;
397                 d_new = SHA256_H0_3;
398                 e_new = SHA256_H0_4;
399                 f_new = SHA256_H0_5;
400                 g_new = SHA256_H0_6;
401                 h_new = SHA256_H0_7;
402             end
403         else
404         begin
405             a_new = SHA224_H0_0;
406             b_new = SHA224_H0_1;
407             c_new = SHA224_H0_2;
408             d_new = SHA224_H0_3;
409             e_new = SHA224_H0_4;
410             f_new = SHA224_H0_5;
411             g_new = SHA224_H0_6;
412             h_new = SHA224_H0_7;

```



```

409         end
410     end
411     else
412     begin
413         a_new = H0_reg;
414         b_new = H1_reg;
415         c_new = H2_reg;
416         d_new = H3_reg;
417         e_new = H4_reg;
418         f_new = H5_reg;
419         g_new = H6_reg;
420         h_new = H7_reg;
421     end
422 end
423
424 if (state_update)
425 begin
426     a_new = t1 + t2;
427     b_new = a_reg;
428     c_new = b_reg;
429     d_new = c_reg;
430     e_new = d_reg + t1;
431     f_new = e_reg;
432     g_new = f_reg;
433     h_new = g_reg;
434     a_h_we = 1;
435 end
436 end // state_logic
437
438
439 //-----
440 // t_ctr
441 //
442 // Update logic for the round counter, a monotonically
443 // increasing counter with reset.
444 //-----
445 always @*
446 begin : t_ctr
447     t_ctr_new = 0;
448     t_ctr_we = 0;
449
450     if (t_ctr_rst)
451     begin
452         t_ctr_new = 0;
453         t_ctr_we = 1;
454     end
455
456     if (t_ctr_inc)
457     begin
458         t_ctr_new = t_ctr_reg + 1'b1;
459         t_ctr_we = 1;
460     end
461 end // t_ctr
462
463
464 //-----
465 // sha256_ctrl_fsm
466 //
467 // Logic for the state machine controlling the core beha-
468 // viour.
469 //-----
470 always @*
471 begin : sha256_ctrl_fsm
472     digest_init = 0;
473     digest_update = 0;
474
475     state_init = 0;
476     state_update = 0;
477
478     first_block = 0;
479     ready_flag = 0;
480
481     w_init = 0;
482     w_next = 0;
483
484     t_ctr_inc = 0;
485     t_ctr_rst = 0;
486
487     digest_valid_new = 0;
488     digest_valid_we = 0;
489
490     sha256_ctrl_new = CTRL_IDLE;
491     sha256_ctrl_we = 0;
492
493     case (sha256_ctrl_reg)
494     CTRL_IDLE:
495     begin
496         ready_flag = 1;
497
498         if (init)
499         begin
500             digest_init = 1;
501             w_init = 1;
502             state_init = 1;
503             first_block = 1;
504             t_ctr_rst = 1;

```

```

505             digest_valid_new = 0;
506             digest_valid_we = 1;
507             sha256_ctrl_new = CTRL_ROUNDS;
508             sha256_ctrl_we = 1;
509         end
510
511         if (next)
512         begin
513             t_ctr_rst = 1;
514             w_init = 1;
515             state_init = 1;
516             digest_valid_new = 0;
517             digest_valid_we = 1;
518             sha256_ctrl_new = CTRL_ROUNDS;
519             sha256_ctrl_we = 1;
520         end
521     end
522
523
524     CTRL_ROUNDS:
525     begin
526         w_next = 1;
527         state_update = 1;
528         t_ctr_inc = 1;
529
530         if (t_ctr_reg == SHA256_ROUNDS)
531         begin
532             sha256_ctrl_new = CTRL_DONE;
533             sha256_ctrl_we = 1;
534         end
535     end
536
537
538     CTRL_DONE:
539     begin
540         digest_update = 1;
541         digest_valid_new = 1;
542         digest_valid_we = 1;
543
544         sha256_ctrl_new = CTRL_IDLE;
545         sha256_ctrl_we = 1;
546     end
547 endcase // case (sha256_ctrl_reg)
548 end // sha256_ctrl_fsm
549
550 endmodule // sha256_core
551
552 //

```

```

=====
553 // EOF sha256_core.v
554 //
=====

```

3. secworks_sha256/src/rtl/sha256_k_constants.v (134 lines)

```

1 //
=====
2 //
3 // sha256_k_constants.v
4 // -----
5 // The table K with constants in the SHA-256 hash function.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013, Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
14 // lowing
15 // conditions are met:
16 // 1. Redistributions of source code must retain the above
17 // copyright
18 // notice, this list of conditions and the following dis-
19 // claimer.
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright
22 // notice, this list of conditions and the following dis-
23 // claimer in
24 // the documentation and/or other materials provided with
25 // the
26 // distribution.
27 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
28 // TRIBUTORS
29 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
30 // BUT NOT
31 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
32 // FITNESS
33 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
34 // THE
35 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
36 // INDIRECT,
37 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
38 // (INCLUDING,

```

```

30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 `default_nettype none
40
41 module sha256_k_constants(
42     input wire [5 : 0] round,
43     output wire [31 : 0] K
44 );
45
46 //-----
47 // Wires.
48 //-----
49
50 reg [31 : 0] tmp_K;
51
52 //-----
53 // Concurrent connectivity for ports etc.
54 //-----
55
56 assign K = tmp_K;
57
58 //-----
59 // round_mux
60 //-----
61
62 always @*
63 begin : round_mux
64     case(round)
65         00: tmp_K = 32'h428a2f98;
66         01: tmp_K = 32'h71374491;
67         02: tmp_K = 32'hb5c0fbcf;
68         03: tmp_K = 32'he9b5dba5;
69         04: tmp_K = 32'h3956c25b;
70         05: tmp_K = 32'h59f111f1;
71         06: tmp_K = 32'h923f82a4;
72         07: tmp_K = 32'hab1c5ed5;
73         08: tmp_K = 32'hd807aa98;
74         09: tmp_K = 32'h12835b01;
75         10: tmp_K = 32'h243185be;
76         11: tmp_K = 32'h550c7dc3;
77         12: tmp_K = 32'h72be5d74;
78         13: tmp_K = 32'h80deb1fe;
79         14: tmp_K = 32'h9bdc06a7;
80         15: tmp_K = 32'hc19bf174;
81         16: tmp_K = 32'he49b69c1;
82         17: tmp_K = 32'hef6be4786;
83         18: tmp_K = 32'h0fc19dc6;
84         19: tmp_K = 32'h240ca1cc;
85         20: tmp_K = 32'h2de92c6f;
86         21: tmp_K = 32'h4a7484aa;
87         22: tmp_K = 32'h5cb0a9dc;
88         23: tmp_K = 32'h76f98da;
89         24: tmp_K = 32'h983e5152;
90         25: tmp_K = 32'ha831c66d;
91         26: tmp_K = 32'hb00327c8;
92         27: tmp_K = 32'hbf597fc7;
93         28: tmp_K = 32'hc6e00bf3;
94         29: tmp_K = 32'hd5a79147;
95         30: tmp_K = 32'h06ca6351;
96         31: tmp_K = 32'h14292967;
97         32: tmp_K = 32'h27b70a85;
98         33: tmp_K = 32'h2e1b2138;
99         34: tmp_K = 32'h4d2c6dfc;
100        35: tmp_K = 32'h53380d13;
101        36: tmp_K = 32'h650a7354;
102        37: tmp_K = 32'h766a0abb;
103        38: tmp_K = 32'h81c2c92e;
104        39: tmp_K = 32'h92722c85;
105        40: tmp_K = 32'ha2bfe8a1;
106        41: tmp_K = 32'ha81a664b;
107        42: tmp_K = 32'hc24b8b70;
108        43: tmp_K = 32'hc76c51a3;
109        44: tmp_K = 32'hd192e819;
110        45: tmp_K = 32'hd6990624;
111        46: tmp_K = 32'hf40e3585;
112        47: tmp_K = 32'h106aa070;
113        48: tmp_K = 32'h19a4c116;
114        49: tmp_K = 32'h1e376c08;
115        50: tmp_K = 32'h2748774c;
116        51: tmp_K = 32'h34b0bcb5;
117        52: tmp_K = 32'h391c0cb3;
118        53: tmp_K = 32'h4ed8aa4a;
119        54: tmp_K = 32'h5b9cca4f;
120        55: tmp_K = 32'h682e6fff3;

```

```

120        56: tmp_K = 32'h748f82ee;
121        57: tmp_K = 32'h78a5636f;
122        58: tmp_K = 32'h84c87814;
123        59: tmp_K = 32'h8cc70208;
124        60: tmp_K = 32'h90befffa;
125        61: tmp_K = 32'ha4506ceb;
126        62: tmp_K = 32'hbef9a3f7;
127        63: tmp_K = 32'hc67178f2;
128        endcase // case (round)
129        end // block: round_mux
130    endmodule // sha256_k_constants
131
132 //
=====

```

```

133 // sha256_k_constants.v
134 //
=====

```

4. secworks_sha256/src/rtl/sha256_w_mem.v (241 lines)

```

1 //
=====
2 //
3 // sha256_w_mem_regs.v
4 //
5 // The W memory. This version uses 16 32-bit registers as a
sliding
6 // window to generate the 64 words.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-
claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 `default_nettype none
41
42 module sha256_w_mem(
43     input wire          clk,
44     input wire          reset_n,
45
46     input wire [511 : 0] block,
47     input wire [5 : 0] round,
48
49     input wire          init,
50     input wire          next,
51     output wire [31 : 0] w
52 );
53
54 //-----
55 // Registers including update variables and write enable.
56 //-----

```

```

58 reg [31 : 0] w_mem [0 : 15];
59 reg [31 : 0] w_mem00_new;
60 reg [31 : 0] w_mem01_new;
61 reg [31 : 0] w_mem02_new;
62 reg [31 : 0] w_mem03_new;
63 reg [31 : 0] w_mem04_new;
64 reg [31 : 0] w_mem05_new;
65 reg [31 : 0] w_mem06_new;
66 reg [31 : 0] w_mem07_new;
67 reg [31 : 0] w_mem08_new;
68 reg [31 : 0] w_mem09_new;
69 reg [31 : 0] w_mem10_new;
70 reg [31 : 0] w_mem11_new;
71 reg [31 : 0] w_mem12_new;
72 reg [31 : 0] w_mem13_new;
73 reg [31 : 0] w_mem14_new;
74 reg [31 : 0] w_mem15_new;
75 reg w_mem_we;
76
77 //-----
78
79 // Wires.
80 //-----
81
81 reg [31 : 0] w_tmp;
82 reg [31 : 0] w_new;
83
84
85 //-----
86
86 // Concurrent connectivity for ports etc.
87 //-----
88
88 assign w = w_tmp;
89
90
91 //-----
92
92 // reg_update
93 // Update functionality for all registers in the core.
94 // All registers are positive edge triggered with synchron-
ous
95 // active low reset. All registers have write enable.
96 //-----
97
97 always @(posedge clk or negedge reset_n)
98 begin : reg_update
99 integer i;
100
101 if (!reset_n)
102 begin
103 for (i = 0 ; i < 16 ; i = i + 1) begin
104 w_mem[i] <= 32'h0;
105 end
106 end
107 else
108 begin
109 if (w_mem_we)
110 begin
111 w_mem[00] <= w_mem00_new;
112 w_mem[01] <= w_mem01_new;
113 w_mem[02] <= w_mem02_new;
114 w_mem[03] <= w_mem03_new;
115 w_mem[04] <= w_mem04_new;
116 w_mem[05] <= w_mem05_new;
117 w_mem[06] <= w_mem06_new;
118 w_mem[07] <= w_mem07_new;
119 w_mem[08] <= w_mem08_new;
120 w_mem[09] <= w_mem09_new;
121 w_mem[10] <= w_mem10_new;
122 w_mem[11] <= w_mem11_new;
123 w_mem[12] <= w_mem12_new;
124 w_mem[13] <= w_mem13_new;
125 w_mem[14] <= w_mem14_new;
126 w_mem[15] <= w_mem15_new;
127 end
128 end
129 end // reg_update
130
131
132 //-----
133
133 // select_w
134 //
135 // Mux for the external read operation. This is where we ex-
ract
136 // the W variable.
137 //-----
138
138 always @*
139 begin : select_w
140 if (round < 16)
141 w_tmp = w_mem[round[3 : 0]];
142 else
143 w_tmp = w_new;
144 end // select_w
145
146
147 //-----

```

```

148 // w_new_logic
149 //
150 // Logic that calculates the next value to be inserted into
151 // the sliding window of the memory.
152 //-----
153
153 always @*
154 begin : w_mem_update_logic
155 reg [31 : 0] w_0;
156 reg [31 : 0] w_1;
157 reg [31 : 0] w_9;
158 reg [31 : 0] w_14;
159 reg [31 : 0] d0;
160 reg [31 : 0] d1;
161
162 w_mem00_new = 32'h0;
163 w_mem01_new = 32'h0;
164 w_mem02_new = 32'h0;
165 w_mem03_new = 32'h0;
166 w_mem04_new = 32'h0;
167 w_mem05_new = 32'h0;
168 w_mem06_new = 32'h0;
169 w_mem07_new = 32'h0;
170 w_mem08_new = 32'h0;
171 w_mem09_new = 32'h0;
172 w_mem10_new = 32'h0;
173 w_mem11_new = 32'h0;
174 w_mem12_new = 32'h0;
175 w_mem13_new = 32'h0;
176 w_mem14_new = 32'h0;
177 w_mem15_new = 32'h0;
178 w_mem_we = 0;
179
180 w_0 = w_mem[0];
181 w_1 = w_mem[1];
182 w_9 = w_mem[9];
183 w_14 = w_mem[14];
184
185 d0 = {w_1[6 : 0], w_1[31 : 7]} ^
186 {w_1[17 : 0], w_1[31 : 18]} ^
187 {3'b000, w_1[31 : 3]};
188
189 d1 = {w_14[16 : 0], w_14[31 : 17]} ^
190 {w_14[18 : 0], w_14[31 : 19]} ^
191 {10'b0000000000, w_14[31 : 10]};
192
193 w_new = d1 + w_9 + d0 + w_0;
194
195 if (init)
196 begin
197 w_mem00_new = block[511 : 480];
198 w_mem01_new = block[479 : 448];
199 w_mem02_new = block[447 : 416];
200 w_mem03_new = block[415 : 384];
201 w_mem04_new = block[383 : 352];
202 w_mem05_new = block[351 : 320];
203 w_mem06_new = block[319 : 288];
204 w_mem07_new = block[287 : 256];
205 w_mem08_new = block[255 : 224];
206 w_mem09_new = block[223 : 192];
207 w_mem10_new = block[191 : 160];
208 w_mem11_new = block[159 : 128];
209 w_mem12_new = block[127 : 96];
210 w_mem13_new = block[95 : 64];
211 w_mem14_new = block[63 : 32];
212 w_mem15_new = block[31 : 0];
213 w_mem_we = 1;
214 end
215
216 if (next && (round > 15))
217 begin
218 w_mem00_new = w_mem[01];
219 w_mem01_new = w_mem[02];
220 w_mem02_new = w_mem[03];
221 w_mem03_new = w_mem[04];
222 w_mem04_new = w_mem[05];
223 w_mem05_new = w_mem[06];
224 w_mem06_new = w_mem[07];
225 w_mem07_new = w_mem[08];
226 w_mem08_new = w_mem[09];
227 w_mem09_new = w_mem[10];
228 w_mem10_new = w_mem[11];
229 w_mem11_new = w_mem[12];
230 w_mem12_new = w_mem[13];
231 w_mem13_new = w_mem[14];
232 w_mem14_new = w_mem[15];
233 w_mem15_new = w_new;
234 w_mem_we = 1;
235 end
236 end // w_mem_update_logic
237 endmodule // sha256_w_mem
238
239 //
=====
240 // sha256_w_mem.v
241 //
=====

```

OpenTitan AES -- 양산 보안 RTL

What. Google/lowRISC 의 양산 보안칩에 들어가는 AES. 코어·제어 FSM(3중)·S-box 6중·마스킹 PRNG·클리어링 PRNG·CTR/GCM·새도우 레지스터·TL-UL 인터페이스.

Why it exists. ‘기능이 맞는 AES’와 ‘출하 가능한 AES’의 차이 전부 가 여기 있다. 알고리즘은 secworks 판과 같다. 나머지는 물리적 공격(전력 분석, 레이저 고장 주입), 오용(잘못된 설정), 잔류 데이터(키 삭제)에 대한 대책이다.

학부 복습 · EECS concepts. 부채널 해석(DPA/CPA) · 임계값 구현(threshold implementation)과 확률적 비간섭 · 고장 허용 설계(이중화·상보 논리) · CMOS 스위칭 전력 모델 $P \propto \alpha CV^2 f$.

석사 수준 · Practice note. aes_sbox_dom.sv(1,077줄)와 aes_sbox_lut.sv(120줄)를 나란히 열어 보라. 수학적으로 같은 함수가 9배 크기가 되는 이유가 보안 IP 가격의 근거다. 그리고 기능 회귀는 둘을 구분하지 못한다 — 모델링 팀이 이 한계를 명확히 알고 있어야 한다.

8 files • 4,898 lines

API — aes.sv

Table 177. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — module / interface / package (1개)

이름	종류
aes	module

Table 178. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — parameter (14개)

파라미터	형/폭	기본값
AES192Enable	bit	1
AESGCMEnable	bit	1
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecStartTriggerDelay	int unsigned	0
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RndCnstClearingLfsrSeed	clearing_lfsr_seed_t	RndCnstClearingLfsrSeedDefault
RndCnstClearingLfsrPerm	clearing_lfsr_perm_t	RndCnstClearingLfsrPermDefault
RndCnstClearingSharePerm	clearing_lfsr_perm_t	RndCnstClearingSharePermDefault
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 179. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — localparam (1개)

상수	형/폭	값
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH

Table 180. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
rst_shadowed_ni	input	logic
idle_o	output	prim_mubi_pkg::mubi4_t
output_valid_o	output	logic
input_ready_o	output	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
clk_edn_i	input	logic
rst_edn_ni	input	logic
edn_o	output	edn_pkg::edn_req_t
edn_i	input	edn_pkg::edn_rsp_t
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]

API — aes_cipher_control.sv**Table 181. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — module / interface / package (1개)**

이름	종류
aes_cipher_control	module

Table 182. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — parameter (3개)

파라미터	형/폭	기본값
CiphOpFwdOnly	bit	0
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 183. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — localparam (2개)

상수	형/폭	값
NumSp2VSig	int unsigned	8
Sp2VEnSecBuf	bit [NumSp2VSig-1:0]	8'b1100_0000

Table 184. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control.sv — 포트 (45개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic
key_clear_i	input	logic
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	sp2v_e
sub_bytes_en_o	output	sp2v_e
sub_bytes_out_req_i	input	sp2v_e
sub_bytes_out_ack_o	output	sp2v_e
add_rk_sel_o	output	add_rk_sel_e
key_expand_op_o	output	ciph_op_e
key_full_sel_o	output	key_full_sel_e

포트	방향	형/폭
key_full_we_o	output	sp2v_e
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	sp2v_e
key_expand_en_o	output	sp2v_e
key_expand_out_req_i	input	sp2v_e
key_expand_out_ack_o	output	sp2v_e
key_expand_clear_o	output	logic
key_expand_round_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e

API — aes_cipher_control_fsm.sv

Table 185. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm.sv — module / interface / package (1개)

이름	종류
aes_cipher_control_fsm	module

Table 186. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 187. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic

포트	방향	형/폭
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic

포트	방향	형/폭
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_n.sv

Table 188. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — module / interface / package (1개)

이름	종류
aes_cipher_control_fsm_n	module

Table 189. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 190. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 191. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_n.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_ni	input	logic
in_ready_no	output	logic
out_valid_no	output	logic
out_ready_ni	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_ni	input	logic
dec_key_gen_ni	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic

포트	방향	형/폭
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_no	output	logic
sub_bytes_en_no	output	logic
sub_bytes_out_req_ni	input	logic
sub_bytes_out_ack_no	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_no	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_no	output	logic
key_expand_en_no	output	logic
key_expand_out_req_ni	input	logic
key_expand_out_ack_no	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_ni	input	logic
crypt_d_no	output	logic
dec_key_gen_q_ni	input	logic
dec_key_gen_d_no	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_p.sv**Table 192. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — module / interface / package (1개)**

이름	종류
aes_cipher_control_fsm_p	module

Table 193. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 194. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 195. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_control_fsm_p.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic

포트	방향	형/폭
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_core.sv

Table 196. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — module / interface / package (1개)

이름	종류
aes_cipher_core	module

Table 197. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — parameter (9개)

파라미터	형/폭	기본값
AES192Enable	bit	1
CiphOpFwdOnly	bit	0
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 198. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — localparam (5개)

상수	형/폭	값
NumShares	int	SecMasking ? 2 : 1
WidthPRDRow	int unsigned	4*WidthPRDSBox
NumSp2VSig	int unsigned	3
Sp2VEnSecBuf	bit [NumSp2VSig-1:0]	{NumSp2VSig{1'b1}}
NumCyclesPerRound	int unsigned	(SecSBoxImpl == SBoxImplDom) ? 5 : 1

Table 199. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic

포트	방향	형/폭
key_clear_i	input	logic
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
alert_fatal_i	input	logic
alert_o	output	logic
force_masks_i	input	logic
data_in_mask_o	output	logic [3:0][3:0][7:0]
entropy_req_o	output	logic
entropy_ack_i	input	logic
entropy_i	input	logic [EntropyWidth-1:0]

API — aes_control.sv

Table 200. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — module / interface / package (1개)

이름	종류
aes_control	module

Table 201. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — parameter (3개)

파라미터	형/폭	기본값
AESGCMEnable	bit	0
SecMasking	bit	0
SecStartTriggerDelay	int unsigned	0

Table 202. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — localparam (3개)

상수	형/폭	값
WidthCounter	int unsigned	\$clog2(SecStartTriggerDelay+1)
NumSp2VSig	int unsigned	7 + NumSlicesCtr
Sp2VEnSecBuf	bit [NumSp2VSig-1:0]	'0

Table 203. lowRISC_opentitan/hw/ip/aes/rtl/aes_control.sv — 포트 (87개) — 앞 60개만 표시(전체 87개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ctrl_qe_i	input	logic
ctrl_we_o	output	logic
ctrl_phase_i	input	logic

포트	방향	형/폭
ctrl_err_storage_i	input	logic
op_i	input	aes_op_e
mode_i	input	aes_mode_e
cipher_op_i	input	ciph_op_e
sideload_i	input	logic
prng_reseed_rate_i	input	prs_rate_e
manual_operation_i	input	logic
key_touch_forces_reseed_i	input	logic
ctrl_gcm_qe_i	input	logic
ctrl_gcm_we_o	output	logic
ctrl_gcm_phase_i	input	logic
gcm_init_done_o	output	logic
gcm_phase_i	input	gcm_phase_e
start_i	input	logic
key_iv_data_in_clear_i	input	logic
data_out_clear_i	input	logic
prng_reseed_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
alert_fatal_i	input	logic
alert_o	output	logic
key_sideload_valid_i	input	logic
iv_qe_i	input	logic [NumRegsIv-1:0]
data_in_qe_i	input	logic [NumRegsData-1:0]
data_out_re_i	input	logic [NumRegsData-1:0]
data_in_we_o	output	logic
data_out_sel_o	output	data_out_sel_e
data_out_we_o	output	sp2v_e
data_in_prev_sel_o	output	dip_sel_e
data_in_prev_we_o	output	sp2v_e
state_in_sel_o	output	si_sel_e
add_state_in_sel_o	output	add_si_sel_e
add_state_out_sel_o	output	add_so_sel_e
ctr_inc32_o	output	sp2v_e

포트	방향	형/폭
ctr_incr_o	output	sp2v_e
ctr_ready_i	input	sp2v_e
ctr_we_i	input	sp2v_e [NumSlicesCtr-1:0]
cipher_in_valid_o	output	sp2v_e
cipher_in_ready_i	input	sp2v_e
cipher_out_valid_i	input	sp2v_e
cipher_out_ready_o	output	sp2v_e
cipher_crypt_o	output	sp2v_e
cipher_crypt_i	input	sp2v_e
cipher_dec_key_gen_o	output	sp2v_e
cipher_dec_key_gen_i	input	sp2v_e
cipher_prng_reseed_o	output	logic
cipher_prng_reseed_i	input	logic
cipher_key_clear_o	output	logic
cipher_key_clear_i	input	logic
cipher_data_out_clear_o	output	logic
cipher_data_out_clear_i	input	logic
ghash_in_valid_o	output	sp2v_e
ghash_in_ready_i	input	sp2v_e
ghash_out_valid_i	input	sp2v_e

API — aes_control_fsm.sv

Table 204. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv — module / interface / package (1개)

이름	종류
aes_control_fsm	module

Table 205. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv — parameter (2개)

파라미터	형/폭	기본값
AESGCMEnable	bit	0
SecMasking	bit	0

Table 206. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv — 포트 (86개) — 앞 60개만 표시(전체 86개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ctrl_qe_i	input	logic

포트	방향	형/폭
ctrl_we_o	output	logic
ctrl_phase_i	input	logic
ctrl_err_storage_i	input	logic
op_i	input	aes_op_e
mode_i	input	aes_mode_e
cipher_op_i	input	ciph_op_e
sideload_i	input	logic
prng_reseed_rate_i	input	prs_rate_e
manual_operation_i	input	logic
key_touch_forces_reseed_i	input	logic
ctrl_gcm_qe_i	input	logic
ctrl_gcm_we_o	output	logic
ctrl_gcm_phase_i	input	logic
gcm_init_done_o	output	logic
gcm_phase_i	input	gcm_phase_e
start_i	input	logic
key_iv_data_in_clear_i	input	logic
data_out_clear_i	input	logic
prng_reseed_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
alert_fatal_i	input	logic
alert_o	output	logic
key_sideload_valid_i	input	logic
key_init_qe_i	input	logic [NumSharesKey-1:0][NumRegsKey-1:0]
iv_qe_i	input	logic [NumRegslv-1:0]
data_in_qe_i	input	logic [NumRegsData-1:0]
data_out_re_i	input	logic [NumRegsData-1:0]
data_in_we_o	output	logic
data_out_sel_o	output	data_out_sel_e
data_out_we_o	output	logic
data_in_prev_sel_o	output	dip_sel_e
data_in_prev_we_o	output	logic
state_in_sel_o	output	si_sel_e

포트	방향	형/폭
add_state_in_sel_o	output	add_si_sel_e
add_state_out_sel_o	output	add_so_sel_e
ctr_inc32_o	output	logic
ctr_incr_o	output	logic
ctr_ready_i	input	logic
ctr_we_i	input	logic [NumSlicesCtr-1:0]
cipher_in_valid_o	output	logic
cipher_in_ready_i	input	logic
cipher_out_valid_i	input	logic
cipher_out_ready_o	output	logic
cipher_crypt_o	output	logic
cipher_crypt_i	input	logic
cipher_dec_key_gen_o	output	logic
cipher_dec_key_gen_i	input	logic
cipher_prng_reseed_o	output	logic
cipher_prng_reseed_i	input	logic
cipher_key_clear_o	output	logic
cipher_key_clear_i	input	logic
cipher_data_out_clear_o	output	logic
cipher_data_out_clear_i	input	logic
ghash_in_valid_o	output	logic
ghash_in_ready_i	input	logic

1. lowRISC_opentitan/hw/ip/aes/rtl/aes.sv (328 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES top-level wrapper
6
7 `include "prim_assert.sv"
8
9 module aes
10   import aes_pkg::*;
11   import aes_reg_pkg::*;
12   #(
13     parameter bit      AES192Enable      = 1, // Can be
0 (disable), or 1 (enable).
14     parameter bit      AESGCMEnable      = 1, // Can be
0 (disable), or 1 (enable).
15     parameter bit      SecMasking        = 1, // Can be
0 (no masking), or
16     // 1
17     // core.
18     // masked
19     parameter sbox_impl_e SecSBoxImpl    = SBoxImpl-
Dom, // See aes_pkg.sv
20     parameter int unsigned SecStartTriggerDelay = 0, // Manual
start trigger delay, useful for
21     // SCA

```

```

measurements. A value of e.g. 40
22                                     // allows
the processor to go into sleep
23                                     // before
AES starts operation.
24     parameter bit      SecAllowForcingMasks = 0, // Allow
forcing masks to constant values using
25                                     //
FORCE_MASKS bit in Auxiliary Control
26                                     // Re-
gister. Useful for SCA only.
27     parameter bit      SecSkipPRNGReseeding = 0, // The
current SCA setup doesn't provide enough
28                                     // re-
sources to implement the infrastructure
29                                     // re-
quired for PRNG reseeding (CSRNG, EDN).
30                                     // To en-
able SCA resistance evaluations, we
31                                     // need to
skip reseeding requests.
32                                     // Useful
for SCA only.
33     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
mAlerts{1'b1}},
34     // Number of cycles a differential skew is tolerated on the
alert signal
35     parameter int unsigned AlertSkewCycles = 1,
36     parameter clearing_lfsr_seed_t RndCnstClearingLfsrSeed =
RndCnstClearingLfsrSeedDefault,
37     parameter clearing_lfsr_perm_t RndCnstClearingLfsrPerm =
RndCnstClearingLfsrPermDefault,
38     parameter clearing_lfsr_perm_t RndCnstClearingSharePerm =
RndCnstClearingSharePermDefault,

```

```

39  parameter masking_lfsr_seed_t  RndCnstMaskingLfsrSeed  =
    RndCnstMaskingLfsrSeedDefault,
40  parameter masking_lfsr_perm_t  RndCnstMaskingLfsrPerm  =
    RndCnstMaskingLfsrPermDefault
41 ) (
42  input  logic                    clk_i,
43  input  logic                    rst_ni,
44  input  logic                    rst_shad-
owed_ni,
45
46  // Idle indicator for clock manager
47  output prim_mubi_pkg::mubi4_t   idle_o,
48
49  // Status indicators for DMA integrations
50  output logic                    out-
put_valid_o,
51  output logic                    in-
put_ready_o,
52
53  // Life cycle
54  input  lc_ctrl_pkg::lc_tx_t     lc_escal-
ate_en_i,
55
56  // Entropy distribution network (EDN) interface
57  input  logic                    clk_edn_i,
58  input  logic                    rst_edn_ni,
59  output edn_pkg::edn_req_t       edn_o,
60  input  edn_pkg::edn_rsp_t       edn_i,
61
62  // Key manager (keymgr) key sideload interface
63  input  keymgr_pkg::hw_key_req_t keym-
gr_key_i,
64
65  // Bus interface
66  input  tlul_pkg::tl_h2d_t        tl_i,
67  output tlul_pkg::tl_d2h_t        tl_o,
68
69  // Alerts
70  input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
71  output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o
72 );
73
74  localparam int unsigned EntropyWidth = edn_pkg::END-
POINT_BUS_WIDTH;
75
76  // Signals
77  aes_reg2hw_t          reg2hw;
78  aes_hw2reg_t          hw2reg;
79
80  logic [NumAlerts-1:0] alert;
81  lc_ctrl_pkg::lc_tx_t  lc_escalate_en;
82
83  logic                    edn_req_int;
84  logic                    edn_req_hold_d, edn_req_hold_q;
85  logic                    edn_req;
86  logic                    edn_ack;
87  logic [EntropyWidth-1:0] edn_data;
88  logic                    unused_edn_fips;
89  logic                    entropy_clearing_req, en-
tropy_masking_req;
90  logic                    entropy_clearing_ack, en-
tropy_masking_ack;
91
92  ////////////
93  // Inputs //
94  ////////////
95
96  // SEC_CM: BUS_INTEGRITY
97  // SEC_CM: AUX.CONFIG.SHADOW
98  // SEC_CM: AUX.CONFIG.REGWEN
99  // SEC_CM: KEY.SW_UNREADABLE
100 // SEC_CM: DATA_REG.SW_UNREADABLE
101 // Register interface
102 logic intg_err_alert;
103 logic shadowed_storage_err, shadowed_update_err;
104 aes_reg_top u_reg (
105   .clk_i,
106   .rst_ni,
107   .rst_shadowed_ni,
108   .tl_i,
109   .tl_o,
110   .reg2hw,
111   .hw2reg,
112   .shadowed_storage_err_o(shadowed_storage_err),
113   .shadowed_update_err_o(shadowed_update_err),
114   .intg_err_o(intg_err_alert)
115 );
116
117 // SEC_CM: LC_ESCALATE_EN.INTERSIG.MUBI
118 // Synchronize life cycle input
119 prim_lc_sync #(
120   .NumCopies (1)
121 ) u_prim_lc_sync (
122   .clk_i,
123   .rst_ni,
124   .lc_en_i ( lc_escalate_en_i ),
125   .lc_en_o ( {lc_escalate_en} )
126 );
127

```

```

128  ////////////
129  // EDN Interface //
130  ////////////
131
132  // Internally, we have up to two PRNGs that share the EDN
interface for reseeding. Here, we just
133  // arbitrate the requests. Upsizing of the entropy to the
correct width is performed inside the
134  // PRNGs.
135  // Reseed operations for the clearing PRNG are initiated by
software. Reseed operations for the
136  // masking PRNG can also be automatically initiated.
137  assign edn_req_int = entropy_clearing_req | en-
tropy_masking_req;
138  // Only forward ACK to PRNG currently requesting entropy.
Give higher priority to clearing PRNG.
139  assign entropy_clearing_ack = entropy_clearing_req &
edn_ack;
140  assign entropy_masking_ack = ~entropy_clearing_req & en-
tropy_masking_req & edn_ack;
141
142  // Upon escalation or detection of a fatal alert, an EDN re-
quest signal can be dropped before
143  // getting acknowledged. This is okay with respect to AES as
the module will need to be reset
144  // anyway. However, to not leave EDN in a strange state, we
hold the request until it's actually
145  // acknowledged.
146  assign edn_req = edn_req_int | edn_req_hold_q;
147  assign edn_req_hold_d = (edn_req_hold_q | edn_req) &
~edn_ack;
148  always_ff @(posedge clk_i or negedge rst_ni) begin :
edn_req_reg
149    if (!rst_ni) begin
150      edn_req_hold_q <= '0;
151    end else begin
152      edn_req_hold_q <= edn_req_hold_d;
153    end
154  end
155
156  // Synchronize EDN interface
157  prim_sync_reqack_data #(
158    .Width(EntropyWidth),
159    .DataSrc2Dst(1'b0),
160    .DataReg(1'b0)
161  ) u_prim_sync_reqack_data (
162    .clk_src_i ( clk_i ),
163    .rst_src_ni ( rst_ni ),
164    .clk_dst_i ( clk_edn_i ),
165    .rst_dst_ni ( rst_edn_ni ),
166    .req_chk_i ( 1'b1 ),
167    .src_req_i ( edn_req ),
168    .src_ack_o ( edn_ack ),
169    .dst_req_o ( edn_o.edn_req ),
170    .dst_ack_i ( edn_i.edn_ack ),
171    .data_i ( edn_i.edn_bus ),
172    .data_o ( edn_data )
173  );
174  // We don't track whether the entropy is pre-FIPS or not in-
side AES.
175  assign unused_edn_fips = edn_i.edn_fips;
176
177  ////////////
178  // Core //
179  ////////////
180
181  // AES core
182  aes_core #(
183    .AES192Enable ( AES192Enable ),
184    .AESGCMEnable ( AESGCMEnable ),
185    .SecMasking ( SecMasking ),
186    .SecSBoxImpl ( SecSBoxImpl ),
187    .SecStartTriggerDelay ( SecStartTriggerDelay ),
188    .SecAllowForcingMasks ( SecAllowForcingMasks ),
189    .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
190    .EntropyWidth ( EntropyWidth ),
191    .RndCnstClearingLfsrSeed ( RndCnstClearingLfsrSeed ),
192    .RndCnstClearingLfsrPerm ( RndCnstClearingLfsrPerm ),
193    .RndCnstClearingSharePerm ( RndCnstClearingSharePerm ),
194    .RndCnstMaskingLfsrSeed ( RndCnstMaskingLfsrSeed ),
195    .RndCnstMaskingLfsrPerm ( RndCnstMaskingLfsrPerm )
196  ) u_aes_core (
197    .clk_i
198    ( clk_i ),
199    .rst_ni
200    ( rst_ni ),
201    .rst_shadowed_ni
202    ( rst_shad-
owed_ni ),
203    .entropy_clearing_req_o ( en-
tropy_clearing_req ),
204    .entropy_clearing_ack_i ( en-
tropy_clearing_ack ),
205    .entropy_clearing_i
206    ( edn_data ),
207    .entropy_masking_req_o ( en-
tropy_masking_req ),
208    .entropy_masking_ack_i ( en-
tropy_masking_ack ),
209    .entropy_masking_i
210    ( edn_data ),

```

```

206
207 .keymgr_key_i      ( keymgr_key_i ),
gr_key_i
208
209 .lc_escalate_en_i  ( lc_escalate_en ),
ate_en
210
211 .shadowed_storage_err_i ( shadowed_storage_err ),
owed_storage_err
212
213 .shadowed_update_err_i ( shadowed_update_err ),
owed_update_err
214
215 .intg_err_alert_i    ( intg_err_alert ),
(intg_err_alert)
216
217 .alert_recov_o       ( alert_recov_o ),
(alert[AlertRecovCtrlUpdateErrIdx])
218
219 .alert_fatal_o       ( alert_fatal_o ),
(alert[AlertFatalFaultIdx])
220
221 .reg2hw              ( reg2hw ),
(reg2hw)
222
223 .hw2reg              ( hw2reg ),
(hw2reg)
224
225 );
226
227 assign idle_o =
prim_mubi_pkg::mubi4_bool_to_mubi(reg2hw.status.idle.q);
228
229 assign output_valid_o = reg2hw.status.output_valid.q;
230
231 assign input_ready_o = reg2hw.status.input_ready.q;
232
233 // Alerts
234 // Alerts
235 // Alerts
236
237 logic [NumAlerts-1:0] alert_test;
238 assign alert_test[AlertRecovCtrlUpdateErrIdx] =
reg2hw.alert_test.recov_ctrl_update_err.q &
reg2hw.alert_test.recov_ctrl_update_err.qe;
239
240 assign alert_test[AlertFatalFaultIdx] =
reg2hw.alert_test.fatal_fault.q &
reg2hw.alert_test.fatal_fault.qe;
241
242 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
243   prim_alert_sender #(
244     .AsyncOn(AlertAsyncOn[i]),
245     .SkewCycles(AlertSkewCycles),
246     .IsFatal(i == AlertFatalFaultIdx)
247   ) u_prim_alert_sender (
248     .clk_i,
249     .rst_ni,
250     .alert_test_i ( alert_test[i] ),
251     .alert_req_i ( alert[i] ),
252     .alert_ack_o ( ),
253     .alert_state_o ( ),
254     .alert_rx_i ( alert_rx_i[i] ),
255     .alert_tx_o ( alert_tx_o[i] )
256   );
257 end
258
259 // Assertions
260 // Assertions
261 // Assertions
262
263 // All outputs should have a known value after reset
264 `ASSERT_KNOWN(T10DValidKnown, tl_o.d_valid)
265 `ASSERT_KNOWN(T10AReadyKnown, tl_o.a_ready)
266 `ASSERT_KNOWN(IdleKnown, idle_o)
267 `ASSERT_KNOWN(OutputValidKnown, output_valid_o)
268 `ASSERT_KNOWN(InputReadyKnown, input_ready_o)
269 `ASSERT_KNOWN(EdnReqKnown, edn_o)
270 `ASSERT_KNOWN(AlertTxKnown, alert_tx_o)
271
272 // Alert assertions for sparse FSMs.
273 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_ctr_fsm_svas
274   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_ctr_fsm_svas_p
275     `AS-
276     SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrlFsmCheck_A,
277       u_aes_core.u_aes_ctrl.gen_fsm[i].gen_fsm.p,
278       u_aes_ctrl_fsm_i.u_aes_ctrl_fsm.u_state_regs,
279       alert_tx_o[1])
280   end else begin : gen_ctr_fsm_svas_n
281     `AS-
282     SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrlFsmCheck_A,
283       u_aes_core.u_aes_ctrl.gen_fsm[i].gen_fsm.n,
284       u_aes_ctrl_fsm_i.u_aes_ctrl_fsm.u_state_regs,
285       alert_tx_o[1])
286   end
287 end
288
289 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_cipher_control_fsm_svas
290   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
gen_cipher_control_fsm_svas_p
291     `AS-
292     SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
293       u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.p,
294       u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
295       alert_tx_o[1])
296   end else begin : gen_cipher_control_fsm_svas_n
297     `AS-
298     SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
299       u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.n,
300       u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
301       alert_tx_o[1])
302   end
303 end
304
305 if (AESGCMEEnable) begin : gen_ghash_fsm_sva
306   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesGhashFsmCheck_A,
307     u_aes_core.gen_ghash.u_aes_ghash.u_state_regs,
308     alert_tx_o[1])
309 end
310
311 if (AESGCMEEnable && SecMasking) begin : gen_ghash_onehot_sva
312   for (genvar s = 0; s < 2; s++) begin :
gen_ghash_onehot_add_in_sva
313     `AS-
314     SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashAadOnehotCheck_A,
315       u_aes_core.gen_ghash.u_aes_ghash.gen_masked_add.gen_add_in_muxes[s],
316       u_prim_onehot_check_add_in_sel,
317       alert_tx_o[1])
318   end
319 end
320
321 SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashMultOnehotCheck_A,
322   u_aes_core.gen_ghash.u_aes_ghash.gen_gf_mult1_mux.u_prim_onehot_che
323   ck_gf_mult1_in_sel,
324   alert_tx_o[1])
325 end
326
327 // Alert assertions for reg_we onehot check
328 `AS-
329 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
330   u_reg, alert_tx_o[1])
331 endmodule

```

```

286 end else begin : gen_ctr_fsm_svas_n
287   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrlFsmCheck_A,
288     u_aes_core.u_aes_ctrl.gen_fsm[i].gen_fsm.n,
289     u_aes_ctrl_fsm_i.u_aes_ctrl_fsm.u_state_regs,
290     alert_tx_o[1])
291 end
292 end
293
294 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_cipher_control_fsm_svas
295   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
gen_cipher_control_fsm_svas_p
296     `AS-
297     SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
298       u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.p,
299       u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
300       alert_tx_o[1])
301   end else begin : gen_cipher_control_fsm_svas_n
302     `AS-
303     SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
304       u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm.n,
305       u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
306       alert_tx_o[1])
307   end
308 end
309
310 if (AESGCMEEnable) begin : gen_ghash_fsm_sva
311   `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesGhashFsmCheck_A,
312     u_aes_core.gen_ghash.u_aes_ghash.u_state_regs,
313     alert_tx_o[1])
314 end
315
316 if (AESGCMEEnable && SecMasking) begin : gen_ghash_onehot_sva
317   for (genvar s = 0; s < 2; s++) begin :
gen_ghash_onehot_add_in_sva
318     `AS-
319     SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashAadOnehotCheck_A,
320       u_aes_core.gen_ghash.u_aes_ghash.gen_masked_add.gen_add_in_muxes[s],
321       u_prim_onehot_check_add_in_sel,
322       alert_tx_o[1])
323   end
324 end
325
326 SERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashMultOnehotCheck_A,
327   u_aes_core.gen_ghash.u_aes_ghash.gen_gf_mult1_mux.u_prim_onehot_che
328   ck_gf_mult1_in_sel,
329   alert_tx_o[1])
330 end
331
332 // Alert assertions for reg_we onehot check
333 `AS-
334 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
335   u_reg, alert_tx_o[1])
336 endmodule

```

2. lowRISC_opentitan/hw/ipl/aes/rtl/aes_cipher_control.sv (485 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // AES cipher core control
7 // This module controls the AES cipher core including the key
8 // expand module.
9 `include "prim_assert.sv"
10
11 module aes_cipher_control import aes_pkg::*;
12 #(
13   parameter bit CiphOpFwdOnly = 0,
14   parameter bit SecMasking = 0,
15   parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
16 ) (
17   input logic clk_i,
18   input logic rst_ni,
19
20   // Input handshake signals
21   input sp2v_e in_valid_i,
22   output sp2v_e in_ready_o,
23
24   // Output handshake signals
25   output sp2v_e out_valid_o,
26   input sp2v_e out_ready_i,
27
28   // Control and sync signals
29   input logic cfg_valid_i,
30   input ciph_op_e op_i,
31   input key_len_e key_len_i,
32   input sp2v_e crypt_i,

```

```

33 output sp2v_e      crypt_o,
34 input  sp2v_e      dec_key_gen_i,
35 output sp2v_e      dec_key_gen_o,
36 input  logic        prng_reseed_i,
37 output logic        prng_reseed_o,
38 input  logic        key_clear_i,
39 output logic        key_clear_o,
40 input  logic        data_out_clear_i,
41 output logic        data_out_clear_o,
42 input  logic        mux_sel_err_i,
43 input  logic        sp_enc_err_i,
44 input  logic        op_err_i,
45 input  logic        alert_fatal_i,
46 output logic        alert_o,
47
48 // Control signals for masking PRNG
49 output logic        prng_update_o,
50 output logic        prng_reseed_req_o,
51 input  logic        prng_reseed_ack_i,
52
53 // Control and sync signals for cipher data path
54 output state_sel_e  state_sel_o,
55 output sp2v_e      state_we_o,
56 output sp2v_e      sub_bytes_en_o,
57 input  sp2v_e      sub_bytes_out_req_i,
58 output sp2v_e      sub_bytes_out_ack_o,
59 output add_rk_sel_e add_rk_sel_o,
60
61 // Control and sync signals for key expand data path
62 output ciph_op_e    key_expand_op_o,
63 output key_full_sel_e key_full_sel_o,
64 output sp2v_e      key_full_we_o,
65 output key_dec_sel_e key_dec_sel_o,
66 output sp2v_e      key_dec_we_o,
67 output sp2v_e      key_expand_en_o,
68 input  sp2v_e      key_expand_out_req_i,
69 output sp2v_e      key_expand_out_ack_o,
70 output logic        key_expand_clear_o,
71 output logic [3:0]  key_expand_round_o,
72 output key_words_sel_e key_words_sel_o,
73 output round_key_sel_e round_key_sel_o,
74 );
75
76 // Signals
77 logic [3:0] rnd_ctr;
78 sp2v_e      crypt_d, crypt_q;
79 sp2v_e      dec_key_gen_d,
80 dec_key_gen_q;
81 logic        prng_reseed_d, prng_reseed_q;
82 logic        key_clear_d,
83 key_clear_q;
84 logic        data_out_clear_d,
85 data_out_clear_q;
86 sp2v_e      sub_bytes_out_req;
87 sp2v_e      key_expand_out_req;
88 sp2v_e      in_valid;
89 sp2v_e      out_ready;
90 sp2v_e      crypt;
91 sp2v_e      dec_key_gen;
92 logic        mux_sel_err;
93 logic        mr_err;
94 logic        sp_enc_err;
95 logic        rnd_ctr_err;
96
97 // Sparsified FSM signals. These are needed for connecting
98 // the individual bits of the Sp2V
99 // signals to the single-rail FSMs.
100 logic [Sp2VWidth-1:0] sp_in_valid;
101 logic [Sp2VWidth-1:0] sp_in_ready;
102 logic [Sp2VWidth-1:0] sp_out_valid;
103 logic [Sp2VWidth-1:0] sp_out_ready;
104 logic [Sp2VWidth-1:0] sp_crypt;
105 logic [Sp2VWidth-1:0] sp_dec_key_gen;
106 logic [Sp2VWidth-1:0] sp_state_we;
107 logic [Sp2VWidth-1:0] sp_sub_bytes_en;
108 logic [Sp2VWidth-1:0] sp_sub_bytes_out_req;
109 logic [Sp2VWidth-1:0] sp_sub_bytes_out_ack;
110 logic [Sp2VWidth-1:0] sp_key_full_we;
111 logic [Sp2VWidth-1:0] sp_key_dec_we;
112 logic [Sp2VWidth-1:0] sp_key_expand_en;
113 logic [Sp2VWidth-1:0] sp_key_expand_out_req;
114 logic [Sp2VWidth-1:0] sp_key_expand_out_ack;
115 logic [Sp2VWidth-1:0] sp_crypt_d;
116 logic [Sp2VWidth-1:0] sp_crypt_q;
117 logic [Sp2VWidth-1:0] sp_dec_key_gen_d;
118 logic [Sp2VWidth-1:0] sp_dec_key_gen_q;
119
120 // Multi-rail signals. These are outputs of the single-rail
121 // FSMs and need combining.
122 logic [Sp2VWidth-1:0] mr_alert;
123 logic [Sp2VWidth-1:0] mr_prng_update;
124 logic [Sp2VWidth-1:0] mr_prng_reseed_req;
125 logic [Sp2VWidth-1:0] mr_key_expand_clear;
126 logic [Sp2VWidth-1:0] mr_prng_reseed_d;
127 logic [Sp2VWidth-1:0] mr_key_clear_d;
128 logic [Sp2VWidth-1:0] mr_data_out_clear_d;
129
130 state_sel_e [Sp2VWidth-1:0] mr_state_sel;
131 add_rk_sel_e [Sp2VWidth-1:0] mr_add_rk_sel;
132 key_full_sel_e [Sp2VWidth-1:0] mr_key_full_sel;
133
134 key_dec_sel_e [Sp2VWidth-1:0] mr_key_dec_sel;
135 key_words_sel_e [Sp2VWidth-1:0] mr_key_words_sel;
136 round_key_sel_e [Sp2VWidth-1:0] mr_round_key_sel;
137
138 logic [Sp2VWidth-1:0][3:0] mr_rnd_ctr;
139
140 // FSM
141
142 // Convert sp2v_e signals to sparsified inputs.
143 assign sp_in_valid = {in_valid};
144 assign sp_out_ready = {out_ready};
145 assign sp_crypt = {crypt};
146 assign sp_dec_key_gen = {dec_key_gen};
147 assign sp_sub_bytes_out_req = {sub_bytes_out_req};
148 assign sp_key_expand_out_req = {key_expand_out_req};
149 assign sp_crypt_q = {crypt_q};
150 assign sp_dec_key_gen_q = {dec_key_gen_q};
151
152 // SEC_CM: CIPHER.FSM.RETURN
153 // SEC_CM: CIPHER.CTR.RETURN
154 // For every bit in the Sp2V signals, one separate rail is
155 // instantiated. The inputs and outputs
156 // of every rail are buffered to prevent aggressive synthesis
157 // optimizations.
158 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
159   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p
160     aes_cipher_control_fsm_p # (
161       .SecMasking ( SecMasking ),
162       .SecSBoxImpl ( SecSBoxImpl ),
163       .u_aes_cipher_control_fsm_i (
164         .clk_i ( clk_i ),
165         .rst_i ( rst_i ),
166         .in_valid_i ( sp_in_val-
167           id[i] ), // Sparsified
168         .in_ready_o ( sp_in_ready[i] ), // Sparsified
169         .out_valid_o ( sp_out_val-
170           id[i] ), // Sparsified
171         .out_ready_i ( sp_out_ready[i] ), // Sparsified
172         .cfg_valid_i ( cfg_valid_i ),
173         .op_i ( op_i ),
174         .key_len_i ( key_len_i ),
175         .crypt_i ( crypt_i ),
176         .sp_crypt[i] ( sp_crypt[i] ), // Sparsified
177         .sp_dec_key_gen[i] ( sp_dec_key_gen[i] ), // Sparsified
178         .prng_reseed_i ( prng_reseed_i ),
179         .key_clear_i ( key_clear_i ),
180         .data_out_clear_i ( data_out_clear_i ),
181         .mux_sel_err_i ( mux_sel_err ),
182         .sp_enc_err_i ( sp_enc_err ),
183         .rnd_ctr_err_i ( rnd_ctr_err ),
184         .op_err_i ( op_err_i ),
185         .alert_fatal_i ( alert_fatal_i ),
186         .alert_o ( alert_o ),
187         .mr_alert[i] ( mr_alert[i] ), // OR-combine
188         .prng_update_o ( mr_prng_update[i] ), // OR-combine
189         .prng_reseed_req_o ( mr_prng_reseed_req[i] ), // OR-combine
190         .prng_reseed_ack_i ( prng_reseed_ack_i ),
191         .state_sel_o ( mr_state_sel[i] ), // OR-combine
192         .state_we_o ( state_we_o ), // Sparsified
193         .sub_bytes_en_o ( sp_sub_bytes_en[i] ), // Sparsified
194         .sub_bytes_out_req_i ( sp_sub_bytes_out_req[i] ), // Sparsified
195         .sub_bytes_out_ack_o ( sp_sub_bytes_out_ack[i] ), // Sparsified
196         .add_rk_sel_o ( mr_add_rk_sel[i] ), // OR-combine
197         .key_full_sel_o ( mr_key_full_sel[i] ), // OR-combine
198         .key_full_we_o ( sp_key_full_we[i] ), // Sparsified
199         .key_dec_sel_o ( mr_key_dec_sel[i] ), // OR-combine
200         .key_dec_we_o ( sp_key_dec_we[i] ), // Sparsified
201         .key_expand_en_o ( sp_key_expand_en[i] ), // Sparsified
202         .key_expand_out_req_i ( sp_key_expand_out_req[i] ), // Sparsified
203         .key_expand_out_ack_o ( sp_key_expand_out_ack[i] ), // Sparsified
204         .key_expand_clear_o ( mr_key_expand_clear[i] ), // OR-combine
205         .rnd_ctr_o ( mr_rnd_ctr[i] ), // OR-combine
206         .key_words_sel_o ( key_words_sel_o )

```

```

( mr_key_words_sel[i] ), // OR-combine
203 .round_key_sel_o
( mr_round_key_sel[i] ), // OR-combine
204
205 .crypt_q_i
( sp_crypt_q[i] ), // Sparsified
206 .crypt_d_o
( sp_crypt_d[i] ), // Sparsified
207 .dec_key_gen_q_i
( sp_dec_key_gen_q[i] ), // Sparsified
208 .dec_key_gen_d_o
( sp_dec_key_gen_d[i] ), // Sparsified
209 .prng_reseed_q_i ( prng_reseed_q ),
210 .prng_reseed_d_o
( mr_prng_reseed_d[i] ), // AND-combine
211 .key_clear_q_i ( key_clear_q ),
212 .key_clear_d_o
( mr_key_clear_d[i] ), // AND-combine
213 .data_out_clear_q_i ( data_out_clear_q ),
214 .data_out_clear_d_o
( mr_data_out_clear_d[i] ) // AND-combine
215 );
216 end else begin : gen_fsm_n
217   aes_cipher_control_fsm_n #(
218     .SecMasking ( SecMasking ),
219     .SecSBoxImpl ( SecSBoxImpl ),
220     ) u_aes_cipher_control_fsm_i (
221     .clk_i ( clk_i ),
222     .rst_ni ( rst_ni ),
223
224     .in_valid_ni ( sp_in_val-
id[i] ), // Sparsified
225     .in_ready_no
( sp_in_ready[i] ), // Sparsified
226
227     .out_valid_no ( sp_out_val-
id[i] ), // Sparsified
228     .out_ready_ni
( sp_out_ready[i] ), // Sparsified
229
230     .cfg_valid_i ( cfg_valid_i ),
231     .op_i ( op_i ),
232     .key_len_i ( key_len_i ),
233     .crypt_ni
( sp_crypt[i] ), // Sparsified
234     .dec_key_gen_ni
( sp_dec_key_gen[i] ), // Sparsified
235     .prng_reseed_i ( prng_reseed_i ),
236     .key_clear_i ( key_clear_i ),
237     .data_out_clear_i ( data_out_clear_i ),
238     .mux_sel_err_i ( mux_sel_err ),
239     .sp_enc_err_i ( sp_enc_err ),
240     .rnd_ctr_err_i ( rnd_ctr_err ),
241     .op_err_i ( op_err_i ),
242     .alert_fatal_i ( alert_fatal_i ),
243     .alert_o
( mr_alert[i] ), // OR-combine
244
245     .prng_update_o
( mr_prng_update[i] ), // OR-combine
246     .prng_reseed_req_o
( mr_prng_reseed_req[i] ), // OR-combine
247     .prng_reseed_ack_i ( prng_reseed_ack_i ),
248
249     .state_sel_o
( mr_state_sel[i] ), // OR-combine
250     .state_we_no
( sp_state_we[i] ), // Sparsified
251     .sub_bytes_en_no
( sp_sub_bytes_en[i] ), // Sparsified
252     .sub_bytes_out_req_ni
( sp_sub_bytes_out_req[i] ), // Sparsified
253     .sub_bytes_out_ack_no
( sp_sub_bytes_out_ack[i] ), // Sparsified
254     .add_rk_sel_o
( mr_add_rk_sel[i] ), // OR-combine
255
256     .key_full_sel_o
( mr_key_full_sel[i] ), // OR-combine
257     .key_full_we_no
( sp_key_full_we[i] ), // Sparsified
258     .key_dec_sel_o
( mr_key_dec_sel[i] ), // OR-combine
259     .key_dec_we_no
( sp_key_dec_we[i] ), // Sparsified
260     .key_expand_en_no ( sp_key-ex-
pand_en[i] ), // Sparsified
261     .key_expand_out_req_ni ( sp_key-ex-
pand_out_req[i] ), // Sparsified
262     .key_expand_out_ack_no ( sp_key-ex-
pand_out_ack[i] ), // Sparsified
263     .key_expand_clear_o ( mr_key-ex-
pand_clear[i] ), // OR-combine
264     .rnd_ctr_o
( mr_rnd_ctr[i] ), // OR-combine
265     .key_words_sel_o
( mr_key_words_sel[i] ), // OR-combine
266     .round_key_sel_o
( mr_round_key_sel[i] ), // OR-combine
267
268     .crypt_q_ni

```

```

( sp_crypt_q[i] ), // Sparsified
269 .crypt_d_no
( sp_crypt_d[i] ), // Sparsified
270 .dec_key_gen_q_ni
( sp_dec_key_gen_q[i] ), // Sparsified
271 .dec_key_gen_d_no
( sp_dec_key_gen_d[i] ), // Sparsified
272 .prng_reseed_q_i ( prng_reseed_q ),
273 .prng_reseed_d_o
( mr_prng_reseed_d[i] ), // AND-combine
274 .key_clear_q_i ( key_clear_q ),
275 .key_clear_d_o
( mr_key_clear_d[i] ), // AND-combine
276 .data_out_clear_q_i ( data_out_clear_q ),
277 .data_out_clear_d_o
( mr_data_out_clear_d[i] ) // AND-combine
278 );
279 end
280 end
281
282 // Convert sparsified outputs to sp2v_e type.
283 assign in_ready_o = sp2v_e'(sp_in_ready);
284 assign out_valid_o = sp2v_e'(sp_out_valid);
285 assign state_we_o = sp2v_e'(sp_state_we);
286 assign sub_bytes_en_o = sp2v_e'(sp_sub_bytes_en);
287 assign sub_bytes_out_ack_o = sp2v_e'(sp_sub_bytes_out_ack);
288 assign key_full_we_o = sp2v_e'(sp_key_full_we);
289 assign key_dec_we_o = sp2v_e'(sp_key_dec_we);
290 assign key_expand_en_o = sp2v_e'(sp_key_expand_en);
291 assign key_expand_out_ack_o = sp2v_e'(sp_key-ex-
pand_out_ack);
292 assign crypt_d = sp2v_e'(sp_crypt_d);
293 assign dec_key_gen_d = sp2v_e'(sp_dec_key_gen_d);
294
295 // Combine single-bit FSM outputs.
296 // OR: One bit is sufficient to drive the corresponding out-
put bit high.
297 assign alert_o = |mr_alert;
298 assign prng_update_o = |mr_prng_update;
299 assign prng_reseed_req_o = |mr_prng_reseed_req;
300 assign key_expand_clear_o = |mr_key_expand_clear;
301 // AND: Only if all bits are high, the corresponding status
is signaled which will lead to
302 // the clearing of these trigger bits.
303 assign prng_reseed_d = &mr_prng_reseed_d;
304 assign key_clear_d = &mr_key_clear_d;
305 assign data_out_clear_d = &mr_data_out_clear_d;
306
307 // Combine multi-bit, sparse FSM outputs. We simply OR them
together. If the FSMs don't provide
308 // the same outputs, two cases are possible:
309 // - An invalid encoding results: A downstream checker will
fire, see mux_sel_err_i.
310 // - A valid encoding results: The outputs are compared be-
low to cover this case, see mr_err;
311 always_comb begin : combine_sparse_signals
312   state_sel_o = state_sel_e'({StateSelWidth{1'b0}});
313   add_rk_sel_o = add_rk_sel_e'({AddRKSelWidth{1'b0}});
314   key_full_sel_o =
key_full_sel_e'({KeyFullSelWidth{1'b0}});
315   key_dec_sel_o = key_dec_sel_e'({KeyDecSelWidth{1'b0}});
316   key_words_sel_o =
key_words_sel_e'({KeyWordsSelWidth{1'b0}});
317   round_key_sel_o =
round_key_sel_e'({RoundKeySelWidth{1'b0}});
318   mr_err = 1'b0;
319
320   for (int i = 0; i < Sp2VWidth; i++) begin
321     state_sel_o = state_sel_e'({state_sel_o} |
{mr_state_sel[i]});
322     add_rk_sel_o = add_rk_sel_e'({add_rk_sel_o} |
{mr_add_rk_sel[i]});
323     key_full_sel_o = key_full_sel_e'({key_full_sel_o} |
{mr_key_full_sel[i]});
324     key_dec_sel_o = key_dec_sel_e'({key_dec_sel_o} |
{mr_key_dec_sel[i]});
325     key_words_sel_o = key_words_sel_e'({key_words_sel_o} |
{mr_key_words_sel[i]});
326     round_key_sel_o = round_key_sel_e'({round_key_sel_o} |
{mr_round_key_sel[i]});
327   end
328
329   for (int i = 0; i < Sp2VWidth; i++) begin
330     if (state_sel_o != mr_state_sel[i] ||
331         add_rk_sel_o != mr_add_rk_sel[i] ||
332         key_full_sel_o != mr_key_full_sel[i] ||
333         key_dec_sel_o != mr_key_dec_sel[i] ||
334         key_words_sel_o != mr_key_words_sel[i] ||
335         round_key_sel_o != mr_round_key_sel[i]) begin
336       mr_err = 1'b1;
337     end
338   end
339 end
340
341 // Collect errors in mux selector signals.
342 assign mux_sel_err = mux_sel_err_i | mr_err;
343
344 // Combine counter signals. We simply OR them together. If
the FSMs don't provide the same
345 // outputs, rnd_ctr_err will be set.
346 always_comb begin : combine_counter_signals

```



```

347   rnd_ctr      = '0;
348   rnd_ctr_err  = 1'b0;
349   for (int i = 0; i < Sp2VWidth; i++) begin
350     rnd_ctr[i] = mr_rnd_ctr[i];
351   end
352
353   for (int i = 0; i < Sp2VWidth; i++) begin
354     if (rnd_ctr != mr_rnd_ctr[i]) begin
355       rnd_ctr_err = 1'b1;
356     end
357   end
358 end
359
360 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
361   if (!rst_ni) begin
362     prng_reseed_q <= 1'b0;
363     key_clear_q   <= 1'b0;
364     data_out_clear_q <= 1'b0;
365   end else begin
366     prng_reseed_q <= prng_reseed_d;
367     key_clear_q   <= key_clear_d;
368     data_out_clear_q <= data_out_clear_d;
369   end
370 end
371
372 // Use separate signal for key expand operation, forward
round.
373   assign key_expand_op_o = (dec_key_gen_d == SP2V_HIGH ||
374     dec_key_gen_q == SP2V_HIGH) ||
CiphOpFwdOnly ? CIPH_FWD : op_i;
375   assign key_expand_round_o = rnd_ctr;
376
377   // Let the main controller know what we are doing.
378   assign crypt_o = crypt_q;
379   assign dec_key_gen_o = dec_key_gen_q;
380   assign prng_reseed_o = prng_reseed_q;
381   assign key_clear_o = key_clear_q;
382   assign data_out_clear_o = data_out_clear_q;
383
384   ///////////////////////////////////////////////////
385   // Sparsely Encoded Signals //
386   ///////////////////////////////////////////////////
387   // SEC_CM: CTRL.SPARSE
388   // We use sparse encodings for various critical signals and
must ensure that:
391   // 1. The synthesis tool doesn't optimize away the sparse
encoding.
392   // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
393   // if a sparse signal takes on an invalid value.
394   // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
395   // This is achieved by driving the control FSM into the
terminal error state whenever any
396   // sparsely encoded signal becomes invalid.
397   //
398   // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts
399   // the out_valid_o signal to prevent any data from being re-
leased.
400
401   // The following primitives are used to place a size-only
constraint on the
402   // flops in order to prevent optimizations on these status
signals.
403   logic [Sp2VWidth-1:0] crypt_q_raw;
404   prim_flop #(
405     .Width      ( Sp2VWidth ),
406     .ResetValue ( Sp2VWidth'(SP2V_LOW) )
407   ) u_crypt_regs (
408     .clk_i ( clk_i ),
409     .rst_ni ( rst_ni ),
410     .d_i   ( crypt_d ),
411     .q_o   ( crypt_q_raw )
412   );
413
414   logic [Sp2VWidth-1:0] dec_key_gen_q_raw;
415   prim_flop #(
416     .Width      ( Sp2VWidth ),
417     .ResetValue ( Sp2VWidth'(SP2V_LOW) )
418   ) u_dec_key_gen_regs (
419     .clk_i ( clk_i ),
420     .rst_ni ( rst_ni ),
421     .d_i   ( dec_key_gen_d ),
422     .q_o   ( dec_key_gen_q_raw )
423   );
424
425   // We use vectors of sparsely encoded signals to reduce code
duplication.
426   localparam int unsigned NumSp2VSig = 8;
427   sp2v_e [NumSp2VSig-1:0] sp2v_sig;
428   sp2v_e [NumSp2VSig-1:0] sp2v_sig_chk;
429   logic [NumSp2VSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
430   logic [NumSp2VSig-1:0] sp2v_sig_err;
431
432   assign sp2v_sig[0] = in_valid_i;
433   assign sp2v_sig[1] = out_ready_i;
434   assign sp2v_sig[2] = crypt_i;

```

```

435   assign sp2v_sig[3] = dec_key_gen_i;
436   assign sp2v_sig[4] = sp2v_e'(crypt_q_raw);
437   assign sp2v_sig[5] = sp2v_e'(dec_key_gen_q_raw);
438   assign sp2v_sig[6] = sub_bytes_out_req_i;
439   assign sp2v_sig[7] = key_expand_out_req_i;
440
441   // All signals inside sp2v_sig except for sub_bytes/key_ex-
pand_out_req_i are driven and consumed
442   // by multi-rail FSMs.
443   localparam bit [NumSp2VSig-1:0] Sp2VEnSecBuf = 8'b1100_0000;
444
445   // Individually check sparsely encoded signals.
446   for (genvar i = 0; i < NumSp2VSig; i++) begin :
gen_sel_buf_chk
447     aes_sel_buf_chk #(
448       .Num      ( Sp2VNum ),
449       .Width     ( Sp2VWidth ),
450       .EnSecBuf ( Sp2VEnSecBuf[i] )
451     ) u_aes_sp2v_sig_buf_chk_i (
452       .clk_i ( clk_i ),
453       .rst_ni ( rst_ni ),
454       .sel_i ( sp2v_sig[i] ),
455       .sel_o ( sp2v_sig_chk_raw[i] ),
456       .err_o ( sp2v_sig_err[i] )
457     );
458     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
459   end
460
461   assign in_valid      = sp2v_sig_chk[0];
462   assign out_ready     = sp2v_sig_chk[1];
463   assign crypt         = sp2v_sig_chk[2];
464   assign dec_key_gen   = sp2v_sig_chk[3];
465   assign crypt_q       = sp2v_sig_chk[4];
466   assign dec_key_gen_q = sp2v_sig_chk[5];
467   assign sub_bytes_out_req = sp2v_sig_chk[6];
468   assign key_expand_out_req = sp2v_sig_chk[7];
469
470   // Collect encoding errors.
471   // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
472   // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
473   assign sp_enc_err = |sp2v_sig_err | sp_enc_err_i;
474
475   ///////////////////////////////////////////////////
476   // Assertions //
477   ///////////////////////////////////////////////////
478   // Selectors must be known/valid
479   `ASSERT(AesCiphOpValid, cfg_valid_i |> op_i inside {
480     CIPH_FWD,
481     CIPH_INV
482   })
483
484
485 endmodule

```

3. lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm.sv (515 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM.
8
9 `include "prim_assert.sv"
10
11 module aes_cipher_control_fsm import aes_pkg::*;
12 #(
13   parameter bit          SecMasking = 0,
14   parameter sbbox_impl_e SecSBoxImpl = SBoxImplDom
15 ) (
16   input logic             clk_i,
17   input logic             rst_ni,
18
19   // Input handshake signals
20   input logic             in_valid_i,           // Sparsify
using multi-rail.
21   output logic            in_ready_o,          // Sparsify
using multi-rail.
22
23   // Output handshake signals
24   output logic            out_valid_o,          // Sparsify
using multi-rail.
25   input logic             out_ready_i,          // Sparsify
using multi-rail.
26
27   // Control and sync signals
28   input logic             cfg_valid_i,          // Used for
SVAs only.
29   input ciph_op_e         op_i,
30   input key_len_e         key_len_i,
31   input logic             crypt_i,             // Sparsify
using multi-rail.
32   input logic             dec_key_gen_i,        // Sparsify
using multi-rail.

```

```

33 input logic prng_reseed_i,
34 input logic key_clear_i,
35 input logic data_out_clear_i,
36 input logic mux_sel_err_i,
37 input logic sp_enc_err_i,
38 input logic rnd_ctr_err_i,
39 input logic op_err_i,
40 input logic alert_fatal_i,
41 output logic alert_o,
42
43 // Control signals for masking PRNG
44 output logic prng_update_o,
45 output logic prng_reseed_req_o,
46 input logic prng_reseed_ack_i,
47
48 // Control and sync signals for cipher data path
49 output state_sel_e state_sel_o,
50 output logic state_we_o, // Sparsify
using multi-rail.
51 output logic sub_bytes_en_o, // Sparsify
using multi-rail.
52 input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
53 output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
54 output add_rk_sel_e add_rk_sel_o,
55
56 // Control and sync signals for key expand data path
57 output key_full_sel_e key_full_sel_o,
58 output logic key_full_we_o, // Sparsify
using multi-rail.
59 output key_dec_sel_e key_dec_sel_o,
60 output logic key_dec_we_o, // Sparsify
using multi-rail.
61 output logic key_expand_en_o, // Sparsify
using multi-rail.
62 input logic key_expand_out_req_i, // Sparsify
using multi-rail.
63 output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
64 output logic key_expand_clear_o,
65 output key_words_sel_e rnd_ctr_o,
66 output key_words_sel_e key_words_sel_o,
67 output round_key_sel_e round_key_sel_o,
68
69 // Register signals
70 input logic crypt_q_i, // Sparsify
using multi-rail.
71 output logic crypt_d_o, // Sparsify
using multi-rail.
72 input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
73 output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
74 input logic prng_reseed_q_i,
75 output logic prng_reseed_d_o,
76 input logic key_clear_q_i,
77 output logic key_clear_d_o,
78 input logic data_out_clear_q_i,
79 output logic data_out_clear_d_o
80 );
81
82 // cfg_valid_i is used for SVAs only.
83 logic unused_cfg_valid;
84 assign unused_cfg_valid = cfg_valid_i;
85
86 // Tie off unused inputs.
87 if (!SecMasking) begin : gen_unused_prng_reseed
88 logic unused_prng_reseed;
89 assign unused_prng_reseed = prng_reseed_i;
90 end
91
92 // Signals
93 aes_cipher_ctrl_e aes_cipher_ctrl_ns, aes_cipher_ctrl_cs;
94 logic advance;
95 logic [2:0] cyc_ctr_d, cyc_ctr_q;
96 logic cyc_ctr_expr;
97 logic prng_reseed_done_d, prng_reseed_done_q;
98 logic [3:0] rnd_ctr_d, rnd_ctr_q;
99 logic [3:0] num_rounds_d, num_rounds_q;
100 logic [3:0] num_rounds_regular;
101
102 // Use separate signal for number of regular rounds.
103 assign num_rounds_regular = num_rounds_q - 4'd1;
104
105 // FSM
106 always_comb begin : aes_cipher_ctrl_fsm
107
108 // Handshake signals
109 in_ready_o = 1'b0;
110 out_valid_o = 1'b0;
111
112 // Masking PRNG signals
113 prng_update_o = 1'b0;
114 prng_reseed_req_o = 1'b0;
115
116 // Cipher data path
117 state_sel_o = STATE_ROUND;
118 state_we_o = 1'b0;
119 add_rk_sel_o = ADD_RK_ROUND;
120 sub_bytes_en_o = 1'b0;
121
122 sub_bytes_out_ack_o = 1'b0;
123
124 // Key expand data path
125 key_full_sel_o = KEY_FULL_ROUND;
126 key_full_we_o = 1'b0;
127 key_dec_sel_o = KEY_DEC_EXPAND;
128 key_dec_we_o = 1'b0;
129 key_expand_en_o = 1'b0;
130 key_expand_out_ack_o = 1'b0;
131 key_expand_clear_o = 1'b0;
132 key_words_sel_o = KEY_WORDS_ZERO;
133 round_key_sel_o = ROUND_KEY_DIRECT;
134
135 // FSM
136 aes_cipher_ctrl_ns = aes_cipher_ctrl_cs;
137 num_rounds_d = num_rounds_q;
138 rnd_ctr_d = rnd_ctr_q;
139 crypt_d_o = crypt_q_i;
140 dec_key_gen_d_o = dec_key_gen_q_i;
141 prng_reseed_d_o = prng_reseed_q_i;
142 key_clear_d_o = key_clear_q_i;
143 data_out_clear_d_o = data_out_clear_q_i;
144 prng_reseed_done_d = prng_reseed_done_q | prng_reseed_ack_i;
145 advance = 1'b0;
146 cyc_ctr_q + 3'd1 : 3'd0;
147
148 // Alert
149 alert_o = 1'b0;
150
151 unique case (aes_cipher_ctrl_cs)
152 CIPHER_CTRL_IDLE: begin
153 cyc_ctr_d = 3'd0;
154
155 // Signal that we are ready, wait for handshake.
156 in_ready_o = 1'b1;
157 if (in_valid_i) begin
158 if (SecMasking && prng_reseed_i && !dec_key_gen_i
&& !crypt_i) begin
159 // Reseed the masking PRNG without starting en-
160 // cryptation/decryption or generation of the
161 // start key for decryption.
162 prng_reseed_d_o = 1'b1;
163 prng_reseed_done_d = 1'b0;
164 aes_cipher_ctrl_ns = CIPHER_CTRL_PRNG_RESEED;
165
166 end else if (key_clear_i || data_out_clear_i) begin
167 // Clear internal key registers. The cipher core
168 // muxes are used to clear the data
169 // output registers.
170 key_clear_d_o = key_clear_i;
171 data_out_clear_d_o = data_out_clear_i;
172
173 // To clear the data output registers, we must
174 // first clear the state.
175 aes_cipher_ctrl_ns = data_out_clear_i ?
176 CIPHER_CTRL_CLEAR_S : CIPHER_CTRL_CLEAR_KD;
177
178 end else if (dec_key_gen_i || crypt_i) begin
179 // Start encryption/decryption or generation of
180 // start key for decryption.
181 crypt_d_o = ~dec_key_gen_i & crypt_i;
182 dec_key_gen_d_o = dec_key_gen_i;
183
184 // Latch whether we shall reseed the masking PRNG.
185 prng_reseed_d_o = SecMasking & prng_reseed_i;
186
187 // Load input data to state
188 state_sel_o = dec_key_gen_i ? STATE_CLEAR :
189 STATE_INIT;
190
191 state_we_o = 1'b1;
192
193 // Make the masking PRNG advance. The current
194 // pseudo-random data is used to mask the
195 // input data.
196 prng_update_o = SecMasking;
197
198 // Init key expand
199 key_expand_clear_o = 1'b1;
200
201 // Load full key
202 key_full_sel_o = dec_key_gen_i ?
203 KEY_FULL_ENC_INIT :
204 (op_i == CIPH_FWD) ?
205 KEY_FULL_ENC_INIT :
206 (op_i == CIPH_INV) ?
207 KEY_FULL_DEC_INIT :
208 KEY_FULL_ENC_INIT;
209
210 key_full_we_o = 1'b1;
211
212 // Load num_rounds, initialize round counter.
213 num_rounds_d = (key_len_i == AES_128) ? 4'd10 :
214 (key_len_i == AES_192) ? 4'd12 :
215 4'd14;
216
217 rnd_ctr_d = '0;
218 aes_cipher_ctrl_ns = CIPHER_CTRL_INIT;
219
220 end else begin

```



```

208      // Handshake without a valid command. We should
never get here. If we do (e.g. via a
209      // malicious glitch), error out immediately.
210      aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
211    end
212  end
213 end
214
215 CIPHER_CTRL_INIT: begin
216   // Initial round: just add key to state
217   add_rk_sel_o = ADD_RK_INIT;
218
219   // Select key words for initial add_round_key
220   key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
221   (key_len_i == AES_128) ?
KEY_WORDS_0123 :
222   (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
223   (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_2345 :
224   (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
225   (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_4567 : KEY_WORDS_ZERO;
226
227   // Clear masking PRNG reseed status.
228   prng_reseed_done_d = 1'b0;
229
230   // AES-256 has two round keys available right from be-
ginning. Pseudo-random data is
231   // required by KeyExpand only.
232   if (key_len_i != AES_256) begin
233     // Advance in sync with KeyExpand. Based on the S-
Box implementation, it can take
234     // multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
235     // only in the first clock cycle and that PRD is
taken from the buffer stage updated
236     // based on key_full_we_o. The PRNG itself is up-
dated in every clock cycle to increase
237     // the noise.
238     advance = key_expand_out_req_i & cyc_ctr_ex-
pr;
239     prng_update_o = SecMasking;
240     key_expand_en_o = 1'b1;
241     if (advance) begin
242       key_expand_out_ack_o = 1'b1;
243       state_we_o = ~dec_key_gen_q_i;
244       key_full_we_o = 1'b1;
245       rnd_ctr_d = rnd_ctr_q + 4'b0001;
246       cyc_ctr_d = 3'd0;
247       aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
248     end
249   end else begin
250     prng_update_o = SecMasking;
251     state_we_o = ~dec_key_gen_q_i;
252     rnd_ctr_d = rnd_ctr_q + 4'b0001;
253     cyc_ctr_d = 3'd0;
254     aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
255   end
256 end
257
258 CIPHER_CTRL_ROUND: begin
259   // Normal rounds
260
261   // Select key words for add_round_key
262   key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
263   (key_len_i == AES_128) ?
KEY_WORDS_0123 :
264   (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
265   (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
266   (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
267   (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
268
269   // Keep requesting PRNG reseeding until it is acknow-
ledged.
270   prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
271
272   // Select round key: direct or mixed (equivalent in-
verse cipher)
273   round_key_sel_o = (op_i == CIPH_FWD) ?
ROUND_KEY_DIRECT :
274   (op_i == CIPH_INV) ?
ROUND_KEY_MIXED : ROUND_KEY_DIRECT;
275
276   // Advance in sync with SubBytes and KeyExpand. Based
on the S-Box implementation, both can
277   // take multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
278   // only in the first clock cycle and that PRD is taken
from the buffer stages updated
279   // with state_we_o / based on key_full_we_o. The PRNG

```

```

itself is updated in every clock
280   // cycle to increase the noise.
281   advance = key_expand_out_req_i & cyc_ctr_expr &
(dec_key_gen_q_i | sub_bytes_out_req_i);
282   prng_update_o = SecMasking;
283   sub_bytes_en_o = ~dec_key_gen_q_i;
284   key_expand_en_o = 1'b1;
285
286   if (advance) begin
287     sub_bytes_out_ack_o = ~dec_key_gen_q_i;
288     key_expand_out_ack_o = 1'b1;
289
290     state_we_o = ~dec_key_gen_q_i;
291     key_full_we_o = 1'b1;
292
293     // Update round
294     rnd_ctr_d = rnd_ctr_q + 4'b0001;
295     cyc_ctr_d = 3'd0;
296
297     // Are we doing the last regular round?
298     if (rnd_ctr_q >= num_rounds_regular) begin
299       aes_cipher_ctrl_ns = CIPHER_CTRL_FINISH;
300
301       if (dec_key_gen_q_i) begin
302         // Write decryption key.
303         key_dec_we_o = 1'b1;
304
305         // Indicate that we are done, try to perform the
handshake. But we don't wait here.
306         // If we don't get the handshake now, we will
wait in the finish state. When using
307         // masking, we only finish if the masking PRNG
has been reseeded.
308         out_valid_o = SecMasking ? (prng_reseed_q_i ?
prng_reseed_done_q : 1'b1) : 1'b1;
309         if (out_valid_o && out_ready_i) begin
310           // Go to idle state directly.
311           dec_key_gen_d_o = 1'b0;
312           prng_reseed_d_o = 1'b0;
313           aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
314         end
315       end
316       end // rnd_ctr_q
317       end // SubBytes/KeyExpand REQ/ACK
318     end
319
320     CIPHER_CTRL_FINISH: begin
321       // Final round
322
323       // Select key words for add_round_key
324       key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
325       (key_len_i == AES_128) ?
KEY_WORDS_0123 :
326       (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
327       (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
328       (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
329       (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
330
331       // Skip mix_columns
332       add_rk_sel_o = ADD_RK_FINAL;
333
334       // Keep requesting PRNG reseeding until it is acknow-
ledged.
335       prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
336
337       // SEC_CM: DATA_REG.KEY.SCA
338       // Once we're done, we won't need the state anymore.
We actually clear it when progressing
339       // to the next state.
340       state_sel_o = STATE_CLEAR;
341
342       // SEC_CM: DATA_REG.LOCAL_ESC
343       // Advance in sync with SubBytes. Based on the S-Box
implementation, it can take multiple
344       // cycles to finish. Only indicate that we are done
if:
345       // - we have valid output (SubBytes finished),
346       // - the masking PRNG has been reseeded (if masking is
used),
347       // - all mux selector signals are valid (don't release
data in case of errors), and
348       // - all sparsely encoded signals are valid (don't re-
lease data in case of errors).
349       // Perform both handshakes simultaneously.
350       advance = (sub_bytes_out_req_i & cyc_ctr_expr)
| dec_key_gen_q_i;
351       sub_bytes_en_o = ~dec_key_gen_q_i;
352       out_valid_o = (mux_sel_err_i || sp_enc_err_i ||
op_err_i) ? 1'b0 :
353       SecMasking ? (prng_reseed_q_i ? prng_reseed_done_q &
advance : advance) : advance;

```

```

354
355 // Stop updating the cycle counter once we have valid
output.
356   cyc_ctr_d =
357     (SecSBoxImpl == SBoxImplDom) ? (!advance ?
cyc_ctr_q + 3'd1 : cyc_ctr_q) : 3'd0;
358
359 // The DOM S-Boxes consume fresh PRD only in the first
clock cycle and that PRD is taken
360 // from the buffer stages updated with state_we_o /
based on key_full_we_o. The PRNG itself
361 // is updated in every but the last processing clock
cycle to increase the noise. Once the
362 // processing is all done (e.g. if we're just waiting
for the PRNG reseeding to finish or
363 // if we're waiting for out_ready_i), the PRNG is no
longer updated to save power. In the
364 // very last clock cycle, we update the PRNG again to
get ready for the next block.
365   prng_update_o =
366     ((SecSBoxImpl == SBoxImplDom) ? !advance : 1'b0) |
(out_valid_o & out_ready_i);
367
368   if (out_valid_o && out_ready_i) begin
369     sub_bytes_out_ack_o = ~dec_key_gen_q_i;
370
371     // Clear the state.
372     state_we_o = 1'b1;
373     crypt_d_o = 1'b0;
374     cyc_ctr_d = 3'd0;
375     // If we were generating the decryption key and
didn't get the handshake in the last
376     // regular round, we should clear dec_key_gen now.
377     dec_key_gen_d_o = 1'b0;
378     prng_reseed_d_o = 1'b0;
379     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
380   end
381 end
382
383 CIPHER_CTRL_PRNG_RESEED: begin
384 // Keep requesting PRNG reseeding until it is acknow-
ledged.
385   prng_reseed_req_o = prng_reseed_q_i & ~prng_re-
seed_done_q;
386
387 // Don't update the cycle counter as we don't need it.
388   cyc_ctr_d = 3'd0;
389
390 // Once we're done, wait for handshake.
391   out_valid_o = prng_reseed_done_q;
392   if (out_valid_o && out_ready_i) begin
393     prng_reseed_d_o = 1'b0;
394     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
395   end
396 end
397
398 CIPHER_CTRL_CLEAR_S: begin
399 // Clear the state with pseudo-random data.
400   state_we_o = 1'b1;
401   state_sel_o = STATE_CLEAR;
402   aes_cipher_ctrl_ns = CIPHER_CTRL_CLEAR_KD;
403 end
404
405 CIPHER_CTRL_CLEAR_KD: begin
406 // Clear internal key registers and/or external data
output registers.
407   if (key_clear_q_i) begin
408     key_full_sel_o = KEY_FULL_CLEAR;
409     key_full_we_o = 1'b1;
410     key_dec_sel_o = KEY_DEC_CLEAR;
411     key_dec_we_o = 1'b1;
412   end
413   if (data_out_clear_q_i) begin
414 // Forward the state (previously cleared with
pseudo-random data).
415     // SEC_CM: DATA_REG.SEC_WIPE
416     add_rk_sel_o = ADD_RK_INIT;
417     key_words_sel_o = KEY_WORDS_ZERO;
418     round_key_sel_o = ROUND_KEY_DIRECT;
419   end
420 // Indicate that we are done, wait for handshake.
421   out_valid_o = 1'b1;
422   if (out_ready_i) begin
423     key_clear_d_o = 1'b0;
424     data_out_clear_d_o = 1'b0;
425     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
426   end
427 end
428
429 CIPHER_CTRL_ERROR: begin
430 // SEC_CM: CIPHER.FSM.LOCAL_ESC
431 // Terminal error state
432   alert_o = 1'b1;
433 end
434
435 // We should never get here. If we do (e.g. via a mali-
cious glitch), error out immediately.
436   default: begin
437     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;

```

```

438   alert_o = 1'b1;
439 end
440 endcase
441
442 // Unconditionally jump into the terminal error state in
case a mux selector or a sparsely
443 // encoded signal becomes invalid, in case we have detec-
ted a fault in the round counter,
444 // or if a fatal alert has been triggered.
445   if (mux_sel_err_i || sp_enc_err_i || rnd_ctr_err_i ||
op_err_i || alert_fatal_i) begin
446     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
447   end
448 end
449
450 // SEC_CM: CIPHER.FSM.SPARSE
451 `PRIM_FLOP_SPARSE_FSM(u_state_regs, aes_cipher_ctrl_ns,
aes_cipher_ctrl_cs, aes_cipher_ctrl_e, CIPHER_CTRL_IDLE)
452
453 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
454   if (!rst_ni) begin
455     prng_reseed_done_q <= 1'b0;
456     rnd_ctr_q <= '0;
457     num_rounds_q <= '0;
458   end else begin
459     prng_reseed_done_q <= prng_reseed_done_d;
460     rnd_ctr_q <= rnd_ctr_d;
461     num_rounds_q <= num_rounds_d;
462   end
463 end
464
465 assign rnd_ctr_o = rnd_ctr_q;
466
467   if (SecSBoxImpl == SBoxImplDom) begin : gen_cyc_ctr
468     always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_cyc_ctr
469       if (!rst_ni) begin
470         cyc_ctr_q <= 3'd0;
471       end else begin
472         cyc_ctr_q <= cyc_ctr_d;
473       end
474     end
475     assign cyc_ctr_expr = cyc_ctr_q >= 3'd4;
476   end else begin : gen_no_cyc_ctr
477     logic [2:0] unused_cyc_ctr;
478     assign cyc_ctr_q = cyc_ctr_d;
479     assign unused_cyc_ctr = cyc_ctr_q;
480     assign cyc_ctr_expr = 1'b1;
481   end
482 end
483
484 // Assertions //
485 // Create a lint error to reduce the risk of accidentally
disabling the masking.
486 `AS-
487   SERT_STATIC_LINT_ERROR(AesCipherControlFsmSecMaskingNonDefault,
SecMasking == 1)
488 // Create a lint error to reduce the risk of accidentally
using a less secure SBox
implementation.
489 `AS-
490   SERT_STATIC_LINT_ERROR(AesCipherControlFsmSecSBoxImplNonDefault,
SecSBoxImpl == SBoxImplDom)
491
492 // Selectors must be known/valid
493 `ASSERT(AesCiphOpValid, cfg_valid_i |> op_i inside {
494   CIPH_FWD,
495   CIPH_INV
496 })
497 `ASSERT(AesKeyLenValid, cfg_valid_i |> key_len_i inside {
498   AES_128,
499   AES_192,
500   AES_256
501 })
502 `ASSERT(AesCipherControlStateValid, !alert_o |>
aes_cipher_ctrl_cs inside {
503   CIPHER_CTRL_IDLE,
504   CIPHER_CTRL_INIT,
505   CIPHER_CTRL_ROUND,
506   CIPHER_CTRL_FINISH,
507   CIPHER_CTRL_PRNG_RESEED,
508   CIPHER_CTRL_CLEAR_S,
509   CIPHER_CTRL_CLEAR_KD
510 })
511 endmodule

```

4. lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm_n.sv (400 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM

```

```

6 //
7 // This module contains the AES cipher core control FSM oper-
ating on and producing the negated
8 // values of important control signals. This is achieved by:
9 // - instantiating the regular AES cipher core control FSM op-
erating on and producing the positive
10 // values of these signals, and
11 // - inverting these signals between the regular FSM and the
prim_buf synthesis barriers.
12 // Synthesis tools will then push the inverters into the actu-
al FSM.
13
14 module aes_cipher_control_fsm_n import aes_pkg::*;
15 #(
16     parameter bit SecMasking = 0,
17     parameter sbbox_impl_e SecSBBoxImpl = SBBoxImplDom
18 ) (
19     input logic clk_i,
20     input logic rst_ni,
21
22     // Input handshake signals
23     input logic in_valid_ni, // Sparsify
using multi-rail.
24     output logic in_ready_no, // Sparsify
using multi-rail.
25
26     // Output handshake signals
27     output logic out_valid_no, // Sparsify
using multi-rail.
28     input logic out_ready_ni, // Sparsify
using multi-rail.
29
30     // Control and sync signals
31     input logic cfg_valid_i, // Used for
SVAs only.
32     input ciph_op_e op_i,
33     input key_len_e key_len_i,
34     input logic crypt_ni, // Sparsify
using multi-rail.
35     input logic dec_key_gen_ni, // Sparsify
using multi-rail.
36     input logic prng_reseed_i,
37     input logic key_clear_i,
38     input logic data_out_clear_i,
39     input logic mux_sel_err_i,
40     input logic sp_enc_err_i,
41     input logic rnd_ctr_err_i,
42     input logic op_err_i,
43     input logic alert_fatal_i,
44     output logic alert_o,
45
46     // Control signals for masking PRNG
47     output logic prng_update_o,
48     output logic prng_reseed_req_o,
49     input logic prng_reseed_ack_i,
50
51     // Control and sync signals for cipher data path
52     output state_sel_e state_sel_o,
53     output logic state_we_no, // Sparsify
using multi-rail.
54     output logic sub_bytes_en_no, // Sparsify
using multi-rail.
55     input logic sub_bytes_out_req_ni, // Sparsify
using multi-rail.
56     output logic sub_bytes_out_ack_no, // Sparsify
using multi-rail.
57     output add_rk_sel_e add_rk_sel_o,
58
59     // Control and sync signals for key expand data path
60     output key_full_sel_e key_full_sel_o,
61     output logic key_full_we_no, // Sparsify
using multi-rail.
62     output key_dec_sel_e key_dec_sel_o,
63     output logic key_dec_we_no, // Sparsify
using multi-rail.
64     output logic key_expand_en_no, // Sparsify
using multi-rail.
65     input logic key_expand_out_req_ni, // Sparsify
using multi-rail.
66     output logic key_expand_out_ack_no, // Sparsify
using multi-rail.
67     output logic key_expand_clear_o,
68     output logic [3:0] rnd_ctr_o,
69     output key_words_sel_e key_words_sel_o,
70     output round_key_sel_e round_key_sel_o,
71
72     // Register signals
73     input logic crypt_q_ni, // Sparsify
using multi-rail.
74     output logic crypt_d_no, // Sparsify
using multi-rail.
75     input logic dec_key_gen_q_ni, // Sparsify
using multi-rail.
76     output logic dec_key_gen_d_no, // Sparsify
using multi-rail.
77     input logic prng_reseed_q_i,
78     output logic prng_reseed_d_o,
79     input logic key_clear_q_i,
80     output logic key_clear_d_o,
81     input logic data_out_clear_q_i,
82     output logic data_out_clear_d_o

```

```

83 );
84
85 // Input Buffering //
86 // Input Buffering //
87 // Input Buffering //
88
89 localparam int NumInBufBits = $bits({
90     in_valid_ni,
91     out_ready_ni,
92     cfg_valid_i,
93     op_i,
94     key_len_i,
95     crypt_ni,
96     dec_key_gen_ni,
97     prng_reseed_i,
98     key_clear_i,
99     data_out_clear_i,
100     mux_sel_err_i,
101     sp_enc_err_i,
102     rnd_ctr_err_i,
103     op_err_i,
104     alert_fatal_i,
105     prng_reseed_ack_i,
106     sub_bytes_out_req_ni,
107     key_expand_out_req_ni,
108     crypt_q_ni,
109     dec_key_gen_q_ni,
110     prng_reseed_q_i,
111     key_clear_q_i,
112     data_out_clear_q_i
113 });
114
115 logic [NumInBufBits-1:0] in, in_buf;
116
117 assign in = {
118     in_valid_ni,
119     out_ready_ni,
120     cfg_valid_i,
121     op_i,
122     key_len_i,
123     crypt_ni,
124     dec_key_gen_ni,
125     prng_reseed_i,
126     key_clear_i,
127     data_out_clear_i,
128     mux_sel_err_i,
129     sp_enc_err_i,
130     rnd_ctr_err_i,
131     op_err_i,
132     alert_fatal_i,
133     prng_reseed_ack_i,
134     sub_bytes_out_req_ni,
135     key_expand_out_req_ni,
136     crypt_q_ni,
137     dec_key_gen_q_ni,
138     prng_reseed_q_i,
139     key_clear_q_i,
140     data_out_clear_q_i
141 };
142
143 // This primitive is used to place a size-only constraint on
the
144 // buffers to act as a synthesis optimization barrier.
145 prim_buf #(
146     .Width(NumInBufBits)
147 ) u_prim_buf_in (
148     .in_i(in),
149     .out_o(in_buf)
150 );
151
152 logic in_valid_n;
153 logic out_ready_n;
154 logic cfg_valid;
155 ciph_op_e op;
156 logic [$bits(op)-1:0] op_raw;
157 key_len_e key_len;
158 logic crypt_n;
159 logic dec_key_gen_n;
160 logic prng_reseed;
161 logic key_clear;
162 logic data_out_clear;
163 logic mux_sel_err;
164 logic sp_enc_err;
165 logic rnd_ctr_err;
166 logic op_err;
167 logic alert_fatal;
168 logic prng_reseed_ack;
169 logic sub_bytes_out_req_n;
170 logic key_expand_out_req_n;
171 logic crypt_q_n;
172 logic dec_key_gen_q_n;
173 logic prng_reseed_q;
174 logic key_clear_q;
175 logic data_out_clear_q;
176
177 assign {in_valid_n,
178     out_ready_n,
179     cfg_valid,
180     op_raw,
181     key_len,

```

```

182     crypt_n,
183     dec_key_gen_n,
184     prng_reseed,
185     key_clear,
186     data_out_clear,
187     mux_sel_err,
188     sp_enc_err,
189     rnd_ctr_err,
190     op_err,
191     alert_fatal,
192     prng_reseed_ack,
193     sub_bytes_out_req_n,
194     key_expand_out_req_n,
195     crypt_q_n,
196     dec_key_gen_q_n,
197     prng_reseed_q,
198     key_clear_q,
199     data_out_clear_q} = in_buf;
200
201     assign op = ciph_op_e'(op_raw);
202
203     // Intermediate output signals
204     logic      in_ready;
205     logic      out_valid;
206     logic      alert;
207     logic      prng_update;
208     logic      prng_reseed_req;
209     state_sel_e state_sel;
210     logic      state_we;
211     logic      sub_bytes_en;
212     logic      sub_bytes_out_ack;
213     add_rk_sel_e add_rk_sel;
214     key_full_sel_e key_full_sel;
215     logic      key_full_we;
216     key_dec_sel_e key_dec_sel;
217     logic      key_dec_we;
218     logic      key_expand_en;
219     logic      key_expand_out_ack;
220     logic      key_expand_clear;
221     key_words_sel_e key_words_sel;
222     round_key_sel_e round_key_sel;
223     logic [3:0] rnd_ctr;
224     logic      crypt_d;
225     logic      dec_key_gen_d;
226     logic      prng_reseed_d;
227     logic      key_clear_d;
228     logic      data_out_clear_d;
229
230     ///////////////////////////////////////////////////
231     // Regular FSM //
232     ///////////////////////////////////////////////////
233
234     // The regular FSM operates on and produces the positive
235     // values of important control signals.
236     // Invert *_n input signals here to get the positive values
237     // for the regular FSM. To obtain the
238     // negated outputs, important output signals are inverted
239     // further below. Thanks to the prim_buf
240     // synthesis optimization barriers, tools will push the in-
241     // verters into the regular FSM.
242     aes_cipher_control_fsm #(
243         .SecMasking ( SecMasking ),
244         .SecSBoxImpl ( SecSBoxImpl )
245     ) u_aes_cipher_control_fsm (
246         .clk_i      ( clk_i      ),
247         .rst_i      ( rst_i      ),
248         .in_valid_i ( ~in_valid_n ), // In-
249         .in_ready_o ( in_ready   ), // In-
250         .out_valid_o ( out_valid  ), // In-
251         .out_ready_i ( ~out_ready_n ), // In-
252         .cfg_valid_i ( cfg_valid  ),
253         .op_i        ( op        ),
254         .key_len_i   ( key_len   ),
255         .crypt_i     ( ~crypt_n  ), // In-
256         .dec_key_gen_i ( ~dec_key_gen_n ), // In-
257         .prng_reseed_i ( prng_reseed ),
258         .key_clear_i ( key_clear ),
259         .data_out_clear_i ( data_out_clear ),
260         .mux_sel_err_i ( mux_sel_err ),
261         .sp_enc_err_i ( sp_enc_err ),
262         .rnd_ctr_err_i ( rnd_ctr_err ),
263         .op_err_i    ( op_err    ),
264         .alert_fatal_i ( alert_fatal ),
265         .alert_o     ( alert     ),
266         .prng_update_o ( prng_update ),
267         .prng_reseed_req_o ( prng_reseed_req ),
268         .prng_reseed_ack_i ( prng_reseed_ack ),
269         .state_sel_o ( state_sel ),
270         .state_we_o ( state_we ), // In-
271         vert below for negated output.

```

```

272         .sub_bytes_en_o ( sub_bytes_en ), // In-
273         vert below for negated output.
274         .sub_bytes_out_req_i ( ~sub_bytes_out_req_n ), // In-
275         .sub_bytes_out_ack_o ( sub_bytes_out_ack ), // In-
276         vert below for negated output.
277         .add_rk_sel_o ( add_rk_sel ),
278         .key_full_sel_o ( key_full_sel ),
279         .key_full_we_o ( key_full_we ), // In-
280         vert below for negated output.
281         .key_dec_sel_o ( key_dec_sel ),
282         .key_dec_we_o ( key_dec_we ), // In-
283         vert below for negated output.
284         .key_expand_en_o ( key_expand_en ), // In-
285         vert below for negated output.
286         .key_expand_out_req_i ( ~key_expand_out_req_n ), // In-
287         .key_expand_out_ack_o ( key_expand_out_ack ), // In-
288         vert below for negated output.
289         .key_expand_clear_o ( key_expand_clear ),
290         .rnd_ctr_o ( rnd_ctr ),
291         .key_words_sel_o ( key_words_sel ),
292         .round_key_sel_o ( round_key_sel ),
293         .crypt_q_i ( ~crypt_q_n ), // In-
294         .crypt_d_o ( crypt_d ), // In-
295         vert below for negated output.
296         .dec_key_gen_q_i ( ~dec_key_gen_q_n ), // In-
297         .dec_key_gen_d_o ( dec_key_gen_d ), // In-
298         vert below for negated output.
299         .key_clear_q_i ( key_clear_q ),
300         .key_clear_d_o ( key_clear_d ),
301         .prng_reseed_q_i ( prng_reseed_q ),
302         .prng_reseed_d_o ( prng_reseed_d ),
303         .data_out_clear_q_i ( data_out_clear_q ),
304         .data_out_clear_d_o ( data_out_clear_d )
305     );
306
307     ///////////////////////////////////////////////////
308     // Output Buffering //
309     ///////////////////////////////////////////////////
310
311     localparam int NumOutBufBits = $bits({
312         in_ready_no,
313         out_valid_no,
314         alert_o,
315         prng_update_o,
316         prng_reseed_req_o,
317         state_sel_o,
318         state_we_no,
319         sub_bytes_en_no,
320         sub_bytes_out_ack_no,
321         add_rk_sel_o,
322         key_full_sel_o,
323         key_full_we_no,
324         key_dec_sel_o,
325         key_dec_we_no,
326         key_expand_en_no,
327         key_expand_out_ack_no,
328         key_expand_clear_o,
329         rnd_ctr_o,
330         key_words_sel_o,
331         round_key_sel_o,
332         crypt_d_no,
333         dec_key_gen_d_no,
334         key_clear_d_o,
335         prng_reseed_d_o,
336         data_out_clear_d_o
337     });
338
339     logic [NumOutBufBits-1:0] out, out_buf;
340
341     // Important output control signals need to be inverted
342     // here. Synthesis tools will push the
343     // inverters back into the regular FSM.
344     assign out = {
345         ~in_ready,
346         ~out_valid,
347         alert,
348         prng_update,
349         prng_reseed_req,
350         state_sel,
351         ~state_we,
352         ~sub_bytes_en,
353         ~sub_bytes_out_ack,
354         add_rk_sel,
355         key_full_sel,
356         ~key_full_we,
357         key_dec_sel,
358         ~key_dec_we,
359         ~key_expand_en,
360         ~key_expand_out_ack,
361         key_expand_clear,
362         rnd_ctr,
363         key_words_sel,
364         round_key_sel,
365         ~crypt_d,
366         ~dec_key_gen_d,

```

```

360   key_clear_d,
361   prng_reseed_d,
362   data_out_clear_d
363 };
364
365 // This primitive is used to place a size-only constraint on
the
366 // buffers to act as a synthesis optimization barrier.
367 prim_buf #(
368   .Width(NumOutBufBits)
369 ) u_prim_buf_out (
370   .in_i(out),
371   .out_o(out_buf)
372 );
373
374 assign {in_ready_no,
375         out_valid_no,
376         alert_o,
377         prng_update_o,
378         prng_reseed_req_o,
379         state_sel_o,
380         state_we_o,
381         sub_bytes_en_no,
382         sub_bytes_out_ack_no,
383         add_rk_sel_o,
384         key_full_sel_o,
385         key_full_we_o,
386         key_dec_sel_o,
387         key_dec_we_o,
388         key_expand_en_no,
389         key_expand_out_ack_no,
390         key_expand_clear_o,
391         rnd_ctr_o,
392         key_words_sel_o,
393         round_key_sel_o,
394         crypt_d_no,
395         dec_key_gen_d_no,
396         key_clear_d_o,
397         prng_reseed_d_o,
398         data_out_clear_d_o} = out_buf;
399
400 endmodule

```

5. lowRISC_opentitan/hw/ip/aes/rtl/ aes_cipher_control_fsm_p.sv (390 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM oper-
ating on
8 // and producing the positive values of important control sig-
nals.
9
10 module aes_cipher_control_fsm_p import aes_pkg::*;
11 #(
12   parameter bit SecMasking = 0,
13   parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
14 ) (
15   input logic clk_i,
16   input logic rst_ni,
17
18   // Input handshake signals
19   input logic in_valid_i, // Sparsify
using multi-rail.
20   output logic in_ready_o, // Sparsify
using multi-rail.
21
22   // Output handshake signals
23   output logic out_valid_o, // Sparsify
using multi-rail.
24   input logic out_ready_i, // Sparsify
using multi-rail.
25
26   // Control and sync signals
27   input logic cfg_valid_i, // Used for
SVAs only.
28   input ciph_op_e op_i,
29   input key_len_e key_len_i,
30   input logic crypt_i, // Sparsify
using multi-rail.
31   input logic dec_key_gen_i, // Sparsify
using multi-rail.
32   input logic prng_reseed_i,
33   input logic key_clear_i,
34   input logic data_out_clear_i,
35   input logic mux_sel_err_i,
36   input logic sp_enc_err_i,
37   input logic rnd_ctr_err_i,
38   input logic op_err_i,
39   input logic alert_fatal_i,
40   output logic alert_o,
41
42   // Control signals for masking PRNG

```

```

43   output logic prng_update_o,
44   output logic prng_reseed_req_o,
45   input logic prng_reseed_ack_i,
46
47   // Control and sync signals for cipher data path
48   output state_sel_e state_sel_o,
49   output logic state_we_o, // Sparsify
using multi-rail.
50   output logic sub_bytes_en_o, // Sparsify
using multi-rail.
51   input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
52   output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
53   output add_rk_sel_e add_rk_sel_o,
54
55   // Control and sync signals for key expand data path
56   output key_full_sel_e key_full_sel_o,
57   output logic key_full_we_o, // Sparsify
using multi-rail.
58   output key_dec_sel_e key_dec_sel_o,
59   output logic key_dec_we_o, // Sparsify
using multi-rail.
60   output logic key_expand_en_o, // Sparsify
using multi-rail.
61   input logic key_expand_out_req_i, // Sparsify
using multi-rail.
62   output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
63   output logic key_expand_clear_o,
64   output logic [3:0] rnd_ctr_o,
65   output key_words_sel_e key_words_sel_o,
66   output round_key_sel_e round_key_sel_o,
67
68   // Register signals
69   input logic crypt_q_i, // Sparsify
using multi-rail.
70   output logic crypt_d_o, // Sparsify
using multi-rail.
71   input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
72   output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
73   input logic prng_reseed_q_i,
74   output logic prng_reseed_d_o,
75   input logic key_clear_q_i,
76   output logic key_clear_d_o,
77   input logic data_out_clear_q_i,
78   output logic data_out_clear_d_o
79 );
80
81 //////////////////////////////////////////////////
82 // Input Buffering //
83 //////////////////////////////////////////////////
84
85 localparam int NumInBufBits = $bits({
86   in_valid_i,
87   out_ready_i,
88   cfg_valid_i,
89   op_i,
90   key_len_i,
91   crypt_i,
92   dec_key_gen_i,
93   prng_reseed_i,
94   key_clear_i,
95   data_out_clear_i,
96   mux_sel_err_i,
97   sp_enc_err_i,
98   rnd_ctr_err_i,
99   op_err_i,
100  alert_fatal_i,
101  prng_reseed_ack_i,
102  sub_bytes_out_req_i,
103  key_expand_out_req_i,
104  crypt_q_i,
105  dec_key_gen_q_i,
106  prng_reseed_q_i,
107  key_clear_q_i,
108  data_out_clear_q_i
109 });
110
111 logic [NumInBufBits-1:0] in, in_buf;
112
113 assign in = {
114   in_valid_i,
115   out_ready_i,
116   cfg_valid_i,
117   op_i,
118   key_len_i,
119   crypt_i,
120   dec_key_gen_i,
121   prng_reseed_i,
122   key_clear_i,
123   data_out_clear_i,
124   mux_sel_err_i,
125   sp_enc_err_i,
126   rnd_ctr_err_i,
127   op_err_i,
128   alert_fatal_i,
129   prng_reseed_ack_i,
130   sub_bytes_out_req_i,

```



```

131     key_expand_out_req_i,
132     crypt_q_i,
133     dec_key_gen_q_i,
134     prng_reseed_q_i,
135     key_clear_q_i,
136     data_out_clear_q_i
137   };
138
139 // This primitive is used to place a size-only constraint on
the
140 // buffers to act as a synthesis optimization barrier.
141 prim_buf #(
142   .Width(NumInBufBits)
143 ) u_prim_buf_in (
144   .in_i(in),
145   .out_o(in_buf)
146 );
147
148 logic          in_valid;
149 logic          out_ready;
150 logic          cfg_valid;
151 ciph_op_e      op;
152 logic [$bits(op)-1:0] op_raw;
153 key_len_e      key_len;
154 logic          crypt;
155 logic          dec_key_gen;
156 logic          prng_reseed;
157 logic          key_clear;
158 logic          data_out_clear;
159 logic          mux_sel_err;
160 logic          sp_enc_err;
161 logic          rnd_ctr_err;
162 logic          op_err;
163 logic          alert_fatal;
164 logic          prng_reseed_ack;
165 logic          sub_bytes_out_req;
166 logic          key_expand_out_req;
167 logic          crypt_q;
168 logic          dec_key_gen_q;
169 logic          prng_reseed_q;
170 logic          key_clear_q;
171 logic          data_out_clear_q;
172
173 assign {in_valid,
174        out_ready,
175        cfg_valid,
176        op_raw,
177        key_len,
178        crypt,
179        dec_key_gen,
180        prng_reseed,
181        key_clear,
182        data_out_clear,
183        mux_sel_err,
184        sp_enc_err,
185        rnd_ctr_err,
186        op_err,
187        alert_fatal,
188        prng_reseed_ack,
189        sub_bytes_out_req,
190        key_expand_out_req,
191        crypt_q,
192        dec_key_gen_q,
193        prng_reseed_q,
194        key_clear_q,
195        data_out_clear_q} = in_buf;
196
197 assign op = ciph_op_e'(op_raw);
198
199 // Intermediate output signals
200 logic          in_ready;
201 logic          out_valid;
202 logic          alert;
203 logic          prng_update;
204 logic          prng_reseed_req;
205 state_sel_e    state_sel;
206 logic          state_we;
207 logic          sub_bytes_en;
208 logic          sub_bytes_out_ack;
209 add_rk_sel_e    add_rk_sel;
210 key_full_sel_e  key_full_sel;
211 logic          key_full_we;
212 key_dec_sel_e   key_dec_sel;
213 logic          key_dec_we;
214 logic          key_expand_en;
215 logic          key_expand_out_ack;
216 logic          key_expand_clear;
217 logic [3:0]     rnd_ctr;
218 key_words_sel_e key_words_sel;
219 round_key_sel_e round_key_sel;
220 logic          crypt_d;
221 logic          dec_key_gen_d;
222 logic          prng_reseed_d;
223 logic          key_clear_d;
224 logic          data_out_clear_d;
225
226 //////////////////////////////////////////////////
227 // Regular FSM //
228 //////////////////////////////////////////////////
229

```

```

230 aes_cipher_control_fsm #(
231   .SecMasking ( SecMasking ),
232   .SecSBoxImpl ( SecSBoxImpl )
233 ) u_aes_cipher_control_fsm (
234   .clk_i          ( clk_i          ),
235   .rst_ni         ( rst_ni         ),
236
237   .in_valid_i     ( in_valid       ),
238   .in_ready_o     ( in_ready       ),
239
240   .out_valid_o    ( out_valid      ),
241   .out_ready_i    ( out_ready      ),
242
243   .cfg_valid_i    ( cfg_valid      ),
244   .op_i           ( op             ),
245   .key_len_i      ( key_len        ),
246   .crypt_i        ( crypt          ),
247   .dec_key_gen_i  ( dec_key_gen    ),
248   .prng_reseed_i  ( prng_reseed    ),
249   .key_clear_i    ( key_clear      ),
250   .data_out_clear_i ( data_out_clear ),
251   .mux_sel_err_i  ( mux_sel_err    ),
252   .sp_enc_err_i   ( sp_enc_err     ),
253   .rnd_ctr_err_i  ( rnd_ctr_err    ),
254   .op_err_i       ( op_err         ),
255   .alert_fatal_i  ( alert_fatal    ),
256   .alert_o        ( alert          ),
257
258   .prng_update_o  ( prng_update    ),
259   .prng_reseed_req_o ( prng_reseed_req ),
260   .prng_reseed_ack_i ( prng_reseed_ack ),
261
262   .state_sel_o    ( state_sel      ),
263   .state_we_o     ( state_we       ),
264   .sub_bytes_en_o ( sub_bytes_en   ),
265   .sub_bytes_out_req_i ( sub_bytes_out_req ),
266   .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
267   .add_rk_sel_o   ( add_rk_sel     ),
268
269   .key_full_sel_o ( key_full_sel    ),
270   .key_full_we_o  ( key_full_we    ),
271   .key_dec_sel_o  ( key_dec_sel     ),
272   .key_dec_we_o   ( key_dec_we     ),
273   .key_expand_en_o ( key_expand_en  ),
274   .key_expand_out_req_i ( key_expand_out_req ),
275   .key_expand_out_ack_o ( key_expand_out_ack ),
276   .key_expand_clear_o ( key_expand_clear ),
277   .rnd_ctr_o      ( rnd_ctr        ),
278   .key_words_sel_o ( key_words_sel  ),
279   .round_key_sel_o ( round_key_sel  ),
280
281   .crypt_q_i      ( crypt_q        ),
282   .crypt_d_o      ( crypt_d        ),
283   .dec_key_gen_q_i ( dec_key_gen_q  ),
284   .dec_key_gen_d_o ( dec_key_gen_d  ),
285   .key_clear_q_i  ( key_clear_q    ),
286   .key_clear_d_o  ( key_clear_d    ),
287   .prng_reseed_q_i ( prng_reseed_q  ),
288   .prng_reseed_d_o ( prng_reseed_d  ),
289   .data_out_clear_q_i ( data_out_clear_q ),
290   .data_out_clear_d_o ( data_out_clear_d ),
291 );
292
293 //////////////////////////////////////////////////
294 // Output Buffering //
295 //////////////////////////////////////////////////
296
297 localparam int NumOutBufBits = $bits({
298   in_ready_o,
299   out_valid_o,
300   alert_o,
301   prng_update_o,
302   prng_reseed_req_o,
303   state_sel_o,
304   state_we_o,
305   sub_bytes_en_o,
306   sub_bytes_out_ack_o,
307   add_rk_sel_o,
308   key_full_sel_o,
309   key_full_we_o,
310   key_dec_sel_o,
311   key_dec_we_o,
312   key_expand_en_o,
313   key_expand_out_ack_o,
314   key_expand_clear_o,
315   rnd_ctr_o,
316   key_words_sel_o,
317   round_key_sel_o,
318   crypt_d_o,
319   dec_key_gen_d_o,
320   key_clear_d_o,
321   prng_reseed_d_o,
322   data_out_clear_d_o
323 });
324
325 logic [NumOutBufBits-1:0] out, out_buf;
326
327 assign out = {
328   in_ready,
329   out_valid,
330   alert,

```

```

331 prng_update,
332 prng_reseed_req,
333 state_sel,
334 state_we,
335 sub_bytes_en,
336 sub_bytes_out_ack,
337 add_rk_sel,
338 key_full_sel,
339 key_full_we,
340 key_dec_sel,
341 key_dec_we,
342 key_expand_en,
343 key_expand_out_ack,
344 key_expand_clear,
345 rnd_ctr,
346 key_words_sel,
347 round_key_sel,
348 crypt_d,
349 dec_key_gen_d,
350 key_clear_d,
351 prng_reseed_d,
352 data_out_clear_d
353 };
354
355 // This primitive is used to place a size-only constraint on
the
356 // buffers to act as a synthesis optimization barrier.
357 prim_buf #(
358     .Width(NumOutBufBits)
359 ) u_prim_buf_out (
360     .in_i(out),
361     .out_o(out_buf)
362 );
363
364 assign {in_ready_o,
365         out_valid_o,
366         alert_o,
367         prng_update_o,
368         prng_reseed_req_o,
369         state_sel_o,
370         state_we_o,
371         sub_bytes_en_o,
372         sub_bytes_out_ack_o,
373         add_rk_sel_o,
374         key_full_sel_o,
375         key_full_we_o,
376         key_dec_sel_o,
377         key_dec_we_o,
378         key_expand_en_o,
379         key_expand_out_ack_o,
380         key_expand_clear_o,
381         rnd_ctr_o,
382         key_words_sel_o,
383         round_key_sel_o,
384         crypt_d_o,
385         dec_key_gen_d_o,
386         key_clear_d_o,
387         prng_reseed_d_o,
388         data_out_clear_d_o} = out_buf;
389
390 endmodule

```

6. lowRISC_opentitan/hw/ip/aes/rtl/aes_cipher_core.sv (932 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core implementation
6 //
7 // This module contains the AES cipher core including, state
register, full key and decryption key
8 // registers as well as key expand module and control unit.
9 //
10 //
11 // Masking
12 // -----
13 //
14 // If the parameter "Masking" is set to one, first-order mask-
ing is applied to the entire
15 // cipher core including key expand module. For details, see
Rivain et al., "Provably secure
16 // higher-order masking of AES" available at https://
eprint.iacr.org/2010/441.pdf .
17 //
18 //
19 // Details on the data formats
20 // -----
21 //
22 // This implementation uses 4-dimensional SystemVerilog arrays to
represent the AES state:
23 //
24 //     logic [3:0][3:0][7:0] state_q [NumShares];
25 //
26 // The fourth dimension (unpacked) corresponds to the differ-

```

ent shares. The first element holds the
27 // (masked) data share whereas the other elements hold the
masks (masked implementation only).

28 // The three packed dimensions correspond to the 128-bit state
matrix per share. This
29 // implementation uses the same encoding as the Advanced En-
cryption Standard (AES) FIPS Publication
30 // 197 available at https://www.nist.gov/publications/ad-
vanced-encryption-standard-aes (see Section
31 // 3.4). An input sequence of 16 bytes (128-bit, left most
byte is the first one)

```

32 //
33 //     b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14 b15
34 //
35 // is mapped to the state matrix as
36 //
37 //     [ b0 b4 b8 b12 ]
38 //     [ b1 b5 b9 b13 ]
39 //     [ b2 b6 b10 b14 ]
40 //     [ b3 b7 b11 b15 ] .
41 //
42 // This is mapped to three packed dimensions of SystemVerilog
array as follows:
43 // - The first dimension corresponds to the rows. Thus,
state_q[0] gives
44 // - The first row of the state matrix [ b0 b4 b8
b12 ], or
45 // - A 32-bit packed SystemVerilog array 32h'{ b12, b8, b4,
b0 }.
46 //
47 // - The second dimension corresponds to the columns. To ac-
cess complete columns, the state matrix
48 // must be transposed first. Thus state_transposed =
aes_pkg::aes_transpose(state_q) and then
49 // state_transposed[1] gives
50 // - The second column of the state matrix [ b4 b5 b6
b7 ], or
51 // - A 32-bit packed SystemVerilog array 32h'{ b7, b6, b5,
b4 }.
52 //
53 // - The third dimension corresponds to the bytes.
54 //
55 // Note that the CSRs are little-endian. The input sequence
above is provided to 32-bit DATA_IN_0 -
56 // DATA_IN_3 registers as
57 //
58 //     MSB                                LSB
59 // - DATA_IN_0 32h'{ b3 , b2 , b1 , b0 }
60 // - DATA_IN_1 32h'{ b7 , b6 , b4 , b4 }
61 // - DATA_IN_2 32h'{ b11, b10, b9 , b8 }
62 // - DATA_IN_3 32h'{ b15, b14, b13, b12 } .
63 //
64 // The input state can thus be obtained by transposing the
content of the DATA_IN_0 - DATA_IN_3
65 // registers.

```

66 // Similarly, the implementation uses a 3-dimensional array to
represent the AES keys:

```

67 //
68 //     logic [7:0][31:0] key_full_q [NumShares]
69 //

```

70 // The third dimension (unpacked) corresponds to the different
shares. The first element holds the
71 // (masked) key share whereas the other elements hold the
masks (masked implementation only).

72 // The two packed dimensions correspond to the 256-bit key per
share. This implementation uses

73 // the same encoding as the Advanced Encryption Standard (AES)
FIPS Publication
74 // 197 available at https://www.nist.gov/publications/ad-
vanced-encryption-standard-aes .

75 //
76 // The first packed dimension corresponds to the 8 key words.
The second packed dimension
77 // corresponds to the 32 bits per key word. A key sequence of
32 bytes (256-bit, left most byte is
78 // the first one)

```

79 //
80 //     b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14
b15 ... .. b28 b29 b30 b31
81 //
82 // is mapped to the key words and registers (little-endian) as
83 //
84 //     MSB                                LSB
85 // - KEY_SHARE0_0 32h'{ b3 , b2 , b1 , b0 }
86 // - KEY_SHARE0_1 32h'{ b7 , b6 , b4 , b4 }
87 // - KEY_SHARE0_2 32h'{ b11, b10, b9 , b8 }
88 // - KEY_SHARE0_3 32h'{ b15, b14, b13, b12 }
89 // - KEY_SHARE0_4 32h'{ . . . . }
90 // - KEY_SHARE0_5 32h'{ . . . . }
91 // - KEY_SHARE0_6 32h'{ . . . . }
92 // - KEY_SHARE0_7 32h'{ b31, b30, b29, b28 } .
93
94 `include "prim_assert.sv"
95
96 module aes_cipher_core import aes_pkg::*;
97
98 parameter bit AES192Enable = 1,

```

```

98 parameter bit CiphOpFwdOnly = 0,
99 parameter bit SecMasking = 1,
100 parameter sbox_impl_e SecSBoxImpl = SBoxImplDom,
101 parameter bit SecAllowForcingMasks = 0,
102 parameter bit SecSkipPRNGReseeding = 0,
103 parameter int unsigned EntropyWidth = edn_pkg::END-
POINT_BUS_WIDTH,
104
105 localparam int NumShares = SecMasking ?
2 : 1, // derived parameter
106
107 parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed = Rnd-
CnstMaskingLfsrSeedDefault,
108 parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm = Rnd-
CnstMaskingLfsrPermDefault
109 ) (
110 input logic clk_i,
111 input logic rst_ni,
112
113 // Input handshake signals
114 input sp2v_e in_valid_i,
115 output sp2v_e in_ready_o,
116
117 // Output handshake signals
118 output sp2v_e out_valid_o,
119 input sp2v_e out_ready_i,
120
121 // Control and sync signals
122 input logic cfg_valid_i, // Used for
gating assertions only.
123 input ciph_op_e op_i,
124 input key_len_e key_len_i,
125 input sp2v_e crypt_i,
126 output sp2v_e crypt_o,
127 input sp2v_e dec_key_gen_i,
128 output sp2v_e dec_key_gen_o,
129 input logic prng_reseed_i,
130 output logic prng_reseed_o,
131 input logic key_clear_i,
132 output logic key_clear_o,
133 input logic data_out_clear_i, // Re-
use the cipher core muxes.
134 output logic data_out_clear_o,
135 input logic alert_fatal_i,
136 output logic alert_o,
137
138 // Pseudo-random data for register clearing
139 input logic [3:0][3:0][7:0] prd_clearing_state_i
[NumShares],
140 input logic [7:0][31:0] prd_clearing_key_i [Num-
Shares],
141
142 // Masking PRNG
143 input logic force_masks_i, // Useful
for SCA only.
144 output logic [3:0][3:0][7:0] data_in_mask_o,
145 output logic entropy_req_o,
146 input logic entropy_ack_i,
147 input logic [EntropyWidth-1:0] entropy_i,
148
149 // I/O data & initial key
150 input logic [3:0][3:0][7:0] state_init_i [Num-
Shares],
151 input logic [7:0][31:0] key_init_i [NumShares],
152 output logic [3:0][3:0][7:0] state_o [NumShares]
153 );
154
155 // Signals
156 logic [3:0][3:0][7:0] state_d [NumShares];
157 logic [3:0][3:0][7:0] state_q [NumShares];
158 sp2v_e state_we_ctrl;
159 sp2v_e state_we;
160 logic [StateSelWidth-1:0] state_sel_raw;
161 state_sel_e state_sel_ctrl;
162 state_sel_e state_sel;
163 logic state_sel_err;
164
165 sp2v_e sub_bytes_en;
166 sp2v_e sub_bytes_out_req;
167 sp2v_e sub_bytes_out_ack;
168 logic sub_bytes_err;
169 logic [3:0][3:0][7:0] sub_bytes_out;
170 logic [3:0][3:0][7:0] sb_in_mask;
171 logic [3:0][3:0][7:0] sb_out_mask;
172 logic [3:0][3:0][7:0] shift_rows_in [Num-
Shares];
173 logic [3:0][3:0][7:0] shift_rows_out [Num-
Shares];
174 logic [3:0][3:0][7:0] mix_columns_out [Num-
Shares];
175 logic [3:0][3:0][7:0] add_round_key_in [Num-
Shares];
176 logic [3:0][3:0][7:0] add_round_key_out [Num-
Shares];
177 logic [AddRKSelWidth-1:0] add_rk_sel_raw;
178 add_rk_sel_e add_rk_sel_ctrl;
179 add_rk_sel_e add_rk_sel;
180 logic add_rk_sel_err;
181

```

```

182 logic [7:0][31:0] key_full_d [NumShares];
183 logic [7:0][31:0] key_full_q [NumShares];
184 sp2v_e key_full_we_ctrl;
185 sp2v_e key_full_we;
186 logic [KeyFullSelWidth-1:0] key_full_sel_raw;
187 key_full_sel_e key_full_sel_ctrl;
188 key_full_sel_e key_full_sel;
189 logic key_full_sel_err;
190 logic [7:0][31:0] key_dec_d [NumShares];
191 logic [7:0][31:0] key_dec_q [NumShares];
192 sp2v_e key_dec_we_ctrl;
193 sp2v_e key_dec_we;
194 logic [KeyDecSelWidth-1:0] key_dec_sel_raw;
195 key_dec_sel_e key_dec_sel_ctrl;
196 key_dec_sel_e key_dec_sel;
197 logic key_dec_sel_err;
198 logic [7:0][31:0] key_expand_out [Num-
Shares];
199 ciph_op_e key_expand_op;
200 sp2v_e key_expand_en;
201 logic key_expand_prd_we;
202 sp2v_e key_expand_out_req;
203 sp2v_e key_expand_out_ack;
204 logic key_expand_err;
205 logic key_expand_clear;
206 logic [3:0] key_expand_round;
207 logic [KeywordsSelWidth-1:0] key_words_sel_raw;
208 key_words_sel_e key_words_sel_ctrl;
209 key_words_sel_e key_words_sel;
210 logic key_words_sel_err;
211 logic [3:0][31:0] key_words [NumShares];
212 logic [3:0][3:0][7:0] key_bytes [NumShares];
213 logic [3:0][3:0][7:0] key_mix_columns_out
[NumShares];
214 logic [3:0][3:0][7:0] round_key [NumShares];
215 logic [RoundKeySelWidth-1:0] round_key_sel_raw;
216 round_key_sel_e round_key_sel_ctrl;
217 round_key_sel_e round_key_sel;
218 logic round_key_sel_err;
219
220 logic cfg_valid;
221 logic mux_sel_err;
222 logic sp_enc_err_q;
223 logic op_err;
224
225 // Pseudo-random data for masking purposes
226 logic [WidthPRDMasking-1:0] prd_masking;
227 logic [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_d;
228 logic [3:0][3:0][WidthPRDSBox-1:0] prd_sub_bytes_q;
229 logic [WidthPRDKey-1:0] prd_key_expand;
230 logic prd_masking_upd;
231 logic prd_masking_rsd_req;
232 logic prd_masking_rsd_ack;
233
234 logic [3:0][3:0][7:0] data_in_mask;
235
236 // op_i is one-hot encoded. Check the provided value and
trigger an alert upon detecting invalid
237 // encodings.
238 assign op_err = ~(op_i == CIPH_FWD || op_i == CIPH_INV);
239 assign cfg_valid = cfg_valid_i & ~op_err;
240
241 // Data //
242 // Data //
243 // Data //
244
245 // SEC_CM: DATA_REG.SEC_WIPE
246 // State registers
247 always_comb begin : state_mux
248     unique case (state_sel)
249         STATE_INIT: state_d = state_init_i;
250         STATE_ROUND: state_d = add_round_key_out;
251         STATE_CLEAR: state_d = prd_clearing_state_i;
252         default: state_d = prd_clearing_state_i;
253     endcase
254 end
255
256 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_reg
257     if (!rst_ni) begin
258         state_q <= '{default: '0};
259     end else if (state_we == SP2V_HIGH) begin
260         state_q <= state_d;
261     end
262 end
263
264 // Masking
265 if (!SecMasking) begin : gen_no_masks
266     // The masks are ignored anyway, they can be 0.
267     assign sb_in_mask = '0;
268     assign prd_masking = '0;
269
270 // Tie-off unused signals.
271 logic unused_entropy_ack;
272 logic [EntropyWidth-1:0] unused_entropy;
273 assign unused_entropy_ack = entropy_ack_i;
274 assign unused_entropy = entropy_i;
275 assign entropy_req_o = '1'b0;
276
277 logic unused_force_masks;

```



```

278 logic unused_prd_masking_upd;
279 logic unused_prd_masking_rsd_req;
280 assign unused_force_masks = force_masks_i;
281 assign unused_prd_masking_upd = prd_masking_upd;
282 assign unused_prd_masking_rsd_req = prd_masking_rsd_req;
283 assign prd_masking_rsd_ack = 1'b0;
284
285 logic [3:0][3:0][7:0] unused_sb_out_mask;
286 assign unused_sb_out_mask = sb_out_mask;
287
288 end else begin : gen_masks
289 // The input mask is the mask share of the state.
290 assign sb_in_mask = state_q[1];
291
292 // The masking PRNG generates:
293 // - the pseudo-random data (PRD) required by SubBytes,
294 // - the PRD required by the key expand module (has 4 S-Boxes internally).
295 aes_prng_masking #(
296     .Width ( WidthPRDMasking ),
297     .EntropyWidth ( EntropyWidth ),
298     .SecAllowForcingMasks ( SecAllowForcingMasks ),
299     .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
300     .RndCnstLfsrcSeed ( RndCnstLfsrcSeed ),
301     .RndCnstLfsrcPerm ( RndCnstLfsrcPerm )
302 ) u_aes_prng_masking (
303     .clk_i ( clk_i ),
304     .rst_ni ( rst_ni ),
305     .force_masks_i ( force_masks_i ),
306     .data_update_i ( prd_masking_upd ),
307     .data_o ( prd_masking ),
308     .reseed_req_i ( prd_masking_rsd_req ),
309     .reseed_ack_o ( prd_masking_rsd_ack ),
310     .entropy_req_o ( entropy_req_o ),
311     .entropy_ack_i ( entropy_ack_i ),
312     .entropy_i ( entropy_i )
313 );
314 end
315
316 // Extract randomness for key expand module and SubBytes.
317 //
318 // The masking PRNG output has the following shape:
319 // prd_masking = { prd_key_expand, prd_sub_bytes_d }
320 assign prd_key_expand = prd_masking[WidthPRDMasking-1 -: WidthPRDKey];
321 assign prd_sub_bytes_d = prd_masking[WidthPRDData-1 -: WidthPRDData];
322
323 // PRD buffering
324 if (!SecMasking) begin : gen_no_prd_buffer
325 // The masks are ignored anyway.
326 assign prd_sub_bytes_q = prd_sub_bytes_d;
327
328 end else begin : gen_prd_buffer
329 // PRD buffer stage to:
330 // 1. Make sure the S-Boxes get always presented new data/
331 // mask inputs together with fresh PRD
332 // for remasking.
333
334 // 2. Prevent glitches originating from inside the masking
335 // PRNG from propagating into the
336 // masked S-Boxes.
337 always_ff @(posedge clk_i or negedge rst_ni) begin :
338 prd_sub_bytes_reg
339 if (!rst_ni) begin
340 prd_sub_bytes_q <= '0;
341 end else if (state_we == SP2V_HIGH) begin
342 prd_sub_bytes_q <= prd_sub_bytes_d;
343 end
344 end
345
346 // Convert the 3-dimensional prd_sub_bytes_q array to a 1-
347 // dimensional packed array for the
348 // aes_prd_get_lsbs() function used below.
349 logic [WidthPRDData-1:0] prd_sub_bytes;
350 assign prd_sub_bytes = prd_sub_bytes_q;
351
352 // Extract randomness for masking the input data.
353 //
354 // The masking PRNG is used for generating both the PRD for
355 // the S-Boxes/SubBytes operation as
356 // well as for the input data masks. When using any of the
357 // masked Canright S-Box implementations,
358 // it is important that the SubBytes input masks (generated
359 // by the PRNG in Round X-1) and the
360 // SubBytes output masks (generated by the PRNG in Round X)
361 // are independent. This can be achieved
362 // by using e.g. an unrolled Bivium stream cipher primitive
363 // inside the PRNG. Since the input data
364
365 // masks become the SubBytes input masks in the first round,
366 // we select the same 8 bit lanes for
367 // the input data masks which are also used to form the Sub-
368 // Bytes output mask for the masked
369 // Canright S-Box implementations, i.e., the 8 LSBs of the
370 // per S-Box PRD. In particular, we have:
371 //
372 // prd_masking = { prd_key_expand, ... , sb_prd[4],
373 // sb_out_mask[4], sb_prd[0], sb_out_mask[0] }
374 //

```

```

361 // Where sb_out_mask[x] contains the SubBytes output mask
362 // for byte x (when using a masked
363 // Canright S-Box implementation) and sb_prd[x] contains ad-
364 // ditional PRD consumed by SubBytes for
365 // byte x.
366 //
367 // When using a masked S-Box implementation other than Can-
368 // right, we still select the 8 LSBs of
369 // the per-S-Box PRD to form the input data mask of the cor-
370 // responding byte. We do this to
371 // distribute the input data masks over all output bits the
372 // masking PRNG. We do the extraction on
373 // a row basis.
374 localparam int unsigned WidthPRDRow = 4*WidthPRDSubBox;
375 for (genvar i = 0; i < 4; i++) begin : gen_in_mask
376 assign data_in_mask[i] = aes_prd_get_lsbs(prd_sub_bytes[i
377 * WidthPRDRow -: WidthPRDRow]);
378 end
379
380 // Rotate the data input masks by 64 bits to ensure the data
381 // input masks are independent
382 // from the PRD fed to the S-Boxes/SubBytes operation.
383 assign data_in_mask_o = {data_in_mask[1], data_in_mask[0],
384 data_in_mask[3], data_in_mask[2]};
385
386 // Cipher data path
387 aes_sub_bytes #(
388     .SecSBoxImpl ( SecSBoxImpl )
389 ) u_aes_sub_bytes (
390     .clk_i ( clk_i ),
391     .rst_ni ( rst_ni ),
392     .en_i ( sub_bytes_en ),
393     .out_req_o ( sub_bytes_out_req ),
394     .out_ack_i ( sub_bytes_out_ack ),
395     .op_i ( op_i ),
396     .data_i ( state_q[0] ),
397     .mask_i ( sb_in_mask ),
398     .prd_i ( prd_sub_bytes_q ),
399     .data_o ( sub_bytes_out ),
400     .mask_o ( sb_out_mask ),
401     .err_o ( sub_bytes_err )
402 );
403
404 for (genvar s = 0; s < NumShares; s++) begin :
405 gen_shares_shift_mix
406 if (s == 0) begin : gen_shift_in_data
407 // The (masked) data share
408 assign shift_rows_in[s] = sub_bytes_out;
409 end else begin : gen_shift_in_mask
410 // The mask share
411 assign shift_rows_in[s] = sb_out_mask;
412 end
413
414 aes_shift_rows u_aes_shift_rows (
415     .op_i ( op_i ),
416     .data_i ( shift_rows_in[s] ),
417     .data_o ( shift_rows_out[s] )
418 );
419
420 aes_mix_columns u_aes_mix_columns (
421     .op_i ( op_i ),
422     .data_i ( shift_rows_out[s] ),
423     .data_o ( mix_columns_out[s] )
424 );
425 end
426
427 always_comb begin : add_round_key_in_mux
428 unique case (add_rk_sel)
429 ADD_RK_INIT: add_round_key_in = state_q;
430 ADD_RK_ROUND: add_round_key_in = mix_columns_out;
431 ADD_RK_FINAL: add_round_key_in = shift_rows_out;
432 default: add_round_key_in = state_q;
433 endcase
434 end
435
436 for (genvar s = 0; s < NumShares; s++) begin :
437 gen_shares_add_round_key
438 assign add_round_key_out[s] = add_round_key_in[s] ^
439 round_key[s];
440 end
441
442 // Key
443 //
444 // SEC_CM: KEY.SEC_WIPE
445 // Full Key registers
446 always_comb begin : key_full_mux
447 unique case (key_full_sel)
448 KEY_FULL_ENC_INIT: key_full_d = key_init_i;
449 KEY_FULL_DEC_INIT: key_full_d = !CiphOpFwdOnly ?
450 key_dec_q : prd_clearing_key_i;
451 KEY_FULL_ROUND: key_full_d = key_expand_out;
452 KEY_FULL_CLEAR: key_full_d = prd_clearing_key_i;
453 default: key_full_d = prd_clearing_key_i;
454 endcase
455 end
456
457 always_ff @(posedge clk_i or negedge rst_ni) begin :
458 key_full_reg

```

```

448 if (!rst_ni) begin
449   key_full_q <= '{default: '0}';
450 end else if (key_full_we == SP2V_HIGH) begin
451   key_full_q <= key_full_d;
452 end
453 end
454
455 if (!CiphOpFwdOnly) begin : gen_key_dec
456   // SEC_CM: KEY.SEC.WIPE
457   // Decryption Key registers
458   always_comb begin : key_dec_mux
459     unique case (key_dec_sel)
460       KEY_DEC_EXPAND: key_dec_d = key_expand_out;
461       KEY_DEC_CLEAR: key_dec_d = prd_clearing_key_i;
462       default:       key_dec_d = prd_clearing_key_i;
463     endcase
464   end
465
466   always_ff @(posedge clk_i or negedge rst_ni) begin :
key_dec_reg
467     if (!rst_ni) begin
468       key_dec_q <= '{default: '0}';
469     end else if (key_dec_we == SP2V_HIGH) begin
470       key_dec_q <= key_dec_d;
471     end
472   end
473 end else begin : gen_no_key_dec
474   // No Decryption Key registers
475   assign key_dec_q = '{default: '0}';
476   assign key_dec_d = key_dec_q;
477
478   // Tie-off unused signals.
479   logic unused_key_dec;
480   always_comb begin
481     unused_key_dec = ^{key_dec_sel, key_dec_we};
482     for (int s = 0; s < NumShares; s++) begin
483       unused_key_dec ^= ^{key_dec_d[s]};
484     end
485   end
486 end
487
488 // Make sure that whenever the data/mask inputs of the S-
Boxes update, the internally buffered
489 // PRD is updated in sync.
490 assign key_expand_prd_we = (key_full_we == SP2V_HIGH) ?
1'b1 : 1'b0;
491
492 // Key expand data path
493 aes_key_expand #(
494   .AES192Enable ( AES192Enable ),
495   .SecMasking   ( SecMasking ),
496   .SecSBoxImpl  ( SecSBoxImpl )
497 ) u_aes_key_expand (
498   .clk_i      ( clk_i ),
499   .rst_ni     ( rst_ni ),
500   .cfg_valid_i ( cfg_valid ),
501   .op_i       ( key_expand_op ),
502   .en_i       ( key_expand_en ),
503   .prd_we_i   ( key_expand_prd_we ),
504   .out_req_o  ( key_expand_out_req ),
505   .out_ack_i  ( key_expand_out_ack ),
506   .clear_i    ( key_expand_clear ),
507   .round_i    ( key_expand_round ),
508   .key_len_i  ( key_len_i ),
509   .key_i      ( key_full_q ),
510   .key_o      ( key_expand_out ),
511   .prd_i      ( prd_key_expand ),
512   .err_o      ( key_expand_err ),
513 );
514
515 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_round_key
516   always_comb begin : key_words_mux
517     unique case (key_words_sel)
518       KEY_WORDS_0123: key_words[s] = key_full_q[s][3:0];
519       KEY_WORDS_2345: key_words[s] = AES192Enable ?
key_full_q[s][5:2] : '0;
520       KEY_WORDS_4567: key_words[s] = key_full_q[s][7:4];
521       KEY_WORDS_ZERO: key_words[s] = '0;
522       default:       key_words[s] = '0;
523     endcase
524   end
525
526   // Convert words to bytes (every key word contains one
column).
527   assign key_bytes[s] = aes_transpose(key_words[s]);
528
529   aes_mix_columns u_aes_key_mix_columns (
530     .op_i ( CIPH_INV ),
531     .data_i ( key_bytes[s] ),
532     .data_o ( key_mix_columns_out[s] )
533   );
534 end
535
536 always_comb begin : round_key_mux
537   unique case (round_key_sel)
538     ROUND_KEY_DIRECT: round_key = key_bytes;
539     ROUND_KEY_MIXED:  round_key = !CiphOpFwdOnly ?
key_mix_columns_out : key_bytes;
540     default:         round_key = key_bytes;
541   endcase

```

```

542 end
543
544 if (CiphOpFwdOnly) begin : gen_unused_key_mix_columns_out
545   // Tie-off unused signals.
546   logic unused_key_mix_columns_out;
547   always_comb begin
548     unused_key_mix_columns_out = 1'b0;
549     for (int s = 0; s < NumShares; s++) begin
550       unused_key_mix_columns_out ^=
^key_mix_columns_out[s];
551     end
552   end
553 end
554
555 // Control
556 // Control
557 // Control
558 // Control
559 aes_cipher_control #(
560   .CiphOpFwdOnly ( CiphOpFwdOnly ),
561   .SecMasking    ( SecMasking ),
562   .SecSBoxImpl   ( SecSBoxImpl )
563 ) u_aes_cipher_control (
564   .clk_i      ( clk_i ),
565   .rst_ni     ( rst_ni ),
566   .in_valid_i ( in_valid_i ),
567   .in_ready_o ( in_ready_o ),
568   .out_valid_o ( out_valid_o ),
569   .out_ready_i ( out_ready_i ),
570   .cfg_valid_i ( cfg_valid ),
571   .op_i       ( op_i ),
572   .key_len_i  ( key_len_i ),
573   .crypt_i    ( crypt_i ),
574   .crypt_o    ( crypt_o ),
575   .dec_key_gen_i ( dec_key_gen_i ),
576   .dec_key_gen_o ( dec_key_gen_o ),
577   .prng_reseed_i ( prng_reseed_i ),
578   .prng_reseed_o ( prng_reseed_o ),
579   .key_clear_i ( key_clear_i ),
580   .key_clear_o ( key_clear_o ),
581   .data_out_clear_i ( data_out_clear_i ),
582   .data_out_clear_o ( data_out_clear_o ),
583   .mux_sel_err_i ( mux_sel_err ),
584   .sp_enc_err_i ( sp_enc_err_q ),
585   .op_err_i     ( op_err ),
586   .alert_fatal_i ( alert_fatal_i ),
587   .alert_o      ( alert_o ),
588   .prng_update_o ( prd_masking_upd ),
589   .prng_reseed_req_o ( prd_masking_rsd_req ),
590   .prng_reseed_ack_i ( prd_masking_rsd_ack ),
591   .state_sel_o ( state_sel_ctrl ),
592   .state_we_o ( state_we_ctrl ),
593   .sub_bytes_en_o ( sub_bytes_en ),
594   .sub_bytes_out_req_i ( sub_bytes_out_req ),
595   .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
596   .add_rk_sel_o ( add_rk_sel_ctrl ),
597   .key_expand_op_o ( key_expand_op ),
598   .key_full_sel_o ( key_full_sel_ctrl ),
599   .key_full_we_o ( key_full_we_ctrl ),
600   .key_dec_sel_o ( key_dec_sel_ctrl ),
601   .key_dec_we_o ( key_dec_we_ctrl ),
602   .key_expand_en_o ( key_expand_en ),
603   .key_expand_out_req_i ( key_expand_out_req ),
604   .key_expand_out_ack_o ( key_expand_out_ack ),
605   .key_expand_clear_o ( key_expand_clear ),
606   .key_expand_round_o ( key_expand_round ),
607   .key_words_sel_o ( key_words_sel_ctrl ),
608   .round_key_sel_o ( round_key_sel_ctrl ),
609 );
610
611 // We use sparse encodings for these mux selector signals
and must ensure that:
612 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
613 // 2. The selector signal is always valid. More precisely,
an alert or SVA is triggered if a
614 // selector signal takes on an invalid value.
615 // 3. The alert signal remains asserted until reset even if
the selector signal becomes valid
616 // again. This is achieved by driving the control FSM
into the terminal error state whenever
617 // any mux selector signal becomes invalid.
618 //
619 // If any mux selector signal becomes invalid, the cipher
core further immediately de-asserts
620 // the out_valid_o signal to prevent any data from being re-
leased.
621
622 aes_sel_buf_chk #(
623   .Num ( StateSelNum ),

```

```

635 .Width ( StateSelWidth ),
636 .EnSecBuf ( 1'b1 )
637 ) u_aes_state_sel_buf_chk (
638 .clk_i ( clk_i ),
639 .rst_ni ( rst_ni ),
640 .sel_i ( state_sel_ctrl ),
641 .sel_o ( state_sel_raw ),
642 .err_o ( state_sel_err )
643 );
644 assign state_sel = state_sel_e'(state_sel_raw);
645
646 aes_sel_buf_chk #(
647 .Num ( AddrKSelNum ),
648 .Width ( AddrKSelWidth ),
649 .EnSecBuf ( 1'b1 )
650 ) u_aes_add_rk_sel_buf_chk (
651 .clk_i ( clk_i ),
652 .rst_ni ( rst_ni ),
653 .sel_i ( add_rk_sel_ctrl ),
654 .sel_o ( add_rk_sel_raw ),
655 .err_o ( add_rk_sel_err )
656 );
657 assign add_rk_sel = add_rk_sel_e'(add_rk_sel_raw);
658
659 aes_sel_buf_chk #(
660 .Num ( KeyFullSelNum ),
661 .Width ( KeyFullSelWidth ),
662 .EnSecBuf ( 1'b1 )
663 ) u_aes_key_full_sel_buf_chk (
664 .clk_i ( clk_i ),
665 .rst_ni ( rst_ni ),
666 .sel_i ( key_full_sel_ctrl ),
667 .sel_o ( key_full_sel_raw ),
668 .err_o ( key_full_sel_err )
669 );
670 assign key_full_sel = key_full_sel_e'(key_full_sel_raw);
671
672 aes_sel_buf_chk #(
673 .Num ( KeyDecSelNum ),
674 .Width ( KeyDecSelWidth ),
675 .EnSecBuf ( 1'b1 )
676 ) u_aes_key_dec_sel_buf_chk (
677 .clk_i ( clk_i ),
678 .rst_ni ( rst_ni ),
679 .sel_i ( key_dec_sel_ctrl ),
680 .sel_o ( key_dec_sel_raw ),
681 .err_o ( key_dec_sel_err )
682 );
683 assign key_dec_sel = key_dec_sel_e'(key_dec_sel_raw);
684
685 aes_sel_buf_chk #(
686 .Num ( KeyWordsSelNum ),
687 .Width ( KeyWordsSelWidth ),
688 .EnSecBuf ( 1'b1 )
689 ) u_aes_key_words_sel_buf_chk (
690 .clk_i ( clk_i ),
691 .rst_ni ( rst_ni ),
692 .sel_i ( key_words_sel_ctrl ),
693 .sel_o ( key_words_sel_raw ),
694 .err_o ( key_words_sel_err )
695 );
696 assign key_words_sel = key_words_sel_e'(key_words_sel_raw);
697
698 aes_sel_buf_chk #(
699 .Num ( RoundKeySelNum ),
700 .Width ( RoundKeySelWidth ),
701 .EnSecBuf ( 1'b1 )
702 ) u_aes_round_key_sel_buf_chk (
703 .clk_i ( clk_i ),
704 .rst_ni ( rst_ni ),
705 .sel_i ( round_key_sel_ctrl ),
706 .sel_o ( round_key_sel_raw ),
707 .err_o ( round_key_sel_err )
708 );
709 assign round_key_sel = round_key_sel_e'(round_key_sel_raw);
710
711 // Signal invalid mux selector signals to control FSM which
will lock up and trigger an alert.
712 assign mux_sel_err = state_sel_err | add_rk_sel_err |
key_full_sel_err |
713 key_dec_sel_err | key_words_sel_err | round_key_sel_err;
714
715 ///////////////////////////////////////////////////
716 // Sparsely Encoded Signals //
717 ///////////////////////////////////////////////////
718
719 // We use sparse encodings for various critical signals and
must ensure that:
720 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
721 // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
722 // if a sparse signal takes on an invalid value.
723 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
724 // This is achieved by driving the control FSM into the
terminal error state whenever any
725 // sparsely encoded signal becomes invalid.
726 //
727 // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts

```

```

728 // the out_valid_o signal to prevent any data from being re-
leased.
729
730 // We use vectors of sparsely encoded signals to reduce code
duplication.
731 localparam int unsigned NumSp2VSig = 3;
732 sp2v_e [NumSp2VSig-1:0] sp2v_sig;
733 sp2v_e [NumSp2VSig-1:0] sp2v_sig_chk;
734 logic [NumSp2VSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
735 logic [NumSp2VSig-1:0] sp2v_sig_err;
736
737 assign sp2v_sig[0] = state_we_ctrl;
738 assign sp2v_sig[1] = key_full_we_ctrl;
739 assign sp2v_sig[2] = key_dec_we_ctrl;
740
741 // All signals inside sp2v_sig are eventually converted to
single-rail signals.
742 localparam bit [NumSp2VSig-1:0] Sp2VEnSecBuf = {Num-
Sp2VSig{1'b1}};
743
744 // Individually check sparsely encoded signals.
745 for (genvar i = 0; i < NumSp2VSig; i++) begin :
gen_sel_buf_chk
746     aes_sel_buf_chk #(
747         .Num ( Sp2VNum ),
748         .Width ( Sp2VWidth ),
749         .EnSecBuf ( Sp2VEnSecBuf[i] )
750     ) u_aes_sp2v_sig_buf_chk_i (
751         .clk_i ( clk_i ),
752         .rst_ni ( rst_ni ),
753         .sel_i ( sp2v_sig[i] ),
754         .sel_o ( sp2v_sig_chk_raw[i] ),
755         .err_o ( sp2v_sig_err[i] )
756     );
757     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
758 end
759
760 assign state_we = sp2v_sig_chk[0];
761 assign key_full_we = sp2v_sig_chk[1];
762 assign key_dec_we = sp2v_sig_chk[2];
763
764 // Collect encoding errors.
765 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
766 // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
767 assign sp_enc_err_d = |sp2v_sig_err | sub_bytes_err |
key_expand_err;
768
769 // We need to register the collected error signal to avoid
circular loops in the cipher core
770
771 // controller related to out_valid_o and detecting errors in
state_we_o and sub_bytes_out_ack.
772 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_sp_enc_err
773     if (!rst_ni) begin
774         sp_enc_err_q <= 1'b0;
775     end else if (sp_enc_err_d) begin
776         sp_enc_err_q <= 1'b1;
777     end
778 end
779
780 ///////////////////////////////////////////////////
781 // Outputs //
782 ///////////////////////////////////////////////////
783
784 // The output of the last round is not stored into the state
register but forwarded directly.
785 assign state_o = add_round_key_out;
786
787 ///////////////////////////////////////////////////
788 // Assertions //
789 ///////////////////////////////////////////////////
790 // Typically assertions already contain this macro, which en-
sures that assertions are only compiled
791 // in simulation and FPV. However, we wrap the entire asser-
tion section with INC_ASSERT so that the
792 // helper logic below is not synthesized either, since that
could cause issues in DC.
793 `ifdef INC_ASSERT
794 //VCS coverage off
795 // pragma coverage off
796
797 // Create a lint error to reduce the risk of accidentally
disabling the masking.
798 `ASSERT_STATIC_LINT_ERROR(AesSecMaskingNonDefault, SecMask-
ing == 1)
799
800 // Cipher core masking requires a masked SBox and vice
versa.
801 `ASSERT_INIT(AesMaskedCoreAndSBox,
802 (SecMasking &&
803 (SecSBoxImpl == SBoxImplCanrightMasked ||
804 SecSBoxImpl == SBoxImplCanrightMaskedNoreuse ||
805 SecSBoxImpl == SBoxImplDom)) ||
806 (!SecMasking &&
807 (SecSBoxImpl == SBoxImplLut ||

```

```

808     SecSBoxImpl == SBoxImplCanright)))
809
810 // Signals used for assertions only.
811 logic prd_clearing_equals_output, un-
used_prd_clearing_equals_output;

812 assign prd_clearing_equals_output = (prd_clearing_state_i ==
add_round_key_out);
813 assign unused_prd_clearing_equals_output = prd_clear-
ing_equals_output;
814
815 // Ensure that the state register gets cleared with pseudo-
random data at the end of the last
816 // round. The following two scenarios are unlikely but not
illegal:
817 // 1. The newly loaded initial state matches the previous
output (the round counter is only
818 // cleared upon loading the new initial state).
819 // 2. The previous pseudo-random data is equal to the previ-
ous output.
820 // Otherwise, we must see an alert e.g. because the state
multiplexer got glitched.
821 `ASSERT(AesSecCmDataRegKeySca, (state_we == SP2V_HIGH) &&
822 ((key_len_i == AES_128 && u_aes_cipher_control.rnd_ctr
== 4'd10) ||
823 (key_len_i == AES_192 && u_aes_cipher_control.rnd_ctr
== 4'd12) ||
824 (key_len_i == AES_256 && u_aes_cipher_control.rnd_ctr
== 4'd14)) |>
825 (state_q != $past(add_round_key_out)) ||
826 (state_q == $past(state_init_i)) ||
827 $past(prd_clearing_equals_output) || alert_o)
828
829 if (SecMasking) begin : gen_sec_cm_key_masking_svas
830 // The number of clock cycles a regular AES round takes
- only used for assertions.
831 localparam int unsigned NumCyclesPerRound = (SecSBoxImpl
== SBoxImplDom) ? 5 : 1;
832 logic unused_param;
833 assign unused_param = (NumCyclesPerRound == 1);
834 // Ensure that SubBytes gets fresh PRD input for every
evaluation unless mask forcing is
835 // enabled. We effectively check that the PRNG has been
updated at least once within the
836 // last NumCyclesPerRound cycles. This also holds for
the very first round, as the PRNG
837 // is always updated in the last cycle of the IDLE state
and/or the first cycle of the
838 // INIT state.
839 `ASSERT(AesSecCmKeyMaskingPrdSubBytes,

840 sub_bytes_en == SP2V_HIGH && ($past(sub_bytes_en) ==
SP2V_LOW ||
841 ($past(sub_bytes_out_req) == SP2V_HIGH &&
842 $past(sub_bytes_out_ack) == SP2V_HIGH)) |>
843 $past(prd_sub_bytes_q) != $past(prd_sub_bytes_q,
NumCyclesPerRound + 1) ||
844 SecAllowForcingMasks && force_masks_i)
845
846 // Ensure that the PRNG has been updated between masking
the input and starting the first
847 // SubBytes evaluation/KeyExpand operation unless mask
forcing is enabled. For AES-256,
848 // we just spend 1 cycle in the INIT state and KeyExpand
isn't evaluating its S-Boxes,
849 // i.e., no fresh randomness is required. For the other
key lengths, KeyExpand evaluates
850 // its S-Boxes which takes NumCyclesPerRound cycles.
When computing the start key for
851 // decryption, the input isn't loaded and the PRNG is
thus not advanced.
852 `ASSERT(AesSecCmKeyMaskingInitialPrngUpdateSubBytes,
853 sub_bytes_en == SP2V_HIGH && $past(sub_bytes_en) ==
SP2V_LOW |>
854 (key_len_i == AES_256 &&
855 $past(prd_masking) != $past(prd_masking, 3)) ||
856 ((key_len_i == AES_128 || key_len_i == AES_192) &&
857 $past(prd_masking) != $past(prd_masking, Num-
CyclesPerRound + 2)) ||
858 (SecAllowForcingMasks && force_masks_i))
859 `ASSERT(AesSecCmKeyMaskingInitialPrngUpdateKeyExpand,
860 key_expand_en == SP2V_HIGH && $past(key_expand_en)
== SP2V_LOW |>
861 (key_len_i == AES_256 &&
862 $past(prd_masking) != $past(prd_masking, 3)) ||
863 ((key_len_i == AES_128 || key_len_i == AES_192) &&
864 $past(prd_masking) != $past(prd_masking, 2)) ||
865 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
866
867 // Ensure none of the state shares keeps being constant
during encryption/decryption
868 // unless mask forcing is enabled. Even though unlikely
it's not impossible that one
869 // share remains constant throughout one round. The SVAs
thus only fire if a share

```

```

870 // remains constant across two rounds.
871 for (genvar s = 0; s < NumShares; s++) begin :
gen_sec_cm_key_masking_share_svas
872 `ASSERT(AesSecCmKeyMaskingStateShare, state_we ==
SP2V_HIGH &&
873 (crypt_i == SP2V_HIGH || crypt_o == SP2V_HIGH) |>
874 state_q[s] != $past(state_q[s], NumCyclesPerRound)
||
875 $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
876 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
877 `ASSERT(AesSecCmKeyMaskingOutputShare,
878 (out_valid_o == SP2V_HIGH && $past(out_valid_o) ==
SP2V_LOW) &&
879 (crypt_o == SP2V_HIGH) |>
880 $past(state_o[s]) != $past(state_q[s], NumCycles-
PerRound) ||
881 $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
882 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
883 end
884 end
885
886 // Make sure the output of the masking PRNG is properly ex-
tracted without creating overlaps
887 // in the data input masks, or between the PRD fed to the
key expand module and SubBytes.
888 if (WidthPRDSBox > 8) begin : gen_prd_extract_assert
889 // For one row of the state matrix, extract the WidthPRDS-
Box-8 MSBs of the per-S-Box PRD from
890 // the PRNG output.
891 function automatic logic [3:0][(WidthPRDSBox-8)-1:0]
aes_prd_get_msbs(
892 logic [(4*WidthPRDSBox)-1:0] in
893 );
894 logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs;
895 for (int i = 0; i < 4; i++) begin
896 prd_msbs[i] = in[(i*WidthPRDSBox) + 8 +: (WidthPRDS-
Box-8)];
897 end
898 return prd_msbs;
899 endfunction
900
901 // For one row of the state matrix, undo the extraction of
LSBs and MSBs of the per-S-Box PRD
902 // from the PRNG output. This can be used to verify proper
extraction (no overlap of output
903 // masks and PRD for masked Canright S-Box implementa-
tions, no unused PRNG output).
904 function automatic logic [4*WidthPRDSBox-1:0] aes_prd_con-
cat_bits(
905 logic [3:0] [7:0] prd_lsb,
906 logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs
907 );
908 logic [(4*WidthPRDSBox)-1:0] prd;
909 for (int i = 0; i < 4; i++) begin
910 prd[(i*WidthPRDSBox) +: WidthPRDSBox] = {prd_msbs[i],
prd_lsb[i]};
911 end
912 return prd;
913 endfunction
914
915 // Check for correct extraction of masking PRNG output
without overlaps.
916 logic [WidthPRDMasking-1:0] unused_prd_masking;
917 logic [3:0][3:0][(WidthPRDSBox-8)-1:0] unused_prd_msbs;
918 for (genvar i = 0; i < 4; i++) begin : gen_unused_prd_msbs
919 assign unused_prd_msbs[i] =
aes_prd_get_msbs(prd_masking[i * WidthPRDRow +: WidthPRDRow]);
920 end
921 for (genvar i = 0; i < 4; i++) begin : gen_un-
used_prd_masking
922 assign unused_prd_masking[i * WidthPRDRow +: Width-
PRDRow] =
923 aes_prd_concat_bits(data_in_mask[i], un-
used_prd_msbs[i]);
924 end
925 assign unused_prd_masking[WidthPRDMasking-1 -: Width-
PRDRow] = prd_key_expand;
926 `ASSERT(AesMskgPrdExtraction, prd_masking == un-
used_prd_masking)
927 end
928 //VCS coverage on
929 // pragma coverage on
930 `endif
931
932 endmodule

```

7. lowRISC_opentitan/hw/ipl/aes/rtl/aes_control.sv (646 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for details.

```

3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES main control
6 //
7 // This module controls the interplay of input/output registers and the AES cipher core.
8
9 `include "prim_assert.sv"
10
11 module aes_control
12   import aes_pkg::*;
13   import aes_reg_pkg::*;
14   #(
15     parameter bit AESGCMEnable = 0,
16     parameter bit SecMasking = 0,
17     parameter int unsigned SecStartTriggerDelay = 0
18   ) (
19     input logic clk_i,
20     input logic rst_ni,
21
22     // Main control signals
23     input logic ctrl_qe_i,
24     output logic ctrl_we_o,
25     input logic ctrl_phase_i,
26     input logic ctrl_err_storage_i,
27     input aes_op_e op_i,
28     input aes_mode_e mode_i,
29     input ciph_op_e cipher_op_i,
30     input logic sideload_i,
31     input prs_rate_e prng_reseed_rate_i,
32     input logic manual_operation_i,
33     input logic key_touch_forces_reseed_i,
34     input logic ctrl_gcm_qe_i,
35     output logic ctrl_gcm_we_o,
36     input logic ctrl_gcm_phase_i,
37     output logic gcm_init_done_o,
38     input gcm_phase_e gcm_phase_i,
39     input logic start_i,
40     input logic key_iv_data_in_clear_i,
41     input logic data_out_clear_i,
42     input logic prng_reseed_i,
43     input logic mux_sel_err_i,
44     input logic sp_enc_err_i,
45     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
46     input logic alert_fatal_i,
47     output logic alert_o,
48
49     // I/O register read/write enables
50     input logic key_sideload_valid_i,
51     input logic [NumRegsKey-1:0] key_init_qe_i [NumShares-
Key],
52     input logic [NumRegsIv-1:0] iv_qe_i,
53     input logic [NumRegsData-1:0] data_in_qe_i,
54     input logic [NumRegsData-1:0] data_out_re_i,
55     output logic data_in_we_o,
56     output data_out_sel_e data_out_sel_o,
57     output sp2v_e data_out_we_o,
58
59     // Previous input data register
60     output dip_sel_e data_in_prev_sel_o,
61     output sp2v_e data_in_prev_we_o,
62
63     // Cipher I/O muxes
64     output si_sel_e state_in_sel_o,
65     output add_si_sel_e add_state_in_sel_o,
66     output add_so_sel_e add_state_out_sel_o,
67
68     // Counter
69     output sp2v_e ctr_inc32_o,
70     output sp2v_e ctr_incr_o,
71     input sp2v_e ctr_ready_i,
72     input sp2v_e [NumSlicesCtr-1:0] ctr_we_i,
73
74     // Cipher core control and sync
75     output sp2v_e cipher_in_valid_o,
76     input sp2v_e cipher_in_ready_i,
77     input sp2v_e cipher_out_valid_i,
78     output sp2v_e cipher_out_ready_o,
79     output sp2v_e cipher_crypt_o,
80     input sp2v_e cipher_crypt_i,
81     output sp2v_e cipher_dec_key_gen_o,
82     input sp2v_e cipher_dec_key_gen_i,
83     output logic cipher_prng_reseed_o,
84     input logic cipher_prng_reseed_i,
85     output logic cipher_key_clear_o,
86     input logic cipher_key_clear_i,
87     output logic cipher_data_out_clear_o,
88     input logic cipher_data_out_clear_i,
89
90     // GHASH control and sync
91     output sp2v_e ghash_in_valid_o,
92     input sp2v_e ghash_in_ready_i,
93     input sp2v_e ghash_out_valid_i,
94     output sp2v_e ghash_out_ready_o,
95     output sp2v_e ghash_load_hash_subkey_o,
96
97     // Initial key registers
98     output key_init_sel_e key_init_sel_o,
99     output sp2v_e [NumRegsKey-1:0] key_init_we_o [NumShares-
Key],
100

```

```

101 // IV registers
102 output iv_sel_e iv_sel_o,
103 output sp2v_e [NumSlicesCtr-1:0] iv_we_o,
104
105 // Pseudo-random number generator interface
106 output logic prng_update_o,
107 output logic prng_reseed_req_o,
108 input logic prng_reseed_ack_i,
109
110 // Trigger register
111 output logic start_o,
112 output logic start_we_o,
113 output logic key_iv_data_in_clear_o,
114 output logic key_iv_data_in_clear_we_o,
115 output logic data_out_clear_o,
116 output logic data_out_clear_we_o,
117 output logic prng_reseed_o,
118 output logic prng_reseed_we_o,
119
120 // Status register
121 output logic idle_o,
122 output logic idle_we_o,
123 output logic stall_o,
124 output logic stall_we_o,
125 input logic output_lost_i,
126 output logic output_lost_o,
127 output logic output_lost_we_o,
128 output logic output_valid_o,
129 output logic output_valid_we_o,
130 output logic input_ready_o,
131 output logic input_ready_we_o,
132 );
133
134 // Optional delay of manual start trigger
135 logic start_trigger;
136
137 // Create a lint error to reduce the risk of accidentally
enabling this feature.
138 `ASSERT_STATIC_LINT_ERROR(AesSecStartTriggerDelayNonDefault,
SecStartTriggerDelay == 0)
139
140 if (SecStartTriggerDelay > 0) begin : gen_start_delay
141   // Delay the manual start trigger input for SCA measure-
ments.
142   localparam int unsigned WidthCounter = $clog2(SecStartTriggerDelay+1);
143   logic [WidthCounter-1:0] count_d, count_q;
144
145   // Clear counter when input goes low. Keep value if the
specified
146   delay is reached.
147   assign count_d = !start_i ? '0 :
start_trigger ? count_q : count_q +
1'b1;
148   assign start_trigger = (count_q == SecStartTrigger-
Delay[WidthCounter-1:0]) ? 1'b1 : 1'b0;
149
150   always_ff @(posedge clk_i or negedge rst_ni) begin
151     if (!rst_ni) begin
152       count_q <= '0;
153     end else begin
154       count_q <= count_d;
155     end
156   end
157
158 end else begin : gen_no_start_delay
159   // Directly forward the manual start trigger input.
160   assign start_trigger = start_i;
161 end
162
163 // Signals
164 sp2v_e ctr_ready;
165 sp2v_e [NumSlicesCtr-1:0] ctr_we;
166 sp2v_e cipher_in_ready;
167 sp2v_e cipher_out_valid;
168 sp2v_e cipher_crypt;
169 sp2v_e cipher_dec_key_gen;
170 sp2v_e ghash_in_ready;
171 sp2v_e ghash_out_valid;
172 logic mux_sel_err;
173 logic mr_err;
174 logic sp_enc_err;
175
176 // Sparsified FSM signals. These are needed for connecting
the individual bits of the Sp2V
177 // signals to the single-rail FSMs.
178 logic [Sp2VWidth-1:0] sp_data_out_we;
179 logic [Sp2VWidth-1:0] sp_data_in_prev_we;
180 logic [Sp2VWidth-1:0] sp_ctr_inc32;
181 logic [Sp2VWidth-1:0] sp_ctr_incr;
182 logic [Sp2VWidth-1:0] sp_ctr_ready;
183 logic [Sp2VWidth-1:0] sp_cipher_in_valid;
184 logic [Sp2VWidth-1:0] sp_cipher_in_ready;
185 logic [Sp2VWidth-1:0] sp_cipher_out_valid;
186 logic [Sp2VWidth-1:0] sp_cipher_out_ready;
187 logic [Sp2VWidth-1:0] sp_in_cipher_crypt;
188 logic [Sp2VWidth-1:0] sp_out_cipher_crypt;
189 logic [Sp2VWidth-1:0] sp_in_cipher_dec_key_gen;
190 logic [Sp2VWidth-1:0] sp_out_cipher_dec_key_gen;
191 logic [Sp2VWidth-1:0] sp_ghash_in_valid;
192 logic [Sp2VWidth-1:0] sp_ghash_in_ready;

```



```

193 logic [Sp2VWidth-1:0] sp_ghash_out_valid;
194 logic [Sp2VWidth-1:0] sp_ghash_out_ready;
195 logic [Sp2VWidth-1:0] sp_ghash_load_hash_subkey;
196
197 // Multi-rail signals. These are outputs of the single-rail
FSMs and need combining.
198 logic [Sp2VWidth-1:0] mr_ctrl_we;
199 logic [Sp2VWidth-1:0] mr_ctrl_gcm_we;
200 logic [Sp2VWidth-1:0] mr_gcm_init_done;
201 logic [Sp2VWidth-1:0] mr_alert;
202 logic [Sp2VWidth-1:0] mr_data_in_we;
203 data_out_sel_e [Sp2VWidth-1:0] mr_data_out_sel;
204 dip_sel_e [Sp2VWidth-1:0] mr_data_in_prev_sel;
205 si_sel_e [Sp2VWidth-1:0] mr_state_in_sel;
206 add_si_sel_e [Sp2VWidth-1:0] mr_add_state_in_sel;
207 add_so_sel_e [Sp2VWidth-1:0] mr_add_state_out_sel;
208 logic [Sp2VWidth-1:0] mr_cipher_prng_reseed;
209 logic [Sp2VWidth-1:0] mr_cipher_key_clear;
210 logic [Sp2VWidth-1:0] mr_cipher_data_out_clear;
211 key_init_sel_e [Sp2VWidth-1:0] mr_key_init_sel;
212 iv_sel_e [Sp2VWidth-1:0] mr_iv_sel;
213 logic [Sp2VWidth-1:0] mr_prng_update;
214 logic [Sp2VWidth-1:0] mr_prng_reseed_req;
215 logic [Sp2VWidth-1:0] mr_start_we;
216 logic [Sp2VWidth-1:0] mr_key_iv_data_in_clear_we;
217 logic [Sp2VWidth-1:0] mr_data_out_clear_we;
218 logic [Sp2VWidth-1:0] mr_prng_reseed;
219 logic [Sp2VWidth-1:0] mr_prng_reseed_we;
220 logic [Sp2VWidth-1:0] mr_idle;
221 logic [Sp2VWidth-1:0] mr_idle_we;
222 logic [Sp2VWidth-1:0] mr_stall;
223 logic [Sp2VWidth-1:0] mr_stall_we;
224 logic [Sp2VWidth-1:0] mr_output_lost;
225 logic [Sp2VWidth-1:0] mr_output_lost_we;
226 logic [Sp2VWidth-1:0] mr_output_valid;
227 logic [Sp2VWidth-1:0] mr_output_valid_we;
228 logic [Sp2VWidth-1:0] mr_input_ready;
229 logic [Sp2VWidth-1:0] mr_input_ready_we;
230
231 // To ease interfacing with the individual FSM rails, some
signals need to be converted to packed
232 // arrays.
233 logic [Sp2VWidth-1:0][NumSharesKey-1:0][NumRegs-
Key-1:0] int_key_init_we;
234 logic [NumSharesKey-1:0][NumRegsKey-1:0]
[Sp2VWidth-1:0] log_key_init_we;
235 logic [NumSharesKey-1:0][NumRegs-
Key-1:0] int_key_init_qe;
236 for (genvar s = 0; s < NumSharesKey; s++) begin :
gen_conv_key_init_wqe_shares
237 for (genvar i = 0; i < NumRegsKey; i++) begin :
gen_conv_key_init_wqe_regs
238 assign int_key_init_qe[s][i] = key_init_qe[i][s];
239 for (genvar j = 0; j < Sp2VWidth; j++) begin :
gen_conv_key_init_wqe_log
240 assign log_key_init_we[s][i][j] = int_key_init_we[j]
[s][i];
241 end
242 assign key_init_we_o[s][i] = sp2v_e'(log_key_init_we[s]
[i]);
243 end
244 end
245 logic [Sp2VWidth-1:0][NumSlicesCtr-1:0]
int_ctr_we;
246 logic [NumSlicesCtr-1:0][Sp2VWidth-1:0]
log_ctr_we;
247 logic [Sp2VWidth-1:0][NumSlicesCtr-1:0]
int_iv_we;
248 logic [NumSlicesCtr-1:0][Sp2VWidth-1:0]
log_iv_we;
249 for (genvar i = 0; i < NumSlicesCtr; i++) begin :
gen_conv_ctr_iv_we_slices
250 assign log_ctr_we[i] = {ctr_we[i]};
251 for (genvar j = 0; j < Sp2VWidth; j++) begin :
gen_conv_ctr_iv_we_log
252 assign int_ctr_we[j][i] = log_ctr_we[i][j];
253 assign log_iv_we[i][j] = int_iv_we[j][i];
254 end
255 assign iv_we_o[i] = sp2v_e'(log_iv_we[i]);
256 end
257
258 // FSM
259 //
260 //
261
262 // Convert sp2v_e signals to sparsified inputs.
263 assign sp_ctr_ready = {ctr_ready};
264 assign sp_cipher_in_ready = {cipher_in_ready};
265 assign sp_cipher_out_valid = {cipher_out_valid};
266 assign sp_in_cipher_crypt = {cipher_crypt};
267 assign sp_in_cipher_dec_key_gen = {cipher_dec_key_gen};
268 assign sp_ghash_in_ready = {ghash_in_ready};
269 assign sp_ghash_out_valid = {ghash_out_valid};
270
271 // SEC_CM: MAIN.FSM.REDUN
272 // For every bit in the Sp2V signals, one separate rail is
instantiated. The inputs and outputs
273 // of every rail are buffered to prevent aggressive syntheses-
is optimizations.
274 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
275 if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p

```

```

276 aes_control_fsm_p #(
277 .AESGCMEnable ( AESGCMEnable ),
278 .SecMasking ( SecMasking )
279 ) u_aes_control_fsm_i (
280 .clk_i
281 .rst_ni
282 ( rst_ni ),
283 .ctrl_qe_i
284 ( ctrl_qe_i ),
285 .ctrl_we_o ( mr_c-
trl_we[i] ), // AND-combine
286 .ctrl_phase_i
287 ( ctrl_phase_i ),
288 .ctrl_err_storage_i ( ctrl_err_stor-
age_i ),
289 .op_i
290 ( op_i ),
291 .mode_i
292 ( mode_i ),
293 .cipher_op_i
294 ( cipher_op_i ),
295 .sideload_i ( side-
load_i ),
296 .prng_reseed_rate_i ( prng_re-
seed_rate_i ),
297 .manual_operation_i ( manu-
al_operation_i ),
298 .key_touch_forces_reseed_i
299 ( key_touch_forces_reseed_i ),
300 .ctrl_gcm_qe_i
301 ( ctrl_gcm_qe_i ),
302 .ctrl_gcm_we_o ( mr_c-
trl_gcm_we[i] ), // AND-combine
303 .ctrl_gcm_phase_i
304 ( ctrl_gcm_phase_i ),
305 .gcm_init_done_o ( mr_gcm_in-
it_done[i] ), // AND-combine
306 .gcm_phase_i
307 ( gcm_phase_i ),
308 .start_i
309 ( start_trigger ),
310 .key_iv_data_in_clear_i
311 ( key_iv_data_in_clear_i ),
312 .data_out_clear_i
313 ( data_out_clear_i ),
314 .prng_reseed_i ( prng_re-
seed_i ),
315 .mux_sel_err_i
316 ( mux_sel_err ),
317 .sp_enc_err_i
318 ( sp_enc_err ),
319 .lc_escalate_en_i ( lc_escal-
ate_en_i ),
320 .alert_fatal_i
321 ( alert_fatal_i ),
322 .alert_o
323 ( mr_alert[i] ), // OR-combine
324 .key_sideload_valid_i ( key_side-
load_valid_i ),
325 .key_init_qe_i ( int_key_in-
it_qe ),
326 .iv_qe_i
327 ( iv_qe_i ),
328 .data_in_qe_i
329 ( data_in_qe_i ),
330 .data_out_re_i
331 ( data_out_re_i ),
332 .data_in_we_o
333 ( mr_data_in_we[i] ), // AND-combine
334 .data_out_sel_o
335 ( mr_data_out_sel[i] ), // OR-combine
336 .data_out_we_o
337 ( sp_data_out_we[i] ), // Sparsified
338 .data_in_prev_sel_o
339 ( mr_data_in_prev_sel[i] ), // OR-combine
340 .data_in_prev_we_o
341 ( sp_data_in_prev_we[i] ), // Sparsified
342 .state_in_sel_o
343 ( mr_state_in_sel[i] ), // OR-combine
344 .add_state_in_sel_o
345 ( mr_add_state_in_sel[i] ), // OR-combine
346 .add_state_out_sel_o
347 ( mr_add_state_out_sel[i] ), // OR-combine
348 .ctr_inc32_o
349 ( sp_ctr_inc32[i] ), // Sparsified
350 .ctr_incr_o ( sp_ctr_in-
cr[i] ), // Sparsified
351 .ctr_ready_i
352 ( sp_ctr_ready[i] ), // Sparsified
353 .ctr_we_i
354 ( int_ctr_we[i] ), // Sparsified
355 .cipher_in_valid_o
356 ( sp_cipher_in_valid[i] ), // Sparsified
357 .cipher_in_ready_i

```

```

( sp_cipher_in_ready[i] ), // Sparsified
332 .cipher_out_valid_i
( sp_cipher_out_valid[i] ), // Sparsified
333 .cipher_out_ready_o
( sp_cipher_out_ready[i] ), // Sparsified
334 .cipher_crypt_o
( sp_out_cipher_crypt[i] ), // Sparsified
335 .cipher_crypt_i
( sp_in_cipher_crypt[i] ), // Sparsified
336 .cipher_dec_key_gen_o
( sp_out_cipher_dec_key_gen[i] ), // Sparsified
337 .cipher_dec_key_gen_i
( sp_in_cipher_dec_key_gen[i] ), // Sparsified
338 .cipher_prng_reseed_o
( mr_cipher_prng_reseed[i] ), // OR-combine
339 .cipher_prng_reseed_i
( cipher_prng_reseed_i ),
340 .cipher_key_clear_o
( mr_cipher_key_clear[i] ), // OR-combine
341 .cipher_key_clear_i
( cipher_key_clear_i ),
342 .cipher_data_out_clear_o
( mr_cipher_data_out_clear[i] ), // OR-combine
343 .cipher_data_out_clear_i
( cipher_data_out_clear_i ),
344
345 .ghash_in_valid_o
( sp_ghash_in_valid[i] ), // Sparsified
346 .ghash_in_ready_i
( sp_ghash_in_ready[i] ), // Sparsified
347 .ghash_out_valid_i
( sp_ghash_out_valid[i] ), // Sparsified
348 .ghash_out_ready_o
( sp_ghash_out_ready[i] ), // Sparsified
349 .ghash_load_hash_subkey_o
( sp_ghash_load_hash_subkey[i] ), // Sparsified
350
351 .key_init_sel_o ( mr_key_in-
it_sel[i] ), // OR-combine
352 .key_init_we_o ( int_key_in-
it_we[i] ), // Sparsified
353
354 .iv_sel_o
( mr_iv_sel[i] ), // OR-combine
355 .iv_we_o
( int_iv_we[i] ), // Sparsified
356
357 .prng_update_o
( mr_prng_update[i] ), // OR-combine
358 .prng_reseed_req_o
( mr_prng_reseed_req[i] ), // OR-combine
359 .prng_reseed_ack_i ( prng_re-
seed_ack_i ),
360
361 .start_we_o
( mr_start_we[i] ), // OR-combine
362 .key_iv_data_in_clear_we_o
( mr_key_iv_data_in_clear_we[i] ), // AND-combine
363 .data_out_clear_we_o
( mr_data_out_clear_we[i] ), // AND-combine
364 .prng_reseed_o
( mr_prng_reseed[i] ), // OR-combine
365 .prng_reseed_we_o
( mr_prng_reseed_we[i] ), // OR-combine
366
367 .idle_o
( mr_idle[i] ), // AND-combine
368 .idle_we_o
( mr_idle_we[i] ), // AND-combine
369 .stall_o
( mr_stall[i] ), // AND-combine
370 .stall_we_o
( mr_stall_we[i] ), // AND-combine
371 .output_lost_i ( out-
put_lost_i ), // AND-combine
372 .output_lost_o ( mr_out-
put_lost[i] ), // AND-combine
373 .output_lost_we_o ( mr_out-
put_lost_we[i] ), // AND-combine
374 .output_valid_o ( mr_out-
put_valid[i] ), // AND-combine
375 .output_valid_we_o ( mr_out-
put_valid_we[i] ), // AND-combine
376 .input_ready_o ( mr_in-
put_ready[i] ), // AND-combine
377 .input_ready_we_o ( mr_in-
put_ready_we[i] ), // AND-combine
378 );
379 end else begin : gen_fsm_n
380 aes_control_fsm_n # (
381 .AESGCMEEnable ( AESGCMEEnable ),
382 .SecMasking ( SecMasking )
383 ) u_aes_control_fsm_i (
384 .clk_i
( clk_i ),
385 .rst_ni
( rst_ni ),
386
387 .ctrl_qe_i
( ctrl_qe_i ),
388 .ctrl_we_o ( mr_c-

```

```

trl_we[i] ), // AND-combine
389 .ctrl_phase_i
( ctrl_phase_i ),
390 .ctrl_err_storage_i ( ctrl_err_stor-
age_i ),
391 .op_i
( op_i ),
392 .mode_i
( mode_i ),
393 .cipher_op_i
( cipher_op_i ),
394 .sideload_i ( side-
load_i ),
395 .prng_reseed_rate_i ( prng_re-
seed_rate_i ),
396 .manual_operation_i ( manu-
al_operation_i ),
397 .key_touch_forces_reseed_i
( key_touch_forces_reseed_i ),
398 .ctrl_gcm_qe_i
( ctrl_gcm_qe_i ),
399 .ctrl_gcm_we_o ( mr_c-
trl_gcm_we[i] ), // AND-combine
400 .ctrl_gcm_phase_i
( ctrl_gcm_phase_i ),
401 .gcm_init_done_o ( mr_gcm_in-
it_done[i] ), // AND-combine
402 .gcm_phase_i
( gcm_phase_i ),
403 .start_i
( start_trigger ),
404 .key_iv_data_in_clear_i
( key_iv_data_in_clear_i ),
405 .data_out_clear_i
( data_out_clear_i ),
406 .prng_reseed_i ( prng_re-
seed_i ),
407 .mux_sel_err_i
( mux_sel_err ),
408 .sp_enc_err_i
( sp_enc_err ),
409 .lc_escalate_en_i ( lc_escal-
ate_en_i ),
410 .alert_fatal_i
( alert_fatal_i ),
411 .alert_o
( mr_alert[i] ), // OR-combine
412
413 .key_sideload_valid_i ( key_side-
load_valid_i ),
414 .key_init_qe_i ( int_key_in-
it_qe ),
415 .iv_qe_i
( iv_qe_i ),
416 .data_in_qe_i
( data_in_qe_i ),
417 .data_out_re_i
( data_out_re_i ),
418 .data_in_we_o
( mr_data_in_we[i] ), // AND-combine
419 .data_out_sel_o
( mr_data_out_sel[i] ), // OR-combine
420 .data_out_we_no
( sp_data_out_we[i] ), // Sparsified
421
422 .data_in_prev_sel_o
( mr_data_in_prev_sel[i] ), // OR-combine
423 .data_in_prev_we_no
( sp_data_in_prev_we[i] ), // Sparsified
424
425 .state_in_sel_o
( mr_state_in_sel[i] ), // OR-combine
426 .add_state_in_sel_o
( mr_add_state_in_sel[i] ), // OR-combine
427 .add_state_out_sel_o
( mr_add_state_out_sel[i] ), // OR-combine
428
429 .ctr_inc32_no
( sp_ctr_inc32[i] ), // Sparsified
430 .ctr_incr_no ( sp_ctr_in-
cr[i] ), // Sparsified
431 .ctr_ready_ni
( sp_ctr_ready[i] ), // Sparsified
432 .ctr_we_ni
( int_ctr_we[i] ), // Sparsified
433
434 .cipher_in_valid_no
( sp_cipher_in_valid[i] ), // Sparsified
435 .cipher_in_ready_ni
( sp_cipher_in_ready[i] ), // Sparsified
436 .cipher_out_valid_ni
( sp_cipher_out_valid[i] ), // Sparsified
437 .cipher_out_ready_no
( sp_cipher_out_ready[i] ), // Sparsified
438 .cipher_crypt_no
( sp_out_cipher_crypt[i] ), // Sparsified
439 .cipher_crypt_ni
( sp_in_cipher_crypt[i] ), // Sparsified
440 .cipher_dec_key_gen_no
( sp_out_cipher_dec_key_gen[i] ), // Sparsified
441 .cipher_dec_key_gen_ni

```

```

( sp_in_cipher_dec_key_gen[i] ), // Sparsified
442   .cipher_prng_reseed_o
( mr_cipher_prng_reseed[i] ), // OR-combine
443   .cipher_prng_reseed_i
( cipher_prng_reseed_i ),
444   .cipher_key_clear_o
( mr_cipher_key_clear[i] ), // OR-combine
445   .cipher_key_clear_i
( cipher_key_clear_i ),
446   .cipher_data_out_clear_o
( mr_cipher_data_out_clear[i] ), // OR-combine
447   .cipher_data_out_clear_i
( cipher_data_out_clear_i ),
448
449   .ghash_in_valid_no
( sp_ghash_in_valid[i] ), // Sparsified
450   .ghash_in_ready_no
( sp_ghash_in_ready[i] ), // Sparsified
451   .ghash_out_valid_no
( sp_ghash_out_valid[i] ), // Sparsified
452   .ghash_out_ready_no
( sp_ghash_out_ready[i] ), // Sparsified
453   .ghash_load_hash_subkey_no
( sp_ghash_load_hash_subkey[i] ), // Sparsified
454
455   .key_init_sel_o      ( mr_key_in-
it_sel[i] ), // OR-combine
456   .key_init_we_no      ( int_key_in-
it_we[i] ), // Sparsified
457
458   .iv_sel_o
( mr_iv_sel[i] ), // OR-combine
459   .iv_we_no
( int_iv_we[i] ), // Sparsified
460
461   .prng_update_o
( mr_prng_update[i] ), // OR-combine
462   .prng_reseed_req_o
( mr_prng_reseed_req[i] ), // OR-combine
463   .prng_reseed_ack_i   ( prng_re-
seed_ack_i ),
464
465   .start_we_o
( mr_start_we[i] ), // OR-combine
466   .key_iv_data_in_clear_we_o
( mr_key_iv_data_in_clear_we[i] ), // AND-combine
467   .data_out_clear_we_o
( mr_data_out_clear_we[i] ), // AND-combine
468   .prng_reseed_o
( mr_prng_reseed[i] ), // OR-combine
469   .prng_reseed_we_o
( mr_prng_reseed_we[i] ), // OR-combine
470
471   .idle_o
( mr_idle[i] ), // AND-combine
472   .idle_we_o
( mr_idle_we[i] ), // AND-combine
473   .stall_o
( mr_stall[i] ), // AND-combine
474   .stall_we_o
( mr_stall_we[i] ), // AND-combine
475   .output_lost_i      ( out-
put_lost_i ), // AND-combine
476   .output_lost_o      ( mr_out-
put_lost[i] ), // AND-combine
477   .output_lost_we_o   ( mr_out-
put_lost_we[i] ), // AND-combine
478   .output_valid_o      ( mr_out-
put_valid[i] ), // AND-combine
479   .output_valid_we_o   ( mr_out-
put_valid_we[i] ), // AND-combine
480   .input_ready_o      ( mr_in-
put_ready[i] ), // AND-combine
481   .input_ready_we_o   ( mr_in-
put_ready_we[i] ), // AND-combine
482   );
483   end
484   end
485
486   // Convert sparsified outputs to sp2v_e type.
487   assign data_out_we_o = sp2v_e'(sp_data_out_we);
488   assign data_in_prev_we_o =
sp2v_e'(sp_data_in_prev_we);
489   assign ctr_inc32_o = sp2v_e'(sp_ctr_inc32);
490   assign ctr_incr_o = sp2v_e'(sp_ctr_incr);
491   assign cipher_in_valid_o =
sp2v_e'(sp_cipher_in_valid);
492   assign cipher_out_ready_o =
sp2v_e'(sp_cipher_out_ready);
493   assign cipher_crypt_o =
sp2v_e'(sp_out_cipher_crypt);
494   assign cipher_dec_key_gen_o =
sp2v_e'(sp_out_cipher_dec_key_gen);
495   assign ghash_in_valid_o =
sp2v_e'(sp_ghash_in_valid);
496   assign ghash_out_ready_o =
sp2v_e'(sp_ghash_out_ready);
497   assign ghash_load_hash_subkey_o =
sp2v_e'(sp_ghash_load_hash_subkey);
498
499   // Combine single-bit FSM outputs.

```

```

500   // OR: One bit is sufficient to drive the corresponding out-
put bit high.
501   assign alert_o = |mr_alert;
502   assign cipher_prng_reseed_o = |mr_cipher_prng_reseed;
503   assign cipher_key_clear_o = |mr_cipher_key_clear;
504   assign cipher_data_out_clear_o = |
mr_cipher_data_out_clear;
505   assign prng_update_o = |mr_prng_update;
506   assign prng_reseed_req_o = |mr_prng_reseed_req;
507   assign start_we_o = |mr_start_we;
508   assign prng_reseed_o = |mr_prng_reseed;
509   assign prng_reseed_we_o = |mr_prng_reseed_we;
510
511   // AND: Only if all bits are high, the corresponding action
should be triggered.
512   assign ctrl_we_o = &mr_ctrl_we;
513   assign ctrl_gcm_we_o = &mr_ctrl_gcm_we;
514   assign gcm_init_done_o = &mr_gcm_init_done;
515   assign data_in_we_o = &mr_data_in_we;
516   assign key_iv_data_in_clear_we_o =
&mr_key_iv_data_in_clear_we;
517   assign data_out_clear_we_o = &mr_data_out_clear_we;
518   assign idle_o = &mr_idle;
519   assign idle_we_o = &mr_idle_we;
520   assign stall_o = &mr_stall;
521   assign stall_we_o = &mr_stall_we;
522   assign output_lost_o = &mr_output_lost;
523   assign output_lost_we_o = &mr_output_lost_we;
524   assign output_valid_o = &mr_output_valid;
525   assign output_valid_we_o = &mr_output_valid_we;
526   assign input_ready_o = &mr_input_ready;
527   assign input_ready_we_o = &mr_input_ready_we;
528
529   // Combine multi-bit, sparse FSM outputs. We simply OR them
together. If the FSMs don't provide
530   // the same outputs, two cases are possible:
531   // - An invalid encoding results: A downstream checker will
fire, see mux_sel_err_i.
532   // - A valid encoding results: The outputs are compared be-
low to cover this case, see mr_err;
533   always_comb begin : combine_sparse_signals
534     data_out_sel_o = data_out_sel_e'({DataOutSel-
Width{1'b0}});
535     data_in_prev_sel_o = dip_sel_e'({DIPSelWidth{1'b0}});
536     state_in_sel_o = si_sel_e'({SISelWidth{1'b0}});
537     add_state_in_sel_o = add_si_sel_e'({AddSISel-
Width{1'b0}});
538     add_state_out_sel_o = add_so_sel_e'({AddSOSel-
Width{1'b0}});
539     key_init_sel_o = key_in-
it_sel_e'({KeyInitSelWidth{1'b0}});
540     iv_sel_o = iv_sel_e'({IVSelWidth{1'b0}});
541     mr_err = 1'b0;
542
543     for (int i = 0; i < Sp2VWidth; i++) begin
544       data_out_sel_o =
data_out_sel_e'({data_out_sel_o | {mr_data_out_sel[i]}});
545       data_in_prev_sel_o =
dip_sel_e'({data_in_prev_sel_o | {mr_data_in_prev_sel[i]}});
546       state_in_sel_o =
si_sel_e'({state_in_sel_o | {mr_state_in_sel[i]}});
547       add_state_in_sel_o =
add_si_sel_e'({add_state_in_sel_o | {mr_add_state_in_sel[i]}});
548       add_state_out_sel_o =
add_so_sel_e'({add_state_out_sel_o | {mr_add_state_out_sel[i]}});
549       key_init_sel_o = key_in-
it_sel_e'({key_init_sel_o | {mr_key_init_sel[i]}});
550       iv_sel_o =
iv_sel_e'({iv_sel_o | {mr_iv_sel[i]}});
551     end
552
553     for (int i = 0; i < Sp2VWidth; i++) begin
554       if (data_out_sel_o != mr_data_out_sel[i] ||
555         data_in_prev_sel_o != mr_data_in_prev_sel[i] ||
556         state_in_sel_o != mr_state_in_sel[i] ||
557         add_state_in_sel_o != mr_add_state_in_sel[i] ||
558         add_state_out_sel_o != mr_add_state_out_sel[i] ||
559         key_init_sel_o != mr_key_init_sel[i] ||
560         iv_sel_o != mr_iv_sel[i]) begin
561         mr_err = 1'b1;
562       end
563     end
564   end
565
566   // Collect errors in mux selector signals.
567   assign mux_sel_err = mux_sel_err_i | mr_err;
568
569   ///////////////////////////////////////////////////
570   // Sparsely Encoded Signals //
571   ///////////////////////////////////////////////////
572
573   // SEC_CM: CTRL.SPARSE
574   // We use sparse encodings for various critical signals and
must ensure that:
575   // 1. The synthesis tool doesn't optimize away the sparse
encoding.
576   // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
577   // 3. if a sparse signal takes on an invalid value.
578   // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again

```



```

579 // This is achieved by driving the control FSM into the
terminal error state whenever any
580 // sparsely encoded signal becomes invalid.
581 //
582 // If any sparsely encoded signal becomes invalid, the con-
troller further immediately de-asserts
583 // data_out_we_o and other write-enable signals to prevent
any data from being released.
584
585 // We use vectors of sparsely encoded signals to reduce code
duplication.
586 localparam int unsigned NumSp2VVSig = 7 + NumSlicesCtr;
587 sp2v_e [NumSp2VVSig-1:0] sp2v_sig;
588 sp2v_e [NumSp2VVSig-1:0] sp2v_sig_chk;
589 logic [NumSp2VVSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
590 logic [NumSp2VVSig-1:0] sp2v_sig_err;
591
592 assign sp2v_sig[0] = cipher_in_ready_i;
593 assign sp2v_sig[1] = cipher_out_valid_i;
594 assign sp2v_sig[2] = cipher_crypt_i;
595 assign sp2v_sig[3] = cipher_dec_key_gen_i;
596 assign sp2v_sig[4] = ghash_in_ready_i;
597 assign sp2v_sig[5] = ghash_out_valid_i;
598 assign sp2v_sig[6] = ctr_ready_i;
599 for (genvar i = 0; i < NumSlicesCtr; i++) begin :
gen_use_ctr_we_i
600     assign sp2v_sig[7+i] = ctr_we_i[i];
601 end
602
603 // All signals inside sp2v_sig are driven and consumed by
multi-rail FSMs.
604 localparam bit [NumSp2VVSig-1:0] Sp2VEnSecBuf = '0;
605
606 // Individually check sparsely encoded signals.
607 for (genvar i = 0; i < NumSp2VVSig; i++) begin :
gen_sel_buf_chk
608     aes_sel_buf_chk #(
609         .Num      ( Sp2VNum      ),
610         .Width    ( Sp2VWidth    ),
611         .EnSecBuf ( Sp2VEnSecBuf[i] )
612     ) u_aes_sp2v_sig_buf_chk_i (
613         .clk_i ( clk_i ),
614         .rst_ni ( rst_ni ),
615         .sel_i ( sp2v_sig[i] ),
616         .sel_o ( sp2v_sig_chk_raw[i] ),
617         .err_o ( sp2v_sig_err[i] )
618     );
619     assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
620 end
621
622 assign cipher_in_ready    = sp2v_sig_chk[0];
623 assign cipher_out_valid  = sp2v_sig_chk[1];
624 assign cipher_crypt      = sp2v_sig_chk[2];
625 assign cipher_dec_key_gen = sp2v_sig_chk[3];
626 assign ghash_in_ready    = sp2v_sig_chk[4];
627 assign ghash_out_valid  = sp2v_sig_chk[5];
628 assign ctr_ready        = sp2v_sig_chk[6];
629 for (genvar i = 0; i < NumSlicesCtr; i++) begin : gen_ctr_we
630     assign ctr_we[i] = sp2v_sig_chk[7+i];
631 end
632
633 // Collect encoding errors.
634 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
635 // are used. Here, we collect also encoding errors detected
in other places of the core.
636 assign sp_enc_err = |sp2v_sig_err | sp_enc_err_i;
637
638 ///////////////////////////////////////////////////
639 // Trigger Register //
640 ///////////////////////////////////////////////////
641 // Most triggers are only ever cleared by control.
642 assign start_o = 1'b0;
643 assign key_iv_data_in_clear_o = 1'b0;
644 assign data_out_clear_o = 1'b0;
645
646 endmodule

```

8. lowRISC_opentitan/hw/ip/aes/rtl/aes_control_fsm.sv (1,202 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES main control FSM
6 //
7 // This module contains the main control FSM handling the in-
terplay of input/output registers and
8 // the AES cipher core.
9
10 `include "prim_assert.sv"
11
12 module aes_control_fsm
13     import aes_pkg::*;
14     import aes_reg_pkg::*;
15     #(

```

```

16     parameter bit AESGCMEEnable = 0,
17     parameter bit SecMasking    = 0
18 ) (
19     input  logic                clk_i,
20     input  logic                rst_ni,
21
22     // Main control signals
23     input  logic                ctrl_qe_i,
24     output logic                ctrl_we_o,
25     input  logic                ctrl_phase_i,
26     input  logic                ctrl_err_storage_i,
27     input  aes_op_e             op_i,
28     input  aes_mode_e           mode_i,
29     input  ciph_op_e            cipher_op_i,
30     input  logic                sideload_i,
31     input  prs_rate_e           prng_re-
seed_rate_i,
32     input  logic                manu-
al_operation_i,
33     input  logic                key_touch_forces_reseed_i,
34     input  logic                ctrl_gcm_qe_i,
35     output logic                ctrl_gcm_we_o,
36     input  logic                ctrl_gcm_phase_i,
37     output logic                gcm_in-
it_done_o,
38     input  gcm_phase_e          gcm_phase_i,
39     input  logic                start_i,
40     input  logic                key_iv_data_in_clear_i,
41     input  logic                data_out_clear_i,
42     input  logic                prng_re-
seed_i,
43     input  logic                mux_sel_err_i,
44     input  logic                sp_enc_err_i,
45     input  lc_ctrl_pkg::lc_tx_t lc_escal-
ate_en_i,
46     input  logic                alert_fatal_i,
47     output logic                alert_o,
48
49     // I/O register read/write enables
50     input  logic                key_side-
load_valid_i,
51     input  logic [NumSharesKey-1:0][NumRegsKey-1:0] key_in-
it_qe_i,
52     input  logic [NumRegsIv-1:0] iv_qe_i,
53     input  logic [NumRegsData-1:0]
54     data_in_qe_i,
55     input  logic [NumRegsData-1:0]
56     data_out_re_i,
57     output logic                data_in_we_o,
58     output logic                data_out_sel_e
59     data_out_sel_o,
60     output logic                data_out_we_o,
61     // Sparsify
62     // Previous input data register
63     output dip_sel_e            data_in_prev_sel_o,
64     output logic                data_in_prev_we_o,
65     // Sparsify
66     // Cipher I/O muxes
67     output si_sel_e             state_in_sel_o,
68     output add_si_sel_e         add_state_in_sel_o,
69     output add_so_sel_e         add_state_out_sel_o,
70
71     // Counter
72     output logic                ctr_inc32_o,
73     output logic                cr_o,
74     input  logic                ctr_ready_i,
75     input  logic                ctr_we_i,
76     // Sparsify
77     // Cipher core control and sync
78     output logic                cipher_in_valid_o,
79     input  logic                cipher_in_ready_i,
80     input  logic                cipher_out_valid_i,
81     output logic                cipher_out_ready_o,
82     output logic                cipher_crypt_o,
83     // Sparsify

```

```

80  input logic
cipher_crypt_i,          // Sparsify
81  output logic
cipher_dec_key_gen_o,    // Sparsify
82  input logic
cipher_dec_key_gen_i,    // Sparsify
83  output logic
cipher_prng_reseed_o,
84  input logic
cipher_prng_reseed_i,
85  output logic
cipher_key_clear_o,
86  input logic
cipher_key_clear_i,
87  output logic
cipher_data_out_clear_o,
88  input logic
cipher_data_out_clear_i,
89
90  // GHASH control and sync
91  output logic
ghash_in_valid_o,        // Sparsify
92  input logic
ghash_in_ready_i,        // Sparsify
93  input logic
ghash_out_valid_i,       // Sparsify
94  output logic
ghash_out_ready_o,       // Sparsify
95  output logic
ghash_load_hash_subkey_o, // Sparsify
96
97  // Initial key registers
98  output key_init_sel_e          key_in-
it_sel_o,
99  output logic [NumSharesKey-1:0][NumRegsKey-1:0] key_in-
it_we_o,          // Sparsify
100
101  // IV registers
102  output iv_sel_e          iv_sel_o,
103  output logic [NumSlicesCtr-1:0]
iv_we_o,          // Sparsify
104
105  // Pseudo-random number generator interface
106  output logic          prng_up-
date_o,
107  output logic          prng_re-
seed_req_o,
108  input logic          prng_re-
seed_ack_i,
109
110  // Trigger register
111  output logic          start_we_o,
112  output logic
key_iv_data_in_clear_we_o,
113  output logic
data_out_clear_we_o,
114  output logic
seed_o,
115  output logic
seed_we_o,
116
117  // Status register
118  output logic          idle_o,
119  output logic          idle_we_o,
120  output logic          stall_o,
121  output logic          stall_we_o,
122  input logic          out-
put_lost_i,
123  output logic          out-
put_lost_o,
124  output logic          out-
put_lost_we_o,
125  output logic          out-
put_valid_o,
126  output logic          out-
put_valid_we_o,
127  output logic          in-
put_ready_o,
128  output logic          in-
put_ready_we_o
129 );
130
131  // Signals
132  aes_ctrl_e          aes_ctrl_ns, aes_ctrl_cs;
133  logic              prng_reseed_done_d, prng_re-
seed_done_q;
134
135  logic              key_init_clear;
136  logic              key_init_new;
137  logic              key_init_new_pulse;
138  logic              key_init_load;
139  logic              key_init_arm;
140  logic              key_init_ready;
141  logic              key_sideload;
142
143  logic [NumSlicesCtr-1:0] iv_qe;
144  logic              iv_clear;
145  logic              iv_load;
146  logic              iv_arm;
147  logic              iv_ready;
148

```

```

149  logic [NumRegsData-1:0] data_in_new_d, data_in_new_q;
150  logic              data_in_new;
151  logic              data_in_load;
152
153  logic [NumRegsData-1:0] data_out_read_d, data_out_read_q;
154  logic              data_out_read;
155  logic              output_valid_q;
156
157  logic              cfg_valid;
158  logic              no_alert;
159  logic              cipher_op_err;
160  logic              start_common, start_ecb,
start_cbc, start_cfb, start_ofb, start_ctr;
161  logic              start;
162  logic              start_core;
163  logic              finish;
164  logic              crypt;
165  logic              cipher_out_done;
166  logic              doing_cbc_enc, doing_cbc_dec;
167  logic              doing_cfb_enc, doing_cfb_dec;
168  logic              doing_ofb;
169  logic              doing_ctr;
170  logic              ctrl_we_q;
171  logic              clear_in_out_status;
172  logic              clear_on_fatal;
173
174  logic              start_we;
175  logic              key_iv_data_in_clear_we;
176  logic              data_out_clear_we;
177  logic              prng_reseed_we;
178
179  logic              idle;
180  logic              idle_we;
181  logic              stall;
182  logic              stall_we;
183  logic              output_lost;
184  logic              output_lost_we;
185  logic              output_valid;
186  logic              output_valid_we;
187  logic              input_ready;
188  logic              input_ready_we;
189
190  logic              ctrl_gcm_we_q;
191  logic              gcm_clear;
192  logic              gcm_init, gcm_restore, gcm_aad,
gcm_txt, gcm_save, gcm_tag;
193  logic              start_common_gcm, start_ghash;
194  logic              start_gcm_init, start_gcm_hsk,
start_gcm_s;
195  logic              start_gcm_restore, start_gcm_aad,
start_gcm_txt, start_gcm_save;
196  logic              start_gcm_tag;
197  logic              doing_gcm_hsk, doing_gcm_s, do-
ing_gcm_txt;
198  logic              hash_subkey_ready_d, hash_sub-
key_ready_q;
199  logic              s_ready_d, s_ready_q;
200  logic              doing_gcm_restore_d, do-
ing_gcm_restore_q;
201  logic              doing_gcm_aad_d, doing_gcm_aad_q;
202  logic              doing_gcm_tag_d, doing_gcm_tag_q;
203  logic              doing_gcm_save_d, do-
ing_gcm_save_q;
204  logic              ghash_out_done;
205  logic              ghash_idle;
206
207  logic              block_ctr_expr;
208  logic              block_ctr_decr;
209
210  // Software updates IV in chunks of 32 bits, the counter up-
dates SliceSizeCtr bits at a time.
211  // Convert word write enable to internal half-word write en-
able.
212  assign iv_qe = {iv_qe_i[3], iv_qe_i[3], iv_qe_i[2],
iv_qe_i[2],
213                  iv_qe_i[1], iv_qe_i[1], iv_qe_i[0],
iv_qe_i[0]};
214
215  // The cipher core is only ever allowed to start or finish
if the control register holds a valid
216  // configuration and if no fatal alert condition occurred.
217  assign cfg_valid = ~(mode_i == AES_NONE) | ctrl_err_stor-
age_i;
218  assign no_alert = ~alert_fatal_i;
219
220  // cipher_op_i is obtained from the configuration of the
control register with additional logic.
221  assign cipher_op_err = ~(cipher_op_i == CIPH_FWD ||
cipher_op_i == CIPH_INV);
222
223  // Check common start conditions. These are needed for any
mode, unless we are running in
224  // manual mode.
225  assign start_common = key_init_ready & data_in_new &
226  // If key sideload is enabled, we only start if the key
is valid.
(sideload_i ? key_sideload_valid_i : 1'b1);
228
229  // Check mode-specific start conditions. If the IV (and
counter) is needed, we only start if
230  // also the IV (and counter) is ready.

```

```

231 assign start_ecb = (mode_i == AES_ECB);
232 assign start_cbc = (mode_i == AES_CBC) & iv_ready;
233 assign start_cfb = (mode_i == AES_CFB) & iv_ready;
234 assign start_ofb = (mode_i == AES_OFB) & iv_ready;
235 assign start_ctr = (mode_i == AES_CTR) & iv_ready &
ctr_ready_i;
236
237 // If set to start manually, we just wait for the trigger.
Otherwise, check common as well as
238 // mode-specific start conditions.
239 assign start = cfg_valid & no_alert &
240 // Manual operation has priority.
241 (manual_operation_i ? start_i :
242 // Check start conditions for automatic operation.
243 ((start_ecb |
244 start_cbc |
245 start_cfb |
246 start_ofb |
247 start_ctr) & start_common) |
248 // Only the initial as well as the ciphertext/plain-
text phases of GCM require operating
249 // the AES cipher core. Common start conditions are
already factored into these signals.
250 (start_gcm_init |
251 start_gcm_txt));
252
253 // If not set to overwrite data, we wait for any previous
output data to be read. data_out_read
254 // synchronously clears output_valid_q, unless new output
data is written in the exact same
255 // clock cycle.
256 assign finish = cfg_valid & no_alert &
257 // Manual operation has priority.
258 (manual_operation_i ? 1'b1 :
259 // Make sure previous output data has been read.
260 (~output_valid_q | data_out_read));
261
262 ///////////////////////////////////////////////////
263 // GCM Start Conditions //
264 ///////////////////////////////////////////////////
265 // Note that only the initial as well as the ciphertext/
plaintext phases of GCM require operating
266 // the AES cipher core. The start conditions for these
phases are factored into the regular start
267 // signal above. In contrast, the restore, AAD, save and tag
phases use the GHASH block only. The
268 // corresponding start conditions are handled separately by
the FSM and we need to explicitly
269 // factor in manual operation below.
270
271 assign gcm_init = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_INIT);
272 assign gcm_restore = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_RESTORE);
273 assign gcm_aad = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_AAD);
274 assign gcm_txt = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_TXT);
275 assign gcm_save = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_SAVE);
276 assign gcm_tag = (mode_i == AES_GCM) & (gcm_phase_i ==
GCM_TAG);
277
278 // The common start conditions for GCM don't include the in-
put data.
279 assign start_common_gcm = key_init_ready &
280 // If key sideload is enabled, we only start if the key
is valid.
281 (sideload_i ? key_sideload_valid_i : 1'b1);
282
283 // For the initial GCM phase, we need key and IV but then
encrypt two blocks. The input data
284 // is not required. Requires operating the AES cipher core
and thus factors into the regular
285 // start signal. Delay GCM initialization while clearing
GCM-related status tracking.
286 assign start_gcm_hsk = gcm_init & ~gcm_clear & ~hash_sub-
key_ready_q & iv_ready & ctr_ready_i;
287 assign start_gcm_s = gcm_init & ~gcm_clear &
~s_ready_q & iv_ready & ctr_ready_i;
288 assign start_gcm_init = (start_gcm_hsk | start_gcm_s) &
start_common_gcm;
289
290 // For the restore phase of GCM, we need key, IV and input
data. If set to start manually, we
291 // just wait for the trigger.
292 assign start_gcm_restore = gcm_restore & cfg_valid &
no_alert &
293 // Manual operation has priority.
294 (manual_operation_i ? start_i : iv_ready & ctr_ready_i &
data_in_new & start_common_gcm);
295
296 // For the AAD and ciphertext/plaintext phases of GCM, we
need key, IV and input data. But for
297 // the AAD phase, only the input data is really used and
marked as used. The start condition for
298 // the AAD phase is handled separately by the FSM and we
need to factor in the manual operation
299 // mode here.

```

```

300 assign start_gcm_aad = gcm_aad & cfg_valid & no_alert &
301 // Manual operation has priority.
302 (manual_operation_i ? start_i : iv_ready & ctr_ready_i &
start_common);
303 // The ciphertext/plaintext phase requires operating AES
cipher core and thus factors into the
304 // regular start signal.
305 assign start_gcm_txt = gcm_txt & iv_ready & ctr_ready_i &
start_common;
306
307 // For the save phase of GCM, we need no inputs. But the
hash subkey and S must not yet have been
308 // cleared (happens upon saving the GHASH state).
309 assign start_gcm_save = gcm_save & cfg_valid & no_alert &
310 // Manual operation has priority.
311 (manual_operation_i ? start_i : hash_subkey_ready_q &
s_ready_q);
312
313 // For the tag phase of GCM, we also need the input data.
314 assign start_gcm_tag = gcm_tag & cfg_valid & no_alert &
315 // Manual operation has priority.
316 (manual_operation_i ? start_i : hash_subkey_ready_q &
s_ready_q & data_in_new);
317
318 // The restore, AAD, save and tag phases use the GHASH block
only.
319 assign start_ghash = start_gcm_restore | start_gcm_aad |
start_gcm_save | start_gcm_tag;
320
321 // The GHASH block is idle whenever it's ready to receive
inputs and we're not about to start
322 // a GCM related operation.
323 assign ghash_idle = ghash_in_ready_i & ~start_ghash;
324
325 // In GCM, the counter performs inc32() instead of inc128(),
i.e., the counter wraps at 32 bits.
326 assign ctr_inc32_o = (mode_i == AES_GCM);
327
328 // Helper signals for FSM
329 assign crypt = cipher_crypt_o | cipher_crypt_i;
330
331 assign doing_cbc_enc = (mode_i == AES_CBC && op_i ==
AES_ENC) & crypt;
332 assign doing_cbc_dec = (mode_i == AES_CBC && op_i ==
AES_DEC) & crypt;
333 assign doing_cfb_enc = (mode_i == AES_CFB && op_i ==
AES_ENC) & crypt;
334 assign doing_cfb_dec = (mode_i == AES_CFB && op_i ==
AES_DEC) & crypt;
335 assign doing_ofb = (mode_i ==
AES_OFB) & crypt;
336 assign doing_ctr = (mode_i ==
AES_CTR) & crypt;
337 assign doing_gcm_hsk = gcm_init & ~hash_sub-
key_ready_q & crypt;
338 assign doing_gcm_s = gcm_init & hash_sub-
key_ready_q & crypt;
339 assign doing_gcm_txt =
gcm_txt & crypt;
340
341 // FSM
342 always_comb begin : aes_ctrl_fsm
343
344 // Previous input data register control
345 data_in_prev_sel_o = DIP_CLEAR;
346 data_in_prev_we_o = 1'b0;
347
348 // Cipher I/O mux control
349 state_in_sel_o = SI_DATA;
350 add_state_in_sel_o = ADD_SI_ZERO;
351 add_state_out_sel_o = ADD_SO_ZERO;
352
353 // Counter control
354 ctr_incr_o = 1'b0;
355
356 // Cipher core control
357 cipher_in_valid_o = 1'b0;
358 cipher_out_ready_o = 1'b0;
359 cipher_out_done = 1'b0;
360 cipher_crypt_o = 1'b0;
361 cipher_dec_key_gen_o = 1'b0;
362 cipher_prng_reseed_o = 1'b0;
363 cipher_key_clear_o = 1'b0;
364 cipher_data_out_clear_o = 1'b0;
365
366 // GHASH control
367 ghash_in_valid_o = 1'b0;
368 ghash_out_ready_o = 1'b0;
369 ghash_out_done = 1'b0;
370 ghash_load_hash_subkey_o = ~hash_subkey_ready_q;
371
372 // Initial key registers
373 key_init_sel_o = sideload_i ? KEY_INIT_KEYMGR : KEY_IN-
IT_INPUT;
374 key_init_we_o = {NumSharesKey * NumRegsKey{1'b0}};
375
376 // IV registers
377 iv_sel_o = IV_INPUT;

```

```

378   iv_we_o = {NumSlicesCtr{1'b0}};
379
380   // Control registers
381   ctrl_we_o = 1'b0;
382   ctrl_gcm_we_o = 1'b0;
383
384   // Alert
385   alert_o = 1'b0;
386
387   // Pseudo-random number generator control
388   prng_update_o = 1'b0;
389   prng_reseed_req_o = 1'b0;
390
391   // Trigger register control
392   start_we = 1'b0;
393   key_iv_data_in_clear_we = 1'b0;
394   data_out_clear_we = 1'b0;
395   prng_reseed_we = 1'b0;
396
397   // Status register
398   idle = 1'b0;
399   idle_we = 1'b0;
400   stall = 1'b0;
401   stall_we = 1'b0;
402
403   // Key, data I/O register control
404   data_in_load = 1'b0;
405   data_in_we_o = 1'b0;
406   data_out_sel_o = DATA_OUT_CIPHER;
407   data_out_we_o = 1'b0;
408
409   // Register status tracker control
410   key_init_clear = 1'b0;
411   key_init_load = 1'b0;
412   key_init_arm = 1'b0;
413   iv_clear = 1'b0;
414   iv_load = 1'b0;
415   iv_arm = 1'b0;
416
417   // Block counter
418   block_ctr_decr = 1'b0;
419
420   // FSM
421   aes_ctrl_ns = aes_ctrl_cs;
422   start_core = 1'b0;
423   prng_reseed_done_d = prng_reseed_done_q | prng_reseed_ack_i;
424   hash_subkey_ready_d = hash_subkey_ready_q;
425   s_ready_d = s_ready_q;
426   doing_gcm_restore_d = doing_gcm_restore_q;
427   doing_gcm_aad_d = doing_gcm_aad_q;
428   doing_gcm_tag_d = doing_gcm_tag_q;
429   doing_gcm_save_d = doing_gcm_save_q;
430
431   unique case (aes_ctrl_cs)
432
433     CTRL_IDLE: begin
434
435       // The core is about to start encryption/decryption or
436       another action.
437       start_core = start | key_iv_data_in_clear_i |
438       data_out_clear_i | prng_reseed_i;
439
440       // Update status register. A write to the main control
441       register (if sideload is enabled)
442       // or writing the last key register can initiate a
443       PRNG reseed operation via trigger
444       // register. To avoid that subsequent writes to the
445       main control, key or IV registers
446       // collide with the start of the reseed operation, de-
447       assert the idle bit. The GHASH block
448
449       // may still be busy while the cipher core is actually
450       idle.
451       idle = ~(start_core | (prng_reseed_o & prng_reseed_we_o)) & ghash_idle;
452       idle_we = 1'b1;
453
454       // Clear the start trigger when seeing invalid configurations or performing automatic
455       // operation.
456       start_we = start_i & ((mode_i == AES_NONE) | ~manual_operation_i);
457
458       if (!start_core) begin
459         // Initial key and IV updates are ignored if the
460         core is about to start. If key sideload
461         // is enabled, software writes to the initial key
462         registers are ignored.
463         key_init_we_o = sideload_i ? {NumSharesKey * NumRegisters[key_sideload]} : key_init_we_i;
464         iv_we_o = iv_we_i;
465       end
466
467       if (!start_core && !start_ghash) begin
468
469         // Updates to the main and GCM control registers are
470         only allowed if the cipher core and
471         // the GHASH block are not about to start and there
472         isn't a storage error. A storage

```

```

459         // error is unrecoverable and requires a reset.
460         ctrl_we_o = !ctrl_err_storage_i ?
ctrl_qe_i : 1'b0;
461         ctrl_gcm_we_o = !ctrl_err_storage_i ?
ctrl_gcm_qe_i : 1'b0;
462
463         // Control register updates clear all register
464         status trackers. As GCM initialization
465         // advances the IV, we have to clear the IV status
466         tracking when re-initializing GCM.
467         key_init_clear = ctrl_we_o;
468         iv_clear = ctrl_we_o | (gcm_init & gcm_clear);
469
470         // Also clear GCM-specific status tracking.
471         if (ctrl_we_o | gcm_clear) begin
472           hash_subkey_ready_d = 1'b0;
473           s_ready_d = 1'b0;
474         end
475
476         if (prng_reseed_i) begin
477           // PRNG reseeding has highest priority.
478           if (!SecMasking) begin
479             prng_reseed_done_d = 1'b0;
480             aes_ctrl_ns = CTRL_PRNG_RESEED;
481           end else begin
482             // In case masking is enabled, also the masking
483             PRNG inside the cipher core needs to
484             // be reseeded.
485             cipher_prng_reseed_o = 1'b1;
486
487             // Perform handshake.
488             cipher_in_valid_o = 1'b1;
489             if (cipher_in_ready_i) begin
490               prng_reseed_done_d = 1'b0;
491               aes_ctrl_ns = CTRL_PRNG_RESEED;
492             end
493           end
494
495           end else if (key_iv_data_in_clear_i ||
496           data_out_clear_i) begin
497             // To clear registers, we update the PRNG and then
498             wait for the GHASH block.
499             prng_update_o = 1'b1;
500
501             // To clear the output data registers, we re-use the
502             muxing resources of the cipher
503             // core. To clear all key material, some key registers
504             inside the cipher core need to
505             // be cleared.
506             cipher_key_clear_o = key_iv_data_in_clear_i;
507             cipher_data_out_clear_o = data_out_clear_i;
508
509             // We have work for the cipher core, perform handshake.
510             cipher_in_valid_o = 1'b1;
511             if (cipher_in_ready_i) begin
512               aes_ctrl_ns = CTRL_GHASH_READY;
513             end
514
515             end else if (start_gcm_restore || start_gcm_aad ||
516             start_gcm_tag) begin
517               // We don't have work for the AES cipher core but
518               for the GHASH block only. Load the
519               // input data into the previous input data register
520               such that it can be loaded into
521               // the GHASH block in unmasked form.
522               data_in_prev_sel_o = DIP_DATA_IN;
523               data_in_prev_we_o = 1'b1;
524
525               // Advance to the loading state where the input data
526               is marked as used.
527               doing_gcm_restore_d = start_gcm_restore;
528               doing_gcm_aad_d = start_gcm_aad;
529               doing_gcm_tag_d = start_gcm_tag;
530               start_we = 1'b1;
531               aes_ctrl_ns = CTRL_LOAD;
532
533             end else if (start_gcm_save) begin
534               // We don't have work for the AES cipher core but
535               for the GHASH block only. Update the
536               // PRNG (to clear the internal state after saving
537               it) and advance.
538               prng_update_o = 1'b1;
539               doing_gcm_save_d = 1'b1;
540               start_we = 1'b1;
541               aes_ctrl_ns = CTRL_GHASH_READY;
542
543             end else if (start) begin
544               // Signal that we want to start encryption/decryption.
545               cipher_crypt_o = 1'b1;
546
547               // Signal if the cipher core shall reseed the masking
548               PRNG.
549               cipher_prng_reseed_o = block_ctr_expr;
550
551               // We got a new initial key, but want to do decryption.
552               The cipher core must first

```

```

539 // generate the start key for decryption.
540 cipher_dec_key_gen_o = (cipher_op_i == CIPH_INV) ?
key_init_new : 1'b0;
541
542 // Previous input data register control
543 data_in_prev_sel_o = doing_cbc_dec ? DIP_DATA_IN :
544 doing_cfb_enc ? DIP_DATA_IN :
545 doing_cfb_dec ? DIP_DATA_IN :
546 doing_ofb ? DIP_DATA_IN :
547 doing_ctr ? DIP_DATA_IN :
548 doing_gcm_txt ? DIP_DATA_IN :
DIP_CLEAR;
549 data_in_prev_we_o = doing_cbc_dec |
550 doing_cfb_enc |
551 doing_cfb_dec |
552 doing_ofb |
553 doing_ctr |
554 doing_gcm_txt;
555
556 // State input mux control
557 state_in_sel_o = doing_cfb_enc ? SI_ZERO :
558 doing_cfb_dec ? SI_ZERO :
559 doing_ofb ? SI_ZERO :
560 doing_ctr ? SI_ZERO :
561 doing_gcm_hsk ? SI_ZERO :
562 doing_gcm_s ? SI_ZERO :
563 doing_gcm_txt ? SI_ZERO :
SI_DATA;
564
565 // State input addition mux control
566 add_state_in_sel_o = doing_cbc_enc ? ADD_SI_IV :
567 doing_cfb_enc ? ADD_SI_IV :
568 doing_cfb_dec ? ADD_SI_IV :
569 doing_ofb ? ADD_SI_IV :
570 doing_ctr ? ADD_SI_IV :
571 doing_gcm_s ? ADD_SI_IV :
572 doing_gcm_txt ? ADD_SI_IV :
ADD_SI_ZERO;
573
574 // We have work for the cipher core, perform hand-
shake.
575 cipher_in_valid_o = 1'b1;
576 if (cipher_in_ready_i) begin
577 // Do not yet clear a possible start trigger if we
are just starting the generation of
578 // the start key for decryption.
579 start_we = ~cipher_dec_key_gen_o;
580 aes_ctrl_ns = CTRL_LOAD;
581 end
582 end
583 end
584 CTRL_LOAD: begin
585 // Signal that we have used the current key, IV, data
input to register status tracking.
587 key_init_load = cipher_dec_key_gen_i; // This key is
no longer "new", but still clean.
588 key_init_arm = ~cipher_dec_key_gen_i; // The key is
still "new", prevent partial updates.
589 iv_load = ~cipher_dec_key_gen_i & (doing_cbc_enc |
590 doing_cbc_dec |
591 doing_cfb_enc |
592 doing_cfb_dec |
593 doing_ofb |
594 doing_ctr |
595 doing_gcm_hsk |
596 doing_gcm_s |
597 do-
ing_gcm_txt);
598 data_in_load = ~cipher_dec_key_gen_i;
599
600 // Trigger counter increment.
601 ctr_incr_o = doing_ctr | doing_gcm_hsk | doing_gcm_s |
doing_gcm_txt;
602
603 // Unless we are just generating the start key for de-
cryption, we must update the PRNG.
604 // Fresh pseudo-random data is used to:
605 // - clear the state in the final cipher round,
606 // - clear any other registers in the CLEAR_IV/CO
states.
607 prng_update_o = !cipher_dec_key_gen_i;
608
609 // Unless we are just generating the start key for de-
cryption, we may need to interface
610 // the GHASH block as well
611 aes_ctrl_ns = !cipher_dec_key_gen_i ?
CTRL_GHASH_READY : CTRL_FINISH;
612 end
613
614 CTRL_GHASH_READY: begin

```

```

615 // IV control in case of ongoing encryption/decryption
616 // - CTR: IV registers are updated by counter during
cipher operation
617 iv_sel_o = doing_ctr ||
618 doing_gcm_hsk ||
619 doing_gcm_s ||
620 doing_gcm_txt ? IV_CTR : IV_INPUT;
621 iv_we_o = doing_ctr ||
622 doing_gcm_hsk ||
623 doing_gcm_s ||
624 doing_gcm_txt ? ctr_we_i :
{NumSlicesCtr{1'b0}};
625
626 // Ongoing encryption/decryption operations have the
highest priority. The clear triggers
627 // might for example have become asserted after the
handshake with the cipher core.
628 if (cipher_crypt_i) begin
629 if (doing_gcm_hsk || doing_gcm_s || doing_gcm_txt)
begin
630 // We actually have some work for the GHASH block.
Make sure it's ready before we move.
631 // We send the valid in a following clock cycle.
Doing this check here, allows to
632 // decouple the valid from the ready.
633 if (ghash_in_ready_i) begin
634 aes_ctrl_ns = CTRL_FINISH;
635 end
636 end else begin
637 // We're not actually using the GHASH block and
don't need to check it's status.
638 aes_ctrl_ns = CTRL_FINISH;
639 end
640 end else if (doing_gcm_restore_q) begin
641 // Pass the previously saved GHASH state to the
GHASH block. We're done after the input
642 // handshake.
643 ghash_in_valid_o = 1'b1;
644 if (ghash_in_ready_i) begin
645 doing_gcm_restore_d = 1'b0;
646 aes_ctrl_ns = CTRL_IDLE;
647 end
648 end else if (doing_gcm_aad_q) begin
649 // Pass the AAD to the GHASH block. We're done after
the input handshake.
650 ghash_in_valid_o = 1'b1;
651 if (ghash_in_ready_i) begin
652 doing_gcm_aad_d = 1'b0;
653 aes_ctrl_ns = CTRL_IDLE;
654 end
655 end else if (doing_gcm_save_q) begin
656 // Perform the handshake with the GHASH block and
then wait for the GHASH block to output
657 // the state.
658 ghash_in_valid_o = 1'b1;
659 if (ghash_in_ready_i) begin
660 aes_ctrl_ns = CTRL_FINISH;
661 end
662 end else if (doing_gcm_tag_q) begin
663 // Pass the AAD and text length to the GHASH block
and wait for the final authentication
664 // tag afterwards.
665 ghash_in_valid_o = 1'b1;
666 if (ghash_in_ready_i) begin
667 aes_ctrl_ns = CTRL_FINISH;
668 end
669 end else if (key_iv_data_in_clear_i ||
data_out_clear_i ||
670 cipher_key_clear_i ||
cipher_data_out_clear_i) begin
671 // We actually have some work for the GHASH block.
Make sure it's ready before we move.
672 // We send the valid in a following clock cycle. Do-
ing this check here, allows to
673 // decouple the valid from the ready.
674 if (ghash_in_ready_i) begin
675 aes_ctrl_ns = CTRL_CLEAR_I;
676 end
677 end else begin
678 // Another write to the trigger register must have
overwritten the trigger bits that
679 // actually caused us to enter this state. Just re-
turn.
680 aes_ctrl_ns = CTRL_IDLE;
681 end
682 end
683 CTRL_PRNG_RESEED: begin
684 // Request a reseed of the clearing PRNG.
685 prng_reseed_req_o = ~prng_reseed_done_q;
686
687 if (!SecMasking) begin
688 if (prng_reseed_done_q) begin
689 // Clear the trigger and return.

```



```

696         prng_reseed_we    = 1'b1;
697         prng_reseed_done_d = 1'b0;
698         aes_ctrl_ns       = CTRL_IDLE;
699     end
700
701     end else begin
702         // In case masking is used, we must also wait for
the cipher core
703         // to reseed the internal
704         // masking PRNG. Perform handshake.
705         cipher_out_ready_o = prng_reseed_done_q;
706         if (cipher_out_ready_o && cipher_out_valid_i) begin
707             // Clear the trigger and return.
708             prng_reseed_we    = 1'b1;
709             prng_reseed_done_d = 1'b0;
710             aes_ctrl_ns       = CTRL_IDLE;
711         end
712     end
713
714     CTRL_FINISH: begin
715         // Wait for cipher core to finish.
716
717         if (cipher_dec_key_gen_i) begin
718             // We are ready.
719             cipher_out_ready_o = 1'b1;
720             if (cipher_out_valid_i) begin
721                 block_ctr_decr = 1'b1;
722                 aes_ctrl_ns     = CTRL_IDLE;
723             end
724         end else if (doing_gcm_save_q || doing_gcm_tag_q) be-
gin
725             // Handshake signals: We are ready once the output
data registers
726             // can be written. Don't
727             // let data propagate in case of mux selector or
sparsely encoded
728             // signals taking on
729             // invalid values.
730             ghash_out_ready_o = finish;
731             ghash_out_done    = finish & ghash_out_valid_i &
~mux_sel_err_i & ~sp_enc_err_i & ~cipher_op_err;
732             // Signal if the GHASH block is stalled (because
previous output
733             // has not yet been read).
734             stall    = ~finish & ghash_out_valid_i;
735             stall_we = 1'b1;
736             // Forward the GHASH output instead of the cipher
core output.
737             data_out_sel_o = DATA_OUT_GHASH;
738
739             // Proceed upon successful handshake. Mark hash sub-
key and s as not ready, the GHASH
740             // block clears those registers after the handshake.
741             if (ghash_out_done) begin
742                 doing_gcm_save_d    = 1'b0;
743                 doing_gcm_tag_d     = 1'b0;
744                 hash_subkey_ready_d = 1'b0;
745                 s_ready_d           = 1'b0;
746                 data_out_we_o       = 1'b1;
747                 aes_ctrl_ns         = CTRL_IDLE;
748             end
749
750             end else begin
751                 // Handshake signals: We are ready once the output
data registers
752                 // can be written. Don't
753                 // let data propagate in case of mux selector or
sparsely encoded
754                 // signals taking on
755                 // invalid values.
756                 cipher_out_ready_o = finish;
757                 cipher_out_done    = finish & cipher_out_valid_i &
~mux_sel_err_i & ~sp_enc_err_i & ~cipher_op_err;
758                 // Signal if the cipher core is stalled (because
previous output
759                 // has not yet been read).
760                 stall    = ~finish & cipher_out_valid_i;
761                 stall_we = 1'b1;
762
763                 // State out addition mux control
764                 add_state_out_sel_o = doing_cbc_dec ? ADD_SO_IV :
765                                     doing_cfb_enc ? ADD_SO_DIP :
766                                     doing_cfb_dec ? ADD_SO_DIP :
767                                     doing_ofb   ? ADD_SO_DIP :
768                                     doing_ctr    ? ADD_SO_DIP :
769                                     doing_gcm_txt ? ADD_SO_DIP :
770                                     ADD_SO_ZERO;
771
772                 // IV control
773
774                 // - CBC/CFB/OFB: IV registers are only updated when
cipher finishes.
775                 // - CTR/GCM: IV registers are updated by counter
during cipher operation.
776                 // - The same holds when we're computing the hash
subkey for GCM.
777                 iv_sel_o = doing_cbc_enc ? IV_DATA_OUT :
778                             doing_cbc_dec ? IV_DATA_IN_PREV :
779                             doing_cfb_enc ? IV_DATA_OUT :
780                             doing_cfb_dec ? IV_DATA_IN_PREV :
781                             doing_ofb   ? IV_DATA_OUT_RAW :
782                             doing_ctr    ? IV_CTR :
783                             doing_gcm_hsk ? IV_CTR :
784                             doing_gcm_s  ? IV_CTR :

```

```

782         doing_gcm_txt ? IV_CTR : IV_IN-
PUT;
783         iv_we_o = doing_cbc_enc ||
784                 doing_cbc_dec ||
785                 doing_cfb_enc ||
786                 doing_cfb_dec ||
787                 doing_ofb   ?
{NumSlicesCtr{cipher_out_done}} :
788                 doing_ctr    ?
ctr_we_i :
789                 doing_gcm_hsk ?
ctr_we_i :
790                 doing_gcm_s  ?
ctr_we_i :
791                 doing_gcm_txt ?
ctr_we_i : {NumSlicesCtr{1'b0}};
792
793         // Arm the IV status tracker: After finishing, the
IV registers can be written again
794         // by software. We need to make sure software does
not partially update the IV.
795         iv_arm = (doing_cbc_enc |
796                 doing_cbc_dec |
797                 doing_cfb_enc |
798                 doing_cfb_dec |
799                 doing_ofb   |
800                 doing_ctr    |
801                 doing_gcm_hsk |
802                 doing_gcm_s  |
803                 doing_gcm_txt) & cipher_out_done;
804
805         // Proceed upon successful handshake. When initial-
izing GHASH for GCM, don't write to the
806         // cipher output to the output data registers but
forward it to the GHASH block and
807         // mark the hash subkey or S as ready.
808         if (cipher_out_done) begin
809             block_ctr_decr = 1'b1;
810             data_out_we_o   = doing_gcm_hsk |
811                             doing_gcm_s  ? 1'b0 : 1'b1;
812             ghash_in_valid_o = doing_gcm_hsk |
813                             doing_gcm_s  ? 1'b0 : 1'b1;
814             doing_gcm_txt ? 1'b1 : 1'b0;
815             hash_subkey_ready_d = doing_gcm_hsk ? 1'b1 :
hash_subkey_ready_q;
816             s_ready_d           = doing_gcm_s  ? 1'b1 :
s_ready_q;
817             aes_ctrl_ns         = CTRL_IDLE;
818         end
819     end
820
821     CTRL_CLEAR_I: begin
822         // Clear input registers such as Initial Key, IV and
input data registers.
823         if (key_iv_data_in_clear_i) begin
824             // Initial Key
825             key_init_sel_o = KEY_INIT_CLEAR;
826             key_init_we_o  = {NumSharesKey * NumRegsKey{1'b1}};
827             key_init_clear = 1'b1;
828
829             // IV
830             iv_sel_o = IV_CLEAR;
831             iv_we_o  = {NumSlicesCtr{1'b1}};
832             iv_clear = 1'b1;
833
834             // Input data
835             data_in_we_o = 1'b1;
836             data_in_prev_sel_o = DIP_CLEAR;
837             data_in_prev_we_o = 1'b1;
838         end
839         aes_ctrl_ns = CTRL_CLEAR_CO;
840     end
841
842     CTRL_CLEAR_CO: begin
843
844         // Wait for cipher core to clear internal Full Key and
Decryption Key registers and/or
845         // the state register and clear output data registers
afterwards.
846
847         // Perform handshake with cipher core.
848         cipher_out_ready_o = 1'b1;
849         if (cipher_out_valid_i) begin
850
851             // Full Key and Decryption Key registers are cleared
by the cipher core.
852             // key_iv_data_in_clear_i is acknowledged by the
cipher core with cipher_key_clear_i.
853             if (cipher_key_clear_i) begin
854                 // Clear the trigger bit.
855                 key_iv_data_in_clear_we = 1'b1;
856
857                 // The GHASH block can now clear it's internal re-
gisters.
858                 ghash_in_valid_o = 1'b1;
859             end
860
861             // To clear the output data registers, we re-use the

```

```

muxing resources of the cipher core.
862 // data_out_clear_i is acknowledged by the cipher
core with cipher_data_out_clear_i.
863 if (cipher_data_out_clear_i) begin
864 // Clear output data and the trigger bit. Don't
release data from cipher core in case
865 // of mux selector or sparsely encoded signals
taking on invalid values.

866 data_out_we_o = ~mux_sel_err_i & ~sp_enc_err_i &
~cipher_op_err;
867 data_out_clear_we = 1'b1;
868 end
869
870 aes_ctrl_ns = CTRL_IDLE;
871 end
872 end
873
874 CTRL_ERROR: begin
875 // SEC_CM: MAIN.FSM.GLOBAL_ESC
876 // SEC_CM: MAIN.FSM.LOCAL_ESC
877 // Terminal error state
878 alert_o = 1'b1;
879 end
880
881 // We should never get here. If we do (e.g. via a mali-
cious glitch), error out immediately.
882 default: begin
883 aes_ctrl_ns = CTRL_ERROR;
884 alert_o = 1'b1;
885 end
886 endcase
887
888 // Unconditionally jump into the terminal error state in
case a mux selector or a sparsely
889 // encoded signal becomes invalid, or if the life cycle
controller triggers an escalation.
890 if (mux_sel_err_i || sp_enc_err_i || cipher_op_err ||
lc_c-
891 trl_pkg::lc_tx_test_true_loose(lc_escalate_en_i)) begin
892 aes_ctrl_ns = CTRL_ERROR;
893 end
894 end
895
896 // SEC_CM: MAIN.FSM.SPARSE
897 'PRIM_FLOP_SPARSE_FSM(u_state_regs, aes_ctrl_ns, aes_c-
trl_cs, aes_ctrl_e, CTRL_IDLE)
898
899 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
900 if (!rst_ni) begin
901 prng_reseed_done_q <= 1'b0;
902 end else begin
903 prng_reseed_done_q <= prng_reseed_done_d;
904 end
905 end
906
907 if (AESGCMEnable) begin : gen_reg_fsm_gcm
908 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_fsm_gcm
909 if (!rst_ni) begin
910 hash_subkey_ready_q <= 1'b0;
911 s_ready_q <= 1'b0;
912 doing_gcm_restore_q <= 1'b0;
913 doing_gcm_aad_q <= 1'b0;
914 doing_gcm_save_q <= 1'b0;
915 doing_gcm_tag_q <= 1'b0;
916 end else begin
917 hash_subkey_ready_q <= hash_subkey_ready_d;
918 s_ready_q <= s_ready_d;
919 doing_gcm_restore_q <= doing_gcm_restore_d;
920 doing_gcm_aad_q <= doing_gcm_aad_d;
921 doing_gcm_save_q <= doing_gcm_save_d;
922 doing_gcm_tag_q <= doing_gcm_tag_d;
923 end
924 end
925 end else begin : gen_no_reg_fsm_gcm
926 // GCM is simply not supported.
927 assign hash_subkey_ready_q = 1'b0;
928 assign s_ready_q = 1'b0;
929 assign doing_gcm_restore_q = 1'b0;
930 assign doing_gcm_aad_q = 1'b0;
931 assign doing_gcm_save_q = 1'b0;
932 assign doing_gcm_tag_q = 1'b0;
933
934 // Tie-off unused signals.
935 logic unused_gcm_d;
936 assign unused_gcm_d = ^{hash_subkey_ready_d,
937 s_ready_d,
938 doing_gcm_restore_d,
939 doing_gcm_aad_d,
940 doing_gcm_save_d,
941 doing_gcm_tag_d};
942 end
943
944 //////////////////////////////////////////////////
945 // Status Tracking //
946 //////////////////////////////////////////////////
947
948 // We only take a new sideload key if sideload is enabled,
if the provided sideload key is marked

```

```

949 // as valid, and after the control register has been written
for the second time. After that

950 // point we don't update the key anymore, as we don't have a
notion of when it actually changes.
951 // This would be required to trigger decryption key genera-
tion for ECB/CBC decryption.
952 // To update the sideload key, software has to:
953 // 1) wait until AES is idle,
954 // 2) wait for the key manager to provide the new key,
955 // 3) start a new message by writing the control register
and providing the IV (if needed).
956 assign key_sideload = sideload_i & key_sideload_valid_i &
ctrl_we_q & ~ctrl_phase_i;
957
958 // We only use clean initial keys. Either software/counter
has updated
959 // - all initial key registers, or
960 // - none of the initial key registers but the registers
were updated in the past.
961 aes_reg_status #(
962 .Width ( $bits(key_init_we_o) )
963 ) u_reg_status_key_init (
964 .clk_i ( clk_i ),
965 .rst_ni ( rst_ni ),
966 .we_i ( key_init_we_o ),
967 .use_i ( key_init_load ),
968 .clear_i ( key_init_clear ),
969 .arm_i ( key_init_arm ),
970 .new_o ( key_init_new ),
971 .new_pulse_o ( key_init_new_pulse ),
972 .clean_o ( key_init_ready )
973 );
974
975 // We only use clean and unused IVs. Either software/counter
has updated
976 // - all IV registers, or

977 // - none of the IV registers but the registers were updated
in the past
978 // and this particular IV has not yet been used.
979 aes_reg_status #(
980 .Width ( $bits(iv_we_o) )
981 ) u_reg_status_iv (
982 .clk_i ( clk_i ),
983 .rst_ni ( rst_ni ),
984 .we_i ( iv_we_o ),
985 .use_i ( iv_load ),
986 .clear_i ( iv_clear ),
987 .arm_i ( iv_arm ),
988 .new_o ( iv_ready ),
989 .new_pulse_o ( ),
990 .clean_o ( )
991 );
992
993 // Input and output data register status tracking detects
if:
994 // - A complete new data input block is available, and
995 // - An output data block has been read completely.
996 // The status tracking needs to be cleared upon writes to
the control register. The clearing is

997 // applied one cycle later here to avoid zero-latency loops.
This additional delay is not
998 // relevant as if we are about to start encryption/decryp-
tion, we anyway don't allow writes
999 // to the control register.
1000 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_c-
trl_we
1001 if (!rst_ni) begin
1002 ctrl_we_q <= 1'b0;
1003 end else begin
1004 ctrl_we_q <= ctrl_we_o;
1005 end
1006 end
1007 assign clear_in_out_status = ctrl_we_q;
1008
1009 // Collect writes to data input registers. Cleared if:
1010 // - data is loaded into cipher core,
1011 // - clearing data input registers with random data (all
data_in_qe_i bits high in next cycle),
1012 // - clearing the status tracking.
1013 assign data_in_new_d = data_in_load || &data_in_qe_i ||
clear_in_out_status ? '0 :
1014 data_in_new_q | data_in_qe_i;
1015 assign data_in_new = &data_in_new_d;
1016
1017 // Collect reads of data output registers. data_out_read is
high for one clock cycle only and

1018 // clears output_valid_q unless new output is written in the
exact same cycle. Cleared if:
1019 // - clearing data output registers with random data,
1020 // - clearing the status tracking.
1021 assign data_out_read_d = &data_out_read_q ||
clear_in_out_status ? '0 :
1022 data_out_read_q | data_out_re_i;
1023 assign data_out_read = &data_out_read_d;
1024
1025 always_ff @(posedge clk_i or negedge rst_ni) begin :

```

```

reg_edge_detection
1026   if (!rst_ni) begin
1027     data_in_new_q <= '0;
1028     data_out_read_q <= '0;
1029   end else begin
1030     data_in_new_q <= data_in_new_d;
1031     data_out_read_q <= data_out_read_d;
1032   end
1033 end
1034
1035 // Status register bits for data input and output
1036 // Cleared to 1 if:
1037 // - data is loaded into cipher core,
1038 // - clearing data input registers with random data,
1039 // - clearing the status tracking.
1040 assign input_ready = ~data_in_new;
1041 assign input_ready_we = data_in_new | data_in_load |
data_in_we_o | clear_in_out_status;
1042
1043 // Cleared if:
1044 // - all data output registers have been read (unless new
output is written in the same cycle),
1045 // - clearing data output registers with random data,
1046 // - clearing the status tracking.
1047 assign output_valid = data_out_we_o & ~data_out_clear_we;
1048 assign output_valid_we = data_out_we_o | data_out_read |
data_out_clear_we |
1049   clear_in_out_status;
1050
1051 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_output_valid
1052   if (!rst_ni) begin
1053     output_valid_q <= '0;
1054   end else if (output_valid_we) begin
1055     output_valid_q <= output_valid;
1056   end
1057 end
1058
1059 // Output lost status register bit
1060 // Cleared when updating the Control Register. Set when
overwriting previous output data that has
1061 // not yet been read.
1062 assign output_lost = ctrl_we_o ? 1'b0 :
1063   output_lost_i ? 1'b1 : out-
put_valid_q & ~data_out_read;
1064 assign output_lost_we = ctrl_we_o | data_out_we_o;
1065
1066 ////////////////////////////////////////////////////
1067 // GCM Status Tracking //
1068 ////////////////////////////////////////////////////
1069
1070 // Clear GCM-specific status tracking if the GCM phase is
set to GCM_INIT after a second write to
1071 // the shadowed GCM control register
1072
1072 assign gcm_clear = (gcm_phase_i == GCM_INIT) & ctrl_gcm_we_q
& ~ctrl_gcm_phase_i;
1073
1074 if (AESGCMEnable) begin : gen_reg_ctrl_gcm_we
1075   always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_ctrl_gcm_we
1076     if (!rst_ni) begin
1077       ctrl_gcm_we_q <= 1'b0;
1078     end else begin
1079       ctrl_gcm_we_q <= ctrl_gcm_we_o;
1080     end
1081   end
1082 end else begin : gen_no_reg_ctrl_gcm_we
1083   assign ctrl_gcm_we_q = 1'b1;
1084 end
1085
1086 // Signal when the GCM initialization is done.
1087 assign gcm_init_done_o = hash_subkey_ready_q & s_ready_q;
1088
1089 // Should fatal alerts clear the status and trigger re-
gister?
1090 assign clear_on_fatal = ClearStatusOnFatalAlert ?
alert_fatal_i : 1'b0;
1091
1092 ////////////////////////////////////////////////////
1093 // Status Register //
1094 ////////////////////////////////////////////////////
1095 assign idle_o = clear_on_fatal ? 1'b0 : idle;
1096 assign idle_we_o = clear_on_fatal ? 1'b1 : idle_we;
1097 assign stall_o = clear_on_fatal ? 1'b0 : stall;
1098 assign stall_we_o = clear_on_fatal ? 1'b1 : stall_we;
1099 assign output_lost_o = clear_on_fatal ? 1'b0 : out-
put_lost;
1100 assign output_lost_we_o = clear_on_fatal ? 1'b1 : out-
put_lost_we;
1101 assign output_valid_o = clear_on_fatal ? 1'b0 : out-
put_valid;
1102 assign output_valid_we_o = clear_on_fatal ? 1'b1 : out-
put_valid_we;
1103 assign input_ready_o = clear_on_fatal ? 1'b0 : in-
put_ready;
1104 assign input_ready_we_o = clear_on_fatal ? 1'b1 : in-
put_ready_we;
1105
1106 ////////////////////////////////////////////////////
1107 // Trigger Register //
1108
1108 ////////////////////////////////////////////////////
1109 // Most triggers are only ever cleared by control. Fatal
alerts clear all bits in the trigger
1110 // register.
1111 assign start_we_o = clear_on_fatal ? 1'b1 :
start_we;
1112 assign key_iv_data_in_clear_we_o = clear_on_fatal ? 1'b1 :
key_iv_data_in_clear_we;
1113 assign data_out_clear_we_o = clear_on_fatal ? 1'b1 :
data_out_clear_we;
1114
1115 // If configured, trigger the reseeding of the PRNGs used
for clearing and masking purposes after
1116 // the key has been updated.
1117 assign prng_reseed_o = clear_on_fatal ? 1'b0 :
1118   key_init_new_pulse ? 1'b1 : 1'b0;
1119 assign prng_reseed_we_o = clear_on_fatal ?
1'b1 :
1120   key_init_new_pulse ?
key_touch_forces_reseed_i : prng_reseed_we;
1121
1122 ////////////////////////////////////////////////////
1123 // PRNG Reseeding Counter //
1124 ////////////////////////////////////////////////////
1125
1125 // Count the number of blocks since the start of the message
to determine when the masking PRNG
1126 // inside the cipher core needs to be reseeded.
1127 if (SecMasking) begin : gen_block_ctr
1128   logic block_ctr_set;
1129   logic [BlockCtrWidth-1:0] block_ctr_d, block_ctr_q;
1130   logic [BlockCtrWidth-1:0] block_ctr_set_val,
block_ctr_decr_val;
1131
1132   assign block_ctr_expr = block_ctr_q == '0;
1133   assign block_ctr_set = ctrl_we_q | (block_ctr_decr &
(block_ctr_expr | cipher_prng_reseed_i));
1134
1135   assign block_ctr_set_val = prng_reseed_rate_i == PER_1 ?
'0 :
1136     prng_reseed_rate_i == PER_64 ?
BlockCtrWidth'(63) :
1137     prng_reseed_rate_i == PER_8K ?
BlockCtrWidth'(8191) : '0;
1138
1139   assign block_ctr_decr_val = block_ctr_q - BlockC-
trWidth'(1);
1140
1141   assign block_ctr_d = block_ctr_set ? block_ctr_set_val :
block_ctr_decr ? block_ctr_decr_val :
block_ctr_q;
1142
1143   always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_block_ctr
1144     if (!rst_ni) begin
1145       block_ctr_q <= '0;
1146     end else begin
1147       block_ctr_q <= block_ctr_d;
1148     end
1149   end
1150 end
1151
1152 end else begin : gen_no_block_ctr
1153   assign block_ctr_expr = 1'b0;
1154
1155   // Tie off unused signals.
1156   logic unused_block_ctr_decr;
1157   prs_rate_e unused_prng_reseed_rate;
1158   logic unused_cipher_prng_reseed;
1159   assign unused_block_ctr_decr = block_ctr_decr;
1160   assign unused_prng_reseed_rate = prng_reseed_rate_i;
1161   assign unused_cipher_prng_reseed = cipher_prng_reseed_i;
1162 end
1163
1164 ////////////////////////////////////////////////////
1165 // Assertions //
1166 ////////////////////////////////////////////////////
1167
1168 // Create a lint error to reduce the risk of accidentally
disabling the masking.
1169 `ASSERT_STATIC_LINT_ERROR(AesControlFsmSecMaskingNonDefault,
SecMasking == 1)
1170
1171 // Selectors must be known/valid
1172 `ASSERT(AesModeValid, !ctrl_err_storage_i |-> mode_i inside
{
1173   AES_ECB,
1174   AES_CBC,
1175   AES_CFB,
1176   AES_OFB,
1177   AES_CTR,
1178   AES_GCM,
1179   AES_NONE
1180 })
1181 `ASSERT(AesOpValid, !ctrl_err_storage_i |-> op_i inside {
1182   AES_ENC,
1183   AES_DEC

```



```

1184     })
1185     `ASSERT(AesCiphOpValid, !cipher_op_err |-> cipher_op_i in-
side {
1186         CIPH_FWD,
1187         CIPH_INV
1188     })
1189     `ASSERT(AesControlStateValid, !alert_o |-> aes_ctrl_cs in-
side {
1190         CTRL_IDLE,
1191         CTRL_LOAD,
1192         CTRL_GHASH_READY,

```

```

1193         CTRL_PRNG_RESEED,
1194         CTRL_FINISH,
1195         CTRL_CLEAR_I,
1196         CTRL_CLEAR_CO
1197     })
1198
1199     // Check parameters
1200     `ASSERT_INIT(AesNumSlicesCtr, NumSlicesCtr == 8)
1201
1202 endmodule

```

OpenTitan HMAC-SHA256

What. HMAC(키 있는 해시)과 SHA-256 을 하나의 블록으로. FIFO · 레지스터 인터페이스 포함.

Why it exists. 해시 자체보다 스트리밍 인터페이스가 블록의 대부분을 차지한다. 메시지 길이가 임의이고 소프트웨어가 조각내 밀어 넣으므로, FIFO · 바이트 정렬 · 마지막 블록 패딩 · 중단/재개를 전부 하드웨어가 처리해야 한다.

학부 복습 · **EECS concepts.** HMAC 구성(ipad/opad) · 스트림 처리 · 백프레셔 · 바이트 인에이블(strobe) 처리.

석사 수준 · **Practice note.** “알고리즘은 60%, 인터페이스가 40%”가 아니다. 대부분의 블록에서 인터페이스가 **70%** 이상이다. 일정 산정에서 이것을 놓치면 항상 지연된다.

4 files • 4,632 lines

API — hmac.sv

Table 207. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — module / interface / package (1개)

이름	종류
hmac	module

Table 208. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — parameter (2개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1

Table 209. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — localparam (3개)

상수	형/폭	값
NumInBufBits	int	\$bits(keymgr_pkg::hw_key_req_t)
MsgFifoDepth	int	32
AlertIsFatal	logic [NumAlerts-1:0]	{1'b1}

Table 210. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
intr_hmac_done_o	output	logic

포트	방향	형/폭
intr_fifo_empty_o	output	logic
intr_hmac_err_o	output	logic
idle_o	output	prim_mubi_pkg::mubi4_t

API — hmac_core.sv

Table 211. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv — module / interface / package (1개)

이름	종류
hmac_core	module

Table 212. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv — localparam (7개)

상수	형/폭	값
BlockSizeSHA256	int unsigned	512
BlockSizeSHA512	int unsigned	1024
BlockSizeBitsSHA256	int unsigned	$\$clog2(\text{BlockSizeSHA256})$
BlockSizeBitsSHA512	int unsigned	$\$clog2(\text{BlockSizeSHA512})$
HashWordBitsSHA256	int unsigned	$\$clog2(\text{\$bits}(\text{sha_word32_t}))$
BlockSizeSHA256in64	bit [63:0]	64'(BlockSizeSHA256)
BlockSizeSHA512in64	bit [63:0]	64'(BlockSizeSHA512)

Table 213. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
secret_key_i	input	[1023:0]
hmac_en_i	input	
digest_size_i	input	digest_mode_e
key_length_i	input	key_length_e
reg_hash_start_i	input	
reg_hash_stop_i	input	
reg_hash_continue_i	input	
reg_hash_process_i	input	
hash_done_o	output	logic
sha_hash_start_o	output	logic
sha_hash_continue_o	output	logic
sha_hash_process_o	output	logic
sha_hash_done_i	input	

포트	방향	형/폭
sha_rvalid_o	output	logic
sha_rdata_o	output	sha_fifo32_t
sha_rready_i	input	
fifo_rvalid_i	input	
fifo_rdata_i	input	sha_fifo32_t
fifo_rready_o	output	logic
fifo_wsel_o	output	logic
fifo_wvalid_o	output	logic
fifo_wdata_sel_o	output	logic [3:0]
fifo_wready_i	input	
message_length_i	input	[63:0]
sha_message_length_o	output	[63:0]
idle_o	output	logic

API — hmac_reg_pkg.sv

Table 214. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_pkg.sv — module / interface / package (1개)

이름	종류
hmac_reg_pkg	package

Table 215. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_pkg.sv — parameter (134개) — 앞 60개만 표시(전체 134개, 본문 소스 참조)

파라미터	형/폭	기본값
NumDigestWords	int	16
NumKeyWords	int	32
NumAlerts	int	1
BlockAw	int	13
NumRegs	int	59
HMAC_INTR_STATE_OFFSET	logic [BlockAw-1:0]	13'h 0
HMAC_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	13'h 4
HMAC_INTR_TEST_OFFSET	logic [BlockAw-1:0]	13'h 8
HMAC_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	13'h c
HMAC_CFG_OFFSET	logic [BlockAw-1:0]	13'h 10
HMAC_CMD_OFFSET	logic [BlockAw-1:0]	13'h 14
HMAC_STATUS_OFFSET	logic [BlockAw-1:0]	13'h 18
HMAC_ERR_CODE_OFFSET	logic [BlockAw-1:0]	13'h 1c
HMAC_WIPE_SECRET_OFFSET	logic [BlockAw-1:0]	13'h 20

파라미터	형/폭	기본값
HMAC_KEY_0_OFFSET	logic [BlockAw-1:0]	13'h 24
HMAC_KEY_1_OFFSET	logic [BlockAw-1:0]	13'h 28
HMAC_KEY_2_OFFSET	logic [BlockAw-1:0]	13'h 2c
HMAC_KEY_3_OFFSET	logic [BlockAw-1:0]	13'h 30
HMAC_KEY_4_OFFSET	logic [BlockAw-1:0]	13'h 34
HMAC_KEY_5_OFFSET	logic [BlockAw-1:0]	13'h 38
HMAC_KEY_6_OFFSET	logic [BlockAw-1:0]	13'h 3c
HMAC_KEY_7_OFFSET	logic [BlockAw-1:0]	13'h 40
HMAC_KEY_8_OFFSET	logic [BlockAw-1:0]	13'h 44
HMAC_KEY_9_OFFSET	logic [BlockAw-1:0]	13'h 48
HMAC_KEY_10_OFFSET	logic [BlockAw-1:0]	13'h 4c
HMAC_KEY_11_OFFSET	logic [BlockAw-1:0]	13'h 50
HMAC_KEY_12_OFFSET	logic [BlockAw-1:0]	13'h 54
HMAC_KEY_13_OFFSET	logic [BlockAw-1:0]	13'h 58
HMAC_KEY_14_OFFSET	logic [BlockAw-1:0]	13'h 5c
HMAC_KEY_15_OFFSET	logic [BlockAw-1:0]	13'h 60
HMAC_KEY_16_OFFSET	logic [BlockAw-1:0]	13'h 64
HMAC_KEY_17_OFFSET	logic [BlockAw-1:0]	13'h 68
HMAC_KEY_18_OFFSET	logic [BlockAw-1:0]	13'h 6c
HMAC_KEY_19_OFFSET	logic [BlockAw-1:0]	13'h 70
HMAC_KEY_20_OFFSET	logic [BlockAw-1:0]	13'h 74
HMAC_KEY_21_OFFSET	logic [BlockAw-1:0]	13'h 78
HMAC_KEY_22_OFFSET	logic [BlockAw-1:0]	13'h 7c
HMAC_KEY_23_OFFSET	logic [BlockAw-1:0]	13'h 80
HMAC_KEY_24_OFFSET	logic [BlockAw-1:0]	13'h 84
HMAC_KEY_25_OFFSET	logic [BlockAw-1:0]	13'h 88
HMAC_KEY_26_OFFSET	logic [BlockAw-1:0]	13'h 8c
HMAC_KEY_27_OFFSET	logic [BlockAw-1:0]	13'h 90
HMAC_KEY_28_OFFSET	logic [BlockAw-1:0]	13'h 94
HMAC_KEY_29_OFFSET	logic [BlockAw-1:0]	13'h 98
HMAC_KEY_30_OFFSET	logic [BlockAw-1:0]	13'h 9c
HMAC_KEY_31_OFFSET	logic [BlockAw-1:0]	13'h a0
HMAC_DIGEST_0_OFFSET	logic [BlockAw-1:0]	13'h a4
HMAC_DIGEST_1_OFFSET	logic [BlockAw-1:0]	13'h a8
HMAC_DIGEST_2_OFFSET	logic [BlockAw-1:0]	13'h ac

파라미터	형/폭	기본값
HMAC_DIGEST_3_OFFSET	logic [BlockAw-1:0]	13'h b0
HMAC_DIGEST_4_OFFSET	logic [BlockAw-1:0]	13'h b4
HMAC_DIGEST_5_OFFSET	logic [BlockAw-1:0]	13'h b8
HMAC_DIGEST_6_OFFSET	logic [BlockAw-1:0]	13'h bc
HMAC_DIGEST_7_OFFSET	logic [BlockAw-1:0]	13'h c0
HMAC_DIGEST_8_OFFSET	logic [BlockAw-1:0]	13'h c4
HMAC_DIGEST_9_OFFSET	logic [BlockAw-1:0]	13'h c8
HMAC_DIGEST_10_OFFSET	logic [BlockAw-1:0]	13'h cc
HMAC_DIGEST_11_OFFSET	logic [BlockAw-1:0]	13'h d0
HMAC_DIGEST_12_OFFSET	logic [BlockAw-1:0]	13'h d4
HMAC_DIGEST_13_OFFSET	logic [BlockAw-1:0]	13'h d8

API — hmac_reg_top.sv

Table 216. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv — module / interface / package (1개)

이름	종류
hmac_reg_top	module

Table 217. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	13
DW	int	32
DBW	int	DW/8

Table 218. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
tl_win_o	output	tlul_pkg::tl_h2d_t
tl_win_i	input	tlul_pkg::tl_d2h_t
reg2hw	output	hmac_reg_pkg::hmac_reg2hw_t
hw2reg	input	hmac_reg_pkg::hmac_hw2reg_t
intg_err_o	output	logic

1. lowRISC_opentitan/hw/ip/hmac/rtl/hmac.sv (968 lines)

```
1 // Copyright lowRISC contributors (OpenTitan project).
```

```
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // HMAC/SHA-2 256/384/512
6
```

```

7 `include "prim_assert.sv"
8
9 module hmac
10   import prim_sha2_pkg::*;
11   import hmac_reg_pkg::*;
12 # (
13   parameter logic [NumAlerts-1:0] AlertAsyncOn = {Num-
mAlerts{1'b1}},
14   // Number of cycles a differential skew is tolerated on the
alert signal
15   parameter int unsigned AlertSkewCycles = 1
16 ) (
17   input clk_i,
18   input rst_ni,
19
20   input tlul_pkg::tl_h2d_t tl_i,
21   output tlul_pkg::tl_d2h_t tl_o,
22
23   input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
24   output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
25
26   // Key manager (keymgr) key sideload interface
27   input keymgr_pkg::hw_key_req_t keymgr_key_i,
28
29   output logic intr_hmac_done_o,
30   output logic intr_fifo_empty_o,
31   output logic intr_hmac_err_o,
32
33   output prim_mubi_pkg::mubi4_t idle_o
34 );
35
36 // TODO(#31026): Consume the key from the keymgr_dpe
37 localparam int NumInBufBits = $bits(keymgr-
gr_pkg::hw_key_req_t);
38 keymgr_pkg::hw_key_req_t unused_key;
39 prim_buf #(
40   .Width (NumInBufBits)
41 ) u_anchor_buf (
42   .in_i (keymgr_key_i),
43   .out_o (unused_key)
44 );
45
46 ///////////////////////////////////////////////////
47 // Signal declarations //
48 ///////////////////////////////////////////////////
49 hmac_reg2hw_t reg2hw;
50 hmac_hw2reg_t hw2reg;
51
52 tlul_pkg::tl_h2d_t tl_win_h2d;
53 tlul_pkg::tl_d2h_t tl_win_d2h;
54
55 logic [1023:0] secret_key, secret_key_d;
56
57 // Logic will support key length <= block size
58 // Will default to key length = block size, if key length >
block size or unsupported value
59 key_length_e key_length_supplied, key_length;
60
61 logic wipe_secret;
62 logic [31:0] wipe_v;
63
64 logic fifo_rvalid;
65 logic fifo_rready;
66 sha_fifo32_t fifo_rdata;
67
68 logic fifo_wvalid, fifo_wready;
69 sha_fifo32_t fifo_wdata;
70 logic fifo_full;
71 logic fifo_empty;
72 logic [5:0] fifo_depth;
73
74 logic msg_fifo_req;
75 logic msg_fifo_gnt;
76 logic msg_fifo_we;
77 logic [31:0] msg_fifo_wdata;
78 logic [31:0] msg_fifo_wmask;
79 logic [31:0] msg_fifo_rdata;
80 logic [1:0] msg_fifo_rerror;
81 logic [31:0] msg_fifo_wdata_endian;
82 logic [31:0] msg_fifo_wmask_endian;
83
84 logic packer_ready;
85 logic packer_flush_done;
86
87 logic reg_fifo_wvalid;
88 sha_word32_t reg_fifo_wdata;
89 sha_word32_t reg_fifo_wmask;
90 logic hmac_fifo_wsel;
91 logic hmac_fifo_wvalid;
92 logic [3:0] hmac_fifo_wdata_sel;
93
94 logic shaf_rvalid;
95 sha_fifo32_t shaf_rdata;
96 logic shaf_rready;
97
98 logic sha_en;
99 logic hmac_en;
100 logic endian_swap;
101 logic digest_swap;

```

```

102 logic key_swap;
103
104 logic reg_hash_start;
105 logic sha_hash_start;
106 logic reg_hash_stop;
107 logic reg_hash_continue;
108 logic sha_hash_continue;
109
110 logic hash_start; // hash_start is reg_hash_start
gated with extra checks
111 logic hash_continue; // hash_continue is
reg_hash_continue gated with extra checks
112 logic hash_process; // hash_process is
reg_hash_process gated with extra checks
113 logic hash_start_or_continue;
114 logic hash_done_event;
115 logic reg_hash_process;
116 logic sha_hash_process;
117 logic digest_on_blk;
118
119 logic reg_hash_done;
120 logic sha_hash_done;
121
122 logic [63:0] message_length, message_length_d;
123 logic [63:0] sha_message_length;
124
125 err_code_e err_code;
126 logic err_valid;
127
128 logic invalid_config; // HMAC/SHA-2 is configured with
invalid digest size/key length
129 logic invalid_config_atstart;
130
131 sha_word64_t [7:0] digest, digest_sw;
132 logic [7:0] digest_sw_we;
133
134 digest_mode_e digest_size, digest_size_supplied;
135 // this is the digest size captured into HMAC when it gets
started
136 digest_mode_e digest_size_started_d, digest_size_started_q;
137
138 hmac_reg2hw_cfg_reg_t cfg_reg;
139 logic cfg_block; // Prevents changing con-
fig
140
141 logic msg_allowed; // MSG_FIFO from software
is allowed
142
143 logic hmac_core_idle;
144 logic sha_core_idle;
145 logic hash_running;
146 logic idle;
147
148 ///////////////////////////////////////////////////
149 // Connect registers //
150 ///////////////////////////////////////////////////
151 assign hw2reg.status.fifo_full.d = fifo_full;
152 assign hw2reg.status.fifo_empty.d = fifo_empty;
153 assign hw2reg.status.fifo_depth.d = fifo_depth;
154 assign hw2reg.status.hmac_idle.d = idle;
155
156 typedef enum logic [1:0] {
157   DoneAwaitCmd,
158   DoneAwaitHashDone,
159   DoneAwaitMessageComplete,
160   DoneAwaitHashComplete
161 } done_state_e;
162
163 done_state_e done_state_d, done_state_q;
164
165 always_comb begin
166   done_state_d = done_state_q;
167   hash_done_event = 1'b0;
168
169   unique case (done_state_q)
170     DoneAwaitCmd: begin
171       if (sha_hash_process) begin
172         // SHA has been told to process the message, so sig-
nal *done* when the hash is done.
173         done_state_d = DoneAwaitHashDone;
174       end else if (reg_hash_stop) begin
175         // SHA has been told to stop, so first wait for the
current message block to be complete.
176         done_state_d = DoneAwaitMessageComplete;
177       end
178     end
179     DoneAwaitHashDone: begin
180       if (reg_hash_done) begin
181         hash_done_event = 1'b1;
182         done_state_d = DoneAwaitCmd;
183       end
184     end
185     DoneAwaitMessageComplete: begin
186       if (digest_on_blk) begin
187         // Once the digest is being computed for the com-
plete message block, wait for the hash to
188         // complete.
189         done_state_d = DoneAwaitHashComplete;
190       end
191     end
192   endcase
193 end

```

```

190     end
191
192     DoneAwaitHashComplete: begin
193         if (!hash_running) begin
194             hash_done_event = 1'b1;
195             done_state_d = DoneAwaitCmd;
196         end
197     end
198
199     default: ;
200 endcase
201 end
202
203 always_ff @(posedge clk_i or negedge rst_ni) begin
204     if (!rst_ni) begin
205         done_state_q <= DoneAwaitCmd;
206     end else begin
207         done_state_q <= done_state_d;
208     end
209 end
210
211 assign wipe_secret = reg2hw.wipe_secret.qe;
212 assign wipe_v      = reg2hw.wipe_secret.q;
213
214 // update secret key
215 always_comb begin : update_secret_key
216     secret_key_d = secret_key;
217     if (wipe_secret) begin
218         secret_key_d = {32(wipe_v)};
219     end else if (!cfg_block) begin
220         // Allow updating secret key only when the engine is in
Idle.
221         for (int i = 0; i < 32; i++) begin
222             if (reg2hw.key[31-i].qe) begin
223                 // swap byte endianness per secret key word if
key_swap = 1
224                 secret_key_d[32*i+:32] = conv_endi-
an32(reg2hw.key[31-i].q, key_swap);
225             end
226         end
227     end
228 end
229
230 always_ff @(posedge clk_i or negedge rst_ni) begin
231     if (!rst_ni) secret_key <= '0;
232     else        secret_key <= secret_key_d;
233 end
234
235 for (genvar i = 0; i < 32; i++) begin : gen_key
236     assign hw2reg.key[31-i].d = '0;
237 end
238
239 // Retain the previous digest in CSRs until HMAC is actually
started with a valid configuration
240 always_comb begin : assign_digest_reg
241     // default
242     // digest SW -> HW
243     digest_sw = '0;
244     digest_sw_we = '0;
245     // digest HW -> SW
246     hw2reg.digest = '0;
247
248     for (int i = 0; i < 8; i++) begin
249         // digest SW -> HW (depends on digest size configured
even before starting/enabling)
250
251         // capturing the intermediate digests written by SW when
restoring context into the SHA-2
252         // engine before it is started
253         if (digest_size == SHA2_256) begin
254             digest_sw[i][31:0] = conv_endian32(reg2hw.digest[i].q,
digest_swap);
255             digest_sw_we[i] = reg2hw.digest[i].qe;
256         end else if ((digest_size == SHA2_384) || (digest_size
== SHA2_512)) begin
257             // digest SW -> HW
258             digest_sw[i][63:32] = reg2hw.digest[2*i].qe ?
conv_endi-
an32(reg2hw.digest[2*i].q, digest_swap) :
259             digest[i][63:32];
260             digest_sw[i][31:0] = reg2hw.digest[2*i+1].qe ?
conv_endi-
an32(reg2hw.digest[2*i+1].q, digest_swap) :
261             digest[i][31:0];
262             digest_sw_we[i] = reg2hw.digest[2*i].qe |
reg2hw.digest[2*i+1].qe;
263         end
264         // digest HW -> SW (depends on configuration that has
been started)
265         if (digest_size_started_q == SHA2_256) begin
266             hw2reg.digest[i].d = conv_endian32(digest[i][31:0],
digest_swap);
267             // replicate digest[0..7] into digest[8..15]. Di-
gest[8..15] are irrelevant for SHA2_256,
268             // but this ensures all digest CSRs are wiped out with
random value (at wipe_secret)
269
270             // across different configurations.
271             hw2reg.digest[i+8].d = conv_endian32(digest[i][31:0],
digest_swap);
272             end else if ((digest_size_started_q == SHA2_384) || (di-
gest_size_started_q == SHA2_512)) begin
273                 // digest HW -> SW
274                 // digest swap only within each 32-bit word of the 64-
bit digest word, not digest swap
275                 // on the entire 64-bit digest word
276                 hw2reg.digest[2*i].d = conv_endian32(digest[i]
[63:32], digest_swap);
277                 hw2reg.digest[2*i+1].d = conv_endian32(digest[i]
[31:0], digest_swap);
278                 end else begin // for SHA2_None
279                     // to ensure secret wiping is always passed to digest
CSRs
280                     hw2reg.digest[i].d = conv_endian32(digest[i][31:0],
digest_swap);
281                     hw2reg.digest[i+8].d = conv_endian32(digest[i][31:0],
digest_swap);
282                     end
283                     end
284                     end
285                     end
286                     end
287                     logic unused_cfg_qe;
288                     assign unused_cfg_qe = ^{cfg_reg.sha_en.qe,
cfg_reg.hmac_en.qe,
289                     cfg_reg.endian_swap.qe, cfg_reg.di-
gest_swap.qe,
290                     cfg_reg.key_swap.qe,   cfg_reg.di-
gest_size.qe,
291                     cfg_reg.key_length.qe };
292                     assign sha_en = cfg_reg.sha_en.q;
293                     assign hmac_en = cfg_reg.hmac_en.q;
294                     assign digest_size_supplied = di-
gest_mode_e'(cfg_reg.digest_size.q);
295                     always_comb begin : cast_digest_size
296                         digest_size = SHA2_None;
297                         unique case (digest_size_supplied)
298                             SHA2_256: digest_size = SHA2_256;
299                             SHA2_384: digest_size = SHA2_384;
300                             SHA2_512: digest_size = SHA2_512;
301                             // unsupported digest size values are mapped to
SHA2_None
302                             // if HMAC/SHA-2 is triggered to start with this digest
size, it is blocked
303                             // and an error is signalled to SW
304                             default: digest_size = SHA2_None;
305                         endcase
306                         end
307                         // Hold the previous digest size till HMAC is started with
the new digest size configured
308                         assign digest_size_started_d = (hash_start_or_continue) ?
digest_size : digest_size_started_q;
309                         always_ff @(posedge clk_i or negedge rst_ni) begin
310                             if (!rst_ni) digest_size_started_q <= SHA2_None;
311                             else        digest_size_started_q <= di-
gest_size_started_d;
312                             end
313                             assign key_length_supplied =
key_length_e'(cfg_reg.key_length.q);
314                             always_comb begin : cast_key_length
315                                 key_length = Key_None;
316                                 unique case (key_length_supplied)
317                                     Key_128: key_length = Key_128;
318                                     Key_256: key_length = Key_256;
319                                     Key_384: key_length = Key_384;
320                                     Key_512: key_length = Key_512;
321                                     Key_1024: key_length = Key_1024;
322                                     // unsupported key length values are mapped to Key_None
323                                     // if HMAC (not SHA-2) is triggered to start with this
key length, it is blocked
324                                     // and an error is signalled to SW
325                                     default: key_length = Key_None;
326                                 endcase
327                                 end
328                                 assign endian_swap = cfg_reg.endian_swap.q;
329                                 assign digest_swap = cfg_reg.digest_swap.q;
330                                 assign key_swap = cfg_reg.key_swap.q;
331                                 assign hw2reg.cfg.hmac_en.d = cfg_reg.hmac_en.q;
332                                 assign hw2reg.cfg.sha_en.d = cfg_reg.sha_en.q;
333                                 assign hw2reg.cfg.digest_size.d = di-
gest_mode_e'(digest_size);
334                                 assign hw2reg.cfg.key_length.d = key_length_e'(key_length);
335                                 assign hw2reg.cfg.endian_swap.d = cfg_reg.endian_swap.q;
336                                 assign hw2reg.cfg.digest_swap.d = cfg_reg.digest_swap.q;
337                                 assign hw2reg.cfg.key_swap.d = cfg_reg.key_swap.q;
338                                 assign reg_hash_start = reg2hw.cmd.hash_start.qe &
reg2hw.cmd.hash_start.q;
339                                 assign reg_hash_stop = reg2hw.cmd.hash_stop.qe &
reg2hw.cmd.hash_stop.q;

```



```

351 assign reg_hash_continue = reg2hw.cmd.hash_continue.qe &
reg2hw.cmd.hash_continue.q;
352 assign reg_hash_process = reg2hw.cmd.hash_process.qe &
reg2hw.cmd.hash_process.q;
353
354 // Error code register
355 assign hw2reg.err_code.de = err_valid;
356 assign hw2reg.err_code.d = err_code;
357
358 // Control signals
359 // Control signals
360 // Control signals

361 assign hash_start = reg_hash_start & sha_en &
~cfg_block & ~invalid_config;

362 assign hash_continue = reg_hash_continue & sha_en &
~cfg_block & ~invalid_config;

363 assign hash_process = reg_hash_process & sha_en &
cfg_block & ~invalid_config;
364 assign hash_start_or_continue = hash_start | hash_continue;
365
366 always_ff @(posedge clk_i or negedge rst_ni) begin
367   if (!rst_ni) begin
368     cfg_block <= '0;
369   end else if (hash_start_or_continue) begin
370     cfg_block <= 1'b 1;
371   end else if (reg_hash_done || reg_hash_stop) begin
372     cfg_block <= 1'b 0;
373   end
374 end
375 // Hold the configuration during the process
376 always_ff @(posedge clk_i or negedge rst_ni) begin
377   if (!rst_ni) begin
378     cfg_reg <= '{
379       hmac_en: '{
380         q: 1'b0,
381         qe: 1'b0
382       },
383       sha_en: '{
384         q: 1'b0,
385         qe: 1'b0
386       },
387       endian_swap: '{
388         q: HMAC_CFG_ENDIAN_SWAP_RESVAL,
389         qe: 1'b0
390       },
391       digest_swap: '{
392         q: HMAC_CFG_DIGEST_SWAP_RESVAL,
393         qe: 1'b0
394       },
395       key_swap: '{
396         q: HMAC_CFG_KEY_SWAP_RESVAL,
397         qe: 1'b0
398       },
399       digest_size: '{
400         q: HMAC_CFG_DIGEST_SIZE_RESVAL,
401         qe: 1'b0
402       },
403       key_length: '{
404         q: HMAC_CFG_KEY_LENGTH_RESVAL,
405         qe: 1'b0
406       },
407       default: '0
408     };
409   end else if (!cfg_block && reg2hw.cfg.hmac_en.qe) begin
410     cfg_reg <= reg2hw.cfg;
411   end
412 end
413
414 // Open up the MSG_FIFO from the TL-UL port when it is ready
415 always_ff @(posedge clk_i or negedge rst_ni) begin
416   if (!rst_ni) begin
417     msg_allowed <= '0;
418   end else if (hash_start_or_continue) begin
419     msg_allowed <= 1'b 1;
420   end else if (packer_flush_done) begin
421     msg_allowed <= 1'b 0;
422   end
423 end
424
425 // Interrupts
426 // Interrupts
427 // Interrupts
428
429 // instantiate interrupt hardware primitive
430 prim_intr_hw #(.Width(1)) intr_hw_hmac_done (
431   .clk_i,
432   .rst_ni,
433   .event_intr_i (hash_done_event),
434   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.hmac_done.q),
435   .reg2hw_intr_test_q_i (reg2hw.intr_test.hmac_done.q),
436   .reg2hw_intr_test_qe_i (reg2hw.intr_test.hmac_done.qe),
437   .reg2hw_intr_state_q_i (reg2hw.intr_state.hmac_done.q),
438   .hw2reg_intr_state_de_o (hw2reg.intr_state.hmac_done.de),
439   .hw2reg_intr_state_d_o (hw2reg.intr_state.hmac_done.d),
440   .intr_o (intr_hmac_done_o)
441 );
442
443 // FIFO empty interrupt

```

```

444 //
445 // The FIFO empty interrupt is **not useful** for software
446 // - The HMAC block is running in HMAC mode and performing
447 //   the second round of computing the
448 //   final hash of the outer key as well as the result of
449 //   the first round using the inner key.
450 // - The FIFO is then managed entirely by the hardware.
451 // - The FIFO is currently not writeable by software.
452 // - Software has already written the Process command. The
453 //   HMAC block will now empty the
454 //   FIFO and load its content into the SHA2 core, add the
455 //   padding and then perform
456 //   the final hashing operation. Software cannot append the
457 //   message further.
458 // - Software has written the Stop command. The HMAC block
459 //   will not wait for further input from
460 //   software after finishing the current block.
461 // - The FIFO empty interrupt can be **useful** for software
462 //   in particular if:
463 // - The FIFO was completely full previously. However, un-
464 //   less the HMAC block is currently
465 //   processing a block, it always empties the message FIFO
466 //   faster than software can fill it up,
467 //   meaning the message FIFO is empty most of the time.
468 // Note, the empty status is signaled only
469
470 // once after the FIFO was completely full. The FIFO needs
471 // to be full again for the empty
472 // status to be signaled again next time it's empty.
473 logic status_fifo_empty, fifo_empty_gate;
474 logic fifo_empty_negedge, fifo_empty_q;
475 logic fifo_full_posedge, fifo_full_q;
476 logic fifo_full_seen_d, fifo_full_seen_q;
477 assign fifo_empty_negedge = fifo_empty_q & ~fifo_empty;
478 assign fifo_full_posedge = ~fifo_full_q & fifo_full;
479
480 // Track whether the FIFO was full after being empty. We
481 // clear the tracking:
482 // - When receiving the Start, Continue, Process or Stop
483 //   command. This is to start over for the
484 //   next message.
485 // - When seeing a negative edge on the empty signal. This
486 //   signals that software has reacted to
487 //   the interrupt and is filling up the FIFO again.
488 assign fifo_full_seen_d =
489   reg_hash_start || reg_hash_continue ||
490   reg_hash_process || reg_hash_stop ? 1'b 0 :
491   fifo_empty_negedge ? 1'b 0 :
492   fifo_full_posedge ? 1'b 1 :
493   fifo_full_seen_q;
494
495 // The interrupt is gated unless software is actually al-
496 // lowed to write the FIFO and the FIFO was
497 // full before.
498 assign fifo_empty_gate = ~msg_allowed || ~fifo_full_seen_q;
499
500 assign status_fifo_empty = fifo_empty_gate ? 1'b 0 :
501   fifo_empty;
502
503 always_ff @(posedge clk_i or negedge rst_ni) begin
504   if (!rst_ni) begin
505     fifo_empty_q <= 1'b 0;
506     fifo_full_q <= 1'b 0;
507     fifo_full_seen_q <= 1'b 0;
508   end else begin
509     fifo_empty_q <= status_fifo_empty;
510     fifo_full_q <= fifo_full;
511     fifo_full_seen_q <= fifo_full_seen_d;
512   end
513 end
514
515 prim_intr_hw #(.Width(1)) intr_hw_hmac_err (
516   .Width(1),
517   .IntrT("Status")
518 ) intr_hw_fifo_empty (
519   .clk_i,
520   .rst_ni,
521   .event_intr_i (status_fifo_empty),
522   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.fifo_empty.q),
523   .reg2hw_intr_test_q_i (reg2hw.intr_test.fifo_empty.q),
524   .reg2hw_intr_test_qe_i (reg2hw.intr_test.fifo_empty.qe),
525   .reg2hw_intr_state_q_i (reg2hw.intr_state.fifo_empty.q),
526   .hw2reg_intr_state_de_o (hw2reg.intr_state.fifo_empty.de),
527   .hw2reg_intr_state_d_o (hw2reg.intr_state.fifo_empty.d),
528   .intr_o (intr_fifo_empty_o)
529 );
530
531 prim_intr_hw #(.Width(1)) intr_hw_hmac_err (
532   .clk_i,
533   .rst_ni,
534   .event_intr_i (err_valid),
535   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.hmac_err.q),
536   .reg2hw_intr_test_q_i (reg2hw.intr_test.hmac_err.q),
537   .reg2hw_intr_test_qe_i (reg2hw.intr_test.hmac_err.qe),
538   .reg2hw_intr_state_q_i (reg2hw.intr_state.hmac_err.q),
539   .hw2reg_intr_state_de_o (hw2reg.intr_state.hmac_err.de),
540   .hw2reg_intr_state_d_o (hw2reg.intr_state.hmac_err.d),
541   .intr_o (intr_hmac_err_o)
542 );

```

```

525
526 // Instances //
527 // Instances //
528 // Instances //
529
530 assign msg_fifo_rdata = '1; // Return all F
531 assign msg_fifo_rerror = '1; // Return error for read access
532 assign msg_fifo_gnt = msg_fifo_req & ~hmac_fifo_wsel &
packer_ready;
533
534 // Unused Signals //
535 // Unused Signals //
536 // Unused Signals //
537 logic unused_signals;
538 assign unused_signals = ^{reg_fifo_wmask[7:1],
reg_fifo_wmask[15:9],
539 reg_fifo_wmask[23:17],
reg_fifo_wmask[31:25]};
540
541 // FIFO control: from packer into message FIFO
542 sha_fifo32_t reg_fifo_wentry;
543 assign reg_fifo_wentry.data = conv_endian32(reg_fifo_wdata,
1'b1); // always convert
544 assign reg_fifo_wentry.mask = {reg_fifo_wmask[0],
reg_fifo_wmask[8],
545 reg_fifo_wmask[16],
reg_fifo_wmask[24]};
546 assign fifo_empty = ~fifo_rvalid;
547 assign fifo_wvalid = (hmac_fifo_wsel && fifo_wready) ?
hmac_fifo_wvalid : reg_fifo_wvalid;
548
549 logic index;
550 always_comb begin : select_fifo_wdata
551 // default when !hmac_fifo_wsel
552 index = 1'b0;
553 fifo_wdata = reg_fifo_wentry;
554
555 if (hmac_fifo_wsel) begin
556 fifo_wdata = '0;
557 if (digest_size == SHA2_256) begin
558 // only reads out lower 32 bits of each digest word
and discards upper 32-bit zero padding
559 fifo_wdata = '{data: digest[hmac_fifo_wdata_sel[2:0]]
[31:0], mask: '1};
560 end else if ((digest_size == SHA2_384) || (digest_size
== SHA2_512)) begin
561 // reads out first upper 32 bits then lower 32 bits of
each digest word
562 index = !hmac_fifo_wdata_sel[0];
563 fifo_wdata = '{data: digest[hmac_fifo_wdata_sel[3:1]]
[32:index+:32], mask: '1};
564 end
565 end
566 end
567
568 // Extended for 1024-bit block
569 localparam int MsgFifoDepth = 32;
570 prim_fifo_sync #(
571 .Width ($bits(sha_fifo32_t)),
572 .Pass (1'b1),
573 .Depth (MsgFifoDepth),
574 .NeverClears (1'b1)
575 ) u_msg_fifo (
576 .clk_i,
577 .rst_ni,
578 .clr_i (1'b0),
579
580 .wvalid_i(fifo_wvalid & sha_en),
581 .wready_o(fifo_wready),
582 .wdata_i (fifo_wdata),
583
584 .depth_o (fifo_depth),
585 .full_o (fifo_full),
586
587 .rvalid_o(fifo_rvalid),
588 .rready_i(fifo_rready),
589 .rdata_o (fifo_rdata),
590 .err_o ()
591 );
592
593 // TL ADAPTER SRAM
594 tlul_adapter_sram #(
595 .SramAw (9),
596 .SramDw (32),
597 .Outstanding (1),
598 .ByteAccess (1),
599 .ErrOnRead (1)
600 ) u_tlul_adapter (
601 .clk_i,
602 .rst_ni,
603 .tl_i (tl_win_h2d),
604 .tl_o (tl_win_d2h),
605 .en_ifetch_i (prim_mubi_pkg::MuBi4False),
606 .req_o (msg_fifo_req),
607 .req_type_o ( ),
608 .gnt_i (msg_fifo_gnt),
609 .we_o (msg_fifo_we),
610 .addr_o ( ), // Doesn't
care the address

```

```

611 // other
than sub-word
612 .wdata_o (msg_fifo_wdata),
613 .wmask_o (msg_fifo_wmask),
614 .intg_error_o ( ),
615 .user_rsvd_o ( ),
616 .rdata_i (msg_fifo_rdata),
617 .rvalid_i (1'b0),
618 .rerror_i (msg_fifo_rerror),
619 .compound_txn_in_progress_o ( ),
620 .readback_en_i (prim_mubi_pkg::MuBi4False),
621 .readback_error_o ( ),
622 .wr_collision_i (1'b0),
623 .write_pending_i (1'b0)
624 );
625
626 // TL-UL to MSG_FIFO byte write handling
627 logic msg_write;
628
629 assign msg_write = msg_fifo_req & msg_fifo_we &
~hmac_fifo_wsel & msg_allowed;
630
631 logic {$clog2(32+1)-1:0} wmask_ones;
632
633 always_comb begin
634 wmask_ones = '0;
635 for (int i = 0; i < 32; i++) begin
636 wmask_ones = wmask_ones + msg_fifo_wmask[i];
637 end
638 end
639
640 // Calculate written message
641 always_ff @(posedge clk_i or negedge rst_ni) begin
642 if (!rst_ni) message_length <= '0;
643 else message_length <= message_length_d;
644 end
645
646 always_comb begin
647 message_length_d = message_length;
648 if (!cfg_block) begin
649 if (reg2hw.msg_length_lower.qe) begin
650 message_length_d[31:0] = reg2hw.msg_length_lower.q;
651 end
652 if (reg2hw.msg_length_upper.qe) begin
653 message_length_d[63:32] = reg2hw.msg_length_upper.q;
654 end
655 end
656
657 if (hash_start) begin
658 message_length_d = '0;
659 end else if (msg_write && sha_en && packer_ready) begin
660 message_length_d = message_length + 64'(wmask_ones);
661 end
662 end
663
664 assign hw2reg.msg_length_upper.d = message_length[63:32];
665 assign hw2reg.msg_length_lower.d = message_length[31:0];
666
667 // Convert endian here
668 // prim_packer always packs to the right, but SHA engine
assumes incoming
669 // to be big-endian, [31:24] comes first. So, the data is
reverted after
670 // prim_packer before the message fifo. here to reverse
if not big-endian
671 // before pushing to the packer.
672
673 assign msg_fifo_wdata_endian = conv_endian32(msg_fifo_wdata,
endian_swap);
674
675 assign msg_fifo_wmask_endian = conv_endian32(msg_fifo_wmask,
endian_swap);
676
677 prim_packer #(
678 .InW (32),
679 .OutW (32),
680 .EnProtection (1'b 0)
681 ) u_packer (
682 .clk_i,
683 .rst_ni,
684
685 .valid_i (msg_write & sha_en),
686 .data_i (msg_fifo_wdata_endian),
687 .mask_i (msg_fifo_wmask_endian),
688 .ready_o (packer_ready),
689
690 .valid_o (reg_fifo_wvalid),
691 .data_o (reg_fifo_wdata),
692 .mask_o (reg_fifo_wmask),
693 .ready_i (fifo_wready & ~hmac_fifo_wsel),
694
695 .flush_i (hash_process),
696 .flush_done_o (packer_flush_done), // ignore at this mo-
ment
697
698 .err_o () // Not used
699 );
700
701 hmac_core u_hmac (
702 .clk_i,

```

```

701 .rst_ni,
702 .secret_key_i (secret_key),
703 .hmac_en_i (hmac_en),
704 .digest_size_i (digest_size),
705 .key_length_i (key_length),
706
707 .reg_hash_start_i (hash_start),
708 .reg_hash_stop_i (reg_hash_stop),
709 .reg_hash_continue_i (hash_continue),
710
711 .reg_hash_process_i (packer_flush_done), // Trigger after
all msg written
712 .hash_done_o (reg_hash_done),
713 .sha_hash_start_o (sha_hash_start),
714 .sha_hash_continue_o (sha_hash_continue),
715 .sha_hash_process_o (sha_hash_process),
716 .sha_hash_done_i (sha_hash_done),
717
718 .sha_rvalid_o (shaf_rvalid),
719 .sha_rdata_o (shaf_rdata),
720 .sha_rready_i (shaf_rready),
721
722 .fifo_rvalid_i (fifo_rvalid),
723 .fifo_rdata_i (fifo_rdata),
724 .fifo_rready_o (fifo_rready),
725
726 .fifo_wsel_o (hmac_fifo_wsel),
727 .fifo_wvalid_o (hmac_fifo_wvalid),
728 .fifo_wdata_sel_o (hmac_fifo_wdata_sel),
729 .fifo_wready_i (fifo_wready),
730
731 .message_length_i (message_length),
732 .sha_message_length_o (sha_message_length),
733
734 .idle_o (hmac_core_idle)
735 );
736
737 // Instantiate SHA-2 256/384/512 engine
738 prim_sha2_32 #(
739     .MultimodeEn(1)
740 ) u_prim_sha2_512 (
741     .clk_i,
742     .rst_ni,
743     .wipe_secret_i (wipe_secret),
744     .wipe_v_i (wipe_v),
745     .fifo_rvalid_i (shaf_rvalid),
746     .fifo_rdata_i (shaf_rdata),
747     .fifo_rready_o (shaf_rready),
748     .sha_en_i (sha_en),
749     .hash_start_i (sha_hash_start),
750     .hash_stop_i (reg_hash_stop),
751     .hash_continue_i (sha_hash_continue),
752     .digest_mode_i (digest_size),
753     .hash_process_i (sha_hash_process),
754     .message_length_i (sha_message_length),
755     .digest_i (digest_sw),
756     .digest_we_i (digest_sw_we),
757     .digest_o (digest),
758     .hash_running_o (hash_running),
759     .digest_on_blk_o (digest_on_blk),
760     .hash_done_o (sha_hash_done),
761     .idle_o (sha_core_idle)
762 );
763
764 // Register top
765 logic [NumAlerts-1:0] alert_test, alerts;
766 hmac_reg_top u_reg (
767     .clk_i,
768     .rst_ni,
769     .tl_i,
770     .tl_o,
771
772     .tl_win_o (tl_win_h2d),
773     .tl_win_i (tl_win_d2h),
774
775     .reg2hw,
776     .hw2reg,
777
778     // SEC_CM: BUS_INTEGRITY
779     .intg_err_o (alerts[0])
780 );
781
782 // Alerts
783 assign alert_test = {
784     reg2hw.alert_test.q &
785     reg2hw.alert_test.qe
786 };
787
788 localparam logic [NumAlerts-1:0] AlertIsFatal = {1'b1};
789 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
790     prim_alert_sender #(
791         .AsyncOn(AlertAsyncOn[i]),
792         .SkewCycles(AlertSkewCycles),
793         .IsFatal(AlertIsFatal[i])
794     ) u_prim_alert_sender (
795         .clk_i,
796         .rst_ni,
797         .alert_test_i ( alert_test[i] ),
798         .alert_req_i ( alerts[0] ),
799         .alert_ack_o ( ),
800         .alert_state_o (
801             .alert_rx_i ( alert_rx_i[i] ),
802             .alert_tx_o ( alert_tx_o[i] )
803         );
804     end
805
806     //////////////////////////////////
807     // HMAC Error Handling //
808     //////////////////////////////////
809     logic hash_start_sha_disabled, update_seckey_inprocess;
810     logic hash_start_active; // 'reg_hash_start' or
'reg_hash_continue' set when hash already active
811     logic msg_push_not_allowed; // Message is received when
'hash_start_or_continue' isn't set
812
813     assign hash_start_sha_disabled = (reg_hash_start |
reg_hash_continue) & ~sha_en;
814     assign hash_start_active = (reg_hash_start |
reg_hash_continue) & cfg_block;
815     assign msg_push_not_allowed = msg_fifo_req & ~msg_allowed;
816
817     // Invalid/unconfigured HMAC/SHA-2: not configured/invalid
digest size or
818     // not configured/invalid key length for HMAC mode or
819     // key_length = 1024-bit for digest_size = SHA2_256 (max
512-bit is supported for SHA-2 256)
820     assign invalid_config = ((digest_size ==
SHA2_None) |
821         ((key_length == Key_None) &&
hmac_en) |
822         ((key_length == Key_1024) && (di-
gest_size == SHA2_256) && hmac_en));
823
824     // invalid_config at reg_hash_start or reg_hash_continue
will signal an error to the SW
825     assign invalid_config_atstart = (reg_hash_start ||
reg_hash_continue) & invalid_config;
826
827     always_comb begin
828         update_seckey_inprocess = 1'b0;
829         if (cfg_block) begin
830             for (int i = 0; i < 32; i++) begin
831                 if (reg2hw.key[i].qe) begin
832                     update_seckey_inprocess = update_seckey_inprocess |
1'b1;
833                 end
834             end
835         end else begin
836             update_seckey_inprocess = 1'b0;
837         end
838     end
839
840     // Update ERR_CODE register and interrupt only when no
pending interrupt.
841
842     // This ensures only the first event of the series of events
can be seen to sw.
843     // It is recommended that the software reads ERR_CODE re-
gister when interrupt
844     // is pending to avoid any race conditions.
845     assign err_valid = ~reg2hw.intr_state.hmac_err.q &
( hash_start_sha_disabled | up-
date_seckey_inprocess
846         | hash_start_active | msg_push_not_allowed
| invalid_config_atstart);
847
848     always_comb begin
849         // default
850         err_code = NoError;
851
852         priority case (1'b1)
853             // SwInvalidConfig has the highest priority: SW config-
ures HMAC incorrectly
854             invalid_config_atstart: begin
855                 err_code = SwInvalidConfig;
856             end
857
858             hash_start_sha_disabled: begin
859                 err_code = SwHashStartWhenShaDisabled;
860             end
861
862             hash_start_active: begin
863                 err_code = SwHashStartWhenActive;
864             end
865
866             msg_push_not_allowed: begin
867                 err_code = SwPushMsgWhenDisallowed;
868             end
869
870             update_seckey_inprocess: begin
871                 err_code = SwUpdateSecretKeyInProgress;
872             end
873
874             default: begin
875                 err_code = NoError;
876             end
877         endcase
878     end
879
880     //////////////////////////////////
881     // Idle output //

```

```

882 //////////////////////////////////////////////////
883 // TBD this should be connected later
884 // Idle: AND condition of:

885 // - packer empty: Currently no way to guarantee the packer
is empty.
886 // temporary, the logic uses packer output
(reg_fifo_wvalid)
887 // - MSG_FIFO --> fifo_rvalid
888 // - HMAC_CORE --> hmac_core_idle
889 // - SHA2_CORE --> sha_core_idle
890 // - Clean interrupt status
891 // ICEBOX(#12958): Revise prim_packer and replace
`reg_fifo_wvalid` to the
892 // empty status.
893 assign idle = !reg_fifo_wvalid && !fifo_rvalid
894 && hmac_core_idle && sha_core_idle;
895
896 prim_mubi_pkg::mubi4_t idle_q, idle_d;
897 assign idle_d = prim_mubi_pkg::mubi4_bool_to_mubi(idle);
898 always_ff @(posedge clk_i or negedge rst_ni) begin
899   if (!rst_ni) begin
900     idle_q <= prim_mubi_pkg::Mubi4False;
901   end else begin
902     idle_q <= idle_d;
903   end
904 end
905 assign idle_o = idle_q;
906
907 //////////////////////////////////////////////////
908 // Assertions, Assumptions, and Coverpoints //
909 //////////////////////////////////////////////////
910
911 `ifndef VERILATOR
912 `ifndef SYNTHESIS
913 // HMAC assumes TL-UL mask is byte-aligned.
914 property wmask_bytealign_p(wmask_byte, clk, rst_n);
915   @(posedge clk) disable iff (rst_n == 0)
916     msg_fifo_req & msg_fifo_we -> wmask_byte inside {0,
`1};
917 endproperty
918
919 for (genvar i = 0 ; i < 4; i++) begin:
gen_assert_wmask_bytealign

920   assert property (wmask_bytealign_p(msg_fifo_wmask[8*i+8],
clk_i, rst_ni));
921   end
922
923 // To pass FPV, this shouldn't add pragma translate_off even
these two signals
924 // are used in Assertion only
925 logic in_process;
926 always_ff @(posedge clk_i or negedge rst_ni) begin
927   if (!rst_ni) in_process <=
1'b0;
928   else if (hash_process || reg_hash_stop) in_process <=
1'b1;
929   else if (reg_hash_done) in_process <=
1'b0;
930 end
931
932 logic initiated;
933 always_ff @(posedge clk_i or negedge rst_ni) begin
934   if (!rst_ni) initiated <=
1'b0;
935   else if (hash_start_or_continue) initiated <=
1'b1;
936   else if (hash_process || reg_hash_stop) initiated <=
1'b0;
937 end
938
939 // the host doesn't write data after hash_process until
hash_start_or_continue.
940 `ASSERT(ValidWriteAssert, msg_fifo_req -> !in_process)
941
942 // Below condition is covered by the design (2020-02-19)
943 `ASSERT(ValidHashStartAssert, hash_start_or_continue -> !
initiated)
944 // `hash_process` or `reg_hash_stop` should be toggled and
paired with `hash_start_or_continue`
945 `ASSERT(ValidHashProcessAssert, (hash_process ||
reg_hash_stop) -> initiated)
946
947 // hmac_en should be modified only when the logic is Idle
948 `ASSERT(ValidHmacEnConditionAssert,
949   hmac_en != $past(hmac_en) -> !in_process && !initiated)
950
951 // When wipe_secret is high, sensitive internal variables
are cleared by extending the wipe
952 // value specified in the register
953 `ASSERT(WipeSecretKeyAssert,
954   wipe_secret == ($bits(secret_key) == ($bits(secret_key)/
$bits(wipe_v))$past(wipe_v)))
955
956 // All outputs should be known value after reset
957 `ASSERT_KNOWN(IntrHmacDoneOKnown, intr_hmac_done_o)
958 `ASSERT_KNOWN(IntrFifoEmptyOKnown, intr_fifo_empty_o)
959 `ASSERT_KNOWN(TLODValidKnown, tl_o.d_valid)

```

```

960 `ASSERT_KNOWN(TLOAReadyKnown, tl_o.a_ready)
961 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
962
963 `endif // SYNTHESIS
964 `endif // VERILATOR
965
966 // Alert assertions for reg_we onehot check
967 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
968 endmodule

```

2. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_core.sv (505 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // HMAC Core implementation
6
7 `include "prim_assert.sv"
8
9 module hmac_core import prim_sha2_pkg::*; (
10   input clk_i,
11   input rst_ni,
12
13   input [1023:0] secret_key_i, // {word0, word1, ...,
word7}
14   input hmac_en_i,
15   input digest_mode_e digest_size_i,
16   input key_length_e key_length_i,
17
18   input reg_hash_start_i,
19   input reg_hash_stop_i,
20   input reg_hash_continue_i,
21   input reg_hash_process_i,
22   output logic hash_done_o,
23   output logic sha_hash_start_o,
24   output logic sha_hash_continue_o,
25   output logic sha_hash_process_o,
26   input sha_hash_done_i,
27
28   // fifo
29   output logic sha_rvalid_o,
30   output sha_fifo32_t sha_rdata_o,
31   input sha_rready_i,
32
33   input fifo_rvalid_i,
34   input sha_fifo32_t fifo_rdata_i,
35   output logic fifo_rready_o,
36
37   // fifo control (select and fifo write data)
38   output logic fifo_wsel_o, // 0: from reg, 1: from
digest
39   output logic fifo_wvalid_o,
40   // 0: digest[0][upper], 1: digest[0][lower] .. 14: digest[7]
[upper], 15: digest[7][lower]
41   output logic [3:0] fifo_wdata_sel_o,
42   input fifo_wready_i,
43
44   input [63:0] message_length_i,
45   output [63:0] sha_message_length_o,
46
47   output logic idle_o
48 );
49
50 localparam int unsigned BlockSizeSHA256 = 512;
51 localparam int unsigned BlockSizeSHA512 = 1024;
52
53 localparam int unsigned BlockSizeBitsSHA256 = $clo-
g2(BlockSizeSHA256);
54 localparam int unsigned BlockSizeBitsSHA512 = $clo-
g2(BlockSizeSHA512);
55
56 localparam int unsigned HashWordBitsSHA256 = $clo-
g2($bits(sha_word32_t));
57
58 localparam bit [63:0] BlockSizeSHA256in64 = 64'(BlockSize-
SHA256);
59 localparam bit [63:0] BlockSizeSHA512in64 = 64'(BlockSize-
SHA512);
60
61 logic hash_start; // generated from internal state ma-
chine
62 logic hash_continue; // generated from internal state ma-
chine
63 logic hash_process; // generated from internal state ma-
chine to trigger hash
64 logic hmac_hash_done;
65
66 logic [BlockSizeSHA256-1:0] i_pad_256;
67 logic [BlockSizeSHA512-1:0] i_pad_512;
68 logic [BlockSizeSHA256-1:0] o_pad_256;
69 logic [BlockSizeSHA512-1:0] o_pad_512;
70
71 logic [63:0] txcount, txcount_d; // works for both digest
lengths

```

```

72
73 logic [BlockSizeBitsSHA512-HashWordBitsSHA256-1:0] pad_in-
dex_512;
74 logic [BlockSizeBitsSHA256-HashWordBitsSHA256-1:0] pad_in-
dex_256;
75 logic clr_txcount, load_txcount, inc_txcount;
76
77 logic hmac_sha_rvalid;
78
79 logic idle_d, idle_q;
80 logic reg_hash_stop_d, reg_hash_stop_q;
81
82 typedef enum logic [1:0] {
83     SelIPad,
84     SelOPad,
85     SelFifo
86 } sel_rdata_t;
87
88 sel_rdata_t sel_rdata;
89
90 typedef enum logic {
91     SelIPadMsg,
92     SelOPadMsg
93 } sel_msglen_t;
94
95 sel_msglen_t sel_msglen;
96
97 typedef enum logic {
98     Inner, // Update when state goes to StIPad
99     Outer // Update when state enters StOPad
100 } round_t;
101
102 logic update_round;
103 round_t round_q, round_d;
104
105 typedef enum logic [2:0] {
106     StIdle,
107     StIPad,
108     StMsg, // Actual Msg, and Digest both
109     StPushToMsgFifo, // Digest --> Msg Fifo
110     StWaitResp, // Hash done( by checking pro-
cessed_length? or hash_done)
111     StOPad,
112     StDone // hmac_done
113 } st_e;
114
115 st_e st_q, st_d;
116
117 logic clr_fifo_wdata_sel;
118 logic txcnt_eq_blksz;
119
120 logic reg_hash_process_flag;
121
122 assign sha_hash_start_o = (hmac_en_i) ? hash_start :
reg_hash_start_i;
123 assign sha_hash_continue_o = (hmac_en_i) ? hash_continue :
reg_hash_continue_i;
124
125 assign sha_hash_process_o = (hmac_en_i) ?
reg_hash_process_i | hash_process : reg_hash_process_i;
126 assign hash_done_o = (hmac_en_i) ?
hmac_hash_done : sha_hash_done_i;
127
128 assign pad_index_512 = tx-
count[BlockSizeBitsSHA512-1:HashWordBitsSHA256];
129 assign pad_index_256 = tx-
count[BlockSizeBitsSHA256-1:HashWordBitsSHA256];
130
131 // adjust inner and outer padding depending on key length
and block size
132 always_comb begin : adjust_key_pad_length
133     // set defaults
134     i_pad_256 = '{default: '0';
135     i_pad_512 = '{default: '0';
136     o_pad_256 = '{default: '0';
137     o_pad_512 = '{default: '0';
138
139     unique case (key_length_i)
140         Key_128: begin
141             i_pad_256 = {secret_key_i[1023:896],
142                 {(BlockSizeSHA256-128){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h36}};
143             i_pad_512 = {secret_key_i[1023:896],
144                 {(BlockSizeSHA512-128){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
145             o_pad_256 = {secret_key_i[1023:896],
146                 {(BlockSizeSHA256-128){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h5c}};
147             o_pad_512 = {secret_key_i[1023:896],
148                 {(BlockSizeSHA512-128){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
149         end
150         Key_256: begin
151             i_pad_256 = {secret_key_i[1023:768],
152                 {(BlockSizeSHA256-256){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h36}};
153             i_pad_512 = {secret_key_i[1023:768],
154                 {(BlockSizeSHA512-256){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
155             o_pad_256 = {secret_key_i[1023:768],
156                 {(BlockSizeSHA256-256){1'b0}}} ^ {(BlockS-

```

```

izeSHA256/8){8'h5c}};
157             o_pad_512 = {secret_key_i[1023:768],
158                 {(BlockSizeSHA512-256){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
159         end
160         Key_384: begin
161             i_pad_256 = {secret_key_i[1023:640],
162                 {(BlockSizeSHA256-384){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h36}};
163             i_pad_512 = {secret_key_i[1023:640],
164                 {(BlockSizeSHA512-384){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
165             o_pad_256 = {secret_key_i[1023:640],
166                 {(BlockSizeSHA256-384){1'b0}}} ^ {(BlockS-
izeSHA256/8){8'h5c}};
167             o_pad_512 = {secret_key_i[1023:640],
168                 {(BlockSizeSHA512-384){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
169         end
170         Key_512: begin
171             i_pad_256 = secret_key_i[1023:512] ^ {(BlockSize-
SHA256/8){8'h36}};
172             i_pad_512 = {secret_key_i[1023:512],
173                 {(BlockSizeSHA512-512){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h36}};
174             o_pad_256 = secret_key_i[1023:512] ^ {(BlockSize-
SHA256/8){8'h5c}};
175             o_pad_512 = {secret_key_i[1023:512],
176                 {(BlockSizeSHA512-512){1'b0}}} ^ {(BlockS-
izeSHA512/8){8'h5c}};
177         end
178         Key_1024: begin // not allowed to be configured for
SHA-2 256
179             // zero out for SHA-2 256
180             i_pad_256 = '{default: '0';
181             i_pad_512 = secret_key_i[1023:0] ^ {(BlockSize-
SHA512/8){8'h36}};
182             // zero out for SHA-2 256
183             o_pad_256 = '{default: '0';
184             o_pad_512 = secret_key_i[1023:0] ^ {(BlockSize-
SHA512/8){8'h5c}};
185         end
186         default: begin
187             end
188         endcase
189     end
190
191     assign fifo_rready_o = (hmac_en_i) ? (st_q == StMsg) &
sha_rready_i : sha_rready_i;
192     // sha_rvalid is controlled by State Machine below.
193     assign sha_rvalid_o = (!hmac_en_i) ? fifo_rvalid_i :
hmac_sha_rvalid;
194     assign sha_rdata_o =
195         (!hmac_en_i) ?
fifo_rdata_i
196         : (sel_rdata == SelIPad && digest_size_i == SHA2_256)
? '{data: i_pad_256[(BlockSize-
SHA256-1)-32*pad_index_256-:32], mask: '1}' :
197         (sel_rdata == SelIPad && ((digest_size_i == SHA2_384) ||
(digest_size_i == SHA2_512)))
? '{data: i_pad_512[(BlockSize-
SHA512-1)-32*pad_index_512-:32], mask: '1}' :
200         (sel_rdata == SelOPad && digest_size_i == SHA2_256)
? '{data: o_pad_256[(BlockSize-
SHA256-1)-32*pad_index_256-:32], mask: '1}' :
202         (sel_rdata == SelOPad && ((digest_size_i == SHA2_384) ||
(digest_size_i == SHA2_512)))
? '{data: o_pad_512[(BlockSize-
SHA512-1)-32*pad_index_512-:32], mask: '1}' :
204         // Well-defined default case `sel_rdata == SelFifo`.
`sel_rdata == 2'b11` cannot occur.
205         fifo_rdata_i;
206
207     logic [63:0] sha_msg_len;
208
209     always_comb begin: assign_sha_message_length
210         sha_msg_len = '0;
211         if (!hmac_en_i) begin
212             sha_msg_len = message_length_i;
213             // HASH = (o_pad || HASH_INTERMEDIATE (i_pad || msg))
214             // message length for HASH_INTERMEDIATE = block size
(i_pad) + message length
215         end else if (sel_msglen == SelIPadMsg) begin
216             if (digest_size_i == SHA2_256) begin
217                 sha_msg_len = message_length_i + BlockSizeSHA256in64;
218             end else if ((digest_size_i == SHA2_384) || (di-
gest_size_i == SHA2_512)) begin
219                 sha_msg_len = message_length_i + BlockSizeSHA512in64;
220                 // `SHA2_None` is a permissible value in the
`SelIPadMsg` case. It can not occur for the
221                 // `SelOPadMsg` message length.
222             end else begin
223                 sha_msg_len = '0;
224             end
225         end else begin // `SelOPadMsg`
226             // message length for HASH = block size (o_pad) +
HASH_INTERMEDIATE digest length
227             if (digest_size_i == SHA2_256) begin
228                 sha_msg_len = BlockSizeSHA256in64 + 64'd256;
229             end else if (digest_size_i == SHA2_384) begin

```



```

230     sha_msg_len = BlockSizeSHA512in64 + 64'd384;
231 end else begin // SHA512
232     sha_msg_len = BlockSizeSHA512in64 + 64'd512;
233 end
234 end
235 end
236
237 assign sha_message_length_o = sha_msg_len;
238
239 always_comb begin
240     txcnt_eq_blkisz = '0;
241
242     unique case (digest_size_i)
243     SHA2_256: txcnt_eq_blkisz = (tx-
count[BlockSizeBitsSHA256-1:0] == '0) && (txcount != '0);
244     SHA2_384: txcnt_eq_blkisz = (tx-
count[BlockSizeBitsSHA512-1:0] == '0) && (txcount != '0);
245     SHA2_512: txcnt_eq_blkisz = (tx-
count[BlockSizeBitsSHA512-1:0] == '0) && (txcount != '0);
246     default;
247     endcase
248 end
249
250 assign inc_txcount = sha_rready_i && sha_rvalid_o;
251
252 // txcount
253 // Looks like txcount can be removed entirely here in
hmac_core
254 // In the first round (InnerPaddedKey), it can just watch
process and hash_done
255 // In the second round, it only needs count 256 bits for
hash digest to trigger
256 // hash_process to SHA2
257 always_comb begin
258     txcount_d = txcount;
259     if (clr_txcount) begin
260         txcount_d = '0;
261     end else if (load_txcount) begin
262         // When loading, add block size to the message length
because the SW-visible message length
263         // does not include the block containing the key xor'ed
with the inner pad.
264         unique case (digest_size_i)
265         SHA2_256: txcount_d = message_length_i + BlockSize-
SHA256in64;
266         SHA2_384: txcount_d = message_length_i + BlockSize-
SHA512in64;
267         SHA2_512: txcount_d = message_length_i + BlockSize-
SHA512in64;
268         default : txcount_d = message_length_i + '0;
269     endcase
270     end else if (inc_txcount) begin
271         txcount_d[63:5] = txcount[63:5] + 1'b1; // increment by
32 (data word size)
272     end
273 end
274
275 always_ff @(posedge clk_i or negedge rst_ni) begin
276     if (!rst_ni) txcount <= '0;
277     else txcount <= txcount_d;
278 end
279
280 // reg_hash_process_i trigger logic
281 always_ff @(posedge clk_i or negedge rst_ni) begin
282     if (!rst_ni) begin
283         reg_hash_process_flag <= 1'b0;
284     end else if (reg_hash_process_i) begin
285         reg_hash_process_flag <= 1'b1;
286     end else if (hmac_hash_done || reg_hash_start_i ||
reg_hash_continue_i) begin
287         reg_hash_process_flag <= 1'b0;
288     end
289 end
290
291 always_ff @(posedge clk_i or negedge rst_ni) begin
292     if (!rst_ni) begin
293         round_q <= Inner;
294     end else if (update_round) begin
295         round_q <= round_d;
296     end
297 end
298
299 always_ff @(posedge clk_i or negedge rst_ni) begin
300     if (!rst_ni) begin
301         fifo_wdata_sel_o <= '0;
302     end else if (clr_fifo_wdata_sel) begin
303         fifo_wdata_sel_o <= '0;
304     end else if (fifo_wsel_o && fifo_wvalid_o) begin
305         fifo_wdata_sel_o <= fifo_wdata_sel_o + 1'b1; // incre-
ment by 1
306     end
307 end
308
309 assign sel_msglen = (round_q == Inner) ? SelIPadMsg : Sel0-
PadMsg ;
310
311 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_ff
312     if (!rst_ni) st_q <= StIdle;
313     else st_q <= st_d;

```

```

314 end
315
316 always_comb begin : next_state
317     hmac_hash_done = 1'b0;
318     hmac_sha_rvalid = 1'b0;
319     clr_txcount = 1'b0;
320     load_txcount = 1'b0;
321     update_round = 1'b0;
322     round_d = Inner;
323     fifo_wsel_o = 1'b0; // from register
324     fifo_wvalid_o = 1'b0;
325     clr_fifo_wdata_sel = 1'b1;
326     sel_rdata = SelFifo;
327     hash_start = 1'b0;
328     hash_continue = 1'b0;
329     hash_process = 1'b0;
330     st_d = st_q;
331
332     unique case (st_q)
333     StIdle: begin
334         // reset round to Inner
335         // we always switch context into inner round since
outer round computes once over
336         // single block at the end (outer key pad + inner
hash)
337         update_round = 1'b1;
338         round_d = Inner;
339         if (hmac_en_i && reg_hash_start_i) begin
340             st_d = StIPad; // start at StIPad if told to start
341
342             clr_txcount = 1'b1;
343             hash_start = 1'b1;
344             end else if (hmac_en_i && reg_hash_continue_i) begin
345                 st_d = StMsg; // skip StIPad if told to continue -
assumed it finished StIPad
346
347                 load_txcount = 1'b1;
348                 hash_continue = 1'b1;
349             end else begin
350                 st_d = StIdle;
351             end
352         end
353
354         StIPad: begin
355             sel_rdata = SelIPad;
356
357             if (txcnt_eq_blkisz) begin
358                 st_d = StMsg;
359
360                 hmac_sha_rvalid = 1'b0; // block new read request
361             end else begin
362                 st_d = StIPad;
363
364                 hmac_sha_rvalid = 1'b1;
365             end
366         end
367
368         StMsg: begin
369             sel_rdata = SelFifo;
370             fifo_wsel_o = (round_q == Outer);
371
372             if ( (((round_q == Inner) && reg_hash_process_flag) ||
(round_q == Outer))
373                 && (txcount >= sha_message_length_o)) begin
374                 st_d = StWaitResp;
375
376                 hmac_sha_rvalid = 1'b0; // block reading words from
MSG FIFO
377                 hash_process = (round_q == Outer);
378                 end else if (txcnt_eq_blkisz && (txcount >= sha_mes-
sage_length_o)
379                     && reg_hash_stop_q && (round_q == Inner))
begin
380                     // wait till all MSG words are pushed out from FIFO
(txcount reaches msg length)
381                     // before transitioning to StWaitResp to wait on
sha_hash_done_i and disabling
382                     // reading from MSG FIFO
383                     st_d = StWaitResp;
384
385                     hmac_sha_rvalid = 1'b0;
386                 end else begin
387                     st_d = StMsg;
388                     hmac_sha_rvalid = fifo_rvalid_i;
389                 end
390             end
391
392             StWaitResp: begin
393                 hmac_sha_rvalid = 1'b0;
394
395                 if (sha_hash_done_i) begin
396                     if (round_q == Outer) begin
397                         st_d = StDone;
398                     end else begin // round_q == Inner
399                         if (reg_hash_stop_q) begin
400                             st_d = StDone;
401                         end else begin
402                             st_d = StPushToMsgFifo;
403                         end

```

```

404         end
405     end else begin
406         st_d = StWaitResp;
407     end
408 end
409
410 StPushToMsgFifo: begin
411     hmac_sha_rvalid = 1'b0;
412     fifo_wsel_o = 1'b1;
413     fifo_wvalid_o = 1'b1;
414     clr_fifo_wdata_sel = 1'b0;
415
416     if (fifo_wready_i && (((fifo_wdata_sel_o == 4'd7) &&
(digest_size_i == SHA2_256)) ||
417     ((fifo_wdata_sel_o == 4'd15) &&
(digest_size_i == SHA2_512)) ||
418     ((fifo_wdata_sel_o == 4'd11) &&
(digest_size_i == SHA2_384)))) begin
419         st_d = StOPad;
420
421         clr_txcount = 1'b1;
422         update_round = 1'b1;
423         round_d = Outer;
424         hash_start = 1'b1;
425     end else begin
426         st_d = StPushToMsgFifo;
427     end
428 end
429
430 end
431
432 StOPad: begin
433     sel_rdata = SelOPad;
434     fifo_wsel_o = 1'b1; // Remained HMAC select to indic-
ate HMAC is in second stage
435
436     if (txcnt_eq_blkksz) begin
437         st_d = StMsg;
438
439         hmac_sha_rvalid = 1'b0; // block new read request
440     end else begin
441         st_d = StOPad;
442
443         hmac_sha_rvalid = 1'b1;
444     end
445 end
446
447 StDone: begin
448     // raise interrupt (hash_done)
449     st_d = StIdle;
450
451     hmac_hash_done = 1'b1;
452 end
453
454 default: begin
455     st_d = StIdle;
456 end
457
458 endcase
459 end
460
461 // raise reg_hash_stop_d flag at reg_hash_stop_i and keep it
until sha_hash_done_i is asserted
462 // to indicate the hashing operation on current block has
completed
463 assign reg_hash_stop_d = (reg_hash_stop_i ==
1'b1)
? 1'b1 :
464 (sha_hash_done_i == 1'b1 &&
reg_hash_stop_q == 1'b1) ? 1'b0 :
465 reg_hash_stop_q;
466
467 always_ff @(posedge clk_i or negedge rst_ni) begin
468     if (!rst_ni) begin
469         reg_hash_stop_q <= 1'b0;
470     end else begin
471         reg_hash_stop_q <= reg_hash_stop_d;
472     end
473 end
474
475 // Idle status signaling: This module ..
476 assign idle_d =
477 // .. is not idle when told to start or continue
(reg_hash_start_i || reg_hash_continue_i) ? 1'b0 :
478 // .. is idle when the FSM is in the Idle state
(st_q == StIdle) ? 1'b1 :
479 // .. is idle when it has processed a complete block of
a message and is told to stop in any
480 // FSM state
(txcnt_eq_blkksz && reg_hash_stop_d) ? 1'b1 :
481 // .. and keeps the current idle state in all other
cases.
482 idle_q;
483
484 assign idle_o = idle_d;
485
486 always_ff @(posedge clk_i or negedge rst_ni) begin
487     if (!rst_ni) begin
488         idle_q <= 1'b1;

```

```

492     end else begin
493         idle_q <= idle_d;
494     end
495 end
496
497 // Assertions //
498 // Assertions //
499 // Assertions //
500
501 `ASSERT(ValidSelRdata_A, hmac_en_i |-> sel_rdata inside
{SelIPad, SelOPad, SelFifo})
502 `ASSERT(ValidDigestSize_A, (hmac_en_i && (sel_msglen ==
SelOPadMsg)) |->
503     digest_size_i inside {SHA2_256, SHA2_384, SHA2_512})
504
505 endmodule

```

3. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_pkg.sv (511 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
structure
6
7 package hmac_reg_pkg;
8
9 // Param list
10 parameter int NumDigestWords = 16;
11 parameter int NumKeyWords = 32;
12 parameter int NumAlerts = 1;
13
14 // Address widths within the block
15 parameter int BlockAw = 13;
16
17 // Number of registers for every interface
18 parameter int NumRegs = 59;
19
20 // Alert indices
21 typedef enum int {
22     AlertFatalFaultIdx = 0
23 } hmac_alert_idx_t;
24
25 // Typedefs for registers //
26 // Typedefs for registers //
27 // Typedefs for registers //
28
29 typedef struct packed {
30     struct packed {
31         logic q;
32     } hmac_err;
33     struct packed {
34         logic q;
35     } fifo_empty;
36     struct packed {
37         logic q;
38     } hmac_done;
39 } hmac_reg2hw_intr_state_reg_t;
40
41 typedef struct packed {
42     struct packed {
43         logic q;
44     } hmac_err;
45     struct packed {
46         logic q;
47     } fifo_empty;
48     struct packed {
49         logic q;
50     } hmac_done;
51 } hmac_reg2hw_intr_enable_reg_t;
52
53 typedef struct packed {
54     struct packed {
55         logic q;
56         logic qe;
57     } hmac_err;
58     struct packed {
59         logic q;
60         logic qe;
61     } fifo_empty;
62     struct packed {
63         logic q;
64         logic qe;
65     } hmac_done;
66 } hmac_reg2hw_intr_test_reg_t;
67
68 typedef struct packed {
69     logic q;
70     logic qe;
71 } hmac_reg2hw_alert_test_reg_t;
72
73 typedef struct packed {
74     struct packed {
75         logic [5:0] q;
76         logic qe;

```

```

77     } key_length;
78     struct packed {
79         logic [3:0] q;
80         logic       qe;
81     } digest_size;
82     struct packed {
83         logic       q;
84         logic       qe;
85     } key_swap;
86     struct packed {
87         logic       q;
88         logic       qe;
89     } digest_swap;
90     struct packed {
91         logic       q;
92         logic       qe;
93     } endian_swap;
94     struct packed {
95         logic       q;
96         logic       qe;
97     } sha_en;
98     struct packed {
99         logic       q;
100        logic       qe;
101    } hmac_en;
102 } hmac_reg2hw_cfg_reg_t;
103
104 typedef struct packed {
105     struct packed {
106         logic       q;
107         logic       qe;
108     } hash_continue;
109     struct packed {
110         logic       q;
111         logic       qe;
112     } hash_stop;
113     struct packed {
114         logic       q;
115         logic       qe;
116     } hash_process;
117     struct packed {
118         logic       q;
119         logic       qe;
120     } hash_start;
121 } hmac_reg2hw_cmd_reg_t;
122
123 typedef struct packed {
124     logic [31:0] q;
125     logic       qe;
126 } hmac_reg2hw_wipe_secret_reg_t;
127
128 typedef struct packed {
129     logic [31:0] q;
130     logic       qe;
131 } hmac_reg2hw_key_mreg_t;
132
133 typedef struct packed {
134     logic [31:0] q;
135     logic       qe;
136 } hmac_reg2hw_digest_mreg_t;
137
138 typedef struct packed {
139     logic [31:0] q;
140     logic       qe;
141 } hmac_reg2hw_msg_length_lower_reg_t;
142
143 typedef struct packed {
144     logic [31:0] q;
145     logic       qe;
146 } hmac_reg2hw_msg_length_upper_reg_t;
147
148 typedef struct packed {
149     struct packed {
150         logic d;
151         logic de;
152     } hmac_err;
153     struct packed {
154         logic d;
155         logic de;
156     } fifo_empty;
157     struct packed {
158         logic d;
159         logic de;
160     } hmac_done;
161 } hmac_hw2reg_intr_state_reg_t;
162
163 typedef struct packed {
164     struct packed {
165         logic [5:0] d;
166     } key_length;
167     struct packed {
168         logic [3:0] d;
169     } digest_size;
170     struct packed {
171         logic d;
172     } key_swap;
173     struct packed {
174         logic d;
175     } digest_swap;
176     struct packed {
177         logic d;
178     } endian_swap;
179     struct packed {
180         logic d;
181     } sha_en;
182     struct packed {
183         logic d;
184     } hmac_en;
185 } hmac_hw2reg_cfg_reg_t;
186
187 typedef struct packed {
188     struct packed {
189         logic [5:0] d;
190     } fifo_depth;
191     struct packed {
192         logic d;
193     } fifo_full;
194     struct packed {
195         logic d;
196     } fifo_empty;
197     struct packed {
198         logic d;
199     } hmac_idle;
200 } hmac_hw2reg_status_reg_t;
201
202 typedef struct packed {
203     logic [31:0] d;
204     logic       de;
205 } hmac_hw2reg_err_code_reg_t;
206
207 typedef struct packed {
208     logic [31:0] d;
209 } hmac_hw2reg_key_mreg_t;
210
211 typedef struct packed {
212     logic [31:0] d;
213 } hmac_hw2reg_digest_mreg_t;
214
215 typedef struct packed {
216     logic [31:0] d;
217 } hmac_hw2reg_msg_length_lower_reg_t;
218
219 typedef struct packed {
220     logic [31:0] d;
221 } hmac_hw2reg_msg_length_upper_reg_t;
222
223 // Register -> HW type
224 typedef struct packed {
225     hmac_reg2hw_intr_state_reg_t intr_state; // [1726:1724]
226     hmac_reg2hw_intr_enable_reg_t intr_enable; // [1723:1721]
227     hmac_reg2hw_intr_test_reg_t intr_test; // [1720:1715]
228     hmac_reg2hw_alert_test_reg_t alert_test; // [1714:1713]
229     hmac_reg2hw_cfg_reg_t cfg; // [1712:1691]
230     hmac_reg2hw_cmd_reg_t cmd; // [1690:1683]
231     hmac_reg2hw_wipe_secret_reg_t wipe_secret; // [1682:1650]
232     hmac_reg2hw_key_mreg_t [31:0] key; // [1649:594]
233     hmac_reg2hw_digest_mreg_t [15:0] digest; // [593:66]
234     hmac_reg2hw_msg_length_lower_reg_t msg_length_lower; //
235     hmac_reg2hw_msg_length_upper_reg_t msg_length_upper; //
236 } hmac_reg2hw_t;
237
238 // HW -> register type
239 typedef struct packed {
240     hmac_hw2reg_intr_state_reg_t intr_state; // [1662:1657]
241     hmac_hw2reg_cfg_reg_t cfg; // [1656:1642]
242     hmac_hw2reg_status_reg_t status; // [1641:1633]
243     hmac_hw2reg_err_code_reg_t err_code; // [1632:1600]
244     hmac_hw2reg_key_mreg_t [31:0] key; // [1599:576]
245     hmac_hw2reg_digest_mreg_t [15:0] digest; // [575:64]
246     hmac_hw2reg_msg_length_lower_reg_t msg_length_lower; //
247     hmac_hw2reg_msg_length_upper_reg_t msg_length_upper; //
248 } hmac_hw2reg_t;
249
250 // Register offsets
251 parameter logic [BlockAw-1:0] HMAC_INTR_STATE_OFFSET = 13'h
0;
252 parameter logic [BlockAw-1:0] HMAC_INTR_ENABLE_OFFSET = 13'h
4;
253 parameter logic [BlockAw-1:0] HMAC_INTR_TEST_OFFSET = 13'h
8;
254 parameter logic [BlockAw-1:0] HMAC_ALERT_TEST_OFFSET = 13'h
c;
255 parameter logic [BlockAw-1:0] HMAC_CFG_OFFSET = 13'h 10;
256 parameter logic [BlockAw-1:0] HMAC_CMD_OFFSET = 13'h 14;
257 parameter logic [BlockAw-1:0] HMAC_STATUS_OFFSET = 13'h 18;
258 parameter logic [BlockAw-1:0] HMAC_ERR_CODE_OFFSET = 13'h
1c;
259 parameter logic [BlockAw-1:0] HMAC_WIPE_SECRET_OFFSET = 13'h
20;
260 parameter logic [BlockAw-1:0] HMAC_KEY_0_OFFSET = 13'h 24;
261 parameter logic [BlockAw-1:0] HMAC_KEY_1_OFFSET = 13'h 28;
262 parameter logic [BlockAw-1:0] HMAC_KEY_2_OFFSET = 13'h 2c;
263 parameter logic [BlockAw-1:0] HMAC_KEY_3_OFFSET = 13'h 30;
264 parameter logic [BlockAw-1:0] HMAC_KEY_4_OFFSET = 13'h 34;
265 parameter logic [BlockAw-1:0] HMAC_KEY_5_OFFSET = 13'h 38;
266 parameter logic [BlockAw-1:0] HMAC_KEY_6_OFFSET = 13'h 3c;

```



```

267 parameter logic [BlockAw-1:0] HMAC_KEY_7_OFFSET = 13'h 40;
268 parameter logic [BlockAw-1:0] HMAC_KEY_8_OFFSET = 13'h 44;
269 parameter logic [BlockAw-1:0] HMAC_KEY_9_OFFSET = 13'h 48;
270 parameter logic [BlockAw-1:0] HMAC_KEY_10_OFFSET = 13'h 4c;
271 parameter logic [BlockAw-1:0] HMAC_KEY_11_OFFSET = 13'h 50;
272 parameter logic [BlockAw-1:0] HMAC_KEY_12_OFFSET = 13'h 54;
273 parameter logic [BlockAw-1:0] HMAC_KEY_13_OFFSET = 13'h 58;
274 parameter logic [BlockAw-1:0] HMAC_KEY_14_OFFSET = 13'h 5c;
275 parameter logic [BlockAw-1:0] HMAC_KEY_15_OFFSET = 13'h 60;
276 parameter logic [BlockAw-1:0] HMAC_KEY_16_OFFSET = 13'h 64;
277 parameter logic [BlockAw-1:0] HMAC_KEY_17_OFFSET = 13'h 68;
278 parameter logic [BlockAw-1:0] HMAC_KEY_18_OFFSET = 13'h 6c;
279 parameter logic [BlockAw-1:0] HMAC_KEY_19_OFFSET = 13'h 70;
280 parameter logic [BlockAw-1:0] HMAC_KEY_20_OFFSET = 13'h 74;
281 parameter logic [BlockAw-1:0] HMAC_KEY_21_OFFSET = 13'h 78;
282 parameter logic [BlockAw-1:0] HMAC_KEY_22_OFFSET = 13'h 7c;
283 parameter logic [BlockAw-1:0] HMAC_KEY_23_OFFSET = 13'h 80;
284 parameter logic [BlockAw-1:0] HMAC_KEY_24_OFFSET = 13'h 84;
285 parameter logic [BlockAw-1:0] HMAC_KEY_25_OFFSET = 13'h 88;
286 parameter logic [BlockAw-1:0] HMAC_KEY_26_OFFSET = 13'h 8c;
287 parameter logic [BlockAw-1:0] HMAC_KEY_27_OFFSET = 13'h 90;
288 parameter logic [BlockAw-1:0] HMAC_KEY_28_OFFSET = 13'h 94;
289 parameter logic [BlockAw-1:0] HMAC_KEY_29_OFFSET = 13'h 98;
290 parameter logic [BlockAw-1:0] HMAC_KEY_30_OFFSET = 13'h 9c;
291 parameter logic [BlockAw-1:0] HMAC_KEY_31_OFFSET = 13'h a0;
292 parameter logic [BlockAw-1:0] HMAC_DIGEST_0_OFFSET = 13'h
a4;
293 parameter logic [BlockAw-1:0] HMAC_DIGEST_1_OFFSET = 13'h
a8;
294 parameter logic [BlockAw-1:0] HMAC_DIGEST_2_OFFSET = 13'h
ac;
295 parameter logic [BlockAw-1:0] HMAC_DIGEST_3_OFFSET = 13'h
b0;
296 parameter logic [BlockAw-1:0] HMAC_DIGEST_4_OFFSET = 13'h
b4;
297 parameter logic [BlockAw-1:0] HMAC_DIGEST_5_OFFSET = 13'h
b8;
298 parameter logic [BlockAw-1:0] HMAC_DIGEST_6_OFFSET = 13'h
bc;
299 parameter logic [BlockAw-1:0] HMAC_DIGEST_7_OFFSET = 13'h
c0;
300 parameter logic [BlockAw-1:0] HMAC_DIGEST_8_OFFSET = 13'h
c4;
301 parameter logic [BlockAw-1:0] HMAC_DIGEST_9_OFFSET = 13'h
c8;
302 parameter logic [BlockAw-1:0] HMAC_DIGEST_10_OFFSET = 13'h
cc;
303 parameter logic [BlockAw-1:0] HMAC_DIGEST_11_OFFSET = 13'h
d0;
304 parameter logic [BlockAw-1:0] HMAC_DIGEST_12_OFFSET = 13'h
d4;
305 parameter logic [BlockAw-1:0] HMAC_DIGEST_13_OFFSET = 13'h
d8;
306 parameter logic [BlockAw-1:0] HMAC_DIGEST_14_OFFSET = 13'h
dc;
307 parameter logic [BlockAw-1:0] HMAC_DIGEST_15_OFFSET = 13'h
e0;

308 parameter logic [BlockAw-1:0] HMAC_MSG_LENGTH_LOWER_OFFSET =
13'h e4;
309 parameter logic [BlockAw-1:0] HMAC_MSG_LENGTH_UPPER_OFFSET =
13'h e8;
310
311 // Reset values for hwext registers and their fields
312 parameter logic [2:0] HMAC_INTR_TEST_RESVAL = 3'h 0;
313 parameter logic [0:0] HMAC_INTR_TEST_HMAC_DONE_RESVAL = 1'h
0;

314 parameter logic [0:0] HMAC_INTR_TEST_FIFO_EMPTY_RESVAL = 1'h
0;
315 parameter logic [0:0] HMAC_INTR_TEST_HMAC_ERR_RESVAL = 1'h
0;
316 parameter logic [0:0] HMAC_ALERT_TEST_RESVAL = 1'h 0;
317 parameter logic [0:0] HMAC_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
318 parameter logic [14:0] HMAC_CFG_RESVAL = 15'h 4100;
319 parameter logic [0:0] HMAC_CFG_ENDIAN_SWAP_RESVAL = 1'h 0;
320 parameter logic [0:0] HMAC_CFG_DIGEST_SWAP_RESVAL = 1'h 0;
321 parameter logic [0:0] HMAC_CFG_KEY_SWAP_RESVAL = 1'h 0;
322 parameter logic [3:0] HMAC_CFG_DIGEST_SIZE_RESVAL = 4'h 8;
323 parameter logic [5:0] HMAC_CFG_KEY_LENGTH_RESVAL = 6'h 20;
324 parameter logic [3:0] HMAC_CMD_RESVAL = 4'h 0;
325 parameter logic [9:0] HMAC_STATUS_RESVAL = 10'h 3;
326 parameter logic [0:0] HMAC_STATUS_HMAC_IDLE_RESVAL = 1'h 1;
327 parameter logic [0:0] HMAC_STATUS_FIFO_EMPTY_RESVAL = 1'h 1;
328 parameter logic [31:0] HMAC_WIPE_SECRET_RESVAL = 32'h 0;
329 parameter logic [31:0] HMAC_KEY_0_RESVAL = 32'h 0;
330 parameter logic [31:0] HMAC_KEY_1_RESVAL = 32'h 0;
331 parameter logic [31:0] HMAC_KEY_2_RESVAL = 32'h 0;
332 parameter logic [31:0] HMAC_KEY_3_RESVAL = 32'h 0;
333 parameter logic [31:0] HMAC_KEY_4_RESVAL = 32'h 0;
334 parameter logic [31:0] HMAC_KEY_5_RESVAL = 32'h 0;
335 parameter logic [31:0] HMAC_KEY_6_RESVAL = 32'h 0;
336 parameter logic [31:0] HMAC_KEY_7_RESVAL = 32'h 0;
337 parameter logic [31:0] HMAC_KEY_8_RESVAL = 32'h 0;
338 parameter logic [31:0] HMAC_KEY_9_RESVAL = 32'h 0;
339 parameter logic [31:0] HMAC_KEY_10_RESVAL = 32'h 0;
340 parameter logic [31:0] HMAC_KEY_11_RESVAL = 32'h 0;
341 parameter logic [31:0] HMAC_KEY_12_RESVAL = 32'h 0;
342 parameter logic [31:0] HMAC_KEY_13_RESVAL = 32'h 0;

343 parameter logic [31:0] HMAC_KEY_14_RESVAL = 32'h 0;
344 parameter logic [31:0] HMAC_KEY_15_RESVAL = 32'h 0;
345 parameter logic [31:0] HMAC_KEY_16_RESVAL = 32'h 0;
346 parameter logic [31:0] HMAC_KEY_17_RESVAL = 32'h 0;
347 parameter logic [31:0] HMAC_KEY_18_RESVAL = 32'h 0;
348 parameter logic [31:0] HMAC_KEY_19_RESVAL = 32'h 0;
349 parameter logic [31:0] HMAC_KEY_20_RESVAL = 32'h 0;
350 parameter logic [31:0] HMAC_KEY_21_RESVAL = 32'h 0;
351 parameter logic [31:0] HMAC_KEY_22_RESVAL = 32'h 0;
352 parameter logic [31:0] HMAC_KEY_23_RESVAL = 32'h 0;
353 parameter logic [31:0] HMAC_KEY_24_RESVAL = 32'h 0;
354 parameter logic [31:0] HMAC_KEY_25_RESVAL = 32'h 0;
355 parameter logic [31:0] HMAC_KEY_26_RESVAL = 32'h 0;
356 parameter logic [31:0] HMAC_KEY_27_RESVAL = 32'h 0;
357 parameter logic [31:0] HMAC_KEY_28_RESVAL = 32'h 0;
358 parameter logic [31:0] HMAC_KEY_29_RESVAL = 32'h 0;
359 parameter logic [31:0] HMAC_KEY_30_RESVAL = 32'h 0;
360 parameter logic [31:0] HMAC_KEY_31_RESVAL = 32'h 0;
361 parameter logic [31:0] HMAC_DIGEST_0_RESVAL = 32'h 0;
362 parameter logic [31:0] HMAC_DIGEST_1_RESVAL = 32'h 0;
363 parameter logic [31:0] HMAC_DIGEST_2_RESVAL = 32'h 0;
364 parameter logic [31:0] HMAC_DIGEST_3_RESVAL = 32'h 0;
365 parameter logic [31:0] HMAC_DIGEST_4_RESVAL = 32'h 0;
366 parameter logic [31:0] HMAC_DIGEST_5_RESVAL = 32'h 0;
367 parameter logic [31:0] HMAC_DIGEST_6_RESVAL = 32'h 0;
368 parameter logic [31:0] HMAC_DIGEST_7_RESVAL = 32'h 0;
369 parameter logic [31:0] HMAC_DIGEST_8_RESVAL = 32'h 0;
370 parameter logic [31:0] HMAC_DIGEST_9_RESVAL = 32'h 0;
371 parameter logic [31:0] HMAC_DIGEST_10_RESVAL = 32'h 0;
372 parameter logic [31:0] HMAC_DIGEST_11_RESVAL = 32'h 0;
373 parameter logic [31:0] HMAC_DIGEST_12_RESVAL = 32'h 0;
374 parameter logic [31:0] HMAC_DIGEST_13_RESVAL = 32'h 0;
375 parameter logic [31:0] HMAC_DIGEST_14_RESVAL = 32'h 0;
376 parameter logic [31:0] HMAC_DIGEST_15_RESVAL = 32'h 0;
377 parameter logic [31:0] HMAC_MSG_LENGTH_LOWER_RESVAL = 32'h
0;
378 parameter logic [31:0] HMAC_MSG_LENGTH_UPPER_RESVAL = 32'h
0;
379
380 // Window parameters
381 parameter logic [BlockAw-1:0] HMAC_MSG_FIFO_OFFSET = 13'h
1000;
382 parameter int unsigned HMAC_MSG_FIFO_SIZE = 'h
1000;
383 parameter int unsigned HMAC_MSG_FIFO_IDX = 0;
384
385 // Register index
386 typedef enum int {
387     HMAC_INTR_STATE,
388     HMAC_INTR_ENABLE,
389     HMAC_INTR_TEST,
390     HMAC_ALERT_TEST,
391     HMAC_CFG,
392     HMAC_CMD,
393     HMAC_STATUS,
394     HMAC_ERR_CODE,
395     HMAC_WIPE_SECRET,
396     HMAC_KEY_0,
397     HMAC_KEY_1,
398     HMAC_KEY_2,
399     HMAC_KEY_3,
400     HMAC_KEY_4,
401     HMAC_KEY_5,
402     HMAC_KEY_6,
403     HMAC_KEY_7,
404     HMAC_KEY_8,
405     HMAC_KEY_9,
406     HMAC_KEY_10,
407     HMAC_KEY_11,
408     HMAC_KEY_12,
409     HMAC_KEY_13,
410     HMAC_KEY_14,
411     HMAC_KEY_15,
412     HMAC_KEY_16,
413     HMAC_KEY_17,
414     HMAC_KEY_18,
415     HMAC_KEY_19,
416     HMAC_KEY_20,
417     HMAC_KEY_21,
418     HMAC_KEY_22,
419     HMAC_KEY_23,
420     HMAC_KEY_24,
421     HMAC_KEY_25,
422     HMAC_KEY_26,
423     HMAC_KEY_27,
424     HMAC_KEY_28,
425     HMAC_KEY_29,
426     HMAC_KEY_30,
427     HMAC_KEY_31,
428     HMAC_DIGEST_0,
429     HMAC_DIGEST_1,
430     HMAC_DIGEST_2,
431     HMAC_DIGEST_3,
432     HMAC_DIGEST_4,
433     HMAC_DIGEST_5,
434     HMAC_DIGEST_6,
435     HMAC_DIGEST_7,
436     HMAC_DIGEST_8,
437     HMAC_DIGEST_9,
438     HMAC_DIGEST_10,
439     HMAC_DIGEST_11,

```

```

440 HMAC_DIGEST_12,
441 HMAC_DIGEST_13,
442 HMAC_DIGEST_14,
443 HMAC_DIGEST_15,
444 HMAC_MSG_LENGTH_LOWER,
445 HMAC_MSG_LENGTH_UPPER
446 } hmac_id_e;
447
448 // Register width information to check illegal writes
449 parameter logic [3:0] HMAC_PERMIT [59] = '{
450 4'b 0001, // index[ 0] HMAC_INTR_STATE
451 4'b 0001, // index[ 1] HMAC_INTR_ENABLE
452 4'b 0001, // index[ 2] HMAC_INTR_TEST
453 4'b 0001, // index[ 3] HMAC_ALERT_TEST
454 4'b 0011, // index[ 4] HMAC_CFG
455 4'b 0001, // index[ 5] HMAC_CMD
456 4'b 0011, // index[ 6] HMAC_STATUS
457 4'b 1111, // index[ 7] HMAC_ERR_CODE
458 4'b 1111, // index[ 8] HMAC_WIPE_SECRET
459 4'b 1111, // index[ 9] HMAC_KEY_0
460 4'b 1111, // index[10] HMAC_KEY_1
461 4'b 1111, // index[11] HMAC_KEY_2
462 4'b 1111, // index[12] HMAC_KEY_3
463 4'b 1111, // index[13] HMAC_KEY_4
464 4'b 1111, // index[14] HMAC_KEY_5
465 4'b 1111, // index[15] HMAC_KEY_6
466 4'b 1111, // index[16] HMAC_KEY_7
467 4'b 1111, // index[17] HMAC_KEY_8
468 4'b 1111, // index[18] HMAC_KEY_9
469 4'b 1111, // index[19] HMAC_KEY_10
470 4'b 1111, // index[20] HMAC_KEY_11
471 4'b 1111, // index[21] HMAC_KEY_12
472 4'b 1111, // index[22] HMAC_KEY_13
473 4'b 1111, // index[23] HMAC_KEY_14
474 4'b 1111, // index[24] HMAC_KEY_15
475 4'b 1111, // index[25] HMAC_KEY_16
476 4'b 1111, // index[26] HMAC_KEY_17
477 4'b 1111, // index[27] HMAC_KEY_18
478 4'b 1111, // index[28] HMAC_KEY_19
479 4'b 1111, // index[29] HMAC_KEY_20
480 4'b 1111, // index[30] HMAC_KEY_21
481 4'b 1111, // index[31] HMAC_KEY_22
482 4'b 1111, // index[32] HMAC_KEY_23
483 4'b 1111, // index[33] HMAC_KEY_24
484 4'b 1111, // index[34] HMAC_KEY_25
485 4'b 1111, // index[35] HMAC_KEY_26
486 4'b 1111, // index[36] HMAC_KEY_27
487 4'b 1111, // index[37] HMAC_KEY_28
488 4'b 1111, // index[38] HMAC_KEY_29
489 4'b 1111, // index[39] HMAC_KEY_30
490 4'b 1111, // index[40] HMAC_KEY_31
491 4'b 1111, // index[41] HMAC_DIGEST_0
492 4'b 1111, // index[42] HMAC_DIGEST_1
493 4'b 1111, // index[43] HMAC_DIGEST_2
494 4'b 1111, // index[44] HMAC_DIGEST_3
495 4'b 1111, // index[45] HMAC_DIGEST_4
496 4'b 1111, // index[46] HMAC_DIGEST_5
497 4'b 1111, // index[47] HMAC_DIGEST_6
498 4'b 1111, // index[48] HMAC_DIGEST_7
499 4'b 1111, // index[49] HMAC_DIGEST_8
500 4'b 1111, // index[50] HMAC_DIGEST_9
501 4'b 1111, // index[51] HMAC_DIGEST_10
502 4'b 1111, // index[52] HMAC_DIGEST_11
503 4'b 1111, // index[53] HMAC_DIGEST_12
504 4'b 1111, // index[54] HMAC_DIGEST_13
505 4'b 1111, // index[55] HMAC_DIGEST_14
506 4'b 1111, // index[56] HMAC_DIGEST_15
507 4'b 1111, // index[57] HMAC_MSG_LENGTH_LOWER
508 4'b 1111, // index[58] HMAC_MSG_LENGTH_UPPER
509 };
510
511 endpackage

```

4. lowRISC_opentitan/hw/ip/hmac/rtl/hmac_reg_top.sv (2,648 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module hmac_reg_top (
10   input clk_i,
11   input rst_ni,
12   input tlul_pkg::tl_h2d_t tl_i,
13   output tlul_pkg::tl_d2h_t tl_o,
14
15   // Output port for window
16   output tlul_pkg::tl_h2d_t tl_win_o,
17   input tlul_pkg::tl_d2h_t tl_win_i,
18
19   // To HW
20   output hmac_reg_pkg::hmac_reg2hw_t reg2hw, // Write
21   input  hmac_reg_pkg::hmac_hw2reg_t hw2reg, // Read
22

```

```

23 // Integrity check errors
24 output logic intg_err_o
25 );
26
27 import hmac_reg_pkg::* ;
28
29 localparam int AW = 13;
30 localparam int DW = 32;
31 localparam int DBW = DW/8; // Byte Width
32
33 // register signals
34 logic reg_we;
35 logic reg_re;
36 logic [AW-1:0] reg_addr;
37 logic [DW-1:0] reg_wdata;
38 logic [DBW-1:0] reg_be;
39 logic [DW-1:0] reg_rdata;
40 logic reg_error;
41
42 logic addrmiss, wr_err;
43
44 logic [DW-1:0] reg_rdata_next;
45 logic reg_busy;
46
47 tlul_pkg::tl_h2d_t tl_reg_h2d;
48 tlul_pkg::tl_d2h_t tl_reg_d2h;
49
50 // incoming payload check
51 logic intg_err;
52 tlul_cmd_intg_chk u_chk (
53   .tl_i(tl_i),
54   .err_o(intg_err)
55 );
56
57 // also check for spurious write enables
58 logic reg_we_err;
59 logic [58:0] reg_we_check;
60 prim_reg_we_check #(
61   .OneHotWidth(59)
62 ) u_prim_reg_we_check (
63   .clk_i(clk_i),
64   .rst_ni(rst_ni),
65   .oh_i (reg_we_check),
66   .en_i (reg_we && !addrmiss),
67   .err_o (reg_we_err)
68 );
69
70 logic err_q;
71 always_ff @(posedge clk_i or negedge rst_ni) begin
72   if (!rst_ni) begin
73     err_q <= '0;
74   end else if (intg_err || reg_we_err) begin
75     err_q <= '1b1;
76   end
77 end
78
79 // integrity error output is permanent and should be used
  for alert generation
80 // register errors are transactional
81 assign intg_err_o = err_q | intg_err | reg_we_err;
82
83 // outgoing integrity generation
84 tlul_pkg::tl_d2h_t tl_o_pre;
85 tlul_rsp_intg_gen #(
86   .EnableRspIntgGen(1),
87   .EnableDataIntgGen(1)
88 ) u_rsp_intg_gen (
89   .tl_i(tl_o_pre),
90   .tl_o(tl_o)
91 );
92
93 tlul_pkg::tl_h2d_t tl_socket_h2d [2];
94 tlul_pkg::tl_d2h_t tl_socket_d2h [2];
95
96 logic [0:0] reg_steer;
97
98 // socket_1n connection
99 assign tl_reg_h2d = tl_socket_h2d[1];
100 assign tl_socket_d2h[1] = tl_reg_d2h;
101
102 assign tl_win_o = tl_socket_h2d[0];
103 assign tl_socket_d2h[0] = tl_win_i;
104
105 // Create Socket_1n
106 tlul_socket_1n #(
107   .N (2),
108   .HReqPass (1'b1),
109   .HRspPass (1'b1),
110   .DReqPass ({2{1'b1}}),
111   .DRspPass ({2{1'b1}}),
112   .HReqDepth (4'h0),
113   .HRspDepth (4'h0),
114   .DReqDepth ({2{4'h0}}),
115   .DRspDepth ({2{4'h0}}),
116   .ExplicitErrs (1'b0)
117 ) u_socket (
118   .clk_i (clk_i),
119   .rst_ni (rst_ni),
120   .tl_h_i (tl_i),
121   .tl_h_o (tl_o_pre),

```

```

123 .tl_d_o (tl_socket_h2d),
124 .tl_d_i (tl_socket_d2h),
125 .dev_select_i (reg_steer)
126 );
127
128 // Create steering logic
129 always_comb begin
130     reg_steer =
131         tl_i_a_address[AW-1:0] inside {[4096:8191]} ? 1'd0 :
132         // Default set to register
133         1'd1;
134
135     // Override this in case of an integrity error
136     if (intg_err) begin
137         reg_steer = 1'd1;
138     end
139 end
140
141 tlul_adapter_reg #(
142     .RegAw(AW),
143     .RegDw(DW),
144     .EnableDataIntgGen(0)
145 ) u_reg_if (
146     .clk_i (clk_i),
147     .rst_ni (rst_ni),
148
149     .tl_i (tl_reg_h2d),
150     .tl_o (tl_reg_d2h),
151
152     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
153     .intg_error_o()),
154
155     .we_o (reg_we),
156     .re_o (reg_re),
157     .addr_o (reg_addr),
158     .wdata_o (reg_wdata),
159     .be_o (reg_be),
160     .busy_i (reg_busy),
161     .rdata_i (reg_rdata),
162     .error_i (reg_error)
163 );
164
165 // cdc oversampling signals
166
167 assign reg_rdata = reg_rdata_next ;
168 assign reg_error = addrmiss | wr_err | intg_err;
169
170 // Define SW related signals
171 // Format: <reg>_<field>_{wd|we|qs}
172 // or <reg>_{wd|we|qs} if field == 1 or 0
173 logic intr_state_we;
174 logic intr_state_hmac_done_qs;
175 logic intr_state_hmac_done_wd;
176 logic intr_state_fifo_empty_qs;
177 logic intr_state_hmac_err_qs;
178 logic intr_state_hmac_err_wd;
179 logic intr_enable_we;
180 logic intr_enable_hmac_done_qs;
181 logic intr_enable_hmac_done_wd;
182 logic intr_enable_fifo_empty_qs;
183 logic intr_enable_fifo_empty_wd;
184 logic intr_enable_hmac_err_qs;
185 logic intr_enable_hmac_err_wd;
186 logic intr_test_we;
187 logic intr_test_hmac_done_wd;
188 logic intr_test_fifo_empty_wd;
189 logic intr_test_hmac_err_wd;
190 logic alert_test_we;
191 logic alert_test_wd;
192 logic cfg_re;
193 logic cfg_we;
194 logic cfg_hmac_en_qs;
195 logic cfg_hmac_en_wd;
196 logic cfg_sha_en_qs;
197 logic cfg_sha_en_wd;
198 logic cfg_endian_swap_qs;
199 logic cfg_endian_swap_wd;
200 logic cfg_digest_swap_qs;
201 logic cfg_digest_swap_wd;
202 logic cfg_key_swap_qs;
203 logic cfg_key_swap_wd;
204 logic [3:0] cfg_digest_size_qs;
205 logic [3:0] cfg_digest_size_wd;
206 logic [5:0] cfg_key_length_qs;
207 logic [5:0] cfg_key_length_wd;
208 logic cmd_we;
209 logic cmd_hash_start_wd;
210 logic cmd_hash_process_wd;
211 logic cmd_hash_stop_wd;
212 logic cmd_hash_continue_wd;
213 logic status_re;
214 logic status_hmac_idle_qs;
215 logic status_fifo_empty_qs;
216 logic status_fifo_full_qs;
217 logic [5:0] status_fifo_depth_qs;
218 logic [31:0] err_code_qs;
219 logic wipe_secret_we;
220 logic [31:0] wipe_secret_wd;
221 logic key_0_we;
222 logic [31:0] key_0_wd;
223 logic key_1_we;
224 logic [31:0] key_1_wd;
225 logic key_2_we;
226 logic [31:0] key_2_wd;
227 logic key_3_we;
228 logic [31:0] key_3_wd;
229 logic key_4_we;
230 logic [31:0] key_4_wd;
231 logic key_5_we;
232 logic [31:0] key_5_wd;
233 logic key_6_we;
234 logic [31:0] key_6_wd;
235 logic key_7_we;
236 logic [31:0] key_7_wd;
237 logic key_8_we;
238 logic [31:0] key_8_wd;
239 logic key_9_we;
240 logic [31:0] key_9_wd;
241 logic key_10_we;
242 logic [31:0] key_10_wd;
243 logic key_11_we;
244 logic [31:0] key_11_wd;
245 logic key_12_we;
246 logic [31:0] key_12_wd;
247 logic key_13_we;
248 logic [31:0] key_13_wd;
249 logic key_14_we;
250 logic [31:0] key_14_wd;
251 logic key_15_we;
252 logic [31:0] key_15_wd;
253 logic key_16_we;
254 logic [31:0] key_16_wd;
255 logic key_17_we;
256 logic [31:0] key_17_wd;
257 logic key_18_we;
258 logic [31:0] key_18_wd;
259 logic key_19_we;
260 logic [31:0] key_19_wd;
261 logic key_20_we;
262 logic [31:0] key_20_wd;
263 logic key_21_we;
264 logic [31:0] key_21_wd;
265 logic key_22_we;
266 logic [31:0] key_22_wd;
267 logic key_23_we;
268 logic [31:0] key_23_wd;
269 logic key_24_we;
270 logic [31:0] key_24_wd;
271 logic key_25_we;
272 logic [31:0] key_25_wd;
273 logic key_26_we;
274 logic [31:0] key_26_wd;
275 logic key_27_we;
276 logic [31:0] key_27_wd;
277 logic key_28_we;
278 logic [31:0] key_28_wd;
279 logic key_29_we;
280 logic [31:0] key_29_wd;
281 logic key_30_we;
282 logic [31:0] key_30_wd;
283 logic key_31_we;
284 logic [31:0] key_31_wd;
285 logic digest_0_re;
286 logic digest_0_we;
287 logic [31:0] digest_0_qs;
288 logic [31:0] digest_0_wd;
289 logic digest_1_re;
290 logic digest_1_we;
291 logic [31:0] digest_1_qs;
292 logic [31:0] digest_1_wd;
293 logic digest_2_re;
294 logic digest_2_we;
295 logic [31:0] digest_2_qs;
296 logic [31:0] digest_2_wd;
297 logic digest_3_re;
298 logic digest_3_we;
299 logic [31:0] digest_3_qs;
300 logic [31:0] digest_3_wd;
301 logic digest_4_re;
302 logic digest_4_we;
303 logic [31:0] digest_4_qs;
304 logic [31:0] digest_4_wd;
305 logic digest_5_re;
306 logic digest_5_we;
307 logic [31:0] digest_5_qs;
308 logic [31:0] digest_5_wd;
309 logic digest_6_re;
310 logic digest_6_we;
311 logic [31:0] digest_6_qs;
312 logic [31:0] digest_6_wd;
313 logic digest_7_re;
314 logic digest_7_we;
315 logic [31:0] digest_7_qs;
316 logic [31:0] digest_7_wd;
317 logic digest_8_re;
318 logic digest_8_we;
319 logic [31:0] digest_8_qs;
320 logic [31:0] digest_8_wd;
321 logic digest_9_re;
322 logic digest_9_we;
323 logic [31:0] digest_9_qs;
324 logic [31:0] digest_9_wd;

```

```

325 logic digest_10_re;
326 logic digest_10_we;
327 logic [31:0] digest_10_qs;
328 logic [31:0] digest_10_wd;
329 logic digest_11_re;
330 logic digest_11_we;
331 logic [31:0] digest_11_qs;
332 logic [31:0] digest_11_wd;
333 logic digest_12_re;
334 logic digest_12_we;
335 logic [31:0] digest_12_qs;
336 logic [31:0] digest_12_wd;
337 logic digest_13_re;
338 logic digest_13_we;
339 logic [31:0] digest_13_qs;
340 logic [31:0] digest_13_wd;
341 logic digest_14_re;
342 logic digest_14_we;
343 logic [31:0] digest_14_qs;
344 logic [31:0] digest_14_wd;
345 logic digest_15_re;
346 logic digest_15_we;
347 logic [31:0] digest_15_qs;
348 logic [31:0] digest_15_wd;
349 logic msg_length_lower_re;
350 logic msg_length_lower_we;
351 logic [31:0] msg_length_lower_qs;
352 logic [31:0] msg_length_lower_wd;
353 logic msg_length_upper_re;
354 logic msg_length_upper_we;
355 logic [31:0] msg_length_upper_qs;
356 logic [31:0] msg_length_upper_wd;
357
358 // Register instances
359 // R[intr_state]: V(False)
360 // F[hmac_done]: 0:0
361 prim_subreg #(
362   .DW (1),
363   .SwAccess(prim_subreg_pkg::SwAccessW1C),
364   .RESVAL (1'h0),
365   .Mubi (1'b0)
366 ) u_intr_state_hmac_done (
367   .clk_i (clk_i),
368   .rst_ni (rst_ni),
369
370   // from register interface
371   .we (intr_state_we),
372   .wd (intr_state_hmac_done_wd),
373
374   // from internal hardware
375   .de (hw2reg.intr_state.hmac_done.de),
376   .d (hw2reg.intr_state.hmac_done.d),
377
378   // to internal hardware
379   .qe (),
380   .q (reg2hw.intr_state.hmac_done.q),
381   .ds (),
382
383   // to register interface (read)
384   .qs (intr_state_hmac_done_qs)
385 );
386
387 // F[fifo_empty]: 1:1
388 prim_subreg #(
389   .DW (1),
390   .SwAccess(prim_subreg_pkg::SwAccessR0),
391   .RESVAL (1'h0),
392   .Mubi (1'b0)
393 ) u_intr_state_fifo_empty (
394   .clk_i (clk_i),
395   .rst_ni (rst_ni),
396
397   // from register interface
398   .we (1'b0),
399   .wd ('0'),
400
401   // from internal hardware
402   .de (hw2reg.intr_state_fifo_empty.de),
403   .d (hw2reg.intr_state_fifo_empty.d),
404
405   // to internal hardware
406   .qe (),
407   .q (reg2hw.intr_state_fifo_empty.q),
408   .ds (),
409
410   // to register interface (read)
411   .qs (intr_state_fifo_empty_qs)
412 );
413
414 // F[hmac_err]: 2:2
415 prim_subreg #(
416   .DW (1),
417   .SwAccess(prim_subreg_pkg::SwAccessW1C),
418   .RESVAL (1'h0),
419   .Mubi (1'b0)
420 ) u_intr_state_hmac_err (
421   .clk_i (clk_i),
422   .rst_ni (rst_ni),
423
424   // from register interface
425   .we (intr_state_we),
426
427   .wd (intr_state_hmac_err_wd),
428
429   // from internal hardware
430   .de (hw2reg.intr_state.hmac_err.de),
431   .d (hw2reg.intr_state.hmac_err.d),
432
433   // to internal hardware
434   .qe (),
435   .q (reg2hw.intr_state.hmac_err.q),
436   .ds (),
437
438   // to register interface (read)
439   .qs (intr_state_hmac_err_qs)
440 );
441
442 // R[intr_enable]: V(False)
443 // F[hmac_done]: 0:0
444 prim_subreg #(
445   .DW (1),
446   .SwAccess(prim_subreg_pkg::SwAccessRW),
447   .RESVAL (1'h0),
448   .Mubi (1'b0)
449 ) u_intr_enable_hmac_done (
450   .clk_i (clk_i),
451   .rst_ni (rst_ni),
452
453   // from register interface
454   .we (intr_enable_we),
455   .wd (intr_enable_hmac_done_wd),
456
457   // from internal hardware
458   .de (1'b0),
459   .d ('0'),
460
461   // to internal hardware
462   .qe (),
463   .q (reg2hw.intr_enable.hmac_done.q),
464   .ds (),
465
466   // to register interface (read)
467   .qs (intr_enable_hmac_done_qs)
468 );
469
470 // F[fifo_empty]: 1:1
471 prim_subreg #(
472   .DW (1),
473   .SwAccess(prim_subreg_pkg::SwAccessRW),
474   .RESVAL (1'h0),
475   .Mubi (1'b0)
476 ) u_intr_enable_fifo_empty (
477   .clk_i (clk_i),
478   .rst_ni (rst_ni),
479
480   // from register interface
481   .we (intr_enable_we),
482   .wd (intr_enable_fifo_empty_wd),
483
484   // from internal hardware
485   .de (1'b0),
486   .d ('0'),
487
488   // to internal hardware
489   .qe (),
490   .q (reg2hw.intr_enable_fifo_empty.q),
491   .ds (),
492
493   // to register interface (read)
494   .qs (intr_enable_fifo_empty_qs)
495 );
496
497 // F[hmac_err]: 2:2
498 prim_subreg #(
499   .DW (1),
500   .SwAccess(prim_subreg_pkg::SwAccessRW),
501   .RESVAL (1'h0),
502   .Mubi (1'b0)
503 ) u_intr_enable_hmac_err (
504   .clk_i (clk_i),
505   .rst_ni (rst_ni),
506
507   // from register interface
508   .we (intr_enable_we),
509   .wd (intr_enable_hmac_err_wd),
510
511   // from internal hardware
512   .de (1'b0),
513   .d ('0'),
514
515   // to internal hardware
516   .qe (),
517   .q (reg2hw.intr_enable.hmac_err.q),
518   .ds (),
519
520   // to register interface (read)
521   .qs (intr_enable_hmac_err_qs)
522 );
523
524 // R[intr_test]: V(True)
525 logic intr_test_qe;
526

```

```

527 logic [2:0] intr_test_flds_we;
528 assign intr_test_qe = &intr_test_flds_we;
529 // F[hmac_done]: 0:0
530 prim_subreg_ext #(
531   .DW (1)
532 ) u_intr_test_hmac_done (
533   .re (1'b0),
534   .we (intr_test_we),
535   .wd (intr_test_hmac_done_wd),
536   .d ('0'),
537   .qre (),
538   .qe (intr_test_flds_we[0]),
539   .q (reg2hw.intr_test.hmac_done.q),
540   .ds (),
541   .qs ()
542 );
543 assign reg2hw.intr_test.hmac_done.qe = intr_test_qe;
544
545 // F[fifo_empty]: 1:1
546 prim_subreg_ext #(
547   .DW (1)
548 ) u_intr_test_fifo_empty (
549   .re (1'b0),
550   .we (intr_test_we),
551   .wd (intr_test_fifo_empty_wd),
552   .d ('0'),
553   .qre (),
554   .qe (intr_test_flds_we[1]),
555   .q (reg2hw.intr_test.fifo_empty.q),
556   .ds (),
557   .qs ()
558 );
559 assign reg2hw.intr_test.fifo_empty.qe = intr_test_qe;
560
561 // F[hmac_err]: 2:2
562 prim_subreg_ext #(
563   .DW (1)
564 ) u_intr_test_hmac_err (
565   .re (1'b0),
566   .we (intr_test_we),
567   .wd (intr_test_hmac_err_wd),
568   .d ('0'),
569   .qre (),
570   .qe (intr_test_flds_we[2]),
571   .q (reg2hw.intr_test.hmac_err.q),
572   .ds (),
573   .qs ()
574 );
575 assign reg2hw.intr_test.hmac_err.qe = intr_test_qe;
576
577 // R[alert_test]: V(True)
578 logic alert_test_qe;
579 logic [0:0] alert_test_flds_we;
580 assign alert_test_qe = &alert_test_flds_we;
581 prim_subreg_ext #(
582   .DW (1)
583 ) u_alert_test (
584   .re (1'b0),
585   .we (alert_test_we),
586   .wd (alert_test_wd),
587   .d ('0'),
588   .qre (),
589   .qe (alert_test_flds_we[0]),
590   .q (reg2hw.alert_test.q),
591   .ds (),
592   .qs ()
593 );
594 assign reg2hw.alert_test.qe = alert_test_qe;
595
596 // R[cfg]: V(True)
597 logic cfg_qe;
598 logic [6:0] cfg_flds_we;
599 assign cfg_qe = &cfg_flds_we;
600 // F[hmac_en]: 0:0
601 prim_subreg_ext #(
602   .DW (1)
603 ) u_cfg_hmac_en (
604   .re (cfg_re),
605   .we (cfg_we),
606   .wd (cfg_hmac_en_wd),
607   .d (hw2reg.cfg.hmac_en.d),
608   .qre (),
609   .qe (cfg_flds_we[0]),
610   .q (reg2hw.cfg.hmac_en.q),
611   .ds (),
612   .qs (cfg_hmac_en_qs)
613 );
614 assign reg2hw.cfg.hmac_en.qe = cfg_qe;
615
616 // F[sha_en]: 1:1
617 prim_subreg_ext #(
618   .DW (1)
619 ) u_cfg_sha_en (
620   .re (cfg_re),
621   .we (cfg_we),
622   .wd (cfg_sha_en_wd),
623   .d (hw2reg.cfg.sha_en.d),
624   .qre (),
625   .qe (cfg_flds_we[1]),
626   .ds (),
627   .qs (cfg_sha_en_qs)
628 );
629 assign reg2hw.cfg.sha_en.qe = cfg_qe;
630
631 // F[endian_swap]: 2:2
632 prim_subreg_ext #(
633   .DW (1)
634 ) u_cfg_endian_swap (
635   .re (cfg_re),
636   .we (cfg_we),
637   .wd (cfg_endian_swap_wd),
638   .d (hw2reg.cfg.endian_swap.d),
639   .qre (),
640   .qe (cfg_flds_we[2]),
641   .q (reg2hw.cfg.endian_swap.q),
642   .ds (),
643   .qs (cfg_endian_swap_qs)
644 );
645 assign reg2hw.cfg.endian_swap.qe = cfg_qe;
646
647 // F[digest_swap]: 3:3
648 prim_subreg_ext #(
649   .DW (1)
650 ) u_cfg_digest_swap (
651   .re (cfg_re),
652   .we (cfg_we),
653   .wd (cfg_digest_swap_wd),
654   .d (hw2reg.cfg.digest_swap.d),
655   .qre (),
656   .qe (cfg_flds_we[3]),
657   .q (reg2hw.cfg.digest_swap.q),
658   .ds (),
659   .qs (cfg_digest_swap_qs)
660 );
661 assign reg2hw.cfg.digest_swap.qe = cfg_qe;
662
663 // F[key_swap]: 4:4
664 prim_subreg_ext #(
665   .DW (1)
666 ) u_cfg_key_swap (
667   .re (cfg_re),
668   .we (cfg_we),
669   .wd (cfg_key_swap_wd),
670   .d (hw2reg.cfg.key_swap.d),
671   .qre (),
672   .qe (cfg_flds_we[4]),
673   .q (reg2hw.cfg.key_swap.q),
674   .ds (),
675   .qs (cfg_key_swap_qs)
676 );
677 assign reg2hw.cfg.key_swap.qe = cfg_qe;
678
679 // F[digest_size]: 8:5
680 prim_subreg_ext #(
681   .DW (4)
682 ) u_cfg_digest_size (
683   .re (cfg_re),
684   .we (cfg_we),
685   .wd (cfg_digest_size_wd),
686   .d (hw2reg.cfg.digest_size.d),
687   .qre (),
688   .qe (cfg_flds_we[5]),
689   .q (reg2hw.cfg.digest_size.q),
690   .ds (),
691   .qs (cfg_digest_size_qs)
692 );
693 assign reg2hw.cfg.digest_size.qe = cfg_qe;
694
695 // F[key_length]: 14:9
696 prim_subreg_ext #(
697   .DW (6)
698 ) u_cfg_key_length (
699   .re (cfg_re),
700   .we (cfg_we),
701   .wd (cfg_key_length_wd),
702   .d (hw2reg.cfg.key_length.d),
703   .qre (),
704   .qe (cfg_flds_we[6]),
705   .q (reg2hw.cfg.key_length.q),
706   .ds (),
707   .qs (cfg_key_length_qs)
708 );
709 assign reg2hw.cfg.key_length.qe = cfg_qe;
710
711 // R[cmd]: V(True)
712 logic cmd_qe;
713 logic [3:0] cmd_flds_we;
714 assign cmd_qe = &cmd_flds_we;
715 // F[hash_start]: 0:0
716 prim_subreg_ext #(
717   .DW (1)
718 ) u_cmd_hash_start (
719   .re (1'b0),
720   .we (cmd_we),
721   .wd (cmd_hash_start_wd),
722   .d ('0'),
723   .qre (),
724   .qe (cmd_flds_we[0]),
725   .ds (),
726   .qs (cmd_hash_start_qs)
727 );
728 assign reg2hw.cmd.hash_start.qe = cmd_qe;

```



```

729 .q (reg2hw.cmd.hash_start.q),
730 .ds (),
731 .qs ()
732 );
733 assign reg2hw.cmd.hash_start.qe = cmd_qe;
734
735 // F[hash_process]: 1:1
736 prim_subreg_ext #(
737 .DW (1)
738 ) u_cmd_hash_process (
739 .re (1'b0),
740 .we (cmd_we),
741 .wd (cmd_hash_process_wd),
742 .d ('0'),
743 .qre (),
744 .qe (cmd_flds_we[1]),
745 .q (reg2hw.cmd.hash_process.q),
746 .ds (),
747 .qs ()
748 );
749 assign reg2hw.cmd.hash_process.qe = cmd_qe;
750
751 // F[hash_stop]: 2:2
752 prim_subreg_ext #(
753 .DW (1)
754 ) u_cmd_hash_stop (
755 .re (1'b0),
756 .we (cmd_we),
757 .wd (cmd_hash_stop_wd),
758 .d ('0'),
759 .qre (),
760 .qe (cmd_flds_we[2]),
761 .q (reg2hw.cmd.hash_stop.q),
762 .ds (),
763 .qs ()
764 );
765 assign reg2hw.cmd.hash_stop.qe = cmd_qe;
766
767 // F[hash_continue]: 3:3
768 prim_subreg_ext #(
769 .DW (1)
770 ) u_cmd_hash_continue (
771 .re (1'b0),
772 .we (cmd_we),
773 .wd (cmd_hash_continue_wd),
774 .d ('0'),
775 .qre (),
776 .qe (cmd_flds_we[3]),
777 .q (reg2hw.cmd.hash_continue.q),
778 .ds (),
779 .qs ()
780 );
781 assign reg2hw.cmd.hash_continue.qe = cmd_qe;
782
783 // R[status]: V(True)
784 // F[hmac_idle]: 0:0
785 prim_subreg_ext #(
786 .DW (1)
787 ) u_status_hmac_idle (
788 .re (status_re),
789 .we (1'b0),
790 .wd ('0'),
791 .d (hw2reg.status.hmac_idle.d),
792 .qre (),
793 .qe (),
794 .q (),
795 .ds (),
796 .qs (status_hmac_idle_qs)
797 );
798
799 // F[fifo_empty]: 1:1
800 prim_subreg_ext #(
801 .DW (1)
802 ) u_status_fifo_empty (
803 .re (status_re),
804 .we (1'b0),
805 .wd ('0'),
806 .d (hw2reg.status.fifo_empty.d),
807 .qre (),
808 .qe (),
809 .q (),
810 .ds (),
811 .qs (status_fifo_empty_qs)
812 );
813
814 // F[fifo_full]: 2:2
815 prim_subreg_ext #(
816 .DW (1)
817 ) u_status_fifo_full (
818 .re (status_re),
819 .we (1'b0),
820 .wd ('0'),
821 .d (hw2reg.status.fifo_full.d),
822 .qre (),
823 .qe (),
824 .q (),
825 .ds (),
826 .qs (status_fifo_full_qs)
827 );
828
829
830 // F[fifo_depth]: 9:4
831 prim_subreg_ext #(
832 .DW (6)
833 ) u_status_fifo_depth (
834 .re (status_re),
835 .we (1'b0),
836 .wd ('0'),
837 .d (hw2reg.status.fifo_depth.d),
838 .qre (),
839 .qe (),
840 .q (),
841 .ds (),
842 .qs (status_fifo_depth_qs)
843 );
844
845 // R[err_code]: V(False)
846 prim_subreg_ext #(
847 .DW (32),
848 .SwAccess(prim_subreg_pkg::SwAccessR0),
849 .RESVAL (32'h0),
850 .Mubi (1'b0)
851 ) u_err_code (
852 .clk_i (clk_i),
853 .rst_ni (rst_ni),
854
855 // from register interface
856 .we (1'b0),
857 .wd ('0'),
858
859 // from internal hardware
860 .de (hw2reg.err_code.de),
861 .d (hw2reg.err_code.d),
862
863 // to internal hardware
864 .qe (),
865 .q (),
866 .ds (),
867
868 // to register interface (read)
869 .qs (err_code_qs)
870 );
871
872 // R[wipe_secret]: V(True)
873 logic wipe_secret_qe;
874 logic [0:0] wipe_secret_flds_we;
875 assign wipe_secret_qe = &wipe_secret_flds_we;
876 prim_subreg_ext #(
877 .DW (32)
878 ) u_wipe_secret (
879 .re (1'b0),
880 .we (wipe_secret_we),
881 .wd (wipe_secret_wd),
882 .d ('0'),
883 .qre (),
884 .qe (wipe_secret_flds_we[0]),
885 .q (reg2hw.wipe_secret.q),
886 .ds (),
887 .qs ()
888 );
889 assign reg2hw.wipe_secret.qe = wipe_secret_qe;
890
891 // Subregister 0 of Multireg key
892 // R[key_0]: V(True)
893 logic key_0_qe;
894 logic [0:0] key_0_flds_we;
895 assign key_0_qe = &key_0_flds_we;
896 prim_subreg_ext #(
897 .DW (32)
898 ) u_key_0 (
899 .re (1'b0),
900 .we (key_0_we),
901 .wd (key_0_wd),
902 .d (hw2reg.key[0].d),
903 .qre (),
904 .qe (key_0_flds_we[0]),
905 .q (reg2hw.key[0].q),
906 .ds (),
907 .qs ()
908 );
909 assign reg2hw.key[0].qe = key_0_qe;
910
911 // Subregister 1 of Multireg key
912 // R[key_1]: V(True)
913 logic key_1_qe;
914 logic [0:0] key_1_flds_we;
915 assign key_1_qe = &key_1_flds_we;
916 prim_subreg_ext #(
917 .DW (32)
918 ) u_key_1 (
919 .re (1'b0),
920 .we (key_1_we),
921 .wd (key_1_wd),
922 .d (hw2reg.key[1].d),
923 .qre (),
924 .qe (key_1_flds_we[0]),
925 .q (reg2hw.key[1].q),
926 .ds (),
927 .qs ()
928 );
929
930

```

```

931     .qs      ()
932 );
933 assign reg2hw.key[1].qe = key_1_qe;
934
935 // Subregister 2 of Multireg key
936 // R[key_2]: V(True)
937 logic key_2_qe;
938 logic [0:0] key_2_flds_we;
939 assign key_2_qe = &key_2_flds_we;
940 prim_subreg_ext #(
941     .DW      (32)
942 ) u_key_2 (
943     .re      (1'b0),
944     .we      (key_2_we),
945     .wd      (key_2_wd),
946     .d       (hw2reg.key[2].d),
947     .qre     (),
948     .qe      (key_2_flds_we[0]),
949     .q       (reg2hw.key[2].q),
950     .ds      (),
951     .qs      ()
952 );
953 assign reg2hw.key[2].qe = key_2_qe;
954
955 // Subregister 3 of Multireg key
956 // R[key_3]: V(True)
957 logic key_3_qe;
958 logic [0:0] key_3_flds_we;
959 assign key_3_qe = &key_3_flds_we;
960 prim_subreg_ext #(
961     .DW      (32)
962 ) u_key_3 (
963     .re      (1'b0),
964     .we      (key_3_we),
965     .wd      (key_3_wd),
966     .d       (hw2reg.key[3].d),
967     .qre     (),
968     .qe      (key_3_flds_we[0]),
969     .q       (reg2hw.key[3].q),
970     .ds      (),
971     .qs      ()
972 );
973 assign reg2hw.key[3].qe = key_3_qe;
974
975 // Subregister 4 of Multireg key
976 // R[key_4]: V(True)
977 logic key_4_qe;
978 logic [0:0] key_4_flds_we;
979 assign key_4_qe = &key_4_flds_we;
980 prim_subreg_ext #(
981     .DW      (32)
982 ) u_key_4 (
983     .re      (1'b0),
984     .we      (key_4_we),
985     .wd      (key_4_wd),
986     .d       (hw2reg.key[4].d),
987     .qre     (),
988     .qe      (key_4_flds_we[0]),
989     .q       (reg2hw.key[4].q),
990     .ds      (),
991     .qs      ()
992 );
993 assign reg2hw.key[4].qe = key_4_qe;
994
995 // Subregister 5 of Multireg key
996 // R[key_5]: V(True)
997 logic key_5_qe;
998 logic [0:0] key_5_flds_we;
999 assign key_5_qe = &key_5_flds_we;
1000 prim_subreg_ext #(
1001     .DW      (32)
1002 ) u_key_5 (
1003     .re      (1'b0),
1004     .we      (key_5_we),
1005     .wd      (key_5_wd),
1006     .d       (hw2reg.key[5].d),
1007     .qre     (),
1008     .qe      (key_5_flds_we[0]),
1009     .q       (reg2hw.key[5].q),
1010     .ds      (),
1011     .qs      ()
1012 );
1013 assign reg2hw.key[5].qe = key_5_qe;
1014
1015 // Subregister 6 of Multireg key
1016 // R[key_6]: V(True)
1017 logic key_6_qe;
1018 logic [0:0] key_6_flds_we;
1019 assign key_6_qe = &key_6_flds_we;
1020 prim_subreg_ext #(
1021     .DW      (32)
1022 ) u_key_6 (
1023     .re      (1'b0),
1024     .we      (key_6_we),
1025     .wd      (key_6_wd),
1026     .d       (hw2reg.key[6].d),

```

```

1032     .qre     (),
1033     .qe      (key_6_flds_we[0]),
1034     .q       (reg2hw.key[6].q),
1035     .ds      (),
1036     .qs      ()
1037 );
1038 assign reg2hw.key[6].qe = key_6_qe;
1039
1040 // Subregister 7 of Multireg key
1041 // R[key_7]: V(True)
1042 logic key_7_qe;
1043 logic [0:0] key_7_flds_we;
1044 assign key_7_qe = &key_7_flds_we;
1045 prim_subreg_ext #(
1046     .DW      (32)
1047 ) u_key_7 (
1048     .re      (1'b0),
1049     .we      (key_7_we),
1050     .wd      (key_7_wd),
1051     .d       (hw2reg.key[7].d),
1052     .qre     (),
1053     .qe      (key_7_flds_we[0]),
1054     .q       (reg2hw.key[7].q),
1055     .ds      (),
1056     .qs      ()
1057 );
1058 assign reg2hw.key[7].qe = key_7_qe;
1059
1060 // Subregister 8 of Multireg key
1061 // R[key_8]: V(True)
1062 logic key_8_qe;
1063 logic [0:0] key_8_flds_we;
1064 assign key_8_qe = &key_8_flds_we;
1065 prim_subreg_ext #(
1066     .DW      (32)
1067 ) u_key_8 (
1068     .re      (1'b0),
1069     .we      (key_8_we),
1070     .wd      (key_8_wd),
1071     .d       (hw2reg.key[8].d),
1072     .qre     (),
1073     .qe      (key_8_flds_we[0]),
1074     .q       (reg2hw.key[8].q),
1075     .ds      (),
1076     .qs      ()
1077 );
1078 assign reg2hw.key[8].qe = key_8_qe;
1079
1080 // Subregister 9 of Multireg key
1081 // R[key_9]: V(True)
1082 logic key_9_qe;
1083 logic [0:0] key_9_flds_we;
1084 assign key_9_qe = &key_9_flds_we;
1085 prim_subreg_ext #(
1086     .DW      (32)
1087 ) u_key_9 (
1088     .re      (1'b0),
1089     .we      (key_9_we),
1090     .wd      (key_9_wd),
1091     .d       (hw2reg.key[9].d),
1092     .qre     (),
1093     .qe      (key_9_flds_we[0]),
1094     .q       (reg2hw.key[9].q),
1095     .ds      (),
1096     .qs      ()
1097 );
1098 assign reg2hw.key[9].qe = key_9_qe;
1099
1100 // Subregister 10 of Multireg key
1101 // R[key_10]: V(True)
1102 logic key_10_qe;
1103 logic [0:0] key_10_flds_we;
1104 assign key_10_qe = &key_10_flds_we;
1105 prim_subreg_ext #(
1106     .DW      (32)
1107 ) u_key_10 (
1108     .re      (1'b0),
1109     .we      (key_10_we),
1110     .wd      (key_10_wd),
1111     .d       (hw2reg.key[10].d),
1112     .qre     (),
1113     .qe      (key_10_flds_we[0]),
1114     .q       (reg2hw.key[10].q),
1115     .ds      (),
1116     .qs      ()
1117 );
1118 assign reg2hw.key[10].qe = key_10_qe;
1119
1120 // Subregister 11 of Multireg key
1121 // R[key_11]: V(True)
1122 logic key_11_qe;
1123 logic [0:0] key_11_flds_we;
1124 assign key_11_qe = &key_11_flds_we;
1125 prim_subreg_ext #(
1126     .DW      (32)
1127 ) u_key_11 (

```

```

1133 .re (1'b0),
1134 .we (key_11_we),
1135 .wd (key_11_wd),
1136 .d (hw2reg.key[11].d),
1137 .qre (),
1138 .qe (key_11_flds_we[0]),
1139 .q (reg2hw.key[11].q),
1140 .ds (),
1141 .qs ();
1142 );
1143 assign reg2hw.key[11].qe = key_11_qe;
1144
1145 // Subregister 12 of Multireg key
1146 // R[key_12]: V(True)
1147 logic key_12_qe;
1148 logic [0:0] key_12_flds_we;
1149 assign key_12_qe = &key_12_flds_we;
1150 prim_subreg_ext #(
1151 .DW (32)
1152 ) u_key_12 (
1153 .re (1'b0),
1154 .we (key_12_we),
1155 .wd (key_12_wd),
1156 .d (hw2reg.key[12].d),
1157 .qre (),
1158 .qe (key_12_flds_we[0]),
1159 .q (reg2hw.key[12].q),
1160 .ds (),
1161 .qs ();
1162 );
1163 assign reg2hw.key[12].qe = key_12_qe;
1164
1165 // Subregister 13 of Multireg key
1166 // R[key_13]: V(True)
1167 logic key_13_qe;
1168 logic [0:0] key_13_flds_we;
1169 assign key_13_qe = &key_13_flds_we;
1170 prim_subreg_ext #(
1171 .DW (32)
1172 ) u_key_13 (
1173 .re (1'b0),
1174 .we (key_13_we),
1175 .wd (key_13_wd),
1176 .d (hw2reg.key[13].d),
1177 .qre (),
1178 .qe (key_13_flds_we[0]),
1179 .q (reg2hw.key[13].q),
1180 .ds (),
1181 .qs ();
1182 );
1183 assign reg2hw.key[13].qe = key_13_qe;
1184
1185 // Subregister 14 of Multireg key
1186 // R[key_14]: V(True)
1187 logic key_14_qe;
1188 logic [0:0] key_14_flds_we;
1189 assign key_14_qe = &key_14_flds_we;
1190 prim_subreg_ext #(
1191 .DW (32)
1192 ) u_key_14 (
1193 .re (1'b0),
1194 .we (key_14_we),
1195 .wd (key_14_wd),
1196 .d (hw2reg.key[14].d),
1197 .qre (),
1198 .qe (key_14_flds_we[0]),
1199 .q (reg2hw.key[14].q),
1200 .ds (),
1201 .qs ();
1202 );
1203 assign reg2hw.key[14].qe = key_14_qe;
1204
1205 // Subregister 15 of Multireg key
1206 // R[key_15]: V(True)
1207 logic key_15_qe;
1208 logic [0:0] key_15_flds_we;
1209 assign key_15_qe = &key_15_flds_we;
1210 prim_subreg_ext #(
1211 .DW (32)
1212 ) u_key_15 (
1213 .re (1'b0),
1214 .we (key_15_we),
1215 .wd (key_15_wd),
1216 .d (hw2reg.key[15].d),
1217 .qre (),
1218 .qe (key_15_flds_we[0]),
1219 .q (reg2hw.key[15].q),
1220 .ds (),
1221 .qs ();
1222 );
1223 assign reg2hw.key[15].qe = key_15_qe;
1224
1225 // Subregister 16 of Multireg key
1226 // R[key_16]: V(True)
1227 logic key_16_qe;
1228 logic [0:0] key_16_flds_we;
1229
1230 assign key_16_qe = &key_16_flds_we;
1231 prim_subreg_ext #(
1232 .DW (32)
1233 ) u_key_16 (
1234 .re (1'b0),
1235 .we (key_16_we),
1236 .wd (key_16_wd),
1237 .d (hw2reg.key[16].d),
1238 .qre (),
1239 .qe (key_16_flds_we[0]),
1240 .q (reg2hw.key[16].q),
1241 .ds (),
1242 .qs ();
1243 );
1244 assign reg2hw.key[16].qe = key_16_qe;
1245
1246 // Subregister 17 of Multireg key
1247 // R[key_17]: V(True)
1248 logic key_17_qe;
1249 logic [0:0] key_17_flds_we;
1250 assign key_17_qe = &key_17_flds_we;
1251 prim_subreg_ext #(
1252 .DW (32)
1253 ) u_key_17 (
1254 .re (1'b0),
1255 .we (key_17_we),
1256 .wd (key_17_wd),
1257 .d (hw2reg.key[17].d),
1258 .qre (),
1259 .qe (key_17_flds_we[0]),
1260 .q (reg2hw.key[17].q),
1261 .ds (),
1262 .qs ();
1263 );
1264 assign reg2hw.key[17].qe = key_17_qe;
1265
1266 // Subregister 18 of Multireg key
1267 // R[key_18]: V(True)
1268 logic key_18_qe;
1269 logic [0:0] key_18_flds_we;
1270 assign key_18_qe = &key_18_flds_we;
1271 prim_subreg_ext #(
1272 .DW (32)
1273 ) u_key_18 (
1274 .re (1'b0),
1275 .we (key_18_we),
1276 .wd (key_18_wd),
1277 .d (hw2reg.key[18].d),
1278 .qre (),
1279 .qe (key_18_flds_we[0]),
1280 .q (reg2hw.key[18].q),
1281 .ds (),
1282 .qs ();
1283 );
1284 assign reg2hw.key[18].qe = key_18_qe;
1285
1286 // Subregister 19 of Multireg key
1287 // R[key_19]: V(True)
1288 logic key_19_qe;
1289 logic [0:0] key_19_flds_we;
1290 assign key_19_qe = &key_19_flds_we;
1291 prim_subreg_ext #(
1292 .DW (32)
1293 ) u_key_19 (
1294 .re (1'b0),
1295 .we (key_19_we),
1296 .wd (key_19_wd),
1297 .d (hw2reg.key[19].d),
1298 .qre (),
1299 .qe (key_19_flds_we[0]),
1300 .q (reg2hw.key[19].q),
1301 .ds (),
1302 .qs ();
1303 );
1304 assign reg2hw.key[19].qe = key_19_qe;
1305
1306 // Subregister 20 of Multireg key
1307 // R[key_20]: V(True)
1308 logic key_20_qe;
1309 logic [0:0] key_20_flds_we;
1310 assign key_20_qe = &key_20_flds_we;
1311 prim_subreg_ext #(
1312 .DW (32)
1313 ) u_key_20 (
1314 .re (1'b0),
1315 .we (key_20_we),
1316 .wd (key_20_wd),
1317 .d (hw2reg.key[20].d),
1318 .qre (),
1319 .qe (key_20_flds_we[0]),
1320 .q (reg2hw.key[20].q),
1321 .ds (),
1322 .qs ();
1323 );
1324 assign reg2hw.key[20].qe = key_20_qe;
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334

```



```

1335 // Subregister 21 of Multireg key
1336 // R[key_21]: V(True)
1337 logic key_21_qe;
1338 logic [0:0] key_21_flds_we;
1339 assign key_21_qe = &key_21_flds_we;
1340 prim_subreg_ext #(
1341     .DW (32)
1342 ) u_key_21 (
1343     .re (1'b0),
1344     .we (key_21_we),
1345     .wd (key_21_wd),
1346     .d (hw2reg.key[21].d),
1347     .qre (),
1348     .qe (key_21_flds_we[0]),
1349     .q (reg2hw.key[21].q),
1350     .ds (),
1351     .qs ()
1352 );
1353 assign reg2hw.key[21].qe = key_21_qe;
1354
1355 // Subregister 22 of Multireg key
1356 // R[key_22]: V(True)
1357 logic key_22_qe;
1358 logic [0:0] key_22_flds_we;
1359 assign key_22_qe = &key_22_flds_we;
1360 prim_subreg_ext #(
1361     .DW (32)
1362 ) u_key_22 (
1363     .re (1'b0),
1364     .we (key_22_we),
1365     .wd (key_22_wd),
1366     .d (hw2reg.key[22].d),
1367     .qre (),
1368     .qe (key_22_flds_we[0]),
1369     .q (reg2hw.key[22].q),
1370     .ds (),
1371     .qs ()
1372 );
1373 assign reg2hw.key[22].qe = key_22_qe;
1374
1375 // Subregister 23 of Multireg key
1376 // R[key_23]: V(True)
1377 logic key_23_qe;
1378 logic [0:0] key_23_flds_we;
1379 assign key_23_qe = &key_23_flds_we;
1380 prim_subreg_ext #(
1381     .DW (32)
1382 ) u_key_23 (
1383     .re (1'b0),
1384     .we (key_23_we),
1385     .wd (key_23_wd),
1386     .d (hw2reg.key[23].d),
1387     .qre (),
1388     .qe (key_23_flds_we[0]),
1389     .q (reg2hw.key[23].q),
1390     .ds (),
1391     .qs ()
1392 );
1393 assign reg2hw.key[23].qe = key_23_qe;
1394
1395 // Subregister 24 of Multireg key
1396 // R[key_24]: V(True)
1397 logic key_24_qe;
1398 logic [0:0] key_24_flds_we;
1399 assign key_24_qe = &key_24_flds_we;
1400 prim_subreg_ext #(
1401     .DW (32)
1402 ) u_key_24 (
1403     .re (1'b0),
1404     .we (key_24_we),
1405     .wd (key_24_wd),
1406     .d (hw2reg.key[24].d),
1407     .qre (),
1408     .qe (key_24_flds_we[0]),
1409     .q (reg2hw.key[24].q),
1410     .ds (),
1411     .qs ()
1412 );
1413 assign reg2hw.key[24].qe = key_24_qe;
1414
1415 // Subregister 25 of Multireg key
1416 // R[key_25]: V(True)
1417 logic key_25_qe;
1418 logic [0:0] key_25_flds_we;
1419 assign key_25_qe = &key_25_flds_we;
1420 prim_subreg_ext #(
1421     .DW (32)
1422 ) u_key_25 (
1423     .re (1'b0),
1424     .we (key_25_we),
1425     .wd (key_25_wd),
1426     .d (hw2reg.key[25].d),
1427     .qre (),
1428     .qe (key_25_flds_we[0]),
1429     .q (reg2hw.key[25].q),
1430     .ds (),
1431     .qs ()
1432 );
1433
1434 // Subregister 26 of Multireg key
1435 // R[key_26]: V(True)
1436 logic key_26_qe;
1437 logic [0:0] key_26_flds_we;
1438 assign key_26_qe = &key_26_flds_we;
1439 prim_subreg_ext #(
1440     .DW (32)
1441 ) u_key_26 (
1442     .re (1'b0),
1443     .we (key_26_we),
1444     .wd (key_26_wd),
1445     .d (hw2reg.key[26].d),
1446     .qre (),
1447     .qe (key_26_flds_we[0]),
1448     .q (reg2hw.key[26].q),
1449     .ds (),
1450     .qs ()
1451 );
1452 assign reg2hw.key[26].qe = key_26_qe;
1453
1454 // Subregister 27 of Multireg key
1455 // R[key_27]: V(True)
1456 logic key_27_qe;
1457 logic [0:0] key_27_flds_we;
1458 assign key_27_qe = &key_27_flds_we;
1459 prim_subreg_ext #(
1460     .DW (32)
1461 ) u_key_27 (
1462     .re (1'b0),
1463     .we (key_27_we),
1464     .wd (key_27_wd),
1465     .d (hw2reg.key[27].d),
1466     .qre (),
1467     .qe (key_27_flds_we[0]),
1468     .q (reg2hw.key[27].q),
1469     .ds (),
1470     .qs ()
1471 );
1472 assign reg2hw.key[27].qe = key_27_qe;
1473
1474 // Subregister 28 of Multireg key
1475 // R[key_28]: V(True)
1476 logic key_28_qe;
1477 logic [0:0] key_28_flds_we;
1478 assign key_28_qe = &key_28_flds_we;
1479 prim_subreg_ext #(
1480     .DW (32)
1481 ) u_key_28 (
1482     .re (1'b0),
1483     .we (key_28_we),
1484     .wd (key_28_wd),
1485     .d (hw2reg.key[28].d),
1486     .qre (),
1487     .qe (key_28_flds_we[0]),
1488     .q (reg2hw.key[28].q),
1489     .ds (),
1490     .qs ()
1491 );
1492 assign reg2hw.key[28].qe = key_28_qe;
1493
1494 // Subregister 29 of Multireg key
1495 // R[key_29]: V(True)
1496 logic key_29_qe;
1497 logic [0:0] key_29_flds_we;
1498 assign key_29_qe = &key_29_flds_we;
1499 prim_subreg_ext #(
1500     .DW (32)
1501 ) u_key_29 (
1502     .re (1'b0),
1503     .we (key_29_we),
1504     .wd (key_29_wd),
1505     .d (hw2reg.key[29].d),
1506     .qre (),
1507     .qe (key_29_flds_we[0]),
1508     .q (reg2hw.key[29].q),
1509     .ds (),
1510     .qs ()
1511 );
1512 assign reg2hw.key[29].qe = key_29_qe;
1513
1514 // Subregister 30 of Multireg key
1515 // R[key_30]: V(True)
1516 logic key_30_qe;
1517 logic [0:0] key_30_flds_we;
1518 assign key_30_qe = &key_30_flds_we;
1519 prim_subreg_ext #(
1520     .DW (32)
1521 ) u_key_30 (
1522     .re (1'b0),
1523     .we (key_30_we),
1524     .wd (key_30_wd),
1525     .d (hw2reg.key[30].d),
1526     .qre (),
1527     .qe (key_30_flds_we[0]),
1528     .q (reg2hw.key[30].q),
1529     .ds (),
1530     .qs ()
1531 );
1532

```

```

1537 .qe (key_30_flds_we[0]),
1538 .q (reg2hw.key[30].q),
1539 .ds (),
1540 .qs ()
1541 );
1542 assign reg2hw.key[30].qe = key_30_qe;
1543
1544 // Subregister 31 of Multireg key
1545 // R[key_31]: V(True)
1546 logic key_31_qe;
1547 logic [0:0] key_31_flds_we;
1548 assign key_31_qe = &key_31_flds_we;
1549 prim_subreg_ext #(
1550 .DW (32)
1551 ) u_key_31 (
1552 .re (1'b0),
1553 .we (key_31_we),
1554 .wd (key_31_wd),
1555 .d (hw2reg.key[31].d),
1556 .qre (),
1557 .qe (key_31_flds_we[0]),
1558 .q (reg2hw.key[31].q),
1559 .ds (),
1560 .qs ()
1561 );
1562 assign reg2hw.key[31].qe = key_31_qe;
1563
1564 // Subregister 0 of Multireg digest
1565 // R[digest_0]: V(True)
1566 logic digest_0_qe;
1567 logic [0:0] digest_0_flds_we;
1568 assign digest_0_qe = &digest_0_flds_we;
1569 prim_subreg_ext #(
1570 .DW (32)
1571 ) u_digest_0 (
1572 .re (digest_0_re),
1573 .we (digest_0_we),
1574 .wd (digest_0_wd),
1575 .d (hw2reg.digest[0].d),
1576 .qre (),
1577 .qe (digest_0_flds_we[0]),
1578 .q (reg2hw.digest[0].q),
1579 .ds (),
1580 .qs (digest_0_qs)
1581 );
1582 assign reg2hw.digest[0].qe = digest_0_qe;
1583
1584 // Subregister 1 of Multireg digest
1585 // R[digest_1]: V(True)
1586 logic digest_1_qe;
1587 logic [0:0] digest_1_flds_we;
1588 assign digest_1_qe = &digest_1_flds_we;
1589 prim_subreg_ext #(
1590 .DW (32)
1591 ) u_digest_1 (
1592 .re (digest_1_re),
1593 .we (digest_1_we),
1594 .wd (digest_1_wd),
1595 .d (hw2reg.digest[1].d),
1596 .qre (),
1597 .qe (digest_1_flds_we[0]),
1598 .q (reg2hw.digest[1].q),
1599 .ds (),
1600 .qs (digest_1_qs)
1601 );
1602 assign reg2hw.digest[1].qe = digest_1_qe;
1603
1604 // Subregister 2 of Multireg digest
1605 // R[digest_2]: V(True)
1606 logic digest_2_qe;
1607 logic [0:0] digest_2_flds_we;
1608 assign digest_2_qe = &digest_2_flds_we;
1609 prim_subreg_ext #(
1610 .DW (32)
1611 ) u_digest_2 (
1612 .re (digest_2_re),
1613 .we (digest_2_we),
1614 .wd (digest_2_wd),
1615 .d (hw2reg.digest[2].d),
1616 .qre (),
1617 .qe (digest_2_flds_we[0]),
1618 .q (reg2hw.digest[2].q),
1619 .ds (),
1620 .qs (digest_2_qs)
1621 );
1622 assign reg2hw.digest[2].qe = digest_2_qe;
1623
1624 // Subregister 3 of Multireg digest
1625 // R[digest_3]: V(True)
1626 logic digest_3_qe;
1627 logic [0:0] digest_3_flds_we;
1628 assign digest_3_qe = &digest_3_flds_we;
1629 prim_subreg_ext #(
1630 .DW (32)
1631 ) u_digest_3 (
1632 .re (digest_3_re),

```

```

1633 .we (digest_3_we),
1634 .wd (digest_3_wd),
1635 .d (hw2reg.digest[3].d),
1636 .qre (),
1637 .qe (digest_3_flds_we[0]),
1638 .q (reg2hw.digest[3].q),
1639 .ds (),
1640 .qs (digest_3_qs)
1641 );
1642 assign reg2hw.digest[3].qe = digest_3_qe;
1643
1644 // Subregister 4 of Multireg digest
1645 // R[digest_4]: V(True)
1646 logic digest_4_qe;
1647 logic [0:0] digest_4_flds_we;
1648 assign digest_4_qe = &digest_4_flds_we;
1649 prim_subreg_ext #(
1650 .DW (32)
1651 ) u_digest_4 (
1652 .re (digest_4_re),
1653 .we (digest_4_we),
1654 .wd (digest_4_wd),
1655 .d (hw2reg.digest[4].d),
1656 .qre (),
1657 .qe (digest_4_flds_we[0]),
1658 .q (reg2hw.digest[4].q),
1659 .ds (),
1660 .qs (digest_4_qs)
1661 );
1662 assign reg2hw.digest[4].qe = digest_4_qe;
1663
1664 // Subregister 5 of Multireg digest
1665 // R[digest_5]: V(True)
1666 logic digest_5_qe;
1667 logic [0:0] digest_5_flds_we;
1668 assign digest_5_qe = &digest_5_flds_we;
1669 prim_subreg_ext #(
1670 .DW (32)
1671 ) u_digest_5 (
1672 .re (digest_5_re),
1673 .we (digest_5_we),
1674 .wd (digest_5_wd),
1675 .d (hw2reg.digest[5].d),
1676 .qre (),
1677 .qe (digest_5_flds_we[0]),
1678 .q (reg2hw.digest[5].q),
1679 .ds (),
1680 .qs (digest_5_qs)
1681 );
1682 assign reg2hw.digest[5].qe = digest_5_qe;
1683
1684 // Subregister 6 of Multireg digest
1685 // R[digest_6]: V(True)
1686 logic digest_6_qe;
1687 logic [0:0] digest_6_flds_we;
1688 assign digest_6_qe = &digest_6_flds_we;
1689 prim_subreg_ext #(
1690 .DW (32)
1691 ) u_digest_6 (
1692 .re (digest_6_re),
1693 .we (digest_6_we),
1694 .wd (digest_6_wd),
1695 .d (hw2reg.digest[6].d),
1696 .qre (),
1697 .qe (digest_6_flds_we[0]),
1698 .q (reg2hw.digest[6].q),
1699 .ds (),
1700 .qs (digest_6_qs)
1701 );
1702 assign reg2hw.digest[6].qe = digest_6_qe;
1703
1704 // Subregister 7 of Multireg digest
1705 // R[digest_7]: V(True)
1706 logic digest_7_qe;
1707 logic [0:0] digest_7_flds_we;
1708 assign digest_7_qe = &digest_7_flds_we;
1709 prim_subreg_ext #(
1710 .DW (32)
1711 ) u_digest_7 (
1712 .re (digest_7_re),
1713 .we (digest_7_we),
1714 .wd (digest_7_wd),
1715 .d (hw2reg.digest[7].d),
1716 .qre (),
1717 .qe (digest_7_flds_we[0]),
1718 .q (reg2hw.digest[7].q),
1719 .ds (),
1720 .qs (digest_7_qs)
1721 );
1722 assign reg2hw.digest[7].qe = digest_7_qe;
1723
1724 // Subregister 8 of Multireg digest
1725 // R[digest_8]: V(True)
1726 logic digest_8_qe;
1727 logic [0:0] digest_8_flds_we;
1728 assign digest_8_qe = &digest_8_flds_we;

```

```

1739 prim_subreg_ext #(
1740     .DW (32)
1741 ) u_digest_8 (
1742     .re (digest_8_re),
1743     .we (digest_8_we),
1744     .wd (digest_8_wd),
1745     .d (hw2reg.digest[8].d),
1746     .qre (),
1747     .qe (digest_8_flds_we[0]),
1748     .q (reg2hw.digest[8].q),
1749     .ds (),
1750     .qs (digest_8_qs)
1751 );
1752 assign reg2hw.digest[8].qe = digest_8_qe;
1753
1754 // Subregister 9 of Multireg digest
1755 // R[digest_9]: V(True)
1756 logic digest_9_qe;
1757 logic [0:0] digest_9_flds_we;
1758 assign digest_9_qe = &digest_9_flds_we;
1759 prim_subreg_ext #(
1760     .DW (32)
1761 ) u_digest_9 (
1762     .re (digest_9_re),
1763     .we (digest_9_we),
1764     .wd (digest_9_wd),
1765     .d (hw2reg.digest[9].d),
1766     .qre (),
1767     .qe (digest_9_flds_we[0]),
1768     .q (reg2hw.digest[9].q),
1769     .ds (),
1770     .qs (digest_9_qs)
1771 );
1772 assign reg2hw.digest[9].qe = digest_9_qe;
1773
1774 // Subregister 10 of Multireg digest
1775 // R[digest_10]: V(True)
1776 logic digest_10_qe;
1777 logic [0:0] digest_10_flds_we;
1778 assign digest_10_qe = &digest_10_flds_we;
1779 prim_subreg_ext #(
1780     .DW (32)
1781 ) u_digest_10 (
1782     .re (digest_10_re),
1783     .we (digest_10_we),
1784     .wd (digest_10_wd),
1785     .d (hw2reg.digest[10].d),
1786     .qre (),
1787     .qe (digest_10_flds_we[0]),
1788     .q (reg2hw.digest[10].q),
1789     .ds (),
1790     .qs (digest_10_qs)
1791 );
1792 assign reg2hw.digest[10].qe = digest_10_qe;
1793
1794 // Subregister 11 of Multireg digest
1795 // R[digest_11]: V(True)
1796 logic digest_11_qe;
1797 logic [0:0] digest_11_flds_we;
1798 assign digest_11_qe = &digest_11_flds_we;
1799 prim_subreg_ext #(
1800     .DW (32)
1801 ) u_digest_11 (
1802     .re (digest_11_re),
1803     .we (digest_11_we),
1804     .wd (digest_11_wd),
1805     .d (hw2reg.digest[11].d),
1806     .qre (),
1807     .qe (digest_11_flds_we[0]),
1808     .q (reg2hw.digest[11].q),
1809     .ds (),
1810     .qs (digest_11_qs)
1811 );
1812 assign reg2hw.digest[11].qe = digest_11_qe;
1813
1814 // Subregister 12 of Multireg digest
1815 // R[digest_12]: V(True)
1816 logic digest_12_qe;
1817 logic [0:0] digest_12_flds_we;
1818 assign digest_12_qe = &digest_12_flds_we;
1819 prim_subreg_ext #(
1820     .DW (32)
1821 ) u_digest_12 (
1822     .re (digest_12_re),
1823     .we (digest_12_we),
1824     .wd (digest_12_wd),
1825     .d (hw2reg.digest[12].d),
1826     .qre (),
1827     .qe (digest_12_flds_we[0]),
1828     .q (reg2hw.digest[12].q),
1829     .ds (),
1830     .qs (digest_12_qs)
1831 );
1832 assign reg2hw.digest[12].qe = digest_12_qe;
1833
1834 // Subregister 13 of Multireg digest
1835
1836 // R[digest_13]: V(True)
1837 logic digest_13_qe;
1838 logic [0:0] digest_13_flds_we;
1839 assign digest_13_qe = &digest_13_flds_we;
1840 prim_subreg_ext #(
1841     .DW (32)
1842 ) u_digest_13 (
1843     .re (digest_13_re),
1844     .we (digest_13_we),
1845     .wd (digest_13_wd),
1846     .d (hw2reg.digest[13].d),
1847     .qre (),
1848     .qe (digest_13_flds_we[0]),
1849     .q (reg2hw.digest[13].q),
1850     .ds (),
1851     .qs (digest_13_qs)
1852 );
1853 assign reg2hw.digest[13].qe = digest_13_qe;
1854
1855 // Subregister 14 of Multireg digest
1856 // R[digest_14]: V(True)
1857 logic digest_14_qe;
1858 logic [0:0] digest_14_flds_we;
1859 assign digest_14_qe = &digest_14_flds_we;
1860 prim_subreg_ext #(
1861     .DW (32)
1862 ) u_digest_14 (
1863     .re (digest_14_re),
1864     .we (digest_14_we),
1865     .wd (digest_14_wd),
1866     .d (hw2reg.digest[14].d),
1867     .qre (),
1868     .qe (digest_14_flds_we[0]),
1869     .q (reg2hw.digest[14].q),
1870     .ds (),
1871     .qs (digest_14_qs)
1872 );
1873 assign reg2hw.digest[14].qe = digest_14_qe;
1874
1875 // Subregister 15 of Multireg digest
1876 // R[digest_15]: V(True)
1877 logic digest_15_qe;
1878 logic [0:0] digest_15_flds_we;
1879 assign digest_15_qe = &digest_15_flds_we;
1880 prim_subreg_ext #(
1881     .DW (32)
1882 ) u_digest_15 (
1883     .re (digest_15_re),
1884     .we (digest_15_we),
1885     .wd (digest_15_wd),
1886     .d (hw2reg.digest[15].d),
1887     .qre (),
1888     .qe (digest_15_flds_we[0]),
1889     .q (reg2hw.digest[15].q),
1890     .ds (),
1891     .qs (digest_15_qs)
1892 );
1893 assign reg2hw.digest[15].qe = digest_15_qe;
1894
1895 // R[msg_length_lower]: V(True)
1896 logic msg_length_lower_qe;
1897 logic [0:0] msg_length_lower_flds_we;
1898 assign msg_length_lower_qe = &msg_length_lower_flds_we;
1899 prim_subreg_ext #(
1900     .DW (32)
1901 ) u_msg_length_lower (
1902     .re (msg_length_lower_re),
1903     .we (msg_length_lower_we),
1904     .wd (msg_length_lower_wd),
1905     .d (hw2reg.msg_length_lower.d),
1906     .qre (),
1907     .qe (msg_length_lower_flds_we[0]),
1908     .q (reg2hw.msg_length_lower.q),
1909     .ds (),
1910     .qs (msg_length_lower_qs)
1911 );
1912 assign reg2hw.msg_length_lower.qe = msg_length_lower_qe;
1913
1914 // R[msg_length_upper]: V(True)
1915 logic msg_length_upper_qe;
1916 logic [0:0] msg_length_upper_flds_we;
1917 assign msg_length_upper_qe = &msg_length_upper_flds_we;
1918 prim_subreg_ext #(
1919     .DW (32)
1920 ) u_msg_length_upper (
1921     .re (msg_length_upper_re),
1922     .we (msg_length_upper_we),
1923     .wd (msg_length_upper_wd),
1924     .d (hw2reg.msg_length_upper.d),
1925     .qre (),
1926     .qe (msg_length_upper_flds_we[0]),
1927     .q (reg2hw.msg_length_upper.q),
1928     .ds (),
1929     .qs (msg_length_upper_qs)
1930 );
1931 assign reg2hw.msg_length_upper.qe = msg_length_upper_qe;
1932

```

```

1941
1942
1943 logic [58:0] addr_hit;
1944 always_comb begin
1945   addr_hit[ 0] = (reg_addr == HMAC_INTR_STATE_OFFSET);
1946   addr_hit[ 1] = (reg_addr == HMAC_INTR_ENABLE_OFFSET);
1947   addr_hit[ 2] = (reg_addr == HMAC_INTR_TEST_OFFSET);
1948   addr_hit[ 3] = (reg_addr == HMAC_ALERT_TEST_OFFSET);
1949   addr_hit[ 4] = (reg_addr == HMAC_CFG_OFFSET);
1950   addr_hit[ 5] = (reg_addr == HMAC_CMD_OFFSET);
1951   addr_hit[ 6] = (reg_addr == HMAC_STATUS_OFFSET);
1952   addr_hit[ 7] = (reg_addr == HMAC_ERR_CODE_OFFSET);
1953   addr_hit[ 8] = (reg_addr == HMAC_WIPE_SECRET_OFFSET);
1954   addr_hit[ 9] = (reg_addr == HMAC_KEY_0_OFFSET);
1955   addr_hit[10] = (reg_addr == HMAC_KEY_1_OFFSET);
1956   addr_hit[11] = (reg_addr == HMAC_KEY_2_OFFSET);
1957   addr_hit[12] = (reg_addr == HMAC_KEY_3_OFFSET);
1958   addr_hit[13] = (reg_addr == HMAC_KEY_4_OFFSET);
1959   addr_hit[14] = (reg_addr == HMAC_KEY_5_OFFSET);
1960   addr_hit[15] = (reg_addr == HMAC_KEY_6_OFFSET);
1961   addr_hit[16] = (reg_addr == HMAC_KEY_7_OFFSET);
1962   addr_hit[17] = (reg_addr == HMAC_KEY_8_OFFSET);
1963   addr_hit[18] = (reg_addr == HMAC_KEY_9_OFFSET);
1964   addr_hit[19] = (reg_addr == HMAC_KEY_10_OFFSET);
1965   addr_hit[20] = (reg_addr == HMAC_KEY_11_OFFSET);
1966   addr_hit[21] = (reg_addr == HMAC_KEY_12_OFFSET);
1967   addr_hit[22] = (reg_addr == HMAC_KEY_13_OFFSET);
1968   addr_hit[23] = (reg_addr == HMAC_KEY_14_OFFSET);
1969   addr_hit[24] = (reg_addr == HMAC_KEY_15_OFFSET);
1970   addr_hit[25] = (reg_addr == HMAC_KEY_16_OFFSET);
1971   addr_hit[26] = (reg_addr == HMAC_KEY_17_OFFSET);
1972   addr_hit[27] = (reg_addr == HMAC_KEY_18_OFFSET);
1973   addr_hit[28] = (reg_addr == HMAC_KEY_19_OFFSET);
1974   addr_hit[29] = (reg_addr == HMAC_KEY_20_OFFSET);
1975   addr_hit[30] = (reg_addr == HMAC_KEY_21_OFFSET);
1976   addr_hit[31] = (reg_addr == HMAC_KEY_22_OFFSET);
1977   addr_hit[32] = (reg_addr == HMAC_KEY_23_OFFSET);
1978   addr_hit[33] = (reg_addr == HMAC_KEY_24_OFFSET);
1979   addr_hit[34] = (reg_addr == HMAC_KEY_25_OFFSET);
1980   addr_hit[35] = (reg_addr == HMAC_KEY_26_OFFSET);
1981   addr_hit[36] = (reg_addr == HMAC_KEY_27_OFFSET);
1982   addr_hit[37] = (reg_addr == HMAC_KEY_28_OFFSET);
1983   addr_hit[38] = (reg_addr == HMAC_KEY_29_OFFSET);
1984   addr_hit[39] = (reg_addr == HMAC_KEY_30_OFFSET);
1985   addr_hit[40] = (reg_addr == HMAC_KEY_31_OFFSET);
1986   addr_hit[41] = (reg_addr == HMAC_DIGEST_0_OFFSET);
1987   addr_hit[42] = (reg_addr == HMAC_DIGEST_1_OFFSET);
1988   addr_hit[43] = (reg_addr == HMAC_DIGEST_2_OFFSET);
1989   addr_hit[44] = (reg_addr == HMAC_DIGEST_3_OFFSET);
1990   addr_hit[45] = (reg_addr == HMAC_DIGEST_4_OFFSET);
1991   addr_hit[46] = (reg_addr == HMAC_DIGEST_5_OFFSET);
1992   addr_hit[47] = (reg_addr == HMAC_DIGEST_6_OFFSET);
1993   addr_hit[48] = (reg_addr == HMAC_DIGEST_7_OFFSET);
1994   addr_hit[49] = (reg_addr == HMAC_DIGEST_8_OFFSET);
1995   addr_hit[50] = (reg_addr == HMAC_DIGEST_9_OFFSET);
1996   addr_hit[51] = (reg_addr == HMAC_DIGEST_10_OFFSET);
1997   addr_hit[52] = (reg_addr == HMAC_DIGEST_11_OFFSET);
1998   addr_hit[53] = (reg_addr == HMAC_DIGEST_12_OFFSET);
1999   addr_hit[54] = (reg_addr == HMAC_DIGEST_13_OFFSET);
2000   addr_hit[55] = (reg_addr == HMAC_DIGEST_14_OFFSET);
2001   addr_hit[56] = (reg_addr == HMAC_DIGEST_15_OFFSET);
2002   addr_hit[57] = (reg_addr == HMAC_MSG_LENGTH_LOWER_OFFSET);
2003   addr_hit[58] = (reg_addr == HMAC_MSG_LENGTH_UPPER_OFFSET);
2004 end
2005
2006 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
2007
2008 // Check sub-word write is permitted
2009 always_comb begin
2010   wr_err = (reg_we &
2011
2012   ((addr_hit[ 0] & |(HMAC_PERMIT[ 0] & ~reg_be)))
2013   |
2014   (addr_hit[ 1] & |(HMAC_PERMIT[ 1] & ~reg_be)))
2015   |
2016   (addr_hit[ 2] & |(HMAC_PERMIT[ 2] & ~reg_be)))
2017   |
2018   (addr_hit[ 3] & |(HMAC_PERMIT[ 3] & ~reg_be)))
2019   |
2020   (addr_hit[ 4] & |(HMAC_PERMIT[ 4] & ~reg_be)))
2021   |
2022   (addr_hit[ 5] & |(HMAC_PERMIT[ 5] & ~reg_be)))
2023   |
2024   (addr_hit[ 6] & |(HMAC_PERMIT[ 6] & ~reg_be)))
2025   |
2026   (addr_hit[ 7] & |(HMAC_PERMIT[ 7] & ~reg_be)))
2027   |
2028   (addr_hit[ 8] & |(HMAC_PERMIT[ 8] & ~reg_be)))
2029   |
2030   (addr_hit[ 9] & |(HMAC_PERMIT[ 9] & ~reg_be)))
2031   |
2032   (addr_hit[10] & |(HMAC_PERMIT[10] & ~reg_be)))
2033   |
2034   (addr_hit[11] & |(HMAC_PERMIT[11] & ~reg_be)))
2035   |
2036   (addr_hit[12] & |(HMAC_PERMIT[12] & ~reg_be)))
2037   |
2038   (addr_hit[13] & |(HMAC_PERMIT[13] & ~reg_be)))
2039   |
2040   (addr_hit[14] & |(HMAC_PERMIT[14] & ~reg_be)))
2041   |
2042   (addr_hit[15] & |(HMAC_PERMIT[15] & ~reg_be)))
2043   |
2044   (addr_hit[16] & |(HMAC_PERMIT[16] & ~reg_be)))
2045   |
2046   (addr_hit[17] & |(HMAC_PERMIT[17] & ~reg_be)))
2047   |
2048   (addr_hit[18] & |(HMAC_PERMIT[18] & ~reg_be)))
2049   |
2050   (addr_hit[19] & |(HMAC_PERMIT[19] & ~reg_be)))
2051   |
2052   (addr_hit[20] & |(HMAC_PERMIT[20] & ~reg_be)))
2053   |
2054   (addr_hit[21] & |(HMAC_PERMIT[21] & ~reg_be)))
2055   |
2056   (addr_hit[22] & |(HMAC_PERMIT[22] & ~reg_be)))
2057   |
2058   (addr_hit[23] & |(HMAC_PERMIT[23] & ~reg_be)))
2059   |
2060   (addr_hit[24] & |(HMAC_PERMIT[24] & ~reg_be)))
2061   |
2062   (addr_hit[25] & |(HMAC_PERMIT[25] & ~reg_be)))
2063   |
2064   (addr_hit[26] & |(HMAC_PERMIT[26] & ~reg_be)))
2065   |
2066   (addr_hit[27] & |(HMAC_PERMIT[27] & ~reg_be)))
2067   |
2068   (addr_hit[28] & |(HMAC_PERMIT[28] & ~reg_be)))
2069   |
2070   (addr_hit[29] & |(HMAC_PERMIT[29] & ~reg_be)))
2071   |
2072   (addr_hit[30] & |(HMAC_PERMIT[30] & ~reg_be)))
2073   |
2074   (addr_hit[31] & |(HMAC_PERMIT[31] & ~reg_be)))
2075   |
2076   (addr_hit[32] & |(HMAC_PERMIT[32] & ~reg_be)))
2077   |
2078   (addr_hit[33] & |(HMAC_PERMIT[33] & ~reg_be)))
2079   |
2080   (addr_hit[34] & |(HMAC_PERMIT[34] & ~reg_be)))
2081   |
2082   (addr_hit[35] & |(HMAC_PERMIT[35] & ~reg_be)))
2083   |
2084   (addr_hit[36] & |(HMAC_PERMIT[36] & ~reg_be)))
2085   |
2086   (addr_hit[37] & |(HMAC_PERMIT[37] & ~reg_be)))
2087   |
2088   (addr_hit[38] & |(HMAC_PERMIT[38] & ~reg_be)))
2089   |
2090   (addr_hit[39] & |(HMAC_PERMIT[39] & ~reg_be)))
2091   |
2092   (addr_hit[40] & |(HMAC_PERMIT[40] & ~reg_be)))
2093   |
2094   (addr_hit[41] & |(HMAC_PERMIT[41] & ~reg_be)))
2095   |
2096   (addr_hit[42] & |(HMAC_PERMIT[42] & ~reg_be)))
2097   |
2098   (addr_hit[43] & |(HMAC_PERMIT[43] & ~reg_be)))

```

```

2055      (addr_hit[44] & !(HMAC_PERMIT[44] & ~reg_be)))
2056      |
2057      (addr_hit[45] & !(HMAC_PERMIT[45] & ~reg_be)))
2058      |
2059      (addr_hit[46] & !(HMAC_PERMIT[46] & ~reg_be)))
2060      |
2061      (addr_hit[47] & !(HMAC_PERMIT[47] & ~reg_be)))
2062      |
2063      (addr_hit[48] & !(HMAC_PERMIT[48] & ~reg_be)))
2064      |
2065      (addr_hit[49] & !(HMAC_PERMIT[49] & ~reg_be)))
2066      |
2067      (addr_hit[50] & !(HMAC_PERMIT[50] & ~reg_be)))
2068      |
2069      (addr_hit[51] & !(HMAC_PERMIT[51] & ~reg_be)))
2070      |
2071      (addr_hit[52] & !(HMAC_PERMIT[52] & ~reg_be)))
2072      |
2073      (addr_hit[53] & !(HMAC_PERMIT[53] & ~reg_be)))
2074      |
2075      (addr_hit[54] & !(HMAC_PERMIT[54] & ~reg_be)))
2076      |
2077      (addr_hit[55] & !(HMAC_PERMIT[55] & ~reg_be)))
2078      |
2079      (addr_hit[56] & !(HMAC_PERMIT[56] & ~reg_be)))
2080      |
2081      (addr_hit[57] & !(HMAC_PERMIT[57] & ~reg_be)))
2082      |
2083      (addr_hit[58] & !(HMAC_PERMIT[58] &
2084      ~reg_be)))));
2085      end
2086      // Generate write-enables
2087      assign intr_state_we = addr_hit[0] & reg_we & !reg_error;
2088      assign intr_state_hmac_done_wd = reg_wdata[0];
2089      assign intr_state_hmac_err_wd = reg_wdata[2];
2090      assign intr_enable_we = addr_hit[1] & reg_we & !reg_error;
2091      assign intr_enable_hmac_done_wd = reg_wdata[0];
2092      assign intr_enable_fifo_empty_wd = reg_wdata[1];
2093      assign intr_enable_hmac_err_wd = reg_wdata[2];
2094      assign intr_test_we = addr_hit[2] & reg_we & !reg_error;
2095      assign intr_test_hmac_done_wd = reg_wdata[0];
2096      assign intr_test_fifo_empty_wd = reg_wdata[1];
2097      assign intr_test_hmac_err_wd = reg_wdata[2];
2098      assign alert_test_we = addr_hit[3] & reg_we & !reg_error;
2099      assign alert_test_wd = reg_wdata[0];
2100      assign cfg_re = addr_hit[4] & reg_re & !reg_error;
2101      assign cfg_we = addr_hit[4] & reg_we & !reg_error;
2102      assign cfg_hmac_en_wd = reg_wdata[0];
2103      assign cfg_sha_en_wd = reg_wdata[1];
2104      assign cfg_endian_swap_wd = reg_wdata[2];
2105      assign cfg_digest_swap_wd = reg_wdata[3];
2106      assign cfg_key_swap_wd = reg_wdata[4];
2107      assign cfg_digest_size_wd = reg_wdata[8:5];
2108      assign cfg_key_length_wd = reg_wdata[14:9];
2109      assign cmd_we = addr_hit[5] & reg_we & !reg_error;
2110      assign cmd_hash_start_wd = reg_wdata[0];
2111      assign cmd_hash_process_wd = reg_wdata[1];
2112      assign cmd_hash_stop_wd = reg_wdata[2];
2113      assign cmd_hash_continue_wd = reg_wdata[3];
2114      assign status_re = addr_hit[6] & reg_re & !reg_error;
2115      assign wipe_secret_we = addr_hit[8] & reg_we & !reg_error;
2116      assign wipe_secret_wd = reg_wdata[31:0];
2117      assign key_0_we = addr_hit[9] & reg_we & !reg_error;
2118      assign key_0_wd = reg_wdata[31:0];

```

```

2119      assign key_1_we = addr_hit[10] & reg_we & !reg_error;
2120      assign key_1_wd = reg_wdata[31:0];
2121      assign key_2_we = addr_hit[11] & reg_we & !reg_error;
2122      assign key_2_wd = reg_wdata[31:0];
2123      assign key_3_we = addr_hit[12] & reg_we & !reg_error;
2124      assign key_3_wd = reg_wdata[31:0];
2125      assign key_4_we = addr_hit[13] & reg_we & !reg_error;
2126      assign key_4_wd = reg_wdata[31:0];
2127      assign key_5_we = addr_hit[14] & reg_we & !reg_error;
2128      assign key_5_wd = reg_wdata[31:0];
2129      assign key_6_we = addr_hit[15] & reg_we & !reg_error;
2130      assign key_6_wd = reg_wdata[31:0];
2131      assign key_7_we = addr_hit[16] & reg_we & !reg_error;
2132      assign key_7_wd = reg_wdata[31:0];
2133      assign key_8_we = addr_hit[17] & reg_we & !reg_error;
2134      assign key_8_wd = reg_wdata[31:0];
2135      assign key_9_we = addr_hit[18] & reg_we & !reg_error;
2136      assign key_9_wd = reg_wdata[31:0];
2137      assign key_10_we = addr_hit[19] & reg_we & !reg_error;
2138      assign key_10_wd = reg_wdata[31:0];
2139      assign key_11_we = addr_hit[20] & reg_we & !reg_error;
2140      assign key_11_wd = reg_wdata[31:0];
2141      assign key_12_we = addr_hit[21] & reg_we & !reg_error;
2142      assign key_12_wd = reg_wdata[31:0];
2143      assign key_13_we = addr_hit[22] & reg_we & !reg_error;
2144      assign key_13_wd = reg_wdata[31:0];
2145      assign key_14_we = addr_hit[23] & reg_we & !reg_error;
2146      assign key_14_wd = reg_wdata[31:0];
2147      assign key_15_we = addr_hit[24] & reg_we & !reg_error;
2148      assign key_15_wd = reg_wdata[31:0];
2149      assign key_16_we = addr_hit[25] & reg_we & !reg_error;
2150      assign key_16_wd = reg_wdata[31:0];
2151      assign key_17_we = addr_hit[26] & reg_we & !reg_error;
2152      assign key_17_wd = reg_wdata[31:0];
2153      assign key_18_we = addr_hit[27] & reg_we & !reg_error;
2154      assign key_18_wd = reg_wdata[31:0];
2155      assign key_19_we = addr_hit[28] & reg_we & !reg_error;
2156      assign key_19_wd = reg_wdata[31:0];
2157      assign key_20_we = addr_hit[29] & reg_we & !reg_error;
2158      assign key_20_wd = reg_wdata[31:0];
2159      assign key_21_we = addr_hit[30] & reg_we & !reg_error;
2160      assign key_21_wd = reg_wdata[31:0];
2161      assign key_22_we = addr_hit[31] & reg_we & !reg_error;
2162      assign key_22_wd = reg_wdata[31:0];
2163      assign key_23_we = addr_hit[32] & reg_we & !reg_error;
2164      assign key_23_wd = reg_wdata[31:0];
2165      assign key_24_we = addr_hit[33] & reg_we & !reg_error;
2166      assign key_24_wd = reg_wdata[31:0];
2167      assign key_25_we = addr_hit[34] & reg_we & !reg_error;
2168      assign key_25_wd = reg_wdata[31:0];
2169      assign key_26_we = addr_hit[35] & reg_we & !reg_error;
2170      assign key_26_wd = reg_wdata[31:0];
2171      assign key_27_we = addr_hit[36] & reg_we & !reg_error;
2172      assign key_27_wd = reg_wdata[31:0];
2173      assign key_28_we = addr_hit[37] & reg_we & !reg_error;
2174      assign key_28_wd = reg_wdata[31:0];
2175      assign key_29_we = addr_hit[38] & reg_we & !reg_error;
2176      assign key_29_wd = reg_wdata[31:0];
2177      assign key_30_we = addr_hit[39] & reg_we & !reg_error;
2178      assign key_30_wd = reg_wdata[31:0];
2179      assign key_31_we = addr_hit[40] & reg_we & !reg_error;
2180      assign key_31_wd = reg_wdata[31:0];
2181      assign digest_0_re = addr_hit[41] & reg_re & !reg_error;
2182      assign digest_0_we = addr_hit[41] & reg_we & !reg_error;
2183      assign digest_0_wd = reg_wdata[31:0];
2184      assign digest_1_re = addr_hit[42] & reg_re & !reg_error;
2185      assign digest_1_we = addr_hit[42] & reg_we & !reg_error;
2186      assign digest_1_wd = reg_wdata[31:0];

```



```

2228 assign digest_2_re = addr_hit[43] & reg_re & !reg_error;
2229 assign digest_2_we = addr_hit[43] & reg_we & !reg_error;
2230
2231 assign digest_2_wd = reg_wdata[31:0];
2232 assign digest_3_re = addr_hit[44] & reg_re & !reg_error;
2233 assign digest_3_we = addr_hit[44] & reg_we & !reg_error;
2234
2235 assign digest_3_wd = reg_wdata[31:0];
2236 assign digest_4_re = addr_hit[45] & reg_re & !reg_error;
2237 assign digest_4_we = addr_hit[45] & reg_we & !reg_error;
2238
2239 assign digest_4_wd = reg_wdata[31:0];
2240 assign digest_5_re = addr_hit[46] & reg_re & !reg_error;
2241 assign digest_5_we = addr_hit[46] & reg_we & !reg_error;
2242
2243 assign digest_5_wd = reg_wdata[31:0];
2244 assign digest_6_re = addr_hit[47] & reg_re & !reg_error;
2245 assign digest_6_we = addr_hit[47] & reg_we & !reg_error;
2246
2247 assign digest_6_wd = reg_wdata[31:0];
2248 assign digest_7_re = addr_hit[48] & reg_re & !reg_error;
2249 assign digest_7_we = addr_hit[48] & reg_we & !reg_error;
2250
2251 assign digest_7_wd = reg_wdata[31:0];
2252 assign digest_8_re = addr_hit[49] & reg_re & !reg_error;
2253 assign digest_8_we = addr_hit[49] & reg_we & !reg_error;
2254
2255 assign digest_8_wd = reg_wdata[31:0];
2256 assign digest_9_re = addr_hit[50] & reg_re & !reg_error;
2257 assign digest_9_we = addr_hit[50] & reg_we & !reg_error;
2258
2259 assign digest_9_wd = reg_wdata[31:0];
2260 assign digest_10_re = addr_hit[51] & reg_re & !reg_error;
2261 assign digest_10_we = addr_hit[51] & reg_we & !reg_error;
2262
2263 assign digest_10_wd = reg_wdata[31:0];
2264 assign digest_11_re = addr_hit[52] & reg_re & !reg_error;
2265 assign digest_11_we = addr_hit[52] & reg_we & !reg_error;
2266
2267 assign digest_11_wd = reg_wdata[31:0];
2268 assign digest_12_re = addr_hit[53] & reg_re & !reg_error;
2269 assign digest_12_we = addr_hit[53] & reg_we & !reg_error;
2270
2271 assign digest_12_wd = reg_wdata[31:0];
2272 assign digest_13_re = addr_hit[54] & reg_re & !reg_error;
2273 assign digest_13_we = addr_hit[54] & reg_we & !reg_error;
2274
2275 assign digest_13_wd = reg_wdata[31:0];
2276 assign digest_14_re = addr_hit[55] & reg_re & !reg_error;
2277 assign digest_14_we = addr_hit[55] & reg_we & !reg_error;
2278
2279 assign digest_14_wd = reg_wdata[31:0];
2280 assign digest_15_re = addr_hit[56] & reg_re & !reg_error;
2281 assign digest_15_we = addr_hit[56] & reg_we & !reg_error;
2282
2283 assign digest_15_wd = reg_wdata[31:0];
2284 assign msg_length_lower_re = addr_hit[57] & reg_re & !
reg_error;
2285 assign msg_length_lower_we = addr_hit[57] & reg_we & !
reg_error;
2286
2287 assign msg_length_lower_wd = reg_wdata[31:0];
2288 assign msg_length_upper_re = addr_hit[58] & reg_re & !
reg_error;
2289 assign msg_length_upper_we = addr_hit[58] & reg_we & !
reg_error;
2290
2291 assign msg_length_upper_wd = reg_wdata[31:0];
2292
2293 // Assign write-enables to checker logic vector.
2294 always_comb begin
2295   reg_we_check[0] = intr_state_we;
2296   reg_we_check[1] = intr_enable_we;
2297   reg_we_check[2] = intr_test_we;
2298   reg_we_check[3] = alert_test_we;
2299   reg_we_check[4] = cfg_we;
2300   reg_we_check[5] = cmd_we;
2301   reg_we_check[6] = 1'b0;
2302   reg_we_check[7] = 1'b0;
2303   reg_we_check[8] = wipe_secret_we;
2304   reg_we_check[9] = key_0_we;
2305   reg_we_check[10] = key_1_we;
2306   reg_we_check[11] = key_2_we;
2307   reg_we_check[12] = key_3_we;
2308   reg_we_check[13] = key_4_we;
2309   reg_we_check[14] = key_5_we;
2310   reg_we_check[15] = key_6_we;
2311   reg_we_check[16] = key_7_we;
2312   reg_we_check[17] = key_8_we;
2313   reg_we_check[18] = key_9_we;
2314   reg_we_check[19] = key_10_we;
2315   reg_we_check[20] = key_11_we;
2316   reg_we_check[21] = key_12_we;
2317   reg_we_check[22] = key_13_we;
2318   reg_we_check[23] = key_14_we;
2319   reg_we_check[24] = key_15_we;
2320   reg_we_check[25] = key_16_we;
2321   reg_we_check[26] = key_17_we;
2322   reg_we_check[27] = key_18_we;
2323   reg_we_check[28] = key_19_we;
2324   reg_we_check[29] = key_20_we;
2325   reg_we_check[30] = key_21_we;
2326   reg_we_check[31] = key_22_we;
2327   reg_we_check[32] = key_23_we;
2328   reg_we_check[33] = key_24_we;
2329   reg_we_check[34] = key_25_we;
2330   reg_we_check[35] = key_26_we;
2331   reg_we_check[36] = key_27_we;
2332   reg_we_check[37] = key_28_we;
2333   reg_we_check[38] = key_29_we;
2334   reg_we_check[39] = key_30_we;
2335   reg_we_check[40] = key_31_we;
2336   reg_we_check[41] = digest_0_we;
2337   reg_we_check[42] = digest_1_we;
2338   reg_we_check[43] = digest_2_we;
2339   reg_we_check[44] = digest_3_we;
2340   reg_we_check[45] = digest_4_we;
2341   reg_we_check[46] = digest_5_we;
2342   reg_we_check[47] = digest_6_we;
2343   reg_we_check[48] = digest_7_we;
2344   reg_we_check[49] = digest_8_we;
2345   reg_we_check[50] = digest_9_we;
2346   reg_we_check[51] = digest_10_we;
2347   reg_we_check[52] = digest_11_we;
2348   reg_we_check[53] = digest_12_we;
2349   reg_we_check[54] = digest_13_we;
2350   reg_we_check[55] = digest_14_we;
2351   reg_we_check[56] = digest_15_we;
2352   reg_we_check[57] = msg_length_lower_we;
2353   reg_we_check[58] = msg_length_upper_we;
2354 end
2355
2356 // Read data return
2357 always_comb begin
2358   reg_rdata_next = '0;
2359   unique case (1'b1)
2360     addr_hit[0]: begin
2361       reg_rdata_next[0] = intr_state_hmac_done_qs;
2362       reg_rdata_next[1] = intr_state_fifo_empty_qs;
2363       reg_rdata_next[2] = intr_state_hmac_err_qs;
2364     end
2365     addr_hit[1]: begin
2366       reg_rdata_next[0] = intr_enable_hmac_done_qs;
2367       reg_rdata_next[1] = intr_enable_fifo_empty_qs;
2368       reg_rdata_next[2] = intr_enable_hmac_err_qs;
2369     end
2370     addr_hit[2]: begin
2371       reg_rdata_next[0] = '0;
2372       reg_rdata_next[1] = '0;
2373       reg_rdata_next[2] = '0;
2374     end
2375     addr_hit[3]: begin
2376       reg_rdata_next[0] = '0;
2377     end
2378     addr_hit[4]: begin
2379       reg_rdata_next[0] = cfg_hmac_en_qs;
2380       reg_rdata_next[1] = cfg_sha_en_qs;
2381       reg_rdata_next[2] = cfg_endian_swap_qs;
2382       reg_rdata_next[3] = cfg_digest_swap_qs;
2383       reg_rdata_next[4] = cfg_key_swap_qs;
2384       reg_rdata_next[8:5] = cfg_digest_size_qs;
2385       reg_rdata_next[14:9] = cfg_key_length_qs;
2386     end
2387     addr_hit[5]: begin
2388       reg_rdata_next[0] = '0;
2389       reg_rdata_next[1] = '0;
2390       reg_rdata_next[2] = '0;
2391       reg_rdata_next[3] = '0;
2392     end
2393     addr_hit[6]: begin
2394       reg_rdata_next[0] = status_hmac_idle_qs;
2395       reg_rdata_next[1] = status_fifo_empty_qs;
2396       reg_rdata_next[2] = status_fifo_full_qs;
2397       reg_rdata_next[9:4] = status_fifo_depth_qs;
2398     end
2399     addr_hit[7]: begin
2400       reg_rdata_next[31:0] = err_code_qs;
2401     end
2402     addr_hit[8]: begin
2403       reg_rdata_next[31:0] = '0;
2404     end
2405     addr_hit[9]: begin
2406       reg_rdata_next[31:0] = '0;
2407     end
2408     addr_hit[10]: begin
2409       reg_rdata_next[31:0] = '0;
2410     end
2411     addr_hit[11]: begin
2412       reg_rdata_next[31:0] = '0;
2413     end
2414   end
2415 end

```

```

2426     addr_hit[12]: begin
2427         reg_rdata_next[31:0] = '0;
2428     end
2429
2430     addr_hit[13]: begin
2431         reg_rdata_next[31:0] = '0;
2432     end
2433
2434     addr_hit[14]: begin
2435         reg_rdata_next[31:0] = '0;
2436     end
2437
2438     addr_hit[15]: begin
2439         reg_rdata_next[31:0] = '0;
2440     end
2441
2442     addr_hit[16]: begin
2443         reg_rdata_next[31:0] = '0;
2444     end
2445
2446     addr_hit[17]: begin
2447         reg_rdata_next[31:0] = '0;
2448     end
2449
2450     addr_hit[18]: begin
2451         reg_rdata_next[31:0] = '0;
2452     end
2453
2454     addr_hit[19]: begin
2455         reg_rdata_next[31:0] = '0;
2456     end
2457
2458     addr_hit[20]: begin
2459         reg_rdata_next[31:0] = '0;
2460     end
2461
2462     addr_hit[21]: begin
2463         reg_rdata_next[31:0] = '0;
2464     end
2465
2466     addr_hit[22]: begin
2467         reg_rdata_next[31:0] = '0;
2468     end
2469
2470     addr_hit[23]: begin
2471         reg_rdata_next[31:0] = '0;
2472     end
2473
2474     addr_hit[24]: begin
2475         reg_rdata_next[31:0] = '0;
2476     end
2477
2478     addr_hit[25]: begin
2479         reg_rdata_next[31:0] = '0;
2480     end
2481
2482     addr_hit[26]: begin
2483         reg_rdata_next[31:0] = '0;
2484     end
2485
2486     addr_hit[27]: begin
2487         reg_rdata_next[31:0] = '0;
2488     end
2489
2490     addr_hit[28]: begin
2491         reg_rdata_next[31:0] = '0;
2492     end
2493
2494     addr_hit[29]: begin
2495         reg_rdata_next[31:0] = '0;
2496     end
2497
2498     addr_hit[30]: begin
2499         reg_rdata_next[31:0] = '0;
2500     end
2501
2502     addr_hit[31]: begin
2503         reg_rdata_next[31:0] = '0;
2504     end
2505
2506     addr_hit[32]: begin
2507         reg_rdata_next[31:0] = '0;
2508     end
2509
2510     addr_hit[33]: begin
2511         reg_rdata_next[31:0] = '0;
2512     end
2513
2514     addr_hit[34]: begin
2515         reg_rdata_next[31:0] = '0;
2516     end
2517
2518     addr_hit[35]: begin
2519         reg_rdata_next[31:0] = '0;
2520     end
2521
2522     addr_hit[36]: begin
2523         reg_rdata_next[31:0] = '0;
2524     end
2525
2526     addr_hit[37]: begin
2527         reg_rdata_next[31:0] = '0;
2528     end
2529
2530     addr_hit[38]: begin
2531         reg_rdata_next[31:0] = '0;
2532     end
2533
2534     addr_hit[39]: begin
2535         reg_rdata_next[31:0] = '0;
2536     end
2537
2538     addr_hit[40]: begin
2539         reg_rdata_next[31:0] = '0;
2540     end
2541
2542     addr_hit[41]: begin
2543         reg_rdata_next[31:0] = digest_0_qs;
2544     end
2545
2546     addr_hit[42]: begin
2547         reg_rdata_next[31:0] = digest_1_qs;
2548     end
2549
2550     addr_hit[43]: begin
2551         reg_rdata_next[31:0] = digest_2_qs;
2552     end
2553
2554     addr_hit[44]: begin
2555         reg_rdata_next[31:0] = digest_3_qs;
2556     end
2557
2558     addr_hit[45]: begin
2559         reg_rdata_next[31:0] = digest_4_qs;
2560     end
2561
2562     addr_hit[46]: begin
2563         reg_rdata_next[31:0] = digest_5_qs;
2564     end
2565
2566     addr_hit[47]: begin
2567         reg_rdata_next[31:0] = digest_6_qs;
2568     end
2569
2570     addr_hit[48]: begin
2571         reg_rdata_next[31:0] = digest_7_qs;
2572     end
2573
2574     addr_hit[49]: begin
2575         reg_rdata_next[31:0] = digest_8_qs;
2576     end
2577
2578     addr_hit[50]: begin
2579         reg_rdata_next[31:0] = digest_9_qs;
2580     end
2581
2582     addr_hit[51]: begin
2583         reg_rdata_next[31:0] = digest_10_qs;
2584     end
2585
2586     addr_hit[52]: begin
2587         reg_rdata_next[31:0] = digest_11_qs;
2588     end
2589
2590     addr_hit[53]: begin
2591         reg_rdata_next[31:0] = digest_12_qs;
2592     end
2593
2594     addr_hit[54]: begin
2595         reg_rdata_next[31:0] = digest_13_qs;
2596     end
2597
2598     addr_hit[55]: begin
2599         reg_rdata_next[31:0] = digest_14_qs;
2600     end
2601
2602     addr_hit[56]: begin
2603         reg_rdata_next[31:0] = digest_15_qs;
2604     end
2605
2606     addr_hit[57]: begin
2607         reg_rdata_next[31:0] = msg_length_lower_qs;
2608     end
2609
2610     addr_hit[58]: begin
2611         reg_rdata_next[31:0] = msg_length_upper_qs;
2612     end
2613
2614     default: begin
2615         reg_rdata_next = '1;
2616     end
2617 endcase
2618 end
2619
2620 // shadow busy
2621 logic shadow_busy;
2622 assign shadow_busy = 1'b0;
2623
2624 // register busy
2625 assign reg_busy = shadow_busy;
2626
2627 // Unused signal tieoff

```

```

2628
2629 // wdata / byte enable are not always fully used
2630 // add a blanket unused statement to handle lint waivers
2631 logic unused_wdata;
2632 logic unused_be;
2633 assign unused_wdata = ^reg_wdata;
2634 assign unused_be = ^reg_be;
2635
2636 // Assertions for Register Interface
2637 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
2638 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
2639
2640 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |=>

```

```

tl_o_pre.d_valid, clk_i, !rst_ni)
2641
2642 `ASSERT(en2addrHit, (reg_we || reg_re) |-> $one-
hot0(addr_hit), clk_i, !rst_ni)
2643
2644 // this is formulated as an assumption such that the FPV
testbenches do disprove this
2645 // property by mistake
2646 `ASSUME(reqParity, tl_reg_h2d.a_valid |->
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
2647
2648 endmodule

```


OpenTitan KMAC (Keccak/SHA-3)

What. SHA-3 계열(Keccak-f[1600]) 과 KMAC. 상태가 1600비트인 순열 기반 해시.

Why it exists. SHA-2 와 구조가 근본적으로 다르다. Merkle–Damgård(압축 함수 반복)가 아니라 스펀지(**sponge**) 구조다 — 큰 상태를 순열로 휘젓고 일부만 흡수/방출한다. 하드웨어에서는 1600비트 레지스터와 넓은 조합 논리가 된다.

학부 복습 · **EECS concepts.** 스펀지 구조 · 순열(permutation) 기반 암호 · 비트 슬라이스 구현 · 넓은 데이터패스의 배선 혼잡(routing congestion).

석사 수준 · **Practice note.** Keccak 은 배선이 면적을 지배 하는 대표적 블록이다. 합성 후 면적이 게이트 수 추정보다 크게 나오면 배선 때문이고, 그것은 RTL 을 고쳐도 잘 안 줄어든다. 초기 면적 추정에 배선 계수를 넣어라.

5 files • 4,651 lines

API — keccak_2share.sv

Table 219. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — module / interface / package (1개)

이름	종류
keccak_2share	module

Table 220. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — parameter (3개)

파라미터	형/폭	기본값
Width	int	1600
EnMasking	bit	1'b0
ForceRandExt	bit	1'b0

Table 221. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — localparam (10개)

상수	형/폭	값
W	int	Width/25
L	int	\$clog2(W)
MaxRound	int	12 + 2*L
RndW	int	\$clog2(MaxRound+1)
Share	int	EnMasking ? 2 : 1
WSheetHalf	int unsigned	\$bits(sheet_t)/2
X1	int	(x + 1) % 5
X2	int	(x + 2) % 5
Offset	int	RhoOffset[5*x+y]%W
ShiftAmt	int	W- Offset

Table 222. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
rnd_i	input	[RndW-1:0]
phase_sel_i	input	mubi4_t
dom_out_low_i	input	
dom_in_low_i	input	
dom_in_rand_ext_i	input	
dom_update_i	input	
rand_i	input	[Width/2-1:0]

Table 223. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv — typedef / using (7개)

새 이름	원래 형
box_t	logic [4:0][4:0][W-1:0]
lane_t	logic [W-1:0]
plane_t	logic [4:0] [W-1:0]
slice_t	logic [4:0][4:0]
sheet_t	logic [4:0][W-1:0]
row_t	logic [4:0]
col_t	logic [4:0]

API — keccak_round.sv

Table 224. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — module / interface / package (1개)

이름	종류
keccak_round	module

Table 225. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — parameter (4개)

파라미터	형/폭	기본값
Width	int	1600
DInWidth	int	64
EnMasking	bit	1'b0
ForceRandExt	bit	1'b0

Table 226. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — localparam (7개)

상수	형/폭	값
W	int	Width/25
L	int	\$clog2(W)

상수	형/폭	값
MaxRound	int	$12 + 2 \cdot L$
RndW	int	$\lceil \log_2(\text{MaxRound} + 1) \rceil$
DInEntry	int	Width / DInWidth
DInAddr	int	$\lceil \log_2(\text{DInEntry}) \rceil$
Share	int	EnMasking ? 2 : 1

Table 227. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
valid_i	input	
addr_i	input	[DInAddr-1:0]
ready_o	output	
run_i	input	
rand_valid_i	input	
rand_early_i	input	
rand_data_i	input	[Width/2-1:0]
rand_aux_i	input	
rand_update_o	output	logic
rand_consumed_o	output	logic
complete_o	output	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
sparse_fsm_error_o	output	logic
round_count_error_o	output	logic
rst_storage_error_o	output	logic
0	input	prim_mubi_pkg::mubi4_t clear_i // Clear internal s...

API — kmac.sv

Table 228. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — module / interface / package (1개)

이름	종류
kmac	module

Table 229. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — parameter (11개)

파라미터	형/폭	기본값
EnMasking	bit	1
SwKeyMasked	bit	0

파라미터	형/폭	기본값
SecCmdDelay	int	0
SecIdleAcceptSwMsg	bit	1'b0
NumAppIntf	int unsigned	4
RndCnstLfsrPerm	lfsr_perm_t	RndCnstLfsrPermDefault
RndCnstLfsrSeed	lfsr_seed_t	RndCnstLfsrSeedDefault
RndCnstBufferLfsrSeed	buffer_lfsr_seed_t	RndCnstBufferLfsrSeedDefault
RndCnstMsgPerm	msg_perm_t	RndCnstMsgPermDefault
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1

Table 230. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — localparam (5개)

상수	형/폭	값
Share	int	(EnMasking) ? 2 : 1
SwKeyShare	int	(EnMasking SwKeyMasked) ? 2 : 1
StateWidth	int	6
NumLcSyncCopies	int unsigned	6
WidthCounter	int unsigned	\$clog2(SecCmdDelay+1)

Table 231. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rst_shadowed_ni	input	
clk_edn_i	input	
rst_edn_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
app_i	input	app_req_t [NumAppIntf-1:0]
app_o	output	app_rsp_t [NumAppIntf-1:0]
entropy_o	output	edn_pkg::edn_req_t
entropy_i	input	edn_pkg::edn_rsp_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
intr_kmac_done_o	output	logic

포트	방향	형/폭
intr_fifo_empty_o	output	logic
intr_kmac_err_o	output	logic
en_masking_o	output	logic
idle_o	output	prim_mubi_pkg::mubi4_t

API — kmac_app.sv

Table 232. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — module / interface / package (1개)

이름	종류
kmac_app	module

Table 233. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — parameter (3개)

파라미터	형/폭	기본값
EnMasking	bit	1'b0
SecIdleAcceptSwMsg	bit	1'b0
NumAppIntf	int unsigned	4

Table 234. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — localparam (9개)

상수	형/폭	값
Share	int	(EnMasking) ? 2 : 1
KeyMgrKeyW	int	\$bits(keymgr_key_i.key[0])
SelKeySize	int	(AppKeyW == 128) ? 0 :
SelDigSize	int	(AppDigestW == 128) ? 0 :
SideloadedKey	key_len_e	KeyLengths[SelKeySize]
OutLenW	int	24
AppIdxW	int unsigned	\$clog2(NumAppIntf)
MaxNumDigestParts	int	(sha3_pkg::StateW - 2 * 128) / DynAppDigestW
DigestCntW	int	\$clog2(MaxNumDigestParts)

Table 235. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv — 포트 (43개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
reg_key_len_i	input	key_len_e
reg_prefix_i	input	[sha3_pkg::NSRegisterSize*8-1:0]
reg_kmac_en_i	input	logic
reg_sha3_mode_i	input	sha3_pkg::sha3_mode_e
reg_keccak_strength_i	input	sha3_pkg::keccak_strength_e

포트	방향	형/폭
sw_valid_i	input	
sw_data_i	input	[MsgWidth-1:0]
sw_strb_i	input	[MsgWidth-1:0]
sw_ready_o	output	logic
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
app_i	input	app_req_t [NumAppIntf-1:0]
app_o	output	app_rsp_t [NumAppIntf-1:0]
key_len_o	output	key_len_e
key_valid_o	output	logic
kmac_valid_o	output	logic
kmac_strb_o	output	logic [MsgWidth-1:0]
kmac_ready_i	input	logic
kmac_bypass_fifo_o	output	logic
kmac_en_o	output	logic
sha3_prefix_o	output	logic [sha3_pkg::NSRegisterSize*8-1:0]
sha3_mode_o	output	sha3_pkg::sha3_mode_e
keccak_strength_o	output	sha3_pkg::keccak_strength_e
keccak_state_valid_i	input	
reg_state_valid_o	output	logic
keymgr_key_en_i	input	logic
sw_cmd_i	input	kmac_cmd_e
absorbed_i	input	prim_mubi_pkg::mubi4_t
squeezing_i	input	logic
cmd_o	output	kmac_cmd_e
absorbed_o	output	prim_mubi_pkg::mubi4_t
app_active_o	output	logic
entropy_fast_process_i	input	logic
entropy_fast_process_o	output	logic
entropy_ready_i	input	prim_mubi_pkg::mubi4_t
error_i	input	
err_processed_i	input	
clear_after_error_o	output	prim_mubi_pkg::mubi4_t
error_o	output	kmac_pkg::err_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
sparse_fsm_error_o	output	logic

포트	방향	형/폭
counter_error_o	output	logic

API — kmac_core.sv

Table 236. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — module / interface / package (1개)

이름	종류
kmac_core	module

Table 237. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — parameter (1개)

파라미터	형/폭	기본값
EnMasking	bit	0

Table 238. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — localparam (2개)

상수	형/폭	값
Share	int	(EnMasking) ? 2 : 1 // derived parameter
StateWidth	int	6

Table 239. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
fifo_valid_i	input	
fifo_strb_i	input	[MsgStrbW-1:0]
fifo_ready_o	output	logic
msg_valid_o	output	logic
msg_strb_o	output	logic [MsgStrbW-1:0]
msg_ready_i	input	
kmac_en_i	input	
mode_i	input	sha3_pkg::sha3_mode_e
strength_i	input	sha3_pkg::keccak_strength_e
key_len_i	input	key_len_e
key_valid_i	input	logic
start_i	input	
process_i	input	
done_i	input	prim_mubi_pkg::mubi4_t
process_o	output	logic
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
sparse_fsm_error_o	output	logic

포트	방향	형/폭
key_index_error_o	output	logic

1. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_2share.sv (563 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module is the single round keccak permutation module
6 // It supports Keccak with up to 1600b of state
7
8 `include "prim_assert.sv"
9
10 module keccak_2share
11   import prim_mubi_pkg::*;
12   #(
13     parameter int Width = 1600, // b= {25, 50, 100, 200, 400,
14       800, 1600}
15     // Derived
16     localparam int W      = Width/25,
17     localparam int L      = $clog2(W),
18     localparam int MaxRound = 12 + 2*L, // Keccak-f
19   )
20   localparam int RndW      = $clog2(MaxRound+1), // Represent-
21     ing up to MaxRound
22   // Control parameters
23   parameter bit EnMasking  = 1'b0, // Enable secure harden-
24     ing
25   parameter bit ForceRandExt = 1'b0, // 1: Always forward ex-
26     ternally provided randomness.
27   // 0: Switch between ex-
28     ternal randomness and internal
29
30   // intermediate state
31   localparam int Share     = EnMasking ? 2 : 1
32   ) (
33     input clk_i,
34     input rst_ni,
35     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i, // Used to
36       disable SVAs when escalating.
37     input [RndW-1:0] rnd_i, // Current round index
38     // Control inputs used when EnMasking = 1.
39     input mubi4_t phase_sel_i, // Output mux control
40     input dom_out_low_i, // DOM multiplier output
41     mux
42     input dom_in_low_i, // DOM multiplier input mux
43     input dom_in_rand_ext_i, // DOM multiplier input
44     randomness mux
45     input dom_update_i, // DOM multiplier pipeline
46     reg write enable
47     input [Width/2-1:0] rand_i, // Randomness for remask-
48       ing.
49     // State input and output
50     input [Width-1:0] s_i [Share],
51     output logic [Width-1:0] s_o [Share]
52   );
53   // Types //
54   //
55   // x y z
56   typedef logic [4:0][4:0][W-1:0] box_t; // (x,y,z) state
57   typedef logic [W-1:0] lane_t; // (z)
58   typedef logic [4:0][W-1:0] plane_t; // (x,z)
59   typedef logic [4:0][4:0] slice_t; // (x,y)
60   typedef logic [4:0][W-1:0] sheet_t; // (y,z) identical
61   to plane_t
62   typedef logic [4:0] row_t; // (x)
63   typedef logic [4:0] col_t; // (y) identical to
64   row_t
65
66   // Keccak-f //
67   //
68   box_t state_in [Share];
69   box_t state_out [Share];
70   box_t theta_data [Share];
71   box_t rho_data [Share];
72   box_t pi_data [Share];
73   box_t chi_data [Share];
74   box_t iota_data [Share];
75   box_t phase1_in [Share];

```

```

76   box_t phase2_in [Share];
77   box_t phase2_out [Share];
78   //
79   // Unused nets //
80   //
81   // Tie off input signals that aren't used in the unmasked
  implementation.
82   if (!EnMasking) begin : gen_tie_unused
83     logic unused_clk;
84     logic unused_rst_n;
85     mubi4_t unused_phase_sel;
86     logic unused_dom_ctrl;
87     logic [Width/2-1:0] unused_rand;
88     assign unused_clk = clk_i;
89     assign unused_rst_n = rst_ni;
90     assign unused_phase_sel = phase_sel_i;
91     assign unused_dom_ctrl =
92       ^{dom_out_low_i, dom_in_low_i, dom_in_rand_ext_i,
93         dom_update_i};
94     assign unused_rand = rand_i;
95   end
96   // Input/output type conversion and interfacing //
97   //
98   for (genvar i = 0 ; i < Share ; i++) begin : g_state_inout
99     assign state_in[i] = bitarray_to_box(s_i[i]);
100    assign s_o[i] = box_to_bitarray(state_out[i]);
101  end : g_state_inout
102
103  if (EnMasking) begin : g_2share_data
104    assign phase1_in = state_in;
105    assign phase2_in = state_in;
106
107    always_comb begin
108      unique case (phase_sel_i)
109        Mubi4False: state_out = phase1_out;
110        Mubi4True: state_out = phase2_out;
111        default: state_out = phase1_out;
112      endcase
113    end
114  end else begin : g_single_data
115    assign phase1_in = state_in;
116    assign phase2_in = phase1_out;
117    assign state_out = phase2_out;
118  end
119
120  // Datapath //
121  //
122  for (genvar i = 0 ; i < Share ; i++) begin : g_datapath
123
124    // Phase 1:
125    assign theta_data[i] = theta(phase1_in[i]);
126    // Commented out rho function as vcs complains z-Offset%W
127    isn't constant
128    // assign rho_data[i] = rho(theta_data[i]);
129    assign pi_data[i] = pi(rho_data[i]);
130
131    // Phase 2 (Cycles 1, 2 and 3):
132    // Chi : See below
133    // Iota: See below
134    end : g_datapath
135
136    assign phase1_out = pi_data;
137
138    // Iota adds Round Constants(RC), so only one share should
  be XORed
139    if (EnMasking) begin : g_2share_iota
140      assign iota_data[0] = iota(chi_data[0], rnd_i);
141      assign iota_data[1] = chi_data[1];
142    end else begin : g_single_iota
143      assign iota_data[0] = iota(chi_data[0], rnd_i);
144    end
145
146    if (EnMasking) begin : g_2share_chi
147      // Domain-Oriented Masking
148      // reference: https://eprint.iacr.org/2017/395.pdf
149
150      localparam int unsigned WSheetHalf = $bits(sheet_t)/2;
151      logic [4:0][WSheetHalf-1:0] in_prd, out_prd;
152
153      //
154      // DOM multipliers //
155      //
156
157      for (genvar x = 0 ; x < 5 ; x++) begin : g_chi_w
158        localparam int X1 = (x + 1) % 5;
159        localparam int X2 = (x + 2) % 5;
160
161        sheet_t sheet0[Share]; // Inverted input X1

```



```

162 sheet_t sheet1[Share]; // X2
163 sheet_t sheet2[Share]; // DOM output
164
165 assign sheet0[0] = ~phase2_in[0][X1];
166 assign sheet0[1] = phase2_in[1][X1];
167
168 assign sheet1[0] = phase2_in[0][X2];
169 assign sheet1[1] = phase2_in[1][X2];
170
171 // Convert sheet_t to 1D arrays, one for the upper and
lower half lane.
172 logic [WSheetHalf-1:0] a0_l, a1_l, b0_l, b1_l;
173 logic [WSheetHalf-1:0] a0_h, a1_h, b0_h, b1_h;
174 logic [WSheetHalf-1:0] a0, a1, b0, b1, q0, q1;
175
176 assign a0_l = {sheet0[0][0][W/2-1:0],
177               sheet0[0][1][W/2-1:0],
178               sheet0[0][2][W/2-1:0],
179               sheet0[0][3][W/2-1:0],
180               sheet0[0][4][W/2-1:0]};
181 assign a1_l = {sheet0[1][0][W/2-1:0],
182               sheet0[1][1][W/2-1:0],
183               sheet0[1][2][W/2-1:0],
184               sheet0[1][3][W/2-1:0],
185               sheet0[1][4][W/2-1:0]};
186
187 assign a0_h = {sheet0[0][0][W-1:W/2],
188               sheet0[0][1][W-1:W/2],
189               sheet0[0][2][W-1:W/2],
190               sheet0[0][3][W-1:W/2],
191               sheet0[0][4][W-1:W/2]};
192 assign a1_h = {sheet0[1][0][W-1:W/2],
193               sheet0[1][1][W-1:W/2],
194               sheet0[1][2][W-1:W/2],
195               sheet0[1][3][W-1:W/2],
196               sheet0[1][4][W-1:W/2]};
197
198 assign b0_l = {sheet1[0][0][W/2-1:0],
199               sheet1[0][1][W/2-1:0],
200               sheet1[0][2][W/2-1:0],
201               sheet1[0][3][W/2-1:0],
202               sheet1[0][4][W/2-1:0]};
203 assign b1_l = {sheet1[1][0][W/2-1:0],
204               sheet1[1][1][W/2-1:0],
205               sheet1[1][2][W/2-1:0],
206               sheet1[1][3][W/2-1:0],
207               sheet1[1][4][W/2-1:0]};
208
209 assign b0_h = {sheet1[0][0][W-1:W/2],
210               sheet1[0][1][W-1:W/2],
211               sheet1[0][2][W-1:W/2],
212               sheet1[0][3][W-1:W/2],
213               sheet1[0][4][W-1:W/2]};
214 assign b1_h = {sheet1[1][0][W-1:W/2],
215               sheet1[1][1][W-1:W/2],
216               sheet1[1][2][W-1:W/2],
217               sheet1[1][3][W-1:W/2],
218               sheet1[1][4][W-1:W/2]};
219
220 // Input muxing
221 assign a0 = dom_in_low_i ? a0_l : a0_h;
222 assign a1 = dom_in_low_i ? a1_l : a1_h;
223 assign b0 = dom_in_low_i ? b0_l : b0_h;
224 assign b1 = dom_in_low_i ? b1_l : b1_h;
225
226 // Randomness muxing
227 if (!ForceRandExt) begin : gen_in_prd_mux
228   // Intermediate results are rotated across rows. The
new Row x depends on
229   // data from Rows x + 1 and x + 2. Hence we don't want
to use intermediate
230   // results from Rows x, x + 1, and x + 2 for remask-
ing.
231   assign in_prd[x] = dom_in_rand_ext_i ? rand_i[x *
WSheetHalf +: WSheetHalf] :
232
233   out_prd[rot_int(x, 5)];
234   end else begin : gen_no_in_prd_mux
235     // Always use the externally provided randomness.
236     assign in_prd[x] = rand_i[x * WSheetHalf +: WSheetH-
alf];
237     // Tie off unused signals.
238     logic unused_out_prd;
239     assign unused_out_prd = ^{dom_in_rand_ext_i,
out_prd[rot_int(x, 5)]};
240   end
241
242   prim_dom_and_2share #(
243     .DW (WSheetHalf), // a half sheet
244     .Pipeline(1) // Process the full sheet in 3 clock
cycles. This reduces
245     // SCA leakage.
246   ) u_dom (
247     .clk_i,
248     .rst_ni,
249     .a0_i (a0),
250     .a1_i (a1),
251     .b0_i (b0),
252     .b1_i (b1),

```

```

253     .z_valid_i (dom_update_i),
254     .z_i (in_prd[x]),
255     .q0_o (q0),
256     .q1_o (q1),
257     .prd_o (out_prd[x])
258   );
259
260   // Output conversion from q0, q1 to sheet_t
261
262   // For simplicity, we forward the generated lane half to
both the upper
263   // and lower lane halves at this point. The actual out-
put muxing/selection
264   // happens after the Iota step when generating
phase2_out from iota_data
265   // and state_in below.
266   assign sheet2[0][4] = {2{q0[W/2*0+:W/2]}};
267   assign sheet2[0][3] = {2{q0[W/2*1+:W/2]}};
268   assign sheet2[0][2] = {2{q0[W/2*2+:W/2]}};
269   assign sheet2[0][1] = {2{q0[W/2*3+:W/2]}};
270   assign sheet2[0][0] = {2{q0[W/2*4+:W/2]}};
271
272   assign sheet2[1][4] = {2{q1[W/2*0+:W/2]}};
273   assign sheet2[1][3] = {2{q1[W/2*1+:W/2]}};
274   assign sheet2[1][2] = {2{q1[W/2*2+:W/2]}};
275   assign sheet2[1][1] = {2{q1[W/2*3+:W/2]}};
276   assign sheet2[1][0] = {2{q1[W/2*4+:W/2]}};
277
278   // Final XOR to generate the output
279   assign chi_data[0][x] = sheet2[0] ^ phase2_in[0][x];
280   assign chi_data[1][x] = sheet2[1] ^ phase2_in[1][x];
281   end : g_chi_w
282
283   // Since Chi and thus Iota are separately applied to the
lower and upper half
284   // lanes, we need to forward the input to the other half.
for (genvar x = 0 ; x < 5 ; x++) begin :
g_2share_phase2_out_row
285     for (genvar y = 0 ; y < 5 ; y++) begin :
g_2share_phase2_out_col
286       assign phase2_out[0][x][y] = dom_out_low_i ?
{ state_in[0][x][y][W-1:W/2], iota_data[0][x][y]
[W/2-1:0] } :
287       {iota_data[0][x][y][W-1:W/2], state_in[0][x][y]
[W/2-1:0]};
288       assign phase2_out[1][x][y] = dom_out_low_i ?
{ state_in[1][x][y][W-1:W/2], iota_data[1][x][y]
[W/2-1:0] } :
289       {iota_data[1][x][y][W-1:W/2], state_in[1][x][y]
[W/2-1:0]};
290     end
291   end else begin : g_single_chi
292     assign chi_data[0] = chi(phase2_in[0]);
293     assign phase2_out = iota_data;
294   end
295
296   // Rho
297   // As RhoOffset[x][y] is considered as variable int in VCS,
// it is replaced with generate statement.
298   // Revised to meet verilog lint. Now RhoOffset is 1-D ar-
ray
299   localparam int RhoOffset [25] = '{
300     //y 0 1 2 3 4 x
301     0, 36, 3, 105, 210, // 0: 0 1 2 3 4
302     1, 300, 10, 45, 66, // 1: 5 6 7 8 9
303     190, 6, 171, 15, 253, // 2: 10 11 12 13 14
304     28, 55, 153, 21, 120, // 3: 15 16 17 18 19
305     91, 276, 231, 136, 78 // 4: 20 21 22 23 24
306   };
307   for (genvar i = 0 ; i < Share ; i++) begin : g_rho
308     box_t rho_in, rho_out;
309     assign rho_in = theta_data[i];
310     assign rho_data[i] = rho_out;
311
312     for (genvar x = 0 ; x < 5 ; x++) begin : gen_rho_x
313       for (genvar y = 0 ; y < 5 ; y++) begin : gen_rho_y
314         localparam int Offset = RhoOffset[5*x+y]%W;
315         localparam int ShiftAmt = W- Offset;
316         if (Offset == 0) begin : gen_offset0
317           assign rho_out[x][y][W-1:0] = rho_in[x][y][W-1:0];
318         end else begin : gen_others
319           assign rho_out[x][y][W-1:0] = {rho_in[x][y]
[0+:ShiftAmt],
320                                           rho_in[x][y]
[ShiftAmt+:Offset]};
321         end
322       end
323     end
324   end : g_rho
325
326   // Assertions //
327   // ASSERT_INIT(ValidWidth_A,
328   // EnMasking == 0 && Width inside {25, 50, 100, 200, 400,
800, 1600} ||

```

```

337     EnMasking == 1 && Width inside {50, 100, 200, 400, 800,
1600})
338     `ASSERT_INIT(ValidW_A, W inside {1, 2, 4, 8, 16, 32, 64})
339     `ASSERT_INIT(ValidL_A, L inside {0, 1, 2, 3, 4, 5, 6})
340     `ASSERT_INIT(ValidRound_A, MaxRound <= 24) // Keccak-f only
341
342     // phase_sel_i shall stay for two cycle after change to 1.
343     lc_ctrl_pkg::lc_tx_t unused_lc_sig;
344     assign unused_lc_sig = lc_escalate_en_i;
345     if (EnMasking) begin : gen_selperiod_chk
346         ASSUME(SelStayTwoCycleIfTrue_A,
347             ($past(phase_sel_i) == MuB14False) && (phase_sel_i ==
MuB14True)
348             | => phase_sel_i == MuB14True, clk_i, !rst_ni ||
349             lc_c-
trl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
350     end
351
352     // Functions //
353     // Convert bitarray to 3D box
354     // Please take a look at FIPS PUB 202
355     // https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
356     // > For all triples (x,y,z) such that 0<=x<5, 0<=y<5, and
0<=z<w,
357     // > A[x,y,z]=S[w(5y+x)+z]
358     function automatic box_t bitarray_to_box(logic [Width-1:0]
s_in);
359     automatic box_t box;
360     for (int y = 0 ; y < 5 ; y++) begin
361         for (int x = 0 ; x < 5 ; x++) begin
362             for (int z = 0 ; z < W ; z++) begin
363                 box[x][y][z] = s_in[w*(5*y+x) + z];
364             end
365         end
366     end
367     return box;
368 endfunction : bitarray_to_box
369
370 // Convert 3D cube to bitarray
371 function automatic logic [Width-1:0] box_to_bitarray(box_t
state);
372 automatic logic [Width-1:0] bitarray;
373 for (int y = 0 ; y < 5 ; y++) begin
374     for (int x = 0 ; x < 5 ; x++) begin
375         for (int z = 0 ; z < W ; z++) begin
376             bitarray[w*(5*y+x)+z] = state[x][y][z];
377         end
378     end
379 end
380 return bitarray;
381 endfunction : box_to_bitarray
382
383 // Rotate integer indices
384 function automatic integer rot_int(integer in, integer num);
385 integer out;
386 if (in == 0) begin
387     out = num - 1;
388 end else begin
389     out = in - 1;
390 end
391 return out;
392 endfunction
393
394 // Step Mapping
395
396 // theta
397 // XOR each bit in the state with the parity of two columns
398 // C[x,z] = A[x,0,z] ^ A[x,1,z] ^ A[x,2,z] ^ A[x,3,z] ^ A[x,
4, z]
399 // D[x,z] = C[x-1,z] ^ C[x+1,z-1]
400 // theta = A[x,y,z] ^ D[x,z]
401 localparam int ThetaIndexX1 [5] = '{4, 0, 1, 2, 3}; //
(x-1)%5
402 localparam int ThetaIndexX2 [5] = '{1, 2, 3, 4, 0}; //
(x+1)%5
403 function automatic box_t theta(box_t state);
404 plane_t c;
405 plane_t d;
406 box_t result;
407 for (int x = 0 ; x < 5 ; x++) begin
408     c[x] = state[x][0] ^ state[x][1] ^ state[x][2] ^
state[x][3] ^ state[x][4];
409 end
410 for (int x = 0 ; x < 5 ; x++) begin
411     for (int z = 0 ; z < W ; z++) begin
412         int index_z;
413         index_z = (z == 0) ? W-1 : z-1; // (z+1)%W
414         d[x][z] = c[ThetaIndexX1[x]][z] ^ c[ThetaIndexX2[x]]
[index_z];
415     end
416 end
417 for (int x = 0 ; x < 5 ; x++) begin
418     for (int y = 0 ; y < 5 ; y++) begin
419         result[x][y] = state[x][y] ^ d[x];
420     end
421 end
422 return result;
423 endfunction : theta

```

```

426 // rho
427 // rho
428 // Commented out entire rho function due to VCS elaboration
error.
429 // ((z-RhoOffset[x][y]%W) isn't considered as a constant in
VCS.
430 // Even changing it to W-RhoOffset[x][y]%W and assign to
ShiftAmt
431 // creates same error.
432 // Offset : Look at Table 2 in FIPS PUB 202
433 //localparam int RhoOffset [5][5] = '{
434 // //y 0 1 2 3 4 x
435 // '{ 0, 36, 3, 105, 210}, // 0
436 // '{ 1, 300, 10, 45, 66}, // 1
437 // '{ 190, 6, 171, 15, 253}, // 2
438 // '{ 28, 55, 153, 21, 120}, // 3
439 // '{ 91, 276, 231, 136, 78} // 4
440 //';
441 // rotate bits of each lane by offset
442 // 1. rho[0,0,z] = A[0,0,z]
443 // 2. Offset swap
444 // a. (x,y) := (1,0)
445 // b. for t [0..23]
446 // i. rho[x,y,z] = A[x,y,z-(t+1)(t+2)/2]
447 // ii. (x,y) = (y, (2x+3y))
448 //function automatic box_t rho(box_t state);
449 // box_t result;
450 // for (int x = 0 ; x < 5 ; x++) begin
451 // for (int y = 0 ; y < 5 ; y++) begin
452 // for (int z = 0 ; z < W ; z++) begin
453 // automatic int index_z;
454 // index_z = (z-RhoOffset[x][y]%W);
455 // result[x][y][z] = state[x][y][(z-RhoOffset[x][y])
%W];
456 // end
457 // end
458 // end
459 // return result;
460 //endfunction : rho
461
462 // pi
463 // rearrange the position of lanes
464 // pi[x,y,z] = state[(x+3y),x,z]
465 localparam int PiRotate [5][5] = '{
466 //y 0 1 2 3 4 x
467 // '{ 0, 3, 1, 4, 2}, // 0
468 // '{ 1, 4, 2, 0, 3}, // 1
469 // '{ 2, 0, 3, 1, 4}, // 2
470 // '{ 3, 1, 4, 2, 0}, // 3
471 // '{ 4, 2, 0, 3, 1} // 4
472 //';
473 function automatic box_t pi(box_t state);
474 box_t result;
475 for (int x = 0 ; x < 5 ; x++) begin
476     for (int y = 0 ; y < 5 ; y++) begin
477         result[x][y][W-1:0] = state[PiRotate[x][y]][x][W-1:0];
478     end
479 end
480 return result;
481 endfunction : pi
482
483 // chi
484 // chi[x,y,z] = state[x,y,z] ^ ((state[x+1,y,z] ^ 1) &
state[x+2,y,z])
485 localparam int ChiIndexX1 [5] = '{1, 2, 3, 4, 0}; // (x+1)%5
486 localparam int ChiIndexX2 [5] = '{2, 3, 4, 0, 1}; // (x+2)%5
487 function automatic box_t chi(box_t state);
488 box_t result;
489 for (int x = 0 ; x < 5 ; x++) begin
490     result[x] = state[x] ^ ((~state[ChiIndexX1[x]]) &
state[ChiIndexX2[x]]);
491 end
492 return result;
493 endfunction : chi
494
495 // iota
496 // XOR (x,y) = (0,0) with Round Constant (RC)
497 // RC parameter: Precomputed by util/keccak_rc.py. Only up-
to 0..L-1 is used
498 // RC = '0
499 // RC[2**j-1] = rc(j+7*rnd)
500 // rc(t) =
501 // 1. t%255 == 0 -> 1
502 // 2. R[0:7] = 'b10000000
503 // 3. for i = [1..t%255]
504 // a. R = 0 || R
505 // b. R[0] = R[0] ^ R[8]
506 // c. R[4] = R[4] ^ R[8]
507 // d. R[5] = R[5] ^ R[8]
508 // e. R[6] = R[6] ^ R[8]
509 // f. R = R[0:7]
510 // 4. return R[0]
511 // RC has L = [0..6]
512
513 // for lower L case, only chopping lower part of 64bit RC is
sufficient.
514 localparam logic [63:0] RC [24] = '{

```

```

518 64'h 0000_0000_0000_0001, // Round 0
519 64'h 0000_0000_0000_8082, // Round 1
520 64'h 8000_0000_0000_808A, // Round 2
521 64'h 8000_0000_8000_8000, // Round 3
522 64'h 0000_0000_0000_808B, // Round 4
523 64'h 0000_0000_8000_0001, // Round 5
524 64'h 8000_0000_8000_8081, // Round 6
525 64'h 8000_0000_0000_8009, // Round 7
526 64'h 0000_0000_0000_008A, // Round 8
527 64'h 0000_0000_0000_0088, // Round 9
528 64'h 0000_0000_8000_8009, // Round 10
529 64'h 0000_0000_8000_000A, // Round 11
530 64'h 0000_0000_8000_808B, // Round 12
531 64'h 8000_0000_0000_008B, // Round 13
532 64'h 8000_0000_0000_8089, // Round 14
533 64'h 8000_0000_0000_8003, // Round 15
534 64'h 8000_0000_0000_8002, // Round 16
535 64'h 8000_0000_0000_0080, // Round 17
536 64'h 0000_0000_0000_800A, // Round 18
537 64'h 8000_0000_8000_000A, // Round 19
538 64'h 8000_0000_8000_8081, // Round 20
539 64'h 8000_0000_0000_8080, // Round 21
540 64'h 0000_0000_8000_0001, // Round 22
541 64'h 8000_0000_8000_8008 // Round 23
542 };
543
544 // iota: XOR with RC for (x,y) = (0,0)
545 function automatic box_t iota(box_t state, logic [RndW-1:0]
rnd);
546     box_t result;
547     result = state;
548     result[0][0][W-1:0] = state[0][0][W-1:0] ^ RC[rnd][W-1:0];
549
550     return result;
551 endfunction : iota
552
553 // Round function : Rnd(A,i_r)
554 // Not used due to rho function issue described above.
555
556 //function automatic box_t keccak_rnd(box_t state, logic
[RndW-1:0] rnd);
557 // box_t keccak_state;
558 // keccak_state = iota(chi(pi(rho(theta(state))))), rnd);
559 //
560 // return keccak_state;
561 //endfunction : keccak_rnd
562
563 endmodule

```

2. lowRISC_opentitan/hw/ip/kmac/rtl/keccak_round.sv (611 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Keccak full round logic based on given input `Width`
6 // e.g. Width 800 requires 22 rounds
7 //
8 // This module has two phases:
9 // 1. Compute Theta, Rho, Pi steps.
10 // 2. Computes Chi and Iota steps.
11 //
12 // If masking is not enabled, the two phases are completed
within a single
13 // clock cycle.
14 //
15 // If masking is enabled, the first phase (Phase1) completes
in one cycle.
16 // The second phase then needs three clock cycles to complete:
17 // 1. In the first clock cycle, the first stage of Chi is com-
puted for the
18 // first lane halves.
19 // 2. In the second clock cycle, the module writes the updated
first lane
20 // halves to the state.
21 // 3. In the third clock cycle, the new second lane halves are
written to
22 // to the state.
23 //
24 // To deter SCA, we randomly decide which lane halves to pro-
cess first on a
25 // per-round basis. We use additional randomness generated by
the PRNG to take
26 // this decision (rand_aux_i). For more details, refer to the
comments inside
27 // the FSM below.
28 //
29 // If masking is enabled, this implementation uses both ran-
domness provided
30 // from an external PRNG as well as intermediate results for
remasking the DOM
31 // multipliers below. Per clock cycle, 800b of pseudo-random
data (PRD) are
32 // required. A carefully designed schedule ensures to only
ever update the input

```

```

33 // data of the DOM multipliers when also providing fresh ran-
domness and vice
34 // versa. Updating one without the other could lead to un-
desired SCA leakage.
35
36 `include "prim_assert.sv"
37
38 module keccak_round
39     import prim_mubi_pkg::*;
40 # (
41     parameter int Width = 1600, // b= {25, 50, 100, 200, 400,
800, 1600}
42
43     // Derived
44     localparam int W      = Width/25,
45     localparam int L      = $clog2(W),
46     localparam int MaxRound = 12 + 2*L, // Keccak-f
only
47     localparam int RndW    = $clog2(MaxRound+1), // Represent-
ing up to MaxRound-1
48
49     // Feed parameters
50     parameter int DINWidth = 64, // currently only 64bit sup-
ported
51     localparam int DINEntry = Width / DINWidth,
52     localparam int DINAddr  = $clog2(DINEntry),
53
54     // Control parameters
55     parameter bit EnMasking = 1'b0, // Enable SCA harden-
ing, requires Width >= 50
56     parameter bit ForceRandExt = 1'b0, // 1: Always forward
externally provided randomness.
57                                     // 0: Switch between
external randomness and internal
58                                     // intermediate
state according to schedule.
59     localparam int Share      = EnMasking ? 2 : 1
60 ) (
61     input clk_i,
62     input rst_ni,
63
64     // Message Feed
65     input          valid_i,
66     input [DINAddr-1:0] addr_i,
67     input [DINWidth-1:0] data_i [Share],
68     output          ready_o,
69
70     // In-process control
71     input          run_i, // Pulse signal to initi-
ates Keccak full round
72     input          rand_valid_i,
73     input          rand_early_i,
74     input [Width/2-1:0] rand_data_i,
75     input          rand_aux_i,
76     output logic    rand_update_o,
77     output logic    rand_consumed_o,
78
79     output logic    complete_o, // Indicates full round is
done
80
81     // State out. This can be used as Digest
82     output logic [Width-1:0] state_o [Share],
83
84     // Life cycle
85     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
86
87     // Errors:
88     // sparse_fsm_error: Checking if FSM state falls into un-
known value
89     output logic    sparse_fsm_error_o,
90     // round_count_error: prim_count checks round value con-
sistency
91     output logic    round_count_error_o,
92
93     // rst_storage_error: check if reset signal asserted out of
the
94     // permitted window
95     output logic    rst_storage_error_o,
96
97     input prim_mubi_pkg::mubi4_t clear_i // Clear internal
state to '0'
98 );
99 import sha3_pkg::*;
100
101 // Control signals //
102 // Control signals //
103 // Control signals //
104
105 // Update storage register
106 logic update_storage;
107
108 // Reset the storage to 0 to initiate new Hash operation
109 logic rst_storage;
110
111 // XOR message into storage register
112 // It only does based on the given DINWidth.
113 // If DINWidth < Width, it takes multiple cycles to XOR all
message
114 logic xor_message;

```

```

115
116 // Select Keccak_p datapath
117 // 0: Select Phase1 (Theta -> Rho -> Pi)
118 // 1: Select Phase2 (Chi -> Iota)
119 // 'phase_sel' needs to be asserted until the Chi stage is
consumed,
120 mubi4_t phase_sel;
121
122 // DOM multiplier input/output mux control
123 // 0: first compute upper, then lower lane halves
124 // 1: first compute lower, then upper lane halves
125 logic low_then_high_d, low_then_high_q;
126 // 0: drive/select upper lane halves
127 // 1: drive/select lower lane halves
128 logic dom_out_low_d, dom_out_low_q;
129 logic dom_in_low_d, dom_in_low_q;
130 // 0: forward external randomness input
131 // 1: forward partial intermediate results
132 logic dom_in_rand_ext_d, dom_in_rand_ext_q;
133 // 0: keep current intermediate results in pipeline re-
gisters
134 // 1: latch new intermediate results into pipeline registers
135 logic dom_update;
136
137 // Increase/ Reset Round number
138 logic inc_rnd_num;
139 logic rst_rnd_num;
140
141 // Round reaches end
142 // This signal indicates the round reaches desired number,
which is MaxRound -1.
143 // MaxRound is dependant on the Width. In case of SHA3/
SHAKE, MaxRound is 24.
144 logic rnd_eq_end;
145
146 // Complete of Keccak_f
147
148 // State machine asserts 'complete_d' when it reaches at the
end of round and
149 // operation (Phase3 if Masked). The stage, the storage
still doesn't have
150 // the valid states. So precisely it is not completed yet.
151 // State generated 'complete_d' is latched with the clock
and creates a pulse
152 // signal one cycle later. The signal is the indication of
completion.
153 //
154 // Intentionally removed any intermediate step (so called
StComplete) in order
155 // to save a clock to proceeds next round.
156 logic complete_d;
157
158 // Datapath Signals //
159 //
160
161 // Single round keccak output data
162 logic [Width-1:0] keccak_out [Share];
163
164 // Keccak Round indicator: range from 0 .. MaxRound
165 logic [RndW-1:0] round;
166
167 // Random value and valid signal used in Keccak_p
168 logic keccak_rand_update;
169 logic keccak_rand_consumed;
170 logic [Width/2-1:0] keccak_rand_data;
171
172 //
173 // Keccak Round FSM //
174 //
175
176 // state inputs
177 assign rnd_eq_end = (int'(round) == MaxRound - 1);
178
179 keccak_st_e keccak_fsm, keccak_st_d;
180 // PRIM_FLOP_SPARSE_FSM(u_state_regs, keccak_st_d, keccak_st,
keccak_st_e, KeccakStIdle)
181
182 // Next state logic and output logic
183 // SEC_CM: FSM.SPARSE
184 always_comb begin
185 // Default values
186 keccak_st_d = keccak_st;
187
188 xor_message = 1'b 0;
189 update_storage = 1'b 0;
190 rst_storage = 1'b 0;
191
192 inc_rnd_num = 1'b 0;
193 rst_rnd_num = 1'b 0;
194
195 keccak_rand_update = 1'b 0;
196 keccak_rand_consumed = 1'b 0;
197
198 phase_sel = MuBi4False;
199 low_then_high_d = low_then_high_q;
200 dom_in_low_d = dom_in_low_q;
201 dom_in_rand_ext_d = dom_in_rand_ext_q;
202 dom_update = 1'b 0;
203
204 complete_d = 1'b 0;
205
206 sparse_fsm_error_o = 1'b 0;
207
208 unique case (keccak_st)
209 KeccakStIdle: begin
210 if (valid_i) begin
211 // State machine allows Sponge Absorbing only in
Idle state.
212 keccak_st_d = KeccakStIdle;
213
214 xor_message = 1'b 1;
215 update_storage = 1'b 1;
216 end else if
(prim_mubi_pkg::mubi4_test_true_strict(clear_i)) begin
217 // Opt1. State machine allows resetting the storage
only in Idle
218 // Opt2. storage resets regardless of states but
clear_i
219 // Both are added in the design at this time. Will
choose the
220 // direction later.
221 keccak_st_d = KeccakStIdle;
222
223 rst_storage = 1'b 1;
224 end else if (EnMasking && run_i) begin
225 // Masked version of Keccak handling
226 keccak_st_d = KeccakStPhase1;
227
228 // Drive DOM multiplier I/O mux signals for Phase 1.
229 dom_in_low_d = low_then_high_q;
230 dom_in_rand_ext_d = 1'b 0;
231 end else if (!EnMasking && run_i) begin
232 // Unmasked version of Keccak handling
233 keccak_st_d = KeccakStActive;
234 end else begin
235 keccak_st_d = KeccakStIdle;
236 end
237 end
238
239 KeccakStActive: begin
240 // Run Keccak single round logic until it reaches
MaxRound - 1
241 update_storage = 1'b 1;
242
243 if (rnd_eq_end) begin
244 keccak_st_d = KeccakStIdle;
245
246 rst_rnd_num = 1'b 1;
247 complete_d = 1'b 1;
248 end else begin
249 keccak_st_d = KeccakStActive;
250
251 inc_rnd_num = 1'b 1;
252 end
253 end
254
255 KeccakStPhase1: begin
256 // Compute Theta, Rho, Pi - The DOM multipliers are
not evaluated at
257 // all: their inputs are driven by the first lane
halves (same values
258 // as in Phase2Cycle3 of the last round). Also, the
intermediate
259 // results we already had in Phase2Cycle3 didn't
change.
260 phase_sel = MuBi4False;
261 dom_update = 1'b 0;
262
263 // Only update the state and move on once we know the
auxiliary
264 // randomness required for Phase2 will be available in
the next clock
265 // cycle.
266 //
267 // It's important that the DOM multipliers inside kec-
cak_2share are
268 // presented the new state (updated with up-
date_storage) at the same
269 // time as the new randomness (updated with rand_up-
date_o). Otherwise,
270 // stale entropy is paired with fresh data or vice
versa. This could
271 // lead to undesired SCA leakage.
272 if (rand_early_i || rand_valid_i) begin
273 keccak_st_d = KeccakStPhase2Cycle1;
274 update_storage = 1'b 1;
275 keccak_rand_update = 1'b 1;
276
277 // Update lane halves processing order for this
round.
278 low_then_high_d = rand_aux_i;
279
280 // Drive DOM multiplier I/O mux signals for next
phase.
281 dom_in_low_d = low_then_high_d;
282 dom_in_rand_ext_d = 1'b 1;
283 end else begin
284 keccak_st_d = KeccakStPhase1;
285 end
286 end

```

```

287
288     KeccakStPhase2Cycle1: begin
289         // Compute first stage of Chi for first lane halves
using the DOM
290         // multipliers. Use the fresh randomness provided by
the PRNG for
291         // remasking.
292         phase_sel = MuBi4True;
293         dom_update = 1'b 1;
294
295         // Trigger randomness update for next cycle.
296         // It's important that the DOM multipliers inside kec-
cak_2share are
297         // presented the second lane halves at the same time
as the new
298         // randomness (updated with rand_update_o). Otherwise,
stale entropy
299         // is paired with fresh data or vice versa. This could
lead to
300         // undesired SCA leakage.
301         keccak_rand_update = 1'b 1;
302
303         // Unconditionally move to next phase/cycle.
304         keccak_st_d = KeccakStPhase2Cycle2;
305
306         // Drive DOM multiplier I/O mux signals for next
phase.
307         dom_in_low_d = ~low_then_high_q;
308         dom_in_rand_ext_d = 1'b 1;
309     end
310
311     KeccakStPhase2Cycle2: begin
312         // Chi Stage 1 for second lane halves.
313         // Chi Stage 2 and Iota for first lane halves.
314
315         // Compute second stage of Chi and Iota for first lane
halves.
316         // Compute first stage of Chi for second lane halves.
Use the fresh
317         // randomness provided by the PRNG for remasking the
DOM multipliers.
318         phase_sel = MuBi4True;
319         dom_update = 1'b 1;
320
321         // Trigger randomness update for next cycle.
322         // It's important that the DOM multipliers inside kec-
cak_2share are
323         // presented the updated state at the same as the new
randomness
324         // (updated with rand_update_o) - even if the DOM mul-
tipliers don't
325         // update the pipeline registers in the next cycle.
Otherwise, stale
326         // entropy is paired with fresh data or vice versa.
This could lead to
327         // undesired SCA leakage.
328         keccak_rand_update = 1'b 1;
329
330         // Trigger auxiliary randomness update for next round.
The rand_aux_i
331         // signal is actually going to change in 2 clock
cycles from now
332
333         // (Phase1) based on the PRNG output in the next cycle
(Phase2Cycle3)
334         // in which the DOM multipliers don't update the
pipeline registers.
335         keccak_rand_consumed = 1'b 1;
336
337         // Update first lane halves.
338         update_storage = 1'b 1;
339
340         // Unconditionally move to next phase/cycle.
341         keccak_st_d = KeccakStPhase2Cycle3;
342
343         // Drive DOM multiplier I/O mux signals for next
phase.
344         dom_in_low_d = low_then_high_q;
345         dom_in_rand_ext_d = 1'b 0;
346     end
347
348     KeccakStPhase2Cycle3: begin
349         // Compute second stage of Chi and Iota for second
lane halves.
350         // Feed again first lane halves to DOM multiplier in-
puts (now
351         // the updated values become visible) together with
intermediate
352         // results of Phase2Cycle2. Don't update the register
stage inside
353         // the DOM multipliers.
354         phase_sel = MuBi4True;
355         dom_update = 1'b 0;
356
357         // Update second lane halves.
358         // We don't need fresh randomness for the next cycle
as the DOM
359         // multipliers inside keccak_2share will keep seeing

```

```

the first
358         // lane halves in the next cycle. If we updated the
randomness,
359         // old data got combined with fresh randomness which
is not
360         // desirable as it could lead to SCA leakage.
361         update_storage = 1'b 1;
362
363         if (rnd_eq_end) begin
364             // We're done.
365             keccak_st_d = KeccakStIdle;
366
367             rst_rnd_num = 1'b 1;
368             complete_d = 1'b 1;
369         end else begin
370             // Continue to the next round.
371             keccak_st_d = KeccakStPhase1;
372
373             inc_rnd_num = 1'b 1;
374
375             // Drive DOM multiplier I/O mux signals for next
phase.
376             dom_in_low_d = low_then_high_q;
377             dom_in_rand_ext_d = 1'b 0;
378         end
379     end
380
381     KeccakStError: begin
382         keccak_st_d = KeccakStError;
383     end
384
385     KeccakStTerminalError: begin
386         //this state is terminal
387         keccak_st_d = keccak_st;
388         sparse_fsm_error_o = 1'b 1;
389     end
390
391     default: begin
392         keccak_st_d = KeccakStTerminalError;
393         sparse_fsm_error_o = 1'b 1;
394     end
395 endcase
396
397 // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
398 // Unconditionally jump into the terminal error state
399 // if the life cycle controller triggers an escalation.
400 if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
begin
401     keccak_st_d = KeccakStTerminalError;
402 end
403 end
404
405 // When taking the lower lane halves in, the upper lane
halves are output and
406 // vice versa.
407 assign dom_out_low_d = ~dom_in_low_d;
408
409 if (EnMasking) begin : gen_regs_dom_ctrl
410     always_ff @(posedge clk_i or negedge rst_ni) begin
411         if (!rst_ni) begin
412             low_then_high_q <= 1'b 0;
413             dom_out_low_q <= 1'b 0;
414             dom_in_low_q <= 1'b 0;
415         end else begin
416             low_then_high_q <= low_then_high_d;
417             dom_out_low_q <= dom_out_low_d;
418             dom_in_low_q <= dom_in_low_d;
419         end
420     end
421
422     if (!ForceRandExt) begin : gen_reg_dom_in_rand_ext
423         always_ff @(posedge clk_i or negedge rst_ni) begin
424             if (!rst_ni) begin
425                 dom_in_rand_ext_q <= 1'b 0;
426             end else begin
427                 dom_in_rand_ext_q <= dom_in_rand_ext_d;
428             end
429         end
430     end else begin : gen_force_dom_in_rand_ext
431         // Always forward the externally provided randomness.
432         assign dom_in_rand_ext_q = 1'b 1;
433         // Tie off unused signals.
434         logic unused_dom_in_rand_ext;
435         assign unused_dom_in_rand_ext = dom_in_rand_ext_d;
436     end
437 end else begin : gen_no_regs_dom_ctrl
438     logic unused_dom_ctrl;
439     assign unused_dom_ctrl =
440         ^{low_then_high_d, dom_out_low_d, dom_in_low_d,
dom_in_rand_ext_d};
441     assign low_then_high_q = 1'b 0;
442     assign dom_out_low_q = 1'b 0;
443     assign dom_in_low_q = 1'b 0;
444     assign dom_in_rand_ext_q = 1'b 0;
445 end
446
447 // Ready indicates the keccak_round is able to receive new
message.
448 // While keccak_round is processing the data, it blocks the
new message to be
449 // XORed into the current state.

```



```

450 assign ready_o = (keccak_st == KeccakStIdle) ? 1'b 1 : 1'b
0;
451
452 //////////////////////////////////////////////////
453 // Keccak state registers //
454 //////////////////////////////////////////////////
455
456 // SEC_CM: LOGIC_INTEGRITY
457 logic rst_n;
458 prim_sec_anchor_buf #(
459   .width(1)
460 ) u_prim_sec_anchor_buf (
461   .in_i(rst_n),
462   .out_o(rst_n)
463 );
464
465 logic [Width-1:0] storage [Share];
466 logic [Width-1:0] storage_d [Share];
467 always_ff @(posedge clk_i or negedge rst_n) begin
468   if (!rst_n) begin
469     storage <= '{default:'0};
470   end else if (rst_storage) begin
471     storage <= '{default:'0};
472   end else if (update_storage) begin
473     storage <= storage_d;
474   end
475 end
476
477 assign state_o = storage;
478
479 // Storage register input
480 // The incoming message is XORed with the existing storage
registers.
481 // The logic can accept not a block size incoming message
chunk but
482 // the size defined in `DInWidth` parameter with its posi-
tion.
483
484 always_comb begin
485   storage_d = keccak_out;
486   if (xor_message) begin
487     for (int j = 0; j < Share; j++) begin
488       for (int unsigned i = 0; i < DInEntry; i++) begin
489         // ICEBOX(#18029): handle If Width is not integer
divisible by DInWidth
490         // Currently it is not allowed to have partial write
491         // Please see the Assertion `WidthDivisibleByDIn-
Width_A`
492         if (addr_i == i[DInAddr-1:0]) begin
493           storage_d[j][i*DInWidth+:DInWidth] =
494             storage[j][i*DInWidth+:DInWidth] ^ data_i[j];
495         end else begin
496           storage_d[j][i*DInWidth+:DInWidth] = storage[j]
[i*DInWidth+:DInWidth];
497         end
498       end // for i
499     end // for j
500   end // if xor_message
501 end
502
503 // Check the rst_storage integrity
504 logic rst_storage_error;
505
506 always_comb begin : chk_rst_storage
507   rst_storage_error = 1'b 0;
508
509   if (rst_storage) begin
510     // FSM should be in KeccakStIdle and clear_i should be
high
511     if ((keccak_st != KeccakStIdle) ||
512         prim_mubi_pkg::mubi4_test_false_loose(clear_i)) begin
513       rst_storage_error = 1'b 1;
514     end
515   end
516 end : chk_rst_storage
517
518 assign rst_storage_error_o = rst_storage_error ;
519
520 //////////////////////////////////////////////////
521 // Datapath //
522 //////////////////////////////////////////////////
523 keccak_2share #(
524   .width(Width),
525   .EnMasking(EnMasking),
526   .ForceRandExt(ForceRandExt)
527 ) u_keccak_p (
528   .clk_i,
529   .rst_ni,
530
531   .lc_escalate_en_i,
532
533   .rnd_i(round),
534
535   .phase_sel_i (phase_sel),
536   .dom_out_low_i (dom_out_low_q),
537   .dom_in_low_i (dom_in_low_q),
538   .dom_in_rand_ext_i(dom_in_rand_ext_q),
539   .dom_update_i (dom_update),
540
541   .rand_i(keccak_rand_data),
542

```

```

543   .s_i(storage),
544   .s_o(keccak_out)
545 );
546
547 // keccak entropy handling
548 assign rand_update_o = keccak_rand_update;
549 assign rand_consumed_o = keccak_rand_consumed;
550
551 assign keccak_rand_data = rand_data_i;
552
553 // Round number
554 // This primitive is used to place a hardened counter
555 // SEC_CM: CTR.REDUN
556 prim_count #(
557   .width(RndW)
558 ) u_round_count (
559   .clk_i,
560   .rst_ni,
561   .clr_i(rst_rnd_num),
562   .set_i(1'b0),
563   .set_cnt_i('0),
564   .incr_en_i(inc_rnd_num),
565   .decr_en_i(1'b0),
566   .step_i(RndW'(1)),
567   .commit_i(1'b1),
568   .cnt_o(round),
569   .cnt_after_commit_o(),
570   .err_o(round_count_error_o)
571 );
572
573 // completion signal
574 always_ff @(posedge clk_i or negedge rst_ni) begin
575   if (!rst_ni) begin
576     complete_o <= 1'b 0;
577   end else begin
578     complete_o <= complete_d;
579   end
580 end
581
582 //////////////////////////////////////////////////
583 // Assertions //
584 //////////////////////////////////////////////////
585
586 // Only allow `DInWidth` that `Width` is integer divisible
by `DInWidth`
587 `ASSERT_INIT(WidthDivisibleByDInWidth_A, (Width % DInWidth)
== 0)
588
589 // If `run_i` triggered, it shall complete
590 // `ASSERT(RunResultComplete_A, run_i ##[MaxRound:] com-
plete_o, clk_i, !rst_ni)
591
592 // valid_i and run_i cannot be asserted at the same time
593 `ASSUME(OneHot0ValidAndRun_A, $onehot0{valid_i, run_i}),
clk_i, !rst_ni)
594
595 // valid_i, run_i only asserted in Idle state
596 `ASSUME(ValidRunAssertStIdle_A, valid_i || run_i |>-> kec-
cak_st == KeccakStIdle, clk_i, !rst_ni)
597
598 // clear_i is assumed to be asserted in Idle state
599 `ASSUME(ClearAssertStIdle_A,
600   prim_mubi_pkg::mubi4_test_true_strict(clear_i)
601   |>-> keccak_st == KeccakStIdle, clk_i, !rst_ni)
602
603 // EnMasking controls the valid states
604 if (EnMasking) begin : gen_mask_st_chk
605   `ASSERT(EnMaskingValidStates_A, keccak_st != KeccakStAct-
ive, clk_i, !rst_ni)
606 end else begin : gen_unmask_st_chk
607   `ASSERT(UnmaskValidStates_A, !(keccak_st
608     inside {KeccakStPhase1, KeccakStPhase2Cycle1, Keccak-
StPhase2Cycle2, KeccakStPhase2Cycle3}),
609     clk_i, !rst_ni)
610 end
611 endmodule

```

3. lowRISC_opentitan/hw/ip/kmac/rtl/kmac.sv (1,606 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // KMAC/SHA3
6
7 `include "prim_assert.sv"
8
9 module kmac
10   import kmac_pkg::*;
11   import kmac_reg_pkg::*;
12 #()
13 // EnMasking: Enable masking security hardening inside kec-
cak_round
14 // If it is enabled, the result digest will be two set of
1600bit.
15   parameter bit EnMasking = 1,
16
17   // In case EnMasking == 0, this defines whether SW can

```

```

provide a masked key or whether Share 1 of
18 // the SW key is simply ignored. In case EnMasking == 1,
this parameter has no meaning, always
19 // both shares of the key provided by SW are used.
20 // This is useful to allow both for area-optimized unmasked
designs as well as unmasked designs
21 // having a SW interface fully compatible with the masked
design.
22 parameter bit SwKeyMasked = 0,
23
24 // Command delay, useful for SCA measurements only. A value
of e.g. 40 allows the processor to go
25 // into sleep before KMAC starts operation. If a value > 0
is chosen, the processor can provide
26 // two commands subsequently and then go to sleep. The
second command is buffered internally and
27 // will be presented to the hardware SecCmdDelay number of
cycles after the first one.
28 parameter int SecCmdDelay = 0,
29
30 // Accept SW message when idle and before receiving a START
command. Useful for SCA only.
31 parameter bit SecIdleAcceptSwMsg = 1'b0,
32 parameter int unsigned NumAppIntf = 4,
33 parameter app_config_t AppCfg[NumAppIntf] = '{AppCfgKeyMgr,
AppCfgLcCtrl,
34                                     AppCfgRomCtrl,
AppCfgG0tbn},
35
36 parameter lfsr_perm_t RndCnstLfsrPerm = RndCnstLfsrPermDe-
fault,
37 parameter lfsr_seed_t RndCnstLfsrSeed = RndCnstLfsrSeedDe-
fault,
38 parameter buffer_lfsr_seed_t RndCnstBufferLfsrSeed = RndCn-
stBufferLfsrSeedDefault,
39 parameter msg_perm_t RndCnstMsgPerm = RndCnstMsgPermDe-
fault,
40
41 parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
mAlerts{1'b1}},
42 // Number of cycles a differential skew is tolerated on the
alert signal
43 parameter int unsigned AlertSkewCycles = 1
44 )
45 input clk_i,
46 input rst_ni,
47
48 input rst_shadowed_ni,
49
50 input clk_edn_i,
51 input rst_edn_ni,
52
53 input tlul_pkg::tl_h2d_t tl_i,
54 output tlul_pkg::tl_d2h_t tl_o,
55
56 // Alerts
57 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
58 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
59
60 // KeyMgr sideload (secret key) interface
61 input keymgr_pkg::hw_key_req_t keymgr_key_i,
62
63 // KeyMgr KDF data path
64 input app_req_t [NumAppIntf-1:0] app_i,
65 output app_rsp_t [NumAppIntf-1:0] app_o,
66
67 // EDN interface
68 output edn_pkg::edn_req_t entropy_o,
69 input edn_pkg::edn_rsp_t entropy_i,
70
71 // Life cycle
72 input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
73
74 // interrupts
75 output logic intr_kmac_done_o,
76 output logic intr_fifo_empty_o,
77 output logic intr_kmac_err_o,
78
79 // parameter consistency check with keymgr
80 output logic en_masking_o,
81
82 // Idle signal
83 output prim_mubi_pkg::mubi4_t idle_o
84 );
85
86 ///////////////////////////////////////////////////
87 // Parameters //
88 ///////////////////////////////////////////////////
89 localparam int Share = (EnMasking) ? 2 : 1 ;
90
91 localparam int SwKeyShare = (EnMasking || SwKeyMasked) ? 2 : 1;
92
93 // Definitions //
94 ///////////////////////////////////////////////////
95 // This state machine is to track the current process based
on SW input and
96 // KMAC operation.

```

```

97 // Encoding generated with:
98 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 6 -n 6 \
99 // -s 1966361510 --language=sv
100 //
101 // Hamming distance histogram:
102 //
103 // 0: --
104 // 1: --
105 // 2: --
106 // 3: ||| (53.33%)
107 // 4: ||| (40.00%)
108 // 5: || (6.67%)
109 // 6: --
110 //
111 // Minimum Hamming distance: 3
112 // Maximum Hamming distance: 5
113 // Minimum Hamming weight: 2
114 // Maximum Hamming weight: 5
115 //
116 localparam int StateWidth = 6;
117 typedef enum logic [StateWidth-1:0] {
118     // Idle state
119     KmacIdle = 6'b001011,
120
121     // When software writes CmdStart @ KmacIdle and kmac_en,
FSM moves to this
122     KmacPrefix = 6'b000110,
123
124     // When SHA3 engine processes Key block, FSM moves to
here.
125     KmacKeyBlock = 6'b111110,
126
127     // Message Feed
128     KmacMsgFeed = 6'b010101,
129
130     // Complete and squeeze
131     KmacDigest = 6'b101101,
132
133     // Error
134     KmacTerminalError = 6'b110000
135 } kmac_st_e;
136
137 kmac_st_e kmac_st, kmac_st_d;
138
139 // Signals //
140 ///////////////////////////////////////////////////
141 kmac_reg2hw_t reg2hw;
142 kmac_hw2reg_t hw2reg;
143
144 // Window
145 typedef enum int {
146     WinState = 0,
147     WinMsgFifo = 1
148 } tl_window_e;
149
150 tlul_pkg::tl_h2d_t tl_win_h2d[2];
151 tlul_pkg::tl_d2h_t tl_win_d2h[2];
152
153 // SHA3 core control signals and its response.
154 // Sequence: start --> process(multiple) --> get absorbed
event --> {run -->} done
155 logic sha3_start, sha3_run, sha3_squeezing;
156 prim_mubi_pkg::mubi4_t sha3_done;
157 prim_mubi_pkg::mubi4_t sha3_done_d;
158 prim_mubi_pkg::mubi4_t sha3_absorbed;
159
160 // Indicate one block processed
161 logic sha3_block_processed;
162
163 // EStatus for entropy
164 logic entropy_in_keyblock;
165
166 // Application interface logic generates absorbed from
sha3_absorbed.
167 // It is active only if SW initiates the hashing engine.
168 prim_mubi_pkg::mubi4_t app_absorbed;
169 logic event_absorbed;
170
171 sha3_pkg::sha3_st_e sha3_fsm;
172
173 // Prefix: kmac_pkg defines Prefix based on N size and S
size.
174 // Then computes left_encode(len(N)) size and left_en-
code(len(S))
175 // For given default value 32, 256 bits, the max
176
177 // encode_string(N) || encode_string(S) is 328. So 11 Prefix
registers are
178 // created.
179 logic [sha3_pkg::NSRegisterSize*8-1:0] reg_ns_prefix;
180 logic [sha3_pkg::NSRegisterSize*8-1:0] ns_prefix;
181
182 // NumWordsPrefix from kmac_reg_pkg
183 `ASSERT_INIT(PrefixRegSameToPrefixPkg_A,
184             kmac_reg_pkg::NumWordsPrefix*4 == sha3_pkg::NS-
RegisterSize)
185
186 // NumEntriesMsgFifo from kmac_reg_pkg must match calculated

```

```

MsgFifoDepth
188 // from kmac_pkg.
189 `ASSERT_INIT(NumEntriesRegSameToNumEntriesPkg_A,
190             kmac_reg_pkg::NumEntriesMsgFifo ==
kmac_pkg::MsgFifoDepth)
191
192 // NumBytesMsgFifoEntry from kmac_reg_pkg must match the
MsgWidth calculated
193 // in kmac_pkg (although MsgWidth is in bits, so we multiply
by 8).
194 `ASSERT_INIT(EntrySizeRegSameToEntrySizePkg_A,
195             kmac_reg_pkg::NumBytesMsgFifoEntry * 8 ==
kmac_pkg::MsgWidth)
196
197 // Output state: this is used to redirect the digest to
KeyMgr or Software
198 // depends on the configuration.
199 logic state_valid;
200 logic [sha3_pkg::StateW-1:0] state [Share];
201
202 // state is de-muxed in keymgr interface logic.
203 // the output from keymgr logic goes into staterd module to
be visible to SW
204 logic reg_state_valid;
205 logic [sha3_pkg::StateW-1:0] reg_state [Share];
206
207 // SHA3 Entropy interface
208 logic sha3_rand_valid, sha3_rand_early, sha3_rand_update,
sha3_rand_consumed;
209 logic [sha3_pkg::StateW/2-1:0] sha3_rand_data;
210 logic sha3_rand_aux;
211
212 // FIFO related signals
213 logic msgfifo_empty, msgfifo_full;
214 logic [kmac_pkg::MsgFifoDepthW-1:0] msgfifo_depth;
215
216 logic msgfifo_valid ;
217 logic [kmac_pkg::MsgWidth-1:0] msgfifo_data [Share];
218 logic [kmac_pkg::MsgStrbW-1:0] msgfifo_strb ;
219 logic msgfifo_ready ;
220
221 // TL-UL Adapter(MSG_FIFO) signals
222 logic tlrām_req;
223 logic tlrām_gnt;
224 logic tlrām_we;
225 logic [8:0] tlrām_addr; // NOT_READ
226 logic [31:0] tlrām_wdata;
227 logic [31:0] tlrām_wmask;
228 logic [31:0] tlrām_rdata;
229 logic tlrām_rvalid;
230 logic [1:0] tlrām_rerror;
231 logic [31:0] tlrām_wdata_endian;
232 logic [31:0] tlrām_wmask_endian;
233
234 logic sw_msg_valid;
235 logic [kmac_pkg::MsgWidth-1:0] sw_msg_data;
236 logic [kmac_pkg::MsgWidth-1:0] sw_msg_strb;
237 logic sw_msg_ready;
238
239 // KeyMgr interface to MSG_FIFO
240 logic mux2fifo_valid;
241 logic [kmac_pkg::MsgWidth-1:0] mux2fifo_data[Share];
242 logic [kmac_pkg::MsgWidth-1:0] mux2fifo_strb;
243 logic mux2fifo_ready;
244 logic mux2fifo_bypass;
245
246 // KMAC to SHA3 core
247 logic msg_valid ;
248 logic [kmac_pkg::MsgWidth-1:0] msg_data [Share];
249 logic [kmac_pkg::MsgWidth-1:0] msg_data_masked [Share];
250 logic [kmac_pkg::MsgStrbW-1:0] msg_strb ;
251 logic msg_ready ;
252
253 // Process control signals
254 // Process pulse propagates from register to SHA3 engine one
by one.
255 // Each module (MSG_FIFO, KMAC core, SHA3 core) generates
the process pulse
256 // after flushing internal data to the next module.
257 logic reg2msgfifo_process, msgfifo2kmac_process,
kmac2sha3_process;
258
259 // Secret Key signals
260 logic [MaxKeyLen-1:0] sw_key_data_reg [SwKeyShare];
261 logic [MaxKeyLen-1:0] sw_key_data [Share];
262 key_len_e sw_key_len;
263 logic [MaxKeyLen-1:0] key_data [Share];
264 key_valid;
265 key_len_e key_len;
266
267 // SHA3 Mode, Strength, KMAC enable for app interface
268 logic reg_kmac_en, app_k-
mac_en;
269 sha3_pkg::sha3_mode_e reg_sha3_mode,
app_sha3_mode;
270 sha3_pkg::keccak_strength_e reg_keccak_strength, app_kec-
cak_strength;
271

```

```

273 // RegIF of enabling unsupported mode & strength
274 logic cfg_en_unsupported_modestrength;
275
276 // Indicating AppIntf is active. This signal is used to
check SW error
277 logic app_active;
278
279 // SEC_CM: SW_CMD.CTRL.SPARSE
280 // Command
281 // sw_cmd is the command written by SW
282 // checked_sw_cmd is checked in the kmac_errchk module.
283 // Invalid command is filtered out in the module.
284 // kmac_cmd is generated in KeyMgr interface.
285 // If SW initiates the KMAC/SHA3, kmac_cmd represents SW
command,
286 // if KeyMgr drives the data, kmac_cmd is controlled in the
state machine
287 // in KeyMgr interface logic.
288 kmac_cmd_e sw_cmd, checked_sw_cmd, kmac_cmd, cmd_q;
289 logic cmd_update;
290
291 // Entropy configurations
292 logic [9:0] wait_timer_prescaler;
293 logic [15:0] wait_timer_limit;
294 logic entropy_refresh_req;
295 logic entropy_seed_update;
296 logic [31:0] entropy_seed_data;
297
298 logic [HashCntW-1:0] entropy_hash_threshold;
299 logic [HashCntW-1:0] entropy_hash_cnt;
300 logic entropy_hash_clr;
301
302 logic entropy_ready;
303 entropy_mode_e entropy_mode;
304 logic reg_entropy_fast_process;
305 logic entropy_fast_process;
306
307 prim_mubi_pkg::mubi4_t entropy_configured;
308
309 // Message Masking
310 logic msg_mask_en, cfg_msg_mask;
311 logic [MsgWidth-1:0] msg_mask;
312
313 // SHA3 Error response
314 sha3_pkg::err_t sha3_err;
315
316 // KeyMgr Error response
317 kmac_pkg::err_t app_err;
318
319 // Entropy Generator Error
320 kmac_pkg::err_t entropy_err;
321
322 // Error checker
323 kmac_pkg::err_t errchecker_err;
324
325 // MsgFIFO Error
326 kmac_pkg::err_t msgfifo_err;
327
328 logic err_processed;
329
330 prim_mubi_pkg::mubi4_t clear_after_error;
331
332 logic alert_fatal, alert_recov_operation;
333 logic alert_intg_err;
334
335 // Life cycle
336 localparam int unsigned NumLcSyncCopies = 6;
337 lc_ctrl_pkg::lc_tx_t [NumLcSyncCopies-1:0] lc_escal-
ate_en_sync;
338 lc_ctrl_pkg::lc_tx_t [NumLcSyncCopies-1:0] lc_escalate_en;
339
340 // Connecting Register IF to logics //
341 // Connecting Register IF to logics //
342
343 // Function-name N and Customization input string S
always_comb begin
344 for (int i = 0 ; i < NumWordsPrefix; ++i) begin
345 reg_ns_prefix[32*i+32] = reg2hw.prefix[i].q;
346 end
347 end
348
349 // Create a lint error to reduce the risk of accidentally
enabling this feature.
350 `ASSERT_STATIC_LINT_ERROR(KmacSecCmdDelayNonDefault, SecCmd-
Delay == 0)
351
352 if (SecCmdDelay > 0) begin : gen_cmd_delay_buf
353 // Delay and buffer commands for SCA measurements.
354 localparam int unsigned WidthCounter = $clog2(SecCmdDelay+1);
355 logic [WidthCounter-1:0] count_d, count_q;
356 logic counting_d, counting_q;
357 logic cmd_buf_empty;
358 kmac_cmd_e cmd_buf_q;
359
360 assign cmd_buf_empty = (cmd_buf_q == CmdNone);
361
362 // When seeing a write to the cmd register, we start
counting. We stop counting once the
363 // counter has expired and the command buffer is empty.
364

```



```

366     assign counting_d = reg2hw.cmd.cmd.qe          ? 1'b1 :
367     counting_q;
368     cmd_update & cmd_buf_empty ? 1'b0 :
369     // Clear counter upon writes to the cmd register or if the
specified delay is reached.
370     assign count_d = reg2hw.cmd.cmd.qe ? '0          :
371     cmd_update ? '0          :
372     counting_q ? count_q + 1'b1 :
count_q;
373
374     // The manual run command cannot be delayed. Software ex-
pects this to be triggered immediately
375     // and will poll the status register to wait for the SHA3
engine to return back to the squeeze
376     // state.
377     assign cmd_update = (cmd_q == CmdManual-
Run)
? 1'b1 :
378     (count_q == SecCmd-
Delay[WidthCounter-1:0]) ? 1'b1 : 1'b0;
379
380     always_ff @(posedge clk_i or negedge rst_ni) begin
381     if (!rst_ni) begin
382     count_q <= '0;
383     counting_q <= 1'b0;
384     end else begin
385     count_q <= count_d;
386     counting_q <= counting_d;
387     end
388     end
389
390     // cmd.q is valid while cmd.qe is high, meaning it needs
to be registered. We buffer one
391     // additional command such that software can write START
followed by PROCESS and then go to
392     // sleep.
393     always_ff @(posedge clk_i or negedge rst_ni) begin
394     if (!rst_ni) begin
395     cmd_q <= CmdNone;
396     cmd_buf_q <= CmdNone;
397     end else begin
398     if (reg2hw.cmd.cmd.qe && cmd_update) begin
399     // New write & counter expired.
400     cmd_q <= cmd_buf_q;
401     cmd_buf_q <= kmac_cmd_e'(reg2hw.cmd.cmd.q);
402     end else if (reg2hw.cmd.cmd.qe) begin
403     // New write.
404     if (counting_q == 1'b0) begin
405     cmd_q <= kmac_cmd_e'(reg2hw.cmd.cmd.q);
406     end else begin
407     cmd_buf_q <= kmac_cmd_e'(reg2hw.cmd.cmd.q);
408     end
409     end else if (cmd_update) begin
410     // Counter expired.
411     cmd_q <= cmd_buf_q;
412     cmd_buf_q <= CmdNone;
413     end
414     end
415     end
416     end
417     end
418     end
419     end else begin : gen_no_cmd_delay_buf
420     // Directly forward signals from register IF.
421     assign cmd_update = reg2hw.cmd.cmd.qe;
422     assign cmd_q = kmac_cmd_e'(reg2hw.cmd.cmd.q);
423     end
424
425     // Command signals
426     assign sw_cmd = (cmd_update) ? cmd_q : CmdNone;
427     `ASSERT_KNOWN(KmacCmd_A, sw_cmd)
428     always_comb begin
429     sha3_start = 1'b 0;
430     sha3_run = 1'b 0;
431     sha3_done_d = prim_mubi_pkg::MuBi4False;
432     reg2msgfifo_process = 1'b 0;
433
434     unique case (kmac_cmd)
435     CmdStart: begin
436     sha3_start = 1'b 1;
437     end
438
439     CmdProcess: begin
440     reg2msgfifo_process = 1'b 1;
441     end
442
443     CmdManualRun: begin
444     sha3_run = 1'b 1;
445     end
446
447     CmdDone: begin
448     sha3_done_d = prim_mubi_pkg::MuBi4True;
449     end
450
451     CmdNone: begin
452     // inactive state
453     end
454
455     default: begin
456     end

```

```

457     endcase
458     end
459
460     // Status register
=====
461     // status.squeeze is valid only when SHA3 engine completes
the Absorb and not
462     // running the manual keccak rounds. This status is for SW
to determine when
463     // to read the STATE values.
464     assign hw2reg.status.sha3_idle.d = sha3_fsm ==
sha3_pkg::StIdle;
465     assign hw2reg.status.sha3_absorb.d = sha3_fsm ==
sha3_pkg::StAbsorb;
466     assign hw2reg.status.sha3_squeeze.d = sha3_fsm ==
sha3_pkg::StSqueeze;
467
468     // FIFO related status
469     assign hw2reg.status.fifo_depth.d[MsgFifoDepthW-1:0] =
msgfifo_depth;
470     if ($bits(hw2reg.status.fifo_depth.d) != MsgFifoDepthW) be-
gin : gen_fifo_depth_tie
471     assign
hw2reg.status.fifo_depth.d[$bits(hw2reg.status.fifo_depth.d)-1:MsgF
ifoDepthW] = '0;
472     end
473     assign hw2reg.status.fifo_empty.d = msgfifo_empty;
474     assign hw2reg.status.fifo_full.d = msgfifo_full;
475
476     // Configuration Register
477     logic engine_stable;
478     assign engine_stable = sha3_fsm == sha3_pkg::StIdle;
479
480     // SEC_CM: CFG_SHADOWED.CONFIG.REGWEN
481     assign hw2reg.cfg_regwen.d = engine_stable;
482
483     // Secret Key
484     // Secret key is defined as external register. So the logic
latches when SW
485     // writes to KEY_SHARE0 , KEY_SHARE1 registers.
486     // SEC_CM: SW_KEY.KEY_MASKING
487     always_ff @(posedge clk_i or negedge rst_ni) begin
488     if (!rst_ni) begin
489     sw_key_data_reg[0] <= '0;
490     end else if (engine_stable) begin
491     for (int j = 0 ; j < MaxKeyLen/32 ; j++) begin
492     if (reg2hw.key_share0[j].qe) begin
493     sw_key_data_reg[0][32*j+:32] <=
reg2hw.key_share0[j].q;
494     end
495     end // for j
496     end // else if engine_stable
497     end // always_ff
498
499     if (EnMasking || SwKeyMasked) begin : gen_key_share1_reg
500     always_ff @(posedge clk_i or negedge rst_ni) begin
501     if (!rst_ni) begin
502     sw_key_data_reg[1] <= '0;
503     end else if (engine_stable) begin
504     for (int j = 0 ; j < MaxKeyLen/32 ; j++) begin
505     if (reg2hw.key_share1[j].qe) begin
506     sw_key_data_reg[1][32*j+:32] <=
reg2hw.key_share1[j].q;
507     end
508     end // for j
509     end // else if engine_stable
510     end // always_ff
511     end else begin : gen_no_key_share1_reg
512     logic unused_key_share1;
513     assign unused_key_share1 = ^reg2hw.key_share1;
514     end
515
516     if (EnMasking || !SwKeyMasked) begin : gen_key_forward
517     // Forward all available key shares as is.
518     assign sw_key_data = sw_key_data_reg;
519     end else begin : gen_key_unmask
520     // Masking is disabled but the SW still provides the key
in two shares.
521     // Unmask the key for processing.
522     assign sw_key_data[0] = sw_key_data_reg[0] ^
sw_key_data_reg[1];
523     end
524
525     assign sw_key_len = key_len_e'(reg2hw.key_len.q);
526
527     // Entropy configurations
528     assign wait_timer_prescaler =
reg2hw.entropy_period.prescaler.q;
529     assign wait_timer_limit =
reg2hw.entropy_period.wait_timer.q;
530     assign entropy_refresh_req = reg2hw.cmd.entropy_req.q
531     && reg2hw.cmd.entropy_req.qe;
532     assign entropy_seed_update = reg2hw.entropy_seed.qe;
533     assign entropy_seed_data = reg2hw.entropy_seed.q;
534
535     assign entropy_hash_threshold =
reg2hw.entropy_refresh_threshold_shadowed.q;
536     assign hw2reg.entropy_refresh_hash_cnt.de = 1'b 1;
537     assign hw2reg.entropy_refresh_hash_cnt.d = en-
tropy_hash_cnt;
538

```

```

539 assign entropy_hash_clr = reg2hw.cmd.hash_cnt_clr.qe
540 && reg2hw.cmd.hash_cnt_clr.q;
541
542 // Entropy config
543 assign entropy_ready = reg2hw.cfg_shadowed.entropy_ready.q
544 & reg2hw.cfg_shadowed.entropy_ready.qe;
545 assign entropy_mode = en-
546   tropy_mode_e'(reg2hw.cfg_shadowed.entropy_mode.q);
547 assign reg_entropy_fast_process =
548   reg2hw.cfg_shadowed.entropy_fast_process.q;
549
550 // msg_mask_en turns on the message LFSR when KMAC is en-
551   abled.
552 assign cfg_msg_mask = reg2hw.cfg_shadowed.msg_mask.q;
553 assign msg_mask_en = cfg_msg_mask & msg_valid & msg_ready;
554
555 // Enable unsupported mode & strength combination
556 assign cfg_en_unsupported_modestrength =
557   reg2hw.cfg_shadowed.en_unsupported_modestrength.q;
558
559 `ASSERT(EntropyReadyLatched_A, $rose(entropy_ready) |> !en-
560   tropy_ready)
561
562 // Idle control (registered output)
563 // The logic checks idle of SHA3 engine, MSG_FIFO,
564   KMAC_CORE, KEYMGR interface
565   always_ff @(posedge clk_i or negedge rst_ni) begin
566     if (!rst_ni) begin
567       idle_o <= prim_mubi_pkg::MuBi4True;
568     end else if ((sha3_fsm == sha3_pkg::StIdle) &&
569   (msgfifo_empty || SecIdleAcceptSwMsg)) begin
570       idle_o <= prim_mubi_pkg::MuBi4True;
571     end else begin
572       idle_o <= prim_mubi_pkg::MuBi4False;
573     end
574   end
575
576 // Clear the error processed
577 assign err_processed = reg2hw.cmd.err_processed.q &
578   reg2hw.cmd.err_processed.qe;
579
580 // Make sure the field has latch in reg_top
581 `ASSERT(ErrProcessedLatched_A, $rose(err_processed) |> !
582   err_processed)
583
584 // App mode, strength, kmac_en
585 assign reg_kmac_en = reg2hw.cfg_shadowed.kmac_en.q;
586 assign reg_sha3_mode =
587   sha3_pkg::sha3_mode_e'(reg2hw.cfg_shadowed.mode.q);
588 assign reg_keccak_strength = sha3_pkg::kec-
589   cak_strength_e'(reg2hw.cfg_shadowed.kstrength.q);
590
591 // Interrupt
592
593 // Hash process absorbed interrupt
594 // Convert mubi4_t to logic to generate interrupts
595 assign event_absorbed =
596   prim_mubi_pkg::mubi4_test_true_strict(app_absorbed);
597
598 prim_intr_hw #(.Width(1)) intr_kmac_done (
599   .clk_i,
600   .rst_ni,
601   .event_intr_i (event_absorbed),
602   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.kmac_done.q),
603   .reg2hw_intr_test_q_i (reg2hw.intr_test.kmac_done.q),
604   .reg2hw_intr_test_qe_i (reg2hw.intr_test.kmac_done.qe),
605   .reg2hw_intr_state_q_i (reg2hw.intr_state.kmac_done.q),
606   .hw2reg_intr_state_de_o (hw2reg.intr_state.kmac_done.d),
607   .hw2reg_intr_state_d_o (hw2reg.intr_state.kmac_done.d),
608   .intr_o (intr_kmac_done_o)
609 );
610
611 `ASSERT(Sha3AbsorbedPulse_A,
612   $rose(prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed)) |>
613   prim_mubi_pkg::mubi4_test_false_strict(sha3_absorbed))
614
615 // Message FIFO empty interrupt
616 // The message FIFO empty interrupt is **not useful** for
617   software if:
618   // - One of the hardware application interfaces is actively
619     using the KMAC block. In this case
620
621 // the message FIFO is managed entirely by the application
622   interface.
623 // - The SHA3 core is not in the Absorb state. Only in this
624   state, the FIFO is writeable by
625   // software anyway.
626 // - Software has already written the Process command. The
627   KMAC block will now empty the
628   // message FIFO and load its content into the SHA3 core,
629   // add the padding and then perform
630   // the final absorption. Software cannot append the mes-
631     sage further.
632 //
633 // The message FIFO empty interrupt can be **useful** for
634   software in particular if:
635 // - The message FIFO was completely full previously. How-

```

```

ever, unless the KMAC block is currently
619 // processing a block or waiting for fresh entropy from
620   EDN, it always empties the message FIFO
621 // faster than software can fill it up, meaning the mes-
622   sage FIFO is empty most of the time.
623 // Note, the empty status is signaled only once after the
624   FIFO was completely full. The FIFO
625 // needs to be full again for the empty status to be
626   signaled again next time it's empty.
627
628 // For further details see also:
629 // https://opentitan.org/book/hw/ip/kmac/doc/the-
630   ory_of_operation.html#fifo-depth-and-empty-status
631 logic status_msgfifo_empty, msgfifo_empty_gate;
632 logic msgfifo_empty_negedge, msgfifo_empty_q;
633 logic msgfifo_full_seen_d, msgfifo_full_seen_q;
634 assign msgfifo_empty_negedge = msgfifo_empty_q &
635   ~msgfifo_empty;
636
637 // Track whether the message FIFO was full after being
638   empty. We clear the tracking:
639 // - When receiving the Process command. This is to start
640   over for the next message.
641 // - When seeing a negative edge on the empty signal. This
642   signals that software has reacted to
643   // the interrupt and is filling up the FIFO again.
644 assign msgfifo_full_seen_d =
645   msgfifo_full ? 1'b 1 :
646   msgfifo_empty_negedge ? 1'b 0 :
647   msgfifo2kmac_process ? 1'b 0 : msgfifo_full_seen_q;
648
649 // The interrupt is gated unless software is performing an
650   absorption operation (but not the
651   // final block) and the FIFO was full before. The msgfifo2k-
652   mac_process pulse is arriving from the
653   // FIFO together with the empty signal.
654 assign msgfifo_empty_gate =
655   app_active ? 1'b 1 :
656   sha3_fsm != sha3_pkg::StAbsorb ? 1'b 1 :
657   msgfifo2kmac_process ? 1'b 1 :
658   ~msgfifo_full_seen_q;
659
660 assign status_msgfifo_empty = msgfifo_empty_gate ? 1'b 0 :
661   msgfifo_empty;
662
663 always_ff @(posedge clk_i or negedge rst_ni) begin
664   if (!rst_ni) begin
665     msgfifo_empty_q <= 1'b 0;
666     msgfifo_full_seen_q <= 1'b 0;
667   end else begin
668     msgfifo_empty_q <= msgfifo_empty;
669     msgfifo_full_seen_q <= msgfifo_full_seen_d;
670   end
671 end
672
673 prim_intr_hw #(
674   .Width(1),
675   .IntrT("Status")
676 ) intr_fifo_empty (
677   .clk_i,
678   .rst_ni,
679   .event_intr_i (status_msgfifo_empty),
680   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.fifo_empty.q),
681   .reg2hw_intr_test_q_i (reg2hw.intr_test.fifo_empty.q),
682   .reg2hw_intr_test_qe_i (reg2hw.intr_test.fifo_empty.qe),
683   .reg2hw_intr_state_q_i (reg2hw.intr_state.fifo_empty.q),
684   .hw2reg_intr_state_de_o (hw2reg.intr_state.fifo_empty.de),
685   .hw2reg_intr_state_d_o (hw2reg.intr_state.fifo_empty.d),
686   .intr_o (intr_fifo_empty_o)
687 );
688
689 // Error
690
691 logic event_error;
692 assign event_error = sha3_err.valid | app_err.valid
693   | entropy_err.valid | errcheck-
694   er_err.valid
695   ;
696
697 // Assigning error code to the register
698 assign hw2reg.err_code.de = event_error;
699
700 always_comb begin
701   hw2reg.err_code.d = '0;
702
703   priority case (1'b 1)
704     // app_err has the highest priority. If SW issues an in-
705     correct command
706     // while app is in active state, the error from AppIntf
707     is passed
708     // through.
709     app_err.valid: begin
710       hw2reg.err_code.d = {app_err.code, app_err.info};
711     end
712
713     errchecker_err.valid: begin
714       hw2reg.err_code.d = {errchecker_err.code, errcheck-
715       er_err.info};
716     end
717
718     sha3_err.valid: begin

```

```

702     hw2reg.err_code.d = {sha3_err.code , sha3_err.info};
703 end
704
705     entropy_err.valid: begin
706         hw2reg.err_code.d = {entropy_err.code, en-
707         tropy_err.info};
708     end
709
710     msgfifo_err.valid: begin
711         hw2reg.err_code.d = {msgfifo_err.code,
712         msgfifo_err.info};
713     end
714
715     default: begin
716         hw2reg.err_code.d = '0';
717     end
718 endcase
719 end
720
721 // Counter errors
722 logic counter_error, sha3_count_error, key_index_error;
723 logic msgfifo_counter_error;
724 logic kmac_entropy_hash_counter_error;
725 logic kmac_app_counter_error;
726 assign counter_error = sha3_count_error
727 | kmac_entropy_hash_counter_error
728 | key_index_error
729 | msgfifo_counter_error
730 | kmac_app_counter_error;
731
732 assign msgfifo_counter_error = msgfifo_err.valid;
733
734 // State Errors
735 logic sparse_fsm_error;
736 logic sha3_state_error, kmac_errchk_state_error;
737 logic kmac_core_state_error, kmac_app_state_error;
738 logic kmac_entropy_state_error, kmac_state_error;
739 assign sparse_fsm_error = sha3_state_error
740 | kmac_errchk_state_error
741 | kmac_core_state_error
742 | kmac_app_state_error
743 | kmac_entropy_state_error
744 | kmac_state_error;
745
746 // Control Signal Integrity Errors
747 logic control_integrity_error;
748 logic sha3_storage_rst_error;
749 assign control_integrity_error = sha3_storage_rst_error;
750
751 prim_intr_hw #(Width(1)) intr_kmac_err (
752     .clk_i,
753     .rst_ni,
754     .event_intr_i      (event_error),
755     .reg2hw_intr_enable_q_i (reg2hw_intr_enable.kmac_err.q),
756     .reg2hw_intr_test_q_i   (reg2hw_intr_test.kmac_err.q),
757     .reg2hw_intr_test_qe_i  (reg2hw_intr_test.kmac_err.qe),
758     .reg2hw_intr_state_q_i  (reg2hw_intr_state.kmac_err.q),
759     .hw2reg_intr_state_de_o (hw2reg_intr_state.kmac_err.de),
760     .hw2reg_intr_state_d_o  (hw2reg_intr_state.kmac_err.d),
761     .intr_o                (intr_kmac_err.o)
762 );
763
764 // State Machine //
765
766 // State FF
767 *PRIM_FLOP_SPARSE_FSM(u_state_regs, kmac_st_d, kmac_st,
768 kmac_st_e, KmacIdle)
769
770 always_comb begin
771     // Default value
772     kmac_st_d = kmac_st;
773
774     entropy_in_keyblock = 1'b 0;
775     kmac_state_error = 1'b 0;
776
777     unique case (kmac_st)
778         KmacIdle: begin
779             if (kmac_cmd == CmdStart) begin
780                 // If cSHAKE turned on
781                 if (sha3_pkg::CShake == app_sha3_mode) begin
782                     kmac_st_d = KmacPrefix;
783                 end else begin
784                     // Jump to Msg feed directly
785                     kmac_st_d = KmacMsgFeed;
786                 end
787             end else begin
788                 kmac_st_d = KmacIdle;
789             end
790         end
791         KmacPrefix: begin
792             // Wait until SHA3 processes one block
793             if (sha3_block_processed) begin
794                 kmac_st_d = (app_kmac_en) ? KmacKeyBlock :
795                 KmacMsgFeed;
796             end else begin
797                 kmac_st_d = KmacPrefix;
798             end
799         end
800     endcase

```

```

799
800     KmacKeyBlock: begin
801         entropy_in_keyblock = 1'b 1;
802         if (sha3_block_processed) begin
803             kmac_st_d = KmacMsgFeed;
804         end else begin
805             kmac_st_d = KmacKeyBlock;
806         end
807     end
808
809     KmacMsgFeed: begin
810         // If absorbed, move to Digest
811         if
812         (prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed) &&
813         prim_mubi_pkg::mubi4_test_true_strict(sha3_done))
814     begin
815         // absorbed and done can be asserted at a cycle if
816         Applications have
817         requested the hash operation. kmac_app FSM issues
818         CmdDone command
819         // if it receives absorbed signal.
820         kmac_st_d = KmacIdle;
821         end else if
822         (prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed) &&
823         prim_mubi_pkg::mubi4_test_false_loose(sha3_done))
824     begin
825         kmac_st_d = KmacDigest;
826         end else begin
827             kmac_st_d = KmacMsgFeed;
828         end
829     end
830
831     KmacDigest: begin
832         // SW can manually run it, wait till done
833         if (prim_mubi_pkg::mubi4_test_true_strict(sha3_done))
834     begin
835         kmac_st_d = KmacIdle;
836         end else begin
837             kmac_st_d = KmacDigest;
838         end
839     end
840
841     KmacTerminalError: begin
842         //this state is terminal
843         kmac_st_d = KmacTerminalError;
844         kmac_state_error = 1'b 1;
845     end
846
847     default: begin
848         kmac_st_d = KmacTerminalError;
849         kmac_state_error = 1'b 1;
850     end
851 endcase
852
853 // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
854 // Unconditionally jump into the terminal error state
855 // if the life cycle controller triggers an escalation.
856
857 if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en[0]))
858 begin
859     kmac_st_d = KmacTerminalError;
860 end
861
862 `ASSERT_KNOWN(KmacStKnown_A, kmac_st)
863
864 // KMAC core
865 kmac_core #(
866     .EnMasking (EnMasking)
867 ) u_kmac_core (
868     .clk_i,
869     .rst_ni,
870
871     // from Msg FIFO
872     .fifo_valid_i (msgfifo_valid),
873     .fifo_data_i   (msgfifo_data ),
874     .fifo_strb_i   (msgfifo_strb ),
875     .fifo_ready_o  (msgfifo_ready),
876
877     // to SHA3 core
878     .msg_valid_o   (msg_valid),
879     .msg_data_o    (msg_data ),
880     .msg_strb_o    (msg_strb ),
881     .msg_ready_i   (msg_ready),
882
883     // Configurations
884     .kmac_en_i     (app_kmac_en),
885     .mode_i        (app_sha3_mode),
886     .strength_i    (app_keccak_strength),
887
888     // Secret key interface
889     .key_data_i    (key_data),
890     .key_len_i     (key_len),
891     .key_valid_i   (key_valid),
892
893     // Controls
894     .start_i       (sha3_start ),

```

```

890 .process_i (msgfifo2kmac_process),
891 .done_i (sha3_done),
892 .process_o (kmac2sha3_process),
893
894 // LC escalation
895 .lc_escalate_en_i (lc_escalate_en[1]),
896
897 // Error detection
898 .sparse_fsm_error_o (kmac_core_state_error),
899 .key_index_error_o (key_index_error)
900 );
901
902 // SHA3 hashing engine
903
904 // msg_data masking
905 if (EnMasking == 1) begin: g_msg_mask
906   logic [MsgWidth-1:0] msg_mask_permuted;
907
908   // Permute the LFSR output to avoid same lfsr applied to
multiple times
909   always_comb begin
910     msg_mask_permuted = '0;
911     for (int unsigned i = 0 ; i < MsgWidth ; i++) begin
912       // Loop through the MsgPerm constant and swap between
the bits
913         msg_mask_permuted[i] = msg_mask[RndCnstMsgPerm[i]];
914       end
915     end
916
917     for (genvar i = 0 ; i < Share ; i++) begin:
g_msg_data_mask
918       assign msg_data_masked[i] = msg_data[i]
919         ^ ({MsgWidth{cfg_msg_mask}} &
msg_mask_permuted);
920     end : g_msg_data_mask
921   end else begin : g_no_msg_mask
922     assign msg_data_masked[0] = msg_data[0];
923     assign msg_mask = '0;
924
925     logic unused_msgmask;
926     assign unused_msgmask = ^{msg_mask, cfg_msg_mask,
msg_mask_en};
927   end
928   sha3 #(
929     .EnMasking (EnMasking)
930   ) u_sha3 (
931     .clk_i,
932     .rst_ni,
933
934     // MSG_FIFO interface (or from KMAC)
935     .msg_valid_i (msg_valid),
936     .msg_data_i (msg_data_masked),
937     .msg_strb_i (msg_strb),
938     .msg_ready_o (msg_ready),
939
940     // Entropy interface
941     .rand_valid_i (sha3_rand_valid),
942     .rand_early_i (sha3_rand_early),
943     .rand_data_i (sha3_rand_data),
944     .rand_aux_i (sha3_rand_aux),
945     .rand_update_o (sha3_rand_update),
946     .rand_consumed_o (sha3_rand_consumed),
947
948     // N, S: Used in cSHAKE mode
949     .ns_data_i (ns_prefix),
950
951     // Configurations
952     .mode_i (app_sha3_mode),
953     .strength_i (app_keccak_strength),
954
955     // Controls (CMD register)
956     .start_i (sha3_start),
957     .process_i (kmac2sha3_process),
958     .run_i (sha3_run),
959     .done_i (sha3_done),
960
961     // LC escalation
962     .lc_escalate_en_i (lc_escalate_en[2]),
963
964     .absorbed_o (sha3_absorbed),
965     .squeezing_o (sha3_squeezing),
966
967     .block_processed_o (sha3_block_processed),
968
969     .sha3_fsm_o (sha3_fsm),
970
971     .state_valid_o (state_valid),
972     .state_o (state), // [Share]
973
974     // REQ/ACK interface to avoid power spikes
975     .run_req_o ( ), // Not used
976     .run_ack_i (1'b 1), // The SHA3 core is always allowed to
process.
977
978     .error_o (sha3_err),
979     .sparse_fsm_error_o (sha3_state_error),
980     .count_error_o (sha3_count_error),
981     .keccak_storage_rst_error_o (sha3_storage_rst_error)
982   );
983
984 // MSG_FIFO window interface to FIFO interface

```

```

=====
985 // Tie the read path
986 assign tlram_rvalid = 1'b 0;
987 assign tlram_rdata = '0;
988 assign tlram_rerror = '0;
989
990 // Convert endian here
991 // prim_packer always packs to the right(bit0). If the
input DWORD is
992 // big-endian, it needs to be swapped to little-endian to
maintain the
993 // order. Internal SHA3(Keccak) runs in little-endian in
contrast to HMAC
994 // So, no endian-swap after prim_packer.
995 assign tlram_wdata_endian = conv_endian32(tlram_wdata,
996
reg2hw.cfg_shadowed.msg_endianness.q);
997 assign tlram_wmask_endian = conv_endian32(tlram_wmask,
998
reg2hw.cfg_shadowed.msg_endianness.q);
999
1000 // TL Adapter
1001 tlul_adapter_sram #(
1002   .SramAw ($clog2(MsgWindowDepth)),
1003   .SramDw (MsgWindowWidth),
1004   .Outstanding (1),
1005   .ByteAccess (1),
1006   .ErrOnRead (1)
1007 ) u_tlul_adapter_msgfifo (
1008   .clk_i,
1009   .rst_ni,
1010   .en_ifetch_i (prim_mubi_pkg::MuBi4False),
1011   .tl_i (tl_win_h2d[WinMsgFifo]),
1012   .tl_o (tl_win_d2h[WinMsgFifo]),
1013
1014   .req_o (tlram_req),
1015   .req_type_o ( ),
1016   .gnt_i (tlram_gnt),
1017   .we_o (tlram_we),
1018   .addr_o (tlram_addr),
1019   .wdata_o (tlram_wdata),
1020   .wmask_o (tlram_wmask),
1021   .intg_error_o ( ),
1022   .user_rsvd_o ( ),
1023   .rdata_i (tlram_rdata),
1024   .rvalid_i (tlram_rvalid),
1025   .rerror_i (tlram_rerror),
1026   .compound_txn_in_progress_o ( ),
1027   .readback_en_i (prim_mubi_pkg::MuBi4False),
1028   .readback_error_o ( ),
1029   .wr_collision_i (1'b0),
1030   .write_pending_i (1'b0)
1031 );
1032
1033 assign sw_msg_valid = tlram_req & tlram_we ;
1034 if (MsgWidth == MsgWindowWidth) begin : gen_sw_msg_samewidth
1035   assign sw_msg_data = tlram_wdata_endian ;
1036   assign sw_msg_strb = tlram_wmask_endian ;
1037 end else begin : gen_sw_msg_diff
1038   assign sw_msg_data = {{MsgWidth-MsgWindowWidth{1'b0}},
tlram_wdata_endian};
1039   assign sw_msg_strb = {{MsgWidth-MsgWindowWidth{1'b0}},
tlram_wmask_endian};
1040 end
1041 assign tlram_gnt = sw_msg_ready ;
1042
1043 logic unused_tlram_addr;
1044 assign unused_tlram_addr = &{1'b0, tlram_addr};
1045
1046 // Application interface Mux/Demux
1047 kmac_app #(
1048   .EnMasking(EnMasking),
1049   .SecIdleAcceptSwMsg(SecIdleAcceptSwMsg),
1050   .NumAppIntf(NumAppIntf),
1051   .AppCfg(AppCfg)
1052 ) u_app_intf (
1053   .clk_i,
1054   .rst_ni,
1055
1056   .reg_key_data_i (sw_key_data),
1057   .reg_key_len_i (sw_key_len),
1058
1059   .reg_prefix_i (reg_ns_prefix),
1060
1061   .reg_kmac_en_i (reg_kmac_en),
1062   .reg_sha3_mode_i (reg_sha3_mode),
1063   .reg_keccak_strength_i (reg_keccak_strength),
1064
1065   // data from tl_adapter
1066   .sw_valid_i (sw_msg_valid),
1067   .sw_data_i (sw_msg_data),
1068   .sw_strb_i (sw_msg_strb),
1069   .sw_ready_o (sw_msg_ready),
1070
1071   // KeyMgr sideloaded key interface
1072   .keymgr_key_i,
1073
1074   // Application data in / digest out interface
1075   .app_i,
1076   .app_o,

```

```

1077
1078 // Secret Key output to KMAC Core
1079 .key_data_o (key_data),
1080 .key_len_o (key_len),
1081 .key_valid_o (key_valid),
1082
1083 // to MSG_FIFO
1084 .kmac_valid_o (mux2fifo_valid),
1085 .kmac_data_o (mux2fifo_data),
1086 .kmac_strb_o (mux2fifo_strb),
1087 .kmac_ready_i (mux2fifo_ready),
1088 .kmac_bypass_fifo_o(mux2fifo_bypass),
1089
1090 // to KMAC Core
1091 .kmac_en_o (app_kmac_en),
1092
1093 // to SHA3 Core
1094 .sha3_prefix_o (ns_prefix),
1095 .sha3_mode_o (app_sha3_mode),
1096 .keccak_strength_o (app_keccak_strength),
1097
1098 // Keccak state from SHA3 core
1099 .keccak_state_valid_i (state_valid),
1100 .keccak_state_i (state),
1101
1102 // to STATE TL Window
1103 .reg_state_valid_o (reg_state_valid),
1104 .reg_state_o (reg_state),
1105
1106 // Configuration: Sideloaded Key
1107 .keymgr_key_en_i (reg2hw_cfg_shadowed.sideload.q),
1108
1109 .absorbed_i (sha3_absorbed), // from SHA3
1110 .squeezing_i (sha3_squeezing), // from SHA3
1111 .absorbed_o (app_absorbed), // to SW
1112
1113 .app_active_o (app_active),
1114
1115 .entropy_fast_process_i (reg_entropy_fast_process),
1116 .entropy_fast_process_o (entropy_fast_process),
1117
1118 .error_i (sha3_err.valid),
1119 .err_processed_i (err_processed),
1120
1121 .clear_after_error_o (clear_after_error),
1122
1123 // Command interface
1124 .sw_cmd_i (checked_sw_cmd),
1125 .cmd_o (kmac_cmd),
1126
1127 // Status
1128 .entropy_ready_i (entropy_configured),
1129
1130 // LC escalation
1131 .lc_escalate_en_i (lc_escalate_en[3]),
1132
1133 // Error report
1134 .error_o (app_err),
1135 .sparse_fsm_error_o (kmac_app_state_error),
1136 .counter_error_o (kmac_app_counter_error)
1137 );
1138
1139 // Message FIFO
1140 kmac_msgfifo #(
1141   .OutWidth (kmac_pkg::MsgWidth),
1142   .MsgDepth (kmac_pkg::MsgFifoDepth),
1143   .EnMasking (EnMasking)
1144 ) u_msgfifo (
1145   .clk_i,
1146   .rst_ni,
1147
1148   .fifo_valid_i (mux2fifo_valid),
1149   .fifo_data_i (mux2fifo_data),
1150   .fifo_strb_i (mux2fifo_strb),
1151   .fifo_ready_o (mux2fifo_ready),
1152   .fifo_bypass_i (mux2fifo_bypass),
1153
1154   .msg_valid_o (msgfifo_valid),
1155   .msg_data_o (msgfifo_data),
1156   .msg_strb_o (msgfifo_strb),
1157   .msg_ready_i (msgfifo_ready),
1158
1159   .fifo_empty_o (msgfifo_empty), // intr and status
1160   .fifo_full_o (msgfifo_full), // connected to status only
1161   .fifo_depth_o (msgfifo_depth),
1162
1163   .clear_i (sha3_done),
1164
1165   .process_i (reg2msgfifo_process),
1166   .process_o (msgfifo2kmac_process),
1167
1168   .err_o (msgfifo_err)
1169 );
1170
1171 logic [sha3_pkg::StateW-1:0] reg_state_tl [Share];
1172 always_comb begin
1173   for (int i = 0 ; i < Share; i++) begin
1174     reg_state_tl[i] = reg_state_valid ? reg_state[i] : '0;
1175   end
1176 end
1177
1178 // State (Digest) reader
1179 kmac_staterd #(
1180   .AddrW (9), // 512B
1181   .EnMasking (EnMasking)
1182 ) u_staterd (
1183   .clk_i,
1184   .rst_ni,
1185
1186   .tl_i (tl_win_h2d[WinState]),
1187   .tl_o (tl_win_d2h[WinState]),
1188
1189   .state_i (reg_state_tl),
1190
1191   .endian_swap_i (reg2hw_cfg_shadowed.state_endianness.q)
1192 );
1193
1194 // Error checker
1195 kmac_errchk #(
1196   .EnMasking (EnMasking)
1197 ) u_errchk (
1198   .clk_i,
1199   .rst_ni,
1200
1201   // Configurations
1202   .cfg_mode_i (reg_sha3_mode),
1203   .cfg_strength_i (reg_keccak_strength),
1204
1205   .kmac_en_i (reg_kmac_en),
1206   .cfg_prefix_6B_i (reg_ns_prefix[47:0]), // first 6B of PRE-
1207   FIX
1208   .cfg_en_unsupported_modestrength_i (cfg_en_unsupported_modestrength),
1209
1210   .entropy_ready_pulse_i (entropy_ready),
1211
1212   // SW commands
1213   .sw_cmd_i (sw_cmd),
1214   .sw_cmd_o (checked_sw_cmd),
1215
1216   // Status from KMAC_APP
1217   .app_active_i (app_active),
1218
1219   // Status from SHA3 core
1220   .sha3_absorbed_i (sha3_absorbed),
1221   .keccak_done_i (sha3_block_processed),
1222
1223   // LC escalation
1224   .lc_escalate_en_i (lc_escalate_en[4]),
1225
1226   .err_processed_i (err_processed),
1227   .clear_after_error_i (clear_after_error),
1228
1229   .error_o (errchecker_err),
1230   .sparse_fsm_error_o (kmac_errchk_state_error)
1231 );
1232
1233 // Entropy Generator
1234 if (EnMasking == 1) begin : gen_entropy
1235   logic entropy_req, entropy_ack;
1236   logic [edn_pkg::ENDPOINT_BUS_WIDTH-1:0] entropy_data;
1237   logic unused_entropy_fips;
1238
1239   // Synchronize EDN interface
1240   prim_sync_reqack_data #(
1241     .Width (edn_pkg::ENDPOINT_BUS_WIDTH),
1242     .DataSrc2Dst (1'b0),
1243     .DataReg (1'b0)
1244   ) u_prim_sync_reqack_data (
1245     .clk_src_i (clk_i),
1246     .rst_src_ni (rst_ni),
1247     .clk_dst_i (clk_edn_i),
1248     .rst_dst_ni (rst_edn_ni),
1249     .req_chk_i (1'b1),
1250     .src_req_i (entropy_req),
1251     .src_ack_o (entropy_ack),
1252     .dst_req_o (entropy_o.edn_req),
1253     .dst_ack_i (entropy_i.edn_ack),
1254     .data_i (entropy_i.edn_bus),
1255     .data_o (entropy_data)
1256   );
1257
1258   // We don't track whether the entropy is pre-FIPS or not
1259   inside KMAC
1260     assign unused_entropy_fips = entropy_i.edn_fips;
1261
1262   kmac_entropy #(
1263     .RndCnstLfsrcPerm (RndCnstLfsrcPerm),
1264     .RndCnstLfsrcSeed (RndCnstLfsrcSeed),
1265     .RndCnstBufferLfsrcSeed (RndCnstBufferLfsrcSeed)
1266   ) u_entropy (
1267     .clk_i,
1268     .rst_ni,
1269
1270     // EDN interface
1271     .entropy_req_o (entropy_req),
1272     .entropy_ack_i (entropy_ack),
1273     .entropy_data_i (entropy_data),
1274
1275     // Entropy to internal logic (DOM AND)

```



```

1276 .rand_valid_o    (sha3_rand_valid),
1277 .rand_early_o    (sha3_rand_early),
1278 .rand_data_o     (sha3_rand_data),
1279 .rand_aux_o      (sha3_rand_aux),
1280 .rand_update_i   (sha3_rand_update),
1281 .rand_consumed_i (sha3_rand_consumed),
1282
1283 // Status from internal logic
1284 //// KMAC secret block handling indicator
1285 .in_keyblock_i (entropy_in_keyblock),
1286
1287 // Configuration
1288 .mode_i        (entropy_mode),
1289 .entropy_ready_i (entropy_ready),
1290 .fast_process_i (entropy_fast_process),
1291
1292 //// Entropy refresh period in clk cycles
1293 .wait_timer_prescaler_i (wait_timer_prescaler),
1294 .wait_timer_limit_i     (wait_timer_limit),
1295
1296 //// Message Masking
1297 .msg_mask_en_i (msg_mask_en),
1298 .msg_mask_o    (msg_mask),
1299
1300 //// SW update of seed
1301 .seed_update_i (entropy_seed_update),
1302 .seed_data_i   (entropy_seed_data),
1303 .entropy_refresh_req_i (entropy_refresh_req),
1304
1305 // Status
1306 .hash_cnt_o    (entropy_hash_cnt),
1307 .hash_cnt_clr_i (entropy_hash_clr),
1308 .hash_threshold_i (entropy_hash_threshold),
1309
1310 .entropy_configured_o (entropy_configured),
1311
1312 // LC escalation
1313 .lc_escalate_en_i (lc_escalate_en[5]),
1314
1315 // Error
1316 .err_o    (entropy_err),
1317 .sparse_fsm_error_o (kmac_entropy_state_error),
1318 .count_error_o    (kmac_entropy_hash_counter_error),
1319 .err_processed_i   (err_processed)
1320 );
1321 end else begin : gen_empty_entropy
1322 // If Masking is not used, no need of entropy. Ignore in-
puts and config; tie output to 0.
1323 edn_pkg::edn_rsp_t unused_entropy_input;
1324 entropy_mode_e     unused_entropy_mode;
1325 logic              unused_entropy_fast_process;
1326
1327 assign unused_entropy_input = entropy_i;
1328 assign unused_entropy_mode = entropy_mode;
1329 assign unused_entropy_fast_process = entropy_fast_process;
1330
1331 assign entropy_o = '{default: '0};
1332
1333 logic unused_sha3_rand_update;
1334 logic unused_sha3_rand_consumed;
1335 assign sha3_rand_valid = 1'b 1;
1336 assign sha3_rand_early = 1'b 1;
1337 assign sha3_rand_data = '0;
1338 assign sha3_rand_aux = '0;
1339 assign unused_sha3_rand_update = sha3_rand_update;
1340 assign unused_sha3_rand_consumed = sha3_rand_consumed;
1341
1342 logic unused_seed_update;
1343 logic unused_seed_data;
1344 logic unused_refresh_period;
1345 logic unused_entropy_refresh_req;
1346 assign unused_seed_data = ^entropy_seed_data;
1347 assign unused_seed_update = entropy_seed_update;
1348 assign unused_refresh_period = ^{wait_timer_limit,
wait_timer_prescaler};
1349 assign unused_entropy_refresh_req = entropy_refresh_req;
1350
1351 logic unused_entropy_hash;
1352 assign unused_entropy_hash = ^{entropy_hash_clr, en-
tropy_hash_threshold};
1353 assign entropy_hash_cnt = '0;
1354
1355 assign entropy_err = '{valid: 1'b 0, code: ErrNone, info:
'0};
1356
1357 assign kmac_entropy_state_error = 1'b 0;
1358 assign kmac_entropy_hash_counter_error = 1'b 0;
1359
1360 logic unused_entropy_status;
1361 assign unused_entropy_status = entropy_in_keyblock;
1362
1363 // If Masking is off, always entropy configured
1364 assign entropy_configured = prim_mubi_pkg::MuBi4True;
1365
1366 logic unused_edn_clk_rst;
1367 assign unused_edn_clk_rst = ^{clk_edn_i, rst_edn_ni};
1368
1369 logic unused_lc_escalate_en5;
1370 assign unused_lc_escalate_en5 = ^lc_escalate_en[5];
1371 end
1372

```

```

1373 // MUBI4 buf
1374 prim_mubi4_sender #(
1375   .AsyncOn (0)
1376 ) u_sha3_done_sender (
1377   .clk_i,
1378   .rst_ni,
1379   .mubi_i (sha3_done_d),
1380   .mubi_o (sha3_done)
1381 );
1382
1383 // Register top
1384 logic [NumAlerts-1:0] alert_test, alerts, alerts_q;
1385
1386 logic shadowed_storage_err, shadowed_update_err;
1387 kmac_reg_top u_reg (
1388   .clk_i,
1389   .rst_ni,
1390   .rst_shadowed_ni,
1391
1392   .tl_i,
1393   .tl_o,
1394
1395   .tl_win_o (tl_win_h2d),
1396   .tl_win_i (tl_win_d2h),
1397
1398   .reg2hw,
1399   .hw2reg,
1400
1401   // SEC_CM: CFG.SHADOWED.CONFIG.SHADOW
1402   .shadowed_storage_err_o (shadowed_storage_err),
1403   .shadowed_update_err_o  (shadowed_update_err),
1404   // SEC_CM: BUS.INTEGRITY
1405   .intg_err_o              (alert_intg_err)
1406 );
1407
1408 logic unused_cfg_shadowed_qe;
1409 assign unused_cfg_shadowed_qe = ^{
1410   reg2hw.cfg_shadowed.kmac_en.qe
1411   , reg2hw.cfg_shadowed.kstrength.qe
1412   , reg2hw.cfg_shadowed.mode.qe
1413   , reg2hw.cfg_shadowed.msg_endianness.qe
1414   , reg2hw.cfg_shadowed.state_endianness.qe
1415   , reg2hw.cfg_shadowed.side_load.qe
1416   , reg2hw.cfg_shadowed.entropy_mode.qe
1417   , reg2hw.cfg_shadowed.entropy_fast_process.qe
1418   , reg2hw.cfg_shadowed.msg_mask.qe
1419   , reg2hw.cfg_shadowed.en_unsupported_modestrength.qe
1420 };
1421
1422 // Alerts
1423 assign alert_test = {
1424   reg2hw.alert_test.fatal_fault_err.q
1425   & reg2hw.alert_test.fatal_fault_err.qe, // [1]
1426   reg2hw.alert_test.recov_operation_err.q
1427   & reg2hw.alert_test.recov_operation_err.qe // [0]
1428 };
1429
1430 assign alerts = {
1431   alert_fatal, // Alerts[1]
1432   alert_recov_operation // Alerts[0]
1433 };
1434
1435 assign alert_recov_operation = shadowed_update_err;
1436
1437 // The recoverable alert is observable via status register
until the KMAC operation is restarted
1438 // by re-writing the Control Register.
1439 logic status_alert_recov_ctrl_update_err;
1440 always_ff @(posedge clk_i or negedge rst_ni) begin
1441   if (!rst_ni) begin
1442     status_alert_recov_ctrl_update_err <= 1'b 0;
1443   end else if (alert_recov_operation) begin
1444     status_alert_recov_ctrl_update_err <= 1'b 1;
1445   end else if (err_processed) begin
1446     status_alert_recov_ctrl_update_err <= 1'b 0;
1447   end
1448 end
1449
1450 assign hw2reg.status.alert_recov_ctrl_update_err.d =
status_alert_recov_ctrl_update_err;
1451
1452 assign alert_fatal = shadowed_storage_err
1453   | alert_intg_err
1454   | sparse_fsm_error
1455   | counter_error
1456   | control_integrity_error
1457 ;
1458
1459 // Make the fatal alert observable via status register.
1460 // Cannot be reset except the hardware reset
1461 logic status_alert_fatal_fault;
1462
1463 always_ff @(posedge clk_i or negedge rst_ni) begin
1464   if (!rst_ni) begin
1465     status_alert_fatal_fault <= 1'b 0;
1466   end else if (alert_fatal) begin
1467     status_alert_fatal_fault <= 1'b 1;
1468   end
1469 end
1470 assign hw2reg.status.alert_fatal_fault.d =
status_alert_fatal_fault;

```

```

1471
1472 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
1473     prim_alert_sender #(
1474         .AsyncOn(AlertAsyncOn[i]),
1475         .SkewCycles(AlertSkewCycles),
1476         .IsFatal(i)
1477     ) u_prim_alert_sender (
1478         .clk_i,
1479         .rst_ni,
1480         .alert_test_i ( alert_test[i] ),
1481         .alert_req_i ( alerts[i] ),
1482         .alert_ack_o ( ),
1483         .alert_state_o ( ),
1484         .alert_rx_i ( alert_rx_i[i] ),
1485         .alert_tx_o ( alert_tx_o[i] )
1486     );
1487 end
1488
1489 // Below assumes NumAlerts == 2
1490 `ASSERT_INIT(NumAlerts2_A, NumAlerts == 2)
1491
1492 always_ff @(posedge clk_i or negedge rst_ni) begin
1493     // break up the combinatorial path for local escalation
1494     if (!rst_ni) begin
1495         alerts_q[1] <= 1'b0;
1496     end else if (alerts[1]) begin
1497         // fatal alerts cannot be cleared
1498         alerts_q[1] <= 1'b1;
1499     end
1500 end
1501
1502 always_ff @(posedge clk_i or negedge rst_ni) begin
1503     // break up the combinatorial path for local escalation
1504     if (!rst_ni) begin
1505         alerts_q[0] <= 1'b0;
1506     end else begin
1507         // recoverable alerts can be cleared so just latch the
1508         alerts_q[0] <= alerts[0];
1509     end
1510 end
1511
1512 // Latched recoverable alert[0] is not used. Rather removing
1513 // keep alert_q[1:0] and make alert_q[0] unused (lint
1514 // waive).
1515 logic unused_alerts_q0;
1516 assign unused_alerts_q0 = alerts_q[0];
1517
1518 // SEC_CM: LC_ESCALATE_EN.INTERSIG.MUBI, FSM.GLOBAL_ESC,
1519 FSM.LOCAL_ESC
1520 lc_ctrl_pkg::lc_tx_t alert_to_lc_tx;
1521 assign alert_to_lc_tx = lc_c-
1522 trl_pkg::lc_tx_bool_to_lc_tx(alerts_q[1]);
1523 for (genvar i = 0; i < NumLcSyncCopies; i++) begin :
1524 gen_or_alert_lc_sync
1525     assign lc_escalate_en[i] = lc_c-
1526 trl_pkg::lc_tx_or_hi(alert_to_lc_tx, lc_escalate_en_sync[i]);
1527 end
1528
1529 // Synchronize life cycle input
1530 prim_lc_sync #(
1531     .NumCopies (NumLcSyncCopies)
1532 ) u_prim_lc_sync (
1533     .clk_i,
1534     .rst_ni,
1535     .lc_en_i ( lc_escalate_en_i ),
1536     .lc_en_o ( lc_escalate_en_sync )
1537 );
1538
1539 assign en_masking_o = EnMasking;
1540
1541 // Assertions //
1542 // Assert known for output values
1543 `ASSERT_KNOWN(KmacDone_A, intr_kmac_done_o)
1544 `ASSERT_KNOWN(FifoEmpty_A, intr_fifo_empty_o)
1545 `ASSERT_KNOWN(KmacErr_A, intr_kmac_err_o)
1546 `ASSERT_KNOWN(Tl0dValidKnown_A, tl_o.d_valid)
1547 `ASSERT_KNOWN(Tl0AReadyKnown_A, tl_o.a_ready)
1548 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
1549 `ASSERT_KNOWN(EnMaskingKnown_A, en_masking_o)
1550
1551 // Parameter as desired
1552 `ASSERT_INIT(SecretKeyDivideBy32_A, (kmac_pkg::MaxKeyLen %
1553 32) == 0)
1554
1555 // Command input should be sparse
1556 `ASSUME(CmdSparse_M, reg2hw.cmd.cmd.qe |-> reg2hw.cmd.cmd.q
1557 inside {CmdStart, CmdProcess,
1558 CmdManualRun, CmdDone, CmdNone})
1559
1560 // redundant counter error
1561 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SentMsgCountCheck_A,
1562 u_sha3.u_pad.u_sentmsg_count,
1563 alert_tx_o[1])

```

```

1559 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RoundCountCheck_A,
1560 u_sha3.u_keccak.u_round_count,
1561 alert_tx_o[1])
1562
1563 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(KeyIndexCountCheck_A,
1564 u_kmac_core.u_key_index_count,
1565 alert_tx_o[1])
1566
1567 `AS-
1568 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(AppDigestCountCheck_A,
1569 u_app_intf.u_digest_part_counter,
1570 alert_tx_o[1])
1571
1572 // Sparse FSM state error
1573 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacCoreFsmCheck_A,
1574 u_kmac_core.u_state_regs, alert_tx_o[1])
1575 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacAppFsmCheck_A,
1576 u_app_intf.u_state_regs, alert_tx_o[1])
1577 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3FsmCheck_A,
1578 u_sha3.u_state_regs, alert_tx_o[1])
1579 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3padFsmCheck_A,
1580 u_sha3.u_pad.u_state_regs, alert_tx_o[1])
1581 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KeccakFsmCheck_A,
1582 u_sha3.u_keccak.u_state_regs,
1583 alert_tx_o[1])
1584 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(ErrorCheckFsmCheck_A,
1585 u_errchk.u_state_regs, alert_tx_o[1])
1586 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacFsmCheck_A,
1587 u_state_regs, alert_tx_o[1])
1588
1589 // prim is only instantiated if masking is enabled
1590 if (EnMasking == 1) begin : g_testassertion
1591     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(EntropyFsmCheck_A,
1592 gen_entropy.u_entropy.u_state_regs,
1593 alert_tx_o[1])
1594
1595     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(HashCountCheck_A,
1596 gen_entropy.u_entropy.u_hash_count,
1597 alert_tx_o[1])
1598
1599 // MsgFifo.Packer
1600 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(
1601     PackerCountCheck_A,
1602     u_msgfifo.u_packer.g_pos_dupcnt.u_pos,
1603     alert_tx_o[1]
1604 )
1605
1606 // MsgFifo.Fifo
1607 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(
1608     MsgFifoWptrCheck_A,
1609     u_msgfifo.u_msgfifo.gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_wp
1610 tr,
1611 alert_tx_o[1]
1612 )
1613 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(
1614     MsgFifoRptrCheck_A,
1615     u_msgfifo.u_msgfifo.gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_rp
1616 tr,
1617 alert_tx_o[1]
1618 )
1619 end
1620
1621 // Alert assertions for reg_we onehot check
1622 `AS-
1623 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
1624 u_reg, alert_tx_o[1])
1625 endmodule

```

4. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_app.sv (1,401 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // KMAC Application interface
7 // This module implements the app interface which arbitrates
8 // between the SW interface and up to
9 // NumAppIntf hardware app interfaces. While a session is act-
10 // ive (either an app or SW), other
11 // requests are stalled.
12 //
13 // There are two kind of app interfaces: Static interfaces
14 // which have a configuration defined at
15 // compile-time and only a one-shot digest can be retrieved. A
16 // dynamic interface can send the
17 // desired hashing configuration at run time and supports XOF
18 // operation.
19
20 include "prim_assert.sv"
21
22 module kmac_app
23 import kmac_pkg::*;
24 #(
25     // App specific configs are defined in kmac_pkg
26     parameter bit EnMasking = 1'b0,

```



```

22  localparam int      Share          = (EnMasking) ?
2 : 1, // derived parameter
23  parameter bit      SecIdleAcceptSwMsg = 1'b0,
24  parameter int unsigned NumAppIntf = 4,

25  parameter app_config_t AppCfg[NumAppIntf] = '{AppCfgKeyMgr,
AppCfgLcCtrl,
26                                     AppCfgRomC-
trl, AppCfg0tbn}
27  ) (
28  input clk_i,
29  input rst_ni,
30
31  // Secret Key from register
32  input [MaxKeyLen-1:0] reg_key_data_i[Share],
33  input key_len_e      reg_key_len_i,
34
35  // Prefix from register
36  input [sha3_pkg::NSRegisterSize*8-1:0] reg_prefix_i,
37
38  // mode, strength, kmac_en from register
39  input logic          reg_kmac_en_i,
40  input sha3_pkg::sha3_mode_e reg_sha3_mode_i,
41  input sha3_pkg::keccak_strength_e reg_keccak_strength_i,
42
43  // Data from Software
44  input          sw_valid_i,
45  input [MsgWidth-1:0] sw_data_i,
46  input [MsgWidth-1:0] sw_strb_i,
47  output logic      sw_ready_o,
48
49  // KeyMgr Sideload Key interface
50  input keymgr_pkg::hw_key_req_t keymgr_key_i,
51
52  // Application Message in/ Digest out interface + control
signals
53  input app_req_t [NumAppIntf-1:0] app_i,
54  output app_rsp_t [NumAppIntf-1:0] app_o,
55
56  // to KMAC Core: Secret key
57  output logic [MaxKeyLen-1:0] key_data_o[Share],
58  output key_len_e      key_len_o,
59  output logic          key_valid_o,
60
61  // to MSG_FIFO
62  output logic          kmac_valid_o,
63  output logic [MsgWidth-1:0] kmac_data_o[Share],

64  // This strobe is on bit level for the packer. The FIFO will
then convert it again to byte level.
65  output logic [MsgWidth-1:0] kmac_strb_o,
66  input logic          kmac_ready_i,
67  output logic          kmac_bypass_fifo_o,
68
69  // Signal to KMAC core that a KMAC operation is ongoing.
70  output logic kmac_en_o,
71
72  // To Sha3 Core
73  output logic [sha3_pkg::NSRegisterSize*8-1:0] sha3_prefix_o,
74  output sha3_pkg::sha3_mode_e      sha3_mode_o,
75  output sha3_pkg::keccak_strength_e      kec-
cak_strength_o,
76
77  // The current keccak state from SHA3 Core
78  input          keccak_state_valid_i,
79  input [sha3_pkg::StateW-1:0] keccak_state_i[Share],
80
81  // The keccak state is exposed to the STATE TL-window if no
application is active. If a key from
82  // the KeyMgr is used, the capacity region is zeroed. If an
app is active the state reads as
83  // zero.
84  output logic          reg_state_valid_o,
85  output logic [sha3_pkg::StateW-1:0] reg_state_o[Share],
86
87  // Controls for SW whether to take the key from the KeyMgr
sideload interface or registers. For
88  // KMAC operations initiated by an app interface, we always
take the sideloaded key.
89  // If 1, the key for KMAC is taken from the KeyMgr sideload
interface.
90  // If 0, the key is taken from the registers.
91  input logic keymgr_key_en_i,
92
93  // Command from software
94  input kmac_cmd_e sw_cmd_i,
95
96  // Status signals from SHA3 core
97  input prim_mubi_pkg::mubi4_t absorbed_i,
98  input logic          squeezing_i,
99
100 // to KMAC
101 output kmac_cmd_e cmd_o,
102
103 // to SW
104 output prim_mubi_pkg::mubi4_t absorbed_o,
105
106 // To status
107 output logic app_active_o,
108
109 // The entropy_fast_process bit is forced to 0 when recover-

```

```

ing an error state.
110 input logic entropy_fast_process_i,
111 output logic entropy_fast_process_o,
112
113 // Entropy must be ready before a KMAC operation can be per-
formed.
114 input prim_mubi_pkg::mubi4_t entropy_ready_i,
115
116 // Error input

117 // This error comes from KMAC/SHA3 engine and is pulsed if a
wrong command sequence is detected

118 // or kept high as long as any control signal has an invalid
value (e.g. if the done mubi signal
119 // is attacked to enforce a known digest value).

120 // This error is reported to the ERR_CODE regardless whether
app or SW is active.
121 input error_i,
122
123 // Pulsed when SW acknowledges error by writing to the
err_processed bit. Lets the interface
124 // return to idle after errors.
125 input err_processed_i,
126
127 output prim_mubi_pkg::mubi4_t clear_after_error_o,
128
129 // Reports error and its type if an FIFO or FSM error oc-
curred.
130 output kmac_pkg::err_t error_o,
131
132 // Life cycle
133 input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
134
135 // Fatal errors
136 output logic sparse_fsm_error_o,
137 output logic counter_error_o
138 );
139
140 // Create a lint error to reduce the risk of accidentally
enabling this feature.
141 `ASSERT_STATIC_LINT_ERROR(KmacSecIdleAcceptSwMsgNonDefault,
SecIdleAcceptSwMsg == 0)

142
143 import sha3_pkg::KeccakBitCapacity;
144 import sha3_pkg::L128;
145 import sha3_pkg::L224;
146 import sha3_pkg::L256;
147 import sha3_pkg::L384;
148 import sha3_pkg::L512;

149
150 ///////////////////////////////////////////////////////////////////
151 // Definitions //
152 ///////////////////////////////////////////////////////////////////

153
154 localparam int KeyMgrKeyW = $bits(keymgr_key_i.key[0]);
155
156 localparam key_len_e KeyLengths [5] = '{Key128, Key192,
Key256, Key384, Key512};

157
158 localparam int SelKeySize = (AppKeyW == 128) ? 0 :
(AppKeyW == 192) ? 1 :
(AppKeyW == 256) ? 2 :
(AppKeyW == 384) ? 3 :
(AppKeyW == 512) ? 4 : 0 ;
159
160 localparam int SelDigSize = (AppDigestW == 128) ? 0 :
(AppDigestW == 192) ? 1 :
(AppDigestW == 256) ? 2 :
(AppDigestW == 384) ? 3 :
(AppDigestW == 512) ? 4 : 0 ;
161
162 // Key length is always the same as the full app digest
width.
163 localparam key_len_e SideloadedKey = KeyLengths[SelKeySize];
164
165 // Define right_encode(outlen) value here
166 // Look at kmac_pkg::key_len_e for the kinds of key size
167 //

174 // These values should be exactly the same as the key length
encodings
175 // in kmac_core.sv, with the only difference being that the
byte representing
176 // the byte-length of the encoded value is in the MSB posi-
tion due to right encoding
177 // instead of in the LSB position (left encoding).
178 localparam int OutLenW = 24;
179 localparam logic [OutLenW-1:0] EncodedOutLen [5] = '{
180 24'h 0001_80, // Key128
181 24'h 0001_C0, // Key192
182 24'h 02_0001, // Key256
183 24'h 02_8001, // Key384
184 24'h 02_0002 // Key512
185 };
186
187 localparam logic [OutLenW-1:0] EncodedOutLenStrb [5] = '{
188 24'h 00FFFF, // Key128,
189 24'h 00FFFF, // Key192
190 24'h FFFFFFF, // Key256
191 24'h FFFFFFF, // Key384
192 24'h FFFFFFF // Key512

```

```

193 };
194
195 // Signals //
196 // Signals //
197 // Signals //
198
199 st_e st, st_d;
200
201 logic keymgr_key_used;
202
203 // Response channel signals
204
205 // The state machine controls the datapath mux selection and
handles the requests and responses.
206 app_rsp_t app_rsp;
207 logic app_data_ready, app_req_ready, app_error_req_ready;
208 logic app_digest_valid;
209 logic app_finish_rsp_valid, app_error_rsp_valid;
210 logic app_finish_rsp_is_error;
211 logic [AppDigestW-1:0] app_digest[2];
212
213 localparam int unsigned AppIdxW = $clog2(NumAppIntf);
214
215 // app_id indicates, which app interface was chosen. various
logic use this
216 // value to get the config or return the data.
217 logic [AppIdxW-1:0] app_id, app_id_d;
218 logic clr_appid, set_appid;
219
220 // Output length
221 logic [OutLenW-1:0] encoded_outlen, encoded_outlen_strb;
222
223 // state output
224 // Mux selection signal
225 app_mux_sel_e mux_sel;
226 app_mux_sel_e mux_sel_buf_output;
227 app_mux_sel_e mux_sel_buf_err_check;
228 app_mux_sel_e mux_sel_buf_kmac;
229
230 // Tracking and error signals
231 kmac_pkg::err_t fsm_err, mux_err;
232 logic err_sha3_during_app_set, err_sha3_during_app_d,
err_sha3_during_app_q;
233
234 // Application arbitration //
235 // Application arbitration //
236 // Latch the selected app ID when FSM grants a request. Per
default select ID 0. The FSM and
237 // datapath ensure that no data is leaked if interface is
not active.
238 always_ff @(posedge clk_i or negedge rst_ni) begin
239   if (!rst_ni) app_id <= AppIdxW'(0);
240   else if (clr_appid) app_id <= AppIdxW'(0);
241   else if (set_appid) app_id <= app_id_d;
242 end
243
244 // The arbitration uses a fixed priority. The assumption is
that the requests normally do no
245 // collide. If this assumption doesn't hold anymore, con-
sider RR arbiter.
246 logic [NumAppIntf-1:0] app_req_valids;
247 logic [$clog2(NumAppIntf)-1:0] arb_idx;
248 logic arb_valid;
249 logic arb_ready;
250
251 // Pick all request valid signals from the interfaces
252 always_comb begin
253   app_req_valids = '0;
254   for (int unsigned i = 0; i < NumAppIntf; i++) begin
255     app_req_valids[i] = app_i[i].req_valid;
256   end
257 end
258
259 prim_arbiter_fixed #(
260   .N (NumAppIntf),
261   .DW(1),
262   .EnDataPort(1'b 0)
263 ) u_appid_arb (
264   .clk_i,
265   .rst_ni,
266
267   .req_i (app_req_valids),
268   .data_i ('{default:'0}),
269   .gnt_o (), // not used
270   .idx_o (arb_idx),
271
272   .valid_o (arb_valid),
273   .data_o (), // not used
274   .ready_i (arb_ready)
275 );
276
277 assign app_id_d = AppIdxW'(arb_idx);
278 assign arb_ready = set_appid;
279
280 // App config selection //
281 // App config selection //
282
283 // Select the new configuration from either the compile-time

```

```

defined configuration or from the
284 // supplied session configuration. This operates on the non-
latched app ID so the configuration
285 // can be latched at the same time the ID is.
286 app_config_t app_cfg;
287 app_ses_config_t app_ses_cfg_raw;
288 app_ses_config_t app_ses_cfg_pending;
289 app_ses_config_t app_ses_cfg_d;
290 app_ses_config_t app_ses_cfg_q;
291
292 // Note, the parsed value of prefix_mode is not actually
used but just here to have a complete
293 // type. It is overwritten below.
294 assign app_ses_cfg_raw = app_ses_con-
fig_t'(app_i[arb_idx].data_s0[$bits(app_ses_config_t)-1:0]);
295
296 always_comb begin
297   app_ses_cfg_pending.mode = Ap-
pcfg[arb_idx].session_cfg.mode;
298   app_ses_cfg_pending.kstrength = Ap-
pcfg[arb_idx].session_cfg.kstrength;
299   app_ses_cfg_pending.en_xof = Ap-
pcfg[arb_idx].session_cfg.en_xof;
300   app_ses_cfg_pending.prefix_mode = Ap-
pcfg[arb_idx].session_cfg.prefix_mode;
301
302 // Override configuration with configuration supplied by
dynamic interface.
303 if (AppCfg[arb_idx].if_type == AppDynamic) begin
304   app_ses_cfg_pending = app_ses_cfg_raw;
305
306 // For KMAC always use prefix defined at compile time.
This saves the prefix check.
307 // Otherwise use prefix from CSR.
308 app_ses_cfg_pending.prefix_mode = app_ses_cfg_pend-
ing.mode == AppKMAC ? 1'b1 : 1'b0;
309 end
310 end
311
312 // KMAC en / SHA3 mode / Strength / configuration latching.
313 logic kmac_en_d, kmac_en_q;
314 sha3_pkg::sha3_mode_e sha3_mode_d, sha3_mode_q;
315 sha3_pkg::keccak_strength_e keccak_strength_d, kec-
cak_strength_q;
316
317 always_comb begin
318   app_ses_cfg_d = app_ses_cfg_q;
319   kmac_en_d = kmac_en_q;
320   sha3_mode_d = sha3_mode_q;
321   keccak_strength_d = keccak_strength_q;
322
323 if (clr_appid) begin
324   // When an app finishes, latch values from CSRs.
325   app_ses_cfg_d = AppSesCfgDefault;
326   kmac_en_d = reg_kmac_en_i;
327   sha3_mode_d = reg_sha3_mode_i;
328   keccak_strength_d = reg_keccak_strength_i;
329 end else if (set_appid) begin
330   // When an app starts, latch its configuration.
331   app_ses_cfg_d = app_ses_cfg_pending;
332   kmac_en_d = app_ses_cfg_pending.mode == App-
KMAC ? 1'b1 : 1'b0;
333   // KMAC is based upon CShake
334   sha3_mode_d = app_ses_cfg_pending.mode == App-
SHA3 ? sha3_pkg::Sha3 :
335   app_ses_cfg_pending.mode == App-
Shake ? sha3_pkg::Shake : sha3_pkg::CShake;
336   keccak_strength_d = app_ses_cfg_pending.kstrength;
337 end else if (st == StIdle) begin
338   // In idle always propagate the latest CSR values as
there is no latch trigger when SW starts
339   // a hashing operation.
340   app_ses_cfg_d = AppSesCfgDefault;
341   kmac_en_d = reg_kmac_en_i;
342   sha3_mode_d = reg_sha3_mode_i;
343   keccak_strength_d = reg_keccak_strength_i;
344 end
345 end
346
347 always_ff @(posedge clk_i or negedge rst_ni) begin
348   if (!rst_ni) begin
349     app_ses_cfg_q <= AppSesCfgDefault;
350     kmac_en_q <= 1'b 0;
351     sha3_mode_q <= sha3_pkg::Sha3;
352     keccak_strength_q <= sha3_pkg::L256;
353   end else begin
354     app_ses_cfg_q <= app_ses_cfg_d;
355     kmac_en_q <= kmac_en_d;
356     sha3_mode_q <= sha3_mode_d;
357     keccak_strength_q <= keccak_strength_d;
358   end
359 end
360
361 assign kmac_en_o = kmac_en_q;
362 assign sha3_mode_o = sha3_mode_q;
363 assign keccak_strength_o = keccak_strength_q;
364
365 // Force the entropy_fast_process bit to 0 when recovering
from error.
366 logic disable_entropy_fast_process;
367 assign entropy_fast_process_o = dis-

```

```

able_entropy_fast_process ? 1'b0 : entropy_fast_process_i;
368
369 // Construct the complete configuration for the active app
from the static and latched
370 // session configuration.
371 always_comb begin
372     app_cfg = AppCfg[app_id];
373     app_cfg.session_cfg = app_cfg.session_cfg;
374 end
375
376 //////////////////////////////////////////////////
377 // App config checker //
378 //////////////////////////////////////////////////
379 logic valid_app_sha3_strength;
380 logic valid_app_shake_strength;
381 logic valid_app_kmac_cfg;
382 logic valid_app_mode_strength_raw;
383 logic valid_app_mode_strength;
384 logic valid_en_xof_cfg;
385 logic valid_app_cfg;
386
387 assign valid_app_sha3_strength = app_cfg.session_cfg.kstrength inside {sha3_pkg::L224,
388
389     sha3_pkg::L256,
390
391     sha3_pkg::L384,
392
393     sha3_pkg::L512};
394
395 assign valid_app_shake_strength = app_cfg.session_cfg.kstrength inside {sha3_pkg::L128,
396
397     sha3_pkg::L256};
398
399 assign valid_app_mode_strength_raw =
400     app_cfg.session_cfg.mode == App-
SHA3 ? valid_app_sha3_strength :
401     app_cfg.session_cfg.mode inside {AppShake, AppCShake,
AppKMAC} ? valid_app_shake_strength :
402     1'b0;
403
404 // Ignore the mode and strength check if app allows unsupported combinations.
405
406 assign valid_app_mode_strength = valid_app_mode_strength_raw
|| app_cfg.en_unsup_comb;
407
408 assign valid_app_kmac_cfg = app_cfg.session_cfg.mode == App-
KMAC ?
409
prim_mubi_pkg::mubi4_test_true_strict(entropy_ready_i) : 1'b1;
410
411 // XOF operation is not supported for SHA3 or KMAC.
412 assign valid_en_xof_cfg = app_cfg.session_cfg.mode inside
{AppSHA3, AppKMAC} ?
413     app_cfg.session_cfg.en_xof ==
1'b0 : 1'b1;
414
415 assign valid_app_cfg = valid_app_mode_strength && valid-
id_en_xof_cfg && valid_app_kmac_cfg;
416
417 // A compile-time defined configuration must always result
in a valid mode-strength and en_xof
418 // configuration.
419 `ASSERT(ConfigAlwaysValidIfStatic_A,
420     app_active_o && app_cfg.if_type == AppStatic
421     |-> valid_app_mode_strength && valid_en_xof_cfg)
422
423 //////////////////////////////////////////////////
424 // Application Mux / Demux //
425 //////////////////////////////////////////////////
426 // The active app's request and response channel is muxed
here from / to local signals.
427 app_req_t app_req;
428 assign app_req = app_i[app_id];
429
430 // There are three ready sources which can accept a request.
The app_data_ready active in the
431 // message phase and is driven by the FIFO. The
app_req_ready is controlled by the FSM to accept
432 // the control requests (start/config, process, and termina-
tion). The app_error_req_ready is used
433 // to accept messages and control requests during the error
handling.
434 // Similarly, there are three response generators. One
drives digest responses, one the finish
435 // response and one the error responses.
436 assign app_rsp = '{
437     req_ready: app_data_ready | app_error_req_ready |
app_req_ready,
438     rsp_valid: app_digest_valid | app_finish_rsp_valid |
app_error_rsp_valid,
439     digest_s0: app_digest[0],

```

```

440     digest_s1: app_digest[1],
441     error: app_error_rsp_valid | app_finish_rsp_is_error
|
442 // Must use _d as error is relevant in cycle
the digest is returned.
443 (err_sha3_during_app_d & app_cfg.if_type ==
AppStatic),
444     rsp_finish: app_finish_rsp_valid
445 };
446
447 // The three response generators may never be active at the
same time because otherwise responses
448 // would collide.
449 `ASSERT(AppOnlyOneRspSourceActive_A,
450     $onehot0({app_digest_valid, app_finish_rsp_valid,
app_error_rsp_valid}))
451
452 always_comb begin
453     for (int unsigned i = 0; i < NumAppIntf; i++) begin
454         if (i == app_id) begin
455             app_o[i] = app_rsp;
456         end else begin
457             app_o[i] = APP_RSP_DEFAULT;
458         end
459     end
460 end
461
462 //////////////////////////////////////////////////
463 // FSM //
464 //////////////////////////////////////////////////
465 logic clear_app_trackers;
466
467 logic any_request_outstanding;
468 logic last_req_pending;
469 logic process_cmd_pending;
470 logic last_req_hs;
471
472 logic last_msg_part_received_d, last_msg_part_received_q;
473 logic last_msg_part_received_set;
474
475 logic process_cmd_sent_d, process_cmd_sent_q;
476 logic process_cmd_sent_set;
477
478 logic pending_digest_rsp_d, pending_digest_rsp_q;
479 logic pending_error_rsp_d, pending_error_rsp_q;
480
481 logic digest_parts_available;
482 logic squeeze_again;
483 logic app_push_digest;
484 logic clear_digest_pusher;
485
486 logic service_rejected_error_d, service_rejected_error_q;
487 logic service_rejected_error_set;
488
489 logic err_key_used_but_invalid;
490 logic err_key_used_but_invalid_set;
491 logic err_key_used_but_invalid_clear;
492 logic err_key_used_but_invalid_d,
err_key_used_but_invalid_q;
493
494 logic err_during_sw_d, err_during_sw_q;
495 logic err_during_sw_set;
496 logic err_processed_d, err_processed_q;
497
498 // State register
499 `PRIM_FLOP_SPARSE_FSM(u_state_regs, st_d, st, st_e, StIdle)
500
501 // Next State & output logic
502 // SEC_CM: FSM.SPARSE
503 always_comb begin
504     st_d = st;
505
506     mux_sel = SecIdleAcceptSwMsg ? SelSw : SelNone;
507
508 // app_id control
509 set_appid = 1'b0;
510 clr_appid = 1'b0;
511
512 // Command towards the KMAC core
513 cmd_o = CmdNone;
514
515 // Software output
516 absorbed_o = prim_mubi_pkg::MuBi4False;
517
518 // Bypass FIFO if masked app is active.
519 kmac_bypass_fifo_o = 1'b0;
520
521 // Ready signal to handshake requests for dynamic inter-
faces.
522 app_req_ready = 1'b0;
523
524 // Control signals for the digest pusher.
525 app_push_digest = 1'b0;
526 clear_digest_pusher = 1'b0;
527
528 // Finish response
529 app_finish_rsp_valid = 1'b0;
530 app_finish_rsp_is_error = 1'b0;
531
532 // Error tracking and handling
533 fsm_err = '{valid: 1'b0, code: Er-
rNone, info: '0};

```

```

528 sparse_fsm_error_o      = 'b0';
529 clear_after_error_o    =
prim_mubi_pkg::MuB14False;
530 service_rejected_error_set = 'b0';
531 err_sha3_during_app_set   = 'b0';
532 err_key_used_but_invalid_clear = 'b0';
533 process_cmd_sent_set     = 'b0';
534 last_msg_part_received_set = 'b0';
535 err_during_sw_set        = 'b0';
536 clear_app_trackers       = 'b0';
537 app_error_req_ready      = 'b0';
538 app_error_rsp_valid      = 'b0';
539
540 disable_entropy_fast_process = 'b0';
541
542 unique case (st)
543   StIdle: begin
544     clear_app_trackers = 'b1';
545
546     if (arb_valid) begin
547       // An app was chosen, latch the ID.
548       st_d = StAppCfg;
549       set_appid = 'b1';
550     end else if (sw_cmd_i == CmdStart) begin
551       // Software initiates the sequence
552       st_d = StSw;
553       cmd_o = CmdStart;
554     end
555   end
556
557   StAppCfg: begin
558     if (!valid_app_cfg) begin
559       // Either the entropy source was not marked as ready
560       // for a KMAC operation, or the configuration in
561       // the registers supplied by SW is
562       // invalid. In this error case we still go to the
563       // "message absorb" phase but no data is
564       // forwarded to the actual SHA3 core. This simpli-
565       // fies the error handling on the
566       // application side.
567       st_d = StErrorAwaitMsg;
568       service_rejected_error_set = 'b1';
569     end else begin
570       // The configuration is valid and also latched. We
571       // can now send the start command and
572       // begin to absorb the message.
573       st_d = StAppMsg;
574       cmd_o = CmdStart;
575     end
576   end
577
578   // Handshake the configuration request also if there
579   // is an error. This is required as an
580   // application interface still must send the message.
581
582   // For static interfaces we stall the first request as
583   // it already contains data.
584   app_req_ready = app_cfg.if_type == AppDynamic;
585   end
586
587   StAppMsg: begin
588     mux_sel = SelApp;
589     kmac_bypass_fifo_o = app_cfg.masked;
590
591     // Accept any process command immediately and do not
592     // forward the request to the message
593     // FIFO.
594     app_req_ready = process_cmd_pending;
595     last_msg_part_received_set = last_req_hs;
596
597     if (last_req_hs) begin
598       if (app_cfg.session_cfg.mode == AppKMAC) begin
599         st_d = StAppOutLen;
600       end else begin
601         st_d = StAppProcess;
602       end
603     end else if (err_sha3_during_app_q) begin
604       // Handshaking last message part has priority as
605       // this error is also checked afterwards.
606       st_d = StErrorAwaitMsg;
607     end
608   end
609
610   StAppOutLen: begin
611     mux_sel = SelOutLen;
612     kmac_bypass_fifo_o = app_cfg.masked;
613
614     if (kmac_valid_o && kmac_ready_i) begin
615       st_d = StAppProcess;
616     end else begin
617       st_d = StAppOutLen;
618     end
619   end
620
621   StAppProcess: begin
622     cmd_o = CmdProcess;
623     st_d = StAppWait;
624
625     // Bypass must be stable until process command has
626     // been sent.
627     kmac_bypass_fifo_o = app_cfg.masked;

```

```

615 process_cmd_sent_set = 'b1';
616 end
617
618 StAppWait: begin
619   // absorbed_i is pulsed once when the first process
620   // command ends. squeezing_i is high once
621   // the processing or subsequent squeezes have fin-
622   // ished. This is only relevant for dynamic
623   // interfaces.
624   if (prim_mubi_pkg::mubi4_test_true_strict(absorbed_i)
625   ||
626       (squeezing_i && app_cfg.if_type == AppDynamic))
627   begin
628     st_d = StAppPushDigest;
629     clear_digest_pusher = 'b1';
630   end
631 end
632
633 StAppPushDigest: begin
634   // Static interface:
635   // - Return full digest in one response and return to
636   // idle (via finish).
637   // Dynamic interface:
638   // - Push the available digest / rate in parts.
639   // - For SHAKE and CSHAKE, if digest / rate is fully
640   // pushed, trigger a squeeze.
641   // - End the session if a termination request arrives.
642   app_push_digest = 'b1';
643
644   if (app_cfg.if_type == AppStatic) begin
645     if (app_rsp.rsp_valid && app_req.rsp_ready) begin
646       // Must be on _d signal to capture the error in
647       // the same cycle.
648       st_d = err_sha3_during_app_d ? StErrorAwaitSw :
649       StAppFinish;
650     end
651   end else begin
652     // Ending a session by handshaking the request takes
653     // priority over 1) handling a SHA3
654     // error as it is checked for when sending the re-
655     // sponse and 2) squeezing to avoid
656     // starting an obsolete squeeze operation.
657     if (last_req_pending) begin
658       st_d = StAppFinish;
659       app_req_ready = 'b1';
660     end else if (err_sha3_during_app_q) begin
661       st_d = StErrorPush;
662     end else if (squeeze_again && !pending_digest_rsp_d)
663   begin
664     // Trigger a squeeze if there should be sent more
665     // digest parts. Ensure that there
666     // is no pending response which would be 'killed'
667     // when changing the state.
668     cmd_o = CmdManualRun;
669     st_d = StAppWait;
670   end
671 end
672
673 StAppFinish: begin
674   if (app_cfg.if_type == AppStatic) begin
675     // Immediately end the session for static inter-
676     // faces.
677     st_d = StIdle;
678     cmd_o = CmdDone;
679     clr_appid = 'b1';
680   end else begin
681     // Await handshake of pending response from last
682     // cycle(s) (digest or error response).
683     // Otherwise the finish response would violate the
684     // valid locked-in principle.
685     app_push_digest = pending_digest_rsp_q;
686     app_error_rsp_valid = pending_error_rsp_q;
687     if (!pending_digest_rsp_q && !pending_error_rsp_q)
688   begin
689     // We now can send the finish response. Send again
690     // the error flag to cover the case the
691     // error occurred whilst the last digest response
692     // was pending.
693     app_finish_rsp_valid = 'b1';
694     app_finish_rsp_is_error = err_sha3_during_app_q;
695
696     // Once the finish response is handshaked the ses-
697     // sion can be terminated.
698     if (app_finish_rsp_valid && app_req.rsp_ready) be-
699   gin
700     st_d = StIdle;
701     cmd_o = CmdDone;
702     clr_appid = 'b1';
703     if (err_sha3_during_app_q) begin
704       clear_after_error_o =
prim_mubi_pkg::MuB14True;
705     end
706   end
707 end
708 end
709 end

```

```

691   StSw: begin
692     mux_sel = SelSw;
693
694     cmd_o = sw_cmd_i;
695     absorbed_o = absorbed_i;
696
697     if (sw_cmd_i == CmdDone) begin
698       st_d = StIdle;
699     end else begin
700       st_d = StSw;
701     end
702   end
703
704   StErrorKeyNotValid: begin
705     // Signal the error to SW and start the error recov-
706     // ery. This state won't accept any
707     // requests, so we cannot miss the last message re-
708     // quest.
709     st_d = StErrorAwaitMsg;
710
711     // Clear the error as we now start handling it.
712     err_key_used_but_invalid_clear = 1'b1;
713
714     // Report the error to SW.
715     fsm_err.valid = 1'b 1;
716     fsm_err.code = ErrKeyNotValid;
717     fsm_err.info = 24'(app_id);
718   end
719
720   StErrorAwaitMsg: begin
721     if (err_during_sw_q) begin
722       // Only wait for SW to ack the error.
723       st_d = StErrorAwaitSw;
724     end else begin
725       // Track if the last message arrives.
726       last_msg_part_received_set = last_req_hs;
727
728       // Once the last message arrived, send error re-
729       // sponse in next cycle. We must use the _q
730       // signal to avoid 1) a timing loop with the ready
731       // and 2) handshaking a pending
732       // termination request in case we enter this error
733       // state after the message has already
734       // been received.
735       if (last_msg_part_received_q) begin
736         st_d = StErrorNotify;
737       end else begin
738         // Continue to absorb data or a process command on
739         // the app interface.
740         app_error_req_ready = 1'b1;
741       end
742     end
743   end
744
745   StErrorNotify: begin
746     // Send error response back to app. Once response is
747     // handshaked, wait for termination
748     // request. Static interfaces don't wait for a termin-
749     // ation request and skip this state.
750     app_error_rsp_valid = 1'b1;
751
752     if (app_rsp.rsp_valid && app_req.rsp_ready) begin
753       st_d = app_cfg.if_type == AppDynamic ? StErrorAwait-
754       Termination : StErrorFinish;
755     end
756   end
757
758   StErrorAwaitTermination: begin
759     // A state entered only for dynamic interfaces. Wait
760     // for the app to send a termination
761     // request.
762     if (app_req.req_valid && app_req.req_last) begin
763       st_d = StErrorFinish;
764       app_req_ready = 1'b1;
765     end
766   end
767
768   StErrorFinish: begin
769     // Send finish response (dynamic) or exit immediately
770     // (static).
771
772     // For service-rejected errors, return to idle without
773     // SW interaction.
774     // For other errors, wait for SW to acknowledge the
775     // error.
776     // A SHA3 error can occur after the service rejected
777     // error is latched. Thus only send an
778     // error-free finish response if the only error case
779     // is a service rejected error.
780     app_finish_rsp_valid = app_cfg.if_type == AppDynam-
781     ic;
782
783     app_finish_rsp_is_error = !service_rejected_error_q ||
784     err_sha3_during_app_q;
785
786     if ((app_finish_rsp_valid && app_req.rsp_ready) ||
787     (app_cfg.if_type == AppStatic)) begin
788       if (!app_finish_rsp_is_error) begin
789         clr_appid = 1'b1;

```

```

771     clear_after_error_o = prim_mubi_pkg::MuBi4True;
772     st_d = StIdle;
773   end else begin
774     st_d = StErrorAwaitSw;
775   end
776 end
777
778 StErrorAwaitSw: begin
779   // Wait for SW to have processed the error.
780   if (err_processed_q) begin
781     // Flush the message FIFO and let the SHA3 engine
782     // compute a digest (which won't be used
783     // but serves to bring the SHA3 engine back to the
784     // idle state). Skip the command if
785     // error occurred after regular PROCESS command has
786     // been sent and the processing is
787     // already ongoing or has already finished. Note,
788     // the process command is not tracked
789
790     // during SW operation to have the same behaviour as
791     // the KMAC before the extended app
792     // interface was added.
793     st_d = StErrorAwaitAbsorbed;
794
795     if (!process_cmd_sent_q) begin
796       cmd_o = CmdProcess;
797     end
798   end
799
800   StErrorAwaitAbsorbed: begin
801     // Wait until the hashing engine has finished comput-
802     // ing the garbage digest. Use the
803     // squeezing_i signal to cover the case when the en-
804     // gine finishes before we wait for it.
805     if (squeezing_i) begin
806       // Clear internal variables, send done command, and
807       // return to idle.
808       clr_appid = 1'b1;
809       clear_after_error_o = prim_mubi_pkg::MuBi4True;
810       cmd_o = CmdDone;
811       st_d = StIdle;
812
813       // If error originated from SW, report 'absorbed' to
814       // SW.
815       if (err_during_sw_q) begin
816         absorbed_o = prim_mubi_pkg::MuBi4True;
817       end
818     end
819
820     StErrorPush: begin
821       // State for dynamic interfaces only.
822       // An error occurred while pushing digest parts. Keep
823       // sending error responses until the
824       // app sends a termination request. Then close the
825       // session normally.
826       // Continue sending pending response to adhere to val-
827       // id locked-in principle.
828       if (pending_digest_rsp_q) begin
829         app_push_digest = 1'b1;
830       end else begin
831         // Now continuously send error responses until ter-
832         // mination request arrives.
833         app_error_rsp_valid = 1'b1;
834         if (app_req.req_valid && app_req.req_last) begin
835           app_req_ready = 1'b1;
836           st_d = StAppFinish;
837         end
838       end
839     end
840
841     StTerminalError: begin
842       // this state is terminal
843       st_d = st;
844       sparse_fsm_error_o = 1'b 1;
845       fsm_err.valid = 1'b 1;
846       fsm_err.code = ErrFatalError;
847       fsm_err.info = 24'(app_id);
848     end
849
850     default: begin
851       st_d = StTerminalError;
852       sparse_fsm_error_o = 1'b 1;
853     end
854   endcase
855
856   // Error out if the key is used but it is invalid. Do not
857   // transition if:
858   // - A request is outstanding (valid request but no hand-
859   // shake in this cycle). Otherwise a valid
860
861   // could be dropped which could bring the hashing engine
862   // in a non recoverable state.
863
864   // - When the key error handling starts. Otherwise the FSM
865   // would get trapped.
866   // Note, the key error is cleared when the error handling

```



```

starts.
851   if (err_key_used_but_invalid && !any_request_outstanding
&& st != StErrorKeyNotValid) begin
852     st_d = StErrorKeyNotValid;
853     // Track if error occurred during SW.
854     err_during_sw_set = st == StSw;
855   end
856
857   // Track whether a hashing engine error occurred.
858   err_sha3_during_app_set =
859     error_i &&
860     (st inside {StAppCfg, StAppMsg, StAppOutLen, StAppPro-
cess, StAppWait, StAppPushDigest});
861
862   // Enforce the entropy_fast_process bit is set to 0 for
the duration of the error handling.
863   // This ensures the hashing updates the PRNG and hence the
masking is fully enabled. This
864   // in turn ensures that no improperly masked hashing oper-
ation can be triggered by an
865   // adversary.
866   disable_entropy_fast_process =
867     st inside {StErrorKeyNotValid, StErrorAwaitMsg, StEr-
rorNotify, StErrorAwaitTermination,
868               StErrorFinish, StErrorAwaitSw, StError-
AwaitAbsorbed};
869
870   // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
871   // Unconditionally jump into the terminal error state if
the life cycle controller triggers an
872   // escalation.
873   if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
begin
874     st_d = StTerminalError;
875   end
876 end
877
878 assign any_request_outstanding = app_req.req_valid && !
app_rsp.req_ready;
879
880 assign last_req_pending = app_req.req_valid &&
app_req.req_last;
881 // An empty message request issues a process command and
does not forward any message data.
882 assign process_cmd_pending = last_req_pending &&
app_req.strb == '0;
883 assign last_req_hs = last_req_pending &&
app_rsp.req_ready;
884
885 // Track when last message part is handshake. Note, this
also tracks process commands.
886 assign last_msg_part_received_d =
887   clear_app_trackers ?
1'b0 : // clear
888   last_msg_part_received_set ?
1'b1 : // set
889
last_msg_part_received_q; // hold
890
891 assign process_cmd_sent_d =
892   clear_app_trackers ? 1'b0 : // clear
893   process_cmd_sent_set ? 1'b1 : // set
894   process_cmd_sent_q; // hold
895
896 // Track when SW acknowledges the error. Only set flag if
this bit is relevant for the app
897 // interface, i.e., a hashing operation is ongoing (this in-
cludes SW- and app-triggered
898 // operations).
899 assign err_processed_d =
900   clear_app_trackers ?
1'b0 : // clear
901   (err_processed_i && (st != StIdle)) ?
1'b1 : // set
902
err_pro-
cessed_q; // hold
903
904 // Track errors occurring in SW mode.
905 assign err_during_sw_d = clear_app_trackers ? 1'b0 : //
clear
906   err_during_sw_set ? 1'b1 : // set
907   err_during_sw_q; // hold
908
909 // Track hashing engine errors
910 assign err_sha3_during_app_d = clear_app_trackers ?
1'b0 : // clear
911   err_sha3_during_app_set ?
1'b1 : // set
912
err_sha3_during_app_q; // hold
913
914 // Track key invalid errors
915 assign err_key_used_but_invalid_d =
clear_app_trackers ? 1'b0 : // clear
916
err_key_used_but_invalid_clear ? 1'b0 : // clear when handled
917
err_key_used_but_invalid_set ? 1'b1 : // set
918
err_key_used_but_invalid_q; // hold

```

```

919
920 // Factor out the clearing as it would affect the state
transition logic but we want all
921 // state dependent transition logic inside the FSM.
922 assign err_key_used_but_invalid =
err_key_used_but_invalid_set || err_key_used_but_invalid_q;
923
924 // Track service rejected errors
925 assign service_rejected_error_d =
clear_app_trackers ? 1'b0 : // clear
926   ser-
vice_rejected_error_set ? 1'b1 : // set
927
service_rejected_error_q; // hold
928
929 always_ff @(posedge clk_i or negedge rst_ni) begin
930   if (!rst_ni) begin
931     last_msg_part_received_q <= 1'b0;
932     process_cmd_sent_q <= 1'b0;
933     pending_digest_rsp_q <= 1'b0;
934     pending_error_rsp_q <= 1'b0;
935     service_rejected_error_q <= 1'b0;
936     err_processed_q <= 1'b0;
937     err_during_sw_q <= 1'b0;
938     err_sha3_during_app_q <= 1'b0;
939     err_key_used_but_invalid_q <= 1'b0;
940   end else begin
941     last_msg_part_received_q <= last_msg_part_received_d;
942     process_cmd_sent_q <= process_cmd_sent_d;
943     pending_digest_rsp_q <= pending_digest_rsp_d;
944     pending_error_rsp_q <= pending_error_rsp_d;
945     service_rejected_error_q <= service_rejected_error_d;
946     err_processed_q <= err_processed_d;
947     err_during_sw_q <= err_during_sw_d;
948     err_sha3_during_app_q <= err_sha3_during_app_d;
949     err_key_used_but_invalid_q <=
err_key_used_but_invalid_d;
950   end
951 end
952
953 // Check that dynamic app only states are never entered for
a static app.
954 `ASSERT(InvalidStateForStaticApp_A,
955         st inside {StErrorPush, StErrorAwaitTermination} |>
(app_cfg.if_type == AppDynamic),
956         clk_i, rst_ni)
957
958 ///////////////////////////////////////////////////
959 // Datapath //
960 ///////////////////////////////////////////////////
961
962 // Encoded output length for the KMAC operation. The length
is based upon the full app interface
963 // response width.
964 assign encoded_outlen = EncodedOutLen[SelDigSize];
965 assign encoded_outlen_strb = EncodedOutLenStrb[SelDigSize];
966
967 // Data mux
968 // This is the main datapath part of the app interface lo-
gic.
969 // Once the FSM selected an interface and checked its con-
figuration the message FIFO interface is
970 // exposed to the interface. For a KMAC operation, as soon
as the app has sent the last message
971 // the FSM pushes the encoded output length into the message
FIFO. This is a predefined value,
972 // see 'EncodeOutLen' parameter above.
973 //
974 // The data is forwarded depending on the masking paramet-
ers:
975 // EnMasking = 1:
976 // - For static and dynamic interface:
977 //   - Masked = 0: Forward share 0. Set share 1 to '0.
978 //   - Masked = 1: Forward both shares.
979 // EnMasking = 0
980 // - For static and dynamic interface:
981 //   - Masked = 0: Forward share 0. Ignore share 1.
982 //   - Masked = 1: Forward XOR of both shares.
983 always_comb begin
984   app_data_ready = 1'b 0;
985
986   // Always accept a SW request even if SW is not active to
avoid deadlocking the software. If
987   // SW is not active, this request is ignored and will
trigger an error (handled outside).
988   sw_ready_o = 1'b 1;
989
990   kmac_valid_o = 1'b 0;
991   kmac_data_o = '{default: '0;
992   kmac_strb_o = '0;
993
994   unique case (mux_sel_buf_kmac)
995     SelApp: begin
996       // Forward message only if it contains data. An empty
message / process command is not
997       // forwarded.
998       kmac_valid_o = app_req.req_valid && !pro-
cess_cmd_pending;

```

```

999     if (EnMasking) begin
1000         kmac_data_o[0] = app_req.data_s0;
1001         kmac_data_o[1] = app_cfg.masked ? app_req.data_s1 :
'0';
1002     end else begin
1003         // If the interface is masked but the KMAC core not,
then we unmask the input data.
1004         kmac_data_o[0] = app_cfg.masked ? app_req.data_s0 ^
app_req.data_s1 : app_req.data_s0;
1005     end
1006     // Expand byte strobe to bits for the prim_packer in-
side MSG_FIFO.
1007     for (int i = 0; i < MsgStrbW; i++) begin
1008         kmac_strb_o[8 * i +: 8] = {8{app_req.strb[i]}};
1009     end
1010     app_data_ready = kmac_ready_i;
1011 end
1012
1013 SelOutLen: begin
1014     // Write encoded output length value as unmasked data
(share 1 = '0').
1015     kmac_valid_o = 1'b1; // always write
1016     kmac_data_o[0] = MsgWidth'(encoded_outlen);
1017     kmac_strb_o = MsgWidth'(encoded_outlen_strb);
1018 end
1019
1020 SelSw: begin
1021     // SW supports only one share
1022     kmac_valid_o = sw_valid_i;
1023     kmac_data_o[0] = sw_data_i;
1024     for (int i = 1; i < Share; i++) begin
1025         kmac_data_o[i] = '0';
1026     end
1027     kmac_strb_o = sw_strb_i;
1028     sw_ready_o = kmac_ready_i;
1029 end
1030
1031 default: begin // Incl. SelNone
1032     kmac_valid_o = 1'b0;
1033     kmac_data_o = '{default: '0}';
1034     kmac_strb_o = '0';
1035 end
1036
1037 endcase
1038 end
1039
1040 // Error checking for Mux
1041 always_comb begin
1042     mux_err = '{valid: 1'b0, code: ErrNone, info: '0};
1043
1044     if (mux_sel_buf_err_check != SelSw && sw_valid_i) begin
1045         // If SW writes message into FIFO
1046         mux_err = '{
1047             valid: 1'b1,
1048             code: ErrSwPushedMsgFifo,
1049             info: 24'({8'h00, 8'(st), 8'(mux_sel_buf_err_check)})
1050         };
1051     end else if (app_active_o && sw_cmd_i != CmdNone) begin
1052         // If SW issues command except start
1053         mux_err = '{
1054             valid: 1'b1,
1055             code: ErrSwIssuedCmdInAppActive,
1056             info: 24'(sw_cmd_i)
1057         };
1058     end
1059 end
1060
1061 logic [AppMuxWidth-1:0] mux_sel_buf_output_logic;
1062 assign mux_sel_buf_output = app_mux_sel_e'(mux_sel_buf_out-
put_logic);
1063
1064 // SEC_CM: LOGIC.INTEGRITY
1065 prim_sec_anchor_buf #(
1066     .Width(AppMuxWidth)
1067 ) u_prim_buf_state_output_sel (
1068     .in_i(mux_sel),
1069     .out_o(mux_sel_buf_output_logic)
1070 );
1071
1072 logic [AppMuxWidth-1:0] mux_sel_buf_err_check_logic;
1073 assign mux_sel_buf_err_check =
app_mux_sel_e'(mux_sel_buf_err_check_logic);
1074
1075 // SEC_CM: LOGIC.INTEGRITY
1076 prim_sec_anchor_buf #(
1077     .Width(AppMuxWidth)
1078 ) u_prim_buf_state_err_check (
1079     .in_i(mux_sel),
1080     .out_o(mux_sel_buf_err_check_logic)
1081 );
1082
1083 logic [AppMuxWidth-1:0] mux_sel_buf_kmac_logic;
1084 assign mux_sel_buf_kmac = app_mux_sel_e'(mux_sel_buf_k-
mac_logic);
1085
1086 // SEC_CM: LOGIC.INTEGRITY
1087 prim_sec_anchor_buf #(
1088     .Width(AppMuxWidth)
1089 ) u_prim_buf_state_kmac_sel (
1090     .in_i(mux_sel),

```

```

1091     .out_o(mux_sel_buf_kmac_logic)
1092 );
1093
1094 // SEC_CM: LOGIC.INTEGRITY
1095 logic reg_state_valid;
1096 prim_sec_anchor_buf #(
1097     .Width(1)
1098 ) u_prim_buf_state_output_valid (
1099     .in_i(reg_state_valid),
1100     .out_o(reg_state_valid_o)
1101 );
1102
1103 // Keccak state Demux
1104 // The state is only exposed to the registers if SW is act-
ive.
1105 always_comb begin
1106     reg_state_valid = 1'b0;
1107     reg_state_o = '{default: '0};
1108
1109     if ((mux_sel_buf_output == SelSw) &&
1110         lc_c-
1111         trl_pkg::lc_tx_test_false_strict(lc_escalate_en_i)) begin
1112         reg_state_valid = keccak_state_valid_i;
1113         reg_state_o = keccak_state_i;
1114         // If key is sideloaded and KMAC is SW initiated hide
the capacity from SW by zeroing.
1115         // See https://github.com/lowRISC/opentitan/issues/
17508.
1116         if (keymgr_key_en_i) begin
1117             for (int i = 0; i < Share; i++) begin
1118                 unique case (keccak_strength_q)
1119                     L128: reg_state_o[i]
[sha3_pkg::StateW-1:KeccakBitCapacity[L128]] = '0;
1120                     L224: reg_state_o[i]
[sha3_pkg::StateW-1:KeccakBitCapacity[L224]] = '0;
1121                     L256: reg_state_o[i]
[sha3_pkg::StateW-1:KeccakBitCapacity[L256]] = '0;
1122                     L384: reg_state_o[i]
[sha3_pkg::StateW-1:KeccakBitCapacity[L384]] = '0;
1123                     L512: reg_state_o[i]
[sha3_pkg::StateW-1:KeccakBitCapacity[L512]] = '0;
1124                     default: reg_state_o[i] = '0;
1125                 endcase
1126             end
1127         end
1128     end
1129
1130     ////////////
1131     // Digest pusher //
1132     ////////////
1133     // The maximal number of digest chunks is defined by the op-
eration with the largest rate. The
1134     // bit-width of the rate for any XOF algorithm can be com-
puted with: StateW - 2 * Strength.
1135     // And as SHA3, SHAKE, cSHAKE and KMAC have always a state
width of 1600, the largest rate has
1136     // SHAKE-128 / cSHAKE-128 / KMAC-128 (the lowest strength).
1137     localparam int MaxNumDigestParts = (sha3_pkg::StateW - 2 *
128) / DynAppDigestW;
1138     localparam int DigestCntW = $clog-
2(MaxNumDigestParts);
1139     logic [DigestCntW-1:0] digest_top;
1140     logic [DigestCntW-1:0] current_digest_idx;
1141
1142     // Expose only the relevant part of the digest depending on
the mode and strength. In case an
1143     // invalid mode or strength is detected, we set the di-
gest_top to the smallest value.
1144     always_comb begin
1145         unique case (sha3_mode_q)
1146             sha3_pkg::Sha3: begin
1147                 unique case (keccak_strength_q)
1148                     sha3_pkg::L128: digest_top = DigestCntW'(128 / Dyn-
AppDigestW);
1149                     // 224 is not a multiple of 64. Round up the number
of digest parts. Note, due to this
1150                     // rounding the interface exposes bits from the rate
which are not part of the hash.
1151                     sha3_pkg::L224: digest_top = DigestCntW'(4);
1152                     sha3_pkg::L256: digest_top = DigestCntW'(256 / Dyn-
AppDigestW);
1153                     sha3_pkg::L384: digest_top = DigestCntW'(384 / Dyn-
AppDigestW);
1154                     sha3_pkg::L512: digest_top = DigestCntW'(512 / Dyn-
AppDigestW);
1155                     default: digest_top = DigestCntW'(128 / Dyn-
AppDigestW);
1156                 endcase
1157             end
1158             sha3_pkg::Shake,
1159             sha3_pkg::cShake: begin
1160                 // Expose the requested output length for KMAC. Expose
the full rate for SHAKE and cSHAKE.
1161                 if (kmac_en_q) begin
1162                     digest_top = AppDigestW / DynAppDigestW;
1163                 end else begin
1164                     unique case (keccak_strength_q)
1165                         sha3_pkg::L128: digest_top = Digest-

```



```

CntW'((sha3_pkg::StateW - 2 * 128) / DynAppDigestW);
1166   sha3_pkg::L224: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 224) / DynAppDigestW);
1167   sha3_pkg::L256: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 256) / DynAppDigestW);
1168   sha3_pkg::L384: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 384) / DynAppDigestW);
1169   sha3_pkg::L512: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 512) / DynAppDigestW);
1170   default: digest_top = Digest-
CntW'((sha3_pkg::StateW - 2 * 512) / DynAppDigestW);
1171   endcase
1172   end
1173   end
1174   default: digest_top = DigestCntW'(128 / DynAppDigestW);
1175   endcase
1176   end
1177
1178   // Limit the response width to 64 bits because its the GCD
for all modes except SHA-224.
1179   `ASSERT_INIT(DynAppDigestWIs64Bit_A, DynAppDigestW == 64)
1180   // Ensure all counter top computations divide properly as
the response channel does not carry any
1181   // valid-strobe information.

1182   `ASSERT_INIT(DigestTopDividesSha3L128_A, 128 % DynAppDigestW
== 0)
1183   // An exception is SHA3-224, where we fix the number of di-
gest parts to 4, so the full hash and
1184   // some additional rate bits are sent.

1185   `ASSERT_INIT(DigestTopDividesSha3L256_A, 256 % DynAppDigestW
== 0)
1186   `ASSERT_INIT(DigestTopDividesSha3L384_A, 384 % DynAppDigestW
== 0)
1187   `ASSERT_INIT(DigestTopDividesSha3L512_A, 512 % DynAppDigestW
== 0)
1188
1189   `ASSERT_INIT(DigestTopDividesKmacAppDigestW_A, AppDigestW %
DynAppDigestW == 0)
1190
1191   `ASSERT_INIT(DigestTopDividesShakeL128_A, (sha3_pkg::StateW
- 2 * 128) % DynAppDigestW == 0)
1192   `ASSERT_INIT(DigestTopDividesShakeL224_A, (sha3_pkg::StateW
- 2 * 224) % DynAppDigestW == 0)
1193   `ASSERT_INIT(DigestTopDividesShakeL256_A, (sha3_pkg::StateW
- 2 * 256) % DynAppDigestW == 0)
1194   `ASSERT_INIT(DigestTopDividesShakeL384_A, (sha3_pkg::StateW
- 2 * 384) % DynAppDigestW == 0)
1195   `ASSERT_INIT(DigestTopDividesShakeL512_A, (sha3_pkg::StateW
- 2 * 512) % DynAppDigestW == 0)
1196
1197   assign digest_parts_available = current_digest_idx < di-
gest_top;
1198
1199   // Send parts of the digest if we are pushing.
1200   // Gate with digest_parts_available to prevent sending capa-
city-region bits when
1201   // current_digest_idx has already reached digest_top (the
rate boundary).
1202   assign app_digest_valid = app_push_digest && di-
gest_parts_available &&
1203   lc_c-
trl_pkg::lc_tx_test_false_strict(lc_escalate_en_i);
1204
1205   // Only allow a squeeze if the app allows it.
1206   logic squeeze_again_allowed;

1207   assign squeeze_again_allowed = app_cfg.if_type == AppDynamic
&& app_cfg.session_cfg.en_xof &&
1208   app_cfg.session_cfg.mode !=
AppSHA3 &&
1209   app_cfg.session_cfg.mode !=
AppKMAC;
1210
1211   // Request a squeeze once we are out of digest parts.
1212   assign squeeze_again = squeeze_again_allowed && !di-
gest_parts_available;
1213
1214   logic digest_part_pushed;
1215   assign digest_part_pushed = app_digest_valid &&
app_req.rsp_ready;
1216

1217   // Register if there is a digest response pending. If one is
pending, any following response
1218   // (finish or error) must wait as otherwise data is changed
which would violate the valid
1219   // locked-in principle.
1220   assign pending_digest_rsp_d = app_digest_valid && !di-
gest_part_pushed;
1221
1222   // Similarly, register if there is a pending error response.
1223   logic error_rsp_pushed;
1224   assign error_rsp_pushed = app_error_rsp_valid &&
app_req.rsp_ready;
1225   assign pending_error_rsp_d = app_error_rsp_valid && !er-
ror_rsp_pushed;
1226

```

```

1227   logic [DynAppDigestW-1:0] digest_part[Share];
1228   for (genvar i = 0; i < Share; i++) begin : g_digest_mux
1229     assign digest_part[i] = keccak_state_i[i][DynAppDigestW *
current_digest_idx +: DynAppDigestW];
1230   end
1231
1232   always_comb begin
1233     app_digest = '{default:'0};
1234     if (app_digest_valid) begin

1235       // Digest has always 2 entries. If !EnMasking, second is
tied to 0.
1236       for (int i = 0; i < Share; i++) begin
1237         // Return the portion of state.
1238         if (app_cfg.if_type == AppStatic) begin
1239           app_digest[i] = keccak_state_i[i][AppDigestW-1:0];
1240         end else if (app_cfg.if_type == AppDynamic) begin
1241           // Only expose a sub part. For SHA3-224, we set the
capacity bits of the last response to
1242           // '0. See digest_top calculation above.
1243           app_digest[i][DynAppDigestW-1:0] = digest_part[i];
1244           if (sha3_mode_q == sha3_pkg::Sha3 && kec-
cak_strength_q == sha3_pkg::L224 &&
1245               current_digest_idx == digest_top - 1) begin
1246             app_digest[i][DynAppDigestW-1:DynAppDigestW/2] =
'0;
1247           end
1248         end
1249       end
1250     end
1251   end

1252   // SEC_CM CTR.REDUN
1253   prim_count #(
1254     .Width(DigestCntW),
1255     .PossibleActions(prim_count_pkg::Clr |
prim_count_pkg::Incr)
1256   ) u_digest_part_counter (
1257     .clk_i,
1258     .rst_ni,
1259     .clr_i (clear_digest_pusher),
1260     .set_i ('b0),
1261     .set_cnt_i ('0),
1262     .incr_en_i ('b1),
1263     .decr_en_i ('b0),
1264     .step_i (DigestCntW'(1)),
1265     .commit_i (digest_part_pushed ||
clear_digest_pusher),
1266     .cnt_o (current_digest_idx),
1267     .cnt_after_commit_o(),
1268     .err_o (counter_error_o)
1269   );
1270
1271   // Key handling
1272   // Secret Key Mux
1273   // Prepare merged key if EnMasking is not set.

1274   // Combine share keys into unpacked array for logic below to
assign easily.
1275   // SEC_CM: KEY.SIDELOAD
1276   logic [MaxKeyLen-1:0] keymgr_key[Share];
1277   if (EnMasking == 1) begin : g_masked_key
1278     for (genvar i = 0; i < Share; i++) begin : gen_key_pad
1279       assign keymgr_key[i] = {(MaxKeyLen-KeyMgrKeyW)'(0),
keymgr_key_i.key[i]};
1280     end
1281   end else begin : g_unmasked_key
1282     always_comb begin
1283       keymgr_key[0] = '0;
1284       for (int i = 0; i < keymgr_pkg::Shares; i++) begin
1285         keymgr_key[0][KeyMgrKeyW-1:0] ^= keymgr_key_i.key[i];
1286       end
1287     end
1288   end

1289   // Sideloaded key expose control
1290   assign err_key_used_but_invalid_set = keymgr_key_used && !
keymgr_key_i.valid;
1291
1292   always_comb begin
1293     keymgr_key_used = 1'b0;
1294     key_len_o = reg_key_len_i;
1295     for (int i = 0; i < Share; i++) begin
1296       key_data_o[i] = reg_key_data_i[i];
1297     end

1298   // The key is considered invalid in all cases that are not
listed below (which includes idle and
1299   // error states).
1300   key_valid_o = 1'b0;

1301   unique case (st)
1302     StAppMsg, StAppOutLen, StAppProcess, StAppWait: begin
1303       // The key from KeyMgr is used if the current HW app
interface does *keyed* MAC. This
1304       // module does not know the precise timing related to
when the kmac_core module uses the
1305       // key during processing. To be on the safe side, we

```

```

check that the key remains valid
1312 // directly after latching the configuration and
throughout the entire processing until the
1313 // digest is received. The key is used by the
kmac_core also after the kmac_app finished

1314 // sending the message. So we must check the key until
we receive the digest because we
1315 // don't know here when the hashing engine starts the
processing.
1316 keymgr_key_used = app_cfg.session_cfg.mode == AppKMAC;
1317 key_len_o = SideloadedKey;
1318 for (int i = 0; i < Share; i++) begin
1319   key_data_o[i] = keymgr_key[i];
1320 end
1321 // Key is valid if the current HW app interface does
*keyed* MAC and the key provided by
1322 // KeyMgr is valid.
1323 key_valid_o = keymgr_key_used && keymgr_key_i.valid;
1324 end
1325
1326 StSw: begin
1327   if (keymgr_key_en_i) begin
1328     // Key from KeyMgr is actually used if *keyed* MAC
is enabled.
1329     keymgr_key_used = kmac_en_q;
1330     key_len_o = SideloadedKey;
1331     for (int i = 0; i < Share; i++) begin
1332       key_data_o[i] = keymgr_key[i];
1333     end
1334   end
1335   // Key is valid if SW does *keyed* MAC and ...
1336   if (kmac_en_q) begin
1337     if (!keymgr_key_en_i) begin
1338       // ... it uses the key from kmac's CSR, or ...
1339       key_valid_o = 1'b1;
1340     end else begin
1341       // ... it uses the key provided by KeyMgr and that
one is valid.
1342       key_valid_o = keymgr_key_i.valid;
1343     end
1344   end
1345 end
1346
1347 default: ;
1348 endcase
1349 end
1350
1351 // Prefix Demux
1352 // For SW, always take the prefix from the register. For
apps chose depending on configuration.
1353 always_comb begin
1354   sha3_prefix_o = '0;
1355
1356   unique case (st)
1357     StAppCfg, StAppMsg, StAppOutLen, StAppProcess,
StAppWait, StAppPushDigest: begin
1358       if (app_cfg.session_cfg.prefix_mode == 1'b0) begin
1359         sha3_prefix_o = reg_prefix_i;
1360       end else begin
1361         sha3_prefix_o = app_cfg.prefix;
1362       end
1363     end
1364
1365     StSw: begin
1366       sha3_prefix_o = reg_prefix_i;
1367     end
1368
1369     default: begin
1370       sha3_prefix_o = reg_prefix_i;
1371     end
1372   endcase
1373 end
1374
1375 // Status - exclude error states so SW still can write mes-
sages during error recovery.
1376 assign app_active_o = (st inside {StAppCfg, StAppMsg, StAp-
pOutLen, StAppProcess, StAppWait,
1377   StAppPushDigest, StAppFin-
ish});
1378
1379 // Error Reporting
=====
1380 always_comb begin
1381   priority casez ({fsm_err.valid, mux_err.valid})
1382     2'b ?1: error_o = mux_err;
1383     2'b 10: error_o = fsm_err;
1384     default: error_o = '{valid: 1'b0, code: ErrNone, info:
'0};
1385   endcase
1386 end
1387
1388 ///////////////////////////////////////////////////////////////////
1389 // Assertions //
1390 ///////////////////////////////////////////////////////////////////
1391
1392 // KeyMgr sideload key and the digest should be in the Key
Length value
1393 `ASSERT_INIT(SideloadKeySameToDigest_A, KeyMgrKeyW <= AppDi-
gestW)

```

```

1394 `ASSERT_INIT(AppIntfInRange_A, AppDigestW inside {128, 192,
256, 384, 512})
1395
1396 // Issue(#13655): Having a coverage that sideload keylen and
CSR keylen are
1397 // different.
1398 `COVER(AppIntfUseDifferentSizeKey_C,
1399   (st == StAppCfg && kmac_en_q) |> reg_key_len_i != Side-
loadedKey)
1400
1401 endmodule

```

5. lowRISC_opentitan/hw/ip/kmac/rtl/kmac_core.sv (470 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // KMAC control and padding logic
6
7 `include "prim_assert.sv"
8
9 module kmac_core
10   import kmac_pkg::*;
11   #(
12     // EnMasking: Enable masking security hardening inside kec-
cak_round
13     // If it is enabled, the result digest will be two set of
1600bit.
14     parameter bit EnMasking = 0,
15     localparam int Share = (EnMasking) ? 2 : 1 // derived para-
meter
16   ) (
17     input clk_i,
18     input rst_ni,
19
20     // From Message FIFO
21     input fifo_valid_i,
22     input [MsgWidth-1:0] fifo_data_i [Share],
23     input [MsgStrbW-1:0] fifo_strb_i,
24     output logic fifo_ready_o,
25
26     // to SHA3 Core
27     output logic msg_valid_o,
28     output logic [MsgWidth-1:0] msg_data_o [Share],
29     output logic [MsgStrbW-1:0] msg_strb_o,
30     input msg_ready_i,
31
32     // Configurations
33
34     // If kmac_en is cleared, Core logic doesn't function but
forward incoming
35     // message to SHA3 core
36     input kmac_en_i,
37     input sha3_pkg::sha3_mode_e mode_i,
38     input sha3_pkg::keccak_strength_e strength_i,
39
40     // Key input from CSR
41     input [MaxKeyLen-1:0] key_data_i [Share],
42     input key_len_e key_len_i,
43     input logic key_valid_i,
44
45     // Controls : same to SHA3 core
46     input start_i,
47     input process_i,
48     input prim_mubi_pkg::mubi4_t done_i,
49
50     // Control to SHA3 core
51     output logic process_o,
52
53     // Life cycle
54     input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
55
56     output logic sparse_fsm_error_o,
57     output logic key_index_error_o
58   );
59
60   import sha3_pkg::KeccakMsgAddrw;
61   import sha3_pkg::KeccakCountw;
62   import sha3_pkg::KeccakRate;
63   import sha3_pkg::L128;
64   import sha3_pkg::L224;
65   import sha3_pkg::L256;
66   import sha3_pkg::L384;
67   import sha3_pkg::L512;
68
69   ///////////////////////////////////////////////////////////////////
70   // Definitions //
71   ///////////////////////////////////////////////////////////////////
72
73   // Encoding generated with:
74   // $ ./util/design/sparse-fsm-encode.py -d 3 -m 5 -n 6 \
75   //   -s 401658243 --language=sv
76   //
77   // Hamming distance histogram:
78   //
79   // 0: --

```

```

80 // 1: --
81 // 2: --
82 // 3: ||| (50.00%)
83 // 4: ||| (40.00%)
84 // 5: ||| (10.00%)
85 // 6: --
86 //
87 // Minimum Hamming distance: 3
88 // Maximum Hamming distance: 5
89 // Minimum Hamming weight: 1
90 // Maximum Hamming weight: 4
91 //
92 localparam int StateWidth = 6;
93 typedef enum logic [StateWidth-1:0] {
94   StKmacIdle = 6'b011000,
95
96   // Secret Key pushing stage
97
98   // The key is sliced by prim_slicer. This state pushes the
99   // SHA3 hashing engine. When it hits the block size limit,
100   // (same as in sha3pad) the state machine moves to Mes-
101   sage.
102   StKey = 6'b010111,
103
104   // Incoming Message
105   // The core does nothing but forwarding the incoming mes-
106   sage to SHA3 hashing
107   // engine by turning off `en_kmac_datapath`.
108   StKmacMsg = 6'b001110,
109
110   // Wait till done signal
111   StKmacFlush = 6'b101011,
112
113   // Terminal Error
114   StTerminalError = 6'b100000
115 } kmac_st_e ;
116
117 // Signals //
118 // represents encode_string(K)
119 logic [MaxEncodedKeyW-1:0] encoded_key [Share];
120
121 // Key slice address
122 // This signal controls the 64 bit output of the sliced
123 secret_key.
124 logic [sha3_pkg::KeccakMsgAddrW-1:0] key_index;
125 logic inc_keyidx, clr_keyidx;
126
127 // `sent_blocksize` indicates that the encoded key is sent
128 to sha3 hashing
129 // engine. If this hits at StKey stage, the state moves to
130 message state.
131 logic [sha3_pkg::KeccakCountW-1:0] block_addr_limit;
132 logic sent_blocksize;
133
134 // Internal message signals
135 logic kmac_valid ;
136 logic [MsgWidth-1:0] kmac_data [Share];
137 logic [MsgStrbW-1:0] kmac_strb ;
138
139 // Control SHA3 core
140 // `kmac_process` is to forward the process signal to SHA3
141 core only after
142
143 // the KMAC core writes the key block in case of the message
144 is empty.
145
146 // If the incoming message is empty, there's chance that the
147 `process_i`
148
149 // signal can be asserted while KMAC core processing the key
150 block.
151 logic kmac_process, process_latched;
152
153 // Indication of Secret key write stage. Only in this stage,
154 the internal
155 // message interface is active.
156 logic en_key_write;
157 logic en_kmac_datapath;
158
159 // Encoded key has wider bits. `key_sliced` is the data to
160 send to sha3
161 logic [MsgWidth-1:0] key_sliced [Share];
162
163 // The following signals are only used in assertions.
164 logic unused_signals;
165 assign unused_signals = ^{mode_i, key_valid_i};
166
167 // FSM //
168 kmac_st_e st, st_d;
169
170 // State register
171 `PRIM_FLOP_SPARSE_FSM(u_state_regs, st_d, st, kmac_st_e,
172 StKmacIdle)
173
174

```

```

163 // Next state and output logic
164 // SEC_CM: FSM.SPARSE
165 always_comb begin
166   st_d = st;
167
168   en_kmac_datapath = 1'b 0;
169   en_key_write = 1'b 0;
170
171   clr_keyidx = 1'b 0;
172
173   kmac_valid = 1'b 0;
174   kmac_process = 1'b 0;
175
176   sparse_fsm_error_o = 1'b 0;
177
178   unique case (st)
179     StKmacIdle: begin
180       if (kmac_en_i && start_i) begin
181         st_d = StKey;
182       end else begin
183         st_d = StKmacIdle;
184       end
185     end
186
187     // If State enters here, regardless of the `process_i`,
188     the state writes
189     // full block size of the key into SHA3 hashing engine.
190     StKey: begin
191       en_kmac_datapath = 1'b 1;
192       en_key_write = 1'b 1;
193
194       if (sent_blocksize) begin
195         st_d = StKmacMsg;
196
197         kmac_valid = 1'b 0;
198         clr_keyidx = 1'b 1;
199       end else begin
200         st_d = StKey;
201
202         kmac_valid = 1'b 1;
203       end
204     end
205
206     StKmacMsg: begin
207       // If process is previously latched, it is sent to
208       SHA3 here.
209       if (process_i || process_latched) begin
210         st_d = StKmacFlush;
211
212         kmac_process = 1'b 1;
213       end else begin
214         st_d = StKmacMsg;
215       end
216     end
217
218     StKmacFlush: begin
219       if (prim_mubi_pkg::mubi4_test_true_strict(done_i)) be-
220 gin
221       st_d = StKmacIdle;
222       end else begin
223         st_d = StKmacFlush;
224       end
225     end
226
227     StTerminalError: begin
228       // this state is terminal
229       st_d = st;
230       sparse_fsm_error_o = 1'b 1;
231     end
232
233     default: begin
234       // this state is terminal
235       st_d = StTerminalError;
236       sparse_fsm_error_o = 1'b 1;
237     end
238   endcase
239
240   // SEC_CM: FSM.GLOBAL_ESC, FSM.LOCAL_ESC
241   // Unconditionally jump into the terminal error state
242   // if the life cycle controller triggers an escalation.
243   if (lc_ctrl_pkg::lc_tx_test_true_loose(lc_escalate_en_i))
244   begin
245     st_d = StTerminalError;
246   end
247
248   // DATA Mux depending on kmac_en
249   // When Key write happens, hold the FIFO request. so
250   fifo_ready_o is tied to 0
251   assign msg_valid_o = (en_kmac_datapath) ? kmac_valid :
252   fifo_valid_i;
253   assign msg_data_o = (en_kmac_datapath) ? kmac_data :
254   fifo_data_i;
255   assign msg_strb_o = (en_kmac_datapath) ? kmac_strb :
256   fifo_strb_i;
257   assign fifo_ready_o = (en_kmac_datapath) ? 1'b 0 :
258   msg_ready_i;
259

```

```

255
256 // secret key write request to SHA3 hashing engine is always
full width write.
257 // KeyMgr is fixed 256 bit output. So `right_encode(256)` is
0x020100 --> strb 3
258 assign kmac_strb = (en_key_write) ? '1' : '0';
259
260 assign kmac_data = (en_key_write) ? key_sliced : '{de-
fault:'0';
261
262 // Process is controlled by the KMAC core always.
263 // This is mainly to prevent process_i asserted while KMAC
core is writing
264 // the secret key to SHA3 hashing engine (the empty message
case)
265 assign process_o = (kmac_en_i) ? kmac_process : process_i ;
266
267 always_ff @(posedge clk_i or negedge rst_ni) begin
268 if (!rst_ni) begin
269 process_latched <= 1'b 0;
270 end else if (process_i && !process_o) begin
271 process_latched <= 1'b 1;
272 end else if (process_o ||
273 prim_mubi_pkg::mubi4_test_true_strict(done_i)) begin
274 process_latched <= 1'b 0;
275 end
276 end
277
278 // bytepad(encoded_string(K), 168 or 136)
=====
279 // 1. Prepare left_encode(w)
280 // 2. Prepare left_encode(len(secret_key))
281 // 3. Concatenate left_encode(len(secret_key)) || secret_key
282 // 4. Concatenate left_encode(w) || en-
code_string(secret_key)
283 // 5. Based on the address, slice out the data into MsgWidth
bits
284
285 // left_encode(w): Same as used in sha3pad logic.
286 logic [15:0] encode_bytepad;
287 assign encode_bytepad = sha3_pkg::en-
code_bytepad_len(strength_i);
288
289 // left_encode(len(secret_key))
290 // encoded length is always byte size. Use MaxEncodedKeyLen-
Byte parameter
291 // from kmac_pkg and add one more byte to indicate how many
bytes used to
292 // represent len(secret_key)
293 // Note that if the secret_key is 128 bit, only lower 16
bits of
294 // `encode_keylen` are valid. Refer `encoded_key` concatena-
tion logic below.
295 // As the encoded string in the spec big-endian, The endian
swap is a must.
296 logic [MaxEncodedKeyLenSize + 8 - 1:0] encode_keylen
[Share];
297
298 always_comb begin
299 // the spec mentioned the key length is encoded in
left_encode()
300 // The number is represented in big-endian. For example:
301 // 384 ==> 0x02 0x01 0x80
302 // The first byte is the number of bytes to represent 384
303 // The second byte represents 2**8 number, which is 256
here.
304 // The third byte represents 2**0 number, which is 128.
305 // The data put into MsgFIFO is little-endian and
SHA3(Keccak) processes in
306 // little-endian. So, below keylen swaps the byte order
307 unique case (key_len_i)
308 // endian-swapped
key_length num_bytes
309 // Key128: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(128)}}, 8'h 01};
310 // Key192: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(192)}}, 8'h 01};
311 // Key256: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(256)}}, 8'h 02};
312 // Key384: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(384)}}, 8'h 02};
313 // Key512: encode_keylen[0] = {<<8{MaxEncodedKeyLenS-
ize'(512)}}, 8'h 02};
314
315 // Vivado does not support stream swap for non context
value. So assign
316 // the value directly.
317 Key128: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'(h
0080_01);
318 Key192: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'(h
00C0_01);
319 Key256: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'(h
0001_02);
320 Key384: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'(h
8001_02);
321 Key512: encode_keylen[0] = (MaxEncodedKeyLenSize+8)'(h
0002_02);
322 default: encode_keylen[0] = '0;

```

```

323 endcase
324 end
325
326 if (EnMasking) begin: gen_encode_keylen_masked
327 assign encode_keylen[1] = '0;
328 end
329
330 // encode_string(secret_key): Concatenate key
331 // Based on the left_encode(len(secret_key)) size, the con-
catenation logic
332 // should be changed. If key length is 128 bit, only lower
16 bits of the
333 // encoded length are used so that the upper 8 bits are pad-
ded with 0 as
334 // defined in bytepad() function.
335
336 for (genvar i = 0 ; i < Share; i++) begin : gen_encoded_key
337 always_comb begin
338 unique case (key_len_i)
339 // In Key 128, 192 case, only lower parts of en-
code_keylen signal is
340 // used. So upper padding requires 8 more bits than
MaxKeyLen - keylen
341 Key128: encoded_key[i] = {(8 + MaxKeyLen - 128)'(0),
342 key_data_i[i][0+:128],
343 encode_keylen[i][0+:MaxEn-
codedKeyLenSize]};
344
345 Key192: encoded_key[i] = {(8 + MaxKeyLen - 192)'(0),
346 key_data_i[i][0+:192],
347 encode_keylen[i][0+:MaxEn-
codedKeyLenSize]};
348
349 Key256: encoded_key[i] = {(MaxKeyLen - 256)'(0),
350 key_data_i[i][0+:256],
351 encode_keylen[i]};
352
353 Key384: encoded_key[i] = {(MaxKeyLen - 384)'(0),
354 key_data_i[i][0+:384],
355 encode_keylen[i]};
356
357 // Assume 512bit is the MaxKeyLen
358 Key512: encoded_key[i] = {key_data_i[i][0+:512],
359 encode_keylen[i]};
360
361 default: encoded_key[i] = '0;
362 endcase
363 end
364 end : gen_encoded_key
365
366 // Above logic assumes MaxKeyLen as 512 bits. Revise if it
is not.
367 `ASSERT_INIT(MaxKeyLenMatchToKey512_A, kmac_pkg::MaxKeyLen
== 512)
368
369 // Combine the bytepad `left_encode(w)` and the `en-
code_string(secret_key)`
370 logic [MaxEncodedKeyW + 16 - 1:0] encoded_key_block [Share];
371
372 assign encoded_key_block[0] = {encoded_key[0], en-
code_bytepad};
373
374 if (EnMasking) begin : gen_encoded_key_block_masked
375 assign encoded_key_block[1] = {encoded_key[1], 16'h 0};
376 end
377
378 // Slicer to slice out 64 bits
379 for (genvar i = 0 ; i < Share ; i++) begin : gen_key_slicer
380 prim_slicer #(
381 .InW (MaxEncodedKeyW+16),
382 .IndexW (KeccakMsgAddrW),
383 .OutW (MsgWidth)
384 ) u_key_slicer (
385 .sel_i (key_index),
386 .data_i (encoded_key_block[i]),
387 .data_o (key_sliced[i])
388 );
389 end
390
391 // `key_index` logic
392 // key_index is used to select MsgWidth data from long `en-
coded_key_block`
393 // It behaves same as `keccak_addr` or `prefix_index` in
sha3pad module.
394 assign inc_keyidx = kmac_valid & msg_ready_i ;
395
396 // This primitive is used to place a hardened counter
397 // SEC_CM: CTR.REDUN
398 prim_count #(
399 .Width (sha3_pkg::KeccakMsgAddrW)
400 ) u_key_index_count (
401 .clk_i,
402 .rst_ni,
403 .clr_i (clr_keyidx),
404 .set_i (1'b0),
405 .set_cnt_i ('0),
406 .incr_en_i (inc_keyidx),
407 .decr_en_i (1'b0),
408 .step_i (sha3_pkg::KeccakMsgAddrW'(1)),
409 .commit_i (1'b1),
410 .cnt_o (key_index),

```

```

411     .cnt_after_commit_o(),
412     .err_o(key_index_error_o)
413 );
414
415 // Block size based on the address.
416 // This is used for bytepad() and also pad10*1()
417 // assign block_addr_limit = KeccakRate[strength_i];
418 // but below is easier to understand
419 always_comb begin
420     unique case (strength_i)
421         L128: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L128]);
422         L224: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L224]);
423         L256: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L256]);
424         L384: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L384]);
425         L512: block_addr_limit = KeccakCoun-
tw'(KeccakRate[L512]);
426         default: block_addr_limit = '0;
427     endcase
428 end
429
430 assign sent_blocksize = (key_index == block_addr_limit);
431
432
433 // Encoded Output Length
434 =====
435 //
436 // KMAC(K,X,L,S) := cSHAKE(newX,L,"KMAC",S)
437 // K : Secret Key
438 // X : Input Message
439 // L : Output Length
440 // S : Customization input string
441 // newX = bytepad(encode_string(key), 168or136) || X ||
right_encode(L)

```

```

442 //
443 // Software writes desired output length as encoded value
into the message
444 // FIFO at the end of the message prior to set !!CMD.pro-
cess.
445
446
447 // Assertions //
448
449
450
451 // If process_latched is set, then at Message state, it
should be cleared
452
453 `ASSERT(ProcessLatchedCleared_A,
454     st == StKmacMsg && process_latched | => !pro-
cess_latched)
455
456 // Assume configuration is stable during the operation
457 `ASSUME(KmacEnStable_M, $changed(kmac_en_i) |-> st inside
{StKmacIdle, StTerminalError})
458 `ASSUME(ModeStable_M, $changed(mode_i) |-> st inside {StKma-
cIdle, StTerminalError})
459 `ASSUME(StrengthStable_M,
460     $changed(strength_i) |->
461     (st inside {StKmacIdle, StTerminalError}) ||
462     ($past(st) == StKmacIdle))
463 `ASSUME(KeyLengthStableWhenValid_M, key_valid_i && !
$rose(key_valid_i) |-> $stable(key_len_i))
464 `ASSUME(KeyDataStableWhenValid_M, key_valid_i && !
$rose(key_valid_i) |-> $stable(key_data_i))
465
466 // no acked to MsgFIFO in StKmacMsg
467 `ASSERT(AckOnlyInMessageState_A,
468     fifo_valid_i && fifo_ready_o && kmac_en_i |-> st ==
StKmacMsg)
469
470 endmodule : kmac_core

```

OpenTitan CSRNG (SP800-90A)

What. NIST SP800-90A 의 CTR_DRBG 를 구현한 결정론적 난수 생성기. 여러 요청자에게 난수를 분배하는 중재 로직 포함.

Why it exists. 암호 난수는 규격이 절차를 규정 한다. 인스턴스화 · 리시드 · 생성 · 언인스턴스화의 상태 기계와 카운터 한계가 표준에 박혀 있고, 인증(FIPS 140-3)을 받으려면 그대로여야 한다. ‘난수를 만든다’가 아니라 ‘표준 절차를 이행한다’가 이 블록의 정체다.

학부 복습 · EECS concepts. 의사난수 생성기(PRNG)와 암호학적 PRNG 의 차이 · 전방/후방 비밀성 · 엔트로피 풀 관리 · 상태 기계 설계.

석사 수준 · Practice note. 이런 블록은 검증이 ‘표준 절차와 같은가’로 정의된다. NIST 가 주는 시험 벡터(CAVP)를 통과시키는 것이 사실상의 명세다. 모델링 팀의 일은 표준 문서를 코드로 정확히 옮기는 것이며, 창의성이 오히려 해롭다.

8 files • 3,151 lines

API — csrng.sv

Table 240. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — module / interface / package (1개)

이름	종류
csrng	module

Table 241. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — parameter (5개)

파라미터	형/폭	기본값
SBoxImpl	aes_pkg::sbox_impl_e	aes_pkg::SBoxImplCanright
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RndCnstCsKeymgrDivNonProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)
RndCnstCsKeymgrDivProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)

Table 242. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — localparam (1개)

상수	형/폭	값
NumHwApps	int unsigned	NumApps - 1 // derived parameter

Table 243. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t

포트	방향	형/폭
otp_en_csrng_sw_app_read_i	input	prim_mubi_pkg::mubi8_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
entropy_src_hw_if_o	output	entropy_src_pkg::entropy_src_hw_if_req_t
entropy_src_hw_if_i	input	entropy_src_pkg::entropy_src_hw_if_rsp_t
csrng_cmd_i	input	csrng_req_t [NumHwApps-1:0]
csrng_cmd_o	output	csrng_rsp_t [NumHwApps-1:0]
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
intr_cs_cmd_req_done_o	output	logic
intr_cs_entropy_req_o	output	logic
intr_cs_hw_inst_exc_o	output	logic
intr_cs_fatal_err_o	output	logic

API — csrng_block_encrypt.sv

Table 244. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — module / interface / package (1개)

이름	종류
csrng_block_encrypt	module

Table 245. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — parameter (1개)

파라미터	형/폭	기본값
SBoxImpl	aes_pkg::sbox_impl_e	aes_pkg::SBoxImplLut

Table 246. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — localparam (1개)

상수	형/폭	값
NumShares	int unsigned	1

Table 247. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
req_vld_i	input	logic
req_rdy_o	output	logic
req_data_i	input	csrng_key_v_t
rsp_vld_o	output	logic
rsp_rdy_i	input	logic
rsp_data_o	output	logic [BlkLen-1:0]

포트	방향	형/폭
cipher_sm_err_o	output	logic

API — csrng_cmd_stage.sv

Table 248. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.sv — module / interface / package (1개)

이름	종류
csrng_cmd_stage	module

Table 249. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.sv — localparam (3개)

상수	형/폭	값
GenBitsFifoWidth	int	1 + BlkLen
GenBitsFifoDepth	int	1
StateWidth	int	8

Table 250. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.sv — 포트 (32개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
cs_enable_i	input	logic
cmd_stage_vld_i	input	logic
cmd_stage_shid_i	input	logic [InstIdWidth-1:0]
cmd_stage_bus_i	input	logic [CmdBusWidth-1:0]
cmd_stage_rdy_o	output	logic
reseed_cnt_reached_i	input	logic
reseed_cnt_alert_o	output	logic
invalid_cmd_seq_alert_o	output	logic
invalid_acmd_alert_o	output	logic
cmd_arb_req_o	output	logic
cmd_arb_sop_o	output	logic
cmd_arb_mop_o	output	logic
cmd_arb_eop_o	output	logic
cmd_arb_gnt_i	input	logic
cmd_arb_bus_o	output	logic [CmdBusWidth-1:0]
cmd_ack_i	input	logic
cmd_ack_sts_i	input	csrng_cmd_sts_e
cmd_stage_ack_o	output	logic
cmd_stage_ack_sts_o	output	csrng_cmd_sts_e

포트	방향	형/폭
genbits_vld_i	input	logic
genbits_bus_i	input	logic [127:0]
genbits_fips_i	input	logic
genbits_vld_o	output	logic
genbits_rdy_i	input	logic
genbits_bus_o	output	logic [127:0]
genbits_fips_o	output	logic
cmd_stage_sfifo_cmd_err_o	output	logic [2:0]
cmd_stage_sfifo_genbits_err_o	output	logic [2:0]
cmd_gen_cnt_err_o	output	logic
cmd_stage_sm_err_o	output	logic

API — csrng_core.sv

Table 251. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — module / interface / package (1개)

이름	종류
csrng_core	module

Table 252. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — parameter (4개)

파라미터	형/폭	기본값
SBoxImpl	aes_pkg::sbox_impl_e	aes_pkg::SBoxImplLut
NumHwApps	int	2
RndCnstCsKeymgrDivNonProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)
RndCnstCsKeymgrDivProduction	cs_keymgr_div_t	CsKeymgrDivWidth'(0)

Table 253. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — localparam (4개)

상수	형/폭	값
ADeepthClog	int unsigned	$\$clog2(\text{CmdMaxClen}) + 1$
CsEnableCopies	int unsigned	51
LcHwDebugCopies	int unsigned	1
Flag0Copies	int unsigned	3

Table 254. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
reg2hw	input	csrng_reg_pkg::csrng_reg2hw_t
hw2reg	output	csrng_reg_pkg::csrng_hw2reg_t

포트	방향	형/폭
otp_en_csrng_sw_app_read_i	input	prim_mubi_pkg::mubi8_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
entropy_src_hw_if_o	output	entropy_src_pkg::entropy_src_hw_if_req_t
entropy_src_hw_if_i	input	entropy_src_pkg::entropy_src_hw_if_rsp_t
csrng_cmd_i	input	csrng_req_t [NumHwApps-1:0]
csrng_cmd_o	output	csrng_rsp_t [NumHwApps-1:0]
recov_alert_test_o	output	logic
fatal_alert_test_o	output	logic
recov_alert_o	output	logic
fatal_alert_o	output	logic
intr_cs_cmd_req_done_o	output	logic
intr_cs_entropy_req_o	output	logic
intr_cs_hw_inst_exc_o	output	logic
intr_cs_fatal_err_o	output	logic

API — csrng_ctr_drbg.sv

Table 255. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv — module / interface / package (1개)

이름	종류
csrng_ctr_drbg	module

Table 256. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv — localparam (3개)

상수	형/폭	값
NumBlkPerUpd	int unsigned	SeedLen/BlkLen
NumBlkPerUpdLg	int unsigned	\$clog2(NumBlkPerUpd)
StateWidth	int	6

Table 257. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
req_vld_i	input	logic
req_rdy_o	output	logic
req_data_i	input	csrng_core_data_t
req_entropy_i	input	logic [SeedLen-1:0]
req_entropy_fips_i	input	logic

포트	방향	형/폭
req_glast_i	input	logic
state_db_wr_o	output	logic
rsp_vld_o	output	logic
rsp_data_o	output	csrng_core_data_t
rsp_sts_o	output	csrng_cmd_sts_e
bits_vld_o	output	logic
bits_data_o	output	logic [BlkLen-1:0]
bits_fips_o	output	logic
block_encrypt_req_vld_o	output	logic
block_encrypt_req_rdy_i	input	logic
block_encrypt_req_data_o	output	csrng_key_v_t
block_encrypt_rsp_vld_i	input	logic
block_encrypt_rsp_rdy_o	output	logic
block_encrypt_rsp_data_i	input	logic [BlkLen-1:0]
ctr_err_o	output	logic
sm_err_o	output	logic

API — csrng_main_sm.sv

Table 258. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_main_sm.sv — module / interface / package (1개)

이름	종류
csrng_main_sm	module

Table 259. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_main_sm.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
acmd_avail_i	input	logic
acmd_accept_o	output	logic
acmd_i	input	acmd_e
acmd_eop_i	input	logic
flag0_i	input	logic
cmd_entropy_req_o	output	logic
cmd_entropy_avail_i	input	logic
cmd_vld_o	output	logic
cmd_rdy_i	input	logic

포트	방향	형/폭
clr_adata_packer_o	output	logic
cmd_complete_i	input	logic
local_escalate_i	input	logic
main_sm_state_o	output	logic [MainSmStateWidth-1:0]
main_sm_err_o	output	logic

API — csrng_pkg.sv

Table 260. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv — module / interface / package (1개)

이름	종류
csrng_pkg	package

Table 261. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv — parameter (21개)

파라미터	형/폭	기본값
NumAppsLg	int unsigned	\$clog2(csrng_reg_pkg::NumApps)
BlkLen	int unsigned	128
KeyLen	int unsigned	256
SeedLen	int unsigned	BlkLen + KeyLen
CtrLen	int unsigned	32
RsCtrWidth	int unsigned	32
GenBitsCtrWidth	int unsigned	12
CmdWidth	int unsigned	3
InstIdWidth	int unsigned	4
CmdBusWidth	int unsigned	32
CmdMaxClen	int unsigned	12
CmdFifoDepth	int unsigned	2
CmdFifoDepthLg	int unsigned	\$clog2(CmdFifoDepth)
GENBITS_BUS_WIDTH	int unsigned	BlkLen
FIPS_GENBITS_BUS_WIDTH	int unsigned	entropy_src_pkg::FIPS_BUS_WIDTH +
CmdStatusWidth	int unsigned	3
CSRNG_REQ_DEFAULT	csrng_req_t	{default: '0}
CSRNG_RSP_DEFAULT	csrng_rsp_t	'0
StateWidth	int unsigned	\$bits(csrng_state_t)
MainSmStateWidth	int unsigned	6
CsKeymgrDivWidth	int	384

Table 262. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv — typedef / using (1개)

새 이름	원래 형
cs_keymgr_div_t	logic [CsKeymgrDivWidth-1:0]

API — csrng_reg_pkg.sv

Table 263. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_reg_pkg.sv — module / interface / package (1개)

이름	종류
csrng_reg_pkg	package

Table 264. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_reg_pkg.sv — parameter (45개)

파라미터	형/폭	기본값
NumApps	int	3
NumAlerts	int	2
BlockAw	int	7
NumRegs	int	24
CSRNG_INTR_STATE_OFFSET	logic [BlockAw-1:0]	7'h 0
CSRNG_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	7'h 4
CSRNG_INTR_TEST_OFFSET	logic [BlockAw-1:0]	7'h 8
CSRNG_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	7'h c
CSRNG_REGWEN_OFFSET	logic [BlockAw-1:0]	7'h 10
CSRNG_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 14
CSRNG_CMD_REQ_OFFSET	logic [BlockAw-1:0]	7'h 18
CSRNG_RESEED_INTERVAL_OFFSET	logic [BlockAw-1:0]	7'h 1c
CSRNG_RESEED_COUNTER_0_OFFSET	logic [BlockAw-1:0]	7'h 20
CSRNG_RESEED_COUNTER_1_OFFSET	logic [BlockAw-1:0]	7'h 24
CSRNG_RESEED_COUNTER_2_OFFSET	logic [BlockAw-1:0]	7'h 28
CSRNG_SW_CMD_STS_OFFSET	logic [BlockAw-1:0]	7'h 2c
CSRNG_GENBITS_VLD_OFFSET	logic [BlockAw-1:0]	7'h 30
CSRNG_GENBITS_OFFSET	logic [BlockAw-1:0]	7'h 34
CSRNG_INT_STATE_READ_ENABLE_OFFSET	logic [BlockAw-1:0]	7'h 38
CSRNG_INT_STATE_READ_ENABLE_REGWEN_OFFSET	logic [BlockAw-1:0]	7'h 3c
CSRNG_INT_STATE_NUM_OFFSET	logic [BlockAw-1:0]	7'h 40
CSRNG_INT_STATE_VAL_OFFSET	logic [BlockAw-1:0]	7'h 44
CSRNG_FIPS_FORCE_OFFSET	logic [BlockAw-1:0]	7'h 48
CSRNG_HW_EXC_STS_OFFSET	logic [BlockAw-1:0]	7'h 4c
CSRNG_RECOV_ALERT_STS_OFFSET	logic [BlockAw-1:0]	7'h 50
CSRNG_ERR_CODE_OFFSET	logic [BlockAw-1:0]	7'h 54
CSRNG_ERR_CODE_TEST_OFFSET	logic [BlockAw-1:0]	7'h 58

파라미터	형/폭	기본값
CSRNG_MAIN_SM_STATE_OFFSET	logic [BlockAw-1:0]	7'h 5c
CSRNG_INTR_TEST_RESVAL	logic [3:0]	4'h 0
CSRNG_INTR_TEST_CS_CMD_REQ_DONE_RESVAL	logic [0:0]	1'h 0
CSRNG_INTR_TEST_CS_ENTROPY_REQ_RESVAL	logic [0:0]	1'h 0
CSRNG_INTR_TEST_CS_HW_INST_EXC_RESVAL	logic [0:0]	1'h 0
CSRNG_INTR_TEST_CS_FATAL_ERR_RESVAL	logic [0:0]	1'h 0
CSRNG_ALERT_TEST_RESVAL	logic [1:0]	2'h 0
CSRNG_ALERT_TEST_RECOV_ALERT_RESVAL	logic [0:0]	1'h 0
CSRNG_ALERT_TEST_FATAL_ALERT_RESVAL	logic [0:0]	1'h 0
CSRNG_RESEED_COUNTER_0_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_0_RESEED_COUNTER_0_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_1_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_1_RESEED_COUNTER_1_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_2_RESVAL	logic [31:0]	32'h 0
CSRNG_RESEED_COUNTER_2_RESEED_COUNTER_2_RESVAL	logic [31:0]	32'h 0
CSRNG_GENBITS_VLD_RESVAL	logic [1:0]	2'h 0
CSRNG_GENBITS_RESVAL	logic [31:0]	32'h 0
CSRNG_INT_STATE_VAL_RESVAL	logic [31:0]	32'h 0

1. lowRISC_opentitan/hw/ip/csrng/rtl/csrng.sv (203 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: csrng top level wrapper file
6
7 `include "prim_assert.sv"
8
9 module csrng
10   import csrng_pkg::*;
11   import csrng_reg_pkg::*;
12   #(
13     parameter aes_pkg::sbox_impl_e SBoxImpl = aes_pkg::SBoxIm-
  pCanright,
14     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
  mAlerts{1'b1}},
15     // Number of cycles a differential skew is tolerated on the
  alert signal
16     parameter int unsigned AlertSkewCycles = 1,
17     parameter cs_keymgr_div_t RndCnstCsKeymgrDivNonProduction =
  CsKeymgrDivWidth'(0),
18     parameter cs_keymgr_div_t RndCnstCsKeymgrDivProduction = Cs-
  KeymgrDivWidth'(0),
19
20     localparam int unsigned NumHwApps = NumApps - 1 // derived
  parameter
21   ) (
22     input  logic clk_i,
23     input  logic rst_ni,
24
25     // TileLink Bus Interface
26     input  tlul_pkg::tl_h2d_t tl_i,
27     output tlul_pkg::tl_d2h_t tl_o,
28
29     // OTP Interface
30     // SEC_CM: INTERSIG.MUBI
31     input  prim_mubi_pkg::mubi8_t otp_en_csrng_sw_app_read_i,
32
33     // Life Cycle Broadcast Inputs
34     input  lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
35

```

```

36     // Entropy Source Interface
37     output entropy_src_pkg::entropy_src_hw_if_req_t en-
  trophy_src_hw_if_o,
38     input  entropy_src_pkg::entropy_src_hw_if_rsp_t en-
  trophy_src_hw_if_i,
39
40     // Application Interfaces
41     input  csrng_req_t [NumHwApps-1:0] csrng_cmd_i,
42     output csrng_rsp_t [NumHwApps-1:0] csrng_cmd_o,
43
44     // Alerts
45     input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
  alert_rx_i,
46     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
  alert_tx_o,
47
48     // Interrupts
49     output logic intr_cs_cmd_req_done_o,
50     output logic intr_cs_entropy_req_o,
51     output logic intr_cs_hw_inst_exc_o,
52     output logic intr_cs_fatal_err_o
53   );
54
55   csrng_reg2hw_t reg2hw;
56   csrng_hw2reg_t hw2reg;
57
58   logic [NumAlerts-1:0] alert_test;
59   logic [NumAlerts-1:0] alert;
60
61   logic [NumAlerts-1:0] intg_err_alert;
62   assign intg_err_alert[0] = 1'b0;
63
64   // SEC_CM: CONFIG.REGWEN
65   // SEC_CM: TILE_LINK.BUS.INTEGRITY
66
67   csrng_reg_top u_reg (
68     .clk_i,
69     .rst_ni,
70     .tl_i,
71     .tl_o,
72     .reg2hw,
73     .hw2reg,
74     .intg_err_o(intg_err_alert[1])
75   );
76
77   csrng_core #(

```



```

78     .SBoxImpl(SBoxImpl),
79     .NumHwApps(NumHwApps),
80     .RndCnstCsKeymgrDivNonProduc-
tion(RndCnstCsKeymgrDivNonProduction),
81     .RndCnstCsKeymgrDivProduc-
tion(RndCnstCsKeymgrDivProduction)
82 ) u_csrng_core (
83     .clk_i,
84     .rst_ni,
85     .reg2hw,
86     .hw2reg,
87
88     // misc inputs
89     .otp_en_csrng_sw_app_read_i(otp_en_csrng_sw_app_read_i),
90     .lc_hw_debug_en_i,
91
92     // Entropy Interface
93     .entropy_src_hw_if_o,
94     .entropy_src_hw_if_i,
95
96     // Application Interfaces
97     .csrng_cmd_i,
98     .csrng_cmd_o,
99
100    // Alerts
101    .recov_alert_test_o(alert_test[0]),
102    .fatal_alert_test_o(alert_test[1]),
103    .recov_alert_o(alert[0]),
104    .fatal_alert_o(alert[1]),
105
106    .intr_cs_cmd_req_done_o,
107    .intr_cs_entropy_req_o,
108    .intr_cs_hw_inst_exc_o,
109    .intr_cs_fatal_err_o
110 );
111
112 ///////////////////////////////////////////////////
113 // Alert generation
114 ///////////////////////////////////////////////////
115 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
116     prim_alert_sender #(
117         .AsyncOn(AlertAsyncOn[i]),
118         .SkewCycles(AlertSkewCycles),
119         .IsFatal(i)
120     ) u_prim_alert_sender (
121         .clk_i,
122         .rst_ni,
123         .alert_test_i ( alert_test[i] ),
124         .alert_req_i ( alert[i] || intg_err_alert[i] ),
125         .alert_ack_o ( ),
126         .alert_state_o ( ),
127         .alert_rx_i ( alert_rx_i[i] ),
128         .alert_tx_o ( alert_tx_o[i] )
129     );
130 end
131
132 // Assertions
133
134 `ASSERT_KNOWN(TlDValidKnown0_A, tl_o.d_valid)
135 `ASSERT_KNOWN(TlAReadyKnown0_A, tl_o.a_ready)
136 `ASSERT_KNOWN(EsReqKnown0_A, entropy_src_hw_if_o.es_req)
137
138 // Application Interface Asserts
139 for (genvar i = 0; i < NumHwApps; i = i+1) begin :
gen_app_if_asserts
140     `ASSERT_KNOWN(CsrngReqReadyKnown0_A,
csrng_cmd_o[i].csrng_req_ready)
141     `ASSERT_KNOWN(CsrngRspAckKnown0_A,
csrng_cmd_o[i].csrng_rsp_ack)
142     `ASSERT_KNOWN(CsrngRspStsKnown0_A,
csrng_cmd_o[i].csrng_rsp_sts)
143     `ASSERT_KNOWN(CsrngGenbitsValidKnown0_A,
csrng_cmd_o[i].genbits_valid)
144     `ASSERT_KNOWN_IF(CsrngGenbitsFipsKnown0_A,
csrng_cmd_o[i].genbits_fips,
csrng_cmd_o[i].genbits_valid)
145     `ASSERT_KNOWN_IF(CsrngGenbitsBusKnown0_A,
csrng_cmd_o[i].genbits_bus,
csrng_cmd_o[i].genbits_valid)
146     end : gen_app_if_asserts
147
148 // Alerts
149 `ASSERT_KNOWN(AlertTxKnown0_A, alert_tx_o)
150
151 `ASSERT_KNOWN(IntrCsCmdReqDoneKnown0_A,
intr_cs_cmd_req_done_o)
152 `ASSERT_KNOWN(IntrCsEntropyReqKnown0_A, intr_cs_en-
tropy_req_o)
153 `ASSERT_KNOWN(IntrCsShwInstExcKnown0_A,
intr_cs_hw_inst_exc_o)
154 `ASSERT_KNOWN(IntrCsFatalErrKnown0_A, intr_cs_fatal_err_o)
155
156 `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CtrDrbgGenAlertCheck_A,
157     u_csrng_core.u_csrng_ctr_drbg.u_prim_count_ctr_drbg,
158     alert_tx_o[1])
159
160 for (genvar i = 0; i < NumHwApps + 1; i++) begin :
gen_cnt_asserts

```

```

165     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
166     u_c-
srng_core.gen_cmd_stage[i].u_csrng_cmd_stage.u_prim_count_cmd_gen_c
ntr,
167     alert_tx_o[1])
168
169     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(DrbgCmdFsmCheck_A,
170     u_c-
srng_core.gen_cmd_stage[i].u_csrng_cmd_stage.u_state_regs,
171     alert_tx_o[1])
172 end
173
174     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlMainFsmCheck_A,
175     u_csrng_core.u_csrng_main_sm.u_state_regs,
176     alert_tx_o[1])
177
178     `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(DrbgGenFsmCheck_A,
179     u_csrng_core.u_csrng_ctr_drbg.u_state_regs,
180     alert_tx_o[1])
181
182 for (genvar i = 0; i < aes_pkg::Sp2VWidth; i++) begin :
gen_aes_cipher_control_fsm_svas
183     if (aes_pkg::SP2V_LOGIC_HIGH[i] == 1'b1) begin :
gen_aes_cipher_control_fsm_svas_p
184         `AS-
SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
185     u_c-
srng_core.u_csrng_block_encrypt.u_aes_cipher_core.u_aes_cipher_cont
rol.gen_fsm[i].
186     gen_fsm_p.u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_sta
te_regs,
187     alert_tx_o[1])
188     end else begin : gen_aes_cipher_control_fsm_svas_n
189         `AS-
SERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
190     u_c-
srng_core.u_csrng_block_encrypt.u_aes_cipher_core.u_aes_cipher_cont
rol.gen_fsm[i].
191     gen_fsm_n.u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_sta
te_regs,
192     alert_tx_o[1])
193     end
194 end
195
196 // Alert assertions for reg_we onehot check
197 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
198     u_reg, alert_tx_o[1])
199
200 // The total number of application interfaces defined in the
hjson must be at least two:
201 // One SW interface and at least one HW interface. Zero HW
interfaces is currently not supported.
202 `ASSERT_INIT(CsrngNumAppsMatch_A, NumApps >= 2)
203 endmodule

```

2. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_block_encrypt.sv (113 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: AES-based block encrypt module for CSRNG
6 //
7 module csrng_block_encrypt import csrng_pkg::*; #(
8     parameter aes_pkg::sbox_impl_e SBoxImpl = aes_pkg::SBoxIm-
pLut
9 ) (
10     input logic clk_i,
11     input logic rst_ni,
12
13     // Global enable
14     input logic enable_i,
15
16     // Request from ctr_drbg
17     input logic req_vld_i,
18     output logic req_rdy_o,
19     input csrng_key_v_t req_data_i,
20
21     // Response to ctr_drbg
22     output logic rsp_vld_o,
23     input logic rsp_rdy_i,
24     output logic [BlkLen-1:0] rsp_data_o,
25
26     // Status and error signals
27     output logic cipher_sm_err_o
28 );
29
30 localparam int unsigned NumShares = 1;
31
32 // Signals
33 aes_pkg::sp2v_e cipher_in_valid;

```

```

35 aes_pkg::sp2v_e      cipher_in_ready;
36 aes_pkg::sp2v_e      cipher_out_valid;
37 aes_pkg::sp2v_e      cipher_out_ready;
38
39 logic [3:0][3:0][7:0] state_init[NumShares];
40
41 logic [7:0][31:0]     key_init[NumShares];
42 logic [3:0][3:0][7:0] state_done[NumShares];
43 logic [3:0][3:0][7:0] state_out;
44
45 //-----
46 // AES cipher core
47 //-----
48
49 assign state_init[0] = aes_pkg::aes_trans-
pose({<<8{req_data_i.v}});
50 assign key_init[0]   = {<<8{req_data_i.key}};
51
52 assign state_out = aes_pkg::aes_transpose(state_done[0]);
53 assign rsp_data_o = {<<8{state_out}};
54
55 assign cipher_in_valid = (enable_i && req_vld_i) ?
aes_pkg::SP2V_HIGH : aes_pkg::SP2V_LOW;
56
57 // SEC_CM: AES_CIPHER.FSM.SPARSE
58 // SEC_CM: AES_CIPHER.FSM.REDUN
59 // SEC_CM: AES_CIPHER.CTRL.SPARSE
60 // SEC_CM: AES_CIPHER.FSM.LOCAL_ESC
61 // SEC_CM: AES_CIPHER.CTRL.REDUN
62 // SEC_CM: AES_CIPHER.DATA_REG.LOCAL_ESC
63
64 aes_cipher_core #(
65     .AES192Enable (1'b0), // AES192Enable disabled
66     .CiphOpFwdOnly(1'b1), // Forward operation only
67     .SecMasking   (1'b0), // Masking disable
68     .SecSBoxImpl  (SBoxImpl)
69 ) u_aes_cipher_core (
70     .clk_i (clk_i),
71     .rst_ni(rst_ni),
72
73     .cfg_valid_i      (1'b1),
74     .in_valid_i       (cipher_in_valid),
75     .in_ready_o        (cipher_in_ready),
76     .out_valid_o       (cipher_out_valid),
77     .out_ready_i       (cipher_out_ready),
78     .op_i             (aes_pkg::CIPH_FWD),
79     .key_len_i         (aes_pkg::AES_256),
80     .crypt_i          (aes_pkg::SP2V_HIGH), // Enable
81     .crypt_o           (),
82     .alert_fatal_i    (1'b0),
83     .alert_o          (cipher_sm_err_o),
84     .dec_key_gen_i    (aes_pkg::SP2V_LOW), // Disable
85     .dec_key_gen_o    (),
86     .prng_reseed_i    (1'b0), // Disable
87     .prng_reseed_o    (),
88     .key_clear_i      (1'b0), // Disable
89     .key_clear_o      (),
90     .data_out_clear_i (1'b0), // Disable
91     .data_out_clear_o (),
92     // These two init values are provided to allow synthesis
93     // to perform optimizations.
94     .prd_clearing_state_i(state_init),
95     .prd_clearing_key_i (key_init),
96     .force_masks_i      (1'b0),
97     .data_in_mask_o     (),
98     .entropy_req_o      (),
99     .entropy_ack_i      (1'b0),
100    .entropy_i           ('0'),
101    .state_init_i        (state_init),
102    .key_init_i          (key_init),
103    .state_o             (state_done)
104 );
105
106 // The cipher determines whether the response is ready for
107 // consumption
108 assign rsp_vld_o = (cipher_out_valid == aes_pkg::SP2V_HIGH);
109
110 // Type conversion for AES compatibility
111 assign req_rdy_o = (cipher_in_ready == aes_pkg::SP2V_HIGH);
112 assign cipher_out_ready = rsp_rdy_i ? aes_pkg::SP2V_HIGH :
aes_pkg::SP2V_LOW;
113 endmodule

```

3. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_cmd_stage.v (548 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Description: CSRNG command staging module.
6 //
7 `include "prim_assert.sv"
8
9 module csrng_cmd_stage import csrng_pkg::*; (
10     input logic          clk_i,

```

```

11     input logic          rst_ni,
12     // Command input.
13     input logic          cs_enable_i,
14     input logic          cmd_stage_vld_i,
15     input logic [InstIdWidth-1:0] cmd_stage_shid_i,
16     input logic [CmdBusWidth-1:0] cmd_stage_bus_i,
17     output logic         cmd_stage_rdy_o,
18     // Command checking interface.
19     input logic          reseed_cnt_reached_i,
20     output logic         reseed_cnt_alert_o,
21     output logic         invalid_cmd_seq_alert_o,
22     output logic         invalid_acmd_alert_o,
23     // Command to arbiter.
24     output logic         cmd_arb_req_o,
25     output logic         cmd_arb_sop_o,
26     output logic         cmd_arb_mop_o,
27     output logic         cmd_arb_eop_o,
28     input logic          cmd_arb_gnt_i,
29     output logic [CmdBusWidth-1:0] cmd_arb_bus_o,
30     // Ack from core.
31     input logic          cmd_ack_i,
32     input csrng_cmd_sts_e cmd_ack_sts_i,
33     // Ack to app i/f.
34     output logic         cmd_stage_ack_o,
35     output csrng_cmd_sts_e cmd_stage_ack_sts_o,
36     // Genbits from core.
37     input logic          genbits_vld_i,
38     input logic [127:0]  genbits_bus_i,
39     input logic          genbits_fips_i,
40     // Genbits to app i/f.
41     output logic         genbits_vld_o,
42     input logic          genbits_rdy_i,
43     output logic [127:0] genbits_bus_o,
44     output logic         genbits_fips_o,
45     // Error indication.
46     output logic [2:0]   cmd_stage_sfifo_cmd_err_o,
47     output logic [2:0]   cmd_stage_sfifo_genbits_err_o,
48     output logic         cmd_gen_cnt_err_o,
49     output logic         cmd_stage_sm_err_o
50 );
51
52 // Genbits parameters.
53 localparam int GenBitsFifoWidth = 1 + BlkLen;
54 localparam int GenBitsFifoDepth = 1;
55
56 // Command FIFO.
57 logic [CmdBusWidth-1:0] sfifo_cmd_rdata;
58 logic [CmdFifoDepth-1:0] sfifo_cmd_depth;
59 logic sfifo_cmd_wvld;
60 logic sfifo_cmd_wrdy;
61 logic [CmdBusWidth-1:0] sfifo_cmd_wdata;
62 logic sfifo_cmd_rrdy;
63 logic [2:0] sfifo_cmd_err;
64 logic sfifo_cmd_rvld;
65
66 // Genbits FIFO.
67 logic [GenBitsFifoWidth-1:0] sfifo_genbits_rdata;
68 logic sfifo_genbits_wvld;
69 logic sfifo_genbits_wrdy;
70 logic [GenBitsFifoWidth-1:0] sfifo_genbits_wdata;
71 logic sfifo_genbits_rrdy;
72 logic [2:0] sfifo_genbits_err;
73 logic sfifo_genbits_rvld;
74
75 // Command signals.
76 logic [3:0] cmd_len;
77 logic cmd_fifo_zero;
78 logic cmd_fifo_pop;
79 logic cmd_len_dec;
80 logic cmd_gen_cnt_dec;
81 logic cmd_gen_1st_req;
82 logic cmd_gen_inc_req;
83 logic cmd_gen_cnt_last;
84 logic cmd_final_ack;
85 logic cmd_err_ack;
86 logic [GenBitsCtrWidth-1:0] cmd_gen_cnt;
87 csrng_cmd_sts_e err_sts;
88 logic reseed_cnt_exceeded;
89 logic invalid_cmd_seq;
90 logic invalid_acmd;
91 logic [2:0] acmd;
92 logic local_escalate;
93
94 // Flops.
95 logic cmd_ack_q, cmd_ack_d;
96 csrng_cmd_sts_e cmd_ack_sts_q, cmd_ack_sts_d;
97 logic [3:0] cmd_len_q, cmd_len_d;
98 logic cmd_gen_flag_q, cmd_gen_flag_d;
99 logic [11:0] cmd_gen_cmd_q, cmd_gen_cmd_d;
100 logic instantiated_d, instantiated_q;
101
102 always_ff @(posedge clk_i or negedge rst_ni) begin
103     if (!rst_ni) begin
104         cmd_ack_q <= '0;
105         cmd_ack_sts_q <= CMD_STS_SUCCESS;
106         cmd_len_q <= '0;
107         cmd_gen_flag_q <= '0;
108         cmd_gen_cmd_q <= '0;
109         instantiated_q <= '0;

```

```

110     end else begin
111         cmd_ack_q      <= cmd_ack_d;
112         cmd_ack_sts_q   <= cmd_ack_sts_d;
113         cmd_len_q       <= cmd_len_d;
114         cmd_gen_flag_q  <= cmd_gen_flag_d;
115         cmd_gen_cmd_q   <= cmd_gen_cmd_d;
116         instantiated_q  <= instantiated_d;
117     end
118 end
119
120 assign cmd_stage_sfifo_cmd_err_o = sfifo_cmd_err;
121 assign cmd_stage_sfifo_genbits_err_o = sfifo_genbits_err;
122
123 //-----
124 // Capture the transfer length of data behind the command.
125 //-----
126
127 prim_fifo_sync #(
128     .Width(CmdBusWidth),
129     .Pass(0),
130     .Depth(CmdFifoDepth),
131     .OutputZeroIfEmpty(1'b0)
132 ) u_prim_fifo_cmd (
133     .clk_i   (clk_i),
134     .rst_ni  (rst_ni),
135     .clr_i   (!cs_enable_i),
136     .wvalid_i(sfifo_cmd_wvld),
137     .wready_o(sfifo_cmd_wrdy),
138     .wdata_i (sfifo_cmd_wdata),
139     .rvalid_o(sfifo_cmd_rvld),
140     .rready_i(sfifo_cmd_rrdy),
141     .rdata_o (sfifo_cmd_rdata),
142     .full_o  (),
143     .depth_o (sfifo_cmd_depth),
144     .err_o   ()
145 );
146
147 assign sfifo_cmd_wdata = cmd_stage_bus_i;
148
149 assign sfifo_cmd_wvld = cs_enable_i && cmd_stage_rdy_o &&
cmd_stage_vld_i;
150
151 assign sfifo_cmd_rrdy = cs_enable_i && cmd_fifo_pop;
152
153 assign cmd_arb_bus_o =
154     cmd_gen_inc_req ?
155     {15'b0, cmd_gen_cnt_last, cmd_stage_shid_i, cmd_gen_cmd_q} :
156     // pad, glast, id, f, clen, cmd
157     cmd_gen_1st_req ?
158     {15'b0, cmd_gen_cnt_last, cmd_stage_shid_i, sfifo_cmd_rdata[11:0]} :
159     cmd_arb_mop_o ? sfifo_cmd_rdata :
160     '0;
161
162 assign cmd_stage_rdy_o = sfifo_cmd_wrdy;
163
164 assign sfifo_cmd_err =
165     {( sfifo_cmd_wvld && !sfifo_cmd_wrdy),
166      ( sfifo_cmd_rrdy && !sfifo_cmd_rvld),
167      (!sfifo_cmd_wrdy && !sfifo_cmd_rvld)};
168
169 // State machine controls.
170 assign cmd_fifo_zero = (sfifo_cmd_depth == '0);
171 assign cmd_len = sfifo_cmd_rdata[7:4];
172
173 // Capture the length of csrng command.
174 assign cmd_len_d =
175     (!cs_enable_i) ? '0 :
176     cmd_arb_sop_o ? cmd_len :
177     cmd_len_dec ? (cmd_len_q-1) :
178     cmd_len_q;
179
180 // Capture the application command type.
181 assign acmd = sfifo_cmd_rdata[2:0];
182
183 // For gen commands, capture information from the original
184 // command for use later.
185 assign cmd_gen_flag_d =
186     (!cs_enable_i) ? '0 :
187     cmd_gen_1st_req ? (acmd == GEN) :
188     cmd_gen_flag_q;
189
190 assign cmd_gen_cmd_d =
191     (!cs_enable_i) ? '0 :
192     cmd_gen_1st_req ? {sfifo_cmd_rdata[11:0]} :
193     cmd_gen_cmd_q;
194
195 // SEC_CM: GEN_CMD.CTR.REDUN
196 prim_count #(
197     .Width(GenBitsCtrWidth),
198     .ResetValue({GenBitsCtrWidth{1'b1}})
199 ) u_prim_count_cmd_gen_ctr (
200     .clk_i,
201     .rst_ni,
202     .clr_i(!cs_enable_i),
203     .set_i(cmd_gen_1st_req),
204     .set_cnt_i(sfifo_cmd_rdata[12+: GenBitsCtrWidth]),
205     .incr_en_i(1'b0),
206     .decr_en_i(cmd_gen_cnt_dec), // Count down.
207     .step_i(GenBitsCtrWidth'(1)),
208     .commit_i(1'b1),
209
210     .cnt_o(cmd_gen_cnt),
211     .cnt_after_commit_o(),
212     .err_o(cmd_gen_cnt_err_o)
213 );
214
215 // For naming consistency.
216 assign local_escalate = cmd_gen_cnt_err_o;
217
218 //-----
219 // state machine to process command
220 //-----
221 // Encoding generated with:
222 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 11 -n 8 \
223 // -s 170131814 --language=sv
224 //
225 // Hamming distance histogram:
226 //
227 // 0: --
228 // 1: --
229 // 2: --
230 // 3: ||||| (21.82%)
231 // 4: ||||| (36.36%)
232 // 5: ||||| (30.91%)
233 // 6: |||| (9.09%)
234 // 7: | (1.82%)
235 // 8: --
236
237 // Minimum Hamming distance: 3
238 // Maximum Hamming distance: 7
239 // Minimum Hamming weight: 1
240 // Maximum Hamming weight: 6
241
242 localparam int StateWidth = 8;
243 typedef enum logic [StateWidth-1:0] {
244     Idle      = 8'b11110101, // idle
245     Flush     = 8'b01011011, // flush command FIFO and start
246     over
247     ArbGnt    = 8'b00011100, // general arbiter request
248     SendSOP   = 8'b00000001, // send sop (start of packet)
249     SendMOP   = 8'b01010110, // send mop (middle of packet)
250     GenCmdChk = 8'b10001101, // gen cmd check
251     CmdAck    = 8'b10000000, // wait for command ack
252     GenReq    = 8'b10010011, // process gen requests
253     GenArbGnt = 8'b11101110, // generate subsequent arb re-
254     quest
255     GenSOP    = 8'b10111010, // generate subsequent request
256     Error     = 8'b01100111 // illegal state reached and hang
257 } state_e;
258
259 // SEC_CM: CMD_STAGE.FSM.SPARSE
260 state_e state_d, state_q;
261 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
262 state_e, Idle)
263
264 always_comb begin
265     state_d = state_q;
266     cmd_fifo_pop = 1'b0;
267     cmd_len_dec = 1'b0;
268     cmd_gen_cnt_dec = 1'b0;
269     cmd_gen_1st_req = 1'b0;
270     cmd_gen_inc_req = 1'b0;
271     cmd_gen_cnt_last = 1'b0;
272     cmd_final_ack = 1'b0;
273     cmd_arb_req_o = 1'b0;
274     cmd_arb_sop_o = 1'b0;
275     cmd_arb_mop_o = 1'b0;
276     cmd_arb_eop_o = 1'b0;
277     cmd_stage_sm_err_o = 1'b0;
278     cmd_err_ack = 1'b0;
279     reseed_cnt_exceeded = 1'b0;
280     invalid_cmd_seq = 1'b0;
281     invalid_acmd = 1'b0;
282     instantiated_d = instantiated_q;
283
284     if (state_q == Error) begin
285         // In case we are in the Error state we must ignore the
286         // local escalate and enable signals.
287         cmd_stage_sm_err_o = 1'b1;
288         end else if (local_escalate) begin
289             // In case local escalate is high we must transition to
290             // the error state.
291             state_d = Error;
292             end else if (!cs_enable_i && state_q inside {Idle, Flush,
293 ArbGnt, SendSOP, SendMOP, GenCmdChk,
294 CmdAck, Gen-
295 Req, GenArbGnt, GenSOP}) begin
296             // In case the module is disabled and we are in a legal
297             // state we must go into idle state.
298             state_d = Idle;
299             instantiated_d = 1'b0;
300             end else begin
301                 // Otherwise do the state machine as normal.
302                 unique case (state_q)
303                     Idle: begin
304                         // Because of the if statement above we won't leave
305                         // idle if enable is low.
306                         if (!cmd_fifo_zero) begin
307                             if (acmd == INS) begin
308                                 if (!instantiated_q) begin
309                                     state_d = ArbGnt;
310                                     instantiated_d = 1'b1;
311                                 end
312                             end
313                         end
314                     end
315                 end
316             end
317         end
318     end

```

```

299     end
300     if (instantiated_q) begin
301         cmd_err_ack = 1'b1;
302         invalid_cmd_seq = 1'b1;
303         state_d = Idle;
304     end
305 end else if (acmd == RES) begin
306     if (instantiated_q) begin
307         state_d = ArbGnt;
308     end
309     if (!instantiated_q) begin
310         cmd_err_ack = 1'b1;
311         invalid_cmd_seq = 1'b1;
312         state_d = Idle;
313     end
314 end else if (acmd == GEN) begin
315     if (instantiated_q) begin
316         // If the issued command is GEN and the reseed
count has already been reached,
317         // send an ack with an error status response.
318         if ((acmd == GEN) && reseed_cnt_reached_i) be-
gin
319             cmd_err_ack = 1'b1;
320             reseed_cnt_exceeded = 1'b1;
321             state_d = Idle;
322         end else begin
323             state_d = ArbGnt;
324         end
325     end
326     if (!instantiated_q) begin
327         cmd_err_ack = 1'b1;
328         invalid_cmd_seq = 1'b1;
329         state_d = Idle;
330     end
331 end else if (acmd == UPD) begin
332     if (instantiated_q) begin
333         state_d = ArbGnt;
334     end
335     if (!instantiated_q) begin
336         cmd_err_ack = 1'b1;
337         invalid_acmd = 1'b1;
338         state_d = Idle;
339     end
340 end else if (acmd == UNI) begin
341     // Set the instantiation to zero.
342     instantiated_d = 1'b0;
343     state_d = ArbGnt;
344 end else begin
345     // Command was not supported.
346     cmd_err_ack = 1'b1;
347     invalid_acmd = 1'b1;
348     state_d = Idle;
349 end
350 // If we received an invalid command, pop it from
the FIFO. Afterwards, absorb any
351 // additional data belonging to the same invalid
command and empty
the FIFO.
352 if (cmd_err_ack) begin
353     cmd_fifo_pop = 1'b1;
354     state_d = Flush;
355 end
356 end
357 end
358 Flush: begin
359     // Keep popping the FIFO until it's empty and we're
not getting new
command input.
360     // The decision whether a command is invalid is
taken based on the
command header but
361     // EDN might continue and send additional data be-
longing to the same
command. We have
362     // to absorb the full invalid command before we can
continue.
363     cmd_fifo_pop = sfifo_cmd_rvld;
364     if (!sfifo_cmd_rvld && !cmd_stage_vld_i) begin
365         state_d = Idle;
366     end
367 end
368 ArbGnt: begin
369     cmd_arb_req_o = 1'b1;
370     if (cmd_arb_gnt_i) begin
371         state_d = SendSOP;
372     end
373 end
374 SendSOP: begin
375     cmd_gen_1st_req = 1'b1;
376     cmd_arb_sop_o = 1'b1;
377     cmd_fifo_pop = 1'b1;
378     if (sfifo_cmd_rdata[12 +: GenBitsCtrWidth] == Gen-
BitsCtrWidth'(1)) begin
379         cmd_gen_cnt_last = 1'b1;
380     end
381     if (cmd_len == '0) begin
382         cmd_arb_eop_o = 1'b1;
383         state_d = GenCmdChk;
384     end else begin
385         state_d = SendMOP;
386     end
387 end
388 SendMOP: begin
389     if (!cmd_fifo_zero) begin

```

```

390         cmd_fifo_pop = 1'b1;
391         cmd_len_dec = 1'b1;
392         if (cmd_len_q == 4'h1) begin
393             cmd_arb_mop_o = 1'b1;
394             cmd_arb_eop_o = 1'b1;
395             state_d = GenCmdChk;
396         end else begin
397             cmd_arb_mop_o = 1'b1;
398         end
399     end
400 end
401 GenCmdChk: begin
402     if (cmd_gen_flag_q) begin
403         cmd_gen_cnt_dec = 1'b1;
404     end
405     state_d = CmdAck;
406 end
407 CmdAck: begin
408     if (cmd_ack_i) begin
409         // The state database has successfully been up-
dated.
410         // In case of Generate commands, we get the gener-
ated bits one clock cycle before
411         // receiving the ACK from the state database (from
csrng_ctr_drbg_gen).
412         state_d = GenReq;
413     end
414 end
415 GenReq: begin
416     // Flag set if a gen request.
417     if (cmd_gen_flag_q) begin
418         // Must stall if genbits fifo is not clear.
419         if (sfifo_genbits_wrdy) begin
420             if (cmd_gen_cnt == '0) begin
421                 cmd_final_ack = 1'b1;
422                 state_d = Idle;
423             end else begin
424                 // Issue a subsequent gen request.
425                 state_d = GenArbGnt;
426             end
427         end
428     end else begin
429         // Ack for the non-gen request case.
430         cmd_final_ack = 1'b1;
431         state_d = Idle;
432     end
433 end
434 GenArbGnt: begin
435     cmd_arb_req_o = 1'b1;
436     if (cmd_arb_gnt_i) begin
437         state_d = GenSOP;
438     end
439 end
440 GenSOP: begin
441     cmd_arb_sop_o = 1'b1;
442     cmd_arb_eop_o = 1'b1;
443     cmd_gen_inc_req = 1'b1;
444     state_d = GenCmdChk;
445     // Check for final genbits beat.
446     if (cmd_gen_cnt == GenBitsCtrWidth'(1)) begin
447         cmd_gen_cnt_last = 1'b1;
448     end
449 end
450 // Error: The error state is now covered by the if
statement above.
451 default: begin
452     state_d = Error;
453     cmd_stage_sm_err_o = 1'b1;
454 end
455 endcase // unique case (state_q)
456 end
457 end
458
459 //-----
460 // Genbits FIFO.
461 //-----
462
463 prim_fifo_sync #(
464     .Width(GenBitsFifoWidth),
465     .Pass(0),
466     .Depth(GenBitsFifoDepth),
467     .OutputZeroIfEmpty(0) // Set to 0, and let last data drive
out.
468 ) u_prim_fifo_genbits (
469     .clk_i (clk_i),
470     .rst_ni (rst_ni),
471     .clr_i (!cs_enable_i),
472     .wvalid_i(sfifo_genbits_wvld),
473     .wready_o(sfifo_genbits_wrdy),
474     .wdata_i(sfifo_genbits_wdata),
475     .rvalid_o(sfifo_genbits_rvld),
476     .rready_i(sfifo_genbits_rrdy),
477     .rdata_o (sfifo_genbits_rdata),
478     .full_o (),
479     .depth_o (), // sfifo_genbits_depth)
480     .err_o ()
481 );
482
483 assign sfifo_genbits_wdata = {genbits_fips_i, genbits_bus_i};

```

```

484
485 // The prim_fifo_sync primitive is constructed to only ac-
cept pushes if there is indeed space
486 // available. Backpressure would actually need to be handled
at the sender (csrng_ctr_drbg_gen).
487 // In this particular case, it is safe to unconditionally
push the genbits FIFO because
488 // the GenSOP FSM state which triggers csrng_ctr_drbg_gen to
generate bits can only be reached
489 // after checking that the genbits FIFO isn't full already.
This condition is checked using an
490 // SVA below.
491 //
492 // If the genbits FIFO got pushed without having space, this
either means the output of a genbits
493 // request is routed to the wrong application interface
(which would be a critical design bug)
494 // or that some fault injection attack is going on. Thus, we
track such cases both with an SVA
495 // and with a fatal alert (identifiable via the ERR_CODE re-
gister).
496 assign sfifo_genbits_wvld = cs_enable_i && genbits_vld_i;
497
498 assign sfifo_genbits_rrdy = genbits_vld_o && genbits_rdy_i;
499
500 assign genbits_vld_o = cs_enable_i && sfifo_genbits_rvld;
501 assign {genbits_fips_o, genbits_bus_o} =
sfifo_genbits_rdata;
502
503 assign sfifo_genbits_err =
504   {( sfifo_genbits_wvld && !sfifo_genbits_wrdy),
505     ( sfifo_genbits_rrdy && !sfifo_genbits_rvld),
506     (!sfifo_genbits_wrdy && !sfifo_genbits_rvld)};
507
508 // We're only allowed to request more bits if the genbits
FIFO has indeed space.
509 `ASSERT(CsrngCmdStageGenbitsFifoFull_A, state_q == GenSOP |>
> sfifo_genbits_wrdy)
510
511 // Pushes to the genbits FIFO outside of the GenCmdChk and
CmdAck states or while handling a
512 // command other than Generate are not allowed.
513 `ASSERT(CsrngCmdStageGenbitsFifoPushExpected_A,
514   sfifo_genbits_wvld |> state_q inside {GenCmdChk, Cm-
dAck} && cmd_gen_flag_q)
515
516 //-----
517 // Ack logic.
518 //-----
519
520 assign cmd_ack_d =
521   (!cs_enable_i) ? '0 :
522   cmd_final_ack || cmd_err_ack;
523
524 assign cmd_stage_ack_o = cmd_ack_q;
525
526 assign err_sts = reseed_cnt_exceeded ? CMD_STS_RE-
SEED_CNT_EXCEEDED :
527   invalid_cmd_seq ? CMD_STS_INVALID-
528   ID_CMD_SEQ :
529   invalid_acmd ? CMD_STS_INVALID-
530   ID_ACMD : CMD_STS_INVALID_ACMD;
531
532 assign cmd_ack_sts_d =
533   (!cs_enable_i) ? CMD_STS_SUCCESS :
534   cmd_err_ack ? err_sts :
535   cmd_final_ack ? cmd_ack_sts_i :
536   cmd_ack_sts_q;
537
538 assign cmd_stage_ack_sts_o = cmd_ack_sts_q;
539
540 assign reseed_cnt_alert_o = reseed_cnt_exceeded;
541 assign invalid_cmd_seq_alert_o = invalid_cmd_seq;
542 assign invalid_acmd_alert_o = invalid_acmd;
543
544 // Make sure that the state machine has a stable error
state. This means that after the error
545 // state is entered it will not exit it unless a reset sig-
nal is received.
546 `ASSERT(CsrngCmdStageErrorStStable_A, state_q == Error |>
$stable(state_q))
547 // If in error state, the error output must be high.
548 `ASSERT(CsrngCmdStageErrorOutput_A, state_q == Error |>
cmd_stage_sm_err_o)
549
550 endmodule

```

4. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_core.v (1,036 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //

```

```

5 // Description: csrng core module
6 //
7
8
9 module csrng_core import csrng_pkg::*; #(
10   parameter aes_pkg::sbox_impl_e SBoxImpl = aes_pkg::SBoxIm-
pLut,
11   parameter int NumHwApps = 2,
12   parameter cs_keymgr_div_t RndCnstCsKeymgrDivNonProduction =
CsKeymgrDivWidth'(0),
13   parameter cs_keymgr_div_t RndCnstCsKeymgrDivProduction = Cs-
KeymgrDivWidth'(0)
14 ) (
15   input logic clk_i,
16   input logic rst_ni,
17
18   input csrng_reg_pkg::csrng_reg2hw_t reg2hw,
19   output csrng_reg_pkg::csrng_hw2reg_t hw2reg,
20
21   // OTP Interface
22   input prim_mubi_pkg::mubi8_t otp_en_c-
srng_sw_app_read_i,
23
24   // Lifecycle broadcast inputs
25   input lc_ctrl_pkg::lc_tx_t lc_hw_de-
bug_en_i,
26
27   // Entropy Interface
28   output entropy_src_pkg::entropy_src_hw_if_req_t en-
tropy_src_hw_if_o,
29   input entropy_src_pkg::entropy_src_hw_if_rsp_t en-
tropy_src_hw_if_i,
30
31   // Application Interfaces
32   input csrng_req_t [NumHwApps-1:0] csrng_cmd_i,
33   output csrng_rsp_t [NumHwApps-1:0] csrng_cmd_o,
34
35   // Alerts
36   output logic re-
cov_alert_test_o,
37   output logic fatal_alert_test_o,
38   output logic re-
cov_alert_o,
39   output logic fatal_alert_o,
40
41   // Interrupts
42   output logic intr_cs_cmd_req_done_o,
43   output logic intr_cs_en-
tropy_req_o,
44   output logic intr_cs_hw_inst_exc_o,
45   output logic intr_cs_fatal_err_o
46 );
47
48 import csrng_reg_pkg::*;
49
50 import prim_mubi_pkg::mubi4_t;
51 import prim_mubi_pkg::mubi4_test_true_strict;
52 import prim_mubi_pkg::mubi4_test_invalid;
53
54 localparam int unsigned ADataDepthClog = $clog2(CmdMaxClen)
+ 1;
55 localparam int unsigned CsEnableCopies = 51;
56 localparam int unsigned LcHwDebugCopies = 1;
57 localparam int unsigned Flag0Copies = 3;
58
59 // signals
60 // interrupt signals
61 logic event_cs_fatal_err;
62 logic [CsEnableCopies-1:1] cs_enable_fo;
63 logic [Flag0Copies-1:0] flag0_fo;
64 logic acmd_flag0_pfa;
65 logic cs_enable_pfa;
66 logic sw_app_enable;
67 logic sw_app_enable_pfe;
68 logic sw_app_enable_pfa;
69 logic read_int_state;
70 logic read_int_state_pfe;
71 logic read_int_state_pfa;
72 logic fips_force_enable;
73 logic fips_force_enable_pfe;
74 logic fips_force_enable_pfa;
75 logic recov_alert_event;
76 logic acmd_avail;
77 logic acmd_sop;
78 logic acmd_mop;
79 logic acmd_eop;
80
81 logic state_db_wr_vld;
82 csrng_state_t state_db_rd_data;
83
84 logic [CmdBusWidth-1:0] acmd_e
acmd_hold;
85
86
87 logic [SeedLen-1:0] packer_adata;
88 logic [ADataDepthClog-1:0] packer_adata_depth;
89 logic packer_adata_pop;

```



```

90 logic packer_adata_clr;
91 logic [SeedLen-1:0] seed_diversification;
92
93 logic cmd_entropy_req;
94 logic cmd_entropy_avail;
95
96 logic ctr_drbg_req_vld;
97 logic ctr_drbg_req_rdy;
98 csrng_core_data_t ctr_drbg_req_data;
99
100 logic ctr_drbg_rsp_vld;
101 csrng_core_data_t ctr_drbg_rsp_data;
102 csrng_cmd_sts_e ctr_drbg_rsp_sts;
103
104 logic ctr_drbg_bits_vld;
105 logic [BlkLen-1:0] ctr_drbg_bits_data;
106 logic ctr_drbg_bits_fips;
107
108 logic acmd_accept;
109 logic main_sm_cmd_vld;
110 logic clr_adata_packer;
111
112 logic fifo_write_err_sum;
113 logic fifo_read_err_sum;
114 logic fifo_status_err_sum;
115 logic ctr_err_sum;
116
117 logic cmd_stage_sm_err_sum;
118 logic main_sm_err_sum;
119 logic cs_main_sm_err;
120 logic [MainSmStateWidth-1:0] cs_main_sm_state;
121 logic ctr_drbg_sm_err_sum;
122 logic ctr_drbg_sm_err;
123 logic ctr_drbg_v_ctr_err;
124 logic block_encrypt_sm_err_sum;
125 logic block_encrypt_sm_err;
126
127 // Signals to and from block cipher
128 logic block_encrypt_req_vld;
129 logic block_encrypt_req_rdy;
130 csrng_key_v_t block_encrypt_req_data;
131
132 logic block_encrypt_rsp_vld;
133 logic block_encrypt_rsp_rdy;
134 logic [BlkLen-1:0] block_encrypt_rsp_data;
135
136
137 logic [2:0] cmd_stage_sfifo_cmd_err[NumApps];
138 logic [NumApps-1:0] cmd_stage_sfifo_cmd_err_sum;
139 logic [NumApps-1:0] cmd_stage_sfifo_cmd_err_wr;
140 logic [NumApps-1:0] cmd_stage_sfifo_cmd_err_rd;
141 logic [NumApps-1:0] cmd_stage_sfifo_cmd_err_st;
142 logic [2:0] cmd_stage_sfifo_genbits_err[NumApps];
143 logic [NumApps-1:0] cmd_stage_sfifo_genbits_err_sum;
144 logic [NumApps-1:0] cmd_stage_sfifo_genbits_err_wr;
145 logic [NumApps-1:0] cmd_stage_sfifo_genbits_err_rd;
146 logic [NumApps-1:0] cmd_stage_sfifo_genbits_err_st;
147 logic [NumApps-1:0] cmd_stage_ctr_err;
148 logic [NumApps-1:0] cmd_stage_sm_err;
149
150 logic [NumApps-1:0] cmd_stage_vld;
151 logic [InstIdWidth-1:0] cmd_stage_shid[NumApps];
152 logic [CmdBusWidth-1:0] cmd_stage_bus[NumApps];
153 logic [NumApps-1:0] cmd_stage_rdy;
154 logic [NumApps-1:0] cmd_arb_req;
155 logic [NumApps-1:0] cmd_arb_gnt;
156 logic [$clog2(NumApps)-1:0] cmd_arb_idx;
157 logic [NumApps-1:0] cmd_arb_sop;
158 logic [NumApps-1:0] cmd_arb_mop;
159 logic [NumApps-1:0] cmd_arb_eop;
160 logic [CmdBusWidth-1:0] cmd_arb_bus[NumApps];
161 logic [NumApps-1:0] cmd_core_ack;
162 csrng_cmd_sts_e [NumApps-1:0] cmd_core_ack_sts;
163 logic [NumApps-1:0] cmd_stage_ack;
164 csrng_cmd_sts_e [NumApps-1:0] cmd_stage_ack_sts;
165 logic [NumApps-1:0] genbits_core_vld;
166 logic [BlkLen-1:0] genbits_core_bus[NumApps];
167 logic [NumApps-1:0] genbits_core_fips;
168 logic [NumApps-1:0] genbits_stage_vld;
169 logic [NumApps-1:0] genbits_stage_fips;
170 logic [BlkLen-1:0] genbits_stage_bus[NumApps];
171 logic [NumApps-1:0] genbits_stage_rdy;
172 logic genbits_stage_vld_sw;
173 logic genbits_stage_bus_rd_sw;
174 logic [31:0] genbits_stage_bus_sw;
175
176 logic [15:0] hw_exception_sts;
177 logic [LcHwDebugCopies-1:0] lc_hw_debug_on_fo;
178 logic state_db_reg_read_en;
179
180 logic [30:0] err_code_test_bit;
181
182 logic cs_rdata_capt_vld;
183 logic cs_bus_cmp_alert;
184 logic [NumApps-1:0] invalid_cmd_seq_alert;
185 logic [NumApps-1:0] invalid_acmd_alert;
186 logic [NumApps-1:0] reseed_cnt_alert;
187 logic [1:0] otp_sw_app_read_en;
188
189 logic [NumApps-1:0][31:0] reseed_counter;
190
191 prim_mubi_pkg::mubi8_t [1:0]
192 otp_sw_app_read_en_mubi;
193 prim_mubi_pkg::mubi4_t [CsEnableCopies-1:0] mubi_cs_en-
194 able_fanout;
195 prim_mubi_pkg::mubi4_t [Flag0Copies-1:0]
196 mubi_flag0_fanout;
197
198 // flops
199 acmd_e acmd_q, acmd_d;
200 logic [NumAppsLg-1:0] inst_id_q, inst_id_d;
201 logic gen_last_q, gen_last_d;
202 mubi4_t flag0_q, flag0_d;
203 logic [NumAppsLg-1:0] cmd_arb_idx_q, cmd_arb_idx_d;
204 logic genbits_stage_fips_sw_q, gen-
205 bits_stage_fips_sw_d;
206 acmd_e ctr_drbg_cmd_q, ctr_drbg_cmd_d;
207 logic [SeedLen-1:0] entropy_src_seed_q, en-
208 tropy_src_seed_d;
209 logic entropy_src_fips_q, en-
210 tropy_src_fips_d;
211 logic [63:0] cs_rdata_capt_q, cs_rdata_capt_d;
212 logic cs_rdata_capt_vld_q,
213 cs_rdata_capt_vld_d;
214 logic sw_rdy_sts_q, sw_rdy_sts_d;
215 logic sw_sts_ack_q, sw_sts_ack_d;
216 logic [NumApps-1:0] reseed_cnt_reached_q, re-
217 seed_cnt_reached_d;
218
219 always_ff @(posedge clk_i or negedge rst_ni) begin
220   if (!rst_ni) begin
221     acmd_q <= INV;
222     inst_id_q <= '0;
223     gen_last_q <= '0;
224     flag0_q <= prim_mubi_pkg::MuBi4False;
225     cmd_arb_idx_q <= '0;
226     genbits_stage_fips_sw_q <= '0;
227     ctr_drbg_cmd_q <= INV;
228     entropy_src_seed_q <= '0;
229     entropy_src_fips_q <= '0;
230     cs_rdata_capt_q <= '0;
231     cs_rdata_capt_vld_q <= '0;
232     sw_rdy_sts_q <= '0;
233     sw_sts_ack_q <= '0;
234     reseed_cnt_reached_q <= '0;
235   end else begin
236     acmd_q <= acmd_d;
237     inst_id_q <= inst_id_d;
238     gen_last_q <= gen_last_d;
239     flag0_q <= flag0_d;
240     cmd_arb_idx_q <= cmd_arb_idx_d;
241     genbits_stage_fips_sw_q <= genbits_stage_fips_sw_d;
242     ctr_drbg_cmd_q <= ctr_drbg_cmd_d;
243     entropy_src_seed_q <= entropy_src_seed_d;
244     entropy_src_fips_q <= entropy_src_fips_d;
245     cs_rdata_capt_q <= cs_rdata_capt_d;
246     cs_rdata_capt_vld_q <= cs_rdata_capt_vld_d;
247     sw_rdy_sts_q <= sw_rdy_sts_d;
248     sw_sts_ack_q <= sw_sts_ack_d;
249     reseed_cnt_reached_q <= reseed_cnt_reached_d;
250   end
251 end
252
253 //-----
254 // instantiate interrupt hardware primitives
255 //-----
256 // All TLUL interrupts are collect in the section.
257
258 prim_intr_hw #(
259   .Width(1)
260 ) u_intr_hw_cs_cmd_req_done (
261   .clk_i (clk_i),
262   .rst_ni (rst_ni),
263   .event_intr_i (cmd_stage_ack[NumApps-1]),
264   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.cs_cmd_req_done
265   .q),
266   .reg2hw_intr_test_q_i
267   (reg2hw.intr_test.cs_cmd_req_done.q),
268   .reg2hw_intr_test_qe_i
269   (reg2hw.intr_test.cs_cmd_req_done.qe),
270   .reg2hw_intr_state_q_i
271   (reg2hw.intr_state.cs_cmd_req_done.q),
272   .hw2reg_intr_state_de_o (hw2reg.intr_state.cs_cmd_req_done.
273   de),
274   .hw2reg_intr_state_d_o
275   (hw2reg.intr_state.cs_cmd_req_done.d),
276   .intr_o (intr_cs_cmd_req_done_o)
277 );
278
279 prim_intr_hw #(
280   .Width(1)
281 ) u_intr_hw_cs_entropy_req (
282   .clk_i (clk_i),
283   .rst_ni (rst_ni),
284   .event_intr_i (entropy_src_hw_if_o.es_req),
285   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.cs_entropy_req.
286   q),
287   .reg2hw_intr_test_q_i
288   (reg2hw.intr_test.cs_entropy_req.q),

```

```

273     .reg2hw_intr_test_qe_i
(reg2hw.intr_test.cs_entropy_req.qe),
274     .reg2hw_intr_state_q_i
(reg2hw.intr_state.cs_entropy_req.q),
275     .hw2reg_intr_state_de_o(hw2reg.intr_state.cs_entropy_req.
de),
276     .hw2reg_intr_state_d_o
(hw2reg.intr_state.cs_entropy_req.d),
277     .intr_o          (intr_cs_entropy_req_o)
278 );
279
280 prim_intr_hw #(
281     .Width(1)
282 ) u_intr_hw_cs_hw_inst_exc (
283     .clk_i          (clk_i),
284     .rst_ni         (rst_ni),
285     .event_intr_i   (|hw_exception_sts),
286     .reg2hw_intr_enable_q_i(reg2hw.intr_enable.cs_hw_inst_exc.
q),
287     .reg2hw_intr_test_q_i
(reg2hw.intr_test.cs_hw_inst_exc.q),
288     .reg2hw_intr_test_qe_i
(reg2hw.intr_test.cs_hw_inst_exc.qe),
289     .reg2hw_intr_state_q_i
(reg2hw.intr_state.cs_hw_inst_exc.q),
290     .hw2reg_intr_state_de_o(hw2reg.intr_state.cs_hw_inst_exc.
de),
291     .hw2reg_intr_state_d_o
(hw2reg.intr_state.cs_hw_inst_exc.d),
292     .intr_o          (intr_cs_hw_inst_exc_o)
293 );
294
295 prim_intr_hw #(
296     .Width(1)
297 ) u_intr_hw_cs_fatal_err (
298     .clk_i          (clk_i),
299     .rst_ni         (rst_ni),
300     .event_intr_i   (event_cs_fatal_err),
301     .reg2hw_intr_enable_q_i(reg2hw.intr_enable.cs_fatal_err.q)
302 ,
303     .reg2hw_intr_test_q_i (reg2hw.intr_test.cs_fatal_err.q),
304     .reg2hw_intr_test_qe_i (reg2hw.intr_test.cs_fatal_err.qe),
305     .reg2hw_intr_state_q_i (reg2hw.intr_state.cs_fatal_err.q),
306     .hw2reg_intr_state_de_o(hw2reg.intr_state.cs_fatal_err.de)
307 ,
308     .hw2reg_intr_state_d_o (hw2reg.intr_state.cs_fatal_err.d),
309     .intr_o          (intr_cs_fatal_err_o)
310 );
311 // Counter and FSM errors are structural errors and are al-
ways active regardless of the
312
313 // functional state. main_sm_err_sum is not included here to
prevent some tools from
314 // inferring combo loops. However, to include the state ma-
chine error for testing,
315 // we use the error code test bit (index 21) directly.
316 logic fatal_loc_events;
317 assign fatal_loc_events = block_encrypt_sm_err_sum ||
cmd_stage_sm_err_sum || ctr_drbg_sm_err_sum
318 || ctr_err_sum ||
err_code_test_bit[21];
319
320 // set the interrupt sources
321 assign event_cs_fatal_err = fatal_loc_events ||
main_sm_err_sum || (cs_enable_fo[1] &&
322 ((|cmd_stage_sfifo_cmd_err_sum)
|| (|cmd_stage_sfifo_genbits_err_sum)
|| fifo_write_err_sum ||
323 fifo_read_err_sum || fifo_status_err_sum));
324
325 // Individual error signals/sources
326 assign cmd_stage_sm_err_sum = (|cmd_stage_sm_err) ||
err_code_test_bit[20];
327 assign main_sm_err_sum = cs_main_sm_err ||
err_code_test_bit[21];
328 assign ctr_drbg_sm_err_sum = ctr_drbg_sm_err ||
err_code_test_bit[22];
329 assign block_encrypt_sm_err_sum = block_encrypt_sm_err ||
err_code_test_bit[25];
330
331 // Collective error signals for each type of error
332 assign ctr_err_sum = (|cmd_stage_ctr_err) ||
ctr_drbg_v_ctr_err || err_code_test_bit[26];
333 assign fifo_write_err_sum = (|
cmd_stage_sfifo_genbits_err_wr) || (|cmd_stage_sfifo_cmd_err_wr) ||
err_code_test_bit[28];
334 assign fifo_read_err_sum = (|
cmd_stage_sfifo_genbits_err_rd) || (|cmd_stage_sfifo_cmd_err_rd) ||
err_code_test_bit[29];
335 assign fifo_status_err_sum = (|
cmd_stage_sfifo_genbits_err_st) || (|cmd_stage_sfifo_cmd_err_st) ||
err_code_test_bit[30];
336
337 // set the err code source bits
338 assign hw2reg.err_code.sfifo_genbits_err.d = 1'b1;
339
340 assign hw2reg.err_code.sfifo_cmd_err.de = cs_enable_fo[2] &&
(|cmd_stage_sfifo_cmd_err_sum);
341
342

```

```

343 assign hw2reg.err_code.sfifo_genbits_err.d = 1'b1;
344 assign hw2reg.err_code.sfifo_genbits_err.de = cs_en-
able_fo[3] &&
345 (|cmd_stage_sfifo_genbits_err_sum);
346
347 assign hw2reg.err_code.cmd_stage_sm_err.d = 1'b1;
348 assign hw2reg.err_code.cmd_stage_sm_err.de = cs_en-
able_fo[19] && cmd_stage_sm_err_sum;
349
350 assign hw2reg.err_code.main_sm_err.d = 1'b1;
351 assign hw2reg.err_code.main_sm_err.de = cs_enable_fo[20] &&
main_sm_err_sum;
352
353 assign hw2reg.err_code.ctr_drbg_sm_err.d = 1'b1;
354
355 assign hw2reg.err_code.ctr_drbg_sm_err.de = cs_enable_fo[21]
&& ctr_drbg_sm_err_sum;
356
357 assign hw2reg.err_code.aes_cipher_sm_err.d = 1'b1;
358 assign hw2reg.err_code.aes_cipher_sm_err.de = cs_en-
able_fo[22] && block_encrypt_sm_err_sum;
359
360 // set the err code type bits
361 assign hw2reg.err_code.ctr_err.d = 1'b1;
362 assign hw2reg.err_code.ctr_err.de = cs_enable_fo[23] &&
ctr_err_sum;
363
364 assign hw2reg.err_code.fifo_write_err.d = 1'b1;
365 assign hw2reg.err_code.fifo_write_err.de = cs_enable_fo[24]
&& fifo_write_err_sum;
366
367 assign hw2reg.err_code.fifo_read_err.d = 1'b1;
368 assign hw2reg.err_code.fifo_read_err.de = cs_enable_fo[25]
&& fifo_read_err_sum;
369
370 assign hw2reg.err_code.fifo_state_err.d = 1'b1;
371 assign hw2reg.err_code.fifo_state_err.de = cs_enable_fo[26]
&& fifo_status_err_sum;
372
373 // Error testing decoding
374 for (genvar i = 0; i < 31; i++) begin :
gen_err_code_test_bit
375     assign err_code_test_bit[i] = (reg2hw.err_code_test.q ==
i) && reg2hw.err_code_test.qe;
376 end : gen_err_code_test_bit
377
378 // alert - send all interrupt sources to the alert for the
fatal case
379 assign fatal_alert_o = event_cs_fatal_err;
380
381 // alert test
382 assign recov_alert_test_o = {
383     reg2hw.alert_test.recov_alert.q &&
384     reg2hw.alert_test.recov_alert.qe
385 };
386 assign fatal_alert_test_o = {
387     reg2hw.alert_test.fatal_alert.q &&
388     reg2hw.alert_test.fatal_alert.qe
389 };
390
391 assign recov_alert_event = cs_enable_pfa ||
392 sw_app_enable_pfa ||
393 read_int_state_pfa ||
394 acmd_flag0_pfa ||
395 |reset_cnt_alert ||
396 |invalid_cmd_seq_alert ||
397 |invalid_acmd_alert ||
398 cs_bus_cmp_alert;
399
400
401 prim_edge_detector #(
402     .Width(1),
403     .ResetValue(0),
404     .EnSync(0)
405 ) u_prim_edge_detector_recov_alert (
406     .clk_i,
407     .rst_ni,
408     .d_i(recov_alert_event),
409     .q_sync_o(),
410     .q_posedge_pulse_o(recov_alert_o),
411     .q_negedge_pulse_o()
412 );
413
414 // check for illegal enable field states, and set alert if
detected
415
416 // SEC_CM: CONFIG.MUBI
417 mubi4_t mubi_cs_enable;
418 assign mubi_cs_enable = mubi4_t'(reg2hw.ctrl.enable.q);
419 assign cs_enable_pfa = mu-
bi4_test_invalid(mubi_cs_enable.fanout[0]);
420 assign hw2reg.recov_alert_sts.enable_field_alert.de = cs_en-
able_pfa;
421 assign hw2reg.recov_alert_sts.enable_field_alert.d = cs_en-
able_pfa;
422
423 for (genvar i = 1; i < CsEnableCopies; i = i+1) begin :
gen_mubi_en_copies
424     assign cs_enable_fo[i] = mu-

```



```

bi4_test_true_strict(mubi_cs_enable_fanout[i]);
426   end : gen_mubi_en_copies
427
428   prim_mubi4_sync #(
429     .NumCopies(CsEnableCopies),
430     .AsyncOn(0)
431   ) u_prim_mubi4_sync_cs_enable (
432     .clk_i,
433     .rst_ni,
434     .mubi_i(mubi_cs_enable),
435     .mubi_o(mubi_cs_enable_fanout)
436   );
437
438   // SEC_CM: CONFIG.MUBI
439   mubi4_t mubi_sw_app_enable;
440   mubi4_t [1:0] mubi_sw_app_enable_fanout;
441   assign mubi_sw_app_enable = mu-
bi4_t'(reg2hw.ctrl.sw_app_enable.q);
442   assign sw_app_enable_pfe = mu-
bi4_test_true_strict(mubi_sw_app_enable_fanout[0]);
443   assign sw_app_enable_pfa = mu-
bi4_test_invalid(mubi_sw_app_enable_fanout[1]);
444   assign hw2reg.recov_alert_sts.sw_app_enable_field_alert.de =
sw_app_enable_pfa;
445   assign hw2reg.recov_alert_sts.sw_app_enable_field_alert.d =
sw_app_enable_pfa;
446
447   prim_mubi4_sync #(
448     .NumCopies(2),
449     .AsyncOn(0)
450   ) u_prim_mubi4_sync_sw_app_enable (
451     .clk_i,
452     .rst_ni,
453     .mubi_i(mubi_sw_app_enable),
454     .mubi_o(mubi_sw_app_enable_fanout)
455   );
456
457   // SEC_CM: CONFIG.MUBI
458   mubi4_t mubi_read_int_state;
459   mubi4_t [1:0] mubi_read_int_state_fanout;
460   assign mubi_read_int_state = mu-
bi4_t'(reg2hw.ctrl.read_int_state.q);
461   assign read_int_state_pfe = mu-
bi4_test_true_strict(mubi_read_int_state_fanout[0]);
462   assign read_int_state_pfa = mu-
bi4_test_invalid(mubi_read_int_state_fanout[1]);
463   assign hw2reg.recov_alert_sts.read_int_state_field_alert.de =
read_int_state_pfa;
464   assign hw2reg.recov_alert_sts.read_int_state_field_alert.d =
read_int_state_pfa;
465
466   prim_mubi4_sync #(
467     .NumCopies(2),
468     .AsyncOn(0)
469   ) u_prim_mubi4_sync_read_int_state (
470     .clk_i,
471     .rst_ni,
472     .mubi_i(mubi_read_int_state),
473     .mubi_o(mubi_read_int_state_fanout)
474   );
475
476   // SEC_CM: CONFIG.MUBI
477   mubi4_t mubi_fips_force_enable;
478   mubi4_t [1:0] mubi_fips_force_enable_fanout;
479   assign mubi_fips_force_enable = mu-
bi4_t'(reg2hw.ctrl.fips_force_enable.q);
480   assign fips_force_enable_pfe = mu-
bi4_test_true_strict(mubi_fips_force_enable_fanout[0]);
481   assign fips_force_enable_pfa = mu-
bi4_test_invalid(mubi_fips_force_enable_fanout[1]);
482   assign
hw2reg.recov_alert_sts.fips_force_enable_field_alert.de =
fips_force_enable_pfa;
483   assign
hw2reg.recov_alert_sts.fips_force_enable_field_alert.d =
fips_force_enable_pfa;
484
485   prim_mubi4_sync #(
486     .NumCopies(2),
487     .AsyncOn(0)
488   ) u_prim_mubi4_sync_fips_force_enable (
489     .clk_i,
490     .rst_ni,
491     .mubi_i(mubi_fips_force_enable),
492     .mubi_o(mubi_fips_force_enable_fanout)
493   );
494
495   // master module enable
496   assign sw_app_enable = sw_app_enable_pfe;
497   assign read_int_state = read_int_state_pfe;
498   assign fips_force_enable = fips_force_enable_pfe;
499
500   //-----
501   // application interface
502   //-----
503   // Each application port has its own
504   // csrng_cmd_stage block to receive the
505   // command, track the state of its completion,
506   // and return any genbits if the command

```

```

507   // is a generate command.
508
509   for (genvar ai = 0; ai < NumApps; ai = ai+1) begin :
gen_cmd_stage
510
511     csrng_cmd_stage u_csrng_cmd_stage (
512       .clk_i (clk_i),
513       .rst_ni (rst_ni),
514       .cs_enable_i (cs_enable_fo[27]),
515       .cmd_stage_vld_i (cmd_stage_vld[ai]),
516       .cmd_stage_shid_i (cmd_stage_shid[ai]),
517       .cmd_stage_bus_i (cmd_stage_bus[ai]),
518       .cmd_stage_rdy_o (cmd_stage_rdy[ai]),
519       .reseed_cnt_reached_i (re-
seed_cnt_reached_q[ai]),
520       .reseed_cnt_alert_o (reseed_cnt_alert[ai]),
521       .invalid_cmd_seq_alert_o (inval-
id_cmd_seq_alert[ai]),
522       .invalid_acmd_alert_o (invalid_acmd_alert[ai]),
523       .cmd_arb_req_o (cmd_arb_req[ai]),
524       .cmd_arb_sop_o (cmd_arb_sop[ai]),
525       .cmd_arb_mop_o (cmd_arb_mop[ai]),
526       .cmd_arb_eop_o (cmd_arb_eop[ai]),
527       .cmd_arb_gnt_i (cmd_arb_gnt[ai]),
528       .cmd_arb_bus_o (cmd_arb_bus[ai]),
529       .cmd_ack_i (cmd_core_ack[ai]),
530       .cmd_ack_sts_i (cmd_core_ack_sts[ai]),
531       .cmd_stage_ack_o (cmd_stage_ack[ai]),
532       .cmd_stage_ack_sts_o (cmd_stage_ack_sts[ai]),
533       .genbits_vld_i (genbits_core_vld[ai]),
534       .genbits_bus_i (genbits_core_bus[ai]),
535       .genbits_fips_i (genbits_core_fips[ai]),
536       .genbits_vld_o (genbits_stage_vld[ai]),
537       .genbits_rdy_i (genbits_stage_rdy[ai]),
538       .genbits_bus_o (genbits_stage_bus[ai]),
539       .genbits_fips_o (genbits_stage_fips[ai]),
540       .cmd_stage_sfifo_cmd_err_o (cmd_stage_sfifo_cmd_err[ai]),
541       .cmd_stage_sfifo_genbits_err_o (cmd_stage_sfifo_genbits-
err[ai]),
542       .cmd_gen_cnt_err_o (cmd_stage_ctr_err[ai]),
543       .cmd_stage_sm_err_o (cmd_stage_sm_err[ai])
544     );
545
546     // Set reseed_cnt_reached_d to true if the max number of
generate requests between reseeds
547     // has been reached for the respective counter.
548     assign reseed_cnt_reached_d[ai] = (ctr_dbg_rsp_vld &&
(ctr_dbg_rsp_data.inst_id == ai)) ?
549
550     (ctr_dbg_rsp_data.rs_ctr >= reg2hw.reseed_interval.q) :
re-
seed_cnt_reached_q[ai];
551
552   end : gen_cmd_stage
553
554   // SW interface connection (only 1, and must be present)
555   // cmd req
556   assign cmd_stage_vld[NumApps-1] = reg2hw.cmd_req.q;
557   assign cmd_stage_bus[NumApps-1] = reg2hw.cmd_req.q;
558   assign cmd_stage_shid[NumApps-1] = InstIdWidth'(NumApps-1);
559
560   assign hw2reg.sw_cmd_sts.cmd_rdy.de = 1'b1;
561   assign hw2reg.sw_cmd_sts.cmd_rdy.d = !
cmd_stage_vld[NumApps-1] && sw_rdy_sts.q;
562
563   assign sw_rdy_sts_d =
564     !cs_enable_fo[28] ? 1'b0 :
565     cmd_stage_vld[NumApps-1] ? 1'b0 :
566     cmd_stage_rdy[NumApps-1] ? 1'b1 :
567     sw_rdy_sts.q;
568
569   // cmd sts ack
570   assign hw2reg.sw_cmd_sts.cmd_ack.de = 1'b1;
571   assign hw2reg.sw_cmd_sts.cmd_ack.d = sw_sts_ack_d;
572   assign sw_sts_ack_d =
573     !cs_enable_fo[28] ? 1'b0 :
574     cmd_stage_vld[NumApps-1] ? 1'b0 :
575     cmd_stage_ack[NumApps-1] ? 1'b1 :
576     sw_sts_ack.q;
577   // cmd ack sts
578   assign hw2reg.sw_cmd_sts.cmd_sts.de =
cmd_stage_ack[NumApps-1];
579   assign hw2reg.sw_cmd_sts.cmd_sts.d =
cmd_stage_ack_sts[NumApps-1];
580   // genbits
581   assign hw2reg.genbits_vld.genbits_vld.d = gen-
bits_stage_vld_sw;
582   assign hw2reg.genbits_vld.genbits_fips.d = gen-
bits_stage_fips_sw;
583   assign hw2reg.genbits.d = (sw_app_enable &&
otp_sw_app_read_en[0]) ? genbits_stage_bus_sw : '0;
584   assign genbits_stage_bus_rd_sw = reg2hw.genbits.re;
585
586   assign otp_sw_app_read_en[0] =
prim_mubi_pkg:mubi8_test_true_strict(otp_sw_app_read_en_mubi[0]);
587   assign otp_sw_app_read_en[1] =
prim_mubi_pkg:mubi8_test_true_strict(otp_sw_app_read_en_mubi[1]);
588
589   prim_mubi8_sync #(
590     .NumCopies(2),

```

```

591 .AsyncOn(1)
592 ) u_prim_mubi8_sync_sw_app_read (
593   .clk_i,
594   .rst_ni,
595   .mubi_i(otp_en_csrng_sw_app_read_i),
596   .mubi_o(otp_sw_app_read_en_mubi)
597 );
598
599 // pack the gen bits into a 32 bit register sized word
600 prim_packer_fifo #(
601   .InW(Blklen),
602   .OutW(32),
603   .ClearOnRead(1'b0)
604 ) u_prim_packer_fifo_sw_genbits (
605   .clk_i (clk_i),
606   .rst_ni (rst_ni),
607   .clr_i (!cs_enable_fo[29]),
608   .wvalid_i(genbits_stage_vld[NumApps-1]),
609   .wdata_i (genbits_stage_bus[NumApps-1]),
610   .wready_o(genbits_stage_rdy[NumApps-1]),
611   .rvalid_o(genbits_stage_vld_sw),
612   .rdata_o (genbits_stage_bus_sw),
613   .rready_i(genbits_stage_bus_rd_sw),
614   .depth_o ()
615 );
616
617 // flops for SW fips status
618 assign genbits_stage_fips_sw_d =
619   (!cs_enable_fo[30]) ? 1'b0 :
620   (genbits_stage_rdy[NumApps-1] && gen-
bits_stage_vld[NumApps-1]) ?
621   gen-
bits_stage_fips[NumApps-1] :
622   gen-
bits_stage_fips_sw_q;
623
624 //-----
625 // data path integrity check
626 // - a countermeasure to detect entropy bus tampering at-
tempts
627 // - checks to make sure repeated data sets off an alert for
sw to handle
628 //-----
629
630 // SEC-CM: SW_GENBITS.BUS.CONSTENCY
631
632 // capture a copy of the genbits data
633 assign cs_rdata_capt_vld = (genbits_stage_vld[NumApps-1] &&
genbits_stage_rdy[NumApps-1]);
634
635 assign cs_rdata_capt_d = cs_rdata_capt_vld ? gen-
bits_stage_bus[NumApps-1][63:0] : cs_rdata_capt_q;
636
637 assign cs_rdata_capt_vld_d =
638   !cs_enable_fo[31] ? 1'b0 :
639   cs_rdata_capt_vld ? 1'b1 :
640   cs_rdata_capt_vld_q;
641
642 // continuous compare of the entropy data for sw port
643 assign cs_bus_cmp_alert = cs_rdata_capt_vld &&
cs_rdata_capt_vld_q &&
644   (cs_rdata_capt_q == genbits_stage_bus[NumApps-1]
[63:0]); // only look at 64 bits
645
646 assign hw2reg.recov_alert_sts.cs_bus_cmp_alert.de =
cs_bus_cmp_alert;
647 assign hw2reg.recov_alert_sts.cs_bus_cmp_alert.d =
cs_bus_cmp_alert;
648
649 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_acmd_alert.de = |inval-
id_acmd_alert;
650 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_acmd_alert.d = |inval-
id_acmd_alert;
651
652 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_cmd_seq_alert.de = |inval-
id_cmd_seq_alert;
653 assign
hw2reg.recov_alert_sts.cmd_stage_invalid_cmd_seq_alert.d = |inval-
id_cmd_seq_alert;
654
655 assign hw2reg.recov_alert_sts.cmd_stage_reseed_cnt_alert.de =
|reseed_cnt_alert;
656 assign hw2reg.recov_alert_sts.cmd_stage_reseed_cnt_alert.d =
|reseed_cnt_alert;
657
658 // HW interface connections (up to 16, numbered 0-14)
659 for (genvar hai = 0; hai < (NumApps-1); hai = hai+1) begin :
gen_app_if
660   // cmd req
661   assign cmd_stage_vld[hai] =
csrng_cmd_i[hai].csrng_req_valid;
662   assign cmd_stage_shid[hai] = hai;
663   assign cmd_stage_bus[hai] =
csrng_cmd_i[hai].csrng_req_bus;
664   assign csrng_cmd_o[hai].csrng_req_ready =
cmd_stage_rdy[hai];

```

```

665   // cmd ack
666   assign csrng_cmd_o[hai].csrng_rsp_ack =
cmd_stage_ack[hai];
667   assign csrng_cmd_o[hai].csrng_rsp_sts =
cmd_stage_ack_sts[hai];
668   // genbits
669   assign csrng_cmd_o[hai].genbits_valid = gen-
bits_stage_vld[hai];
670   assign csrng_cmd_o[hai].genbits_fips = gen-
bits_stage_fips[hai];
671   assign csrng_cmd_o[hai].genbits_bus = gen-
bits_stage_bus[hai];
672   assign genbits_stage_rdy[hai] =
csrng_cmd_i[hai].genbits_ready;
673   end : gen_app_if
674
675   // Assign ack status
676   for (genvar i = 0; i < 16; i = i+1) begin : gen_app_if_sts
677     if (i < NumApps) begin : gen_assign
678       assign hw_exception_sts[i] = cmd_stage_ack[i] &&
(cmd_stage_ack_sts[i] != CMD_STS_SUCCESS);
679     end else begin : gen_dummy_assign
680       assign hw_exception_sts[i] = 1'b0;
681     end
682   end : gen_app_if_sts
683
684   // set fifo err status bits
685   for (genvar i = 0; i < NumApps; i = i+1) begin :
gen_fifo_sts
686     assign cmd_stage_sfifo_cmd_err_sum[i] = (|
cmd_stage_sfifo_cmd_err[i] ||
687     err_code_test_bit[0]);
688     assign cmd_stage_sfifo_cmd_err_wr[i] =
cmd_stage_sfifo_cmd_err[i][2];
689     assign cmd_stage_sfifo_cmd_err_rd[i] =
cmd_stage_sfifo_cmd_err[i][1];
690     assign cmd_stage_sfifo_cmd_err_st[i] =
cmd_stage_sfifo_cmd_err[i][0];
691     assign cmd_stage_sfifo_genbits_err_sum[i] = (|
cmd_stage_sfifo_genbits_err[i] ||
692     err_code_test_bit[1]);
693     assign cmd_stage_sfifo_genbits_err_wr[i] =
cmd_stage_sfifo_genbits_err[i][2];
694     assign cmd_stage_sfifo_genbits_err_rd[i] =
cmd_stage_sfifo_genbits_err[i][1];
695     assign cmd_stage_sfifo_genbits_err_st[i] =
cmd_stage_sfifo_genbits_err[i][0];
696   end : gen_fifo_sts
697
698   //-----
699   // app command arbiter and state machine
700   //-----
701   // All commands that arrive from the
702   // application ports are arbitrated for
703   // and processed by the main state machine
704   // logic block.
705
706   assign cmd_arb_idx_d = (acmd_avail && acmd_accept) ?
cmd_arb_idx : cmd_arb_idx_q;
707
708   assign acmd_sop = cmd_arb_sop[cmd_arb_idx_q]; // Start-of-
Packet
709   assign acmd_mop = cmd_arb_mop[cmd_arb_idx_q]; // Mid-of-
Packet
710   assign acmd_eop = cmd_arb_eop[cmd_arb_idx_q]; // End-of-
Packet
711   assign acmd_bus = cmd_arb_bus[cmd_arb_idx_q]; // Actual bus
data; content changes with s/m/eop
712
713   prim_arbiter_ppc #(
714     .EnDataPort(0), // Ignore data port
715     .N(NumApps), // Number of request ports
716     .DW(1) // Data width
717   ) u_prim_arbiter_ppc_acmd (
718     .clk_i (clk_i),
719     .rst_ni (rst_ni),
720     .req_chk_i(cs_enable_fo[1]),
721     .req_i (cmd_arb_req),
722     .data_i ('{default: 1'b0}),
723     .gnt_o (cmd_arb_gnt),
724     .idx_o (cmd_arb_idx),
725     .valid_o (acmd_avail), // 1 req
726     .data_o (), //NC
727     .ready_i (acmd_accept) // 1 fsm rdy
728   );
729
730   assign acmd_flag0_pfa = mubi4_test_invalid(flag0_q);
731   assign hw2reg.recov_alert_sts.acmd_flag0_field_alert.de =
acmd_flag0_pfa;
732   assign hw2reg.recov_alert_sts.acmd_flag0_field_alert.d =
acmd_flag0_pfa;
733
734   // parse the command bus
735   assign acmd_hold = acmd_sop ? acmd_e'(acmd_bus[Cmd-
width-1:0]) : acmd_q;
736
737   // TODO(#28153) rewrite as an always_comb block
738   assign acmd_d =
739     (!cs_enable_fo[32]) ? INV :

```

```

740     acmd_sop ? acmd_e'(acmd_bus[CmdWidth-1:0]) :
741     acmd_q;
742
743     assign inst_id_d =
744         (!cs_enable_fo[33]) ? '0 :
745         acmd_sop ? acmd_bus[12+: NumAppsLg] :
746         inst_id_q;
747
748     assign gen_last_d =
749         (!cs_enable_fo[34]) ? '0 :
750         acmd_sop ? acmd_bus[16] :
751         gen_last_q;
752
753     assign flag0_d =
754         (!cs_enable_fo[35]) ? prim_mubi_pkg::MuBi4False :
755         (acmd_sop && ((acmd_bus[2:0] == INS) ||
756         (acmd_bus[2:0] == RES))) ?
757         mubi4_t'(acmd_bus[11:8]) : flag0_q;
758
759     // SEC_CM: CTRL.MUBI
760     mubi4_t mubi_flag0;
761     assign mubi_flag0 = flag0_q;
762
763     for (genvar i = 0; i < Flag0Copies; i = i+1) begin :
764     gen_mubi_flag0_copies
765         assign flag0_fo[i] = mu-
766         bi4_test_true_strict(mubi_flag0_fanout[i]);
767     end : gen_mubi_flag0_copies
768
769     prim_mubi4_sync #(
770         .NumCopies(Flag0Copies),
771         .AsyncOn(0)
772     ) u_prim_mubi4_sync_flag0 (
773         .clk_i,
774         .rst_ni,
775         .mubi_i(mubi_flag0),
776         .mubi_o(mubi_flag0_fanout)
777     );
778
779     // sm to process all instantiation requests
780     // SEC_CM: MAIN.SM.CTR.LOCAL_ESC
781     csrng_main_sm u_csrng_main_sm (
782         .clk_i (clk_i),
783         .rst_ni (rst_ni),
784         .enable_i (cs_enable_fo[36]),
785         .acmd_avail_i (acmd_avail),
786         .acmd_accept_o (acmd_accept),
787         .acmd_i (acmd_hold),
788         .acmd_eop_i (acmd_eop),
789         .flag0_i (flag0_fo[0]),
790         .cmd_entropy_req_o (cmd_entropy_req),
791         .cmd_entropy_avail_i (cmd_entropy_avail),
792         .cmd_vld_o (main_sm_cmd_vld),
793         .cmd_rdy_i (ctr_drbg_req_rdy),
794         .clr_adata_packer_o (clr_adata_packer),
795         .cmd_complete_i (ctr_drbg_rsp_vld),
796         .local_escalate_i (fatal_loc_events),
797         .main_sm_state_o (cs_main_sm_state),
798         .main_sm_err_o (cs_main_sm_err)
799     );
800
801     // entropy available
802     assign cmd_entropy_avail = entropy_src_hw_if_i.es_ack;
803
804     for (genvar csi = 0; csi < NumApps; csi = csi+1) begin :
805     gen_cmd_ack
806         assign cmd_core_ack[csi] = ctr_drbg_rsp_vld &&
807         (ctr_drbg_rsp_data.inst_id == csi);
808         assign cmd_core_ack_sts[csi] = ctr_drbg_rsp_sts;
809         assign genbits_core_vld[csi] = ctr_drbg_bits_vld &&
810         (ctr_drbg_rsp_data.inst_id == csi);
811         assign genbits_core_bus[csi] = ctr_drbg_bits_data;
812         assign genbits_core_fips[csi] = ctr_drbg_bits_fips;
813     end : gen_cmd_ack
814
815     prim_packer_fifo #(
816         .InW(32),
817         .OutW(SeedLen),
818         .ClearOnRead(1'b1)
819     ) u_prim_packer_fifo_adata (
820         .clk_i (clk_i),
821         .rst_ni (rst_ni),
822         .clr_i (!cs_enable_fo[37] || packer_adata_clr),
823         .wvalid_i(acmd_mop),
824         .wdata_i (acmd_bus),
825         .wready_o(),
826         .rvalid_o(),
827         .rdata_o (packer_adata),
828         .rready_i(packer_adata_pop),
829         .depth_o (packer_adata_depth)
830     );
831
832     assign packer_adata_pop = cs_enable_fo[38] &&
833     clr_adata_packer && (packer_adata_depth == AData-
834     aDepthClog'(CmdMaxClen));
835
836     assign packer_adata_clr = cs_enable_fo[39] &&
837     clr_adata_packer && (packer_adata_depth < ADataDepth-
838     Clog'(CmdMaxClen));
839

```

```

833     //-----
834     // csrng_state_db instantiation
835     //-----
836     // Holds the internal state of each csrng instance. Gets up-
837     dated after every command.
838     assign state_db_reg_read_en = cs_enable_fo[40] &&
839     read_int_state && otp_sw_app_read_en[1];
840
841     csrng_state_db u_csrng_state_db (
842         .clk_i (clk_i),
843         .rst_ni (rst_ni),
844         .enable_i(cs_enable_fo[41]),
845         .rd_inst_id_i(inst_id_q),
846         .rd_state_o (state_db_rd_data),
847         .wr_vld_i (state_db_wr_vld),
848         .wr_data_i (ctr_drbg_rsp_data),
849         .reg_rd_otp_en_i (state_db_reg_read_en),
850         .reg_rd_regfile_en_i(reg2hw.int_state_read_enable.q),
851         .reg_rd_id_vld_i(reg2hw.int_state_num.qe),
852         .reg_rd_id_i (reg2hw.int_state_num.q),
853         .reg_rd_strb_i (reg2hw.int_state_val.re),
854         .reg_rd_val_o (hw2reg.int_state_val.d),
855         .reseed_counter_o(reseed_counter)
856     );
857
858     // Forward the reseed counter values to the register inter-
859     face.
860     always_comb begin : reseed_counter_assign
861         for (int i = 0; i < NumApps; i++) begin
862             hw2reg.reseed_counter[i].d = reseed_counter[i];
863         end
864     end
865
866     //-----
867     // entropy interface
868     //-----
869     // Basic interface logic with the entropy_src block
870
871     assign entropy_src_hw_if_o.es_req = cs_enable_fo[43] &&
872     cmd_entropy_req;
873
874     // SEC_CM: CONSTANTS.LC.GATED
875     assign seed_diversification = lc_hw_debug_on_fo[0] ? RndCn-
876     stCsKeymgrDivNonProduction :
877     RndCn-
878     stCsKeymgrDivProduction;
879
880     // Capture entropy from entropy_src
881     assign entropy_src_seed_d =
882     flag0_fo[1] ? '0 : // special case where zero is used
883     cmd_entropy_req && cmd_entropy_avail ?
884     (entropy_src_hw_if_i.es_bits ^ seed_diversifica-
885     tion) :
886     entropy_src_seed_q;
887     assign entropy_src_fips_d =
888     // Use inst_id_d here such that u_csrng_ctr_drbg_cmd
889     gets the inst_id_q and the proper
890     // entropy_src_fips_q in the next clock cycle.
891     fips_force_enable && reg2hw.fips_force.q[inst_id_d] ?
892     1'b1 :
893     flag0_fo[2] ? '0 : // special case where zero is used
894     cmd_entropy_req && cmd_entropy_avail ? en-
895     tropy_src_hw_if_i.es_fips :
896     entropy_src_fips_q;
897
898     //-----
899     // CTR DRBG instantiation
900     //-----
901
902     assign ctr_drbg_req_vld = !cs_enable_fo[45] ? 1'b0 :
903     main_sm_cmd_vld;
904     assign ctr_drbg_cmd_d = !cs_enable_fo[44] ? INV :
905     acmd_hold;
906
907     assign ctr_drbg_req_data = '{
908         inst_id: inst_id_q,
909         cmd: ctr_drbg_cmd_q,
910         key: state_db_rd_data.key,
911         v: state_db_rd_data.v,
912         pdata: packer_adata,
913         rs_ctr: state_db_rd_data.rs_ctr,
914         fips: state_db_rd_data.fips
915     };
916
917     csrng_ctr_drbg u_csrng_ctr_drbg (
918         .clk_i (clk_i),
919         .rst_ni (rst_ni),
920         .enable_i(cs_enable_fo[49]),
921         .req_vld_i (ctr_drbg_req_vld),
922         .req_rdy_o (ctr_drbg_req_rdy),
923         .req_data_i (ctr_drbg_req_data),
924         .req_entropy_i (entropy_src_seed_q),

```

```

921 .req_entropy_fips_i(entropy_src_fips_q),
922 .req_glast_i      (gen_last_q),
923
924 .rsp_vld_o (ctr_drbg_rsp_vld),
925 .rsp_data_o(ctr_drbg_rsp_data),
926 .rsp_sts_o (ctr_drbg_rsp_sts),
927
928 .bits_vld_o (ctr_drbg_bits_vld),
929 .bits_data_o(ctr_drbg_bits_data),
930 .bits_fips_o(ctr_drbg_bits_fips),
931
932 .state_db_wr_o(state_db_wr_vld),
933
934 .block_encrypt_req_vld_o (block_encrypt_req_vld),
935 .block_encrypt_req_rdy_i (block_encrypt_req_rdy),
936 .block_encrypt_req_data_o(block_encrypt_req_data),
937
938 .block_encrypt_rsp_vld_i (block_encrypt_rsp_vld),
939 .block_encrypt_rsp_rdy_o (block_encrypt_rsp_rdy),
940 .block_encrypt_rsp_data_i(block_encrypt_rsp_data),
941
942 .ctr_err_o(ctr_drbg_v_ctr_err),
943 .sm_err_o (ctr_drbg_sm_err)
944 );
945
946 //-----
947 // csrng_block_encrypt instantiation
948 //-----
949
950 csrng_block_encrypt #(
951   .SBoxImpl(SBoxImpl)
952 ) u_csrng_block_encrypt (
953   .clk_i   (clk_i),
954   .rst_ni  (rst_ni),
955   .enable_i(cs_enable_fo[48]),
956
957   .req_vld_i (block_encrypt_req_vld),
958   .req_rdy_o (block_encrypt_req_rdy),
959   .req_data_i(block_encrypt_req_data),
960
961   .rsp_vld_o (block_encrypt_rsp_vld),
962   .rsp_rdy_i (block_encrypt_rsp_rdy),
963   .rsp_data_o(block_encrypt_rsp_data),
964
965   .cipher_sm_err_o(block_encrypt_sm_err)
966 );
967
968 //-----
969 // life cycle logic
970 //-----
971 // The chip level life cycle control
972 // provide control logic to determine
973 // how certain debug features are controlled.
974
975 lc_ctrl_pkg::lc_tx_t [LcHwDebugCopies-1:0] lc_hw_de-
bug_en_out;
976
977 prim_lc_sync #(
978   .NumCopies(LcHwDebugCopies)
979 ) u_prim_lc_sync (
980   .clk_i,
981   .rst_ni,
982   .lc_en_i(lc_hw_debug_en_i),
983   .lc_en_o({lc_hw_debug_en_out})
984 );
985
986 for (genvar i = 0; i < LcHwDebugCopies; i = i+1) begin :
gen_lc_dbg_copies
987   assign lc_hw_debug_on_fo[i] = lc_c-
trl_pkg::lc_tx_test_true_strict(lc_hw_debug_en_out[i]);
988 end : gen_lc_dbg_copies
989
990 //-----
991 // observe state machine
992 //-----
993
994 assign hw2reg.main_sm_state.de = 1'b1;
995 assign hw2reg.main_sm_state.d = cs_main_sm_state;
996
997 //-----
998 // report csrng request summary
999 //-----
1000 // Misc status
1001
1002 assign hw2reg.hw_exc_sts.de = cs_enable_fo[50];
1003 assign hw2reg.hw_exc_sts.d = hw_exception_sts;
1004
1005 // unused signals
1006 logic      unused_err_code_test_bit;
1007 logic      unused_enable_fo;
1008 logic      unused_reg2hw_genbits;
1009 logic      unused_int_state_val;
1010 logic      unused_reseed_interval;
1011 logic [SeedLen-1:0] unused_gen_rsp_pdata;
1012 logic      unused_state_db_inst_state;
1013
1014 assign unused_err_code_test_bit = err_code_test_bit[27] ||
(!err_code_test_bit[24:23]) ||
1015
1016 err_code_test_bit[19:2]);
1017 assign unused_enable_fo = (!cs_enable_fo[47:46]) || cs-en-

```

```

able_fo[42] || (!cs_enable_fo[18:4]);
1017 assign unused_reg2hw_genbits = (!reg2hw.genbits.q);
1018 assign unused_int_state_val = (!reg2hw.int_state_val.q);
1019 assign unused_reseed_interval = reg2hw.reseed_interval.q;
1020 assign unused_gen_rsp_pdata = ctr_drbg_rsp_data.pdata;
1021 assign unused_state_db_inst_state =
state_db_rd_data.inst_state;
1022
1023 //-----
1024 // Assertions
1025 //-----
1026 `ifdef INC_ASSERT
1027   logic state_db_zeroize;
1028   assign state_db_zeroize = state_db_wr_vld &&
(ctr_drbg_rsp_data.cmd == UNI);
1029   `ASSERT(CsrngUniZeroizeFips_A, state_db_zeroize ->
(ctr_drbg_rsp_data.fips == '0))
1030   `ASSERT(CsrngUniZeroizeKey_A, state_db_zeroize ->
(ctr_drbg_rsp_data.key == '0))
1031   `ASSERT(CsrngUniZeroizeV_A, state_db_zeroize ->
(ctr_drbg_rsp_data.v == '0))
1032   `ASSERT(CsrngUniZeroizeRc_A, state_db_zeroize ->
(ctr_drbg_rsp_data.rs_ctr == '0))
1033   `ASSERT(CsrngUniZeroizeSts_A, state_db_zeroize ->
(ctr_drbg_rsp_sts == CMD_STS_SUCCESS))
1034 `endif
1035
1036 endmodule // csrng_core

```

5. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_ctr_drbg.sv (533 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Implementation of the NIST SP800-90A CTR DRBG algorithm, no de-
rivation function.
6
7 `include "prim_assert.sv"
8
9 module csrng_ctr_drbg import csrng_pkg::*; (
10   input logic      clk_i,
11   input logic      rst_ni,
12
13   // Global enable
14   input logic      enable_i,
15
16   // Command interface request, arbitrated from command stages
17   input logic      req_vld_i,
18   output logic     req_rdy_o,
19   input csrng_core_data_t req_data_i,
20   input logic [SeedLen-1:0] req_entropy_i,
21   input logic      req_entropy_fips_i,
22   input logic      req_glast_i,
23
24   // Command interface response to state db and command stages
25   output logic     state_db_wr_o,
26   output logic     rsp_vld_o,
27   output csrng_core_data_t rsp_data_o,
28   output csrng_cmd_sts_e  rsp_sts_o,
29
30   // Generated bits output to command stages
31   output logic     bits_vld_o,
32   output logic [BlkLen-1:0] bits_data_o,
33   output logic     bits_fips_o,
34
35   // Block encrypt interface request
36   output logic     block_encrypt_req_vld_o,
37   input logic      block_encrypt_req_rdy_i,
38   output csrng_key_v_t block_encrypt_req_data_o,
39
40   // Block encrypt interface response
41   input logic      block_encrypt_rsp_vld_i,
42   output logic     block_encrypt_rsp_rdy_o,
43   input logic [BlkLen-1:0] block_encrypt_rsp_data_i,
44
45   // Error status outputs
46   output logic     ctr_err_o,
47   output logic     sm_err_o,
48 );
49
50 import csrng_reg_pkg::NumApps;
51
52 // Number of calls to the block cipher required for each
call of UPDate()
53 localparam int unsigned NumBlkPerUpd = SeedLen/BlkLen;
54 localparam int unsigned NumBlkPerUpdLg = $clog2(NumBlkPerUpd);
55
56 // The GENERate command consists of up to three sub commands
typedef enum logic [1:0] {
57   UPD_BEGIN,
58   GEN_LOOP,
59   UPD_FINAL,
60 } gen_subcmd_e;
61
62

```

```

63 //-----
64 // Sparse FSM encoding and instantiation
65 //-----
66
67 // Encoding generated with:
68 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 8 -n 6 \
69 //   -s 923483574589 --language=sv
70 //
71 // Hamming distance histogram:
72 //
73 // 0: --
74 // 1: --
75 // 2: --
76 // 3: ||||| (57.14%)
77 // 4: ||||| (42.86%)
78 // 5: --
79 // 6: --
80 //
81 // Minimum Hamming distance: 3
82 // Maximum Hamming distance: 4
83 // Minimum Hamming weight: 1
84 // Maximum Hamming weight: 5
85 //
86 localparam int StateWidth = 6;
87 typedef enum logic [StateWidth-1:0] {
88     Idle      = 6'b101110,
89     CtrInc    = 6'b100001,
90     ReqSend   = 6'b110100,
91     RspWait   = 6'b010111,
92     Hndshk    = 6'b001101,
93     HndshkGen = 6'b011000,
94     HndshkLoad = 6'b111011,
95     Error     = 6'b000010
96 } state_e;
97
98 state_e state_q, state_d;
99
100 // SEC_CM: CTR_DRBG.FSM.SPARSE
101 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, Idle)
102
103 //-----
104 // Signals
105 //-----
106
107 // Main data structure that encapsulates most of the
(meta)data required in this module
108 csrng_core_data_t core_data;
109
110 // Control and data signals for the v and concat counters
111 logic [BlkLen-1:0] v_ctr_sized;
112 logic [CtrLen-1:0] v_ctr;
113 logic v_ctr_load;
114 logic v_ctr_inc;
115 csrng_key_v_t concat_key_v;
116 logic concat_ctr_inc;
117 logic concat_ctr_done;
118
119 //-----
120 // Registers, including sequential process
121 //-----
122
123 // adata flops for each app interface, plus one valid bit
for each
124 // This rather large bank of flops is unfortunately required
due to the way that we handle
125 // GENerate commands with more than one block of generated
output bits: Instead of generating all
126 // blocks in one go, we always generate one block and then
allow the arbiter to give another app
127 // interface access to the data path. However, on the last
generated block, we require the adata
128 // that are supplied at the very beginning of the GENerate
command for the final update() step
129 // (compare NIST specification). Since all app interfaces
share a single packer for adata, its
130 // content can get overwritten after the first block of gen-
erated bits, and hence we must buffer
131 // adata for each app interface locally for the full "life-
time" of a multi-block GENerate command.
132 // TODO(#28153) Find a smaller solution (giving each app in-
terface an unpacker does not help)
133 csrng_key_v_t [NumApps-1:0] generate_adata_q, gener-
ate_adata_d;
134 logic [NumApps-1:0] generate_adata_vld_q, gener-
ate_adata_vld_d;
135
136 // Scratch register for commands that call block_encrypt
more than once and pointer into it
137 logic [SeedLen-1:0] concat_key_v_q, concat_key_v_d;
138 logic [NumBlkPerUpdLg-1:0] concat_ctr_q, concat_ctr_d;
139
140 // Keep track of current sub command for GENerates
141 gen_subcmd_e gen_subcmd_q, gen_subcmd_d;
142
143 always_ff @(posedge clk_i or negedge rst_ni) begin
144     if (!rst_ni) begin
145         generate_adata_q <= '{default: '0};
146         generate_adata_vld_q <= '0;
147         concat_ctr_q <= '0;

```

```

148     concat_key_v_q <= '0;
149     gen_subcmd_q <= UPD_BEGIN;
150     end else begin
151         generate_adata_q <= generate_adata_d;
152         generate_adata_vld_q <= generate_adata_vld_d;
153         concat_ctr_q <= concat_ctr_d;
154         concat_key_v_q <= concat_key_v_d;
155         gen_subcmd_q <= gen_subcmd_d;
156     end
157 end
158
159 //-----
160 // Prepare/mux input values depending on cmd
161 //-----
162
163 always_comb begin
164     core_data = req_data_i;
165
166     unique case (req_data_i.cmd)
167     INS: begin
168         // Reset key, v, and reseed counter; inject entropy
and FIPS status
169         core_data.key = '0;
170         core_data.v = '0;
171         core_data.rs_ctr = '0;
172         core_data.pdata = req_data_i.pdata ^ req_entropy_i;
173         core_data.fips = req_entropy_fips_i;
174     end
175     RES: begin
176         // Reset reseed counter; inject entropy and FIPS
status
177         core_data.rs_ctr = '0;
178         core_data.pdata = req_data_i.pdata ^ req_entropy_i;
179         core_data.fips = req_entropy_fips_i;
180     end
181     UPD, GEN: begin
182         /* leave everything as-is */
183     end
184     UNI: begin
185         // Set everything to zero
186         core_data.key = '0;
187         core_data.v = '0;
188         core_data.rs_ctr = '0;
189         core_data.pdata = '0;
190         core_data.fips = '1'b0;
191     end
192     default: begin
193         // Invalid command, should never happen
194         core_data.cmd = INV;
195     end
196 endcase
197 end
198
199 //-----
200 // Handling of generate_adata(_vld)
201 //-----
202
203 for (genvar i = 0; i < NumApps; i++) begin :
g_assign_gen_adata
204     logic capt_adata;
205     logic clear_adata_vld;
206     // 1) Write adata to local buffer upon a GENerate command
and _vld not being set
207     // 2) Clear _vld when the last GENerate beat appears on
the cmd_rsp port
208     assign capt_adata = req_vld_i && (core_data.cmd == GEN) &&
(core_data.inst_id == i) &&
!generate_adata_vld_q[i];
209     assign clear_adata_vld = rsp_vld_o && req_glast_i &&
(core_data.inst_id == i);
210
211     always_comb begin
212         generate_adata_vld_d[i] = generate_adata_vld_q[i];
213         generate_adata_d[i] = generate_adata_q[i];
214
215         if (!enable_i) begin
216             generate_adata_vld_d[i] = '1'b0;
217             generate_adata_d[i] = '0;
218         end else if (capt_adata) begin
219             generate_adata_vld_d[i] = '1'b1;
220             generate_adata_d[i] = core_data.pdata;
221         end else if (clear_adata_vld) begin
222             generate_adata_vld_d[i] = '1'b0;
223         end
224     end
225 end
226
227 //-----
228 // Counter logic for v
229 //-----
230
231 // SEC_CM: CTR_DRBG.CTR.REGUN
232 prim_count #(
233     .Width(CtrLen),
234     .PossibleActions(prim_count_pkg::Clr |
prim_count_pkg::Set |
prim_count_pkg::Incr)
235 ) u_prim_count_ctr_drbg (
236     .clk_i,
237     .rst_ni,

```



```

241
242 .clr_i (!enable_i),
243 .set_i (v_ctr_load),
244 .set_cnt_i(core_data.v[BlkLen-1:0]),
245
246 .incr_en_i(v_ctr_inc),
247 .decr_en_i(1'b0),
248 .step_i (CtrLen'(1)),
249 .commit_i (1'b1),
250
251 .cnt_o (v_ctr),
252 .cnt_after_commit_o(),
253 .err_o (ctr_err_o)
254 );
255
256 // Combine the MSBs of the initial v from the state db with
the current counter value as LSBs
257 assign v_ctr_sized = {core_data.v[BlkLen-1:CtrLen], v_ctr};
258
259 //-----
260 // Block encrypt request data assign
261 //-----
262
263 assign block_encrypt_req_data_o = '{
264   key: core_data.key,
265   v: v_ctr_sized
266 };
267
268 //-----
269 // Scratch register for block-wise derivation of new {key,v}
270 //-----
271
272 // Count the number of blocks that have been received back
from block_encrypt until we have enough
273 // blocks for at least SeedLen bits in total.
274 assign concat_ctr_done = (concat_ctr_q >= NumBlkPerUpd);
275
276 // Type conversion for easier usage
277 assign concat_key_v = csrng_key_v_t'(concat_key_v_q);
278
279 always_comb begin
280   concat_ctr_d = concat_ctr_q;
281   concat_key_v_d = concat_key_v_q;
282
283   if (!enable_i || concat_ctr_done) begin
284     concat_ctr_d = '0;
285     concat_key_v_d = '0;
286   end else if (concat_ctr_inc) begin
287     concat_ctr_d = concat_ctr_q + 1;
288     // Steer the v-response from block encrypt to the cor-
rect lane, MSB down.
289     // concat_ctr_q must be lower than NumBlkPerUpd, other-
wise concat_ctr_done is set and we end
290     // up in the if branch above, i.e., the index into con-
cat_key_v_d is always valid.
291     concat_key_v_d[(NumBlkPerUpd - 1 - concat_ctr_q) *
BlkLen +: BlkLen]
292     = block_encrypt_rsp_data_i;
293   end
294 end
295
296 //-----
297 // FSM to orchestrate everything
298 //-----
299
300 always_comb begin
301   state_d = state_q;
302   gen_subcmd_d = gen_subcmd_q;
303
304   // Handshake and data validity
305   req_rdy_o = 1'b0;
306   rsp_vld_o = 1'b0;
307   bits_vld_o = 1'b0;
308   state_db_wr_o = 1'b0;
309
310   block_encrypt_req_vld_o = 1'b0;
311   block_encrypt_rsp_rdy_o = 1'b0;
312
313   // Counter control
314   v_ctr_load = 1'b0;
315   v_ctr_inc = 1'b0;
316   concat_ctr_inc = 1'b0;
317
318   // Error output
319   sm_err_o = 1'b0;
320
321   unique case (state_q)
322     Idle: begin
323       if (enable_i && req_vld_i) begin
324         unique case (core_data.cmd)
325           UNI: begin
326             // No further action needed, write to state db
and flag response valid
327             req_rdy_o = 1'b1;
328             rsp_vld_o = 1'b1;
329             state_db_wr_o = 1'b1;
330           end
331           INS, RES, UPD: begin
332             // Load the counter with v from state db
333             v_ctr_load = 1'b1;
334             state_d = CtrInc;

```

```

335     end
336     GEN: begin
337       v_ctr_load = 1'b1;
338       // Decide which GENerate sub command to execute
339       // TODO(#28153) Clarify what the exact condition
for skipping the initial UPDate()
340       // call on GENerate commands is (pdata/adata be-
ing an all-zero vector or pdata/adata
341       // LENGTH being zero).
342       if (core_data.pdata != '0) begin
343         gen_subcmd_d = UPD_BEGIN;
344       end else begin
345         gen_subcmd_d = GEN_LOOP;
346       end
347       state_d = CtrInc;
348     end
349     default: begin
350       // Everything else is an error
351       state_d = Error;
352     end
353   endcase
354   end else begin
355     // There is no "clear" on the AES cipher, so just
flush out any outstanding responses
356     // Otherwise, there will be erroneous handshakes
when re-enabling the CSRNG
357     block_encrypt_rsp_rdy_o = 1'b1;
358   end
359   end
360   ReqSend: begin
361     // Once a request has been accepted, also process its
response.
362     block_encrypt_req_vld_o = 1'b1;
363     if (block_encrypt_req_rdy_i) begin
364       state_d = RspWait;
365     end
366   end
367   RspWait: begin
368     block_encrypt_rsp_rdy_o = 1'b1;
369     if (block_encrypt_rsp_vld_i) begin
370       // In the actual generate part of GENerate, the re-
sponse of the cipher is the generated
371       // random bits. In all other cases, the response
ends up in the scratch register.
372       if ((core_data.cmd == GEN) && (gen_subcmd_q ==
GEN_LOOP)) begin
373         bits_vld_o = 1'b1;
374       end else begin
375         v_ctr_inc = 1'b1;
376         concat_ctr_inc = 1'b1;
377       end
378       // GENerate requires a more involved, separate logic
on how to continue
379       if (core_data.cmd == GEN) begin
380         state_d = HndshkGen;
381       end else begin
382         state_d = Hndshk;
383       end
384     end
385   end
386   Hndshk: begin
387     if (concat_ctr_done) begin
388       // Enough data has been received to make up the new
{key,v}; command is completed
389       req_rdy_o = 1'b1;
390       rsp_vld_o = 1'b1;
391       state_db_wr_o = 1'b1;
392       state_d = Idle;
393     end else begin
394       // More data required
395       state_d = ReqSend;
396     end
397   end
398   HndshkGen: begin
399     unique case (gen_subcmd_q)
400       UPD_BEGIN: begin
401         if (concat_ctr_done) begin
402           // We have to commit the derived {key,v} to the
state db as they are the basis for the
403           // UPD_FINAL step which we potentially must ex-
ecute, also requiring the scratch reg
404           state_db_wr_o = 1'b1;
405           gen_subcmd_d = GEN_LOOP;
406           state_d = HndshkLoad;
407         end else begin
408           state_d = ReqSend;
409         end
410       end
411       GEN_LOOP: begin
412         if (req_glast_i) begin
413           // Do the initial v increment for the UPD_FINAL
step that follows
414           v_ctr_inc = 1'b1;
415           gen_subcmd_d = UPD_FINAL;
416           state_d = ReqSend;
417         end else begin
418           // This was not the last beat of the generate
loop; signal the command as completed
419           req_rdy_o = 1'b1;

```

```

420         rsp_vld_o = 1'b1;
421         state_db_wr_o = 1'b1;
422         state_d = Idle;
423     end
424 end
425 UPD_FINAL: begin
426     if (concat_ctr_done) begin
427         // Enough data has been received to make up the
new {key,v}; command is completed
428         req_rdy_o = 1'b1;
429         rsp_vld_o = 1'b1;
430         state_db_wr_o = 1'b1;
431         state_d = Idle;
432     end else begin
433         // More data required
434         state_d = ReqSend;
435     end
436 end
437 default: begin
438     // We should never get here
439     state_d = Error;
440 end
441 endcase
442 end
443 HndshkLoad: begin
444     // Transitional state to load the prim_count with the
new v
445     v_ctr_load = 1'b1;
446     state_d = CtrInc;
447 end
448 CtrInc: begin
449     // Transitional state to perform initial v-increment
after (re)loading it from state db
450     v_ctr_inc = 1'b1;
451     state_d = ReqSend;
452 end
453 Error: begin
454     sm_err_o = 1'b1;
455 end
456 default: begin
457     state_d = Error;
458     sm_err_o = 1'b1;
459 end
460 endcase
461
462 // Avoid wrapping every state body in an `if (enable_i)`
by collectively handling `enable_i`
463 // here, at least for most states.
464 if (!enable_i && (state_q == RspWait)) begin
465     // We have to handle this edge case, where there is an
outstanding response from block_encrypt
466     // while CSRNG gets disabled. Wait until the response
arrives; otherwise, it can arrive at a
467     // somewhat random time during other states and might
corrupt or deadlock the control flow
468     if (block_encrypt_rsp_vld_i) begin
469         state_d = Idle;
470     end
471 end else if (!(state_q inside {Idle, Error}) && (state_d !=
Error)) begin
472     // Be careful to not enable error-state escape by acci-
dent (checked by assertions).
473     if (!enable_i) begin
474         state_d = Idle;
475     end
476 end
477 end
478
479 //-----
480 // Response/data out assignments
481 //-----
482
483 // Make sure the command stages get an error response in
case we received an invalid command
484 assign rsp_sts_o = (core_data.cmd == INV) ? CMD_STS_INVAL-
ID_ACMD : CMD_STS_SUCCESS;
485
486 always_comb begin
487     // Default case only relevant for UNInstantiate and inval-
id commands
488     rsp_data_o = core_data;
489
490     // For all commands that update the internal state, send
new {key,v} to state db
491
492     // For GENERate commands, we need a more involved handling
of the three sub commands
493     if (core_data.cmd inside {INS, RES, UPD}) begin
494         rsp_data_o.key = concat_key_v.key ^
core_data.pdata[BlkLen -: KeyLen];
495         rsp_data_o.v = concat_key_v.v ^
core_data.pdata[BlkLen-1:0];
496     end else if (core_data.cmd == GEN) begin
497         unique case (gen_subcmd_q)
498             UPD_BEGIN: begin
499                 rsp_data_o.key = concat_key_v.key ^
core_data.pdata[BlkLen -: KeyLen];
500                 rsp_data_o.v = concat_key_v.v ^
core_data.pdata[BlkLen-1:0];
501             end
502             GEN_LOOP: begin

```

```

502         rsp_data_o.key = core_data.key; // Unchanged
503         rsp_data_o.v = v_ctr_sized; // Old value + 1
504     end
505     UPD_FINAL: begin
506         rsp_data_o.key = concat_key_v.key ^ gener-
ate_adata_q[core_data.inst_id].key;
507         rsp_data_o.v = concat_key_v.v ^ gener-
ate_adata_q[core_data.inst_id].v;
508         // Increase the reseed counter on the last beat of
every GENERate
509         rsp_data_o.rs_ctr = core_data.rs_ctr + 1;
510     end
511     default: /* We should never get here */ ;
512 endcase
513 end
514 end
515
516 // Genbits out; actual bits taken directly from block cipher
517 assign bits_data_o = block_encrypt_rsp_data_i;
518 assign bits_fips_o = core_data.fips;
519
520 //-----
521 // Assertions and unused signal tie-off
522 //-----
523
524 // Currently not supported, but most probably never intended
(would yield a very wide counter)
525 `ASSERT_INIT(CsrngCtrLenLessBlkLen, CtrLen < BlkLen)
526
527 // Make sure the FSM has a stable error state that cannot be
escaped
528 `ASSERT(CsrngCtrDrbgSmErrorStStable_A, state_q == Error |>
$stable(state_q))
529
530 // Outside of any non-error state, the FSM error output must
be asserted
531 `ASSERT(CsrngCtrDrbgSmErrorOutput_A, !(state_q inside
{Idle, CtrInc, ReqSend, RspWait, Hndshk, HndshkGen,
HndshkLoad}))
532 |> sm_err_o)
533 endmodule

```

6. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_main_sm.sv (136 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: csrng app cmd request state machine module
6 //
7 // - handles all app cmd requests from all requesting inter-
faces
8
9 `include "prim_assert.sv"
10
11 module csrng_main_sm import csrng_pkg::*; (
12     input logic clk_i,
13     input logic rst_ni,
14     input logic enable_i,
15
16     input logic acmd_avail_i,
17     output logic acmd_accept_o,
18     input acmd_e acmd_i,
19     input logic acmd_eop_i,
20
21     input logic flag0_i,
22
23     output logic cmd_entropy_req_o,
24     input logic cmd_entropy_avail_i,
25
26     output logic cmd_vld_o,
27     input logic cmd_rdy_i,
28     output logic clr_adata_packer_o,
29     input logic cmd_complete_i,
30
31     input logic local_escalate_i,
32
33     output logic [MainSmStateWidth-1:0] main_sm_state_o,
34     output logic main_sm_err_o
35 );
36
37 // SEC_CM: MAIN_SM.FSM.SPARSE
38 main_sm_state_e state_d, state_q;
39 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
main_sm_state_e, MainSmIdle)
40
41 assign main_sm_state_o = {state_q};
42
43 always_comb begin
44     state_d = state_q;
45     acmd_accept_o = 1'b0;
46     cmd_entropy_req_o = 1'b0;
47     cmd_vld_o = 1'b0;
48     clr_adata_packer_o = 1'b0;
49     main_sm_err_o = 1'b0;

```



```

50
51   if (state_q == MainSmError) begin
52     // In case we are in the Error state we must ignore the
local escalate and enable signals.
53     main_sm_err_o = 1'b1;
54   end else if (local_escalate_i) begin
55     // In case local escalate is high we must transition to
the error state.
56     state_d = MainSmError;
57   end else if (!enable_i && state_q inside {MainSmIdle,
MainSmParseCmd, MainSmEntropyReq,
58                                     MainSmCmdPrep,
MainSmCmdVld,
59                                     MainSmClrAData,
MainSmCmdCompWait}) begin
60     // In case the module is disabled and we are in a legal
state we must go into idle state.
61     state_d = MainSmIdle;
62   end else begin
63     // Otherwise do the state machine as normal.
64     unique case (state_q)
65       MainSmIdle: begin
66         // Signal the arbiter to grant this request.
67         if (acmd_avail_i) begin
68           acmd_accept_o = 1'b1;
69           state_d = MainSmParseCmd;
70         end
71       end
72       MainSmParseCmd: begin
73         if (acmd_eop_i) begin
74           unique case (acmd_i)
75             INS, RES: state_d = MainSmEntropyReq; //
Command may require entropy
76             GEN, UPD, UNI: state_d = MainSmCmdPrep; //
Command does not require entropy
77             default: state_d = MainSmIdle; //
Command was not supported
78           endcase
79         end
80       end
81       MainSmEntropyReq: begin
82         if (flag0_i) begin
83           // With flag0 set, no entropy is required.
84           state_d = MainSmCmdVld;
85         end else begin
86           // Delay one clock to fix timing issue.
87           cmd_entropy_req_o = 1'b1;
88           if (cmd_entropy_avail_i) begin
89             state_d = MainSmCmdVld;
90           end
91         end
92       end
93       MainSmCmdPrep: begin
94         // Assumes all adata is present now.
95         state_d = MainSmCmdVld;
96       end
97       MainSmCmdVld: begin
98         cmd_vld_o = 1'b1;
99         if (cmd_rdy_i) begin
100           if (cmd_complete_i) begin
101             clr_adata_packer_o = 1'b1;
102             state_d = MainSmIdle;
103           end else begin
104             state_d = MainSmClrAData;
105           end
106         end
107       end
108       MainSmClrAData: begin
109         clr_adata_packer_o = 1'b1;
110         if (cmd_complete_i) begin
111           state_d = MainSmIdle;
112         end else begin
113           state_d = MainSmCmdCompWait;
114         end
115       end
116       MainSmCmdCompWait: begin
117         if (cmd_complete_i) begin
118           state_d = MainSmIdle;
119         end
120       end
121       // Error: The error state is now covered by the if
statement above.
122       default: begin
123         state_d = MainSmError;
124         main_sm_err_o = 1'b1;
125       end
126     endcase
127   end
128 end
129
130 // Make sure that the state machine has a stable error
state. This means that after the error
131 // state is entered it will not exit it unless a reset sig-
nal is received.
132 `ASSERT(CsrngMainErrorStStable_A, state_q == MainSmError ==>
$stable(state_q))
133 // If in error state, the error output must be high.
134 `ASSERT(CsrngMainErrorOutput_A, state_q == MainSmError ==>
main_sm_err_o)

```

```

135
136 endmodule

```

7. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_pkg.sv (161 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package csrng_pkg;
7
8   parameter int unsigned NumAppsLg = $clog-
g2(csrng_reg_pkg::NumApps);
9
10  // Main design parameters
11  // We use the suffix 'Len' (as opposed to 'Width') to stick
to the symbols used in the
12  // NIST SP 800-90A algorithm specification.
13
14  // Size of the generated random number blocks in bits and
the internal state variable 'v'
15  parameter int unsigned BlkLen = 128;
16  // Cipher key size in bits
17  parameter int unsigned KeyLen = 256;
18
19  // Seed size in bits; must be equal to the sum of BlkLen and
KeyLen
20  parameter int unsigned SeedLen = BlkLen + KeyLen;
21  // Width of the counter field used as input to the cipher
22  // Caution: Not to be confused with the reseed counter
width! These two are independent
23  parameter int unsigned CtrLen = 32;
24  parameter int unsigned RsCtrWidth = 32;
25
26  // Width of the counter in the command stages to count the
amount of generated random bits
27  parameter int unsigned GenBitsCtrWidth = 12;
28
29  // Commonly used internal signal widths
30  parameter int unsigned CmdWidth = 3;
31  parameter int unsigned InstIdWidth = 4;
32
33  // Width of the application command bus (matches the 32b
TLUL bus width)
34  parameter int unsigned CmdBusWidth = 32;
35
36  // Maximum number of 32b words additionally supplied on the
application interfaces
37  parameter int unsigned CmdMaxClen = 12;
38  parameter int unsigned CmdFifoDepth = 2;
39  parameter int unsigned CmdFifoDepthLg = $clog-
g2(CmdFifoDepth);
40
41  parameter int unsigned GENBITS_BUS_WIDTH = BlkLen;
42  parameter int unsigned FIPS_GENBITS_BUS_WIDTH = en-
tropy_src_pkg::FIPS_BUS_WIDTH +
43  GENBITS_BUS_WIDTH;
44
45  //-----
46  // Application Interface
47  //-----
48
49  typedef struct packed {
50    logic [CmdBusWidth-1:0] csrng_req_valid;
51    logic [CmdBusWidth-1:0] csrng_req_bus;
52    logic genbits_ready;
53  } csrng_req_t;
54
55  parameter int unsigned CmdStatusWidth = 3;
56
57  typedef enum logic [CmdStatusWidth-1:0] {
58    CMD_STS_SUCCESS = 'h0,
59    CMD_STS_INVALID_ACMD = 'h1,
60    CMD_STS_INVALID_GEN_CMD = 'h2,
61    CMD_STS_INVALID_CMD_SEQ = 'h3,
62    CMD_STS_RESEED_CNT_EXCEEDED = 'h4,
63    CMD_STS_UNDRIVEN = 'z
64  } csrng_cmd_sts_e;
65
66  typedef struct packed {
67    logic csrng_req_ready;
68    logic csrng_rsp_ack;
69    logic csrng_cmd_sts_e;
70    logic genbits_valid;
71    logic genbits_fips;
72    logic [GENBITS_BUS_WIDTH-1:0] genbits_bus;
73  } csrng_rsp_t;
74
75  parameter csrng_req_t CSRNG_REQ_DEFAULT = '{default: '0};
76  parameter csrng_rsp_t CSRNG_RSP_DEFAULT = '0;
77
78  typedef enum logic [CmdWidth-1:0] {
79    INV = 3'h0,
80    INS = 3'h1,
81    RES = 3'h2,
82    GEN = 3'h3,
83    UPD = 3'h4,

```

```

83     UNI = 3'h5
84 } acmd_e;
85
86 typedef struct packed {
87     logic [7:0] resv;
88     logic [11:0] glen;
89     logic [3:0] flag0;
90     logic [3:0] clen;
91     logic      gap; // acmd is defined as 4 bits wide but
only 3 are used
92     acmd_e      acmd;
93 } csrng_cmd_t;
94
95 //-----
96 // Struct data types for the core data path
97 //-----
98
99 typedef struct packed {
100     logic      fips;
101     logic [RsCtrWidth-1:0] rs_ctr;
102     logic [SeedLen-1:0] pdata;
103     logic [NumAppsLg-1:0] inst_id;
104     acmd_e      cmd;
105     logic [KeyLen-1:0] key;
106     logic [BlkLen-1:0] v;
107 } csrng_core_data_t;
108
109 typedef struct packed {
110     logic [KeyLen-1:0] key;
111     logic [BlkLen-1:0] v;
112 } csrng_key_v_t;
113
114 // Do not reorder these fields - software expects them in
this order when doing the raw
115 // state readout through the register interface.
116 typedef struct packed {
117     logic      fips;
118     logic      inst_state;
119     logic [KeyLen-1:0] key;
120     logic [BlkLen-1:0] v;
121     logic [RsCtrWidth-1:0] rs_ctr;
122 } csrng_state_t;
123
124 parameter int unsigned StateWidth = $bits(csrng_state_t);
125
126 parameter int unsigned MainSmStateWidth = 6;
127
128 // Encoding generated with:
129 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 8 -n 6 \
130 // -s 137328258 --language=sv
131 //
132 // Hamming distance histogram:
133 //
134 // 0: --
135 // 1: --
136 // 2: --
137 // 3: ||||| (57.14%)
138 // 4: ||||| (42.86%)
139 // 5: --
140 // 6: --
141 //
142 // Minimum Hamming distance: 3
143 // Maximum Hamming distance: 4
144 // Minimum Hamming weight: 1
145 // Maximum Hamming weight: 5
146 //
147 typedef enum logic [MainSmStateWidth-1:0] {
148     MainSmIdle = 6'b110111, // idle
149     MainSmParseCmd = 6'b011101, // parse the cmd
150     MainSmEntropyReq = 6'b001110, // request entropy if ne-
cessary
151     MainSmCmdPrep = 6'b000011, // delay cycle for command
request (?)
152     MainSmCmdVld = 6'b010000, // command request to core
data path
153     MainSmClrAData = 6'b111010, // clear out the additional
data packer fifo
154     MainSmCmdCompWait = 6'b100100, // wait for command to com-
plete
155     MainSmError = 6'b101001 // error state, results in
fatal alert
156 } main_sm_state_e;
157
158 parameter int CsKeymgrDivWidth = 384;
159 typedef logic [CsKeymgrDivWidth-1:0] cs_keymgr_div_t;
160
161 endpackage : csrng_pkg

```

8. lowRISC_opentitan/hw/ip/csrng/rtl/csrng_reg_pkg.sv (421 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data

```

```

structure
6
7 package csrng_reg_pkg;
8
9 // Param list
10 parameter int NumApps = 3;
11 parameter int NumAlerts = 2;
12
13 // Address widths within the block
14 parameter int BlockAw = 7;
15
16 // Number of registers for every interface
17 parameter int NumRegs = 24;
18
19 // Alert indices
20 typedef enum int {
21     AlertRecovAlertIdx = 0,
22     AlertFatalAlertIdx = 1
23 } csrng_alert_idx_t;
24
25 //-----
26 // Typedefs for registers //
27 //-----
28
29 typedef struct packed {
30     struct packed {
31         logic      q;
32     } cs_fatal_err;
33     struct packed {
34         logic      q;
35     } cs_hw_inst_exc;
36     struct packed {
37         logic      q;
38     } cs_entropy_req;
39     struct packed {
40         logic      q;
41     } cs_cmd_req_done;
42 } csrng_reg2hw_intr_state_reg_t;
43
44 typedef struct packed {
45     struct packed {
46         logic      q;
47     } cs_fatal_err;
48     struct packed {
49         logic      q;
50     } cs_hw_inst_exc;
51     struct packed {
52         logic      q;
53     } cs_entropy_req;
54     struct packed {
55         logic      q;
56     } cs_cmd_req_done;
57 } csrng_reg2hw_intr_enable_reg_t;
58
59 typedef struct packed {
60     struct packed {
61         logic      q;
62         logic      qe;
63     } cs_fatal_err;
64     struct packed {
65         logic      q;
66         logic      qe;
67     } cs_hw_inst_exc;
68     struct packed {
69         logic      q;
70         logic      qe;
71     } cs_entropy_req;
72     struct packed {
73         logic      q;
74         logic      qe;
75     } cs_cmd_req_done;
76 } csrng_reg2hw_intr_test_reg_t;
77
78 typedef struct packed {
79     struct packed {
80         logic      q;
81         logic      qe;
82     } fatal_alert;
83     struct packed {
84         logic      q;
85         logic      qe;
86     } recov_alert;
87 } csrng_reg2hw_alert_test_reg_t;
88
89 typedef struct packed {
90     struct packed {
91         logic [3:0] q;
92     } fips_force_enable;
93     struct packed {
94         logic [3:0] q;
95     } read_int_state;
96     struct packed {
97         logic [3:0] q;
98     } sw_app_enable;
99     struct packed {
100         logic [3:0] q;
101     } enable;
102 } csrng_reg2hw_ctrl_reg_t;
103
104 typedef struct packed {
105     logic [31:0] q;

```

```

106     logic         qe;
107 } csrng_reg2hw_cmd_req_reg_t;
108
109 typedef struct packed {
110     logic [31:0] q;
111     logic         qe;
112 } csrng_reg2hw_reseed_interval_reg_t;
113
114 typedef struct packed {
115     logic [31:0] q;
116     logic         re;
117 } csrng_reg2hw_genbits_reg_t;
118
119 typedef struct packed {
120     logic [2:0] q;
121 } csrng_reg2hw_int_state_read_enable_reg_t;
122
123 typedef struct packed {
124     logic [3:0] q;
125     logic         qe;
126 } csrng_reg2hw_int_state_num_reg_t;
127
128 typedef struct packed {
129     logic [31:0] q;
130     logic         re;
131 } csrng_reg2hw_int_state_val_reg_t;
132
133 typedef struct packed {
134     logic [2:0] q;
135 } csrng_reg2hw_fips_force_reg_t;
136
137 typedef struct packed {
138     logic [4:0] q;
139     logic         qe;
140 } csrng_reg2hw_err_code_test_reg_t;
141
142 typedef struct packed {
143     struct packed {
144         logic d;
145         logic de;
146     } cs_fatal_err;
147     struct packed {
148         logic d;
149         logic de;
150     } cs_hw_inst_exc;
151     struct packed {
152         logic d;
153         logic de;
154     } cs_entropy_req;
155     struct packed {
156         logic d;
157         logic de;
158     } cs_cmd_req_done;
159 } csrng_hw2reg_intr_state_reg_t;
160
161 typedef struct packed {
162     logic [31:0] d;
163 } csrng_hw2reg_reseed_counter_mreg_t;
164
165 typedef struct packed {
166     struct packed {
167         logic [2:0] d;
168         logic         de;
169     } cmd_sts;
170     struct packed {
171         logic d;
172         logic de;
173     } cmd_ack;
174     struct packed {
175         logic d;
176         logic de;
177     } cmd_rdy;
178 } csrng_hw2reg_sw_cmd_sts_reg_t;
179
180 typedef struct packed {
181     struct packed {
182         logic d;
183     } genbits_fips;
184     struct packed {
185         logic d;
186     } genbits_vld;
187 } csrng_hw2reg_genbits_vld_reg_t;
188
189 typedef struct packed {
190     logic [31:0] d;
191 } csrng_hw2reg_genbits_reg_t;
192
193 typedef struct packed {
194     logic [31:0] d;
195 } csrng_hw2reg_int_state_val_reg_t;
196
197 typedef struct packed {
198     logic [15:0] d;
199     logic         de;
200 } csrng_hw2reg_hw_exc_sts_reg_t;
201
202 typedef struct packed {
203     struct packed {
204         logic d;
205         logic de;
206     } cmd_stage_reseed_cnt_alert;

```

```

207     struct packed {
208         logic d;
209         logic de;
210     } cmd_stage_invalid_cmd_seq_alert;
211     struct packed {
212         logic d;
213         logic de;
214     } cmd_stage_invalid_acmd_alert;
215     struct packed {
216         logic d;
217         logic de;
218     } cs_bus_cmp_alert;
219     struct packed {
220         logic d;
221         logic de;
222     } acmd_flag0_field_alert;
223     struct packed {
224         logic d;
225         logic de;
226     } fips_force_enable_field_alert;
227     struct packed {
228         logic d;
229         logic de;
230     } read_int_state_field_alert;
231     struct packed {
232         logic d;
233         logic de;
234     } sw_app_enable_field_alert;
235     struct packed {
236         logic d;
237         logic de;
238     } enable_field_alert;
239 } csrng_hw2reg_recov_alert_sts_reg_t;
240
241 typedef struct packed {
242     struct packed {
243         logic d;
244         logic de;
245     } fifo_state_err;
246     struct packed {
247         logic d;
248         logic de;
249     } fifo_read_err;
250     struct packed {
251         logic d;
252         logic de;
253     } fifo_write_err;
254     struct packed {
255         logic d;
256         logic de;
257     } ctr_err;
258     struct packed {
259         logic d;
260         logic de;
261     } aes_cipher_sm_err;
262     struct packed {
263         logic d;
264         logic de;
265     } ctr_drbg_sm_err;
266     struct packed {
267         logic d;
268         logic de;
269     } main_sm_err;
270     struct packed {
271         logic d;
272         logic de;
273     } cmd_stage_sm_err;
274     struct packed {
275         logic d;
276         logic de;
277     } sfifo_genbits_err;
278     struct packed {
279         logic d;
280         logic de;
281     } sfifo_cmd_err;
282 } csrng_hw2reg_err_code_reg_t;
283
284 typedef struct packed {
285     logic [5:0] d;
286     logic         de;
287 } csrng_hw2reg_main_sm_state_reg_t;
288
289 // Register -> HW type
290 typedef struct packed {
291     csrng_reg2hw_intr_state_reg_t intr_state; // [184:181]
292     csrng_reg2hw_intr_enable_reg_t intr_enable; // [180:177]
293     csrng_reg2hw_intr_test_reg_t intr_test; // [176:169]
294     csrng_reg2hw_alert_test_reg_t alert_test; // [168:165]
295     csrng_reg2hw_ctrl_reg_t ctrl; // [164:149]
296     csrng_reg2hw_cmd_req_reg_t cmd_req; // [148:116]
297     csrng_reg2hw_reseed_interval_reg_t reseed_interval; //
[115:83]
298     csrng_reg2hw_genbits_reg_t genbits; // [82:50]
299     csrng_reg2hw_int_state_read_enable_reg_t
int_state_read_enable; // [49:47]
300     csrng_reg2hw_int_state_num_reg_t int_state_num; // [46:42]
301     csrng_reg2hw_int_state_val_reg_t int_state_val; // [41:9]
302     csrng_reg2hw_fips_force_reg_t fips_force; // [8:6]
303     csrng_reg2hw_err_code_test_reg_t err_code_test; // [5:0]
304 } csrng_reg2hw_t;
305

```

```

306 // HW -> register type
307 typedef struct packed {
308     csrng_hw2reg_intr_state_reg_t intr_state; // [239:232]
309     csrng_hw2reg_reseed_counter_mreg_t [2:0] re-
seed_counter; // [231:136]
310     csrng_hw2reg_sw_cmd_sts_reg_t sw_cmd_sts; // [135:128]
311     csrng_hw2reg_genbits_vld_reg_t genbits_vld; // [127:126]
312     csrng_hw2reg_genbits_reg_t genbits; // [125:94]
313     csrng_hw2reg_int_state_val_reg_t int_state_val; // [93:62]
314     csrng_hw2reg_hw_exc_sts_reg_t hw_exc_sts; // [61:45]
315     csrng_hw2reg_recov_alert_sts_reg_t recov_alert_sts; //
[44:27]
316     csrng_hw2reg_err_code_reg_t err_code; // [26:7]
317     csrng_hw2reg_main_sm_state_reg_t main_sm_state; // [6:0]
318 } csrng_hw2reg_t;
319
320 // Register offsets
321 parameter logic [BlockAw-1:0] CSRNG_INTR_STATE_OFFSET = 7'h
0;
322 parameter logic [BlockAw-1:0] CSRNG_INTR_ENABLE_OFFSET = 7'h
4;
323 parameter logic [BlockAw-1:0] CSRNG_INTR_TEST_OFFSET = 7'h
8;
324 parameter logic [BlockAw-1:0] CSRNG_ALERT_TEST_OFFSET = 7'h
c;
325 parameter logic [BlockAw-1:0] CSRNG_REGWEN_OFFSET = 7'h 10;
326 parameter logic [BlockAw-1:0] CSRNG_CTRL_OFFSET = 7'h 14;
327 parameter logic [BlockAw-1:0] CSRNG_CMD_REQ_OFFSET = 7'h 18;
328 parameter logic [BlockAw-1:0] CSRNG_RESEED_INTERVAL_OFFSET =
7'h 1c;
329 parameter logic [BlockAw-1:0] CSRNG_RESEED_COUNTER_0_OFFSET
= 7'h 20;
330 parameter logic [BlockAw-1:0] CSRNG_RESEED_COUNTER_1_OFFSET
= 7'h 24;
331 parameter logic [BlockAw-1:0] CSRNG_RESEED_COUNTER_2_OFFSET
= 7'h 28;
332 parameter logic [BlockAw-1:0] CSRNG_SW_CMD_STS_OFFSET = 7'h
2c;
333 parameter logic [BlockAw-1:0] CSRNG_GENBITS_VLD_OFFSET = 7'h
30;
334 parameter logic [BlockAw-1:0] CSRNG_GENBITS_OFFSET = 7'h 34;
335 parameter logic [BlockAw-1:0]
CSRNG_INT_STATE_READ_ENABLE_OFFSET = 7'h 38;
336 parameter logic [BlockAw-1:0]
CSRNG_INT_STATE_READ_ENABLE_REGWEN_OFFSET = 7'h 3c;
337 parameter logic [BlockAw-1:0] CSRNG_INT_STATE_NUM_OFFSET =
7'h 40;
338 parameter logic [BlockAw-1:0] CSRNG_INT_STATE_VAL_OFFSET =
7'h 44;
339 parameter logic [BlockAw-1:0] CSRNG_FIPS_FORCE_OFFSET = 7'h
48;
340 parameter logic [BlockAw-1:0] CSRNG_HW_EXC_STS_OFFSET = 7'h
4c;
341 parameter logic [BlockAw-1:0] CSRNG_RECOV_ALERT_STS_OFFSET =
7'h 50;
342 parameter logic [BlockAw-1:0] CSRNG_ERR_CODE_OFFSET = 7'h
54;
343 parameter logic [BlockAw-1:0] CSRNG_ERR_CODE_TEST_OFFSET =
7'h 58;
344 parameter logic [BlockAw-1:0] CSRNG_MAIN_SM_STATE_OFFSET =
7'h 5c;
345
346 // Reset values for hwext registers and their fields
347 parameter logic [3:0] CSRNG_INTR_TEST_RESVAL = 4'h 0;
348 parameter logic [0:0] CSRNG_INTR_TEST_CS_CMD_REQ_DONE_RESVAL =
1'h 0;
349 parameter logic [0:0] CSRNG_INTR_TEST_CS_ENTROPY_REQ_RESVAL
= 1'h 0;
350 parameter logic [0:0] CSRNG_INTR_TEST_CS_HW_INST_EXC_RESVAL
= 1'h 0;
351 parameter logic [0:0] CSRNG_INTR_TEST_CS_FATAL_ERR_RESVAL =
1'h 0;
352 parameter logic [1:0] CSRNG_ALERT_TEST_RESVAL = 2'h 0;

```

```

353 parameter logic [0:0] CSRNG_ALERT_TEST_RECOV_ALERT_RESVAL =
1'h 0;
354 parameter logic [0:0] CSRNG_ALERT_TEST_FATAL_ALERT_RESVAL =
1'h 0;
355 parameter logic [31:0] CSRNG_RESEED_COUNTER_0_RESVAL = 32'h
0;
356 parameter logic [31:0]
CSRNG_RESEED_COUNTER_0_RESEED_COUNTER_0_RESVAL = 32'h 0;
357 parameter logic [31:0] CSRNG_RESEED_COUNTER_1_RESVAL = 32'h
0;
358 parameter logic [31:0]
CSRNG_RESEED_COUNTER_1_RESEED_COUNTER_1_RESVAL = 32'h 0;
359 parameter logic [31:0] CSRNG_RESEED_COUNTER_2_RESVAL = 32'h
0;
360 parameter logic [31:0]
CSRNG_RESEED_COUNTER_2_RESEED_COUNTER_2_RESVAL = 32'h 0;
361 parameter logic [1:0] CSRNG_GENBITS_VLD_RESVAL = 2'h 0;
362 parameter logic [31:0] CSRNG_GENBITS_RESVAL = 32'h 0;
363 parameter logic [31:0] CSRNG_INT_STATE_VAL_RESVAL = 32'h 0;
364
365 // Register index
366 typedef enum int {
367     CSRNG_INTR_STATE,
368     CSRNG_INTR_ENABLE,
369     CSRNG_INTR_TEST,
370     CSRNG_ALERT_TEST,
371     CSRNG_REGWEN,
372     CSRNG_CTRL,
373     CSRNG_CMD_REQ,
374     CSRNG_RESEED_INTERVAL,
375     CSRNG_RESEED_COUNTER_0,
376     CSRNG_RESEED_COUNTER_1,
377     CSRNG_RESEED_COUNTER_2,
378     CSRNG_SW_CMD_STS,
379     CSRNG_GENBITS_VLD,
380     CSRNG_GENBITS,
381     CSRNG_INT_STATE_READ_ENABLE,
382     CSRNG_INT_STATE_READ_ENABLE_REGWEN,
383     CSRNG_INT_STATE_NUM,
384     CSRNG_INT_STATE_VAL,
385     CSRNG_FIPS_FORCE,
386     CSRNG_HW_EXC_STS,
387     CSRNG_RECOV_ALERT_STS,
388     CSRNG_ERR_CODE,
389     CSRNG_ERR_CODE_TEST,
390     CSRNG_MAIN_SM_STATE
391 } csrng_id_e;
392
393 // Register width information to check illegal writes
394 parameter logic [3:0] CSRNG_PERMIT [24] = '{
395     4'b 0001, // index[ 0] CSRNG_INTR_STATE
396     4'b 0001, // index[ 1] CSRNG_INTR_ENABLE
397     4'b 0001, // index[ 2] CSRNG_INTR_TEST
398     4'b 0001, // index[ 3] CSRNG_ALERT_TEST
399     4'b 0001, // index[ 4] CSRNG_REGWEN
400     4'b 0011, // index[ 5] CSRNG_CTRL
401     4'b 1111, // index[ 6] CSRNG_CMD_REQ
402     4'b 1111, // index[ 7] CSRNG_RESEED_INTERVAL
403     4'b 1111, // index[ 8] CSRNG_RESEED_COUNTER_0
404     4'b 1111, // index[ 9] CSRNG_RESEED_COUNTER_1
405     4'b 1111, // index[10] CSRNG_RESEED_COUNTER_2
406     4'b 0001, // index[11] CSRNG_SW_CMD_STS
407     4'b 0001, // index[12] CSRNG_GENBITS_VLD
408     4'b 1111, // index[13] CSRNG_GENBITS
409     4'b 0001, // index[14] CSRNG_INT_STATE_READ_ENABLE
410     4'b 0001, // index[15] CSRNG_INT_STATE_READ_ENABLE_REGWEN
411     4'b 0001, // index[16] CSRNG_INT_STATE_NUM
412     4'b 1111, // index[17] CSRNG_INT_STATE_VAL
413     4'b 0001, // index[18] CSRNG_FIPS_FORCE
414     4'b 0011, // index[19] CSRNG_HW_EXC_STS
415     4'b 0011, // index[20] CSRNG_RECOV_ALERT_STS
416     4'b 1111, // index[21] CSRNG_ERR_CODE
417     4'b 0001, // index[22] CSRNG_ERR_CODE_TEST
418     4'b 0001 // index[23] CSRNG_MAIN_SM_STATE
419 };
420
421 endpackage

```

OpenTitan 엔트로피원 (SP800-90B)

What. 물리 잡음원에서 엔트로피를 뽑고 건강검사(health test)를 돌리는 블록.

Why it exists. CSRNG 에 먹일 ‘진짜’ 무작위가 필요하다. 그런데 물리 잡음원은 고장날 수 있고(고착·편향), 고장난 채로 난수를 공급하면 키가 예측 가능해진다. 그래서 표준이 반복 카운트 검사와 적응 비율 검사를 상시 돌리도록 규정한다. 이 블록의 절반이 그 검사다.

학부 복습 · EECS concepts. 엔트로피의 정보이론적 정의(min-entropy) · 통계 검정 · 링 오실레이터 지터의 물리(열잡음 → 위상 잡음 → 비트 무작위성).

석사 수준 · Practice note. 엔트로피원은 시뮬레이션으로 검증할 수 없는 대표 블록 이다. RTL 검증은 ‘검사 로직이 맞게 도는가’까지이고, 실제 엔트로피 품질은 실리콘에서 측정한다. 모델링 팀이 이 경계를 문서에 명시해야 한다.

13 files • 4,920 lines

API — entropy_src.sv

Table 265. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — module / interface / package (1개)

이름	종류
entropy_src	module

Table 266. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — parameter (8개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RngBusWidth	int	4
RngBusBitSelWidth	int	2
HealthTestWindowWidth	int	18
EsFifoDepth	int	3
DistrFifoDepth	int	3
Stub	bit	1'b0

Table 267. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — localparam (5개)

상수	형/폭	값
BucketHtDataWidth	int	bucket_ht_data_width(RngBusWidth)
NumBuckets	int	2**BucketHtDataWidth
NumBucketHtInst	int unsigned	num_bucket_ht_inst(RngBusWidth)
StubLfsrWidth	int	64
Copies	int	CSRNG_BUS_WIDTH / StubLfsrWidth

Table 268. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
otp_en_entropy_src_fw_read_i	input	mubi8_t
otp_en_entropy_src_fw_over_i	input	mubi8_t
rng_fips_o	output	logic
entropy_src_hw_if_i	input	entropy_src_hw_if_req_t
entropy_src_hw_if_o	output	entropy_src_hw_if_rsp_t
entropy_src_rng_enable_o	output	logic
entropy_src_rng_valid_i	input	logic
entropy_src_rng_bits_i	input	logic [RngBusWidth-1:0]
entropy_src_xht_valid_o	output	logic
entropy_src_xht_bits_o	output	logic [RngBusWidth-1:0]
entropy_src_xht_bit_sel_o	output	logic [RngBusBitSelWidth-1:0]
entropy_src_xht_health_test_window_o	output	logic [HealthTestWindowWidth-1:0]
entropy_src_xht_meta_o	output	entropy_src_xht_meta_req_t
entropy_src_xht_meta_i	input	entropy_src_xht_meta_rsp_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
intr_es_entropy_valid_o	output	logic
intr_es_health_test_failed_o	output	logic
intr_es_observe_fifo_ready_o	output	logic
intr_es_fatal_err_o	output	logic

API — entropy_src_ack_sm.sv

Table 269. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm.sv — module / interface / package (1개)

이름	종류
entropy_src_ack_sm	module

Table 270. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic

포트	방향	형/폭
req_i	input	logic
ack_o	output	logic
fifo_not_empty_i	input	logic
local_escalate_i	input	logic
fifo_pop_o	output	logic
ack_sm_err_o	output	logic

API — entropy_src_ack_sm_pkg.sv

Table 271. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm_pkg.sv — module / interface / package (1개)

이름	종류
entropy_src_ack_sm_pkg	package

Table 272. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm_pkg.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	6

API — entropy_src_adaptp_ht.sv

Table 273. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv — module / interface / package (1개)

이름	종류
entropy_src_adaptp_ht	module

Table 274. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv — parameter (3개)

파라미터	형/폭	기본값
RegWidth	int	16
RngBusWidth	int	4
RngBusBitSelWidth	int	2

Table 275. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
entropy_bit_i	input	logic [RngBusWidth-1:0]
entropy_bit_vld_i	input	logic
rng_bit_en_i	input	logic
rng_bit_sel_i	input	logic [RngBusBitSelWidth-1:0]
clear_i	input	logic

포트	방향	형/폭
active_i	input	logic
thresh_hi_i	input	logic [RegWidth-1:0]
thresh_lo_i	input	logic [RegWidth-1:0]
window_wrap_pulse_i	input	logic
threshold_scope_i	input	logic
test_cnt_hi_o	output	logic [RegWidth-1:0]
test_cnt_lo_o	output	logic [RegWidth-1:0]
test_fail_hi_pulse_o	output	logic
test_fail_lo_pulse_o	output	logic
count_err_o	output	logic

API — entropy_src_bucket_ht.sv

Table 276. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — module / interface / package (1개)

이름	종류
entropy_src_bucket_ht	module

Table 277. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — parameter (2개)

파라미터	형/폭	기본값
RegWidth	int	16
RngBusWidth	int	4

Table 278. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — localparam (1개)

상수	형/폭	값
NUM_BINS	int	2**RngBusWidth

Table 279. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
entropy_bit_i	input	logic [RngBusWidth-1:0]
entropy_bit_vld_i	input	logic
clear_i	input	logic
active_i	input	logic
thresh_i	input	logic [RegWidth-1:0]
window_wrap_pulse_i	input	logic
test_cnt_o	output	logic [RegWidth-1:0]

포트	방향	형/폭
test_fail_pulse_o	output	logic
count_err_o	output	logic

API — entropy_src_cntr_reg.sv

Table 280. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cntr_reg.sv — module / interface / package (1개)

이름	종류
entropy_src_cntr_reg	module

Table 281. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cntr_reg.sv — parameter (1개)

파라미터	형/폭	기본값
RegWidth	int	16

Table 282. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cntr_reg.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clear_i	input	logic
event_i	input	logic
step_i	input	logic [RegWidth-1:0]
value_o	output	logic [RegWidth-1:0]
err_o	output	logic

API — entropy_src_core.sv

Table 283. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — module / interface / package (1개)

이름	종류
entropy_src_core	module

Table 284. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — parameter (7개)

파라미터	형/폭	기본값
RngBusWidth	int	4
RngBusBitSelWidth	int	2
HealthTestWindowWidth	int	18
EsFifoDepth	int	3
DistrFifoDepth	int	2
BucketHtDataWidth	int	4
NumBucketHtInst	int	prim_util_pkg::ceil_div(RngBusWidth

Table 285. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — localparam (21개)

상수	형/폭	값
EsFifoDepthW	int	prim_util_pkg::vbits(EsFifoDepth + 1)
PostHTWidth	int	32
PostHTFifoDepthW	int	prim_util_pkg::vbits(PostHTWidth / RngBusWidth + 1)
HalfRegWidth	int	16
FullRegWidth	int	32
EighthRegWidth	int	4
SeedLen	int	384
DistrFifoWidth	int	32
DistrFifoDepthW	int	prim_util_pkg::vbits(DistrFifoDepth + 1)
ObserveFifoWidth	int	32
PreCondWidth	int	64
PreCondFifoDepthW	int	prim_util_pkg::vbits(PreCondWidth / PostHTWidth + 1)
PipelineDepthW	int	prim_util_pkg::vbits(1 + DistrFifoDepth + PreCondWidth / Pos...
Clog2ObserveFifoDepth	int	\$clog2(ObserveFifoDepth)
EsEnableCopies	int	20
EsEnPulseCopies	int	1
NumSwReadsSeed	int unsigned	SeedLen / FullRegWidth
LastSwRead	int unsigned	NumSwReadsSeed - 1
SwReadIdxWidth	int unsigned	prim_util_pkg::vbits(NumSwReadsSeed)
Sha3EnMasking	bit	0
Sha3Share	int	(Sha3EnMasking) ? 2 : 1

Table 286. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
reg2hw	input	entropy_src_reg_pkg::entropy_src_reg2hw_t
hw2reg	output	entropy_src_reg_pkg::entropy_src_hw2reg_t
otp_en_entropy_src_fw_read_i	input	prim_mubi_pkg::mubi8_t
otp_en_entropy_src_fw_over_i	input	prim_mubi_pkg::mubi8_t
rng_fips_o	output	logic
entropy_src_hw_if_i	input	entropy_src_hw_if_req_t
entropy_src_hw_if_o	output	entropy_src_hw_if_rsp_t
entropy_src_rng_enable_o	output	logic

포트	방향	형/폭
entropy_src_rng_valid_i	input	logic
entropy_src_rng_bits_i	input	logic [RngBusWidth-1:0]
entropy_src_xht_valid_o	output	logic
entropy_src_xht_bits_o	output	logic [RngBusWidth-1:0]
entropy_src_xht_bit_sel_o	output	logic[RngBusBitSelWidth-1:0]
entropy_src_xht_health_test_window_o	output	logic [HealthTestWindowWidth-1:0]
entropy_src_xht_meta_o	output	entropy_src_xht_meta_req_t
entropy_src_xht_meta_i	input	entropy_src_xht_meta_rsp_t
recov_alert_test_o	output	logic
fatal_alert_test_o	output	logic
recov_alert_o	output	logic
fatal_alert_o	output	logic
intr_es_entropy_valid_o	output	logic
intr_es_health_test_failed_o	output	logic
intr_es_observe_fifo_ready_o	output	logic
intr_es_fatal_err_o	output	logic

API — entropy_src_enable_delay.sv

Table 287. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv — module / interface / package (1개)

이름	종류
entropy_src_enable_delay	module

Table 288. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv — localparam (1개)

상수	형/폭	값
MaxFifoWait	int	4

Table 289. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
esrng_fifo_not_empty_i	input	logic
esbit_fifo_not_empty_i	input	logic
postht_fifo_not_empty_i	input	logic
distr_fifo_not_empty_i	input	logic
sha3_block_busy_i	input	logic

포트	방향	형/폭
sha3_block_processed_i	input	logic
bypass_mode_i	input	logic
enable_o	output	logic

API — entropy_src_field_en.sv

Table 290. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — module / interface / package (1개)

이름	종류
entropy_src_field_en	module

Table 291. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — parameter (2개)

파라미터	형/폭	기본값
FieldW	int	4
FieldEnVal	int	'ha

Table 292. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — localparam (2개)

상수	형/폭	값
FieldEnValBit	logic [FieldW-1:0]	FieldW'(FieldEnVal)
FieldEnValBitRevert	logic [FieldW-1:0]	~FieldEnValBit

Table 293. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
wvalid_i	input	logic
wdata_i	input	logic [FieldW-1:0]
enable_o	output	logic

API — entropy_src_main_sm.sv

Table 294. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm.sv — module / interface / package (1개)

이름	종류
entropy_src_main_sm	module

Table 295. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic

포트	방향	형/폭
fw_ov_ent_insert_i	input	logic
fw_ov_sha3_start_i	input	logic
ht_done_pulse_i	input	logic
pd_cntr_zero_i	input	logic
ht_fail_pulse_i	input	logic
alert_thresh_fail_i	input	logic
alert_cntr_clr_ok_o	output	logic
bypass_mode_i	input	logic
bypass_stage_rdy_i	input	logic
sha3_state_vld_i	input	logic
main_stage_push_o	output	logic
bypass_stage_pop_o	output	logic
boot_phase_done_o	output	logic
sha3_start_o	output	logic
sha3_process_o	output	logic
sha3_done_o	output	prim_mubi_pkg::mubi4_t
local_escalate_i	input	logic
main_sm_alert_o	output	logic
main_sm_idle_o	output	logic
main_sm_state_o	output	logic [StateWidth-1:0]
main_sm_err_o	output	logic

API — entropy_src_main_sm_pkg.sv

Table 296. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm_pkg.sv — module / interface / package (1개)

이름	종류
entropy_src_main_sm_pkg	package

Table 297. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm_pkg.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	9

API — entropy_src_markov_ht.sv

Table 298. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv — module / interface / package (1개)

이름	종류
entropy_src_markov_ht	module

Table 299. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv — parameter (3개)

파라미터	형/폭	기본값
RegWidth	int	16
RngBusWidth	int	4
RngBusBitSelWidth	int	2

Table 300. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
entropy_bit_i	input	logic [RngBusWidth-1:0]
entropy_bit_vld_i	input	logic
rng_bit_en_i	input	logic
rng_bit_sel_i	input	logic [RngBusBitSelWidth-1:0]
clear_i	input	logic
active_i	input	logic
thresh_hi_i	input	logic [RegWidth-1:0]
thresh_lo_i	input	logic [RegWidth-1:0]
window_wrap_pulse_i	input	logic
threshold_scope_i	input	logic
test_cnt_hi_o	output	logic [RegWidth-1:0]
test_cnt_lo_o	output	logic [RegWidth-1:0]
test_fail_hi_pulse_o	output	logic
test_fail_lo_pulse_o	output	logic
count_err_o	output	logic

API — entropy_src_pkg.sv**Table 301. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_pkg.sv — module / interface / package (1개)**

이름	종류
entropy_src_pkg	package

Table 302. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_pkg.sv — parameter (9개)

파라미터	형/폭	기본값
CSRNG_BUS_WIDTH	int	384
FIPS_BUS_WIDTH	int	1
FIPS_CSRNG_BUS_WIDTH	int	FIPS_BUS_WIDTH + CSRNG_BUS_WIDTH
BucketHtDataMaxWidth	int	4

파라미터	형/폭	기본값
ENTROPY_SRC_HW_IF_REQ_DEFAULT	entropy_src_hw_if_req_t	{default: '0}
ENTROPY_SRC_HW_IF_RSP_DEFAULT	entropy_src_hw_if_rsp_t	{default: '0}
ENTROPY_SRC_XHT_META_REQ_DEFAULT	entropy_src_xht_meta_req_t	{default: '0}
ENTROPY_SRC_XHT_META_RSP_DEFAULT	entropy_src_xht_meta_rsp_t	{test_cnt_lo: 16'hffff}
HT_WATERMARK_NUM_WIDTH	int	4

1. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src.sv

(481 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src top level wrapper file
6
7 `include "prim_assert.sv"
8
9
10 module entropy_src
11     import entropy_src_pkg::*;
12     import entropy_src_reg_pkg::*;
13     import prim_mubi_pkg::mubi8_t;
14 # (
15     parameter logic [NumAlerts-1:0] AlertAsyncOn    = {Num-
mAlerts{1'b1}},
16     // Number of cycles a differential skew is tolerated on the
alert signal
17     parameter int unsigned AlertSkewCycles           = 1,
18     parameter int RngBusWidth                       = 4,
19     parameter int RngBusBitSelWidth                 = 2,
20     parameter int HealthTestWindowWidth             = 18,
21     parameter int EsFifoDepth                       = 3,
22     parameter int DistrFifoDepth                    = 3,
23     parameter bit Stub                              = 1'b0
24 ) (
25     input logic clk_i,
26     input logic rst_ni,
27
28     // Bus Interface
29     input tlul_pkg::tl_h2d_t tl_i,
30     output tlul_pkg::tl_d2h_t tl_o,
31
32     // OTP Interface
33     // SEC_CM: INTERSIG.MUBI
34     input mubi8_t otp_en_entropy_src_fw_read_i,
35     // SEC_CM: INTERSIG.MUBI
36     input mubi8_t otp_en_entropy_src_fw_over_i,
37
38     // RNG Interface
39     output logic rng_fips_o,
40
41     // Entropy Interface
42     input entropy_src_hw_if_req_t entropy_src_hw_if_i,
43     output entropy_src_hw_if_rsp_t entropy_src_hw_if_o,
44
45     // RNG Interface
46     output logic          entropy_src_rng_enable_o,
47     input logic          entropy_src_rng_valid_i,
48     input logic [RngBusWidth-1:0] entropy_src_rng_bits_i,
49
50     // External Health Test Interface
51     output logic          entropy_src_xht_valid_o,
52     output logic [RngBusWidth-1:0] entropy_src_xht_bits_o,
53     output logic [RngBusBitSelWidth-1:0] entropy_src_xht_bit_sel_o,
54     output logic [HealthTestWindowWidth-1:0] entropy_src_xht_health_test_window_o,
55     output entropy_src_xht_meta_req_t entropy_src_xht_meta_req_o,
56     input entropy_src_xht_meta_rsp_t entropy_src_xht_meta_rsp_i,
57
58     // Alerts
59     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0] alert_rx_i,
60     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o,
61
62     // Interrupts
63     output logic intr_es_entropy_valid_o,
64     output logic intr_es_health_test_failed_o,
65     output logic intr_es_observe_fifo_ready_o,

```

```

66     output logic intr_es_fatal_err_o
67 );
68
69 // For a data width of N, there are 2^N buckets in the bucket
health test, each with its own
70 // counter. To make this health test scale reasonably well
with `RngBusWidth`, we limit the bucket
71 // health test to 4 bit. If `RngBusWidth` is larger than 4,
the bucket health test gets
72 // instantiated multiple times, once per 4 bit.
73 localparam int BucketHtDataWidth = buck-
et_ht_data_width(RngBusWidth);
74 localparam int NumBuckets = 2**BucketHtDataWidth; // bucket
health test bin count
75 localparam int unsigned NumBucketHtInst = num_buck-
et_ht_inst(RngBusWidth);
76
77 // Register interface supports up to 256-bit wide noise
source bus widths. Ensure the that the
78 // parameter does not exceed that limit.
79 `ASSERT_INIT(RngBusBitWidthMaxValue_A, RngBusWidth <= 256)
80 // The IP needs to define both RngBusWidth, RngBusBitSel-
Width, and HealthTestWindowWidth, although
81 // the last two are derived from the first. This is needed
to allow topgen to pick up the widths
82 // for interfaces IP-level interfaces based on these para-
meters.
83 `ASSERT_INIT(RngBusBitSelWidthSameAsComputed_A,
RngBusBitSelWidth ==
prim_util_pkg::vbits(RngBusWidth))
84 `ASSERT_INIT(HealthTestWindowWidthSameAsComputed_A,
HealthTestWindowWidth == 16 +
prim_util_pkg::vbits(RngBusWidth))
85
86 // common signals
87 entropy_src_hw2reg_t hw2reg;
88 entropy_src_reg2hw_t reg2hw;
89 logic [NumAlerts-1:0] alert_test;
90 logic [NumAlerts-1:0] alert;
91
92 // core signals
93 logic core_rst_n;
94 entropy_src_hw2reg_t core_hw2reg;
95 entropy_src_hw_if_rsp_t core_entropy_hw_if;
96 logic core_rng_enable;
97 entropy_src_xht_meta_req_t core_xht_meta;
98 logic core_xht_valid;
99 logic [RngBusWidth-1:0] core_xht_bits;
100 logic [RngBusBitSelWidth-1:0] core_xht_bit_sel;
101 logic [HealthTestWindowWidth-1:0]
core_xht_health_test_window;
102 logic core_intr_es_entropy_valid;
103 logic core_intr_es_health_test_failed;
104 logic core_intr_es_observe_fifo_ready;
105 logic core_intr_es_fatal_err;
106 logic [NumAlerts-1:0] core_alert_test;
107 logic [NumAlerts-1:0] core_alert;
108
109 // stub signals
110 localparam int StubLfsrWidth = 64;
111 localparam int Copies = CSRNG_BUS_WIDTH / StubLfsrWidth;
112 entropy_src_hw2reg_t stub_hw2reg;
113 entropy_src_hw_if_rsp_t stub_entropy_hw_if;
114 logic stub_es_valid;
115 logic [NumAlerts-1:0] stub_alert_test;
116 logic [NumAlerts-1:0] stub_alert;
117 logic [StubLfsrWidth-1:0] stub_lfsr_value;
118
119 // Selecting between core and stub
120
121 assign hw2reg = Stub ? stub_h-
w2reg : core_hw2reg;
122 assign core_rst_n = Stub ?
'0 : rst_ni;
123 assign entropy_src_hw_if_o = Stub ?
stub_entropy_hw_if : core_entropy_hw_if;
124 assign entropy_src_rng_enable_o = Stub ?
'1 : core_rng_enable;
125 assign entropy_src_xht_valid_o = Stub ?
'0 : core_xht_valid;
126 assign entropy_src_xht_bits_o = Stub ?

```

```

'0          : core_xht_bits;
131 assign entropy_src_xht_bit_sel_o          = Stub ?
'0          : core_xht_bit_sel;
132 assign entropy_src_xht_health_test_window_o = Stub ?
'0          :
133
core_xht_health_test_window;
134 assign entropy_src_xht_meta_o          = Stub ?
'0          : core_xht_meta;
135 assign intr_es_entropy_valid_o          = Stub ?
stub_es_valid :
136
core_intr_es_entropy_valid;
137 assign intr_es_health_test_failed_o      = Stub ?
'0          :
138
core_intr_es_health_test_failed;
139 assign intr_es_observe_fifo_ready_o      = Stub ?
'0          :
140
core_intr_es_observe_fifo_ready;
141 assign intr_es_fatal_err_o              = Stub ?
'0          : core_intr_es_fatal_err;
142 assign alert_test                      = Stub ?
stub_alert_test : core_alert_test;
143 assign alert                          = Stub ?
stub_alert : core_alert;
144
145 ///////////////////////////////////////////////////////////////////
146 // core entropy operation
147 ///////////////////////////////////////////////////////////////////
148
149 logic [NumAlerts-1:0] intg_err_alert;
150 assign intg_err_alert[0] = 1'b0;
151
152 // SEC_CM: CONFIG.REGWEN
153 // SEC_CM: TILE_LINK.BUS.INTEGRITY
154
155 entropy_src_reg_top u_reg (
156   .clk_i,
157   .rst_ni,
158   .tl_i,
159   .tl_o,
160   .reg2hw,
161   .hw2reg(hw2reg),
162   .intg_err_o(intg_err_alert[1]) // Assign this alert to the
fatal alert index.
163 );
164
165 entropy_src_core #(
166   .RngBusWidth(RngBusWidth),
167   .RngBusBitSelWidth(RngBusBitSelWidth),
168   .HealthTestWindowWidth(HealthTestWindowWidth),
169   .EsFifoDepth(EsFifoDepth),
170   .DistrFifoDepth(DistrFifoDepth),
171   .BucketHtDataWidth(BucketHtDataWidth),
172   .NumBucketHtInst(NumBucketHtInst)
173 ) u_entropy_src_core (
174   .clk_i,
175   .rst_ni(core_rst_n),
176   .reg2hw,
177   .hw2reg(core_hw2reg),
178
179   .otp_en_en-
trophy_src_fw_read_i(otp_en_entropy_src_fw_read_i),
180   .otp_en_en-
trophy_src_fw_over_i(otp_en_entropy_src_fw_over_i),
181   .rng_fips_o,
182
183   .entropy_src_hw_if_o(core_entropy_hw_if),
184   .entropy_src_hw_if_i,
185
186   .entropy_src_xht_valid_o(core_xht_valid),
187   .entropy_src_xht_bits_o(core_xht_bits),
188   .entropy_src_xht_bit_sel_o(core_xht_bit_sel),
189   .entropy_src_xht_meta_o(core_xht_meta),
190   .en-
trophy_src_xht_health_test_window_o(core_xht_health_test_window),
191   .entropy_src_xht_meta_i,
192
193   .entropy_src_rng_enable_o(core_rng_enable),
194   .entropy_src_rng_valid_i,
195   .entropy_src_rng_bits_i,
196
197   .recov_alert_o(core_alert[0]),
198   .fatal_alert_o(core_alert[1]),
199
200   .recov_alert_test_o(core_alert_test[0]),
201   .fatal_alert_test_o(core_alert_test[1]),
202
203   .intr_es_entropy_valid_o(core_intr_es_entropy_valid),
204   .intr_es_health_test_failed_o(core_intr_es_health_test_fa
iled),
205   .intr_es_ob-
serve_fifo_ready_o(core_intr_es_observe_fifo_ready),
206   .intr_es_fatal_err_o(core_intr_es_fatal_err)
207 );
208
209 ///////////////////////////////////////////////////////////////////
210 // stub entropy operation

```

```

211 ///////////////////////////////////////////////////////////////////
212
213 assign stub_alert = '0;
214 assign stub_alert_test = '0;
215 assign stub_entropy_hw_if = '{
216   es_ack: '1,
217   es_bits: {Copies{stub_lfsr_value}},
218   es_fips: '1
219 };
220 // once enabled, stub entropy is always available
221
222 import prim_mubi_pkg::mubi4_t;
223 import prim_mubi_pkg::mubi4_test_true_strict;
224
225 mubi4_t mubi_module_en;
226 assign mubi_module_en = mubi4_t'(reg2hw.module_enable.q);
227 assign stub_es_valid = mu-
bi4_test_true_strict(mubi_module_en);
228
229 if (Stub) begin : gen_stub_entropy_src
230   prim_lfsr #(
231     .LfsrDw(StubLfsrWidth),
232     .StateOutDw(StubLfsrWidth)
233   ) u_prim_lfsr (
234     .clk_i          (clk_i),
235     .rst_ni         (rst_ni),
236     .seed_en_i      ('0),
237     .seed_i         ('0),
238     .lfsr_en_i      (stub_es_valid),
239     .entropy_i       ('0),
240     .state_o        (stub_lfsr_value)
241   );
242
243   // hardwire hw2reg inputs
244   always_comb begin
245     stub_hw2reg = '0;
246
247     // as long as enable is 1, do not allow registers to be
written
248     stub_hw2reg.fw_ov_rd_data.d = stub_lfsr_value[31:0];
249     stub_hw2reg.entropy_data.d = stub_lfsr_value[31:0];
250     stub_hw2reg.debug_status.main_sm_idle.d = 1'b1;
251     stub_hw2reg.main_sm_state.d = en-
trophy_src_main_sm_pkg::Idle;
252
253     stub_hw2reg.intr_state.es_entropy_valid.de =
stub_es_valid;
254     stub_hw2reg.intr_state.es_entropy_valid.d = 1'b1;
255
256   end
257 end else begin : gen_stub_tieoff
258   assign stub_hw2reg = '0;
259   assign stub_lfsr_value = '0;
260 end
261
262 ///////////////////////////////////////////////////////////////////
263 // Alert generation
264 ///////////////////////////////////////////////////////////////////
265 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
266   prim_alert_sender #(
267     .AsyncOn(AlertAsyncOn[i]),
268     .SkewCycles(AlertSkewCycles),
269     .IsFatal(i)
270   ) u_prim_alert_sender (
271     .clk_i,
272     .rst_ni,
273     .alert_test_i ( alert_test[i]
274     .alert_req_i  ( alert[i] || intg_err_alert[i]
275     .alert_ack_o   (
276     .alert_state_o (
277     .alert_rx_i    ( alert_rx_i[i]
278     .alert_tx_o    ( alert_tx_o[i]
279   );
280 end
281
282 // Outputs should have a known value after reset
283 `ASSERT_KNOWN(TLdValidKnown0_A, tl_o.d_valid)
284 `ASSERT_KNOWN(TLReadyKnown0_A, tl_o.a_ready)
285
286 // Entropy Interface
287 `ASSERT_KNOWN(EsHwIfEsAckKnown0_A, en-
trophy_src_hw_if_o.es_ack)
288 `ASSERT_KNOWN_IF(EsHwIfEsBitsKnown0_A, en-
trophy_src_hw_if_o.es_bits,
289   entropy_src_hw_if_o.es_ack)
290 `ASSERT_KNOWN_IF(EsHwIfEsFipsKnown0_A, en-
trophy_src_hw_if_o.es_fips,
291   entropy_src_hw_if_o.es_ack)
292
293 // RNG Interface
294 `ASSERT_KNOWN(EsRngEnableKnown0_A, entropy_src_rng_enable_o)
295
296 // External Health Test Interface
297 `ASSERT_KNOWN_IF(EsXhtEntropyBitKnown0_A, en-
trophy_src_xht_bits_o, entropy_src_xht_valid_o)
298 `ASSERT_KNOWN(EsXhtEntropyBitValidKnown0_A, en-
trophy_src_xht_valid_o)
299 `ASSERT_KNOWN(EsXhtClearKnown0_A, en-
trophy_src_xht_meta_o.clear)
300 `ASSERT_KNOWN(EsXhtActiveKnown0_A, en-
trophy_src_xht_meta_o.active)

```

```

301 `ASSERT_KNOWN(EsXhtThreshHiKnown0_A, en-
tropy_src_xht_meta_o.thresh_hi)
302 `ASSERT_KNOWN(EsXhtThreshLoKnown0_A, en-
tropy_src_xht_meta_o.thresh_lo)
303 `ASSERT_KNOWN(EsXhtWindowKnown0_A, en-
tropy_src_xht_meta_o.window_wrap_pulse)
304
305 // Alerts
306 `ASSERT_KNOWN(AlertTxKnown0_A, alert_tx_o)
307
308 // Interrupts
309 `ASSERT_KNOWN(IntrEsEntropyValidKnown0_A, intr_es_en-
tropy_valid_o)
310 `ASSERT_KNOWN(IntrEsHealthTestFailedKnown0_A,
intr_es_health_test_failed_o)
311 `ASSERT_KNOWN(IntrEsFifoErrKnown0_A, intr_es_fatal_err_o)
312
313 // prim_count alerts
314 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck0_A,
u_entropy_src_core.u_prim_count_window_cntr,
alert_tx_o[1])
315
316 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
gen_bit_cntrs
317   `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck1_A,
u_en-
318   tropy_src_core.u_entropy_src_adaptp_ht.gen_cntrs[sh].u_prim_count_t
est_cnt,
319   alert_tx_o[1])
320 end : gen_bit_cntrs
321
322 for (genvar j = 0; j < NumBucketHtInst; j = j + 1) begin :
gen_symbol_match_inst_loop
323   for (genvar i = 0; i < NumBuckets; i = i + 1) begin :
gen_symbol_match
324     `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
u_en-
325     tropy_src_core.gen_health_test[j].u_entropy_src_bucket_ht.gen_symbol_
l_match[i],
326     u_prim_count_bin_cntr, alert_tx_o[1])
327   end : gen_symbol_match
328 end : gen_symbol_match_inst_loop
329
330 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
gen_pair_cntrs
331   `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
u_en-
332   tropy_src_core.u_entropy_src_markov_ht.gen_cntrs[sh].u_prim_count_p
air_cntr,
333   alert_tx_o[1])
334 end : gen_pair_cntrs
335
336 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
gen_rep_cntrs
337   `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck_A,
u_en-
338   tropy_src_core.u_entropy_src_repcnt_ht.gen_cntrs[sh].u_prim_count_r
ep_cntr,
339   alert_tx_o[1])
340 end : gen_rep_cntrs
341
342 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck7_A,
u_en-
343   tropy_src_core.u_entropy_src_repcnts_ht.u_prim_count_rep_cntr,
alert_tx_o[1])
344
345 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlMainFsmCheck_A,
u_entropy_src_core.u_entropy_src_main_sm.u_state_regs,
alert_tx_o[1])
346
347 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlAckFsmCheck_A,
u_entropy_src_core.u_entropy_src_ack_sm.u_state_regs,
alert_tx_o[1])
348
349 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(PipelineDepthCheck_A,
u_entropy_src_core.u_prim_count_pipeline_depth,
alert_tx_o[1])
350
351 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3FsmCheck_A,
u_entropy_src_core.u_sha3.u_state_regs, alert_tx_o[1])
352
353 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KeccakRoundFsmCheck_A,
u_entropy_src_core.u_sha3.u_keccak.u_state_regs,
alert_tx_o[1])
354
355 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SHA3padFsmCheck_A,
u_entropy_src_core.u_sha3.u_pad.u_state_regs,
alert_tx_o[1])
356
357 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SentMsgCountCheck_A,
u_entropy_src_core.u_sha3.u_pad.u_sentmsg_count,
alert_tx_o[1])
358
359 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RoundCountCheck_A,
u_entropy_src_core.u_sha3.u_keccak.u_round_count,
alert_tx_o[1])
360
361 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck8_A,
u_en-
362   tropy_src_core.u_entropy_src_cntr_reg_repcnt.u_prim_count_cntr_reg,
alert_tx_o[1])
363

```

```

374
375 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck9_A,
u_en-
376   tropy_src_core.u_entropy_src_cntr_reg_repcnts.u_prim_count_cntr_reg,
alert_tx_o[1])
377
378 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck10_A,
u_en-
379   tropy_src_core.u_entropy_src_cntr_reg_adaptp_hi.u_prim_count_cntr_r
eg,
380   alert_tx_o[1])
381
382 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck11_A,
u_en-
383   tropy_src_core.u_entropy_src_cntr_reg_adaptp_lo.u_prim_count_cntr_r
eg,
384   alert_tx_o[1])
385
386 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck12_A,
u_en-
387   tropy_src_core.u_entropy_src_cntr_reg_bucket.u_prim_count_cntr_reg,
alert_tx_o[1])
388
389 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck13_A,
u_en-
390   tropy_src_core.u_entropy_src_cntr_reg_markov_hi.u_prim_count_cntr_r
eg,
391   alert_tx_o[1])
392
393 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck14_A,
u_en-
394   tropy_src_core.u_entropy_src_cntr_reg_markov_lo.u_prim_count_cntr_r
eg,
395   alert_tx_o[1])
396
397 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck15_A,
u_en-
398   tropy_src_core.u_entropy_src_cntr_reg_extht_hi.u_prim_count_cntr_reg
,
399   alert_tx_o[1])
400
401 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck16_A,
u_en-
402   tropy_src_core.u_entropy_src_cntr_reg_extht_lo.u_prim_count_cntr_reg
,
403   alert_tx_o[1])
404
405 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck17_A,
u_en-
406   tropy_src_core.u_entropy_src_cntr_reg_any_alert_fails.u_prim_count_
cntr_reg,
407   alert_tx_o[1])
408
409 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck18_A,
u_en-
410   tropy_src_core.u_entropy_src_cntr_reg_repcnt_alert_fails.u_prim_cou
nt_cntr_reg,
411   alert_tx_o[1])
412
413 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck19_A,
u_en-
414   tropy_src_core.u_entropy_src_cntr_reg_repcnts_alert_fails.u_prim_co
unt_cntr_reg,
415   alert_tx_o[1])
416
417 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck20_A,
u_en-
418   tropy_src_core.u_entropy_src_cntr_reg_adaptp_hi_alert_fails.u_prim_
count_cntr_reg,
419   alert_tx_o[1])
420
421 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck21_A,
u_en-
422   tropy_src_core.u_entropy_src_cntr_reg_adaptp_lo_alert_fails.u_prim_
count_cntr_reg,
423   alert_tx_o[1])
424
425 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck22_A,
u_en-
426   tropy_src_core.u_entropy_src_cntr_reg_bucket_alert_fails.u_prim_cou
nt_cntr_reg,
427   alert_tx_o[1])
428
429 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck23_A,
u_en-
430   tropy_src_core.u_entropy_src_cntr_reg_markov_hi_alert_fails.u_prim_
count_cntr_reg,
431   alert_tx_o[1])
432
433 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck24_A,
u_en-
434   tropy_src_core.u_entropy_src_cntr_reg_markov_lo_alert_fails.u_prim_
count_cntr_reg,
435   alert_tx_o[1])
436
437 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck25_A,
u_en-
438   tropy_src_core.u_entropy_src_cntr_reg_extht_hi_alert_fails.u_prim_c
ount_cntr_reg,
439   alert_tx_o[1])
440
441
442
443
444
445

```

```

446 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntAlertCheck26_A,
447   u_en-
448   tropy_src_core.u_entropy_src_cntr_reg_extht_lo_alert_fails.u_prim_c
449   ount_cntr_reg,
450   alert_tx_o[1])
451 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsrngFifoWptrCheck_A,
452   u_en-
453   tropy_src_core.u_prim_fifo_sync_esrng.gen_normal_fifo.u_fifo_cnt.ge
454   n_secure_ptrs.u_wptr,
455   alert_tx_o[1])
456 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsrngFifoRptrCheck_A,
457   u_en-
458   tropy_src_core.u_prim_fifo_sync_distr.gen_normal_fifo.u_fifo_cnt.ge
459   n_secure_ptrs.u_wptr,
460   alert_tx_o[1])
461 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(DistrFifoRptrCheck_A,
462   u_en-
463   tropy_src_core.u_prim_fifo_sync_distr.gen_normal_fifo.u_fifo_cnt.ge
464   n_secure_ptrs.u_rptr,
465   alert_tx_o[1])
466 `AS-
467 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ObserveFifoWptrCheck_A,
468   u_en-
469   tropy_src_core.u_prim_fifo_sync_observe.gen_normal_fifo.u_fifo_cnt.
470   gen_secure_ptrs.u_wptr,
471   alert_tx_o[1])
472 `AS-
473 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ObserveFifoRptrCheck_A,
474   u_en-
475   tropy_src_core.u_prim_fifo_sync_observe.gen_normal_fifo.u_fifo_cnt.
476   gen_secure_ptrs.u_rptr,
477   alert_tx_o[1])
478 `AS-
479 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsfinalFifoWptrCheck_A,
480   u_en-
481   tropy_src_core.u_prim_fifo_sync_esfinal.gen_normal_fifo.u_fifo_cnt.
482   gen_secure_ptrs.u_wptr,
483   alert_tx_o[1])
484 `AS-
485 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(EsfinalFifoRptrCheck_A,
486   u_en-
487   tropy_src_core.u_prim_fifo_sync_esfinal.gen_normal_fifo.u_fifo_cnt.
488   gen_secure_ptrs.u_rptr,
489   alert_tx_o[1])
490 // Alert assertions for reg_we onehot check
491 `AS-
492 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
493   u_reg, alert_tx_o[1])
494 endmodule

```

2. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm.sv (66 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Description: interface between a req/ack interface and a
6 // fifo
7 //
8 // include "prim_assert.sv"
9 //
10 module entropy_src_ack_sm (
11   input logic clk_i,
12   input logic rst_ni,
13
14   input logic enable_i,
15   input logic req_i,
16   output logic ack_o,
17   input logic fifo_not_empty_i,
18   input logic local_escalate_i,
19   output logic fifo_pop_o,
20   output logic ack_sm_err_o
21 );
22
23 import entropy_src_ack_sm_pkg::*;
24
25 state_e state_d, state_q;
26
27 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
28 state_e, Idle)
29
30 always_comb begin
31   state_d = state_q;
32   ack_o = 1'b0;
33   fifo_pop_o = 1'b0;

```

```

33   ack_sm_err_o = 1'b0;
34   unique case (state_q)
35     Idle: begin
36       if (enable_i) begin
37         if (req_i) begin
38           state_d = Wait;
39         end
40       end
41     end
42     Wait: begin
43       if (!enable_i) begin
44         state_d = Idle;
45       end else begin
46         if (fifo_not_empty_i) begin
47           ack_o = 1'b1;
48           fifo_pop_o = 1'b1;
49           state_d = Idle;
50         end
51       end
52     end
53     Error: begin
54       ack_sm_err_o = 1'b1;
55     end
56     default: begin
57       state_d = Error;
58       ack_sm_err_o = 1'b1;
59     end
60   endcase
61   if (local_escalate_i) begin
62     state_d = Error;
63   end
64 end
65
66 endmodule

```

3. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_ack_sm_pkg.sv (35 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // State definitions for entropy_src_ack_sm, provided as a
6 // separate package for use in DV
7 package entropy_src_ack_sm_pkg;
8
9 // Encoding generated with:
10 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 3 -n 6 \
11 //   -s 1236774883 --language=sv
12 //
13 // Hamming distance histogram:
14 //
15 // 0: --
16 // 1: --
17 // 2: --
18 // 3: ||||| (33.33%)
19 // 4: ||||| (33.33%)
20 // 5: ||||| (33.33%)
21 // 6: --
22 //
23 // Minimum Hamming distance: 3
24 // Maximum Hamming distance: 5
25 // Minimum Hamming weight: 1
26 // Maximum Hamming weight: 4
27 //
28 localparam int StateWidth = 6;
29 typedef enum logic [StateWidth-1:0] {
30   Idle = 6'b011101, // idle
31   Wait = 6'b101100, // wait until the fifo has an entry
32   Error = 6'b000010 // illegal state reached and hang
33 } state_e;
34
35 endpackage

```

4. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_adaptp_ht.sv (138 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Description: entropy_src Adaptive Proportion Test module
6 //
7
8 module entropy_src_adaptp_ht #(
9   parameter int RegWidth = 16,
10  parameter int RngBusWidth = 4,
11  parameter int RngBusBitSelWidth = 2
12 ) (
13   input logic clk_i,
14   input logic rst_ni,
15
16   // ins req interface
17   input logic [RngBusWidth-1:0] entropy_bit_i,

```

```

18 input logic entropy_bit_vld_i,
19 input logic rng_bit_en_i,
20 input logic [RngBusBitSelWidth-1:0] rng_bit_sel_i,
21 input logic clear_i,
22 input logic active_i,
23 input logic [RegWidth-1:0] thresh_hi_i,
24 input logic [RegWidth-1:0] thresh_lo_i,
25 input logic window_wrap_pulse_i,
26 input logic threshold_scope_i,
27 output logic [RegWidth-1:0] test_cnt_hi_o,
28 output logic [RegWidth-1:0] test_cnt_lo_o,
29 output logic test_fail_hi_pulse_o,
30 output logic test_fail_lo_pulse_o,
31 output logic count_err_o
32 );
33
34 // signals
35 logic [RegWidth-1:0] test_cnt_max;
36 logic [RegWidth-1:0] test_cnt_min,
test_cnt_min_tmp;
37 logic [RegWidth-1:0] test_cnt_sum;
38 logic [RngBusWidth-1:0][RegWidth-1:0] test_cnt;
39 logic [RegWidth-1:0] rng_bit_cnt;
40 logic [RngBusWidth-1:0] test_cnt_err;
41
42 // Adaptive Proportion Test
43 //
44 // Test operation
45 // This is an approved modification of the NIST Adaptive
Proportion test in that
46 // instead of counting the first sampled value (1'b1 or
1'b0), it will count
47 // only the 1's on all four bit streams and accumulate for
the during of the
48 // window size (W) of the test.
49
50 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
gen_cntrs
51
52 // cumulative ones counter
53 // SEC_CM: CTR.REDUN
54 prim_count #(
55 .Width(RegWidth)
56 ) u_prim_count_test_cnt (
57 .clk_i,
58 .rst_ni,
59 .clr_i(window_wrap_pulse_i),
60 .set_i(!active_i || clear_i),
61 .set_cnt_i(RegWidth'(0)),
62 .incr_en_i(entropy_bit_vld_i),
63 .decr_en_i(1'b0),
64 .step_i(RegWidth'(entropy_bit_i[sh])),
65 .commit_i(1'b1),
66 .cnt_o(test_cnt[sh]),
67 .cnt_after_commit_o(),
68 .err_o(test_cnt_err[sh])
69 );
70 end : gen_cntrs
71
72 // determine the highest counter counter value
73 prim_max_tree #(
74 .NumSrc(RngBusWidth),
75 .Width(RegWidth)
76 ) u_max (
77 .clk_i (clk_i),
78 .rst_ni (rst_ni),
79 .values_i (test_cnt),
80 .valid_i ({RngBusWidth{1'b1}}),
81 .max_value_o (test_cnt_max),
82 .max_idx_o (),
83 .max_valid_o ()
84 );
85
86 // determine the lowest counter value
87 // Negate the inputs and outputs of prim_max_tree to find
the minimum
88 // For this unsigned application, one's complement negation
(i.e. logical inversion) is fine.
89 prim_max_tree #(
90 .NumSrc(RngBusWidth),
91 .Width(RegWidth)
92 ) u_min (
93 .clk_i (clk_i),
94 .rst_ni (rst_ni),
95 .values_i (~test_cnt),
96 .valid_i ({RngBusWidth{1'b1}}),
97 .max_value_o (test_cnt_min_tmp),
98 .max_idx_o (),
99 .max_valid_o ()
100 );
101
102 assign test_cnt_min = ~test_cnt_min_tmp;
103
104 prim_sum_tree #(
105 .NumSrc(RngBusWidth),
106 .InWidth(RegWidth)
107 ) u_sum (
108 .clk_i (clk_i),
109 .rst_ni (rst_ni),
110 .values_i (test_cnt),
111 .valid_i ({RngBusWidth{1'b1}}),

```

```

112 .sum_value_o (test_cnt_sum),
113 .sum_valid_o ()
114 );
115
116 always_comb begin
117   rng_bit_cnt = '0;
118
119   for (int i = 0; i < RngBusWidth; i++) begin
120     if (rng_bit_sel_i == i) begin
121       rng_bit_cnt = test_cnt[i];
122     end
123   end
124 end
125
126 assign test_cnt_hi_o = rng_bit_en_i ? rng_bit_cnt :
threshold_scope_i ? test_cnt_sum :
test_cnt_max;
127
128 assign test_cnt_lo_o = rng_bit_en_i ? rng_bit_cnt :
threshold_scope_i ? test_cnt_sum :
test_cnt_min;
129
130 // the pulses will be only one clock in length
131 assign test_fail_hi_pulse_o = active_i && win-
dow_wrap_pulse_i && (test_cnt_hi_o > thresh_hi_i);
132 assign test_fail_lo_pulse_o = active_i && win-
dow_wrap_pulse_i && (test_cnt_lo_o < thresh_lo_i);
133 assign count_err_o = |test_cnt_err;
134
135 endmodule

```

5. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_bucket_ht.sv (90 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src bucket health test module
6 //
7
8 module entropy_src_bucket_ht #(
9   parameter int RegWidth = 16,
10  parameter int RngBusWidth = 4
11 ) (
12   input logic clk_i,
13   input logic rst_ni,
14
15   // ins req interface
16   input logic [RngBusWidth-1:0] entropy_bit_i,
17   input logic entropy_bit_vld_i,
18   input logic clear_i,
19   input logic active_i,
20   input logic [RegWidth-1:0] thresh_i,
21   input logic window_wrap_pulse_i,
22   output logic [RegWidth-1:0] test_cnt_o,
23   output logic test_fail_pulse_o,
24   output logic count_err_o
25 );
26
27 localparam int NUM_BINS = 2**RngBusWidth;
28
29 // signals
30 logic [NUM_BINS-1:0] bin_incr;
31 logic [NUM_BINS-1:0] bin_cnt_exceeds_thresh;
32 logic [NUM_BINS - 1:0][RegWidth - 1:0] bin_cntr;
33 logic [NUM_BINS-1:0] bin_cntr_err;
34 logic [RegWidth-1:0] bin_max;
35
36 // Bucket Test
37 //
38 // Test operation
39 // This test will look at 4 bit symbols and increment one
of sixteen
40 // counters, or buckets, to show a histogram of the data
stream.
41 // An error will occur if one of the counters reaches the
thresh
42 // value.
43
44 // Analyze the incoming symbols
45
46 for (genvar i = 0; i < NUM_BINS; i = i + 1) begin : gen_sym-
bol_match
47
48 // set the bin incremter if the symbol matches that bin
49 assign bin_incr[i] = entropy_bit_vld_i && (entropy_bit_i
== i);
50
51 // use the bin incremter to increase the bin total count
52 // SEC_CM: CTR.REDUN
53 prim_count #(
54 .Width(RegWidth)
55 ) u_prim_count_bin_cntr (
56 .clk_i,
57 .rst_ni,
58 .clr_i(window_wrap_pulse_i),
59 .set_i(!active_i || clear_i),
60 .set_cnt_i(RegWidth'(0)),
61 .incr_en_i(bin_incr[i]),

```



```

61     .decr_en_i(1'b0),
62     .step_i(RegWidth'(1)),
63     .commit_i(1'b1),
64     .cnt_o(bin_cnr[i]),
65     .cnt_after_commit_o(),
66     .err_o(bin_cnr_err[i])
67 );
68 assign bin_cnt_exceeds_thresh[i] = (bin_cnr[i] >
thresh_i);
69 end : gen_symbol_match
70
71 prim_max_tree #(
72     .NumSrc(NUM_BINS),
73     .Width(RegWidth)
74 ) u_prim_max_tree_bin_cnr_max (
75     .clk_i,
76     .rst_ni,
77     .values_i (bin_cnr),
78     .valid_i ({RegWidth{1'b1}}),
79     .max_value_o(bin_max),
80     .max_idx_o (),
81     .max_valid_o()
82 );
83
84 // the pulses will be only one clock in length
85 assign test_fail_pulse_o = active_i && window_wrap_pulse_i
&& (|bin_cnt_exceeds_thresh);
86 assign test_cnt_o = bin_max;
87 assign count_err_o = |bin_cnr_err;
88
89
90 endmodule

```

6. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_cnr_reg.sv (45 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src counter register module
6 //
7
8 module entropy_src_cnr_reg #(
9     parameter int RegWidth = 16
10 ) (
11     input logic          clk_i,
12     input logic          rst_ni,
13
14     // functional interface
15     input logic          clear_i,
16     input logic          event_i,
17     input logic [RegWidth-1:0] step_i,
18     output logic [RegWidth-1:0] value_o,
19     output logic          err_o
20 );
21
22 // signals
23 logic [RegWidth-1:0] counter_value;
24
25 // counter will not wrap when full value is reached
26 prim_count #(
27     .Width(RegWidth)
28 ) u_prim_count_cnr_reg (
29     .clk_i,
30     .rst_ni,
31     .clr_i(clear_i),
32     .set_i(1'b0),
33     .set_cnt_i(RegWidth'(0)),
34     .incr_en_i(event_i && (counter_value != '0)),
35     .decr_en_i(1'b0),
36     .step_i,
37     .commit_i(1'b1),
38     .cnt_o(counter_value),
39     .cnt_after_commit_o(),
40     .err_o(err_o)
41 );
42
43 assign value_o = counter_value;
44
45 endmodule

```

7. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_core.sv (3,313 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src core module
6 //
7 `include "prim_assert.sv"
8
9 module entropy_src_core import entropy_src_pkg::*; #(
10     parameter int RngBusWidth = 4,

```

```

11     parameter int RngBusBitSelWidth = 2,
12     parameter int HealthTestWindowWidth = 18,
13     parameter int EsFifoDepth = 3,
14     parameter int DistrFifoDepth = 2,
15     parameter int BucketHtDataWidth = 4,
16     parameter int NumBucketHtInst =
prim_util_pkg::ceil_div(RngBusWidth, BucketHtDataWidth)
17 ) (
18     input logic clk_i,
19     input logic rst_ni,
20
21     input entropy_src_reg_pkg::entropy_src_reg2hw_t reg2hw,
22     output entropy_src_reg_pkg::entropy_src_hw2reg_t hw2reg,
23
24     // Efuse Interface
25     input prim_mubi_pkg::mubi8_t otp_en_entropy_src_fw_read_i,
26     input prim_mubi_pkg::mubi8_t otp_en_entropy_src_fw_over_i,
27
28     // RNG Interface
29     output logic rng_fips_o,
30
31     // Entropy Interface
32     input entropy_src_hw_if_req_t entropy_src_hw_if_i,
33     output entropy_src_hw_if_rsp_t entropy_src_hw_if_o,
34
35     // RNG Interface
36     output logic          entropy_src_rng_enable_o,
37     input logic          entropy_src_rng_valid_i,
38     input logic [RngBusWidth-1:0] entropy_src_rng_bits_i,
39
40     // External Health Test Interface
41     output logic          en-
entropy_src_xht_valid_o,
42     output logic [RngBusWidth-1:0] en-
entropy_src_xht_bits_o,
43     output logic [RngBusBitSelWidth-1:0] en-
entropy_src_xht_bit_sel_o,
44     output logic [HealthTestWindowWidth-1:0] en-
entropy_src_xht_health_test_window_o,
45     output entropy_src_xht_meta_req_t en-
entropy_src_xht_meta_o,
46     input entropy_src_xht_meta_rsp_t en-
entropy_src_xht_meta_i,
47
48     output logic          recov_alert_test_o,
49     output logic          fatal_alert_test_o,
50     output logic          recov_alert_o,
51     output logic          fatal_alert_o,
52
53     output logic          intr_es_entropy_valid_o,
54     output logic          intr_es_health_test_failed_o,
55     output logic          intr_es_observe_fifo_ready_o,
56     output logic          intr_es_fatal_err_o
57 );
58
59 import entropy_src_reg_pkg::*;
60 import prim_mubi_pkg::mubi4_t;
61 import prim_mubi_pkg::mubi4_test_true_strict;
62 import prim_mubi_pkg::mubi4_test_true_loose;
63 import prim_mubi_pkg::mubi4_and_hi;
64 import prim_mubi_pkg::mubi4_test_false_loose;
65 import prim_mubi_pkg::mubi4_test_invalid;
66
67 localparam int EsFifoDepthW =
prim_util_pkg::vbits(EsFifoDepth + 1);
68 localparam int PostHTWidth = 32;
69 localparam int PostHTFifoDepthW =
prim_util_pkg::vbits(PostHTWidth / RngBusWidth + 1);
70 localparam int HalfRegWidth = 16;
71 localparam int FullRegWidth = 32;
72 localparam int EighthRegWidth = 4;
73 localparam int SeedLen = 384;
74 localparam int DistrFifoWidth = 32;
75 localparam int DistrFifoDepthW =
prim_util_pkg::vbits(DistrFifoDepth + 1);
76 localparam int ObserveFifoWidth = 32;
77 localparam int PreCondWidth = 64;
78 localparam int PreCondFifoDepthW =
prim_util_pkg::vbits(PreCondWidth / PostHTWidth + 1);
79 // We can have 1 32-bit word in the post HT FIFO, DistrFi-
foDepth words in the distr FIFO, and
80 // 2 words in the precond FIFO. The final +1 is required for
when the pipeline is entirely empty.
81 localparam int PipelineDepthW =
82     prim_util_pkg::vbits(1 + DistrFifoDepth + PreCondWidth /
PostHTWidth + 1);
83 localparam int Clog2ObserveFifoDepth = $clog2(ObserveFifoDepth);
84 localparam int EsEnableCopies = 20;
85 localparam int EsEnPulseCopies = 1;
86 localparam int unsigned NumSwReadsSeed = SeedLen / FullReg-
Width;
87 localparam int unsigned LastSwRead = NumSwReadsSeed - 1;
88 localparam int unsigned SwReadIdxWidth =
prim_util_pkg::vbits(NumSwReadsSeed);
89
90 //-----
91 // SHA3parameters
92 //-----
93 // Do not enable masking

```

```

94  localparam bit Sha3EnMasking = 0;
95  // derived parameter
96  localparam int Sha3Share = (Sha3EnMasking) ? 2 : 1;
97
98  // signals
99  logic          fw_ov_mode;
100 logic          fw_ov_mode_pfe;
101 logic          fw_ov_mode_pfa;
102 logic          fw_ov_wr_fifo_full;
103 logic          fw_ov_mode_entropy_insert;
104 logic          fw_ov_entropy_insert_pfe;
105 logic          fw_ov_entropy_insert_pfa;
106 logic          fw_ov_sha3_start_pfe;
107 logic          fw_ov_sha3_start_pfe_q;
108 logic          fw_ov_sha3_start_pfa;
109 logic          fw_ov_sha3_disable_pulse;
110 logic [ObserveFifoWidth-1:0] fw_ov_wr_data;
111 logic          fw_ov_fifo_rd_pulse;
112 logic          fw_ov_fifo_wr_pulse;
113 logic          es_enable_pfa;
114
115 logic          fips_enable_pfe;
116 logic          fips_enable_pfa;
117 logic          fips_flag_pfe;
118 logic          fips_flag_pfa;
119 logic          rng_fips_pfe;
120 logic          rng_fips_pfa;
121
122 logic          rng_bit_en;
123 logic          rng_bit_enable_pfe;
124 logic          rng_bit_enable_pfa;
125 logic [RngBusBitSelWidth-1:0] rng_bit_sel;
126 logic          rng_enable_q, rng_enable_d;
127 logic          entropy_data_reg_en_pfe;
128 logic          entropy_data_reg_en_pfa;
129 logic          es_data_reg_rd_en;
130 logic          event_es_entropy_valid;
131 logic          event_es_health_test_failed;
132 logic          event_es_observe_fifo_ready;
133 logic          event_es_fatal_err;
134
135 logic [RngBusWidth-1:0] sfifo_esrng_wdata;
136 logic [RngBusWidth-1:0] sfifo_esrng_rdata;
137 logic          sfifo_esrng_push;
138 logic          sfifo_esrng_pop;
139 logic          sfifo_esrng_clr;
140 logic          sfifo_esrng_full;
141 logic          sfifo_esrng_not_empty;
142 logic          sfifo_esrng_not_full;
143 logic          sfifo_esrng_int_err;
144 logic [2:0]          sfifo_esrng_err;
145
146 logic [DistrFifoWidth-1:0] sfifo_distr_wdata;
147 logic [DistrFifoWidth-1:0] sfifo_distr_rdata;
148 logic          sfifo_distr_push;
149 logic          sfifo_distr_pop;
150 logic          sfifo_distr_clr;
151 logic          sfifo_distr_not_full;
152 logic          sfifo_distr_full;
153 logic          sfifo_distr_not_empty;
154 logic [DistrFifoDepthW-1:0] sfifo_distr_depth;
155 logic          sfifo_distr_int_err;
156 logic [2:0]          sfifo_distr_err;
157
158 logic [ObserveFifoWidth-1:0] sfifo_observe_wdata;
159 logic [ObserveFifoWidth-1:0] sfifo_observe_rdata;
160 logic          sfifo_observe_push;
161 logic          sfifo_observe_pop;
162 logic          sfifo_observe_full;
163 logic          sfifo_observe_clr;
164 logic          sfifo_observe_not_empty;
165 logic [Clog2ObserveFifoDepth:0] sfifo_observe_depth;
166 logic          sfifo_observe_int_err;
167 logic [2:0]          sfifo_observe_err;
168
169 logic [EsFifoDepthW-1:0] sfifo_esfinal_depth;
170 logic [(1+SeedLen)-1:0] sfifo_esfinal_wdata;
171 logic [(1+SeedLen)-1:0] sfifo_esfinal_rdata;
172 logic          sfifo_esfinal_push;
173 logic          sfifo_esfinal_pop;
174 logic          sfifo_esfinal_clr;
175 logic          sfifo_esfinal_not_full;
176 logic          sfifo_esfinal_full;
177 logic          sfifo_esfinal_not_empty;
178 logic          sfifo_esfinal_int_err;
179 logic [2:0]          sfifo_esfinal_err;
180
181 logic          es_sfifo_int_err;
182
183 logic [SeedLen-1:0] esfinal_data;
184 logic          esfinal_fips_flag;
185
186 logic          any_fail_pulse;
187 logic          main_stage_push;
188 logic          main_stage_push_raw;
189 logic          bypass_stage_pop;
190 logic          boot_phase_done;
191 logic [HalfRegWidth-1:0] any_fail_count;
192 logic          any_fails_cntr_err;
193 logic          alert_threshold_fail;
194 logic [HalfRegWidth-1:0] alert_threshold;
195
196 logic [HalfRegWidth-1:0] alert_threshold_inv;
197 logic [Clog2ObserveFifoDepth:0] observe_fifo_thresh;
198 logic          observe_fifo_thresh_met;
199 logic          repcnt_active;
200 logic          repcnts_active;
201 logic          adaptp_active;
202 logic          bucket_active;
203 logic          markov_active;
204 logic          extht_active;
205 logic          alert_cntrs_clr;
206 logic          health_test_clr;
207 logic [RngBusWidth-1:0] health_test_done_pulse;
208 logic          health_test_esbus;
209 logic          health_test_esbus_vld;
210 logic          es_route_pfe;
211 logic          es_route_pfa;
212 logic          es_type_pfe;
213 logic          es_type_pfa;
214 logic          es_route_to_sw;
215 logic          es_bypass_to_sw;
216 logic          es_bypass_mode;
217 logic          alert_cntr_clr_ok;
218 logic          threshold_scope;
219 logic          threshold_scope_pfe;
220 logic          threshold_scope_pfa;
221 logic          fips_compliance;
222
223 logic [HalfRegWidth-1:0] health_test_fips_window;
224 logic [HalfRegWidth-1:0] health_test_bypass_window;
225 logic [HealthTestWindowWidth-1:0] health_test_window;
226 logic [HealthTestWindowWidth-1:0] window_cntr_step;
227
228 logic threshold_oneway_pfe;
229 logic threshold_oneway_pfa;
230
231 logic [HalfRegWidth-1:0] repcnt_threshold;
232 logic [HalfRegWidth-1:0] repcnt_threshold_oneway;
233 logic          repcnt_threshold_wr;
234 logic [HalfRegWidth-1:0] repcnt_event_cnt;
235 logic [FullRegWidth-1:0] repcnt_total_fails;
236 logic [EighthRegWidth-1:0] repcnt_fail_count;
237 logic          repcnt_fail_pulse;
238 logic          repcnt_fails_cntr_err;
239 logic          repcnt_alert_cntr_err;
240
241 logic [HalfRegWidth-1:0] repcnts_threshold;
242 logic [HalfRegWidth-1:0] repcnts_threshold_oneway;
243 logic          repcnts_threshold_wr;
244 logic [HalfRegWidth-1:0] repcnts_event_cnt;
245 logic [FullRegWidth-1:0] repcnts_total_fails;
246 logic [EighthRegWidth-1:0] repcnts_fail_count;
247 logic          repcnts_fail_pulse;
248 logic          repcnts_fails_cntr_err;
249 logic          repcnts_alert_cntr_err;
250
251 logic [HalfRegWidth-1:0] adaptp_hi_threshold;
252 logic [HalfRegWidth-1:0] adaptp_hi_threshold_oneway;
253 logic          adaptp_hi_threshold_wr;
254 logic [HalfRegWidth-1:0] adaptp_lo_threshold;
255 logic [HalfRegWidth-1:0] adaptp_lo_threshold_oneway;
256 logic          adaptp_lo_threshold_wr;
257 logic [HalfRegWidth-1:0] adaptp_hi_event_cnt;
258 logic [HalfRegWidth-1:0] adaptp_lo_event_cnt;
259 logic [FullRegWidth-1:0] adaptp_hi_total_fails;
260 logic [FullRegWidth-1:0] adaptp_lo_total_fails;
261 logic [EighthRegWidth-1:0] adaptp_hi_fail_count;
262 logic [EighthRegWidth-1:0] adaptp_lo_fail_count;
263 logic          adaptp_hi_fail_pulse;
264 logic          adaptp_lo_fail_pulse;
265 logic          adaptp_hi_fails_cntr_err;
266 logic          adaptp_lo_fails_cntr_err;
267 logic          adaptp_hi_alert_cntr_err;
268 logic          adaptp_lo_alert_cntr_err;
269
270 logic [HalfRegWidth-1:0] bucket_threshold;
271 logic [HalfRegWidth-1:0] bucket_threshold_oneway;
272 logic          bucket_threshold_wr;
273 logic [NumBucketHtInst-1:0][HalfRegWidth-1:0] bucket_event_cnt;
274 logic [HalfRegWidth-1:0] bucket_event_cnt_max;
275 logic [FullRegWidth-1:0] bucket_total_fails;
276 logic [EighthRegWidth-1:0] bucket_fail_count;
277 logic [NumBucketHtInst-1:0] bucket_fail_pulse;
278 logic [prim_util_pkg::vbits(NumBucketHtInst+1)-1:0] bucket_fail_pulse_step;
279 logic          bucket_fails_cntr_err;
280 logic          bucket_alert_cntr_err;
281
282 logic [HalfRegWidth-1:0] markov_hi_threshold;
283 logic [HalfRegWidth-1:0] markov_hi_threshold_oneway;
284 logic          markov_hi_threshold_wr;
285 logic [HalfRegWidth-1:0] markov_lo_threshold;
286 logic [HalfRegWidth-1:0] markov_lo_threshold_oneway;
287 logic          markov_lo_threshold_wr;
288 logic [HalfRegWidth-1:0] markov_hi_event_cnt;
289 logic [HalfRegWidth-1:0] markov_lo_event_cnt;
290 logic [FullRegWidth-1:0] markov_hi_total_fails;
291 logic [FullRegWidth-1:0] markov_lo_total_fails;
292 logic [EighthRegWidth-1:0] markov_hi_fail_count;
293 logic [EighthRegWidth-1:0] markov_lo_fail_count;
294 logic          markov_hi_fail_pulse;

```



```

294 logic markov_lo_fail_pulse;
295 logic markov_hi_fails_cntr_err;
296 logic markov_lo_fails_cntr_err;
297 logic markov_hi_alert_cntr_err;
298 logic markov_lo_alert_cntr_err;
299
300 logic [HalfRegWidth-1:0] extht_hi_threshold;
301 logic [HalfRegWidth-1:0] extht_hi_threshold_oneway;
302 logic extht_hi_threshold_wr;
303 logic [HalfRegWidth-1:0] extht_lo_threshold;
304 logic [HalfRegWidth-1:0] extht_lo_threshold_oneway;
305 logic extht_lo_threshold_wr;
306 logic [HalfRegWidth-1:0] extht_event_cnt_hi;
307 logic [HalfRegWidth-1:0] extht_event_cnt_lo;
308 logic [FullRegWidth-1:0] extht_hi_total_fails;
309 logic [FullRegWidth-1:0] extht_lo_total_fails;
310 logic [EighthRegWidth-1:0] extht_hi_fail_count;
311 logic [EighthRegWidth-1:0] extht_lo_fail_count;
312 logic extht_hi_fail_pulse;
313 logic extht_lo_fail_pulse;
314 logic extht_cnt_test;
315 logic extht_hi_fails_cntr_err;
316 logic extht_lo_fails_cntr_err;
317 logic extht_hi_alert_cntr_err;
318 logic extht_lo_alert_cntr_err;
319
320 ht_watermark_num_e ht_watermark_num_reg_if;
321 logic ht_watermark_high;
322 logic ht_watermark_event;
323 logic ht_watermark_event_pre;
324 logic [HalfRegWidth-1:0] ht_watermark_cnt;
325 logic [HalfRegWidth-1:0] ht_watermark;
326
327 logic pfifo_esbit_wdata;
328 logic [RngBusWidth-1:0] pfifo_esbit_rdata;
329 logic pfifo_esbit_not_empty;
330 logic pfifo_esbit_not_full;
331 logic pfifo_esbit_push;
332 logic pfifo_esbit_clr;
333 logic pfifo_esbit_pop;
334
335 logic [RngBusWidth-1:0] pfifo_postht_wdata;
336 logic [PostHTWidth-1:0] pfifo_postht_rdata;
337 logic pfifo_postht_not_empty;
338 logic pfifo_postht_not_full;
339 logic pfifo_postht_push;
340 logic pfifo_postht_clr;
341 logic pfifo_postht_pop;
342 logic [PostHTFifoDepthW-1:0] pfifo_postht_depth;
343
344 logic [PreCondWidth-1:0] pfifo_cond_wdata;
345 logic [SeedLen-1:0] pfifo_cond_rdata;
346 logic pfifo_cond_push;
347
348 logic [ObserveFifoWidth-1:0] pfifo_precon_wdata;
349 logic [PreCondWidth-1:0] pfifo_precon_rdata;
350 logic pfifo_precon_not_empty;
351 logic pfifo_precon_not_full;
352 logic pfifo_precon_push;
353 logic pfifo_precon_clr;
354 logic pfifo_precon_pop;
355 logic [PreCondFifoDepthW-1:0] pfifo_precon_depth;
356
357 logic [PostHTWidth-1:0] pfifo_bypass_wdata;
358 logic [SeedLen-1:0] pfifo_bypass_rdata;
359 logic pfifo_bypass_not_empty;
360 logic pfifo_bypass_not_full;
361 logic pfifo_bypass_push;
362 logic pfifo_bypass_clr;
363 logic pfifo_bypass_pop;
364
365 logic [PipelineDepthW-1:0] pipeline_depth;
366 logic pipeline_depth_cntr_err;
367
368 logic [SwReadIdxWidth-1:0] swread_idx_d, swread_idx_q;
369 logic swread_idx_incr, swread_idx_clr;
370 logic [FullRegWidth-1:0] swread_data, swread_data_buf;
371 logic swread_done;
372
373 logic [SeedLen-1:0] final_es_data;
374 logic es_hw_if_req;
375 logic es_hw_if_ack;
376 logic es_hw_if_fifo_pop;
377 logic sfifo_esrng_err_sum;
378 logic sfifo_distr_err_sum;
379 logic sfifo_observe_err_sum;
380 logic sfifo_esfinal_err_sum;
381 // For fifo errors that are generated through the
382 // ERR_CODE_TEST register, but are not associated
383 // with any errors:
384 logic sfifo_test_err_sum;
385 logic es_ack_sm_err_sum;
386 logic es_ack_sm_err;
387 logic es_main_sm_err_sum;
388 logic es_main_sm_err;
389 logic es_main_sm_alert;
390 logic es_bus_cmp_alert;
391 logic es_thresh_cfg_alert;
392 logic es_main_sm_idle;
393 logic [8:0] es_main_sm_state;
394 logic fifo_write_err_sum;
395 logic fifo_read_err_sum;
396 logic fifo_status_err_sum;
397 logic [30:0] err_code_test_bit;
398 logic sha3_msgfifo_ready;
399 logic sha3_state_vld;
400 logic sha3_start_raw;
401 logic sha3_start;
402 logic sha3_process;
403 logic sha3_block_processed;
404 prim_mubi_pkg::mubi4_t sha3_done;
405 prim_mubi_pkg::mubi4_t sha3_absorbed;
406 logic sha3_squeezing;
407 logic [2:0] sha3_fsm;
408 logic [32:0] sha3_err;
409 logic sha3_block_busy;
410 logic [HealthTestWindowWidth-1:0] window_cntr;
411 logic window_cntr_incr_en;
412
413 logic [sha3_pkg::StateW-1:0] sha3_state[Sha3Share];
414 logic [PreCondWidth-1:0] msg_data[Sha3Share];
415 logic es_rdata_capt_vld;
416 logic window_cntr_err;
417 logic repcnt_cntr_err;
418 logic repcnts_cntr_err;
419 logic adaptp_cntr_err;
420 logic [NumBucketHtInst-1:0] bucket_cntr_err;
421 logic markov_cntr_err;
422 logic es_cntr_err;
423 logic es_cntr_err_sum;
424 logic sha3_state_error_sum;
425 logic sha3_rst_storage_err_sum;
426 logic efuse_es_sw_reg_en;
427 logic efuse_es_sw_ov_en;
428
429 logic sha3_state_error;
430 logic sha3_count_error;
431 logic sha3_rst_storage_err;
432 logic es_hw_regwen;
433 logic recov_alert_state;
434 logic es_fw_ov_wr_alert;
435 logic es_fw_ov_disable_alert;
436 logic fw_ov_corrupted;
437 logic postht_entropy_drop_alert;
438
439 logic stale_seed_processing;
440 logic main_sm_enable;
441 logic main_sm_done_pulse;
442 logic main_sm_ht_failed;
443
444 logic unused_err_code_test_bit;
445 logic unused_sha3_state;
446 logic unused_entropy_data;
447 logic unused_fw_ov_rd_data;
448 logic unused_sfifo_esrng_not_full;
449 logic unused_sfifo_esfinal_not_full;
450
451 prim_mubi_pkg::mubi8_t en_entropy_src_fw_read;
452 prim_mubi_pkg::mubi8_t en_entropy_src_fw_over;
453
454 mubi4_t mubi_es_enable;
455 mubi4_t mubi_module_en_pulse;
456
457 mubi4_t mubi_module_en_raw;
458 mubi4_t [2:0] mubi_module_en_raw_fanout;
459
460 mubi4_t [EsEnableCopies-1:0] mubi_es_enable_fanout;
461 logic [EsEnableCopies-1:0] es_enable_fo;
462
463 mubi4_t [EsEnPulseCopies-1:0] mubi_module_en_pulse_fanout;
464 logic [EsEnPulseCopies-1:0] module_en_pulse_fo;
465
466 // A delayed copy of the enable signal, and enable pulse
467 // which are needed to cleanly handle any
468 // long-duration operations from the previous run.
469 logic es_delayed_enable;
470
471 mubi4_t [1:0] mubi_rng_bit_en_fanout;
472 mubi4_t mubi_rng_bit_en;
473
474 // flops
475 logic ht_failed_q, ht_failed_q, ht_failed_d;
476 logic ht_done_pulse_q, ht_done_pulse_q,
477 ht_done_pulse_d;
478 logic sha3_err_q, sha3_err_d;
479 logic [63:0] es_rdata_capt_q, es_rdata_capt_d;
480 logic es_rdata_capt_vld_q, es_rdata_capt_vld_d;
481 mubi4_t mubi_mod_en_dly_d, mubi_mod_en_dly_q;
482
483 ht_watermark_num_e ht_watermark_num_q, ht_watermark_num_d;
484
485 logic sha3_start_mask_q, sha3_start_mask_d;
486 logic sha3_flush_q, sha3_flush_d;
487 logic [1:0] fw_ov_corrupted_q, fw_ov_corrupted_d;
488
489 always_ff @(posedge clk_i or negedge rst_ni) begin
490   if (!rst_ni) begin
491     ht_failed_q <= '0;
492     ht_failed_qq <= '0;
493     ht_done_pulse_q <= '0;
494     ht_done_pulse_qq <= '0;
495     sha3_err_q <= '0;

```

```

494     es_rdata_capt_q      <= '0';
495     es_rdata_capt_vld_q  <= '0';
496     fw_ov_sha3_start_pfe_q <= '0';
497     mubi_mod_en_dly_q    <= prim_mubi_pkg::MuBi4False;
498     sha3_flush_q        <= '0';
499     sha3_start_mask_q    <= '0';
500     fw_ov_corrupted_q    <= 2'b00;
501     rng_enable_q         <= 1'b 0;
502   end else begin
503     ht_failed_q          <= ht_failed_d;
504     ht_failed_q          <= ht_failed_q;
505     ht_done_pulse_q      <= ht_done_pulse_d;
506     ht_done_pulse_q      <= ht_done_pulse_q;
507     sha3_err_q           <= sha3_err_d;
508     es_rdata_capt_q      <= es_rdata_capt_d;
509     es_rdata_capt_vld_q  <= es_rdata_capt_vld_d;
510     fw_ov_sha3_start_pfe_q <= fw_ov_sha3_start_pfe;
511     sha3_flush_q         <= sha3_flush_d;
512     sha3_start_mask_q    <= sha3_start_mask_d;
513     mubi_mod_en_dly_q    <= mubi_mod_en_dly_d;
514     fw_ov_corrupted_q    <= fw_ov_corrupted_d;
515     rng_enable_q         <= rng_enable_d;
516   end
517 end
518
519 assign fw_ov_sha3_disable_pulse = fw_ov_sha3_start_pfe_q &
~fw_ov_sha3_start_pfe;
520
521 //-----
522 // register lock gating
523 //-----
524
525 // Allow writes only if
526 // 1. SW_REGUPD is true,
527 // 2. The DUT is disabled
528 // Block writes if enabled or if internal activities are
still in progress (as indicated by
529 // es_delayed_enable).
530 assign es_hw_regwen = reg2hw.sw_regupd.q &&
531   mu-
bi4_test_false_loose(mubi_module_en_raw_fanout[0]) &&
532   !es_delayed_enable;
533 assign hw2reg.regwen.de = 1'b1;
534 assign hw2reg.regwen.d = es_hw_regwen;
535
536 //-----
537 // set up secure enable bits
538 //-----
539
540 // check for illegal enable field states, and set alert if
detected
541
542 // SEC_CM: CONFIG.MUBI
543 assign mubi_module_en_raw = mu-
bi4_t'(reg2hw.module_enable.q);
544 assign es_enable_pfa = mu-
bi4_test_invalid(mubi_module_en_raw_fanout[1]);
545
546 assign hw2reg.recov_alert_sts.module_enable_field_alert.de =
es_enable_pfa;
547
548 assign hw2reg.recov_alert_sts.module_enable_field_alert.d =
es_enable_pfa;
549
550 prim_mubi4_sync #(
551   .NumCopies(3),
552   .AsyncOn(0)
553 ) u_prim_mubi4_sync_entropy_module_en (
554   .clk_i,
555   .rst_ni,
556   .mubi_i(mubi_module_en_raw),
557   .mubi_o(mubi_module_en_raw_fanout)
558 );
559
560 // Generation of enable pulse, and main enable signals.
561 //
562 // This module creates a single mubi_encoded mod-
ule_enable_pulse, as well as
563 // a general mubi_es_enable signal.
564 //
565 // The module enable pulse is MuBi4True for a single clock
after the
566 // module_enable register is asserted MuBi4True. However in
this first clock cycle
567 // most of the module is not enabled. (The enable pulse is
for clearing the
568 // residual internal state.)
569 //
570 // The rest of the module is enabled in the second clock
cycle after setting the
571 // module_enable register to MuBi4True.
572 //
573 // When module_enable is set to MuBi4False the module is
disabled. No further
574 // RNG input is accepted and the state machines transition
to Idle as soon as
575 // possible. Some delay may occur if the SHA3 conditioning
block is busy, though
576 // otherwise the main_sm transitions to Idle without delay.
577
578 assign mubi_mod_en_dly_d = mubi_module_en_raw_fanout[2];

```

```

577   assign mubi_module_en_pulse = mu-
bi4_and_hi(mubi_mod_en_dly_d, mubi4_t'(~mubi_mod_en_dly_q));
578   assign mubi_es_enable = mubi4_and_hi(mubi_mod_en_dly_d,
mubi_mod_en_dly_q);
579
580   for (genvar i = 0; i < EsEnableCopies; i = i+1) begin :
gen_mubi_en_copies
581     assign es_enable_fo[i] = mu-
bi4_test_true_strict(mubi_es_enable_fanout[i]);
582   end : gen_mubi_en_copies
583
584   prim_mubi4_sync #(
585     .NumCopies(EsEnableCopies),
586     .AsyncOn(0)
587   ) u_prim_mubi4_sync_es_enable (
588     .clk_i,
589     .rst_ni,
590     .mubi_i(mubi_es_enable),
591     .mubi_o(mubi_es_enable_fanout)
592   );
593
594   for (genvar i = 0; i < EsEnPulseCopies; i = i+1) begin :
gen_mubi_en_pulse_copies
595     assign module_en_pulse_fo[i] = mu-
bi4_test_true_strict(mubi_module_en_pulse_fanout[i]);
596   end : gen_mubi_en_pulse_copies
597
598   prim_mubi4_sync #(
599     .NumCopies(EsEnPulseCopies),
600     .AsyncOn(0)
601   ) u_prim_mubi4_sync_es_enable_pulse (
602     .clk_i,
603     .rst_ni,
604     .mubi_i(mubi_module_en_pulse),
605     .mubi_o(mubi_module_en_pulse_fanout)
606   );
607
608   entropy_src_enable_delay u_enable_delay (
609     .clk_i,
610     .rst_ni,
611     .enable_i(es_enable_fo[0]),
612     .esrng_fifo_not_empty_i(sfifo_esrng_not_empty),
613     .esbit_fifo_not_empty_i(pfifo_esbit_not_empty),
614     .postht_fifo_not_empty_i(pfifo_postht_not_empty),
615     .distr_fifo_not_empty_i(sfifo_distr_not_empty),
616     .sha3_block_busy_i(sha3_block_busy),
617     .sha3_block_processed_i(sha3_block_processed),
618     .bypass_mode_i(es_bypass_mode),
619     .enable_o(es_delayed_enable)
620   );
621
622   mubi4_t mubi_fips_en;
623   mubi4_t [1:0] mubi_fips_en_fanout;
624   assign mubi_fips_en = mubi4_t'(reg2hw.conf.fips_enable.q);
625   assign fips_enable_pfe = mu-
bi4_test_true_strict(mubi_fips_en_fanout[0]);
626   assign fips_enable_pfa = mu-
bi4_test_invalid(mubi_fips_en_fanout[1]);
627   assign hw2reg.recov_alert_sts.fips_enable_field_alert.de =
fips_enable_pfa;
628   assign hw2reg.recov_alert_sts.fips_enable_field_alert.d =
fips_enable_pfa;
629
630   prim_mubi4_sync #(
631     .NumCopies(2),
632     .AsyncOn(0)
633   ) u_prim_mubi4_sync_entropy_fips_en (
634     .clk_i,
635     .rst_ni,
636     .mubi_i(mubi_fips_en),
637     .mubi_o(mubi_fips_en_fanout)
638   );
639
640   mubi4_t mubi_fips_flag;
641   mubi4_t [1:0] mubi_fips_flag_fanout;
642   assign mubi_fips_flag = mubi4_t'(reg2hw.conf.fips_flag.q);
643   assign fips_flag_pfa = mu-
bi4_test_invalid(mubi_fips_flag_fanout[1]);
644   assign fips_flag_pfe = mu-
bi4_test_true_strict(mubi_fips_flag_fanout[0]);
645   assign hw2reg.recov_alert_sts.fips_flag_field_alert.de =
fips_flag_pfa;
646   assign hw2reg.recov_alert_sts.fips_flag_field_alert.d =
fips_flag_pfa;
647
648   prim_mubi4_sync #(
649     .NumCopies(2),
650     .AsyncOn(0)
651   ) u_prim_mubi4_sync_entropy_fips_flag (
652     .clk_i,
653     .rst_ni,
654     .mubi_i(mubi_fips_flag),
655     .mubi_o(mubi_fips_flag_fanout)
656   );
657
658   mubi4_t mubi_rng_fips;
659   mubi4_t [1:0] mubi_rng_fips_fanout;
660   assign mubi_rng_fips = mubi4_t'(reg2hw.conf.rng_fips.q);
661   assign rng_fips_pfa = mu-
bi4_test_invalid(mubi_rng_fips_fanout[1]);
662   assign rng_fips_pfe =

```

```

prim_mubi_pkg::mubi4_test_true_loose(mubi_rng_fips_fanout[0]);
663 assign hw2reg.recov_alert_sts.rng_fips_field_alert.de =
rng_fips_pfa;
664 assign hw2reg.recov_alert_sts.rng_fips_field_alert.d =
rng_fips_pfa;
665
666 prim_mubi4_sync #(
667   .NumCopies(2),
668   .AsyncOn(0)
669 ) u_prim_mubi4_sync_entropy_rng_fips (
670   .clk_i,
671   .rst_ni,
672   .mubi_i(mubi_rng_fips),
673   .mubi_o(mubi_rng_fips_fanout)
674 );
675
676 // SEC_CM: CONFIG.MUBI
677 mubi4_t mubi_entropy_reg_en;
678 mubi4_t [1:0] mubi_entropy_reg_en_fanout;
679 assign mubi_entropy_reg_en = mu-
bi4_t'(reg2hw.conf_entropy_data_reg_enable.q);
680 assign entropy_data_reg_en_pfe = mu-
bi4_test_true_strict(mubi_entropy_reg_en_fanout[0]);
681 assign entropy_data_reg_en_pfa = mu-
bi4_test_invalid(mubi_entropy_reg_en_fanout[1]);
682 assign
hw2reg.recov_alert_sts.entropy_data_reg_en_field_alert.de = en-
tropy_data_reg_en_pfa;
683 assign
hw2reg.recov_alert_sts.entropy_data_reg_en_field_alert.d = en-
tropy_data_reg_en_pfa;
684
685 prim_mubi4_sync #(
686   .NumCopies(2),
687   .AsyncOn(0)
688 ) u_prim_mubi4_sync_entropy_data_reg_en (
689   .clk_i,
690   .rst_ni,
691   .mubi_i(mubi_entropy_reg_en),
692   .mubi_o(mubi_entropy_reg_en_fanout)
693 );
694
695 // SEC_CM: CONFIG.MUBI
696 mubi4_t mubi_threshold_oneway;
697 mubi4_t [1:0] mubi_threshold_oneway_fanout;
698 assign mubi_threshold_oneway = mu-
bi4_t'(reg2hw.threshold_oneway.q);
699 assign threshold_oneway_pfe = mu-
bi4_test_true_loose(mubi_threshold_oneway_fanout[0]);
700 assign threshold_oneway_pfa = mu-
bi4_test_invalid(mubi_threshold_oneway_fanout[1]);
701 assign
hw2reg.recov_alert_sts.threshold_oneway_field_alert.de =
threshold_oneway_pfa;
702 assign hw2reg.recov_alert_sts.threshold_oneway_field_alert.d =
threshold_oneway_pfa;
703
704 prim_mubi4_sync #(
705   .NumCopies(2),
706   .AsyncOn(0)
707 ) u_prim_mubi4_sync_treshold_oneway (
708   .clk_i,
709   .rst_ni,
710   .mubi_i(mubi_threshold_oneway),
711   .mubi_o(mubi_threshold_oneway_fanout)
712 );
713
714 assign observe_fifo_thresh = reg2hw.observe_fifo_thresh.q;
715
716 // SEC_CM: CONFIG.MUBI
717 mubi4_t mubi_fw_ov_mode;
718 mubi4_t [1:0] mubi_fw_ov_mode_fanout;
719 assign mubi_fw_ov_mode = mu-
bi4_t'(reg2hw_fw_ov_control_fw_ov_mode.q);
720 assign fw_ov_mode_pfe = mu-
bi4_test_true_strict(mubi_fw_ov_mode_fanout[0]);
721 assign fw_ov_mode_pfa = mu-
bi4_test_invalid(mubi_fw_ov_mode_fanout[1]);
722 assign hw2reg.recov_alert_sts.fw_ov_mode_field_alert.de =
fw_ov_mode_pfa;
723 assign hw2reg.recov_alert_sts.fw_ov_mode_field_alert.d =
fw_ov_mode_pfa;
724
725 prim_mubi4_sync #(
726   .NumCopies(2),
727   .AsyncOn(0)
728 ) u_prim_mubi4_sync_fw_ov_mode (
729   .clk_i,
730   .rst_ni,
731   .mubi_i(mubi_fw_ov_mode),
732   .mubi_o(mubi_fw_ov_mode_fanout)
733 );
734
735 // SEC_CM: CONFIG.MUBI
736 mubi4_t mubi_fw_ov_entropy_insert;
737 mubi4_t [1:0] mubi_fw_ov_entropy_insert_fanout;
738 assign mubi_fw_ov_entropy_insert = mu-
bi4_t'(reg2hw_fw_ov_control_fw_ov_entropy_insert.q);
739 assign fw_ov_entropy_insert_pfe = mu-
bi4_test_true_strict(mubi_fw_ov_entropy_insert_fanout[0]);

```

```

740 assign fw_ov_entropy_insert_pfa = mu-
bi4_test_invalid(mubi_fw_ov_entropy_insert_fanout[1]);
741 assign
hw2reg.recov_alert_sts.fw_ov_entropy_insert_field_alert.de =
fw_ov_entropy_insert_pfa;
742 assign
hw2reg.recov_alert_sts.fw_ov_entropy_insert_field_alert.d =
fw_ov_entropy_insert_pfa;
743
744 prim_mubi4_sync #(
745   .NumCopies(2),
746   .AsyncOn(0)
747 ) u_prim_mubi4_sync_fw_ov_entropy_insert (
748   .clk_i,
749   .rst_ni,
750   .mubi_i(mubi_fw_ov_entropy_insert),
751   .mubi_o(mubi_fw_ov_entropy_insert_fanout)
752 );
753
754 // SEC_CM: CONFIG.MUBI
755 mubi4_t mubi_fw_ov_sha3_start;
756 mubi4_t [1:0] mubi_fw_ov_sha3_start_fanout;
757 assign mubi_fw_ov_sha3_start = mu-
bi4_t'(reg2hw_fw_ov_sha3_start.q);
758 assign fw_ov_sha3_start_pfe = mu-
bi4_test_true_strict(mubi_fw_ov_sha3_start_fanout[0]);
759 assign fw_ov_sha3_start_pfa = mu-
bi4_test_invalid(mubi_fw_ov_sha3_start_fanout[1]);
760 assign
hw2reg.recov_alert_sts.fw_ov_sha3_start_field_alert.de =
fw_ov_sha3_start_pfa;
761 assign
hw2reg.recov_alert_sts.fw_ov_sha3_start_field_alert.d =
fw_ov_sha3_start_pfa;
762
763 prim_mubi4_sync #(
764   .NumCopies(2),
765   .AsyncOn(0)
766 ) u_prim_mubi4_sync_fw_ov_sha3_start (
767   .clk_i,
768   .rst_ni,
769   .mubi_i(mubi_fw_ov_sha3_start),
770   .mubi_o(mubi_fw_ov_sha3_start_fanout)
771 );
772
773 // firmware override controls
774 assign fw_ov_mode = efuse_es_sw_ov_en && fw_ov_mode_pfe;
775 assign fw_ov_mode_entropy_insert = fw_ov_mode && fw_ov_en-
tropy_insert_pfe;
776 assign fw_ov_fifo_rd_pulse = reg2hw_fw_ov_rd_data.re;
777 assign hw2reg_fw_ov_rd_data.d = sfifo_observe_rdata;
778 assign fw_ov_fifo_wr_pulse = reg2hw_fw_ov_wr_data.qe;
779 assign fw_ov_wr_data = reg2hw_fw_ov_wr_data.q;
780
781 assign efuse_es_sw_ov_en =
prim_mubi_pkg::mubi8_test_true_strict(en_entropy_src_fw_over);
782
783 prim_mubi8_sync #(
784   .NumCopies(1),
785   .AsyncOn(1) // must be set to one, see note below
786 ) u_prim_mubi8_sync_es_fw_over (
787   .clk_i,
788   .rst_ni,
789   .mubi_i(otp_en_entropy_src_fw_over_i),
790   .mubi_o({en_entropy_src_fw_over})
791 );
792
793 // note: the input to the above sync module is from the OTP
block.
794 // It is assumed that the source is in a different
time domain,
795 // and requires the AsyncOn parameter to be set.
796
797 // rng_enable is being used in other clock domains. Need to
latch the
798 // signal.
799 assign rng_enable_d = es_enable_fo[1] &&
800   es_delayed_enable;
801
802 assign entropy_src_rng_enable_o = rng_enable_q;
803
804 //-----
805 // instantiate interrupt hardware primitives
806 //-----
807
808 prim_intr_hw #(
809   .Width(1)
810 ) u_intr_hw_es_entropy_valid (
811   .clk_i (clk_i),
812   .rst_ni (rst_ni),
813   .event_intr_i (event_es_entropy_valid),
814   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable.es_entropy_valid.q),
815   .reg2hw_intr_test_q_i
(reg2hw_intr_test.es_entropy_valid.q),
816   .reg2hw_intr_test_qe_i
(reg2hw_intr_test.es_entropy_valid.qe),
817   .reg2hw_intr_state_q_i
(reg2hw_intr_state.es_entropy_valid.q),
818   .hw2reg_intr_state_de_o
(hw2reg_intr_state.es_entropy_valid.de),

```

```

819 .hw2reg_intr_state_d_o
(hw2reg.intr_state.es_entropy_valid.d),
820 .intr_o (intr_es_entropy_valid_o)
821 );
822
823 prim_intr_hw #(
824 .Width(1)
825 ) u_intr_hw_es_health_test_failed (
826 .clk_i (clk_i),
827 .rst_ni (rst_ni),
828 .event_intr_i (event_es_health_test_failed),
829 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.es_health_test_failed.q),
830 .reg2hw_intr_test_q_i
(reg2hw.intr_test.es_health_test_failed.q),
831 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.es_health_test_failed.qe),
832 .reg2hw_intr_state_q_i
(reg2hw.intr_state.es_health_test_failed.q),
833 .hw2reg_intr_state_de_o
(hw2reg.intr_state.es_health_test_failed.de),
834 .hw2reg_intr_state_d_o
(hw2reg.intr_state.es_health_test_failed.d),
835 .intr_o (intr_es_health_test_failed_o)
836 );
837
838
839 prim_intr_hw #(
840 .Width(1)
841 ) u_intr_hw_es_observe_fifo_ready (
842 .clk_i (clk_i),
843 .rst_ni (rst_ni),
844 .event_intr_i (event_es_observe_fifo_ready),
845 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.es_observe_fifo_ready.q),
846 .reg2hw_intr_test_q_i
(reg2hw.intr_test.es_observe_fifo_ready.q),
847 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.es_observe_fifo_ready.qe),
848 .reg2hw_intr_state_q_i
(reg2hw.intr_state.es_observe_fifo_ready.q),
849 .hw2reg_intr_state_de_o
(hw2reg.intr_state.es_observe_fifo_ready.de),
850 .hw2reg_intr_state_d_o
(hw2reg.intr_state.es_observe_fifo_ready.d),
851 .intr_o (intr_es_observe_fifo_ready_o)
852 );
853
854 prim_intr_hw #(
855 .Width(1)
856 ) u_intr_hw_es_fatal_err (
857 .clk_i (clk_i),
858 .rst_ni (rst_ni),
859 .event_intr_i (event_es_fatal_err),
860 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.es_fatal_err.q),
861 .reg2hw_intr_test_q_i (reg2hw.intr_test.es_fatal_err.q),
862 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.es_fatal_err.qe),
863 .reg2hw_intr_state_q_i
(reg2hw.intr_state.es_fatal_err.q),
864 .hw2reg_intr_state_de_o
(hw2reg.intr_state.es_fatal_err.de),
865 .hw2reg_intr_state_d_o
(hw2reg.intr_state.es_fatal_err.d),
866 .intr_o (intr_es_fatal_err_o)
867 );
868
869 //-----
870 // tlul register settings
871 //-----
872
873 // set the interrupt event when enabled
874 assign event_es_entropy_valid =
875 sfifo_esfinal_not_empty && es_route_to_sw &&
876 es_data_reg_rd_en && es_enable_fo[2];
877
878
879 // Collect all internal FIFO errors.
880 assign es_sfifo_int_err = sfifo_esrng_int_err ||
881 sfifo_distr_int_err ||
882 sfifo_observe_int_err ||
883 sfifo_esfinal_int_err;
884
885 // Counter, internal FIFO errors and FSM errors are struc-
886 tural errors and are always active
887 // regardless of the functional state.
888 logic fatal_loc_events;
889 assign fatal_loc_events = es_cntr_err_sum ||
890 es_sfifo_int_err ||
891 es_main_sm_err_sum ||
892 es_ack_sm_err_sum ||
893 sha3_state_error_sum;
894
895 // set the interrupt sources
896 assign event_es_fatal_err = (es_enable_fo[3] &&
897 (sfifo_esrng_err_sum ||
898 sfifo_distr_err_sum ||
899 sfifo_observe_err_sum ||
900 sfifo_esfinal_err_sum ||

```

```

900 sfifo_test_err_sum) ) ||
901 sha3_rst_storage_err_sum ||
902 fatal_loc_events;
903
904 // set fifo errors that are single instances of source
905 assign sfifo_esrng_err_sum = (|sfifo_esrng_err) ||
906 err_code_test_bit[0];
907 assign sfifo_distr_err_sum = (|sfifo_distr_err) ||
908 err_code_test_bit[1];
909 assign sfifo_observe_err_sum = (|sfifo_observe_err) ||
910 err_code_test_bit[2];
911 assign sfifo_esfinal_err_sum = (|sfifo_esfinal_err) ||
912 err_code_test_bit[3];
913
914 // The following test bits help normally diagnose the _type_
915 of
916 // error when they are triggered by the fifo. However when
917 // they are triggered by software they are not linked to a
918 // particular sfifo and do not trigger an alert, unless
919 // we capture them here.
920 assign sfifo_test_err_sum = err_code_test_bit[28] ||
921 err_code_test_bit[29] ||
922 err_code_test_bit[30];
923
924 assign es_ack_sm_err_sum = es_ack_sm_err ||
925 err_code_test_bit[20];
926 assign es_main_sm_err_sum = es_main_sm_err ||
927 err_code_test_bit[21];
928 assign es_cntr_err_sum = es_cntr_err ||
929 err_code_test_bit[22];
930 assign sha3_state_error_sum = sha3_state_error ||
931 err_code_test_bit[23];
932 assign sha3_rst_storage_err_sum = sha3_rst_storage_err ||
933 err_code_test_bit[24];
934 assign fifo_write_err_sum =
935 sfifo_esrng_err[2] ||
936 sfifo_observe_err[2] ||
937 sfifo_distr_err[2] ||
938 sfifo_esfinal_err[2] ||
939 err_code_test_bit[28];
940 assign fifo_read_err_sum =
941 sfifo_esrng_err[1] ||
942 sfifo_observe_err[1] ||
943 sfifo_distr_err[1] ||
944 sfifo_esfinal_err[1] ||
945 err_code_test_bit[29];
946 assign fifo_status_err_sum =
947 sfifo_esrng_err[0] ||
948 sfifo_observe_err[0] ||
949 sfifo_distr_err[0] ||
950 sfifo_esfinal_err[0] ||
951 err_code_test_bit[30];
952
953 // set the err code source bits
954 assign hw2reg.err_code.sfifo_esrng_err.d = 'b1;
955 assign hw2reg.err_code.sfifo_esrng_err.de =
956 sfifo_esrng_err_sum;
957 assign hw2reg.err_code.sfifo_distr_err.d = 'b1;
958 assign hw2reg.err_code.sfifo_distr_err.de =
959 sfifo_distr_err_sum;
960 assign hw2reg.err_code.sfifo_observe_err.d = 'b1;
961 assign hw2reg.err_code.sfifo_observe_err.de =
962 sfifo_observe_err_sum;
963 assign hw2reg.err_code.sfifo_esfinal_err.d = 'b1;
964 assign hw2reg.err_code.sfifo_esfinal_err.de =
965 sfifo_esfinal_err_sum;
966 assign hw2reg.err_code.es_ack_sm_err.d = 'b1;
967 assign hw2reg.err_code.es_ack_sm_err.de = es_ack_sm_err_sum;
968 assign hw2reg.err_code.es_main_sm_err.d = 'b1;
969 assign hw2reg.err_code.es_main_sm_err.de =
970 es_main_sm_err_sum;
971 assign hw2reg.err_code.es_cntr_err.d = 'b1;
972 assign hw2reg.err_code.es_cntr_err.de = es_cntr_err_sum;
973
974 assign hw2reg.err_code.sha3_state_err.d = 'b1;
975 assign hw2reg.err_code.sha3_state_err.de =
976 sha3_state_error_sum;
977 assign hw2reg.err_code.sha3_rst_storage_err.d = 'b1;
978 assign hw2reg.err_code.sha3_rst_storage_err.de =
979 sha3_rst_storage_err_sum;
980
981 // set the err code type bits
982 assign hw2reg.err_code.fifo_write_err.d = 'b1;
983 assign hw2reg.err_code.fifo_write_err.de =
984 fifo_write_err_sum;
985 assign hw2reg.err_code.fifo_read_err.d = 'b1;
986 assign hw2reg.err_code.fifo_read_err.de = fifo_read_err_sum;
987
988 assign hw2reg.err_code.fifo_state_err.d = 'b1;
989 assign hw2reg.err_code.fifo_state_err.de =
990 fifo_status_err_sum;

```

```

990
991 // Error forcing
992 for (genvar i = 0; i < 31; i = i+1) begin :
gen_err_code_test_bit
993   assign err_code_test_bit[i] = (reg2hw.err_code_test.q ==
994 i) && reg2hw.err_code_test.qe;
995   end : gen_err_code_test_bit
996 // alert - send all interrupt sources to the alert for the
fatal case
997   assign fatal_alert_o = (event_es_fatal_err |
998   sfifo_distr_int_err |
sfifo_esrng_int_err |
999   sfifo_observe_int_err |
sfifo_esfinal_int_err);
1000
1001 // alert test
1002 assign recov_alert_test_o = {
1003   reg2hw.alert_test.recov_alert.q &&
1004   reg2hw.alert_test.recov_alert.qe
1005 };
1006 assign fatal_alert_test_o = {
1007   reg2hw.alert_test.fatal_alert.q &&
1008   reg2hw.alert_test.fatal_alert.qe
1009 };
1010
1011 // set the debug status reg
1012 assign hw2reg.debug_status.entropy_fifo_depth.d =
sfifo_esfinal_depth;
1013 assign hw2reg.debug_status.sha3_fsm.d = sha3_fsm;
1014 assign hw2reg.debug_status.sha3_block_pr.d = sha3_b-
lock_processed;
1015 assign hw2reg.debug_status.sha3_squeezing.d = sha3_squeez-
ing;
1016 assign hw2reg.debug_status.sha3_absorbed.d =
1017   prim_mubi_pkg::mubi4_test_true_strict(sha3_absorbed) ? 1'b
1 : 1'b 0;
1018 assign hw2reg.debug_status.sha3_err.d = sha3_err_q;
1019
1020 assign sha3_err_d =
1021   es_enable_fo[4] ? 1'b0 :
1022   {!sha3_err} ? 1'b1 :
1023   sha3_err_q;
1024
1025 // state machine status
1026 assign hw2reg.debug_status.main_sm_idle.d = es_main_sm_idle;
1027 assign hw2reg.debug_status.main_sm_boot_done.d =
boot_phase_done;
1028 assign hw2reg.main_sm_state.de = 1'b1;
1029 assign hw2reg.main_sm_state.d = es_main_sm_state;
1030
1031 // fw override wr data status indication
1032 assign fw_ov_wr_fifo_full = fw_ov_mode_entropy_insert &&
1033   (es_bypass_mode ? !
1034   pfifo_bypass_not_full : !pfifo_precon_not_full);
1035
1036 assign hw2reg.fw_ov_wr_fifo_full.d = fw_ov_wr_fifo_full;
1037
1038 //-----
1039 // receive in RNG bus input
1040 //-----
1041 // SEC_CM: FIFO.CTR.REDUN
1042 prim_fifo_sync #(
1043   .Width(RngBusWidth),
1044   .Pass(0),
1045   .Depth(2),
1046   .OutputZeroIfEmpty(0),
1047   .Secure(1)
1048 ) u_prim_fifo_sync_esrng (
1049   .clk_i      (clk_i),
1050   .rst_ni     (rst_ni),
1051   .clr_i      (sfifo_esrng_clr),
1052   .wvalid_i   (sfifo_esrng_push),
1053   .wdata_i    (sfifo_esrng_wdata),
1054   .wready_o   (sfifo_esrng_not_full),
1055   .rvalid_o   (sfifo_esrng_not_empty),
1056   .rdata_o    (sfifo_esrng_rdata),
1057   .rready_i   (sfifo_esrng_pop),
1058   .full_o     (sfifo_esrng_full),
1059   .depth_o    (),
1060   .err_o      (sfifo_esrng_int_err)
1061 );
1062
1063 // fifo controls
1064 // We can't handle any backpressure at this point. Unless
the ENTROPY_SRC block is turned off,
1065 // the input coming from the noise source / RNG needs to be
accepted without dropping samples.
1066 assign sfifo_esrng_push = es_enable_fo[5] &&
es_delayed_enable && entropy_src_rng_valid_i &&
1067   rng_enable.q;
1068
1069 assign sfifo_esrng_clr = ~es_delayed_enable;
1070 assign sfifo_esrng_wdata = entropy_src_rng_bits_i;
1071 // We can't apply any backpressure at this point. Every
sample is presented to the health tests
1072 // for exactly one clock cycle. If the receiving FIFO is

```

```

full, the sample is dropped but the
1073 // health tests are still performed and the results accumu-
lated.
1074 assign sfifo_esrng_pop = sfifo_esrng_not_empty;
1075
1076 // fifo err
1077 // The esrng FIFO must never be pushed when it's full as no
backpressure can be applied to the
1078 // noise source / RNG. However, we can't raise an error and
abort operation of the ENTROPY_SRC
1079 // block as this would mean a catastrophic failure of the
whole chip. Instead we need to catch
1080 // this in simulation only.
1081 assign sfifo_esrng_err =
1082   {1'b0,
1083   (sfifo_esrng_pop && !sfifo_esrng_not_empty),
1084   (sfifo_esrng_full && !sfifo_esrng_not_empty) ||
sfifo_esrng_int_err};
1085 `ASSERT(RngBackpressureNotAllowed_A, sfifo_esrng_push |->
sfifo_esrng_not_full)
1086
1087 // Read the health test data from the esrng FIFO.
1088 assign health_test_esbus = sfifo_esrng_rdata;
1089 // Perform the health tests whenever data is valid and the
receiving FIFO is not being cleared.
1090 // This doesn't mean the receiving FIFO is pushed. The re-
ceiving FIFO is only pushed if it has
1091 // indeed space. This means in case of heavy backpressure,
we keep testing the noise source
1092
1093 // samples, but we drop them before the postht FIFO. If this
happens, the window counter isn't
1094 // incremented. This way we can keep the number of bits go-
ing into the conditioner constant,
1095 // independent of potential backpressure within the
pipeline.
1096 assign health_test_esbus_vld =
1097   rng_bit_en ? sfifo_esrng_not_empty && !pfifo_esbit_clr :
1098   sfifo_esrng_not_empty && !pfifo_postht_clr;
1099
1100 // Health test any data that comes in on the RNG interface.
1101 assign repcnt_active = 1'b1;
1102 assign repcnts_active = 1'b1;
1103 assign adaptp_active = 1'b1;
1104 assign bucket_active = 1'b1;
1105 assign markov_active = 1'b1;
1106 assign extht_active = 1'b1;
1107
1108 // Only reset health tests on re-enable
1109 assign health_test_clr = module_en_pulse_fo[0];
1110
1111 assign health_test_fips_window =
reg2hw.health_test_windows.fips_window.q;
1112 assign health_test_bypass_window =
reg2hw.health_test_windows.bypass_window.q;
1113
1114 // The bypass window size is given in bits. The FIPS window
is given in samples. If we are using
1115 // the single lane mode, the number of samples is equivalent
to the number of bits. If we are
1116 // using the multi-channel mode, we need to multiply with
the noise source width.
1117 assign health_test_window =
1118   es_bypass_mode ? HealthTestWin-
dowWidth'(health_test_bypass_window) :
1119   rng_bit_en ? HealthTestWin-
dowWidth'(health_test_fips_window) :
1120   HealthTestWin-
dowWidth'(health_test_fips_window * RngBusWidth);
1121
1122 // Window sizes other than 384 bits (the seed length) are
currently not tested nor supported in
1123 // bypass or boot-time mode.
1124 `ASSERT(EsBootTestHtWindowSizeSupported_A,
1125   main_sm_enable && es_bypass_mode && !
fw_ov_mode_entropy_insert
1126   |-> health_test_bypass_window == HalfRegWidth'(SeedLen))
1127
1128 //-----
1129 // repcnt one-way threshold
1130 //-----
1131 assign repcnt_threshold = reg2hw.repcnt_threshold.q;
1132 assign repcnt_threshold_wr = reg2hw.repcnt_threshold.qe;
1133 assign hw2reg.repcnt_threshold.d = repcnt_threshold_oneway;
1134
1135 entropy_src_watermark_reg #(
1136   .RegWidth(HalfRegWidth),
1137   .ResVal({HalfRegWidth{1'b1}})
1138 ) u_entropy_src_watermark_reg_repcnt_thresh (
1139   .clk_i      (clk_i),
1140   .rst_ni     (rst_ni),
1141   .high_i     (1'b0),
1142   .clear_i    (1'b0),
1143   .oneway_i   (threshold_oneway_pfe),
1144   .event_i    (repcnt_threshold_wr),
1145   .value_i    (repcnt_threshold),
1146   .value_o    (repcnt_threshold_oneway)
1147 );
1148
1149 //-----

```



```

1151 // repcnts one-way threshold
1152 //-----
1153 assign repcnts_threshold = reg2hw.repcnts_threshold.q;
1154 assign repcnts_threshold_wr = reg2hw.repcnts_threshold.qe;
1155 assign hw2reg.repcnts_threshold.d = rep-
cnts_threshold_oneway;
1156
1157 entropy_src_watermark_reg #(
1158   .RegWidth(HalfRegWidth),
1159   .ResVal({HalfRegWidth{1'b1}})
1160 ) u_entropy_src_watermark_reg_repcnts_thresh (
1161   .clk_i      (clk_i),
1162   .rst_ni     (rst_ni),
1163   .high_i     (1'b0),
1164   .clear_i    (1'b0),
1165   .oneway_i   (threshold_oneway_pfe),
1166   .event_i    (repcnts_threshold_wr),
1167   .value_i    (repcnts_threshold),
1168   .value_o    (repcnts_threshold_oneway)
1169 );
1170
1171 //-----
1172 // adaptp one-way thresholds
1173 //-----
1174 assign adaptp_hi_threshold = reg2hw.adaptp_hi_threshold.q;
1175 assign adaptp_hi_threshold_wr =
reg2hw.adaptp_hi_threshold.qe;
1176 assign hw2reg.adaptp_hi_threshold.d = ad-
aptp_hi_threshold_oneway;
1177
1178 entropy_src_watermark_reg #(
1179   .RegWidth(HalfRegWidth),
1180   .ResVal({HalfRegWidth{1'b1}})
1181 ) u_entropy_src_watermark_reg_adaptp_hi_thresh (
1182   .clk_i      (clk_i),
1183   .rst_ni     (rst_ni),
1184   .high_i     (1'b0),
1185   .clear_i    (1'b0),
1186   .oneway_i   (threshold_oneway_pfe),
1187   .event_i    (adaptp_hi_threshold_wr),
1188   .value_i    (adaptp_hi_threshold),
1189   .value_o    (adaptp_hi_threshold_oneway)
1190 );
1191
1192 assign adaptp_lo_threshold = reg2hw.adaptp_lo_threshold.q;
1193 assign adaptp_lo_threshold_wr =
reg2hw.adaptp_lo_threshold.qe;
1194 assign hw2reg.adaptp_lo_threshold.d = ad-
aptp_lo_threshold_oneway;
1195
1196 entropy_src_watermark_reg #(
1197   .RegWidth(HalfRegWidth),
1198   .ResVal('0)
1199 ) u_entropy_src_watermark_reg_adaptp_lo_thresh (
1200   .clk_i      (clk_i),
1201   .rst_ni     (rst_ni),
1202   .high_i     (1'b1),
1203   .clear_i    (1'b0),
1204   .oneway_i   (threshold_oneway_pfe),
1205   .event_i    (adaptp_lo_threshold_wr),
1206   .value_i    (adaptp_lo_threshold),
1207   .value_o    (adaptp_lo_threshold_oneway)
1208 );
1209
1210 //-----
1211 // bucket one-way threshold
1212 //-----
1213 assign bucket_threshold = reg2hw.bucket_threshold.q;
1214 assign bucket_threshold_wr = reg2hw.bucket_threshold.qe;
1215 assign hw2reg.bucket_threshold.d = bucket_threshold_oneway;
1216
1217 entropy_src_watermark_reg #(
1218   .RegWidth(HalfRegWidth),
1219   .ResVal({HalfRegWidth{1'b1}})
1220 ) u_entropy_src_watermark_reg_bucket_thresh (
1221   .clk_i      (clk_i),
1222   .rst_ni     (rst_ni),
1223   .high_i     (1'b0),
1224   .clear_i    (1'b0),
1225   .oneway_i   (threshold_oneway_pfe),
1226   .event_i    (bucket_threshold_wr),
1227   .value_i    (bucket_threshold),
1228   .value_o    (bucket_threshold_oneway)
1229 );
1230
1231 //-----
1232 // markov one-way thresholds
1233 //-----
1234 assign markov_hi_threshold = reg2hw.markov_hi_threshold.q;
1235 assign markov_hi_threshold_wr =
reg2hw.markov_hi_threshold.qe;
1236 assign hw2reg.markov_hi_threshold.d =
markov_hi_threshold_oneway;
1237
1238 entropy_src_watermark_reg #(
1239   .RegWidth(HalfRegWidth),
1240   .ResVal({HalfRegWidth{1'b1}})
1241 ) u_entropy_src_watermark_reg_markov_hi_thresh (
1242   .clk_i      (clk_i),
1243   .rst_ni     (rst_ni),
1244   .high_i     (1'b0),
1245   .clear_i    (1'b0),
1246   .oneway_i   (threshold_oneway_pfe),
1247   .event_i    (markov_hi_threshold_wr),
1248   .value_i    (markov_hi_threshold),
1249   .value_o    (markov_hi_threshold_oneway)
1250 );
1251
1252 assign markov_lo_threshold = reg2hw.markov_lo_threshold.q;
1253 assign markov_lo_threshold_wr =
reg2hw.markov_lo_threshold.qe;
1254 assign hw2reg.markov_lo_threshold.d =
markov_lo_threshold_oneway;
1255
1256 entropy_src_watermark_reg #(
1257   .RegWidth(HalfRegWidth),
1258   .ResVal('0)
1259 ) u_entropy_src_watermark_reg_markov_lo_thresh (
1260   .clk_i      (clk_i),
1261   .rst_ni     (rst_ni),
1262   .high_i     (1'b1),
1263   .clear_i    (1'b0),
1264   .oneway_i   (threshold_oneway_pfe),
1265   .event_i    (markov_lo_threshold_wr),
1266   .value_i    (markov_lo_threshold),
1267   .value_o    (markov_lo_threshold_oneway)
1268 );
1269
1270 //-----
1271 // extht one-way thresholds
1272 //-----
1273 assign extht_hi_threshold = reg2hw.extht_hi_threshold.q;
1274 assign extht_hi_threshold_wr = reg2hw.extht_hi_threshold.qe;
1275 assign hw2reg.extht_hi_threshold.d =
extht_hi_threshold_oneway;
1276
1277 entropy_src_watermark_reg #(
1278   .RegWidth(HalfRegWidth),
1279   .ResVal({HalfRegWidth{1'b1}})
1280 ) u_entropy_src_watermark_reg_extht_hi_thresh (
1281   .clk_i      (clk_i),
1282   .rst_ni     (rst_ni),
1283   .high_i     (1'b0),
1284   .clear_i    (1'b0),
1285   .oneway_i   (threshold_oneway_pfe),
1286   .event_i    (extht_hi_threshold_wr),
1287   .value_i    (extht_hi_threshold),
1288   .value_o    (extht_hi_threshold_oneway)
1289 );
1290
1291 assign extht_lo_threshold = reg2hw.extht_lo_threshold.q;
1292 assign extht_lo_threshold_wr = reg2hw.extht_lo_threshold.qe;
1293 assign hw2reg.extht_lo_threshold.d =
extht_lo_threshold_oneway;
1294
1295 entropy_src_watermark_reg #(
1296   .RegWidth(HalfRegWidth),
1297   .ResVal('0)
1298 ) u_entropy_src_watermark_reg_extht_lo_thresh (
1299   .clk_i      (clk_i),
1300   .rst_ni     (rst_ni),
1301   .high_i     (1'b1),
1302   .clear_i    (1'b0),
1303   .oneway_i   (threshold_oneway_pfe),
1304   .event_i    (extht_lo_threshold_wr),
1305   .value_i    (extht_lo_threshold),
1306   .value_o    (extht_lo_threshold_oneway)
1307 );
1308
1309 //-----
1310 // misc control settings
1311 //-----
1312
1313 assign event_es_health_test_failed = es_main_sm_alert;
1314
1315 assign event_es_observe_fifo_ready = ob-
serve_fifo_thresh_met;
1316
1317 // SEC_CM: CONFIG.MUBI
1318 mubi4_t mubi_es_route;
1319 mubi4_t [1:0] mubi_es_route_fanout;
1320 assign mubi_es_route = mu-
bi4_t'(reg2hw.entropy_control.es_route.q);
1321 assign es_route_pfe = mu-
bi4_test_true_strict(mubi_es_route_fanout[0]);
1322 assign es_route_pfa = mu-
bi4_test_invalid(mubi_es_route_fanout[1]);
1323 assign hw2reg.recov_alert_sts.es_route_field_alert.de =
es_route_pfa;
1324 assign hw2reg.recov_alert_sts.es_route_field_alert.d =
es_route_pfa;
1325
1326 prim_mubi4_sync #(
1327   .NumCopies(2),
1328   .AsyncOn(0)
1329 ) u_prim_mubi4_sync_es_route (
1330   .clk_i,
1331   .rst_ni,
1332   .mubi_i(mubi_es_route),
1333   .mubi_o(mubi_es_route_fanout)
1334 );
1335

```

```

1336 // SEC_CM: CONFIG.MUBI
1337 mubi4_t mubi_es_type;
1338 mubi4_t [1:0] mubi_es_type_fanout;
1339 assign mubi_es_type = mu-
bi4_t'(reg2hw.entropy_control.es_type.q);
1340 assign es_type_pfe = mu-
bi4_test_true_strict(mubi_es_type_fanout[0]);
1341 assign es_type_pfa = mu-
bi4_test_invalid(mubi_es_type_fanout[1]);
1342 assign hw2reg.recov_alert_sts.es_type_field_alert.de =
es_type_pfa;
1343 assign hw2reg.recov_alert_sts.es_type_field_alert.d =
es_type_pfa;
1344
1345 prim_mubi4_sync #(
1346     .NumCopies(2),
1347     .AsyncOn(0)
1348 ) u_prim_mubi4_sync_es_type (
1349     .clk_i,
1350     .rst_ni,
1351     .mubi_i(mubi_es_type),
1352     .mubi_o(mubi_es_type_fanout)
1353 );
1354
1355 // SEC_CM: CONFIG.MUBI
1356 mubi4_t mubi_thresh_scope;
1357 assign mubi_thresh_scope = mu-
bi4_t'(reg2hw.conf.threshold_scope.q);
1358 assign threshold_scope_pfe = mu-
bi4_test_true_strict(mubi_thresh_scope);
1359 assign threshold_scope_pfa = mu-
bi4_test_invalid(mubi_thresh_scope);

1360 assign hw2reg.recov_alert_sts.threshold_scope_field_alert.de =
threshold_scope_pfa;

1361 assign hw2reg.recov_alert_sts.threshold_scope_field_alert.d =
threshold_scope_pfa;
1362
1363 assign es_route_to_sw = es_route_pfe;
1364 assign es_bypass_to_sw = es_type_pfe;
1365 assign threshold_scope = threshold_scope_pfe;
1366
1367 // The es_bypass_mode signal determines whether the condi-
tioner is bypassed or not.
1368 // The conditioner can be bypassed by either disabling
fips_enable_pfe or by enabling
1369 // both es_bypass_to_sw and es_route_to_sw.
1370 // The combination of es_bypass_to_sw and es_route_to_sw al-
lows for four distinct cases:
1371 //
1372 // | es_bypass_to_sw | es_route_to_sw |
1373 // |-----|-----|
1374 // | 0 | 0 | In this case the en-
1375 // | 0 | 0 | tropy passes through the conditioner
1376 // | 0 | 0 | and the entropy is
1377 // | 0 | 0 | forwarded to the HW endpoints.
1378 // | 0 | 1 | In this case the en-
1379 // | 0 | 1 | tropy passes through the conditioner
1380 // | 0 | 1 | and the entropy is
1381 // | 0 | 1 | forwarded to software.
1382 // | 1 | 0 | In this case nothing
1383 // | 1 | 0 | happens and whether the conditioner
1384 // | 1 | 0 | is bypassed solely
1385 // | 1 | 0 | depends on fips_enable_pfe.
1386 // | 1 | 1 | In this case the
1387 // | 1 | 1 | conditioner is bypassed and the entropy
1388 // | 1 | 1 | is forwarded to
1389 // | 1 | 1 | software.
1390 // | 1 | 1 |
1391 // | 1 | 1 |
1392 // | 1 | 1 |
1393 // | 1 | 1 |
1394 // | 1 | 1 |
1395 // | 1 | 1 |
1396 // | 1 | 1 |
1397 // | 1 | 1 |
1398 // | 1 | 1 |
1399 // | 1 | 1 |
1400 // | 1 | 1 |
1401 // | 1 | 1 |
1402 // | 1 | 1 |
1403 // | 1 | 1 |
1404 // | 1 | 1 |
1405 // | 1 | 1 |
1406 // | 1 | 1 |
1407 // | 1 | 1 |
1408 // | 1 | 1 |
1409 // | 1 | 1 |
1410 // | 1 | 1 |
1411 // | 1 | 1 |
1412 // | 1 | 1 |
1413 // | 1 | 1 |
1414 // | 1 | 1 |
1415 // | 1 | 1 |
1416 // | 1 | 1 |
1417 // | 1 | 1 |
1418 // | 1 | 1 |
1419 // | 1 | 1 |
1420 // | 1 | 1 |
1421 // | 1 | 1 |
1422 // | 1 | 1 |
1423 // | 1 | 1 |
1424 // | 1 | 1 |
1425 // | 1 | 1 |
1426 // | 1 | 1 |
1427 // | 1 | 1 |
1428 // | 1 | 1 |
1429 // | 1 | 1 |
1430 // | 1 | 1 |
1431 // | 1 | 1 |
1432 // | 1 | 1 |
1433 // | 1 | 1 |
1434 // | 1 | 1 |
1435 // | 1 | 1 |
1436 // | 1 | 1 |
1437 // | 1 | 1 |
1438 // | 1 | 1 |
1439 // | 1 | 1 |
1440 // | 1 | 1 |
1441 // | 1 | 1 |
1442 // | 1 | 1 |
1443 // | 1 | 1 |
1444 // | 1 | 1 |
1445 // | 1 | 1 |
1446 // | 1 | 1 |
1447 // | 1 | 1 |
1448 // | 1 | 1 |
1449 // | 1 | 1 |
1450 // | 1 | 1 |
1451 // | 1 | 1 |
1452 // | 1 | 1 |
1453 // | 1 | 1 |
1454 // | 1 | 1 |
1455 // | 1 | 1 |
1456 // | 1 | 1 |
1457 // | 1 | 1 |
1458 // | 1 | 1 |
1459 // | 1 | 1 |
1460 // | 1 | 1 |
1461 // | 1 | 1 |
1462 // | 1 | 1 |
1463 // | 1 | 1 |
1464 // | 1 | 1 |
1465 // | 1 | 1 |
1466 // | 1 | 1 |
1467 // | 1 | 1 |
1468 // | 1 | 1 |
1469 // | 1 | 1 |
1470 // | 1 | 1 |
1471 // | 1 | 1 |
1472 // | 1 | 1 |
1473 // | 1 | 1 |
1474 // | 1 | 1 |
1475 // | 1 | 1 |
1476 // | 1 | 1 |
1477 // | 1 | 1 |
1478 // | 1 | 1 |
1479 // | 1 | 1 |
1480 // | 1 | 1 |
1481 // | 1 | 1 |
1482 // | 1 | 1 |
1483 // | 1 | 1 |
1484 // | 1 | 1 |
1485 // | 1 | 1 |
1486 // | 1 | 1 |
1487 // | 1 | 1 |
1488 // | 1 | 1 |
1489 // | 1 | 1 |
1490 // | 1 | 1 |
1491 // | 1 | 1 |
1492 // | 1 | 1 |
1493 // | 1 | 1 |
1494 // | 1 | 1 |
1495 // | 1 | 1 |
1496 // | 1 | 1 |
1497 // | 1 | 1 |
1498 // | 1 | 1 |
1499 // | 1 | 1 |
1500 // | 1 | 1 |
1501 // | 1 | 1 |
1502 // | 1 | 1 |
1503 // | 1 | 1 |
1504 // | 1 | 1 |
1505 // | 1 | 1 |
1506 // | 1 | 1 |
1507 // | 1 | 1 |
1508 // | 1 | 1 |
1509 // | 1 | 1 |
1510 // | 1 | 1 |
1511 // | 1 | 1 |
1512 // | 1 | 1 |
1513 // | 1 | 1 |
1514 // | 1 | 1 |
1515 // | 1 | 1 |
1516 // | 1 | 1 |
1517 // | 1 | 1 |
1518 // | 1 | 1 |
1519 // | 1 | 1 |
1520 // | 1 | 1 |
1521 // | 1 | 1 |
1522 // | 1 | 1 |
1523 // | 1 | 1 |
1524 // | 1 | 1 |
1525 // | 1 | 1 |
1526 // | 1 | 1 |
1527 // | 1 | 1 |
1528 // | 1 | 1 |
1529 // | 1 | 1 |
1530 // | 1 | 1 |
1531 // | 1 | 1 |
1532 // | 1 | 1 |
1533 // | 1 | 1 |
1534 // | 1 | 1 |
1535 // | 1 | 1 |
1536 // | 1 | 1 |
1537 // | 1 | 1 |
1538 // | 1 | 1 |
1539 // | 1 | 1 |
1540 // | 1 | 1 |
1541 // | 1 | 1 |
1542 // | 1 | 1 |
1543 // | 1 | 1 |
1544 // | 1 | 1 |
1545 // | 1 | 1 |
1546 // | 1 | 1 |
1547 // | 1 | 1 |
1548 // | 1 | 1 |
1549 // | 1 | 1 |
1550 // | 1 | 1 |
1551 // | 1 | 1 |
1552 // | 1 | 1 |
1553 // | 1 | 1 |
1554 // | 1 | 1 |
1555 // | 1 | 1 |
1556 // | 1 | 1 |
1557 // | 1 | 1 |
1558 // | 1 | 1 |
1559 // | 1 | 1 |
1560 // | 1 | 1 |
1561 // | 1 | 1 |
1562 // | 1 | 1 |
1563 // | 1 | 1 |
1564 // | 1 | 1 |
1565 // | 1 | 1 |
1566 // | 1 | 1 |
1567 // | 1 | 1 |
1568 // | 1 | 1 |
1569 // | 1 | 1 |
1570 // | 1 | 1 |
1571 // | 1 | 1 |
1572 // | 1 | 1 |
1573 // | 1 | 1 |
1574 // | 1 | 1 |
1575 // | 1 | 1 |
1576 // | 1 | 1 |
1577 // | 1 | 1 |
1578 // | 1 | 1 |
1579 // | 1 | 1 |
1580 // | 1 | 1 |
1581 // | 1 | 1 |
1582 // | 1 | 1 |
1583 // | 1 | 1 |
1584 // | 1 | 1 |
1585 // | 1 | 1 |
1586 // | 1 | 1 |
1587 // | 1 | 1 |
1588 // | 1 | 1 |
1589 // | 1 | 1 |
1590 // | 1 | 1 |
1591 // | 1 | 1 |
1592 // | 1 | 1 |
1593 // | 1 | 1 |
1594 // | 1 | 1 |
1595 // | 1 | 1 |
1596 // | 1 | 1 |
1597 // | 1 | 1 |
1598 // | 1 | 1 |
1599 // | 1 | 1 |
1600 // | 1 | 1 |
1601 // | 1 | 1 |
1602 // | 1 | 1 |
1603 // | 1 | 1 |
1604 // | 1 | 1 |
1605 // | 1 | 1 |
1606 // | 1 | 1 |
1607 // | 1 | 1 |
1608 // | 1 | 1 |
1609 // | 1 | 1 |
1610 // | 1 | 1 |
1611 // | 1 | 1 |
1612 // | 1 | 1 |
1613 // | 1 | 1 |
1614 // | 1 | 1 |
1615 // | 1 | 1 |
1616 // | 1 | 1 |
1617 // | 1 | 1 |
1618 // | 1 | 1 |
1619 // | 1 | 1 |
1620 // | 1 | 1 |
1621 // | 1 | 1 |
1622 // | 1 | 1 |
1623 // | 1 | 1 |
1624 // | 1 | 1 |
1625 // | 1 | 1 |
1626 // | 1 | 1 |
1627 // | 1 | 1 |
1628 // | 1 | 1 |
1629 // | 1 | 1 |
1630 // | 1 | 1 |
1631 // | 1 | 1 |
1632 // | 1 | 1 |
1633 // | 1 | 1 |
1634 // | 1 | 1 |
1635 // | 1 | 1 |
1636 // | 1 | 1 |
1637 // | 1 | 1 |
1638 // | 1 | 1 |
1639 // | 1 | 1 |
1640 // | 1 | 1 |
1641 // | 1 | 1 |
1642 // | 1 | 1 |
1643 // | 1 | 1 |
1644 // | 1 | 1 |
1645 // | 1 | 1 |
1646 // | 1 | 1 |
1647 // | 1 | 1 |
1648 // | 1 | 1 |
1649 // | 1 | 1 |
1650 // | 1 | 1 |
1651 // | 1 | 1 |
1652 // | 1 | 1 |
1653 // | 1 | 1 |
1654 // | 1 | 1 |
1655 // | 1 | 1 |
1656 // | 1 | 1 |
1657 // | 1 | 1 |
1658 // | 1 | 1 |
1659 // | 1 | 1 |
1660 // | 1 | 1 |
1661 // | 1 | 1 |
1662 // | 1 | 1 |
1663 // | 1 | 1 |
1664 // | 1 | 1 |
1665 // | 1 | 1 |
1666 // | 1 | 1 |
1667 // | 1 | 1 |
1668 // | 1 | 1 |
1669 // | 1 | 1 |
1670 // | 1 | 1 |
1671 // | 1 | 1 |
1672 // | 1 | 1 |
1673 // | 1 | 1 |
1674 // | 1 | 1 |
1675 // | 1 | 1 |
1676 // | 1 | 1 |
1677 // | 1 | 1 |
1678 // | 1 | 1 |
1679 // | 1 | 1 |
1680 // | 1 | 1 |
1681 // | 1 | 1 |
1682 // | 1 | 1 |
1683 // | 1 | 1 |
1684 // | 1 | 1 |
1685 // | 1 | 1 |
1686 // | 1 | 1 |
1687 // | 1 | 1 |
1688 // | 1 | 1 |
1689 // | 1 | 1 |
1690 // | 1 | 1 |
1691 // | 1 | 1 |
1692 // | 1 | 1 |
1693 // | 1 | 1 |
1694 // | 1 | 1 |
1695 // | 1 | 1 |
1696 // | 1 | 1 |
1697 // | 1 | 1 |
1698 // | 1 | 1 |
1699 // | 1 | 1 |
1700 // | 1 | 1 |
1701 // | 1 | 1 |
1702 // | 1 | 1 |
1703 // | 1 | 1 |
1704 // | 1 | 1 |
1705 // | 1 | 1 |
1706 // | 1 | 1 |
1707 // | 1 | 1 |
1708 // | 1 | 1 |
1709 // | 1 | 1 |
1710 // | 1 | 1 |
1711 // | 1 | 1 |
1712 // | 1 | 1 |
1713 // | 1 | 1 |
1714 // | 1 | 1 |
1715 // | 1 | 1 |
1716 // | 1 | 1 |
1717 // | 1 | 1 |
1718 // | 1 | 1 |
1719 // | 1 | 1 |
1720 // | 1 | 1 |
1721 // | 1 | 1 |
1722 // | 1 | 1 |
1723 // | 1 | 1 |
1724 // | 1 | 1 |
1725 // | 1 | 1 |
1726 // | 1 | 1 |
1727 // | 1 | 1 |
1728 // | 1 | 1 |
1729 // | 1 | 1 |
1730 // | 1 | 1 |
1731 // | 1 | 1 |
1732 // | 1 | 1 |
1733 // | 1 | 1 |
1734 // | 1 | 1 |
1735 // | 1 | 1 |
1736 // | 1 | 1 |
1737 // | 1 | 1 |
1738 // | 1 | 1 |
1739 // | 1 | 1 |
1740 // | 1 | 1 |
1741 // | 1 | 1 |
1742 // | 1 | 1 |
1743 // | 1 | 1 |
1744 // | 1 | 1 |
1745 // | 1 | 1 |
1746 // | 1 | 1 |
1747 // | 1 | 1 |
1748 // | 1 | 1 |
1749 // | 1 | 1 |
1750 // | 1 | 1 |
1751 // | 1 | 1 |
1752 // | 1 | 1 |
1753 // | 1 | 1 |
1754 // | 1 | 1 |
1755 // | 1 | 1 |
1756 // | 1 | 1 |
1757 // | 1 | 1 |
1758 // | 1 | 1 |
1759 // | 1 | 1 |
1760 // | 1 | 1 |
1761 // | 1 | 1 |
1762 // | 1 | 1 |
1763 // | 1 | 1 |
1764 // | 1 | 1 |
1765 // | 1 | 1 |
1766 // | 1 | 1 |
1767 // | 1 | 1 |
1768 // | 1 | 1 |
1769 // | 1 | 1 |
1770 // | 1 | 1 |
1771 // | 1 | 1 |
1772 // | 1 | 1 |
1773 // | 1 | 1 |
1774 // | 1 | 1 |
1775 // | 1 | 1 |
1776 // | 1 | 1 |
1777 // | 1 | 1 |
1778 // | 1 | 1 |
1779 // | 1 | 1 |
1780 // | 1 | 1 |
1781 // | 1 | 1 |
1782 // | 1 | 1 |
1783 // | 1 | 1 |
1784 // | 1 | 1 |
1785 // | 1 | 1 |
1786 // | 1 | 1 |
1787 // | 1 | 1 |
1788 // | 1 | 1 |
1789 // | 1 | 1 |
1790 // | 1 | 1 |
1791 // | 1 | 1 |
1792 // | 1 | 1 |
1793 // | 1 | 1 |
1794 // | 1 | 1 |
1795 // | 1 | 1 |
1796 // | 1 | 1 |
1797 // | 1 | 1 |
1798 // | 1 | 1 |
1799 // | 1 | 1 |
1800 // | 1 | 1 |
1801 // | 1 | 1 |
1802 // | 1 | 1 |
1803 // | 1 | 1 |
1804 // | 1 | 1 |
1805 // | 1 | 1 |
1806 // | 1 | 1 |
1807 // | 1 | 1 |
1808 // | 1 | 1 |
1809 // | 1 | 1 |
1810 // | 1 | 1 |
1811 // | 1 | 1 |
1812 // | 1 | 1 |
1813 // | 1 | 1 |
1814 // | 1 | 1 |
1815 // | 1 | 1 |
1816 // | 1 | 1 |
1817 // | 1 | 1 |
1818 // | 1 | 1 |
1819 // | 1 | 1 |
1820 // | 1 | 1 |
1821 // | 1 | 1 |
1822 // | 1 | 1 |
1823 // | 1 | 1 |
1824 // | 1 | 1 |
1825 // | 1 | 1 |
1826 // | 1 | 1 |
1827 // | 1 | 1 |
1828 // | 1 | 1 |
1829 // | 1 | 1 |
1830 // | 1 | 1 |
1831 // | 1 | 1 |
1832 // | 1 | 1 |
1833 // | 1 | 1 |
1834 // | 1 | 1 |
1835 // | 1 | 1 |
1836 // | 1 | 1 |
1837 // | 1 | 1 |
1838 // | 1 | 1 |
1839 // | 1 | 1 |
1840 // | 1 | 1 |
1841 // | 1 | 1 |
1842 // | 1 | 1 |
1843 // | 1 | 1 |
1844 // | 1 | 1 |
1845 // | 1 | 1 |
1846 // | 1 | 1 |
1847 // | 1 | 1 |
1848 // | 1 | 1 |
1849 // | 1 | 1 |
1850 // | 1 | 1 |
1851 // | 1 | 1 |
1852 // | 1 | 1 |
1853 // | 1 | 1 |
1854 // | 1 | 1 |
1855 // | 1 | 1 |
1856 // | 1 | 1 |
1857 // | 1 | 1 |
1858 // | 1 | 1 |
1859 // | 1 | 1 |
1860 // | 1 | 1 |
1861 // | 1 | 1 |
1862 // | 1 | 1 |
1863 // | 1 | 1 |
1864 // | 1 | 1 |
1865 // | 1 | 1 |
1866 // | 1 | 1 |
1867 // | 1 | 1 |
1868 // | 1 | 1 |
1869 // | 1 | 1 |
1870 // | 1 | 1 |
1871 // | 1 | 1 |
1872 // | 1 | 1 |
1873 // | 1 | 1 |
1874 // | 1 | 1 |
1875 // | 1 | 1 |
1876 // | 1 | 1 |
1877 // | 1 | 1 |
1878 // | 1 | 1 |
1879 // | 1 | 1 |
1880 // | 1 | 1 |
1881 // | 1 | 1 |
1882 // | 1 | 1 |
1883 // | 1 | 1 |
1884 // | 1 | 1 |
1885 // | 1 | 1 |
1886 // | 1 | 1 |
1887 // | 1 | 1 |
1888 // | 1 | 1 |
1889 // | 1 | 1 |
1890 // | 1 | 1 |
1891 // | 1 | 1 |
1892 // | 1 | 1 |
1893 // | 1 | 1 |
1894 // | 1 | 1 |
1895 // | 1 | 1 |
1896 // | 1 | 1 |
1897 // | 1 | 1 |
1898 // | 1 | 1 |
1899 // | 1 | 1 |
1900 // | 1 | 1 |
1901 // | 1 | 1 |
1902 // | 1 | 1 |
1903 // | 1 | 1 |
1904 // | 1 | 1 |
1905 // | 1 | 1 |
1906 // | 1 | 1 |
1907 // | 1 | 1 |
1908 // | 1 | 1 |
1909 // | 1 | 1 |
1910 // | 1 | 1 |
1911 // | 1 | 1 |
1912 // | 1 | 1 |
1913 // | 1 | 1 |
1914 // | 1 | 1 |
1915 // | 1 | 1 |
1916 // | 1 | 1 |
1917 // | 1 | 1 |
1918 // | 1 | 1 |
1919 // | 1 | 1 |
1920 // | 1 | 1 |
1921 // | 1 | 1 |
1922 // | 1 | 1 |
1923 // | 1 | 1 |
1924 // | 1 | 1 |
1925 // | 1 | 1 |
1926 // | 1 | 1 |
1927 // | 1 | 1 |
1928 // | 1 | 1 |
1929 // | 1 | 1 |
1930 // | 1 | 1 |
1931 // | 1 | 1 |
1932 // | 1 | 1 |
1933 // | 1 | 1 |
1934 // | 1 | 1 |
1935 // | 1 | 1 |
1936 // | 1 | 1 |
1937 // | 1 | 1 |
1938 // | 1 | 1 |
1939 // | 1 | 1 |
1940 // | 1 | 1 |
1941 // | 1 | 1 |
1942 // | 1 | 1 |
1943 // | 1 | 1 |
1944 // | 1 | 1 |
1945 // | 1 | 1 |
1946 // | 1 | 1 |
1947 // | 1 | 1 |
1948 // | 1 | 1 |
1949 // | 1 | 1 |
1950 // | 1 | 1 |
1951 // | 1 | 1 |
1952 // | 1 | 1 |
1953 // | 1 | 1 |
1954 // | 1 | 1 |
1955 // | 1 | 1 |
1956 // | 1 | 1 |
1957 // | 1 | 1 |
1958 // | 1 | 1 |
1959 // | 1 | 1 |
1960 // | 1 | 1 |
1961 // | 1 | 1 |
1962 // | 1 | 1 |
1963 // | 1 | 1 |
1964 // | 1 | 1 |
1965 // | 1 | 1 |
1966 // | 1 | 1 |
1967 // | 1 | 1 |
1968 // | 1 | 1 |
1969 // | 1 | 1 |
1970 // | 1 | 1 |
1971 // | 1 | 1 |
1972 // | 1 | 1 |
1973 // | 1 | 1 |
1974 // | 1 | 1 |
1975 // | 1 | 1 |
1976 // | 1 | 1 |
1977 // | 1 | 1 |
1978 // | 1 | 1 |
1979 // | 1 | 1 |
1980 // | 1 | 1 |
1981 // | 1 | 1 |
1982 // | 1 | 1 |
1983 // | 1 | 1 |
1984 // | 1 | 1 |
1985 // | 1 | 1 |
1986 // | 1 | 1 |
1987 // | 1 | 1 |
1988 // | 1 | 1 |
1989 // | 1 | 1 |
1990 // | 1 | 1 |
1991 // | 1 | 1 |
1992 // | 1 | 1 |
1993 // | 1 | 1 |
1994 // | 1 | 1 |
1995 // | 1 | 1 |
1996 // | 1 | 1 |
1997 // | 1 | 1 |
1998 // | 1 | 1 |
1999 // | 1 | 1 |
2000 // | 1 | 1 |

```

```

1405 assign window_cntr_incr_en =
1406     rng_bit_en ? pfifo_esbit_push && pfifo_esbit_not_full
&& !pfifo_esbit_clr :
1407     pfifo_postht_push && pfifo_postht_not_full
&& !pfifo_postht_clr;
1408
1409 // When in single lane mode, only increment with 1-bit, oth-
erwise use the full noise source width
1410 assign window_cntr_step = unsigned'(rng_bit_en ? HealthT-
estWindowWidth'(1) :
1411     HealthT-
estWindowWidth'(RngBusWidth));
1412
1413 prim_count #(
1414     .Width(HealthTestWindowWidth)
1415 ) u_prim_count_window_cntr (
1416     .clk_i,
1417     .rst_ni,
1418     .clr_i(!es_delayed_enable),
1419     .set_i(health_test_done_pulse),
1420     .set_cnt_i(HealthTestWindowWidth'(0)),
1421     .incr_en_i(window_cntr_incr_en),
1422     .decr_en_i(1'b0),
1423     .step_i(window_cntr_step),
1424     .commit_i(1'b1),
1425     .cnt_o(window_cntr),
1426     .cnt_after_commit_o(),
1427     .err_o(window_cntr_err)
1428 );
1429
1430 // Window wrap condition
1431 // The counter is incremented with every tested symbol and
when we've tested sufficiently many
1432 // symbols, we decide whether the current window passed or
failed the tests. Using the
1433 // health_test_done_pulse, we then restart the counter, we
update the watermark registers, and
1434 // we clear the window-based health tests to get ready for
the next window. This means, we can't
1435 // actually test another symbol in this cycle, i.e., the
maximum rate of the noise source is
1436 // limited to one symbol every two clock cycles.
1437 assign health_test_done_pulse = (window_cntr >=
health_test_window);
1438
1439 // Summary of counter errors
1440 assign es_cntr_err =
1441     (window_cntr_err ||
1442     repcnt_cntr_err ||
1443     repcnts_cntr_err ||
1444     adaptp_cntr_err ||
1445     (|bucket_cntr_err) ||
1446     markov_cntr_err ||
1447     repcnt_fails_cntr_err ||
1448     repcnt_alert_cntr_err ||
1449     repcnts_fails_cntr_err ||
1450     repcnts_alert_cntr_err ||
1451     adaptp_hi_fails_cntr_err ||
1452     adaptp_lo_fails_cntr_err ||
1453     adaptp_hi_alert_cntr_err ||
1454     adaptp_lo_alert_cntr_err ||
1455     bucket_fails_cntr_err ||
1456     bucket_alert_cntr_err ||
1457     markov_hi_fails_cntr_err ||
1458     markov_lo_fails_cntr_err ||
1459     markov_hi_alert_cntr_err ||
1460     markov_lo_alert_cntr_err ||
1461     extht_hi_fails_cntr_err ||
1462     extht_lo_fails_cntr_err ||
1463     extht_hi_alert_cntr_err ||
1464     extht_lo_alert_cntr_err ||
1465     any_fails_cntr_err ||
1466     pipeline_depth_cntr_err ||
1467     sha3_count_error);
1468
1469 //-----
1470 // repetitive count test
1471 //-----
1472
1473 // SEC_CM: RNG.BKGN_CHK
1474 entropy_src_repcnt_ht #(
1475     .RegWidth(HalfRegWidth),
1476     .RngBusBitSelWidth(RngBusBitSelWidth),
1477     .RngBusWidth(RngBusWidth)
1478 ) u_entropy_src_repcnt_ht (
1479     .clk_i,
1480     .rst_ni,
1481     .entropy_bit_i (health_test_esbus),
1482     .entropy_bit_vld_i (health_test_esbus_vld),
1483     .rng_bit_en_i (rng_bit_en),
1484     .rng_bit_sel_i (rng_bit_sel),
1485     .clear_i (health_test_clr),
1486     .active_i (repcnt_active),
1487     .thresh_i (repcnt_threshold_oneway),
1488     .test_cnt_o (repcnt_event_cnt),
1489     .test_fail_pulse_o (repcnt_fail_pulse),
1490     .count_err_o (repcnt_cntr_err)
1491 );
1492
1493 // SEC_CM: CTR.REDUN
1494 entropy_src_cntr_reg #(

```



```

1495 .RegWidth(FullRegWidth)
1496 ) u_entropy_src_ctr_reg_repnt (
1497   .clk_i      (clk_i),
1498   .rst_ni     (rst_ni),
1499   .clear_i    (health_test_clr),
1500   .event_i    (repnt_fail_pulse),
1501   .step_i     (FullRegWidth'(1)),
1502   .value_o    (repnt_total_fails),
1503   .err_o      (repnt_fails_ctr_err)
1504 );
1505
1506 assign hw2reg.repnt_total_fails.d = repnt_total_fails;
1507
1508 //-----
1509 // repetitive count symbol test
1510 //-----
1511
1512 // SEC_CM: RNG.BKGN_CHK
1513 entropy_src_repnts_ht #(
1514   .RegWidth(HalfRegWidth),
1515   .RngBusWidth(RngBusWidth)
1516 ) u_entropy_src_repnts_ht (
1517   .clk_i      (clk_i),
1518   .rst_ni     (rst_ni),
1519   .entropy_bit_i (health_test_esbus),
1520   .entropy_bit_vld_i (health_test_esbus_vld),
1521   .clear_i    (health_test_clr),
1522   .active_i   (repnts_active),
1523   .thresh_i   (repnts_threshold_oneway),
1524   .test_cnt_o (repnts_event_cnt),
1525   .test_fail_pulse_o (repnts_fail_pulse),
1526   .count_err_o (repnts_ctr_err)
1527 );
1528
1529 // SEC_CM: CTR.REDUN
1530 entropy_src_ctr_reg #(
1531   .RegWidth(FullRegWidth)
1532 ) u_entropy_src_ctr_reg_repnts (
1533   .clk_i      (clk_i),
1534   .rst_ni     (rst_ni),
1535   .clear_i    (health_test_clr),
1536   .event_i    (repnts_fail_pulse),
1537   .step_i     (FullRegWidth'(1)),
1538   .value_o    (repnts_total_fails),
1539   .err_o      (repnts_fails_ctr_err)
1540 );
1541
1542 assign hw2reg.repnts_total_fails.d = repnts_total_fails;
1543
1544 //-----
1545 // Adaptive Proportion Test
1546 //-----
1547
1548 // SEC_CM: RNG.BKGN_CHK
1549 entropy_src_adaptpt_ht #(
1550   .RegWidth(HalfRegWidth),
1551   .RngBusWidth(RngBusWidth),
1552   .RngBusBitSelWidth(RngBusBitSelWidth)
1553 ) u_entropy_src_adaptpt_ht (
1554   .clk_i      (clk_i),
1555   .rst_ni     (rst_ni),
1556   .entropy_bit_i (health_test_esbus),
1557   .entropy_bit_vld_i (health_test_esbus_vld),
1558   .rng_bit_en_i (rng_bit_en),
1559   .rng_bit_sel_i (health_test_sel),
1560   .clear_i      (health_test_clr),
1561   .active_i     (adaptpt_active),
1562   .thresh_hi_i  (adaptpt_hi_threshold_oneway),
1563   .thresh_lo_i  (adaptpt_lo_threshold_oneway),
1564   .window_wrap_pulse_i (health_test_done_pulse),
1565   .threshold_scope_i (threshold_scope),
1566   .test_cnt_hi_o (adaptpt_hi_event_cnt),
1567   .test_cnt_lo_o (adaptpt_lo_event_cnt),
1568   .test_fail_hi_pulse_o (adaptpt_hi_fail_pulse),
1569   .test_fail_lo_pulse_o (adaptpt_lo_fail_pulse),
1570   .count_err_o   (adaptpt_ctr_err)
1571 );
1572
1573 // SEC_CM: CTR.REDUN
1574 entropy_src_ctr_reg #(
1575   .RegWidth(FullRegWidth)
1576 ) u_entropy_src_ctr_reg_adaptpt_hi (
1577   .clk_i      (clk_i),
1578   .rst_ni     (rst_ni),
1579   .clear_i    (health_test_clr),
1580   .event_i    (adaptpt_hi_fail_pulse),
1581   .step_i     (FullRegWidth'(1)),
1582   .value_o    (adaptpt_hi_total_fails),
1583   .err_o      (adaptpt_hi_fails_ctr_err)
1584 );
1585
1586 assign hw2reg.adaptpt_hi_total_fails.d = adaptpt_hi_total_fails;
1587
1588 // SEC_CM: CTR.REDUN
1589 entropy_src_ctr_reg #(
1590   .RegWidth(FullRegWidth)
1591 ) u_entropy_src_ctr_reg_adaptpt_lo (
1592   .clk_i      (clk_i),
1593   .rst_ni     (rst_ni),
1594   .clear_i    (health_test_clr),

```

```

1595   .event_i      (adaptpt_lo_fail_pulse),
1596   .step_i       (FullRegWidth'(1)),
1597   .value_o      (adaptpt_lo_total_fails),
1598   .err_o        (adaptpt_lo_fails_ctr_err)
1599 );
1600
1601 assign hw2reg.adaptpt_lo_total_fails.d = adaptpt_lo_total_fails;
1602
1603 //-----
1604 // bucket test
1605 //-----
1606
1607 for (genvar i = 0; i < NumBucketHtInst; i++) begin :
gen_health_test
1608   // SEC_CM: RNG.BKGN_CHK
1609   entropy_src_bucket_ht #(
1610     .RegWidth(HalfRegWidth),
1611     .RngBusWidth(BucketHtDataWidth)
1612   ) u_entropy_src_bucket_ht (
1613     .clk_i      (clk_i),
1614     .rst_ni     (rst_ni),
1615     .entropy_bit_i (health_test_esbus[i*BucketHtDataWidth+:BucketHtDataWidth]),
1616     .entropy_bit_vld_i (health_test_esbus_vld),
1617     .clear_i      (health_test_clr),
1618     .active_i     (bucket_active),
1619     .thresh_i     (bucket_threshold_oneway),
1620     .window_wrap_pulse_i (health_test_done_pulse),
1621     .test_cnt_o   (bucket_event_cnt[i]),
1622     .test_fail_pulse_o (bucket_fail_pulse[i]),
1623     .count_err_o  (bucket_ctr_err[i])
1624   );
1625 end
1626
1627 // Extract the maximum bin counter value from all NumBucketHtInst groups of BucketHtDataWidth
1628 // channels.
1629 if (NumBucketHtInst > 1) begin : gen_bucket_max_tree
1630   prim_max_tree #(
1631     .NumSrc(NumBucketHtInst),
1632     .Width(HalfRegWidth)
1633   ) u_prim_max_tree_bin_ctr_max (
1634     .clk_i,
1635     .rst_ni,
1636     .values_i (bucket_event_cnt),
1637     .valid_i  ({NumBucketHtInst{1'b1}}),
1638     .max_value_o (bucket_event_cnt_max),
1639     .max_idx_o  (),
1640     .max_valid_o()
1641   );
1642 end else begin : gen_no_bucket_max_tree
1643   assign bucket_event_cnt_max = bucket_event_cnt[0];
1644 end
1645
1646 // Add the failure pulses from all NumBucketHtInst groups.
1647 always_comb begin
1648   bucket_fail_pulse_step = 0;
1649   for (int i = 0; i < NumBucketHtInst; i++) begin
1650     bucket_fail_pulse_step += bucket_fail_pulse[i];
1651   end
1652 end
1653
1654 // SEC_CM: CTR.REDUN
1655 entropy_src_ctr_reg #(
1656   .RegWidth(FullRegWidth)
1657 ) u_entropy_src_ctr_reg_bucket (
1658   .clk_i      (clk_i),
1659   .rst_ni     (rst_ni),
1660   .clear_i    (health_test_clr),
1661   .event_i    (|bucket_fail_pulse),
1662   .step_i     (FullReg-
Width'(bucket_fail_pulse_step)),
1663   .value_o    (bucket_total_fails),
1664   .err_o      (bucket_fails_ctr_err)
1665 );
1666
1667 assign hw2reg.bucket_total_fails.d = bucket_total_fails;
1668
1669 //-----
1670 // Markov test
1671 //-----
1672
1673 // SEC_CM: RNG.BKGN_CHK
1674 entropy_src_markov_ht #(
1675   .RegWidth(HalfRegWidth),
1676   .RngBusWidth(RngBusWidth),
1677   .RngBusBitSelWidth(RngBusBitSelWidth)
1678 ) u_entropy_src_markov_ht (
1679   .clk_i      (clk_i),
1680   .rst_ni     (rst_ni),
1681   .entropy_bit_i (health_test_esbus),
1682   .entropy_bit_vld_i (health_test_esbus_vld),
1683   .rng_bit_en_i (rng_bit_en),
1684   .rng_bit_sel_i (rng_bit_sel),
1685   .clear_i      (health_test_clr),
1686   .active_i     (markov_active),
1687   .thresh_hi_i  (markov_hi_threshold_oneway),
1688   .thresh_lo_i  (markov_lo_threshold_oneway),
1689   .window_wrap_pulse_i (health_test_done_pulse),
1690   .threshold_scope_i (threshold_scope),

```

```

1691 .test_cnt_hi_o      (markov_hi_event_cnt),
1692 .test_cnt_lo_o      (markov_lo_event_cnt),
1693 .test_fail_hi_pulse_o (markov_hi_fail_pulse),
1694 .test_fail_lo_pulse_o (markov_lo_fail_pulse),
1695 .count_err_o        (markov_cntr_err)
1696 );
1697
1698 // SEC_CM: CTR.REDUN
1699 entropy_src_cntr_reg #(
1700   .RegWidth(FullRegWidth)
1701 ) u_entropy_src_cntr_reg_markov_hi (
1702   .clk_i      (clk_i),
1703   .rst_ni     (rst_ni),
1704   .clear_i    (health_test_clr),
1705   .event_i    (markov_hi_fail_pulse),
1706   .step_i     (FullRegWidth'(1)),
1707   .value_o    (markov_hi_total_fails),
1708   .err_o      (markov_hi_fails_cntr_err)
1709 );
1710
1711 assign hw2reg.markov_hi_total_fails.d =
markov_hi_total_fails;
1712
1713 // SEC_CM: CTR.REDUN
1714 entropy_src_cntr_reg #(
1715   .RegWidth(FullRegWidth)
1716 ) u_entropy_src_cntr_reg_markov_lo (
1717   .clk_i      (clk_i),
1718   .rst_ni     (rst_ni),
1719   .clear_i    (health_test_clr),
1720   .event_i    (markov_lo_fail_pulse),
1721   .step_i     (FullRegWidth'(1)),
1722   .value_o    (markov_lo_total_fails),
1723   .err_o      (markov_lo_fails_cntr_err)
1724 );
1725
1726 assign hw2reg.markov_lo_total_fails.d =
markov_lo_total_fails;
1727
1728 //-----
1729 // External health test
1730 //-----
1731
1732 // set outputs to external health test
1733 assign entropy_src_xht_valid_o = health_test_esbus_vld;
1734 assign entropy_src_xht_bits_o = health_test_esbus;
1735 assign entropy_src_xht_bit_sel_o = rng_bit_sel;
1736 assign entropy_src_xht_health_test_window_o =
health_test_window;
1737 assign entropy_src_xht_meta_o.rng_bit_en = rng_bit_en;
1738 assign entropy_src_xht_meta_o.clear = health_test_clr;
1739 assign entropy_src_xht_meta_o.active = extht_active;
1740 assign entropy_src_xht_meta_o.thresh_hi =
extht_hi_threshold_oneway;
1741 assign entropy_src_xht_meta_o.thresh_lo =
extht_lo_threshold_oneway;
1742 assign entropy_src_xht_meta_o.window_wrap_pulse =
health_test_done_pulse;
1743 assign entropy_src_xht_meta_o.threshold_scope =
threshold_scope;
1744 // get inputs from external health test
1745 assign extht_event_cnt_hi = en-
tropy_src_xht_meta_i.test_cnt_hi;
1746 assign extht_event_cnt_lo = en-
tropy_src_xht_meta_i.test_cnt_lo;
1747 assign extht_hi_fail_pulse = en-
tropy_src_xht_meta_i.test_fail_hi_pulse;
1748 assign extht_lo_fail_pulse = en-
tropy_src_xht_meta_i.test_fail_lo_pulse;
1749 assign extht_cont_test = en-
tropy_src_xht_meta_i.continuous_test;
1750
1751 // SEC_CM: CTR.REDUN
1752 entropy_src_cntr_reg #(
1753   .RegWidth(FullRegWidth)
1754 ) u_entropy_src_cntr_reg_extht_hi (
1755   .clk_i      (clk_i),
1756   .rst_ni     (rst_ni),
1757   .clear_i    (health_test_clr),
1758   .event_i    (extht_hi_fail_pulse),
1759   .step_i     (FullRegWidth'(1)),
1760   .value_o    (extht_hi_total_fails),
1761   .err_o      (extht_hi_fails_cntr_err)
1762 );
1763
1764 assign hw2reg.extht_hi_total_fails.d = extht_hi_total_fails;
1765
1766 // SEC_CM: CTR.REDUN
1767 entropy_src_cntr_reg #(
1768   .RegWidth(FullRegWidth)
1769 ) u_entropy_src_cntr_reg_extht_lo (
1770   .clk_i      (clk_i),
1771   .rst_ni     (rst_ni),
1772   .clear_i    (health_test_clr),
1773   .step_i     (FullRegWidth'(1)),
1774   .event_i    (extht_lo_fail_pulse),
1775   .value_o    (extht_lo_total_fails),
1776   .err_o      (extht_lo_fails_cntr_err)
1777 );
1778
1779 assign hw2reg.extht_lo_total_fails.d = extht_lo_total_fails;

```

```

1780
1781 //-----
1782 // watermark register
1783 //-----
1784
1785 // Get and resolve HT_WATERMARK_NUM values from register in-
terface.
1786 assign ht_watermark_num_reg_if = ht_water-
mark_num_e'(reg2hw.ht_watermark_num.q);
1787 always_comb begin
1788   unique case (ht_watermark_num_reg_if)
1789     REPCNT_HI,
1790     REPCNTS_HI,
1791     ADAPTP_HI,
1792     ADAPTP_LO,
1793     BUCKET_HI,
1794     MARKOV_HI,
1795     MARKOV_LO,
1796     EXTHT_HI,
1797     EXTHT_LO: ht_watermark_num_d = ht_watermark_num_reg_if;
1798     default: ht_watermark_num_d = REPCNT_HI; // Unsuppor-
ted values are mapped to REPCNT_HI.
1799   endcase
1800 end
1801
1802 // Write the resolved value to the actual register.
1803 always_ff @(posedge clk_i or negedge rst_ni) begin
1804   if (!rst_ni) begin
1805     ht_watermark_num_q <= REPCNT_HI;
1806   end else if (reg2hw.ht_watermark_num.qe) begin
1807     ht_watermark_num_q <= ht_watermark_num_d;
1808   end
1809 end
1810 assign hw2reg.ht_watermark_num.d = ht_watermark_num_q;
1811
1812 // Select the health test for which we want to record the
watermark based on the HT_WATERMARK_NUM
1813 // register.
1814 always_comb begin
1815   unique case (ht_watermark_num_q)
1816     REPCNT_HI: begin
1817       ht_watermark_high      = 1'b1;
1818       ht_watermark_event_pre = 1'b1; // continuous
1819       ht_watermark_cnt      = repcnt_event_cnt;
1820     end
1821     REPCNTS_HI: begin
1822       ht_watermark_high      = 1'b1;
1823       ht_watermark_event_pre = 1'b1; // continuous
1824       ht_watermark_cnt      = repcnts_event_cnt;
1825     end
1826     ADAPTP_HI: begin
1827       ht_watermark_high      = 1'b1;
1828       ht_watermark_event_pre = health_test_done_pulse;
1829       ht_watermark_cnt      = adaptp_hi_event_cnt;
1830     end
1831     ADAPTP_LO: begin
1832       ht_watermark_high      = 1'b0;
1833       ht_watermark_event_pre = health_test_done_pulse;
1834       ht_watermark_cnt      = adaptp_lo_event_cnt;
1835     end
1836     BUCKET_HI: begin
1837       ht_watermark_high      = 1'b1;
1838       ht_watermark_event_pre = health_test_done_pulse;
1839       ht_watermark_cnt      = bucket_event_cnt_max;
1840     end
1841     MARKOV_HI: begin
1842       ht_watermark_high      = 1'b1;
1843       ht_watermark_event_pre = health_test_done_pulse;
1844       ht_watermark_cnt      = markov_hi_event_cnt;
1845     end
1846     MARKOV_LO: begin
1847       ht_watermark_high      = 1'b0;
1848       ht_watermark_event_pre = health_test_done_pulse;
1849       ht_watermark_cnt      = markov_lo_event_cnt;
1850     end
1851     EXTHT_HI: begin
1852       ht_watermark_high      = 1'b1;
1853       ht_watermark_event_pre = extht_cont_test ||
health_test_done_pulse;
1854       ht_watermark_cnt      = extht_event_cnt_hi;
1855     end
1856     EXTHT_LO: begin
1857       ht_watermark_high      = 1'b0;
1858       ht_watermark_event_pre = extht_cont_test ||
health_test_done_pulse;
1859       ht_watermark_cnt      = extht_event_cnt_lo;
1860     end
1861     default: begin // Unsupported values are mapped to REP-
CNT_HI.
1862       ht_watermark_high      = 1'b1;
1863       ht_watermark_event_pre = 1'b1; // continuous
1864       ht_watermark_cnt      = repcnt_event_cnt;
1865     end
1866   endcase
1867 end
1868
1869 // Prevent watermark register updates while the module dis-
abled. Upon enabling, we then clear
1870 // the watermark register before we start recording.
1871 assign ht_watermark_event = ht_watermark_event_pre &&
es_delayed_enable;

```

```

1872
1873 entropy_src_watermark_reg #(
1874   .RegWidth(HalfRegWidth),
1875   .ResVal('0)
1876 ) u_entropy_src_ht_watermark_reg (
1877   .clk_i   (clk_i),
1878   .rst_ni  (rst_ni),
1879   .high_i  (ht_watermark_high),
1880   .clear_i (health_test_clr),
1881   .oneway_i(1'b1), // For the watermark register, the one-
way behavior is always enabled. Upon
1882
1883 // enabling, the register is then cleared
according to ht_watermark_high.
1883   .event_i (ht_watermark_event),
1884   .value_i (ht_watermark_cnt),
1885   .value_o (ht_watermark)
1886 );
1887
1888 assign hw2reg.ht_watermark.d = ht_watermark;
1889
1890 //-----
1891 // summary and alert registers
1892 //-----
1893
1894 // We clear the alert counters when clearing the health
tests (upon module enable), and...
1895 assign alert_cntrs_clr = health_test_clr ||
1896 // ...whenever a health test window ends without a fail-
ure (and when the main state machine
1897 // actually allows the clearing).
1898 (health_test_done_pulse && !ht_failed_d &&
alert_cntr_clr_ok);
1899
1900 // SEC_CM: CTR.REDUN
1901 entropy_src_cntr_reg #(
1902   .RegWidth(HalfRegWidth)
1903 ) u_entropy_src_cntr_reg_any_alert_fails (
1904   .clk_i   (clk_i),
1905   .rst_ni  (rst_ni),
1906   .clear_i (alert_cntrs_clr),
1907   .event_i (any_fail_pulse),
1908   .step_i  (HalfRegWidth'(1)),
1909   .value_o (any_fail_count),
1910   .err_o   (any_fails_cntr_err)
1911 );
1912
1913 assign any_fail_pulse =
1914   repcnt_fail_pulse ||
1915   repcnts_fail_pulse ||
1916   adaptp_hi_fail_pulse || adaptp_lo_fail_pulse ||
1917   (|bucket_fail_pulse) ||
1918   markov_hi_fail_pulse || markov_lo_fail_pulse ||
1919   extht_hi_fail_pulse || extht_lo_fail_pulse;
1920
1921 // The failure pulses of the window-based health tests are
aligned with the
1922
1923 // health_test_done_pulse. In contrast, the continous health
tests (repcnt and repcnts) can
1924
1925 // signal failures at any point in time and we have to latch
these.
1926 assign ht_failed_d =
1927 // The ht_failed_d/q/qq pulses get consumed together
with ht_done_pulse_d/q/qq. Hence, we
1928 // clear them together.
1929 ht_done_pulse_q ? 1'b0 :
1930 // Latch continuous health test failures which can
happen at any point point in a window.
1931 any_fail_pulse ? 1'b1 :
1932 // Eventually clear when disabling. Note, it may be
possible that the last symbol tested
1933 // before disabling triggers a failure. We want to
record this failure. Thus, the clearing
1934 // has the lowest priority and the delayed enable
signal is used as well.
1935 !(es_enable_fo[6] || es_delayed_enable) ? 1'b0 :
1936 ht_failed_q;
1937
1938 // delay health pulse so that main_sm will
1939 // get the correct threshold value comparisons
1940 assign ht_done_pulse_d = health_test_done_pulse;
1941
1942 assign hw2reg.alert_summary_fail_counts.d = any_fail_count;
1943
1944 // signal an alert
1945 // SEC_CM: CONFIG.REDUN
1946 assign alert_threshold =
reg2hw.alert_threshold.alert_threshold.q;
1947 assign alert_threshold_inv =
reg2hw.alert_threshold.alert_threshold_inv.q;
1948 assign es_thresh_cfg_alert = (~alert_threshold_inv !=
alert_threshold);
1949
1950 assign alert_threshold_fail =
1951 ((any_fail_count >= ~alert_threshold_inv) &&
(~alert_threshold_inv != '0)) ||
1952 (any_fail_count >= alert_threshold) &&
(alert_threshold != '0);

```

```

1952
1953
1954 prim_edge_detector #(
1955   .Width(1),
1956   .ResetValue(0),
1957   .EnSync(0)
1958 ) u_prim_edge_detector_recov_alert (
1959   .clk_i,
1960   .rst_ni,
1961   .d_i(recov_alert_state),
1962   .q_sync_o(),
1963   .q_posedge_pulse_o(recov_alert_o),
1964   .q_negedge_pulse_o()
1965 );
1966
1967 assign recov_alert_state =
1968   es_enable_pfa ||
1969   fips_enable_pfa ||
1970   fips_flag_pfa ||
1971   rng_fips_pfa ||
1972   entropy_data_reg_en_pfa ||
1973   threshold_scope_pfa ||
1974   rng_bit_enable_pfa ||
1975   fw_ov_mode_pfa ||
1976   fw_ov_entropy_insert_pfa ||
1977   fw_ov_sha3_start_pfa ||
1978   es_route_pfa ||
1979   es_type_pfa ||
1980   es_main_sm_alert ||
1981   es_bus_cmp_alert ||
1982   es_thresh_cfg_alert ||
1983   es_fw_ov_wr_alert ||
1984   es_fw_ov_disable_alert ||
1985   postht_entropy_drop_alert;
1986
1987 assign hw2reg.recov_alert_sts.es_main_sm_alert.de =
es_main_sm_alert;
1988 assign hw2reg.recov_alert_sts.es_main_sm_alert.d =
es_main_sm_alert;
1989
1990 assign hw2reg.recov_alert_sts.es_bus_cmp_alert.de =
es_bus_cmp_alert;
1991 assign hw2reg.recov_alert_sts.es_bus_cmp_alert.d =
es_bus_cmp_alert;
1992
1993 assign hw2reg.recov_alert_sts.es_thresh_cfg_alert.de =
es_thresh_cfg_alert;
1994 assign hw2reg.recov_alert_sts.es_thresh_cfg_alert.d =
es_thresh_cfg_alert;
1995
1996 assign hw2reg.recov_alert_sts.es_fw_ov_wr_alert.de =
es_fw_ov_wr_alert;
1997 assign hw2reg.recov_alert_sts.es_fw_ov_wr_alert.d =
es_fw_ov_wr_alert;
1998
1999 assign hw2reg.recov_alert_sts.es_fw_ov_disable_alert.de =
es_fw_ov_disable_alert;
2000 assign hw2reg.recov_alert_sts.es_fw_ov_disable_alert.d =
es_fw_ov_disable_alert;
2001
2002 // repcnt fail counter
2003 // SEC_CM: CTR.REDUN
2004 entropy_src_cntr_reg #(
2005   .RegWidth(EighthRegWidth)
2006 ) u_entropy_src_cntr_reg_repcnt_alert_fails (
2007   .clk_i   (clk_i),
2008   .rst_ni  (rst_ni),
2009   .clear_i (alert_cntrs_clr),
2010   .event_i (repcnt_fail_pulse),
2011   .step_i  (EighthRegWidth'(1)),
2012   .value_o (repcnt_fail_count),
2013   .err_o   (repcnt_alert_cntr_err)
2014 );
2015
2016 assign hw2reg.alert_fail_counts.repcnt_fail_count.d = rep-
cnt_fail_count;
2017
2018 // repcnts fail counter
2019 // SEC_CM: CTR.REDUN
2020 entropy_src_cntr_reg #(
2021   .RegWidth(EighthRegWidth)
2022 ) u_entropy_src_cntr_reg_repcnts_alert_fails (
2023   .clk_i   (clk_i),
2024   .rst_ni  (rst_ni),
2025   .clear_i (alert_cntrs_clr),
2026   .event_i (repcnts_fail_pulse),
2027   .step_i  (EighthRegWidth'(1)),
2028   .value_o (repcnts_fail_count),
2029   .err_o   (repcnts_alert_cntr_err)
2030 );
2031
2032 assign hw2reg.alert_fail_counts.repcnts_fail_count.d = rep-
cnts_fail_count;
2033
2034 // adaptp fail counter hi and lo
2035 // SEC_CM: CTR.REDUN
2036 entropy_src_cntr_reg #(
2037   .RegWidth(EighthRegWidth)
2038 ) u_entropy_src_cntr_reg_adaptp_hi_alert_fails (
2039   .clk_i   (clk_i),
2040   .rst_ni  (rst_ni),

```

```

2041 .clear_i          (alert_cntrs_clr),
2042 .event_i          (adaptp_hi_fail_pulse),
2043 .step_i           (EighthRegWidth'1)),
2044 .value_o          (adaptp_hi_fail_count),
2045 .err_o            (adaptp_hi_alert_cntr_err)
2046 );
2047
2048 assign hw2reg.alert_fail_counts.adaptp_hi_fail_count.d = ad-
adaptp_hi_fail_count;
2049
2050 // SEC_CM: CTR.REDUN
2051 entropy_src_cntr_reg #(
2052   .RegWidth(EighthRegWidth)
2053 ) u_entropy_src_cntr_reg_adaptp_lo_alert_fails (
2054   .clk_i          (clk_i),
2055   .rst_ni         (rst_ni),
2056   .clear_i        (alert_cntrs_clr),
2057   .event_i        (adaptp_lo_fail_pulse),
2058   .step_i         (EighthRegWidth'1)),
2059   .value_o        (adaptp_lo_fail_count),
2060   .err_o          (adaptp_lo_alert_cntr_err)
2061 );
2062
2063 assign hw2reg.alert_fail_counts.adaptp_lo_fail_count.d = ad-
adaptp_lo_fail_count;
2064
2065 // bucket fail counter
2066 // SEC_CM: CTR.REDUN
2067 entropy_src_cntr_reg #(
2068   .RegWidth(EighthRegWidth)
2069 ) u_entropy_src_cntr_reg_bucket_alert_fails (
2070   .clk_i          (clk_i),
2071   .rst_ni         (rst_ni),
2072   .clear_i        (alert_cntrs_clr),
2073   .event_i        (|bucket_fail_pulse),
2074   .step_i         (EighthRegWidth'1)),
2075   .value_o        (bucket_fail_count),
2076   .err_o          (bucket_alert_cntr_err)
2077 );
2078
2079 assign hw2reg.alert_fail_counts.bucket_fail_count.d = buck-
et_fail_count;
2080
2081 // markov fail counter hi and lo
2082 // SEC_CM: CTR.REDUN
2083 entropy_src_cntr_reg #(
2084   .RegWidth(EighthRegWidth)
2085 ) u_entropy_src_cntr_reg_markov_hi_alert_fails (
2086   .clk_i          (clk_i),
2087   .rst_ni         (rst_ni),
2088   .clear_i        (alert_cntrs_clr),
2089   .event_i        (markov_hi_fail_pulse),
2090   .step_i         (EighthRegWidth'1)),
2091   .value_o        (markov_hi_fail_count),
2092   .err_o          (markov_hi_alert_cntr_err)
2093 );
2094
2095 assign hw2reg.alert_fail_counts.markov_hi_fail_count.d =
markov_hi_fail_count;
2096
2097 // SEC_CM: CTR.REDUN
2098 entropy_src_cntr_reg #(
2099   .RegWidth(EighthRegWidth)
2100 ) u_entropy_src_cntr_reg_markov_lo_alert_fails (
2101   .clk_i          (clk_i),
2102   .rst_ni         (rst_ni),
2103   .clear_i        (alert_cntrs_clr),
2104   .event_i        (markov_lo_fail_pulse),
2105   .step_i         (EighthRegWidth'1)),
2106   .value_o        (markov_lo_fail_count),
2107   .err_o          (markov_lo_alert_cntr_err)
2108 );
2109
2110 assign hw2reg.alert_fail_counts.markov_lo_fail_count.d =
markov_lo_fail_count;
2111
2112 // extht fail counter hi and lo
2113 // SEC_CM: CTR.REDUN
2114 entropy_src_cntr_reg #(
2115   .RegWidth(EighthRegWidth)
2116 ) u_entropy_src_cntr_reg_extht_hi_alert_fails (
2117   .clk_i          (clk_i),
2118   .rst_ni         (rst_ni),
2119   .clear_i        (alert_cntrs_clr),
2120   .event_i        (extht_hi_fail_pulse),
2121   .step_i         (EighthRegWidth'1)),
2122   .value_o        (extht_hi_fail_count),
2123   .err_o          (extht_hi_alert_cntr_err)
2124 );
2125
2126 assign hw2reg.extht_fail_counts.extht_hi_fail_count.d =
extht_hi_fail_count;
2127
2128 // SEC_CM: CTR.REDUN
2129 entropy_src_cntr_reg #(
2130   .RegWidth(EighthRegWidth)
2131 ) u_entropy_src_cntr_reg_extht_lo_alert_fails (
2132   .clk_i          (clk_i),
2133   .rst_ni         (rst_ni),
2134   .clear_i        (alert_cntrs_clr),

```

```

2136 .event_i          (extht_lo_fail_pulse),
2137 .step_i           (EighthRegWidth'1)),
2138 .value_o          (extht_lo_fail_count),
2139 .err_o            (extht_lo_alert_cntr_err)
2140 );
2141
2142 assign hw2reg.extht_fail_counts.extht_lo_fail_count.d =
extht_lo_fail_count;
2143
2144 //-----
2145 // Signal a recoverable alert when dropping tested entropy
bits.
2146 //-----
2147 //-----
2148
2149 // Post-health test entropy bits can be dropped from the
pipeline in case of e.g. backpressure
2150 // from the conditioner. The conditioner will still use the
amount of bits configured in
2151 // HEALTH_TEST_WINDOW.FIPS_WINDOW to produce the seed, and
the produced seed is okay to use.
2152 // But as the dropped bits are still tested, the effective
test window increases beyond the
2153 // value configured in HEALTH_TEST_WINDOW.FIPS_WINDOW.
2154 //
2155 // Signaling this condition serves the following purposes:
2156 // 1. When running in Firmware Override: Observe mode, drop-
ping post-health test entropy bits
2157 // may cause the entropy bits observed from the Observe
FIFO to be non-contiguous, causing
2158 // the observed bits to be not usable for validation pur-
poses. Note that Firmware Override:
2159 // Extract & Insert mode is not affected by this.
2160
2161 // 2. It allows the DV environment to know when entropy bits
have been dropped which simplifies
2162 // DV.
2163 // 3. It helps tuning the depth of the distr FIFO which can
be used to absorb the backpressure
2164 // of the conditioner.
2165
2166 // The pop signal of the preceding esrng FIFO is uncondi-
tional, meaning samples are dropped
2167 // whenever the esbit or postht FIFO is full in single-chan-
nel or multi-channel mode,
2168 // respectively.
2169 assign postht_entropy_drop_alert = sfifo_esrng_not_empty &&
pfifo_postht_not_full;
2170
2171 assign hw2reg.recov_alert_sts.postht_entropy_drop_alert.de =
postht_entropy_drop_alert;
2172
2173 assign hw2reg.recov_alert_sts.postht_entropy_drop_alert.d =
postht_entropy_drop_alert;
2174 //-----
2175 // Pack health tested esrng bus into single bit packer FIFO.
2176 //-----
2177
2178 // SEC_CM: CONFIG.MUBI
2179 assign mubi_rng_bit_en = mu-
bi4_t'(reg2hw.conf.rng_bit_enable.q);
2180 assign rng_bit_enable_pfe = mu-
bi4_test_true_strict(mubi_rng_bit_en_fanout[0]);
2181 assign rng_bit_enable_pfa = mu-
bi4_test_invalid(mubi_rng_bit_en_fanout[1]);
2182 assign hw2reg.recov_alert_sts.rng_bit_enable_field_alert.de
= rng_bit_enable_pfa;
2183 assign hw2reg.recov_alert_sts.rng_bit_enable_field_alert.d
= rng_bit_enable_pfa;
2184
2185 prim_mubi4_sync #(
2186   .NumCopies(2),
2187   .AsyncOn(0)
2188 ) u_prim_mubi4_sync_rng_bit_en (
2189   .clk_i,
2190   .rst_ni,
2191   .mubi_i(mubi_rng_bit_en),
2192   .mubi_o(mubi_rng_bit_en_fanout)
2193 );
2194
2195 assign rng_bit_en = rng_bit_enable_pfe;
2196 assign rng_bit_sel =
reg2hw.conf.rng_bit_sel.q[RngBusBitSelWidth-1:0];
2197
2198 // If RngBusWidth is less than 256 (8-bit), we have unused
bits to read to avoid linting errors
2199 if (RngBusBitSelWidth < 8) begin : gen_read_unused_bits
2200   logic unused_rng_bit_sel;
2201   assign unused_rng_bit_sel =
^reg2hw.conf.rng_bit_sel.q[7:RngBusBitSelWidth];
2202 end
2203
2204 prim_packer_fifo #(

```

```

2206 .InW(1),
2207 .OutW(RngBusWidth),
2208 .ClearOnRead(1'b0)
2209 ) u_prim_packer_fifo_esbit (
2210 .clk_i (clk_i),
2211 .rst_ni (rst_ni),
2212 .clr_i (pfifo_esbit_clr),
2213 .wvalid_i (pfifo_esbit_push),
2214 .wdata_i (pfifo_esbit_wdata),
2215 .wready_o (pfifo_esbit_not_full),
2216 .rvalid_o (pfifo_esbit_not_empty),
2217 .rdata_o (pfifo_esbit_rdata),
2218 .rready_i (pfifo_esbit_pop),
2219 .depth_o ()
2220 );
2221
2222 // The prim_packer_fifo primitive is constructed to only accept pushes if there is indeed space
2223 // available. The pop signal of the preceeding esrng FIFO is unconditional, meaning samples can
2224 // be dropped before the esbit FIFO in case of backpressure. The samples are however still
2225 // tested.
2226 assign pfifo_esbit_push = rng_bit_en && sfifo_esrng_not_empty;
2227 assign pfifo_esbit_clr = ~es_delayed_enable;
2228 assign pfifo_esbit_pop = rng_bit_en && pfifo_esbit_not_empty && pfifo_postht_not_full;
2229
2230 always_comb begin
2231   pfifo_esbit_wdata = '0;
2232   for (int i = 0; i < RngBusWidth; i++) begin
2233     if (rng_bit_sel == i) begin
2234       pfifo_esbit_wdata = sfifo_esrng_rdata[i];
2235     end
2236   end
2237 end
2238
2239 //-----
2240 // pack tested entropy into 32 bit packer
2241 //-----
2242
2243 prim_packer_fifo #(
2244 .InW(RngBusWidth),
2245 .OutW(PostHTWidth),
2246 .ClearOnRead(1'b0)
2247 ) u_prim_packer_fifo_postht (
2248 .clk_i (clk_i),
2249 .rst_ni (rst_ni),
2250 .clr_i (pfifo_postht_clr),
2251 .wvalid_i (pfifo_postht_push),
2252 .wdata_i (pfifo_postht_wdata),
2253 .wready_o (pfifo_postht_not_full),
2254 .rvalid_o (pfifo_postht_not_empty),
2255 .rdata_o (pfifo_postht_rdata),
2256 .rready_i (pfifo_postht_pop),
2257 .depth_o (pfifo_postht_depth)
2258 );
2259
2260 // The prim_packer_fifo primitive is constructed to only accept pushes if there is indeed space
2261 // available. In case the single-bit mode is enabled, the pop signal of the preceeding esbit FIFO
2262 // is conditional on the full status of the postht FIFO, meaning backpressure can be handled. In
2263 // case the single-bit mode is disabled, the pop signal of the preceeding esrng FIFO is
2264 // unconditional, meaning samples can be dropped before the esbit in case of backpressure. The
2265 // samples are however still tested.
2266 // assign pfifo_postht_push = rng_bit_en ? pfifo_esbit_not_empty : sfifo_esrng_not_empty;
2267 assign pfifo_postht_wdata = rng_bit_en ? pfifo_esbit_rdata : sfifo_esrng_rdata;
2268
2269 // For verification purposes, let post-disable data continue through to the SHA engine if it has
2270 // made it past the health checks, when in standard (non-fw_ov) mode. This allows scoreboards
2271 // to use the same data set for computing both the SHA engine outputs and the health-check stats.
2272 //
2273 // In fw_ov mode it is preferable (from a verification standpoint) to clear all FIFOs whenever
2274 // disabled. Given the lack of handshaking on the fw_ov register path, this is easier to predict.
2275
2276 // Also, there is no association between SHA data and health test windows in FW_OV mode, so there
2277 // is no benefit in this mode to clearing the SHA FIFOs at the same time we clear the HT
2278 // statistics.
2279 assign pfifo_postht_clr = fw_ov_mode_entropy_insert ? !es_enable_fo[7] : !es_delayed_enable;
2280
2281

```

```

2283 // Pop whenever the distribution FIFO is not full. The distribution FIFO can be sized such that
2284 // it's never going to be full even under pessimistic operating conditions.
2285 assign pfifo_postht_pop = sfifo_distr_push & sfifo_distr_not_full;
2286
2287 //-----
2288 // buffer entropy in ditribution FIFO
2289 //-----
2290
2291 // The purpose of this FIFO is to buffer postht entropy bits in case the conditioner cannot
2292 // accept them at the moment, i.e., because it's busy. By properly sizing this FIFO, it can be
2293 // guaranteed that even under pessimistic operating conditions (see
2294 // entropy_src_rng_max_rate test), entropy bits never need to be dropped from the hardware
2295 // pipeline.
2296
2297 // SEC_CM: FIFO.CTR.REDUN
2298 prim_fifo_sync #(
2299 .Width(DistrFifoWidth),
2300 .Pass(1),
2301 .Depth(DistrFifoDepth),
2302 .OutputZeroIfEmpty(0),
2303 .Secure(1)
2304 ) u_prim_fifo_sync_distr (
2305 .clk_i (clk_i),
2306 .rst_ni (rst_ni),
2307 .clr_i (sfifo_distr_clr),
2308 .wvalid_i (sfifo_distr_push),
2309 .wdata_i (sfifo_distr_wdata),
2310 .wready_o (sfifo_distr_not_full),
2311 .rvalid_o (sfifo_distr_not_empty),
2312 .rdata_o (sfifo_distr_rdata),
2313 .rready_i (sfifo_distr_pop),
2314 .full_o (sfifo_distr_full),
2315 .depth_o (sfifo_distr_depth),
2316 .err_o (sfifo_distr_int_err)
2317 );
2318
2319 // The prim_fifo_sync primitive is constructed to only accept pushes if there is indeed space
2320 // available. Backpressure is handled at the sender.
2321 assign sfifo_distr_push = pfifo_postht_not_empty;
2322 assign sfifo_distr_wdata = pfifo_postht_rdata;
2323
2324 assign sfifo_distr_clr = fw_ov_mode_entropy_insert ? !es_enable_fo[19] : !es_delayed_enable;
2325
2326 // In firmware override mode with extract & insert enabled, post-health test entropy bits can
2327 // only move into the observe FIFO. Once the observe FIFO is full, post-health test entropy is
2328 // just discarded.
2329 assign sfifo_distr_pop = fw_ov_mode_entropy_insert ? sfifo_distr_not_empty :
2330 // In firmware override mode (observe only) or during normal
2331 // operation, post-health test entropy bits continue to flow
2332 // through the hardware pipeline.
2333 // es_bypass_mode ?
2334 pfifo_bypass_push : pfifo_precon_push & pfifo_precon_not_full;
2335
2336 // fifo err
2337 // Note that for the used prim_fifo_sync and prim_packer_fifo primitives it is not an error to
2338 // push to a FIFO that is full. The primitives simply don't accept the data when full. The
2339 // backpressure needs to be handled at the sender.
2340 assign sfifo_distr_err =
2341 {1'b0,
2342 (sfifo_distr_pop && !sfifo_distr_not_empty),
2343 (sfifo_distr_full && !sfifo_distr_not_empty) || sfifo_distr_int_err};
2344
2345 //-----
2346 // store entropy into a 32 entry deep FIFO
2347 //-----
2348
2349 // SEC_CM: FIFO.CTR.REDUN
2350 prim_fifo_sync #(
2351 .Width(ObserveFifoWidth),
2352 .Pass(0),
2353 .Depth(ObserveFifoDepth),
2354 .OutputZeroIfEmpty(1), // Prevent SVA from firing due unknown module outputs.
2355 .Secure(1)
2356 ) u_prim_fifo_sync_observe (
2357 .clk_i (clk_i),
2358 .rst_ni (rst_ni),
2359 .clr_i (sfifo_observe_clr),
2360 .wvalid_i (sfifo_observe_push),
2361 .wdata_i (sfifo_observe_wdata),

```



```

2362 .wready_o    (),
2363 .rvalid_o    (sfifo_observe_not_empty),
2364 .rdata_o     (sfifo_observe_rdata),
2365 .rready_i    (sfifo_observe_pop),
2366 .full_o      (sfifo_observe_full),
2367 .depth_o     (sfifo_observe_depth),
2368 .err_o       (sfifo_observe_int_err)
2369 );
2370
2371 // The Observe fifo is intended to hold kilobits of contiguous data, yet still gracefully
2372 // drop data when full. This flop gates the observe fifo.
2373 // If it ever overflows, no new data is
2374 // allowed until it is empty. Thus if the rate of CSR uptake almost matches the RNG data rate
2375 // the FIFO avoids unnecessary segmentation, and guarantees that the remaining RNG data is as
2376 // contiguous as possible.
2377 logic sfifo_observe_gate_d, sfifo_observe_gate_q;
2378 assign sfifo_observe_gate_d = (sfifo_distr_pop && sfifo_observe_full) ? 1'b0 :
2379 !
2380 sfifo_observe_not_empty ? 1'b1 :
2381 sfifo_observe_gate_q;
2382
2383 always_ff @(posedge clk_i or negedge rst_ni) begin
2384 if (!rst_ni) begin
2385 sfifo_observe_gate_q <= 1'b1;
2386 end else begin
2387 sfifo_observe_gate_q <= sfifo_observe_gate_d;
2388 end
2389 end
2390
2391 assign hw2reg.fw_ov_rd_fifo_overflow.d = !
sfifo_observe_gate_d;
2392 assign hw2reg.fw_ov_rd_fifo_overflow.de = 1'b1;
2393
2394 assign observe_fifo_thresh_met = fw_ov_mode && (observe_fifo_thresh != '0) &&
2395 (observe_fifo_thresh <= sfifo_observe_depth) &&
es_enable_fo[8];
2396
2397 assign hw2reg.observe_fifo_depth.d = sfifo_observe_depth;
2398
2399 // fifo controls
2400 assign sfifo_observe_push = fw_ov_mode && sfifo_distr_pop &&
(sfifo_observe_gate_q || !
sfifo_observe_not_empty);
2401
2402 assign sfifo_observe_clr = ~es_enable_fo[9];
2403
2404 assign sfifo_observe_wdata = sfifo_distr_rdata;
2405
2406 assign sfifo_observe_pop =
2407 (fw_ov_mode && fw_ov_fifo_rd_pulse);
2408
2409 // fifo err
2410 // Note that for the used prim_fifo_sync and prim_pack-
er_fifo primitives it is not an error to
2411 // push to a FIFO that is full. The primitives simply don't
accept the data when full. The
2412 // backpressure needs to be handled at the sender.
2413 assign sfifo_observe_err =
2414 {1'b0,
2415 (sfifo_observe_pop && !sfifo_observe_not_empty),
2416 (sfifo_observe_full && !sfifo_observe_not_empty) ||
sfifo_observe_int_err};
2417
2418
2419 //-----
2420 // pack entropy into 64 bit packer
2421 //-----
2422
2423 prim_packer_fifo #(
2424 .InW(ObserveFifoWidth),
2425 .OutW(PreCondWidth),
2426 .ClearOnRead(1'b0)
2427 ) u_prim_packer_fifo_precon (
2428 .clk_i    (clk_i),
2429 .rst_ni   (rst_ni),
2430 .clr_i    (pfifo_precon_clr),
2431 .wvalid_i (pfifo_precon_push),
2432 .wdata_i  (pfifo_precon_wdata),
2433 .wready_o (pfifo_precon_not_full),
2434 .rvalid_o (pfifo_precon_not_empty),
2435 .rdata_o  (pfifo_precon_rdata),
2436 .rready_i (pfifo_precon_pop),
2437 .depth_o  (pfifo_precon_depth)
2438 );
2439
2440 // When bypassing the hardware conditioning - due to a) dis-
abling FIPS mode or b) routing entropy
2441
2442 // to the ENTROPY_DATA register (ES_ROUTE) and bypassing the
conditioner (ES_TYPE) - nothing is
2443 // going into the conditioner.
2444 assign pfifo_precon_push = es_bypass_mode ? 1'b0 :
2445
2446 // In firmware override mode with
extract & insert enabled, only bits

```

```

2445 // inserted by firmware continue
down the pipeline.
2446 fw_ov_mode_entropy_insert ?
2447 // Otherwise post-health test en-
tropy bits continue to flow
2448 // downstream. This includes ob-
serve-only firmware override mode.
2449 sfifo_distr_not_empty;
2450
2451 assign pfifo_precon_wdata = fw_ov_mode_entropy_insert ?
fw_ov_wr_data :
2452 sfifo_distr_rdata;
2453
2454 // For verification purposes, let post-disable data continue
through to the SHA engine if it has
2455 // made it past the health checks, when in standard (non-
fw_ov) mode. This allows scoreboards
2456 // to use the same data set for computing both the SHA en-
gine outputs and the health-check stats.
2457 //
2458 // In fw_ov mode it is preferable (from a verification
standpoint) to clear all FIFOs whenever
2459 // disabled. Given the lack of handshaking on the fw_ov re-
gister path, this is easier to predict.
2460
2461 // Also, there is no association between SHA data and health
test windows in FW_OV mode, so there
2462 // is no benefit in this mode to clearing the SHA FIFOs at
the same time we clear the HT
2463 // statistics.
2464 // Corner case: Even in FW_OV mode, if a full SHA word is
ready as the disable comes in,
2465 // let it stay in the FIFO until the SHA engine has picked
it up, as verification has no way
2466 // of knowing if a word will get stalled by SHA backpres-
sure. This is not a problem however
2467 // as the reset is only important for clearing 32-bit half-
SHA-words.
2468 assign pfifo_precon_clr = fw_ov_mode_entropy_insert ?
~es_enable_fo[10] &
~pfifo_precon_not_empty :
2469 ~es_delayed_enable &
~pfifo_precon_not_empty;
2470
2471 assign pfifo_precon_pop = (pfifo_cond_push &&
sha3_msgfifo_ready);
2472
2473 assign es_fw_ov_wr_alert = fw_ov_mode &&
fw_ov_mode_entropy_insert &&
2474 fw_ov_fifo_wr_pulse && fw_ov_wr_fifo_full;
2475
2476 assign es_fw_ov_disable_alert = fw_ov_mode &&
fw_ov_mode_entropy_insert &&
2477 !es_bypass_mode && fw_ov_sha3_disable_pulse &&
fw_ov_wr_fifo_full;
2478
2479 //-----
2480 // entropy conditioner
2481 //-----
2482 // This block takes in either post-health test entropy bits
2483 // (when running in FIPS/CC compliant
2484 // mode) or entropy injected by software (when running in
Firmware Override: Extract & Insert
2485 // mode).
2486
2487 // The amount of entropy consumed to generate a 384-bit seed
depends on the mode of operation:
2488 // - In FIPS/CC compliant mode,
HEALTH_TEST_WINDOWS.FIPS_WINDOW x 4 bits (by default 2048 bits)
2489
2490 // are required to produce one 384-bit seed. For the first
seed after start up, the number of
2491 // bits consumed is doubled to align with the required
1024 samples of 4 bits for startup
2492 // health testing.
2493 // For windows which fail a health test, the entropy is
still absorbed by the SHA3 engine
2494 // but no seed is produced. In order for the SHA3 engine
to produce a seed, the last window it
2495 // absorbed must have passed the health tests.
2496 // - In Firmware Override: Extract & Insert mode, the opera-
tion of the SHA3 engine is software
2497 // defined.
2498 // Note that the final absorption operation of the SHA3 en-
gine is triggered by the main state
2499 // machine (upon receiving the health-test done pulse). De-
pending on the backpressure currently
2500
2501 // experienced in the pipeline, the sha3_process pulse needs
to be delayed to allow all bits in
2502 // the postht, distr and precon FIFOs to be absorbed first.
The SHA3 engine is also triggered
2503 // internally by the padding logic whenever 832 bits (= the
rate or block size of SHA3-384) have
2504 // been received.
2505 // Note on backpressure handling from the SHA3 engine:

```

```

2505 // To avoid inferring a combo loop, the msg_valid_i input
(pfifo_cond_push signal) must not
2506 // depend on the msg_ready_o output (sha3_msgfifo_ready).
However, we can always push into the
2507 // SHA3 engine as long as the precon FIFO contains valid
data. The ready output is just used to
2508 // determine when to pop from the precon FIFO.
2509
2510 // The postht FIFO must be completely empty when the health-
test done pulse is signaled to the
2511 // main state machine. Note that since the postht FIFO is a
packer FIFO, observing the not_empty
2512 // output signal is not sufficient. The only exception is
when the pipeline is entirely full.
2513 logic [PostHTFifoDepthW-1:0] unused_pfifo_postht_depth;
2514 assign unused_pfifo_postht_depth = pfifo_postht_depth;

2515 `ASSERT(PosthtFifoEmptyWhenHTDonePulse_A, main_sm_done_pulse
|-> pfifo_postht_depth == '0 ||
2516 !pfifo_postht_not_full && sfifo_distr_full)
2517
2518 // Pipeline depth computation
2519 assign pipeline_depth =
2520 PipelineDepthW'(!pfifo_postht_not_full) +
2521 PipelineDepthW'(sfifo_distr_depth) +
2522 PipelineDepthW'(pfifo_precon_depth);
2523
2524 // We only ever push full 64-bit words into the conditioner
and for this reason, we just track
2525 // the number of full 64-bit words that still have to be ab-
sorbed before triggering the final

2526 // processing. In the worst case, the pipeline is completely
full when the health-test done
2527 // pulse is signaled, meaning there are 1 (postht FIFO) +
DistrFifoDepth + 2 (precon FIFO)

2528 // number of 32-bit words in the pipeline. As a consequence,
this number needs to be divisible

2529 // by two and the distribution FIFO depth needs to be an odd
number.
2530 `ASSERT_INIT(DistrFifoDepthOdd_A, DistrFifoDepth % 2 == 1)
2531
2532 logic pipeline_depth_cntr_set;
2533 logic pipeline_depth_cntr_decr_en;
2534 logic [PipelineDepthW-1:0] pipeline_depth_cntr_set_cnt;
2535 logic [PipelineDepthW-1:0] pipeline_depth_cntr;
2536 logic pipeline_depth_cntr_zero;
2537
2538 assign pipeline_depth_cntr_set =
2539 main_sm_done_pulse & ~main_sm_ht_failed &
~fw_ov_mode_entropy_insert;
2540 assign pipeline_depth_cntr_decr_en = pfifo_cond_push &
sha3_msgfifo_ready;
2541 // When we set the counter while the current contents of
precon FIFO get pushed into the
2542 // conditioner, we have to correct the start value of the
counter accordingly.
2543 assign pipeline_depth_cntr_set_cnt =
2544 pipeline_depth_cntr_decr_en ? pipeline_depth -
PipelineDepthW'(2) : pipeline_depth;
2545
2546 prim_count #(
2547 .Width(PipelineDepthW),
2548 .PossibleActions(prim_count_pkg::Clr |
2549 prim_count_pkg::Set |
2550 prim_count_pkg::Decr)
2551 ) u_prim_count_pipeline_depth (
2552 .clk_i,
2553 .rst_ni,
2554 .clr_i(!es_delayed_enable),
2555 .set_i(pipeline_depth_cntr_set),
2556 .set_cnt_i(pipeline_depth_cntr_set_cnt),
2557 .incr_en_i(1'b0),
2558 .decr_en_i(pipeline_depth_cntr_decr_en),
2559 .step_i(PipelineDepthW'(2)),
2560 .commit_i(1'b1),
2561 .cnt_o(pipeline_depth_cntr),
2562 .cnt_after_commit_o(),
2563 .err_o(pipeline_depth_cntr_err)
2564 );
2565
2566 assign pipeline_depth_cntr_zero = pipeline_depth_cntr == '0;
2567
2568 // Backpressure handling
2569 assign pfifo_cond_push = pfifo_precon_not_empty && !es_by-
pass_mode;
2570 assign pfifo_cond_wdata = pfifo_precon_rdata;
2571
2572 // The SHA3 engine is unmasked - only connect Share 0.
2573 assign msg_data[0] = pfifo_cond_wdata;
2574 assign pfifo_cond_rdata = sha3_state[0][SeedLen-1:0];
2575
2576 // SHA3 hashing engine
2577 sha3 #(
2578 .EnMasking (Sha3EnMasking)
2579 ) u_sha3 (
2580 .clk_i,
2581 .rst_ni,

```

```

2583 // MSG_FIFO interface
2584 .msg_valid_i (pfifo_cond_push),
2585 .msg_data_i (msg_data),
2586 .msg_strb_i ({8{pfifo_cond_push}}),
2587 .msg_ready_o (sha3_msgfifo_ready),
2588
2589 // Entropy interface - not using
2590 .rand_valid_i (1'b0),
2591 .rand_early_i (1'b0),
2592 .rand_data_i ('0),
2593 .rand_aux_i ('0),
2594 .rand_update_o (),
2595 .rand_consumed_o (),
2596
2597 // N, S: Used in cSHAKE mode
2598 .ns_data_i ('0), // ns_prefix),
2599
2600 // Configurations
2601 .mode_i (sha3_pkg::Sha3), // Use SHA3 mode
2602 .strength_i (sha3_pkg::L384), // Use keccak_strength_e of
L384
2603
2604 // Controls (CMD register)
2605 .start_i (sha3_start ),
2606 .process_i (sha3_process ),
2607 .run_i (1'b0 ), // For software applica-
tion
2608 .done_i (sha3_done ),
2609
2610 // LC escalation
2611 .lc_escalate_en_i (lc_ctrl_pkg::0ff),
2612
2613 .absorbed_o (sha3_absorbed),
2614 .squeezing_o (sha3_squeezing),
2615
2616 .block_processed_o (sha3_block_processed),
2617
2618 .sha3_fsm_o (sha3_fsm),
2619
2620 .state_valid_o (sha3_state_vld),
2621 .state_o (sha3_state),
2622
2623 // REQ/ACK interface to avoid power spikes
2624 .run_req_o(sha3_block_busy),
2625 .run_ack_i(1'b1),
2626
2627 .error_o (sha3_err),
2628 .sparse_fsm_error_o (sha3_state_error),
2629 .count_error_o (sha3_count_error),
2630 .keccak_storage_rst_error_o (sha3_rst_storage_err)
2631 );
2632
2633 //-----
2634 // bypass SHA conditioner path
2635 //-----
2636
2637 prim_packer_fifo #(
2638 .InW(PostHTWidth),
2639 .OutW(SeedLen),
2640 .ClearOnRead(1'b0)
2641 ) u_prim_packer_fifo_bypass (
2642 .clk_i (clk_i),
2643 .rst_ni (rst_ni),
2644 .clr_i (pfifo_bypass_clr),
2645 .wvalid_i (pfifo_bypass_push),
2646 .wdata_i (pfifo_bypass_wdata),
2647 .wready_o (pfifo_bypass_not_full),
2648 .rvalid_o (pfifo_bypass_not_empty),
2649 .rdata_o (pfifo_bypass_rdata),
2650 .rready_i (pfifo_bypass_pop),
2651 .depth_o ()
2652 );
2653
2654 // Unless the hardware conditioning is bypassed - due to a)
disabling FIPS mode or b) routing
2655 // entropy to the ENTROPY_DATA register (ES_ROUTE) and by-
passing the conditioner (ES_TYPE) -
2656 // nothing is going into the bypass FIFO.
2657 assign pfifo_bypass_push = !es_bypass_mode ? 1'b0 :
2658
2659 // In firmware override mode with
extract & insert enabled, only bits
2659 // inserted by firmware continue
down the pipeline
2660 fw_ov_mode_entropy_insert ?
2661 // Otherwise post-health test en-
tropy bits continue to flow
2662 // downstream. This includes ob-
serve-only firmware override mode.
2663 sfifo_distr_not_empty;
2664
2665 assign pfifo_bypass_wdata = fw_ov_mode_entropy_insert ?
fw_ov_wr_data :
2666 sfifo_distr_rdata;
2667
2668 assign pfifo_bypass_clr = !es_enable_fo[11];
2669
2670 // Corner case: If the main state machine encounters an
alert, drain the

```



```

2671 // bypass fifo, to get rid of the seeds and let the HT stats
continue.
2672 assign pfifo_bypass_pop =
2673     fw_ov_mode_entropy_insert ? pfifo_bypass_not_empty :
2674     bypass_stage_pop;
2675
2676 // mux to select between fips and bypass mode
2677 assign final_es_data = es_bypass_mode ? pfifo_bypass_rdata :
pfifo_cond_rdata;
2678
2679 //-----
2680 // state machine to coordinate fifo flow
2681 //-----
2682 //-----
2683 assign main_sm_done_pulse = rng_bit_en ? ht_done_pulse_qq :
ht_done_pulse_q;
2684 assign main_sm_ht_failed = rng_bit_en ? ht_failed_qq :
ht_failed_q;
2685
2686 // SEC_CM: CTR.LOCAL_ESC
2687 // SEC_CM: MAIN_SM.FSM.SPARSE
2688 entropy_src_main_sm
2689     u_entropy_src_main_sm (
2690         .clk_i          (clk_i),
2691         .rst_ni         (rst_ni),
2692         .enable_i       (main_sm_enable),
2693         .fw_ov_ent_insert_i (fw_ov_mode_entropy_insert),
2694         .fw_ov_sha3_start_i (fw_ov_sha3_start_pfe),
2695         .ht_done_pulse_i (main_sm_done_pulse),
2696         .pd_cntr_zero_i (pipeline_depth_cntr_zero),
2697         .ht_fail_pulse_i (main_sm_ht_failed),
2698         .alert_thresh_fail_i (alert_threshold_fail),
2699         .alert_cntr_clr_ok_o (alert_cntr_clr_ok),
2700         .bypass_mode_i (es_bypass_mode),
2701         .bypass_stage_rdy_i (pfifo_bypass_not_empty),
2702         .sha3_state_vld_i (sha3_state_vld),
2703         .main_stage_push_o (main_stage_push_raw),
2704         .bypass_stage_pop_o (bypass_stage_pop),
2705         .boot_phase_done_o (boot_phase_done),
2706         .sha3_start_o (sha3_start_raw),
2707         .sha3_process_o (sha3_process),
2708         .sha3_done_o (sha3_done),
2709         .local_escalate_i (fatal_loc_events),
2710         .main_sm_alert_o (es_main_sm_alert),
2711         .main_sm_idle_o (es_main_sm_idle),
2712         .main_sm_state_o (es_main_sm_state),
2713         .main_sm_err_o (es_main_sm_err)
2714     );
2715 //-----
2716 // Corner case masking of main_sm inputs/outputs
2717 //-----
2718 //-----
2719
2720 // When operating in RNG mode the state machine does not re-
spond
2721 // immediately to disable requests if it processing the SHA
output
2722 // (here indicated by the sha3_block_busy signal). The SHA
engine
2723 // will continue to process even if the module is disabled.
These seeds
2724 // that continue to process after the disable signal are re-
ferred to as
2725 // stale.
2726 //
2727 // If the SHA processing were instantaneous, stale seeds
would be discarded
2728 // when the esfinal fifo was cleared on disable. Though since
processing
2729 // can push through a disable pulse, stale seeds need to be
identified,
2730 // and held back from the esfinal FIFO.
2731 //
2732 // There is at most one seed processing at a time so, we
simply need to
2733 // detect when a stale seed has commenced processing, and
mask the following
2734 // main_stage_push signal.
2735
2736 assign stale_seed_processing = ~es_bypass_mode &
~fw_ov_mode_entropy_insert &
2737     sha3_block_busy & ~es_en-
able_fo[12];
2738 assign sha3_flush_d = stale_seed_processing ? 1'b1 :
2739     main_stage_push_raw ? 1'b0 :
2740     sha3_flush_q;
2741
2742 // If the user incorrectly disables the fw_ov SHA3 pro-
cessing while
2743 // data is in the pipeline, it can potentially scramble two
outputs.
2744 // Thus in addition to triggering a recoverable alert, we
mark the
2745 // following _two_ outputs as corrupted and to not let them
in the
2746 // esfinal FIFO
2747 assign fw_ov_corrupted_d = es_fw_ov_disable_alert ? 2'b11 :
2748     !es_bypass_mode &&

```

```

main_stage_push_raw ? {1'b0, fw_ov_corrupted_q[1]} :
2749     fw_ov_corrupted_q;
2750
2751 assign fw_ov_corrupted = (!fw_ov_corrupted_q) & !es_by-
pass_mode;
2752
2753
2754 assign main_stage_push = main_stage_push_raw & !sha3_flush_q
& !fw_ov_corrupted;
2755
2756 // Use the delayed enable signal to keep the Main SM enabled
while Data is in flight, and
2757 // to make sure it receives a delayed disable pulse after
finishing any final SHA processing
2758 // commands
2759 assign main_sm_enable = es_delayed_enable;
2760
2761 // The main SM can also generate redundant start pulses.
After data can be pushed into SHA,
2762 // the SM can be disabled leaving entropy in the SHA
sponge. This is fine, but the SM will
2763 // have no recollection of this previous start pulse. We
track redundant start pulses
2764 // outside the SM and suppress them as needed.
2765 assign sha3_start_mask_d = sha3_start_raw ? 1'b1 :
2766     sha3_process ? 1'b0 :
2767     sha3_start_mask_q;
2768 assign sha3_start = sha3_start_raw & ~sha3_start_mask_q;
2769
2770 //-----
2771 // send processed entropy to final fifo
2772 //-----
2773
2774 // SEC_CM: FIFO.CTR.REDUN
2775 prim_fifo_sync #(
2776     .Width(1+SeedLen),
2777     .Pass(0),
2778     .Depth(EsFifoDepth),
2779     .OutputZeroIfEmpty(0),
2780     .Secure(1)
2781 ) u_prim_fifo_sync_esfinal (
2782     .clk_i          (clk_i),
2783     .rst_ni         (rst_ni),
2784     .clr_i          (sfifo_esfinal_clr),
2785     .wvalid_i       (sfifo_esfinal_push),
2786     .wready_o       (sfifo_esfinal_not_full),
2787     .wdata_i        (sfifo_esfinal_wdata),
2788     .rvalid_o       (sfifo_esfinal_not_empty),
2789     .rready_i       (sfifo_esfinal_pop),
2790     .rdata_o        (sfifo_esfinal_rdata),
2791     .full_o         (sfifo_esfinal_full),
2792     .depth_o        (sfifo_esfinal_depth),
2793     .err_o          (sfifo_esfinal_int_err)
2794 );
2795
2796 // The FIPS flag is fully determined in SW. This has to be
the case since we don't know
2797 // which mode of operation will be validated/certified. An-
other reason is that SW can
2798 // set the threshold values arbitrarily, which on its own
makes the FIPS bit basically
2799 // SW defined.
2800 assign fips_compliance = es_enable_fo[13] && fips_flag_pfe;
2801
2802 // fifo controls
2803
2804 // No backpressure is possible at this point. If the esfinal
FIFO is already full and a new seed
2805 // is pushed, the push is ignored and the seed is lost.
2806 assign sfifo_esfinal_push =
2807     fw_ov_mode_entropy_insert && es_bypass_mode ?
pfifo_bypass_not_empty :
2808     main_stage_push;
2809
2810 assign sfifo_esfinal_clr = !es_enable_fo[14];
2811 assign sfifo_esfinal_wdata = {fips_compli-
ance, final_es_data};
2812 assign sfifo_esfinal_pop = es_route_to_sw ? swread_done :
es_hw_if_fifo_pop;
2813 assign {esfinal_fips_flag, esfinal_data} =
sfifo_esfinal_rdata;
2814
2815 // fifo err
2816 // Note that for the used prim_fifo_sync and prim_pack-
er_fifo primitives it is not an error to
2817 // push to a FIFO that is full. The primitives simply don't
accept the data when full. The
2818 // backpressure needs to be handled at the sender.
2819 assign sfifo_esfinal_err =
2820     {1'b0,
2821     (sfifo_esfinal_pop && !sfifo_esfinal_not_empty),
2822     (sfifo_esfinal_full && !sfifo_esfinal_not_empty) ||
sfifo_esfinal_int_err};
2823
2824 // drive out hw interface
2825 assign es_hw_if_req = entropy_src_hw_if_i.es_req;
2826 assign entropy_src_hw_if_o.es_ack = es_hw_if_ack;
2827 assign entropy_src_hw_if_o.es_bits = esfinal_data;
2828 assign entropy_src_hw_if_o.es_fips = esfinal_fips_flag;

```

```

2828
2829 // SEC_CM: ACK_SM.FSM.SPARSE
2830 entropy_src_ack_sm u_entropy_src_ack_sm (
2831     .clk_i          (clk_i),
2832     .rst_ni         (rst_ni),
2833     .enable_i       (es_enable_fo[15]),
2834     .req_i          (es_hw_if_req),
2835     .ack_o          (es_hw_if_ack),
2836     .fifo_not_empty_i (sfifo_esfinal_not_empty && !
es_route_to_sw),
2837     .local_escalate_i (fatal_loc_events),
2838     .fifo_pop_o      (es_hw_if_fifo_pop),
2839     .ack_sm_err_o    (es_ack_sm_err)
2840 );
2841
2842 //-----
2843 // data path integrity check
2844 // - a countermeasure to detect entropy bus tampering at-
tempts
2845 // - checks to make sure repeated data sets off
2846 //   an alert for sw to handle
2847 //-----
2848
2849 // SEC_CM: ESFINAL_RDATA.BUS.CONSTISTENCY
2850
2851 // capture a copy of the entropy data
2852 assign es_rdata_capt_vld = (sfifo_esfinal_pop &&
sfifo_esfinal_not_empty);
2853
2854 assign es_rdata_capt_d = es_rdata_capt_vld ?
sfifo_esfinal_rdata[63:0] : es_rdata_capt_q;
2855
2856 assign es_rdata_capt_vld_d =
2857     !es_enable_fo[16] ? 1'b0 :
2858     es_rdata_capt_vld ? 1'b1 :
2859     es_rdata_capt_vld_q;
2860
2861 // continuous compare of the entropy data
2862 assign es_bus_cmp_alert = es_rdata_capt_vld &&
es_rdata_capt_vld_q &&
2863     (es_rdata_capt_q == sfifo_esfinal_rdata[63:0]);
2864
2865 //-----
2866 // software es read path via ENTROPY_DATA register
2867 //-----
2868
2869 // Sync and evaluate the OTP input.
2870 prim_mubi8_sync #(
2871     .NumCopies(1),
2872     .AsyncOn(1)
2873 ) u_prim_mubi8_sync_es_fw_read (
2874     .clk_i,
2875     .rst_ni,
2876     .mubi_i(otp_en_entropy_src_fw_read_i),
2877     .mubi_o({en_entropy_src_fw_read})
2878 );
2879
2880 assign efuse_es_sw_reg_en =
prim_mubi_pkg::mubi8_test_true_strict(en_entropy_src_fw_read);
2881
2882 // Is the ENTROPY_DATA register readable?
2883 assign es_data_reg_rd_en = es_enable_fo[17] &&
efuse_es_sw_reg_en && entropy_data_reg_en_pfe;
2884
2885 // Interface the esfinal FIFO and cut its 384-bit output
into 32-bit chunks for software.
2886 // We're done after the last read.

2887 assign swread_done = reg2hw.entropy_data.re && (swread_idx_q
== LastSwRead[SwReadIdxWidth-1:0]);
2888
2889 // Increment the index counter upon reads from software.
2890 assign swread_idx_incr = reg2hw.entropy_data.re;
2891
2892 // Unless the ENTROPY_DATA CSR is readable, keep clearing
the index counter. Clear the counter
2893 // after a full seed has been read.
2894 assign swread_idx_clr = !es_data_reg_rd_en || swread_done;
2895
2896 // Index counter
2897 assign swread_idx_d = !es_enable_fo[18] ?
'0
:
2898     swread_idx_clr ?
'0
:
2899     swread_idx_incr ? swread_idx_q +
1'b1 : swread_idx_q;
2900 always_ff @(posedge clk_i or negedge rst_ni) begin :
swread_idx_reg
2901     if (!rst_ni) begin
2902         swread_idx_q <= '0;
2903     end else begin
2904         swread_idx_q <= swread_idx_d;
2905     end
2906 end
2907
2908 // Forward the relevant part of the valid esfinal output or
0 if ES_ROUTE is off.
2909 assign swread_data = sfifo_esfinal_not_empty &&
es_route_to_sw ?
2910     esfinal_data[swread_idx_q * FullRegWidth+: FullReg-
Width] : '0;

```

```

2911
2912 // This primitive is used to place a size-only constraint on
the buffers to act as a synthesis
2913 // optimization barrier. This ensures ES_ROUTE and EN-
TROPY_DATA_REG_ENABLE together with the OTP
2914 // input are taken into account in separately.
2915 prim_buf #(
2916     .Width(FullRegWidth)
2917 ) u_prim_buf_swread_data (
2918     .in_i (swread_data),
2919     .out_o(swread_data_buf)
2920 );
2921
2922 // Forward the data to the ENTROPY_DATA CSR or 0 if the CSR
is not readable.
2923 assign hw2reg.entropy_data.d = es_data_reg_rd_en ?
swread_data_buf : '0;
2924
2925 //-----
2926 // unused signals
2927 //-----
2928
2929 assign unused_err_code_test_bit = {
{err_code_test_bit[27:25],err_code_test_bit[19:4]};
2930
2931 assign unused_sha3_state = ({sha3_state[0]
[sha3_pkg::StateW-1:SeedLen]});
2932 assign unused_entropy_data = {reg2hw.entropy_data.q};
2933 assign unused_fw_ov_rd_data = {reg2hw.fw_ov_rd_data.q};
2934 assign unused_sfifo_esrng_not_full = sfifo_esrng_not_full;
2935 assign unused_sfifo_esfinal_not_full =
sfifo_esfinal_not_full;
2936
2937 //-----
2938 // Assertions
2939 //-----
2940 `ifdef INC_ASSERT
2941 `include "prim_macros.svh"
2942
2943 // Assert that we request high quality entropy only when the
rng_fips field of the conf register
2944 // is set to Mubi4True.
2945 `ASSERT(RngFipsOutputHighInFipsMode_A,
2946
prim_mubi_pkg::mubi4_test_true_loose(mubi4_t'(reg2hw.conf.rng_fips.q
)) == rng_fips_o)
2947
2948 // Count number of disables since last reset.
2949 logic [63:0] disable_cnt_d, disable_cnt_q;
2950 always_comb begin
2951     disable_cnt_d = disable_cnt_q;
2952     if (!mubi4_test_true_strict(mubi_mod_en_dly_d) &&
mubi4_test_true_strict(mubi_mod_en_dly_q)) begin
2953         disable_cnt_d += 1;
2954     end
2955 end
2956
2957
2958 // Assert that no entropy gets dropped during FIPS-compliant
operation mode.
2959 //
2960 // The following code, which includes counters, small FSMs,
and assertions, tracks entropy bits

2961 // from the noise source input through Entropy Source to the
hardware interface output and checks
2962 // that in FIPS-compliant mode entropy does not get dropped
unless it should get dropped. The
2963 // code is arranged in the same order as entropy flows
through Entropy Source, so having Entropy
2964 // Source's block diagram ready when reading this code is
highly recommended.
2965
2966 // Delay `es_delayed_enable` by one clock cycle to track
when Entropy Source actually accepts RNG
2967 // inputs.
2968 logic es_delayed_enable_d, es_delayed_enable_q;
2969 assign es_delayed_enable_d = es_delayed_enable;
2970
2971 // Count number of valid bits from RNG input (RngBusWidth
wide) while Entropy Source is enabled.
2972 logic [63:0] rng_valid_bit_cnt_d, rng_valid_bit_cnt_q;
2973 assign rng_valid_bit_cnt_d = entropy_src_rng_valid_i &&
es_enable_fo[5] &&
2974     es_delayed_enable_q ?
2975     rng_valid_bit_cnt_q + Rng-
BusWidth :
2976     rng_valid_bit_cnt_q;
2977
2978 // Count number of bits pushed into esrng FIFO (RngBusWidth
wide).
2979 logic [63:0] esrng_push_bit_cnt_d, esrng_push_bit_cnt_q;
2980 assign esrng_push_bit_cnt_d = sfifo_esrng_push &
sfifo_esrng_not_full ?
2981     esrng_push_bit_cnt_q + Rng-
BusWidth :
2982     esrng_push_bit_cnt_q;
2983
2984 // Assert that as many bits got pushed into the esrng FIFO

```

```

(destination) as there were valid RNG

2985 // input bits (source). RngBusWidth bits may get lost after
every re-enable; add a margin to
2986 // account for that.
2987 `AS-
SERT_AT_RESET_AND_FINAL(ValidRngBitsPushedIntoEsrngFifo_A,
2988 `WITH-
IN_MARGIN(esrng_push_bit_cnt_q, // actual
2989 rng_val-
id_bit_cnt_q, // expected
2990 disable_cnt_q *
RngBusWidth, // allowed less
2991 0)) // allowed more
2992
2993 // Count number of bits pushed into esbit FIFO (1 wide in-
put, RngBusWidth wide output).
2994 logic [63:0] esbit_push_bit_cnt_d, esbit_push_bit_cnt_q;
2995 assign esbit_push_bit_cnt_d = pfifo_esbit_push &
pfifo_esbit_not_full ?
2996 esbit_push_bit_cnt_q + 1 :
2997 esbit_push_bit_cnt_q;
2998
2999 // Count number of bits pushed into postht FIFO (RngBusWidth
wide input, PostHTWidth wide output).
3000 logic [63:0] postht_push_bit_cnt_d, postht_push_bit_cnt_q;
3001 assign postht_push_bit_cnt_d = pfifo_postht_push &
pfifo_postht_not_full ?
3002 postht_push_bit_cnt_q + Rng-
BusWidth :
3003 postht_push_bit_cnt_q;
3004
3005 // Count number of bits pushed into postht FIFO from esrng
FIFO.
3006 logic [63:0] postht_from_esrng_push_bit_cnt_d, pos-
ttht_from_esrng_push_bit_cnt_q;
3007 assign postht_from_esrng_push_bit_cnt_d =
3008 pfifo_postht_push & pfifo_postht_not_full &
~rng_bit_en ?
3009 postht_from_esrng_push_bit_cnt_q + RngBusWidth :
3010 postht_from_esrng_push_bit_cnt_q;
3011
3012 // Assert that as many bits got pushed into the esbit FIFO
or the postht FIFO (destinations) as
3013 // into the esrng FIFO (source). The number of bits pushed
into the esbit FIFO has to be
3014 // multiplied by the output width of the esrng FIFO (Rng-
BusWidth) because only 1 bit gets pushed
3015 // into the esbit FIFO for every pop from the esrng FIFO.
Add a margin (allowed less) because:
3016 // - RngBusWidth bits may get lost after every re-enable
3017 // - one entry may just have been pushed into esrng FIFO
when the assertion gets evaluated.
3018 `AS-
SERT_AT_RESET_AND_FINAL(EsrngFifoPushedIntoEsbitOrPosthtFifos_A,
3019 `WITH-
IN_MARGIN((esbit_push_bit_cnt_q * RngBusWidth +
3020 ttht_from_esrng_push_bit_cnt_q), // actual pos-
3021 es-
rng_push_bit_cnt_q, // expected
3022 (disable_cnt_q +
1) * RngBusWidth, // allowed less
3023 0)) // allowed more
3024
3025 // Count number of bits pushed into postht FIFO from esbit
FIFO.
3026 logic [63:0] postht_from_esbit_push_bit_cnt_d, pos-
ttht_from_esbit_push_bit_cnt_q;
3027 assign postht_from_esbit_push_bit_cnt_d =
3028 pfifo_postht_push & pfifo_postht_not_full & rng_bit_en ?
3029 postht_from_esbit_push_bit_cnt_q + RngBusWidth :
3030 postht_from_esbit_push_bit_cnt_q;
3031
3032 // Assert that as many bits got pushed into the postht FIFO
(destination) as into the esbit FIFO
3033 // (source) when the latter was selected as source. Add a
margin (allowed less) because:
3034 // - RngBusWidth bits may get lost after every re-enable
3035 // - esbit FIFO may be partially full when the assertion
gets evaluated.
3036 `ASSERT_AT_RESET_AND_FINAL(EsbitFifoPushedIntoPosthtFifo_A,
3037 `WITH-
IN_MARGIN(postht_from_esbit_push_bit_cnt_q, // actual
3038 es-
bit_push_bit_cnt_q, // expected
3039 (disable_cnt_q +
1) * RngBusWidth - 1, // allowed less
3040 0)) // allowed more
3041
3042 // Assert that as many bits got pushed into the postht FIFO
as got counted from the esrng FIFO or
3043 // the esbit FIFO. This assertion checks more the complete-
ness of the other assertions than the
3044 // design itself.
3045 `AS-
SERT_AT_RESET_AND_FINAL(PosthtFifoPushedFromEsbitOrEsrngFifos_A,

```

```

3046 postht_push_bit_cnt_q == (pos-
ttht_from_esrng_push_bit_cnt_q +
3047 ttht_from_esbit_push_bit_cnt_q)) pos-
3048
3049 // Count number of bits popped from postht FIFO (PostHTWidth
wide output) when bypass mode was
3050 // disabled.
3051 logic [63:0] postht_non_bypass_pop_bit_cnt_d, pos-
ttht_non_bypass_pop_bit_cnt_q;
3052 assign postht_non_bypass_pop_bit_cnt_d =
3053 pfifo_postht_pop & pfifo_postht_not_empty & ~es_by-
pass_mode ?
3054 postht_non_bypass_pop_bit_cnt_q + PostHTWidth :
3055 postht_non_bypass_pop_bit_cnt_q;
3056
3057 // Count number of bits pushed into distr FIFO (DistrFifoW-
idth wide input and output).
3058 logic [63:0] distr_non_bypass_push_bit_cnt_d, dis-
tr_non_bypass_push_bit_cnt_q;
3059 assign distr_non_bypass_push_bit_cnt_d =
3060 sfifo_distr_push & sfifo_distr_not_full & ~es_by-
pass_mode ?
3061 distr_non_bypass_push_bit_cnt_q + DistrFifoWidth :
3062 distr_non_bypass_push_bit_cnt_q;
3063
3064 // Assert that as many bits got pushed into the distr FIFO
(destination) as got popped from the
3065 // postht FIFO when bypass mode was disabled (source).
3066 `ASSERT_AT_RESET_AND_FINAL(PosthtFifoPushedIntoDistrFifo_A,
3067 distr_non_bypass_push_bit_cnt_q
== postht_non_bypass_pop_bit_cnt_q)
3068
3069 // Count number of bits popped from distr FIFO (DistrFifoW-
idth wide output) when bypass mode was
3070 // disabled.
3071 logic [63:0] distr_non_bypass_pop_bit_cnt_d, dis-
tr_non_bypass_pop_bit_cnt_q;
3072 assign distr_non_bypass_pop_bit_cnt_d =
3073 sfifo_distr_pop & sfifo_distr_not_empty & ~es_by-
pass_mode ?
3074 distr_non_bypass_pop_bit_cnt_q + DistrFifoWidth :
3075 distr_non_bypass_pop_bit_cnt_q;
3076
3077 // Count number of bits pushed into precon FIFO (ObserveFi-
fowidth wide input, PreCondWidth wide
3078 // output).
3079 logic [63:0] precon_push_bit_cnt_d, precon_push_bit_cnt_q;
3080 assign precon_push_bit_cnt_d = pfifo_precon_push &
pfifo_precon_not_full ?
3081 precon_push_bit_cnt_q + Ob-
serveFifoWidth :
3082 precon_push_bit_cnt_q;
3083
3084 // Assert that as many bits got pushed into the precon FIFO
(destination) as got popped from the
3085 // distr FIFO when bypass mode was disabled (source).
3086 `ASSERT_AT_RESET_AND_FINAL(DistrFifoPushedIntoPreconFifo_A,
3087 precon_push_bit_cnt_q == dis-
tr_non_bypass_pop_bit_cnt_q)
3088
3089 // Track when boot and startup checks are completing.
3090 logic boot_startup_checks_completing;
3091 assign boot_startup_checks_completing =
3092 (u_entropy_src_main_sm.state_q == en-
tropy_src_main_sm_pkg::StartupPass1 &
3093 u_entropy_src_main_sm.ht_done_pulse_i &
3094 ~u_entropy_src_main_sm.ht_fail_pulse_i);
3095
3096 // Track state of boot and startup checks.
3097 typedef enum logic [1:0] {
3098 BscStIncomplete, // checks incomplete
3099 BscStPassed, // checks passed
3100 BscStPushed // entropy from passed checks pushed
} bsc_state_e;
3101 bsc_state_e bsc_state_d, bsc_state_q;
3102 always_comb begin
3103 bsc_state_d = bsc_state_q;
3104 unique case (bsc_state_q)
3105 BscStIncomplete: begin
3106 if (boot_startup_checks_completing) begin
3107 // Checks have just completed.
3108 bsc_state_d = BscStPassed;
3109 end
3110 end
3111 BscStPassed: begin
3112 if (main_stage_push) begin
3113 // Entropy from passed checks is being pushed.
3114 bsc_state_d = BscStPushed;
3115 end
3116 end
3117 end
3118 BscStPushed: begin
3119 // Boot and startup checks remained passed and their
entropy pushed until Entropy Source
3120 // gets disabled again (which is handled below).
3121 bsc_state_d = bsc_state_q;
3122 end
3123 default: bsc_state_d = BscStIncomplete;
3124 endcase
3125 // If not enabled, always clear to incomplete.

```

```

3126     if (!es_delayed_enable) begin
3127         bsc_state_d = BscStIncomplete;
3128     end
3129 end
3130
3131 // Count number of bits pushed into precon FIFO (ObserveFi-
3132 fowidth wide input, PreCondWidth wide
3133 // output) after boot and startup checks.
3134 logic [63:0] precon_post_startup_push_bit_cnt_d, pre-
3135 con_post_startup_push_bit_cnt_q;
3136 assign precon_post_startup_push_bit_cnt_d =
3137     pfifo_precon_push & pfifo_precon_not_full &
3138     (bsc_state_q != BscStIncomplete) ?
3139     precon_post_startup_push_bit_cnt_q + ObserveFifowidth :
3140     precon_post_startup_push_bit_cnt_q;
3141
3142 // Track when esfinal FIFO gets pushed while bypass mode is
3143 disabled.
3144 logic esfinal_non_bypass_push;
3145 assign esfinal_non_bypass_push = sfifo_esfinal_push &
3146     sfifo_esfinal_not_full & ~es_bypass_mode;
3147
3148 // Count number of bits pushed into esfinal FIFO (SeedLen
3149 bits per push) while bypass mode was
3150 // disabled.
3151 logic [63:0] esfinal_non_bypass_push_cnt_d,
3152 esfinal_non_bypass_push_cnt_q;
3153 assign esfinal_non_bypass_push_cnt_d =
3154     esfinal_non_bypass_push & (bsc_state_q != BscStIncom-
3155 plete) ?
3156     esfinal_post_startup_push_bit_cnt_q + SeedLen :
3157     esfinal_post_startup_push_bit_cnt_q;
3158
3159 // Assert that all bits pushed into the esfinal FIFO came
3160 from or after passed startup checks.
3161 `ASSERT_AT_RESET_AND_FINAL(EsfinalFifoPushed_A,
3162     esfinal_non_bypass_push_cnt_q ==
3163     esfinal_post_startup_push_bit_cnt_q)
3164
3165 // Track result of health tests after boot and startup
3166 tests.
3167 typedef enum logic [1:0] {
3168     HtStNoResult, // no health test result is currently avail-
3169 able
3170     HtStPassed, // last health test has passed and entropy
3171     has not propagated from conditioner yet
3172     HtStFailed // last health test has failed
3173 } ht_state_e;
3174 ht_state_e ht_state_d, ht_state_q;
3175 always_comb begin
3176     ht_state_d = ht_state_q;
3177     if (bsc_state_q == BscStPushed) begin
3178         if (ht_state_q inside {HtStNoResult, HtStFailed}) begin
3179             if (health_test_done_pulse) begin
3180                 if (!any_fail_pulse && !alert_threshold_fail) begin
3181                     ht_state_d = HtStPassed;
3182                 end else begin
3183                     ht_state_d = HtStFailed;
3184                 end
3185             end
3186         end
3187     end else if (ht_state_q == HtStPassed) begin
3188         if (main_stage_push_raw)
3189             ht_state_d = HtStNoResult;
3190     end else begin
3191         ht_state_d = HtStNoResult;
3192     end
3193     if (ht_state_q == HtStFailed) begin
3194         `ASSERT_I(NoPushAfterFailedHealthTest_A, rst_ni !=
3195 1'b1 || !main_stage_push_raw)
3196     end
3197 end
3198 // If not enabled, always clear to no result.
3199 if (!es_delayed_enable) begin
3200     ht_state_d = HtStNoResult;
3201 end
3202
3203 // Count number of bits that are expected to have gotten
3204 pushed into precon FIFO and into esfinal
3205 // FIFO after boot and startup checks and while bypass mode
3206 was disabled.
3207 logic [63:0] precon_post_startup_exp_push_bit_cnt_d, pre-
3208 con_post_startup_exp_push_bit_cnt_q;
3209 logic [63:0] esfinal_post_startup_exp_push_bit_cnt_d,
3210 esfinal_post_startup_exp_push_bit_cnt_q;
3211 always_comb begin
3212     esfinal_post_startup_exp_push_bit_cnt_d =
3213     esfinal_post_startup_exp_push_bit_cnt_q;

```

```

3203     precon_post_startup_exp_push_bit_cnt_d = pre-
3204     con_post_startup_exp_push_bit_cnt_q;
3205     if (bsc_state_q == BscStPassed && bsc_state_d == BscSt-
3206     Pushed) begin
3207         // On the completion of boot and startup checks, SeedLen
3208         bits are expected to be pushed into
3209         // the esfinal FIFO.
3210         esfinal_post_startup_exp_push_bit_cnt_d += SeedLen;
3211     end
3212     if (bsc_state_q != BscStIncomplete &&
3213     health_test_done_pulse) begin
3214         // Once boot and startup checks have completed,
3215         health_test_window bits are
3216         // expected to have gotten pushed into the precon FIFO.
3217         precon_post_startup_exp_push_bit_cnt_d +=
3218         health_test_window;
3219     end
3220     if (ht_state_q == HtStPassed && main_stage_push_raw &&
3221     sfifo_esfinal_not_full) begin
3222         // If none of the health tests failed and the alert
3223         threshold has not been exceeded, SeedLen
3224         // bits are expected to be pushed into the esfinal FIFO
3225         -- unless we expect them to get
3226         // dropped (see above).
3227         esfinal_post_startup_exp_push_bit_cnt_d += SeedLen;
3228     end
3229     // When Entropy Source gets disabled after boot and star-
3230     tup checks have been completed, add the
3231     // number of bits that have been pushed into precon FIFO
3232     since the last conditioner output to
3233     // the expected number of bits.
3234     if ((bsc_state_q != BscStIncomplete) && (bsc_state_d ==
3235     BscStIncomplete)) begin
3236         logic [63:0] diff_;
3237         diff_ = precon_post_startup_push_bit_cnt_q - pre-
3238         con_post_startup_exp_push_bit_cnt_q;
3239         // Assert that the difference is not negative.
3240         `ASSERT_I(PreconPostStartupDiffNonNegative_A,
3241             rst_ni != 1'b1 || (pre-
3242             con_post_startup_push_bit_cnt_q >=
3243             precon_post_startup_exp_push_bit_cnt_q))
3244         // Assert that the difference is smaller than the number
3245         of bits that would have sufficed to
3246         // get pushed into the conditioner.
3247         `ASSERT_I(PreconPostStartupDiffSmall_A, rst_ni != 1'b1
3248         || diff_ < health_test_window)
3249         precon_post_startup_exp_push_bit_cnt_d += diff_;
3250     end
3251     // This code assumes that `health_test_window` does not
3252     change dynamically; capture that in an
3253     // assertion ensuring it only changes when entropy_src is
3254     not enabled.
3255     `ASSERT(HealthTestWindowStableWhenEnabled_A,
3256         mubi4_test_true_strict(mubi_es_enable) |->
3257         $stable(health_test_window))
3258
3259 // Assert that all bits pushed into the esfinal FIFO after
3260 startup checks were expected.
3261 `ASSERT_AT_RESET_AND_FINAL(EsfinalFifoPushedPostStartup_A,
3262     esfinal_post_startup_push_bit_cnt_q ==
3263     esfinal_post_startup_exp_push_bit_cnt_q)
3264
3265 // Assert that the expected number of bits pushed into the
3266 precon FIFO based on the number of
3267 // outputs of the conditioner matches the actual number of
3268 bits pushed into the precon FIFO after
3269 // startup checks. Add a margin (allowed more) as the simu-
3270 lation may end when bits in the precon
3271 // FIFO have not resulted in conditioner output and Entropy
3272 Source has not been disabled.
3273 logic [63:0] ppspb_allowed_more;
3274
3275 assign ppspb_allowed_more = bsc_state_q != BscStIncomplete ?
3276     health_test_window : '0;
3277 `ASSERT_AT_RESET_AND_FINAL(PreconFifoPushedPostStartup_A,
3278     WITH-
3279     IN_MARGIN(precon_post_startup_push_bit_cnt_q, // actual
3280     precon_post_startup_exp_push_bit_cnt_q, // expected
3281     0, // allowed less
3282     ppspb_allowed_more)) // allowed more
3283
3284 always_ff @(posedge clk_i, negedge rst_ni) begin
3285     if (!rst_ni) begin
3286         esbit_push_bit_cnt_q <= '0;
3287         esfinal_non_bypass_push_cnt_q <= '0;
3288         esfinal_post_startup_push_bit_cnt_q <= '0;
3289         esrng_push_bit_cnt_q <= '0;
3290         postht_from_esbit_push_bit_cnt_q <= '0;
3291         postht_from_esrng_push_bit_cnt_q <= '0;
3292         postht_non_bypass_pop_bit_cnt_q <= '0;
3293         postht_push_bit_cnt_q <= '0;
3294         distr_non_bypass_pop_bit_cnt_q <= '0;

```

```

3269     distr_non_bypass_push_bit_cnt_q   <= '0';
3270     precon_post_startup_push_bit_cnt_q <= '0';
3271     precon_push_bit_cnt_q              <= '0';
3272     rng_valid_bit_cnt_q                <= '0';
3273   end else if (es_delayed_enable & !
fw_ov_mode_entropy_insert) begin
3274     // All these counters get updated if and only if en-
tropy_src is enabled and the firmware
3275     // override entropy insertion mode is disabled. Other-
wise, there are no guarantees on how
3276     // much entropy from the noise source gets dropped due
to backpressure.
3277     esbit_push_bit_cnt_q               <= es-
bit_push_bit_cnt_d;
3278     esfinal_non_bypass_push_cnt_q      <=
esfinal_non_bypass_push_cnt_d;
3279     esfinal_post_startup_push_bit_cnt_q <=
esfinal_post_startup_push_bit_cnt_d;
3280     esrng_push_bit_cnt_q               <= es-
rng_push_bit_cnt_d;
3281     postht_from_esbit_push_bit_cnt_q   <= pos-
ttht_from_esbit_push_bit_cnt_d;
3282     postht_from_esrng_push_bit_cnt_q   <= pos-
ttht_from_esrng_push_bit_cnt_d;
3283     postht_non_bypass_pop_bit_cnt_q    <= pos-
ttht_non_bypass_pop_bit_cnt_d;
3284     postht_push_bit_cnt_q              <= pos-
ttht_push_bit_cnt_d;
3285     distr_non_bypass_pop_bit_cnt_q     <= dis-
tr_non_bypass_pop_bit_cnt_d;
3286     distr_non_bypass_push_bit_cnt_q    <= dis-
tr_non_bypass_push_bit_cnt_d;
3287     precon_post_startup_push_bit_cnt_q <= pre-
con_post_startup_push_bit_cnt_d;
3288     precon_push_bit_cnt_q              <= pre-
con_push_bit_cnt_d;
3289     rng_valid_bit_cnt_q                <= rng_val-
id_bit_cnt_d;
3290   end
3291 end
3292
3293 always_ff @(posedge clk_i, negedge rst_ni) begin
3294   if (!rst_ni) begin
3295     bsc_state_q <= BscStIncom-
plete;
3296     disable_cnt_q <= '0';
3297     es_delayed_enable_q <= '0';
3298     esfinal_post_startup_exp_push_bit_cnt_q <= '0';
3299     ht_state_q <= HtStNoResult;
3300     precon_post_startup_exp_push_bit_cnt_q <= '0';
3301   end else begin
3302     bsc_state_q <= bsc_state_d;
3303     disable_cnt_q <= dis-
able_cnt_d;
3304     es_delayed_enable_q <=
es_delayed_enable_d;
3305     esfinal_post_startup_exp_push_bit_cnt_q <=
esfinal_post_startup_exp_push_bit_cnt_d;
3306     ht_state_q <= ht_state_d;
3307     precon_post_startup_exp_push_bit_cnt_q <= pre-
con_post_startup_exp_push_bit_cnt_d;
3308   end
3309 end
3310
3311 `endif
3312
3313 endmodule

```

8. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_enable_delay.sv (111 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: Logic module indicate ongoing activity of
after disablement of entropy_src_core
6 //
7 // The entropy_src has a great deal of internal state, and
when shutting down most of this internal
8 // state should be cleared. (The two notable exceptions are the
health test statistics, which are
9 // only cleared at the next enable, and the SHA3 conditioning
sponge which accumulates unused
10 // entropy until a seed is generated). There are delays as
incoming RNG data is processed, and for
11 // ease of verification we insist all elements of the data
pipeline are cleared consistently. This
12 // means that once an RNG sample enters the pipeline, that
sample should be reflected in the health

13 // tests. The SHA3 conditioner is also assumed to successfully
absorb every 64-bits that enters the
14 // module.
15 //

16 // To achieve this consistency goal the entropy_src delays the
clearing of internal data buffers

```

```

17 // and the state machine until:
18 // 1. Any unprocessed data has been counted at the health
checks (regardless of the mode)
19 // 2. Any RNG data bound for the SHA conditioner has been re-
ceived at the conditioner.
20 // 3. Any ongoing SHA processing operations have completed,
and the main FSM has been forced
21 // back to idle.
22 //
23 // This block creates a modified version of the enable pulse
which:
24 // 1. Postpones the disable event until any flowing data has
passed through the RNG, ESBIT, POSTHT
25 // and DISTR FIFOs. If backpressure is encountered at the
Precon FIFO, the stalled data can
26 // be discarded, and so a has a maximum time limit of
MaxFifoWait=4 clocks is given for this
27 // check.
28 // 2. Once the disable signal is received, the rising edge
does not occur until:
29 // 2a. One clock after the falling edge OR
30 // 2b. One clock after the SHA engine completes,
31 // Whichever comes later.
32 //
33 module entropy_src_enable_delay import prim_mubi_pkg::*; (
34   input logic clk_i,
35   input logic rst_ni,
36
37   input logic enable_i,
38
39   // Unconsumed FIFO inputs
40   input logic esrng_fifo_not_empty_i,
41   input logic esbit_fifo_not_empty_i,
42   input logic postht_fifo_not_empty_i,
43   input logic distr_fifo_not_empty_i,
44
45   // SHA3 conditioner inputs
46   input logic sha3_block_busy_i,
47   input logic sha3_block_processed_i,
48
49   input logic bypass_mode_i,
50
51   output logic enable_o
52 );
53
54 // Maximum number of cycles to wait for FIFOs to clear out.
55 // Set to 4 to allow one cycle for each FIFO in the
pipeline.
56 localparam int MaxFifoWait = 4;
57
58 logic suppress_reenable;
59 logic extend_enable;
60
61 logic data_in_flight;
62 logic [3:0] fifos_not_empty;
63
64 // Flops
65 logic [MaxFifoWait - 1:0] fifo_timer_d, fifo_timer_q;
66 logic sha3_active_post_en_d, sha3_act-
ive_post_en_q;
67 logic sha3_block_processed_q;
68 logic extend_enable_q;
69
70 always_ff @(posedge clk_i or negedge rst_ni) begin
71   if (!rst_ni) begin
72     sha3_active_post_en_q <= 1'b0;
73     fifo_timer_q <= '0;
74     sha3_block_processed_q <= 1'b0;
75     extend_enable_q <= 1'b0;
76   end else begin
77     sha3_active_post_en_q <= sha3_active_post_en_d;
78     fifo_timer_q <= fifo_timer_d;
79     sha3_block_processed_q <= sha3_block_processed_i;
80     extend_enable_q <= extend_enable;
81   end
82 end
83
84 // Output definition
85 assign enable_o = (enable_i & ~suppress_reenable) | ex-
tend_enable;
86
87 // In flight data monitoring.
88 // The 'fifo_timer' is a small shift register to count out
the maximum number of cycles to wait
89 // for the FIFOs to drain. Since this timer is very small (3
cycles), it is implemented as a shift
90 // register.
91 assign fifo_timer_d = enable_i ? {MaxFifoWait{1'b1}} :
{fifo_timer_q[MaxFifoWait-2:0], 1'b0};
92 assign fifos_not_empty = {esrng_fifo_not_empty_i, es-
bit_fifo_not_empty_i,
93                           !bypass_mode_i & pos-
ttht_fifo_not_empty_i,
94                           !bypass_mode_i & dis-
tr_fifo_not_empty_i};
95 assign data_in_flight = |fifo_timer_q && |fifos_not_empty;
96
97 // Extend the enable by at least one clock to give the FSM
time to receive any last
98 // Health checks.

```



```

99 assign extend_enable = ((fifo_timer_q[0] | data_in_flight) &
~enable_i);
100
101 // Pulse to extend from the falling edge of the incoming en-
able pulse until one cycle after the
102 // SHA engine has finished processing the current block.
103 assign sha3_active_post_en_d = sha3_block_busy_i && !en-
able_i ? 1'b1 :
104 sha3_block_processed_q ?
1'b0 :
105 sha3_active_post_en_q;
106
107 // Force the output to be low until sha3_active_post_en_q
falls or
108 // for one more cycle after the falling edge of ex-
tend_enable
109 assign suppress_reenable = sha3_active_post_en_q | ex-
tend_enable_q;
110
111 endmodule

```

9. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_field_en.sv (52 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Supports writing a field that enables
6 // a function within a module.
7 // Requirements are that the function will
8 // only be enabled when the field is written
9 // the it is the inverse of the current field
10 // setting. The can only toggle between the
11 // the on value and the off value.
12
13
14 module entropy_src_field_en #(
15     parameter int FieldW = 4,
16     parameter int FieldEnVal = 'ha
17 ) (
18     input logic clk_i,
19     input logic rst_ni,
20     input logic wvalid_i,
21     input logic [FieldW-1:0] wdata_i,
22
23     output logic enable_o
24 );
25
26 localparam logic [FieldW-1:0] FieldEnValBit =
FieldW'(FieldEnVal);
27 localparam logic [FieldW-1:0] FieldEnValBitRevert =
~FieldEnValBit;
28
29 // signal
30 logic field_update;
31
32 // flops
33 logic [FieldW-1:0] field_q, field_d;
34
35 always_ff @(posedge clk_i or negedge rst_ni) begin
36     if (!rst_ni) begin
37         field_q <= FieldEnValBitRevert;
38     end else begin
39         field_q <= field_d;
40     end
41 end
42
43 assign field_update = wvalid_i && (field_q == ~wdata_i) &&
((wdata_i == FieldEnValBit) ||
44 (wdata_i == FieldEnValBitRevert));
45
46 assign field_d = field_update ? wdata_i : field_q;
47
48 assign enable_o = (field_q == FieldEnValBit);
49
50
51
52 endmodule

```

10. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm.sv (287 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src main state machine module
6 //
7 // determines when new entropy is ready to be forwarded
8
9 `include "prim_assert.sv"
10
11 module entropy_src_main_sm
12     import entropy_src_main_sm_pkg::*;

```

```

13 (
14     input logic clk_i,
15     input logic rst_ni,
16
17     input logic enable_i,
18     input logic fw_ov_ent_insert_i,
19     input logic fw_ov_sha3_start_i,
20     input logic ht_done_pulse_i,
21     input logic pd_cntr_zero_i,
22     input logic ht_fail_pulse_i,
23     input logic alert_thresh_fail_i,
24     output logic alert_cntr_clr_ok_o,
25     input logic bypass_mode_i,
26     input logic bypass_stage_rdy_i,
27     input logic sha3_state_vld_i,
28     output logic main_stage_push_o,
29     output logic bypass_stage_pop_o,
30     output logic boot_phase_done_o,
31     output logic sha3_start_o,
32     output logic sha3_process_o,
33     output prim_mubi_pkg::mubi4_t sha3_done_o,
34     input logic local_escalate_i,
35     output logic main_sm_alert_o,
36     output logic main_sm_idle_o,
37     output logic [StateWidth-1:0] main_sm_state_o,
38     output logic main_sm_err_o
39 );
40
41 // The definition of state_e, the sparse FSM state enum, is
in entropy_src_main_sm_pkg.sv
42 state_e state_d, state_q;
43
44 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, Idle)
45
46 assign main_sm_state_o = state_q;
47
48 always_comb begin
49     state_d = state_q;
50     alert_cntr_clr_ok_o = 1'b0;
51     main_stage_push_o = 1'b0;
52     bypass_stage_pop_o = 1'b0;
53     boot_phase_done_o = 1'b0;
54     sha3_start_o = 1'b0;
55     sha3_process_o = 1'b0;
56     sha3_done_o = prim_mubi_pkg::MuBi4False;
57     main_sm_alert_o = 1'b0;
58     main_sm_idle_o = 1'b0;
59     main_sm_err_o = 1'b0;
60     unique case (state_q)
61         Idle: begin
62             main_sm_idle_o = 1'b1;
63             if (enable_i) begin
64                 // running fw override mode and in sha3 mode
65                 if (fw_ov_ent_insert_i && !bypass_mode_i) begin
66                     sha3_start_o = 1'b1;
67                     if (fw_ov_sha3_start_i) begin
68                         state_d = FWInsertMsg;
69                     end else begin
70                         state_d = FWInsertStart;
71                     end
72                 // running in bypass_mode and not fw override mode
73             end else if (bypass_mode_i && !fw_ov_ent_insert_i)
begin
74                 state_d = BootHTRunning;
75                 // running in bypass_mode and fw override mode
76             end else if (bypass_mode_i && fw_ov_ent_insert_i)
begin
77                 state_d = Idle;
78             end else begin
79                 state_d = StartupHTStart;
80             end
81         end
82     end
83     BootHTRunning: begin
84         alert_cntr_clr_ok_o = 1'b1;
85         if (!enable_i) begin
86             state_d = Idle;
87         end else if (ht_done_pulse_i) begin
88             if (ht_fail_pulse_i) begin
89                 if (bypass_stage_rdy_i) begin
90                     // Remove failed data
91                     bypass_stage_pop_o = 1'b1;
92                 end
93             end
94             if (alert_thresh_fail_i) begin
95                 state_d = AlertState;
96             end else begin
97                 state_d = Idle;
98             end
99         end else begin
100             // Window sizes other than 384 bits (the seed
length) are currently not tested nor
101             // supported in bypass or boot-time mode.
102             state_d = BootPostHTChk;
103         end
104     end
105     BootPostHTChk: begin
106         if (!enable_i) begin
107             state_d = Idle;
108         end else begin

```

```

109         if (!bypass_stage_rdy_i) begin
110             end else begin
111                 bypass_stage_pop_o = 1'b1;
112                 main_stage_push_o = 1'b1;
113                 state_d = BootPhaseDone;
114             end
115         end
116     end
117     BootPhaseDone: begin
118         boot_phase_done_o = 1'b1;
119         if (!enable_i) begin
120             state_d = Idle;
121         end
122     // Even when stalled we keep monitoring for alerts and
maintaining alert statistics.
123     // However, we don't signal alerts or clear HT stats
in FW_OV mode.
124     if (!fw_ov_ent_insert_i) begin
125         alert_cntr_clr_ok_o = 1'b1;
126         if (ht_done_pulse_i && alert_thresh_fail_i) begin
127             state_d = AlertState;
128         end
129     end
130 end
131 StartupHTStart: begin
132     if (!enable_i) begin
133         state_d = Idle;
134     end else begin
135         sha3_start_o = 1'b1;
136         state_d = StartupPhase1;
137     end
138 end
139 StartupPhase1: begin
140     if (!enable_i) begin
141         state_d = Idle;
142     end else begin
143         alert_cntr_clr_ok_o = 1'b1;
144         if (ht_done_pulse_i) begin
145             if (ht_fail_pulse_i) begin
146                 state_d = StartupFail1;
147             end else begin
148                 state_d = StartupPass1;
149             end
150         end
151     end
152 end
153 StartupPass1: begin
154     if (!enable_i) begin
155         state_d = Idle;
156     end else begin
157         alert_cntr_clr_ok_o = 1'b1;
158         if (ht_done_pulse_i) begin
159             if (ht_fail_pulse_i) begin
160                 state_d = StartupFail1;
161             end else begin
162                 // We've now passed two consecutive test windows
of the configured window length.
163                 // Next, we're going to compress the collected
entropy to produce a single seed.
164                 state_d = Sha3Process;
165             end
166         end
167     end
168 end
169 StartupFail1: begin
170     if (!enable_i) begin
171         state_d = Idle;
172     end else begin
173         alert_cntr_clr_ok_o = 1'b1;
174         if (ht_done_pulse_i) begin
175             if (ht_fail_pulse_i) begin
176                 // Failed two consecutive tests
177                 state_d = AlertState;
178             end else begin
179                 state_d = StartupPass1;
180             end
181         end
182     end
183 end
184 ContHTStart: begin
185     if (!enable_i) begin
186         state_d = Idle;
187     end else begin
188         // Send the Start trigger to the SHA3 engine. After
this point, collected raw entropy is
189         // going to be moved into the conditioner in chunks
of 64 bits. The SHA3 engine itself
190         // will hash the received raw entropy whenever
either of the following two conditions are
191         // met:
192         // 1) 13 64-bit chunks have been received (13 x 64 =
832 = the rate or block size of
193         // the employed SHA3-384 engine). This condition
is checked internally to the SHA3
194         // engine.
195         // 2) The total amount of received entropy is equal
to the configured health test window
196         // size. This condition is checked by this FSM

```

```

which then triggers the SHA3 engine
197     // using the sha3_process_o signal.
198     sha3_start_o = 1'b1;
199     state_d = ContHTRunning;
200 end
201 end
202 ContHTRunning: begin
203     if (!enable_i) begin
204         state_d = Idle;
205     end else begin
206         alert_cntr_clr_ok_o = 1'b1;
207         if (ht_done_pulse_i) begin
208             // We've finished testing the current window and
all the collected and tested entropy
209             // has been forwarded to the SHA3 engine and has
at least partially been absorbed.
210             if (alert_thresh_fail_i) begin
211                 state_d = AlertState;
212             end else if (!ht_fail_pulse_i) begin
213                 // Move forward and get the conditioner ready to
finish the absorption process.
214                 state_d = Sha3Process;
215             end
216         end
217     end
218 end
219 FWInsertStart: begin
220     if (!enable_i) begin
221         state_d = Idle;
222     end else if (fw_ov_sha3_start_i) begin
223         state_d = FWInsertMsg;
224     end
225 end
226 FWInsertMsg: begin
227     if (!enable_i) begin
228         state_d = Idle;
229     end else if (!fw_ov_sha3_start_i) begin
230         state_d = Sha3Process;
231     end
232 end
233 Sha3Process: begin
234     // Wait for words belonging to the current window to
flow into the conditioner before
235     // triggering the final absorption operation of the
SHA3 engine.
236     if (pd_cntr_zero_i) begin
237         sha3_process_o = 1'b1;
238         state_d = Sha3Valid;
239     end
240 end
241 Sha3Valid: begin
242     if (sha3_state_vld_i) begin
243         state_d = Sha3Done;
244     end
245 end
246 Sha3Done: begin
247     if (!enable_i) begin
248         sha3_done_o = prim_mubi_pkg::MuBi4True;
249         state_d = Sha3MsgDone;
250     end else begin
251         // Push the digest produced by the SHA3 engine into
the final FIFO and clear the
252         // internal state of the SHA3 engine to start from
scratch for the next seed.
253         sha3_done_o = prim_mubi_pkg::MuBi4True;
254         main_stage_push_o = 1'b1;
255         state_d = Sha3MsgDone;
256     end
257 end
258 Sha3MsgDone: begin
259     if (!enable_i || fw_ov_ent_insert_i) begin
260         state_d = Idle;
261     end else begin
262         state_d = ContHTStart;
263     end
264 end
265 AlertState: begin
266     main_sm_alert_o = 1'b1;
267     state_d = AlertHang;
268 end
269 AlertHang: begin
270     if (!enable_i) begin
271         state_d = Idle;
272     end
273 end
274 Error: begin
275     main_sm_err_o = 1'b1;
276 end
277 default: begin
278     state_d = Error;
279     main_sm_err_o = 1'b1;
280 end
281 endcase
282 if (local_escalate_i) begin
283     state_d = Error;
284 end
285 end
286
287 endmodule

```


11. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_main_sm_pkg.sv (54 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // State definitions for entropy_src_main_sm, provided as a
  separate package for use in DV
6
7 package entropy_src_main_sm_pkg;
8
9 // Encoding generated with:
10 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 20 -n 9 \
11 //   -s 2359261201 --language=sv
12 //
13 // Hamming distance histogram:
14 //
15 // 0: --
16 // 1: --
17 // 2: --
18 // 3: ||| (18.42%)
19 // 4: ||| (29.47%)
20 // 5: ||| (26.84%)
21 // 6: ||| (16.32%)
22 // 7: ||| (6.84%)
23 // 8: | (2.11%)
24 // 9: --
25 //
26 // Minimum Hamming distance: 3
27 // Maximum Hamming distance: 8
28 // Minimum Hamming weight: 1
29 // Maximum Hamming weight: 8
30 //
31 localparam int StateWidth = 9;
32 typedef enum logic [StateWidth-1:0] {
33     Idle = 'b01110101, // idle
34     BootHTRunning = 'b111010010, // boot mode, wait for
  health test done pulse
35     BootPostHTChk = 'b101101110, // boot mode, wait for post
  health test packer not empty state
36     BootPhaseDone = 'b010001110, // boot mode, stay here un-
  til master enable is off
37     StartupHTStart = 'b000101100, // startup mode, pulse the
  sha3 start input
38     StartupPhase1 = 'b100000001, // startup mode, look for
  first test pass/fail
39     StartupPass1 = 'b110100101, // startup mode, look for
  first test pass/fail, done if pass
40     StartupFail1 = 'b000010111, // startup mode, look for
  second fail, alert if fail
41     ContHTStart = 'b001000000, // continuous test mode,
  pulse the sha3 start input
42     ContHTRunning = 'b110100010, // continuous test mode,
  wait for health test done pulse
43     FWInsertStart = 'b011000011, // fw ov mode, start the
  sha3 block
44     FWInsertMsg = 'b001011001, // fw ov mode, insert fw
  message into sha3 block
45     Sha3MsgDone = 'b100001111, // sha3 mode, sha3 state
  cleared, go to continuous check mode
46     Sha3Process = 'b011111000, // sha3 mode, pulse the
  sha3 process input
47     Sha3Valid = 'b010111111, // sha3 mode, wait for sha3
  valid indication
48     Sha3Done = 'b110011000, // sha3 mode, capture sha3
  result, pulse done input
49     AlertState = 'b111001101, // if some alert condition
  occurs, pulse an alert indication
50     AlertHang = 'b111111011, // after pulsing alert sig-
  nal, hang here until sw handles
51     Error = 'b001110011 // illegal state reached
  and hang
52 } state_e;
53
54 endpackage

```

12. lowRISC_opentitan/hw/ip/entropy_src/rtl/entropy_src_markov_ht.sv (168 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: entropy_src Markov health test module
6 //
7
8 module entropy_src_markov_ht #(
9     parameter int RegWidth = 16,
10    parameter int RngBusWidth = 4,
11    parameter int RngBusBitSelWidth = 2
12 ) (

```

```

13    input logic clk_i,
14    input logic rst_ni,
15
16    // ins req interface
17    input logic [RngBusWidth-1:0] entropy_bit_i,
18    input logic entropy_bit_vld_i,
19    input logic rng_bit_en_i,
20    input logic [RngBusBitSelWidth-1:0] rng_bit_sel_i,
21    input logic clear_i,
22    input logic active_i,
23    input logic [RegWidth-1:0] thresh_hi_i,
24    input logic [RegWidth-1:0] thresh_lo_i,
25    input logic window_wrap_pulse_i,
26    input logic threshold_scope_i,
27    output logic [RegWidth-1:0] test_cnt_hi_o,
28    output logic [RegWidth-1:0] test_cnt_lo_o,
29    output logic test_fail_hi_pulse_o,
30    output logic test_fail_lo_pulse_o,
31    output logic count_err_o
32 );
33
34 // signals
35 logic [RngBusWidth-1:0] samples_no_match_pulse;
36 logic [RegWidth-1:0] pair_cntr_max;
37 logic [RegWidth-1:0] pair_cntr_min;
38 pair_cntr_min_tmp;
39 logic [RegWidth-1:0] pair_cntr_sum;
40 logic [RngBusWidth-1:0][RegWidth-1:0] pair_cntr;
41 logic [RegWidth-1:0] rng_bit_cnt;
42 logic [RngBusWidth-1:0] pair_cntr_err;
43
44 // flops
45 logic toggle_q, toggle_d;
46 logic [RngBusWidth-1:0] prev_sample_q, prev_sample_d;
47
48 always_ff @(posedge clk_i or negedge rst_ni)
49     if (!rst_ni) begin
50         toggle_q <= '0;
51         prev_sample_q <= '0;
52     end else begin
53         toggle_q <= toggle_d;
54         prev_sample_q <= prev_sample_d;
55     end
56
57 // Markov Test
58 //
59 // Test operation
60 // This test will look at pairs of bit levels per bit-
  stream. A counter for
61 // stream will only count when the pair equals 0b01 or
  0b10.
62
63 for (genvar sh = 0; sh < RngBusWidth; sh = sh+1) begin :
  gen_cntrs
64     // bit sampler
65     assign prev_sample_d[sh] = (!active_i || clear_i) ? '0 :
66     window_wrap_pulse_i ? '0 :
67     entropy_bit_vld_i ? en-
68     tropy_bit_i[sh] :
69     prev_sample_q[sh];
70
71     // pair check
72     assign samples_no_match_pulse[sh] = entropy_bit_vld_i &&
73     toggle_q &&
74     (prev_sample_q[sh] == !entropy_bit_i[sh]);
75
76     // pair counter
77     prim_count #(
78         .Width(RegWidth)
79     ) u_prim_count_pair_cntr (
80         .clk_i,
81         .rst_ni,
82         .clr_i(window_wrap_pulse_i),
83         .set_i(!active_i || clear_i),
84         .set_cnt_i(RegWidth'(0)),
85         .incr_en_i(samples_no_match_pulse[sh]),
86         .decr_en_i(1'b0),
87         .step_i(RegWidth'(1)),
88         .commit_i(1'b1),
89         .cnt_o(pair_cntr[sh]),
90         .cnt_after_commit_o(),
91         .err_o(pair_cntr_err[sh])
92     );
93 end : gen_cntrs
94
95 // create a toggle signal to sample pairs with
96 assign toggle_d = (!active_i || clear_i) ? '0 :
97 window_wrap_pulse_i ? '0 :
98 entropy_bit_vld_i ? (!toggle_q) :
99 toggle_q;
100
101 // determine the highest counter pair counter value
102 prim_max_tree #(
103     .NumSrc(RngBusWidth),
104     .Width(RegWidth)
105 ) u_max (
106     .clk_i (clk_i),
107     .rst_ni (rst_ni),

```

```

108 .values_i (pair_cntr),
109 .valid_i ({RngBusWidth{1'b1}}),
110 .max_value_o (pair_cntr_max),
111 .max_idx_o (),
112 .max_valid_o ()
113 );
114
115 // determine the lowest counter pair counter value
116 // Negate the inputs and outputs of prim_max_tree to find
the minimum
117 // For this unsigned application, one's complement negation
(i.e. logical inversion) is fine.
118 prim_max_tree #(
119 .NumSrc(RngBusWidth),
120 .Width(RegWidth)
121 ) u_min (
122 .clk_i (clk_i),
123 .rst_ni (rst_ni),
124 .values_i (~pair_cntr),
125 .valid_i ({RngBusWidth{1'b1}}),
126 .max_value_o (pair_cntr_min_tmp),
127 .max_idx_o (),
128 .max_valid_o ()
129 );
130
131 // Invert the output back.
132 assign pair_cntr_min = ~pair_cntr_min_tmp;
133
134 prim_sum_tree #(
135 .NumSrc(RngBusWidth),
136 .InWidth(RegWidth)
137 ) u_sum (
138 .clk_i (clk_i),
139 .rst_ni (rst_ni),
140 .values_i (pair_cntr),
141 .valid_i ({RngBusWidth{1'b1}}),
142 .sum_value_o (pair_cntr_sum),
143 .sum_valid_o ()
144 );
145
146 always_comb begin
147   rng_bit_cnt = '0;
148
149   for (int i = 0; i < RngBusWidth; i++) begin
150     if (rng_bit_sel_i == i) begin
151       rng_bit_cnt = pair_cntr[i];
152     end
153   end
154 end
155
156 assign test_cnt_hi_o = rng_bit_en_i ? rng_bit_cnt :
157   threshold_scope_i ? pair_cntr_sum :
158   pair_cntr_max;
159 assign test_cnt_lo_o = rng_bit_en_i ? rng_bit_cnt :
160   threshold_scope_i ? pair_cntr_sum :
161   pair_cntr_min;
162
163 // the pulses will be only one clock in length
164 assign test_fail_hi_pulse_o = active_i && win-
165   dow_wrap_pulse_i && (test_cnt_hi_o > thresh_hi_i);
166 assign test_fail_lo_pulse_o = active_i && win-
167   dow_wrap_pulse_i && (test_cnt_lo_o < thresh_lo_i);
168 assign count_err_o = (!pair_cntr_err);
169 endmodule

```

13. lowRISC_opentitan/hw/ip/entropy_src/rtl/ entropy_src_pkg.sv (80 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package entropy_src_pkg;
7
8 //-----
9 // Entropy Interface
10 //-----

```

```

11
12 parameter int CSRNG_BUS_WIDTH = 384;
13 parameter int FIPS_BUS_WIDTH = 1;
14 parameter int FIPS_CSRNG_BUS_WIDTH = FIPS_BUS_WIDTH +
CSRNG_BUS_WIDTH;
15
16 // For a data width of N, there are 2^N buckets in the buck-
et health test, each with its own
17 // counter. To make this health test scale reasonably well
with `RngBusWidth`, we limit the bucket
18 // health test to 4 bit. If `RngBusWidth` is larger than 4,
the bucket health test gets
19 // instantiated multiple times, once per 4 bit.
20 parameter int BucketHtDataMaxWidth = 4;
21
22 function automatic integer bucket_ht_data_width(integer
rng_bus_width);
23   return rng_bus_width >= BucketHtDataMaxWidth ? BucketHt-
DataMaxWidth : rng_bus_width;
24 endfunction
25
26 function automatic integer num_bucket_ht_inst(integer
rng_bus_width);
27   return prim_util_pkg::ceil_div(rng_bus_width, buck-
et_ht_data_width(rng_bus_width));
28 endfunction
29
30 // es entropy i/f
31 typedef struct packed {
32   logic es_ack;
33   logic [CSRNG_BUS_WIDTH-1:0] es_bits;
34   logic [FIPS_BUS_WIDTH-1:0] es_fips;
35 } entropy_src_hw_if_rsp_t;
36
37 typedef struct packed {
38   logic es_req;
39 } entropy_src_hw_if_req_t;
40
41 parameter entropy_src_hw_if_req_t EN-
TROPY_SRC_HW_IF_REQ_DEFAULT = '{default: '0'};
42 parameter entropy_src_hw_if_rsp_t EN-
TROPY_SRC_HW_IF_RSP_DEFAULT = '{default: '0'};
43
44 // external health test i/f
45 typedef struct packed {
46   logic rng_bit_en;
47   logic clear;
48   logic active;
49   logic [15:0] thresh_hi;
50   logic [15:0] thresh_lo;
51   logic window_wrap_pulse;
52   logic threshold_scope;
53 } entropy_src_xht_meta_req_t;
54
55 typedef struct packed {
56   logic [15:0] test_cnt_hi;
57   logic [15:0] test_cnt_lo;
58   logic continuous_test;
59   logic test_fail_hi_pulse;
60   logic test_fail_lo_pulse;
61 } entropy_src_xht_meta_rsp_t;
62
63 parameter entropy_src_xht_meta_req_t EN-
TROPY_SRC_XHT_META_REQ_DEFAULT = '{default: '0'};
64 parameter entropy_src_xht_meta_rsp_t EN-
TROPY_SRC_XHT_META_RSP_DEFAULT =
65   '{test_cnt_lo: 16'hffff, default: '0'};
66
67 parameter int HT_WATERMARK_NUM_WIDTH = 4;
68 typedef enum logic [HT_WATERMARK_NUM_WIDTH-1:0] {
69   REPCNT_HI = 0,
70   REPCNTS_HI = 1,
71   ADAPTP_HI = 2,
72   ADAPTP_LO = 3,
73   BUCKET_HI = 4,
74   MARKOV_HI = 5,
75   MARKOV_LO = 6,
76   EXTHT_HI = 7,
77   EXTHT_LO = 8
78 } ht_watermark_num_e;
79
80 endpackage : entropy_src_pkg

```

OpenTitan OTBN -- 빅넘버 코프로세서

What. 256비트 이상의 큰 정수 연산 전용 프로세서. 자체 ISA · 레지스터 파일 · 모듈러 곱셈 유닛.

Why it exists. RSA/ECC 는 수백~수천 비트 정수의 모듈러 거듭제곱 이 필요하다. 범용 CPU 로 하면 느리고, 고정 기능 회로로 하면 알고리즘 변경(곡선 교체, 양자내성 전환)에 대응 못 한다. 그래서 프로그래머블 코프로세서라는 절충이 나온다.

학부 복습 · **EECS concepts.** 모듈러 산술 · Montgomery 곱셈(나눗셈 없는 모듈러 감산) · 상수 시간 구현 · 명령어 집합 설계.

석사 수준 · **Practice note.** “고정 기능이나 프로그래머블이냐”는 IP 사업의 핵심 결정이다. 표준이 자주 바뀌는 영역은 프로그래머블이 이긴다. 양자내성 암호 전환이 지금 그 상황이다.

4 files • 5,083 lines

API — otbn.sv

Table 303. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — module / interface / package (1개)

이름	종류
otbn	module

Table 304. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — parameter (14개)

파라미터	형/폭	기본값
Stub	bit	1'b0
RegFile	regfile_e	RegFileFF
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RndCnstUrndPrngSeed	urnd_prng_seed_t	RndCnstUrndPrngSeedDefault
RndCnstUrndPerm	urnd_perm_t	RndCnstUrndPermDefault
SecSkipUrndReseedAtStart	bit	1'b0
SecFixMaiOpSeq	bit	1'b0
SecFixMacOpSeq	bit	1'b0
FeatStubMai	bit	1'b0
RndCnstBnMacUrndPerm	bn_mac_urnd_perm_t	RndCnstBnMacUrndPermDefault
RndCnstMaiUrndPerm	mai_urnd_perm_t	RndCnstMaiUrndPermDefault
RndCnstOtbnKey	otp_ctrl_pkg::otbn_key_t	RndCnstOtbnKeyDefault
RndCnstOtbnNonce	otp_ctrl_pkg::otbn_nonce_t	RndCnstOtbnNonceDefault

Table 305. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — localparam (13개)

상수	형/폭	값
ImemSizeByte	int	int'(otbn_reg_pkg::OTBN_IMEM_SIZE)
DmemBusSizeByte	int	int'(otbn_reg_pkg::OTBN_DMEM_SIZE)
DmemSizeByte	int	DmemBusSizeByte + DmemScratchSizeByte
ImemAddrWidth	int	vbits(ImemSizeByte)
DmemBusAddrWidth	int	vbits(DmemBusSizeByte)
DmemAddrWidth	int	vbits(DmemSizeByte)
LcEscalateCopies	int unsigned	2
ImemSizeWords	int	ImemSizeByte / 4
ImemIndexWidth	int	vbits(ImemSizeWords)
DmemSizeWords	int	DmemSizeByte / (WLEN / 8)
DmemIndexWidth	int	vbits(DmemSizeWords)
DmemBusSizeWords	int	DmemBusSizeByte / (WLEN / 8)
DmemBusIndexWidth	int	vbits(DmemBusSizeWords)

Table 306. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
idle_o	output	prim_mubi_pkg::mubi4_t
intr_done_o	output	logic
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_rma_req_i	input	lc_ctrl_pkg::lc_tx_t
lc_rma_ack_o	output	lc_ctrl_pkg::lc_tx_t
ram_cfg_imem_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_dmem_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_imem_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
ram_cfg_dmem_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
clk_edn_i	input	
rst_edn_ni	input	
edn_rnd_o	output	edn_pkg::edn_req_t
edn_rnd_i	input	edn_pkg::edn_rsp_t

포트	방향	형/폭
edn_urnd_o	output	edn_pkg::edn_req_t
edn_urnd_i	input	edn_pkg::edn_rsp_t
clk_otp_i	input	
rst_otp_ni	input	
otbn_otp_key_o	output	otp_ctrl_pkg::otbn_otp_key_req_t
otbn_otp_key_i	input	otp_ctrl_pkg::otbn_otp_key_rsp_t
keymgr_key_i	input	keymgr_pkg::otbn_key_req_t
kmac_data_o	output	kmac_pkg::app_req_t
kmac_data_i	input	kmac_pkg::app_rsp_t

Table 307. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv — #define / `define 매크로 (1개)

매크로	인자	정의
DEF_FAC_BIT	(NAME)	\

API — otbn_alu_base.sv

Table 308. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_base.sv — module / interface / package (1개)

이름	종류
otbn_alu_base	module

Table 309. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_base.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
operation_i	input	alu_base_operation_t
comparison_i	input	alu_base_comparison_t
operation_result_o	output	logic [31:0]
comparison_result_o	output	logic

API — otbn_alu_bignum.sv

Table 310. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv — module / interface / package (1개)

이름	종류
otbn_alu_bignum	module

Table 311. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv — localparam (29개)

상수	형/폭	값
NFlagsSrcs	int	5
NNoIntgIspr	int	15

상수	형/폭	값
IsprRndNoIntg	int	0
IsprUrndNoIntg	int	1
IsprMaiCtrlNoIntg	int	2
IsprMaiStatusNoIntg	int	3
IsprFlagsNoIntg	int	4
IsprKeyS0LNoIntg	int	5
IsprKeyS0HNoIntg	int	6
IsprKeyS1LNoIntg	int	7
IsprKeyS1HNoIntg	int	8
IsprKmacStatusNoIntg	int	9
IsprKmacCfgNoIntg	int	10
IsprKmacStrbNoIntg	int	11
IsprInsnCntNoIntg	int	12
IsprUrndStateNoIntg	int	13
IsprUrndStatusNoIntg	int	14
NIntgIspr	int	10
IsprModIntg	int	0
IsprAccIntg	int	1
IsprMaiResS0Intg	int	2
IsprMaiResS1Intg	int	3
IsprMaiIn0S0Intg	int	4
IsprMaiIn0S1Intg	int	5
IsprMaiIn1S0Intg	int	6
IsprMaiIn1S1Intg	int	7
IsprKmacDataS0Intg	int	8
IsprKmacDataS1Intg	int	9
IsprNoIntg	int	10

Table 312. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv — 포트 (82개) — 앞 60개만 표시(전체 82개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
operation_i	input	alu_bignum_operation_t
operation_valid_i	input	logic

포트	방향	형/폭
operation_commit_i	input	logic
operation_result_o	output	logic [WLEN-1:0]
selection_flag_o	output	logic
alu_bignum_predec_i	input	alu_bignum_predec_t
ispr_bignum_predec_i	input	ispr_bignum_predec_t
ispr_addr_i	input	ispr_e
ispr_base_wdata_i	input	logic [31:0]
ispr_base_wr_en_i	input	logic [BaseWordsPerWLEN-1:0]
ispr_bignum_wdata_intg_i	input	logic [ExtWLEN-1:0]
ispr_bignum_wr_en_i	input	logic
ispr_flags_wr_i	input	logic [NFlagGroups-1:0]
ispr_wr_commit_i	input	logic
ispr_init_i	input	logic
ispr_rdata_intg_o	output	logic [ExtWLEN-1:0]
ispr_rd_en_i	input	logic
ispr_acc_intg_i	input	logic [ExtWLEN-1:0]
ispr_acc_wr_data_intg_o	output	logic [ExtWLEN-1:0]
ispr_acc_wr_en_o	output	logic
ispr_mod_intg_o	output	logic [ExtWLEN-1:0]
ispr_mai_ctrl_wr_o	output	logic
ispr_mai_ctrl_wdata_o	output	logic [31:0]
ispr_mai_ctrl_rdata_i	input	logic [31:0]
ispr_mai_status_rdata_i	input	logic [31:0]
ispr_mai_in0_s0_wr_o	output	logic
ispr_mai_in0_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_in0_s1_wr_o	output	logic
ispr_mai_in0_s1_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_in1_s0_wr_o	output	logic
ispr_mai_in1_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_in1_s1_wr_o	output	logic
ispr_mai_in1_s1_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_res_s0_wr_o	output	logic
ispr_mai_res_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_mai_res_s1_wr_o	output	logic
ispr_mai_res_s1_wdata_o	output	logic [ExtWLEN-1:0]

포트	방향	형/폭
ispr_mai_in0_s0_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_in0_s1_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_in1_s0_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_in1_s1_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_res_s0_rdata_i	input	logic [ExtWLEN-1:0]
ispr_mai_res_s1_rdata_i	input	logic [ExtWLEN-1:0]
ispr_kmac_status_wr_o	output	logic
ispr_kmac_status_wdata_o	output	logic [31:0]
ispr_kmac_ctrl_wr_o	output	logic
ispr_kmac_ctrl_wdata_o	output	logic [31:0]
ispr_kmac_cfg_wr_o	output	logic
ispr_kmac_cfg_wdata_o	output	logic [31:0]
ispr_kmac_strb_wr_o	output	logic
ispr_kmac_strb_wdata_o	output	logic [31:0]
ispr_kmac_data_s0_wr_o	output	logic
ispr_kmac_data_s0_wdata_o	output	logic [ExtWLEN-1:0]
ispr_kmac_data_s1_wr_o	output	logic
ispr_kmac_data_s1_wdata_o	output	logic [ExtWLEN-1:0]
ispr_kmac_status_rdata_i	input	logic [31:0]
ispr_kmac_cfg_rdata_i	input	logic [31:0]
ispr_kmac_strb_rdata_i	input	logic [31:0]

API — otbn_controller.sv

Table 313. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — module / interface / package (1개)

이름	종류
otbn_controller	module

Table 314. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — parameter (2개)

파라미터	형/폭	기본값
ImemSizeByte	int	4096
DmemSizeByte	int	4096

Table 315. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — localparam (2개)

상수	형/폭	값
ImemAddrWidth	int	prim_util_pkg::vbits(ImemSizeByte)
DmemAddrWidth	int	prim_util_pkg::vbits(DmemSizeByte)

Table 316. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_controller.sv — 포트 (113개) — 앞 60개만 표시(전체 113개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
start_i	input	logic
locking_o	output	logic
err_bit_clear_i	input	logic
fatal_escalate_en_i	input	prim_mubi_pkg::mubi4_t
recov_escalate_en_i	input	prim_mubi_pkg::mubi4_t
rma_req_i	input	prim_mubi_pkg::mubi4_t
err_bits_o	output	controller_err_bits_t
recoverable_err_o	output	logic
insn_fetch_req_valid_o	output	logic
insn_fetch_req_valid_raw_o	output	logic
insn_fetch_req_addr_o	output	logic [ImemAddrWidth-1:0]
insn_fetch_resp_clear_o	output	logic
insn_valid_i	input	logic
insn_illegal_i	input	logic
insn_addr_i	input	logic [ImemAddrWidth-1:0]
insn_dec_base_i	input	insn_dec_base_t
insn_dec_bignum_i	input	insn_dec_bignum_t
insn_dec_shared_i	input	insn_dec_shared_t
rf_base_wr_addr_o	output	logic [4:0]
rf_base_wr_en_o	output	logic
rf_base_wr_commit_o	output	logic
rf_base_wr_data_no_intg_o	output	logic [31:0]
rf_base_wr_data_intg_o	output	logic [BaseIntgWidth-1:0]
rf_base_wr_data_intg_sel_o	output	logic
rf_base_rd_addr_a_o	output	logic [4:0]
rf_base_rd_en_a_o	output	logic
rf_base_rd_data_a_intg_i	input	logic [BaseIntgWidth-1:0]
rf_base_rd_addr_b_o	output	logic [4:0]
rf_base_rd_en_b_o	output	logic
rf_base_rd_data_b_intg_i	input	logic [BaseIntgWidth-1:0]
rf_base_rd_commit_o	output	logic

포트	방향	형/폭
rf_base_call_stack_sw_err_i	input	logic
rf_base_call_stack_hw_err_i	input	logic
rf_bignum_wr_addr_o	output	logic [4:0]
rf_bignum_wr_en_o	output	logic [1:0]
rf_bignum_wr_commit_o	output	logic
rf_bignum_wr_data_no_intg_o	output	logic [WLEN-1:0]
rf_bignum_wr_data_intg_o	output	logic [ExtWLEN-1:0]
rf_bignum_wr_data_intg_sel_o	output	logic
rf_bignum_rd_addr_a_o	output	logic [4:0]
rf_bignum_rd_en_a_o	output	logic
rf_bignum_rd_data_a_intg_i	input	logic [ExtWLEN-1:0]
rf_bignum_rd_addr_b_o	output	logic [4:0]
rf_bignum_rd_en_b_o	output	logic
rf_bignum_rd_data_b_intg_i	input	logic [ExtWLEN-1:0]
rf_bignum_intg_err_i	input	logic
rf_bignum_spurious_we_err_i	input	logic
rf_bignum_rd_a_indirect_onehot_o	output	logic [NWdr-1:0]
rf_bignum_rd_b_indirect_onehot_o	output	logic [NWdr-1:0]
rf_bignum_wr_indirect_onehot_o	output	logic [NWdr-1:0]
rf_bignum_indirect_en_o	output	logic
alu_base_operation_o	output	alu_base_operation_t
alu_base_comparison_o	output	alu_base_comparison_t
alu_base_operation_result_i	input	logic [31:0]
alu_base_comparison_result_i	input	logic
alu_bignum_operation_o	output	alu_bignum_operation_t
alu_bignum_operation_valid_o	output	logic
alu_bignum_operation_commit_o	output	logic

1. lowRISC_opentitan/hw/ip/otbn/rtl/otbn.sv (1,654 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6 `include "prim_fifo_assert.svh"
7
8 /**
9  * OpenTitan Big Number Accelerator (OTBN)
10  */
11 module otbn
12     import prim_alert_pkg::*;
13     import otbn_pkg::*;
14     import otbn_reg_pkg::*;
15 #(

```

```

16     parameter bit          Stub          = 1'b0,
17     parameter regfile_e    RegFile       = RegFileFF,
18     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
mAlerts{1'b1}},
19     // Number of cycles a differential skew is tolerated on the
  alert signal
20     parameter int unsigned AlertSkewCycles = 1,
21
22     // Default seed for URND PRNG
23     parameter urnd_prng_seed_t RndCnstUrndPrngSeed = RndCn-
stUrndPrngSeedDefault,
24     // Compile-time permutation applied to the primary URND out-
  put in otbn_rnd
25     parameter urnd_perm_t RndCnstUrndPerm = RndCnstUrndPermDe-
fault,
26
27     // Skip URND re-seed at the start of an operation. Useful
  for SCA only.
28     parameter bit SecSkipUrndReseedAtStart = 1'b0,
29     // Masking accelerator interface will not randomize operand
  start indexes.

```

```

30 parameter bit SecFixMaiOpSeq = 1'b0,
31 // BN MAC will not randomize the order in which the vector
elements are processed for vectorized
32 // multiplication instructions.
33 parameter bit SecFixMacOpSeq = 1'b0,
34
35 // Masking accelerator is not present. Useful for resource-
bound targets only.
36 parameter bit FeatStubMai = 1'b0,
37
38 // Compile-time permutation for URND permutation in BN MAC
39 parameter bn_mac_urnd_perm_t RndCnstBnMacUrndPerm = RndCnstBnMacUrndPermDefault,
40
41 // Compile-time permutation for URND permutation in MAI
42 parameter mai_urnd_perm_t RndCnstMaiUrndPerm = RndCnstMaiUrndPermDefault,
43
44 // Default seed and nonce for scrambling
45 parameter otp_ctrl_pkg::otbn_key_t RndCnstOtbnKey = RndCnstOtbnKeyDefault,
46 parameter otp_ctrl_pkg::otbn_nonce_t RndCnstOtbnNonce = RndCnstOtbnNonceDefault
47 ) (
48   input clk_i,
49   input rst_ni,
50
51   input tlul_pkg::tl_h2d_t tl_i,
52   output tlul_pkg::tl_d2h_t tl_o,
53
54   // Inter-module signals
55   output prim_mubi_pkg::mubi4_t idle_o,
56
57   // Interrupts
58   output logic intr_done_o,
59
60   // Alerts
61   input prim_alert_pkg::alert_rx_t [NumAlerts-1:0] alert_rx_i,
62   output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o,
63
64   // Lifecycle interfaces
65   input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
66
67   input lc_ctrl_pkg::lc_tx_t lc_rma_req_i,
68   output lc_ctrl_pkg::lc_tx_t lc_rma_ack_o,
69
70   // Memory configuration
71   input prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_imem_i,
72   input prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_dmem_i,
73   output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_imem_o,
74   output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_dmem_o,
75
76   // EDN clock and interface
77   input clk_edn_i,
78   input rst_edn_ni,
79   output edn_pkg::edn_req_t edn_rnd_o,
80   input edn_pkg::edn_rsp_t edn_rnd_i,
81
82   output edn_pkg::edn_req_t edn_urnd_o,
83   input edn_pkg::edn_rsp_t edn_urnd_i,
84
85   // Key request to OTP (running on clk_fixed)
86   input clk_otp_i,
87   input rst_otp_ni,
88   output otp_ctrl_pkg::otbn_otp_key_req_t otbn_otp_key_o,
89   input otp_ctrl_pkg::otbn_otp_key_rsp_t otbn_otp_key_i,
90
91   input keymgr_pkg::otbn_key_req_t keymgr_key_i,
92
93   // KMAC application interface.
94   output kmac_pkg::app_req_t kmac_data_o,
95   input kmac_pkg::app_rsp_t kmac_data_i
96 );
97
98 import prim_mubi_pkg::*;
99 import prim_util_pkg::vbits;
100
101 logic rst_n;
102
103 // hold module in reset permanently when stubbing
104 if (Stub) begin : gen_stub_otbn
105   assign rst_n = 1'b0;
106 end else begin : gen_real_otbn
107   assign rst_n = rst_ni;
108 end
109
110 // The OTBN_*_SIZE parameters are auto-generated by regtool
and come from the bus window sizes;
111 // they are given in bytes and must be powers of two.
112 //
113 // DMEM is actually a bit bigger than OTBN_DMEN_SIZE: there
are an extra DmemScratchSizeByte bytes
114 // that aren't accessible over the bus.
115 localparam int ImemSizeByte =
int'(otbn_reg_pkg::OTBN_IMEM_SIZE);
116 localparam int DmemBusSizeByte = int'(otbn_reg_pkg::OTBN_D-
MEM_SIZE);
117 localparam int DmemSizeByte = DmemBusSizeByte + Dmem-
ScratchSizeByte;

```

```

118
119 localparam int ImemAddrWidth = vbits(ImemSizeByte);
120 localparam int DmemBusAddrWidth = vbits(DmemBusSizeByte);
121 localparam int DmemAddrWidth = vbits(DmemSizeByte);
122
123 `ASSERT_INIT(ImemSizePowerOfTwo, 2 ** ImemAddrWidth == Imem-
SizeByte)
124 `ASSERT_INIT(DmemSizePowerOfTwo, 2 ** DmemAddrWidth == Dmem-
SizeByte)
125
126 logic start_d, start_q;
127 logic busy_execute_d, busy_execute_q;
128 logic wfi_pending;
129 logic wfi_resume_d, wfi_resume_q;
130 logic done, done_core, locking, locking_q;
131 logic busy_secure_wipe;
132 logic init_sec_wipe_done_d, init_sec_wipe_done_q;
133 logic illegal_bus_access_d, illegal_bus_access_q;
134 logic missed_gnt_error_d, missed_gnt_error_q;
135 logic dmem_sec_wipe;
136 logic imem_sec_wipe;
137 logic mems_sec_wipe;
138 logic req_sec_wipe_urnd_keys;
139 logic [127:0] dmem_sec_wipe_urnd_key,
imem_sec_wipe_urnd_key;
140
141 logic core_recoverable_err, recoverable_err_d, recover-
able_err_q;
142 mubi4_t core_escalate_en;
143
144 core_err_bits_t core_err_bits;
145 non_core_err_bits_t non_core_err_bits, non_core_err_bits_d,
non_core_err_bits_q;
146 err_bits_t err_bits, err_bits_d, err_bits_q;
147 logic err_bits_en;
148
149 // ERR_BITS register should be cleared due to a write re-
quest from the host processor
150 // when OTBN is not running.
151 logic err_bits_clear;
152
153 logic software_errs_fatal_q, software_errs_fatal_d;
154 logic wfi_enabled_q, wfi_enabled_d;
155 logic urnd_ctrl_enabled_q, urnd_ctrl_enabled_d;
156
157 otbn_reg2hw_t reg2hw;
158 otbn_hw2reg_t hw2reg;
159 status_e status_d, status_q;
160
161 // Bus device windows, as specified in otbn.hjson
162 typedef enum logic {
163   TlWinImem = 1'b0,
164   TlWinDmem = 1'b1
165 } tl_win_e;
166
167 tlul_pkg::tl_h2d_t tl_win_h2d[2];
168 tlul_pkg::tl_d2h_t tl_win_d2h[2];
169
170 // The clock can be gated and some registers can be updated
as long as OTBN isn't currently
171 // running. Other registers can only be updated when OTBN is
in the Idle state (which also implies
172 // we are not locked). In addition, OTBN may not be gated
when it is paused so a RESUME command
173 // can be properly detected.
174 logic is_not_running_d, is_not_running_q;
175 logic otbn_dmem_scramble_key_req_busy,
otbn_imem_scramble_key_req_busy;
176
177 assign is_not_running_d = ~({busy_execute_d,
wfi_pending,
178   otbn_d-
mem_scramble_key_req_busy,
179   otbn_imem_scramble_key_req_busy,
180   busy_secure_wipe});
181
182 always_ff @(posedge clk_i or negedge rst_ni) begin
183   if (!rst_ni) begin
184     // OTBN starts busy, performing the initial secure wipe.
185     is_not_running_q <= 1'b0;
186   end else begin
187     is_not_running_q <= is_not_running_d;
188   end
189 end
190
191 // Inter-module signals
=====
192
193 // Note: This is not the same thing as STATUS == IDLE. For
example, we want to allow clock gating
194 // when locked.
195 prim_mubi4_sender #(
196   .ResetValue(prim_mubi_pkg::MuBi4True)
197 ) u_prim_mubi4_sender (
198   .clk_i,
199   .rst_ni,
200   .mubi_i(mubi4_bool_to_mubi(is_not_running_q)),
201   .mubi_o(idle_o)
202 );
203

```

```

204
205 // Lifecycle
=====
206
207 localparam int unsigned LcEscalateCopies = 2;
208 lc_ctrl_pkg::lc_tx_t [LcEscalateCopies-1:0] lc_escalate_en;
209 prim_lc_sync #(
210     .NumCopies(LcEscalateCopies)
211 ) u_lc_escalate_en_sync (
212     .clk_i,
213     .rst_ni,
214     .lc_en_i(lc_escalate_en_i),
215     .lc_en_o(lc_escalate_en)
216 );
217
218 lc_ctrl_pkg::lc_tx_t lc_rma_req;
219 prim_lc_sync #(
220     .NumCopies(1)
221 ) u_lc_rma_req_sync (
222     .clk_i,
223     .rst_ni,
224     .lc_en_i(lc_rma_req_i),
225     .lc_en_o({lc_rma_req})
226 );
227
228 // Internally, OTBN uses MUBI types.
229 mubi4_t mubi_rma_req, mubi_rma_ack;
230 assign mubi_rma_req = lc_ctrl_pkg::lc_to_mubi4(lc_rma_req);
231
232 // When stubbing, forward req to ack.
233 if (Stub) begin : gen_stub_rma_ack
234     assign lc_rma_ack_o = lc_rma_req;
235 end else begin : gen_real_rma_ack
236     assign lc_rma_ack_o = lc_c-
trl_pkg::mubi4_to_lc(mubi_rma_ack);
237 end
238
239 // Interrupts
=====
240
241 // Interrupt is set when OTBN is no longer busy. This is the
case when it gets paused or ends
242 // execution. The done interrupt is also set when a direct
paused to locked transition occurs.
243 // This can happen under some faults where the secure wipe
is skipped. These faults raise a fatal
244 // alert but for consistency we still set the interrupt. The
faults are:
245 // - An invalid MuBi on lc_escalate_en_i
246 // - Spurious URND reseed ACK
247 assign done = ((is_busy_status(status_q) &
-is_busy_status(status_d)) |
248     (status_q == StatusPaused) & (status_d ==
StatusLocked)) & init_sec_wipe_done_q;
249
250 prim_intr_hw #(
251     .Width(1)
252 ) u_intr_hw_done (
253     .clk_i,
254     .rst_ni,
255     .event_intr_i (done),
256     .reg2hw_intr_enable_q_i(reg2hw.intr_enable.q),
257     .reg2hw_intr_test_q_i (reg2hw.intr_test.q),
258     .reg2hw_intr_test_qe_i (reg2hw.intr_test.qe),
259     .reg2hw_intr_state_q_i (reg2hw.intr_state.q),
260     .hw2reg_intr_state_de_o(hw2reg.intr_state.de),
261     .hw2reg_intr_state_do (hw2reg.intr_state.d),
262     .intr_o (intr_done_o)
263 );
264
265 // Instruction Memory (IMEM)
=====
266
267 localparam int ImemSizeWords = ImemSizeByte / 4;
268 localparam int ImemIndexWidth = vbits(ImemSizeWords);
269
270 // Access select to IMEM: core (1), or bus (0)
271 logic imem_access_core;
272
273 logic imem_req;
274 logic imem_gnt;
275 logic imem_write;
276 logic imem_wr_collision;
277 logic imem_wpending;
278 logic [ImemIndexWidth-1:0] imem_index;
279 logic [38:0] imem_wdata;
280 logic [38:0] imem_wmask;
281 logic [38:0] imem_rdata;
282 logic imem_rvalid;
283 logic imem_illegal_bus_access;
284 logic imem_missed_gnt;
285
286 logic imem_req_core;
287 logic imem_write_core;
288 logic [ImemIndexWidth-1:0] imem_index_core;
289 logic [38:0] imem_rdata_core;
290 logic imem_rvalid_core;
291
292 logic imem_req_bus;

```

```

293 logic imem_dummy_response_q, imem_dummy_response_d;
294 logic imem_write_bus;
295 logic [ImemIndexWidth-1:0] imem_index_bus;
296 logic [38:0] imem_wdata_bus;
297 logic [38:0] imem_wmask_bus;
298 logic [38:0] imem_rdata_bus, imem_rdata_bus_raw;
299 logic imem_rdata_bus_en_q, imem_rdata_bus_en_d;
300 logic [top_pkg::TL_DBW-1:0] imem_byte_mask_bus;
301 logic imem_rvalid_bus;
302 logic [1:0] imem_error_bus;
303
304 logic imem_bus_intg_violation;
305
306 typedef struct packed {
307     logic imem;
308     logic [14:0] index;
309     logic [31:0] wr_data;
310 } mem_crc_data_in_t;
311
312 logic mem_crc_data_in_valid;
313 mem_crc_data_in_t mem_crc_data_in;
314 logic set_crc;
315 logic [31:0] crc_in, crc_out;
316
317 logic [ImemAddrWidth-1:0] imem_addr_core;
318 assign imem_index_core = imem_addr_core[ImemAddrWidth-1:2];
319
320 logic [1:0] unused_imem_addr_core_wordbits;
321 assign unused_imem_addr_core_wordbits = imem_addr_core[1:0];
322
323 otp_ctrl_pkg::otbn_key_t otbn_imem_scramble_key;
324 otbn_imem_nonce_t otbn_imem_scramble_nonce;
325 logic otbn_imem_scramble_valid;
326 logic un-
used_otbn_imem_scramble_key_seed_valid;
327
328 otp_ctrl_pkg::otbn_key_t otbn_dmem_scramble_key;
329 otbn_dmem_nonce_t otbn_dmem_scramble_nonce;
330 logic otbn_dmem_scramble_valid;
331 logic un-
used_otbn_dmem_scramble_key_seed_valid;
332
333 logic otbn_scramble_state_error;
334
335 // SEC_CM: SCRAMBLE.KEY.SIDELoad
336 otbn_scramble_ctrl #(
337     .RndCnstOtbnKey (RndCnstOtbnKey),
338     .RndCnstOtbnNonce(RndCnstOtbnNonce)
339 ) u_otbn_scramble_ctrl (
340     .clk_i,
341     .rst_ni,
342     .clk_otpn_i,
343     .rst_otpn_i,
344     .otbn_otpn_key_o,
345     .otbn_otpn_key_i,
346     .otbn_dmem_scramble_key_o (otbn_d-
mem_scramble_key),
347     .otbn_dmem_scramble_nonce_o (otbn_d-
mem_scramble_nonce),
348     .otbn_dmem_scramble_valid_o (otbn_d-
mem_scramble_valid),
349     .otbn_d-
mem_scramble_key_seed_valid_o(unused_otbn_dmem_scramble_key_seed_va
lid),
350     .otbn_imem_scramble_key_o
(otbn_imem_scramble_key),
351     .otbn_imem_scramble_nonce_o
(otbn_imem_scramble_nonce),
352     .otbn_imem_scramble_valid_o
(otbn_imem_scramble_valid),
353     .otbn_imem_scramble_key_seed_valid_o(unused_otbn_imem_scr
amble_key_seed_valid),
354     .otbn_dmem_scramble_sec_wipe_i (dmem_sec_wipe),
355     .otbn_d-
mem_scramble_sec_wipe_key_i(dmem_sec_wipe_urnd_key),
356     .otbn_imem_scramble_sec_wipe_i (imem_sec_wipe),
357     .otbn_imem_scramble_sec_wipe_key_i(imem_sec_wipe_urnd_key)
,
358     .otbn_d-
mem_scramble_key_req_busy_o(otbn_dmem_scramble_key_req_busy),
359     .otbn_imem_scramble_key_req_busy_o(otbn_imem_scramble_key-
req_busy),
360     .state_error_o(otbn_scramble_state_error)
);
361
362 // SEC_CM: MEM.SCRAMBLE
363 prim_ram_1p_scr #(
364     .Width (39),
365     .Depth (ImemSizeWords),
366     .InstDepth (ImemSizeWords),
367     .DataBitsPerMask(39),
368     .EnableParity (0)
369 ) u_imem (

```

```

379 .clk_i,
380 .rst_ni(rst_n),
381
382 .key_valid_i(otbn_imem_scramble_valid),
383 .key_i      (otbn_imem_scramble_key),
384 .nonce_i    (otbn_imem_scramble_nonce),
385
386 .req_i      (imem_req),
387 .gnt_o      (imem_gnt),
388 .write_i    (imem_write),
389 .addr_i     (imem_index),
390 .wdata_i    (imem_wdata),
391 .wmask_i    (imem_wmask),
392 .intg_error_i(locking),
393
394 .rdata_o    (imem_rdata),
395 .rvalid_o   (imem_rvalid),
396 .raddr_o    (),
397 .rerror_o   (),
398 .cfg_i      (ram_cfg_imem_i),
399 .cfg_o      (ram_cfg_imem_o),
400
401 .wr_collision_o (imem_wr_collision),
402 .write_pending_o (imem_wpending),
403
404 .alert_o ()
405 );
406
407 // We should never see a request that doesn't get granted. A
fatal error is raised if this occurs.
408 assign imem_missed_gnt = imem_req & ~imem_gnt;
409
410 // IMEM access from main TL-UL bus
411 logic imem_gnt_bus;
412 // Always grant to bus accesses, when OTBN is running a
dummy response is returned
413 assign imem_gnt_bus = imem_req_bus;
414
415 tlul_adapter_sram #(
416   .SramAw      (ImemIndexWidth),
417   .SramDw      (32),
418   .Outstanding (1),
419   .ByteAccess  (0),
420   .ErrOnRead   (0),
421   .EnableDataIntgPt(1),
422   .SecFifoPtr  (1) // SEC_CM: TLUL_FIFO.CTR.REDUN
423 ) u_tlul_adapter_sram_imem (
424   .clk_i,
425   .rst_ni      (rst_n),
426   .tl_i        (tl_win_h2d[TLWinImem]),
427   .tl_o        (tl_win_d2h[TLWinImem]),
428   .en_ifetch_i (MuBi4False),
429   .req_o       (imem_req_bus),
430   .req_type_o  (),
431   .gnt_i       (imem_gnt_bus),
432   .we_o        (imem_write_bus),
433   .addr_o      (imem_index_bus),
434   .wdata_o     (imem_wdata_bus),
435   .wmask_o     (imem_wmask_bus),
436   .intg_error_o (imem_bus_intg_violation),
437   .user_rsvd_o (),
438   .rdata_i     (imem_rdata_bus),
439   .rvalid_i    (imem_rvalid_bus),
440   .rerror_i    (imem_rerror_bus),
441   .compound_txn_in_progress_o (),
442   .readback_en_i (prim_mubi_pkg::MuBi4False),
443   .readback_error_o (),
444   .wr_collision_i (imem_wr_collision),
445   .write_pending_i (imem_wpending)
446 );
447
448 // IMEM access is granted to the core (not the bus) if the
core is executing, being started, or
449 // is paused by a WFI instruction.
450 assign imem_access_core = busy_execute_q | start_q |
wfi_pending;
451
452 assign imem_req_core = imem_access_core ?
imem_req_core : imem_req_bus;
453 assign imem_write_core = imem_access_core ?
imem_write_core : imem_write_bus;
454 assign imem_index_core = imem_access_core ? imem_in-
dex_core : imem_index_bus;
455 assign imem_wdata_core = imem_access_core ?
imem_wdata_core : imem_wdata_bus;
456
457 assign imem_illegal_bus_access = imem_req_bus & imem_ac-
cess_core;
458
459 assign imem_dummy_response_d = imem_illegal_bus_access;
460 always_ff @(posedge clk_i or negedge rst_ni) begin
461   if (!rst_ni) begin
462     imem_dummy_response_q <= 1'b0;
463   end else begin
464     imem_dummy_response_q <= imem_dummy_response_d;
465   end
466 end
467
468 // The instruction memory only supports 32b word writes, so

```

```

we hardcoded its
470 // wmask here.
471 //
472 // Since this could cause confusion if the bus tried to do a
partial write
473 // (which wasn't caught in the TLUL adapter for some reas-
on), we assert that
474 // the wmask signal from the bus is indeed '1' when it re-
quests a write. We
475 // don't have the corresponding check for writes from the
core because the
476 // core cannot perform writes (and has no imem_wmask_o
port).
477 assign imem_wmask = imem_access_core ? '1 : imem_wmask_bus;
478 `ASSERT(ImemWmaskBusIsFullWord_A, imem_req_bus &&
imem_write_bus |> imem_wmask_bus == '1)
479
480 // SEC_CM: DATA_REG_SW.SCA
481 // Blank bus read data interface during core operation to
avoid leaking the currently executed
482 // instruction from IMEM through the bus unintentionally.
Also blank when OTBN is paused, is
483 // returning a dummy response (responding to an illegal bus
access), and when OTBN is locked.
484 assign imem_rdata_bus_en_d =
485   ~(busy_execute_d | start_d | wfi_pending) &
~imem_dummy_response_d & ~locking;
486
487 always_ff @(posedge clk_i or negedge rst_ni) begin
488   if (!rst_ni) begin
489     imem_rdata_bus_en_q <= 1'b1;
490   end else begin
491     imem_rdata_bus_en_q <= imem_rdata_bus_en_d;
492   end
493 end
494
495 prim_blanker #(.Width(39)) u_imem_rdata_bus_blanker (
496   .in_i (imem_rdata),
497   .en_i (imem_rdata_bus_en_q),
498   .out_o(imem_rdata_bus_raw)
499 );
500
501 // When OTBN is locked all imem bus reads should return 0.
The blanker produces the 0s, this adds
502 // the appropriate ECC. When OTBN is not locked the output
of the blanker is passed straight
503 // through. Data bits are always left un-modified. A re-
gistered version of `locking` is used for
504 // timing reasons. When a read comes in when `locking` has
just been asserted, `locking_q` will be
505
506 // set the following cycle and the rdata will be forced to 0
with appropriate ECC. When `locking`
507 // is asserted the cycle the rdata is being returned no
locking was occurring when the request
508
509 // came in so it is reasonable to proceed with returning the
supplied integrity.
510 assign imem_rdata_bus =
511   {locking_q ? prim_secdd_pkg::SecddInv3932ZeroEcc :
imem_rdata_bus_raw[38:32],
512   imem_rdata_bus_raw[31:0]};
513
514 `ASSERT(ImemRdataBusDisabledWhenCoreAccess_A, imem_ac-
cess_core |> !imem_rdata_bus_en_q)
515 `ASSERT(ImemRdataBusEnabledWhenIdle_A, status_q ==
StatusIdle |> imem_rdata_bus_en_q)
516 `ASSERT(ImemRdataBusDisabledWhenLocked_A, locking |> !
imem_rdata_bus_en_q)
517 `ASSERT(ImemRdataBusReadAsZeroWhenLocked_A,
518   imem_rvalid_bus & locking |> imem_rdata_bus_raw == '0)
519
520 assign imem_rdata_core = imem_rdata;
521
522 // When an illegal bus access is seen, always return a dummy
response the follow cycle.
523
524 assign imem_rvalid_bus = (~imem_access_core & imem_rvalid) |
imem_dummy_response_q;
525 assign imem_rvalid_core = imem_access_core ? imem_rvalid :
1'b0;
526
527 assign imem_byte_mask_bus = tl_win_h2d[TLWinImem].a_mask;
528
529 // No imem errors reported for bus reads. Integrity is car-
ried through on the bus so integrity
530 // checking on TL responses will pick up any errors.
531 assign imem_rerror_bus = 2'b00;
532
533 // Data Memory (DMEM)
534
535 localparam int DmemSizeWords = DmemSizeByte / (WLEN / 8);
536 localparam int DmemIndexWidth = vbits(DmemSizeWords);
537
538 localparam int DmemBusSizeWords = DmemBusSizeByte / (WLEN /
8);
539 localparam int DmemBusIndexWidth = vbits(DmemBusSizeWords);
540

```



```

538 // Access select to DMEM: core (1), or bus (0)
539 logic dmem_access_core;
540
541 logic dmem_req;
542 logic dmem_gnt;
543 logic dmem_write;
544 logic [DmemIndexWidth-1:0] dmem_index;
545 logic [ExtWLEN-1:0] dmem_wdata;
546 logic [ExtWLEN-1:0] dmem_wmask;
547 logic [ExtWLEN-1:0] dmem_rdata;
548 logic dmem_rvalid;
549 logic [BaseWordsPerWLEN*2-1:0] dmem_rerror_vec;
550 logic dmem_rerror;
551 logic dmem_illegal_bus_access;
552 logic dmem_missed_gnt;
553
554 logic dmem_req_core;
555 logic dmem_write_core;
556 logic [DmemIndexWidth-1:0] dmem_index_core;
557 logic [ExtWLEN-1:0] dmem_wdata_core;
558 logic [ExtWLEN-1:0] dmem_wmask_core;
559 logic [BaseWordsPerWLEN-1:0] dmem_rmask_core_q,
dmem_rmask_core_d;
560 logic [ExtWLEN-1:0] dmem_rdata_core;
561 logic dmem_rvalid_core;
562 logic dmem_rerror_core;
563
564 logic dmem_req_bus;
565 logic dmem_dummy_response_q, dmem_dummy_response_d;
566 logic dmem_write_bus;
567 logic [DmemBusIndexWidth-1:0] dmem_index_bus;
568 logic [ExtWLEN-1:0] dmem_wdata_bus;
569 logic [ExtWLEN-1:0] dmem_wmask_bus;
570 logic [ExtWLEN-1:0] dmem_rdata_bus, dmem_rdata_bus_raw;
571 logic dmem_rdata_bus_en_q, dmem_rdata_bus_en_d;
572 logic [DmemBusAddrWidth-1:0] dmem_addr_bus;
573 logic unused_dmem_addr_bus;
574 logic [31:0] dmem_wdata_narrow_bus;
575 logic [top_pkg::TL_DBW-1:0] dmem_byte_mask_bus;
576 logic dmem_rvalid_bus;
577 logic dmem_wr_collision;
578 logic dmem_wpending;
579 logic [1:0] dmem_rerror_bus;
580
581 logic dmem_bus_intg_violation;
582
583 logic [DmemAddrWidth-1:0] dmem_addr_core;
584 assign dmem_index_core = dmem_ad-
dr_core[DmemAddrWidth-1:DmemAddrWidth-DmemIndexWidth];
585
586 logic unused_dmem_addr_core_wordbits;
587 assign unused_dmem_addr_core_wordbits = ^dmem_ad-
dr_core[DmemAddrWidth-DmemIndexWidth-1:0];
588
589 // SEC_CM: MEM.SCRAMBLE
590 prim_ram_ip_scr #(
591   .Width      (ExtWLEN),
592   .Depth      (DmemSizeWords),
593   .InstDepth  (DmemSizeWords),
594   .DataBitsPerMask (39),
595   .EnableParity (0),
596   .ReplicateKeyStream(1)
597 ) u_dmem (
598   .clk_i,
599   .rst_ni(rst_n),
600
601   .key_valid_i(otbn_dmem_scramble_valid),
602   .key_i      (otbn_dmem_scramble_key),
603   .nonce_i    (otbn_dmem_scramble_nonce),
604
605   .req_i      (dmem_req),
606   .gnt_o      (dmem_gnt),
607   .write_i    (dmem_write),
608   .addr_i     (dmem_index),
609   .wdata_i    (dmem_wdata),
610   .wmask_i    (dmem_wmask),
611   .intg_error_i(locking),
612
613   .rdata_o    (dmem_rdata),
614   .rvalid_o   (dmem_rvalid),
615   .raddr_o    (),
616   .rerror_o   (),
617   .cfg_i      (ram_cfg_dmem_i),
618   .cfg_o      (ram_cfg_dmem_o),
619
620   .wr_collision_o (dmem_wr_collision),
621   .write_pending_o (dmem_wpending),
622
623   .alert_o ()
624 );
625
626 // We should never see a request that doesn't get granted. A
fatal error is raised if this occurs.
627 assign dmem_missed_gnt = dmem_req & !dmem_gnt;
628
629 always_ff @(posedge clk_i or negedge rst_ni) begin
630   if (!rst_ni) begin
631     dmem_rmask_core_q <= '0;
632   end else begin
633     if (dmem_req_core) begin

```

```

634     dmem_rmask_core_q <= dmem_rmask_core_d;
635   end
636 end
637 end
638
639 // SEC_CM: DATA.MEM.INTEGRITY
640 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; ++i_word)
begin : g_dmem_intg_check
641   logic [1:0] dmem_rerror_raw;
642
643   // Separate check for dmem read data integrity outside of
'u_dmem' as 'prim_ram_ip_adv' doesn't
644   // have functionality for only integrity checking, just
fully integrated ECC. Integrity bits are
645   // implemented on a 32-bit granule so separate checks are
required for each.
646   prim_secdec_inv_39_32_dec u_dmem_intg_check (
647     .data_i    (dmem_rdata[i_word*39+:39]),
648     .data_o    (),
649     .syndrome_o(),
650     .err_o     (dmem_rerror_raw)
651   );
652
653   // Only report an error where the word was actually ac-
cessed. Otherwise uninitialised memory
654   // that OTBN isn't using will cause false errors.
dmem_rerror is only reported for reads from
655
656   // OTBN. For Ibex reads integrity checking on TL responses
will serve the same purpose.
657   assign dmem_rerror_vec[i_word*2+:2] = dmem_rerror_raw &
{2{dmem_rmask_core_q[i_word] & dmem_rvalid & dmem_ac-
cess_core}};
658 end
659
660 // dmem_rerror_vec is 2 bits wide and is used to report ECC
errors. Bit 1 is set if there's an
661 // uncorrectable error and bit 0 is set if there's a cor-
rectable error. However, we're treating
662 // all errors as fatal, so OR the two signals together.
663 assign dmem_rerror = |dmem_rerror_vec;
664
665 // DMEM access from main TL-UL bus
666 logic dmem_gnt_bus;
667 // Always grant to bus accesses, when OTBN is running a
dummy response is returned
668 assign dmem_gnt_bus = dmem_req_bus;
669
670 tlul_adapter_sram #(
671   .SramAw      (DmemBusIndexWidth),
672   .SramDw      (WLEN),
673   .Outstanding (1),
674   .ByteAccess  (0),
675   .ErrOnRead   (0),
676   .EnableDataIntgPt(1),
677   .SecFifoPtr  (1) // SEC_CM: TLUL_FIFO.CTR.REDUN
678 ) u_tlul_adapter_sram_dmem (
679   .clk_i,
680   .rst_ni      (rst_n),
681   .tl_i        (tl_win_h2d[TLWinDmem]),
682   .tl_o        (tl_win_d2h[TLWinDmem]),
683   .en_ifetch_i (MuBi4False),
684   .req_o       (dmem_req_bus),
685   .req_type_o  (),
686   .gnt_i       (dmem_gnt_bus),
687   .we_o        (dmem_write_bus),
688   .addr_o      (dmem_index_bus),
689   .wdata_o     (dmem_wdata_bus),
690   .wmask_o     (dmem_wmask_bus),
691   .intg_error_o (dmem_bus_intg_violation),
692   .user_rsvd_o (),
693   .rdata_i     (dmem_rdata_bus),
694   .rvalid_i    (dmem_rvalid_bus),
695   .rerror_i    (dmem_rerror_bus),
696   .compound_txn_in_progress_o (),
697   .readback_en_i (prim_mubi_pkg::MuBi4False),
698   .readback_error_o (),
699   .wr_collision_i (dmem_wr_collision),
700   .write_pending_i (dmem_wpending)
701 );
702
703 // Mux core and bus access into dmem. While paused, busy_ex-
ecute_q is low so the DMEM is
704 // accessible to the bus.
705 assign dmem_access_core = busy_execute_q;
706
707 assign dmem_req = dmem_access_core ? dmem_req_core :
dmem_req_bus;
708 assign dmem_write = dmem_access_core ? dmem_write_core :
dmem_write_bus;
709 assign dmem_wmask = dmem_access_core ? dmem_wmask_core :
dmem_wmask_bus;
710 // SEC_CM: DATA.MEM.SW_NOACCESS
711 assign dmem_index = dmem_access_core ? dmem_index_core :
712 {{(DmemIndexWidth-DmemBusIndexWidth){1'b0}}, {dmem_in-
dex_bus}};
713 assign dmem_wdata = dmem_access_core ? dmem_wdata_core :
dmem_wdata_bus;
714
715 assign dmem_illegal_bus_access = dmem_req_bus & dmem_ac-

```



```

cess_core;
716
717 assign dmem_dummy_response_d = dmem_illegal_bus_access;
718 always_ff @(posedge clk_i or negedge rst_ni) begin
719   if (!rst_ni) begin
720     dmem_dummy_response_q <= 1'b0;
721   end else begin
722     dmem_dummy_response_q <= dmem_dummy_response_d;
723   end
724 end
725
726 // SEC_CM: DATA_REG_SW.SCA
727 // Blank bus read data interface during core operation to
avoid leaking DMEM data through the bus
728 // unintentionally. Also blank when OTBN is returning a
dummy response (responding to an illegal
729 // bus access) and when OTBN is locked.
730 assign dmem_rdata_bus_en_d = ~(busy_execute_d | start_d) &
~dmem_dummy_response_d & ~locking;
731
732 always_ff @(posedge clk_i or negedge rst_ni) begin
733   if (!rst_ni) begin
734     dmem_rdata_bus_en_q <= 1'b1;
735   end else begin
736     dmem_rdata_bus_en_q <= dmem_rdata_bus_en_d;
737   end
738 end
739
740 prim_blanker #(Width(ExtWLEN)) u_dmem_rdata_bus_blanker (
741   .in_i (dmem_rdata),
742   .en_i (dmem_rdata_bus_en_q),
743   .out_o(dmem_rdata_bus_raw)
744 );
745
746 // When OTBN is locked all dmem bus reads should return 0.
The blanker produces the 0s, this adds
747 // the appropriate ECC. When OTBN is not locked the output
of the blanker is passed straight
748 // through. Data bits are always left un-modified. A re-
gistered version of `locking` is used for
749 // timing reasons. When a read comes in when `locking` has
just been asserted, `locking_q` will be
750 // timing reasons. When a read comes in when `locking` has
just been asserted, `locking_q` will be
751 // set the following cycle and the rdata will be forced to 0
with appropriate ECC. When `locking`
752 // is asserted the cycle the rdata is being returned no
locking was occurring when the request
753 // came in so it is reasonable to proceed with returning the
supplied integrity.
754 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; ++i_word)
begin : g_dmem_rdata_bus
755   assign dmem_rdata_bus[i_word*39+39] =
756     {locking_q ? prim_secdd_pkg::SecddInv3932ZeroEcc :
dmem_rdata_bus_raw[i_word*39+32+7],
757     dmem_rdata_bus_raw[i_word*39+32]};
758 end
759
760 `ASSERT(DmemRDataBusDisabledWhenCoreAccess_A, dmem_ac-
cess_core |> !dmem_rdata_bus_en_q)
761 `ASSERT(DmemRDataBusEnabledWhenIdle_A, status_q ==
StatusIdle |> dmem_rdata_bus_en_q)
762 `ASSERT(DmemRDataBusDisabledWhenLocked_A, locking |> !
dmem_rdata_bus_en_q)
763 `ASSERT(DmemRDataBusReadAsZeroWhenLocked_A,
dmem_rvalid_bus & locking |> dmem_rdata_bus_raw == '0)
764
765 assign dmem_rdata_core = dmem_rdata;
766
767
768 // When an illegal bus access is seen, always return a dummy
response the follow cycle.
769 assign dmem_rvalid_bus = (~dmem_access_core & dmem_rvalid)
| dmem_dummy_response_q;
770 assign dmem_rvalid_core = dmem_access_core ? dmem_rvalid :
1'b0;
771
772 // No dmem errors reported for bus reads. Integrity is car-
ried through on the bus so integrity
773 // checking on TL responses will pick up any errors.
774 assign dmem_error_bus = 2'b00;
775 assign dmem_error_core = dmem_error;
776
777 assign dmem_addr_bus = tl_win_h2d[TLWinD-
mem].a_address[DmemBusAddrWidth-1:0];
778 assign dmem_wdata_narrow_bus = tl_win_h2d[TLWinD-
mem].a_data[31:0];
779 assign dmem_byte_mask_bus = tl_win_h2d[TLWinDmem].a_mask;
780
781 // Memory Load Integrity
=====
782 // CRC logic below assumes an incoming data bus width of 32
bits and considers
783 // 15 bits of the 32-bit word based addresses.
784 `ASSERT_INIT(TLDWIs32Bit_A, top_pkg::TL_DW == 32)
785 `ASSERT_INIT(ImemAddrTooWideForLoadCrc_A, ImemIndexWidth <=
15)
786 `ASSERT_INIT(DmemBusAddrTooWideForLoadCrc_A, (DmemBusAd-

```

```

drwidth - 2) <= 15)
787
788 // Only advance CRC calculation on full 32-bit writes;
789 assign mem_crc_data_in_valid = ~(dmem_access_core |
imem_access_core) &
790   ((imem_req_bus & (imem_byte_mask_bus == 4'hf)) |
791    (dmem_req_bus & (dmem_byte_mask_bus == 4'hf)));
792
793 assign mem_crc_data_in.wr_data = imem_req_bus ?
imem_wdata_bus[31:0] :
794   dmem_wdata_narrow_bus[31:0];
795
796 // The CRC operates on the 32 bits from the bus. For IMEM we
can take the index. But for DMEM
797 // we must take the relevant part of dmem_addr_bus because
dmem_index_bus indexes 256-bit words.
798 assign mem_crc_data_in.index = imem_req_bus ? {{15 - Ime-
mIndexWidth{1'b0}}, imem_index_bus} :
799   {{15 -
(DmemBusAddrWidth - 2){1'b0}},
800   dmem_ad-
dr_bus[DmemBusAddrWidth-1:2]};
801 assign mem_crc_data_in.imem = imem_req_bus;
802
803 // Only the bits that factor into the dmem index and dmem
word enables are required
804 assign unused_dmem_addr_bus = ^dmem_addr_bus[1:0];
805
806 // SEC_CM: WRITE.MEM.INTEGRITY
807 prim_crc32 #(
808   .BytesPerWord(6)
809   ) u_mem_load_crc32 (
810     .clk_i (clk_i),
811     .rst_ni(rst_ni),
812     .set_crc_i(set_crc),
813     .crc_in_i (crc_in),
814     .data_valid_i(mem_crc_data_in_valid),
815     .data_i (mem_crc_data_in),
816     .crc_out_o (crc_out)
817 );
818
819 assign set_crc = reg2hw.load_checksum.q;
820 assign crc_in = reg2hw.load_checksum.q;
821 assign hw2reg.load_checksum.d = crc_out;
822
823 // Registers
=====
824
825 logic reg_bus_intg_violation;
826
827
828 otbn_reg_top u_reg (
829   .clk_i,
830   .rst_ni (rst_n),
831   .tl_i,
832   .tl_o,
833   .tl_win_o(tl_win_h2d),
834   .tl_win_i(tl_win_d2h),
835   .reg2hw,
836   .hw2reg,
837   .intg_err_o(reg_bus_intg_violation)
838 );
839
840 // SEC_CM: BUS.INTEGRITY
841 // SEC_CM: TLUL_FIFO.CTR.REDUN
842 logic bus_intg_violation;
843 assign bus_intg_violation = (imem_bus_intg_violation |
dmem_bus_intg_violation |
844   reg_bus_intg_violation);
845
846 // CMD register
847 always_comb begin
848
849 // start and resume are flopped to avoid long timing paths
from the TL fabric into OTBN
850 // internals.
851 start_d = 1'b0;
852 wfi_resume_d = 1'b0;
853 dmem_sec_wipe = 1'b0;
854 imem_sec_wipe = 1'b0;
855
856 // Can only start a new command when idle.
857 if (status_q == StatusIdle) begin
858   if (reg2hw.cmd.q) begin
859     unique case (reg2hw.cmd.q)
860       CmdExecute: start_d = 1'b1;
861       CmdSecWipeDmem: dmem_sec_wipe = 1'b1;
862       CmdSecWipeImem: imem_sec_wipe = 1'b1;
863       default: ;
864     endcase
865   end
866 end else if (busy_execute_q || status_q == StatusPaused)
begin
867
868 // OTBN can command a secure wipe of IMEM and DMEM. This
occurs when OTBN encounters a fatal
869 // error.

```

```

870     if (mems_sec_wipe) begin
871         dmem_sec_wipe = 1'b1;
872         imem_sec_wipe = 1'b1;
873     end
874     // Resume execution once the cmd arrives.
875     if (status_q == StatusPaused) begin
876         if (reg2hw.cmd.qe && (reg2hw.cmd.q == CmdResume)) be-
gin
877             wfi_resume_d = 1'b1;
878         end
879     end
880 end
881 end
882
883 assign req_sec_wipe_urnd_keys = dmem_sec_wipe |
imem_sec_wipe;
884
885 assign illegal_bus_access_d = dmem_illegal_bus_access |
imem_illegal_bus_access;
886
887 // It should not be possible to request an imem or dmem ac-
cess without it being granted. Either
888 // a scramble key is present so the request will be granted
or the core is busy obtaining a new
889
889 // key, so no request can occur (the core won't generate one
whilst awaiting a scrambling key and
890 // the bus requests get an immediate dummy response by-
passing the dmem or imem). A fatal error is
891 // raised if request is seen without a grant.
892 assign missed_gnt_error_d = dmem_missed_gnt |
imem_missed_gnt;
893
894 // Flop `illegal_bus_access_q` and `missed_gnt_error_q` to
break timing paths from the TL
895 // interface into the OTBN core.
896 always_ff @(posedge clk_i or negedge rst_ni) begin
897     if (!rst_ni) begin
898         start_q <= 1'b0;
899         illegal_bus_access_q <= 1'b0;
900         missed_gnt_error_q <= 1'b0;
901     end else begin
902         start_q <= start_d;
903         illegal_bus_access_q <= illegal_bus_access_d;
904         missed_gnt_error_q <= missed_gnt_error_d;
905     end
906 end
907
908 // STATUS register
909 // imem/dmem scramble req can be busy when locked, so use a
priority selection so locked status
910 // always takes priority.
911 //
912 // Note that these signals are all "a cycle early". For ex-
ample, the locking signal gets asserted
913 // combinatorially on the cycle that an error is injected.
The STATUS register change, done
914 // interrupt and any change to the idle signal will be
delayed by 2 cycles.
915 assign status_d = locking ? Status-
Locked :
916     busy_secure_wipe ? Status-
BusySecWipeInt :
917     wfi_pending & ~wfi_resume_q ? Status-
Paused :
918     busy_execute_d ? Status-
BusyExecute :
919     otbn_dmem_scramble_key_req_busy ? Status-
BusySecWipeDmem :
920     otbn_imem_scramble_key_req_busy ? Status-
BusySecWipeImem :
921     StatusIdle;
922 always_ff @(posedge clk_i or negedge rst_ni) begin
923     if (!rst_ni) begin
924         status_q <= StatusBusySecWipeInt;
925     end else begin
926         status_q <= status_d;
927     end
928 end
929
930 assign hw2reg.status.d = status_q;
931 assign hw2reg.status.de = 1'b1;
932
933 // Only certain combinations of the state variable {locking,
busy_execute_d,
934 // otbn_dmem_scramble_key_req_busy,
otbn_imem_scramble_key_req_busy} are possible.
935 //
936 // - Once locking is high, we guarantee never to see a new
execution or the start of a key
937 // rotation. (Assertion: NoStartWhenLocked_A)
938
939 `ASSERT(NoStartWhenLocked_A,
940     locking |>= !($rose(busy_execute_d) ||
941         $rose(otbn_dmem_scramble_key_req_busy)
||
942         $rose(otbn_imem_scramble_key_req_busy)))

```

```

943
944 // CTRL register
945 assign software_errs_fatal_d =
946     reg2hw.ctrl.software_errs_fatal.qe && (status_q ==
StatusIdle) ?
947     reg2hw.ctrl.software_errs_fatal.q : soft-
ware_errs_fatal_q;
948
949 assign wfi_enabled_d =
950     reg2hw.ctrl.wfi_enabled.qe && (status_q == StatusIdle) ?
951     reg2hw.ctrl.wfi_enabled.q : wfi_enabled_q;
952
953 // The URND control enable bit must be stable during an OTBN
execution because it is used to
954 // control a blanker inside otbn_rnd.sv
955 assign urnd_ctrl_enabled_d =
956     reg2hw.ctrl.urnd_ctrl_enabled.qe && (status_q ==
StatusIdle) ?
957     reg2hw.ctrl.urnd_ctrl_enabled.q : urnd_ctrl_enabled_q;
958
959 always_ff @(posedge clk_i or negedge rst_ni) begin
960     if (!rst_ni) begin
961         software_errs_fatal_q <= 1'b0;
962         wfi_enabled_q <= 1'b0;
963         urnd_ctrl_enabled_q <= 1'b0;
964     end else begin
965         software_errs_fatal_q <= software_errs_fatal_d;
966         wfi_enabled_q <= wfi_enabled_d;
967         urnd_ctrl_enabled_q <= urnd_ctrl_enabled_d;
968     end
969 end
970
971 assign hw2reg.ctrl.software_errs_fatal.d = soft-
ware_errs_fatal_q;
972 assign hw2reg.ctrl.wfi_enabled.d = wfi_enabled_q;
973 assign hw2reg.ctrl.urnd_ctrl_enabled.d = urnd_c-
trl_enabled_q;
974
975 // ERR_BITS register
976 // The error bits for an OTBN operation get stored on the
cycle that done is
977 // asserted. Software is expected to read them out before
starting the next operation.
978
979 assign hw2reg.err_bits.bad_data_addr.d =
err_bits_q.bad_data_addr;
980 assign hw2reg.err_bits.bad_insn_addr.d =
err_bits_q.bad_insn_addr;
981 assign hw2reg.err_bits.call_stack.d = err_bits_q.call_stack;
982 assign hw2reg.err_bits.illegal_insn.d =
err_bits_q.illegal_insn;
983 assign hw2reg.err_bits.loop.d = err_bits_q.loop;
984 assign hw2reg.err_bits.key_invalid.d =
err_bits_q.key_invalid;
985 assign hw2reg.err_bits.rnd_rep_chk_fail.d =
err_bits_q.rnd_rep_chk_fail;
986 assign hw2reg.err_bits.rnd_fips_chk_fail.d =
err_bits_q.rnd_fips_chk_fail;
987 assign hw2reg.err_bits.imem_intg_violation.d =
err_bits_q.imem_intg_violation;
988 assign hw2reg.err_bits.dmem_intg_violation.d =
err_bits_q.dmem_intg_violation;
989 assign hw2reg.err_bits.reg_intg_violation.d =
err_bits_q.reg_intg_violation;
990 assign hw2reg.err_bits.bus_intg_violation.d =
err_bits_q.bus_intg_violation;
991 assign hw2reg.err_bits.bad_internal_state.d =
err_bits_q.bad_internal_state;
992 assign hw2reg.err_bits.illegal_bus_access.d =
err_bits_q.illegal_bus_access;
993 assign hw2reg.err_bits.lifecycle_escalation.d =
err_bits_q.lifecycle_escalation;
994 assign hw2reg.err_bits.fatal_software.d =
err_bits_q.fatal_software;
995 assign hw2reg.err_bits.mai_software_error.d =
err_bits_q.mai_error;
996
997 assign err_bits_clear = reg2hw.err_bits.bad_data_addr.qe &
is_not_running_q;
998 assign err_bits_d = err_bits_clear ? '0 : err_bits;
999 assign err_bits_en = err_bits_clear | done_core;
1000
1001 logic unused_reg2hw_err_bits;
1002
1003 // Majority of reg2hw.err_bits is unused as write values are
ignored, all writes clear the
1004 // register to 0.
1005 assign unused_reg2hw_err_bits =
^ {reg2hw.err_bits.bad_data_addr.q,
1006     reg2hw.err_bits.bad_insn_addr,
1007     reg2hw.err_bits.call_stack,
1008     reg2hw.err_bits.illegal_insn,
1009     reg2hw.err_bits.loop,
1010     reg2hw.err_bits.key_invalid,
1011

```

```

reg2hw.err_bits.rnd_rep_chk_fail,
1012
reg2hw.err_bits.rnd_fips_chk_fail,
1013
reg2hw.err_bits.imem_intg_violation,
1014
reg2hw.err_bits.dmem_intg_violation,
1015
reg2hw.err_bits.reg_intg_violation,
1016
reg2hw.err_bits.bus_intg_violation,
1017
reg2hw.err_bits.bad_internal_state,
1018
reg2hw.err_bits.illegal_bus_access,
1019
reg2hw.err_bits.lifecycle_escalation,
1020
reg2hw.err_bits.fatal_software,
1021
reg2hw.err_bits.mai_software_error};
1022
1023 always_ff @(posedge clk_i or negedge rst_ni) begin
1024   if (!rst_ni) begin
1025     err_bits_q <= '0;
1026   end else if (err_bits_en) begin
1027     err_bits_q <= err_bits_d;
1028   end
1029 end
1030
1031 // Latch the recoverable error signal from the core. This
will be generated as a pulse some time

1032 // during the run (and before secure wipe finishes). Collect
up this bit, clearing on the start or
1033 // end of an operation (start_q / done_core, respectively)
1034 assign recoverable_err_d = (recoverable_err_q | core_recov-
erable_err) & ~(start_q | done_core);
1035 always_ff @(posedge clk_i or negedge rst_ni) begin
1036   if (!rst_ni) begin
1037     recoverable_err_q <= '0;
1038   end else begin
1039     recoverable_err_q <= recoverable_err_d;
1040   end
1041 end
1042
1043 // FATAL_ALERT_CAUSE register. The .de and .d values are
equal for each bit, so that it can only
1044 // be set, not cleared.
1045 `define
DEF_FAC_BIT(NAME)
1046   assign hw2reg.fatal_alert_cause.``NAME``.d = \
1'b1;
1047   assign hw2reg.fatal_alert_cause.``NAME``.de =
err_bits.``NAME``;
1048
1049 `DEF_FAC_BIT(fatal_software)
1050 `DEF_FAC_BIT(lifecycle_escalation)
1051 `DEF_FAC_BIT(illegal_bus_access)
1052 `DEF_FAC_BIT(bad_internal_state)
1053 `DEF_FAC_BIT(bus_intg_violation)
1054 `DEF_FAC_BIT(reg_intg_violation)
1055 `DEF_FAC_BIT(dmem_intg_violation)
1056 `DEF_FAC_BIT(imem_intg_violation)
1057
1058 `undef DEF_FAC_BIT
1059
1060 // INSN_CNT register
1061 logic [31:0] insn_cnt;
1062 logic insn_cnt_clear;
1063 logic unused_insn_cnt_q;
1064 assign hw2reg.insn_cnt.d = insn_cnt;
1065 assign insn_cnt_clear = reg2hw.insn_cnt.qe & is_not_run-
ning_q;
1066 // Ignore all write data to insn_cnt. All writes zero the
register.
1067 assign unused_insn_cnt_q = ^reg2hw.insn_cnt.q;
1068
1069 // Alerts
=====
1070
1071 logic [NumAlerts-1:0] alert_test;
1072 assign alert_test[AlertFatal] = reg2hw.alert_test.fatal.q &
reg2hw.alert_test.fatal.qe;
1073 assign alert_test[AlertRecov] = reg2hw.alert_test.recov.q &
reg2hw.alert_test.recov.qe;
1074
1075 logic [NumAlerts-1:0] alerts;
1076 assign alerts[AlertFatal] = {err_bits.fatal_software,
1077   err_bits.lifecycle_escalation,
1078   err_bits.illegal_bus_access,
1079   err_bits.bad_internal_state,
1080   err_bits.bus_intg_violation,
1081   err_bits.reg_intg_violation,
1082   err_bits.dmem_intg_violation,
1083   err_bits.imem_intg_violation};
1084
1085 assign alerts[AlertRecov] = (core_recoverable_err | recover-
able_err_q) & done_core;
1086
1087 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx

```

```

1088   prim_alert_sender #(
1089     .AsyncOn(AlertAsyncOn[i]),
1090     .SkewCycles(AlertSkewCycles),
1091     .IsFatal(i == AlertFatal)
1092   ) u_prim_alert_sender (
1093     .clk_i,
1094     .rst_ni (rst_n),
1095     .alert_test_i (alert_test[i]),
1096     .alert_req_i (alerts[i]),
1097     .alert_ack_o (),
1098     .alert_state_o(),
1099     .alert_rx_i (alert_rx_i[i]),
1100     .alert_tx_o (alert_tx_o[i])
1101   );
1102 end
1103
1104 // EDN Connections
=====
1105
1106 logic edn_rnd_req, edn_rnd_ack;
1107 logic [EdnDataWidth-1:0] edn_rnd_data;
1108 logic edn_rnd_fips, edn_rnd_err;
1109
1110 // This EDN request module synchronizes the data coming from
EDN and stack the 32 bit EDN words
1111 // to achieve an internal entropy width of 256 bit.
1112 prim_edn_req #(
1113   .EnRstChks(1'b1),
1114   .Outwidth(EdnDataWidth),
1115   // SEC_CM: RND.BUS.CONSTENCY
1116   .RepCheck(1'b1)
1117 ) u_prim_edn_req (
1118   .clk_i,
1119   .rst_ni (rst_n),
1120   .req_chk_i (1'b1),
1121   .req_i (edn_rnd_req),
1122   .ack_o (edn_rnd_ack),
1123   .data_o (edn_rnd_data),
1124   .fips_o (edn_rnd_fips),
1125   .err_o (edn_rnd_err),
1126   .clk_edn_i,
1127   .rst_edn_ni,
1128   .edn_o (edn_rnd_o),
1129   .edn_i (edn_rnd_i)
1130 );
1131
1132 edn_pkg::edn_req_t edn_urnd_req;
1133 edn_pkg::edn_rsp_t edn_urnd_rsp;
1134
1135 // Synchronize the data from the EDN network to the main
clock of OTBN.
1136 prim_sync_reqack_data #(
1137   .Width(edn_pkg::ENDPOINT_BUS_WIDTH + 32'd1),
1138   .EnRstChks(1'b1),
1139   .DataSrc2Dst(1'b0),
1140   .DataReg(1'b0)
1141 ) u_prim_sync_reqack_data_urnd (
1142   .clk_src_i (clk_i),
1143   .rst_src_ni (rst_n),
1144   .clk_dst_i (clk_edn_i),
1145   .rst_dst_ni (rst_edn_ni),
1146   .req_chk_i (1'b1),
1147   .src_req_i (edn_urnd_req.edn_req),
1148   .src_ack_o (edn_urnd_rsp.edn_ack),
1149   .dst_req_o (edn_urnd_o.edn_req),
1150   .dst_ack_i (edn_urnd_i.edn_ack),
1151   .data_i ({edn_urnd_i.edn_fips,
edn_urnd_i.edn_bus}),
1152   .data_o ({edn_urnd_rsp.edn_fips, edn_urnd_rsp.edn_bus})
1153 );
1154
1155 // OTBN Core
=====
1156
1157 always_ff @(posedge clk_i or negedge rst_n) begin
1158   if (!rst_n) begin
1159     busy_execute_q <= 1'b0;
1160     wfi_resume_q <= 1'b0;
1161     init_sec_wipe_done_q <= 1'b0;
1162   end else begin
1163     busy_execute_q <= busy_execute_d;
1164     wfi_resume_q <= wfi_resume_d;
1165     init_sec_wipe_done_q <= init_sec_wipe_done_d;
1166   end
1167 end
1168 // OTBN is busy executing once started or resumed until it
is done or gets paused. The
1169 // wfi_pending signal has only effect if we haven't already
received the resume command.
1170 assign busy_execute_d = wfi_resume_d |
1171   ((busy_execute_q | start_d) &
1172     ~done_core | (wfi_pending &
~wfi_resume_q));
1173 assign init_sec_wipe_done_d = init_sec_wipe_done_q |
~wfi_secure_wipe;
1174
1175 otbn_core #(
1176   .RegFile(RegFile),
1177   .DmemSizeByte(DmemSizeByte),
1178   .ImemSizeByte(ImemSizeByte),

```

```

1179 .RndCnstUrndPrngSeed(RndCnstUrndPrngSeed),
1180 .RndCnstUrndPerm(RndCnstUrndPerm),
1181 .SecFixMaiOpSeq(SecFixMaiOpSeq),
1182 .SecFixMacOpSeq(SecFixMacOpSeq),
1183 .FeatStubMai(FeatStubMai),
1184 .SecSkipUrndReseedAtStart(SecSkipUrndReseedAtStart),
1185 .RndCnstBnMacUrndPerm(RndCnstBnMacUrndPerm),
1186 .RndCnstMaiUrndPerm(RndCnstMaiUrndPerm)
1187 ) u_otbn_core (
1188   .clk_i,
1189   .rst_ni                                (rst_n),
1190
1191   .start_i                                (start_q),
1192   .done_o                                (done_core),
1193   .locking_o                              (locking),
1194   .secure_wipe_running_o                 (busy_secure_wipe),
1195
1196   .err_bits_o                             (core_err_bits),
1197   .recoverable_err_o                     (core_recoverable_err),
1198
1199   .imem_req_o                             (imem_req_core),
1200   .imem_addr_o                           (imem_addr_core),
1201   .imem_rdata_i                          (imem_rdata_core),
1202   .imem_rvalid_i                         (imem_rvalid_core),
1203
1204   .dmem_req_o                             (dmem_req_core),
1205   .dmem_write_o                          (dmem_write_core),
1206   .dmem_addr_o                           (dmem_addr_core),
1207   .dmem_wdata_o                          (dmem_wdata_core),
1208   .dmem_wmask_o                          (dmem_wmask_core),
1209   .dmem_rmask_o                          (dmem_rmask_core_d),
1210   .dmem_rdata_i                          (dmem_rdata_core),
1211   .dmem_rvalid_i                         (dmem_rvalid_core),
1212   .dmem_rerror_i                         (dmem_rerror_core),
1213
1214   .edn_rnd_req_o                         (edn_rnd_req),
1215   .edn_rnd_ack_i                         (edn_rnd_ack),
1216   .edn_rnd_data_i                       (edn_rnd_data),
1217   .edn_rnd_fips_i                       (edn_rnd_fips),
1218   .edn_rnd_err_i                         (edn_rnd_err),
1219
1220   .edn_urnd_o                            (edn_urnd_req),
1221   .edn_urnd_i                            (edn_urnd_rsp),
1222
1223   .wfi_enabled_i                         (wfi_enabled_q),
1224   .wfi_pending_o                         (wfi_pending),
1225   .wfi_resume_i                          (wfi_resume_q),
1226
1227   .urnd_ctrl_enabled_i                  (urnd_ctrl_enabled_q),
1228
1229   .insn_cnt_o                           (insn_cnt),
1230   .insn_cnt_clear_i                     (insn_cnt_clear),
1231
1232   .mems_sec_wipe_o                       (mems_sec_wipe),
1233   .dmem_sec_wipe_urnd_key_o              (dmem_sec_wipe_urnd_key),
1234   .imem_sec_wipe_urnd_key_o              (imem_sec_wipe_urnd_key),
1235   .req_sec_wipe_urnd_keys_i              (req_sec_wipe_urnd_keys),
1236
1237   .escalate_en_i                         (core_escalate_en),
1238   .rma_req_i                             (mubi_rma_req),
1239   .rma_ack_o                             (mubi_rma_ack),
1240
1241   .software_errs_fatal_i                 (software_errs_fatal_q),
1242
1243   .sideload_key_shares_i                 (keymgr_key_i.key),
1244   .sideload_key_shares_valid_i           ({keymgr_key_i.valid}},
1245
1246   // The naming kmac_data is just to be consistent with oth-
1247   // er IPs connecting to KMAC. From here
1248   // on use more sensible name.
1249   .kmac_app_req_o(kmac_data_o),
1250   .kmac_app_rsp_i(kmac_data_i)
1251 );
1252 always_ff @(posedge clk_i or negedge rst_n) begin
1253   if (!rst_n) begin
1254     locking_q <= 1'b0;
1255   end else begin
1256     locking_q <= locking;
1257   end
1258 end
1259
1260 // Collect up the error bits that don't come from the core
1261 // itself and latch them so that they'll
1262 // be available when an operation finishes.
1263 assign non_core_err_bits = '{
1264   lifecycle_escalation: lc_c-
1265   trl_pkg::lc_tx_test_true_loose(lc_escalate_en[0]),
1266   illegal_bus_access:   illegal_bus_access_q,
1267   bad_internal_state:   otbn_scramble_state_error |
1268   missed_gnt_error_q,
1269   bus_intg_violation:   bus_intg_violation
1270 };
1271 assign non_core_err_bits_d = non_core_err_bits_q |
1272 non_core_err_bits;
1273 always_ff @(posedge clk_i or negedge rst_ni) begin
1274   if (!rst_ni) begin
1275     non_core_err_bits_q <= '0;
1276   end else begin
1277     non_core_err_bits_q <= non_core_err_bits_d;
1278   end
1279 end
1280
1281 end
1282
1283 // Construct a full set of error bits from the core output
1284 assign err_bits = '{
1285   fatal_software:      core_err_bits.fatal_software,
1286   lifecycle_escalation: core_err_bits.fatal_software,
1287   non_core_err_bits_d.lifecycle_escalation,
1288   illegal_bus_access:  core_err_bits.fatal_software,
1289   non_core_err_bits_d.illegal_bus_access,
1290   bad_internal_state:  |{core_err_bits.bad_internal_state,
1291   non_core_err_bits_d.bad_internal_state},
1292   bus_intg_violation:  core_err_bits.dmem_intg_violation,
1293   non_core_err_bits_d.bus_intg_violation,
1294   reg_intg_violation:  core_err_bits.reg_intg_violation,
1295   dmem_intg_violation: core_err_bits.dmem_intg_violation,
1296   imem_intg_violation: core_err_bits.imem_intg_violation,
1297   rnd_fips_chk_fail:   core_err_bits.rnd_fips_chk_fail,
1298   rnd_rep_chk_fail:    core_err_bits.rnd_rep_chk_fail,
1299   key_invalid:         core_err_bits.key_invalid,
1300   loop:                core_err_bits.loop,
1301   illegal_insn:        core_err_bits.illegal_insn,
1302   call_stack:          core_err_bits.call_stack,
1303   bad_insn_addr:       core_err_bits.bad_insn_addr,
1304   bad_data_addr:       core_err_bits.bad_data_addr,
1305   mai_error:           core_err_bits.mai_error
1306 };
1307
1308 // An error signal going down into the core to show that it
1309 // should locally escalate. In
1310 // accordance with the lc_ctrl spec, all values of the
1311 // lc_escalate_en signal other than the OFF
1312 // value must be interpreted as ON.
1313 assign core_escalate_en = mubi4_bool_to_mubi(
1314   |{non_core_err_bits.illegal_bus_access,
1315   non_core_err_bits.bad_internal_state,
1316   non_core_err_bits.bus_intg_violation,
1317   lc_c-
1318   trl_pkg::lc_tx_test_true_loose(lc_escalate_en[1])});
1319
1320 // The core can never signal a write to IMEM
1321 assign imem_write_core = 1'b0;
1322
1323 // Asserts
1324 =====
1325 for (genvar i = 0; i < LoopStackDepth; ++i) begin :
1326   gen_loop_stack_ctr_asserts
1327   `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1328     LoopStackCntAlertCheck_A,
1329     u_otbn_core.u_otbn_controller.u_otbn_loop_controller.g_loop_counters
1330     [i].u_loop_count,
1331     alert_tx_o[AlertFatal]
1332   )
1333 end
1334
1335 // GPR assertions for secure wipe
1336 // 1. urnd_reseed_err disables the assertion because secure
1337 // wipe finishes with failure and OTBN
1338 // goes to LOCKED state immediately after this error which
1339 // means that it's not guaranteed to have
1340 // secure wiping complete.
1341
1342 // 2. mubi_err_d of start_stop_control disables the internal
1343 // secure wipe related assertion
1344 // because a fatal error affecting internal secure wiping
1345 // could cause an immediate locking
1346 // behaviour in which it's not guaranteed to see a success-
1347 // ful secure wipe.
1348 for (genvar i = 2; i < NGpr; ++i) begin :
1349   gen_sec_wipe_gpr_asserts
1350   // Initial secure wipe needs to initialise all registers
1351   // to nonzero
1352   `ASSERT(InitSecWipeNonZeroBaseRegs_A,
1353     $fell(busy_secure_wipe) |>
1354   )
1355   u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.g_rf_
1356   flops[i].rf_reg_q !=
1357   EccZeroWord,
1358   clk_i,
1359   !rst_ni || u_otbn_core.urnd_reseed_err ||
1360   u_otbn_core.u_otbn_start_stop_control.mubi_err_d ||
1361   u_otbn_core.u_otbn_start_stop_control.state_error_d)
1362   // After execution, it's expected to see a change result-
1363   // ing with a nonzero register value
1364   `ASSERT(SecWipeChangedBaseRegs_A,
1365     $rose(busy_secure_wipe) |> ((#[0:$]
1366   )
1367   u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.g_rf_
1368   flops[i].rf_reg_q !=
1369   EccZeroWord &&
1370   $changed(
1371   u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.g_rf_
1372   flops[i].rf_reg_q))
1373   within ($rose(busy_secure_wipe) #[0:$] $fell(busy_se-
1374   cure_wipe))),
1375   clk_i,
1376   !rst_ni || u_otbn_core.urnd_reseed_err ||

```

```

u_otbn_core.u_otbn_start_stop_control.mubi_err_d ||
1348     u_otbn_core.u_otbn_start_stop_control.state_error_d)
1349 end
1350
1351 // We have several assertions that check that secure wipe
worked properly. However, we've also got
1352 // some tests where we force nets, stopping it from working
properly! That's fine, and the tests
1353 // are checking that some other mechanism catches the prob-
lem. However, we don't want the
1354 // simulation to die with a failed assertion, so we put
everything in a named block which we can
1355 // turn off with $assertoff.
1356 //
1357 // The silly-looking name is to avoid a lint warning.
Verible (correctly) points out that
1358 // SystemVerilog doesn't allow bare begin/end blocks at mod-
ule level. So I cheated and put
1359 // everything in an if(1) block. But this is treated as a
generate block, and our lint rules
1360 // therefore expect its name to start with a "g.".
1361 if (1) begin : g_secure_wipe_assertions
1362 // WDR assertions for secure wipe
1363 // 1. urnd_reseed_err disables the assertion because se-
cure wipe finishes with failure and OTBN
1364 // goes to LOCKED state immediately after this error which
means that it's not guaranteed to
1365 // have secure wiping complete.
1366 // 2. mubi_err_d of start_stop_control disables the in-
ternal secure wipe related assertion
1367 // because a fatal error affecting internal secure wiping
could cause an immediate locking
1368 // behaviour in which it's not guaranteed to see a suc-
cessful secure wipe.
1369 for (genvar i = 0; i < NWdr; ++i) begin :
gen_sec_wipe_wdr_asserts
1370 // Initial secure wipe needs to initialise all registers
to nonzero
1371 `ASSERT(InitSecWipeNonZeroWideRegs_A,
1372     $fell(busy_secure_wipe) |>
1373 )
1374 u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.rf[i] !=
1375     EccWideZeroWord,
1376     clk_i,
1377     !rst_ni || u_otbn_core.urnd_reseed_err ||
1378 u_otbn_core.u_otbn_start_stop_control.mubi_err_d ||
1379 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1380 // After execution, it's expected to see a change res-
ulting with a nonzero register value
1381 `ASSERT(SecWipeChangedWideRegs_A,
1382     $rose(busy_secure_wipe) |> ((#[0:$]
1383 )
1384 u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.rf[i] !=
1385     EccWideZeroWord &&
1386     $changed(
1387 u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.rf[i]))
1388     within ($rose(busy_secure_wipe) #[0:$]
1389 $fell(busy_secure_wipe))),
1390     clk_i, !rst_ni || u_otbn_core.urnd_reseed_err ||
1391 u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1392 end
1393 // Secure wipe needs to invalidate call and loop stack, ini-
tialize MOD, ACC to nonzero and set
1394 // FLAGS CSR to zero
1395 // 1. urnd_reseed_err disables the assertion because secure
wipe finishes with failure and OTBN
1396 // goes to LOCKED state immediately after this error which
means that it's not guaranteed to have
1397 // secure wiping complete.
1398 // 2. mubi_err_d of start_stop_control disables the secure
wipe related assertions because a
1399 // fatal error affecting internal secure wiping could cause
an immediate locking behaviour
1400 // in which it's not guaranteed to see a successful secure
wipe.
1401 `ASSERT(SecWipeInvalidCallStack_A,
1402     $fell(busy_secure_wipe) |> (!
1403 u_otbn_core.u_otbn_rf_base.u_call_stack.top_valid_o),
1404     clk_i,
1405     !rst_ni || u_otbn_core.urnd_reseed_err ||
1406     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1407 ||
1408 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1409 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1410 `ASSERT(SecWipeInvalidLoopStack_A,
1411     $fell(busy_secure_wipe) |>
1412     (!

```

```

u_otbn_core.u_otbn_controller.u_otbn_loop_controller.loop_info_stack
.top_valid_o),
1411     clk_i,
1412     !rst_ni || u_otbn_core.urnd_reseed_err ||
1413     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1414 ||
1415 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1416 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1417 `ASSERT(SecWipeNonZeroMod_A,
1418     $fell(busy_secure_wipe) |>
1419 u_otbn_core.u_otbn_alu_bignum.mod_intg_q != EccWideZeroWord,
1420     clk_i,
1421     !rst_ni || u_otbn_core.urnd_reseed_err ||
1422     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1423 ||
1424 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1425 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1426 `ASSERT(SecWipeNonZeroACC_A,
1427     $fell(busy_secure_wipe) |>
1428     u_otbn_core.u_otbn_alu_bignum.ispr_acc_intg_i !=
EccWideZeroWord,
1429     clk_i,
1430     !rst_ni || u_otbn_core.urnd_reseed_err ||
1431     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1432 ||
1433 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1434 `ASSERT(SecWipeNonZeroFlags_A,
1435     $fell(busy_secure_wipe) |> (!
1436 u_otbn_core.u_otbn_alu_bignum.flags_flattened),
1437     clk_i,
1438     !rst_ni || u_otbn_core.urnd_reseed_err ||
1439     u_otbn_core.u_otbn_start_stop_control.mubi_err_d
1440 ||
1441 u_otbn_core.u_otbn_start_stop_control.state_error_d ||
1442 u_otbn_core.u_otbn_start_stop_control.state_error_d)
1443 // Secure wipe of IMEM and DMEM first happens with a key
change from URND (while valid is zero)
1444 `ASSERT(ImemSecWipeRequiresUrndKey_A,
1445     $rose(imem_sec_wipe) |> (otbn_imem_scramble_key ==
1446 $past(imem_sec_wipe_urnd_key)),
1447     clk_i,
1448     !rst_ni || u_otbn_core.urnd_reseed_err ||
1449     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1450 `ASSERT(DmemSecWipeRequiresUrndKey_A,
1451     $rose(dmem_sec_wipe) |> (otbn_dmem_scramble_key ==
1452 $past(dmem_sec_wipe_urnd_key)),
1453     clk_i,
1454     !rst_ni || u_otbn_core.urnd_reseed_err ||
1455     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1456 // Then it is guaranteed to have a valid key from OTP inter-
face which is different from URND key
1457 `ASSERT(ImemSecWipeRequiresOtpKey_A,
1458     $rose(imem_sec_wipe) ##1 (otbn_imem_scramble_key ==
1459 $past(imem_sec_wipe_urnd_key)) |>
1460     #[0:$] otbn_imem_scramble_valid &&
1461 $changed(otbn_imem_scramble_key),
1462     clk_i,
1463     !rst_ni || u_otbn_core.urnd_reseed_err ||
1464     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1465 `ASSERT(DmemSecWipeRequiresOtpKey_A,
1466     $rose(dmem_sec_wipe) ##1 (otbn_dmem_scramble_key ==
1467 $past(dmem_sec_wipe_urnd_key)) |>
1468     #[0:$] otbn_dmem_scramble_valid &&
1469 $changed(otbn_dmem_scramble_key),
1470     clk_i,
1471     !rst_ni || u_otbn_core.urnd_reseed_err ||
1472     u_otbn_core.u_otbn_start_stop_control.mubi_err_d)
1473 // All outputs should be known value after reset
1474 `ASSERT_KNOWN(Tl0ValidKnown_A, tl_o.d_valid)
1475 `ASSERT_KNOWN(Tl0AReadyKnown_A, tl_o.a_ready)
1476 `ASSERT_KNOWN(IdleOKKnown_A, idle_o)
1477 `ASSERT_KNOWN(IntrDoneOKKnown_A, intr_done_o)
1478 `ASSERT_KNOWN(AlertTxOKKnown_A, alert_tx_o)
1479 `ASSERT_KNOWN(LcRmaAckKnown_A, lc_rma_ack_o)
1480 `ASSERT_KNOWN(RamCfgDmemKnown_A, ram_cfg_dmem_o)
1481 `ASSERT_KNOWN(RamCfgImemKnown_A, ram_cfg_imem_o)
1482 `ASSERT_KNOWN(EdnRndOKKnown_A, edn_rnd_o, clk_edn_i, !
rst_edn_ni)
1483 `ASSERT_KNOWN(EdnUrndOKKnown_A, edn_urnd_o, clk_edn_i, !
rst_edn_ni)
1484 `ASSERT_KNOWN(otbnOtpKey0_A, otbn_otp_key_o, clk_otp_i, !
rst_otp_ni)
1485 `ASSERT_KNOWN(ErrBitsKnown_A, err_bits)
1486 // The data part of the request directly originates from
WSRs. These are non resettable flops.
1487 // When a simulation starts, these are still X as only a se-
cure wipe will set a value. We thus
1488 // only check whether the data is known when the valid is

```



```

set.
1483 `ASSERT_KNOWN(KmacReqKnown_A, {kmac_data_o.req_last,
kmac_data_o.req_valid,
1484                                     kmac_data_o.rsp_ready,
kmac_data_o.strb})
1485 `ASSERT_KNOWN_IF(KmacReqDataKnown_A, {kmac_data_o.data_s0,
kmac_data_o.data_s1},
1486                                     kmac_data_o.req_valid)
1487
1488 // Incoming key must be valid (other inputs go via prim mod-
ules that handle the X checks).
1489 `ASSERT_KNOWN(KeyMgrKeyValid_A, keymgr_key_i.valid)
1490
1491 // In locked state, the readable registers INSN_CNT, IMEM,
and DMEM are expected to always read 0
1492 // when accessed from the bus. For INSN_CNT, we use "|>=" so
that the assertion lines up with
1493 // "status.q" (a signal that isn't directly accessible
here).
1494 `ASSERT(LockedInsnCntReadsZero_A, (hw2reg.status.d ==
StatusLocked) |>= insn_cnt == 'd0)
1495 `ASSERT(ExecuteOrLockedImemReadsZero_A,
1496         (hw2reg.status.d inside {StatusBusyExecute, Status-
Locked}) & imem_rvalid_bus
1497         |>= imem_rdata_bus == 'd0)
1498 `ASSERT(ExecuteOrLockedDmemReadsZero_A,
1499         (hw2reg.status.d inside {StatusBusyExecute, Status-
Locked}) & dmem_rvalid_bus
1500         |>= dmem_rdata_bus == 'd0)
1501
1502 // While paused OTBN must not report idle, so the clock
keeps running to observe CMD.RESUME.
1503 `ASSERT(WfiPendingNotIdle_A, wfi_pending |>= (idle_o ==
prim_mubi_pkg::MuBi4False))
1504
1505 // While paused the DMEM is released to the bus until the
resume command arrived.
1506 `ASSERT(PausedDmemReleased_A, (status_q == StatusPaused)
&& !wfi_resume_q |>= !dmem_access_core)
1507
1508 // The paused status and the core's pending WFI cannot drift
apart except when a wipe starts on a
1509 // WFI instruction.
1510 `ASSERT(PausedImpliesWfiPending_A, (status_q == Status-
Paused) && !busy_secure_wipe
1511         |>= wfi_pending)
1512
1513 // OTBN can only be paused if the WFI instruction is en-
abled.
1514 `ASSERT(WfiPendingImpliesEnabled_A, wfi_pending |>= wfi_en-
abled_q)
1515
1516 // CTRL.wfi_enabled is only writable while idle, so it must
not change otherwise.
1517 `ASSERT(WfiEnabledStableUnlessIdle_A, (status_q !=
StatusIdle) |>= $stable(wfi_enabled_q))
1518
1519 // From the cycle the core is told to start to when it is
done, it must always be busy executing,
1520 // locking, or paused -- even if the core is never done. We
use this property to enable blanking
1521 // while the core is executing or locking, and this asser-
tion ensures that there is no gap
1522 // between execution and locking (except a valid pause).
1523 `ASSERT(BusyOrLockingFromStartToDone_A,
1524         $rose(start_q) |>= (busy_execute_d | locking |
wfi_pending) |>= ##[0:$] $rose(done_core))
1525
1526 // Error handling: if we pass an error signal down to the
core then we should also be setting an
1527 // error flag. Note that this uses err_bits, not err_bits_q,
because the latter signal only gets
1528 // asserted when an operation finishes.
1529 `ASSERT(ErrBitIfEscalate_A, mu-
bi4_test_true_loose(core_escalate_en) |>= |err_bits)
1530
1531 // Constraint from package, check here as we cannot have
`ASSERT_INIT in package
1532 `ASSERT_INIT(WsrSizeMatchesParameter_A, $bits(wsr_e) == Ws-
rNumWidth)
1533
1534 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1535     OtnbStartStopFsmCheck_A,
1536     u_otbn_core.u_otbn_start_stop_control.u_state_regs,
1537
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1538 )
1539 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1540     OtnbControllerFsmCheck_A,
1541     u_otbn_core.u_otbn_controller.u_state_regs,
1542
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1543 )
1544 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1545     OtnbScrambleCtrlFsmCheck_A,
1546     u_otbn_scramble_ctrl.u_state_regs,
1547
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1548 )
1549
1550 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT_IN(
1551     OtnbKmacFsmCheck_A,
1552     u_otbn_core.u_otbn_kmac_if.u_state_regs,
1553
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1554 )
1555
1556 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1557     OtnbCallStackWrPtrAlertCheck_A,
1558     u_otbn_core.u_otbn_rf_base.u_call_stack.u_stack_wr_ptr,
1559
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1560 )
1561
1562 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1563     OtnbLoopInfoStackWrPtrAlertCheck_A,
1564     u_otbn_core.u_otbn_controller.u_otbn_loop_controller.loop_info_stack
.u_stack_wr_ptr,
1565
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1566 )
1567
1568 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1569     OtnbKmacMsgSelectorCntAlertCheck_A,
1570     u_otbn_core.u_otbn_kmac_if.u_msg_selector_count,
1571
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1572 )
1573
1574 `ASSERT_ERROR_TRIGGER_ALERT_IN(
1575     OtnbBnMacCycleCountAlertCheck_A,
1576     u_otbn_core.u_otbn_mac_bignum.u_mac_bignum_fsm,
1577
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i,
1578     0, 2, state_err_o)
1579
1580 // Alert assertions for reg_we onehot check
1581 `ASSERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT_IN(
1582     RegWeOnehotCheck_A,
1583     u_reg,
1584
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1585 )
1586
1587 // other onehot checks
1588 `ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT_IN(
1589     RfBaseOnehotCheck_A,
1590
u_otbn_core.u_otbn_rf_base.gen_rf_base_ff.u_otbn_rf_base_inner.u_pr
im_onehot_check,
1591
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1592 )
1593
1594 `ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT_IN(
1595     RfBignumOnehotCheck_A,
1596
u_otbn_core.u_otbn_rf_bignum.gen_rf_bignum_ff.u_otbn_rf_bignum_inner
.u_prim_onehot_check,
1597
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1598 )
1599
1600 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1601     DmemRspFifo,
1602     u_tlu_adapter_sram_dmem.u_rspfifo,
1603
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1604 )
1605
1606 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1607     DmemSramReqFifo,
1608     u_tlu_adapter_sram_dmem.u_sramreqfifo,
1609
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1610 )
1611
1612 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1613     ImemRspFifo,
1614     u_tlu_adapter_sram_imem.u_rspfifo,
1615
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1616 )
1617
1618 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1619     ImemSramReqFifo,
1620     u_tlu_adapter_sram_imem.u_sramreqfifo,
1621
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1622 )
1623
1624 `ASSERT_PRIM_FIFO_SYNC_ERROR_TRIGGERS_ALERT1_IN(
1625     ImemReqFifo,
1626     u_tlu_adapter_sram_imem.u_reqfifo,
1627
gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1628 )
1629
1630 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(

```

```

1628     MaiMaskFifoWptrCheck_A,
1629 u_otbn_core.gen_mai.u_otbn_mai.u_otbn_mask_accelerator.u_prim_fifo_
sync_mask
1630     .gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_wptr,
1631 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1632 )
1633     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1634         MaiMaskFifoRptrCheck_A,
1635 u_otbn_core.gen_mai.u_otbn_mai.u_otbn_mask_accelerator.u_prim_fifo_
sync_mask
1636     .gen_normal_fifo.u_fifo_cnt.gen_secure_ptrs.u_rptr,
1637 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1638 )
1639     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1640         OtbmMaiMaskCntAlertCheck_A,
1641 u_otbn_core.gen_mai.u_otbn_mai.u_otbn_mask_accelerator.u_otbn_sec_a
dd_mod.u_prim_count_add_inp,
1642 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1643 )
1644     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1645         OtbmMaiInputCntAlertCheck_A,
1646 u_otbn_core.gen_mai.u_otbn_mai.u_prim_count_input_word_select,
1647 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1648 )
1649     ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT_IN(
1650         OtbmMaiOutputCntAlertCheck_A,
1651 u_otbn_core.gen_mai.u_otbn_mai.u_prim_count_output_word_select,
1652 gen_alert_tx[AlertFatalIdx].u_prim_alert_sender.alert_req_i
1653 )
1654 endmodule

```

2. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_base.sv (137 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * OTBN execute block for the base instruction subset
9  *
10  * This ALU supports the execution of all of OTBN's base in-
struction subset.
11  */
12 module otbn_alu_base
13     import otbn_pkg::*;
14 (
15     // Block is combinatorial; clk/rst are for assertions only.
16     input logic clk_i,
17     input logic rst_ni,
18
19     input alu_base_operation_t operation_i,
20     input alu_base_comparison_t comparison_i,
21
22     output logic [31:0] operation_result_o,
23     output logic      comparison_result_o
24 );
25
26     logic [32:0] adder_op_a, adder_op_b;
27     logic      adder_op_b_negate;
28     logic [32:0] adder_result;
29
30     logic [31:0] and_result;
31     logic [31:0] or_result;
32     logic [31:0] xor_result;
33     logic [31:0] not_result;
34
35     logic      is_equal;
36
37     // Adder //
38     // //
39     // //
40
41 // Adder takes in 33-bit operands. The addition of the input
operands occurs on bits [32:1],
42 // setting addr_op_b_negate will cause a carry-in into bit
1. Combined with an inversion of
43 // operation_i.operand_b this gives a two's-complement nega-
tion (~b + 1)
44
45     assign adder_op_b_negate = operation_i.op == AluOpBaseSub;
46
47     assign adder_op_a = {operation_i.operand_a, 1'b1};
48     assign adder_op_b = adder_op_b_negate ? {-opera-
tion_i.operand_b, 1'b1} :

```

```

49     { opera-
tion_i.operand_b, 1'b0};
50
51     assign adder_result = adder_op_a + adder_op_b;
52
53     // Bit-wise logical ops //
54     // //
55     // //
56
57     assign and_result = operation_i.operand_a & opera-
tion_i.operand_b;
58     assign or_result  = operation_i.operand_a | opera-
tion_i.operand_b;
59     assign xor_result = operation_i.operand_a ^ opera-
tion_i.operand_b;
60     assign not_result = ~operation_i.operand_a;
61
62     // Shifter //
63     // //
64
65     logic [32:0] shift_in;
66     logic [4:0] shift_amt;
67     logic [31:0] operand_a_reverse;
68     logic [32:0] shift_out;
69     logic [31:0] shift_out_reverse;
70
71     for (genvar i = 0; i < 32; i++) begin : g_shifter_reverses
72         assign operand_a_reverse[i] = operation_i.operand_a[31-i];
73         assign shift_out_reverse[i] = shift_out[31-i];
74     end
75
76     assign shift_amt = operation_i.operand_b[4:0];
77     // Shifter performs right arithmetic 33-bit shifts. Force
top bit to 0 to get logical shifting
78     // otherwise replicate top bit of shift_in. Left shifts per-
formed by reversing the input and
79     // output.
80     assign shift_in[31:0] = (operation_i.op == AluOpBaseSll) ?
operand_a_reverse :
81     operation_i.operand_a;
82     assign shift_in[32] = (operation_i.op == AluOpBaseSra) ? op-
eration_i.operand_a[31] : 1'b0;
83
84     logic signed [32:0] shift_in_signed;
85     assign shift_in_signed = signed'(shift_in);
86     assign shift_out = unsigned'(shift_in_signed >>> shift_amt);
87
88     // Output Mux //
89     // //
90     // //
91
92     always_comb begin
93         operation_result_o = adder_result[32:1];
94
95         unique case (operation_i.op)
96             AluOpBaseAnd: operation_result_o = and_result;
97             AluOpBaseOr:  operation_result_o = or_result;
98             AluOpBaseXor: operation_result_o = xor_result;
99             AluOpBaseNot: operation_result_o = not_result;
100             AluOpBaseSra: operation_result_o = shift_out[31:0];
101             AluOpBaseSrl: operation_result_o = shift_out[31:0];
102             AluOpBaseSll: operation_result_o = shift_out_reverse;
103             default: ;
104         endcase
105     end
106
107     // Comparisons //
108     // //
109     // //
110
111     // Dedicated comparator to deal with branches. As only
Equal/Not-Equal is required no need to use
112
113     // the adder (which frees it to compute the branch target in
the same cycle). No point in using
114
115     // existing XOR logic as area added from extra mux to choose
xor operands negates area saving from
116     // avoiding an dedicated comparator.
117     assign is_equal = comparison_i.operand_a == comparis-
on_i.operand_b;
118
119     assign comparison_result_o = (comparison_i.op == Comparis-
onOpBaseEq) ? is_equal : ~is_equal;
120
121     // The bottom bit of adder_result is discarded. It simply
corresponds to the carry in used to
122     // produce twos complement subtraction from an addition.
123     logic unused_adder_result_bit;
124
125     // The top bit of shift_out is discarded. shift_in contains
an extra bit to deal with sign
126     // extension which isn't needed in the shift_out result.
127     logic unused_shift_out_result_bit;
128     assign unused_shift_out_result_bit = shift_out[32];
129
130     assign unused_adder_result_bit = adder_result[0];
131
132     // clk_i, rst_ni are only used by assertions

```



```

132 logic unused_clk;
133 logic unused_rst_n;
134
135 assign unused_clk = clk_i;
136 assign unused_rst_n = rst_ni;
137 endmodule

```

3. lowRISC_opentitan/hw/ip/otbn/rtl/otbn_alu_bignum.sv

(1,612 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6 /**
7  * OTBN alu block for the bignum instruction subset
8  *
9  * This ALU supports all of the 'plain' arithmetic and logic
  bignum instructions and some
10  * vectorized arithmetic instructions. BN.MULQACC and its vec-
  torized counterparts are
11  * implemented in a separate block.
12  *
13  * One barrel shifter and two adders (X and Y) are implemented
  along with the logic operators
14  * (AND, OR, XOR, NOT), a vector transposer and 24-bit to 32-bit
  packing and unpacking logic.
15  *
16  * The adders have 256-bit operands with a carry_in and op-
  tional invert on the second operand. This
17  * can be used to implement subtraction (a - b == a + ~b + 1).
  BN.SUBB/BN.ADDC are implemented by
18  * feeding in the carry flag as carry in rather than a fixed 0 or
  1. These 256-bit operands can
19  * also be interpreted as vectors with 32-bit elements.
20  *
21  * The shifter takes a 512-bit input (to implement BN.RSHI,
  concatenate and right shift) and shifts
22  * right by up to 256-bits. The lower (256-bit) half of the
  input and output can be reversed to
23  * allow left shift implementation. There is no concatenate
  and left shift instruction so reversing
24  * isn't required over the full width.
25  *
26  * The dataflow between the adders and shifter is in the dia-
  gram below. This arrangement allows the
27  * implementation of the pseudo-mod (BN.ADDM(V)/BN.SUBM(V))
  instructions in a single cycle whilst
28  * minimising the critical path. The pseudo-mod instructions
  do not have a shifted input so X can
29  * compute the initial add/sub and Y computes the pseudo-mod
  result. The result is selected based
30  * on the carries of adder X and Y. This selection logic is
  implemented in the modulo result
31  * selector. For all other add/sub operations, Y computes the
  operation with one of the inputs
32  * supplied by the shifter and the other from operand_a. The
  result is then pushed through the
33  * modulo result selector. This allows to save one input to
  the final result selection mux.
34  *
35  * The shifter gets supplied with either operand_a and oper-
  and_b or a dense representation thereof
36  * to implement the packing functionality. In addition, the
  shifter gets a shift amount (shift_amt)
37  * and can use 0 instead of operand_a / its dense representa-
  tion. The shifter concatenates both
38  * inputs together before shifting with operand_a in the upper
  (256-bit) half (A/0, B). This
39  * allows the shifter to pass through operand_b simply by not
  performing a shift.
40  *
41  * The shifter result has no direct connection to the final
  result MUX but the result is pushed
42  * through the logic block with operand_a set to 0 and the lo-
  gic operation set to OR. This saves
43  * one input to the result selection mux.
44  *
45  * The vector packing and unpacking is implemented using the
  shifter's concatenation feature and
46  * some wiring.
47  *
48  * Blanking is employed on the ALU data paths. This holds un-
  used data paths to 0 to reduce side
49  * channel leakage. The lower-case 'b' on the diagram below
  indicates points in the data path that
50  * get blanked. Note that Adder X is never used in isolation,
  it is always combined with Adder Y so
51  * there is no need for blanking between Adder X and Adder Y.
52  *
53  * Due to this blanking only one of the possible results can
  be active at the same time. This
54  * allows to implement the final result selection with one

```

256-bit OR. This is more area as well

55 * as timing efficient compared to a regular 256-bit mux.

```

56 *
57 *
58 *
59 *
60 *
61 *
62 *
63 *
64 *
65 *
66 *
67 *
68 *
69 *
70 *
71 *
72 *
73 *
74 *
75 *
76 *
77 *
78 *
79 *
80 *
81 *
82 *
83 *
84 *
85 *
86 *
87 *
88 *
89 *
90 *
91 *
92 *
93 *
94 *
95 *
96 *
97 *
98 *
99 *
100 *
101 *
102 *
103 *
104 *
105 */
106 module otbn_alu_bignum
107   import otbn_pkg::*;
108 (
109   input logic clk_i,
110   input logic rst_ni,
111
112   input alu_bignum_operation_t operation_i,
113   input logic operation_valid_i,
114   input logic operation_commit_i, // used
  for SVAs only
115   output logic [WLEN-1:0] operation_result_o,
116   output logic selection_flag_o,
117
118   input alu_bignum_predec_t alu_bignum_predec_i,
119   input ispr_bignum_predec_t ispr_bignum_predec_i,
120
121   input ispr_e ispr_addr_i,
122   input logic [31:0] ispr_base_wdata_i,
123   input logic [BaseWordsPerWLEN-1:0] ispr_base_wr_en_i,
124   input logic [ExtWLEN-1:0] ispr_bignum_wdata_intg_i,
125   input logic ispr_bignum_wr_en_i,
126   input logic [NFlagGroups-1:0] ispr_flags_wr_i,
127   input logic ispr_wr_commit_i,
128   input logic ispr_init_i,
129   output logic [ExtWLEN-1:0] ispr_rdata_intg_o,
130   input logic ispr_rd_en_i,

```

```

131
132 input logic [ExtWLEN-1:0] ispr_acc_intg_i,
133 output logic [ExtWLEN-1:0] ispr_acc_wr_data_intg_o,
134 output logic ispr_acc_wr_en_o,
135
136 output logic [ExtWLEN-1:0] ispr_mod_intg_o,
137
138 output logic ispr_mai_ctrl_wr_o,
139 output logic [31:0] ispr_mai_ctrl_wdata_o,
140 input logic [31:0] ispr_mai_ctrl_rdata_i,
141 input logic [31:0] ispr_mai_status_rdata_i,
142 output logic ispr_mai_in0_s0_wr_o,
143 output logic [ExtWLEN-1:0] ispr_mai_in0_s0_wdata_o,
144 output logic ispr_mai_in0_s1_wr_o,
145 output logic [ExtWLEN-1:0] ispr_mai_in0_s1_wdata_o,
146 output logic ispr_mai_in1_s0_wr_o,
147 output logic [ExtWLEN-1:0] ispr_mai_in1_s0_wdata_o,
148 output logic ispr_mai_in1_s1_wr_o,
149 output logic [ExtWLEN-1:0] ispr_mai_in1_s1_wdata_o,
150 output logic ispr_mai_res_s0_wr_o,
151 output logic [ExtWLEN-1:0] ispr_mai_res_s0_wdata_o,
152 output logic ispr_mai_res_s1_wr_o,
153 output logic [ExtWLEN-1:0] ispr_mai_res_s1_wdata_o,
154 input logic [ExtWLEN-1:0] ispr_mai_in0_s0_rdata_i,
155 input logic [ExtWLEN-1:0] ispr_mai_in0_s1_rdata_i,
156 input logic [ExtWLEN-1:0] ispr_mai_in1_s0_rdata_i,
157 input logic [ExtWLEN-1:0] ispr_mai_in1_s1_rdata_i,
158 input logic [ExtWLEN-1:0] ispr_mai_res_s0_rdata_i,
159 input logic [ExtWLEN-1:0] ispr_mai_res_s1_rdata_i,
160
161 output logic ispr_kmac_status_wr_o,
162 output logic [31:0] ispr_k-,
163 output logic ispr_kmac_ctrl_wr_o,
164 output logic [31:0] ispr_kmac_ctrl_wdata_o,
165 output logic ispr_kmac_cfg_wr_o,
166 output logic [31:0] ispr_kmac_cfg_wdata_o,
167 output logic ispr_kmac_strb_wr_o,
168 output logic [31:0] ispr_kmac_strb_wdata_o,
169 output logic ispr_kmac_data_s0_wr_o,
170 output logic [ExtWLEN-1:0] ispr_k-,
171 output logic ispr_kmac_data_s1_wr_o,
172 output logic [ExtWLEN-1:0] ispr_k-,
173 input logic [31:0] ispr_k-,
174 input logic [31:0] ispr_kmac_cfg_rdata_i,
175 input logic [31:0] ispr_kmac_strb_rdata_i,
176 input logic [ExtWLEN-1:0] ispr_k-,
177 input logic [ExtWLEN-1:0] ispr_k-,
178 output logic ispr_kmac_data_s0_rd_o,
179 output logic ispr_kmac_data_s1_rd_o,
180
181 output logic ispr_urnd_ctrl_wr_o,
182 output logic [31:0] ispr_urnd_ctrl_wdata_o,
183 input logic [31:0] ispr_urnd_status_rdata_i,
184 output logic ispr_urnd_state_wr_o,
185 output logic [UrndPartialSeedwidth-1:0] ispr_urnd_state_wdata_o,
186 input logic [WLEN-1:0] ispr_urnd_state_rdata_i,
187
188 output logic reg_intg_violation_err_o,
189
190 input logic sec_wipe_mod_urnd_i,
191 input logic sec_wipe_running_i,
192 output logic sec_wipe_err_o,
193
194 input flags_t mac_operation_flags_i,
195 input flags_t mac_opera-,
196
197 input logic [WLEN-1:0] rnd_data_i,
198 input logic [WLEN-1:0] urnd_data_i,
199 input logic [31:0] insn_cnt_i,
200
201 input logic [1:0][SideloadKeyWidth-1:0] side-
load_key_shares_i,
202
203 output logic alu_predec_error_o,
204 output logic ispr_predec_error_o
205 );
206
207 logic [WLEN-1:0] adder_y_res;
208 logic [NVecProc-1:0] adder_y_carries_out;
209 logic [WLEN-1:0] logical_res;
210
211 //////////////////////////////////////////////////
212 // ISPRs //
213 //////////////////////////////////////////////////
214
215 flags_t flags_d [NFlagGroups];
216 flags_t flags_q [NFlagGroups];
217 logic [NFlagGroups*FlagsWidth-1:0] flags_flattened;

218 flags_t selected_flags;
219 flags_t adder_update_flags;
220 logic ad-,
221
222 der_update_flags_en_raw;
223 flags_t logic_update_flags
224 [NFlagGroups];
225 logic lo-,
226
227 gic_update_flags_en_raw;
228 flags_t mac_update_flags
229 [NFlagGroups];
230 logic [NFlagGroups-1:0] mac_up-,
231
232 date_z_flag_en_blanked;
233 flags_t ispr_update_flags
234 [NFlagGroups];
235
236 logic [NIspr-1:0] expected_ispr_rd_en_onehot;
237 logic [NIspr-1:0] expected_ispr_wr_en_onehot;
238 logic ispr_wr_en;
239
240 logic [NFlagGroups-1:0] expected_flag_group_sel;
241 flags_t expected_flag_sel;
242 logic [NFlagGroups-1:0] expected_flags_keep;
243 logic [NFlagGroups-1:0] expected_flags_adder_update;
244 logic [NFlagGroups-1:0] expected_flags_logic_update;
245 logic [NFlagGroups-1:0] expected_flags_mac_update;
246 logic [NFlagGroups-1:0] expected_flags_ispr_wr;
247
248 //////////////////////////////////////////////////
249 // Flags Selection //
250 //////////////////////////////////////////////////
251
252 always_comb begin
253     expected_flag_group_sel = '0;
254     expected_flag_group_sel[operation_i.flag_group] = '1'b1;
255 end
256 assign expected_flag_sel.C = operation_i.sel_flag == FlagC;
257 assign expected_flag_sel.M = operation_i.sel_flag == FlagM;
258 assign expected_flag_sel.L = operation_i.sel_flag == FlagL;
259 assign expected_flag_sel.Z = operation_i.sel_flag == FlagZ;
260
261 // SEC_CM: DATA_REG_SW.SCA
262 prim_onehot_mux #(
263     .Width(FlagsWidth),
264     .Inputs(NFlagGroups)
265 ) u_flags_q_mux (
266     .clk_i,
267     .rst_ni,
268     .in_i (flags_q),
269     .sel_i(alu_bignum_predec_i.flag_group_sel),
270     .out_o(selected_flags)
271 );
272
273 `ASSERT(BlankingSelectedFlags_A, expected_flag_group_sel ==
'0 |> selected_flags == '0, clk_i,
274     !rst_ni || alu_predec_error_o || !operation_commit_i)
275
276 logic flag_mux_in [FlagsWidth];
277 logic [FlagsWidth-1:0] flag_mux_sel;
278 assign flag_mux_in = '{selected_flags.C,
279     selected_flags.M,
280     selected_flags.L,
281     selected_flags.Z};
282
283 assign flag_mux_sel = {alu_bignum_predec_i.flag_sel.Z,
284     alu_bignum_predec_i.flag_sel.L,
285     alu_bignum_predec_i.flag_sel.M,
286     alu_bignum_predec_i.flag_sel.C};
287
288 // SEC_CM: DATA_REG_SW.SCA
289 prim_onehot_mux #(
290     .Width(1),
291     .Inputs(FlagsWidth)
292 ) u_flag_mux (
293     .clk_i,
294     .rst_ni,
295     .in_i (flag_mux_in),
296     .sel_i(flag_mux_sel),
297     .out_o(selection_flag_o)
298 );
299
300 `ASSERT(BlankingSelectionFlag_A, expected_flag_sel == '0 |>
selection_flag_o == '0, clk_i,
301     !rst_ni || alu_predec_error_o || !operation_commit_i)
302
303 //////////////////////////////////////////////////
304 // Flags Update //
305 //////////////////////////////////////////////////
306
307 // Flags are only updated for regular 256b operations.
308 // Vectorized operations do not update the flags.
309 `ASSERT(VecAndModAndRshOpsNoFlagUpdate_A,
310     operation_i.op inside {AluOpBignumAddv, AluOpBignum-
311     Subv,
312     AluOpBignumAddm, AluOpBignu-
313     mAddvm,
314     AluOpBignumSubm, AluOpBignum-
315     Subvm,
316     AluOpBignumTrn1, AluOp-
317     BignumTrn2,
318     AluOpBignumShv,
319     AluOpBignumPack, AluOpBignu-

```

```

mUnpk,
306             AluOpBignumRshi}
307     |-> !adder_update_flags_en_raw && !lo-
gic_update_flags_en_raw,
308         clk_i, !rst_ni || !operation_commit_i)
309
310 // Note that the flag zeroing triggered by ispr_init_i and
secure wipe is achieved by not
311 // selecting any inputs in the one-hot muxes below. The in-
struction fetch/predecoder stage
312 // is driving the selector inputs accordingly.
313
314 always_comb begin
315     expected_flags_adder_update = '0;
316     expected_flags_logic_update = '0;
317     expected_flags_mac_update = '0;
318
319     expected_flags_adder_update[operation_i.flag_group] = op-
eration_i.alu_flag_en &
320         ad-
der_update_flags_en_raw;
321     expected_flags_logic_update[operation_i.flag_group] = op-
eration_i.alu_flag_en &
322         lo-
gic_update_flags_en_raw;
323     expected_flags_mac_update[operation_i.flag_group] = op-
eration_i.mac_flag_en;
324 end
325 assign expected_flags_ispr_wr = ispr_flags_wr_i;
326
327 assign expected_flags_keep = ~(expected_flags_adder_update |
328     expected_flags_logic_update |
329     expected_flags_mac_update |
330     expected_flags_ispr_wr);
331
332 // Adder operations update all flags.
333 assign adder_update_flags.C = (operation_i.op == AluOpBignu-
mAdd ||
334     operation_i.op == AluOpBignu-
mAddc) ?
335     ad-
der_y_carries_out[NVecProc-1] : ~adder_y_carries_out[NVecProc-1];
336
337 assign adder_update_flags.M = adder_y_res[WLEN-1];
338 assign adder_update_flags.L = adder_y_res[0];
339 assign adder_update_flags.Z = ~|adder_y_res;
340
341 for (genvar i_fg = 0; i_fg < NFlagGroups; i_fg++) begin :
g_update_flag_groups
342
343     // Logical operations only update M, L and Z; C must re-
main at its old value.
344     assign logic_update_flags[i_fg].C = flags_q[i_fg].C;
345     assign logic_update_flags[i_fg].M = logical_res[WLEN-1];
346     assign logic_update_flags[i_fg].L = logical_res[0];
347     assign logic_update_flags[i_fg].Z = ~|logical_res;
348
349     // MAC Flags //
350     // MAC operations don't update C.
351     assign mac_update_flags[i_fg].C = flags_q[i_fg].C;
352
353     // Tie off unused signals.
354     logic unused_mac_operation_flags;
355     assign unused_mac_operation_flags = macopera-
tion_flags_i.C ^ mac_operation_flags_en_i.C;
356
357     // MAC operations update M and L depending on the opera-
tion. The individual enable signals for
358     // M and L are generated from flopped instruction bits
with minimal logic. They are not data
359     // dependent.
360     assign mac_update_flags[i_fg].M = macopera-
tion_flags_en_i.M ?
361         macopera-
tion_flags_i.M : flags_q[i_fg].M;
362     assign mac_update_flags[i_fg].L = macopera-
tion_flags_en_i.L ?
363         macopera-
tion_flags_i.L : flags_q[i_fg].L;
364
365     // MAC operations update Z depending on the operation and
data. For BN.MULQACC.S0, already the
366     // enable signal is data dependent (it depends on the
lower half of the accumulator result). As
367     // a result the enable signal might change back and forth
during instruction execution which may
368     // lead to SCA leakage. There is nothing that can really
be done to avoid this other than
369
370 // pipelining the flag computation which has a performance
impact.
371 //
372
373 // By blanking the enable signal for the other flag group,
we can at least avoid leakage related
374
375 // to the other flag group, i.e., we give the programmer a
way to control where the leakage

```

```

376 // happens.
377 // SEC_CM: DATA_REG_SW.SCA
378 prim_blanker #(.Width(1)) u_mac_z_flag_en_blanker (
379     .in_i (mac_operation_flags_en_i.Z),
380     .en_i (alu_bignum_predec_i.flags_mac_update[i_fg]),
381     .out_o(mac_update_z_flag_en_blanked[i_fg])
382 );
383 assign mac_update_flags[i_fg].Z = mac_up-
date_z_flag_en_blanked[i_fg] ?
384     macopera-
tion_flags_i.Z : flags_q[i_fg].Z;
385
386 // For ISPR writes, we get the full write data from the
base ALU and will select the relevant
387 // parts using the blankers and one-hot muxes below.
388 assign ispr_update_flags[i_fg] =
ispr_base_wdata_i[i_fg*FlagsWidth+:FlagsWidth];
389 end
390
391 localparam int NFlagsSrcs = 5;
392 for (genvar i_fg = 0; i_fg < NFlagGroups; i_fg++) begin :
g_flag_groups
393
394     flags_t             flags_d_mux_in [NFlagsSrcs];
395     logic [NFlagsSrcs-1:0] flags_d_mux_sel;
396     assign flags_d_mux_in = '{ispr_update_flags[i_fg],
397         mac_update_flags[i_fg],
398         logic_update_flags[i_fg],
399         adder_update_flags,
400         flags_q[i_fg]};
401     assign flags_d_mux_sel =
{alu_bignum_predec_i.flags_keep[i_fg],
402         alu_bignum_predec_i.flags_adder_update[i_fg],
403         alu_bignum_predec_i.flags_logic_update[i_fg],
404         alu_bignum_predec_i.flags_mac_update[i_fg],
405         alu_bignum_predec_i.flags_ispr_wr[i_fg]};
406
407     // SEC_CM: DATA_REG_SW.SCA
408     prim_onehot_mux #(
409         .Width(FlagsWidth),
410         .Inputs(NFlagsSrcs)
411     ) u_flags_d_mux (
412         .clk_i,
413         .rst_ni,
414         .in_i (flags_d_mux_in),
415         .sel_i (flags_d_mux_sel),
416         .out_o(flags_d[i_fg])
417     );
418
419     always_ff @(posedge clk_i or negedge rst_ni) begin
420         if (!rst_ni) begin
421             flags_q[i_fg] <= '{Z : 1'b0, L : 1'b0, M : 1'b0, C :
1'b0};
422         end else begin
423             flags_q[i_fg] <= flags_d[i_fg];
424         end
425     end
426
427     assign flags_flattened[i_fg*FlagsWidth+:FlagsWidth] =
flags_q[i_fg];
428 end
429
430 // MOD //
431 // MOD //
432
433 logic [ExtWLEN-1:0] mod_intg_q;
434 logic [ExtWLEN-1:0] mod_intg_d;
435 logic [BaseWordsPerWLEN-1:0] mod_ispr_wr_en;
436 logic [BaseWordsPerWLEN-1:0] mod_wr_en;
437
438 logic [ExtWLEN-1:0] ispr_mod_bignum_wdata_intg_blanked;
439
440 // SEC_CM: DATA_REG_SW.SCA
441 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mod_bignum_wdata_blanker (
442     .in_i (ispr_bignum_wdata_intg_i),
443     .en_i (ispr_bignum_predec_i.ispr_wr_en[IsprMod]),
444     .out_o(ispr_mod_bignum_wdata_intg_blanked)
445 );
446 // If the blanker is enabled, the output will not carry the
correct ECC bits. This is not
447
448 // a problem because a blanked value should never be written
to the register. If the blanked
449 // value is written to the register nonetheless, an integ-
rity error arises.
450
451 logic [WLEN-1:0] mod_no_intg_d;
452 logic [WLEN-1:0] mod_no_intg_q;
453 logic [WLEN-1:0] mod_no_intg_q_replicated;
454 logic [ExtWLEN-1:0] mod_intg_calc;
455 logic [2*BaseWordsPerWLEN-1:0] mod_intg_err;
456
457 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_mod_words
458     prim_secdec_inv_39_32_enc i_secdec_enc (

```

```

458     .data_i(mod_no_intg_d[i_word*32+:32]),
459     .data_o(mod_intg_calc[i_word*39+:39])
460 );
461 prim_secdec_inv_39_32_dec i_secdec_dec (
462     .data_i (mod_intg_q[i_word*39+:39]),
463     .data_o  (* unused because we abort on any integrity
error */),
464     .syndrome_o(* unused */),
465     .err_o      (mod_intg_err[i_word*2+:2])
466 );
467 assign mod_no_intg_q[i_word*32+:32] =
mod_intg_q[i_word*39+:32];
468
469 always_ff @(posedge clk_i) begin
470     if (mod_wr_en[i_word]) begin
471         mod_intg_q[i_word*39+:39] <= mod_intg_d[i_word*39+:
39];
472     end
473 end
474
475 always_comb begin
476     mod_no_intg_d[i_word*32+:32] = '0;
477     unique case (i'b1)
478         // Non-encoded inputs have to be encoded before writ-
ing to the register.
479         sec_wipe_mod_urnd_i: begin
480             // In a secure wipe, `urnd_data_i` is written to the
register before the zero word. The
481             // ECC bits should not matter between the two
writes, but nonetheless we encode
482             // `urnd_data_i` so there is no spurious integrity
error.
483             mod_no_intg_d[i_word*32+:32] =
urnd_data_i[i_word*32+:32];
484             mod_intg_d[i_word*39+:39] =
mod_intg_calc[i_word*39+:39];
485         end
486         // Pre-encoded inputs can directly be written to the
register.
487         default: mod_intg_d[i_word*39+:39] =
ispr_mod_bignum_wdata_intg_blanked[i_word*39+:39];
488     endcase
489
490     unique case (i'b1)
491         ispr_init_i: mod_intg_d[i_word*39+:39] = EccZeroWord;
492         ispr_base_wr_en_i[i_word]: begin
493             mod_no_intg_d[i_word*32+:32] = ispr_base_wdata_i;
494             mod_intg_d[i_word*39+:39] =
mod_intg_calc[i_word*39+:39];
495         end
496         default: ;
497     endcase
498 end
499
500 `ASSERT(MODWrSelOneHot, $onehot0({ispr_init_i,
ispr_base_wr_en_i[i_word]}))
501
502 assign mod_ispr_wr_en[i_word] = (ispr_addr_i == Is-
prMod)
&
503 (ispr_base_wr_en_i[i_word] |
ispr_bignum_wr_en_i) &
504 ispr_wr_commit_i;
505
506 assign mod_wr_en[i_word] = ispr_init_i |
507 mod_ispr_wr_en[i_word] |
508 sec_wipe_mod_urnd_i;
509 end
510
511 // Replicate the modulus value to match the ELEN
512 always_comb begin
513     unique case (alu_bignum_predec_i.alu_elen)
514         AluElen32: mod_no_intg_q_replicated = {(WLEN/32)
{mod_no_intg_q[31:0]}};
515         AluElen256: mod_no_intg_q_replicated = mod_no_intg_q;
516         default: mod_no_intg_q_replicated = mod_no_intg_q;
517     endcase
518 end
519
520 // Output current MOD value towards BN MAC
521 assign ispr_mod_intg_o = mod_intg_q;
522
523 // ACC
524 // ACC
525 // ACC
526
527 assign ispr_acc_wr_en_o =
528 ((ispr_addr_i == IsprAcc) & ispr_bignum_wr_en_i &
ispr_wr_commit_i) | ispr_init_i;
529
530
531 logic [ExtWLEN-1:0] ispr_acc_bignum_wdata_intg_blanked;
532
533 // SEC_CM: DATA_REG_SW.SCA
534 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_acc_bignum_wdata_intg_blanker (
535     .in_i (ispr_bignum_wdata_intg_i),
536     .en_i (ispr_bignum_predec_i.ispr_wr_en[IsprAcc]),
537     .out_o(ispr_acc_bignum_wdata_intg_blanked)
538 );

```

```

539 // If the blanker is enabled, the output will not carry the
correct ECC bits. This is not
540 // a problem because a blanked value should never be used.
If the blanked value is used
541 // nonetheless, an integrity error arises.
542
543 assign ispr_acc_wr_data_intg_o = ispr_init_i ? EccWideZero-
Word
544 :
ispr_acc_bignum_wdata_intg_blanked;
545
546 // MAI Write
547 // MAI Write
548 // MAI Write
549
550 assign ispr_mai_in0_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn0S0];
551 // SEC_CM: DATA_REG_SW.SCA
552 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in0_s0_wdata_blanker (
553     .in_i (ispr_bignum_wdata_intg_i),
554     .en_i (ispr_mai_in0_s0_wr_o),
555     .out_o(ispr_mai_in0_s0_wdata_o)
556 );
557
558 assign ispr_mai_in0_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn0S1];
559 // SEC_CM: DATA_REG_SW.SCA
560 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in0_s1_wdata_blanker (
561     .in_i (ispr_bignum_wdata_intg_i),
562     .en_i (ispr_mai_in0_s1_wr_o),
563     .out_o(ispr_mai_in0_s1_wdata_o)
564 );
565
566 assign ispr_mai_in1_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn1S0];
567 // SEC_CM: DATA_REG_SW.SCA
568 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in1_s0_wdata_blanker (
569     .in_i (ispr_bignum_wdata_intg_i),
570     .en_i (ispr_mai_in1_s0_wr_o),
571     .out_o(ispr_mai_in1_s0_wdata_o)
572 );
573
574 assign ispr_mai_in1_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiIn1S1];
575 // SEC_CM: DATA_REG_SW.SCA
576 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_in1_s1_wdata_blanker (
577     .in_i (ispr_bignum_wdata_intg_i),
578     .en_i (ispr_mai_in1_s1_wr_o),
579     .out_o(ispr_mai_in1_s1_wdata_o)
580 );
581
582 assign ispr_mai_res_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiResS0];
583 // SEC_CM: DATA_REG_SW.SCA
584 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_res_s0_wdata_blanker (
585     .in_i (ispr_bignum_wdata_intg_i),
586     .en_i (ispr_mai_res_s0_wr_o),
587     .out_o(ispr_mai_res_s0_wdata_o)
588 );
589
590 assign ispr_mai_res_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiResS1];
591 // SEC_CM: DATA_REG_SW.SCA
592 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_mai_res_s1_wdata_blanker (
593     .in_i (ispr_bignum_wdata_intg_i),
594     .en_i (ispr_mai_res_s1_wr_o),
595     .out_o(ispr_mai_res_s1_wdata_o)
596 );
597
598 assign ispr_mai_ctrl_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprMaiCtrl];
599 // SEC_CM: DATA_REG_SW.SCA
600 prim_blanker #(.Width(32'd32)) u_ispr_mai_ctrl_wdata_blanker (
601     .in_i (ispr_base_wdata_i),
602     .en_i (ispr_mai_ctrl_wr_o),
603     .out_o(ispr_mai_ctrl_wdata_o)
604 );
605
606 // KMAC Write
607 // KMAC Write
608 // KMAC Write
609
610 assign ispr_kmac_status_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacStatus];
611 // SEC_CM: DATA_REG_SW.SCA
612 prim_blanker #(.Width(32'd32)) u_is-
pr_kmac_status_wdata_blanker (
613     .in_i (ispr_base_wdata_i),
614     .en_i (ispr_kmac_status_wr_o),
615     .out_o(ispr_kmac_status_wdata_o)
616 );
617
618 assign ispr_kmac_ctrl_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacCtrl];

```

```

619 // SEC_CM: DATA_REG_SW.SCA
620 prim_blanker #(.Width(32'd32)) u_is-
pr_kmac_ctrl_wdata_blanker (
621   .in_i (ispr_base_wdata_i),
622   .en_i (ispr_kmac_ctrl_wr_o),
623   .out_o(ispr_kmac_ctrl_wdata_o)
624 );
625
626 assign ispr_kmac_cfg_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacCfg];
627 // SEC_CM: DATA_REG_SW.SCA

628 prim_blanker #(.Width(32'd32)) u_ispr_kmac_cfg_wdata_blanker (
629   .in_i (ispr_base_wdata_i),
630   .en_i (ispr_kmac_cfg_wr_o),
631   .out_o(ispr_kmac_cfg_wdata_o)
632 );
633
634 assign ispr_kmac_strb_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacStrb];
635 // SEC_CM: DATA_REG_SW.SCA
636 prim_blanker #(.Width(32'd32)) u_is-
pr_kmac_strb_wdata_blanker (
637   .in_i (ispr_base_wdata_i),
638   .en_i (ispr_kmac_strb_wr_o),
639   .out_o(ispr_kmac_strb_wdata_o)
640 );
641
642 assign ispr_kmac_data_s0_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacDataS0];
643 // SEC_CM: DATA_REG_SW.SCA
644 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_kmac_data_s0_wdata_blanker (
645   .in_i (ispr_bignum_wdata_intg_i),
646   .en_i (ispr_kmac_data_s0_wr_o),
647   .out_o(ispr_kmac_data_s0_wdata_o)
648 );
649
650 assign ispr_kmac_data_s1_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprKmacDataS1];
651 // SEC_CM: DATA_REG_SW.SCA
652 prim_blanker #(.Width(ExtWLEN)) u_is-
pr_kmac_data_s1_wdata_blanker (
653   .in_i (ispr_bignum_wdata_intg_i),
654   .en_i (ispr_kmac_data_s1_wr_o),
655   .out_o(ispr_kmac_data_s1_wdata_o)
656 );
657
658 assign ispr_kmac_data_s0_rd_o =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS0];
659 assign ispr_kmac_data_s1_rd_o =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS1];
660
661 //////////////////////////////////////////////////
662 // URND Write //
663 //////////////////////////////////////////////////
664 // No blanker is required in front of a register holding
known control values when writing values
665 // from base registers.
666 assign ispr_urnd_ctrl_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprUrndCtrl];
667 assign ispr_urnd_ctrl_wdata_o = ispr_base_wdata_i;
668
669 // We drop the integrity towards the rnd module. SW can
verify the update by reading back the
670 // state.
671 logic [WLEN-1:0] ispr_urnd_state_wdata_no_intg;
672 for (genvar i = 0; i < BaseWordsPerWLEN; i++) begin :
g_urnd_state_drop_intg
673   assign ispr_urnd_state_wdata_no_intg[i * 32 +: 32] =
674     ispr_bignum_wdata_intg_i[i * BaseIntgWidth +: 32];
675 end
676
677 assign ispr_urnd_state_wr_o =
ispr_bignum_predec_i.ispr_wr_en[IsprUrndState];
678 // SEC_CM: DATA_REG_SW.SCA
679 prim_blanker #(.Width(UrndPartialSeedWidth)) u_is-
pr_urnd_state_wdata_blanker (
680   .in_i
(ispr_urnd_state_wdata_no_intg[UrndPartialSeedWidth-1:0]),
681   .en_i (ispr_urnd_state_wr_o),
682   .out_o(ispr_urnd_state_wdata_o)
683 );
684
685 logic unused_urnd_state_wdata;
686 assign unused_urnd_state_wdata =
^ispr_urnd_state_wdata_no_intg[WLEN-1:UrndPartialSeedWidth];
687
688 //////////////////////////////////////////////////
689 // ISPR Read //
690 //////////////////////////////////////////////////
691
692 // ISPR read data is muxed out in two stages:
693 // 1. Select amongst the ISPRs that have no integrity bits.
The output has integrity calculated
694 // for it.
695 // 2. Select between the ISPRs that have integrity bits and
the result of the first stage.
696
697 // Number of ISPRs that have no integrity protection
698 localparam int NNoIntgIspr = 15;

```

```

699 // IDs for ISPRs without integrity
700 localparam int IsprRndNoIntg = 0;
701 localparam int IsprUrndNoIntg = 1;
702 localparam int IsprMaiCtrlNoIntg = 2;
703 localparam int IsprMaiStatusNoIntg = 3;
704 localparam int IsprFlagsNoIntg = 4;
705 localparam int IsprKeyS0LNoIntg = 5;
706 localparam int IsprKeyS0HNoIntg = 6;
707 localparam int IsprKeyS1LNoIntg = 7;
708 localparam int IsprKeyS1HNoIntg = 8;
709 localparam int IsprKmacStatusNoIntg = 9;
710 localparam int IsprKmacCfgNoIntg = 10;
711 localparam int IsprKmacStrbNoIntg = 11;
712 localparam int IsprInsCntNoIntg = 12;
713 localparam int IsprUrndStateNoIntg = 13;
714 localparam int IsprUrndStatusNoIntg = 14;

715 // KMAC_CTRL and URND_CTRL always read as '0. Thus we do not
need a mux input.
716
717 logic [NNoIntgIspr-1:0] ispr_rdata_no_intg_mux_sel;
718
719 // Number of ISPRs that have integrity protection
720 localparam int NIntgIspr = 10;
721 // IDs for ISPRs with integrity
722 localparam int IsprModIntg = 0;
723 localparam int IsprAccIntg = 1;
724 localparam int IsprMaiResS0Intg = 2;
725 localparam int IsprMaiResS1Intg = 3;
726 localparam int IsprMaiInS0Intg = 4;
727 localparam int IsprMaiInS1Intg = 5;
728 localparam int IsprMaiInIS0Intg = 6;
729 localparam int IsprMaiInIS1Intg = 7;
730 localparam int IsprKmacDataS0Intg = 8;
731 localparam int IsprKmacDataS1Intg = 9;
732 // ID representing all ISPRs with no integrity
733 localparam int IsprNoIntg = 10;
734
735 logic [NIntgIspr:0] ispr_rdata_intg_mux_sel;
736 logic [ExtWLEN-1:0] ispr_rdata_intg_mux_in [NIntgIspr+1];
737 logic [WLEN-1:0] ispr_rdata_no_intg_mux_in [NNoIntgIspr];
738
739 // First stage
740 assign ispr_rdata_no_intg_mux_in[IsprRndNoIntg] =
rnd_data_i;
741 assign ispr_rdata_no_intg_mux_in[IsprUrndNoIntg] =
urnd_data_i;
742 assign ispr_rdata_no_intg_mux_in[IsprUrndStateNoIntg] =
ispr_urnd_state_rdata_i;
743 assign ispr_rdata_no_intg_mux_in[IsprUrndStatusNoIntg] =
744   {{{(WLEN - 32){1'b0}}, ispr_urnd_status_rdata_i};
745 assign ispr_rdata_no_intg_mux_in[IsprMaiCtrlNoIntg] =
746   {{{(WLEN - 32){1'b0}}, ispr_mai_ctrl_rdata_i};
747 assign ispr_rdata_no_intg_mux_in[IsprMaiStatusNoIntg] =
748   {{{(WLEN - 32){1'b0}}, ispr_mai_status_rdata_i};
749 assign ispr_rdata_no_intg_mux_in[IsprFlagsNoIntg] =
750   {{{(WLEN - (NFlagGroups * FlagsWidth)){1'b0}},
flags_flattened}};
751
752 // SEC_CM: KEY.SIDELOAD
753 assign ispr_rdata_no_intg_mux_in[IsprKeyS0LNoIntg] = side-
load_key_shares_i[0][255:0];
754 assign ispr_rdata_no_intg_mux_in[IsprKeyS0HNoIntg] =
755   {{{(WLEN - (SideLoadKeyWidth - 256)){1'b0}}, side-
load_key_shares_i[0][SideLoadKeyWidth-1:256]}};
756 assign ispr_rdata_no_intg_mux_in[IsprKeyS1LNoIntg] = side-
load_key_shares_i[1][255:0];
757 assign ispr_rdata_no_intg_mux_in[IsprKeyS1HNoIntg] =
758   {{{(WLEN - (SideLoadKeyWidth - 256)){1'b0}}, side-
load_key_shares_i[1][SideLoadKeyWidth-1:256]}};
759 assign ispr_rdata_no_intg_mux_in[IsprInsCntNoIntg] =
760   {{{(WLEN - 32){1'b0}}, insn_cnt_i}};
761
762 assign ispr_rdata_no_intg_mux_in[IsprKmacStatusNoIntg] =
763   {{{(WLEN - 32){1'b0}}, ispr_kmac_status_rdata_i};
764 assign ispr_rdata_no_intg_mux_in[IsprKmacCfgNoIntg] =
765   {{{(WLEN - 32){1'b0}}, ispr_kmac_cfg_rdata_i};
766 assign ispr_rdata_no_intg_mux_in[IsprKmacStrbNoIntg] =
767   {{{(WLEN - 32){1'b0}}, ispr_kmac_strb_rdata_i};
768
769 assign ispr_rdata_no_intg_mux_sel[IsprRndNoIntg] =
770   ispr_bignum_predec_i.ispr_rd_en[IsprRnd];
771 assign ispr_rdata_no_intg_mux_sel[IsprUrndNoIntg] =
772   ispr_bignum_predec_i.ispr_rd_en[IsprUrnd];
773 assign ispr_rdata_no_intg_mux_sel[IsprUrndStateNoIntg] =
774   ispr_bignum_predec_i.ispr_rd_en[IsprUrndState];
775 assign ispr_rdata_no_intg_mux_sel[IsprUrndStatusNoIntg] =
776   ispr_bignum_predec_i.ispr_rd_en[IsprUrndStatus];
777 assign ispr_rdata_no_intg_mux_sel[IsprMaiCtrlNoIntg] =
778   ispr_bignum_predec_i.ispr_rd_en[IsprMaiCtrl];
779 assign ispr_rdata_no_intg_mux_sel[IsprMaiStatusNoIntg] =
780   ispr_bignum_predec_i.ispr_rd_en[IsprMaiStatus];
781 assign ispr_rdata_no_intg_mux_sel[IsprFlagsNoIntg] =
782   ispr_bignum_predec_i.ispr_rd_en[IsprFlags];
783 assign ispr_rdata_no_intg_mux_sel[IsprKeyS0LNoIntg] =
784   ispr_bignum_predec_i.ispr_rd_en[IsprKeyS0L];
785 assign ispr_rdata_no_intg_mux_sel[IsprKeyS0HNoIntg] =
786   ispr_bignum_predec_i.ispr_rd_en[IsprKeyS0H];
787 assign ispr_rdata_no_intg_mux_sel[IsprKeyS1LNoIntg] =
788   ispr_bignum_predec_i.ispr_rd_en[IsprKeyS1L];
789 assign ispr_rdata_no_intg_mux_sel[IsprKeyS1HNoIntg] =

```



```

789     ispr_bignum_predec_i.ispr_rd_en[IsprKeyS1H];
790     assign ispr_rdata_no_intg_mux_sel[IsprKmacStatusNoIntg] =
791       ispr_bignum_predec_i.ispr_rd_en[IsprKmacStatus];
792     assign ispr_rdata_no_intg_mux_sel[IsprKmacCfgNoIntg] =
793       ispr_bignum_predec_i.ispr_rd_en[IsprKmacCfg];
794     assign ispr_rdata_no_intg_mux_sel[IsprKmacStrbNoIntg] =
795       ispr_bignum_predec_i.ispr_rd_en[IsprKmacStrb];
796     // KMAC_CTRL and URND_CTRL always read as '0'. We use the
onehot MUX to output '0' by not having an
797     // input for these. We thus also don't need to factor in
their read enable signals into the MUX
798     // select signal.
799
800     assign ispr_rdata_no_intg_mux_sel[IsprInsnCntNoIntg] =
801       ispr_bignum_predec_i.ispr_rd_en[IsprInsnCnt];
802
803     logic [WLEN-1:0] ispr_rdata_no_intg;
804     logic [ExtWLEN-1:0] ispr_rdata_intg_calc;
805
806     // SEC_CM: DATA_REG_SW.SCA
807     prim_onehot_mux #(
808       .Width (WLEN),
809       .Inputs (NNoIntgIspr)
810     ) u_ispr_rdata_no_intg_mux (
811       .clk_i,
812       .rst_ni,
813       .in_i (ispr_rdata_no_intg_mux_in),
814       .sel_i (ispr_rdata_no_intg_mux_sel),
815       .out_o (ispr_rdata_no_intg)
816     );
817
818     for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_rdata_enc
819       prim_secdec_inv_39_32_enc i_secdec_enc (
820         .data_i (ispr_rdata_no_intg[i_word * 32 +: 32]),
821         .data_o (ispr_rdata_intg_calc[i_word * 39 +: 39])
822       );
823     end
824
825     // Second stage
826     assign ispr_rdata_intg_mux_in[IsprModIntg] =
mod_intg_q;
827     assign ispr_rdata_intg_mux_in[IsprAccIntg] =
ispr_acc_intg_i;
828     assign ispr_rdata_intg_mux_in[IsprMaiResS0Intg] =
ispr_mai_res_s0_rdata_i;
829     assign ispr_rdata_intg_mux_in[IsprMaiResS1Intg] =
ispr_mai_res_s1_rdata_i;
830     assign ispr_rdata_intg_mux_in[IsprMaiInS0Intg] =
ispr_mai_in0_s0_rdata_i;
831     assign ispr_rdata_intg_mux_in[IsprMaiInS1Intg] =
ispr_mai_in0_s1_rdata_i;
832     assign ispr_rdata_intg_mux_in[IsprMaiIn1S0Intg] =
ispr_mai_in1_s0_rdata_i;
833     assign ispr_rdata_intg_mux_in[IsprMaiIn1S1Intg] =
ispr_mai_in1_s1_rdata_i;
834     assign ispr_rdata_intg_mux_in[IsprKmacDataS0Intg] = ispr_k-
mac_data_s0_rdata_i;
835     assign ispr_rdata_intg_mux_in[IsprKmacDataS1Intg] = ispr_k-
mac_data_s1_rdata_i;
836     assign ispr_rdata_intg_mux_in[IsprNoIntg] =
ispr_rdata_intg_calc;
837
838     assign ispr_rdata_intg_mux_sel[IsprModIntg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMod];
839     assign ispr_rdata_intg_mux_sel[IsprAccIntg] =
ispr_bignum_predec_i.ispr_rd_en[IsprAcc];
840     assign ispr_rdata_intg_mux_sel[IsprMaiResS0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiResS0];
841     assign ispr_rdata_intg_mux_sel[IsprMaiResS1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiResS1];
842     assign ispr_rdata_intg_mux_sel[IsprMaiInS0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiInS0];
843     assign ispr_rdata_intg_mux_sel[IsprMaiInS1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiInS1];
844     assign ispr_rdata_intg_mux_sel[IsprMaiIn0S0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiIn0S0];
845     assign ispr_rdata_intg_mux_sel[IsprMaiIn0S1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiIn0S1];
846     assign ispr_rdata_intg_mux_sel[IsprMaiIn1S0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiIn1S0];
847     assign ispr_rdata_intg_mux_sel[IsprMaiIn1S1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprMaiIn1S1];
848     assign ispr_rdata_intg_mux_sel[IsprKmacDataS0Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS0];
849     assign ispr_rdata_intg_mux_sel[IsprKmacDataS1Intg] =
ispr_bignum_predec_i.ispr_rd_en[IsprKmacDataS1];
850
851     assign ispr_rdata_intg_mux_sel[IsprNoIntg] =
|{ispr_bignum_predec_i.ispr_rd_en[IsprInsnCnt],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndState],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprUrndCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacStrb],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacCfg],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprKmacStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprMaiCtrl],
ispr_bignum_predec_i.ispr_rd_en[IsprMaiStatus],
ispr_bignum_predec_i.ispr_rd_en[IsprKeyS1H:IsprKeyS0L],
ispr_bignum_predec_i.ispr_rd_en[IsprUrnd],
ispr_bignum_predec_i.ispr_rd_en[IsprFlags],
ispr_bignum_predec_i.ispr_rd_en[IsprRnd]};
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874

```

```

875     // If we're reading from an ISPR we must be using the
ispr_rdata_intg_mux
876     `ASSERT(IsprRdataIntgMuxSelIfIsprRd_A,
877       |{ispr_bignum_predec_i.ispr_rd_en |-> |
ispr_rdata_intg_mux_sel)
878
879     // If we're reading from MOD or ACC we must not take the
read data from the calculated integrity
880     // path
881     `ASSERT(IsprModMustTakeIntg_A,
882       ispr_bignum_predec_i.ispr_rd_en[IsprMod] |-> !
ispr_rdata_intg_mux_sel[IsprNoIntg])
883
884     `ASSERT(IsprAccMustTakeIntg_A,
885       ispr_bignum_predec_i.ispr_rd_en[IsprAcc] |-> !
ispr_rdata_intg_mux_sel[IsprNoIntg])
886
887
888     prim_onehot_mux #(
889       .Width (ExtWLEN),
890       .Inputs (NIntgIspr+1)
891     ) u_ispr_rdata_intg_mux (
892       .clk_i,
893       .rst_ni,
894       .in_i (ispr_rdata_intg_mux_in),
895       .sel_i (ispr_rdata_intg_mux_sel),
896       .out_o (ispr_rdata_intg_o)
897     );
898
899     ///////////////////////////////////
900     // ISPR PreDec Checks //
901     ///////////////////////////////////
902     prim_onehot_enc #(
903       .OneHotWidth (NIspr)
904     ) u_expected_ispr_rd_en_enc (
905       .in_i (ispr_addr_i),
906       .en_i (ispr_rd_en_i),
907       .out_o (expected_ispr_rd_en_onehot)
908     );
909
910     assign ispr_wr_en = |{ispr_bignum_wr_en_i,
ispr_base_wr_en_i};
911
912     prim_onehot_enc #(
913       .OneHotWidth (NIspr)
914     ) u_expected_ispr_wr_en_enc (
915       .in_i (ispr_addr_i),
916       .en_i (ispr_wr_en),
917       .out_o (expected_ispr_wr_en_onehot)
918     );
919
920     // SEC_CM: CTRL.REDUN
921     assign ispr_predec_error_o =
|{expected_ispr_rd_en_onehot !=
ispr_bignum_predec_i.ispr_rd_en,
expected_ispr_wr_en_onehot !=
ispr_bignum_predec_i.ispr_wr_en};
922
923     ///////////////////////////////////
924     // Shifter & Packing //
925     ///////////////////////////////////
926
927     logic [8*24-1:0] operand_a_dense;
928     logic [8*24-1:0] operand_b_dense;
929     logic [WLEN-1:0] operand_a_dense_extended;
930     logic [WLEN-1:0] operand_b_dense_extended;
931     logic [WLEN-1:0] shifter_operands_upper [2];
932     logic [WLEN-1:0] shifter_operands_lower [2];
933     logic [WLEN-1:0] shifter_in_upper;
934     logic [WLEN-1:0] shifter_in_lower;
935     logic [WLEN-1:0] shifter_res;
936
937     // Pack both operands into a 8 * 24-bit dense format by
dropping the upper 8 bits per element.
938     for (genvar i = 0; i < 8; i++) begin : gen_packing
939       assign operand_a_dense[i * 24+:24] = opera-
tion_i.operand_a[i * 32+:24];
940       assign operand_b_dense[i * 24+:24] = opera-
tion_i.operand_b[i * 32+:24];
941     end
942
943     // Extend to 256 bits. The lower dense vector is placed at
the upper part such that the
944     // shifting can operate on the concatenated dense vectors.
945     assign operand_a_dense_extended = {64'd0, operand_a_dense};
946     assign operand_b_dense_extended = {operand_b_dense, 64'd0};
947
948     // Using onehot MUXs to select between regular inputs and
dense representations reduces the
949     // overall glitching activity compared to blanking the oper-
ands and using predecoded MUXs.
950     // Note that the packer may not contain any logic!
951
952     assign shifter_operands_upper[AluShiftOpFull] = opera-
tion_i.operand_a;
953     assign shifter_operands_upper[AluShiftOpDense] = oper-
and_a_dense_extended;
954     assign shifter_operands_lower[AluShiftOpFull] = opera-
tion_i.operand_b;
955     assign shifter_operands_lower[AluShiftOpDense] = oper-
and_b_dense_extended;

```

```

958
959 // SEC_CM: DATA_REG_SW.SCA
960 prim_onehot_mux #(
961     .Width (WLEN),
962     .Inputs(2)
963 ) u_shifter_operand_upper_mux (
964     .clk_i,
965     .rst_ni,
966     .in_i (shifter_operands_upper),
967     .sel_i(alu_bignum_predec_i.shift_op_a_sel),
968     .out_o(shifter_in_upper)
969 );
970
971 // SEC_CM: DATA_REG_SW.SCA
972 prim_onehot_mux #(
973     .Width (WLEN),
974     .Inputs(2)
975 ) u_shifter_operand_lower_mux (
976     .clk_i,
977     .rst_ni,
978     .in_i (shifter_operands_lower),
979     .sel_i(alu_bignum_predec_i.shift_op_b_sel),
980     .out_o(shifter_in_lower)
981 );
982
983 otbn_vec_shifter u_vec_shifter (
984     .clk_i,
985     .rst_ni,
986     .shifter_in_upper_i(shifter_in_upper),
987     .shifter_in_lower_i(shifter_in_lower),
988     .shift_direction_i (alu_bignum_predec_i.shift_dir),
989     .shift_amt_i (alu_bignum_predec_i.shift_amt),
990     .vector_mask_i ({8{alu_bignum_predec_i.shift_mask}}),
991     .shifter_res_o (shifter_res)
992 );
993
994 // Unpack the shifted result. Zero extend the 24-bit elements
995 // from the lower 8*24 bits
996 // to 32-bit elements.
997 logic [8*24-1:0] unpack_input_blanked;
998 logic [WLEN-1:0] unpacked_res;
999
1000 // SEC_CM: DATA_REG_SW.SCA
1001 prim_blanker #(.Width(8*24)) u_unpack_input_blanker (
1002     .in_i (shifter_res[8*24-1:0]),
1003     .en_i (alu_bignum_predec_i.unpack_shifter_en),
1004     .out_o(unpack_input_blanked)
1005 );
1006
1007 for (genvar i = 0; i < 8; i++) begin : gen_unpacking
1008     assign unpacked_res[i*32+:32] = {8'b0, unpack_input_blanked[i*24+:24]};
1009 end
1010
1011 // Adders X & Y
1012
1013
1014 logic [WLEN-1:0] adder_x_op_a_blanked, adder_x_op_b_blanked;
1015 logic [WLEN-1:0] adder_x_res;
1016 logic [NVecProc-1:0] adder_x_carries_out;
1017 logic [WLEN-1:0] adder_y_op_a_blanked;
1018 logic [WLEN-1:0] adder_y_op_shifter_res_blanked;
1019 logic [WLEN-1:0] shift_mod_mux_out;
1020 logic [WLEN-1:0] x_res_operand_a_mux_out;
1021 logic adder_y_carry_0_in;
1022
1023 // SEC_CM: DATA_REG_SW.SCA
1024 prim_blanker #(.Width(WLEN)) u_adder_x_op_a_blanked (
1025     .in_i (operation_i.operand_a),
1026     .en_i (alu_bignum_predec_i.adder_x_en),
1027     .out_o(adder_x_op_a_blanked)
1028 );
1029
1030 // SEC_CM: DATA_REG_SW.SCA
1031 prim_blanker #(.Width(WLEN)) u_adder_x_op_b_blanked (
1032     .in_i (operation_i.operand_b),
1033     .en_i (alu_bignum_predec_i.adder_x_en),
1034     .out_o(adder_x_op_b_blanked)
1035 );
1036
1037 otbn_vec_adder #(
1038     .LVLEN(VLEN),
1039     .LVChunkLEN(VChunkLEN)
1040 ) u_vec_adder_x (
1041     .operand_a_i (adder_x_op_a_blanked),
1042     .operand_b_i (adder_x_op_b_blanked),
1043     .oper-
1044 and_b_invert_i(alu_bignum_predec_i.adder_x_op_b_invert),
1045     .carries_in_i
1046     (alu_bignum_predec_i.adder_x_carries_in),
1047     .use_ext_carry_i ({NVec-
1048 Proc{alu_bignum_predec_i.adder_carry_sel}}),
1049     .sum_o (adder_x_res),
1050     .carries_out_o (adder_x_carries_out)
1051 );
1052
1053 // SEC_CM: DATA_REG_SW.SCA
1054 prim_blanker #(.Width(WLEN)) u_adder_y_op_a_blanked (
1055     .in_i (operation_i.operand_a),

```

```

1053     .en_i (alu_bignum_predec_i.adder_y_op_a_en),
1054     .out_o(adder_y_op_a_blanked)
1055 );
1056
1057 assign x_res_operand_a_mux_out =
1058     alu_bignum_predec_i.x_res_operand_a_sel ? adder_x_res :
1059     adder_y_op_a_blanked;
1060
1061 // SEC_CM: DATA_REG_SW.SCA
1062 prim_blanker #(.Width(WLEN)) u_adder_y_op_shifter_blanked (
1063     .in_i (shifter_res),
1064     .en_i (alu_bignum_predec_i.adder_y_op_shifter_en),
1065     .out_o(adder_y_op_shifter_res_blanked)
1066 );
1067
1068 // Despite mod_no_intg_q_replicated is not blanked, a regular
1069 // predecoded MUX is sufficient. The
1070
1071 // reason is that SW can control the value of MOD to prevent
1072 // undesirable SCA leakage due to
1073 // combining MOD and the operand_b of adder Y.
1074 assign shift_mod_mux_out =
1075     alu_bignum_predec_i.shift_mod_sel ? adder_y_op_shifter_res_blanked
1076     :
1077     mod_no_intg_q_replicated;
1078
1079 // The carries_in bits are predecoded except the LSB. It
1080 // cannot be predecoded because its value
1081 // depends on the actual flag value.
1082 otbn_vec_adder #(
1083     .LVLEN(VLEN),
1084     .LVChunkLEN(VChunkLEN)
1085 ) u_vec_adder_y (
1086     .operand_a_i (x_res_operand_a_mux_out),
1087     .operand_b_i (shift_mod_mux_out),
1088     .oper-
1089     and_b_invert_i(alu_bignum_predec_i.adder_y_op_b_invert),
1090     .carries_in_i
1091     ({alu_bignum_predec_i.adder_y_carries_top, adder_y_carry_0_in}),
1092     .use_ext_carry_i ({NVec-
1093 Proc{alu_bignum_predec_i.adder_carry_sel}}),
1094     .sum_o (adder_y_res),
1095     .carries_out_o (adder_y_carries_out)
1096 );
1097
1098 // Modulo Result Selection
1099
1100 logic [WLEN-1:0] arithmetic_result;
1101 logic arithmetic_result_used_adder_y;
1102
1103 otbn_mod_result_selector u_mod_result_selector (
1104     .result_x_i (adder_x_res),
1105     .carries_x_i (adder_x_carries_out),
1106     .result_y_i (adder_y_res),
1107     .carries_y_i (adder_y_carries_out),
1108     .is_subtraction_i(alu_bignum_predec_i.mod_is_subtraction),
1109     // Adder X is exclusively used for modulo instructions
1110     .is_modulo_i (alu_bignum_predec_i.adder_x_en),
1111     .elen_i (alu_bignum_predec_i.alu_elen),
1112     .result_o (arithmetic_result),
1113     .adder_y_used_o (arithmetic_result_used_adder_y)
1114 );
1115
1116 // Shifter, Adders, Logic & Transposer control
1117
1118 logic [NVecProc-1:0] expected_adder_x_carries_in;
1119 logic expected_adder_x_op_b_invert;
1120 logic [NVecProc-2:0] expected_adder_y_carries_top;
1121 logic expected_adder_y_op_b_invert;
1122
1123 logic expected_adder_x_en;
1124 logic expected_x_res_operand_a_sel;
1125 logic expected_adder_y_op_a_en;
1126 logic expected_adder_y_op_shifter_en;
1127
1128 logic [1:0] expected_shift_op_a_sel;
1129 logic [1:0] expected_shift_op_b_sel;
1130 logic expected_shift_en;
1131 logic expected_shift_right;
1132 logic expected_shift_mod_sel;
1133 logic expected_logic_a_en;
1134 logic expected_logic_shifter_en;
1135 logic [3:0] expected_logic_res_sel;
1136
1137 logic expected_mod_is_subtraction;
1138 logic expected_trn_en;
1139 logic expected_trn_is_trn1;
1140 logic expected_unpack_shifter_en;
1141
1142 always_comb begin
1143     // We use vectorized adders which support 8x32b or 256b
1144     // elements stored in a WDR. These adders
1145     // are split into 8 32b processing elements. Thus we have
1146     // a carry for each processing element.
1147     // These carries are predecoded except the LSB carry for
1148     // adder Y as it depends on the flag
1149     // value.
1150     expected_adder_x_carries_in = '0;

```



```

1141 expected_adder_x_op_b_invert = 'b0;
1142 expected_adder_y_carries_top = '0;
1143 adder_y_carry_0_in = 'b0;
1144 expected_adder_y_op_b_invert = 'b0;
1145 adder_update_flags_en_raw = 'b0;
1146 logic_update_flags_en_raw = 'b0;
1147
1148 expected_adder_x_en = 'b0;
1149 expected_x_res_operand_a_sel = 'b0;
1150 expected_adder_y_op_a_en = 'b0;
1151 expected_adder_y_op_shifter_en = 'b0;
1152 expected_shift_op_a_sel = '0;
1153 expected_shift_op_b_sel = '0;
1154 expected_shift_en = 'b0;
1155 expected_shift_right = 'b0;
1156 expected_shift_mod_sel = 'b1;
1157 expected_logic_a_en = 'b0;
1158 expected_logic_shifter_en = 'b0;
1159 expected_logic_res_sel = '0;
1160
1161 expected_mod_is_subtraction = 'b0;
1162 expected_trn_en = 'b0;
1163 expected_trn_is_trn1 = 'b0;
1164
1165 expected_unpack_shifter_en = 'b0;
1166
1167 unique case (operation_i.op)
1168   AluOpBigNumAdd,
1169   AluOpBigNumAddv: begin
1170     // Shifter computes B [>>|<<] shift_amt, no shift for
vectorized variant.
1171     // Y computes A + shifter_res
1172     // X ignored
1173     expected_adder_y_carries_top = '0;
1174     adder_y_carry_0_in = 'b0;
1175     expected_adder_y_op_b_invert = 'b0;
1176
1177     expected_adder_y_op_shifter_en = 'b1;
1178     expected_adder_y_op_a_en = 'b1;
1179     expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1180     expected_shift_en = 'b1;
1181
1182     if (operation_i.op == AluOpBigNumAdd) begin
1183       adder_update_flags_en_raw = 'b1;
1184       expected_shift_right = operation_i.shift_right;
1185     end
1186   end
1187   AluOpBigNumAddv: begin
1188     // Shifter computes B [>>|<<] shift_amt
1189     // Y computes A + shifter_res + flags.C
1190     // X ignored
1191     expected_adder_y_carries_top = '0;
1192     adder_y_carry_0_in = selected_flags.C;
1193     expected_adder_y_op_b_invert = 'b0;
1194     adder_update_flags_en_raw = 'b1;
1195
1196     expected_adder_y_op_shifter_en = 'b1;
1197     expected_adder_y_op_a_en = 'b1;
1198     expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1199     expected_shift_en = 'b1;
1200     expected_shift_right = operation_i.shift_right;
1201   end
1202   AluOpBigNumAddm,
1203   AluOpBigNumAddvm: begin
1204     // X computes A + B
1205
1206     // Y computes adder_x_res - mod = adder_x_res + ~mod +
1
1207     // Shifter ignored
1208
1209     // Mod result selector chooses result based on carries
of X and Y (whether mod subtraction
1210     // in Y should be applied or not).
1211     expected_adder_x_carries_in = '0;
1212     expected_adder_x_op_b_invert = 'b0;
1213     expected_adder_y_carries_top = operation_i.op ==
AluOpBigNumAddm ? '0 :
1214
1215     {(NVecProc-1){1'b1}};
1216     adder_y_carry_0_in = 'b1;
1217     expected_adder_y_op_b_invert = 'b1;
1218
1219     expected_adder_x_en = 'b1;
1220     expected_x_res_operand_a_sel = 'b1;
1221     expected_shift_mod_sel = 'b0;
1222   end
1223   AluOpBigNumSub,
1224   AluOpBigNumSubv: begin
1225     // Shifter computes B [>>|<<] shift_amt, no shift for
vectorized variant.
1226     // Y computes A - shifter_res = A + ~shifter_res + 1
1227     // X ignored
1228     expected_adder_y_carries_top = operation_i.op ==
AluOpBigNumSub ? '0 :
1229
1230     {(NVecProc-1){1'b1}};
1231     adder_y_carry_0_in = 'b1;
1232     expected_adder_y_op_b_invert = 'b1;

```

```

1229 expected_adder_y_op_shifter_en = 'b1;
1230
1231 expected_adder_y_op_a_en = 'b1;
1232 expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1233 expected_shift_en = 'b1;
1234
1235 if (operation_i.op == AluOpBigNumSub) begin
1236   adder_update_flags_en_raw = 'b1;
1237   expected_shift_right = operation_i.shift_right;
1238 end
1239 end
1240 AluOpBigNumSubv: begin
1241   // Shifter computes B [>>|<<] shift_amt
1242   // Y computes A - shifter_res + ~flags.C = A +
~shifter_res + flags.C
1243   // X ignored
1244   expected_adder_y_carries_top = '0;
1245   adder_y_carry_0_in = ~selected_flags.C;
1246   expected_adder_y_op_b_invert = 'b1;
1247   adder_update_flags_en_raw = 'b1;
1248   expected_adder_y_op_shifter_en = 'b1;
1249
1250   expected_adder_y_op_a_en = 'b1;
1251   expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1252   expected_shift_en = 'b1;
1253   expected_shift_right = operation_i.shift_right;
1254 end
1255 AluOpBigNumSubm,
1256 AluOpBigNumSubvm: begin
1257   // X computes A - B = A + ~B + 1
1258   // Y computes adder_x_res + mod
1259   // Shifter ignored
1260   // Mod result selector chooses result based on carry
of X (whether subtraction in Y should
1261   // be applied or not).
1262   expected_adder_x_carries_in = operation_i.op ==
AluOpBigNumSubm ? NVecProc'(1) :
1263
1264   {(NVecProc){1'b1}};
1265   expected_adder_x_op_b_invert = 'b1;
1266   expected_adder_y_carries_top = '0;
1267   adder_y_carry_0_in = 'b0;
1268   expected_adder_y_op_b_invert = 'b0;
1269
1270   expected_adder_x_en = 'b1;
1271   expected_x_res_operand_a_sel = 'b1;
1272   expected_shift_mod_sel = 'b0;
1273   expected_mod_is_subtraction = 'b1;
1274 end
1275 AluOpBigNumRshi: begin
1276   // Shifter computes {A, B} >> shift_amt
1277   // X, Y ignored
1278   // Feed blanked shifter output (adder_y_op_shifter_res_blanked) to Y to avoid undesired
1279   // leakage in the zero flag computation.
1280   expected_shift_op_a_sel[AluShiftOpFull] = 'b1;
1281   expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1282   expected_shift_en = 'b1;
1283   expected_shift_right = 'b0;
1284   // The shifter result is pushed through the logic
block to save an input to the result mux.
1285   // logic operation set to OR.
1286   expected_logic_shifter_en = 'b1;
1287   expected_logic_res_sel[AluOpLogicOr] = 'b1;
1288 end
1289 AluOpBigNumXor,
1290 AluOpBigNumOr,
1291 AluOpBigNumAnd,
1292 AluOpBigNumNot: begin
1293   // Shift computes one operand for the logical operation
1294   // X & Y ignored
1295   // Feed blanked shifter output (adder_y_op_shifter_res_blanked) to Y to avoid undesired
1296   // leakage in the zero flag computation.
1297   logic_update_flags_en_raw = 'b1;
1298
1299   expected_shift_op_b_sel[AluShiftOpFull] = 'b1;
1300   expected_shift_en = 'b1;
1301   expected_shift_right = operation_i.shift_right;
1302   expected_logic_a_en = operation_i.op != AluOpBigNumNot;
1303   expected_logic_shifter_en = 'b1;
1304   expected_logic_res_sel[AluOpLogicXor] = operation_i.op == AluOpBigNumXor;
1305   expected_logic_res_sel[AluOpLogicOr] = operation_i.op == AluOpBigNumOr;
1306   expected_logic_res_sel[AluOpLogicAnd] = operation_i.op == AluOpBigNumAnd;
1307   expected_logic_res_sel[AluOpLogicNot] = operation_i.op == AluOpBigNumNot;
1308 end
1309 AluOpBigNumTrn1,
1310 AluOpBigNumTrn2: begin
1311   expected_trn_en = 'b1;
1312   expected_trn_is_trn1 = operation_i.op == AluOp-
BigNumTrn1;
1313 end

```

```

1313     AluOpBignumShv: begin
1314         expected_shift_op_b_sel[AluShiftOpFull] = 'b1';
1315         expected_shift_en = 'b1';
1316         expected_shift_right = operation_i.shift_right;
1317         // The shifter result is pushed through the logic
1318         // block to save an input to the result mux.
1319         // logic operation set to OR.
1320         expected_logic_shifter_en = 'b1';
1321         expected_logic_res_sel[AluOpLogicOr] = 'b1';
1322     end
1323     AluOpBignumPack: begin
1324         expected_shift_op_a_sel[AluShiftOpDense] = 'b1';
1325         expected_shift_op_b_sel[AluShiftOpDense] = 'b1';
1326         expected_shift_en = 'b1';
1327         expected_shift_right = 'b1';
1328         // The packed result (shifter result) is pushed
1329         // through the logic block with operand A
1330         // blanked and the logic operation set to OR.
1331         expected_logic_shifter_en = 'b1';
1332         expected_logic_res_sel[AluOpLogicOr] = 'b1';
1333     end
1334     AluOpBignumUnpk: begin
1335         expected_shift_op_a_sel[AluShiftOpFull] = 'b1';
1336         expected_shift_op_b_sel[AluShiftOpFull] = 'b1';
1337         expected_shift_en = 'b1';
1338         expected_shift_right = 'b1';
1339         expected_unpack_shifter_en = 'b1';
1340     end
1341     // No operation, do nothing.
1342     AluOpBignumNone: ;
1343     default: ;
1344 endcase
1345 end
1346
1347 alu_elen_e          expected_alu_elen;
1348 trn_elen_e         expected_trn_elen;
1349 logic              expected_adder_carry_sel;
1350 logic [$clog2(WLEN)-1:0] expected_shift_amt;
1351 logic [VChunkLEN-1:0] expected_shift_mask;
1352 logic [1:0]         expected_shift_dir;
1353
1354 assign expected_alu_elen      = operation_i.alu_elen;
1355 assign expected_trn_elen     = operation_i.trn_elen;
1356 assign expected_adder_carry_sel = operation_i.adder_carry_sel;
1357 assign expected_shift_amt     = operation_i.shift_amt;
1358 assign expected_shift_mask   = operation_i.shift_mask;
1359
1360 assign expected_shift_dir[AluShiftDirLeft] = expected_shift_en &
1361                                     !expected_shift_right;
1362 assign expected_shift_dir[AluShiftDirRight] = expected_shift_en &
1363                                     expected_shift_right;
1364
1365 // SEC_CM: CTRL.REDUN
1366 assign alu_predec_error_o =
1367     !{expected_adder_x_carries_in !=
1368     alu_bignum_predec_i.adder_x_carries_in,
1369     expected_adder_x_op_b_invert !=
1370     alu_bignum_predec_i.adder_x_op_b_invert,
1371     expected_adder_y_carries_top !=
1372     alu_bignum_predec_i.adder_y_carries_top,
1373     expected_adder_y_op_b_invert !=
1374     alu_bignum_predec_i.adder_y_op_b_invert,
1375     expected_adder_x_en != alu_bignum_predec_i.adder_x_en,
1376     expected_x_res_operand_a_sel !=
1377     alu_bignum_predec_i.x_res_operand_a_sel,
1378     expected_adder_y_op_a_en !=
1379     alu_bignum_predec_i.adder_y_op_a_en,
1380     expected_adder_y_op_shifter_en !=
1381     alu_bignum_predec_i.adder_y_op_shifter_en,
1382     expected_shift_op_a_sel !=
1383     alu_bignum_predec_i.shift_op_a_sel,
1384     expected_shift_op_b_sel !=
1385     alu_bignum_predec_i.shift_op_b_sel,
1386     expected_shift_dir != alu_bignum_predec_i.shift_dir,
1387     expected_shift_amt != alu_bignum_predec_i.shift_amt,
1388     expected_shift_mod_sel !=
1389     alu_bignum_predec_i.shift_mod_sel,
1390     expected_logic_a_en != alu_bignum_predec_i.logic_a_en,
1391     expected_logic_shifter_en !=
1392     alu_bignum_predec_i.logic_shifter_en,
1393     expected_logic_res_sel !=
1394     alu_bignum_predec_i.logic_res_sel,
1395     expected_flag_group_sel !=
1396     alu_bignum_predec_i.flag_group_sel,
1397     expected_flag_sel != alu_bignum_predec_i.flag_sel,
1398     expected_flags_keep != alu_bignum_predec_i.flags_keep,
1399     expected_flags_adder_update !=
1400     alu_bignum_predec_i.flags_adder_update,
1401     expected_flags_logic_update !=
1402     alu_bignum_predec_i.flags_logic_update,
1403     expected_flags_mac_update !=
1404     alu_bignum_predec_i.flags_mac_update,
1405     expected_flags_ispr_wr !=
1406     alu_bignum_predec_i.flags_ispr_wr,
1407     expected_alu_elen != alu_bignum_predec_i.alu_elen,

```

```

1389     expected_trn_elen != alu_bignum_predec_i.trn_elen,
1390     expected_adder_carry_sel !=
1391     alu_bignum_predec_i.adder_carry_sel,
1392     expected_mod_is_subtraction !=
1393     alu_bignum_predec_i.mod_is_subtraction,
1394     expected_shift_mask != alu_bignum_predec_i.shift_mask,
1395     expected_trn_en != alu_bignum_predec_i.trn_en,
1396     expected_trn_is_trn1 != alu_bignum_predec_i.trn_is_trn1,
1397     expected_unpack_shifter_en !=
1398     alu_bignum_predec_i.unpack_shifter_en;
1399
1400 // Logical operations //
1401 // =====
1402 logic [WLEN-1:0] logical_res_mux_in [4];
1403 logic [WLEN-1:0] logical_op_a_blanked;
1404 logic [WLEN-1:0] logical_op_shifter_res_blanked;
1405
1406 // SEC_CM: DATA_REG_SW.SCA
1407 prim_blanker #(.Width(WLEN)) u_logical_op_a_blanker (
1408     .in_i (operation_i.operand_a),
1409     .en_i (alu_bignum_predec_i.logic_a_en),
1410     .out_o(logical_op_a_blanked)
1411 );
1412
1413 // SEC_CM: DATA_REG_SW.SCA
1414 prim_blanker #(.Width(WLEN)) u_logical-
1415 al_op_shifter_res_blanker (
1416     .in_i (shifter_res),
1417     .en_i (alu_bignum_predec_i.logic_shifter_en),
1418     .out_o(logical_op_shifter_res_blanked)
1419 );
1420
1421 assign logical_res_mux_in[AluOpLogicXor] = logic-
1422 al_op_a_blanked ^ logical_op_shifter_res_blanked;
1423 assign logical_res_mux_in[AluOpLogicOr] = logic-
1424 al_op_a_blanked | logical_op_shifter_res_blanked;
1425 assign logical_res_mux_in[AluOpLogicAnd] = logic-
1426 al_op_a_blanked & logical_op_shifter_res_blanked;
1427 assign logical_res_mux_in[AluOpLogicNot] = ~logic-
1428 al_op_shifter_res_blanked;
1429
1430 // SEC_CM: DATA_REG_SW.SCA
1431 prim_onehot_mux #(
1432     .Width(WLEN),
1433     .Inputs(4)
1434 ) u_logical_res_mux (
1435     .clk_i,
1436     .rst_ni,
1437     .in_i (logical_res_mux_in),
1438     .sel_i (alu_bignum_predec_i.logic_res_sel),
1439     .out_o(logical_res)
1440 );
1441
1442 // Vector Transposer //
1443 // =====
1444 logic [WLEN-1:0] vec_transposer_op_a_blanked;
1445 logic [WLEN-1:0] vec_transposer_op_b_blanked;
1446 logic [WLEN-1:0] vec_transposer_res;
1447
1448 // SEC_CM: DATA_REG_SW.SCA
1449 prim_blanker #(.Width(WLEN)) u_vec_transposer_op_a_blanker (
1450     .in_i (operation_i.operand_a),
1451     .en_i (alu_bignum_predec_i.trn_en),
1452     .out_o(vec_transposer_op_a_blanked)
1453 );
1454
1455 // SEC_CM: DATA_REG_SW.SCA
1456 prim_blanker #(.Width(WLEN)) u_vec_transposer_op_b_blanker (
1457     .in_i (operation_i.operand_b),
1458     .en_i (alu_bignum_predec_i.trn_en),
1459     .out_o(vec_transposer_op_b_blanked)
1460 );
1461
1462 otbn_vec_transposer u_vec_transposer (
1463     .clk_i,
1464     .rst_ni,
1465     .operand_a_i (vec_transposer_op_a_blanked),
1466     .operand_b_i (vec_transposer_op_b_blanked),
1467     .is_trn1_i (alu_bignum_predec_i.trn_is_trn1),
1468     .elen_i (alu_bignum_predec_i.trn_elen),
1469     .result_o (vec_transposer_res)
1470 );
1471
1472 // Output multiplexer //
1473 // =====
1474 // Due to the blanking only one input to the output multi-
1475 plexer is non zero at the same time.
1476 // This allows to implement the selection with one large OR
1477 instead of a regular mux.
1478 // Note, the shifter result is pushed through the logic
1479 block. Similarly is the non modulo result
1480 // pushed through the modulo result selector.
1481
1482 assign operation_result_o = arithmetic_result | unpacked_res
1483 | logical_res | vec_transposer_res;

```

```

1477 `ASSERT(OnlyOneResultActive_A,
1478     $onehot0({|arithmetic_result,
1479         |unpacked_res,
1480         |logical_res,
1481         |vec_transposer_res})),
1482     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1483
1484 // We need to generate a signal indicating whether we have
used the value of the MOD WSR and thus
1485 // need to check its integrity. See otbn_mod_result_selector
for more details.
1486 logic adder_y_res_used;
1487 always_comb begin
1488     unique case(operation_i.op)
1489         AluOpBignumAdd,
1490         AluOpBignumAddv,
1491         AluOpBignumAddc,
1492         AluOpBignumSub,
1493         AluOpBignumSubv,
1494         AluOpBignumSubb,
1495         AluOpBignumAddm,
1496         AluOpBignumAddvm,
1497         AluOpBignumSubm,
1498         AluOpBignumSubvm: adder_y_res_used = arithmet-
ic_result_used_adder_y;
1499
1500         AluOpBignumRshi,
1501         AluOpBignumXor,
1502         AluOpBignumOr,
1503         AluOpBignumAnd,
1504         AluOpBignumNot,
1505         AluOpBignumTrn1,
1506         AluOpBignumTrn2,
1507         AluOpBignumShv,
1508         AluOpBignumPack,
1509         AluOpBignumUnpk: adder_y_res_used = 1'b0;
1510
1511         // Most arithmetic operations use the adder Y result, so
mark it as used as save default.
1512         default: adder_y_res_used = 1'b1;
1513     endcase
1514 end
1515
1516 // Tie off unused signals.
1517 logic unused_operation_commit;
1518 assign unused_operation_commit = operation_commit_i;
1519
1520 // Determine if `mod_intg_q` is used. The control signals
are only valid if `operation_i.op` is
1521 // not none. If `shift_mod_sel` is low, `mod_intg_q` flows
into `adder_y_op_b` and from there
1522 // into `adder_y_res`. In this case, `mod_intg_q` is used
iff `adder_y_res` flows into
1523 // `operation_result_o`.
1524 logic mod_used;
1525 assign mod_used = operation_valid_i & (operation_i.op !=
AluOpBignumNone) &
1526     !alu_bignum_predec_i.shift_mod_sel & ad-
der_y_res_used;
1527 `ASSERT_KNOWN(ModUsed_A, mod_used)
1528
1529 // Raise a register integrity violation error iff
`mod_intg_q` is used and (at least partially)
1530 // invalid.
1531 assign reg_intg_violation_err_o = mod_used & |
(mod_intg_err);
1532 `ASSERT_KNOWN(RegIntgErrKnown_A, reg_intg_violation_err_o)
1533
1534 // Detect and signal unexpected secure wipe signals.
1535 assign sec_wipe_err_o = sec_wipe_mod_urnd_i &
~sec_wipe_running_i;
1536
1537 // Blanking Assertions
1538 // All blanking assertions are reset with predec_error or
overall error in the whole system
1539 // -indicated by operation_commit_i port- as OTBN does not
guarantee blanking in the case
1540 // of an error.
1541
1542 // adder_x_res related blanking
1543 `ASSERT(BlankingBignumAluXOp_A,
1544     !expected_adder_x_en
1545     |> {adder_x_op_a_blanked, adder_x_op_b_blanked, ad-
der_x_res, adder_x_carries_out} == '0,
1546     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1547
1548 // adder_y_res related blanking
1549 `ASSERT(BlankingBignumAluYOpA_A,
1550     !expected_adder_y_op_a_en |> adder_y_op_a_blanked
== '0,
1551     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1552
1553 `ASSERT(BlankingBignumAluYOpShft_A,
1554     !expected_adder_y_op_shifter_en |> ad-
der_y_op_shifter_res_blanked == '0,
1555     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1556
1557 // Vector transposer related blanking
1558 `ASSERT(BlankingBignumAluVecTrn_A,
1559     !expected_trn_en
1560     |> {vec_transposer_op_a_blanked, vec_trans-
poser_op_b_blanked} == '0,
1561     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1562
1563 // Vector unpack related blanking
1564 `ASSERT(BlankingBignumAluVecUnpk_A,
1565     !expected_unpack_shifter_en |> unpacked_res ==
'0,
1566     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1567
1568 // MOD ISPR Blanking
1569 `ASSERT(BlankingIsprMod_A,
1570     !(|mod_wr_en) |> ispr_mod_bignum_wdata_intg_blanked
== '0,
1571     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1572
1573 // ACC ISPR Blanking
1574 `ASSERT(BlankingIsprACC_A,
1575     !(|ispr_acc_wr_en) |>
ispr_acc_bignum_wdata_intg_blanked == '0,
1576     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1577
1578 endmodule

```

```

1555 // Adder Y must be blanked when its result is not used, with
one exception: For `BN.SUBM` and
1556 // `BN.SUBVM` with `a >= b` (thus the result of Adder X has
the carry bit set), the result of
1557 // Adder Y is not used but it cannot be blanked solely based
on the carry bit.
1558 `ASSERT(BlankingBignumAluYResUsed_A,
1559     !adder_y_res_used &&
1560     !((operation_i.op == AluOpBignumSubm || opera-
tion_i.op == AluOpBignumSubvm)
1561     && !arithmetic_result_used_adder_y)
1562     |> {x_res_operand_a_mux_out, shift_mod_mux_out} ==
'0,
1563     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1564
1565 // shifter_res related blanking
1566 `ASSERT(BlankingBignumAluShftA_A,
1567     (expected_shift_op_a_sel == '0) |> shifter_in_upper
== '0,
1568     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1569
1570 `ASSERT(BlankingBignumAluShftB_A,
1571     (expected_shift_op_b_sel == '0) |> shifter_in_lower
== '0,
1572     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1573
1574 `ASSERT(BlankingBignumAluShftRes_A,
1575     (expected_shift_op_a_sel == '0) && (expec-
ted_shift_op_b_sel == '0) |> shifter_res == '0,
1576     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1577
1578 // logical_res related blanking
1579 `ASSERT(BlankingBignumAluLogicOpA_A,
1580     !expected_logic_a_en |> logical_op_a_blanked == '0,
1581     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1582
1583 `ASSERT(BlankingBignumAluLogicShft_A,
1584     !expected_logic_shifter_en |> logic-
al_op_shifter_res_blanked == '0,
1585     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1586
1587 `ASSERT(BlankingBignumAluLogicRes_A,
1588     !expected_logic_a_en || expected_logic_shifter_en)
|> logical_res == '0,
1589     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1590
1591 // Vector transposer related blanking
1592 `ASSERT(BlankingBignumAluVecTrn_A,
1593     !expected_trn_en
1594     |> {vec_transposer_op_a_blanked, vec_trans-
poser_op_b_blanked} == '0,
1595     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1596
1597 // Vector unpack related blanking
1598 `ASSERT(BlankingBignumAluVecUnpk_A,
1599     !expected_unpack_shifter_en |> unpacked_res ==
'0,
1600     clk_i, !rst_ni || alu_predec_error_o || !opera-
tion_commit_i)
1601
1602 // MOD ISPR Blanking
1603 `ASSERT(BlankingIsprMod_A,
1604     !(|mod_wr_en) |> ispr_mod_bignum_wdata_intg_blanked
== '0,
1605     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1606
1607 // ACC ISPR Blanking
1608 `ASSERT(BlankingIsprACC_A,
1609     !(|ispr_acc_wr_en) |>
ispr_acc_bignum_wdata_intg_blanked == '0,
1610     clk_i, !rst_ni || ispr_predec_error_o || alu_pre-
dec_error_o || !operation_commit_i)
1611
1612 endmodule

```

4. lowRISC opentitan/hw/ip/otbn/rtl/otbn_controller.sv (1,680 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6

```

```

7 /**
8  * OTBN Controller
9  */
10 module otbn_controller
11     import otbn_pkg::*;
12     #(
13         // Size of the instruction memory, in bytes
14         parameter int ImemSizeByte = 4096,
15         // Size of the data memory, in bytes
16         parameter int DmemSizeByte = 4096,
17
18         localparam int ImemAddrWidth =
19             prim_util_pkg::vbits(ImemSizeByte),
20         localparam int DmemAddrWidth =
21             prim_util_pkg::vbits(DmemSizeByte)
22     ) (
23         input logic clk_i,
24         input logic rst_ni,
25
26         input logic start_i, // start the processing at address
27         zero
28         output logic locking_o, // Controller is in or is entering
29         the locked state
30         input logic err_bit_clear_i,
31
32         input prim_mubi_pkg::mubi4_t fatal_escalate_en_i,
33         input prim_mubi_pkg::mubi4_t recov_escalate_en_i,
34         input prim_mubi_pkg::mubi4_t rma_req_i,
35         output controller_err_bits_t err_bits_o,
36         output logic recoverable_err_o,
37
38         // Next instruction selection (to instruction fetch)
39         output logic insn_fetch_req_valid_o,
40         output logic insn_fetch_req_valid_raw_o,
41         output logic [ImemAddrWidth-1:0] insn_fetch_req_addr_o,
42         output logic insn_fetch_resp_clear_o,
43
44         // Fetched/decoded instruction
45         input logic insn_valid_i,
46         input logic insn_illegal_i,
47         input logic [ImemAddrWidth-1:0] insn_addr_i,
48
49         // Decoded instruction data
50         input insn_dec_base_t insn_dec_base_i,
51         input insn_dec_bignum_t insn_dec_bignum_i,
52         input insn_dec_shared_t insn_dec_shared_i,
53
54         // Base register file
55         output logic [4:0] rf_base_wr_addr_o,
56         output logic rf_base_wr_en_o,
57         output logic rf_base_wr_commit_o,
58         output logic [31:0] rf_base_wr_data_no_intg_o,
59         output logic [BaseIntgWidth-1:0] rf_base_wr_data_intg_o,
60         output logic rf_base_wr_data_intg_sel_o,
61
62         output logic [4:0] rf_base_rd_addr_a_o,
63         output logic rf_base_rd_en_a_o,
64         input logic [BaseIntgWidth-1:0] rf_base_rd_data_a_intg_i,
65         output logic [4:0] rf_base_rd_addr_b_o,
66         output logic rf_base_rd_en_b_o,
67         input logic [BaseIntgWidth-1:0] rf_base_rd_data_b_intg_i,
68         output logic rf_base_rd_commit_o,
69
70         input logic rf_base_call_stack_sw_err_i,
71         input logic rf_base_call_stack_hw_err_i,
72
73         // Bignum register file (WDRs)
74         output logic [4:0] rf_bignum_wr_addr_o,
75         output logic [1:0] rf_bignum_wr_en_o,
76         output logic rf_bignum_wr_commit_o,
77         output logic [WLEN-1:0] rf_bignum_wr_data_no_intg_o,
78         output logic [ExtWLEN-1:0] rf_bignum_wr_data_intg_o,
79         output logic rf_bignum_wr_data_intg_sel_o,
80
81         output logic [4:0] rf_bignum_rd_addr_a_o,
82         output logic rf_bignum_rd_en_a_o,
83         input logic [ExtWLEN-1:0] rf_bignum_rd_data_a_intg_i,
84
85         output logic [4:0] rf_bignum_rd_addr_b_o,
86         output logic rf_bignum_rd_en_b_o,
87         input logic [ExtWLEN-1:0] rf_bignum_rd_data_b_intg_i,
88
89         input logic rf_bignum_intg_err_i,
90         input logic rf_bignum_spurious_we_err_i,
91
92         output logic [NWdr-1:0] rf_bignum_rd_a_indirect_onehot_o,
93         output logic [NWdr-1:0] rf_bignum_rd_b_indirect_onehot_o,
94         output logic [NWdr-1:0] rf_bignum_wr_indirect_onehot_o,
95         output logic rf_bignum_indirect_en_o,
96
97         // Execution units
98         // Base ALU
99         output alu_base_operation_t alu_base_operation_o,
100         output alu_base_comparison_t alu_base_comparison_o,
101         input logic [31:0] alu_base_operation_result_i,
102         input logic alu_base_comparison_result_i,
103
104         // Bignum ALU
105         output alu_bignum_operation_t alu_bignum_operation_o,
106         output logic alu_bignum_operation_valid_o,
107
108         output logic alu_bignum_operation_commit_o,
109         input logic [WLEN-1:0] alu_bignum_operation_result_i,
110         input logic alu_bignum_operation_valid_i,
111         output logic mac_bignum_en_o,
112         output logic mac_bignum_commit_o,
113
114         // LSU
115         output logic lsu_load_req_o,
116         output logic lsu_store_req_o,
117         output logic insn_subset_e,
118         output logic [DmemAddrWidth-1:0] lsu_addr_o,
119         input logic lsu_addr_en_predec_i,
120
121         output logic [BaseIntgWidth-1:0] lsu_base_wdata_o,
122         output logic [ExtWLEN-1:0] lsu_bignum_wdata_o,
123
124         input logic [BaseIntgWidth-1:0] lsu_base_rdata_i,
125         input logic [ExtWLEN-1:0] lsu_bignum_rdata_i,
126
127         // Internal Special-Purpose Registers (ISPRs)
128         output ispr_e ispr_addr_o,
129         output logic [31:0] ispr_base_wdata_o,
130         output logic [BaseWordsPerWLEN-1:0] ispr_base_wr_en_o,
131         output logic [ExtWLEN-1:0] ispr_bignum_wdata_intg_o,
132         output logic ispr_bignum_wr_en_o,
133         output logic [NFlagGroups-1:0] ispr_flags_wr_o,
134         output logic ispr_wr_commit_o,
135         input logic [ExtWLEN-1:0] ispr_rdata_intg_i,
136         output logic ispr_rd_en_o,
137
138         // MAI
139         input logic mai_software_error_i,
140
141         // RND interface
142         output logic rnd_req_o,
143         output logic rnd_prefetch_req_o,
144         input logic rnd_valid_i,
145
146         input logic urnd_reseed_err_i,
147
148         // WFI
149         output logic wfi_pending_o,
150         input logic wfi_resume_i,
151
152         // Secure Wipe
153         output logic secure_wipe_req_o,
154         input logic secure_wipe_ack_i,
155         input logic sec_wipe_zero_i,
156         input logic secure_wipe_running_i,
157         input logic sec_wipe_err_i,
158
159         input logic state_reset_i,
160         output logic [31:0] insn_cnt_o,
161         input logic insn_cnt_clear_ext_i,
162         input logic insn_cnt_clear_int_i,
163         output logic mems_sec_wipe_o,
164
165         input logic software_errs_fatal_i,
166
167         input logic [1:0] sideload_key_shares_valid_i,
168
169         // Prefetch stage control
170         output logic prefetch_en_o,
171         output logic prefetch_loop_active_o,
172         output logic [31:0] prefetch_loop_iterations_o,
173         output logic [ImemAddrWidth:0] prefetch_loop_end_addr_o,
174         output logic [ImemAddrWidth-1:0] prefetch_loop_jump_addr_o,
175         output logic prefetch_ignore_errs_o,
176
177         // Predecoded control
178         input ctrl_flow_predec_t ctrl_flow_predec_i,
179         input logic [ImemAddrWidth-1:0] ctrl_flow_target_predec_i,
180         output logic predec_error_o,
181     );
182     import prim_mubi_pkg::*;
183
184     otbn_state_e state_q, state_d;
185
186     controller_err_bits_t err_bits_q, err_bits_d;
187
188     // The specific error signals that go into err_bits_d
189     logic fatal_software_err, bad_internal_state_err,
190     reg_intg_violation_err, key_invalid_err;
191     logic illegal_insn_err, bad_data_addr_err,
192     call_stack_sw_err, bad_insn_addr_err;
193
194     logic err;
195     logic internal_err;
196     logic recoverable_err;
197     logic software_err;
198     logic non_insn_addr_software_err;
199     logic fatal_err;
200     logic internal_fatal_err;
201     logic done_complete;

```

```

202 logic executing;
203 logic state_error, state_error_d, state_error_q;
204 logic spurious_secure_wipe_ack_q, spuri-
ous_secure_wipe_ack_d;
205 logic sec_wipe_err_q, sec_wipe_err_d;
206 logic mubi_err_q, mubi_err_d;
207
208 logic          insn_fetch_req_valid_raw;
209 logic [ImemAddrWidth-1:0] insn_fetch_req_addr_last;
210
211 logic stall;
212 logic ispr_stall;
213 logic mem_stall;
214 logic rf_indirect_stall;
215 logic mac_bignum_stall;
216 logic wfi_stall;
217 logic wfi_pending_d, wfi_pending_q;
218 logic jump_or_branch;
219 logic branch_taken;
220 logic insn_executing;
221 logic ld_insn_with_addr_from_call_stack, st_in-
sn_with_addr_from_call_stack;
222 logic [ImemAddrWidth-1:0] branch_target;
223 logic          branch_target_overflow;
224 logic [ImemAddrWidth:0] next_insn_addr_wide;
225 logic [ImemAddrWidth-1:0] next_insn_addr;
226
227 csr_e          csr_addr;
228 logic [$clog2(BaseWordsPerWLEN)-1:0] csr_sub_addr;
229 logic [31:0] csr_rdata_raw;
230 logic [31:0] csr_rdata;
231 logic [BaseWordsPerWLEN-1:0] csr_rdata_mux [32];
232 logic [31:0] csr_wdata_raw;
233 logic [31:0] csr_wdata;
234
235 wsr_e          wsr_addr;
236 logic [WLEN-1:0] wsr_wdata;
237
238 ispr_e          ispr_addr_base;
239 logic [$clog2(BaseWordsPerWLEN)-1:0] ispr_word_addr_base;
240 logic [BaseWordsPerWLEN-1:0] ispr_word_sel_base;
241
242 ispr_e          ispr_addr_bignum;
243
244 logic          ispr_wr_insn,
ispr_rd_insn;
245 logic          ispr_wr_base_insn;
246 logic          ispr_wr_bignum_insn;
247 logic          ispr_rd_bignum_insn;
248
249 logic          lsu_load_req_raw;
250 logic          lsu_store_req_raw;
251 logic [DmemAddrWidth-1:0] lsu_addr, lsu_addr_blanked,
lsu_addr_saved_d, lsu_addr_saved_q;
252 logic          lsu_addr_saved_sel;
253 logic          expected_lsu_addr_en;
254
255 logic          expected_call_stack_push, expec-
ted_call_stack_pop;
256 logic          lsu_preddec_error,
branch_target_preddec_error, ctrl_preddec_error;
257
258 logic rnd_req_raw;
259
260 // Register read data with integrity stripped off
261 logic [31:0] rf_base_rd_data_a_no_intg;
262 logic [31:0] rf_base_rd_data_b_no_intg;
263 logic [WLEN-1:0] rf_bignum_rd_data_a_no_intg;
264 logic [WLEN-1:0] rf_bignum_rd_data_b_no_intg;
265
266 logic [ExtWLEN-1:0] rf_bignum_rd_data_b_intg_blanked;
267 logic [ExtWLEN-1:0] selection_result;
268
269 logic [1:0] rf_bignum_wr_en_unbuf;
270 logic [4:0] rf_bignum_wr_addr_unbuf;
271 logic [4:0] rf_bignum_rd_addr_a_unbuf;
272 logic          rf_bignum_rd_en_a_unbuf;
273 logic [4:0] rf_bignum_rd_addr_b_unbuf;
274 logic          rf_bignum_rd_en_b_unbuf;
275
276 logic rf_bignum_rd_a_indirect_en;
277 logic rf_bignum_rd_b_indirect_en;
278 logic rf_bignum_wr_indirect_en;
279
280 // Computed increments for indirect register index and
memory address in BN.LID/BN.SID/BN.MOVR
281 // instructions.
282 logic [5:0] rf_base_rd_data_a_inc;
283 logic [5:0] rf_base_rd_data_b_inc;
284 logic [26:0] rf_base_rd_data_a_wlen_word_inc;
285
286 // Read/Write enables for base register file before illegal
instruction encoding are factored in
287 logic rf_base_rd_en_a_raw, rf_base_rd_en_b_raw,
rf_base_wr_en_raw;
288
289 // Output of mux taking the above increments as inputs and
choosing one to write back to base
290
291 // register file with appropriate zero extension and padding
to give a 32-bit result.

```

```

291 logic [31:0]          increment_out;
292
293 // Loop control, used to start a new loop
294 logic          loop_start_req;
295 logic          loop_start_commit;
296 logic          loop_reset;
297 logic [11:0] loop_bodysize;
298 logic [31:0] loop_iterations;
299
300 // Loop generated jumps. The loop controller asks to jump
when execution reaches the end of a loop
301 // body that hasn't completed all of its iterations.
302 logic          loop_jump;
303 logic [ImemAddrWidth-1:0] loop_jump_addr;
304
305 logic [WLEN-1:0] mac_bignum_rf_wr_data;
306
307 logic loop_hw_err, loop_preddec_err;
308 logic csr_illegal_addr, wsr_illegal_addr, ispr_illegal_addr;
309 logic imem_addr_err, loop_sw_err, ispr_err;
310 logic dmem_addr_err_check, dmem_addr_err;
311 logic dmem_addr_unaligned_base, dmem_addr_unaligned_bignum,
dmem_addr_overflow;
312 logic illegal_insn_static;
313 logic key_invalid;
314
315 logic rf_a_indirect_err, rf_b_indirect_err, rf_dindir-
ect_err, rf_indirect_err;
316
317 // If we are doing an indirect access to the bignum register
file, it's possible that the
318 // address that we use for the access is architecturally un-
known. This happens if it came from x1
319 // and we've underflowed the call stack. When this happens,
we want to ignore any read data
320 // integrity errors and spurious write enable errors since
the access to the bignum register file
321 // didn't happen architecturally anyway.
322 logic ignore_rf_bignum_intg_errs;
323 logic rf_bignum_intg_err;
324 logic ignore_rf_bignum_spurious_we_errs;
325 logic rf_bignum_spurious_we_err;
326
327 logic ispr_rdata_intg_err;
328
329 logic [31:0] insn_cnt_d, insn_cnt_q;
330 logic          insn_cnt_clear;
331
332 logic [4:0] insn_bignum_rd_addr_a_q,
insn_bignum_rd_addr_b_q, insn_bignum_wr_addr_q;
333
334 logic          start_secure_wipe;
335 logic          secure_wipe_running_q, secure_wipe_running_d;
336
337 assign secure_wipe_running_d = start_secure_wipe | (se-
cure_wipe_running_q & ~secure_wipe_ack_i);
338
339 always_ff @(posedge clk_i or negedge rst_ni) begin
340   if (!rst_ni) begin
341     secure_wipe_running_q <= 1'b0;
342   end else begin
343     secure_wipe_running_q <= secure_wipe_running_d;
344   end
345 end
346 assign secure_wipe_req_o = start_secure_wipe | se-
cure_wipe_running_q;
347
348 // Spot spurious acks on the secure wipe interface. There is a
an ack at the end of the initial
349 // secure wipe, and as `secure_wipe_running_q` is only high
during secure wipes triggered by this
350 // controller, we have to ignore acks before the initial se-
cure wipe is done. Register this
351 // signal to break a circular path (a secure wipe can be
triggered by a stop, and a spurious
352 // secure wipe ack can trigger a stop).
353 always_ff @(posedge clk_i or negedge rst_ni) begin
354   if (!rst_ni) begin
355     spurious_secure_wipe_ack_q <= 1'b0;
356   end else begin
357     spurious_secure_wipe_ack_q <= spuri-
ous_secure_wipe_ack_d;
358   end
359 end
360 assign spurious_secure_wipe_ack_d = spuri-
ous_secure_wipe_ack_q |
361 (secure_wipe_ack_i
&
362 ~secure_wipe_running_q
&
363 ~se-
cure_wipe_running_i);
364
365 // Detect and latch unexpected secure wipe signals.
366 always_ff @(posedge clk_i or negedge rst_ni) begin
367   if (!rst_ni) begin
368     sec_wipe_err_q <= 1'b0;
369   end else begin
370     sec_wipe_err_q <= sec_wipe_err_d;

```



```

371     end
372   end
373   assign sec_wipe_err_d = sec_wipe_err_q |
374     sec_wipe_err_i |
375     (sec_wipe_zero_i & ~se-
cure_wipe_running_i);
376
377   // Stall a cycle on loads to allow load data writeback to
happen the following cycle. Stall not
378   // required on stores as there is no response to deal with.
379   assign mem_stall = lsu_load_req_raw;
380
381   // Reads to RND must stall until data is available
382   assign ispr_stall = rnd_req_raw & ~rnd_valid_i;
383
384   assign rf_indirect_stall = insn_valid_i &
385     (state_q != OtnbStateStall) &
386     (insn_dec_shared_i.subset == In-
snSubsetBignum) &
387     (insn_dec_bignum_i.rf_a_indirect
|
388     insn_dec_bignum_i.rf_b_indirect
|
389     insn_dec_bignum_i.rf_d_indirect);
390
391   // Stall while a vectorized multicycle MAC calculation is
ongoing
392   assign mac_bignum_stall = insn_valid_i &
393     (insn_dec_shared_i.subset == Insn-
SubsetBignum) &
394     insn_dec_bignum_i.mac_en &
395     (~mac_bignum_operation_valid_i);
396
397   // Stall until the resume command arrives.
398   assign wfi_stall = insn_valid_i &&
insn_dec_shared_i.wfi_insn && !wfi_resume_i;
399   assign wfi_pending_d = wfi_stall;
400
401   // The resume command should only be issued when we are
already waiting.
402   `ASSERT(WfiResumeOnlyIfStalling, !wfi_pending_q |> !wfi_re-
sume_i)
403
404   // The wfi_pending is registered to avoid long paths into
the IMEM/DMEM core/bus mux.
405   always_ff @(posedge clk_i or negedge rst_ni) begin
406     if (!rst_ni) begin
407       wfi_pending_q <= 1'b0;
408     end else begin
409       wfi_pending_q <= wfi_pending_d;
410     end
411   end
412   assign wfi_pending_o = wfi_pending_q;
413
414   assign stall = mem_stall | ispr_stall | rf_indirect_stall |
mac_bignum_stall | wfi_stall;
415
416   // OTBN is done when it was executing something (in state
OtnbStateRun or OtnbStateStall)
417   // and either it executes an ecall or an error occurs. A
pulse on the done signal raises the
418   // 'done' interrupt and also tells the top-level to update
its ERR_BITS status
419   // register. The calculation that ecall triggered done is
factored out as 'done_complete' to
420   // avoid logic loops in the error handling logic.
421   assign done_complete = (insn_valid_i &
insn_dec_shared_i.ecall_insn);
422   assign executing = (state_q == OtnbStateRun) ||
423     (state_q == OtnbStateStall);
424
425   // Set the *locking* output when the next state is the
*locked* state and no secure wipe is
426   // running or there is a URND reseed error. 'locking_o' is
thus set only after the secure wipe
427   // has completed or if it cannot complete due to an URND re-
seed error (in which case
428   // 'secure_wipe_req_o' and 'urnd_reseed_err_i' will remain
high). The condition for secure wipe
429   // running involves 'secure_wipe_running_i', which is high
for the initial secure wipe, and
430   // 'secure_wipe_req_o', which is high for post-execution se-
cure wipes.
431   assign locking_o = (state_d == OtnbStateLocked) & (~(se-
cure_wipe_running_i | secure_wipe_req_o) |
432     urnd_re-
seed_err_i | mubi_err_d);
433
434   assign start_secure_wipe = executing & (done_complete |
err);
435
436   assign jump_or_branch = (insn_valid_i &
437     (insn_dec_shared_i.branch_insn |
insn_dec_shared_i.jump_insn));
438
439   // Branch taken when there is a valid branch instruction and
comparison passes or a valid jump
440   // instruction (which is always taken)
441   assign branch_taken = insn_valid_i &

```

```

442     ((insn_dec_shared_i.branch_insn &
alu_base_comparison_result_i) |
443     insn_dec_shared_i.jump_insn);
444   // Branch target computed by base ALU (PC + imm)
445   assign branch_target =
alu_base_operation_result_i[ImemAddrWidth-1:0];
446   assign branch_target_overflow = |
alu_base_operation_result_i[31:ImemAddrWidth];
447
448   assign next_insn_addr_wide = {1'b0, insn_addr_i} + 'd4;
449   assign next_insn_addr = next_in-
sn_addr_wide[ImemAddrWidth-1:0];
450
451   // Record address for fetch request so it can be retried
when an invalid response is received
452   always_ff @(posedge clk_i) begin
453     if (insn_fetch_req_valid_raw) begin
454       insn_fetch_req_addr_last <= insn_fetch_req_addr_o;
455     end
456   end
457
458   always_comb begin
459     state_d = state_q;
460     // 'insn_fetch_req_valid_raw' is the value
'insn_fetch_req_valid_o' before any errors are
461     // considered.
462     insn_fetch_req_valid_raw = 1'b0;
463     insn_fetch_req_addr_o = '0;
464     insn_fetch_resp_clear_o = 1'b1;
465     prefetch_en_o = 1'b0;
466
467     state_error = 1'b0;
468
469     unique case (state_q)
470       OtnbStateHalt: begin
471         if (start_i) begin
472           state_d = OtnbStateRun;
473
474           insn_fetch_req_addr_o = '0;
475           insn_fetch_req_valid_raw = 1'b1;
476           prefetch_en_o = 1'b1;
477         end
478       end
479       OtnbStateRun: begin
480         insn_fetch_req_valid_raw = 1'b1;
481         prefetch_en_o = 1'b1;
482
483         if (!insn_valid_i) begin
484           insn_fetch_req_addr_o = insn_fetch_req_addr_last;
485         end else if (done_complete) begin
486           state_d = OtnbStateHalt;
487           insn_fetch_req_valid_raw = 1'b0;
488           prefetch_en_o = 1'b0;
489         end else begin
490           if (stall) begin
491             // When stalling don't request a new fetch and
don't clear response either to keep
492             // current instruction.
493             state_d = OtnbStateStall;
494             insn_fetch_req_valid_raw = 1'b0;
495             insn_fetch_resp_clear_o = 1'b0;
496           end else begin
497             if (branch_taken) begin
498               insn_fetch_req_addr_o = branch_target;
499             end else if (loop_jump) begin
500               insn_fetch_req_addr_o = loop_jump_addr;
501             end else begin
502               insn_fetch_req_addr_o = next_insn_addr;
503             end
504           end
505         end
506       end
507       OtnbStateStall: begin
508         prefetch_en_o = 1'b1;
509         // When stalling refetch the same instruction to keep
decode inputs constant
510         if (stall) begin
511           state_d = OtnbStateStall;
512           //insn_fetch_req_addr_o = insn_addr_i;
513           insn_fetch_req_valid_raw = 1'b0;
514           insn_fetch_resp_clear_o = 1'b0;
515         end else begin
516           insn_fetch_req_valid_raw = 1'b1;
517
518           if (loop_jump) begin
519             insn_fetch_req_addr_o = loop_jump_addr;
520           end else begin
521             insn_fetch_req_addr_o = next_insn_addr;
522           end
523
524           state_d = OtnbStateRun;
525         end
526       end
527       OtnbStateLocked: begin
528         insn_fetch_req_valid_raw = 1'b0;
529         state_d = OtnbStateLocked;
530       end
531       default: begin
532         // We should never get here. If we do (e.g. via a ma-
licious glitch), error out immediately.
533         // SEC_CM: CONTROLLER.FSM.LOCAL_ESC

```

```

534     state_d = OtbNStateLocked;
535     state_error = 1'b1;
536   end
537 endcase
538
539 // On any error immediately halt, either going to OtbN-
StateLocked or OtbNStateHalt depending on
540 // whether it was a fatal error.
541 if (err) begin
542   insn_fetch_resp_clear_o = 1'b1;
543
544   if (fatal_err) begin
545     // SEC_CM: CONTROLLER.FSM.GLOBAL_ESC
546     state_d = OtbNStateLocked;
547   end else begin
548     state_d = OtbNStateHalt;
549   end
550 end
551
552 // Regardless of what happens above enforce staying in Ot-
nbStateLocked.
553 if (state_q == OtbNStateLocked) begin
554   state_d = OtbNStateLocked;
555 end
556 end
557
558 assign state_error_d = state_error | state_error_q;
559
560 prim_flop #(
561   .Width(1),
562   .ResetValue('0)
563 ) u_state_error_flop (
564   .clk_i,
565   .rst_ni,
566
567   .d_i(state_error_d),
568   .q_o(state_error_q)
569 );
570
571 `ASSERT(InsnAlwaysValidInStall, state_q == OtbNStateStall | -
> insn_valid_i)
572
573 // Anything that moves us or keeps us in the stall state
should cause 'stall' to be asserted
574 `ASSERT(StallIfNextStateStall, insn_valid_i & (state_d ==
OtbNStateStall) |-> stall)
575
576 // The raw signal is needed by the instruction fetch stage
for generating instruction address
577 // errors (where instruction fetch and prefetch disagree on
address). 'err' will factor this in so
578 // using the qualified signal results in a combinational
loop.
579 assign insn_fetch_req_valid_raw_o =
insn_fetch_req_valid_raw;
580 assign insn_fetch_req_valid_o = err ? 1'b0 :
insn_fetch_req_valid_raw;
581
582 // Determine if there are any errors related to the Imem
fetch address.
583 always_comb begin
584   imem_addr_err = 1'b0;
585
586   if (insn_fetch_req_valid_raw) begin
587     if (!insn_fetch_req_addr_o[1:0]) begin
588       // Imem address is unaligned
589       imem_addr_err = 1'b1;
590     end else if (branch_taken) begin
591       imem_addr_err = branch_target_overflow;
592     end else begin
593       imem_addr_err = next_insn_addr_wide[ImemAddrWidth] &
insn_valid_i;
594     end
595   end
596 end
597
598 // Signal error if MuBi input signals take on invalid values
as this means something bad is
599 // happening. Register the error signal to break circular
paths (instruction fetch errors factor
600 // into fatal_escalate_en_i, RND errors factor into re-
cov_escalate_en_i).
601 assign mubi_err_d = {mu-
bi4_test_invalid(fatal_escalate_en_i),
602   mu-
bi4_test_invalid(recov_escalate_en_i),
603   mubi_err_q};
604 always_ff @(posedge clk_i or negedge rst_ni) begin
605   if (!rst_ni) begin
606     mubi_err_q <= 1'b0;
607   end else begin
608     mubi_err_q <= mubi_err_d;
609   end
610 end
611
612 // Instruction is illegal based on the static properties of
the instruction bits (illegal encoding
613 // or illegal WSR/CSR referenced).
614 assign illegal_insn_static = insn_illegal_i | ispr_err;
615

```

```

616   assign fatal_software_err = software_err & soft-
ware_errs_fatal_i;
617   assign bad_internal_state_err = {state_error_d,
loop_hw_err, rf_base_call_stack_hw_err_i,
618   rf_bignum_spurious_we_err, spurious_secure_wipe_ack_q,
619   sec_wipe_err_q,
mubi_err_q};
620   assign reg_intg_violation_err = rf_bignum_intg_err |
ispr_rdata_intg_err;
621   assign key_invalid_err = ispr_rd_bignum_insn &
insn_valid_i & key_invalid;
622   assign illegal_insn_err = illegal_insn_static |
rf_indirect_err;
623   assign call_stack_sw_err =
rf_base_call_stack_sw_err_i;
624
625   // Flag a bad data address error if the data memory address
is invalid and it does not come from
626   // an empty call stack. The second case cannot be decided
as bad data address because the address
627   // on top of the empty call stack may or may not be valid.
(Also, in most RTL simulators an empty
628   // call stack that has never been pushed contains an unknown
value, so this error bit would become
629   // unknown.) Thus, a data memory address coming from an
empty call stack raises a call stack
630   // error but never a bad data address error.
631   assign bad_data_addr_err = dmem_addr_err &
632     ~(call_stack_sw_err &
633       (ld_in-
sn_with_addr_from_call_stack |
634         st_in-
sn_with_addr_from_call_stack));
635
636   // Identify load instructions that take the memory address
from the call stack.
637   assign ld_insn_with_addr_from_call_stack = insn_val-
id_i
638     &
insn_dec_shared_i.ld_insn &
639     insn_dec_base_i.rf_ren_a &
640     (insn_dec_base_i.a == 5'd1);
641
642   // Identify store instructions that take the memory address
from the call stack.
643   assign st_insn_with_addr_from_call_stack = insn_val-
id_i
644     &
insn_dec_shared_i.st_insn &
645     insn_dec_base_i.rf_ren_a &
646     (insn_dec_base_i.a == 5'd1);
647
648   // All software errors that aren't bad_insn_addr. Factored
into bad_insn_addr so it is only raised
649   // if other software errors haven't occurred. As bad_in-
sn_addr relates to the next instruction
650   // begin fetched it cannot occur if the current instruction
has seen an error and failed to
651   // execute.
652   assign non_insn_addr_software_err = {key_invalid_err,
loop_sw_err,
653     illegal_insn_err,
654     call_stack_sw_err,
655     bad_data_addr_err,
656     bad_insn_addr_err,
657     mai_software_error_i};
658
659   assign bad_insn_addr_err = imem_addr_err & ~non_in-
sn_addr_software_err;
660
661   assign err_bits_d = '{
662     fatal_software_err,
663     bad_internal_state_err,
664     reg_intg_violation_err,
665     key_invalid_err,
666     loop_sw_err,
667     illegal_insn_err,
668     call_stack_sw_err,
669     bad_data_addr_err,
670     bad_insn_addr_err,
671     mai_error,
672   };
673
674   always_ff @(posedge clk_i or negedge rst_ni) begin
675     if (!rst_ni) begin
676       err_bits_q <= '0;
677     end else begin
678       if (err_bit_clear_i && !locking_o) begin
679         err_bits_q <= '0;
680       end else begin
681         err_bits_q <= err_bits_q | err_bits_d;
682       end
683     end
684   end
685   assign err_bits_o = err_bits_q;
686

```



```

687 assign software_err = non_insn_addr_software_err | bad_in-
insn_addr_err;
688
689 assign recoverable_err = mu-
bi4_test_true_loose(recov_escalate_en_i);
690
691 assign internal_fatal_err = |{fatal_software_err,
692 bad_internal_state_err,
693 reg_intg_violation_err};
694
695 // In case of an RMA request, just lock up the controller.
This triggers the rotation of the
696 // scrambling keys. The start/stop controller takes care of
initiating the internal secure wipe
697 // and eventually acknowledging the RMA request.
698 assign fatal_err = |{internal_fatal_err,
699 mu-
bi4_test_true_loose(fatal_escalate_en_i),
700 mubi4_test_true_strict(rma_req_i)};
701
702 assign recoverable_err_o = recoverable_err | (software_err &
~software_errs_fatal_i);
703 assign mems_sec_wipe_o = (state_d == OtbNStateLocked) &
(state_q != OtbNStateLocked);
704
705 assign internal_err = software_err | internal_fatal_err;
706 assign err = software_err | recoverable_err |
fatal_err;
707
708 assign prefetch_ignore_errs_o = internal_err;
709
710 // Instructions must not execute if there is an error
711 assign insn_executing = insn_valid_i & ~err;
712
713 `ASSERT(ErrBitSetOnErr,
714 err & (mubi4_test_false_strict(fatal_escalate_en_i) &
715 mubi4_test_false_strict(recov_escalate_en_i) &
716 mubi4_test_false_loose(rma_req_i)) |>
err_bits_o)
717
718 `ASSERT(ErrSetOnFatalErr, fatal_err |> err)
719 `ASSERT(SoftwareErrIfNonInsnAddrSoftwareErr, non_in-
insn_addr_software_err |> software_err)
720
721 `ASSERT(ControllerStateValid,
722 state_q inside {OtbNStateHalt, OtbNStateRun, OtbN-
StateStall, OtbNStateLocked})
723 // Branch only takes effect in OtbNStateRun so must not go
into stall state for branch
724 // instructions.
725 `ASSERT(NoStallOnBranch,
726 insn_valid_i & insn_dec_shared_i.branch_insn |>
state_q != OtbNStateStall)
727
728 // SEC_CM: CONTROLLER.FSM.SPARSE
729 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
otbn_state_e, OtbNStateHalt)
730
731 // SEC_CM: CTRL_FLOW.COUNT
732 // Two explicit clear controls, one comes from external to
otbn_core and the other is generated
733 // internally (by otbn_start_stop_control).
734 assign insn_cnt_clear =
735 (state_q == OtbNStateLocked) | insn_cnt_clear_ext_i |
insn_cnt_clear_int_i;
736
737 always_comb begin
738 if (insn_cnt_clear) begin
739 insn_cnt_d = 32'd0;
740 end else if (insn_executing & ~stall & (insn_cnt_q !=
32'hfffffff)) begin
741 insn_cnt_d = insn_cnt_q + 32'd1;
742 end else begin
743 insn_cnt_d = insn_cnt_q;
744 end
745 end
746
747 always_ff @(posedge clk_i or negedge rst_ni) begin
748 if (!rst_ni) begin
749 insn_cnt_q <= 32'd0;
750 end else begin
751 insn_cnt_q <= insn_cnt_d;
752 end
753 end
754
755 assign insn_cnt_o = insn_cnt_q;
756
757 assign loop_reset = state_reset_i | sec_wipe_zero_i;
758
759 otbn_loop_controller #(
760 .ImemAddrWidth(ImemAddrWidth)
761 ) u_otbn_loop_controller (
762 .clk_i,
763 .rst_ni,
764
765 .state_reset_i(loop_reset),
766
767 .insn_valid_i,
768 .insn_addr_i,
769 .next_insn_addr_i(next_insn_addr),
770

```

```

771 .loop_start_req_i (loop_start_req),
772 .loop_start_commit_i (loop_start_commit),
773 .loop_bodysize_i (loop_bodysize),
774 .loop_iterations_i (loop_iterations),
775 .loop_end_addr_predec_i (ctrl_flow_target_predec_i),
776
777 .loop_jump_o (loop_jump),
778 .loop_jump_addr_o(loop_jump_addr),
779
780 .sw_err_o (loop_sw_err),
781 .hw_err_o (loop_hw_err),
782 .predec_err_o (loop_predec_err),
783
784 .jump_or_branch_i(jump_or_branch),
785 .otbn_stall_i (stall),
786
787 .prefetch_loop_active_o,
788 .prefetch_loop_iterations_o,
789 .prefetch_loop_end_addr_o,
790 .prefetch_loop_jump_addr_o
791 );
792
793 // loop_start_req indicates the instruction wishes to start
a loop, loop_start_commit confirms it
794 // should occur.
795 assign loop_start_req = insn_valid_i &
insn_dec_shared_i.loop_insn;
796 assign loop_start_commit = insn_executing;
797 assign loop_bodysize = insn_dec_base_i.loop_bodysize;
798 assign loop_iterations = insn_dec_base_i.loop_immediate ?
insn_dec_base_i.i :
799
rf_base_rd_data_a_no_intg;
800
801 // Compute increments which can be optionally applied to in-
direct register accesses and memory
802 // addresses in BN.LID/BN.SID/BN.MOVR instructions.
803 assign rf_base_rd_data_a_inc =
rf_base_rd_data_a_no_intg[4:0] + 1'b1;
804 assign rf_base_rd_data_b_inc =
rf_base_rd_data_b_no_intg[4:0] + 1'b1;
805 // We can avoid a full 32-bit adder here because the offset
is 32-bit aligned, so we know the
806 // load/store address will only be valid if
rf_base_rd_data_a_no_intg[4:0] is zero.
807 assign rf_base_rd_data_a_wlen_word_inc =
rf_base_rd_data_a_no_intg[31:5] + 27'h1;
808
809 // Choose increment to write back to base register file,
only one increment can be written as
810 // there is only one write port. Note that where an instruc-
tion is incrementing the indirect
811 // reference to its destination register
(insn_dec_bignum_i.d_inc) that reference is read on the
812 // B read port so the B increment is written back.
813 always_comb begin
814 unique case (1'b1)
815 insn_dec_bignum_i.a_inc: begin
816 increment_out = {26'b0, rf_base_rd_data_a_inc};
817 end
818 insn_dec_bignum_i.b_inc: begin
819 increment_out = {26'b0, rf_base_rd_data_b_inc};
820 end
821 insn_dec_bignum_i.d_inc: begin
822 increment_out = {26'b0, rf_base_rd_data_b_inc};
823 end
824 insn_dec_bignum_i.a_wlen_word_inc: begin
825 increment_out = {rf_base_rd_data_a_wlen_word_inc,
5'b0};
826 end
827 default: begin
828 // Whenever increment_out is written back to the re-
gister file, exactly one of the
829 // increment selector signals is high. To prevent the
automatic inference of latches in
830 // case nothing is written back (rf_wdata_sel != RfWd-
SelIncr) and to save logic, we choose
831 // a valid output as default.
832 increment_out = {26'b0, rf_base_rd_data_a_inc};
833 end
834 endcase
835 end
836
837 // Base RF read/write address, enable and commit control
always_comb begin
838 rf_base_rd_addr_a_o = insn_dec_base_i.a;
839 rf_base_rd_addr_b_o = insn_dec_base_i.b;
840 rf_base_wr_addr_o = insn_dec_base_i.d;
841
842 // Only commit read or write if the instruction is execut-
ing (in particular a read commit pops
843 // the call stack so must not occur where a valid instruc-
tion sees an error and doesn't
844 // execute).
845 rf_base_rd_commit_o = insn_executing;
846 rf_base_wr_commit_o = insn_executing;
847
848 rf_base_rd_en_a_raw = 1'b0;
849 rf_base_rd_en_b_raw = 1'b0;
850 rf_base_wr_en_raw = 1'b0;
851
852

```

```

853   if (insn_valid_i) begin
854     if (insn_dec_shared_i.st_insn) begin
855       // For stores, both base reads happen in the first
cycle of the store instruction. For base
856       // stores this is the same cycle as the request. For
bignum stores this is the cycle before
857       // the request (as the indirect register read to get
the store data occurs the following
858       // cycle).
859       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
860       (rf_indirect_stall | (insn_dec_shared_i.subset ==
InsSubsetBase));
861       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
862       (rf_indirect_stall | (insn_dec_shared_i.subset ==
InsSubsetBase));
863
864       // Bignum stores can update the base register file
where an increment is used.
865       rf_base_wr_en_raw = (insn_dec_shared_i.subset == In-
snSubsetBignum) &
866       insn_dec_base_i.rf_we
867       & rf_indirect_stall;
868     end else if (insn_dec_shared_i.ld_insn) begin

869       // For loads, both base reads happen in the same cycle
as the request. The address is
870       // required for the request and the indirect destina-
tion register (only used for Bignum
871       // loads) is flopped in ld_insn_bignum_wr_addr_q to
correctly deal with the case where it's
872       // updated by an increment.
873       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
lsu_load_req_raw;
874       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
lsu_load_req_raw;
875
876       if (insn_dec_shared_i.subset == InsSubsetBignum) be-
gin
877         // Bignum loads can update the base register file
where an increment is used. This must

878         // always happen in the same cycle as the request as
this is where both registers are
879         // read.
880         rf_base_wr_en_raw = insn_dec_base_i.rf_we &
lsu_load_req_raw & rf_indirect_stall;
881       end else begin
882         // For Base loads write the base register file when
the instruction is unstalled (meaning
883         // the load data is available).
884         rf_base_wr_en_raw = insn_dec_base_i.rf_we & ~stall;
885       end
886     end else if (insn_dec_bignum_i.rf_wdata_sel == RfwdSel-
MovSel) begin
887       // For MOVR base register reads occur in the first
cycle of the instruction. The indirect
888       // register read for the bignum data occurs in the
following cycle.
889       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
rf_indirect_stall;
890       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
rf_indirect_stall;
891       rf_base_wr_en_raw = insn_dec_base_i.rf_we &
rf_indirect_stall;
892     end else begin
893       // For all other instructions the read and write hap-
pen when the instruction is unstalled.
894       rf_base_rd_en_a_raw = insn_dec_base_i.rf_ren_a &
~stall;
895       rf_base_rd_en_b_raw = insn_dec_base_i.rf_ren_b &
~stall;
896       rf_base_wr_en_raw = insn_dec_base_i.rf_we &
~stall;
897     end
898   end
899
900   if (insn_dec_shared_i.subset == InsSubsetBignum) begin
901     unique case (i'b1)
902       insn_dec_bignum_i.a_inc,
903       insn_dec_bignum_i.a_wlen_word_inc: begin
904         rf_base_wr_addr_o = insn_dec_base_i.a;
905       end
906
907       insn_dec_bignum_i.b_inc,
908       insn_dec_bignum_i.d_inc: begin
909         rf_base_wr_addr_o = insn_dec_base_i.b;
910       end
911     default: ;
912   endcase
913 end
914
915 rf_base_rd_en_a_o = rf_base_rd_en_a_raw & ~illeg-
al_insn_static;
916 rf_base_rd_en_b_o = rf_base_rd_en_b_raw & ~illeg-
al_insn_static;
917 rf_base_wr_en_o = rf_base_wr_en_raw & ~illeg-
al_insn_static;
918 end
919
920 // Base ALU Operand A MUX

```

```

921   always_comb begin
922     unique case (insn_dec_base_i.op_a_sel)
923       OpASelRegister: alu_base_operation_o.operand_a =
rf_base_rd_data_a_no_intg;
924       OpASelZero: alu_base_operation_o.operand_a = '0;

925       OpASelCurrPc: alu_base_operation_o.operand_a = {{(32 -
ImemAddrWidth)}1'b0}}, insn_addr_i;
926     default: alu_base_operation_o.operand_a =
rf_base_rd_data_a_no_intg;
927   endcase
928 end
929
930 // Base ALU Operand B MUX
931   always_comb begin
932     unique case (insn_dec_base_i.op_b_sel)
933       OpBSelRegister: alu_base_operation_o.operand_b =
rf_base_rd_data_b_no_intg;
934       OpBSelImmediate: alu_base_operation_o.operand_b =
insn_dec_base_i.i;
935     default: alu_base_operation_o.operand_b =
rf_base_rd_data_b_no_intg;
936   endcase
937 end
938
939   assign alu_base_operation_o.op = insn_dec_base_i.alu_op;
940
941   assign alu_base_comparison_o.operand_a =
rf_base_rd_data_a_no_intg;
942   assign alu_base_comparison_o.operand_b =
rf_base_rd_data_b_no_intg;
943   assign alu_base_comparison_o.op =
insn_dec_base_i.comparison_op;
944
945   assign rf_base_rd_data_a_no_intg =
rf_base_rd_data_a_intg_i[31:0];
946   assign rf_base_rd_data_b_no_intg =
rf_base_rd_data_b_intg_i[31:0];
947
948   logic unused_rf_base_rd_a_intg_bits;
949
950   // TODO(#18266): Implement GPR to ISPR end to end integrity
path (ISPR writes from GPR take data
951   // from base RF port A)
952   assign unused_rf_base_rd_a_intg_bits = |
rf_base_rd_data_a_intg_i[38:32];
953
954   // Base register file write MUX. Depending on the data
source, integrity bits do or don't have to
955   // be appended:
956   // - Data sources that require appending integrity bits go
into `rf_base_wr_data_no_intg_o` and
957   // `rf_base_wr_data_intg_sel_o` is low.
958   // - Data sources that already come with integrity bits go
into `rf_base_wr_data_intg_o` and
959   // `rf_base_wr_data_intg_sel_o` is high.
960   always_comb begin
961     // Default values
962     rf_base_wr_data_no_intg_o = alu_base_operation_result_i;
963     rf_base_wr_data_intg_o = '0;
964     rf_base_wr_data_intg_sel_o = 1'b0;
965
966     unique case (insn_dec_base_i.rf_wdata_sel)
967       RfwdSelEx: begin
968         rf_base_wr_data_no_intg_o =
alu_base_operation_result_i;
969       end
970       RfwdSelNextPc: begin
971         rf_base_wr_data_no_intg_o = {{(32-(ImemAddrWidth+1))
{1'b0}}, next_insn_addr_wide};
972       end
973       RfwdSelIspr: begin
974         rf_base_wr_data_no_intg_o = csr_rdata;
975       end
976       RfwdSelIncr: begin
977         rf_base_wr_data_no_intg_o = increment_out;
978       end
979       RfwdSelLsu: begin
980         rf_base_wr_data_intg_sel_o = 1'b1;
981         rf_base_wr_data_intg_o = lsu_base_rdata_i;
982       end
983     default: ;
984   endcase
985 end
986
987   for (genvar i = 0; i < BaseWordsPerWLEN; ++i) begin :
g_rf_bignum_rd_data
988     assign rf_bignum_rd_data_a_no_intg[i*32+:32] =
rf_bignum_rd_data_a_intg_i[i*39+:32];
989     assign rf_bignum_rd_data_b_no_intg[i*32+:32] =
rf_bignum_rd_data_b_intg_i[i*39+:32];
990   end
991
992   // Bignum RF control signals from the controller aren't ac-
tually used, instead the predecoded

993   // one-hot versions are. The predecoded versions get checked
against the signals produced here.

994   // Buffer them to ensure they don't get optimised away (with a
functionally correct OTBN / if

```

```

995 // there is no FI attack they will always be identical).
996
997 // The rf_bignum_rd_en_stall ensures that the BN RFs data is
only read when it is actually used.
998 // For all instructions except multi-cycle multiplication we
thus must suppress the read if the
999 // core stalls. The multi-cycle multiplications however read
the data also while stalling and
1000 // thus we force the read stall signal to low. This works as
none of these instructions can
1001 // trigger any of the other stall sources (mem_stall,
ispr_stall or rf_indirect_stall). The only
1002 // possible stall reason is the MAC module itself.
1003 logic rf_bignum_rd_en_stall;
1004 assign rf_bignum_rd_en_stall = insn_dec_bignum_i.mac_en ?
1'b0 : stall;
1005
1006 assign rf_bignum_rd_addr_a_unbuf =
insn_dec_bignum_i.rf_a_indirect ? insn_bignum_rd_addr_a_q :
1007
insn_dec_bignum_i.a;
1008
1009 prim_buf #(
1010   .Width(WdrAw)
1011 ) u_rf_bignum_rd_addr_a_buf (
1012   .in_i (rf_bignum_rd_addr_a_unbuf),
1013   .out_o(rf_bignum_rd_addr_a_o)
1014 );
1015
1016 assign rf_bignum_rd_en_a_unbuf = insn_dec_bignum_i.rf_ren_a
&
1017
insn_valid_i
&
1018
~rf_bignum_rd_en_stall;
1019
1020 prim_buf #(
1021   .Width(1)
1022 ) u_rf_bignum_rd_en_a_buf (
1023   .in_i (rf_bignum_rd_en_a_unbuf),
1024   .out_o(rf_bignum_rd_en_a_o)
1025 );
1026
1027 assign rf_bignum_rd_addr_b_unbuf =
insn_dec_bignum_i.rf_b_indirect ? insn_bignum_rd_addr_b_q :
1028
insn_dec_bignum_i.b;
1029
1030 prim_buf #(
1031   .Width(WdrAw)
1032 ) u_rf_bignum_rd_addr_b_buf (
1033   .in_i (rf_bignum_rd_addr_b_unbuf),
1034   .out_o(rf_bignum_rd_addr_b_o)
1035 );
1036
1037 assign rf_bignum_rd_en_b_unbuf = insn_dec_bignum_i.rf_ren_b
&
1038
insn_valid_i
&
1039
~rf_bignum_rd_en_stall;
1040
1041 prim_buf #(
1042   .Width(1)
1043 ) u_rf_bignum_rd_en_b_buf (
1044   .in_i (rf_bignum_rd_en_b_unbuf),
1045   .out_o(rf_bignum_rd_en_b_o)
1046 );
1047
1048 assign alu_bignum_operation_o.operand_a =
rf_bignum_rd_data_a_no_intg;
1049
1050 // Bignum ALU Operand B MUX
1051 always_comb begin
1052   unique case (insn_dec_bignum_i.alu_op_b_sel)
1053     OpBSelRegister: alu_bignum_operation_o.operand_b =
rf_bignum_rd_data_b_no_intg;
1054     OpBSelImmediate: alu_bignum_operation_o.operand_b =
insn_dec_bignum_i.i;
1055     default: alu_bignum_operation_o.operand_b =
rf_bignum_rd_data_b_no_intg;
1056   endcase
1057 end
1058
1059 assign alu_bignum_operation_o.op =
insn_dec_bignum_i.alu_op;
1060 assign alu_bignum_operation_o.alu_elen =
insn_dec_bignum_i.alu_elen;
1061 assign alu_bignum_operation_o.trn_elen =
insn_dec_bignum_i.trn_elen;
1062 assign alu_bignum_operation_o.adder_carry_sel =
insn_dec_bignum_i.alu_adder_carry_sel;
1063 assign alu_bignum_operation_o.shift_right =
insn_dec_bignum_i.alu_shift_right;
1064 assign alu_bignum_operation_o.shift_amt =
insn_dec_bignum_i.alu_shift_amt;
1065 assign alu_bignum_operation_o.shift_mask =

```

```

insn_dec_bignum_i.alu_shift_mask;
1066 assign alu_bignum_operation_o.flag_group =
insn_dec_bignum_i.alu_flag_group;
1067 assign alu_bignum_operation_o.sel_flag =
insn_dec_bignum_i.alu_sel_flag;
1068 assign alu_bignum_operation_o.alu_flag_en =
insn_dec_bignum_i.alu_flag_en & insn_valid_i;
1069 assign alu_bignum_operation_o.mac_flag_en =
insn_dec_bignum_i.mac_flag_en & insn_valid_i;
1070
1071 assign alu_bignum_operation_valid_o = insn_valid_i;
1072 assign alu_bignum_operation_commit_o = insn_executing;
1073
1074 assign mac_bignum_operation_o.operand_a =
rf_bignum_rd_data_a_no_intg;
1075 assign mac_bignum_operation_o.operand_b =
rf_bignum_rd_data_b_no_intg;
1076 assign mac_bignum_operation_o.op_a_qw_sel_raw =
insn_dec_bignum_i.mac_op_a_qw_sel_raw;
1077 assign mac_bignum_operation_o.op_b_elem0_sel_raw =
insn_dec_bignum_i.mac_op_b_elem0_sel_raw;
1078 assign mac_bignum_operation_o.op_b_elem1_sel_raw =
insn_dec_bignum_i.mac_op_b_elem1_sel_raw;
1079 assign mac_bignum_operation_o.wr_hw_sel_upper =
insn_dec_bignum_i.mac_wr_hw_sel_upper;
1080 assign mac_bignum_operation_o.pre_acc_shift_imm =
insn_dec_bignum_i.mac_pre_acc_shift;
1081 assign mac_bignum_operation_o.acc_add_en =
insn_dec_bignum_i.mac_acc_add_en;
1082 assign mac_bignum_operation_o.shift_acc =
insn_dec_bignum_i.mac_shift_out;
1083 assign mac_bignum_operation_o.is_vec =
insn_dec_bignum_i.mac_is_vec;
1084 assign mac_bignum_operation_o.is_mod =
insn_dec_bignum_i.mac_is_mod;
1085 assign mac_bignum_operation_o.is_lane =
insn_dec_bignum_i.mac_is_lane;
1086 assign mac_bignum_operation_o.lane_index =
insn_dec_bignum_i.mac_lane_index;
1087 assign mac_bignum_operation_o.elen =
insn_dec_bignum_i.mac_elen;
1088 assign mac_bignum_operation_o.adder_carry_sel =
insn_dec_bignum_i.mac_adder_carry_sel;
1089
1090 assign mac_bignum_en_o = insn_valid_i &
insn_dec_bignum_i.mac_en;
1091 assign mac_bignum_commit_o = insn_executing;
1092
1093 // Move / Conditional Select. Only select B register data
when a selection instruction is being
1094 // executed and the selection flag isn't set. To avoid un-
desirable SCA leakage between the two
1095 // registers for non-selection instructions, the B register
is blanked except for selection
1096 // instructions.
1097 // Note that blanking both registers is not feasible nor ab-
solutely required because:
1098 // - The flag group selection and flag selection are known
in the predecoder stage but the actual
1099 // flag isn't.
1100 // - Selecting the flag in the predecoder stage using com-
binatorial inputs may lead to SCA leakage
1101 // between the still combinatorial flag groups and flags
within a group which might be
1102 // undesirable as well.
1103
1104 // - When executing a selection instruction, programmers can
expected that there will be some SCA
1105 // leakage between the two options. But it may be much
less expected for such leakage to occur
1106 // for other instructions.
1107 `ASSERT(SelFlagValid, insn_valid_i &
insn_dec_bignum_i.sel_insn |>
1108 insn_dec_bignum_i.alu_sel_flag inside {FlagC, FlagL,
FlagM, FlagZ})
1109
1110 // SEC_CM: DATA_REG_SW.SCA
1111 prim_blanker #(Width(ExtWLEN))
u_rf_bignum_rd_data_b_intg_blanker (
1112   .in_i (rf_bignum_rd_data_b_intg_i),
1113   .en_i (ctrl_flow_predec_i.sel_insn),
1114   .out_o(rf_bignum_rd_data_b_intg_blanked)
1115 );
1116
1117 `ASSERT(BlankingBignumRdDataBSel,
~(insn_valid_i & insn_dec_bignum_i.sel_insn) |>
rf_bignum_rd_data_b_intg_blanked == '0,
1118 clk_i, !rst_ni || ctrl_flow_predec_error || !insn_executing)
1119
1120 assign selection_result =
1121 ~ctrl_flow_predec_i.sel_insn |
alu_bignum_selection_flag_i ? rf_bignum_rd_data_a_intg_i :
1122
rf_bignum_rd_data_b_intg_blanked;
1123
1124 // Bignum Register file write control
1125
1126 always_comb begin
1127   // By default write nothing
1128   rf_bignum_wr_en_unbuf = 2'b00;

```

```

1129
1130 // Only write if valid instruction wants a bignum rf write
and it isn't stalled. If instruction
1131 // doesn't execute (e.g. due to an error detected in the
previous cycle) the write won't
1132 // commit. The write enable is also high during the execu-
tion cycles of multi-cycle
1133 // multiplications. However, the destination register may
not be updated until the instruction
1134 // finishes to avoid overwriting a source register (if
source = destination). As the OTBN
1135 // stalls during these cycles, the write commit signal is
not set and the update will only be
1136 // committed in the last cycle where the OTBN un-stalls.
1137 if (insn_valid_i && insn_dec_bignum_i.rf_we && !rf_indir-
ect_stall) begin
1138   if (insn_dec_bignum_i.mac_en &&
insn_dec_bignum_i.mac_shift_out &&
1139       !insn_dec_bignum_i.mac_is_vec) begin
1140     // Special handling for BN.MULQACC.S0, only enable up-
per or lower half depending on
1141     // mac_wr_hw_sel_upper.
1142     rf_bignum_wr_en_unbuf =
insn_dec_bignum_i.mac_wr_hw_sel_upper ? 2'b10 : 2'b01;
1143   end else begin
1144     // For everything else write both halves immediately.
1145     rf_bignum_wr_en_unbuf = 2'b11;
1146   end
1147 end
1148 end
1149
1150 // Bignum RF control signals from the controller aren't ac-
tually used, instead the predecoded
1151 // one-hot versions are. The predecoded versions get checked
against the signals produced here.
1152 // Buffer them to ensure they don't get optimised away (with
a functionally correct OTBN they will
1153 // always be identical).
1154 prim_buf #(
1155   .width(2)
1156 ) u_bignum_wr_en_buf (
1157   .in_i (rf_bignum_wr_en_unbuf),
1158   .out_o (rf_bignum_wr_en_o)
1159 );
1160
1161 assign rf_bignum_wr_commit_o = |rf_bignum_wr_en_o & insn_ex-
ecuting & !stall;
1162
1163 assign rf_bignum_indirect_en_o = insn_executing & rf_in-
direct_stall;
1164 assign rf_bignum_rd_a_indirect_en = insn_executing &
insn_dec_bignum_i.rf_a_indirect;
1165 assign rf_bignum_rd_b_indirect_en = insn_executing &
insn_dec_bignum_i.rf_b_indirect;
1166 assign rf_bignum_wr_indirect_en = insn_executing &
insn_dec_bignum_i.rf_d_indirect;
1167
1168 prim_onehot_enc #(
1169   .OneHotWidth(NWdr)
1170 ) rf_bignum_rd_a_indirect_onehot_enc (
1171   .in_i (rf_base_rd_data_a_no_intg[4:0]),
1172   .en_i (rf_bignum_rd_a_indirect_en),
1173   .out_o (rf_bignum_rd_a_indirect_onehot_o)
1174 );
1175
1176 prim_onehot_enc #(
1177   .OneHotWidth(NWdr)
1178 ) rf_bignum_rd_b_indirect_onehot_enc (
1179   .in_i (rf_base_rd_data_b_no_intg[4:0]),
1180   .en_i (rf_bignum_rd_b_indirect_en),
1181   .out_o (rf_bignum_rd_b_indirect_onehot_o)
1182 );
1183
1184 prim_onehot_enc #(
1185   .OneHotWidth(NWdr)
1186 ) rf_bignum_wr_indirect_onehot_enc (
1187   .in_i (rf_base_rd_data_b_no_intg[4:0]),
1188   .en_i (rf_bignum_wr_indirect_en),
1189   .out_o (rf_bignum_wr_indirect_onehot_o)
1190 );
1191
1192
1193 // For BN.LID sample the indirect destination register index
in first cycle as an increment might
1194 // change it for the second cycle where the load data is
written to the bignum register file.
1195 always_ff @(posedge clk_i) begin
1196   if (insn_dec_bignum_i.rf_d_indirect) begin
1197     insn_bignum_wr_addr_q <= rf_base_rd_data_b_no_intg[4:0];
1198   end
1199
1200   if (insn_dec_bignum_i.rf_a_indirect) begin
1201     insn_bignum_rd_addr_a_q <=
rf_base_rd_data_a_no_intg[4:0];
1202   end
1203
1204   if (insn_dec_bignum_i.rf_b_indirect) begin

```

```

1205     insn_bignum_rd_addr_b_q <=
rf_base_rd_data_b_no_intg[4:0];
1206   end
1207 end
1208
1209 // Bignum RF control signals from the controller aren't ac-
tually used, instead the predecoded
1210 // one-hot versions are. The predecoded versions get checked
against the signals produced here.
1211 // Buffer them to ensure they don't get optimised away (with
a functionally correct OTBN they will
1212 // always be identical).
1213 assign rf_bignum_wr_addr_unbuf =
insn_dec_bignum_i.rf_d_indirect ? insn_bignum_wr_addr_q :
1214
1215   insn_dec_bignum_i.d;
1216   prim_buf #(
1217     .width(WdrAw)
1218   ) u_rf_bignum_wr_addr_buf (
1219     .in_i (rf_bignum_wr_addr_unbuf),
1220     .out_o (rf_bignum_wr_addr_o)
1221   );
1222
1223 // For the shift-out variant of BN.MULQACC the bottom half
of the MAC result is written to one
1224 // half of a desatination register specified by the instruc-
tion (mac_wr_hw_sel_upper). The bottom
1225 // half of the MAC result must be placed in the appropriate
half of the write data (the RF only
1226 // accepts write data for the top half in the top half of
the write data input). Otherwise
1227 // (shift-out to bottom half and all other BN.MULQACC in-
structions) simply pass the MAC result
1228 // through unchanged as write data.
1229 assign mac_bignum_rf_wr_data[WLEN-1:WLEN/2] =
1230   (insn_dec_bignum_i.mac_wr_hw_sel_upper &&
1231     insn_dec_bignum_i.mac_shift_out &&
1232     !insn_dec_bignum_i.mac_is_vec) ?
mac_bignum_operation_result_i[WLEN/2-1:0] :
1233   mac_bignum_operation_result_i[WLEN-1:WLEN/2];
1234
1235 assign mac_bignum_rf_wr_data[WLEN/2-1:0] =
mac_bignum_operation_result_i[WLEN/2-1:0];
1236
1237 // Bignum register file write MUX. Depending on the data
source, integrity bits do or don't have
1238 // to be appended; see comments on the "Base register file
write MUX" for details.
1239 always_comb begin
1240   // Default values
1241   rf_bignum_wr_data_intg_sel_o = 1'b0;
1242   rf_bignum_wr_data_intg_o = '0;
1243   rf_bignum_wr_data_no_intg_o =
alu_bignum_operation_result_i;
1244
1245   unique case (insn_dec_bignum_i.rf_wdata_sel)
1246     RfwdSelEx: begin
1247       rf_bignum_wr_data_no_intg_o =
alu_bignum_operation_result_i;
1248     end
1249     RfwdSelMac: begin
1250       rf_bignum_wr_data_no_intg_o = mac_bignum_rf_wr_data;
1251     end
1252     RfwdSelIspr: begin
1253       rf_bignum_wr_data_intg_sel_o = 1'b1;
1254       rf_bignum_wr_data_intg_o = ispr_rdata_intg_i;
1255     end
1256     RfwdSelMovSel: begin
1257       rf_bignum_wr_data_intg_sel_o = 1'b1;
1258       rf_bignum_wr_data_intg_o = selection_result;
1259     end
1260     RfwdSelLsu: begin
1261       rf_bignum_wr_data_intg_sel_o = 1'b1;
1262       //SEC_CM: BUS INTEGRITY
1263       rf_bignum_wr_data_intg_o = lsu_bignum_rdata_i;
1264     end
1265     default: ;
1266   endcase
1267 end
1268
1269 assign rf_a_indirect_err =
insn_dec_bignum_i.rf_a_indirect &
1270   (|
rf_base_rd_data_a_no_intg[31:5]) &
1271   ~rf_base_call_stack_sw_err_i &
1272   rf_base_rd_en_a_o;
1273
1274 assign rf_b_indirect_err =
insn_dec_bignum_i.rf_b_indirect &
1275   (|
rf_base_rd_data_b_no_intg[31:5]) &
1276   ~rf_base_call_stack_sw_err_i &
1277   rf_base_rd_en_b_o;
1278

```

```

1279 assign rf_d_indirect_err =
insn_dec_bignum_i.rf_d_indirect &
1280 rf_base_rd_data_b_no_intg[31:5]) &
1281 ~rf_base_call_stack_sw_err_i &
1282 rf_base_rd_en_b_o;
1283
1284 assign rf_indirect_err =
1285 insn_valid_i & (rf_a_indirect_err | rf_b_indirect_err |
rf_d_indirect_err);
1286
1287 // If the source registers are indirectly indexed and there
is a stack error, the source
1288 // register indices were illegal due to a stack pop error.
In this case, ignore bignum RF read
1289 // integrity errors.
1290 assign ignore_rf_bignum_intg_errs =
1291 (insn_dec_bignum_i.rf_a_indirect |
1292 insn_dec_bignum_i.rf_b_indirect) &
1293 rf_base_call_stack_sw_err_i;
1294
1295 assign rf_bignum_intg_err = rf_bignum_intg_err_i & ~ig-
nore_rf_bignum_intg_errs;
1296
1297 // If the destination register is indirectly indexed and
there is a stack error, the destination
1298 // register index was illegal due to a stack pop error. In
this case, ignore bignum RF
1299 // write-enable errors.
1300 assign ignore_rf_bignum_spurious_we_errs =
insn_dec_bignum_i.rf_d_indirect &
1301 rf_base_call_stack_sw_err_i;
1302
1303 assign rf_bignum_spurious_we_err =
rf_bignum_spurious_we_err_i &
1304 ~ig-
nore_rf_bignum_spurious_we_errs;
1305
1306 // CSR/WSR/ISPR handling
1307 // ISPRs (Internal Special Purpose Registers) are the in-
ternal registers. CSRs and WSRs are the
1308 // ISA visible versions of those registers in the base and
bignum ISAs respectively.
1309
1310 assign csr_addr = csr_e'(insn_dec_base_i.i[11:0]);
1311 assign csr_sub_addr =
insn_dec_base_i.i[$clog2(BaseWordsPerWLEN)-1:0];
1312
1313 always_comb begin
1314 ispr_addr_base = IsprMod;
1315 ispr_word_addr_base = '0;
1316 csr_illegal_addr = '1'b0;
1317
1318 unique case (csr_addr)
1319 CsrFlags, CsrFg0, CsrFg1: begin
1320 ispr_addr_base = IsprFlags;
1321 ispr_word_addr_base = '0;
1322 end
1323 CsrMod0, CsrMod1, CsrMod2, CsrMod3, CsrMod4, CsrMod5,
CsrMod6, CsrMod7: begin
1324 ispr_addr_base = IsprMod;
1325 ispr_word_addr_base = csr_sub_addr;
1326 end
1327 CsrRndPrefetch: begin
1328 // Reading from RND_PREFETCH results in 0, there is no
ISPR to read so no address is set.
1329 // The csr_rdata mux logic takes care of producing the
0.
1330 end
1331 CsrRnd: begin
1332 ispr_addr_base = IsprRnd;
1333 ispr_word_addr_base = '0;
1334 end
1335 CsrUrnd: begin
1336 ispr_addr_base = IsprUrnd;
1337 ispr_word_addr_base = '0;
1338 end
1339 CsrUrndCtrl: begin
1340 ispr_addr_base = IsprUrndCtrl;
1341 ispr_word_addr_base = '0;
1342 end
1343 CsrUrndStatus: begin
1344 ispr_addr_base = IsprUrndStatus;
1345 ispr_word_addr_base = '0;
1346 end
1347 CsrKmacStatus: begin
1348 ispr_addr_base = IsprKmacStatus;
1349 ispr_word_addr_base = '0;
1350 end
1351 CsrKmacCtrl: begin
1352 ispr_addr_base = IsprKmacCtrl;
1353 ispr_word_addr_base = '0;
1354 end
1355 CsrKmacCfg: begin

```

```

1356 ispr_addr_base = IsprKmacCfg;
1357 ispr_word_addr_base = '0;
1358 end
1359 CsrKmacStrb: begin
1360 ispr_addr_base = IsprKmacStrb;
1361 ispr_word_addr_base = '0;
1362 end
1363 CsrMaiCtrl: begin
1364 ispr_addr_base = IsprMaiCtrl;
1365 ispr_word_addr_base = '0;
1366 end
1367 CsrMaiStatus: begin
1368 ispr_addr_base = IsprMaiStatus;
1369 ispr_word_addr_base = '0;
1370 end
1371 CsrInsCnt: begin
1372 ispr_addr_base = IsprInsCnt;
1373 ispr_word_addr_base = '0;
1374 end
1375 default: csr_illegal_addr = '1'b1;
1376 endcase
1377 end
1378
1379 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_ispr_word_sel_base
1380 assign ispr_word_sel_base[i_word] = ispr_word_addr_base ==
i_word;
1381 end
1382
1383 // Decode wide ISPR read data.
1384 logic [WLEN-1:0] ispr_rdata;
1385 logic [2*BaseWordsPerWLEN-1:0] ispr_rdata_intg_err_wide;
1386 logic [BaseWordsPerWLEN-1:0] ispr_rdata_intg_err_narrow;
1387
1388 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_ispr_rdata_dec
1389 prim_secdec_inv_39_32_dec i_secdec_dec (
1390 .data_i (ispr_rdata_intg_i[i_word*39+:39]),
1391 .data_o (* unused because we abort on any integrity
error */),
1392 .syndrome_o (* unused */),
1393 .err_o (ispr_rdata_intg_err_wide[i_word*2+:2])
1394 );
1395 assign ispr_rdata[i_word*32+:32] =
ispr_rdata_intg_i[i_word*39+:32];
1396 assign ispr_rdata_intg_err_narrow[i_word] = |
(ispr_rdata_intg_err_wide[i_word*2+:2]);
1397 end
1398 // Propagate integrity error only if wide ISPR is used.
1399
1400 // Handle ISPR integrity error detection. We've got a bit-
mask of ISPR words that failed their
1401 // integrity check (ispr_rdata_intg_err_narrow), but a
nonzero entry may not be a problem if we
1402 // don't actually use the data.
1403 //
1404 // The situations when the data is actually used are:
1405 //
1406 // (1) This is a bignum instruction that writes back to
the bignum register file by reading an
1407 // ISPR. In this case, we actually pass the data
through with integrity bits, but it
1408 // shouldn't hurt to add fault detection at this
point.
1409 //
1410 // (2) This instruction consumes the data by selecting a
word from an ISPR and then writing it
1411 // back. This happens for things like CSRRS instruc-
tions, where the data flows to the base
1412 // register file through rf_base_wr_data_no_intg_o and
back to the ISPR through
1413 // ispr_base_wdata_o. The word used is given by the
onehot ispr_word_sel_base mask.
1414 //
1415 // In both cases, there's a special case for the
RND_PREFETCH register, which doesn't actually
1416 // have any backing data. It reads as zero with invalid in-
tegrity bits which we want to ignore.
1417 //
1418 // Are we reading all the ISPR data? (case (1) above)
1419 logic all_ispr_words_used;
1420 assign all_ispr_words_used = (insn_dec_bignum_i.rf_wdata_sel
== RfwdSelIspr);
1421
1422 // Are we reading just one word of the ISPR data? (case (2)
above).
1423 logic one_ispr_word_used;
1424 assign one_ispr_word_used = ispr_rd_insn &
(insn_dec_shared_i.subset == InsnSubsetBase);
1425
1426 // A bit-mask giving which ISPR words are being read
1427 logic [BaseWordsPerWLEN-1:0] ispr_read_mask;
1428 assign ispr_read_mask = all_ispr_words_used ? '1 :
one_ispr_word_used ?
1429 ispr_word_sel_base : '0;

```



```

1430
1431 // Use ispr_read_mask to qualify the error bit-mask that
came out of the integrity decoder.
1432 logic [BaseWordsPerWLEN-1:0] ispr_rdata_used_intg_err;
1433 assign ispr_rdata_used_intg_err = ispr_read_mask &
ispr_rdata_intg_err_narrow;
1434
1435 // We only architecturally read the ISPR when there's a non-
stalled instruction. This is also the
1436 // place where we factor in the special RND_PREFETCH beha-
viour. We also need to squash any
1437 // integrity errors if we're reading a sideload key which
isn't currently valid (this will
1438 // generate a key_invalid error, but we shouldn't have any
behaviour that depends on what happens
1439 // to be on the pins)
1440 logic non_prefetch_insn_running;
1441 assign non_prefetch_insn_running = (insn_valid_i & ~stall &
1442                                     (csr_addr != CsrRnd-
Prefetch) & ~key_invalid);
1443
1444 assign ispr_rdata_intg_err = non_prefetch_insn_running & |
(ispr_rdata_used_intg_err);
1445
1446 `ASSERT_KNOWN(IsprRdataIntgErrKnown_A, ispr_rdata_intg_err)
1447
1448 for (genvar i_bit = 0; i_bit < 32; i_bit++) begin :
g_csr_rdata_mux
1449   for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word+
+) begin : g_csr_rdata_mux_inner
1450     assign csr_rdata_mux[i_bit][i_word] =
1451       ispr_rdata[i_word*32 + i_bit] &
ispr_word_sel_base[i_word];
1452   end
1453
1454   assign csr_rdata_raw[i_bit] = |csr_rdata_mux[i_bit];
1455 end
1456
1457 // Specialised read data handling for CSR reads where raw
read data needs modification.
1458 always_comb begin
1459   csr_rdata = csr_rdata_raw;
1460
1461   unique case (csr_addr)
1462     // For FG0/FG1 select out appropriate bits from FLAGS
ISPR and pad the rest with zeros.
1463     CsrFg0:   csr_rdata = {28'b0, csr_rdata_raw[3:0]};
1464     CsrFg1:   csr_rdata = {28'b0, csr_rdata_raw[7:4]};
1465     CsrRndPrefetch: csr_rdata = '0;
1466     default: ;
1467   endcase
1468 end
1469
1470 assign csr_wdata_raw = insn_dec_shared_i.ispr_rs_insn ?
csr_rdata | rf_base_rd_data_a_no_intg :
1471 rf_base_rd_data_a_no_intg;
1472
1473 // Specialised write data handling for CSR writes where raw
write data needs modification.
1474 always_comb begin
1475   csr_wdata = csr_wdata_raw;
1476
1477   unique case (csr_addr)
1478     // For FG0/FG1 only modify relevant part of FLAGS ISPR.
1479     CsrFg0:   csr_wdata = {24'b0, csr_rdata_raw[7:4],
csr_wdata_raw[3:0]};
1480     CsrFg1:   csr_wdata = {24'b0, csr_wdata_raw[3:0],
csr_rdata_raw[3:0]};
1481     default: ;
1482   endcase
1483 end
1484
1485 // ISPR RS (read and set) must not be combined with ISPR RD
or WR (read or write). ISPR RD and
1486 // WR (read and write) is allowed.
1487 `ASSERT(NoIsprRorWAndRs, insn_valid_i |->
~(insn_dec_shared_i.ispr_rs_insn &
1488   (insn_dec_shared_i.ispr_rd_insn |
1489     insn_dec_shared_i.ispr_wr_insn)))
1490
1491
1492 assign wsr_addr =
wsr_e'(insn_dec_bignum_i.i[WsNumWidth-1:0]);
1493
1494 always_comb begin
1495   ispr_addr_bignum = IsprMod;
1496   wsr_illegal_addr = 1'b0;
1497   key_invalid = 1'b0;
1498
1499   unique case (wsr_addr)
1500     WsrMod:   ispr_addr_bignum = IsprMod;
1501     WsrRnd:   ispr_addr_bignum = IsprRnd;
1502     WsrUrnd:  ispr_addr_bignum = IsprUrnd;
1503     WsrAcc:   ispr_addr_bignum = IsprAcc;
1504     WsrKeyS0L: begin
1505       ispr_addr_bignum = IsprKeyS0L;
1506       key_invalid = ~sideload_key_shares_valid_i[0];
1507     end
1508     WsrKeyS0H: begin
1509       ispr_addr_bignum = IsprKeyS0H;
1510       key_invalid = ~sideload_key_shares_valid_i[0];
1511     end
1512     WsrKeyS1L: begin
1513       ispr_addr_bignum = IsprKeyS1L;
1514       key_invalid = ~sideload_key_shares_valid_i[1];
1515     end
1516     WsrKeyS1H: begin
1517       ispr_addr_bignum = IsprKeyS1H;
1518       key_invalid = ~sideload_key_shares_valid_i[1];
1519     end
1520     WsrMaiResS0:   ispr_addr_bignum = IsprMaiResS0;
1521     WsrMaiResS1:   ispr_addr_bignum = IsprMaiResS1;
1522     WsrMaiIn0S0:   ispr_addr_bignum = IsprMaiIn0S0;
1523     WsrMaiIn0S1:   ispr_addr_bignum = IsprMaiIn0S1;
1524     WsrMaiIn1S0:   ispr_addr_bignum = IsprMaiIn1S0;
1525     WsrMaiIn1S1:   ispr_addr_bignum = IsprMaiIn1S1;
1526     WsrKmacDataS0: ispr_addr_bignum = IsprKmacDataS0;
1527     WsrKmacDataS1: ispr_addr_bignum = IsprKmacDataS1;
1528     WsrUrndState:  ispr_addr_bignum = IsprUrndState;
1529     default: wsr_illegal_addr = 1'b1;
1530   endcase
1531 end
1532
1533 assign wsr_wdata = insn_dec_shared_i.ispr_rs_insn ?
ispr_rdata | rf_bignum_rd_data_a_no_intg :
1534 rf_bignum_rd_data_a_no_intg;
1535
1536 assign ispr_illegal_addr = insn_dec_shared_i.subset == Insn-
SubsetBase ? csr_illegal_addr :
1537 wsr_illegal_addr;
1538
1539 assign ispr_err = ispr_illegal_addr & insn_valid_i &
(insn_dec_shared_i.ispr_rd_insn |
1540   insn_dec_shared_i.ispr_wr_insn |
1541   insn_dec_shared_i.ispr_rs_insn);
1542
1543 assign ispr_wr_insn = insn_dec_shared_i.ispr_wr_insn |
insn_dec_shared_i.ispr_rs_insn;
1544 assign ispr_rd_insn = insn_dec_shared_i.ispr_rd_insn |
insn_dec_shared_i.ispr_rs_insn;
1545
1546 assign ispr_flags_wr_o = insn_dec_shared_i.ispr_flags_wr;
1547
1548 // Write to RND_PREFETCH must not produce ISR write
1549 assign ispr_wr_base_insn =
1550   ispr_wr_insn & (insn_dec_shared_i.subset == InsnSubset-
Base) & (csr_addr != CsrRndPrefetch);
1551
1552 assign ispr_wr_bignum_insn = ispr_wr_insn &
(insn_dec_shared_i.subset == InsnSubsetBignum);
1553 assign ispr_rd_bignum_insn = ispr_rd_insn &
(insn_dec_shared_i.subset == InsnSubsetBignum);
1554
1555 assign ispr_addr_o = insn_dec_shared_i.subset == In-
snSubsetBase ? ispr_addr_base :
1556   ispr_addr_bignum;
1557 assign ispr_base_wdata_o = csr_wdata;
1558 assign ispr_base_wr_en_o = {BaseWordsPer-
WLEN(ispr_wr_base_insn & insn_valid_i)} &
1559   ispr_word_sel_base;
1560
1561 for (genvar i_word = 0; i_word < BaseWordsPerWLEN; i_word++)
begin : g_ispr_bignum_wdata_enc
1562   prim_secded_inv_39_32_enc_i.secded_enc (
1563     .data_i(wsr_wdata[i_word*32+:32]),
1564     .data_o(ispr_bignum_wdata_intg_o[i_word*39+:39])
1565   );
1566 end
1567 assign ispr_bignum_wr_en_o = ispr_wr_bignum_insn & insn_val-
id_i;
1568
1569 assign ispr_wr_commit_o = ispr_wr_insn & insn_executing;
1570 assign ispr_rd_en_o = ispr_rd_insn & insn_valid_i &
1571   ~(insn_dec_shared_i.subset == InsnSubsetBase) & (csr_addr
== CsrRndPrefetch));
1572
1573 // For BN.SID the LSU address is computed in the first cycle
by the base ALU. The store request
1574 // itself occurs in the second cycle when the store data is
available (from the indirect register
1575 // read). The calculated address is saved in a flop here so
it's available for use in the second
1576 // cycle.
1577 assign lsu_addr_saved_d =
alu_base_operation_result_i[DmemAddrwidth-1:0];
1578 always_ff @(posedge clk_i) begin
1579   lsu_addr_saved_q <= lsu_addr_saved_d;
1580 end
1581

```

```

1582 //assign expected_lsu_addr_en_predec = insn_valid &
insn_dec_shared_i.ld_insn
1583
1584 // lsu_load_req_raw/lsu_store_req_raw indicate an instruc-
tion wishes to perform a store or a load.
1585 // lsu_load_req_o/lsu_store_req_o factor in whether an in-
struction is actually executing (it may

1586 // be suppressed due an error) and command the load or store
to happen when asserted.
1587 assign lsu_load_req_raw = insn_valid_i &
insn_dec_shared_i.ld_insn & (state_q == 0tbnStateRun);
1588 assign lsu_load_req_o = insn_executing & lsu_load_req_raw;
1589
1590 assign lsu_store_req_raw = insn_valid_i &
insn_dec_shared_i.st_insn & ~rf_indirect_stall;
1591 assign lsu_store_req_o = insn_executing &
lsu_store_req_raw;
1592
1593 assign lsu_req_subset_o = insn_dec_shared_i.subset;
1594
1595 // To simplify blanking logic all two cycle memory opera-
tions (BN.LID, BN.SID, LW) present the
1596 // calculated address in their first cycle and the saved ad-
dress in the second cycle. This results
1597 // in lsu_addr_o remaining stable for the entire instruc-
tion. Only SW is a single cycle
1598 // instruction so it only presents the calculated address.
The stability property is checked by an
1599 // assertion.
1600 assign lsu_addr_saved_sel =
1601   insn_valid_i & ((insn_dec_shared_i.subset == InsnSubset-
Bignum) ||
1602   insn_dec_shared_i.ld_insn
1603   ? ~stall : 1'b0);
1604 assign lsu_addr = lsu_addr_saved_sel ? lsu_ad-
dr_saved_q
1605   :
1606   alu_base_operation_result_i[DmemAddrWidth-1:0];
1607 // SEC_CM: CTRL.REDUN
1608 assign expected_lsu_addr_en =
1609   insn_valid_i & (insn_dec_shared_i.ld_insn |
insn_dec_shared_i.st_insn);
1610
1611 assign lsu_predec_error = expected_lsu_addr_en != lsu_ad-
dr_en_predec_i;
1612
1613 assign expected_call_stack_push =
1614   insn_valid_i & insn_dec_base_i.rf_we & rf_base_wr_addr_o
== 5'd1;
1615
1616 assign expected_call_stack_pop = insn_valid_i &
1617   ((insn_dec_base_i.rf_ren_a
& rf_base_rd_addr_a_o == 5'd1) |
1618   (insn_dec_base_i.rf_ren_b
& rf_base_rd_addr_b_o == 5'd1));
1619
1620 // Check branch target against the precalculated target from
pre-decode. Pre-decode cannot
1621 // calculate the jump target of a JALR as it requires a re-
gister read so this is excluded from the
1622 // check (by looking at the ALU op a selection).
1623 assign branch_target_predec_error =
1624   insn_dec_shared_i.branch_insn
&
1625   insn_dec_shared_i.jump_insn & insn_dec_base_i.op_a_sel !=
OpASelRegister &
1626   (ctrl_flow_target_predec_i != branch_target);
1627
1628 assign ctrl_predec_error =
1629   |(ctrl_flow_predec_i.jump_insn
1630   !=
(insn_dec_shared_i.jump_insn & insn_valid_i),

```

```

1630   ctrl_flow_predec_i.loop_insn
!=
(insn_dec_shared_i.loop_insn & insn_valid_i),
1631   ctrl_flow_predec_i.branch_insn
!=
(insn_dec_shared_i.branch_insn & insn_valid_i),
1632   ctrl_flow_predec_i.sel_insn
!=
(insn_dec_bignum_i.sel_insn & insn_valid_i),
1633   ctrl_flow_predec_i.call_stack_push != expec-
ted_call_stack_push,
1634   ctrl_flow_predec_i.call_stack_pop != expec-
ted_call_stack_pop,
1635   branch_target_predec_error,
1636   loop_predec_err};
1637
1638 assign predec_error_o = lsu_predec_error | ctrl_pre-
dec_error;
1639
1640 // SEC_CM: DATA.REG.SW.SCA
1641 prim_blanker #(.Width(DmemAddrWidth)) u_lsu_addr_blanker (
1642   .in_i (lsu_addr),
1643   .en_i (lsu_addr_en_predec_i),
1644   .out_o (lsu_addr_blanked)
1645 );
1646
1647 // Check stability property described above (see the lsu_ad-
dr_saved_sel signal) holds.
1648 `ASSERT(LsuAddrBlankedStable_A, insn_valid_i & stall & ~err
|=> $stable(lsu_addr_blanked))
1649
1650 assign lsu_addr_o = lsu_addr_blanked;
1651
1652 assign lsu_base_wdata_o = rf_base_rd_data_b_intg_i;
1653 assign lsu_bignum_wdata_o = rf_bignum_rd_data_b_intg_i;
1654
1655 assign dmem_addr_unaligned_bignum =
1656   (lsu_req_subset_o == InsnSubsetBignum) & (lsu_ad-
dr_o[$clog2(WLEN/8)-1:0]);
1657 assign dmem_addr_unaligned_base =
1658   (lsu_req_subset_o == InsnSubsetBase) & (lsu_ad-
dr_o[1:0]);
1659 assign dmem_addr_overflow = |
alu_base_operation_result_i[31:DmemAddrWidth];
1660
1661 // A dmem address is checked the cycle it is available. For
bignum stores this is the first cycle
1662 // where the base register file read occurs, with the store
request occurring the following cycle.
1663 // For all other loads and stores the dmem address is avail-
able the same cycle as the request.
1664 assign dmem_addr_err_check =
1665   (lsu_req_subset_o == InsnSubsetBignum) &
1666   insn_dec_shared_i.st_insn
1667   ? rf_indir-
ect_stall :
1668   lsu_load_req_raw
| lsu_store_req_raw;
1669
1670 assign dmem_addr_err =
1671   insn_valid_i & dmem_addr_err_check & (dmem_ad-
dr_overflow
1672   |
dmem_ad-
dr_unaligned_bignum |
dmem_ad-
dr_unaligned_base);
1673
1674 assign rnd_req_raw = insn_valid_i & ispr_rd_insn & (ispr_ad-
dr_o == IsprRnd);
1675 // Don't factor rnd_rep/fips_err_i into rnd_req_o. This
would lead to a combo loop.
1676 assign rnd_req_o = rnd_req_raw & insn_valid_i & ~(soft-
ware_err | fatal_err);
1677
1678 assign rnd_prefetch_req_o = insn_executing & ispr_wr_insn &
1679   (insn_dec_shared_i.subset == InsnSubsetBase) & (csr_addr
== CsrRndPrefetch);
1680 endmodule

```


SPI 호스트

What. SPI 마스터 컨트롤러. 클럭 분주 · 모드(CPOL/CPHA) · FIFO · 세그먼트 전송.

Why it exists. 가장 흔한 주변장치 인터페이스이고, ‘간단해 보이지만 변종이 많은’ 블록의 전형 이다. 모드 4가지 · 비트 순서 · 반이중/전이중 · 듀얼/쿼드 IO · 칩선택 타이밍이 장치마다 다르다.

학부 복습 · **EECS concepts.** 직렬 통신 · 클럭 위상/극성 · 셋업/홀드 마진 · 클럭 분주기 설계.

석사 수준 · **Practice note.** SPI 버그의 대부분은 **CPOL/CPHA** 조합과 칩선택 타이밍에서 난다. 검증에서 네 모드를 모두 돌리지 않으면 현장에서 터진다 — 파라미터 조합 커버리지의 교과서적 사례.

12 files • 4,482 lines

API — spi_host.sv

Table 317. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — module / interface / package (1개)

이름	종류
spi_host	module

Table 318. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — parameter (7개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
NumCS	int unsigned	1
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelWinRXDATA	top_racl_pkg::racl_policy_sel_t	0
RaclPolicySelWinTXDATA	top_racl_pkg::racl_policy_sel_t	0

Table 319. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — localparam (1개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)

Table 320. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]

포트	방향	형/폭
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
cio_sck_o	output	logic
cio_sck_en_o	output	logic
cio_csb_o	output	logic [NumCS-1:0]
cio_csb_en_o	output	logic [NumCS-1:0]
cio_sd_o	output	logic [3:0]
cio_sd_en_o	output	logic [3:0]
cio_sd_i	input	[3:0]
passthrough_i	input	spi_device_pkg::passthrough_req_t
passthrough_o	output	spi_device_pkg::passthrough_rsp_t
lsio_trigger_o	output	logic
intr_error_o	output	logic
intr_spi_event_o	output	logic

API — spi_host_byte_merge.sv

Table 321. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_merge.sv — module / interface / package (1개)

이름	종류
spi_host_byte_merge	module

Table 322. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_merge.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
byte_i	input	[7:0]
byte_last_i	input	
byte_valid_i	input	
byte_ready_o	output	logic
word_o	output	logic [31:0]
word_valid_o	output	logic
word_ready_i	input	
sw_rst_i	input	

API — spi_host_byte_select.sv**Table 323. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_select.sv — module / interface / package (1개)**

이름	종류
spi_host_byte_select	module

Table 324. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_byte_select.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
word_i	input	[31:0]
word_be_i	input	[3:0]
word_valid_i	input	
word_ready_o	output	logic
byte_o	output	logic [7:0]
byte_valid_o	output	logic
byte_ready_i	input	
flush_i	input	
sw_rst_i	input	

API — spi_host_cmd_pkg.sv**Table 325. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_cmd_pkg.sv — module / interface / package (1개)**

이름	종류
spi_host_cmd_pkg	package

Table 326. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_cmd_pkg.sv — parameter (1개)

파라미터	형/폭	기본값
CmdSize	int	56

API — spi_host_command_queue.sv**Table 327. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — module / interface / package (1개)**

이름	종류
spi_host_command_queue	module

Table 328. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — parameter (2개)

파라미터	형/폭	기본값
CmdDepth	int	4
NumCS	int unsigned	1

Table 329. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — localparam (2개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)
CmdDepthW	int	prim_util_pkg::vbits(CmdDepth+1)

Table 330. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_command_queue.sv — 포트 (13개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
command_i	input	spi_host_cmd_pkg::command_t
command_csid_i	input	logic [CSW-1:0]
command_valid_i	input	
command_busy_o	output	logic
core_command_o	output	spi_host_cmd_pkg::command_t
core_command_csid_o	output	logic [CSW-1:0]
core_command_valid_o	output	logic
core_command_ready_i	input	
qd_o	output	logic [3:0]
error_busy_o	output	logic
sw_rst_i	input	

API — spi_host_core.sv**Table 331. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — module / interface / package (1개)**

이름	종류
spi_host_core	module

Table 332. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — parameter (1개)

파라미터	형/폭	기본값
NumCS	int	1

Table 333. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — localparam (1개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)

Table 334. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
command_i	input	spi_host_cmd_pkg::command_t

포트	방향	형/폭
command_csid_i	input	logic [CSW-1:0]
command_valid_i	input	
command_ready_o	output	
en_i	input	
tx_data_i	input	[31:0]
tx_be_i	input	[3:0]
tx_valid_i	input	
tx_ready_o	output	logic
tx_byte_select_full_o	output	logic
rx_data_o	output	logic [31:0]
rx_valid_o	output	logic
rx_ready_i	input	
sw_rst_i	input	
sck_o	output	logic
csb_o	output	logic [NumCS-1:0]
sd_o	output	logic [3:0]
sd_en_o	output	logic [3:0]
sd_i	input	[3:0]
rx_stall_o	output	logic
tx_stall_o	output	logic
active_o	output	logic

API — spi_host_data_fifos.sv

Table 335. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — module / interface / package (1개)

이름	종류
spi_host_data_fifos	module

Table 336. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — parameter (3개)

파라미터	형/폭	기본값
TxDepth	int	72
RxDepth	int	64
SwapBytes	logic	0

Table 337. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — localparam (2개)

상수	형/폭	값
RxDepthW	int	prim_util_pkg::vbits(RxDepth+1)

상수	형/폭	값
TxDpthW	int	prim_util_pkg::vbits(TxDpth+1)

Table 338. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tx_data_i	input	[31:0]
tx_be_i	input	[3:0]
tx_valid_i	input	
tx_ready_o	output	logic
tx_watermark_i	input	[7:0]
core_tx_data_o	output	logic [31:0]
core_tx_be_o	output	logic [3:0]
core_tx_valid_o	output	logic
core_tx_ready_i	input	
core_tx_byte_select_full_i	input	
core_rx_data_i	input	[31:0]
core_rx_valid_i	input	
core_rx_ready_o	output	logic
rx_data_o	output	logic [31:0]
rx_valid_o	output	logic
rx_ready_i	input	
rx_watermark_i	input	[7:0]
sw_rst_i	input	
tx_empty_o	output	logic
tx_full_o	output	logic
tx_qd_o	output	logic [7:0]
tx_wm_o	output	logic
rx_empty_o	output	logic
rx_full_o	output	logic
rx_qd_o	output	logic [7:0]
rx_wm_o	output	logic

API — spi_host_fsm.sv**Table 339. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — module / interface / package (17개)**

이름	종류
spi_host_fsm	module

Table 340. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — parameter (1개)

파라미터	형/폭	기본값
NumCS	int	1

Table 341. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — localparam (1개)

상수	형/폭	값
CSW	int	prim_util_pkg::vbits(NumCS)

Table 342. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
en_i	input	
command_i	input	command_t
command_csid_i	input	logic [CSW-1:0]
command_valid_i	input	
command_ready_o	output	logic
sck_o	output	logic
csb_o	output	logic [NumCS-1:0]
sd_en_o	output	logic [3:0]
last_read_o	output	logic
last_write_o	output	logic
wr_en_o	output	logic
sr_wr_ready_i	input	
rd_en_o	output	logic
sr_rd_ready_i	input	
sample_en_o	output	logic
shift_en_o	output	logic
speed_o	output	logic [1:0]
full_cyc_o	output	logic
rx_stall_o	output	logic
tx_stall_o	output	logic
active_o	output	logic
sw_rst_i	input	

API — spi_host_reg_pkg.sv**Table 343. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_pkg.sv — module / interface / package (1개)**

이름	종류
spi_host_reg_pkg	package

Table 344. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_pkg.sv — parameter (35개)

파라미터	형/폭	기본값
ByteOrder	logic	1
TxDpeth	int	72
RxDpeth	int	64
CmdDepth	int	4
NumAlerts	int	1
BlockAw	int	6
NumRegs	int	12
SPI_HOST_INTR_STATE_OFFSET	logic [BlockAw-1:0]	6'h 0
SPI_HOST_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 4
SPI_HOST_INTR_TEST_OFFSET	logic [BlockAw-1:0]	6'h 8
SPI_HOST_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	6'h c
SPI_HOST_CONTROL_OFFSET	logic [BlockAw-1:0]	6'h 10
SPI_HOST_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 14
SPI_HOST_CONFIGOPTS_OFFSET	logic [BlockAw-1:0]	6'h 18
SPI_HOST_CSID_OFFSET	logic [BlockAw-1:0]	6'h 1c
SPI_HOST_COMMAND_OFFSET	logic [BlockAw-1:0]	6'h 20
SPI_HOST_ERROR_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 2c
SPI_HOST_ERROR_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 30
SPI_HOST_EVENT_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 34
SPI_HOST_INTR_TEST_RESVAL	logic [1:0]	2'h 0
SPI_HOST_INTR_TEST_ERROR_RESVAL	logic [0:0]	1'h 0
SPI_HOST_INTR_TEST_SPI_EVENT_RESVAL	logic [0:0]	1'h 0
SPI_HOST_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
SPI_HOST_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
SPI_HOST_COMMAND_RESVAL	logic [24:0]	25'h 0
SPI_HOST_COMMAND_CSAAT_RESVAL	logic [0:0]	1'h 0
SPI_HOST_COMMAND_SPEED_RESVAL	logic [1:0]	2'h 0
SPI_HOST_COMMAND_DIRECTION_RESVAL	logic [1:0]	2'h 0
SPI_HOST_COMMAND_LEN_RESVAL	logic [19:0]	20'h 0

파라미터	형/폭	기본값
SPI_HOST_RXDATA_OFFSET	logic [BlockAw-1:0]	6'h 24
SPI_HOST_RXDATA_SIZE	int unsigned	'h 4
SPI_HOST_RXDATA_IDX	int unsigned	0
SPI_HOST_TXDATA_OFFSET	logic [BlockAw-1:0]	6'h 28
SPI_HOST_TXDATA_SIZE	int unsigned	'h 4
SPI_HOST_TXDATA_IDX	int unsigned	1

API — spi_host_reg_top.sv

Table 345. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — module / interface / package (1개)

이름	종류
spi_host_reg_top	module

Table 346. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacl	bit	1'b0
RaclErrorRsp	bit	1'b1

Table 347. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	6
DW	int	32
DBW	int	DW/8

Table 348. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	spi_host_reg_pkg::spi_host_reg2hw_t
hw2reg	input	spi_host_reg_pkg::spi_host_hw2reg_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

API — spi_host_shift_register.sv

Table 349. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_shift_register.sv — module / interface / package (1개)

이름	종류
spi_host_shift_register	module

Table 350. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_shift_register.sv — 포트 (23개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
wr_en_i	input	
wr_ready_o	output	logic
rd_en_i	input	
rd_ready_o	output	logic
speed_i	input	[1:0]
shift_en_i	input	
sample_en_i	input	
full_cyc_i	input	
last_read_i	input	
last_write_i	input	
tx_data_i	input	[7:0]
tx_valid_i	input	
tx_ready_o	output	
tx_flush_o	output	
rx_data_o	output	logic [7:0]
rx_valid_o	output	logic
rx_ready_i	input	
rx_last_o	output	logic
sd_i	input	[3:0]
sd_o	output	logic [3:0]
sw_rst_i	input	

API — spi_host_window.sv

Table 351. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — module / interface / package (1개)

이름	종류
spi_host_window	module

Table 352. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — parameter (4개)

파라미터	형/폭	기본값
EnableRac1	bit	1'b0

파라미터	형/폭	기본값
RaClErrorRsp	bit	1'b1
RaClPolicySelWinRXDATA	top_racl_pkg::racl_policy_sel_t	0
RaClPolicySelWinTXDATA	top_racl_pkg::racl_policy_sel_t	0

Table 353. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — localparam (4개)

상수	형/폭	값
AW	int	spi_host_reg_pkg::BlockAw
DW	int	32
ByteMaskW	int	DW / 8
RaClPolicySelRangesTXDATA	top_racl_pkg::racl_range_t [0:0]	{

Table 354. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_window.sv — 포트 (14개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rx_win_i	input	tlul_pkg::tl_h2d_t
rx_win_o	output	tlul_pkg::tl_d2h_t
tx_win_i	input	tlul_pkg::tl_h2d_t
tx_win_o	output	tlul_pkg::tl_d2h_t
tx_data_o	output	logic [31:0]
tx_be_o	output	logic [3:0]
tx_valid_o	output	logic
rx_data_i	input	[31:0]
rx_ready_o	output	logic
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_tx_o	output	top_racl_pkg::racl_error_log_t
racl_error_rx_o	output	top_racl_pkg::racl_error_log_t

1. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host.sv (661 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Serial Peripheral Interface (SPI) Host module.
6 //
7 //
8
9 `include "prim_assert.sv"
10
11 module spi_host
12   import spi_host_reg_pkg::*;
13   #(
14     parameter logic [NumAlerts-1:0]      AlertAsyn-
cOn      = {NumAlerts{1'b1}},
15     // Number of cycles a differential skew is tolerated on the
alert signal
16     parameter int unsigned              AlertSkew-
cycles      = 1,

```

```

17     parameter int unsigned              NumCS      = 1,
18     parameter bit                      EnableR-
acl      = 1'b0,
19     parameter bit                      RaclEr-
rorRsp      = EnableRacl,
20     parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
cySelVec[NumRegs] = '{NumRegs{0}},
21     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelWin-
RXDATA      = 0,
22     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySel-
WinTXDATA    = 0
23   ) (
24     input          clk_i,
25     input          rst_ni,
26
27     // Register interface
28     input          tlul_pkg::tl_h2d_t tl_i,
29     output         tlul_pkg::tl_d2h_t tl_o,
30
31     // Alerts
32     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
33     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]

```

```

alert_tx_o,
34
35 // RACL interface
36 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
37 output top_racl_pkg::racl_error_log_t racl_error_o,
38
39 // SPI Interface
40 output logic cio_sck_o,
41 output logic cio_sck_en_o,
42 output logic [NumCS-1:0] cio_csb_o,
43 output logic [NumCS-1:0] cio_csb_en_o,
44 output logic [3:0] cio_sd_o,
45 output logic [3:0] cio_sd_en_o,
46 input [3:0] cio_sd_i,
47
48 // Passthrough interface
49 input spi_device_pkg::passthrough_req_t passthrough_i,
50 output spi_device_pkg::passthrough_rsp_t passthrough_o,
51
52 output logic lsio_trigger_o,
53
54 output logic intr_error_o,
55 output logic intr_spi_event_o
56 );
57
58 import spi_host_cmd_pkg::*;
59
60 localparam int CSW = prim_util_pkg::vbits(NumCS);
61
62 spi_host_reg2hw_t reg2hw;
63 spi_host_hw2reg_t hw2reg;
64
65 top_racl_pkg::racl_error_log_t racl_error[3];
66 if (EnableRacl) begin : gen_racl_error_arb
67 // Arbitrate between all simultaneously valid error log
68 requests.
69 prim_racl_error_arb #(
70 .N ( 3 )
71 ) u_prim_err_arb (
72 .clk_i,
73 .rst_ni,
74 .error_log_i ( racl_error ),
75 .error_log_o ( racl_error_o )
76 );
77 end else begin : gen_no_racl_error_arb
78 logic unused_signals;
79 always_comb begin
80 unused_signals = ^{racl_error[0] ^ racl_error[1] ^
81 racl_error[2]};
82 racl_error_o = '0;
83 end
84 end
85
86 tlul_pkg::tl_h2d_t fifo_win_h2d [2];
87 tlul_pkg::tl_d2h_t fifo_win_d2h [2];
88
89 // Register module
90 logic [NumAlerts-1:0] alert_test, alerts;
91 spi_host_reg_top #(
92 .EnableRacl ( EnableRacl ),
93 .RaclErrorRsp ( RaclErrorRsp ),
94 .RaclPolicySelVec ( RaclPolicySelVec )
95 ) u_reg (
96 .clk_i,
97 .rst_ni,
98 .tl_i ( tl_i ),
99 .tl_o ( tl_o ),
100 .tl_win_o ( fifo_win_h2d ),
101 .tl_win_i ( fifo_win_d2h ),
102 .reg2hw,
103 .hw2reg,
104 // SEC_CM: BUS INTEGRITY
105 .intg_err_o ( alerts[0] ),
106 .racl_policies_i ( racl_policies_i ),
107 .racl_error_o ( racl_error[0] )
108 );
109 // Alerts
110 assign alert_test = {
111 reg2hw.alert_test.q &
112 reg2hw.alert_test.qe
113 };
114
115 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
116 prim_alert_sender #(
117 .AsyncOn(AlertAsyncOn[i]),
118 .SkewCycles(AlertSkewCycles),
119 .IsFatal(1'b1)
120 ) u_prim_alert_sender (
121 .clk_i,
122 .rst_ni,
123 .alert_test_i ( alert_test[i] ),
124 .alert_req_i ( alerts[0] ),
125 .alert_ack_o ( ),
126 .alert_state_o ( ),
127 .alert_rx_i ( alert_rx_i[i] ),
128 .alert_tx_o ( alert_tx_o[i] )
129 );
130 end
131 logic sck;
132
133 logic [NumCS-1:0] csb;
134 logic [3:0] sd_out;
135 logic [3:0] sd_en, sd_en_core;
136 logic [3:0] sd_i;
137 logic output_en;
138
139 assign output_en = reg2hw.control.output_en;
140
141 assign sd_en = output_en ? sd_en_core : 4'h0;
142
143 if (NumCS == 1) begin : gen_passthrough_implementation
144 logic passthrough_en;
145 assign passthrough_en = passthrough_i.passthrough_en;
146
147 logic pt_sck;
148 logic pt_sck_en;
149 logic [0:0] pt_csb;
150 logic [0:0] pt_csb_en;
151 logic [3:0] pt_sd_out;
152 logic [3:0] pt_sd_en;
153
154 assign pt_sck = passthrough_i.sck;
155 assign pt_sck_en = passthrough_i.sck_en;
156 assign pt_csb[0] = passthrough_i.csb;
157 assign pt_csb_en[0] = passthrough_i.csb_en;
158 assign pt_sd_out = passthrough_i.s;
159 assign pt_sd_en = passthrough_i.s_en;
160
161 assign cio_sck_o = passthrough_en ? pt_sck : sck;
162 assign cio_sck_en_o = passthrough_en ? pt_sck_en : out-
163 put_en;
164 assign cio_csb_o = passthrough_en ? pt_csb : csb;
165 assign cio_csb_en_o = passthrough_en ? pt_csb_en : out-
166 put_en;
167 assign cio_sd_o = passthrough_en ? pt_sd_out : sd_out;
168 assign cio_sd_en_o = passthrough_en ? pt_sd_en : sd_en;
169
170 end
171 else begin : gen_passthrough_ignore
172 // Passthrough only supported for instances with one CSb
173 line
174 `ASSERT(PassthroughNumCSCompat_A, !
175 passthrough_i.passthrough_en, clk_i, rst_ni)
176
177 assign cio_sck_o = sck;
178 assign cio_sck_en_o = output_en;
179 assign cio_csb_o = csb;
180 assign cio_csb_en_o = {NumCS{output_en}};
181 assign cio_sd_o = sd_out;
182 assign cio_sd_en_o = sd_en;
183
184 logic unused_pt_en;
185 logic unused_pt_sck;
186 logic unused_pt_sck_en;
187 logic unused_pt_csb;
188 logic unused_pt_csb_en;
189 logic [3:0] unused_pt_sd_out;
190 logic [3:0] unused_pt_sd_en;
191
192 assign unused_pt_en = passthrough_i.passthrough_en;
193 assign unused_pt_sck = passthrough_i.sck;
194 assign unused_pt_sck_en = passthrough_i.sck_en;
195 assign unused_pt_csb = passthrough_i.csb;
196 assign unused_pt_csb_en = passthrough_i.csb_en;
197 assign unused_pt_sd_out = passthrough_i.s;
198 assign unused_pt_sd_en = passthrough_i.s_en;
199
200 end : gen_passthrough_ignore
201
202 assign passthrough_o.s = cio_sd_i;
203 assign sd_i = cio_sd_i;
204
205 assign hw2reg.status.byteorder.d = ByteOrder;
206 assign hw2reg.status.byteorder.de = 1'b1;
207
208 logic command_valid;
209 logic core_command_valid;
210 logic command_busy;
211 logic core_command_ready;
212
213 command_t core_command, command;
214 logic [CSW-1:0] core_command_csid, command_csid;
215 logic error_csid_inval;
216 logic error_cmd_inval;
217 logic error_busy;
218 logic test_csid_inval;
219 logic test_dir_inval;
220 logic test_speed_inval;
221
222 assign test_csid_inval = (reg2hw.csid.q >= NumCS);
223
224 always_comb begin
225 test_speed_inval = 1'b1;
226 test_dir_inval = 1'b1;
227 unique case (reg2hw.command.speed.q)
228 Standard: begin
229 test_dir_inval = 1'b0;
230 test_speed_inval = 1'b0;
231 end
232 Dual, Quad: begin
233 test_dir_inval = (reg2hw.command.direction.q == Bid-

```

```

ir));
229     test_speed_inval = 1'b0;
230     end
231     default: begin
232     end
233 endcase
234 end
235
236 always_comb begin
237     command.segment.cmd_rd_en = 1'b0;
238     command.segment.cmd_wr_en = 1'b0;
239     unique case (reg2hw.command.direction.q)
240     RdOnly: begin
241         command.segment.cmd_rd_en = 1'b1;
242     end
243     WrOnly: begin
244         command.segment.cmd_wr_en = 1'b1;
245     end
246     Bidir: begin
247         command.segment.cmd_rd_en = 1'b1;
248         command.segment.cmd_wr_en = 1'b1;
249     end
250     default: begin
251     end
252 endcase
253 end
254
255 assign error_csid_inval = command_valid & ~command_busy &
256     test_csid_inval;
257 assign error_cmd_inval = command_valid & ~command_busy &
258     (test_speed_inval | test_dir_in-
val);
259
260 assign command_csid = (test_csid_inval) ? '0 :
reg2hw.csid.q[CSW-1:0];
261
262 assign command.configopts.clkdiv =
reg2hw.configopts.clkdiv.q;
263 assign command.configopts.csnidle =
reg2hw.configopts.csnidle.q;
264 assign command.configopts.csnlead =
reg2hw.configopts.csnlead.q;
265 assign command.configopts.csntail =
reg2hw.configopts.csntail.q;
266 assign command.configopts.full_cyc =
reg2hw.configopts.fullcyc.q;
267 assign command.configopts.cpha =
reg2hw.configopts.cpha.q;
268 assign command.configopts.cpol =
reg2hw.configopts.cpol.q;
269
270 assign command.segment.len = reg2hw.command.len.q;
271 assign command.segment.csaat = reg2hw.command.csaat.q;
272 assign command.segment.speed = reg2hw.command.speed.q;
273
274
275 logic [3:0] cmd_qes;
276
277 assign cmd_qes = {
278     reg2hw.command.len.qe,
279     reg2hw.command.speed.qe,
280     reg2hw.command.direction.qe,
281     reg2hw.command.csaat.qe
282 };
283
284 // Any qe pin from COMMAND will suffice.
285 assign command_valid = |cmd_qes;
286
287 logic active;
288 logic rx_stall;
289 logic tx_stall;
290
291 assign hw2reg.status.ready.d = ~command_busy;
292 assign hw2reg.status.active.d = active;
293 assign hw2reg.status.rxstall.d = rx_stall;
294 assign hw2reg.status.txstall.d = tx_stall;
295
296 assign hw2reg.status.ready.de = 1'b1;
297 assign hw2reg.status.active.de = 1'b1;
298 assign hw2reg.status.rxstall.de = 1'b1;
299 assign hw2reg.status.txstall.de = 1'b1;
300
301 logic sw_rst;
302
303 logic [3:0] cmd_qd;
304
305 spi_host_command_queue #(
306     .CmdDepth(CmdDepth),
307     .NumCS(NumCS)
308 ) u_cmd_queue (
309     .clk_i,
310     .rst_ni,
311     .command_i (command),
312     .command_csid_i (command_csid),
313     .command_valid_i (command_valid),
314     .command_busy_o (command_busy),
315     .core_command_o (core_command),
316     .core_command_csid_o (core_command_csid),
317     .core_command_valid_o (core_command_valid),
318     .core_command_ready_i (core_command_ready),
319     .error_busy_o (error_busy),

```

```

320     .qd_o (cmd_qd),
321     .sw_rst_i (sw_rst)
322 );
323
324 logic [31:0] tx_data;
325 logic [3:0] tx_be;
326 logic tx_valid;
327 logic tx_ready;
328
329 logic [31:0] rx_data;
330 logic rx_valid;
331 logic rx_ready;
332
333 spi_host_window #(
334     .EnableRacl ( EnableRacl ),
335     .RaclErrorRsp ( RaclErrorRsp ),
336     .RaclPolicySelWinRXDATA ( RaclPolicySelWinRXDATA ),
337     .RaclPolicySelWinTXDATA ( RaclPolicySelWinTXDATA )
338 ) u_window (
339     .clk_i,
340     .rst_ni,
341     .rx_win_i ( fifo_win_h2d[0] ),
342     .rx_win_o ( fifo_win_d2h[0] ),
343     .tx_win_i ( fifo_win_h2d[1] ),
344     .tx_win_o ( fifo_win_d2h[1] ),
345     .tx_data_o ( tx_data ),
346     .tx_be_o ( tx_be ),
347     .tx_valid_o ( tx_valid ),
348     .rx_data_i ( rx_data ),
349     .rx_ready_o ( rx_ready ),
350     .racl_policies_i ( racl_policies_i ),
351     .racl_error_tx_o ( racl_error[1] ),
352     .racl_error_rx_o ( racl_error[2] )
353 );
354
355 logic [31:0] core_tx_data;
356 logic [3:0] core_tx_be;
357 logic core_tx_valid;
358 logic core_tx_ready;
359 logic core_tx_byte_select_full;
360
361 logic [31:0] core_rx_data;
362 logic core_rx_valid;
363 logic core_rx_ready;
364
365 logic [7:0] rx_watermark;
366 logic [7:0] tx_watermark;
367 logic [7:0] rx_qd;
368 logic [7:0] tx_qd;
369
370 logic tx_empty, tx_full, tx_wm;
371 logic rx_empty, rx_full, rx_wm;
372
373 assign rx_watermark = reg2hw.control.rx_watermark.q;
374 assign tx_watermark = reg2hw.control.tx_watermark.q;
375
376 assign hw2reg.status.txqd.d = tx_qd;
377 assign hw2reg.status.rxqd.d = rx_qd;
378 assign hw2reg.status.cmdqd.d = cmd_qd;
379 assign hw2reg.status.txwm.d = tx_wm;
380 assign hw2reg.status.rxwm.d = rx_wm;
381 assign hw2reg.status.rxempty.d = rx_empty;
382 assign hw2reg.status.txempty.d = tx_empty;
383 assign hw2reg.status.rxfull.d = rx_full;
384 assign hw2reg.status.txfull.d = tx_full;
385
386 assign hw2reg.status.txqd.de = 1'b1;
387 assign hw2reg.status.rxqd.de = 1'b1;
388 assign hw2reg.status.cmdqd.de = 1'b1;
389 assign hw2reg.status.txwm.de = 1'b1;
390 assign hw2reg.status.rxwm.de = 1'b1;
391 assign hw2reg.status.rxempty.de = 1'b1;
392 assign hw2reg.status.txempty.de = 1'b1;
393 assign hw2reg.status.rxfull.de = 1'b1;
394 assign hw2reg.status.txfull.de = 1'b1;
395
396 logic error_overflow, error_underflow;
397 logic error_access_inval;
398
399 // Since the DATA FIFOs are essentially directly connected
to SW registers, it is an error if
400 // there is ever a need for flow control.
401 assign error_overflow = tx_valid & ~tx_ready;
402 assign error_underflow = rx_ready & ~rx_valid;
403 logic access_valid;
404 assign error_access_inval = tx_valid & ~access_valid;
405
406 always_comb begin
407     unique case (tx_be)
408     4'b1000,
409     4'b0100,
410     4'b0010,
411     4'b0001,
412     4'b1100,
413     4'b0110,
414     4'b0011,
415     4'b1111: begin
416         access_valid = 1'b1;
417     end
418     default: begin
419         access_valid = 1'b0;

```

```

420     end
421   endcase
422 end
423
424 logic tx_valid_checked;
425 assign tx_valid_checked = tx_valid & ~error_overflow & ~error_access_inval;
426
427 // Note on ByteOrder and ByteSwapping.
428 // ByteOrder == 1 is for Little-Endian transmission (i.e.
LSB first), which is achieved by
429 // default with the prim_packer_fifo implementation. Thus
we have to swap if Big-Endian
430 // transmission is required (i.e. if ByteOrder == 0).
431 spi_host_data_fifos #(
432   .TxDepth(TxDepth),
433   .RxDepth(RxDepth),
434   .SwapBytes(~ByteOrder)
435 ) u_data_fifos (
436   .clk_i,
437   .rst_ni,
438
439   .tx_data_i      (tx_data),
440   .tx_be_i        (tx_be),
441   .tx_valid_i     (tx_valid_checked),
442   .tx_ready_o     (tx_ready),
443   .tx_watermark_i (tx_watermark),
444
445   .core_tx_data_o (core_tx_data),
446   .core_tx_be_o   (core_tx_be),
447   .core_tx_valid_o (core_tx_valid),
448   .core_tx_ready_i (core_tx_ready),
449   .core_tx_byte_select_full_i (core_tx_byte_select_full),
450
451   .core_rx_data_i (core_rx_data),
452   .core_rx_valid_i (core_rx_valid),
453   .core_rx_ready_o (core_rx_ready),
454
455   .rx_data_o      (rx_data),
456   .rx_valid_o     (rx_valid),
457   .rx_ready_i     (rx_ready),
458   .rx_watermark_i (rx_watermark),
459
460   .tx_empty_o      (tx_empty),
461   .tx_full_o       (tx_full),
462   .tx_qd_o         (tx_qd),
463   .tx_wm_o         (tx_wm),
464   .rx_empty_o      (rx_empty),
465   .rx_full_o       (rx_full),
466   .rx_qd_o         (rx_qd),
467   .rx_wm_o         (rx_wm),
468
469   .sw_rst_i        (sw_rst)
470 );
471
472 logic en_sw;
473 logic enb_error;
474 logic en;
475
476 assign en      = en_sw & ~enb_error;
477 assign sw_rst = reg2hw.control.sw_rst.q;
478 assign en_sw  = reg2hw.control.spilen.q;
479
480 spi_host_core #(
481   .NumCS(NumCS)
482 ) u_spi_core (
483   .clk_i,
484   .rst_ni,
485
486   .command_i      (core_command),
487   .command_csid_i (core_command_csid),
488   .command_valid_i (core_command_valid),
489   .command_ready_o (core_command_ready),
490   .en_i           (en),
491   .tx_data_i      (core_tx_data),
492   .tx_be_i        (core_tx_be),
493   .tx_valid_i     (core_tx_valid),
494   .tx_ready_o     (core_tx_ready),
495   .tx_byte_select_full_o (core_tx_byte_select_full),
496   .rx_data_o      (core_rx_data),
497   .rx_valid_o     (core_rx_valid),
498   .rx_ready_i     (core_rx_ready),
499   .sck_o          (sck),
500   .csb_o          (csb),
501   .sd_o           (sd_out),
502   .sd_en_o        (sd_en_core),
503   .sd_i,
504   .rx_stall_o     (rx_stall),
505   .tx_stall_o     (tx_stall),
506   .sw_rst_i       (sw_rst),
507   .active_o       (active)
508 );
509
510 logic event_error;
511
512 logic [5:0] error_vec;
513 logic [5:0] error_mask;
514 logic [5:0] sw_error_status;
515
516 assign error_vec = {
517   error_access_inval,
518   error_csid_inval,
519   error_cmd_inval,
520   error_underflow,
521   error_overflow,
522   error_busy
523 };
524
525 // This mask dictates what classes of error events are to be
escalated to error interrupts.
526 // (Assume here that error interrupts will be enabled)
527 // Software generated errors can be configured to _not_ generate
error events.
528 // Bus errors (such as invalid write access) always generate
error events and therefore
529 // interrupts.
530 assign error_mask = {
531   1'b1, // invalid access: always an error event
532   reg2hw.error_enable.csidinval.q,
533   reg2hw.error_enable.cmdinval.q,
534   reg2hw.error_enable.underflow.q,
535   reg2hw.error_enable.overflow.q,
536   reg2hw.error_enable.cmbusy.q
537 };
538
539 assign hw2reg.error_status.accessinval.d = error_access_inval;
540 assign hw2reg.error_status.csidinval.d = error_csid_inval;
541 assign hw2reg.error_status.cmdinval.d = error_cmd_inval;
542 assign hw2reg.error_status.underflow.d = error_underflow;
543 assign hw2reg.error_status.overflow.d = error_overflow;
544 assign hw2reg.error_status.cmbusy.d = error_busy;
545
546 // Write the status register whenever the corresponding
event occurs.
547 // Only clear them from software.
548 assign hw2reg.error_status.accessinval.de = error_access_inval;
549 assign hw2reg.error_status.csidinval.de = error_csid_inval;
550 assign hw2reg.error_status.cmdinval.de = error_cmd_inval;
551 assign hw2reg.error_status.underflow.de = error_underflow;
552 assign hw2reg.error_status.overflow.de = error_overflow;
553 assign hw2reg.error_status.cmbusy.de = error_busy;
554
555 assign sw_error_status[5] =
reg2hw.error_status.accessinval.q;
556 assign sw_error_status[4] = reg2hw.error_status.csidinval.q;
557 assign sw_error_status[3] = reg2hw.error_status.cmdinval.q;
558 assign sw_error_status[2] = reg2hw.error_status.underflow.q;
559 assign sw_error_status[1] = reg2hw.error_status.overflow.q;
560 assign sw_error_status[0] = reg2hw.error_status.cmbusy.q;
561
562 assign event_error = |(error_vec & error_mask);
563 assign enb_error = |sw_error_status;
564
565 prim_intr_hw #(.Width(1)) intr_hw_error (
566   .clk_i,
567   .rst_ni,
568   .event_intr_i      (event_error),
569   .reg2hw_intr_enable_q_i (reg2hw.intr_enable.error.q),
570   .reg2hw_intr_test_q_i  (reg2hw.intr_test.error.q),
571   .reg2hw_intr_test_qe_i (reg2hw.intr_test.error.qe),
572   .reg2hw_intr_state_q_i (reg2hw.intr_state.error.q),
573   .hw2reg_intr_state_de_o (hw2reg.intr_state.error.de),
574   .hw2reg_intr_state_d_o (hw2reg.intr_state.error.d),
575   .intr_o              (intr_error_o)
576 );
577
578 logic status_spi_event;
579
580 logic status_idle, status_ready, status_tx_wm, status_rx_wm,
status_tx_empty, status_rx_full;
581 logic [5:0] event_vector;
582 logic [5:0] event_mask;
583
584 assign status_idle = ~active;
585 assign status_ready = ~command_busy;
586 assign status_tx_wm = tx_wm;
587 assign status_rx_wm = rx_wm;
588 assign status_tx_empty = tx_empty;
589 assign status_rx_full = rx_full;
590
591 assign event_vector = {
592   status_idle,
593   status_ready,
594   status_tx_wm,
595   status_rx_wm,
596   status_tx_empty,
597   status_rx_full
598 };
599
600 assign event_mask = {
601   reg2hw.event_enable.idle.q,
602   reg2hw.event_enable.ready.q,
603   reg2hw.event_enable.txwm.q,
604   reg2hw.event_enable.rxwm.q,
605   reg2hw.event_enable.txempty.q,
606   reg2hw.event_enable.rxfull.q
607 };

```



```

606   };
607
608   // Qualify interrupt sources individually with dedicated
mask register CSR.EVENT_ENABLE
609   assign status_spi_event = |(event_vector & event_mask);
610
611   // Flop trigger signal to avoid glitches on the output
612   always_ff @(posedge clk_i or negedge rst_ni) begin
613     if (!rst_ni) begin
614       lsio_trigger_o <= 1'b0;
615     end else begin
616       lsio_trigger_o <= tx_wm | rx_wm;
617     end
618   end
619
620   prim_intr_hw #(
621     .Width(1),
622     .IntrT("Status")
623   ) intr_hw_spi_event (
624     .clk_i,
625     .rst_ni,
626     .event_intr_i      (status_spi_event),
627     .reg2hw_intr_enable_q_i (reg2hw_intr_enable_spi_event.q),
628     .reg2hw_intr_test_q_i  (reg2hw_intr_test_spi_event.q),
629     .reg2hw_intr_test_qe_i (reg2hw_intr_test_spi_event.qe),
630     .reg2hw_intr_state_q_i (reg2hw_intr_state_spi_event.q),
631     .hw2reg_intr_state_de_o (hw2reg_intr_state_spi_event.de),
632     .hw2reg_intr_state_do  (hw2reg_intr_state_spi_event.d),
633     .intr_o                (intr_spi_event_o)
634   );
635
636   `ASSERT_KNOWN(TIDValidKnown0_A, tl_o.d_valid)
637   `ASSERT_KNOWN(TIARReadyKnown0_A, tl_o.a_ready)
638   `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
639   `ASSERT_KNOWN(CioSckEnKnown0_A, cio_sck_o)
640   `ASSERT_KNOWN(CioSckEnKnown0_A, cio_sck_en_o)
641   `ASSERT_KNOWN(CioCsbEnKnown0_A, cio_csb_o)
642   `ASSERT_KNOWN(CioCsbEnKnown0_A, cio_csb_en_o)
643   `ASSERT_KNOWN(CioSdEnKnown0_A, cio_sd_o, !
644   `ASSERT_KNOWN_IF(CioSdKnown0_A, cio_sd_o, !
passthrough_i.passthrough_en |
645   (passthrough_i.passthrough_en && passthrough_i.csb_en && !
passthrough_i.csb),
646   passthrough_i.sck_en & passthrough_i.sck)
647   `ASSERT_KNOWN(CioSdEnKnown0_A, cio_sd_en_o)
648   `ASSERT_KNOWN(IntrSpiEventKnown0_A, intr_spi_event_o)
649   `ASSERT_KNOWN(IntrErrorKnown0_A, intr_error_o)
650   `ASSERT_KNOWN(LsioTriggerKnown0_A, lsio_trigger_o)
651   `ASSERT_KNOWN(RaclErrorValidKnown0_A, racl_error_o.valid)
652
653   // passthrough_o.s is passed through to spi_device, it may
contain unknown data,
654   // but the unknown data won't be used based on the SPI pro-
655   tocol.
656   // Hence, instead of checking known data, here does a con-
657   nectivity check.
658   `ASSERT(PassthroughConn_A, passthrough_o.s == cio_sd_i)
659   // Alert assertions for reg_we onehot check
660   `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
661 endmodule : spi_host

```

2. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_byte_merge.sv (66 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Byte-merge module for collecting words in SPI Host IP
6 //
7
8 module spi_host_byte_merge (
9   input          clk_i,
10  input          rst_ni,
11  input [7:0]    byte_i,
12  input          byte_last_i,
13  input          byte_valid_i,
14  input          byte_ready_o,
15  output logic   word_o,
16  output logic   word_valid_o,
17  input          word_ready_i,
18  input          sw_rst_i
19 );
20
21 logic          clr;
22 logic          byte_valid;
23 logic          byte_ready;
24 logic          last_q;
25 logic          last_d;
26 logic          do_fill;
27 logic          byte_incoming;
28
29 // Don't clear data except on reset;
30 assign clr = sw_rst_i;

```

```

31 assign byte_incoming = byte_valid_i & byte_ready_o;
32
33 assign last_d        = sw_rst_i                ? 1'b0 :
34   byte_incoming && byte_last_i ? 1'b1 :
35   word_valid_o       ? 1'b0 :
36   last_q;
37
38 assign do_fill       = last_q & ~word_valid_o;
39 assign byte_valid    = do_fill | byte_valid_i;
40 assign byte_ready_o  = byte_ready & ~do_fill;
41
42 always_ff @(posedge clk_i or negedge rst_ni) begin
43   if (!rst_ni) begin
44     last_q <= 1'b0;
45   end else begin
46     last_q <= last_d;
47   end
48 end
49
50 prim_packer_fifo #(
51   .InW(8),
52   .OutW(32)
53 ) u_packer (
54   .clk_i,
55   .rst_ni,
56   .clr_i      (clr),
57   .wdata_i    (do_fill ? 8'h0 : byte_i),
58   .wvalid_i   (byte_valid),
59   .wready_o   (byte_ready),
60   .rdata_o    (word_o),
61   .rvalid_o   (word_valid_o),
62   .rready_i   (word_ready_i),
63   .depth_o   ()
64 );
65
66 endmodule : spi_host_byte_merge

```

3. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_byte_select.sv (53 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Byte-select module for dispensing words in SPI Host IP
6 //
7 module spi_host_byte_select (
8   input          clk_i,
9   input          rst_ni,
10  input [31:0]    word_i,
11  input [3:0]     word_be_i,
12  input          word_valid_i,
13  output logic    word_ready_o,
14  output logic [7:0] byte_o,
15  output logic    byte_valid_o,
16  input          byte_ready_i,
17  input          flush_i,
18  input          sw_rst_i
19 );
20
21 logic [35:0]    wdata_be;
22 logic           do_drain;
23 logic           byte_en;
24 logic           byte_valid;
25 logic           byte_ready;
26 logic           clr;
27
28 assign clr      = flush_i | sw_rst_i;
29 assign byte_valid_o = byte_valid & byte_en;
30 assign do_drain  = byte_valid & ~byte_en;
31 assign byte_ready  = byte_ready_i | do_drain;
32
33 for(genvar ii = 0; ii < 4; ii = ii + 1) begin :
gen_map_data_be
34   assign wdata_be[9*ii +: 9] = { word_be_i[ii], word_i[8*ii
+: 8] };
35 end : gen_map_data_be
36
37 prim_packer_fifo #(
38   .InW(36),
39   .OutW(9)
40 ) u_packer (
41   .clk_i,
42   .rst_ni,
43   .clr_i      (clr),
44   .wdata_i    (wdata_be),
45   .wvalid_i   (word_valid_i),
46   .wready_o   (word_ready_o),
47   .rdata_o    ({byte_en, byte_o}),
48   .rvalid_o   (byte_valid),
49   .rready_i   (byte_ready),
50   .depth_o   ()
51 );
52
53 endmodule : spi_host_byte_select

```

4. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_cmd_pkg.sv (51 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Command & Configuration Options structure for SPI HOST.
6 //
7
8 package spi_host_cmd_pkg;
9
10 parameter int CmdSize = 56;
11
12 // For decoding the direction register
13 typedef enum logic [1:0] {
14     Dummy = 2'b00,
15     RdOnly = 2'b01,
16     WrOnly = 2'b10,
17     Bidir = 2'b11
18 } reg_direction_t;
19
20 // For decoding the direction register
21 typedef enum logic [1:0] {
22     Standard = 2'b00,
23     Dual = 2'b01,
24     Quad = 2'b10,
25     RsvdSpd = 2'b11
26 } speed_t;
27
28 typedef struct packed {
29     logic [15:0] clkdiv;
30     logic [3:0] csnsidle;
31     logic [3:0] csnlead;
32     logic [3:0] csntrail;
33     logic full_cyc;
34     logic cpha;
35     logic cpol;
36 } configopts_t;
37
38 typedef struct packed {
39     logic [1:0] speed;
40     logic cmd_wr_en;
41     logic cmd_rd_en;
42     logic [19:0] len;
43     logic csaat;
44 } segment_t;
45
46 typedef struct packed {
47     segment_t segment;
48     configopts_t configopts;
49 } command_t;
50
51 endpackage

```

5. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_command_queue.sv (63 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Queue for SPI_HOST commands
6 //
7
8 module spi_host_command_queue #(
9     parameter int CmdDepth = 4,
10    parameter int unsigned NumCS = 1,
11    localparam int CSW =
    prim_util_pkg::vbits(NumCS)
12 ) (
13     input clk_i,
14     input rst_ni,
15
16     input spi_host_cmd_pkg::command_t command_i,
17     input logic [CSW-1:0] command_csid_i,
18     input command_valid_i,
19     output logic command_busy_o,
20
21     output spi_host_cmd_pkg::command_t core_command_o,
22     output logic [CSW-1:0] core_command_csid_o,
23     output logic core_command_valid_o,
24     input core_command_ready_i,
25
26     output logic [3:0] qd_o,
27
28     output logic error_busy_o,
29
30     input sw_rst_i
31 );
32
33 localparam int CmdDepthW = prim_util_pkg::vbits(CmdDepth+1);
34
35 logic command_ready;

```

```

36
37 assign command_busy_o = ~command_ready;
38 assign error_busy_o = command_valid_i & command_busy_o;
39
40 logic [CmdDepthW-1:0] cmd_depth;
41
42 prim_fifo_sync #(
43     .Width($bits(spi_host_cmd_pkg::command_t) + CSW),
44     .Pass(0),
45     .Depth(CmdDepth)
46 ) cmd_fifo (
47     .clk_i,
48     .rst_ni,
49     .clr_i (sw_rst_i),
50     .wvalid_i (command_valid_i),
51     .wready_o (command_ready),
52     .wdata_i ({command_i, command_csid_i}),
53     .rvalid_o (core_command_valid_o),
54     .rready_i (core_command_ready_i),
55     .rdata_o ({core_command_o, core_command_csid_o}),
56     .full_o (),
57     .depth_o (cmd_depth),
58     .err_o ()
59 );
60
61 assign qd_o = 4'(cmd_depth);
62
63 endmodule : spi_host_command_queue

```

6. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_core.sv (155 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Core Implementation module for Serial Peripheral Interface
  (SPI) Host IP.
6 //
7
8 module spi_host_core #(
9     parameter int NumCS = 1,
10    localparam int CSW = prim_util_pkg::vbits(NumCS)
11 ) (
12     input clk_i,
13     input rst_ni,
14
15     input spi_host_cmd_pkg::command_t command_i,
16     input logic [CSW-1:0] command_csid_i,
17     input command_valid_i,
18     output command_ready_o,
19     input en_i,
20
21     input [31:0] tx_data_i,
22     input [3:0] tx_be_i,
23     input tx_valid_i,
24     output logic tx_ready_o,
25     output logic tx_byte_select_full_o,
26
27     output logic [31:0] rx_data_o,
28     output logic rx_valid_o,
29     input rx_ready_i,
30
31     input sw_rst_i,
32
33     // SPI Interface
34     output logic sck_o,
35     output logic [NumCS-1:0] csb_o,
36     output logic [3:0] sd_o,
37     output logic [3:0] sd_en_o,
38     input [3:0] sd_i,
39
40     // Status bits
41     output logic rx_stall_o,
42     output logic tx_stall_o,
43     output logic active_o
44 );
45
46 logic rx_valid_sr;
47 logic rx_ready_sr;
48 logic rx_last_sr;
49 logic [7:0] rx_data_sr;
50 logic tx_valid_sr;
51 logic tx_ready_sr;
52 logic tx_flush_sr;
53 logic [7:0] tx_data_sr;
54
55 logic wr_en;
56 logic rd_en;
57 logic wr_ready;
58 logic rd_ready;
59
60 logic sample_en;
61 logic shift_en;
62 logic [1:0] speed;
63 logic full_cyc;
64 logic last_read;
65 logic last_write;

```

```

66
67 spi_host_byte_merge u_merge (
68     .clk_i,
69     .rst_ni,
70     .byte_i      (rx_data_sr),
71     .byte_last_i (rx_last_sr),
72     .byte_valid_i (rx_valid_sr),
73     .byte_ready_o (rx_ready_sr),
74     .word_o      (rx_data_o),
75     .word_valid_o (rx_valid_o),
76     .word_ready_i (rx_ready_i),
77     .sw_rst_i
78 );
79
80 spi_host_byte_select u_select (
81     .clk_i,
82     .rst_ni,
83     .word_i      (tx_data_i),
84     .word_be_i   (tx_be_i),
85     .word_valid_i (tx_valid_i),
86     .word_ready_o (tx_ready_o),
87     .byte_o      (tx_data_sr),
88     .byte_valid_o (tx_valid_sr),
89     .byte_ready_i (tx_ready_sr),
90     .flush_i     (tx_flush_sr),
91     .sw_rst_i
92 );
93 // The byte_select module greedily pops data from the TX
FIFO, so
94 // the FIFO is observed as having an effective depth of N+1.
95 // The byte_valid_o signal indicates when the byte_select
contains
96 // a valid word. Pass this on up to the TX FIFO to adjust
the
97 // reported depth/full/empty flags.
98 assign tx_byte_select_full_o = tx_valid_sr;
99
100 spi_host_shift_register u_shift_reg (
101     .clk_i,
102     .rst_ni,
103     .wr_en_i   (wr_en),
104     .wr_ready_o (wr_ready),
105     .rd_en_i   (rd_en),
106     .rd_ready_o (rd_ready),
107     .speed_i   (speed),
108     .shift_en_i (shift_en),
109     .sample_en_i (sample_en),
110     .last_read_i (last_read),
111     .last_write_i (last_write),
112     .full_cyc_i (full_cyc),
113     .tx_data_i   (tx_data_sr),
114     .tx_valid_i   (tx_valid_sr),
115     .tx_ready_o   (tx_ready_sr),
116     .tx_flush_o   (tx_flush_sr),
117     .rx_data_o    (rx_data_sr),
118     .rx_valid_o    (rx_valid_sr),
119     .rx_ready_i    (rx_ready_sr),
120     .rx_last_o     (rx_last_sr),
121     .sw_rst_i,
122     .sd_o          (sd_o),
123     .sd_i          (sd_i)
124 );
125
126 spi_host_fsm #(
127     .NumCS(NumCS)
128 ) u_fsm (
129     .clk_i,
130     .rst_ni,
131     .en_i,
132     .command_i,
133     .command_csid_i,
134     .command_valid_i,
135     .command_ready_o,
136     .sck_o,
137     .csb_o,
138     .sd_en_o,
139     .last_read_o   (last_read),
140     .last_write_o   (last_write),
141     .wr_en_o        (wr_en),
142     .sr_wr_ready_i  (wr_ready),
143     .rd_en_o        (rd_en),
144     .sr_rd_ready_i  (rd_ready),
145     .sample_en_o    (sample_en),
146     .shift_en_o     (shift_en),
147     .speed_o        (speed),
148     .full_cyc_o     (full_cyc),
149     .sw_rst_i,
150     .rx_stall_o,
151     .tx_stall_o,
152     .active_o
153 );
154
155 endmodule : spi_host_core

```

7. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_data_fifos.v (135 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for

details.

```

3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Module for SPI_HOST RX and TX queues
6 //
7
8 `include "prim_assert.sv"
9
10 module spi_host_data_fifos #(
11     parameter int    TxDepth    = 72,
12     parameter int    RxDepth    = 64,
13     parameter logic   SwapBytes = 0
14 ) (
15     input            clk_i,
16     input            rst_ni,
17
18     input            [31:0] tx_data_i,
19     input            [3:0]  tx_be_i,
20     input            tx_valid_i,
21     output logic      tx_ready_o,
22     input            [7:0]  tx_watermark_i,
23
24     output logic [31:0] core_tx_data_o,
25     output logic [3:0]  core_tx_be_o,
26     output logic      core_tx_valid_o,
27     input            core_tx_ready_i,
28     input            core_tx_byte_select_full_i,
29
30     input            [31:0] core_rx_data_i,
31     input            core_rx_valid_i,
32     output logic      core_rx_ready_o,
33
34     output logic [31:0] rx_data_o,
35     output logic      rx_valid_o,
36     input            rx_ready_i,
37     input            [7:0]  rx_watermark_i,
38
39     input            sw_rst_i,
40
41     output logic      tx_empty_o,
42     output logic      tx_full_o,
43     output logic [7:0] tx_qd_o,
44     output logic      tx_wm_o,
45     output logic      rx_empty_o,
46     output logic      rx_full_o,
47     output logic [7:0] rx_qd_o,
48     output logic      rx_wm_o
49 );
50
51 localparam int RxDepthW = prim_util_pkg::vbits(RxDepth+1);
52 localparam int TxDepthW = prim_util_pkg::vbits(TxDepth+1);
53
54 logic [31:0] tx_data_ordered;
55 logic [3:0]  tx_be_ordered;
56 logic [31:0] rx_data_unordered;
57
58 if (SwapBytes) begin : gen_swap
59     assign tx_data_ordered = { << 8 {tx_data_i} };
60     assign tx_be_ordered   = { << {tx_be_i} };
61     assign rx_data_o       = { << 8 {rx_data_unordered} };
62 end else begin : gen_do_not_swap
63     assign tx_data_ordered = tx_data_i;
64     assign tx_be_ordered   = tx_be_i;
65     assign rx_data_o       = rx_data_unordered;
66 end : gen_do_not_swap
67
68 logic [35:0] tx_data_be;
69 logic [35:0] core_tx_data_be;
70
71 logic [TxDepthW-1:0] tx_depth;
72
73 // The byte_select module greedily pops data from the TX
FIFO, so
74 // the FIFO should be observed as having an effective depth
of N+1.
75 assign tx_qd_o = 8'(tx_depth) + {7'b0,
core_tx_byte_select_full_i};
76
77 assign tx_data_be = { tx_data_ordered, tx_be_ordered };
78 assign { core_tx_data_o, core_tx_be_o } = core_tx_data_be;
79
80 prim_fifo_sync #(
81     .Width(36),
82     .Pass(1),
83     .Depth(TxDepth)
84 ) u_tx_fifo (
85     .clk_i,
86     .rst_ni,
87     .clr_i   (sw_rst_i),
88     .wvalid_i (tx_valid_i),
89     .wready_o (tx_ready_o),
90     .wdata_i   (tx_data_be),
91     .rvalid_o  (core_tx_valid_o),
92     .rready_i  (core_tx_ready_i),
93     .rdata_o   (core_tx_data_be),
94     .full_o    (),
95     .depth_o   (tx_depth),
96     .err_o     ()
97 );
98
99 logic [RxDepthW-1:0] rx_depth;

```

```

100
101 assign rx_qd_o = 8'(rx_depth);
102
103 prim_fifo_sync #(
104     .Width(32),
105     .Pass(1),
106     .Depth(RxDepth)
107 ) u_rx_fifo (
108     .clk_i,
109     .rst_ni,
110     .clr_i (sw_rst_i),
111     .wvalid_i (core_rx_valid_i),
112     .wready_o (core_rx_ready_o),
113     .wdata_i (core_rx_data_i),
114     .rvalid_o (rx_valid_o),
115     .rready_i (rx_ready_i),
116     .rdata_o (rx_data_unordered),
117     .full_o (),
118     .depth_o (rx_depth),
119     .err_o ()
120 );
121
122 // Create the status outputs
123 // - Include the 1-word storage in byte_select for the TX
datapath
124 assign tx_empty_o = (tx_qd_o == 0);
125 assign rx_empty_o = (rx_qd_o == 0);
126 assign tx_full_o = (tx_qd_o >= 8'(TxDepth) + 1); // inc.
byte_sel
127 assign rx_full_o = (rx_qd_o >= 8'(RxDepth));
128 assign tx_wm_o = (tx_qd_o < tx_watermark_i);
129 assign rx_wm_o = (rx_qd_o >= rx_watermark_i);
130
131 // Due to the addition of the byte_sel as a N+1'th fifo
stage, ensure
132 // we have adequate resolution in our counter (tx_qd_o) to
represent this.
133 `ASSERT_INIT(TxDepth_A, TxDepth < (2**$size(tx_qd_o)) - 1)
134
135 endmodule : spi_host_data_fifos

```

8. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_fsm.sv (626 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Core Implementation module for Serial Peripheral Interface
(SPI) Host IP.
6 //
7
8 module spi_host_fsm
9     import spi_host_cmd_pkg::*;
10
11 #(
12     parameter int NumCS = 1,
13     localparam int CSW = prim_util_pkg::vbits(NumCS)
14 ) (
15     input clk_i,
16     input rst_ni,
17     input en_i,
18     input command_t command_i,
19     input logic [CSW-1:0] command_csid_i,
20     input logic [CSW-1:0] command_valid_i,
21     input logic [CSW-1:0] command_ready_o,
22     output logic [NumCS-1:0] sck_o,
23     output logic [3:0] csb_o,
24     output logic [3:0] sd_en_o,
25     output logic last_read_o,
26     output logic last_write_o,
27     output logic wr_en_o,
28     input sr_wr_ready_i,
29     output logic rd_en_o,
30     input sr_rd_ready_i,
31     output logic sample_en_o,
32     output logic shift_en_o,
33     output logic [1:0] speed_o,
34     output logic full_cyc_o,
35     output logic rx_stall_o,
36     output logic tx_stall_o,
37     output logic active_o,
38     input sw_rst_i
39 );
40
41 logic is_idle;
42 logic [15:0] clkdiv, clkdiv_q;
43 logic [15:0] clk_cntr_q, clk_cntr_d;
44 logic clk_cntr_en;
45
46 logic [CSW-1:0] csid;
47 logic [CSW-1:0] csid_q;
48
49 logic [3:0] csnidle, csntrail, csnlead;
50 logic [3:0] csnidle_q, csntrail_q, csnlead_q;
51 logic full_cyc_q, cpha, cpol;
52 logic full_cyc_q, cpha_q, cpol_q;
53

```

```

54 // Unlike the configopts fields, the segment fields can
change in back-to-back segment operation.
55 // (If a change in only the configopts fields is detected,
the FSM transitions to idle instead).
56 // For that reason, when using the segment fields in back-
to-back operations, we have to bear
57 // in mind context and determine whether it is appropriate
to use the value for the previous
58 // segment or the following segment.
59 // For instance, the cmd_rd_en signal is sometimes used to
push segment bytes into the shift
60 // register, and so in that case it is important to use
cmd_rd_en_q in that sense. The same
61 // signal is consulted to load the bit counter at the /be-
ginning/ of each byte or dummy cycle,
62 // and so in this context it is appropriate to use
cmd_rd_en_d (which refers to the value from
63 // the immediately following segment).
64 logic [1:0] cmd_speed_d, cmd_speed_q;
65 logic cmd_wr_en_d, cmd_wr_en_q;
66 logic cmd_rd_en_d, cmd_rd_en_q;
67 logic [19:0] cmd_len_d, cmd_len_q;
68 logic csat;
69 logic csat_q;
70
71 logic [2:0] bit_cntr_d, bit_cntr_q;
72 logic [19:0] byte_cntr_cpha0_d, byte_cntr_cpha1_d,
byte_cntr_cpha0_q, byte_cntr_cpha1_q;
73 logic [19:0] byte_cntr_early, byte_cntr_late;
74 logic [3:0] wait_cntr_d, wait_cntr_q;
75 logic last_bit, last_byte;
76
77 logic state_changing;
78 logic byte_starting, byte_starting_cpha0,
byte_starting_cpha1;
79 logic bit_shifting, bit_shifting_cpha0,
bit_shifting_cpha1;
80 logic byte_ending, byte_ending_cpha0, byte_end-
ing_cpha1;
81
82 logic sample_en_d, sample_en_q, sample_en_q2;
83
84 logic config_changed;
85 logic fsm_en;
86
87 // new_command: signals a new segment input
88 logic new_command;
89
90 logic csb_single_d;
91 logic [NumCS-1:0] csb_q;
92 logic sck_d, sck_q;
93
94 logic wr_en_internal, rd_en_internal, sample_en_internal,
shift_en_internal;
95
96 logic stall;
97
98 assign stall = rx_stall_o | tx_stall_o;
99
100 // suppress output pulses if stalled.
101 assign wr_en_o = wr_en_internal & ~stall;
102 assign rd_en_o = rd_en_internal & ~stall;
103 assign sample_en_o = sample_en_internal & ~stall;
104 assign shift_en_o = shift_en_internal & ~stall;
105
106 typedef enum logic [2:0] {
107     Idle,
108     WaitLead,
109     InternalClkLow,
110     InternalClkHigh,
111     WaitTrail,
112     WaitIdle,
113     CSBSwitch,
114     IdleCSBActive
115 } spi_host_st_e;
116
117 spi_host_st_e state_q, state_d;
118
119 logic command_ready_int;
120 assign command_ready_o = command_ready_int & ~stall;
121
122
123 assign new_command = command_valid_i && com-
mand_ready_int;
124 assign config_changed = (command_i.configopts.cpol !=
cpol_q) ||
125 (command_i.configopts.cpha !=
cpha_q) ||
126 (command_i.configopts.full_cyc !=
full_cyc_q) ||
127 (command_i.configopts.csnidle !=
csnidle_q) ||
128 (command_i.configopts.csntrail !=
csntrail_q) ||
129 (command_i.configopts.csnlead !=
csnlead_q) ||
130 (command_i.configopts.clkdiv !=
clkdiv_q);
131
132 always_comb begin
133     csid = new_command ? command_csid_i : csid_q;

```

```

134 cpol      = new_command ? command_i.configopts.cpol :
cpol_q;
135 cpha      = new_command ? command_i.configopts.cpha :
cpha_q;
136 full_cyc  = new_command ? command_i.configopts.full_cyc :
full_cyc_q;
137 csnidle   = new_command ? command_i.configopts.csnidle :
csnidle_q;
138 csnlead   = new_command ? command_i.configopts.csnlead :
csnlead_q;
139 csntrail  = new_command ? command_i.configopts.csntrail :
csntrail_q;
140 clkdiv    = new_command ? command_i.configopts.clkdiv :
clkdiv_q;
141 csaat     = new_command ? command_i.segment.csaat :
csaat_q;
142 cmd_len_d = new_command ? command_i.segment.len :
cmd_len_q;
143 cmd_wr_en_d = new_command ? command_i.segment.cmd_wr_en :
cmd_wr_en_q;
144 cmd_rd_en_d = new_command ? command_i.segment.cmd_rd_en :
cmd_rd_en_q;
145 cmd_speed_d = new_command ? command_i.segment.speed :
cmd_speed_q;
146 end
147
148 always_ff @(posedge clk_i or negedge rst_ni) begin
149   if (!rst_ni) begin
150     csid_q    <= {CSW{1'b0}};
151     cpol_q    <= 1'b0;
152     cpha_q    <= 1'b0;
153     full_cyc_q <= 1'b0;
154     csnidle_q <= 4'h0;
155     csnlead_q <= 4'h0;
156     csntrail_q <= 4'h0;
157     clkdiv_q  <= 16'h0;
158     csaat_q   <= 1'b0;
159     cmd_rd_en_q <= 1'b0;
160     cmd_wr_en_q <= 1'b0;
161     cmd_speed_q <= 2'b00;
162     cmd_len_q <= 20'h0;
163   end else begin
164     csid_q    <= (new_command && !stall) ? csid : csid_q;
165     cpol_q    <= (new_command && !stall) ? cpol : cpol_q;
166     cpha_q    <= (new_command && !stall) ? cpha : cpha_q;
167     full_cyc_q <= (new_command && !stall) ? full_cyc :
full_cyc_q;
168     csnidle_q <= (new_command && !stall) ? csnidle :
csnidle_q;
169     csnlead_q <= (new_command && !stall) ? csnlead : csn-
lead_q;
170     csntrail_q <= (new_command && !stall) ? csntrail :
csntrail_q;
171     clkdiv_q  <= (new_command && !stall) ? clkdiv : clk-
div_q;
172     csaat_q   <= (new_command && !stall) ? csaat :
csaat_q;
173     cmd_wr_en_q <= (new_command && !stall) ? cmd_wr_en_d :
cmd_wr_en_q;
174     cmd_rd_en_q <= (new_command && !stall) ? cmd_rd_en_d :
cmd_rd_en_q;
175     cmd_speed_q <= (new_command && !stall) ? cmd_speed_d :
cmd_speed_q;
176     cmd_len_q <= (new_command && !stall) ? cmd_len_d :
cmd_len_q;
177   end
178 end
179
180 assign is_idle      = (state_q == Idle) || (state_q == IdleC-
SBActive);
181
182 assign active_o     = ~is_idle;
183
184 // New commands must always ensure there is the full setup
time available.
185 assign clk_cntr_d = sw_rst_i      ? 16'h0 :
!clk_cntr_en      ? clk_cntr_q :
186 new_command      ? clkdiv :
187 is_idle          ? clk_cntr_q :
188 (clk_cntr_q == 16'h0) ? clkdiv :
189 clk_cntr_q - 1;
190
191
192 assign tx_stall_o = wr_en_internal & ~sr_wr_ready_i;
193 assign rx_stall_o = rd_en_internal & ~sr_rd_ready_i;
194 assign clk_cntr_en = en_i;
195 assign fsm_en = (clk_cntr_en && ((clk_cntr_q == 0) ||
is_idle));
196
197 spi_host_st_e next_state_after_idle;
198 always_comb begin
199   if (command_valid_i) begin
200     if (config_changed) begin
201       next_state_after_idle = CSBSwitch;
202     end else begin
203       next_state_after_idle = WaitLead;
204     end
205   end else begin
206     next_state_after_idle = Idle;
207   end
208 end
209

```

```

210 spi_host_st_e next_state_after_idle_csb_active;
211 logic      command_ready_idle_csb_active;
212 always_comb begin
213   if (command_valid_i) begin
214     if (command_csid_i != csid_q) begin
215       //
216       // Do not acknowledge the command now, as it will
trigger
217
218       // an update of the internal command and configuration
registers.
219       // *Silently* transition to WaitTrail. The command
// will be acknowledged later, at the end of the
WaitIdle state.
220       //
221       next_state_after_idle_csb_active = WaitTrail;
222       // Explicitly *suppress* command_ready
command_ready_idle_csb_active = 1'b0;
223     end else begin
224       if (cpha_q) begin
225         next_state_after_idle_csb_active = InternalClkHigh;
226       end else begin
227         next_state_after_idle_csb_active = InternalClkLow;
228       end
229       command_ready_idle_csb_active = 1'b1;
230     end
231   end else begin
232     next_state_after_idle_csb_active = IdleCSBActive;
233     command_ready_idle_csb_active = 1'b1;
234   end
235 end
236
237 //
238 // FSM main body: Controls state transitions and com-
mand_ready_o signaling
239 //
240 always_comb begin
241   state_d = state_q;
242   command_ready_int = 1'b0;
243   if (sw_rst_i) begin
244     state_d = Idle;
245   end else if (fsm_en) begin
246     unique case (state_q)
247       Idle: begin
248         // Initial state, wait for commands.
249         command_ready_int = 1'b1;
250         state_d = next_state_after_idle;
251       end
252       WaitLead: begin
253         // Transaction lead: CSB is low, waiting to start
sck pulses.
254         if (wait_cntr_q == 4'h0) begin
255           state_d = InternalClkHigh;
256         end
257       end
258       InternalClkLow: begin
259         // Launch clock and full-cycle sampling clock for
CPHA=0.
260         // Half-cycle sampling clock for CPHA=1.
261         // End-of-transaction clock level for both CPHA=0
and CPHA=1.
262         // One of two active clock states. sck is low when
CPOL=0.
263         if (!last_bit || !last_byte || !cpha_q) begin
264           state_d = InternalClkHigh;
265         end else if (!csaat_q) begin
266           state_d = WaitTrail;
267         end else begin
268           state_d = next_state_after_idle_csb_active;
269           command_ready_int = command_ready_idle_csb_active;
270         end
271       end
272       InternalClkHigh: begin
273         // Half-cycle sampling clock for CPHA=0.
274         // Launch clock for CPHA=1. Note: Full-cycle
sampling is not compatible with CPHA=1.
275         // One of two active clock states. sck is low when
CPOL=0.
276         // Typically often the last state in a command, and
so the next state depends on CSAAT,
277         // and of CSAAT is asserted, the details of the sub-
sequent command.
278         if (!last_bit || !last_byte || cpha_q) begin
279           state_d = InternalClkLow;
280         end
281       end
282       // Check value of csaat for the previously submitted
segment
283       end else if (!csaat_q) begin
284         state_d = WaitTrail;
285       end else begin
286         state_d = next_state_after_idle_csb_active;
287         command_ready_int = command_ready_idle_csb_active;
288       end
289     end
290     WaitTrail: begin
291       // Prepare to enter CSB high idle state by waiting
csntrail cycles.
292       if (wait_cntr_q == 4'h0) begin
293         state_d = WaitIdle;
294       end
295     end
296   end
297 end
298

```



```

295     WaitIdle: begin
296         // Once CSB is high, wait for the designated number
of cycles before accepting commands.
297         if (wait_cntr_q == 4'h0) begin
298             // ready to accept new command
299             command_ready_int = 1'b1;
300             state_d = next_state_after_idle;
301         end
302     end
303     CSBSwitch: begin
304         // Insert extra idle cycles when swtiching between
CSID, this allows time to switch CPHA,
305         // CPOL and clkdiv settings, as well as guarantee
that the idle delay requirements have
306         // been observed for the new device.
307         if (wait_cntr_q == 4'h0) begin
308             state_d = WaitLead;
309         end else begin
310             state_d = CSBSwitch;
311         end
312     end
313     IdleCSBActive: begin
314         // Wait for new commands, but with CSB held active
low.
315         state_d = next_state_after_idle_csb_active;
316         command_ready_int = command_ready_idle_csb_active;
317     end
318     default: begin
319         command_ready_int = 1'b0;
320         state_d = Idle;
321     end
322 endcase
323 end
324 end
325
326 // All register updates freeze when a stall is detected.
327 // The definition of the stall signal looks ahead to determ-
ine whether a conflict is looming.
328 // Thus stall depends on state_d. Making state_d depend on
stall
329 // would create a circular logic loop, and lint errors.
Therefore stall is applied here, not
330 // in the previous always_comb block;
331 always_ff @(posedge clk_i or negedge rst_ni) begin
332     if (!rst_ni) begin
333         state_q <= Idle;
334         clk_cntr_q <= 16'h0;
335     end else begin
336         state_q <= stall ? state_q : state_d;
337         clk_cntr_q <= stall ? clk_cntr_q : clk_cntr_d;
338     end
339 end
340
341 logic segment_rd_en, segment_rd_en_cpha0, seg-
ment_rd_en_cpha1;
342
343 assign state_changing = (state_q != state_d);
344 assign byte_starting_cpha0 = ~sw_rst_i & state_changing &
((state_d == WaitLead) |
345 (state_d == InternalClkLow &
bit_cntr_q==0));
346 assign bit_shifting_cpha0 = ~sw_rst_i & state_changing &
(state_d == InternalClkLow &
bit_cntr_q != 0);
347 assign byte_ending_cpha0 = ~sw_rst_i & state_changing &
(state_q == InternalClkHigh &
bit_cntr_q == 0);
348
349 assign segment_rd_en_cpha0 = cmd_rd_en_q;
350
351 // We can calculate byte transitions for CPHA=1 by noting
352 // that in this implementation, the sck edges have a 1-1
353 // correspondence with FSM transitions.
354 // New bytes are loaded exactly one state transition behind
the time
355 // when they would be loaded if CPHA=0
356 always_ff @(posedge clk_i or negedge rst_ni) begin
357     if (!rst_ni) begin
358         segment_rd_en_cpha1 <= 1'b0;
359     end else if (state_changing && !stall) begin
360         segment_rd_en_cpha1 <= segment_rd_en_cpha0;
361     end
362 end
363
364 // For CPHA=1, the launch and capture states flip roles, and
data output
365 // does not begin in the WaitIdle state, when CSB first as-
serts. They
366 // otherwise look very similar to the CPHA=0 case.
367 assign byte_starting_cpha1 = ~sw_rst_i & state_changing &
(state_d == InternalClkHigh &
bit_cntr_q==0);
368 assign bit_shifting_cpha1 = ~sw_rst_i & state_changing &
(state_d == InternalClkHigh &
bit_cntr_q != 0);
369 assign byte_ending_cpha1 = ~sw_rst_i & state_changing &
(state_q == InternalClkLow &
bit_cntr_q == 0);
370
371
372

```

```

378 assign byte_starting = (cpha == 1'b0) ? byte_start-
ing_cpha0 :
379 byte_starting_cpha1;
380
381 assign byte_ending = (cpha == 1'b0) ? byte_ending_cpha0 :
byte_ending_cpha1;
382
383 assign bit_shifting = (cpha == 1'b0) ? bit_shifting_cpha0 :
bit_shifting_cpha1;
384
385 assign speed_o = cmd_speed_q;
386
387 assign segment_rd_en = (cpha == 1'b0) ? segment_rd_en_cpha0 :
segment_rd_en_cpha1;
388
389 assign byte_cntr_early = (cpha == 1'b0) ?
byte_cntr_cpha0_d :
390 byte_cntr_cpha1_d;
391
392 assign byte_cntr_late = (cpha == 1'b0) ?
byte_cntr_cpha0_q :
393 byte_cntr_cpha1_q;
394
395 logic [2:0] shift_size;
396 logic [2:0] start_bit;
397
398 always_comb begin
399     if (!cmd_rd_en_d && !cmd_wr_en_d) begin
400         // direction == 0, means to send out
401         // a fixed number of pulses, NOT bytes.
402         // thus "last_bit" is always asserted,
403         // and the number of pulses is counted
404         // by byte_cntr_d.
405         shift_size = 0;
406         start_bit = 3'h0;
407     end else begin
408         unique case (cmd_speed_d)
409             Standard: begin
410                 shift_size = 3'h1;
411                 start_bit = 3'h7;
412             end
413             Dual: begin
414                 shift_size = 3'h2;
415                 start_bit = 3'h6;
416             end
417             Quad: begin
418                 shift_size = 3'h4;
419                 start_bit = 3'h4;
420             end
421             default: begin
422                 // Invalid speed;
423                 shift_size = 3'h1;
424                 start_bit = 3'h1;
425             end
426         endcase
427     end
428 end
429
430 assign bit_cntr_d = sw_rst_i ? 3'h0 :
!fsm_en ? bit_cntr_q :
431 byte_starting ? start_bit :
432 bit_shifting ? bit_cntr_q -
shift_size :
433 bit_cntr_q;
434
435 assign last_bit = (bit_cntr_q == 3'h0);
436
437 // The variable last_byte is only used for updating the FSM
state.
438 // For CPHA=1 operation, either byte_cntr_cpha0_q or
byte_cntr_cpha1_q
439 // can drive the FSM properly. However, we explicitly
choose
440 // byte_cntr_cpha0_q to avoid a combinational logic loop.
441 //
442 always_comb begin
443     if (cpha_q) begin
444         last_byte = (byte_cntr_cpha1_q == 20'h0);
445     end else begin
446         last_byte = (byte_cntr_cpha0_q == 20'h0);
447     end
448 end
449
450 // Note: when updating the byte_cntr in CPHA=0 mode with a
new command value, the length must
451 // be pulled in directly from the command bus, cmd_len_d;
452 assign byte_cntr_cpha0_d = sw_rst_i ? 20'h0 :
!fsm_en ? byte_cntr_cpha0_q :
453 new_command ? cmd_len_d :
454 byte_ending_cpha0 ?
byte_cntr_cpha0_q - 1 :
455 byte_cntr_cpha0_q;
456
457 assign byte_cntr_cpha1_d = sw_rst_i ? 20'h0 :
!fsm_en ?
458 new_command ? cmd_len_d :
459 byte_ending_cpha1 ?
byte_cntr_cpha1_q - 1 :
460 byte_cntr_cpha1_q;
461
462 always_comb begin

```

```

468 if(sw_rst_i) begin
469     wait_cntr_d = 4'b0;
470 end else if (!fsm_en) begin
471     wait_cntr_d = wait_cntr_q;
472 end else if (state_changing) begin
473     unique case (state_d)
474         WaitLead: begin
475             wait_cntr_d = csnlead;
476         end
477         WaitTrail: begin
478             wait_cntr_d = csntrail;
479         end
480         WaitIdle: begin
481             wait_cntr_d = csnidle;
482         end
483         CSBSwitch: begin
484             wait_cntr_d = csnidle;
485         end
486         default: begin
487             // Hold wait_cntr to zero
488             // for states that don't use it
489             wait_cntr_d = 4'b0;
490         end
491     endcase
492 end else if (wait_cntr_q == 0) begin
493     wait_cntr_d = 4'h0;
494 end else begin
495     wait_cntr_d = wait_cntr_q - 1;
496 end
497 end
498
499 always_ff @(posedge clk_i or negedge rst_ni) begin
500     if (!rst_ni) begin
501         bit_cntr_q    <= 3'h0;
502         byte_cntr_cpha0_q <= 20'h0;
503         byte_cntr_cpha1_q <= 20'h0;
504         wait_cntr_q    <= 4'h0;
505     end else begin
506         bit_cntr_q    <= stall ? bit_cntr_q :
507         byte_cntr_cpha0_q <= stall ? byte_cntr_cpha0_q :
508         byte_cntr_cpha1_q <= stall ? byte_cntr_cpha1_q :
509         wait_cntr_q    <= stall ? wait_cntr_q :
510     end
511 end
512
513 assign wr_en_internal = byte_starting & cmd_wr_en_d;
514 assign shift_en_internal = bit_shifting;
515
516 assign rd_en_internal = byte_ending & segment_rd_en;
517 assign sample_en_d = byte_starting | shift_en_o;
518 assign full_cyc_o = full_cyc;
519
520 assign last_read_o = (byte_cntr_late == 'h0) & rd_en_o &
521 sr_rd_ready_i;
522
523 assign last_write_o = (byte_cntr_early == 'h0) &
524 wr_en_o & sr_wr_ready_i;
525
526 always_ff @(posedge clk_i or negedge rst_ni) begin
527     if (!rst_ni) begin
528         sample_en_q <= 1'b0;
529         sample_en_q2 <= 1'b0;
530     end else begin
531         sample_en_q <= (fsm_en && !stall) ? sample_en_d :
532         sample_en_q2 <= (fsm_en && !stall) ? sample_en_q :
533         sample_en_q2;
534     end
535 end
536
537 assign sample_en_internal = full_cyc_o ? sample_en_q2 :
538 sample_en_q;
539
540 always_comb begin
541     unique case (state_d)
542         WaitLead, InternalClkLow, InternalClkHigh, IdleCSBAct-
543         ive, WaitTrail:
544             csb_single_d = 1'b0;
545         default:
546             csb_single_d = 1'b1;
547     endcase
548 end
549
550 assign sck_d = cpol ? (state_d != InternalClkHigh) :
551 (state_d == InternalClkHigh);
552
553 assign sck_o = sck_q;
554
555 prim_flop_en u_sck_flop (
556     .clk_i,
557     .rst_ni,
558     .en_i(~stall),
559     .d_i(sck_d),
560     .q_o(sck_q)
561 );
562
563 for (genvar ii = 0; ii < NumCS; ii = ii + 1) begin :

```

```

gen_csb_gen
558 always_ff @(posedge clk_i or negedge rst_ni) begin
559     if (!rst_ni) begin
560         csb_q[ii] <= 1'b1;
561     end else begin
562         csb_q[ii] <= (csid != ii) ? 1'b1 :
563         !stall ? csb_single_d :
564         csb_q[ii];
565     end
566 end
567 end : gen_csb_gen
568
569 // When 'cmd_wr_en_q' deasserts, we need to stretch the en-
570 able used
571 // to drive sd_en_o[0] until the end of the bit period. This
572 is because
573 // 'cmd_wr_en_q' can change up to 1/2 cycle before the end
574 of the period
575 // (depending on cpol/cpha).
576 // The signal 'last_bit' is active in the final period, so
577 use that to start
578
579 // stretching the enable, and 'byte_ending' marks the period
580 end.
581 logic cmd_wr_en_last_bit;
582 always_ff @(posedge clk_i or negedge rst_ni) begin
583     if (!rst_ni) begin
584         cmd_wr_en_last_bit <= 1'b0;
585     end else if (cmd_wr_en_q & last_bit) begin
586         cmd_wr_en_last_bit <= 1'b1;
587     end else if (byte_ending) begin
588         cmd_wr_en_last_bit <= 1'b0;
589     end else begin // Stretching..
590         cmd_wr_en_last_bit <= cmd_wr_en_last_bit;
591     end
592 end
593
594 assign csb_o = csb_q;
595
596 always_comb begin
597     if (&csb_o) begin
598         sd_en_o[3:0] = 4'h0;
599     end else begin
600         unique case (speed_o)
601             Standard: begin
602                 // Observing 'last_bit' ensures we do not deassert
603                 the enable too early
604                 sd_en_o[0] = cmd_wr_en_q | cmd_wr_en_last_bit;
605                 sd_en_o[1] = 1'b0;
606                 sd_en_o[3:2] = 2'b00;
607             end
608             Dual: begin
609                 sd_en_o[1:0] = {2{cmd_wr_en_q}};
610                 sd_en_o[3:2] = 2'b00;
611             end
612             Quad: begin
613                 sd_en_o[3:0] = {4{cmd_wr_en_q}};
614             end
615             default: begin
616                 // invalid speed
617                 sd_en_o[3:0] = 4'h0;
618             end
619         endcase
620     end
621 end
622
623 `ASSERT(BidirOnlyInStdMode_A,
624 cmd_speed_d == Standard || !(cmd_rd_en_d &&
625 cmd_wr_en_d),
626 clk_i, rst_ni)
627 `ASSERT(ValidSpeed_A, cmd_speed_d != RsVdSpd, clk_i, rst_ni)
628 `ASSERT(ValidCSID_A, csid < NumCS, clk_i, rst_ni)
629
630 endmodule

```

9. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_pkg.sv (375 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // Register Package auto-generated by `reggen` containing data
7 structure
8
9 package spi_host_reg_pkg;
10
11 // Param list
12 parameter logic ByteOrder = 1;
13 parameter int TxDepth = 72;
14 parameter int RxDepth = 64;
15 parameter int CmdDepth = 4;

```



```

14  parameter int NumAlerts = 1;
15
16  // Address widths within the block
17  parameter int BlockAw = 6;
18
19  // Number of registers for every interface
20  parameter int NumRegs = 12;
21
22  // Alert indices
23  typedef enum int {
24      AlertFatalFaultIdx = 0
25  } spi_host_alert_idx_t;
26
27  ///////////////////////////////////////////////////
28  // Typedefs for registers //
29  ///////////////////////////////////////////////////
30
31  typedef struct packed {
32      struct packed {
33          logic q;
34      } spi_event;
35      struct packed {
36          logic q;
37      } error;
38  } spi_host_reg2hw_intr_state_reg_t;
39
40  typedef struct packed {
41      struct packed {
42          logic q;
43      } spi_event;
44      struct packed {
45          logic q;
46      } error;
47  } spi_host_reg2hw_intr_enable_reg_t;
48
49  typedef struct packed {
50      struct packed {
51          logic q;
52          logic qe;
53      } spi_event;
54      struct packed {
55          logic q;
56          logic qe;
57      } error;
58  } spi_host_reg2hw_intr_test_reg_t;
59
60  typedef struct packed {
61      logic q;
62      logic qe;
63  } spi_host_reg2hw_alert_test_reg_t;
64
65  typedef struct packed {
66      struct packed {
67          logic q;
68      } spien;
69      struct packed {
70          logic q;
71      } sw_rst;
72      struct packed {
73          logic q;
74      } output_en;
75      struct packed {
76          logic [7:0] q;
77      } tx_watermark;
78      struct packed {
79          logic [7:0] q;
80      } rx_watermark;
81  } spi_host_reg2hw_control_reg_t;
82
83  typedef struct packed {
84      struct packed {
85          logic q;
86      } cpol;
87      struct packed {
88          logic q;
89      } cpha;
90      struct packed {
91          logic q;
92      } fullcyc;
93      struct packed {
94          logic [3:0] q;
95      } csnlead;
96      struct packed {
97          logic [3:0] q;
98      } csntail;
99      struct packed {
100         logic [3:0] q;
101     } csnidle;
102     struct packed {
103         logic [15:0] q;
104     } clkdiv;
105 } spi_host_reg2hw_configopts_reg_t;
106
107 typedef struct packed {
108     logic [31:0] q;
109 } spi_host_reg2hw_csid_reg_t;
110
111 typedef struct packed {
112     struct packed {
113         logic [19:0] q;
114         logic qe;
115     } len;
116     struct packed {
117         logic [1:0] q;
118         logic qe;
119     } direction;
120     struct packed {
121         logic [1:0] q;
122         logic qe;
123     } speed;
124     struct packed {
125         logic q;
126         logic qe;
127     } csat;
128 } spi_host_reg2hw_command_reg_t;
129
130 typedef struct packed {
131     struct packed {
132         logic q;
133     } csidinvl;
134     struct packed {
135         logic q;
136     } cmdinvl;
137     struct packed {
138         logic q;
139     } underflow;
140     struct packed {
141         logic q;
142     } overflow;
143     struct packed {
144         logic q;
145     } cmdbusy;
146 } spi_host_reg2hw_error_enable_reg_t;
147
148 typedef struct packed {
149     struct packed {
150         logic q;
151     } accessinvl;
152     struct packed {
153         logic q;
154     } csidinvl;
155     struct packed {
156         logic q;
157     } cmdinvl;
158     struct packed {
159         logic q;
160     } underflow;
161     struct packed {
162         logic q;
163     } overflow;
164     struct packed {
165         logic q;
166     } cmdbusy;
167 } spi_host_reg2hw_error_status_reg_t;
168
169 typedef struct packed {
170     struct packed {
171         logic q;
172     } idle;
173     struct packed {
174         logic q;
175     } ready;
176     struct packed {
177         logic q;
178     } txwm;
179     struct packed {
180         logic q;
181     } rxwm;
182     struct packed {
183         logic q;
184     } txempty;
185     struct packed {
186         logic q;
187     } rxfull;
188 } spi_host_reg2hw_event_enable_reg_t;
189
190 typedef struct packed {
191     struct packed {
192         logic d;
193         logic de;
194     } spi_event;
195     struct packed {
196         logic d;
197         logic de;
198     } error;
199 } spi_host_hw2reg_intr_state_reg_t;
200
201 typedef struct packed {
202     struct packed {
203         logic d;
204         logic de;
205     } ready;
206     struct packed {
207         logic d;
208         logic de;
209     } active;
210     struct packed {
211         logic d;
212         logic de;
213     } txfull;
214     struct packed {
215         logic d;

```

```

216     logic      de;
217 } txempty;
218 struct packed {
219     logic      d;
220     logic      de;
221 } txstall;
222 struct packed {
223     logic      d;
224     logic      de;
225 } txwm;
226 struct packed {
227     logic      d;
228     logic      de;
229 } rxfull;
230 struct packed {
231     logic      d;
232     logic      de;
233 } rxempty;
234 struct packed {
235     logic      d;
236     logic      de;
237 } rxstall;
238 struct packed {
239     logic      d;
240     logic      de;
241 } byteorder;
242 struct packed {
243     logic      d;
244     logic      de;
245 } rxwm;
246 struct packed {
247     logic [3:0] d;
248     logic      de;
249 } cmdqd;
250 struct packed {
251     logic [7:0] d;
252     logic      de;
253 } rxqd;
254 struct packed {
255     logic [7:0] d;
256     logic      de;
257 } txqd;
258 } spi_host_hw2reg_status_reg_t;
259
260 typedef struct packed {
261     struct packed {
262         logic      d;
263         logic      de;
264     } accessinval;
265     struct packed {
266         logic      d;
267         logic      de;
268     } csidinval;
269     struct packed {
270         logic      d;
271         logic      de;
272     } cmdinval;
273     struct packed {
274         logic      d;
275         logic      de;
276     } underflow;
277     struct packed {
278         logic      d;
279         logic      de;
280     } overflow;
281     struct packed {
282         logic      d;
283         logic      de;
284     } cmdbusy;
285 } spi_host_hw2reg_error_status_reg_t;
286
287 // Register -> HW type
288 typedef struct packed {
289     spi_host_reg2hw_intr_state_reg_t intr_state; // [137:136]
290     spi_host_reg2hw_intr_enable_reg_t intr_enable; //
291     spi_host_reg2hw_intr_test_reg_t intr_test; // [133:130]
292     spi_host_reg2hw_alert_test_reg_t alert_test; // [129:128]
293     spi_host_reg2hw_control_reg_t control; // [127:109]
294     spi_host_reg2hw_configopts_reg_t configopts; // [108:78]
295     spi_host_reg2hw_csid_reg_t csid; // [77:46]
296     spi_host_reg2hw_command_reg_t command; // [45:17]
297     spi_host_reg2hw_error_enable_reg_t error_enable; //
298     spi_host_reg2hw_error_status_reg_t error_status; // [11:6]
299     spi_host_reg2hw_event_enable_reg_t event_enable; // [5:0]
300 } spi_host_reg2hw_t;
301
302 // HW -> register type
303 typedef struct packed {
304     spi_host_hw2reg_intr_state_reg_t intr_state; // [60:57]
305     spi_host_hw2reg_status_reg_t status; // [56:12]
306     spi_host_hw2reg_error_status_reg_t error_status; // [11:0]
307 } spi_host_hw2reg_t;
308
309 // Register offsets
310 parameter logic [BlockAw-1:0] SPI_HOST_INTR_STATE_OFFSET =
6'h 0;
311 parameter logic [BlockAw-1:0] SPI_HOST_INTR_ENABLE_OFFSET =
6'h 4;
312 parameter logic [BlockAw-1:0] SPI_HOST_INTR_TEST_OFFSET =

```

```

6'h 8;
313 parameter logic [BlockAw-1:0] SPI_HOST_ALERT_TEST_OFFSET =
6'h c;
314 parameter logic [BlockAw-1:0] SPI_HOST_CONTROL_OFFSET = 6'h
10;
315 parameter logic [BlockAw-1:0] SPI_HOST_STATUS_OFFSET = 6'h
14;
316 parameter logic [BlockAw-1:0] SPI_HOST_CONFIGOPTS_OFFSET =
6'h 18;
317 parameter logic [BlockAw-1:0] SPI_HOST_CSID_OFFSET = 6'h 1c;
318 parameter logic [BlockAw-1:0] SPI_HOST_COMMAND_OFFSET = 6'h
20;
319 parameter logic [BlockAw-1:0] SPI_HOST_ERROR_ENABLE_OFFSET =
6'h 2c;
320 parameter logic [BlockAw-1:0] SPI_HOST_ERROR_STATUS_OFFSET =
6'h 30;
321 parameter logic [BlockAw-1:0] SPI_HOST_EVENT_ENABLE_OFFSET =
6'h 34;
322
323 // Reset values for hwext registers and their fields
324 parameter logic [1:0] SPI_HOST_INTR_TEST_RESVAL = 2'h 0;
325 parameter logic [0:0] SPI_HOST_INTR_TEST_ERROR_RESVAL = 1'h
0;
326 parameter logic [0:0] SPI_HOST_INTR_TEST_SPI_EVENT_RESVAL =
1'h 0;
327 parameter logic [0:0] SPI_HOST_ALERT_TEST_RESVAL = 1'h 0;
328 parameter logic [0:0] SPI_HOST_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
329 parameter logic [24:0] SPI_HOST_COMMAND_RESVAL = 25'h 0;
330 parameter logic [0:0] SPI_HOST_COMMAND_CSAAT_RESVAL = 1'h 0;
331 parameter logic [1:0] SPI_HOST_COMMAND_SPEED_RESVAL = 2'h 0;
332 parameter logic [1:0] SPI_HOST_COMMAND_DIRECTION_RESVAL =
2'h 0;
333 parameter logic [19:0] SPI_HOST_COMMAND_LEN_RESVAL = 20'h 0;
334
335 // Window parameters
336 parameter logic [BlockAw-1:0] SPI_HOST_RXDATA_OFFSET = 6'h
24;
337 parameter int unsigned SPI_HOST_RXDATA_SIZE = 'h 4;
338 parameter int unsigned SPI_HOST_RXDATA_IDX = 0;
339 parameter logic [BlockAw-1:0] SPI_HOST_TXDATA_OFFSET = 6'h
28;
340 parameter int unsigned SPI_HOST_TXDATA_SIZE = 'h 4;
341 parameter int unsigned SPI_HOST_TXDATA_IDX = 1;
342
343 // Register index
344 typedef enum int {
345     SPI_HOST_INTR_STATE,
346     SPI_HOST_INTR_ENABLE,
347     SPI_HOST_INTR_TEST,
348     SPI_HOST_ALERT_TEST,
349     SPI_HOST_CONTROL,
350     SPI_HOST_STATUS,
351     SPI_HOST_CONFIGOPTS,
352     SPI_HOST_CSID,
353     SPI_HOST_COMMAND,
354     SPI_HOST_ERROR_ENABLE,
355     SPI_HOST_ERROR_STATUS,
356     SPI_HOST_EVENT_ENABLE
357 } spi_host_id_e;
358
359 // Register width information to check illegal writes
360 parameter logic [3:0] SPI_HOST_PERMIT [12] = '{
361     4'b 0001, // index[ 0] SPI_HOST_INTR_STATE
362     4'b 0001, // index[ 1] SPI_HOST_INTR_ENABLE
363     4'b 0001, // index[ 2] SPI_HOST_INTR_TEST
364     4'b 0001, // index[ 3] SPI_HOST_ALERT_TEST
365     4'b 1111, // index[ 4] SPI_HOST_CONTROL
366     4'b 1111, // index[ 5] SPI_HOST_STATUS
367     4'b 1111, // index[ 6] SPI_HOST_CONFIGOPTS
368     4'b 1111, // index[ 7] SPI_HOST_CSID
369     4'b 1111, // index[ 8] SPI_HOST_COMMAND
370     4'b 0001, // index[ 9] SPI_HOST_ERROR_ENABLE
371     4'b 0001, // index[10] SPI_HOST_ERROR_STATUS
372     4'b 0001 // index[11] SPI_HOST_EVENT_ENABLE
373 };
374
375 endpackage

```

10. lowRISC_opentitan/hw/ip/spi_host/rtl/spi_host_reg_top.sv (2,055 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module spi_host_reg_top
10 # (
11     parameter bit      EnableRac1           = 1'b0,
12     parameter bit      Rac1ErrorRsp         = 1'b1,

```

```

13   parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
cySelVec[spi_host_reg_pkg::NumRegs] =
14   '{spi_host_reg_pkg::NumRegs{0}}
15   ) (
16   input clk_i,
17   input rst_ni,
18   input tlul_pkg::tl_h2d_t tl_i,
19   output tlul_pkg::tl_d2h_t tl_o,
20
21   // Output port for window
22   output tlul_pkg::tl_h2d_t tl_win_o [2],
23   input  tlul_pkg::tl_d2h_t tl_win_i [2],
24
25   // To HW
26   output spi_host_reg_pkg::spi_host_reg2hw_t reg2hw, // Write
27   input  spi_host_reg_pkg::spi_host_hw2reg_t hw2reg, // Read
28
29   // RACL interface
30   input top_racl_pkg::racl_policy_vec_t racl_policies_i,
31   output top_racl_pkg::racl_error_log_t  racl_error_o,
32
33   // Integrity check errors
34   output logic intg_err_o
35 );
36
37 import spi_host_reg_pkg::* ;
38
39 localparam int AW = 6;
40 localparam int DW = 32;
41 localparam int DBW = DW/8; // Byte Width
42
43 // register signals
44 logic reg_we;
45 logic reg_re;
46 logic [AW-1:0] reg_addr;
47 logic [DW-1:0] reg_wdata;
48 logic [DBW-1:0] reg_be;
49 logic [DW-1:0] reg_rdata;
50 logic reg_error;
51
52 logic addrmiss, wr_err;
53
54 logic [DW-1:0] reg_rdata_next;
55 logic reg_busy;
56
57 tlul_pkg::tl_h2d_t tl_reg_h2d;
58 tlul_pkg::tl_d2h_t tl_reg_d2h;
59
60
61 // incoming payload check
62 logic intg_err;
63 tlul_cmd_intg_chk u_chk (
64   .tl_i(tl_i),
65   .err_o(intg_err)
66 );
67
68 // also check for spurious write enables
69 logic reg_we_err;
70 logic [11:0] reg_we_check;
71 prim_reg_we_check #(
72   .OneHotWidth(12)
73 ) u_prim_reg_we_check (
74   .clk_i(clk_i),
75   .rst_ni(rst_ni),
76   .oh_i (reg_we_check),
77   .en_i (reg_we && !addrmiss),
78   .err_o (reg_we_err)
79 );
80
81 logic err_q;
82 always_ff @(posedge clk_i or negedge rst_ni) begin
83   if (!rst_ni) begin
84     err_q <= '0;
85   end else if (intg_err || reg_we_err) begin
86     err_q <= 1'b1;
87   end
88 end
89
90 // integrity error output is permanent and should be used
for alert generation
91 // register errors are transactional
92 assign intg_err_o = err_q | intg_err | reg_we_err;
93
94 // outgoing integrity generation
95 tlul_pkg::tl_d2h_t tl_o_pre;
96 tlul_rsp_intg_gen #(
97   .EnableRspIntgGen(1),
98   .EnableDataIntgGen(1)
99 ) u_rsp_intg_gen (
100   .tl_i(tl_o_pre),
101   .tl_o(tl_o)
102 );
103
104 tlul_pkg::tl_h2d_t tl_socket_h2d [3];
105 tlul_pkg::tl_d2h_t tl_socket_d2h [3];
106
107 logic [1:0] reg_steer;
108
109 // socket_1n connection
110 assign tl_reg_h2d = tl_socket_h2d[2];
111 assign tl_socket_d2h[2] = tl_reg_d2h;
112
113 assign tl_win_o[0] = tl_socket_h2d[0];
114 assign tl_socket_d2h[0] = tl_win_i[0];
115 assign tl_win_o[1] = tl_socket_h2d[1];
116 assign tl_socket_d2h[1] = tl_win_i[1];
117
118 // Create Socket_1n
119 tlul_socket_1n #(
120   .N (3),
121   .HReqPass (1'b1),
122   .HRspPass (1'b1),
123   .DReqPass ({3{1'b1}}),
124   .DRspPass ({3{1'b1}}),
125   .HReqDepth (4'h0),
126   .HRspDepth (4'h0),
127   .DReqDepth ({3{4'h0}}),
128   .DRspDepth ({3{4'h0}}),
129   .ExplicitErrs (1'b0)
130 ) u_socket (
131   .clk_i (clk_i),
132   .rst_ni (rst_ni),
133   .tl_h_i (tl_i),
134   .tl_h_o (tl_o_pre),
135   .tl_d_o (tl_socket_h2d),
136   .tl_d_i (tl_socket_d2h),
137   .dev_select_i (reg_steer)
138 );
139
140 // Create steering logic
141 always_comb begin
142   reg_steer =
143     tl_i.a.address[AW-1:0] inside {[36:39]} ? 2'd0 :
144     tl_i.a.address[AW-1:0] inside {[40:43]} ? 2'd1 :
145     // Default set to register
146     2'd2;
147
148   // Override this in case of an integrity error
149   if (intg_err) begin
150     reg_steer = 2'd2;
151   end
152 end
153
154 tlul_adapter_reg #(
155   .RegAw(AW),
156   .RegDw(DW),
157   .EnableDataIntgGen(0)
158 ) u_reg_if (
159   .clk_i (clk_i),
160   .rst_ni (rst_ni),
161
162   .tl_i (tl_reg_h2d),
163   .tl_o (tl_reg_d2h),
164
165   .en_ifetch_i(prim_mubi_pkg::MuBi4False),
166   .intg_error_o(),
167
168   .we_o (reg_we),
169   .re_o (reg_re),
170   .addr_o (reg_addr),
171   .wdata_o (reg_wdata),
172   .be_o (reg_be),
173   .busy_i (reg_busy),
174   .rdata_i (reg_rdata),
175   // Translate RACL error to TLUL error if enabled
176   .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
177 );
178
179 // cdc oversampling signals
180
181 assign reg_rdata = reg_rdata_next ;
182 assign reg_error = addrmiss | wr_err | intg_err;
183
184 // Define SW related signals
185 // Format: <reg>_<field>_{wd|we|qs}
186 // or <reg>_{wd|we|qs} if field == 1 or 0
187 logic intr_state_we;
188 logic intr_state_error_qs;
189 logic intr_state_error_wd;
190 logic intr_state_spi_event_qs;
191 logic intr_enable_we;
192 logic intr_enable_error_qs;
193 logic intr_enable_error_wd;
194 logic intr_enable_spi_event_qs;
195 logic intr_enable_spi_event_wd;
196 logic intr_test_we;
197 logic intr_test_error_wd;
198 logic intr_test_spi_event_wd;
199 logic alert_test_we;
200 logic alert_test_wd;
201 logic control_we;
202 logic [7:0] control_rx_watermark_qs;
203 logic [7:0] control_rx_watermark_wd;
204 logic [7:0] control_tx_watermark_qs;
205 logic [7:0] control_tx_watermark_wd;
206 logic control_output_en_qs;
207 logic control_output_en_wd;
208 logic control_sw_rst_qs;
209 logic control_sw_rst_wd;
210 logic control_spieen_qs;
211 logic control_spieen_wd;
212 logic [7:0] status_txqd_qs;

```

```

213 logic [7:0] status_rxqd_qs;
214 logic [3:0] status_cmdqd_qs;
215 logic status_rxwm_qs;
216 logic status_byteorder_qs;
217 logic status_rxstall_qs;
218 logic status_rxempty_qs;
219 logic status_rxfull_qs;
220 logic status_txwm_qs;
221 logic status_txstall_qs;
222 logic status_txempty_qs;
223 logic status_txfull_qs;
224 logic status_active_qs;
225 logic status_ready_qs;
226 logic configopts_we;
227 logic [15:0] configopts_clkdiv_qs;
228 logic [15:0] configopts_clkdiv_wd;
229 logic [3:0] configopts_csnidle_qs;
230 logic [3:0] configopts_csnidle_wd;
231 logic [3:0] configopts_csntrail_qs;
232 logic [3:0] configopts_csntrail_wd;
233 logic [3:0] configopts_csnlead_qs;
234 logic [3:0] configopts_csnlead_wd;
235 logic configopts_fullcyc_qs;
236 logic configopts_fullcyc_wd;
237 logic configopts_cpha_qs;
238 logic configopts_cpha_wd;
239 logic configopts_cpol_qs;
240 logic configopts_cpol_wd;
241 logic csid_we;
242 logic [31:0] csid_qs;
243 logic [31:0] csid_wd;
244 logic command_we;
245 logic command_csaat_wd;
246 logic [1:0] command_speed_wd;
247 logic [1:0] command_direction_wd;
248 logic [19:0] command_len_wd;
249 logic error_enable_we;
250 logic error_enable_cmdbusy_qs;
251 logic error_enable_cmdbusy_wd;
252 logic error_enable_overflow_qs;
253 logic error_enable_overflow_wd;
254 logic error_enable_underflow_qs;
255 logic error_enable_underflow_wd;
256 logic error_enable_cmdinval_qs;
257 logic error_enable_cmdinval_wd;
258 logic error_enable_csidinval_qs;
259 logic error_enable_csidinval_wd;
260 logic error_status_we;
261 logic error_status_cmdbusy_qs;
262 logic error_status_cmdbusy_wd;
263 logic error_status_overflow_qs;
264 logic error_status_overflow_wd;
265 logic error_status_underflow_qs;
266 logic error_status_underflow_wd;
267 logic error_status_cmdinval_qs;
268 logic error_status_cmdinval_wd;
269 logic error_status_csidinval_qs;
270 logic error_status_csidinval_wd;
271 logic error_status_accessinval_qs;
272 logic error_status_accessinval_wd;
273 logic event_enable_we;
274 logic event_enable_rxfull_qs;
275 logic event_enable_rxfull_wd;
276 logic event_enable_txempty_qs;
277 logic event_enable_txempty_wd;
278 logic event_enable_rxwm_qs;
279 logic event_enable_rxwm_wd;
280 logic event_enable_txwm_qs;
281 logic event_enable_txwm_wd;
282 logic event_enable_ready_qs;
283 logic event_enable_ready_wd;
284 logic event_enable_idle_qs;
285 logic event_enable_idle_wd;
286
287 // Register instances
288 // R[intr_state]: V(False)
289 // F[error]: 0:0
290 prim_subreg #(
291   .DW (1),
292   .SwAccess(prim_subreg_pkg::SwAccessW1C),
293   .RESVAL (1'h0),
294   .Mubi (1'b0)
295 ) u_intr_state_error (
296   .clk_i (clk_i),
297   .rst_ni (rst_ni),
298
299   // from register interface
300   .we (intr_state_we),
301   .wd (intr_state_error_wd),
302
303   // from internal hardware
304   .de (hw2reg.intr_state.error.de),
305   .d (hw2reg.intr_state.error.d),
306
307   // to internal hardware
308   .qe (),
309   .q (reg2hw.intr_state.error.q),
310   .ds (),
311
312   // to register interface (read)
313   .qs (intr_state_error_qs)
314 );
315
316 // F[spi_event]: 1:1
317 prim_subreg #(
318   .DW (1),
319   .SwAccess(prim_subreg_pkg::SwAccessRO),
320   .RESVAL (1'h0),
321   .Mubi (1'b0)
322 ) u_intr_state_spi_event (
323   .clk_i (clk_i),
324   .rst_ni (rst_ni),
325
326   // from register interface
327   .we (1'b0),
328   .wd ('0'),
329
330   // from internal hardware
331   .de (hw2reg.intr_state.spi_event.de),
332   .d (hw2reg.intr_state.spi_event.d),
333
334   // to internal hardware
335   .qe (),
336   .q (reg2hw.intr_state.spi_event.q),
337   .ds (),
338
339   // to register interface (read)
340   .qs (intr_state_spi_event_qs)
341 );
342
343 // R[intr_enable]: V(False)
344 // F[error]: 0:0
345 prim_subreg #(
346   .DW (1),
347   .SwAccess(prim_subreg_pkg::SwAccessRW),
348   .RESVAL (1'h0),
349   .Mubi (1'b0)
350 ) u_intr_enable_error (
351   .clk_i (clk_i),
352   .rst_ni (rst_ni),
353
354   // from register interface
355   .we (intr_enable_we),
356   .wd (intr_enable_error_wd),
357
358   // from internal hardware
359   .de (1'b0),
360   .d ('0'),
361
362   // to internal hardware
363   .qe (),
364   .q (reg2hw.intr_enable.error.q),
365   .ds (),
366
367   // to register interface (read)
368   .qs (intr_enable_error_qs)
369 );
370
371 // F[spi_event]: 1:1
372 prim_subreg #(
373   .DW (1),
374   .SwAccess(prim_subreg_pkg::SwAccessRW),
375   .RESVAL (1'h0),
376   .Mubi (1'b0)
377 ) u_intr_enable_spi_event (
378   .clk_i (clk_i),
379   .rst_ni (rst_ni),
380
381   // from register interface
382   .we (intr_enable_we),
383   .wd (intr_enable_spi_event_wd),
384
385   // from internal hardware
386   .de (1'b0),
387   .d ('0'),
388
389   // to internal hardware
390   .qe (),
391   .q (reg2hw.intr_enable.spi_event.q),
392   .ds (),
393
394   // to register interface (read)
395   .qs (intr_enable_spi_event_qs)
396 );
397
398 // R[intr_test]: V(True)
399 logic intr_test_qe;
400 logic [1:0] intr_test_flds_we;
401 assign intr_test_qe = &intr_test_flds_we;
402 // F[error]: 0:0
403 prim_subreg_ext #(
404   .DW (1)
405 ) u_intr_test_error (
406   .re (1'b0),
407   .we (intr_test_we),
408   .wd (intr_test_error_wd),
409   .d ('0'),
410   .gre (),
411   .qe (intr_test_flds_we[0]),
412   .q (reg2hw.intr_test.error.q),
413
414 );

```

```

415 .ds      (),
416 .qs      ()
417 );
418 assign reg2hw.intr_test.error.qe = intr_test_qe;
419
420 // F[spi_event]: 1:1
421 prim_subreg_ext #(
422   .DW      (1)
423 ) u_intr_test_spi_event (
424   .re      (1'b0),
425   .we      (intr_test_we),
426   .wd      (intr_test_spi_event_wd),
427   .d       ('0'),
428   .qre     (),
429   .qe      (intr_test_flds_we[1]),
430   .q       (reg2hw.intr_test.spi_event.q),
431   .ds      (),
432   .qs      ()
433 );
434 assign reg2hw.intr_test.spi_event.qe = intr_test_qe;
435
436 // R[alert_test]: V(True)
437 logic alert_test_qe;
438 logic [0:0] alert_test_flds_we;
439 assign alert_test_qe = &alert_test_flds_we;
440 prim_subreg_ext #(
441   .DW      (1)
442 ) u_alert_test (
443   .re      (1'b0),
444   .we      (alert_test_we),
445   .wd      (alert_test_wd),
446   .d       ('0'),
447   .qre     (),
448   .qe      (alert_test_flds_we[0]),
449   .q       (reg2hw.alert_test.q),
450   .ds      (),
451   .qs      ()
452 );
453 assign reg2hw.alert_test.qe = alert_test_qe;
454
455 // R[control]: V(False)
456 // F[rx_watermark]: 7:0
457 prim_subreg #(
458   .DW      (8),
459   .SwAccess(prim_subreg_pkg::SwAccessRW),
460   .RESVAL  (8'h7f),
461   .Mubi     (1'b0)
462 ) u_control_rx_watermark (
463   .clk_i    (clk_i),
464   .rst_ni   (rst_ni),
465
466   // from register interface
467   .we      (control_we),
468   .wd      (control_rx_watermark_wd),
469
470   // from internal hardware
471   .de      (1'b0),
472   .d       ('0'),
473
474   // to internal hardware
475   .qe      (),
476   .q       (reg2hw.control.rx_watermark.q),
477   .ds      (),
478
479   // to register interface (read)
480   .qs      (control_rx_watermark_qs)
481 );
482
483 // F[tx_watermark]: 15:8
484 prim_subreg #(
485   .DW      (8),
486   .SwAccess(prim_subreg_pkg::SwAccessRW),
487   .RESVAL  (8'h0),
488   .Mubi     (1'b0)
489 ) u_control_tx_watermark (
490   .clk_i    (clk_i),
491   .rst_ni   (rst_ni),
492
493   // from register interface
494   .we      (control_we),
495   .wd      (control_tx_watermark_wd),
496
497   // from internal hardware
498   .de      (1'b0),
499   .d       ('0'),
500
501   // to internal hardware
502   .qe      (),
503   .q       (reg2hw.control.tx_watermark.q),
504   .ds      (),
505
506   // to register interface (read)
507   .qs      (control_tx_watermark_qs)
508 );
509
510 // F[output_en]: 29:29
511 prim_subreg #(
512   .DW      (1),
513   .SwAccess(prim_subreg_pkg::SwAccessRW),

```

```

516 .RESVAL  (1'h0),
517 .Mubi     (1'b0)
518 ) u_control_output_en (
519   .clk_i   (clk_i),
520   .rst_ni  (rst_ni),
521
522   // from register interface
523   .we      (control_we),
524   .wd      (control_output_en_wd),
525
526   // from internal hardware
527   .de      (1'b0),
528   .d       ('0'),
529
530   // to internal hardware
531   .qe      (),
532   .q       (reg2hw.control.output_en.q),
533   .ds      (),
534
535   // to register interface (read)
536   .qs      (control_output_en_qs)
537 );
538
539 // F[sw_rst]: 30:30
540 prim_subreg #(
541   .DW      (1),
542   .SwAccess(prim_subreg_pkg::SwAccessRW),
543   .RESVAL  (1'h0),
544   .Mubi     (1'b0)
545 ) u_control_sw_rst (
546   .clk_i    (clk_i),
547   .rst_ni   (rst_ni),
548
549   // from register interface
550   .we      (control_we),
551   .wd      (control_sw_rst_wd),
552
553   // from internal hardware
554   .de      (1'b0),
555   .d       ('0'),
556
557   // to internal hardware
558   .qe      (),
559   .q       (reg2hw.control.sw_rst.q),
560   .ds      (),
561
562   // to register interface (read)
563   .qs      (control_sw_rst_qs)
564 );
565
566 // F[spien]: 31:31
567 prim_subreg #(
568   .DW      (1),
569   .SwAccess(prim_subreg_pkg::SwAccessRW),
570   .RESVAL  (1'h0),
571   .Mubi     (1'b0)
572 ) u_control_spie (
573   .clk_i    (clk_i),
574   .rst_ni   (rst_ni),
575
576   // from register interface
577   .we      (control_we),
578   .wd      (control_spie_wd),
579
580   // from internal hardware
581   .de      (1'b0),
582   .d       ('0'),
583
584   // to internal hardware
585   .qe      (),
586   .q       (reg2hw.control.spie.q),
587   .ds      (),
588
589   // to register interface (read)
590   .qs      (control_spie_qs)
591 );
592
593 // R[status]: V(False)
594 // F[txqd]: 7:0
595 prim_subreg #(
596   .DW      (8),
597   .SwAccess(prim_subreg_pkg::SwAccessRO),
598   .RESVAL  (8'h0),
599   .Mubi     (1'b0)
600 ) u_status_txqd (
601   .clk_i    (clk_i),
602   .rst_ni   (rst_ni),
603
604   // from register interface
605   .we      (1'b0),
606   .wd      ('0'),
607
608   // from internal hardware
609   .de      (hw2reg.status.txqd.de),
610   .d       (hw2reg.status.txqd.d),
611
612   // to internal hardware
613   .qe      (),
614   .q       (),
615   .ds      (),
616

```

```

617
618 // to register interface (read)
619 .qs (status_txqd_qs)
620 );
621
622 // F[rxqd]: 15:8
623 prim_subreg #(
624     .DW (8),
625     .SwAccess(prim_subreg_pkg::SwAccessR0),
626     .RESVAL (8'h0),
627     .Mubi (1'b0)
628 ) u_status_rxqd (
629     .clk_i (clk_i),
630     .rst_ni (rst_ni),
631
632     // from register interface
633     .we (1'b0),
634     .wd ('0'),
635
636     // from internal hardware
637     .de (hw2reg.status.rxqd.de),
638     .d (hw2reg.status.rxqd.d),
639
640     // to internal hardware
641     .qe (),
642     .q (),
643     .ds (),
644
645     // to register interface (read)
646     .qs (status_rxqd_qs)
647 );
648
649 // F[cmdqd]: 19:16
650 prim_subreg #(
651     .DW (4),
652     .SwAccess(prim_subreg_pkg::SwAccessR0),
653     .RESVAL (4'h0),
654     .Mubi (1'b0)
655 ) u_status_cmdqd (
656     .clk_i (clk_i),
657     .rst_ni (rst_ni),
658
659     // from register interface
660     .we (1'b0),
661     .wd ('0'),
662
663     // from internal hardware
664     .de (hw2reg.status.cmdqd.de),
665     .d (hw2reg.status.cmdqd.d),
666
667     // to internal hardware
668     .qe (),
669     .q (),
670     .ds (),
671
672     // to register interface (read)
673     .qs (status_cmdqd_qs)
674 );
675
676 // F[rxwm]: 20:20
677 prim_subreg #(
678     .DW (1),
679     .SwAccess(prim_subreg_pkg::SwAccessR0),
680     .RESVAL (1'h0),
681     .Mubi (1'b0)
682 ) u_status_rxwm (
683     .clk_i (clk_i),
684     .rst_ni (rst_ni),
685
686     // from register interface
687     .we (1'b0),
688     .wd ('0'),
689
690     // from internal hardware
691     .de (hw2reg.status.rxwm.de),
692     .d (hw2reg.status.rxwm.d),
693
694     // to internal hardware
695     .qe (),
696     .q (),
697     .ds (),
698
699     // to register interface (read)
700     .qs (status_rxwm_qs)
701 );
702
703 // F[byteorder]: 22:22
704 prim_subreg #(
705     .DW (1),
706     .SwAccess(prim_subreg_pkg::SwAccessR0),
707     .RESVAL (1'h0),
708     .Mubi (1'b0)
709 ) u_status_byteorder (
710     .clk_i (clk_i),
711     .rst_ni (rst_ni),
712
713     // from register interface
714     .we (1'b0),
715     .wd ('0'),
716
717     // from internal hardware

```

```

718     .de (hw2reg.status.byteorder.de),
719     .d (hw2reg.status.byteorder.d),
720
721     // to internal hardware
722     .qe (),
723     .q (),
724     .ds (),
725
726     // to register interface (read)
727     .qs (status_byteorder_qs)
728 );
729
730 // F[rxstall]: 23:23
731 prim_subreg #(
732     .DW (1),
733     .SwAccess(prim_subreg_pkg::SwAccessR0),
734     .RESVAL (1'h0),
735     .Mubi (1'b0)
736 ) u_status_rxstall (
737     .clk_i (clk_i),
738     .rst_ni (rst_ni),
739
740     // from register interface
741     .we (1'b0),
742     .wd ('0'),
743
744     // from internal hardware
745     .de (hw2reg.status.rxstall.de),
746     .d (hw2reg.status.rxstall.d),
747
748     // to internal hardware
749     .qe (),
750     .q (),
751     .ds (),
752
753     // to register interface (read)
754     .qs (status_rxstall_qs)
755 );
756
757 // F[rxempty]: 24:24
758 prim_subreg #(
759     .DW (1),
760     .SwAccess(prim_subreg_pkg::SwAccessR0),
761     .RESVAL (1'h0),
762     .Mubi (1'b0)
763 ) u_status_rxempty (
764     .clk_i (clk_i),
765     .rst_ni (rst_ni),
766
767     // from register interface
768     .we (1'b0),
769     .wd ('0'),
770
771     // from internal hardware
772     .de (hw2reg.status.rxempty.de),
773     .d (hw2reg.status.rxempty.d),
774
775     // to internal hardware
776     .qe (),
777     .q (),
778     .ds (),
779
780     // to register interface (read)
781     .qs (status_rxempty_qs)
782 );
783
784 // F[rxfull]: 25:25
785 prim_subreg #(
786     .DW (1),
787     .SwAccess(prim_subreg_pkg::SwAccessR0),
788     .RESVAL (1'h0),
789     .Mubi (1'b0)
790 ) u_status_rxfull (
791     .clk_i (clk_i),
792     .rst_ni (rst_ni),
793
794     // from register interface
795     .we (1'b0),
796     .wd ('0'),
797
798     // from internal hardware
799     .de (hw2reg.status.rxfull.de),
800     .d (hw2reg.status.rxfull.d),
801
802     // to internal hardware
803     .qe (),
804     .q (),
805     .ds (),
806
807     // to register interface (read)
808     .qs (status_rxfull_qs)
809 );
810
811 // F[txwm]: 26:26
812 prim_subreg #(
813     .DW (1),
814     .SwAccess(prim_subreg_pkg::SwAccessR0),
815     .RESVAL (1'h0),
816     .Mubi (1'b0)
817 ) u_status_txwm (
818     .clk_i (clk_i),

```



```

819 .rst_ni (rst_ni),
820
821 // from register interface
822 .we (1'b0),
823 .wd ('0'),
824
825 // from internal hardware
826 .de (hw2reg.status.txwm.de),
827 .d (hw2reg.status.txwm.d),
828
829 // to internal hardware
830 .qe (),
831 .q (),
832 .ds (),
833
834 // to register interface (read)
835 .qs (status_txwm_qs)
836 );
837
838 // F[txstall]: 27:27
839 prim_subreg #(
840 .DW (1),
841 .SwAccess(prim_subreg_pkg::SwAccessR0),
842 .RESVAL (1'h0),
843 .Mubi (1'b0)
844 ) u_status_txstall (
845 .clk_i (clk_i),
846 .rst_ni (rst_ni),
847
848 // from register interface
849 .we (1'b0),
850 .wd ('0'),
851
852 // from internal hardware
853 .de (hw2reg.status.txstall.de),
854 .d (hw2reg.status.txstall.d),
855
856 // to internal hardware
857 .qe (),
858 .q (),
859 .ds (),
860
861 // to register interface (read)
862 .qs (status_txstall_qs)
863 );
864
865 // F[txempty]: 28:28
866 prim_subreg #(
867 .DW (1),
868 .SwAccess(prim_subreg_pkg::SwAccessR0),
869 .RESVAL (1'h0),
870 .Mubi (1'b0)
871 ) u_status_txempty (
872 .clk_i (clk_i),
873 .rst_ni (rst_ni),
874
875 // from register interface
876 .we (1'b0),
877 .wd ('0'),
878
879 // from internal hardware
880 .de (hw2reg.status.txempty.de),
881 .d (hw2reg.status.txempty.d),
882
883 // to internal hardware
884 .qe (),
885 .q (),
886 .ds (),
887
888 // to register interface (read)
889 .qs (status_txempty_qs)
890 );
891
892 // F[txfull]: 29:29
893 prim_subreg #(
894 .DW (1),
895 .SwAccess(prim_subreg_pkg::SwAccessR0),
896 .RESVAL (1'h0),
897 .Mubi (1'b0)
898 ) u_status_txfull (
899 .clk_i (clk_i),
900 .rst_ni (rst_ni),
901
902 // from register interface
903 .we (1'b0),
904 .wd ('0'),
905
906 // from internal hardware
907 .de (hw2reg.status.txfull.de),
908 .d (hw2reg.status.txfull.d),
909
910 // to internal hardware
911 .qe (),
912 .q (),
913 .ds (),
914
915 // to register interface (read)
916 .qs (status_txfull_qs)
917 );
918
919 // F[active]: 30:30

```

```

920 prim_subreg #(
921 .DW (1),
922 .SwAccess(prim_subreg_pkg::SwAccessR0),
923 .RESVAL (1'h0),
924 .Mubi (1'b0)
925 ) u_status_active (
926 .clk_i (clk_i),
927 .rst_ni (rst_ni),
928
929 // from register interface
930 .we (1'b0),
931 .wd ('0'),
932
933 // from internal hardware
934 .de (hw2reg.status.active.de),
935 .d (hw2reg.status.active.d),
936
937 // to internal hardware
938 .qe (),
939 .q (),
940 .ds (),
941
942 // to register interface (read)
943 .qs (status_active_qs)
944 );
945
946 // F[ready]: 31:31
947 prim_subreg #(
948 .DW (1),
949 .SwAccess(prim_subreg_pkg::SwAccessR0),
950 .RESVAL (1'h0),
951 .Mubi (1'b0)
952 ) u_status_ready (
953 .clk_i (clk_i),
954 .rst_ni (rst_ni),
955
956 // from register interface
957 .we (1'b0),
958 .wd ('0'),
959
960 // from internal hardware
961 .de (hw2reg.status.ready.de),
962 .d (hw2reg.status.ready.d),
963
964 // to internal hardware
965 .qe (),
966 .q (),
967 .ds (),
968
969 // to register interface (read)
970 .qs (status_ready_qs)
971 );
972
973 // R[configopts]: V(False)
974 // F[clkdiv]: 15:0
975 prim_subreg #(
976 .DW (16),
977 .SwAccess(prim_subreg_pkg::SwAccessRW),
978 .RESVAL (16'h0),
979 .Mubi (1'b0)
980 ) u_configopts_clkdiv (
981 .clk_i (clk_i),
982 .rst_ni (rst_ni),
983
984 // from register interface
985 .we (configopts_we),
986 .wd (configopts_clkdiv_wd),
987
988 // from internal hardware
989 .de (1'b0),
990 .d ('0'),
991
992 // to internal hardware
993 .qe (),
994 .q (reg2hw.configopts.clkdiv.q),
995 .ds (),
996
997 // to register interface (read)
998 .qs (configopts_clkdiv_qs)
999 );
1000
1001 // F[csnidle]: 19:16
1002 prim_subreg #(
1003 .DW (4),
1004 .SwAccess(prim_subreg_pkg::SwAccessRW),
1005 .RESVAL (4'h0),
1006 .Mubi (1'b0)
1007 ) u_configopts_csnidle (
1008 .clk_i (clk_i),
1009 .rst_ni (rst_ni),
1010
1011 // from register interface
1012 .we (configopts_we),
1013 .wd (configopts_csnidle_wd),
1014
1015 // from internal hardware
1016 .de (1'b0),
1017 .d ('0'),
1018
1019 // to internal hardware
1020

```



```

1021 .qe      ( ),
1022 .q      (reg2hw.configopts.csntail.q),
1023 .ds      ( ),
1024
1025 // to register interface (read)
1026 .qs      (configopts_csntail_qs)
1027 );
1028
1029 // F[csntail]: 23:20
1030 prim_subreg #(
1031     .DW      (4),
1032     .SwAccess(prim_subreg_pkg::SwAccessRW),
1033     .RESVAL   (4'h0),
1034     .Mubi     (1'b0)
1035 ) u_configopts_csntail (
1036     .clk_i    (clk_i),
1037     .rst_ni   (rst_ni),
1038
1039     // from register interface
1040     .we        (configopts_we),
1041     .wd        (configopts_csntail_wd),
1042
1043     // from internal hardware
1044     .de        (1'b0),
1045     .d         ('0'),
1046
1047     // to internal hardware
1048     .qe        ( ),
1049     .q         (reg2hw.configopts.csntail.q),
1050     .ds        ( ),
1051
1052     // to register interface (read)
1053     .qs        (configopts_csntail_qs)
1054 );
1055
1056 // F[csnlead]: 27:24
1057 prim_subreg #(
1058     .DW      (4),
1059     .SwAccess(prim_subreg_pkg::SwAccessRW),
1060     .RESVAL   (4'h0),
1061     .Mubi     (1'b0)
1062 ) u_configopts_csnlead (
1063     .clk_i    (clk_i),
1064     .rst_ni   (rst_ni),
1065
1066     // from register interface
1067     .we        (configopts_we),
1068     .wd        (configopts_csnlead_wd),
1069
1070     // from internal hardware
1071     .de        (1'b0),
1072     .d         ('0'),
1073
1074     // to internal hardware
1075     .qe        ( ),
1076     .q         (reg2hw.configopts.csnlead.q),
1077     .ds        ( ),
1078
1079     // to register interface (read)
1080     .qs        (configopts_csnlead_qs)
1081 );
1082
1083 // F[fullcyc]: 29:29
1084 prim_subreg #(
1085     .DW      (1),
1086     .SwAccess(prim_subreg_pkg::SwAccessRW),
1087     .RESVAL   (1'h0),
1088     .Mubi     (1'b0)
1089 ) u_configopts_fullcyc (
1090     .clk_i    (clk_i),
1091     .rst_ni   (rst_ni),
1092
1093     // from register interface
1094     .we        (configopts_we),
1095     .wd        (configopts_fullcyc_wd),
1096
1097     // from internal hardware
1098     .de        (1'b0),
1099     .d         ('0'),
1100
1101     // to internal hardware
1102     .qe        ( ),
1103     .q         (reg2hw.configopts.fullcyc.q),
1104     .ds        ( ),
1105
1106     // to register interface (read)
1107     .qs        (configopts_fullcyc_qs)
1108 );
1109
1110 // F[cpha]: 30:30
1111 prim_subreg #(
1112     .DW      (1),
1113     .SwAccess(prim_subreg_pkg::SwAccessRW),
1114     .RESVAL   (1'h0),
1115     .Mubi     (1'b0)
1116 ) u_configopts_cpha (
1117     .clk_i    (clk_i),
1118     .rst_ni   (rst_ni),
1119
1120     // from register interface
1121     .we        (configopts_we),
1122
1123     .wd        (configopts_cpha_wd),
1124
1125     // from internal hardware
1126     .de        (1'b0),
1127     .d         ('0'),
1128
1129     // to internal hardware
1130     .qe        ( ),
1131     .q         (reg2hw.configopts.cpha.q),
1132     .ds        ( ),
1133
1134     // to register interface (read)
1135     .qs        (configopts_cpha_qs)
1136 );
1137
1138 // F[cpol]: 31:31
1139 prim_subreg #(
1140     .DW      (1),
1141     .SwAccess(prim_subreg_pkg::SwAccessRW),
1142     .RESVAL   (1'h0),
1143     .Mubi     (1'b0)
1144 ) u_configopts_cpol (
1145     .clk_i    (clk_i),
1146     .rst_ni   (rst_ni),
1147
1148     // from register interface
1149     .we        (configopts_we),
1150     .wd        (configopts_cpol_wd),
1151
1152     // from internal hardware
1153     .de        (1'b0),
1154     .d         ('0'),
1155
1156     // to internal hardware
1157     .qe        ( ),
1158     .q         (reg2hw.configopts.cpol.q),
1159     .ds        ( ),
1160
1161     // to register interface (read)
1162     .qs        (configopts_cpol_qs)
1163 );
1164
1165 // R[csid]: V(False)
1166 prim_subreg #(
1167     .DW      (32),
1168     .SwAccess(prim_subreg_pkg::SwAccessRW),
1169     .RESVAL   (32'h0),
1170     .Mubi     (1'b0)
1171 ) u_csids (
1172     .clk_i    (clk_i),
1173     .rst_ni   (rst_ni),
1174
1175     // from register interface
1176     .we        (csid_we),
1177     .wd        (csid_wd),
1178
1179     // from internal hardware
1180     .de        (1'b0),
1181     .d         ('0'),
1182
1183     // to internal hardware
1184     .qe        ( ),
1185     .q         (reg2hw.csid.q),
1186     .ds        ( ),
1187
1188     // to register interface (read)
1189     .qs        (csid_qs)
1190 );
1191
1192 // R[command]: V(True)
1193 logic command_qe;
1194 logic [3:0] command_flds_we;
1195 assign command_qe = &command_flds_we;
1196 // F[csaat]: 0:0
1197 prim_subreg_ext #(
1198     .DW      (1)
1199 ) u_command_csaat (
1200     .re       (1'b0),
1201     .we        (command_we),
1202     .wd        (command_csaat_wd),
1203     .d         ('0'),
1204     .qre       ( ),
1205     .qe        (command_flds_we[0]),
1206     .q         (reg2hw.command.csaat.q),
1207     .ds        ( ),
1208     .qs        ( ),
1209
1210 );
1211 assign reg2hw.command.csaat.qe = command_qe;
1212
1213 // F[speed]: 2:1
1214 prim_subreg_ext #(
1215     .DW      (2)
1216 ) u_command_speed (
1217     .re       (1'b0),
1218     .we        (command_we),
1219     .wd        (command_speed_wd),
1220     .d         ('0'),
1221     .qre       ( ),
1222     .qe        (command_flds_we[1]),

```

```

1223 .q      (reg2hw.command.speed.q),
1224 .ds    (),
1225 .qs    ()
1226 );
1227 assign reg2hw.command.speed.qe = command_qe;
1228
1229 // F[direction]: 4:3
1230 prim_subreg_ext #(
1231     .DW      (2)
1232 ) u_command_direction (
1233     .re      (1'b0),
1234     .we      (command_we),
1235     .wd      (command_direction_wd),
1236     .d       ('0'),
1237     .qre     (),
1238     .qe      (command_flds_we[2]),
1239     .q       (reg2hw.command.direction.q),
1240     .ds      (),
1241     .qs      ()
1242 );
1243 assign reg2hw.command.direction.qe = command_qe;
1244
1245 // F[len]: 24:5
1246 prim_subreg_ext #(
1247     .DW      (20)
1248 ) u_command_len (
1249     .re      (1'b0),
1250     .we      (command_we),
1251     .wd      (command_len_wd),
1252     .d       ('0'),
1253     .qre     (),
1254     .qe      (command_flds_we[3]),
1255     .q       (reg2hw.command.len.q),
1256     .ds      (),
1257     .qs      ()
1258 );
1259 assign reg2hw.command.len.qe = command_qe;
1260
1261 // R[error_enable]: V(False)
1262 // F[cmdbusy]: 0:0
1263 prim_subreg #(
1264     .DW      (1),
1265     .SwAccess(prim_subreg_pkg::SwAccessRW),
1266     .RESVAL  (1'h1),
1267     .Mubi    (1'b0)
1268 ) u_error_enable_cmdbusy (
1269     .clk_i   (clk_i),
1270     .rst_ni  (rst_ni),
1271
1272     // from register interface
1273     .we      (error_enable_we),
1274     .wd      (error_enable_cmdbusy_wd),
1275
1276     // from internal hardware
1277     .de      (1'b0),
1278     .d       ('0'),
1279
1280     // to internal hardware
1281     .qe      (),
1282     .q       (reg2hw.error_enable.cmdbusy.q),
1283     .ds      (),
1284
1285     // to register interface (read)
1286     .qs      (error_enable_cmdbusy_qs)
1287 );
1288
1289 // F[overflow]: 1:1
1290 prim_subreg #(
1291     .DW      (1),
1292     .SwAccess(prim_subreg_pkg::SwAccessRW),
1293     .RESVAL  (1'h1),
1294     .Mubi    (1'b0)
1295 ) u_error_enable_overflow (
1296     .clk_i   (clk_i),
1297     .rst_ni  (rst_ni),
1298
1299     // from register interface
1300     .we      (error_enable_we),
1301     .wd      (error_enable_overflow_wd),
1302
1303     // from internal hardware
1304     .de      (1'b0),
1305     .d       ('0'),
1306
1307     // to internal hardware
1308     .qe      (),
1309     .q       (reg2hw.error_enable.overflow.q),
1310     .ds      (),
1311
1312     // to register interface (read)
1313     .qs      (error_enable_overflow_qs)
1314 );
1315
1316 // F[underflow]: 2:2
1317 prim_subreg #(
1318     .DW      (1),
1319     .SwAccess(prim_subreg_pkg::SwAccessRW),
1320     .RESVAL  (1'h1),
1321     .Mubi    (1'b0)
1322 ) u_error_enable_underflow (
1323
1324     .clk_i   (clk_i),
1325     .rst_ni  (rst_ni),
1326
1327     // from register interface
1328     .we      (error_enable_we),
1329     .wd      (error_enable_underflow_wd),
1330
1331     // from internal hardware
1332     .de      (1'b0),
1333     .d       ('0'),
1334
1335     // to internal hardware
1336     .qe      (),
1337     .q       (reg2hw.error_enable.underflow.q),
1338     .ds      (),
1339
1340     // to register interface (read)
1341     .qs      (error_enable_underflow_qs)
1342 );
1343
1344 // F[cmdinval]: 3:3
1345 prim_subreg #(
1346     .DW      (1),
1347     .SwAccess(prim_subreg_pkg::SwAccessRW),
1348     .RESVAL  (1'h1),
1349     .Mubi    (1'b0)
1350 ) u_error_enable_cmdinval (
1351     .clk_i   (clk_i),
1352     .rst_ni  (rst_ni),
1353
1354     // from register interface
1355     .we      (error_enable_we),
1356     .wd      (error_enable_cmdinval_wd),
1357
1358     // from internal hardware
1359     .de      (1'b0),
1360     .d       ('0'),
1361
1362     // to internal hardware
1363     .qe      (),
1364     .q       (reg2hw.error_enable.cmdinval.q),
1365     .ds      (),
1366
1367     // to register interface (read)
1368     .qs      (error_enable_cmdinval_qs)
1369 );
1370
1371 // F[csdinval]: 4:4
1372 prim_subreg #(
1373     .DW      (1),
1374     .SwAccess(prim_subreg_pkg::SwAccessRW),
1375     .RESVAL  (1'h1),
1376     .Mubi    (1'b0)
1377 ) u_error_enable_csdinval (
1378     .clk_i   (clk_i),
1379     .rst_ni  (rst_ni),
1380
1381     // from register interface
1382     .we      (error_enable_we),
1383     .wd      (error_enable_csdinval_wd),
1384
1385     // from internal hardware
1386     .de      (1'b0),
1387     .d       ('0'),
1388
1389     // to internal hardware
1390     .qe      (),
1391     .q       (reg2hw.error_enable.csdinval.q),
1392     .ds      (),
1393
1394     // to register interface (read)
1395     .qs      (error_enable_csdinval_qs)
1396 );
1397
1398 // R[error_status]: V(False)
1399 // F[cmdbusy]: 0:0
1400 prim_subreg #(
1401     .DW      (1),
1402     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1403     .RESVAL  (1'h0),
1404     .Mubi    (1'b0)
1405 ) u_error_status_cmdbusy (
1406     .clk_i   (clk_i),
1407     .rst_ni  (rst_ni),
1408
1409     // from register interface
1410     .we      (error_status_we),
1411     .wd      (error_status_cmdbusy_wd),
1412
1413     // from internal hardware
1414     .de      (hw2reg.error_status.cmdbusy.de),
1415     .d       (hw2reg.error_status.cmdbusy.d),
1416
1417     // to internal hardware
1418     .qe      (),
1419     .q       (reg2hw.error_status.cmdbusy.q),
1420     .ds      (),
1421
1422     // to register interface (read)
1423     .qs      (error_status_cmdbusy_qs)
1424 );

```

```

1425 );
1426 // F[overflow]: 1:1
1427 prim_subreg #(
1428     .DW (1),
1429     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1430     .RESVAL (1'h0),
1431     .Mubi (1'b0)
1432 ) u_error_status_overflow (
1433     .clk_i (clk_i),
1434     .rst_ni (rst_ni),
1435
1436     // from register interface
1437     .we (error_status_we),
1438     .wd (error_status_overflow_wd),
1439
1440     // from internal hardware
1441     .de (hw2reg.error_status.overflow.de),
1442     .d (hw2reg.error_status.overflow.d),
1443
1444     // to internal hardware
1445     .qe (),
1446     .q (reg2hw.error_status.overflow.q),
1447     .ds (),
1448
1449     // to register interface (read)
1450     .qs (error_status_overflow_qs)
1451 );
1452
1453 // F[underflow]: 2:2
1454 prim_subreg #(
1455     .DW (1),
1456     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1457     .RESVAL (1'h0),
1458     .Mubi (1'b0)
1459 ) u_error_status_underflow (
1460     .clk_i (clk_i),
1461     .rst_ni (rst_ni),
1462
1463     // from register interface
1464     .we (error_status_we),
1465     .wd (error_status_underflow_wd),
1466
1467     // from internal hardware
1468     .de (hw2reg.error_status.underflow.de),
1469     .d (hw2reg.error_status.underflow.d),
1470
1471     // to internal hardware
1472     .qe (),
1473     .q (reg2hw.error_status.underflow.q),
1474     .ds (),
1475
1476     // to register interface (read)
1477     .qs (error_status_underflow_qs)
1478 );
1479
1480 // F[cmdinval]: 3:3
1481 prim_subreg #(
1482     .DW (1),
1483     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1484     .RESVAL (1'h0),
1485     .Mubi (1'b0)
1486 ) u_error_status_cmdinval (
1487     .clk_i (clk_i),
1488     .rst_ni (rst_ni),
1489
1490     // from register interface
1491     .we (error_status_we),
1492     .wd (error_status_cmdinval_wd),
1493
1494     // from internal hardware
1495     .de (hw2reg.error_status.cmdinval.de),
1496     .d (hw2reg.error_status.cmdinval.d),
1497
1498     // to internal hardware
1499     .qe (),
1500     .q (reg2hw.error_status.cmdinval.q),
1501     .ds (),
1502
1503     // to register interface (read)
1504     .qs (error_status_cmdinval_qs)
1505 );
1506
1507 // F[csidinval]: 4:4
1508 prim_subreg #(
1509     .DW (1),
1510     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1511     .RESVAL (1'h0),
1512     .Mubi (1'b0)
1513 ) u_error_status_csidinval (
1514     .clk_i (clk_i),
1515     .rst_ni (rst_ni),
1516
1517     // from register interface
1518     .we (error_status_we),
1519     .wd (error_status_csidinval_wd),
1520
1521     // from internal hardware
1522     .de (hw2reg.error_status.csidinval.de),
1523     .d (hw2reg.error_status.csidinval.d),
1524
1525     // to internal hardware
1526     .qe (),
1527     .q (reg2hw.error_status.csidinval.q),
1528     .ds (),
1529
1530     // to register interface (read)
1531     .qs (error_status_csidinval_qs)
1532 );
1533
1534 // F[accessinval]: 5:5
1535 prim_subreg #(
1536     .DW (1),
1537     .SwAccess(prim_subreg_pkg::SwAccessW1C),
1538     .RESVAL (1'h0),
1539     .Mubi (1'b0)
1540 ) u_error_status_accessinval (
1541     .clk_i (clk_i),
1542     .rst_ni (rst_ni),
1543
1544     // from register interface
1545     .we (error_status_we),
1546     .wd (error_status_accessinval_wd),
1547
1548     // from internal hardware
1549     .de (hw2reg.error_status.accessinval.de),
1550     .d (hw2reg.error_status.accessinval.d),
1551
1552     // to internal hardware
1553     .qe (),
1554     .q (reg2hw.error_status.accessinval.q),
1555     .ds (),
1556
1557     // to register interface (read)
1558     .qs (error_status_accessinval_qs)
1559 );
1560
1561 // R[event_enable]: V(False)
1562 // F[rxfull]: 0:0
1563 prim_subreg #(
1564     .DW (1),
1565     .SwAccess(prim_subreg_pkg::SwAccessRW),
1566     .RESVAL (1'h0),
1567     .Mubi (1'b0)
1568 ) u_event_enable_rxfull (
1569     .clk_i (clk_i),
1570     .rst_ni (rst_ni),
1571
1572     // from register interface
1573     .we (event_enable_we),
1574     .wd (event_enable_rxfull_wd),
1575
1576     // from internal hardware
1577     .de (1'b0),
1578     .d ('0'),
1579
1580     // to internal hardware
1581     .qe (),
1582     .q (reg2hw.event_enable.rxfull.q),
1583     .ds (),
1584
1585     // to register interface (read)
1586     .qs (event_enable_rxfull_qs)
1587 );
1588
1589 // F[tEMPTY]: 1:1
1590 prim_subreg #(
1591     .DW (1),
1592     .SwAccess(prim_subreg_pkg::SwAccessRW),
1593     .RESVAL (1'h0),
1594     .Mubi (1'b0)
1595 ) u_event_enable_txempty (
1596     .clk_i (clk_i),
1597     .rst_ni (rst_ni),
1598
1599     // from register interface
1600     .we (event_enable_we),
1601     .wd (event_enable_txempty_wd),
1602
1603     // from internal hardware
1604     .de (1'b0),
1605     .d ('0'),
1606
1607     // to internal hardware
1608     .qe (),
1609     .q (reg2hw.event_enable.txempty.q),
1610     .ds (),
1611
1612     // to register interface (read)
1613     .qs (event_enable_txempty_qs)
1614 );
1615
1616 // F[rxwm]: 2:2
1617 prim_subreg #(
1618     .DW (1),
1619     .SwAccess(prim_subreg_pkg::SwAccessRW),
1620     .RESVAL (1'h0),
1621     .Mubi (1'b0)
1622 ) u_event_enable_rxwm (
1623     .clk_i (clk_i),
1624     .rst_ni (rst_ni),

```

```

1627
1628 // from register interface
1629 .we (event_enable_we),
1630 .wd (event_enable_rxwm_wd),
1631
1632 // from internal hardware
1633 .de (1'b0),
1634 .d ('0'),
1635
1636 // to internal hardware
1637 .qe (),
1638 .q (reg2hw.event_enable.rxwm.q),
1639 .ds (),
1640
1641 // to register interface (read)
1642 .qs (event_enable_rxwm_qs)
1643 );
1644
1645 // F[txwm]: 3:3
1646 prim_subreg #(
1647 .DW (1),
1648 .SwAccess(prim_subreg_pkg::SwAccessRW),
1649 .RESVAL (1'h0),
1650 .Mubi (1'b0)
1651 ) u_event_enable_txwm (
1652 .clk_i (clk_i),
1653 .rst_ni (rst_ni),
1654
1655 // from register interface
1656 .we (event_enable_we),
1657 .wd (event_enable_txwm_wd),
1658
1659 // from internal hardware
1660 .de (1'b0),
1661 .d ('0'),
1662
1663 // to internal hardware
1664 .qe (),
1665 .q (reg2hw.event_enable.txwm.q),
1666 .ds (),
1667
1668 // to register interface (read)
1669 .qs (event_enable_txwm_qs)
1670 );
1671
1672 // F[ready]: 4:4
1673 prim_subreg #(
1674 .DW (1),
1675 .SwAccess(prim_subreg_pkg::SwAccessRW),
1676 .RESVAL (1'h0),
1677 .Mubi (1'b0)
1678 ) u_event_enable_ready (
1679 .clk_i (clk_i),
1680 .rst_ni (rst_ni),
1681
1682 // from register interface
1683 .we (event_enable_we),
1684 .wd (event_enable_ready_wd),
1685
1686 // from internal hardware
1687 .de (1'b0),
1688 .d ('0'),
1689
1690 // to internal hardware
1691 .qe (),
1692 .q (reg2hw.event_enable.ready.q),
1693 .ds (),
1694
1695 // to register interface (read)
1696 .qs (event_enable_ready_qs)
1697 );
1698
1699 // F[idle]: 5:5
1700 prim_subreg #(
1701 .DW (1),
1702 .SwAccess(prim_subreg_pkg::SwAccessRW),
1703 .RESVAL (1'h0),
1704 .Mubi (1'b0)
1705 ) u_event_enable_idle (
1706 .clk_i (clk_i),
1707 .rst_ni (rst_ni),
1708
1709 // from register interface
1710 .we (event_enable_we),
1711 .wd (event_enable_idle_wd),
1712
1713 // from internal hardware
1714 .de (1'b0),
1715 .d ('0'),
1716
1717 // to internal hardware
1718 .qe (),
1719 .q (reg2hw.event_enable.idle.q),
1720 .ds (),
1721
1722 // to register interface (read)
1723 .qs (event_enable_idle_qs)
1724 );
1725
1726
1727

```

```

1728 logic [11:0] addr_hit;
1729 top_racl_pkg::racl_role_vec_t racl_role_vec;
1730 top_racl_pkg::racl_role_t racl_role;
1731
1732 logic [11:0] racl_addr_hit_read;
1733 logic [11:0] racl_addr_hit_write;
1734
1735 if (EnableRacl) begin : gen_racl_role_logic
1736 // Retrieve RACL role from user bits and one-hot encode
that for the comparison bitmap
1737 assign racl_role =
top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a.user.rsvd);
1738
1739 prim_onehot_enc #(
1740 .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
1741 ) u_racl_role_encode (
1742 .in_i ( racl_role ),
1743 .en_i ( 1'b1 ),
1744 .out_o( racl_role_vec )
1745 );
1746 end else begin : gen_no_racl_role_logic
1747 assign racl_role = '0;
1748 assign racl_role_vec = '0;
1749 end
1750
1751 always_comb begin
1752 racl_addr_hit_read = '0;
1753 racl_addr_hit_write = '0;
1754 addr_hit[ 0] = (reg_addr == SPI_HOST_INTR_STATE_OFFSET);
1755 addr_hit[ 1] = (reg_addr == SPI_HOST_INTR_ENABLE_OFFSET);
1756 addr_hit[ 2] = (reg_addr == SPI_HOST_INTR_TEST_OFFSET);
1757 addr_hit[ 3] = (reg_addr == SPI_HOST_ALERT_TEST_OFFSET);
1758 addr_hit[ 4] = (reg_addr == SPI_HOST_CONTROL_OFFSET);
1759 addr_hit[ 5] = (reg_addr == SPI_HOST_STATUS_OFFSET);
1760 addr_hit[ 6] = (reg_addr == SPI_HOST_CONFIGOPTS_OFFSET);
1761 addr_hit[ 7] = (reg_addr == SPI_HOST_CSID_OFFSET);
1762 addr_hit[ 8] = (reg_addr == SPI_HOST_COMMAND_OFFSET);
1763 addr_hit[ 9] = (reg_addr == SPI_HOST_ERROR_ENABLE_OFFSET);
1764 addr_hit[10] = (reg_addr == SPI_HOST_ERROR_STATUS_OFFSET);
1765 addr_hit[11] = (reg_addr == SPI_HOST_EVENT_ENABLE_OFFSET);
1766
1767 if (EnableRacl) begin : gen_racl_hit
1768 for (int unsigned slice_idx = 0; slice_idx < 12;
slice_idx++) begin
1769 racl_addr_hit_read[slice_idx] =
1770 addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].read_perm
1771 & racl_role_vec));
1772 racl_addr_hit_write[slice_idx] =
1773 addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].write_perm
1774 & racl_role_vec));
1775 end
1776 end else begin : gen_no_racl
1777 racl_addr_hit_read = addr_hit;
1778 racl_addr_hit_write = addr_hit;
1779 end
1780 end
1781
1782 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
1783 // A valid address hit, access, but failed the RACL check
1784 assign racl_error_o.valid = |addr_hit & ((reg_re & ~|
racl_addr_hit_read) |
1785 (reg_we & ~|
racl_addr_hit_write));
1786 assign racl_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
1787 assign racl_error_o.racl_role = racl_role;
1788 assign racl_error_o.overflow = 1'b0;
1789
1790 if (EnableRacl) begin : gen_racl_log
1791 assign racl_error_o.ctn_uid =
top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a.user.rsvd);
1792 assign racl_error_o.read_access = tl_i.a.opcode ==
tlul_pkg::Get;
1793 end else begin : gen_no_racl_log
1794 assign racl_error_o.ctn_uid = '0;
1795 assign racl_error_o.read_access = 1'b0;
1796 end
1797
1798 // Check sub-word write is permitted
1799 always_comb begin
1800 wr_err = (reg_we &
1801 ((racl_addr_hit_write[ 0] & (|
(SPI_HOST_PERMIT[ 0] & ~reg_be))) |
1802 (racl_addr_hit_write[ 1] & (|
(SPI_HOST_PERMIT[ 1] & ~reg_be))) |
1803 (racl_addr_hit_write[ 2] & (|
(SPI_HOST_PERMIT[ 2] & ~reg_be))) |
1804 (racl_addr_hit_write[ 3] & (|
(SPI_HOST_PERMIT[ 3] & ~reg_be))) |
1805 (racl_addr_hit_write[ 4] & (|
(SPI_HOST_PERMIT[ 4] & ~reg_be))) |
1806 (racl_addr_hit_write[ 5] & (|
(SPI_HOST_PERMIT[ 5] & ~reg_be))) |
1807 (racl_addr_hit_write[ 6] & (|
(SPI_HOST_PERMIT[ 6] & ~reg_be))) |
1808 (racl_addr_hit_write[ 7] & (|
(SPI_HOST_PERMIT[ 7] & ~reg_be))) |
1809 (racl_addr_hit_write[ 8] & (|
(SPI_HOST_PERMIT[ 8] & ~reg_be)))))) |

```

```

1810      (rac1_addr_hit_write[ 9] & (!
(SPI_HOST_PERMIT[ 9] & ~reg_be))) |
1811      (rac1_addr_hit_write[10] & (!
(SPI_HOST_PERMIT[10] & ~reg_be))) |
1812      (rac1_addr_hit_write[11] & (!
(SPI_HOST_PERMIT[11] & ~reg_be)))));
1813  end
1814
1815  // Generate write-enables
1816  assign intr_state_we = rac1_addr_hit_write[0] & reg_we & !
reg_error;
1817
1818  assign intr_state_error_wd = reg_wdata[0];
1819  assign intr_enable_we = rac1_addr_hit_write[1] & reg_we & !
reg_error;
1820
1821  assign intr_enable_error_wd = reg_wdata[0];
1822
1823  assign intr_enable_spi_event_wd = reg_wdata[1];
1824  assign intr_test_we = rac1_addr_hit_write[2] & reg_we & !
reg_error;
1825
1826  assign intr_test_error_wd = reg_wdata[0];
1827
1828  assign intr_test_spi_event_wd = reg_wdata[1];
1829  assign alert_test_we = rac1_addr_hit_write[3] & reg_we & !
reg_error;
1830
1831  assign alert_test_wd = reg_wdata[0];
1832  assign control_we = rac1_addr_hit_write[4] & reg_we & !
reg_error;
1833
1834  assign control_rx_watermark_wd = reg_wdata[7:0];
1835
1836  assign control_tx_watermark_wd = reg_wdata[15:8];
1837
1838  assign control_output_en_wd = reg_wdata[29];
1839
1840  assign control_sw_rst_wd = reg_wdata[30];
1841
1842  assign control_spilen_wd = reg_wdata[31];
1843  assign configopts_we = rac1_addr_hit_write[6] & reg_we & !
reg_error;
1844
1845  assign configopts_clkdiv_wd = reg_wdata[15:0];
1846
1847  assign configopts_csnidle_wd = reg_wdata[19:16];
1848
1849  assign configopts_csntail_wd = reg_wdata[23:20];
1850
1851  assign configopts_csnlead_wd = reg_wdata[27:24];
1852
1853  assign configopts_fullcyc_wd = reg_wdata[29];
1854
1855  assign configopts_cpha_wd = reg_wdata[30];
1856
1857  assign configopts_cpol_wd = reg_wdata[31];
1858  assign csid_we = rac1_addr_hit_write[7] & reg_we & !reg_er-
ror;
1859
1860  assign csid_wd = reg_wdata[31:0];
1861  assign command_we = rac1_addr_hit_write[8] & reg_we & !
reg_error;
1862
1863  assign command_csaat_wd = reg_wdata[0];
1864
1865  assign command_speed_wd = reg_wdata[2:1];
1866
1867  assign command_direction_wd = reg_wdata[4:3];
1868
1869  assign command_len_wd = reg_wdata[24:5];
1870  assign error_enable_we = rac1_addr_hit_write[9] & reg_we & !
reg_error;
1871
1872  assign error_enable_cmdbusy_wd = reg_wdata[0];
1873
1874  assign error_enable_overflow_wd = reg_wdata[1];
1875
1876  assign error_enable_underflow_wd = reg_wdata[2];
1877
1878  assign error_enable_cmdinval_wd = reg_wdata[3];
1879
1880  assign error_enable_csidinval_wd = reg_wdata[4];
1881  assign error_status_we = rac1_addr_hit_write[10] & reg_we
& !reg_error;
1882
1883  assign error_status_cmdbusy_wd = reg_wdata[0];
1884
1885  assign error_status_overflow_wd = reg_wdata[1];
1886
1887  assign error_status_underflow_wd = reg_wdata[2];
1888
1889  assign error_status_cmdinval_wd = reg_wdata[3];
1890
1891  assign error_status_csidinval_wd = reg_wdata[4];
1892
1893  assign error_status_accessinval_wd = reg_wdata[5];
1894  assign event_enable_we = rac1_addr_hit_write[11] & reg_we
& !reg_error;
1895
1896  assign event_enable_rxfull_wd = reg_wdata[0];

```

```

1897
1898  assign event_enable_txempty_wd = reg_wdata[1];
1899
1900  assign event_enable_rxwm_wd = reg_wdata[2];
1901
1902  assign event_enable_txwm_wd = reg_wdata[3];
1903
1904  assign event_enable_ready_wd = reg_wdata[4];
1905
1906  assign event_enable_idle_wd = reg_wdata[5];
1907
1908  // Assign write-enables to checker logic vector.
1909  always_comb begin
1910    reg_we_check[0] = intr_state_we;
1911    reg_we_check[1] = intr_enable_we;
1912    reg_we_check[2] = intr_test_we;
1913    reg_we_check[3] = alert_test_we;
1914    reg_we_check[4] = control_we;
1915    reg_we_check[5] = 1'b0;
1916    reg_we_check[6] = configopts_we;
1917    reg_we_check[7] = csid_we;
1918    reg_we_check[8] = command_we;
1919    reg_we_check[9] = error_enable_we;
1920    reg_we_check[10] = error_status_we;
1921    reg_we_check[11] = event_enable_we;
1922  end
1923
1924  // Read data return
1925  always_comb begin
1926    reg_rdata_next = '0;
1927    unique case (1'b1)
1928      rac1_addr_hit_read[0]: begin
1929        reg_rdata_next[0] = intr_state_error_qs;
1930        reg_rdata_next[1] = intr_state_spi_event_qs;
1931      end
1932
1933      rac1_addr_hit_read[1]: begin
1934        reg_rdata_next[0] = intr_enable_error_qs;
1935        reg_rdata_next[1] = intr_enable_spi_event_qs;
1936      end
1937
1938      rac1_addr_hit_read[2]: begin
1939        reg_rdata_next[0] = '0;
1940        reg_rdata_next[1] = '0;
1941      end
1942
1943      rac1_addr_hit_read[3]: begin
1944        reg_rdata_next[0] = '0;
1945      end
1946
1947      rac1_addr_hit_read[4]: begin
1948        reg_rdata_next[7:0] = control_rx_watermark_qs;
1949        reg_rdata_next[15:8] = control_tx_watermark_qs;
1950        reg_rdata_next[29] = control_output_en_qs;
1951        reg_rdata_next[30] = control_sw_rst_qs;
1952        reg_rdata_next[31] = control_spilen_qs;
1953      end
1954
1955      rac1_addr_hit_read[5]: begin
1956        reg_rdata_next[7:0] = status_txqd_qs;
1957        reg_rdata_next[15:8] = status_rxqd_qs;
1958        reg_rdata_next[19:16] = status_cmdqd_qs;
1959        reg_rdata_next[20] = status_rxwm_qs;
1960        reg_rdata_next[22] = status_byteorder_qs;
1961        reg_rdata_next[23] = status_rxstall_qs;
1962        reg_rdata_next[24] = status_rxempty_qs;
1963        reg_rdata_next[25] = status_rxfull_qs;
1964        reg_rdata_next[26] = status_txwm_qs;
1965        reg_rdata_next[27] = status_txstall_qs;
1966        reg_rdata_next[28] = status_txempty_qs;
1967        reg_rdata_next[29] = status_txfull_qs;
1968        reg_rdata_next[30] = status_active_qs;
1969        reg_rdata_next[31] = status_ready_qs;
1970      end
1971
1972      rac1_addr_hit_read[6]: begin
1973        reg_rdata_next[15:0] = configopts_clkdiv_qs;
1974        reg_rdata_next[19:16] = configopts_csnidle_qs;
1975        reg_rdata_next[23:20] = configopts_csntail_qs;
1976        reg_rdata_next[27:24] = configopts_csnlead_qs;
1977        reg_rdata_next[29] = configopts_fullcyc_qs;
1978        reg_rdata_next[30] = configopts_cpha_qs;
1979        reg_rdata_next[31] = configopts_cpol_qs;
1980      end
1981
1982      rac1_addr_hit_read[7]: begin
1983        reg_rdata_next[31:0] = csid_qs;
1984      end
1985
1986      rac1_addr_hit_read[8]: begin
1987        reg_rdata_next[0] = '0;
1988        reg_rdata_next[2:1] = '0;
1989        reg_rdata_next[4:3] = '0;
1990        reg_rdata_next[24:5] = '0;
1991      end
1992
1993      rac1_addr_hit_read[9]: begin
1994        reg_rdata_next[0] = error_enable_cmdbusy_qs;
1995        reg_rdata_next[1] = error_enable_overflow_qs;
1996        reg_rdata_next[2] = error_enable_underflow_qs;
1997        reg_rdata_next[3] = error_enable_cmdinval_qs;

```

```

1998     reg_rdata_next[4] = error_enable_csinvalid_qs;
1999 end
2000
2001     rac1_addr_hit_read[10]: begin
2002         reg_rdata_next[0] = error_status_cmdbusy_qs;
2003         reg_rdata_next[1] = error_status_overflow_qs;
2004         reg_rdata_next[2] = error_status_underflow_qs;
2005         reg_rdata_next[3] = error_status_cmdinvalid_qs;
2006         reg_rdata_next[4] = error_status_csinvalid_qs;
2007         reg_rdata_next[5] = error_status_accessinvalid_qs;
2008     end
2009
2010     rac1_addr_hit_read[11]: begin
2011         reg_rdata_next[0] = event_enable_rxfull_qs;
2012         reg_rdata_next[1] = event_enable_txempty_qs;
2013         reg_rdata_next[2] = event_enable_rxwm_qs;
2014         reg_rdata_next[3] = event_enable_txwm_qs;
2015         reg_rdata_next[4] = event_enable_ready_qs;
2016         reg_rdata_next[5] = event_enable_idle_qs;
2017     end
2018
2019     default: begin
2020         reg_rdata_next = '1';
2021     end
2022 endcase
2023 end
2024
2025 // shadow busy
2026 logic shadow_busy;
2027 assign shadow_busy = 1'b0;
2028
2029 // register busy
2030 assign reg_busy = shadow_busy;
2031
2032 // Unused signal tieoff
2033
2034 // wdata / byte enable are not always fully used
2035 // add a blanket unused statement to handle lint waivers
2036 logic unused_wdata;
2037 logic unused_be;
2038 assign unused_wdata = ^reg_wdata;
2039 assign unused_be = ^reg_be;
2040 logic unused_policy_sel;
2041 assign unused_policy_sel = ^rac1_policies_i;
2042
2043 // Assertions for Register Interface
2044 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
2045 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
2046
2047 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>
tlo_pre_d_valid, clk_i, !rst_ni)
2048
2049 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
hot0(addr_hit), clk_i, !rst_ni)
2050
2051 // this is formulated as an assumption such that the FPV
testbenches do disprove this
2052 // property by mistake
2053 `ASSUME(reqParity, tl_reg_h2d.a_valid |>
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
2054
2055 endmodule

```

11. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_shift_register.sv (115 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Shift Register for Serial Peripheral Interface (SPI) Host
IP.
6 //
7
8 `include "prim_assert.sv"
9
10 module spi_host_shift_register (
11     input          clk_i,
12     input          rst_ni,
13     input          wr_en_i,
14     output logic   wr_ready_o,
15     input          rd_en_i,
16     output logic   rd_ready_o,
17
18     input          [1:0] speed_i,
19     input          shift_en_i,
20     input          sample_en_i,
21     input          full_cyc_i,
22     input          last_read_i,
23     input          last_write_i,
24
25     input          [7:0] tx_data_i,
26     input          tx_valid_i,
27     output         tx_ready_o,
28     output         tx_flush_o,
29
30     output logic   [7:0] rx_data_o,
31     output logic   rx_valid_o,

```

```

32     input          rx_ready_i,
33     output logic   rx_last_o,
34
35     input          [3:0] sd_i,
36     output logic   [3:0] sd_o,
37
38     input          sw_rst_i
39 );
40 import spi_host_cmd_pkg::*;
41
42     logic          [7:0] sr_q;
43     logic          [7:0] sr_d;
44     logic          [3:0] sd_i_q, sd_i_d;
45     logic          [3:0] next_bits;
46     logic          [7:0] sr_shifted;
47
48     // 9-bit buffer to hold shift register data
49     // and the last_read bit to indicate whether
50     // it is the last in a series.
51     logic          [8:0] rx_buf_q;
52     logic          [8:0] rx_buf_d;
53     logic          rx_buf_valid_q;
54     logic          rx_buf_valid_d;
55
56     `ASSERT(SpeedValid, speed_i != RsvdSpd, clk_i, rst_ni)
57
58     assign next_bits = full_cyc_i ? sd_i : sd_i_q;
59     assign sr_shifted = (speed_i == Standard) ? {sr_q[6:0],
next_bits[1]} :
60                                     (speed_i == Dual) ? {sr_q[5:0],
next_bits[1:0]} :
61                                     (speed_i == Quad) ? {sr_q[3:0],
next_bits[3:0]} :
62                                     8'h00;
63
64     assign sd_o = (speed_i == Standard) ? {3'b000,
sr_q[7]} :
65                                     (speed_i == Dual) ? {2'b00,
sr_q[7:6]} :
66                                     (speed_i == Quad) ? {sr_q[7:4] :
4'h0;
67
68     // Buffer the rx_data outputs to simplify three-way flow
control
69     // between fsm, shift reg and byte_merge.
70     assign rd_ready_o = ~rx_buf_valid_q | (rx_val-
id_o & rx_ready_i);
71     assign rx_valid_o = rx_buf_valid_q;
72
73     assign rx_buf_d = sw_rst_i ?
9'h0 :
74                                     (rd_en_i && rd_ready_o) ?
{last_read_i, sr_shifted} :
75                                     rx_buf_q;
76
77     assign rx_buf_valid_d = sw_rst_i ?
1'b0 :
78                                     (rd_en_i && rd_ready_o) ?
1'b1 :
79                                     (rx_valid_o & rx_ready_i) ?
1'b0 :
80                                     rx_buf_valid_q;
81
82     assign sd_i_d = sw_rst_i ? 4'b0 :
83                                     sample_en_i ? sd_i :
84                                     sd_i_q;
85
86     assign {rx_last_o, rx_data_o} = rx_buf_q;
87
88     // tx_data cannot be buffered, because of the
89     // need to transmit tx_flush as we fetch the
90     // last-needed byte.
91     assign wr_ready_o = tx_valid_i;
92     assign tx_ready_o = wr_en_i;
93     assign tx_flush_o = last_write_i;
94
95     assign sr_d = sw_rst_i ? 8'h0 :
96                                     (wr_en_i & wr_ready_o) ? tx_data_i :
97                                     shift_en_i ? sr_shifted :
98                                     sr_q;
99
100     always_ff @(posedge clk_i or negedge rst_ni) begin
101         if (!rst_ni) begin
102             sd_i_q <= 4'h0;
103             sr_q <= 8'h0;
104             rx_buf_valid_q <= 1'b0;
105             rx_buf_q <= 9'h0;
106         end else begin
107             sd_i_q <= sd_i_d;
108             sr_q <= sr_d;
109             rx_buf_valid_q <= rx_buf_valid_d;
110             rx_buf_q <= rx_buf_d;
111         end
112     end
113 end
114
115 endmodule : spi_host_shift_register

```


12. lowRISC_opentitan/hw/ip/spi_host/rtl/ spi_host_window.sv (127 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Module to manage TX & RX FIFO windows for Serial Peripheral In-
  terface (SPI) host IP.
6 //
7
8 `include "prim_assert.sv"
9
10 module spi_host_window
11 #(
12     parameter bit EnableR-
13     parameter bit RACLer-
14     parameter bit RORRsp = 1'b1,
15     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelWin-
16     RXDATA = 0,
17     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySel-
18     WinTXDATA = 0
19 ) (
20     input clk_i,
21     input rst_ni,
22     input tlul_pkg::tl_h2d_t rx_win_i,
23     output tlul_pkg::tl_d2h_t rx_win_o,
24     input tlul_pkg::tl_h2d_t tx_win_i,
25     output tlul_pkg::tl_d2h_t tx_win_o,
26     output logic [31:0] tx_data_o,
27     output logic [3:0] tx_be_o,
28     output logic tx_valid_o,
29     input [31:0] rx_data_i,
30     output logic rx_ready_o,
31     // RACL interface
32     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
33     output top_racl_pkg::racl_error_log_t racl_error_tx_o,
34     output top_racl_pkg::racl_error_log_t racl_error_rx_o
35 );
36
37 localparam int AW = spi_host_reg_pkg::BlockAw;
38 localparam int DW = 32;
39 localparam int ByteMaskW = DW / 8;
40 localparam top_racl_pkg::racl_range_t [0:0] RaclPoli-
41 cySelRangesTXDATA = '{
42     '{
43         base: {top_pkg::TL_AW{1'b0}},
44         limit: {top_pkg::TL_AW{1'b1}},
45         policy_sel:
46         top_racl_pkg::racl_policy_sel_t'(RaclPolicySelWinTXDATA),
47         enable: 1'b1
48     }
49 };
50
51 logic rx_we;
52
53 // Only support reads from the data RX fifo window
54 logic rx_access_error;
55 assign rx_access_error = rx_we;
56
57 tlul_adapter_reg_racl #(
58     .RegAw ( AW ),
59     .RegDw ( DW ),
60     .EnableDataIntgGen ( 0 ),
61     .EnableRacl ( EnableRacl ),

```

```

62     .RaclErrorRsp ( RaclErrorRsp ),
63     .RaclPolicySelVec ( RaclPolicySelWinRXDATA ),
64 ) u_adapter_rx (
65     .clk_i,
66     .rst_ni,
67     .tl_i ( rx_win_i ),
68     .tl_o ( rx_win_o ),
69     .en_ifetch_i ( prim_mubi_pkg::MuBi4False ),
70     .intg_error_o ( ),
71     .racl_policies_i ( racl_policies_i ),
72     .racl_error_o ( racl_error_rx_o ),
73     .we_o ( rx_we ),
74     .re_o ( rx_ready_o ),
75     .addr_o ( ),
76     .wdata_o ( ),
77     .be_o ( ),
78     .rdata_i ( rx_data_i ),
79     .error_i ( rx_access_error ),
80     .busy_i ( '0'
81 );
82
83 // translate bitmask to byte mask
84 logic [DW-1:0] bit_mask;
85 for (genvar i = 0; i < ByteMaskW; i++) begin : gen_byte_mask
86     assign tx_be_o[i] = |bit_mask[i*8 +: 8];
87
88 // all the bits of particular byte must be the same
89 `ASSERT(BitMaskCheck_A, (|bit_mask[i*8 +: 8] == 1'b0) ||
90     (&bit_mask[i*8 +: 8] == 1'b1))
91
92 end
93
94 // Only support writes to the data TX fifo window
95 tlul_adapter_sram_racl #(
96     .SramAw(AW),
97     .SramDw(DW),
98     .Outstanding(1),
99     .ByteAccess(1),
100     .ErrOnWrite(0),
101     .ErrOnRead(1),
102     .EnableRacl(EnableRacl),
103     .RaclErrorRsp(RaclErrorRsp),
104     .RaclPolicySelNumRanges(1)
105 ) u_adapter_tx (
106     .clk_i,
107     .rst_ni,
108     .tl_i(tx_win_i),
109     .tl_o(tx_win_o),
110     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
111     .req_o(tx_valid_o),
112     .req_type_o(),
113     .gnt_i(1'b1),
114     .we_o(),
115     .addr_o(),
116     .wdata_o(tx_data_o),
117     .wmask_o(bit_mask),
118     .intg_error_o(),
119     .user_rsvd_o(),
120     .rdata_i('0),
121     .rvalid_i('0),
122     .error_i('0),
123     .compound_txn_in_progress_o(),
124     .readback_en_i(prim_mubi_pkg::MuBi4False),
125     .readback_error_o(),
126     .wr_collision_i(1'b0),
127     .write_pending_i(1'b0),
128     .racl_policies_i ( racl_policies_i ),
129     .racl_error_o ( racl_error_tx_o ),
130     .racl_policy_sel_ranges_i(RaclPolicySelRangesTXDATA)
131 );
132
133 endmodule : spi_host_window

```


I2C 컨트롤러

What. I2C 마스터/타깃. 클럭 스트레칭 · 중재 · 시작/정지 조건 · 다중 마스터 지원.

Why it exists. I2C 는 오픈 드레인 양방향 버스라 디지털 설계 관점에서 특이하다. 타깃이 클럭을 늘릴 수 있고(스트레칭), 여러 마스터가 동시에 말하면 중재가 일어난다. 이 두 가지가 단순한 시프트 레지스터로 끝나지 않게 만든다.

학부 복습 · **EECS concepts.** 오픈 드레인/와이어드 AND · 버스 중재 · 준정적 타이밍 · 입력 디바운싱과 필터.

석사 수준 · **Practice note.** 클럭 스트레칭을 지원하지 않는 구현이 흔하고, 그게 현장 버그의 단골이다. IP 데이터시트에서 이 항목을 반드시 확인하라. 파는 쪽이라면 명시하라.

8 files • 3,857 lines

API — i2c.sv

Table 355. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv — module / interface / package (1개)

이름	종류
i2c	module

Table 356. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv — parameter (5개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
InputDelayCycles	int unsigned	0
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl

Table 357. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv — 포트 (32개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t

포트	방향	형/폭
cio_scl_i	input	
cio_scl_o	output	logic
cio_scl_en_o	output	logic
cio_sda_i	input	
cio_sda_o	output	logic
cio_sda_en_o	output	logic
lsio_trigger_o	output	logic
intr_fmt_threshold_o	output	logic
intr_rx_threshold_o	output	logic
intr_acq_threshold_o	output	logic
intr_rx_overflow_o	output	logic
intr_controller_halt_o	output	logic
intr_scl_interference_o	output	logic
intr_sda_interference_o	output	logic
intr_stretch_timeout_o	output	logic
intr_sda_unstable_o	output	logic
intr_cmd_complete_o	output	logic
intr_tx_stretch_o	output	logic
intr_tx_threshold_o	output	logic
intr_acq_stretch_o	output	logic
intr_unexp_stop_o	output	logic
intr_host_timeout_o	output	logic

API — i2c_bus_monitor.sv

Table 358. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_bus_monitor.sv — module / interface / package (1개)

이름	종류
i2c_bus_monitor	module

Table 359. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_bus_monitor.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
scl_i	input	
sda_i	input	
controller_enable_i	input	
multi_controller_enable_i	input	

포트	방향	형/폭
target_enable_i	input	
target_idle_i	input	
thd_dat_i	input	[12:0]
t_buf_i	input	[12:0]
bus_active_timeout_i	input	[29:0]
bus_active_timeout_en_i	input	
bus_inactive_timeout_i	input	[19:0]
bus_free_o	output	logic
start_detect_o	output	
stop_detect_o	output	
event_bus_active_timeout_o	output	
event_host_timeout_o	output	

API — i2c_controller_fsm.sv

Table 360. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — module / interface / package (1개)

이름	종류
i2c_controller_fsm	module

Table 361. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — parameter (1개)

파라미터	형/폭	기본값
FifoDepth	int	64

Table 362. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — localparam (1개)

상수	형/폭	값
FifoDepthWidth	int	\$clog2(FifoDepth+1)

Table 363. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.sv — 포트 (44개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
scl_i	input	
scl_o	output	logic
sda_i	input	
sda_o	output	logic
bus_free_i	input	
transmitting_o	output	logic
host_enable_i	input	

포트	방향	형/폭
halt_controller_i	input	
fmt_fifo_rvalid_i	input	
fmt_fifo_depth_i	input	[FifoDepthWidth-1:0]
fmt_fifo_rready_o	output	logic
fmt_byte_i	input	[7:0]
fmt_flag_start_before_i	input	
fmt_flag_stop_after_i	input	
fmt_flag_read_bytes_i	input	
fmt_flag_read_continue_i	input	
fmt_flag_nak_ok_i	input	
unhandled_unexp_nak_i	input	
unhandled_nak_timeout_i	input	
rx_fifo_wvalid_o	output	logic
rx_fifo_wdata_o	output	logic [RX_FIFO_WIDTH-1:0]
host_idle_o	output	logic
thigh_i	input	[12:0]
tlow_i	input	[12:0]
t_r_i	input	[12:0]
t_f_i	input	[12:0]
thd_sta_i	input	[12:0]
tsu_sta_i	input	[12:0]
tsu_sto_i	input	[12:0]
thd_dat_i	input	[12:0]
sda_interference_i	input	
stretch_timeout_i	input	[29:0]
timeout_enable_i	input	
host_nack_handler_timeout_i	input	[30:0]
host_nack_handler_timeout_en_i	input	
event_nak_o	output	logic
event_unhandled_nak_timeout_o	output	logic
event_arbitration_lost_o	output	logic
event_scl_interference_o	output	logic
event_stretch_timeout_o	output	logic
event_sda_unstable_o	output	logic
complete	output	logic event_cmd_complete_o // Command is

API — i2c_core.sv**Table 364. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — module / interface / package (1개)**

이름	종류
i2c_core	module

Table 365. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — parameter (1개)

파라미터	형/폭	기본값
InputDelayCycles	int unsigned	0

Table 366. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — localparam (4개)

상수	형/폭	값
FifoDepthW	int unsigned	\$clog2(FifoDepth + 1)
AcqFifoDepthW	int unsigned	\$clog2(AcqFifoDepth+1)
MaxFifoDepthW	int unsigned	12
RoundTripCycles	int unsigned	InputDelayCycles + 2 + 1

Table 367. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
reg2hw	input	i2c_reg_pkg::i2c_reg2hw_t
hw2reg	output	i2c_reg_pkg::i2c_hw2reg_t
scl_i	input	
scl_o	output	logic
sda_i	input	
sda_o	output	logic
lsio_trigger_o	output	logic
intr_fmt_threshold_o	output	logic
intr_rx_threshold_o	output	logic
intr_acq_threshold_o	output	logic
intr_rx_overflow_o	output	logic
intr_controller_halt_o	output	logic
intr_scl_interference_o	output	logic
intr_sda_interference_o	output	logic
intr_stretch_timeout_o	output	logic
intr_sda_unstable_o	output	logic

포트	방향	형/폭
intr_cmd_complete_o	output	logic
intr_tx_stretch_o	output	logic
intr_tx_threshold_o	output	logic
intr_acq_stretch_o	output	logic
intr_unexp_stop_o	output	logic
intr_host_timeout_o	output	logic

API — i2c_fifo_sync_sram_adapter.sv

Table 368. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — module / interface / package (1개)

이름	종류
i2c_fifo_sync_sram_adapter	module

Table 369. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — parameter (4개)

파라미터	형/폭	기본값
Width	int unsigned	1
Depth	int unsigned	3
SramAw	int unsigned	1
SramBaseAddr	logic [SramAw-1:0]	'0

Table 370. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — localparam (9개)

상수	형/폭	값
DepthW	int unsigned	prim_util_pkg::vbits(Depth + 1)
InpBufDepth	int unsigned	2
OupBufDepth	int unsigned	2
SramFifoDepth	int	Depth - OupBufDepth
InpBufDepthW	int unsigned	prim_util_pkg::vbits(InpBufDepth + 1)
OupBufDepthW	int unsigned	prim_util_pkg::vbits(OupBufDepth + 1)
SramPtrW	int unsigned	prim_util_pkg::vbits(SramFifoDepth)
SramDepthW	int unsigned	prim_util_pkg::vbits(SramFifoDepth + 1)
SramAddrLeadingZeros	int unsigned	SramAw - SramPtrW

Table 371. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv — 포트 (19개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic

포트	방향	형/폭
fifo_wvalid_i	input	logic
fifo_wready_o	output	logic
fifo_wdata_i	input	logic [Width-1:0]
fifo_rvalid_o	output	logic
fifo_rready_i	input	logic
fifo_rdata_o	output	logic [Width-1:0]
fifo_depth_o	output	logic [DepthW-1:0]
sram_req_o	output	logic
sram_gnt_i	input	logic
sram_write_o	output	logic
sram_addr_o	output	logic [SramAw-1:0]
sram_wdata_o	output	logic [Width-1:0]
sram_wmask_o	output	logic [Width-1:0]
sram_rdata_i	input	logic [Width-1:0]
sram_rvalid_i	input	logic
err_o	output	logic

API — i2c_fifos.sv

Table 372. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifos.sv — module / interface / package (17개)

이름	종류
i2c_fifos	module

Table 373. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifos.sv — localparam (12개)

상수	형/폭	값
FifoDepthW	int unsigned	$\$clog2(\text{FifoDepth} + 1)$
AcqFifoDepthW	int unsigned	$\$clog2(\text{AcqFifoDepth} + 1)$
RamDepth	int unsigned	464
RamAw	int unsigned	$\text{prim_util_pkg::vbits}(\text{RamDepth})$
RamWidth	int unsigned	13
FmtFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(0)$
RxFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(1 * \text{FifoDepth})$
TxFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(2 * \text{FifoDepth})$
AcqFifoRamBaseAddr	logic [RamAw-1:0]	$\text{RamAw}'(3 * \text{FifoDepth})$
RamArbN	int unsigned	4
RamArbIdxW	int unsigned	$\text{prim_util_pkg::vbits}(\text{RamArbN})$
RamArbDw	int unsigned	$1 + \text{RamAw} + \text{RamWidth}$

Table 374. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifos.sv — 포트 (36개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
fmt_fifo_clr_i	input	logic
fmt_fifo_depth_o	output	logic [FifoDepthW-1:0]
fmt_fifo_wvalid_i	input	logic
fmt_fifo_wready_o	output	logic
fmt_fifo_wdata_i	input	logic [FMT_FIFO_WIDTH-1:0]
fmt_fifo_rvalid_o	output	logic
fmt_fifo_rready_i	input	logic
fmt_fifo_rdata_o	output	logic [FMT_FIFO_WIDTH-1:0]
rx_fifo_clr_i	input	logic
rx_fifo_depth_o	output	logic [FifoDepthW-1:0]
rx_fifo_wvalid_i	input	logic
rx_fifo_wready_o	output	logic
rx_fifo_wdata_i	input	logic [RX_FIFO_WIDTH-1:0]
rx_fifo_rvalid_o	output	logic
rx_fifo_rready_i	input	logic
rx_fifo_rdata_o	output	logic [RX_FIFO_WIDTH-1:0]
tx_fifo_clr_i	input	logic
tx_fifo_depth_o	output	logic [FifoDepthW-1:0]
tx_fifo_wvalid_i	input	logic
tx_fifo_wready_o	output	logic
tx_fifo_wdata_i	input	logic [TX_FIFO_WIDTH-1:0]
tx_fifo_rvalid_o	output	logic
tx_fifo_rready_i	input	logic
tx_fifo_rdata_o	output	logic [TX_FIFO_WIDTH-1:0]
acq_fifo_clr_i	input	logic
acq_fifo_depth_o	output	logic [AcqFifoDepthW-1:0]
acq_fifo_wvalid_i	input	logic
acq_fifo_wready_o	output	logic
acq_fifo_wdata_i	input	logic [ACQ_FIFO_WIDTH-1:0]
acq_fifo_rvalid_o	output	logic

포트	방향	형/폭
acq_fifo_rready_i	input	logic
acq_fifo_rdata_o	output	logic [ACQ_FIFO_WIDTH-1:0]

API — i2c_pkg.sv

Table 375. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_pkg.sv — module / interface / package (1개)

이름	종류
i2c_pkg	package

Table 376. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_pkg.sv — parameter (5개)

파라미터	형/폭	기본값
I2C_ACQ_BYTE_ID_WIDTH	int unsigned	3
FMT_FIFO_WIDTH	int unsigned	8 + 5
RX_FIFO_WIDTH	int unsigned	8
TX_FIFO_WIDTH	int unsigned	8
ACQ_FIFO_WIDTH	int unsigned	I2C_ACQ_BYTE_ID_WIDTH + 8

API — i2c_reg_pkg.sv

Table 377. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_reg_pkg.sv — module / interface / package (1개)

이름	종류
i2c_reg_pkg	package

Table 378. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_reg_pkg.sv — parameter (69개) — 앞 60개만 표시(전체 69개, 본문 소스 참조)

파라미터	형/폭	기본값
FifoDepth	int	64
AcqFifoDepth	int	268
NumAlerts	int	1
BlockAw	int	7
NumRegs	int	32
I2C_INTR_STATE_OFFSET	logic [BlockAw-1:0]	7'h 0
I2C_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	7'h 4
I2C_INTR_TEST_OFFSET	logic [BlockAw-1:0]	7'h 8
I2C_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	7'h c
I2C_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 10
I2C_STATUS_OFFSET	logic [BlockAw-1:0]	7'h 14
I2C_RDATA_OFFSET	logic [BlockAw-1:0]	7'h 18
I2C_FDATA_OFFSET	logic [BlockAw-1:0]	7'h 1c

파라미터	형/폭	기본값
I2C_FIFO_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 20
I2C_HOST_FIFO_CONFIG_OFFSET	logic [BlockAw-1:0]	7'h 24
I2C_TARGET_FIFO_CONFIG_OFFSET	logic [BlockAw-1:0]	7'h 28
I2C_HOST_FIFO_STATUS_OFFSET	logic [BlockAw-1:0]	7'h 2c
I2C_TARGET_FIFO_STATUS_OFFSET	logic [BlockAw-1:0]	7'h 30
I2C_OVRD_OFFSET	logic [BlockAw-1:0]	7'h 34
I2C_VAL_OFFSET	logic [BlockAw-1:0]	7'h 38
I2C_TIMING0_OFFSET	logic [BlockAw-1:0]	7'h 3c
I2C_TIMING1_OFFSET	logic [BlockAw-1:0]	7'h 40
I2C_TIMING2_OFFSET	logic [BlockAw-1:0]	7'h 44
I2C_TIMING3_OFFSET	logic [BlockAw-1:0]	7'h 48
I2C_TIMING4_OFFSET	logic [BlockAw-1:0]	7'h 4c
I2C_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 50
I2C_TARGET_ID_OFFSET	logic [BlockAw-1:0]	7'h 54
I2C_ACQDATA_OFFSET	logic [BlockAw-1:0]	7'h 58
I2C_TXDATA_OFFSET	logic [BlockAw-1:0]	7'h 5c
I2C_HOST_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 60
I2C_TARGET_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 64
I2C_TARGET_NACK_COUNT_OFFSET	logic [BlockAw-1:0]	7'h 68
I2C_TARGET_ACK_CTRL_OFFSET	logic [BlockAw-1:0]	7'h 6c
I2C_ACQ_FIFO_NEXT_DATA_OFFSET	logic [BlockAw-1:0]	7'h 70
I2C_HOST_NACK_HANDLER_TIMEOUT_OFFSET	logic [BlockAw-1:0]	7'h 74
I2C_CONTROLLER_EVENTS_OFFSET	logic [BlockAw-1:0]	7'h 78
I2C_TARGET_EVENTS_OFFSET	logic [BlockAw-1:0]	7'h 7c
I2C_INTR_TEST_RESVAL	logic [14:0]	15'h 0
I2C_INTR_TEST_FMT_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_RX_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_ACQ_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_RX_OVERFLOW_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_CONTROLLER_HALT_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_SCL_INTERFERENCE_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_SDA_INTERFERENCE_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_STRETCH_TIMEOUT_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_SDA_UNSTABLE_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_CMD_COMPLETE_RESVAL	logic [0:0]	1'h 0

파라미터	형/폭	기본값
I2C_INTR_TEST_TX_STRETCH_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_TX_THRESHOLD_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_ACQ_STRETCH_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_UNEXP_STOP_RESVAL	logic [0:0]	1'h 0
I2C_INTR_TEST_HOST_TIMEOUT_RESVAL	logic [0:0]	1'h 0
I2C_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
I2C_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
I2C_STATUS_RESVAL	logic [10:0]	11'h 33c
I2C_STATUS_FMEMPTY_RESVAL	logic [0:0]	1'h 1
I2C_STATUS_HOSTIDLE_RESVAL	logic [0:0]	1'h 1
I2C_STATUS_TARGETIDLE_RESVAL	logic [0:0]	1'h 1
I2C_STATUS_RXEMPTY_RESVAL	logic [0:0]	1'h 1

1. lowRISC_opentitan/hw/ip/i2c/rtl/i2c.sv (184 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: I2C top level wrapper file
6
7 `include "prim_assert.sv"
8
9 module i2c
10   import i2c_reg_pkg::*;
11   #(
12     parameter logic [NumAlerts-1:0]      AlertAsyn-
13       = {NumAlerts{1'b1}},
14     // Number of cycles a differential skew is tolerated on the
15     alert signal
16     parameter int unsigned                AlertSkew-
17       = 1,
18     parameter int unsigned                InputDelay-
19       = 0,
20     parameter bit                         EnableR-
21       = 1'b0,
22     parameter bit                         RaclEr-
23       = EnableRacl,
24     parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
25     cySelVec[NumRegs] = '{NumRegs{0}}
26   ) (
27     input          clk_i,
28     input          rst_ni,
29     input  prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_i,
30     output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_o,
31
32     // Bus Interface
33     input  tlul_pkg::tl_h2d_t tl_i,
34     output tlul_pkg::tl_d2h_t tl_o,
35
36     // Alerts
37     input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
38     alert_rx_i,
39     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
40     alert_tx_o,
41
42     // RACL interface
43     input  top_racl_pkg::racl_policy_vec_t racl_policies_i,
44     output top_racl_pkg::racl_error_log_t  racl_error_o,
45
46     // Generic IO
47     input          cio_scl_i,
48     output logic   cio_scl_o,
49     output logic   cio_scl_en_o,
50     input          cio_sda_i,
51     output logic   cio_sda_o,
52     output logic   cio_sda_en_o,
53
54     output logic   lsio_trigger_o,
55
56     // Interrupts
57     output logic   intr_fmt_threshold_o,
58
59     output logic   intr_rx_threshold_o,
60     output logic   intr_acq_threshold_o,
61     output logic   intr_rx_overflow_o,
62     output logic   intr_controller_halt_o,
63     output logic   intr_scl_interference_o,
64     output logic   intr_sda_interference_o,
65     output logic   intr_stretch_timeout_o,
66     output logic   intr_sda_unstable_o,
67     output logic   intr_cmd_complete_o,
68     output logic   intr_tx_stretch_o,
69     output logic   intr_tx_threshold_o,
70     output logic   intr_acq_stretch_o,
71     output logic   intr_unexp_stop_o,
72     output logic   intr_host_timeout_o
73   );
74
75   i2c_reg2hw_t reg2hw;
76   i2c_hw2reg_t hw2reg;
77
78   logic [NumAlerts-1:0] alert_test, alerts;
79
80   i2c_reg_top #(
81     .EnableRacl(EnableRacl),
82     .RaclErrorRsp(RaclErrorRsp),
83     .RaclPolicySelVec(RaclPolicySelVec)
84   ) u_reg (
85     .clk_i,
86     .rst_ni,
87     .tl_i,
88     .tl_o,
89     .reg2hw,
90     .hw2reg,
91     .racl_policies_i,
92     .racl_error_o,
93     // SEC_CM: BUS.INTEGRITY
94     .intg_err_o(alerts[0])
95   );
96
97   assign alert_test = {
98     reg2hw.alert_test.q &
99     reg2hw.alert_test.qe
100   };
101
102   for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
103     prim_alert_sender #(
104       .AsyncOn(AlertAsyncOn[i]),
105       .SkewCycles(AlertSkewCycles),
106       .IsFatal(1'b1)
107     ) u_prim_alert_sender (
108       .clk_i,
109       .rst_ni,
110       .alert_test_i ( alert_test[i] ),
111       .alert_req_i   ( alerts[i] ),
112       .alert_ack_o   (          ),
113       .alert_state_o (          ),
114       .alert_rx_i    ( alert_rx_i[i] ),
115       .alert_tx_o    ( alert_tx_o[i] )
116     );
117   end
118
119   logic scl_int;
120   logic sda_int;

```

```

112 i2c_core #(
113     .InputDelayCycles(InputDelayCycles)
114 ) i2c_core (
115     .clk_i,
116     .rst_ni,
117     .ram_cfg_i,
118     .ram_cfg_o,
119
120     .reg2hw,
121     .hw2reg,
122
123     .scl_i(cio_scl_i),
124     .scl_o(scl_int),
125     .sda_i(cio_sda_i),
126     .sda_o(sda_int),
127
128     .lsio_trigger_o,
129
130     .intr_fmt_threshold_o,
131     .intr_rx_threshold_o,
132     .intr_acq_threshold_o,
133     .intr_rx_overflow_o,
134     .intr_controller_halt_o,
135     .intr_scl_interference_o,
136     .intr_sda_interference_o,
137     .intr_stretch_timeout_o,
138     .intr_sda_unstable_o,
139     .intr_cmd_complete_o,
140     .intr_tx_stretch_o,
141     .intr_tx_threshold_o,
142     .intr_acq_stretch_o,
143     .intr_unexp_stop_o,
144     .intr_host_timeout_o
145 );
146
147 // For I2C, in standard, fast and fast-plus modes, outputs
simulated as open-drain outputs.
148 // Asserting scl or sda high should be equivalent to a tri-
state output.
149 // The output, when enabled, should only assert low.
150
151 assign cio_scl_o = 1'b0;
152 assign cio_sda_o = 1'b0;
153
154 assign cio_scl_en_o = ~scl_int;
155 assign cio_sda_en_o = ~sda_int;
156
157 `ASSERT_KNOWN(TIDValidKnown0_A, tl_o.d_valid)
158 `ASSERT_KNOWN(TIARReadyKnown0_A, tl_o.a_ready)
159 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
160 `ASSERT_KNOWN(CioSclKnown0_A, cio_scl_o)
161 `ASSERT_KNOWN(CioSclEnKnown0_A, cio_scl_en_o)
162 `ASSERT_KNOWN(CioSdaKnown0_A, cio_sda_o)
163 `ASSERT_KNOWN(CioSdaEnKnown0_A, cio_sda_en_o)
164 `ASSERT_KNOWN(IntrFmtWtmkKnown0_A, intr_fmt_threshold_o)
165 `ASSERT_KNOWN(IntrRxWtmkKnown0_A, intr_rx_threshold_o)
166 `ASSERT_KNOWN(IntrAcqWtmkKnown0_A, intr_acq_threshold_o)
167 `ASSERT_KNOWN(IntrRxOflwKnown0_A, intr_rx_overflow_o)
168 `ASSERT_KNOWN(IntrControllerHaltKnown0_A, intr_control-
ler_halt_o)
169 `ASSERT_KNOWN(IntrSclInterfKnown0_A, intr_scl_interfer-
ence_o)
170 `ASSERT_KNOWN(IntrSdaInterfKnown0_A, intr_sda_interfer-
ence_o)
171 `ASSERT_KNOWN(IntrStretchTimeoutKnown0_A,
intr_stretch_timeout_o)
172 `ASSERT_KNOWN(IntrSdaUnstableKnown0_A, intr_sda_unstable_o)
173 `ASSERT_KNOWN(IntrCommandCompleteKnown0_A, intr_cmd_com-
plete_o)
174 `ASSERT_KNOWN(IntrTxStretchKnown0_A, intr_tx_stretch_o)
175 `ASSERT_KNOWN(IntrTxWtmkKnown0_A, intr_tx_threshold_o)
176 `ASSERT_KNOWN(IntrAcqStretchKnown0_A, intr_acq_stretch_o)
177 `ASSERT_KNOWN(IntrUnexpStopKnown0_A, intr_unexp_stop_o)
178 `ASSERT_KNOWN(IntrHostTimeoutKnown0_A, intr_host_timeout_o)
179 `ASSERT_KNOWN(LsioTriggerKnown0_A, lsio_trigger_o)
180 `ASSERT_KNOWN(RaclErrorValidKnown0_A, racl_error_o.valid)
181
182 // Alert assertions for reg_we onehot check
183 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
184 endmodule

```

2. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_bus_monitor.sv (306 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: I2C bus idle and timeout monitoring, plus
Start/Stop
6 // detection.
7
8 module i2c_bus_monitor import i2c_pkg::*;
9 (
10     input          clk_i,
11     input          rst_ni,

```

```

12
13     input          scl_i,
14     input          sda_i,
15
16     input          controller_enable_i,
17     input          multi_controller_enable_i,
18     input          target_enable_i,
19     input          target_idle_i,
20     input [12:0]   thd_dat_i, //
Data hold time(< 200 ns, < thd_sta)
21     input [12:0]   t_buf_i, //
Bus free time (< 5 us)
22     input [29:0]   bus_active_timeout_i, //
SCL held low (~25 ms)
23     input          bus_active_timeout_en_i,
24     input [19:0]   bus_inactive_timeout_i, //
SCL held high (~50 us)
25
26     output logic   bus_free_o,
27     output         start_detect_o,
28     output         stop_detect_o,
29
30     output         event_bus_active_timeout_o,
31     output         event_host_timeout_o,
32
33 );
34
35 // Only activate this monitor if at least one of the modules is
enabled.
36 logic monitor_enable, monitor_enable_q;
37 assign monitor_enable = controller_enable_i | tar-
get_enable_i | multi_controller_enable_i;
38
39 always_ff @(posedge clk_i or negedge rst_ni) begin
40     if (!rst_ni) begin
41         monitor_enable_q <= 1'b0;
42     end else begin
43         monitor_enable_q <= monitor_enable;
44     end
45 end
46
47 // SDA and SCL at the previous clock edge
48 logic scl_i_q, sda_i_q;
49 always_ff @(posedge clk_i or negedge rst_ni) begin :
bus_prev
50     if (!rst_ni) begin
51         scl_i_q <= 1'b1;
52         sda_i_q <= 1'b1;
53     end else begin
54         scl_i_q <= scl_i;
55         sda_i_q <= sda_i;
56     end
57 end
58
59 // Start and Stop detection
60 //
61 // To resolve ambiguity with early SDA arrival, reject con-
trol symbols when
62 // SCL goes low too soon. The hold time for ordinary data/
ACK bits is too
63 // short to reliably see SCL change before SDA. Use the hold
time for
64 // control signals to ensure a Start or Stop symbol was ac-
tually received.
65 // Requirements: thd_dat + 1 < thd_sta
66 // The extra (+1) here is to account for a late SDA ar-
rival due to CDC
67 // skew.
68 //
69 // Note that this counter combines Start and Stop detection
into one
70 // counter. A controller-only reset scenario could end up
with a Stop
71 // following shortly after a Start, with the requisite setup
time not
72 // observed.
73 logic start_det_trigger, start_det_pending;
74 logic start_det; // indicates start or repeated
start is detected on the bus
75 logic stop_det_trigger, stop_det_pending;
76 logic stop_det; // indicates stop is detected on
the bus
77
78 // Stop / Start detection counter
79 logic [13:0] ctrl_det_count;
80
81 always_ff @(posedge clk_i or negedge rst_ni) begin
82     if (!rst_ni) begin
83         ctrl_det_count <= '0;
84     end else if (start_det_trigger || stop_det_trigger) begin
85         ctrl_det_count <= 14'd1;
86     end else if (start_det_pending || stop_det_pending) begin
87         ctrl_det_count <= ctrl_det_count + 1'b1;
88     end
89 end
90
91 always_ff @(posedge clk_i or negedge rst_ni) begin

```

```

92   if (!rst_ni) begin
93     start_det_pending <= 1'b0;
94   end else if (start_det_trigger) begin
95     start_det_pending <= 1'b1;
96   end else if (!monitor_enable || !scl_i || start_det ||
stop_det_trigger) begin
97     start_det_pending <= 1'b0;
98   end
99 end
100
101 always_ff @(posedge clk_i or negedge rst_ni) begin
102   if (!rst_ni) begin
103     stop_det_pending <= 1'b0;
104   end else if (stop_det_trigger) begin
105     stop_det_pending <= 1'b1;
106   end else if (!monitor_enable || !scl_i || stop_det ||
start_det_trigger) begin
107     stop_det_pending <= 1'b0;
108   end
109 end
110
111 // (Repeated) Start condition detection by target
112 assign start_det_trigger = monitor_enable && (scl_i_q &&
scl_i) & (sda_i_q && !sda_i);
113 assign start_det = monitor_enable && start_det_pending &&
(ctrl_det_count >= 14'(thd_dat_i));
114 assign start_detect_o = start_det;
115
116 // Stop condition detection by target
117 assign stop_det_trigger = monitor_enable && (scl_i_q &&
scl_i) & (!sda_i_q && sda_i);
118 assign stop_det = monitor_enable && stop_det_pending &&
(ctrl_det_count >= 14'(thd_dat_i));
119 assign stop_detect_o = stop_det;
120
121 //
122 // Bus timeout logic
123 //
124
125 // Detection of bus in a released state.
126 logic bus_idling;
127 assign bus_idling = scl_i && (sda_i == sda_i_q);
128
129 logic [29:0] bus_release_cnt, bus_release_cnt_sel;
130 logic bus_release_cnt_load, bus_release_cnt_dec;
131
132 // bus_inactive_timeout_det is only high for the case where
the bus release
133
134 // counter reaches zero for bus idling, not the wait after a
Stop condition.
135 logic bus_inactive_timeout_det;
136
137 logic bus_inactive_timeout_en;
138 assign bus_inactive_timeout_en = (bus_inactive_timeout_i >
'0);
139
140 // bus_active_timeout latches high when SCL has been held
low for too long.
141
142 // This can be done intentionally or unintentionally, as the
response for
143
144 // target devices should be to abort the current transaction
and release any
145 // hold on the bus. Note that the bus doesn't immediately
transition to
146 // "free" status, since the controller can continue holding
SCL low for some
147 // time.
148 logic bus_active_timeout_det_d, bus_active_timeout_det_q;
149 always_ff @(posedge clk_i or negedge rst_ni) begin
150   if (!rst_ni) begin
151     bus_active_timeout_det_q <= 1'b0;
152   end else begin
153     bus_active_timeout_det_q <= bus_active_timeout_det_d;
154   end
155 end
156
157 // Bus release counter.
158 // Together with the FSM below, this counter detects bus
timeouts and the end
159 // of the bus free time following a Stop condition.
160 always_ff @ (posedge clk_i or negedge rst_ni) begin :
bus_idle
161   if (!rst_ni) begin
162     bus_release_cnt <= '0;
163   end else if (monitor_enable && !monitor_enable_q) begin
164     // The rising edge of monitor enable resets the counter
for
165     // multi-controller mode.
166     if (multi_controller_enable_i) begin
167       // For the multi-controller case, wait until the bus
isn't busy before
168       // transmitting.
169       bus_release_cnt <= 30'(bus_inactive_timeout_i);
170     end
171   end else if (bus_release_cnt_load) begin
172     bus_release_cnt <= bus_release_cnt_sel;
173   end else if (bus_release_cnt_dec && (bus_release_cnt !=
'0)) begin

```

```

171     bus_release_cnt <= bus_release_cnt - 1'b1;
172   end
173 end
174
175 typedef enum logic [1:0] {
176   // Bus is currently free. Can transmit.
177   StBusFree,
178   // Bus is busy and not held with SCL high.
179   StBusBusyLow,
180   // Bus is currently busy, but SCL is held high.
181   StBusBusyHigh,
182   // Bus is currently busy, but saw a Stop. Count down to
Bus Free.
183   StBusBusyStop
184 } bus_state_e;
185
186 bus_state_e state_q, state_d;
187
188 always_comb begin
189   state_d = state_q;
190   bus_release_cnt_load = 1'b0;
191   bus_release_cnt_sel = 30'(t_buf_i);
192   bus_release_cnt_dec = 1'b0;
193   bus_inactive_timeout_det = 1'b0;
194   bus_active_timeout_det_d = bus_active_timeout_det_q;
195
196   unique case (state_q)
197     StBusFree: begin
198       bus_active_timeout_det_d = 1'b0;
199
200       if (!scl_i || !sda_i) begin
201         state_d = StBusBusyLow;
202         bus_release_cnt_load = 1'b1;
203         bus_release_cnt_sel = bus_active_timeout_i;
204       end
205     end
206
207     StBusBusyLow: begin
208       bus_release_cnt_dec = !scl_i;
209
210       if (stop_det) begin
211         state_d = StBusBusyStop;
212         bus_release_cnt_load = 1'b1;
213         bus_release_cnt_sel = 30'(t_buf_i);
214       end else if (bus_idling && bus_inactive_timeout_en)
begin
215         state_d = StBusBusyHigh;
216         bus_release_cnt_load = 1'b1;
217         bus_release_cnt_sel = 30'(bus_inactive_timeout_i);
218       end else if (scl_i) begin
219         bus_release_cnt_load = 1'b1;
220         bus_release_cnt_sel = bus_active_timeout_i;
221       end if (bus_active_timeout_det_q) begin
222         // SCL was released due to the bus timeout, so go
to BusFree.
223         state_d = StBusFree;
224       end
225     end else if (bus_release_cnt == 30'd1) begin
226       // The active timeout occurs when SCL has been held
continuously low
227       // for too long. Both the controller and target
should respond to
228       // this timeout and release the bus. We don't con-
sider the bus free
229       // yet, though: SCL must be released first.
230       bus_active_timeout_det_d = bus_active_timeout_en_i;
231     end
232   end
233
234   StBusBusyHigh: begin
235     bus_release_cnt_dec = 1'b1;
236
237     if (stop_det) begin
238       state_d = StBusBusyStop;
239       bus_release_cnt_load = 1'b1;
240       bus_release_cnt_sel = 30'(t_buf_i);
241     end else if (!bus_idling) begin
242       state_d = StBusBusyLow;
243       bus_release_cnt_load = 1'b1;
244       bus_release_cnt_sel = bus_active_timeout_i;
245     end else if (bus_release_cnt == 30'd1) begin
246       // The host_timeout interrupt occurs regardless of
which value of
247       // SDA was present, but only transition to StBusFree
if we entered
248       // this state with SDA high. If SDA is low, a change
to SCL will
249       // cause a transition back to StBusBusyLow. If SDA
changes from low
250       // to high, we get a Stop condition and transition
to StBusBusyStop.
251       bus_inactive_timeout_det = bus_inactive_timeout_en;
252       if (sda_i) begin
253         state_d = StBusFree;
254       end
255     end
256   end
257
258   StBusBusyStop: begin

```

```

259     bus_release_cnt_dec = 1'b1;
260
261     if (!scl_i || !sda_i) begin
262         state_d = StBusBusyLow;
263     end else if (bus_release_cnt == 30'd1) begin
264         state_d = StBusFree;
265     end
266 end
267
268 default: begin
269     state_d = StBusFree;
270 end
271 endcase
272 end
273
274 always_ff @(posedge clk_i or negedge rst_ni) begin
275     if (!rst_ni) begin
276         state_q <= StBusFree;
277     end else if (!monitor_enable) begin
278         state_q <= StBusFree;
279     end else if (monitor_enable && !monitor_enable_q) begin
280         if (multi_controller_enable_i) begin
281             // For the multi-controller case, wait until the bus
282             // isn't busy before
283             // transmitting.
284             state_q <= StBusBusyHigh;
285         end else begin
286             state_q <= StBusFree;
287         end
288     end else begin
289         state_q <= state_d;
290     end
291 end
292
293 always_comb begin
294     if (multi_controller_enable_i) begin
295         bus_free_o = (state_q == StBusFree);
296     end else begin
297         // For single-controller cases, the bus is only "busy"
298         // while waiting for the "bus free" time
299         // after a Stop condition. In other words, that is the
300         // only time our controller can't
301         // continue to the next transaction.
302         bus_free_o = (state_q != StBusBusyStop);
303     end
304 end
305
306 assign event_bus_active_timeout_o = bus_active_timeout_det_d
307 && !bus_active_timeout_det_q;
308 assign event_host_timeout_o = !target_idle_i && bus_inactive_timeout_det;
309
310 endmodule

```

3. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_controller_fsm.v (995 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Description: I2C finite state machine
6
7 `include "prim_assert.sv"
8
9 module i2c_controller_fsm import i2c_pkg::*;
10 #(
11     parameter int FifoDepth = 64,
12     localparam int FifoDepthWidth = $clog2(FifoDepth+1)
13 ) (
14     input          clk_i, // clock
15     input          rst_ni, // active low reset
16
17     input          scl_i, // serial clock input from i2c bus
18     output logic   scl_o, // serial clock output to i2c bus
19     input          sda_i, // serial data input from i2c bus
20     output logic   sda_o, // serial data output to i2c bus
21     input          bus_free_i, // High if the bus is free for
22     // a new transmission
23     output logic   transmitting_o, // Controller is currently
24     // transmitting SDA
25     input          host_enable_i, // enable host functionality
26     input          halt_controller_i, // Halt the controller FSM
27
28     // in idle
29     input          fmt_fifo_rvalid_i, // indicates
30     // there is valid data in fmt_fifo
31     input [FifoDepthWidth-1:0] fmt_fifo_depth_i, //
32     // fmt_fifo_depth
33     output logic   fmt_fifo_rready_o, // populates
34     // fmt_fifo
35     input [7:0]    fmt_byte_i, // byte in
36     // fmt_fifo to be sent to target
37
38     input          fmt_flag_start_before_i, // issue
39     // start before sending byte

```

```

32     input          fmt_flag_stop_after_i, // issue
33     // stop after sending byte
34     input          fmt_flag_read_bytes_i, // in-
35     // dicates byte is an number of reads
36     input          fmt_flag_read_continue_i, // host
37     // to send Ack to final byte read
38     input          fmt_flag_nak_ok_i, // no
39     // Ack is expected
40     input          unhandled_unexp_nak_i,
41     // input          unhandled_nak_timeout_i, // NACK
42     // handler timeout event not cleared
43
44     output logic   rx_fifo_wvalid_o, // high
45     // if there is valid data in rx_fifo
46     output logic [RX_FIFO_WIDTH-1:0] rx_fifo_wdata_o, // byte
47     // in rx_fifo read from target
48
49     output logic   host_idle_o, // indicates the host
50     // is idle
51
52     input [12:0]   thigh_i, // high period of the SCL in clock
53     // units
54     input [12:0]   tlow_i, // low period of the SCL in clock
55     // units
56
57     input [12:0]   t_r_i, // rise time of both SDA and SCL in
58     // clock units
59
60     input [12:0]   t_f_i, // fall time of both SDA and SCL in
61     // clock units
62     input [12:0]   thd_sta_i, // hold time for (repeated) START
63     // in clock units
64
65     input [12:0]   tsu_sta_i, // setup time for repeated START in
66     // clock units
67     input [12:0]   tsu_sto_i, // setup time for STOP in clock
68     // units
69     input [12:0]   thd_dat_i, // data hold time in clock units
70     // sda_interference_i, // High when SCL is high
71     // and SDA doesn't match while transmitting
72     input [29:0]   stretch_timeout_i, // max time target connec-
73     // ted to this host may stretch the clock
74     input          timeout_enable_i, // assert if target
75     // stretches clock past max
76     // threshold for unhandled Host-Mode 'nak' irq.
77     input          host_nack_handler_timeout_i, // Timeout
78     // threshold for unhandled Host-Mode 'nak' irq.
79     input          host_nack_handler_timeout_en_i,
80
81     output logic   event_nak_o, // target didn't Ack
82     // when expected
83     output logic   event_unhandled_nak_timeout_o, // SW didn't
84     // handle the NACK in time
85     output logic   event_arbitration_lost_o, // Lost arbitration
86     // after beginning a transaction
87     output logic   event_scl_interference_o, // other device for-
88     // cing SCL low
89     output logic   event_stretch_timeout_o, // target stretches
90     // clock past max time
91     output logic   event_sda_unstable_o, // SDA is not con-
92     // stant during SCL pulse
93     output logic   event_cmd_complete_o, // Command is com-
94     // plete
95 );
96
97 // I2C bus clock timing variables
98 logic [15:0] tcount_q; // current counter for setting
99 delays
100 logic [15:0] tcount_d; // next counter for setting
101 delays
102 logic load_tcount; // indicates counter must be
103 loaded
104 logic [30:0] stretch_idle_cnt; // counter for clock being
105 stretched by target
106 // or clock idle by host.
107 logic [30:0] unhandled_nak_cnt; // In Host-mode, count
108 // cycles where the FSM is halted awaiting
109
110 // the nak irq to be handled by
111 // SW.
112 logic incr_nak_cnt;
113
114 // (IP in HOST-Mode) This bit is active when the FSM is in a
115 // state where a TARGET might
116
117 // be trying to stretch the clock, preventing the controller
118 // FSM from continuing.
119 logic stretch_en;
120
121 // Bit and byte counter variables
122 logic [2:0] bit_index; // bit being transmitted to or
123 // read from the bus
124 logic bit_decr; // indicates bit_index must be
125 // decremented by 1
126 logic bit_clr; // indicates bit_index must be
127 // reset to 7
128 logic [8:0] byte_num; // number of bytes to read
129 logic [8:0] byte_index; // byte being read from the bus
130 logic byte_decr; // indicates byte_index must be
131 // decremented by 1

```



```

88 logic      byte_clr;      // indicates byte_index must be
reset to byte_num
89
90 // Other internal variables
91 logic      scl_d;          // scl internal
92 logic      sda_d;          // data internal
93 logic      scl_i_q;        // scl_i delayed by one clock
94 logic      sda_i_q;        // sda_i delayed by one clock
95 logic [7:0] read_byte;    // register for reads from tar-
get
96 logic      read_byte_clr;  // clear read_byte contents
97 logic      shift_data_en;  // indicates data must be shif-
ted in from the bus
98 logic      trans_started; // indicates a transaction has
started
99 logic      pend_restart;  // there is a pending restart
waiting to be processed
100 logic     req_restart;    // request restart
101 logic     log_start;      // indicates start is been is-
sued
102 logic     log_stop;       // indicates stop is been issued
103 logic     auto_stop_d, auto_stop_q; // Tracks whether a
Stop symbol was autonomously generated.
104 logic     ctrl_symbol_failed; // If SCL pulls low before
the controller can issue Start/Stop
105
106 // Clock counter implementation
107 typedef enum logic [3:0] {
108     tSetupStart,
109     tHoldStart,
110     tClockStart,
111     tClockLow,
112     tClockPulse,
113     tClockHigh,
114     tHoldBit,
115     tClockStop,
116     tSetupStop,
117     tNoDelay
118 } tcount_sel_e;
119
120 tcount_sel_e tcount_sel;
121
122 always_comb begin : counter_functions
123     tcount_d = tcount_q;
124     if (load_tcount) begin
125         unique case (tcount_sel)
126             tSetupStart : tcount_d = 16'(t_r_i) + 16'(tsu_sta_i);
127             tHoldStart  : tcount_d = 16'(t_f_i) + 16'(thd_sta_i);
128             tClockStart : tcount_d = 16'(thd_dat_i);
129             tClockLow   : tcount_d = 16'(tlow_i) - 16'(thd_dat_i);
130             tClockPulse : tcount_d = 16'(t_r_i) + 16'(thigh_i);
131             tClockHigh  : tcount_d = 16'(thigh_i);
132             tHoldBit    : tcount_d = 16'(t_f_i) + 16'(thd_dat_i);
133             tClockStop  : tcount_d = 16'(t_f_i) + 16'(tlow_i) -
16'(thd_dat_i);
134             tSetupStop  : tcount_d = 16'(t_r_i) + 16'(tsu_sto_i);
135             tNoDelay    : tcount_d = 16'h0001;
136             default     : tcount_d = 16'h0001;
137         endcase
138     end else if (
139         host_enable_i ||
140         // If we disable Host-Mode mid-txn, keep counting until
the end of
141         // byte, at which point we create a STOP condition then
return to Idle.
142         (!host_idle_o && !host_enable_i)) begin
143         tcount_d = tcount_q - 1'b1;
144     end
145 end
146
147 always_ff @ (posedge clk_i or negedge rst_ni) begin :
clk_counter
148     if (!rst_ni) begin
149         tcount_q <= '1;
150     end else begin
151         tcount_q <= tcount_d;
152     end
153 end
154
155 // Clock stretching/idle detection when i2c_ctrl.
156 // When in host mode, this is a stretch count for how long
an external target
157 // has stretched the clock.
158 // When in target mode, this is an idle count for how long
an external host
159 // has kept the clock idle after a START indication.
160 always_ff @ (posedge clk_i or negedge rst_ni) begin :
clk_stretch
161     if (!rst_ni) begin
162         stretch_idle_cnt <= '0;
163     end else if (stretch_en && !scl_i) begin
164         // HOST-mode count of clock stretching
165         stretch_idle_cnt <= stretch_idle_cnt + 1'b1;
166     end else begin
167         stretch_idle_cnt <= '0;
168     end
169 end
170
171 // The TARGET can stretch the clock during any time that the
host drives SCL to 0.

```

```

172 // However, we (the HOST) cannot know it is being stretched
until we release SCL,
173 // usually trying to create the next clock pulse.
174 // There is a minimum 3-cycle round trip (1-cycle output
flop, 2-cycle input synchronizer),
175 // between releasing the clock and observing the effect of
releasing the clock on
176 // the inputs. However, this is really '1 + t_r + 2' as the
bus also needs to slew to '1'
177 // before can observe it. Even if the TARGET is not stretch-
ing the clock, we cannot
178 // confirm it until at-least this amount of time has
elapsed.
179 //
180 // 'stretch_predict_cnt_expired' becomes active once we have
observed (4 + t_r) cycles of
181 // delay, and if !scl_i at this point we know that the TAR-
GET is stretching the clock.
182 // > This implementation requires 'thigh >= 4' to guarantee
we don't miss stretching.
183 logic [30:0] stretch_cnt_threshold;
184 assign stretch_cnt_threshold = 31'd2 + 31'(t_r_i);
185
186 logic stretch_predict_cnt_expired;
187 always_ff @ (posedge clk_i or negedge rst_ni) begin
188     if (!rst_ni) begin
189         stretch_predict_cnt_expired <= 1'b0;
190     end else begin
191         if (stretch_idle_cnt == stretch_cnt_threshold) begin
192             stretch_predict_cnt_expired <= 1'b1;
193         end else if (!stretch_en) begin
194             stretch_predict_cnt_expired <= 1'b0;
195         end
196     end
197 end
198
199 // While the FSM is halted due to an unhandled 'nak' irq in
Host-Mode, this counter can
200 // be used to trigger a timeout which disables Host-Mode and
creates a STOP condition to
201 // end the transaction.
202 logic unhandled_nak_cnt_expired;
203 always_ff @ (posedge clk_i or negedge rst_ni) begin : un-
handled_nak_cnt_b
204     if (!rst_ni) begin
205         unhandled_nak_cnt <= '0;
206         unhandled_nak_cnt_expired <= 1'b0;
207     end else if (incr_nak_cnt) begin
208         // Increment the counter while the FSM is halted in
Idle.
209         unhandled_nak_cnt <= unhandled_nak_cnt + 1'b1;
210         if (unhandled_nak_cnt > host_nack_handler_timeout_i) be-
gin
211             unhandled_nak_cnt_expired <= 1'b1;
212         end
213     end else begin
214         unhandled_nak_cnt <= '0;
215         unhandled_nak_cnt_expired <= 1'b0;
216     end
217 end
218
219 assign event_unhandled_nak_timeout_o = un-
handled_nak_cnt_expired;
220
221 // Bit index implementation
222 always_ff @ (posedge clk_i or negedge rst_ni) begin :
bit_counter
223     if (!rst_ni) begin
224         bit_index <= 3'd7;
225     end else if (bit_clr) begin
226         bit_index <= 3'd7;
227     end else if (bit_decr) begin
228         bit_index <= bit_index - 1'b1;
229     end else begin
230         bit_index <= bit_index;
231     end
232 end
233
234 // Deserializer for a byte read from the bus
235 always_ff @ (posedge clk_i or negedge rst_ni) begin :
read_register
236     if (!rst_ni) begin
237         read_byte <= 8'h00;
238     end else if (read_byte_clr) begin
239         read_byte <= 8'h00;
240     end else if (shift_data_en) begin
241         read_byte[7:0] <= {read_byte[6:0], sda_i}; // MSB goes
in first
242     end
243 end
244
245 // Number of bytes to read
246 always_comb begin : byte_number
247     if (!fmt_flag_read_bytes_i) byte_num = 9'd0;
248     else if (fmt_byte_i == '0) byte_num = 9'd256;
249     else byte_num = 9'(fmt_byte_i);
250 end
251
252 // Byte index implementation

```

```

253 always_ff @(posedge clk_i or negedge rst_ni) begin :
byte_counter
254 if (!rst_ni) begin
255   byte_index <= '0;
256 end else if (byte_clr) begin
257   byte_index <= byte_num;
258 end else if (byte_decr) begin
259   byte_index <= byte_index - 1'b1;
260 end else begin
261   byte_index <= byte_index;
262 end
263 end
264
265 // SDA and SCL at the previous clock edge
266 always_ff @(posedge clk_i or negedge rst_ni) begin :
bus_prev
267 if (!rst_ni) begin
268   scl_i_q <= 1'b1;
269   sda_i_q <= 1'b1;
270 end else begin
271   scl_i_q <= scl_i;
272   sda_i_q <= sda_i;
273 end
274 end
275
276 // SCL going low early is just clock synchronization. SCL
switching so fast that the controller
277 // can't even sense its outputs is cause for a bus error.
278 assign event_arbitration_lost_o = event_sda_unstable_o ||
sda_interference_i ||
279                               ctrl_symbol_failed;
280
281 // Registers whether a transaction start has been observed.
282 // A transaction start does not include a "restart", but
rather
283 // the first start after enabling i2c, or a start observed
after a
284 // stop.
285 always_ff @(posedge clk_i or negedge rst_ni) begin
286   if (!rst_ni) begin
287     trans_started <= '0;
288   end else if (trans_started && (!host_enable_i ||
event_arbitration_lost_o)) begin
289     trans_started <= '0;
290   end else if (log_start) begin
291     trans_started <= 1'b1;
292   end else if (log_stop) begin
293     trans_started <= 1'b0;
294   end
295 end
296
297 always_ff @(posedge clk_i or negedge rst_ni) begin
298   if (!rst_ni) begin
299     pend_restart <= '0;
300   end else if (pend_restart && !host_enable_i) begin
301     pend_restart <= '0;
302   end else if (req_restart) begin
303     pend_restart <= 1'b1;
304   end else if (log_start) begin
305     pend_restart <= '0;
306   end
307 end
308
309 always_ff @(posedge clk_i or negedge rst_ni) begin
310   if (!rst_ni) begin
311     auto_stop_q <= 1'b0;
312   end else begin
313     auto_stop_q <= auto_stop_d;
314   end
315 end
316
317 // State definitions
318 typedef enum logic [4:0] {
319   Idle,
320   // Host function states
321   // Host function starts a transaction
322   SetupStart, HoldStart, ClockStart,
323   // Host function stops a transaction
324   SetupStop, HoldStop, ClockStop,
325   // Host function transmits a bit to the external target
326   ClockLow, ClockPulse, HoldBit,
327   // Host function receives an ack from the external target
328   ClockLowAck, ClockPulseAck, HoldDevAck,
329   // Host function reads a bit from the external target
330   ReadClockLow, ReadClockPulse, ReadHoldBit,
331   // Host function transmits an ack to the external target
332   HostClockLowAck, HostClockPulseAck, HostHoldBitAck
333 } state_e;
334
335 state_e state_q, state_d;
336
337 // Increment the NACK timeout count if the controller is
halted in Idle and
338 // the timeout hasn't yet occurred.
339
340
343 assign incr_nak_cnt = unhandled_unexp_nak_i && host_enable_i
&& (state_q == Idle) &&

```

```

344 host_nack_handler_timeout_en_i && !un-
handled_nak_timeout_i;
345
346 // Outputs for each state
347 always_comb begin : state_outputs
348   host_idle_o = 1'b1;
349   sda_d = 1'b1;
350   scl_d = 1'b1;
351   transmitting_o = 1'b0;
352   log_start = 1'b0;
353   log_stop = 1'b0;
354   fmt_fifo_rready_o = 1'b0;
355   rx_fifo_wvalid_o = 1'b0;
356   rx_fifo_wdata_o = RX_FIFO_WIDTH'(0);
357   event_nak_o = 1'b0;
358   event_scl_interference_o = 1'b0;
359   ctrl_symbol_failed = 1'b0;
360   event_sda_unstable_o = 1'b0;
361   event_cmd_complete_o = 1'b0;
362   stretch_en = 1'b0;
363   unique case (state_q)
364     // Idle: initial state, SDA is released (high), SCL is
released if the
365     // bus is idle. Otherwise, if no STOP condition has been
sent yet,
366     // continue pulling SCL low in host mode.
367     Idle : begin
368       sda_d = 1'b1;
369       if (trans_started) begin
370         host_idle_o = 1'b0;
371         scl_d = 1'b0;
372       end else begin
373         host_idle_o = 1'b1;
374         scl_d = 1'b1;
375       end
376     end
377     // HOST MODE //
378     // SetupStart: SDA and SCL are released
379     SetupStart : begin
380       host_idle_o = 1'b0;
381       sda_d = 1'b1;
382       scl_d = 1'b1;
383       transmitting_o = 1'b1;
384       // If this is a restart, SCL was last low, and a tar-
get could be stretching the clock.
385       stretch_en = trans_started;
386       if (trans_started && !scl_i && scl_i_q) begin
387         // If this is a repeated Start, an early clock pre-
vents issuing the symbol. If it's not
388         // a repeated start, the FSM will just go back to
Idle and wait for the bus to go free
389         // again.
390         ctrl_symbol_failed = 1'b1;
391       end else if (tcount_q == 16'd1) begin
392         log_start = 1'b1;
393         event_cmd_complete_o = pend_restart;
394       end
395     end
396     // HoldStart: SDA is pulled low, SCL is released
397     HoldStart : begin
398       host_idle_o = 1'b0;
399       sda_d = 1'b0;
400       scl_d = 1'b1;
401       transmitting_o = 1'b1;
402       if (scl_i_q && !scl_i) event_scl_interference_o =
1'b1;
403     end
404     // ClockStart: SCL is pulled low, SDA stays low
405     ClockStart : begin
406       host_idle_o = 1'b0;
407       sda_d = 1'b0;
408       scl_d = 1'b0;
409       transmitting_o = 1'b1;
410     end
411     // ClockLow : begin
412     ClockLow : begin
413       host_idle_o = 1'b0;
414       if (pend_restart) begin
415         sda_d = 1'b1;
416       end else begin
417         sda_d = fmt_byte_i[bit_index];
418       end
419       scl_d = 1'b0;
420       transmitting_o = 1'b1;
421     end
422     // ClockPulse: SCL is released, SDA keeps the indexed
bit value
423     ClockPulse : begin
424       host_idle_o = 1'b0;
425       sda_d = fmt_byte_i[bit_index];
426       scl_d = 1'b1;
427       transmitting_o = 1'b1;
428       stretch_en = 1'b1;
429       if (scl_i_q && !scl_i) event_scl_interference_o =
1'b1;
430     end
431     if (scl_i_q && scl_i && (sda_i_q != sda_i)) begin
432       // Unexpected Stop / Start

```

```

435     event_sda_unstable_o = '1'b1;
436 end
437 end
438 // HoldBit: SCL is pulled low
439 HoldBit : begin
440     host_idle_o = '1'b0;
441     sda_d = fmt_byte_i[bit_index];
442     scl_d = '1'b0;
443     transmitting_o = '1'b1;
444 end
445 // ClockLowAck: SCL pulled low, SDA is released
446 ClockLowAck : begin
447     host_idle_o = '1'b0;
448     sda_d = '1'b1;
449     scl_d = '1'b0;
450 end
451 // ClockPulseAck: SCL is released
452 ClockPulseAck : begin
453     host_idle_o = '1'b0;
454     sda_d = '1'b1;
455     scl_d = '1'b1;
456     if (!scli_q && scl_i && sda_i && !fmt_flag_nak_ok_i)
event_nak_o = '1'b1;
457     stretch_en = '1'b1;
458     if (scli_q && !scl_i) event_scl_interference_o =
1'b1;
459     if (scli_q && scl_i && (sda_i_q != sda_i)) begin
460         // Unexpected Stop / Start
461         event_sda_unstable_o = '1'b1;
462     end
463 end
464 // HoldDevAck: SCL is pulled low
465 HoldDevAck : begin
466     host_idle_o = '1'b0;
467     sda_d = '1'b1;
468     scl_d = '1'b0;
469 end
470 // ReadClockLow: SCL is pulled low, SDA is released
471 ReadClockLow : begin
472     host_idle_o = '1'b0;
473     sda_d = '1'b1;
474     scl_d = '1'b0;
475 end
476 // ReadClockPulse: SCL is released, the indexed bit
value is read off SDA
477 ReadClockPulse : begin
478     host_idle_o = '1'b0;
479     scl_d = '1'b1;
480     stretch_en = '1'b1;
481     if (scli_q && !scl_i) event_scl_interference_o =
1'b1;
482     if (scli_q && scl_i && (sda_i_q != sda_i)) begin
483         // Unexpected Stop / Start
484         event_sda_unstable_o = '1'b1;
485     end
486 end
487 // ReadHoldBit: SCL is pulled low
488 ReadHoldBit : begin
489     host_idle_o = '1'b0;
490     scl_d = '1'b0;
491     if (bit_index == '0 && tcount_q == 16'd1) begin
492         rx_fifo_wvalid_o = '1'b1; // assert that rx_fifo
has valid data
493         rx_fifo_wdata_o = read_byte; // transfer read data
to rx_fifo
494     end
495 end
496 // HostClockLowAck: SCL pulled low, SDA is conditional
497 HostClockLowAck : begin
498     host_idle_o = '1'b0;
499     scl_d = '1'b0;
500     transmitting_o = '1'b1;
501
502 // If it is the last byte of a read, send a NAK before
the stop.
503 // Otherwise send the ack.
504 if (fmt_flag_read_continue_i) sda_d = '1'b0;
505 else if (byte_index == 9'd1) sda_d = '1'b1;
506 else sda_d = '1'b0;
507 end
508 // HostClockPulseAck: SCL is released
509 HostClockPulseAck : begin
510     host_idle_o = '1'b0;
511     if (fmt_flag_read_continue_i) sda_d = '1'b0;
512     else if (byte_index == 9'd1) sda_d = '1'b1;
513     else sda_d = '1'b0;
514     scl_d = '1'b1;
515     transmitting_o = '1'b1;
516     stretch_en = '1'b1;
517     if (scli_q && !scl_i) event_scl_interference_o =
1'b1;
518     if (scli_q && scl_i && (sda_i_q != sda_i)) begin
519         // Unexpected Stop / Start
520         event_sda_unstable_o = '1'b1;
521     end
522 end
523 // HostHoldBitAck: SCL is pulled low
524 HostHoldBitAck : begin
525     host_idle_o = '1'b0;

```

```

526     if (fmt_flag_read_continue_i) sda_d = '1'b0;
527     else if (byte_index == 9'd1) sda_d = '1'b1;
528     else sda_d = '1'b0;
529     scl_d = '1'b0;
530     transmitting_o = '1'b1;
531 end
532 // ClockStop: SCL is pulled low, SDA stays low
533 ClockStop : begin
534     host_idle_o = '1'b0;
535     sda_d = '1'b0;
536     scl_d = '1'b0;
537     transmitting_o = '1'b1;
538 end
539 // SetupStop: SDA is pulled low, SCL is released
540 SetupStop : begin
541     host_idle_o = '1'b0;
542     sda_d = '1'b0;
543     scl_d = '1'b1;
544     transmitting_o = '1'b1;
545     stretch_en = '1'b1;
546     if (!scli_q && scl_i_q) begin
547         // Failed to issue Stop before some other device
could pull SCL low.
548         ctrl_symbol_failed = '1'b1;
549     end
550 end
551 // HoldStop: SDA and SCL are released
552 HoldStop : begin
553     host_idle_o = '1'b0;
554     sda_d = '1'b1;
555     scl_d = '1'b1;
556     event_cmd_complete_o = '1'b1;
557     if (!sda_i && !scl_i) begin
558         // Failed to issue Stop before some other device
could pull SCL low.
559         ctrl_symbol_failed = '1'b1;
560     end else if (sda_i) begin
561         log_stop = '1'b1;
562     end
563 end
564 // Active: continue while keeping SCL low
565 Active : begin
566     host_idle_o = '1'b0;
567
568 // If this is a transaction start, do not drive scl
low
569 // since in the next state we will drive it high to
initiate
570 // the start bit.
571 // If this is a restart, continue driving the clock
low.
572     scl_d = fmt_flag_start_before_i && !trans_started;
573 end
574 // PopFmtFifo: populate fmt_fifo
575 PopFmtFifo : begin
576     host_idle_o = '1'b0;
577     if (fmt_flag_stop_after_i) scl_d = '1'b1;
578     else scl_d = '1'b0;
579     fmt_fifo_rready_o = '1'b1;
580 end
581
582 // default
583 default : begin
584     host_idle_o = '1'b1;
585     sda_d = '1'b1;
586     scl_d = '1'b1;
587     transmitting_o = '1'b0;
588     log_start = '1'b0;
589     log_stop = '1'b0;
590     event_scl_interference_o = '1'b0;
591     ctrl_symbol_failed = '1'b0;
592     fmt_fifo_rready_o = '1'b0;
593     rx_fifo_wvalid_o = '1'b0;
594     rx_fifo_wdata_o = RX_FIFO_WIDTH'(0);
595     event_nak_o = '1'b0;
596     event_sda_unstable_o = '1'b0;
597     event_cmd_complete_o = '1'b0;
598 end
599 endcase // unique case (state_q)
600 end
601
602 // Conditional state transition
603 always_comb begin : state_functions
604     state_d = state_q;
605     load_tcount = '1'b0;
606     tcount_sel = tNoDelay;
607     bit_decr = '1'b0;
608     bit_clr = '1'b0;
609     byte_decr = '1'b0;
610     byte_clr = '1'b0;
611     read_byte_clr = '1'b0;
612     shift_data_en = '1'b0;
613     req_restart = '1'b0;
614     auto_stop_d = auto_stop_q;
615
616     unique case (state_q)
617         // Idle: initial state, SDA is released (high), and SCL
is released if there is no ongoing
618         // transaction.
619         Idle : begin
620             if (host_enable_i) begin

```

```

621         if (unhandled_unexp_nak_i || unhandled_nak_timeout_i
|| halt_controller_i) begin
622             // If we are awaiting software to handle an unex-
pected NACK, halt the FSM here.
623             // The current transaction does not end, and SCL
remains in its current state.
624             // Software typically should handle an unexpected
NACK by either disabling the
625             // controller (causing host_enable_i to fall) or
by the following sequence:
626             // 1. Clear and/or populate the FMT FIFO.
627             // 2. Clear CONTROLLER_EVENTS.NACK
628             // Note that if the timeout feature is enabled,
the controller will be forced to
629             // issue a Stop if software takes too long to ad-
dress the NACK. A short timeout
630             // could also be used to automatically issue a
Stop whenever an unexpected NACK
631             // occurs.
632             // Note that we may also halt here on a bus
timeout or if arbitration was lost, so
633             // software may fix up the FIFOs before beginning
a new transaction.
634             if (trans_started && unhandled_nak_cnt_expired)
begin
635                 // If our timeout counter expires, generate a
STOP condition automatically.
636                 auto_stop_d = 1'b1;
637                 state_d = ClockStop;
638                 load_tcount = 1'b1;
639                 tcount_sel = tClockStop;
640             end
641             end else if (fmt_fifo_rvalid_i) begin
642                 if (trans_started || bus_free_i) begin
643                     state_d = Active;
644                 end
645             end
646             end else if (trans_started && !host_enable_i) begin
647                 auto_stop_d = 1'b1;
648                 state_d = ClockStop;
649                 load_tcount = 1'b1;
650                 tcount_sel = tClockStop;
651             end
652         end
653         //////////////////////////////////
654         // HOST MODE //
655         //////////////////////////////////
656         // SetupStart: SDA and SCL are released
657         SetupStart : begin
658             if (!trans_started && !scl_i) begin
659                 // This was the start of a transaction, but another
device beat us to access. Go back to
660                 // Idle, and wait for the next turn.
661                 state_d = Idle;
662             end else if (trans_started && !scl_i && !scl_i_q &&
stretch_predict_cnt_expired) begin
663                 // Saw stretching. Remain in this state and don't
count down until we see SCL high.
664                 state_d = SetupStart;
665                 load_tcount = 1'b1;
666                 // This double-counts the rise time, unfortunately.
667                 tcount_sel = tSetupStart;
668             end else if (trans_started && !scl_i && scl_i_q) begin
669                 // Failed to issue repeated Start. Effectively lost
arbitration.
670                 state_d = Idle;
671             end else if (tcount_q == 16'd1) begin
672                 state_d = HoldStart;
673                 load_tcount = 1'b1;
674                 tcount_sel = tHoldStart;
675             end
676         end
677         // HoldStart: SDA is pulled low, SCL is released
678         HoldStart : begin
679             if (tcount_q == 16'd1 || (!scl_i && scl_i_q)) begin
680                 state_d = ClockStart;
681                 load_tcount = 1'b1;
682                 tcount_sel = tClockStart;
683             end
684         end
685         // ClockStart: SCL is pulled low, SDA stays low
686         ClockStart : begin
687             if (tcount_q == 16'd1) begin
688                 state_d = ClockLow;
689                 load_tcount = 1'b1;
690                 tcount_sel = tClockLow;
691             end
692         end
693         // ClockLow: SCL stays low, shift indexed bit onto SDA
694         ClockLow : begin
695             if (tcount_q == 16'd1) begin
696                 load_tcount = 1'b1;
697                 if (pend_restart) begin
698                     state_d = SetupStart;
699                     tcount_sel = tSetupStart;
700                 end else begin
701                     state_d = ClockPulse;
702                     tcount_sel = tClockPulse;
703                 end
704             end

```

```

705         end
706     end
707 end
708 // ClockPulse: SCL is released, SDA keeps the indexed
bit value
709 ClockPulse : begin
710     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
711         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
712         load_tcount = 1'b1;
713         tcount_sel = tClockHigh;
714     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
715         // Unexpected Stop / Start
716         state_d = Idle;
717     end else if (tcount_q == 16'd1 || (!scl_i && scl_i_q))
begin
718         // Transition either when we finish counting our
high period or
719         // another controller pulls clock low.
720         state_d = HoldBit;
721         load_tcount = 1'b1;
722         tcount_sel = tHoldBit;
723     end
724 end
725 // HoldBit: SCL is pulled low
726 HoldBit : begin
727     if (tcount_q == 16'd1) begin
728         load_tcount = 1'b1;
729         tcount_sel = tClockLow;
730         if (bit_index == '0) begin
731             state_d = ClockLowAck;
732             bit_clr = 1'b1;
733         end else begin
734             state_d = ClockLow;
735             bit_decr = 1'b1;
736         end
737     end
738 end
739 // ClockLowAck: Target is allowed to drive ack back
740 // to host (dut)
741 ClockLowAck : begin
742     if (tcount_q == 16'd1) begin
743         state_d = ClockPulseAck;
744         load_tcount = 1'b1;
745         tcount_sel = tClockPulse;
746     end
747 end
748 // ClockPulseAck: SCL is released
749 ClockPulseAck : begin
750     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
751         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
752         load_tcount = 1'b1;
753         tcount_sel = tClockHigh;
754     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
755         // Unexpected Stop / Start
756         state_d = Idle;
757     end else begin
758         if (tcount_q == 16'd1 || (!scl_i && scl_i_q)) begin
759             state_d = HoldDevAck;
760             load_tcount = 1'b1;
761             tcount_sel = tHoldBit;
762         end
763     end
764 end
765 // HoldDevAck: SCL is pulled low
766 HoldDevAck : begin
767     if (tcount_q == 16'd1) begin
768         if (fmt_flag_stop_after_i) begin
769             state_d = ClockStop;
770             load_tcount = 1'b1;
771             tcount_sel = tClockStop;
772         end else begin
773             state_d = PopFmtFifo;
774             load_tcount = 1'b1;
775             tcount_sel = tNoDelay;
776         end
777     end
778 end
779 // ReadClockLow: SCL is pulled low, SDA is released
780 ReadClockLow : begin
781     if (tcount_q == 16'd1) begin
782         state_d = ReadClockPulse;
783         load_tcount = 1'b1;
784         tcount_sel = tClockPulse;
785     end
786 end
787 // ReadClockPulse: SCL is released, the indexed bit
value is read off SDA
788 ReadClockPulse : begin
789     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
790         // Saw stretching. Remain in this state and don't

```

```

count down until we see SCL high.
791     load_tcount = 1'b1;
792     tcount_sel = tClockHigh;
793     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
794         // Unexpected Stop / Start
795         state_d = Idle;

796     end else if (tcount_q == 16'd1 || (!scl_i && scl_i_q))
begin
797         state_d = ReadHoldBit;
798         load_tcount = 1'b1;
799         tcount_sel = tHoldBit;

800     shift_data_en = 1'b1; // SDA is sampled on the final
clk_i cycle of the SCL pulse.
801     end
802 end
803 // ReadHoldBit: SCL is pulled low
804 ReadHoldBit : begin
805     if (tcount_q == 16'd1) begin
806         load_tcount = 1'b1;
807         tcount_sel = tClockLow;
808         if (bit_index == '0) begin
809             state_d = HostClockLowAck;
810             bit_clr = 1'b1;
811             read_byte_clr = 1'b1;
812         end else begin
813             state_d = ReadClockLow;
814             bit_decr = 1'b1;
815         end
816     end
817 end
818 // HostClockLowAck: SCL is pulled low, SDA is condition-
al based on
819 // byte position
820 HostClockLowAck : begin
821     if (tcount_q == 16'd1) begin
822         state_d = HostClockPulseAck;
823         load_tcount = 1'b1;
824         tcount_sel = tClockPulse;
825     end
826 end
827 // HostClockPulseAck: SCL is released
828 HostClockPulseAck : begin

829     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
830         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
831         load_tcount = 1'b1;
832         tcount_sel = tClockHigh;
833     end else if (scl_i_q && scl_i && (sda_i_q != sda_i))
begin
834         // Unexpected Stop / Start
835         state_d = Idle;

836     end else if (tcount_q == 16'd1 || (!scl_i && scl_i_q))
begin
837         state_d = HostHoldBitAck;
838         load_tcount = 1'b1;
839         tcount_sel = tHoldBit;
840     end
841 end
842 // HostHoldBitAck: SCL is pulled low
843 HostHoldBitAck : begin
844     if (tcount_q == 16'd1) begin
845         if (byte_index == 9'd1) begin
846             if (fmt_flag_stop_after_i) begin
847                 state_d = ClockStop;
848                 load_tcount = 1'b1;
849                 tcount_sel = tClockStop;
850             end else begin
851                 state_d = PopFmtFifo;
852                 load_tcount = 1'b1;
853                 tcount_sel = tNoDelay;
854             end
855         end else begin
856             state_d = ReadClockLow;
857             load_tcount = 1'b1;
858             tcount_sel = tClockLow;
859             byte_decr = 1'b1;
860         end
861     end
862 end
863 // ClockStop: SCL is pulled low, SDA stays low
864 ClockStop : begin
865     if (tcount_q == 16'd1) begin
866         state_d = SetupStop;
867         load_tcount = 1'b1;
868         tcount_sel = tSetupStop;
869     end
870 end
871 // SetupStop: SDA is pulled low, SCL is released
872 SetupStop : begin

873     if (!scl_i && !scl_i_q && stretch_predict_cnt_expired)
begin
874         // Saw stretching. Remain in this state and don't
count down until we see SCL high.
875         load_tcount = 1'b1;

```

```

876         tcount_sel = tSetupStop;
877     end else if (!scl_i && scl_i_q) begin
878         // Failed to issue Stop before some other device
could pull SCL low.
879         state_d = Idle;
880     end else if (tcount_q == 16'd1) begin
881         state_d = HoldStop;
882     end
883 end
884 // HoldStop: SDA and SCL are released
885 HoldStop : begin
886     if (!sda_i && !scl_i) begin
887         // Failed to issue Stop before some other device
could pull SCL low.
888         state_d = Idle;
889         auto_stop_d = 1'b0;
890     end else if (sda_i) begin
891         auto_stop_d = 1'b0;
892     end if (auto_stop_q) begin
893         // If this Stop symbol was generated automatic-
ally, go back to Idle.
894         state_d = Idle;
895         load_tcount = 1'b1;
896         tcount_sel = tNoDelay;
897     end else begin
898         state_d = PopFmtFifo;
899         load_tcount = 1'b1;
900         tcount_sel = tNoDelay;
901     end
902 end
903 end
904 // Active: continue while keeping SCL low
905 Active : begin
906     if (fmt_flag_read_bytes_i) begin
907         byte_clr = 1'b1;
908         state_d = ReadClockLow;
909         load_tcount = 1'b1;
910         tcount_sel = tClockLow;
911     end else if (fmt_flag_start_before_i && !
trans_started) begin
912         state_d = SetupStart;
913         load_tcount = 1'b1;
914         tcount_sel = tSetupStart;
915     end else begin
916         state_d = ClockLow;
917         load_tcount = 1'b1;
918         req_restart = fmt_flag_start_before_i;
919         tcount_sel = tClockLow;
920     end
921 end
922 // PopFmtFifo: pop fmt_fifo item
923 PopFmtFifo : begin
924     if (!host_enable_i && trans_started) begin
925         auto_stop_d = 1'b1;
926         state_d = ClockStop;
927         load_tcount = 1'b1;
928         tcount_sel = tClockStop;
929     end else if (!host_enable_i || (fmt_fifo_depth_i ==
7'h1) ||
930         unhandled_unexp_nak_i || !trans_started)
begin
931         state_d = Idle;
932         load_tcount = 1'b1;
933         tcount_sel = tNoDelay;
934     end else begin
935         state_d = Active;
936         load_tcount = 1'b1;
937         tcount_sel = tNoDelay;
938     end
939 end
940 // default
941 default : begin
942     state_d = Idle;
943     load_tcount = 1'b0;
944     tcount_sel = tNoDelay;
945     bit_decr = 1'b0;
946     bit_clr = 1'b0;
947     byte_decr = 1'b0;
948     byte_clr = 1'b0;
949     read_byte_clr = 1'b0;
950     shift_data_en = 1'b0;
951     auto_stop_d = 1'b0;
952 end
953 end
954 endcase // unique case (state_q)
955
956     if (trans_started && (sda_interference_i || ctrl_sym-
bol_failed)) begin
957         state_d = Idle;
958     end
959 end
960
961 // Synchronous state transition
962 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_transition
963     if (!rst_ni) begin
964         state_q <= Idle;
965     end else begin
966         state_q <= state_d;
967     end
968 end

```



```

969
970 assign scl_o = scl_d;
971 assign sda_o = sda_d;
972
973 // Target stretched clock beyond timeout
974 assign event_stretch_timeout_o = stretch_en &&
timeout_enable_i &&
975         (stretch_idle_cnt >
31'(stretch_timeout_i));
976
977 // Make sure we never attempt to send a single cycle glitch
978 `ASSERT(SclOutputGlitch_A, $rose(scl_o) |-> ##1 scl_o)
979
980 // TODO: Handle the assertion below
981 // // I2C bus outputs
982 // always_ff @(posedge clk_i or negedge rst_ni) begin
983 //     if (!rst_ni) begin
984 //         scl_q <= 'b1;
985 //         sda_q <= 'b1;
986 //     end else begin
987 //         scl_q <= scl_d;
988 //         sda_q <= sda_d;
989 //     end
990 // end
991 //
992 // // Check that we don't change SCL and SDA in the same
clock cycle in host mode.
993 // `ASSERT(SclSdaChangeNotSimultaneous_A, !(host_enable_i &&
(scl_d != scl_q) && (sda_d != sda_q)))
994
995 endmodule

```

4. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_core.sv (891 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: I2C core module
6
7 `include "prim_assert.sv"
8
9 module i2c_core import i2c_pkg::*;
10 #(
11     parameter int unsigned InputDelayCycles = 0
12 ) (
13     input                clk_i,
14     input                rst_ni,
15     input  prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_i,
16     output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_o,
17
18     input i2c_reg_pkg::i2c_reg2hw_t      reg2hw,
19     output i2c_reg_pkg::i2c_hw2reg_t      hw2reg,
20
21     input                scl_i,
22     output logic         scl_o,
23     input                sda_i,
24     output logic         sda_o,
25
26     output logic         lsio_trigger_o,
27
28     output logic         intr_fmt_threshold_o,
29     output logic         intr_rx_threshold_o,
30     output logic         intr_acq_threshold_o,
31     output logic         intr_rx_overflow_o,
32     output logic         intr_control-
ler_halt_o,
33     output logic         intr_scl_interfer-
ence_o,
34     output logic         intr_sda_interfer-
ence_o,
35     output logic         intr_stretch_timeout_o,
36     output logic         intr_sda_un-
stable_o,
37     output logic         intr_cmd_com-
plete_o,
38     output logic         intr_tx_stretch_o,
39     output logic         intr_acq_stretch_o,
40     output logic         intr_unexp_stop_o,
41     output logic         intr_host_timeout_o
42 );
43
44 import i2c_reg_pkg::FifoDepth;
45 import i2c_reg_pkg::AcqFifoDepth;
46
47 // Number of bits required to represent the FIFO level/
depth.
48 localparam int unsigned FifoDepthW = $clog2(FifoDepth + 1);
49 localparam int unsigned AcqFifoDepthW = $clog2(AcqFifoDepth+1);
50
51 // Maximum number of bits required to represent the level/

```

```

depth of any FIFO.
53 localparam int unsigned MaxFifoDepthW = 12;
54
55 // Round-trip delay for outputs to appear on the inputs, not
including rise
56 // time. This is the input delay external to this IP, plus
the output flop,
57 // plus the 2-flop synchronizer on the input. The total
value here
58 // represents the minimum allowed high and low times for
SCL.
59 localparam int unsigned RoundTripCycles = InputDelayCycles + 2
+ 1;
60
61 logic [12:0] thigh;
62 logic [12:0] tlow;
63 logic [12:0] t_r;
64 logic [12:0] t_f;
65 logic [12:0] thd_sta;
66 logic [12:0] tsu_sta;
67 logic [12:0] tsu_sto;
68 logic [12:0] tsu_dat;
69 logic [12:0] thd_dat;
70 logic [12:0] t_buf;
71 logic [29:0] bus_active_timeout;
72 logic         stretch_timeout_enable;
73 logic         bus_timeout_enable;
74 logic [19:0] host_timeout;
75 logic [30:0] nack_timeout;
76 logic         nack_timeout_en;
77 logic [30:0] host_nack_handler_timeout;
78 logic         host_nack_handler_timeout_en;
79
80 logic scl_sync;
81 logic sda_sync;
82 logic scl_out_controller_fsm, sda_out_controller_fsm;
83 logic scl_out_target_fsm, sda_out_target_fsm;
84 logic scl_out_fsm;
85 logic sda_out_fsm;
86 logic controller_transmitting;
87 logic target_transmitting;
88
89 // bus_event_detect goes low after any drive change from
this IP, and it
90 // returns to high once enough time has passed for the out-
put change to
91 // reach the FSMs. This is used to qualify detection of un-
expected bus events,
92 // where some other I2C controller or target is driving SCL
or SDA
93 // simultaneously.
94 logic bus_event_detect;
95 logic [10:0] bus_event_detect_cnt;
96 logic sda_released_but_low;
97 logic controller_sda_interference;
98 logic target_arbitration_lost;
99
100 logic bus_free;
101 logic start_detect;
102 logic stop_detect;
103
104 logic event_rx_overflow;
105 logic status_controller_halt;
106 logic event_nak;
107 logic event_unhandled_nak_timeout;
108 logic event_controller_arbitration_lost;
109 logic event_scl_interference;
110 logic event_sda_interference;
111 logic event_bus_active_timeout;
112 logic event_stretch_timeout;
113 logic event_sda_unstable;
114 logic event_read_cmd_received;
115 logic event_cmd_complete;
116 logic event_controller_cmd_complete,
event_target_cmd_complete;
117 logic event_target_nack;
118 logic event_tx_arbitration_lost;
119 logic event_tx_bus_timeout;
120 logic event_tx_stretch;
121 logic event_acq_stretch;
122 logic event_unexp_stop;
123 logic event_host_timeout;
124
125 logic unhandled_unexp_nak;
126 logic unhandled_tx_stretch_event;
127 logic target_ack_ctrl_stretching;
128 logic target_ack_ctrl_sw_nack;
129
130 logic [15:0] scl_rx_val;
131 logic [15:0] sda_rx_val;
132
133 logic override;
134
135 logic         fmt_fifo_wvalid;
136 logic         fmt_fifo_wready;
137 logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_wdata;
138 logic [FifoDepthW-1:0]     fmt_fifo_depth;
139 logic         fmt_fifo_rvalid;
140 logic         fmt_fifo_rready;

```

```

141 logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_rdata;
142 logic [7:0] fmt_byte;
143 logic fmt_flag_start_before;
144 logic fmt_flag_stop_after;
145 logic fmt_flag_read_bytes;
146 logic fmt_flag_read_continue;
147 logic fmt_flag_nak_ok;
148
149 logic i2c_fifo_rxrst;
150 logic i2c_fifo_fmtrst;
151 logic [MaxFifoDepthW-1:0] i2c_fifo_rx_thresh;
152 logic [MaxFifoDepthW-1:0] i2c_fifo_fmt_thresh;
153
154 logic rx_fifo_wvalid;
155 logic rx_fifo_wready;
156 logic [RX_FIFO_WIDTH-1:0] rx_fifo_wdata;
157 logic [FifoDepthW-1:0] rx_fifo_depth;
158 logic rx_fifo_rvalid;
159 logic rx_fifo_rready;
160 logic [RX_FIFO_WIDTH-1:0] rx_fifo_rdata;
161
162 // FMT FIFO level below programmed threshold?
163 logic fmt_lt_threshold;
164 // Rx FIFO level above programmed threshold?
165 logic rx_gt_threshold;
166
167 logic tx_fifo_wvalid;
168 logic tx_fifo_wready;
169 logic [TX_FIFO_WIDTH-1:0] tx_fifo_wdata;
170 logic [FifoDepthW-1:0] tx_fifo_depth;
171 logic tx_fifo_rvalid;
172 logic tx_fifo_rready;
173 logic [TX_FIFO_WIDTH-1:0] tx_fifo_rdata;
174
175 logic acq_fifo_wvalid;
176 logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_wdata;
177 logic [AcqFifoDepthW-1:0] acq_fifo_depth;
178 logic acq_fifo_full;
179 logic acq_fifo_rvalid;
180 logic acq_fifo_rready;
181 logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_rdata;
182
183 logic i2c_fifo_txrst;
184 logic i2c_fifo_acqrst;
185 logic [MaxFifoDepthW-1:0] i2c_fifo_tx_thresh;
186 logic [MaxFifoDepthW-1:0] i2c_fifo_acq_thresh;
187
188 // Tx FIFO level below programmed threshold?
189 logic tx_lt_threshold;
190 // ACQ FIFO level above programmed threshold?
191 logic acq_gt_threshold;
192
193 logic host_idle;
194 logic target_idle;
195
196 logic host_enable;
197 logic target_enable;
198 logic line_loopback;
199 logic target_loopback;
200
201 logic [6:0] target_address0;
202 logic [6:0] target_mask0;
203 logic [6:0] target_address1;
204 logic [6:0] target_mask1;
205
206 // Unused parts of exposed bits
207 logic unused_rx_thr_qe;
208 logic unused_fmt_thr_qe;
209 logic unused_tx_thr_qe;
210 logic unused_acq_thr_qe;
211 logic [7:0] unused_rx_fifo_rdata_q;
212 logic [7:0] unused_acq_fifo_adata_q;
213 logic [ACQ_FIFO_WIDTH-9:0] unused_acq_fifo_signal_q;
214 logic unused_alert_test_qe;
215 logic unused_alert_test_q;
216
217 assign hw2reg.status.fmtfull.d = ~fmt_fifo_wready;
218 assign hw2reg.status.rxfull.d = ~rx_fifo_wready;
219 assign hw2reg.status.fmtempty.d = ~fmt_fifo_rvalid;
220 assign hw2reg.status.hostidle.d = host_idle;
221 assign hw2reg.status.targetidle.d = target_idle;
222 assign hw2reg.status.rxempty.d = ~rx_fifo_rvalid;
223 assign hw2reg.status.ack_ctrl_stretch.d = target_ack_ctrl_stretching;
224
225 assign hw2reg.rdata.d = rx_fifo_rdata;
226 assign hw2reg.host_fifo_status.fmtlvl.d = MaxFifoDepthW'(fmt_fifo_depth);
227 assign hw2reg.host_fifo_status.rxlvl.d = MaxFifoDepthW'(rx_fifo_depth);
228 assign hw2reg.val.scl_rx.d = scl_rx_val;
229 assign hw2reg.val.sda_rx.d = sda_rx_val;
230
231 assign hw2reg.status.txfull.d = ~tx_fifo_wready;
232 assign hw2reg.status.acqfull.d = acq_fifo_full;
233 assign hw2reg.status.txempty.d = ~tx_fifo_rvalid;
234 assign hw2reg.status.acqempty.d = ~acq_fifo_rready;
235 assign hw2reg.target_fifo_status.txlvl.d = MaxFifoDepthW'(tx_fifo_depth);
236 assign hw2reg.target_fifo_status.acqlvl.d = MaxFifoDepthW'(acq_fifo_depth);
237 assign hw2reg.acqdata.abyte.d = acq_fifo_rdata[7:0];
238 assign hw2reg.acqdata.signal.d = acq_fifo_rdata[ACQ_FIFO_WIDTH-1:8];
239
240 // Add one to the target NACK count if this target has sent a NACK and if
241 // counter has not saturated.
242 assign hw2reg.target_nack_count.de = event_target_nack && (reg2hw.target_nack_count.q < 8'hFF);
243 assign hw2reg.target_nack_count.d = reg2hw.target_nack_count.q + 1;
244
245 assign override = reg2hw.ovrd.txovrden;
246
247 assign scl_o = override ? reg2hw.ovrd.sclval : scl_out_fsm;
248 assign sda_o = override ? reg2hw.ovrd.sdaval : sda_out_fsm;
249
250 assign host_enable = reg2hw.ctrl.enablehost.q;
251 assign target_enable = reg2hw.ctrl.enabletarget.q;
252 assign line_loopback = reg2hw.ctrl.llpbk.q;
253
254 assign event_cmd_complete = event_controller_cmd_complete | event_target_cmd_complete;
255
256 // Target loopback simply plays back whatever is received from the external host
257 // back to it.
258 assign target_loopback = target_enable & line_loopback;
259
260 assign target_address0 = reg2hw.target_id.address0.q;
261 assign target_mask0 = reg2hw.target_id.mask0.q;
262 assign target_address1 = reg2hw.target_id.address1.q;
263 assign target_mask1 = reg2hw.target_id.mask1.q;
264
265 // Flop I2C bus outputs
266 always_ff @(posedge clk_i or negedge rst_ni) begin
267   if (!rst_ni) begin
268     scl_out_fsm <= 1'b1;
269     sda_out_fsm <= 1'b1;
270   end else begin
271     // Drive 0 if any FSM requests it.
272     scl_out_fsm <= scl_out_controller_fsm & scl_out_target_fsm;
273     sda_out_fsm <= sda_out_controller_fsm & sda_out_target_fsm;
274   end
275 end
276
277 // Sample scl_i and sda_i at system clock
278 always_ff @(posedge clk_i or negedge rst_ni) begin : rx_oversampling
279   if (!rst_ni) begin
280     scl_rx_val <= 16'h0;
281     sda_rx_val <= 16'h0;
282   end else begin
283     scl_rx_val <= {scl_rx_val[14:0], scl_sync};
284     sda_rx_val <= {sda_rx_val[14:0], sda_sync};
285   end
286 end
287
288 assign thigh = reg2hw.timing0.thigh.q;
289 assign tlow = reg2hw.timing0.tlow.q;
290 assign t_r = 13'(reg2hw.timing1.t_r.q);
291 assign t_f = 13'(reg2hw.timing1.t_f.q);
292 assign tsu_sta = reg2hw.timing2.tsu_sta.q;
293 assign thd_sta = reg2hw.timing2.thd_sta.q;
294 assign tsu_dat = 13'(reg2hw.timing3.tsu_dat.q);
295 assign thd_dat = reg2hw.timing3.thd_dat.q;
296 assign tsu_sto = reg2hw.timing4.tsu_sto.q;
297 assign t_buf = reg2hw.timing4.t_buf.q;
298 assign bus_active_timeout = reg2hw.timeout_ctrl.val.q;
299 assign stretch_timeout_enable = reg2hw.timeout_ctrl.en.q && (reg2hw.timeout_ctrl.mode.q == StretchTimeoutMode);
300
301 assign bus_timeout_enable = reg2hw.timeout_ctrl.en.q && (reg2hw.timeout_ctrl.mode.q == BusTimeoutMode);
302
303 assign host_timeout = reg2hw.host_timeout_ctrl.q;
304 assign nack_timeout = reg2hw.target_timeout_ctrl.val.q;
305 assign nack_timeout_en = reg2hw.target_timeout_ctrl.en.q;
306 assign host_nack_handler_timeout = reg2hw.host_nack_handler_timeout.val.q;
307 assign host_nack_handler_timeout_en = reg2hw.host_nack_handler_timeout.en.q;
308 assign target_ack_ctrl_sw_nack = reg2hw.target_ack_ctrl.nack.qe &

```



```

309 reg2hw.target_ack_ctrl.nack.q;
310
311 assign i2c_fifo_rxrst = reg2hw.fifo_ctrl.rxrst.q &
reg2hw.fifo_ctrl.rxrst.qe;
312 assign i2c_fifo_fmtrst = reg2hw.fifo_ctrl.fmtrst.q &
reg2hw.fifo_ctrl.fmtrst.qe;
313 assign i2c_fifo_rx_thresh =
reg2hw.host_fifo_config.rx_thresh.q;
314 assign i2c_fifo_fmt_thresh =
reg2hw.host_fifo_config.fmt_thresh.q;
315
316 assign i2c_fifo_txrst = reg2hw.fifo_ctrl.txrst.q &
reg2hw.fifo_ctrl.txrst.qe;
317 assign i2c_fifo_acqrst = reg2hw.fifo_ctrl.acqrst.q &
reg2hw.fifo_ctrl.acqrst.qe;
318 assign i2c_fifo_tx_thresh =
reg2hw.target_fifo_config.tx_thresh.q;
319 assign i2c_fifo_acq_thresh =
reg2hw.target_fifo_config.acq_thresh.q;
320
321 // FMT FIFO level below programmed threshold?
322 assign fmt_lt_threshold = (MaxFifoDepthW'(fmt_fifo_depth) <
i2c_fifo_fmt_thresh);
323 // Rx FIFO level above programmed threshold?
324 assign rx_gt_threshold = (MaxFifoDepthW'(rx_fifo_depth) >
i2c_fifo_rx_thresh);
325 // Tx FIFO level below programmed threshold?
326 assign tx_lt_threshold = (MaxFifoDepthW'(tx_fifo_depth) <
i2c_fifo_tx_thresh);
327 // ACQ FIFO level above programmed threshold?
328 assign acq_gt_threshold = (MaxFifoDepthW'(acq_fifo_depth) >
i2c_fifo_acq_thresh);
329
330 assign lsio_trigger_o = rx_gt_threshold;
331 assign event_rx_overflow = rx_fifo_wvalid & ~rx_fifo_wready;
332 assign event_acq_stretch = acq_fifo_full || tar-
get_ack_ctrl.stretching;
333
334 // The fifo write enable is controlled by fbyte, start,
stop, read, rcont,
335 // and nakok field qe bits.
336 // When all qe bits are asserted, fdata is injected into the
fifo.
337 assign fmt_fifo_wvalid = reg2hw.fdata.fbyte.qe &
reg2hw.fdata.start.qe &
338 reg2hw.fdata.stop.qe &
339 reg2hw.fdata.readb.qe &
340 reg2hw.fdata.rcont.qe &
341 reg2hw.fdata.nakok.qe;
342
343 assign fmt_fifo_wdata[7:0] = reg2hw.fdata.fbyte.q;
344 assign fmt_fifo_wdata[8] = reg2hw.fdata.start.q;
345 assign fmt_fifo_wdata[9] = reg2hw.fdata.stop.q;
346 assign fmt_fifo_wdata[10] = reg2hw.fdata.readb.q;
347 assign fmt_fifo_wdata[11] = reg2hw.fdata.rcont.q;
348 assign fmt_fifo_wdata[12] = reg2hw.fdata.nakok.q;
349
350 assign fmt_byte = fmt_fifo_rvalid ?
fmt_fifo_rdata[7:0] : '0;
351 assign fmt_flag_start_before = fmt_fifo_rvalid ?
fmt_fifo_rdata[8] : '0;
352 assign fmt_flag_stop_after = fmt_fifo_rvalid ?
fmt_fifo_rdata[9] : '0;
353 assign fmt_flag_read_bytes = fmt_fifo_rvalid ?
fmt_fifo_rdata[10] : '0;
354 assign fmt_flag_read_continue = fmt_fifo_rvalid ?
fmt_fifo_rdata[11] : '0;
355 assign fmt_flag_nak_ok = fmt_fifo_rvalid ?
fmt_fifo_rdata[12] : '0;
356
357 // Operating this HWIP as a controller-transmitter, the ad-
dressed Target device
358 // may NACK our bytes. In Byte-Formatted Programming Mode,
each FDATA format indicator
359 // can set the 'NAKOK' bit to ignore the Target's NACK and
proceed to the next item in
360 // the FMTFIFO. If 'NAKOK' is not set, the 'controller_halt'
interrupt is asserted, and the FSM
361 // halts until software intervenes. In addition, the 'con-
troller_events.nack' status bit is set.
362 // To acknowledge the 'NACK', software should clear the
status bit by writing 1 to it in the
363 // CONTROLLER_EVENTS register.
364 assign unhandled_unexp_nak =
reg2hw.controller_events.nack.q;
365
366 assign unused_rx_thr_qe =
reg2hw.host_fifo_config.rx_thresh.qe;
367 assign unused_fmt_thr_qe =
reg2hw.host_fifo_config.fmt_thresh.qe;
368 assign unused_tx_thr_qe =
reg2hw.target_fifo_config.tx_thresh.qe;
369 assign unused_acq_thr_qe =
reg2hw.target_fifo_config.acq_thresh.qe;
370 assign unused_rx_fifo_rdata_q = reg2hw.rdata.q;
371 assign unused_acq_fifo_adata_q = reg2hw.acqdata.abyte.q;
372 assign unused_acq_fifo_signal_q = reg2hw.acqdata.signal.q;
373 assign unused_alert_test_qe = reg2hw.alert_test.qe;
374 assign unused_alert_test_q = reg2hw.alert_test.q;

```

```

375
376 i2c_fifos u_fifos (
377 .clk_i,
378 .rst_ni,
379 .ram_cfg_i,
380 .ram_cfg_o,
381
382 .fmt_fifo_clr_i (i2c_fifo_fmtrst),
383 .fmt_fifo_depth_o (fmt_fifo_depth),
384 .fmt_fifo_wvalid_i (fmt_fifo_wvalid),
385 .fmt_fifo_wready_o (fmt_fifo_wready),
386 .fmt_fifo_wdata_i (fmt_fifo_wdata),
387 .fmt_fifo_rvalid_o (fmt_fifo_rvalid),
388 .fmt_fifo_rready_i (fmt_fifo_rready),
389 .fmt_fifo_rdata_o (fmt_fifo_rdata),
390
391 .rx_fifo_clr_i (i2c_fifo_rxrst),
392 .rx_fifo_depth_o (rx_fifo_depth),
393 .rx_fifo_wvalid_i (rx_fifo_wvalid),
394 .rx_fifo_wready_o (rx_fifo_wready),
395 .rx_fifo_wdata_i (rx_fifo_wdata),
396 .rx_fifo_rvalid_o (rx_fifo_rvalid),
397 .rx_fifo_rready_i (rx_fifo_rready),
398 .rx_fifo_rdata_o (rx_fifo_rdata),
399
400 .tx_fifo_clr_i (i2c_fifo_txrst),
401 .tx_fifo_depth_o (tx_fifo_depth),
402 .tx_fifo_wvalid_i (tx_fifo_wvalid),
403 .tx_fifo_wready_o (tx_fifo_wready),
404 .tx_fifo_wdata_i (tx_fifo_wdata),
405 .tx_fifo_rvalid_o (tx_fifo_rvalid),
406 .tx_fifo_rready_i (tx_fifo_rready),
407 .tx_fifo_rdata_o (tx_fifo_rdata),
408
409 .acq_fifo_clr_i (i2c_fifo_acqrst),
410 .acq_fifo_depth_o (acq_fifo_depth),
411 .acq_fifo_wvalid_i (acq_fifo_wvalid),
412 .acq_fifo_wready_o (),
413 .acq_fifo_wdata_i (acq_fifo_wdata),
414 .acq_fifo_rvalid_o (acq_fifo_rvalid),
415 .acq_fifo_rready_i (acq_fifo_rready),
416 .acq_fifo_rdata_o (acq_fifo_rdata)
417 );
418
419 assign rx_fifo_rready = reg2hw.rdata.re;
420
421 // Need to add a valid qualification to write only payload
bytes
422 logic valid_target_lb_wr;
423 i2c_acq_byte_id_e acq_type;
424 assign acq_type =
i2c_acq_byte_id_e'(acq_fifo_rdata[ACQ_FIFO_WIDTH-1:8]);
425
426 assign valid_target_lb_wr = target_enable & (acq_type == Ac-
qData);
427
428 // only write into tx fifo if it's payload
429 assign tx_fifo_wvalid = target_loopback ? acq_fifo_rvalid &
valid_target_lb_wr : reg2hw.txdata.qe;
430 assign tx_fifo_wdata = target_loopback ?
acq_fifo_rdata[7:0] : reg2hw.txdata.q;
431
432 // During line loopback, pop from acquisition fifo only when
there is space in
433 // the tx_fifo. We are also allowed to pop even if there is
no space if th acq entry
434 // is not data payload.
435 assign acq_fifo_rready = (reg2hw.acqdata.abyte.re &
reg2hw.acqdata.signal.re) |
436 (target_loopback & (tx_fifo_wready
| (acq_type != AcqData)));
437
438 // sync the incoming SCL and SDA signals
439 prim_flop_2sync #(
440 .Width(1),
441 .ResetValue(1'b1)
442 ) u_i2c_sync_scl (
443 .clk_i,
444 .rst_ni,
445 .d_i (scl_i),
446 .q_o (scl_sync)
447 );
448
449 prim_flop_2sync #(
450 .Width(1),
451 .ResetValue(1'b1)
452 ) u_i2c_sync_sda (
453 .clk_i,
454 .rst_ni,
455 .d_i (sda_i),
456 .q_o (sda_sync)
457 );
458
459 // Various bus collision events are detected while SCL is
high.
460 logic sda_fsm, sda_fsm_q;
461 logic scl_fsm, scl_fsm_q;
462 assign sda_fsm = sda_out_controller_fsm & sda_out_tar-
get_fsm;

```

```

463 assign scl_fsm = scl_out_controller_fsm & scl_out_tar-
get_fsm;
464
465 always_ff @(posedge clk_i or negedge rst_ni) begin
466   if (!rst_ni) begin
467     sda_fsm_q <= 1'b1;
468     scl_fsm_q <= 1'b1;
469   end else begin
470     sda_fsm_q <= sda_fsm;
471     scl_fsm_q <= scl_fsm;
472   end
473 end
474
475 always_ff @(posedge clk_i or negedge rst_ni) begin
476   if (!rst_ni) begin
477     bus_event_detect_cnt <= '1;
478   end else if ((scl_fsm != scl_fsm_q) || (sda_fsm !=
sda_fsm_q)) begin
479     // Wait for the round-trip time on SCL changes or on SDA
changes. The
480     // latter handles Start and Stop conditions, so changes
while SCL is high
481     // are allowed to propagate. The rise time is used here
because it
482     // should be the longer value. The (-1) term here is to
account for the
483     // delay in the counter.
484     // Note that there are limits to this method of detect-
ing arbitration.
485     // A separate, buggy device that drives clock low faster
than the counter
486     // can expire would not trigger loss of arbitration.
487     bus_event_detect_cnt <= reg2hw.timing1.t_r_q +
10'(RoundTripCycles - 1);
488   end else if (bus_event_detect_cnt != '0) begin
489     bus_event_detect_cnt <= bus_event_detect_cnt - 1'b1;
490   end
491 end
492 assign bus_event_detect = (bus_event_detect_cnt == '0);
493 assign sda_released_but_low = bus_event_detect && scl_sync
&& (sda_fsm_q != sda_sync);
494 // What about unexpected start / stop on the bits that are
read?
495 assign controller_sda_interference = controller_transmitting
&& sda_released_but_low;
496 assign target_arbitration_lost = target_transmitting &&
sda_released_but_low;
497
498 assign event_sda_interference = controller_sda_interference;
499
500 // The bus monitor detects starts, stops, and bus timeouts.
It also reports
501 // when the bus is free for the controller to transmit.
502 i2c_bus_monitor u_i2c_bus_monitor (
503   .clk_i,
504   .rst_ni,
505   .scl_i
506     (scl_sync),
507   .sda_i
508     (sda_sync),
509   .controller_enable_i
510     (host_enable),
511   .multi_controller_enable_i
512     (reg2hw.ctrl.multi_controller_monitor_en.q),
513   .target_enable_i
514     (target_enable),
515   .target_idle_i
516     (target_idle),
517   .thd_dat_i
518     (thd_dat),
519   .t_buf_i
520     (t_buf),
521   .bus_active_timeout_i
522     (bus_active_timeout),
523   .bus_active_timeout_en_i
524     (bus_timeout_enable),
525   .bus_inactive_timeout_i
526     (host_timeout),
527   .bus_free_o
528     (bus_free),
529   .start_detect_o
530     (start_detect),
531   .stop_detect_o
532     (stop_detect),
533   .event_bus_active_timeout_o
534     (event_bus_active_timeout),
535   .event_host_timeout_o
536     (event_host_timeout)
537 );
538
539 i2c_controller_fsm #(
540   .FifoDepth(FifoDepth)
541 ) u_i2c_controller_fsm (
542   .clk_i,
543   .rst_ni,
544   .scl_i
545     (scl_sync),
546   .scl_o
547     (scl_out_controller_fsm),
548   .sda_i
549     (sda_sync),
550   .sda_o
551     (sda_out_controller_fsm),
552   .bus_free_i
553     (bus_free),
554   .transmitting_o
555     (controller_transmitting),
556   .host_enable_i
557     (host_enable),
558   .halt_controller_i
559     (status_controller_halt),
560   .fmt_fifo_rvalid_i
561     (fmt_fifo_rvalid),
562   .fmt_fifo_depth_i
563     (fmt_fifo_depth),

```

```

564   .fmt_fifo_rready_o
565     (fmt_fifo_rready),
566   .fmt_byte_i
567     (fmt_byte),
568   .fmt_flag_start_before_i
569     (fmt_flag_start_before),
570   .fmt_flag_stop_after_i
571     (fmt_flag_stop_after),
572   .fmt_flag_read_bytes_i
573     (fmt_flag_read_bytes),
574   .fmt_flag_read_continue_i
575     (fmt_flag_read_continue),
576   .fmt_flag_nak_ok_i
577     (fmt_flag_nak_ok),
578   .unhandled_unexp_nak_i
579     (unhandled_unexp_nak),
580   .unhandled_nak_timeout_i
581     (reg2hw.controller_events.unhandled_nak_timeout.q),
582   .rx_fifo_wvalid_o
583     (rx_fifo_wvalid),
584   .rx_fifo_wdata_o
585     (rx_fifo_wdata),
586   .host_idle_o
587     (host_idle),
588   .thigh_i
589     (thigh),
590   .tlow_i
591     (tlow),
592   .t_r_i
593     (t_r),
594   .t_f_i
595     (t_f),
596   .thd_sta_i
597     (thd_sta),
598   .tsu_sta_i
599     (tsu_sta),
600   .tsu_sto_i
601     (tsu_sto),
602   .thd_dat_i
603     (thd_dat),
604   .sda_interference_i
605     (control-
ler_sda_interference),
606   .stretch_timeout_i
607     (bus_active_timeout),
608   .timeout_enable_i
609     (stretch_timeout_enable),
610   .host_nack_handler_timeout_i
611     (host_n-
ack_handler_timeout),
612   .host_nack_handler_timeout_en_i
613     (host_n-
ack_handler_timeout_en),
614   .event_nak_o
615     (event_nak),
616   .event_unhandled_nak_timeout_o
617     (event_unhandled_nak_timeout),
618   .event_arbitration_lost_o
619     (event_controller_arbitration_lost),
620   .event_scl_interference_o
621     (event_scl_interference),
622   .event_stretch_timeout_o
623     (event_stretch_timeout),
624   .event_sda_unstable_o
625     (event_sda_unstable),
626   .event_cmd_complete_o
627     (event_controller_cmd_complete)
628 );
629
630 i2c_target_fsm #(
631   .AcqFifoDepth(AcqFifoDepth)
632 ) u_i2c_target_fsm (
633   .clk_i,
634   .rst_ni,
635   .scl_i
636     (scl_sync),
637   .scl_o
638     (scl_out_target_fsm),
639   .sda_i
640     (sda_sync),
641   .sda_o
642     (sda_out_target_fsm),
643   .start_detect_i
644     (start_detect),
645   .stop_detect_i
646     (stop_detect),
647   .transmitting_o
648     (target_transmitting),
649   .target_enable_i
650     (target_enable),
651   .tx_fifo_rvalid_i
652     (tx_fifo_rvalid),
653   .tx_fifo_rready_o
654     (tx_fifo_rready),
655   .tx_fifo_rdata_i
656     (tx_fifo_rdata),
657   .acq_fifo_wvalid_o
658     (acq_fifo_wvalid),
659   .acq_fifo_wdata_o
660     (acq_fifo_wdata),
661   .acq_fifo_rdata_i
662     (acq_fifo_rdata),
663   .acq_fifo_full_o
664     (acq_fifo_full),
665   .acq_fifo_depth_i
666     (acq_fifo_depth),
667   .target_idle_o
668     (target_idle),
669   .t_r_i
670     (t_r),
671   .tsu_dat_i
672     (tsu_dat),
673   .thd_dat_i
674     (thd_dat),
675   .nak_timeout_i
676     (nak_timeout),
677   .nak_timeout_en_i
678     (nak_timeout_en),
679   .nak_addr_after_timeout_i
680     (reg2hw.ctrl.nak_addr_after_timeout.q),
681   .arbitration_lost_i
682     (target_arbitration_lost),
683   .bus_timeout_i
684     (event_bus_active_timeout),
685   .unhandled_tx_stretch_event_i
686     (un-
handled_tx_stretch_event),
687   .ack_ctrl_mode_i
688     (reg2hw.ctrl.ack_ctrl_mode.q),
689   .auto_ack_cnt_o
690     (hw2reg.target_ack_ctrl.nbytes.d),
691   .auto_ack_load_i
692     (reg2hw.target_ack_ctrl.nbytes.qe),
693   .auto_ack_load_value_i
694     (reg2hw.target_ack_ctrl.nbytes.q),
695   .sw_nack_i
696     (target_ack_ctrl_sw_nack),
697   .ack_ctrl_stretching_o
698     (tar-
get_ack_ctrl_stretching),
699   .acq_fifo_next_data_o
700     (hw2reg.acq_fifo_next_data.d),
701   .target_address0_i
702     (target_address0),
703   .target_mask0_i
704     (target_mask0),
705   .target_address1_i
706     (target_address1),

```

```

630 .target_mask1_i          (target_mask1),
631 .event_target_nack_o     (event_target_nack),
632 .event_read_cmd_received_o (event_read_cmd_received),
633 .event_cmd_complete_o
(event_target_cmd_complete),
634 .event_tx_stretch_o      (event_tx_stretch),
635 .event_unexp_stop_o      (event_unexp_stop),
636 .event_tx_arbitration_lost_o
(event_tx_arbitration_lost),
637 .event_tx_bus_timeout_o  (event_tx_bus_timeout)
638 );
639
640 prim_intr_hw #(
641   .Width(1),
642   .IntrT("Status")
643 ) intr_hw_fmt_threshold (
644   .clk_i,
645   .rst_ni,
646   .event_intr_i          (fmt_lt_threshold),
647   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_fmt_threshold.q),
648   .reg2hw_intr_test_q_i
(reg2hw_intr_test_fmt_threshold.q),
649   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_fmt_threshold.qe),
650   .reg2hw_intr_state_q_i
(reg2hw_intr_state_fmt_threshold.q),
651   .hw2reg_intr_state_de_o
(hw2reg_intr_state_fmt_threshold.de),
652   .hw2reg_intr_state_d_o
(hw2reg_intr_state_fmt_threshold.d),
653   .intr_o                (intr_fmt_threshold_o)
654 );
655
656 prim_intr_hw #(
657   .Width(1),
658   .IntrT("Status")
659 ) intr_hw_rx_threshold (
660   .clk_i,
661   .rst_ni,
662   .event_intr_i          (rx_gt_threshold),
663   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_rx_threshold.q),
664   .reg2hw_intr_test_q_i
(reg2hw_intr_test_rx_threshold.q),
665   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_rx_threshold.qe),
666   .reg2hw_intr_state_q_i
(reg2hw_intr_state_rx_threshold.q),
667   .hw2reg_intr_state_de_o
(hw2reg_intr_state_rx_threshold.de),
668   .hw2reg_intr_state_d_o
(hw2reg_intr_state_rx_threshold.d),
669   .intr_o                (intr_rx_threshold_o)
670 );
671
672 prim_intr_hw #(
673   .Width(1),
674   .IntrT("Status")
675 ) intr_hw_acq_threshold (
676   .clk_i,
677   .rst_ni,
678   .event_intr_i          (acq_gt_threshold),
679   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_acq_threshold.q),
680   .reg2hw_intr_test_q_i
(reg2hw_intr_test_acq_threshold.q),
681   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_acq_threshold.qe),
682   .reg2hw_intr_state_q_i
(reg2hw_intr_state_acq_threshold.q),
683   .hw2reg_intr_state_de_o
(hw2reg_intr_state_acq_threshold.de),
684   .hw2reg_intr_state_d_o
(hw2reg_intr_state_acq_threshold.d),
685   .intr_o                (intr_acq_threshold_o)
686 );
687
688 prim_intr_hw #(Width(1)) intr_hw_rx_overflow (
689   .clk_i,
690   .rst_ni,
691   .event_intr_i          (event_rx_overflow),
692   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_rx_overflow.q),
693   .reg2hw_intr_test_q_i
(reg2hw_intr_test_rx_overflow.q),
694   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_rx_overflow.qe),
695   .reg2hw_intr_state_q_i
(reg2hw_intr_state_rx_overflow.q),
696   .hw2reg_intr_state_de_o
(hw2reg_intr_state_rx_overflow.de),
697   .hw2reg_intr_state_d_o
(hw2reg_intr_state_rx_overflow.d),
698   .intr_o                (intr_rx_overflow_o)
699 );
700
701 prim_intr_hw #(
702   .Width(1),
703   .IntrT("Status")
704 ) intr_hw_controller_halt (
705   .clk_i,
706   .rst_ni,
707   .event_intr_i          (status_controller_halt),
708   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_controller_halt.q),

```

```

709   .reg2hw_intr_test_q_i
(reg2hw_intr_test_controller_halt.q),
710   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_controller_halt.qe),
711   .reg2hw_intr_state_q_i
(reg2hw_intr_state_controller_halt.q),
712   .hw2reg_intr_state_de_o
(hw2reg_intr_state_controller_halt.de),
713   .hw2reg_intr_state_d_o
(hw2reg_intr_state_controller_halt.d),
714   .intr_o                (intr_controller_halt_o)
715 );
716
717 prim_intr_hw #(Width(1)) intr_hw_scl_interference (
718   .clk_i,
719   .rst_ni,
720   .event_intr_i          (event_scl_interference),
721   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_scl_interference.q),
722   .reg2hw_intr_test_q_i
(reg2hw_intr_test_scl_interference.q),
723   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_scl_interference.qe),
724   .reg2hw_intr_state_q_i
(reg2hw_intr_state_scl_interference.q),
725   .hw2reg_intr_state_de_o
(hw2reg_intr_state_scl_interference.de),
726   .hw2reg_intr_state_d_o
(hw2reg_intr_state_scl_interference.d),
727   .intr_o                (intr_scl_interference_o)
728 );
729
730 prim_intr_hw #(Width(1)) intr_hw_sda_interference (
731   .clk_i,
732   .rst_ni,
733   .event_intr_i          (event_sda_interference),
734   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_sda_interference.q),
735   .reg2hw_intr_test_q_i
(reg2hw_intr_test_sda_interference.q),
736   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_sda_interference.qe),
737   .reg2hw_intr_state_q_i
(reg2hw_intr_state_sda_interference.q),
738   .hw2reg_intr_state_de_o
(hw2reg_intr_state_sda_interference.de),
739   .hw2reg_intr_state_d_o
(hw2reg_intr_state_sda_interference.d),
740   .intr_o                (intr_sda_interference_o)
741 );
742
743 prim_intr_hw #(Width(1)) intr_hw_stretch_timeout (
744   .clk_i,
745   .rst_ni,
746   .event_intr_i          (event_stretch_timeout),
747   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_stretch_timeout.q),
748   .reg2hw_intr_test_q_i
(reg2hw_intr_test_stretch_timeout.q),
749   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_stretch_timeout.qe),
750   .reg2hw_intr_state_q_i
(reg2hw_intr_state_stretch_timeout.q),
751   .hw2reg_intr_state_de_o
(hw2reg_intr_state_stretch_timeout.de),
752   .hw2reg_intr_state_d_o
(hw2reg_intr_state_stretch_timeout.d),
753   .intr_o                (intr_stretch_timeout_o)
754 );
755
756 prim_intr_hw #(Width(1)) intr_hw_sda_unstable (
757   .clk_i,
758   .rst_ni,
759   .event_intr_i          (event_sda_unstable),
760   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_sda_unstable.q),
761   .reg2hw_intr_test_q_i
(reg2hw_intr_test_sda_unstable.q),
762   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_sda_unstable.qe),
763   .reg2hw_intr_state_q_i
(reg2hw_intr_state_sda_unstable.q),
764   .hw2reg_intr_state_de_o
(hw2reg_intr_state_sda_unstable.de),
765   .hw2reg_intr_state_d_o
(hw2reg_intr_state_sda_unstable.d),
766   .intr_o                (intr_sda_unstable_o)
767 );
768
769 prim_intr_hw #(Width(1)) intr_hw_cmd_complete (
770   .clk_i,
771   .rst_ni,
772   .event_intr_i          (event_cmd_complete),
773   .reg2hw_intr_enable_q_i
(reg2hw_intr_enable_cmd_complete.q),
774   .reg2hw_intr_test_q_i
(reg2hw_intr_test_cmd_complete.q),
775   .reg2hw_intr_test_qe_i
(reg2hw_intr_test_cmd_complete.qe),
776   .reg2hw_intr_state_q_i
(reg2hw_intr_state_cmd_complete.q),
777   .hw2reg_intr_state_de_o
(hw2reg_intr_state_cmd_complete.de),

```

```

778 .hw2reg_intr_state_d_o
(hw2reg.intr_state.cmd_complete.d),
779 .intr_o (intr_cmd_complete_o)
780 );
781
782 prim_intr_hw #(
783 .Width(1),
784 .IntrT("Status")
785 ) intr_hw_tx_stretch (
786 .clk_i,
787 .rst_ni,
788 .event_intr_i (event_tx_stretch),
789 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_stretch.q),
790 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_stretch.q),
791 .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_stretch.qe),
792 .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_stretch.q),
793 .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_stretch.de),
794 .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_stretch.d),
795 .intr_o (intr_tx_stretch_o)
796 );
797
798 prim_intr_hw #(
799 .Width(1),
800 .IntrT("Status")
801 ) intr_hw_tx_threshold (
802 .clk_i,
803 .rst_ni,
804 .event_intr_i (tx_lt_threshold),
805 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_threshold.q),
806 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_threshold.q),
807 .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_threshold.qe),
808 .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_threshold.q),
809 .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_threshold.de),
810 .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_threshold.d),
811 .intr_o (intr_tx_threshold_o)
812 );
813
814 prim_intr_hw #(
815 .Width(1),
816 .IntrT("Status")
817 ) intr_hw_acq_overflow (
818 .clk_i,
819 .rst_ni,
820 .event_intr_i (event_acq_stretch),
821 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.acq_stretch.q),
822 .reg2hw_intr_test_q_i (reg2hw.intr_test.acq_stretch.q),
823 .reg2hw_intr_test_qe_i (reg2hw.intr_test.acq_stretch.qe),
824 .reg2hw_intr_state_q_i (reg2hw.intr_state.acq_stretch.q),
825 .hw2reg_intr_state_de_o (hw2reg.intr_state.acq_stretch.de),
826 .hw2reg_intr_state_d_o (hw2reg.intr_state.acq_stretch.d),
827 .intr_o (intr_acq_stretch_o)
828 );
829
830 prim_intr_hw #(.Width(1)) intr_hw_unexp_stop (
831 .clk_i,
832 .rst_ni,
833 .event_intr_i (event_unexp_stop),
834 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.unexp_stop.q),
835 .reg2hw_intr_test_q_i (reg2hw.intr_test.unexp_stop.q),
836 .reg2hw_intr_test_qe_i (reg2hw.intr_test.unexp_stop.qe),
837 .reg2hw_intr_state_q_i (reg2hw.intr_state.unexp_stop.q),
838 .hw2reg_intr_state_de_o (hw2reg.intr_state.unexp_stop.de),
839 .hw2reg_intr_state_d_o (hw2reg.intr_state.unexp_stop.d),
840 .intr_o (intr_unexp_stop_o)
841 );
842
843 prim_intr_hw #(.Width(1)) intr_hw_host_timeout (
844 .clk_i,
845 .rst_ni,
846 .event_intr_i (event_host_timeout),
847 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.host_timeout.q),
848 .reg2hw_intr_test_q_i (reg2hw.intr_test.host_timeout.q),
849 .reg2hw_intr_test_qe_i (reg2hw.intr_test.host_timeout.qe),
850 .reg2hw_intr_state_q_i (reg2hw.intr_state.host_timeout.q),
851 .hw2reg_intr_state_de_o (hw2reg.intr_state.host_timeout.de),
852 .hw2reg_intr_state_d_o (hw2reg.intr_state.host_timeout.d),
853 .intr_o (intr_host_timeout_o)
854 );
855
856 assign hw2reg.controller_events.nack.d = 1'b1;
857 assign hw2reg.controller_events.nack.de = event_nak;
858 assign hw2reg.controller_events.unhandled_nack_timeout.d = 1'b1;
859 assign hw2reg.controller_events.unhandled_nack_timeout.de = event_unhandled_nak_timeout;
860 assign hw2reg.controller_events.bus_timeout.d = 1'b1;
861 assign hw2reg.controller_events.bus_timeout.de = event_bus_active_timeout & !host_idle;
862 assign hw2reg.controller_events.arbitration_lost.d = 1'b1;

```

```

863 assign hw2reg.controller_events.arbitration_lost.de = event_controller_arbitration_lost;
864 assign status_controller_halt = | {
865 reg2hw.controller_events.nack.q,
866 reg2hw.controller_events.unhandled_nack_timeout.q,
867 reg2hw.controller_events.bus_timeout.q,
868 reg2hw.controller_events.arbitration_lost.q
869 };
870
871 assign hw2reg.target_events.tx_pending.d = 1'b1;
872 assign hw2reg.target_events.tx_pending.de = event_read_cmd_received &&
873 reg2hw.ctrl.tx_stretch_ctrl_en.q;
874 assign hw2reg.target_events.bus_timeout.d = 1'b1;
875 assign hw2reg.target_events.bus_timeout.de = event_tx_bus_timeout;
876 assign hw2reg.target_events.arbitration_lost.d = 1'b1;
877 assign hw2reg.target_events.arbitration_lost.de = event_tx_arbitration_lost;
878 assign unhandled_tx_stretch_event = | {
879 reg2hw.target_events.tx_pending.q,
880 reg2hw.target_events.bus_timeout.q,
881 reg2hw.target_events.arbitration_lost.q
882 };
883
884 ///////////////////////////////////////////////////////////////////
885 // ASSERTIONS //
886 ///////////////////////////////////////////////////////////////////
887
888 `ASSERT_INIT(FifoDepthValid_A, FifoDepth > 0 && FifoDepthW
<= MaxFifoDepthW)
889 `ASSERT_INIT(AcqFifoDepthValid_A, AcqFifoDepth > 0 && AcqFifoDepthW
<= MaxFifoDepthW)
890
891 endmodule

```

5. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo_sync_sram_adapter.sv (268 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This adapter module allows using a single-port SRAM like a
6 // FIFO. It has been written for I2C,
7 // but it could later be generalized into a prim.
8
9 `include "prim_assert.sv"
10
11 module i2c_fifo_sync_sram_adapter #(
12     parameter int unsigned Width = 1,
13     // Total depth of the FIFO (in number of entries), including
14     // the output buffer in this module.
15     // Must be at least 3.
16     parameter int unsigned Depth = 3,
17     // Address width of the SRAM
18     parameter int unsigned SramAw = 1,
19     // Base address of the FIFO in the SRAM
20     parameter logic [SramAw-1:0] SramBaseAddr = '0,
21     // Derived parameters; don't override!
22     localparam int unsigned DepthW = prim_util_pkg::vbits(Depth
+ 1)
23 ) (
24     input logic clk_i,
25     input logic rst_ni,
26     input logic clr_i,
27
28     // FIFO interface
29     input logic fifo_wvalid_i,
30     output logic fifo_wready_o,
31     input logic [Width-1:0] fifo_wdata_i,
32     output logic fifo_rvalid_o,
33     input logic fifo_rready_i,
34     output logic [Width-1:0] fifo_rdata_o,
35     output logic [DepthW-1:0] fifo_depth_o,
36
37     // SRAM interface. When this module reads from the SRAM, it
38     // expects the read data in the next
39     // cycle.
40     output logic sram_req_o,
41     input logic sram_gnt_i,
42     output logic sram_write_o,
43     output logic [SramAw-1:0] sram_addr_o,
44     output logic [Width-1:0] sram_wdata_o,
45     output logic [Width-1:0] sram_wmask_o,
46     input logic [Width-1:0] sram_rdata_i,
47     input logic sram_rvalid_i,
48
49     output logic err_o
50 );
51
52 // Depth of the input buffer.
53 localparam int unsigned InpBufDepth = 2;

```

```

53 // Depth of the output buffer. To support a read in the
cycle after N consecutive cycles in which
54 // this module simultaneously got written and read, the
depth needs to be N + 1 (one entry extra
55 // to cover the single-cycle latency of SRAM reads). We
currently support this for 1 cycle.
56 localparam int unsigned OupBufDepth = 2;
57
58 // Depth of the SRAM-stored FIFO; simply the entries that
don't fit into the output buffer.
59 localparam int SramFifoDepth = Depth - OupBufDepth;
60
61 // While it would theoretically be possible that the output
buffer is as deep as the entire FIFO,
62 // the current implementation doesn't support this.
63 `ASSERT_INIT(MinimalSramFifoDepth_A, SramFifoDepth >= 1)
64
65 localparam int unsigned InpBufDepthW =
prim_util_pkg::vbits(InpBufDepth + 1);
66 localparam int unsigned OupBufDepthW =
prim_util_pkg::vbits(OupBufDepth + 1);
67 localparam int unsigned SramPtrW =
prim_util_pkg::vbits(SramFifoDepth);
68 localparam int unsigned SramDepthW =
prim_util_pkg::vbits(SramFifoDepth + 1);
69
70 `ASSERT_INIT(MinimalSramAw_A, SramAw >= SramPtrW)
71
72 localparam int unsigned SramAddrLeadingZeros = SramAw -
SramPtrW;
73
74 // Input buffer
75 logic                               inp_buf_wvalid, inp_buf_wready,
76                               inp_buf_rvalid, inp_buf_rready,
77                               inp_buf_err;
78 logic [Width-1:0]               inp_buf_rdata;
79 logic [InpBufDepthW-1:0] inp_buf_depth;
80 prim_fifo_sync #(
81   .Width(Width),
82   .Pass (1'b1),
83   .Depth(InpBufDepth)
84 ) u_inp_buf (
85   .clk_i,
86   .rst_ni,
87   .clr_i,
88   .wvalid_i(inp_buf_wvalid),
89   .wready_o(inp_buf_wready),
90   .wdata_i (fifo_wdata_i),
91   .rvalid_o(inp_buf_rvalid),
92   .rready_i(inp_buf_rready),
93   .rdata_o (inp_buf_rdata),
94   .full_o  (* unused *),
95   .depth_o (inp_buf_depth),
96   .err_o   (inp_buf_err)
97 );
98
99 // Output buffer
100 logic                               oup_buf_wvalid, oup_buf_wready,
oup_buf_full, oup_buf_almost_full,
101                               oup_buf_err;
102 logic [Width-1:0]               oup_buf_wdata;
103 logic [OupBufDepthW-1:0] oup_buf_depth;
104 prim_fifo_sync #(
105   .Width(Width),
106   .Pass (1'b1),
107   .Depth(OupBufDepth)
108 ) u_oup_buf (
109   .clk_i,
110   .rst_ni,
111   .clr_i,
112   .wvalid_i(oup_buf_wvalid),
113   .wready_o(oup_buf_wready),
114   .wdata_i (oup_buf_wdata),
115   .rvalid_o(fifo_rvalid_o),
116   .rready_i(fifo_rready_i),
117   .rdata_o (fifo_rdata_o),
118   .full_o  (oup_buf_full),
119   .depth_o (oup_buf_depth),
120   .err_o   (oup_buf_err)
121 );
122 assign inp_buf_wvalid = fifo_wvalid_i && fifo_wready_o;
123 assign oup_buf_almost_full = oup_buf_depth >= OupBuf-
DepthW'(OupBufDepth - 1);
124
125 // Signal whether we access the SRAM in this cycle
126 logic sram_access;
127 assign sram_access = sram_req_o && sram_gnt_i;
128
129 // SRAM read and write addresses
130 logic [SramAw-1:0] sram_wr_addr, sram_rd_addr;
131 logic [SramPtrW-1:0] sram_wr_ptr, sram_rd_ptr;
132 logic [SramDepthW-1:0] sram_depth;
133 logic sram_incr_wr_ptr, sram_incr_rd_ptr;
134 logic sram_full, sram_empty;
135 logic sram_ptrs_err;
136 prim_fifo_sync_cnt #(
137   .Depth (SramFifoDepth),
138   .Secure(1'b0)
139 ) u_sram_ptrs (
140   .clk_i,
141   .rst_ni,

```

```

142   .clr_i,
143   .incr_wptr_i(sram_incr_wr_ptr),
144   .incr_rptr_i(sram_incr_rd_ptr),
145   .wptr_o      (sram_wr_ptr),
146   .rptr_o      (sram_rd_ptr),
147   .full_o      (sram_full),
148   .empty_o     (sram_empty),
149   .depth_o     (sram_depth),
150   .err_o       (sram_ptrs_err)
151 );
152 assign sram_incr_wr_ptr = sram_access && sram_write_o;
153 assign sram_incr_rd_ptr = sram_access && !sram_write_o;
154 if (SramAddrLeadingZeros > 0) begin :
gen_zero_extend_sram_addrs
155   assign sram_wr_addr = {{SramAddrLeadingZeros{1'b0}}},
sram_wr_ptr + SramBaseAddr;
156   assign sram_rd_addr = {{SramAddrLeadingZeros{1'b0}}},
sram_rd_ptr + SramBaseAddr;
157 end else begin : gen_no_zero_extend_sram_addrs
158   assign sram_wr_addr = sram_wr_ptr + SramBaseAddr;
159   assign sram_rd_addr = sram_rd_ptr + SramBaseAddr;
160 end
161
162 // FF to remember whether we read from the SRAM in the pre-
vious clock cycle.
163 logic sram_read_in_prev_cyc_d, sram_read_in_prev_cyc_q;
164 assign sram_read_in_prev_cyc_d = clr_i ? 1'b0 : sram_in-
cr_rd_ptr;
165 always_ff @(posedge clk_i or negedge rst_ni) begin
166   if (!rst_ni) begin
167     sram_read_in_prev_cyc_q <= 1'b0;
168   end else begin
169     sram_read_in_prev_cyc_q <= sram_read_in_prev_cyc_d;
170   end
171 end
172
173 // Control logic for FIFO interface wready, output buffer
writes, and SRAM requests
174 logic state_err;
175 always_comb begin
176   inp_buf_rready = 1'b0;
177   oup_buf_wvalid = 1'b0;
178   oup_buf_wdata = inp_buf_rdata;
179   sram_req_o = 1'b0;
180   sram_write_o = 1'b0;
181   sram_addr_o = sram_wr_addr;
182   state_err = 1'b0;
183
184   // If the SRAM was read in the previous cycle, write the
read data to the output buffer.
185   if (sram_read_in_prev_cyc_q) begin
186     oup_buf_wvalid = 1'b1;
187     oup_buf_wdata = sram_rdata_i;
188     // The output buffer must be ready; otherwise we are in
an erroneous state.
189     state_err = !oup_buf_wready;
190   end
191
192   // If the SRAM is not empty, data from the input buffer
must flow via the SRAM.
193   if (!sram_empty) begin
194
195     // Prioritize refilling of the output buffer: if the
output buffer is not being filled to the
196     // last slot in the current cycle or it is being read in
the current cycle, read from the SRAM
197     // so the output buffer can be written in the next
cycle.
198     if (!(oup_buf_almost_full && oup_buf_wvalid) ||
oup_buf_full)
199       || (fifo_rvalid_o && fifo_rready_i)) begin
200       sram_req_o = 1'b1;
201       sram_write_o = 1'b0;
202       sram_addr_o = sram_rd_addr;
203
204       // If the output buffer is full (and not being read),
drain the input buffer into the SRAM.
205       end else begin
206         sram_req_o = !sram_full && inp_buf_rvalid;
207         sram_write_o = 1'b1;
208         sram_addr_o = sram_wr_addr;
209         inp_buf_rready = !sram_full && sram_gnt_i;
210       end
211
212       // If the SRAM is empty, data must flow from the input
buffer to the output buffer if the output
213       // buffer is ready and is not receiving data read from the
SRAM in the previous cycle
214       end else if (oup_buf_wready && !sram_read_in_prev_cyc_q)
begin
215         oup_buf_wvalid = inp_buf_rvalid;
216         oup_buf_wdata = inp_buf_rdata;
217         inp_buf_rready = oup_buf_wready;
218
219         // Otherwise the SRAM is empty but the output buffer can-
not take the data from the input buffer,
220         // so drain the input buffer into the SRAM.
221         end else begin
222           sram_req_o = !sram_full && inp_buf_rvalid;

```



```

223     sram_write_o = 'b1;
224     sram_addr_o = sram_wr_addr;
225     inp_buf_rready = !sram_full && sram_gnt_i;
226 end
227 end
228
229 // Error output is high if any of the internal errors is
high
230 assign err_o = {inp_buf_err, oup_buf_err, sram_ptr_err,
state_err};
231
232 // The SRAM and the output buffer together form the entire
architectural capacity of the FIFO.
233 // Thus, when both are full, the FIFO is no longer ready to
receive writes even though the input
234 // buffer could store one additional entry. This ensures
that in the cycle after an entry has
235 // been read from the full FIFO, the next entry can be writ-
ten to the FIFO.
236 // (It may be possible that all input buffer slots except
one can be counted to the architectural
237 // capacity of the FIFO, but this is a relatively small op-
timization left for future work.)
238 assign fifo_wready_o = inp_buf_wready && !(sram_full &&
oup_buf_full);
239
240 // The current depth of the FIFO represented by this module
is the depth of all buffers plus the
241 // FIFO in the SRAM plus one if there's an entry in trans-
ition between SRAM and output buffer.
242 assign fifo_depth_o = DepthW'(inp_buf_depth) +
DepthW'(sram_depth) + DepthW'(oup_buf_depth) +
243     DepthW'(sram_read_in_prev_cyc_q);
244
245 // SRAM write data always comes from the input buffer.
assign sram_wdata_o = inp_buf_rdata;
246
247 assign sram_wmask_o = '1;
248
249 // `sram_rvalid_i` is only used for assertions.
logic unused_sram_rvalid;
250
251 assign unused_sram_rvalid = sram_rvalid_i;
252
253 // When we read from the SRAM in the previous cycle, we ex-
pect valid read data in this cycle.
254 `ASSERT(SramValidAfterRead_A, sram_read_in_prev_cyc_q |>
sram_rvalid_i)
255
256 // When we read from the SRAM in the previous cycle, the
output buffer must be ready to store data
257 // in this cycle.
258 `ASSERT(OupBufWreadyAfterSramRead_A, sram_read_in_prev_cyc_q
|> oup_buf_wready)
259
260 // We must never write the SRAM when it is full.
261 `ASSERT(NoSramWriteWhenFull_A, sram_full |> !sram_in-
cr_wr_ptr)
262
263 // We must never read from the SRAM when it is empty.
264 `ASSERT(NoSramReadWhenEmpty_A, sram_empty |> !sram_in-
cr_rd_ptr)
265
266 // We must never be in an erroneous state (impossible
without functional defects).
267 `ASSERT(NoErr_A, !err_o)
268 endmodule

```

6. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_fifo.sv (322 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 module i2c_fifo
8 import i2c_pkg::*;
9 import i2c_reg_pkg::FifoDepth;
10 import i2c_reg_pkg::AcqFifoDepth;
11 #(
12     localparam int unsigned FifoDepthW = $clog2(FifoDepth + 1),
13     localparam int unsigned AcqFifoDepthW = $clog2(AcqFifoDepth
+ 1)
14 ) (
15     input logic clk_i,
16     input logic rst_ni,
17     input prim_ram_1p_pkg::ram_1p_cfg_req_t ram_cfg_i,
18     output prim_ram_1p_pkg::ram_1p_cfg_rsp_t ram_cfg_o,
19
20     input logic fmt_fifo_clr_i,
21     output logic [FifoDepthW-1:0] fmt_fifo_depth_o,
22     // FMT FIFO: writes controlled by CSR
23     input logic fmt_fifo_wvalid_i,
24     output logic fmt_fifo_wready_o,
25     input logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_wdata_i,
26     // FMT FIFO: reads controlled by FSM
27     output logic fmt_fifo_rvalid_o,
28     input logic fmt_fifo_rready_i,

```

```

29     output logic [FMT_FIFO_WIDTH-1:0] fmt_fifo_rdata_o,
30
31     input logic rx_fifo_clr_i,
32     output logic [FifoDepthW-1:0] rx_fifo_depth_o,
33     // RX FIFO: writes controlled by FSM
34     input logic rx_fifo_wvalid_i,
35     output logic rx_fifo_wready_o,
36     input logic [RX_FIFO_WIDTH-1:0] rx_fifo_wdata_i,
37     // RX FIFO: reads controlled by CSR
38     output logic rx_fifo_rvalid_o,
39     input logic rx_fifo_rready_i,
40     output logic [RX_FIFO_WIDTH-1:0] rx_fifo_rdata_o,
41
42     input logic tx_fifo_clr_i,
43     output logic [FifoDepthW-1:0] tx_fifo_depth_o,
44     // TX FIFO: writes controlled by CSR
45     input logic tx_fifo_wvalid_i,
46     output logic tx_fifo_wready_o,
47     input logic [TX_FIFO_WIDTH-1:0] tx_fifo_wdata_i,
48     // TX FIFO: reads controlled by FSM
49     output logic tx_fifo_rvalid_o,
50     input logic tx_fifo_rready_i,
51     output logic [TX_FIFO_WIDTH-1:0] tx_fifo_rdata_o,
52
53     input logic acq_fifo_clr_i,
54     output logic [AcqFifoDepthW-1:0] acq_fifo_depth_o,
55     // ACQ FIFO: writes controlled by FSM
56     input logic acq_fifo_wvalid_i,
57     output logic acq_fifo_wready_o,
58     input logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_wdata_i,
59     // ACQ FIFO: reads controlled by CSR
60     output logic acq_fifo_rvalid_o,
61     input logic acq_fifo_rready_i,
62     output logic [ACQ_FIFO_WIDTH-1:0] acq_fifo_rdata_o
63 );
64
65 // RAM synthesis parameters
66 localparam int unsigned RamDepth = 464;
67
68 `ASSERT_INIT(RamDepthSuffices_A, RamDepth >= 3 * FifoDepth +
AcqFifoDepth)
69 localparam int unsigned RamAw =
prim_util_pkg::vbits(RamDepth);
70 localparam int unsigned RamWidth = 13;
71 `ASSERT_INIT(RamWidthSuffices_A,
72     RamWidth >= FMT_FIFO_WIDTH &&
73     RamWidth >= RX_FIFO_WIDTH &&
74     RamWidth >= TX_FIFO_WIDTH &&
75     RamWidth >= ACQ_FIFO_WIDTH)
76
77 // RAM address map
78 localparam logic [RamAw-1:0] FmtFifoRamBaseAddr = RamAw'(0);
79 localparam logic [RamAw-1:0] RxFifoRamBaseAddr = RamAw'(1 *
FifoDepth);
80 localparam logic [RamAw-1:0] TxFifoRamBaseAddr = RamAw'(2 *
FifoDepth);
81 localparam logic [RamAw-1:0] AcqFifoRamBaseAddr = RamAw'(3 *
FifoDepth);
82
83 // RAM read data, which is a common signal to all adapters
84 logic [RamWidth-1:0] ram_rdata;
85
86 // RAM adapter for FMT FIFO
87 logic [RamWidth-1:0] fmt_fifo_wdata, fmt_fifo_rdata,
fmt_ram_wdata;
88 logic fmt_ram_req, fmt_ram_gnt, fmt_ram_write, fmt_ram_rval-
id;
89 logic [RamAw-1:0] fmt_ram_addr;
90 i2c_fifo_sync_sram_adapter #(
91     .Width (RamWidth),
92     .Depth (FifoDepth),
93     .SramAw (RamAw),
94     .SramBaseAddr (FmtFifoRamBaseAddr)
95 ) u_fmt_fifo_sram_adapter (
96     .clk_i,
97     .rst_ni,
98     .clr_i (fmt_fifo_clr_i),
99     .fifo_wvalid_i (fmt_fifo_wvalid_i),
100     .fifo_wready_o (fmt_fifo_wready_o),
101     .fifo_wdata_i (fmt_fifo_wdata),
102     .fifo_rvalid_o (fmt_fifo_rvalid_o),
103     .fifo_rready_i (fmt_fifo_rready_i),
104     .fifo_rdata_o (fmt_fifo_rdata),
105     .fifo_depth_o (fmt_fifo_depth_o),
106     .sram_req_o (fmt_ram_req),
107     .sram_gnt_i (fmt_ram_gnt),
108     .sram_write_o (fmt_ram_write),
109     .sram_addr_o (fmt_ram_addr),
110     .sram_wdata_o (fmt_ram_wdata),
111     .sram_wmask_o (* unused *),
112     .sram_rdata_i (ram_rdata),
113     .sram_rvalid_i (fmt_ram_rvalid),
114     .err_o (* unused *)
115 );
116 assign fmt_fifo_wdata = fmt_fifo_wdata_i;
117 assign fmt_fifo_rdata_o =
fmt_fifo_rdata[FMT_FIFO_WIDTH-1:0];
118

```

```

118 // RAM adapter for RX FIFO
119 logic [RamWidth-1:0] rx_fifo_wdata, rx_fifo_rdata,
rx_fifo_wdata;
120 logic rx_ram_req, rx_ram_gnt, rx_ram_write, rx_ram_rvalid;
121 logic [RamAw-1:0] rx_ram_addr;
122 i2c_fifo_sync_sram_adapter #(
123     .Width      (RamWidth),
124     .Depth      (FifoDepth),
125     .SramAw      (RamAw),
126     .SramBaseAddr(RxFifoRamBaseAddr)
127 ) u_rx_fifo_sram_adapter (
128     .clk_i,
129     .rst_ni,
130     .clr_i      (rx_fifo_clr_i),
131     .fifo_wvalid_i(rx_fifo_wvalid_i),
132     .fifo_wready_o(rx_fifo_wready_o),
133     .fifo_wdata_i (rx_fifo_wdata),
134     .fifo_rvalid_o(rx_fifo_rvalid_o),
135     .fifo_rready_i(rx_fifo_rready_i),
136     .fifo_rdata_o (rx_fifo_rdata),
137     .fifo_depth_o (rx_fifo_depth_o),
138     .sram_req_o   (rx_ram_req),
139     .sram_gnt_i   (rx_ram_gnt),
140     .sram_write_o (rx_ram_write),
141     .sram_addr_o  (rx_ram_addr),
142     .sram_wdata_o (rx_fifo_wdata),
143     .sram_wmask_o (* unused */,
144     .sram_rdata_i (ram_rdata),
145     .sram_rvalid_i(rx_fifo_rvalid),
146     .err_o       (* unused */)
147 );
148 assign rx_fifo_wdata = {{{(RamWidth - RX_FIFO_WIDTH){1'b0}}},
rx_fifo_wdata_i};
149 assign rx_fifo_rdata_o = rx_fifo_rdata[RX_FIFO_WIDTH-1:0];
150 logic unused_rx_fifo_rdata;
151 assign unused_rx_fifo_rdata = ^rx_fifo_rdata[(RamWidth-1):
(RX_FIFO_WIDTH-1)];
152
153 // RAM adapter for TX FIFO
154 logic [RamWidth-1:0] tx_fifo_wdata, tx_fifo_rdata,
tx_fifo_wdata;
155 logic tx_ram_req, tx_ram_gnt, tx_ram_write, tx_ram_rvalid;
156 logic [RamAw-1:0] tx_ram_addr;
157 i2c_fifo_sync_sram_adapter #(
158     .Width      (RamWidth),
159     .Depth      (FifoDepth),
160     .SramAw      (RamAw),
161     .SramBaseAddr(TxFifoRamBaseAddr)
162 ) u_tx_fifo_sram_adapter (
163     .clk_i,
164     .rst_ni,
165     .clr_i      (tx_fifo_clr_i),
166     .fifo_wvalid_i(tx_fifo_wvalid_i),
167     .fifo_wready_o(tx_fifo_wready_o),
168     .fifo_wdata_i (tx_fifo_wdata),
169     .fifo_rvalid_o(tx_fifo_rvalid_o),
170     .fifo_rready_i(tx_fifo_rready_i),
171     .fifo_rdata_o (tx_fifo_rdata),
172     .fifo_depth_o (tx_fifo_depth_o),
173     .sram_req_o   (tx_ram_req),
174     .sram_gnt_i   (tx_ram_gnt),
175     .sram_write_o (tx_ram_write),
176     .sram_addr_o  (tx_ram_addr),
177     .sram_wdata_o (tx_fifo_wdata),
178     .sram_wmask_o (* unused */,
179     .sram_rdata_i (ram_rdata),
180     .sram_rvalid_i(tx_fifo_rvalid),
181     .err_o       (* unused */)
182 );
183 assign tx_fifo_wdata = {{{(RamWidth - TX_FIFO_WIDTH){1'b0}}},
tx_fifo_wdata_i};
184 assign tx_fifo_rdata_o = tx_fifo_rdata[TX_FIFO_WIDTH-1:0];
185 logic unused_tx_fifo_rdata;
186 assign unused_tx_fifo_rdata = ^tx_fifo_rdata[(RamWidth-1):
(TX_FIFO_WIDTH-1)];
187
188 // RAM adapter for ACQ FIFO
189 logic [RamWidth-1:0] acq_fifo_wdata, acq_fifo_rdata,
acq_fifo_wdata;
190 logic acq_ram_req, acq_ram_gnt, acq_ram_write, acq_ram_rval-
id;
191 logic [RamAw-1:0] acq_ram_addr;
192 i2c_fifo_sync_sram_adapter #(
193     .Width      (RamWidth),
194     .Depth      (AcqFifoDepth),
195     .SramAw      (RamAw),
196     .SramBaseAddr(AcqFifoRamBaseAddr)
197 ) u_acq_fifo_sram_adapter (
198     .clk_i,
199     .rst_ni,
200     .clr_i      (acq_fifo_clr_i),
201     .fifo_wvalid_i(acq_fifo_wvalid_i),
202     .fifo_wready_o(acq_fifo_wready_o),
203     .fifo_wdata_i (acq_fifo_wdata),
204     .fifo_rvalid_o(acq_fifo_rvalid_o),
205     .fifo_rready_i(acq_fifo_rready_i),
206     .fifo_rdata_o (acq_fifo_rdata),
207     .fifo_depth_o (acq_fifo_depth_o),
208     .sram_req_o   (acq_ram_req),
209     .sram_gnt_i   (acq_ram_gnt),
210     .sram_write_o (acq_ram_write),
211     .sram_addr_o  (acq_ram_addr),
212     .sram_wdata_o (acq_fifo_wdata),
213     .sram_wmask_o (* unused */,
214     .sram_rdata_i (ram_rdata),
215     .sram_rvalid_i(acq_fifo_rvalid),
216     .err_o       (* unused */)
217 );
218 assign acq_fifo_wdata = {{{(RamWidth - ACQ_FIFO_WIDTH)
{1'b0}}}, acq_fifo_wdata_i};
219 assign acq_fifo_rdata_o =
acq_fifo_rdata[ACQ_FIFO_WIDTH-1:0];
220 logic unused_acq_fifo_rdata;
221 assign unused_acq_fifo_rdata = ^acq_fifo_rdata[(RamWidth-1):
(ACQ_FIFO_WIDTH-1)];
222
223 // RAM signals
224 logic ram_req, ram_write, ram_rvalid;
225 logic [RamAw-1:0] ram_addr;
226 logic [RamWidth-1:0] ram_wdata;
227
228 // RAM arbiter
229 localparam int unsigned RamArbN = 4;
230 localparam int unsigned RamArbIdxW =
prim_util_pkg::vbits(RamArbN);
231 // Data for arbiter consists of write bit, address, and
write data
232 localparam int unsigned RamArbDw = 1 + RamAw + RamWidth;
233 logic [RamArbN-1:0] ram_arb_req, ram_arb_gnt;
234 logic [RamArbIdxW-1:0] ram_arb_idx;
235 logic [RamArbDw-1:0] ram_arb_inp_data[RamArbN];
236 logic [RamArbDw-1:0] ram_arb_oup_data;
237 prim_arbiter_tree #(
238     .N      (RamArbN),
239     .DW      (RamArbDw),
240     .EnDataPort(1)
241 ) u_ram_arbiter (
242     .clk_i,
243     .rst_ni,
244     .req_chk_i(1'b1),
245     .req_i      (ram_arb_req),
246     .data_i      (ram_arb_inp_data),
247     .gnt_o      (ram_arb_gnt),
248     .idx_o      (ram_arb_idx),
249     .valid_o     (ram_req),
250     .data_o     (ram_arb_oup_data),
251     .ready_i     (1'b1)
252 );
253 assign ram_arb_req[0] = fmt_ram_req;
254 assign ram_arb_inp_data[0] = {fmt_ram_write, fmt_ram_addr,
fmt_ram_wdata};
255 assign fmt_ram_gnt = ram_arb_gnt[0];
256 assign ram_arb_req[1] = rx_ram_req;
257 assign ram_arb_inp_data[1] = {rx_ram_write, rx_ram_addr,
rx_ram_wdata};
258 assign rx_ram_gnt = ram_arb_gnt[1];
259 assign ram_arb_req[2] = tx_ram_req;
260 assign ram_arb_inp_data[2] = {tx_ram_write, tx_ram_addr,
tx_ram_wdata};
261 assign tx_ram_gnt = ram_arb_gnt[2];
262 assign ram_arb_req[3] = acq_ram_req;
263 assign ram_arb_inp_data[3] = {acq_ram_write, acq_ram_addr,
acq_ram_wdata};
264 assign acq_ram_gnt = ram_arb_gnt[3];
265
266 // Demux `ram_rvalid` based on arbiter index one cycle
earlier.
267 logic [RamArbIdxW-1:0] ram_arb_idx_q;
268 always_ff @(posedge clk_i or negedge rst_ni) begin
269     if (!rst_ni) begin
270         ram_arb_idx_q <= '0;
271     end else begin
272         ram_arb_idx_q <= ram_arb_idx;
273     end
274 end
275 assign fmt_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(0))) && ram_rvalid;
276 assign rx_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(1))) && ram_rvalid;
277 assign tx_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(2))) && ram_rvalid;
278 assign acq_ram_rvalid = (ram_arb_idx_q == un-
signed'(RamArbIdxW'(3))) && ram_rvalid;
279
280 // RAM instance
281 prim_ram_1p_adv #(
282     .Depth      (RamDepth),
283     .Width      (RamWidth),
284     .DataBitsPerMask (RamWidth),
285     .EnableECC      (1'b0),
286     .EnableParity    (1'b0),
287     .EnableInputPipeline(1'b0),
288     .EnableOutputPipeline(1'b0)
289 ) u_ram_1p (
290     .clk_i,
291     .rst_ni,
292     .req_i      (ram_req),
293     .write_i     (ram_write),
294     .addr_i      (ram_addr),
295     .wdata_i     (ram_wdata),

```



```

298 .wmask_i ('1),
299 .rdata_o (ram_rdata),
300 .rvalid_o (ram_rvalid),
301 .rerror_o (* unused *),
302 .cfg_i (ram_cfg_i),
303 .cfg_o (ram_cfg_o),
304 .alert_o (* unused *)
305 );
306 assign {ram_write, ram_addr, ram_wdata} = ram_arb_oup_data;
307
308 // For the FMT, ACQ, and TX FIFOs, we assume that a write
request stays stable until it has been
309 // granted. Put differently, we assume this module for
those FIFOs will never see a write that
310 // would be dropped (e.g., because the FIFO is full). This
does not hold for the RX FIFO, for

311 // which `i2c_core` will register an overflow event if there
is a wvalid without a wready.
312 `ASSUME(FmtWriteStableBeforeHandshake_A,
313         fmt_fifo_wvalid_i && !fmt_fifo_wready_o
314         |=> $stable(fmt_fifo_wvalid_i) &&
$stable(fmt_fifo_wdata_i))
315 `ASSUME(AcqWriteStableBeforeHandshake_A,
316         acq_fifo_wvalid_i && !acq_fifo_wready_o
317         |=> $stable(acq_fifo_wvalid_i) &&
$stable(acq_fifo_wdata_i))
318 `ASSUME(TxWriteStableBeforeHandshake_A,
319         tx_fifo_wvalid_i && !tx_fifo_wready_o
320         |=> $stable(tx_fifo_wvalid_i) &&
$stable(tx_fifo_wdata_i))
321
322 endmodule

```

7. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_pkg.sv (62 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // i2c package
6 //
7 package i2c_pkg;
8     parameter int unsigned I2C_ACQ_BYTE_ID_WIDTH = 3;
9
10 // TODO(#22028) encode this more efficiently in the ACQ
FIFO. Each entry in the
11 // ACQ FIFO does not need to contain both an 8 bit data
field and a 3 bit
12 // identifier. We should have the ACQ FIFO be 9 bits wide
where the MSB
13 // indicates whether it is a data byte or a control byte.
This way we can
14 // add more values to this enum without having to widen the
ACQ FIFO width.
15 typedef enum logic [I2C_ACQ_BYTE_ID_WIDTH-1:0] {
16     AcqData      = 3'b000,
17     AcqStart     = 3'b001,
18     AcqStop      = 3'b010,
19     AcqRestart   = 3'b011,
20     // AcqNack means one of two things:
21     // 1. We received a read request to our address, but had
to NACK the
22     // address because our ACQ FIFO is full.
23     // 2. We received too many bytes in a write request and
had to NACK a data
24     // byte. The NACK'ed data byte is still in the data field
for inspection.
25     AcqNack      = 3'b100,
26
27     // AcqNackStart means that we were addressed on this item,
but we timed
28     // out after stretching.
29     AcqNackStart = 3'b101,
30     // AcqNackStop means that we were addressed during the
transaction, but we
31     // timed out after stretching and received the Stop to end
the
32     // transaction.
33     AcqNackStop  = 3'b110
34 } i2c_acq_byte_id_e;
35
36 typedef enum logic {
37     StretchTimeoutMode = 1'b0,
38     BusTimeoutMode     = 1'b1
39 } i2c_timeout_mode_e;
40
41 // Width of each entry in the FMT FIFO with enough space for an
8-bit data
42 // byte and 5 flags.
43 parameter int unsigned FMT_FIFO_WIDTH = 8 + 5;
44
45 // Width of each entry in the RX and TX FIFO: just an 8-bit
data byte.
46 parameter int unsigned RX_FIFO_WIDTH = 8;
47 parameter int unsigned TX_FIFO_WIDTH = 8;

```

```

47
48 // Width of each entry in the ACQ FIFO with enough space for an
8-bit data
49 // byte and an identifier defined by i2c_acq_byte_id_e.
50 parameter int unsigned ACQ_FIFO_WIDTH =
I2C_ACQ_BYTE_ID_WIDTH + 8;
51
52 typedef enum bit {
53     WRITE = 1'b0,
54     READ  = 1'b1
55 } rw_e;
56
57 typedef enum bit {
58     ACK = 1'b0,
59     NACK = 1'b1
60 } acknack_e;
61
62 endpackage : i2c_pkg

```

8. lowRISC_opentitan/hw/ip/i2c/rtl/i2c_reg_pkg.sv (829 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
structure
6
7 package i2c_reg_pkg;
8
9 // Param list
10 parameter int FifoDepth = 64;
11 parameter int AcqFifoDepth = 268;
12 parameter int NumAlerts = 1;
13
14 // Address widths within the block
15 parameter int BlockAw = 7;
16
17 // Number of registers for every interface
18 parameter int NumRegs = 32;
19
20 // Alert indices
21 typedef enum int {
22     AlertFatalFaultIdx = 0
23 } i2c_alert_idx_t;
24
25 ///////////////////////////////////////////////////
26 // Typedefs for registers //
27 ///////////////////////////////////////////////////
28
29 typedef struct packed {
30     struct packed {
31         logic q;
32     } host_timeout;
33     struct packed {
34         logic q;
35     } unexp_stop;
36     struct packed {
37         logic q;
38     } acq_stretch;
39     struct packed {
40         logic q;
41     } tx_threshold;
42     struct packed {
43         logic q;
44     } tx_stretch;
45     struct packed {
46         logic q;
47     } cmd_complete;
48     struct packed {
49         logic q;
50     } sda_unstable;
51     struct packed {
52         logic q;
53     } stretch_timeout;
54     struct packed {
55         logic q;
56     } sda_interference;
57     struct packed {
58         logic q;
59     } scl_interference;
60     struct packed {
61         logic q;
62     } controller_halt;
63     struct packed {
64         logic q;
65     } rx_overflow;
66     struct packed {
67         logic q;
68     } acq_threshold;
69     struct packed {
70         logic q;
71     } rx_threshold;
72     struct packed {
73         logic q;
74     } fmt_threshold;

```

```

75 } i2c_reg2hw_intr_state_reg_t;
76
77 typedef struct packed {
78     struct packed {
79         logic q;
80     } host_timeout;
81     struct packed {
82         logic q;
83     } unexp_stop;
84     struct packed {
85         logic q;
86     } acq_stretch;
87     struct packed {
88         logic q;
89     } tx_threshold;
90     struct packed {
91         logic q;
92     } tx_stretch;
93     struct packed {
94         logic q;
95     } cmd_complete;
96     struct packed {
97         logic q;
98     } sda_unstable;
99     struct packed {
100         logic q;
101     } stretch_timeout;
102     struct packed {
103         logic q;
104     } sda_interference;
105     struct packed {
106         logic q;
107     } scl_interference;
108     struct packed {
109         logic q;
110     } controller_halt;
111     struct packed {
112         logic q;
113     } rx_overflow;
114     struct packed {
115         logic q;
116     } acq_threshold;
117     struct packed {
118         logic q;
119     } rx_threshold;
120     struct packed {
121         logic q;
122     } fmt_threshold;
123 } i2c_reg2hw_intr_enable_reg_t;
124
125 typedef struct packed {
126     struct packed {
127         logic q;
128         logic qe;
129     } host_timeout;
130     struct packed {
131         logic q;
132         logic qe;
133     } unexp_stop;
134     struct packed {
135         logic q;
136         logic qe;
137     } acq_stretch;
138     struct packed {
139         logic q;
140         logic qe;
141     } tx_threshold;
142     struct packed {
143         logic q;
144         logic qe;
145     } tx_stretch;
146     struct packed {
147         logic q;
148         logic qe;
149     } cmd_complete;
150     struct packed {
151         logic q;
152         logic qe;
153     } sda_unstable;
154     struct packed {
155         logic q;
156         logic qe;
157     } stretch_timeout;
158     struct packed {
159         logic q;
160         logic qe;
161     } sda_interference;
162     struct packed {
163         logic q;
164         logic qe;
165     } scl_interference;
166     struct packed {
167         logic q;
168         logic qe;
169     } controller_halt;
170     struct packed {
171         logic q;
172         logic qe;
173     } rx_overflow;
174     struct packed {
175         logic q;
176         logic qe;
177     } acq_threshold;
178     struct packed {
179         logic q;
180         logic qe;
181     } rx_threshold;
182     struct packed {
183         logic q;
184         logic qe;
185     } fmt_threshold;
186 } i2c_reg2hw_intr_test_reg_t;
187
188 typedef struct packed {
189     logic q;
190     logic qe;
191 } i2c_reg2hw_alert_test_reg_t;
192
193 typedef struct packed {
194     struct packed {
195         logic q;
196     } tx_stretch_ctrl_en;
197     struct packed {
198         logic q;
199     } multi_controller_monitor_en;
200     struct packed {
201         logic q;
202     } ack_ctrl_en;
203     struct packed {
204         logic q;
205     } nack_addr_after_timeout;
206     struct packed {
207         logic q;
208     } llpbk;
209     struct packed {
210         logic q;
211     } enabletarget;
212     struct packed {
213         logic q;
214     } enablehost;
215 } i2c_reg2hw_ctrl_reg_t;
216
217 typedef struct packed {
218     logic [7:0] q;
219     logic re;
220 } i2c_reg2hw_rdata_reg_t;
221
222 typedef struct packed {
223     struct packed {
224         logic q;
225         logic qe;
226     } nakok;
227     struct packed {
228         logic q;
229         logic qe;
230     } rcont;
231     struct packed {
232         logic q;
233         logic qe;
234     } readb;
235     struct packed {
236         logic q;
237         logic qe;
238     } stop;
239     struct packed {
240         logic q;
241         logic qe;
242     } start;
243     struct packed {
244         logic [7:0] q;
245         logic qe;
246     } fbyte;
247 } i2c_reg2hw_fdata_reg_t;
248
249 typedef struct packed {
250     struct packed {
251         logic q;
252         logic qe;
253     } txrst;
254     struct packed {
255         logic q;
256         logic qe;
257     } acqrst;
258     struct packed {
259         logic q;
260         logic qe;
261     } fmtrst;
262     struct packed {
263         logic q;
264         logic qe;
265     } rxrst;
266 } i2c_reg2hw_fifo_ctrl_reg_t;
267
268 typedef struct packed {
269     struct packed {
270         logic [11:0] q;
271         logic qe;
272     } fmt_thresh;
273     struct packed {
274         logic [11:0] q;
275         logic qe;
276     } rx_thresh;

```

```

277 } i2c_reg2hw_host_fifo_config_reg_t;
278
279 typedef struct packed {
280     struct packed {
281         logic [11:0] q;
282         logic       qe;
283     } acq_thresh;
284     struct packed {
285         logic [11:0] q;
286         logic       qe;
287     } tx_thresh;
288 } i2c_reg2hw_target_fifo_config_reg_t;
289
290 typedef struct packed {
291     struct packed {
292         logic       q;
293     } sdaval;
294     struct packed {
295         logic       q;
296     } sclval;
297     struct packed {
298         logic       q;
299     } txovrden;
300 } i2c_reg2hw_ovrd_reg_t;
301
302 typedef struct packed {
303     struct packed {
304         logic [12:0] q;
305     } tlow;
306     struct packed {
307         logic [12:0] q;
308     } thigh;
309 } i2c_reg2hw_timing0_reg_t;
310
311 typedef struct packed {
312     struct packed {
313         logic [8:0] q;
314     } t_f;
315     struct packed {
316         logic [9:0] q;
317     } t_r;
318 } i2c_reg2hw_timing1_reg_t;
319
320 typedef struct packed {
321     struct packed {
322         logic [12:0] q;
323     } thd_sta;
324     struct packed {
325         logic [12:0] q;
326     } tsu_sta;
327 } i2c_reg2hw_timing2_reg_t;
328
329 typedef struct packed {
330     struct packed {
331         logic [12:0] q;
332     } thd_dat;
333     struct packed {
334         logic [8:0] q;
335     } tsu_dat;
336 } i2c_reg2hw_timing3_reg_t;
337
338 typedef struct packed {
339     struct packed {
340         logic [12:0] q;
341     } t_buf;
342     struct packed {
343         logic [12:0] q;
344     } tsu_sto;
345 } i2c_reg2hw_timing4_reg_t;
346
347 typedef struct packed {
348     struct packed {
349         logic       q;
350     } en;
351     struct packed {
352         logic       q;
353     } mode;
354     struct packed {
355         logic [29:0] q;
356     } val;
357 } i2c_reg2hw_timeout_ctrl_reg_t;
358
359 typedef struct packed {
360     struct packed {
361         logic [6:0] q;
362     } mask1;
363     struct packed {
364         logic [6:0] q;
365     } address1;
366     struct packed {
367         logic [6:0] q;
368     } mask0;
369     struct packed {
370         logic [6:0] q;
371     } address0;
372 } i2c_reg2hw_target_id_reg_t;
373
374 typedef struct packed {
375     struct packed {
376         logic [2:0] q;
377         logic       re;
378     } signal;
379     struct packed {
380         logic [7:0] q;
381         logic       re;
382     } abyte;
383 } i2c_reg2hw_acqdata_reg_t;
384
385 typedef struct packed {
386     logic [7:0] q;
387     logic       qe;
388 } i2c_reg2hw_txdata_reg_t;
389
390 typedef struct packed {
391     logic [19:0] q;
392 } i2c_reg2hw_host_timeout_ctrl_reg_t;
393
394 typedef struct packed {
395     struct packed {
396         logic       q;
397     } en;
398     struct packed {
399         logic [30:0] q;
400     } val;
401 } i2c_reg2hw_target_timeout_ctrl_reg_t;
402
403 typedef struct packed {
404     logic [7:0] q;
405 } i2c_reg2hw_target_nack_count_reg_t;
406
407 typedef struct packed {
408     struct packed {
409         logic       q;
410         logic       qe;
411     } nack;
412     struct packed {
413         logic [8:0] q;
414         logic       qe;
415     } nbytes;
416 } i2c_reg2hw_target_ack_ctrl_reg_t;
417
418 typedef struct packed {
419     struct packed {
420         logic       q;
421     } en;
422     struct packed {
423         logic [30:0] q;
424     } val;
425 } i2c_reg2hw_host_nack_handler_timeout_reg_t;
426
427 typedef struct packed {
428     struct packed {
429         logic       q;
430     } arbitration_lost;
431     struct packed {
432         logic       q;
433     } bus_timeout;
434     struct packed {
435         logic       q;
436     } unhandled_nack_timeout;
437     struct packed {
438         logic       q;
439     } nack;
440 } i2c_reg2hw_controller_events_reg_t;
441
442 typedef struct packed {
443     struct packed {
444         logic       q;
445     } arbitration_lost;
446     struct packed {
447         logic       q;
448     } bus_timeout;
449     struct packed {
450         logic       q;
451     } tx_pending;
452 } i2c_reg2hw_target_events_reg_t;
453
454 typedef struct packed {
455     struct packed {
456         logic       d;
457         logic       de;
458     } host_timeout;
459     struct packed {
460         logic       d;
461         logic       de;
462     } unexp_stop;
463     struct packed {
464         logic       d;
465         logic       de;
466     } acq_stretch;
467     struct packed {
468         logic       d;
469         logic       de;
470     } tx_threshold;
471     struct packed {
472         logic       d;
473         logic       de;
474     } tx_stretch;
475     struct packed {
476         logic       d;
477         logic       de;
478     } cmd_complete;

```

```

479 struct packed {
480     logic d;
481     logic de;
482 } sda_unstable;
483 struct packed {
484     logic d;
485     logic de;
486 } stretch_timeout;
487 struct packed {
488     logic d;
489     logic de;
490 } sda_interference;
491 struct packed {
492     logic d;
493     logic de;
494 } scl_interference;
495 struct packed {
496     logic d;
497     logic de;
498 } controller_halt;
499 struct packed {
500     logic d;
501     logic de;
502 } rx_overflow;
503 struct packed {
504     logic d;
505     logic de;
506 } acq_threshold;
507 struct packed {
508     logic d;
509     logic de;
510 } rx_threshold;
511 struct packed {
512     logic d;
513     logic de;
514 } fmt_threshold;
515 } i2c_hw2reg_intr_state_reg_t;
516
517 typedef struct packed {
518     struct packed {
519         logic d;
520     } ack_ctrl_stretch;
521     struct packed {
522         logic d;
523     } acqempty;
524     struct packed {
525         logic d;
526     } txempty;
527     struct packed {
528         logic d;
529     } acqfull;
530     struct packed {
531         logic d;
532     } txfull;
533     struct packed {
534         logic d;
535     } rxempty;
536     struct packed {
537         logic d;
538     } targetidle;
539     struct packed {
540         logic d;
541     } hostidle;
542     struct packed {
543         logic d;
544     } fmtempty;
545     struct packed {
546         logic d;
547     } rxfull;
548     struct packed {
549         logic d;
550     } fmtfull;
551 } i2c_hw2reg_status_reg_t;
552
553 typedef struct packed {
554     logic [7:0] d;
555 } i2c_hw2reg_rdata_reg_t;
556
557 typedef struct packed {
558     struct packed {
559         logic [11:0] d;
560     } rxlvl;
561     struct packed {
562         logic [11:0] d;
563     } fmtrlvl;
564 } i2c_hw2reg_host_fifo_status_reg_t;
565
566 typedef struct packed {
567     struct packed {
568         logic [11:0] d;
569     } acqlvl;
570     struct packed {
571         logic [11:0] d;
572     } txlvl;
573 } i2c_hw2reg_target_fifo_status_reg_t;
574
575 typedef struct packed {
576     struct packed {
577         logic [15:0] d;
578     } sda_rx;
579     struct packed {

```

```

580         logic [15:0] d;
581     } scl_rx;
582 } i2c_hw2reg_val_reg_t;
583
584 typedef struct packed {
585     struct packed {
586         logic [2:0] d;
587     } signal;
588     struct packed {
589         logic [7:0] d;
590     } abyte;
591 } i2c_hw2reg_acqdata_reg_t;
592
593 typedef struct packed {
594     logic [7:0] d;
595     logic de;
596 } i2c_hw2reg_target_nack_count_reg_t;
597
598 typedef struct packed {
599     struct packed {
600         logic [8:0] d;
601     } nbytes;
602 } i2c_hw2reg_target_ack_ctrl_reg_t;
603
604 typedef struct packed {
605     logic [7:0] d;
606 } i2c_hw2reg_acq_fifo_next_data_reg_t;
607
608 typedef struct packed {
609     struct packed {
610         logic d;
611         logic de;
612     } arbitration_lost;
613     struct packed {
614         logic d;
615         logic de;
616     } bus_timeout;
617     struct packed {
618         logic d;
619         logic de;
620     } unhandled_nack_timeout;
621     struct packed {
622         logic d;
623         logic de;
624     } nack;
625 } i2c_hw2reg_controller_events_reg_t;
626
627 typedef struct packed {
628     struct packed {
629         logic d;
630         logic de;
631     } arbitration_lost;
632     struct packed {
633         logic d;
634         logic de;
635     } bus_timeout;
636     struct packed {
637         logic d;
638         logic de;
639     } tx_pending;
640 } i2c_hw2reg_target_events_reg_t;
641
642 // Register -> HW type
643 typedef struct packed {
644     i2c_reg2hw_intr_state_reg_t intr_state; // [471:457]
645     i2c_reg2hw_intr_enable_reg_t intr_enable; // [456:442]
646     i2c_reg2hw_intr_test_reg_t intr_test; // [441:412]
647     i2c_reg2hw_alert_test_reg_t alert_test; // [411:410]
648     i2c_reg2hw_ctrl_reg_t ctrl; // [409:403]
649     i2c_reg2hw_rdata_reg_t rdata; // [402:394]
650     i2c_reg2hw_fdata_reg_t fdata; // [393:375]
651     i2c_reg2hw_fifo_ctrl_reg_t fifo_ctrl; // [374:367]
652     i2c_reg2hw_host_fifo_config_reg_t host_fifo_config; //
[366:341]
653     i2c_reg2hw_target_fifo_config_reg_t target_fifo_config; //
[340:315]
654     i2c_reg2hw_ovrd_reg_t ovrd; // [314:312]
655     i2c_reg2hw_timing0_reg_t timing0; // [311:286]
656     i2c_reg2hw_timing1_reg_t timing1; // [285:267]
657     i2c_reg2hw_timing2_reg_t timing2; // [266:241]
658     i2c_reg2hw_timing3_reg_t timing3; // [240:219]
659     i2c_reg2hw_timing4_reg_t timing4; // [218:193]
660     i2c_reg2hw_timeout_ctrl_reg_t timeout_ctrl; // [192:161]
661     i2c_reg2hw_target_id_reg_t target_id; // [160:133]
662     i2c_reg2hw_acqdata_reg_t acqdata; // [132:120]
663     i2c_reg2hw_txdata_reg_t txdata; // [119:111]
664     i2c_reg2hw_host_timeout_ctrl_reg_t host_timeout_ctrl; //
[110:91]
665     i2c_reg2hw_target_timeout_ctrl_reg_t tar-
get_timeout_ctrl; // [90:59]
666     i2c_reg2hw_target_nack_count_reg_t target_nack_count; //
[58:51]
667     i2c_reg2hw_target_ack_ctrl_reg_t target_ack_ctrl; //
[50:39]
668     i2c_reg2hw_host_nack_handler_timeout_reg_t host_n-
ack_handler_timeout; // [38:7]
669     i2c_reg2hw_controller_events_reg_t controller_events; //
[6:3]
670     i2c_reg2hw_target_events_reg_t target_events; // [2:0]
671 } i2c_reg2hw_t;

```

```

672 // HW -> register type
673 typedef struct packed {
674     i2c_hw2reg_intr_state_reg_t intr_state; // [179:150]
675     i2c_hw2reg_status_reg_t status; // [149:139]
676     i2c_hw2reg_rdata_reg_t rdata; // [138:131]
677     i2c_hw2reg_host_fifo_status_reg_t host_fifo_status; //
[130:107]
678
679     i2c_hw2reg_target_fifo_status_reg_t target_fifo_status; //
[106:83]
680     i2c_hw2reg_val_reg_t val; // [82:51]
681     i2c_hw2reg_acqdata_reg_t acqdata; // [50:40]
682     i2c_hw2reg_target_nack_count_reg_t target_nack_count; //
[39:31]
683     i2c_hw2reg_target_ack_ctrl_reg_t target_ack_ctrl; //
[30:22]
684
685     i2c_hw2reg_acq_fifo_next_data_reg_t acq_fifo_next_data; //
[21:14]
686     i2c_hw2reg_controller_events_reg_t controller_events; //
[13:6]
687     i2c_hw2reg_target_events_reg_t target_events; // [5:0]
688 } i2c_hw2reg_t;
689
690 // Register offsets
691 parameter logic [BlockAw-1:0] I2C_INTR_STATE_OFFSET = 7'h 0;
692 parameter logic [BlockAw-1:0] I2C_INTR_ENABLE_OFFSET = 7'h
4;
693 parameter logic [BlockAw-1:0] I2C_INTR_TEST_OFFSET = 7'h 8;
694 parameter logic [BlockAw-1:0] I2C_ALERT_TEST_OFFSET = 7'h c;
695 parameter logic [BlockAw-1:0] I2C_CTRL_OFFSET = 7'h 10;
696 parameter logic [BlockAw-1:0] I2C_STATUS_OFFSET = 7'h 14;
697 parameter logic [BlockAw-1:0] I2C_RDATA_OFFSET = 7'h 18;
698 parameter logic [BlockAw-1:0] I2C_FDATA_OFFSET = 7'h 1c;
699 parameter logic [BlockAw-1:0] I2C_FIFO_CTRL_OFFSET = 7'h 20;
700 parameter logic [BlockAw-1:0] I2C_HOST_FIFO_CONFIG_OFFSET =
7'h 24;
701 parameter logic [BlockAw-1:0] I2C_TARGET_FIFO_CONFIG_OFFSET
= 7'h 28;
702 parameter logic [BlockAw-1:0] I2C_HOST_FIFO_STATUS_OFFSET =
7'h 2c;
703 parameter logic [BlockAw-1:0] I2C_TARGET_FIFO_STATUS_OFFSET
= 7'h 30;
704 parameter logic [BlockAw-1:0] I2C_OVRD_OFFSET = 7'h 34;
705 parameter logic [BlockAw-1:0] I2C_VAL_OFFSET = 7'h 38;
706 parameter logic [BlockAw-1:0] I2C_TIMING0_OFFSET = 7'h 3c;
707 parameter logic [BlockAw-1:0] I2C_TIMING1_OFFSET = 7'h 40;
708 parameter logic [BlockAw-1:0] I2C_TIMING2_OFFSET = 7'h 44;
709 parameter logic [BlockAw-1:0] I2C_TIMING3_OFFSET = 7'h 48;
710 parameter logic [BlockAw-1:0] I2C_TIMING4_OFFSET = 7'h 4c;
711 parameter logic [BlockAw-1:0] I2C_TIMEOUT_CTRL_OFFSET = 7'h
50;
712 parameter logic [BlockAw-1:0] I2C_TARGET_ID_OFFSET = 7'h 54;
713 parameter logic [BlockAw-1:0] I2C_ACQDATA_OFFSET = 7'h 58;
714 parameter logic [BlockAw-1:0] I2C_TXDATA_OFFSET = 7'h 5c;
715 parameter logic [BlockAw-1:0] I2C_HOST_TIMEOUT_CTRL_OFFSET =
7'h 60;
716 parameter logic [BlockAw-1:0] I2C_TARGET_TIMEOUT_CTRL_OFFSET =
7'h 64;
717 parameter logic [BlockAw-1:0] I2C_TARGET_NACK_COUNT_OFFSET =
7'h 68;
718 parameter logic [BlockAw-1:0] I2C_TARGET_ACK_CTRL_OFFSET =
7'h 6c;
719 parameter logic [BlockAw-1:0] I2C_ACQ_FIFO_NEXT_DATA_OFFSET
= 7'h 70;
720 parameter logic [BlockAw-1:0] I2C_HOST_NACK_HANDLER_TIMEOUT_OFFSET
= 7'h 74;
721 parameter logic [BlockAw-1:0] I2C_CONTROLLER_EVENTS_OFFSET =
7'h 78;
722 parameter logic [BlockAw-1:0] I2C_TARGET_EVENTS_OFFSET = 7'h
7c;
723 // Reset values for hwext registers and their fields
724 parameter logic [14:0] I2C_INTR_TEST_RESVAL = 15'h 0;
725 parameter logic [0:0] I2C_INTR_TEST_FMT_THRESHOLD_RESVAL =
1'h 0;
726 parameter logic [0:0] I2C_INTR_TEST_RX_THRESHOLD_RESVAL =
1'h 0;
727 parameter logic [0:0] I2C_INTR_TEST_ACQ_THRESHOLD_RESVAL =
1'h 0;
728 parameter logic [0:0] I2C_INTR_TEST_RX_OVERFLOW_RESVAL = 1'h
0;
729 parameter logic [0:0] I2C_INTR_TEST_CONTROLLER_HALT_RESVAL =
1'h 0;
730 parameter logic [0:0] I2C_INTR_TEST_SCL_INTERFERENCE_RESVAL
= 1'h 0;
731 parameter logic [0:0] I2C_INTR_TEST_SDA_INTERFERENCE_RESVAL
= 1'h 0;
732 parameter logic [0:0] I2C_INTR_TEST_STRETCH_TIMEOUT_RESVAL =
1'h 0;
733 parameter logic [0:0] I2C_INTR_TEST_SDA_UNSTABLE_RESVAL =
1'h 0;

```

```

734 parameter logic [0:0] I2C_INTR_TEST_CMD_COMPLETE_RESVAL =
1'h 0;
735 parameter logic [0:0] I2C_INTR_TEST_TX_STRETCH_RESVAL = 1'h
0;
736 parameter logic [0:0] I2C_INTR_TEST_TX_THRESHOLD_RESVAL =
1'h 0;
737 parameter logic [0:0] I2C_INTR_TEST_ACQ_STRETCH_RESVAL = 1'h
0;
738 parameter logic [0:0] I2C_INTR_TEST_UNEXP_STOP_RESVAL = 1'h
0;
739 parameter logic [0:0] I2C_INTR_TEST_HOST_TIMEOUT_RESVAL =
1'h 0;
740 parameter logic [0:0] I2C_ALERT_TEST_RESVAL = 1'h 0;
741 parameter logic [0:0] I2C_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
742 parameter logic [10:0] I2C_STATUS_RESVAL = 11'h 33c;
743 parameter logic [0:0] I2C_STATUS_FMTEMPTY_RESVAL = 1'h 1;
744 parameter logic [0:0] I2C_STATUS_HOSTIDLE_RESVAL = 1'h 1;
745 parameter logic [0:0] I2C_STATUS_TARGETIDLE_RESVAL = 1'h 1;
746 parameter logic [0:0] I2C_STATUS_RXEMPTY_RESVAL = 1'h 1;
747 parameter logic [0:0] I2C_STATUS_TXEMPTY_RESVAL = 1'h 1;
748 parameter logic [0:0] I2C_STATUS_ACQEMPTY_RESVAL = 1'h 1;
749 parameter logic [7:0] I2C_RDATA_RESVAL = 8'h 0;
750 parameter logic [27:0] I2C_HOST_FIFO_STATUS_RESVAL = 28'h 0;
751 parameter logic [27:0] I2C_TARGET_FIFO_STATUS_RESVAL = 28'h
0;
752 parameter logic [31:0] I2C_VAL_RESVAL = 32'h 0;
753 parameter logic [10:0] I2C_ACQDATA_RESVAL = 11'h 0;
754 parameter logic [31:0] I2C_TARGET_ACK_CTRL_RESVAL = 32'h 0;
755 parameter logic [7:0] I2C_ACQ_FIFO_NEXT_DATA_RESVAL = 8'h 0;
756
757 // Register index
758 typedef enum int {
759     I2C_INTR_STATE,
760     I2C_INTR_ENABLE,
761     I2C_INTR_TEST,
762     I2C_ALERT_TEST,
763     I2C_CTRL,
764     I2C_STATUS,
765     I2C_RDATA,
766     I2C_FDATA,
767     I2C_FIFO_CTRL,
768     I2C_HOST_FIFO_CONFIG,
769     I2C_TARGET_FIFO_CONFIG,
770     I2C_HOST_FIFO_STATUS,
771     I2C_TARGET_FIFO_STATUS,
772     I2C_OVRD,
773     I2C_VAL,
774     I2C_TIMING0,
775     I2C_TIMING1,
776     I2C_TIMING2,
777     I2C_TIMING3,
778     I2C_TIMING4,
779     I2C_TIMEOUT_CTRL,
780     I2C_TARGET_ID,
781     I2C_ACQDATA,
782     I2C_TXDATA,
783     I2C_HOST_TIMEOUT_CTRL,
784     I2C_TARGET_TIMEOUT_CTRL,
785     I2C_TARGET_NACK_COUNT,
786     I2C_TARGET_ACK_CTRL,
787     I2C_ACQ_FIFO_NEXT_DATA,
788     I2C_HOST_NACK_HANDLER_TIMEOUT,
789     I2C_CONTROLLER_EVENTS,
790     I2C_TARGET_EVENTS
791 } i2c_id_e;
792
793 // Register width information to check illegal writes
794 parameter logic [3:0] I2C_PERMIT [32] = '{
795     4'b 0011, // index[ 0] I2C_INTR_STATE
796     4'b 0011, // index[ 1] I2C_INTR_ENABLE
797     4'b 0011, // index[ 2] I2C_INTR_TEST
798     4'b 0001, // index[ 3] I2C_ALERT_TEST
799     4'b 0001, // index[ 4] I2C_CTRL
800     4'b 0011, // index[ 5] I2C_STATUS
801     4'b 0001, // index[ 6] I2C_RDATA
802     4'b 0011, // index[ 7] I2C_FDATA
803     4'b 0011, // index[ 8] I2C_FIFO_CTRL
804     4'b 1111, // index[ 9] I2C_HOST_FIFO_CONFIG
805     4'b 1111, // index[10] I2C_TARGET_FIFO_CONFIG
806     4'b 1111, // index[11] I2C_HOST_FIFO_STATUS
807     4'b 1111, // index[12] I2C_TARGET_FIFO_STATUS
808     4'b 0001, // index[13] I2C_OVRD
809     4'b 1111, // index[14] I2C_VAL
810     4'b 1111, // index[15] I2C_TIMING0
811     4'b 1111, // index[16] I2C_TIMING1
812     4'b 1111, // index[17] I2C_TIMING2
813     4'b 1111, // index[18] I2C_TIMING3
814     4'b 1111, // index[19] I2C_TIMING4
815     4'b 1111, // index[20] I2C_TIMEOUT_CTRL
816     4'b 1111, // index[21] I2C_TARGET_ID
817     4'b 0011, // index[22] I2C_ACQDATA
818     4'b 0001, // index[23] I2C_TXDATA
819     4'b 0111, // index[24] I2C_HOST_TIMEOUT_CTRL
820     4'b 1111, // index[25] I2C_TARGET_TIMEOUT_CTRL
821     4'b 0001, // index[26] I2C_TARGET_NACK_COUNT
822     4'b 1111, // index[27] I2C_TARGET_ACK_CTRL
823     4'b 0001, // index[28] I2C_ACQ_FIFO_NEXT_DATA
824     4'b 1111, // index[29] I2C_HOST_NACK_HANDLER_TIMEOUT
825     4'b 0001, // index[30] I2C_CONTROLLER_EVENTS

```

```
826     4'b 0001 // index[31] I2C_TARGET_EVENTS  
827 };
```

```
828  
829 endpackage
```

UART

What. 비동기 직렬 송수신기. 보울 생성 · 오버샘플링 수신 · FIFO · 오류 플래그.

Why it exists. 브링업의 생명선이다. 칩이 켜졌을 때 가장 먼저 동작해야 하는 것이 UART 인 경우가 많다. 그래서 가장 단순하고 가장 견고해야 한다.

학부 복습 · **EECS concepts.** 비동기 통신 · 오버샘플링과 다수결 · 클럭 주파수 오차 허용 범위 계산 (보통 $\pm 2\%$ 이내) · 프레임িং 오류.

석사 수준 · **Practice note.** 보울 오차 계산을 반드시 해 두라. 시스템 클럭과 목표 보울의 비가 정수로 안 떨어지면 누적 오차가 프레임 끝에서 비트를 놓친다. 이 계산을 데이터시트에 표로 넣는 것이 좋은 **IP** 문서의 조건이다.

6 files • 3,190 lines

API — uart.sv

Table 379. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv — module / interface / package (1개)

이름	종류
uart	module

Table 380. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv — parameter (4개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl

Table 381. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv — 포트 (21개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
lsio_trigger_o	output	logic
cio_rx_i	input	
cio_tx_o	output	logic

포트	방향	형/폭
cio_tx_en_o	output	logic
intr_tx_watermark_o	output	logic
intr_tx_empty_o	output	logic
intr_rx_watermark_o	output	logic
intr_tx_done_o	output	logic
intr_rx_overflow_o	output	logic
intr_rx_frame_err_o	output	logic
intr_rx_break_err_o	output	logic
intr_rx_timeout_o	output	logic
intr_rx_parity_err_o	output	logic

API — uart_core.sv

Table 382. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv — module / interface / package (1개)

이름	종류
uart_core	module

Table 383. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv — localparam (3개)

상수	형/폭	값
NcoWidth	int	\$bits(reg2hw.ctrl.nco.q)
TxFifoDepthw	int	\$clog2(TxFifoDepth)+1
RxFifoDepthw	int	\$clog2(RxFifoDepth)+1

Table 384. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
reg2hw	input	uart_reg_pkg::uart_reg2hw_t
hw2reg	output	uart_reg_pkg::uart_hw2reg_t
rx	input	
tx	output	logic
lsio_trigger_o	output	logic
intr_tx_watermark_o	output	logic
intr_tx_empty_o	output	logic
intr_rx_watermark_o	output	logic
intr_tx_done_o	output	logic
intr_rx_overflow_o	output	logic

포트	방향	형/폭
intr_rx_frame_err_o	output	logic
intr_rx_break_err_o	output	logic
intr_rx_timeout_o	output	logic
intr_rx_parity_err_o	output	logic

API — uart_reg_pkg.sv

Table 385. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_pkg.sv — module / interface / package (1개)

이름	종류
uart_reg_pkg	package

Table 386. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_pkg.sv — parameter (38개)

파라미터	형/폭	기본값
RxFifoDepth	int	64
TxFifoDepth	int	32
NumAlerts	int	1
BlockAw	int	6
NumRegs	int	13
UART_INTR_STATE_OFFSET	logic [BlockAw-1:0]	6'h 0
UART_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	6'h 4
UART_INTR_TEST_OFFSET	logic [BlockAw-1:0]	6'h 8
UART_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	6'h c
UART_CTRL_OFFSET	logic [BlockAw-1:0]	6'h 10
UART_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 14
UART_RDATA_OFFSET	logic [BlockAw-1:0]	6'h 18
UART_WDATA_OFFSET	logic [BlockAw-1:0]	6'h 1c
UART_FIFO_CTRL_OFFSET	logic [BlockAw-1:0]	6'h 20
UART_FIFO_STATUS_OFFSET	logic [BlockAw-1:0]	6'h 24
UART_OVRD_OFFSET	logic [BlockAw-1:0]	6'h 28
UART_VAL_OFFSET	logic [BlockAw-1:0]	6'h 2c
UART_TIMEOUT_CTRL_OFFSET	logic [BlockAw-1:0]	6'h 30
UART_INTR_TEST_RESVAL	logic [8:0]	9'h 0
UART_INTR_TEST_TX_WATERMARK_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_WATERMARK_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_TX_DONE_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_OVERFLOW_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_FRAME_ERR_RESVAL	logic [0:0]	1'h 0

파라미터	형/폭	기본값
UART_INTR_TEST_RX_BREAK_ERR_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_TIMEOUT_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_RX_PARITY_ERR_RESVAL	logic [0:0]	1'h 0
UART_INTR_TEST_TX_EMPTY_RESVAL	logic [0:0]	1'h 0
UART_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
UART_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
UART_STATUS_RESVAL	logic [5:0]	6'h 3c
UART_STATUS_TXEMPTY_RESVAL	logic [0:0]	1'h 1
UART_STATUS_TXIDLE_RESVAL	logic [0:0]	1'h 1
UART_STATUS_RXIDLE_RESVAL	logic [0:0]	1'h 1
UART_STATUS_RXEMPTY_RESVAL	logic [0:0]	1'h 1
UART_RDATA_RESVAL	logic [7:0]	8'h 0
UART_FIFO_STATUS_RESVAL	logic [23:0]	24'h 0
UART_VAL_RESVAL	logic [15:0]	16'h 0

API — uart_reg_top.sv

Table 387. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — module / interface / package (1개)

이름	종류
uart_reg_top	module

Table 388. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacL	bit	1'b0
RacLErrorRsp	bit	1'b1

Table 389. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	6
DW	int	32
DBW	int	DW/8

Table 390. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t

포트	방향	형/폭
reg2hw	output	uart_reg_pkg::uart_reg2hw_t
hw2reg	input	uart_reg_pkg::uart_hw2reg_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

API — uart_rx.sv

Table 391. lowRISC_opentitan/hw/ip/uart/rtl/uart_rx.sv — module / interface / package (1개)

이름	종류
uart_rx	module

Table 392. lowRISC_opentitan/hw/ip/uart/rtl/uart_rx.sv — 포트 (13개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rx_enable	input	
tick_baud_x16	input	
parity_enable	input	
parity_odd	input	
tick_baud	output	logic
rx_valid	output	logic
rx_data	output	[7:0]
idle	output	logic
frame_err	output	
rx_parity_err	output	
rx	input	

API — uart_tx.sv

Table 393. lowRISC_opentitan/hw/ip/uart/rtl/uart_tx.sv — module / interface / package (1개)

이름	종류
uart_tx	module

Table 394. lowRISC_opentitan/hw/ip/uart/rtl/uart_tx.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tx_enable	input	

포트	방향	형/폭
tick_baud_x16	input	
parity_enable	input	logic
wr	input	
wr_parity	input	logic
wr_data	input	[7:0]
idle	output	
tx	output	logic

1. lowRISC_opentitan/hw/ip/uart/rtl/uart.sv (144 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART top level wrapper file
6
7 `include "prim_assert.sv"
8
9 module uart
10     import uart_reg_pkg::*;
11
12     #(
13         parameter logic [NumAlerts-1:0]      AlertAsyn-
14         cOn                                     = {NumAlerts{1'b1}},
15         // Number of cycles a differential skew is tolerated on the
16         alert signal
17         parameter int unsigned                AlertSkew-
18         Cycles                                = 1,
19         parameter bit                        EnableR-
20         acl                                   = 1'b0,
21         parameter bit                        RaclEr-
22         rorRsp                               = EnableRacl,
23         parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
24         cySelVec[NumRegs] = '{NumRegs{0}}
25     ) (
26         input          clk_i,
27         input          rst_ni,
28
29         // Bus Interface
30         input  tlul_pkg::tl_h2d_t tl_i,
31         output tlul_pkg::tl_d2h_t tl_o,
32
33         // Alerts
34         input  prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
35         alert_rx_i,
36         output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
37         alert_tx_o,
38
39         // RACL interface
40         input  top_racl_pkg::racl_policy_vec_t racl_policies_i,
41         output top_racl_pkg::racl_error_log_t  racl_error_o,
42
43         output logic      lsio_trigger_o,
44
45         // Generic IO
46         input            cio_rx_i,
47         output logic      cio_tx_o,
48         output logic      cio_tx_en_o,
49
50         // Interrupts
51         output logic      intr_tx_watermark_o ,
52         output logic      intr_tx_empty_o ,
53         output logic      intr_rx_watermark_o ,
54         output logic      intr_tx_done_o ,
55         output logic      intr_rx_overflow_o ,
56         output logic      intr_rx_frame_err_o ,
57         output logic      intr_rx_break_err_o ,
58         output logic      intr_rx_timeout_o ,
59         output logic      intr_rx_parity_err_o
60     );
61
62     logic [NumAlerts-1:0] alert_test, alerts;
63     uart_reg2hw_t reg2hw;
64     uart_hw2reg_t hw2reg;
65
66     uart_reg_top #(
67         .EnableRacl(EnableRacl),
68         .RaclErrorRsp(RaclErrorRsp),
69         .RaclPolicySelVec(RaclPolicySelVec)
70     ) u_reg (
71         .clk_i,
72         .rst_ni,

```

```

64     .tl_i,
65     .tl_o,
66     .reg2hw,
67     .hw2reg,
68     .racl_policies_i,
69     .racl_error_o,
70     // SEC_CM: BUS_INTEGRITY
71     .intg_err_o (alerts[0])
72 );
73
74 uart_core uart_core (
75     .clk_i,
76     .rst_ni,
77     .reg2hw,
78     .hw2reg,
79
80     .rx      (cio_rx_i  ),
81     .tx      (cio_tx_o  ),
82
83     .lsio_trigger_o,
84
85     .intr_tx_watermark_o,
86     .intr_tx_empty_o,
87     .intr_rx_watermark_o,
88     .intr_tx_done_o,
89     .intr_rx_overflow_o,
90     .intr_rx_frame_err_o,
91     .intr_rx_break_err_o,
92     .intr_rx_timeout_o,
93     .intr_rx_parity_err_o
94 );
95
96 // Alerts
97 assign alert_test = {
98     reg2hw.alert_test.q &
99     reg2hw.alert_test.qe
100 };
101
102 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
103     prim_alert_sender #(
104         .AsyncOn(AlertAsyncOn[i]),
105         .SkewCycles(AlertSkewCycles),
106         .IsFatal(1'b1)
107     ) u_prim_alert_sender (
108         .clk_i,
109         .rst_ni,
110         .alert_test_i ( alert_test[i] ),
111         .alert_req_i   ( alerts[i] ),
112         .alert_ack_o   (             ),
113         .alert_state_o (             ),
114         .alert_rx_i    ( alert_rx_i[i] ),
115         .alert_tx_o    ( alert_tx_o[i] )
116     );
117 end
118
119 // always enable the driving out of TX
120 assign cio_tx_en_o = 1'b1;
121
122 // Assert Known for outputs
123 `ASSERT(TxEnIsOne_A, cio_tx_en_o === 1'b1)
124 `ASSERT_KNOWN(TxKnown_A, cio_tx_o, clk_i, !rst_ni || !
125 cio_tx_en_o)
126
127 // Assert Known for alerts
128 `ASSERT_KNOWN(AlertsKnown_A, alert_tx_o)
129
130 // Assert Known for interrupts
131 `ASSERT_KNOWN(TxWatermarkKnown_A, intr_tx_watermark_o)
132 `ASSERT_KNOWN(TxEmptyKnown_A, intr_tx_empty_o)
133 `ASSERT_KNOWN(RxWatermarkKnown_A, intr_rx_watermark_o)
134 `ASSERT_KNOWN(TxDoneKnown_A, intr_tx_done_o)
135 `ASSERT_KNOWN(RxOverflowKnown_A, intr_rx_overflow_o)
136 `ASSERT_KNOWN(RxFrameErrKnown_A, intr_rx_frame_err_o)
137 `ASSERT_KNOWN(RxBreakErrKnown_A, intr_rx_break_err_o)
138 `ASSERT_KNOWN(RxTimeoutKnown_A, intr_rx_timeout_o)
139 `ASSERT_KNOWN(RxParityErrKnown_A, intr_rx_parity_err_o)
140 `ASSERT_KNOWN(LsioTriggerKnown_A, lsio_trigger_o)

```

```

140 `ASSERT_KNOWN(RaclErrorKnown_A, racl_error_o.valid)
141
142 // Alert assertions for reg_we onehot check
143 `AS-
144 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
144 endmodule

```

2. lowRISC_opentitan/hw/ip/uart/rtl/uart_core.sv (525 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART core module
6 //
7
8 `include "prim_assert.sv"
9
10 module uart_core (
11     input                clk_i,
12     input                rst_ni,
13
14     input  uart_reg_pkg::uart_reg2hw_t reg2hw,
15     output uart_reg_pkg::uart_hw2reg_t hw2reg,
16
17     input                rx,
18     output logic         tx,
19
20     output logic         lsio_trigger_o,
21
22     output logic         intr_tx_watermark_o,
23     output logic         intr_tx_empty_o,
24     output logic         intr_rx_watermark_o,
25     output logic         intr_tx_done_o,
26     output logic         intr_rx_overflow_o,
27     output logic         intr_rx_frame_err_o,
28     output logic         intr_rx_break_err_o,
29     output logic         intr_rx_timeout_o,
30     output logic         intr_rx_parity_err_o
31 );
32
33 import uart_reg_pkg::*;
34
35 localparam int NcoWidth = $bits(reg2hw.ctrl.nco.q);
36 localparam int TxFifoDepthW = $clog2(TxFifoDepth)+1;
37 localparam int RxFifoDepthW = $clog2(RxFifoDepth)+1;
38
39 // The design does not support FIFOs deeper than 255 ele-
ments with the current CSR layout.
40 `ASSERT_INIT(TxFifoDepth_A, TxFifoDepth < 256)
41 `ASSERT_INIT(RxFifoDepth_A, RxFifoDepth < 256)
42
43 logic [15:0] rx_val_q;
44 logic [7:0]  uart_rdata;
45 logic        tick_baud_x16, rx_tick_baud;
46
47 logic [TxFifoDepthW-1:0] tx_fifo_depth;
48 logic [RxFifoDepthW-1:0] rx_fifo_depth;
49 logic [RxFifoDepthW-1:0] rx_fifo_depth_prev_q;
50
51 logic [23:0] rx_timeout_count_d, rx_timeout_count_q,
uart_rxto_val;
52 logic        rx_fifo_depth_changed, uart_rxto_en;
53 logic        tx_enable, rx_enable;
54 logic        sys_loopback, line_loopback, rxnf_enable;
55 logic        uart_fifo_rrst, uart_fifo_txrst;
56 logic [2:0]  uart_fifo_rxilvl;
57 logic [2:0]  uart_fifo_txilvl;
58 logic        ovr_d_tx_en, ovr_d_tx_val;
59 logic [7:0]  tx_fifo_data;
60 logic        tx_fifo_rready, tx_fifo_rvalid;
61 logic        tx_fifo_wready, tx_uart_idle;
62 logic        tx_out;
63 logic        tx_out_q;
64 logic [7:0]  rx_fifo_data;
65 logic        rx_valid, rx_fifo_wvalid, rx_fifo_rvalid;
66 logic        rx_fifo_wready, rx_uart_idle;
67 logic        rx_sync;
68 logic        rx_in;
69 logic        break_err;
70 logic [4:0]  allzero_cnt_d, allzero_cnt_q;
71 logic        allzero_err, not_allzero_char;
72 logic        event_tx_watermark, event_tx_empty,
event_rx_watermark, event_tx_done;
73 logic        event_rx_overflow, event_rx_frame_err,
event_rx_break_err, event_rx_timeout;
74 logic        event_rx_parity_err;
75 logic        tx_uart_idle_q;
76
77 assign tx_enable = reg2hw.ctrl.tx.q;
78 assign rx_enable = reg2hw.ctrl.rx.q;
79 assign rxnf_enable = reg2hw.ctrl.nf.q;
80 assign sys_loopback = reg2hw.ctrl.slpbk.q;
81 assign line_loopback = reg2hw.ctrl.llpbk.q;
82
83 assign uart_fifo_rrst = reg2hw.fifo_ctrl.rrst.q &
reg2hw.fifo_ctrl.rrst.qe;

```

```

84 assign uart_fifo_txrst = reg2hw.fifo_ctrl.txrst.q &
reg2hw.fifo_ctrl.txrst.qe;
85 assign uart_fifo_rxilvl = reg2hw.fifo_ctrl.rxilvl.q;
86 assign uart_fifo_txilvl = reg2hw.fifo_ctrl.txilvl.q;
87
88 assign ovr_d_tx_en = reg2hw.ovrd.txen.q;
89 assign ovr_d_tx_val = reg2hw.ovrd.txval.q;
90
91 typedef enum logic {
92     BRK_CHK,
93     BRK_WAIT
94 } break_st_e ;
95
96 break_st_e break_st_q;
97
98 assign not_allzero_char = rx_valid & (~event_rx_frame_err |
(rx_fifo_data != 8'h0));
99 assign allzero_err = event_rx_frame_err & (rx_fifo_data ==
8'h0);
100
101
102 assign allzero_cnt_d = (break_st_q == BRK_WAIT ||
not_allzero_char) ? 5'h0 :
103
104 //allzero_cnt_q[4] never be 1b
105 //allzero_cnt_q[4] ? allzero_cnt_q :
106 //allzero_err ? allzero_cnt_q + 5'd1 :
107 //allzero_cnt_q;
108
109 always_ff @(posedge clk_i or negedge rst_ni) begin
110     if (!rst_ni) allzero_cnt_q <= '0;
111     else if (rx_enable) allzero_cnt_q <= allzero_cnt_d;
112 end
113
114 // break_err edges in same cycle as event_rx_frame_err
edges ; that way the
115 // reset-on-read works the same way for break and frame er-
rors interrupts.
116
117 always_comb begin
118     unique case (reg2hw.ctrl.rxbvl.q)
119         2'h0: break_err = allzero_cnt_d >= 5'd2;
120         2'h1: break_err = allzero_cnt_d >= 5'd4;
121         2'h2: break_err = allzero_cnt_d >= 5'd8;
122         default: break_err = allzero_cnt_d >= 5'd16;
123     endcase
124 end
125
126 always_ff @(posedge clk_i or negedge rst_ni) begin
127     if (!rst_ni) break_st_q <= BRK_CHK;
128     else begin
129         unique case (break_st_q)
130             BRK_CHK: begin
131                 if (event_rx_break_err) break_st_q <= BRK_WAIT;
132             end
133             BRK_WAIT: begin
134                 if (rx_in) break_st_q <= BRK_CHK;
135             end
136             default: begin
137                 break_st_q <= BRK_CHK;
138             end
139         endcase
140     end
141 end
142
143 assign hw2reg.val.d = rx_val_q;
144
145 assign hw2reg.rdata.d = uart_rdata;
146
147 assign hw2reg.status.rxempty.d = ~rx_fifo_rvalid;
148 assign hw2reg.status.rxidle.d = rx_uart_idle;
149 assign hw2reg.status.txidle.d = tx_uart_idle &
~tx_fifo_rvalid;
150 assign hw2reg.status.txempty.d = ~tx_fifo_rvalid;
151 assign hw2reg.status.rxfull.d = ~rx_fifo_wready;
152 assign hw2reg.status.txfull.d = ~tx_fifo_wready;
153
154 assign hw2reg.fifo_status.txilvl.d = 8'(tx_fifo_depth);
155 assign hw2reg.fifo_status.rxilvl.d = 8'(rx_fifo_depth);
156
157 // resets are self-clearing, so need to update FIFO_CTRL
158 assign hw2reg.fifo_ctrl.rxilvl.de = 1'b0;
159 assign hw2reg.fifo_ctrl.rxilvl.d = 3'h0;
160 assign hw2reg.fifo_ctrl.txilvl.de = 1'b0;
161 assign hw2reg.fifo_ctrl.txilvl.d = 3'h0;
162
163
164 // NCO 16x Baud Generator
165 // output clock rate is:
166 // Fin * (NCO/2**NcoWidth)
167 logic [NcoWidth:0] nco_sum_q; // extra bit to get the
carry
168
169 always_ff @(posedge clk_i or negedge rst_ni) begin
170     if (!rst_ni) begin
171         nco_sum_q <= 17'h0;
172     end else if (tx_enable || rx_enable) begin
173         nco_sum_q <= {1'b0, nco_sum_q[NcoWidth-1:0]} +
{1'b0, reg2hw.ctrl.nco.q[NcoWidth-1:0]};
174     end
175 end

```

```

175 end
176
177 assign tick_baud_x16 = nco_sum_q[16];
178
179 // TX Logic //
180 // TX Logic //
181 // TX Logic //
182
183 assign tx_fifo_rready = tx_uart_idle & tx_fifo_rvalid &
tx_enable;
184
185 prim_fifo_sync #(
186     .Width      (8),
187     .Pass       (1'b0),
188     .Depth      (TxFifoDepth)
189 ) u_uart_txfifo (
190     .clk_i,
191     .rst_ni,
192     .clr_i      (uart_fifo_txrst),
193     .wvalid_i   (reg2hw.wdata.qe),
194     .wready_o   (tx_fifo_wready),
195     .wdata_i    (reg2hw.wdata.q),
196     .depth_o    (tx_fifo_depth),
197     .full_o     (),
198     .rvalid_o   (tx_fifo_rvalid),
199     .rready_i   (tx_fifo_rready),
200     .rdata_o    (tx_fifo_data),
201     .err_o      ()
202 );
203
204 uart_tx uart_tx (
205     .clk_i,
206     .rst_ni,
207     .tx_enable,
208     .tick_baud_x16,
209     .parity_enable (reg2hw.ctrl.parity_en.q),
210     .wr            (tx_fifo_rready),
211     .wr_parity     ((^tx_fifo_data) ^
reg2hw.ctrl.parity_odd.q),
212     .wr_data       (tx_fifo_data),
213     .idle          (tx_uart_idle),
214     .tx            (tx_out)
215 );
216
217 assign tx = line_loopback ? rx : tx_out_q ;
218 always_ff @(posedge clk_i or negedge rst_ni) begin
219     if (!rst_ni) begin
220         tx_out_q <= 1'b1;
221     end else if (ovrd_tx_en) begin
222         tx_out_q <= ovrd_tx_val ;
223     end else if (sys_loopback) begin
224         tx_out_q <= 1'b1;
225     end else begin
226         tx_out_q <= tx_out;
227     end
228 end
229
230 // RX Logic //
231 // RX Logic //
232 // RX Logic //
233
234 // sync the incoming data
235 prim_flop_2sync #(
236     .Width(1),
237     .ResetValue(1'b1)
238 ) sync_rx (
239     .clk_i,
240     .rst_ni,
241     .d_i(rx),
242     .q_o(rx_sync)
243 );
244
245 // Based on: en.wikipedia.org/wiki/Repetition_code mentions
the use of a majority filter
246 // in UART to ignore brief noise spikes
247 logic rx_sync_q1, rx_sync_q2, rx_in_mx, rx_in_maj;
248
249 always_ff @(posedge clk_i or negedge rst_ni) begin
250     if (!rst_ni) begin
251         rx_sync_q1 <= 1'b1;
252         rx_sync_q2 <= 1'b1;
253     end else begin
254         rx_sync_q1 <= rx_sync;
255         rx_sync_q2 <= rx_sync_q1;
256     end
257 end
258
259 assign rx_in_maj = (rx_sync & rx_sync_q1) |
260                  (rx_sync & rx_sync_q2) |
261                  (rx_sync_q1 & rx_sync_q2);
262 assign rx_in_mx  = rxnf_enable ? rx_in_maj : rx_sync;
263
264 assign rx_in = sys_loopback ? tx_out :
265               line_loopback ? 1'b1 :
266               rx_in_mx;
267
268 uart_rx uart_rx (
269     .clk_i,
270     .rst_ni,
271     .rx_enable,
272     .tick_baud_x16,

```

```

273     .parity_enable (reg2hw.ctrl.parity_en.q),
274     .parity_odd    (reg2hw.ctrl.parity_odd.q),
275     .tick_baud     (rx_tick_baud),
276     .rx_valid,
277     .rx_data       (rx_fifo_data),
278     .idle          (rx_uart_idle),
279     .frame_err     (event_rx_frame_err),
280     .rx            (rx_in),
281     .rx_parity_err (event_rx_parity_err)
282 );
283
284 assign rx_fifo_wvalid = rx_valid & ~event_rx_frame_err &
~event_rx_parity_err;
285
286 prim_fifo_sync #(
287     .Width      (8),
288     .Pass       (1'b0),
289     .Depth      (RxFifoDepth)
290 ) u_uart_rxfifo (
291     .clk_i,
292     .rst_ni,
293     .clr_i      (uart_fifo_rxrst),
294     .wvalid_i   (rx_fifo_wvalid),
295     .wready_o   (rx_fifo_wready),
296     .wdata_i    (rx_fifo_data),
297     .depth_o    (rx_fifo_depth),
298     .full_o     (),
299     .rvalid_o   (rx_fifo_rvalid),
300     .rready_i   (reg2hw.rdata.re),
301     .rdata_o    (uart_rdata),
302     .err_o      ()
303 );
304
305 always_ff @(posedge clk_i or negedge rst_ni) begin
306     if (!rst_ni) rx_val_q <= 16'h0;
307     else if (tick_baud_x16) rx_val_q <= {rx_val_q[14:0],
rx_in};
308 end
309
310 // Interrupt & Status //
311 // Interrupt & Status //
312 // Interrupt & Status //
313
314 logic [TxFifoDepthW-1:0] tx_watermark_thresh;
315 always_comb begin
316     // Create power of two thresholds.
317     // The threshold saturates at half the FIFO depth.
318     if (uart_fifo_txilvl >= (TxFifoDepthW-2)) begin
319         tx_watermark_thresh = TxFifoDepthW'(TxFifoDepth/2);
320     end else begin
321         tx_watermark_thresh = 1'b1 << uart_fifo_txilvl;
322     end
323     event_tx_watermark = tx_fifo_depth < tx_watermark_thresh;
324 end
325
326 assign event_tx_empty = tx_fifo_depth == '0;
327
328 assign event_tx_done = ~tx_fifo_rvalid & ~tx_uart_idle_q &
tx_uart_idle;
329
330 always_ff @(posedge clk_i or negedge rst_ni) begin
331     if (!rst_ni) begin
332         tx_uart_idle_q <= 1'b1;
333     end else begin
334         tx_uart_idle_q <= tx_uart_idle;
335     end
336 end
337
338 logic [RxFifoDepthW-1:0] rx_watermark_thresh;
339 always_comb begin
340     // Create power of two thresholds.
341     if (uart_fifo_rxilvl > (RxFifoDepthW-1)) begin
342         // This results in the comparison always returning 0 be-
low because RxFifoDepth can
343         // encode depths up to 2*RxFifoDepth-1.
344         rx_watermark_thresh = {RxFifoDepthW{1'b1}};
345     end else if (uart_fifo_rxilvl == (RxFifoDepthW-1)) begin
346         // The maximum valid threshold threshold is an exception
and saturates at RxFifoDepth-2.
347         rx_watermark_thresh = RxFifoDepthW'(RxFifoDepth-2);
348     end else begin
349         rx_watermark_thresh = 1'b1 << uart_fifo_rxilvl;
350     end
351     event_rx_watermark = rx_fifo_depth >= rx_watermark_thresh;
352 end
353
354 // Flop trigger signal to avoid glitches on the output
355 always_ff @(posedge clk_i or negedge rst_ni) begin
356     if (!rst_ni) begin
357         lsio_trigger_o <= 1'b0;
358     end else begin
359         lsio_trigger_o <= event_tx_watermark |
event_rx_watermark;
360     end
361 end
362
363 // rx timeout interrupt
364 assign uart_rxto_en = reg2hw.timeout_ctrl.en.q;
365 assign uart_rxto_val = reg2hw.timeout_ctrl.val.q;
366

```



```

367 assign rx_fifo_depth_changed = (rx_fifo_depth !=
rx_fifo_depth_prev_q);
368
369 assign rx_timeout_count_d =
370 // don't count if timeout feature not enabled ;
371 // will never reach timeout val + lower power
372 (uart_rxto_en == 1'b0) ? 24'd0 :
373 // reset count if timeout interrupt is set
374 event_rx_timeout ? 24'd0 :

375 // reset count upon change in fifo level: covers
both read and receiving a new byte
376 rx_fifo_depth_changed ? 24'd0 :
377 // reset count if no bytes are pending
378 (rx_fifo_depth == '0) ? 24'd0 :
379 // stop the count at timeout value (this will
set the interrupt)
380 // Removed below line as when the timeout
reaches the value,
381 // event occurred, and timeout value reset to
0h.
382 //(rx_timeout_count_q == uart_rxto_val) ?
rx_timeout_count_q :
383 // increment if at rx baud tick
384 rx_tick_baud ?
(rx_timeout_count_q + 24'd1) :
385 rx_timeout_count_q;
386
387 assign event_rx_timeout = (rx_timeout_count_q ==
uart_rxto_val) & uart_rxto_en;
388
389 always_ff @(posedge clk_i or negedge rst_ni) begin
390 if (!rst_ni) begin
391 rx_timeout_count_q <= 24'd0;
392 rx_fifo_depth_prev_q <= '0;
393 end else begin
394 rx_timeout_count_q <= rx_timeout_count_d;
395 rx_fifo_depth_prev_q <= rx_fifo_depth;
396 end
397 end
398
399 assign event_rx_overflow = rx_fifo_wvalid &
~rx_fifo_wready;
400 assign event_rx_break_err = break_err & (break_st_q ==
BRK_CHK);
401
402 // instantiate interrupt hardware primitives
403
404 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_tx_wa-
termark (
405 .clk_i,
406 .rst_ni,
407 .event_intr_i (event_tx_watermark),
408 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.tx_watermark.q),
409 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_watermark.q),
410 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.tx_watermark.qe),
411 .reg2hw_intr_state_q_i
(reg2hw.intr_state.tx_watermark.q),
412 .hw2reg_intr_state_de_o
(hw2reg.intr_state.tx_watermark.de),
413 .hw2reg_intr_state_d_o
(hw2reg.intr_state.tx_watermark.d),
414 .intr_o (intr_tx_watermark_o)
415 );
416
417 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_tx_empty (
418 .clk_i,
419 .rst_ni,
420 .event_intr_i (event_tx_empty),
421 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_empty.q),
422 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_empty.q),
423 .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_empty.qe),
424 .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_empty.q),
425 .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_empty.de),
426 .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_empty.d),
427 .intr_o (intr_tx_empty_o)
428 );
429
430 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_rx_wa-
termark (
431 .clk_i,
432 .rst_ni,
433 .event_intr_i (event_rx_watermark),
434 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_watermark.q),
435 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_watermark.q),
436 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_watermark.qe),
437 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_watermark.q),
438 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_watermark.de),
439 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_watermark.d),
440 .intr_o (intr_rx_watermark_o)
441 );
442
443 prim_intr_hw #(.Width(1)) intr_hw_tx_done (
444 .clk_i,
445 .rst_ni,
446 .event_intr_i (event_tx_done),
447 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.tx_done.q),
448 .reg2hw_intr_test_q_i (reg2hw.intr_test.tx_done.q),
449 .reg2hw_intr_test_qe_i (reg2hw.intr_test.tx_done.qe),
450 .reg2hw_intr_state_q_i (reg2hw.intr_state.tx_done.q),
451 .hw2reg_intr_state_de_o (hw2reg.intr_state.tx_done.de),
452 .hw2reg_intr_state_d_o (hw2reg.intr_state.tx_done.d),
453 .intr_o (intr_tx_done_o)
454 );
455
456 prim_intr_hw #(.Width(1)) intr_hw_rx_overflow (
457 .clk_i,
458 .rst_ni,
459 .event_intr_i (event_rx_overflow),
460 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_overflow.q),
461 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_overflow.q),
462 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_overflow.qe),
463 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_overflow.q),
464 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_overflow.de),
465 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_overflow.d),
466 .intr_o (intr_rx_overflow_o)
467 );
468
469 prim_intr_hw #(.Width(1)) intr_hw_rx_frame_err (
470 .clk_i,
471 .rst_ni,
472 .event_intr_i (event_rx_frame_err),
473 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_frame_err.q),
474 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_frame_err.q),
475 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_frame_err.qe),
476 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_frame_err.q),
477 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_frame_err.de),
478 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_frame_err.d),
479 .intr_o (intr_rx_frame_err_o)
480 );
481
482 prim_intr_hw #(.Width(1)) intr_hw_rx_break_err (
483 .clk_i,
484 .rst_ni,
485 .event_intr_i (event_rx_break_err),
486 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_break_err.q),
487 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_break_err.q),
488 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_break_err.qe),
489 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_break_err.q),
490 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_break_err.de),
491 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_break_err.d),
492 .intr_o (intr_rx_break_err_o)
493 );
494
495 prim_intr_hw #(.Width(1)) intr_hw_rx_timeout (
496 .clk_i,
497 .rst_ni,
498 .event_intr_i (event_rx_timeout),
499 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_timeout.q),
500 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_timeout.q),
501 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_timeout.qe),
502 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_timeout.q),
503 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_timeout.de),
504 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_timeout.d),
505 .intr_o (intr_rx_timeout_o)
506 );
507
508 prim_intr_hw #(.Width(1)) intr_hw_rx_parity_err (
509 .clk_i,
510 .rst_ni,
511 .event_intr_i (event_rx_parity_err),
512 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.rx_parity_err.q),
513 .reg2hw_intr_test_q_i
(reg2hw.intr_test.rx_parity_err.q),
514 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.rx_parity_err.qe),
515 .reg2hw_intr_state_q_i
(reg2hw.intr_state.rx_parity_err.q),
516 .hw2reg_intr_state_de_o
(hw2reg.intr_state.rx_parity_err.de),
517 .hw2reg_intr_state_d_o
(hw2reg.intr_state.rx_parity_err.d),
518 .intr_o (intr_rx_parity_err_o)
519 );
520
521 // unused registers
522 logic unused_reg;
523 assign unused_reg = ^reg2hw.alert_test;
524
525 endmodule

```

3. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_pkg.sv (421 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
  structure
6
7 package uart_reg_pkg;
8
9   // Param list
10  parameter int RxFifoDepth = 64;
11  parameter int TxFifoDepth = 32;
12  parameter int NumAlerts = 1;
13
14  // Address widths within the block
15  parameter int BlockAw = 6;
16
17  // Number of registers for every interface
18  parameter int NumRegs = 13;
19
20  // Alert indices
21  typedef enum int {
22      AlertFatalFaultIdx = 0
23  } uart_alert_idx_t;
24
25  //////////////////////////////////////////////////
26  // Typedefs for registers //
27  //////////////////////////////////////////////////
28
29  typedef struct packed {
30      struct packed {
31          logic q;
32      } tx_empty;
33      struct packed {
34          logic q;
35      } rx_parity_err;
36      struct packed {
37          logic q;
38      } rx_timeout;
39      struct packed {
40          logic q;
41      } rx_break_err;
42      struct packed {
43          logic q;
44      } rx_frame_err;
45      struct packed {
46          logic q;
47      } rx_overflow;
48      struct packed {
49          logic q;
50      } tx_done;
51      struct packed {
52          logic q;
53      } rx_watermark;
54      struct packed {
55          logic q;
56      } tx_watermark;
57  } uart_reg2hw_intr_state_reg_t;
58
59  typedef struct packed {
60      struct packed {
61          logic q;
62      } tx_empty;
63      struct packed {
64          logic q;
65      } rx_parity_err;
66      struct packed {
67          logic q;
68      } rx_timeout;
69      struct packed {
70          logic q;
71      } rx_break_err;
72      struct packed {
73          logic q;
74      } rx_frame_err;
75      struct packed {
76          logic q;
77      } rx_overflow;
78      struct packed {
79          logic q;
80      } tx_done;
81      struct packed {
82          logic q;
83      } rx_watermark;
84      struct packed {
85          logic q;
86      } tx_watermark;
87  } uart_reg2hw_intr_enable_reg_t;
88
89  typedef struct packed {
90      struct packed {
91          logic q;
92          logic qe;
93      } tx_empty;
94
95      struct packed {
96          logic q;
97          logic qe;
98      } rx_parity_err;
99
100     struct packed {
101         logic q;
102         logic qe;
103     } rx_timeout;
104
105     struct packed {
106         logic q;
107         logic qe;
108     } rx_break_err;
109
110     struct packed {
111         logic q;
112         logic qe;
113     } rx_frame_err;
114
115     struct packed {
116         logic q;
117         logic qe;
118     } rx_overflow;
119
120     struct packed {
121         logic q;
122         logic qe;
123     } tx_done;
124
125     struct packed {
126         logic q;
127         logic qe;
128     } rx_watermark;
129
130     struct packed {
131         logic q;
132         logic qe;
133     } tx_watermark;
134 } uart_reg2hw_intr_test_reg_t;
135
136 typedef struct packed {
137     logic q;
138     logic qe;
139 } uart_reg2hw_alert_test_reg_t;
140
141 typedef struct packed {
142     struct packed {
143         logic [15:0] q;
144     } nco;
145     struct packed {
146         logic [1:0] q;
147     } rxblvl;
148     struct packed {
149         logic q;
150     } parity_odd;
151     struct packed {
152         logic q;
153     } parity_en;
154     struct packed {
155         logic q;
156     } llpbk;
157     struct packed {
158         logic q;
159     } slpbk;
160     struct packed {
161         logic q;
162     } nf;
163     struct packed {
164         logic q;
165     } rx;
166     struct packed {
167         logic q;
168     } tx;
169 } uart_reg2hw_ctrl_reg_t;
170
171 typedef struct packed {
172     struct packed {
173         logic q;
174         logic re;
175     } rxempty;
176     struct packed {
177         logic q;
178         logic re;
179     } rxidle;
180     struct packed {
181         logic q;
182         logic re;
183     } txidle;
184     struct packed {
185         logic q;
186         logic re;
187     } txfull;
188     struct packed {
189         logic q;
190         logic re;
191     } txfull;
192 } uart_reg2hw_status_reg_t;
193
194 typedef struct packed {
195     logic [7:0] q;
196     logic re;
197 } uart_reg2hw_rdata_reg_t;

```

```

195 typedef struct packed {
196     logic [7:0] q;
197     logic      qe;
198 } uart_reg2hw_wdata_reg_t;
199
200 typedef struct packed {
201     struct packed {
202         logic [2:0] q;
203         logic      qe;
204     } txlvl;
205     struct packed {
206         logic [2:0] q;
207         logic      qe;
208     } rxlvl;
209     struct packed {
210         logic      q;
211         logic      qe;
212     } txrst;
213     struct packed {
214         logic      q;
215         logic      qe;
216     } rxrst;
217 } uart_reg2hw_fifo_ctrl_reg_t;
218
219 typedef struct packed {
220     struct packed {
221         logic      q;
222     } txval;
223     struct packed {
224         logic      q;
225     } txen;
226 } uart_reg2hw_ovrd_reg_t;
227
228 typedef struct packed {
229     struct packed {
230         logic      q;
231     } en;
232     struct packed {
233         logic [23:0] q;
234     } val;
235 } uart_reg2hw_timeout_ctrl_reg_t;
236
237 typedef struct packed {
238     struct packed {
239         logic      d;
240         logic      de;
241     } tx_empty;
242     struct packed {
243         logic      d;
244         logic      de;
245     } rx_parity_err;
246     struct packed {
247         logic      d;
248         logic      de;
249     } rx_timeout;
250     struct packed {
251         logic      d;
252         logic      de;
253     } rx_break_err;
254     struct packed {
255         logic      d;
256         logic      de;
257     } rx_frame_err;
258     struct packed {
259         logic      d;
260         logic      de;
261     } rx_overflow;
262     struct packed {
263         logic      d;
264         logic      de;
265     } tx_done;
266     struct packed {
267         logic      d;
268         logic      de;
269     } rx_watermark;
270     struct packed {
271         logic      d;
272         logic      de;
273     } tx_watermark;
274 } uart_hw2reg_intr_state_reg_t;
275
276 typedef struct packed {
277     struct packed {
278         logic      d;
279     } rxempty;
280     struct packed {
281         logic      d;
282     } rxidle;
283     struct packed {
284         logic      d;
285     } txidle;
286     struct packed {
287         logic      d;
288     } txempty;
289     struct packed {
290         logic      d;
291     } rxfull;
292     struct packed {
293         logic      d;
294     } txfull;
295 } uart_hw2reg_status_reg_t;

```

```

296
297 typedef struct packed {
298     logic [7:0] d;
299 } uart_hw2reg_rdata_reg_t;
300
301 typedef struct packed {
302     struct packed {
303         logic [2:0] d;
304         logic      de;
305     } txlvl;
306     struct packed {
307         logic [2:0] d;
308         logic      de;
309     } rxlvl;
310 } uart_hw2reg_fifo_ctrl_reg_t;
311
312 typedef struct packed {
313     struct packed {
314         logic [7:0] d;
315     } rxlvl;
316     struct packed {
317         logic [7:0] d;
318     } txlvl;
319 } uart_hw2reg_fifo_status_reg_t;
320
321 typedef struct packed {
322     logic [15:0] d;
323 } uart_hw2reg_val_reg_t;
324
325 // Register -> HW type
326 typedef struct packed {
327     uart_reg2hw_intr_state_reg_t intr_state; // [131:123]
328     uart_reg2hw_intr_enable_reg_t intr_enable; // [122:114]
329     uart_reg2hw_intr_test_reg_t intr_test; // [113:96]
330     uart_reg2hw_alert_test_reg_t alert_test; // [95:94]
331     uart_reg2hw_ctrl_reg_t ctrl; // [93:69]
332     uart_reg2hw_status_reg_t status; // [68:57]
333     uart_reg2hw_rdata_reg_t rdata; // [56:48]
334     uart_reg2hw_wdata_reg_t wdata; // [47:39]
335     uart_reg2hw_fifo_ctrl_reg_t fifo_ctrl; // [38:27]
336     uart_reg2hw_ovrd_reg_t ovrd; // [26:25]
337     uart_reg2hw_timeout_ctrl_reg_t timeout_ctrl; // [24:0]
338 } uart_reg2hw_t;
339
340 // HW -> register type
341 typedef struct packed {
342     uart_hw2reg_intr_state_reg_t intr_state; // [71:54]
343     uart_hw2reg_status_reg_t status; // [53:48]
344     uart_hw2reg_rdata_reg_t rdata; // [47:40]
345     uart_hw2reg_fifo_ctrl_reg_t fifo_ctrl; // [39:32]
346     uart_hw2reg_fifo_status_reg_t fifo_status; // [31:16]
347     uart_hw2reg_val_reg_t val; // [15:0]
348 } uart_hw2reg_t;
349
350 // Register offsets
351 parameter logic [BlockAw-1:0] UART_INTR_STATE_OFFSET = 6'h
0;
352 parameter logic [BlockAw-1:0] UART_INTR_ENABLE_OFFSET = 6'h
4;
353 parameter logic [BlockAw-1:0] UART_INTR_TEST_OFFSET = 6'h
8;
354 parameter logic [BlockAw-1:0] UART_ALERT_TEST_OFFSET = 6'h
c;
355 parameter logic [BlockAw-1:0] UART_CTRL_OFFSET = 6'h
10;
356 parameter logic [BlockAw-1:0] UART_STATUS_OFFSET = 6'h
14;
357 parameter logic [BlockAw-1:0] UART_RDATA_OFFSET = 6'h
18;
358 parameter logic [BlockAw-1:0] UART_WDATA_OFFSET = 6'h
1c;
359 parameter logic [BlockAw-1:0] UART_FIFO_CTRL_OFFSET = 6'h
20;
360 parameter logic [BlockAw-1:0] UART_FIFO_STATUS_OFFSET = 6'h
24;
361 parameter logic [BlockAw-1:0] UART_OVRD_OFFSET = 6'h
28;
362 parameter logic [BlockAw-1:0] UART_VAL_OFFSET = 6'h
2c;
363 parameter logic [BlockAw-1:0] UART_TIMEOUT_CTRL_OFFSET = 6'h
30;
364
365 // Reset values for hwext registers and their fields
366 parameter logic [8:0] UART_INTR_TEST_RESVAL = 9'h
0;
367 parameter logic [0:0] UART_INTR_TEST_TX_WATERMARK_RESVAL =
1'h
0;
368 parameter logic [0:0] UART_INTR_TEST_RX_WATERMARK_RESVAL =
1'h
0;
369 parameter logic [0:0] UART_INTR_TEST_TX_DONE_RESVAL = 1'h
0;
370 parameter logic [0:0] UART_INTR_TEST_RX_OVERFLOW_RESVAL =
1'h
0;
371 parameter logic [0:0] UART_INTR_TEST_RX_FRAME_ERR_RESVAL =
1'h
0;
372 parameter logic [0:0] UART_INTR_TEST_RX_BREAK_ERR_RESVAL =
1'h
0;
373 parameter logic [0:0] UART_INTR_TEST_RX_TIMEOUT_RESVAL = 1'h
0;
374 parameter logic [0:0] UART_INTR_TEST_RX_PARITY_ERR_RESVAL =
1'h
0;
375 parameter logic [0:0] UART_INTR_TEST_TX_EMPTY_RESVAL = 1'h
0;
376 parameter logic [0:0] UART_ALERT_TEST_RESVAL = 1'h
0;
377 parameter logic [0:0] UART_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h
0;
378 parameter logic [5:0] UART_STATUS_RESVAL = 6'h
3c;
379 parameter logic [0:0] UART_STATUS_TXEMPTY_RESVAL = 1'h
1;

```

```

380 parameter logic [0:0] UART_STATUS_TXIDLE_RESVAL = 1'h 1;
381 parameter logic [0:0] UART_STATUS_RXIDLE_RESVAL = 1'h 1;
382 parameter logic [0:0] UART_STATUS_RXEMPTY_RESVAL = 1'h 1;
383 parameter logic [7:0] UART_RDATA_RESVAL = 8'h 0;
384 parameter logic [23:0] UART_FIFO_STATUS_RESVAL = 24'h 0;
385 parameter logic [15:0] UART_VAL_RESVAL = 16'h 0;
386
387 // Register index
388 typedef enum int {
389     UART_INTR_STATE,
390     UART_INTR_ENABLE,
391     UART_INTR_TEST,
392     UART_ALERT_TEST,
393     UART_CTRL,
394     UART_STATUS,
395     UART_RDATA,
396     UART_WDATA,
397     UART_FIFO_CTRL,
398     UART_FIFO_STATUS,
399     UART_OVRD,
400     UART_VAL,
401     UART_TIMEOUT_CTRL
402 } uart_id_e;
403
404 // Register width information to check illegal writes
405 parameter logic [3:0] UART_PERMIT [13] = '{
406     4'b 0011, // index[ 0] UART_INTR_STATE
407     4'b 0011, // index[ 1] UART_INTR_ENABLE
408     4'b 0011, // index[ 2] UART_INTR_TEST
409     4'b 0001, // index[ 3] UART_ALERT_TEST
410     4'b 1111, // index[ 4] UART_CTRL
411     4'b 0001, // index[ 5] UART_STATUS
412     4'b 0001, // index[ 6] UART_RDATA
413     4'b 0001, // index[ 7] UART_WDATA
414     4'b 0001, // index[ 8] UART_FIFO_CTRL
415     4'b 0111, // index[ 9] UART_FIFO_STATUS
416     4'b 0001, // index[10] UART_OVRD
417     4'b 0011, // index[11] UART_VAL
418     4'b 1111 // index[12] UART_TIMEOUT_CTRL
419 };
420
421 endpackage

```

4. lowRISC_opentitan/hw/ip/uart/rtl/uart_reg_top.sv (1,916 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module uart_reg_top
10 # (
11     parameter bit          EnableRacl          = 1'b0,
12     parameter bit          RclErrorRsp         = 1'b1,
13     parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
cySelVec[uart_reg_pkg::NumRegs] =
14     '{uart_reg_pkg::NumRegs{0}}
15 ) (
16     input clk_i,
17     input rst_ni,
18     input tlul_pkg::tl_h2d_t tl_i,
19     output tlul_pkg::tl_d2h_t tl_o,
20     // To HW
21     output uart_reg_pkg::uart_reg2hw_t reg2hw, // Write
22     input  uart_reg_pkg::uart_hw2reg_t hw2reg, // Read
23
24     // RACL interface
25     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
26     output top_racl_pkg::racl_error_log_t  racl_error_o,
27
28     // Integrity check errors
29     output logic intg_err_o
30 );
31
32 import uart_reg_pkg::*;
33
34 localparam int AW = 6;
35 localparam int DW = 32;
36 localparam int DBW = DW/8; // Byte Width
37
38 // register signals
39 logic          reg_we;
40 logic          reg_re;
41 logic [AW-1:0] reg_addr;
42 logic [DW-1:0] reg_wdata;
43 logic [DBW-1:0] reg_be;
44 logic [DW-1:0] reg_rdata;
45 logic          reg_error;
46
47 logic          addrmiss, wr_err;
48
49 logic [DW-1:0] reg_rdata_next;
50 logic reg_busy;
51

```

```

52 tlul_pkg::tl_h2d_t tl_reg_h2d;
53 tlul_pkg::tl_d2h_t tl_reg_d2h;
54
55
56 // incoming payload check
57 logic intg_err;
58 tlul_cmd_intg_chk u_chk (
59     .tl_i(tl_i),
60     .err_o(intg_err)
61 );
62
63 // also check for spurious write enables
64 logic reg_we_err;
65 logic [12:0] reg_we_check;
66 prim_reg_we_check #(
67     .OneHotWidth(13)
68 ) u_prim_reg_we_check (
69     .clk_i(clk_i),
70     .rst_ni(rst_ni),
71     .oh_i  (reg_we_check),
72     .en_i  (reg_we && !addrmiss),
73     .err_o (reg_we_err)
74 );
75
76 logic err_q;
77 always_ff @(posedge clk_i or negedge rst_ni) begin
78     if (!rst_ni) begin
79         err_q <= '0;
80     end else if (intg_err || reg_we_err) begin
81         err_q <= 1'b1;
82     end
83 end
84
85 // integrity error output is permanent and should be used
  for alert generation
86 // register errors are transactional
87 assign intg_err_o = err_q | intg_err | reg_we_err;
88
89 // outgoing integrity generation
90 tlul_pkg::tl_d2h_t tl_o_pre;
91 tlul_rsp_intg_gen #(
92     .EnableRspIntgGen(1),
93     .EnableDataIntgGen(1)
94 ) u_rsp_intg_gen (
95     .tl_i(tl_o_pre),
96     .tl_o(tl_o)
97 );
98
99 assign tl_reg_h2d = tl_i;
100 assign tl_o_pre   = tl_reg_d2h;
101
102 tlul_adapter_reg #(
103     .RegAw(AW),
104     .RegDw(DW),
105     .EnableDataIntgGen(0)
106 ) u_reg_if (
107     .clk_i  (clk_i),
108     .rst_ni (rst_ni),
109
110     .tl_i (tl_reg_h2d),
111     .tl_o (tl_reg_d2h),
112
113     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
114     .intg_error_o(),
115
116     .we_o  (reg_we),
117     .re_o  (reg_re),
118     .addr_o (reg_addr),
119     .wdata_o (reg_wdata),
120     .be_o  (reg_be),
121     .busy_i  (reg_busy),
122     .rdata_i (reg_rdata),
123     // Translate RACL error to TLUL error if enabled
124     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
125 );
126
127 // cdc oversampling signals
128
129 assign reg_rdata = reg_rdata_next;
130 assign reg_error = addrmiss | wr_err | intg_err;
131
132 // Define SW related signals
133 // Format: <reg>_<field>_<wd|we|qs>
134 // or <reg>_<wd|we|qs> if field == 1 or 0
135 logic intr_state_we;
136 logic intr_state_tx_watermark_qs;
137 logic intr_state_rx_watermark_qs;
138 logic intr_state_tx_done_qs;
139 logic intr_state_tx_done_wd;
140 logic intr_state_rx_overflow_qs;
141 logic intr_state_rx_overflow_wd;
142 logic intr_state_rx_frame_err_qs;
143 logic intr_state_rx_frame_err_wd;
144 logic intr_state_rx_break_err_qs;
145 logic intr_state_rx_break_err_wd;
146 logic intr_state_rx_timeout_qs;
147 logic intr_state_rx_timeout_wd;
148 logic intr_state_rx_parity_err_qs;
149 logic intr_state_rx_parity_err_wd;
150 logic intr_state_tx_empty_qs;
151 logic intr_enable_we;

```

```

152 logic intr_enable_tx_watermark_qs;
153 logic intr_enable_tx_watermark_wd;
154 logic intr_enable_rx_watermark_qs;
155 logic intr_enable_rx_watermark_wd;
156 logic intr_enable_tx_done_qs;
157 logic intr_enable_tx_done_wd;
158 logic intr_enable_rx_overflow_qs;
159 logic intr_enable_rx_overflow_wd;
160 logic intr_enable_rx_frame_err_qs;
161 logic intr_enable_rx_frame_err_wd;
162 logic intr_enable_rx_break_err_qs;
163 logic intr_enable_rx_break_err_wd;
164 logic intr_enable_rx_timeout_qs;
165 logic intr_enable_rx_timeout_wd;
166 logic intr_enable_rx_parity_err_qs;
167 logic intr_enable_rx_parity_err_wd;
168 logic intr_enable_tx_empty_qs;
169 logic intr_enable_tx_empty_wd;
170 logic intr_test_we;
171 logic intr_test_tx_watermark_wd;
172 logic intr_test_rx_watermark_wd;
173 logic intr_test_tx_done_wd;
174 logic intr_test_rx_overflow_wd;
175 logic intr_test_rx_frame_err_wd;
176 logic intr_test_rx_break_err_wd;
177 logic intr_test_rx_timeout_wd;
178 logic intr_test_rx_parity_err_wd;
179 logic intr_test_tx_empty_wd;
180 logic alert_test_we;
181 logic alert_test_wd;
182 logic ctrl_we;
183 logic ctrl_tx_qs;
184 logic ctrl_tx_wd;
185 logic ctrl_rx_qs;
186 logic ctrl_rx_wd;
187 logic ctrl_nf_qs;
188 logic ctrl_nf_wd;
189 logic ctrl_slpbk_qs;
190 logic ctrl_slpbk_wd;
191 logic ctrl_llpbk_qs;
192 logic ctrl_llpbk_wd;
193 logic ctrl_parity_en_qs;
194 logic ctrl_parity_en_wd;
195 logic ctrl_parity_odd_qs;
196 logic ctrl_parity_odd_wd;
197 logic [1:0] ctrl_rxbvlv_qs;
198 logic [1:0] ctrl_rxbvlv_wd;
199 logic [15:0] ctrl_nco_qs;
200 logic [15:0] ctrl_nco_wd;
201 logic status_re;
202 logic status_txfull_qs;
203 logic status_rxfull_qs;
204 logic status_txempty_qs;
205 logic status_txidle_qs;
206 logic status_rxidle_qs;
207 logic status_rxempty_qs;
208 logic rdata_re;
209 logic [7:0] rdata_qs;
210 logic wdata_we;
211 logic [7:0] wdata_wd;
212 logic fifo_ctrl_we;
213 logic fifo_ctrl_rxrst_wd;
214 logic fifo_ctrl_txrst_wd;
215 logic [2:0] fifo_ctrl_rxilvl_qs;
216 logic [2:0] fifo_ctrl_rxilvl_wd;
217 logic [2:0] fifo_ctrl_txilvl_qs;
218 logic [2:0] fifo_ctrl_txilvl_wd;
219 logic fifo_status_re;
220 logic [7:0] fifo_status_txlvl_qs;
221 logic [7:0] fifo_status_rxlvl_qs;
222 logic ovrd_we;
223 logic ovrd_txen_qs;
224 logic ovrd_txen_wd;
225 logic ovrd_txval_qs;
226 logic ovrd_txval_wd;
227 logic val_re;
228 logic [15:0] val_qs;
229 logic timeout_ctrl_we;
230 logic [23:0] timeout_ctrl_val_qs;
231 logic [23:0] timeout_ctrl_val_wd;
232 logic timeout_ctrl_en_qs;
233 logic timeout_ctrl_en_wd;
234
235 // Register instances
236 // R[intr_state]: V(False)
237 // F[tx_watermark]: 0:0
238 prim_subreg #(
239     .DW (1),
240     .SwAccess(prim_subreg_pkg::SwAccessR0),
241     .RESVAL (1'h1),
242     .Mubi (1'b0)
243 ) u_intr_state_tx_watermark (
244     .clk_i (clk_i),
245     .rst_ni (rst_ni),
246
247     // from register interface
248     .we (1'b0),
249     .wd ('0'),
250
251     // from internal hardware
252     .de (hw2reg.intr_state.tx_watermark.de),
253     .d (hw2reg.intr_state.tx_watermark.d),
254
255     // to internal hardware
256     .qe (),
257     .q (reg2hw.intr_state.tx_watermark.q),
258     .ds (),
259
260     // to register interface (read)
261     .qs (intr_state_tx_watermark_qs)
262 );
263
264 // F[rx_watermark]: 1:1
265 prim_subreg #(
266     .DW (1),
267     .SwAccess(prim_subreg_pkg::SwAccessR0),
268     .RESVAL (1'h0),
269     .Mubi (1'b0)
270 ) u_intr_state_rx_watermark (
271     .clk_i (clk_i),
272     .rst_ni (rst_ni),
273
274     // from register interface
275     .we (1'b0),
276     .wd ('0'),
277
278     // from internal hardware
279     .de (hw2reg.intr_state.rx_watermark.de),
280     .d (hw2reg.intr_state.rx_watermark.d),
281
282     // to internal hardware
283     .qe (),
284     .q (reg2hw.intr_state.rx_watermark.q),
285     .ds (),
286
287     // to register interface (read)
288     .qs (intr_state_rx_watermark_qs)
289 );
290
291 // F[tx_done]: 2:2
292 prim_subreg #(
293     .DW (1),
294     .SwAccess(prim_subreg_pkg::SwAccessW1C),
295     .RESVAL (1'h0),
296     .Mubi (1'b0)
297 ) u_intr_state_tx_done (
298     .clk_i (clk_i),
299     .rst_ni (rst_ni),
300
301     // from register interface
302     .we (intr_state_we),
303     .wd (intr_state_tx_done_wd),
304
305     // from internal hardware
306     .de (hw2reg.intr_state.tx_done.de),
307     .d (hw2reg.intr_state.tx_done.d),
308
309     // to internal hardware
310     .qe (),
311     .q (reg2hw.intr_state.tx_done.q),
312     .ds (),
313
314     // to register interface (read)
315     .qs (intr_state_tx_done_qs)
316 );
317
318 // F[rx_overflow]: 3:3
319 prim_subreg #(
320     .DW (1),
321     .SwAccess(prim_subreg_pkg::SwAccessW1C),
322     .RESVAL (1'h0),
323     .Mubi (1'b0)
324 ) u_intr_state_rx_overflow (
325     .clk_i (clk_i),
326     .rst_ni (rst_ni),
327
328     // from register interface
329     .we (intr_state_we),
330     .wd (intr_state_rx_overflow_wd),
331
332     // from internal hardware
333     .de (hw2reg.intr_state.rx_overflow.de),
334     .d (hw2reg.intr_state.rx_overflow.d),
335
336     // to internal hardware
337     .qe (),
338     .q (reg2hw.intr_state.rx_overflow.q),
339     .ds (),
340
341     // to register interface (read)
342     .qs (intr_state_rx_overflow_qs)
343 );
344
345 // F[rx_frame_err]: 4:4
346 prim_subreg #(
347     .DW (1),
348     .SwAccess(prim_subreg_pkg::SwAccessW1C),
349     .RESVAL (1'h0),
350     .Mubi (1'b0)
351 ) u_intr_state_rx_frame_err (
352     .clk_i (clk_i),
353     .rst_ni (rst_ni),

```

```

354
355 // from register interface
356 .we (intr_state_we),
357 .wd (intr_state_rx_frame_err_wd),
358
359 // from internal hardware
360 .de (hw2reg.intr_state.rx_frame_err.de),
361 .d (hw2reg.intr_state.rx_frame_err.d),
362
363 // to internal hardware
364 .qe (),
365 .q (reg2hw.intr_state.rx_frame_err.q),
366 .ds (),
367
368 // to register interface (read)
369 .qs (intr_state_rx_frame_err_qs)
370 );
371
372 // F[rx_break_err]: 5:5
373 prim_subreg #(
374 .DW (1),
375 .SwAccess(prim_subreg_pkg::SwAccessW1C),
376 .RESVAL (1'h0),
377 .Mubi (1'b0)
378 ) u_intr_state_rx_break_err (
379 .clk_i (clk_i),
380 .rst_ni (rst_ni),
381
382 // from register interface
383 .we (intr_state_we),
384 .wd (intr_state_rx_break_err_wd),
385
386 // from internal hardware
387 .de (hw2reg.intr_state.rx_break_err.de),
388 .d (hw2reg.intr_state.rx_break_err.d),
389
390 // to internal hardware
391 .qe (),
392 .q (reg2hw.intr_state.rx_break_err.q),
393 .ds (),
394
395 // to register interface (read)
396 .qs (intr_state_rx_break_err_qs)
397 );
398
399 // F[rx_timeout]: 6:6
400 prim_subreg #(
401 .DW (1),
402 .SwAccess(prim_subreg_pkg::SwAccessW1C),
403 .RESVAL (1'h0),
404 .Mubi (1'b0)
405 ) u_intr_state_rx_timeout (
406 .clk_i (clk_i),
407 .rst_ni (rst_ni),
408
409 // from register interface
410 .we (intr_state_we),
411 .wd (intr_state_rx_timeout_wd),
412
413 // from internal hardware
414 .de (hw2reg.intr_state.rx_timeout.de),
415 .d (hw2reg.intr_state.rx_timeout.d),
416
417 // to internal hardware
418 .qe (),
419 .q (reg2hw.intr_state.rx_timeout.q),
420 .ds (),
421
422 // to register interface (read)
423 .qs (intr_state_rx_timeout_qs)
424 );
425
426 // F[rx_parity_err]: 7:7
427 prim_subreg #(
428 .DW (1),
429 .SwAccess(prim_subreg_pkg::SwAccessW1C),
430 .RESVAL (1'h0),
431 .Mubi (1'b0)
432 ) u_intr_state_rx_parity_err (
433 .clk_i (clk_i),
434 .rst_ni (rst_ni),
435
436 // from register interface
437 .we (intr_state_we),
438 .wd (intr_state_rx_parity_err_wd),
439
440 // from internal hardware
441 .de (hw2reg.intr_state.rx_parity_err.de),
442 .d (hw2reg.intr_state.rx_parity_err.d),
443
444 // to internal hardware
445 .qe (),
446 .q (reg2hw.intr_state.rx_parity_err.q),
447 .ds (),
448
449 // to register interface (read)
450 .qs (intr_state_rx_parity_err_qs)
451 );
452
453 // F[tx_empty]: 8:8
454 prim_subreg #(
455 .DW (1),
456 .SwAccess(prim_subreg_pkg::SwAccessR0),
457 .RESVAL (1'h1),
458 .Mubi (1'b0)
459 ) u_intr_state_tx_empty (
460 .clk_i (clk_i),
461 .rst_ni (rst_ni),
462
463 // from register interface
464 .we (1'b0),
465 .wd ('0'),
466
467 // from internal hardware
468 .de (hw2reg.intr_state.tx_empty.de),
469 .d (hw2reg.intr_state.tx_empty.d),
470
471 // to internal hardware
472 .qe (),
473 .q (reg2hw.intr_state.tx_empty.q),
474 .ds (),
475
476 // to register interface (read)
477 .qs (intr_state_tx_empty_qs)
478 );
479
480 // R[intr_enable]: V(False)
481 // F[tx_watermark]: 0:0
482 prim_subreg #(
483 .DW (1),
484 .SwAccess(prim_subreg_pkg::SwAccessRW),
485 .RESVAL (1'h0),
486 .Mubi (1'b0)
487 ) u_intr_enable_tx_watermark (
488 .clk_i (clk_i),
489 .rst_ni (rst_ni),
490
491 // from register interface
492 .we (intr_enable_we),
493 .wd (intr_enable_tx_watermark_wd),
494
495 // from internal hardware
496 .de (1'b0),
497 .d ('0'),
498
499 // to internal hardware
500 .qe (),
501 .q (reg2hw.intr_enable.tx_watermark.q),
502 .ds (),
503
504 // to register interface (read)
505 .qs (intr_enable_tx_watermark_qs)
506 );
507
508 // F[rx_watermark]: 1:1
509 prim_subreg #(
510 .DW (1),
511 .SwAccess(prim_subreg_pkg::SwAccessRW),
512 .RESVAL (1'h0),
513 .Mubi (1'b0)
514 ) u_intr_enable_rx_watermark (
515 .clk_i (clk_i),
516 .rst_ni (rst_ni),
517
518 // from register interface
519 .we (intr_enable_we),
520 .wd (intr_enable_rx_watermark_wd),
521
522 // from internal hardware
523 .de (1'b0),
524 .d ('0'),
525
526 // to internal hardware
527 .qe (),
528 .q (reg2hw.intr_enable.rx_watermark.q),
529 .ds (),
530
531 // to register interface (read)
532 .qs (intr_enable_rx_watermark_qs)
533 );
534
535 // F[tx_done]: 2:2
536 prim_subreg #(
537 .DW (1),
538 .SwAccess(prim_subreg_pkg::SwAccessRW),
539 .RESVAL (1'h0),
540 .Mubi (1'b0)
541 ) u_intr_enable_tx_done (
542 .clk_i (clk_i),
543 .rst_ni (rst_ni),
544
545 // from register interface
546 .we (intr_enable_we),
547 .wd (intr_enable_tx_done_wd),
548
549 // from internal hardware
550 .de (1'b0),
551 .d ('0'),
552
553 // to internal hardware
554 .qe (),

```



```

556 .q      (reg2hw.intr_enable.tx_done.q),
557 .ds      (),
558
559 // to register interface (read)
560 .qs      (intr_enable_tx_done_qs)
561 );
562
563 // F[rx_overflow]: 3:3
564 prim_subreg #(
565     .DW      (1),
566     .SwAccess(prim_subreg_pkg::SwAccessRW),
567     .RESVAL   (1'h0),
568     .Mubi     (1'b0)
569 ) u_intr_enable_rx_overflow (
570     .clk_i    (clk_i),
571     .rst_ni   (rst_ni),
572
573     // from register interface
574     .we       (intr_enable_we),
575     .wd       (intr_enable_rx_overflow_wd),
576
577     // from internal hardware
578     .de       (1'b0),
579     .d         ('0'),
580
581     // to internal hardware
582     .qe       (),
583     .q         (reg2hw.intr_enable.rx_overflow.q),
584     .ds       (),
585
586     // to register interface (read)
587     .qs       (intr_enable_rx_overflow_qs)
588 );
589
590 // F[rx_frame_err]: 4:4
591 prim_subreg #(
592     .DW      (1),
593     .SwAccess(prim_subreg_pkg::SwAccessRW),
594     .RESVAL   (1'h0),
595     .Mubi     (1'b0)
596 ) u_intr_enable_rx_frame_err (
597     .clk_i    (clk_i),
598     .rst_ni   (rst_ni),
599
600     // from register interface
601     .we       (intr_enable_we),
602     .wd       (intr_enable_rx_frame_err_wd),
603
604     // from internal hardware
605     .de       (1'b0),
606     .d         ('0'),
607
608     // to internal hardware
609     .qe       (),
610     .q         (reg2hw.intr_enable.rx_frame_err.q),
611     .ds       (),
612
613     // to register interface (read)
614     .qs       (intr_enable_rx_frame_err_qs)
615 );
616
617 // F[rx_break_err]: 5:5
618 prim_subreg #(
619     .DW      (1),
620     .SwAccess(prim_subreg_pkg::SwAccessRW),
621     .RESVAL   (1'h0),
622     .Mubi     (1'b0)
623 ) u_intr_enable_rx_break_err (
624     .clk_i    (clk_i),
625     .rst_ni   (rst_ni),
626
627     // from register interface
628     .we       (intr_enable_we),
629     .wd       (intr_enable_rx_break_err_wd),
630
631     // from internal hardware
632     .de       (1'b0),
633     .d         ('0'),
634
635     // to internal hardware
636     .qe       (),
637     .q         (reg2hw.intr_enable.rx_break_err.q),
638     .ds       (),
639
640     // to register interface (read)
641     .qs       (intr_enable_rx_break_err_qs)
642 );
643
644 // F[rx_timeout]: 6:6
645 prim_subreg #(
646     .DW      (1),
647     .SwAccess(prim_subreg_pkg::SwAccessRW),
648     .RESVAL   (1'h0),
649     .Mubi     (1'b0)
650 ) u_intr_enable_rx_timeout (
651     .clk_i    (clk_i),
652     .rst_ni   (rst_ni),
653
654     // from register interface
655     .we       (intr_enable_we),
656     .wd       (intr_enable_rx_timeout_wd),
657
658     // from internal hardware
659     .de       (1'b0),
660     .d         ('0'),
661
662     // to internal hardware
663     .qe       (),
664     .q         (reg2hw.intr_enable.rx_timeout.q),
665     .ds       (),
666
667     // to register interface (read)
668     .qs       (intr_enable_rx_timeout_qs)
669 );
670
671 // F[rx_parity_err]: 7:7
672 prim_subreg #(
673     .DW      (1),
674     .SwAccess(prim_subreg_pkg::SwAccessRW),
675     .RESVAL   (1'h0),
676     .Mubi     (1'b0)
677 ) u_intr_enable_rx_parity_err (
678     .clk_i    (clk_i),
679     .rst_ni   (rst_ni),
680
681     // from register interface
682     .we       (intr_enable_we),
683     .wd       (intr_enable_rx_parity_err_wd),
684
685     // from internal hardware
686     .de       (1'b0),
687     .d         ('0'),
688
689     // to internal hardware
690     .qe       (),
691     .q         (reg2hw.intr_enable.rx_parity_err.q),
692     .ds       (),
693
694     // to register interface (read)
695     .qs       (intr_enable_rx_parity_err_qs)
696 );
697
698 // F[tx_empty]: 8:8
699 prim_subreg #(
700     .DW      (1),
701     .SwAccess(prim_subreg_pkg::SwAccessRW),
702     .RESVAL   (1'h0),
703     .Mubi     (1'b0)
704 ) u_intr_enable_tx_empty (
705     .clk_i    (clk_i),
706     .rst_ni   (rst_ni),
707
708     // from register interface
709     .we       (intr_enable_we),
710     .wd       (intr_enable_tx_empty_wd),
711
712     // from internal hardware
713     .de       (1'b0),
714     .d         ('0'),
715
716     // to internal hardware
717     .qe       (),
718     .q         (reg2hw.intr_enable.tx_empty.q),
719     .ds       (),
720
721     // to register interface (read)
722     .qs       (intr_enable_tx_empty_qs)
723 );
724
725 // R[intr_test]: V(True)
726 logic intr_test_qe;
727 logic [8:0] intr_test_flds_we;
728 assign intr_test_qe = &intr_test_flds_we;
729 // F[tx_watermark]: 0:0
730 prim_subreg_ext #(
731     .DW      (1)
732 ) u_intr_test_tx_watermark (
733     .re       (1'b0),
734     .we       (intr_test_we),
735     .wd       (intr_test_tx_watermark_wd),
736     .d         ('0'),
737     .qre      (),
738     .qe       (intr_test_flds_we[0]),
739     .q         (reg2hw.intr_test.tx_watermark.q),
740     .ds       (),
741     .qs       ()
742 );
743 assign reg2hw.intr_test.tx_watermark.qe = intr_test_qe;
744
745 // F[rx_watermark]: 1:1
746 prim_subreg_ext #(
747     .DW      (1)
748 ) u_intr_test_rx_watermark (
749     .re       (1'b0),
750     .we       (intr_test_we),
751     .wd       (intr_test_rx_watermark_wd),
752     .d         ('0'),
753     .qre      (),
754     .qe       (intr_test_flds_we[1]),
755     .q         (reg2hw.intr_test.rx_watermark.q),
756     .ds       (),
757

```



```

758 .qs      ()
759 );
760 assign reg2hw.intr_test.rx_watermark.qe = intr_test_qe;
761
762 // F[tx_done]: 2:2
763 prim_subreg_ext #(
764   .DW      (1)
765 ) u_intr_test_tx_done (
766   .re      (1'b0),
767   .we      (intr_test_we),
768   .wd      (intr_test_tx_done_wd),
769   .d       ('0'),
770   .qre     (),
771   .qe      (intr_test_flds_we[2]),
772   .q       (reg2hw.intr_test.tx_done.q),
773   .ds      (),
774   .qs      ()
775 );
776 assign reg2hw.intr_test.tx_done.qe = intr_test_qe;
777
778 // F[rx_overflow]: 3:3
779 prim_subreg_ext #(
780   .DW      (1)
781 ) u_intr_test_rx_overflow (
782   .re      (1'b0),
783   .we      (intr_test_we),
784   .wd      (intr_test_rx_overflow_wd),
785   .d       ('0'),
786   .qre     (),
787   .qe      (intr_test_flds_we[3]),
788   .q       (reg2hw.intr_test.rx_overflow.q),
789   .ds      (),
790   .qs      ()
791 );
792 assign reg2hw.intr_test.rx_overflow.qe = intr_test_qe;
793
794 // F[rx_frame_err]: 4:4
795 prim_subreg_ext #(
796   .DW      (1)
797 ) u_intr_test_rx_frame_err (
798   .re      (1'b0),
799   .we      (intr_test_we),
800   .wd      (intr_test_rx_frame_err_wd),
801   .d       ('0'),
802   .qre     (),
803   .qe      (intr_test_flds_we[4]),
804   .q       (reg2hw.intr_test.rx_frame_err.q),
805   .ds      (),
806   .qs      ()
807 );
808 assign reg2hw.intr_test.rx_frame_err.qe = intr_test_qe;
809
810 // F[rx_break_err]: 5:5
811 prim_subreg_ext #(
812   .DW      (1)
813 ) u_intr_test_rx_break_err (
814   .re      (1'b0),
815   .we      (intr_test_we),
816   .wd      (intr_test_rx_break_err_wd),
817   .d       ('0'),
818   .qre     (),
819   .qe      (intr_test_flds_we[5]),
820   .q       (reg2hw.intr_test.rx_break_err.q),
821   .ds      (),
822   .qs      ()
823 );
824 assign reg2hw.intr_test.rx_break_err.qe = intr_test_qe;
825
826 // F[rx_timeout]: 6:6
827 prim_subreg_ext #(
828   .DW      (1)
829 ) u_intr_test_rx_timeout (
830   .re      (1'b0),
831   .we      (intr_test_we),
832   .wd      (intr_test_rx_timeout_wd),
833   .d       ('0'),
834   .qre     (),
835   .qe      (intr_test_flds_we[6]),
836   .q       (reg2hw.intr_test.rx_timeout.q),
837   .ds      (),
838   .qs      ()
839 );
840 assign reg2hw.intr_test.rx_timeout.qe = intr_test_qe;
841
842 // F[rx_parity_err]: 7:7
843 prim_subreg_ext #(
844   .DW      (1)
845 ) u_intr_test_rx_parity_err (
846   .re      (1'b0),
847   .we      (intr_test_we),
848   .wd      (intr_test_rx_parity_err_wd),
849   .d       ('0'),
850   .qre     (),
851   .qe      (intr_test_flds_we[7]),
852   .q       (reg2hw.intr_test.rx_parity_err.q),
853   .ds      (),
854   .qs      ()
855 );
856 assign reg2hw.intr_test.rx_parity_err.qe = intr_test_qe;
857
858 // F[tx_empty]: 8:8

```

```

859 prim_subreg_ext #(
860   .DW      (1)
861 ) u_intr_test_tx_empty (
862   .re      (1'b0),
863   .we      (intr_test_we),
864   .wd      (intr_test_tx_empty_wd),
865   .d       ('0'),
866   .qre     (),
867   .qe      (intr_test_flds_we[8]),
868   .q       (reg2hw.intr_test.tx_empty.q),
869   .ds      (),
870   .qs      ()
871 );
872 assign reg2hw.intr_test.tx_empty.qe = intr_test_qe;
873
874 // R[alert_test]: V(True)
875 logic alert_test_qe;
876 logic [0:0] alert_test_flds_we;
877 assign alert_test_qe = &alert_test_flds_we;
878 prim_subreg_ext #(
879   .DW      (1)
880 ) u_alert_test (
881   .re      (1'b0),
882   .we      (alert_test_we),
883   .wd      (alert_test_wd),
884   .d       ('0'),
885   .qre     (),
886   .qe      (alert_test_flds_we[0]),
887   .q       (reg2hw.alert_test.q),
888   .ds      (),
889   .qs      ()
890 );
891 assign reg2hw.alert_test.qe = alert_test_qe;
892
893 // R[ctrl]: V(False)
894 // F[tx]: 0:0
895 prim_subreg #(
896   .DW      (1),
897   .SwAccess(prim_subreg_pkg::SwAccessRW),
898   .RESVAL  (1'h0),
899   .Mubi     (1'b0)
900 ) u_ctrl_tx (
901   .clk_i    (clk_i),
902   .rst_ni   (rst_ni),
903
904   // from register interface
905   .we       (ctrl_we),
906   .wd       (ctrl_tx_wd),
907
908   // from internal hardware
909   .de       (1'b0),
910   .d        ('0'),
911
912   // to internal hardware
913   .qe       (),
914   .q        (reg2hw.ctrl.tx.q),
915   .ds       (),
916
917   // to register interface (read)
918   .qs       (ctrl_tx_qs)
919 );
920
921 // F[rx]: 1:1
922 prim_subreg #(
923   .DW      (1),
924   .SwAccess(prim_subreg_pkg::SwAccessRW),
925   .RESVAL  (1'h0),
926   .Mubi     (1'b0)
927 ) u_ctrl_rx (
928   .clk_i    (clk_i),
929   .rst_ni   (rst_ni),
930
931   // from register interface
932   .we       (ctrl_we),
933   .wd       (ctrl_rx_wd),
934
935   // from internal hardware
936   .de       (1'b0),
937   .d        ('0'),
938
939   // to internal hardware
940   .qe       (),
941   .q        (reg2hw.ctrl.rx.q),
942   .ds       (),
943
944   // to register interface (read)
945   .qs       (ctrl_rx_qs)
946 );
947
948 // F[nf]: 2:2
949 prim_subreg #(
950   .DW      (1),
951   .SwAccess(prim_subreg_pkg::SwAccessRW),
952   .RESVAL  (1'h0),
953   .Mubi     (1'b0)
954 ) u_ctrl_nf (
955   .clk_i    (clk_i),
956   .rst_ni   (rst_ni),

```

```

960 // from register interface
961 .we (ctrl_we),
962 .wd (ctrl_nf_wd),
963
964 // from internal hardware
965 .de (1'b0),
966 .d ('0'),
967
968 // to internal hardware
969 .qe (),
970 .q (reg2hw.ctrl.nf.q),
971 .ds (),
972
973 // to register interface (read)
974 .qs (ctrl_nf_qs)
975 );
976
977 // F[slpbk]: 4:4
978 prim_subreg #(
979 .DW (1),
980 .SwAccess(prim_subreg_pkg::SwAccessRW),
981 .RESVAL (1'h0),
982 .Mubi (1'b0)
983 ) u_ctrl_slpbk (
984 .clk_i (clk_i),
985 .rst_ni (rst_ni),
986
987 // from register interface
988 .we (ctrl_we),
989 .wd (ctrl_slpbk_wd),
990
991 // from internal hardware
992 .de (1'b0),
993 .d ('0'),
994
995 // to internal hardware
996 .qe (),
997 .q (reg2hw.ctrl.slpbk.q),
998 .ds (),
999
1000 // to register interface (read)
1001 .qs (ctrl_slpbk_qs)
1002 );
1003
1004 // F[llpbk]: 5:5
1005 prim_subreg #(
1006 .DW (1),
1007 .SwAccess(prim_subreg_pkg::SwAccessRW),
1008 .RESVAL (1'h0),
1009 .Mubi (1'b0)
1010 ) u_ctrl_llpbk (
1011 .clk_i (clk_i),
1012 .rst_ni (rst_ni),
1013
1014 // from register interface
1015 .we (ctrl_we),
1016 .wd (ctrl_llpbk_wd),
1017
1018 // from internal hardware
1019 .de (1'b0),
1020 .d ('0'),
1021
1022 // to internal hardware
1023 .qe (),
1024 .q (reg2hw.ctrl.llpbk.q),
1025 .ds (),
1026
1027 // to register interface (read)
1028 .qs (ctrl_llpbk_qs)
1029 );
1030
1031 // F[parity_en]: 6:6
1032 prim_subreg #(
1033 .DW (1),
1034 .SwAccess(prim_subreg_pkg::SwAccessRW),
1035 .RESVAL (1'h0),
1036 .Mubi (1'b0)
1037 ) u_ctrl_parity_en (
1038 .clk_i (clk_i),
1039 .rst_ni (rst_ni),
1040
1041 // from register interface
1042 .we (ctrl_we),
1043 .wd (ctrl_parity_en_wd),
1044
1045 // from internal hardware
1046 .de (1'b0),
1047 .d ('0'),
1048
1049 // to internal hardware
1050 .qe (),
1051 .q (reg2hw.ctrl.parity_en.q),
1052 .ds (),
1053
1054 // to register interface (read)
1055 .qs (ctrl_parity_en_qs)
1056 );
1057
1058 // F[parity_odd]: 7:7
1059 prim_subreg #(
1060 .DW (1),

```

```

1061 .SwAccess(prim_subreg_pkg::SwAccessRW),
1062 .RESVAL (1'h0),
1063 .Mubi (1'b0)
1064 ) u_ctrl_parity_odd (
1065 .clk_i (clk_i),
1066 .rst_ni (rst_ni),
1067
1068 // from register interface
1069 .we (ctrl_we),
1070 .wd (ctrl_parity_odd_wd),
1071
1072 // from internal hardware
1073 .de (1'b0),
1074 .d ('0'),
1075
1076 // to internal hardware
1077 .qe (),
1078 .q (reg2hw.ctrl.parity_odd.q),
1079 .ds (),
1080
1081 // to register interface (read)
1082 .qs (ctrl_parity_odd_qs)
1083 );
1084
1085 // F[rxblvl]: 9:8
1086 prim_subreg #(
1087 .DW (2),
1088 .SwAccess(prim_subreg_pkg::SwAccessRW),
1089 .RESVAL (2'h0),
1090 .Mubi (1'b0)
1091 ) u_ctrl_rxblvl (
1092 .clk_i (clk_i),
1093 .rst_ni (rst_ni),
1094
1095 // from register interface
1096 .we (ctrl_we),
1097 .wd (ctrl_rxblvl_wd),
1098
1099 // from internal hardware
1100 .de (1'b0),
1101 .d ('0'),
1102
1103 // to internal hardware
1104 .qe (),
1105 .q (reg2hw.ctrl.rxblvl.q),
1106 .ds (),
1107
1108 // to register interface (read)
1109 .qs (ctrl_rxblvl_qs)
1110 );
1111
1112 // F[nco]: 31:16
1113 prim_subreg #(
1114 .DW (16),
1115 .SwAccess(prim_subreg_pkg::SwAccessRW),
1116 .RESVAL (16'h0),
1117 .Mubi (1'b0)
1118 ) u_ctrl_nco (
1119 .clk_i (clk_i),
1120 .rst_ni (rst_ni),
1121
1122 // from register interface
1123 .we (ctrl_we),
1124 .wd (ctrl_nco_wd),
1125
1126 // from internal hardware
1127 .de (1'b0),
1128 .d ('0'),
1129
1130 // to internal hardware
1131 .qe (),
1132 .q (reg2hw.ctrl.nco.q),
1133 .ds (),
1134
1135 // to register interface (read)
1136 .qs (ctrl_nco_qs)
1137 );
1138
1139 // R[status]: V(True)
1140 // F[txfull]: 0:0
1141 prim_subreg_ext #(
1142 .DW (1)
1143 ) u_status_txfull (
1144 .re (status_re),
1145 .we (1'b0),
1146 .wd ('0'),
1147 .d (hw2reg.status.txfull.d),
1148 .qre (reg2hw.status.txfull.re),
1149 .qe (),
1150 .q (reg2hw.status.txfull.q),
1151 .ds (),
1152 .qs (status_txfull_qs)
1153 );
1154
1155 // F[rxfull]: 1:1
1156 prim_subreg_ext #(
1157 .DW (1)
1158 ) u_status_rxfull (
1159 .re (status_re),
1160 .we (1'b0),

```

```

1162 .wd ('0),
1163 .d (hw2reg.status.rxfull.d),
1164 .qre (reg2hw.status.rxfull.re),
1165 .qe (),
1166 .q (reg2hw.status.rxfull.q),
1167 .ds (),
1168 .qs (status_rxfull_qs)
1169 );
1170
1171 // F[txempty]: 2:2
1172 prim_subreg_ext #(
1173     .DW (1)
1174 ) u_status_txempty (
1175     .re (status_re),
1176     .we (1'b0),
1177     .wd ('0),
1178     .d (hw2reg.status.txempty.d),
1179     .qre (reg2hw.status.txempty.re),
1180     .qe (),
1181     .q (reg2hw.status.txempty.q),
1182     .ds (),
1183     .qs (status_txempty_qs)
1184 );
1185
1186 // F[txidle]: 3:3
1187 prim_subreg_ext #(
1188     .DW (1)
1189 ) u_status_txidle (
1190     .re (status_re),
1191     .we (1'b0),
1192     .wd ('0),
1193     .d (hw2reg.status.txidle.d),
1194     .qre (reg2hw.status.txidle.re),
1195     .qe (),
1196     .q (reg2hw.status.txidle.q),
1197     .ds (),
1198     .qs (status_txidle_qs)
1199 );
1200
1201 // F[rxidle]: 4:4
1202 prim_subreg_ext #(
1203     .DW (1)
1204 ) u_status_rxidle (
1205     .re (status_re),
1206     .we (1'b0),
1207     .wd ('0),
1208     .d (hw2reg.status.rxidle.d),
1209     .qre (reg2hw.status.rxidle.re),
1210     .qe (),
1211     .q (reg2hw.status.rxidle.q),
1212     .ds (),
1213     .qs (status_rxidle_qs)
1214 );
1215
1216 // F[rxempty]: 5:5
1217 prim_subreg_ext #(
1218     .DW (1)
1219 ) u_status_rxempty (
1220     .re (status_re),
1221     .we (1'b0),
1222     .wd ('0),
1223     .d (hw2reg.status.rxempty.d),
1224     .qre (reg2hw.status.rxempty.re),
1225     .qe (),
1226     .q (reg2hw.status.rxempty.q),
1227     .ds (),
1228     .qs (status_rxempty_qs)
1229 );
1230
1231
1232 // R[rdata]: V(True)
1233 prim_subreg_ext #(
1234     .DW (8)
1235 ) u_rdata (
1236     .re (rdata_re),
1237     .we (1'b0),
1238     .wd ('0),
1239     .d (hw2reg.rdata.d),
1240     .qre (reg2hw.rdata.re),
1241     .qe (),
1242     .q (reg2hw.rdata.q),
1243     .ds (),
1244     .qs (rdata_qs)
1245 );
1246
1247
1248 // R[wdata]: V(False)
1249 logic wdata_qe;
1250 logic [0:0] wdata_flds_we;
1251 prim_flop #(
1252     .Width(1),
1253     .ResetValue(0)
1254 ) u_wdata0_qe (
1255     .clk_i (clk_i),
1256     .rst_ni (rst_ni),
1257     .d_i (&wdata_flds_we),
1258     .q_o (wdata_qe)
1259 );
1260 prim_subreg #(
1261     .DW (8),
1262     .SwAccess(prim_subreg_pkg::SwAccessW0),
1263     .RESVAL (8'h0),
1264     .Mubi (1'b0)
1265 ) u_wdata (
1266     .clk_i (clk_i),
1267     .rst_ni (rst_ni),
1268
1269     // from register interface
1270     .we (wdata_we),
1271     .wd (wdata_wd),
1272
1273     // from internal hardware
1274     .de (1'b0),
1275     .d ('0),
1276
1277     // to internal hardware
1278     .qe (wdata_flds_we[0]),
1279     .q (reg2hw.wdata.q),
1280     .ds (),
1281
1282     // to register interface (read)
1283     .qs ()
1284 );
1285 assign reg2hw.wdata.qe = wdata_qe;
1286
1287
1288 // R[fifo_ctrl]: V(False)
1289 logic fifo_ctrl_qe;
1290 logic [3:0] fifo_ctrl_flds_we;
1291 prim_flop #(
1292     .Width(1),
1293     .ResetValue(0)
1294 ) u_fifo_ctrl0_qe (
1295     .clk_i (clk_i),
1296     .rst_ni (rst_ni),
1297     .d_i (&fifo_ctrl_flds_we),
1298     .q_o (fifo_ctrl_qe)
1299 );
1300 // F[rxrst]: 0:0
1301 prim_subreg #(
1302     .DW (1),
1303     .SwAccess(prim_subreg_pkg::SwAccessW0),
1304     .RESVAL (1'h0),
1305     .Mubi (1'b0)
1306 ) u_fifo_ctrl_rxrst (
1307     .clk_i (clk_i),
1308     .rst_ni (rst_ni),
1309
1310     // from register interface
1311     .we (fifo_ctrl_we),
1312     .wd (fifo_ctrl_rxrst_wd),
1313
1314     // from internal hardware
1315     .de (1'b0),
1316     .d ('0),
1317
1318     // to internal hardware
1319     .qe (fifo_ctrl_flds_we[0]),
1320     .q (reg2hw.fifo_ctrl.rxrst.q),
1321     .ds (),
1322
1323     // to register interface (read)
1324     .qs ()
1325 );
1326 assign reg2hw.fifo_ctrl.rxrst.qe = fifo_ctrl_qe;
1327
1328 // F[txrst]: 1:1
1329 prim_subreg #(
1330     .DW (1),
1331     .SwAccess(prim_subreg_pkg::SwAccessW0),
1332     .RESVAL (1'h0),
1333     .Mubi (1'b0)
1334 ) u_fifo_ctrl_txrst (
1335     .clk_i (clk_i),
1336     .rst_ni (rst_ni),
1337
1338     // from register interface
1339     .we (fifo_ctrl_we),
1340     .wd (fifo_ctrl_txrst_wd),
1341
1342     // from internal hardware
1343     .de (1'b0),
1344     .d ('0),
1345
1346     // to internal hardware
1347     .qe (fifo_ctrl_flds_we[1]),
1348     .q (reg2hw.fifo_ctrl.txrst.q),
1349     .ds (),
1350
1351     // to register interface (read)
1352     .qs ()
1353 );
1354 assign reg2hw.fifo_ctrl.txrst.qe = fifo_ctrl_qe;
1355
1356 // F[rxilvl]: 4:2
1357 prim_subreg #(
1358     .DW (3),
1359     .SwAccess(prim_subreg_pkg::SwAccessRW),
1360     .RESVAL (3'h0),
1361     .Mubi (1'b0)
1362 ) u_fifo_ctrl_rxilvl (
1363     .clk_i (clk_i),

```

```

1364 .rst_ni (rst_ni),
1365
1366 // from register interface
1367 .we (fifo_ctrl_we),
1368 .wd (fifo_ctrl_rxilvl_wd),
1369
1370 // from internal hardware
1371 .de (hw2reg.fifo_ctrl_rxilvl.de),
1372 .d (hw2reg.fifo_ctrl_rxilvl.d),
1373
1374 // to internal hardware
1375 .qe (fifo_ctrl_flds_we[2]),
1376 .q (reg2hw.fifo_ctrl_rxilvl.q),
1377 .ds (),
1378
1379 // to register interface (read)
1380 .qs (fifo_ctrl_rxilvl_qs)
1381 );
1382 assign reg2hw.fifo_ctrl_rxilvl.qe = fifo_ctrl_qe;
1383
1384 // F[txilvl]: 7:5
1385 prim_subreg #(
1386 .DW (3),
1387 .SwAccess(prim_subreg_pkg::SwAccessRW),
1388 .RESVAL (3'h0),
1389 .Mubi (1'b0)
1390 ) u_fifo_ctrl_txilvl (
1391 .clk_i (clk_i),
1392 .rst_ni (rst_ni),
1393
1394 // from register interface
1395 .we (fifo_ctrl_we),
1396 .wd (fifo_ctrl_txilvl_wd),
1397
1398 // from internal hardware
1399 .de (hw2reg.fifo_ctrl_txilvl.de),
1400 .d (hw2reg.fifo_ctrl_txilvl.d),
1401
1402 // to internal hardware
1403 .qe (fifo_ctrl_flds_we[3]),
1404 .q (reg2hw.fifo_ctrl_txilvl.q),
1405 .ds (),
1406
1407 // to register interface (read)
1408 .qs (fifo_ctrl_txilvl_qs)
1409 );
1410 assign reg2hw.fifo_ctrl_txilvl.qe = fifo_ctrl_qe;
1411
1412 // R[fifo_status]: V(True)
1413 // F[txlvl]: 7:0
1414 prim_subreg_ext #(
1415 .DW (8)
1416 ) u_fifo_status_txlvl (
1417 .re (fifo_status_re),
1418 .we (1'b0),
1419 .wd ('0'),
1420 .d (hw2reg.fifo_status_txlvl.d),
1421 .gre (),
1422 .qe (),
1423 .q (),
1424 .ds (),
1425 .qs (fifo_status_txlvl_qs)
1426 );
1427
1428 // F[rxlvl]: 23:16
1429 prim_subreg_ext #(
1430 .DW (8)
1431 ) u_fifo_status_rxlvl (
1432 .re (fifo_status_re),
1433 .we (1'b0),
1434 .wd ('0'),
1435 .d (hw2reg.fifo_status_rxlvl.d),
1436 .gre (),
1437 .qe (),
1438 .q (),
1439 .ds (),
1440 .qs (fifo_status_rxlvl_qs)
1441 );
1442
1443 // R[ovrd]: V(False)
1444 // F[txen]: 0:0
1445 prim_subreg #(
1446 .DW (1),
1447 .SwAccess(prim_subreg_pkg::SwAccessRW),
1448 .RESVAL (1'h0),
1449 .Mubi (1'b0)
1450 ) u_ovrd_txen (
1451 .clk_i (clk_i),
1452 .rst_ni (rst_ni),
1453
1454 // from register interface
1455 .we (ovrd_we),
1456 .wd (ovrd_txen_wd),
1457
1458 // from internal hardware
1459 .de (1'b0),
1460 .d ('0'),
1461
1462 // to internal hardware

```

```

1465 .qe (),
1466 .q (reg2hw.ovrd_txen.q),
1467 .ds (),
1468
1469 // to register interface (read)
1470 .qs (ovrd_txen_qs)
1471 );
1472
1473 // F[txval]: 1:1
1474 prim_subreg #(
1475 .DW (1),
1476 .SwAccess(prim_subreg_pkg::SwAccessRW),
1477 .RESVAL (1'h0),
1478 .Mubi (1'b0)
1479 ) u_ovrd_txval (
1480 .clk_i (clk_i),
1481 .rst_ni (rst_ni),
1482
1483 // from register interface
1484 .we (ovrd_we),
1485 .wd (ovrd_txval_wd),
1486
1487 // from internal hardware
1488 .de (1'b0),
1489 .d ('0'),
1490
1491 // to internal hardware
1492 .qe (),
1493 .q (reg2hw.ovrd_txval.q),
1494 .ds (),
1495
1496 // to register interface (read)
1497 .qs (ovrd_txval_qs)
1498 );
1499
1500 // R[val]: V(True)
1501 prim_subreg_ext #(
1502 .DW (16)
1503 ) u_val (
1504 .re (val_re),
1505 .we (1'b0),
1506 .wd ('0'),
1507 .d (hw2reg.val.d),
1508 .gre (),
1509 .qe (),
1510 .q (),
1511 .ds (),
1512 .qs (val_qs)
1513 );
1514
1515 // R[timeout_ctrl]: V(False)
1516 // F[val]: 23:0
1517 prim_subreg #(
1518 .DW (24),
1519 .SwAccess(prim_subreg_pkg::SwAccessRW),
1520 .RESVAL (24'h0),
1521 .Mubi (1'b0)
1522 ) u_timeout_ctrl_val (
1523 .clk_i (clk_i),
1524 .rst_ni (rst_ni),
1525
1526 // from register interface
1527 .we (timeout_ctrl_we),
1528 .wd (timeout_ctrl_val_wd),
1529
1530 // from internal hardware
1531 .de (1'b0),
1532 .d ('0'),
1533
1534 // to internal hardware
1535 .qe (),
1536 .q (reg2hw.timeout_ctrl.val.q),
1537 .ds (),
1538
1539 // to register interface (read)
1540 .qs (timeout_ctrl_val_qs)
1541 );
1542
1543 // F[en]: 31:31
1544 prim_subreg #(
1545 .DW (1),
1546 .SwAccess(prim_subreg_pkg::SwAccessRW),
1547 .RESVAL (1'h0),
1548 .Mubi (1'b0)
1549 ) u_timeout_ctrl_en (
1550 .clk_i (clk_i),
1551 .rst_ni (rst_ni),
1552
1553 // from register interface
1554 .we (timeout_ctrl_we),
1555 .wd (timeout_ctrl_en_wd),
1556
1557 // from internal hardware
1558 .de (1'b0),
1559 .d ('0'),
1560
1561 // to internal hardware
1562 .qe (),
1563 .q (reg2hw.timeout_ctrl.en.q),
1564
1565

```

```

1566 .ds      (),
1567
1568 // to register interface (read)
1569 .qs      (timeout_ctrl_en_qs)
1570 );
1571
1572
1573
1574 logic [12:0] addr_hit;
1575 top_racl_pkg::racl_role_vec_t racl_role_vec;
1576 top_racl_pkg::racl_role_t racl_role;
1577
1578 logic [12:0] racl_addr_hit_read;
1579 logic [12:0] racl_addr_hit_write;
1580
1581 if (EnableRacl) begin : gen_racl_role_logic
1582 // Retrieve RACL role from user bits and one-hot encode
that for the comparison bitmap
1583 assign racl_role =
top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a.user.rsvd);
1584
1585 prim_onehot_enc #(
1586 .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
1587 ) u_racl_role_encode (
1588 .in_i ( racl_role ),
1589 .en_i ( 1'b1 ),
1590 .out_o( racl_role_vec )
1591 );
1592 end else begin : gen_no_racl_role_logic
1593 assign racl_role = '0;
1594 assign racl_role_vec = '0;
1595 end
1596
1597 always_comb begin
1598 racl_addr_hit_read = '0;
1599 racl_addr_hit_write = '0;
1600 addr_hit[ 0] = (reg_addr == UART_INTR_STATE_OFFSET);
1601 addr_hit[ 1] = (reg_addr == UART_INTR_ENABLE_OFFSET);
1602 addr_hit[ 2] = (reg_addr == UART_INTR_TEST_OFFSET);
1603 addr_hit[ 3] = (reg_addr == UART_ALERT_TEST_OFFSET);
1604 addr_hit[ 4] = (reg_addr == UART_CTRL_OFFSET);
1605 addr_hit[ 5] = (reg_addr == UART_STATUS_OFFSET);
1606 addr_hit[ 6] = (reg_addr == UART_RDATA_OFFSET);
1607 addr_hit[ 7] = (reg_addr == UART_WDATA_OFFSET);
1608 addr_hit[ 8] = (reg_addr == UART_FIFO_CTRL_OFFSET);
1609 addr_hit[ 9] = (reg_addr == UART_FIFO_STATUS_OFFSET);
1610 addr_hit[10] = (reg_addr == UART_OVRD_OFFSET);
1611 addr_hit[11] = (reg_addr == UART_VAL_OFFSET);
1612 addr_hit[12] = (reg_addr == UART_TIMEOUT_CTRL_OFFSET);
1613
1614 if (EnableRacl) begin : gen_racl_hit
1615 for (int unsigned slice_idx = 0; slice_idx < 13;
slice_idx++) begin
1616 racl_addr_hit_read[slice_idx] =
1617 addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].read_perm
& racl_role_vec));
1618
1619 racl_addr_hit_write[slice_idx] =
1620 addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].write_perm
& racl_role_vec));
1621
1622 end
1623 end else begin : gen_no_racl
1624 racl_addr_hit_read = addr_hit;
1625 racl_addr_hit_write = addr_hit;
1626 end
1627 end
1628
1629 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
1630 // A valid address hit, access, but failed the RACL check
1631 assign racl_error_o.valid = |addr_hit & ((reg_re & ~|
racl_addr_hit_read) |
1632 (reg_we & ~|
racl_addr_hit_write));
1633 assign racl_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
1634 assign racl_error_o.ctn_uid = racl_role;
1635 assign racl_error_o.overflow = 1'b0;
1636
1637 if (EnableRacl) begin : gen_racl_log
1638 assign racl_error_o.ctn_uid =
top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a.user.rsvd);
1639 assign racl_error_o.read_access = tl_i.a.opcode ==
tlul_pkg::Get;
1640 end else begin : gen_no_racl_log
1641 assign racl_error_o.ctn_uid = '0;
1642 assign racl_error_o.read_access = 1'b0;
1643 end
1644
1645 // Check sub-word write is permitted
1646 always_comb begin
1647 wr_err = (reg_we &
1648 ((racl_addr_hit_write[ 0] & (|(UART_PERMIT[ 0] &
~reg_be))) |
1649 (racl_addr_hit_write[ 1] & (|(UART_PERMIT[ 1] &
~reg_be))) |
1650 (racl_addr_hit_write[ 2] & (|(UART_PERMIT[ 2] &
~reg_be))) |

```

```

1651 (racl_addr_hit_write[ 3] & (|(UART_PERMIT[ 3] &
~reg_be))) |
1652 (racl_addr_hit_write[ 4] & (|(UART_PERMIT[ 4] &
~reg_be))) |
1653 (racl_addr_hit_write[ 5] & (|(UART_PERMIT[ 5] &
~reg_be))) |
1654 (racl_addr_hit_write[ 6] & (|(UART_PERMIT[ 6] &
~reg_be))) |
1655 (racl_addr_hit_write[ 7] & (|(UART_PERMIT[ 7] &
~reg_be))) |
1656 (racl_addr_hit_write[ 8] & (|(UART_PERMIT[ 8] &
~reg_be))) |
1657 (racl_addr_hit_write[ 9] & (|(UART_PERMIT[ 9] &
~reg_be))) |
1658 (racl_addr_hit_write[10] & (|(UART_PERMIT[10] &
~reg_be))) |
1659 (racl_addr_hit_write[11] & (|(UART_PERMIT[11] &
~reg_be))) |
1660 (racl_addr_hit_write[12] & (|(UART_PERMIT[12] &
~reg_be)))));
1661 end
1662
1663 // Generate write-enables
1664 assign intr_state_we = racl_addr_hit_write[0] & reg_we & !
reg_error;
1665
1666 assign intr_state_tx_done_wd = reg_wdata[2];
1667
1668 assign intr_state_rx_overflow_wd = reg_wdata[3];
1669
1670 assign intr_state_rx_frame_err_wd = reg_wdata[4];
1671
1672 assign intr_state_rx_break_err_wd = reg_wdata[5];
1673
1674 assign intr_state_rx_timeout_wd = reg_wdata[6];
1675
1676 assign intr_state_rx_parity_err_wd = reg_wdata[7];
1677 assign intr_enable_we = racl_addr_hit_write[1] & reg_we & !
reg_error;
1678
1679 assign intr_enable_tx_watermark_wd = reg_wdata[0];
1680
1681 assign intr_enable_rx_watermark_wd = reg_wdata[1];
1682
1683 assign intr_enable_tx_done_wd = reg_wdata[2];
1684
1685 assign intr_enable_rx_overflow_wd = reg_wdata[3];
1686
1687 assign intr_enable_rx_frame_err_wd = reg_wdata[4];
1688
1689 assign intr_enable_rx_break_err_wd = reg_wdata[5];
1690
1691 assign intr_enable_rx_timeout_wd = reg_wdata[6];
1692
1693 assign intr_enable_rx_parity_err_wd = reg_wdata[7];
1694
1695 assign intr_enable_tx_empty_wd = reg_wdata[8];
1696 assign intr_test_we = racl_addr_hit_write[2] & reg_we & !
reg_error;
1697
1698 assign intr_test_tx_watermark_wd = reg_wdata[0];
1699
1700 assign intr_test_rx_watermark_wd = reg_wdata[1];
1701
1702 assign intr_test_tx_done_wd = reg_wdata[2];
1703
1704 assign intr_test_rx_overflow_wd = reg_wdata[3];
1705
1706 assign intr_test_rx_frame_err_wd = reg_wdata[4];
1707
1708 assign intr_test_rx_break_err_wd = reg_wdata[5];
1709
1710 assign intr_test_rx_timeout_wd = reg_wdata[6];
1711
1712 assign intr_test_rx_parity_err_wd = reg_wdata[7];
1713
1714 assign intr_test_tx_empty_wd = reg_wdata[8];
1715 assign alert_test_we = racl_addr_hit_write[3] & reg_we & !
reg_error;
1716
1717 assign alert_test_wd = reg_wdata[0];
1718 assign ctrl_we = racl_addr_hit_write[4] & reg_we & !reg_er-
ror;
1719
1720 assign ctrl_tx_wd = reg_wdata[0];
1721
1722 assign ctrl_rx_wd = reg_wdata[1];
1723
1724 assign ctrl_nf_wd = reg_wdata[2];
1725
1726 assign ctrl_slpbk_wd = reg_wdata[4];

```

```

1727
1728 assign ctrl_llpbk_wd = reg_wdata[5];
1729
1730 assign ctrl_parity_en_wd = reg_wdata[6];
1731
1732 assign ctrl_parity_odd_wd = reg_wdata[7];
1733
1734 assign ctrl_rxbvlv_wd = reg_wdata[9:8];
1735
1736 assign ctrl_nco_wd = reg_wdata[31:16];
1737 assign status_re = rac_l_addr_hit_read[5] & reg_re & !reg_er-
ror;
1738 assign rdata_re = rac_l_addr_hit_read[6] & reg_re & !reg_er-
ror;
1739 assign wdata_we = rac_l_addr_hit_write[7] & reg_we & !reg_er-
ror;
1740
1741 assign wdata_wd = reg_wdata[7:0];
1742 assign fifo_ctrl_we = rac_l_addr_hit_write[8] & reg_we & !
reg_error;
1743
1744 assign fifo_ctrl_rxrst_wd = reg_wdata[0];
1745
1746 assign fifo_ctrl_txrst_wd = reg_wdata[1];
1747
1748 assign fifo_ctrl_rxilvl_wd = reg_wdata[4:2];
1749
1750 assign fifo_ctrl_txilvl_wd = reg_wdata[7:5];
1751 assign fifo_status_re = rac_l_addr_hit_read[9] & reg_re & !
reg_error;
1752 assign ovrd_we = rac_l_addr_hit_write[10] & reg_we & !reg_er-
ror;
1753
1754 assign ovrd_txen_wd = reg_wdata[0];
1755
1756 assign ovrd_txval_wd = reg_wdata[1];
1757 assign val_re = rac_l_addr_hit_read[11] & reg_re & !reg_er-
ror;
1758 assign timeout_ctrl_we = rac_l_addr_hit_write[12] & reg_we
& !reg_error;
1759
1760 assign timeout_ctrl_val_wd = reg_wdata[23:0];
1761
1762 assign timeout_ctrl_en_wd = reg_wdata[31];
1763
1764 // Assign write-enables to checker logic vector.
1765 always_comb begin
1766   reg_we_check[0] = intr_state_we;
1767   reg_we_check[1] = intr_enable_we;
1768   reg_we_check[2] = intr_test_we;
1769   reg_we_check[3] = alert_test_we;
1770   reg_we_check[4] = ctrl_we;
1771   reg_we_check[5] = 1'b0;
1772   reg_we_check[6] = 1'b0;
1773   reg_we_check[7] = wdata_we;
1774   reg_we_check[8] = fifo_ctrl_we;
1775   reg_we_check[9] = 1'b0;
1776   reg_we_check[10] = ovrd_we;
1777   reg_we_check[11] = 1'b0;
1778   reg_we_check[12] = timeout_ctrl_we;
1779 end
1780
1781 // Read data return
1782 always_comb begin
1783   reg_rdata_next = '0;
1784   unique case (1'b1)
1785     rac_l_addr_hit_read[0]: begin
1786       reg_rdata_next[0] = intr_state_tx_watermark_qs;
1787       reg_rdata_next[1] = intr_state_rx_watermark_qs;
1788       reg_rdata_next[2] = intr_state_tx_done_qs;
1789       reg_rdata_next[3] = intr_state_rx_overflow_qs;
1790       reg_rdata_next[4] = intr_state_rx_frame_err_qs;
1791       reg_rdata_next[5] = intr_state_rx_break_err_qs;
1792       reg_rdata_next[6] = intr_state_rx_timeout_qs;
1793       reg_rdata_next[7] = intr_state_rx_parity_err_qs;
1794       reg_rdata_next[8] = intr_state_tx_empty_qs;
1795     end
1796
1797     rac_l_addr_hit_read[1]: begin
1798       reg_rdata_next[0] = intr_enable_tx_watermark_qs;
1799       reg_rdata_next[1] = intr_enable_rx_watermark_qs;
1800       reg_rdata_next[2] = intr_enable_tx_done_qs;
1801       reg_rdata_next[3] = intr_enable_rx_overflow_qs;
1802       reg_rdata_next[4] = intr_enable_rx_frame_err_qs;
1803       reg_rdata_next[5] = intr_enable_rx_break_err_qs;
1804       reg_rdata_next[6] = intr_enable_rx_timeout_qs;
1805       reg_rdata_next[7] = intr_enable_rx_parity_err_qs;
1806       reg_rdata_next[8] = intr_enable_tx_empty_qs;
1807     end
1808
1809     rac_l_addr_hit_read[2]: begin
1810       reg_rdata_next[0] = '0;
1811       reg_rdata_next[1] = '0;
1812       reg_rdata_next[2] = '0;
1813       reg_rdata_next[3] = '0;
1814       reg_rdata_next[4] = '0;
1815       reg_rdata_next[5] = '0;
1816       reg_rdata_next[6] = '0;
1817       reg_rdata_next[7] = '0;
1818       reg_rdata_next[8] = '0;
1819     end
1820
1821     rac_l_addr_hit_read[3]: begin
1822       reg_rdata_next[0] = '0;
1823     end
1824
1825     rac_l_addr_hit_read[4]: begin
1826       reg_rdata_next[0] = ctrl_tx_qs;
1827       reg_rdata_next[1] = ctrl_rx_qs;
1828       reg_rdata_next[2] = ctrl_nf_qs;
1829       reg_rdata_next[4] = ctrl_slpbk_qs;
1830       reg_rdata_next[5] = ctrl_llpbk_qs;
1831       reg_rdata_next[6] = ctrl_parity_en_qs;
1832       reg_rdata_next[7] = ctrl_parity_odd_qs;
1833       reg_rdata_next[9:8] = ctrl_rxbvlv_qs;
1834       reg_rdata_next[31:16] = ctrl_nco_qs;
1835     end
1836
1837     rac_l_addr_hit_read[5]: begin
1838       reg_rdata_next[0] = status_txfull_qs;
1839       reg_rdata_next[1] = status_rxfull_qs;
1840       reg_rdata_next[2] = status_txempty_qs;
1841       reg_rdata_next[3] = status_txidle_qs;
1842       reg_rdata_next[4] = status_rxidle_qs;
1843       reg_rdata_next[5] = status_rxempty_qs;
1844     end
1845
1846     rac_l_addr_hit_read[6]: begin
1847       reg_rdata_next[7:0] = rdata_qs;
1848     end
1849
1850     rac_l_addr_hit_read[7]: begin
1851       reg_rdata_next[7:0] = '0;
1852     end
1853
1854     rac_l_addr_hit_read[8]: begin
1855       reg_rdata_next[0] = '0;
1856       reg_rdata_next[1] = '0;
1857       reg_rdata_next[4:2] = fifo_ctrl_rxilvl_qs;
1858       reg_rdata_next[7:5] = fifo_ctrl_txilvl_qs;
1859     end
1860
1861     rac_l_addr_hit_read[9]: begin
1862       reg_rdata_next[7:0] = fifo_status_txlvl_qs;
1863       reg_rdata_next[23:16] = fifo_status_rxlvl_qs;
1864     end
1865
1866     rac_l_addr_hit_read[10]: begin
1867       reg_rdata_next[0] = ovrd_txen_qs;
1868       reg_rdata_next[1] = ovrd_txval_qs;
1869     end
1870
1871     rac_l_addr_hit_read[11]: begin
1872       reg_rdata_next[15:0] = val_qs;
1873     end
1874
1875     rac_l_addr_hit_read[12]: begin
1876       reg_rdata_next[23:0] = timeout_ctrl_val_qs;
1877       reg_rdata_next[31] = timeout_ctrl_en_qs;
1878     end
1879
1880     default: begin
1881       reg_rdata_next = '1;
1882     end
1883   endcase
1884 end
1885
1886 // shadow busy
1887 logic shadow_busy;
1888 assign shadow_busy = 1'b0;
1889
1890 // register busy
1891 assign reg_busy = shadow_busy;
1892
1893 // Unused signal tieoff
1894
1895 // wdata / byte enable are not always fully used
1896 // add a blanket unused statement to handle lint waivers
1897 logic unused_wdata;
1898 assign unused_wdata = ^reg_wdata;
1899 assign unused_be = ^reg_be;
1900 logic unused_policy_sel;
1901 assign unused_policy_sel = ^rac_l_policies_i;
1902
1903 // Assertions for Register Interface
1904 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
1905 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
1906 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>
tlo_pre.d_valid, clk_i, !rst_ni)
1907
1908 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
hot0(addr_hit), clk_i, !rst_ni)
1909
1910 // this is formulated as an assumption such that the FPV
testbenches do disprove this
1911 // property by mistake
1912 `ASSUME(reqParity, tl_reg_h2d.a_valid |>
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
1913
1914 endmodule

```


5. lowRISC_opentitan/hw/ip/uart/rtl/uart_rx.sv (105 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART Receive Module
6 //
7
8 module uart_rx (
9     input          clk_i,
10    input          rst_ni,
11
12    input          rx_enable,
13    input          tick_baud_x16,
14    input          parity_enable,
15    input          parity_odd,
16
17    output logic    tick_baud,
18    output logic    rx_valid,
19    output [7:0]    rx_data,
20    output logic    idle,
21    output logic    frame_err,
22    output          rx_parity_err,
23
24    input          rx
25 );
26
27 logic          rx_valid_q;
28 logic [10:0]   sreg_q, sreg_d;
29 logic [3:0]    bit_cnt_q, bit_cnt_d;
30 logic [3:0]    baud_div_q, baud_div_d;
31 logic          tick_baud_d, tick_baud_q;
32 logic          idle_d, idle_q;
33
34 assign tick_baud = tick_baud_q;
35 assign idle      = idle_q;
36
37 always_ff @(posedge clk_i or negedge rst_ni) begin
38     if (!rst_ni) begin
39         sreg_q <= 11'h0;
40         bit_cnt_q <= 4'h0;
41         baud_div_q <= 4'h0;
42         tick_baud_q <= 1'b0;
43         idle_q <= 1'b1;
44     end else begin
45         sreg_q <= sreg_d;
46         bit_cnt_q <= bit_cnt_d;
47         baud_div_q <= baud_div_d;
48         tick_baud_q <= tick_baud_d;
49         idle_q <= idle_d;
50     end
51 end
52
53 always_comb begin
54     if (!rx_enable) begin
55         sreg_d = 11'h0;
56         bit_cnt_d = 4'h0;
57         baud_div_d = 4'h0;
58         tick_baud_d = 1'b0;
59         idle_d = 1'b1;
60     end else begin
61         tick_baud_d = 1'b0;
62         sreg_d = sreg_q;
63         bit_cnt_d = bit_cnt_q;
64         baud_div_d = baud_div_q;
65         idle_d = idle_q;
66         if (tick_baud_x16) begin
67             {tick_baud_d, baud_div_d} = {1'b0, baud_div_q} + 5'h1;
68         end
69     end
70     if (idle_q && !rx) begin
71         // start of char, sample in the middle of the bit time
72         baud_div_d = 4'd8;
73         tick_baud_d = 1'b0;
74         bit_cnt_d = (parity_enable ? 4'd11 : 4'd10);
75         sreg_d = 11'h0;
76         idle_d = 1'b0;
77     end else if (!idle_q && tick_baud_q) begin
78         if ((bit_cnt_q == (parity_enable ? 4'd11 : 4'd10)) &&
79 rx) begin
80             // must have been a glitch on the input, start bit
81             is not set
82             // in the middle of the bit time, abort
83             idle_d = 1'b1;
84             bit_cnt_d = 4'h0;
85         end else begin
86             sreg_d = {rx, sreg_q[10:1]};
87             bit_cnt_d = bit_cnt_q - 4'h1;
88             idle_d = (bit_cnt_q == 4'h1);
89         end
90     end
91 end
92
93 always_ff @(posedge clk_i or negedge rst_ni) begin
94     if (!rst_ni) rx_valid_q <= 1'b0;

```

```

94     else          rx_valid_q <= tick_baud_q & (bit_cnt_q ==
4'h1);
95
96     end
97
98     assign rx_valid      = rx_valid_q;
99     assign rx_data       = parity_enable ? sreg_q[8:1] :
sreg_q[9:2];
100    // (rx_parity      = sreg_q[9])
101    assign frame_err      = rx_valid_q & ~sreg_q[10];
102    assign rx_parity_err  = parity_enable & rx_valid_q &
(^{sreg_q[9:1], parity_odd});
103
104
105 endmodule

```

6. lowRISC_opentitan/hw/ip/uart/rtl/uart_tx.sv (79 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Description: UART Transmit Module
6 //
7
8 module uart_tx (
9     input          clk_i,
10    input          rst_ni,
11
12    input          tx_enable,
13    input          tick_baud_x16,
14    input logic     parity_enable,
15
16    input          wr,
17    input logic     wr_parity,
18    input [7:0]     wr_data,
19    output          idle,
20
21    output logic     tx
22 );
23
24 logic [3:0] baud_div_q;
25 logic       tick_baud_q;
26
27 logic [3:0] bit_cnt_q, bit_cnt_d;
28 logic [10:0] sreg_q, sreg_d;
29 logic       tx_q, tx_d;
30
31 assign tx = tx_q;
32
33 always_ff @(posedge clk_i or negedge rst_ni) begin
34     if (!rst_ni) begin
35         baud_div_q <= 4'h0;
36         tick_baud_q <= 1'b0;
37     end else if (tick_baud_x16) begin
38         {tick_baud_q, baud_div_q} <= {1'b0, baud_div_q} + 5'h1;
39     end else begin
40         tick_baud_q <= 1'b0;
41     end
42 end
43
44 always_ff @(posedge clk_i or negedge rst_ni) begin
45     if (!rst_ni) begin
46         bit_cnt_q <= 4'h0;
47         sreg_q <= 11'h7ff;
48         tx_q <= 1'b1;
49     end else begin
50         bit_cnt_q <= bit_cnt_d;
51         sreg_q <= sreg_d;
52         tx_q <= tx_d;
53     end
54 end
55
56 always_comb begin
57     if (!tx_enable) begin
58         bit_cnt_d = 4'h0;
59         sreg_d = 11'h7ff;
60         tx_d = 1'b1;
61     end else begin
62         bit_cnt_d = bit_cnt_q;
63         sreg_d = sreg_q;
64         tx_d = tx_q;
65         if (wr) begin
66             sreg_d = {1'b1, (parity_enable ? wr_parity : 1'b1),
67 wr_data, 1'b0};
68             bit_cnt_d = (parity_enable ? 4'd11 : 4'd10);
69         end else if (tick_baud_q && (bit_cnt_q != 4'h0)) begin
70             sreg_d = {1'b1, sreg_q[10:1]};
71             tx_d = sreg_q[0];
72             bit_cnt_d = bit_cnt_q - 4'h1;
73         end
74     end
75 end
76
77 assign idle = (tx_enable) ? (bit_cnt_q == 4'h0) : 1'b1;
78
79 endmodule

```


USB 디바이스

What. USB full-speed 디바이스 컨트롤러. 엔드포인트 · 패킷 처리 · 버스 상태기계.

Why it exists. USB 는 프로토콜 계층이 깊은 블록의 예다. 물리 신호(차동 J/K, SE0) 위에 패킷 · 트랜잭션 · 전송 (transfer) · 열거(enumeration)가 쌓인다. 어느 계층까지 하드웨어가 하고 어디부터 펌웨어가 하는지가 설계 결정이다.

학부 복습 · **EECS concepts.** 계층 프로토콜 · 비트 스타핑 · NRZI 부호화 · 클럭 복원(SOF 동기) · CRC.

석사 수준 · **Practice note.** 계층 분할 결정을 명세에 명시하라. “열거는 펌웨어가 한다”를 안 적으면 고객이 하드웨어가 다 해 줄 것으로 기대한다.

13 files • 4,805 lines

API — usb_consts_pkg.sv

Table 395. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_consts_pkg.sv — module / interface / package (1개)

이름	종류
usb_consts_pkg	package

API — usb_fs_nb_in_pe.sv

Table 396. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — module / interface / package (1개)

이름	종류
usb_fs_nb_in_pe	module

Table 397. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — parameter (3개)

파라미터	형/폭	기본값
NumInEps	logic [4:0]	12
MaxInPktSizeByte	int unsigned	32
AckTimeoutCnt	int unsigned	$((17 * 4) + 6) - 1$

Table 398. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — localparam (3개)

상수	형/폭	값
InEpW	int unsigned	$\$ \log_2(\text{NumInEps})$
PktW	int unsigned	$\$ \log_2(\text{MaxInPktSizeByte})$
AckTimeoutCntW	int unsigned	$\$ \log_2(\text{AckTimeoutCnt})$ // derived parameter

Table 399. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv — 포트 (38개)

포트	방향	형/폭
clk_48mhz_i	input	logic
rst_ni	input	logic

포트	방향	형/폭
link_reset_i	input	logic
link_active_i	input	logic
dev_addr_i	input	logic [6:0]
in_xact_starting_o	output	logic
in_xact_start_ep_o	output	logic [3:0]
in_ep_current_o	output	logic [3:0]
in_ep_rollback_o	output	logic
in_ep_xact_end_o	output	logic
in_ep_get_addr_o	output	logic [PktW - 1:0]
in_ep_data_get_o	output	logic
in_ep_newpkt_o	output	logic
in_ep_enabled_i	input	logic [NumInEps-1:0]
in_ep_stall_i	input	logic [NumInEps-1:0]
in_ep_has_data_i	input	logic [NumInEps-1:0]
in_ep_data_i	input	logic [7:0]
in_ep_data_done_i	input	logic [NumInEps-1:0]
in_ep_iso_i	input	logic [NumInEps-1:0]
in_data_toggle_o	output	logic [NumInEps-1:0]
in_datatog_we_i	input	logic
in_datatog_status_i	input	logic [NumInEps-1:0]
in_datatog_mask_i	input	logic [NumInEps-1:0]
rx_pkt_start_i	input	logic
rx_pkt_end_i	input	logic
rx_pkt_valid_i	input	logic
rx_pid_i	input	logic [3:0]
rx_addr_i	input	logic [6:0]
rx_endp_i	input	logic [3:0]
tx_pkt_start_o	output	logic
tx_pkt_end_i	input	logic
tx_pid_o	output	logic [3:0]
tx_data_avail_o	output	logic
tx_data_get_i	input	logic
tx_data_o	output	logic [7:0]
event_timeout_in_o	output	logic
event_nak_in_o	output	logic

포트	방향	형/폭
event_nodata_in_o	output	logic

API — usb_fs_nb_out_pe.sv

Table 400. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — module / interface / package (1개)

이름	종류
usb_fs_nb_out_pe	module

Table 401. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — parameter (3개)

파라미터	형/폭	기본값
NumOutEps	logic [4:0]	2
MaxOutPktSizeByte	int unsigned	32
AckTimeoutCnt	int unsigned	17 * 4

Table 402. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — localparam (3개)

상수	형/폭	값
OutEpW	int unsigned	\$clog2(NumOutEps)
PktW	int unsigned	\$clog2(MaxOutPktSizeByte)
AckTimeoutCntW	int unsigned	\$clog2(AckTimeoutCnt) // derived parameter

Table 403. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv — 포트 (33개)

포트	방향	형/폭
clk_48mhz_i	input	logic
rst_ni	input	logic
link_reset_i	input	logic
dev_addr_i	input	logic [6:0]
out_ep_current_o	output	logic [3:0]
out_ep_data_put_o	output	logic
out_ep_put_addr_o	output	logic [PktW - 1:0]
out_ep_data_o	output	logic [7:0]
out_ep_newpkt_o	output	logic
out_ep_acked_o	output	logic
out_ep_rollback_o	output	logic
out_ep_setup_o	output	logic [NumOutEps-1:0]
out_ep_enabled_i	input	logic [NumOutEps-1:0]
out_ep_control_i	input	logic [NumOutEps-1:0]
out_ep_full_i	input	logic [NumOutEps-1:0]
out_ep_stall_i	input	logic [NumOutEps-1:0]

포트	방향	형/폭
out_ep_iso_i	input	logic [NumOutEps-1:0]
out_data_toggle_o	output	logic [NumOutEps-1:0]
out_datatog_we_i	input	logic
out_datatog_status_i	input	logic [NumOutEps-1:0]
out_datatog_mask_i	input	logic [NumOutEps-1:0]
rx_pkt_start_i	input	logic
rx_pkt_end_i	input	logic
rx_pkt_valid_i	input	logic
rx_pid_i	input	logic [3:0]
rx_addr_i	input	logic [6:0]
rx_endp_i	input	logic [3:0]
rx_data_put_i	input	logic
rx_data_i	input	logic [7:0]
tx_pkt_start_o	output	logic
tx_pkt_end_i	input	logic
tx_pid_o	output	logic [3:0]
event_datatog_out_o	output	logic

API — usb_fs_nb_pe.sv

Table 404. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — module / interface / package (1개)

이름	종류
usb_fs_nb_pe	module

Table 405. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — parameter (3개)

파라미터	형/폭	기본값
NumOutEps	int unsigned	2
NumInEps	int unsigned	2
MaxPktSizeByte	int unsigned	32

Table 406. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — localparam (1개)

상수	형/폭	값
PktW	int unsigned	\$clog2(MaxPktSizeByte) // derived parameter

Table 407. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv — 포트 (66개) — 앞 60개만 표시(전체 66개, 본문 소스 참조)

포트	방향	형/폭
clk_48mhz_i	input	logic

포트	방향	형/폭
rst_ni	input	logic
link_reset_i	input	logic
link_active_i	input	logic
dev_addr_i	input	logic [6:0]
cfg_eop_single_bit_i	input	logic
cfg_use_diff_rcvr_i	input	logic
cfg_pinflip_i	input	logic
tx_osc_test_mode_i	input	logic
out_data_toggle_o	output	logic [NumOutEps-1:0]
out_datatog_we_i	input	logic
out_datatog_status_i	input	logic [NumOutEps-1:0]
out_datatog_mask_i	input	logic [NumOutEps-1:0]
in_data_toggle_o	output	logic [NumInEps-1:0]
in_datatog_we_i	input	logic
in_datatog_status_i	input	logic [NumInEps-1:0]
in_datatog_mask_i	input	logic [NumInEps-1:0]
diff_rx_ok_i	input	logic
out_ep_current_o	output	logic [3:0]
out_ep_data_put_o	output	logic
out_ep_put_addr_o	output	logic [PktW - 1:0]
out_ep_data_o	output	logic [7:0]
out_ep_newpkt_o	output	logic
out_ep_acked_o	output	logic
out_ep_rollback_o	output	logic
out_ep_setup_o	output	logic [NumOutEps-1:0]
out_ep_enabled_i	input	logic [NumOutEps-1:0]
out_ep_control_i	input	logic [NumOutEps-1:0]
out_ep_full_i	input	logic [NumOutEps-1:0]
out_ep_stall_i	input	logic [NumOutEps-1:0]
out_ep_iso_i	input	logic [NumOutEps-1:0]
in_xact_starting_o	output	logic
in_xact_start_ep_o	output	logic [3:0]
in_ep_current_o	output	logic [3:0]
in_ep_rollback_o	output	logic
in_ep_xact_end_o	output	logic

포트	방향	형/폭
in_ep_get_addr_o	output	logic [PktW - 1:0]
in_ep_data_get_o	output	logic
in_ep_newpkt_o	output	logic
in_ep_enabled_i	input	logic [NumInEps-1:0]
in_ep_stall_i	input	logic [NumInEps-1:0]
in_ep_has_data_i	input	logic [NumInEps-1:0]
in_ep_data_i	input	logic [7:0]
in_ep_data_done_i	input	logic [NumInEps-1:0]
in_ep_iso_i	input	logic [NumInEps-1:0]
sof_detected_o	output	logic
sof_valid_o	output	logic
frame_index_o	output	logic [10:0]
rx_idle_det_o	output	logic
rx_j_det_o	output	logic
rx_crc5_err_o	output	logic
rx_crc16_err_o	output	logic
rx_pid_err_o	output	logic
rx_bitstuff_err_o	output	logic
usb_d_i	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
usb_d_o	output	logic
usb_se0_o	output	logic
usb_dp_o	output	logic

API — usb_fs_rx.sv

Table 408. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv — module / interface / package (1개)

이름	종류
usb_fs_rx	module

Table 409. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv — localparam (4개)

상수	형/폭	값
DT	logic [2:0]	3'b100
DJ	logic [2:0]	3'b010
DK	logic [2:0]	3'b001
SE0	logic [2:0]	3'b000

Table 410. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
link_reset_i	input	logic
cfg_eop_single_bit_i	input	logic
cfg_use_diff_rcvr_i	input	logic
cfg_pinflip_i	input	logic
diff_rx_ok_i	input	logic
usb_d_i	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
tx_en_i	input	logic
bit_strobe_o	output	logic
pkt_start_o	output	logic
pkt_end_o	output	logic
pid_o	output	logic [3:0]
addr_o	output	logic [6:0]
endp_o	output	logic [3:0]
frame_num_o	output	logic [10:0]
rx_data_put_o	output	logic
rx_data_o	output	logic [7:0]
valid_pid_o	output	logic
valid_packet_o	output	logic
rx_idle_det_o	output	logic
rx_j_det_o	output	logic
crc5_error_o	output	logic
crc16_error_o	output	logic
pid_error_o	output	logic
bitstuff_error_o	output	logic

API — usb_fs_tx.sv**Table 411. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv — module / interface / package (1개)**

이름	종류
usb_fs_tx	module

Table 412. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
link_reset_i	input	logic
cfg_pinflip_i	input	logic
tx_osc_test_mode_i	input	logic
bit_strobe_i	input	logic
usb_oe_o	output	logic
usb_d_o	output	logic
usb_se0_o	output	logic
usb_dp_o	output	logic
usb_dn_o	output	logic
pkt_start_i	input	logic
pkt_end_o	output	logic
pid_i	input	logic [3:0]
tx_data_avail_i	input	logic
tx_data_get_o	output	logic
tx_data_i	input	logic [7:0]

Table 413. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv — typedef / using (2개)

새 이름	원래 형
state_e	enum logic [2:0] {Idle, Sync, Pid, DataOrCrc160, Crc161, Eop, OscTest}
out_state_e	enum logic [1:0] {OsIdle, OsWaitByte, OsTransmit}

API — usb_fs_tx_mux.sv

Table 414. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx_mux.sv — module / interface / package (1개)

이름	종류
usb_fs_tx_mux	module

Table 415. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx_mux.sv — 포트 (6개)

포트	방향	형/폭
in_tx_pkt_start_i	input	logic
in_tx_pid_i	input	logic [3:0]
out_tx_pkt_start_i	input	logic
out_tx_pid_i	input	logic [3:0]
tx_pkt_start_o	output	logic
tx_pid_o	output	logic [3:0]

API — usbdev.sv**Table 416. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — module / interface / package (1개)**

이름	종류
usbdev	module

Table 417. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — parameter (4개)

파라미터	형/폭	기본값
Stub	bit	1'b0
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
RcvrWakeTimeUs	int unsigned	1

Table 418. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — localparam (15개)

상수	형/폭	값
SramDw	int	32
SramDepth	int	512
MaxPktSizeByte	int	64
SramAw	int	$\$clog2(SramDepth)$
SizeWidth	int	$\$clog2(MaxPktSizeByte)$
NBuf	int	$(SramDepth * SramDw) / (MaxPktSizeByte * 8)$
NBufWidth	int	$\$clog2(NBuf)$
AVFifoWidth	int	NBufWidth
AVSetupFifoDepth	int	4
AVOutFifoDepth	int	8
RXFifoWidth	int	$NBufWidth + (1 + SizeWidth) + 4 + 1$
RXFifoDepth	int	8
RXFifoDepthW	int	$prim_util_pkg::vbits(RXFifoDepth+1)$
AlertIsFatal	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
RcvrWakeTimeWidth	int	$vbits(RcvrWakeTimeUs + 2)$

Table 419. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clk_aon_i	input	logic
rst_aon_ni	input	logic
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t

포트	방향	형/폭
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
cio_usb_dp_i	input	logic
cio_usb_dn_i	input	logic
usb_rx_d_i	input	logic
cio_usb_dp_o	output	logic
cio_usb_dp_en_o	output	logic
cio_usb_dn_o	output	logic
cio_usb_dn_en_o	output	logic
usb_tx_se0_o	output	logic
usb_tx_d_o	output	logic
cio_sense_i	input	logic
usb_dp_pullup_o	output	logic
usb_dn_pullup_o	output	logic
usb_rx_enable_o	output	logic
usb_tx_use_d_se0_o	output	logic
usb_aon_suspend_req_o	output	logic
usb_aon_wake_ack_o	output	logic
usb_aon_bus_reset_i	input	logic
usb_aon_sense_lost_i	input	logic
usb_aon_bus_not_idle_i	input	logic
usb_aon_wake_detect_active_i	input	logic
usb_ref_val_o	output	logic
usb_ref_pulse_o	output	logic
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t
intr_pkt_received_o	output	logic
intr_pkt_sent_o	output	logic
intr_powered_o	output	logic
intr_disconnected_o	output	logic
intr_host_lost_o	output	logic
intr_link_reset_o	output	logic
intr_link_suspend_o	output	logic
intr_link_resume_o	output	logic
intr_av_out_empty_o	output	logic

포트	방향	형/폭
intr_rx_full_o	output	logic
intr_av_overflow_o	output	logic
intr_link_in_err_o	output	logic
intr_link_out_err_o	output	logic
intr_rx_crc_err_o	output	logic
intr_rx_pid_err_o	output	logic
intr_rx_bitstuff_err_o	output	logic
intr_frame_o	output	logic
intr_av_setup_empty_o	output	logic

API — usbdev_aon_wake.sv

Table 420. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_aon_wake.sv — module / interface / package (1개)

이름	종류
usbdev_aon_wake	module

Table 421. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_aon_wake.sv — 포트 (16개)

포트	방향	형/폭
clk_aon_i	input	logic
rst_aon_ni	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
usb_sense_i	input	logic
usbdev_dppullup_en_i	input	logic
usbdev_dnpullup_en_i	input	logic
wake_ack_aon_i	input	logic
suspend_req_aon_i	input	logic
usb_dppullup_en_o	output	logic
usb_dnpullup_en_o	output	logic
wake_req_aon_o	output	logic
bus_not_idle_aon_o	output	logic
bus_reset_aon_o	output	logic
sense_lost_aon_o	output	logic
wake_detect_active_aon_o	output	logic

API — usbdev_counter.sv

Table 422. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — module / interface / package (1개)

이름	종류
usbdev_counter	module

Table 423. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — parameter (3개)

파라미터	형/폭	기본값
NEndpoints	int unsigned	12
NEvents	int unsigned	1
Width	int unsigned	8

Table 424. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — localparam (1개)

상수	형/폭	값
EpW	int unsigned	prim_util_pkg::vbits(NEndpoints) // derived parameter

Table 425. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
reset_i	input	logic
event_i	input	logic [NEvents-1:0]
ep_i	input	logic [EpW-1:0]
ev_qe_i	input	logic
ev_i	input	logic [NEvents-1:0]
ev_o	output	logic [NEvents-1:0]
endp_qe_i	input	logic
endpoints_i	input	logic [NEndpoints-1:0]
endpoints_o	output	logic [NEndpoints-1:0]

API — usbdev_iomux.sv**Table 426. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_iomux.sv — module / interface / package (1개)**

이름	종류
usbdev_iomux	module

Table 427. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_iomux.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
hw2reg_sense_o	output	usbdev_hw2reg_phy_pins_sense_reg_t
reg2hw_drive_i	input	usbdev_reg2hw_phy_pins_drive_reg_t
usb_rx_d_i	input	logic

포트	방향	형/폭
usb_rx_dp_i	input	logic
usb_rx_dn_i	input	logic
cio_usb_sense_i	input	logic
usb_tx_d_o	output	logic
usb_tx_se0_o	output	logic
usb_tx_dp_o	output	logic
usb_tx_dn_o	output	logic
usb_tx_oe_o	output	logic
usb_dp_pullup_en_o	output	logic
usb_dn_pullup_en_o	output	logic
usb_rx_enable_o	output	logic
usb_rx_d_o	output	logic
usb_rx_dp_o	output	logic
usb_rx_dn_o	output	logic
usb_tx_d_i	input	logic
usb_tx_se0_i	input	logic
usb_tx_dp_i	input	logic
usb_tx_dn_i	input	logic
usb_tx_oe_i	input	logic
usb_dp_pullup_en_i	input	logic
usb_dn_pullup_en_i	input	logic
usb_rx_enable_i	input	logic
usb_pwr_sense_o	output	logic

API — usbdev_linkstate.sv

Table 428. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv — module / interface / package (1개)

이름	종류
usbdev_linkstate	module

Table 429. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv — localparam (3개)

상수	형/폭	값
SUSPEND_TIMEOUT	logic [11:0]	12'd3000
RESET_TIMEOUT	logic [2:0]	3'd3
SOF_TIMEOUT	logic [9:0]	10'd1005

Table 430. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv — 포트 (21개)

포트	방향	형/폭
clk_48mhz_i	input	logic
rst_ni	input	logic
us_tick_i	input	logic
usb_sense_i	input	logic
usb_dp_i	input	logic
usb_dn_i	input	logic
usb_oe_i	input	logic
usb_pullup_en_i	input	logic
rx_idle_det_i	input	logic
rx_j_det_i	input	logic
sof_detected_i	input	logic
resume_link_active_i	input	logic
link_disconnect_o	output	logic
link_powered_o	output	logic
link_reset_o	output	logic
link_active_o	output	logic
link_suspend_o	output	logic
link_resume_o	output	logic
host_lost_o	output	logic
sof_missed_o	output	logic
link_state_o	output	logic [2:0]

API — usbdev_pkg.sv

Table 431. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_pkg.sv — module / interface / package (1개)

이름	종류
usbdev_pkg	package

```

1. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_consts_pkg.sv (66
lines)
1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 package usb_consts_pkg;
6
7 // Definitions for USB Packet Identifier (called PID in the
  spec)
8
9 // Bottom two bits of the PID are type
10 typedef enum logic [1:0] {
11     UsbPidTypeSpecial = 2'b00,
12     UsbPidTypeToken   = 2'b01,
13     UsbPidTypeHandshake = 2'b10,
14     UsbPidTypeData    = 2'b11
15 } usb_pid_type_e;
16
17 // Direct from USB 2.0 Spec Table 8-1
18
19 typedef enum logic [3:0] {
20     // TOKEN
21     UsbPidOut   = 4'b0001,
22     UsbPidIn    = 4'b1001,
23     UsbPidSof   = 4'b0101,
24     UsbPidSetup = 4'b1101,
25     // DATA
26     UsbPidData0 = 4'b0011,
27     UsbPidData1 = 4'b1011,
28     UsbPidData2 = 4'b0111, // Only valid for High Speed
29     UsbPidMDData = 4'b1111, // Only valid for High Speed
30     // HANDSHAKE
31     UsbPidAck   = 4'b0010,
32     UsbPidNak   = 4'b1010,
33     UsbPidStall = 4'b1110,
34     UsbPidNyet  = 4'b0110,
35     // SPECIAL
36     UsbPidPre   = 4'b1100
37 } usb_pid_e;
38
39 // USB SETUP packet commands (USB2.0 Table 9-4)
40 typedef enum logic [7:0] {
41     SetupGetStatus = 8'd0,
42     SetupClearFeature = 8'd1,

```



```

43 SetupSetFeature      = 8'd3,
44 SetupSetAddress      = 8'd5,
45 SetupGetDescriptor    = 8'd6,
46 SetupSetDescriptor    = 8'd7,
47 SetupGetConfiguration = 8'd8,
48 SetupSetConfiguration = 8'd9,
49 SetupGetInterface     = 8'd10,
50 SetupSetInterface     = 8'd11,
51 SetupSynchFrame       = 8'd12
52 } usb_setup_request_e;
53
54 // USB Descriptor Types (USB2.0 Table 9-5)
55 typedef enum logic [7:0] {
56     DscrTypeDevice      = 8'd1,
57     DscrTypeConfiguration = 8'd2,
58     DscrTypeString      = 8'd3,
59     DscrTypeInterface    = 8'd4,
60     DscrTypeEndpoint     = 8'd5,
61     DscrTypeDevQual      = 8'd6,
62     DscrTypeOthrSpd      = 8'd7,
63     DscrTypeIntPwr       = 8'd8
64 } usb_dscr_type_e;
65
66 endpackage

```

2. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_in_pe.sv (431 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
  hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 // USB Full Speed Non-Buffered Protocol Engine for IN end-
  points
7 //
8 // Decode IN requests from host and respond with data if there is
  any
9 //
10 // Based on usb_fs_in_pe.v from the TinyFPGA-Bootloader pro-
  ject but
11 // this version contains no packet buffers
12
13 `include "prim_assert.sv"
14
15 module usb_fs_nb_in_pe #(
16     parameter logic [4:0] NumInEps = 12,
17     parameter int unsigned MaxInPktSizeByte = 32,
18     // Targeting 17 bit times for the timeout. The counter runs
  on the 48 MHz
19     // clock, the SOP delay to rx_pkt_start_i is 1.5 bit times
  (synchronizer flops and K detection),
20     // and the EOP delay to tx_pkt_end_i is essentially 0. This
  timeout originates from Section
21     // 7.1.19.1 of the USB 2.0 specification.
22     parameter int unsigned AckTimeoutCnt = ((17 * 4) + 6) -
  1, // -1 because [N-1:0] counted
23     localparam int unsigned InEpW = $clog2(NumInEps), // derived
  parameter
24     localparam int unsigned PktW = $clog2(MaxInPktSizeByte), //
  derived parameter
25     localparam int unsigned AckTimeoutCntW = $clog2(
  AckTimeoutCnt) // derived parameter
26 ) (
27     input logic clk_48mhz_i,
28     input logic rst_ni,
29     input logic link_reset_i,
30     input logic link_active_i,
31     input logic [6:0] dev_addr_i,
32
33     // Transaction starting
34     // of an IN transaction; capture details
35     output logic [3:0] in_xact_start_ep_o, // End-
  point starting an IN transaction
36
37     // endpoint interface
38     output logic [3:0] in_ep_current_o, // Other sig-
  nals addressed to this ep
39     output logic in_ep_rollback_o, // Bad ter-
  mination, rollback transaction
40     output logic in_ep_xact_end_o, // good ter-
  mination, transaction complete
41     output logic [PktW - 1:0] in_ep_get_addr_o, // Offset
  requested (0..pktlen)
42     output logic in_ep_data_get_o, // Accept
  data (get_addr advances too)
43     output logic in_ep_newpkt_o, // New IN
  packet starting (updates in_ep_current_o)
44     input logic [NumInEps-1:0] in_ep_enabled_i, // Endpoint

```

```

  is enabled and produces handshakes
50     input logic [NumInEps-1:0] in_ep_stall_i, // Endpoint in
  a stall state
51     input logic [NumInEps-1:0] in_ep_has_data_i, // Endpoint
  has data to supply
52     input logic [7:0] in_ep_data_i, // Data for cur-
  rent get_addr
53     input logic [NumInEps-1:0] in_ep_data_done_i, // Set when
  out of data
54     input logic [NumInEps-1:0] in_ep_iso_i, // Configure end-
  point in isochronous mode
55     output logic [NumInEps-1:0] in_data_toggle_o, // Current
  state of IN data toggles
56     input logic in_datatog_we_i, // IN data
  toggles write strobe from software
57     input logic [NumInEps-1:0] in_datatog_status_i, // New
  state of selected IN data toggles
58     input logic [NumInEps-1:0] in_datatog_mask_i, // Which IN EP
  data toggles to modify
59
60     // rx path
61     // rx path
62     // rx path
63     // rx path
64     // rx path
65     // Strobed on reception of packet.
66     input logic rx_pkt_start_i,
67     input logic rx_pkt_end_i,
68     input logic rx_pkt_valid_i,
69
70     // Most recent packet received.
71     input logic [3:0] rx_pid_i,
72     input logic [6:0] rx_addr_i,
73     input logic [3:0] rx_endp_i,
74
75     // tx path
76     // tx path
77     // tx path
78
79     // Strobe to send new packet.
80     output logic tx_pkt_start_o,
81     input logic tx_pkt_end_i,
82     // Packet type to send
83     output logic [3:0] tx_pid_o,
84
85     // Data payload to send if any
86     output logic tx_data_avail_o,
87     input logic tx_data_get_i,
88     output logic [7:0] tx_data_o,
89
90     // event counters
91     // event counters
92     // event counters
93     output logic event_timeout_in_o,
94     output logic event_nak_in_o,
95     output logic event_nodata_in_o
96 );
97
98 // in transaction state machine
99 // in transaction state machine
100 // in transaction state machine
101 // in transaction state machine
102
103 import usb_consts_pkg::*;
104
105 typedef enum logic [2:0] {
106     StIdle,
107     StRcvdIn,
108     StSendData,
109     StWaitTxEnd,
110     StWaitAckStart,
111     StWaitAck
112 } state_in_e;
113
114 state_in_e in_xact_state;
115 state_in_e in_xact_state_next;
116
117 logic in_xact_end;
118
119 assign in_ep_xact_end_o = in_xact_end;
120
121 // data toggle state
122 logic [NumInEps-1:0] data_toggle_q, data_toggle_d;
123
124 // endpoint data buffer
125 logic token_received, setup_token_received,
  in_token_received, ack_received, nak_received;
126 logic more_data_to_send;
127 logic ep_in_hw, ep_active;
128 logic [3:0] in_ep_current_d;
129
130 // More syntax so can compare with enum
131 usb_pid_type_e rx_pid_type;
132 usb_pid_e rx_pid;
133 assign rx_pid_type = usb_pid_type_e'(rx_pid_i[1:0]);
134 assign rx_pid = usb_pid_e'(rx_pid_i);
135 assign token_received =

```

```

136 rx_pkt_end_i &&
137 rx_pkt_valid_i &&
138 rx_pid_type == UsbPidTypeToken &&
139 rx_addr_i == dev_addr_i;
140
141 assign setup_token_received =
142 token_received &&
143 rx_pid == UsbPidSetup;
144
145 assign in_token_received =
146 token_received &&
147 rx_pid == UsbPidIn;
148
149 assign ack_received =
150 rx_pkt_end_i &&
151 rx_pkt_valid_i &&
152 rx_pid == UsbPidAck;
153
154 assign nak_received =
155 rx_pkt_end_i &&
156 rx_pkt_valid_i &&
157 rx_pid == UsbPidNak;
158
159 // Is the specified endpoint actually implemented in hard-
ware?
160 assign ep_in_hw = {1'b0, rx_endp_i} < NumInEps;
161 assign in_ep_current_d = ep_in_hw ? rx_endp_i : '0;
162
163 // Make widths work - in_ep_current_d/in_ep_current_o only
hold implemented endpoint IDs.
164 // These signals can be used to index signals of NumInEps
width.
165 // They are only valid if ep_active is set, i.e., if the
specified endpoint is implemented.
166 logic [InEpW-1:0] in_ep_index;
167 logic [InEpW-1:0] in_ep_index_d;
168 assign in_ep_index = in_ep_current_o[0+:InEpW];
169 assign in_ep_index_d = in_ep_current_d[0+:InEpW];
170
171 // Is the endpoint active?
172 assign ep_active = in_ep_enabled_i[in_ep_index_d] &
ep_in_hw;
173
174 // Retain whether the current IN transmission has associated
data; an Isochronous transaction
175 // for which there was no packet data available must not
raise a 'pkt_sent' interrupt.
176 logic has_data_q;
177
178 assign more_data_to_send = has_data_q &
~in_ep_data_done_i[in_ep_index];
179
180 assign tx_data_avail_o = logic'(in_xact_state == StSendData) &
more_data_to_send;
181
182 // Transaction is starting on this IN endpoint; capture the
packet details.
183
184
185 logic in_starting;
186 assign in_starting = (in_xact_state == StIdle || in_x-
act_state == StWaitAck) && in_token_received;
187
188 assign in_xact_starting_o = in_starting & ep_active;
189 assign in_xact_start_ep_o = in_ep_current_d;
190
191
192 // in transaction state machine
193
194
195 logic [AckTimeoutCntW-1:0] timeout_cntdown_d,
timeout_cntdown_q;
196 logic rollback_in_xact;
197
198 always_comb begin
199 in_xact_state_next = in_xact_state;
200 in_xact_end = 1'b0;
201 tx_pkt_start_o = 1'b0;
202 tx_pid_o = 4'b0000;
203 rollback_in_xact = 1'b0;
204 timeout_cntdown_d = AckTimeoutCnt[AckTimeoutCntW-1:0];
205 unique case (in_xact_state)
206 StIdle: begin
207 if (ep_active && in_token_received) begin
208 in_xact_state_next = StRcvdIn;
209 end else begin
210 // Ignore tokens to inactive endpoints. Send no re-
sponse.
211 in_xact_state_next = StIdle;
212 end
213 end
214
215 StRcvdIn: begin
216 tx_pkt_start_o = 1'b1; // Need to transmit NAK/STALL
or DATA

```

```

217
218 if (in_ep_iso_i[in_ep_index]) begin
219 // ISO endpoint
220 // We always need to transmit. When no data is
available, we send
221 // a zero-length packet.
222 // DATA0 always for full-speed isochronous endpoints
223 in_xact_state_next = StSendData;
224 tx_pid_o = {UsbPidData0};
225 end else if (in_ep_stall_i[in_ep_index]) begin
226 in_xact_state_next = StIdle;
227 tx_pid_o = {UsbPidStall}; // STALL
228 end else if (has_data_q) begin
229 in_xact_state_next = StSendData;
230 tx_pid_o = {data_toggle_q[in_ep_index], 1'b0, {Us-
bPidTypeData}}; // DATA0/1
231 end else begin
232 in_xact_state_next = StIdle;
233 tx_pid_o = {UsbPidNak}; // NAK
234 end
235 end
236
237 StSendData: begin
238 // Use &in_ep_get_addr so width can vary, looking for
all ones
239 if ((!more_data_to_send) || (&in_ep_get_addr_o) &&
tx_data_get_i) begin
240 if (in_ep_iso_i[in_ep_index]) begin
241 in_xact_state_next = StIdle; // no ACK for ISO EPs
242 in_xact_end = has_data_q;
243 end else begin
244 if (tx_pkt_end_i) begin
245 in_xact_state_next = StWaitAckStart;
246 end else begin
247 in_xact_state_next = StWaitTxEnd;
248 end
249 end
250 end else begin
251 in_xact_state_next = StSendData;
252 end
253 end
254
255 StWaitTxEnd: begin
256 if (tx_pkt_end_i) begin
257 in_xact_state_next = StWaitAckStart;
258 end
259 end
260
261 StWaitAckStart: begin
262 // The spec says we have up to 18 bit times to wait
for the host
263 // response. If it doesn't arrive in time, we must in-
validate the
264 // transaction.
265 timeout_cntdown_d = timeout_cntdown_q - 1'b1;
266
267 if (rx_pkt_start_i) begin
268 in_xact_state_next = StWaitAck;
269 end else if (timeout_cntdown_q == '0) begin
270 in_xact_state_next = StIdle;
271 rollback_in_xact = 1'b1;
272 end else begin
273 in_xact_state_next = StWaitAckStart;
274 end
275 end
276
277 StWaitAck: begin
278 if (ack_received) begin
279 in_xact_state_next = StIdle;
280 in_xact_end = 1'b1;
281 end else if (in_token_received) begin
282 // Handshake response is missing.
283 in_xact_state_next = ep_active ? StRcvdIn : StIdle;
284 rollback_in_xact = 1'b1;
285 end else if (rx_pkt_end_i) begin
286 // Includes NAK
287 in_xact_state_next = StIdle;
288 rollback_in_xact = 1'b1;
289 end else begin
290 in_xact_state_next = StWaitAck;
291 end
292 end
293
294 default: in_xact_state_next = StIdle;
295 endcase
296 end
297
298 `ASSERT(InXactStateValid_A,
299 in_xact_state inside {StIdle, StRcvdIn, StSendData,
StWaitTxEnd, StWaitAckStart, StWaitAck},
300 clk_48mhz_i)
301
302 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
303 if (!rst_ni) begin
304 timeout_cntdown_q <= AckTimeoutCnt[AckTimeoutCntW-1:0];
305 end else begin
306 timeout_cntdown_q <= timeout_cntdown_d;
307 end
308 end
309
310 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin

```

```

311 if (!rst_ni) begin
312   tx_data_o <= '0';
313 end else begin
314   tx_data_o <= in_ep_data_i;
315 end
316 end
317
318 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
319   if (!rst_ni) begin
320     in_xact_state <= StIdle;
321     in_ep_rollback_o <= 1'b0;
322   end else if (link_reset_i || !link_active_i) begin
323     in_xact_state <= StIdle;
324     in_ep_rollback_o <= 1'b0;
325   end else begin
326     in_xact_state <= in_xact_state_next;
327     in_ep_rollback_o <= rollback_in_xact;
328   end
329 end
330
331 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
332   if (!rst_ni) begin
333     in_ep_get_addr_o <= '0';
334   end else begin
335     if (in_xact_state == StIdle) begin
336       in_ep_get_addr_o <= '0';
337     end else if ((in_xact_state == StSendData) &&
tx_data_get_i) begin
338       in_ep_get_addr_o <= in_ep_get_addr_o + 1'b1;
339     end
340   end
341 end
342
343 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
344   if (!rst_ni) begin
345     in_ep_newpkt_o <= 1'b0;
346     in_ep_current_o <= '0';
347     has_data_q <= 1'b0;
348   end else begin
349     if (in_token_received) begin
350       in_ep_current_o <= in_ep_current_d;
351       in_ep_newpkt_o <= 1'b1;
352       has_data_q <= in_ep_has_data_i[in_ep_index_d];
353     end else begin
354       in_ep_newpkt_o <= 1'b0;
355     end
356   end
357 end
358
359 // Updating of data toggles
360 always_comb begin : proc_data_toggle_d
361   data_toggle_d = data_toggle_q;
362
363   if (setup_token_received && ep_active) begin
364     data_toggle_d[in_ep_index_d] = 1'b1;
365   end else if ((in_xact_state == StWaitAck) && ack_received)
begin
366     data_toggle_d[in_ep_index] =
~data_toggle_q[in_ep_index];
367   end
368   // Selective modification by software
369   if (in_datatog_we_i) begin
370     data_toggle_d = (data_toggle_d & ~in_datatog_mask_i) |
371       (in_datatog_status_i & in_data-
tog_mask_i);
372   end
373 end
374 // Supply current data toggles to register interface
375 assign in_data_toggle_o = data_toggle_q;
376
377 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
378   if (!rst_ni) begin
379     data_toggle_q <= '0'; // Clear for all endpoints
380   end else if (link_reset_i) begin
381     data_toggle_q <= '0'; // Clear for all endpoints
382   end else begin
383     data_toggle_q <= data_toggle_d;
384   end
385 end
386
387 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
388   if (!rst_ni) begin
389     in_ep_data_get_o <= 1'b0;
390   end else begin
391     if ((in_xact_state == StSendData) && tx_data_get_i) be-
gin
392       in_ep_data_get_o <= 1'b1;
393     end else begin
394       in_ep_data_get_o <= 1'b0;
395     end
396   end
397 end
398
399 ///////////////////////////////////////////////////
400 // Count non-Isynchronous IN transactions not receiving a
handshake response.
401 //
402 // - no handshake response within the timeout period
403 // - packet reception started within the timeout period but

```

```

it was not ACK/NAK.
404 ///////////////////////////////////////////////////
405 assign event_timeout_in_o = (in_xact_state == StWaitAckStart
||
406   (in_xact_state == StWaitAck &&
rx_pkt_end_i && !nak_received))
& rollback_in_xact;
407
408 ///////////////////////////////////////////////////
409 // Count IN transactions that are actively NAKed by the
host.
410 //
411 ///////////////////////////////////////////////////
412 assign event_nak_in_o = (in_xact_state == StWaitAck) &&
nak_received;
413
414 ///////////////////////////////////////////////////
415 // Count the number of IN requests for which data is not
available, including
416 // endpoints which are supported but not presently enabled/
configured.
417 // (These may be ignored using the event counter configura-
tion if required.)
418 ///////////////////////////////////////////////////
419 logic event_nodata_in_q;
420 // This event signal must be delayed by a single cycle so
that 'in_ep_current_o' becomes valid.
421 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
422   if (!rst_ni) event_nodata_in_q <= 1'b0;
423   else event_nodata_in_q <= in_starting & (ep_in_hw ? !
in_ep_has_data_i[in_ep_index_d] : 1'b0);
424 end
425 assign event_nodata_in_o = event_nodata_in_q;
426
427 ///////////////////////////////////////////////////
428 // Assertions //
429 ///////////////////////////////////////////////////
430
431 endmodule

```

3. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_out_pe.sv (443 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 // USB Full Speed Non-Buffered Protocol Engine for OUT/SETUP
endpoints
7 //
8 // Decode OUT/SETUP requests from host and accept data unless
buffers are full
9 // (SETUP is a special form of OUT and starts a transaction
sequence)
10 //
11 // Based on usb_fs_out_pe.v from the TinyFPGA-Bootloader pro-
ject but
12 // this version contains no packet buffers
13
14 `include "prim_assert.sv"
15
16 module usb_fs_nb_out_pe #(
17   parameter logic [4:0] NumOutEps = 2,
18   parameter int unsigned MaxOutPktSizeByte = 32,
19   // Targeting 17 bit times for the timeout. The counter runs
on the 48 MHz
20   // clock. The SOP delay to rx_pkt_start_i is 3 bit times,
but the two
21   // packets go in the same direction, so this is negated.
This timeout
22   // originates from Section 7.1.19.1 of the USB 2.0 specific-
ation.
23   parameter int unsigned AckTimeoutCnt = 17 * 4,
24   localparam int unsigned OutEpW = $clog2(NumOutEps), // de-
rived parameter
25   localparam int unsigned PktW = $clog2(MaxOutPktSizeByte), //
derived parameter
26   localparam int unsigned AckTimeoutCntW = $clog2(AckTimeoutCnt) // derived parameter
27 ) (
28   input logic clk_48mhz_i,
29   input logic rst_ni,
30   input logic link_reset_i,
31   input logic [6:0] dev_addr_i,
32
33   ///////////////////////////////////////////////////
34   // endpoint interface //
35   ///////////////////////////////////////////////////
36   output logic [3:0] out_ep_current_o, // Other
signals address to this ep,

```

```

37 // stable
for several cycles

38 output logic out_ep_data_put_o, // put the
data (put addr advances after)
39 output logic [PKTW - 1:0] out_ep_put_addr_o, // Offset
to put data (0..pktlen)
40 output logic [7:0] out_ep_data_o,
41 output logic out_ep_newpkt_o, // new pack-
et, current was set
42 output logic out_ep_acked_o, // good ter-
mination, device has acked
43 output logic out_ep_rollback_o, // bad
termination, discard data
44 output logic [NumOutEps-1:0] out_ep_setup_o,

45 input logic [NumOutEps-1:0] out_ep_enabled_i, // Endpoint is
enabled and produces handshakes
46 input logic [NumOutEps-1:0] out_ep_control_i, // Is a
control endpoint: Accepts SETUP packets
47 input logic [NumOutEps-1:0] out_ep_full_i, // Cannot ac-
cept data
48 input logic [NumOutEps-1:0] out_ep_stall_i, // Stalled
49 input logic [NumOutEps-1:0] out_ep_iso_i, // Configure
endpoint in isochronous mode
50

51 output logic [NumOutEps-1:0] out_data_toggle_o, // Current
state of OUT data toggles

52 input logic out_datatog_we_i, // OUT data
toggles write strobe from software
53 input logic [NumOutEps-1:0] out_datatog_status_i, // New
state of selected OUT data toggles
54 input logic [NumOutEps-1:0] out_datatog_mask_i, // Which
OUT EP data toggles to modify
55
56 // rx path //
57 // rx path //
58 // rx path //
59
60 // Strobed on reception of packet.
61 input logic rx_pkt_start_i,
62 input logic rx_pkt_end_i,
63 input logic rx_pkt_valid_i,
64
65 // Most recent packet received.
66 input logic [3:0] rx_pid_i,
67 input logic [6:0] rx_addr_i,
68 input logic [3:0] rx_endp_i,
69
70 // rx_data is pushed into endpoint controller.
71 input logic rx_data_put_i,
72 input logic [7:0] rx_data_i,
73
74
75 // tx path //
76 // tx path //
77 // tx path //
78
79 // Strobe to send new packet.
80 output logic tx_pkt_start_o,
81 input logic tx_pkt_end_i,
82 output logic [3:0] tx_pid_o,
83
84 // event counters
85 output logic event_datatog_out_o
86 );
87
88 // suppress warnings
89 logic unused_1;
90 assign unused_1 = tx_pkt_end_i;
91
92 // out transaction state machine //
93 // out transaction state machine //
94 // out transaction state machine //
95 import usb_consts_pkg::*;
96
97 typedef enum logic [2:0] {
98     StIdle,
99     StRcvdOut,
100     StRcvdDataStart,
101     StRcvdDataEnd,
102     StRcvdIsoDataEnd
103 } state_out_e;
104
105 state_out_e out_xact_state;
106 state_out_e out_xact_state_next;
107
108 logic out_xact_start;
109 logic new_pkt_end;
110 logic rollback_data;
111
112 // set when the endpoint buffer is unable to receive the out
transaction
113 logic nak_out_transaction;
114
115 // data toggle state
116 logic [NumOutEps - 1:0] data_toggle_q, data_toggle_d;
117

```

```

118 // Decode the rx token
119 logic token_received, out_token_received,
setup_token_received;
120 logic invalid_packet_received, data_packet_received,
non_data_packet_received;
121 logic bad_data_toggle;
122 logic ep_in_hw, ep_active, ep_is_control;
123 logic [3:0] out_ep_current_d;
124
125 // 1: If the current transaction is a SETUP, 0: OUT
126 logic current_xact_setup_q;
127
128 // More syntax so can compare with enum
129 usb_pid_type_e rx_pid_type;
130 usb_pid_e rx_pid;
131 assign rx_pid_type = usb_pid_type_e'(rx_pid_i[1:0]);
132 assign rx_pid = usb_pid_e'(rx_pid_i);
133
134 assign token_received =
135     rx_pkt_end_i &&
136     rx_pkt_valid_i &&
137     rx_pid_type == UsbPidTypeToken &&
138     rx_addr_i == dev_addr_i;
139
140 assign out_token_received =
141     token_received &&
142     rx_pid == UsbPidOut;
143
144 assign setup_token_received =
145     token_received &&
146     rx_pid == UsbPidSetup;
147
148 assign invalid_packet_received =
149     rx_pkt_end_i &&
150     !rx_pkt_valid_i;
151
152 assign data_packet_received =
153     rx_pkt_end_i &&
154     rx_pkt_valid_i &&
155     ((rx_pid == UsbPidData0) || (rx_pid == UsbPidData1));
156
157 assign non_data_packet_received =
158     rx_pkt_end_i &&
159     rx_pkt_valid_i &&
160     !((rx_pid == UsbPidData0) || (rx_pid == UsbPidData1));
161
162 // Is the specified endpoint actually implemented in hard-
ware?
163 assign ep_in_hw = {1'b0, rx_endp_i} < NumOutEps;
164 assign out_ep_current_d = ep_in_hw ? rx_endp_i : '0;
165
166 // Make widths work - out_ep_current_d/out_ep_current_o only
hold implemented endpoint IDs.
167 // These signals can be used to index signals of NumOutEps
width.
168 // They are only valid if ep_in_hw is set, i.e., if the spe-
cified endpoint is implemented.
169 logic [OutEpW-1:0] out_ep_index;
170 logic [OutEpW-1:0] out_ep_index_d;
171 assign out_ep_index = out_ep_current_o[0+: OutEpW];
172 assign out_ep_index_d = out_ep_current_d[0+: OutEpW];
173
174 // Is the endpoint active?
175 assign ep_active = out_ep_enabled_i[out_ep_index_d] &
ep_in_hw;
176 assign ep_is_control = out_ep_control_i[out_ep_index_d];
177
178 assign bad_data_toggle =
179     data_packet_received &&
180     ep_active &&
181     rx_pid_i[3] != data_toggle_q[out_ep_index_d];
182
183 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
184     if (!rst_ni) begin
185         out_ep_setup_o <= '0;
186     end else begin
187         if (setup_token_received && ep_active) begin
188             out_ep_setup_o[out_ep_index_d] <= 1'b1;
189         end else if (out_token_received && ep_active) begin
190             out_ep_setup_o[out_ep_index_d] <= 1'b0;
191         end
192     end
193 end
194
195 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
196     if (!rst_ni) begin
197         out_ep_data_o <= 0;
198     end else begin
199         if (rx_data_put_i) begin
200             out_ep_data_o <= rx_data_i;
201         end
202     end
203 end
204
205 // out transaction state machine //
206 // out transaction state machine //
207 // out transaction state machine //
208
209 logic [AckTimeoutCntW-1:0] timeout_cntdown_d,
timeout_cntdown_q;

```

```

210
211 always_comb begin
212     out_ep_acked_o = 1'b0;
213     out_xact_start = 1'b0;
214     out_xact_state_next = out_xact_state;
215     tx_pkt_start_o = 1'b0;
216     tx_pid_o = 4'b0000;
217     new_pkt_end = 1'b0;
218     rollback_data = 1'b0;
219     timeout_cntdown_d = AckTimeoutCnt[AckTimeoutCntW-1:0];
220
221     unique case (out_xact_state)
222         StIdle: begin
223             // For unimplemented EPs, transactions are ignored.
224
225             // SETUP transactions are also ignored for non-control
226             EPs.
227             if (ep_active && (out_token_received ||
228             (setup_token_received && ep_is_control))) begin
229                 out_xact_state_next = StRcvdOut;
230                 out_xact_start = 1'b1;
231             end else begin
232                 out_xact_state_next = StIdle;
233             end
234         end
235         StRcvdOut: begin
236             // The spec says we have up to 18 bit times to wait
237             for the host's
238             // data packet. If it doesn't arrive in time, we must
239             invalidate the
240             // transaction.
241             timeout_cntdown_d = timeout_cntdown_q - 1'b1;
242
243             if (rx_pkt_start_i) begin
244                 out_xact_state_next = StRcvdDataStart;
245             end else if (timeout_cntdown_q == '0) begin
246                 out_xact_state_next = StIdle;
247             end else begin
248                 out_xact_state_next = StRcvdOut;
249             end
250         end
251         StRcvdDataStart: begin
252             if (!ep_is_control && out_ep_iso_i[out_ep_index] &&
253             data_packet_received) begin
254                 // ISO EP: Don't send a handshake, ignore toggle.
255                 // But ignore iso bit for endpoints marked control.
256                 This is
257                 // a configuration error.
258                 out_xact_state_next = StRcvdIsoDataEnd;
259             end else if (bad_data_toggle && !
260             out_ep_stall_i[out_ep_index]) begin
261                 // The DATA toggle was wrong (skipped when this EP
262                 is stalled)
263                 // Note: bad_data_toggle is meaningless for unimple-
264                 mented EPs.
265                 out_xact_state_next = StIdle;
266                 rollback_data = 1'b1;
267                 tx_pkt_start_o = 1'b1;
268                 tx_pid_o = {UsbPidAck}; // ACK by spec because this
269                 is most likely previous ACK was lost
270             end else if (invalid_packet_received ||
271             non_data_packet_received) begin
272                 // in these cases eg bad CRC, send no response (not
273                 a NAK)
274                 out_xact_state_next = StIdle;
275                 rollback_data = 1'b1;
276             end else if (data_packet_received) begin
277                 out_xact_state_next = StRcvdDataEnd;
278             end else begin
279                 out_xact_state_next = StRcvdDataStart;
280             end
281         end
282         StRcvdDataEnd: begin
283             out_xact_state_next = StIdle;
284             tx_pkt_start_o = 1'b1;
285
286             if (current_xact_setup_q) begin
287                 // SETUP transactions end here
288                 if (nak_out_transaction |
289                 out_ep_full_i[out_ep_index]) begin
290                     // SETUP transactions that fail to be received are
291                     dropped without a response.
292                     tx_pkt_start_o = 1'b0;
293                     rollback_data = 1'b1;
294                 end else begin
295                     tx_pid_o = {UsbPidAck};
296                     new_pkt_end = 1'b1;
297                     out_ep_acked_o = 1'b1;
298                 end
299             end else begin
300                 // Non-isochronous OUT transactions end here
301                 if (out_ep_stall_i[out_ep_index]) begin
302                     tx_pid_o = {UsbPidStall}; // STALL
303                     rollback_data = 1'b1; // Packet not accepted
304                 end else if (nak_out_transaction |
305                 out_ep_full_i[out_ep_index]) begin
306                     tx_pid_o = {UsbPidNak}; // NAK -- the endpoint

```

```

could not accept the data at the moment
294     rollback_data = 1'b1;
295     end else begin
296         tx_pid_o = {UsbPidAck}; // ACK
297         new_pkt_end = 1'b1;
298         out_ep_acked_o = 1'b1;
299     end
300 end
301 end
302
303 StRcvdIsoDataEnd: begin
304     // Isochronous OUT transactions end here
305     out_xact_state_next = StIdle;
306
307     if (nak_out_transaction | out_ep_full_i[out_ep_index])
308     begin
309         // We got a valid packet, but can't store it (error
310         that the software must resolve)
311         rollback_data = 1'b1;
312     end else begin
313         // We got a valid packet, but we don't send an ACK
314         on the bus
315         new_pkt_end = 1'b1;
316         out_ep_acked_o = 1'b1;
317     end
318     end
319     default: out_xact_state_next = StIdle;
320 endcase
321 end
322 `ASSERT(OutXactStateValid_A,
323 out_xact_state inside {StIdle, StRcvdOut, StRcvdDataStart,
324 StRcvdDataEnd, StRcvdIsoDataEnd},
325 clk_48mhz_i)
326
327 // could flop this if needed
328 assign out_ep_rollback_o = rollback_data;
329
330 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
331     if (!rst_ni) begin
332         timeout_cntdown_q <= AckTimeoutCnt[AckTimeoutCntW-1:0];
333     end else begin
334         timeout_cntdown_q <= timeout_cntdown_d;
335     end
336 end
337
338 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
339     if (!rst_ni) begin
340         out_xact_state <= StIdle;
341     end else begin
342         out_xact_state <= link_reset_i ? StIdle : out_x-
343         act_state_next;
344     end
345 end
346
347 // Updating of data toggles
348 always_comb begin : proc_data_toggle_d
349     data_toggle_d = data_toggle_q;
350
351     if (setup_token_received && ep_active) begin
352         data_toggle_d[out_ep_index_d] = 1'b0;
353     end else if (new_pkt_end) begin
354         data_toggle_d[out_ep_index] =
355         ~data_toggle_q[out_ep_index];
356     end
357 end
358
359 // Selective modification by software
360 if (out_datatog_we_i) begin
361     data_toggle_d = (data_toggle_d & ~out_datatog_mask_i) |
362     (out_datatog_status_i & out_data-
363     tog_mask_i);
364 end
365
366 // Supply current data toggles to register interface
367 assign out_data_toggle_o = data_toggle_q;
368
369 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
370     if (!rst_ni) begin
371         data_toggle_q <= '0; // Clear for all endpoints
372     end else if (link_reset_i) begin
373         data_toggle_q <= '0; // Clear for all endpoints
374     end else begin
375         data_toggle_q <= data_toggle_d;
376     end
377 end
378
379 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
380     if (!rst_ni) begin
381         out_ep_newpkt_o <= 1'b0;
382         out_ep_current_o <= '0;
383         current_xact_setup_q <= 1'b0;
384     end else begin
385         if (out_xact_start) begin
386             out_ep_newpkt_o <= 1'b1;
387             out_ep_current_o <= out_ep_current_d;
388             current_xact_setup_q <= setup_token_received;
389         end else begin
390             out_ep_newpkt_o <= 1'b0;
391         end
392     end

```



```

385     end
386   end
387
388   // put data strobe follows the rx strobe (which will latch
the data)
389   always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
390     if (!rst_ni) begin
391       out_ep_data_put_o <= 1'b0;
392     end else begin
393       out_ep_data_put_o <= ((out_xact_state == StRcvdData-
Start) && rx_data_put_i);
394     end
395   end
396
397   // nak an OUT if any data comes in with the endpoint full
398
399   // Note that if there is a full size packet buffer this will
only be all or nothing
400   // but in the case there was a FIFO with less than a max
packet size free you
401   // could get lucky and the packet received be small and fit
with no need to NAK
402   always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
403     if (!rst_ni) begin
404       nak_out_transaction <= 1'b0;
405     end else begin
406       if ((out_xact_state == StIdle) || (out_xact_state ==
StRcvdOut)) begin
407         nak_out_transaction <= 1'b0;
408       end else if (out_ep_data_put_o &&
out_ep_full_i[out_ep_index]) begin
409         nak_out_transaction <= 1'b1;
410       end
411     end
412
413     // address increment whenever there is a data put unless
414     // -- already going to NAK transaction and replay things
415     // -- the address is at max packet size
416     // NOTE if more than max packet size received then data is
lost
417     logic increment_addr;
418     assign increment_addr = !nak_out_transaction &&
(~&out_ep_put_addr_o) && out_ep_data_put_o;
419
420     always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
421       if (!rst_ni) begin
422         out_ep_put_addr_o <= '0;
423       end else begin
424         if (out_xact_state == StRcvdOut) begin
425           out_ep_put_addr_o <= '0;
426         end else if ((out_xact_state == StRcvdDataStart) && in-
crement_addr) begin
427           out_ep_put_addr_o <= out_ep_put_addr_o + 1;
428         end
429       end
430     end
431
432     ///////////////////////////////////////////////////////////////////
433     // Count OUT transactions that have been ACKed and dropped
because of unexpected Data Toggles.
434     ///////////////////////////////////////////////////////////////////
435     assign event_datatog_out_o = (out_xact_state == StRcvdData-
Start) &&
436                                     (ep_is_control || !
out_ep_iso_i[out_ep_index]) &&
437                                     !out_ep_stall_i[out_ep_index]
&& bad_data_toggle;
438
439     ///////////////////////////////////////////////////////////////////
440     // Assertions //
441     ///////////////////////////////////////////////////////////////////
442
443   endmodule

```

4. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_nb_pe.sv (353 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 // USB Full Speed Non-Buffered Protocol Engine
7 //
8 // Decodes low level USB link protocol from external
usb_fs_tx/rx
9 // Provides upper level with
10 // - OUT endpoint interface (host to device) which also covers
SETUP
11 // - IN endpoint interface (device to host)
12 // - Start of Frame interface
13 //
14 // Based on usb_fs_pe.v from the TinyFPGA-Bootloader project

```

```

but
15 // this version contains no packet buffers
16
17 `include "prim_assert.sv"
18
19 module usb_fs_nb_pe #(
20   // Currently only accepts NumOutEps == NumInEps
21   parameter int unsigned NumOutEps = 2,
22   parameter int unsigned NumInEps = 2,
23   parameter int unsigned MaxPktSizeByte = 32,
24   localparam int unsigned PktW = $clog2(MaxPktSizeByte) // de-
rived parameter
25 ) (
26   input logic clk_48mhz_i,
27   input logic rst_ni, // Async. re-
set, active low
28   input logic link_reset_i, // USB reset,
sync to 48 MHz, active high
29   input logic link_active_i, // Device is
in Default/Addressed/Configured state
30                                     // and may re-
spond to transactions
31   input logic [6:0] dev_addr_i,
32
33   input logic cfg_eop_single_bit_i, // 1:
detect a single SE0 bit as EOP
34   input logic cfg_use_diff_rcvr_i, // 1:
use usb_d_i from a differential receiver
35   input logic cfg_pinflip_i, // 1: USB-side D+
and D- pins are flipped.
36                                     // Change val-
ues in logic to accommodate.
37   input logic tx_osc_test_mode_i, // Oscil-
lator test mode (constantly output JK)
38   output logic [NumOutEps-1:0] out_data_toggle_o, // Current
state of OUT data toggles
39   input logic out_datatog_we_i, // OUT data
toggles write strobe from software
40   input logic [NumOutEps-1:0] out_datatog_status_i, // New
state of selected OUT data toggles
41   input logic [NumOutEps-1:0] out_datatog_mask_i, // Which
OUT EP data toggles to modify
42   output logic [NumInEps-1:0] in_data_toggle_o, // Current
state of IN data toggles
43   input logic in_datatog_we_i, // IN data
toggles write strobe from software
44   input logic [NumInEps-1:0] in_datatog_status_i, // New
state of selected IN data toggles
45   input logic [NumInEps-1:0] in_datatog_mask_i, // Which
IN EP data toggles to modify
46   input logic diff_rx_ok_i, // 1: received
differential data symbols are valid.
47                                     // Set low if K
and J symbols might be invalid, such
48                                     // as when an
external differential receiver is
49                                     // powering on.
50
51   ///////////////////////////////////////////////////////////////////
52   // USB Endpoint Interface //
53   ///////////////////////////////////////////////////////////////////
54   ///////////////////////////////////////////////////////////////////
55   // global endpoint interface //
56   ///////////////////////////////////////////////////////////////////
57
58   // out endpoint interfaces
59   output logic [3:0] out_ep_current_o, // Other
signals address to this ep
60
61   output logic out_ep_data_put_o, // put the
data (put addr advances after)
62   output logic [PktW - 1:0] out_ep_put_addr_o, // Offset
to put data (0..pktlen)
63   output logic [7:0] out_ep_data_o,
64   output logic out_ep_newpkt_o, // New OUT
pkt start (with in_ep_current_o update)
65   output logic out_ep_acked_o, // good ter-
mination, device has acked
66   output logic out_ep_rollback_o, // bad
termination, discard data
67   output logic [NumOutEps-1:0] out_ep_setup_o,
68
69   input logic [NumOutEps-1:0] out_ep_enabled_i, // Endpoint is
enabled and produces handshakes
70   input logic [NumOutEps-1:0] out_ep_control_i, // Is a
control endpoint: Accepts SETUP packets
71   input logic [NumOutEps-1:0] out_ep_full_i, // Cannot ac-
cept data
72   input logic [NumOutEps-1:0] out_ep_stall_i, // Stalled
73   input logic [NumOutEps-1:0] out_ep_iso_i, // Configure
endpoint in isochronous mode
74
75   // in endpoint interfaces
76   output logic in_xact_starting_o, // Start
of an IN transaction
77   output logic [3:0] in_xact_start_ep_o, // Cap-
ture IN packet details

```

```

76
77 output logic [3:0] in_ep_current_o, // Other
signals addressed to this ep
78 output logic in_ep_rollback_o, // Bad ter-
mination, rollback transaction
79 output logic in_ep_xact_end_o, // good
termination, transaction complete
80 output logic [PktW - 1:0] in_ep_get_addr_o, // Offset
requested (0..pktlen)
81 output logic in_ep_data_get_o, // Accept
data (get_addr advances too)

82 output logic in_ep_newpkt_o, // New IN pkt
start (with in_ep_current_o update)
83 input logic [NumInEps-1:0] in_ep_enabled_i, // Endpoint
is enabled and produces handshakes

84 input logic [NumInEps-1:0] in_ep_stall_i, // Endpoint in a
stall state

85 input logic [NumInEps-1:0] in_ep_has_data_i, // Endpoint
has data to supply
86 input logic [7:0] in_ep_data_i, // Data for
current get_addr
87 input logic [NumInEps-1:0] in_ep_data_done_i, // Set
when out of data
88 input logic [NumInEps-1:0] in_ep_iso_i, // Configure en-
dpoint in isochronous mode
89
90 // sof interface
91 output logic sof_detected_o, // SOF detec-
ted, PID valid
92 output logic sof_valid_o, // SOF packet
valid; frame number may be trusted
93 output logic [10:0] frame_index_o,
94
95 // RX line status
96 output logic rx_idle_det_o,
97 output logic rx_j_det_o,
98
99 // RX errors
100 output logic rx_crc5_err_o,
101 output logic rx_crc16_err_o,
102 output logic rx_pid_err_o,
103 output logic rx_bitstuff_err_o,
104
105 ///////////////////////////////////////////////////
106 // USB RX Interface (synchronous) //
107 ///////////////////////////////////////////////////
108 input logic usb_d_i,
109 input logic usb_dp_i,
110 input logic usb_dn_i,
111
112 ///////////////////////////////////////////////////
113 // USB TX Interface (synchronous) //
114 ///////////////////////////////////////////////////
115 output logic usb_d_o,
116 output logic usb_se0_o,
117 output logic usb_dp_o,
118 output logic usb_dn_o,
119 output logic usb_oe_o,
120
121 // event counters
122 output logic event_datatog_out_o,
123 output logic event_timeout_in_o,
124 output logic event_nak_in_o,
125 output logic event_nodata_in_o
126 );
127
128 import usb_consts_pkg::*;
129

130 // The code below assumes the number of OUT endpoints and IN
endpoints are
131 // interchangeable. Require them to be equal.
132 `ASSERT_INIT(NumOutEpsEqualsNumInEps_A, NumOutEps == Nu-
mInEps)
133
134 // rx interface
135 logic bit_strobe;
136 logic rx_pkt_start;
137 logic rx_pkt_end;
138 logic [3:0] rx_pid;
139 logic [6:0] rx_addr;
140 logic [3:0] rx_endp;
141 logic [10:0] rx_frame_num;
142 logic rx_data_put;
143 logic [7:0] rx_data;
144 logic rx_pid_valid;
145 logic rx_pkt_valid;
146
147 // tx mux
148 logic in_tx_pkt_start;
149 logic [3:0] in_tx_pid;
150 logic out_tx_pkt_start;
151 logic [3:0] out_tx_pid;
152
153 // tx interface
154 logic tx_pkt_start;
155 logic tx_pkt_end;
156 logic [3:0] tx_pid;

```

```

157 logic tx_data_avail;
158 logic tx_data_get;
159 logic [7:0] tx_data;
160
161 logic usb_oe;
162
163 // Start Of Frame handling
164 //
165 // - SOF is detected and may be used for timing/synchroniza-
tion purposes if the PID is valid.
166 assign sof_detected_o = rx_pkt_end & rx_pid_valid &
(usb_pid_e'(rx_pid) == UsbPidSof);
167 // - The SOF packet is considered valid iff the packet is
also free of bit stuffing violations
168 // and the CRC5 is correct.
169 assign sof_valid_o = rx_pkt_end & rx_pkt_valid &
(usb_pid_e'(rx_pid) == UsbPidSof);
170 // - The frame number shall be used only if the SOF packet
is valid.
171 assign frame_index_o = rx_frame_num;
172 assign usb_oe_o = usb_oe;
173
174 // IN ep type configuration
175 logic [NumInEps-1:0] in_ep_iso_not_control;
176 assign in_ep_iso_not_control = in_ep_iso_i & ~out_ep_con-
trol_i;
177
178 usb_fs_nb_in_pe #(
179     .NumInEps (NumInEps),
180     .MaxInPktSizeByte (MaxPktSizeByte)
181 ) u_usb_fs_nb_in_pe (
182     .clk_48mhz_i (clk_48mhz_i),
183     .rst_ni (rst_ni),
184     .link_reset_i (link_reset_i),
185     .link_active_i (link_active_i),
186     .dev_addr_i (dev_addr_i),
187
188     // transaction starting
189     .in_xact_starting_o (in_xact_starting_o),
190     .in_xact_start_ep_o (in_xact_start_ep_o),
191
192     // endpoint interface
193     .in_ep_current_o (in_ep_current_o),
194     .in_ep_rollback_o (in_ep_rollback_o),
195     .in_ep_xact_end_o (in_ep_xact_end_o),
196     .in_ep_get_addr_o (in_ep_get_addr_o),
197     .in_ep_data_get_o (in_ep_data_get_o),
198     .in_ep_newpkt_o (in_ep_newpkt_o),
199     .in_ep_enabled_i (in_ep_enabled_i),
200     .in_ep_stall_i (in_ep_stall_i),
201     .in_ep_has_data_i (in_ep_has_data_i),
202     .in_ep_data_i (in_ep_data_i),
203     .in_ep_data_done_i (in_ep_data_done_i),
204     .in_ep_iso_i (in_ep_iso_not_control),
205
206     .in_data_toggle_o (in_data_toggle_o),
207     .in_datatog_we_i (in_datatog_we_i),
208     .in_datatog_status_i (in_datatog_status_i),
209     .in_datatog_mask_i (in_datatog_mask_i),
210
211     // rx path
212     .rx_pkt_start_i (rx_pkt_start),
213     .rx_pkt_end_i (rx_pkt_end),
214     .rx_pkt_valid_i (rx_pkt_valid),
215     .rx_pid_i (rx_pid),
216     .rx_addr_i (rx_addr),
217     .rx_endp_i (rx_endp),
218
219     // tx path
220     .tx_pkt_start_o (in_tx_pkt_start),
221     .tx_pkt_end_i (tx_pkt_end),
222     .tx_pid_o (in_tx_pid),
223     .tx_data_avail_o (tx_data_avail),
224     .tx_data_get_i (tx_data_get),
225     .tx_data_o (tx_data),
226
227     // event counters
228     .event_timeout_in_o (event_timeout_in_o),
229     .event_nak_in_o (event_nak_in_o),
230     .event_nodata_in_o (event_nodata_in_o)
231 );
232
233 usb_fs_nb_out_pe #(
234     .NumOutEps (NumOutEps),
235     .MaxOutPktSizeByte (MaxPktSizeByte)
236 ) u_usb_fs_nb_out_pe (
237     .clk_48mhz_i (clk_48mhz_i),
238     .rst_ni (rst_ni),
239     .link_reset_i (link_reset_i),
240     .dev_addr_i (dev_addr_i),
241
242     // endpoint interface
243     .out_ep_current_o (out_ep_current_o),
244     .out_ep_data_put_o (out_ep_data_put_o),
245     .out_ep_put_addr_o (out_ep_put_addr_o),
246     .out_ep_data_o (out_ep_data_o),
247     .out_ep_newpkt_o (out_ep_newpkt_o),
248     .out_ep_acked_o (out_ep_acked_o),
249     .out_ep_rollback_o (out_ep_rollback_o),
250     .out_ep_setup_o (out_ep_setup_o),
251     .out_ep_enabled_i (out_ep_enabled_i),

```



```

252 .out_ep_control_i      (out_ep_control_i),
253 .out_ep_full_i        (out_ep_full_i),
254 .out_ep_stall_i       (out_ep_stall_i),
255 .out_ep_iso_i         (out_ep_iso_i),
256
257 .out_data_toggle_o    (out_data_toggle_o),
258 .out_datatog_we_i     (out_datatog_we_i),
259 .out_datatog_status_i (out_datatog_status_i),
260 .out_datatog_mask_i   (out_datatog_mask_i),
261
262 // rx path
263 .rx_pkt_start_i       (rx_pkt_start),
264 .rx_pkt_end_i         (rx_pkt_end),
265 .rx_pkt_valid_i       (rx_pkt_valid),
266 .rx_pid_i             (rx_pid),
267 .rx_addr_i           (rx_addr),
268 .rx_endp_i           (rx_endp),
269 .rx_data_put_i        (rx_data_put),
270 .rx_data_i            (rx_data),
271
272 // tx path
273 .tx_pkt_start_o       (out_tx_pkt_start),
274 .tx_pkt_end_i         (tx_pkt_end),
275 .tx_pid_o             (out_tx_pid),
276
277 // event counters
278 .event_datatog_out_o  (event_datatog_out_o)
279 );
280
281 usb_fs_rx u_usb_fs_rx (
282     .clk_i            (clk_48mhz_i),
283     .rst_ni           (rst_ni),
284     .link_reset_i     (link_reset_i),
285     .cfg_eop_single_bit_i (cfg_eop_single_bit_i),
286     .cfg_use_diff_rcvr_i (cfg_use_diff_rcvr_i),
287     .cfg_pinflip_i     (cfg_pinflip_i),
288     .diff_rx_ok_i     (diff_rx_ok_i),
289     .usb_d_i          (usb_d_i),
290     .usb_dp_i         (usb_dp_i),
291     .usb_dn_i         (usb_dn_i),
292     .tx_en_i          (usb_oe),
293     .bit_strobe_o     (bit_strobe),
294     .pkt_start_o      (rx_pkt_start),
295     .pkt_end_o        (rx_pkt_end),
296     .pid_o            (rx_pid),
297     .addr_o           (rx_addr),
298     .endp_o           (rx_endp),
299     .frame_num_o      (rx_frame_num),
300     .rx_data_put_o    (rx_data_put),
301     .rx_data_o        (rx_data),
302     .valid_pid_o      (rx_pid_valid),
303     .valid_packet_o   (rx_pkt_valid),
304     .rx_idle_det_o    (rx_idle_det_o),
305     .rx_j_det_o       (rx_j_det_o),
306     .crc5_error_o     (rx_crc5_err_o),
307     .crc16_error_o    (rx_crc16_err_o),
308     .pid_error_o      (rx_pid_err_o),
309     .bitstuff_error_o (rx_bitstuff_err_o)
310 );
311
312 usb_fs_tx_mux u_usb_fs_tx_mux (
313     // interface to IN Protocol Engine
314     .in_tx_pkt_start_i (in_tx_pkt_start),
315     .in_tx_pid_i       (in_tx_pid),
316
317     // interface to OUT Protocol Engine
318     .out_tx_pkt_start_i (out_tx_pkt_start),
319     .out_tx_pid_i       (out_tx_pid),
320
321     // interface to tx module
322     .tx_pkt_start_o     (tx_pkt_start),
323     .tx_pid_o           (tx_pid)
324 );
325
326 usb_fs_tx u_usb_fs_tx (
327     .clk_i            (clk_48mhz_i),
328     .rst_ni           (rst_ni),
329     .link_reset_i     (link_reset_i),
330     .cfg_pinflip_i     (cfg_pinflip_i),
331     .tx_osc_test_mode_i (tx_osc_test_mode_i),
332     .bit_strobe_i     (bit_strobe),
333     .usb_d_o          (usb_d_o),
334     .usb_se0_o        (usb_se0_o),
335     .usb_dp_o         (usb_dp_o),
336     .usb_dn_o         (usb_dn_o),
337     .usb_oe_o         (usb_oe),
338     .pkt_start_i      (tx_pkt_start),
339     .pkt_end_o        (tx_pkt_end),
340     .pid_i            (tx_pid),
341     .tx_data_avail_i  (tx_data_avail),
342     .tx_data_get_o    (tx_data_get),
343     .tx_data_i        (tx_data)
344 );
345
346 ///////////////////////////////////////////////////////////////////
347 // Assertions //
348 ///////////////////////////////////////////////////////////////////
349 `ASSERT_INIT(ParamNumEpsOutAndInEqual, NumOutEps == NumInEps)
350 `ASSERT_INIT(ParamNumOutEpsValid, (NumOutEps > 0) && (NumOutEps <= 16))

```

```

351 `ASSERT_INIT(ParamNumInEpsValid, (NumInEps > 0) && (NumInEps <= 16))
352
353 `ASSERT_INIT(ParamMaxPktSizeByteValid, (MaxPktSizeByte >= 8)
354 && (MaxPktSizeByte <= 64))
355 endmodule

```

5. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_rx.sv (673 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Copyright Luke Valenty (TinyFPGA project, https://git-
6 // hub.com/tinyfpga/TinyFPGA-Bootloader).
7 module usb_fs_rx (
8     // A 48MHz clock is required to recover the clock from the
9     // incoming data.
10     input logic clk_i,
11     input logic rst_ni,
12     input logic link_reset_i,
13
14     // configuration
15     input logic cfg_eop_single_bit_i,
16     input logic cfg_use_diff_rcvr_i,
17     input logic cfg_pinflip_i,
18     input logic diff_rx_ok_i,
19
20     // USB data+ and data- lines (synchronous)
21     input logic usb_d_i,
22     input logic usb_dp_i,
23     input logic usb_dn_i,
24
25     // Transmit enable disables the receiver
26     input logic tx_en_i,
27
28     // pulse on every bit transition.
29     output logic bit_strobe_o,
30
31     // Pulse on detecting a start of packet symbol (i.e. the be-
32     // ginning of sync)
33     output logic pkt_start_o,
34
35     // Pulse on end of current packet.
36     output logic pkt_end_o,
37
38     // Most recent packet decoded.
39     output logic [3:0] pid_o,
40     output logic [6:0] addr_o,
41     output logic [3:0] endp_o,
42     output logic [10:0] frame_num_o,
43
44     // Pulse on valid data on rx_data. Will not go high for
45     // token or handshake
46     // packets.
47     output logic rx_data_put_o,
48     output logic [7:0] rx_data_o,
49
50     // Most recent packet passes PID check
51     output logic valid_pid_o,
52     // Most recent packet passes PID, length and CRC checks
53     output logic valid_packet_o,
54
55     // line status for the status detection (actual rx bits
56     // after clock recovery)
57     output logic rx_idle_det_o,
58     output logic rx_j_det_o,
59
60     // Error detection
61     output logic crc5_error_o,
62     output logic crc16_error_o,
63     output logic pid_error_o,
64     output logic bitstuff_error_o
65 );
66
67 import usb_consts_pkg::*;
68
69 logic [6:0] bitstuff_history_q, bitstuff_history_d;
70 logic bitstuff_error;
71 logic bitstuff_error_q, bitstuff_error_d;
72
73 ///////////////////////////////////////////////////////////////////
74 // usb receive path //
75 ///////////////////////////////////////////////////////////////////
76
77 // Adjust inputs when D+/D- are flipped on the USB side
78 logic usb_dp_flipped, usb_dn_flipped, usb_d_flipped;
79 assign usb_dp_flipped = cfg_pinflip_i ? usb_dn_i : usb_dp_i;
80 assign usb_dn_flipped = cfg_pinflip_i ? usb_dp_i : usb_dn_i;
81 assign usb_d_flipped = usb_d_i ^ cfg_pinflip_i;
82
83 ///////////////////////////////////////////////////////////////////
84 // line state recovery state machine //
85 ///////////////////////////////////////////////////////////////////
86
87 // If the receive path is set NOT to use an external differ-
88 // ential receiver:

```

```

82 // This block samples data purely from the usb_dp_i/usb_dn_i
pair.
83 // There is a chance that one of the signals in the pair
will appear to have
84 // changed to the new state while the other is still in the
old state. The
85 // following state machine detects transitions and waits an
extra sampling clock
86 // before decoding the state on the dp/dn pair. This trans-
ition period will

87 // only ever last for one clock as long as there is no noise on
the line.

88 // If there is enough noise on the line then the data may be
corrupted and the
89 // packet will fail the data integrity checks.
90
91 // If the receive path uses an external differential receiv-
er:
92 // This block uses the usb_d_i input to detect K and J sym-
bols.
93 // The individual signals of the differential pair must
still be connected
94 // to this block to detect SE0.
95 // Note that the spec warns in section 7.1.4.1:
96 // Both D+ and D- may temporarily be less than VIH (min)
during differential

97 // signal transitions. This period can be up to 14 ns (TFST)
for full-speed
98 // transitions and up to 210 ns (TLST) for low-speed trans-
itions. Logic in the
99 // receiver must ensure that this is not interpreted as an
SE0.
100 // Since the 48MHz sample clock is 20.833ns period we will
either miss this or
101 // sample it only once, so it will be covered by
line_state=DT and the next
102 // sample will not be SE0 unless this was a real SE0 trans-
ition
103 // Note: if it is a real SE0 the usb_d_i input could be do-
ing anything.
104
105 logic [2:0] line_state_qq, line_state_q, line_state_d;
106 logic [2:0] diff_state_q, diff_state_d;
107 logic [2:0] line_state_rx;
108 logic
109     use_se;
110
111 localparam logic [2:0] DT = 3'b100; // transition state
112 localparam logic [2:0] DJ = 3'b010; // J - idle line state
113 localparam logic [2:0] DK = 3'b001; // K - inverse of J

114 localparam logic [2:0] SE0 = 3'b000; // single-ended 0 - end
of packet or detached
115 // localparam logic [2:0] SE1 = 3'b011; // single-ended 1 -
illegal
116
117 // Mute the input if we're transmitting
118 // dpair is the usb_dp_i/usb_dn_i pair, used in both modes.
With
119 // an external differential receiver, it is only used for
detecting SE0 and
120 // transitions. Without an external differential receiver
driving the
121 // usb_d_i input, it is used for all symbols.
122 // ddiff is the decoded data input from an external differ-
ential receiver,
123 // if available, and it is only for K and J symbols, plus
transition detection.
124 logic [1:0] dpair, ddiff;
125 always_comb begin : proc_dpair_mute
126     if (tx_en_i) begin
127         dpair = DJ[1:0]; // J
128         ddiff = DJ[1:0]; // J
129     end else begin
130         dpair = {usb_dp_flipped, usb_dn_flipped};
131         ddiff = usb_d_flipped ? DJ[1:0] : DK[1:0]; // equiv to
{usb_d_i, ~usb_d_i}
132     end
133 end
134
135 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_line_state_q
136     if (!rst_ni) begin
137         line_state_q <= SE0;
138         line_state_qq <= SE0;
139         diff_state_q <= SE0;
140     end else begin
141         if (link_reset_i) begin
142             line_state_q <= SE0;
143             line_state_qq <= SE0;
144             diff_state_q <= SE0;
145         end else begin
146             line_state_q <= line_state_d;
147             line_state_qq <= line_state_q;
148             diff_state_q <= diff_state_d;
149         end
150     end
151 end

152 always_comb begin : proc_line_state_d
153     // Default assignment
154     line_state_d = line_state_q;
155
156     if (line_state_q == DT) begin
157         // if we are in a transition state, then we can sample
the pair and
158         // move to the next corresponding line state
159         line_state_d = {1'b0, dpair};
160     end else begin
161         // if we are in a valid line state and the value of the
pair changes,
162         // then we need to move to the transition state
163         if (dpair != line_state_q[1:0]) begin
164             line_state_d = DT;
165         end
166     end
167 end

168 always_comb begin : proc_diff_state_d
169     // Default assignment
170     diff_state_d = diff_state_q;
171
172     if (diff_state_q == DT) begin
173         // if we are in a transition state, then we can sample
the diff input and
174         // move to the next corresponding line state
175         diff_state_d = {1'b0, ddiff};
176     end else begin
177         // if we are in a valid line state and the value of the
diff input changes,
178         // then we need to move to the transition state
179         if (ddiff != diff_state_q[1:0]) begin
180             diff_state_d = DT;
181         end
182     end
183 end

184 // The received line state depends on how the receiver is
configured:
185 // NOT using a differential receiver: it is just the
line_state_q that was captured
186 // Using a differential receiver: recovered from the differ-
ential receiver (diff_state_q)
187 // unless the diff pair in-
dicate SE0 when the differential
188 // receiver could produce any
value
189 //
190 // Transition where the dp/dn pair happen to see SE0 will
look like (driven by diff DT)
191 // line_state D? DT D?...
192 // diff_state Dx DT Dy (expect Dy to be inverse
of Dx since diff changed)
193 //
194 // Transition to SE0 when usb_d_i changes will look like:
195 // line_state DT D? D? D? DT SE0 SE0... (DT is the first
sample at SE0)
196 // diff_state DT Dx Dx Dx DT ?? ??... (diff saw trans-
ition as line went SE0)
197
198 // --> out DT Dx Dx Dx DT SE0 SE0 (if no transition
then DT would be Dx and n=3)
199
200 // bit_phase n 0 1 2 3 0 1 (n=3 unless there
was a clock resync)
201 //
202 // Transition to SE0 when usb_d_i does not change will look
like:
203 // line_state DT D? D? D? DT SE0 SE0... (DT is the first
sample at SE0)
204 // diff_state DT Dx Dx Dx Dx ?? ??... (diff no trans-
ition as line went SE0)
205
206 // --> out DT Dx Dx Dx Dx SE0 SE0 (if no transition
then DT would be Dx and n=3)
207
208 // bit_phase n 0 1 2 3 0 1 (n=3 unless there
was a clock resync)
209 //
210 // Transition to SE0 when usb_d_i does not change and clock
resync earlier:
211 //
212 // line_state DT D? D? DT SE0 SE0 SE0... (DT is the first
sample at SE0, should resync clock)
213 // diff_state DT Dx Dx Dx ?? ??... (diff no trans-
ition as line went SE0)
214 // --> out DT Dx Dx Dx SE0 SE0 SE0 (if no trans-
ition then DT would be Dx and n=3)
215 // bit_phase n 0 1 2 3 0 1 (n=3 unless
there was a clock resync)
216 //
217 // On transition back from SE0 want to generate a DT to re-
sync the clock
218 // since SE0 could have gone on a while no idea what
bit_phase is
219 // line_state SE0 SE0 DT D? D? D?

```

```

220 // diff_state ?? ?? ?? Dx Dx Dx
221 // --> out SE0 SE0 DT Dx Dx Dx
222 // bit_phase ? ? ? 0 1 2
223
224 assign use_se = (line_state_q == SE0) || ((line_state_q ==
DT) && (line_state_q == SE0));
225 assign line_state_rx = cfg_use_diff_rcvr_i ? (use_se ?
line_state_q : diff_state_q) :
line_state_q;
226
227 // clock recovery
228 // clock recovery
229 // clock recovery
230 // clock recovery
231
232 // the DT state from the line state recovery state machine
is used to align to
233 // transmit clock. the line state is sampled in the middle
of the bit time.
234
235 // example of signal relationships
236 // -----
237 // line_state DT DJ DJ DJ DT DK DK DK DK DK
DK DT DJ DJ DJ
238 // line_state_valid -----
239 // bit_phase 0 0 1 2 3 0 1 2 3 0
1 2 0 1 2
240
241 logic [1:0] bit_phase_q, bit_phase_d;
242 logic line_state_valid;
243
244 assign line_state_valid = (bit_phase_q == 2'd1);
245 assign bit_strobe_o = (bit_phase_q == 2'd2);
246
247 // keep track of phase within each bit
248
249 assign bit_phase_d = (line_state_rx == DT) ? 0 : bit_phase_q +
1;
250
251 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bit_phase_q
252 if (!rst_ni) begin
253 bit_phase_q <= 0;
254 end else begin
255 if (link_reset_i) begin
256 bit_phase_q <= 0;
257 end else begin
258 bit_phase_q <= bit_phase_d;
259 end
260 end
261 end
262
263 // packet detection
264 // packet detection
265 // packet detection
266 // packet detection
267
268 // usb uses a sync to denote the beginning of a packet's PID
and two
269 // single-ended-0 to denote the end of a packet. this state
machine
270 // recognizes the beginning and end of packets for sub-
sequent layers to
271 // process.
272
273 logic [11:0] line_history_q, line_history_d;
274 logic packet_valid_q, packet_valid_d;
275 logic see_sop, see_eop, see_preamble, packet_start, pack-
et_end;
276 logic in_packet_d, in_packet_q;
277
278 // A bit of a misnomer: packet_start pulses when the PID be-
gins, not SOP.
279 assign packet_start = packet_valid_d & ~packet_valid_q;
280 assign packet_end = ~packet_valid_d & packet_valid_q;
281
282 // EOP detection is configurable for 1/2 bit periods of SE0.
283 // The standard (Table 7-7) mandates min = 82 ns = 1 bit
period.
284 // We also trigger an EOP on seeing a bitstuff error or a
PRE PID.
285 assign see_eop = (cfg_eop_single_bit_i && line_his-
tory_q[1:0] == 2'b00);
286
287 || (line_history_q[3:0] == 4'b0000) || bitstuff_error_q ||
see_preamble;
288
289 // SOP is the transition from idle (J) to K
289 assign see_sop = (line_history_q[3:0] == 4'b1001) & ~tx_en_i &
~in_packet_q;
290 assign in_packet_d = see_eop ? 1'b0 :
291 see_sop ? 1'b1 : in_packet_q;
292
293 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg_in_packet
294 if (!rst_ni) begin
295 in_packet_q <= 1'b0;
296 end else begin
297 in_packet_q <= in_packet_d;
298 end
299 end
300
301 always_comb begin : proc_packet_valid_d
302 if (line_state_valid) begin
303 // If the differential K and J symbols are not valid,
reject the
304 // containing packet as invalid.
305 if (~diff_rx_ok_i) begin
306 packet_valid_d = 0;
307 // check for packet start: KJKJKK, we use the last 6
bits
308 end else if (!packet_valid_q && line_history_q[11:0] ==
12'b011001100101) begin
309 packet_valid_d = 1;
310 end
311
312 // check for packet end: SE0 SE0
313 else if (packet_valid_q && see_eop) begin
314 packet_valid_d = 0;
315
316 end else begin
317 packet_valid_d = packet_valid_q;
318 end
319 end else begin
320 packet_valid_d = packet_valid_q;
321 end
322 end
323
324 // keep a history of the last two states on the line
325 assign line_history_d = line_state_valid ? {line_his-
tory_q[9:0], line_state_rx[1:0]} :
line_history_q;
326
327 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg_pkt_line
328 if (!rst_ni) begin
329 packet_valid_q <= 0;
330 line_history_q <= 12'b0101010101010; // all K
331 end else begin
332 if (link_reset_i) begin
333 packet_valid_q <= 0;
334 line_history_q <= 12'b0101010101010; // all K
335 end else begin
336 packet_valid_q <= packet_valid_d;
337 line_history_q <= line_history_d;
338 end
339 end
340 end
341 end
342
343 // mask out idle detection when transmitting (because rx may
be forced to
344 // J and look like an idle symbol)
345 logic rx_idle_det_d, rx_idle_det_q;
346 assign rx_idle_det_d = (~tx_en_i & line_state_valid) ?
(line_state_q == DJ) : rx_idle_det_q;
347 assign rx_idle_det_o = rx_idle_det_q;
348
349 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg_idle_det
350 if (!rst_ni) begin
351 rx_idle_det_q <= 1'b0;
352 end else begin
353 rx_idle_det_q <= rx_idle_det_d;
354 end
355 end
356
357 // Used for seeing a J after the completion of resume sig-
naling
358 assign rx_j_det_o = diff_rx_ok_i & ~tx_en_i & (line_his-
tory_q[1:0] == 2'b10);
359
360 // NRZI decode
361 // NRZI decode
362 // NRZI decode
363
364 // in order to ensure there are enough bit transitions for a
receiver to recover
365 // the clock usb uses NRZI encoding.
366
367 // https://en.wikipedia.org/wiki/Non-return-to-zero
368
369 logic dvalid_raw;
370 logic din;
371
372 always_comb begin
373 unique case (line_history_q[3:0])
374 4'b0101 : din = 1;
375 4'b0110 : din = 0;
376 4'b1001 : din = 0;
377 4'b1010 : din = 1;
378 default : din = 0;
379 endcase
380
381 if (packet_valid_q && line_state_valid) begin

```

```

382     unique case (line_history_q[3:0])
383     4'b0101 : dvalid_raw = 1;
384     4'b0110 : dvalid_raw = 1;
385     4'b1001 : dvalid_raw = 1;
386     4'b1010 : dvalid_raw = 1;
387     default : dvalid_raw = 0;
388   endcase
389   end else begin
390     dvalid_raw = 0;
391   end
392 end
393
394 ///////////////////////////////////////////////////////////////////
395 // Undo bit stuffing and detect bit stuffing errors //
396 ///////////////////////////////////////////////////////////////////
397
398 always_comb begin : proc_bitstuff_history_d
399   if (packet_end) begin
400     bitstuff_history_d = '0;
401   end else if (dvalid_raw) begin
402     bitstuff_history_d = {bitstuff_history_q[5:0], din};
403   end else begin
404     bitstuff_history_d = bitstuff_history_q;
405   end
406 end
407
408 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bitstuff_history_q
409   if (!rst_ni) begin
410     bitstuff_history_q <= 0;
411   end else begin
412     if (link_reset_i) begin
413       bitstuff_history_q <= 0;
414     end else begin
415       bitstuff_history_q <= bitstuff_history_d;
416     end
417   end
418 end
419
420 logic dvalid;
421 assign dvalid = dvalid_raw && !(bitstuff_history_q[5:0] ==
6'b111111);
422
423 // 7 consecutive ones should not be seen on the bus
424 // USB spec, 7.1.9.1: "If the receiver sees seven
425 // consecutive ones anywhere in the packet, then a bit
stuffing error
426 // has occurred and the packet should be ignored."
427 assign bitstuff_error = bitstuff_history_q == 7'b1111111;
428
429 // remember the bitstuff errors
430 always_comb begin : proc_bitstuff_error_d
431   bitstuff_error_d = bitstuff_error_q;
432   if (packet_start) begin
433     bitstuff_error_d = 0;
434   end else if (bitstuff_error && dvalid_raw) begin
435     bitstuff_error_d = 1;
436   end
437 end
438
439 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bitstuff_error_q
440   if (!rst_ni) begin
441     bitstuff_error_q <= 0;
442   end else begin
443     bitstuff_error_q <= bitstuff_error_d;
444   end
445 end
446
447 assign bitstuff_error_o = bitstuff_error_q && packet_end;
448
449
450 ///////////////////////////////////////////////////////////////////
451 // save and check pid //
452 ///////////////////////////////////////////////////////////////////
453
454 // shift in the entire 8-bit pid with an additional 9th bit
used as a sentinel.
455
456 logic [8:0] full_pid_q, full_pid_d;
457 logic pid_valid, pid_complete;
458
459 assign pid_valid = full_pid_q[4:1] == ~full_pid_q[8:5];
460 assign pid_complete = full_pid_q[0];
461
462 always_comb begin : proc_full_pid_d
463   if (dvalid && !pid_complete) begin
464     full_pid_d = {din, full_pid_q[8:1]};
465   end else if (packet_start) begin
466     full_pid_d = 9'b100000000;
467   end else begin
468     full_pid_d = full_pid_q;
469   end
470 end
471
472 ///////////////////////////////////////////////////////////////////
473 // check crc5 //
474 ///////////////////////////////////////////////////////////////////
475 logic [4:0] crc5_q, crc5_d;
476 logic crc5_valid, crc5_invert;
477 assign crc5_valid = crc5_q == 5'b01100;

```

```

478 assign crc5_invert = din ^ crc5_q[4];
479
480 always_comb begin
481   crc5_d = crc5_q; // default value
482
483   if (packet_start) begin
484     crc5_d = 5'b11111;
485   end
486
487   if (dvalid && pid_complete) begin
488     crc5_d = {crc5_q[3:0], 1'b0} ^ ({5{crc5_invert}} &
5'b00101);
489   end
490 end
491
492
493 ///////////////////////////////////////////////////////////////////
494 // check crc16 //
495 ///////////////////////////////////////////////////////////////////
496 logic [15:0] crc16_q, crc16_d;
497 logic crc16_valid, crc16_invert;
498
499 assign crc16_valid = crc16_q == 16'b100000000000101;
500 assign crc16_invert = din ^ crc16_q[15];
501
502 always_comb begin
503   crc16_d = crc16_q; // default value
504
505   if (packet_start) begin
506     crc16_d = 16'b111111111111111;
507   end
508
509   if (dvalid && pid_complete) begin
510     crc16_d = {crc16_q[14:0], 1'b0} ^ ({16{crc16_invert}} &
16'b100000000000101);
511   end
512 end
513
514
515 ///////////////////////////////////////////////////////////////////
516 // output control signals //
517 ///////////////////////////////////////////////////////////////////
518 logic valid_token_len, valid_data_len, valid_handshake_len;
519
520 logic pkt_is_token, pkt_is_data, pkt_is_handshake;
521 assign pkt_is_token = full_pid_q[2:1] == 2'b01;
522 assign pkt_is_data = full_pid_q[2:1] == 2'b11;
523 assign pkt_is_handshake = full_pid_q[2:1] == 2'b10;
524
525 assign see_preamble = packet_valid_q & pid_valid & pid_com-
plete &&
526   (usb_pid_e'(full_pid_q[4:1]) == Us-
bPidPre);
527
528 assign valid_pid_o = pid_valid;
529
530 assign valid_packet_o = pid_valid && !bitstuff_error_q &&
531   ((pkt_is_handshake && valid_handshake_len) ||
532   (pkt_is_data && valid_data_len && crc16_valid) ||
533   (pkt_is_token && valid_token_len && crc5_valid));
534
535 // Detect CRC errors
536 assign crc5_error_o = pkt_is_token & packet_end & !
crc5_valid;
537 assign crc16_error_o = pkt_is_data & packet_end & !
crc16_valid;
538
539 // Detect PID errors
540 assign pid_error_o = !pid_valid && packet_end;
541
542 logic [11:0] token_payload_q, token_payload_d;
543 logic token_payload_done;
544
545 assign token_payload_done = token_payload_q[0];
546
547 logic [6:0] addr_q, addr_d;
548 logic [3:0] endp_q, endp_d;
549 logic [10:0] frame_num_q, frame_num_d;
550
551 always_comb begin
552   token_payload_d = token_payload_q; // default
553
554   if (packet_start) begin
555     token_payload_d = 12'b100000000000;
556   end
557
558   if (dvalid && pid_complete && pkt_is_token && !
token_payload_done) begin
559     token_payload_d = {din, token_payload_q[11:1]};
560   end
561 end
562
563 always_comb begin
564   // defaults
565   addr_d = addr_q;
566   endp_d = endp_q;
567   frame_num_d = frame_num_q;
568
569   if (token_payload_done && pkt_is_token) begin
570     addr_d = token_payload_q[7:1];
571     endp_d = token_payload_q[11:8];

```

```

572     frame_num_d = token_payload_q[11:1];
573 end
574 end
575
576 assign addr_o      = addr_q;
577 assign endp_o      = endp_q;
578 assign frame_num_o = frame_num_q;
579 assign pid_o       = full_pid_q[4:1];
580
581 assign pkt_start_o = see_sop;
582 assign pkt_end_o   = packet_end;
583
584
585 //////////////////////////////////////////////////
586 // deserialize and output data //
587 //////////////////////////////////////////////////
588 logic [8:0] rx_data_buffer_q, rx_data_buffer_d;
589 logic rx_data_buffer_full;
590
591 assign rx_data_buffer_full = rx_data_buffer_q[0];
592 assign rx_data_put_o      = rx_data_buffer_full;
593 assign rx_data_o          = rx_data_buffer_q[8:1];
594
595 always_comb begin
596     rx_data_buffer_d = rx_data_buffer_q; // default
597
598     if (packet_start || rx_data_buffer_full) begin
599         rx_data_buffer_d = 9'b100000000;
600     end
601
602     if (dvalid && pid_complete && pkt_is_data) begin
603         rx_data_buffer_d = {din, rx_data_buffer_q[8:1]};
604     end
605 end
606
607 // Counting packet length in bits; we need to know whether
the length is byte-aligned,
608 // and whether it's equal to 16 (token packet) or at least
16 (data packet; CRC16 required).
609 logic rx_data_len16_q, rx_data_len16_d;
610 logic [3:0] rx_data_len_q, rx_data_len_d;
611
612 // Valid packet length indications.
613 assign valid_token_len = rx_data_len16_q & ~|
rx_data_len_q;
614 assign valid_data_len = rx_data_len16_q & ~|
rx_data_len_q[2:0]; // Must be byte-aligned.
615 assign valid_handshake_len = ~|{rx_data_len16_q,
rx_data_len_q};
616
617 always_comb begin
618     rx_data_len16_d = rx_data_len16_q;
619     rx_data_len_d   = rx_data_len_q;
620
621     if (packet_start) begin
622         rx_data_len16_d = 1'b0;
623         rx_data_len_d   = 4'h0;
624     end
625     if (dvalid && pid_complete) begin
626         rx_data_len16_d = rx_data_len16_q | &rx_data_len_q;
627         rx_data_len_d   = rx_data_len_q + 4'h1;
628     end
629 end
630
631 //////////////////////////////////////////////////
632 // Registers //
633 //////////////////////////////////////////////////
634 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_gp_regs
635     if (!rst_ni) begin
636         full_pid_q      <= 0;
637         crc16_q         <= 0;
638         crc5_q          <= 0;
639         token_payload_q <= 0;
640         addr_q          <= 0;
641         endp_q          <= 0;
642         frame_num_q     <= 0;
643         rx_data_buffer_q <= 0;
644         rx_data_len16_q <= 0;
645         rx_data_len_q   <= 0;
646     end else begin
647         if (link_reset_i) begin
648             full_pid_q      <= 0;
649             crc16_q         <= 0;
650             crc5_q          <= 0;
651             token_payload_q <= 0;
652             addr_q          <= 0;
653             endp_q          <= 0;
654             frame_num_q     <= 0;
655             rx_data_buffer_q <= 0;
656             rx_data_len16_q <= 0;
657             rx_data_len_q   <= 0;
658         end else begin
659             full_pid_q      <= full_pid_d;
660             crc16_q         <= crc16_d;
661             crc5_q          <= crc5_d;
662             token_payload_q <= token_payload_d;
663             addr_q          <= addr_d;
664             endp_q          <= endp_d;
665             frame_num_q     <= frame_num_d;
666             rx_data_buffer_q <= rx_data_buffer_d;

```

```

667     rx_data_len16_q      <= rx_data_len16_d;
668     rx_data_len_q       <= rx_data_len_d;
669 end
670 end
671 end
672
673 endmodule // usb_fs_rx

```

6. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx.sv (508 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 module usb_fs_tx (
7     // A 48MHz clock is required to receive USB data at 12MHz
8     // it's simpler to just use 48MHz everywhere
9     input logic clk_i,
10    input logic rst_ni, // async reset
11    input logic link_reset_i, // USB reset, sync to 48 MHz,
active high
12    input logic cfg_pinflip_i,
13
14    // Oscillator test mode (constantly output JK)
15    input logic tx_osc_test_mode_i,
16
17    // bit strobe from rx to align with senders clock
18    input logic bit_strobe_i,
19
20    // output enable to take ownership of bus and data out
21    output logic usb_oe_o,
22    output logic usb_d_o,
23    output logic usb_se0_o,
24    output logic usb_dp_o,
25    output logic usb_dn_o,
26
27    // pulse to initiate new packet transmission
28    input logic pkt_start_i,
29    output logic pkt_end_o,
30
31    // pid_i to send
32    input logic [3:0] pid_i,
33
34    // tx logic pulls data until there is nothing available
35    input logic tx_data_avail_i,
36    output logic tx_data_get_o,
37    input logic [7:0] tx_data_i
38);
39
40
41 typedef enum logic [2:0] {Idle, Sync, Pid, Data0rCrc160,
Crc161, Eop, OscTest} state_e;
42 typedef enum logic [1:0] {0sIdle, 0sWaitByte, 0sTransmit}
out_state_e;
43
44 //////////////////////////////////////////////////
45 // Signal Declarations //
46 //////////////////////////////////////////////////
47 //////////////////////////////////////////////////
48 logic [3:0] pid_q, pid_d;
49 logic bitstuff;
50 logic bitstuff_q;
51 logic bitstuff_q2;
52 logic bitstuff_q3;
53 logic bitstuff_q4;
54
55 logic [5:0] bit_history;
56
57 state_e state_d, state_q;
58 out_state_e out_state_d, out_state_q;
59
60
61 logic [7:0] data_shift_reg_q, data_shift_reg_d;
62 logic [7:0] oe_shift_reg_q, oe_shift_reg_d;
63 logic [7:0] se0_shift_reg_q, se0_shift_reg_d;
64 logic data_payload_q, data_payload_d;
65 logic tx_data_get_q, tx_data_get_d;
66 logic byte_strobe_q, byte_strobe_d;
67 logic [4:0] bit_history_d, bit_history_q;
68 logic [2:0] bit_count_d, bit_count_q;
69
70 logic [15:0] crc16_d, crc16_q;
71
72 logic oe_q, oe_d;
73 logic usb_d_q, usb_d_d;
74 logic usb_se0_q, usb_se0_d;
75 logic [2:0] dp_eop_q, dp_eop_d;
76
77 logic test_mode_start;
78 logic serial_tx_data;
79 logic serial_tx_oe;
80 logic serial_tx_se0;
81 logic crc16_invert;
82 logic pkt_end;
83 logic out_nrzi_en;
84

```



```

85 // save packet parameters at pkt_start_i
86 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_pid
87   if (!rst_ni) begin
88     pid_q <= 0;
89   end else begin
90     if (link_reset_i) begin
91       pid_q <= 0;
92     end else begin
93       pid_q <= pid_d;
94     end
95   end
96 end
97
98 assign pid_d = pkt_start_i ? pid_i : pid_q;
99
100
101 assign serial_tx_data = data_shift_reg_q[0];
102 assign serial_tx_oe = oe_shift_reg_q[0];
103 assign serial_tx_se0 = se0_shift_reg_q[0];
104
105
106 // serialize sync, pid_i, data payload, and crc16
107 assign bit_history = {serial_tx_data, bit_history_q};
108 assign bitstuff    = bit_history == 6'b111111;
109
110 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_bitstuff
111   if (!rst_ni) begin
112     bitstuff_q <= 0;
113     bitstuff_q2 <= 0;
114     bitstuff_q3 <= 0;
115     bitstuff_q4 <= 0;
116   end else begin
117     if (link_reset_i) begin
118       bitstuff_q <= 0;
119       bitstuff_q2 <= 0;
120       bitstuff_q3 <= 0;
121       bitstuff_q4 <= 0;
122     end else begin
123       bitstuff_q <= bitstuff;
124       bitstuff_q2 <= bitstuff_q;
125       bitstuff_q3 <= bitstuff_q2;
126       bitstuff_q4 <= bitstuff_q3;
127     end
128   end
129 end
130
131 assign pkt_end    = bit_strobe_i && se0_shift_reg_q[1:0] ==
2'b01;
132 assign pkt_end_o = pkt_end;
133
134
135 // FSM //
136 // FSM //
137 // FSM //
138 always_comb begin : proc_fsm
139   // Default assignments
140   state_d = state_q;
141   data_shift_reg_d = data_shift_reg_q;
142   oe_shift_reg_d = oe_shift_reg_q;
143   se0_shift_reg_d = se0_shift_reg_q;
144   data_payload_d = data_payload_q;
145   tx_data_get_d = tx_data_get_q;
146   bit_history_d = bit_history_q;
147   bit_count_d = bit_count_q;
148   test_mode_start = 0;
149
150   unique case (state_q)
151     Idle : begin
152       if (tx_osc_test_mode_i) begin
153         state_d = OscTest;
154         test_mode_start = 1;
155       end else if (pkt_start_i) begin
156         state_d = Sync;
157       end
158     end
159
160     Sync : begin
161       if (byte_strobe_q) begin
162         state_d = Pid;
163         data_shift_reg_d = 8'b10000000;
164         oe_shift_reg_d = 8'b11111111;
165         se0_shift_reg_d = 8'b00000000;
166       end
167     end
168
169     Pid : begin
170       if (byte_strobe_q) begin
171         if (pid_q[1:0] == 2'b11) begin
172           state_d = Data0rCrc160;
173         end else begin
174           state_d = Eop;
175         end
176
177         data_shift_reg_d = {~pid_q, pid_q};
178         oe_shift_reg_d = 8'b11111111;
179         se0_shift_reg_d = 8'b00000000;
180       end
181     end
182

```

```

183   Data0rCrc160 : begin
184     if (byte_strobe_q) begin
185       if (tx_data_avail_i) begin
186         state_d = Data0rCrc160;
187         data_payload_d = 1;
188         tx_data_get_d = 1;
189         data_shift_reg_d = tx_data_i;
190         oe_shift_reg_d = 8'b11111111;
191         se0_shift_reg_d = 8'b00000000;
192       end else begin
193         state_d = Crc161;
194         data_payload_d = 0;
195         tx_data_get_d = 0;
196         data_shift_reg_d = ~(crc16_q[8], crc16_q[9],
183         crc16_q[10], crc16_q[11],
184         crc16_q[12], crc16_q[13],
185         crc16_q[14], crc16_q[15]);
197         oe_shift_reg_d = 8'b11111111;
198         se0_shift_reg_d = 8'b00000000;
199       end
200     end else begin
201       tx_data_get_d = 0;
202     end
203   end
204 end
205
206   Crc161 : begin
207     if (byte_strobe_q) begin
208       state_d = Eop;
209       data_shift_reg_d = ~(crc16_q[0], crc16_q[1],
183         crc16_q[2], crc16_q[3],
184         crc16_q[4], crc16_q[5],
185         crc16_q[6], crc16_q[7]);
210       oe_shift_reg_d = 8'b11111111;
211       se0_shift_reg_d = 8'b00000000;
212     end
213   end
214 end
215
216   Eop : begin
217     if (byte_strobe_q) begin
218       state_d = Idle;
219       oe_shift_reg_d = 8'b00000011;
220       se0_shift_reg_d = 8'b00000011;
221     end
222   end
223 end
224
225   OscTest: begin
226     // Oscillator test mode: toggle constantly
227     if (!tx_osc_test_mode_i && byte_strobe_q) begin
228       oe_shift_reg_d = 8'b00000000;
229       state_d = Idle;
230     end else if (byte_strobe_q) begin
231       data_shift_reg_d = 8'b00000000;
232       oe_shift_reg_d = 8'b11111111;
233       se0_shift_reg_d = 8'b00000000;
234     end
235   end
236 end
237
238   default: state_d = Idle;
239 endcase
240
241 // Logic closely coupled to the FSM
242 if (pkt_start_i) begin
243   // We need to have an inter-packet delay between
244   // 2 and 6.5 bit times (see USB 2.0 spec / 7.1.18.1)
245   // The latency in the rest of the system is approxi-
246   ately (measured)
247   // 3.68 bit-times, so we only introduce 1 bit-time here
248   bit_count_d = 7; // 8-7 = 1
249   bit_history_d = 0;
250 end else if (bit_strobe_i) begin
251   // bitstuff
252   if (bitstuff /* && !serial_tx_se0 */) begin
253     bit_history_d = bit_history[5:1];
254     data_shift_reg_d[0] = 0;
255   end
256
257   // normal deserialize
258   end else begin
259     bit_count_d = bit_count_q + 1;
260
261     data_shift_reg_d = (data_shift_reg_q >> 1);
262     oe_shift_reg_d = (oe_shift_reg_q >> 1);
263     se0_shift_reg_d = (se0_shift_reg_q >> 1);
264
265     bit_history_d = bit_history[5:1];
266   end
267 end
268
269   `ASSERT(StateValid_A, state_q inside {Idle, Sync, Pid,
Data0rCrc160, Crc161, Eop, OscTest})
270
271   always_comb begin : proc_byte_str
272     if (bit_strobe_i && !bitstuff && !pkt_start_i) begin
273       byte_strobe_d = (bit_count_q == 3'b000);
274     end else begin
275       byte_strobe_d = 0;
276     end
277   end
278 end
279

```

```

278 assign tx_data_get_o = tx_data_get_q;
279
280 // calculate crc16
281 assign crc16_invert = serial_tx_data ^ crc16_q[15];
282
283 always_comb begin : proc_crc16
284   crc16_d = crc16_q; // default assignment
285
286   if (pkt_start_i) begin
287     crc16_d = 16'b1111111111111111;
288   end
289
290   if (bit_strobe_i && data_payload_q && !bitstuff_q4 && !
pkt_start_i) begin
291     crc16_d = {crc16_q[14:0], 1'b0} ^ ({16{crc16_invert}} &
16'b10000000000000101);
292   end
293 end
294
295 // Regular Registers //
296 // Regular Registers //
297
298 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_reg
299   if (!rst_ni) begin
300     state_q <= Idle;
301     data_payload_q <= 0;
302     data_shift_reg_q <= 0;
303     oe_shift_reg_q <= 0;
304     se0_shift_reg_q <= 0;
305     tx_data_get_q <= 0;
306     byte_strobe_q <= 0;
307     bit_history_q <= 0;
308     bit_count_q <= 0;
309     crc16_q <= 0;
310   end else begin
311     if (link_reset_i) begin
312       state_q <= Idle;
313       data_payload_q <= 0;
314       data_shift_reg_q <= 0;
315       oe_shift_reg_q <= 0;
316       se0_shift_reg_q <= 0;
317       tx_data_get_q <= 0;
318       byte_strobe_q <= 0;
319       bit_history_q <= 0;
320       bit_count_q <= 0;
321       crc16_q <= 0;
322     end else begin
323       state_q <= state_d;
324       data_payload_q <= data_payload_d;
325       data_shift_reg_q <= data_shift_reg_d;
326       oe_shift_reg_q <= oe_shift_reg_d;
327       se0_shift_reg_q <= se0_shift_reg_d;
328       tx_data_get_q <= tx_data_get_d;
329       byte_strobe_q <= byte_strobe_d;
330       bit_history_q <= bit_history_d;
331       bit_count_q <= bit_count_d;
332       crc16_q <= crc16_d;
333     end
334   end
335 end
336
337 // nrzi and differential driving //
338 // nrzi and differential driving //
339
340 // Output FSM
341 always_comb begin : proc_out_fsm
342   out_state_d = out_state_q;
343   out_nrzi_en = 1'b0;
344
345   unique case (out_state_q)
346     OsIdle: begin
347       if (pkt_start_i || test_mode_start) begin
348         out_state_d = OsWaitByte;
349       end
350     end
351     OsWaitByte: begin
352       if (byte_strobe_q) begin
353         out_state_d = OsTransmit;
354       end
355     end
356     OsTransmit: begin
357       out_nrzi_en = 1'b1;
358       if ((bit_strobe_i && !serial_tx_oe)) begin
359         out_state_d = OsIdle;
360       end
361     end
362     default : out_state_d = OsIdle;
363   endcase
364 end
365
366 `ASSERT(OutStateValid_A, out_state_q inside {OsIdle, OsWait-
Byte, OsTransmit})
367
368 always_comb begin : proc_diff
369   usb_d_d = usb_d_q;

```

```

375   usb_se0_d = usb_se0_q;
376   oe_d = oe_q;
377   dp_eop_d = dp_eop_q;
378
379   if (pkt_start_i) begin
380     usb_d_d = 1; // J -> first bit will be K (start of sync)
381     dp_eop_d = 3'b100; // Eop: {SE0, SE0, J}
382   end else if (bit_strobe_i && out_nrzi_en) begin
383     oe_d = serial_tx_oe;
384
385     if (serial_tx_se0) begin
386       // Eop
387       dp_eop_d = dp_eop_q >> 1;
388
389       if (dp_eop_q[0]) begin
390         // last bit of Eop: J
391         usb_d_d = 1;
392         usb_se0_d = 0;
393       end else begin
394         // first two bits of Eop: SE0
395         usb_se0_d = 1;
396       end
397     end
398   end else if (serial_tx_data) begin
399     // value should stay the same, do nothing
400   end else begin
401     usb_d_d = !usb_d_q;
402   end
403 end
404
405 // Set to J state when OE=0 to avoid
406 // glitches
407 if (!oe_d) begin
408   usb_d_d = 1;
409 end
410
411 always_ff @(posedge clk_i or negedge rst_ni) begin :
proc_diff_reg
412   if (!rst_ni) begin
413     dp_eop_q <= 0;
414     out_state_q <= OsIdle;
415   end else begin
416     if (link_reset_i) begin
417       dp_eop_q <= 0;
418       out_state_q <= OsIdle;
419     end else begin
420       dp_eop_q <= dp_eop_d;
421       out_state_q <= out_state_d;
422     end
423   end
424 end
425
426 prim_flop u_oe_flop (
427   .clk_i,
428   .rst_ni,
429   .d_i(link_reset_i ? 1'b0 : oe_d),
430   .q_o(oe_q)
431 );
432
433 prim_flop #(
434   .ResetValue(1) // J state = idle state
435 ) u_usb_d_flop (
436   .clk_i,
437   .rst_ni,
438   .d_i(link_reset_i ? 1'b1 : usb_d_d),
439   .q_o(usb_d_q)
440 );
441
442 prim_flop u_usb_se0_flop (
443   .clk_i,
444   .rst_ni,
445   .d_i(link_reset_i ? 1'b0 : usb_se0_d),
446   .q_o(usb_se0_q)
447 );
448
449 // Handle the D+ / D- pin flip on the USB side, and provide
both the
450 // dp/dn and d/se0 interfaces, for compatibility with mul-
tiple
451 driver types.
452 logic usb_se0_flipped, usb_dp_flipped, usb_dn_flipped;
453
454 always_comb begin
455   if (link_reset_i) begin
456     usb_se0_flipped = 1'b0;
457     usb_dp_flipped = !cfg_pinflip_i;
458     usb_dn_flipped = !cfg_pinflip_i;
459   end else begin
460     usb_se0_flipped = usb_se0_d;
461     usb_dp_flipped = (cfg_pinflip_i ? ~usb_d_d : usb_d_d) &
~usb_se0_d;
462     usb_dn_flipped = (cfg_pinflip_i ? usb_d_d : ~usb_d_d) &
~usb_se0_d;
463   end
464 end

```



```

469 // Use registered outputs for the I/Os
470 prim_flop #(
471   .ResetValue(1) // J state = idle state
472 ) u_usb_d_o_flop (
473   .clk_i,
474   .rst_ni,
475   .d_i(usb_dp_flipped), // Note: single-ended 'D' output
mirrors D+
476   .q_o(usb_d_o)
477 );
478
479 prim_flop #(
480   .ResetValue(0) // J state = idle state
481 ) u_usb_se0_o_flop (
482   .clk_i,
483   .rst_ni,
484   .d_i(usb_se0_flipped),
485   .q_o(usb_se0_o)
486 );
487
488 prim_flop #(
489   .ResetValue(1) // J state = idle state
490 ) u_usb_dp_o_flop (
491   .clk_i,
492   .rst_ni,
493   .d_i(usb_dp_flipped),
494   .q_o(usb_dp_o)
495 );
496
497 prim_flop #(
498   .ResetValue(0) // J state = idle state
499 ) u_usb_dn_o_flop (
500   .clk_i,
501   .rst_ni,
502   .d_i(usb_dn_flipped),
503   .q_o(usb_dn_o)
504 );
505
506 assign usb_oe_o = oe_q;
507
508 endmodule

```

7. lowRISC_opentitan/hw/ip/usbdev/rtl/usb_fs_tx_mux.sv (23 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 // Copyright Luke Valenty (TinyFPGA project, https://git-
  hub.com/tinyfpga/TinyFPGA-Bootloader).
5
6 module usb_fs_tx_mux (
7   // interface to IN Protocol Engine
8   input logic in_tx_pkt_start_i,
9   input logic [3:0] in_tx_pid_i,
10
11   // interface to OUT Protocol Engine
12   input logic out_tx_pkt_start_i,
13   input logic [3:0] out_tx_pid_i,
14
15   // interface to tx module
16   output logic tx_pkt_start_o,
17   output logic [3:0] tx_pid_o
18 );
19
20 assign tx_pkt_start_o = in_tx_pkt_start_i |
  out_tx_pkt_start_i;
21 assign tx_pid_o = out_tx_pkt_start_i ? out_tx_pid_i :
  in_tx_pid_i;
22
23 endmodule

```

8. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev.sv (1,470 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // USB Full Speed Device Interface (usbdev).
6
7 `include "prim_assert.sv"
8
9 module usbdev
10   import usbdev_pkg::*;
11   import usbdev_reg_pkg::*;
12   import prim_util_pkg::vbits;
13 # (
14   parameter bit Stub = 1'b0,
15   parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
  mAlerts{1'b1}},
16   // Number of cycles of differential skew tolerated on the
  alert signal.
17   parameter int unsigned AlertSkewCycles = 1,
18   // Max time (in microseconds) from rx_enable_o high to the
19   // external differential receiver outputting valid data

```

```

(when
20   // configured to use one).
21   parameter int unsigned RcvrWakeTimeUs = 1
22 ) (
23   input logic clk_i,
24   input logic rst_ni,
25   input logic clk_aon_i,
26   input logic rst_aon_ni,
27
28   // Register interface
29   input tlul_pkg::tl_h2d_t tl_i,
30   output tlul_pkg::tl_d2h_t tl_o,
31
32   // Alerts
33   input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
  alert_rx_i,
34   output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
  alert_tx_o,
35
36   // Data inputs
37   input logic cio_usb_dp_i, // differential P, can be
  used in single-ended mode to detect SE0
38   input logic cio_usb_dn_i, // differential N, can be
  used in single-ended mode to detect SE0
39   input logic usb_rx_d_i, // single-ended input from
  the differential receiver
40
41   // Data outputs
42   output logic cio_usb_dp_o,
43   output logic cio_usb_dp_en_o,
44   output logic cio_usb_dn_o,
45   output logic cio_usb_dn_en_o,
46   output logic usb_tx_se0_o, // single-ended zero output
47   output logic usb_tx_d_o,
48
49   // Non-data I/O
50   input logic cio_sense_i,
51   output logic usb_dp_pullup_o,
52   output logic usb_dn_pullup_o,
53   output logic usb_rx_enable_o,
54   output logic usb_tx_use_d_se0_o,
55
56   // Direct pinmux aon detect connections
57   output logic usb_aon_suspend_req_o,
58   output logic usb_aon_wake_ack_o,
59
60   // Events and state from wakeup module
61   input logic usb_aon_bus_reset_i,
62   input logic usb_aon_sense_lost_i,
63   input logic usb_aon_bus_not_idle_i,
64   input logic usb_aon_wake_detect_active_i,
65
66   // SOF reference for clock calibration
67   output logic usb_ref_val_o,
68   output logic usb_ref_pulse_o,
69
70   // memory configuration
71   input prim_ram_ip_pkg::ram_ip_cfg_req_t ram_cfg_i,
72   output prim_ram_ip_pkg::ram_ip_cfg_rsp_t ram_cfg_o,
73
74   // Interrupts
75   output logic intr_pkt_received_o, // Packet received
76   output logic intr_pkt_sent_o, // Packet sent
77   output logic intr_powered_o,
78   output logic intr_disconnected_o,
79   output logic intr_host_lost_o,
80   output logic intr_link_reset_o,
81   output logic intr_link_suspend_o,
82   output logic intr_link_resume_o,
83   output logic intr_av_out_empty_o,
84   output logic intr_rx_full_o,
85   output logic intr_av_overflow_o,
86   output logic intr_link_in_err_o,
87   output logic intr_link_out_err_o,
88   output logic intr_rx_crc_err_o,
89   output logic intr_rx_pid_err_o,
90   output logic intr_rx_bitstuff_err_o,
91   output logic intr_frame_o,
92   output logic intr_av_setup_empty_o
93 );
94
95 // Could make SramDepth, MaxPktSizeByte, AVSetupFifoDepth,
  AVOutFifoDepth and RXFifoDepth
96 // module parameters but may need to fix register def for
  the first two
97 localparam int SramDw = 32; // Places packing bytes to SRAM
  assume this
98 localparam int SramDepth = 512; // 2kB, SRAM width is DW
99 localparam int MaxPktSizeByte = 64;
100
101 localparam int SramAw = $clog2(SramDepth);
102 localparam int SizeWidth = $clog2(MaxPktSizeByte);
103
104 localparam int NBuf = (SramDepth * SramDw) / (MaxPktSizeByte *
  8);
105
106 localparam int NBufWidth = $clog2(NBuf);
107
108 // AV SETUP and AV OUT fifos just store buffer numbers
109 localparam int AVFifoWidth = NBufWidth;
110 localparam int AVSetupFifoDepth = 4;
111 localparam int AVOutFifoDepth = 8;

```

```

110
111 // RX fifo stores          buf# + size(0-MaxPktSize-
Byte) + EP# + Type
112 localparam int RXFifoWidth = NBufWidth +
(1+SizeWidth) + 4 + 1;
113 localparam int RXFifoDepth = 8;
114 // derived parameter
115 localparam int RXFifoDepthW =
prim_util_pkg::vbits(RXFifoDepth+1);
116
117 usbdev_reg2hw_t reg2hw, reg2hw_regtop;
118 usbdev_hw2reg_t hw2reg, hw2reg_regtop;
119
120 logic rst_n;
121 if (Stub) begin : gen_stubbed_reset
122     assign rst_n = '0;
123 end else begin : gen_no_stubbed_reset
124     assign rst_n = rst_n1;
125 end
126
127 tlul_pkg::tl_h2d_t tl_sram_h2d;
128 tlul_pkg::tl_d2h_t tl_sram_d2h;
129
130 // Software access to the Packet Buffer RAM
131 logic sw_mem_a_req;
132 logic sw_mem_a_gnt;
133 logic sw_mem_a_write;
134 logic [SramAw-1:0] sw_mem_a_addr;
135 logic [SramDw-1:0] sw_mem_a_wdata;
136 logic sw_mem_a_rvalid;
137 logic [SramDw-1:0] sw_mem_a_rdata;
138 logic [1:0] sw_mem_a_rerror;
139
140 // usbdev hardware access to the Packet Buffer RAM
141 logic usb_mem_b_req;
142 logic usb_mem_b_write;
143 logic [SramAw-1:0] usb_mem_b_addr;
144 logic [SramDw-1:0] usb_mem_b_wdata;
145 logic [SramDw-1:0] usb_mem_b_rdata;
146
147 logic clr_devaddr;
148
149 logic event_av_setup_empty, event_av_out_empty,
event_av_overflow, event_rx_full;
150 logic link_reset, link_suspend;
151 logic host_lost, link_disconnect, link_powered;
152 logic event_link_reset, event_link_suspend,
event_link_resume;
153 logic event_host_lost, event_disconnect,
event_powered;
154 logic event_rx_crc5_err, event_rx_crc16_err;
155 logic event_rx_crc_err, event_rx_pid_err;
156 logic event_rx_bitstuff_err;
157 logic event_in_err;
158 logic event_out_err;
159 logic event_frame, event_sof;
160 logic link_active;
161
162 // Diagnostic visibility of OUT-side exceptional events
163 logic event_ign_avsetup, event_drop_avout,
event_drop_rx, event_datatog_out;
164 // Diagnostic visibility of IN-side exceptional events
165 logic event_timeout_in, event_nak_in,
event_nodata_in;
166 // Interrupt to software reports both types of CRC error;
they are separated only for the
167 // purpose of diagnostic event counting.
168 assign event_rx_crc_err = event_rx_crc5_err |
event_rx_crc16_err;
169
170 logic [10:0] frame;
171 logic [2:0] link_state;
172 logic connect_en;
173 logic resume_link_active;
174
175 // Current state of OUT data toggles
176 logic [NEndpoints-1:0] out_data_toggle;
177 // Write strobe from register interface
178 logic out_datatog_we;
179 // Write data from register interface
180 logic [NEndpoints-1:0] out_datatog_status;
181 logic [NEndpoints-1:0] out_datatog_mask;
182
183 // Current state of IN data toggles
184 logic [NEndpoints-1:0] in_data_toggle;
185 // Write strobe from register interface
186 logic in_datatog_we;
187 // Write data from register interface
188 logic [NEndpoints-1:0] in_datatog_status;
189 logic [NEndpoints-1:0] in_datatog_mask;
190
191 ///////////////////////////////////////////////////
192 // USB RX after CDC & muxing //
193 ///////////////////////////////////////////////////
194 logic usb_rx_d;
195 logic usb_rx_dp;
196 logic usb_rx_dn;
197 ///////////////////////////////////////////////////
198 // USB TX after CDC & muxing //
199 ///////////////////////////////////////////////////

```

```

200 logic usb_tx_d;
201 logic usb_tx_se0;
202 logic usb_tx_dp;
203 logic usb_tx_dn;
204 logic usb_tx_oe;
205 ///////////////////////////////////////////////////
206 // USB control pins after CDC //
207 ///////////////////////////////////////////////////
208 logic usb_pwr_sense;
209 logic usb_pullup_en;
210 logic usb_dp_pullup_en;
211 logic usb_dn_pullup_en;
212
213 ///////////////////////////////////////////////////
214 // Microsecond timing reference //
215 ///////////////////////////////////////////////////
216 // us_tick ticks for one cycle every us, and it is based off a
free-running
217 // counter.
218 logic [5:0] ns_cnt;
219 logic us_tick;
220
221 assign us_tick = (ns_cnt == 6'd47);
222 always_ff @(posedge clk_i or negedge rst_n) begin
223     if (!rst_n) begin
224         ns_cnt <= '0;
225     end else begin
226         if (us_tick) begin
227             ns_cnt <= '0;
228         end else begin
229             ns_cnt <= ns_cnt + 1'b1;
230         end
231     end
232 end
233
234 ///////////////////////////////////////////////////
235 // Receive interface fifos //
236 ///////////////////////////////////////////////////
237
238 logic avsetup_fifo_rst;
239 logic avsetup_fifo_wready;
240 logic avout_fifo_rst;
241 logic avout_fifo_wready;
242 logic event_pkt_received;
243 logic avsetup_rvalid, avsetup_rready;
244 logic avout_rvalid, avout_rready;
245 logic rx_fifo_rst;
246 logic rx_wvalid, rx_wready;
247 logic rx_wready_setup, rx_wready_out;
248 logic rx_fifo_rvalid;
249 logic rx_fifo_re;
250
251 logic [AVFifoWidth - 1:0] avsetup_rdata;
252 logic [AVFifoWidth - 1:0] avout_rdata;
253 logic [RXFifoWidth - 1:0] rx_wdata, rx_rdata;
254
255 logic [NEndpoints-1:0] clear_rxenable_out;
256
257 // Software reset signals
258 assign avsetup_fifo_rst = reg2hw.fifo_ctrl.avsetup_rst.qe &
reg2hw.fifo_ctrl.avsetup_rst.q;
259 assign avout_fifo_rst = reg2hw.fifo_ctrl.avout_rst.qe &
reg2hw.fifo_ctrl.avout_rst.q;
260 assign rx_fifo_rst = reg2hw.fifo_ctrl.rx_rst.qe &
reg2hw.fifo_ctrl.rx_rst.q;
261
262 // Separate 'FIFO empty' interrupts for the OUT and SETUP
FIFOs because each interrupt cannot be
263 // cleared without writing a buffer into the FIFO
264 assign event_av_setup_empty = connect_en & ~avsetup_rvalid;
265 assign event_av_out_empty = connect_en & ~avout_rvalid;
266 // A single 'overflow' interrupt suffices since this indic-
ates a programming error
267 assign event_av_overflow = (reg2hw.avsetupbuffer.qe &
(~avsetup_fifo_wready))
268 | (reg2hw.avoutbuffer.qe &
(~avout_fifo_wready));
269 assign hw2reg.usbstat.rx_empty.d = connect_en &
~rx_fifo_rvalid;
270
271 // Available SETUP Buffer FIFO
272 prim_fifo_sync #(
273     .Width(AVFifoWidth),
274     .Pass(1'b0),
275     .Depth(AVSetupFifoDepth),
276     .OutputZeroIfEmpty(1'b0)
277 ) usbdev_avsetupfifo (
278     .clk_i,
279     .rst_ni (rst_n),
280     .clr_i (avsetup_fifo_rst),
281
282     .wvalid_i (reg2hw.avsetupbuffer.qe),
283     .wready_o (avsetup_fifo_wready),
284     .wdata_i (reg2hw.avsetupbuffer.q),
285
286     .rvalid_o (avsetup_rvalid),
287     .rready_i (avsetup_rready),
288     .rdata_o (avsetup_rdata),
289     .full_o (hw2reg.usbstat.av_setup_full.d),
290     .depth_o (hw2reg.usbstat.av_setup_depth.d),

```

```

291 .err_o    ()
292 );
293
294 // Available OUT Buffer FIFO
295 prim_fifo_sync #(
296   .Width(AVOutFifoWidth),
297   .Pass(1'b0),
298   .Depth(AVOutFifoDepth),
299   .OutputZeroIfEmpty(1'b0)
300 ) usbdev_avoutfifo (
301   .clk_i,
302   .rst_ni    (rst_n),
303   .clr_i     (avout_fifo_rst),
304
305   .wvalid_i  (reg2hw.avoutbuffer.qe),
306   .wready_o  (avout_fifo_wready),
307   .wdata_i   (reg2hw.avoutbuffer.q),
308
309   .rvalid_o  (avout_rvalid),
310   .rready_i  (avout_rready),
311   .rdata_o   (avout_rdata),
312   .full_o    (hw2reg.usbstat.av_out_full.d),
313   .depth_o   (hw2reg.usbstat.av_out_depth.d),
314   .err_o     ()
315 );
316
317 assign rx_fifo_re = reg2hw.rxfifo.ep.re |
reg2hw.rxfifo.setup.re |
318 reg2hw.rxfifo.size.re |
reg2hw.rxfifo.buffer.re;
319
320 // The number of used entries in the Received Buffer FIFO is
presented to the software
321 logic [RXFifoDepthW-1:0] rx_depth;
322 assign hw2reg.usbstat.rx_depth.d = rx_depth;
323
324 // We can always accept a SETUP packet if the Received Buf-
fer FIFO is not full...
325 assign rx_wready_setup = rx_wready;
326 // ... but regular OUT packets are not permitted to use the
final entry; still qualified
327 // with 'rx_wready' for when reset is asserted.
328 assign rx_wready_out = rx_wready & (rx_depth < RXFi-
foDepthW'(RXFifoDepth - 1));
329
330 // Received Buffer FIFO
331 prim_fifo_sync #(
332   .Width(RXFifoWidth),
333   .Pass(1'b0),
334   .Depth(RXFifoDepth),
335   .OutputZeroIfEmpty(1'b1)
336 ) usbdev_rxfifo (
337   .clk_i,
338   .rst_ni    (rst_n),
339   .clr_i     (rx_fifo_rst),
340
341   .wvalid_i  (rx_wvalid),
342   .wready_o  (rx_wready),
343   .wdata_i   (rx_wdata),
344
345   .rvalid_o  (rx_fifo_rvalid),
346   .rready_i  (rx_fifo_re),
347   .rdata_o   (rx_rdata),
348   .full_o    (event_rx_full),
349   .depth_o   (rx_depth),
350   .err_o     ()
351 );
352
353 assign hw2reg.rxfifo.ep.d = rx_rdata[16:13];
354 assign hw2reg.rxfifo.setup.d = rx_rdata[12];
355 assign hw2reg.rxfifo.size.d = rx_rdata[11:5];
356 assign hw2reg.rxfifo.buffer.d = rx_rdata[4:0];
357 assign event_pkt_received = rx_fifo_rvalid;
358
359 // The rxfifo register is hrw, but we just need the read en-
ables.
360 logic [16:0] unused_rxfifo_q;
361 assign unused_rxfifo_q = {reg2hw.rxfifo.ep.q,
reg2hw.rxfifo.setup.q,
362 reg2hw.rxfifo.size.q,
reg2hw.rxfifo.buffer.q};
363
364 ///////////////////////////////////////////////////////////////////
365 // IN (Transmit) interface config //
366 ///////////////////////////////////////////////////////////////////
367 logic [NBufWidth-1:0] in_buf [NEndpoints];
368 logic [SizeWidth:0] in_size [NEndpoints];
369 logic [3:0] in_endpoint;
370 logic in_endpoint_val;
371 logic [NEndpoints-1:0] in_rdy;
372 logic [NEndpoints-1:0] clear_rdybit, set_sentbit, up-
date_pend, set_sending;
373 logic setup_received, in_ep_xact_end;
374 logic [NEndpoints-1:0] ep_out_iso, ep_in_iso;
375 logic [NEndpoints-1:0] enable_out, enable_setup,
in_ep_stall, out_ep_stall;
376 logic [NEndpoints-1:0] ep_set_nak_on_out;
377 logic [NEndpoints-1:0] ep_in_enable, ep_out_enable;
378 logic [3:0] out_endpoint;
379 logic out_endpoint_val;

```

```

380 logic use_diff_rcvr, usb_diff_rx_ok;
381
382 // Endpoint enables
383 always_comb begin : proc_map_ep_enable
384   for (int i = 0; i < NEndpoints; i++) begin
385     ep_in_enable[i] = reg2hw.ep_in_enable[i].q;
386     ep_out_enable[i] = reg2hw.ep_out_enable[i].q;
387   end
388 end
389
390 // RX enables
391 logic [NEndpoints-1:0] rxenable_out;
392 always_comb begin : proc_map_rxenable
393   for (int i = 0; i < NEndpoints; i++) begin
394     enable_setup[i] = reg2hw.rxenable_setup[i].q;
395     enable_out[i] = rxenable_out[i];
396     ep_set_nak_on_out[i] = reg2hw.set_nak_out[i].q;
397   end
398 end
399
400 // STALL for both directions
401 always_comb begin : proc_map_stall
402   for (int i = 0; i < NEndpoints; i++) begin
403     in_ep_stall[i] = reg2hw.in_stall[i];
404     out_ep_stall[i] = reg2hw.out_stall[i];
405   end
406 end
407
408 always_comb begin : proc_map_iso
409   for (int i = 0; i < NEndpoints; i++) begin
410     ep_out_iso[i] = reg2hw.out_iso[i].q;
411     ep_in_iso[i] = reg2hw.in_iso[i].q;
412   end
413 end
414
415 always_comb begin : proc_map_buf_size
416   for (int i = 0; i < NEndpoints; i++) begin
417     in_buf[i] = reg2hw.configin[i].buffer.q;
418     in_size[i] = reg2hw.configin[i].size.q;
419   end
420 end
421
422 always_comb begin : proc_map_rdy_reg2hw
423   for (int i = 0; i < NEndpoints; i++) begin
424     in_rdy[i] = reg2hw.configin[i].rdy.q;
425   end
426 end
427
428 // Captured properties of current IN buffer, maintained
throughout packet collection as
429 // protection against change during packet retraction by FW.
430 logic [NBufWidth-1:0] in_buf_q, in_buf_d;
431 logic [SizeWidth:0] in_size_q, in_size_d;
432 logic in_xact_starting;
433 logic [3:0] in_xact_start_ep;
434
435 always_ff @(posedge clk_i or negedge rst_n) begin
436   if (!rst_n) begin
437     in_buf_q <= '0;
438     in_size_q <= '0;
439   end else begin
440     in_buf_q <= in_buf_d;
441     in_size_q <= in_size_d;
442   end
443 end
444 assign in_buf_d = in_xact_starting ? in_buf[in_x-
act_start_ep] : in_buf_q;
445 assign in_size_d = in_xact_starting ? in_s-
ize[in_xact_start_ep] : in_size_q;
446
447 // OUT data toggles are maintained with the packet engine
but may be set or
448 // cleared by software
449 assign out_datatog_we = reg2hw.out_data_toggle.status.qe &
reg2hw.out_data_toggle.mask.qe;
450 assign out_datatog_status = reg2hw.out_data_toggle.status.q;
451 assign out_datatog_mask = reg2hw.out_data_toggle.mask.q;
452 // Software may read them at any time
453 assign hw2reg.out_data_toggle.status.d = out_data_toggle;
454 assign hw2reg.out_data_toggle.mask.d = {NEndpoints{1'b0}};
455
456 // IN data toggles are maintained with the packet engine but
may be set or
457 // cleared by software
458 assign in_datatog_we = reg2hw.in_data_toggle.status.qe &
reg2hw.in_data_toggle.mask.qe;
459 assign in_datatog_status = reg2hw.in_data_toggle.status.q;
460 assign in_datatog_mask = reg2hw.in_data_toggle.mask.q;
461 // Software may read them at any time
462 assign hw2reg.in_data_toggle.status.d = in_data_toggle;
463 assign hw2reg.in_data_toggle.mask.d = {NEndpoints{1'b0}};
464
465 always_comb begin
466   set_sentbit = '0;
467   if (in_ep_xact_end && in_endpoint_val) begin
468     set_sentbit[in_endpoint] = 1'b1;
469   end
470 end
471
472 always_comb begin : proc_map_sent

```

```

473   for (int i = 0; i < NEndpoints; i++) begin
474       hw2reg.in_sent[i].de = set_sentbit[i];
475       hw2reg.in_sent[i].d = 1'b1;
476   end
477 end
478
479 // This must be held level for the interrupt, so no sent
480 packets are missed.
481 logic sent_event_pending;
482 always_comb begin
483     sent_event_pending = 1'b0;
484     for (int i = 0; i < NEndpoints; i++) begin
485         sent_event_pending |= reg2hw.in_sent[i].q;
486     end
487 end
488 // rxenable_out register must be implemented as 'hwext' so
489 that we can treat each endpoint
490 // individually and prevent sw/hw races from inadvertently
491 re-enabling an endpoint that was
492 // recently disabled by the hardware.
493 logic [NEndpoints-1:0] rxenable_out_we;
494 assign rxenable_out_we = {NEndpoints}{reg2hw.rxenable_out.out.qe &
495 reg2hw.rxenable_out.preserve.qe} &
496 ~reg2hw.rxenable_out.preserve.q;
497 always_ff @(posedge clk_i or negedge rst_ni) begin
498     if (!rst_ni) begin
499         rxenable_out <= 'b0;
500     end else begin
501         for (int unsigned i = 0; i < NEndpoints; i++) begin
502             if (rxenable_out_we[i]) begin
503                 // Software wins for backwards compatibility; an
504                 // earlier implementation employed the
505                 // module `prim_subreg_arb` which exhibits this be-
506                 // havior.
507                 rxenable_out[i] <= reg2hw.rxenable_out.out.q[i];
508             end else if (clear_rxenable_out[i]) begin
509                 rxenable_out[i] <= 1'b0;
510             end
511         end
512     end
513 end
514 // If so configured, for every received transaction on a
515 given endpoint, clear
516 // the rxenable_out bit. In this configuration, hardware de-
517 faults to NAKing
518 // any subsequent transaction, so software has time to de-
519 cide the next
520 // response.
521 always_comb begin
522     clear_rxenable_out = '0;
523     if (rx_wvalid && out_endpoint_val) begin
524         clear_rxenable_out[out_endpoint] =
525 ep_set_nak_on_out[out_endpoint];
526     end
527 end
528
529 always_comb begin
530     clear_rdybit = '0;
531     update_pend = '0;
532     if (event_link_reset) begin
533         clear_rdybit = {NEndpoints{1'b1}};
534         update_pend = {NEndpoints{1'b1}};
535     end else begin
536         if (setup_received & out_endpoint_val) begin
537             // Clear pending when a SETUP is received
538             clear_rdybit[out_endpoint] = 1'b1;
539             update_pend[out_endpoint] = 1'b1;
540         end else if (in_ep_xact_end & in_endpoint_val) begin
541             // Clear rdy and sending when an IN transmission was
542             // successful
543             clear_rdybit[in_endpoint] = 1'b1;
544         end
545     end
546 end
547
548 // IN transaction starting on any endpoint?
549 always_comb begin
550     set_sending = '0;
551     if (in_xact_starting) set_sending[in_xact_start_ep] =
552 1'b1;
553 end
554
555 // Clearing of rdy bit in response to successful IN packet
556 transmission or packet cancellation
557 // through link reset or SETUP packet reception.
558 always_comb begin : proc_map_rdy_hw2reg
559     for (int i = 0; i < NEndpoints; i++) begin
560         hw2reg.configin[i].rdy.de = clear_rdybit[i];
561         hw2reg.configin[i].rdy.d = 1'b0;
562     end
563 end
564
565 // Update the pending bit by copying the ready bit that is
566 about to clear
567 always_comb begin : proc_map_pend

```

```

559   for (int i = 0; i < NEndpoints; i++) begin
560       hw2reg.configin[i].pend.de = update_pend[i];
561       hw2reg.configin[i].pend.d = reg2hw.configin[i].rdy.q |
562 reg2hw.configin[i].pend.q;
563   end
564 end
565
566 // Update the sending bit to mark that collection of the
567 packet by the USB host has been
568 // attempted and FW shall not attempt retraction of the
569 packet.
570 always_comb begin : proc_map_sending
571     for (int i = 0; i < NEndpoints; i++) begin
572         hw2reg.configin[i].sending.de = set_sending[i] |
573 set_sentbit[i] | update_pend[i];
574         hw2reg.configin[i].sending.d = ~set_sentbit[i] & ~up-
575 date_pend[i];
576     end
577 end
578
579 // USB interface -- everything is in USB clock domain //
580 ////////////////////////////////////////////////////////////////////
581 logic cfg_pinflip;
582 assign cfg_pinflip = reg2hw.phy_config.pinflip.q;
583 assign usb_dp_pullup_en = cfg_pinflip ? 1'b0 : usb_pul-
584 lup_en;
585 assign usb_dn_pullup_en = !cfg_pinflip ? 1'b0 : usb_pul-
586 lup_en;
587
588 usbdev_usbif #(
589     .NEndpoints (NEndpoints),
590     .AVFifoWidth (AVFifoWidth),
591     .RXFifoWidth (RXFifoWidth),
592     .MaxPktSizeByte (MaxPktSizeByte),
593     .NBuf (NBuf),
594     .SramAw (SramAw)
595 ) usbdev_impl (
596     .clk_48mhz_i (clk_i),
597     .rst_ni (rst_n),
598
599     // Pins
600     .usb_d_i (usb_rx_d),
601     .usb_dp_i (usb_rx_dp),
602     .usb_dn_i (usb_rx_dn),
603     .usb_oe_o (usb_tx_oe),
604     .usb_d_o (usb_tx_d),
605     .usb_se0_o (usb_tx_se0),
606     .usb_dp_o (usb_tx_dp),
607     .usb_dn_o (usb_tx_dn),
608     .usb_sense_i (usb_pwr_sense),
609     .usb_pullup_en_o (usb_pullup_en),
610
611     // receive side
612     .rx_setup_i (enable_setup),
613     .rx_out_i (enable_out),
614     .rx_stall_i (out_ep_stall),
615     .avsetup_rvalid_i (avsetup_rvalid),
616     .avsetup_rready_o (avsetup_rready),
617     .avsetup_rdata_i (avsetup_rdata),
618     .avout_rvalid_i (avout_rvalid),
619     .avout_rready_o (avout_rready),
620     .avout_rdata_i (avout_rdata),
621
622     .rx_wvalid_o (rx_wvalid),
623     .rx_wready_setup_i (rx_wready_setup),
624     .rx_wready_out_i (rx_wready_out),
625     .rx_wdata_o (rx_wdata),
626     .setup_received_o (setup_received),
627     .out_endpoint_o (out_endpoint), // will be stable
628     .out_endpoint_val_o (out_endpoint_val),
629
630     // transmit side
631     .in_xact_starting_o (in_xact_starting),
632     .in_xact_start_ep_o (in_xact_start_ep),
633     .in_buf_i (in_buf_q),
634     .in_size_i (in_size_q),
635     .in_stall_i (in_ep_stall),
636     .in_rdy_i (in_rdy),
637     .in_ep_xact_end_o (in_ep_xact_end),
638     .in_endpoint_o (in_endpoint),
639     .in_endpoint_val_o (in_endpoint_val),
640
641     // memory
642     .mem_req_o (usb_mem_b_req),
643     .mem_write_o (usb_mem_b_write),
644     .mem_addr_o (usb_mem_b_addr),
645     .mem_wdata_o (usb_mem_b_wdata),
646     .mem_rdata_i (usb_mem_b_rdata),
647
648     // time reference
649     .us_tick_i (us_tick),
650
651     // control
652     .connect_en_i (connect_en),
653     .devaddr_i (reg2hw.usbctrl.device_address.q),
654     .clr_devaddr_o (clr_devaddr),
655     .in_ep_enabled_i (ep_in_enable),
656     .out_ep_enabled_i (ep_out_enable),

```

```

652 .out_ep_iso_i      (ep_out_iso),
653 .in_ep_iso_i       (ep_in_iso),
654 .diff_rx_ok_i      (usb_diff_rx_ok),
655 .cfg_eop_single_bit_i
(reg2hw.phy_config.eop_single_bit.q), // cdc ok: quasi-static
656 .tx_osc_test_mode_i
(reg2hw.phy_config.tx_osc_test_mode.q), // cdc ok: quasi-static
657 .cfg_use_diff_rcvr_i (usb_rx_enable_o),
658 .cfg_pinflip_i      (cfg_pinflip),
659 .out_data_toggle_o  (out_data_toggle),
660 .out_datatog_we_i   (out_datatog_we),
661 .out_datatog_status_i (out_datatog_status),
662 .out_datatog_mask_i (out_datatog_mask),
663 .in_data_toggle_o   (in_data_toggle),
664 .in_datatog_we_i    (in_datatog_we),
665 .in_datatog_status_i (in_datatog_status),
666 .in_datatog_mask_i  (in_datatog_mask),
667
668 .resume_link_active_i (resume_link_active),
669
670 // status
671 .frame_o              (frame),
672 .frame_start_o        (event_frame),
673 .sof_detected_o       (event_sof),
674 .link_state_o         (link_state),
675 .link_disconnect_o    (link_disconnect),
676 .link_powered_o       (link_powered),
677 .link_reset_o         (link_reset),
678 .link_active_o        (link_active),
679 .link_suspend_o       (link_suspend),
680 .link_resume_o        (event_link_resume),
681 .host_lost_o          (host_lost),
682 .link_in_err_o        (event_in_err),
683 .link_out_err_o       (event_out_err),
684 .rx_crc5_err_o        (event_rx_crc5_err),
685 .rx_crc16_err_o       (event_rx_crc16_err),
686 .rx_pid_err_o         (event_rx_pid_err),
687 .rx_bitstuff_err_o    (event_rx_bitstuff_err),
688
689 // event counters
690 .event_ign_avsetup_o  (event_ign_avsetup),
691 .event_drop_avout_o   (event_drop_avout),
692 .event_drop_rx_o      (event_drop_rx),
693 .event_datatog_out_o  (event_datatog_out),
694 .event_timeout_in_o   (event_timeout_in),
695 .event_nak_in_o       (event_nak_in),
696 .event_nodata_in_o    (event_nodata_in)
697 );
698
699 ///////////////////////////////////////////////////
700 // Control signal / status CDC //
701 ///////////////////////////////////////////////////
702
703 assign hw2reg.usbstat.link_state.d = link_state;
704 assign hw2reg.usbstat.frame.d = frame;
705
706 assign connect_en = reg2hw.usbctrl.enable.q;
707 assign resume_link_active =
reg2hw.usbctrl.resume_link_active.qe &
reg2hw.usbctrl.resume_link_active.q;
708
709 // Just want a pulse to ensure only one interrupt for an
event
710 prim_edge_detector #(
711   .Width(5),
712   .EnSync(1'b0)
713 ) gen_event (
714   .clk_i,
715   .rst_ni      (rst_n),
716   .d_i          ({link_disconnect, link_reset, link_suspend,
717   .host_lost, link_powered}),
718   .q_sync_o     (),
719   .q_posedge_pulse_o({event_disconnect, event_link_reset,
event_link_suspend,
720   .event_host_lost, event_powered}),
721   .q_negedge_pulse_o()
722 );
723
724
725 assign hw2reg.usbstat.host_lost.d = host_lost;
726
727 // resets etc cause the device address to clear
728 assign hw2reg.usbctrl.device_address.de = clr_devaddr;
729 assign hw2reg.usbctrl.device_address.d = '0;
730
731 // Clear the stall flag when a SETUP is received
732
733 always_comb begin : proc_stall_tieoff
734   for (int i = 0; i < NEndpoints; i++) begin
735     hw2reg.in_stall[i].d = 1'b0;
736     hw2reg.out_stall[i].d = 1'b0;
737     if (setup_received && out_endpoint_val && out_endpoint
== 4'(unsigned'(i))) begin
738       hw2reg.out_stall[i].de = 1'b1;
739       hw2reg.in_stall[i].de = 1'b1;
740     end else begin
741       hw2reg.out_stall[i].de = 1'b0;
742       hw2reg.in_stall[i].de = 1'b0;
743     end
744   end
end

```

```

745 end
746
747 if (Stub) begin : gen_stubbed_memory
748   // Stub this window off with an error responder if
stubbed.
749   tlul_err_resp u_tlul_err_resp (
750     .clk_i,
751     .rst_ni,
752     .tl_h_i(tl_sram_h2d),
753     .tl_h_o(tl_sram_d2h)
754   );
755
756   // Tie off unused signals
757   assign sw_mem_a_req = '0;
758   assign sw_mem_a_gnt = '0;
759   assign sw_mem_a_write = '0;
760   assign sw_mem_a_addr = '0;
761   assign sw_mem_a_wdata = '0;
762   assign sw_mem_a_rvalid = '0;
763   assign sw_mem_a_rdata = '0;
764   assign sw_mem_a_rerror = '0;
765
766   assign usb_mem_b_rdata = '0;
767   assign ram_cfg_o =
prim_ram_1p_pkg::RAM_1P_CFG_RSP_DEFAULT;
768
769   logic unused_usb_mem_b_sigs;
770   assign unused_usb_mem_b_sigs = ^{
771     ram_cfg_i,
772     usb_mem_b_req,
773     usb_mem_b_write,
774     usb_mem_b_addr,
775     usb_mem_b_wdata,
776     usb_mem_b_rdata
777   };
778   end else begin : gen_no_stubbed_memory
779     // TL-UL to SRAM adapter
780     tlul_adapter_sram #(
781       .SramAw(SramAw),
782       .ByteAccess(0)
783     ) u_tlul2sram (
784       .clk_i,
785       .rst_ni,
786
787       .tl_i              (tl_sram_h2d),
788       .tl_o              (tl_sram_d2h),
789       .en_ifetch_i       (prim_mubi_pkg::MuBi4False),
790       .req_o              (sw_mem_a_req),
791       .req_type_o        (),
792       .gnt_i              (sw_mem_a_gnt),
793       .we_o              (sw_mem_a_write),
794       .addr_o             (sw_mem_a_addr),
795       .wdata_o            (sw_mem_a_wdata),
796       .wmask_o            (), // Not used
797       .intg_error_o       (),
798       .user_rsvd_o        (),
799       .rdata_i            (sw_mem_a_rdata),
800       .rvalid_i           (sw_mem_a_rvalid),
801       .rerror_i           (sw_mem_a_rerror),
802       .compound_txn_in_progress_o (),
803       .readback_en_i      (prim_mubi_pkg::MuBi4False),
804       .readback_error_o   (),
805       .wr_collision_i     (1'b0),
806       .write_pending_i    (1'b0)
807     );
808
809     // Single Port RAM implementation, which will award the
`usb` port absolute priority and
810     // delay `sw` access by in the event of a collision.
811     //
812     // In practice the `usb` access to memory is sporadic (4x
oversampling of bits, and read/write
813     // operations transfer 32 bits so on average the probability
of collision is just 1/128 even
814     // during active USB traffic, if the TL-UL interface were
active on every cycle).
815
816     // usb access has absolute priority, followed by any deferred
write, and then any sw access.
817     logic mem_req;
818     logic mem_write;
819     logic [SramAw-1:0] mem_addr;
820     logic [SramDw-1:0] mem_wdata;
821     assign mem_req = usb_mem_b_req | sw_mem_a_req;
822     assign mem_write = usb_mem_b_req ? usb_mem_b_write :
sw_mem_a_write;
823     assign mem_addr = usb_mem_b_req ? usb_mem_b_addr :
sw_mem_a_addr;
824     assign mem_wdata = usb_mem_b_req ? usb_mem_b_wdata :
sw_mem_a_wdata;
825
826     logic mem_rvalid;
827     logic [SramDw-1:0] mem_rdata;
828     logic [1:0] mem_rerror;
829     logic mem_rsteering;
830
831     // Always grant when no `usb` request.
832     assign sw_mem_a_gnt = !usb_mem_b_req;
833
834     // `usb` relies upon its read data remaining static after
read.
end

```



```

835 logic mem_b_read_q;
836 logic [SramDw-1:0] mem_b_rdata_q;
837
838 // Remember granted read accesses.
839 // NOTE: No pipelining within the RAM model.
840 always_ff @(posedge clk_i or negedge rst_ni) begin
841   if (!rst_ni) begin
842     mem_rsteering <= 1'b0;
843     mem_b_read_q <= 1'b0;
844     mem_b_rdata_q <= {SramDw{1'b0}};
845   end else begin
846     mem_rsteering <= usb_mem_b_req;
847     mem_b_read_q <= usb_mem_b_req & !usb_mem_b_write;
848     // Capture the `usb` read data.
849     if (mem_b_read_q)
850       mem_b_rdata_q <= mem_rdata;
851   end
852 end
853
854 // Read responses.
855 assign sw_mem_a_rvalid = mem_rvalid & !mem_rsteering;
856 assign sw_mem_a_rerror = {2{sw_mem_a_rvalid}} & mem_rer-
ror;
857
858 // We may safely return the read data to both (no security
859 // implications), but `usb` rdata
860 // must be held static after the read, and be unaffected
861 // by `sw` reads.
862 assign sw_mem_a_rdata = mem_rdata;
863 assign usb_mem_b_rdata = mem_b_read_q ? mem_rdata :
mem_b_rdata_q;
864
865 // SRAM Wrapper
866 prim_ram_1p_adv #(
867   .Depth (SramDepth),
868   .Width (SramDw), // 32 x 512 --> 2kB
869   .DataBitsPerMask(8),
870   .EnableECC (0), // No Protection
871   .EnableParity (0),
872   .EnableInputPipeline (0),
873   .EnableOutputPipeline(0)
874 ) u_memory_1p (
875   .clk_i,
876   .rst_ni,
877   .req_i (mem_req),
878   .write_i (mem_write),
879   .addr_i (mem_addr),
880   .wdata_i (mem_wdata),
881   .wmask_i ({SramDw{1'b1}}),
882   .rdata_o (mem_rdata),
883   .rvalid_o (mem_rvalid),
884   .rerror_o (mem_rerror),
885   .cfg_i (ram_cfg_i),
886   .cfg_o (ram_cfg_o),
887   .alert_o ()
888 );
889 end : gen_no_stubbed_memory
890
891 logic [NumAlerts-1:0] alert_test, alerts;
892
893 // Register module
894 usbdev_reg_top u_reg (
895   .clk_i,
896   .rst_ni, // this reset is not stubbed off so that the re-
897   gisters are still accessible.
898   .clk_aon_i,
899   .rst_aon_ni, // this reset is not stubbed off so that the
900   registers are still accessible.
901   .tl_i (tl_i),
902   .tl_o (tl_o),
903   .tl_win_o (tl_sram_h2d),
904   .tl_win_i (tl_sram_d2h),
905   .reg2hw(reg2hw_regtop),
906   .hw2reg(hw2reg_regtop),
907   // SEC_CM: BUS_INTEGRITY
908   .intg_err_o (alerts[0])
909 );
910
911 // Stub off all register connections to reg_top.
912 if (Stub) begin : gen_stubbed
913   logic unused_sigs;
914   assign reg2hw = '0;
915   assign hw2reg_regtop = '0;
916   assign unused_sigs = ^{reg2hw_regtop, hw2reg};
917 end else begin : gen_not_stubbed
918   assign reg2hw = reg2hw_regtop;
919   assign hw2reg_regtop = hw2reg;
920 end
921
922 // Alerts
923 assign alert_test = {
924   reg2hw.alert_test.q &
925   reg2hw.alert_test.qe
926 };
927
928

```

```

929 // Alerts not stubbed off because registers and T-L access
930 // still present.
931 localparam logic [NumAlerts-1:0] AlertIsFatal = {Nu-
mAlerts{1'b1}};
932 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
933   prim_alert_sender #(
934     .AsyncOn(AlertAsyncOn[i]),
935     .SkewCycles(AlertSkewCycles),
936     .IsFatal(AlertIsFatal[i])
937   ) u_prim_alert_sender (
938     .clk_i,
939     .rst_ni, // this reset is not stubbed off so that the
940     pings still work.
941     .alert_test_i ( alert_test[i] ),
942     .alert_req_i ( alerts[i] ),
943     .alert_ack_o ( ),
944     .alert_state_o ( ),
945     .alert_rx_i ( alert_rx_i[i] ),
946     .alert_tx_o ( alert_tx_o[i] )
947   );
948 end
949
950 // Interrupts
951 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_pkt-re-
ceived (
952   .clk_i,
953   .rst_ni, // not stubbed off so that the interrupt regs
954   still work.
955   .event_intr_i (event_pkt_received),
956   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.pkt_received.q),
957   .reg2hw_intr_test_q_i (reg2hw_intr_test.pkt_received.q),
958   .reg2hw_intr_test_qe_i (reg2hw_intr_test.pkt_received.qe),
959   .reg2hw_intr_state_q_i (reg2hw_intr_state.pkt_received.q),
960   .hw2reg_intr_state_de_o (hw2reg_intr_state.pkt_received.de),
961   .hw2reg_intr_state_d_o (hw2reg_intr_state.pkt_received.d),
962   .intr_o (intr_pkt_received_o)
963 );
964
965 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_hw_pkt_sent (
966   .clk_i,
967   .rst_ni, // not stubbed off so that the interrupt regs
968   still work.
969   .event_intr_i (sent_event_pending),
970   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.pkt_sent.q),
971   .reg2hw_intr_test_q_i (reg2hw_intr_test.pkt_sent.q),
972   .reg2hw_intr_test_qe_i (reg2hw_intr_test.pkt_sent.qe),
973   .reg2hw_intr_state_q_i (reg2hw_intr_state.pkt_sent.q),
974   .hw2reg_intr_state_de_o (hw2reg_intr_state.pkt_sent.de),
975   .hw2reg_intr_state_d_o (hw2reg_intr_state.pkt_sent.d),
976   .intr_o (intr_pkt_sent_o)
977 );
978
979 prim_intr_hw #(.Width(1)) intr_disconnected (
980   .clk_i,
981   .rst_ni, // not stubbed off so that the interrupt regs
982   still work.
983   .event_intr_i (event_disconnect),
984   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.disconnected.q),
985   .reg2hw_intr_test_q_i (reg2hw_intr_test.disconnected.q),
986   .reg2hw_intr_test_qe_i (reg2hw_intr_test.disconnected.qe),
987   .reg2hw_intr_state_q_i (reg2hw_intr_state.disconnected.q),
988   .hw2reg_intr_state_de_o (hw2reg_intr_state.disconnected.de),
989   .hw2reg_intr_state_d_o (hw2reg_intr_state.disconnected.d),
990   .intr_o (intr_disconnected_o)
991 );
992
993 prim_intr_hw #(.Width(1)) intr_powered (
994   .clk_i,
995   .rst_ni, // not stubbed off so that the interrupt regs
996   still work.
997   .event_intr_i (event_powered),
998   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.powered.q),
999   .reg2hw_intr_test_q_i (reg2hw_intr_test.powered.q),
1000   .reg2hw_intr_test_qe_i (reg2hw_intr_test.powered.qe),
1001   .reg2hw_intr_state_q_i (reg2hw_intr_state.powered.q),
1002   .hw2reg_intr_state_de_o (hw2reg_intr_state.powered.de),
1003   .hw2reg_intr_state_d_o (hw2reg_intr_state.powered.d),
1004   .intr_o (intr_powered_o)
1005 );
1006
1007 prim_intr_hw #(.Width(1)) intr_host_lost (
1008   .clk_i,
1009   .rst_ni, // not stubbed off so that the interrupt regs
1010   still work.
1011   .event_intr_i (event_host_lost),
1012   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.host_lost.q),
1013   .reg2hw_intr_test_q_i (reg2hw_intr_test.host_lost.q),
1014   .reg2hw_intr_test_qe_i (reg2hw_intr_test.host_lost.qe),
1015   .reg2hw_intr_state_q_i (reg2hw_intr_state.host_lost.q),
1016   .hw2reg_intr_state_de_o (hw2reg_intr_state.host_lost.de),

```

```

1010 .hw2reg_intr_state_d_o (hw2reg.intr_state.host_lost.d),
1011 .intr_o (intr_host_lost_o)
1012 );
1013
1014 prim_intr_hw #(.Width(1)) intr_link_reset (
1015 .clk_i,
1016 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1017 .event_intr_i (event_link_reset),
1018 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_reset.q),
1019 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_reset.q),
1020 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_reset.qe),
1021 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_reset.q),
1022 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_reset.de),
1023 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_reset.d),
1024 .intr_o (intr_link_reset_o)
1025 );
1026
1027 prim_intr_hw #(.Width(1)) intr_link_suspend (
1028 .clk_i,
1029 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1030 .event_intr_i (event_link_suspend),
1031 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_suspend.q),
1032 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_suspend.q),
1033 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_suspend.qe),
1034 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_suspend.q),
1035 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_suspend.de),
1036 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_suspend.d),
1037 .intr_o (intr_link_suspend_o)
1038 );
1039
1040 prim_intr_hw #(.Width(1)) intr_link_resume (
1041 .clk_i,
1042 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1043 .event_intr_i (event_link_resume),
1044 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_resume.q),
1045 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_resume.q),
1046 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_resume.qe),
1047 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_resume.q),
1048 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_resume.de),
1049 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_resume.d),
1050 .intr_o (intr_link_resume_o)
1051 );
1052
1053 prim_intr_hw #(.Width(1), .IntrT("Status"))
intr_av_out_empty (
1054 .clk_i,
1055 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1056 .event_intr_i (event_av_out_empty),
1057 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.av_out_empty.q),
1058 .reg2hw_intr_test_q_i (reg2hw.intr_test.av_out_empty.q),
1059 .reg2hw_intr_test_qe_i (reg2hw.intr_test.av_out_empty.qe),
1060 .reg2hw_intr_state_q_i (reg2hw.intr_state.av_out_empty.q),
1061 .hw2reg_intr_state_de_o (hw2reg.intr_state.av_out_empty.de),
1062 .hw2reg_intr_state_d_o (hw2reg.intr_state.av_out_empty.d),
1063 .intr_o (intr_av_out_empty_o)
1064 );
1065
1066 prim_intr_hw #(.Width(1), .IntrT("Status")) intr_rx_full (
1067 .clk_i,
1068 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1069 .event_intr_i (event_rx_full),
1070 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_full.q),
1071 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_full.q),
1072 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_full.qe),
1073 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_full.q),
1074 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_full.de),
1075 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_full.d),
1076 .intr_o (intr_rx_full_o)
1077 );
1078
1079 prim_intr_hw #(.Width(1)) intr_av_overflow (
1080 .clk_i,
1081 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1082 .event_intr_i (event_av_overflow),
1083 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.av_overflow.q),
1084 .reg2hw_intr_test_q_i (reg2hw.intr_test.av_overflow.q),
1085 .reg2hw_intr_test_qe_i (reg2hw.intr_test.av_overflow.qe),
1086 .reg2hw_intr_state_q_i (reg2hw.intr_state.av_overflow.q),
1087 .hw2reg_intr_state_de_o (hw2reg.intr_state.av_overflow.de),
1088 .hw2reg_intr_state_d_o (hw2reg.intr_state.av_overflow.d),
1089 .intr_o (intr_av_overflow_o)
1090 );
1091
1092 prim_intr_hw #(.Width(1)) intr_link_in_err (
1093 .clk_i,
1094 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1095 .event_intr_i (event_in_err),
1096 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_in_err.q),
1097 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_in_err.q),
1098 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_in_err.qe),
1099 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_in_err.q),
1100 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_in_err.de),
1101 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_in_err.d),
1102 .intr_o (intr_link_in_err_o)
1103 );
1104
1105 prim_intr_hw #(.Width(1)) intr_link_out_err (
1106 .clk_i,
1107 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1108 .event_intr_i (event_out_err),
1109 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.link_out_err.q),
1110 .reg2hw_intr_test_q_i (reg2hw.intr_test.link_out_err.q),
1111 .reg2hw_intr_test_qe_i (reg2hw.intr_test.link_out_err.qe),
1112 .reg2hw_intr_state_q_i (reg2hw.intr_state.link_out_err.q),
1113 .hw2reg_intr_state_de_o (hw2reg.intr_state.link_out_err.de),
1114 .hw2reg_intr_state_d_o (hw2reg.intr_state.link_out_err.d),
1115 .intr_o (intr_link_out_err_o)
1116 );
1117
1118 prim_intr_hw #(.Width(1)) intr_rx_crc_err (
1119 .clk_i,
1120 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1121 .event_intr_i (event_rx_crc_err),
1122 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_crc_err.q),
1123 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_crc_err.q),
1124 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_crc_err.qe),
1125 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_crc_err.q),
1126 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_crc_err.de),
1127 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_crc_err.d),
1128 .intr_o (intr_rx_crc_err_o)
1129 );
1130
1131 prim_intr_hw #(.Width(1)) intr_rx_pid_err (
1132 .clk_i,
1133 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1134 .event_intr_i (event_rx_pid_err),
1135 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_pid_err.q),
1136 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_pid_err.q),
1137 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_pid_err.qe),
1138 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_pid_err.q),
1139 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_pid_err.de),
1140 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_pid_err.d),
1141 .intr_o (intr_rx_pid_err_o)
1142 );
1143
1144 prim_intr_hw #(.Width(1)) intr_rx_bitstuff_err (
1145 .clk_i,
1146 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1147 .event_intr_i (event_rx_bitstuff_err),
1148 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.rx_bitstuff_err.q),
1149 .reg2hw_intr_test_q_i (reg2hw.intr_test.rx_bitstuff_err.q),
1150 .reg2hw_intr_test_qe_i (reg2hw.intr_test.rx_bitstuff_err.qe),
1151 .reg2hw_intr_state_q_i (reg2hw.intr_state.rx_bitstuff_err.q),
1152 .hw2reg_intr_state_de_o (hw2reg.intr_state.rx_bitstuff_err.de),
1153 .hw2reg_intr_state_d_o (hw2reg.intr_state.rx_bitstuff_err.d),
1154 .intr_o (intr_rx_bitstuff_err_o)
1155 );
1156
1157 prim_intr_hw #(.Width(1)) intr_frame (
1158 .clk_i,
1159 .rst_ni, // not stubbed off so that the interrupt regs
still work.
1160 .event_intr_i (event_frame),
1161 .reg2hw_intr_enable_q_i (reg2hw.intr_enable.frame.q),
1162 .reg2hw_intr_test_q_i (reg2hw.intr_test.frame.q),
1163 .reg2hw_intr_test_qe_i (reg2hw.intr_test.frame.qe),
1164 .reg2hw_intr_state_q_i (reg2hw.intr_state.frame.q),
1165 .hw2reg_intr_state_de_o (hw2reg.intr_state.frame.de),
1166 .hw2reg_intr_state_d_o (hw2reg.intr_state.frame.d),
1167 .intr_o (intr_frame_o)
1168 );
1169
1170 prim_intr_hw #(.Width(1), .IntrT("Status"))
intr_av_setup_empty (

```



```

1171 .clk_i,
1172 .rst_n1, // not stubbed off so that the interrupt regs
still work.
1173 .event_intr_i      (event_av_setup_empty),
1174 .reg2hw_intr_enable_q_i
(reg2hw.intr_enable.av_setup_empty.q),
1175 .reg2hw_intr_test_q_i
(reg2hw.intr_test.av_setup_empty.q),
1176 .reg2hw_intr_test_qe_i
(reg2hw.intr_test.av_setup_empty.qe),
1177 .reg2hw_intr_state_q_i
(reg2hw.intr_state.av_setup_empty.q),
1178 .hw2reg_intr_state_de_o
(hw2reg.intr_state.av_setup_empty.de),
1179 .hw2reg_intr_state_d_o
(hw2reg.intr_state.av_setup_empty.d),
1180 .intr_o      (intr_av_setup_empty_o)
1181 );
1182
1183 ///////////////////////////////////////////////////
1184 // USB IO Muxing //
1185 ///////////////////////////////////////////////////
1186 logic cio_usb_oe;
1187 logic usb_rx_enable;
1188 assign cio_usb_dp_en_o = cio_usb_oe;
1189 assign cio_usb_dn_en_o = cio_usb_oe;
1190 assign usb_tx_use_d_se0_o =
reg2hw.phy_config.tx_use_d_se0.q; // cdc ok: quasi-static
assign hw2reg.usbstat.sense.d = usb_pwr_sense;
1191
1192 usbdev_iomux i_usbdev_iomux (
1193     .clk_i,
1194     .rst_n1      (rst_n),
1195
1196     // Register interface
1197     .hw2reg_sense_o      (hw2reg.phy_pins_sense),
1198     .reg2hw_drive_i      (reg2hw.phy_pins_drive),
1199
1200     // Chip IO
1201     .usb_rx_d_i      (usb_rx_d_i),
1202     .usb_rx_dp_i      (cio_usb_dp_i),
1203     .usb_rx_dn_i      (cio_usb_dn_i),
1204     .cio_usb_sense_i      (cio_sense_i),
1205     .usb_tx_d_o      (usb_tx_d_o),
1206     .usb_tx_se0_o      (usb_tx_se0_o),
1207     .usb_tx_dp_o      (cio_usb_dp_o),
1208     .usb_tx_dn_o      (cio_usb_dn_o),
1209     .usb_tx_oe_o      (cio_usb_oe),
1210     .usb_dp_pullup_en_o      (usb_dp_pullup_o),
1211     .usb_dn_pullup_en_o      (usb_dn_pullup_o),
1212     .usb_rx_enable_o      (usb_rx_enable_o),
1213
1214     // Internal interface
1215     .usb_rx_d_o      (usb_rx_d),
1216     .usb_rx_dp_o      (usb_rx_dp),
1217     .usb_rx_dn_o      (usb_rx_dn),
1218     .usb_tx_d_i      (usb_tx_d),
1219     .usb_tx_se0_i      (usb_tx_se0),
1220     .usb_tx_dp_i      (usb_tx_dp),
1221     .usb_tx_dn_i      (usb_tx_dn),
1222     .usb_tx_oe_i      (usb_tx_oe),
1223     .usb_pwr_sense_o      (usb_pwr_sense),
1224     .usb_dp_pullup_en_i      (usb_dp_pullup_en),
1225     .usb_dn_pullup_en_i      (usb_dn_pullup_en),
1226     .usb_rx_enable_i      (usb_rx_enable)
1227 );
1228
1229 // Differential receiver enable
1230 assign use_diff_rcvr = reg2hw.phy_config.use_diff_rcvr.q;
1231
1232 // enable rx only when the single-ended input is enabled and
the device is
1233 // not suspended (unless it is forced on in the I/O mux).
1234 assign usb_rx_enable = use_diff_rcvr & ~link_suspend;
1235
1236 // Symbols from the differential receiver are invalid until
it has finished
1237 // waking up / powering on
1238
1239 // Add 1 to the specified time to account for uncertainty in
the
1240 // free-running counter for us_tick.
1241 localparam int RcvrWakeTimeWidth = vbits(RcvrWakeTimeUs +
2);
1242 logic [RcvrWakeTimeWidth-1:0] usb_rcvr_ok_counter_d, usb_r-
cvr_ok_counter_q;
1243
1244 assign usb_diff_rx_ok = (usb_rcvr_ok_counter_q == '0);
1245 always_comb begin
1246     // When don't need to use a differential receiver, RX is
always ready
1247     usb_rcvr_ok_counter_d = '0;
1248     if (use_diff_rcvr & !usb_rx_enable_o) begin
1249         usb_rcvr_ok_counter_d = RcvrWake-
TimeWidth'(RcvrWakeTimeUs + 1);
1250     end else if (us_tick && (usb_rcvr_ok_counter_q > '0)) be-
gin
1251         usb_rcvr_ok_counter_d = usb_rcvr_ok_counter_q - 1;
1252     end
1253
1254 always_ff @(posedge clk_i or negedge rst_n) begin
1255     if (!rst_n) begin
1256         usb_rcvr_ok_counter_q <= RcvrWake-
TimeWidth'(RcvrWakeTimeUs + 1);
1257     end else begin
1258         usb_rcvr_ok_counter_q <= usb_rcvr_ok_counter_d;
1259     end
1260 end
1261
1262 ///////////////////////////////////////////////////
1263 // SOF Reference for Clock Calibration //
1264 ///////////////////////////////////////////////////
1265
1266 logic usb_ref_val_d, usb_ref_val_q;
1267 logic usb_ref_disable;
1268 assign usb_ref_disable =
reg2hw.phy_config.usb_ref_disable.q;
1269
1270 // Directly forward the pulse unless disabled.
1271 assign usb_ref_pulse_o = usb_ref_disable ? 1'b0 : event_sof;
1272
1273 // The first pulse is always ignored, but causes the valid
to be asserted.
1274 // The valid signal is deasserted when:
1275 // - The link is no longer active.
1276 // - The host is lost (no SOF for 4ms).
1277 // - The reference generation is disabled.
1278 assign usb_ref_val_d =
usb_ref_pulse_o ? 1'b1 :
(!link_active || host_lost || usb_ref_disable) ? 1'b0 :
usb_ref_val_q;
1279
1280 always_ff @(posedge clk_i or negedge rst_n) begin
1281     if (!rst_n) begin
1282         usb_ref_val_q <= 1'b0;
1283     end else begin
1284         usb_ref_val_q <= usb_ref_val_d;
1285     end
1286 end
1287
1288 assign usb_ref_val_o = usb_ref_val_q;
1289
1290 ///////////////////////////////////////////////////
1291 // USB aon detector signaling //
1292 ///////////////////////////////////////////////////
1293 assign usb_aon_suspend_req_o =
reg2hw.wake_control.suspend_req.qe &
1294 reg2hw.wake_control.suspend_req.q;
1295
1296 assign usb_aon_wake_ack_o = reg2hw.wake_control.wake_ack.qe
&
1297 reg2hw.wake_control.wake_ack.q;
1298
1299 ///////////////////////////////////////////////////
1300 // capture async event and debug info //
1301 ///////////////////////////////////////////////////
1302
1303 assign hw2reg.wake_events.module_active.de = 1'b1;
1304 assign hw2reg.wake_events.module_active.d =
usb_aon_wake_detect_active_i;
1305 assign hw2reg.wake_events.bus_not_idle.de = 1'b1;
1306 assign hw2reg.wake_events.bus_not_idle.d =
usb_aon_bus_not_idle_i;
1307 assign hw2reg.wake_events.disconnected.de = 1'b1;
1308 assign hw2reg.wake_events.disconnected.d =
usb_aon_sense_lost_i;
1309 assign hw2reg.wake_events.bus_reset.de = 1'b1;
1310 assign hw2reg.wake_events.bus_reset.d = usb_aon_bus_reset_i;
1311
1312 ///////////////////////////////////////////////////
1313 // Diagnostic/performance counters //
1314 ///////////////////////////////////////////////////
1315
1316 // SW write strobes for the event enables of the counters.
1317 logic ctr_out_ev_qe;
1318 logic ctr_in_ev_qe;
1319 logic ctr_errors_ev_qe;
1320 assign ctr_out_ev_qe = &{reg2hw.count_out.ign_avsetup.qe,
reg2hw.count_out.drop_avout.qe,
reg2hw.count_out.drop_rx.qe,
reg2hw.count_out.datatog_out.qe};
1321
1322 assign ctr_in_ev_qe = &{reg2hw.count_in.timeout.qe,
reg2hw.count_in.nak.qe,
reg2hw.count_in.nodata.qe};
1323
1324 assign ctr_errors_ev_qe = &{reg2hw.count_errors.crc5.qe,
reg2hw.count_errors.crc16.qe,
reg2hw.count_errors.bitstuff.qe,
reg2hw.count_errors.pid_invalid.qe};
1325
1326 // Counters use 'rst_n' and remain at zero in Stubbed imple-
mentation
1327 usbdev_counter #(NEndpoints(NEndpoints), .NEvents(4))
u_ctr_out(
1328     .clk_i      (clk_i),
1329     .rst_n1      (rst_n),
1330     .reset_i      (reg2hw.count_out.rst.qe &
reg2hw.count_out.rst.q),
1331     .event_i      ({event_ign_avsetup,
event_drop_avout,
event_drop_rx,

```

```

1340         event_datatog_out}},
1341     .ep_i         (out_endpoint),
1342     // Set of events being counted.
1343     .ev_qe_i      (ctr_out_ev_qe),
1344     .ev_i         ({reg2hw.count_out.ign_avsetup.q,
1345                    reg2hw.count_out.drop_avout.q,
1346                    reg2hw.count_out.drop_rx.q,
1347                    reg2hw.count_out.datatog_out.q}),
1348     .ev_o         ({hw2reg.count_out.ign_avsetup.d,
1349                    hw2reg.count_out.drop_avout.d,
1350                    hw2reg.count_out.drop_rx.d,
1351                    hw2reg.count_out.datatog_out.d}),
1352     // Endpoints being monitored.
1353     .endp_qe_i    (reg2hw.count_out.endpoints.qe),
1354     .endpoints_i  (reg2hw.count_out.endpoints.q),
1355     .endpoints_o  (hw2reg.count_out.endpoints.d),
1356     .count_o      (hw2reg.count_out.count.d)
1357 );
1358
1359 usbdev_counter #(.NEndpoints(NEndpoints), .NEvents(3))
u_ctr_in(
1360     .clk_i        (clk_i),
1361     .rst_ni       (rst_n),
1362     .reset_i      (reg2hw.count_in.rst.qe &
reg2hw.count_in.rst.q),
1363     .event_i      ({event_timeout_in,
1364                    event_nak_in,
1365                    event_nodata_in}),
1366     .ep_i         (in_endpoint),
1367     // Set of events being counted.
1368     .ev_qe_i      (ctr_in_ev_qe),
1369     .ev_i         ({reg2hw.count_in.timeout.q,
1370                    reg2hw.count_in.nak.q,
1371                    reg2hw.count_in.nodata.q}),
1372     .ev_o         ({hw2reg.count_in.timeout.d,
1373                    hw2reg.count_in.nak.d,
1374                    hw2reg.count_in.nodata.d}),
1375     // Endpoints being monitored.
1376     .endp_qe_i    (reg2hw.count_in.endpoints.qe),
1377     .endpoints_i  (reg2hw.count_in.endpoints.q),
1378     .endpoints_o  (hw2reg.count_in.endpoints.d),
1379     .count_o      (hw2reg.count_in.count.d)
1380 );
1381
1382 usbdev_counter #(.NEndpoints(NEndpoints), .NEvents(1))
u_ctr_nodata_in(
1383     .clk_i        (clk_i),
1384     .rst_ni       (rst_n),
1385     .reset_i      (reg2hw.count_nodata_in.rst.qe &
reg2hw.count_nodata_in.rst.q),
1386     .event_i      (event_nodata_in),
1387     .ep_i         (in_endpoint),
1388     // Single event for this counter, so enables not required.
1389     .ev_qe_i      (1'b1),
1390     .ev_i         (1'b1),
1391     .ev_o         (), // not used
1392     // Endpoints being monitored.
1393     .endp_qe_i    (reg2hw.count_nodata_in.endpoints.qe),
1394     .endpoints_i  (reg2hw.count_nodata_in.endpoints.q),
1395     .endpoints_o  (hw2reg.count_nodata_in.endpoints.d),
1396     .count_o      (hw2reg.count_nodata_in.count.d)
1397 );
1398
1399 usbdev_counter #(.NEndpoints(1), .NEvents(4)) u_ctr_errors(
1400     .clk_i        (clk_i),
1401     .rst_ni       (rst_n),
1402     .reset_i      (reg2hw.count_errors.rst.qe &
reg2hw.count_errors.rst.q),
1403     .event_i      ({event_rx_crc5_err,
1404                    event_rx_crc16_err,
1405                    event_rx_bitstuff_err,
1406                    event_rx_pid_err}),
1407     .ep_i         (1'b0),
1408     // Set of events being counted.
1409     .ev_qe_i      (ctr_errors_ev_qe),
1410     .ev_i         ({reg2hw.count_errors.crc5.q,
1411                    reg2hw.count_errors.crc16.q,
1412                    reg2hw.count_errors.bitstuff.q,
1413                    reg2hw.count_errors.pid_invalid.q}),
1414     .ev_o         ({hw2reg.count_errors.crc5.d,
1415                    hw2reg.count_errors.crc16.d,
1416                    hw2reg.count_errors.bitstuff.d,
1417                    hw2reg.count_errors.pid_invalid.d}),
1418     // This events are not reliably associated with specific
endpoints, so no endpoint enables.
1419     .endp_qe_i    (1'b1),
1420     .endpoints_i  (1'b1),
1421     .endpoints_o  (), // not used
1422     .count_o      (hw2reg.count_errors.count.d)
1423 );
1424
1425 ///////////////////////////////////////////////////////////////////
1426 // Xprop assertions on outputs //
1427 ///////////////////////////////////////////////////////////////////
1428
1429 `ASSERT_KNOWN(T10DValidKnown_A, tl_o.d_valid)
1430 `ASSERT_KNOWN(T10AReadyKnown_A, tl_o.a_ready)
1431 // These pins are not necessarily associated with any clock
but it probably makes most sense to
1432 // check them on the fastest clock.
1433 `ASSERT_KNOWN(USBTxDKnown_A, usb_tx_d_o)

```

```

1434 `ASSERT_KNOWN(C10DpKnown_A, cio_usb_dp_o)
1435 `ASSERT_KNOWN(C10DpEnKnown_A, cio_usb_dp_en_o)
1436 `ASSERT_KNOWN(C10DnKnown_A, cio_usb_dn_o)
1437 `ASSERT_KNOWN(C10DnEnKnown_A, cio_usb_dn_en_o)
1438 `ASSERT_KNOWN(USBTxSe0Known_A, usb_tx_se0_o)
1439 `ASSERT_KNOWN(USBDpPUKnown_A, usb_dp_pullup_o)
1440 `ASSERT_KNOWN(USBDnPUKnown_A, usb_dn_pullup_o)
1441 `ASSERT_KNOWN(USBRxEnableKnown_A, usb_rx_enable_o)
1442 `ASSERT_KNOWN(USBAonSuspendReqKnown_A, usb_aon_suspend_req_o)
1443 `ASSERT_KNOWN(USBAonWakeAckKnown_A, usb_aon_wake_ack_o)
1444 `ASSERT_KNOWN(USBRefValKnown_A, usb_ref_val_o, clk_i, !
rst_ni)
1445 `ASSERT_KNOWN(USBRefPulseKnown_A, usb_ref_pulse_o, clk_i, !
rst_ni)
1446 // Assert Known for alerts
1447 `ASSERT_KNOWN(AlertsKnown_A, alert_tx_o)
1448 //Interrupt signals
1449 `ASSERT_KNOWN(USBIntrPktRcvdKnown_A, intr_pkt_received_o)
1450 `ASSERT_KNOWN(USBIntrPktSentKnown_A, intr_pkt_sent_o)
1451 `ASSERT_KNOWN(USBIntrPwrKnown_A, intr_powered_o)
1452 `ASSERT_KNOWN(USBIntrDisConKnown_A, intr_disconnected_o)
1453 `ASSERT_KNOWN(USBIntrHostLostKnown_A, intr_host_lost_o)
1454 `ASSERT_KNOWN(USBIntrLinkRstKnown_A, intr_link_reset_o)
1455 `ASSERT_KNOWN(USBIntrLinkSusKnown_A, intr_link_suspend_o)
1456 `ASSERT_KNOWN(USBIntrLinkResKnown_A, intr_link_resume_o)
1457 `ASSERT_KNOWN(USBIntrAvOutEmptyKnown_A, intr_av_out_empty_o)
1458 `ASSERT_KNOWN(USBIntrRxFullKnown_A, intr_rx_full_o)
1459 `ASSERT_KNOWN(USBIntrAvOverflowKnown_A, intr_av_overflow_o)
1460 `ASSERT_KNOWN(USBIntrLinkInErrKnown_A, intr_link_in_err_o)
1461 `ASSERT_KNOWN(USBIntrLinkOutErrKnown_A, intr_link_out_err_o)
1462 `ASSERT_KNOWN(USBIntrRxCrCErrKnown_A, intr_rx_crc_err_o)
1463 `ASSERT_KNOWN(USBIntrRxPidErrKnown_A, intr_rx_pid_err_o)
1464 `ASSERT_KNOWN(USBIntrRxBitstuffErrKnown_A, intr_rx_bitstuff_err_o)
1465 `ASSERT_KNOWN(USBIntrFrameKnown_A, intr_frame_o)
1466 `ASSERT_KNOWN(USBIntrAvSetupEmptyKnown_A,
intr_av_setup_empty_o)
1467
1468 // Alert assertions for reg_we onehot check
1469 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
1470 endmodule

```

9. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_aon_wake.sv (206 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Always On USB wake detect
6 //
7
8 `include "prim_assert.sv"
9
10 module usbdev_aon_wake import usbdev_pkg::*;
11     input logic clk_aon_i,
12     input logic rst_aon_ni,
13
14     // These come from the chip pin
15     input logic usb_dp_i,
16     input logic usb_dn_i,
17     input logic usb_sense_i,
18
19     // These come from the IP
20     input logic usbdev_dppullup_en_i,
21     input logic usbdev_dnpullup_en_i,
22
23     // Register signals from the IP, which must already be syn-
chronized to the AON domain.
24     input logic wake_ack_aon_i,
25     input logic suspend_req_aon_i,
26
27     // The I/Os that need to be maintained in low-power mode
28     output logic usb_dppullup_en_o,
29     output logic usb_dnpullup_en_o,
30
31     // wake/powerup request
32     output logic wake_req_aon_o,
33
34     // Event signals that indicate what happened while monitor-
ing
35     output logic bus_not_idle_aon_o,
36     output logic bus_reset_aon_o,
37     output logic sense_lost_aon_o,
38
39     // state debug information
40     output logic wake_detect_active_aon_o
41 );
42
43 // Whether this AON wake detector is active and controlling
the pull-ups.
44     logic wake_detect_active_d, wake_detect_active_q;
45
46 // Detect whether the USB has moved from an idle state.
47     logic not_idle_async;

```

```

48 logic event_not_idle;
49
50 // In suspend it is the device pullup that sets the line
state
51 // so if the input value differs then the host is doing
something
52 // This covers both host generated wake (J->K) and host gen-
erated reset (J->SE0)
53 // Use of the pullups takes care of pinflipping
54 assign not_idle_async = (usb_dp_i != usb_dppullup_en_o) |
55 (usb_dn_i != usb_dnpullup_en_o);
56
57 // aon clock is ~200kHz so 4 cycle filter is about 20us
58 // as well as noise debounce this gives the main IP time to
detect resume if it didn't turn off
59 prim_filter #(
60 .AsyncOn(1), // Instantiate 2-stage synchronizer
61 .Cycles(4)
62 ) filter_activity (
63 .clk_i (clk_aon_i),
64 .rst_ni (rst_aon_ni),
65 .enable_i (1'b1),
66 .filter_i (not_idle_async),
67 .filter_o (event_not_idle)
68 );
69
70 // Detect bus reset and VBUS removal events.
71 // Hold the detectors in reset when this module is in the
idle state, to
72 // avoid sampling / hysteresis issues that carry over when
the link is
73 // active.
74 logic se0_async, sense_lost_async;
75 logic event_bus_reset, event_sense_lost;
76 logic bus_not_idle_d, bus_not_idle_q;
77 logic bus_reset_d, bus_reset_q;
78 logic sense_lost_d, sense_lost_q;
79
80 assign se0_async = ~usb_dp_i & ~usb_dn_i;
81 assign sense_lost_async = ~usb_sense_i;
82
83 prim_filter #(
84 .AsyncOn(1),
85 .Cycles(3)
86 ) filter_bus_reset (
87 .clk_i (clk_aon_i),
88 .rst_ni (rst_aon_ni),
89 .enable_i (1'b1),
90 .filter_i (se0_async),
91 .filter_o (event_bus_reset)
92 );
93
94 prim_filter #(
95 .AsyncOn(1),
96 .Cycles(3)
97 ) filter_sense (
98 .clk_i (clk_aon_i),
99 .rst_ni (rst_aon_ni),
100 .enable_i (1'b1),
101 .filter_i (sense_lost_async),
102 .filter_o (event_sense_lost)
103 );
104
105 // USB has become non-idle while monitoring.
106 assign bus_not_idle_d = (event_not_idle | bus_not_idle_q) &
wake_detect_active_q;
107 // USB has issued a Bus Reset condition while monitoring.
108 assign bus_reset_d = (event_bus_reset | bus_reset_q) &
wake_detect_active_q;
109 // USB SENSE signal has been lost (Disconnection) while mon-
itoring.
110 assign sense_lost_d = (event_sense_lost | sense_lost_q) &
wake_detect_active_q;
111
112 // Detected events
113 assign bus_not_idle_aon_o = bus_not_idle_q;
114 assign bus_reset_aon_o = bus_reset_q;
115 assign sense_lost_aon_o = sense_lost_q;
116
117 logic wake_req_d, wake_req_q;
118 assign wake_req_d = wake_detect_active_q &
(event_not_idle | event_bus_reset |
event_sense_lost | wake_req_q);
119 assign wake_req_aon_o = wake_req_q;
120
121
122 always_ff @(posedge clk_aon_i or negedge rst_aon_ni) begin :
proc_reg_events
123 if (!rst_aon_ni) begin
124 bus_not_idle_q <= 1'b0;
125 bus_reset_q <= 1'b0;
126 sense_lost_q <= 1'b0;
127 wake_req_q <= 1'b0;
128 end else begin
129 bus_not_idle_q <= bus_not_idle_d;
130 bus_reset_q <= bus_reset_d;
131 sense_lost_q <= sense_lost_d;
132 wake_req_q <= wake_req_d;
133 end
134 end
135

```

```

136 always_comb begin : proc_awk_fsm
137 wake_detect_active_d = wake_detect_active_q;
138
139 unique case (wake_detect_active_q)
140 // No aon suspend entry has been requested or detected
141 1'b0: begin
142 if (suspend_req_aon_i) begin
143 wake_detect_active_d = 1'b1;
144 end
145 end
146
147 // The USB IP has passed control to this module for
handling the suspend
148 // request. wake_ack_i must be asserted to bring control
back to the USB
149 // IP.
150 1'b1: begin
151 if (wake_ack_aon_i) begin
152 wake_detect_active_d = 1'b0;
153 end
154 end
155 default : wake_detect_active_d = 1'b0;
156 endcase
157 end
158
159 `ASSERT_KNOWN(WakeDetectActiveAonKnown_A, wake_de-
tect_active_aon_o,
160 clk_aon_i, !rst_aon_ni)
161
162 always_ff @(posedge clk_aon_i or negedge rst_aon_ni) begin :
proc_reg_awk
163 if (!rst_aon_ni) begin
164 wake_detect_active_q <= 1'b0;
165 end else begin
166 wake_detect_active_q <= wake_detect_active_d;
167 end
168 end
169
170 assign wake_detect_active_aon_o = wake_detect_active_q;
171
172 // Control the pullup enable outputs from the AON module
when it's active
173 logic usbdev_dppullup_en_aon, usbdev_dnpullup_en_aon;
174 logic aon_dppullup_en_d, aon_dppullup_en_q;
175 logic aon_dnpullup_en_d, aon_dnpullup_en_q;
176
177 prim_flop_2sync #(
178 .Width(2)
179 ) u_pullup_en_cdc (
180 .clk_i(clk_aon_i),
181 .rst_ni(rst_aon_ni),
182 .d_i({usbdev_dppullup_en_i, usbdev_dnpullup_en_i}),
183 .q_o({usbdev_dppullup_en_aon, usbdev_dnpullup_en_aon})
184 );
185
186 assign aon_dppullup_en_d = wake_detect_active_q ? aon_dppul-
lup_en_q
187 : usb-
dev_dppullup_en_aon;
188 assign aon_dnpullup_en_d = wake_detect_active_q ? aon_dnpul-
lup_en_q
189 : usb-
dev_dnpullup_en_aon;
190
191 always_ff @(posedge clk_aon_i or negedge rst_aon_ni) begin :
proc_reg_pullup_en
192 if (!rst_aon_ni) begin
193 aon_dppullup_en_q <= 1'b0;
194 aon_dnpullup_en_q <= 1'b0;
195 end else begin
196 aon_dppullup_en_q <= aon_dppullup_en_d;
197 aon_dnpullup_en_q <= aon_dnpullup_en_d;
198 end
199 end
200
201 assign usb_dppullup_en_o = wake_detect_active_q ? aon_dppul-
lup_en_q
202 : usb-
dev_dppullup_en_i;
203 assign usb_dnpullup_en_o = wake_detect_active_q ? aon_dnpul-
lup_en_q
204 : usb-
dev_dnpullup_en_i;
205
206 endmodule

```

10. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_counter.sv

(88 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // USB device event counter
6 //

```

```

7 // At every rising edge within 'event_i' the counter samples
the current endpoint as indicated by

8 // 'ep_i' and increments the count if the endpoint in question is
enabled at that moment.
9 //
10 // This means the set of endpoints for which events are coun-
ted may be changed over time by
11 // software, whilst the USB device is active, multiplexing a
single counter amongst multiple
12 // sets of endpoints.
13 //

14 // The counter value saturates at its maximum and may be reset by
software at any point.
15
16 module usbdev_counter
17 #(
18     parameter int unsigned NEndpoints = 12, // Number of end-
points supported.
19     parameter int unsigned NEvents = 1, // Number of events be-
ing counted.
20     parameter int unsigned Width = 8, // Counter width, in
bits.
21     localparam int unsigned EpW =
prim_util_pkg::vbits(NEndpoints) // derived parameter
22 ) (
23     input logic clk_i,
24     input logic rst_ni,
25     input logic reset_i, // Software reset
26     input logic [NEvents-1:0] event_i, // Event pulse
27     input logic [EpW-1:0] ep_i, // Endpoint on which
event occurred.
28     // Set of events being counted.
29     input logic ev_qe_i, // SW write to the
event enables.
30     input logic [NEvents-1:0] ev_i, // Per-event enable/dis-
able.
31     output logic [NEvents-1:0] ev_o, // Current event en-
ables.
32     // Endpoints being monitored.
33     input logic endp_qe_i, // SW write to the
endpoint enables.
34     input logic [NEndpoints-1:0] endpoints_i, // Per-endpoint
enable/disable.
35     output logic [NEndpoints-1:0] endpoints_o, // Current end-
point enables.
36     output logic [Width-1:0] count_o // Current value of
counter.
37 );
38
39 // Current per-endpoint and per-event enables.
40 logic [NEndpoints-1:0] endpoints;
41 logic [NEvents-1:0] ev_enables;
42 always_ff @(posedge clk_i or negedge rst_ni) begin
43     if (!rst_ni) begin
44         endpoints <= NEndpoints'(0);
45         ev_enables <= NEvents'(0);
46     end else begin
47         // Software writes to set the current enables.
48         if (endp_qe_i) endpoints <= endpoints_i;
49         if (ev_qe_i) ev_enables <= ev_i;
50     end
51 end
52
53 // Respond to events on this endpoint?
54 logic ep_enabled;
55 if (NEndpoints > 1) begin : gen_multi
56     assign ep_enabled = (ep_i < EpW'(NEndpoints)) ? end-
points[ep_i] : 1'b0;
57 end else begin : gen_single
58     logic unused_ep;
59     assign unused_ep = ^ep_i; // Endpoint number not required
60     assign ep_enabled = endpoints[0];
61 end
62
63 // Saturating event counter.
64 logic [Width-1:0] count;
65 logic [NEvents-1:0] event_q;
66
67 always_ff @(posedge clk_i or negedge rst_ni) begin
68     if (!rst_ni) begin
69         count <= Width'(0);
70         event_q <= NEvents'(0);
71     end else if (reset_i) begin
72         count <= Width'(0);
73     end
74 end
75
76 // Do not modify 'event_q' here because the event inputs
may still be asserted.
77 end else begin
78     // Positive-edge triggered, saturating count of events.
79     if (!(ev_enables & event_i & ~event_q) & ep_enabled)
count <= count + ~count;
80     // Retain previous state of event input signals.
81     event_q <= event_i;
82 end
83

```

```

84 assign ev_o = ev_enables;
85 assign endpoints_o = endpoints;
86 assign count_o = count;
87
88 endmodule

```

11. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_iomux.sv

(147 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // USB IO Mux
6 //
7 // Muxes the USB IO signals between the override CSRs and the
USB engine. The

8 // incoming signals are also synchronized to the corresponding
clock domain.
9
10 module usbdev_iomux
11 import usbdev_reg_pkg::*;
12 (
13     input logic clk_i,
14     input logic rst_ni,
15
16     // Register interface
17     output usbdev_hw2reg_phy_pins_sense_reg_t hw2reg_sense_o,
18     input usbdev_reg2hw_phy_pins_drive_reg_t reg2hw_drive_i,
19
20     // External USB Inputs (async)
21     input logic usb_rx_d_i,
22     input logic usb_rx_dp_i,
23     input logic usb_rx_dn_i,
24     input logic cio_usb_sense_i,
25
26     // External USB outputs (usb clk)
27     output logic usb_tx_d_o,
28     output logic usb_tx_se0_o,
29     output logic usb_tx_dp_o,
30     output logic usb_tx_dn_o,
31     output logic usb_tx_oe_o,
32
33     output logic usb_dp_pullup_en_o,
34     output logic usb_dn_pullup_en_o,
35     output logic usb_rx_enable_o,
36
37     // Internal USB Interface (usb clk)
38     output logic usb_rx_d_o,
39     output logic usb_rx_dp_o,
40     output logic usb_rx_dn_o,
41     input logic usb_tx_d_i,
42     input logic usb_tx_se0_i,
43     input logic usb_tx_dp_i,
44     input logic usb_tx_dn_i,
45     input logic usb_tx_oe_i,
46     input logic usb_dp_pullup_en_i,
47     input logic usb_dn_pullup_en_i,
48     input logic usb_rx_enable_i,
49     output logic usb_pwr_sense_o
50 );
51
52 // CDCs
53 //
54 //
55
56 // USB input pins
57 prim_flop_2sync #(
58     .Width (4)
59 ) cdc_io_to_usb (
60     .clk_i,
61     .rst_ni,
62     .d_i ({usb_rx_dp_i,
63            usb_rx_dn_i,
64            usb_rx_d_i,
65            cio_usb_sense_i}),
66     .q_o ({usb_rx_dp_o,
67            usb_rx_dn_o,
68            usb_rx_d_o,
69            usb_pwr_sense_o}
70 );
71
72 // USB pins sense
73 assign hw2reg_sense_o.rx_dp_i.d = usb_rx_dp_o;
74 assign hw2reg_sense_o.rx_dn_i.d = usb_rx_dn_o;
75 assign hw2reg_sense_o.rx_d_i.d = usb_rx_d_o;
76 assign hw2reg_sense_o.pwr_sense.d = usb_pwr_sense_o;
77 assign hw2reg_sense_o.tx_dp_o.d = usb_tx_dp_i;
78 assign hw2reg_sense_o.tx_dn_o.d = usb_tx_dn_i;
79 assign hw2reg_sense_o.tx_d_o.d = usb_tx_d_i;
80 assign hw2reg_sense_o.tx_se0_o.d = usb_tx_se0_i;
81 assign hw2reg_sense_o.tx_oe_o.d = usb_tx_oe_i;
82
83
84 //
85 // USB output pin mux //
86 //

```

```

87 always_comb begin : proc_drive_out
88   if (reg2hw_drive_i.en.q) begin
89     // Override from registers
90     usb_dp_pullup_en_o = reg2hw_drive_i.dp_pullup_en_o.q;
91     usb_dn_pullup_en_o = reg2hw_drive_i.dn_pullup_en_o.q;
92     usb_rx_enable_o = reg2hw_drive_i.rx_enable_o.q;
93   end else begin
94     // Signals from the peripheral core
95     usb_dp_pullup_en_o = usb_dp_pullup_en_i;
96     usb_dn_pullup_en_o = usb_dn_pullup_en_i;
97     usb_rx_enable_o = usb_rx_enable_i;
98   end
99 end
100

101 // Use explicit muxes for the critical output signals, we do
this
102 // to avoid glitches from synthesized logic on these sig-
nals.

103 // Clock muxes should be used here to achieve the best match
between

104 // rising and falling edges on an ASIC. This mismatch on the
data line
105 // degrades performance in the JK-KJ jitter test.
106 prim_clock_mux2 #(
107   .NoFpgaBufG(1)
108 ) i_mux_tx_dp (
109   .clk0_i (usb_tx_dp_i),
110   .clk1_i (reg2hw_drive_i.dp_o.q),
111   .sel_i (reg2hw_drive_i.en.q),
112   .clk_o (usb_tx_dp_o)
113 );
114 prim_clock_mux2 #(
115   .NoFpgaBufG(1)
116 ) i_mux_tx_dn (
117   .clk0_i (usb_tx_dn_i),
118   .clk1_i (reg2hw_drive_i.dn_o.q),
119   .sel_i (reg2hw_drive_i.en.q),
120   .clk_o (usb_tx_dn_o)
121 );
122 prim_clock_mux2 #(
123   .NoFpgaBufG(1)
124 ) i_mux_tx_d (
125   .clk0_i (usb_tx_d_i),
126   .clk1_i (reg2hw_drive_i.d_o.q),
127   .sel_i (reg2hw_drive_i.en.q),
128   .clk_o (usb_tx_d_o)
129 );
130 prim_clock_mux2 #(
131   .NoFpgaBufG(1)
132 ) i_mux_tx_se0 (
133   .clk0_i (usb_tx_se0_i),
134   .clk1_i (reg2hw_drive_i.se0_o.q),
135   .sel_i (reg2hw_drive_i.en.q),
136   .clk_o (usb_tx_se0_o)
137 );
138 prim_clock_mux2 #(
139   .NoFpgaBufG(1)
140 ) i_mux_tx_oe (
141   .clk0_i (usb_tx_oe_i),
142   .clk1_i (reg2hw_drive_i.oe_o.q),
143   .sel_i (reg2hw_drive_i.en.q),
144   .clk_o (usb_tx_oe_o)
145 );
146
147 endmodule

```

12. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_linkstate.sv (390 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Link state detection
6 //
7
8 `include "prim_assert.sv"
9
10 module usbdev_linkstate (
11   input logic clk_48mhz_i,
12   input logic rst_ni,
13   input logic us_tick_i,
14   input logic usb_sense_i,
15   input logic usb_dp_i,
16   input logic usb_dn_i,
17   input logic usb_oe_i,
18   input logic usb_pullup_en_i,
19   input logic rx_idle_det_i,
20   input logic rx_j_det_i,
21   input logic sof_detected_i,
22   input logic resume_link_active_i, // pulse
23
24   output logic link_disconnect_o, // level
25   output logic link_powered_o, // level
26   output logic link_reset_o, // level

```

```

27   output logic link_active_o, // level
28   output logic link_suspend_o, // level
29   output logic link_resume_o, // pulse
30   output logic host_lost_o, // level
31   output logic sof_missed_o, // pulse
32
33   output logic [2:0] link_state_o
34 );
35
36 // Suspend signaling is 3ms of J by spec.
37 localparam logic [11:0] SUSPEND_TIMEOUT = 12'd3000;
38 // Reset is 2.5us - 10ms of SE0 by spec, though care should
be taken to not
39 // confuse the 2 *low-speed* bit times (1.33us) of SE0 that
terminate resume
40 // signaling. Use 3us here.
41 localparam logic [2:0] RESET_TIMEOUT = 3'd3;
42
43 // Consider an SOF lost after 1.005 ms. The extra 5 us helps
accommodate
44 // the worst case frequency difference between the host and
device, due to a
45 // +/- 2500 ppm range around 12 MHz.
46 localparam logic [9:0] SOF_TIMEOUT = 10'd1005;
47
48 typedef enum logic [2:0] {
49   // No power and/or no pull-up connected state
50   LinkDisconnected = 0,
51   // Powered / connected states
52   LinkPowered = 1,
53   LinkPoweredSuspended = 2,
54   // Active states
55   LinkActiveNoSOF = 5,
56   LinkActive = 3,
57   LinkSuspended = 4,
58   LinkResuming = 6
59 } link_state_e;
60
61 typedef enum logic [1:0] {
62   NoRst,
63   RstCnt,
64   RstPend
65 } link_rst_state_e;
66
67 typedef enum logic [1:0] {
68   Active,
69   InactCnt,
70   InactPend
71 } link_inac_state_e;
72
73 link_state_e link_state_d, link_state_q;
74 logic see_pwr_sense;
75
76 // Reset FSM
77 logic [2:0] link_rst_timer_d, link_rst_timer_q;
78 link_rst_state_e link_rst_state_d, link_rst_state_q;
79 logic link_reset; // reset detected (level)
80
81 // Link inactivity detection
82 logic monitor_inac; // monitor link inactivity
83 link_inac_state_e link_inac_timer_d, link_inac_timer_q;
84 link_inac_state_e link_inac_state_d, link_inac_state_q;
85
86 // Events that are not triggered by a timeout
87 logic ev_bus_active;
88
89 // Events that are triggered by timeout
90 logic ev_bus_inactive, ev_reset;
91
92 assign link_disconnect_o = (link_state_q == LinkDisconnec-
ted);
93 assign link_powered_o = see_pwr_sense;
94 assign link_suspend_o = (link_state_q == LinkSuspended ||
link_state_q == LinkPoweredSuspended);
95 assign link_active_o = (link_state_q == LinkActive) ||
(link_state_q == LinkActiveNoSOF);
96 // Link state is stable, so we can output it to the register
assign link_state_o = link_state_q;
97
98 // If the PHY reflects the line state on rx pins when the
device is driving
99 // then the usb_oe_i check isn't needed here. But it seems
best to do the check
100 // to be robust in the face of different PHY designs.
101 assign line_se0_raw = (usb_dn_i == 1'b0) & (usb_dp_i ==
1'b0) & (usb_oe_i == 1'b0);
102
103 // four ticks is a bit time
104 // Could completely filter out 2-cycle EOP SE0 here but
// does not seem needed
105 prim_filter #(
106   .AsyncOn(0), // No synchronizer required
107   .Cycles(6)
108 ) filter_se0 (
109   .clk_i (clk_48mhz_i),
110   .rst_ni (rst_ni),
111   .enable_i (1'b1),
112   .filter_i (line_se0_raw),
113   .filter_o (see_se0)
114 );

```



```

119 );
120
121 prim_filter #(
122     .AsyncOn(0), // No synchronizer required
123     .Cycles(6)
124 ) filter_pwr_sense (
125     .clk_i    (clk_48mhz_i),
126     .rst_ni   (rst_ni),
127     .enable_i (1'b1),
128     .filter_i (usb_sense_i),
129     .filter_o (see_pwr_sense)
130 );
131
132 // Simple events
133 assign ev_bus_active = !rx_idle_det_i;
134
135 assign monitor_inac = see_pwr_sense ? ((link_state_q ==
LinkPowered) | link_active_o) :
136     1'b0;
137
138 always_comb begin
139     link_state_d = link_state_q;
140     link_resume_o = 0;
141
142     // If VBUS ever goes away the link has disconnected (like-
wise if the
143     // pull-up goes away / user requested disconnection)
144     if (!see_pwr_sense || !usb_pullup_en_i) begin
145         link_state_d = LinkDisconnected;
146     end else begin
147         unique case (link_state_q)
148             // No USB supply detected (USB spec: Attached)
149             LinkDisconnected: begin
150                 if (see_pwr_sense & usb_pullup_en_i) begin
151                     link_state_d = LinkPowered;
152                 end
153             end
154
155             LinkPowered: begin
156                 if (ev_reset) begin
157                     link_state_d = LinkActiveNoSOF;
158                 end else if (resume_link_active_i) begin
159                     // Software-directed jump to resume from LinkSus-
pending, in case
160                     // this module was previously powered down.
161                     link_state_d = LinkResuming;
162                 end else if (ev_bus_inactive) begin
163                     link_state_d = LinkPoweredSuspended;
164                 end
165             end
166
167             LinkPoweredSuspended: begin
168                 if (ev_reset) begin
169                     link_state_d = LinkActiveNoSOF;
170                 end else if (ev_bus_active) begin
171                     link_resume_o = 1;
172                     link_state_d = LinkPowered;
173                 end
174             end
175
176             // An event occurred that brought the link out of
LinkSuspended, but
177             // the end-of-resume signaling may not have occurred
yet.
178             // Park here before starting to count towards not see-
ing SOF. Wait for
179             // the end of resume signaling before expecting SOF.
The host will
180             // return the link to idle after a low-speed EOP. In-
stead of trying to
181             // capture the termination of resume signaling dir-
ectly, wait for
182             // any J / idle symbol (or a bus reset).
183             LinkResuming: begin
184                 if (rx_j_det_i | ev_reset) begin
185                     link_resume_o = 1;
186                     link_state_d = LinkActiveNoSOF;
187                 end
188             end
189
190             // Active but not yet seen a frame
191             // One reason for getting stuck here is the host
thinks it is a LS link
192             // which could happen if the flipped bit does not
match the actual pins
193             // Another is the SI is bad so good data is not re-
covered from the link
194             LinkActiveNoSOF: begin
195                 if (ev_bus_inactive) begin
196                     link_state_d = LinkSuspended;
197                 end else if (sof_detected_i) begin
198                     link_state_d = LinkActive;
199                 end
200             end
201
202             // Active (USB spec: Default / Address / Configured)
LinkActive: begin
203                 if (ev_bus_inactive) begin
204                     link_state_d = LinkSuspended;
205                 end else if (ev_reset) begin
206                     link_state_d = LinkActiveNoSOF;
207
208                 end
209             end
210
211             LinkSuspended: begin
212                 if (ev_reset) begin
213                     link_resume_o = 1;
214                     link_state_d = LinkActiveNoSOF;
215                 end else if (ev_bus_active) begin
216                     link_state_d = LinkResuming;
217                 end
218             end
219
220             default: begin
221                 link_state_d = LinkDisconnected;
222             end
223         endcase // case (link_state_q)
224     end
225 end
226
227 `ASSERT(LinkStateValid_A, link_state_q inside {LinkDiscon-
nected, LinkPowered,
228     LinkPoweredSuspended, LinkResuming, LinkActiveNoSOF,
LinkActive, LinkSuspended}, clk_48mhz_i)
229
230 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
231     if (!rst_ni) begin
232         link_state_q <= LinkDisconnected;
233     end else begin
234         link_state_q <= link_state_d;
235     end
236 end
237
238 // Reset detection //
239 // Here we clean up the SE0 signal and generate a single
ev_reset at
240 // the end of a valid reset
241
242 always_comb begin : proc_rst_fsm
243     link_rst_state_d = link_rst_state_q;
244     link_rst_timer_d = link_rst_timer_q;
245     ev_reset = 1'b0;
246     link_reset = 1'b0;
247
248     unique case (link_rst_state_q)
249         // No reset signal detected
250         NoRst: begin
251             if (see_se0) begin
252                 link_rst_state_d = RstCnt;
253                 link_rst_timer_d = 0;
254             end
255         end
256
257         // Reset signal detected -> counting
258         RstCnt: begin
259             if (!see_se0) begin
260                 link_rst_state_d = NoRst;
261             end else begin
262                 if (us_tick_i) begin
263                     if (link_rst_timer_q == RESET_TIMEOUT) begin
264                         link_rst_state_d = RstPend;
265                     end else begin
266                         link_rst_timer_d = link_rst_timer_q + 1;
267                     end
268                 end
269             end
270         end
271
272         // Detected reset -> wait for falling edge
273         RstPend: begin
274             if (!see_se0) begin
275                 link_rst_state_d = NoRst;
276                 ev_reset = 1'b1;
277             end
278             link_reset = 1'b1;
279         end
280
281         default : link_rst_state_d = NoRst;
282     endcase
283 end
284
285 `ASSERT(LinkRstStateValid_A, link_rst_state_q inside {NoRst,
RstCnt, RstPend}, clk_48mhz_i)
286
287 assign link_reset_o = link_reset;
288
289 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin :
proc_reg_rst
290     if (!rst_ni) begin
291         link_rst_state_q <= NoRst;
292         link_rst_timer_q <= 0;
293     end else begin
294         link_rst_state_q <= link_rst_state_d;
295         link_rst_timer_q <= link_rst_timer_d;
296     end
297 end
298
299 // Idle detection //

```

```

303 ///////////////////////////////////////////////////
304 // Here we clean up the idle signal and generate a single
ev_bus_inactive
305 // after the timer expires
306 always_comb begin : proc_idle_det
307     link_inac_state_d = link_inac_state_q;
308     link_inac_timer_d = link_inac_timer_q;
309     ev_bus_inactive = 0;
310
311     unique case (link_inac_state_q)
312         // Active or disabled
313         Active: begin
314             link_inac_timer_d = 0;
315             if (!ev_bus_active && monitor_inac) begin
316                 link_inac_state_d = InactCnt;
317             end
318         end
319
320         // Got an inactivity signal -> count duration
321         InactCnt: begin
322             if (ev_bus_active || !monitor_inac) begin
323                 link_inac_state_d = Active;
324             end else if (us_tick_i) begin
325                 if (link_inac_timer_q == SUSPEND_TIMEOUT) begin
326                     link_inac_state_d = InactPend;
327                     ev_bus_inactive = 1;
328                 end else begin
329                     link_inac_timer_d = link_inac_timer_q + 1;
330                 end
331             end
332         end
333
334         // Counter expired & event sent, wait here
335         InactPend: begin
336             if (ev_bus_active || !monitor_inac) begin
337                 link_inac_state_d = Active;
338             end
339         end
340
341         default : link_inac_state_d = Active;
342     endcase
343 end
344
345 `ASSERT(LincInacStateValid_A, link_inac_state_q inside {Active, InactCnt, InactPend}, clk_48mhz_i)
346
347 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin :
proc_reg_idle_det
348     if (!rst_ni) begin
349         link_inac_state_q <= Active;
350         link_inac_timer_q <= 0;
351     end else begin
352         link_inac_state_q <= link_inac_state_d;
353         link_inac_timer_q <= link_inac_timer_d;
354     end
355 end

```

```

356
357 ///////////////////////////////////////////////////
358 // Host loss and missing sof detection //
359 ///////////////////////////////////////////////////
360 // sof_missed if no SOF was observed in 1.005ms and the link
is active
361 // host_lost if 4 frames have gone by without observing a
SOF
362 logic [2:0] missed_sof_count;
363 logic [9:0] missing_sof_timer;
364
365 assign host_lost_o = missed_sof_count[2];
366 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
367     if (!rst_ni) begin
368         missed_sof_count <= '0;
369     end else begin
370         if (sof_detected_i || !link_active_o || link_reset) be-
gin
371             missed_sof_count <= '0;
372         end else if (sof_missed_o && !host_lost_o) begin
373             missed_sof_count <= missed_sof_count + 1;
374         end
375     end
376 end
377
378 assign sof_missed_o = (missing_sof_timer == SOF_TIMEOUT);
379 always_ff @(posedge clk_48mhz_i or negedge rst_ni) begin
380     if (!rst_ni) begin
381         missing_sof_timer <= '0;
382     end else begin
383         if (sof_missed_o || sof_detected_i || !link_active_o ||
link_reset) begin
384             missing_sof_timer <= '0;
385         end else if (us_tick_i) begin
386             missing_sof_timer <= missing_sof_timer + 1;
387         end
388     end
389 end
390 endmodule

```

13. lowRISC_opentitan/hw/ip/usbdev/rtl/usbdev_pkg.sv (7 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 package usbdev_pkg;
6
7 endpackage : usbdev_pkg

```


RISC-V 타이머

What. mtime/mtimecmp 레지스터와 타이머 인터럽트 생성.

Why it exists. 규격이 정한 최소 블록의 예다. RISC-V 권한 규격이 mtime 의 동작을 정의하므로, 구현 자유도가 거의 없다. 그래서 검증도 단순하다 — 규격대로인가만 본다.

학부 복습 · **EECS concepts.** 타이머/카운터 · 인터럽트 컨트롤러 · 64비트 카운터의 원자적 읽기 문제.

석사 수준 · **Practice note.** 64비트 카운터를 32비트 버스로 읽을 때의 찢김(**tearing**) 이 고전적 함정이다. 상위/하위를 따로 읽는 사이에 값이 넘어가면 엉뚱한 값이 나온다. 규격이 읽기 순서를 정하거나 새도우 레지스터를 둔다.

4 files • 1,088 lines

API — rv_timer.sv

Table 432. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv — module / interface / package (1개)

이름	종류
rv_timer	module

Table 433. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv — parameter (4개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl

Table 434. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
rac1_policies_i	input	top_rac1_pkg::rac1_policy_vec_t
rac1_error_o	output	top_rac1_pkg::rac1_error_log_t
intr_timer_expired_hart0_timer0_o	output	logic

API — rv_timer_reg_pkg.sv

Table 435. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_pkg.sv — module / interface / package (1개)

이름	종류
rv_timer_reg_pkg	package

Table 436. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_pkg.sv — parameter (18개)

파라미터	형/폭	기본값
N_HARTS	int	1
N_TIMERS	int	1
NumAlerts	int	1
BlockAw	int	9
NumRegs	int	10
RV_TIMER_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	9'h 0
RV_TIMER_CTRL_OFFSET	logic [BlockAw-1:0]	9'h 4
RV_TIMER_INTR_ENABLE0_OFFSET	logic [BlockAw-1:0]	9'h 100
RV_TIMER_INTR_STATE0_OFFSET	logic [BlockAw-1:0]	9'h 104
RV_TIMER_INTR_TEST0_OFFSET	logic [BlockAw-1:0]	9'h 108
RV_TIMER_CFG0_OFFSET	logic [BlockAw-1:0]	9'h 10c
RV_TIMER_TIMER_V_LOWER0_OFFSET	logic [BlockAw-1:0]	9'h 110
RV_TIMER_TIMER_V_UPPER0_OFFSET	logic [BlockAw-1:0]	9'h 114
RV_TIMER_COMPARE_LOWER0_0_OFFSET	logic [BlockAw-1:0]	9'h 118
RV_TIMER_COMPARE_UPPER0_0_OFFSET	logic [BlockAw-1:0]	9'h 11c
RV_TIMER_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
RV_TIMER_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
RV_TIMER_INTR_TEST0_RESVAL	logic [0:0]	1'h 0

API — rv_timer_reg_top.sv**Table 437. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — module / interface / package (1개)**

이름	종류
rv_timer_reg_top	module

Table 438. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacl	bit	1'b0
RaclErrorRsp	bit	1'b1

Table 439. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	9
DW	int	32

상수	형/폭	값
DBW	int	DW/8

Table 440. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	rv_timer_reg_pkg::rv_timer_reg2hw_t
hw2reg	input	rv_timer_reg_pkg::rv_timer_hw2reg_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

API — timer_core.sv**Table 441. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv — module / interface / package (1개)**

이름	종류
timer_core	module

Table 442. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv — parameter (1개)

파라미터	형/폭	기본값
N	int	1

Table 443. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
active	input	
prescaler	input	[11:0]
step	input	[7:0]
tick	output	logic
mtime_d	output	logic [63:0]
mtime	input	[63:0]
intr	output	logic [N-1:0]

1. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer.sv (181 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.

```

```

3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6
7 `include "prim_assert.sv"
8
9 module rv_timer import rv_timer_reg_pkg::*;
10 #(

```

```

11 parameter logic [NumAlerts-1:0] AlertAsyn-
con = {NumAlerts{1'b1}},
12 // Number of cycles a differential skew is tolerated on the
alert signal
13 parameter int unsigned AlertSkew-
Cycles = 1,
14 parameter bit EnableR-
acl = 1'b0,
15 parameter bit RaciEr-
rorRsp = EnableRaci,
16 parameter top_racl_pkg::racl_policy_sel_t RaciPol-
cySelVec[NumRegs] = '{NumRegs{0}}
17 ) (
18 input clk_i,
19 input rst_ni,
20
21 input tlul_pkg::tl_h2d_t tl_i,
22 output tlul_pkg::tl_d2h_t tl_o,
23
24 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
25 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
26
27 // RACL interface
28 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
29 output top_racl_pkg::racl_error_log_t racl_error_o,
30
31 output logic intr_timer_expired_hart0_timer0_o
32 );
33
34 rv_timer_reg2hw_t reg2hw;
35 rv_timer_hw2reg_t hw2reg;
36
37 logic [N_HARTS-1:0] active;
38
39 logic [11:0] prescaler [N_HARTS];
40 logic [7:0] step [N_HARTS];
41
42 logic [N_HARTS-1:0] tick;
43
44 logic [63:0] mtime_d [N_HARTS];
45 logic [63:0] mtime [N_HARTS];
46 logic [63:0] mtimecmp [N_HARTS][N_TIMERS]; // Only [harts]
[0] is connected to mtimecmp CSRs
47 logic mtimecmp_update [N_HARTS][N_TIMERS];
48
49 logic [N_HARTS*N_TIMERS-1:0] intr_timer_set;
50 logic [N_HARTS*N_TIMERS-1:0] intr_timer_en;
51 logic [N_HARTS*N_TIMERS-1:0] intr_timer_test_q;
52 logic [N_HARTS-1:0] intr_timer_test_qe;
53 logic [N_HARTS*N_TIMERS-1:0] intr_timer_state_q;
54 logic [N_HARTS-1:0] intr_timer_state_de;
55 logic [N_HARTS*N_TIMERS-1:0] intr_timer_state_d;
56
57 logic [N_HARTS*N_TIMERS-1:0] intr_out;
58
59 // Connecting register interface to the signal //
60 // Connecting register interface to the signal //
61 // Connecting register interface to the signal //
62
63 // Once reggen supports nested multireg, the following can
be automated. For the moment, it must
64 // be connected manually.
65 assign active[0] = reg2hw.ctrl[0].q;
66 assign prescaler = '{reg2hw.cfg0.prescale.q};
67 assign step = '{reg2hw.cfg0.step.q};
68
69 assign hw2reg.timer_v_upper0.de = tick[0];
70 assign hw2reg.timer_v_lower0.de = tick[0];
71 assign hw2reg.timer_v_upper0.d = mtime_d[0][63:32];
72 assign hw2reg.timer_v_lower0.d = mtime_d[0][31:0];
73 assign mtime[0] = {reg2hw.timer_v_upper0.q,
reg2hw.timer_v_lower0.q};
74 assign mtimecmp =
'{{reg2hw.compare_upper0_0.q, reg2hw.compare_lower0_0.q}};
75 assign mtimecmp_update[0][0] = reg2hw.compare_upper0_0.qe |
reg2hw.compare_lower0_0.qe;
76
77 assign intr_timer_expired_hart0_timer0_o = intr_out[0];
78 assign intr_timer_en = reg2hw.intr_enable[0].q;
79 assign intr_timer_state_q = reg2hw.intr_state[0].q;
80 assign intr_timer_test_q = reg2hw.intr_test[0].q;
81 assign intr_timer_test_qe = reg2hw.intr_test[0].qe;
82 assign hw2reg.intr_state[0].de = intr_timer_state_de |
mtimecmp_update[0][0];
83 assign hw2reg.intr_state[0].d = intr_timer_state_d &
~mtimecmp_update[0][0];
84
85
86 for (genvar h = 0 ; h < N_HARTS ; h++) begin : gen_harts
87 prim_intr_hw #(
88 .Width(N_TIMERS)
89 ) u_intr_hw (
90 .clk_i,
91 .rst_ni,
92 .event_intr_i (intr_timer_set),
93
94 .reg2hw_intr_enable_q_i
(intr_timer_en[h*N_TIMERS+:N_TIMERS]),
95 .reg2hw_intr_test_q_i

```

```

(intr_timer_test_q[h*N_TIMERS+:N_TIMERS]),
96 .reg2hw_intr_test_qe_i (intr_timer_test_qe[h]),
97 .reg2hw_intr_state_q_i
(intr_timer_state_q[h*N_TIMERS+:N_TIMERS]),
98 .hw2reg_intr_state_de_o (intr_timer_state_de),
99 .hw2reg_intr_state_d_o
(intr_timer_state_d[h*N_TIMERS+:N_TIMERS]),
100
101 .intr_o (intr_out[h*N_TIMERS+:N_TIMERS])
102 );
103
104 timer_core #(
105 .N (N_TIMERS)
106 ) u_core (
107 .clk_i,
108 .rst_ni,
109
110 .active (active[h]),
111 .prescaler (prescaler[h]),
112 .step (step[h]),
113
114 .tick (tick[h]),
115
116 .mtime_d (mtime_d[h]),
117 .mtime (mtime[h]),
118 .mtimecmp (mtimecmp[h]),
119
120 .intr (intr_timer_set[h*N_TIMERS+:N_TIMERS])
121 );
122 end : gen_harts
123
124 // Register module
125 logic [NumAlerts-1:0] alert_test, alerts;
126 rv_timer_reg_top #(
127 .EnableRaci(EnableRaci),
128 .RaciErrorRsp(RaciErrorRsp),
129 .RaciPolicySelVec(RaciPolicySelVec)
130 ) u_reg (
131 .clk_i,
132 .rst_ni,
133
134 .tl_i,
135 .tl_o,
136
137 .reg2hw,
138 .hw2reg,
139
140 .racl_policies_i,
141 .racl_error_o,
142
143 // SEC_CM: BUS INTEGRITY
144 .intg_err_o (alerts[0])
145 );
146
147 // Alerts
148 assign alert_test = {
149 reg2hw.alert_test.q &
150 reg2hw.alert_test.qe
151 };
152
153 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
154 prim_alert_sender #(
155 .AsyncOn(AlertAsyncOn[i]),
156 .SkewCycles(AlertSkewCycles),
157 .IsFatal(1'b1)
158 ) u_prim_alert_sender (
159 .clk_i,
160 .rst_ni,
161 .alert_test_i ( alert_test[i] ),
162 .alert_req_i ( alerts[i] ),
163 .alert_ack_o ( ),
164 .alert_state_o ( ),
165 .alert_rx_i ( alert_rx_i[i] ),
166 .alert_tx_o ( alert_tx_o[i] )
167 );
168 end
169
170 // Assertions //
171 // Assertions //
172 // Assertions //
173 `ASSERT_KNOWN(TL0DValidKnown, tl_o.d_valid)
174 `ASSERT_KNOWN(TL0AReadyKnown, tl_o.a_ready)
175 `ASSERT_KNOWN(AlertsKnown_A, alert_tx_o)
176 `ASSERT_KNOWN_IF(RaciError0Known_A, racl_error_o, racl_er-
ror_o.valid)
177 `ASSERT_KNOWN(IntrTimerExpiredHart0Timer0Known,
intr_timer_expired_hart0_timer0_o)
178
179 // Alert assertions for reg_we onehot check
180 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[0])
181 endmodule

```

2. lowRISC opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_pkg.sv (159 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for

```

details.
3 // SPDX-License-Identifier: Apache-2.0
4 //

5 // Register Package auto-generated by `reggen` containing data
  structure
6
7   package rv_timer_reg_pkg;
8
9   // Param list
10  parameter int N_HARTS = 1;
11  parameter int N_TIMERS = 1;
12  parameter int NumAlerts = 1;
13
14  // Address widths within the block
15  parameter int BlockAw = 9;
16
17  // Number of registers for every interface
18  parameter int NumRegs = 10;
19
20  // Alert indices
21  typedef enum int {
22    AlertFatalFaultIdx = 0
23  } rv_timer_alert_idx_t;
24
25  ///////////////////////////////////////////////////
26  // Typedefs for registers //
27  ///////////////////////////////////////////////////
28
29  typedef struct packed {
30    logic q;
31    logic qe;
32  } rv_timer_reg2hw_alert_test_reg_t;
33
34  typedef struct packed {
35    logic q;
36  } rv_timer_reg2hw_ctrl_mreg_t;
37
38  typedef struct packed {
39    logic q;
40  } rv_timer_reg2hw_intr_enable0_mreg_t;
41
42  typedef struct packed {
43    logic q;
44  } rv_timer_reg2hw_intr_state0_mreg_t;
45
46  typedef struct packed {
47    logic q;
48    logic qe;
49  } rv_timer_reg2hw_intr_test0_mreg_t;
50
51  typedef struct packed {
52    struct packed {
53      logic [7:0] q;
54    } step;
55    struct packed {
56      logic [11:0] q;
57    } prescale;
58  } rv_timer_reg2hw_cfg0_reg_t;
59
60  typedef struct packed {
61    logic [31:0] q;
62  } rv_timer_reg2hw_timer_v_lower0_reg_t;
63
64  typedef struct packed {
65    logic [31:0] q;
66  } rv_timer_reg2hw_timer_v_upper0_reg_t;
67
68  typedef struct packed {
69    logic [31:0] q;
70    logic qe;
71  } rv_timer_reg2hw_compare_lower0_0_reg_t;
72
73  typedef struct packed {
74    logic [31:0] q;
75    logic qe;
76  } rv_timer_reg2hw_compare_upper0_0_reg_t;
77
78  typedef struct packed {
79    logic d;
80    logic de;
81  } rv_timer_hw2reg_intr_state0_mreg_t;
82
83  typedef struct packed {
84    logic [31:0] d;
85    logic de;
86  } rv_timer_hw2reg_timer_v_lower0_reg_t;
87
88  typedef struct packed {
89    logic [31:0] d;
90    logic de;
91  } rv_timer_hw2reg_timer_v_upper0_reg_t;
92
93  // Register -> HW type
94  typedef struct packed {
95    rv_timer_reg2hw_alert_test_reg_t alert_test; // [156:155]
96    rv_timer_reg2hw_ctrl_mreg_t [0:0] ctrl; // [154:154]
97
98    rv_timer_reg2hw_intr_enable0_mreg_t [0:0] intr_enable0; //
[153:153]
99    rv_timer_reg2hw_intr_state0_mreg_t [0:0] intr_state0; //

```

```

[152:152]
99    rv_timer_reg2hw_intr_test0_mreg_t [0:0] intr_test0; //
[151:150]
100    rv_timer_reg2hw_cfg0_reg_t cfg0; // [149:130]
101    rv_timer_reg2hw_timer_v_lower0_reg_t timer_v_lower0; //
[129:98]
102    rv_timer_reg2hw_timer_v_upper0_reg_t timer_v_upper0; //
[97:66]
103    rv_timer_reg2hw_compare_lower0_0_reg_t com-
pare_lower0_0; // [65:33]
104    rv_timer_reg2hw_compare_upper0_0_reg_t com-
pare_upper0_0; // [32:0]
105  } rv_timer_reg2hw_t;
106
107  // HW -> register type
108  typedef struct packed {
109    rv_timer_hw2reg_intr_state0_mreg_t [0:0] intr_state0; //
[67:66]
110    rv_timer_hw2reg_timer_v_lower0_reg_t timer_v_lower0; //
[65:33]
111    rv_timer_hw2reg_timer_v_upper0_reg_t timer_v_upper0; //
[32:0]
112  } rv_timer_hw2reg_t;
113
114  // Register offsets
115  parameter logic [BlockAw-1:0] RV_TIMER_ALERT_TEST_OFFSET =
9'h 0;
116  parameter logic [BlockAw-1:0] RV_TIMER_CTRL_OFFSET = 9'h 4;
117  parameter logic [BlockAw-1:0] RV_TIMER_INTR_ENABLE0_OFFSET =
9'h 100;
118  parameter logic [BlockAw-1:0] RV_TIMER_INTR_STATE0_OFFSET =
9'h 104;
119  parameter logic [BlockAw-1:0] RV_TIMER_INTR_TEST0_OFFSET =
9'h 108;
120  parameter logic [BlockAw-1:0] RV_TIMER_CFG0_OFFSET = 9'h
100;
121  parameter logic [BlockAw-1:0] RV_TIMER_TIMER_V_LOWER0_OFFSET =
9'h 110;
122  parameter logic [BlockAw-1:0] RV_TIMER_TIMER_V_UPPER0_OFFSET =
9'h 114;
123  parameter logic [BlockAw-1:0]
RV_TIMER_COMPARE_LOWER0_0_OFFSET = 9'h 118;
124  parameter logic [BlockAw-1:0]
RV_TIMER_COMPARE_UPPER0_0_OFFSET = 9'h 11c;
125
126  // Reset values for hwext registers and their fields
127  parameter logic [0:0] RV_TIMER_ALERT_TEST_RESVAL = 1'h 0;
128  parameter logic [0:0] RV_TIMER_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
129  parameter logic [0:0] RV_TIMER_INTR_TEST0_RESVAL = 1'h 0;
130
131  // Register index
132  typedef enum int {
133    RV_TIMER_ALERT_TEST,
134    RV_TIMER_CTRL,
135    RV_TIMER_INTR_ENABLE0,
136    RV_TIMER_INTR_STATE0,
137    RV_TIMER_INTR_TEST0,
138    RV_TIMER_CFG0,
139    RV_TIMER_TIMER_V_LOWER0,
140    RV_TIMER_TIMER_V_UPPER0,
141    RV_TIMER_COMPARE_LOWER0_0,
142    RV_TIMER_COMPARE_UPPER0_0
143  } rv_timer_id_e;
144
145  // Register width information to check illegal writes
146  parameter logic [3:0] RV_TIMER_PERMIT [10] = '{
147    4'b 0001, // index[0] RV_TIMER_ALERT_TEST
148    4'b 0001, // index[1] RV_TIMER_CTRL
149    4'b 0001, // index[2] RV_TIMER_INTR_ENABLE0
150    4'b 0001, // index[3] RV_TIMER_INTR_STATE0
151    4'b 0001, // index[4] RV_TIMER_INTR_TEST0
152    4'b 0111, // index[5] RV_TIMER_CFG0
153    4'b 1111, // index[6] RV_TIMER_TIMER_V_LOWER0
154    4'b 1111, // index[7] RV_TIMER_TIMER_V_UPPER0
155    4'b 1111, // index[8] RV_TIMER_COMPARE_LOWER0_0
156    4'b 1111 // index[9] RV_TIMER_COMPARE_UPPER0_0
157  };
158
159 endpackage

```

3. lowRISC_opentitan/hw/ip/rv_timer/rtl/rv_timer_reg_top.sv (700 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module rv_timer_reg_top
10  # (

```

```

11     parameter bit      EnableRacl      = 1'b0,
12     parameter bit      RaclErrorRsp    = 1'b1,
13     parameter top_racl_pkg::racl_policy_sel_t RaclPolli-
cySelVec[rv_timer_reg_pkg::NumRegs] =
14     '{rv_timer_reg_pkg::NumRegs{0}}
15 ) (
16     input clk_i,
17     input rst_ni,
18     input tlul_pkg::tl_h2d_t tl_i,
19     output tlul_pkg::tl_d2h_t tl_o,
20     // To HW
21     output rv_timer_reg_pkg::rv_timer_reg2hw_t reg2hw, // Write
22     input  rv_timer_reg_pkg::rv_timer_hw2reg_t hw2reg, // Read
23
24     // RACL interface
25     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
26     output top_racl_pkg::racl_error_log_t  racl_error_o,
27
28     // Integrity check errors
29     output logic intg_err_o
30 );
31
32 import rv_timer_reg_pkg::* ;
33
34 localparam int AW = 9;
35 localparam int DW = 32;
36 localparam int DBW = DW/8;          // Byte Width
37
38 // register signals
39 logic      reg_we;
40 logic      reg_re;
41 logic [AW-1:0] reg_addr;
42 logic [DW-1:0] reg_wdata;
43 logic [DBW-1:0] reg_be;
44 logic [DW-1:0] reg_rdata;
45 logic      reg_error;
46
47 logic      addrmiss, wr_err;
48
49 logic [DW-1:0] reg_rdata_next;
50 logic reg_busy;
51
52 tlul_pkg::tl_h2d_t tl_reg_h2d;
53 tlul_pkg::tl_d2h_t tl_reg_d2h;
54
55 // incoming payload check
56 logic intg_err;
57 tlul_cmd_intg_chk u_chk (
58     .tl_i(tl_i),
59     .err_o(intg_err)
60 );
61
62 // also check for spurious write enables
63 logic reg_we_err;
64 logic [9:0] reg_we_check;
65 prim_reg_we_check #(
66     .OneHotWidth(10)
67 ) u_prim_reg_we_check (
68     .clk_i(clk_i),
69     .rst_ni(rst_ni),
70     .oh_i (reg_we_check),
71     .en_i (reg_we && !addrmiss),
72     .err_o (reg_we_err)
73 );
74
75 logic err_q;
76 always_ff @(posedge clk_i or negedge rst_ni) begin
77     if (!rst_ni) begin
78         err_q <= '0;
79     end else if (intg_err || reg_we_err) begin
80         err_q <= 1'b1;
81     end
82 end
83
84 // integrity error output is permanent and should be used
85 for alert generation
86 // register errors are transactional
87 assign intg_err_o = err_q | intg_err | reg_we_err;
88
89 // outgoing integrity generation
90 tlul_pkg::tl_d2h_t tl_o_pre;
91 tlul_rsp_intg_gen #(
92     .EnableRspIntgGen(1),
93     .EnableDataIntgGen(1)
94 ) u_rsp_intg_gen (
95     .tl_i(tl_o_pre),
96     .tl_o(tl_o)
97 );
98
99 assign tl_reg_h2d = tl_i;
100 assign tl_o_pre = tl_reg_d2h;
101
102 tlul_adapter_reg #(
103     .RegAw(AW),
104     .RegDw(DW),
105     .EnableDataIntgGen(0)
106 ) u_reg_if (
107     .clk_i (clk_i),
108     .rst_ni (rst_ni),
109
110     .tl_i (tl_reg_h2d),
111     .tl_o (tl_reg_d2h),
112
113     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
114     .intg_error_o(),
115
116     .we_o (reg_we),
117     .re_o (reg_re),
118     .addr_o (reg_addr),
119     .wdata_o (reg_wdata),
120     .be_o (reg_be),
121     .busy_i (reg_busy),
122     .rdata_i (reg_rdata),
123     // Translate RACL error to TLUL error if enabled
124     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
125 );
126
127 // cdc oversampling signals
128
129 assign reg_rdata = reg_rdata_next ;
130 assign reg_error = addrmiss | wr_err | intg_err;
131
132 // Define SW related signals
133 // Format: <reg>_<field>_<wd|we|qs>
134 // or <reg>_<wd|we|qs> if field == 1 or 0
135 logic alert_test_we;
136 logic alert_test_wd;
137 logic ctrl_we;
138 logic ctrl_qs;
139 logic ctrl_wd;
140 logic intr_enable0_we;
141 logic intr_enable0_qs;
142 logic intr_enable0_wd;
143 logic intr_state0_we;
144 logic intr_state0_qs;
145 logic intr_state0_wd;
146 logic intr_test0_we;
147 logic intr_test0_wd;
148 logic cfg0_we;
149 logic [11:0] cfg0_prescale_qs;
150 logic [11:0] cfg0_prescale_wd;
151 logic [7:0] cfg0_step_qs;
152 logic [7:0] cfg0_step_wd;
153 logic timer_v_lower0_we;
154 logic [31:0] timer_v_lower0_qs;
155 logic [31:0] timer_v_lower0_wd;
156 logic timer_v_upper0_we;
157 logic [31:0] timer_v_upper0_qs;
158 logic [31:0] timer_v_upper0_wd;
159 logic compare_lower0_0_we;
160 logic [31:0] compare_lower0_0_qs;
161 logic [31:0] compare_lower0_0_wd;
162 logic compare_upper0_0_we;
163 logic [31:0] compare_upper0_0_qs;
164 logic [31:0] compare_upper0_0_wd;
165
166 // Register instances
167 // R[alert_test]: V(True)
168 logic alert_test_qe;
169 logic [0:0] alert_test_flds_we;
170 assign alert_test_qe = &alert_test_flds_we;
171 prim_subreg_ext #(
172     .DW (1)
173 ) u_alert_test (
174     .re (1'b0),
175     .we (alert_test_we),
176     .wd (alert_test_wd),
177     .d ('0),
178     .qre (),
179     .qe (alert_test_flds_we[0]),
180     .q (reg2hw.alert_test.q),
181     .ds (),
182     .qs ()
183 );
184 assign reg2hw.alert_test.qe = alert_test_qe;
185
186 // Subregister 0 of Multireg ctrl
187 // R[ctrl]: V(False)
188 prim_subreg #(
189     .DW (1),
190     .SwAccess(prim_subreg_pkg::SwAccessRW),
191     .RESVAL (1'h0),
192     .Mubi (1'b0)
193 ) u_ctrl (
194     .clk_i (clk_i),
195     .rst_ni (rst_ni),
196
197     // from register interface
198     .we (ctrl_we),
199     .wd (ctrl_wd),
200
201     // from internal hardware
202     .de (1'b0),
203     .d ('0),
204
205     // to internal hardware
206     .qe (),
207     .q (reg2hw.ctrl[0].q),
208     .ds (),
209
210

```

```

211 // to register interface (read)
212 .qs (ctrl_qs)
213 );
214
215 // Subregister 0 of Multireg intr_enable0
216 // R[intr_enable0]: V(False)
217 prim_subreg #(
218 .DW (1),
219 .SwAccess(prim_subreg_pkg::SwAccessRW),
220 .RESVAL (1'h0),
221 .Mubi (1'b0)
222 ) u_intr_enable0 (
223 .clk_i (clk_i),
224 .rst_ni (rst_ni),
225
226 // from register interface
227 .we (intr_enable0_we),
228 .wd (intr_enable0_wd),
229
230 // from internal hardware
231 .de (1'b0),
232 .d ('0'),
233
234 // to internal hardware
235 .qe (),
236 .q (reg2hw.intr_enable0[0].q),
237 .ds (),
238
239 // to register interface (read)
240 .qs (intr_enable0_qs)
241 );
242
243 // Subregister 0 of Multireg intr_state0
244 // R[intr_state0]: V(False)
245 prim_subreg #(
246 .DW (1),
247 .SwAccess(prim_subreg_pkg::SwAccessW1C),
248 .RESVAL (1'h0),
249 .Mubi (1'b0)
250 ) u_intr_state0 (
251 .clk_i (clk_i),
252 .rst_ni (rst_ni),
253
254 // from register interface
255 .we (intr_state0_we),
256 .wd (intr_state0_wd),
257
258 // from internal hardware
259 .de (hw2reg.intr_state0[0].de),
260 .d (hw2reg.intr_state0[0].d),
261
262 // to internal hardware
263 .qe (),
264 .q (reg2hw.intr_state0[0].q),
265 .ds (),
266
267 // to register interface (read)
268 .qs (intr_state0_qs)
269 );
270
271 // Subregister 0 of Multireg intr_test0
272 // R[intr_test0]: V(True)
273 logic intr_test0_qe;
274 logic [0:0] intr_test0_flds_we;
275 assign intr_test0_qe = &intr_test0_flds_we;
276 prim_subreg_ext #(
277 .DW (1)
278 ) u_intr_test0 (
279 .re (1'b0),
280 .we (intr_test0_we),
281 .wd (intr_test0_wd),
282 .d ('0'),
283 .qre (),
284 .qe (intr_test0_flds_we[0]),
285 .q (reg2hw.intr_test0[0].q),
286 .ds (),
287 .qs ()
288 );
289 assign reg2hw.intr_test0[0].qe = intr_test0_qe;
290
291 // R[cfg0]: V(False)
292 // F[prescale]: 11:0
293 prim_subreg #(
294 .DW (12),
295 .SwAccess(prim_subreg_pkg::SwAccessRW),
296 .RESVAL (12'h0),
297 .Mubi (1'b0)
298 ) u_cfg0_prescale (
299 .clk_i (clk_i),
300 .rst_ni (rst_ni),
301
302 // from register interface
303 .we (cfg0_we),
304 .wd (cfg0_prescale_wd),
305
306 // from internal hardware
307 .de (1'b0),

```

```

312 .d ('0'),
313
314 // to internal hardware
315 .qe (),
316 .q (reg2hw.cfg0.prescale.q),
317 .ds (),
318
319 // to register interface (read)
320 .qs (cfg0_prescale_qs)
321 );
322
323 // F[step]: 23:16
324 prim_subreg #(
325 .DW (8),
326 .SwAccess(prim_subreg_pkg::SwAccessRW),
327 .RESVAL (8'h1),
328 .Mubi (1'b0)
329 ) u_cfg0_step (
330 .clk_i (clk_i),
331 .rst_ni (rst_ni),
332
333 // from register interface
334 .we (cfg0_step_we),
335 .wd (cfg0_step_wd),
336
337 // from internal hardware
338 .de (1'b0),
339 .d ('0'),
340
341 // to internal hardware
342 .qe (),
343 .q (reg2hw.cfg0.step.q),
344 .ds (),
345
346 // to register interface (read)
347 .qs (cfg0_step_qs)
348 );
349
350 // R[timer_v_lower0]: V(False)
351 prim_subreg #(
352 .DW (32),
353 .SwAccess(prim_subreg_pkg::SwAccessRW),
354 .RESVAL (32'h0),
355 .Mubi (1'b0)
356 ) u_timer_v_lower0 (
357 .clk_i (clk_i),
358 .rst_ni (rst_ni),
359
360 // from register interface
361 .we (timer_v_lower0_we),
362 .wd (timer_v_lower0_wd),
363
364 // from internal hardware
365 .de (hw2reg.timer_v_lower0.de),
366 .d (hw2reg.timer_v_lower0.d),
367
368 // to internal hardware
369 .qe (),
370 .q (reg2hw.timer_v_lower0.q),
371 .ds (),
372
373 // to register interface (read)
374 .qs (timer_v_lower0_qs)
375 );
376
377 // R[timer_v_upper0]: V(False)
378 prim_subreg #(
379 .DW (32),
380 .SwAccess(prim_subreg_pkg::SwAccessRW),
381 .RESVAL (32'h0),
382 .Mubi (1'b0)
383 ) u_timer_v_upper0 (
384 .clk_i (clk_i),
385 .rst_ni (rst_ni),
386
387 // from register interface
388 .we (timer_v_upper0_we),
389 .wd (timer_v_upper0_wd),
390
391 // from internal hardware
392 .de (hw2reg.timer_v_upper0.de),
393 .d (hw2reg.timer_v_upper0.d),
394
395 // to internal hardware
396 .qe (),
397 .q (reg2hw.timer_v_upper0.q),
398 .ds (),
399
400 // to register interface (read)
401 .qs (timer_v_upper0_qs)
402 );
403
404 // R[compare_lower0_0]: V(False)
405 logic compare_lower0_0_qe;
406 logic [0:0] compare_lower0_0_flds_we;
407 prim_flop #(
408 .Width(1),
409 .ResetValue(0)

```



```

413 ) u_compare_lower0_00_qe (
414   .clk_i(clk_i),
415   .rst_ni(rst_ni),
416   .d_i(&compare_lower0_0_flds_we),
417   .q_o(compare_lower0_0_qe)
418 );
419 prim_subreg #(
420   .DW      (32),
421   .SwAccess(prim_subreg_pkg::SwAccessRW),
422   .RESVAL  (32'hffffffff),
423   .Mubi    (1'b0)
424 ) u_compare_lower0_0 (
425   .clk_i  (clk_i),
426   .rst_ni (rst_ni),
427
428   // from register interface
429   .we      (compare_lower0_0_we),
430   .wd      (compare_lower0_0_wd),
431
432   // from internal hardware
433   .de      (1'b0),
434   .d        ('0'),
435
436   // to internal hardware
437   .qe      (compare_lower0_0_flds_we[0]),
438   .q        (reg2hw.compare_lower0_0.q),
439   .ds      (),
440
441   // to register interface (read)
442   .qs      (compare_lower0_0_qs)
443 );
444 assign reg2hw.compare_lower0_0.qe = compare_lower0_0_qe;
445
446 // R[compare_upper0_0]: V(False)
447 logic compare_upper0_0_qe;
448 logic [0:0] compare_upper0_0_flds_we;
449 prim_flop #(
450   .Width(1),
451   .ResetValue(0)
452 ) u_compare_upper0_00_qe (
453   .clk_i(clk_i),
454   .rst_ni(rst_ni),
455   .d_i(&compare_upper0_0_flds_we),
456   .q_o(compare_upper0_0_qe)
457 );
458 prim_subreg #(
459   .DW      (32),
460   .SwAccess(prim_subreg_pkg::SwAccessRW),
461   .RESVAL  (32'hffffffff),
462   .Mubi    (1'b0)
463 ) u_compare_upper0_0 (
464   .clk_i  (clk_i),
465   .rst_ni (rst_ni),
466
467   // from register interface
468   .we      (compare_upper0_0_we),
469   .wd      (compare_upper0_0_wd),
470
471   // from internal hardware
472   .de      (1'b0),
473   .d        ('0'),
474
475   // to internal hardware
476   .qe      (compare_upper0_0_flds_we[0]),
477   .q        (reg2hw.compare_upper0_0.q),
478   .ds      (),
479
480   // to register interface (read)
481   .qs      (compare_upper0_0_qs)
482 );
483 assign reg2hw.compare_upper0_0.qe = compare_upper0_0_qe;
484
485 logic [9:0] addr_hit;
486 top_racl_pkg::racl_role_vec_t racl_role_vec;
487 top_racl_pkg::racl_role_t racl_role;
488
489 logic [9:0] racl_addr_hit_read;
490 logic [9:0] racl_addr_hit_write;
491
492 if (EnableRacl) begin : gen_racl_role_logic
493   // Retrieve RACL role from user bits and one-hot encode
494   that for the comparison bitmap
495   assign racl_role =
496   top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a_user.rsvd);
497
498   prim_onehot_enc #(
499     .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
500   ) u_racl_role_encode (
501     .in_i ( racl_role ),
502     .en_i ( 1'b1 ),
503     .out_o( racl_role_vec )
504   );
505 end else begin : gen_no_racl_role_logic
506   assign racl_role = '0;
507   assign racl_role_vec = '0;
508 end
509
510 always_comb begin

```

```

512   racl_addr_hit_read = '0;
513   racl_addr_hit_write = '0;
514   addr_hit[0] = (reg_addr == RV_TIMER_ALERT_TEST_OFFSET);
515   addr_hit[1] = (reg_addr == RV_TIMER_CTRL_OFFSET);
516   addr_hit[2] = (reg_addr == RV_TIMER_INTR_ENABLE0_OFFSET);
517   addr_hit[3] = (reg_addr == RV_TIMER_INTR_STATE0_OFFSET);
518   addr_hit[4] = (reg_addr == RV_TIMER_INTR_TEST0_OFFSET);
519   addr_hit[5] = (reg_addr == RV_TIMER_CFG0_OFFSET);
520   addr_hit[6] = (reg_addr ==
RV_TIMER_TIMER_V_LOWER0_OFFSET);
521   addr_hit[7] = (reg_addr ==
RV_TIMER_TIMER_V_UPPER0_OFFSET);
522   addr_hit[8] = (reg_addr ==
RV_TIMER_COMPARE_LOWER0_0_OFFSET);
523   addr_hit[9] = (reg_addr ==
RV_TIMER_COMPARE_UPPER0_0_OFFSET);
524
525   if (EnableRacl) begin : gen_racl_hit
526     for (int unsigned slice_idx = 0; slice_idx < 10;
slice_idx++) begin
527       racl_addr_hit_read[slice_idx] =
528         addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].read_perm
& racl_role_vec));
529       racl_addr_hit_write[slice_idx] =
530         addr_hit[slice_idx] & (|
(racl_policies_i[RaclPolicySelVec[slice_idx]].write_perm
& racl_role_vec));
531     end
532   end else begin : gen_no_racl
533     racl_addr_hit_read = addr_hit;
534     racl_addr_hit_write = addr_hit;
535   end
536
537   assign addrmiss = (reg_re || reg_we) ? ~(addr_hit : 1'b0 ;
538   // A valid address hit, access, but failed the RACL check
539   assign racl_error_o.valid = |addr_hit & ((reg_re & ~|
racl_addr_hit_read) |
540   (reg_we & ~|
racl_addr_hit_write));
541   assign racl_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
542   assign racl_error_o.racl_role = racl_role;
543   assign racl_error_o.overflow = 1'b0;
544
545   if (EnableRacl) begin : gen_racl_log
546     assign racl_error_o.ctn_uid =
top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a_user.rsvd);
547     assign racl_error_o.read_access = tl_i.a_opcode ==
tlul_pkg::Get;
548   end else begin : gen_no_racl_log
549     assign racl_error_o.ctn_uid = '0;
550     assign racl_error_o.read_access = 1'b0;
551   end
552
553   // Check sub-word write is permitted
554   always_comb begin
555     wr_err = (reg_we &
556     ((racl_addr_hit_write[0] & (|(RV_TIMER_PERMIT[0] &
~reg_be))) |
557     (racl_addr_hit_write[1] & (|(RV_TIMER_PERMIT[1] &
~reg_be))) |
558     (racl_addr_hit_write[2] & (|(RV_TIMER_PERMIT[2] &
~reg_be))) |
559     (racl_addr_hit_write[3] & (|(RV_TIMER_PERMIT[3] &
~reg_be))) |
560     (racl_addr_hit_write[4] & (|(RV_TIMER_PERMIT[4] &
~reg_be))) |
561     (racl_addr_hit_write[5] & (|(RV_TIMER_PERMIT[5] &
~reg_be))) |
562     (racl_addr_hit_write[6] & (|(RV_TIMER_PERMIT[6] &
~reg_be))) |
563     (racl_addr_hit_write[7] & (|(RV_TIMER_PERMIT[7] &
~reg_be))) |
564     (racl_addr_hit_write[8] & (|(RV_TIMER_PERMIT[8] &
~reg_be))) |
565     (racl_addr_hit_write[9] & (|(RV_TIMER_PERMIT[9] &
~reg_be)))));
566   end
567
568   // Generate write-enables
569   assign alert_test_we = racl_addr_hit_write[0] & reg_we & !
reg_error;
570
571   assign alert_test_wd = reg_wdata[0];
572   assign ctrl_we = racl_addr_hit_write[1] & reg_we & !reg_er-
ror;
573
574   assign ctrl_wd = reg_wdata[0];
575   assign intr_enable0_we = racl_addr_hit_write[2] & reg_we & !

```

```

reg_error;
579 assign intr_enable0_wd = reg_wdata[0];
580 assign intr_state0_we = rACL_addr_hit_write[3] & reg_we & !
reg_error;
582 assign intr_state0_wd = reg_wdata[0];
584 assign intr_test0_we = rACL_addr_hit_write[4] & reg_we & !
reg_error;
586 assign intr_test0_wd = reg_wdata[0];
587 assign cfg0_we = rACL_addr_hit_write[5] & reg_we & !reg_er-
ror;
588 assign cfg0_prescale_wd = reg_wdata[11:0];
590
591 assign cfg0_step_wd = reg_wdata[23:16];
592 assign timer_v_lower0_we = rACL_addr_hit_write[6] & reg_we
& !reg_error;
593
594 assign timer_v_lower0_wd = reg_wdata[31:0];
595 assign timer_v_upper0_we = rACL_addr_hit_write[7] & reg_we
& !reg_error;
596
597 assign timer_v_upper0_wd = reg_wdata[31:0];

598 assign compare_lower0_0_we = rACL_addr_hit_write[8] & reg_we
& !reg_error;
599
600 assign compare_lower0_0_wd = reg_wdata[31:0];

601 assign compare_upper0_0_we = rACL_addr_hit_write[9] & reg_we
& !reg_error;
602
603 assign compare_upper0_0_wd = reg_wdata[31:0];
604
605 // Assign write-enables to checker logic vector.
606 always_comb begin
607   reg_we_check[0] = alert_test_we;
608   reg_we_check[1] = ctrl_we;
609   reg_we_check[2] = intr_enable0_we;
610   reg_we_check[3] = intr_state0_we;
611   reg_we_check[4] = intr_test0_we;
612   reg_we_check[5] = cfg0_we;
613   reg_we_check[6] = timer_v_lower0_we;
614   reg_we_check[7] = timer_v_upper0_we;
615   reg_we_check[8] = compare_lower0_0_we;
616   reg_we_check[9] = compare_upper0_0_we;
617 end
618
619 // Read data return
620 always_comb begin
621   reg_rdata_next = '0;
622   unique case (1'b1)
623     rACL_addr_hit_read[0]: begin
624       reg_rdata_next[0] = '0;
625     end
626
627     rACL_addr_hit_read[1]: begin
628       reg_rdata_next[0] = ctrl_qs;
629     end
630
631     rACL_addr_hit_read[2]: begin
632       reg_rdata_next[0] = intr_enable0_qs;
633     end
634
635     rACL_addr_hit_read[3]: begin
636       reg_rdata_next[0] = intr_state0_qs;
637     end
638
639     rACL_addr_hit_read[4]: begin
640       reg_rdata_next[0] = '0;
641     end
642
643     rACL_addr_hit_read[5]: begin
644       reg_rdata_next[11:0] = cfg0_prescale_qs;
645       reg_rdata_next[23:16] = cfg0_step_qs;
646     end
647
648     rACL_addr_hit_read[6]: begin
649       reg_rdata_next[31:0] = timer_v_lower0_qs;
650     end
651
652     rACL_addr_hit_read[7]: begin
653       reg_rdata_next[31:0] = timer_v_upper0_qs;
654     end
655
656     rACL_addr_hit_read[8]: begin
657       reg_rdata_next[31:0] = compare_lower0_0_qs;
658     end
659
660     rACL_addr_hit_read[9]: begin
661       reg_rdata_next[31:0] = compare_upper0_0_qs;
662     end
663
664     default: begin

```

```

665       reg_rdata_next = '1;
666     end
667   endcase
668 end
669
670 // shadow busy
671 logic shadow_busy;
672 assign shadow_busy = 1'b0;
673
674 // register busy
675 assign reg_busy = shadow_busy;
676
677 // Unused signal tieoff
678
679 // wdata / byte enable are not always fully used
680 // add a blanket unused statement to handle lint waivers
681 logic unused_wdata;
682 logic unused_be;
683 assign unused_wdata = ^reg_wdata;
684 assign unused_be = ^reg_be;
685 logic unused_policy_sel;
686 assign unused_policy_sel = ^rACL_policies_i;
687
688 // Assertions for Register Interface
689 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
690 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
691
692 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>=
tl_o_pre.d_valid, clk_i, !rst_ni)
693
694 `ASSERT(en2addrHit, (reg_we || reg_re) |>= $one-
hot0(addr_hit), clk_i, !rst_ni)
695
696 // this is formulated as an assumption such that the FPV
testbenches do disprove this
697 // property by mistake
698 // `ASSUME(reqParity, tl_reg_h2d.a_valid |>=
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
699
700 endmodule

```

4. lowRISC_opentitan/hw/ip/rv_timer/rtl/timer_core.sv (48 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6
7 module timer_core #(
8   parameter int N = 1
9 ) (
10  input clk_i,
11  input rst_ni,
12
13  input active,
14  input [11:0] prescaler,
15  input [7:0] step,
16
17  output logic tick,
18  output logic [63:0] mtime_d,
19  input [63:0] mtime,
20  input [63:0] mtimecmp [N],
21
22  output logic [N-1:0] intr
23 );
24
25 logic [11:0] tick_count;
26
27 always_ff @(posedge clk_i or negedge rst_ni) begin : gener-
ate_tick
28   if (!rst_ni) begin
29     tick_count <= 12'h0;
30   end else if (!active) begin
31     tick_count <= 12'h0;
32   end else if (tick_count == prescaler) begin
33     tick_count <= 12'h0;
34   end else begin
35     tick_count <= tick_count + 1'b1;
36   end
37 end
38
39 assign tick = active & (tick_count >= prescaler);
40
41 assign mtime_d = mtime + 64'(step);
42
43 // interrupt is generated if mtime is greater than or equal
to mtimecmp
44 for (genvar t = 0 ; t < N ; t++) begin : gen_intr
45   assign intr[t] = active & (mtime >= mtimecmp[t]);
46 end
47
48 endmodule : timer_core

```

RISC-V 디버그 모듈

What. RISC-V Debug Spec 의 디버그 모듈. JTAG(DTM) → 디버그 모듈 → 코어 제어.

Why it exists. 칩이 안 뜰 때 볼 수 있는 유일한 창 이다. 이것이 없으면 브링업이 불가능하다. 그래서 디버그 모듈은 코어가 멈춰 있어도 동작해야 하고, 별도 클럭/리셋 도메인에 있는 경우가 많다.

학부 복습 · **EECS concepts.** JTAG TAP 상태기계(IEEE 1149.1) · 경계 스캔 · 디버그 트랜스포트 · 클럭 도메인 교차.

석사 수준 · **Practice note.** 디버그 경로가 보안 경계를 뚫는 통로 라는 점을 잊지 마라. 양산 칩은 수명주기 상태에 따라 디버그를 잠근다(OpenTitan 의 lc_ctrl 참고). 디버그와 보안의 충돌은 모든 보안칩의 설계 쟁점이다.

7 files • 2,548 lines

API — jtag_pkg.sv

Table 444. lowRISC_opentitan/hw/ip/rv_dm/rtl/jtag_pkg.sv — module / interface / package (1개)

이름	종류
jtag_pkg	package

Table 445. lowRISC_opentitan/hw/ip/rv_dm/rtl/jtag_pkg.sv — parameter (2개)

파라미터	형/폭	기본값
JTAG_REQ_DEFAULT	jtag_req_t	'0
JTAG_RSP_DEFAULT	jtag_rsp_t	'0

API — rv_dm.sv

Table 446. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — module / interface / package (1개)

이름	종류
rv_dm	module

Table 447. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — parameter (8개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
IdcodeValue	logic [31:0]	32'h 0000_0001
UseDmiInterface	bit	1'b0
SecVolatileRawUnlockEn	bit	0
TlulHostUserRsvdBits	logic [tlul_pkg::RsvdWidth-1:0]	0
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl

Table 448. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — localparam (6개)

상수	형/폭	값
DebugHartInfo	dm::hartinfo_t	'{
BusWidth	int	32
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
LcEnDebugReqVal	int	4 - 1
LcEnResetReqVal	int	LcEnDebugReqVal + NrHarts
RvDmLcEnSize	int	\$clog2(LcEnResetReqVal + 2)

Table 449. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv — 포트 (35개)

포트	방향	형/폭
clk_i	input	logic
clk_lc_i	input	logic
rst_ni	input	logic
rst_lc_ni	input	logic
next_dm_addr_i	input	logic [31:0]
lc_hw_debug_clr_i	input	lc_ctrl_pkg::lc_tx_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_dft_en_i	input	lc_ctrl_pkg::lc_tx_t
pinmux_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_check_byp_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_init_done_i	input	lc_ctrl_pkg::lc_tx_t
strap_en_i	input	
strap_en_override_i	input	
otp_dis_rv_dm_late_debug_i	input	prim_mubi_pkg::mubi8_t
scanmode_i	input	prim_mubi_pkg::mubi4_t
scan_rst_ni	input	
ndmreset_req_o	output	logic
dmactive_o	output	logic
debug_req_o	output	logic [NrHarts-1:0]
unavailable_i	input	logic [NrHarts-1:0]
regs_tl_d_i	input	tlul_pkg::tl_h2d_t
regs_tl_d_o	output	tlul_pkg::tl_d2h_t
mem_tl_d_i	input	tlul_pkg::tl_h2d_t
mem_tl_d_o	output	tlul_pkg::tl_d2h_t

포트	방향	형/폭
sba_tl_h_o	output	tlul_pkg::tl_h2d_t
sba_tl_h_i	input	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
jtag_i	input	jtag_pkg::jtag_req_t
jtag_o	output	jtag_pkg::jtag_rsp_t
dbg_tl_d_i	input	tlul_pkg::tl_h2d_t
dbg_tl_d_o	output	tlul_pkg::tl_d2h_t

API — rv_dm_dbg_reg_top.sv

Table 450. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dbg_reg_top.sv — module / interface / package (1개)

이름	종류
rv_dm_dbg_reg_top	module

Table 451. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dbg_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
tl_win_o	output	tlul_pkg::tl_h2d_t
tl_win_i	input	tlul_pkg::tl_d2h_t
intg_err_o	output	logic

API — rv_dm_dmi_gate.sv

Table 452. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv — module / interface / package (1개)

이름	종류
rv_dm_dmi_gate	module

Table 453. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv — parameter (1개)

파라미터	형/폭	기본값
SecVolatileRawUnlockEn	bit	0

Table 454. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
strap_en_override_i	input	
strap_en_i	input	
lc_hw_debug_clr_i	input	lc_ctrl_pkg::lc_tx_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_check_byp_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
dbg_tl_h2d_win_i	input	tlul_pkg::tl_h2d_t
dbg_tl_d2h_win_o	output	tlul_pkg::tl_d2h_t
dmi_req_valid_o	output	
dmi_req_ready_i	input	
dmi_req_o	output	dm::dmi_req_t
dmi_rsp_valid_i	input	
dmi_rsp_ready_o	output	
dmi_rsp_i	input	dm::dmi_resp_t
intg_error_o	output	

API — rv_dm_pkg.sv

Table 455. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv — module / interface / package (1개)

이름	종류
rv_dm_pkg	package

Table 456. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv — parameter (1개)

파라미터	형/폭	기본값
NEXT_DM_ADDR_DEFAULT	next_dm_addr_t	'0

Table 457. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv — typedef / using (1개)

새 이름	원래 형
next_dm_addr_t	logic [31:0]

API — rv_dm_reg_pkg.sv

Table 458. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_reg_pkg.sv — module / interface / package (1개)

이름	종류
rv_dm_reg_pkg	package

Table 459. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_reg_pkg.sv — parameter (300개) — 앞 60개만 표시(전체 300개, 본문 소스 참조)

파라미터	형/폭	기본값
NrHarts	int	1
NumAlerts	int	1
RegsAw	int	4
MemAw	int	12
DbgAw	int	9
NumRegsRegs	int	3
NumRegsMem	int	281
NumRegsDbg	int	0
RV_DM_ALERT_TEST_OFFSET	logic [RegsAw-1:0]	4'h 0
RV_DM_LATE_DEBUG_ENABLE_REGWEN_OFFSET	logic [RegsAw-1:0]	4'h 4
RV_DM_LATE_DEBUG_ENABLE_OFFSET	logic [RegsAw-1:0]	4'h 8
RV_DM_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
RV_DM_ALERT_TEST_FATAL_FAULT_RESVAL	logic [0:0]	1'h 0
RV_DM_HALTED_OFFSET	logic [MemAw-1:0]	12'h 100
RV_DM_GOING_OFFSET	logic [MemAw-1:0]	12'h 108
RV_DM_RESUMING_OFFSET	logic [MemAw-1:0]	12'h 110
RV_DM_EXCEPTION_OFFSET	logic [MemAw-1:0]	12'h 118
RV_DM_WHERETO_OFFSET	logic [MemAw-1:0]	12'h 300
RV_DM_ABSTRACTCMD_0_OFFSET	logic [MemAw-1:0]	12'h 338
RV_DM_ABSTRACTCMD_1_OFFSET	logic [MemAw-1:0]	12'h 33c
RV_DM_ABSTRACTCMD_2_OFFSET	logic [MemAw-1:0]	12'h 340
RV_DM_ABSTRACTCMD_3_OFFSET	logic [MemAw-1:0]	12'h 344
RV_DM_ABSTRACTCMD_4_OFFSET	logic [MemAw-1:0]	12'h 348
RV_DM_ABSTRACTCMD_5_OFFSET	logic [MemAw-1:0]	12'h 34c
RV_DM_ABSTRACTCMD_6_OFFSET	logic [MemAw-1:0]	12'h 350
RV_DM_ABSTRACTCMD_7_OFFSET	logic [MemAw-1:0]	12'h 354
RV_DM_ABSTRACTCMD_8_OFFSET	logic [MemAw-1:0]	12'h 358
RV_DM_ABSTRACTCMD_9_OFFSET	logic [MemAw-1:0]	12'h 35c
RV_DM_PROGRAM_BUFFER_0_OFFSET	logic [MemAw-1:0]	12'h 360
RV_DM_PROGRAM_BUFFER_1_OFFSET	logic [MemAw-1:0]	12'h 364
RV_DM_PROGRAM_BUFFER_2_OFFSET	logic [MemAw-1:0]	12'h 368
RV_DM_PROGRAM_BUFFER_3_OFFSET	logic [MemAw-1:0]	12'h 36c
RV_DM_PROGRAM_BUFFER_4_OFFSET	logic [MemAw-1:0]	12'h 370
RV_DM_PROGRAM_BUFFER_5_OFFSET	logic [MemAw-1:0]	12'h 374
RV_DM_PROGRAM_BUFFER_6_OFFSET	logic [MemAw-1:0]	12'h 378

파라미터	형/폭	기본값
RV_DM_PROGRAM_BUFFER_7_OFFSET	logic [MemAw-1:0]	12'h 37c
RV_DM_DATAADDR_0_OFFSET	logic [MemAw-1:0]	12'h 380
RV_DM_DATAADDR_1_OFFSET	logic [MemAw-1:0]	12'h 384
RV_DM_FLAGS_0_OFFSET	logic [MemAw-1:0]	12'h 400
RV_DM_FLAGS_1_OFFSET	logic [MemAw-1:0]	12'h 404
RV_DM_FLAGS_2_OFFSET	logic [MemAw-1:0]	12'h 408
RV_DM_FLAGS_3_OFFSET	logic [MemAw-1:0]	12'h 40c
RV_DM_FLAGS_4_OFFSET	logic [MemAw-1:0]	12'h 410
RV_DM_FLAGS_5_OFFSET	logic [MemAw-1:0]	12'h 414
RV_DM_FLAGS_6_OFFSET	logic [MemAw-1:0]	12'h 418
RV_DM_FLAGS_7_OFFSET	logic [MemAw-1:0]	12'h 41c
RV_DM_FLAGS_8_OFFSET	logic [MemAw-1:0]	12'h 420
RV_DM_FLAGS_9_OFFSET	logic [MemAw-1:0]	12'h 424
RV_DM_FLAGS_10_OFFSET	logic [MemAw-1:0]	12'h 428
RV_DM_FLAGS_11_OFFSET	logic [MemAw-1:0]	12'h 42c
RV_DM_FLAGS_12_OFFSET	logic [MemAw-1:0]	12'h 430
RV_DM_FLAGS_13_OFFSET	logic [MemAw-1:0]	12'h 434
RV_DM_FLAGS_14_OFFSET	logic [MemAw-1:0]	12'h 438
RV_DM_FLAGS_15_OFFSET	logic [MemAw-1:0]	12'h 43c
RV_DM_FLAGS_16_OFFSET	logic [MemAw-1:0]	12'h 440
RV_DM_FLAGS_17_OFFSET	logic [MemAw-1:0]	12'h 444
RV_DM_FLAGS_18_OFFSET	logic [MemAw-1:0]	12'h 448
RV_DM_FLAGS_19_OFFSET	logic [MemAw-1:0]	12'h 44c
RV_DM_FLAGS_20_OFFSET	logic [MemAw-1:0]	12'h 450
RV_DM_FLAGS_21_OFFSET	logic [MemAw-1:0]	12'h 454

API — rv_dm_regs_reg_top.sv

Table 460. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — module / interface / package (17개)

이름	종류
rv_dm_regs_reg_top	module

Table 461. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRac1	bit	1'b0
Rac1ErrorRsp	bit	1'b1

Table 462. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	4
DW	int	32
DBW	int	DW/8

Table 463. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	rv_dm_reg_pkg::rv_dm_regs_reg2hw_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

1. lowRISC_opentitan/hw/ip/rv_dm/rtl/jtag_pkg.sv (24 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package jtag_pkg;
7
8   typedef struct packed {
9     logic tck;
10    logic tms;
11    logic trst_n;
12    logic tdi;
13  } jtag_req_t;
14
15  parameter jtag_req_t JTAG_REQ_DEFAULT = '0;
16
17  typedef struct packed {
18    logic tdo;
19    logic tdo_oe;
20  } jtag_rsp_t;
21
22  parameter jtag_rsp_t JTAG_RSP_DEFAULT = '0;
23
24 endpackage : jtag_pkg

```

2. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm.sv (783 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Top-level debug module (DM)
6 //
7 // This module implements the RISC-V debug specification ver-
  sion 0.13,
8 //
9 // This toplevel wraps the PULP debug module available from
10 // https://github.com/pulp-platform/riscv-dbg to match the
  needs of
11 // the TL-UL-based lowRISC chip design.
12
13 `include "prim_assert.sv"
14
15 module rv_dm
16   import rv_dm_reg_pkg::*;
17   #(
18     parameter logic [NumAlerts-1:0]          AlertAsyn-
19     = {NumAlerts{1'b1}},
20     cOn // Number of cycles a differential skew is tolerated on the

```

```

alert signal
20   parameter int unsigned          AlertSkew-
  Cycles = 1,
21   parameter logic [31:0]          Idcode-
  Value = 32'h 0000_0001,
22   parameter bit                    UseDmiInter-
  face = 1'b0,
23   parameter bit                    SecVolatileRawUn-
  lockEn = 0,
24   parameter logic [tlul_pkg::RsvdWidth-1:0] TlulHostUserRsvd-
  Bits = 0,
25   parameter bit                    EnableR-
  acl = 1'b0,
26   parameter bit                    RaclEr-
  rorRsp = EnableRacl,
27   parameter top_racl_pkg::racl_policy_sel_t RaclPoli-
  cySelVecRegs[NumRegsRegs] = '{NumRegsRegs{0}}
28 ) (
29   input logic                      clk_i,      // clock
30   input logic                      clk_lc_i,   // only declared
  here so that the topgen
31
32   // tooling connects
  the correct clk/rst domains.
33   input logic                      rst_ni,      // asynchronous re-
  set active low, connect PoR
34   // here, not the
  system reset
35   input logic                      rst_lc_ni,   // asynchronous re-
  set active low, connect the lc
36
37   // reset here. this is
  only used for NDM reset tracking.
38   input logic [31:0]              next_dm_addr_i, // static word
  address of the next debug module.
39   // SEC_CM: LC_HW_DEBUG_CLR.INTERSIG.MUBI
  input lc_ctrl_pkg::lc_tx_t lc_hw_debug_clr_i,
40   // SEC_CM: LC_HW_DEBUG_EN.INTERSIG.MUBI
  input lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
41   // HW Debug lifecycle enable signal (live version from the
  life cycle controller)
42   input lc_ctrl_pkg::lc_tx_t lc_dft_en_i,
43   // SEC_CM: LC_DFT_EN.INTERSIG.MUBI
  input lc_ctrl_pkg::lc_tx_t lc_dft_en_i,
44   // HW DFT lifecycle enable signal (live version from the
  life cycle controller)
45   input lc_ctrl_pkg::lc_tx_t lc_dft_en_i,
46   // HW Debug lifecycle enable signal (latched version from
  pinmux, only used for JTAG/TAP gating)
47   input lc_ctrl_pkg::lc_tx_t pinmux_hw_debug_en_i,
48   input lc_ctrl_pkg::lc_tx_t lc_check_byp_en_i,
49   input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
50   // Lifecycle init done indicator used to clear NDM latch
  after LC re-initialization
51   input lc_ctrl_pkg::lc_tx_t lc_init_done_i,
52   input strap_en_i,
53   input strap_en_override_i,
54   // SEC_CM: OTP_DIS_RV_DM_LATE_DEBUG.INTERSIG.MUBI
  // Late debug enable disablement signal coming from the OTP

```

```

HW_CFG1 partition.
55 input prim_mubi_pkg::mubi8_t otp_dis_rv_dm_late_debug_i,
56 input prim_mubi_pkg::mubi4_t scanmode_i,
57 input scan_rst_ni,
58 output logic ndmreset_req_o, // non-debug
module reset
59 output logic dmactive_o, // debug module is
active
60 output logic [NrHarts-1:0] debug_req_o, // async debug re-
quest
61 input logic [NrHarts-1:0] unavailable_i, // communicate
whether the hart is unavailable
62 // (e.g.: power
down)
63
64 // bus device for comfortable CSR access
65 input tlul_pkg::tl_h2d_t regs_tl_d_i,
66 output tlul_pkg::tl_d2h_t regs_tl_d_o,
67
68 // bus device with debug memory, for an execution based
technique
69 input tlul_pkg::tl_h2d_t mem_tl_d_i,
70 output tlul_pkg::tl_d2h_t mem_tl_d_o,
71
72 // bus host, for system bus accesses
73 output tlul_pkg::tl_h2d_t sba_tl_h_o,
74 input tlul_pkg::tl_d2h_t sba_tl_h_i,
75
76 // Alerts
77 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_rx_i,
78 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
alert_tx_o,
79
80 // RACL interface
81 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
82 output top_racl_pkg::racl_error_log_t racl_error_o,
83
84 // JTAG TAP
85 input jtag_pkg::jtag_req_t jtag_i,
86 output jtag_pkg::jtag_rsp_t jtag_o,
87 // TL-UL-based DMI
88 input tlul_pkg::tl_h2d_t dbg_tl_d_i,
89 output tlul_pkg::tl_d2h_t dbg_tl_d_o
90 );
91
92 ///////////////////////////////////////////////////
93 // Parameter Definitions //
94 ///////////////////////////////////////////////////
95
96 import prim_mubi_pkg::mubi4_test_true_strict;
97 import prim_mubi_pkg::mubi8_test_true_strict;
98 import prim_mubi_pkg::mubi32_test_true_strict;
99 import lc_ctrl_pkg::lc_tx_test_true_strict;
100
101 `ASSERT_INIT(paramCheckNrHarts, NrHarts > 0)
102
103 // static debug hartinfo
104 localparam dm::hartinfo_t DebugHartInfo = '{
105   zero0: 0,
106   nscratch: 2, // Debug module needs at least two scratch
regs
107   zero0: 0,
108   dataaccess: 1'b1, // data registers are memory mapped in
the debugger
109   datasize: dm::DataCount,
110   dataaddr: dm::DataAddr
111 };
112
113 dm::hartinfo_t [NrHarts-1:0] hartinfo;
114 for (genvar i = 0; i < NrHarts; i++) begin :
gen_dm_hart_ctrl
115   assign hartinfo[i] = DebugHartInfo;
116 end
117
118 // Currently only 32 bit busses are supported by our TL-UL
IP
119 localparam int BusWidth = 32;
120 // all harts have contiguous IDs
121 localparam logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}};
122
123 ///////////////////////////////////////////////////
124 // CSR Nodes //
125 ///////////////////////////////////////////////////
126
127 tlul_pkg::tl_h2d_t mem_tl_win_h2d;
128 tlul_pkg::tl_d2h_t mem_tl_win_d2h;
129 rv_dm_reg_pkg::rv_dm_regs_reg2hw_t regs_reg2hw;
130 logic regs_intg_error, rom_intg_error, dmi_intg_error,
dbg_intg_error;
131 logic sba_gate_intg_error, rom_gate_intg_error;
132
133 rv_dm_regs_reg_top #(
134   .EnableRacl(EnableRacl),
135   .RaclErrorRsp(RaclErrorRsp),
136   .RaclPolicySelVec(RaclPolicySelVecRegs)
137 ) u_reg_regs (
138   .clk_i,
139   .rst_ni,
140   .tl_i (regs_tl_d_i ),

```

```

141   .tl_o (regs_tl_d_o ),
142   .reg2hw (regs_reg2hw ),
143   .racl_policies_i,
144   .racl_error_o,
145   // SEC_CM: BUS.INTEGRITY
146   .intg_err_o(regs_intg_error)
147 );
148
149 // We are not instantiating the generated rv_dm_mem_reg_top,
since the registers are actually
150 // implemented inside the vendored-in rv_dm module from the
PULP project.
151 assign mem_tl_win_h2d = mem_tl_d_i;
152 assign mem_tl_d_o = mem_tl_win_d2h;
153
154 // Alerts
155 logic [NumAlerts-1:0] alert_test, alerts;
156
157 assign alerts[0] = regs_intg_error | rom_intg_error |
dmi_intg_error | dbg_intg_error |
158   sba_gate_intg_error | rom_g-
ate_intg_error;
159
160 assign alert_test = {
161   regs_reg2hw.alert_test.q &
162   regs_reg2hw.alert_test.qe
163 };
164
165 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
166   prim_alert_sender #(
167     .AsyncOn(AlertAsyncOn[i]),
168     .SkewCycles(AlertSkewCycles),
169     .IsFatal(1'b1)
170   ) u_prim_alert_sender (
171     .clk_i,
172     .rst_ni,
173     .alert_test_i ( alert_test[i] ),
174     .alert_req_i ( alerts[i] ),
175     .alert_ack_o ( ),
176     .alert_state_o ( ),
177     .alert_rx_i ( alert_rx_i[i] ),
178     .alert_tx_o ( alert_tx_o[i] )
179   );
180 end
181
182 // Decode multibit scanmode enable
183 logic testmode;
184 assign testmode = mubi4_test_true_strict(scanmode_i);
185
186 ///////////////////////////////////////////////////
187 // Life Cycle Gating //
188 ///////////////////////////////////////////////////
189
190 // Debug enable gating.
191 localparam int LcEnDebugReqVal = 4 - 1;
192 localparam int LcEnResetReqVal = LcEnDebugReqVal + NrHarts;
193 // +1 to get number of bits and another +1 because LcEnLast-
Pos is one more than LcEnResetReq.
194 localparam int RvDmLcEnSize = $clog2(LcEnResetReqVal +
2);
195 typedef enum logic [RvDmLcEnSize-1:0] {
196   LcEnFetch,
197   LcEnRom,
198   LcEnSba,
199   // LcEnDebugReq[NrHarts], <= this unfortunately does not
work - SV-LRM mandates the use of
200   // integral numbers. Parameters are not allowed in this
context.
201   LcEnDebugReq,
202   // The above literal accommodates NrHarts number of debug
requests - so we number the next
203   // literal accordingly.
204   LcEnResetReq = RvDmLcEnSize'(LcEnResetReqVal),
205   // LcEnLastPos must immediately follow LcEnResetReq to
calculate RvDmLcEnSize.
206   LcEnLastPos
207 } rv_dm_lc_en_e;
208 // These must be equal so that the difference between
LcEnResetReq and LcEnDebugReq is NrHarts.
209 `ASSERT(RvDmLcEnDebugVal_A, int'(LcEnDebugReq) == LcEnDebu-
gReqVal)
210
211 lc_ctrl_pkg::lc_tx_t lc_hw_debug_en;
212 prim_lc_sync #(
213   .NumCopies(1)
214 ) u_prim_lc_sync_lc_hw_debug_en (
215   .clk_i,
216   .rst_ni,
217   .lc_en_i(lc_hw_debug_en_i),
218   .lc_en_o({lc_hw_debug_en})
219 );
220
221 lc_ctrl_pkg::lc_tx_t lc_dft_en;
222 prim_lc_sync #(
223   .NumCopies(1)
224 ) u_prim_lc_sync_lc_dft_en (
225   .clk_i,
226   .rst_ni,
227   .lc_en_i(lc_dft_en_i),
228   .lc_en_o({lc_dft_en})

```

```

229 );
230
231 prim_mubi_pkg::mubi8_t [lc_ctrl_pkg::TxWidth-1:0]
otp_dis_rv_dm_late_debug;
232 prim_mubi8_sync #(
233   .NumCopies (lc_ctrl_pkg::TxWidth)
234 ) u_prim_mubi8_sync_otp_dis_rv_dm_late_debug (
235   .clk_i,
236   .rst_ni,
237   .mubi_i(otp_dis_rv_dm_late_debug_i),
238   .mubi_o(otp_dis_rv_dm_late_debug)
239 );
240
241 prim_mubi_pkg::mubi32_t [lc_ctrl_pkg::TxWidth-1:0] late_de-
bug_enable;
242 prim_mubi32_sync #(
243   .NumCopies (lc_ctrl_pkg::TxWidth),
244   .AsyncOn(0) // No synchronization required since the input
signal is already synchronous.
245 ) u_prim_mubi32_sync_late_debug_enable (
246   .clk_i,
247   .rst_ni,
248   .mubi_i(prim_mubi_pkg::mubi32_t'(regs_reg2hw.late_debug_e
nable)),
249   .mubi_o(late_debug_enable)
250 );
251
252 // SEC_CM: DM_EN.CTRL.LC_GATED

253 // This implements a hardened MuBi multiplexor circuit where
each output bitlane has its own
254 // associated comparators for the enablement condition.
255 logic [lc_ctrl_pkg::TxWidth-1:0] lc_hw_debug_en_raw;
256 logic [lc_ctrl_pkg::TxWidth-1:0] lc_dft_en_raw;
257 logic [lc_ctrl_pkg::TxWidth-1:0] lc_hw_debug_en_gated_raw;
258 assign lc_hw_debug_en_raw = lc_hw_debug_en;
259 assign lc_dft_en_raw = lc_dft_en;
260 for (genvar k = 0; k < lc_ctrl_pkg::TxWidth; k++) begin :
gen_mubi_mux
261   assign lc_hw_debug_en_gated_raw[k] = (mu-
bi8_test_true_strict(otp_dis_rv_dm_late_debug[k]) ||
262     mu-
bi32_test_true_strict(late_debug_enable[k])) ?
263     lc_hw_de-
bug_en_raw[k] :
264     lc_dft_en_raw[k];
265 end
266

267 // The lc_hw_debug_en_gated signal modulates gating logic on
the bus-side of the RV_DM.

268 // The pinmux_hw_debug_en signal on the other hand modulates
the TAP side of the RV_DM.
269 // In order for the RV_DM to remain response during a NDM
reset request, the TAP side
270 // is not further modulated with the LATE_DEBUG_ENABLE CSR.
271 lc_ctrl_pkg::lc_tx_t [LcEnLastPos-1:0] lc_hw_debug_en_gated;
272 prim_lc_sync #(
273   .NumCopies(int'(LcEnLastPos)),
274   .AsyncOn(0) // No synchronization required since the input
signal is already synchronous.
275 ) u_lc_en_sync_copies (
276   .clk_i,
277   .rst_ni,
278   .lc_en_i(lc_ctrl_pkg::lc_tx_t'(lc_hw_debug_en_gated_raw)),
279   .lc_en_o(lc_hw_debug_en_gated)
280 );
281
282 // Keep debug gating asserted during an active NDM reset by
latching the
283 // current LC debug enable and ORing it into the gating
while the reset is
284 // pending. Clear the latch once LC initialization has com-
pleted again.
285 lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_latched_q;
286 lc_ctrl_pkg::lc_tx_t lc_init_done_sync;
287 prim_lc_sync #(
288   .NumCopies(1)
289 ) u_lc_init_done_sync (
290   .clk_i,
291   .rst_ni,
292   .lc_en_i(lc_init_done_i),
293   .lc_en_o{lc_init_done_sync})
294 );
295
296 // Version of lc_hw_debug_en_gated with NDM latch applied
(defined below)
297 lc_ctrl_pkg::lc_tx_t [LcEnLastPos-1:0] lc_hw_de-
bug_en_gated_ndm;
298
299 dm::dmi_req_t dmi_req;
300 dm::dmi_rsp_t dmi_rsp;
301 logic dmi_req_valid, dmi_req_ready;
302 logic dmi_rsp_valid, dmi_rsp_ready;
303 logic dmi_rst_n;
304
305 ///////////////////////////////////////////////////
306 // NDM Reset Tracking //

```

```

307 ///////////////////////////////////////////////////
308
309 logic reset_req_en;
310 logic ndmreset_req, ndmreset_ack;
311 logic ndmreset_req_qual;
312 // SEC_CM: DM_EN.CTRL.LC_GATED
313 assign reset_req_en =
lc_tx_test_true_strict(lc_hw_debug_en_gated_ndm[LcEnResetReq]);
314 assign ndmreset_req_o = ndmreset_req_qual & reset_req_en;
315
316 // Sample the processor reset to detect lc reset assertion.
317 logic lc_rst_asserted_async;
318 prim_flop_2sync #(
319   .Width(1),
320   .ResetValue(1) // Resets to 1 to indicate assertion.
321 ) u_prim_flop_2sync_lc_rst_assert (
322   .clk_i, // Use RV_DM clock
323   .rst_ni(rst_lc_ni), // Use LC reset here that resets the
entire system except the RV_DM.
324   .d_i(1'b0), // Set to 0 to indicate deassertion.
325   .q_o(lc_rst_asserted_async)
326 );
327

328 // Note that the output of the above flops can be metastable
at reset assertion, since the reset
329 // signal is coming from a different clock domain and has
not been synchronized with clk_i.
330 logic lc_rst_asserted;
331 prim_flop_2sync #(
332   .Width(1)
333 ) u_prim_flop_2sync_lc_rst_sync (
334   .clk_i,
335   .rst_ni,
336   .d_i(lc_rst_asserted_async),
337   .q_o(lc_rst_asserted)
338 );
339
340 // The acknowledgement pulse sets the dm-
status.allhavereset / dmstatus.anyhavereset registers in
341 // RV_DM. It should only be asserted once an NDM reset re-
quest has been fully completed.
342 logic ndmreset_pending_q;
343 logic lc_rst_pending_q;
344 always_ff @(posedge clk_i or negedge rst_ni) begin :
p_ndm_reset
345   if (!rst_ni) begin
346     ndmreset_pending_q <= 1'b0;
347     lc_rst_pending_q <= 1'b0;
348     lc_hw_debug_en_latched_q <= lc_ctrl_pkg::0ff;
349   end else begin
350     // Only set this if there was no previous pending NDM
request.
351     if (ndmreset_req && !ndmreset_pending_q) begin
352       ndmreset_pending_q <= 1'b1;
353     end
354   end
355 end

356 // Latch current LC debug enable value at the start of
NDM reset
357 lc_hw_debug_en_latched_q <= lc_hw_debug_en;
358 end else if (ndmreset_ack && ndmreset_pending_q) begin
359   ndmreset_pending_q <= 1'b0;
360 end
361 // We only track lc resets that are asserted during an
active ndm reset request..
362 if (ndmreset_pending_q && lc_rst_asserted) begin
363   lc_rst_pending_q <= 1'b1;
364 end else if (ndmreset_ack && lc_rst_pending_q) begin
365   lc_rst_pending_q <= 1'b0;
366 end
367 // Clear the latch after lifecycle initialization com-
pletes
368 if (lc_c-
trl_pkg::lc_tx_test_true_strict(lc_init_done_sync)) begin
369   lc_hw_debug_en_latched_q <= lc_ctrl_pkg::0ff;
370 end
371 end

372 // In order to ACK the following conditions must be met
373 // 1) an NDM reset request was asserted and is pending
374 // 2) a lc reset was asserted after the NDM reset request
375 // 3) the NDM reset request was deasserted
376 // 4) the NDM lc request was deasserted
377 // 5) the debug module has been ungated for operation (de-
pending on LC state, OTP config and CSR)
378 assign ndmreset_ack = ndmreset_pending_q &&
lc_rst_pending_q &&
!ndmreset_req &&
!lc_rst_asserted &&
reset_req_en;
379
380 // Apply the latched LC value only while an NDM reset is
pending.
381 for (genvar k = 0; k < int'(LcEnLastPos); k++) begin :
gen_ndm_gated
382   assign lc_hw_debug_en_gated_ndm[k] = (ndmre-
set_pending_q) ?
383     lc_ctrl_pkg::lc_tx_or_hi(lc_hw_debug_en_gated[k],
lc_hw_debug_en_latched_q) :
384     lc_hw_debug_en_gated[k];
385 end

```

```

389
390 //////////////////////////////////////////////////
391 // System Bus Access Port (TL-UL Host) //
392 //////////////////////////////////////////////////
393
394 logic                host_req;
395 logic [BusWidth-1:0] host_add;
396 logic                host_we;
397 logic [BusWidth-1:0] host_wdata;
398 logic [BusWidth/8-1:0] host_be;
399 logic                host_gnt;
400 logic                host_r_valid;
401 logic [BusWidth-1:0] host_r_rdata;
402 logic                host_r_err;
403 logic                host_r_other_err;
404
405 // SEC_CM: DM_EN.CTRL.LC_GATED
406 // SEC_CM: SBA_TL_LC_GATE.FSM.SPARSE
407 tlul_pkg::tl_h2d_t sba_tl_h_o_int;
408 tlul_pkg::tl_d2h_t sba_tl_h_i_int;
409 tlul_lc_gate #(
410     .NumGatesPerDirection(2)
411 ) u_tlul_lc_gate_sba (
412     .clk_i,
413     .rst_ni,
414     .tl_h2d_i(sba_tl_h_o_int),
415     .tl_d2h_o(sba_tl_h_i_int),
416     .tl_h2d_o(sba_tl_h_o),
417     .tl_d2h_i(sba_tl_h_i),
418     .lc_en_i (lc_hw_debug_en_gated_ndm[LcEnSba]),
419     .err_o   (sba_gate_intg_error),
420     .flush_req_i('0),
421     .flush_ack_o(),
422     .resp_pending_o()
423 );
424
425 tlul_adapter_host #(
426     .MAX_REQS(1),
427     .EnableDataIntgGen(1),
428     .EnableRspDataIntgCheck(1)
429 ) t_adapter_host_sba (
430     .clk_i,
431     .rst_ni,
432     .req_i   (host_req),
433     .instr_type_i (prim_mubi_pkg::MuBi4False),
434     .gnt_o   (host_gnt),
435     .addr_i   (host_add),
436     .we_i     (host_we),
437     .wdata_i   (host_wdata),
438     .wdata_intg_i ('0),
439     .be_i     (host_be),
440     .user_rsvd_i (TlulHostUserRsvdBits),
441     .valid_o   (host_r_valid),
442     .rdata_o   (host_r_rdata),
443     .rdata_intg_o (),
444     .err_o     (host_r_err),
445     // Note: This bus integrity error is not connected to the
446     // alert due to a few reasons:
447     // 1) the SBA module is not active in production life
448     // cycle states.
449     // 2) there is value in being able to accept incoming
450     // transactions with integrity
451     // errors during test / debug life cycle states so that
452     // the system can be debugged
453     // without triggering alerts.
454     // 3) the error condition is hooked up to an error CSR
455     // that can be read out by the debugger
456     // via JTAG or DMI so that bus integrity errors can be
457     // told apart from regular bus errors.
458     .intg_err_o (host_r_other_err),
459     .tl_o       (sba_tl_h_o_int),
460     .tl_i       (sba_tl_h_i_int)
461 );
462
463 //////////////////////////////////////////////////
464 // Debug Memory Port (TL-UL Device) //
465 //////////////////////////////////////////////////
466
467 logic                device_req;
468 logic                device_we;
469 logic                device_re;
470 logic [BusWidth/8-1:0] device_be;
471 logic [BusWidth-1:0] device_wdata;
472 logic [BusWidth-1:0] device_rdata;
473 logic                device_err;
474
475 logic [BusWidth-1:0] device_addr_aligned;
476 logic [MemAw-1:0] device_addr;
477
478 assign device_addr_aligned = BusWidth'(device_addr);
479
480 logic [NrHarts-1:0] debug_req_en;
481 logic [NrHarts-1:0] debug_req;
482 for (genvar i = 0; i < NrHarts; i++) begin : gen_de-
483 bug_req_hart
484     // SEC_CM: DM_EN.CTRL.LC_GATED
485     assign debug_req_en[i] =
486 lc_tx_test_true_strict(lc_hw_debug_en_gated_ndm[LcEnDebugReq + i]);
487 end
488 assign debug_req_o = debug_req & debug_req_en;

```

```

481
482 if (UseDmiInterface) begin : gen_dmi_gating
483     //////////////////////////////////////////////////
484     // TL-UL-based Debug Module Interface (DMI) //
485     //////////////////////////////////////////////////
486
487     // If DMIDirectTAP is defined, a bound-in DPI module re-
488     // places the TAP that's defined
489     // within the ifndef block
490     `ifndef DMIDirectTAP
491         tlul_pkg::tl_h2d_t dbg_tl_h2d_win;
492         tlul_pkg::tl_d2h_t dbg_tl_d2h_win;
493
494         rv_dm_dbg_reg_top u_rv_dm_dbg_reg_top (
495             .clk_i,
496             .rst_ni,
497             .tl_i   (dbg_tl_d_i),
498             .tl_o   (dbg_tl_d_o),
499             .tl_win_o (dbg_tl_h2d_win),
500             .tl_win_i (dbg_tl_d2h_win),
501             .intg_err_o(dbg_intg_error)
502         );
503
504         rv_dm_dmi_gate #(
505             .SecVolatileRawUnlockEn(SecVolatileRawUnlockEn)
506         ) u_rv_dm_dmi_gate (
507             .clk_i,
508             .rst_ni,
509             .strap_en_override_i,
510             .strap_en_i,
511             .lc_hw_debug_clr_i,
512             .lc_hw_debug_en_i,
513             .lc_check_byp_en_i,
514             .lc_escalate_en_i,
515             .dbg_tl_h2d_win_i ( dbg_tl_h2d_win ),
516             .dbg_tl_d2h_win_o ( dbg_tl_d2h_win ),
517             .dmi_req_valid_o ( dmi_req_valid ),
518             .dmi_req_ready_i ( dmi_req_ready ),
519             .dmi_req_o       ( dmi_req ),
520             .dmi_rsp_valid_i ( dmi_rsp_valid ),
521             .dmi_rsp_ready_o ( dmi_rsp_ready ),
522             .dmi_rsp_i       ( dmi_rsp ),
523             // Integrity error
524             .intg_error_o( dmi_intg_error)
525         );
526
527         // This only clears the DMI FIFO inside the dm_csrs imple-
528         // mentation.
529         // Since the JTAG DTM used in this system can always drain
530         // this FIFO,
531         // no additional reset request should be needed in order
532         // to clear it.
533         assign dmi_rst_n = rst_ni;
534         `else
535             assign dmi_intg_error = 1'b0;
536             assign dbg_tl_d_o = tlul_pkg::TL_D2H_DEFAULT;
537         `endif
538
539         // Tied-off signals from the JTAG interface and read unused
540         // signals
541         assign jtag_o = '0;
542         logic unused_signals;
543         assign unused_signals = ^{jtag_i, scan_rst_ni, pin-
544 mux_hw_debug_en_i};
545         end else begin : gen_jtag_gating
546             // Gating of JTAG signals
547             jtag_pkg::jtag_req_t jtag_in_int;
548             jtag_pkg::jtag_rsp_t jtag_out_int;
549
550             typedef enum logic [1:0] {
551                 PmEnDmiReq,
552                 PmEnJtagIn,
553                 PmEnJtagOut,
554                 PmEnLastPos
555             } rv_dm_pm_en_e;
556
557             lc_ctrl_pkg::lc_tx_t [PmEnLastPos-1:0] pinmux_hw_debug_en;
558             prim_lc_sync #(
559                 .NumCopies(int'(PmEnLastPos))
560             ) u_pm_en_sync (
561                 .clk_i,
562                 .rst_ni,
563                 .lc_en_i(pinmux_hw_debug_en_i),
564                 .lc_en_o(pinmux_hw_debug_en)
565             );
566
567             assign jtag_in_int =
568 (lc_tx_test_true_strict(pinmux_hw_debug_en[PmEnJtagIn])) ? jtag_i :
569 '0;
570             assign jtag_o =
571 (lc_tx_test_true_strict(pinmux_hw_debug_en[PmEnJtagOut])) ?
572 jtag_out_int : '0;
573
574             logic dmi_en;
575
576             // SEC_CM: DM_EN.CTRL.LC_GATED
577             assign dmi_en =
578 lc_tx_test_true_strict(pinmux_hw_debug_en[PmEnDmiReq]);

```

```

569
570 // If DMIDirectTAP is not defined, a bound-in DPI module
replaces the TAP and TL-UL DMI that
571 // is defined within the ifdef block
572 `ifndef DMIDirectTAP
573 logic tck_muxed;
574 logic trst_n_muxed;
575 logic dmi_req_valid_raw, dmi_rsp_ready_raw;
576
577 prim_clock_mux2 #(
578   .NoFpgaBufG(1'b1)
579 ) u_prim_clock_mux2 (
580   .clk0_i(jtag_in_int.tck),
581   .clk1_i(clk_i),
582   .sel_i (testmode),
583   .clk_o (tck_muxed)
584 );
585
586 prim_rst_mux2 #(
587   .NoFpgaBufG(1'b1)
588 ) u_prim_rst_n_mux2 (
589   .clk0_i(jtag_in_int.trst_n),
590   .clk1_i(scan_rst_ni),
591   .sel_i (testmode),
592   .clk_o (trst_n_muxed)
593 );
594
595 // JTAG TAP
596 dmi_jtag #(
597   .IdcodeValue (IdcodeValue),
598   .NumDmiWordAbits(7)
599 ) dap (
600   .clk_i      (clk_i),
601   .rst_ni     (rst_ni),
602   .testmode_i (testmode),
603   .test_rst_ni (scan_rst_ni),
604
605   .dmi_rst_no (dmi_rst_n),
606   .dmi_req_o  (dmi_req),
607   .dmi_req_valid_o (dmi_req_valid_raw),
608   .dmi_req_ready_i (dmi_req_ready & dmi_en),
609
610   .dmi_resp_i (dmi_rsp),
611   .dmi_resp_ready_o (dmi_rsp_ready_raw),
612   .dmi_resp_valid_i (dmi_rsp_valid & dmi_en),
613
614   //JTAG
615   .tck_i      (tck_muxed),
616   .tms_i      (jtag_in_int.tms),
617   .trst_ni    (trst_n_muxed),
618   .td_i       (jtag_in_int.tdi),
619   .td_o       (jtag_out_int.tdo),
620   .tdo_oe_o   (jtag_out_int.tdo_oe)
621 );
622
623 // Gated DMI signals
624 assign dmi_req_valid = dmi_req_valid_raw & dmi_en;
625 assign dmi_rsp_ready = dmi_rsp_ready_raw & dmi_en;
626 `endif
627 // Tied-off and ignore signals from the DMI interface
628 assign dmi_intg_error = 1'b0;
629 assign dbg_intg_error = 1'b0;
630 assign dbg_tl_d_o = tlu1_pkg::TL_D2H_DEFAULT;
631
632 logic unused_signals;
633 assign unused_signals = ^{dbg_tl_d_i,
634   lc_check_byp_en_i,
635   lc_hw_debug_clr_i,
636   lc_escalate_en_i,
637   strap_en_i,
638   strap_en_override_i};
639
640 end
641
642 // SEC_CM: DM_EN.CTRL.LC_GATED
643 // SEC_CM: MEM_TL_LC_GATE.FSM.SPARSE
644 tlu1_pkg::tl_h2d_t mem_tl_win_h2d_gated;
645 tlu1_pkg::tl_d2h_t mem_tl_win_d2h_gated;
646 tlu1_lc_gate #(
647   .NumGatesPerDirection(2)
648 ) u_tlu1_lc_gate_rom (
649   .clk_i,
650   .rst_ni,
651   .tl_h2d_i(mem_tl_win_h2d),
652   .tl_d2h_o(mem_tl_win_d2h),
653   .tl_h2d_o(mem_tl_win_h2d_gated),
654   .tl_d2h_i(mem_tl_win_d2h_gated),
655   .flush_req_i(ndmreset_req),
656   .flush_ack_o(ndmreset_req_qual),
657   .resp_pending_o(),
658   .lc_en_i (lc_hw_debug_en_gated_ndm[LcEnRom]),
659   .err_o (rom_gate_intg_error)
660 );
661
662 prim_mubi_pkg::mubi4_t en_ifetch;
663 // SEC_CM: DM_EN.CTRL.LC_GATED, EXEC.CTRL.MUBI
664 assign en_ifetch = lc_c-
trl_pkg::lc_to_mubi4(lc_hw_debug_en_gated_ndm[LcEnFetch]);
665
666 tlu1_adapter_reg #(
667   .CmdIntgCheck (1),
668   .EnableRspIntgGen (1),

```

```

668   .EnableDataIntgGen(1),
669   .RegAw (MemAw),
670   .RegDw (BusWidth),
671   .AccessLatency (1)
672 ) i_tlu1_adapter_reg (
673   .clk_i,
674   .rst_ni,
675   .tl_i (mem_tl_win_h2d_gated),
676   .tl_o (mem_tl_win_d2h_gated),
677   // SEC_CM: EXEC.CTRL.MUBI
678   .en_ifetch_i (en_ifetch),
679   // SEC_CM: BUS.INTEGRITY
680   .intg_error_o(rom_intg_error),
681   .re_o (device_re),
682   .we_o (device_we),
683   .addr_o (device_addr),
684   .wdata_o (device_wdata),
685   .be_o (device_be),
686   .busy_i (1'b0),
687   .rdata_i (device_rdata),
688   .error_i (device_err)
689 );
690
691 assign device_req = device_we || device_re;
692
693 ///////////////////////////////////////////////////////////////////
694 // Debug Module Instance //
695 ///////////////////////////////////////////////////////////////////
696
697 dm_top #(
698   .NrHarts (NrHarts),
699   .BusWidth (BusWidth),
700   .SelectableHarts(SelectableHarts),
701   // The debug module provides a simplified ROM for systems
that map the debug ROM to offset 0x0
702   // on the system bus. In that case, only one scratch re-
gister has to be implemented in the core.
703   // However, we require that the DM can be placed at arbit-
rary offsets in the system, which
704   // requires the generalized debug ROM implementation and
two scratch registers. We hence set
705   // this parameter to a non-zero value (inside dm_mem, this
just feeds into a comparison with 0).
706   .DmBaseAddress (1)
707 ) u_dm_top (
708   .clk_i,
709   .rst_ni,
710   .next_dm_addr_i,
711   .testmode_i (testmode),
712   .ndmreset_o (ndmreset_req),
713   .ndmreset_ack_i (ndmreset_ack),
714   .dmactive_o,
715   .debug_req_o (debug_req),
716   .unavailable_i,
717   .hartinfo_i (hartinfo),
718   .slave_req_i (device_req),
719   .slave_we_i (device_we),
720   .slave_addr_i (device_addr_aligned),
721   .slave_be_i (device_be),
722   .slave_wdata_i (device_wdata),
723   .slave_rdata_o (device_rdata),
724   .slave_err_o (device_err),
725   .master_req_o (host_req),
726   .master_add_o (host_add),
727   .master_we_o (host_we),
728   .master_wdata_o (host_wdata),
729   .master_be_o (host_be),
730   .master_gnt_i (host_gnt),
731   .master_r_valid_i (host_r_valid),
732   .master_r_err_i (host_r_err),
733   .master_r_other_err_i (host_r_other_err),
734   .master_r_rdata_i (host_r_rdata),
735   .dmi_rst_ni (dmi_rst_n),
736   .dmi_req_valid_i (dmi_req_valid),
737   .dmi_req_ready_o (dmi_req_ready),
738   .dmi_req_i (dmi_req),
739   .dmi_resp_valid_o (dmi_rsp_valid),
740   .dmi_resp_ready_i (dmi_rsp_ready),
741   .dmi_resp_o (dmi_rsp)
742 );
743
744 ///////////////////////////////////////////////////////////////////
745 // Assertions //
746 ///////////////////////////////////////////////////////////////////
747
748 `ASSERT_KNOWN(TlRegsDValidKnown_A, regs_tl_d_o.d_valid)
749 `ASSERT_KNOWN(TlRegsAReadyKnown_A, regs_tl_d_o.a_ready)
750
751 `ASSERT_KNOWN(TlMemDValidKnown_A, mem_tl_d_o.d_valid)
752 `ASSERT_KNOWN(TlMemAReadyKnown_A, mem_tl_d_o.a_ready)
753
754 `ASSERT_KNOWN(TlSbaAValidKnown_A, sba_tl_h_o.a_valid)
755 `ASSERT_KNOWN(TlSbaDReadyKnown_A, sba_tl_h_o.d_ready)
756
757 `ASSERT_KNOWN(TlDmiDValidKnown_A, dbg_tl_d_o.d_valid)
758 `ASSERT_KNOWN(TlDmiAReadyKnown_A, dbg_tl_d_o.a_ready)
759
760 `ASSERT_KNOWN(NdmresetOKnown_A, ndmreset_req_o)
761 `ASSERT_KNOWN(DmactiveOKnown_A, dmactive_o)
762 `ASSERT_KNOWN(DebugReqOKnown_A, debug_req_o)

```



```

763 `ASSERT_KNOWN_IF(RaclErrorKnown_A, racl_error_o, racl_er-
ror_o.valid)
764
765 if (UseDmiInterface) begin : gen_dmi_assertions
766 `ASSERT(DmiRspOneCycleAfterReq_A, dmi_req_valid |>=
dmi_rsp_valid)
767 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(DbgTlLcGateFsm_A,
gen_dmi_gat-
ing.u_rv_dm_dmi_gate.u_tlul_lc_gate_dbg.u_state_regs, alert_tx_o[0])
768 end else begin : gen_jtag_assertions
769 // JTAG TDO is driven by an inverted TCK in
dmi_jtag_tap.sv
770
771 `ASSERT_KNOWN(JtagRspOTdoKnown_A, jtag_o.tdo, !
jtag_i.tck, !jtag_i.trst_n)
772 `ASSERT_KNOWN(JtagRspOTdoOeKnown_A, jtag_o.tdo_oe, !
jtag_i.tck, !jtag_i.trst_n)
773 end
774
775 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SbaTlLcGateFsm_A,
u_tlul_lc_gate_sba.u_state_regs, alert_tx_o[0])
776
777 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(RomTlLcGateFsm_A,
u_tlul_lc_gate_rom.u_state_regs, alert_tx_o[0])
778
779 // Alert assertions for reg_we onehot check
780
781 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegsWeOnehotCheck_A,
u_reg_regs, alert_tx_o[0])
782 endmodule

```

3. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dbg_reg_top.sv (60 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module rv_dm_dbg_reg_top (
10 input clk_i,
11 input rst_ni,
12 input tlul_pkg::tl_h2d_t tl_i,
13 output tlul_pkg::tl_d2h_t tl_o,
14
15 // Output port for window
16 output tlul_pkg::tl_h2d_t tl_win_o,
17 input tlul_pkg::tl_d2h_t tl_win_i,
18
19 // To HW
20
21 // Integrity check errors
22 output logic intg_err_o
23 );
24
25 import rv_dm_reg_pkg::* ;
26
27
28 // Add an unloaded flop to make use of clock / reset
29 // This is done to specifically address lint complaints of
unused clocks/resets
30 // Since the flop is unloaded it will be removed during syn-
thesis
31 logic unused_reg;
32 always_ff @(posedge clk_i or negedge rst_ni) begin
33 if (!rst_ni) begin
34 unused_reg <= '0;
35 end else begin
36 unused_reg <= tl_i.a_valid;
37 end
38 end
39
40
41
42 // Since there are no registers in this block, commands are
routed through to windows which
43 // can report their own integrity errors.
44 assign intg_err_o = 1'b0;
45
46 // outgoing integrity generation
47 tlul_pkg::tl_d2h_t tl_o_pre;
48 tlul_rsp_intg_gen #(
49 .EnableRspIntgGen(1),
50 .EnableDataIntgGen(0)
51 ) u_rsp_intg_gen (
52 .tl_i(tl_o_pre),
53 .tl_o(tl_o)
54 );
55
56 assign tl_win_o = tl_i;
57 assign tl_o_pre = tl_win_i;
58
59 // Unused signal tieoff
60 endmodule

```

4. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_dmi_gate.sv (304 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // DMI gate for the RISC-V Debug Module
6 //
7 // This module latches the power-on strap for DM enablement
from lc_ctrl and
8 // gates the DMI if debugging is not enabled. It also monitors for
lc_ctrl
9 // events that would invalidate the strap.
10
11 `include "prim_assert.sv"
12
13 module rv_dm_dmi_gate
14 import lc_ctrl_pkg::*;
15 # (
16 parameter bit SecVolatileRawUnlockEn = 0
17 ) (
18 input clk_i,
19 input rst_ni,
20 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
21
22 // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
ATE
23 // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
24 // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
25 //
26 // Strap sampling override that is only used when SecVolat-
ileRawUnlockEn = 1, Otherwise this input
27 // is unused. This needs to be synchronized since it is com-
ing from a different clock domain.
28 // This signal goes from 0 -> 1 and then stays high, since
we only ever re-sample once. The
29 // synchronization logic can therefore just detect the edge
to create the sampling pulse
30 // internally.
31 input strap_en_override_i,
32 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
33
34 input strap_en_i,
35 // LC signals for DMI qualification
36 input lc_ctrl_pkg::lc_tx_t lc_hw_debug_clr_i,
37 input lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
38 input lc_ctrl_pkg::lc_tx_t lc_check_byp_en_i,
39 input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
40
41 // TL-UL-based DMI
42 input tlul_pkg::tl_h2d_t dbg_tl_h2d_win_i,
43 output tlul_pkg::tl_d2h_t dbg_tl_d2h_win_o,
44
45 // Gated Rocket Legacy DMI
46 output dmi_req_valid_o,
47 input dmi_req_ready_i,
48 output dmi_req_o,
49 input dmi_rsp_valid_i,
50 output dmi_rsp_ready_o,
51 input dmi_rsp_i,
52
53 // Integrity error
54 output intg_error_o
55 );
56
57 // Life cycle signal synchronizers //
58 // -----
59 lc_tx_t [0:0] lc_hw_debug_clr;
60 prim_lc_sync #(
61 .NumCopies(1)
62 ) u_prim_lc_sync_lc_hw_debug_clr (
63 .clk_i,
64 .rst_ni,
65 .lc_en_i(lc_hw_debug_clr_i),
66 .lc_en_o(lc_hw_debug_clr)
67 );
68 lc_tx_t [0:0] lc_hw_debug_en;
69 prim_lc_sync #(
70 .NumCopies(1)
71 ) u_prim_lc_sync_lc_hw_debug_en (
72 .clk_i,
73 .rst_ni,
74 .lc_en_i(lc_hw_debug_en_i),
75 .lc_en_o(lc_hw_debug_en)
76 );
77 lc_tx_t [0:0] lc_check_byp_en;
78 prim_lc_sync #(
79 .NumCopies(1)
80 ) u_prim_lc_sync_lc_check_byp_en (
81 .clk_i,
82 .rst_ni,
83 .lc_en_i(lc_check_byp_en_i),

```



```

83     .lc_en_o(lc_check_byp_en)
84 );
85 lc_tx_t [0:0] lc_escalate_en;
86 prim_lc_sync #(
87     .NumCopies(1)
88 ) u_prim_lc_sync_lc_escalate_en (
89     .clk_i,
90     .rst_ni,
91     .lc_en_i(lc_escalate_en_i),
92     .lc_en_o(lc_escalate_en)
93 );
94
95 ///////////////////////////////////////////////////
96 // Strap Sampling Logic //
97 ///////////////////////////////////////////////////
98
99 logic strap_en;
100 if (SecVolatileRawUnlockEn) begin : gen_strap_override
101     logic strap_en_override_d, strap_en_override_q;
102
103     prim_flop_2sync #(
104         .Width(1),
105         .ResetValue(0)
106     ) u_prim_flop_2sync (
107         .clk_i,
108         .rst_ni,
109         .d_i(strap_en_override_i),
110         .q_o(strap_en_override_d)
111     );
112
113     always_ff @(posedge clk_i or negedge rst_ni) begin :
p_strap_override_reg
114         if(!rst_ni) begin
115             strap_en_override_q <= 1'b0;
116         end else begin
117             strap_en_override_q <= strap_en_override_d;
118         end
119     end
120
121     // Detect a posedge on the override signal and reflect it
for just a cycle on strap_en
122     // (ignoring the fact that strap_en_override_i will stay
high afterwards).
123     assign strap_en = strap_en_i || (strap_en_override_d && !
strap_en_override_q);
124
125     // Once strap_en_override_i is asserted, it will always
stay high
126     `ASSUME(LcCtrlStrapSampleOverrideOnce_A,
strap_en_override_i | => strap_en_override_i)
127
128     end else begin : gen_no_strap_override
129         logic unused_strap_en_override;
130         assign unused_strap_en_override = strap_en_override_i;
131         assign strap_en = strap_en_i;
132     end
133
134 ///////////////////////////////////////////////////
135 // LC_HW_DEBUG_EN Latching //
136 ///////////////////////////////////////////////////
137
138 // In order to keep a RV_DM JTAG debug session alive during
NDM reset, we need to memorize the
139 // state of lc_hw_debug_en, since OTP/LC will be reset as
part of NDM reset (the only parts not
140 // reset are in the pwr/rst/clkmgrs, RV_DM and this strap
sampling module). We sample the life
141 // cycle signal state when the strap sampling pulse is as-
serted but the PWRMGR. This pulse is
142 // asserted once during boot (and not after an NDM reset).
143 //
144 // Note that we make sure to invalidate the sampled
lc_hw_debug_en whenever lc_check_byp_en or
145 // lc_escalate_en are not OFF. lc_escalate_en is asserted as
part of an escalation, and
146 // lc_check_byp_en is asserted whenever a life cycle trans-
ition is initiated (it causes the OTP
147 // controller to skip background checks on the life cycle
partition as it undergoes
148 // modification). This makes sure that the sampled value
here does not survive a life cycle
149 // transition.
150 //
151 // Finally, note that there is secondary gating on the RV_DM
and DFT TAPS that is always consuming
152 // live lc_hw_debug_en and lc_dft_en signals for added pro-
tection.
153
154 // Convert the strap enable pulse to a mubi signal and mask
lc_hw_debug_en with it.
155 lc_tx_t lc_strap_en, lc_hw_debug_en_masked;
156 assign lc_strap_en = lc_tx_bool_to_lc_tx(strap_en);
157 assign lc_hw_debug_en_masked = lc_tx_and_hi(lc_strap_en,
lc_hw_debug_en[0]);
158
159 // Output ON if
160 // - If the strap sampling pulse is asserted and lc_hw-de-
bug_en is ON
161 // - If the strap_hw_debug_en_q is already set to ON (this

```

is the latching feedback loop)

```

162 // Note: make sure we use a hardened, rectifying OR function
since otherwise two non-strict
163 // values may produce a strict ON value.
164 lc_tx_t hw_debug_en_set, strap_hw_debug_en_q;
165 prim_lc_or_hardenen #(
166     .ActVal(On)
167 ) u_prim_lc_or_hardenen (
168     .clk_i,
169     .rst_ni,
170     .lc_en_a_i(lc_hw_debug_en_masked),
171     .lc_en_b_i(strap_hw_debug_en_q),
172     .lc_en_o (hw_debug_en_set)
173 );
174
175 // Output ON if lc_check_byp_en AND lc_escalate_en are
strictly OFF; otherwise output a non-ON
176 // value (which may be different from OFF).
177 lc_tx_t neither_check_byp_nor_escalate;
178 assign neither_check_byp_nor_escalate =
lc_tx_inv(lc_tx_and_lo(lc_check_byp_en[0],
179 lc_escalate_en[0]));
180
181 // Output ON if the signal above is strictly ON AND
lc_hw_debug_clr is strictly OFF; otherwise
182 // output a non-ON value (which may be different from OFF).
183 lc_tx_t hw_debug_en_gating;
184 assign hw_debug_en_gating =
lc_tx_and_hi(neither_check_byp_nor_escalate,
185 lc_tx_inv(lc_hw_de-
bug_clr[0]));
186
187 // Gate the hw_debug_en_set signal and feed it into the
latching flop.
188 lc_tx_t strap_hw_debug_en_d;
189 assign strap_hw_debug_en_d = lc_tx_and_hi(hw_debug_en_set,
hw_debug_en_gating);
190
191 prim_lc_sender u_prim_lc_sender_strap_hw_debug_en (
192     .clk_i,
193     .rst_ni,
194     .lc_en_i(strap_hw_debug_en_d),
195     .lc_en_o(strap_hw_debug_en_q)
196 );
197
198 typedef enum logic [1:0] {
199     StEnDmiTlulAdapter,
200     StEnDmiTlulLcGate,
201     StEnDmiTlulLast
202 } strap_hw_debug_en_e;
203
204 lc_tx_t [StEnDmiTlulLast-1:0] strap_hw_debug_en;
205 prim_lc_sync #(
206     .NumCopies(int'(StEnDmiTlulLast)),
207     .AsyncOn(0) // no sync needed
208 ) u_prim_lc_sync_strap_hw_debug_en (
209     .clk_i,
210     .rst_ni,
211     .lc_en_i(strap_hw_debug_en_q),
212     .lc_en_o(strap_hw_debug_en)
213 );
214
215 // SEC_CM: DM_EN.CTRL.LC_GATED
216 logic dmi_en;
217 assign dmi_en =
lc_tx_test_true_strict(strap_hw_debug_en[StEnDmiTlulAdapter]);
218
219 ///////////////////////////////////////////////////
220 // TL-UL to Rocket Legacy DMI //
221 ///////////////////////////////////////////////////
222
223 logic dbg_gate_intg_error, dmi_intg_error;
224 assign intg_error_o = dbg_gate_intg_error | dmi_intg_error;
225
226 logic tlul_resp_pending;
227
228 tlul_pkg::tl_h2d_t dbg_tl_h2d_gated;
229 tlul_pkg::tl_d2h_t dbg_tl_d2h_gated;
230 tlul_lc_gate #(
231     // If the DMI side is gated, the spec requires the DM to
return a valid all-zero response.
232     .ReturnBlankResp(1)
233 ) u_tlul_lc_gate_dbg (
234     .clk_i,
235     .rst_ni,
236     .tl_h2d_i (dbg_tl_h2d_win_i),
237     .tl_d2h_o (dbg_tl_d2h_win_o),
238     .tl_h2d_o (dbg_tl_h2d_gated),
239     .tl_d2h_i (dbg_tl_d2h_gated),
240
241 // The DMI side should not be flushed upon NDM-reset since
we keep the
242 // debugger side of the connection stable during an NDM-
reset request.
243     .flush_req_i (1'b0),
244     .flush_ack_o (),
245     .resp_pending_o(tlul_resp_pending),
246     .lc_en_i (strap_hw_debug_en[StEnDmiTlulLcGate]),

```

```

246     .err_o          (dbg_gate_intg_error)
247 );
248
249 logic dmi_req_ready, dmi_rsp_valid;
250 tlul_adapter_dmi u_tlul_adapter_dmi (
251     .clk_i,
252     .rst_ni,
253     .tl_h2d_i        (dbg_tl_h2d_gated),
254     .tl_d2h_o        (dbg_tl_d2h_gated),
255     .intg_error_o     (dmi_intg_error),
256     .dmi_req_valid_o  (dmi_req_valid_o),
257     .dmi_req_ready_i  (dmi_req_ready),
258     .dmi_req_o        (dmi_req_o),
259     .dmi_resp_valid_i (dmi_rsp_valid),
260     .dmi_resp_ready_o (dmi_rsp_ready_o),
261     .dmi_resp_i       (dmi_rsp_i)
262 );
263
264 // Connect the TL-UL adapter to the DMI handshake if the DMI
is enabled. Let responses also
265 // propagate from the DMI to TL-UL if any responses are
pending.
266 assign dmi_req_ready = dmi_req_ready_i & dmi_en;
267 assign dmi_rsp_valid = dmi_rsp_valid_i & (dmi_en |
tlul_resp_pending);
268
269 ///////////////////////////////////////////////////////////////////
270 // Assertions //
271 ///////////////////////////////////////////////////////////////////
272
273 // Check that we can correctly latch upon strap_en_i
274 `ASSERT(LcHwDebugEnSet_A,
275     (lc_tx_test_true_strict(lc_hw_debug_en[0]) ||
276      lc_tx_test_true_strict(strap_hw_debug_en_q)) &&
277      lc_tx_test_false_strict(lc_hw_debug_clr[0]) &&
278      lc_tx_test_false_strict(lc_check_byp_en[0]) &&
279      lc_tx_test_false_strict(lc_escalate_en[0]) &&
280      strap_en_i
281      |>
282      lc_tx_test_true_strict(strap_hw_debug_en_q))
283
284 // Check that latching ON can only occur if lc_hw_debug_en_i
is set.
285 `ASSERT(LcHwDebugEnSetRev0_A,
286     lc_tx_test_false_loose(strap_hw_debug_en_q) ##1
287     lc_tx_test_true_strict(strap_hw_debug_en_q)
288     |>
289     $past(lc_tx_test_true_strict(lc_hw_debug_en[0])))
290 // Check that latching ON can only occur if strap_en is set.
291 `ASSERT(LcHwDebugEnSetRev1_A,
292     lc_tx_test_false_loose(strap_hw_debug_en_q) ##1
293     lc_tx_test_true_strict(strap_hw_debug_en_q)
294     |>
295     $past(strap_en))
296 // Check that any non-OFF value on lc_check_byp_en_i and
297 // lc_escalate_en_i clears the latched value.
298 `ASSERT(LcHwDebugEnClear_A,
299     lc_tx_test_true_loose(lc_check_byp_en[0]) ||
300     lc_tx_test_true_loose(lc_escalate_en[0]) ||
301     lc_tx_test_true_loose(lc_hw_debug_clr[0])
302     |>
303     lc_tx_test_false_loose(strap_hw_debug_en_q))
304 endmodule

```

5. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_pkg.sv (12 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package rv_dm_pkg;
7
8     typedef logic [31:0] next_dm_addr_t;
9
10     parameter next_dm_addr_t NEXT_DM_ADDR_DEFAULT = '0;
11
12 endpackage : rv_dm_pkg

```

6. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_reg_pkg.sv (1,002 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
structure
6
7 package rv_dm_reg_pkg;
8
9     // Param list

```

```

10     parameter int NrHarts = 1;
11     parameter int NumAlerts = 1;
12
13     // Address widths within the block
14     parameter int RegsAw = 4;
15     parameter int MemAw = 12;
16     parameter int DbgAw = 9;
17
18     // Number of registers for every interface
19     parameter int NumRegsRegs = 3;
20     parameter int NumRegsMem = 281;
21     parameter int NumRegsDbg = 0;
22
23     // Alert indices
24     typedef enum int {
25         AlertFatalFaultIdx = 0
26     } rv_dm_alert_idx_t;
27
28     ///////////////////////////////////////////////////////////////////
29     // Typedefs for registers for regs interface //
30     ///////////////////////////////////////////////////////////////////
31
32     typedef struct packed {
33         logic q;
34         logic qe;
35     } rv_dm_reg2hw_alert_test_reg_t;
36
37     typedef struct packed {
38         logic [31:0] q;
39     } rv_dm_reg2hw_late_debug_enable_reg_t;
40
41     // Register -> HW type for regs interface
42     typedef struct packed {
43         rv_dm_reg2hw_alert_test_reg_t alert_test; // [33:32]
44
45         rv_dm_reg2hw_late_debug_enable_reg_t late_debug_enable; //
[31:0]
46     } rv_dm_regs_reg2hw_t;
47
48     // Register offsets for regs interface
49     parameter logic [RegsAw-1:0] RV_DM_ALERT_TEST_OFFSET = 4'h
0;
50     parameter logic [RegsAw-1:0] RV_DM_ALERT_TEST_RESVAL = 1'h 0;
51     parameter logic [RegsAw-1:0] RV_DM_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
52
53     // Reset values for hwext registers and their fields for
regs interface
54     parameter logic [0:0] RV_DM_ALERT_TEST_RESVAL = 1'h 0;
55     parameter logic [0:0] RV_DM_ALERT_TEST_FATAL_FAULT_RESVAL =
1'h 0;
56
57     // Register index for regs interface
58     typedef enum int {
59         RV_DM_ALERT_TEST,
60         RV_DM_LATE_DEBUG_ENABLE_REGWEN,
61         RV_DM_LATE_DEBUG_ENABLE
62     } rv_dm_regs_id_e;
63
64     // Register width information to check illegal writes for
regs interface
65     parameter logic [3:0] RV_DM_REGS_PERMIT [3] = '{
66         4'b 0001, // index[0] RV_DM_ALERT_TEST
67         4'b 0001, // index[1] RV_DM_LATE_DEBUG_ENABLE_REGWEN
68         4'b 1111 // index[2] RV_DM_LATE_DEBUG_ENABLE
69     };
70
71     ///////////////////////////////////////////////////////////////////
72     // Typedefs for registers for mem interface //
73     ///////////////////////////////////////////////////////////////////
74
75     typedef struct packed {
76         logic q;
77     } rv_dm_reg2hw_halted_reg_t;
78
79     typedef struct packed {
80         logic q;
81     } rv_dm_reg2hw_going_reg_t;
82
83     typedef struct packed {
84         logic q;
85     } rv_dm_reg2hw_resuming_reg_t;
86
87     typedef struct packed {
88         logic q;
89     } rv_dm_reg2hw_exception_reg_t;
90
91     typedef struct packed {
92         logic [31:0] q;
93     } rv_dm_reg2hw_where_to_reg_t;
94
95     typedef struct packed {
96         logic [31:0] q;
97     } rv_dm_reg2hw_abstractcmd_mreg_t;
98
99     typedef struct packed {
100         logic [31:0] q;
101     } rv_dm_reg2hw_program_buffer_mreg_t;
102
103     typedef struct packed {

```

```

103     logic [31:0] q;
104 } rv_dm_reg2hw_dataaddr_mreg_t;
105
106 typedef struct packed {
107     logic [31:0] q;
108 } rv_dm_reg2hw_flags_mreg_t;
109
110 typedef struct packed {
111     logic d;
112     logic de;
113 } rv_dm_hw2reg_halted_reg_t;
114
115 typedef struct packed {
116     logic [31:0] d;
117     logic de;
118 } rv_dm_hw2reg_where_to_reg_t;
119
120 // Register -> HW type for mem interface
121 typedef struct packed {
122     rv_dm_reg2hw_halted_reg_t halted; // [8867:8867]
123     rv_dm_reg2hw_going_reg_t going; // [8866:8866]
124     rv_dm_reg2hw_resuming_reg_t resuming; // [8865:8865]
125     rv_dm_reg2hw_exception_reg_t exception; // [8864:8864]
126     rv_dm_reg2hw_where_to_reg_t where_to; // [8863:8832]
127     rv_dm_reg2hw_abstractcmd_mreg_t [9:0] abstractcmd; //
[8831:8512]
128     rv_dm_reg2hw_program_buffer_mreg_t [7:0] pro-
gram_buffer; // [8511:8256]
129     rv_dm_reg2hw_dataaddr_mreg_t [1:0] dataaddr; //
[8255:8192]
130     rv_dm_reg2hw_flags_mreg_t [255:0] flags; // [8191:0]
131 } rv_dm_mem_reg2hw_t;
132
133 // HW -> register type for mem interface
134 typedef struct packed {
135     rv_dm_hw2reg_halted_reg_t halted; // [34:33]
136     rv_dm_hw2reg_where_to_reg_t where_to; // [32:0]
137 } rv_dm_mem_hw2reg_t;
138
139 // Register offsets for mem interface
140 parameter logic [MemAw-1:0] RV_DM_HALTED_OFFSET = 12'h 100;
141 parameter logic [MemAw-1:0] RV_DM_GOING_OFFSET = 12'h 108;
142 parameter logic [MemAw-1:0] RV_DM_RESUMING_OFFSET = 12'h
110;
143 parameter logic [MemAw-1:0] RV_DM_EXCEPTION_OFFSET = 12'h
118;
144 parameter logic [MemAw-1:0] RV_DM_WHERETO_OFFSET = 12'h 300;
145 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_0_OFFSET =
12'h 338;
146 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_1_OFFSET =
12'h 33c;
147 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_2_OFFSET =
12'h 340;
148 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_3_OFFSET =
12'h 344;
149 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_4_OFFSET =
12'h 348;
150 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_5_OFFSET =
12'h 34c;
151 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_6_OFFSET =
12'h 350;
152 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_7_OFFSET =
12'h 354;
153 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_8_OFFSET =
12'h 358;
154 parameter logic [MemAw-1:0] RV_DM_ABSTRACTCMD_9_OFFSET =
12'h 35c;
155 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_0_OFFSET =
12'h 360;
156 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_1_OFFSET =
12'h 364;
157 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_2_OFFSET =
12'h 368;
158 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_3_OFFSET =
12'h 36c;
159 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_4_OFFSET =
12'h 370;
160 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_5_OFFSET =
12'h 374;
161 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_6_OFFSET =
12'h 378;
162 parameter logic [MemAw-1:0] RV_DM_PROGRAM_BUFFER_7_OFFSET =
12'h 37c;
163 parameter logic [MemAw-1:0] RV_DM_DATAADDR_0_OFFSET = 12'h
380;
164 parameter logic [MemAw-1:0] RV_DM_DATAADDR_1_OFFSET = 12'h
384;
165 parameter logic [MemAw-1:0] RV_DM_FLAGS_0_OFFSET = 12'h 400;
166 parameter logic [MemAw-1:0] RV_DM_FLAGS_1_OFFSET = 12'h 404;
167 parameter logic [MemAw-1:0] RV_DM_FLAGS_2_OFFSET = 12'h 408;
168 parameter logic [MemAw-1:0] RV_DM_FLAGS_3_OFFSET = 12'h 40c;
169 parameter logic [MemAw-1:0] RV_DM_FLAGS_4_OFFSET = 12'h 410;
170 parameter logic [MemAw-1:0] RV_DM_FLAGS_5_OFFSET = 12'h 414;
171 parameter logic [MemAw-1:0] RV_DM_FLAGS_6_OFFSET = 12'h 418;
172 parameter logic [MemAw-1:0] RV_DM_FLAGS_7_OFFSET = 12'h 41c;
173 parameter logic [MemAw-1:0] RV_DM_FLAGS_8_OFFSET = 12'h 420;
174 parameter logic [MemAw-1:0] RV_DM_FLAGS_9_OFFSET = 12'h 424;
175 parameter logic [MemAw-1:0] RV_DM_FLAGS_10_OFFSET = 12'h
428;
176 parameter logic [MemAw-1:0] RV_DM_FLAGS_11_OFFSET = 12'h
42c;

```

```

177 parameter logic [MemAw-1:0] RV_DM_FLAGS_12_OFFSET = 12'h
430;
178 parameter logic [MemAw-1:0] RV_DM_FLAGS_13_OFFSET = 12'h
434;
179 parameter logic [MemAw-1:0] RV_DM_FLAGS_14_OFFSET = 12'h
438;
180 parameter logic [MemAw-1:0] RV_DM_FLAGS_15_OFFSET = 12'h
43c;
181 parameter logic [MemAw-1:0] RV_DM_FLAGS_16_OFFSET = 12'h
440;
182 parameter logic [MemAw-1:0] RV_DM_FLAGS_17_OFFSET = 12'h
444;
183 parameter logic [MemAw-1:0] RV_DM_FLAGS_18_OFFSET = 12'h
448;
184 parameter logic [MemAw-1:0] RV_DM_FLAGS_19_OFFSET = 12'h
44c;
185 parameter logic [MemAw-1:0] RV_DM_FLAGS_20_OFFSET = 12'h
450;
186 parameter logic [MemAw-1:0] RV_DM_FLAGS_21_OFFSET = 12'h
454;
187 parameter logic [MemAw-1:0] RV_DM_FLAGS_22_OFFSET = 12'h
458;
188 parameter logic [MemAw-1:0] RV_DM_FLAGS_23_OFFSET = 12'h
45c;
189 parameter logic [MemAw-1:0] RV_DM_FLAGS_24_OFFSET = 12'h
460;
190 parameter logic [MemAw-1:0] RV_DM_FLAGS_25_OFFSET = 12'h
464;
191 parameter logic [MemAw-1:0] RV_DM_FLAGS_26_OFFSET = 12'h
468;
192 parameter logic [MemAw-1:0] RV_DM_FLAGS_27_OFFSET = 12'h
46c;
193 parameter logic [MemAw-1:0] RV_DM_FLAGS_28_OFFSET = 12'h
470;
194 parameter logic [MemAw-1:0] RV_DM_FLAGS_29_OFFSET = 12'h
474;
195 parameter logic [MemAw-1:0] RV_DM_FLAGS_30_OFFSET = 12'h
478;
196 parameter logic [MemAw-1:0] RV_DM_FLAGS_31_OFFSET = 12'h
47c;
197 parameter logic [MemAw-1:0] RV_DM_FLAGS_32_OFFSET = 12'h
480;
198 parameter logic [MemAw-1:0] RV_DM_FLAGS_33_OFFSET = 12'h
484;
199 parameter logic [MemAw-1:0] RV_DM_FLAGS_34_OFFSET = 12'h
488;
200 parameter logic [MemAw-1:0] RV_DM_FLAGS_35_OFFSET = 12'h
48c;
201 parameter logic [MemAw-1:0] RV_DM_FLAGS_36_OFFSET = 12'h
490;
202 parameter logic [MemAw-1:0] RV_DM_FLAGS_37_OFFSET = 12'h
494;
203 parameter logic [MemAw-1:0] RV_DM_FLAGS_38_OFFSET = 12'h
498;
204 parameter logic [MemAw-1:0] RV_DM_FLAGS_39_OFFSET = 12'h
49c;
205 parameter logic [MemAw-1:0] RV_DM_FLAGS_40_OFFSET = 12'h
4a0;
206 parameter logic [MemAw-1:0] RV_DM_FLAGS_41_OFFSET = 12'h
4a4;
207 parameter logic [MemAw-1:0] RV_DM_FLAGS_42_OFFSET = 12'h
4a8;
208 parameter logic [MemAw-1:0] RV_DM_FLAGS_43_OFFSET = 12'h
4ac;
209 parameter logic [MemAw-1:0] RV_DM_FLAGS_44_OFFSET = 12'h
4b0;
210 parameter logic [MemAw-1:0] RV_DM_FLAGS_45_OFFSET = 12'h
4b4;
211 parameter logic [MemAw-1:0] RV_DM_FLAGS_46_OFFSET = 12'h
4b8;
212 parameter logic [MemAw-1:0] RV_DM_FLAGS_47_OFFSET = 12'h
4bc;
213 parameter logic [MemAw-1:0] RV_DM_FLAGS_48_OFFSET = 12'h
4c0;
214 parameter logic [MemAw-1:0] RV_DM_FLAGS_49_OFFSET = 12'h
4c4;
215 parameter logic [MemAw-1:0] RV_DM_FLAGS_50_OFFSET = 12'h
4c8;
216 parameter logic [MemAw-1:0] RV_DM_FLAGS_51_OFFSET = 12'h
4cc;
217 parameter logic [MemAw-1:0] RV_DM_FLAGS_52_OFFSET = 12'h
4d0;
218 parameter logic [MemAw-1:0] RV_DM_FLAGS_53_OFFSET = 12'h
4d4;
219 parameter logic [MemAw-1:0] RV_DM_FLAGS_54_OFFSET = 12'h
4d8;
220 parameter logic [MemAw-1:0] RV_DM_FLAGS_55_OFFSET = 12'h
4dc;
221 parameter logic [MemAw-1:0] RV_DM_FLAGS_56_OFFSET = 12'h
4e0;
222 parameter logic [MemAw-1:0] RV_DM_FLAGS_57_OFFSET = 12'h
4e4;
223 parameter logic [MemAw-1:0] RV_DM_FLAGS_58_OFFSET = 12'h
4e8;
224 parameter logic [MemAw-1:0] RV_DM_FLAGS_59_OFFSET = 12'h
4ec;
225 parameter logic [MemAw-1:0] RV_DM_FLAGS_60_OFFSET = 12'h
4f0;
226 parameter logic [MemAw-1:0] RV_DM_FLAGS_61_OFFSET = 12'h
4f4;
227 parameter logic [MemAw-1:0] RV_DM_FLAGS_62_OFFSET = 12'h

```



```

68c;
329 parameter logic [MemAw-1:0] RV_DM_FLAGS_164_OFFSET = 12'h
690;
330 parameter logic [MemAw-1:0] RV_DM_FLAGS_165_OFFSET = 12'h
694;
331 parameter logic [MemAw-1:0] RV_DM_FLAGS_166_OFFSET = 12'h
698;
332 parameter logic [MemAw-1:0] RV_DM_FLAGS_167_OFFSET = 12'h
69c;
333 parameter logic [MemAw-1:0] RV_DM_FLAGS_168_OFFSET = 12'h
6a0;
334 parameter logic [MemAw-1:0] RV_DM_FLAGS_169_OFFSET = 12'h
6a4;
335 parameter logic [MemAw-1:0] RV_DM_FLAGS_170_OFFSET = 12'h
6a8;
336 parameter logic [MemAw-1:0] RV_DM_FLAGS_171_OFFSET = 12'h
6ac;
337 parameter logic [MemAw-1:0] RV_DM_FLAGS_172_OFFSET = 12'h
6b0;
338 parameter logic [MemAw-1:0] RV_DM_FLAGS_173_OFFSET = 12'h
6b4;
339 parameter logic [MemAw-1:0] RV_DM_FLAGS_174_OFFSET = 12'h
6b8;
340 parameter logic [MemAw-1:0] RV_DM_FLAGS_175_OFFSET = 12'h
6bc;
341 parameter logic [MemAw-1:0] RV_DM_FLAGS_176_OFFSET = 12'h
6c0;
342 parameter logic [MemAw-1:0] RV_DM_FLAGS_177_OFFSET = 12'h
6c4;
343 parameter logic [MemAw-1:0] RV_DM_FLAGS_178_OFFSET = 12'h
6c8;
344 parameter logic [MemAw-1:0] RV_DM_FLAGS_179_OFFSET = 12'h
6cc;
345 parameter logic [MemAw-1:0] RV_DM_FLAGS_180_OFFSET = 12'h
6d0;
346 parameter logic [MemAw-1:0] RV_DM_FLAGS_181_OFFSET = 12'h
6d4;
347 parameter logic [MemAw-1:0] RV_DM_FLAGS_182_OFFSET = 12'h
6d8;
348 parameter logic [MemAw-1:0] RV_DM_FLAGS_183_OFFSET = 12'h
6dc;
349 parameter logic [MemAw-1:0] RV_DM_FLAGS_184_OFFSET = 12'h
6e0;
350 parameter logic [MemAw-1:0] RV_DM_FLAGS_185_OFFSET = 12'h
6e4;
351 parameter logic [MemAw-1:0] RV_DM_FLAGS_186_OFFSET = 12'h
6e8;
352 parameter logic [MemAw-1:0] RV_DM_FLAGS_187_OFFSET = 12'h
6ec;
353 parameter logic [MemAw-1:0] RV_DM_FLAGS_188_OFFSET = 12'h
6f0;
354 parameter logic [MemAw-1:0] RV_DM_FLAGS_189_OFFSET = 12'h
6f4;
355 parameter logic [MemAw-1:0] RV_DM_FLAGS_190_OFFSET = 12'h
6f8;
356 parameter logic [MemAw-1:0] RV_DM_FLAGS_191_OFFSET = 12'h
6fc;
357 parameter logic [MemAw-1:0] RV_DM_FLAGS_192_OFFSET = 12'h
700;
358 parameter logic [MemAw-1:0] RV_DM_FLAGS_193_OFFSET = 12'h
704;
359 parameter logic [MemAw-1:0] RV_DM_FLAGS_194_OFFSET = 12'h
708;
360 parameter logic [MemAw-1:0] RV_DM_FLAGS_195_OFFSET = 12'h
70c;
361 parameter logic [MemAw-1:0] RV_DM_FLAGS_196_OFFSET = 12'h
710;
362 parameter logic [MemAw-1:0] RV_DM_FLAGS_197_OFFSET = 12'h
714;
363 parameter logic [MemAw-1:0] RV_DM_FLAGS_198_OFFSET = 12'h
718;
364 parameter logic [MemAw-1:0] RV_DM_FLAGS_199_OFFSET = 12'h
71c;
365 parameter logic [MemAw-1:0] RV_DM_FLAGS_200_OFFSET = 12'h
720;
366 parameter logic [MemAw-1:0] RV_DM_FLAGS_201_OFFSET = 12'h
724;
367 parameter logic [MemAw-1:0] RV_DM_FLAGS_202_OFFSET = 12'h
728;
368 parameter logic [MemAw-1:0] RV_DM_FLAGS_203_OFFSET = 12'h
72c;
369 parameter logic [MemAw-1:0] RV_DM_FLAGS_204_OFFSET = 12'h
730;
370 parameter logic [MemAw-1:0] RV_DM_FLAGS_205_OFFSET = 12'h
734;
371 parameter logic [MemAw-1:0] RV_DM_FLAGS_206_OFFSET = 12'h
738;
372 parameter logic [MemAw-1:0] RV_DM_FLAGS_207_OFFSET = 12'h
73c;
373 parameter logic [MemAw-1:0] RV_DM_FLAGS_208_OFFSET = 12'h
740;
374 parameter logic [MemAw-1:0] RV_DM_FLAGS_209_OFFSET = 12'h
744;
375 parameter logic [MemAw-1:0] RV_DM_FLAGS_210_OFFSET = 12'h
748;
376 parameter logic [MemAw-1:0] RV_DM_FLAGS_211_OFFSET = 12'h
74c;
377 parameter logic [MemAw-1:0] RV_DM_FLAGS_212_OFFSET = 12'h
750;
378 parameter logic [MemAw-1:0] RV_DM_FLAGS_213_OFFSET = 12'h
754;

```

```

379 parameter logic [MemAw-1:0] RV_DM_FLAGS_214_OFFSET = 12'h
758;
380 parameter logic [MemAw-1:0] RV_DM_FLAGS_215_OFFSET = 12'h
75c;
381 parameter logic [MemAw-1:0] RV_DM_FLAGS_216_OFFSET = 12'h
760;
382 parameter logic [MemAw-1:0] RV_DM_FLAGS_217_OFFSET = 12'h
764;
383 parameter logic [MemAw-1:0] RV_DM_FLAGS_218_OFFSET = 12'h
768;
384 parameter logic [MemAw-1:0] RV_DM_FLAGS_219_OFFSET = 12'h
76c;
385 parameter logic [MemAw-1:0] RV_DM_FLAGS_220_OFFSET = 12'h
770;
386 parameter logic [MemAw-1:0] RV_DM_FLAGS_221_OFFSET = 12'h
774;
387 parameter logic [MemAw-1:0] RV_DM_FLAGS_222_OFFSET = 12'h
778;
388 parameter logic [MemAw-1:0] RV_DM_FLAGS_223_OFFSET = 12'h
77c;
389 parameter logic [MemAw-1:0] RV_DM_FLAGS_224_OFFSET = 12'h
780;
390 parameter logic [MemAw-1:0] RV_DM_FLAGS_225_OFFSET = 12'h
784;
391 parameter logic [MemAw-1:0] RV_DM_FLAGS_226_OFFSET = 12'h
788;
392 parameter logic [MemAw-1:0] RV_DM_FLAGS_227_OFFSET = 12'h
78c;
393 parameter logic [MemAw-1:0] RV_DM_FLAGS_228_OFFSET = 12'h
790;
394 parameter logic [MemAw-1:0] RV_DM_FLAGS_229_OFFSET = 12'h
794;
395 parameter logic [MemAw-1:0] RV_DM_FLAGS_230_OFFSET = 12'h
798;
396 parameter logic [MemAw-1:0] RV_DM_FLAGS_231_OFFSET = 12'h
79c;
397 parameter logic [MemAw-1:0] RV_DM_FLAGS_232_OFFSET = 12'h
7a0;
398 parameter logic [MemAw-1:0] RV_DM_FLAGS_233_OFFSET = 12'h
7a4;
399 parameter logic [MemAw-1:0] RV_DM_FLAGS_234_OFFSET = 12'h
7a8;
400 parameter logic [MemAw-1:0] RV_DM_FLAGS_235_OFFSET = 12'h
7ac;
401 parameter logic [MemAw-1:0] RV_DM_FLAGS_236_OFFSET = 12'h
7b0;
402 parameter logic [MemAw-1:0] RV_DM_FLAGS_237_OFFSET = 12'h
7b4;
403 parameter logic [MemAw-1:0] RV_DM_FLAGS_238_OFFSET = 12'h
7b8;
404 parameter logic [MemAw-1:0] RV_DM_FLAGS_239_OFFSET = 12'h
7bc;
405 parameter logic [MemAw-1:0] RV_DM_FLAGS_240_OFFSET = 12'h
7c0;
406 parameter logic [MemAw-1:0] RV_DM_FLAGS_241_OFFSET = 12'h
7c4;
407 parameter logic [MemAw-1:0] RV_DM_FLAGS_242_OFFSET = 12'h
7c8;
408 parameter logic [MemAw-1:0] RV_DM_FLAGS_243_OFFSET = 12'h
7cc;
409 parameter logic [MemAw-1:0] RV_DM_FLAGS_244_OFFSET = 12'h
7d0;
410 parameter logic [MemAw-1:0] RV_DM_FLAGS_245_OFFSET = 12'h
7d4;
411 parameter logic [MemAw-1:0] RV_DM_FLAGS_246_OFFSET = 12'h
7d8;
412 parameter logic [MemAw-1:0] RV_DM_FLAGS_247_OFFSET = 12'h
7dc;
413 parameter logic [MemAw-1:0] RV_DM_FLAGS_248_OFFSET = 12'h
7e0;
414 parameter logic [MemAw-1:0] RV_DM_FLAGS_249_OFFSET = 12'h
7e4;
415 parameter logic [MemAw-1:0] RV_DM_FLAGS_250_OFFSET = 12'h
7e8;
416 parameter logic [MemAw-1:0] RV_DM_FLAGS_251_OFFSET = 12'h
7ec;
417 parameter logic [MemAw-1:0] RV_DM_FLAGS_252_OFFSET = 12'h
7f0;
418 parameter logic [MemAw-1:0] RV_DM_FLAGS_253_OFFSET = 12'h
7f4;
419 parameter logic [MemAw-1:0] RV_DM_FLAGS_254_OFFSET = 12'h
7f8;
420 parameter logic [MemAw-1:0] RV_DM_FLAGS_255_OFFSET = 12'h
7fc;
421
422 // Window parameters for mem interface
423 parameter logic [MemAw-1:0] RV_DM_ROM_OFFSET = 12'h 800;
424 parameter int unsigned RV_DM_ROM_SIZE = 'h 800;
425 parameter int unsigned RV_DM_ROM_IDX = 0;
426
427 // Register index for mem interface
428 typedef enum int {
429     RV_DM_HALTED,
430     RV_DM_GOING,
431     RV_DM_RESUMING,
432     RV_DM_EXCEPTION,
433     RV_DM_WHERE_TO,
434     RV_DM_ABSTRACTCMD_0,
435     RV_DM_ABSTRACTCMD_1,
436     RV_DM_ABSTRACTCMD_2,
437     RV_DM_ABSTRACTCMD_3,

```

438	RV_DM_ABSTRACTCMD_4,	539	RV_DM_FLAGS_85,
439	RV_DM_ABSTRACTCMD_5,	540	RV_DM_FLAGS_86,
440	RV_DM_ABSTRACTCMD_6,	541	RV_DM_FLAGS_87,
441	RV_DM_ABSTRACTCMD_7,	542	RV_DM_FLAGS_88,
442	RV_DM_ABSTRACTCMD_8,	543	RV_DM_FLAGS_89,
443	RV_DM_ABSTRACTCMD_9,	544	RV_DM_FLAGS_90,
444	RV_DM_PROGRAM_BUFFER_0,	545	RV_DM_FLAGS_91,
445	RV_DM_PROGRAM_BUFFER_1,	546	RV_DM_FLAGS_92,
446	RV_DM_PROGRAM_BUFFER_2,	547	RV_DM_FLAGS_93,
447	RV_DM_PROGRAM_BUFFER_3,	548	RV_DM_FLAGS_94,
448	RV_DM_PROGRAM_BUFFER_4,	549	RV_DM_FLAGS_95,
449	RV_DM_PROGRAM_BUFFER_5,	550	RV_DM_FLAGS_96,
450	RV_DM_PROGRAM_BUFFER_6,	551	RV_DM_FLAGS_97,
451	RV_DM_PROGRAM_BUFFER_7,	552	RV_DM_FLAGS_98,
452	RV_DM_DATAADDR_0,	553	RV_DM_FLAGS_99,
453	RV_DM_DATAADDR_1,	554	RV_DM_FLAGS_100,
454	RV_DM_FLAGS_0,	555	RV_DM_FLAGS_101,
455	RV_DM_FLAGS_1,	556	RV_DM_FLAGS_102,
456	RV_DM_FLAGS_2,	557	RV_DM_FLAGS_103,
457	RV_DM_FLAGS_3,	558	RV_DM_FLAGS_104,
458	RV_DM_FLAGS_4,	559	RV_DM_FLAGS_105,
459	RV_DM_FLAGS_5,	560	RV_DM_FLAGS_106,
460	RV_DM_FLAGS_6,	561	RV_DM_FLAGS_107,
461	RV_DM_FLAGS_7,	562	RV_DM_FLAGS_108,
462	RV_DM_FLAGS_8,	563	RV_DM_FLAGS_109,
463	RV_DM_FLAGS_9,	564	RV_DM_FLAGS_110,
464	RV_DM_FLAGS_10,	565	RV_DM_FLAGS_111,
465	RV_DM_FLAGS_11,	566	RV_DM_FLAGS_112,
466	RV_DM_FLAGS_12,	567	RV_DM_FLAGS_113,
467	RV_DM_FLAGS_13,	568	RV_DM_FLAGS_114,
468	RV_DM_FLAGS_14,	569	RV_DM_FLAGS_115,
469	RV_DM_FLAGS_15,	570	RV_DM_FLAGS_116,
470	RV_DM_FLAGS_16,	571	RV_DM_FLAGS_117,
471	RV_DM_FLAGS_17,	572	RV_DM_FLAGS_118,
472	RV_DM_FLAGS_18,	573	RV_DM_FLAGS_119,
473	RV_DM_FLAGS_19,	574	RV_DM_FLAGS_120,
474	RV_DM_FLAGS_20,	575	RV_DM_FLAGS_121,
475	RV_DM_FLAGS_21,	576	RV_DM_FLAGS_122,
476	RV_DM_FLAGS_22,	577	RV_DM_FLAGS_123,
477	RV_DM_FLAGS_23,	578	RV_DM_FLAGS_124,
478	RV_DM_FLAGS_24,	579	RV_DM_FLAGS_125,
479	RV_DM_FLAGS_25,	580	RV_DM_FLAGS_126,
480	RV_DM_FLAGS_26,	581	RV_DM_FLAGS_127,
481	RV_DM_FLAGS_27,	582	RV_DM_FLAGS_128,
482	RV_DM_FLAGS_28,	583	RV_DM_FLAGS_129,
483	RV_DM_FLAGS_29,	584	RV_DM_FLAGS_130,
484	RV_DM_FLAGS_30,	585	RV_DM_FLAGS_131,
485	RV_DM_FLAGS_31,	586	RV_DM_FLAGS_132,
486	RV_DM_FLAGS_32,	587	RV_DM_FLAGS_133,
487	RV_DM_FLAGS_33,	588	RV_DM_FLAGS_134,
488	RV_DM_FLAGS_34,	589	RV_DM_FLAGS_135,
489	RV_DM_FLAGS_35,	590	RV_DM_FLAGS_136,
490	RV_DM_FLAGS_36,	591	RV_DM_FLAGS_137,
491	RV_DM_FLAGS_37,	592	RV_DM_FLAGS_138,
492	RV_DM_FLAGS_38,	593	RV_DM_FLAGS_139,
493	RV_DM_FLAGS_39,	594	RV_DM_FLAGS_140,
494	RV_DM_FLAGS_40,	595	RV_DM_FLAGS_141,
495	RV_DM_FLAGS_41,	596	RV_DM_FLAGS_142,
496	RV_DM_FLAGS_42,	597	RV_DM_FLAGS_143,
497	RV_DM_FLAGS_43,	598	RV_DM_FLAGS_144,
498	RV_DM_FLAGS_44,	599	RV_DM_FLAGS_145,
499	RV_DM_FLAGS_45,	600	RV_DM_FLAGS_146,
500	RV_DM_FLAGS_46,	601	RV_DM_FLAGS_147,
501	RV_DM_FLAGS_47,	602	RV_DM_FLAGS_148,
502	RV_DM_FLAGS_48,	603	RV_DM_FLAGS_149,
503	RV_DM_FLAGS_49,	604	RV_DM_FLAGS_150,
504	RV_DM_FLAGS_50,	605	RV_DM_FLAGS_151,
505	RV_DM_FLAGS_51,	606	RV_DM_FLAGS_152,
506	RV_DM_FLAGS_52,	607	RV_DM_FLAGS_153,
507	RV_DM_FLAGS_53,	608	RV_DM_FLAGS_154,
508	RV_DM_FLAGS_54,	609	RV_DM_FLAGS_155,
509	RV_DM_FLAGS_55,	610	RV_DM_FLAGS_156,
510	RV_DM_FLAGS_56,	611	RV_DM_FLAGS_157,
511	RV_DM_FLAGS_57,	612	RV_DM_FLAGS_158,
512	RV_DM_FLAGS_58,	613	RV_DM_FLAGS_159,
513	RV_DM_FLAGS_59,	614	RV_DM_FLAGS_160,
514	RV_DM_FLAGS_60,	615	RV_DM_FLAGS_161,
515	RV_DM_FLAGS_61,	616	RV_DM_FLAGS_162,
516	RV_DM_FLAGS_62,	617	RV_DM_FLAGS_163,
517	RV_DM_FLAGS_63,	618	RV_DM_FLAGS_164,
518	RV_DM_FLAGS_64,	619	RV_DM_FLAGS_165,
519	RV_DM_FLAGS_65,	620	RV_DM_FLAGS_166,
520	RV_DM_FLAGS_66,	621	RV_DM_FLAGS_167,
521	RV_DM_FLAGS_67,	622	RV_DM_FLAGS_168,
522	RV_DM_FLAGS_68,	623	RV_DM_FLAGS_169,
523	RV_DM_FLAGS_69,	624	RV_DM_FLAGS_170,
524	RV_DM_FLAGS_70,	625	RV_DM_FLAGS_171,
525	RV_DM_FLAGS_71,	626	RV_DM_FLAGS_172,
526	RV_DM_FLAGS_72,	627	RV_DM_FLAGS_173,
527	RV_DM_FLAGS_73,	628	RV_DM_FLAGS_174,
528	RV_DM_FLAGS_74,	629	RV_DM_FLAGS_175,
529	RV_DM_FLAGS_75,	630	RV_DM_FLAGS_176,
530	RV_DM_FLAGS_76,	631	RV_DM_FLAGS_177,
531	RV_DM_FLAGS_77,	632	RV_DM_FLAGS_178,
532	RV_DM_FLAGS_78,	633	RV_DM_FLAGS_179,
533	RV_DM_FLAGS_79,	634	RV_DM_FLAGS_180,
534	RV_DM_FLAGS_80,	635	RV_DM_FLAGS_181,
535	RV_DM_FLAGS_81,	636	RV_DM_FLAGS_182,
536	RV_DM_FLAGS_82,	637	RV_DM_FLAGS_183,
537	RV_DM_FLAGS_83,	638	RV_DM_FLAGS_184,
538	RV_DM_FLAGS_84,	639	RV_DM_FLAGS_185,

```

640 RV_DM_FLAGS_186,
641 RV_DM_FLAGS_187,
642 RV_DM_FLAGS_188,
643 RV_DM_FLAGS_189,
644 RV_DM_FLAGS_190,
645 RV_DM_FLAGS_191,
646 RV_DM_FLAGS_192,
647 RV_DM_FLAGS_193,
648 RV_DM_FLAGS_194,
649 RV_DM_FLAGS_195,
650 RV_DM_FLAGS_196,
651 RV_DM_FLAGS_197,
652 RV_DM_FLAGS_198,
653 RV_DM_FLAGS_199,
654 RV_DM_FLAGS_200,
655 RV_DM_FLAGS_201,
656 RV_DM_FLAGS_202,
657 RV_DM_FLAGS_203,
658 RV_DM_FLAGS_204,
659 RV_DM_FLAGS_205,
660 RV_DM_FLAGS_206,
661 RV_DM_FLAGS_207,
662 RV_DM_FLAGS_208,
663 RV_DM_FLAGS_209,
664 RV_DM_FLAGS_210,
665 RV_DM_FLAGS_211,
666 RV_DM_FLAGS_212,
667 RV_DM_FLAGS_213,
668 RV_DM_FLAGS_214,
669 RV_DM_FLAGS_215,
670 RV_DM_FLAGS_216,
671 RV_DM_FLAGS_217,
672 RV_DM_FLAGS_218,
673 RV_DM_FLAGS_219,
674 RV_DM_FLAGS_220,
675 RV_DM_FLAGS_221,
676 RV_DM_FLAGS_222,
677 RV_DM_FLAGS_223,
678 RV_DM_FLAGS_224,
679 RV_DM_FLAGS_225,
680 RV_DM_FLAGS_226,
681 RV_DM_FLAGS_227,
682 RV_DM_FLAGS_228,
683 RV_DM_FLAGS_229,
684 RV_DM_FLAGS_230,
685 RV_DM_FLAGS_231,
686 RV_DM_FLAGS_232,
687 RV_DM_FLAGS_233,
688 RV_DM_FLAGS_234,
689 RV_DM_FLAGS_235,
690 RV_DM_FLAGS_236,
691 RV_DM_FLAGS_237,
692 RV_DM_FLAGS_238,
693 RV_DM_FLAGS_239,
694 RV_DM_FLAGS_240,
695 RV_DM_FLAGS_241,
696 RV_DM_FLAGS_242,
697 RV_DM_FLAGS_243,
698 RV_DM_FLAGS_244,
699 RV_DM_FLAGS_245,
700 RV_DM_FLAGS_246,
701 RV_DM_FLAGS_247,
702 RV_DM_FLAGS_248,
703 RV_DM_FLAGS_249,
704 RV_DM_FLAGS_250,
705 RV_DM_FLAGS_251,
706 RV_DM_FLAGS_252,
707 RV_DM_FLAGS_253,
708 RV_DM_FLAGS_254,
709 RV_DM_FLAGS_255
710 } rv_dm_mem_id_e;
711
712 // Register width information to check illegal writes for
mem interface
713 parameter logic [3:0] RV_DM_MEM_PERMIT [281] = '{
714     4'b 0001, // index[ 0] RV_DM_HALTED
715     4'b 0001, // index[ 1] RV_DM_GOING
716     4'b 0001, // index[ 2] RV_DM_RESUMING
717     4'b 0001, // index[ 3] RV_DM_EXCEPTION
718     4'b 1111, // index[ 4] RV_DM_WHERETO
719     4'b 1111, // index[ 5] RV_DM_ABSTRACTCMD_0
720     4'b 1111, // index[ 6] RV_DM_ABSTRACTCMD_1
721     4'b 1111, // index[ 7] RV_DM_ABSTRACTCMD_2
722     4'b 1111, // index[ 8] RV_DM_ABSTRACTCMD_3
723     4'b 1111, // index[ 9] RV_DM_ABSTRACTCMD_4
724     4'b 1111, // index[10] RV_DM_ABSTRACTCMD_5
725     4'b 1111, // index[11] RV_DM_ABSTRACTCMD_6
726     4'b 1111, // index[12] RV_DM_ABSTRACTCMD_7
727     4'b 1111, // index[13] RV_DM_ABSTRACTCMD_8
728     4'b 1111, // index[14] RV_DM_ABSTRACTCMD_9
729     4'b 1111, // index[15] RV_DM_PROGRAM_BUFFER_0
730     4'b 1111, // index[16] RV_DM_PROGRAM_BUFFER_1
731     4'b 1111, // index[17] RV_DM_PROGRAM_BUFFER_2
732     4'b 1111, // index[18] RV_DM_PROGRAM_BUFFER_3
733     4'b 1111, // index[19] RV_DM_PROGRAM_BUFFER_4
734     4'b 1111, // index[20] RV_DM_PROGRAM_BUFFER_5
735     4'b 1111, // index[21] RV_DM_PROGRAM_BUFFER_6
736     4'b 1111, // index[22] RV_DM_PROGRAM_BUFFER_7
737     4'b 1111, // index[23] RV_DM_DATAADDR_0
738     4'b 1111, // index[24] RV_DM_DATAADDR_1
739     4'b 1111, // index[25] RV_DM_FLAGS_0

```

```

740 4'b 1111, // index[26] RV_DM_FLAGS_1
741 4'b 1111, // index[27] RV_DM_FLAGS_2
742 4'b 1111, // index[28] RV_DM_FLAGS_3
743 4'b 1111, // index[29] RV_DM_FLAGS_4
744 4'b 1111, // index[30] RV_DM_FLAGS_5
745 4'b 1111, // index[31] RV_DM_FLAGS_6
746 4'b 1111, // index[32] RV_DM_FLAGS_7
747 4'b 1111, // index[33] RV_DM_FLAGS_8
748 4'b 1111, // index[34] RV_DM_FLAGS_9
749 4'b 1111, // index[35] RV_DM_FLAGS_10
750 4'b 1111, // index[36] RV_DM_FLAGS_11
751 4'b 1111, // index[37] RV_DM_FLAGS_12
752 4'b 1111, // index[38] RV_DM_FLAGS_13
753 4'b 1111, // index[39] RV_DM_FLAGS_14
754 4'b 1111, // index[40] RV_DM_FLAGS_15
755 4'b 1111, // index[41] RV_DM_FLAGS_16
756 4'b 1111, // index[42] RV_DM_FLAGS_17
757 4'b 1111, // index[43] RV_DM_FLAGS_18
758 4'b 1111, // index[44] RV_DM_FLAGS_19
759 4'b 1111, // index[45] RV_DM_FLAGS_20
760 4'b 1111, // index[46] RV_DM_FLAGS_21
761 4'b 1111, // index[47] RV_DM_FLAGS_22
762 4'b 1111, // index[48] RV_DM_FLAGS_23
763 4'b 1111, // index[49] RV_DM_FLAGS_24
764 4'b 1111, // index[50] RV_DM_FLAGS_25
765 4'b 1111, // index[51] RV_DM_FLAGS_26
766 4'b 1111, // index[52] RV_DM_FLAGS_27
767 4'b 1111, // index[53] RV_DM_FLAGS_28
768 4'b 1111, // index[54] RV_DM_FLAGS_29
769 4'b 1111, // index[55] RV_DM_FLAGS_30
770 4'b 1111, // index[56] RV_DM_FLAGS_31
771 4'b 1111, // index[57] RV_DM_FLAGS_32
772 4'b 1111, // index[58] RV_DM_FLAGS_33
773 4'b 1111, // index[59] RV_DM_FLAGS_34
774 4'b 1111, // index[60] RV_DM_FLAGS_35
775 4'b 1111, // index[61] RV_DM_FLAGS_36
776 4'b 1111, // index[62] RV_DM_FLAGS_37
777 4'b 1111, // index[63] RV_DM_FLAGS_38
778 4'b 1111, // index[64] RV_DM_FLAGS_39
779 4'b 1111, // index[65] RV_DM_FLAGS_40
780 4'b 1111, // index[66] RV_DM_FLAGS_41
781 4'b 1111, // index[67] RV_DM_FLAGS_42
782 4'b 1111, // index[68] RV_DM_FLAGS_43
783 4'b 1111, // index[69] RV_DM_FLAGS_44
784 4'b 1111, // index[70] RV_DM_FLAGS_45
785 4'b 1111, // index[71] RV_DM_FLAGS_46
786 4'b 1111, // index[72] RV_DM_FLAGS_47
787 4'b 1111, // index[73] RV_DM_FLAGS_48
788 4'b 1111, // index[74] RV_DM_FLAGS_49
789 4'b 1111, // index[75] RV_DM_FLAGS_50
790 4'b 1111, // index[76] RV_DM_FLAGS_51
791 4'b 1111, // index[77] RV_DM_FLAGS_52
792 4'b 1111, // index[78] RV_DM_FLAGS_53
793 4'b 1111, // index[79] RV_DM_FLAGS_54
794 4'b 1111, // index[80] RV_DM_FLAGS_55
795 4'b 1111, // index[81] RV_DM_FLAGS_56
796 4'b 1111, // index[82] RV_DM_FLAGS_57
797 4'b 1111, // index[83] RV_DM_FLAGS_58
798 4'b 1111, // index[84] RV_DM_FLAGS_59
799 4'b 1111, // index[85] RV_DM_FLAGS_60
800 4'b 1111, // index[86] RV_DM_FLAGS_61
801 4'b 1111, // index[87] RV_DM_FLAGS_62
802 4'b 1111, // index[88] RV_DM_FLAGS_63
803 4'b 1111, // index[89] RV_DM_FLAGS_64
804 4'b 1111, // index[90] RV_DM_FLAGS_65
805 4'b 1111, // index[91] RV_DM_FLAGS_66
806 4'b 1111, // index[92] RV_DM_FLAGS_67
807 4'b 1111, // index[93] RV_DM_FLAGS_68
808 4'b 1111, // index[94] RV_DM_FLAGS_69
809 4'b 1111, // index[95] RV_DM_FLAGS_70
810 4'b 1111, // index[96] RV_DM_FLAGS_71
811 4'b 1111, // index[97] RV_DM_FLAGS_72
812 4'b 1111, // index[98] RV_DM_FLAGS_73
813 4'b 1111, // index[99] RV_DM_FLAGS_74
814 4'b 1111, // index[100] RV_DM_FLAGS_75
815 4'b 1111, // index[101] RV_DM_FLAGS_76
816 4'b 1111, // index[102] RV_DM_FLAGS_77
817 4'b 1111, // index[103] RV_DM_FLAGS_78
818 4'b 1111, // index[104] RV_DM_FLAGS_79
819 4'b 1111, // index[105] RV_DM_FLAGS_80
820 4'b 1111, // index[106] RV_DM_FLAGS_81
821 4'b 1111, // index[107] RV_DM_FLAGS_82
822 4'b 1111, // index[108] RV_DM_FLAGS_83
823 4'b 1111, // index[109] RV_DM_FLAGS_84
824 4'b 1111, // index[110] RV_DM_FLAGS_85
825 4'b 1111, // index[111] RV_DM_FLAGS_86
826 4'b 1111, // index[112] RV_DM_FLAGS_87
827 4'b 1111, // index[113] RV_DM_FLAGS_88
828 4'b 1111, // index[114] RV_DM_FLAGS_89
829 4'b 1111, // index[115] RV_DM_FLAGS_90
830 4'b 1111, // index[116] RV_DM_FLAGS_91
831 4'b 1111, // index[117] RV_DM_FLAGS_92
832 4'b 1111, // index[118] RV_DM_FLAGS_93
833 4'b 1111, // index[119] RV_DM_FLAGS_94
834 4'b 1111, // index[120] RV_DM_FLAGS_95
835 4'b 1111, // index[121] RV_DM_FLAGS_96
836 4'b 1111, // index[122] RV_DM_FLAGS_97
837 4'b 1111, // index[123] RV_DM_FLAGS_98
838 4'b 1111, // index[124] RV_DM_FLAGS_99
839 4'b 1111, // index[125] RV_DM_FLAGS_100
840 4'b 1111, // index[126] RV_DM_FLAGS_101

```



```

841 4'b 1111, // index[127] RV_DM_FLAGS_102
842 4'b 1111, // index[128] RV_DM_FLAGS_103
843 4'b 1111, // index[129] RV_DM_FLAGS_104
844 4'b 1111, // index[130] RV_DM_FLAGS_105
845 4'b 1111, // index[131] RV_DM_FLAGS_106
846 4'b 1111, // index[132] RV_DM_FLAGS_107
847 4'b 1111, // index[133] RV_DM_FLAGS_108
848 4'b 1111, // index[134] RV_DM_FLAGS_109
849 4'b 1111, // index[135] RV_DM_FLAGS_110
850 4'b 1111, // index[136] RV_DM_FLAGS_111
851 4'b 1111, // index[137] RV_DM_FLAGS_112
852 4'b 1111, // index[138] RV_DM_FLAGS_113
853 4'b 1111, // index[139] RV_DM_FLAGS_114
854 4'b 1111, // index[140] RV_DM_FLAGS_115
855 4'b 1111, // index[141] RV_DM_FLAGS_116
856 4'b 1111, // index[142] RV_DM_FLAGS_117
857 4'b 1111, // index[143] RV_DM_FLAGS_118
858 4'b 1111, // index[144] RV_DM_FLAGS_119
859 4'b 1111, // index[145] RV_DM_FLAGS_120
860 4'b 1111, // index[146] RV_DM_FLAGS_121
861 4'b 1111, // index[147] RV_DM_FLAGS_122
862 4'b 1111, // index[148] RV_DM_FLAGS_123
863 4'b 1111, // index[149] RV_DM_FLAGS_124
864 4'b 1111, // index[150] RV_DM_FLAGS_125
865 4'b 1111, // index[151] RV_DM_FLAGS_126
866 4'b 1111, // index[152] RV_DM_FLAGS_127
867 4'b 1111, // index[153] RV_DM_FLAGS_128
868 4'b 1111, // index[154] RV_DM_FLAGS_129
869 4'b 1111, // index[155] RV_DM_FLAGS_130
870 4'b 1111, // index[156] RV_DM_FLAGS_131
871 4'b 1111, // index[157] RV_DM_FLAGS_132
872 4'b 1111, // index[158] RV_DM_FLAGS_133
873 4'b 1111, // index[159] RV_DM_FLAGS_134
874 4'b 1111, // index[160] RV_DM_FLAGS_135
875 4'b 1111, // index[161] RV_DM_FLAGS_136
876 4'b 1111, // index[162] RV_DM_FLAGS_137
877 4'b 1111, // index[163] RV_DM_FLAGS_138
878 4'b 1111, // index[164] RV_DM_FLAGS_139
879 4'b 1111, // index[165] RV_DM_FLAGS_140
880 4'b 1111, // index[166] RV_DM_FLAGS_141
881 4'b 1111, // index[167] RV_DM_FLAGS_142
882 4'b 1111, // index[168] RV_DM_FLAGS_143
883 4'b 1111, // index[169] RV_DM_FLAGS_144
884 4'b 1111, // index[170] RV_DM_FLAGS_145
885 4'b 1111, // index[171] RV_DM_FLAGS_146
886 4'b 1111, // index[172] RV_DM_FLAGS_147
887 4'b 1111, // index[173] RV_DM_FLAGS_148
888 4'b 1111, // index[174] RV_DM_FLAGS_149
889 4'b 1111, // index[175] RV_DM_FLAGS_150
890 4'b 1111, // index[176] RV_DM_FLAGS_151
891 4'b 1111, // index[177] RV_DM_FLAGS_152
892 4'b 1111, // index[178] RV_DM_FLAGS_153
893 4'b 1111, // index[179] RV_DM_FLAGS_154
894 4'b 1111, // index[180] RV_DM_FLAGS_155
895 4'b 1111, // index[181] RV_DM_FLAGS_156
896 4'b 1111, // index[182] RV_DM_FLAGS_157
897 4'b 1111, // index[183] RV_DM_FLAGS_158
898 4'b 1111, // index[184] RV_DM_FLAGS_159
899 4'b 1111, // index[185] RV_DM_FLAGS_160
900 4'b 1111, // index[186] RV_DM_FLAGS_161
901 4'b 1111, // index[187] RV_DM_FLAGS_162
902 4'b 1111, // index[188] RV_DM_FLAGS_163
903 4'b 1111, // index[189] RV_DM_FLAGS_164
904 4'b 1111, // index[190] RV_DM_FLAGS_165
905 4'b 1111, // index[191] RV_DM_FLAGS_166
906 4'b 1111, // index[192] RV_DM_FLAGS_167
907 4'b 1111, // index[193] RV_DM_FLAGS_168
908 4'b 1111, // index[194] RV_DM_FLAGS_169
909 4'b 1111, // index[195] RV_DM_FLAGS_170
910 4'b 1111, // index[196] RV_DM_FLAGS_171
911 4'b 1111, // index[197] RV_DM_FLAGS_172
912 4'b 1111, // index[198] RV_DM_FLAGS_173
913 4'b 1111, // index[199] RV_DM_FLAGS_174
914 4'b 1111, // index[200] RV_DM_FLAGS_175
915 4'b 1111, // index[201] RV_DM_FLAGS_176
916 4'b 1111, // index[202] RV_DM_FLAGS_177
917 4'b 1111, // index[203] RV_DM_FLAGS_178
918 4'b 1111, // index[204] RV_DM_FLAGS_179
919 4'b 1111, // index[205] RV_DM_FLAGS_180
920 4'b 1111, // index[206] RV_DM_FLAGS_181
921 4'b 1111, // index[207] RV_DM_FLAGS_182
922 4'b 1111, // index[208] RV_DM_FLAGS_183
923 4'b 1111, // index[209] RV_DM_FLAGS_184
924 4'b 1111, // index[210] RV_DM_FLAGS_185
925 4'b 1111, // index[211] RV_DM_FLAGS_186
926 4'b 1111, // index[212] RV_DM_FLAGS_187
927 4'b 1111, // index[213] RV_DM_FLAGS_188
928 4'b 1111, // index[214] RV_DM_FLAGS_189
929 4'b 1111, // index[215] RV_DM_FLAGS_190
930 4'b 1111, // index[216] RV_DM_FLAGS_191
931 4'b 1111, // index[217] RV_DM_FLAGS_192
932 4'b 1111, // index[218] RV_DM_FLAGS_193
933 4'b 1111, // index[219] RV_DM_FLAGS_194
934 4'b 1111, // index[220] RV_DM_FLAGS_195
935 4'b 1111, // index[221] RV_DM_FLAGS_196
936 4'b 1111, // index[222] RV_DM_FLAGS_197
937 4'b 1111, // index[223] RV_DM_FLAGS_198
938 4'b 1111, // index[224] RV_DM_FLAGS_199
939 4'b 1111, // index[225] RV_DM_FLAGS_200
940 4'b 1111, // index[226] RV_DM_FLAGS_201
941 4'b 1111, // index[227] RV_DM_FLAGS_202

```

```

942 4'b 1111, // index[228] RV_DM_FLAGS_203
943 4'b 1111, // index[229] RV_DM_FLAGS_204
944 4'b 1111, // index[230] RV_DM_FLAGS_205
945 4'b 1111, // index[231] RV_DM_FLAGS_206
946 4'b 1111, // index[232] RV_DM_FLAGS_207
947 4'b 1111, // index[233] RV_DM_FLAGS_208
948 4'b 1111, // index[234] RV_DM_FLAGS_209
949 4'b 1111, // index[235] RV_DM_FLAGS_210
950 4'b 1111, // index[236] RV_DM_FLAGS_211
951 4'b 1111, // index[237] RV_DM_FLAGS_212
952 4'b 1111, // index[238] RV_DM_FLAGS_213
953 4'b 1111, // index[239] RV_DM_FLAGS_214
954 4'b 1111, // index[240] RV_DM_FLAGS_215
955 4'b 1111, // index[241] RV_DM_FLAGS_216
956 4'b 1111, // index[242] RV_DM_FLAGS_217
957 4'b 1111, // index[243] RV_DM_FLAGS_218
958 4'b 1111, // index[244] RV_DM_FLAGS_219
959 4'b 1111, // index[245] RV_DM_FLAGS_220
960 4'b 1111, // index[246] RV_DM_FLAGS_221
961 4'b 1111, // index[247] RV_DM_FLAGS_222
962 4'b 1111, // index[248] RV_DM_FLAGS_223
963 4'b 1111, // index[249] RV_DM_FLAGS_224
964 4'b 1111, // index[250] RV_DM_FLAGS_225
965 4'b 1111, // index[251] RV_DM_FLAGS_226
966 4'b 1111, // index[252] RV_DM_FLAGS_227
967 4'b 1111, // index[253] RV_DM_FLAGS_228
968 4'b 1111, // index[254] RV_DM_FLAGS_229
969 4'b 1111, // index[255] RV_DM_FLAGS_230
970 4'b 1111, // index[256] RV_DM_FLAGS_231
971 4'b 1111, // index[257] RV_DM_FLAGS_232
972 4'b 1111, // index[258] RV_DM_FLAGS_233
973 4'b 1111, // index[259] RV_DM_FLAGS_234
974 4'b 1111, // index[260] RV_DM_FLAGS_235
975 4'b 1111, // index[261] RV_DM_FLAGS_236
976 4'b 1111, // index[262] RV_DM_FLAGS_237
977 4'b 1111, // index[263] RV_DM_FLAGS_238
978 4'b 1111, // index[264] RV_DM_FLAGS_239
979 4'b 1111, // index[265] RV_DM_FLAGS_240
980 4'b 1111, // index[266] RV_DM_FLAGS_241
981 4'b 1111, // index[267] RV_DM_FLAGS_242
982 4'b 1111, // index[268] RV_DM_FLAGS_243
983 4'b 1111, // index[269] RV_DM_FLAGS_244
984 4'b 1111, // index[270] RV_DM_FLAGS_245
985 4'b 1111, // index[271] RV_DM_FLAGS_246
986 4'b 1111, // index[272] RV_DM_FLAGS_247
987 4'b 1111, // index[273] RV_DM_FLAGS_248
988 4'b 1111, // index[274] RV_DM_FLAGS_249
989 4'b 1111, // index[275] RV_DM_FLAGS_250
990 4'b 1111, // index[276] RV_DM_FLAGS_251
991 4'b 1111, // index[277] RV_DM_FLAGS_252
992 4'b 1111, // index[278] RV_DM_FLAGS_253
993 4'b 1111, // index[279] RV_DM_FLAGS_254
994 4'b 1111, // index[280] RV_DM_FLAGS_255
995 };
996
997 // Window parameters for dbg interface
998 parameter logic [DbgAw-1:0] RV_DM_DBG_OFFSET = 9'h 0;
999 parameter int unsigned RV_DM_DBG_SIZE = 'h 200;
1000 parameter int unsigned RV_DM_DBG_IDX = 0;
1001
1002 endpackage

```

7. lowRISC_opentitan/hw/ip/rv_dm/rtl/rv_dm_regs_reg_top.sv (363 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module rv_dm_regs_reg_top
10 # (
11     parameter bit EnableRacl = 1'b0,
12     parameter bit RaclErrorRsp = 1'b1,
13     parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelVec[rv_dm_reg_pkg::NumRegsRegs] =
14         '{rv_dm_reg_pkg::NumRegsRegs{0}}
15 ) (
16     input clk_i,
17     input rst_ni,
18     input tlul_pkg::tl_h2d_t tl_i,
19     output tlul_pkg::tl_d2h_t tl_o,
20     // To HW
21     output rv_dm_reg_pkg::rv_dm_regs_reg2hw_t reg2hw, // Write
22
23     // RACL interface
24     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
25     output top_racl_pkg::racl_error_log_t racl_error_o,
26
27     // Integrity check errors
28     output logic intg_err_o
29 );
30
31 import rv_dm_reg_pkg::*;
32

```

```

33 localparam int AW = 4;
34 localparam int DW = 32;
35 localparam int DBW = DW/8; // Byte Width
36
37 // register signals
38 logic reg_we;
39 logic reg_re;
40 logic [AW-1:0] reg_addr;
41 logic [DW-1:0] reg_wdata;
42 logic [DBW-1:0] reg_be;
43 logic [DW-1:0] reg_rdata;
44 logic reg_error;
45
46 logic addrmiss, wr_err;
47
48 logic [DW-1:0] reg_rdata_next;
49 logic reg_busy;
50
51 tlul_pkg::tl_h2d_t tl_reg_h2d;
52 tlul_pkg::tl_d2h_t tl_reg_d2h;
53
54 // incoming payload check
55 logic intg_err;
56 tlul_cmd_intg_chk u_chk (
57     .tl_i(tl_i),
58     .err_o(intg_err)
59 );
60
61 // also check for spurious write enables
62 logic reg_we_err;
63 logic [2:0] reg_we_check;
64 prim_reg_we_check #(
65     .OneHotWidth(3)
66 ) u_prim_reg_we_check (
67     .clk_i(clk_i),
68     .rst_ni(rst_ni),
69     .oh_i (reg_we_check),
70     .en_i (reg_we && !addrmiss),
71     .err_o (reg_we_err)
72 );
73
74 logic err_q;
75 always_ff @(posedge clk_i or negedge rst_ni) begin
76     if (!rst_ni) begin
77         err_q <= '0;
78     end else if (intg_err || reg_we_err) begin
79         err_q <= 1'b1;
80     end
81 end
82
83 // integrity error output is permanent and should be used
84 for alert generation
85 // register errors are transactional
86 assign intg_err_o = err_q | intg_err | reg_we_err;
87
88 // outgoing integrity generation
89 tlul_pkg::tl_d2h_t tl_o_pre;
90 tlul_rsp_intg_gen #(
91     .EnableRspIntgGen(1),
92     .EnableDataIntgGen(1)
93 ) u_rsp_intg_gen (
94     .tl_i(tl_o_pre),
95     .tl_o(tl_o)
96 );
97
98 assign tl_reg_h2d = tl_i;
99 assign tl_o_pre = tl_reg_d2h;
100
101 tlul_adapter_reg #(
102     .RegAw(AW),
103     .RegDw(DW),
104     .EnableDataIntgGen(0)
105 ) u_reg_if (
106     .clk_i (clk_i),
107     .rst_ni (rst_ni),
108
109     .tl_i (tl_reg_h2d),
110     .tl_o (tl_reg_d2h),
111
112     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
113     .intg_error_o(),
114
115     .we_o (reg_we),
116     .re_o (reg_re),
117     .addr_o (reg_addr),
118     .wdata_o (reg_wdata),
119     .be_o (reg_be),
120     .busy_i (reg_busy),
121     .rdata_i (reg_rdata),
122     // Translate RACL error to TLUL error if enabled
123     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
124 );
125
126 // cdc oversampling signals
127
128 assign reg_rdata = reg_rdata_next ;
129 assign reg_error = addrmiss | wr_err | intg_err;
130
131 // Define SW related signals
132 // Format: <reg>_<field>_<wd|we|qs>

```

```

133 // or <reg>_<wd|we|qs> if field == 1 or 0
134 logic alert_test_we;
135 logic alert_test_wd;
136 logic late_debug_enable_regwen_we;
137 logic late_debug_enable_regwen_qs;
138 logic late_debug_enable_regwen_wd;
139 logic late_debug_enable_we;
140 logic [31:0] late_debug_enable_qs;
141 logic [31:0] late_debug_enable_wd;
142
143 // Register instances
144 // R[alert_test]: V(True)
145 logic alert_test_qe;
146 logic [0:0] alert_test_flds_we;
147 assign alert_test_qe = &alert_test_flds_we;
148 prim_subreg_ext #(
149     .DW (1)
150 ) u_alert_test (
151     .re (1'b0),
152     .we (alert_test_we),
153     .wd (alert_test_wd),
154     .d ('0),
155     .qre (),
156     .qe (alert_test_flds_we[0]),
157     .q (reg2hw.alert_test.q),
158     .ds (),
159     .qs ()
160 );
161 assign reg2hw.alert_test.qe = alert_test_qe;
162
163 // R[late_debug_enable_regwen]: V(False)
164 prim_subreg #(
165     .DW (1),
166     .SwAccess(prim_subreg_pkg::SwAccessW0C),
167     .RESVAL (1'h1),
168     .Mubi (1'b0)
169 ) u_late_debug_enable_regwen (
170     .clk_i (clk_i),
171     .rst_ni (rst_ni),
172
173     // from register interface
174     .we (late_debug_enable_regwen_we),
175     .wd (late_debug_enable_regwen_wd),
176
177     // from internal hardware
178     .de (1'b0),
179     .d ('0),
180
181     // to internal hardware
182     .qe (),
183     .q (),
184     .ds (),
185
186     // to register interface (read)
187     .qs (late_debug_enable_regwen_qs)
188 );
189
190 // R[late_debug_enable]: V(False)
191 // Create REGWEN-gated WE signal
192 logic late_debug_enable_gated_we;
193 assign late_debug_enable_gated_we = late_debug_enable_we &
194 late_debug_enable_regwen_qs;
195 prim_subreg #(
196     .DW (32),
197     .SwAccess(prim_subreg_pkg::SwAccessRW),
198     .RESVAL (32'h69696969),
199     .Mubi (1'b1)
200 ) u_late_debug_enable (
201     .clk_i (clk_i),
202     .rst_ni (rst_ni),
203
204     // from register interface
205     .we (late_debug_enable_gated_we),
206     .wd (late_debug_enable_wd),
207
208     // from internal hardware
209     .de (1'b0),
210     .d ('0),
211
212     // to internal hardware
213     .qe (),
214     .q (reg2hw.late_debug_enable.q),
215     .ds (),
216
217     // to register interface (read)
218     .qs (late_debug_enable_qs)
219 );
220
221 logic [2:0] addr_hit;
222 top_racl_pkg::racl_role_vec_t racl_role_vec;
223 top_racl_pkg::racl_role_t racl_role;
224
225 logic [2:0] racl_addr_hit_read;
226 logic [2:0] racl_addr_hit_write;
227
228 if (EnableRacl) begin : gen_racl_role_logic
229     // Retrieve RACL role from user bits and one-hot encode

```

```

that for the comparison bitmap
233   assign rac1_role =
top_rac1_pkg::tlul_extract_rac1_role_bits(tl_i.a_user.rsvd);
234
235   prim_onehot_enc #(
236     .OneHotWidth( $bits(top_rac1_pkg::rac1_role_vec_t) )
237   ) u_rac1_role_encode (
238     .in_i ( rac1_role ),
239     .en_i ( 1'b1 ),
240     .out_o( rac1_role_vec )
241   );
242 end else begin : gen_no_rac1_role_logic
243   assign rac1_role = '0;
244   assign rac1_role_vec = '0;
245 end
246
247 always_comb begin
248   rac1_addr_hit_read = '0;
249   rac1_addr_hit_write = '0;
250   addr_hit[0] = (reg_addr == RV_DM_ALERT_TEST_OFFSET);
251   addr_hit[1] = (reg_addr ==
RV_DM_LATE_DEBUG_ENABLE_REGWEN_OFFSET);
252   addr_hit[2] = (reg_addr ==
RV_DM_LATE_DEBUG_ENABLE_OFFSET);
253
254   if (EnableRac1) begin : gen_rac1_hit
255     for (int unsigned slice_idx = 0; slice_idx < 3;
slice_idx++) begin
256       rac1_addr_hit_read[slice_idx] =
257         addr_hit[slice_idx] & (|
(rac1_policies_i[Rac1PolicySelVec[slice_idx]].read_perm
& rac1_role_vec));
258       rac1_addr_hit_write[slice_idx] =
259         addr_hit[slice_idx] & (|
(rac1_policies_i[Rac1PolicySelVec[slice_idx]].write_perm
& rac1_role_vec));
260     end
261   end else begin : gen_no_rac1
262     rac1_addr_hit_read = addr_hit;
263     rac1_addr_hit_write = addr_hit;
264   end
265 end
266
267 end
268
269 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
270 // A valid address hit, access, but failed the RACL check
271 assign rac1_error_o.valid = |addr_hit & ((reg_re & ~|
rac1_addr_hit_read) |
272                                     (reg_we & ~|
rac1_addr_hit_write));
273 assign rac1_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
274 assign rac1_error_o.rac1_role = rac1_role;
275 assign rac1_error_o.overflow = 1'b0;
276
277 if (EnableRac1) begin : gen_rac1_log
278   assign rac1_error_o.ctn_uid =
top_rac1_pkg::tlul_extract_ctn_uid_bits(tl_i.a_user.rsvd);
279   assign rac1_error_o.read_access = tl_i.a_opcode ==
tlul_pkg::Get;
280 end else begin : gen_no_rac1_log
281   assign rac1_error_o.ctn_uid = '0;
282   assign rac1_error_o.read_access = 1'b0;
283 end
284
285 // Check sub-word write is permitted
286 always_comb begin
287   wr_err = (reg_we &
288             ((rac1_addr_hit_write[0] & (|
(RV_DM_REGS_PERMIT[0] & ~reg_be))) |
289             (rac1_addr_hit_write[1] & (|
(RV_DM_REGS_PERMIT[1] & ~reg_be))) |
290             (rac1_addr_hit_write[2] & (|
(RV_DM_REGS_PERMIT[2] & ~reg_be)))));
291   end
292
293 // Generate write-enables
294 assign alert_test_we = rac1_addr_hit_write[0] & reg_we & !

```

```

reg_error;
295
296 assign alert_test_wd = reg_wdata[0];
297 assign late_debug_enable_regwen_we = rac1_addr_hit_write[1]
& reg_we & !reg_error;
298
299 assign late_debug_enable_regwen_wd = reg_wdata[0];
300 assign late_debug_enable_we = rac1_addr_hit_write[2] &
reg_we & !reg_error;
301
302 assign late_debug_enable_wd = reg_wdata[31:0];
303
304 // Assign write-enables to checker logic vector.
305 always_comb begin
306   reg_we_check[0] = alert_test_we;
307   reg_we_check[1] = late_debug_enable_regwen_we;
308   reg_we_check[2] = late_debug_enable_gated_we;
309 end
310
311 // Read data return
312 always_comb begin
313   reg_rdata_next = '0;
314   unique case (1'b1)
315     rac1_addr_hit_read[0]: begin
316       reg_rdata_next[0] = '0;
317     end
318
319     rac1_addr_hit_read[1]: begin
320       reg_rdata_next[0] = late_debug_enable_regwen_qs;
321     end
322
323     rac1_addr_hit_read[2]: begin
324       reg_rdata_next[31:0] = late_debug_enable_qs;
325     end
326
327     default: begin
328       reg_rdata_next = '1;
329     end
330   endcase
331 end
332
333 // shadow busy
334 logic shadow_busy;
335 assign shadow_busy = 1'b0;
336
337 // register busy
338 assign reg_busy = shadow_busy;
339
340 // Unused signal tieoff
341
342 // wdata / byte enable are not always fully used
343 // add a blanket unused statement to handle lint waivers
344 logic unused_wdata;
345 logic unused_be;
346 assign unused_wdata = ^reg_wdata;
347 assign unused_be = ^reg_be;
348 logic unused_policy_sel;
349 assign unused_policy_sel = ^rac1_policies_i;
350
351 // Assertions for Register Interface
352 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
353 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
354
355 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>
tl_o_pre.d_valid, clk_i, !rst_ni)
356
357 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
hot0(addr_hit), clk_i, !rst_ni)
358
359 // this is formulated as an assumption such that the FPV
testbenches do disprove this
360 // property by mistake
361 //`ASSUME(reqParity, tl_reg_h2d.a_valid |>
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
362
363 endmodule

```

TL-UL 버스 구조

What. TileLink Uncached Lightweight 버스의 어댑터 · 소켓 · FIFO · 오류 응답.

Why it exists. 모든 주변장치를 같은 방식으로 붙이기 위해 서다. 버스를 통일하면 블록마다 인터페이스를 새로 만들 필요가 없고, 레지스터 생성기 · 검증 IP · 주소 디코더를 재사용할 수 있다. IP 재사용의 실질적 기반이 버스 표준화다.

학부 복습 · **EECS concepts.** 버스 프로토콜 · 핸드셰이크 · 주소 디코딩 · 중재 · 파이프라이닝과 처리량.

석사 수준 · **Practice note.** 버스 어댑터는 ‘공짜’가 아니다. 지연이 붙고 면적을 먹는다. 작은 블록에서는 어댑터가 본체보다 클 수도 있다. IP 자원 보고에 어댑터 포함/제외를 명시하라.

28 files • 4,683 lines

API — sram2tlul.sv

Table 464. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — module / interface / package (1개)

이름	종류
sram2tlul	module

Table 465. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — parameter (3개)

파라미터	형/폭	기본값
SramAw	int	12
SramDw	int	32
TlBaseAddr	logic [top_pkg::TL_AW-1:0]	'h0 // Base address of SRAM request

Table 466. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — localparam (1개)

상수	형/폭	값
SRAM_DWB	int unsigned	\$clog2(SramDw/8)

Table 467. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_o	output	tlul_pkg::tl_h2d_t
tl_i	input	tlul_pkg::tl_d2h_t
mem_req_i	input	
mem_write_i	input	
mem_addr_i	input	[SramAw-1:0]
mem_wdata_i	input	[SramDw-1:0]

포트	방향	형/폭
mem_rvalid_o	output	logic
mem_rdata_o	output	logic [SramDw-1:0]
mem_error_o	output	logic [1:0]

API — tlul_adapter_dmi.sv

Table 468. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — module / interface / package (1개)

이름	종류
tlul_adapter_dmi	module

Table 469. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — parameter (3개)

파라미터	형/폭	기본값
CmdIntgCheck	bit	1
EnableRspIntgGen	bit	1
EnableDataIntgGen	bit	1 // 1: Generate response data integrity

Table 470. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — localparam (2개)

상수	형/폭	값
IdW	int	\$bits(tl_h2d_i.a_source)
SzW	int	\$bits(tl_h2d_i.a_size)

Table 471. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h2d_i	input	tl_h2d_t
tl_d2h_o	output	tl_d2h_t
intg_error_o	output	logic
dmi_req_valid_o	output	logic
dmi_req_ready_i	input	logic
dmi_req_o	output	dm::dmi_req_t
dmi_resp_valid_i	input	logic
dmi_resp_ready_o	output	logic
dmi_resp_i	input	dm::dmi_resp_t

API — tlul_adapter_host.sv

Table 472. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — module / interface / package (1개)

이름	종류
tlul_adapter_host	module

Table 473. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — parameter (3개)

파라미터	형/폭	기본값
MAX_REQS	int unsigned	2
EnableDataIntgGen	bit	0
EnableRspDataIntgCheck	bit	0

Table 474. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — localparam (6개)

상수	형/폭	값
WordSize	int unsigned	$\$clog2(top_pkg::TL_DBW)$
ReqNumW	int	$\$clog2(MAX_REQS)$
MaxSource	int unsigned	$MAX_REQS - 1$
ReqNumOne	logic [ReqNumW-1:0]	$ReqNumW'(1'b1)$
OutstandingReqCntW	int	$(MAX_REQS == 2 ** \$clog2(MAX_REQS)) ? \$clog2(MAX_REQS) + 1 : \dots$
OutstandingReqCntOne	logic [OutstandingReqCntW-1:0]	$OutstandingReqCntW'(1'b1)$

Table 475. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_i	input	
gnt_o	output	logic
addr_i	input	logic [top_pkg::TL_AW-1:0]
we_i	input	logic
wdata_i	input	logic [top_pkg::TL_DW-1:0]
wdata_intg_i	input	logic [DataIntgWidth-1:0]
be_i	input	logic [top_pkg::TL_DBW-1:0]
instr_type_i	input	mubi4_t
user_rsvd_i	input	logic [RsvdWidth-1:0]
valid_o	output	logic
rdata_o	output	logic [top_pkg::TL_DW-1:0]
rdata_intg_o	output	logic [DataIntgWidth-1:0]
err_o	output	logic
intg_err_o	output	logic

포트	방향	형/폭
tl_o	output	tl_h2d_t
tl_i	input	tl_d2h_t

API — tlul_adapter_racl.sv

Table 476. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.sv — module / interface / package (1개)

이름	종류
tlul_adapter_racl	module

Table 477. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.sv — parameter (3개)

파라미터	형/폭	기본값
EnableRacl	bit	0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelNumRanges	int	1 // Number of ranges with a RACL policy

Table 478. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h2d_i	input	tlul_pkg::tl_h2d_t
tl_d2h_o	output	tlul_pkg::tl_d2h_t
tl_filtered_h2d_o	output	tlul_pkg::tl_h2d_t
tl_filtered_d2h_i	input	tlul_pkg::tl_d2h_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
racl_policy_sel_ranges_i	input	top_racl_pkg::racl_range_t [RaclPolicySelNumRanges...

API — tlul_adapter_reg.sv

Table 479. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — module / interface / package (1개)

이름	종류
tlul_adapter_reg	module

Table 480. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — parameter (6개)

파라미터	형/폭	기본값
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0
RegAw	int	8

파라미터	형/폭	기본값
RegDw	int	32
AccessLatency	int	0

Table 481. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — localparam (3개)

상수	형/폭	값
RegBw	int	RegDw/8
IW	int	\$bits(tl_i.a_source)
SZW	int	\$bits(tl_i.a_size)

Table 482. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv — 포트 (14개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
intg_error_o	output	logic
re_o	output	logic
we_o	output	logic
addr_o	output	logic [RegAw-1:0]
wdata_o	output	logic [RegDw-1:0]
be_o	output	logic [RegBw-1:0]
busy_i	input	
rdata_i	input	[RegDw-1:0]
error_i	input	

API — tlul_adapter_reg_racl.sv

Table 483. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — module / interface / package (1개)

이름	종류
tlul_adapter_reg_racl	module

Table 484. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — parameter (9개)

파라미터	형/폭	기본값
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0

파라미터	형/폭	기본값
RegAw	int	8
RegDw	int	32
AccessLatency	int	0
EnableRacl	bit	0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelVec	top_racl_pkg::racl_policy_sel_t	0

Table 485. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — localparam (1개)

상수	형/폭	값
RegBw	int	RegDw/8

Table 486. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
intg_error_o	output	logic
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
re_o	output	logic
we_o	output	logic
addr_o	output	logic [RegAw-1:0]
wdata_o	output	logic [RegDw-1:0]
be_o	output	logic [RegBw-1:0]
busy_i	input	
rdata_i	input	[RegDw-1:0]
error_i	input	

API — tlul_adapter_shim.sv

Table 487. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv — module / interface / package (1개)

이름	종류
tlul_adapter_vh	module

Table 488. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv — parameter (8개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	top_pkg::TL_AW
DATA_WIDTH	int	top_pkg::TL_DW
MASK_WIDTH	int	DATA_WIDTH >> 3
USER_WIDTH	int	32
ID_WIDTH	int	top_pkg::TL_AIW
VH_REGISTER_ADDRESS_OFFSET	bit [ADDR_WIDTH-1:0]	32'h0000_0100
EnableDataIntgGen	bit	1
EnableRspDataIntgCheck	bit	1

Table 489. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_o	output	tl_h2d_t
tl_i	input	tl_d2h_t
dv_i	input	logic
hld_o	output	logic
addr_i	input	logic [ADDR_WIDTH-1:0]
write_i	input	logic
wdata_i	input	logic [DATA_WIDTH-1:0]
wstrb_i	input	logic [MASK_WIDTH-1:0]
size_i	input	logic [2:0]
rdata_o	output	logic [DATA_WIDTH-1:0]
error_o	output	logic
last_i	input	logic
user_i	input	logic [USER_WIDTH-1:0]
id_i	input	logic [ID_WIDTH-1:0]
int_dv_o	output	logic
int_hld_i	input	logic
int_addr_o	output	logic [ADDR_WIDTH-1:0]
int_write_o	output	logic
int_wdata_o	output	logic [DATA_WIDTH-1:0]
int_wstrb_o	output	logic [MASK_WIDTH-1:0]
int_size_o	output	logic [2:0]
int_rdata_i	input	logic [DATA_WIDTH-1:0]

포트	방향	형/폭
int_error_i	input	logic
int_last_o	output	logic
int_user_o	output	logic [USER_WIDTH-1:0]
int_id_o	output	logic [ID_WIDTH-1:0]

API — tlul_adapter_sram.sv

Table 490. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — module / interface / package (1개)

이름	종류
tlul_adapter_sram	module

Table 491. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — parameter (15개)

파라미터	형/폭	기본값
SramAw	int	12
SramDw	int	32
SramDepth	int	2**SramAw
Outstanding	int	1
SramBusBankAW	int	12
ByteAccess	bit	1
ErrOnWrite	bit	0
ErrOnRead	bit	0
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0
EnableDataIntgPt	bit	0
SecFifoPtr	bit	0
EnableReadback	bit	0
DataXorAddr	bit	0

Table 492. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — localparam (11개)

상수	형/폭	값
WidthMult	int	SramDw / top_pkg::TL_DW
IntgWidth	int	tlul_pkg::DataIntgWidth * WidthMult
DataOutW	int	EnableDataIntgPt ? SramDw + IntgWidth : SramDw
SramByte	int	SramDw/8
DataBitWidth	int	prim_util_pkg::vbits(SramByte)
WoffsetWidth	int	(SramByte == top_pkg::TL_DBW) ? 1 :

상수	형/폭	값
SramReqWidth	int	\$bits(sram_req_t)
SramReqFifoWidth	int	SramReqWidth + (DataXorAddr ? SramBusBankAW : 0)
ReqFifoWidth	int	\$bits(req_t)
RspFifoWidth	int	\$bits(rsp_t)
DataWidth	int	EnableDataIntgPt ? top_pkg::TL_DW + DataIntgWidth : top_pkg::...

Table 493. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv — 포트 (22개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
req_o	output	logic
req_type_o	output	mubi4_t
gnt_i	input	
we_o	output	logic
addr_o	output	logic [SramAw-1:0]
wdata_o	output	logic [DataOutW-1:0]
wmask_o	output	logic [DataOutW-1:0]
intg_error_o	output	logic
user_rsvd_o	output	logic [RsvdWidth-1:0]
rdata_i	input	[DataOutW-1:0]
rvalid_i	input	
rerror_i	input	[1:0]
compound_txn_in_progress_o	output	logic
readback_en_i	input	mubi4_t
readback_error_o	output	logic
wr_collision_i	input	logic
write_pending_i	input	logic

API — tlul_adapter_sram_racl.sv

Table 494. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — module / interface / package (1개)

이름	종류
tlul_adapter_sram_racl	module

Table 495. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — parameter (18개)

파라미터	형/폭	기본값
SramAw	int	12
SramDw	int	32
SramDepth	int	2**SramAw
Outstanding	int	1
SramBusBankAW	int	12
ByteAccess	bit	1
ErrOnWrite	bit	0
ErrOnRead	bit	0
CmdIntgCheck	bit	0
EnableRspIntgGen	bit	0
EnableDataIntgGen	bit	0
EnableDataIntgPt	bit	0
SecFifoPtr	bit	0
EnableReadback	bit	0
DataXorAddr	bit	0
EnableRacl	bit	0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelNumRanges	int	1

Table 496. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — localparam (3개)

상수	형/폭	값
WidthMult	int	SramDw / top_pkg::TL_DW
IntgWidth	int	tlul_pkg::DataIntgWidth * WidthMult
DataOutW	int	EnableDataIntgPt ? SramDw + IntgWidth : SramDw

Table 497. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv — 포트 (25개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
tl_o	output	tl_d2h_t
en_ifetch_i	input	mubi4_t
req_o	output	logic
req_type_o	output	mubi4_t
gnt_i	input	

포트	방향	형/폭
we_o	output	logic
addr_o	output	logic [SramAw-1:0]
wdata_o	output	logic [DataOutW-1:0]
wmask_o	output	logic [DataOutW-1:0]
intg_error_o	output	logic
user_rsvd_o	output	logic [RsrdWidth-1:0]
rdata_i	input	[DataOutW-1:0]
rvalid_i	input	
rerror_i	input	[1:0]
compound_txn_in_progress_o	output	logic
readback_en_i	input	mubi4_t
readback_error_o	output	logic
wr_collision_i	input	logic
write_pending_i	input	logic
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
racl_policy_sel_ranges_i	input	top_racl_pkg::racl_range_t [RaclPolicySelNumRanges...

API — tlul_adapter_vh.sv

Table 498. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv — module / interface / package (1개)

이름	종류
tlul_adapter_vh	module

Table 499. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv — parameter (8개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	top_pkg::TL_AW
DATA_WIDTH	int	top_pkg::TL_DW
MASK_WIDTH	int	DATA_WIDTH >> 3
USER_WIDTH	int	32
ID_WIDTH	int	top_pkg::TL_AIW
VH_REGISTER_ADDRESS_OFFSET	bit [ADDR_WIDTH-1:0]	32'h0000_0100
EnableDataIntgGen	bit	1
EnableRspDataIntgCheck	bit	1

Table 500. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_o	output	tl_h2d_t
tl_i	input	tl_d2h_t
dv_i	input	logic
hld_o	output	logic
addr_i	input	logic [ADDR_WIDTH-1:0]
write_i	input	logic
wdata_i	input	logic [DATA_WIDTH-1:0]
wstrb_i	input	logic [MASK_WIDTH-1:0]
size_i	input	logic [2:0]
rdata_o	output	logic [DATA_WIDTH-1:0]
error_o	output	logic
last_i	input	logic
user_i	input	logic [USER_WIDTH-1:0]
id_i	input	logic [ID_WIDTH-1:0]
int_dv_o	output	logic
int_hld_i	input	logic
int_addr_o	output	logic [ADDR_WIDTH-1:0]
int_write_o	output	logic
int_wdata_o	output	logic [DATA_WIDTH-1:0]
int_wstrb_o	output	logic [MASK_WIDTH-1:0]
int_size_o	output	logic [2:0]
int_rdata_i	input	logic [DATA_WIDTH-1:0]
int_error_i	input	logic
int_last_o	output	logic
int_user_o	output	logic [USER_WIDTH-1:0]
int_id_o	output	logic [ID_WIDTH-1:0]

API — tlul_assert.sv

Table 501. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — module / interface / package (1개)

이름	종류
tlul_assert	module

Table 502. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — parameter (1개)

파라미터	형/폭	기본값
EndpointType		"Device" // can be either "Host" or "Device"

Table 503. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
h2d	input	tlul_pkg::tl_h2d_t
d2h	input	tlul_pkg::tl_d2h_t

Table 504. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv — #define / `define 매크로 (3개)

매크로	인자	정의
TLUL_COVER	(SEQ)	`COVER(`SEQ``_C, ``SEQ``_S, !clk_i, !rst_ni disable_sva)
TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED	(NAME)	\
TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED	(NAME)	\

API — tlul_assert_multiple.sv**Table 505. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv — module / interface / package (1개)**

이름	종류
tlul_assert_multiple	module

Table 506. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv — parameter (2개)

파라미터	형/폭	기본값
N	int unsigned	2
EndpointType		"Device" // can be "Device" or "Host"

Table 507. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv — 포트 (2개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	

API — tlul_cmd_intg_chk.sv**Table 508. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_chk.sv — module / interface / package (1개)**

이름	종류
tlul_cmd_intg_chk	module

Table 509. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_chk.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_h2d_t
err_o	output	logic

API — tlul_cmd_intg_gen.sv**Table 510. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv — module / interface / package (1개)**

이름	종류
tlul_cmd_intg_gen	module

Table 511. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv — parameter (1개)

파라미터	형/폭	기본값
EnableDataIntgGen	bit	1'b1

Table 512. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_h2d_t
tl_o	output	tl_h2d_t

API — tlul_data_integ_dec.sv**Table 513. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_dec.sv — module / interface / package (1개)**

이름	종류
tlul_data_integ_dec	module

Table 514. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_dec.sv — 포트 (2개)

포트	방향	형/폭
data_intg_i	input	[DataMaxWidth+DataIntgWidth-1:0]
data_err_o	output	logic

API — tlul_data_integ_enc.sv**Table 515. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_enc.sv — module / interface / package (1개)**

이름	종류
tlul_data_integ_enc	module

Table 516. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_enc.sv — 포트 (2개)

포트	방향	형/폭
data_i	input	[DataMaxWidth-1:0]
data_intg_o	output	logic [DataMaxWidth+DataIntgWidth-1:0]

API — tlul_err.sv**Table 517. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv — module / interface / package (1개)**

이름	종류
tlul_err	module

Table 518. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv — localparam (6개)

상수	형/폭	값
IW	int	\$bits(tl_i.a_source)
SZW	int	\$bits(tl_i.a_size)
DW	int	\$bits(tl_i.a_data)
MW	int	\$bits(tl_i.a_mask)
SubAW	int	\$clog2(DW/8)
MaskOne	bit [MW-1:0]	1

Table 519. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tl_h2d_t
err_o	output	logic

API — tlul_err_resp.sv**Table 520. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv — module / interface / package (1개)**

이름	종류
tlul_err_resp	module

Table 521. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv — parameter (1개)

파라미터	형/폭	기본값
ReturnBlankResp	bit	0

Table 522. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t

API — tlul_fifo_async.sv**Table 523. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — module / interface / package (1개)**

이름	종류
tlul_fifo_async	module

Table 524. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — parameter (2개)

파라미터	형/폭	기본값
ReqDepth	int unsigned	4

파라미터	형/폭	기본값
RspDepth	int unsigned	4

Table 525. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — localparam (2개)

상수	형/폭	값
REQFIFO_WIDTH	int unsigned	$\$bits(tlul_pkg::tl_h2d_t)-2$
RSPFIFO_WIDTH	int unsigned	$\$bits(tlul_pkg::tl_d2h_t)-2$

Table 526. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv — 포트 (8개)

포트	방향	형/폭
clk_h_i	input	
rst_h_ni	input	
clk_d_i	input	
rst_d_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t
tl_d_o	output	tlul_pkg::tl_h2d_t
tl_d_i	input	tlul_pkg::tl_d2h_t

API — tlul_fifo_sync.sv**Table 527. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — module / interface / package (1개)**

이름	종류
tlul_fifo_sync	module

Table 528. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — parameter (6개)

파라미터	형/폭	기본값
ReqPass	bit	1'b1
RspPass	bit	1'b1
ReqDepth	int unsigned	2
RspDepth	int unsigned	2
SpareReqW	int unsigned	1
SpareRspW	int unsigned	1

Table 529. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — localparam (6개)

상수	형/폭	값
REQFIFO_INTG_WIDTH	int unsigned	tlul_pkg::H2DCmdIntgWidth + tlul_pkg::DataIntgWidth
REQFIFO_WIDTH	int unsigned	$\$bits(tlul_pkg::tl_h2d_t)-2 + \text{SpareReqW}$
NumBufferBitsReqFifoIntg	int	$\$bits(\{$
RSPFIFO_INTG_WIDTH	int unsigned	$\$bits(tlul_pkg::tl_d_user_t)$

상수	형/폭	값
RSPFIFO_WIDTH	int unsigned	\$bits(tlul_pkg::tl_d2h_t) - 2 + SpareRspW -
NumBufferBitsRspFifoIntg	int	\$bits({

Table 530. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t
tl_d_o	output	tlul_pkg::tl_h2d_t
tl_d_i	input	tlul_pkg::tl_d2h_t
spare_req_i	input	[SpareReqW-1:0]
spare_req_o	output	[SpareReqW-1:0]
spare_rsp_i	input	[SpareRspW-1:0]
spare_rsp_o	output	[SpareRspW-1:0]

API — tlul_jtag_dtm.sv

Table 531. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv — module / interface / package (1개)

이름	종류
tlul_jtag_dtm	module

Table 532. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv — parameter (2개)

파라미터	형/폭	기본값
IdcodeValue	logic [31:0]	32'h00000001
NumDmiByteAbits	int unsigned	18

Table 533. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
jtag_i	input	jtag_pkg::jtag_req_t
jtag_o	output	jtag_pkg::jtag_rsp_t
scan_rst_ni	input	
scanmode_i	input	prim_mubi_pkg::mubi4_t
tl_h2d_o	output	tlul_pkg::tl_h2d_t
tl_d2h_i	input	tlul_pkg::tl_d2h_t

API — tlul_lc_gate.sv**Table 534. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — module / interface / package (1개)**

이름	종류
tlul_lc_gate	module

Table 535. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — parameter (3개)

파라미터	형/폭	기본값
NumGatesPerDirection	int	2
ReturnBlankResp	bit	0
Outstanding	int unsigned	3

Table 536. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	9

Table 537. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h2d_i	input	tl_h2d_t
tl_d2h_o	output	tl_d2h_t
tl_h2d_o	output	tl_h2d_t
tl_d2h_i	input	tl_d2h_t
flush_req_i	input	
flush_ack_o	output	logic
resp_pending_o	output	logic
lc_en_i	input	lc_tx_t
err_o	output	logic

API — tlul_pkg.sv**Table 538. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv — module / interface / package (1개)**

이름	종류
tlul_pkg	package

Table 539. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv — parameter (15개)

파라미터	형/폭	기본값
ArbiterImpl		"PPC"
H2DCmdMaxWidth	int	57
H2DCmdIntgWidth	int	7

파라미터	형/폭	기본값
H2DCmdFullWidth	int	H2DCmdMaxWidth + H2DCmdIntgWidth
D2HRspMaxWidth	int	57
D2HRspIntgWidth	int	7
D2HRspFullWidth	int	D2HRspMaxWidth + D2HRspIntgWidth
DataMaxWidth	int	32
DataIntgWidth	int	7
DataFullWidth	int	DataMaxWidth + DataIntgWidth
RsvdWidth	int	top_pkg::TL_AUW - prim_mubi_pkg::MuBi4Width -
DataWhenInstrError	logic [top_pkg::TL_DW-1:0]	'0
DataWhenError	logic [top_pkg::TL_DW-1:0]	{top_pkg::TL_DW{1'b1}}
TL_A_USER_DEFAULT	tl_a_user_t	'{
TL_D_USER_DEFAULT	tl_d_user_t	'{

Table 540. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv — localparam (3개)

상수	형/폭	값
BlankedAData	logic [top_pkg::TL_DW-1:0]	{top_pkg::TL_DW{1'b1}}
TL_H2D_DEFAULT	tl_h2d_t	'{
TL_D2H_DEFAULT	tl_d2h_t	'{

API — tlul_request_loopback.sv

Table 541. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv — module / interface / package (1개)

이름	종류
tlul_request_loopback	module

Table 542. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv — parameter (1개)

파라미터	형/폭	기본값
ErrorRsp	bit	1 // 1: Return TLUL error on on squash

Table 543. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
squash_req_i	input	logic
tl_h2d_i	input	tlul_pkg::tl_h2d_t
tl_d2h_o	output	tlul_pkg::tl_d2h_t
tl_h2d_o	output	tlul_pkg::tl_h2d_t

포트	방향	형/폭
target	input	tlul_pkg::tl_d2h_t tl_d2h_i // Response from the

API — tlul_rsp_intg_chk.sv

Table 544. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv — module / interface / package (1개)

이름	종류
tlul_rsp_intg_chk	module

Table 545. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv — parameter (1개)

파라미터	형/폭	기본값
EnableRspDataIntgCheck	bit	0

Table 546. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_d2h_t
err_o	output	logic

API — tlul_rsp_intg_gen.sv

Table 547. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv — module / interface / package (1개)

이름	종류
tlul_rsp_intg_gen	module

Table 548. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv — parameter (4개)

파라미터	형/폭	기본값
EnableRspIntgGen	bit	1'b1
EnableDataIntgGen	bit	1'b1
UserInIsZero	bit	1'b0
RspIntgInIsZero	bit	UserInIsZero

Table 549. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv — 포트 (2개)

포트	방향	형/폭
tl_i	input	tl_d2h_t
tl_o	output	tl_d2h_t

API — tlul_socket_1n.sv

Table 550. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — module / interface / package (1개)

이름	종류
tlul_socket_1n	module

Table 551. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — parameter (8개)

파라미터	형/폭	기본값
N	int unsigned	4
HReqPass	bit	1'b1
HRspPass	bit	1'b1
DReqPass	bit [N-1:0]	{N{1'b1}}
DRspPass	bit [N-1:0]	{N{1'b1}}
HReqDepth	bit [3:0]	4'h1
HRspDepth	bit [3:0]	4'h1
ExplicitErrs	bit	1'b1

Table 552. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — localparam (3개)

상수	형/폭	값
NWD	int unsigned	$\$clog2(ExplicitErrs ? N+1 : N)$
MaxOutstanding	int	$2^{**top_pkg::TL_AIW}$
OutstandingW	int	$\$clog2(MaxOutstanding+1)$

Table 553. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_h_i	input	tlul_pkg::tl_h2d_t
tl_h_o	output	tlul_pkg::tl_d2h_t
dev_select_i	input	[NWD-1:0]

API — tlul_socket_m1.sv

Table 554. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — module / interface / package (1개)

이름	종류
tlul_socket_m1	module

Table 555. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — parameter (7개)

파라미터	형/폭	기본값
M	int unsigned	4
HReqPass	bit [M-1:0]	{M{1'b1}}
HRspPass	bit [M-1:0]	{M{1'b1}}
DReqPass	bit	1'b1
DRspPass	bit	1'b1
DReqDepth	bit [3:0]	4'h1
DRspDepth	bit [3:0]	4'h1

Table 556. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — localparam (2개)

상수	형/폭	값
IDW	int unsigned	top_pkg::TL_AIW
STIDW	int unsigned	\$clog2(M)

Table 557. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_d_o	output	tlul_pkg::tl_h2d_t
tl_d_i	input	tlul_pkg::tl_d2h_t

1. lowRISC_opentitan/hw/ip/tlul/rtl/sram2tlul.sv (60 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // SRAM interface to TL-UL converter
6 // Current version only supports if TL-UL width and SRAM
width are same
7 // If SRAM interface requests more than MaxOutstanding
cap, it generates
8 // error in simulation but not in Silicon.
9
10 `include "prim_assert.sv"
11
12 module sram2tlul #(
13     parameter int          SramAw = 12,
14     parameter int          SramDw = 32,
15     parameter logic [top_pkg::TL_AW-1:0] TlBaseAddr = 'h0 //
Base address of SRAM request
16 ) (
17     input clk_i,
18     input rst_ni,
19
20     output tlul_pkg::tl_h2d_t tl_o,
21     input  tlul_pkg::tl_d2h_t tl_i,
22
23     // SRAM
24     input          mem_req_i,
25     input          mem_write_i,
26     input [SramAw-1:0] mem_addr_i,
27     input [SramDw-1:0] mem_wdata_i,
28     output logic    mem_rvalid_o,
29     output logic [SramDw-1:0] mem_rdata_o,
30     output logic [1:0] mem_error_o
31 );
32
33 import tlul_pkg::*;
34
35 `ASSERT_INIT(wrongSramDw, SramDw == top_pkg::TL_DW)
36
37 localparam int unsigned SRAM_DWB = $clog2(SramDw/8);
38
39 assign tl_o.a_valid = mem_req_i;
40 assign tl_o.a_opcode = (mem_write_i) ? PutFullData : Get;
41 assign tl_o.a_param = '0;
42 assign tl_o.a_size = top_pkg::TL_SZW'(SRAM_DWB); // Max
Size always
43 assign tl_o.a_source = '0;
44 assign tl_o.a_address = TlBaseAddr |
45 {{(top_pkg::TL_AW-SramAw-SRAM_DWB)
46 {1'b0}}, mem_addr_i, {(SRAM_DWB){1'b0}}}};
47 assign tl_o.a_mask = '1;
48 assign tl_o.a_data = mem_wdata_i;
49 assign tl_o.a_user = '0;
50 assign tl_o.d_ready = '1b1;
51
52 assign mem_rvalid_o = tl_i.d_valid && (tl_i.d_opcode ==
AccessAckData);
53 assign mem_rdata_o = tl_i.d_data;
54 assign mem_error_o = {2{tl_i.d_error}};
55
56 // below assertion fails when TL-UL doesn't accept request
in a cycle,
57 // which is currently not supported by sram2tlul
58 `ASSERT(validNotReady, tl_o.a_valid |> tl_i.a_ready)

```

```

59
60 endmodule

```

2. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_dmi.sv (183 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Tile-Link UL adapter for RISC-V debug module from the PULP
project.
6 //
7 // Note that this module can only handle one outstanding re-
quest at a time.
8
9 `include "prim_assert.sv"
10
11 module tlul_adapter_dmi
12 import tlul_pkg::*;
13 #(
14     parameter bit CmdIntgCheck      = 1, // 1: Enable command
integrity check
15     parameter bit EnableRspIntgGen = 1, // 1: Generate re-
sponse integrity
16     parameter bit EnableDataIntgGen = 1 // 1: Generate re-
sponse data integrity
17 ) (
18     input clk_i,
19     input rst_ni,
20
21     // TL-UL interface
22     input tl_h2d_t tl_h2d_i,
23     output tl_d2h_t tl_d2h_o,
24
25     // control interface
26     output logic    intg_error_o,
27
28     output logic    dmi_req_valid_o,
29     input logic     dmi_req_ready_i,
30     output dm::dmi_req_t dmi_req_o,
31
32     input logic     dmi_resp_valid_i,
33     output logic    dmi_resp_ready_o,
34     input dm::dmi_resp_t dmi_resp_i
35 );
36
37 localparam int Idw = $bits(tl_h2d_i.a_source);
38 localparam int SzW = $bits(tl_h2d_i.a_size);
39
40 logic a_ack, d_ack; // Indicates whether the corresponding
valid/ready handshake completes.
41
42 logic outstanding_q; // Indicates whether there is a pending
request.
43
44 assign a_ack = tl_h2d_i.a_valid & tl_d2h_o.a_ready;
45 assign d_ack = tl_d2h_o.d_valid & tl_h2d_i.d_ready;
46 always_ff @(posedge clk_i or negedge rst_ni) begin
47     if (!rst_ni) outstanding_q <= 'b0;
48     else if (a_ack) outstanding_q <= 'b1;
49     else if (d_ack) outstanding_q <= 'b0;
50 end
51
52 // Decode Requests
53 logic rd_req, wr_req;
54 assign wr_req = tl_h2d_i.a_valid && ((tl_h2d_i.a_opcode ==
PutFullData) ||
55                                     (tl_h2d_i.a_opcode ==

```

```

PutPartialData));
54 assign rd_req = tl_h2d_i.a_valid && (tl_h2d_i.a_opcode ==
Get);
55
56 // If we detect an error, the response will be returned im-
mediately in the next cycle
57 // and we do not send a request out to the DMI.
58 logic error_d, error_q;
59 assign dmi_req_valid_o = (wr_req || rd_req) && !error_d;
60 // The DMI response can be accepted if there is a pending
request without error.
61 assign dmi_resp_ready_o = outstanding_q && !error_q;
62

63 // We expect a word-aligned address here, otherwise an error is
returned (see further below).
64 assign dmi_req_o.addr = {2'b00, tl_h2d_i.a_ad-
dress[top_pkg::TL_AW-1:2]};
65 assign dmi_req_o.data = tl_h2d_i.a_data;
66 assign dmi_req_o.op = (wr_req) ? dm::DTM_WRITE :
67 (rd_req) ? dm::DTM_READ :
dm::DTM_NOP;
68
69 logic [IdW-1:0] reqid_q;
70 logic [SzW-1:0] reqsz_q;
71 tl_d_op_e rspop_q;
72 logic wr_req_q, rd_req_q;
73 always_ff @(posedge clk_i or negedge rst_ni) begin
74 if (!rst_ni) begin
75 reqid_q <= '0;
76 reqsz_q <= '0;
77 rspop_q <= AccessAck;
78 error_q <= 1'b0;
79 wr_req_q <= 1'b0;
80 rd_req_q <= 1'b0;
81 end else if (a_ack) begin
82 reqid_q <= tl_h2d_i.a_source;
83 reqsz_q <= tl_h2d_i.a_size;
84 // Return AccessAckData regardless of error
85 rspop_q <= (rd_req) ? AccessAckData : AccessAck;
86 rd_req_q <= rd_req;
87 wr_req_q <= wr_req;
88 error_q <= error_d;
89 end
90 end

91 // Send back a bus error if the dm_csrs module responded
with an error.
92 logic error_resp;
93 assign error_resp = error_q || (dmi_resp_i.resp ==
dm::DTM_ERR);
94
95 tlul_pkg::tl_d2h_t tl_d2h_o_pre;
96 assign tl_d2h_o_pre = '{
97 // We are ready to accept a request if there is no out-
standing one and if the DMI is ready.
98 a_ready: dmi_req_ready_i && !outstanding_q,
99 // A data response is valid if the DMI response is valid
and we've got an outstanding request.
100
101 // If the request resulted in an internal error, the error
response is returned without a DMI
102 // response, since the request to the DMI has been sup-
pressed by the error.
103 d_valid: (dmi_resp_valid_i || error_q) && outstanding_q,
104 d_opcode: rspop_q,
105 d_param: '0,
106 d_size: reqsz_q,
107 d_source: reqid_q,
108 d_sink: '0,
109 // Blank data upon error in the same way as other TL-UL
adapters would in OpenTitan.
110 d_data: (rd_req_q && !(error_resp || wr_req_q)) ?
dmi_resp_i.data : {top_pkg::TL_DW{1'b1}},
111 d_user: '0,
112 d_error: error_resp
113 };
114
115 // outgoing integrity generation
116 tlul_rsp_intg_gen #(
117 .EnableRspIntgGen(EnableRspIntgGen),
118 .EnableDataIntgGen(EnableDataIntgGen),
119 .UserInIsZero(1'b1)
120 ) u_rsp_intg_gen (
121 .tl_i(tl_d2h_o_pre),
122 .tl_o(tl_d2h_o)
123 );
124
125 logic intg_error;
126 if (CmdIntgCheck) begin : gen_cmd_intg_check
127 logic intg_error_q;
128 tlul_cmd_intg_chk u_cmd_intg_chk (
129 .tl_i(tl_h2d_i),
130 .err_o(intg_error)
131 );
132 // permanently latch integrity error until reset
133 always_ff @(posedge clk_i or negedge rst_ni) begin
134 if (!rst_ni) begin
135 intg_error_q <= 1'b0;
136 end else if (intg_error) begin
137 intg_error_q <= 1'b1;

```

```

138 end
139 end
140 assign intg_error_o = intg_error_q;
141 end else begin : gen_no_cmd_intg_check
142 assign intg_error = 1'b0;
143 assign intg_error_o = 1'b0;
144 end
145
146 ///////////////////////////////////////////////////
147 // Error Handling //
148 ///////////////////////////////////////////////////
149
150 logic addr_align_err; // Size and alignment
151 logic be_err; // Byte enable error.
152 logic malformed_meta_err; // User signal format error or un-
supported
153 logic tl_err; // Common TL-UL error checker
154 assign error_d = addr_align_err ||
155 be_err ||
156 malformed_meta_err ||
157 tl_err ||
158 intg_error;
159
160 // addr_align_err
161 // Raised if addr isn't aligned with the size
162 // Read size error is checked in tlul_assert.sv
163 // Here is it added due to the limitation of register in-
terface.
164 // Only word-align is accepted based on comportability
spec.
165 assign addr_align_err = (wr_req) && |tl_h2d_i.a_ad-
dress[1:0];
166
167 // The DMI does not support a non-all-ones byte enable.
168 assign be_err = (wr_req) && !(tl_h2d_i.a_mask);
169
170 // Don't allow unsupported values.
171 assign malformed_meta_err = tl_a_user_chk(tl_h2d_i.a_user);
172
173 // tl_err : separate checker
174 tlul_err u_err (
175 .clk_i,
176 .rst_ni,
177 .tl_i(tl_h2d_i),
178 .err_o(tl_err)
179 );
180
181 `ASSERT_INIT(DwMatchedWithDm_A, top_pkg::TL_DW ==
$bits(dmi_req_o.data))
182
183 endmodule : tlul_adapter_dmi

```

3. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_host.sv (190 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // tlul_adapter (Host adapter) converts basic req/grant/rvalid
into TL-UL interface. If
6 // MAX_REQS == 1 it is purely combinational logic. If MAX_REQS > 1
flops are required.
7 //
8 // The host driving the adapter is responsible for ensuring it
doesn't have more requests in flight
9 // than the specified MAX_REQS.
10 //
11 // The outgoing address is always word aligned. The access
size is always the word size (as
12 // specified by TL_DW). For write accesses that occupy all
lanes the operation is PutFullData,
13 // otherwise it is PutPartialData, mask is generated from
be_i. For reads all lanes are enabled as
14 // required by TL-UL (every bit in mask set).
15 //
16 // When MAX_REQS > 1 tlul_adapter_host does not do anything to
order responses from the TL-UL
17 // interface which could return them out of order. It is the
host's responsibility to either only
18 // have outstanding requests to an address space it knows will
return responses in order or to not
19 // care about out of order responses (note that if read data
is returned out of order there is no
20 // way to determine this).
21
22 `include "prim_assert.sv"
23
24 module tlul_adapter_host
25 import tlul_pkg::*;
26 import prim_mubi_pkg::mubi4_t;
27 #(
28 parameter int unsigned MAX_REQS = 2,

```

```

29 parameter bit EnableDataIntgGen = 0;
30 parameter bit EnableRspDataIntgCheck = 0
31 )
32 input clk_i,
33 input rst_ni,
34
35 input req_i,
36 output logic gnt_o,
37 input logic [top_pkg::TL_AW-1:0] addr_i,
38 input logic we_i,
39 input logic [top_pkg::TL_DW-1:0] wdata_i,
40 input logic [DataIntgWidth-1:0] wdata_intg_i,
41 input logic [top_pkg::TL_DBW-1:0] be_i,
42 input mubi4_t instr_type_i,
43 input logic [RsvdWidth-1:0] user_rsvd_i,
44
45 output logic valid_o,
46 output logic [top_pkg::TL_DW-1:0] rdata_o,
47 output logic [DataIntgWidth-1:0] rdata_intg_o,
48 output logic err_o,
49 output logic intg_err_o,
50
51 output tl_h2d_t tl_o,
52 input tl_d2h_t tl_i,
53 );
54 localparam int unsigned WordSize = $clog2(top_pkg::TL_DBW);
55
56 logic [top_pkg::TL_AIW-1:0] tl_source;
57 logic [top_pkg::TL_DBW-1:0] tl_be;
58 tl_h2d_t tl_out;
59
60 if (MAX_REQS == 1) begin : g_single_req
61   assign tl_source = '0;
62 end else begin : g_multiple_reqs
63   localparam int ReqNumW = $clog2(MAX_REQS);
64   localparam int unsigned MaxSource = MAX_REQS - 1;
65   localparam logic [ReqNumW-1:0] ReqNumOne = ReqNumW'(1'b1);
66
67   logic [ReqNumW-1:0] source_d;
68   logic [ReqNumW-1:0] source_q;
69
70   always_ff @(posedge clk_i or negedge rst_ni) begin
71     if (!rst_ni) begin
72       source_q <= '0;
73     end else begin
74       source_q <= source_d;
75     end
76   end
77
78   always_comb begin
79     source_d = source_q;
80
81     if (req_i && gnt_o) begin
82       if (source_q == MaxSource[ReqNumW-1:0]) begin
83         source_d = '0;
84       end else begin
85         source_d = source_q + ReqNumOne;
86       end
87     end
88   end
89
90   assign tl_source = top_pkg::TL_AIW'(source_q);
91 end
92
93 // For TL-UL Get opcode all active bytes must have their
mask bit set, so all reads get all tl_be
94 // bits set. For writes the supplied be_i is used as the
mask.
95 assign tl_be = ~we_i ? {top_pkg::TL_DBW{1'b1}} : be_i;
96
97 assign tl_out = '{
98   a_valid: req_i,
99   a_opcode: (~we_i) ? Get :
100   (&be_i) ? PutFullData :
101   PutPartialData,
102   a_param: 3'h0,
103   a_size: top_pkg::TL_SZW'(WordSize),
104   a_mask: tl_be,
105   a_source: tl_source,
106   a_address: {addr_i[31:WordSize], {WordSize{1'b0}}},
107   a_data: wdata_i,
108   a_user: '{default: '0, data_intg: wdata_intg_i, in-
str_type: instr_type_i, rsvd: user_rsvd_i},
109   d_ready: 1'b1
110 };
111
112 tlul_cmd_intg_gen #(EnableDataIntgGen (EnableDataIntgGen))
u_cmd_intg_gen (
113   .tl_i(tl_i),
114   .tl_o(tl_o)
115 );
116
117 assign gnt_o = tl_i.a_ready;
118
119 assign valid_o = tl_i.d_valid;
120 assign rdata_o = tl_i.d_data;
121 assign rdata_intg_o = tl_i.d_user.data_intg;
122
123 logic intg_err;
124 tlul_rsp_intg_chk #(
125   .EnableRspDataIntgCheck(EnableRspDataIntgCheck)

```

```

126 ) u_rsp_chk (
127   .tl_i,
128   .err_o(intg_err)
129 );
130
131 logic intg_err_q;
132 always_ff @(posedge clk_i or negedge rst_ni) begin
133   if (!rst_ni) begin
134     intg_err_q <= '0;
135   end else if (intg_err) begin
136     intg_err_q <= 1'b1;
137   end
138 end
139
140 // err_o is transactional. This allows the host to continue
141 // debug without receiving an endless stream of errors.
142 assign err_o = tl_i.d_error | intg_err;
143
144 // intg_err_o is permanent once detected, and should be used
145 // to trigger alerts
146 assign intg_err_o = intg_err_q | intg_err;
147
148 // Addresses are assumed to be word-aligned, and the bottom
bits are ignored
149 logic unused_addr_bottom_bits;
150 assign unused_addr_bottom_bits = ^addr_i[WordSize-1:0];
151
152 // Explicitly ignore unused fields of tl_i
153 logic unused_tl_i_fields;
154 assign unused_tl_i_fields = ^{tl_i.d_opcode, tl_i.d_param,
tl_i.d_size, tl_i.d_source,
tl_i.d_sink,
tl_i.d_user};
155
156 `ifdef INC_ASSERT
157 //VCS coverage off
158 // pragma coverage off
159 localparam int OutstandingReqCntW =
160   (MAX_REQS == 2 ** $clog2(MAX_REQS)) ? $clog2(MAX_REQS) +
1 : $clog2(MAX_REQS);
161 localparam logic [OutstandingReqCntW-1:0] OutstandingRe-
qCntOne = OutstandingReqCntW'(1'b1);
162
163 logic [OutstandingReqCntW-1:0] outstanding_reqs_q;
164 logic [OutstandingReqCntW-1:0] outstanding_reqs_d;
165
166 always_comb begin
167   outstanding_reqs_d = outstanding_reqs_q;
168
169   if ((req_i && gnt_o) && !valid_o) begin
170     outstanding_reqs_d = outstanding_reqs_q + OutstandingRe-
qCntOne;
171   end else if (!(req_i && gnt_o) && valid_o) begin
172     outstanding_reqs_d = outstanding_reqs_q - OutstandingRe-
qCntOne;
173   end
174 end
175
176 always_ff @(posedge clk_i or negedge rst_ni) begin
177   if (!rst_ni) begin
178     outstanding_reqs_q <= '0;
179   end else begin
180     outstanding_reqs_q <= outstanding_reqs_d;
181   end
182 end
183
184 //VCS coverage on
185 // pragma coverage on
186
187 `ASSERT(DontExceedMaxReqs, req_i |-> outstanding_reqs_d <=
MAX_REQS)
188 `endif
189 endmodule

```

4. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_racl.sv (114 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL adapter with RACL support
9  *
10  * - It allows filtering TLUL requests based on their range
using an allow-list approach. A request
11  * is allowed if its address matches a racl range which al-
lows access for the racl role associated
12  * with this request.
13  */
14 module tlul_adapter_racl
15   import tlul_pkg::*;
16   import prim_mubi_pkg::mubi4_t;
17 #()
18   parameter bit EnableRacl = 0, // 1: En-
able RACL checks on access
19   parameter bit RaclErrorRsp = EnableRacl, // 1: Re-

```

```

turn TLUL error on RACL errors

20 parameter int RACLPolicySelNumRanges = 1 // Number of
ranges with a RACL policy
21 ) (
22 in-
put
23 in-
put
24 // TL-UL interface
25 input
tlul_pkg::tl_h2d_t
26 output
tlul_pkg::tl_d2h_t
27 output
tlul_pkg::tl_h2d_t
28 input
tl_filtered_h2d_o,
tlul_pkg::tl_d2h_t
29 input
tl_filtered_d2h_i,
29 // RACL interface
30 input
top_racl_pkg::racl_policy_vec_t
racl_policies_i,
31 output
top_racl_pkg::racl_error_log_t
racl_er-
ror_o,
32 input top_racl_pkg::racl_range_t [RACLPolicySelNum-
Ranges-1:0] racl_policy_sel_ranges_i
33 );
34 if (EnableRacl) begin : gen_racl_role_logic
35 // Retrieve RACL role from user bits and one-hot encode
that for the comparison bitmap
36 top_racl_pkg::racl_role_t racl_role;
37 assign racl_role =
top_racl_pkg::tlul_extract_racl_role_bits(tl_h2d_i.a_user.rsvd);
38
39 top_racl_pkg::racl_role_vec_t racl_role_vec;
40 prim_onehot_enc #(
41 .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
42 ) u_racl_role_encode (
43 .in_i ( racl_role ),
44 .en_i ( 1'b1 ),
45 .out_o ( racl_role_vec );
46 );
47
48 logic rd_req, wr_req, racl_read_allowed,
racl_write_allowed, racl_error;
49 logic [RACLPolicySelNumRanges-1:0] range_read_allowed;
50 logic [RACLPolicySelNumRanges-1:0] range_write_allowed;
51
52 for (genvar r = 0; r < RACLPolicySelNumRanges; r++) be-
gin : gen_racl_range_check
53 top_racl_pkg::racl_range_t range;
54 top_racl_pkg::racl_policy_t policy;
55 logic range_match;
56 assign range = racl_policy_sel_ranges_i[r];
57 assign policy = racl_policies_i[range.policy_sel];
58 // Check if the address is within range
59 assign range_match = range.enable
60 & tl_h2d_i.a_address >= range.base
61 & tl_h2d_i.a_address <=
range.limit;
62 // If address matches, lookup permissions for policy
defined for this range
63 assign range_read_allowed[r] = range_match & |
(policy.read_perm & racl_role_vec);
64 assign range_write_allowed[r] = range_match & |
(policy.write_perm & racl_role_vec);
65 end
66
67 assign racl_read_allowed = |range_read_allowed;
68 assign racl_write_allowed = |range_write_allowed;
69
70 assign rd_req = tl_h2d_i.a_valid &
(tl_h2d_i.a_opcode == tlul_pkg::Get);
71 assign wr_req = tl_h2d_i.a_valid &
(tl_h2d_i.a_opcode == tlul_pkg::PutFullData |
72 tl_h2d_i.a_opcode == tlul_pkg::PutPartialData);
73 assign racl_error = (rd_req & ~racl_read_allowed)
| (wr_req & ~racl_write_allowed);
74 assign racl_error_o.valid = racl_error & tl_d2h_o.a_ready;
75
76 tlul_request_loopback #(
77 .ErrorRsp ( RaclErrorRsp )
78 ) u_loopback (
79 .clk_i ( clk_i ),
80 .rst_ni ( rst_ni ),
81 .squash_req_i ( racl_error ),
82 .tl_h2d_i ( tl_h2d_i ),
83 .tl_d2h_o ( tl_d2h_o ),
84 .tl_h2d_o ( tl_filtered_h2d_o ),
85 .tl_d2h_i ( tl_filtered_d2h_i );
86 );
87
88 // Collect RACL error information
89 assign racl_error_o.overflow = 1'b0;
90 assign racl_error_o.read_access = tl_h2d_i.a_opcode ==
tlul_pkg::Get;
91 assign racl_error_o.racl_role = racl_role;

```

```

92 assign racl_error_o.ctn_uid =
top_racl_pkg::tlul_extract_ctn_uid_bits(tl_h2d_i.a_user.rsvd);
93 assign racl_error_o.request_address = tl_h2d_i.a_address;
94 end else begin : gen_no_racl_role_logic
95 // Pass through and default assignments
96 assign tl_filtered_h2d_o = tl_h2d_i;
97 assign tl_d2h_o = tl_filtered_d2h_i;
98 assign racl_error_o = '0;
99
100 // Some signals are not read when RACL is disabled
101 logic unused_signals;
102 assign unused_signals = ^racl_policy_sel_ranges_i;
103 end
104
105 // Not all RACL policies are used, even if RACL is enabled
106 logic unused_policy_sel;
107 assign unused_policy_sel = ^racl_policies_i;
108
109 `ASSERT(RaclAdapterNumRanges, EnableRacl |-> RACLPolicySel-
NumRanges > 0)
110
111 // Ensure that RACL signals are not undefined
112 `ASSERT_KNOWN(RaclAdapterErrorKnown_A, racl_error_o.valid)
113
114 endmodule

```

5. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg.sv (307 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 // Tile-Link UL to register interface adapter
8 //
9 // This module acts as a device, addressed through the TL-UL
bus. It receives Get, PutPartialData
10 // and PutFullData messages on the A channel. These are read
or write requests, which are passed to
11 // the register interface on the same cycle. The response from
the register interface is sent back
12 // to the host as a message on the D channel on the next
cycle.
13 //
14 // A module instance performs several checks on A-channel mes-
sages and only forwards a request to
15 // the register interface if they all pass. These checks are:
16 //
17 // - A_OPCODE must be Get, PutPartialData or PutFullData.
18 //
19 // - A_SIZE must be 0, 1 or 2 (giving a 1-, 2- or 4-byte op-
eration, respectively).
20 //
21 // - A_ADDRESS must be naturally aligned for the operation
byte width signaled in A_SIZE.
22 //
23 // - A_MASK must only be true for active byte-lanes. The pos-
ition of the first of these active
24 // lanes is calculated from the address, modulo the width
of the TL-UL data bus.
25 //
26 // - If the A_USER.INSTR_TYPE value is MuBi4True, A_OPCODE
must be Get.
27 //
28 // - Either the A_USER.INSTR_TYPE value must be a value other
than MuBi4True or the en_ifetch_i
29 // port must be MuBi4True.
30 //
31 // - If the CmdIntgCheck parameter is true then the command
integrity check must pass.
32 //
33 //
34 // ** Parameters
35 //
36 // CmdIntgCheck: If this parameter is true then there is a
"command integrity check". This uses a
37 // tlul_cmd_intg_chk instance to compare integ-
rity checksums from requests on the A
38 // channel. If the integrity checksums don't
match, the adapter doesn't pass the
39 // request to the register interface through
re_o/we_o and the response on the D
40 // channel will then have its D_ERROR flag set.
41 //
42 // EnableRspIntgGen: OpenTitan sends response integrity with
messages on the TL-UL bus. If this
43 // parameter is true, the module will per-
form calculations to send a correct
44 // integrity value in the response's
A_USER.RSP_INTG field.
45 //
46 // EnableDataIntgGen: This parameter is analogous to EnableR-
spIntgGen but controls whether the
47 // module performs calculations to send a

```



```

correct integrity value in the
48 //           response's A_USER.DATA_INTG field.
49 //
50 // RegAw: The number of bits used for addresses in the re-
gister interface. Higher bits of the
51 //           A_ADDRESS field in an A channel message are ignored.
52 //
53 // RegDw: The number of bits used for data words in the re-
gister interface. This is assumed to match
54 //           the data width of the TL-UL bus (see the Matched-
Width_A assertion).
55 //
56 // AccessLatency: This adapter always returns TileLink re-
sponses after one cycle, but this parameter

57 //           allows it to handle different timings on the
register interface. If AccessLatency
58 //           is zero then the adapter expects a combinat-
orial response on the register
59 //           interface and flops the result to break
paths and add a cycle delay. If
60 //           AccessLatency is one then the adapter ex-
pects a response on the register interface

61 //           in the cycle after the re_o/we_o signal goes
high, but then forwards it
62 //           combinatorially to the TileLink interface.
Only values 0 and 1 are allowed for
63 //           this parameter.
64 //
65 //
66 // ** Ports
67 //
68 // clk_i/rst_ni: Clock and reset.
69 // tl_i/tl_o: Connections to the TL-UL bus.
70 // en_ifetch_i: This mubi4_t port controls whether to allow
read requests from Get messages whose

71 //           A_USER.INSTR_TYPE is MuBi4True (considered to be
processor "fetch" requests).
72 // intg_error_o: This port is high when the adapter has detec-
ted a command integrity error. That can
73 //           only happen if the CmdIntgCheck parameter is
set.

74 // re_o/we_o: The read-enable and write-enable ports in the
register interface.
75 // addr_o: The address to read or write through the re-
gister interface.
76 // wdata_o: The value to write through the register in-
terface if we_o is true.
77 // be_o: A byte-enable signal, which is true for each
byte-lane where the TL-UL operation's
78 // mask is true.
79 // busy_i: If this input is high then the A channel in-
terface is stalled (by dropping
80 //           A_READY), which means that the adapter will
not accept any new operations.
81 // rdata_i: Read data being returned for a read request
through the register interface.

82 // error_i: An error flag in the register interface. This
signals that an error has been seen,
83 //           which can be seen in the D channel response
by D_ERROR being high. The D_DATA
84 //           response is also set to '1, squashing any
data that might have been returned
85 //           through rdata_i.
86 //
87 // ICEBOX(#15822): Note that due to some modules with special
needs (like the vendored-in RV_DM),
88 //           this module has been extended so that it
supports use cases outside of the
89 //           generated reg_top module. This makes this
adapter and its parameterization
90 //           options a bit heavy.
91 //
92 //           We should in the future come back to this
and refactor / align the module and its
93 //           parameterization needs.
94 //
95 module tlul_adapter_reg
96 import tlul_pkg::*;
97 import prim_mubi_pkg::mubi4_t;
98 #(
99     parameter bit CmdIntgCheck = 0, // 1: Enable command
integrity check
100     parameter bit EnableRspIntgGen = 0, // 1: Generate re-
sponse integrity
101     parameter bit EnableDataIntgGen = 0, // 1: Generate re-
sponse data integrity
102     parameter int RegAw = 8, // Width of register
address
103     parameter int RegDw = 32, // Shall be matched
with TL_DW
104     parameter int AccessLatency = 0, // 0: same cycle, 1:
next cycle
105     localparam int RegBw = RegDw/8
106 ) (
107     input clk_i,
108     input rst_ni,

```

```

109
110 // TL-UL interface
111 input tl_h2d_t tl_i,
112 output tl_d2h_t tl_o,
113
114 // control interface
115 input mubi4_t en_ifetch_i,
116 output logic intg_error_o,
117
118 // Register interface
119 output logic re_o,
120 output logic we_o,
121 output logic [RegAw-1:0] addr_o,
122 output logic [RegDw-1:0] wdata_o,
123 output logic [RegBw-1:0] be_o,
124 input busy_i,
125 // The following two signals are expected
126 // to be returned in AccessLatency cycles.
127 input [RegDw-1:0] rdata_i,
128 // This can be a write or read error.
129 input error_i
130 );
131
132 `ASSERT_INIT(AllowedLatency_A, AccessLatency inside {0, 1})
133
134 localparam int IW = $bits(tl_i.a_source);
135 localparam int SZW = $bits(tl_i.a_size);
136
137 logic outstanding_q; // Indicates current request is
pending
138 logic a_ack, d_ack;
139
140 logic [RegDw-1:0] rdata, rdata_q;

141 logic error_q, error, err_internal, instr_error,
intg_error;
142
143 logic addr_align_err; // Size and alignment
144 logic tl_err; // Common TL-UL error checker
145
146 logic [IW-1:0] reqid_q;
147 logic [SZW-1:0] reqsz_q;
148 tl_d_op_e rspop_q;
149
150 logic rd_req, wr_req;
151
152 assign a_ack = tl_i.a_valid & tl_o.a_ready;
153 assign d_ack = tl_o.d_valid & tl_i.d_ready;
154 // Request signal
155 assign wr_req = a_ack & ((tl_i.a_opcode == PutFullData) |
(tl_i.a_opcode == PutPartialData));
156 assign rd_req = a_ack & (tl_i.a_opcode == Get);
157
158 assign we_o = wr_req & ~err_internal;
159 assign re_o = rd_req & ~err_internal;
160 assign wdata_o = tl_i.a_data;
161 assign be_o = tl_i.a_mask;
162
163 if (RegAw <= 2) begin : gen_only_one_reg
164     assign addr_o = '0;
165 end else begin : gen_more_regs
166     assign addr_o = {tl_i.a_address[RegAw-1:2], 2'b00}; //
generate always word-align
167 end
168
169 always_ff @(posedge clk_i or negedge rst_ni) begin
170     if (!rst_ni) outstanding_q <= 'b0;
171     else if (a_ack) outstanding_q <= 'b1;
172     else if (d_ack) outstanding_q <= 'b0;
173 end
174
175 always_ff @(posedge clk_i or negedge rst_ni) begin
176     if (!rst_ni) begin
177         reqid_q <= '0;
178         reqsz_q <= '0;
179         rspop_q <= AccessAck;
180     end else if (a_ack) begin
181         reqid_q <= tl_i.a_source;
182         reqsz_q <= tl_i.a_size;
183         // Return AccessAckData regardless of error
184         rspop_q <= (rd_req) ? AccessAckData : AccessAck ;
185     end
186 end
187
188 if (AccessLatency == 1) begin : gen_access_latency1
189     logic a_ack_q, err_internal_q, wr_req_q;
190     always_ff @(posedge clk_i or negedge rst_ni) begin
191         if (!rst_ni) begin
192             a_ack_q <= 'b0;
193             err_internal_q <= 'b0;
194             wr_req_q <= 'b0;
195             rdata_q <= '0;
196             error_q <= 'b0;
197         end else begin
198             a_ack_q <= a_ack;
199             err_internal_q <= err_internal;
200             wr_req_q <= wr_req;
201
202             rdata_q <= rdata;
203             error_q <= error;
204         end

```

```

205     end
206     always_comb begin
207         if (a_ack_q) begin
208             rdata = (error_i || err_internal_q || wr_req_q) ? '1 :
rdata_i;
209             error = error_i || err_internal_q;
210         end else begin
211             rdata = rdata_q;
212             error = error_q;
213         end
214     end
215 end else begin : gen_access_latency0
216     always_ff @(posedge clk_i or negedge rst_ni) begin
217         if (!rst_ni) begin
218             rdata_q <= '0;
219             error_q <= 1'b0;
220         end else if (a_ack) begin
221             rdata_q <= (error_i || err_internal || wr_req) ? '1 :
rdata_i;
222             error_q <= error_i || err_internal;
223         end
224     end
225     assign rdata = rdata_q;
226     assign error = error_q;
227 end
228
229 tlul_pkg::tl_d2h_t tl_o_pre;
230 assign tl_o_pre = '{
231     a_ready: ~(outstanding_q | busy_i),
232     d_valid: outstanding_q,
233     d_opcode: rspop_q,
234     d_param: '0,
235     d_size: reqsz_q,
236     d_source: reqid_q,
237     d_sink: '0,
238     d_data: rdata,
239     d_user: '0,
240     d_error: error
241 };
242
243 // outgoing integrity generation
244 tlul_rsp_intg_gen #(
245     .EnableRspIntgGen(EnableRspIntgGen),
246     .EnableDataIntgGen(EnableDataIntgGen),
247     .UserInIsZero(1'b1)
248 ) u_rsp_intg_gen (
249     .tl_i(tl_o_pre),
250     .tl_o(tl_o)
251 );
252
253 if (CmdIntgCheck) begin : gen_cmd_intg_check
254     logic intg_error_q;
255     tlul_cmd_intg_chk u_cmd_intg_chk (
256         .tl_i(tl_i),
257         .err_o(intg_error)
258     );
259     // permanently latch integrity error until reset
260     always_ff @(posedge clk_i or negedge rst_ni) begin
261         if (!rst_ni) begin
262             intg_error_q <= 1'b0;
263         end else if (intg_error) begin
264             intg_error_q <= 1'b1;
265         end
266     end
267     assign intg_error_o = intg_error_q;
268 end else begin : gen_no_cmd_intg_check
269     assign intg_error = 1'b0;
270     assign intg_error_o = 1'b0;
271 end
272
273 // Error Handling //
274 // Error Handling //
275 // Error Handling //
276
277 // An instruction type transaction is only valid if
en_ifetch is enabled
278 assign instr_error =
prim_mubi_pkg::mubi4_test_true_strict(tl_i.a_user.instr_type) &
279
prim_mubi_pkg::mubi4_test_false_loose(en_ifetch_i);
280
281 assign err_internal = addr_align_err | tl_err | instr_error
| intg_error;
282
283 // addr_align_err
284 // Raised if addr isn't aligned with the size
285 // Read size error is checked in tlul_assert.sv
286 // Here is it added due to the limitation of register in-
terface.
287 always_comb begin
288     if (wr_req) begin
289         // Only word-align is accepted based on comportability
spec
290         addr_align_err = |tl_i.a_address[1:0];
291     end else begin
292         // No request
293         addr_align_err = 1'b0;
294     end
295 end
296

```

```

297 // tl_err : separate checker
298 tlul_err u_err (
299     .clk_i,
300     .rst_ni,
301     .tl_i,
302     .err_o (tl_err)
303 );
304
305 `ASSERT_INIT(MatchedWidth_A, RegDw == top_pkg::TL_DW)
306
307 endmodule

```

6. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_reg_racl.sv (137 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL adapter for Register interface with RACL pro-
tection
9  */
10
11 module tlul_adapter_reg_racl
12     import tlul_pkg::*;
13     import prim_mubi_pkg::mubi4_t;
14     #(
15         parameter bit CmdIntgCheck      = 0,           // 1: Enable
command integrity check
16         parameter bit EnableRspIntgGen  = 0,           // 1: Gener-
ate response integrity
17         parameter bit EnableDataIntgGen = 0,           // 1: Gener-
ate response data integrity
18         parameter int RegAw              = 8,           // Width of
register address
19         parameter int RegDw              = 32,           // Shall be
matched with TL_DW
20         parameter int AccessLatency      = 0,           // 0: same
cycle, 1: next cycle
21         parameter bit EnableRacl         = 0,           // 1: Enable
RACL checks on access
22         parameter bit RaclErrorRsp       = EnableRacl, // 1: Return
TLUL error on RACL errors
23         parameter top_racl_pkg::racl_policy_sel_t RaclPolicySelVec =
0, // RACL policy for this reg
24
25         // adapter
26         localparam int RegBw              = RegDw/8
27     ) (
28         input clk_i,
29         input rst_ni,
30
31         // TL-UL interface
32         input tl_h2d_t tl_i,
33         output tl_d2h_t tl_o,
34
35         // control interface
36         input mubi4_t en_ifetch_i,
37         output logic intg_error_o,
38
39         // RACL interface
40         input top_racl_pkg::racl_policy_vec_t racl_policies_i,
41         output top_racl_pkg::racl_error_log_t racl_error_o,
42
43         // Register interface
44         output logic re_o,
45         output logic we_o,
46         output logic [RegAw-1:0] addr_o,
47         output logic [RegDw-1:0] wdata_o,
48         output logic [RegBw-1:0] be_o,
49         input busy_i,
50
51         // The following two signals are expected
52         // to be returned in AccessLatency cycles.
53         input [RegDw-1:0] rdata_i,
54         // This can be a write or read error.
55         input error_i
56     );
57
58 tl_h2d_t tl_h2d_filtered;
59 tl_d2h_t tl_d2h_filtered;
60
61 if (EnableRacl) begin : gen_racl_role_logic
62     // Retrieve RACL role from user bits and one-hot encode
that for the comparison bitmap
63     top_racl_pkg::racl_role_t racl_role;
64     assign racl_role =
top_racl_pkg::tlul_extract_racl_role_bits(tl_i.a_user.rsvd);
65
66     top_racl_pkg::racl_role_vec_t racl_role_vec;
67     prim_onehot_enc #(
68         .OneHotWidth( $bits(top_racl_pkg::racl_role_vec_t) )
69     )

```

```

66 ) u_racl_role_encode (
67   .in_i ( racl_role ),
68   .en_i ( 1'b1 ),
69   .out_o ( racl_role_vec )
70 );
71
72 logic rd_req, wr_req, racl_read_allowed,
73 racl_write_allowed, racl_error;
74 assign rd_req = tl_i.a_valid & (tl_i.a_opcode
== tlul_pkg::Get);
75 assign wr_req = tl_i.a_valid & (tl_i.a_opcode
== tlul_pkg::PutFullData |
76 tl_i.a_opcode
== tlul_pkg::PutPartialData);
77 assign racl_read_allowed = (!
78 (racl_policies_i[RaclPolicySelVec].read_perm & racl_role_vec));
79 assign racl_write_allowed = (!
80 (racl_policies_i[RaclPolicySelVec].write_perm & racl_role_vec));
81 assign racl_error = (rd_req & ~racl_read_allowed)
82 | (wr_req & ~racl_write_allowed);
83 assign racl_error_o.valid = racl_error & tl_o.a_ready;
84
85 tlul_request_loopback #(
86   .ErrorRsp(RaclErrorRsp)
87 ) u_loopback (
88   .clk_i,
89   .rst_ni,
90   .squash_req_i ( racl_error ),
91   .tl_h2d_i ( tl_i ),
92   .tl_d2h_o ( tl_o ),
93   .tl_h2d_o ( tl_h2d_filtered ),
94   .tl_d2h_i ( tl_d2h_filtered )
95 );
96
97 // Collect RACL error information
98 assign racl_error_o.overflow = 1'b0;
99 assign racl_error_o.read_access = tl_i.a_opcode ==
100 tlul_pkg::Get;
101 assign racl_error_o.racl_role = racl_role;
102 assign racl_error_o.ctn_uid =
103 top_racl_pkg::tlul_extract_ctn_uid_bits(tl_i.a_user.rsvd);
104 assign racl_error_o.request_address = tl_i.a_address;
105 end else begin : gen_no_racl_role_logic
106 // Pass through and default assignments
107 assign tl_h2d_filtered = tl_i;
108 assign tl_o = tl_d2h_filtered;
109 assign racl_error_o = '0;
110 end
111
112 tlul_adapter_reg #(
113   .CmdIntgCheck (CmdIntgCheck),
114   .EnableRspIntgGen (EnableRspIntgGen),
115   .EnableDataIntgGen (EnableDataIntgGen),
116   .RegAw (RegAw),
117   .RegDw (RegDw),
118   .AccessLatency (AccessLatency)
119 ) tlul_adapter_reg (
120   .clk_i,
121   .rst_ni,
122   .tl_i(tl_h2d_filtered),
123   .tl_o(tl_d2h_filtered),
124   .en_ifetch_i,
125   .intg_error_o,
126   .re_o,
127   .we_o,
128   .addr_o,
129   .wdata_o,
130   .be_o,
131   .busy_i,
132   .rdata_i,
133   .error_i
134 );
135
136 // Not all RACL policies are used, even if RACL is enabled
137 logic unused_policy;
138 assign unused_policy = ^racl_policies_i;
139
140 // Ensure that RACL signals are not undefined
141 `ASSERT_KNOWN(RaclAdapterRegErrorKnown_A, racl_er-
142 ror_o.valid)
143
144 endmodule

```

7. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_shim.sv (149 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5
6 `include "prim_assert.sv"
7
8 module tlul_adapter_vh
9   import tlul_pkg::*;
10   import prim_mubi_pkg::mubi4_t;
11
12   #(
13     parameter int ADDR_WIDTH = top_pkg::TL_AW,
14     parameter int DATA_WIDTH = top_pkg::TL_DW,

```

```

13   parameter int MASK_WIDTH = DATA_WIDTH >> 3,
14   parameter int USER_WIDTH = 32,
15   parameter int ID_WIDTH = top_pkg::TL_AIW,
16
17   parameter bit [ADDR_WIDTH-1:0] VH_REGISTER_ADDRESS_OFFSET =
32'h0000_0100,
18
19   parameter bit EnableDataIntgGen = 1,
20   parameter bit EnableRspDataIntgCheck = 1
21 ) (
22   input clk_i,
23   input rst_ni,
24
25   output tl_h2d_t tl_o,
26   input tl_d2h_t tl_i,
27
28   // Valid-Hold device interface (VH to TLUL).
29   input logic dv_i,
30   output logic hld_o,
31   input logic [ADDR_WIDTH-1:0] addr_i,
32   input logic write_i,
33   input logic [DATA_WIDTH-1:0] wdata_i,
34   input logic [MASK_WIDTH-1:0] wstrb_i,
35   input logic [2:0] size_i,
36   output logic [DATA_WIDTH-1:0] rdata_o,
37   output logic error_o,
38   input logic last_i,
39   input logic [USER_WIDTH-1:0] user_i,
40   input logic [ID_WIDTH-1:0] id_i,
41
42   // Valid-Hold host interface (VH to internal registers). The
43   signals from the VH device interface
44   // are routed to the VH host interface for every internal
45   access, see the `internal_access` signal.
46   output logic int_dv_o,
47   input logic int_hld_i,
48   output logic [ADDR_WIDTH-1:0] int_addr_o,
49   output logic int_write_o,
50   output logic [DATA_WIDTH-1:0] int_wdata_o,
51   output logic [MASK_WIDTH-1:0] int_wstrb_o,
52   output logic [2:0] int_size_o,
53   input logic [DATA_WIDTH-1:0] int_rdata_i,
54   input logic int_error_i,
55   output logic int_last_o,
56   output logic [USER_WIDTH-1:0] int_user_o,
57   output logic [ID_WIDTH-1:0] int_id_o,
58 );
59
60 // The adapter distinguishes between two types of VH ac-
61 cesses: those going to an internal register
62 // file and those targeting OpenTitan registers, with the
63 latter ones being translated into TLUL
64 // requests while the other requests are relayed to the in-
65 ternal register file.
66 logic internal_access;
67 assign internal_access = addr_i >= VH_RE-
68 GISTER_ADDRESS_OFFSET;
69
70 // Differentiate between two levels of acknowledgements:
71
72 // - `req_ack`: A host-to-device request is acknowledged by
73 the device, meaning that the response
74 // - `resp_ack`: The device-to-host response is acknow-
75 ledged.
76 logic pending_d, pending_q;
77 logic req_ack, resp_ack;
78
79 assign req_ack = dv_i & tl_i.a_ready & tl_o.a_valid & ~in-
80 ternal_access;
81 assign resp_ack = dv_i & tl_o.d_ready & tl_i.d_valid & ~in-
82 ternal_access;
83
84 always_comb begin
85   pending_d = pending_q;
86   if (req_ack) begin
87     pending_d = 1'b1;
88   end else if (resp_ack) begin
89     pending_d = 1'b0;
90   end
91 end
92
93 always_ff @(posedge clk_i or negedge rst_ni) begin
94   if (!rst_ni) begin
95     pending_q <= 1'b0;
96   end else begin
97     pending_q <= pending_d;
98   end
99 end
100
101 // Only make a new request (through `a_valid`) to the device
102 when none are pending. If the
103
104 // integrity checks are enabled, the `a_user` fields will be
105 set by the corresponding module (see
106 // `tlul_cmd_intg_gen`).
107 tlul_pkg::tl_h2d_t tl_o_pre;
108 assign tl_o_pre = '{
109   a_valid: dv_i & ~pending_q & ~internal_access,

```

```

97  a_opcode: ~write_i ? Get : (&wstrb_i ? PutFullData : Put-
PartialData),
98  a_size: top_pkg::TL_SZW'(size_i),
99  a_mask: wstrb_i,
100 a_source: id_i,
101 a_address: addr_i,
102 a_data: wdata_i,
103 a_user: '{default: '0, instr_type:
prim_mubi_pkg::Mubi4False},
104 d_ready: '1b1,
105 // unused
106 a_param: '3'h0
107 };
108
109 // Accesses to device registers are acknowledged with the
TLUL d-channel handshake.

110 assign hld_o = !pending_q && internal_access && !int_hld_i ?
1'b0 :
111         pending_q && tl_o.d_ready && tl_i.d_valid ?
1'b0 : '1b1;
112
113 assign rdata_o = internal_access ? int_rdata_i :
tl_i.d_data;
114
115 // Route signals to the VH host interface.
116 assign int_dv_o = dv_i & internal_access;
117 assign int_addr_o = addr_i;
118 assign int_write_o = write_i;
119 assign int_wdata_o = wdata_i;
120 assign int_wstrb_o = wstrb_i;
121 assign int_size_o = size_i;
122 assign int_last_o = last_i;
123 assign int_user_o = user_i;
124 assign int_id_o = id_i;
125
126 tlul_cmd_intg_gen #(
127     .EnableDataIntgGen (EnableDataIntgGen)
128 ) u_cmd_intg_gen (
129     .tli ( tl_o_pre ),
130     .tlo ( tl_o )
131 );
132
133 logic intg_err_chk;
134 tlul_rsp_intg_chk #(
135     .EnableRspDataIntgCheck(EnableRspDataIntgCheck)
136 ) u_rsp_chk (
137     .tli ( tli ),
138     .err_o ( intg_err_chk )
139 );
140
141 assign error_o = tl_i.d_error | int_error_i | intg_err_chk;
142
143 // Make sure the VH parameters are compatible with TLUL
datapath widths.
144 `ASSERT_INIT(TlInvalidAddrWidth_A, ADDR_WIDTH ==
top_pkg::TL_AW)
145 `ASSERT_INIT(TlInvalidDataWidth_A, DATA_WIDTH ==
top_pkg::TL_DW)
146 `ASSERT_INIT(TlInvalidMaskWidth_A, MASK_WIDTH ==
top_pkg::TL_DBW)
147 `ASSERT_INIT(TlInvalidUserWidth_A, ID_WIDTH ==
top_pkg::TL_AIW)
148
149 endmodule

```

8. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram.sv (688 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL adapter for SRAM-like devices
9  *
10  * - Intentionally omitted BaseAddr in case of multiple memory
maps are used in a SoC,
11  *   it means that aliasing can happen if target device size
in TL-UL crossbar is bigger
12  *   than SRAM size
13  * - At most one of EnableDataIntgGen / EnableDataIntgPt can
be enabled. However it
14  *   possible for both to be disabled.
15  * - A module can neither generate an integrity response nor
pass through any pre-existing
16  *   integrity. This might be the case for non-security crit-
ical memories where there is
17  *   no stored integrity AND another entity upstream is
already generating returning integrity.
18  * - There is however no case where EnableDataIntgGen and En-
ableDataIntgPt are both true.
19  */
20 module tlul_adapter_sram

```

```

21 import tlul_pkg::*;
22 import prim_mubi_pkg::mubi4_t;
23 #(
24     parameter int SramAw          = 12,
25     parameter int SramDw          = 32, // Must be multiple of
the TL width
26     parameter int SramDepth       = 2**SramAw, // Must be <=
2**SramAw
27     parameter int Outstanding     = 1, // Only one request is
accepted
28     parameter int SramBusBankAW  = 12, // SRAM bus address
width of the SRAM bank. Only used
29                                     // when DataXorAddr=1.
30     parameter bit ByteAccess      = 1, // 1: Enables sub-word
write transactions. Note that this
31                                     // results in read-
modify-write operations for integrity
32                                     // re-generation if
EnableDataIntgPt is set to 1.
33     parameter bit ErrOnWrite      = 0, // 1: Writes not al-
lowed, automatically error
34     parameter bit ErrOnRead       = 0, // 1: Reads not al-
lowed, automatically error
35     parameter bit CmdIntgCheck    = 0, // 1: Enable command
integrity check
36     parameter bit EnableRspIntgGen = 0, // 1: Generate re-
sponse integrity
37     parameter bit EnableDataIntgGen = 0, // 1: Generate re-
sponse data integrity
38     parameter bit EnableDataIntgPt = 0, // 1: Passthrough com-
mand/response data integrity
39     parameter bit SecFifoPtr      = 0, // 1: Duplicated fifo
pointers
40     parameter bit EnableReadback  = 0, // 1: Readback and
check written/read data.
41     parameter bit DataXorAddr     = 0, // 1: XOR data and ad-
dress for address protection
42     localparam int WidthMult      = SramDw / top_pkg::TL_DW,
43     localparam int IntgWidth      = tlul_pkg::DataIntgWidth *
WidthMult,
44     localparam int DataOutW       = EnableDataIntgPt ? SramDw
+ IntgWidth : SramDw
45 ) (
46     input  clk_i,
47     input  rst_ni,
48
49     // TL-UL interface
50     input  tl_h2d_t      tli,
51     output tl_d2h_t      tlo,
52
53     // control interface
54     input  mubi4_t en_ifetch_i,
55
56     // SRAM interface
57     output logic      req_o,
58     output mubi4_t    req_type_o,
59     input  logic      gnt_i,
60     output logic      we_o,
61     output logic [SramAw-1:0] addr_o,
62     output logic [DataOutW-1:0] wdata_o,
63     output logic [DataOutW-1:0] wmask_o,
64     output logic      intg_error_o,
65     output logic [RsvdWidth-1:0] user_rsvd_o,
66     input  logic [DataOutW-1:0] rdata_i,
67     input  logic      rvalid_i,
68     input  logic [1:0] error_i, // 2 bit error [1]:
Uncorrectable, [0]: Correctable
69     output logic      compound_txn_in_progress_o,
70     input  mubi4_t    readback_en_i,
71     output logic      readback_error_o,
72     input  logic      wr_collision_i,
73     input  logic      write_pending_i
74 );
75
76 localparam int SramByte = SramDw/8;
77 localparam int DataBitWidth =
prim_util_pkg::vbits(SramByte);
78 localparam int WoffsetWidth = (SramByte ==
top_pkg::TL_DBW) ? 1 :
79                                     DataBitWidth -
prim_util_pkg::vbits(top_pkg::TL_DBW);
80
81 logic error_det; // Internal protocol error checker
82 logic error_internal; // Internal protocol error checker
83 logic wr_attr_error;
84 logic instr_error;
85 logic wr_vld_error;
86 logic rd_vld_error;
87 logic rsp_fifo_error;
88 logic sramreqfifo_error;
89 logic reqfifo_error;
90 logic intg_error;
91 logic tlul_error;
92 logic readback_error;
93 logic sram_byte_readback_error;
94 logic addr_miss_error;
95

```

```

96 // readback check
97 logic readback_error_q;
98 if (EnableReadback) begin : gen_cmd_readback_check
99   assign readback_error = sram_byte_readback_error;
100   // permanently latch readback error until reset
101   always_ff @(posedge clk_i or negedge rst_ni) begin
102     if (!rst_ni) begin
103       readback_error_q <= '0;
104     end else if (readback_error) begin
105       readback_error_q <= '1'b1;
106     end
107   end
108 end else begin : gen_no_readback_check
109   logic unused_sram_byte_readback_error;
110   assign unused_sram_byte_readback_error =
sram_byte_readback_error;
111   assign readback_error = '0;
112   assign readback_error_q = '0;
113 end
114

115 // readback error output is permanent and should be used for
alert generation
116 // or other downstream effects
117 assign readback_error_o = readback_error | readback_error_q;
118
119 // integrity check
120 if (CmdIntgCheck) begin : gen_cmd_intg_check
121   tlul_cmd_intg_chk u_cmd_intg_chk (
122     .tl_i(tl_i),
123     .err_o(intg_error)
124   );
125 end else begin : gen_no_cmd_intg_check
126   assign intg_error = '0;
127 end
128
129 // permanently latch integrity error until reset
130 logic intg_error_q;
131 always_ff @(posedge clk_i or negedge rst_ni) begin
132   if (!rst_ni) begin
133     intg_error_q <= '0;
134   end else if (intg_error || rsp_fifo_error || sram-
reqfifo_error || reqfifo_error) begin
135     intg_error_q <= '1'b1;
136   end
137 end
138
139 // integrity error output is permanent and should be used
for alert generation
140 // or other downstream effects
141 assign intg_error_o = intg_error | rsp_fifo_error | sram-
reqfifo_error |
142   reqfifo_error | intg_error_q;
143
144 // wr_attr_error is true if this is a PUT with an unsuppor-
ted request size or mask. This is only
145 // possible if ByteAccess is not allowed.
146 assign wr_attr_error = (ByteAccess == 0) &&
147   (tl_i.a_opcode == PutFullData ||
tl_i.a_opcode == PutPartialData) &&
148   (tl_i.a_mask != '1 || tl_i.a_size !=
2'h2);
149
150 // An instruction type transaction is only valid if
en_ifetch is enabled
151 // If the instruction type is completely invalid, also con-
sidered an instruction error
152 assign instr_error =
prim_mubi_pkg::mubi4_test_invalid(tl_i.a.user.instr_type) |
153   (prim_mubi_pkg::mubi4_test_true_strict(tl_i.a.user.instr_type) &
prim_mubi_pkg::mubi4_test_false_loose(en_ifetch_i));
154
155 if (ErrOnWrite == 1) begin : gen_no_writes
156   assign wr_vld_error = tl_i.a_opcode != Get;
157 end else begin : gen_writes_allowed
158   assign wr_vld_error = '1'b0;
159 end
160
161 if (ErrOnRead == 1) begin: gen_no_reads
162   assign rd_vld_error = tl_i.a_opcode == Get;
163 end else begin : gen_reads_allowed
164   assign rd_vld_error = '1'b0;
165 end
166
167 if (2*SramAw == SramDepth) begin : gen_no_addr_chk
168   // The depth of the memory is a power of two. Here, we
only ever get to see addresses that hit
169   // in the memory.
170   assign addr_miss_error = '1'b0;
171 end else begin : gen_addr_chk
172   // The depth of the memory is not a power of two. We have
to signal an error if the address
173   // hits the unmapped range.
174   assign addr_miss_error = tl_i.a.address[DataBitWidth +:
SramAw] >= SramDepth;
175 end
176
177 // tlul protocol check
178 tlul_err u_err (

```

```

180   .clk_i,
181   .rst_ni,
182   .tl_i(tl_i),
183   .err_o (tlul_error)
184 );
185
186 // error return is transactional and thus does not used the
"latched" intg_err signal
187 assign error_det = wr_attr_error | wr_vld_error | rd_vld_er-
ror | instr_error |
188   tlul_error | intg_error |
addr_miss_error;
189
190 // from sram_byte to adapter logic
191 tl_h2d_t tl_i_int;
192 // from adapter logic to sram_byte
193 tl_d2h_t tl_o_int;
194 // from sram_byte to rsp_gen
195 tl_d2h_t tl_out;
196
197 // not all parts of tl_i_int are used
198 logic unused_tl_i_int;
199 assign unused_tl_i_int = ^tl_i_int;
200
201 tlul_rsp_intg_gen #(
202   .EnableRspIntgGen(EnableRspIntgGen),
203   .EnableDataIntgGen(EnableDataIntgGen),
204   .RspIntgInIsZero('1'b1)
205 ) u_rsp_gen (
206   .tl_i(tl_out),
207   .tl_o
208 );
209
210 // byte handling for integrity
211 tlul_sram_byte #(
212   .EnableIntg(ByteAccess & EnableDataIntgPt & !ErrOnWrite),
213   .Outstanding(Outstanding),
214   .EnableReadback(EnableReadback)
215 ) u_sram_byte (
216   .clk_i,
217   .rst_ni,
218   .tl_i,
219   .tl_o(tl_out),
220   .tl_sram_o(tl_i_int),
221   .tl_sram_i(tl_o_int),
222   .error_i(error_det),
223   .error_o(error_internal),
224   .alert_o(sram_byte_readback_error),
225   .compound_txn_in_progress_o,
226   .readback_en_i,
227   .wr_collision_i,
228   .write_pending_i
229 );
230
231 typedef struct packed {
232   logic [top_pkg::TL_DBW-1:0] mask ; // Byte mask within the
TL-UL word
233   logic [WoffsetWidth-1:0] woffset ; // Offset of the TL-
UL word within the SRAM word
234 } sram_req_t ;
235
236 typedef struct packed {
237   logic [SramBusBankAW-1:0] addr; // Address of the request
going to the memory.
238 } sram_req_addr_t ;
239
240 typedef struct packed {
241   logic is_read ;
242   logic error ;
243   prim_mubi_pkg::mubi4_t instr_type;
244   logic [top_pkg::TL_SZW-1:0] size ;
245   logic [top_pkg::TL_AIW-1:0] source ;
246 } req_t ;
247
248 typedef struct packed {
249   logic [top_pkg::TL_DW-1:0] data ;
250   logic [DataIntgWidth-1:0] data_intg ;
251   logic error ;
252 } rsp_t ;
253
254 localparam int SramReqWidth = $bits(sram_req_t);
255 localparam int SramReqFifoWidth = SramReqWidth + (DataXorAd-
dr ? SramBusBankAW : 0);
256 localparam int ReqFifoWidth = $bits(req_t) ;
257 localparam int RspFifoWidth = $bits(rsp_t) ;
258
259 // FIFO signal in case OutStand is greater than 1
260 // If request is latched, {write, source} is pushed to req
fifo.
261 // Req fifo is popped when D channel is acknowledged (v & r)
262 // D channel valid is asserted if it is write request or rsp
fifo not empty if read.
263 logic reqfifo_wvalid, reqfifo_wready;
264 logic reqfifo_rvalid, reqfifo_rready;
265 req_t reqfifo_wdata, reqfifo_rdata;
266
267 logic sramreqfifo_wvalid, sramreqfifo_wready;
268 logic sramreqfifo_rready;
269

```



```

270 // An item in u_sramreqfifo is the request itself, together
271 // (if DataXorAddr is nonzero) with some
272 // bits of the request address. These values are in
273 sram_req_*data and sram_addr_*data, which get
274 // combined to fifo items in sramreqfifo_*data.
275 sram_req_t sram_req_wdata, sram_req_rdata;
276 logic [SramBusBankAW-1:0] sram_addr_wdata, sram_addr_rdata;
277 logic [SramReqFifoWidth-1:0] sramreqfifo_wdata, sramreqfifo_rdata;
278 if (DataXorAddr) begin : gen_combine_with_addr
279   assign sramreqfifo_wdata = {sram_addr_wdata,
280 sram_req_wdata};
281   assign {sram_addr_rdata, sram_req_rdata} = sramreqfifo_rdata;
282 end else begin : gen_combine_without_addr
283   assign sramreqfifo_wdata = sram_req_wdata;
284   assign sram_addr_rdata = '0;
285   assign sram_req_rdata = sramreqfifo_rdata;
286 end
287 logic rspfifo_wvalid, rspfifo_wready;
288 logic rspfifo_rvalid, rspfifo_rready;
289 rsp_t rspfifo_wdata, rspfifo_rdata;
290 logic a_ack, d_ack, sram_ack;
291 assign a_ack = tl_i_int.a_valid & tl_o_int.a_ready;
292 assign d_ack = tl_o_int.d_valid & tl_i_int.d_ready;
293 assign sram_ack = req_o & gnt_i;
294 // Valid handling
295 logic d_valid, d_error;
296 always_comb begin
297   d_valid = 1'b0;
298   if (reqfifo_rvalid) begin
299     if (reqfifo_rdata.error) begin
300       // Return error response. Assume no request went out
301       to SRAM
302         d_valid = 1'b1;
303       end else if (reqfifo_rdata.is_read) begin
304         d_valid = rspfifo_rvalid;
305       end else begin
306         // Write without error
307         d_valid = 1'b1;
308       end
309     end else begin
310       d_valid = 1'b0;
311     end
312   end
313 end
314 always_comb begin
315   d_error = 1'b0;
316   if (reqfifo_rvalid) begin
317     if (reqfifo_rdata.is_read) begin
318       d_error = rspfifo_rdata.error | reqfifo_rdata.error;
319     end else begin
320       d_error = reqfifo_rdata.error;
321     end
322   end else begin
323     d_error = 1'b0;
324   end
325 end
326 logic vld_rd_rsp;
327 assign vld_rd_rsp = d_valid & rspfifo_rvalid &
328 reqfifo_rdata.is_read;
329 // If the response data is not valid, we set it to an illegal
330 // blanking value which is determined
331 // by whether the current transaction is an instruction
332 // fetch or a regular read operation.
333 logic [top_pkg::TL_DW-1:0] error_blanking_data;
334 assign error_blanking_data =
335 (prim_mubi_pkg::mubi4_test_true_strict(reqfifo_rdata.instr_type)) ?
336 DataWhenInstrError :
337 DataWhenError;
338 // Since DataWhenInstrError and DataWhenError can be arbitrary
339 // parameters
340 // we statically calculate the correct integrity values for
341 // these parameters here so that
342 // they do not have to be supplied externally.
343 logic [top_pkg::TL_DW-1:0] unused_instr, unused_data;
344 logic [DataIntgWidth-1:0] error_instr_integ, error_data_integ;
345 tlul_data_integ_enc u_tlul_data_integ_enc_instr (
346   .data_i(DataMaxWidth'(DataWhenInstrError)),
347   .data_intg_o({error_instr_integ, unused_instr})
348 );
349 tlul_data_integ_enc u_tlul_data_integ_enc_data (
350   .data_i(DataMaxWidth'(DataWhenError)),
351   .data_intg_o({error_data_integ, unused_data})
352 );
353 logic [DataIntgWidth-1:0] error_blanking_integ;
354 assign error_blanking_integ =
355 (prim_mubi_pkg::mubi4_test_true_strict(reqfifo_rdata.instr_type)) ?

```

```

356 error_instr_integ :
357 error_data_integ;
358 logic [top_pkg::TL_DW-1:0] d_data;
359 assign d_data = (vld_rd_rsp & ~d_error) ? rsp-
360 fifo_rdata.data : // valid read
361 : error_blanking_data; // write or TL-UL error
362 // If this a write response with data fields set to 0, we
363 // have to set all ECC bits correctly
364 // since we are using an inverted Hsiao code.
365 logic [DataIntgWidth-1:0] data_intg;
366 assign data_intg = (reqfifo_rdata.error) ? error_blanking_integ : // TL-UL error
367 (vld_rd_rsp) ? rsp-
368 fifo_rdata.data_intg : // valid read
369 prim_secdded_pkg::SecddedInv3932ZeroEcc; // valid write
370 // When an error is seen on an incoming transaction it gets
371 // an immediate response without
372 // performing an SRAM request. It may be the transaction receives
373 // a ready the first cycle it is
374 // seen, but if not we force a ready the following cycle.
375 // This avoids factoring the error
376 // calculation into the outgoing ready preventing a feed-
377 // through path from the incoming tilelink
378 // signals to the outgoing tilelink signals.
379 logic missed_err_gnt_d, missed_err_gnt_q;
380 // Track whether we've seen an incoming transaction with an
381 // error that didn't get a ready
382 assign missed_err_gnt_d = error_internal & tl_i_int.a_valid
383 & ~tl_o_int.a_ready;
384 always_ff @(posedge clk_i or negedge rst_ni) begin
385   if (!rst_ni) begin
386     missed_err_gnt_q <= 1'b0;
387   end else begin
388     missed_err_gnt_q <= missed_err_gnt_d;
389   end
390 end
391 assign tl_o_int = '{
392   d_valid : d_valid,
393   d_opcode : (d_valid && !reqfifo_rdata.is_read) ? Access-
394   sAck : AccessAckData,
395   d_param : '0,
396   d_size : (d_valid) ? reqfifo_rdata.size : '0,
397   d_source : (d_valid) ? reqfifo_rdata.source : '0,
398   d_sink : 1'b0,
399   d_data : d_data,
400   d_user : '{default: '0, data_intg: data_intg},
401   d_error : d_valid && d_error,
402   a_ready : (gnt_i | missed_err_gnt_q) & reqfifo_wready &
403   sramreqfifo_wready
404 };
405 // a_ready depends on the FIFO full condition and grant from
406 // SRAM (or SRAM arbiter)
407 // assemble response, including read response, write re-
408 // sponse, and error for unsupported stuff
409 // Output to SRAM:
410 // Generate request only when no internal error occurs.
411 // If error occurs, the request should be
412 // dropped and returned error response to the host. So,
413 // error to be pushed to reqfifo.
414 // In this case, it is assumed the request is granted
415 // (may cause ordering issue later?)
416 assign req_o = tl_i_int.a_valid & reqfifo_wready &
417 ~error_internal;
418 assign req_type_o = tl_i_int.a_user.instr_type;
419 assign we_o = tl_i_int.a_valid & (tl_i_int.a_opcode
420 inside {PutFullData, PutPartialData});
421 assign addr_o = (tl_i_int.a_valid) ? tl_i_int.a_ad-
422 dress[DataBitWidth+SramAw] : '0;
423 assign user_rsvd_o = (tl_i_int.a_valid) ?
424 tl_i_int.a_user.rsvd : '0;
425 // Support SRAMs wider than the TL-UL word width by mapping
426 // the parts of the
427 // TL-UL address which are more fine-granular than the SRAM
428 // width to the
429 // SRAM write mask.
430 logic [WoffsetWidth-1:0] woffset;
431 if (top_pkg::TL_DW != SramBw) begin : gen_wordwidthadapt
432   assign woffset = tl_i_int.a_ad-
433   dress[DataBitWidth-1:prim_util_pkg::vbits(top_pkg::TL_DBW)];
434 end else begin : gen_no_wordwidthadapt
435   assign woffset = '0;
436 end
437 // The size of the data/wmask depends on whether passthrough
438 // integrity is enabled.
439 // If passthrough integrity is enabled, the data is concat-
440 // enated with the integrity passed through

```

```

426 // the user bits. Otherwise, it is the data only.
427 localparam int DataWidth = EnableDataIntgPt ? top_pkg::TL_DW +
DataIntgWidth : top_pkg::TL_DW;
428
429 // Final combined wmask / wdata
430 logic [WidthMult-1:0][DataWidth-1:0] wmask_combined;
431 logic [WidthMult-1:0][DataWidth-1:0] wdata_combined;
432
433 // Original tlul portion
434 logic [WidthMult-1:0][top_pkg::TL_DW-1:0] wmask_int;
435 logic [WidthMult-1:0][top_pkg::TL_DW-1:0] wdata_int;
436
437 // Integrity portion
438 logic [WidthMult-1:0][DataIntgWidth-1:0] wmask_intg;
439 logic [WidthMult-1:0][DataIntgWidth-1:0] wdata_intg;
440
441 always_comb begin
442   wmask_int = '0;
443   wdata_int = '0;
444
445   if (tl_i_int.a_valid) begin
446     for (int i = 0 ; i < top_pkg::TL_DW/8 ; i++) begin
447       wmask_int[woffset][8*i +: 8] =
{8{tl_i_int.a_mask[i]}};
448       wdata_int[woffset][8*i +: 8] = (tl_i_int.a_mask[i] &&
we_o) ? tl_i_int.a_data[8*i+:8] : '0;
449     end
450   end
451 end
452
453 always_comb begin
454   wmask_intg = '0;
455   wdata_intg = '0;
456
457   if (tl_i_int.a_valid) begin
458     wmask_intg[woffset] = {DataIntgWidth{1'b1}};
459     wdata_intg[woffset] = tl_i_int.a_user.data_intg;
460   end
461 end
462
463 for (genvar i = 0; i < WidthMult; i++) begin :
gen_write_output
464   if (EnableDataIntgPt) begin : gen_combined_output
465     assign wmask_combined[i] = {wmask_int[i],
wmask_int[i]};
466     assign wdata_combined[i] = {wdata_int[i],
wdata_int[i]};
467   end else begin : gen_ft_output
468     logic unused_w;
469     assign wmask_combined[i] = wmask_int[i];
470     assign wdata_combined[i] = wdata_int[i];
471     assign unused_w = |wmask_intg & |wdata_intg;
472   end
473 end
474
475 assign wmask_o = wmask_combined;
476 assign wdata_o = wdata_combined;
477
478 assign reqfifo_wvalid = a_ack ; // Push to FIFO only when
granted
479 assign reqfifo_wdata = '{
480   is_read: tl_i_int.a_opcode == Get,
481   error: error_internal,
482   instr_type: tl_i_int.a_user.instr_type,
483   size: tl_i_int.a_size,
484   source: tl_i_int.a_source
485 }; // Store the request only. Doesn't have to store data
486 assign reqfifo_rready = d_ack ;
487
488 // push together with ReqFIFO, pop upon returning read
489 assign sram_req_wdata = '{
490   mask : tl_i_int.a_mask,
491   woffset : woffset
492 };
493 assign sramreqfifo_wvalid = sram_ack & ~we_o;
494 assign sramreqfifo_rready = rspfifo_wvalid;
495
496 assign rspfifo_wvalid = rvalid_i & reqfifo_rvalid;
497
498 assign sram_addr_wdata = tl_i_int.a_ad-
dress[DataBitWidth+:SramBusBankAW];
499
500 // Make sure only requested bytes are forwarded
501 logic [WidthMult-1:0][DataWidth-1:0] rdata_resaped;
502 logic [DataWidth-1:0] rdata_tlword;
503
504 // This just changes the array format so that the correct
word can be selected by indexing.
505 assign rdata_resaped = rdata_i;
506
507 if (EnableDataIntgPt) begin : gen_no_rmask
508   always_comb begin
509     // If the read mask is set to zero, all read data is
zeroed out by the mask.
510     // We have to set the ECC bits accordingly since we are
using an inverted Hsiao code.
511     rdata_tlword = prim_secdded_pkg::SecddedInv3932ZeroWord;
512     // Otherwise, if at least one mask bit is nonzero, we
are passing through the integrity.
513     // In that case we need to feed back the entire word

```

```

since otherwise the integrity
514 // will not calculate correctly.
515 if (!sram_req_rdata.mask) begin
516   // Select correct word.
517   if (DataXorAddr) begin : gen_data_xor_addr
518     // When DataXorAddr is enabled, on a read, the ad-
dress is XORed with the data fetched from
519     // the memory in the underlying memory controller
(e.g., flash controller). At this point,
520     // the address is again removed. If the address in
the read transaction has been modified,
521     // e.g., due to a fault, rdata now contains faulty
data, which is detected by the
522     // integrity mechanism.
523     rdata_tlword = {
524       rdata_resaped[sram_req_rdata.woffset][DataW-
idth-1:top_pkg::TL_DW],
525       rdata_resaped[sram_req_rdata.woffset]
[top_pkg::TL_DW-1:0] ^
526       {{{(top_pkg::TL_DW-SramBusBankAW){1'b0}}},
sram_addr_rdata}
527   };
528   end else begin : gen_no_data_xor_addr
529     rdata_tlword =
rdata_resaped[sram_req_rdata.woffset];
530   end
531 end
532
533 end else begin : gen_rmask
534   logic [DataWidth-1:0] rmask;
535   always_comb begin
536     rmask = '0;
537     for (int i = 0 ; i < top_pkg::TL_DW/8 ; i++) begin
538       rmask[8*i +: 8] = {8{sram_req_rdata.mask[i]}};
539     end
540   end
541   // Select correct word and mask it.
542   assign rdata_tlword =
rdata_resaped[sram_req_rdata.woffset] & rmask;
543 end
544
545 assign rspfifo_wdata = '{
546   data : rdata_tlword[top_pkg::TL_DW-1:0],
547   data_intg : EnableDataIntgPt ? rdata_tlword[DataWidth-1 -:
DataIntgWidth] : '0,
548   error : error_i[1] // Only care for Uncorrectable er-
ror
549 };
550 assign rspfifo_rready = reqfifo_rdata.is_read &
~reqfifo_rdata.error & reqfifo_rready;
551
552 // This module only cares about uncorrectable errors.
553 logic unused_error;
554 assign unused_error = error_i[0];
555
556 // FIFO instance: REQ, RSP
557
558 // ReqFIFO is to store the Access type to match to the Re-
sponse data.
559 // For instance, SRAM accepts the write request but
doesn't return the
560 // acknowledge. In this case, it may be hard to determine
when the D
561 // response for the write data should send out if reads/
writes are
562 // interleaved. So, to make it in-order (even TL-UL al-
lows out-of-order
563 // responses), storing the request is necessary. And if
the read entry
564 // is write op, it is safe to return the response right
away. If it is
565 // read request, then D response is waiting until read
data arrives.
566 prim_fifo_sync #(
567   .Width (ReqFifoWidth),
568   .Pass (1'b0),
569   .Depth (Outstanding),
570   .NeverClears (1'b1),
571   .Secure (SecFifoPtr)
572 ) u_reqfifo (
573   .clk_i,
574   .rst_ni,
575   .clr_i (1'b0),
576   .wvalid_i(reqfifo_wvalid),
577   .wready_o(reqfifo_wready),
578   .wdata_i (reqfifo_wdata),
579   .rvalid_o(reqfifo_rvalid),
580   .rready_i(reqfifo_rready),
581   .rdata_o (reqfifo_rdata),
582   .full_o (),
583   .depth_o (),
584   .err_o (reqfifo_error)
585 );
586
587 // sramreqfifo:
588 // While the ReqFIFO holds the request until it is sent
back via TL-UL, the
589 // sramreqfifo only needs to hold the mask and word off-
set until the read

```



```

590 // data returns from memory.
591 prim_fifo_sync #(
592   .Width      (SramReqFifoWidth),
593   .Pass       (1'b0),
594   .Depth      (Outstanding),
595   .NeverClears (1'b1),
596   .Secure     (SecFifoPtr),
597   .OutputZeroIfEmpty (1)
598 ) u_sramreqfifo (
599   .clk_i,
600   .rst_ni,
601   .clr_i (1'b0),
602   .wvalid_i(sramreqfifo_wvalid),
603   .wready_o(sramreqfifo_wready),
604   .wdata_i (sramreqfifo_wdata),
605   .rvalid_o(),
606   .rready_i(sramreqfifo_rready),
607   .rdata_o (sramreqfifo_rdata),
608   .full_o (),
609   .depth_o (),
610   .err_o (sramreqfifo_error)
611 );
612
613 if (!DataXorAddr) begin : gen_no_data_xor_addr_fifo
614   // If u_sramreqfifo doesn't contain any address data,
615   // sram_addr_rdata. Tie them off with an unused signal.
616   logic unused_sram_addresses;
617   assign unused_sram_addresses = ^{sram_addr_wdata, sram_ad-
dr_rdata};
618 end
619
620 // Rationale having #Outstanding depth in response FIFO.
621 // In normal case, if the host or the crossbar accepts
622 // the response data,
623 // response FIFO isn't needed. But if in any case it has
624 // a chance to be
625 // back pressured, the response FIFO should store the re-
turned data not to
626 // lose the data from the SRAM interface. Remember, SRAM
interface doesn't
627 // have back-pressure signal such as read_ready.
628 prim_fifo_sync #(
629   .Width      (RspFifoWidth),
630   .Pass       (1'b1),
631   .Depth      (Outstanding),
632   .NeverClears (1'b1),
633   .Secure     (SecFifoPtr)
634 ) u_rspfifo (
635   .clk_i,
636   .rst_ni,
637   .clr_i (1'b0),
638   .wvalid_i(rspfifo_wvalid),
639   .wready_o(rspfifo_wready),
640   .wdata_i (rspfifo_wdata),
641   .rvalid_o(rspfifo_rvalid),
642   .rready_i(rspfifo_rready),
643   .rdata_o (rspfifo_rdata),
644   .full_o (),
645   .depth_o (),
646   .err_o (rsp_fifo_error)
647 );
648 // below assertion fails when SRAM rvalid is asserted even
649 // though ReqFifo is empty
650 `ASSERT(rvalidHighReqFifoEmpty, rvalid_i |-> reqfifo_rvalid)
651
652 // below assertion fails when outstanding value is too small
653 // (SRAM rvalid is asserted
654 // even though the RspFifo is full)
655 `ASSERT(rvalidHighWhenRspFifoFull, rvalid_i |-> rsp-
fifo_wready)
656
657 // If both ErrOnWrite and ErrOnRead are set, this block is
658 // useless
659 `ASSERT_INIT(adapterNoReadOrWrite, (ErrOnWrite & ErrOnRead)
== 0)
660
661 `ASSERT_INIT(SramDwHasByteGranularity_A, SramDw % 8 == 0)
662 `ASSERT_INIT(SramDwIsMultipleOfTlulWidth_A, SramDw %
top_pkg::TL_DW == 0)
663 // Either the memory has a power-of-two depth and the ad-
dress width perfectly matches, or the
664 // depth is not a power of two but the address width is
665 // still minimal.
666 `ASSERT_INIT(SramAwCorrectlySizedForDepth_A,
667 ((SramDepth & (SramDepth - 1)) == 0) ? (2**SramAw ==
SramDepth) :
668 (SramDepth == SramAw))
669
670 // These parameter options cannot both be true at the same
671 // time
672 `ASSERT_INIT(DataIntgOptions_A, ~(EnableDataIntgGen & En-
ableDataIntgPt))
673
674 // Make sure that outputs are defined (a special case for
675 // tl_o is explained separately below)
676 `ASSERT_KNOWN(ReqOutKnown_A, req_o )
677 `ASSERT_KNOWN(WeOutKnown_A, we_o )

```

```

671 `ASSERT_KNOWN(AddrOutKnown_A, addr_o )
672 `ASSERT_KNOWN(WdataOutKnown_A, wdata_o)
673 `ASSERT_KNOWN(WmaskOutKnown_A, wmask_o)
674
675 // We'd like to claim that the payload of the TL output is
676 // known, but this isn't necessarily true!
677 // This block is just an adapter that converts from an SRAM
678 // interface to a TL interface. To make
679
680 // the assertion true, we need to weaken it to say that that
681 // tl_o is only X if the SRAM supplied
682 // that X.
683 //
684 // This is a bit tricky to track because SRAM responses get
685 // stored in u_rspfifo. Assuming that the
686 // FIFO doesn't manufacture X's (an assertion in
687 // prim_fifo_sync), the only stage of the path
688 // needed in this file is the following:
689 `ASSERT_KNOWN(TlOutValidKnown_A, tl_o.d_valid)
690 `ASSERT(TlOutKnownIfFifoKnown_A, !$isunknown(rspfifo_rdata)
-> !$isunknown(tl_o))
691
692 // The definition of d_valid leads to the assertion below.
693 `ASSERT(DvalidNeedsReqFifoRvalid_A, d_valid ->
reqfifo_rvalid)
694 endmodule

```

9. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_sram_racl.sv (146 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5
6 include "prim_assert.sv"
7
8 /**
9  * Tile-Link UL adapter for SRAM-like devices with RACL sup-
10  * port
11  *
12  * - Intentionally omitted BaseAddr in case of multiple memory
13  *   maps are used in a SoC,
14  *   it means that aliasing can happen if target device size
15  *   in TL-UL crossbar is bigger
16  *   than SRAM size
17  * - At most one of EnableDataIntgGen / EnableDataIntgPt can
18  *   be enabled. However it
19  *   is possible for both to be disabled.
20  * - A module can neither generate an integrity response nor
21  *   pass through any pre-existing
22  *   integrity. This might be the case for non-security crit-
23  *   ical memories where there is
24  *   no stored integrity AND another entity upstream is
25  *   already generating returning integrity.
26  * - There is however no case where EnableDataIntgGen and En-
27  *   ableDataIntgPt are both true.
28  */
29 module tlul_adapter_sram_racl
30 import tlul_pkg::*;
31 import prim_mubi_pkg::mubi4_t;
32 #(
33   parameter int SramAw      = 12,
34   parameter int SramDw      = 32, // Must be
35   multiple of the TL width
36   parameter int SramDepth   = 2**SramAw, // Must be <=
37   2**SramAw
38   parameter int Outstanding = 1, // Only one
39   request is accepted
40   parameter int SramBusBankAW = 12, // SRAM bus
41   address width of the SRAM bank. Only
42   // used when
43   DataXorAddr=1.
44   parameter bit ByteAccess  = 1, // 1: Enables
45   sub-word write transactions. Note that
46   // this
47   results in read-modify-write operations
48   // for in-
49   integrity re-generation if
50   // En-
51   ableDataIntgPt is set to 1.
52   parameter bit ErrOnWrite  = 0, // 1: Writes
53   not allowed, automatically error
54   parameter bit ErrOnRead   = 0, // 1: Reads
55   not allowed, automatically error
56   parameter bit CmdIntgCheck = 0, // 1: Enable
57   command integrity check
58
59   parameter bit EnableRspIntgGen = 0, // 1: Generate
60   response integrity
61
62   parameter bit EnableDataIntgGen = 0, // 1: Generate
63   response data integrity
64   parameter bit EnableDataIntgPt = 0, // 1:
65   Passthrough command/response data integrity
66   parameter bit SecFifoPtr      = 0, // 1: Duplic-
67   ated fifo pointers

```

```

41 parameter bit EnableReadback = 0, // 1: Readback
and check written/read data.

42 parameter bit DataXorAddr = 0, // 1: XOR data
and address for address protection
43 parameter bit EnableRacl = 0, // 1: Enable
RACL checks on access
44 parameter bit RclErrorRsp = EnableRacl, // 1: Return
TLUL error on RACL errors
45 parameter int RclPolicySelNumRanges = 1, // Number of
ranges with a RACL policy
46 localparam int WidthMult = SramDw / top_pkg::TL_DW,
47 localparam int IntgWidth *
WidthMult,
48 localparam int DataOutW = EnableDataIntgPt ? SramDw
+ IntgWidth : SramDw
49 ) (
50 input clk_i,
51 input rst_ni,
52
53 // TL-UL interface
54 input tl_h2d_t tl_i,
55 output tl_d2h_t tl_o,
56
57 // control interface
58 input mubi4_t en_ifetch_i,
59
60 // SRAM interface
61 output logic req_o,
62 output mubi4_t req_type_o,
63 input gnt_i,
64 output logic we_o,
65 output logic [SramAw-1:0] addr_o,
66 output logic [DataOutW-1:0] wdata_o,
67 output logic [DataOutW-1:0] wmask_o,
68 output logic intg_error_o,
69 output logic [RsvdWidth-1:0] user_rsvd_o,
70 input [DataOutW-1:0] rdata_i,
71 input rvalid_i,
72 input [1:0] error_i, // 2 bit error [1]:
Uncorrectable, [0]: Correctable
73 output logic compound_txn_in_progress_o,
74 input mubi4_t readback_en_i,
75 output logic readback_error_o,
76 input logic wr_collision_i,
77 input logic write_pending_i,
78 // RACL interface
79 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
80 output top_racl_pkg::racl_error_log_t racl_error_o,
81 input top_racl_pkg::racl_range_t [RaclPolicySelNum-
Ranges-1:0] racl_policy_sel_ranges_i
82 );
83 tl_h2d_t tl_h2d_filtered;
84 tl_d2h_t tl_d2h_filtered;
85
86 tlul_adapter_racl #(
87 .EnableRacl ( EnableRacl ),
88 .RaclErrorRsp ( RaclErrorRsp ),
89 .RaclPolicySelNumRanges ( RaclPolicySelNumRanges )
90 ) tlul_adapter_racl(
91 .clk_i ( clk_i ),
92 .rst_ni ( rst_ni ),
93 .tl_h2d_i ( tl_i ),
94 .tl_d2h_o ( tl_o ),
95 .tl_filtered_h2d_o ( tl_h2d_filtered ),
96 .tl_filtered_d2h_i ( tl_d2h_filtered ),
97 .racl_policies_i ( racl_policies_i ),
98 .racl_error_o ( racl_error_o ),
99 .racl_policy_sel_ranges_i ( racl_policy_sel_ranges_i )
100 );
101
102 tlul_adapter_sram #(
103 .SramAw ( SramAw ),
104 .SramDw ( SramDw ),
105 .SramDepth ( SramDepth ),
106 .Outstanding ( Outstanding ),
107 .SramBusBankAW ( SramBusBankAW ),
108 .ByteAccess ( ByteAccess ),
109 .ErrOnWrite ( ErrOnWrite ),
110 .ErrOnRead ( ErrOnRead ),
111 .CmdIntgCheck ( CmdIntgCheck ),
112 .EnableRspIntgGen ( EnableRspIntgGen ),
113 .EnableDataIntgGen ( EnableDataIntgGen ),
114 .EnableDataIntgPt ( EnableDataIntgPt ),
115 .SecFifoPtr ( SecFifoPtr ),
116 .EnableReadback ( EnableReadback ),
117 .DataXorAddr ( DataXorAddr )
118 ) tlul_adapter_sram (
119 .clk_i,
120 .rst_ni,
121 .tl_i(tl_h2d_filtered),
122 .tl_o(tl_d2h_filtered),
123 .en_ifetch_i,
124 .req_o,
125 .req_type_o,
126 .gnt_i,
127 .we_o,
128 .addr_o,
129 .wdata_o,
130 .wmask_o,
131 .intg_error_o,

```

```

132 .user_rsvd_o,
133 .rdata_i,
134 .rvalid_i,
135 .error_i,
136 .compound_txn_in_progress_o,
137 .readback_en_i,
138 .readback_error_o,
139 .wr_collision_i,
140 .write_pending_i
141 );
142
143 // Ensure that RACL signals are not undefined
144 `ASSERT_KNOWN(RaclAdapterSramErrorKnown_A, racl_er-
ror_o.valid)
145
146 endmodule

```

10. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_adapter_vh.sv (149 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 module tlul_adapter_vh
8 import tlul_pkg::*;
9 import prim_mubi_pkg::mubi4_t;
10 #(
11 parameter int ADDR_WIDTH = top_pkg::TL_AW,
12 parameter int DATA_WIDTH = top_pkg::TL_DW,
13 parameter int MASK_WIDTH = DATA_WIDTH >> 3,
14 parameter int USER_WIDTH = 32,
15 parameter int ID_WIDTH = top_pkg::TL_AIW,
16
17 parameter bit [ADDR_WIDTH-1:0] VH_REGISTER_ADDRESS_OFFSET =
32'h0000_0100,
18
19 parameter bit EnableDataIntgGen = 1,
20 parameter bit EnableRspDataIntgCheck = 1
21 ) (
22 input clk_i,
23 input rst_ni,
24
25 output tl_h2d_t tl_o,
26 input tl_d2h_t tl_i,
27
28 // Valid-Hold device interface (VH to TLUL).
29 input logic dv_i,
30 output logic hld_o,
31 input logic [ADDR_WIDTH-1:0] addr_i,
32 input logic write_i,
33 input logic [DATA_WIDTH-1:0] wdata_i,
34 input logic [MASK_WIDTH-1:0] wstrb_i,
35 input logic [2:0] size_i,
36 output logic [DATA_WIDTH-1:0] rdata_o,
37 output logic error_o,
38 input logic last_i,
39 input logic [USER_WIDTH-1:0] user_i,
40 input logic [ID_WIDTH-1:0] id_i,
41
42 // Valid-Hold host interface (VH to internal registers). The
signals from the VH device interface
43 // are routed to the VH host interface for every internal
access, see the `internal_access` signal.
44 output logic int_dv_o,
45 input logic int_hld_i,
46 output logic [ADDR_WIDTH-1:0] int_addr_o,
47 output logic int_write_o,
48 output logic [DATA_WIDTH-1:0] int_wdata_o,
49 output logic [MASK_WIDTH-1:0] int_wstrb_o,
50 output logic [2:0] int_size_o,
51 input logic [DATA_WIDTH-1:0] int_rdata_i,
52 input logic int_error_i,
53 output logic int_last_o,
54 output logic [USER_WIDTH-1:0] int_user_o,
55 output logic [ID_WIDTH-1:0] int_id_o
56 );
57
58 // The adapter distinguishes between two types of VH ac-
cesses: those going to an internal register
59 // file and those targeting OpenTitan registers, with the
latter ones being translated into TLUL
60 // requests while the other requests are relayed to the in-
ternal register file.
61 logic internal_access;
62 assign internal_access = addr_i >= VH_RE-
GISTER_ADDRESS_OFFSET;
63
64 // Differentiate between two levels of acknowledgements:
65 // - `req_ack`: A host-to-device request is acknowledged by
the device, meaning that the response
66 // is pending.
67 // - `resp_ack`: The device-to-host response is acknow-
ledged.

```

```

68 logic pending_d, pending_q;
69 logic req_ack, resp_ack;
70
71 assign req_ack = dv_i & tl_i.a_ready & tl_o.a_valid & ~internal_access;
72 assign resp_ack = dv_i & tl_o.d_ready & tl_i.d_valid & ~internal_access;
73
74 always_comb begin
75     pending_d = pending_q;
76     if (req_ack) begin
77         pending_d = 1'b1;
78     end else if (resp_ack) begin
79         pending_d = 1'b0;
80     end
81 end
82
83 always_ff @(posedge clk_i or negedge rst_ni) begin
84     if (!rst_ni) begin
85         pending_q <= 1'b0;
86     end else begin
87         pending_q <= pending_d;
88     end
89 end
90
91 // Only make a new request (through `a_valid`) to the device
when none are pending. If the
92 // integrity checks are enabled, the `a_user` fields will be
set by the corresponding module (see
93 // `tlul_cmd_intg_gen`).
94 tlul_pkg::tl_h2d_t tl_o_pre;
95 assign tl_o_pre = '{
96     a_valid: dv_i & ~pending_q & ~internal_access,
97     a_opcode: ~write_i ? Get : (&wstrb_i ? PutFullData : Put-
PartialData),
98     a_size: top_pkg::TL_SZW'(size_i),
99     a_mask: wstrb_i,
100     a_source: id_i,
101     a_address: addr_i,
102     a_data: wdata_i,
103     a_user: '{default: '0, instr_type:
prim_mubi_pkg::MuB14False},
104     d_ready: 1'b1,
105     // unused
106     a_param: 3'h0
107 };
108
109 // Accesses to device registers are acknowledged with the
TLUL d-channel handshake.
110 assign hld_o = !pending_q && internal_access && !int_hld_i ?
1'b0 :
111     pending_q && tl_o.d_ready && tl_i.d_valid ?
1'b0 : 1'b1;
112
113 assign rdata_o = internal_access ? int_rdata_i :
tl_i.d_data;
114
115 // Route signals to the VH host interface.
116 assign int_dv_o = dv_i & internal_access;
117 assign int_addr_o = addr_i;
118 assign int_write_o = write_i;
119 assign int_wdata_o = wdata_i;
120 assign int_wstrb_o = wstrb_i;
121 assign int_size_o = size_i;
122 assign int_last_o = last_i;
123 assign int_user_o = user_i;
124 assign int_id_o = id_i;
125
126 tlul_cmd_intg_gen #(
127     .EnableDataIntgGen (EnableDataIntgGen)
128 ) u_cmd_intg_gen (
129     .tl_i ( tl_o_pre ),
130     .tl_o ( tl_o )
131 );
132
133 logic intg_err_chk;
134 tlul_rsp_intg_chk #(
135     .EnableRspDataIntgCheck(EnableRspDataIntgCheck)
136 ) u_rsp_chk (
137     .tl_i ( tl_i ),
138     .err_o ( intg_err_chk )
139 );
140
141 assign error_o = tl_i.d_error | int_error_i | intg_err_chk;
142
143 // Make sure the VH parameters are compatible with TLUL
datapath widths.
144 `ASSERT_INIT(TlInvalidAddrWidth_A, ADDR_WIDTH ==
top_pkg::TL_AW)
145 `ASSERT_INIT(TlInvalidDataWidth_A, DATA_WIDTH ==
top_pkg::TL_DW)
146 `ASSERT_INIT(TlInvalidMaskWidth_A, MASK_WIDTH ==
top_pkg::TL_DBW)
147 `ASSERT_INIT(TlInvalidUserWidth_A, ID_WIDTH ==
top_pkg::TL_AIW)
148
149 endmodule

```

11. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert.sv (502 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // Protocol checker for TL-UL ports using assertions. Supports in-
terface widths
6 // up to 64bit.
7
8 `include "prim_assert.sv"
9
10 module tlul_assert #(
11     parameter EndpointType = "Device" // can be either "Host" or
"Device"
12 ) (
13     input clk_i,
14     input rst_ni,
15
16     // tile link ports
17     input tlul_pkg::tl_h2d_t h2d,
18     input tlul_pkg::tl_d2h_t d2h
19 );
20
21 `ifndef VERILATOR
22 `ifndef SYNTHESIS
23
24 `ifdef UVM
25     import uvm_pkg::*;
26     `include "uvm_macros.svh"
27 `endif
28     import tlul_pkg::*;
29     import top_pkg::*;
30
31 // To test TL-UL error cases in UVM tests, pass the
tlul_assert_en argument in the UVM config db,
32 // which will cause disable_sva to be set to 1. This dis-
ables some assertions about the
33 // well-formedness of the TL input.
34 bit disable_sva;
35
36 default disable iff disable_sva || !rst_ni;
37 default clocking @(posedge clk_i); endclocking
38
39 ///////////////////////////////////////////////////
40 // check requests and responses //
41 ///////////////////////////////////////////////////
42
43 // There are up to 2**TL-AIW possible source-IDs. Below ar-
ray "pend_req" has one entry
44 // for each source-ID. Each entry has the following fields:
45 // - pend : is set to 1 to indicate up to 1 pending re-
quest for the source ID
46 // - opcode : "Get" requires "AccessAckData" response,
"Put*" require "AccessAck"
47 // - size : d_size of response must match a_size of re-
quest
48 // - mask : is used to allow X for bytes whose mask bit
is 0
49 typedef struct packed {
50     bit pend; // set to 1 to indicate
a pending request
51     tl_a_op_e opcode;
52     logic [top_pkg::TL_SZW-1:0] size;
53     logic [top_pkg::TL_DBW-1:0] mask;
54 } pend_req_t;
55
56 pend_req_t [2**TL-AIW-1:0] pend_req;
57
58 ///////////////////////////////////////////////////
59 // Current request //
60 ///////////////////////////////////////////////////
61 pend_req_t curr_req;
62 always_comb begin
63     // Indicates whether a request is being accepted.
64     curr_req.pend = h2d.a_valid & d2h.a_ready;
65     // Properties of current request.
66     curr_req.opcode = h2d.a_opcode;
67     curr_req.size = h2d.a_size;
68     curr_req.mask = h2d.a_mask;
69 end
70
71 // Forward the current request for checking?
72 // This supports combinational responses. Only defined when
`d_valid` asserted, i.e. it may
73 // be used unqualified in the `consequent` of an assertion,
where the `antecedent` tests
74 // assertion of `d_valid`.
75 logic curr_fwd;
76 assign curr_fwd = curr_req.pend & (d2h.d_source ==
h2d.a_source);
77
78 logic [7:0] a_mask, d_mask;
79 logic [63:0] a_data, d_data;
80 assign a_mask = 8'(h2d.a_mask);

```

```

81 assign a_data = 64'(h2d.a_data);
82 assign d_mask = 8'(curr_fwd ? curr_req.mask :
pend_req[d2h.d_source].mask);
83 assign d_data = 64'(d2h.d_data);
84
85 ///////////////////////////////////////////////////
86 // keep track of pending requests //
87 ///////////////////////////////////////////////////
88
89 always_ff @(posedge clk_i or negedge rst_ni) begin
90   if (!rst_ni) begin
91     pend_req <= '0;
92   end else begin
93     // Update pend_req if there is an A channel transaction
on this cycle.
94     //
95     // Note that we can safely do this if there was a com-
binatorial response. In that case, it
96     // will be removed again just below.
97     if (curr_req.pend) begin
98       pend_req[h2d.a_source] <= curr_req;
99     end
100
101     // If there is a D channel transaction on this cycle,
clear any pending request for the
102     // associated d_source.
103     if (d2h.d_valid && h2d.d_ready) begin
104       pend_req[d2h.d_source] <= '0;
105     end
106   end
107 end
108
109 ///////////////////////////////////////////////////
110 // define sequences for request checks //
111 ///////////////////////////////////////////////////
112
113 sequence h2d_pre_S;
114   h2d.a_valid;
115 endsequence
116
117 // a_opcode: only 3 opcodes are legal for requests
118 sequence legalAopcode_S;
119   (h2d.a_opcode == PutFullData) ||
120   (h2d.a_opcode == Get) ||
121   (h2d.a_opcode == PutPartialData);
122 endsequence
123
124 // a_param is reserved
125 sequence legalAParam_S;
126   h2d.a_param == '0;
127 endsequence
128
129 // a_size: Size shouldn't be greater than the bus width in
TL-UL (not in TL-UH)
130 // This assertion can be covered by below
131 // (a_size must less than or equal to ones of
a_mask)
132
133 // a_size: 2**a_size must greater than or equal to $count-
ones(a_mask) for PutPartial and Get
134 sequence sizeGTEMask_S;
135   (h2d.a_opcode == PutFullData) || ((1 << h2d.a_size) >=
$countones(h2d.a_mask));
136 endsequence
137
138 // a_size: 2**a_size must equal to $countones(a_mask) for
PutFull
139 sequence sizeMatchesMask_S;
140   (h2d.a_opcode inside {PutPartialData, Get}) ||
141   ((1 << h2d.a_size) == $countones(h2d.a_mask));
142 endsequence
143
144 // a_source: there should be no more than one pending re-
quest per each source ID
145 sequence pendingReqPerSrc_S;
146   pend_req[h2d.a_source].pend == 0;
147 endsequence
148
149 // a_address must be aligned to a_size: a_address & ((1 <<
a_size) - 1) == 0
150 sequence addrSizeAligned_S;
151   (h2d.a_address & ((1 << h2d.a_size)-1)) == '0;
152 endsequence
153
154 // a_mask must be contiguous for Get and PutFullData re-
quests
155 // the spec talks about "naturally aligned". Does this
mean that bit [0] of
156 // mask is always 1? If that's true, then below code
could be much simpler.
157 // However, the spec shows a timing diagram where bit 0
of mask is 0.
158 sequence contigMask_pre_S;
159   h2d.a_opcode != PutPartialData;
160 endsequence
161
162 sequence contigMask_S;
163   $countones(h2d.a_mask ^
{h2d.a_mask[$bits(h2d.a_mask)-2:0], 1'b0}) <= 2;
164 endsequence
165

```

```

166 // a_data must be known for opcode == Put*(depending on mask
bits)
167 sequence aDataKnown_pre_S;
168   (h2d.a_opcode != Get);
169 endsequence
170
171 sequence aDataKnown_S;
172   // no check if this lane mask is inactive
173   ((!a_mask[0]) || (a_mask[0] && !$isunknown(a_data[8*0 +
8])))) &&
174   ((!a_mask[1]) || (a_mask[1] && !$isunknown(a_data[8*1 +
8])))) &&
175   ((!a_mask[2]) || (a_mask[2] && !$isunknown(a_data[8*2 +
8])))) &&
176   ((!a_mask[3]) || (a_mask[3] && !$isunknown(a_data[8*3 +
8])))) &&
177   ((!a_mask[4]) || (a_mask[4] && !$isunknown(a_data[8*4 +
8])))) &&
178   ((!a_mask[5]) || (a_mask[5] && !$isunknown(a_data[8*5 +
8])))) &&
179   ((!a_mask[6]) || (a_mask[6] && !$isunknown(a_data[8*6 +
8])))) &&
180   ((!a_mask[7]) || (a_mask[7] && !$isunknown(a_data[8*7 +
8]))));
181 endsequence
182
183 ///////////////////////////////////////////////////
184 // define sequences for request checks //
185 ///////////////////////////////////////////////////
186
187 sequence d2h_pre_S;
188   d2h.d_valid;
189 endsequence
190
191 // d_opcode: if request was Get, then response must be Ac-
cessAckData
192 sequence respOpcode_S;
193   d2h.d_opcode == ((curr_fwd ? curr_req.opcode :
pend_req[d2h.d_source].opcode) == Get) ?
194   AccessAckData : AccessAck;
195 endsequence
196
197 // d_param is reserved
198 sequence legalDParam_S;
199   d2h.d_param == '0;
200 endsequence
201
202 // d_size must equal the a_size of the corresponding request
203 sequence respSzEqReqSz_S;
204   d2h.d_size == (curr_fwd ? curr_req.size :
pend_req[d2h.d_source].size);
205 endsequence
206
207 // d_source: each non-combinational response should have a
pending using same source ID
208 sequence respMustHaveReq_S;
209   curr_fwd | pend_req[d2h.d_source].pend;
210 endsequence
211
212 // d_data must be known for AccessAckData (depending on mask
bits)
213 sequence dDataKnown_pre_S;
214   d2h.d_opcode == AccessAckData;
215 endsequence
216
217 sequence dDataKnown_S;
218   // no check if this lane mask is inactive
219   ((!d_mask[0]) || (d_mask[0] && !$isunknown(d_data[8*0 +
8])))) &&
220   ((!d_mask[1]) || (d_mask[1] && !$isunknown(d_data[8*1 +
8])))) &&
221   ((!d_mask[2]) || (d_mask[2] && !$isunknown(d_data[8*2 +
8])))) &&
222   ((!d_mask[3]) || (d_mask[3] && !$isunknown(d_data[8*3 +
8])))) &&
223   ((!d_mask[4]) || (d_mask[4] && !$isunknown(d_data[8*4 +
8])))) &&
224   ((!d_mask[5]) || (d_mask[5] && !$isunknown(d_data[8*5 +
8])))) &&
225   ((!d_mask[6]) || (d_mask[6] && !$isunknown(d_data[8*6 +
8])))) &&
226   ((!d_mask[7]) || (d_mask[7] && !$isunknown(d_data[8*7 +
8]))));
227 endsequence
228
229 ///////////////////////////////////////////////////
230 // define sequences for d_error checks //
231 ///////////////////////////////////////////////////
232
233 sequence d_error_pre_S;
234   h2d.a_valid && d2h.a_ready;
235 endsequence
236
237 sequence legalAopcodeErr_S;
238   !(h2d.a_opcode inside {PutFullData, Get, PutPartialData});
239 endsequence
240
241 sequence sizeGTEMaskErr_S;
242   (1 << h2d.a_size) < $countones(h2d.a_mask);
243 endsequence

```

```

244
245 sequence sizeMatchesMaskErr_S;
246 (h2d.a_opcode == PutFullData) && ((1 << h2d.a_size) !=
$countones(h2d.a_mask));
247 endsequence
248
249 sequence addrSizeAlignedErr_S;
250 (h2d.a_address & ((1 << h2d.a_size)-1)) != '0';
251 endsequence
252
253 ///////////////////////////////////////////////////////////////////
254 // assemble properties and check //
255 ///////////////////////////////////////////////////////////////////
256
257 // note: use negedge clk to avoid possible race conditions
258 // in this case all signals coming from the device side have
an assumed property
259 if (EndpointType == "Host") begin : gen_host
260 // h2d
261 if (1) begin : gen_h2d
262 legalAOpcode_A:
263 assert property (h2d_pre_S |> legalAOpcode_S)
264 else `ASSERT_ERROR(legalAOpcode_A)
265 legalAParam_A:
266 assert property (h2d_pre_S |> legalAParam_S)
267 else `ASSERT_ERROR(legalAParam_A)
268 sizeGTEMask_A:
269 assert property (h2d_pre_S |> sizeGTEMask_S)
270 else `ASSERT_ERROR(sizeGTEMask_A)
271 sizeMatchesMask_A:
272 assert property (h2d_pre_S |> sizeMatchesMask_S)
273 else `ASSERT_ERROR(sizeMatchesMask_A)
274 pendingReqPerSrc_A:
275 assert property (h2d_pre_S |> pendingReqPerSrc_S)
276 else `ASSERT_ERROR(pendingReqPerSrc_A)
277 addrSizeAligned_A:
278 assert property (h2d_pre_S |> addrSizeAligned_S)
279 else `ASSERT_ERROR(addrSizeAligned_A)
280 contigMask_A:
281 assert property (h2d_pre_S and contigMask_pre_S |>
contigMask_S)
282 else `ASSERT_ERROR(contigMask_A)
283 aDataKnown_A:
284 assert property (h2d_pre_S and aDataKnown_pre_S |>
aDataKnown_S)
285 else `ASSERT_ERROR(aDataKnown_A)
286 end
287 // d2h
288 if (1) begin : gen_d2h
289 respOpcode_M:
290 assume property (d2h_pre_S |> respOpcode_S)
291 else `ASSERT_ERROR(respOpcode_M)
292 legalDParam_M:
293 assume property (d2h_pre_S |> legalDParam_S)
294 else `ASSERT_ERROR(legalDParam_M)
295 respSzEqReqSz_M:
296 assume property (d2h_pre_S |> respSzEqReqSz_S)
297 else `ASSERT_ERROR(respSzEqReqSz_M)
298 respMustHaveReq_M:
299 assume property (d2h_pre_S |> respMustHaveReq_S)
300 else `ASSERT_ERROR(respMustHaveReq_M)
301 dDataKnown_M:
302 assume property (d2h_pre_S and dDataKnown_pre_S |>
dDataKnown_S)
303 else `ASSERT_ERROR(dDataKnown_M)
304 end
305 // in this case all signals coming from the host side have
an assumed property
306 end else if (EndpointType == "Device") begin : gen_device
307 // h2d
308 if (1) begin : gen_h2d
309 legalAParam_M:
310 assume property (h2d_pre_S |> legalAParam_S)
311 else `ASSERT_ERROR(legalAParam_M)
312 pendingReqPerSrc_M:
313 assume property (h2d_pre_S |> pendingReqPerSrc_S)
314 else `ASSERT_ERROR(pendingReqPerSrc_M)
315 aDataKnown_M:
316 assume property (h2d_pre_S and aDataKnown_pre_S |>
aDataKnown_S)
317 else `ASSERT_ERROR(aDataKnown_M)
318 contigMask_M:
319 assume property (h2d_pre_S and contigMask_pre_S |>
contigMask_S)
320 else `ASSERT_ERROR(contigMask_M)
321 end
322 // d2h
323 if (1) begin : gen_d2h
324 respOpcode_A:
325 assert property (d2h_pre_S |> respOpcode_S)
326 else `ASSERT_ERROR(respOpcode_A)
327 legalDParam_A:
328 assert property (d2h_pre_S |> legalDParam_S)
329 else `ASSERT_ERROR(legalDParam_A)
330 respSzEqReqSz_A:
331 assert property (d2h_pre_S |> respSzEqReqSz_S)
332 else `ASSERT_ERROR(respSzEqReqSz_A)
333 respMustHaveReq_A:
334 assert property (d2h_pre_S |> respMustHaveReq_S)
335 else `ASSERT_ERROR(respMustHaveReq_A)

```

```

336 dDataKnown_A:
337 assert property (d2h_pre_S and dDataKnown_pre_S |>
dDataKnown_S)
338 else `ASSERT_ERROR(dDataKnown_A)
339
340 // d2h error cases
341 legalAOpcodeErr_A:
342 assert property (d_error_pre_S and legalAOpcodeErr_S |
=>
343 s_eventually (d2h.d_valid &&
d2h.d_error))
344 else `ASSERT_ERROR(legalAOpcodeErr_A)
345 sizeGTEMaskErr_A:
346 assert property (d_error_pre_S and sizeGTEMaskErr_S |
=>
347 s_eventually (d2h.d_valid &&
d2h.d_error))
348 else `ASSERT_ERROR(sizeGTEMaskErr_A)
349 sizeMatchesMaskErr_A:
350 assert property (d_error_pre_S and sizeMatchesMask-
Err_S |>=>
351 s_eventually (d2h.d_valid &&
d2h.d_error))
352 else `ASSERT_ERROR(sizeMatchesMaskErr_A)
353 addrSizeAlignedErr_A:
354 assert property (d_error_pre_S and addrSize-
AlignedErr_S |>=>
355 s_eventually (d2h.d_valid &&
d2h.d_error))
356 else `ASSERT_ERROR(addrSizeAlignedErr_A)
357 end
358 end else begin : gen_unknown
359 initial begin : p_unknown
360 `ASSERT_I(unknownConfig_A, 0 == 1)
361 end
362 end
363
364 initial begin : p_dbw
365 // widths up to 64bit / 8 Byte are supported
366 `ASSERT_I(TLdbw_A, TL_DBW <= 8)
367 end
368
369 // make sure all "pending" bits are 0 at the end of the sim
370 for (genvar ii = 0; ii < 2*`TL_AIW; ii++) begin :
gen_assert_final
371 `ASSERT_FINAL(noOutstandingReqsAtEndOfSim_A,
(pend_req[ii].pend == 0))
372 end
373
374 ///////////////////////////////////////////////////////////////////
375 // additional checks for X values //
376 ///////////////////////////////////////////////////////////////////
377
378 // a_* should be known when a_valid == 1 (a_opcode and
a_param are already covered above)
379 // This also covers ASSERT_KNOWN of a_valid
380 `ASSERT_KNOWN_IF(aKnown_A, {h2d.a_size, h2d.a_source,
h2d.a_address, h2d.a_mask, h2d.a_user},
381 h2d.a_valid)
382
383 // d_* should be known when d_valid == 1 (d_opcode, d_param,
d_size already covered above)
384 // This also covers ASSERT_KNOWN of d_valid
385 `ASSERT_KNOWN_IF(dKnown_A, {d2h.d_source, d2h.d_sink,
d2h.d_error, d2h.d_user}, d2h.d_valid)
386
387 // make sure ready is not X after reset
388 `ASSERT_KNOWN(aReadyKnown_A, d2h.a_ready)
389 `ASSERT_KNOWN(dReadyKnown_A, h2d.d_ready)
390
391 ///////////////////////////////////////////////////////////////////
392 // SVA coverage //
393 ///////////////////////////////////////////////////////////////////
394 `define TLUL_COVER(SEQ) `COVER(`SEQ`_C, `SEQ`_S, !
clk_i, !rst_ni || disable_sva)
395
396 // host sends back2back requests
397 sequence b2bReq_S;
398 h2d.a_valid && d2h.a_ready ##1 h2d.a_valid;
399 endsequence
400
401 // device sends back2back responses
402 sequence b2bRsp_S;
403 d2h.d_valid && h2d.d_ready ##1 d2h.d_valid;
404 endsequence
405
406 // host sends back2back requests with same address
407 // UVM RAL can't issue this scenario, add this cover to make
sure it's tested in some other seq
408 sequence b2bReqWithSameAddr_S;
409 bit [top_pkg:TL_AW-1:0] pre_addr;
410 (h2d.a_valid && d2h.a_ready, pre_addr = h2d.a_address)
411 ##1 h2d.a_valid && pre_addr == h2d.a_address;
412 endsequence
413
414 // a_valid is dropped without a_ready
415 sequence aValidNotAccepted_S;
416 h2d.a_valid && !d2h.a_ready ##1 !h2d.a_valid;
417 endsequence

```



```

418
419 // d_valid is dropped without a_ready
420 sequence dValidNotAccepted_S;
421 d2h.d_valid && !h2d.d_ready ##1 !d2h.d_valid;
422 endsequence
423
424 // host uses same source for back2back items
425 sequence b2bSameSource_S;
426 bit [top_pkg::TL_AIW-1:0] pre_source;
427 (h2d.a_valid && d2h.a_ready, pre_source = h2d.a_source)
##1 h2d.a_valid[->1]
428 ##0 pre_source == h2d.a_source;
429 endsequence
430
431 // This macro defines a sequence where a value is provided
for the given signal on the A channel
432 // which isn't accepted and then a different value is
provided on the next cycle (without a_valid
433 // dropping).
434 //
435 // It also defines a coverpoint for the sequence.
436 `define TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(NAME)
\
437 sequence a_`NAME`ChangedNotAccepted_S;
\
438 h2d.a_valid && !d2h.a_ready ##1
\
439 h2d.a_valid && $changed(h2d.a_`NAME`);
\
440 endsequence
441 `TLUL_COVER(a_`NAME`ChangedNotAccepted)
442
443 // This macro is like the previous one
(TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED), but applies to
444 // the D channel.
445 `define TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(NAME)
\
446 sequence d_`NAME`ChangedNotAccepted_S;
\
447 d2h.d_valid && !h2d.d_ready ##1
\
448 d2h.d_valid && $changed(d2h.d_`NAME`);
\
449 endsequence
450 `TLUL_COVER(d_`NAME`ChangedNotAccepted)
451
452 if (EndpointType == "Host") begin : gen_host_cov // DUT is
host
453 `TLUL_COVER(b2bRsp)
454 `TLUL_COVER(dValidNotAccepted)
455 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(data)
456 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(opcode)
457 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(size)
458 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(source)
459 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(sink)
460 `TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED(error)
461 end else if (EndpointType == "Device") begin :
gen_device_cov // DUT is device
462 `TLUL_COVER(b2bReq)
463 `TLUL_COVER(b2bReqWithSameAddr)
464 `TLUL_COVER(aValidNotAccepted)
465 `TLUL_COVER(b2bSameSource)
466 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(address)
467 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(data)
468 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(opcode)
469 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(size)
470 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(source)
471 `TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED(mask)
472 end else begin : gen_unknown_cov
473 initial begin : p_unknown_cov
474 `ASSERT_I(unknownConfig_A, 0 == 1)
475 end
476 end
477
478 `ifdef UVM
479 initial forever begin
480 automatic string hier_path = $sformatf("%m");
481 bit tlul_assert_en;
482 uvm_config_db#(bit)::wait_modified(null, hier_path,
"tlul_assert_en");
483 if (!uvm_config_db#(bit)::get(null, hier_path,
"tlul_assert_en", tlul_assert_en)) begin
484 uvm_fatal(hier_path, "Can't find tlul_assert_en")
485 end
486 disable_sva = !tlul_assert_en;
487 end
488 `else
489 // Set default values for the disable_*_sva signals (not
disabling the assertions)
490 always_ff @(posedge clk_i or negedge rst_ni) begin
491 if (!rst_ni) begin
492 disable_sva <= 0;
493 end
494 end
495 `endif
496
497 `undef TLUL_COVER
498 `undef TLUL_A_CHAN_CONTENT_CHANGED_WO_ACCEPTED
499 `undef TLUL_D_CHAN_CONTENT_CHANGED_WO_ACCEPTED

```

```

500 `endif
501 `endif
502 endmodule : tlul_assert

```

12. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_assert_multiple.sv (31 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // Protocol checker for multiple TL-UL ports
6
7 module tlul_assert_multiple #(
8 parameter int unsigned N = 2,
9 parameter EndpointType = "Device" // can be "Device" or
"Host"
10 ) (
11 input clk_i,
12 input rst_ni,
13
14 // tile link ports
15 input tlul_pkg::tl_h2d_t h2d [N],
16 input tlul_pkg::tl_d2h_t d2h [N]
17 );
18
19 // instantiate N tlul_assert modules
20 for (genvar ii = 0; ii < N; ii++) begin : gen_assert
21 tlul_assert #(
22 EndpointType(EndpointType)
23 ) tlul_assert (
24 .clk_i,
25 .rst_ni,
26 // TL-UL ports
27 .h2d (h2d[ii]),
28 .d2h (d2h[ii])
29 );
30 end
31 endmodule

```

13. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_chk.sv (53 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8 * Tile-Link UL command integrity check
9 */
10
11 module tlul_cmd_intg_chk import tlul_pkg::*; (
12 // TL-UL interface
13 input tl_h2d_t tli,
14
15 // error output
16 output logic err_o
17 );
18
19 logic [1:0] err;
20 logic data_err;
21 tl_h2d_cmd_intg_t cmd;
22 assign cmd = extract_h2d_cmd_intg(tli);
23
24 prim_secded_inv_64_57_dec u_chk (
25 .data_i({tli.a_user.cmd_intg, H2DCmdMaxWidth'(cmd)}),
26 .data_o(),
27 .syndrome_o(),
28 .err_o(err)
29 );
30
31 tlul_data_integ_dec u_tlul_data_integ_dec (
32 .data_intg_i({tli.a_user.data_intg, DataMax-
Width'(tli.a_data)}),
33 .data_err_o(data_err)
34 );
35
36 // error output is transactional, it is up to the instanti-
ating module
37 // to determine if a permanent latch is feasible
38 // [LOWRISC] err and data_err is unknown when a_valid is
low, so we can't cover
39 // the condition coverage - (|err| (|data_err)) == 0/1,
when a_valid = 0, which is
40 // fine as driving unknown is better. `err_o` is used as a
condition in other places,
41 // which needs to be covered with 0 and 1, so it's OK to
disable the entire coverage.
42 //VCS coverage off
43 // pragma coverage off
44 assign err_o = tli.a_valid & (|err| (|data_err));
45 //VCS coverage on
46 // pragma coverage on

```

```

47
48 logic unused_tl;
49 assign unused_tl = |tl_i;
50
51 `ASSERT_INIT(PayLoadWidthCheck, $bits(tl_h2d_cmd_intg_t) <=
H2DCmdMaxWidth)
52
53 endmodule // tlul_payload_chk

```

14. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_cmd_intg_gen.sv (58 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL command integrity generator
9  */
10
11 module tlul_cmd_intg_gen import tlul_pkg::*; #(
12   parameter bit EnableDataIntgGen = 1'b1
13 ) (
14   // TL-UL interface
15   input  tl_h2d_t tl_i,
16   output tl_h2d_t tl_o
17 );
18
19   tl_h2d_cmd_intg_t cmd;
20   assign cmd = extract_h2d_cmd_intg(tl_i);
21   logic [H2DCmdMaxWidth-1:0] unused_cmd_payload;
22
23   logic [H2DCmdIntgWidth-1:0] cmd_intg;
24   prim_secded_inv_64_57_enc u_cmd_gen (
25     .data_i(H2DCmdMaxWidth'(cmd)),
26     .data_o({cmd_intg, unused_cmd_payload})
27   );
28
29   logic [top_pkg::TL_DW-1:0] data_final;
30   logic [DataIntgWidth-1:0] data_intg;
31
32   if (EnableDataIntgGen) begin : gen_data_intg
33     assign data_final = tl_i.a_data;
34
35     logic [DataMaxWidth-1:0] unused_data;
36     prim_secded_inv_39_32_enc u_data_gen (
37       .data_i(DataMaxWidth'(data_final)),
38       .data_o({data_intg, unused_data})
39     );
40   end else begin : gen_passthrough_data_intg
41     assign data_final = tl_i.a_data;
42     assign data_intg = tl_i.a_user.data_intg;
43   end
44
45   always_comb begin
46     tl_o = tl_i;
47     tl_o.a_data = data_final;
48     tl_o.a_user.cmd_intg = cmd_intg;
49     tl_o.a_user.data_intg = data_intg;
50   end
51
52   logic unused_tl;
53   assign unused_tl = ^tl_i;
54
55
56 `ASSERT_INIT(PayMaxWidthCheck_A, $bits(tl_h2d_cmd_intg_t) <=
H2DCmdMaxWidth)
57
58 endmodule : tlul_cmd_intg_gen

```

15. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_dec.sv (27 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Data integrity decoder for bus integrity scheme
9  */
10
11 module tlul_data_integ_dec import tlul_pkg::*; (
12   // TL-UL interface
13   input  [DataMaxWidth+DataIntgWidth-1:0] data_intg_i,
14   output logic                               data_err_o
15 );
16
17   logic [1:0] data_err;
18   prim_secded_inv_39_32_dec u_data_chk (
19     .data_i(data_intg_i),

```

```

20     .data_o(),
21     .syndrome_o(),
22     .err_o(data_err)
23   );
24
25   assign data_err_o = |data_err;
26
27 endmodule : tlul_data_integ_dec

```

16. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_data_integ_enc.sv (22 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Data integrity encoder for bus integrity scheme
9  */
10
11 module tlul_data_integ_enc import tlul_pkg::*; (
12   // TL-UL interface
13   input  [DataMaxWidth-1:0] data_i,
14   output logic [DataMaxWidth+DataIntgWidth-1:0] data_intg_o
15 );
16
17   prim_secded_inv_39_32_enc u_data_gen (
18     .data_i,
19     .data_o(data_intg_o)
20   );
21
22 endmodule : tlul_data_integ_enc

```

17. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err.sv (99 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 module tlul_err import tlul_pkg::*; (
8   input clk_i,
9   input rst_ni,
10
11   input tl_h2d_t tl_i,
12
13   output logic err_o
14 );
15
16   localparam int IW = $bits(tl_i.a_source);
17   localparam int SZW = $bits(tl_i.a_size);
18   localparam int DW = $bits(tl_i.a_data);
19   localparam int MW = $bits(tl_i.a_mask);
20   localparam int SubAW = $clog2(DW/8);
21
22   logic opcode_allowed, a_config_allowed;
23
24   logic op_full, op_partial, op_get;
25   assign op_full = (tl_i.a_opcode == PutFullData);
26   assign op_partial = (tl_i.a_opcode == PutPartialData);
27   assign op_get = (tl_i.a_opcode == Get);
28
29   // An instruction type transaction cannot be write
30   logic instr_wr_err;
31   assign instr_wr_err =
prim_mubi_pkg::mubi4_test_true_strict(tl_i.a_user.instr_type) &
32     (op_full | op_partial);
33
34   logic instr_type_err;
35   assign instr_type_err =
prim_mubi_pkg::mubi4_test_invalid(tl_i.a_user.instr_type);
36
37   // Anything that doesn't fall into the permitted category,
it raises an error
38   assign err_o = ~(opcode_allowed & a_config_allowed) | in-
str_wr_err | instr_type_err;
39
40   // opcode check
41   assign opcode_allowed = (tl_i.a_opcode == PutFullData)
42     | (tl_i.a_opcode == PutPartialData)
43     | (tl_i.a_opcode == Get);
44
45   // a channel configuration check
46   logic addr_sz_chk; // address and size alignment check
47   logic mask_chk; // inactive lane a_mask check
48   logic fulldata_chk; // PutFullData should have size match
to mask
49
50   localparam bit [MW-1:0] MaskOne = 1;
51   logic [MW-1:0] mask;
52
53

```



```

54 assign mask = MaskOne << tl_i.a_address[SubAW-1:0];
55
56 always_comb begin
57   addr_sz_chk = 1'b0;
58   mask_chk = 1'b0;
59   fulldata_chk = 1'b0; // Only valid when opcode is Put-
FullData
60
61   if (tl_i.a_valid) begin
62     unique case (tl_i.a_size)
63       'h0: begin // 1 Byte
64         addr_sz_chk = 1'b1;
65         mask_chk = ~(tl_i.a_mask & ~mask);
66         fulldata_chk = |(tl_i.a_mask & mask);
67       end
68
69       'h1: begin // 2 Byte
70         addr_sz_chk = ~tl_i.a_address[0];
71         // check inactive lanes if lower 2B, check
a_mask[3:2], if uppper 2B, a_mask[1:0]
72         mask_chk = (tl_i.a_address[1]) ? ~(tl_i.a_mask
& 4'b0011) : ~(tl_i.a_mask & 4'b1100);
73         fulldata_chk = (tl_i.a_address[1]) ?
&tl_i.a_mask[3:2] : &tl_i.a_mask[1:0] ;
74       end
75     end
76
77     'h2: begin // 4 Byte
78       addr_sz_chk = ~tl_i.a_address[SubAW-1:0];
79       mask_chk = 1'b1;
80       fulldata_chk = &tl_i.a_mask[3:0];
81     end
82
83     default: begin // else
84       addr_sz_chk = 1'b0;
85       mask_chk = 1'b0;
86       fulldata_chk = 1'b0;
87     end
88   endcase
89 end
90 end
91
92 assign a_config_allowed = addr_sz_chk
93   & mask_chk
94   & (op_get | op_partial |
fulldata_chk) ;
95
96 // Only 32 bit data width for current tlul_err
97 `ASSERT_INIT(dataWidthOnly32_A, DW == 32)
98
99 endmodule

```

18. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_err_resp.sv (75 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // TL-UL error responder module, used by tlul_socket_in to
help response
6 // to requests to no correct address space. Responses are al-
ways one cycle
7 // after request with no stalling unless response is stuck on
the way out.
8
9 module tlul_err_resp #(
10   // By default, we return a proper bus error. In some cases,
we need to return a blank all-zero
11   // response without setting the error bit, and for those
cases ReturnBlankResp can be set to 1.
12   parameter bit ReturnBlankResp = 0
13 ) (
14   input clk_i,
15   input rst_ni,
16   input tlul_pkg::tl_h2d_t tl_h_i,
17   output tlul_pkg::tl_d2h_t tl_h_o
18 );
19 import tlul_pkg::*;
20 import prim_mubi_pkg::*;
21
22 tl_a_op_e err_opcode;
23 logic [$bits(tl_h_i.a_source)-1:0] err_source;
24 logic [$bits(tl_h_i.a_size)-1:0] err_size;
25 logic err_rsp_pending;
26 mubi4_t err_instr_type;
27 tlul_pkg::tl_d2h_t tl_h_o_int;
28
29 tlul_rsp_intg_gen #(
30   .EnableRspIntgGen(1),
31   .EnableDataIntgGen(1)
32 ) u_intg_gen (
33   .tl_i(tl_h_o_int),
34   .tl_o(tl_h_o)
35 );
36
37 always_ff @(posedge clk_i or negedge rst_ni) begin
38   if (!rst_ni) begin
39     err_rsp_pending <= 1'b0;

```

```

40   err_source <= {top_pkg::TL_AIW{1'b0}};
41   err_opcode <= Get;
42   err_size <= '0;
43   err_instr_type <= MuBi4False;
44   end else if (err_rsp_pending && tl_h_i.d_ready) begin
45     err_rsp_pending <= 1'b0;
46   end else if (tl_h_i.a_valid && tl_h_o_int.a_ready) begin
47     err_rsp_pending <= 1'b1;
48     err_source <= tl_h_i.a_source;
49     err_opcode <= tl_h_i.a_opcode;
50     err_size <= tl_h_i.a_size;
51     err_instr_type <= tl_h_i.a_user.instr_type;
52   end
53 end
54
55 assign tl_h_o_int.a_ready = ~err_rsp_pending;
56 assign tl_h_o_int.d_valid = err_rsp_pending;
57 if (ReturnBlankResp) begin : gen_zero_resp
58   assign tl_h_o_int.d_data = '0;
59 end else begin : gen_err_resp
60   assign tl_h_o_int.d_data = (mu-
bi4_test_true_strict(err_instr_type)) ? DataWhenInstrError :
61
62   DataWhenError;
63 end
64 assign tl_h_o_int.d_source = err_source;
65 assign tl_h_o_int.d_sink = '0;
66 assign tl_h_o_int.d_param = '0;
67 assign tl_h_o_int.d_size = err_size;
68 assign tl_h_o_int.d_opcode = (err_opcode == Get) ? Acces-
AckData : AccessAck;
69 assign tl_h_o_int.d_user = '0;
70 assign tl_h_o_int.d_error = ~ReturnBlankResp;
71 // Waive unused bits of tl_h_i
72 logic unused_tl_h;
73 assign unused_tl_h = ^{tl_h_i, err_instr_type};
74
75 endmodule

```

19. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_fifo_async.sv (98 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Asynchronous TL-UL FIFO, used to cross asynchronous clock
domains.
6 // This instantiates two FIFOs, one for the request side,
7 // and one for the response side.
8
9 `include "prim_assert.sv"
10
11 module tlul_fifo_async #(
12   parameter int unsigned ReqDepth = 4,
13   parameter int unsigned RspDepth = 4
14 ) (
15   input clk_h_i,
16   input rst_h_ni,
17   input clk_d_i,
18   input rst_d_ni,
19   input tlul_pkg::tl_h2d_t tl_h_i,
20   output tlul_pkg::tl_d2h_t tl_h_o,
21   output tlul_pkg::tl_h2d_t tl_d_o,
22   input tlul_pkg::tl_d2h_t tl_d_i
23 );
24
25 // Put everything on the request side into one FIFO
26 localparam int unsigned REQ_FIFO_WIDTH =
$bits(tlul_pkg::tl_h2d_t)-2;
27
28 prim_fifo_async #(
29   .Width(REQ_FIFO_WIDTH),
30   .Depth(ReqDepth),
31   .OutputZeroIfInvalid(1)
32 ) req_fifo (
33   .clk_wr_i (clk_h_i),
34   .rst_wr_ni (rst_h_ni),
35   .clk_rd_i (clk_d_i),
36   .rst_rd_ni (rst_d_ni),
37   .wvalid_i (tl_h_i.a_valid),
38   .wready_o (tl_h_o.a_ready),
39   .wdata_i ({tl_h_i.a_opcode ,
40             tl_h_i.a_param ,
41             tl_h_i.a_size ,
42             tl_h_i.a_source ,
43             tl_h_i.a_address ,
44             tl_h_i.a_mask ,
45             tl_h_i.a_data ,
46             tl_h_i.a_user })),
47   .rvalid_o (tl_d_o.a_valid),
48   .rready_i (tl_d_i.a_ready),
49   .rdata_o ({tl_d_o.a_opcode ,
50             tl_d_o.a_param ,
51             tl_d_o.a_size ,
52             tl_d_o.a_source ,
53             tl_d_o.a_address ,

```

```

54         tl_d_o.a_mask      ,
55         tl_d_o.a_data      ,
56         tl_d_o.a_user      }},
57     .wdepth_o      (),
58     .rdepth_o      ()
59 );
60
61 // Put everything on the response side into the other FIFO
62
63 localparam int unsigned RSPFIFO_WIDTH =
$bits(tlul_pkg::tl_d2h_t) -2;
64
65 prim_fifo_async #(
66     .Width(RSPFIFO_WIDTH),
67     .Depth(RspDepth),
68     .OutputZeroIfInvalid(1)
69 ) rspfifo (
70     .clk_wr_i      (clk_d_i),
71     .rst_wr_ni     (rst_d_ni),
72     .clk_rd_i      (clk_h_i),
73     .rst_rd_ni     (rst_h_ni),
74     .wvalid_i      (tl_d_i.d_valid),
75     .wready_o      (tl_d_o.d_ready),
76     .wdata_i       ({tl_d_i.d_opcode,
77                     tl_d_i.d_param ,
78                     tl_d_i.d_size ,
79                     tl_d_i.d_source,
80                     tl_d_i.d_sink ,
81                     tl_d_i.d_data ,
82                     tl_d_i.d_user ,
83                     tl_d_i.d_error }},
84     .rvalid_o      (tl_h_o.d_valid),
85     .rready_i      (tl_h_i.d_ready),
86     .rdata_o       ({tl_h_o.d_opcode,
87                     tl_h_o.d_param ,
88                     tl_h_o.d_size ,
89                     tl_h_o.d_source,
90                     tl_h_o.d_sink ,
91                     tl_h_o.d_data ,
92                     tl_h_o.d_user ,
93                     tl_h_o.d_error }},
94     .wdepth_o      (),
95     .rdepth_o      ()
96 );
97
98 endmodule

```

20. lowRISC opentitan/hw/ip/tlul/rtl/tlul_fifo_sync.sv (191 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Synchronous TL-UL FIFO, used to add elasticity (the ability for
6 // transactions to stall on one side without affecting the
  other side)
7 // to a TL-UL bus. This instantiates two pairs (data + integ-
  rity) of FIFOs:
8 // one pair for the request side, and another pair for the re-
  sponse side.
9
10 module tlul_fifo_sync #(
11     parameter bit      ReqPass = 1'b1,
12     parameter bit      RspPass = 1'b1,
13     parameter int unsigned ReqDepth = 2,
14     parameter int unsigned RspDepth = 2,
15     parameter int unsigned SpareReqW = 1,
16     parameter int unsigned SpareRspW = 1
17 ) (
18     input               clk_i,
19     input               rst_ni,
20     input tlul_pkg::tl_h2d_t tl_h_i,
21     output tlul_pkg::tl_d2h_t tl_h_o,
22     output tlul_pkg::tl_h2d_t tl_d_o,
23     input tlul_pkg::tl_d2h_t tl_d_i,
24     input [SpareReqW-1:0] spare_req_i,
25     output [SpareReqW-1:0] spare_req_o,
26     input [SpareRspW-1:0] spare_rsp_i,
27     output [SpareRspW-1:0] spare_rsp_o
28 );
29
30 // Put everything on the request side into two FIFOs.
31 // The first FIFO holds all data except the integrity bits
  of the `a_user` field. The second FIFO
32 // only holds the command and data integrity bits of the
  `a_user` field. This is a FI
33
34 // countermeasure, as a fault into one of the two FIFOs will
  trigger an integrity error once the
35 // integrity bits are checked.
36 localparam int unsigned REQFIFO_INTG_WIDTH = tlul_pkg::H2D-
  CmdIntgWidth + tlul_pkg::DataIntgWidth;
37 localparam int unsigned RSPFIFO_WIDTH =
  $bits(tlul_pkg::tl_h2d_t) -2 + SpareReqW -
38 REQFIFO_INTG_WIDTH;
39

```

```

39 prim_fifo_sync
#(.Width(REQFIFO_WIDTH), .Pass(ReqPass), .Depth(ReqDepth)) reqfifo (
40     .clk_i,
41     .rst_ni,
42     .clr_i      (1'b0 ),
43     .wvalid_i   (tl_h_i.a_valid),
44     .wready_o   (tl_h_o.a_ready),
45     .wdata_i    ({tl_h_i.a_opcode,
46                 tl_h_i.a_param,
47                 tl_h_i.a_size,
48                 tl_h_i.a_source,
49                 tl_h_i.a_address,
50                 tl_h_i.a_mask,
51                 tl_h_i.a_data,
52                 tl_h_i.a_user.rsvd,
53                 tl_h_i.a_user.instr_type,
54                 spare_req_i}),
55     .rvalid_o   (tl_d_o.a_valid),
56     .rready_i   (tl_d_i.a_ready),
57     .rdata_o    ({tl_d_o.a_opcode,
58                 tl_d_o.a_param,
59                 tl_d_o.a_size,
60                 tl_d_o.a_source,
61                 tl_d_o.a_address,
62                 tl_d_o.a_mask,
63                 tl_d_o.a_data,
64                 tl_d_o.a_user.rsvd,
65                 tl_d_o.a_user.instr_type,
66                 spare_req_o}),
67     .full_o     (),
68     .depth_o    (),
69     .err_o      ());
70
71 // Buffer the inputs of the FIFO holding the integrity to
  avoid synthesis optimizations.
72 localparam int NumBufferBitsReqFifoIntg = $bits({
73     tl_h_i.a_valid,
74     tl_d_i.a_ready
75 });
76
77 logic [NumBufferBitsReqFifoIntg-1:0] buf_reqfifo_intg_in,
  buf_reqfifo_intg_out;
78 logic reqfifo_intg_a_valid_buf;
79 logic reqfifo_intg_a_ready_buf;
80
81 assign buf_reqfifo_intg_in = {
82     tl_h_i.a_valid,
83     tl_d_i.a_ready
84 };
85
86 assign {
87     reqfifo_intg_a_valid_buf,
88     reqfifo_intg_a_ready_buf
89 } = buf_reqfifo_intg_out;
90
91 prim_buf #(
92     .Width(NumBufferBitsReqFifoIntg)
93 ) u_reqfifo_intg_prim_buf (
94     .in_i(buf_reqfifo_intg_in),
95     .out_o(buf_reqfifo_intg_out)
96 );
97
98 prim_fifo_sync
#(.Width(REQFIFO_INTG_WIDTH), .Pass(ReqPass), .Depth(ReqDepth))
  reqfifo_intg (
99     .clk_i,
100    .rst_ni,
101    .clr_i      (1'b0 ),
102    .wvalid_i   (reqfifo_intg_a_valid_buf),
103    .wready_o   (),
104    .wdata_i    ({tl_h_i.a_user.cmd_intg,
105                 tl_h_i.a_user.data_intg}),
106    .rvalid_o   (),
107    .rready_i   (reqfifo_intg_a_ready_buf),
108    .rdata_o    ({tl_d_o.a_user.cmd_intg,
109                 tl_d_o.a_user.data_intg}),
110    .full_o     (),
111    .depth_o    (),
112    .err_o      ());
113
114 // Put everything on the response side into the other two
  FIFOs.
115 // The integrity bits and all other bits are separated into
  two different FIFOs for the same
116 // reason as above.
117 localparam int unsigned RSPFIFO_INTG_WIDTH =
  $bits(tlul_pkg::tl_d_user_t);
118 localparam int unsigned RSPFIFO_WIDTH =
  $bits(tlul_pkg::tl_d2h_t) -2 + SpareRspW -
119 RSP-
  FIFO_INTG_WIDTH;
120
121 prim_fifo_sync
#(.Width(RSPFIFO_WIDTH), .Pass(RspPass), .Depth(RspDepth)) rspfifo (
122     .clk_i,
123     .rst_ni,
124     .clr_i      (1'b0 ),
125     .wvalid_i   (tl_d_i.d_valid),
126     .wready_o   (tl_d_o.d_ready),
127     .wdata_i    ({tl_d_i.d_opcode,
128                 tl_d_i.d_param ,

```

```

129         tl_d_i.d_size ,
130         tl_d_i.d_source,
131         tl_d_i.d_sink ,
132         (tl_d_i.d_opcode == tlul_pkg::AccessAck-
Data) ? tl_d_i.d_data :
133 {top_pkg::TL_DW{1'b0}} ,
134         tl_d_i.d_error ,
135         spare_rsp_i}),
136         .rvalid_o      (tl_h_o.d_valid),
137         .rready_i      (tl_h_i.d_ready),
138         .rdata_o       ({tl_h_o.d_opcode,
139                          tl_h_o.d_param ,
140                          tl_h_o.d_size ,
141                          tl_h_o.d_source,
142                          tl_h_o.d_sink ,
143                          tl_h_o.d_data ,
144                          tl_h_o.d_error ,
145                          spare_rsp_o}),
146         .full_o        (),
147         .depth_o       (),
148         .err_o         ());
149
150 // Buffer the inputs of the FIFO holding the integrity to
avoid synthesis optimizations.
151 localparam int NumBufferBitsRspFifoIntg = $bits({
152     tl_d_i.d_valid,
153     tl_h_i.d_ready
154 });
155
156 logic [NumBufferBitsRspFifoIntg-1:0] buf_rspfifo_intg_in,
buf_rspfifo_intg_out;
157 logic rspfifo_intg_d_valid_buf;
158 logic rspfifo_intg_d_ready_buf;
159
160 assign buf_rspfifo_intg_in = {
161     tl_d_i.d_valid,
162     tl_h_i.d_ready
163 };
164
165 assign {
166     rspfifo_intg_d_valid_buf,
167     rspfifo_intg_d_ready_buf
168 } = buf_rspfifo_intg_out;
169
170 prim_buf #(
171     .Width(NumBufferBitsRspFifoIntg)
172 ) u_rspfifo_intg_prim_buf (
173     .in_i(buf_rspfifo_intg_in),
174     .out_o(buf_rspfifo_intg_out)
175 );
176
177 prim_fifo_sync
#(.Width(RSPFIFO_INTG_WIDTH), .Pass(RspPass), .Depth(RspDepth)) rsp-
fifo_intg (
178     .clk_i,
179     .rst_ni,
180     .clr_i      (1'b0),
181     .wvalid_i   (rspfifo_intg_d_valid_buf),
182     .wready_o   (),
183     .wdata_i    (tl_d_i.d_user),
184     .rvalid_o   (),
185     .rready_i   (rspfifo_intg_d_ready_buf),
186     .rdata_o    (tl_h_o.d_user),
187     .full_o     (),
188     .depth_o    (),
189     .err_o      ());
190
191 endmodule

```

21. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_jtag_dtm.sv (150 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // JTAG DTM to TL-UL Converter.
6 // This implements a JTAG DTM according to the RISC-V external de-
bug v0.13 specification and
7 // uses the TL-UL protocol to transport read/write operations
to the RISC-V debug module
8 // (i.e. the DMI is implemented with the TL-UL protocol):
9 // https://github.com/riscv/riscv-debug-spec/blob/release/
riscv-debug-release.pdf
10 //
11
12 module tlul_jtag_dtm #(
13     // Idcode value for the JTAG.
14     parameter logic [31:0] IdcodeValue = 32'h00000001,
15     parameter int unsigned NumDmiByteAbits = 18
16 ) (
17     input                                clk_i,
18     input                                rst_ni,
19     input  jtag_pkg::jtag_req_t         jtag_i,
20     output jtag_pkg::jtag_rsp_t         jtag_o,
21     // This bypasses the clock inverter inside the JTAG TAP for

```

```

scanmode.
22 input
scan_rst_ni,
23 input  prim_mubi_pkg::mubi4_t         scan-
mode_i,
24 // TL-UL-based DMI
25 output tlul_pkg::tl_h2d_t             tl_h2d_o,
26 input  tlul_pkg::tl_d2h_t             tl_d2h_i
27 );
28
29 dm::dmi_req_t dmi_req;
30 logic dmi_req_valid;
31 logic dmi_req_ready;
32 dm::dmi_resp_t dmi_resp;
33 logic dmi_resp_ready;
34 logic dmi_resp_valid;
35
36 logic scanmode;
37 prim_mubi4_dec u_prim_mubi4_dec (
38     .mubi_i(scanmode_i),
39     .mubi_dec_o(scanmode)
40 );
41
42 logic tck_muxed;
43 logic trst_n_muxed;
44 prim_clock_mux2 #(
45     .NoFpgaBufG(1'b1)
46 ) u_prim_clock_mux2 (
47     .clk0_i(jtag_i.tck),
48     .clk1_i(clk_i),
49     .sel_i (scanmode),
50     .clk_o (tck_muxed)
51 );
52
53 prim_clock_mux2 #(
54     .NoFpgaBufG(1'b1)
55 ) u_prim_rst_n_mux2 (
56     .clk0_i(jtag_i.trst_n),
57     .clk1_i(scan_rst_ni),
58     .sel_i (scanmode),
59     .clk_o (trst_n_muxed)
60 );
61
62 // TODO: At the moment, this uses the JTAG DTM implementa-
tion from the PULP project.
63 // This module should be refactored in the future to support
TL-UL directly, instead
64 // of a custom DMI protocol which necessitates another
translation layer to TL-UL.
65 logic req_ready;
66 assign req_ready = dmi_req_ready && dmi_resp_ready;
67
68 dmi_jtag #(
69     .IdcodeValue(IdcodeValue),
70     .NumDmiWordAbits(NumDmiByteAbits - 2)
71 ) u_dmi_jtag (
72     .clk_i,
73     .rst_ni,
74     .testmode_i   ( scanmode ),
75     .test_rst_ni  ( scan_rst_ni ),
76     .dmi_rst_no   ( ), // unused
77     .dmi_req_o    ( dmi_req ),
78     .dmi_req_valid_o ( dmi_req_valid ),
79     // unless there is room for response, stall
80     .dmi_req_ready_i ( req_ready ),
81     .dmi_resp_i     ( dmi_resp ),
82     .dmi_resp_ready_o ( dmi_resp_ready ),
83     .dmi_resp_valid_i ( dmi_resp_valid ),
84     .tck_i          ( tck_muxed ),
85     .tms_i          ( jtag_i.tms ),
86     .trst_ni        ( trst_n_muxed ),
87     .tdi             ( jtag_i.tdi ),
88     .tdo             ( jtag_o.tdo ),
89     .tdo_oe_o       ( jtag_o.tdo_oe )
90 );
91
92 // Outstanding request handling.
93 logic pending_req_q;
94 always_ff @(posedge clk_i or negedge rst_ni) begin : p_pend-
ing
95     if (!rst_ni) begin
96         pending_req_q <= 1'b0;
97     end else begin
98         // New request
99         if (dmi_req_ready && dmi_req_valid && (!pending_req_q))
100             pending_req_q <= 1'b1;
101         end else if (dmi_resp_valid) begin
102             pending_req_q <= 1'b0;
103         end
104     end
105 end
106
107 // DMI to TL-UL transducing
108 logic dmi_error;
109 tlul_adapter_host #(
110     // Only make one request at a time.
111     .MAX_REQS(1),
112     .EnableDataIntgGen(1)
113 ) u_tap_tlul_host (

```

```

114 .clk_i,
115 .rst_ni,
116 // Do not make more than one request at a time
117 .req_i ( dmi_req_valid && !pending_req_q ),
118 .gnt_o ( dmi_req_ready ),
119 .addr_i ( top_pkg::TL_AW'({dmi_req.addr, 2'b00}) ),
120 .we_i ( dmi_req.op == dm::DTM_WRITE ),
121 .wdata_i ( dmi_req.data ),
122 .wdata_intg_i ('0),
123 .be_i ( {top_pkg::TL_DBW{1'b1}} ),
124 .user_rsvd_i ('0),
125 .instr_type_i ( prim_mubi_pkg::MuBi4False ),
126 .valid_o ( dmi_resp_valid ),
127 .rdata_o ( dmi_resp.data ),
128 .rdata_intg_o ('0),
129 .err_o ( dmi_error ),
130 .intg_err_o ('0),
131 .tl_o ( tl_h2d_o ),
132 .tl_i ( tl_d2h_i ),
133 );
134
135 // We use the user channel to transmit the DMI response
type.
136 assign dmi_resp.resp = (dmi_error) ? dm::DTM_ERR :
dm::DTM_SUCCESS;
137
138 // These signals are unused
139 logic unused_sigs;
140 assign unused_sigs = ^{
141   dmi_req.addr[31:30],
142   tl_d2h_i.d_opcode,
143   tl_d2h_i.d_param,
144   tl_d2h_i.d_size,
145   tl_d2h_i.d_source,
146   tl_d2h_i.d_sink,
147   tl_d2h_i.d_error
148 };
149
150 endmodule : tlul_jtag_dtm

```

22. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_lc_gate.sv (267 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Life cycle gating module for TL-UL protocol.
6 // Transactions are passed through when lc_en_i == ON.
7 // In all other cases (lc_en_i != ON) incoming transactions
return a bus error.
8 //
9 // Note that the lc_en_i should be synchronized and buffered
outside of this module using
10 // an instance of prim_lc_sync.
11
12 `include "prim_assert.sv"
13
14 module tlul_lc_gate
15   import tlul_pkg::*;
16   import lc_ctrl_pkg::*;
17   #(
18     // Number of LC gating muxes in each direction.
19     // It is recommended to set this parameter to 2, which res-
ults
20     // in a total of 4 gating muxes.
21     parameter int NumGatesPerDirection = 2,
22     // By default we return a TL-UL bus error response if the
bus is
gated. However, in some special
23     // cases we need to be able to return valid, all-zero re-
sponses
instead (e.g. for the RV_DM). In
24     // those cases, ReturnBlankResp can be set to 1.
25     parameter bit ReturnBlankResp = 0,
26     // The maximum number of outstanding TL-UL requests at the
output
27     parameter int unsigned Outstanding = 3
28   ) (
29     input clk_i,
30     input rst_ni,
31
32     // To host
33     input tl_h2d_t tl_h2d_i,
34     output tl_d2h_t tl_d2h_o,
35
36     // To device
37     output tl_h2d_t tl_h2d_o,
38     input tl_d2h_t tl_d2h_i,
39
40     // Flush control signaling
41     input flush_req_i,
42     output logic flush_ack_o,
43
44     // Indicates whether there are pending responses on the
device
side.
45     output logic resp_pending_o,
46
47     // LC control signal
48     input lc_tx_t lc_en_i,
49     output logic err_o

```

```

50 );
51
52 //////////////////////////////////////////////////
53 // Access Gates //
54 //////////////////////////////////////////////////
55
56 lc_tx_t err_en;
57 lc_tx_t [NumGatesPerDirection-1:0] err_en_buf;
58
59 prim_lc_sync #(
60   .NumCopies(NumGatesPerDirection),
61   .AsyncOn(0)
62 ) u_err_en_sync (
63   .clk_i,
64   .rst_ni,
65   .lc_en_i(err_en),
66   .lc_en_o(err_en_buf)
67 );
68
69 tl_h2d_t tl_h2d_int [NumGatesPerDirection+1];
70 tl_d2h_t tl_d2h_int [NumGatesPerDirection+1];
71 for (genvar k = 0; k < NumGatesPerDirection; k++) begin :
gen_lc_gating_muxes
72   // H -> D path.
73   prim_blanker #(
74     .Width($bits(tl_h2d_t))
75   ) u_prim_blanker_h2d (
76     .in_i(tl_h2d_int[k]),
77     .en_i(lc_tx_test_false_strict(err_en_buf[k])),
78     .out_o(tl_h2d_int[k+1])
79   );
80
81   // D -> H path.
82   prim_blanker #(
83     .Width($bits(tl_d2h_t))
84   ) u_prim_blanker_d2h (
85     .in_i(tl_d2h_int[k+1]),
86     .en_i(lc_tx_test_false_strict(err_en_buf[k])),
87     .out_o(tl_d2h_int[k])
88   );
89 end
90
91 // Assign signals on the device side.
92 assign tl_h2d_o = tl_h2d_int[NumGatesPerDirection];
93 assign tl_d2h_int[NumGatesPerDirection] = tl_d2h_i;
94
95 //////////////////////////////////////////////////
96 // Host Side Interposing //
97 //////////////////////////////////////////////////
98
99 // Encoding generated with:
100 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 4 -n 8 \
101 //   -s 3379253306 --language=sv
102 //
103 // Hamming distance histogram:
104 //
105 // 0: --
106 // 1: --
107 // 2: --
108 // 3: --
109 // 4: --
110 // 5: ||| (66.67%)
111 // 6: ||| (33.33%)
112 // 7: --
113 // 8: --
114 //
115 // Minimum Hamming distance: 5
116 // Maximum Hamming distance: 6
117 // Minimum Hamming weight: 3
118 // Maximum Hamming weight: 5
119 //
120 // Encoding generated with:
121 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 5 -n 9 \
122 //   -s 686407169 --language=sv
123 //
124 // Hamming distance histogram:
125 //
126 // 0: --
127 // 1: --
128 // 2: --
129 // 3: --
130 // 4: --
131 // 5: ||| (60.00%)
132 // 6: ||| (40.00%)
133 // 7: --
134 // 8: --
135 // 9: --
136 //
137 // Minimum Hamming distance: 5
138 // Maximum Hamming distance: 6
139 // Minimum Hamming weight: 3
140 // Maximum Hamming weight: 6
141 //
142 localparam int StateWidth = 9;
143 typedef enum logic [StateWidth-1:0] {
144   StActive = 9'b100100001,
145   StOutstanding = 9'b011100111,
146   StFlush = 9'b001001100,
147   StError = 9'b010111010,
148   StErrorOutstanding = 9'b100010110
149 } state_e;

```

```

150
151 state_e state_d, state_q;
152 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StError)
153
154 logic [prim_util_pkg::vbits(Outstanding+1)-1:0] outstand-
ing_txn;
155 logic a_ack;
156 logic d_ack;
157 assign a_ack = tl_h2d_i.a_valid & tl_d2h_o.a_ready;
158 assign d_ack = tl_h2d_i.d_ready & tl_d2h_o.d_valid;
159
160 always_ff @(posedge clk_i or negedge rst_ni) begin
161   if (!rst_ni) begin
162     outstanding_txn <= '0;
163   end else if (a_ack && !d_ack) begin
164     outstanding_txn <= outstanding_txn + 1'b1;
165   end else if (d_ack && !a_ack) begin
166     outstanding_txn <= outstanding_txn - 1'b1;
167   end
168 end
169
170 logic block_cmd;
171 always_comb begin
172   block_cmd = '0;
173   state_d = state_q;
174   err_en = 0;
175   err_o = '0;
176   flush_ack_o = '0;
177   resp_pending_o = 1'b0;
178
179   unique case (state_q)
180     StActive: begin
181       if (lc_tx_test_false_loose(lc_en_i) || flush_req_i)
182         state_d = StOutstanding;
183       end
184       if (outstanding_txn != '0) begin
185         resp_pending_o = 1'b1;
186       end
187     end
188     StOutstanding: begin
189       block_cmd = 1'b1;
190       if (outstanding_txn == '0) begin
191         state_d = lc_tx_test_false_loose(lc_en_i) ? StEr-
ror : StFlush;
192       end else begin
193         resp_pending_o = 1'b1;
194       end
195     end
196     StFlush: begin
197       block_cmd = 1'b1;
198       flush_ack_o = 1'b1;
199       if (lc_tx_test_false_loose(lc_en_i)) begin
200         state_d = StError;
201       end else if (!flush_req_i) begin
202         state_d = StActive;
203       end
204     end
205     StError: begin
206       err_en = 0;
207       if (lc_tx_test_true_strict(lc_en_i)) begin
208         state_d = StErrorOutstanding;
209       end
210     end
211     StErrorOutstanding: begin
212       err_en = 0;
213       block_cmd = 1'b1;
214       if (outstanding_txn == '0) begin
215         state_d = StActive;
216       end
217     end
218     default: begin
219       err_o = 1'b1;
220       err_en = 0;
221     end
222   endcase // unique case (state_q)
223 end
224
225 // At the host side, we interpose the ready / valid signals
226 so that we can return a bus error
227 // in case the lc signal is not set to ON. Note that this
228 logic does not have to be duplicated
229 // since erroring back is considered a convenience feature
230 so that the bus does not lock up.
231 tl_h2d_t tl_h2d_error;
232 tl_d2h_t tl_d2h_error;
233 always_comb begin
234   tl_h2d_int[0] = tl_h2d_i;
235   tl_d2h_o = tl_d2h_int[0];
236   tl_h2d_error = '0;
237
238   if (lc_tx_test_true_loose(err_en)) begin
239     tl_h2d_error = tl_h2d_i;

```

```

244   tl_d2h_o = tl_d2h_error;
245 end
246
247 if (block_cmd) begin
248   tl_d2h_o.a_ready = 1'b0;
249   tl_h2d_int[0].a_valid = 1'b0;
250   tl_h2d_error.a_valid = 1'b0;
251 end
252 end
253
254 tlul_err_resp #(
255   .ReturnBlankResp(ReturnBlankResp)
256 ) u_tlul_err_resp (
257   .clk_i,
258   .rst_ni,
259   .tl_h_i(tl_h2d_error),
260   .tl_h_o(tl_d2h_error)
261 );
262
263 // Add assertion
264 `ASSERT(OutStanding0vfl_A, (outstanding_txn == Outstanding)
|> ~a_ack)
265 `ASSERT(SizeOutstandingTxn_A, (2 ** $bits(outstanding_txn))
- 1 >= Outstanding)
266
267 endmodule : tlul_lc_gate

```

23. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_pkg.sv (209 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 package tlul_pkg;
6
7 // this can be either PPC or BINTREE
8 // there is no functional difference, but timing and area
behavior is different
9 // between the two instances. PPC can result in smaller im-
plementations when timing
10
11 // is not critical, whereas BINTREE is favorable when timing
pressure is high (but this
12 // may also result in a larger implementation). on FPGA tar-
gets, BINTREE is favorable
13 // both in terms of area and timing.
14 parameter ArbiterImpl = "PPC";
15
16 typedef enum logic [2:0] {
17   PutFullData = 3'h 0,
18   PutPartialData = 3'h 1,
19   Get = 3'h 4
20 } tl_a_op_e;
21
22 typedef enum logic [2:0] {
23   AccessAck = 3'h 0,
24   AccessAckData = 3'h 1
25 } tl_d_op_e;
26
27 parameter int H2DCmdMaxWidth = 57;
28 parameter int H2DCmdIntgWidth = 7;
29 parameter int H2DCmdFullWidth = H2DCmdMaxWidth + H2DCmdIntg-
Width;
30 parameter int D2HRspMaxWidth = 57;
31 parameter int D2HRspIntgWidth = 7;
32 parameter int D2HRspFullWidth = D2HRspMaxWidth + D2HRspIntg-
Width;
33 parameter int DataMaxWidth = 32;
34 parameter int DataIntgWidth = 7;
35 parameter int DataFullWidth = DataMaxWidth + DataIntg-
Width;
36 parameter int RsvdWidth = top_pkg::TL_AUW -
prim_mubi_pkg::MuBi4Width -
H2DCmdIntgWidth - DataIntg-
Width;
37
38 // Data that is returned upon an a TL-UL error belonging to
an instruction fetch.
39
40 // Note that this data will be returned with the correct bus
integrity value.
41 parameter logic [top_pkg::TL_DW-1:0] DataWhenInstrError =
'0;
42
43 // Data that is returned upon an a TL-UL error not belonging to
an instruction fetch.
44
45 // Note that this data will be returned with the correct bus
integrity value.
46 parameter logic [top_pkg::TL_DW-1:0] DataWhenError =
{top_pkg::TL_DW{1'b1}};
47
48 typedef struct packed {
49   logic [RsvdWidth-1:0] rsvd;
50   prim_mubi_pkg::mubi4_t instr_type;
51   logic [H2DCmdIntgWidth-1:0] cmd_intg;
52   logic [DataIntgWidth-1:0] data_intg;

```



```

51 } tl_a_user_t;
52
53 parameter tl_a_user_t TL_A_USER_DEFAULT = '{
54   rsvd: '0,
55   instr_type: prim_mubi_pkg::Mubi4False,
56   cmd_intg: {H2DCmdIntgWidth{1'b1}},
57   data_intg: {DataIntgWidth{1'b1}}
58 };
59
60 typedef struct packed {
61   prim_mubi_pkg::mubi4_t      instr_type;
62   logic [top_pkg::TL_AW-1:0] addr;
63   tl_a_op_e                   opcode;
64   logic [top_pkg::TL_DBW-1:0] mask;
65 } tl_h2d_cmd_intg_t;
66
67 typedef struct packed {
68   logic                      a_valid;
69   tl_a_op_e                  a_opcode;
70   logic [2:0]                a_param;
71   logic [top_pkg::TL_SZW-1:0] a_size;
72   logic [top_pkg::TL_AIW-1:0] a_source;
73   logic [top_pkg::TL_AW-1:0]  a_address;
74   logic [top_pkg::TL_DBW-1:0] a_mask;
75   logic [top_pkg::TL_DW-1:0]  a_data;
76   tl_a_user_t                a_user;
77
78   logic                      d_ready;
79 } tl_h2d_t;
80
81 // The choice of all 1's as the blanked value is deliberate.
82 // It is assumed that most security features of the design
83 // are opt-in instead
84 // of opt-out.
85 // Given the opt-in nature, if a 0 were to propagate, the
86 // feature would be turned
87 // off. Whereas if a 1 were to propagate, it would either
88 // stay on or be turned on.
89 // There is however no perfect value for this purpose.
90 localparam logic [top_pkg::TL_DW-1:0] BlankedAData =
91 {top_pkg::TL_DW{1'b1}};
92
93 localparam tl_h2d_t TL_H2D_DEFAULT = '{
94   d_ready: 1'b1,
95   a_opcode: tl_a_op_e('0),
96   a_user:   TL_A_USER_DEFAULT,
97   a_data:   BlankedAData,
98   default: '0
99 };
100
101 typedef struct packed {
102   logic [D2HRspIntgWidth-1:0] rsp_intg;
103   logic [DataIntgWidth-1:0]   data_intg;
104 } tl_d_user_t;
105
106 parameter tl_d_user_t TL_D_USER_DEFAULT = '{
107   rsp_intg: {D2HRspIntgWidth{1'b1}},
108   data_intg: {DataIntgWidth{1'b1}}
109 };
110
111 typedef struct packed {
112   logic d_valid;
113   tl_d_op_e d_opcode;
114   logic [2:0] d_param;
115   logic [top_pkg::TL_SZW-1:0] d_size; // Bouncing back
116   a_size
117   logic [top_pkg::TL_AIW-1:0] d_source;
118   logic [top_pkg::TL_DIW-1:0] d_sink;
119   logic [top_pkg::TL_DW-1:0] d_data;
120   tl_d_user_t d_user;
121   logic d_error;
122   logic a_ready;
123 } tl_d2h_t;
124
125 typedef struct packed {
126   tl_d_op_e opcode;
127   logic [top_pkg::TL_SZW-1:0] size;
128 } // Temporarily removed because source changes throughout
129 // the fabric
130 // and thus cannot be used for end-to-end checking.
131
132 // A different PR will propose a work-around (a hoaky one)
133 // to see if
134 // it gets the job done.
135 // logic [top_pkg::TL_AIW-1:0] source;
136 // error;
137 } tl_d2h_rsp_intg_t;
138
139 localparam tl_d2h_t TL_D2H_DEFAULT = '{
140   a_ready: 1'b1,
141   d_opcode: tl_d_op_e('0),
142   d_user:   TL_D_USER_DEFAULT,
143   default: '0
144 };
145
146 // Check user for unsupported values
147 function automatic logic tl_a_user_chk(tl_a_user_t user);
148   logic malformed_err;
149   logic unused_user;

```

```

144   unused_user = user;
145   malformed_err =
146   prim_mubi_pkg::mubi4_test_invalid(user.instr_type);
147   return malformed_err;
148   endfunction // tl_a_user_chk
149
150 // extract variables used for command checking
151 function automatic tl_h2d_cmd_intg_t ex-
152 tract_h2d_cmd_intg(tl_h2d_t tl);
153   tl_h2d_cmd_intg_t payload;
154   logic unused_tlul;
155   unused_tlul = ^tl;
156   payload.addr = tl.a_address;
157   payload.opcode = tl.a_opcode;
158   payload.mask = tl.a_mask;
159   payload.instr_type = tl.a_user.instr_type;
160   return payload;
161   endfunction // extract_h2d_payload
162
163 // extract variables used for response checking
164 function automatic tl_d2h_rsp_intg_t ex-
165 tract_d2h_rsp_intg(tl_d2h_t tl);
166   tl_d2h_rsp_intg_t payload;
167   logic unused_tlul;
168   unused_tlul = ^tl;
169   payload.opcode = tl.d_opcode;
170   payload.size = tl.d_size;
171   //payload.source = tl.d_source;
172   payload.error = tl.d_error;
173   return payload;
174   endfunction // extract_d2h_rsp_intg
175
176 // calculate ecc for command checking
177 function automatic logic [H2DCmdIntgWidth-1:0]
178 get_cmd_intg(tl_h2d_t tl);
179   logic [H2DCmdIntgWidth-1:0] cmd_intg;
180   logic [H2DCmdMaxWidth-1:0] unused_cmd_payload;
181   tl_h2d_cmd_intg_t cmd;
182   cmd = extract_h2d_cmd_intg(tl);
183   {cmd_intg, unused_cmd_payload} =
184   {cmd_intg, unused_cmd_payload};
185   endfunction
186
187 prim_secdded_pkg::prim_secdded_inv_64_57_enc(H2DCmdMaxWidth'(cmd));
188   return cmd_intg;
189   endfunction // get_cmd_intg
190
191 // calculate ecc for data checking
192 function automatic logic [DataIntgWidth-1:0]
193 get_data_intg(logic [top_pkg::TL_DW-1:0] data);
194   logic [DataIntgWidth-1:0] data_intg;
195   logic [top_pkg::TL_DW-1:0] unused_data;
196   logic [DataIntgWidth + top_pkg::TL_DW - 1 : 0] enc_data;
197   enc_data =
198   prim_secdded_pkg::prim_secdded_inv_39_32_enc(data);
199   data_intg = enc_data[DataIntgWidth + top_pkg::TL_DW - 1 :
200   top_pkg::TL_DW];
201   unused_data = enc_data[top_pkg::TL_DW - 1 : 0];
202   return data_intg;
203   endfunction // get_data_intg
204
205 // return inverted integrity for command payload
206 function automatic logic [H2DCmdIntgWidth-1:0]
207 get_bad_cmd_intg(tl_h2d_t tl);
208   logic [H2DCmdIntgWidth-1:0] cmd_intg;
209   cmd_intg = get_cmd_intg(tl);
210   return ~cmd_intg;
211   endfunction // get_bad_cmd_intg
212
213 // return inverted integrity for data payload
214 function automatic logic [H2DCmdIntgWidth-1:0]
215 get_bad_data_intg(logic [top_pkg::TL_DW-1:0] data);
216   logic [H2DCmdIntgWidth-1:0] data_intg;
217   data_intg = get_data_intg(data);
218   return ~data_intg;
219   endfunction // get_bad_data_intg
220
221 endpackage

```

24. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_request_loopback.sv (122 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 module tlul_request_loopback
6   import tlul_pkg::*;
7   #(
8     parameter bit ErrorRsp = 1 // 1: Return TLUL error on on
9   )
10   input logic clk_i,
11   input logic rst_ni,
12   input logic squash_req_i,
13   input tlul_pkg::tl_h2d_t tl_h2d_i, // Incoming request
14   output tlul_pkg::tl_d2h_t tl_d2h_o, // Response
15   output tlul_pkg::tl_h2d_t tl_h2d_o, // Feed through re-
16   quest if not squashed to the target
17   input tlul_pkg::tl_d2h_t tl_d2h_i // Response from the

```

```

target
17 );
18
19 // A valid request is looped back if the squash_req_i is as-
serted in the same cycle
20 // Regardless of the request being squashed or not, the re-
quest payload is NOT modified by this
21 // module
22 logic loopback_request;
23 assign loopback_request = tl_h2d_i.a_valid & squash_req_i;
24
25 tlul_pkg::tl_h2d_t tl_muxed_h2d [2];
26 tlul_pkg::tl_d2h_t tl_muxed_d2h [2];
27 tlul_pkg::tl_h2d_t tl_error_h2d;
28 tlul_pkg::tl_d2h_t tl_error_d2h;
29
30 // Device port is Idx0
31 assign tl_h2d_o = tl_muxed_h2d[0];
32 assign tl_muxed_d2h[0] = tl_d2h_i;
33
34 // Error port is Idx1
35 assign tl_error_h2d = tl_muxed_h2d[1];
36 assign tl_muxed_d2h[1] = tl_error_d2h;
37
38 // Use a tlul_socket_in to mux between the genuine response
and the error response. The
39 // tlul_socket_in keeps track of outstanding transactions
and ensures that all responses
40 // happen in the correct order.
41 tlul_socket_in #(
42     .N(2),
43     .ExplicitErrs(0)
44 ) u_socket (
45     .clk_i,
46     .rst_ni,
47     .tl_h_i      (tl_h2d_i),
48     .tl_h_o      (tl_d2h_o),
49     .tl_d_o      (tl_muxed_d2h),
50     .tl_d_i      (tl_muxed_d2h),
51     .dev_select_i (loopback_request)
52 );
53
54 // Assemble the All-Zero response with error set if the re-
quest is squashed
55 tlul_pkg::tl_d2h_t tl_razwi_rsp_pre_intg;
56 logic loopback_valid_q;
57
58 prim_flop #(
59     .Width(1)
60 ) u_loopback_valid (
61     .clk_i ( clk_i ),
62     .rst_ni ( rst_ni ),
63     .d_i   ( tl_error_h2d.a_valid ),
64     .q_o   ( loopback_valid_q )
65 );
66
67 logic [$bits(tlul_pkg::tl_d_op_e)-1:0] d_opcode_q;
68 prim_flop #(
69     .Width($bits(tlul_pkg::tl_d_op_e)),
70     .ResetValue({AccessAck})
71 ) u_rsp_opcode (
72     .clk_i
73     ( clk_i ),
74     .rst_ni
75     ( rst_ni ),
76     .d_i
77     ( (tl_error_h2d.a_opcode == Get) ? AccessAckData :
AccessAck ),
78     .q_o
79     ( d_opcode_q )
80 );
81 assign tl_razwi_rsp_pre_intg.d_opcode =
tlul_pkg::tl_d_op_e'(d_opcode_q);
82
83 prim_flop #(
84     .Width(top_pkg::TL_SZW)
85 ) u_rsp_size (
86     .clk_i ( clk_i ),
87     .rst_ni ( rst_ni ),
88     .d_i   ( tl_error_h2d.a_size ),
89     .q_o   ( tl_razwi_rsp_pre_intg.d_size )
90 );
91
92 prim_flop #(
93     .Width(top_pkg::TL_AIW)
94 ) u_rsp_source (
95     .clk_i ( clk_i ),
96     .rst_ni ( rst_ni ),
97     .d_i   ( tl_error_h2d.a_source ),
98     .q_o   ( tl_razwi_rsp_pre_intg.d_source )
99 );
100
101 // Assemble the RAZWI (Return as Zero, Write Ignored) re-
sponse
102 assign tl_razwi_rsp_pre_intg.d_valid = loopback_valid_q;
103 assign tl_razwi_rsp_pre_intg.a_ready = 1'b1;
104
105 assign tl_razwi_rsp_pre_intg.d_param = '0;
106 assign tl_razwi_rsp_pre_intg.d_sink = '0;
107 assign tl_razwi_rsp_pre_intg.d_data = '0;
108 assign tl_razwi_rsp_pre_intg.d_user = '0;

```

```

105 assign tl_razwi_rsp_pre_intg.d_error = ErrorRsp;
106
107 // Compute integrity bits from the manually assembled RAZWI
response
108 tlul_rsp_intg_gen gen_intg_razwi_rsp (
109     .tl_i ( tl_razwi_rsp_pre_intg ),
110     .tl_o ( tl_error_d2h )
111 );
112
113 // Not all TLUL signals are read from the request muxed to
the error leg
114 logic unused_tlul_signals;
115 assign unused_tlul_signals = ^{tl_error_h2d.a_param,
116     tl_error_h2d.a_address,
117     tl_error_h2d.a_mask,
118     tl_error_h2d.a_data,
119     tl_error_h2d.a_user,
120     tl_error_h2d.d_ready};
121
122 endmodule

```

22. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_chk.sv (54 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8  * Tile-Link UL response integrity check
9  */
10
11 module tlul_rsp_intg_chk import tlul_pkg::*; #(
12     parameter bit EnableRspDataIntgCheck = 0
13 ) (
14     // TL-UL interface
15     input  tl_d2h_t tl_i,
16
17     // error output
18     output logic err_o
19 );
20
21 logic [1:0] rsp_err;
22 tl_d2h_rsp_intg_t rsp;
23 assign rsp = extract_d2h_rsp_intg(tl_i);
24
25 prim_secded_inv_64_57_dec u_chk (
26     .data_i({tl_i.d.user.rsp_intg, D2HRspMaxWidth'(rsp)}),
27     .data_o(),
28     .syndrome_o(),
29     .err_o(rsp_err)
30 );
31
32 logic rsp_data_err;
33 if (EnableRspDataIntgCheck) begin : gen_rsp_data_intg_check
34     tlul_data_integ_dec u_tlul_data_integ_dec (
35         .data_intg_i({tl_i.d.user.data_intg, DataMax-
width'(tl_i.d.data)}),
36         .data_err_o(rsp_data_err)
37     );
38 end else begin : gen_no_rsp_data_intg_check
39     assign rsp_data_err = 1'b0;
40 end
41
42 // error is not permanently latched as rsp_intg_chk is typ-
ically
43 // used near the host.
44 // if the error is permanent, it would imply the host could
forever
45 // receive bus errors and lose all ability to debug.
46 // It should be up to the host to determine the permanence
of this error.
47 assign err_o = tl_i.d_valid & (|rsp_err | rsp_data_err);
48
49 logic unused_tl;
50 assign unused_tl = |tl_i;
51
52 `ASSERT_INIT(PayloadWidthCheck, $bits(tl_d2h_rsp_intg_t) <=
D2HRspMaxWidth)
53
54 endmodule // tlul_rsp_intg_chk

```

26. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_rsp_intg_gen.sv (87 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `include "prim_assert.sv"
6
7 /**
8

```



```

9 Tile-Link UL response integrity generator
10

11 This generates integrity bits that get stored in the rsp_intg
and data_intg fields of tl_o.d_user.
12
13 If EnableRspIntgGen is true then the rsp_intg field is gener-
ated from the opcode, d_size and
14 d_error fields of the response (extracted with tlul_pkg::ex-
tract_d2h_rsp_intg). If it is false then
15 the rsp_intg field either comes from the same field in the
tl_i input (if UserInIsZero is false) or
16 is wired to zero (if UserInIsZero is true).
17
18 If EnableDataIntgGen is true then the data_intg field is gen-
erated from the d_data field of the
19 response. If it is false then the rsp_intg field either comes
from the same field in the tl_i input
20 (if UserInIsZero is false) or is wired to zero (if UserIn-
IsZero is true).
21
22 */
23
24 module tlul_rsp_intg_gen import tlul_pkg::*; #(
25     parameter bit EnableRspIntgGen = 1'b1,
26     parameter bit EnableDataIntgGen = 1'b1,
27     parameter bit UserInIsZero = 1'b0,
28     parameter bit RspIntgInIsZero = UserInIsZero
29 ) (
30     // TL-UL interface
31     input  tl_d2h_t tl_i,
32     output tl_d2h_t tl_o
33 );
34
35     logic [D2HRspIntgWidth-1:0] rsp_intg;
36     if (EnableRspIntgGen) begin : gen_rsp_intg
37         tl_d2h_rsp_intg_t rsp;
38         logic [D2HRspMaxWidth-1:0] unused_payload;
39
40         assign rsp = extract_d2h_rsp_intg(tl_i);
41
42         prim_secded_inv_64_57_enc u_rsp_gen (
43             .data_i(D2HRspMaxWidth'(rsp)),
44             .data_o({rsp_intg, unused_payload})
45         );
46     end else if (RspIntgInIsZero) begin : gen_zero_rsp_intg
47         assign rsp_intg = 0;
48     end else begin : gen_passthrough_rsp_intg
49         assign rsp_intg = tl_i.d_user.rsp_intg;
50     end
51
52     logic [DataIntgWidth-1:0] data_intg;
53     if (EnableDataIntgGen) begin : gen_data_intg
54         logic [DataMaxWidth-1:0] unused_data;
55         tlul_data_integ_enc u_tlul_data_integ_enc (
56             .data_i(DataMaxWidth'(tl_i.d_data)),
57             .data_intg_o({data_intg, unused_data})
58         );
59     end else if (UserInIsZero) begin : gen_zero_data_intg
60         assign data_intg = 0;
61     end else begin : gen_passthrough_data_intg
62         assign data_intg = tl_i.d_user.data_intg;
63     end
64
65     always_comb begin
66         tl_o = tl_i;
67         tl_o.d_user.rsp_intg = rsp_intg;
68         tl_o.d_user.data_intg = data_intg;
69     end
70
71     logic unused_tl;
72     assign unused_tl = ^tl_i;
73
74
75     `ASSERT_INIT(PayLoadWidthCheck, $bits(tl_d2h_rsp_intg_t) <=
D2HRspMaxWidth)
76     `ASSERT_INIT(DataWidthCheck_A, $bits(tl_i.d_data) <=
DataMaxWidth)
77
78 // the code below is not meant to be synthesized,
79 // but it is intended to be used in simulation and FPV
80 `ifndef SYNTHESIS
81     always @(tl_i) begin
82         `ASSERT_I(RspZero_A, tl_i.d_valid & RspIntgInIsZero -> ~|
tl_i.d_user.rsp_intg)
83         `ASSERT_I(UserZero_A, tl_i.d_valid & UserInIsZero -> ~|
tl_i.d_user)
84     end
85 `endif
86
87 endmodule // tlul_rsp_intg_gen

```

27. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_1n.sv (255 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.

```

```

3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // TL-UL socket 1:N module
6 //
7 // configuration settings
8 //     device_count: 4
9 //
10 // Verilog parameters
11 //     HReqPass:      if 1 then host requests can pass through
on empty fifo,
12 //
13 //     HRspPass:      if 1 then host responses can pass through
on empty fifo,
14 //
15 //     DReqPass:      default 1
16 //                   (one per device_count) if 1 then device i
requests can
17 //
18 //     DRspPass:      pass through on empty fifo, default 1
19 //                   (one per device_count) if 1 then device i
responses can
20 //
21 //     HReqDepth:     Depth of host request FIFO, default 2
22 //     HRspDepth:     Depth of host response FIFO, default 2
23 //     DReqDepth:     (one per device_count) Depth of device i
request FIFO,
24 //
25 //     DRspDepth:     default 2
26 //                   (one per device_count) Depth of device i
response FIFO,
27 //
28 //     default 2
29 //
30 // ExplicitErrs: This module always returns a request error if
dev_select_i
31 // is greater than N-1. If ExplicitErrs is
32 // set then the width
33 // of the dev_select_i signal will be chosen
34 // to make sure that
35 // this is possible. This only makes a dif-
36 // ference if N is a
37 // power of 2.
38 //
39 // Requests must stall to one device until all responses from
other devices
40 // have returned. Need to keep a counter of all outstanding
requests and
41 // wait until that counter is zero before switching devices.
42 //
43 // This module will return a request error if the input value
of 'dev_select_i'
44 // is not within the range 0..N-1. Thus the instantiator of
the socket
45 // can indicate error by any illegal value of dev_select_i.
4'b1111 is
46 // recommended for visibility
47 //
48 // The maximum value of N is 63
49 //
50 `include "prim_assert.sv"
51
52 module tlul_socket_1n #(
53     parameter int unsigned N = 4,
54     parameter bit HReqPass = 1'b1,
55     parameter bit HRspPass = 1'b1,
56     parameter bit [N-1:0] DReqPass = {N{1'b1}},
57     parameter bit [N-1:0] DRspPass = {N{1'b1}},
58     parameter bit [3:0] HReqDepth = 4'h1,
59     parameter bit [3:0] HRspDepth = 4'h1,
60     parameter bit [N*4-1:0] DReqDepth = {N{4'h1}},
61     parameter bit [N*4-1:0] DRspDepth = {N{4'h1}},
62     parameter bit ExplicitErrs = 1'b1,
63
64 // The width of dev_select_i. We must be able to select any
of the N devices
65 // (i.e. values 0..N-1). If ExplicitErrs is set, we also
need to be able to
66 // represent N.
67 localparam int unsigned NWD = $clog2(ExplicitErrs ? N+1 : N)
68 ) (
69     input          clk_i,
70     input          rst_ni,
71     input  tlul_pkg::tl_h2d_t tl_h_i,
72     output tlul_pkg::tl_d2h_t tl_h_o,
73     output tlul_pkg::tl_h2d_t tl_d_o [N],
74     input  tlul_pkg::tl_d2h_t tl_d_i [N],
75     input  [NWD-1:0] dev_select_i
76 );
77
78 `ASSERT_INIT(maxN, N < 64)
79
80 // Since our steering is done after potential FIFOing, we
need to
81 // shove our device select bits into spare bits of reqfifo
82 //
83 // instantiate the host fifo, create intermediate bus 't'
84 //
85 // FIFO'd version of device select
86 logic [NWD-1:0] dev_select_t;
87
88 tlul_pkg::tl_h2d_t tl_t_o;
89 tlul_pkg::tl_d2h_t tl_t_i;
90
91 tlul_fifo_sync #(
92     .ReqPass(HReqPass),

```

```

85     .RspPass(HRspPass),
86     .ReqDepth(HReqDepth),
87     .RspDepth(HRspDepth),
88     .SpareReqW(NWD)
89 ) fifo_h (
90     .clk_i,
91     .rst_ni,
92     .tl_h_i,
93     .tl_h_o,
94     .tl_d_o (tl_t_o),
95     .tl_d_i (tl_t_i),
96     .spare_req_i (dev_select_i),
97     .spare_req_o (dev_select_t),
98     .spare_rsp_i (1'b0),
99     .spare_rsp_o ());
100
101
102 // We need to keep track of how many requests are outstanding,
103 // and to which device. New requests are compared to this
104 // stall until that number is zero.
105 localparam int MaxOutstanding = 2**top_pkg::TL_AIW; // Up to
256 outstanding
106 localparam int OutstandingW = $clog2(MaxOutstanding+1);
107 logic [OutstandingW-1:0] num_req_outstanding;
108 logic [NWD-1:0] dev_select_outstanding;
109 logic hold_all_requests;
110 logic accept_t_req, accept_t_rsp;
111
112 assign accept_t_req = tl_t_o.a_valid & tl_t_i.a_ready;
113 assign accept_t_rsp = tl_t_i.d_valid & tl_t_o.d_ready;
114
115 always_ff @(posedge clk_i or negedge rst_ni) begin
116     if (!rst_ni) begin
117         num_req_outstanding <= '0;
118         dev_select_outstanding <= '0;
119     end else if (accept_t_req) begin
120         if (!accept_t_rsp) begin
121             num_req_outstanding <= num_req_outstanding + 1'b1;
122         end
123         dev_select_outstanding <= dev_select_t;
124     end else if (accept_t_rsp) begin
125         num_req_outstanding <= num_req_outstanding - 1'b1;
126     end
127 end
128
129 `ASSERT(NotOverflowed_A,
130
131     accept_t_req && !accept_t_rsp -> num_req_outstanding
132     <= MaxOutstanding)
133
134     assign hold_all_requests =
135         (num_req_outstanding != '0) &
136         (dev_select_t != dev_select_outstanding);
137
138 // Make N copies of 't' request side with modified reqvalid,
139 call
140 // them 'u[0]' .. 'u[n-1]'.
141
142 tlul_pkg::tl_h2d_t tl_u_o [N+1];
143 tlul_pkg::tl_d2h_t tl_u_i [N+1];
144
145 // ensure that when a device is not selected, both command
146 // data integrity can never match
147 tlul_pkg::tl_a_user_t blanked_auser;
148 assign blanked_auser = '{
149     rsvd: tl_t_o.a_user.rsvd,
150     instr_type: tl_t_o.a_user.instr_type,
151     cmd_intg: tlul_pkg::get_bad_cmd_intg(tl_t_o),
152     data_intg:
153     tlul_pkg::get_bad_data_intg(tlul_pkg::BlankedAData)
154 };
155
156 // if a host is not selected, or if requests are held off,
157 blank the bus
158 for (genvar i = 0 ; i < N ; i++) begin : gen_u_o
159     logic dev_select;
160     assign dev_select = dev_select_t == NWD'(i) &
161     ~hold_all_requests;
162
163     assign tl_u_o[i].a_valid = tl_t_o.a_valid & dev_select;
164     assign tl_u_o[i].a_opcode = tl_t_o.a_opcode;
165     assign tl_u_o[i].a_param = tl_t_o.a_param;
166     assign tl_u_o[i].a_size = tl_t_o.a_size;
167     assign tl_u_o[i].a_source = tl_t_o.a_source;
168     assign tl_u_o[i].a_address = tl_t_o.a_address;
169     assign tl_u_o[i].a_mask = tl_t_o.a_mask;
170     assign tl_u_o[i].a_data = dev_select ?
171         tl_t_o.a_data :
172         tlul_pkg::BlankedAData;
173     assign tl_u_o[i].a_user = dev_select ?
174         tl_t_o.a_user :
175         blanked_auser;
176
177     assign tl_u_o[i].d_ready = tl_t_o.d_ready;
178 end
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255

```

```

175     tlul_pkg::tl_d2h_t tl_t_p ;
176
177 // for the returning reqready, only look at the device we're
178 addressing
179 logic hfifo_reqready;
180 always_comb begin
181     hfifo_reqready = tl_u_i[N].a_ready; // default to error
182     for (int idx = 0 ; idx < N ; idx++) begin
183         //if (dev_select_outstanding == NWD'(idx))
184         hfifo_reqready = tl_u_i[idx].a_ready;
185         if (dev_select_t == NWD'(idx)) hfifo_reqready =
186             tl_u_i[idx].a_ready;
187         end
188         if (hold_all_requests) hfifo_reqready = 1'b0;
189     end
190 // Adding a_valid as a qualifier. This prevents the a_ready
191 from having unknown value
192 // when the address is unknown and the Host TL-UL FIFO is
193 bypass mode.
194 assign tl_t_i.a_ready = tl_t_o.a_valid & hfifo_reqready;
195
196 always_comb begin
197     tl_t_p = tl_u_i[N];
198     for (int idx = 0 ; idx < N ; idx++) begin
199         if (dev_select_outstanding == NWD'(idx)) tl_t_p =
200             tl_u_i[idx];
201     end
202     assign tl_t_i.d_valid = tl_t_p.d_valid ;
203     assign tl_t_i.d_opcode = tl_t_p.d_opcode;
204     assign tl_t_i.d_param = tl_t_p.d_param ;
205     assign tl_t_i.d_size = tl_t_p.d_size ;
206     assign tl_t_i.d_source = tl_t_p.d_source;
207     assign tl_t_i.d_sink = tl_t_p.d_sink ;
208     assign tl_t_i.d_data = tl_t_p.d_data ;
209     assign tl_t_i.d_user = tl_t_p.d_user ;
210     assign tl_t_i.d_error = tl_t_p.d_error ;
211
212 // Instantiate all the device FIFOs
213 for (genvar i = 0 ; i < N ; i++) begin : gen_dfifo
214     tlul_fifo_sync #(
215         .ReqPass(DReqPass[i]),
216         .RspPass(DRspPass[i]),
217         .ReqDepth(DReqDepth[i*4:4]),
218         .RspDepth(DRspDepth[i*4:4])
219     ) fifo_d (
220         .clk_i,
221         .rst_ni,
222         .tl_h_i (tl_u_o[i]),
223         .tl_h_o (tl_u_i[i]),
224         .tl_d_o (tl_d_o[i]),
225         .tl_d_i (tl_d_i[i]),
226         .spare_req_i (1'b0),
227         .spare_req_o (),
228         .spare_rsp_i (1'b0),
229         .spare_rsp_o ());
230 end
231
232 // Instantiate the error responder. It's only needed if a
233 value greater than
234 // N-1 is actually representable in NWD bits.
235 if ($clog2(N+1) <= NWD) begin : gen_err_resp
236     assign tl_u_o[N].d_ready = tl_t_o.d_ready;
237     assign tl_u_o[N].a_valid = tl_t_o.a_valid &
238         (dev_select_t >= NWD'(N)) &
239         ~hold_all_requests;
240     assign tl_u_o[N].a_opcode = tl_t_o.a_opcode;
241     assign tl_u_o[N].a_param = tl_t_o.a_param;
242     assign tl_u_o[N].a_size = tl_t_o.a_size;
243     assign tl_u_o[N].a_source = tl_t_o.a_source;
244     assign tl_u_o[N].a_address = tl_t_o.a_address;
245     assign tl_u_o[N].a_mask = tl_t_o.a_mask;
246     assign tl_u_o[N].a_data = tl_t_o.a_data;
247     assign tl_u_o[N].a_user = tl_t_o.a_user;
248     tlul_err_resp err_resp (
249         .clk_i,
250         .rst_ni,
251         .tl_h_i (tl_u_o[N]),
252         .tl_h_o (tl_u_i[N])
253     );
254 end else begin : gen_no_err_resp // block: gen_err_resp
255     assign tl_u_o[N] = '0;
256     assign tl_u_i[N] = '0;
257     logic unused_sig;
258     assign unused_sig = ^tl_u_o[N];
259 end
260
261 endmodule

```

28. lowRISC_opentitan/hw/ip/tlul/rtl/tlul_socket_m1.sv (260 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 //
5 // TL-UL socket M:1 module

```

```

6 //
7 // Verilog parameters
8 // M: Number of host ports.
9 // HReqPass: M bit array to allow requests to pass
through the host i
10 // FIFO with no clock delay if the request
FIFO is empty. If
11 // 1'b0, at least one clock cycle of latency
is created.
12 // Default is 1'b1.
13 // HRspPass: Same as HReqPass but for host response
FIFO.
14 // HReqDepth: Mx4 bit array. bit[i*4+:4] is depth of
host i request FIFO.
15 // Depth of zero is allowed if ReqPass is
true. A maximum value
16 // of 16 is allowed, default is 2.
17 // HRspDepth: Same as HReqDepth but for host response
FIFO.
18 // DReqPass: Same as HReqPass but for device request
FIFO.
19 // DRspPass: Same as HReqPass but for device response
FIFO.
20 // DReqDepth: Same as HReqDepth but for device request
FIFO.
21 // DRspDepth: Same as HReqDepth but for device response
FIFO.
22
23 `include "prim_assert.sv"
24
25 module tlul_socket_m1 #(
26     parameter int unsigned M = 4,
27     parameter bit [M-1:0] HReqPass = {M{1'b1}},
28     parameter bit [M-1:0] HRspPass = {M{1'b1}},
29     parameter bit [M*4-1:0] HReqDepth = {M{4'h1}},
30     parameter bit [M*4-1:0] HRspDepth = {M{4'h1}},
31     parameter bit DReqPass = 1'b1,
32     parameter bit DRspPass = 1'b1,
33     parameter bit [3:0] DReqDepth = 4'h1,
34     parameter bit [3:0] DRspDepth = 4'h1
35 ) (
36     input clk_i,
37     input rst_ni,
38
39     input tlul_pkg::tl_h2d_t tl_h_i [M],
40     output tlul_pkg::tl_d2h_t tl_h_o [M],
41
42     output tlul_pkg::tl_h2d_t tl_d_o,
43     input tlul_pkg::tl_d2h_t tl_d_i
44 );
45
46 `ASSERT_INIT(maxM, M < 16)
47
48 // Signals
49 //
50 // tl_h_i/o[0] | tl_h_i/o[1] | ... | tl_h_i/o[M-1]
51 // | | | | |
52 // | | | | |
53 // u_hostfifo[0] u_hostfifo[1] u_hostfifo[M-1]
54 // | | | | |
55 // hreq_fifo_o(i) / hrsp_fifo_i(i)
56 // | | | | |
57 // request/grant/req_data
58 // | | | | |
59 // PRIM_ARBITER
60 // | | | | |
61 // arb_valid / arb_ready / arb_data
62 // | | | | |
63 // | | | | |
64 // dreq_fifo_i / drsp_fifo_o
65 // | | | | |
66 // u_devicefifo
67 // | | | | |
68 // tl_d_o/i
69 //
70 // Required ID width to distinguish between host ports
71 // Used in response steering
72 localparam int unsigned IDW = top_pkg::TL_AIW;
73 localparam int unsigned STIDW = $clog2(M);
74
75 tlul_pkg::tl_h2d_t hreq_fifo_o [M];
76 tlul_pkg::tl_d2h_t hrsp_fifo_i [M];
77
78 logic [M-1:0] hrequest;
79 logic [M-1:0] hgrant;
80
81 tlul_pkg::tl_h2d_t dreq_fifo_i;
82 tlul_pkg::tl_d2h_t drsp_fifo_o;
83
84 logic arb_valid;
85 logic arb_ready;
86 tlul_pkg::tl_h2d_t arb_data;
87
88 // Host Req/Rsp FIFO
89 for (genvar i = 0; i < M; i++) begin : gen_host_fifo
90     tlul_pkg::tl_h2d_t hreq_fifo_i;
91
92     // ID Shifting
93     logic [STIDW-1:0] reqid_sub;
94     logic [IDW-1:0] shifted_id;
95     assign reqid_sub = i; // can cause conversion error?

```

```

96     assign shifted_id = {
97         tl_h_i[i].a_source[0+: (IDW-STIDW)],
98         reqid_sub
99     };
100
101     `ASSERT(idInRange, tl_h_i[i].a_valid |->
tl_h_i[i].a_source[IDW-1 -:STIDW] == '0)
102
103     // assign not connected bits to nc_* signal to make lint
happy
104     logic [IDW-1 : IDW-STIDW] unused_tl_h_source;
105     assign unused_tl_h_source = tl_h_i[i].a_source[IDW-1 -:
STIDW];
106
107     // Put shifted ID
108     assign hreq_fifo_i = '{
109         a_valid: tl_h_i[i].a_valid,
110         a_opcode: tl_h_i[i].a_opcode,
111         a_param: tl_h_i[i].a_param,
112         a_size: tl_h_i[i].a_size,
113         a_source: shifted_id,
114         a_address: tl_h_i[i].a_address,
115         a_mask: tl_h_i[i].a_mask,
116         a_data: tl_h_i[i].a_data,
117         a_user: tl_h_i[i].a_user,
118         d_ready: tl_h_i[i].d_ready
119     };
120
121     tlul_fifo_sync #(
122         .ReqPass (HReqPass[i]),
123         .RspPass (HRspPass[i]),
124         .ReqDepth (HReqDepth[i*4+:4]),
125         .RspDepth (HRspDepth[i*4+:4]),
126         .SpareReqW (1)
127     ) u_hostfifo (
128         .clk_i,
129         .rst_ni,
130         .tl_h_i (hreq_fifo_i),
131         .tl_h_o (tl_h_o[i]),
132         .tl_d_o (hreq_fifo_o[i]),
133         .tl_d_i (hrsp_fifo_i[i]),
134         .spare_req_i (1'b0),
135         .spare_req_o (),
136         .spare_rsp_i (1'b0),
137         .spare_rsp_o ()
138     );
139 end
140
141 // Device Req/Rsp FIFO
142 tlul_fifo_sync #(
143     .ReqPass (DReqPass),
144     .RspPass (DRspPass),
145     .ReqDepth (DReqDepth),
146     .RspDepth (DRspDepth),
147     .SpareReqW (1)
148 ) u_devicefifo (
149     .clk_i,
150     .rst_ni,
151     .tl_h_i (dreq_fifo_i),
152     .tl_h_o (drsp_fifo_o),
153     .tl_d_o (tl_d_o),
154     .tl_d_i (tl_d_i),
155     .spare_req_i (1'b0),
156     .spare_req_o (),
157     .spare_rsp_i (1'b0),
158     .spare_rsp_o ()
159 );
160
161 // Request Arbiter
162 for (genvar i = 0; i < M; i++) begin : gen_arbreqnt
163     assign hrequest[i] = hreq_fifo_o[i].a_valid;
164 end
165
166 assign arb_ready = drsp_fifo_o.a_ready;
167
168 if (tlul_pkg::ArbiterImpl == "PPC") begin : gen_arb_ppc
169     prim_arbiter_ppc #(
170         .N (M),
171         .DW ($bits(tlul_pkg::tl_h2d_t))
172     ) u_reqarb (
173         .clk_i,
174         .rst_ni,
175         .req_chk_i (1'b0), // TL-UL allows dropping
valid without ready. See #3354.
176         .req_i (hrequest),
177         .data_i (hreq_fifo_o),
178         .gnt_o (hgrant),
179         .idx_o (),
180         .valid_o (arb_valid),
181         .data_o (arb_data),
182         .ready_i (arb_ready)
183     );
184 end else if (tlul_pkg::ArbiterImpl == "BINTREE") begin :
gen_tree_arb
185     prim_arbiter_tree #(
186         .N (M),
187         .DW ($bits(tlul_pkg::tl_h2d_t))
188     ) u_reqarb (
189         .clk_i,
190         .rst_ni,
191         .req_chk_i (1'b0), // TL-UL allows dropping

```

```

valid without ready. See #3354.
192   .req_i    ( hrequest ),
193   .data_i    ( hreq_fifo_o ),
194   .gnt_o    ( hgrant ),
195   .idx_o    ( ),
196   .valid_o   ( arb_valid ),
197   .data_o    ( arb_data ),
198   .ready_i   ( arb_ready )
199 );
200 end else begin : gen_unknown
201   ASSERT_INIT(UnknownArbImpl_A, 0)
202 end
203
204 logic [ M-1:0] hfifo_rspvalid;
205 logic [ M-1:0] dfifo_rspready;
206 logic [IDW-1:0] hfifo_rspid;
207 logic dfifo_rspready_merged;
208
209 // arb_data --> dreq_fifo_i
210 // dreq_fifo_i.hd_rspready <= dfifo_rspready
211
212 assign dfifo_rspready_merged = |dfifo_rspready;
213 assign dreq_fifo_i = '{
214   a_valid:   arb_valid,
215   a_opcode:  arb_data.a_opcode,
216   a_param:   arb_data.a_param,
217   a_size:   arb_data.a_size,
218   a_source:  arb_data.a_source,
219   a_address: arb_data.a_address,
220   a_mask:   arb_data.a_mask,
221   a_data:   arb_data.a_data,
222   a_user:   arb_data.a_user,
223
224   d_ready:  dfifo_rspready_merged
225 };
226
227 // Response ID steering
228 // drsp_fifo_o --> hrsp_fifo_i[i]

```

```

229
230 // Response ID shifting before put into host fifo
231 assign hfifo_rspid = {
232   {STIDW{1'b0}},
233   drsp_fifo_o.d_source[IDW-1:STIDW]
234 };
235 for (genvar i = 0 ; i < M ; i++) begin : gen_idrouting
236   assign hfifo_rspvalid[i] = drsp_fifo_o.d_valid &
237                                     (drsp_fifo_o.d_source[0+:STIDW]
238 == i);
239   assign dfifo_rspready[i] =
240     hreq_fifo_o[i].d_ready
241                                     &
242                                     (drsp_fifo_o.d_source[0+:STIDW]
243 == i) &
244                                     drsp_fifo_o.d_valid;
245
246   assign hrsp_fifo_i[i] = '{
247     d_valid:  hfifo_rspvalid[i],
248     d_opcode: drsp_fifo_o.d_opcode,
249     d_param:  drsp_fifo_o.d_param,
250     d_size:   drsp_fifo_o.d_size,
251     d_source: hfifo_rspid,
252     d_sink:   drsp_fifo_o.d_sink,
253     d_data:   drsp_fifo_o.d_data,
254     d_user:   drsp_fifo_o.d_user,
255     d_error:  drsp_fifo_o.d_error,
256     a_ready:  hgrant[i]
257   };
258 end
259
260 // this assertion fails when rspid[0+:STIDW] not in [0..M-1]
261 `ASSERT(rspidInRange, drsp_fifo_o.d_valid |->
262   drsp_fifo_o.d_source[0+:STIDW] >= 0 &&
263   drsp_fifo_o.d_source[0+:STIDW] < M)
264 endmodule

```

SRAM 컨트롤러(스크램블)

What. SRAM 접근을 주소/데이터 스크램블하고 ECC 를 붙이는 컨트롤러.

Why it exists. 메모리에 담긴 비밀이 물리적으로 읽히는 것을 막기 위해서다. 칩을 열어 SRAM 셀을 직접 읽는 공격이 실재하므로, 저장할 때 주소와 데이터를 섞는다. 동시에 소프트 에러(우주선/알파입자)를 잡기 위해 ECC 를 건다.

학부 복습 · **EECS concepts.** 경량 블록암호(PRINCE 등) · 오류정정부호(SECDED 해밍) · 소프트 에러율(SER)과 FIT 단위.

석사 수준 · **Practice note.** ECC 를 붙이면 지연이 1사이클 늘어난다. 파이프라인이 그것을 흡수할 수 있는지 초기에 확인하라. 나중에 발견하면 타이밍 재설계다.

5 files • 1,933 lines

API — sram_ctrl.sv

Table 558. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — module / interface / package (1개)

이름	종류
sram_ctrl	module

Table 559. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — parameter (17개)

파라미터	형/폭	기본값
MemSizeRam	int	32'h1000
InstSize	int	MemSizeRam
NumRamInst	int	1
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
InstrExec	bit	1
NumPrinceRoundsHalf	int	3
NumAddrScrRounds	int	2
Outstanding	int	2
EccCorrection	bit	0
EnableRacl	bit	1'b0
RaclErrorRsp	bit	EnableRacl
RaclPolicySelNumRangesRam	int unsigned	1
RndCnstSramKey	otp_ctrl_pkg::sram_key_t	RndCnstSramKeyDefault
RndCnstSramNonce	otp_ctrl_pkg::sram_nonce_t	RndCnstSramNonceDefault
RndCnstLfsrSeed	lfsr_seed_t	RndCnstLfsrSeedDefault

파라미터	형/폭	기본값
RndCnstLfsrPerm	lfsr_perm_t	RndCnstLfsrPermDefault

Table 560. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — localparam (4개)

상수	형/폭	값
Depth	int unsigned	MemSizeRam >> 2
InstDepth	int unsigned	InstSize >> 2
AddrWidth	int unsigned	prim_util_pkg::vbits(Depth)
TotalAnchorWidth	int	\$bits(otp_ctrl_pkg::sram_key_t) +

Table 561. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv — 포트 (21개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clk_otp_i	input	logic
rst_otp_ni	input	logic
ram_tl_i	input	tlul_pkg::tl_h2d_t
ram_tl_o	output	tlul_pkg::tl_d2h_t
regs_tl_i	input	tlul_pkg::tl_h2d_t
regs_tl_o	output	tlul_pkg::tl_d2h_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
racl_policy_sel_ranges_ram_i	input	top_racl_pkg::racl_range_t [RaclPolicySelNumRanges...
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_hw_debug_en_i	input	lc_ctrl_pkg::lc_tx_t
otp_en_sram_ifetch_i	input	prim_mubi_pkg::mubi8_t
sram_otp_key_o	output	otp_ctrl_pkg::sram_otp_key_req_t
sram_otp_key_i	input	otp_ctrl_pkg::sram_otp_key_rsp_t
ram_cfg_i	input	prim_ram_1p_pkg::ram_1p_cfg_req_t [NumRamInst-1:0]
ram_cfg_o	output	prim_ram_1p_pkg::ram_1p_cfg_rsp_t [NumRamInst-1:0]
sram_rerror_o	output	sram_ctrl_pkg::sram_error_t

API — sram_ctrl_pkg.sv**Table 562. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv — module / interface / package (17개)**

이름	종류
sram_ctrl_pkg	package

Table 563. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv — parameter (8개)

파라미터	형/폭	기본값
DataWidth	int	32 + tlul_pkg::DataIntgWidth
NonceWidth	int	64
RndCnstSramKeyDefault	otp_ctrl_pkg::sram_key_t	128'hbecda03b34bc0418a30a33861a610f71
RndCnstSramNonceDefault	otp_ctrl_pkg::sram_nonce_t	128'h22f296f8f95efb84a75cd435a5541e9f
LfsrWidth	int	64
RndCnstLfsrSeedDefault	lfsr_seed_t	64'hb496209a_10a81ea5
RndCnstLfsrPermDefault	lfsr_perm_t	{
LfsrOutWidth	int	32

Table 564. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv — typedef / using (2개)

새 이름	원래 형
lfsr_seed_t	logic [LfsrWidth-1:0]
lfsr_perm_t	logic [LfsrWidth-1:0][\$clog2(LfsrWidth)-1:0]

API — sram_ctrl_ram_reg_top.sv

Table 565. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv — module / interface / package (1개)

이름	종류
sram_ctrl_ram_reg_top	module

Table 566. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacl	bit	1'b0
RaclErrorRsp	bit	1'b1

Table 567. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

API — sram_ctrl_reg_pkg.sv**Table 568. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_reg_pkg.sv — module / interface / package (1개)**

이름	종류
sram_ctrl_reg_pkg	package

Table 569. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_reg_pkg.sv — parameter (14개)

파라미터	형/폭	기본값
NumAlerts	int	1
RegsAw	int	6
NumRegsRegs	int	9
SRAM_CTRL_ALERT_TEST_OFFSET	logic [RegsAw-1:0]	6'h 0
SRAM_CTRL_STATUS_OFFSET	logic [RegsAw-1:0]	6'h 4
SRAM_CTRL_EXEC_REGWEN_OFFSET	logic [RegsAw-1:0]	6'h 8
SRAM_CTRL_EXEC_OFFSET	logic [RegsAw-1:0]	6'h c
SRAM_CTRL_CTRL_REGWEN_OFFSET	logic [RegsAw-1:0]	6'h 10
SRAM_CTRL_CTRL_OFFSET	logic [RegsAw-1:0]	6'h 14
SRAM_CTRL_SCR_KEY_ROTATED_OFFSET	logic [RegsAw-1:0]	6'h 18
SRAM_CTRL_READBACK_REGWEN_OFFSET	logic [RegsAw-1:0]	6'h 1c
SRAM_CTRL_READBACK_OFFSET	logic [RegsAw-1:0]	6'h 20
SRAM_CTRL_ALERT_TEST_RESVAL	logic [0:0]	1'h 0
SRAM_CTRL_ALERT_TEST_FATAL_ERROR_RESVAL	logic [0:0]	1'h 0

API — sram_ctrl_regs_reg_top.sv**Table 570. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — module / interface / package (1개)**

이름	종류
sram_ctrl_regs_reg_top	module

Table 571. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — parameter (2개)

파라미터	형/폭	기본값
EnableRacL	bit	1'b0
RacLErrorRsp	bit	1'b1

Table 572. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	6
DW	int	32
DBW	int	DW/8

Table 573. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	sram_ctrl_reg_pkg::sram_ctrl_regs_reg2hw_t
hw2reg	input	sram_ctrl_reg_pkg::sram_ctrl_regs_hw2reg_t
racl_policies_i	input	top_racl_pkg::racl_policy_vec_t
racl_error_o	output	top_racl_pkg::racl_error_log_t
intg_err_o	output	logic

1. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl.sv (799 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // SRAM controller.
6 //
7
8 `include "prim_assert.sv"
9
10 module sram_ctrl
11 import sram_ctrl_pkg::*;
12 import sram_ctrl_reg_pkg::*;
13 #(
14 // Number of words stored in the SRAM.
15 parameter int MemSizeRam
16 = 32'h1000,
17 parameter int InstSize
18 = MemSizeRam,
19 parameter int NumRamInst
20 = 1,
21 // Enable asynchronous transitions on alerts.
22 parameter logic [NumAlerts-1:0] AlertAsyncOn
23 = {NumAlerts{1'b1}},
24 // Number of cycles a differential skew is tolerated on the
  alert signal
25 parameter int unsigned AlertSkewCycles
26 = 1,
27 // Enables the execute from SRAM feature.
28 parameter bit InstrExec
29 = 1,
30 // Number of PRINCE half rounds for the SRAM scrambling fea-
  ture, can be [1..5].
31 // Note that this needs to be low-latency, hence we have to
  keep the amount of cipher rounds low.
32 // PRINCE has 5 half rounds in its original form, which cor-
  responds to 2*5 + 1 effective rounds.
33 // Setting this to 3 lowers this to approximately 7 effective
  rounds.
34 parameter int NumPrinceRoundsHalf
35 = 3,
36 // Number of address scrambling rounds. Setting this to 0
  disables address scrambling.
37 parameter int NumAddrScrRounds
38 = 2,
39 // Number of outstanding TLUL transfers
40 parameter int Outstanding
41 = 2,
42 // Enable single-bit error correction and error logging
43 parameter bit EccCorrection
44 = 0,
45 // RACL configuration
46 parameter bit EnableRacl
47 = 1'b0,
48 parameter bit RaclErrorRsp
49 = EnableRacl,
50 parameter top_racl_pkg::racl_policy_sel_t RaclPolicySel
51 = RaclPolicySelVecRegs[NumRegsRegs] = '{NumRegsRegs{0}},
52 parameter int unsigned RaclPolicySelNumRangesRam
53 = 1,
54 // Random netlist constants
55 parameter otp_ctrl_pkg::sram_key_t RndCnstSramKey
56 = RndCnstSramKeyDefault,

```

```

42 parameter otp_ctrl_pkg::sram_nonce_t RndCnstSramNonce =
  RndCnstSramNonceDefault,
43 parameter lfsr_seed_t RndCnstLfsrSeed =
  RndCnstLfsrSeedDefault,
44 parameter lfsr_perm_t RndCnstLfsrPerm =
  RndCnstLfsrPermDefault
45 ) (
46 // SRAM Clock
47 input logic clk_i,
48 input logic rst_ni,
49 // OTP Clock (for key interface)
50 input logic clk_otp_i,
51 input logic rst_otp_ni,
52 // Bus Interface (device) for SRAM
53 input tlul_pkg::tl_h2d_t ram_tl_i,
54 output tlul_pkg::tl_d2h_t ram_tl_o,
55 // Bus Interface (device) for CSRs
56 input tlul_pkg::tl_h2d_t regs_tl_i,
57 output tlul_pkg::tl_d2h_t regs_tl_o,
58 // Alert outputs.
59 input prim_alert_pkg::alert_rx_t [NumAlerts-1:0] alert_rx_i,
60 output prim_alert_pkg::alert_tx_t [NumAlerts-1:0] alert_tx_o,
61 // RACL interface
62 input top_racl_pkg::racl_policy_vec_t racl_policies_i,
63 output top_racl_pkg::racl_error_log_t racl_error_o,
64 input top_racl_pkg::racl_range_t [RaclPolicySelNumRanges-
  Ram-1:0] racl_policy_sel_ranges_ram_i,
65 // Life-cycle escalation input (scraps the scrambling keys)
66 // SEC_CM: LC_ESCALATE_EN.INTERSIG.MUBI
67 input lc_ctrl_pkg::lc_tx_t lc_escalate_en_i,
68 // SEC_CM: LC_HW_DEBUG_EN.INTERSIG.MUBI
69 input lc_ctrl_pkg::lc_tx_t lc_hw_debug_en_i,
70 // Otp configuration for sram execution
71 // SEC_CM: EXEC.INTERSIG.MUBI
72 input prim_mubi_pkg::mubi8_t otp_en_sram_ifetch_i,
73 // Key request to OTP (running on clk_fixed)
74 // SEC_CM: SCRAMBLE.KEY.SIDELoad
75 output otp_ctrl_pkg::sram_otp_key_req_t sram_otp_key_o,
76 input otp_ctrl_pkg::sram_otp_key_rsp_t sram_otp_key_i,
77 // config
78 input prim_ram_1p_pkg::ram_1p_cfg_req_t [NumRamInst-1:0] ram_cfg_i,
79 output prim_ram_1p_pkg::ram_1p_cfg_rsp_t [NumRamInst-1:0] ram_cfg_o,
80 // Error record
81 output sram_ctrl_pkg::sram_error_t sram_error_o
82 );
83
84 import lc_ctrl_pkg::lc_tx_t;
85 import lc_ctrl_pkg::lc_tx_test_true_loose;
86 import lc_ctrl_pkg::lc_tx_bool_to_lc_tx;

```

```

87 import lc_ctrl_pkg::lc_tx_or_hi;
88 import lc_ctrl_pkg::lc_tx_inv;
89 import lc_ctrl_pkg::lc_to_mubi4;
90 import prim_mubi_pkg::mubi4_t;
91 import prim_mubi_pkg::mubi8_t;
92 import prim_mubi_pkg::MuBi4True;
93 import prim_mubi_pkg::MuBi4False;
94 import prim_mubi_pkg::mubi8_test_true_strict;
95
96 // The memory can have a non-power-of-2 size (checked inside
prim_ram_1p_scr) but the size needs
97 // to be divisible by 4.
98 `ASSERT_INIT(MemSizeRamDivisibleBy4_A, MemSizeRam % 4 == 0)
99
100 // This is later on pruned to the correct width at the SRAM
wrapper
interface.
101 localparam int unsigned Depth = MemSizeRam >> 2;
102 localparam int unsigned InstDepth = InstSize >> 2;
103 localparam int unsigned AddrWidth =
prim_util_pkg::vbits(Depth);
104
105 `ASSERT_INIT(NumRamInstSameAsComputed_A,
106 NumRamInst ==
prim_util_pkg::ceil_div(MemSizeRam, InstSize))
107
108 `ASSERT_INIT(NonceWidthsLessThanSource_A, NonceWidth + Lfsr-
Width <= otp_ctrl_pkg::SramNonceWidth)
109
110 top_racl_pkg::racl_error_log_t racl_error[2];
111 if (EnableRacl) begin : gen_racl_error_arb
112 // Arbitrate between all simultaneously valid error log
requests.
113 prim_racl_error_arb #(
114 .N ( 2 )
115 ) u_prim_err_arb (
116 .clk_i,
117 .rst_ni,
118 .error_log_i ( racl_error ),
119 .error_log_o ( racl_error_o )
120 );
121 end else begin : gen_no_racl_error_arb
122 logic unused_signals;
123 assign unused_signals = ^{racl_error[0] ^ racl_error[1]};
124 assign racl_error_o = '0;
125 end
126
127 ///////////////////////////////////////////////////
128 // Anchor incoming seeds and constants
129 ///////////////////////////////////////////////////
130 localparam int TotalAnchorWidth = $bits(otp_c-
trl_pkg::sram_key_t) +
131 $bits(otp_c-
trl_pkg::sram_nonce_t);
132
133 otp_ctrl_pkg::sram_key_t cnst_sram_key;
134 otp_ctrl_pkg::sram_nonce_t cnst_sram_nonce;
135
136 prim_sec_anchor_buf #(
137 .Width(TotalAnchorWidth)
138 ) u_seed_anchor (
139 .in_i({RndCnstSramKey,
140 RndCnstSramNonce}),
141 .out_o({cnst_sram_key,
142 cnst_sram_nonce})
143 );
144
145 ///////////////////////////////////////////////////
146 // CSR Node and Mapping //
147 ///////////////////////////////////////////////////
148
149 // We've got two bus interfaces + an lc_gate in this module,
150 // hence three integ failure sources.
151 logic [2:0] bus_integ_error;
152
153 sram_ctrl_regs_reg2hw_t reg2hw;
154 sram_ctrl_regs_hw2reg_t hw2reg;
155
156 // SEC_CM: CTRL.CONFIG.REGWEN
157 // SEC_CM: EXEC.CONFIG.REGWEN
158 // SEC_CM: READBACK.CONFIG.REGWEN
159 sram_ctrl_regs_reg_top #(
160 .EnableRacl ( EnableRacl ),
161 .RaclErrorRsp ( RaclErrorRsp ),
162 .RaclPolicySelVec ( RaclPolicySelVecRegs )
163 ) u_reg_regs (
164 .clk_i,
165 .rst_ni,
166 .tl_i ( regs_tl_i ),
167 .tl_o ( regs_tl_o ),
168 .reg2hw,
169 .hw2reg,
170 // RACL interface
171 .racl_policies_i ( racl_policies_i ),
172 .racl_error_o ( racl_error[0] ),
173 // SEC_CM: BUS.INTEGRITY
174 .integ_err_o ( bus_integ_error[0] )
175 );
176
177 // Key and attribute outputs to scrambling device

```

```

179 logic [otp_ctrl_pkg::SramKeyWidth-1:0] key_d, key_q;
180 logic [otp_ctrl_pkg::SramNonceWidth-1:0] nonce_d, nonce_q;
181
182 // tie-off unused nonce bits
183 if (otp_ctrl_pkg::SramNonceWidth > LfsrWidth + NonceWidth)
begin : gen_nonce_tieoff
184 logic unused_nonce;
185 assign unused_nonce =
^nonce_q[otp_ctrl_pkg::SramNonceWidth-1:LfsrWidth + NonceWidth];
186 end
187
188 ///////////////////////////////////////////////////
189 // Alert Sender //
190 ///////////////////////////////////////////////////
191
192 logic alert_test;
193 assign alert_test = reg2hw.alert_test.q &
reg2hw.alert_test.qe;
194
195 assign hw2reg.status.bus_integ_error.d = 'b1;
196 assign hw2reg.status.bus_integ_error.de = |bus_integ_error;
197
198 logic init_error;
199 assign hw2reg.status.init_error.d = 'b1;
200 assign hw2reg.status.init_error.de = init_error;
201
202 logic readback_error;
203 assign hw2reg.status.readback_error.d = 'b1;
204 assign hw2reg.status.readback_error.de = readback_error;
205
206 logic sram_alert;
207 assign hw2reg.status.sram_alert.d = 'b1;
208 assign hw2reg.status.sram_alert.de = sram_alert;
209
210 logic alert_req;
211 assign alert_req = (|bus_integ_error) | init_error | read-
back_error | sram_alert;
212
213 prim_alert_sender #(
214 .AsyncOn(AlertAsyncOn[0]),
215 .SkewCycles(AlertSkewCycles),
216 .IsFatal(1)
217 ) u_prim_alert_sender_parity (
218 .clk_i,
219 .rst_ni,
220 .alert_test_i ( alert_test ),
221 .alert_req_i ( alert_req ),
222 .alert_ack_o ( ),
223 .alert_state_o ( ),
224 .alert_rx_i ( alert_rx_i[0] ),
225 .alert_tx_o ( alert_tx_o[0] )
226 );
227
228 ///////////////////////////////////////////////////
229 // Escalation Triggers //
230 ///////////////////////////////////////////////////
231
232 lc_tx_t [1:0] escalate_en;
233 prim_lc_sync #(
234 .NumCopies (2)
235 ) u_prim_lc_sync (
236 .clk_i,
237 .rst_ni,
238 .lc_en_i (lc_escalate_en_i),
239 .lc_en_o (escalate_en)
240 );
241
242 // SEC_CM: KEY.GLOBAL_ESC
243 logic escalate;
244 assign escalate = lc_tx_test_true_loose(escalate_en[0]);
245 assign hw2reg.status.escalated.d = 'b1;
246 assign hw2reg.status.escalated.de = escalate;
247
248 // SEC_CM: KEY.LOCAL_ESC
249 // Aggregate external and internal escalation sources.
250 // This is used in countermeasures further below (key reset
and transaction blocking).
251 logic local_esc, local_esc_reg;
252
253 // This signal only aggregates registered escalation signals
and is used for transaction
254 // blocking further below, which is on a timing-critical
path.
255 assign local_esc_reg = reg2hw.status.escalated.q |
256 reg2hw.status.init_error.q |
257 reg2hw.status.bus_integ_error.q |
258 reg2hw.status.sram_alert.q |
259 reg2hw.status.readback_error.q;
260
261 // This signal aggregates all escalation trigger signals,
including the ones that are generated
262 // in the same cycle such as init_error, sram alert, and
bus_integ_error. It is used for
263 // countermeasures that are not on the critical path (such
as clearing the scrambling keys).
264 assign local_esc = escalate |
265 init_error |
266 (|bus_integ_error) |
267 sram_alert |
268 readback_error |
269 local_esc_reg;

```

```

269 // Convert registered, local escalation sources to a multi-
it signal and combine this with

270 // the incoming escalation enable signal before feeding into
the TL-UL gate further below.
271 lc_tx_t lc_tlul_gate_en;
272 assign lc_tlul_gate_en = lc_tx_inv(lc_tx_or_hi(escal-
ate_en[1],
273
lc_tx_bool_to_lc_tx(local_esc_reg)));
274 ///////////////////////////////////////////////////
275 // HW Initialization //
276 ///////////////////////////////////////////////////
277
278 // A write to the init register reloads the LFSR seed, re-
sets the init counter and
279 // sets init_q to flag a pending initialization request.
280 logic init_trig, init_q;
281 assign init_trig = reg2hw.ctrl.init.q &&
282 reg2hw.ctrl.init.qe &&
283 !init_q; // Ignore new requests while
memory init is already pending.
284
285 logic init_d, init_done;
286 assign init_d = (init_done) ? 1'b0 :
287 (init_trig) ? 1'b1 : init_q;
288
289 always_ff @(posedge clk_i or negedge rst_ni) begin : p_in-
it_reg
290 if(!rst_ni) begin
291 init_q <= 1'b0;
292 end else begin
293 init_q <= init_d;
294 end
295 end
296
297 // This waits until the scrambling keys are actually valid
(this allows the SW to trigger
298 // key renewal and initialization at the same time).
299 logic init_req;
300 logic [AddrWidth-1:0] init_cnt;
301 logic key_req_pending_d, key_req_pending_q;
302 assign init_req = init_q & ~key_req_pending_q;
303 assign init_done = (init_cnt == AddrWidth'(Depth - 1)) &
init_req;
304
305 // We employ two redundant counters to guard against FI at-
tacks.
306 // If any of the two is glitched and the two counter states
do not agree,
307 // we trigger an alert.
308 // SEC_CM: INIT.CTR.RETURN
309 prim_count #(
310 .Width(AddrWidth)
311 ) u_prim_count (
312 .clk_i,
313 .rst_ni,
314 .clr_i(init_trig),
315 .set_i(1'b0),
316 .set_cnt_i(0),
317 .incr_en_i(init_req),
318 .decr_en_i(1'b0),
319 .step_i(AddrWidth'(1)),
320 .commit_i(1'b1),
321 .cnt_o(init_cnt),
322 .cnt_after_commit_o(),
323 .err_o(init_error)
324 );
325
326 // Clear this bit on local escalation.

327 assign hw2reg.status.init_done.d = init_done & ~init_trig &
~local_esc;
328 assign hw2reg.status.init_done.de = init_done | init_trig |
local_esc;
329 ///////////////////////////////////////////////////
330 // Scrambling Key Request //
331 ///////////////////////////////////////////////////
332
333
334 // The scrambling key and nonce have to be requested from
the OTP controller via a req/ack
335 // protocol. Since the OTP controller works in a different
clock domain, we have to synchronize
336 // the req/ack protocol as described in more details in the
OTP controller documentation.
337 //
338 // This is specialised for different tops that use it but
the req/ack protocol is the same in each
339 // case. For one example, see
340 //
341 // https://opentitan.org/book/hw/top_earlgrey/
342 // ip_autogen/otp_ctrl/doc/interfaces.html#interfaces-to-
sram-and-otbn-scramblers
343 logic key_req, key_ack;
344 assign key_req = reg2hw.ctrl.renew_scr_key.q &&
345 reg2hw.ctrl.renew_scr_key.qe &&
346 !key_req_pending_q && // Ignore new re-
quests while a request is already pending.
347 !init_q; // Ignore new requests while
memory init is already pending.

```

```

348
349 assign key_req_pending_d = (key_req) ? 1'b1 :
350 (key_ack) ? 1'b0 : key_req_pend-
ing_q;
351
352 // Clear this bit on local escalation.
353 assign hw2reg.status.scr_key_valid.d = key_ack & ~key_req
& ~local_esc;

354 assign hw2reg.status.scr_key_valid.de = key_req | key_ack |
local_esc;
355
356 // As opposed to scr_key_valid, SW is responsible for clear-
ing this register.
357 // It is not automatically cleared by HW, except when escal-
ating.
358 assign hw2reg.scr_key_rotated.d = (key_ack & ~local_esc) ?
MuBi4True : MuBi4False;
359 assign hw2reg.scr_key_rotated.de = key_ack | local_esc;
360
361 // Clear this bit on local escalation.
362 logic key_seed_valid;
363 assign hw2reg.status.scr_key_seed_valid.d = key_seed_valid
& ~local_esc;
364 assign hw2reg.status.scr_key_seed_valid.de = key_ack | loc-
al_esc;
365
366 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
367 if (!rst_ni) begin
368 key_req_pending_q <= 1'b0;
369 // reset case does not use buffered values as the
370 // reset value will be directly encoded into flop types
371 key_q <= RndCnstSramKey;
372 nonce_q <= RndCnstSramNonce;
373 end else begin
374 key_req_pending_q <= key_req_pending_d;
375 if (key_ack) begin
376 key_q <= key_d;
377 nonce_q <= nonce_d;
378 end
379 // This scraps the keys.
380 // SEC_CM: KEY.GLOBAL_ESC
381 // SEC_CM: KEY.LOCAL_ESC
382 if (local_esc) begin
383 key_q <= cnst_sram_key;
384 nonce_q <= cnst_sram_nonce;
385 end
386 end
387 end
388
389 prim_sync_reqack_data #(
390 .Width($bits(otp_ctrl_pkg::sram_otp_key_rsp_t)-1),
391 .DataSrc2Dst(1'b0)
392 ) u_prim_sync_reqack_data (
393 .clk_src_i ( clk_i ),
394 .rst_src_ni ( rst_ni ),
395 .clk_dst_i ( clk_otp_i ),
396 .rst_dst_ni ( rst_otp_ni ),
397 .req_chk_i ( 1'b1 ),
398 .src_req_i ( key_req_pending_q ),
399 .src_ack_o ( key_ack ),
400 .dst_req_o ( sram_otp_key_o.req ),
401 .dst_ack_i ( sram_otp_key_i.ack ),
402 .data_i ( {sram_otp_key_i.key,
sram_otp_key_i.nonce,
sram_otp_key_i.seed_valid} ),
403 .data_o ( {key_d,
nonce_d,
key_seed_valid} )
404 );
405
406
407
408
409
410 logic unused_csr_sigs;
411 assign unused_csr_sigs = ^{reg2hw.status.init_done.q,
reg2hw.status.scr_key_seed_valid.q};
412
413 ///////////////////////////////////////////////////
414 // SRAM Execution //
415 ///////////////////////////////////////////////////
416
417
418 mubi4_t en_ifetch;
419 if (InstrExec) begin : gen_instr_ctrl
420 lc_tx_t lc_hw_debug_en;
421 prim_lc_sync #(
422 .NumCopies (1)
423 ) u_prim_lc_sync_hw_debug_en (
424 .clk_i,
425 .rst_ni,
426 .lc_en_i (lc_hw_debug_en_i),
427 .lc_en_o ({lc_hw_debug_en})
428 );
429
430 mubi8_t otp_en_sram_ifetch;
431 prim_mubi8_sync #(
432 .NumCopies (1)
433 ) u_prim_mubi8_sync_otp_en_sram_ifetch (
434 .clk_i,
435 .rst_ni,
436 .mubi_i(otp_en_sram_ifetch_i),
437 .mubi_o({otp_en_sram_ifetch})
438 );

```

```

439
440     mubi4_t lc_ifetch_en;
441     mubi4_t reg_ifetch_en;
442     // SEC_CM: INSTR.BUS.LC_GATED
443     assign lc_ifetch_en = lc_to_mubi4(lc_hw_debug_en);
444     // SEC_CM: EXEC.CONFIG.MUBI
445     assign reg_ifetch_en = mubi4_t'(reg2hw.exec.q);
446     // SEC_CM: EXEC.INTERSIG.MUBI
447     assign en_ifetch = (mu-
bi8_test_true_strict(otp_en_sram_ifetch)) ? reg_ifetch_en :
448     lc_ifetch_en;
449     end else begin : gen_tieoff
450         assign en_ifetch = MUBI4False;
451
452         // tie off unused signals
453         logic unused_sigs;
454         assign unused_sigs = ^{lc_hw_debug_en_i,
455             reg2hw.exec.q,
456             otp_en_sram_ifetch_i};
457     end
458
459     // Initialization LFSR //
460     // Initialization LFSR //
461     // Initialization LFSR //
462
463     logic [LfsrOutWidth-1:0] lfsr_out;
464     prim_lfsr #(
465         .LfsrDw ( LfsrWidth ),
466         .EntropyDw ( LfsrWidth ),
467         .StateOutDw ( LfsrOutWidth ),
468         .DefaultSeed ( RndCnstLfsrSeed ),
469         .StatePermEn ( 1'b1 ),
470         .StatePerm ( RndCnstLfsrPerm )
471     ) u_lfsr (
472         .clk_i,
473         .rst_ni,
474         .lfsr_en_i(init_req),
475         .seed_en_i(init_trig),
476         .seed_i(nonce_q[NonceWidth+: LfsrWidth]),
477         .entropy_i('0),
478         .state_o(lfsr_out)
479     );
480
481     // Compute the correct integrity alongside for the pseudo-
random
482     initialization values.
483     logic [DataWidth - 1 :0] lfsr_out_integ;
484     tlul_data_integ_enc u_tlul_data_integ_enc (
485         .data_i(lfsr_out),
486         .data_intg_o(lfsr_out_integ)
487     );
488
489     // SRAM TL-UL Access Gate //
490     // SRAM TL-UL Access Gate //
491
492     logic tl_gate_resp_pending;
493     tlul_pkg::tl_h2d_t ram_tl_in_gated;
494     tlul_pkg::tl_d2h_t ram_tl_out_gated;
495
496     // SEC_CM: RAM_TL_LC_GATE.FSM.SPARSE
497     tlul_lc_gate #(
498         .NumGatesPerDirection(2),
499         .Outstanding(Outstanding)
500     ) u_tlul_lc_gate (
501         .clk_i,
502         .rst_ni,
503         .tl_h2d_i(ram_tl_in),
504         .tl_d2h_o(ram_tl_o),
505         .tl_h2d_o(ram_tl_in_gated),
506         .tl_d2h_i(ram_tl_out_gated),
507         .flush_req_i('0),
508         .flush_ack_o(),
509         .resp_pending_o(tl_gate_resp_pending),
510         .lc_en_i (lc_tlul_gate_en),
511         .err_o (bus_integ_error[2])
512     );
513
514     // SRAM with scrambling device //
515     // SRAM with scrambling device //
516
517     logic tlul_req, tlul_gnt, tlul_we;
518     logic [AddrWidth-1:0] tlul_addr;
519     logic [DataWidth-1:0] tlul_wdata, tlul_wmask;
520
521     logic sram_intg_error, sram_req, sram_gnt, sram_we, sram_r-
valid, sram_rvalid_scr;
522     logic [1:0] sram_error, sram_error_scr;
523     logic [AddrWidth-1:0] sram_addr, sram_error_addr_scr;
524     logic [DataWidth-1:0] sram_wdata, sram_wmask, sram_rdata,
sram_rdata_scr;
525     logic
526         sram_wpending, sram_wr_collision;
527
528     logic sram_compound_txn_in_progress;
529
530
531     // SEC_CM: MEM.READBACK
532     mubi4_t reg_readback_en;
533     assign reg_readback_en = mubi4_t'(reg2hw.readback.q);

```

```

534
535     tlul_adapter_sram_racl #(
536         .SramAw(AddrWidth),
537         .SramDw(DataWidth - tlul_pkg::DataIntgWidth),
538         .SramDepth(Depth),
539         .Outstanding(Outstanding),
540         .ByteAccess(1),
541         .CmdIntgCheck(1),
542         .EnableRspIntgGen(1),
543         .EnableDataIntgGen(0),
544         .EnableDataIntgPt(1), // SEC_CM: MEM.INTEGRITY
545         .SecFifoPtr (1), // SEC_CM: TLUL_FIFO.CTR.REDUN
546         .EnableReadback (1), // SEC_CM: MEM.READBACK
547         .EnableRacl(EnableRacl),
548         .RaclErrorRsp(RaclErrorRsp),
549         .RaclPolicySelNumRanges(RaclPolicySelNumRangesRam)
550     ) u_tlul_adapter_sram_racl (
551         .clk_i,
552         .rst_ni,
553         .tl_i (ram_tl_in_gated),
554         .tl_o (ram_tl_out_gated),
555         .en_ifetch_i (en_ifetch),
556         .req_o (tlul_req),
557         .req_type_o (),
558         .gnt_i (tlul_gnt),
559         .we_o (tlul_we),
560         .addr_o (tlul_addr),
561         .wdata_o (tlul_wdata),
562         .wmask_o (tlul_wmask),
563         // SEC_CM: BUS.INTEGRITY
564         .intg_error_o (bus_integ_error[1]),
565         .user_rsvd_o (),
566         .rdata_i (sram_rdata),
567         .rvalid_i (sram_rvalid),
568         .rerror_i (sram_error),
569         .compound_txn_in_progress_o (sram_com-
pound_txn_in_progress),
570         .readback_en_i (reg_readback_en),
571         .readback_error_o (readback_error),
572         .wr_collision_i (sram_wr_collision),
573         .write_pending_i (sram_wpending),
574         // RACL interface
575         .racl_policies_i (racl_policies_i),
576         .racl_error_o (racl_error[1]),
577         .racl_policy_sel_ranges_i (racl_policy_sel_ranges_ram_i)
578     );
579
580     logic key_valid;
581
582     // Interposing mux logic for initialization with pseudo ran-
dom
583     data.
584     assign sram_req = tlul_req | init_req;
585     // This grant signal acts more like a ready internally in
tlul_adapter_sram. In particular it's
586     // fine to assert it when tlul_req is low (it has no ef-
fect). So here tlul_gnt is asserted when
587     // a request from tlul_req will be granted regardless of
whether a request exists. This is done
588     // for timing reasons so that the output TL ready isn't com-
binatorially connected to the incoming
589
590     // TL valid. In particular we must not use `sram_gnt` in the
expression to avoid this.
591     assign tlul_gnt = key_valid & ~init_req;
592     assign sram_we = tlul_we | init_req;
593     assign sram_intg_error = |bus_integ_error[2:1] & ~init_req;
594     assign sram_addr = (init_req) ? init_cnt :
tlul_addr;
595     assign sram_wdata = (init_req) ? lfsr_out_integ :
tlul_wdata;
596     assign sram_wmask = (init_req) ? {DataWidth{1'b1}} :
tlul_wmask;
597
598     if (EccCorrection) begin : gen_ecc_correction
599         // Detect ECC errors and decode data. If data is correct-
able, data is genuine and we
600         // can use it further and re-encode
601         logic [31:0] dec_data;
602         logic [1:0] ecc_error;
603         prim_secdec_inv_39_32_dec u_dec (
604             .data_i (sram_rdata_scr),
605             .data_o (dec_data),
606             .syndrome_o (),
607             .err_o (ecc_error)
608         );
609
610         logic [DataWidth-1:0] ecc_enc_data;
611         prim_secdec_inv_39_32_enc u_enc (
612             .data_i(dec_data),
613             .data_o(ecc_enc_data)
614         );
615
616         logic uncorrectable_error_q;
617         prim_flop #(
618             .Width(1)
619         ) u_flop_uncorr_error (
620             .clk_i,
621             .rst_ni,
622             .d_i(ecc_error[1]),
623             .q_o(uncorrectable_error_q)
624         );

```

```

623
624 // Correctable errors are corrected.
625 // Uncorrectable errors are passed through to the re-
quester.
626 assign sram_error[0] = 1'b0;
627 assign sram_error[1] = uncorrectable_error_q;
628
629 // Error log if any error happened
630 assign sram_error_o.valid = sram_rvalid_scr & |
ecc_error;
631 assign sram_error_o.correctable = sram_rvalid_scr &
~ecc_error[1];
632
633 // Translate word address to byte address and fill remain-
ing bits with 0
634 always_comb begin
635     sram_error_o.address = '0;
636     sram_error_o.address[2+:AddrWidth] = sram_rer-
ror_addr_scr;
637 end
638
639 prim_flop #(
640     .Width(1)
641 ) u_flop_rvalid (
642     .clk_i,
643     .rst_ni,
644     .d_i(sram_rvalid_scr),
645     .q_o(sram_rvalid)
646 );
647
648 prim_flop #(
649     .Width(DataWidth)
650 ) u_flop_enc_data (
651     .clk_i,
652     .rst_ni,
653     .d_i(ecc_enc_data),
654     .q_o(sram_rdata)
655 );
656
657 // We don't use the read error from the prim, we re-com-
pute it here
658 logic unused_error;
659 assign unused_error = ^sram_error_scr;
660 end else begin : gen_no_ecc_correction
661     assign sram_rdata = sram_rdata_scr;
662     assign sram_error = sram_error_scr;
663     assign sram_rvalid = sram_rvalid_scr;
664     // ECC errors are not detected (and thus not reported
either) in this configuration.
665     assign sram_error_o = '0;
666
667     // Error address not used here
668     logic unused_error_addr;
669     assign unused_error_addr = ^sram_error_addr_scr;
670 end
671
672 // The SRAM scrambling wrapper will not accept any transac-
tions while the
673 // key req is pending or if we have escalated. Note that
we're not using
674 // the scr_key_valid CSR here, such that the SRAM can be
used right after
675 // reset, where the keys are reset to the default netlist
constant.
676 //
677 // If we have escalated, but there is a pending request in
the TL gate, we may have a pending
678 // read-modify-write transaction or readback in the SRAM ad-
apter. In that case we force key_valid
679 // high to enable that to complete so it returns a response,
the TL gate won't accept any new
680 // transactions and the SRAM keys have been clobbered
already.
681 assign key_valid =
682     (key_req_pending_q) ? 1'b0 :
683     (reg2hw.status.escalated.q) ? (tl_gate_resp_pending &
sram_compound_txn_in_progress) : 1'b1;
684
685 // SEC_CM: MEM.SCRAMBLE, ADDR.SCRAMBLE, PRIM_RAM.CTRL.MUBI
686 prim_ram_1p_scr #(
687     .Width(DataWidth),
688     .Depth(Depth),
689     .InstDepth(InstDepth),
690     .EnableParity(0),
691     .DataBitsPerMask(DataWidth),
692     .NumPrinceRoundsHalf(NumPrinceRoundsHalf),
693     .NumAddrScrRounds(NumAddrScrRounds)
694 ) u_prim_ram_1p_scr (
695     .clk_i,
696     .rst_ni,
697
698     .key_valid_i (key_valid),
699     .key_i (key_q),
700     .nonce_i (nonce_q[NonceWidth-1:0]),
701
702     .req_i (sram_req),
703     .intg_error_i (sram_intg_error),
704     .gnt_o (sram_gnt),
705     .write_i (sram_we),
706     .addr_i (sram_addr),

```

```

707     .wdata_i (sram_wdata),
708     .wmask_i (sram_wmask),
709     .rdata_o (sram_rdata_scr),
710     .rvalid_o (sram_rvalid_scr),
711     .error_o (sram_error_scr),
712     .raddr_o (sram_error_addr_scr),
713     .cfg_i (ram_cfg_i),
714     .cfg_o (ram_cfg_o),
715     .wr_collision_o (sram_wr_collision),
716     .write_pending_o (sram_wpending),
717     .alert_o (sram_alert)
718 );
719
720 logic unused_sram_gnt;
721 // Ignore sram_gnt signal to avoid creating a bad timing
path, see comment on 'tlul_gnt' above for
722 // more details.
723 assign unused_sram_gnt = sram_gnt;
724
725 // Assertions //
726 // Assertions //
727 // Assertions //
728
729 `ASSERT_KNOWN(RegsTlOutKnown_A, regs_tl_o)
730 `ASSERT_KNOWN(RamTlOutKnown_A, ram_tl_o.d_valid)
731 `ASSERT_KNOWN_IF(RamTlOutPayloadKnown_A, ram_tl_o,
ram_tl_o.d_valid)
732 `ASSERT_KNOWN(AlertOutKnown_A, alert_tx_o)
733 `ASSERT_KNOWN(SramOtpKeyKnown_A, sram_otp_key_o)
734 `ASSERT_KNOWN(RaclErrorValidKnown_A, racl_error_o.valid)
735 `ASSERT_KNOWN(SramErrorKnown_A, sram_error_o)
736
737 // Alert assertions for redundant counters.
738 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CntCheck_A,
u_prim_count, alert_tx_o[0])
739 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(LcGateFsmCheck_A,
u_tlul_lc_gate.u_state_regs, alert_tx_o[0])
740
741 // Alert assertions for reg_we onehot check.
742 `AS-
743 SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg_regs, alert_tx_o[0])
744
745 // Alert assertions for redundant counters.
746 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RspFifoWptrCheck_A,
u_tlul_adapter_sram_racl.tlul_adapter_sram.u_rspfifo.gen_normal_fifo
.u_fifo_cnt
747     .gen_secure_ptrs.u_wptr,
748     alert_tx_o[0])
749 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(RspFifoRptrCheck_A,
u_tlul_adapter_sram_racl.tlul_adapter_sram.u_rspfifo.gen_normal_fifo
.u_fifo_cnt
750     .gen_secure_ptrs.u_rptr,
751     alert_tx_o[0])
752 `AS-
753 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SramReqFifoWptrCheck_A,
u_tlul_adapter_sram_racl.tlul_adapter_sram.u_sramreqfifo.gen_normal_
fifo.u_fifo_cnt
754     .gen_secure_ptrs.u_wptr,
755     alert_tx_o[0])
756 `AS-
757 SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(SramReqFifoRptrCheck_A,
u_tlul_adapter_sram_racl.tlul_adapter_sram.u_sramreqfifo.gen_normal_
fifo.u_fifo_cnt
758     .gen_secure_ptrs.u_rptr,
759     alert_tx_o[0])
760 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ReqFifoWptrCheck_A,
u_tlul_adapter_sram_racl.tlul_adapter_sram.u_reqfifo.gen_normal_fifo
.u_fifo_cnt
761     .gen_secure_ptrs.u_wptr,
762     alert_tx_o[0])
763 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ReqFifoRptrCheck_A,
u_tlul_adapter_sram_racl.tlul_adapter_sram.u_reqfifo.gen_normal_fifo
.u_fifo_cnt
764     .gen_secure_ptrs.u_rptr,
765     alert_tx_o[0])
766 // `tlul_gnt` doesn't factor in `sram_gnt` for timing reas-
ons. This assertions checks that
767 // `tlul_gnt` is the same as `sram_gnt` when there's an act-
ive `tlul_req` that isn't being ignored
768 // because the SRAM is initializing.
769 `ASSERT(TlulGntIsCorrect_A, tlul_req |-> (sram_gnt &
~init_req) == tlul_gnt)
770
771 `ifdef FI_SIM_Z01X
772
773 // Check if there are any TL-UL integrity errors caused by
faults that Z01X has introduced.
774 // Specific to fault injection simulation as Z01X expects
that those strobing points are
775 // available in the design.
776 wire ram_tl_intg_err;
777 tlul_rsp_intg_chk #(
778     .EnableRspDataIntgCheck(1)

```



```

785 ) u_rsp_chk_ram (
786   .tl_i (ram_tl_o),
787   .err_o(ram_tl_intg_err)
788 );
789
790 wire regs_tl_intg_err;
791 tlul_rsp_intg_chk #(
792   .EnableRspDataIntgCheck(1)
793 ) u_rsp_chk_regs (
794   .tl_i (regs_tl_o),
795   .err_o(regs_tl_intg_err)
796 );
797 `endif
798
799 endmodule : sram_ctrl

```

2. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_pkg.sv (50 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 package sram_ctrl_pkg;
7
8 // Parameters //
9 // Parameters //
10 // Parameters //
11
12 // The width of this RAM is currently restricted to 39
  (32bit data + 7bit integrity).
13 parameter int DataWidth = 32 + tlul_pkg::DataIntgWidth;
14 parameter int NonceWidth = 64;
15
16 // RndCnst //
17 // RndCnst //
18 // RndCnst //
19
20 parameter otp_ctrl_pkg::sram_key_t RndCnstSramKeyDefault =
  128'hbecda03b34bc0418a30a33861a610f71;
21
22 parameter otp_ctrl_pkg::sram_nonce_t RndCnstSramNonceDefault =
  128'h22f296f8f95efb84a75cd435a5541e9f;
23
24 // These LFSR parameters have been generated with
25 // $ ./util/design/gen-lfsr-seed.py --width 64 --seed
  3296833456 --prefix ""
26
27 parameter int LfsrWidth = 64;
28 typedef logic [LfsrWidth-1:0] lfsr_seed_t;
29 typedef logic [LfsrWidth-1:0][$log2(LfsrWidth)-1:0]
  lfsr_perm_t;
30 parameter lfsr_seed_t RndCnstLfsrSeedDefault =
  64'hb496209a_10a81ea5;
31 parameter lfsr_perm_t RndCnstLfsrPermDefault = {
  32   128'hf7963515_f8af8e60_fbfec4c0_f1edd9e2,
  33
  256'h41e1c6d4_273d5046_2da7165d_1c1db882_693146c2_a33aa048_43762bed_
  0ecabea5
  34 };
35
36 // The LFSR has an internal state width of 64 bits but we
  just use the lowest 32 bits of the
37 // permuted state for initializing the RAM.
38 parameter int LfsrOutWidth = 32;
39
40 // Type definitions //
41 // Type definitions //
42 // Type definitions //
43
44 typedef struct packed {
45   logic          valid;
46   logic          correctable;
47   logic [top_pkg::TL_AW-1:0] address;
48 } sram_error_t;
49
50 endpackage : sram_ctrl_pkg

```

3. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_ram_reg_top.sv (54 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module sram_ctrl_ram_reg_top
10 # (
11   parameter bit          EnableRacl          = 1'b0,
12   parameter bit          RclErrorRsp         = 1'b1,
13   parameter top_racl_pkg::racl_policy_sel_t RclPol-
  icySelVec[sram_ctrl_reg_pkg::NumRegsRam] =

```

```

14   '{sram_ctrl_reg_pkg::NumRegsRam{0}}
15 ) (
16   input clk_i,
17   input rst_ni,
18   input tlul_pkg::tl_h2d_t tl_i,
19   output tlul_pkg::tl_d2h_t tl_o,
20   // To HW
21
22   // RAcl interface
23   input top_racl_pkg::racl_policy_vec_t racl_policies_i,
24   output top_racl_pkg::racl_error_log_t  racl_error_o,
25
26   // Integrity check errors
27   output logic intg_err_o
28 );
29
30 import sram_ctrl_reg_pkg::* ;
31
32
33
34 // Since there are no registers in this block, commands are
  routed through to windows which
35 // can report their own integrity errors.
36 assign intg_err_o = 1'b0;
37
38 // outgoing integrity generation
39 tlul_pkg::tl_d2h_t tl_o_pre;
40 tlul_rsp_intg_gen #(
41   .EnableRspIntgGen(1),
42   .EnableDataIntgGen(1)
43 ) u_rsp_intg_gen (
44   .tl_i(tl_o_pre),
45   .tl_o(tl_o)
46 );
47
48 assign tl_reg_h2d = tl_i;
49 assign tl_o_pre   = tl_reg_d2h;
50
51 // Unused signal tieoff
52 logic unused_policy_sel;
53 assign unused_policy_sel = ^racl_policies_i;
54 endmodule

```

4. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_reg_pkg.sv (171 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
  structure
6
7 package sram_ctrl_reg_pkg;
8
9 // Param list
10 parameter int NumAlerts = 1;
11
12 // Address widths within the block
13 parameter int RegsAw = 6;
14
15 // Number of registers for every interface
16 parameter int NumRegsRegs = 9;
17
18 // Alert indices
19 typedef enum int {
20   AlertFatalErrorIdx = 0
21 } sram_ctrl_alert_idx_t;
22
23 // Typedefs for registers for regs interface //
24 // Typedefs for registers for regs interface //
25 // Typedefs for registers for regs interface //
26
27 typedef struct packed {
28   logic q;
29   logic qe;
30 } sram_ctrl_reg2hw_alert_test_reg_t;
31
32 typedef struct packed {
33   struct packed {
34     logic q;
35   } sram_alert;
36   struct packed {
37     logic q;
38   } readback_error;
39   struct packed {
40     logic q;
41   } init_done;
42   struct packed {
43     logic q;
44   } scr_key_seed_valid;
45   struct packed {
46     logic q;
47   } escalated;
48   struct packed {
49     logic q;
50   } init_error;

```



```

51  struct packed {
52      logic q;
53  } bus_integ_error;
54  } sram_ctrl_reg2hw_status_reg_t;
55
56  typedef struct packed {
57      logic [3:0] q;
58  } sram_ctrl_reg2hw_exec_reg_t;
59
60  typedef struct packed {
61      struct packed {
62          logic q;
63          logic qe;
64      } init;
65      struct packed {
66          logic q;
67          logic qe;
68      } renew_scr_key;
69  } sram_ctrl_reg2hw_ctrl_reg_t;
70
71  typedef struct packed {
72      logic [3:0] q;
73  } sram_ctrl_reg2hw_readback_reg_t;
74
75  typedef struct packed {
76      struct packed {
77          logic d;
78          logic de;
79      } sram_alert;
80      struct packed {
81          logic d;
82          logic de;
83      } readback_error;
84      struct packed {
85          logic d;
86          logic de;
87      } init_done;
88      struct packed {
89          logic d;
90          logic de;
91      } scr_key_seed_valid;
92      struct packed {
93          logic d;
94          logic de;
95      } scr_key_valid;
96      struct packed {
97          logic d;
98          logic de;
99      } escalated;
100     struct packed {
101         logic d;
102         logic de;
103     } init_error;
104     struct packed {
105         logic d;
106         logic de;
107     } bus_integ_error;
108 } sram_ctrl_hw2reg_status_reg_t;
109
110 typedef struct packed {
111     logic [3:0] d;
112     logic de;
113 } sram_ctrl_hw2reg_scr_key_rotated_reg_t;
114
115 // Register -> HW type for regs interface
116 typedef struct packed {
117     sram_ctrl_reg2hw_alert_test_reg_t alert_test; // [20:19]
118     sram_ctrl_reg2hw_status_reg_t status; // [18:12]
119     sram_ctrl_reg2hw_exec_reg_t exec; // [11:8]
120     sram_ctrl_reg2hw_ctrl_reg_t ctrl; // [7:4]
121     sram_ctrl_reg2hw_readback_reg_t readback; // [3:0]
122 } sram_ctrl_regs_reg2hw_t;
123
124 // HW -> register type for regs interface
125 typedef struct packed {
126     sram_ctrl_hw2reg_status_reg_t status; // [20:5]
127
128     sram_ctrl_hw2reg_scr_key_rotated_reg_t scr_key_rotated; //
129     [4:0]
130 } sram_ctrl_regs_hw2reg_t;
131
132 // Register offsets for regs interface
133 parameter logic [RegsAw-1:0] SRAM_CTRL_ALERT_TEST_OFFSET =
6'h 0;
134 parameter logic [RegsAw-1:0] SRAM_CTRL_STATUS_OFFSET = 6'h
4;
135 parameter logic [RegsAw-1:0] SRAM_CTRL_EXEC_REGWEN_OFFSET =
6'h 8;
136 parameter logic [RegsAw-1:0] SRAM_CTRL_EXEC_OFFSET = 6'h c;
137 parameter logic [RegsAw-1:0] SRAM_CTRL_CTRL_REGWEN_OFFSET =
6'h 10;
138 parameter logic [RegsAw-1:0] SRAM_CTRL_CTRL_OFFSET = 6'h 14;
139 parameter logic [RegsAw-1:0] SRAM_C-
TRL_SCR_KEY_ROTATED_OFFSET = 6'h 18;
140 parameter logic [RegsAw-1:0] SRAM_C-
TRL_READBACK_REGWEN_OFFSET = 6'h 1c;
141
142 parameter logic [RegsAw-1:0] SRAM_CTRL_READBACK_OFFSET = 6'h
20;
143
144 // Reset values for hwext registers and their fields for

```

```

regs interface
142 parameter logic [0:0] SRAM_CTRL_ALERT_TEST_RESVAL = 1'h 0;
143 parameter logic [0:0] SRAM_C-
TRL_ALERT_TEST_FATAL_ERROR_RESVAL = 1'h 0;
144
145 // Register index for regs interface
146 typedef enum int {
147     SRAM_CTRL_ALERT_TEST,
148     SRAM_CTRL_STATUS,
149     SRAM_CTRL_EXEC_REGWEN,
150     SRAM_CTRL_EXEC,
151     SRAM_CTRL_CTRL_REGWEN,
152     SRAM_CTRL_CTRL,
153     SRAM_CTRL_SCR_KEY_ROTATED,
154     SRAM_CTRL_READBACK_REGWEN,
155     SRAM_CTRL_READBACK
156 } sram_ctrl_regs_id_e;
157
158 // Register width information to check illegal writes for
regs interface
159 parameter logic [3:0] SRAM_CTRL_REGS_PERMIT [9] = '{
160     4'b 0001, // index[0] SRAM_CTRL_ALERT_TEST
161     4'b 0001, // index[1] SRAM_CTRL_STATUS
162     4'b 0001, // index[2] SRAM_CTRL_EXEC_REGWEN
163     4'b 0001, // index[3] SRAM_CTRL_EXEC
164     4'b 0001, // index[4] SRAM_CTRL_CTRL_REGWEN
165     4'b 0001, // index[5] SRAM_CTRL_CTRL
166     4'b 0001, // index[6] SRAM_CTRL_SCR_KEY_ROTATED
167     4'b 0001, // index[7] SRAM_CTRL_READBACK_REGWEN
168     4'b 0001 // index[8] SRAM_CTRL_READBACK
169 };
170
171 endpackage

```

5. lowRISC_opentitan/hw/ip/sram_ctrl/rtl/sram_ctrl_regs_reg_top.sv (859 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // Register Top module auto-generated by `reggen`
7
8 `include "prim_assert.sv"
9
10 module sram_ctrl_regs_reg_top
11 # (
12     parameter bit EnableRacl = 1'b0,
13     parameter bit RclErrorRsp = 1'b1,
14     parameter top_racl_pkg::racl_policy_sel_t RclPol-
15 cySelVec[sram_ctrl_reg_pkg::NumRegsRegs] =
16     '{sram_ctrl_reg_pkg::NumRegsRegs{0}}
17 ) (
18     input clk_i,
19     input rst_ni,
20     input tlul_pkg::tl_h2d_t tl_i,
21     output tlul_pkg::tl_d2h_t tl_o,
22     // To HW
23
24     output sram_ctrl_reg_pkg::sram_ctrl_regs_reg2hw_t reg2hw, //
25     Write
26
27     input sram_ctrl_reg_pkg::sram_ctrl_regs_hw2reg_t hw2reg, //
28     Read
29
30     // RACL interface
31     input top_racl_pkg::racl_policy_vec_t racl_policies_i,
32     output top_racl_pkg::racl_error_log_t racl_error_o,
33
34     // Integrity check errors
35     output logic integ_err_o
36 );
37
38 import sram_ctrl_reg_pkg::* ;
39
40 localparam int AW = 6;
41 localparam int DW = 32;
42 localparam int DBW = DW/8; // Byte Width
43
44 // register signals
45 logic reg_we;
46 logic reg_re;
47 logic [AW-1:0] reg_addr;
48 logic [DW-1:0] reg_wdata;
49 logic [DBW-1:0] reg_be;
50 logic [DW-1:0] reg_rdata;
51 logic reg_error;
52
53 logic addrmiss, wr_err;
54
55 logic [DW-1:0] reg_rdata_next;
56 logic reg_busy;
57
58 tlul_pkg::tl_h2d_t tl_reg_h2d;
59 tlul_pkg::tl_d2h_t tl_reg_d2h;
60
61 // incoming payload check

```

```

57 logic intg_err;
58 tlul_cmd_intg_chk u_chk (
59     .tl_i(tl_i),
60     .err_o(intg_err)
61 );
62
63 // also check for spurious write enables
64 logic reg_we_err;
65 logic [8:0] reg_we_check;
66 prim_reg_we_check #(
67     .OneHotWidth(9)
68 ) u_prim_reg_we_check (
69     .clk_i(clk_i),
70     .rst_ni(rst_ni),
71     .oh_i (reg_we_check),
72     .en_i (reg_we && !addrmiss),
73     .err_o (reg_we_err)
74 );
75
76 logic err_q;
77 always_ff @(posedge clk_i or negedge rst_ni) begin
78     if (!rst_ni) begin
79         err_q <= '0;
80     end else if (intg_err || reg_we_err) begin
81         err_q <= 1'b1;
82     end
83 end
84
85 // integrity error output is permanent and should be used
for alert generation
86 // register errors are transactional
87 assign intg_err_o = err_q | intg_err | reg_we_err;
88
89 // outgoing integrity generation
90 tlul_pkg::tl_d2h_t tl_o_pre;
91 tlul_rsp_intg_gen #(
92     .EnableRspIntgGen(1),
93     .EnableDataIntgGen(1)
94 ) u_rsp_intg_gen (
95     .tl_i(tl_o_pre),
96     .tl_o(tl_o)
97 );
98
99 assign tl_reg_h2d = tl_i;
100 assign tl_o_pre = tl_reg_d2h;
101
102 tlul_adapter_reg #(
103     .RegAw(AW),
104     .RegDw(DW),
105     .EnableDataIntgGen(0)
106 ) u_reg_if (
107     .clk_i (clk_i),
108     .rst_ni (rst_ni),
109
110     .tl_i (tl_reg_h2d),
111     .tl_o (tl_reg_d2h),
112
113     .en_ifetch_i(prim_mubi_pkg::MuBi4False),
114     .intg_error_o(),
115
116     .we_o (reg_we),
117     .re_o (reg_re),
118     .addr_o (reg_addr),
119     .wdata_o (reg_wdata),
120     .be_o (reg_be),
121     .busy_i (reg_busy),
122     .rdata_i (reg_rdata),
123     // Translate RACL error to TLUL error if enabled
124     .error_i (reg_error | (RaclErrorRsp & racl_error_o.valid))
125 );
126
127 // cdc oversampling signals
128
129 assign reg_rdata = reg_rdata_next ;
130 assign reg_error = addrmiss | wr_err | intg_err;
131
132 // Define SW related signals
133 // Format: <reg>_<field>_{wd|we|qs}
134 // or <reg>_{wd|we|qs} if field == 1 or 0
135 logic alert_test_we;
136 logic alert_test_wd;
137 logic status_bus_integ_error_qs;
138 logic status_init_error_qs;
139 logic status_escalated_qs;
140 logic status_scr_key_valid_qs;
141 logic status_scr_key_seed_valid_qs;
142 logic status_init_done_qs;
143 logic status_readback_error_qs;
144 logic status_sram_alert_qs;
145 logic exec_regwen_we;
146 logic exec_regwen_qs;
147 logic exec_regwen_wd;
148 logic exec_we;
149 logic [3:0] exec_qs;
150 logic [3:0] exec_wd;
151 logic ctrl_regwen_we;
152 logic ctrl_regwen_qs;
153 logic ctrl_regwen_wd;
154 logic ctrl_we;
155 logic ctrl_renew_scr_key_wd;
156 logic ctrl_init_wd;

157 logic scr_key_rotated_we;
158 logic [3:0] scr_key_rotated_qs;
159 logic [3:0] scr_key_rotated_wd;
160 logic readback_regwen_we;
161 logic readback_regwen_qs;
162 logic readback_regwen_wd;
163 logic readback_we;
164 logic [3:0] readback_qs;
165 logic [3:0] readback_wd;
166
167 // Register instances
168 // R[alert_test]: V(True)
169 logic alert_test_qe;
170 logic [0:0] alert_test_flds_we;
171 assign alert_test_qe = &alert_test_flds_we;
172 prim_subreg_ext #(
173     .DW (1)
174 ) u_alert_test (
175     .re (1'b0),
176     .we (alert_test_we),
177     .wd (alert_test_wd),
178     .d ('0),
179     .qre (),
180     .qe (alert_test_flds_we[0]),
181     .q (reg2hw.alert_test.q),
182     .ds (),
183     .qs ()
184 );
185 assign reg2hw.alert_test.qe = alert_test_qe;
186
187 // R[status]: V(False)
188 // F[bus_integ_error]: 0:0
189 prim_subreg #(
190     .DW (1),
191     .SwAccess(prim_subreg_pkg::SwAccessR0),
192     .RESVAL (1'h0),
193     .Mubi (1'b0)
194 ) u_status_bus_integ_error (
195     .clk_i (clk_i),
196     .rst_ni (rst_ni),
197
198     // from register interface
199     .we (1'b0),
200     .wd ('0),
201
202     // from internal hardware
203     .de (hw2reg.status.bus_integ_error.de),
204     .d (hw2reg.status.bus_integ_error.d),
205
206     // to internal hardware
207     .qe (),
208     .q (reg2hw.status.bus_integ_error.q),
209     .ds (),
210
211     // to register interface (read)
212     .qs (status_bus_integ_error_qs)
213 );
214
215 // F[init_error]: 1:1
216 prim_subreg #(
217     .DW (1),
218     .SwAccess(prim_subreg_pkg::SwAccessR0),
219     .RESVAL (1'h0),
220     .Mubi (1'b0)
221 ) u_status_init_error (
222     .clk_i (clk_i),
223     .rst_ni (rst_ni),
224
225     // from register interface
226     .we (1'b0),
227     .wd ('0),
228
229     // from internal hardware
230     .de (hw2reg.status.init_error.de),
231     .d (hw2reg.status.init_error.d),
232
233     // to internal hardware
234     .qe (),
235     .q (reg2hw.status.init_error.q),
236     .ds (),
237
238     // to register interface (read)
239     .qs (status_init_error_qs)
240 );
241
242 // F[escalated]: 2:2
243 prim_subreg #(
244     .DW (1),
245     .SwAccess(prim_subreg_pkg::SwAccessR0),
246     .RESVAL (1'h0),
247     .Mubi (1'b0)
248 ) u_status_escalated (
249     .clk_i (clk_i),
250     .rst_ni (rst_ni),
251
252     // from register interface
253     .we (1'b0),
254     .wd ('0),
255
256     // from internal hardware

```

```

258 .de (hw2reg.status.escalated.de),
259 .d (hw2reg.status.escalated.d),
260
261 // to internal hardware
262 .qe (),
263 .q (reg2hw.status.escalated.q),
264 .ds (),
265
266 // to register interface (read)
267 .qs (status_escalated_qs)
268 );
269
270 // F[scr_key_valid]: 3:3
271 prim_subreg #(
272 .DW (1),
273 .SwAccess(prim_subreg_pkg::SwAccessR0),
274 .RESVAL (1'h0),
275 .Mubi (1'b0)
276 ) u_status_scr_key_valid (
277 .clk_i (clk_i),
278 .rst_ni (rst_ni),
279
280 // from register interface
281 .we (1'b0),
282 .wd ('0'),
283
284 // from internal hardware
285 .de (hw2reg.status.scr_key_valid.de),
286 .d (hw2reg.status.scr_key_valid.d),
287
288 // to internal hardware
289 .qe (),
290 .q (),
291 .ds (),
292
293 // to register interface (read)
294 .qs (status_scr_key_valid_qs)
295 );
296
297 // F[scr_key_seed_valid]: 4:4
298 prim_subreg #(
299 .DW (1),
300 .SwAccess(prim_subreg_pkg::SwAccessR0),
301 .RESVAL (1'h0),
302 .Mubi (1'b0)
303 ) u_status_scr_key_seed_valid (
304 .clk_i (clk_i),
305 .rst_ni (rst_ni),
306
307 // from register interface
308 .we (1'b0),
309 .wd ('0'),
310
311 // from internal hardware
312 .de (hw2reg.status.scr_key_seed_valid.de),
313 .d (hw2reg.status.scr_key_seed_valid.d),
314
315 // to internal hardware
316 .qe (),
317 .q (reg2hw.status.scr_key_seed_valid.q),
318 .ds (),
319
320 // to register interface (read)
321 .qs (status_scr_key_seed_valid_qs)
322 );
323
324 // F[init_done]: 5:5
325 prim_subreg #(
326 .DW (1),
327 .SwAccess(prim_subreg_pkg::SwAccessR0),
328 .RESVAL (1'h0),
329 .Mubi (1'b0)
330 ) u_status_init_done (
331 .clk_i (clk_i),
332 .rst_ni (rst_ni),
333
334 // from register interface
335 .we (1'b0),
336 .wd ('0'),
337
338 // from internal hardware
339 .de (hw2reg.status.init_done.de),
340 .d (hw2reg.status.init_done.d),
341
342 // to internal hardware
343 .qe (),
344 .q (reg2hw.status.init_done.q),
345 .ds (),
346
347 // to register interface (read)
348 .qs (status_init_done_qs)
349 );
350
351 // F[readback_error]: 6:6
352 prim_subreg #(
353 .DW (1),
354 .SwAccess(prim_subreg_pkg::SwAccessR0),
355 .RESVAL (1'h0),
356 .Mubi (1'b0)
357 ) u_status_readback_error (
358 .clk_i (clk_i),
359
360 .rst_ni (rst_ni),
361
362 // from register interface
363 .we (1'b0),
364 .wd ('0'),
365
366 // from internal hardware
367 .de (hw2reg.status.readback_error.de),
368 .d (hw2reg.status.readback_error.d),
369
370 // to internal hardware
371 .qe (),
372 .q (reg2hw.status.readback_error.q),
373 .ds (),
374
375 // to register interface (read)
376 .qs (status_readback_error_qs)
377 );
378
379 // F[sram_alert]: 7:7
380 prim_subreg #(
381 .DW (1),
382 .SwAccess(prim_subreg_pkg::SwAccessR0),
383 .RESVAL (1'h0),
384 .Mubi (1'b0)
385 ) u_status_sram_alert (
386 .clk_i (clk_i),
387 .rst_ni (rst_ni),
388
389 // from register interface
390 .we (1'b0),
391 .wd ('0'),
392
393 // from internal hardware
394 .de (hw2reg.status.sram_alert.de),
395 .d (hw2reg.status.sram_alert.d),
396
397 // to internal hardware
398 .qe (),
399 .q (reg2hw.status.sram_alert.q),
400 .ds (),
401
402 // to register interface (read)
403 .qs (status_sram_alert_qs)
404 );
405
406 // R[exec_regwen]: V(False)
407 prim_subreg #(
408 .DW (1),
409 .SwAccess(prim_subreg_pkg::SwAccessW0C),
410 .RESVAL (1'h1),
411 .Mubi (1'b0)
412 ) u_exec_regwen (
413 .clk_i (clk_i),
414 .rst_ni (rst_ni),
415
416 // from register interface
417 .we (exec_regwen_we),
418 .wd (exec_regwen_wd),
419
420 // from internal hardware
421 .de (1'b0),
422 .d ('0'),
423
424 // to internal hardware
425 .qe (),
426 .q (),
427 .ds (),
428
429 // to register interface (read)
430 .qs (exec_regwen_qs)
431 );
432
433 // R[exec]: V(False)
434 // Create REGWEN-gated WE signal
435 logic exec_gated_we;
436 assign exec_gated_we = exec_we & exec_regwen_qs;
437 prim_subreg #(
438 .DW (4),
439 .SwAccess(prim_subreg_pkg::SwAccessRW),
440 .RESVAL (4'h9),
441 .Mubi (1'b1)
442 ) u_exec (
443 .clk_i (clk_i),
444 .rst_ni (rst_ni),
445
446 // from register interface
447 .we (exec_gated_we),
448 .wd (exec_wd),
449
450 // from internal hardware
451 .de (1'b0),
452 .d ('0'),
453
454 // to internal hardware
455 .qe (),
456 .q (reg2hw.exec.q),
457 .ds (),
458
459

```

```

460 // to register interface (read)
461 .qs (exec_qs)
462 );
463
464 // R[ctrl_regwen]: V(False)
465 prim_subreg #(
466 .DW (1),
467 .SwAccess(prim_subreg_pkg::SwAccessW0C),
468 .RESVAL (1'h1),
469 .Mubi (1'b0)
470 ) u_ctrl_regwen (
471 .clk_i (clk_i),
472 .rst_ni (rst_ni),
473
474 // from register interface
475 .we (ctrl_regwen_we),
476 .wd (ctrl_regwen_wd),
477
478 // from internal hardware
479 .de (1'b0),
480 .d ('0),
481
482 // to internal hardware
483 .qe (),
484 .q (),
485 .ds (),
486
487 // to register interface (read)
488 .qs (ctrl_regwen_qs)
489 );
490
491 // R[ctrl]: V(False)
492 logic ctrl_qe;
493 logic [1:0] ctrl_flds_we;
494 prim_flop #(
495 .Width(1),
496 .ResetValue(0)
497 ) u_ctrl0_qe (
498 .clk_i(clk_i),
499 .rst_ni(rst_ni),
500 .d_i(&ctrl_flds_we),
501 .q_o(ctrl_qe)
502 );
503 // Create REGWEN-gated WE signal
504 logic ctrl_gated_we;
505 assign ctrl_gated_we = ctrl_we & ctrl_regwen_qs;
506 // F[renew_scr_key]: 0:0
507 prim_subreg #(
508 .DW (1),
509 .SwAccess(prim_subreg_pkg::SwAccessW0),
510 .RESVAL (1'h0),
511 .Mubi (1'b0)
512 ) u_ctrl_renew_scr_key (
513 .clk_i (clk_i),
514 .rst_ni (rst_ni),
515
516 // from register interface
517 .we (ctrl_gated_we),
518 .wd (ctrl_renew_scr_key_wd),
519
520 // from internal hardware
521 .de (1'b0),
522 .d ('0),
523
524 // to internal hardware
525 .qe (ctrl_flds_we[0]),
526 .q (reg2hw.ctrl.renew_scr_key.q),
527 .ds (),
528
529 // to register interface (read)
530 .qs ()
531 );
532 assign reg2hw.ctrl.renew_scr_key.qe = ctrl_qe;
533
534 // F[init]: 1:1
535 prim_subreg #(
536 .DW (1),
537 .SwAccess(prim_subreg_pkg::SwAccessW0),
538 .RESVAL (1'h0),
539 .Mubi (1'b0)
540 ) u_ctrl_init (
541 .clk_i (clk_i),
542 .rst_ni (rst_ni),
543
544 // from register interface
545 .we (ctrl_gated_we),
546 .wd (ctrl_init_wd),
547
548 // from internal hardware
549 .de (1'b0),
550 .d ('0),
551
552 // to internal hardware
553 .qe (ctrl_flds_we[1]),
554 .q (reg2hw.ctrl.init.q),
555 .ds (),
556
557 // to register interface (read)
558 .qs ()
559 );
560
561 );
562 assign reg2hw.ctrl.init.qe = ctrl_qe;
563
564 // R[scr_key_rotated]: V(False)
565 prim_subreg #(
566 .DW (4),
567 .SwAccess(prim_subreg_pkg::SwAccessW1C),
568 .RESVAL (4'h9),
569 .Mubi (1'b1)
570 ) u_scr_key_rotated (
571 .clk_i (clk_i),
572 .rst_ni (rst_ni),
573
574 // from register interface
575 .we (scr_key_rotated_we),
576 .wd (scr_key_rotated_wd),
577
578 // from internal hardware
579 .de (hw2reg.scr_key_rotated.de),
580 .d (hw2reg.scr_key_rotated.d),
581
582 // to internal hardware
583 .qe (),
584 .q (),
585 .ds (),
586
587 // to register interface (read)
588 .qs (scr_key_rotated_qs)
589 );
590
591 // R[readback_regwen]: V(False)
592 prim_subreg #(
593 .DW (1),
594 .SwAccess(prim_subreg_pkg::SwAccessW0C),
595 .RESVAL (1'h1),
596 .Mubi (1'b0)
597 ) u_readback_regwen (
598 .clk_i (clk_i),
599 .rst_ni (rst_ni),
600
601 // from register interface
602 .we (readback_regwen_we),
603 .wd (readback_regwen_wd),
604
605 // from internal hardware
606 .de (1'b0),
607 .d ('0),
608
609 // to internal hardware
610 .qe (),
611 .q (),
612 .ds (),
613
614 // to register interface (read)
615 .qs (readback_regwen_qs)
616 );
617
618 // R[readback]: V(False)
619 // Create REGWEN-gated WE signal
620 logic readback_gated_we;
621 assign readback_gated_we = readback_we & readback_regwen_qs;
622 prim_subreg #(
623 .DW (4),
624 .SwAccess(prim_subreg_pkg::SwAccessRW),
625 .RESVAL (4'h9),
626 .Mubi (1'b1)
627 ) u_readback (
628 .clk_i (clk_i),
629 .rst_ni (rst_ni),
630
631 // from register interface
632 .we (readback_gated_we),
633 .wd (readback_wd),
634
635 // from internal hardware
636 .de (1'b0),
637 .d ('0),
638
639 // to internal hardware
640 .qe (),
641 .q (reg2hw.readback.q),
642 .ds (),
643
644 // to register interface (read)
645 .qs (readback_qs)
646 );
647
648 logic [8:0] addr_hit;
649 top_racl_pkg::racl_role_vec_t racl_role_vec;
650 top_racl_pkg::racl_role_t racl_role;
651
652 logic [8:0] racl_addr_hit_read;
653 logic [8:0] racl_addr_hit_write;
654
655 if (EnableRacl) begin : gen_racl_role_logic
656 // Retrieve RACL role from user bits and one-hot encode

```

```

that for the comparison bitmap
662   assign rac1_role =
top_rac1_pkg::tlul_extract_rac1_role_bits(tl_i.a_user.rsvd);
663
664   prim_onehot_enc #(
665     .OneHotWidth( $bits(top_rac1_pkg::rac1_role_vec_t) )
666   ) u_rac1_role_encode (
667     .in_i ( rac1_role ),
668     .en_i ( 1'b1 ),
669     .out_o( rac1_role_vec )
670   );
671 end else begin : gen_no_rac1_role_logic
672   assign rac1_role = '0;
673   assign rac1_role_vec = '0;
674 end
675
676 always_comb begin
677   rac1_addr_hit_read = '0;
678   rac1_addr_hit_write = '0;
679   addr_hit[0] = (reg_addr == SRAM_CTRL_ALERT_TEST_OFFSET);
680   addr_hit[1] = (reg_addr == SRAM_CTRL_STATUS_OFFSET);
681   addr_hit[2] = (reg_addr == SRAM_CTRL_EXEC_REGWEN_OFFSET);
682   addr_hit[3] = (reg_addr == SRAM_CTRL_EXEC_OFFSET);
683   addr_hit[4] = (reg_addr == SRAM_CTRL_CTRL_REGWEN_OFFSET);
684   addr_hit[5] = (reg_addr == SRAM_CTRL_CTRL_OFFSET);
685   addr_hit[6] = (reg_addr == SRAM_C-
TRL_SCR_KEY_ROTATED_OFFSET);
686   addr_hit[7] = (reg_addr == SRAM_C-
TRL_READBACK_REGWEN_OFFSET);
687   addr_hit[8] = (reg_addr == SRAM_CTRL_READBACK_OFFSET);
688
689   if (EnableRac1) begin : gen_rac1_hit
690     for (int unsigned slice_idx = 0; slice_idx < 9;
slice_idx++) begin
691       rac1_addr_hit_read[slice_idx] =
692         addr_hit[slice_idx] & (|
(rac1_policies_i[Rac1PolicySelVec[slice_idx]].read_perm
693           & rac1_role_vec));
694       rac1_addr_hit_write[slice_idx] =
695         addr_hit[slice_idx] & (|
(rac1_policies_i[Rac1PolicySelVec[slice_idx]].write_perm
696           & rac1_role_vec));
697     end
698   end else begin : gen_no_rac1
699     rac1_addr_hit_read = addr_hit;
700     rac1_addr_hit_write = addr_hit;
701   end
702 end
703
704 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
705 // A valid address hit, access, but failed the RACL check
706 assign rac1_error_o.valid = |addr_hit & ((reg_re & ~|
rac1_addr_hit_read) |
707   (reg_we & ~|
rac1_addr_hit_write));
708 assign rac1_error_o.request_address =
top_pkg::TL_AW'(reg_addr);
709 assign rac1_error_o.rac1_role = rac1_role;
710 assign rac1_error_o.overflow = 1'b0;
711
712 if (EnableRac1) begin : gen_rac1_log
713   assign rac1_error_o.ctn_uid =
top_rac1_pkg::tlul_extract_ctn_uid_bits(tl_i.a_user.rsvd);
714   assign rac1_error_o.read_access = tl_i.a_opcode ==
tlul_pkg::Get;
715 end else begin : gen_no_rac1_log
716   assign rac1_error_o.ctn_uid = '0;
717   assign rac1_error_o.read_access = 1'b0;
718 end
719
720 // Check sub-word write is permitted
721 always_comb begin
722   wr_err = (reg_we &
723     ((rac1_addr_hit_write[0] & (|(SRAM_C-
TRL_REGS_PERMIT[0] & ~reg_be))) |
724     (rac1_addr_hit_write[1] & (|(SRAM_C-
TRL_REGS_PERMIT[1] & ~reg_be))) |
725     (rac1_addr_hit_write[2] & (|(SRAM_C-
TRL_REGS_PERMIT[2] & ~reg_be))) |
726     (rac1_addr_hit_write[3] & (|(SRAM_C-
TRL_REGS_PERMIT[3] & ~reg_be))) |
727     (rac1_addr_hit_write[4] & (|(SRAM_C-
TRL_REGS_PERMIT[4] & ~reg_be))) |
728     (rac1_addr_hit_write[5] & (|(SRAM_C-
TRL_REGS_PERMIT[5] & ~reg_be))) |
729     (rac1_addr_hit_write[6] & (|(SRAM_C-
TRL_REGS_PERMIT[6] & ~reg_be))) |
730     (rac1_addr_hit_write[7] & (|(SRAM_C-
TRL_REGS_PERMIT[7] & ~reg_be))) |
731     (rac1_addr_hit_write[8] & (|(SRAM_C-
TRL_REGS_PERMIT[8] & ~reg_be)))));
732   end
733
734 // Generate write-enables
735 assign alert_test_we = rac1_addr_hit_write[0] & reg_we & !
reg_error;
736
737 assign alert_test_wd = reg_wdata[0];
738 assign exec_regwen_we = rac1_addr_hit_write[2] & reg_we & !
reg_error;
739

```

```

740 assign exec_regwen_wd = reg_wdata[0];
741 assign exec_we = rac1_addr_hit_write[3] & reg_we & !reg_er-
ror;
742
743 assign exec_wd = reg_wdata[3:0];
744 assign ctrl_regwen_we = rac1_addr_hit_write[4] & reg_we & !
reg_error;
745
746 assign ctrl_regwen_wd = reg_wdata[0];
747 assign ctrl_we = rac1_addr_hit_write[5] & reg_we & !reg_er-
ror;
748
749 assign ctrl_renew_scr_key_wd = reg_wdata[0];
750
751 assign ctrl_init_wd = reg_wdata[1];
752 assign scr_key_rotated_we = rac1_addr_hit_write[6] & reg_we
& !reg_error;
753
754 assign scr_key_rotated_wd = reg_wdata[3:0];
755 assign readback_regwen_we = rac1_addr_hit_write[7] & reg_we
& !reg_error;
756
757 assign readback_regwen_wd = reg_wdata[0];
758 assign readback_we = rac1_addr_hit_write[8] & reg_we & !
reg_error;
759
760 assign readback_wd = reg_wdata[3:0];
761
762 // Assign write-enables to checker logic vector.
763 always_comb begin
764   reg_we_check[0] = alert_test_we;
765   reg_we_check[1] = 1'b0;
766   reg_we_check[2] = exec_regwen_we;
767   reg_we_check[3] = exec_gated_we;
768   reg_we_check[4] = ctrl_regwen_we;
769   reg_we_check[5] = ctrl_gated_we;
770   reg_we_check[6] = scr_key_rotated_we;
771   reg_we_check[7] = readback_regwen_we;
772   reg_we_check[8] = readback_gated_we;
773 end
774
775 // Read data return
776 always_comb begin
777   reg_rdata_next = '0;
778   unique case (1'b1)
779     rac1_addr_hit_read[0]: begin
780       reg_rdata_next[0] = '0;
781     end
782
783     rac1_addr_hit_read[1]: begin
784       reg_rdata_next[0] = status_bus_integ_error_qs;
785       reg_rdata_next[1] = status_init_error_qs;
786       reg_rdata_next[2] = status_escalated_qs;
787       reg_rdata_next[3] = status_scr_key_valid_qs;
788       reg_rdata_next[4] = status_scr_key_seed_valid_qs;
789       reg_rdata_next[5] = status_init_done_qs;
790       reg_rdata_next[6] = status_readback_error_qs;
791       reg_rdata_next[7] = status_sram_alert_qs;
792     end
793
794     rac1_addr_hit_read[2]: begin
795       reg_rdata_next[0] = exec_regwen_qs;
796     end
797
798     rac1_addr_hit_read[3]: begin
799       reg_rdata_next[3:0] = exec_qs;
800     end
801
802     rac1_addr_hit_read[4]: begin
803       reg_rdata_next[0] = ctrl_regwen_qs;
804     end
805
806     rac1_addr_hit_read[5]: begin
807       reg_rdata_next[0] = '0;
808       reg_rdata_next[1] = '0;
809     end
810
811     rac1_addr_hit_read[6]: begin
812       reg_rdata_next[3:0] = scr_key_rotated_qs;
813     end
814
815     rac1_addr_hit_read[7]: begin
816       reg_rdata_next[0] = readback_regwen_qs;
817     end
818
819     rac1_addr_hit_read[8]: begin
820       reg_rdata_next[3:0] = readback_qs;
821     end
822
823     default: begin
824       reg_rdata_next = '1;
825     end
826   endcase
827 end
828
829 // shadow busy
830 logic shadow_busy;
831 assign shadow_busy = 1'b0;
832
833 // register busy
834 assign reg_busy = shadow_busy;

```

```

835
836 // Unused signal tieoff
837
838 // wdata / byte enable are not always fully used
839 // add a blanket unused statement to handle lint waivers
840 logic unused_wdata;
841 logic unused_be;
842 assign unused_wdata = ^reg_wdata;
843 assign unused_be = ^reg_be;
844 logic unused_policy_sel;
845 assign unused_policy_sel = ^racl_policies_i;
846
847 // Assertions for Register Interface
848 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
849 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)

```

```

850
851 `ASSERT(reAfterRv, $rose(reg_re || reg_we) |>
tl_o_pre.d_valid, clk_i, !rst_ni)
852
853 `ASSERT(en2addrHit, (reg_we || reg_re) |> $one-
hot0(addr_hit), clk_i, !rst_ni)
854
855 // this is formulated as an assumption such that the FPV
testbenches do disprove this
856 // property by mistake
857 // ASSUME(reqParity, tl_reg_h2d.a_valid |>
tl_reg_h2d.a_user.chk_en == tlul_pkg::CheckDis)
858
859 endmodule

```

OTP 컨트롤러

What. 일회성 프로그래머블 메모리의 컨트롤러.

Why it exists. 지울 수 없는 저장소가 필요하다 — 칩 고유 키, 수명주기 상태, 보정값. 한 번 쓰면 못 되돌리므로 쓰기 절차가 극도로 보수적이다.

학부 복습 · **EECS concepts.** 비휘발성 메모리 물리(eFuse, 안티퓨즈) · 오류정정 · 먹등 쓰기.

석사 수준 · **Practice note.** OTP 관련 버그는 수정 불가능한 칩을 만든다. 이 블록의 검증은 다른 어떤 블록보다 보수적으로 한다.

1 files • 201 lines

API — otp_ctrl_pkg.sv

Table 574. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — module / interface / package (1개)

이름	종류
otp_ctrl_pkg	package

Table 575. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — parameter (15개)

파라미터	형/폭	기본값
DeviceIdWidth	int	256
ManufStateWidth	int	256
ScramblKeyWidth	int	128
ScramblBlockWidth	int	64
OTP_LC_DATA_DEFAULT	otp_lc_data_t	{
NvmKeySeedWidth	int	256
SramKeySeedWidth	int	128
NvmKeyWidth	int	128
SramKeyWidth	int	128
SramNonceWidth	int	128
OtbnKeyWidth	int	128
OtbnNonceWidth	int	64
OTP_KEYMGR_KEY_DEFAULT	otp_keymgr_key_t	{
NVM_OTP_KEY_RSP_DEFAULT	nvm_otp_key_rsp_t	{
SRAM_OTP_KEY_RSP_DEFAULT	sram_otp_key_rsp_t	{

Table 576. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — localparam (3개)

상수	형/폭	값
OtbnNonceSel	int	OtbnNonceWidth / ScramblBlockWidth
NvmNonceSel	int	NvmKeyWidth / ScramblBlockWidth
SramNonceSel	int	SramNonceWidth / ScramblBlockWidth

Table 577. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv — typedef / using (6개)

새 이름	원래 형
otp_device_id_t	logic [DeviceIdWidth-1:0]
otp_manuf_state_t	logic [ManufStateWidth-1:0]
sram_key_t	logic [SramKeyWidth-1:0]
sram_nonce_t	logic [SramNonceWidth-1:0]
otbn_key_t	logic [OtbnKeyWidth-1:0]
otbn_nonce_t	logic [OtbnNonceWidth-1:0]

1. lowRISC_opentitan/hw/ip/otp_ctrl/rtl/otp_ctrl_pkg.sv (201 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This package can be imported by generic IPs:
6 // - It does not import otp_ctrl_reg_pkg, which is generated
  and top-specific.
7
8 `include "prim_assert.sv"
9
10 package otp_ctrl_pkg;
11
12 ///////////////////////////////////////////////////////////////////
13 // General Parameters //
14 ///////////////////////////////////////////////////////////////////
15
16 parameter int DeviceIdWidth = 256;
17 typedef logic [DeviceIdWidth-1:0] otp_device_id_t;
18 // Device ID defined in the keymgr_dpe package must match
19 `ASSERT_STATIC_IN_PACKAGE(DeviceIdWidth_A,
20                           DeviceIdWidth == keymgr_dpe_pkg::DeviceIdWidth)
21
22 parameter int ManufStateWidth = 256;
23 typedef logic [ManufStateWidth-1:0] otp_manuf_state_t;
24
25 ///////////////////////////////////////////////////////////////////
26 // Typedefs for OTP Scrambling //
27 ///////////////////////////////////////////////////////////////////
28
29 parameter int ScramblKeyWidth = 128;
30 parameter int ScramblBlockWidth = 64;
31
32 ///////////////////////////////////////////////////////////////////
33 // Typedefs for LC Interface //
34 ///////////////////////////////////////////////////////////////////
35
36 // The tokens below are all hash post-images
37 typedef struct packed {
38     logic                                valid;
39     logic                                error;
40     // Use lc_state_t and lc_cnt_t here as very wide enumerations
41     // (> 64 bits)
42     // are not supported for virtual interfaces by Xcelium yet
43     // https://github.com/lowRISC/opentitan/issues/8884 (Cadenance issue: cds_46570160)
44
45     // The enumeration types lc_state_e and lc_cnt_e are still ok
46     // in other circumstances
47     lc_ctrl_state_pkg::lc_state_t        state;
48     lc_ctrl_state_pkg::lc_cnt_t          count;
49     // This is set to "On" if the partition containing the
50     // root secrets have been locked. In that case, the device
51     // is considered "personalized".
52     lc_ctrl_pkg::lc_tx_t                  secrets_valid;
53     // This is set to "On" if the partition containing the
54     // test tokens has been locked.
55     lc_ctrl_pkg::lc_tx_t                  test_tokens_valid;
56
57     lc_ctrl_state_pkg::lc_token_t        test_unlock_token;
58     lc_ctrl_state_pkg::lc_token_t        test_exit_token;
59     // This is set to "On" if the partition containing the
60     // rma token has been locked.
61     lc_ctrl_pkg::lc_tx_t                  rma_token_valid;
62     lc_ctrl_state_pkg::lc_token_t        rma_token;
63 } otp_lc_data_t;
64
65 // Default for dangling connection.
66 // Note that we put the life cycle into
67 // TEST_UNLOCKED0 by default such that top levels without
68 // the OTP controller can still function.
69 parameter otp_lc_data_t OTP_LC_DATA_DEFAULT = '{
70     valid: 1'b1,
71     error: 1'b0,
72     state: lc_ctrl_state_pkg::LcStTestUnlocked0,
73     count: lc_ctrl_state_pkg::LcCnt1,
74     secrets_valid: lc_ctrl_pkg::Off,
75     test_tokens_valid: lc_ctrl_pkg::Off,
76     test_unlock_token: '0,
77     test_exit_token: '0,
78     rma_token_valid: lc_ctrl_pkg::Off,
79     rma_token: '0
80 };
81
82 typedef struct packed {
83     logic req;
84     lc_ctrl_state_pkg::lc_state_e state;
85     lc_ctrl_state_pkg::lc_cnt_e count;
86 } lc_otp_program_req_t;
87
88 typedef struct packed {
89     logic err;
90     logic ack;
91 } lc_otp_program_rsp_t;
92
93 ///////////////////////////////////////////////////////////////////
94 // Typedefs for Key Broadcast //
95 ///////////////////////////////////////////////////////////////////
96
97 parameter int NvmKeySeedWidth = 256;
98 parameter int SramKeySeedWidth = 128;
99 parameter int NvmKeyWidth = 128;
100 parameter int SramKeyWidth = 128;
101 parameter int SramNonceWidth = 128;
102 parameter int OtbnKeyWidth = 128;
103 parameter int OtbnNonceWidth = 64;
104
105 typedef logic [SramKeyWidth-1:0] sram_key_t;
106 typedef logic [SramNonceWidth-1:0] sram_nonce_t;
107 typedef logic [OtbnKeyWidth-1:0] otbn_key_t;
108 typedef logic [OtbnNonceWidth-1:0] otbn_nonce_t;
109
110 localparam int OtbnNonceSel = OtbnNonceWidth / ScramblBlockWidth;
111 localparam int NvmNonceSel = NvmKeyWidth / ScramblBlockWidth;
112 localparam int SramNonceSel = SramNonceWidth / ScramblBlockWidth;
113
114 typedef struct packed {
115     logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] creat-
116         or_root_key_share0;
117     logic                                         creat-
118         or_root_key_share0_valid;

```

```

113 logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] creat-
or_root_key_share1;
114 logic creat-
or_root_key_share1_valid;
115 logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] creator_seed;
116 logic creat-
or_seed_valid;
117 logic [keymgr_dpe_pkg::KeyMgrKeyWidth-1:0] owner_seed;
118 logic own-
er_seed_valid;
119 } otp_keymgr_key_t;
120
121 parameter otp_keymgr_key_t OTP_KEYMGR_KEY_DEFAULT = '{
122 creator_root_key_share0: 256'hfeb7ea7ee90093cf4aff-
d9aaa2d6c0ec446cfd5f2d5a0bfd7e2d93edc63a102,
123 creator_root_key_share0_valid: 1'b1,
124 creator_root_key_share1:
256'h56d24a00181de99e0f690b447a8dde2a1ffb8b-
c306707107aa6e2410f15cfc37,
125 creator_root_key_share1_valid: 1'b1,
126 creator_seed: 256'hc7c50b38655cc87f821e5b07fed-
85d2c07e222a9e00bef308b3eccba0ba406fa,
127 creator_seed_valid: 1'b1,
128 owner_seed:
256'hf5052c0f14782d8b066be9f49c0b2000d3643ff3723ea7d-
b972f69cd3e2e3e68,
129 owner_seed_valid: 1'b1
130 };
131
132 typedef struct packed {
133 logic data_req; // Requests static key for data scram-
bling.
134 logic addr_req; // Requests static key for address scram-
bling.
135 } nvm_otp_key_req_t;
136
137 typedef struct packed {
138 logic req; // Requests ephemeral scrambling key and nonce.
139 } sram_otp_key_req_t;
140
141 typedef struct packed {
142 logic req; // Requests ephemeral scrambling key and nonce.
143 } otbn_otp_key_req_t;
144
145 typedef struct packed {
146 logic data_ack; // Ack for data key.
147 logic addr_ack; // Ack for address key.
148 logic [NvmKeyWidth-1:0] key; // 128bit static scram-
bling key.
149 logic [NvmKeyWidth-1:0] rand_key;
150 logic seed_valid; // Set to 1 if the key
seed has been provisioned and is
151 // valid.
152 } nvm_otp_key_rsp_t;

```

```

153
154 // Default for dangling connection
155 parameter nvm_otp_key_rsp_t NVM_OTP_KEY_RSP_DEFAULT = '{
156 data_ack: 1'b1,
157 addr_ack: 1'b1,
158 key: '0,
159 rand_key: '0,
160 seed_valid: 1'b1
161 };
162
163 typedef struct packed {
164 logic ack; // Ack for key.
165 sram_key_t key; // 128bit ephemeral scrambling
key.
166 sram_nonce_t nonce; // 128bit nonce.
167 logic seed_valid; // Set to 1 if the key seed has
been provisioned and is valid.
168 } sram_otp_key_rsp_t;
169
170 // Default for dangling connection
171 parameter sram_otp_key_rsp_t SRAM_OTP_KEY_RSP_DEFAULT = '{
172 ack: 1'b1,
173 key: '0,
174 nonce: '0,
175 seed_valid: 1'b1
176 };
177
178 typedef struct packed {
179 logic ack; // Ack for key.
180 otbn_key_t key; // 128bit ephemeral scrambling
key.
181 otbn_nonce_t nonce; // 256bit nonce.
182 logic seed_valid; // Set to 1 if the key seed has
been provisioned and is valid.
183 } otbn_otp_key_rsp_t;
184
185 ///////////////////////////////////////////////////
186 // Power/Reset Ctrl Interface //
187 ///////////////////////////////////////////////////
188
189 typedef struct packed {
190 logic init;
191 } pwr_otp_init_req_t;
192
193 typedef struct packed {
194 logic done;
195 } pwr_otp_init_rsp_t;
196
197 typedef struct packed {
198 logic idle;
199 } otp_pwr_state_t;
200
201 endpackage : otp_ctrl_pkg

```

키 관리자

What. 루트 키에서 용도별 파생 키를 만들고 수명주기에 따라 접근을 통제하는 블록.

Why it exists. 키를 소프트웨어에 절대 노출하지 않기 위해 서다. 소프트웨어는 “이 용도의 키로 서명해 달라”고 요청할 뿐, 키 값을 못 본다. 키는 하드웨어 경로로만 암호 엔진에 전달된다.

학부 복습 · **EECS concepts.** 키 파생 함수(KDF) · 신뢰의 사슬(chain of trust) · 최소 권한 원칙 · 하드웨어 격리.

석사 수준 · **Practice note.** ‘하드웨어 키 경로’가 실제로 격리됐는지는 **RTL** 을 읽어야 한다. 디버그 경로, 스캔 체인, 테스트 모드로 새는 경로가 없는지가 보안 감사의 핵심 항목이다. 스캔 체인은 모든 플립플롭을 읽을 수 있다 — 그래서 양산 칩은 스캔을 잠근다.

10 files • 3,528 lines

API — keymgr .sv

Table 578. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr .sv — module / interface / package (1개)

이름	종류
keymgr	module

Table 579. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr .sv — parameter (18개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
UseOtpSeedsInsteadOfFlash	bit	1'b0
KmacEnMasking	bit	1'b1
RndCnstLfsrSeed	lfsr_seed_t	RndCnstLfsrSeedDefault
RndCnstLfsrPerm	lfsr_perm_t	RndCnstLfsrPermDefault
RndCnstRandPerm	rand_perm_t	RndCnstRandPermDefault
RndCnstRevisionSeed	seed_t	RndCnstRevisionSeedDefault
RndCnstCreatorIdentitySeed	seed_t	RndCnstCreatorIdentitySeedDefault
RndCnstOwnerIntIdentitySeed	seed_t	RndCnstOwnerIntIdentitySeedDefault
RndCnstOwnerIdentitySeed	seed_t	RndCnstOwnerIdentitySeedDefault
RndCnstSoftOutputSeed	seed_t	RndCnstSoftOutputSeedDefault
RndCnstHardOutputSeed	seed_t	RndCnstHardOutputSeedDefault
RndCnstNoneSeed	seed_t	RndCnstNoneSeedDefault
RndCnstAesSeed	seed_t	RndCnstAesSeedDefault
RndCnstOtbnSeed	seed_t	RndCnstOtbnSeedDefault
RndCnstKmacSeed	seed_t	RndCnstKmacSeedDefault

파라미터	형/폭	기본값
RndCnstCdi	seed_t	RndCnstCdiDefault

Table 580. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv — localparam (6개)

상수	형/폭	값
NumRomDigestInputs	int unsigned	1
TotalSeedWidth	int	KeyWidth * 10
RndConstSeed	logic [TotalSeedWidth-1:0]	{RndCnstRevisionSeed
AdvLfsrcopies	int	AdvDataWidth / 32
IdLfsrcopies	int	IdDataWidth / 32
GenLfsrcopies	int	GenDataWidth / 32

Table 581. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
rst_shadowed_ni	input	
clk_edn_i	input	
rst_edn_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
aes_key_o	output	hw_key_req_t
kmac_key_o	output	hw_key_req_t
otbn_key_o	output	otbn_key_req_t
kmac_data_o	output	kmac_pkg::app_req_t
kmac_data_i	input	kmac_pkg::app_rsp_t
kmac_en_masking_i	input	
lc_keymgr_en_i	input	lc_ctrl_pkg::lc_tx_t
lc_keymgr_div_i	input	lc_ctrl_pkg::lc_keymgr_div_t
otp_key_i	input	otp_ctrl_pkg::otp_keymgr_key_t
otp_device_id_i	input	otp_ctrl_pkg::otp_device_id_t
flash_i	input	flash_ctrl_pkg::keymgr_flash_t
edn_o	output	edn_pkg::edn_req_t
edn_i	input	edn_pkg::edn_rsp_t
rom_digest_i	input	rom_ctrl_pkg::keymgr_data_t
intr_op_done_o	output	logic
alert_rx_i	input	prim_alert_pkg::alert_rx_t [keymgr_reg_pkg::NumAle...

포트	방향	형/폭
alert_tx_o	output	prim_alert_pkg::alert_tx_t[keymgr_reg_pkg::NumAle...

API — keymgr_cfg_en.sv

Table 582. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv — module / interface / package (1개)

이름	종류
keymgr_cfg_en	module

Table 583. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv — parameter (1개)

파라미터	형/폭	기본값
NonInitClr	bit	1'b1

Table 584. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
init_i	input	
en_i	input	
set_i	input	
clr_i	input	
out_o	output	logic

API — keymgr_ctrl.sv

Table 585. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — module / interface / package (1개)

이름	종류
keymgr_ctrl	module

Table 586. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — parameter (1개)

파라미터	형/폭	기본값
KmacEnMasking	bit	1'b1

Table 587. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — localparam (9개)

상수	형/폭	값
EntropyWidth	int	LfsrWidth / 2
EntropyRounds	int	KeyWidth / EntropyWidth
EntropyRndWidth	int	prim_util_pkg::vbits(EntropyRounds)
CntWidth	int	EntropyRounds > CDIs ? EntropyRndWidth : CdiWidth
EccDataWidth	int	64
EccWidth	int	8

상수	형/폭	값
EccWords	int	KeyWidth / EccDataWidth
TotalEccWords	int	EccWords * Shares * CDIs
StateWidth	int	10

Table 588. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv — 포트 (44개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
en_i	input	
regfile_intg_err_i	input	
shadowed_update_err_i	input	
shadowed_storage_err_i	input	
reseed_cnt_err_i	input	
sideload_sel_err_i	input	
sideload_fsm_err_i	input	
op_start_i	input	
op_i	input	keymgr_ops_e
op_cdi_sel_i	input	[CdiWidth-1:0]
op_done_o	output	logic
status_o	output	keymgr_op_status_e
error_o	output	logic [ErrLastPos-1:0]
fault_o	output	logic [FaultLastPos-1:0]
data_hw_en_o	output	logic
data_sw_en_o	output	logic
data_valid_o	output	logic
wipe_key_o	output	logic
working_state_o	output	keymgr_working_state_e
sw_binding_unlock_o	output	logic
init_o	output	logic
root_key_i	input	otp_ctrl_pkg::otp_keymgr_key_t
hw_sel_o	output	prim_mubi_pkg::mubi4_t
stage_sel_o	output	keymgr_stage_e
invalid_stage_sel_o	output	logic
cdi_sel_o	output	logic [CdiWidth-1:0]
adv_en_o	output	logic

포트	방향	형/폭
id_en_o	output	logic
gen_en_o	output	logic
key_o	output	hw_key_req_t
kmac_done_i	input	
kmac_input_invalid_i	input	
kmac_fsm_err_i	input	
kmac_op_err_i	input	
kmac_done_err_i	input	
kmac_cmd_err_i	input	
kmac_data_i	input	[Shares-1:0][KeyWidth-1:0]
entropy_i	input	[Shares-1:0][RandWidth-1:0]
prng_reseed_done_i	input	
prng_reseed_ack_i	input	
prng_reseed_req_o	output	logic
prng_en_o	output	logic

API — keymgr_data_en_state.sv

Table 589. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv — module / interface / package (1개)

이름	종류
keymgr_data_en_state	module

Table 590. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv — localparam (1개)

상수	형/폭	값
DataStateWidth	int	10

Table 591. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
hw_sel_i	input	prim_mubi_pkg::mubi4_t
adv_en_i	input	
id_en_i	input	
gen_en_i	input	
op_done_i	input	
op_start_i	input	
data_hw_en_o	output	logic

포트	방향	형/폭
data_sw_en_o	output	logic
fsm_err_o	output	logic

API — keymgr_err.sv

Table 592. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_err.sv — module / interface / package (1개)

이름	종류
keymgr_err	module

Table 593. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_err.sv — 포트 (32개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
invalid_op_i	input	
disabled_i	input	
invalid_i	input	
kmac_input_invalid_i	input	
shadowed_update_err_i	input	
kmac_op_err_i	input	
invalid_kmac_out_i	input	
sideload_sel_err_i	input	
kmac_cmd_err_i	input	
kmac_fsm_err_i	input	
kmac_done_err_i	input	
regfile_intg_err_i	input	
shadowed_storage_err_i	input	
ctrl_fsm_err_i	input	
data_fsm_err_i	input	
op_fsm_err_i	input	
ecc_err_i	input	
state_change_err_i	input	
op_state_cmd_err_i	input	
cnt_err_i	input	
reseed_cnt_err_i	input	
sideload_fsm_err_i	input	
op_update_i	input	
op_done_i	input	

포트	방향	형/폭
sync_err_o	output	logic [SyncErrLastIdx-1:0]
async_err_o	output	logic [AsyncErrLastIdx-1:0]
sync_fault_o	output	logic [SyncFaultLastIdx-1:0]
async_fault_o	output	logic [AsyncFaultLastIdx-1:0]
error_o	output	logic [ErrLastPos-1:0]
fault_o	output	logic [FaultLastPos-1:0]

API — keymgr_input_checks.sv

Table 594. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — module / interface / package (1개)

이름	종류
keymgr_input_checks	module

Table 595. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — parameter (2개)

파라미터	형/폭	기본값
KmacEnMasking	bit	1'b1
NumRomDigestInputs	int unsigned	1

Table 596. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — localparam (1개)

상수	형/폭	값
KeyShares	int	KmacEnMasking ? Shares : 1

Table 597. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv — 포트 (15개)

포트	방향	형/폭
rom_digest_i	input	rom_ctrl_pkg::keymgr_data_t [NumRomDigestInputs-1:...
cur_max_key_version_i	input	[KeyVersionWidth-1:0]
key_i	input	hw_key_req_t
key_version_i	input	[31:0]
creator_seed_i	input	[KeyWidth-1:0]
owner_seed_i	input	[KeyWidth-1:0]
devid_i	input	[DevidWidth-1:0]
health_state_i	input	[HealthStateWidth-1:0]
creator_seed_vld_o	output	logic
owner_seed_vld_o	output	logic
devid_vld_o	output	logic
health_state_vld_o	output	logic
key_version_vld_o	output	logic

포트	방향	형/폭
key_vld_o	output	logic
rom_digest_vld_o	output	logic

API — keymgr_kmac_if.sv

Table 598. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — module / interface / package (1개)

이름	종류
keymgr_kmac_if	module

Table 599. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — parameter (2개)

파라미터	형/폭	기본값
RndCnstRandPerm	rand_perm_t	RndCnstRandPermDefault
MaxAdvDataWidth	int	AdvDataWidth

Table 600. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — localparam (21개)

상수	형/폭	값
StateWidth	int	10
AdvRem	int	MaxAdvDataWidth % KmacDataIfWidth
IdRem	int	IdDataWidth % KmacDataIfWidth
GenRem	int	GenDataWidth % KmacDataIfWidth
AdvRounds	int	(MaxAdvDataWidth + KmacDataIfWidth - 1) / KmacDataIfWidth
IdRounds	int	(IdDataWidth + KmacDataIfWidth - 1) / KmacDataIfWidth
GenRounds	int	(GenDataWidth + KmacDataIfWidth - 1) / KmacDataIfWidth
MaxRounds	int	KDFMaxWidth / KmacDataIfWidth
CntWidth	int	\$clog2(MaxRounds)
IfBytes	int	KmacDataIfWidth / 8
DecoyCopies	int	KmacDataIfWidth / RandWidth
DecoyOutputCopies	int	(kmac_pkg::AppDigestW / RandWidth)
LastAdvRoundInt	int unsigned	AdvRounds - 1
LastIdRoundInt	int unsigned	IdRounds - 1
LastGenRoundInt	int unsigned	GenRounds - 1
LastAdvRound	bit [CntWidth-1:0]	LastAdvRoundInt[CntWidth-1:0]
LastIdRound	bit [CntWidth-1:0]	LastIdRoundInt[CntWidth-1:0]
LastGenRound	bit [CntWidth-1:0]	LastGenRoundInt[CntWidth-1:0]
AdvByteMask	logic [IfBytes-1:0]	(AdvRem > 0) ? (2**(AdvRem/8)-1) : {IfBytes{1'b1}}
IdByteMask	logic [IfBytes-1:0]	(IdRem > 0) ? (2**(IdRem/8)-1) : {IfBytes{1'b1}}
GenByteMask	logic [IfBytes-1:0]	(GenRem > 0) ? (2**(GenRem/8)-1) : {IfBytes{1'b1}}

Table 601. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv — 포트 (20개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
adv_data_i	input	[MaxAdvDataWidth-1:0]
id_data_i	input	[IdDataWidth-1:0]
gen_data_i	input	[GenDataWidth-1:0]
inputs_invalid_i	input	[3:0]
inputs_invalid_o	output	logic
adv_en_i	input	
id_en_i	input	
gen_en_i	input	
done_o	output	logic
data_o	output	logic [Shares-1:0][kmac_pkg::AppDigestW-1:0]
kmac_data_o	output	kmac_pkg::app_req_t
kmac_data_i	input	kmac_pkg::app_rsp_t
prng_en_o	output	logic
entropy_i	input	[Shares-1:0][RandWidth-1:0]
fsm_error_o	output	logic
kmac_error_o	output	logic
kmac_done_error_o	output	logic
cmd_error_o	output	logic

API — keymgr_op_state_ctrl.sv**Table 602. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_op_state_ctrl.sv — module / interface / package (1개)**

이름	종류
keymgr_op_state_ctrl	module

Table 603. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_op_state_ctrl.sv — localparam (1개)

상수	형/폭	값
OpStateWidth	int	8

Table 604. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_op_state_ctrl.sv — 포트 (15개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
adv_req_i	input	

포트	방향	형/폭
dis_req_i	input	
id_req_i	input	
gen_req_i	input	
cnt_i	input	[CdiWidth-1:0]
op_ack_o	output	logic
op_busy_o	output	logic
op_update_o	output	logic
kmac_done_i	input	
adv_en_o	output	logic
id_en_o	output	logic
gen_en_o	output	logic
op_fsm_err_o	output	logic

API — keymgr_pkg.sv

Table 605. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.sv — module / interface / package (1개)

이름	종류
keymgr_pkg	package

Table 606. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.sv — parameter (37개)

파라미터	형/폭	기본값
KeyWidth	int	256
CDIs	int	2
CdiWidth	int	prim_util_pkg::vbits(CDIs)
OtbnKeyWidth	int	384
DigestWidth	int	128
KmacDataIfWidth	int	64
KeyMgrStages	int	3
SwBindingWidth	int	32 * keymgr_reg_pkg::NumSwBindingReg
SaltWidth	int	32 * keymgr_reg_pkg::NumSaltReg
Shares	int	2
KeyVersionWidth	int	32
HealthStateWidth	int	128
DevIdWidth	int	256
MaxWidth	int	256
RndCnstRevisionSeedDefault	seed_t	256'h69802e51_bacf8874_e650d692_e3d8a646_2d3f158f_0bf7961d_d...

파라미터	형/폭	기본값
RndCnstCreatorIdentity-SeedDefault	seed_t	256'h344bff84_d3edf83e_aa565c5e_6c2d22f2_75283734_15cf6d83_3...
RndCnstOwnerIntIdentity-SeedDefault	seed_t	256'h32a37bf9_2627382b_47180548_1fc4b847_a7eba1ce_c13e80ab_0...
RndCnstOwnerIdentity-SeedDefault	seed_t	256'hc1ba416c_822ad676_e4f54f36_33d8dc7e_00ccfb8b_92209f1b_d...
RndCnstSoftOutputSeedDefault	seed_t	256'h23cbe85e_62e39992_4ab8d6a6_acbf12f0_0ca6f488_63eaa428_f...
RndCnstHardOutputSeedDefault	seed_t	256'h877ef0b9_aceaefc5_693b1aa7_e43a7f3d_ee7d63b6_4e73e182_b...
RndCnstNoneSeedDefault	seed_t	256'h6013ba4d_8d04fe4f_80cb0472_536c9679_08b82f1d_98a4e405_c...
RndCnstAesSeedDefault	seed_t	256'he16a8fa9_5b613cd5_fb9ad23f_bd8347e1_64e45dac_5d08a41b_e...
RndCnstKmacSeedDefault	seed_t	256'hc57f4c0b_b308e83f_3fc4bc63_d87dd67d_9071dc1c_e19484c8_3...
RndCnstOtbnSeedDefault	seed_t	256'hcb-cb4d2d_0abeb81b_ca7451ae_d1e2479d_ba13530a_d046b945_6...
RndCnstCdiDefault	seed_t	256'h54180905_d14c1d2f_2dda1522_f332bc0e_fcd6b92f_f0f9db75_3...
LfsrWidth	int	64
RndCnstLfsrSeedDefault	lfsr_seed_t	64'h22d326255bd24320
RndCnstLfsrPermDefault	lfsr_perm_t	{
RandWidth	int	LfsrWidth / 2
RndCnstRandPermDefault	rand_perm_t	{
AdvDataWidth	int	SwBindingWidth + 2*KeyWidth + DevIdWidth + HealthStateWidth
IdDataWidth	int	KeyWidth
GenDataWidth	int	32 + SaltWidth + KeyWidth*2
StageWidth	int	\$clog2(KeyMgrStages)
KDFMaxWidth	int	1984
HW_KEY_REQ_DEFAULT	hw_key_req_t	{
OTBN_KEY_REQ_DEFAULT	otbn_key_req_t	{

Table 607. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.sv — typedef / using (47#)

새 이름	원래 형
seed_t	logic [KeyWidth-1:0]
lfsr_seed_t	logic [LfsrWidth-1:0]
lfsr_perm_t	logic [LfsrWidth-1:0][\$clog2(LfsrWidth)-1:0]
rand_perm_t	logic [RandWidth-1:0][\$clog2(RandWidth)-1:0]

API — keymgr_reg_pkg.sv

Table 608. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_reg_pkg.sv — module / interface / package (17#)

이름	종류
keymgr_reg_pkg	package

Table 609. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_reg_pkg.sv — parameter (79개) — 앞 60개만 표시(전체 79개, 본문 소스 참조)

파라미터	형/폭	기본값
NumSaltReg	int	8
NumSwBindingReg	int	8
NumOutReg	int	8
NumKeyVersion	int	1
NumAlerts	int	2
BlockAw	int	8
NumRegs	int	63
KEYMGR_INTR_STATE_OFFSET	logic [BlockAw-1:0]	8'h 0
KEYMGR_INTR_ENABLE_OFFSET	logic [BlockAw-1:0]	8'h 4
KEYMGR_INTR_TEST_OFFSET	logic [BlockAw-1:0]	8'h 8
KEYMGR_ALERT_TEST_OFFSET	logic [BlockAw-1:0]	8'h c
KEYMGR_CFG_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 10
KEYMGR_START_OFFSET	logic [BlockAw-1:0]	8'h 14
KEYMGR_CONTROL_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 18
KEYMGR_SIDELOAD_CLEAR_OFFSET	logic [BlockAw-1:0]	8'h 1c
KEYMGR_RESEED_INTERVAL_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 20
KEYMGR_RESEED_INTERVAL_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 24
KEYMGR_SW_BINDING_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 28
KEYMGR_SEALING_SW_BINDING_0_OFFSET	logic [BlockAw-1:0]	8'h 2c
KEYMGR_SEALING_SW_BINDING_1_OFFSET	logic [BlockAw-1:0]	8'h 30
KEYMGR_SEALING_SW_BINDING_2_OFFSET	logic [BlockAw-1:0]	8'h 34
KEYMGR_SEALING_SW_BINDING_3_OFFSET	logic [BlockAw-1:0]	8'h 38
KEYMGR_SEALING_SW_BINDING_4_OFFSET	logic [BlockAw-1:0]	8'h 3c
KEYMGR_SEALING_SW_BINDING_5_OFFSET	logic [BlockAw-1:0]	8'h 40
KEYMGR_SEALING_SW_BINDING_6_OFFSET	logic [BlockAw-1:0]	8'h 44
KEYMGR_SEALING_SW_BINDING_7_OFFSET	logic [BlockAw-1:0]	8'h 48
KEYMGR_ATTEST_SW_BINDING_0_OFFSET	logic [BlockAw-1:0]	8'h 4c
KEYMGR_ATTEST_SW_BINDING_1_OFFSET	logic [BlockAw-1:0]	8'h 50
KEYMGR_ATTEST_SW_BINDING_2_OFFSET	logic [BlockAw-1:0]	8'h 54
KEYMGR_ATTEST_SW_BINDING_3_OFFSET	logic [BlockAw-1:0]	8'h 58
KEYMGR_ATTEST_SW_BINDING_4_OFFSET	logic [BlockAw-1:0]	8'h 5c

파라미터	형/폭	기본값
KEYMGR_ATTEST_SW_BINDING_5_OFFSET	logic [BlockAw-1:0]	8'h 60
KEYMGR_ATTEST_SW_BINDING_6_OFFSET	logic [BlockAw-1:0]	8'h 64
KEYMGR_ATTEST_SW_BINDING_7_OFFSET	logic [BlockAw-1:0]	8'h 68
KEYMGR_SALT_0_OFFSET	logic [BlockAw-1:0]	8'h 6c
KEYMGR_SALT_1_OFFSET	logic [BlockAw-1:0]	8'h 70
KEYMGR_SALT_2_OFFSET	logic [BlockAw-1:0]	8'h 74
KEYMGR_SALT_3_OFFSET	logic [BlockAw-1:0]	8'h 78
KEYMGR_SALT_4_OFFSET	logic [BlockAw-1:0]	8'h 7c
KEYMGR_SALT_5_OFFSET	logic [BlockAw-1:0]	8'h 80
KEYMGR_SALT_6_OFFSET	logic [BlockAw-1:0]	8'h 84
KEYMGR_SALT_7_OFFSET	logic [BlockAw-1:0]	8'h 88
KEYMGR_KEY_VERSION_OFFSET	logic [BlockAw-1:0]	8'h 8c
KEYMGR_MAX_CREATOR_KEY_VER_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 90
KEYMGR_MAX_CREATOR_KEY_VER_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 94
KEYMGR_MAX_OWNER_INT_KEY_VER_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h 98
KEYMGR_MAX_OWNER_INT_KEY_VER_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h 9c
KEYMGR_MAX_OWNER_KEY_VER_REGWEN_OFFSET	logic [BlockAw-1:0]	8'h a0
KEYMGR_MAX_OWNER_KEY_VER_SHADOWED_OFFSET	logic [BlockAw-1:0]	8'h a4
KEYMGR_SW_SHARE0_OUTPUT_0_OFFSET	logic [BlockAw-1:0]	8'h a8
KEYMGR_SW_SHARE0_OUTPUT_1_OFFSET	logic [BlockAw-1:0]	8'h ac
KEYMGR_SW_SHARE0_OUTPUT_2_OFFSET	logic [BlockAw-1:0]	8'h b0
KEYMGR_SW_SHARE0_OUTPUT_3_OFFSET	logic [BlockAw-1:0]	8'h b4
KEYMGR_SW_SHARE0_OUTPUT_4_OFFSET	logic [BlockAw-1:0]	8'h b8
KEYMGR_SW_SHARE0_OUTPUT_5_OFFSET	logic [BlockAw-1:0]	8'h bc
KEYMGR_SW_SHARE0_OUTPUT_6_OFFSET	logic [BlockAw-1:0]	8'h c0
KEYMGR_SW_SHARE0_OUTPUT_7_OFFSET	logic [BlockAw-1:0]	8'h c4
KEYMGR_SW_SHARE1_OUTPUT_0_OFFSET	logic [BlockAw-1:0]	8'h c8
KEYMGR_SW_SHARE1_OUTPUT_1_OFFSET	logic [BlockAw-1:0]	8'h cc
KEYMGR_SW_SHARE1_OUTPUT_2_OFFSET	logic [BlockAw-1:0]	8'h d0

1. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr.sv (814 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager top level
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr

```

```

11 import keymgr_pkg::*;
12 import keymgr_reg_pkg::*;
13 #(
14     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
mAlerts{1'b1}},
15     // Number of cycles a differential skew is tolerated on the
alert signal
16     parameter int unsigned AlertSkewCycles = 1,
17     // In case this is set to true, the keymgr will ignore the
creator / owner seeds
18     // on the flash_i port and use the seeds provided in
otp_key_i instead.
19     parameter bit UseOtpSeedsInsteadOfFlash = 1'b0,
20     parameter bit KmacEnMasking = 1'b1,
21     parameter lfsr_seed_t RndCnstLfsrSeed = RndCnstLfsr-

```

```

SeedDefault,
22 parameter lfsr_perm_t RndCnstLfsrPerm = RndCnstLfs-
rPermDefault,
23 parameter rand_perm_t RndCnstRandPerm = RndCnstRand-
PermDefault,
24 parameter seed_t RndCnstRevisionSeed = RndCnstRevi-
sionSeedDefault,
25 parameter seed_t RndCnstCreatorIdentitySeed = RndCnstCreat-
orIdentitySeedDefault,
26 parameter seed_t RndCnstOwnerIntIdentitySeed = RndCnstOwner-
IntIdentitySeedDefault,
27 parameter seed_t RndCnstOwnerIdentitySeed = RndCnstOwn-
erIdentitySeedDefault,
28 parameter seed_t RndCnstSoftOutputSeed = RndCnst-
SoftOutputSeedDefault,
29 parameter seed_t RndCnstHardOutputSeed = RndCn-
stHardOutputSeedDefault,
30 parameter seed_t RndCnstNoneSeed = RndCnstNone-
SeedDefault,
31 parameter seed_t RndCnstAesSeed = RndCnstAesS-
eedDefault,
32 parameter seed_t RndCnstOtbnSeed = RndCnstOtbn-
SeedDefault,
33 parameter seed_t RndCnstKmacSeed = RndCnstKmac-
SeedDefault,
34 parameter seed_t RndCnstCdi = RndCnstCdiDe-
fault
35 ) (
36 input clk_i,
37 input rst_ni,
38 input rst_shadowed_ni,
39 input clk_edn_i,
40 input rst_edn_i,
41
42 // Bus Interface
43 input tlul_pkg::tl_h2d_t tl_i,
44 output tlul_pkg::tl_d2h_t tl_o,
45
46 // key interface to crypto modules
47 output hw_key_req_t aes_key_o,
48 output hw_key_req_t kmac_key_o,
49 output otbn_key_req_t otbn_key_o,
50
51 // data interface to/from crypto modules
52 output kmac_pkg::app_req_t kmac_data_o,
53 input kmac_pkg::app_rsp_t kmac_data_i,
54
55 // whether kmac is masked
56 // Note this input is not driving ANY logic directly. In-
stead it is only used
57 // as part of assertions. This is done because if boundary
optimization were
58 // ever disabled, it would provide a VERY obvious location
for attacks.
59 input kmac_en_masking_i,
60
61 // the following signals should eventually be wrapped into
structs
from other modules
62 // SEC_CM: LC_CTRL.INTERSIG.MUBI
63 input lc_ctrl_pkg::lc_tx_t lc_keymgr_en_i,
64 input lc_ctrl_pkg::lc_keymgr_div_t lc_keymgr_div_i,
65 input otp_ctrl_pkg::otp_keymgr_key_t otp_key_i,
66 input otp_ctrl_pkg::otp_device_id_t otp_device_id_i,
67 input flash_ctrl_pkg::keymgr_flash_t flash_i,
68
69 // connection to edn
70 output edn_pkg::edn_req_t edn_o,
71 input edn_pkg::edn_rsp_t edn_i,
72
73 // connection to rom_ctrl
74 input rom_ctrl_pkg::keymgr_data_t rom_digest_i,
75
76 // interrupts and alerts
77 output logic intr_op_done_o,
78 input prim_alert_pkg::alert_rx_t [keym-
gr_reg_pkg::NumAlerts-1:0] alert_rx_i,
79 output prim_alert_pkg::alert_tx_t [keym-
gr_reg_pkg::NumAlerts-1:0] alert_tx_o
80 );
81
82 `ASSERT_INIT(AdvDataWidth_A, AdvDataWidth <= KDFMaxWidth)
83 `ASSERT_INIT(IdDataWidth_A, IdDataWidth <= KDFMaxWidth)
84 `ASSERT_INIT(GenDataWidth_A, GenDataWidth <= KDFMaxWidth)
85 `ASSERT_INIT(MaxWidthDivisible_A, KDFMaxWidth %
KmacDataIfWidth == 0)
86 `ASSERT_INIT(OutputKeyDiff_A, RndCnstHardOutputSeed != Rnd-
CnstSoftOutputSeed)
87
88 import prim_mubi_pkg::mubi4_test_true_strict;
89 import prim_mubi_pkg::mubi4_test_false_strict;
90 import lc_ctrl_pkg::lc_tx_test_true_strict;
91 import lc_ctrl_pkg::lc_tx_t;
92
93 localparam int unsigned NumRomDigestInputs = 1;
94
95 //////////////////////////////////////////////////
96 // Anchor incoming seeds and constants
97 //////////////////////////////////////////////////
98 localparam int TotalSeedWidth = KeyWidth * 10;
99 seed_t revision_seed;
100 seed_t creator_identity_seed;

```

```

101 seed_t owner_int_identity_seed;
102 seed_t owner_identity_seed;
103 seed_t soft_output_seed;
104 seed_t hard_output_seed;
105 seed_t aes_seed;
106 seed_t otbn_seed;
107 seed_t kmac_seed;
108 seed_t none_seed;
109
110 localparam logic [TotalSeedWidth-1:0] RndConstSeed = {RndCn-
stRevisionSeed,
111 RndCn-
stCreatorIdentitySeed,
112 RndCn-
stOwnerIntIdentitySeed,
113 RndCn-
stOwnerIdentitySeed,
114 RndCn-
stSoftOutputSeed,
115 RndCn-
stHardOutputSeed,
116 RndCn-
stAesSeed,
117 RndCn-
stOtbnSeed,
118 RndCn-
stKmacSeed,
119 RndCn-
stNoneSeed};
120
121 prim_sec_anchor_const #(
122 .Width(TotalSeedWidth),
123 .ConstVal(RndConstSeed)
124 ) u_seed_anchor_const (
125 .out_o({revision_seed,
126 creator_identity_seed,
127 owner_int_identity_seed,
128 owner_identity_seed,
129 soft_output_seed,
130 hard_output_seed,
131 aes_seed,
132 otbn_seed,
133 kmac_seed,
134 none_seed})
135 );
136
137 // Register module
138 keymgr_reg2hw_t reg2hw;
139 keymgr_hw2reg_t hw2reg;
140
141 logic regfile_intg_err;
142 logic shadowed_storage_err;
143 logic shadowed_update_err;
144 // SEC_CM: BUS INTEGRITY
145 // SEC_CM: CONFIG.SHADOW
146 // SEC_CM: OP.CONFIG.REGWEN, RESEED.CONFIG.REGWEN, SW_BIND-
ING.CONFIG.REGWEN
147 // SEC_CM: MAX_KEY_VER.CONFIG.REGWEN
148 keymgr_reg_top u_reg (
149 .clk_i,
150 .rst_ni,
151 .rst_shadowed_ni,
152 .tl_i,
153 .tl_o,
154 .reg2hw,
155 .hw2reg,
156 .shadowed_storage_err_o (shadowed_storage_err),
157 .shadowed_update_err_o (shadowed_update_err),
158 .intg_err_o (regfile_intg_err)
159 );
160
161 //////////////////////////////////////////////////
162 // Synchronize lc_ctrl control inputs
163 // Data inputs are not synchronized and assumed quasi-stat-
ic
164 //////////////////////////////////////////////////
165 lc_tx_t [KeyMgrEnLast-1:0] lc_keymgr_en;
166
167 prim_lc_sync #(
168 .NumCopies(int'(KeyMgrEnLast))
169 ) u_lc_keymgr_en_sync (
170 .clk_i,
171 .rst_ni,
172 .lc_en_i(lc_keymgr_en_i),
173 .lc_en_o(lc_keymgr_en)
174 );
175
176 //////////////////////////////////////////////////
177 // LFSR
178 //////////////////////////////////////////////////
179
180 // A fairly large lfsr is used here as entropy in multiple
places.
181 // - populate the default working state
182 // - generate random inputs when a bad input is selected
183 //
184 // The first case is sensitive, and thus the working state
is constructed
185 // through multiple rounds of the Lfsr
186 // The second case is less sensitive and is applied dir-

```

```

ectly. If the inputs
188 // have more bits than the lfsr output, the lfsr value is
simply replicated
189
190 logic lfsr_en;
191 logic seed_en;
192 logic [LfsrWidth-1:0] seed;
193 logic reseed_req;
194 logic reseed_ack;
195 logic reseed_done;
196 logic reseed_cnt_err;
197
198 keymgr_reseed_ctrl u_reseed_ctrl (
199   .clk_i,
200   .rst_ni,
201   .clk_edn_i,
202   .rst_edn_ni,
203   .reseed_req_i(reseed_req),
204   .reseed_ack_o(reseed_ack),
205   .reseed_done_o(reseed_done),
206   .reseed_interval_i(reg2hw.reseed_interval_shadowed.q),
207   .edn_o,
208   .edn_i,
209   .lfsr_en_i(lfsr_en),
210   .seed_en_o(seed_en),
211   .seed_o(seed),
212   .cnt_err_o(reseed_cnt_err)
213 );
214
215 logic [63:0] lfsr;
216 logic ctrl_lfsr_en, data_lfsr_en, sideload_lfsr_en;
217 assign lfsr_en = ctrl_lfsr_en | data_lfsr_en | side-
load_lfsr_en;
218
219 prim_lfsr #(
220   .LfsrDw(LfsrWidth),
221   .StateOutDw(LfsrWidth),
222   .DefaultSeed(RndCnstLfsrSeed),
223   .StatePermEn(1'b1),
224   .StatePerm(RndCnstLfsrPerm),
225   .NonLinearOut(1'b1)
226 ) u_lfsr (
227   .clk_i,
228   .rst_ni,
229   .lfsr_en_i(lfsr_en),
230   .seed_en_i(seed_en),
231   .seed_i(seed),
232   .entropy_i('0),
233   .state_o(lfsr)
234 );
235 `ASSERT_INIT(LfsrWidth_A, LfsrWidth == 64)
236
237
238 logic [Shares-1:0][RandWidth-1:0] ctrl_rand;
239 logic [Shares-1:0][RandWidth-1:0] data_rand;
240
241 assign ctrl_rand[0] = lfsr[63:32];
242 assign ctrl_rand[1] = perm_data[lfsr[31:0], RndCnstRand-
Perm);
243
244 assign data_rand[0] = lfsr[31:0];
245 assign data_rand[1] = perm_data[lfsr[63:32], RndCnstRand-
Perm);
246
247 //////////////////////////////////////////////////
248 // Key Manager Control
249 //////////////////////////////////////////////////
250
251 keymgr_stage_e stage_sel;
252 logic invalid_stage_sel;
253 prim_mubi_pkg::mubi4_t hw_key_sel;
254 logic adv_en, id_en, gen_en;
255 logic wipe_key;
256 hw_key_req_t kmac_key;
257 logic op_done;
258 logic init;
259 logic data_valid;
260 logic data_hw_en;
261 logic data_sw_en;
262 logic kmac_done;
263 logic kmac_input_invalid;
264 logic kmac_cmd_err;
265 logic kmac_fsm_err;
266 logic kmac_op_err;
267 logic kmac_done_err;
268 logic [Shares-1:0][kmac_pkg::AppDigestW-1:0] kmac_data;
269 logic [Shares-1:0][KeyWidth-1:0] kmac_data_truncated;
270 logic [ErrLastPos-1:0] err_code;
271 logic [FaultLastPos-1:0] fault_code;
272 logic sw_binding_unlock;
273 logic [CdiWidth-1:0] cdi_sel;
274 logic sideload_fsm_err;
275 logic sideload_sel_err;
276
277 for (genvar i = 0; i < Shares; i++) begin : gen_trun-
cate_data
278   assign kmac_data_truncated[i] = kmac_data[i][Key-
Width-1:0];
279 end
280
281 logic op_start;

```

```

282 assign op_start = reg2hw.start.q;
283 keymgr_ctrl #(
284   .KmacEnMasking(KmacEnMasking)
285 ) u_ctrl (
286   .clk_i,
287   .rst_ni,
288   .en_i(lc_tx_test_true_strict(lc_keymgr_en[KeyMgrEnCtrl])),
289   .regfile_intg_err_i(regfile_intg_err),
290   .shadowed_update_err_i(shadowed_update_err),
291   .shadowed_storage_err_i(shadowed_storage_err),
292   .reseed_cnt_err_i(reseed_cnt_err),
293   .sideload_sel_err_i(sideload_sel_err),
294   .sideload_fsm_err_i(sideload_fsm_err),
295   .prng_reseed_req_o(reseed_req),
296   .prng_reseed_ack_i(reseed_ack),
297   .prng_reseed_done_i(reseed_done),
298   .prng_en_o(ctrl_lfsr_en),
299   .entropy_i(ctrl_rand),
300   .op_i(keymgr_ops_e'(reg2hw.control_shadowed.operation.q)),
301   .op_start_i(op_start),
302   .op_cdi_sel_i(reg2hw.control_shadowed.cdi_sel.q),
303   .op_done_o(op_done),
304   .init_o(init),
305   .sw_binding_unlock_o(sw_binding_unlock),
306   .status_o(hw2reg.op_status.d),
307   .fault_o(fault_code),
308   .error_o(err_code),
309   .data_hw_en_o(data_hw_en),
310   .data_sw_en_o(data_sw_en),
311   .data_valid_o(data_valid),
312   .working_state_o(hw2reg.working_state.d),
313   .root_key_i(otp_key_i),
314   .hw_sel_o(hw_key_sel),
315   .stage_sel_o(stage_sel),
316   .invalid_stage_sel_o(invalid_stage_sel),
317   .cdi_sel_o(cdi_sel),
318   .wipe_key_o(wipe_key),
319   .adv_en_o(adv_en),
320   .id_en_o(id_en),
321   .gen_en_o(gen_en),
322   .key_o(kmac_key),
323   .kmac_done_i(kmac_done),
324   .kmac_input_invalid_i(kmac_input_invalid),
325   .kmac_fsm_err_i(kmac_fsm_err),
326   .kmac_op_err_i(kmac_op_err),
327   .kmac_done_err_i(kmac_done_err),
328   .kmac_cmd_err_i(kmac_cmd_err),
329   .kmac_data_i(kmac_data_truncated)
330 );
331
332 assign hw2reg.start.d = '0;
333 assign hw2reg.start.de = op_done;
334 // as long as operation is ongoing, capture status
335 assign hw2reg.op_status.de = op_start;
336
337 // working state is always visible
338 assign hw2reg.working_state.de = 1'b1;
339
340 logic cfg_regwen;
341
342 // key manager registers cannot be changed once an operation
starts
343 keymgr_cfg_en u_cfgen (
344   .clk_i,
345   .rst_ni,
346   .init_i(1'b1), // cfg_regwen does not care about init
347   .en_i(lc_tx_test_true_strict(lc_keymgr_en[KeyMgrEnCfgEn]))
348 ,
349   .set_i(op_start & op_done),
350   .clr_i(op_start),
351   .out_o(cfg_regwen)
352 );
353
354 assign hw2reg.cfg_regwen.d = cfg_regwen;
355
356 logic sw_binding_clr;
357 logic sw_binding_regwen;
358
359 // this is w0c
360 assign sw_binding_clr = reg2hw.sw_binding_regwen.qe &
~reg2hw.sw_binding_regwen.q;
361
362 // software clears the enable
363 // hardware restores it upon successful advance
364 keymgr_cfg_en #(
365   .NonInitClr(1'b1) // clear has an effect regardless of
init state
366 ) u_sw_binding_regwen (
367   .clk_i,
368   .rst_ni,
369   .init_i(init),
370   .en_i(lc_tx_test_true_strict(lc_keymgr_en[KeyMgrEnSwBindi-
ngEn])),
371   .set_i(sw_binding_unlock),
372   .clr_i(sw_binding_clr),
373   .out_o(sw_binding_regwen)
374 );
375
376 assign hw2reg.sw_binding_regwen.d = sw_binding_regwen &

```

```

cfg_regwen;
377
378 //////////////////////////////////////////////////
379 // Key Manager Input Construction
380 //////////////////////////////////////////////////
381
382 // The various arrays of inputs for each operation
383 logic rom_digest_vld;
384 logic [2**StageWidth-1:0][AdvDataWidth-1:0] adv_matrix;
385 logic [2**StageWidth-1:0] adv_dvalid;
386 logic [2**StageWidth-1:0][IdDataWidth-1:0] id_matrix;
387 logic [GenDataWidth-1:0] gen_in;
388
389 // The max key version for each stage
390 logic [2**StageWidth-1:0][31:0] max_key_versions;
391
392 // Number of times the lfsr output fits into the inputs
393 localparam int AdvLfsrCopies = AdvDataWidth / 32;
394 localparam int IdLfsrCopies = IdDataWidth / 32;
395 localparam int GenLfsrCopies = GenDataWidth / 32;
396
397 // input checking
398 logic creator_seed_vld;
399 logic owner_seed_vld;
400 logic devid_vld;
401 logic health_state_vld;
402 logic key_version_vld;
403
404 // software binding
405 logic [SwBindingWidth-1:0] sw_binding;
406 assign sw_binding = (cdi_sel == 0) ?
reg2hw.sealing_sw_binding :
407 (cdi_sel == 1) ?
reg2hw.attest_sw_binding : RndCnstCdi;
408
409 // Advance state operation input construction
410 for (genvar i = KeyMgrStages; i < 2**StageWidth; i++) be-
gin : gen_adv_matrix_fill
411 assign adv_matrix[i] = {AdvLfsrCopies{data_rand[0]}};
412 assign adv_dvalid[i] = '1'b1;
413 end
414
415 // Advance to creator_root_key
416 // The values coming from otp_ctrl / lc_ctrl are treat as
quasi-static for CDC purposes
417 logic [KeyWidth-1:0] creator_seed;
418 logic unused_creator_seed;
419 if (UseOtpSeedsInsteadOfFlash) begin : gen_otp_creator_seed
420 assign unused_creator_seed =
^({flash_i.seeds[flash_ctrl_pkg::CreatorSeedIdx],
421 otp_key_i.creat-
or_seed_vld});
422 assign creator_seed = otp_key_i.creator_seed;
423 end else begin : gen_flash_creator_seed
424 assign unused_creator_seed = ^{otp_key_i.creator_seed,
425 otp_key_i.creat-
or_seed_vld};
426 assign creator_seed =
flash_i.seeds[flash_ctrl_pkg::CreatorSeedIdx];
427 end
428 // TODO(opentitan-integrated/issues/251):
429 // replace below code with commented code once SW and DV
model can handle multiple
430 // // ROM_CTRL digests.
431 // logic [KeyWidth*NumRomDigestInputs-1:0] rom_digests;
432 // always_comb begin
433 // rom_digests = '0;
434 // for (int k = 0; k < NumRomDigestInputs; k++) begin
435 // rom_digests[KeyWidth*k +: KeyWidth] = rom_di-
gest_i[k].data;
436 // end
437 // end
438 // assign adv_matrix[Creator] = AdvDataWidth'({sw_binding,
439 //
otp_device_id_i,
440 // lc_keym-
gr_div_i,
441 // rom_digests,
442 // revi-
sion_seed});
443 assign adv_matrix[Creator] = AdvDataWidth'({sw_binding,
444 otp_device_id_i,
445 lc_keymgr_div_i,
446 rom_di-
gest_i.data,
447 revision_seed});
448
449 assign adv_dvalid[Creator] = creator_seed_vld &
450 devid_vld &
451 health_state_vld &
452 rom_digest_vld;
453
454 // Advance to owner_intermediate_key
455 logic [KeyWidth-1:0] owner_seed;
456 logic unused_owner_seed;
457 if (UseOtpSeedsInsteadOfFlash) begin : gen_otp_owner_seed
458 assign unused_owner_seed =
^({flash_i.seeds[flash_ctrl_pkg::OwnerSeedIdx],
459 otp_key_i.owner_seed_vld});
460 assign owner_seed = otp_key_i.owner_seed;
461 end else begin : gen_flash_owner_seed
462 assign unused_owner_seed = ^{otp_key_i.owner_seed,
463 otp_key_i.owner_seed_vld};
464 assign owner_seed =
flash_i.seeds[flash_ctrl_pkg::OwnerSeedIdx];
465 end
466
467 assign adv_dvalid[OwnerInt] = owner_seed_vld;
468
469 // Advance to owner_key
470 assign adv_matrix[Owner] = AdvDataWidth'({sw_binding, own-
er_seed});
471 assign adv_dvalid[Owner] = '1'b1;
472
473 // Generate Identity operation input construction
474 for (genvar i = KeyMgrStages; i < 2**StageWidth; i++) be-
gin : gen_id_matrix_fill
475 assign id_matrix[i] = {IdLfsrCopies{data_rand[0]}};
476 end
477
478 assign id_matrix[Creator] = creator_identity_seed;
479 assign id_matrix[OwnerInt] = owner_int_identity_seed;
480 assign id_matrix[Owner] = owner_identity_seed;
481
482 // Generate output operation input construction
483 logic [KeyWidth-1:0] output_key;
484 keymgr_key_dest_e dest_sel;
485 logic [KeyWidth-1:0] dest_seed;
486
487 assign dest_sel = keym-
gr_key_dest_e'(reg2hw.control_shadowed.dest_sel.q);
488 assign dest_seed = dest_sel == Aes ? aes_seed :
489 dest_sel == Kmac ? kmac_seed :
490 dest_sel == Otbn ? otbn_seed :
491 none_seed;
492 assign output_key = mubi4_test_true_strict(hw_key_sel) ?
hard_output_seed :
493 soft_output_seed;
494 assign gen_in = invalid_stage_sel ? {GenLfsrCop-
ies{lfscr[31:0]}} : {reg2hw.key_version,
495
reg2hw.salt,
496
dest_seed,
497
output_key};
498
499 // Advance state operation input construction
500 for (genvar i = KeyMgrStages; i < 2**StageWidth; i++) be-
gin : gen_key_version_fill
501 assign max_key_versions[i] = '0;
502 end
503
504 assign max_key_versions[Creator] =
reg2hw.max_creator_key_ver_shadowed.q;
505 assign max_key_versions[OwnerInt] =
reg2hw.max_owner_int_key_ver_shadowed.q;
506 assign max_key_versions[Owner] =
reg2hw.max_owner_key_ver_shadowed.q;
507
508 logic [KeyVersionWidth-1:0] cur_max_key_version;
509 assign cur_max_key_version = max_key_versions[stage_sel];
510
511 // General module for checking inputs
512 logic key_vld;
513 // SEC_CM: CONSTANTS.CONSTISTENCY
514 // SEC_CM: INTERSIG.CONSTISTENCY
515 keymgr_input_checks #(
516 .KmacEnMasking(KmacEnMasking),
517 .NumRomDigestInputs(NumRomDigestInputs)
518 ) u_checks (
519 .rom_digest_i,
520 .cur_max_key_version_i(cur_max_key_version),
521 .key_version_i(reg2hw.key_version),
522 .creator_seed_i(creator_seed),
523 .owner_seed_i(owner_seed),
524 .key_i(kmac_key_o),
525 .devid_i(otp_device_id_i),
526 .health_state_i(HealthStateWidth'(lc_keymgr_div_i)),
527 .creator_seed_vld_o(creator_seed_vld),
528 .owner_seed_vld_o(owner_seed_vld),
529 .devid_vld_o(devid_vld),
530 .health_state_vld_o(health_state_vld),
531 .key_version_vld_o(key_version_vld),
532 .key_vld_o(key_vld),
533 .rom_digest_vld_o(rom_digest_vld)
534 );
535
536 assign hw2reg.debug.invalid_creator_seed.d = '1'b1;
537 assign hw2reg.debug.invalid_owner_seed.d = '1'b1;
538 assign hw2reg.debug.invalid_dev_id.d = '1'b1;
539 assign hw2reg.debug.invalid_health_state.d = '1'b1;
540 assign hw2reg.debug.invalid_key_version.d = '1'b1;
541 assign hw2reg.debug.invalid_key.d = '1'b1;
542 assign hw2reg.debug.invalid_digest.d = '1'b1;
543
544 logic valid_op;

```

```

545 assign valid_op = adv_en | id_en | gen_en;
546 assign hw2reg.debug.invalid_creator_seed.de = adv_en &
(stage_sel == Creator) & ~creator_seed_vld;
547 assign hw2reg.debug.invalid_owner_seed.de = adv_en &
(stage_sel == OwnerInt) & ~owner_seed_vld;
548 assign hw2reg.debug.invalid_dev_id.de = adv_en & (stage_sel
== Creator) & ~dev_id_vld;
549 assign hw2reg.debug.invalid_health_state.de = adv_en &
(stage_sel == Creator) & ~health_state_vld;
550 assign hw2reg.debug.invalid_key_version.de = gen_en &
~key_version_vld;
551 assign hw2reg.debug.invalid_key.de = valid_op & ~key_vld;
552 assign hw2reg.debug.invalid_digest.de = adv_en & (stage_sel
== Creator) & ~rom_digest_vld;
553
554 ///////////////////////////////////////////////////
555 // KMAC Control
556 ///////////////////////////////////////////////////
557
558 logic [3:0] invalid_data;
559 assign invalid_data[0pAdvance] = ~key_vld | ~adv_dval-
id[stage_sel];
560 assign invalid_data[0pGenId] = ~key_vld;
561 assign invalid_data[0pGenSwOut] = ~key_vld | ~key_ver-
sion_vld;
562 assign invalid_data[0pGenHwOut] = ~key_vld | ~key_ver-
sion_vld;
563
564 keymgr_kmac_if #(
565   .RndCnstRandPerm(RndCnstRandPerm)
566 ) u_kmac_if (
567   .clk_i,
568   .rst_ni,
569   .prng_en_o(data_lfsr_en),
570   .adv_data_i(adv_matrix[stage_sel]),
571   .id_data_i(id_matrix[stage_sel]),
572   .gen_data_i(gen_in),
573   .inputs_invalid_i(invalid_data),
574   .inputs_invalid_o(kmac_input_invalid),
575   .adv_en_i(adv_en),
576   .id_en_i(id_en),
577   .gen_en_i(gen_en),
578   .done_o(kmac_done),
579   .data_o(kmac_data),
580   .kmac_data_o,
581   .kmac_data_i,
582   .entropy_i(data_rand),
583   .fsm_error_o(kmac_fsm_err),
584   .kmac_error_o(kmac_op_err),
585   .kmac_done_error_o(kmac_done_err),
586   .cmd_error_o(kmac_cmd_err)
587 );
588
589
590 ///////////////////////////////////////////////////
591 // Side load key storage
592 ///////////////////////////////////////////////////
593 // SEC_CM: HW_KEY_SW_NOACCESS
594 keymgr_sideload_key_ctrl u_sideload_ctrl (
595   .clk_i,
596   .rst_ni,
597   .init_i(init),
598   .entropy_i(data_rand),
599   .clr_key_i(keym-
gr_sideload_clr_e'(reg2hw_sideload_clear.q)),
600   .wipe_key_i(wipe_key),
601   .dest_sel_i(dest_sel),
602   .hw_key_sel_i(hw_key_sel),
603   // SEC_CM: OUTPUT_KEYS.CTRL.REDUN
604   .data_en_i(data_hw_en),
605   .data_valid_i(data_valid),
606   .key_i(kmac_key),
607   .data_i(kmac_data),
608   .prng_en_o(sideload_lfsr_en),
609   .aes_key_o,
610   .otbn_key_o,
611   .kmac_key_o,
612   .sideload_sel_err_o(sideload_sel_err),
613   .fsm_err_o(sideload_fsm_err)
614 );
615
616 for (genvar i = 0; i < 8; i++) begin : gen_sw_assigns
617
618   prim_mubi_pkg::mubi4_t [1:0] hw_key_sel_buf;
619   prim_mubi4_sync #(
620     .NumCopies(2),
621     .AsyncOn0(0)
622   ) u_mubi_buf (
623     .clk_i,
624     .rst_ni,
625     .mubi_i(hw_key_sel),
626     .mubi_o(hw_key_sel_buf)
627   );
628
629   // SEC_CM: OUTPUT_KEYS.CTRL.REDUN
630   prim_sec_anchor_buf #(
631     .Width(32)
632   ) u_prim_buf_share0_d (
633     .in_i(~data_sw_en | wipe_key ? data_rand[0] :
kmac_data[0][i*32 +: 32]),
634     .out_o(hw2reg.sw_share0_output[i].d)

```

```

635   );
636
637   prim_sec_anchor_buf #(
638     .Width(32)
639   ) u_prim_buf_share1_d (
640     .in_i(~data_sw_en | wipe_key ? data_rand[1] :
kmac_data[1][i*32 +: 32]),
641     .out_o(hw2reg.sw_share1_output[i].d)
642   );
643
644   prim_sec_anchor_buf #(
645     .Width(1)
646   ) u_prim_buf_share0_de (
647     .in_i(wipe_key | data_valid & mu-
bi4_test_false_strict(hw_key_sel_buf[0])),
648     .out_o(hw2reg.sw_share0_output[i].de)
649   );
650
651   prim_sec_anchor_buf #(
652     .Width(1)
653   ) u_prim_buf_share1_de (
654     .in_i(wipe_key | data_valid & mu-
bi4_test_false_strict(hw_key_sel_buf[1])),
655     .out_o(hw2reg.sw_share1_output[i].de)
656   );
657 end
658
659 ///////////////////////////////////////////////////
660 // Alerts and Interrupts
661 ///////////////////////////////////////////////////
662
663 prim_intr_hw #(Width(1)) u_intr_op_done (
664   .clk_i,
665   .rst_ni,
666   .event_intr_i (op_done),
667   .reg2hw_intr_enable_q_i (reg2hw_intr_enable.q),
668   .reg2hw_intr_test_q_i (reg2hw_intr_test.q),
669   .reg2hw_intr_test_qe_i (reg2hw_intr_test.qe),
670   .reg2hw_intr_state_q_i (reg2hw_intr_state.q),
671   .hw2reg_intr_state_de_o (hw2reg_intr_state.de),
672   .hw2reg_intr_state_d_o (hw2reg_intr_state.d),
673   .intr_o (intr_op_done_o)
674 );
675
676 assign hw2reg.err_code.invalid_op.d = 1'b1;
677 assign hw2reg.err_code.invalid_kmac_input.d = 1'b1;
678 assign hw2reg.err_code.invalid_shadow_update.d = 1'b1;
679 assign hw2reg.err_code.invalid_op.de =
err_code[ErrInvalidOp];
680 assign hw2reg.err_code.invalid_kmac_input.de =
err_code[ErrInvalidIn];
681 assign hw2reg.err_code.invalid_shadow_update.de =
err_code[ErrShadowUpdate];
682
683 assign hw2reg.fault_status.cmd.de =
fault_code[FaultKmacCmd];
684 assign hw2reg.fault_status.kmac_fsm.de =
fault_code[FaultKmacFsm];
685 assign hw2reg.fault_status.kmac_op.de =
fault_code[FaultKmacOp];
686 assign hw2reg.fault_status.kmac_done.de =
fault_code[FaultKmacDone];
687 assign hw2reg.fault_status.kmac_out.de =
fault_code[FaultKmacOut];
688 assign hw2reg.fault_status.regfile_intg.de =
fault_code[FaultRegIntg];
689 assign hw2reg.fault_status.shadow.de =
fault_code[FaultShadow];
690 assign hw2reg.fault_status.ctrl_fsm_intg.de =
fault_code[FaultCtrlFsm];
691 assign hw2reg.fault_status.ctrl_fsm_chk.de =
fault_code[FaultCtrlFsmChk];
692 assign hw2reg.fault_status.ctrl_fsm_cnt.de =
fault_code[FaultCtrlCnt];
693 assign hw2reg.fault_status.reseed_cnt.de =
fault_code[FaultReseedCnt];
694 assign hw2reg.fault_status.side_ctrl_fsm.de =
fault_code[FaultSideFsm];
695 assign hw2reg.fault_status.side_ctrl_sel.de =
fault_code[FaultSideSel];
696 assign hw2reg.fault_status.key_ecc.de =
fault_code[FaultKeyEcc];
697 assign hw2reg.fault_status.cmd.d = 1'b1;
698 assign hw2reg.fault_status.kmac_fsm.d = 1'b1;
699 assign hw2reg.fault_status.kmac_done.d = 1'b1;
700 assign hw2reg.fault_status.kmac_op.d = 1'b1;
701 assign hw2reg.fault_status.kmac_out.d = 1'b1;
702 assign hw2reg.fault_status.regfile_intg.d = 1'b1;
703 assign hw2reg.fault_status.shadow.d = 1'b1;
704 assign hw2reg.fault_status.ctrl_fsm_intg.d = 1'b1;
705 assign hw2reg.fault_status.ctrl_fsm_chk.d = 1'b1;
706 assign hw2reg.fault_status.ctrl_fsm_cnt.d = 1'b1;
707 assign hw2reg.fault_status.reseed_cnt.d = 1'b1;
708 assign hw2reg.fault_status.side_ctrl_fsm.d = 1'b1;
709 assign hw2reg.fault_status.side_ctrl_sel.d = 1'b1;
710 assign hw2reg.fault_status.key_ecc.d = 1'b1;
711
712 // There are two types of alerts
713 // - alerts for hardware errors, these could not have been
generated by software.
714 // - alerts for errors that may have been generated by soft-

```



```

ware.
715
716 logic fault_errs, fault_err_req_q, fault_err_req_d,
fault_err_ack;
717 logic op_errs, op_err_req_q, op_err_req_d, op_err_ack;
718
719 // Fault status can happen independently of any operation
720 assign fault_errs = |reg2hw.fault_status;
721
722 assign fault_err_req_d = fault_errs ? 1'b1 :
723 fault_err_ack ? 1'b0 :
fault_err_req_q;
724
725 assign op_errs = |err_code;
726 assign op_err_req_d = op_errs ? 1'b1 :
727 op_err_ack ? 1'b0 : op_err_req_q;
728
729 always_ff @(posedge clk_i or negedge rst_ni) begin
730 if (!rst_ni) begin
731 fault_err_req_q <= '0;
732 op_err_req_q <= '0;
733 end else begin
734 fault_err_req_q <= fault_err_req_d;
735 op_err_req_q <= op_err_req_d;
736 end
737 end
738
739 logic fault_alert_test;
740 assign fault_alert_test =
reg2hw.alert_test.fatal_fault_err.q &
741
reg2hw.alert_test.fatal_fault_err.qe;
742 prim_alert_sender #(
743 .AsyncOn(AlertAsyncOn[1]),
744 .SkewCycles(AlertSkewCycles),
745 .IsFatal(1)
746 ) u_fault_alert (
747 .clk_i,
748 .rst_ni,
749 .alert_test_i(fault_alert_test),
750 .alert_req_i(fault_err_req_q),
751 .alert_ack_o(fault_err_ack),
752 .alert_state_o(),
753 .alert_rx_i(alert_rx_i[1]),
754 .alert_tx_o(alert_tx_o[1])
755 );
756
757 logic op_err_alert_test;
758 assign op_err_alert_test =
reg2hw.alert_test.recov_operation_err.q &
759
reg2hw.alert_test.recov_operation_err.qe;
760 prim_alert_sender #(
761 .AsyncOn(AlertAsyncOn[0]),
762 .SkewCycles(AlertSkewCycles),
763 .IsFatal(0)
764 ) u_op_err_alert (
765 .clk_i,
766 .rst_ni,
767 .alert_test_i(op_err_alert_test),
768 .alert_req_i(op_err_req_q),
769 .alert_ack_o(op_err_ack),
770 .alert_state_o(),
771 .alert_rx_i(alert_rx_i[0]),
772 .alert_tx_o(alert_tx_o[0])
773 );
774
775 // known asserts
776 `ASSERT_KNOWN(TlDValidKnown0_A, tl_o.d_valid)
777 `ASSERT_KNOWN(TlAReadyKnown0_A, tl_o.a_ready)
778 `ASSERT_KNOWN(IntrKnown0_A, intr_op_done_o)
779 `ASSERT_KNOWN(AlertKnown0_A, alert_tx_o)
780
781 `ASSERT_KNOWN(AesKeyKnown0_A, aes_key_o)
782 `ASSERT_KNOWN(KmacKeyKnown0_A, kmac_key_o)
783 `ASSERT_KNOWN(OtbnKeyKnown0_A, otbn_key_o)
784 `ASSERT_KNOWN(KmacDataKnown0_A, kmac_data_o)
785
786
787 // kmac parameter consistency
788 // Both modules must be consistent with regards to masking
assumptions
789 logic unused_kmac_en_masking;
790 assign unused_kmac_en_masking = kmac_en_masking_i;
791
792 `ASSERT_INIT_NET(KmacMaskCheck_A, KmacEnMasking ==
kmac_en_masking_i)
793
794 // Ensure all parameters are consistent
795 `ASSERT_INIT(FaultCntMatch_A, FaultLastPos == AsyncFault-
LastIdx + SyncFaultLastIdx)
796 `ASSERT_INIT(ErrCntMatch_A, ErrLastPos == AsyncErrLastIdx +
SyncErrLastIdx)
797 `ASSERT_INIT(StageMatch_A, KeyMgrStages == Disable)
798
799 `ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(CtrlCntAlertCheck_A,
u_ctrl.u_cnt, alert_tx_o[1])
800 `AS-
SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(KmacIfCntAlertCheck_A, u_k-
mac_if.u_cnt, alert_tx_o[1])
801 `AS-

```

```

SERT_PRIM_COUNT_ERROR_TRIGGER_ALERT(ReseedCtrlCntAlertCheck_A, u_re-
seed_ctrl.u_reseed_cnt,
802 alert_tx_o[1])
803 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlMainFsmCheck_A,
u_ctrl.u_state_regs, alert_tx_o[1])
804 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlDataFsmCheck_A,
805 u_ctrl.u_data_en.u_state_regs, alert_tx_o[1])
806 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlOpFsmCheck_A,
807 u_ctrl.u_op_state.u_state_regs, alert_tx_o[1])
808 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(KmacIfFsmCheck_A, u_k-
mac_if.u_state_regs, alert_tx_o[1])
809 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(SideLoadCtrlFsmCheck_A,
810 u_side_load_ctrl.u_state_regs, alert_tx_o[1])
811
812 // Alert assertions for reg_we onehot check
813 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
u_reg, alert_tx_o[1])
814 endmodule // keymgr

```

2. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_cfg_en.sv (63 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager CFGEN
6 // TBD This should be enhanced in the future to contain a
shadow copy
7
8 `include "prim_assert.sv"
9
10 module keymgr_cfg_en #(
11 // controls whether clear has an effect on output value dur-
ing non-init
12 parameter bit NonInitClr = 1'b1
13 ) (
14 input clk_i,
15 input rst_ni,
16 input init_i,
17 input en_i,
18 input set_i,
19 input clr_i,
20 output logic out_o
21 );
22
23 logic out_q;
24 logic init_q;
25
26 logic vld_clr;
27 logic vld_set;
28 logic vld_dis;
29
30 assign vld_clr = init_q && clr_i;
31 assign vld_set = init_q && set_i;
32 assign vld_dis = init_q && !en_i;
33
34 // the same cycle where clear is asserted should already
block future
35 // configuration
36 logic out_clr;
37 assign out_clr = NonInitClr ? clr_i : vld_clr;
38 assign out_o = ~out_clr & out_q & en_i;
39
40 always_ff @(posedge clk_i or negedge rst_ni) begin
41 if (!rst_ni) begin
42 init_q <= '0;
43 end else if (init_q && !en_i) begin
44 init_q <= '0;
45 end else if (init_i && en_i) begin
46 init_q <= 1'b1;
47 end
48 end
49
50 // clearing the configure enable always has higher priority
than setting
51 always_ff @(posedge clk_i or negedge rst_ni) begin
52 if (!rst_ni) begin
53 out_q <= 1'b1;
54 end else if (vld_dis) begin
55 out_q <= 1'b0;
56 end else if (vld_set) begin
57 out_q <= 1'b1;
58 end else if (out_clr) begin
59 out_q <= 1'b0;
60 end
61 end
62
63 endmodule // keymgr_cfg_en

```

3. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_ctrl.sv (897 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for

```

```

details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager top level
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_ctrl
11     import keymgr_pkg::*;
12     import keymgr_reg_pkg::*;
13     #(
14         parameter bit KmacEnMasking = 1'b1
15     ) (
16         input clk_i,
17         input rst_ni,
18
19         // lifecycle enforcement
20         // SEC_CM: CTRL.FSM.GLOBAL_ESC
21         input en_i,
22
23         // faults that can occur outside of operations
24         input regfile_intg_err_i,
25         input shadowed_update_err_i,
26         input shadowed_storage_err_i,
27         input reseed_cnt_err_i,
28         input sideload_sel_err_i,
29         input sideload_fsm_err_i,
30
31         // Software interface
32         input op_start_i,
33         input keymgr_ops_e op_i,
34         input [CdiWidth-1:0] op_cdi_sel_i,
35         output logic op_done_o,
36         output keymgr_op_status_e status_o,
37         output logic [ErrLastPos-1:0] error_o,
38         output logic [FaultLastPos-1:0] fault_o,
39         output logic data_hw_en_o,
40         output logic data_sw_en_o,
41         output logic data_valid_o,
42         output logic wipe_key_o,
43         output keymgr_working_state_e working_state_o,
44         output logic sw_binding_unlock_o,
45         output logic init_o,
46
47         // Data input
48         input otp_ctrl_pkg::otp_keymgr_key_t root_key_i,
49         output prim_mubi_pkg::mubi4_t hw_sel_o,
50         output keymgr_stage_e stage_sel_o,
51         output logic invalid_stage_sel_o,
52         output logic [CdiWidth-1:0] cdi_sel_o,
53
54         // KMAC ctrl interface
55         output logic adv_en_o,
56         output logic id_en_o,
57         output logic gen_en_o,
58         output hw_key_req_t key_o,
59         input kmac_done_i,
60         input kmac_input_invalid_i, // asserted when selected data
        fails criteria check
61         input kmac_fsm_err_i, // asserted when kmac fsm reaches un-
        expected state
62         input kmac_op_err_i, // asserted when kmac itself reports
        an error
63         input kmac_done_err_i, // asserted when kmac unexpectedly
        toggles done
64         input kmac_cmd_err_i, // asserted when more than one command
        given to kmac
65         input [Shares-1:0][KeyWidth-1:0] kmac_data_i,
66
67         // prng control interface
68         input [Shares-1:0][RandWidth-1:0] entropy_i,
69         input prng_reseed_done_i,
70         input prng_reseed_ack_i,
71         output logic prng_reseed_req_o,
72         output logic prng_en_o
73     );
74
75     localparam int EntropyWidth = LfsrWidth / 2;
76     localparam int EntropyRounds = KeyWidth / EntropyWidth;
77     localparam int EntropyRndWidth =
        prim_util_pkg::vbits(EntropyRounds);
78     localparam int CntWidth = EntropyRounds > CDIs ? EntropyRnd-
        Width : CdiWidth;
79     localparam int EccDataWidth = 64;
80     localparam int EccWidth = 8;
81     localparam int EccWords = KeyWidth / EccDataWidth;
82     localparam int TotalEccWords = EccWords * Shares * CDIs;
83
84
85     // Enumeration for working state
86     // Encoding generated with:
87     // $ ./util/design/sparse-fsm-encode.py -d 5 -m 11 -n 10 \
88     // -s 4101887575 --language=sv
89     //
90     // Hamming distance histogram:
91     //
92     // 0: --
93     // 1: --
94     // 2: --

```

```

95 // 3: --
96 // 4: --
97 // 5: ||||| (54.55%)
98 // 6: ||||| (45.45%)
99 // 7: --
100 // 8: --
101 // 9: --
102 // 10: --
103 //
104 // Minimum Hamming distance: 5
105 // Maximum Hamming distance: 6
106 // Minimum Hamming weight: 2
107 // Maximum Hamming weight: 8
108 //
109 localparam int StateWidth = 10;
110 typedef enum logic [StateWidth-1:0] {
111     StCtrlReset = 10'b101100001,
112     StCtrlEntropyReSeed = 10'b1110010010,
113     StCtrlRandom = 10'b0011110100,
114     StCtrlRootKey = 10'b0110101111,
115     StCtrlInit = 10'b0100000100,
116     StCtrlCreatorRootKey = 10'b1000011101,
117     StCtrlOwnerIntKey = 10'b0001001010,
118     StCtrlOwnerKey = 10'b1101111110,
119     StCtrlDisabled = 10'b1010101000,
120     StCtrlWipe = 10'b0000110011,
121     StCtrlInvalid = 10'b1011000111
122 } state_e;
123 state_e state_q, state_d;
124
125 // A variable that represents differentiates states before
        root key and after root key.
126 logic initialized;
127
128 // There are two versions of the key state, one for sealing
        one for attestation
129 // Among each version, there are multiple shares
130 // Each share is a fixed multiple of the entropy width
131 logic [CDIs-1:0][Shares-1:0][EntropyRounds-1:0][Entropy-
        Width-1:0] key_state_d;
132 logic [CDIs-1:0][Shares-1:0][EccWords-1:0][EccDataWidth-1:0]
        key_state_ecc_words_d;
133 logic [CDIs-1:0][Shares-1:0][EccWords-1:0][EccDataWidth-1:0]
        key_state_q;
134 logic [CDIs-1:0][Shares-1:0][EccWords-1:0][EccWidth-1:0]
        key_state_ecc_q;
135 logic [CntWidth-1:0] cnt;
136 logic [CdiWidth-1:0] cdi_cnt;
137
138 // error conditions
139 logic invalid_kmac_out;
140 logic invalid_op;
141 logic cnt_err;
142 // states fall out of sparsely encoded range
143 logic state_intg_err_q, state_intg_err_d;
144
145 //
146 // General operation decode
147 //
148
149 logic adv_op, dis_op, gen_id_op, gen_sw_op, gen_hw_op,
        gen_op;
150 assign adv_op = (op_i == OpAdvance);
151 assign gen_id_op = (op_i == OpGenId);
152 assign gen_sw_op = (op_i == OpGenSwOut);
153 assign gen_hw_op = (op_i == OpGenHwOut);
154 assign dis_op = ~(op_i inside {OpAdvance, OpGenId, Op-
        GenSwOut, OpGenHwOut});
155 assign gen_op = (gen_id_op | gen_sw_op | gen_hw_op);
156
157 //
158 // interaction between software and main fsm
159 //
160 // disable is treated like an advanced call
161 logic advance_sel;
162 logic disable_sel;
163 logic gen_out_hw_sel;
164
165 assign advance_sel = op_start_i & adv_op & en_i;
166 assign gen_out_hw_sel = op_start_i & gen_hw_op & en_i;
167
168 // disable is selected whenever a normal operation is not
        set
169 assign disable_sel = (op_start_i & dis_op) | !en_i;
170
171
172 //
173 // interaction between main control fsm and operation fsm
174 //
175
176 // req/ack interface with op handling fsm
177 logic op_req;
178 logic op_ack;
179 logic op_update;
180 logic op_busy;
181 logic disabled;
182 logic invalid;
183
184 logic adv_req, dis_req, id_req, gen_req;

```



```

185 assign adv_req = op_req & adv_op;
186 assign dis_req = op_req & dis_op;
187 assign id_req = op_req & gen_id_op;
188 assign gen_req = op_req & (gen_sw_op | gen_hw_op);
189
190 //////////////////////////////////////////////////
191 // interaction between operation fsm and software
192 //////////////////////////////////////////////////
193 // categories of keymgr errors
194 logic [SyncErrLastIdx-1:0] sync_err;
195 logic [SyncFaultLastIdx-1:0] sync_fault;
196 logic [AsyncFaultLastIdx-1:0] async_fault;
197
198 logic op_err;
199 logic op_fault_err;
200
201 // unlock sw binding configuration whenever an advance call
is made without errors
202 assign sw_binding_unlock_o = adv_req & op_ack & ~(op_err |
op_fault_err);
203
204 // error definition
205 // check incoming kmac data validity
206 // Only check during the periods when there is actual kmac
output
207 assign invalid_kmac_out = (op_update | op_ack) &
208 (~valid_data_chk(kmac_data_i[0]) |
209 (~valid_data_chk(kmac_data_i[1]) &
KmacEnMasking));
210
211 // async errors have nothing to do with the operation and
thus should not
212 // impact operation results.
213 assign op_err = |sync_err;
214
215 assign op_fault_err = |{sync_fault, async_fault};
216
217 //////////////////////////////////////////////////
218 // key update controls
219 //////////////////////////////////////////////////
220
221 // update select can come from both main and operation fsm's
222 keymgr_key_update_e update_sel, op_update_sel;
223
224 // req from main control fsm to key update controls
225 logic wipe_req;
226 logic random_req;
227 logic random_ack;
228
229 // wipe and initialize take precedence
230 assign update_sel = wipe_req ? KeyUpdateWipe :
231 random_req ? KeyUpdateRandom :
232 init_o ? KeyUpdateRoot :
op_update_sel;
233
234 //////////////////////////////////////////////////
235 // interaction between main fsm and prng
236 //////////////////////////////////////////////////
237
238 // Upon entering StCtrlDisabled or StCtrlInvalid, the PRNG
is kept advancing until it has been
239 // reseeded twice (through the reseeding mechanism inside
keymgr_reseed_ctrl.sv).
240 logic [1:0] prng_en_dis_inv_d, prng_en_dis_inv_q;
241 logic prng_en_dis_inv_set;
242
243 assign prng_en_dis_inv_d =
244 prng_en_dis_inv_set ? 2'b11 :
245 prng_reseed_done_i ? {1'b0, prng_en_dis_inv_q[1]} :
prng_en_dis_inv_q;
246
247 always_ff @(posedge clk_i or negedge rst_ni) begin
248 if (!rst_ni) begin
249 prng_en_dis_inv_q <= '0;
250 end else begin
251 prng_en_dis_inv_q <= prng_en_dis_inv_d;
252 end
253 end
254
255 assign prng_en_o = random_req | wipe_req |
prng_en_dis_inv_q[0];
256
257 //////////////////////////////////////////////////
258 // Main Control FSM
259 //////////////////////////////////////////////////
260 // SEC_CM: CTRL.FSM.SPARSE
261 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StCtrlReset)
262
263 always_ff @(posedge clk_i or negedge rst_ni) begin
264 if (!rst_ni) begin
265 state_intg_err_q <= '0;
266 end else begin
267 state_intg_err_q <= state_intg_err_d;
268 end
269 end
270
271 // prevents unknowns from reaching the outside world.
272 // - whatever operation causes the input data select to be

```

```

disabled should not expose the key
273 // state.
274 // - when there are no operations, the key state also should
be exposed.
275 assign key_o.valid = op_req;
276
277 assign cdi_sel_o = advance_sel ? cdi_cnt : op_cdi_sel_i;
278
279 assign invalid_stage_sel_o = ~(stage_sel_o inside {Creator,
OwnerInt, Owner});
280 for (genvar i = 0; i < Shares; i++) begin :
gen_key_out_assign
281 assign key_o.key[i] = invalid_stage_sel_o ?
282 {EntropyRounds(entropy_i[i])} :
283 key_state_q[cdi_sel_o][i];
284 end
285
286
287 //SEC_CM: CTRL.KEY.INTEGRITY
288 assign key_state_ecc_words_d = key_state_d;
289 always_ff @(posedge clk_i or negedge rst_ni) begin
290 if (!rst_ni) begin
291 key_state_q <= '0;
292 key_state_ecc_q <= {TotalEc-
cWords{prim_secdd_pkg::SecddInv7264ZeroEcc}};
293 end else begin
294 for (int i = 0; i < CDIs; i++) begin
295 for (int j = 0; j < Shares; j++) begin
296 for (int k = 0; k < EccWords; k++) begin
297 {key_state_ecc_q[i][j][k], key_state_q[i][j][k]}
<=
298 prim_secdd_pkg::prim_secdd_inv_72_64_enc(key_state_ecc_words_d[i]
[j][k]);
299 end
300 end
301 end
302 end
303 end
304
305 logic [CDIs-1:0][Shares-1:0][EccWords-1:0] ecc_errs;
306 for (genvar i = 0; i < CDIs; i++) begin : gen_ecc_loop_cdi
307 for (genvar j = 0; j < Shares; j++) begin :
gen_ecc_loop_shares
308 for (genvar k = 0; k < EccWords; k++) begin :
gen_ecc_loop_words
309 logic [1:0] errs;
310 prim_secdd_inv_72_64_dec u_dec (
311 .data_i({key_state_ecc_q[i][j][k], key_state_q[i][j]
[k]}),
312 .data_o(),
313 .syndrome_o(),
314 .err_o(errs)
315 );
316 assign ecc_errs[i][j][k] = |errs;
317 end
318 end
319 end
320
321 // These are consumed one level above in keymgr.sv
322 logic unused_otp_sigs;
323 assign unused_otp_sigs = ^{root_key_i.creator_seed,
root_key_i.creator_seed_valid,
root_key_i.owner_seed,
root_key_i.owner_seed_valid};
324
325 // root key valid sync
326 logic root_key_valid_q;
327
328 prim_flop_2sync # (
329 .Width(1)
330 ) u_key_valid_sync (
331 .clk_i,
332 .rst_ni,
333 // Both valid signals are flopped in OTP_CTRL, and they
only ever transition from 0 -> 1.
334 // It is hence ok to AND them here before the synchron-
izer, since we don't expect this
335 // to create glitches.
336 .d_i(root_key_i.creator_root_key_share0_valid &&
root_key_i.creator_root_key_share1_valid),
337 .q_o(root_key_valid_q)
338 );
339
340 // Do not let the count toggle unless an advance operation
is
341 // selected
342 assign cdi_cnt = op_req ? cnt[CDiWidth-1:0] : '0;
343
344 always_comb begin
345 key_state_d = key_state_q;
346 data_valid_o = 1'b0;
347 wipe_key_o = 1'b0;
348
349 // if a wipe request arrives, immediately destroy the
350 // keys regardless of current state
351 unique case (update_sel)
352 KeyUpdateRandom: begin
353 for (int i = 0; i < CDIs; i++) begin
354 for (int j = 0; j < Shares; j++) begin

```

```

359          // Load each share with the same randomness so we
can
360          // later simply XOR root key on them
361          key_state_d[i][j][cnt[EntropyRndWidth-1:0]] = en-
tropy_i[i];
362      end
363  end
364  end
365
366  KeyUpdateRoot: begin
367      if (root_key_valid_q) begin
368          for (int i = 0; i < CDIs; i++) begin
369              if (KmacEnMasking) begin : gen_two_share_key
370                  key_state_d[i][0] ^= root_key_i.creat-
or_root_key_share0;
371                  key_state_d[i][1] ^= root_key_i.creat-
or_root_key_share1;
372              end else begin : gen_one_share_key
373                  key_state_d[i][0] = root_key_i.creat-
or_root_key_share0 ^
374                      root_key_i.creat-
or_root_key_share1;
375                  key_state_d[i][1] = '0;
376              end
377          end
378      end else begin
379          // if root key is not valid, load and invalid value
380          for (int i = 0; i < CDIs; i++) begin
381              key_state_d[i][0] = '0;
382              key_state_d[i][1] = '{default: '1;
383          end
384      end
385  end
386
387  KeyUpdateKmac: begin
388      data_valid_o = gen_op;
389      key_state_d[cdi_sel_o] = (adv_op || dis_op) ?
kmac_data_i : key_state_q[cdi_sel_o];
390  end
391
392  KeyUpdateWipe: begin
393      wipe_key_o = 'b1;
394      for (int i = 0; i < CDIs; i++) begin
395          for (int j = 0; j < Shares; j++) begin
396              key_state_d[i][j] = {EntropyRounds[entropy_i[j]]};
397          end
398      end
399  end
400
401  default;;
402  endcase // unique case (update_sel)
403  end
404
405  // SEC_CM: CTRL.CTR.REDUN
406  prim_count #(
407      .Width(CntWidth)
408  ) u_cnt (
409      .clk_i,
410      .rst_ni,
411      .clr_i(op_ack | random_ack),
412      .set_i('0),
413      .set_cnt_i('0),
414      .incr_en_i(op_update | random_req),
415      .decr_en_i(1'b0),
416      .step_i(CntWidth'(1'b1)),
417      .commit_i(1'b1),
418      .cnt_o(cnt),
419      .cnt_after_commit_o(),
420      .err_o(cnt_err)
421  );
422
423  prim_mubi4_sender u_hw_sel (
424      .clk_i,
425      .rst_ni,
426      .mubi_i
427      (prim_mubi_pkg:mubi4_bool_to_mubi(gen_out_hw_sel)),
428      .mubi_o (hw_sel_o)
429  );
430
431  // when in a state that accepts commands, look at op_ack for
completion
432  // when in a state that does not accept commands, wait for
other triggers.
433  assign op_done_o = op_req ? op_ack :
434      (init_o | invalid_op);
435
436  // There are 3 possibilities
437  // advance to next state (software command)
438  // advance to disabled state (software command)
439  // advance to invalid state (detected fault)
440  logic adv_state;
441  logic dis_state;
442  logic inv_state;
443  assign adv_state = op_ack & adv_req & ~op_err;
444  assign dis_state = op_ack & dis_req;
445
446  // SEC_CM: CTRL.FSM.LOCAL_ESC
447  // begin invalidation when faults are observed.

```

```

449  // sync faults only invalidate on transaction boundaries
450  // async faults begin invalidating immediately
451  assign inv_state = |fault_o;
452
453  always_comb begin
454      // persistent data
455      state_d = state_q;
456
457      // request to op handling
458      op_req = 1'b0;
459      random_req = 1'b0;
460      random_ack = 1'b0;
461
462      // request to key updates
463      wipe_req = 1'b0;
464
465      // invalid operation issued
466      invalid_op = '0;
467
468      // data update and select signals
469      stage_sel_o = Disable;
470
471      // indication that state is disabled
472      disabled = 1'b0;
473
474      // indication that state is invalid
475      invalid = 1'b0;
476
477      // Don't request final PRNG updating and reseeding.
478      prng_en_dis_inv_set = 1'b0;
479
480      // Request PRNG reseeding.
481      prng_reseed_req_o = 1'b0;
482
483      // initialization complete
484      init_o = 1'b0;
485
486      // Most states are initialized, mark the exceptions
487      initialized = 1'b1;
488
489      // if state is ever faulted, hold on to this indication
490      // until reset.
491      state_intg_err_d = state_intg_err_q;
492
493  unique case (state_q)
494      // Only advance can be called from reset state
495      StCtrlReset: begin
496          initialized = 1'b0;
497
498          // always use random data for advance, since out of
reset state
499          // the key state will be randomized.
500          stage_sel_o = Disable;
501
502          // key state is updated when it is an advance call
503          // all other operations are invalid, including disable
504          invalid_op = op_start_i & ~advance_sel;
505
506          // if there was a structural fault before anything
began, wipe immediately
507          if (inv_state) begin
508              state_d = StCtrlWipe;
509          end else if (advance_sel) begin
510              state_d = StCtrlEntropyReseed;
511          end
512      end
513
514      // reseed entropy
515      StCtrlEntropyReseed: begin
516          initialized = 1'b0;
517          prng_reseed_req_o = 1'b1;
518
519          if (prng_reseed_ack_i) begin
520              state_d = StCtrlRandom;
521          end
522      end
523
524      // This state does not accept any command.
525      StCtrlRandom: begin
526          initialized = 1'b0;
527          random_req = 1'b1;
528
529          // when mask population is complete, xor the root_key
into the zero share
530          // if in the future the root key is updated to 2
shares, it will directly overwrite
531          // the values here
532          if (int'(cnt) == EntropyRounds-1) begin
533              random_ack = 1'b1;
534              state_d = StCtrlRootKey;
535          end
536      end
537
538      // load the root key.
539      StCtrlRootKey: begin
540          init_o = 1'b1;
541          initialized = 1'b1;
542          state_d = (en_i && root_key_valid_q) ? StCtrlInit :
StCtrlWipe;
543      end
544

```

```

545 // Beginning from the Init state, operations are accep-
ted.
546 // Only valid operation is advance state. If invalid
command received,
547 // random data is selected for operation and no persist-
ent state is changed.
548 StCtrlInit: begin
549   op_req = op_start_i;
550
551 // when advancing select creator data, otherwise use
random input
552   stage_sel_o = advance_sel ? Creator : Disable;
553   invalid_op = op_start_i & ~(advance_sel | dis-
able_sel);
554
555   if (!en_i || inv_state) begin
556     state_d = StCtrlWipe;
557   end else if (dis_state) begin
558     state_d = StCtrlDisabled;
559     prng_en_dis_inv_set = 1'b1;
560   end else if (adv_state) begin
561     state_d = StCtrlCreatorRootKey;
562   end
563 end
564
565 // all commands are valid during this stage
566 StCtrlCreatorRootKey: begin
567   op_req = op_start_i;
568
569 // when generating, select creator data input
570 // when advancing, select owner intermediate key as
target
571 // when disabling, select random data input
572   stage_sel_o = disable_sel ? Disable :
573     advance_sel ? OwnerInt : Creator;
574
575   if (!en_i || inv_state) begin
576     state_d = StCtrlWipe;
577   end else if (dis_state) begin
578     state_d = StCtrlDisabled;
579     prng_en_dis_inv_set = 1'b1;
580   end else if (adv_state) begin
581     state_d = StCtrlOwnerIntKey;
582   end
583 end
584
585 // all commands are valid during this stage
586 StCtrlOwnerIntKey: begin
587   op_req = op_start_i;
588
589 // when generating, select owner intermediate data in-
put
590 // when advancing, select owner as target
591 // when disabling, select random data input
592   stage_sel_o = disable_sel ? Disable :
593     advance_sel ? Owner : OwnerInt;
594
595   if (!en_i || inv_state) begin
596     state_d = StCtrlWipe;
597   end else if (dis_state) begin
598     state_d = StCtrlDisabled;
599     prng_en_dis_inv_set = 1'b1;
600   end else if (adv_state) begin
601     state_d = StCtrlOwnerKey;
602   end
603 end
604
605 // all commands are valid during this stage
606 // however advance goes directly to disabled state
607 StCtrlOwnerKey: begin
608   op_req = op_start_i;
609
610 // when generating, select owner data input
611 // when advancing, select disable as target
612 // when disabling, select random data input
613   stage_sel_o = disable_sel | advance_sel ? Disable :
Owner;
614
615   if (!en_i || inv_state) begin
616     state_d = StCtrlWipe;
617   end else if (adv_state || dis_state) begin
618     state_d = StCtrlDisabled;
619     prng_en_dis_inv_set = 1'b1;
620   end
621 end
622
623 // The wipe state immediately clears out the key state,
but waits for any ongoing
624 // transaction to finish before going to disabled state.
625 // Unlike the random state, this is an immediate shut-
down request, so all parts of the
626 // key are wiped.
627 StCtrlWipe: begin
628   wipe_req = 1'b1;
629
630 // if there was already an operation ongoing, maintain
the request until completion
631   op_req = op_busy;
632   invalid_op = op_start_i;
633
634 // If the enable is dropped during the middle of a

```

```

transaction, we clear and wait for that
634 // transaction to gracefully complete (if it can).
635 // There are two scenarios:
636 // 1. the operation completed right when we started
wiping, in which case the done would
637 // clear the start.
638 // 2. the operation completed before we started wip-
ing, or there was never an operation to
639 // begin with (op_start_i == 0), in this case,
don't wait and immediately transition
640 if (!op_start_i) begin
641   state_d = StCtrlInvalid;
642   prng_en_dis_inv_set = 1'b1;
643 end
644 end
645
646 // StCtrlDisabled and StCtrlInvalid are almost function-
ally equivalent
647 // The only difference is that Disabled is entered
through software invocation,
648 // while Invalid is entered through life cycle disable
or operational fault.
649 //
650 // Both states continue to kick off random transactions
651 // All transactions are treated as invalid despite com-
pleting
652 StCtrlDisabled: begin
653   op_req = op_start_i;
654   disabled = 1'b1;
655
656   if (!en_i || inv_state) begin
657     state_d = StCtrlWipe;
658   end
659 end
660
661 StCtrlInvalid: begin
662   invalid_op = op_start_i;
663   invalid = 1'b1;
664 end
665
666 // latch the fault indication and start to wipe the key
manager
667 default: begin
668   state_intg_err_d = 1'b1;
669   state_d = StCtrlWipe;
670 end
671
672 endcase // unique case (state_q)
673 end // always_comb
674
675 // Current working state provided for software read
676 // Certain states are collapsed for simplicity
677 keymgr_working_state_e last_working_st;
678 logic update_en;
679
680 always_ff @(posedge clk_i or negedge rst_ni) begin
681   if (!rst_ni) begin
682     last_working_st <= StReset;
683   end else if (update_en) begin
684     last_working_st <= working_state_o;
685   end
686 end
687
688 always_comb begin
689   update_en = 1'b1;
690   working_state_o = StInvalid;
691
692   unique case (state_q)
693     StCtrlReset, StCtrlEntropyReSeed, StCtrlRandom:
694       working_state_o = StReset;
695
696     StCtrlRootKey, StCtrlInit:
697       working_state_o = StInit;
698
699     StCtrlCreatorRootKey:
700       working_state_o = StCreatorRootKey;
701
702     StCtrlOwnerIntKey:
703       working_state_o = StOwnerIntKey;
704
705     StCtrlOwnerKey:
706       working_state_o = StOwnerKey;
707
708     StCtrlDisabled:
709       working_state_o = StDisabled;
710
711     StCtrlWipe: begin
712       update_en = 1'b0;
713       working_state_o = last_working_st;
714     end
715
716     StCtrlInvalid:
717       working_state_o = StInvalid;
718
719     default:
720       working_state_o = StInvalid;
721   endcase // unique case (state_q)
722 end
723
724 always_comb begin
725   status_o = OpIdle;

```

```

726   if (op_done_o) begin
727     // It is possible for an operation to finish the same
cycle en_i goes low.
728     // The main fsm handling is one cycle behind, but still
report operation
729     // fail.
730     status_o = {error_o, fault_o} ? OpDoneFail : OpDoneSuc-
cess;
731   end else if (op_start_i) begin
732     status_o = OpWip;
733   end
734 end
735
736 //////////////////////////////////////////////////
737 // Operation state, handle advance and generate
738 //////////////////////////////////////////////////
739
740 logic op_fsm_err;
741 keymgr_op_state_ctrl u_op_state (
742   .clk_i,
743   .rst_ni,
744   .adv_req_i(adv_req),
745   .dis_req_i(dis_req),
746   .id_req_i(id_req),
747   .gen_req_i(gen_req),
748   .cnt_i(cdi_cnt),
749   .op_ack_o(op_ack),
750   .op_busy_o(op_busy),
751   .op_update_o(op_update),
752   .kmac_done_i,
753   .adv_en_o,
754   .id_en_o,
755   .gen_en_o,
756   .op_fsm_err_o(op_fsm_err)
757 );
758
759 // operational state cross check. The state value must be
consistent with
760 // the input operations.
761 logic op_state_cmd_err;
762 assign op_state_cmd_err = (adv_en_o & ~(advance_sel | dis-
able_sel)) |
763   (gen_en_o & ~gen_op);
764
765 // operations fsm update precedence
766 // when in invalid state, always update.
767 // when in disabled state, always update unless a fault is
encountered.
768
769 assign op_update_sel = (op_ack | op_update) & invalid ?
KeyUpdateKmac :
770   (op_ack | op_update) & op_fault_err ?
KeyUpdateWipe :
771   (op_ack | op_update) & disabled ?
KeyUpdateKmac :
772   (op_ack | op_update) & op_err ?
KeyUpdateIdle :
773   (op_ack | op_update) ?
KeyUpdateKmac : KeyUpdateIdle;
774
775 //////////////////////////////////////////////////
776 // Suppress kmac return data
777 //////////////////////////////////////////////////
778
779 logic data_fsm_err;
780 keymgr_data_en_state u_data_en (
781   .clk_i,
782   .rst_ni,
783   .hw_sel_i(hw_sel_o),
784   .adv_en_i(adv_en_o),
785   .id_en_i(id_en_o),
786   .gen_en_i(gen_en_o),
787   .op_done_i(op_done_o),
788   .op_start_i,
789   .data_hw_en_o,
790   .data_sw_en_o,
791   .fsm_err_o(data_fsm_err)
792 );
793
794 //////////////////////////////////////////////////
795 // Cross-checks, errors and faults
796 //////////////////////////////////////////////////
797
798 logic vld_state_change_d, vld_state_change_q;
799 assign vld_state_change_d = (state_d != state_q) &
800   (state_d inside {StCtrlRootKey,
801     StCtrlCreator-
RootKey,
802     StCtrlOwner-
IntKey,
803     StCtrlOwner-
Key});
804
805 // capture for cross check in following cycle
806 always_ff @(posedge clk_i or negedge rst_ni) begin
807

```

```

808   if (!rst_ni) begin
809     vld_state_change_q <= '0;
810   end else begin
811     vld_state_change_q <= vld_state_change_d;
812   end
813 end
814
815 // state cross check
816 // if the state advanced, ensure that it was due to an ad-
vanced operation
817 logic state_change_err;
818 assign state_change_err = vld_state_change_q & !adv_op;
819
820 keymgr_err_u_err (
821   .clk_i,
822   .rst_ni,
823   .invalid_op_i(invalid_op),
824   .disabled_i(disabled | (initialized & ~en_i)),
825   .invalid_i(invalid),
826   .kmac_input_invalid_i,
827   .shadowed_update_err_i,
828   .kmac_op_err_i,
829   .invalid_kmac_out_i(invalid_kmac_out),
830   .sideload_sel_err_i,
831   .kmac_cmd_err_i,
832   .kmac_fsm_err_i,
833   .kmac_done_err_i,
834   .regfile_intg_err_i,
835   .shadowed_storage_err_i,
836   .ctrl_fsm_err_i(state_intg_err_q | state_intg_err_d),
837   .data_fsm_err_i(data_fsm_err),
838   .op_fsm_err_i(op_fsm_err),
839   .ecc_err_i(|ecc_errs),
840   .state_change_err_i(state_change_err),
841   .op_state_cmd_err_i(op_state_cmd_err),
842   .cnt_err_i(cnt_err),
843   .reseed_cnt_err_i,
844   .sideload_fsm_err_i,
845
846   .op_update_i(op_update),
847   .op_done_i(op_done_o),
848
849   .sync_err_o(sync_err),
850   .async_err_o(),
851   .sync_fault_o(sync_fault),
852   .async_fault_o(async_fault),
853   .error_o,
854   .fault_o
855 );
856
857 //////////////////////////////////////////////////
858 // Assertions
859 //////////////////////////////////////////////////
860
861 // This assertion will not work if fault_status ever takes
on metafields such as
862 // qe / re etc.
863 `ASSERT_INIT(SameErrCnt_A, $bits(keym-
gr_reg2hw_fault_status_reg_t) ==
864   (SyncFaultLastIdx + AsyncFault-
LastIdx))
865
866 // stage select should always be Disable whenever it is not
enabled
867 `ASSERT(StageDisableSel_A, !en_i |-> stage_sel_o == Disable)
868
869 // Unless it is a legal command, only select disable
870 `ASSERT(InitLegalCommands_A, op_start_i & en_i & state_q in-
side {StCtrlInit} &
871   !(op_i inside {OpAdvance}) |->
stage_sel_o == Disable)
872
873 // All commands are legal, so select disable only if opera-
tion is disable
874 `ASSERT(GeneralLegalCommands_A, op_start_i & en_i &
state_q inside {StCtrlCreat-
orRootKey, StCtrlOwnerIntKey} &
875   (op_i inside {OpDisable}) |->
> stage_sel_o == Disable)
876
877 `ASSERT(OwnerLegalCommands_A, op_start_i & en_i & state_q
inside {StCtrlOwnerKey} &
878   (op_i inside {OpAdvance, Op-
Disable}) |-> stage_sel_o == Disable)
879
880 // load_key should not be high if there is no ongoing opera-
tion
881 `ASSERT(LoadKey_A, key_o.valid |-> op_start_i)
882
883 // The count value should always be 0 when a transaction
start
884 `ASSERT(CntZero_A, $rose(op_start_i) |-> cnt == '0)
885
886 // Whenever a transaction completes, data_en must return to
0 on the next cycle
887 `ASSERT(DataEndDis_A, op_start_i & op_done_o |>
~data_hw_en_o && ~data_sw_en_o)
888
889 // Whenever data enable asserts, it must be the case that
there was a generate or
890 // id operation
891

```

```

892 `ASSERT(DataEn_A, data_hw_en_o | data_sw_en_o |-> (id_en_o |
gen_en_o) & ~adv_en_o)
893
894 // Check that the FSM is linear and does not contain any
loops
895 `ASSERT_FPV_LINEAR_FSM(SecCmCFILinear_A, state_q, state_e)
896
897 endmodule

```

4. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_data_en_state.sv (136 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager data enable generation
6 // This is a redundant alternative to data_valid
7
8 `include "prim_assert.sv"
9
10 module keymgr_data_en_state
11   import keymgr_pkg::*;
12   import keymgr_reg_pkg::*;
13 (
14   input clk_i,
15   input rst_ni,
16   input prim_mubi_pkg::mubi4_t hw_sel_i,
17   input adv_en_i,
18   input id_en_i,
19   input gen_en_i,
20   input op_done_i,
21   input op_start_i,
22   output logic data_hw_en_o,
23   output logic data_sw_en_o,
24   output logic fsm_err_o
25 );
26
27   import prim_mubi_pkg::mubi4_test_true_strict;
28   import prim_mubi_pkg::mubi4_test_true_loose;
29   import prim_mubi_pkg::mubi4_test_false_strict;
30   import prim_mubi_pkg::mubi4_test_false_loose;
31
32 // This is a separate data path from the FSM used to control
the data_en outputs
33 // Encoding generated with:
34 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 6 -n 10 \
35 // -s 2015444891 --language=sv
36 //
37 // Hamming distance histogram:
38 //
39 // 0: --
40 // 1: --
41 // 2: --
42 // 3: --
43 // 4: --
44 // 5: ||||| (33.33%)
45 // 6: ||||| (40.00%)
46 // 7: ||||| (26.67%)
47 // 8: --
48 // 9: --
49 // 10: --
50 //
51 // Minimum Hamming distance: 5
52 // Maximum Hamming distance: 7
53 // Minimum Hamming weight: 2
54 // Maximum Hamming weight: 7
55 //
56 localparam int DataStateWidth = 10;
57 typedef enum logic [DataStateWidth-1:0] {
58   StCtrlDataIdle = 10'b100010000,
59   StCtrlDataHwEn = 10'b0001100100,
60   StCtrlDataSwEn = 10'b1110101110,
61   StCtrlDataDis = 10'b0010011111,
62   StCtrlDataWait = 10'b0111110011,
63   StCtrlDataInvalid = 10'b1111001001
64 } state_e;
65
66   state_e state_d, state_q;
67
68 // SEC_CM: DATA.FSM.SPARSE
69 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StCtrlDataIdle)
70
71 // The below control path is used for modulating the
datapath to sideload and sw keys.
72 // This path is separate from the data_valid_o path, thus
creating two separate attack points.
73 // The data is only enabled when a non-advance operation is
invoked.
74 // When an advance operation is called, the data is dis-
abled. It will stay disabled until an
75 // entire completion sequence is seen (op_done_o assert ->
start_i de-assertion).
76 // When a generate operation is called, the data is en-
abled. However, any indication of this

```

```

77 // supposedly being an advance call will force the path to
disable again.
78 always_comb begin
79   state_d = state_q;
80   fsm_err_o = 1'b0;
81   data_hw_en_o = 1'b0;
82   data_sw_en_o = 1'b0;
83   unique case (state_q)
84
85     StCtrlDataIdle: begin
86       if (adv_en_i) begin
87         state_d = StCtrlDataDis;
88       end else if ((id_en_i || gen_en_i) && mu-
bi4_test_true_strict(hw_sel_i)) begin
89         state_d = StCtrlDataHwEn;
90       end else if ((id_en_i || gen_en_i) && mu-
bi4_test_false_strict(hw_sel_i)) begin
91         state_d = StCtrlDataSwEn;
92       end else if (id_en_i || gen_en_i) begin
93         state_d = StCtrlDataDis;
94       end
95     end
96
97     StCtrlDataHwEn: begin
98       data_hw_en_o = 1'b1;
99       if (op_done_i) begin
100         state_d = StCtrlDataWait;
101       end else if (adv_en_i || mu-
bi4_test_false_loose(hw_sel_i)) begin
102         state_d = StCtrlDataDis;
103       end
104     end
105
106     StCtrlDataSwEn: begin
107       data_sw_en_o = 1'b1;
108       if (op_done_i) begin
109         state_d = StCtrlDataWait;
110       end else if (adv_en_i || mu-
bi4_test_true_loose(hw_sel_i)) begin
111         state_d = StCtrlDataDis;
112       end
113     end
114
115     StCtrlDataDis: begin
116       if (op_done_i) begin
117         state_d = StCtrlDataWait;
118       end
119     end
120
121     StCtrlDataWait: begin
122       if (!op_start_i) begin
123         state_d = StCtrlDataIdle;
124       end
125     end
126
127     default: begin
128       fsm_err_o = 1'b1;
129     end
130
131   endcase // unique case (state_q)
132 end
133
134
135
136 endmodule // keymgr_data_en_state

```

5. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_err.sv (153 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager error and fault collection
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_err
11   import keymgr_pkg::*;
12   import keymgr_reg_pkg::*;
13 (
14   input clk_i,
15   input rst_ni,
16
17   input invalid_op_i,
18   input disabled_i,
19   input invalid_i,
20   input kmac_input_invalid_i,
21   input shadowed_update_err_i,
22   input kmac_op_err_i,
23   input invalid_kmac_out_i,
24   input sideload_sel_err_i,
25   input kmac_cmd_err_i,
26   input kmac_fsm_err_i,
27   input kmac_done_err_i,
28   input regfile_intg_err_i,
29   input shadowed_storage_err_i,

```



```

30 input ctrl_fsm_err_i,
31 input data_fsm_err_i,
32 input op_fsm_err_i,
33 input ecc_err_i,
34 input state_change_err_i,
35 input op_state_cmd_err_i,
36 input cnt_err_i,
37 input reseed_cnt_err_i,
38 input sideload_fsm_err_i,
39
40 input op_update_i,
41 input op_done_i,
42
43 // The following outputs are very similar, but have slightly
different timing for
44 // for CDIs on sync errors/faults.
45 // Advance operations must go through for all CDIs.
46 // The sync_err/fault outputs register when any CDI com-
pletes and helps with
47 // the appropriate behavior on key state change.
48 // The sync_err_o/fault_o outputs on the other hand only
output when the entire
49 // operation is complete, which could be multiple CDIs.
50 output logic [SyncErrLastIdx-1:0] sync_err_o,
51 output logic [AsyncErrLastIdx-1:0] async_err_o,
52 output logic [SyncFaultLastIdx-1:0] sync_fault_o,
53 output logic [AsyncFaultLastIdx-1:0] async_fault_o,
54 output logic [ErrLastPos-1:0] error_o,
55 output logic [FaultLastPos-1:0] fault_o
56 );
57
58 // Advance calls are made up of multiple rounds of kmac op-
erations.
59 // Any sync error that occurs is treated as an error of the
entire call.
60 // Therefore sync errors that happen before the end of the
call must be
61 // latched.
62 logic[SyncErrLastIdx-1:0] sync_err_q, sync_err_d;
63 logic[SyncFaultLastIdx-1:0] sync_fault_q, sync_fault_d;
64
65 logic err_vld;
66 assign err_vld = op_update_i | op_done_i;
67
68 // sync errors
69 // When an operation encounters a fault, the operation is
always rejected as the FSM
70 // transitions to wipe. When an operation is ongoing and en
drops, it is also rejected.
71 assign sync_err_d[SyncErrInvalidOp] = err_vld & (inval-
id_op_i |
72                                     disabled_i
73 |
74                                     invalid_i |
75                                     (|
76                                     fault_o));
77 assign sync_err_d[SyncErrInvalidIn] = err_vld & kmac_in-
put_invalid_i;
78
79 always_ff @(posedge clk_i or negedge rst_ni) begin
80   if (!rst_ni) begin
81     sync_err_q <= '0;
82   end else if (op_done_i) begin
83     sync_err_q <= '0;
84   end else if (op_update_i) begin
85     sync_err_q <= sync_err_d;
86   end
87 end
88 assign sync_err_o = sync_err_q | sync_err_d;
89
90 // async errors
91 assign async_err_o[AsyncErrShadowUpdate] = shad-
owed_update_err_i;
92
93 // sync faults
94 assign sync_fault_d[SyncFaultKmacOp] = err_vld &
kmac_op_err_i;
95 assign sync_fault_d[SyncFaultKmacOut] = err_vld & inval-
id_kmac_out_i;
96 assign sync_fault_d[SyncFaultSideSel] = err_vld & side-
load_sel_err_i;
97
98 always_ff @(posedge clk_i or negedge rst_ni) begin
99   if (!rst_ni) begin
100     sync_fault_q <= '0;
101   end else if (op_done_i) begin
102     sync_fault_q <= '0;
103   end else if (op_update_i) begin
104     sync_fault_q <= sync_fault_d;
105   end
106 end
107 assign sync_fault_o = sync_fault_q | sync_fault_d;
108
109 // async faults
110 logic [AsyncFaultLastIdx-1:0] async_fault_q, async_fault_d;
111 always_ff @(posedge clk_i or negedge rst_ni) begin
112   if (!rst_ni) begin
113     async_fault_q <= '0;
114   end else begin
115     async_fault_q <= async_fault_o;
116   end
117 end
118
119 end
120
121 assign async_fault_o = async_fault_q | async_fault_d;
122 assign async_fault_d[AsyncFaultKmacCmd] = kmac_cmd_err_i;
123 assign async_fault_d[AsyncFaultKmacFsm] = kmac_fsm_err_i;
124 assign async_fault_d[AsyncFaultKmacDone] = kmac_done_err_i;
125 assign async_fault_d[AsyncFaultRegIntg] = reg-
file_intg_err_i;
126 assign async_fault_d[AsyncFaultShadow] = shad-
owed_storage_err_i;
127 assign async_fault_d[AsyncFaultFsmIntg] = ctrl_fsm_err_i |
data_fsm_err_i | op_fsm_err_i;
128 assign async_fault_d[AsyncFaultKeyEcc] = ecc_err_i;
129
130 // SEC_CM: CTRL.FSM.CONSTISTENCY
131 assign async_fault_d[AsyncFaultFsmChk] =
state_change_err_i | op_state_cmd_err_i;
132 assign async_fault_d[AsyncFaultCntErr] = cnt_err_i;
133 assign async_fault_d[AsyncFaultRCntErr] = reseed_cnt_err_i;
134 assign async_fault_d[AsyncFaultSideErr] = side-
load_fsm_err_i;
135
136 // certain errors/faults can only happen when there's an ac-
tual kmac transaction,
137 // others can happen with or without.
138 assign error_o[ErrInvalidOp] = op_done_i &
sync_err_o[SyncErrInvalidOp];
139 assign error_o[ErrInvalidIn] = op_done_i &
sync_err_o[SyncErrInvalidIn];
140 assign error_o[ErrShadowUpdate] =
async_err_o[AsyncErrShadowUpdate];
141
142 // output to fault code register
143 assign fault_o[FaultKmacOp] = op_done_i &
sync_fault_o[SyncFaultKmacOp];
144 assign fault_o[FaultKmacOut] = op_done_i &
sync_fault_o[SyncFaultKmacOut];
145 assign fault_o[FaultSideSel] = op_done_i &
sync_fault_o[SyncFaultSideSel];
146 assign fault_o[FaultKmacCmd] =
async_fault_o[AsyncFaultKmacCmd];
147 assign fault_o[FaultKmacFsm] =
async_fault_o[AsyncFaultKmacFsm];
148 assign fault_o[FaultKmacDone] =
async_fault_o[AsyncFaultKmacDone];
149 assign fault_o[FaultRegIntg] =
async_fault_o[AsyncFaultRegIntg];
150 assign fault_o[FaultShadow] =
async_fault_o[AsyncFaultShadow];
151 assign fault_o[FaultCtrlFsm] =
async_fault_o[AsyncFaultFsmIntg];
152 assign fault_o[FaultCtrlFsmChk] =
async_fault_o[AsyncFaultFsmChk];
153 assign fault_o[FaultCtrlCnt] =
async_fault_o[AsyncFaultCntErr];
154 assign fault_o[FaultReseedCnt] =
async_fault_o[AsyncFaultRCntErr];
155 assign fault_o[FaultSideFsm] =
async_fault_o[AsyncFaultSideErr];
156 assign fault_o[FaultKeyEcc] =
async_fault_o[AsyncFaultKeyEcc];
157
158 endmodule // keymgr_err

```

6. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_input_checks.sv (121 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager input checks
6 // Checks input data for errors
7
8 `include "prim_assert.sv"
9
10 // We should also check for input validity
11 module keymgr_input_checks import keymgr_pkg::*; #(
12   parameter bit KmacEnMasking = 1'b1,
13   parameter int unsigned NumRomDigestInputs = 1
14 ) (
15   input rom_ctrl_pkg::keymgr_data_t [NumRomDigestInputs-1:0]
rom_digest_i,
16   input [KeyVersionWidth-1:0] cur_max_key_version_i,
17   input hw_key_req_t key_i,
18   input [31:0] key_version_i,
19   input [KeyWidth-1:0] creator_seed_i,
20   input [KeyWidth-1:0] owner_seed_i,
21   input [DevidWidth-1:0] devid_i,
22   input [HealthStateWidth-1:0] health_state_i,
23   output logic creator_seed_vld_o,
24   output logic owner_seed_vld_o,
25   output logic devid_vld_o,
26   output logic health_state_vld_o,
27   output logic key_version_vld_o,
28   output logic key_vld_o,
29   output logic rom_digest_vld_o

```

```

30 );
31 // checks for all 0's or all 1's of value
32 function automatic logic valid_chk (logic [MaxWidth-1:0]
value);
33     return |value & ~&value;
34 endfunction // valid_chk
35
36 // key version must be smaller than or equal to max version
37 assign key_version_vld_o = key_version_i <= cur_max_key_ver-
sion_i;
38
39 // general data check
40 logic [MaxWidth-1:0] creator_seed_padded, owner_seed_padded,
devid_padded, health_state_padded;
41
42 prim_msb_extend #(
43     .InWidth(KeyWidth),
44     .OutWidth(MaxWidth)
45 ) u_creator_seed (
46     .in_i(creator_seed_i),
47     .out_o(creator_seed_padded)
48 );
49
50 prim_msb_extend #(
51     .InWidth(KeyWidth),
52     .OutWidth(MaxWidth)
53 ) u_owner_seed (
54     .in_i(owner_seed_i),
55     .out_o(owner_seed_padded)
56 );
57
58 prim_msb_extend #(
59     .InWidth(DevIdWidth),
60     .OutWidth(MaxWidth)
61 ) u_devid (
62     .in_i(devid_i),
63     .out_o(devid_padded)
64 );
65
66 prim_msb_extend #(
67     .InWidth(HealthStateWidth),
68     .OutWidth(MaxWidth)
69 ) u_health_state (
70     .in_i(health_state_i),
71     .out_o(health_state_padded)
72 );
73
74 assign creator_seed_vld_o = valid_chk(creator_seed_padded);
75 assign owner_seed_vld_o = valid_chk(owner_seed_padded);
76 assign devid_vld_o = valid_chk(devid_padded);
77 assign health_state_vld_o = valid_chk(health_state_padded);
78
79 // key check
80 logic unused_key_vld;
81 assign unused_key_vld = key_i.valid;
82
83 localparam int KeyShares = KmacEnMasking ? Shares : 1;
84 logic [KeyShares-1:0][MaxWidth-1:0] key_padded;
85 logic [KeyShares-1:0] key_chk;
86
87 for (genvar i = 0; i < KeyShares; i++) begin : gen_key_chk
88     prim_msb_extend #(
89         .InWidth(KeyWidth),
90         .OutWidth(MaxWidth)
91     ) u_key_pad (
92         .in_i(key_i.key[i]),
93         .out_o(key_padded[i])
94     );
95
96     assign key_chk[i] = valid_chk(key_padded[i]);
97 end
98
99 assign key_vld_o = &key_chk;
100
101 logic [NumRomDigestInputs-1:0][MaxWidth-1:0] rom_di-
gest_padded;
102
103 for (genvar k = 0; k < NumRomDigestInputs; k++) begin :
gen_digest_pad
104     prim_msb_extend #(
105         .InWidth($bits(rom_digest_i[k].data)),
106         .OutWidth(MaxWidth)
107     ) u_rom_digest_pad (
108         .in_i(rom_digest_i[k].data),
109         .out_o(rom_digest_padded[k])
110     );
111 end
112
113 always_comb begin
114     rom_digest_vld_o = 'b1;
115     for (int k = 0; k < NumRomDigestInputs; k++) begin
116         rom_digest_vld_o &= rom_digest_i[k].valid &&
valid_chk(rom_digest_padded[k]);
117     end
118 end
119
120
121 endmodule // keymgr_input_checks

```

7. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_kmac_if.sv (408 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager interface to kmac
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_kmac_if
11 import keymgr_pkg::*;
12 #(
13     parameter rand_perm_t RndCnstRandPerm = RndCnstRandPermDe-
fault,
14     parameter int MaxAdvDataWidth = AdvDataWidth
15 ) (
16     input clk_i,
17     input rst_ni,
18
19     // data input interfaces
20     input [MaxAdvDataWidth-1:0] adv_data_i,
21     input [IdDataWidth-1:0] id_data_i,
22     input [GenDataWidth-1:0] gen_data_i,
23     input [3:0] inputs_invalid_i,
24     output logic inputs_invalid_o,
25
26     // keymgr control to select appropriate inputs
27     input adv_en_i,
28     input id_en_i,
29     input gen_en_i,
30     output logic done_o,
31     output logic [Shares-1:0][kmac_pkg::AppDigestW-1:0] data_o,
32
33     // actual connection to kmac
34     output kmac_pkg::app_req_t kmac_data_o,
35     input kmac_pkg::app_rsp_t kmac_data_i,
36
37     // entropy input
38     output logic prng_en_o,
39     input [Shares-1:0][Randwidth-1:0] entropy_i,
40
41     // error outputs
42     output logic fsm_error_o,
43     output logic kmac_error_o,
44     output logic kmac_done_error_o,
45     output logic cmd_error_o
46 );
47
48
49 // Encoding generated with:
50 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 6 -n 10 \
51 // -s 2292624416 --language=sv
52 //
53 // Hamming distance histogram:
54 //
55 // 0: --
56 // 1: --
57 // 2: --
58 // 3: --
59 // 4: --
60 // 5: ||||| (46.67%)
61 // 6: ||||| (40.00%)
62 // 7: |||| (13.33%)
63 // 8: --
64 // 9: --
65 // 10: --
66 //
67 // Minimum Hamming distance: 5
68 // Maximum Hamming distance: 7
69 // Minimum Hamming weight: 2
70 // Maximum Hamming weight: 9
71 //
72 localparam int StateWidth = 10;
73 typedef enum logic [StateWidth-1:0] {
74     StIdle = 10'b110100010,
75     StTx = 10'b0010011011,
76     StTxLast = 10'b0101000000,
77     StOpWait = 10'b1000101001,
78     StClean = 10'b111111101,
79     StError = 10'b001101110
80 } data_state_e;
81
82 localparam int AdvRem = MaxAdvDataWidth % KmacDataIfWidth;
83 localparam int IdRem = IdDataWidth % KmacDataIfWidth;
84 localparam int GenRem = GenDataWidth % KmacDataIfWidth;
85
86 // the remainder must be in number of bytes
87 `ASSERT_INIT(AdvRemBytes_A, AdvRem % 8 == 0)
88 `ASSERT_INIT(IdRemBytes_A, IdRem % 8 == 0)
89 `ASSERT_INIT(GenRemBytes_A, GenRem % 8 == 0)
90
91 // Number of kmac transactions required
92 localparam int AdvRounds = (MaxAdvDataWidth +
KmacDataIfWidth - 1) / KmacDataIfWidth;

```



```

93  localparam int IdRounds = (IdDataWidth + KmacDataIfWidth -
1) / KmacDataIfWidth;
94  localparam int GenRounds = (GenDataWidth + KmacDataIfWidth -
1) / KmacDataIfWidth;
95  localparam int MaxRounds = KDFMaxWidth / KmacDataIfWidth;
96
97  // calculated parameters for number of rounds and interface
width
98  localparam int CntWidth = $clog2(MaxRounds);
99  localparam int IfBytes = KmacDataIfWidth / 8;
100 localparam int DecoyCopies = KmacDataIfWidth / RandWidth;
101 localparam int DecoyOutputCopies = (kmac_pkg::AppDigestW /
RandWidth);
102
103 localparam int unsigned LastAdvRoundInt = AdvRounds - 1;
104 localparam int unsigned LastIdRoundInt = IdRounds - 1;
105 localparam int unsigned LastGenRoundInt = GenRounds - 1;
106 localparam bit [CntWidth-1:0] LastAdvRound = LastAdvRound-
Int[CntWidth-1:0];
107 localparam bit [CntWidth-1:0] LastIdRound = LastIdRound-
Int[CntWidth-1:0];
108 localparam bit [CntWidth-1:0] LastGenRound = LastGenRound-
Int[CntWidth-1:0];
109
110 // byte mask for the last transfer
111 localparam logic [IfBytes-1:0] AdvByteMask = (AdvRem > 0) ?
(2**(AdvRem/8)-1) : {IfBytes{1'b1}};
112 localparam logic [IfBytes-1:0] IdByteMask = (IdRem > 0) ?
(2**(IdRem/8)-1) : {IfBytes{1'b1}};
113 localparam logic [IfBytes-1:0] GenByteMask = (GenRem > 0) ?
(2**(GenRem/8)-1) : {IfBytes{1'b1}};
114
115 logic [MaxRounds-1:0][KmacDataIfWidth-1:0] adv_data;
116 logic [MaxRounds-1:0][KmacDataIfWidth-1:0] id_data;
117 logic [MaxRounds-1:0][KmacDataIfWidth-1:0] gen_data;
118 logic [CntWidth-1:0] cnt;
119 logic [CntWidth-1:0] rounds;
120 logic [KmacDataIfWidth-1:0] decoy_data;
121 logic valid;
122 logic last;
123 logic [IfBytes-1:0] strb;
124 logic cnt_clr, cnt_set, cnt_en;
125 logic start;
126 logic [3:0] inputs_invalid_d, inputs_invalid_q;
127 logic clr_err;
128 logic kmac_done_vld;
129 logic cmd_chk;
130
131 data_state_e state_q, state_d;
132
133 // 0 pad to the appropriate width
134 // this is basically for scenarios where *DataWidth %
KmacDataIfWidth != 0
135 assign adv_data = KDFMaxWidth'(adv_data_i);
136 assign id_data = KDFMaxWidth'(id_data_i);
137 assign gen_data = KDFMaxWidth'(gen_data_i);
138
139 assign start = adv_en_i | id_en_i | gen_en_i;
140
141 logic cnt_err;
142 // SEC_CM: KMAC_IF.CTR.REDUN
143 prim_count #(
144   .Width(CntWidth),
145   .ResetValue({CntWidth{1'b1}})
146 ) u_cnt (
147   .clk_i,
148   .rst_ni,
149   .clr_i(cnt_clr),
150   .set_i(cnt_set),
151   .set_cnt_i(rounds),
152   .incr_en_i(1'b0),
153   .decr_en_i(cnt_en),
154   .step_i(CntWidth'(1'b1)),
155   .commit_i(1'b1),
156   .cnt_o(cnt),
157   .cnt_after_commit_o(),
158   .err_o(cnt_err)
159 );
160
161 always_ff @(posedge clk_i or negedge rst_ni) begin
162   if (!rst_ni) begin
163     inputs_invalid_q <= '0;
164   end else begin
165     inputs_invalid_q <= inputs_invalid_d;
166   end
167 end
168
169 // SEC_CM: KMAC_IF.FSM.SPARSE
170 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
data_state_e, StIdle)
171
172 always_comb begin
173   cnt_clr = 1'b0;
174   cnt_set = 1'b0;
175   cnt_en = 1'b0;
176   valid = 1'b0;
177   last = 1'b0;
178   strb = '0;
179   done_o = 1'b0;
180   state_d = state_q;
181
182   rounds = '0;
183   clr_err = '0;
184   fsm_error_o = '0;
185   kmac_error_o = '0;
186
187   kmac_done_vld = '0;
188
189   cmd_chk = 1'b1;
190
191   unique case (state_q)
192     StIdle: begin
193       // if for some reason multiple bits are set, adv_en
has priority
194       // as the current key state will be destroyed
195       // cross check for commands once transaction begins
196       cmd_chk = '0;
197       if (start) begin
198         cnt_set = 1'b1;
199         if (adv_en_i) begin
200           rounds = LastAdvRound;
201         end else if (id_en_i) begin
202           rounds = LastIdRound;
203         end else if (gen_en_i) begin
204           rounds = LastGenRound;
205         end
206       // in case we are sending only 1 entry
207       state_d = (rounds == 0) ? StTxLast : StTx;
208     end
209   end
210
211   StTx: begin
212     valid = 1'b1;
213     strb = {IfBytes{1'b1}};
214
215     // transaction accepted
216     if (kmac_data_i.req_ready) begin
217       cnt_en = 1'b1;
218
219       // second to last beat
220       if (cnt == CntWidth'(1'b1)) begin
221         state_d = StTxLast;
222       end
223     end
224
225     end
226
227   StTxLast: begin
228     valid = 1'b1;
229     last = 1'b1;
230
231     if (adv_en_i) begin
232       strb = AdvByteMask;
233     end else if (id_en_i) begin
234       strb = IdByteMask;
235     end else if (gen_en_i) begin
236       strb = GenByteMask;
237     end
238
239     // transaction accepted
240     cnt_clr = kmac_data_i.req_ready;
241     state_d = kmac_data_i.req_ready ? StOpWait : StTxLast;
242   end
243
244   StOpWait: begin
245     kmac_done_vld = 1'b1;
246     if (kmac_data_i.rsp_valid) begin
247       kmac_error_o = kmac_data_i.error;
248       done_o = 1'b1;
249       state_d = StClean;
250     end
251   end
252
253   StClean: begin
254     cmd_chk = '0;
255     done_o = 1'b1;
256
257     // wait for control side to ack done by waiting start
de-assertion
258     if (!start) begin
259       done_o = 1'b0;
260       clr_err = 1'b1;
261       state_d = StIdle;
262     end
263   end
264
265   // trigger error
266   default: begin
267     // This state is terminal
268     done_o = 1'b1;
269     fsm_error_o = 1'b1;
270   end
271 endcase // unique case (state_q)
272
273 // unconditional error transitions
274 // counter errors may disturb the fsm flow and are
275 // treated like fsm errors

```

```

280     if (cnt_err) begin
281         state_d = StError;
282         fsm_error_o = 1;
283         done_o = 1'b1;
284     end
285 end
286
287 // when transaction is not complete, populate the data with
random
288 assign data_o = start && done_o ?
289     {kmac_data_i.digest_s1,
290      kmac_data_i.digest_s0} :
291     {{DecoyOutputCopies{entropy_i[1]}},
292      {DecoyOutputCopies{entropy_i[0]}}};
293
294 // The input invalid check is done whenever transactions are
ongoing with kmac
295 // once set, it cannot be unset until transactions are fully
complete
296 always_comb begin
297     inputs_invalid_d = inputs_invalid_q;
298
299     if (clr_err) begin
300         inputs_invalid_d = '0;
301     end else if (valid) begin
302         inputs_invalid_d[OpAdvance] = adv_en_i & (in-
puts_invalid_i[OpAdvance] |
303                                     in-
puts_invalid_q[OpAdvance]);
304         inputs_invalid_d[OpGenId] = id_en_i & (in-
puts_invalid_i[OpGenId] |
305                                     in-
puts_invalid_q[OpGenId]);
306         inputs_invalid_d[OpGenSwOut] = gen_en_i & (in-
puts_invalid_i[OpGenSwOut] |
307                                     in-
puts_invalid_q[OpGenSwOut]);
308         inputs_invalid_d[OpGenHwOut] = gen_en_i & (in-
puts_invalid_i[OpGenHwOut] |
309                                     in-
puts_invalid_q[OpGenHwOut]);
310     end
311 end
312
313 // immediately assert errors
314 assign inputs_invalid_o = |inputs_invalid_d;
315
316 // Permute Share 1 of the entropy input once more to get the
decoy data.
317 // Share 0 and 1 are used as is for data_o (connected to the
sideload ports).
318 logic [RandWidth-1:0] decoy_entropy;
319 assign decoy_entropy = perm_data(entropy_i[1], RndCnstRand-
Perm);
320 assign decoy_data = {DecoyCopies{decoy_entropy}};
321
322 logic [CntWidth-1:0] adv_sel, id_sel, gen_sel;
323 assign adv_sel = LastAdvRound - cnt;
324 assign id_sel = LastIdRound - cnt;
325 assign gen_sel = LastGenRound - cnt;
326
327 // The count is maintained as a downcount
328 // so a subtract is necessary to send the right byte
329 // alternatively we can also reverse the order of the input
330 always_comb begin
331     kmac_data_o.data_s0 = decoy_data;
332     if (|cmd_error_o || inputs_invalid_o || fsm_error_o) begin
333         kmac_data_o.data_s0 = decoy_data;
334     end else if (valid && adv_en_i) begin
335         kmac_data_o.data_s0 = adv_data[adv_sel];
336     end else if (valid && id_en_i) begin
337         kmac_data_o.data_s0 = id_data[id_sel];
338     end else if (valid && gen_en_i) begin
339         kmac_data_o.data_s0 = gen_data[gen_sel];
340     end
341 end
342
343 // Key manager operates on unshared data and accepts digest
responses immediately.
344 assign kmac_data_o.data_s1 = '0;
345 assign kmac_data_o.req_valid = valid;
346 assign kmac_data_o.req_last = last;
347 assign kmac_data_o.strb = strb;
348 assign kmac_data_o.rsp_ready = 1'b1;
349
350 // A static interface has no finish response.
351 logic unused_kmac_finish_rsp;
352 assign unused_kmac_finish_rsp = kmac_data_i.rsp_finish;
353
354 // kmac done is asserted outside of expected window
355 // SEC_CM: KMAC_IF_DONE.CTRL.CONSISTENCY
356 logic kmac_done_err_q, kmac_done_err_d;
357 assign kmac_done_err_d = ~kmac_done_vld &
kmac_data_i.rsp_valid |
358                                     kmac_done_err_q;
359 assign kmac_done_error_o = kmac_done_err_q;
360

```

```

361
362 // the enables must be 1 hot
363 logic [2:0] enables_d, enables_q, enables_sub;
364 assign enables_d = {adv_en_i, id_en_i, gen_en_i};
365 assign enables_sub = enables_d - 1'b1;
366
367 // cross check to ensure the one-hot command that kicked off
368 // the transaction remains consistent throughout.
369 logic cmd_consty_err_q, cmd_consty_err_d;
370 always_ff @(posedge clk_i or negedge rst_ni) begin
371     if (!rst_ni) begin
372         enables_q <= '0;
373     end else if (cnt_set) begin
374         enables_q <= enables_d;
375     end
376 end
377 assign cmd_consty_err_d = (cmd_chk & (enables_q != en-
ables_d)) |
378                                     cmd_consty_err_q;
379
380 // if a one hot error occurs, latch onto it permanently
381 // SEC_CM: KMAC_IF_CMD.CTRL.CONSISTENCY
382 logic one_hot_err_q, one_hot_err_d;
383 assign one_hot_err_d = |(enables_d & enables_sub) |
one_hot_err_q;
384
385 always_ff @(posedge clk_i or negedge rst_ni) begin
386     if (!rst_ni) begin
387         one_hot_err_q <= '0;
388         kmac_done_err_q <= '0;
389         cmd_consty_err_q <= '0;
390     end else begin
391         one_hot_err_q <= one_hot_err_d;
392         kmac_done_err_q <= kmac_done_err_d;
393         cmd_consty_err_q <= cmd_consty_err_d;
394     end
395 end
396
397 // command error occurs if kmac errors or if the command it-
self is invalid
398 assign cmd_error_o = one_hot_err_q | cmd_consty_err_q;
399
400 // request entropy to churn whenever a transaction is accep-
ted
401 assign prng_en_o = kmac_data_o.req_valid &
kmac_data_i.req_ready;
402
403 // as long as we are transmitting, the strobe should never
be 0.
404 `ASSERT(LastStrb_A, valid |-> strb != '0)
405
406
407
408 endmodule // keymgr_kmac_if

```

8. lowRISC_opentitan/hw/ip/keymgr/rtl/ keymgr_op_state_ctrl.sv (110 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Key manager operation state control
6 //
7
8 `include "prim_assert.sv"
9
10 module keymgr_op_state_ctrl
11     import keymgr_pkg::*;
12     import keymgr_reg_pkg::*;
13 (
14     input clk_i,
15     input rst_ni,
16
17     input adv_req_i,
18     input dis_req_i,
19     input id_req_i,
20     input gen_req_i,
21     input [CdiWidth-1:0] cnt_i,
22     output logic op_ack_o,
23     output logic op_busy_o,
24     output logic op_update_o,
25
26     input kmac_done_i,
27     output logic adv_en_o,
28     output logic id_en_o,
29     output logic gen_en_o,
30
31     output logic op_fsm_err_o
32 );
33
34 localparam int OpStateWidth = 8;
35 typedef enum logic [OpStateWidth-1:0] {
36     StIdle = 8'b10010101,
37     StAdv = 8'b00101000,
38     StAdvAck = 8'b01000011,
39     StWait = 8'b11111110
40 } state_e;

```

```

42
43 state_e state_q, state_d;
44 PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, StIdle)
45
46 logic gen_en;
47 assign id_en_o = gen_en & id_req_i;
48 assign gen_en_o = gen_en & gen_req_i;
49
50 always_comb begin
51 state_d = state_q;
52 op_update_o = 1'b0;
53 op_ack_o = 1'b0;
54 op_busy_o = 1'b1;
55
56 // output to kmac interface
57 adv_en_o = 1'b0;
58
59 gen_en = 1'b0;
60 op_fsm_err_o = 1'b0;
61
62 unique case (state_q)
63 StIdle: begin
64 op_busy_o = '0;
65 if (adv_req_i || dis_req_i) begin
66 state_d = StAdv;
67 end else if (id_req_i || gen_req_i) begin
68 state_d = StWait;
69 end
70 end
71
72 StAdv: begin
73 adv_en_o = 1'b1;
74
75 if (kmac_done_i && (int'(cnt_i) == CDIs-1)) begin
76 op_ack_o = 1'b1;
77 state_d = StIdle;
78 end else if (kmac_done_i && (int'(cnt_i) < CDIs-1))
begin
79 op_update_o = 1'b1;
80 state_d = StAdvAck;
81 end
82 end
83
84 // drop adv_en_o to allow kmac interface handshake
85 StAdvAck: begin
86 state_d = StAdv;
87 end
88
89 // Not an advanced operation
90 StWait: begin
91 gen_en = 1'b1;
92
93 if (kmac_done_i) begin
94 op_ack_o = 1'b1;
95 state_d = StIdle;
96 end
97 end
98
99 // error state
100 default: begin
101 // allow completion of transaction
102 op_ack_o = 1'b1;
103 op_fsm_err_o = 1'b1;
104 end
105
106 endcase // unique case (adv_state_q)
107 end
108
109
110 endmodule // keymgr_op_state_ctrl

```

9. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_pkg.v (279 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // key manager package
6 //
7
8 package keymgr_pkg;
9
10 parameter int KeyWidth = 256;
11 parameter int CDIs = 2; // 2 different CDIs, sealing / at-
testation
12 parameter int CdiWidth = prim_util_pkg::vbits(CDIs);
13 parameter int OtbnKeyWidth = 384;
14 parameter int DigestWidth = 128; // uses truncated hash
15 parameter int KmacDataIfWidth = 64; // KMAC interface data
width
16 parameter int KeyMgrStages = 3; // Number of key manager
stages (creator, ownerInt, owner)
17 parameter int SwBindingWidth = 32 * keym-
gr_reg_pkg::NumSwBindingReg;
18 parameter int SaltWidth = 32 * keymgr_reg_pkg::NumSaltReg;
19 parameter int Shares = 2; // number of key shares

```

```

20 parameter int KeyVersionWidth = 32; // Key version length
for individual DICE stage
21
22 // These should be defined in another module's package
23 parameter int HealthStateWidth = 128;
24 parameter int DevIdWidth = 256;
25 parameter int MaxWidth = 256;
26
27 // Default seeds
28 // These have been generated with the following command by
incrementing the --seed argument
29 // for every seed.
30 // util/design/gen-lfsr-seed.py --width 256 --seed 7535190
--prefix ""
31 typedef logic [KeyWidth-1:0] seed_t;
32 parameter seed_t RndCnstRevisionSeedDefault =
33
34 256'h69802e51_bacf8874_e650d692_e3d8a646_2d3f158f_0bf7961d_d346f880_
b4d52170;
35 parameter seed_t RndCnstCreatorIdentitySeedDefault =
36
37 256'h344bfff84_d3edf83e_aa565c5e_6c2d22f2_75283734_15cf6d83_32cd3752_
731817c;
38 parameter seed_t RndCnstOwnerIntIdentitySeedDefault =
39
40 256'h32a37b-
f9_2627382b_47180548_1fc4b847_a7eba1ce_c13e80ab_030f847e_9bd902e;
41 parameter seed_t RndCnstOwnerIdentitySeedDefault =
42
43 256'hc1ba416c_822ad676_e4f54f36_33d8dc7e_00ccfb8b_92209f1b_d3229348_
275ebda;
44 parameter seed_t RndCnstSoftOutputSeedDefault =
45
46 256'h23cbe85e_62e39992_4ab8d6a6_acbf12f0_0ca6f488_63eaa428_ffb11b26_
c5282b14;
47 parameter seed_t RndCnstHardOutputSeedDefault =
48
49 256'h877e-
f0b9_aceaeefc5_693b1aa7_e43a7f3d_ee7d63b6_4e73e182_b49cbf87_2872f2c;
50 // Target based derivation seeds
51 // These are used during the generation stages for sideload
52 parameter seed_t RndCnstNoneSeedDefault =
53
54 256'h6013ba4d_8d04fe4f_80cb0472_536c9679_08b82f1d_98a4e405_cb4680ec_
3a4a8d7a;
55 parameter seed_t RndCnstAesSeedDefault =
56
57 256'he16a8-
fa9_5b613cd5_fb9ad23f_bd8347e1_64e45dac_5d08a41b_e83caa37_e03d9482;
58 parameter seed_t RndCnstKmacSeedDefault =
59
60 256'hc57f4c0b_b308e83f_3fc4bc63_d87dd67d_9071dc1c_e19484c8_3c94fb97_
dd634369;
61 parameter seed_t RndCnstOtbnSeedDefault =
62
63 256'hcb-
cb4d2d_0abeb81b_ca7451ae_d1e2479d_ba13530a_d046b945_646aa127_bd4f6a3
8;
64 parameter seed_t RndCnstCdiDefault =
65
66 256'h54180905_d14c1d2f_2dda1522_f332bc0e_fcd6b92f_f0f9db75_3a9a9544_
26a42eab;
67 // Default Lfsr configurations
68 // These LFSR parameters have been generated with
69 // $ util/design/gen-lfsr-seed.py --width 64 --seed
691876113 --prefix ""
70 parameter int LfsrWidth = 64;
71 typedef logic [LfsrWidth-1:0] lfsr_seed_t;
72 typedef logic [LfsrWidth-1:0][$clog2(LfsrWidth)-1:0]
lfsr_perm_t;
73 parameter lfsr_seed_t RndCnstLfsrSeedDefault =
74
75 'h22d326255bd24320;
76 parameter lfsr_perm_t RndCnstLfsrPermDefault = {
77
78 128'h16108c9f9008aa37e5118d1dec1df64a7,
79 256'h24f3f1b73537f42d38383ee8f897286df81d49-
ab54b6bbb666cbd1a16c41252
80 };
81 // Random permutation
82 parameter int RandWidth = LfsrWidth / 2;
83 typedef logic [RandWidth-1:0][$clog2(RandWidth)-1:0]
rand_perm_t;
84 parameter rand_perm_t RndCnstRandPermDefault = {
85
86 160'h62089181d2a6be2ce145e2e27099ededbd7dceb0
87 };
88 // Width calculations
89 // These are the largest calculations in use across all
stages
90 parameter int AdvDataWidth = SwBindingWidth + 2*KeyWidth +
DevIdWidth + HealthStateWidth;
91 parameter int IdDataWidth = KeyWidth;
92 // key version + salt + key ID + constant
93 parameter int GenDataWidth = 32 + SaltWidth + KeyWidth*2;
94 parameter int StageWidth = $clog2(KeyMgrStages);
95 // Max Payload Width to derivation function
96 // see security strength description https://keccak.team/
keccak.html
97 // The max width here is chosen arbitrarily to ensure we do
not get out of hand.
98
99 // Since KMAC is a MAC operation, the data can be as long as we
need.

```

```

88 parameter int KDFMaxWidth = 1984;
89
90 // Enumeration for operations
91 typedef enum logic [1:0] {
92     Creator,
93     OwnerInt,
94     Owner,
95     Disable
96 } keymgr_stage_e;
97
98 // Enumeration for sideload sel
99 typedef enum logic [1:0] {
100     None,
101     Aes,
102     Kmac,
103     Otbn
104 } keymgr_key_dest_e;
105
106 // Enumeration for actual key slot idx
107 typedef enum logic [1:0] {
108     AesIdx,
109     KmacIdx,
110     OtbnIdx,
111     LastIdx
112 } keymgr_sideload_slot_idx_e;
113
114 // Enumeration for key select
115 typedef enum logic {
116     HwKey = 0,
117     SwKey = 1
118 } keymgr_gen_out_e;
119
120 // Enumeration for operation
121 typedef enum logic [2:0] {
122     OpAdvance = 0,
123     OpGenId = 1,
124     OpGenSwOut = 2,
125     OpGenHwOut = 3,
126     OpDisable = 4
127 } keymgr_ops_e;
128
129 // Enumeration for working state exposed to software
130 typedef enum logic [2:0] {
131     StReset,
132     StInit,
133     StCreatorRootKey,
134     StOwnerIntKey,
135     StOwnerKey,
136     StDisabled,
137     StInvalid
138 } keymgr_working_state_e;
139
140 // Enumeration for operation status
141 typedef enum logic [1:0] {
142     OpIdle = 0,
143     OpWip = 1,
144     OpDoneSuccess = 2,
145     OpDoneFail = 3
146 } keymgr_op_status_e;
147
148 // keymgr has 4 categories of errors
149 // sync errors - recoverable errors that happen during
keymgr operation
150 // async errors - recoverable errors that happen asynchron-
ously
151 // sync faults - fatal errors that happen during keymgr op-
eration
152 // async faults - fatal errors that happen asynchronously
153
154 typedef enum logic [1:0] {
155     SyncErrInvalidOp,
156     SyncErrInvalidIn,
157     SyncErrLastIdx
158 } keymgr_sync_error_e;
159
160 typedef enum logic [1:0] {
161     AsyncErrShadowUpdate,
162     AsyncErrLastIdx
163 } keymgr_async_error_e;
164
165 typedef enum logic [1:0] {
166     SyncFaultKmacOp,
167     SyncFaultKmacOut,
168     SyncFaultSideSel,
169     SyncFaultLastIdx
170 } keymgr_sync_fault_e;
171
172 typedef enum logic [3:0] {
173     AsyncFaultKmacCmd,
174     AsyncFaultKmacFsm,
175     AsyncFaultKmacDone,
176     AsyncFaultRegIntg,
177     AsyncFaultShadow,
178     AsyncFaultFsmIntg,
179     AsyncFaultFsmChk,
180     AsyncFaultCntErr,
181     AsyncFaultRCntErr,
182     AsyncFaultSideErr,
183     AsyncFaultKeyEcc,
184     AsyncFaultLastIdx
185 } keymgr_async_fault_e;

```

```

186
187
188 // Bit position of error code
189 // Error is encoded as 1 error per bit
190 typedef enum logic [2:0] {
191     ErrInvalidOp,
192     ErrInvalidIn,
193     ErrShadowUpdate,
194     ErrLastPos
195 } keymgr_err_pos_e;
196
197 // Bit position of fault status
198 typedef enum logic [3:0] {
199     FaultKmacCmd,
200     FaultKmacFsm,
201     FaultKmacDone,
202     FaultKmacOp,
203     FaultKmacOut,
204     FaultRegIntg,
205     FaultShadow,
206     FaultCtrlFsm,
207     FaultCtrlFsmChk,
208     FaultCtrlCnt,
209     FaultReseedCnt,
210     FaultSideFsm,
211     FaultSideSel,
212     FaultKeyEcc,
213     FaultLastPos
214 } keymgr_fault_pos_e;
215
216 typedef enum logic [2:0] {
217     KeyUpdateIdle,
218     KeyUpdateRandom,
219     KeyUpdateRoot,
220     KeyUpdateKmac,
221     KeyUpdateWipe
222 } keymgr_key_update_e;
223
224 typedef enum logic [2:0] {
225     SideloadClrIdle,
226     SideloadClrAes,
227     SideloadClrKmac,
228     SideloadClrOtbn
229 } keymgr_sideload_clr_e;
230
231 // Key connection to various symmetric modules
232 typedef struct packed {
233     logic valid;
234     logic [Shares-1:0][KeyWidth-1:0] key;
235 } hw_key_req_t;
236
237 // Key connection to otbn
238 typedef struct packed {
239     logic valid;
240     logic [Shares-1:0][OtbnKeyWidth-1:0] key;
241 } otbn_key_req_t;
242
243 parameter hw_key_req_t HW_KEY_REQ_DEFAULT = '{
244     valid: 1'b0,
245     key: {Shares{KeyWidth'(32'hDEADBEEF)}}
246 };
247
248 parameter otbn_key_req_t OTBN_KEY_REQ_DEFAULT = '{
249     valid: 1'b0,
250     key: {Shares{OtbnKeyWidth'(32'hDEADBEEF)}}
251 };
252
253 // The following structs should be sourced from other mod-
ules
254 // defined here temporarily
255
256 // lc keymgr enable usage
257 typedef enum logic [1:0] {
258     KeyMgrEnCtrl,
259     KeyMgrEnCfgEn,
260     KeyMgrEnSwBindingEn,
261     KeyMgrEnLast
262 } keymgr_lc_en_usage_e;
263
264 // perm_data
265 function automatic logic[RandWidth-1:0] perm_data (logic
[RandWidth-1:0] data,
266     rand_perm_t perm_sel);
267
268     for (int k = 0; k < 32; k++) begin : gen_data_loop
269         perm_data[k] = data[perm_sel[k]];
270     end
271
272 endfunction
273
274 // checks for all 0's or all 1's of value
275 function automatic logic valid_data_chk (logic [Key-
width-1:0] value);
276     return |value & ~&value;
277 endfunction
278
279 endpackage : keymgr_pkg

```

10. lowRISC_opentitan/hw/ip/keymgr/rtl/keymgr_reg_pkg.sv (547 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Package auto-generated by `reggen` containing data
  structure
6
7 package keymgr_reg_pkg;
8
9 // Param list
10 parameter int NumSaltReg = 8;
11 parameter int NumSwBindingReg = 8;
12 parameter int NumOutReg = 8;
13 parameter int NumKeyVersion = 1;
14 parameter int NumAlerts = 2;
15
16 // Address widths within the block
17 parameter int BlockAw = 8;
18
19 // Number of registers for every interface
20 parameter int NumRegs = 63;
21
22 // Alert indices
23 typedef enum int {
24     AlertRecovOperationErrIdx = 0,
25     AlertFatalFaultErrIdx = 1
26 } keymgr_alert_idx_t;
27
28 ///////////////////////////////////////////////////
29 // Typedefs for registers //
30 ///////////////////////////////////////////////////
31
32 typedef struct packed {
33     logic q;
34 } keymgr_reg2hw_intr_state_reg_t;
35
36 typedef struct packed {
37     logic q;
38 } keymgr_reg2hw_intr_enable_reg_t;
39
40 typedef struct packed {
41     logic q;
42     logic qe;
43 } keymgr_reg2hw_intr_test_reg_t;
44
45 typedef struct packed {
46     struct packed {
47         logic q;
48         logic qe;
49     } fatal_fault_err;
50     struct packed {
51         logic q;
52         logic qe;
53     } recov_operation_err;
54 } keymgr_reg2hw_alert_test_reg_t;
55
56 typedef struct packed {
57     logic q;
58 } keymgr_reg2hw_start_reg_t;
59
60 typedef struct packed {
61     struct packed {
62         logic [1:0] q;
63     } dest_sel;
64     struct packed {
65         logic q;
66     } cdi_sel;
67     struct packed {
68         logic [2:0] q;
69     } operation;
70 } keymgr_reg2hw_control_shadowed_reg_t;
71
72 typedef struct packed {
73     logic [2:0] q;
74 } keymgr_reg2hw_sideload_clear_reg_t;
75
76 typedef struct packed {
77     logic [15:0] q;
78 } keymgr_reg2hw_reseed_interval_shadowed_reg_t;
79
80 typedef struct packed {
81     logic q;
82     logic qe;
83 } keymgr_reg2hw_sw_binding_regwen_reg_t;
84
85 typedef struct packed {
86     logic [31:0] q;
87 } keymgr_reg2hw_sealing_sw_binding_mreg_t;
88
89 typedef struct packed {
90     logic [31:0] q;
91 } keymgr_reg2hw_attest_sw_binding_mreg_t;
92
93 typedef struct packed {
94     logic [31:0] q;
95 } keymgr_reg2hw_salt_mreg_t;
96
97 typedef struct packed {
98     logic [31:0] q;
99 } keymgr_reg2hw_key_version_mreg_t;
100
101 typedef struct packed {
102     logic [31:0] q;
103 } keymgr_reg2hw_max_creator_key_ver_shadowed_reg_t;
104
105 typedef struct packed {
106     logic [31:0] q;
107 } keymgr_reg2hw_max_owner_int_key_ver_shadowed_reg_t;
108
109 typedef struct packed {
110     logic [31:0] q;
111 } keymgr_reg2hw_max_owner_key_ver_shadowed_reg_t;
112
113 typedef struct packed {
114     struct packed {
115         logic q;
116     } key_ecc;
117     struct packed {
118         logic q;
119     } side_ctrl_sel;
120     struct packed {
121         logic q;
122     } side_ctrl_fsm;
123     struct packed {
124         logic q;
125     } reseed_cnt;
126     struct packed {
127         logic q;
128     } ctrl_fsm_cnt;
129     struct packed {
130         logic q;
131     } ctrl_fsm_chk;
132     struct packed {
133         logic q;
134     } ctrl_fsm_intg;
135     struct packed {
136         logic q;
137     } shadow;
138     struct packed {
139         logic q;
140     } regfile_intg;
141     struct packed {
142         logic q;
143     } kmac_out;
144     struct packed {
145         logic q;
146     } kmac_op;
147     struct packed {
148         logic q;
149     } kmac_done;
150     struct packed {
151         logic q;
152     } kmac_fsm;
153     struct packed {
154         logic q;
155     } cmd;
156 } keymgr_reg2hw_fault_status_reg_t;
157
158 typedef struct packed {
159     logic d;
160     logic de;
161 } keymgr_hw2reg_intr_state_reg_t;
162
163 typedef struct packed {
164     logic d;
165 } keymgr_hw2reg_cfg_regwen_reg_t;
166
167 typedef struct packed {
168     logic d;
169     logic de;
170 } keymgr_hw2reg_start_reg_t;
171
172 typedef struct packed {
173     logic d;
174 } keymgr_hw2reg_sw_binding_regwen_reg_t;
175
176 typedef struct packed {
177     logic [31:0] d;
178     logic de;
179 } keymgr_hw2reg_sw_share0_output_mreg_t;
180
181 typedef struct packed {
182     logic [31:0] d;
183     logic de;
184 } keymgr_hw2reg_sw_share1_output_mreg_t;
185
186 typedef struct packed {
187     logic [2:0] d;
188     logic de;
189 } keymgr_hw2reg_working_state_reg_t;
190
191 typedef struct packed {
192     logic [1:0] d;
193     logic de;

```



```

194 } keymgr_hw2reg_op_status_reg_t;
195
196 typedef struct packed {
197     struct packed {
198         logic d;
199         logic de;
200     } invalid_shadow_update;
201     struct packed {
202         logic d;
203         logic de;
204     } invalid_kmac_input;
205     struct packed {
206         logic d;
207         logic de;
208     } invalid_op;
209 } keymgr_hw2reg_err_code_reg_t;
210
211 typedef struct packed {
212     struct packed {
213         logic d;
214         logic de;
215     } key_ecc;
216     struct packed {
217         logic d;
218         logic de;
219     } side_ctrl_sel;
220     struct packed {
221         logic d;
222         logic de;
223     } side_ctrl_fsm;
224     struct packed {
225         logic d;
226         logic de;
227     } reseed_cnt;
228     struct packed {
229         logic d;
230         logic de;
231     } ctrl_fsm_cnt;
232     struct packed {
233         logic d;
234         logic de;
235     } ctrl_fsm_chk;
236     struct packed {
237         logic d;
238         logic de;
239     } ctrl_fsm_intg;
240     struct packed {
241         logic d;
242         logic de;
243     } shadow;
244     struct packed {
245         logic d;
246         logic de;
247     } regfile_intg;
248     struct packed {
249         logic d;
250         logic de;
251     } kmac_out;
252     struct packed {
253         logic d;
254         logic de;
255     } kmac_op;
256     struct packed {
257         logic d;
258         logic de;
259     } kmac_done;
260     struct packed {
261         logic d;
262         logic de;
263     } kmac_fsm;
264     struct packed {
265         logic d;
266         logic de;
267     } cmd;
268 } keymgr_hw2reg_fault_status_reg_t;
269
270 typedef struct packed {
271     struct packed {
272         logic d;
273         logic de;
274     } invalid_digest;
275     struct packed {
276         logic d;
277         logic de;
278     } invalid_key;
279     struct packed {
280         logic d;
281         logic de;
282     } invalid_key_version;
283     struct packed {
284         logic d;
285         logic de;
286     } invalid_health_state;
287     struct packed {
288         logic d;
289         logic de;
290     } invalid_dev_id;
291     struct packed {
292         logic d;
293         logic de;
294     } invalid_owner_seed;

```

```

295     struct packed {
296         logic d;
297         logic de;
298     } invalid_creator_seed;
299 } keymgr_hw2reg_debug_reg_t;
300
301 // Register -> HW type
302 typedef struct packed {
303     keymgr_reg2hw_intr_state_reg_t intr_state; // [945:945]
304     keymgr_reg2hw_intr_enable_reg_t intr_enable; // [944:944]
305     keymgr_reg2hw_intr_test_reg_t intr_test; // [943:942]
306     keymgr_reg2hw_alert_test_reg_t alert_test; // [941:938]
307     keymgr_reg2hw_start_reg_t start; // [937:937]
308     keymgr_reg2hw_control_shadowed_reg_t control_shadowed; //
[936:931]
309     keymgr_reg2hw_sideload_clear_reg_t sideload_clear; //
[930:928]
310     keymgr_reg2hw_reseed_interval_shadowed_reg_t re-
seed_interval_shadowed; // [927:912]
311     keymgr_reg2hw_sw_binding_regwen_reg_t sw_bind-
ing_regwen; // [911:910]
312     keymgr_reg2hw_sealing_sw_binding_mreg_t [7:0] seal-
ing_sw_binding; // [909:654]
313     keymgr_reg2hw_attest_sw_binding_mreg_t [7:0] at-
test_sw_binding; // [653:398]
314     keymgr_reg2hw_salt_mreg_t [7:0] salt; // [397:142]
315     keymgr_reg2hw_key_version_mreg_t [0:0] key_version; //
[141:110]
316     keymgr_reg2hw_max_creator_key_ver_shadowed_reg_t max_cre-
ator_key_ver_shadowed; // [109:78]
317     keymgr_reg2hw_max_owner_int_key_ver_shadowed_reg_t
max_owner_int_key_ver_shadowed; // [77:46]
318     keymgr_reg2hw_max_owner_key_ver_shadowed_reg_t max_ow-
ner_key_ver_shadowed; // [45:14]
319     keymgr_reg2hw_fault_status_reg_t fault_status; // [13:0]
320 } keymgr_reg2hw_t;
321
322 // HW -> register type
323 typedef struct packed {
324     keymgr_hw2reg_intr_state_reg_t intr_state; // [588:587]
325     keymgr_hw2reg_cfg_regwen_reg_t cfg_regwen; // [586:586]
326     keymgr_hw2reg_start_reg_t start; // [585:584]
327     keymgr_hw2reg_sw_binding_regwen_reg_t sw_bind-
ing_regwen; // [583:583]
328     keymgr_hw2reg_sw_share0_output_mreg_t [7:0]
sw_share0_output; // [582:319]
329     keymgr_hw2reg_sw_share1_output_mreg_t [7:0]
sw_share1_output; // [318:55]
330     keymgr_hw2reg_working_state_reg_t working_state; //
[54:51]
331     keymgr_hw2reg_op_status_reg_t op_status; // [50:48]
332     keymgr_hw2reg_err_code_reg_t err_code; // [47:42]
333     keymgr_hw2reg_fault_status_reg_t fault_status; // [41:14]
334     keymgr_hw2reg_debug_reg_t debug; // [13:0]
335 } keymgr_hw2reg_t;
336
337 // Register offsets
338 parameter logic [BlockAw-1:0] KEYMGR_INTR_STATE_OFFSET = 8'h
0;
339 parameter logic [BlockAw-1:0] KEYMGR_INTR_ENABLE_OFFSET =
8'h 4;
340 parameter logic [BlockAw-1:0] KEYMGR_INTR_TEST_OFFSET = 8'h
8;
341 parameter logic [BlockAw-1:0] KEYMGR_ALERT_TEST_OFFSET = 8'h
c;
342 parameter logic [BlockAw-1:0] KEYMGR_CFG_REGWEN_OFFSET = 8'h
10;
343 parameter logic [BlockAw-1:0] KEYMGR_START_OFFSET = 8'h 14;
344 parameter logic [BlockAw-1:0] KEYMGR_CONTROL_SHADOWED_OFFSET =
8'h 18;
345 parameter logic [BlockAw-1:0] KEYMGR_SIDELOAD_CLEAR_OFFSET =
8'h 1c;
346 parameter logic [BlockAw-1:0] KEYM-
GR_RESEED_INTERVAL_REGWEN_OFFSET = 8'h 20;
347 parameter logic [BlockAw-1:0] KEYM-
GR_RESEED_INTERVAL_SHADOWED_OFFSET = 8'h 24;
348 parameter logic [BlockAw-1:0] KEYM-
GR_SW_BINDING_REGWEN_OFFSET = 8'h 28;
349 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_0_OFFSET = 8'h 2c;
350 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_1_OFFSET = 8'h 30;
351 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_2_OFFSET = 8'h 34;
352 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_3_OFFSET = 8'h 38;
353 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_4_OFFSET = 8'h 3c;
354 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_5_OFFSET = 8'h 40;
355 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_6_OFFSET = 8'h 44;
356 parameter logic [BlockAw-1:0] KEYM-
GR_SEALING_SW_BINDING_7_OFFSET = 8'h 48;
357 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_0_OFFSET = 8'h 4c;

```

```

358 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_1_OFFSET = 8'h 50;
359 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_2_OFFSET = 8'h 54;
360 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_3_OFFSET = 8'h 58;
361 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_4_OFFSET = 8'h 5c;
362 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_5_OFFSET = 8'h 60;
363 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_6_OFFSET = 8'h 64;
364 parameter logic [BlockAw-1:0] KEYM-
GR_ATTEST_SW_BINDING_7_OFFSET = 8'h 68;
365 parameter logic [BlockAw-1:0] KEYMGR_SALT_0_OFFSET = 8'h 6c;
366 parameter logic [BlockAw-1:0] KEYMGR_SALT_1_OFFSET = 8'h 70;
367 parameter logic [BlockAw-1:0] KEYMGR_SALT_2_OFFSET = 8'h 74;
368 parameter logic [BlockAw-1:0] KEYMGR_SALT_3_OFFSET = 8'h 78;
369 parameter logic [BlockAw-1:0] KEYMGR_SALT_4_OFFSET = 8'h 7c;
370 parameter logic [BlockAw-1:0] KEYMGR_SALT_5_OFFSET = 8'h 80;
371 parameter logic [BlockAw-1:0] KEYMGR_SALT_6_OFFSET = 8'h 84;
372 parameter logic [BlockAw-1:0] KEYMGR_SALT_7_OFFSET = 8'h 88;
373 parameter logic [BlockAw-1:0] KEYMGR_KEY_VERSION_OFFSET =
8'h 8c;
374 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_CREATOR_KEY_VER_REGWEN_OFFSET = 8'h 90;
375 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_CREATOR_KEY_VER_SHADOWED_OFFSET = 8'h 94;
376 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_INT_KEY_VER_REGWEN_OFFSET = 8'h 98;
377 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_INT_KEY_VER_SHADOWED_OFFSET = 8'h 9c;
378 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_KEY_VER_REGWEN_OFFSET = 8'h a0;
379 parameter logic [BlockAw-1:0] KEYM-
GR_MAX_OWNER_KEY_VER_SHADOWED_OFFSET = 8'h a4;
380 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_0_OFFSET = 8'h a8;
381 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_1_OFFSET = 8'h ac;
382 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_2_OFFSET = 8'h b0;
383 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_3_OFFSET = 8'h b4;
384 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_4_OFFSET = 8'h b8;
385 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_5_OFFSET = 8'h bc;
386 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_6_OFFSET = 8'h c0;
387 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE0_OUTPUT_7_OFFSET = 8'h c4;
388 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_0_OFFSET = 8'h c8;
389 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_1_OFFSET = 8'h cc;
390 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_2_OFFSET = 8'h d0;
391 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_3_OFFSET = 8'h d4;
392 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_4_OFFSET = 8'h d8;
393 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_5_OFFSET = 8'h dc;
394 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_6_OFFSET = 8'h e0;
395 parameter logic [BlockAw-1:0] KEYM-
GR_SW_SHARE1_OUTPUT_7_OFFSET = 8'h e4;
396 parameter logic [BlockAw-1:0] KEYMGR_WORKING_STATE_OFFSET =
8'h e8;
397 parameter logic [BlockAw-1:0] KEYMGR_OP_STATUS_OFFSET = 8'h
ec;
398 parameter logic [BlockAw-1:0] KEYMGR_ERR_CODE_OFFSET = 8'h
f0;
399 parameter logic [BlockAw-1:0] KEYMGR_FAULT_STATUS_OFFSET =
8'h f4;
400 parameter logic [BlockAw-1:0] KEYMGR_DEBUG_OFFSET = 8'h f8;
401
402 // Reset values for hwext registers and their fields
403 parameter logic [0:0] KEYMGR_INTR_TEST_RESVAL = 1'h 0;
404 parameter logic [0:0] KEYMGR_INTR_TEST_OP_DONE_RESVAL = 1'h
0;
405 parameter logic [1:0] KEYMGR_ALERT_TEST_RESVAL = 2'h 0;
406 parameter logic [0:0] KEYM-
GR_ALERT_TEST_RECOV_OPERATION_ERR_RESVAL = 1'h 0;
407 parameter logic [0:0] KEYM-
GR_ALERT_TEST_FATAL_FAULT_ERR_RESVAL = 1'h 0;
408 parameter logic [0:0] KEYMGR_CFG_REGWEN_RESVAL = 1'h 1;
409 parameter logic [0:0] KEYMGR_CFG_REGWEN_EN_RESVAL = 1'h 1;
410 parameter logic [0:0] KEYMGR_SW_BINDING_REGWEN_RESVAL = 1'h
1;
411 parameter logic [0:0] KEYMGR_SW_BINDING_REGWEN_EN_RESVAL =
1'h 1;
412
413 // Register index
414 typedef enum int {
415     KEYMGR_INTR_STATE,
416     KEYMGR_INTR_ENABLE,
417     KEYMGR_INTR_TEST,
418     KEYMGR_ALERT_TEST,
419     KEYMGR_CFG_REGWEN,

```

```

420 KEYMGR_START,
421 KEYMGR_CONTROL_SHADOWED,
422 KEYMGR_SIDELOAD_CLEAR,
423 KEYMGR_RESEED_INTERVAL_REGWEN,
424 KEYMGR_RESEED_INTERVAL_SHADOWED,
425 KEYMGR_SW_BINDING_REGWEN,
426 KEYMGR_SEALING_SW_BINDING_0,
427 KEYMGR_SEALING_SW_BINDING_1,
428 KEYMGR_SEALING_SW_BINDING_2,
429 KEYMGR_SEALING_SW_BINDING_3,
430 KEYMGR_SEALING_SW_BINDING_4,
431 KEYMGR_SEALING_SW_BINDING_5,
432 KEYMGR_SEALING_SW_BINDING_6,
433 KEYMGR_SEALING_SW_BINDING_7,
434 KEYMGR_ATTEST_SW_BINDING_0,
435 KEYMGR_ATTEST_SW_BINDING_1,
436 KEYMGR_ATTEST_SW_BINDING_2,
437 KEYMGR_ATTEST_SW_BINDING_3,
438 KEYMGR_ATTEST_SW_BINDING_4,
439 KEYMGR_ATTEST_SW_BINDING_5,
440 KEYMGR_ATTEST_SW_BINDING_6,
441 KEYMGR_ATTEST_SW_BINDING_7,
442 KEYMGR_SALT_0,
443 KEYMGR_SALT_1,
444 KEYMGR_SALT_2,
445 KEYMGR_SALT_3,
446 KEYMGR_SALT_4,
447 KEYMGR_SALT_5,
448 KEYMGR_SALT_6,
449 KEYMGR_SALT_7,
450 KEYMGR_KEY_VERSION,
451 KEYMGR_MAX_CREATOR_KEY_VER_REGWEN,
452 KEYMGR_MAX_CREATOR_KEY_VER_SHADOWED,
453 KEYMGR_MAX_OWNER_INT_KEY_VER_REGWEN,
454 KEYMGR_MAX_OWNER_INT_KEY_VER_SHADOWED,
455 KEYMGR_MAX_OWNER_KEY_VER_REGWEN,
456 KEYMGR_MAX_OWNER_KEY_VER_SHADOWED,
457 KEYMGR_SW_SHARE0_OUTPUT_0,
458 KEYMGR_SW_SHARE0_OUTPUT_1,
459 KEYMGR_SW_SHARE0_OUTPUT_2,
460 KEYMGR_SW_SHARE0_OUTPUT_3,
461 KEYMGR_SW_SHARE0_OUTPUT_4,
462 KEYMGR_SW_SHARE0_OUTPUT_5,
463 KEYMGR_SW_SHARE0_OUTPUT_6,
464 KEYMGR_SW_SHARE0_OUTPUT_7,
465 KEYMGR_SW_SHARE1_OUTPUT_0,
466 KEYMGR_SW_SHARE1_OUTPUT_1,
467 KEYMGR_SW_SHARE1_OUTPUT_2,
468 KEYMGR_SW_SHARE1_OUTPUT_3,
469 KEYMGR_SW_SHARE1_OUTPUT_4,
470 KEYMGR_SW_SHARE1_OUTPUT_5,
471 KEYMGR_SW_SHARE1_OUTPUT_6,
472 KEYMGR_SW_SHARE1_OUTPUT_7,
473 KEYMGR_WORKING_STATE,
474 KEYMGR_OP_STATUS,
475 KEYMGR_ERR_CODE,
476 KEYMGR_FAULT_STATUS,
477 KEYMGR_DEBUG
478 } keymgr_id_e;
479
480 // Register width information to check illegal writes
481 parameter logic [3:0] KEYMGR_PERMIT [63] = '{
482     4'b 0001, // index[ 0] KEYMGR_INTR_STATE
483     4'b 0001, // index[ 1] KEYMGR_INTR_ENABLE
484     4'b 0001, // index[ 2] KEYMGR_INTR_TEST
485     4'b 0001, // index[ 3] KEYMGR_ALERT_TEST
486     4'b 0001, // index[ 4] KEYMGR_CFG_REGWEN
487     4'b 0001, // index[ 5] KEYMGR_START
488     4'b 0011, // index[ 6] KEYMGR_CONTROL_SHADOWED
489     4'b 0001, // index[ 7] KEYMGR_SIDELOAD_CLEAR
490     4'b 0001, // index[ 8] KEYMGR_RESEED_INTERVAL_REGWEN
491     4'b 0011, // index[ 9] KEYMGR_RESEED_INTERVAL_SHADOWED
492     4'b 0001, // index[10] KEYMGR_SW_BINDING_REGWEN
493     4'b 1111, // index[11] KEYMGR_SEALING_SW_BINDING_0
494     4'b 1111, // index[12] KEYMGR_SEALING_SW_BINDING_1
495     4'b 1111, // index[13] KEYMGR_SEALING_SW_BINDING_2
496     4'b 1111, // index[14] KEYMGR_SEALING_SW_BINDING_3
497     4'b 1111, // index[15] KEYMGR_SEALING_SW_BINDING_4
498     4'b 1111, // index[16] KEYMGR_SEALING_SW_BINDING_5
499     4'b 1111, // index[17] KEYMGR_SEALING_SW_BINDING_6
500     4'b 1111, // index[18] KEYMGR_SEALING_SW_BINDING_7
501     4'b 1111, // index[19] KEYMGR_ATTEST_SW_BINDING_0
502     4'b 1111, // index[20] KEYMGR_ATTEST_SW_BINDING_1
503     4'b 1111, // index[21] KEYMGR_ATTEST_SW_BINDING_2
504     4'b 1111, // index[22] KEYMGR_ATTEST_SW_BINDING_3
505     4'b 1111, // index[23] KEYMGR_ATTEST_SW_BINDING_4
506     4'b 1111, // index[24] KEYMGR_ATTEST_SW_BINDING_5
507     4'b 1111, // index[25] KEYMGR_ATTEST_SW_BINDING_6
508     4'b 1111, // index[26] KEYMGR_ATTEST_SW_BINDING_7
509     4'b 1111, // index[27] KEYMGR_SALT_0
510     4'b 1111, // index[28] KEYMGR_SALT_1
511     4'b 1111, // index[29] KEYMGR_SALT_2
512     4'b 1111, // index[30] KEYMGR_SALT_3
513     4'b 1111, // index[31] KEYMGR_SALT_4
514     4'b 1111, // index[32] KEYMGR_SALT_5
515     4'b 1111, // index[33] KEYMGR_SALT_6
516     4'b 1111, // index[34] KEYMGR_SALT_7
517     4'b 1111, // index[35] KEYMGR_KEY_VERSION
518     4'b 0001, // index[36] KEYMGR_MAX_CREATOR_KEY_VER_REGWEN
519     4'b 1111, // index[37] KEYMGR_MAX_CREATOR_KEY_VER_SHADOWED
520     4'b 0001, // index[38] KEYMGR_MAX_OWNER_INT_KEY_VER_REGWEN

```



```

521      4'b 1111, // index[39] KEYM-
GR_MAX_OWNER_INT_KEY_VER_SHADOWED
522      4'b 0001, // index[40] KEYMGR_MAX_OWNER_KEY_VER_REGWEN
523      4'b 1111, // index[41] KEYMGR_MAX_OWNER_KEY_VER_SHADOWED
524      4'b 1111, // index[42] KEYMGR_SW_SHARE0_OUTPUT_0
525      4'b 1111, // index[43] KEYMGR_SW_SHARE0_OUTPUT_1
526      4'b 1111, // index[44] KEYMGR_SW_SHARE0_OUTPUT_2
527      4'b 1111, // index[45] KEYMGR_SW_SHARE0_OUTPUT_3
528      4'b 1111, // index[46] KEYMGR_SW_SHARE0_OUTPUT_4
529      4'b 1111, // index[47] KEYMGR_SW_SHARE0_OUTPUT_5
530      4'b 1111, // index[48] KEYMGR_SW_SHARE0_OUTPUT_6
531      4'b 1111, // index[49] KEYMGR_SW_SHARE0_OUTPUT_7
532      4'b 1111, // index[50] KEYMGR_SW_SHARE1_OUTPUT_0
533      4'b 1111, // index[51] KEYMGR_SW_SHARE1_OUTPUT_1

534      4'b 1111, // index[52] KEYMGR_SW_SHARE1_OUTPUT_2
535      4'b 1111, // index[53] KEYMGR_SW_SHARE1_OUTPUT_3
536      4'b 1111, // index[54] KEYMGR_SW_SHARE1_OUTPUT_4
537      4'b 1111, // index[55] KEYMGR_SW_SHARE1_OUTPUT_5
538      4'b 1111, // index[56] KEYMGR_SW_SHARE1_OUTPUT_6
539      4'b 1111, // index[57] KEYMGR_SW_SHARE1_OUTPUT_7
540      4'b 0001, // index[58] KEYMGR_WORKING_STATE
541      4'b 0001, // index[59] KEYMGR_OP_STATUS
542      4'b 0001, // index[60] KEYMGR_ERR_CODE
543      4'b 0011, // index[61] KEYMGR_FAULT_STATUS
544      4'b 0001 // index[62] KEYMGR_DEBUG
545  };
546
547 endpackage

```

수명주기 컨트롤러

What. 칩의 수명 단계(RAW → TEST → DEV → PROD → RMA)를 관리하고 단계별로 기능을 열고 닫는 블록.

Why it exists. 공장에서는 열려 있어야 하고 현장에서는 닫혀 있어야 하는 기능 이 있기 때문이다. 테스트 모드·디버그·키 주입이 그렇다. 단계 전이는 되돌릴 수 없고, 각 전이는 토큰(비밀값)으로 인증된다.

학부 복습 · **EECS concepts.** 상태기계 · 접근 제어 · 비가역 전이 · 안전한 기본값(fail-closed).

석사 수준 · **Practice note.** 이 블록은 ‘제품 전략’이 실리콘에 박힌 것 이다. 어떤 기능을 언제 잠글지는 엔지니어링이 아니라 사업 결정이고, 그것을 RTL 로 옮기는 것이 이 블록이다. 요구사항 변경이 가장 잦은 블록이기도 하다.

9 files • 5,016 lines

API — lc_ctrl.sv

Table 610. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — module / interface / package (1개)

이름	종류
lc_ctrl	module

Table 611. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — parameter (17개)

파라미터	형/폭	기본값
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
AlertSkewCycles	int unsigned	1
SiliconCreatorId	logic [SiliconCreatorIdWidth-1:0]	'0
ProductId	logic [ProductIdWidth-1:0]	'0
RevisionId	logic [RevisionIdWidth-1:0]	'0
IdcodeValue	logic [31:0]	32'h00000001
UseDmiInterface	bit	1'b0
NumRmaAckSigs	int unsigned	2
RndCnstLcKeymgrDivInvalid	lc_keymgr_div_t	LcKeymgrDivWidth'(0)
RndCnstLcKeymgrDivTestUnlocked	lc_keymgr_div_t	LcKeymgrDivWidth'(1)
RndCnstLcKeymgrDivDev	lc_keymgr_div_t	LcKeymgrDivWidth'(2)
RndCnstLcKeymgrDivProduction	lc_keymgr_div_t	LcKeymgrDivWidth'(3)
RndCnstLcKeymgrDivRma	lc_keymgr_div_t	LcKeymgrDivWidth'(4)
RndCnstInvalidTokens	lc_token_mux_t	{TokenMuxBits{1'b1}}
SecVolatileRawUnlockEn	bit	0
EscNumSeverities	int	4
EscPingCountWidth	int	16

Table 612. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — localparam (2개)

상수	형/폭	값
HwRevWidth	int	\$bits(lc_hw_rev_t)
HwRev	lc_hw_rev_t	{silicon_creator_id: SiliconCreatorId

Table 613. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv — 포트 (53개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clk_kmac_i	input	
rst_kmac_ni	input	
regs_tl_i	input	tlul_pkg::tl_h2d_t
regs_tl_o	output	tlul_pkg::tl_d2h_t
dmi_tl_i	input	tlul_pkg::tl_h2d_t
dmi_tl_o	output	tlul_pkg::tl_d2h_t
jtag_i	input	jtag_pkg::jtag_req_t
jtag_o	output	jtag_pkg::jtag_rsp_t
scan_rst_ni	input	
scanmode_i	input	prim_mubi_pkg::mubi4_t
alert_rx_i	input	prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
alert_tx_o	output	prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
esc_scrap_state0_tx_i	input	prim_esc_pkg::esc_rx_t
esc_scrap_state0_rx_o	output	prim_esc_pkg::esc_tx_t
esc_scrap_state1_tx_i	input	prim_esc_pkg::esc_rx_t
esc_scrap_state1_rx_o	output	prim_esc_pkg::esc_tx_t
pwr_lc_i	input	pwr_lc_req_t
pwr_lc_o	output	pwr_lc_rsp_t
strap_en_override_o	output	logic
lc_otp_vendor_test_o	output	otp_macro_pkg::otp_test_req_t
lc_otp_vendor_test_i	input	otp_macro_pkg::otp_test_rsp_t
lc_otp_program_o	output	otp_ctrl_pkg::lc_otp_program_req_t
lc_otp_program_i	input	otp_ctrl_pkg::lc_otp_program_rsp_t
kmac_data_i	input	kmac_pkg::app_rsp_t
kmac_data_o	output	kmac_pkg::app_req_t
otp_lc_data_i	input	otp_ctrl_pkg::otp_lc_data_t
lc_init_done_o	output	lc_tx_t

포트	방향	형/폭
lc_dft_en_o	output	lc_tx_t
lc_raw_test_rma_o	output	lc_tx_t
lc_nvm_debug_en_o	output	lc_tx_t
lc_hw_debug_clr_o	output	lc_tx_t
lc_hw_debug_en_o	output	lc_tx_t
lc_cpu_en_o	output	lc_tx_t
lc_creator_seed_sw_rw_en_o	output	lc_tx_t
lc_owner_seed_sw_rw_en_o	output	lc_tx_t
lc_iso_part_sw_rd_en_o	output	lc_tx_t
lc_iso_part_sw_wr_en_o	output	lc_tx_t
lc_seed_hw_rd_en_o	output	lc_tx_t
lc_rma_state_o	output	lc_tx_t
lc_keymgr_en_o	output	lc_tx_t
lc_escalate_en_o	output	lc_tx_t
lc_check_byp_en_o	output	lc_tx_t
lc_clk_byp_req_o	output	lc_tx_t
lc_clk_byp_ack_i	input	lc_tx_t
lc_nvm_rma_seed_o	output	lc_nvm_rma_seed_t
lc_nvm_rma_req_o	output	lc_tx_t
lc_nvm_rma_ack_i	input	lc_tx_t [NumRmaAckSigs-1:0]
lc_keymgr_div_o	output	lc_keymgr_div_t
otp_device_id_i	input	otp_ctrl_pkg::otp_device_id_t
otp_manuf_state_i	input	otp_ctrl_pkg::otp_device_id_t
hw_rev_o	output	lc_hw_rev_t

API — lc_ctrl_dmi_reg_top.sv

Table 614. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_dmi_reg_top.sv — module / interface / package (1개)

이름	종류
lc_ctrl_dmi_reg_top	module

Table 615. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_dmi_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t

포트	방향	형/폭
tl_o	output	tlul_pkg::tl_d2h_t
tl_win_o	output	tlul_pkg::tl_h2d_t
tl_win_i	input	tlul_pkg::tl_d2h_t
intg_err_o	output	logic

API — lc_ctrl_fsm.sv

Table 616. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — module / interface / package (1개)

이름	종류
lc_ctrl_fsm	module

Table 617. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — parameter (8개)

파라미터	형/폭	기본값
NumRmaAckSigs	int unsigned	2
RndCnstLcKeymgrDivInvalid	lc_keymgr_div_t	LcKeymgrDivWidth'(0)
RndCnstLcKeymgrDivTestUnlocked	lc_keymgr_div_t	LcKeymgrDivWidth'(1)
RndCnstLcKeymgrDivDev	lc_keymgr_div_t	LcKeymgrDivWidth'(2)
RndCnstLcKeymgrDivProduction	lc_keymgr_div_t	LcKeymgrDivWidth'(3)
RndCnstLcKeymgrDivRma	lc_keymgr_div_t	LcKeymgrDivWidth'(4)
RndCnstInvalidTokens	lc_token_mux_t	{TokenMuxBits{1'b1}}
SecVolatileRawUnlockEn	bit	0

Table 618. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — localparam (1개)

상수	형/폭	값
NumStrapDelayRegs	int	10

Table 619. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv — 포트 (65개) — 앞 60개만 표시(전체 65개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
init_req_i	input	
init_done_o	output	logic
idle_o	output	logic
esc_scrap_state0_i	input	
esc_scrap_state1_i	input	
lc_state_valid_i	input	
lc_state_i	input	lc_state_e

포트	방향	형/폭
lc_cnt_i	input	lc_cnt_e
secrets_valid_i	input	lc_tx_t
use_ext_clock_i	input	
ext_clock_switched_o	output	logic
volatile_raw_unlock_i	input	logic
strap_en_override_o	output	logic
test_unlock_token_i	input	lc_token_t
test_exit_token_i	input	lc_token_t
test_tokens_valid_i	input	lc_tx_t
rma_token_i	input	lc_token_t
rma_token_valid_i	input	lc_tx_t
trans_cmd_i	input	
trans_target_i	input	ext_dec_lc_state_t
dec_lc_state_o	output	ext_dec_lc_state_t
dec_lc_cnt_o	output	dec_lc_cnt_t
dec_lc_id_state_o	output	dec_lc_id_state_e
token_hash_req_o	output	logic
token_hash_req_chk_o	output	logic
token_hash_ack_i	input	
token_hash_err_i	input	
token_if_fsm_err_i	input	
hashed_token_i	input	lc_token_t
unhashed_token_i	input	lc_token_t
otp_prog_req_o	output	logic
otp_prog_lc_state_o	output	lc_state_e
otp_prog_lc_cnt_o	output	lc_cnt_e
otp_prog_ack_i	input	
otp_prog_err_i	input	
trans_success_o	output	logic
trans_cnt_oflw_error_o	output	logic
trans_invalid_error_o	output	logic
token_invalid_error_o	output	logic
nvm_rma_error_o	output	logic
otp_prog_error_o	output	logic
state_invalid_error_o	output	logic

포트	방향	형/폭
lc_raw_test_rma_o	output	lc_tx_t
lc_init_done_o	output	lc_tx_t
lc_dft_en_o	output	lc_tx_t
lc_nvm_debug_en_o	output	lc_tx_t
lc_hw_debug_clr_o	output	lc_tx_t
lc_hw_debug_en_o	output	lc_tx_t
lc_cpu_en_o	output	lc_tx_t
lc_creator_seed_sw_rw_en_o	output	lc_tx_t
lc_owner_seed_sw_rw_en_o	output	lc_tx_t
lc_iso_part_sw_rd_en_o	output	lc_tx_t
lc_iso_part_sw_wr_en_o	output	lc_tx_t
lc_seed_hw_rd_en_o	output	lc_tx_t
lc_rma_state_o	output	lc_tx_t
lc_keymgr_en_o	output	lc_tx_t
lc_escalate_en_o	output	lc_tx_t
lc_check_byp_en_o	output	lc_tx_t

API — lc_ctrl_kmac_if.sv

Table 620. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv — module / interface / package (1개)

이름	종류
lc_ctrl_kmac_if	module

Table 621. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv — localparam (1개)

상수	형/폭	값
StateWidth	int	8

Table 622. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv — 포트 (13개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clk_kmac_i	input	
rst_kmac_ni	input	
kmac_data_i	input	kmac_pkg::app_rsp_t
kmac_data_o	output	kmac_pkg::app_req_t
transition_token_i	input	lc_token_t
token_hash_req_i	input	

포트	방향	형/폭
token_hash_req_chk_i	input	
token_hash_ack_o	output	logic
token_hash_err_o	output	logic
token_if_fsm_err_o	output	logic
hashed_token_o	output	lc_token_t

API — lc_ctrl_pkg.sv

Table 623. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — module / interface / package (1개)

이름	종류
lc_ctrl_pkg	package

Table 624. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — parameter (10개)

파라미터	형/폭	기본값
PWR_LC_RSP_DEFAULT	pwr_lc_rsp_t	'{
NumTokens	int	6
TokenIdxWidth	int	vbits(NumTokens)
TokenMuxBits	int	2**TokenIdxWidth*LcTokenWidth
TxWidth	int	4
LC_TX_DEFAULT	lc_tx_t	lc_tx_t'(Off)
RmaSeedWidth	int	64
LC_NVM_RMA_SEED_DEFAULT	lc_nvm_rma_seed_t	'0
LcKeymgrDivWidth	int	128
TransTokenIdxMatrix	token_idx_e [NumLcStates-1:0][NumLcState...	{

Table 625. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — localparam (1개)

상수	형/폭	값
FsmStateWidth	int	16

Table 626. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — typedef / using (3개)

새 이름	원래 형
lc_token_mux_t	logic [TokenMuxBits-1:0]
lc_nvm_rma_seed_t	logic [RmaSeedWidth-1:0]
lc_keymgr_div_t	logic [LcKeymgrDivWidth-1:0]

Table 627. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv — #define / `define 매크로 (2개)

매크로	인자	정의
TEST_UNLOCKED	(idx)	\
TEST_LOCKED	(idx)	\

API — lc_ctrl_reg_pkg.sv**Table 628. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_reg_pkg.sv — module / interface / package (1개)**

이름	종류
lc_ctrl_reg_pkg	package

Table 629. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_reg_pkg.sv — parameter (94개) — 앞 60개만 표시(전체 94개, 본문 소스 참조)

파라미터	형/폭	기본값
SiliconCreatorIdWidth	int	16
ProductIdWidth	int	16
RevisionIdWidth	int	8
NumTokenWords	int	4
CsrLcStateWidth	int	30
CsrLcCountWidth	int	5
CsrLcIdStateWidth	int	32
CsrOtpTestCtrlWidth	int	32
CsrOtpTestStatusWidth	int	32
NumDeviceIdWords	int	8
NumManufStateWords	int	8
NumAlerts	int	3
RegsAw	int	8
DmiAw	int	12
NumRegsRegs	int	35
NumRegsDmi	int	0
LC_CTRL_ALERT_TEST_OFFSET	logic [RegsAw-1:0]	8'h 0
LC_CTRL_STATUS_OFFSET	logic [RegsAw-1:0]	8'h 4
LC_CTRL_CLAIM_TRANSITION_IF_REGWEN_OFFSET	logic [RegsAw-1:0]	8'h 8
LC_CTRL_CLAIM_TRANSITION_IF_OFFSET	logic [RegsAw-1:0]	8'h c
LC_CTRL_TRANSITION_REGWEN_OFFSET	logic [RegsAw-1:0]	8'h 10
LC_CTRL_TRANSITION_CMD_OFFSET	logic [RegsAw-1:0]	8'h 14
LC_CTRL_TRANSITION_CTRL_OFFSET	logic [RegsAw-1:0]	8'h 18
LC_CTRL_TRANSITION_TOKEN_0_OFFSET	logic [RegsAw-1:0]	8'h 1c
LC_CTRL_TRANSITION_TOKEN_1_OFFSET	logic [RegsAw-1:0]	8'h 20
LC_CTRL_TRANSITION_TOKEN_2_OFFSET	logic [RegsAw-1:0]	8'h 24
LC_CTRL_TRANSITION_TOKEN_3_OFFSET	logic [RegsAw-1:0]	8'h 28
LC_CTRL_TRANSITION_TARGET_OFFSET	logic [RegsAw-1:0]	8'h 2c
LC_CTRL_OTP_VENDOR_TEST_CTRL_OFFSET	logic [RegsAw-1:0]	8'h 30

파라미터	형/폭	기본값
LC_CTRL_OTP_VENDOR_TEST_STATUS_OFFSET	logic [RegsAw-1:0]	8'h 34
LC_CTRL_LC_STATE_OFFSET	logic [RegsAw-1:0]	8'h 38
LC_CTRL_LC_TRANSITION_CNT_OFFSET	logic [RegsAw-1:0]	8'h 3c
LC_CTRL_LC_ID_STATE_OFFSET	logic [RegsAw-1:0]	8'h 40
LC_CTRL_HW_REVISION0_OFFSET	logic [RegsAw-1:0]	8'h 44
LC_CTRL_HW_REVISION1_OFFSET	logic [RegsAw-1:0]	8'h 48
LC_CTRL_DEVICE_ID_0_OFFSET	logic [RegsAw-1:0]	8'h 4c
LC_CTRL_DEVICE_ID_1_OFFSET	logic [RegsAw-1:0]	8'h 50
LC_CTRL_DEVICE_ID_2_OFFSET	logic [RegsAw-1:0]	8'h 54
LC_CTRL_DEVICE_ID_3_OFFSET	logic [RegsAw-1:0]	8'h 58
LC_CTRL_DEVICE_ID_4_OFFSET	logic [RegsAw-1:0]	8'h 5c
LC_CTRL_DEVICE_ID_5_OFFSET	logic [RegsAw-1:0]	8'h 60
LC_CTRL_DEVICE_ID_6_OFFSET	logic [RegsAw-1:0]	8'h 64
LC_CTRL_DEVICE_ID_7_OFFSET	logic [RegsAw-1:0]	8'h 68
LC_CTRL_MANUF_STATE_0_OFFSET	logic [RegsAw-1:0]	8'h 6c
LC_CTRL_MANUF_STATE_1_OFFSET	logic [RegsAw-1:0]	8'h 70
LC_CTRL_MANUF_STATE_2_OFFSET	logic [RegsAw-1:0]	8'h 74
LC_CTRL_MANUF_STATE_3_OFFSET	logic [RegsAw-1:0]	8'h 78
LC_CTRL_MANUF_STATE_4_OFFSET	logic [RegsAw-1:0]	8'h 7c
LC_CTRL_MANUF_STATE_5_OFFSET	logic [RegsAw-1:0]	8'h 80
LC_CTRL_MANUF_STATE_6_OFFSET	logic [RegsAw-1:0]	8'h 84
LC_CTRL_MANUF_STATE_7_OFFSET	logic [RegsAw-1:0]	8'h 88
LC_CTRL_ALERT_TEST_RESVAL	logic [2:0]	3'h 0
LC_CTRL_ALERT_TEST_FATAL_PROG_ERROR_RESVAL	logic [0:0]	1'h 0
LC_CTRL_ALERT_TEST_FATAL_STATE_ERROR_RESVAL	logic [0:0]	1'h 0
LC_CTRL_ALERT_TEST_FATAL_BUS_INTEG_ERROR_RESVAL	logic [0:0]	1'h 0
LC_CTRL_STATUS_RESVAL	logic [11:0]	12'h 0
LC_CTRL_CLAIM_TRANSITION_IF_RESVAL	logic [7:0]	8'h 69
LC_CTRL_CLAIM_TRANSITION_IF_MUTEX_RESVAL	logic [7:0]	8'h 69
LC_CTRL_TRANSITION_REGWEN_RESVAL	logic [0:0]	1'h 0
LC_CTRL_TRANSITION_REGWEN_TRANSITION_REGWEN_RESVAL	logic [0:0]	1'h 0

API — lc_ctrl_regs_reg_top.sv

Table 630. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv — module / interface / package (1개)

이름	종류
lc_ctrl_regs_reg_top	module

Table 631. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv — localparam (3개)

상수	형/폭	값
AW	int	8
DW	int	32
DBW	int	DW/8

Table 632. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
tl_i	input	tlul_pkg::tl_h2d_t
tl_o	output	tlul_pkg::tl_d2h_t
reg2hw	output	lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t
hw2reg	input	lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t
intg_err_o	output	logic

API — lc_ctrl_signal_decode.sv

Table 633. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_signal_decode.sv — module / interface / package (1개)

이름	종류
lc_ctrl_signal_decode	module

Table 634. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_signal_decode.sv — parameter (5개)

파라미터	형/폭	기본값
RndCnstLcKeymgrDivInvalid	lc_keymgr_div_t	LcKeymgrDivWidth'(0)
RndCnstLcKeymgrDivTestUnlocked	lc_keymgr_div_t	LcKeymgrDivWidth'(1)
RndCnstLcKeymgrDivDev	lc_keymgr_div_t	LcKeymgrDivWidth'(2)
RndCnstLcKeymgrDivProduction	lc_keymgr_div_t	LcKeymgrDivWidth'(3)
RndCnstLcKeymgrDivRma	lc_keymgr_div_t	LcKeymgrDivWidth'(4)

Table 635. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_signal_decode.sv — 포트 (22개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
lc_state_valid_i	input	logic
lc_state_i	input	lc_state_e

포트	방향	형/폭
fsm_state_i	input	fsm_state_e
secrets_valid_i	input	lc_tx_t
lc_raw_test_rma_o	output	lc_tx_t
lc_init_done_o	output	lc_tx_t
lc_dft_en_o	output	lc_tx_t
lc_nvm_debug_en_o	output	lc_tx_t
lc_hw_debug_clr_o	output	lc_tx_t
lc_hw_debug_en_o	output	lc_tx_t
lc_cpu_en_o	output	lc_tx_t
lc_creator_seed_sw_rw_en_o	output	lc_tx_t
lc_owner_seed_sw_rw_en_o	output	lc_tx_t
lc_iso_part_sw_rd_en_o	output	lc_tx_t
lc_iso_part_sw_wr_en_o	output	lc_tx_t
lc_seed_hw_rd_en_o	output	lc_tx_t
lc_rma_state_o	output	lc_tx_t
lc_keymgr_en_o	output	lc_tx_t
lc_escalate_en_o	output	lc_tx_t
lc_keymgr_div_o	output	lc_keymgr_div_t

API — lc_ctrl_state_decode.sv

Table 636. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_state_decode.sv — module / interface / package (1개)

이름	종류
lc_ctrl_state_decode	module

Table 637. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_state_decode.sv — 포트 (9개)

포트	방향	형/폭
lc_state_valid_i	input	logic
lc_state_i	input	lc_state_e
lc_cnt_i	input	lc_cnt_e
secrets_valid_i	input	lc_tx_t
fsm_state_i	input	fsm_state_e
dec_lc_state_o	output	ext_dec_lc_state_t
dec_lc_id_state_o	output	dec_lc_id_state_e
dec_lc_cnt_o	output	dec_lc_cnt_t
state_invalid_error_o	output	logic [5:0]

1. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl.sv (912 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Life cycle controller top.
6 //
7
8 `include "prim_assert.sv"
9
10 module lc_ctrl
11   import lc_ctrl_pkg::*;
12   import lc_ctrl_reg_pkg::*;
13   import lc_ctrl_state_pkg::*;
14   # (
15     // Enable asynchronous transitions on alerts.
16     parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
17       mAlerts[1'b1]},
18     // Number of cycles a differential skew is tolerated on the
19     alert and escalation signal
20     parameter int unsigned AlertSkewCycles = 1,
21     // Hardware revision numbers exposed in the CSRs.
22
23     parameter logic [SiliconCreatorIdWidth-1:0] SiliconCreatorId =
24       '0,
25
26     parameter logic [ProductIdWidth-1:0] ProductId =
27       '0,
28
29     parameter logic [RevisionIdWidth-1:0] RevisionId =
30       '0,
31
32     // Idcode value for the JTAG.
33     parameter logic [31:0] IdcodeValue = 32'h00000001,
34     parameter bit UseDmiInterface = 1'b0,
35     parameter int unsigned NumRmaAckSigs = 2,
36     // Random netlist constants
37     parameter lc_keymgr_div_t RndCnstLcKeymgrDivInvalid =
38       LcKeymgrDivWidth'(0),
39     parameter lc_keymgr_div_t RndCnstLcKeymgrDivTestUnlocked =
40       LcKeymgrDivWidth'(1),
41     parameter lc_keymgr_div_t RndCnstLcKeymgrDivDev =
42       LcKeymgrDivWidth'(2),
43     parameter lc_keymgr_div_t RndCnstLcKeymgrDivProduction =
44       LcKeymgrDivWidth'(3),
45     parameter lc_keymgr_div_t RndCnstLcKeymgrDivRma =
46       LcKeymgrDivWidth'(4),
47     parameter lc_token_mux_t RndCnstInvalidTokens =
48       {TokenMuxBits[1'b1]},
49     parameter bit SecVolatileRawUnlockEn =
50       0,
51     parameter int EscNumSeverities =
52       4,
53     parameter int EscPingCountWidth =
54       16
55   ) (
56     // Life cycle controller clock
57     input clk_i,
58     input rst_ni,
59     // Clock for KMAC interface
60     input clk_k,
61     input rst_k,
62     mac_i,
63     mac_ni,
64     // Bus Interface (device)
65     input tlul_pkg::tl_h2d_t
66     regs_tl_i,
67     output tlul_pkg::tl_d2h_t
68     regs_tl_o,
69     // TL-UL-based DMI
70     input tlul_pkg::tl_h2d_t dmi_tl_i,
71     output tlul_pkg::tl_d2h_t dmi_tl_o,
72     // JTAG TAP.
73     input jtag_pkg::jtag_req_t jtag_i,
74     output jtag_pkg::jtag_rsp_t jtag_o,
75     // This bypasses the clock inverter inside the JTAG TAP for
76     scanmode.
77     input scan_rst_ni,
78     input prim_mubi_pkg::mubi4_t scan-
79     mode_i,
80     // Alert outputs.
81     input prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
82     alert_rx_i,
83     output prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
84     alert_tx_o,
85     // Escalation inputs (severity 1 and 2).
86     // These need not be synchronized since the alert handler is
87     // in the same clock domain as the LC controller.
88     input prim_esc_pkg::esc_rx_t
89     esc_scrap_state0_tx_i,
90     output prim_esc_pkg::esc_tx_t
91     esc_scrap_state0_rx_o,
92     input prim_esc_pkg::esc_rx_t
93     esc_scrap_state1_tx_i,
94     output prim_esc_pkg::esc_tx_t
95     esc_scrap_state1_rx_o,
96
97     // Power manager interface (inputs are synced to lifecycle
98     clock domain).
99     input pwr_lc_req_t pwr_lc_i,
100    output pwr_lc_rsp_t pwr_lc_o,
101    // Strap sampling override that is only used when SecVolat-
102    ileRawUnlockEn = 1,
103    // Otherwise this output is tied off to 0.
104    output logic
105    strap_en_override_o,
106    // Strap override - this is only used when
107    // Macro-specific test registers going to lifecycle TAP/DMI
108    output otp_macro_pkg::otp_test_req_t
109    lc_otp_vendor_test_o,
110    input otp_macro_pkg::otp_test_rsp_t
111    lc_otp_vendor_test_i,
112    // Life cycle transition command interface.
113    // No sync required since LC and OTP are in the same clock
114    domain.
115    output otp_ctrl_pkg::lc_otp_program_req_t
116    lc_otp_program_o,
117    input otp_ctrl_pkg::lc_otp_program_rsp_t
118    lc_otp_program_i,
119    // Life cycle hashing interface for raw unlock
120    // Synchronized in the life cycle controller.
121    // SEC_CM: TOKEN.DIGEST
122    input kmac_pkg::app_rsp_t
123    kmac_data_i,
124    output kmac_pkg::app_req_t
125    kmac_data_o,
126    // OTP broadcast outputs
127    // No sync required since LC and OTP are in the same clock
128    domain.
129    // SEC_CM: TOKEN_VALID.CTRL.MUBI
130    input otp_ctrl_pkg::otp_lc_data_t
131    otp_lc_data_i,
132    // Life cycle broadcast outputs (all of them are re-
133    gistered).
134    // SEC_CM: INTERSIG.MUBI
135    output lc_tx_t lc_in-
136    it_done_o,
137    output lc_tx_t
138    lc_dft_en_o,
139    output lc_tx_t
140    lc_raw_test_rma_o,
141    output lc_tx_t
142    lc_nvm_debug_en_o,
143    output lc_tx_t
144    lc_hw_de-
145    bug_clr_o,
146    output lc_tx_t
147    lc_hw_de-
148    bug_en_o,
149    output lc_tx_t
150    lc_cpu_en_o,
151    output lc_tx_t
152    lc_creat-
153    or_seed_sw_rw_en_o,
154    output lc_tx_t
155    lc_own-
156    er_seed_sw_rw_en_o,
157    output lc_tx_t
158    lc_iso_part_sw_rd_en_o,
159    output lc_tx_t
160    lc_iso_part_sw_wr_en_o,
161    output lc_tx_t
162    lc_seed_hw_rd_en_o,
163    output lc_tx_t
164    lc_rma_state_o,
165    output lc_tx_t
166    lc_keym-
167    gr_en_o,
168    output lc_tx_t
169    lc_escal-
170    ate_en_o,
171    output lc_tx_t
172    lc_check_byp_en_o,
173    // Request and feedback to/from clock manager and AST.
174    // The ack is synced to the lc clock domain using
175    prim_lc_sync.
176    // SEC_CM: INTERSIG.MUBI
177    output lc_tx_t
178    lc_clk_byp_req_o,
179    input lc_tx_t
180    lc_clk_byp_ack_i,
181    // Request and feedback to/from nvm controller.
182    // The ack is synced to the lc clock domain using
183    prim_lc_sync.
184    output lc_nvm_rma_seed_t
185    lc_nvm_rma_seed_o,
186    // SEC_CM: INTERSIG.MUBI
187    output lc_tx_t
188    lc_nvm_rma_req_o,
189    input lc_tx_t [NumRmaAckSigs-1:0]
190    lc_nvm_rma_ack_i,
191    // State group diversification value for keymgr.
192    output lc_keymgr_div_t
193    lc_keym-
194    gr_div_o,
195    // Hardware config input, needed for the DEVICE_ID field.
196    input otp_ctrl_pkg::otp_device_id_t
197    otp_device_id_i,
198    // Hardware config input, needed for the MANUF_STATE field.
199    input otp_ctrl_pkg::otp_device_id_t
200    otp_manuf_state_i,
201    // Hardware revision output (static)
202    output lc_hw_rev_t hw_rev_o
203  );

```

```

128 import prim_mubi_pkg::mubi8_t;
129 import prim_mubi_pkg::MuBi8False;
130 import prim_mubi_pkg::mubi8_test_true_strict;
131 import prim_mubi_pkg::mubi8_test_false_loose;
132
133 // Integration Checks //
134 // Integration Checks //
135 // Integration Checks //
136
137 // Check that the CSR parameters correspond with the ones
used in the design.
138 `ASSERT_INIT(DecLcStateWidthCheck_A, CsrLcStateWidth == Ext-
DecLcStateWidth)
139 `ASSERT_INIT(DecLcCountWidthCheck_A, CsrLcCountWidth == De-
cLcCountWidth)
140 `ASSERT_INIT(DecLcIdStateWidthCheck_A, CsrLcIdStateWidth ==
ExtDecLcIdStateWidth)
141 `ASSERT_INIT(NumTokenWordsCheck_A, NumTokenWords == LcToken-
Width/32)
142 `ASSERT_INIT(0tpTestCtrlWidth_A,
otp_macro_pkg::0tpTestCtrlWidth == Csr0tpTestCtrlWidth)
143
144 // Regfile //
145 // Regfile //
146 // Regfile //
147
148 lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t reg2hw;
149 lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t hw2reg;
150
151 // SEC_CM: TRANSITION.CONFIG.REGWEN, STATE.CONFIG.SPARSE
152 logic fatal_bus_integ_error_q, fatal_bus_integ_error_csr_d,
fatal_bus_integ_error_tap_dmi_d;
153 lc_ctrl_regs_reg_top u_reg_regs (
154   .clk_i,
155   .rst_ni,
156   .tl_i    ( reg2hw.tl_i ),
157   .tl_o    ( reg2hw.tl_o ),
158   .reg2hw  ( reg2hw ),
159   .hw2reg  ( hw2reg ),
160   // SEC_CM: BUS.INTEGRITY
161   .integ_err_o( fatal_bus_integ_error_csr_d )
162 );
163
164 // Life Cycle TAP/DMI Regs //
165 // Life Cycle TAP/DMI Regs //
166 // Life Cycle TAP/DMI Regs //
167
168 lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t tap_dmi_reg2hw;
169 lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t tap_dmi_hw2reg;
170
171 tlul_pkg::tl_h2d_t tap_dmi_tl_h2d;
172 tlul_pkg::tl_d2h_t tap_dmi_tl_d2h;
173 tlul_pkg::tl_h2d_t tap_tl_h2d;
174 tlul_pkg::tl_d2h_t tap_tl_d2h;
175
176 // Statically mux DMI TLUL port and the one coming from the
JTAG TAP
177 if (UseDmiInterface) begin : gen_dmi_tlul_ports
178   assign tap_dmi_tl_h2d = dmi_tl_i;
179   assign dmi_tl_o      = tap_dmi_tl_d2h;
180
181   // Tie-off other port
182   assign tap_tl_h2d = '0;
183   assign tap_tl_d2h = '0;
184   assign jtag_o      = '0;
185   logic unused_signal;
186   assign unused_signal = ^{jtag_i, tap_tl_h2d, tap_tl_d2h};
187 end else begin : gen_tap_tlul_ports
188   assign tap_dmi_tl_h2d = tap_tl_h2d;
189   assign tap_dmi_tl_d2h = tap_tl_d2h;
190   // Tie-off other port
191   assign dmi_tl_o = tlul_pkg::TL_D2H_DEFAULT;
192   logic unused_signal;
193   assign unused_signal = ^{dmi_tl_i};
194 end
195
196 lc_ctrl_regs_reg_top u_reg_tap_dmi (
197   .clk_i,
198   .rst_ni,
199   .tl_i    ( tap_dmi_tl_h2d ),
200   .tl_o    ( tap_dmi_tl_d2h ),
201   .reg2hw  ( tap_dmi_reg2hw ),
202   .hw2reg  ( tap_dmi_hw2reg ),
203   // SEC_CM: BUS.INTEGRITY
204
205   // While the TAP does not have bus integrity, it does have a
WE checker
206   // that feeds into integ_err_o - hence this is wired up to
the fatal_bus_integ_error.
207   .integ_err_o( fatal_bus_integ_error_tap_dmi_d )
208 );
209 if (!UseDmiInterface) begin : gen_tap_tlul
210   // This reuses the JTAG DTM and DMI from the RISC-V ex-
ternal
211
212   // debug v0.13 specification to read and write the lc_ctrl
CSRs:
213   // https://github.com/riscv/riscv-debug-spec/blob/release/
riscv-debug-release.pdf
214   // The register addresses correspond to the byte offsets

```

```

of the lc_ctrl CSRs, divided by 4.
214 // Note that the DMI reset does not affect the LC control-
ler in any way.
215 dm::dmi_req_t dmi_req;
216 logic dmi_req_valid;
217 logic dmi_req_ready;
218 dm::dmi_resp_t dmi_resp;
219 logic dmi_resp_ready;
220 logic dmi_resp_valid;
221
222 logic scanmode;
223 prim_mubi4_dec u_prim_mubi4_dec (
224   .mubi_i(scanmode_i),
225   .mubi_dec_o(scanmode)
226 );
227
228 logic tck_muxed;
229 logic trst_n_muxed;
230 prim_clock_mux2 #(
231   .NoFpgaBufG(1'b1)
232 ) u_prim_clock_mux2 (
233   .clk0_i(jtag_i.tck),
234   .clk1_i(clk_i),
235   .sel_i (scanmode),
236   .clk_o (tck_muxed)
237 );
238
239 prim_clock_mux2 #(
240   .NoFpgaBufG(1'b1)
241 ) u_prim_rst_n_mux2 (
242   .clk0_i(jtag_i.trst_n),
243   .clk1_i(scan_rst_ni),
244   .sel_i (scanmode),
245   .clk_o (trst_n_muxed)
246 );
247
248 logic req_ready;
249 assign req_ready = dmi_req_ready & dmi_resp_ready;
250 dmi_jtag #(
251   .IdcodeValue(IdcodeValue),
252   .NumDmiWordAbits(7)
253 ) u_dmi_jtag (
254   .clk_i,
255   .rst_ni,
256   .testmode_i    ( scanmode ),
257   .test_rst_ni   ( scan_rst_ni ),
258   .dmi_rst_no    ( ), // unused
259   .dmi_req_o     ( dmi_req ),
260   .dmi_req_valid_o ( dmi_req_valid ),
261   // unless there is room for response, stall
262   .dmi_req_ready_i ( req_ready ),
263   .dmi_resp_i      ( dmi_resp ),
264   .dmi_resp_ready_o ( dmi_resp_ready ),
265   .dmi_resp_valid_i ( dmi_resp_valid ),
266   .tck_i          ( tck_muxed ),
267   .tms_i          ( jtag_i.tms ),
268   .trst_ni        ( trst_n_muxed ),
269   .td_i           ( jtag_i.tdi ),
270   .td_o           ( jtag_o.tdo ),
271   .tdo_oe_o       ( jtag_o.tdo_oe )
272 );
273
274 // DMI to TL-UL transducing
275 tlul_adapter_host #(
276   .EnableDataIntgGen(1)
277 ) u_tap_tlul_host (
278   .clk_i,
279   .rst_ni,
280   // do not make a request unless there is room for the
response
281   .req_i    ( dmi_req_valid &
dmi_resp_ready ),
282   .gnt_o    ( ),
283   .addr_i   ( top_pkg::TL_AW'{{dmi_req.addr,
2'b00}} ),
284   .we_i     ( dmi_req.op ==
dm::DTM_WRITE ),
285   .wdata_i  ( ),
286   .wdata_intg_i ( ),
287   .be_i     ( ),
288   .user_rsvd_i ( {top_pkg::TL_DBW{1'b1}} ),
289   .instr_type_i ( prim_mubi_pkg::MuBi4False ),
290   .valid_o   ( ),
291   .rdata_o   ( ),
292   .rdata_intg_o ( ),
293   .err_o     ( ),
294   .integ_err_o ( ),
295   .tl_o      ( tap_tl_h2d ),
296   .tl_i      ( tap_tl_d2h )

```



```

297 );
298 // TL-UL to DMI transducing
299 assign dmi_resp.resp = '0; // unused inside dmi_jtag
300
301 // These signals are unused
302 logic unused_tap_tl_d2h;
303 assign unused_tap_tl_d2h = ^{
304   dmi_req.addr[31:30],
305   tap_tl_d2h.d_opcode,
306   tap_tl_d2h.d_param,
307   tap_tl_d2h.d_size,
308   tap_tl_d2h.d_source,
309   tap_tl_d2h.d_sink,
310   tap_tl_d2h.d_user,
311   tap_tl_d2h.d_error
312 };
313
314 end else begin : gen_dmi_tlul
315   // No scan chain needed in the DMI configuration
316   logic unused_signals;
317   assign unused_signals = ^{scan_rst_ni, scanmode_i};
318 end
319
320 //////////////////////////////////////////////////
321 // Transition Interface and HW Mutex //
322 //////////////////////////////////////////////////
323
324 // All registers are Hwext
325 logic trans_success_d, trans_success_q;
326 logic trans_cnt_oflw_error_d,
trans_cnt_oflw_error_q;
327 logic trans_invalid_error_d, trans_invalid_error_q;
328 logic token_invalid_error_d, token_invalid_error_q;
329 logic nvm_rma_error_d, nvm_rma_error_q;
330 logic otp_prog_error_d, fatal_prog_error_q;
331 logic state_invalid_error_d, fatal_state_error_q;
332 logic otp_part_error_q;
333 mubi8_t sw_claim_transition_if_d,
sw_claim_transition_if_q;
334 mubi8_t tap_dmi_claim_transition_if_d,
tap_dmi_claim_transition_if_q;
335 logic transition_cmd;
336 lc_token_t transition_token_d, transition_token_q;
337 ext_dec_lc_state_t transition_target_d, transition_target_q;
338 // No need to register these.
339 ext_dec_lc_state_t dec_lc_state;
340 dec_lc_cnt_t dec_lc_cnt;
341 dec_lc_id_state_e dec_lc_id_state;
342
343 logic lc_idle_d, lc_done_d;
344
345 // Create the hardware revision with the anchor const that
instantiates specific standard cells.
346 // This ensures the individual bits are not combined through
logic optimization and remain visible
347 // in the netlist, thereby enabling metal fixes to be re-
flected in the hardware revision.
348 localparam int HwRevWidth = $bits(lc_hw_rev_t);
349 localparam lc_hw_rev_t HwRev = '{silicon_creator_id: Silic-
onCreatorId,
350                                   product_id: Pro-
ductId,
351                                   revision_id: Revi-
sionId,
352                                   reserved: '0};
353
354 // We need to first cast HwRev to a logic array and then
back to lc_hw_rev_t
355 // This explicit cast is required because some tools misin-
terpret packed struct parameters as
356 // integers, leading to MSB truncation
357 logic [HwRevWidth-1:0] hw_rev_raw;
358 prim_const #(
359   .Width(HwRevWidth),
360   .ConstVal(HwRevWidth'(HwRev))
361 ) u_hw_rev_const (
362   .out_o(hw_rev_raw)
363 );
364 assign hw_rev_o = lc_hw_rev_t'(hw_rev_raw);
365
366 // OTP Vendor control bits
367 logic ext_clock_switched;
368 logic use_ext_clock_d, use_ext_clock_q;
369 logic volatile_raw_unlock_d, volatile_raw_unlock_q;
370 logic [CsrOtpTestCtrlWidth-1:0] otp_vendor_test_ctrl_d,
otp_vendor_test_ctrl_q;
371 logic [CsrOtpTestStatusWidth-1:0] otp_vendor_test_status;
372
373 always_comb begin : p_csr_assign_outputs
374   hw2reg = '0;
375   hw2reg.status.initialized = lc_done_d;
376   hw2reg.status.ready = lc_idle_d;
377   hw2reg.status.ext_clock_switched = ext_clock_switched;
378   hw2reg.status.transition_successful = trans_success_q;
379   hw2reg.status.transition_count_error =
trans_cnt_oflw_error_q;
380   hw2reg.status.transition_error =
trans_invalid_error_q;
381   hw2reg.status.token_error =
token_invalid_error_q;

```

```

382   hw2reg.status.nvm_rma_error = nvm_rma_error_q;
383   hw2reg.status.otp_error = fatal_prog_error_q;
384   hw2reg.status.state_error =
fatal_state_error_q;
385   hw2reg.status.otp_partition_error = otp_part_error_q;
386   hw2reg.status.bus_integ_error =
fatal_bus_integ_error_q;
387   hw2reg.lc_state = dec_lc_state;
388   hw2reg.lc_transition_cnt = dec_lc_cnt;
389   hw2reg.lc_id_state = {DecLcIdStateNum-
Rep{dec_lc_id_state}};
390   hw2reg.device_id = otp_device_id_i;
391   hw2reg.manuf_state = otp_manuf_state_i;
392   hw2reg.hw_revision0.silicon_creator_id = hw_rev_o.silic-
on_creator_id;
393   hw2reg.hw_revision0.product_id =
hw_rev_o.product_id;
394   hw2reg.hw_revision1.revision_id = hw_rev_o.revi-
sion_id;
395   hw2reg.hw_revision1.reserved = '0;
396
397   // The assignments above are identical for the TAP/DMI.
398   tap_dmi_hw2reg = hw2reg;
399
400   // Assignments gated by mutex. Again, the DMI has prior-
ity.
401   tap_dmi_hw2reg.claim_transition_if =
tap_dmi_claim_transition_if_q;
402   hw2reg.claim_transition_if = sw_claim_transition_if_q;
403   if (mubi8_test_true_strict(tap_dmi_claim_transition_if_q))
begin
404     tap_dmi_hw2reg.transition_ctrl.ext_clock_en = use_ext_c-
lock_q;
405     tap_dmi_hw2reg.transition_ctrl.volatile_raw_unlock =
volatile_raw_unlock_q;
406     tap_dmi_hw2reg.transition_token = transition_token_q;
407     tap_dmi_hw2reg.transition_target = transition_target_q;
408     // SEC_CM: TRANSITION.CONFIG.REGWEN
409     tap_dmi_hw2reg.transition_regwen = lc_idle_d;
410     tap_dmi_hw2reg.otp_vendor_test_ctrl =
otp_vendor_test_ctrl_q;
411     tap_dmi_hw2reg.otp_vendor_test_status =
otp_vendor_test_status;
412     end else if (mu-
bi8_test_true_strict(sw_claim_transition_if_q)) begin
413       hw2reg.transition_ctrl.ext_clock_en = use_ext_clock_q;
414       hw2reg.transition_ctrl.volatile_raw_unlock = volat-
ile_raw_unlock_q;
415       hw2reg.transition_token = transition_token_q;
416       hw2reg.transition_target = transition_target_q;
417       // SEC_CM: TRANSITION.CONFIG.REGWEN
418       hw2reg.transition_regwen = lc_idle_d;
419       hw2reg.otp_vendor_test_ctrl =
otp_vendor_test_ctrl_q;
420       hw2reg.otp_vendor_test_status =
otp_vendor_test_status;
421     end
422   end
423
424   always_comb begin : p_csr_assign_inputs
425     sw_claim_transition_if_d = sw_claim_transition_if_q;
426     tap_dmi_claim_transition_if_d =
tap_dmi_claim_transition_if_q;
427     transition_token_d = transition_token_q;
428     transition_target_d = transition_target_q;
429     transition_cmd = '1'b0;
430     otp_vendor_test_ctrl_d = otp_vendor_test_ctrl_q;
431     use_ext_clock_d = use_ext_clock_q;
432     volatile_raw_unlock_d = volatile_raw_unlock_q;
433
434     // Note that the mutex claims from the TAP/DMI and SW side
could arrive within the same cycle.
435     // In that case we give priority to the TAP/DMI mutex
claim in order to avoid a race condition.
436     // TAP/DMI mutex claim.
437     if (mubi8_test_false_loose(sw_claim_transition_if_q) &&
tap_dmi_reg2hw.claim_transition_if.qe) begin
438       tap_dmi_claim_transition_if_d = mu-
bi8_t'(tap_dmi_reg2hw.claim_transition_if.q);
439       // SW mutex claim.
440     end else if (mu-
bi8_test_false_loose(tap_dmi_claim_transition_if_q) &&
tap_dmi_reg2hw.claim_transition_if.qe) begin
441       sw_claim_transition_if_d = mu-
bi8_t'(reg2hw.claim_transition_if.q);
442     end
443
444     // The idle signal serves as the REGWEN in this case.
445     if (lc_idle_d) begin
446       // The DMI has priority.
447     end if (mu-
bi8_test_true_strict(tap_dmi_claim_transition_if_q)) begin
448       transition_cmd = tap_dmi_reg2hw.transition_cmd.q &
tap_dmi_reg2hw.transition_cmd.qe;
449     end if (tap_dmi_reg2hw.transition_ctrl.ext_clock_en.qe)
begin
450       use_ext_clock_d |=

```

```

tap_dmi_reg2hw.transition_ctrl.ext_clock_en.q;
456     end
457
458     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
459     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
460     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
461     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
462     //
-----
463     if
(tap_dmi_reg2hw.transition_ctrl.volatile_raw_unlock.q) begin
464         volatile_raw_unlock_d =
tap_dmi_reg2hw.transition_ctrl.volatile_raw_unlock.q;
465     end

466     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
467
468     for (int k = 0; k < LcTokenWidth/32; k++) begin
469         if (tap_dmi_reg2hw.transition_token[k].qe) begin
470             transition_token_d[k*32+: 32] =
tap_dmi_reg2hw.transition_token[k].q;
471         end
472     end
473
474     if (tap_dmi_reg2hw.transition_target.qe) begin
475         for (int k = 0; k < DecLcStateNumRep; k++) begin
476             transition_target_d[k] = dec_lc_state_e'(
477
tap_dmi_reg2hw.transition_target.q[k*DecLcStateWidth+: DecLc-
StateWidth]);
478         end
479     end
480
481     if (tap_dmi_reg2hw.otp_vendor_test_ctrl.qe) begin
482         otp_vendor_test_ctrl_d =
tap_dmi_reg2hw.otp_vendor_test_ctrl.q;
483     end
484     end else if (mu-
bi8_test_true_strict(sw_claim_transition_if_q)) begin
485         transition_cmd = reg2hw.transition_cmd.q &
reg2hw.transition_cmd.qe;
486     end
487
488     if (reg2hw.transition_ctrl.ext_clock_en.qe) begin
489         use_ext_clock_d |=
reg2hw.transition_ctrl.ext_clock_en.q;
490     end
491
492     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
493     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
494     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
495     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
496     //
-----
497     if (reg2hw.transition_ctrl.volatile_raw_unlock.qe) be-
gin
498         volatile_raw_unlock_d =
reg2hw.transition_ctrl.volatile_raw_unlock.q;
499     end

500     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
501
502     for (int k = 0; k < LcTokenWidth/32; k++) begin
503         if (reg2hw.transition_token[k].qe) begin
504             transition_token_d[k*32+: 32] =
reg2hw.transition_token[k].q;
505         end
506     end
507
508     if (reg2hw.transition_target.qe) begin
509         for (int k = 0; k < DecLcStateNumRep; k++) begin
510             transition_target_d[k] = dec_lc_state_e'(
511                 reg2hw.transition_target.q[k*DecLcStateWidth
+: DecLcStateWidth]);
512         end
513     end
514
515     if (reg2hw.otp_vendor_test_ctrl.qe) begin
516         otp_vendor_test_ctrl_d =
reg2hw.otp_vendor_test_ctrl.q;
517     end
518     end
519     end
520 end
521
522 always_ff @(posedge clk_i or negedge rst_ni) begin : p_csrs
523     if (!rst_ni) begin
524         trans_success_q          <= 1'b0;
525         trans_cnt_oflw_error_q   <= 1'b0;
526         trans_invalid_error_q    <= 1'b0;
527         token_invalid_error_q    <= 1'b0;
528         nvm_rma_error_q          <= 1'b0;

```

```

529         fatal_prog_error_q      <= 1'b0;
530         fatal_state_error_q      <= 1'b0;
531         sw_claim_transition_if_q  <= MuBi8False;
532         tap_dmi_claim_transition_if_q <= MuBi8False;
533         transition_token_q       <= 1'b0;
534         transition_target_q      <= {DecLcStateNum-
Rep{DecLcStw}};
535         otp_part_error_q         <= 1'b0;
536         fatal_bus_integ_error_q  <= 1'b0;
537         otp_vendor_test_ctrl_q   <= 1'b0;
538         use_ext_clock_q          <= 1'b0;
539     end else begin
540         // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
-----
541         // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
542         // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED
VIA
543         // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
544         //
-----
545         // In case of a volatile RAW unlock, this bit has to be
cleared when the volatile
546         // unlock is followed by a real transition.
547         // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
548         if (SecVolatileRawUnlockEn && transition_cmd && !volat-
ile_raw_unlock_q) begin
549             trans_success_q <= 1'b0;
550         end else begin
551             trans_success_q <= trans_success_d | trans_success_q;
552         end
553         // All other status and error bits are terminal and re-
quire a reset cycle.
554         trans_cnt_oflw_error_q <= trans_cnt_oflw_error_d |
trans_cnt_oflw_error_q;
555         trans_invalid_error_q <= trans_invalid_error_d |
trans_invalid_error_q;
556         token_invalid_error_q <= token_invalid_error_d |
token_invalid_error_q;
557         nvm_rma_error_q <= nvm_rma_error_d |
nvm_rma_error_q;
558         fatal_prog_error_q <= otp_prog_error_d |
fatal_prog_error_q;
559         fatal_state_error_q <= state_invalid_error_d |
fatal_state_error_q;
560         otp_part_error_q <= otp_lc_data_i.error |
otp_part_error_q;
561         fatal_bus_integ_error_q <= fatal_bus_integ_error_csr_d |
fatal_bus_integ_error_tap_dmi_d |
562         fatal_bus_integ_error_q;
563         // Other regs, gated by mutex further below.
564         sw_claim_transition_if_d <=
565         tap_dmi_claim_transition_if_q <=
566         tap_dmi_claim_transition_if_d;
567         transition_token_q <= transition_token_d;
568         transition_target_q <= transition_target_d;
569         otp_vendor_test_ctrl_q <= otp_vendor_test_ctrl_d;
570         use_ext_clock_q <= use_ext_clock_d;
571     end
572 end
573
574 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
-----
575 // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
ATE
576 // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
577 // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
578 //
-----
579 // If not enabled, this register will become a constant.
580 if (SecVolatileRawUnlockEn) begin : gen_volat-
ile_raw_unlock_reg
581     always_ff @(posedge clk_i or negedge rst_ni) begin :
p_volatile_raw_unlock_reg
582         if (!rst_ni) begin
583             volatile_raw_unlock_q <= 1'b0;
584         end else begin
585             volatile_raw_unlock_q <= volatile_raw_unlock_d;
586         end
587     end
588 end else begin : gen_volatile_raw_unlock_const
589     logic unused_volatile_raw_unlock;
590     assign unused_volatile_raw_unlock = ^volat-
ile_raw_unlock_d;
591     assign volatile_raw_unlock_q = 1'b0;
592 end
593 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
594
595 assign lc_nvm_rma_seed_o = trans-
ition_token_q[RmaSeedWidth-1:0];
596
597 // Gate the vendor specific test ctrl/status bits to zero in

```

```

production states.
598 // Buffer the enable signal to prevent optimization of the
multibit signal.
599 lc_tx_t lc_raw_test_rma;
600 lc_tx_t [1:0] lc_raw_test_rma_buf;
601 prim_lc_sync #(
602     .NumCopies(2),
603     .AsyncOn(0)
604 ) u_prim_lc_sync (
605     .clk_i,
606     .rst_ni,
607     .lc_en_i(lc_raw_test_rma),
608     .lc_en_o(lc_raw_test_rma_buf)
609 );
610
611 assign lc_raw_test_rma_o = lc_raw_test_rma;
612
613 assign lc_otp_vendor_test_o.ctrl =
(lc_tx_test_true_strict(lc_raw_test_rma_buf[0])) ?
614
615 otp_vendor_test_ctrl_q : '0';
616 assign otp_vendor_test_status =
(lc_tx_test_true_strict(lc_raw_test_rma_buf[1])) ?
617
618 lc_otp_vendor_test_i.status : '0';
619
620 // Alert Sender //
621
622 logic [NumAlerts-1:0] alerts;
623 logic [NumAlerts-1:0] alert_test;
624 logic [NumAlerts-1:0] tap_dmi_alert_test;
625
626 assign alerts = {
627     fatal_bus_integ_error_q,
628     fatal_state_error_q,
629     fatal_prog_error_q
630 };
631
632 assign alert_test = {
633     reg2hw.alert_test.fatal_bus_integ_error.q &
634     reg2hw.alert_test.fatal_bus_integ_error.qe,
635     reg2hw.alert_test.fatal_state_error.q &
636     reg2hw.alert_test.fatal_state_error.qe,
637     reg2hw.alert_test.fatal_prog_error.q &
638     reg2hw.alert_test.fatal_prog_error.qe
639 };
640
641 assign tap_dmi_alert_test = {
642     tap_dmi_reg2hw.alert_test.fatal_bus_integ_error.q &
643     tap_dmi_reg2hw.alert_test.fatal_bus_integ_error.qe,
644     tap_dmi_reg2hw.alert_test.fatal_state_error.q &
645     tap_dmi_reg2hw.alert_test.fatal_state_error.qe,
646     tap_dmi_reg2hw.alert_test.fatal_prog_error.q &
647     tap_dmi_reg2hw.alert_test.fatal_prog_error.qe
648 };
649
650 for (genvar k = 0; k < NumAlerts; k++) begin : gen_alert_tx
651     prim_alert_sender #(
652         .AsyncOn(AlertAsyncOn[k]),
653         .SkewCycles(AlertSkewCycles),
654         .IsFatal(1)
655     ) u_prim_alert_sender (
656         .clk_i,
657         .rst_ni,
658         .alert_test_i ( alert_test[k] |
659             tap_dmi_alert_test[k] ),
660         .alert_req_i ( alerts[k] ),
661         .alert_ack_o ( ),
662         .alert_state_o ( ),
663         .alert_rx_i ( alert_rx_i[k] ),
664         .alert_tx_o ( alert_tx_o[k] )
665     );
666 end
667
668 // Escalation Receivers //
669
670 // SEC_CM: MAIN.FSM.GLOBAL_ESC
671 // We still have two escalation receivers here for historic-
672 al reasons.
673 // The two actions "wipe secrets" and "scrap lifecycle
674 state" have been
675 // combined in order to simplify both DV and the design, as
676 otherwise
677 // this separation of very intertwined actions would have
678 caused too many
679 // unnecessary corner cases. The escalation receivers are
680 now redundant and
681 // trigger both actions at once.
682 // This escalation action moves the life cycle
683 // state into a temporary "SCRAP" state named "ESCALATE",
684 // and asserts the lc_escalate_en life cycle control signal.
685 logic esc_scrap_state0;
686 prim_esc_receiver #(
687     .N_ESC_SEV (EscNumSeverities),
688     .PING_CNT_DW (EscPingCountWidth),
689     .SkewCycles (AlertSkewCycles)
690 ) u_prim_esc_receiver0 (
691     .clk_i,
692     .rst_ni,
693     .esc_req_o (esc_scrap_state0),
694     .esc_rx_o (esc_scrap_state0_rx_o),
695     .esc_tx_i (esc_scrap_state0_tx_i)
696 );
697
698 // This escalation action moves the life cycle
699 // state into a temporary "SCRAP" state named "ESCALATE".
700 logic esc_scrap_state1;
701 prim_esc_receiver #(
702     .N_ESC_SEV (EscNumSeverities),
703     .PING_CNT_DW (EscPingCountWidth),
704     .SkewCycles (AlertSkewCycles)
705 ) u_prim_esc_receiver1 (
706     .clk_i,
707     .rst_ni,
708     .esc_req_o (esc_scrap_state1),
709     .esc_rx_o (esc_scrap_state1_rx_o),
710     .esc_tx_i (esc_scrap_state1_tx_i)
711 );
712
713 // Synchronization of IOs //
714
715 // Signals going to and coming from power manager.
716 logic lc_init;
717 prim_flop_2sync #(
718     .Width(1)
719 ) u_prim_flop_2sync_init (
720     .clk_i,
721     .rst_ni,
722     .d_i(pwr_lc_i.lc_init),
723     .q_o(lc_init)
724 );
725
726 logic lc_done_q;
727 logic lc_idle_q;
728
729 always_ff @(posedge clk_i or negedge rst_ni) begin :
p_sync_regs
730     if (!rst_ni) begin
731         lc_done_q <= 1'b0;
732         lc_idle_q <= 1'b0;
733     end else begin
734         lc_done_q <= lc_done_d;
735         lc_idle_q <= lc_idle_d;
736     end
737 end
738
739 assign pwr_lc_o.lc_done = lc_done_q;
740 assign pwr_lc_o.lc_idle = lc_idle_q;
741
742 // KMAC Interface //
743
744 logic token_hash_req, token_hash_req_chk, token_hash_ack,
token_hash_err, token_if_fsm_err;
745 lc_token_t hashed_token;
746 lc_ctrl_kmac_if u_lc_ctrl_kmac_if (
747     .clk_i,
748     .rst_ni,
749     .clk_kmac_i,
750     .rst_kmac_ni,
751     .kmac_data_i,
752     .kmac_data_o,
753     .transition_token_i ( transition_token_q ),
754     .token_hash_req_i ( token_hash_req ),
755     .token_hash_req_chk_i ( token_hash_req_chk ),
756     .token_hash_ack_o ( token_hash_ack ),
757     .token_hash_err_o ( token_hash_err ),
758     .token_if_fsm_err_o ( token_if_fsm_err ),
759     .hashed_token_o ( hashed_token )
760 );
761
762 // LC FSM //
763
764 // SEC_CM: MANUF.STATE.SPARSE
765 // SEC_CM: TRANSITION.CTR.SPARSE
766 lc_ctrl_fsm #(
767     .NumRmaAckSigs ( NumRmaAck-
768         ),
769     .RndCnstLcKeymgrDivInvalid ( RndCnstLcKeymgrDivInval-
770         id ),
771     .RndCnstLcKeymgrDivTestUn-
772         locked( RndCnstLcKeymgrDivTestUnlocked ),
773     .RndCnstLcKeymgrDivDev ( RndCnstLcKeymgrDi-
774         vDev ),
775     .RndCnstLcKeymgrDivProduction ( RndCnstLcKeymgrDivProduc-
776         tion ),
777     .RndCnstLcKeymgrDivRma ( RndCnstLcKeymgrDi-
778         vRma ),
779     .RndCnstInvalidTokens ( RndCnstInvalid-
780         Tokens ),
781     .SecVolatileRawUnlockEn ( SecVolatileRawUnlock-
782         En )
783 )

```

```

779 ) u_lc_ctrl_fsm (
780   .clk_i,
781   .rst_ni,
782   .init_req_i      ( lc_in-
it
783   .init_done_o
( lc_done_d          ),
784   .idle_o
( lc_idle_d          ),
785   .esc_scrap_state0_i
( esc_scrap_state0   ),
786   .esc_scrap_state1_i
( esc_scrap_state1   ),
787   .lc_state_valid_i
( otp_lc_data_i.valid ),
788   .lc_state_i
( lc_state_e'(otp_lc_data_i.state) ),
789   .secrets_valid_i
( otp_lc_data_i.secrets_valid ),
790   .lc_cnt_i
( lc_cnt_e'(otp_lc_data_i.count) ),
791   .use_ext_clock_i  ( use_ext_c-
lock_q
792   .ext_clock_switched_o ( ext_c-
lock_switched
793   .volatile_raw_unlock_i ( volat-
ile_raw_unlock_q
794   .strap_en_override_o,
795   .test_unlock_token_i
( otp_lc_data_i.test_unlock_token ),
796   .test_exit_token_i
( otp_lc_data_i.test_exit_token ),
797   .test_tokens_valid_i
( otp_lc_data_i.test_tokens_valid ),
798   .rma_token_i
( otp_lc_data_i.rma_token ),
799   .rma_token_valid_i
( otp_lc_data_i.rma_token_valid ),
800   .trans_cmd_i      ( trans-
ition_cmd
801   .trans_target_i   ( trans-
ition_target_q
802   .dec_lc_state_o
( dec_lc_state
803   .dec_lc_cnt_o
( dec_lc_cnt
804   .dec_lc_id_state_o
( dec_lc_id_state
805   .token_hash_req_o
( token_hash_req
806   .token_hash_req_chk_o
( token_hash_req_chk
807   .token_hash_ack_i
( token_hash_ack
808   .token_hash_err_i
( token_hash_err
809   .token_if_fsm_err_i
( token_if_fsm_err
810   .hashed_token_i
( hashed_token
811   .unhashed_token_i ( trans-
ition_token_q
812   .otp_prog_req_o   ( lc_otp_pro-
gram.o.req
813   .otp_prog_lc_state_o ( lc_otp_pro-
gram.o.state
814   .otp_prog_lc_cnt_o ( lc_otp_pro-
gram.o.count
815   .otp_prog_ack_i    ( lc_otp_pro-
gram.i.ack
816   .otp_prog_err_i    ( lc_otp_pro-
gram.i.err
817   .trans_success_o
( trans_success_d
818   .trans_cnt_oflw_error_o
( trans_cnt_oflw_error_d
819   .trans_invalid_error_o
( trans_invalid_error_d
820   .token_invalid_error_o
( token_invalid_error_d
821   .nvm_rma_error_o  ( nvm_rma_er-
ror_d
822   .otp_prog_error_o
( otp_prog_error_d
823   .state_invalid_error_o
( state_invalid_error_d
824   .lc_raw_test_rma_o
( lc_raw_test_rma
825   .lc_init_done_o,
826   .lc_dft_en_o,
827   .lc_nvm_debug_en_o,
828   .lc_hw_debug_clr_o,
829   .lc_hw_debug_en_o,
830   .lc_cpu_en_o,
831   .lc_creator_seed_sw_rw_en_o,
832   .lc_owner_seed_sw_rw_en_o,
833   .lc_iso_part_sw_rd_en_o,
834   .lc_iso_part_sw_wr_en_o,
835   .lc_seed_hw_rd_en_o,
836   .lc_rma_state_o,
837   .lc_keymgr_en_o,

```

```

838   .lc_escalate_en_o,
839   .lc_check_byp_en_o,
840   .lc_clk_byp_req_o,
841   .lc_clk_byp_ack_i,
842   .lc_nvm_rma_req_o,
843   .lc_nvm_rma_ack_i,
844   .lc_keymgr_div_o
845 );
846
847 //////////////////////////////////////////////////////////////////
848 // Assertions //
849 //////////////////////////////////////////////////////////////////
850
851 `ASSERT_KNOWN(RegsTl0Known,
regs_tl_o
852 `ASSERT_KNOWN(DmiTl0Known,
dmi_tl_o
853 `ASSERT_KNOWN(JtagTD0Known_A,
jtag_o
854 `ASSERT_KNOWN(AlertTxKnown_A,
alert_tx_o
855 `ASSERT_KNOWN(EscScrapState0Known_A,
esc_scrap_state0_rx_o
856 `ASSERT_KNOWN(EscScrapState1Known_A,
esc_scrap_state1_rx_o
857 `ASSERT_KNOWN(PwrLcKnown_A,
pwr_lc_o
858 `ASSERT_KNOWN(StrapEnOverrideKnown_A,
strap_en_override_o
859 `ASSERT_KNOWN(VendorTestReqKnown_A,
lc_otp_vendor_test_o
860 `ASSERT_KNOWN(LcOtpProgramKnown_A,   lc_otp_pro-
gram.o
861 `ASSERT_KNOWN(LcOtpTokenKnown_A,
kmac_data_o
862 `ASSERT_KNOWN(LcInitDoneKnown_A,   lc_in-
it_done_o
863 `ASSERT_KNOWN(LcDftEnKnown_A,
lc_dft_en_o
864 `ASSERT_KNOWN(LcRawTestRmaKnown_A,
lc_raw_test_rma_o
865 `ASSERT_KNOWN(LcNvmDebugEnKnown_A,   lc_nvm_de-
bug_en_o
866 `ASSERT_KNOWN(LcHwDebugClrKnown_A,   lc_hw_de-
bug_clr_o
867 `ASSERT_KNOWN(LcHwDebugEnKnown_A,   lc_hw_de-
bug_en_o
868 `ASSERT_KNOWN(LcCpuEnKnown_A,
lc_cpu_en_o
869 `ASSERT_KNOWN(LcCreatorSwRwEn_A,   lc_creat-
or_seed_sw_rw_en_o
870 `ASSERT_KNOWN(LcOwnerSwRwEn_A,   lc_own-
er_seed_sw_rw_en_o
871 `ASSERT_KNOWN(LcIsoSwRwEn_A,
lc_iso_part_sw_rd_en_o
872 `ASSERT_KNOWN(LcIsoSwWrEn_A,
lc_iso_part_sw_wr_en_o
873 `ASSERT_KNOWN(LcSeedHwRdEn_A,
lc_seed_hw_rd_en_o
874 `ASSERT_KNOWN(LcRmaState_A,
lc_rma_state_o
875 `ASSERT_KNOWN(LcKeymgrEnKnown_A,   lc_keym-
gr_en_o
876 `ASSERT_KNOWN(LcEscalateEnKnown_A,   lc_escal-
ate_en_o
877 `ASSERT_KNOWN(LcCheckBypassEnKnown_A,
lc_check_byp_en_o
878 `ASSERT_KNOWN(LcClkBypReqKnown_A,
lc_clk_byp_req_o
879 `ASSERT_KNOWN(LcNvmRmaSeedKnown_A,
lc_nvm_rma_seed_o
880 `ASSERT_KNOWN(LcNvmRmaReqKnown_A,
lc_nvm_rma_req_o
881 `ASSERT_KNOWN(LcKeymgrDiv_A,   lc_keym-
gr_div_o
882 `ASSERT_KNOWN(HwRevKnown_A,
hw_rev_o
883
884 `ASSERT(LcInitDoneSticky_A,
885   #1 !$fell(lc_tx_test_true_strict(lc_init_done_o)))
886
887 // Alert assertions for sparse FSMs.
888 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlLcFsmCheck_A,
u_lc_ctrl_fsm.u_fsm_state_regs, alert_tx_o[1])
889 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlLcStateCheck_A,
u_lc_ctrl_fsm.u_state_regs, alert_tx_o[1],
890   !$past(otp_lc_data_i.valid) ||
891   u_lc_ctrl_fsm.fsm_state_q inside {ResetSt, EscalateSt,
PostTransSt, InvalidSt, ScrapSt} ||
892   u_lc_ctrl_fsm.esc_scrap_state0_i ||
893   u_lc_ctrl_fsm.esc_scrap_state1_i)
894 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlLcCntCheck_A,
u_lc_ctrl_fsm.u_cnt_regs, alert_tx_o[1],
895   !$past(otp_lc_data_i.valid) ||
896   u_lc_ctrl_fsm.fsm_state_q inside {ResetSt, EscalateSt,
PostTransSt, InvalidSt, ScrapSt} ||
897   u_lc_ctrl_fsm.esc_scrap_state0_i ||
898   u_lc_ctrl_fsm.esc_scrap_state1_i)
899 `ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(CtrlKmacIfFsmCheck_A,
u_lc_ctrl_fsm.fsm_state_q inside {EscalateSt} ||
900   u_lc_ctrl_fsm.kmac_if.u_state_regs, alert_tx_o[1],
901   u_lc_ctrl_fsm.fsm_state_q inside {EscalateSt} ||
902   u_lc_ctrl_fsm.fsm_state_q inside {EscalateSt} ||
903   u_lc_ctrl_fsm.fsm_state_q inside {EscalateSt} ||
904   u_lc_ctrl_fsm.fsm_state_q inside {EscalateSt} ||

```

```

905     u_lc_ctrl_fsm.esc_scrap_state0_i ||
906     u_lc_ctrl_fsm.esc_scrap_state1_i)
907
908 // Alert assertions for reg_we onehot check
909 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegsWeOnehotCheck_A,
u_reg_regs, alert_tx_o[2])
910 `AS-
SERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(TapDmiWeOnehotCheck_A,
911 u_reg_tap_dmi, alert_tx_o[2], 0)
912 endmodule : lc_ctrl

```

2. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_dmi_reg_top.sv (60 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"
8
9 module lc_ctrl_dmi_reg_top (
10     input clk_i,
11     input rst_ni,
12     input tlul_pkg::tl_h2d_t tl_i,
13     output tlul_pkg::tl_d2h_t tl_o,
14
15     // Output port for window
16     output tlul_pkg::tl_h2d_t tl_win_o,
17     input tlul_pkg::tl_d2h_t tl_win_i,
18
19     // To HW
20
21     // Integrity check errors
22     output logic intg_err_o
23 );
24
25 import lc_ctrl_reg_pkg::* ;
26
27
28 // Add an unloaded flop to make use of clock / reset
29 // This is done to specifically address lint complaints of
  unused clocks/resets
30 // Since the flop is unloaded it will be removed during syn-
  thesis
31 logic unused_reg;
32 always_ff @(posedge clk_i or negedge rst_ni) begin
33     if (!rst_ni) begin
34         unused_reg <= '0;
35     end else begin
36         unused_reg <= tl_i.a_valid;
37     end
38 end
39
40
41 // Since there are no registers in this block, commands are
  routed through to windows which
42 // can report their own integrity errors.
43 assign intg_err_o = 1'b0;
44
45 // outgoing integrity generation
46 tlul_pkg::tl_d2h_t tl_o_pre;
47 tlul_rsp_intg_gen #(
48     .EnableRspIntgGen(1),
49     .EnableDataIntgGen(0)
50 ) u_rsp_intg_gen (
51     .tl_i(tl_o_pre),
52     .tl_o(tl_o)
53 );
54
55 assign tl_win_o = tl_i;
56 assign tl_o_pre = tl_win_i;
57
58 // Unused signal tieoff
59 endmodule

```

3. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_fsm.sv (893 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Main Life Cycle Controller FSM.
6
7 `include "prim_assert.sv"
8
9 module lc_ctrl_fsm
10     import lc_ctrl_pkg::*;
11     import lc_ctrl_reg_pkg::*;
12     import lc_ctrl_state_pkg::*;
13     #(

```

```

14     parameter int unsigned    NumRmaAckSigs          = 2,
15     // Random netlist constants
16     parameter lc_keymgr_div_t RndCnstLcKeymgrDivInvalid =
  LcKeymgrDivWidth'(0),
17     parameter lc_keymgr_div_t RndCnstLcKeymgrDivTestUnlocked =
  LcKeymgrDivWidth'(1),
18     parameter lc_keymgr_div_t RndCnstLcKeymgrDivDev      =
  LcKeymgrDivWidth'(2),
19     parameter lc_keymgr_div_t RndCnstLcKeymgrDivProduction =
  LcKeymgrDivWidth'(3),
20     parameter lc_keymgr_div_t RndCnstLcKeymgrDivRma      =
  LcKeymgrDivWidth'(4),
21     parameter lc_token_mux_t  RndCnstInvalidTokens      =
  {TokenMuxBits{1'b1}},
22     parameter bit              SecVolatileRawUnlockEn    = 0
23 ) (
24     // This module is combinational, but we
25     // need the clock and reset for the assertions.
26     input clk_i,
27     input rst_ni,
28     // Initialization request from power manager.
29     input init_req_i,
30     output logic init_done_o,
31     output logic idle_o,
32     // Escalation input
33     input esc_scrap_state0_i,
34     input esc_scrap_state1_i,
35     // Life cycle state vector from OTP.
36     input lc_state_valid_i,
37     input lc_state_e      lc_state_i,
38     input lc_cnt_e        lc_cnt_i,
39     input lc_tx_t          secrets_valid_i,
40     // Defines whether we switch to an external clock when ini-
  tiating a transition.
41     input use_ext_clock_i,
42     output logic ext_clock_switched_o,
43     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
44     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
  ATE
45     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
46     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
  DEVICES.
47     //
48     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
49     input logic volatile_raw_unlock_i,
50     output logic strap_en_override_o,
51     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
52     // Token input from OTP (these are all hash post-images).
53     input lc_token_t test_unlock_token_i,
54     input lc_token_t test_exit_token_i,
55     input lc_tx_t     test_tokens_valid_i,
56     input lc_token_t  rma_token_i,
57     input lc_tx_t     rma_token_valid_i,
58     // Transition trigger interface.
59     input trans_cmd_i,
60     input ext_dec_lc_state_t trans_target_i,
61     // Decoded life cycle state for CSRs.
62     output ext_dec_lc_state_t dec_lc_state_o,
63     output dec_lc_cnt_t      dec_lc_cnt_o,
64     output dec_lc_id_state_e dec_lc_id_state_o,
65     // Token hashing interface
66     output logic token_hash_req_o,
67     output logic token_hash_req_chk_o,
68     input token_hash_ack_i,
69     input token_hash_err_i,
70     input token_if_fsm_err_i,
71     input lc_token_t hashed_token_i,
72     input lc_token_t unhashed_token_i,
73     // OTP programming interface
74     output logic otp_prog_req_o,
75     output lc_state_e otp_prog_lc_state_o,
76     output lc_cnt_e   otp_prog_lc_cnt_o,
77     input otp_prog_ack_i,
78     input otp_prog_err_i,
79     // Error outputs going to CSRs
80     output logic trans_success_o,
81     output logic trans_cnt_oflw_error_o,
82     output logic trans_invalid_error_o,
83     output logic token_invalid_error_o,
84     output logic nvm_rma_error_o,
85     output logic otp_prog_error_o,
86     output logic state_invalid_error_o,
87     // Local life cycle signal
88     output lc_tx_t lc_raw_test_rma_o,
89     // Life cycle broadcast outputs.
90     output lc_tx_t lc_init_done_o,
91     output lc_tx_t lc_dft_en_o,
92     output lc_tx_t lc_nvm_debug_en_o,
93     output lc_tx_t lc_hw_debug_clr_o,
94     output lc_tx_t lc_hw_debug_en_o,
95     output lc_tx_t lc_cpu_en_o,
96     output lc_tx_t lc_creator_seed_sw_rw_en_o,
97     output lc_tx_t lc_owner_seed_sw_rw_en_o,
98     output lc_tx_t lc_iso_part_sw_rd_en_o,
99     output lc_tx_t lc_iso_part_sw_wr_en_o,
100    output lc_tx_t lc_seed_hw_rd_en_o,
101    output lc_tx_t lc_rma_state_o,
102    output lc_tx_t lc_keymgr_en_o,
103    output lc_tx_t lc_escalate_en_o,

```



```

103 output lc_tx_t lc_check_byp_en_o,
104 // Request and feedback to/from clock manager and AST.
105 output lc_tx_t lc_clk_byp_req_o,
106 input lc_tx_t lc_clk_byp_ack_i,
107 // Request and feedback to/from nvm controller
108 output lc_tx_t lc_nvm_rma_req_o,
109 input lc_tx_t [NumRmaAckSigs-1:0] lc_nvm_rma_ack_i,
110 // State group diversification value for keymgr
111 output lc_keymgr_div_t lc_keymgr_div_o
112 );
113 import lc_ctrl_token_pkg::*;
114
115 ///////////////////////////////////////////////////
116 // Synchronizers / Buffers //
117 ///////////////////////////////////////////////////
118
119 // We use multiple copies of these signals in the
120 // FSM checks below.
121 lc_tx_t [3:0] lc_clk_byp_ack;
122 prim_lc_sync #(
123     .NumCopies(4)
124 ) u_prim_lc_sync_clk_byp_ack (
125     .clk_i,
126     .rst_ni,
127     .lc_en_i(lc_clk_byp_ack_i),
128     .lc_en_o(lc_clk_byp_ack)
129 );
130
131 // Indication for CSRs
132 assign ext_clock_switched_o =
lc_tx_test_true_strict(lc_clk_byp_ack[3]);
133
134 // We have multiple response channels for this signal since
the RMA wiping requests can go to
135 // multiple modules that perform wiping in parallel. For security
reasons, this signal is not
136 // daisy-chained - see #19136 for context. Synchronize ACK
signals
separately, combine with
137 // bitwise LC AND function and feed into FSM.
138 lc_tx_t [NumRmaAckSigs-1:0] lc_nvm_rma_ack;
139 for (genvar k = 0; k < NumRmaAckSigs; k++) begin : gen_syncs
140     prim_lc_sync #(
141         .NumCopies(1)
142     ) u_prim_lc_sync_nvm_rma_ack(
143         .clk_i,
144         .rst_ni,
145         .lc_en_i(lc_nvm_rma_ack_i[k]),
146         .lc_en_o({lc_nvm_rma_ack[k]})
147     );
148 end
149
150 lc_tx_t lc_nvm_rma_ack_combined;
151 always_comb begin
152     lc_nvm_rma_ack_combined = 0n;
153     for (int k = 0; k < NumRmaAckSigs; k++) begin
154         lc_nvm_rma_ack_combined =
lc_tx_and_hi(lc_nvm_rma_ack_combined, lc_nvm_rma_ack[k]);
155     end
156 end
157
158 // Make buffered copies for consumption in the FSM below.
159 lc_tx_t [2:0] lc_nvm_rma_ack_buf;
160 prim_lc_sync #(
161     .NumCopies(3),
162     .AsyncOn(0)
163 ) u_prim_lc_sync_nvm_rma_ack_buf (
164     .clk_i,
165     .rst_ni,
166     .lc_en_i(lc_nvm_rma_ack_combined),
167     .lc_en_o(lc_nvm_rma_ack_buf)
168 );
169
170 ///////////////////////////////////////////////////
171 // FSM Logic //
172 ///////////////////////////////////////////////////
173 fsm_state_e fsm_state_d, fsm_state_q;
174
175 // Continuously feed in valid signal for LC state.
176 logic lc_state_valid_d, lc_state_valid_q;
177 assign lc_state_valid_d = lc_state_valid_i;
178
179 // Encoded state vector.
180 lc_state_e lc_state_d, lc_state_q, next_lc_state;
181 lc_cnt_e lc_cnt_d, lc_cnt_q, next_lc_cnt;
182
183 // Feed the next lc state reg back to the programming inter-
face of OTP.
184 assign otp_prog_lc_state_o = next_lc_state;
185 assign otp_prog_lc_cnt_o = next_lc_cnt;
186
187 // Conditional LC signal outputs
188 lc_tx_t lc_clk_byp_req, lc_nvm_rma_req, lc_check_byp_en;
189
190 `ASSERT_KNOWN(LcStateKnown_A, lc_state_q )
191 `ASSERT_KNOWN(LcCntKnown_A, lc_cnt_q )
192 `ASSERT_KNOWN(FsmStateKnown_A, fsm_state_q )
193
194 // Hashed token to compare against.
195 logic [1:0] hashed_token_valid_mux;
196 lc_token_t hashed_token_mux;
197
198 // Multibit state error from state decoder
199 logic [5:0] state_invalid_error;
200
201 // Strap sample override signal.
202 logic set_strap_en_override;
203
204 // Registers whether volatile unlock has been successful
205 prim_mubi_pkg::mubi8_t volatile_raw_unlock_success_d, volatil-
ile_raw_unlock_success_q;
206
207 // SEC_CM: MAIN.CTRL_FLOW.CONSTISTENCY
208 always_comb begin : p_fsm
209     // FSM default state assignments.
210     fsm_state_d = fsm_state_q;
211     lc_state_d = lc_state_q;
212     lc_cnt_d = lc_cnt_q;
213
214     // Token hashing.
215     token_hash_req_o = 1'b0;
216     token_hash_req_chk_o = 1'b1;
217
218     // OTP Interface
219     otp_prog_req_o = 1'b0;
220
221     // Defaults for status/error signals.
222     token_invalid_error_o = 1'b0;
223     otp_prog_error_o = 1'b0;
224     nvm_rma_error_o = 1'b0;
225     trans_success_o = 1'b0;
226     state_invalid_error_o = 1'b0;
227
228     // Status indication going to power manager.
229     init_done_o = 1'b1;
230     idle_o = 1'b0;
231
232     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
233     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MIT-
IGATE
234     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED
VIA
235     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
236     //
237     set_strap_en_override = 1'b0;
238     volatile_raw_unlock_success_d = volatil-
ile_raw_unlock_success_q;
239     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
240
241     // These signals remain asserted once set to On.
242     // Note that the remaining life cycle signals are decoded
in
243     // the lc_ctrl_signal_decode submodule.
244     lc_clk_byp_req = lc_clk_byp_req_o;
245     lc_nvm_rma_req = lc_nvm_rma_req_o;
246     lc_check_byp_en = lc_check_byp_en_o;
247
248     unique case (fsm_state_q)
249     ///////////////////////////////////////////////////
250     // Wait here until OTP has initialized and the
251     // power manager sends an initialization request.
252     ResetSt: begin
253         init_done_o = 1'b0;
254         lc_clk_byp_req = Off;
255         lc_nvm_rma_req = Off;
256         lc_check_byp_en = Off;
257         if (init_req_i && lc_state_valid_q) begin
258             fsm_state_d = IdleSt;
259             // Fetch LC state vector from OTP.
260             lc_state_d = lc_state_i;
261             lc_cnt_d = lc_cnt_i;
262         end
263     end
264     ///////////////////////////////////////////////////
250     // Idle state where life cycle control signals are
broadcast.
266     // Note that the life cycle signals are decoded and
broadcast
267     // in the lc_ctrl_signal_decode submodule.
268     IdleSt: begin
269         idle_o = 1'b1;
270
271     // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
272     // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
273     // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
274     // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
275     //
276     // Note that if the volatile unlock mechanism is
available,
277     // we have to stop fetching the OTP value after a
volatile unlock has succeeded.

```

```

278 // Otherwise we unconditionally fetch from OTP in this
state.
279 if (!(SecVolatileRawUnlockEn && lc_state_q == LcStTest-
Unlocked0 && lc_cnt_q != LcCnt0) ||
280 prim_mubi_pkg::mubi8_test_false_loose(volatile_raw_unlock_success_q)
) begin
281 // Continuously fetch LC state vector from OTP.
282 // The state is locked in once a transition is star-
ted.
283 lc_state_d = lc_state_i;
284 lc_cnt_d = lc_cnt_i;
285 end

286 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
287
288 // If the life cycle state is SCRAP, we move the FSM
into a terminal
289 // SCRAP state that does not allow any transitions to
be initiated anymore.
290 if (lc_state_q == LcStScrap) begin
291 fsm_state_d = ScrapSt;
292
293 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION
START -----
294 // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO
MITIGATE
295 // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DIS-
ABLED VIA
296 // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUC-
TION DEVICES.
297 //
-----

298 // Only enter here if volatile RAW unlock is available
and enabled.
299 end else if (SecVolatileRawUnlockEn && volat-
ile_raw_unlock_i && trans_cmd_i) begin
300 // We only allow transitions from RAW -> TEST_UN-
LOCKED0
301 if (lc_state_q == LcStRaw &&
trans_target_i == {DecLcStateNum-
Rep{DecLcStTestUnlocked0}} &&
302 !trans_invalid_error_o) begin
303 // 128bit token check (without passing it through
the KMAC)
304 if (unhashed_token_i == RndCnstRawUnlockToken-
Hashed) begin
305 // We stay in Idle, but update the life cycle
state register (volatile).
306 lc_state_d = LcStTestUnlocked0;
307
308 // If the count is 0, we set it to 1 - otherwise
we just leave it as is so that the
309 // register value is in sync with what has been
programmed to OTP already (there may
310 // have been unsuccessful raw unlock attempts be-
fore that already incremented it).
311 lc_cnt_d = (lc_cnt_q == LcCnt0) ? LcCnt1 :
lc_cnt_q;
312 // Re-sample the DFT straps in the pinmux.

313 // This signal will be delayed by several cycles
so that the LC_CTRL signals
314 // have time to propagate.
315 set_strap_en_override = 1'b1;
316 // We have to remember that the transition was
successful in order
317 // to correctly
318 // disable the continuous sampling of the life
cycle state vector coming from OTP.
319 volatile_raw_unlock_success_d =
prim_mubi_pkg::MuBi8True;
320 // Indicate that the transition was successful.
321 trans_success_o = 1'b1;
322 end else begin
323 token_invalid_error_o = 1'b1;
324 fsm_state_d = PostTransSt;
325 end
326 end else begin
327 // Transition invalid error is set by lc_c-
trl_state_transition module.
328 fsm_state_d = PostTransSt;
329 end

329 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
-----
330 // Initiate a transition. This will first increment
the
331 // life cycle counter before hashing and checking the
token.
332 end else if (trans_cmd_i) begin
333 fsm_state_d = ClkMuxSt;
334 end

335 // If we are in a non-PROD life cycle state, steer the
clock mux if requested. This
336 // action is available in IdleSt so that the mux can
be steered without having to initiate
337 // a life cycle transition. If a transition is initi-
ated however, the life cycle controller

```

```

338 // will wait for the clock mux acknowledgement in the
ClkMuxSt state before proceeding.
339 if (lc_state_q inside {LcStRaw,
340 LcStTestLocked0,
341 LcStTestLocked1,
342 LcStTestLocked2,
343 LcStTestLocked3,
344 LcStTestLocked4,
345 LcStTestLocked5,
346 LcStTestLocked6,
347 LcStTestUnlocked0,
348 LcStTestUnlocked1,
349 LcStTestUnlocked2,
350 LcStTestUnlocked3,
351 LcStTestUnlocked4,
352 LcStTestUnlocked5,
353 LcStTestUnlocked6,
354 LcStTestUnlocked7,
355 LcStRma}) begin
356 if (use_ext_clock_i) begin
357 lc_clk_byp_req = 0n;
358 end
359 end
360 end
361 //////////////////////////////////////////////////
362 // Clock mux state. If we are in RAW, TEST* or RMA, it
is permissible
363 // to switch to an external clock source. If the bypass
request is
364 // asserted, we have to wait until the clock mux and
clock manager
365 // have switched the mux and the clock divider. Also, we
disable the
366 // life cycle partition checks at this point since we
are going to
367 // alter the contents in the OTP memory array, which
could lead to
368 // spurious escalations.
369 ClkMuxSt: begin
370 lc_check_byp_en = 0n;
371 if (lc_state_q inside {LcStRaw,
372 LcStTestLocked0,
373 LcStTestLocked1,
374 LcStTestLocked2,
375 LcStTestLocked3,
376 LcStTestLocked4,
377 LcStTestLocked5,
378 LcStTestLocked6,
379 LcStTestUnlocked0,
380 LcStTestUnlocked1,
381 LcStTestUnlocked2,
382 LcStTestUnlocked3,
383 LcStTestUnlocked4,
384 LcStTestUnlocked5,
385 LcStTestUnlocked6,
386 LcStTestUnlocked7,
387 LcStRma}) begin
388 if (use_ext_clock_i) begin
389 lc_clk_byp_req = 0n;
390 if (lc_tx_test_true_strict(lc_clk_byp_ack[0])) be-
gin
391 fsm_state_d = CntIncrSt;
392 end
393 end else begin
394 fsm_state_d = CntIncrSt;
395 end
396 end else begin
397 fsm_state_d = CntIncrSt;
398 end
399 end
400 //////////////////////////////////////////////////
401 // This increments the life cycle counter state.
402 CntIncrSt: begin
403 // If the counter has reached the maximum, bail out.
404 if (trans_cnt_oflw_error_o) begin
405 fsm_state_d = PostTransSt;
406 end else begin
407 fsm_state_d = CntProgSt;
408 end
409 end
410 //////////////////////////////////////////////////
411 // This programs the life cycle counter state.
412 CntProgSt: begin
413 otp_prog_req_o = 1'b1;
414
415 // If the clock mux has been steered, double check
that this is still the case.
416 // Otherwise abort the transition operation.
417 if (lc_clk_byp_req_o != lc_clk_byp_ack[1]) begin
418 fsm_state_d = PostTransSt;
419 otp_prog_error_o = 1'b1;
420 end
421
422 // Check return value and abort if there
// was an error.
423 if (otp_prog_ack_i) begin
424 if (otp_prog_err_i) begin
425

```



```

426         fsm_state_d = PostTransSt;
427         otp_prog_error_o = 1'b1;
428     end else begin
429         fsm_state_d = TransCheckSt;
430     end
431 end
432 end
433 //////////////////////////////////////////////////
434 // First transition valid check. This will be repeated
several
435 // times below.
436 TransCheckSt: begin
437     if (trans_invalid_error_o) begin
438         fsm_state_d = PostTransSt;
439     end else begin
440         fsm_state_d = TokenHashSt;
441     end
442 end
443 //////////////////////////////////////////////////
444 // Hash and compare the token, no matter whether this
transition
445 // is conditional or not. Unconditional transitions just
use a known
446 // all-zero token value. That way, we always compare a
hashed token
447 // and guarantee that no other control flow path exists
that could
448 // bypass the token check.
449 // SEC_CM: TOKEN.DIGEST
450 TokenHashSt: begin
451     token_hash_req_o = 1'b1;
452     if (token_hash_ack_i) begin
453         // This is the first comparison.
454         // The token is compared two more times further be-
low.
455         // Also note that conditional transitions won't be
possible if the
456         // corresponding token is not valid. This only ap-
plies to tokens
457         // stored in
458         // OTP. I.e., these tokens first have to be provi-
sioned, before
459         // they can be used.
460         if (hashed_token_i == hashed_token_mux &&
461             !token_hash_err_i &&
462             &hashed_token_valid_mux) begin
463             fsm_state_d = NvmRmaSt;
464         end else begin
465             fsm_state_d = PostTransSt;
466             token_invalid_error_o = 1'b1;
467         end
468     end
469 //////////////////////////////////////////////////
470 // NVM RMA state. Note that we check the nvm response
again
471 // two times later below.
472 NvmRmaSt: begin
473     if (trans_target_i == {DecLcStateNumRep{DecLcStRma}})
begin
474         lc_nvm_rma_req = 0n;
475         if (lc_tx_test_true_strict(lc_nvm_rma_ack_buf[0]))
begin
476             fsm_state_d = TokenCheck0St;
477         end
478     end else begin
479         fsm_state_d = TokenCheck0St;
480     end
481 //////////////////////////////////////////////////
482 // Check again two times whether this transition and the
hashed
483 // token are valid. Also check again whether the nvm RMA
484 // response is valid.
485 // SEC_CM: TOKEN.DIGEST
486 TokenCheck0St:
487 TokenCheck1St: begin
488     if (trans_invalid_error_o) begin
489         fsm_state_d = PostTransSt;
490     end else begin
491         // If any of these RMA are conditions are true,
492         // all of them must be true at the same time.
493         if ((trans_target_i != {DecLcStateNum-
Rep{DecLcStRma}} &&
494             lc_tx_test_false_strict(lc_nvm_rma_req_o) &&
495             lc_tx_test_false_strict(lc_nvm_rma_ack_buf[1]))
||
496             (trans_target_i == {DecLcStateNum-
Rep{DecLcStRma}} &&
497             lc_tx_test_true_strict(lc_nvm_rma_req_o) &&
498             lc_tx_test_true_strict(lc_nvm_rma_ack_buf[1])))
begin
499             if (hashed_token_i == hashed_token_mux &&
500                 !token_hash_err_i &&
501                 &hashed_token_valid_mux) begin

```

```

502         if (fsm_state_q == TokenCheck1St) begin
503             // This is the only way we can get into the
504             // programming state.
505             fsm_state_d = TransProgSt;
506         end else begin
507             fsm_state_d = TokenCheck1St;
508         end
509     end else begin
510         fsm_state_d = PostTransSt;
511         token_invalid_error_o = 1'b1;
512     end
513 // The NVM RMA process failed.
514 end else begin
515     fsm_state_d = PostTransSt;
516     nvm_rma_error_o = 1'b1;
517 end
518 end
519 end
520 //////////////////////////////////////////////////
521 // Initiate OTP transaction. Note that the concurrent
522 // LC state check is continuously checking whether the
523 // new LC state remains valid. Once the ack returns we
are
524 // done with the transition and can go into the terminal
PostTransSt.
525 TransProgSt: begin
526     otp_prog_req_o = 1'b1;
527
528     // If the clock mux has been steered, double check
that this is
529     // still the case.
530     // Otherwise abort the transition operation.
531     if (lc_clk_byp_req_o != lc_clk_byp_ack[2]) begin
532         fsm_state_d = PostTransSt;
533         otp_prog_error_o = 1'b1;
534     // Also double check that the RMA signals remain
stable.
535     // Otherwise abort the transition operation.
536     end else if ((trans_target_i != {DecLcStateNum-
Rep{DecLcStRma}} &&
537         (lc_nvm_rma_req_o != Off ||
538         lc_nvm_rma_ack_buf[2] != Off)) ||
539         (trans_target_i == {DecLcStateNum-
Rep{DecLcStRma}} &&
540         (lc_nvm_rma_req_o != On ||
541         lc_nvm_rma_ack_buf[2] != On))) begin
542         fsm_state_d = PostTransSt;
543         nvm_rma_error_o = 1'b1;
544     end else if (otp_prog_ack_i) begin
545         fsm_state_d = PostTransSt;
546         otp_prog_error_o = otp_prog_err_i;
547         trans_success_o = ~otp_prog_err_i;
548     end
549 //////////////////////////////////////////////////
550 // Terminal states.
551 ScrapSt,
552 PostTransSt: ;
553
554 EscalateSt: begin
555     // During an escalation it is okay to de-assert
token_hash_req
556     // without receiving ACK.
557     token_hash_req_chk_o = 1'b0;
558 end
559 InvalidSt: begin
560     // During an escalation it is okay to de-assert
token_hash_req
561     // without receiving ACK.
562     token_hash_req_chk_o = 1'b0;
563     state_invalid_error_o = 1'b1;
564 end
565 //////////////////////////////////////////////////
566 // Go to terminal error state if we get here.
567 default: begin
568     fsm_state_d = InvalidSt;
569     state_invalid_error_o = 1'b1;
570 end
571 //////////////////////////////////////////////////
572 endcase
573
574 // SEC_CM: MAIN.FSM.GLOBAL_ESC
575 if (esc_scrap_state0_i || esc_scrap_state1_i) begin
576     fsm_state_d = EscalateSt;
577 // SEC_CM: MAIN.FSM.LOCAL_ESC
578 // If at any time the life cycle state encoding or any
other FSM
579 // state within this module
580 // is not valid, we jump into the terminal error state
right away.
581 // Note that state_invalid_error is a multibit error sig-
nal
582 // with different error sources - need to reduce this to
one bit here.
583 end else if (([state_invalid_error | token_if_fsm_err_i]
&& (fsm_state_q != EscalateSt))) begin
584     fsm_state_d = InvalidSt;
585     state_invalid_error_o = 1'b1;

```

```

583     end
584 end
585
586 ////////////////
587 // State Flops //
588 ////////////////
589
590 `PRIM_FLOP_SPARSE_FSM(u_fsm_state_regs, fsm_state_d,
fsm_state_q, fsm_state_e, ResetSt)
591 `PRIM_FLOP_SPARSE_FSM(u_state_regs, lc_state_d, lc_state_q,
lc_state_e, LcStScrap)
592 `PRIM_FLOP_SPARSE_FSM(u_cnt_regs, lc_cnt_d, lc_cnt_q,
lc_cnt_e, LcCnt24)
593
594 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
595   if (!rst_ni) begin
596     lc_state_valid_q <= 1'b0;
597   end else begin
598     lc_state_valid_q <= lc_state_valid_d;
599   end
600 end
601
602 // ----- VOLATILE_TEST_UNLOCKED CODE SECTION START
603
604 // NOTE THAT THIS IS A FEATURE FOR TEST CHIPS ONLY TO MITIG-
ATE
605 // THE RISK OF A BROKEN OTP MACRO. THIS WILL BE DISABLED VIA
606 // SecVolatileRawUnlockEn AT COMPILETIME FOR PRODUCTION
DEVICES.
607 //
608 // if (SecVolatileRawUnlockEn) begin : gen_strap_delay_regs
609 //   The delay on the life cycle signals is 1 sender + 2 re-
ceiver domain
610 //   cycles. We are delaying this cycle several cycles more
than that so
611 //   that the life cycle signals have time to propagate (for
good measure).
612 //   localparam int NumStrapDelayRegs = 10;
613 //   logic [NumStrapDelayRegs-1:0] strap_en_override_q;
614 //   always_ff @(posedge clk_i or negedge rst_ni) begin :
p_volatile_raw_unlock_reg
615 //     if (!rst_ni) begin
616 //       strap_en_override_q <= '0;
617 //       volatile_raw_unlock_success_q <=
prim_mubi_pkg::MuB18False;
618 //     end else begin
619 //       strap_en_override_q <=
{strap_en_override_q[NumStrapDelayRegs-2:0],
620 //       // This is a set-reg that will
stay high until the next reset.
621 //       set_strap_en_override ||
strap_en_override_q[0]};
622 //       volatile_raw_unlock_success_q <= volat-
ile_raw_unlock_success_d;
623 //     end
624 //   end
625 //   assign strap_en_override_o =
strap_en_override_q[NumStrapDelayRegs-1];
626 //   end else begin : gen_no_strap_delay_regs
627 //     // In this case we tie the strap sampling off.
628 //     logic unused_sigs;
629 //     assign unused_sigs = ^{set_strap_en_override,
630 //     volatile_raw_unlock_success_d};
631 //     assign strap_en_override_o = 1'b0;
632 //     assign volatile_raw_unlock_success_q =
prim_mubi_pkg::MuB18False;
633 //   end
634 //   // ----- VOLATILE_TEST_UNLOCKED CODE SECTION END
635
636 ////////////////
637 // Token mux //
638 ////////////////
639
640 lc_ctrl_pkg::lc_tx_t [3:0] rma_token_valid;
641 prim_lc_sync #(
642   .NumCopies(4),
643   .AsyncOn(0),
644   .ResetValueIsOn(0)
645 ) u_prim_lc_sync_rma_token_valid (
646   .clk_i,
647   .rst_ni,
648   .lc_en_i(rma_token_valid_i),
649   .lc_en_o(rma_token_valid)
650 );
651
652 lc_ctrl_pkg::lc_tx_t [7:0] test_tokens_valid;
653 prim_lc_sync #(
654   .NumCopies(8),
655   .AsyncOn(0),
656   .ResetValueIsOn(0)
657 ) u_prim_lc_sync_test_token_valid (
658   .clk_i,
659   .rst_ni,
660   .lc_en_i(test_tokens_valid_i),
661   .lc_en_o(test_tokens_valid)
662 );

```

```

663
664 // SEC_CM: TOKEN_MUX.CTRL.REDUN
665 // The token mux is split into two halves for which we use
separate mux select signals
666 // that have both been generated from separately buffered
multibit lifecycle signals.
667 logic [2**TokenIdxWidth-1:0][LcTokenWidth/2-1:0]
hashed_tokens_lower, hashed_tokens_upper;
668 // These helper signals are only there to increase readabil-
ity of the mux code below.
669 logic [LcTokenWidth/2-1:0] test_unlock_token_lower, test_un-
lock_token_upper;
670 logic [LcTokenWidth/2-1:0] test_exit_token_lower,
test_exit_token_upper;
671 logic [LcTokenWidth/2-1:0] rma_token_lower, rma_token_upper;
672 assign {test_unlock_token_lower, test_unlock_token_upper} =
test_unlock_token_i;
673 assign {test_exit_token_lower, test_exit_token_upper} =
test_exit_token_i;
674 assign {rma_token_lower, rma_token_upper} =
rma_token_i;
675
676 // SEC_CM: TOKEN.DIGEST
677 // This indexes the correct token, based on the transition
arc.
678 // Note that we always perform a token comparison, even in
case of
679 // unconditional transitions. In the case of unconditional
tokens
680 // we just pass an all-zero constant through the hashing
function.
681 always_comb begin : p_token_assign
682 // Set the invalid token indices to a random netlist con-
stant, rather than all-zero.
683 {hashed_tokens_lower, hashed_tokens_upper} = RndCnstInval-
idTokens;
684 // All-zero token for unconditional transitions.
685 {hashed_tokens_lower[ZeroTokenIdx],
686 hashed_tokens_upper[ZeroTokenIdx]} = AllZeroTokenHashed;
687 {hashed_tokens_lower[RawUnlockTokenIdx],
688 hashed_tokens_upper[RawUnlockTokenIdx]} = RndCnstRawUn-
lockTokenHashed;
689 // This mux has two separate halves, steered with separ-
ately buffered life cycle signals.
690 if (lc_tx_test_true_strict(test_tokens_valid[0])) begin
691 hashed_tokens_lower[TestUnlockTokenIdx] = test_un-
lock_token_lower;
692 end
693 if (lc_tx_test_true_strict(test_tokens_valid[1])) begin
694 hashed_tokens_upper[TestUnlockTokenIdx] = test_un-
lock_token_upper;
695 end
696 // This mux has two separate halves, steered with separ-
ately buffered life cycle signals.
697 if (lc_tx_test_true_strict(test_tokens_valid[2])) begin
698 hashed_tokens_lower[TestExitTokenIdx] =
test_exit_token_lower;
699 end
700 if (lc_tx_test_true_strict(test_tokens_valid[3])) begin
701 hashed_tokens_upper[TestExitTokenIdx] =
test_exit_token_upper;
702 end
703 // This mux has two separate halves, steered with separ-
ately buffered life cycle signals.
704 if (lc_tx_test_true_strict(rma_token_valid[0])) begin
705 hashed_tokens_lower[RmaTokenIdx] = rma_token_lower;
706 end
707 if (lc_tx_test_true_strict(rma_token_valid[1])) begin
708 hashed_tokens_upper[RmaTokenIdx] = rma_token_upper;
709 end
710 end
711
712 // SEC_CM: TOKEN_VALID.MUX.REDUN
713 // The token valid mux is duplicated.
714 logic [TokenIdxWidth-1:0] token_idx0, token_idx1;
715 logic [2**TokenIdxWidth-1:0] hashed_tokens_valid0,
hashed_tokens_valid1;
716 always_comb begin : p_token_valid_assign
717 // First mux
718 hashed_tokens_valid0
719 hashed_tokens_valid0[ZeroTokenIdx] = 1'b1; // always
valid
720 hashed_tokens_valid0[RawUnlockTokenIdx] = 1'b1; // always
valid
721 hashed_tokens_valid0[TestUnlockTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[4]);
722 hashed_tokens_valid0[TestExitTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[5]);
723 hashed_tokens_valid0[RmaTokenIdx] =
lc_tx_test_true_strict(rma_token_valid[2]);
724 hashed_tokens_valid0[InvalidTokenIdx] = 1'b0; // always
invalid
725 // Second mux
726 hashed_tokens_valid1
727 hashed_tokens_valid1[ZeroTokenIdx] = 1'b1; // always
valid

```

```

728 hashed_tokens_valid1[RawUnlockTokenIdx] = 1'b1; // always
valid
729 hashed_tokens_valid1[TestUnlockTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[6]);
730 hashed_tokens_valid1[TestExitTokenIdx] =
lc_tx_test_true_strict(test_tokens_valid[7]);
731 hashed_tokens_valid1[RmaTokenIdx] =
lc_tx_test_true_strict(rma_token_valid[3]);
732 hashed_tokens_valid1[InvalidTokenIdx] = 1'b0; // always
invalid
733 end
734
735 // SEC_CM: STATE.CONFIG.SPARSE
736 // The trans_target_i signal comes from the CSR and uses a
replication encoding,
737 // hence we can use different indices of the array.
738 assign token_idx0 = (int'(dec_lc_state_o[0]) < NumLcStates
&&
739 int'(trans_target_i[0]) < NumLc-
States) ?
740 TransTokenIdxMatrix[dec_lc_state_o[0]]
[trans_target_i[0]] :
741 InvalidTokenIdx;
742 assign token_idx1 = (int'(dec_lc_state_o[1]) < NumLcStates
&&
743 int'(trans_target_i[1]) < NumLc-
States) ?
744 TransTokenIdxMatrix[dec_lc_state_o[1]]
[trans_target_i[1]] :
745 InvalidTokenIdx;
746 assign hashed_token_mux = {hashed_tokens_lower[token_idx0],
747 hashed_tokens_upper[token_idx1]};
748 assign hashed_token_valid_mux =
{hashed_tokens_valid0[token_idx0],
749 hashed_tokens_valid1[token_idx1]};
750
751 // If the indices are inconsistent, we also trigger a trans-
ition error.
752 // We do not trigger an alert right away if this happens,
since it could
753 // be due to an invalid value programmed to the CSRs.
754 logic trans_invalid_error;
755 assign trans_invalid_error_o = trans_invalid_error ||
(token_idx0 != token_idx1);
756
757 ///////////////////////////////////////////////////
758 // Decoding and transition logic for redundantly encoded LC
state //
759 ///////////////////////////////////////////////////
760
761 // This decodes the state into a format that can be exposed
in the CSRs,
762 // and flags any errors in the state encoding. Errors will
move the
763 // main FSM into INVALID right away.
764 lc_ctrl_state_decode u_lc_ctrl_state_decode (
765 .lc_state_valid_i ( lc_state_valid_q ),
766 .lc_state_i ( lc_state_q ),
767 .lc_cnt_i ( lc_cnt_q ),
768 .secrets_valid_i,
769 .fsm_state_i ( fsm_state_q ),
770 .dec_lc_state_o,
771 .dec_lc_id_state_o,
772 .dec_lc_cnt_o,
773 .state_invalid_error_o ( state_invalid_error )
774 );
775
776 // LC transition checker logic and next state generation.
777 lc_ctrl_state_transition #(
778 .SecVolatileRawUnlockEn(SecVolatileRawUnlockEn)
779 ) u_lc_ctrl_state_transition (
780 .lc_state_i ( lc_state_q ),
781 .lc_cnt_i ( lc_cnt_q ),
782 .dec_lc_state_i ( dec_lc_state_o ),
783 .fsm_state_i ( fsm_state_q ),
784 .trans_target_i,
785 .volatile_raw_unlock_i,
786 .trans_cmd_i,
787 .next_lc_state_o ( next_lc_state ),
788 .next_lc_cnt_o ( next_lc_cnt ),
789 .trans_cnt_oflw_error_o,
790 .trans_invalid_error_o ( trans_invalid_error )
791 );
792
793 // LC signal decoder and broadcasting logic.
794 lc_ctrl_signal_decode #(
795 .RndCnstLcKeymgrDivInvalid ( RndCnstLcKeymgrDivInval-
id
796 ),
797 .RndCnstLcKeymgrDivTestUn-
locked( RndCnstLcKeymgrDivTestUnlocked ),
798 .RndCnstLcKeymgrDivDev ( RndCnstLcKeymgrDi-
vDev
799 ),
800 .RndCnstLcKeymgrDivProduction ( RndCnstLcKeymgrDivProduc-
tion
801 ),
802 .RndCnstLcKeymgrDivRma ( RndCnstLcKeymgrDi-
vRma
803 )
804 ) u_lc_ctrl_signal_decode (

```

```

801 .clk_i,
802 .rst_ni,
803 .lc_state_valid_i ( lc_state_valid_q ),
804 .lc_state_i ( lc_state_q ),
805 .secrets_valid_i,
806 .fsm_state_i ( fsm_state_q ),
807 .lc_init_done_o,
808 .lc_raw_test_rma_o,
809 .lc_dft_en_o,
810 .lc_nvm_debug_en_o,
811 .lc_hw_debug_clr_o,
812 .lc_hw_debug_en_o,
813 .lc_cpu_en_o,
814 .lc_creator_seed_sw_rw_en_o,
815 .lc_owner_seed_sw_rw_en_o,
816 .lc_iso_part_sw_rd_en_o,
817 .lc_iso_part_sw_wr_en_o,
818 .lc_seed_hw_rd_en_o,
819 .lc_rma_state_o,
820 .lc_keymgr_en_o,
821 .lc_escalate_en_o,
822 .lc_keymgr_div_o
823 );
824
825 // Conditional signals set by main FSM.
826 prim_lc_sender u_prim_lc_sender_clk_byp_req (
827 .clk_i,
828 .rst_ni,
829 .lc_en_i(lc_clk_byp_req),
830 .lc_en_o(lc_clk_byp_req_o)
831 );
832
833 prim_lc_sender u_prim_lc_sender_nvm_rma_req (
834 .clk_i,
835 .rst_ni,
836 .lc_en_i(lc_nvm_rma_req),
837 .lc_en_o(lc_nvm_rma_req_o)
838 );
839 prim_lc_sender u_prim_lc_sender_check_byp_en (
840 .clk_i,
841 .rst_ni,
842 .lc_en_i(lc_check_byp_en),
843 .lc_en_o(lc_check_byp_en_o)
844 );
845
846 ///////////////////////////////////////////////////
847 // Assertions //
848 ///////////////////////////////////////////////////
849
850 `ASSERT(EscStaysOnOnceAsserted_A,
851 lc_tx_test_true_strict(lc_escalate_en_o)
852 |>
853 lc_tx_test_true_strict(lc_escalate_en_o))
854
855 `ASSERT(ClkBypStaysOnOnceAsserted_A,
856 lc_tx_test_true_strict(lc_clk_byp_req_o)
857 |>
858 lc_tx_test_true_strict(lc_clk_byp_req_o))
859
860 `ASSERT(NvmRmaStaysOnOnceAsserted_A,
861 lc_tx_test_true_strict(lc_nvm_rma_req_o)
862 |>
863 lc_tx_test_true_strict(lc_nvm_rma_req_o))
864
865 `ASSERT(NoClkBypInProdStates_A,
866 lc_state_q inside {LcStProd, LcStProdEnd, LcStDev}
867 |>
868 lc_tx_test_false_strict(lc_clk_byp_req_o))
869
870 `ASSERT(SecCmCFITerminal0_A,
871 fsm_state_q == PostTransSt
872 |>
873 fsm_state_q inside {PostTransSt, InvalidSt, EscalateSt})
874
875 `ASSERT(SecCmCFITerminal1_A,
876 fsm_state_q == ScrapSt
877 |>
878 fsm_state_q inside {ScrapSt, InvalidSt, EscalateSt})
879
880 `ASSERT(SecCmCFITerminal2_A,
881 fsm_state_q == EscalateSt
882 |>
883 fsm_state_q == EscalateSt)
884
885 `ASSERT(SecCmCFITerminal3_A,
886 fsm_state_q == InvalidSt
887 |>
888 fsm_state_q inside {InvalidSt, EscalateSt})
889
890 // Check that the FSM is linear and does not contain any
loops
891 `ASSERT(FPV_LINEAR_FSM(SecCmCFILinear_A, fsm_state_q,
fsm_state_e)
892
893 endmodule : lc_ctrl_fsm

```

4. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_kmac_if.sv (219 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Synchronization interface between LC FSM and KMAC.
6 //
7
8 `include "prim_assert.sv"
9
10 module lc_ctrl_kmac_if
11 import lc_ctrl_pkg::*;
12 import lc_ctrl_state_pkg::*;
13 (
14 // Life cycle controller clock
15 input clk_i,
16 input rst_ni,
17 // Clock for KMAC interface
18 input clk_kmac_i,
19 input rst_kmac_ni,
20 // Life cycle hashing interface for raw unlock
21 // Synchronized in the life cycle controller.
22 input kmac_pkg::app_rsp_t kmac_data_i,
23 output kmac_pkg::app_req_t kmac_data_o,
24 // Token hashing interface to LC FSM
25 input lc_token_t transition_token_i,
26 input token_hash_req_i,
27 // Used for gating assertions inside CDC primitives.
28 input token_hash_req_chk_i,
29 output logic token_hash_ack_o,
30 output logic token_hash_err_o,
31 output logic token_if_fsm_err_o,
32 output lc_token_t hashed_token_o
33 );
34
35 ///////////////////////////////////////////////////
36 // Data and Handshake Synchronizers //
37 ///////////////////////////////////////////////////
38
39 // The transition_token_i register is guaranteed to remain
40 stable once a life cycle
41 // transition has been initiated.
42 // Hence no further synchronization registers are required
43 on the outgoing data.
44 lc_token_t kmac_transition_token;
45 assign kmac_transition_token = transition_token_i;
46
47 // SRC domain cannot change data while waiting for ACK.
48 `ASSERT(DataStable_A, token_hash_req_i && !token_hash_ack_o
49 |> $stable(transition_token_i))
50
51 // Second synchronizer instance for handshake and return
52 data synchronization.
53 logic kmac_req, kmac_ack;
54 logic token_hash_req;
55 logic token_hash_ack_d, token_hash_ack_q;
56 logic token_hash_err_q, token_hash_err_d;
57 lc_token_t hashed_token_q, hashed_token_d;
58 prim_sync_reqack_data #(
59 // Token + Error bit
60 .Width (LcTokenWidth + 1),
61 .DataSrc2Dst(1'b0),
62 // This instantiates a data register
63 // on the destination side.
64 .DataReg (1'b1)
65 ) u_prim_sync_reqack_data_in (
66 .clk_src_i ( clk_i ),
67 .rst_src_ni ( rst_ni ),
68 .clk_dst_i ( clk_kmac_i ),
69 .rst_dst_ni ( rst_kmac_ni ),
70 .req_chk_i ( token_hash_req_chk_i ),
71 .src_req_i ( token_hash_req ),
72 .src_ack_o ( token_hash_ack_d ),
73 .dst_req_o ( kmac_req ),
74 .dst_ack_i ( kmac_ack ),
75 // Truncate hash to 128bit and remove masking (not re-
76 quired here).
77 .data_i ( {kmac_data_i.error,
78 kmac_data_i.digest_s0[LcTokenWidth-1:0] ^
79 kmac_data_i.digest_s1[LcTokenWidth-1:0]} ),
80 .data_o ( {token_hash_err_d,
81 hashed_token_d} )
82 );
83
84 logic unused_sigs;
85 assign unused_sigs = ^{
86 kmac_data_i.digest_s0[LcTokenWidth+: (kmac_pkg::AppDi-
87 gestW - LcTokenWidth)],
88 kmac_data_i.digest_s1[LcTokenWidth+: (kmac_pkg::AppDi-
89 gestW - LcTokenWidth)],
90 // A static interface has no finish response.
91 kmac_data_i.rsp_finish
92 };
93

```

```

88 // Hashed Token Register Running on LC Clock
89 always_ff @(posedge clk_i or negedge rst_ni) begin :
  p_lc_regs
90 if (!rst_ni) begin
91 token_hash_ack_q <= 1'b0;
92 token_hash_err_q <= 1'b0;
93 hashed_token_q <= {LcTokenWidth{1'b1}};
94 end else begin
95 token_hash_ack_q <= token_hash_ack_d;
96 // Latch synchronized token and error bit
97 if (token_hash_req_i && token_hash_ack_d) begin
98 token_hash_err_q <= token_hash_err_d;
99 hashed_token_q <= hashed_token_d;
100 end
101 end
102 end
103
104 assign token_hash_ack_o = token_hash_ack_q;
105 assign token_hash_err_o = token_hash_err_q;
106 assign hashed_token_o = hashed_token_q;
107
108 // Stop requesting tokens upon latching on LC side.
109 assign token_hash_req = token_hash_req_i &
~token_hash_ack_q;
110
111 // Need to synchronize this error signal separately.
112 logic kmac_fsm_err_d, kmac_fsm_err_q;
113 prim_flop_2sync #(
114 .Width(1),
115 .ResetValue(0)
116 ) u_prim_flop_2sync (
117 .clk_i,
118 .rst_ni,
119 .d_i(kmac_fsm_err_q),
120 .q_o(token_if_fsm_err_o)
121 );
122
123 ///////////////////////////////////////////////////
124 // Serialization FSM Running on KMAC Clock //
125 ///////////////////////////////////////////////////
126
127 // SEC_CM: KMAC.FSM.SPARSE
128 // Encoding generated with:
129 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 4 -n 8 \
130 // -s 3343913945 --language=sv
131 //
132 // Hamming distance histogram:
133 //
134 // 0: --
135 // 1: --
136 // 2: --
137 // 3: --
138 // 4: --
139 // 5: ||||| (66.67%)
140 // 6: ||||| (33.33%)
141 // 7: --
142 // 8: --
143 //
144 // Minimum Hamming distance: 5
145 // Maximum Hamming distance: 6
146 // Minimum Hamming weight: 2
147 // Maximum Hamming weight: 6
148 //
149 localparam int StateWidth = 8;
150 typedef enum logic [StateWidth-1:0] {
151 FirstSt = 8'b01011011,
152 SecondSt = 8'b10010100,
153 WaitSt = 8'b11100111,
154 DoneSt = 8'b00101000
155 } state_e;
156
157 state_e state_d, state_q;
158
159 // Serialize the 128bit token into two 64bit beats.
160 always_comb begin : p_kmac
161 state_d = state_q;
162 kmac_ack = 1'b0;
163 kmac_fsm_err_d = 1'b0;
164
165 // LC ctrl operates on unshared data and accepts digest
166 responses immediately.
167 kmac_data_o = '0;
168 kmac_data_o.data_s1 = '0;
169 kmac_data_o.rsp_ready = 1'b1;
170
171 unique case (state_q)
172 // Wait for request and transfer first half of
173 // LC token.
174 FirstSt: begin
175 if (kmac_req) begin
176 kmac_data_o.req_valid = 1'b1;
177 kmac_data_o.strb = 8'hFF;
178 kmac_data_o.data_s0 = kmac_transition_token[0+:
64];
179 if (kmac_data_i.req_ready) begin
180 state_d = SecondSt;
181 end
182 end
183 // Transfer second half of LC token.
184 SecondSt: begin

```

```

185     kmac_data_o.req_valid = 1'b1;
186     kmac_data_o.strb     = 8'hFF;
187     kmac_data_o.req_last = 1'b1;
188     kmac_data_o.data_s0  = kmac_transition_token[64 +
64];
189     if (kmac_data_i.req_ready) begin
190         state_d = WaitSt;
191     end
192 end
193 // Wait for hashed token response and go to terminal
state.
194 WaitSt: begin
195     if (kmac_data_i.rsp_valid) begin
196         kmac_ack = 1'b1;
197         state_d = DoneSt;
198     end
199 end
200 // Terminal state (by design we can only perform
201 // one token hashing operation per reset cycle).
202 DoneSt: ;
203 default: begin
204     kmac_fsm_err_d = 1'b1;
205 end
206 endcase // state_q
207 end
208
209 `PRIM_FLOP_SPARSE_FSM(u_state_regs, state_d, state_q,
state_e, FirstSt, clk_kmac_i, rst_kmac_ni)
210
211 always_ff @(posedge clk_kmac_i or negedge rst_kmac_ni) be-
gin : p_kmac_fsm_err
212     if (!rst_kmac_ni) begin
213         kmac_fsm_err_q <= 1'b0;
214     end else begin
215         kmac_fsm_err_q <= kmac_fsm_err_d;
216     end
217 end
218
219 endmodule : lc_ctrl_kmac_if

```

5. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_pkg.sv (395 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5
6 `include "prim_assert.sv"
7
8 package lc_ctrl_pkg;
9
10 import prim_util_pkg::vbits;
11 import lc_ctrl_state_pkg::*;
12
13 // pwrmgr to lifecycle
14 typedef struct packed {
15     logic lc_init;
16 } pwr_lc_req_t;
17
18 // lifecycle to pwrmgr
19 typedef struct packed {
20     logic lc_done;
21     logic lc_idle;
22 } pwr_lc_rsp_t;
23
24 // default value (for dangling ports)
25 parameter pwr_lc_rsp_t PWR_LC_RSP_DEFAULT = '{
26     lc_done: 1'b1,
27     lc_idle: 1'b1
28 };
29
30 // Netlist Constants (Hashed Tokens) //
31 ///////////////////////////////////////////////////////////////////
32 ///////////////////////////////////////////////////////////////////
33
34 parameter int NumTokens = 6;
35 parameter int TokenIdxWidth = vbits(NumTokens);
36 typedef enum logic [TokenIdxWidth-1:0] {
37     // This is the index for the hashed all-zero constant.
38     // All unconditional transitions use this token.
39     ZeroTokenIdx = 3'h0,
40     RawUnlockTokenIdx = 3'h1,
41     TestUnlockTokenIdx = 3'h2,
42     TestExitTokenIdx = 3'h3,
43     RmaTokenIdx = 3'h4,
44     // This is the index for an all-zero value (i.e., hashed
value = '0).
45     // This is used as an additional blocker for some invalid
state transition edges.
46     InvalidTokenIdx = 3'h5
47 } token_idx_e;
48
49 parameter int TokenMuxBits = 2**TokenIdxWidth*LcTokenWidth;
50 typedef logic [TokenMuxBits-1:0] lc_token_mux_t;
51
52 ///////////////////////////////////////////////////////////////////
53 // Typedefs for LC Interfaces //
54 ///////////////////////////////////////////////////////////////////

```

```

55
56 parameter int TxWidth = 4;
57
58 // Note that changing this encoding has implications on
isolation cell
59 // values in RTL. Do not change this unless absolutely
needed.
60 typedef enum logic [TxWidth-1:0] {
61     On = 4'b0101,
62     Off = 4'b1010
63 } lc_tx_t;
64 parameter lc_tx_t LC_TX_DEFAULT = lc_tx_t'(Off);
65
66 parameter int RmaSeedWidth = 64;
67 typedef logic [RmaSeedWidth-1:0] lc_nvm_rma_seed_t;
68 parameter lc_nvm_rma_seed_t LC_NVM_RMA_SEED_DEFAULT = '0;
69
70 parameter int LcKeymgrDivWidth = 128;
71 typedef logic [LcKeymgrDivWidth-1:0] lc_keymgr_div_t;
72
73 typedef struct packed {
74     logic [lc_ctrl_reg_pkg::SiliconCreatorIdWidth-1:0] silic-
on_creator_id;
75     logic [lc_ctrl_reg_pkg::ProductIdWidth-1:0]
product_id;
76     logic [lc_ctrl_reg_pkg::RevisionIdWidth-1:0]
revision_id;
77     logic [32-lc_ctrl_reg_pkg::RevisionIdWidth-1:0]
re-
served;
78 } lc_hw_rev_t;
79
80 ///////////////////////////////////////////////////////////////////
81 // Helper Functions for Life Cycle Signals //
82 ///////////////////////////////////////////////////////////////////
83
84 // This is a prerequisite for the multibit functions below
to work.
85 `ASSERT_STATIC_IN_PACKAGE(CheckLcTxValsComplementary_A, On
== ~Off)
86 // Check for bit-width matching between lc_tx_t and mubi4_t
87
88 `ASSERT_STATIC_IN_PACKAGE(LcMuBi4WidthCheck_A, $bits(TxWidth) ==
$bits(prim_mubi4_pkg::MuBi4Width))
89
90 // Convert a life cycle signal to mubi4
91 // If in the future other versions are desired, this should
really be
92 // moved to prim_mubi4_pkg
93 // The On ^ MuBi4True determines the bit differences between
94 // an lc_ctrl_pkg::On and prim_mubi4_pkg::MuBi4True.
95 // Once the required inversions are determined, it is then
applied
96 // to the incoming value. If the incoming value is true, it
will
97 // appropriately flip to the correct MuBi4Value.
98 // Since the false value is always complement of the true
value,
99 // this mechanism will also work for the other polarity.
100 function automatic prim_mubi4_pkg::mubi4_t lc_to_mu-
bi4(lc_tx_t val);
101     return prim_mubi4_pkg::mubi4_t'(val ^ (On ^
prim_mubi4_pkg::MuBi4True));
102 endfunction : lc_to_mubi4
103
104 // Same as lc_to_mubi4 but, but for an input that is Off,
return MuBi4True
105 // for an input that is On, return MuBi4False
106 function automatic prim_mubi4_pkg::mubi4_t lc_to_mu-
bi4_inv(lc_tx_t val);
107     return prim_mubi4_pkg::mubi4_t'(val ^ (Off ^
prim_mubi4_pkg::MuBi4True));
108 endfunction : lc_to_mubi4_inv
109
110 // Convert a life cycle signal to mubi8. See also lc_to_mu-
bi4.
111 function automatic prim_mubi4_pkg::mubi8_t lc_to_mu-
bi8(lc_tx_t val);
112     return prim_mubi4_pkg::mubi8_t'({
113         lc_to_mubi4_inv(val),
114         lc_to_mubi4(val)
115     });
116 endfunction : lc_to_mubi8
117
118 function automatic lc_tx_t mu-
bi4_to_lc(prim_mubi4_pkg::mubi4_t val);
119     return lc_tx_t'(val ^ (prim_mubi4_pkg::MuBi4True ^ On));
120 endfunction : mubi4_to_lc
121
122 // same function as above, but for an input that is Mu-
Bi4True, return Off
123 // for an input that is MuBi4False, return On
124 function automatic lc_tx_t mu-
bi4_to_lc_inv(prim_mubi4_pkg::mubi4_t val);
125     return lc_tx_t'(val ^ (prim_mubi4_pkg::MuBi4True ^ Off));
126 endfunction : mubi4_to_lc_inv
127
128 // Test whether the value is supplied is one of the valid
enumerations
129 function automatic logic lc_tx_test_invalid(lc_tx_t val);

```



```

130     return ~(val inside {On, Off});
131 endfunction : lc_tx_test_invalid
132
133 // Convert a 1 input value to a lc_tx output
134 function automatic lc_tx_t lc_tx_bool_to_lc_tx(logic val);
135     return (val ? On : Off);
136 endfunction : lc_tx_bool_to_lc_tx
137
138 // Test whether the multibit value signals an "enabled" con-
dition.
139 // The strict version of this function requires
140 // the multibit value to equal True.
141 function automatic logic lc_tx_test_true_strict(lc_tx_t
val);
142     return On == val;
143 endfunction : lc_tx_test_true_strict
144
145 // Test whether the multibit value signals a "disabled" con-
dition.
146 // The strict version of this function requires
147 // the multibit value to equal False.
148 function automatic logic lc_tx_test_false_strict(lc_tx_t
val);
149     return Off == val;
150 endfunction : lc_tx_test_false_strict
151
152 // Test whether the multibit value signals an "enabled" con-
dition.
153 // The loose version of this function interprets all
154 // values other than False as "enabled".
155 function automatic logic lc_tx_test_true_loose(lc_tx_t val);
156     return Off != val;
157 endfunction : lc_tx_test_true_loose
158
159 // Test whether the multibit value signals a "disabled" con-
dition.
160 // The loose version of this function interprets all
161 // values other than True as "disabled".
162 function automatic logic lc_tx_test_false_loose(lc_tx_t
val);
163     return On != val;
164 endfunction : lc_tx_test_false_loose
165
166 // Performs a logical OR operation between two multibit val-
ues.
167 // This treats "act" as logical 1, and all other values are
168 // treated as 0. Truth table:
169 //
170 // A   | B   | OUT
171 //-----+-----
172 // !act | !act | !act
173 // act  | !act | act
174 // !act | act  | act
175 // act  | act  | act
176 //
177
178 // Note: due to the nature of the lc_tx_or() function, it is
possible that two
179 // non-strictly "act" values may produce a strictly "act"
value. If this is
180 // of concern, e.g. if the output is consumed with a strict
check on "act",
181 // consider using the prim_lc_or_hardened primitive instead.
182 function automatic lc_tx_t lc_tx_or(lc_tx_t a, lc_tx_t b,
lc_tx_t act);
183     logic [TxWidth-1:0] a_in, b_in, act_in, out;
184     a_in = a;
185     b_in = b;
186     act_in = act;
187     for (int k = 0; k < TxWidth; k++) begin
188         if (act_in[k]) begin
189             out[k] = a_in[k] || b_in[k];
190         end else begin
191             out[k] = a_in[k] && b_in[k];
192         end
193     end
194     return lc_tx_t'(out);
195 endfunction : lc_tx_or
196
197 // Performs a logical AND operation between two multibit
values.
198 // This treats "act" as logical 1, and all other values are
199 // treated as 0. Truth table:
200 //
201 // A   | B   | OUT
202 //-----+-----
203 // !act | !act | !act
204 // act  | !act | !act
205 // !act | act  | !act
206 // act  | act  | act
207 //
208 // Note: The lc_tx_and() function does not suffer from the
strictness problem
209 // that the lc_tx_or function above does, since only one
output value in the
210 // truth table is strictly "act". It can hence be used in
most scenarios without issues.
211 // If however the lc_tx_and() function should be strictly
rectifying (i.e., only
212 // output "act" or ~"act"), the prim_lc_and_hardened can be

```

```

used.
213 function automatic lc_tx_t lc_tx_and(lc_tx_t a, lc_tx_t b,
lc_tx_t act);
214     logic [TxWidth-1:0] a_in, b_in, act_in, out;
215     a_in = a;
216     b_in = b;
217     act_in = act;
218     for (int k = 0; k < TxWidth; k++) begin
219         if (act_in[k]) begin
220             out[k] = a_in[k] && b_in[k];
221         end else begin
222             out[k] = a_in[k] || b_in[k];
223         end
224     end
225     return lc_tx_t'(out);
226 endfunction : lc_tx_and
227
228 // Performs a logical OR operation between two multibit val-
ues.
229 // This treats "True" as logical 1, and all other values are
230 // treated as 0.
231 function automatic lc_tx_t lc_tx_or_hi(lc_tx_t a, lc_tx_t
b);
232     return lc_tx_or(a, b, On);
233 endfunction : lc_tx_or_hi
234
235 // Performs a logical AND operation between two multibit
values.
236 // This treats "True" as logical 1, and all other values are
237 // treated as 0.
238 function automatic lc_tx_t lc_tx_and_hi(lc_tx_t a, lc_tx_t
b);
239     return lc_tx_and(a, b, On);
240 endfunction : lc_tx_and_hi
241
242 // Performs a logical OR operation between two multibit val-
ues.
243 // This treats "False" as logical 1, and all other values
are
244 // treated as 0.
245 function automatic lc_tx_t lc_tx_or_lo(lc_tx_t a, lc_tx_t
b);
246     return lc_tx_or(a, b, Off);
247 endfunction : lc_tx_or_lo
248
249 // Performs a logical AND operation between two multibit
values.
250 // Tlo treats "False" as logical 1, and all other values
are
251 // treated as 0.
252 function automatic lc_tx_t lc_tx_and_lo(lc_tx_t a, lc_tx_t
b);
253     return lc_tx_and(a, b, Off);
254 endfunction : lc_tx_and_lo
255
256 // Inverts the logical meaning of the multibit value.
257 function automatic lc_tx_t lc_tx_inv(lc_tx_t a);
258     return lc_tx_t'(~TxWidth'a);
259 endfunction : lc_tx_inv
260
261 ///////////////////////////////////////////////////
262 // Main FSM State //
263 ///////////////////////////////////////////////////
264
265 // SEC_CM: MAIN.FSM.SPARSE
266 // Encoding generated with:
267 // $ ./util/design/sparse-fsm-encode.py -d 5 -m 15 -n 16 \
268 // -s 2934212379 --language=sv
269 //
270 // Hamming distance histogram:
271 //
272 // 0: --
273 // 1: --
274 // 2: --
275 // 3: --
276 // 4: --
277 // 5: ||||| (7.62%)
278 // 6: ||||| (9.52%)
279 // 7: ||||| (17.14%)
280 // 8: ||||| (20.95%)
281 // 9: ||||| (18.10%)
282 // 10: ||||| (14.29%)
283 // 11: ||||| (6.67%)
284 // 12: ||| (3.81%)
285 // 13: | (1.90%)
286 // 14: --
287 // 15: --
288 // 16: --
289 //
290 // Minimum Hamming distance: 5
291 // Maximum Hamming distance: 13
292 // Minimum Hamming weight: 3
293 // Maximum Hamming weight: 11
294 //
295 localparam int FsmStateWidth = 16;
296 typedef enum logic [FsmStateWidth-1:0] {
297     ResetSt      = 16'b111010101011100,
298     IdleSt       = 16'b0000011101010101,
299     ClkMuxSt     = 16'b1100111011001001,
300     CntIncrSt    = 16'b0011001111000111,
301     CntProgSt    = 16'b0000110001010100,

```

```

302 TransCheckSt = 16'b0110111010110000,
303 TokenHashSt = 16'b1101001000111111,
304 NvmRmaSt = 16'b1110100010001111,
305 TokenCheck0St = 16'b0010000011000000,
306 TokenCheck1St = 16'b1101010101011111,
307 TransProgSt = 16'b1000000110101011,
308 PostTransSt = 16'b0110110100101100,
309 ScrapSt = 16'b1010100001010001,
310 EscalateSt = 16'b1011110110011011,
311 InvalidSt = 16'b0011000101001100
312 } fsm_state_e;
313
314 //////////////////////////////////////////////////
315 // Manufacturing State Transition Matrix //
316 //////////////////////////////////////////////////
317
318 // Helper macro to assemble the token index matrix below.
319 // From TEST_UNLOCKED(N)
320 // -> SCRAP, RMA
321 // -> PROD, PROD_END, DEV
322 // -> TEST_UNLOCKED(N+1)-7
323 // -> TEST_LOCKED(N)-6
324 // -> TEST_UNLOCKED0-(N), RAW
325 `define TEST_UNLOCKED(idx) \
326     {2{ZeroTokenIdx}, \
327     {3{TestExitTokenIdx}, \
328     {(7-idx){InvalidTokenIdx}, \
329     ZeroTokenIdx}, \
330     {(2*idx+2){InvalidTokenIdx}}}
331
332 // Helper macro to assemble the token index matrix below.
333 // From TEST_LOCKED(N)
334 // -> SCRAP
335 // -> RMA
336 // -> PROD, PROD_END, DEV
337 // -> TEST_UNLOCKED(N+1)-7
338 // -> TEST_LOCKED(N)-6
339 // -> TEST_UNLOCKED0-(N), RAW
340 `define TEST_LOCKED(idx) \
341     ZeroTokenIdx, \
342     InvalidTokenIdx, \
343     {3{TestExitTokenIdx}, \
344     {(7-idx){TestUnlockTokenIdx}, \
345     InvalidTokenIdx}, \
346     {(2*idx+2){InvalidTokenIdx}}}
347
348 // The token index matrix below encodes 1) which transition
349 // edges are valid and 2) which token
350 // to use for a given transition edge. Note that uncondi-
351 // tional but otherwise valid transitions
352 // are assigned the ZeroTokenIdx, whereas invalid trans-
353 // itions are assigned an InvalidTokenIdx.
354 parameter token_idx_e [NumLcStates-1:0][NumLcStates-1:0]
355 TransTokenIdxMatrix = {
356     // SCRAP
357     {21{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
358     LOCKED0-7, DEV, PROD, PROD_END, RMA, SCRAP
359     // RMA
360     ZeroTokenIdx, // -> SCRAP
361     {20{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
362     LOCKED0-7, DEV, PROD, PROD_END, RMA
363     // PROD_END
364     ZeroTokenIdx, // -> SCRAP
365     {20{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
366     LOCKED0-7, DEV, PROD, PROD_END, RMA
367     // PROD
368     ZeroTokenIdx, // -> SCRAP
369     RmaTokenIdx, // -> RMA
370     {19{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
371     LOCKED0-7, DEV, PROD, PROD_END
372     // DEV
373     ZeroTokenIdx, // -> SCRAP
374     RmaTokenIdx, // -> RMA
375     {19{InvalidTokenIdx}}, // -> TEST_LOCKED0-6, TEST_UN-
376     LOCKED0-7, DEV, PROD, PROD_END
377     // TEST_UNLOCKED0-7, TEST_LOCKED0-6
378     `TEST_UNLOCKED(7),
379     `TEST_LOCKED(6),
380     `TEST_UNLOCKED(6),
381     `TEST_LOCKED(5),
382     `TEST_UNLOCKED(5),
383     `TEST_LOCKED(4),
384     `TEST_UNLOCKED(4),
385     `TEST_LOCKED(3),
386     `TEST_UNLOCKED(3),
387     `TEST_LOCKED(2),
388     `TEST_UNLOCKED(2),
389     `TEST_LOCKED(1),
390     `TEST_UNLOCKED(1),
391     `TEST_LOCKED(0),
392     `TEST_UNLOCKED(0),
393     // RAW
394     ZeroTokenIdx, // -> SCRAP
395     {4{InvalidTokenIdx}}, // -> RMA, PROD, PROD_END, DEV
396     {8{RawUnlockTokenIdx}, // -> TEST_UNLOCKED0-7
397     InvalidTokenIdx} // -> RAW, TEST_LOCKED0-6
398 };
399
400 // These macros are only used locally.
401 `undef TEST_LOCKED
402 `undef TEST_UNLOCKED

```

```

394
395 endpackage : lc_ctrl_pkg

```

6. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_reg_pkg.sv (395 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 // Register Package auto-generated by `reggen` containing data
6 // structure
7 package lc_ctrl_reg_pkg;
8
9 // Param list
10 parameter int SiliconCreatorIdWidth = 16;
11 parameter int ProductIdWidth = 16;
12 parameter int RevisionIdWidth = 8;
13 parameter int NumTokenWords = 4;
14 parameter int CsrLcStateWidth = 30;
15 parameter int CsrLcCountWidth = 5;
16 parameter int CsrLcIdStateWidth = 32;
17 parameter int CsrOtpTestCtrlWidth = 32;
18 parameter int CsrOtpTestStatusWidth = 32;
19 parameter int NumDeviceIdWords = 8;
20 parameter int NumManufStateWords = 8;
21 parameter int NumAlerts = 3;
22
23 // Address widths within the block
24 parameter int RegsAw = 8;
25 parameter int DmiAw = 12;
26
27 // Number of registers for every interface
28 parameter int NumRegsRegs = 35;
29 parameter int NumRegsDmi = 0;
30
31 // Alert indices
32 typedef enum int {
33     AlertFatalProgErrorIdx = 0,
34     AlertFatalStateErrorIdx = 1,
35     AlertFatalBusIntegErrorIdx = 2
36 } lc_ctrl_alert_idx_t;
37
38 //////////////////////////////////////////////////
39 // Typedefs for registers for regs interface //
40 //////////////////////////////////////////////////
41
42 typedef struct packed {
43     struct packed {
44         logic q;
45         logic qe;
46     } fatal_bus_integ_error;
47     struct packed {
48         logic q;
49         logic qe;
50     } fatal_state_error;
51     struct packed {
52         logic q;
53         logic qe;
54     } fatal_prog_error;
55 } lc_ctrl_reg2hw_alert_test_reg_t;
56
57 typedef struct packed {
58     logic [7:0] q;
59     logic qe;
60 } lc_ctrl_reg2hw_claim_transition_if_reg_t;
61
62 typedef struct packed {
63     logic q;
64     logic qe;
65 } lc_ctrl_reg2hw_transition_cmd_reg_t;
66
67 typedef struct packed {
68     struct packed {
69         logic q;
70         logic qe;
71     } volatile_raw_unlock;
72     struct packed {
73         logic q;
74         logic qe;
75     } ext_clock_en;
76 } lc_ctrl_reg2hw_transition_ctrl_reg_t;
77
78 typedef struct packed {
79     logic [31:0] q;
80     logic qe;
81 } lc_ctrl_reg2hw_transition_token_mreg_t;
82
83 typedef struct packed {
84     logic [29:0] q;
85     logic qe;
86 } lc_ctrl_reg2hw_transition_target_reg_t;
87
88 typedef struct packed {
89     logic [31:0] q;
90     logic qe;

```



```

91 } lc_ctrl_reg2hw_otp_vendor_test_ctrl_reg_t;
92
93 typedef struct packed {
94     struct packed {
95         logic d;
96     } otp_partition_error;
97     struct packed {
98         logic d;
99     } bus_integ_error;
100     struct packed {
101         logic d;
102     } state_error;
103     struct packed {
104         logic d;
105     } otp_error;
106     struct packed {
107         logic d;
108     } nvm_rma_error;
109     struct packed {
110         logic d;
111     } token_error;
112     struct packed {
113         logic d;
114     } transition_error;
115     struct packed {
116         logic d;
117     } transition_count_error;
118     struct packed {
119         logic d;
120     } transition_successful;
121     struct packed {
122         logic d;
123     } ext_clock_switched;
124     struct packed {
125         logic d;
126     } ready;
127     struct packed {
128         logic d;
129     } initialized;
130 } lc_ctrl_hw2reg_status_reg_t;
131
132 typedef struct packed {
133     logic [7:0] d;
134 } lc_ctrl_hw2reg_claim_transition_if_reg_t;
135
136 typedef struct packed {
137     logic d;
138 } lc_ctrl_hw2reg_transition_regwen_reg_t;
139
140 typedef struct packed {
141     struct packed {
142         logic d;
143     } volatile_raw_unlock;
144     struct packed {
145         logic d;
146     } ext_clock_en;
147 } lc_ctrl_hw2reg_transition_ctrl_reg_t;
148
149 typedef struct packed {
150     logic [31:0] d;
151 } lc_ctrl_hw2reg_transition_token_mreg_t;
152
153 typedef struct packed {
154     logic [29:0] d;
155 } lc_ctrl_hw2reg_transition_target_reg_t;
156
157 typedef struct packed {
158     logic [31:0] d;
159 } lc_ctrl_hw2reg_otp_vendor_test_ctrl_reg_t;
160
161 typedef struct packed {
162     logic [31:0] d;
163 } lc_ctrl_hw2reg_otp_vendor_test_status_reg_t;
164
165 typedef struct packed {
166     logic [29:0] d;
167 } lc_ctrl_hw2reg_lc_state_reg_t;
168
169 typedef struct packed {
170     logic [4:0] d;
171 } lc_ctrl_hw2reg_lc_transition_cnt_reg_t;
172
173 typedef struct packed {
174     logic [31:0] d;
175 } lc_ctrl_hw2reg_lc_id_state_reg_t;
176
177 typedef struct packed {
178     struct packed {
179         logic [15:0] d;
180     } silicon_creator_id;
181     struct packed {
182         logic [15:0] d;
183     } product_id;
184 } lc_ctrl_hw2reg_hw_revision0_reg_t;
185
186 typedef struct packed {
187     struct packed {
188         logic [23:0] d;
189     } reserved;
190     struct packed {
191         logic [7:0] d;
192     } revision_id;
193 } lc_ctrl_hw2reg_hw_revision1_reg_t;
194
195 typedef struct packed {
196     logic [31:0] d;
197 } lc_ctrl_hw2reg_device_id_mreg_t;
198
199 typedef struct packed {
200     logic [31:0] d;
201 } lc_ctrl_hw2reg_manuf_state_mreg_t;
202
203 // Register -> HW type for regs interface
204 typedef struct packed {
205     lc_ctrl_reg2hw_alert_test_reg_t alert_test; // [216:211]
206     lc_ctrl_reg2hw_claim_transition_if_reg_t
207 claim_transition_if; // [210:202]
208     lc_ctrl_reg2hw_transition_cmd_reg_t transition_cmd; //
209 [201:200]
210     lc_ctrl_reg2hw_transition_ctrl_reg_t transition_ctrl; //
211 [199:196]
212     lc_ctrl_reg2hw_transition_token_mreg_t [3:0] trans-
213 ition_token; // [195:64]
214     lc_ctrl_reg2hw_transition_target_reg_t trans-
215 ition_target; // [63:33]
216     lc_ctrl_reg2hw_otp_vendor_test_ctrl_reg_t
217 otp_vendor_test_ctrl; // [32:0]
218 } lc_ctrl_regs_reg2hw_t;
219
220 // HW -> register type for regs interface
221 typedef struct packed {
222     lc_ctrl_hw2reg_status_reg_t status; // [887:876]
223     lc_ctrl_hw2reg_claim_transition_if_reg_t
224 claim_transition_if; // [875:868]
225     lc_ctrl_hw2reg_transition_regwen_reg_t trans-
226 ition_regwen; // [867:867]
227     lc_ctrl_hw2reg_transition_ctrl_reg_t transition_ctrl; //
228 [866:865]
229     lc_ctrl_hw2reg_transition_token_mreg_t [3:0] trans-
230 ition_token; // [864:737]
231     lc_ctrl_hw2reg_transition_target_reg_t trans-
232 ition_target; // [736:707]
233     lc_ctrl_hw2reg_otp_vendor_test_ctrl_reg_t
234 otp_vendor_test_ctrl; // [706:675]
235     lc_ctrl_hw2reg_otp_vendor_test_status_reg_t
236 otp_vendor_test_status; // [674:643]
237     lc_ctrl_hw2reg_lc_state_reg_t lc_state; // [642:613]
238     lc_ctrl_hw2reg_lc_transition_cnt_reg_t lc.trans-
239 ition_cnt; // [612:608]
240     lc_ctrl_hw2reg_lc_id_state_reg_t lc_id_state; // [607:576]
241     lc_ctrl_hw2reg_hw_revision0_reg_t hw_revision0; //
242 [575:544]
243     lc_ctrl_hw2reg_hw_revision1_reg_t hw_revision1; //
244 [543:512]
245     lc_ctrl_hw2reg_device_id_mreg_t [7:0] device_id; //
246 [511:256]
247     lc_ctrl_hw2reg_manuf_state_mreg_t [7:0] manuf_state; //
248 [255:0]
249 } lc_ctrl_regs_hw2reg_t;
250
251 // Register offsets for regs interface
252
253 parameter logic [RegsAw-1:0] LC_CTRL_ALERT_TEST_OFFSET = 8'h
0;
254
255 parameter logic [RegsAw-1:0] LC_CTRL_STATUS_OFFSET = 8'h 4;
256
257 parameter logic [RegsAw-1:0] LC_C-
258 TRL_CLAIM_TRANSITION_IF_REGWEN_OFFSET = 8'h 8;
259
260 parameter logic [RegsAw-1:0] LC_C-
261 TRL_CLAIM_TRANSITION_IF_OFFSET = 8'h c;
262
263 parameter logic [RegsAw-1:0] LC_C-
264 TRL_TRANSITION_REGWEN_OFFSET = 8'h 10;
265
266 parameter logic [RegsAw-1:0] LC_CTRL_TRANSITION_CMD_OFFSET =
267 8'h 14;
268
269 parameter logic [RegsAw-1:0] LC_CTRL_TRANSITION_CTRL_OFFSET
270 = 8'h 18;
271
272 parameter logic [RegsAw-1:0] LC_C-
273 TRL_TRANSITION_TOKEN_0_OFFSET = 8'h 1c;
274
275 parameter logic [RegsAw-1:0] LC_C-
276 TRL_TRANSITION_TOKEN_1_OFFSET = 8'h 20;
277
278 parameter logic [RegsAw-1:0] LC_C-
279 TRL_TRANSITION_TOKEN_2_OFFSET = 8'h 24;
280
281 parameter logic [RegsAw-1:0] LC_C-
282 TRL_TRANSITION_TOKEN_3_OFFSET = 8'h 28;
283
284 parameter logic [RegsAw-1:0] LC_C-
285 TRL_TRANSITION_TARGET_OFFSET = 8'h 2c;
286
287 parameter logic [RegsAw-1:0] LC_C-
288 TRL_OTP_VENDOR_TEST_CTRL_OFFSET = 8'h 30;
289
290 parameter logic [RegsAw-1:0] LC_C-
291 TRL_OTP_VENDOR_TEST_STATUS_OFFSET = 8'h 34;
292
293 parameter logic [RegsAw-1:0] LC_CTRL_LC_STATE_OFFSET = 8'h
38;
294
295 parameter logic [RegsAw-1:0] LC_C-
296 TRL_LC_TRANSITION_CNT_OFFSET = 8'h 3c;
297
298 parameter logic [RegsAw-1:0] LC_CTRL_LC_ID_STATE_OFFSET =
299 8'h 40;
300
301 parameter logic [RegsAw-1:0] LC_CTRL_HW_REVISION0_OFFSET =
302 8'h 44;
303
304 parameter logic [RegsAw-1:0] LC_CTRL_HW_REVISION1_OFFSET =
305 8'h 48;
306
307 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_0_OFFSET =
308 8'h 4c;

```

```

254 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_1_OFFSET =
8'h 50;
255 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_2_OFFSET =
8'h 54;
256 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_3_OFFSET =
8'h 58;
257 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_4_OFFSET =
8'h 5c;
258 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_5_OFFSET =
8'h 60;
259 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_6_OFFSET =
8'h 64;
260 parameter logic [RegsAw-1:0] LC_CTRL_DEVICE_ID_7_OFFSET =
8'h 68;
261 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_0_OFFSET =
8'h 6c;
262 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_1_OFFSET =
8'h 70;
263 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_2_OFFSET =
8'h 74;
264 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_3_OFFSET =
8'h 78;
265 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_4_OFFSET =
8'h 7c;
266 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_5_OFFSET =
8'h 80;
267 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_6_OFFSET =
8'h 84;
268 parameter logic [RegsAw-1:0] LC_CTRL_MANUF_STATE_7_OFFSET =
8'h 88;
269
270 // Reset values for hwext registers and their fields for
regs interface
271 parameter logic [2:0] LC_CTRL_ALERT_TEST_RESVAL = 3'h 0;
272 parameter logic [0:0] LC_C-
TRL_ALERT_TEST_FATAL_PROG_ERROR_RESVAL = 1'h 0;
273 parameter logic [0:0] LC_C-
TRL_ALERT_TEST_FATAL_STATE_ERROR_RESVAL = 1'h 0;
274 parameter logic [0:0] LC_C-
TRL_ALERT_TEST_FATAL_BUS_INTEG_ERROR_RESVAL = 1'h 0;
275 parameter logic [11:0] LC_CTRL_STATUS_RESVAL = 12'h 0;
276 parameter logic [7:0] LC_CTRL_CLAIM_TRANSITION_IF_RESVAL =
8'h 69;
277 parameter logic [7:0] LC_C-
TRL_CLAIM_TRANSITION_IF_MUTEX_RESVAL = 8'h 69;

278 parameter logic [0:0] LC_CTRL_TRANSITION_REGWEN_RESVAL = 1'h
0;
279 parameter logic [0:0] LC_C-
TRL_TRANSITION_REGWEN_TRANSITION_REGWEN_RESVAL = 1'h 0;
280 parameter logic [0:0] LC_CTRL_TRANSITION_CMD_RESVAL = 1'h 0;
281 parameter logic [1:0] LC_CTRL_TRANSITION_CTRL_RESVAL = 2'h
0;
282 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_0_RESVAL =
32'h 0;
283 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_1_RESVAL =
32'h 0;
284 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_2_RESVAL =
32'h 0;
285 parameter logic [31:0] LC_CTRL_TRANSITION_TOKEN_3_RESVAL =
32'h 0;
286 parameter logic [29:0] LC_CTRL_TRANSITION_TARGET_RESVAL =
30'h 0;

287 parameter logic [31:0] LC_CTRL_OTP_VENDOR_TEST_CTRL_RESVAL =
32'h 0;
288 parameter logic [31:0] LC_CTRL_OTP_VENDOR_TEST_STATUS_RESVAL =
32'h 0;
289 parameter logic [29:0] LC_CTRL_LC_STATE_RESVAL = 30'h 0;

290 parameter logic [4:0] LC_CTRL_LC_TRANSITION_CNT_RESVAL = 5'h
0;
291 parameter logic [31:0] LC_CTRL_LC_ID_STATE_RESVAL = 32'h 0;
292 parameter logic [31:0] LC_CTRL_HW_REVISION0_RESVAL = 32'h 0;
293 parameter logic [31:0] LC_CTRL_HW_REVISION1_RESVAL = 32'h 0;
294 parameter logic [23:0] LC_CTRL_HW_REVISION1_RESERVED_RESVAL
= 24'h 0;
295 parameter logic [31:0] LC_CTRL_DEVICE_ID_0_RESVAL = 32'h 0;
296 parameter logic [31:0] LC_CTRL_DEVICE_ID_1_RESVAL = 32'h 0;
297 parameter logic [31:0] LC_CTRL_DEVICE_ID_2_RESVAL = 32'h 0;
298 parameter logic [31:0] LC_CTRL_DEVICE_ID_3_RESVAL = 32'h 0;
299 parameter logic [31:0] LC_CTRL_DEVICE_ID_4_RESVAL = 32'h 0;
300 parameter logic [31:0] LC_CTRL_DEVICE_ID_5_RESVAL = 32'h 0;
301 parameter logic [31:0] LC_CTRL_DEVICE_ID_6_RESVAL = 32'h 0;
302 parameter logic [31:0] LC_CTRL_DEVICE_ID_7_RESVAL = 32'h 0;
303 parameter logic [31:0] LC_CTRL_MANUF_STATE_0_RESVAL = 32'h
0;
304 parameter logic [31:0] LC_CTRL_MANUF_STATE_1_RESVAL = 32'h
0;
305 parameter logic [31:0] LC_CTRL_MANUF_STATE_2_RESVAL = 32'h
0;
306 parameter logic [31:0] LC_CTRL_MANUF_STATE_3_RESVAL = 32'h
0;
307 parameter logic [31:0] LC_CTRL_MANUF_STATE_4_RESVAL = 32'h
0;
308 parameter logic [31:0] LC_CTRL_MANUF_STATE_5_RESVAL = 32'h
0;
309 parameter logic [31:0] LC_CTRL_MANUF_STATE_6_RESVAL = 32'h
0;
310 parameter logic [31:0] LC_CTRL_MANUF_STATE_7_RESVAL = 32'h

```

```

0;
311
312 // Register index for regs interface
313 typedef enum int {
314     LC_CTRL_ALERT_TEST,
315     LC_CTRL_STATUS,
316     LC_CTRL_CLAIM_TRANSITION_IF_REGWEN,
317     LC_CTRL_CLAIM_TRANSITION_IF,
318     LC_CTRL_TRANSITION_REGWEN,
319     LC_CTRL_TRANSITION_CMD,
320     LC_CTRL_TRANSITION_CTRL,
321     LC_CTRL_TRANSITION_TOKEN_0,
322     LC_CTRL_TRANSITION_TOKEN_1,
323     LC_CTRL_TRANSITION_TOKEN_2,
324     LC_CTRL_TRANSITION_TOKEN_3,
325     LC_CTRL_TRANSITION_TARGET,
326     LC_CTRL_OTP_VENDOR_TEST_CTRL,
327     LC_CTRL_OTP_VENDOR_TEST_STATUS,
328     LC_CTRL_LC_STATE,
329     LC_CTRL_LC_TRANSITION_CNT,
330     LC_CTRL_LC_ID_STATE,
331     LC_CTRL_HW_REVISION0,
332     LC_CTRL_HW_REVISION1,
333     LC_CTRL_DEVICE_ID_0,
334     LC_CTRL_DEVICE_ID_1,
335     LC_CTRL_DEVICE_ID_2,
336     LC_CTRL_DEVICE_ID_3,
337     LC_CTRL_DEVICE_ID_4,
338     LC_CTRL_DEVICE_ID_5,
339     LC_CTRL_DEVICE_ID_6,
340     LC_CTRL_DEVICE_ID_7,
341     LC_CTRL_MANUF_STATE_0,
342     LC_CTRL_MANUF_STATE_1,
343     LC_CTRL_MANUF_STATE_2,
344     LC_CTRL_MANUF_STATE_3,
345     LC_CTRL_MANUF_STATE_4,
346     LC_CTRL_MANUF_STATE_5,
347     LC_CTRL_MANUF_STATE_6,
348     LC_CTRL_MANUF_STATE_7
349 } lc_ctrl_regs_id_e;
350
351 // Register width information to check illegal writes for
regs interface
352 parameter logic [3:0] LC_CTRL_REGS_PERMIT [35] = '{
353     4'b 0001, // index[ 0] LC_CTRL_ALERT_TEST
354     4'b 0011, // index[ 1] LC_CTRL_STATUS
355     4'b 0001, // index[ 2] LC_CTRL_CLAIM_TRANSITION_IF_REGWEN
356     4'b 0001, // index[ 3] LC_CTRL_CLAIM_TRANSITION_IF
357     4'b 0001, // index[ 4] LC_CTRL_TRANSITION_REGWEN
358     4'b 0001, // index[ 5] LC_CTRL_TRANSITION_CMD
359     4'b 0001, // index[ 6] LC_CTRL_TRANSITION_CTRL
360     4'b 1111, // index[ 7] LC_CTRL_TRANSITION_TOKEN_0
361     4'b 1111, // index[ 8] LC_CTRL_TRANSITION_TOKEN_1
362     4'b 1111, // index[ 9] LC_CTRL_TRANSITION_TOKEN_2
363     4'b 1111, // index[10] LC_CTRL_TRANSITION_TOKEN_3
364     4'b 1111, // index[11] LC_CTRL_TRANSITION_TARGET
365     4'b 1111, // index[12] LC_CTRL_OTP_VENDOR_TEST_CTRL
366     4'b 1111, // index[13] LC_CTRL_OTP_VENDOR_TEST_STATUS
367     4'b 1111, // index[14] LC_CTRL_LC_STATE
368     4'b 0001, // index[15] LC_CTRL_LC_TRANSITION_CNT
369     4'b 1111, // index[16] LC_CTRL_LC_ID_STATE
370     4'b 1111, // index[17] LC_CTRL_HW_REVISION0
371     4'b 1111, // index[18] LC_CTRL_HW_REVISION1
372     4'b 1111, // index[19] LC_CTRL_DEVICE_ID_0
373     4'b 1111, // index[20] LC_CTRL_DEVICE_ID_1
374     4'b 1111, // index[21] LC_CTRL_DEVICE_ID_2
375     4'b 1111, // index[22] LC_CTRL_DEVICE_ID_3
376     4'b 1111, // index[23] LC_CTRL_DEVICE_ID_4
377     4'b 1111, // index[24] LC_CTRL_DEVICE_ID_5
378     4'b 1111, // index[25] LC_CTRL_DEVICE_ID_6
379     4'b 1111, // index[26] LC_CTRL_DEVICE_ID_7
380     4'b 1111, // index[27] LC_CTRL_MANUF_STATE_0
381     4'b 1111, // index[28] LC_CTRL_MANUF_STATE_1
382     4'b 1111, // index[29] LC_CTRL_MANUF_STATE_2
383     4'b 1111, // index[30] LC_CTRL_MANUF_STATE_3
384     4'b 1111, // index[31] LC_CTRL_MANUF_STATE_4
385     4'b 1111, // index[32] LC_CTRL_MANUF_STATE_5
386     4'b 1111, // index[33] LC_CTRL_MANUF_STATE_6
387     4'b 1111, // index[34] LC_CTRL_MANUF_STATE_7
388 };
389
390 // Window parameters for dmi interface
391 parameter logic [DmiAw-1:0] LC_CTRL_DMI_OFFSET = 12'h 0;
392 parameter int unsigned LC_CTRL_DMI_SIZE = 'h 1000;
393 parameter int unsigned LC_CTRL_DMI_IDX = 0;
394
395 endpackage

```

7. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_regs_reg_top.sv (1,525 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Register Top module auto-generated by `reggen`
6
7 `include "prim_assert.sv"

```

```

8
9 module lc_ctrl_regs_reg_top (
10     input clk_i,
11     input rst_ni,
12     input tlul_pkg::tl_h2d_t tl_i,
13     output tlul_pkg::tl_d2h_t tl_o,
14     // To HW
15     output lc_ctrl_reg_pkg::lc_ctrl_regs_reg2hw_t reg2hw, //
Write
16     input lc_ctrl_reg_pkg::lc_ctrl_regs_hw2reg_t hw2reg, //
Read
17
18     // Integrity check errors
19     output logic intg_err_o
20 );
21
22 import lc_ctrl_reg_pkg::* ;
23
24 localparam int AW = 8;
25 localparam int DW = 32;
26 localparam int DBW = DW/8; // Byte Width
27
28 // register signals
29 logic reg_we;
30 logic reg_re;
31 logic [AW-1:0] reg_addr;
32 logic [DW-1:0] reg_wdata;
33 logic [DBW-1:0] reg_be;
34 logic [DW-1:0] reg_rdata;
35 logic reg_error;
36
37 logic addrmiss, wr_err;
38
39 logic [DW-1:0] reg_rdata_next;
40 logic reg_busy;
41
42 tlul_pkg::tl_h2d_t tl_reg_h2d;
43 tlul_pkg::tl_d2h_t tl_reg_d2h;
44
45 // incoming payload check
46 logic intg_err;
47 tlul_cmd_intg_chk u_chk (
48     .tl_i(tl_i),
49     .err_o(intg_err)
50 );
51
52 // also check for spurious write enables
53 logic reg_we_err;
54 logic [34:0] reg_we_check;
55 prim_reg_we_check #(
56     .OneHotWidth(35)
57 ) u_prim_reg_we_check (
58     .clk_i(clk_i),
59     .rst_ni(rst_ni),
60     .oh_i (reg_we_check),
61     .en_i (reg_we && !addrmiss),
62     .err_o (reg_we_err)
63 );
64
65 logic err_q;
66 always_ff @(posedge clk_i or negedge rst_ni) begin
67     if (!rst_ni) begin
68         err_q <= '0;
69     end else if (intg_err || reg_we_err) begin
70         err_q <= '1'b1;
71     end
72 end
73
74 // integrity error output is permanent and should be used
for alert generation
75 // register errors are transactional
76 assign intg_err_o = err_q | intg_err | reg_we_err;
77
78 // outgoing integrity generation
79 tlul_pkg::tl_d2h_t tl_o_pre;
80 tlul_rsp_intg_gen #(
81     .EnableRspIntgGen(1),
82     .EnableDataIntgGen(1)
83 ) u_rsp_intg_gen (
84     .tl_i(tl_o_pre),
85     .tl_o(tl_o)
86 );
87
88 assign tl_reg_h2d = tl_i;
89 assign tl_o_pre = tl_reg_d2h;
90
91 tlul_adapter_reg #(
92     .RegAw(AW),
93     .RegDw(DW),
94     .EnableDataIntgGen(0)
95 ) u_reg_if (
96     .clk_i (clk_i),
97     .rst_ni (rst_ni),
98
99     .tl_i (tl_reg_h2d),
100    .tl_o (tl_reg_d2h),
101
102    .en_ifetch_i(prim_mubi_pkg::MuBi4False),
103    .intg_error_o(),
104
105    .we_o (reg_we),
106    .re_o (reg_re),
107    .addr_o (reg_addr),
108    .wdata_o (reg_wdata),
109    .be_o (reg_be),
110    .busy_i (reg_busy),
111    .rdata_i (reg_rdata),
112    .error_i (reg_error)
113 );
114
115 // cdc oversampling signals
116
117 assign reg_rdata = reg_rdata_next ;
118 assign reg_error = addrmiss | wr_err | intg_err;
119
120 // Define SW related signals
121 // Format: <reg>_<field>_<wd|we|qs>
122 // or <reg>_<wd|we|qs> if field == 1 or 0
123 logic alert_test_we;
124 logic alert_test_fatal_prog_error_wd;
125 logic alert_test_fatal_state_error_wd;
126 logic alert_test_fatal_bus_integ_error_wd;
127 logic status_re;
128 logic status_initialized_qs;
129 logic status_ready_qs;
130 logic status_ext_clock_switched_qs;
131 logic status_transition_successful_qs;
132 logic status_transition_count_error_qs;
133 logic status_transition_error_qs;
134 logic status_token_error_qs;
135 logic status_nvm_rma_error_qs;
136 logic status_otp_error_qs;
137 logic status_state_error_qs;
138 logic status_bus_integ_error_qs;
139 logic status_otp_partition_error_qs;
140 logic claim_transition_if_regwen_we;
141 logic claim_transition_if_regwen_qs;
142 logic claim_transition_if_regwen_wd;
143 logic claim_transition_if_re;
144 logic claim_transition_if_we;
145 logic [7:0] claim_transition_if_qs;
146 logic [7:0] claim_transition_if_wd;
147 logic transition_regwen_re;
148 logic transition_regwen_qs;
149 logic transition_cmd_we;
150 logic transition_cmd_wd;
151 logic transition_ctrl_re;
152 logic transition_ctrl_we;
153 logic transition_ctrl_ext_clock_en_qs;
154 logic transition_ctrl_ext_clock_en_wd;
155 logic transition_ctrl_volatile_raw_unlock_qs;
156 logic transition_ctrl_volatile_raw_unlock_wd;
157 logic transition_token_0_re;
158 logic transition_token_0_we;
159 logic [31:0] transition_token_0_qs;
160 logic [31:0] transition_token_0_wd;
161 logic transition_token_1_re;
162 logic transition_token_1_we;
163 logic [31:0] transition_token_1_qs;
164 logic [31:0] transition_token_1_wd;
165 logic transition_token_2_re;
166 logic transition_token_2_we;
167 logic [31:0] transition_token_2_qs;
168 logic [31:0] transition_token_2_wd;
169 logic transition_token_3_re;
170 logic transition_token_3_we;
171 logic [31:0] transition_token_3_qs;
172 logic [31:0] transition_token_3_wd;
173 logic transition_target_re;
174 logic transition_target_we;
175 logic [29:0] transition_target_qs;
176 logic [29:0] transition_target_wd;
177 logic otp_vendor_test_ctrl_re;
178 logic otp_vendor_test_ctrl_we;
179 logic [31:0] otp_vendor_test_ctrl_qs;
180 logic [31:0] otp_vendor_test_ctrl_wd;
181 logic otp_vendor_test_status_re;
182 logic [31:0] otp_vendor_test_status_qs;
183 logic lc_state_re;
184 logic [29:0] lc_state_qs;
185 logic lc_transition_cnt_re;
186 logic [4:0] lc_transition_cnt_qs;
187 logic lc_id_state_re;
188 logic [31:0] lc_id_state_qs;
189 logic hw_revision0_re;
190 logic [15:0] hw_revision0_product_id_qs;
191 logic [15:0] hw_revision0_silicon_creator_id_qs;
192 logic hw_revision1_re;
193 logic [7:0] hw_revision1_revision_id_qs;
194 logic [23:0] hw_revision1_reserved_qs;
195 logic device_id_0_re;
196 logic [31:0] device_id_0_qs;
197 logic device_id_1_re;
198 logic [31:0] device_id_1_qs;
199 logic device_id_2_re;
200 logic [31:0] device_id_2_qs;
201 logic device_id_3_re;
202 logic [31:0] device_id_3_qs;
203 logic device_id_4_re;
204 logic [31:0] device_id_4_qs;
205 logic device_id_5_re;
206

```

```

207 logic [31:0] device_id_5_qs;
208 logic device_id_6_re;
209 logic [31:0] device_id_6_qs;
210 logic device_id_7_re;
211 logic [31:0] device_id_7_qs;
212 logic manif_state_0_re;
213 logic [31:0] manif_state_0_qs;
214 logic manif_state_1_re;
215 logic [31:0] manif_state_1_qs;
216 logic manif_state_2_re;
217 logic [31:0] manif_state_2_qs;
218 logic manif_state_3_re;
219 logic [31:0] manif_state_3_qs;
220 logic manif_state_4_re;
221 logic [31:0] manif_state_4_qs;
222 logic manif_state_5_re;
223 logic [31:0] manif_state_5_qs;
224 logic manif_state_6_re;
225 logic [31:0] manif_state_6_qs;
226 logic manif_state_7_re;
227 logic [31:0] manif_state_7_qs;
228
229 // Register instances
230 // R[alert_test]: V(True)
231 logic alert_test_qe;
232 logic [2:0] alert_test_flds_we;
233 assign alert_test_qe = &alert_test_flds_we;
234 // F[fatal_prog_error]: 0:0
235 prim_subreg_ext #(
236   .DW (1)
237 ) u_alert_test_fatal_prog_error (
238   .re (1'b0),
239   .we (alert_test_we),
240   .wd (alert_test_fatal_prog_error_wd),
241   .d ('0'),
242   .qre (),
243   .qe (alert_test_flds_we[0]),
244   .q (reg2hw.alert_test_fatal_prog_error.q),
245   .ds (),
246   .qs ()
247 );
248 assign reg2hw.alert_test_fatal_prog_error.qe =
alert_test_qe;
249
250 // F[fatal_state_error]: 1:1
251 prim_subreg_ext #(
252   .DW (1)
253 ) u_alert_test_fatal_state_error (
254   .re (1'b0),
255   .we (alert_test_we),
256   .wd (alert_test_fatal_state_error_wd),
257   .d ('0'),
258   .qre (),
259   .qe (alert_test_flds_we[1]),
260   .q (reg2hw.alert_test_fatal_state_error.q),
261   .ds (),
262   .qs ()
263 );
264 assign reg2hw.alert_test_fatal_state_error.qe =
alert_test_qe;
265
266 // F[fatal_bus_integ_error]: 2:2
267 prim_subreg_ext #(
268   .DW (1)
269 ) u_alert_test_fatal_bus_integ_error (
270   .re (1'b0),
271   .we (alert_test_we),
272   .wd (alert_test_fatal_bus_integ_error_wd),
273   .d ('0'),
274   .qre (),
275   .qe (alert_test_flds_we[2]),
276   .q (reg2hw.alert_test_fatal_bus_integ_error.q),
277   .ds (),
278   .qs ()
279 );
280 assign reg2hw.alert_test_fatal_bus_integ_error.qe =
alert_test_qe;
281
282 // R[status]: V(True)
283 // F[initialized]: 0:0
284 prim_subreg_ext #(
285   .DW (1)
286 ) u_status_initialized (
287   .re (status_re),
288   .we (1'b0),
289   .wd ('0'),
290   .d (hw2reg.status.initialized.d),
291   .qre (),
292   .qe (),
293   .q (),
294   .ds (),
295   .qs (status_initialized_qs)
296 );
297
298 // F[ready]: 1:1
299 prim_subreg_ext #(
300   .DW (1)
301 ) u_status_ready (
302   .re (status_re),
303   .we (1'b0),
304
305   .wd ('0'),
306   .d (hw2reg.status.ready.d),
307   .qre (),
308   .qe (),
309   .q (),
310   .ds (),
311   .qs (status_ready_qs)
312 );
313
314 // F[ext_clock_switched]: 2:2
315 prim_subreg_ext #(
316   .DW (1)
317 ) u_status_ext_clock_switched (
318   .re (status_re),
319   .we (1'b0),
320   .wd ('0'),
321   .d (hw2reg.status.ext_clock_switched.d),
322   .qre (),
323   .qe (),
324   .q (),
325   .ds (),
326   .qs (status_ext_clock_switched_qs)
327 );
328
329 // F[transition_successful]: 3:3
330 prim_subreg_ext #(
331   .DW (1)
332 ) u_status_transition_successful (
333   .re (status_re),
334   .we (1'b0),
335   .wd ('0'),
336   .d (hw2reg.status.transition_successful.d),
337   .qre (),
338   .qe (),
339   .q (),
340   .ds (),
341   .qs (status_transition_successful_qs)
342 );
343
344 // F[transition_count_error]: 4:4
345 prim_subreg_ext #(
346   .DW (1)
347 ) u_status_transition_count_error (
348   .re (status_re),
349   .we (1'b0),
350   .wd ('0'),
351   .d (hw2reg.status.transition_count_error.d),
352   .qre (),
353   .qe (),
354   .q (),
355   .ds (),
356   .qs (status_transition_count_error_qs)
357 );
358
359 // F[transition_error]: 5:5
360 prim_subreg_ext #(
361   .DW (1)
362 ) u_status_transition_error (
363   .re (status_re),
364   .we (1'b0),
365   .wd ('0'),
366   .d (hw2reg.status.transition_error.d),
367   .qre (),
368   .qe (),
369   .q (),
370   .ds (),
371   .qs (status_transition_error_qs)
372 );
373
374 // F[token_error]: 6:6
375 prim_subreg_ext #(
376   .DW (1)
377 ) u_status_token_error (
378   .re (status_re),
379   .we (1'b0),
380   .wd ('0'),
381   .d (hw2reg.status.token_error.d),
382   .qre (),
383   .qe (),
384   .q (),
385   .ds (),
386   .qs (status_token_error_qs)
387 );
388
389 // F[nvm_rma_error]: 7:7
390 prim_subreg_ext #(
391   .DW (1)
392 ) u_status_nvm_rma_error (
393   .re (status_re),
394   .we (1'b0),
395   .wd ('0'),
396   .d (hw2reg.status.nvm_rma_error.d),
397   .qre (),
398   .qe (),
399   .q (),
400   .ds (),
401   .qs (status_nvm_rma_error_qs)
402 );
403
404 // F[otp_error]: 8:8
405 prim_subreg_ext #(

```

```

406 .DW (1)
407 ) u_status_otp_error (
408 .re (status_re),
409 .we (1'b0),
410 .wd ('0'),
411 .d (hw2reg.status.otp_error.d),
412 .qre (),
413 .qe (),
414 .q (),
415 .ds (),
416 .qs (status_otp_error_qs)
417 );
418
419 // F[state_error]: 9:9
420 prim_subreg_ext #(
421 .DW (1)
422 ) u_status_state_error (
423 .re (status_re),
424 .we (1'b0),
425 .wd ('0'),
426 .d (hw2reg.status.state_error.d),
427 .qre (),
428 .qe (),
429 .q (),
430 .ds (),
431 .qs (status_state_error_qs)
432 );
433
434 // F[bus_integ_error]: 10:10
435 prim_subreg_ext #(
436 .DW (1)
437 ) u_status_bus_integ_error (
438 .re (status_re),
439 .we (1'b0),
440 .wd ('0'),
441 .d (hw2reg.status.bus_integ_error.d),
442 .qre (),
443 .qe (),
444 .q (),
445 .ds (),
446 .qs (status_bus_integ_error_qs)
447 );
448
449 // F[otp_partition_error]: 11:11
450 prim_subreg_ext #(
451 .DW (1)
452 ) u_status_otp_partition_error (
453 .re (status_re),
454 .we (1'b0),
455 .wd ('0'),
456 .d (hw2reg.status.otp_partition_error.d),
457 .qre (),
458 .qe (),
459 .q (),
460 .ds (),
461 .qs (status_otp_partition_error_qs)
462 );
463
464 // R[claim_transition_if_regwen]: V(False)
465 prim_subreg_ext #(
466 .DW (1),
467 .SwAccess(prim_subreg_pkg::SwAccessW0C),
468 .RESVAL (1'h1),
469 .Mubi (1'b0)
470 ) u_claim_transition_if_regwen (
471 .clk_i (clk_i),
472 .rst_ni (rst_ni),
473
474 // from register interface
475 .we (claim_transition_if_regwen_we),
476 .wd (claim_transition_if_regwen_wd),
477
478 // from internal hardware
479 .de (1'b0),
480 .d ('0'),
481
482 // to internal hardware
483 .qe (),
484 .q (),
485 .ds (),
486
487 // to register interface (read)
488 .qs (claim_transition_if_regwen_qs)
489 );
490
491 // R[claim_transition_if]: V(True)
492 logic claim_transition_if_qe;
493 logic [0:0] claim_transition_if_flds_we;
494 assign claim_transition_if_qe =
&claim_transition_if_flds_we;
495 // Create REGWEN-gated WE signal
496 logic claim_transition_if_gated_we;
497
498 assign claim_transition_if_gated_we = claim_transition_if_we &
claim_transition_if_regwen_qs;
499 prim_subreg_ext #(
500 .DW (8)
501 ) u_claim_transition_if (
502 .re (claim_transition_if_re),

```

```

504 .we (claim_transition_if_gated_we),
505 .wd (claim_transition_if_wd),
506 .d (hw2reg.claim_transition_if.d),
507 .qre (),
508 .qe (claim_transition_if_flds_we[0]),
509 .q (reg2hw.claim_transition_if.q),
510 .ds (),
511 .qs (claim_transition_if_qs)
512 );
513 assign reg2hw.claim_transition_if.qe =
claim_transition_if_qe;
514
515 // R[transition_regwen]: V(True)
516 prim_subreg_ext #(
517 .DW (1)
518 ) u_transition_regwen (
519 .re (transition_regwen_re),
520 .we (1'b0),
521 .wd ('0'),
522 .d (hw2reg.transition_regwen.d),
523 .qre (),
524 .qe (),
525 .q (),
526 .ds (),
527 .qs (transition_regwen_qs)
528 );
529
530 // R[transition_cmd]: V(True)
531 logic transition_cmd_qe;
532 logic [0:0] transition_cmd_flds_we;
533 assign transition_cmd_qe = &transition_cmd_flds_we;
534 // Create REGWEN-gated WE signal
535 logic transition_cmd_gated_we;
536 assign transition_cmd_gated_we = transition_cmd_we & trans-
ition_regwen_qs;
537 prim_subreg_ext #(
538 .DW (1)
539 ) u_transition_cmd (
540 .re (1'b0),
541 .we (transition_cmd_gated_we),
542 .wd (transition_cmd_wd),
543 .d ('0'),
544 .qre (),
545 .qe (transition_cmd_flds_we[0]),
546 .q (reg2hw.transition_cmd.q),
547 .ds (),
548 .qs ()
549 );
550 assign reg2hw.transition_cmd.qe = transition_cmd_qe;
551
552 // R[transition_ctrl]: V(True)
553 logic transition_ctrl_qe;
554 logic [1:0] transition_ctrl_flds_we;
555 assign transition_ctrl_qe = &transition_ctrl_flds_we;
556 // Create REGWEN-gated WE signal
557 logic transition_ctrl_gated_we;
558 assign transition_ctrl_gated_we = transition_ctrl_we &
transition_regwen_qs;
559 // F[ext_clock_en]: 0:0
560 prim_subreg_ext #(
561 .DW (1)
562 ) u_transition_ctrl_ext_clock_en (
563 .re (transition_ctrl_re),
564 .we (transition_ctrl_gated_we),
565 .wd (transition_ctrl_ext_clock_en_wd),
566 .d (hw2reg.transition_ctrl_ext_clock_en.d),
567 .qre (),
568 .qe (transition_ctrl_flds_we[1]),
569 .q (reg2hw.transition_ctrl_ext_clock_en.q),
570 .ds (),
571 .qs (transition_ctrl_ext_clock_en_qs)
572 );
573 assign reg2hw.transition_ctrl_ext_clock_en.qe = trans-
ition_ctrl_qe;
574
575 // F[volatile_raw_unlock]: 1:1
576 prim_subreg_ext #(
577 .DW (1)
578 ) u_transition_ctrl_volatile_raw_unlock (
579 .re (transition_ctrl_re),
580 .we (transition_ctrl_gated_we),
581 .wd (transition_ctrl_volatile_raw_unlock_wd),
582 .d (hw2reg.transition_ctrl_volatile_raw_unlock.d),
583 .qre (),
584 .qe (transition_ctrl_flds_we[1]),
585 .q (reg2hw.transition_ctrl.volatile_raw_unlock.q),
586 .ds (),
587 .qs (transition_ctrl_volatile_raw_unlock_qs)
588 );
589 assign reg2hw.transition_ctrl.volatile_raw_unlock.qe =
transition_ctrl_qe;
590
591 // Subregister 0 of Multireg transition_token
592 // R[transition_token_0]: V(True)
593 logic transition_token_0_qe;
594 logic [0:0] transition_token_0_flds_we;
595 assign transition_token_0_qe = &transition_token_0_flds_we;

```



```

600 // Create REGWEN-gated WE signal
601 logic transition_token_0_gated_we;

602 assign transition_token_0_gated_we = transition_token_0_we &
transition_regwen_qs;
603 prim_subreg_ext #(
604     .DW (32)
605 ) u_transition_token_0 (
606     .re (transition_token_0_re),
607     .we (transition_token_0_gated_we),
608     .wd (transition_token_0_wd),
609     .d (hw2reg.transition_token[0].d),
610     .qre (),
611     .qe (transition_token_0_flds_we[0]),
612     .q (reg2hw.transition_token[0].q),
613     .ds (),
614     .qs (transition_token_0_qs)
615 );
616 assign reg2hw.transition_token[0].qe = trans-
ition_token_0_qe;
617
618 // Subregister 1 of Multireg transition_token
619 // R[transition_token_1]: V(True)
620 logic transition_token_1_qe;
621 logic [0:0] transition_token_1_flds_we;
622 assign transition_token_1_qe = &transition_token_1_flds_we;
623 // Create REGWEN-gated WE signal
624 logic transition_token_1_gated_we;

626 assign transition_token_1_gated_we = transition_token_1_we &
transition_regwen_qs;
627 prim_subreg_ext #(
628     .DW (32)
629 ) u_transition_token_1 (
630     .re (transition_token_1_re),
631     .we (transition_token_1_gated_we),
632     .wd (transition_token_1_wd),
633     .d (hw2reg.transition_token[1].d),
634     .qre (),
635     .qe (transition_token_1_flds_we[0]),
636     .q (reg2hw.transition_token[1].q),
637     .ds (),
638     .qs (transition_token_1_qs)
639 );
640 assign reg2hw.transition_token[1].qe = trans-
ition_token_1_qe;
641
642 // Subregister 2 of Multireg transition_token
643 // R[transition_token_2]: V(True)
644 logic transition_token_2_qe;
645 logic [0:0] transition_token_2_flds_we;
646 assign transition_token_2_qe = &transition_token_2_flds_we;
647 // Create REGWEN-gated WE signal
648 logic transition_token_2_gated_we;

650 assign transition_token_2_gated_we = transition_token_2_we &
transition_regwen_qs;
651 prim_subreg_ext #(
652     .DW (32)
653 ) u_transition_token_2 (
654     .re (transition_token_2_re),
655     .we (transition_token_2_gated_we),
656     .wd (transition_token_2_wd),
657     .d (hw2reg.transition_token[2].d),
658     .qre (),
659     .qe (transition_token_2_flds_we[0]),
660     .q (reg2hw.transition_token[2].q),
661     .ds (),
662     .qs (transition_token_2_qs)
663 );
664 assign reg2hw.transition_token[2].qe = trans-
ition_token_2_qe;
665
666 // Subregister 3 of Multireg transition_token
667 // R[transition_token_3]: V(True)
668 logic transition_token_3_qe;
669 logic [0:0] transition_token_3_flds_we;
670 assign transition_token_3_qe = &transition_token_3_flds_we;
671 // Create REGWEN-gated WE signal
672 logic transition_token_3_gated_we;

674 assign transition_token_3_gated_we = transition_token_3_we &
transition_regwen_qs;
675 prim_subreg_ext #(
676     .DW (32)
677 ) u_transition_token_3 (
678     .re (transition_token_3_re),
679     .we (transition_token_3_gated_we),
680     .wd (transition_token_3_wd),
681     .d (hw2reg.transition_token[3].d),
682     .qre (),
683     .qe (transition_token_3_flds_we[0]),
684     .q (reg2hw.transition_token[3].q),
685     .ds (),
686     .qs (transition_token_3_qs)
687 );
688 assign reg2hw.transition_token[3].qe = trans-
ition_token_3_qe;

```

```

689
690 // R[transition_target]: V(True)
691 logic transition_target_qe;
692 logic [0:0] transition_target_flds_we;
693 assign transition_target_qe = &transition_target_flds_we;
694 // Create REGWEN-gated WE signal
695 logic transition_target_gated_we;
696 assign transition_target_gated_we = transition_target_we &
transition_regwen_qs;
697 prim_subreg_ext #(
698     .DW (30)
699 ) u_transition_target (
700     .re (transition_target_re),
701     .we (transition_target_gated_we),
702     .wd (transition_target_wd),
703     .d (hw2reg.transition_target.d),
704     .qre (),
705     .qe (transition_target_flds_we[0]),
706     .q (reg2hw.transition_target.q),
707     .ds (),
708     .qs (transition_target_qs)
709 );
710 assign reg2hw.transition_target.qe = transition_target_qe;
711
712 // R[otp_vendor_test_ctrl]: V(True)
713 logic otp_vendor_test_ctrl_qe;
714 logic [0:0] otp_vendor_test_ctrl_flds_we;
715 assign otp_vendor_test_ctrl_qe =
&otp_vendor_test_ctrl_flds_we;
716 // Create REGWEN-gated WE signal
717 logic otp_vendor_test_ctrl_gated_we;
718 assign otp_vendor_test_ctrl_gated_we =
otp_vendor_test_ctrl_we & transition_regwen_qs;
719 prim_subreg_ext #(
720     .DW (32)
721 ) u_otp_vendor_test_ctrl (
722     .re (otp_vendor_test_ctrl_re),
723     .we (otp_vendor_test_ctrl_gated_we),
724     .wd (otp_vendor_test_ctrl_wd),
725     .d (hw2reg.otp_vendor_test_ctrl.d),
726     .qre (),
727     .qe (otp_vendor_test_ctrl_flds_we[0]),
728     .q (reg2hw.otp_vendor_test_ctrl.q),
729     .ds (),
730     .qs (otp_vendor_test_ctrl_qs)
731 );
732 assign reg2hw.otp_vendor_test_ctrl.qe =
otp_vendor_test_ctrl_qe;
733
734 // R[otp_vendor_test_status]: V(True)
735 prim_subreg_ext #(
736     .DW (32)
737 ) u_otp_vendor_test_status (
738     .re (otp_vendor_test_status_re),
739     .we (1'b0),
740     .wd ('0'),
741     .d (hw2reg.otp_vendor_test_status.d),
742     .qre (),
743     .qe (),
744     .q (),
745     .ds (),
746     .qs (otp_vendor_test_status_qs)
747 );
748
749 // R[lc_state]: V(True)
750 prim_subreg_ext #(
751     .DW (30)
752 ) u_lc_state (
753     .re (lc_state_re),
754     .we (1'b0),
755     .wd ('0'),
756     .d (hw2reg.lc_state.d),
757     .qre (),
758     .qe (),
759     .q (),
760     .ds (),
761     .qs (lc_state_qs)
762 );
763
764 // R[lc_transition_cnt]: V(True)
765 prim_subreg_ext #(
766     .DW (5)
767 ) u_lc_transition_cnt (
768     .re (lc_transition_cnt_re),
769     .we (1'b0),
770     .wd ('0'),
771     .d (hw2reg.lc_transition_cnt.d),
772     .qre (),
773     .qe (),
774     .q (),
775     .ds (),
776     .qs (lc_transition_cnt_qs)
777 );
778
779 // R[lc_id_state]: V(True)

```

```

786 prim_subreg_ext #(
787     .DW (32)
788 ) u_lc_id_state (
789     .re (lc_id_state_re),
790     .we (1'b0),
791     .wd ('0'),
792     .d (hw2reg.lc_id_state.d),
793     .qre (),
794     .qe (),
795     .q (),
796     .ds (),
797     .qs (lc_id_state_qs)
798 );
799
800 // R[hw_revision0]: V(True)
801 // F[product_id]: 15:0
802 prim_subreg_ext #(
803     .DW (16)
804 ) u_hw_revision0_product_id (
805     .re (hw_revision0_re),
806     .we (1'b0),
807     .wd ('0'),
808     .d (hw2reg.hw_revision0.product_id.d),
809     .qre (),
810     .qe (),
811     .q (),
812     .ds (),
813     .qs (hw_revision0_product_id_qs)
814 );
815
816 // F[silicon_creator_id]: 31:16
817 prim_subreg_ext #(
818     .DW (16)
819 ) u_hw_revision0_silicon_creator_id (
820     .re (hw_revision0_re),
821     .we (1'b0),
822     .wd ('0'),
823     .d (hw2reg.hw_revision0.silicon_creator_id.d),
824     .qre (),
825     .qe (),
826     .q (),
827     .ds (),
828     .qs (hw_revision0_silicon_creator_id_qs)
829 );
830
831 // R[hw_revision1]: V(True)
832 // F[revision_id]: 7:0
833 prim_subreg_ext #(
834     .DW (8)
835 ) u_hw_revision1_revision_id (
836     .re (hw_revision1_re),
837     .we (1'b0),
838     .wd ('0'),
839     .d (hw2reg.hw_revision1.revision_id.d),
840     .qre (),
841     .qe (),
842     .q (),
843     .ds (),
844     .qs (hw_revision1_revision_id_qs)
845 );
846
847 // F[reserved]: 31:8
848 prim_subreg_ext #(
849     .DW (24)
850 ) u_hw_revision1_reserved (
851     .re (hw_revision1_re),
852     .we (1'b0),
853     .wd ('0'),
854     .d (hw2reg.hw_revision1.reserved.d),
855     .qre (),
856     .qe (),
857     .q (),
858     .ds (),
859     .qs (hw_revision1_reserved_qs)
860 );
861
862 // Subregister 0 of Multireg device_id
863 // R[device_id_0]: V(True)
864 prim_subreg_ext #(
865     .DW (32)
866 ) u_device_id_0 (
867     .re (device_id_0_re),
868     .we (1'b0),
869     .wd ('0'),
870     .d (hw2reg.device_id[0].d),
871     .qre (),
872     .qe (),
873     .q (),
874     .ds (),
875     .qs (device_id_0_qs)
876 );
877
878 // Subregister 1 of Multireg device_id
879 // R[device_id_1]: V(True)
880 prim_subreg_ext #(
881     .DW (32)
882 ) u_device_id_1 (

```

```

887     .re (device_id_1_re),
888     .we (1'b0),
889     .wd ('0'),
890     .d (hw2reg.device_id[1].d),
891     .qre (),
892     .qe (),
893     .q (),
894     .ds (),
895     .qs (device_id_1_qs)
896 );
897
898 // Subregister 2 of Multireg device_id
899 // R[device_id_2]: V(True)
900 prim_subreg_ext #(
901     .DW (32)
902 ) u_device_id_2 (
903     .re (device_id_2_re),
904     .we (1'b0),
905     .wd ('0'),
906     .d (hw2reg.device_id[2].d),
907     .qre (),
908     .qe (),
909     .q (),
910     .ds (),
911     .qs (device_id_2_qs)
912 );
913
914 // Subregister 3 of Multireg device_id
915 // R[device_id_3]: V(True)
916 prim_subreg_ext #(
917     .DW (32)
918 ) u_device_id_3 (
919     .re (device_id_3_re),
920     .we (1'b0),
921     .wd ('0'),
922     .d (hw2reg.device_id[3].d),
923     .qre (),
924     .qe (),
925     .q (),
926     .ds (),
927     .qs (device_id_3_qs)
928 );
929
930 // Subregister 4 of Multireg device_id
931 // R[device_id_4]: V(True)
932 prim_subreg_ext #(
933     .DW (32)
934 ) u_device_id_4 (
935     .re (device_id_4_re),
936     .we (1'b0),
937     .wd ('0'),
938     .d (hw2reg.device_id[4].d),
939     .qre (),
940     .qe (),
941     .q (),
942     .ds (),
943     .qs (device_id_4_qs)
944 );
945
946 // Subregister 5 of Multireg device_id
947 // R[device_id_5]: V(True)
948 prim_subreg_ext #(
949     .DW (32)
950 ) u_device_id_5 (
951     .re (device_id_5_re),
952     .we (1'b0),
953     .wd ('0'),
954     .d (hw2reg.device_id[5].d),
955     .qre (),
956     .qe (),
957     .q (),
958     .ds (),
959     .qs (device_id_5_qs)
960 );
961
962 // Subregister 6 of Multireg device_id
963 // R[device_id_6]: V(True)
964 prim_subreg_ext #(
965     .DW (32)
966 ) u_device_id_6 (
967     .re (device_id_6_re),
968     .we (1'b0),
969     .wd ('0'),
970     .d (hw2reg.device_id[6].d),
971     .qre (),
972     .qe (),
973     .q (),
974     .ds (),
975     .qs (device_id_6_qs)
976 );
977
978 // Subregister 7 of Multireg device_id
979 // R[device_id_7]: V(True)
980 prim_subreg_ext #(
981     .DW (32)
982 ) u_device_id_7 (

```



```

988 ) u_device_id_7 (
989     .re (device_id_7_re),
990     .we (1'b0),
991     .wd ('0'),
992     .d (hw2reg.device_id[7].d),
993     .qre (),
994     .qe (),
995     .q (),
996     .ds (),
997     .qs (device_id_7_qs)
998 );
999
1000 // Subregister 0 of Multireg manif_state
1001 // R[manuf_state_0]: V(True)
1002 prim_subreg_ext #(
1003     .DW (32)
1004 ) u_manuf_state_0 (
1005     .re (manuf_state_0_re),
1006     .we (1'b0),
1007     .wd ('0'),
1008     .d (hw2reg.manuf_state[0].d),
1009     .qre (),
1010     .qe (),
1011     .q (),
1012     .ds (),
1013     .qs (manuf_state_0_qs)
1014 );
1015
1016 // Subregister 1 of Multireg manif_state
1017 // R[manuf_state_1]: V(True)
1018 prim_subreg_ext #(
1019     .DW (32)
1020 ) u_manuf_state_1 (
1021     .re (manuf_state_1_re),
1022     .we (1'b0),
1023     .wd ('0'),
1024     .d (hw2reg.manuf_state[1].d),
1025     .qre (),
1026     .qe (),
1027     .q (),
1028     .ds (),
1029     .qs (manuf_state_1_qs)
1030 );
1031
1032 // Subregister 2 of Multireg manif_state
1033 // R[manuf_state_2]: V(True)
1034 prim_subreg_ext #(
1035     .DW (32)
1036 ) u_manuf_state_2 (
1037     .re (manuf_state_2_re),
1038     .we (1'b0),
1039     .wd ('0'),
1040     .d (hw2reg.manuf_state[2].d),
1041     .qre (),
1042     .qe (),
1043     .q (),
1044     .ds (),
1045     .qs (manuf_state_2_qs)
1046 );
1047
1048 // Subregister 3 of Multireg manif_state
1049 // R[manuf_state_3]: V(True)
1050 prim_subreg_ext #(
1051     .DW (32)
1052 ) u_manuf_state_3 (
1053     .re (manuf_state_3_re),
1054     .we (1'b0),
1055     .wd ('0'),
1056     .d (hw2reg.manuf_state[3].d),
1057     .qre (),
1058     .qe (),
1059     .q (),
1060     .ds (),
1061     .qs (manuf_state_3_qs)
1062 );
1063
1064 // Subregister 4 of Multireg manif_state
1065 // R[manuf_state_4]: V(True)
1066 prim_subreg_ext #(
1067     .DW (32)
1068 ) u_manuf_state_4 (
1069     .re (manuf_state_4_re),
1070     .we (1'b0),
1071     .wd ('0'),
1072     .d (hw2reg.manuf_state[4].d),
1073     .qre (),
1074     .qe (),
1075     .q (),
1076     .ds (),
1077     .qs (manuf_state_4_qs)
1078 );
1079
1080 // Subregister 5 of Multireg manif_state
1081 // R[manuf_state_5]: V(True)
1082 prim_subreg_ext #(

```

```

1089     .DW (32)
1090 ) u_manuf_state_5 (
1091     .re (manuf_state_5_re),
1092     .we (1'b0),
1093     .wd ('0'),
1094     .d (hw2reg.manuf_state[5].d),
1095     .qre (),
1096     .qe (),
1097     .q (),
1098     .ds (),
1099     .qs (manuf_state_5_qs)
1100 );
1101
1102 // Subregister 6 of Multireg manif_state
1103 // R[manuf_state_6]: V(True)
1104 prim_subreg_ext #(
1105     .DW (32)
1106 ) u_manuf_state_6 (
1107     .re (manuf_state_6_re),
1108     .we (1'b0),
1109     .wd ('0'),
1110     .d (hw2reg.manuf_state[6].d),
1111     .qre (),
1112     .qe (),
1113     .q (),
1114     .ds (),
1115     .qs (manuf_state_6_qs)
1116 );
1117
1118 // Subregister 7 of Multireg manif_state
1119 // R[manuf_state_7]: V(True)
1120 prim_subreg_ext #(
1121     .DW (32)
1122 ) u_manuf_state_7 (
1123     .re (manuf_state_7_re),
1124     .we (1'b0),
1125     .wd ('0'),
1126     .d (hw2reg.manuf_state[7].d),
1127     .qre (),
1128     .qe (),
1129     .q (),
1130     .ds (),
1131     .qs (manuf_state_7_qs)
1132 );
1133
1134 logic [34:0] addr_hit;
1135 always_comb begin
1136     addr_hit[ 0] = (reg_addr == LC_CTRL_ALERT_TEST_OFFSET);
1137     addr_hit[ 1] = (reg_addr == LC_CTRL_STATUS_OFFSET);
1138     addr_hit[ 2] = (reg_addr == LC_C-
1139     TR_LCLAIM_TRANSITION_IF_REGWEN_OFFSET);
1140     addr_hit[ 3] = (reg_addr == LC_C-
1141     TR_LCLAIM_TRANSITION_IF_OFFSET);
1142     addr_hit[ 4] = (reg_addr == LC_C-
1143     TR_LTRANSITION_REGWEN_OFFSET);
1144     addr_hit[ 5] = (reg_addr == LC_C-
1145     TR_LTRANSITION_CMD_OFFSET);
1146     addr_hit[ 6] = (reg_addr == LC_C-
1147     TR_LTRANSITION_CTRL_OFFSET);
1148     addr_hit[ 7] = (reg_addr == LC_C-
1149     TR_LTRANSITION_TOKEN_0_OFFSET);
1150     addr_hit[ 8] = (reg_addr == LC_C-
1151     TR_LTRANSITION_TOKEN_1_OFFSET);
1152     addr_hit[ 9] = (reg_addr == LC_C-
1153     TR_LTRANSITION_TOKEN_2_OFFSET);
1154     addr_hit[10] = (reg_addr == LC_C-
1155     TR_LTRANSITION_TOKEN_3_OFFSET);
1156     addr_hit[11] = (reg_addr == LC_C-
1157     TR_LTRANSITION_TARGET_OFFSET);
1158     addr_hit[12] = (reg_addr == LC_C-
1159     TR_OTP_VENDOR_TEST_CTRL_OFFSET);
1160     addr_hit[13] = (reg_addr == LC_C-
1161     TR_OTP_VENDOR_TEST_STATUS_OFFSET);
1162     addr_hit[14] = (reg_addr == LC_CTRL_LC_STATE_OFFSET);
1163     addr_hit[15] = (reg_addr == LC_C-
1164     TR_LC_TRANSITION_CNT_OFFSET);
1165     addr_hit[16] = (reg_addr == LC_CTRL_LC_ID_STATE_OFFSET);
1166     addr_hit[17] = (reg_addr == LC_CTRL_HW_REVISION0_OFFSET);
1167     addr_hit[18] = (reg_addr == LC_CTRL_HW_REVISION1_OFFSET);
1168     addr_hit[19] = (reg_addr == LC_CTRL_DEVICE_ID_0_OFFSET);
1169     addr_hit[20] = (reg_addr == LC_CTRL_DEVICE_ID_1_OFFSET);
1170     addr_hit[21] = (reg_addr == LC_CTRL_DEVICE_ID_2_OFFSET);
1171     addr_hit[22] = (reg_addr == LC_CTRL_DEVICE_ID_3_OFFSET);
1172     addr_hit[23] = (reg_addr == LC_CTRL_DEVICE_ID_4_OFFSET);
1173     addr_hit[24] = (reg_addr == LC_CTRL_DEVICE_ID_5_OFFSET);
1174     addr_hit[25] = (reg_addr == LC_CTRL_DEVICE_ID_6_OFFSET);
1175     addr_hit[26] = (reg_addr == LC_CTRL_DEVICE_ID_7_OFFSET);
1176     addr_hit[27] = (reg_addr == LC_CTRL_MANUF_STATE_0_OFFSET);
1177     addr_hit[28] = (reg_addr == LC_CTRL_MANUF_STATE_1_OFFSET);
1178     addr_hit[29] = (reg_addr == LC_CTRL_MANUF_STATE_2_OFFSET);
1179     addr_hit[30] = (reg_addr == LC_CTRL_MANUF_STATE_3_OFFSET);
1180     addr_hit[31] = (reg_addr == LC_CTRL_MANUF_STATE_4_OFFSET);
1181     addr_hit[32] = (reg_addr == LC_CTRL_MANUF_STATE_5_OFFSET);
1182     addr_hit[33] = (reg_addr == LC_CTRL_MANUF_STATE_6_OFFSET);
1183     addr_hit[34] = (reg_addr == LC_CTRL_MANUF_STATE_7_OFFSET);
1184 end
1185

```

```

1177 assign addrmiss = (reg_re || reg_we) ? ~|addr_hit : 1'b0 ;
1178
1179 // Check sub-word write is permitted
1180 always_comb begin
1181   wr_err = (reg_we &
1182     ((addr_hit[ 0] & ((LC_CTRL_REGS_PERMIT[ 0] &
~reg_be))) |
1183      (addr_hit[ 1] & ((LC_CTRL_REGS_PERMIT[ 1] &
~reg_be))) |
1184      (addr_hit[ 2] & ((LC_CTRL_REGS_PERMIT[ 2] &
~reg_be))) |
1185      (addr_hit[ 3] & ((LC_CTRL_REGS_PERMIT[ 3] &
~reg_be))) |
1186      (addr_hit[ 4] & ((LC_CTRL_REGS_PERMIT[ 4] &
~reg_be))) |
1187      (addr_hit[ 5] & ((LC_CTRL_REGS_PERMIT[ 5] &
~reg_be))) |
1188      (addr_hit[ 6] & ((LC_CTRL_REGS_PERMIT[ 6] &
~reg_be))) |
1189      (addr_hit[ 7] & ((LC_CTRL_REGS_PERMIT[ 7] &
~reg_be))) |
1190      (addr_hit[ 8] & ((LC_CTRL_REGS_PERMIT[ 8] &
~reg_be))) |
1191      (addr_hit[ 9] & ((LC_CTRL_REGS_PERMIT[ 9] &
~reg_be))) |
1192      (addr_hit[10] & ((LC_CTRL_REGS_PERMIT[10] &
~reg_be))) |
1193      (addr_hit[11] & ((LC_CTRL_REGS_PERMIT[11] &
~reg_be))) |
1194      (addr_hit[12] & ((LC_CTRL_REGS_PERMIT[12] &
~reg_be))) |
1195      (addr_hit[13] & ((LC_CTRL_REGS_PERMIT[13] &
~reg_be))) |
1196      (addr_hit[14] & ((LC_CTRL_REGS_PERMIT[14] &
~reg_be))) |
1197      (addr_hit[15] & ((LC_CTRL_REGS_PERMIT[15] &
~reg_be))) |
1198      (addr_hit[16] & ((LC_CTRL_REGS_PERMIT[16] &
~reg_be))) |
1199      (addr_hit[17] & ((LC_CTRL_REGS_PERMIT[17] &
~reg_be))) |
1200      (addr_hit[18] & ((LC_CTRL_REGS_PERMIT[18] &
~reg_be))) |
1201      (addr_hit[19] & ((LC_CTRL_REGS_PERMIT[19] &
~reg_be))) |
1202      (addr_hit[20] & ((LC_CTRL_REGS_PERMIT[20] &
~reg_be))) |
1203      (addr_hit[21] & ((LC_CTRL_REGS_PERMIT[21] &
~reg_be))) |
1204      (addr_hit[22] & ((LC_CTRL_REGS_PERMIT[22] &
~reg_be))) |
1205      (addr_hit[23] & ((LC_CTRL_REGS_PERMIT[23] &
~reg_be))) |
1206      (addr_hit[24] & ((LC_CTRL_REGS_PERMIT[24] &
~reg_be))) |
1207      (addr_hit[25] & ((LC_CTRL_REGS_PERMIT[25] &
~reg_be))) |
1208      (addr_hit[26] & ((LC_CTRL_REGS_PERMIT[26] &
~reg_be))) |
1209      (addr_hit[27] & ((LC_CTRL_REGS_PERMIT[27] &
~reg_be))) |
1210      (addr_hit[28] & ((LC_CTRL_REGS_PERMIT[28] &
~reg_be))) |
1211      (addr_hit[29] & ((LC_CTRL_REGS_PERMIT[29] &
~reg_be))) |
1212      (addr_hit[30] & ((LC_CTRL_REGS_PERMIT[30] &
~reg_be))) |
1213      (addr_hit[31] & ((LC_CTRL_REGS_PERMIT[31] &
~reg_be))) |
1214      (addr_hit[32] & ((LC_CTRL_REGS_PERMIT[32] &
~reg_be))) |
1215      (addr_hit[33] & ((LC_CTRL_REGS_PERMIT[33] &
~reg_be))) |
1216      (addr_hit[34] & ((LC_CTRL_REGS_PERMIT[34] &
~reg_be)))));
1217 end
1218
1219 // Generate write-enables
1220 assign alert_test_we = addr_hit[0] & reg_we & !reg_error;
1221
1222 assign alert_test_fatal_prog_error_wd = reg_wdata[0];
1223
1224 assign alert_test_fatal_state_error_wd = reg_wdata[1];
1225
1226 assign alert_test_fatal_bus_integ_error_wd = reg_wdata[2];
1227 assign status_re = addr_hit[1] & reg_re & !reg_error;
1228 assign claim_transition_if_regwen_we = addr_hit[2] & reg_we
& !reg_error;
1229
1230 assign claim_transition_if_regwen_wd = reg_wdata[0];
1231 assign claim_transition_if_re = addr_hit[3] & reg_re & !
reg_error;
1232 assign claim_transition_if_we = addr_hit[3] & reg_we & !
reg_error;
1233
1234 assign claim_transition_if_wd = reg_wdata[7:0];
1235 assign transition_regwen_re = addr_hit[4] & reg_re & !
reg_error;
1236 assign transition_cmd_we = addr_hit[5] & reg_we & !reg_er-
ror;
1237

```

```

1238 assign transition_cmd_wd = reg_wdata[0];
1239 assign transition_ctrl_re = addr_hit[6] & reg_re & !reg_er-
ror;
1240 assign transition_ctrl_we = addr_hit[6] & reg_we & !reg_er-
ror;
1241
1242 assign transition_ctrl_ext_clock_en_wd = reg_wdata[0];
1243
1244 assign transition_ctrl_volatile_raw_unlock_wd =
reg_wdata[1];
1245 assign transition_token_0_re = addr_hit[7] & reg_re & !
reg_error;
1246 assign transition_token_0_we = addr_hit[7] & reg_we & !
reg_error;
1247
1248 assign transition_token_0_wd = reg_wdata[31:0];
1249 assign transition_token_1_re = addr_hit[8] & reg_re & !
reg_error;
1250 assign transition_token_1_we = addr_hit[8] & reg_we & !
reg_error;
1251
1252 assign transition_token_1_wd = reg_wdata[31:0];
1253 assign transition_token_2_re = addr_hit[9] & reg_re & !
reg_error;
1254 assign transition_token_2_we = addr_hit[9] & reg_we & !
reg_error;
1255
1256 assign transition_token_2_wd = reg_wdata[31:0];
1257 assign transition_token_3_re = addr_hit[10] & reg_re & !
reg_error;
1258 assign transition_token_3_we = addr_hit[10] & reg_we & !
reg_error;
1259
1260 assign transition_token_3_wd = reg_wdata[31:0];
1261 assign transition_target_re = addr_hit[11] & reg_re & !
reg_error;
1262 assign transition_target_we = addr_hit[11] & reg_we & !
reg_error;
1263
1264 assign transition_target_wd = reg_wdata[29:0];
1265 assign otp_vendor_test_ctrl_re = addr_hit[12] & reg_re & !
reg_error;
1266 assign otp_vendor_test_ctrl_we = addr_hit[12] & reg_we & !
reg_error;
1267
1268 assign otp_vendor_test_ctrl_wd = reg_wdata[31:0];
1269 assign otp_vendor_test_status_re = addr_hit[13] & reg_re & !
reg_error;
1270 assign lc_state_re = addr_hit[14] & reg_re & !reg_error;
1271 assign lc_transition_cnt_re = addr_hit[15] & reg_re & !
reg_error;
1272 assign lc_id_state_re = addr_hit[16] & reg_re & !reg_error;
1273 assign hw_revision0_re = addr_hit[17] & reg_re & !reg_error;
1274 assign hw_revision1_re = addr_hit[18] & reg_re & !reg_error;
1275 assign device_id_0_re = addr_hit[19] & reg_re & !reg_error;
1276 assign device_id_1_re = addr_hit[20] & reg_re & !reg_error;
1277 assign device_id_2_re = addr_hit[21] & reg_re & !reg_error;
1278 assign device_id_3_re = addr_hit[22] & reg_re & !reg_error;
1279 assign device_id_4_re = addr_hit[23] & reg_re & !reg_error;
1280 assign device_id_5_re = addr_hit[24] & reg_re & !reg_error;
1281 assign device_id_6_re = addr_hit[25] & reg_re & !reg_error;
1282 assign device_id_7_re = addr_hit[26] & reg_re & !reg_error;
1283 assign manuf_state_0_re = addr_hit[27] & reg_re & !reg_er-
ror;
1284 assign manuf_state_1_re = addr_hit[28] & reg_re & !reg_er-
ror;
1285 assign manuf_state_2_re = addr_hit[29] & reg_re & !reg_er-
ror;
1286 assign manuf_state_3_re = addr_hit[30] & reg_re & !reg_er-
ror;
1287 assign manuf_state_4_re = addr_hit[31] & reg_re & !reg_er-
ror;
1288 assign manuf_state_5_re = addr_hit[32] & reg_re & !reg_er-
ror;
1289 assign manuf_state_6_re = addr_hit[33] & reg_re & !reg_er-
ror;
1290 assign manuf_state_7_re = addr_hit[34] & reg_re & !reg_er-
ror;
1291
1292 // Assign write-enables to checker logic vector.
1293 always_comb begin
1294   reg_we_check[0] = alert_test_we;
1295   reg_we_check[1] = 1'b0;
1296   reg_we_check[2] = claim_transition_if_regwen_we;
1297   reg_we_check[3] = claim_transition_if_gated_we;
1298   reg_we_check[4] = 1'b0;
1299   reg_we_check[5] = transition_cmd_gated_we;
1300   reg_we_check[6] = transition_ctrl_gated_we;
1301   reg_we_check[7] = transition_token_0_gated_we;
1302   reg_we_check[8] = transition_token_1_gated_we;
1303   reg_we_check[9] = transition_token_2_gated_we;
1304   reg_we_check[10] = transition_token_3_gated_we;
1305   reg_we_check[11] = transition_target_gated_we;
1306   reg_we_check[12] = otp_vendor_test_ctrl_gated_we;
1307   reg_we_check[13] = 1'b0;
1308   reg_we_check[14] = 1'b0;
1309   reg_we_check[15] = 1'b0;
1310   reg_we_check[16] = 1'b0;
1311   reg_we_check[17] = 1'b0;
1312   reg_we_check[18] = 1'b0;
1313   reg_we_check[19] = 1'b0;

```

```

1314 reg_we_check[20] = 1'b0;
1315 reg_we_check[21] = 1'b0;
1316 reg_we_check[22] = 1'b0;
1317 reg_we_check[23] = 1'b0;
1318 reg_we_check[24] = 1'b0;
1319 reg_we_check[25] = 1'b0;
1320 reg_we_check[26] = 1'b0;
1321 reg_we_check[27] = 1'b0;
1322 reg_we_check[28] = 1'b0;
1323 reg_we_check[29] = 1'b0;
1324 reg_we_check[30] = 1'b0;
1325 reg_we_check[31] = 1'b0;
1326 reg_we_check[32] = 1'b0;
1327 reg_we_check[33] = 1'b0;
1328 reg_we_check[34] = 1'b0;
1329 end
1330
1331 // Read data return
1332 always_comb begin
1333   reg_rdata_next = '0;
1334   unique case (1'b1)
1335     addr_hit[0]: begin
1336       reg_rdata_next[0] = '0;
1337       reg_rdata_next[1] = '0;
1338       reg_rdata_next[2] = '0;
1339     end
1340
1341     addr_hit[1]: begin
1342       reg_rdata_next[0] = status_initialized_qs;
1343       reg_rdata_next[1] = status_ready_qs;
1344       reg_rdata_next[2] = status_ext_clock_switched_qs;
1345       reg_rdata_next[3] = status_transition_successful_qs;
1346       reg_rdata_next[4] = status_transition_count_error_qs;
1347       reg_rdata_next[5] = status_transition_error_qs;
1348       reg_rdata_next[6] = status_token_error_qs;
1349       reg_rdata_next[7] = status_nvm_rma_error_qs;
1350       reg_rdata_next[8] = status_otp_error_qs;
1351       reg_rdata_next[9] = status_state_error_qs;
1352       reg_rdata_next[10] = status_bus_integ_error_qs;
1353       reg_rdata_next[11] = status_otp_partition_error_qs;
1354     end
1355
1356     addr_hit[2]: begin
1357       reg_rdata_next[0] = claim_transition_if_regwen_qs;
1358     end
1359
1360     addr_hit[3]: begin
1361       reg_rdata_next[7:0] = claim_transition_if_qs;
1362     end
1363
1364     addr_hit[4]: begin
1365       reg_rdata_next[0] = transition_regwen_qs;
1366     end
1367
1368     addr_hit[5]: begin
1369       reg_rdata_next[0] = '0;
1370     end
1371
1372     addr_hit[6]: begin
1373       reg_rdata_next[0] = transition_ctrl_ext_clock_en_qs;
1374       reg_rdata_next[1] = transition_ctrl_volatile_raw_unlock_qs;
1375     end
1376
1377     addr_hit[7]: begin
1378       reg_rdata_next[31:0] = transition_token_0_qs;
1379     end
1380
1381     addr_hit[8]: begin
1382       reg_rdata_next[31:0] = transition_token_1_qs;
1383     end
1384
1385     addr_hit[9]: begin
1386       reg_rdata_next[31:0] = transition_token_2_qs;
1387     end
1388
1389     addr_hit[10]: begin
1390       reg_rdata_next[31:0] = transition_token_3_qs;
1391     end
1392
1393     addr_hit[11]: begin
1394       reg_rdata_next[29:0] = transition_target_qs;
1395     end
1396
1397     addr_hit[12]: begin
1398       reg_rdata_next[31:0] = otp_vendor_test_ctrl_qs;
1399     end
1400
1401     addr_hit[13]: begin
1402       reg_rdata_next[31:0] = otp_vendor_test_status_qs;
1403     end
1404
1405     addr_hit[14]: begin
1406       reg_rdata_next[29:0] = lc_state_qs;
1407     end
1408
1409     addr_hit[15]: begin
1410       reg_rdata_next[4:0] = lc_transition_cnt_qs;
1411     end
1412
1413     addr_hit[16]: begin
1414       reg_rdata_next[31:0] = lc_id_state_qs;
1415     end
1416
1417     addr_hit[17]: begin
1418       reg_rdata_next[15:0] = hw_revision0_product_id_qs;
1419       reg_rdata_next[31:16] = hw_revision0_silicon_creator_id_qs;
1420     end
1421
1422     addr_hit[18]: begin
1423       reg_rdata_next[7:0] = hw_revision1_revision_id_qs;
1424       reg_rdata_next[31:8] = hw_revision1_reserved_qs;
1425     end
1426
1427     addr_hit[19]: begin
1428       reg_rdata_next[31:0] = device_id_0_qs;
1429     end
1430
1431     addr_hit[20]: begin
1432       reg_rdata_next[31:0] = device_id_1_qs;
1433     end
1434
1435     addr_hit[21]: begin
1436       reg_rdata_next[31:0] = device_id_2_qs;
1437     end
1438
1439     addr_hit[22]: begin
1440       reg_rdata_next[31:0] = device_id_3_qs;
1441     end
1442
1443     addr_hit[23]: begin
1444       reg_rdata_next[31:0] = device_id_4_qs;
1445     end
1446
1447     addr_hit[24]: begin
1448       reg_rdata_next[31:0] = device_id_5_qs;
1449     end
1450
1451     addr_hit[25]: begin
1452       reg_rdata_next[31:0] = device_id_6_qs;
1453     end
1454
1455     addr_hit[26]: begin
1456       reg_rdata_next[31:0] = device_id_7_qs;
1457     end
1458
1459     addr_hit[27]: begin
1460       reg_rdata_next[31:0] = manuf_state_0_qs;
1461     end
1462
1463     addr_hit[28]: begin
1464       reg_rdata_next[31:0] = manuf_state_1_qs;
1465     end
1466
1467     addr_hit[29]: begin
1468       reg_rdata_next[31:0] = manuf_state_2_qs;
1469     end
1470
1471     addr_hit[30]: begin
1472       reg_rdata_next[31:0] = manuf_state_3_qs;
1473     end
1474
1475     addr_hit[31]: begin
1476       reg_rdata_next[31:0] = manuf_state_4_qs;
1477     end
1478
1479     addr_hit[32]: begin
1480       reg_rdata_next[31:0] = manuf_state_5_qs;
1481     end
1482
1483     addr_hit[33]: begin
1484       reg_rdata_next[31:0] = manuf_state_6_qs;
1485     end
1486
1487     addr_hit[34]: begin
1488       reg_rdata_next[31:0] = manuf_state_7_qs;
1489     end
1490
1491     default: begin
1492       reg_rdata_next = '1;
1493     end
1494   endcase
1495 end
1496
1497 // shadow busy
1498 logic shadow_busy;
1499 assign shadow_busy = 1'b0;
1500
1501 // register busy
1502 assign reg_busy = shadow_busy;
1503
1504 // Unused signal tieoff
1505
1506 // wdata / byte enable are not always fully used
1507 // add a blanket unused statement to handle lint waivers
1508 logic unused_wdata;
1509 logic unused_be;
1510 assign unused_wdata = ^reg_wdata;
1511 assign unused_be = ^reg_be;
1512
1513 // Assertions for Register Interface

```

```

1514 `ASSERT_PULSE(wePulse, reg_we, clk_i, !rst_ni)
1515 `ASSERT_PULSE(rePulse, reg_re, clk_i, !rst_ni)
1516
1517 `ASSERT(reAfterRv, $rose(reg_re || reg_we) | =>
t1_o_pre_d_valid, clk_i, !rst_ni)
1518
1519 `ASSERT(en2addrHit, (reg_we || reg_re) | -> $one-
hot0(addr_hit), clk_i, !rst_ni)
1520
1521 // this is formulated as an assumption such that the FPV
testbenches do disprove this
1522 // property by mistake
1523 // ASSUME(reqParity, t1_reg_h2d.a_valid | ->
t1_reg_h2d.a_user.chk_en == t1u_pkg::CheckDis)
1524
1525 endmodule

```

8. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/ lc_ctrl_signal_decode.sv (450 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Life cycle signal decoder and sender module.
6
7 `include "prim_assert.sv"
8
9 module lc_ctrl_signal_decode
10 import lc_ctrl_pkg::*;
11 import lc_ctrl_state_pkg::*;
12 #(
13 // Random netlist constants
14 // SCRAP, RAW, TEST_LOCKED*, INVALID
15 parameter lc_keymgr_div_t RndCnstLcKeymgrDivInvalid =
LcKeymgrDivWidth'(0),
16 // TEST_UNLOCKED*
17 parameter lc_keymgr_div_t RndCnstLcKeymgrDivTestUnlocked =
LcKeymgrDivWidth'(1),
18 // DEV
19 parameter lc_keymgr_div_t RndCnstLcKeymgrDivDev =
LcKeymgrDivWidth'(2),
20 // PROD, PROD_END
21 parameter lc_keymgr_div_t RndCnstLcKeymgrDivProduction =
LcKeymgrDivWidth'(3),
22 // RMA
23 parameter lc_keymgr_div_t RndCnstLcKeymgrDivRma =
LcKeymgrDivWidth'(4)
24 ) (
25 input clk_i,
26 input rst_ni,
27 // Life cycle state vector.
28 input logic lc_state_valid_i,
29 input lc_state_e lc_state_i,
30 input fsm_state_e fsm_state_i,
31 input lc_tx_t secrets_valid_i,
32 // Local life cycle signal
33 output lc_tx_t lc_raw_test_rma_o,
34 // Life cycle broadcast outputs.
35 output lc_tx_t lc_init_done_o,
36 output lc_tx_t lc_dft_en_o,
37 output lc_tx_t lc_nvm_debug_en_o,
38 output lc_tx_t lc_hw_debug_clr_o,
39 output lc_tx_t lc_hw_debug_en_o,
40 output lc_tx_t lc_cpu_en_o,
41 output lc_tx_t lc_creator_seed_sw_rw_en_o,
42 output lc_tx_t lc_owner_seed_sw_rw_en_o,
43 output lc_tx_t lc_iso_part_sw_rd_en_o,
44 output lc_tx_t lc_iso_part_sw_wr_en_o,
45 output lc_tx_t lc_seed_hw_rd_en_o,
46 output lc_tx_t lc_rma_state_o,
47 output lc_tx_t lc_keymgr_en_o,
48 output lc_tx_t lc_escalate_en_o,
49 // State group diversification value for keymgr
50 output lc_keymgr_div_t lc_keymgr_div_o
51 );
52
53 //////////////////////////////////////////////////
54 // Signal Decoder Logic //
55 //////////////////////////////////////////////////
56
57 lc_tx_t lc_init_done, lc_raw_test_rma;
58 lc_tx_t lc_dft_en, lc_nvm_debug_en, lc_hw_debug_clr,
lc_hw_debug_en, lc_cpu_en, lc_keymgr_en,
59 lc_escalate_en;
60 lc_tx_t lc_creator_seed_sw_rw_en, lc_owner_seed_sw_rw_en,
lc_iso_part_sw_rd_en;
61 lc_tx_t lc_iso_part_sw_wr_en, lc_seed_hw_rd_en;
62 lc_tx_t lc_rma_state;
63 lc_keymgr_div_t lc_keymgr_div_d, lc_keymgr_div_q;
64
65 always_comb begin : p_lc_signal_decode
66 // Life cycle control signal defaults
67 lc_init_done = Off;
68 lc_raw_test_rma = Off;
69 lc_dft_en = Off;
70 lc_nvm_debug_en = Off;
71 lc_hw_debug_clr = Off;

```

```

72 lc_hw_debug_en = Off;
73 lc_cpu_en = Off;
74 lc_creator_seed_sw_rw_en = Off;
75 lc_owner_seed_sw_rw_en = Off;
76 lc_iso_part_sw_rd_en = Off;
77 lc_iso_part_sw_wr_en = Off;
78 lc_seed_hw_rd_en = Off;
79 lc_rma_state = Off;
80 lc_keymgr_en = Off;
81 // This ensures that once escalation has been triggered,
it cannot go back to Off.
82 lc_escalate_en = lc_tx_or_hi(Off, lc_escal-
ate_en_o);
83 // Set to invalid diversification value by default.
84 lc_keymgr_div_d = RndCnstLcKeymgrDivInvalid;
85
86 unique case (fsm_state_i)
87 //////////////////////////////////////////////////
88 // Don't broadcast anything in this state.
89 ResetSt: ;
90 //////////////////////////////////////////////////
91 // Broadcasting of most signals is only enabled during
the following life cycle states.
92 IdleSt,
93 ClkMuxSt,
94 CntIncrSt,
95 CntProgSt,
96 TransCheckSt,
97 NvmRmaSt,
98 TokenHashSt,
99 TokenCheck0St,
100 TokenCheck1St,
101 TransProgSt: begin
102 if (lc_state_valid_i) begin
103 // After this, all life cycle controller broadcast
signals are valid and based on the
104 // OTP LC state.
105 lc_init_done = On;
106 unique case (lc_state_i)
107 //////////////////////////////////////////////////
108 // Only enable life cycle TAP register for OTP
test mechanisms.
109 LcStRaw,
110 LcStTestLocked0,
111 LcStTestLocked1,
112 LcStTestLocked2,
113 LcStTestLocked3,
114 LcStTestLocked4,
115 LcStTestLocked5,
116 LcStTestLocked6: begin
117 lc_raw_test_rma = On;
118 end
119 //////////////////////////////////////////////////
120 // Enable DFT and debug functionality, including
the CPU in the
121 // test unlocked states.
122 LcStTestUnlocked0,
123 LcStTestUnlocked1,
124 LcStTestUnlocked2,
125 LcStTestUnlocked3,
126 LcStTestUnlocked4,
127 LcStTestUnlocked5,
128 LcStTestUnlocked6: begin
129 lc_raw_test_rma = On;
130 lc_dft_en = On;
131 lc_nvm_debug_en = On;
132 lc_hw_debug_en = On;
133 lc_cpu_en = On;
134 lc_iso_part_sw_wr_en = On;
135 lc_keymgr_div_d = RndCnstLcKeymgrDivTestUn-
locked;
136 end
137 //////////////////////////////////////////////////
138 // This is the last TEST_UNLOCKED state. The same
feature set is enabled
139 // as in the other TEST_UNLOCKED states above, ex-
cept for NVM debug en,
140 // which is disabled in this state.
141 LcStTestUnlocked7: begin
142 lc_raw_test_rma = On;
143 lc_dft_en = On;
144 lc_hw_debug_en = On;
145 lc_cpu_en = On;
146 lc_iso_part_sw_wr_en = On;
147 lc_keymgr_div_d = RndCnstLcKeymgrDivTestUn-
locked;
148 end
149 //////////////////////////////////////////////////
150 // Enable production functions
151 LcStProd,
152 LcStProdEnd: begin
153 lc_cpu_en = On;
154 lc_keymgr_en = On;
155 lc_owner_seed_sw_rw_en = On;
156 lc_iso_part_sw_wr_en = On;

```

```

157         lc_iso_part_sw_rd_en = On;
158         lc_keymgr_div_d      = RndCnstLcKeymgrDivPro-
duction;

159         // Only allow provisioning if the device has not
yet been personalized.
160         // If secrets_valid_i is set to ON, we output
OFF.
161         // Note that we can convert ON to OFF with a
bitwise inversion due to the encoding.
162         lc_creator_seed_sw_rw_en =
lc_tx_t'(~secrets_valid_i);

163         // Only allow hardware to consume the seeds once
personalized.
164         // If secrets_valid_i is set to ON, we output
ON.
165         lc_seed_hw_rd_en = secrets_valid_i;
166         end
167         //////////////////////////////////////
168         // Similar functions as PROD, with the following
differences:
169         // - hardware debug functionality (CPU TAP) is en-
abled,
170         // - access to the isolated NVM partition is dis-
abled.
171         LcStDev: begin
172             lc_hw_debug_en      = On;
173             lc_cpu_en           = On;
174             lc_keymgr_en        = On;
175             lc_owner_seed_sw_rw_en = On;
176             lc_iso_part_sw_wr_en = On;
177             lc_keymgr_div_d      = RndCnstLcKeymgrDivDev;

178         // Only allow provisioning if the device has not
yet been personalized.
179         // If secrets_valid_i is set to ON, we output
OFF.
180         // Note that we can convert ON to OFF with a
bitwise inversion due to the encoding.
181         lc_creator_seed_sw_rw_en =
lc_tx_t'(~secrets_valid_i);

182         // Only allow hardware to consume the seeds once
personalized.
183         // If secrets_valid_i is set to ON, we output
ON.
184         lc_seed_hw_rd_en = secrets_valid_i;
185         end
186         //////////////////////////////////////
187         // Enable all test and production functions.
188         LcStRma: begin
189             lc_raw_test_rma      = On;
190             lc_dft_en            = On;
191             lc_nvm_debug_en      = On;
192             lc_hw_debug_en       = On;
193             lc_cpu_en            = On;
194             lc_keymgr_en         = On;
195             lc_creator_seed_sw_rw_en = On;
196             lc_owner_seed_sw_rw_en = On;
197             lc_iso_part_sw_wr_en = On;
198             lc_iso_part_sw_rd_en = On;
199             lc_seed_hw_rd_en     = On;
200             lc_rma_state         = On;
201             lc_keymgr_div_d      = RndCnstLcKeymgrDi-
vRma;
202         end
203         //////////////////////////////////////
204         // Invalid or scrapped life cycle state, make sure
the escalation
205         // signal is also asserted in this case.
206         default: begin
207             lc_escalate_en = On;
208             lc_hw_debug_clr = On;
209         end
210         endcase // lc_state_i
211     end else begin
212         lc_escalate_en = On;
213         lc_hw_debug_clr = On;
214     end
215 end
216 //////////////////////////////////////
217 // Post-transition state. Behaves similarly to the vir-
tual scrap
218 // states below, with the exception that escalate_en is
NOT asserted,
219 // since that could trigger unwanted alerts / escala-
tions and system resets.
220 PostTransSt: begin
221     // At this point, all life cycle controller broadcast
signals are valid and based on the
222     // OTP LC state
223     lc_init_done = On;
224     // Explicitly clear any flops holding the lc_hw_de-
bug_en signal. Those flops would retain
225     // their state if just the lc_hw_debug_en signal is

```

```

switched off, in order to let debug
226     // connections survive a reset of lc_ctrl.
227     lc_hw_debug_clr = On;
228     end
229     //////////////////////////////////////
230     // Virtual scrap states, make sure the escalation signal
is
231     // also asserted in this case.
232     ScrapSt,
233     EscalateSt,
234     InvalidSt: begin
235         // At this point, all life cycle controller broadcast
signals are valid and based on the
236         // OTP LC state.
237         lc_init_done = On;
238         lc_escalate_en = On;
239         lc_hw_debug_clr = On;
240     end
241     default: begin
242         lc_escalate_en = On;
243         lc_hw_debug_clr = On;
244     end
245 endcase // fsm_state_i
246 end
247
248 //////////////////////////////////////
249 // Control signal output flops //
250 //////////////////////////////////////
251
252 prim_lc_sender u_prim_lc_sender_init_done (
253     .clk_i,
254     .rst_ni,
255     .lc_en_i(lc_init_done),
256     .lc_en_o(lc_init_done_o)
257 );
258 prim_lc_sender u_prim_lc_sender_raw_test_rma (
259     .clk_i,
260     .rst_ni,
261     .lc_en_i(lc_raw_test_rma),
262     .lc_en_o(lc_raw_test_rma_o)
263 );
264 prim_lc_sender u_prim_lc_sender_dft_en (
265     .clk_i,
266     .rst_ni,
267     .lc_en_i(lc_dft_en),
268     .lc_en_o(lc_dft_en_o)
269 );
270 prim_lc_sender u_prim_lc_sender_nvm_debug_en (
271     .clk_i,
272     .rst_ni,
273     .lc_en_i(lc_nvm_debug_en),
274     .lc_en_o(lc_nvm_debug_en_o)
275 );
276 prim_lc_sender u_prim_lc_sender_hw_debug_clr (
277     .clk_i,
278     .rst_ni,
279     .lc_en_i(lc_hw_debug_clr),
280     .lc_en_o(lc_hw_debug_clr_o)
281 );
282 prim_lc_sender u_prim_lc_sender_hw_debug_en (
283     .clk_i,
284     .rst_ni,
285     .lc_en_i(lc_hw_debug_en),
286     .lc_en_o(lc_hw_debug_en_o)
287 );
288 prim_lc_sender u_prim_lc_sender_cpu_en (
289     .clk_i,
290     .rst_ni,
291     .lc_en_i(lc_cpu_en),
292     .lc_en_o(lc_cpu_en_o)
293 );
294 prim_lc_sender u_prim_lc_sender_creator_seed_sw_rw_en (
295     .clk_i,
296     .rst_ni,
297     .lc_en_i(lc_creator_seed_sw_rw_en),
298     .lc_en_o(lc_creator_seed_sw_rw_en_o)
299 );
300 prim_lc_sender u_prim_lc_sender_owner_seed_sw_rw_en (
301     .clk_i,
302     .rst_ni,
303     .lc_en_i(lc_owner_seed_sw_rw_en),
304     .lc_en_o(lc_owner_seed_sw_rw_en_o)
305 );
306 prim_lc_sender u_prim_lc_sender_iso_part_sw_rd_en (
307     .clk_i,
308     .rst_ni,
309     .lc_en_i(lc_iso_part_sw_rd_en),
310     .lc_en_o(lc_iso_part_sw_rd_en_o)
311 );
312 prim_lc_sender u_prim_lc_sender_iso_part_sw_wr_en (
313     .clk_i,
314     .rst_ni,
315     .lc_en_i(lc_iso_part_sw_wr_en),
316     .lc_en_o(lc_iso_part_sw_wr_en_o)
317 );
318 prim_lc_sender u_prim_lc_sender_seed_hw_rd_en (
319     .clk_i,
320     .rst_ni,
321     .lc_en_i(lc_seed_hw_rd_en),

```



```

322     .lc_en_o(lc_seed_hw_rd_en_o)
323 );
324 prim_lc_sender u_prim_lc_sender_rma_state (
325     .clk_i,
326     .rst_ni,
327     .lc_en_i(lc_rma_state),
328     .lc_en_o(lc_rma_state_o)
329 );
330 prim_lc_sender u_prim_lc_sender_keymgr_en (
331     .clk_i,
332     .rst_ni,
333     .lc_en_i(lc_keymgr_en),
334     .lc_en_o(lc_keymgr_en_o)
335 );
336 prim_lc_sender u_prim_lc_sender_escalate_en (
337     .clk_i,
338     .rst_ni,
339     .lc_en_i(lc_escalate_en),
340     .lc_en_o(lc_escalate_en_o)
341 );
342
343 assign lc_keymgr_div_o = lc_keymgr_div_q;
344
345 prim_flop #(
346     .Width(LcKeymgrDivWidth),
347     .ResetValue(RndCnstLcKeymgrDivInvalid)
348 ) u_prim_flop_keymgr_div (
349     .clk_i ( clk_i ),
350     .rst_ni ( rst_ni ),
351     .d_i ( lc_keymgr_div_d ),
352     .q_o ( lc_keymgr_div_q )
353 );
354
355 //////////////////////////////////////////////////
356 // Assertions //
357 //////////////////////////////////////////////////
358
359 // Need to make sure that the random netlist constants are
360 // all unique.
361 `ASSERT_INIT(LcKeymgrDivUnique0_A,
362     !(RndCnstLcKeymgrDivInvalid inside {RndCnstLcKeymgrDivTestUnlocked,
363                                         RndCnstLcKeymgrDivDev,
364                                         RndCnstLcKeymgrDivRma,
365                                         RndCnstLcKeymgrDivProduction}))
366 `ASSERT_INIT(LcKeymgrDivUnique1_A,
367     !(RndCnstLcKeymgrDivTestUnlocked inside {RndCnstLcKeymgrDivInvalid,
368                                                RndCnstLcKeymgrDivDev,
369                                                RndCnstLcKeymgrDivRma,
370                                                RndCnstLcKeymgrDivProduction}))
371 `ASSERT_INIT(LcKeymgrDivUnique2_A,
372     !(RndCnstLcKeymgrDivDev inside {RndCnstLcKeymgrDivInvalid,
373                                     RndCnstLcKeymgrDivTestUnlocked,
374                                     RndCnstLcKeymgrDivRma,
375                                     RndCnstLcKeymgrDivProduction}))
376 `ASSERT_INIT(LcKeymgrDivUnique3_A,
377     !(RndCnstLcKeymgrDivRma inside {RndCnstLcKeymgrDivInvalid,
378                                     RndCnstLcKeymgrDivTestUnlocked,
379                                     RndCnstLcKeymgrDivDev,
380                                     RndCnstLcKeymgrDivProduction}))
381 `ASSERT(SignalsAreOffWhenNotEnabled_A,
382     !lc_state_valid_i
383     |>
384     lc_tx_test_false_strict(lc_raw_test_rma_o) &&
385     lc_tx_test_false_strict(lc_dft_en_o) &&
386     lc_tx_test_false_strict(lc_nvm_debug_en_o) &&
387     lc_tx_test_false_strict(lc_hw_debug_en_o) &&
388     lc_tx_test_false_strict(lc_cpu_en_o) &&
389     lc_tx_test_false_strict(lc_creator_seed_sw_rw_en_o) &&
390     lc_tx_test_false_strict(lc_owner_seed_sw_rw_en_o) &&
391     lc_tx_test_false_strict(lc_iso_part_sw_rd_en_o) &&
392     lc_tx_test_false_strict(lc_iso_part_sw_wr_en_o) &&
393     lc_tx_test_false_strict(lc_seed_hw_rd_en_o) &&
394     lc_tx_test_false_strict(lc_rma_state_o) &&
395     lc_tx_test_false_strict(lc_keymgr_en_o) &&
396     lc_tx_test_false_strict(lc_dft_en_o) &&
397     lc_keymgr_div_o == RndCnstLcKeymgrDivInvalid)
398
399
400 `ASSERT(FsmInScrap_A,
401     !(fsm_state_i inside {ResetSt,
402                           TransProgSt,
403                           IdleSt,
404                           ClkMuxSt,
405                           CntIncrSt,
406                           CntProgSt,
407                           TransCheckSt,

```

```

408                           NvmRmaSt,
409                           TokenHashSt,
410                           TokenCheck0St,
411                           TokenCheck1St,
412                           PostTransSt}))
413     |>
414     lc_tx_test_true_strict(lc_escalate_en_o))
415
416 `ASSERT(StateInScrap_A,
417     lc_state_valid_i &&
418     fsm_state_i inside {IdleSt,
419                         ClkMuxSt,
420                         CntIncrSt,
421                         CntProgSt,
422                         TransCheckSt,
423                         NvmRmaSt,
424                         TokenHashSt,
425                         TokenCheck0St,
426                         TokenCheck1St} &&
427     !(lc_state_i inside {LcStRaw,
428                         LcStTestUnlocked0,
429                         LcStTestUnlocked1,
430                         LcStTestUnlocked2,
431                         LcStTestUnlocked3,
432                         LcStTestUnlocked4,
433                         LcStTestUnlocked5,
434                         LcStTestUnlocked6,
435                         LcStTestUnlocked7,
436                         LcStTestLocked0,
437                         LcStTestLocked1,
438                         LcStTestLocked2,
439                         LcStTestLocked3,
440                         LcStTestLocked4,
441                         LcStTestLocked5,
442                         LcStTestLocked6,
443                         LcStDev,
444                         LcStProd,
445                         LcStProdEnd,
446                         LcStRma}))
447     |>
448     lc_tx_test_true_strict(lc_escalate_en_o))
449
450 endmodule : lc_ctrl_signal_decode

```

9. lowRISC_opentitan/hw/ip/lc_ctrl/rtl/lc_ctrl_state_decode.sv (167 lines)

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for details.

3 // SPDX-License-Identifier: Apache-2.0

4 //

5 // Life cycle state decoder. This is a purely combinational module.

```

6
7 module lc_ctrl_state_decode
8     import lc_ctrl_pkg::*;
9     import lc_ctrl_state_pkg::*;
10
11 // Life cycle state vector.
12 input logic lc_state_valid_i,
13 input lc_state_e lc_state_i,
14 input lc_cnt_e lc_cnt_i,
15 input lc_tx_t secrets_valid_i,
16 // Main FSM state.
17 input fsm_state_e fsm_state_i,
18 // Decoded state vector.
19 output ext_dec_lc_state_t dec_lc_state_o,
20 output dec_lc_id_state_e dec_lc_id_state_o,
21 output dec_lc_cnt_t dec_lc_cnt_o,
22 output logic [5:0] state_invalid_error_o
23 );
24
25 //////////////////////////////////////////////////
26 // Signal Decoder Logic //
27 //////////////////////////////////////////////////
28
29 // SEC_CM: STATE.CONFIG.SPARSE
30 // The decoded life cycle state uses a redundant representation that is used internally
31 // and in the CSR node.
32 ext_dec_lc_state_t dec_lc_state;
33 logic [$bits(ext_dec_lc_state_t)-1:0] dec_lc_state_buf;
34 prim_sec_anchor_buf #(
35     .Width($bits(ext_dec_lc_state_t))
36 ) u_prim_sec_anchor_buf (
37     .in_i(dec_lc_state),
38     .out_o(dec_lc_state_buf)
39 );
40
41 // This cast is needed so that VCS does not throw warnings.
42 for (genvar k = 0; k < DecLcStateNumRep; k++) begin :
43     gen_enum_casts
44         assign dec_lc_state_o[k] =
45         dec_lc_state_e'(dec_lc_state_buf[k*DecLcStateWidth +:
46         DecLcStateWidth]);
47     end

```

```

46 // The decoder logic below decodes the life cycle state vec-
tor and counter
47 // into a format that can be exposed in the CSRs. If the
state is invalid,
48 // this will be flagged as well.
49
50 always_comb begin : p_lc_state_decode
51 // Decoded state defaults
52 dec_lc_state = {DecLcStateNumRep{DecLcStInvalid}};
53 dec_lc_cnt_o = {DecLcCountWidth{1'b1}};
54 dec_lc_id_state_o = DecLcIdInvalid;
55 state_invalid_error_o = '0;
56
57 unique case (fsm_state_i)
58 // Don't decode anything in ResetSt
59 ResetSt :
60 // These are temporary, terminal states that are not en-
coded
61 // in the persistent LC state vector from OTP, hence we
decode them first.
62 EscalateSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStEscalate}};
63 PostTransSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStPostTrans}};
64 InvalidSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStInvalid}};
65 ScrapSt: dec_lc_state = {DecLcStateNum-
Rep{DecLcStScrap}};
66 // Otherwise check and decode the life cycle state con-
tinuously.
67 default: begin
68
69 // Note that we require that the valid signal from OTP is
in ResetSt.
70 // This will trigger an invalid_state_error when the
OTP partition
71 // is corrupt and moved into an error state, where the
valid bit is
72 // deasserted.
73 state_invalid_error_o[0] = ~lc_state_valid_i;
74
75 unique case (lc_state_i)
76 LcStRaw: dec_lc_state = {DecLcStateNum-
Rep{DecLcStRaw}};
77 LcStTestUnlocked0: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked0}};
78 LcStTestLocked0: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked0}};
79 LcStTestUnlocked1: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked1}};
80 LcStTestLocked1: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked1}};
81 LcStTestUnlocked2: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked2}};
82 LcStTestLocked2: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked2}};
83 LcStTestUnlocked3: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked3}};
84 LcStTestLocked3: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked3}};
85 LcStTestUnlocked4: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked4}};
86 LcStTestLocked4: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked4}};
87 LcStTestUnlocked5: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked5}};
88 LcStTestLocked5: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked5}};
89 LcStTestUnlocked6: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked6}};
90 LcStTestLocked6: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestLocked6}};
91 LcStTestUnlocked7: dec_lc_state = {DecLcStateNum-
Rep{DecLcStTestUnlocked7}};
92 LcStDev: dec_lc_state = {DecLcStateNum-
Rep{DecLcStDev}};
93 LcStProd: dec_lc_state = {DecLcStateNum-

```

```

Rep{DecLcStProd}};
94 LcStProdEnd: dec_lc_state = {DecLcStateNum-
Rep{DecLcStProdEnd}};
95 LcStRma: dec_lc_state = {DecLcStateNum-
Rep{DecLcStRma}};
96 LcStScrap: dec_lc_state = {DecLcStateNum-
Rep{DecLcStScrap}};
97 // SEC_CM: MANUF.STATE.BKGN_CHK
98 default: state_invalid_error_o[1] = 1'b1;
99 endcase // lc_state_i
100
101 unique case (lc_cnt_i)
102 LcCnt0: dec_lc_cnt_o = 5'd0;
103 LcCnt1: dec_lc_cnt_o = 5'd1;
104 LcCnt2: dec_lc_cnt_o = 5'd2;
105 LcCnt3: dec_lc_cnt_o = 5'd3;
106 LcCnt4: dec_lc_cnt_o = 5'd4;
107 LcCnt5: dec_lc_cnt_o = 5'd5;
108 LcCnt6: dec_lc_cnt_o = 5'd6;
109 LcCnt7: dec_lc_cnt_o = 5'd7;
110 LcCnt8: dec_lc_cnt_o = 5'd8;
111 LcCnt9: dec_lc_cnt_o = 5'd9;
112 LcCnt10: dec_lc_cnt_o = 5'd10;
113 LcCnt11: dec_lc_cnt_o = 5'd11;
114 LcCnt12: dec_lc_cnt_o = 5'd12;
115 LcCnt13: dec_lc_cnt_o = 5'd13;
116 LcCnt14: dec_lc_cnt_o = 5'd14;
117 LcCnt15: dec_lc_cnt_o = 5'd15;
118 LcCnt16: dec_lc_cnt_o = 5'd16;
119 LcCnt17: dec_lc_cnt_o = 5'd17;
120 LcCnt18: dec_lc_cnt_o = 5'd18;
121 LcCnt19: dec_lc_cnt_o = 5'd19;
122 LcCnt20: dec_lc_cnt_o = 5'd20;
123 LcCnt21: dec_lc_cnt_o = 5'd21;
124 LcCnt22: dec_lc_cnt_o = 5'd22;
125 LcCnt23: dec_lc_cnt_o = 5'd23;
126 LcCnt24: dec_lc_cnt_o = 5'd24;
127 // SEC_CM: TRANSITION.CTR.BKGN_CHK
128 default: state_invalid_error_o[2] = 1'b1;
129 endcase // lc_cnt_i
130
131 // SEC_CM: MANUF.STATE.BKGN_CHK
132 unique case (secrets_valid_i)
133 // If the secrets have not been provisioned, the ID
state is "blank".
134 Off: dec_lc_id_state_o = DecLcIdBlank;
135 // If the secrets have been provisioned, the ID
state is "personalized".
136 On: dec_lc_id_state_o = DecLcIdPersonalized;
137 default: state_invalid_error_o[3] = 1'b1;
138 endcase // secrets_valid_i
139
140 // Require that any non-raw state has a valid, nonzero
transition count.
141 // SEC_CM: TRANSITION.CTR.BKGN_CHK
142 if (lc_state_i != LcStRaw && lc_cnt_i == LcCnt0) begin
143 state_invalid_error_o[4] = 1'b1;
144 end
145
146 // We can't have a personalized device that is
// still in RAW or any of the test states.
147 // SEC_CM: MANUF.STATE.BKGN_CHK
148 if (lc_tx_test_true_strict(secrets_valid_i) &&
!(lc_state_i inside {LcStDev,
149 LcStProd,
150 LcStProdEnd,
151 LcStRma,
152 LcStScrap})) begin
153 state_invalid_error_o[5] = 1'b1;
154 end
155 end
156 endcase // lc_id_state_i
157 end
158
159 ///////////////////////////////////////////////////////////////////
160 // Assertions //
161 ///////////////////////////////////////////////////////////////////
162
163 endmodule : lc_ctrl_state_decode

```


플래시 컨트롤러

1 files • 29 lines

API — flash_ctrl_pkg.sv

Table 638. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv — module / interface / package (1개)

이름	종류
flash_ctrl_pkg	package

Table 639. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv — parameter (6개)

파라미터	형/폭	기본값
NumSeeds	int	2
CreatorSeedIdx	bit [0:0]	0
OwnerSeedIdx	bit [0:0]	1
SeedWidth	int	256
KeyWidth	int	128
KEYMGR_FLASH_DEFAULT	keymgr_flash_t	'{

Table 640. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv — typedef / using (1개)

새 이름	원래 형
flash_key_t	logic [KeyWidth-1:0]

1. lowRISC_opentitan/hw/ip/flash_ctrl/rtl/flash_ctrl_pkg.sv (29 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Flash Controller generic package.
6 // This has no top-specific items so it is okay to use by gen-
  eric IPs.
7
8 package flash_ctrl_pkg;
9 // Parameters for keymgr.
10 parameter int NumSeeds = 2;
11 parameter bit [0:0] CreatorSeedIdx = 0;
12 parameter bit [0:0] OwnerSeedIdx = 1;
```

```

13 parameter int SeedWidth = 256;
14 parameter int KeyWidth = 128;
15 typedef logic [KeyWidth-1:0] flash_key_t;
16
17 // flash_ctrl to keymgr
18 typedef struct packed {
19     logic [NumSeeds-1:0][SeedWidth-1:0] seeds;
20 } keymgr_flash_t;
21
22 parameter keymgr_flash_t KEYMGR_FLASH_DEFAULT = '{
23     seeds: '{
24         256'h9152e32c9380a4bcc3e0-
ab263581e6b0e8825186e1e445631646e8bef8c45d47,
25         256'hfa365df52da48cd752fb3a026a8e608f0098cfe5-
fa9810494829d0cd9479eb78
26     }
27 };
28
29 endpackage : flash_ctrl_pkg
```

OpenTitan 기본 셀 라이브러리

What. 동기화기 · FIFO · 카운터 · LFSR · 아비터 · 패리티/ECC · 보안 프리미티브 (이중화 레지스터, 상보 신호) 모음.

Why it exists. 같은 것을 두 번 만들지 않기 위해서 이고, 더 중요하게는 미묘한 것을 한 번만 제대로 만들기 위해서다. 동기화기를 블록마다 각자 쓰면 그중 하나는 틀린다. 라이브러리에 두고 검증하면 한 번만 맞으면 된다.

학부 복습 · **EECS concepts.** 메타스테이빌리티와 MTBF · 그레이 부호 · LFSR 과 원시다항식 · 라운드로빈 중재의 공정성.

석사 수준 · **Practice note.** 1인 디자인 하우스가 가장 먼저 만들어야 할 것이 이 라이브러리다. 블록을 팔려면 그 안의 기본 셀들이 이미 검증돼 있어야 한다. OpenTitan 의 `prim_` 접두 모듈을 읽고 자기 것을 만들어라 — 라이선스 (Apache-2.0)가 허용한다.

28 files • 5,123 lines

API — `prim_alert_pkg.sv`

Table 641. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_pkg.sv` — module / interface / package (1개)

이름	종류
<code>prim_alert_pkg</code>	package

Table 642. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_pkg.sv` — parameter (2개)

파라미터	형/폭	기본값
<code>ALERT_TX_DEFAULT</code>	<code>alert_tx_t</code>	<code>{alert_p: 1'b0}</code>
<code>ALERT_RX_DEFAULT</code>	<code>alert_rx_t</code>	<code>{ping_p: 1'b0}</code>

API — `prim_alert_receiver.sv`

Table 643. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv` — module / interface / package (1개)

이름	종류
<code>prim_alert_receiver</code>	module

Table 644. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv` — parameter (2개)

파라미터	형/폭	기본값
<code>AsyncOn</code>	bit	<code>1'b0</code>
<code>SkewCycles</code>	int unsigned	1

Table 645. `lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv` — 포트 (9개)

포트	방향	형/폭
<code>clk_i</code>	input	
<code>rst_ni</code>	input	

포트	방향	형/폭
init_trig_i	input	mubi4_t
ping_req_i	input	
ping_ok_o	output	logic
integ_fail_o	output	logic
alert_o	output	logic
alert_rx_o	output	alert_rx_t
alert_tx_i	input	alert_tx_t

Table 646. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [2:0] {Idle, HsAckWait, Pause0, Pause1, InitReq, InitAckWait}

API — prim_alert_sender.sv

Table 647. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv — module / interface / package (1개)

이름	종류
prim_alert_sender	module

Table 648. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv — parameter (3개)

파라미터	형/폭	기본값
Async0n	bit	1'b1
SkewCycles	int unsigned	1
IsFatal	bit	1'b0

Table 649. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
alert_test_i	input	
alert_req_i	input	
alert_ack_o	output	logic
alert_state_o	output	logic
alert_rx_i	input	alert_rx_t
alert_tx_o	output	alert_tx_t

API — prim_alert_to_diff.sv

Table 650. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv — module / interface / package (1개)

이름	종류
prim_alert_to_diff	module

Table 651. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv — parameter (2개)

파라미터	형/폭	기본값
AsyncOn	bit	1'b0
SkewCycles	int unsigned	1

Table 652. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
alert_rx_o	output	prim_alert_pkg::alert_rx_t
alert_tx_i	input	prim_alert_pkg::alert_tx_t
diff_po	output	logic
diff_no	output	logic

API — prim_arbiter_fixed.sv**Table 653. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — module / interface / package (1개)**

이름	종류
prim_arbiter_fixed	module

Table 654. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — parameter (3개)

파라미터	형/폭	기본값
N	int	8
DW	int	32
EnDataPort	bit	1

Table 655. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — localparam (6개)

상수	형/폭	값
IdxW	int	$\$clog2(N)$
Base0	int	$(2^{**level})-1$
Base1	int	$(2^{**level+1})-1$
Pa	int	Base0 + offset
C0	int	Base1 + 2*offset
C1	int	Base1 + 2*offset + 1

Table 656. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	

API — prim_arbiter_ppc.sv

Table 657. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — module / interface / package (1개)

이름	종류
prim_arbiter_ppc	module

Table 658. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — parameter (3개)

파라미터	형/폭	기본값
N	int unsigned	8
DW	int unsigned	32
EnDataPort	bit	1

Table 659. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — localparam (1개)

상수	형/폭	값
IdxW	int	prim_util_pkg::vbits(N)

Table 660. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_chk_i	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	

API — prim_arbiter_tree.sv**Table 661. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — module / interface / package (1개)**

이름	종류
prim_arbiter_tree	module

Table 662. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — parameter (3개)

파라미터	형/폭	기본값
N	int	8
DW	int	32
EnDataPort	bit	1

Table 663. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — localparam (6개)

상수	형/폭	값
IdxW	int	$\$clog2(N)$
Base0	int	$(2^{**level})-1$
Base1	int	$(2^{**(\text{level}+1)})-1$
Pa	int	Base0 + offset
C0	int	Base1 + 2*offset
C1	int	Base1 + 2*offset + 1

Table 664. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_chk_i	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	

API — prim_arbiter_tree_dup.sv**Table 665. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — module / interface / package (1개)**

이름	종류
prim_arbiter_tree_dup	module

Table 666. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — parameter (4개)

파라미터	형/폭	기본값
N	int	8
DW	int	32
EnDataPort	bit	1
FixedArb	bit	0

Table 667. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — localparam (2개)

상수	형/폭	값
IdxW	int	\$clog2(N)
ArbInstances	int	2

Table 668. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
req_chk_i	input	
req_i	input	[N-1:0]
gnt_o	output	logic [N-1:0]
idx_o	output	logic [IdxW-1:0]
valid_o	output	logic
data_o	output	logic [DW-1:0]
ready_i	input	
err_o	output	logic

API — prim_ascon_duplex.sv

Table 669. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_duplex.sv — module / interface / package (1개)

이름	종류
prim_ascon_duplex	module

Table 670. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_duplex.sv — 포트 (24개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
ascon_variant	input	duplex_variant_e
ascon_operation	input	duplex_op_e
start_i	input	logic
done_o	output	logic
idle_o	output	prim_mubi_pkg::mubi4_t

포트	방향	형/폭
no_ad_i	input	prim_mubi_pkg::mubi4_t
no_msg_i	input	prim_mubi_pkg::mubi4_t
key_i	input	logic [127:0]
nonce_i	input	logic [127:0]
data_in_i	input	logic [127:0]
data_in_valid_bytes_i	input	logic [4:0]
last_block_ad_i	input	prim_mubi_pkg::mubi4_t
last_block_msg_i	input	prim_mubi_pkg::mubi4_t
data_in_valid_i	input	logic
data_in_ready_o	output	logic
data_out_o	output	logic [127:0]
data_out_ready_i	input	logic
data_out_valid_o	output	logic
tag_out_o	output	logic [127:0]
tag_out_valid_o	output	logic
fsm_state_o	output	duplex_fsm_state_e
err_o	output	logic

API — prim_ascon_pkg.sv

Table 671. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv — module / interface / package (1개)

이름	종류
prim_ascon_pkg	package

Table 672. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv — parameter (8개)

파라미터	형/폭	기본값
AsconRoundCountW	int	4
DUPLEX_OP_WIDTH	int	3
DUPLEX_VARIANT_WIDTH	int	2
KEY_HI_LOW_MUX_WIDTH	int	1
WORD_LOW_KEY_HI_MUX_WIDTH	int	1
ASCON_WORD_MUX_WIDTH	int	2
ROUND_INPUT_MUX_WIDTH	int	1
PADDING_MUX_WIDTH	int	2

Table 673. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv — localparam (4개)

상수	형/폭	값
IV_128	logic [63:0]	64'h80400c0600000000
IV_128A	logic [63:0]	64'h80800c0800000000
ROUND_MAX	logic [3:0]	4'b1011
AsconDuplexFSMStateWidth	int	10

API — prim_ascon_round.sv

Table 674. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_round.sv — module / interface / package (1개)

이름	종류
prim_ascon_round	module

Table 675. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_round.sv — 포트 (3개)

포트	방향	형/폭
state_o	output	logic [319:0]
state_i	input	logic [319:0]
rcon_i	input	logic [7:0]

API — prim_ascon_sbox.sv

Table 676. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_sbox.sv — module / interface / package (1개)

이름	종류
prim_ascon_sbox	module

Table 677. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_sbox.sv — 포트 (2개)

포트	방향	형/폭
state_o	output	logic [4:0]
state_i	input	logic [4:0]

API — prim_assert.sv

Table 678. lowRISC_opentitan/hw/ip/prim/rtl/prim_assert.sv — localparam (1개)

상수	형/폭	값
__name	int	(__prop) ? 1 : 2

Table 679. lowRISC_opentitan/hw/ip/prim/rtl/prim_assert.sv — #define / `define 매크로 (16개)

매크로	인자	정의
PRIM_ASSERT_SV		////////////////////////////////
ASSERT_DEFAULT_CLK		clk_i
ASSERT_DEFAULT_RST		!rst_ni
PRIM_STRINGIFY	(__x)	`"__x"
ASSERT_ERROR	(__name)	\

매크로	인자	정의
ASSERT_STATIC_LINT_ERROR	(__name, __prop)	\
ASSERT_STATIC_IN_PACKAGE	(__name, __prop)	\
INC_ASSERT		`else
INC_ASSERT		`endif
ASSERT_PULSE	(__name, __sig, __clk = `ASSERT_DEFAULT_CLK, __rst = `ASSERT...	\
ASSERT_IF	(__name, __prop, __enable, __clk = `ASSERT_DEFAULT_CLK, __rs...	\
ASSERT_KNOWN_IF	(__name, __sig, __enable, __clk = `ASSERT_DEFAULT_CLK, __rst...	\
ASSUME_FPV	(__name, __prop, __clk = `ASSERT_DEFAULT_CLK, __rst = `ASSER...	\
ASSUME_I_FPV	(__name, __prop)	\
COVER_FPV	(__name, __prop, __clk = `ASSERT_DEFAULT_CLK, __rst = `ASSER...	\
ASSERT_FPV_LINEAR_FSM	(__name, __state, __type, __clk = `ASSERT_DEFAULT_CLK, __rst...	\

API — prim_blanker.sv

Table 680. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv — module / interface / package (1개)

이름	종류
prim_blanker	module

Table 681. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv — parameter (1개)

파라미터	형/폭	기본값
Width	int	1

Table 682. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv — 포트 (3개)

포트	방향	형/폭
in_i	input	logic [Width-1:0]
en_i	input	logic
out_o	output	logic [Width-1:0]

API — prim_cdc_rand_delay.sv

Table 683. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv — module / interface / package (1개)

이름	종류
prim_cdc_rand_delay	module

Table 684. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv — parameter (2개)

파라미터	형/폭	기본값
DataWidth	int	1
Enable	bit	1

Table 685. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
prev_data_i	input	logic [DataWidth-1:0]
src_data_i	input	logic [DataWidth-1:0]
dst_data_o	output	logic [DataWidth-1:0]

API — prim_cipher_pkg.sv

Table 686. lowRISC_opentitan/hw/ip/prim/rtl/prim_cipher_pkg.sv — module / interface / package (1개)

이름	종류
prim_cipher_pkg	package

Table 687. lowRISC_opentitan/hw/ip/prim/rtl/prim_cipher_pkg.sv — parameter (16개)

파라미터	형/폭	기본값
PRINCE_SBOX4	logic [15:0][3:0]	{4'h4
PRINCE_SBOX4_INV	logic [15:0][3:0]	{4'h1
PRINCE_SHIFT_ROWS64	logic [15:0][3:0]	{4'hF
PRINCE_SHIFT_ROWS64_INV	logic [15:0][3:0]	{4'hF
PRINCE_ROUND_CONST	logic [11:0][63:0]	{64'hC0AC29B7C97C50DD
PRINCE_ALPHA_CONST	logic [63:0]	64'hC0AC29B7C97C50DD
PRINCE_SHIFT_ROWS_CONST0	logic [15:0]	16'h7BDE
PRINCE_SHIFT_ROWS_CONST1	logic [15:0]	16'hBDE7
PRINCE_SHIFT_ROWS_CONST2	logic [15:0]	16'hDE7B
PRINCE_SHIFT_ROWS_CONST3	logic [15:0]	16'hE7BD
PRESENT_SBOX4	logic [15:0][3:0]	{4'h2
PRESENT_SBOX4_INV	logic [15:0][3:0]	{4'hA
PRESENT_PERM32	logic [31:0][4:0]	{5'd31
PRESENT_PERM32_INV	logic [31:0][4:0]	{5'd31
PRESENT_PERM64	logic [63:0][5:0]	{6'd63
PRESENT_PERM64_INV	logic [63:0][5:0]	{6'd63

API — prim_clock_gating_sync.sv

Table 688. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_gating_sync.sv — module / interface / package (1개)

이름	종류
prim_clock_gating_sync	module

Table 689. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_gating_sync.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
test_en_i	input	
async_en_i	input	
en_o	output	logic
clk_o	output	logic

API — prim_clock_meas.sv

Table 690. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — module / interface / package (1개)

이름	종류
prim_clock_meas	module

Table 691. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — parameter (4개)

파라미터	형/폭	기본값
Cnt	int	16
RefCnt	int	1
ClkTimeOutChkEn	bit	1
RefTimeOutChkEn	bit	1

Table 692. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — localparam (6개)

상수	형/폭	값
CntWidth	int	prim_util_pkg::vbits(Cnt)
RefCntWidth	int	prim_util_pkg::vbits(RefCnt)
TimeOutChkEn	bit	ClkTimeOutChkEn RefTimeOutChkEn
ClkRatio	int	Cnt / RefCnt
MaxClkCdcLatency	int	$(1 + 2 * \text{ClkRatio} + 1 * \text{ClkRatio} + 2) * 2$
MaxRefCdcLatency	int	$1 + 1 + 2 + 2$

Table 693. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv — 포트 (12개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clk_ref_i	input	
rst_ref_ni	input	
en_i	input	
max_cnt	input	[CntWidth-1:0]
min_cnt	input	[CntWidth-1:0]

포트	방향	형/폭
valid_o	output	logic
fast_o	output	logic
slow_o	output	logic
timeout_clk_ref_o	output	logic
ref_timeout_clk_o	output	logic

API — prim_clock_timeout.sv

Table 694. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — module / interface / package (1개)

이름	종류
prim_clock_timeout	module

Table 695. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — parameter (1개)

파라미터	형/폭	기본값
TimeOutCnt	int	16

Table 696. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — localparam (1개)

상수	형/폭	값
CntWidth	int	prim_util_pkg::vbits(TimeOutCnt+1)

Table 697. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv — 포트 (6개)

포트	방향	형/폭
clk_chk_i	input	
rst_chk_ni	input	
clk_i	input	
rst_ni	input	
en_i	input	
timeout_o	output	logic

API — prim_count.sv

Table 698. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — module / interface / package (1개)

이름	종류
prim_count	module

Table 699. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — parameter (4개)

파라미터	형/폭	기본값
Width	int	2
ResetValue	logic [Width-1:0]	'0
EnableAlertTriggerSVA	bit	1

파라미터	형/폭	기본값
PossibleActions	action_mask_t	{ \$bits(action_mask_t){1'b1} }

Table 700. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — localparam (2개)

상수	형/폭	값
NumCnt	int	2
ResetValues	logic [NumCnt-1:0][Width-1:0]	{{Width{1'b1}} - ResetValue

Table 701. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv — 포트 (12개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
clr_i	input	
set_i	input	
set_cnt_i	input	[Width-1:0]
incr_en_i	input	
decr_en_i	input	
step_i	input	[Width-1:0]
commit_i	input	
cnt_o	output	logic [Width-1:0]
cnt_after_commit_o	output	logic [Width-1:0]
err_o	output	logic

API — prim_count_pkg.sv**Table 702. lowRISC_opentitan/hw/ip/prim/rtl/prim_count_pkg.sv — module / interface / package (1개)**

이름	종류
prim_count_pkg	package

Table 703. lowRISC_opentitan/hw/ip/prim/rtl/prim_count_pkg.sv — typedef / using (1개)

새 이름	원래 형
action_mask_t	logic [3:0]

API — prim_crc32.sv**Table 704. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv — module / interface / package (1개)**

이름	종류
prim_crc32	module

Table 705. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv — parameter (1개)

파라미터	형/폭	기본값
BytesPerWord	int unsigned	4

Table 706. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv — 포트 (7개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
set_crc_i	input	logic
crc_in_i	input	logic [31:0]
data_valid_i	input	logic
data_i	input	logic [BytesPerWord*8-1:0]
crc_out_o	output	logic [31:0]

API — prim_diff_decode.sv**Table 707. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — module / interface / package (1개)**

이름	종류
prim_diff_decode	module

Table 708. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — parameter (2개)

파라미터	형/폭	기본값
AsyncOn	bit	1'b0
SkewCycles	int unsigned	1

Table 709. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
diff_pi	input	
diff_ni	input	
level_o	output	logic
rise_o	output	logic
fall_o	output	logic
event_o	output	logic
sigint_o	output	logic

Table 710. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [1:0] {IsStd, IsSkewing, SigInt}

API — prim_diff_encode.sv**Table 711. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_encode.sv — module / interface / package (1개)**

이름	종류
prim_diff_encode	module

Table 712. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_encode.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
req_i	input	logic
diff_po	output	logic
diff_no	output	logic

API — prim_diff_to_alert.sv**Table 713. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv — module / interface / package (1개)**

이름	종류
prim_diff_to_alert	module

Table 714. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv — parameter (3개)

파라미터	형/폭	기본값
Async0n	bit	1'b1
SkewCycles	int unsigned	1
IsFatal	bit	1'b1

Table 715. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
diff_pi	input	logic
diff_ni	input	logic
alert_rx_i	input	prim_alert_pkg::alert_rx_t
alert_tx_o	output	prim_alert_pkg::alert_tx_t

API — prim_dom_and_2share.sv**Table 716. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv — module / interface / package (1개)**

이름	종류
prim_dom_and_2share	module

Table 717. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv — parameter (2개)

파라미터	형/폭	기본값
DW	int	64
Pipeline	bit	1'b0 // Enable full pipelining

Table 718. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv — 포트 (11개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
a0_i	input	[DW-1:0]
a1_i	input	[DW-1:0]
b0_i	input	[DW-1:0]
b1_i	input	[DW-1:0]
z_valid_i	input	
z_i	input	[DW-1:0]
q0_o	output	logic [DW-1:0]
q1_o	output	logic [DW-1:0]
instances	output	logic [DW-1:0] prd_o // pseudo-random data for oth...

API — prim_double_lfsr.sv**Table 719. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — module / interface / package (1개)**

이름	종류
prim_double_lfsr	module

Table 720. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — parameter (13개)

파라미터	형/폭	기본값
LfsrType		"GAL_XOR"
LfsrDw	int unsigned	32
EntropyDw	int unsigned	8
StateOutDw	int unsigned	8
DefaultSeed	logic [LfsrDw-1:0]	LfsrDw'(1)
CustomCoeffs	logic [LfsrDw-1:0]	'0
StatePermEn	bit	1'b0
StatePerm	logic [LfsrDw-1:0][LfsrIdxDw-1:0]	'0
MaxLenSVA	bit	1'b1
LockupSVA	bit	1'b1
ExtSeedSVA	bit	1'b1
NonLinearOut	bit	1'b0

파라미터	형/폭	기본값
EnableAlertTriggerSVA	bit	1

Table 721. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — localparam (1개)

상수	형/폭	값
LfsrIdxDw	int unsigned	\$clog2(LfsrDw)

Table 722. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
seed_en_i	input	
seed_i	input	[LfsrDw-1:0]
lfsr_en_i	input	
entropy_i	input	[EntropyDw-1:0]
state_o	output	logic [StateOutDw-1:0]
err_o	output	logic

API — prim_edge_detector.sv**Table 723. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv — module / interface / package (1개)**

이름	종류
prim_edge_detector	module

Table 724. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv — parameter (3개)

파라미터	형/폭	기본값
Width	int unsigned	1
ResetValue	logic [Width-1:0]	'0
EnSync	bit	1'b 1

Table 725. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	
rst_ni	input	
d_i	input	[Width-1:0]
q_sync_o	output	logic [Width-1:0]
q_posedge_pulse_o	output	logic [Width-1:0]
q_negedge_pulse_o	output	logic [Width-1:0]

1. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_pkg.sv (27 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 package prim_alert_pkg;
6
7   typedef struct packed {
8     logic alert_p;
9     logic alert_n;
10  } alert_tx_t;
11
12   typedef struct packed {
13     logic ping_p;
14     logic ping_n;
15     logic ack_p;
16     logic ack_n;
17  } alert_rx_t;
18
19   parameter alert_tx_t ALERT_TX_DEFAULT = '{alert_p: 1'b0,
20                                           alert_n: 1'b1};
21
22   parameter alert_rx_t ALERT_RX_DEFAULT = '{ping_p: 1'b0,
23                                           ping_n: 1'b1,
24                                           ack_p: 1'b0,
25                                           ack_n: 1'b1};
26
27 endpackage : prim_alert_pkg

```

2. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_receiver.sv (391 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // The alert receiver primitive decodes alerts that have been
  differentially
6 // encoded and transmitted via a handshake protocol on
  alert_p/n and
7 // ack_p/n. In case an alert handshake is initiated, the out-
  put alert_o will
8 // immediately be asserted (even before completion of the
  handshake).
9 //
10 // In case the differential input is not correctly encoded,
  this module will
11 // raise an error by asserting integ_fail_o.
12 //
13 // Further, the module supports ping testing of the alert diff
  pair. In order to
14 // initiate a ping test, ping_req_i shall be set to 1'b1 until
  ping_ok_o is
15 // asserted for one cycle. The signal may be de-asserted (e.g.
  after a long)
16 // timeout period. However note that all ping responses that
  come in after
17 // deasserting ping_req_i will be treated as native alerts.
18 //
19 // The protocol works in both asynchronous and synchronous
  cases. In the
20 // asynchronous case, the parameter AsyncOn must be set to
  1'b1 in order to
21 // instantiate additional synchronization logic. Further, it
  must be ensured
22 // that the timing skew between all diff pairs is smaller than
  the shortest
23 // clock period of the involved clocks.
24 //
25 // Note that in case of synchronous operation, alerts on the
  diffpair are
26 // decoded combinatorially and forwarded on alert_o within the
  same cycle.
27 //
28 // See also: prim_alert_sender, prim_diff_decode,
  alert_handler
29
30 `include "prim_assert.sv"
31
32 module prim_alert_receiver
33   import prim_alert_pkg::*;
34   import prim_mubi_pkg::mubi4_t;
35 #()
36   // enables additional synchronization logic
37   parameter bit AsyncOn = 1'b0,
38   // Number of cycles a differential skew is tolerated on the
  differential alert signal.

```

```

39   parameter int unsigned SkewCycles = 1
40 ) (
41   input          clk_i,
42   input          rst_ni,
43   // if set to lc_ctrl_pkg::0n, this triggers the in-band
  alert channel
44   // reset, which resets both the sender and receiver FSMs
  into IDLE.
45   input mubi4_t   init_trig_i,
46   // this triggers a ping test. keep asserted
47   // until ping_ok_o is asserted.
48   input          ping_req_i,
49   output logic    ping_ok_o,
50   // asserted if signal integrity issue detected
51   output logic    integ_fail_o,
52   // alert output (pulsed high) if a handshake is initiated
53   // on alert_p/n and no ping request is outstanding
54   output logic    alert_o,
55   // ping input diff pair and ack diff pair
56   output alert_rx_t alert_rx_o,
57   // alert output diff pair
58   input alert_tx_t alert_tx_i
59 );
60
61 import prim_mubi_pkg::mubi4_test_true_strict;
62
63 ///////////////////////////////////////////////////////////////////
64 // decode differential signals //
65 ///////////////////////////////////////////////////////////////////
66 logic alert_level, alert_sigint, alert_p, alert_n;
67
68 // This prevents further tool optimizations of the differen-
  tial signal.
69 prim_sec_anchor_buf #(
70   .Width(2)
71 ) u_prim_buf_in (
72   .in_i({alert_tx_i.alert_n,
73         alert_tx_i.alert_p}),
74   .out_o({alert_n,
75         alert_p})
76 );
77
78 prim_diff_decode #(
79   .AsyncOn(AsyncOn),
80   .SkewCycles(SkewCycles)
81 ) u_decode_alert (
82   .clk_i,
83   .rst_ni,
84   .diff_pi ( alert_p ),
85   .diff_ni ( alert_n ),
86   .level_o ( alert_level ),
87   .rise_o ( ),
88   .fall_o ( ),
89   .event_o ( ),
90   .sigint_o ( alert_sigint )
91 );
92
93 ///////////////////////////////////////////////////////////////////
94 // main protocol FSM that drives the diff outputs //
95 ///////////////////////////////////////////////////////////////////
96 typedef enum logic [2:0] {Idle, HsAckWait, Pause0, Pause1,
  InitReq, InitAckWait} state_e;
97 state_e state_d, state_q;
98 logic ping_rise;
99 logic ping_tog_pd, ping_tog_pq, ping_tog_dn, ping_tog_nq;
100 logic ack_pd, ack_pq, ack_dn, ack_nq;
101 logic ping_req_d, ping_req_q;
102 logic ping_pending_d, ping_pending_q;
103 logic send_init;
104 logic send_ping;
105
106 // signal ping request upon positive transition on
  ping_req_i
107 // signalling is performed by a level change event on the
  diff output
108 assign ping_req_d = ping_req_i;
109 assign ping_rise = ping_req_d && !ping_req_q;
110 assign ping_tog_pd = (send_init) ? 1'b0 :
111                    (send_ping) ? ~ping_tog_pq :
112
113 // in-band reset is performed by sending out an integrity
  error on purpose.
114 assign ack_dn = (send_init) ? ack_pd : ~ack_pd;
115 assign ping_tog_dn = ~ping_tog_pd;
116
117 // This prevents further tool optimizations of the differen-
  tial signal.
118 prim_sec_anchor_flop #(
119   .Width (2),
120   .ResetValue(2'b10)
121 ) u_prim_generic_flop_ack (
122   .clk_i,
123   .rst_ni,
124   .d_i({ack_dn,
125         ack_pd}),
126   .q_o({ack_nq,
127         ack_pq})
128 );
129
130 prim_sec_anchor_flop #(

```

```

131     .Width      (2),
132     .ResetValue(2'b10)
133 ) u_prim_generic_flop_ping (
134     .clk_i,
135     .rst_ni,
136     .d_i({ping_tog_dn,
137           ping_tog_pd}),
138     .q_o({ping_tog_nq,
139           ping_tog_pq})
140 );
141
142 // the ping pending signal is used in the FSM to distinguish
whether the
143 // incoming handshake shall be treated as an alert or a ping
response.
144 // it is important that this is only set on a rising ping_en
level change, since
145 // otherwise the ping enable signal could be abused to
"mask" all native alerts
146 // as ping responses by constantly tying it to 1.
147 assign ping_pending_d = ping_rise | ((~ping_ok_o) &
ping_req_i & ping_pending_q);
148
149 // diff pair outputs
150 assign alert_rx_o.ack_p = ack_pq;
151 assign alert_rx_o.ack_n = ack_nq;
152
153 assign alert_rx_o.ping_p = ping_tog_pq;
154 assign alert_rx_o.ping_n = ping_tog_nq;
155
156 // this FSM receives the four phase handshakes from the
alert receiver
157 // note that the latency of the alert_p/n input diff pair is
at least one
158 // cycle until it enters the receiver FSM. the same holds
for the ack_* diff
159 // pair outputs.
160 always_comb begin : p_fsm
161     // default
162     state_d = state_q;
163     ack_pd = 1'b0;
164     ping_ok_o = 1'b0;
165     integ_fail_o = 1'b0;
166     alert_o = 1'b0;
167     send_init = 1'b0;
168     // by default, a ping request leads to a toggle on the
differential ping pair
169     send_ping = ping_rise;
170
171     unique case (state_q)
172     Idle: begin
173         // wait for handshake to be initiated
174         if (alert_level) begin
175             state_d = HSAckWait;
176             ack_pd = 1'b1;
177             // signal either an alert or ping received on the
output
178             if (ping_pending_q) begin
179                 ping_ok_o = 1'b1;
180             end else begin
181                 alert_o = 1'b1;
182             end
183         end
184     end
185     // waiting for deassertion of alert to complete HS
186     HSAckWait: begin
187         if (!alert_level) begin
188             state_d = Pause0;
189         end else begin
190             ack_pd = 1'b1;
191         end
192     end
193     // pause cycles between back-to-back handshakes
194     Pause0: state_d = Pause1;
195     Pause1: state_d = Idle;
196     // this state is only reached if an in-band reset is
197     // requested via the low-power logic.
198     InitReq: begin
199         // we deliberately place a sigint error on the ack and
ping lines in this case.
200         send_init = 1'b1;
201         // suppress any toggles on the ping line while we are
in the init phase.
202         send_ping = 1'b0;
203         // As long as init req is asserted, we remain in this
state and acknowledge all incoming
204         // ping requests. As soon as the init request is
dropped however, ping requests are not
205         // acked anymore such that the ping mechanism can also
flag alert channels that got stuck
206         // in the initialization sequence.
207         if (mubi4_test_true_strict(init_trig_i)) begin
208             ping_ok_o = ping_pending_q;
209             // the sender will respond to the sigint error above
with a sigint error on the alert lines.

```

```

210         // hence we treat the alert_sigint like an acknow-
ledge in this case.
211     end else if (alert_sigint) begin
212         state_d = InitAckWait;
213     end
214 end
215 // We get here if the sender has responded with
alert_sigint, and init_trig_i==lc_ctrl_pkg::On
216 // has been deasserted. At this point, we need to wait
for the alert_sigint to drop again
217 // before resuming normal operation.
218 InitAckWait: begin
219     // suppress any toggles on the ping line while we are
in the init phase.
220     send_ping = 1'b0;
221     if (!alert_sigint) begin
222         state_d = Pause0;
223         // If we get a ping request in this cycle, or if we
realize that there is an unhandled
224         // ping request that came in during initialization
(but after init_trig_i has been
225         // deasserted), we signal this to the alert sender
by toggling the request line.
226         send_ping = ping_rise || ping_pending_q;
227     end
228 end
229 default: state_d = Idle;
230 endcase
231
232 // once the initialization sequence has been triggered,
233 // overrides are not allowed anymore until the initializa-
tion has been completed.
234 if (!(state_q inside {InitReq, InitAckWait})) begin
235     // in this case, abort and jump into the initialization
sequence
236     if (mubi4_test_true_strict(init_trig_i)) begin
237         state_d = InitReq;
238         ack_pd = 1'b0;
239         ping_ok_o = 1'b0;
240         integ_fail_o = 1'b0;
241         alert_o = 1'b0;
242         send_init = 1'b1;
243     end
244 // if we're not busy with an init request, we clamp down
all outputs
245 // and indicate an integrity failure.
246 end else if (alert_sigint) begin
247     state_d = Idle;
248     ack_pd = 1'b0;
249     ping_ok_o = 1'b0;
250     integ_fail_o = 1'b1;
251     alert_o = 1'b0;
252 end
253 end
254
255 always_ff @(posedge clk_i or negedge rst_ni) begin : p_reg
256     if (!rst_ni) begin
257         // Reset into the init request so that an alert handler
reset implicitly
258         // triggers an in-band reset of all alert channels.
259         state_q <= InitReq;
260         ping_req_q <= 1'b0;
261         ping_pending_q <= 1'b0;
262     end else begin
263         state_q <= state_d;
264         ping_req_q <= ping_req_d;
265         ping_pending_q <= ping_pending_d;
266     end
267 end
268
269 // assertions //
270 ///////////////////////////////////////////////////////////////////
271 // assertions //
272 ///////////////////////////////////////////////////////////////////
273
274 `ifdef INC_ASSERT
275 import prim_mubi_pkg:mubi4_test_false_loose;
276 `endif
277
278 // check whether all outputs have a good known state after
reset
279 `ASSERT_KNOWN(PingOkKnown0_A, ping_ok_o)
280 `ASSERT_KNOWN(IntegFailKnown0_A, integ_fail_o)
281 `ASSERT_KNOWN(AlertKnown0_A, alert_o)
282 `ASSERT_KNOWN(PingPKnown0_A, alert_rx_o)
283
284 // check encoding of outgoing diffpairs. note that during
init, the outgoing diffpairs are
285 // supposed to be incorrectly encoded on purpose.
286 // shift sequence two cycles to the right to avoid reset ef-
fects.
287 `ASSERT(PingDiffOk_A, alert_rx_o.ping_p ^ alert_rx_o.ping_n)
288 `ASSERT(AckDiffOk_A, ##2 $past(send_init) ^ alert_rx_o.ack_p ^
alert_rx_o.ack_n)
289 `ASSERT(InitReq_A, mubi4_test_true_strict(init_trig_i) &&
290         !(state_q inside {InitReq, InitAckWait})) | =>
send_init)
291
292 // If there is a ping request on the input then we should

```

```

see an encoded ping request. This is
293 // squashed if we are in state InitReq or InitAckWait (be-
cause we are still initialising), or if
294 // we see the init_trig_i signal go high (because it will
start an initialisation).
295 `ASSERT(PingRequest0_A,
296     $rose(ping_req_i) |> $changed(alert_rx_o.ping_p),
297     clk_i, !rst_ni || init_trig_i || state_q inside
{InitReq, InitAckWait})
298
299 // ping response implies it has been requested
300 `ASSERT(PingResponse0_A, ping_ok_o |> ping_pending_q)
301 // correctly latch ping request
302 `ASSERT(PingPending_A, ##1 $rose(ping_req_i) |> ping_pend-
ing_q)
303
304 if (AsyncOn) begin : gen_async_assert
305 // signal integrity check propagation
306 `ASSERT(SigInt_A,
307     alert_tx_i.alert_p == alert_tx_i.alert_n [*2] ##2
308     !(state_q inside {InitReq, InitAckWait}) &&
309     mubi4_test_false_loose(init_trig_i)
310     |>
311     ##[0:SkewCycles] integ_fail_o)
312 `ASSERT(PingResponse1_A,
313     ##1 $rose(alert_tx_i.alert_p) &&
314     (alert_tx_i.alert_p ^ alert_tx_i.alert_n) ##2
315     state_q == Idle && ping_pending_q
316     |>
317     ##[0:SkewCycles] ping_ok_o,
318     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
319 // alert
320 `ASSERT(Alert_A,
321     ##1 $rose(alert_tx_i.alert_p) &&
322     (alert_tx_i.alert_p ^ alert_tx_i.alert_n) ##2
323     state_q == Idle &&
324     !ping_pending_q
325     |>
326     ##[0:SkewCycles] alert_o,
327     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
328 end else begin : gen_sync_assert
329 // signal integrity check propagation
330 `ASSERT(SigInt_A,
331     alert_tx_i.alert_p == alert_tx_i.alert_n &&
332     !(state_q inside {InitReq, InitAckWait}) &&
333     mubi4_test_false_loose(init_trig_i)
334     |>
335     integ_fail_o)
336 // ping response
337 `ASSERT(PingResponse1_A,
338     ##1 $rose(alert_tx_i.alert_p) &&
339     state_q == Idle &&
340     ping_pending_q
341     |>
342     ping_ok_o,
343     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
344 // alert
345 `ASSERT(Alert_A,
346     ##1 $rose(alert_tx_i.alert_p) &&
347     state_q == Idle &&
348     !ping_pending_q
349     |>
350     alert_o,
351     clk_i, !rst_ni || integ_fail_o || mu-
bi4_test_true_strict(init_trig_i))
352 end
353
354 // check in-band init request is always accepted
355 `ASSERT(InBandInitRequest_A,
356     mubi4_test_true_strict(init_trig_i) &&
357     state_q != InitAckWait
358     |>
359     state_q == InitReq)
360 // check in-band init sequence moves FSM into IDLE state
361 `ASSERT(InBandInitSequence_A,
362     (state_q == InitReq &&
363     mubi4_test_true_strict(init_trig_i)) ##1
364     (alert_sigint &&
365     mubi4_test_false_loose(init_trig_i)) [*1:$] ##1
366     (!alert_sigint &&
367     mubi4_test_false_loose(init_trig_i)) [*3]
368     |>
369     state_q == Idle)
370 // check there are no spurious alerts during init
371 `ASSERT(NoSpuriousAlertsDuringInit_A,
372     mubi4_test_true_strict(init_trig_i) ||
373     (state_q inside {InitReq, InitAckWait})
374     |>
375     !alert_o)
376 // check that there are no spurious ping OKs
377 `ASSERT(NoSpuriousPingOKsDuringInit_A,
378     (mubi4_test_true_strict(init_trig_i) ||
379     (state_q inside {InitReq, InitAckWait})) &&
380     !ping_pending_q
381     |>
382     !ping_ok_o)
383 // check ping request is bypassed when in init state
384 `ASSERT(PingOkBypassDuringInit_A,

```

```

385     $rose(ping_req_i) ##1
386     state_q == InitReq &&
387     mubi4_test_true_strict(init_trig_i)
388     |>
389     ping_ok_o)
390
391 endmodule : prim_alert_receiver

```

3. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_sender.sv (398 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // The alert sender primitive module differentially encodes
and transmits an
6 // alert signal to the prim_alert_receiver module. An alert
will be signalled
7 // by a full handshake on alert_p/n and ack_p/n. The
alert_req_i signal may
8 // be continuously asserted, in which case the alert sig-
nalling handshake
9 // will be repeatedly initiated.
10 //
11 // The alert_req_i signal may also be used as part of req/ack.
The parent module
12 // can keep alert_req_i asserted until it has been ack'd
(transferred to the alert
13 // receiver). The parent module is not required to use this.
14 //
15 // In case the alert sender parameter IsFatal is set to 1, an
incoming alert
16 // alert_req_i is latched in a local register until the next
reset, causing the
17 // alert sender to behave as if alert_req_i were continuously
asserted.
18 // The alert_state_o output reflects the state of this intern-
al latching register.
19 //
20 // The alert sender also exposes an alert test input, which
can be used to trigger
21 // single alert handshakes. This input behaves exactly the
same way as the
22 // alert_req_i input with IsFatal set to 0. Test alerts do not
cause alert_ack_o
23 // to be asserted, nor are they latched until reset (regard-
less of the value of the
24 // IsFatal parameter).
25 //
26 // Further, this module supports in-band ping testing, which
means that a level
27 // change on the ping_p/n diff pair will result in a full-
handshake response
28 // on alert_p/n and ack_p/n.
29 //
30 // The protocol works in both asynchronous and synchronous
cases. In the
31 // asynchronous case, the parameter AsyncOn must be set to
1'b1 in order to
32 // instantiate additional synchronization logic. Further, it
must be ensured
33 // that the timing skew between all diff pairs is smaller than
the shortest
34 // clock period of the involved clocks.
35 //
36 // Incorrectly encoded diff inputs can be detected and will be
signalled
37 // to the receiver by placing an inconsistent diff value on
the differential
38 // output (and continuously toggling it).
39 //
40 // See also: prim_alert_receiver, prim_diff_decode,
alert_handler
41
42 `include "prim_assert.sv"
43
44 module prim_alert_sender
45     import prim_alert_pkg::*;
46 (
47     // enables additional synchronization logic
48     parameter bit AsyncOn = 1'b1,
49     // Number of cycles a differential skew is tolerated on the
differential ack/ping signal.
50     parameter int unsigned SkewCycles = 1,
51     // alert sender will latch the incoming alert event perman-
ently and
52     // keep on sending alert events until the next reset.
53     parameter bit IsFatal = 1'b0
54 ) (
55     input          clk_i,
56     input          rst_ni,
57     // alert test trigger (this will never be latched, even if
IsFatal == 1)

```



```

58 input          alert_test_i,
59 // native alert from the peripheral
60 input          alert_req_i,
61 output logic    alert_ack_o,
62 // state of the alert latching register
63 output logic    alert_state_o,
64 // ping input diff pair and ack diff pair
65 input alert_rx_t alert_rx_i,
66 // alert output diff pair
67 output alert_tx_t alert_tx_o
68 );
69
70
71 ///////////////////////////////////////////////////
72 // decode differential signals //
73 ///////////////////////////////////////////////////
74 logic ping_sigint, ping_event, ping_n, ping_p;
75
76 // This prevents further tool optimizations of the differen-
77 tial signal.
78 prim_sec_anchor_buf #(
79   .Width(2)
80 ) u_prim_buf_ping (
81   .in_i({alert_rx_i.ping_n,
82         alert_rx_i.ping_p}),
83   .out_o({ping_n,
84           ping_p})
85 );
86
87 prim_diff_decode #(
88   .AsyncOn(AsyncOn),
89   .SkewCycles(SkewCycles)
90 ) u_decode_ping (
91   .clk_i,
92   .rst_ni,
93   .diff_pi ( ping_p ),
94   .diff_ni ( ping_n ),
95   .level_o (      ),
96   .rise_o  (      ),
97   .fall_o  (      ),
98   .event_o ( ping_event ),
99   .sigint_o ( ping_sigint )
100 );
101
102 logic ack_sigint, ack_level, ack_n, ack_p;
103
104 // This prevents further tool optimizations of the differen-
105 tial signal.
106 prim_sec_anchor_buf #(
107   .Width(2)
108 ) u_prim_buf_ack (
109   .in_i({alert_rx_i.ack_n,
110         alert_rx_i.ack_p}),
111   .out_o({ack_n,
112           ack_p})
113 );
114
115 prim_diff_decode #(
116   .AsyncOn(AsyncOn),
117   .SkewCycles(SkewCycles)
118 ) u_decode_ack (
119   .clk_i,
120   .rst_ni,
121   .diff_pi ( ack_p ),
122   .diff_ni ( ack_n ),
123   .level_o ( ack_level ),
124   .rise_o  (      ),
125   .fall_o  (      ),
126   .event_o (      ),
127   .sigint_o ( ack_sigint )
128 );
129
130 ///////////////////////////////////////////////////
131 // main protocol FSM that drives the diff output //
132 ///////////////////////////////////////////////////
133 typedef enum logic [2:0] {
134   Idle,
135   AlertHsPhase1,
136   AlertHsPhase2,
137   PingHsPhase1,
138   PingHsPhase2,
139   Pause0,
140   Pause1
141 } state_e;
142 state_e state_d, state_q;
143 logic alert_pq, alert_nq, alert_pd, alert_nd;
144 logic sigint_detected;
145
146 assign sigint_detected = ack_sigint | ping_sigint;
147
148 // diff pair output
149 assign alert_tx_o.alert_p = alert_pq;
150 assign alert_tx_o.alert_n = alert_nq;
151
152 // alert and ping set regs
153 logic alert_set_d, alert_set_q, alert_clr;
154 logic alert_test_set_d, alert_test_set_q;
155 logic ping_set_d, ping_set_q, ping_clr;
156 logic alert_req_trigger, alert_test_trigger, ping_trigger;
157
158 // if handshake is ongoing, capture additional alert re-
159 quests.
160 logic alert_req;
161 prim_sec_anchor_buf #(
162   .Width(1)
163 ) u_prim_buf_in_req (
164   .in_i(alert_req_i),
165   .out_o(alert_req)
166 );
167
168 assign alert_req_trigger = alert_req | alert_set_q;
169 if (IsFatal) begin : gen_fatal
170   assign alert_set_d = alert_req_trigger;
171 end else begin : gen_recov
172   assign alert_set_d = (alert_clr) ? 1'b0 :
173   alert_req_trigger;
174 end
175
176 // the alert test request is always cleared.
177 assign alert_test_trigger = alert_test_i | alert_test_set_q;
178 assign alert_test_set_d = (alert_clr) ? 1'b0 :
179 alert_test_trigger;
180
181 logic alert_trigger;
182 assign alert_trigger = alert_req_trigger |
183 alert_test_trigger;
184
185 assign ping_trigger = ping_set_q | ping_event;
186 assign ping_set_d = (ping_clr) ? 1'b0 : ping_trigger;
187
188 // alert event acknowledge and state (not affected by
189 alert_test_i)
190 assign alert_ack_o = alert_clr & alert_set_q;
191 assign alert_state_o = alert_set_q;
192
193 // this FSM performs a full four phase handshake upon a ping
194 or alert trigger.
195 // note that the latency of the alert_p/n diff pair is at
196 least one cycle
197 // until it enters the receiver FSM. the same holds for the
198 ack_* diff pair
199 // input. in case a signal integrity issue is detected, the
200 FSM bails out,
201 // sets the alert_p/n diff pair to the same value and
202 toggles it in order to
203 // signal that condition over to the receiver.
204 always_comb begin : p_fsm
205   // default
206   state_d = state_q;
207   alert_pd = 1'b0;
208   alert_nd = 1'b1;
209   ping_clr = 1'b0;
210   alert_clr = 1'b0;
211
212   unique case (state_q)
213     Idle: begin
214       // alert always takes precedence
215       if (alert_trigger || ping_trigger) begin
216         state_d = (alert_trigger) ? AlertHsPhase1 : PingHs-
217 Phase1;
218       end
219       alert_pd = 1'b1;
220       alert_nd = 1'b0;
221     end
222     // waiting for ack from receiver
223     AlertHsPhase1: begin
224       if (ack_level) begin
225         state_d = AlertHsPhase2;
226       end else begin
227         alert_pd = 1'b1;
228         alert_nd = 1'b0;
229       end
230     end
231     // wait for deassertion of ack
232     AlertHsPhase2: begin
233       if (!ack_level) begin
234         state_d = Pause0;
235         alert_clr = 1'b1;
236       end
237     end
238     // waiting for ack from receiver
239     PingHsPhase1: begin
240       if (ack_level) begin
241         state_d = PingHsPhase2;
242       end else begin
243         alert_pd = 1'b1;
244         alert_nd = 1'b0;
245       end
246     end
247     // wait for deassertion of ack
248     PingHsPhase2: begin
249       if (!ack_level) begin
250         ping_clr = 1'b1;
251         state_d = Pause0;
252       end
253     end
254     Pause0: begin
255       // pause cycles between back-to-back handshakes
256       Pause0: begin

```

```

246     state_d = Pause1;
247 end
248 // clear and ack alert request if it was set
249 Pause1: begin
250     state_d = Idle;
251 end
252 // catch parasitic states
253 default : state_d = Idle;
254 endcase
255
256 // we have a signal integrity issue at one of the incoming
257 // diff pairs. this condition is
258 // signalled by setting the output diffpair to zero. If
259 // the sigint has disappeared, we clear
260 // the ping request state of this sender and go back to
261 // idle.
262 if (sigint_detected) begin
263     state_d = Idle;
264     alert_pd = 1'b0;
265     alert_nd = 1'b0;
266     ping_clr = 1'b1;
267     alert_clr = 1'b0;
268 end
269 end
270 // This prevents further tool optimizations of the differen-
271 // tial signal.
272 prim_sec_anchor_flop #(
273     .Width (2),
274     .ResetValue(2'b10)
275 ) u_prim_flop_alert (
276     .clk_i,
277     .rst_ni,
278     .d_i({alert_nd, alert_pd}),
279     .q_o({alert_nq, alert_pq})
280 );
281
282 always_ff @(posedge clk_i or negedge rst_ni) begin : p_reg
283     if (!rst_ni) begin
284         state_q <= Idle;
285         alert_set_q <= 1'b0;
286         alert_test_set_q <= 1'b0;
287         ping_set_q <= 1'b0;
288     end else begin
289         state_q <= state_d;
290         alert_set_q <= alert_set_d;
291         alert_test_set_q <= alert_test_set_d;
292         ping_set_q <= ping_set_d;
293     end
294 end
295
296 // assertions
297
298 // however, since we use sequence constructs below, we need to
299 // wrap the entire block again.
300 // typically, the ASSERT macros already contain this
301 // INC_ASSERT macro.
302 `ifdef INC_ASSERT
303 // check whether all outputs have a good known state after
304 // reset
305 `ASSERT_KNOWN(AlertPKnown0_A, alert_tx_o)
306
307 if (Async0n) begin : gen_async_assert
308     sequence PingSigInt_S;
309     alert_rx_i.ping_p == alert_rx_i.ping_n [*2];
310 endsequence
311 sequence AckSigInt_S;
312 alert_rx_i.ping_p == alert_rx_i.ping_n [*2];
313 endsequence
314
315 `ifndef FPV_ALERT_NO_SIGINT_ERR
316 // check propagation of sigint issues to output within
317 // three cycles, or four due to CDC
318 // shift sequence to the right to avoid reset effects.
319 `ASSERT(SigIntPing_A, ##1 PingSigInt_S |>
320     ##[SkewCycles+2:SkewCycles+3] alert_tx_o.alert_p ==
321     alert_tx_o.alert_n)
322 `ASSERT(SigIntAck_A, ##1 AckSigInt_S |>
323     ##[SkewCycles+2:SkewCycles+3] alert_tx_o.alert_p ==
324     alert_tx_o.alert_n)
325 `endif
326
327 // Test in-band FSM reset request (via signal integrity
328 // error)
329 `ASSERT(InBandInitFsm_A, PingSigInt_S or AckSigInt_S |>
330     state_q == Idle)
331 `ASSERT(InBandInitPing_A, PingSigInt_S or AckSigInt_S |
332     => !ping_set_q)
333 // output must be driven diff unless sigint issue detected
334 `ASSERT(DiffEncoding_A, (alert_rx_i.ack_p ^
335     alert_rx_i.ack_n) &&
336     (alert_rx_i.ping_p ^ alert_rx_i.ping_n) |>
337     alert_tx_o.alert_p ^ alert_tx_o.alert_n)
338 // handshakes can take indefinite time if blocked due to
339 // sigint on outgoing
340 // lines (which is not visible here). thus, we only check
341 // whether the handshake
342 // is correctly initiated and defer the full handshake
343 // checking to the testbench.
344 `ASSERT(PingHs_A, ##1 $changed(alert_rx_i.ping_p) &&
345     state_q == Idle |>
346     $rose(alert_tx_o.alert_p), clk_i, !rst_ni ||
347     (alert_tx_o.alert_p == alert_tx_o.alert_n))
348 end
349 // Test the alert state output.
350 `ASSERT(AlertState0_A, alert_set_q == alert_state_o)
351
352 if (IsFatal) begin : gen_fatal_assert
353     `ASSERT(AlertState1_A, alert_req_i |> alert_state_o)
354     `ASSERT(AlertState2_A, alert_state_o |>
355         $stable(alert_state_o))
356     `ASSERT(AlertState3_A, alert_ack_o |> alert_state_o)
357 end else begin : gen_recov_assert
358     `ASSERT(AlertState1_A, alert_req_i && !alert_clr |>
359         alert_state_o)
360     `ASSERT(AlertState2_A, alert_req_i && alert_ack_o |> !
361         alert_state_o)
362 end
363
364 // The alert test input should not set the alert state re-
365 // gister.
366 `ASSERT(AlertTest1_A, alert_test_i && !alert_req_i && !
367     alert_state_o |> $stable(alert_state_o))
368
369 // if alert_req_i is true, handshakes should be continuously
370 // repeated
371 `ASSERT(AlertHs_A, alert_req_i && state_q == Idle |>
372     $rose(alert_tx_o.alert_p),
373     clk_i, !rst_ni || (alert_tx_o.alert_p ==
374     alert_tx_o.alert_n))
375
376 // if alert_test_i is true, handshakes should be continu-
377 // ously repeated
378 `ASSERT(AlertTestHs_A, alert_test_i && state_q == Idle |>
379     $rose(alert_tx_o.alert_p),
380     clk_i, !rst_ni || (alert_tx_o.alert_p ==
381     alert_tx_o.alert_n))
382 `endif
383
384 // Test in-band FSM reset request (via signal integrity
385 // error)
386 `ASSERT(InBandInitFsm_A, PingSigInt_S or AckSigInt_S |>
387     ##[SkewCycles+2:SkewCycles+3] state_q == Idle)
388 `ASSERT(InBandInitPing_A, PingSigInt_S or AckSigInt_S |>
389     ##[SkewCycles+2:SkewCycles+3] !ping_set_q)
390 // output must be driven diff unless sigint issue detected
391 `ASSERT(DiffEncoding_A, (alert_rx_i.ack_p ^
392     alert_rx_i.ack_n) &&
393     (alert_rx_i.ping_p ^ alert_rx_i.ping_n) |>
394     alert_tx_o.alert_p ^ alert_tx_o.alert_n)
395 // handshakes can take indefinite time if blocked due to

```

```

sigint on outgoing
332 // lines (which is not visible here). thus, we only check
333 // whether the
334 // handshake is correctly initiated and defer the full
335 // handshake checking to the testbench.
336 `ASSERT(PingHs_A, ##1 $changed(alert_rx_i.ping_p) &&
337     (alert_rx_i.ping_p ^ alert_rx_i.ping_n) ##2 state_q ==
338     Idle |>
339     ##[0:1] $rose(alert_tx_o.alert_p), clk_i,
340     !rst_ni || (alert_tx_o.alert_p == alert_tx_o.alert_n))
341 end else begin : gen_sync_assert
342     sequence PingSigInt_S;
343     alert_rx_i.ping_p == alert_rx_i.ping_n;
344 endsequence
345 sequence AckSigInt_S;
346 alert_rx_i.ping_p == alert_rx_i.ping_n;
347 endsequence
348
349 `ifndef FPV_ALERT_NO_SIGINT_ERR
350 // check propagation of sigint issues to output within one
351 // cycle
352 `ASSERT(SigIntPing_A, PingSigInt_S |>
353     alert_tx_o.alert_p == alert_tx_o.alert_n)
354 `ASSERT(SigIntAck_A, AckSigInt_S |>
355     alert_tx_o.alert_p == alert_tx_o.alert_n)
356 `endif
357
358 // Test in-band FSM reset request (via signal integrity
359 // error)
360 `ASSERT(InBandInitFsm_A, PingSigInt_S or AckSigInt_S |>
361     state_q == Idle)
362 `ASSERT(InBandInitPing_A, PingSigInt_S or AckSigInt_S |
363     => !ping_set_q)
364 // output must be driven diff unless sigint issue detected
365 `ASSERT(DiffEncoding_A, (alert_rx_i.ack_p ^
366     alert_rx_i.ack_n) &&
367     (alert_rx_i.ping_p ^ alert_rx_i.ping_n) |>
368     alert_tx_o.alert_p ^ alert_tx_o.alert_n)
369 // handshakes can take indefinite time if blocked due to
370 // sigint on outgoing
371 // lines (which is not visible here). thus, we only check
372 // whether the handshake
373 // is correctly initiated and defer the full handshake
374 // checking to the testbench.
375 `ASSERT(PingHs_A, ##1 $changed(alert_rx_i.ping_p) &&
376     state_q == Idle |>
377     $rose(alert_tx_o.alert_p), clk_i, !rst_ni ||
378     (alert_tx_o.alert_p == alert_tx_o.alert_n))
379 end
380 // Test the alert state output.
381 `ASSERT(AlertState0_A, alert_set_q == alert_state_o)
382
383 if (IsFatal) begin : gen_fatal_assert
384     `ASSERT(AlertState1_A, alert_req_i |> alert_state_o)
385     `ASSERT(AlertState2_A, alert_state_o |>
386         $stable(alert_state_o))
387     `ASSERT(AlertState3_A, alert_ack_o |> alert_state_o)
388 end else begin : gen_recov_assert
389     `ASSERT(AlertState1_A, alert_req_i && !alert_clr |>
390         alert_state_o)
391     `ASSERT(AlertState2_A, alert_req_i && alert_ack_o |> !
392         alert_state_o)
393 end
394
395 // The alert test input should not set the alert state re-
396 // gister.
397 `ASSERT(AlertTest1_A, alert_test_i && !alert_req_i && !
398     alert_state_o |> $stable(alert_state_o))
399
400 // if alert_req_i is true, handshakes should be continuously
401 // repeated
402 `ASSERT(AlertHs_A, alert_req_i && state_q == Idle |>
403     $rose(alert_tx_o.alert_p),
404     clk_i, !rst_ni || (alert_tx_o.alert_p ==
405     alert_tx_o.alert_n))
406
407 // if alert_test_i is true, handshakes should be continu-
408 // ously repeated
409 `ASSERT(AlertTestHs_A, alert_test_i && state_q == Idle |>
410     $rose(alert_tx_o.alert_p),
411     clk_i, !rst_ni || (alert_tx_o.alert_p ==
412     alert_tx_o.alert_n))
413 `endif
414
415 // Assumptions for FPV security countermeasures to ensure
416 // the alert protocol functions correctly.
417 `ASSUME_FPV(AckPFollowsAlertP_S, alert_rx_i.ack_p ==
418     $past(alert_tx_o.alert_p))
419 `ASSUME_FPV(AckNFollowsAlertN_S, alert_rx_i.ack_n ==
420     $past(alert_tx_o.alert_n))
421 `ASSUME_FPV(TriggerAlertInit_S, $stable(rst_ni) == 0 |>
422     alert_rx_i.ping_p == alert_rx_i.ping_n)
423 `ASSUME_FPV(PingDiffPair_S, ##2 alert_rx_i.ping_p !=
424     alert_rx_i.ping_n)
425 `endif
426 endmodule : prim_alert_sender

```

4. lowRISC_opentitan/hw/ip/prim/rtl/prim_alert_to_diff.sv (63 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_alert_to_diff: This module receives an alert signal
  (using the alert protocol) and
6 // translates it into a differentially encoded signal (without the
  alert protocol).
7 // It also handles integrity errors from the alert receiver,
  treating them as alerts.
8 //
9 // IMPORTANT: This primitive is NOT intended to be used in
  OpenTitan top-level designs.
10 // In top-level designs, prim_alert_{sender,receiver} pairs
  should be used instead.
11 // The intended use case of this primitive is to signal an
  alert through a differential
12 // encoding (for minimal integrity protection) to modules that do
  not support
13 // OpenTitan's alert protocol.
14
15 module prim_alert_to_diff #(
16     // AsyncOn: Enables additional synchronization logic within
  the alert receiver.
17     parameter bit AsyncOn = 1'b0,
18     // Number of cycles a differential skew is tolerated on the
  alert signal
19     parameter int unsigned SkewCycles = 1
20 ) (
21     input logic                clk_i,
22     input logic                rst_ni,
23     // Alert pair (interface signals for the alert protocol)
24     output prim_alert_pkg::alert_rx_t alert_rx_o,
25     input  prim_alert_pkg::alert_tx_t alert_tx_i,
26     // Output diff pair (differentially encoded alert signal)
27     output logic               diff_po,
28     output logic               diff_no
29 );
30
31 logic integ_error;
32 logic alert;
33
34 // u_prim_alert_receiver: Instantiates the alert receiver
35 module.
36     prim_alert_receiver #(
37         .AsyncOn(AsyncOn),
38         .SkewCycles(SkewCycles)
39     ) u_prim_alert_receiver (
40         .clk_i,
41         .rst_ni,
42         .init_trig_i (prim_mubi_pkg::MuBi4False),
43         .ping_req_i  (1'b0),
44         .ping_ok_o   (),
45         .integ_fail_o (integ_error),
46         .alert_o     (alert),
47         .alert_rx_o,
48         .alert_tx_i
49     );
50 // Combines the decoded alert and the integrity error signal.
51 // An alert is triggered if either a valid alert is received or
  an integrity error occurs.
52 logic combined_alert;
53 assign combined_alert = integ_error | alert;
54
55 // Instantiates the differential encoder module.
56 prim_diff_encode u_prim_diff_encode (
57     .clk_i,
58     .rst_ni,
59     .req_i(combined_alert), // Input request signal (combined
  alert).
60     .diff_po,
61     .diff_no
62 );
63 endmodule

```

5. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_fixed.sv (170 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // N:1 fixed priority arbiter module (index 0 has highest
  prio)
6 //
7 // Verilog parameter

```

```

8 // N:          Number of request ports
9 // DW:         Data width
10 // DataPort:   Set to 1 to enable the data port. Otherwise
  that port will be ignored.
11 //
12 // See also: prim_arbiter_ppc, prim_arbiter_tree
13
14 `include "prim_assert.sv"
15
16 module prim_arbiter_fixed #(
17     parameter int N = 8,
18     parameter int DW = 32,
19
20     // Configurations
21     // EnDataPort: {0, 1}, if 0, input data will be ignored
22     parameter bit EnDataPort = 1,
23
24     // Derived parameters
25     localparam int IdxW = $clog2(N)
26 ) (
27     // used for assertions only
28     input clk_i,
29     input rst_ni,
30
31     input [N-1:0] req_i,
32     input [DW-1:0] data_i [N],
33     output logic [N-1:0] gnt_o,
34     output logic [IdxW-1:0] idx_o,
35
36     output logic valid_o,
37     output logic [DW-1:0] data_o,
38     input ready_i
39 );
40
41 `ASSERT_INIT(CheckNGreaterZero_A, N > 0)
42
43 // this case is basically just a bypass
44 if (N == 1) begin : gen_degenerate_case
45
46     assign valid_o = req_i[0];
47     assign data_o = data_i[0];
48     assign gnt_o[0] = valid_o & ready_i;
49     assign idx_o = '0;
50
51 end else begin : gen_normal_case
52
53     // align to powers of 2 for simplicity
54     // a full binary tree with N levels has 2**N + 2**N-1
55 nodes
56     logic [2**(IdxW+1)-2:0] req_tree;
57     logic [2**(IdxW+1)-2:0] gnt_tree;
58     logic [2**(IdxW+1)-2:0][IdxW-1:0] idx_tree;
59     logic [2**(IdxW+1)-2:0][DW-1:0] data_tree;
60
61     for (genvar level = 0; level < IdxW+1; level++) begin :
  gen_tree
62         //
63         // level+1 C0 C1 <- "Base1" points to the first
  node on "level+1",
64         // \ / these nodes are the children
  of the nodes one level below
65         // level Pa <- "Base0", points to the first
  node on "level",
66         // these nodes are the parents
  of the nodes one level above
67         //
68         // hence we have the following indices for the Pa, C0,
  C1 nodes:
69         // Pa = 2**level - 1 + offset = Base0 + offset
70         // C0 = 2**(level+1) - 1 + 2*offset = Base1 + 2*off-
  set
71         // C1 = 2**(level+1) - 1 + 2*offset + 1 = Base1 + 2*off-
  set + 1
72         //
73         localparam int Base0 = (2**level)-1;
74         localparam int Base1 = (2**(level+1))-1;
75         for (genvar offset = 0; offset < 2**level; offset++) be-
  gin : gen_level
76             localparam int Pa = Base0 + offset;
77             localparam int C0 = Base1 + 2*offset;
78             localparam int C1 = Base1 + 2*offset + 1;
79
80             // this assigns the gated interrupt source signals,
  their
81             // corresponding IDs and priorities to the tree leafs
82             if (level == IdxW) begin : gen_leafs
83                 if (offset < N) begin : gen_assign
84                     // forward path
85                     assign req_tree[Pa] = req_i[offset];
86                     assign idx_tree[Pa] = offset;
87                     assign data_tree[Pa] = data_i[offset];
88                     // backward (grant) path
89                     assign gnt_o[offset] = gnt_tree[Pa];
90
91             end else begin : gen_tie_off
92                 // forward path
93                 assign req_tree[Pa] = '0;
94                 assign idx_tree[Pa] = '0;
95                 assign data_tree[Pa] = '0;
96                 logic unused_sigs;

```

```

97         assign unused_sigs = gnt_tree[Pa];
98     end
99     // this creates the node assignments
100 end else begin : gen_nodes
101     // forward path
102     logic sel; // local helper variable
103     always_comb begin : p_node
104         // this always gives priority to the left child
105         sel = ~req_tree[C0];
106         // propagate requests
107         req_tree[Pa] = req_tree[C0] | req_tree[C1];
108         // data and index muxes
109         idx_tree[Pa] = (sel) ? idx_tree[C1] :
idx_tree[C0];
110     data_tree[Pa] = (sel) ? data_tree[C1] :
data_tree[C0];
111     // propagate the grants back to the input
112     gnt_tree[C0] = gnt_tree[Pa] & ~sel;
113     gnt_tree[C1] = gnt_tree[Pa] & sel;
114     end
115 end
116 end : gen_level
117 end : gen_tree
118
119 // the results can be found at the tree root
120 if (EnDataPort) begin : gen_data_port
121     assign data_o = data_tree[0];
122 end else begin : gen_no_dataport
123     logic [DW-1:0] unused_data;
124     assign unused_data = data_tree[0];
125     assign data_o = '1;
126 end
127
128 assign idx_o = idx_tree[0];
129 assign valid_o = req_tree[0];
130
131 // this propagates a grant back to the input
132 assign gnt_tree[0] = valid_o & ready_i;
133 end
134
135 // assertions
136 // assertions
137 // assertions
138
139 // KNOWN assertions on outputs, except for data as that may
be partially X in simulation
140 // e.g. when used on a BUS
141 `ASSERT_KNOWN(ValidKnown_A, valid_o)
142 `ASSERT_KNOWN(GrantKnown_A, gnt_o)
143 `ASSERT_KNOWN(IdxKnown_A, idx_o)
144
145 // Make sure no higher prio req is asserted
146 `ASSERT(Priority_A, |req_i |-> req_i[idx_o] && (((N'(1'b1)
<< idx_o) - 1'b1) & req_i) == '0)
147
148 // we can only grant one requester at a time
149 `ASSERT(CheckHotOne_A, $onehot0(gnt_o))
150 // A grant implies that the sink is ready
151 `ASSERT(GntImpliesReady_A, |gnt_o |-> ready_i)
152 // A grant implies that the arbiter asserts valid as well
153 `ASSERT(GntImpliesValid_A, |gnt_o |-> valid_o)
154 // A request and a sink that is ready imply a grant
155 `ASSERT(ReqAndReadyImpliesGrant_A, |req_i && ready_i |-> |
gnt_o)
156 // A request and a sink that is ready imply a grant
157 `ASSERT(ReqImpliesValid_A, |req_i |-> valid_o)
158 // Both conditions above combined and reversed
159 `ASSERT(ReadyAndValidImpliesGrant_A, ready_i && valid_o |-> |
gnt_o)
160 // Both conditions above combined and reversed
161 `ASSERT(NoReadyValidNoGrant_A, !(ready_i || valid_o) |->
gnt_o == 0)
162 // check index / grant correspond
163 `ASSERT(IndexIsCorrect_A, ready_i && valid_o |->
gnt_o[idx_o] && req_i[idx_o])
164
165 if (EnDataPort) begin: gen_data_port_assertion
166     // data flow
167     `ASSERT(DataFlow_A, ready_i && valid_o |-> data_o ==
data_i[idx_o])
168 end
169
170 endmodule : prim_arbiter_fixed

```

6. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_ppc.sv (210 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // N:1 arbiter module
6 //
7 // Verilog parameter
8 // N: Number of request ports
9 // DW: Data width
10 // DataPort: Set to 1 to enable the data port. Otherwise
that port will be ignored.

```

```

11 //
12 // This is the original implementation of the arbiter which
relies on parallel prefix computing
13 // optimization to optimize the request / arbiter tree. Not
all synthesis tools may support this.
14 //
15 // Note that the currently winning request is held if the data
sink is not ready. This behavior is
16 // required by some interconnect protocols (AXI, TL). The mod-
ule contains an assertion that checks
17 // this behavior.
18 //
19 // Also, this module contains a request stability assertion
that checks that requests stay asserted
20 // until they have been served. This assertion can be gated by
driving the req_chk_i low. This is
21 // a non-functional input and does not affect the designs be-
havior.
22 //
23 // See also: prim_arbiter_tree
24
25 `include "prim_assert.sv"
26
27 module prim_arbiter_ppc #(
28     parameter int unsigned N = 8,
29     parameter int unsigned DW = 32,
30
31     // Configurations
32     // EnDataPort: {0, 1}, if 0, input data will be ignored
33     parameter bit EnDataPort = 1,
34
35     // Derived parameters
36     localparam int IdxW = prim_util_pkg::vbits(N)
37 ) (
38     input clk_i,
39     input rst_ni,
40
41     input req_chk_i, // Used for gating as-
sertions. Drive to 1 during normal
42 // operation.
43     input [N-1:0] req_i,
44     input [DW-1:0] data_i [N],
45     output logic [N-1:0] gnt_o,
46     output logic [IdxW-1:0] idx_o,
47
48     output logic valid_o,
49     output logic [DW-1:0] data_o,
50     input ready_i
51 );
52
53 // req_chk_i is used for gating assertions only.
54 logic unused_req_chk;
55 assign unused_req_chk = req_chk_i;
56
57 `ASSERT_INIT(CheckNGreaterZero_A, N > 0)
58
59 // this case is basically just a bypass
60 if (N == 1) begin : gen_degenerate_case
61
62     assign valid_o = req_i[0];
63     assign data_o = data_i[0];
64     assign gnt_o[0] = valid_o & ready_i;
65     assign idx_o = '0;
66
67 end else begin : gen_normal_case
68     logic [N-1:0] masked_req;
69     logic [N-1:0] ppc_out;
70     logic [N-1:0] arb_req;
71     logic [N-1:0] mask, mask_next;
72     logic [N-1:0] winner;
73
74     assign masked_req = mask & req_i;
75     assign arb_req = (!masked_req) ? masked_req : req_i;
76
77     prim_leading_one_ppc #(
78         .N ( N )
79     ) u_leading_one (
80         .in_i ( arb_req ),
81         .leading_one_o ( winner ),
82         .ppc_out_o ( ppc_out ),
83         .idx_o ( idx_o )
84     );
85     assign gnt_o = (ready_i) ? winner : '0;
86     assign valid_o = |req_i;
87     // Mask Generation
88     assign mask_next = {ppc_out[N-2:0], 1'b0};
89     always_ff @(posedge clk_i or negedge rst_ni) begin
90         if (!rst_ni) begin
91             mask <= '0;
92         end else if (valid_o && ready_i) begin
93             // Latch only when requests accepted
94             mask <= mask_next;
95         end else if (valid_o && !ready_i) begin
96
97             // Downstream isn't yet ready so, keep current request
alive. (First come first serve)
98             mask <= ppc_out;
99         end

```

```

100
101   if (EnDataPort == 1) begin: gen_datapath
102     always_comb begin
103       data_o = '0;
104       for (int i = 0; i < N; i++) begin
105         if (winner[i]) begin
106           data_o = data_i[i];
107         end
108       end
109     end
110   end else begin: gen_nodatapath
111     assign data_o = '1;
112     // The following signal is used to avoid possible lint
113 errors.
114     logic [DW-1:0] unused_data [N];
115     assign unused_data = data_i;
116   end
117
118   // assertions //
119   // assertions //
120   // assertions //
121
122   // KNOWN assertions on outputs, except for data as that may
123 be partially X in simulation
124   // e.g. when used on a BUS
125   `ASSERT_KNOWN(ValidKnown_A, valid_o)
126   `ASSERT_KNOWN(GrantKnown_A, gnt_o)
127   `ASSERT_KNOWN(IdxKnown_A, idx_o)
128   // grant index shall be higher index than previous index,
129 unless no higher requests exist.
130   `ASSERT(RoundRobin_A,
131     #1 valid_o && ready_i && $past(ready_i) && $past(val-
132 id_o) &&
133     |((req_i & ~(N'(1) << $past(idx_o)+1) - 1)) |>->
134     idx_o > $past(idx_o))
135   // we can only grant one requester at a time
136   `ASSERT(CheckHotOne_A, $onehot0(gnt_o))
137   // A grant implies that the sink is ready
138   `ASSERT(GntImpliesReady_A, |gnt_o |>-> ready_i)
139   // A grant implies that the arbiter asserts valid as well
140   `ASSERT(GntImpliesValid_A, |gnt_o |>-> valid_o)
141   // A request and a sink that is ready imply a grant
142   `ASSERT(ReqAndReadyImpliesGrant_A, |req_i && ready_i |>-> |
143 gnt_o)
144   // A request and a sink that is ready imply a grant
145   `ASSERT(ReqImpliesValid_A, |req_i |>-> valid_o)
146   // Both conditions above combined and reversed
147   `ASSERT(ReadyAndValidImpliesGrant_A, ready_i && valid_o |>-> |
148 gnt_o)
149   // Both conditions above combined and reversed
150   `ASSERT(NoReadyValidNoGrant_A, !(ready_i || valid_o) |>->
151 gnt_o == 0)
152   // check index / grant correspond
153   `ASSERT(IndexIsCorrect_A, ready_i && valid_o |>->
154 gnt_o[idx_o] && req_i[idx_o])
155
156   if (EnDataPort) begin: gen_data_port_assertion
157     // data flow
158     `ASSERT(DataFlow_A, ready_i && valid_o |>-> data_o ==
159 data_i[idx_o])
160   end
161
162   // requests must stay asserted until they have been granted
163   `ASSUME(ReqStaysHighUntilGranted0_M, |req_i && !ready_i |>=
164 (req_i & $past(req_i)) == $past(req_i), clk_i, !rst_ni
165 || !req_chk_i)
166   // check that the arbitration decision is held if the sink
167 is not ready
168   `ASSERT(LockArbDecision_A, |req_i && !ready_i |>= idx_o ==
169 $past(idx_o),
170   clk_i, !rst_ni || !req_chk_i)
171
172   // FPV-only assertions with symbolic variables
173   `ifdef FPV_ON
174     // symbolic variables
175     bit [IdxW-1:0] k;
176     bit ReadyIsStable;
177     bit ReqsAreStable;
178
179     // constraints for symbolic variables
180     `ASSUME(KStable_M, #1 $stable(k))
181     `ASSUME(KRange_M, k < N)
182     // this is used enable checking for stable and unstable
183 ready_i and req_i signals in the same run.
184     // the symbolic variables act like a switch that the solver
185 can turn on and off.
186     `ASSUME(ReadyIsStable_M, #1 $stable(ReadyIsStable))
187     `ASSUME(ReqsAreStable_M, #1 $stable(ReqsAreStable))
188     `ASSUME(ReadyStable_M, #1 !ReadyIsStable ||
189 $stable(ready_i))
190     `ASSUME(ReqsStable_M, #1 !ReqsAreStable || $stable(req_i))
191
192     // A grant implies a request
193     `ASSERT(GntImpliesReq_A, gnt_o[k] |>-> req_i[k])
194
195     // if request and ready are constantly held at 1, we should
196 eventually get a grant
197     `ASSERT(NoStarvation_A,
198       ReqsAreStable && ReadyIsStable && ready_i && req_i[k] |>-

```

```

>
185     strong(#[0:$] gnt_o[k]))
186
187   // if N requests are constantly asserted and ready is con-
188 stant 1, each request must
189   // be granted exactly once over a time window of N cycles
190 for the arbiter to be fair.
191   for (genvar n = 1; n <= N; n++) begin : gen_fairness
192     integer gnt_cnt;
193     `ASSERT(Fairness_A,
194       ReqsAreStable && ReadyIsStable && ready_i && req_i[k]
195 &&
196       $countones(req_i) == n |>->
197       ##n gnt_cnt == $past(gnt_cnt, n) + 1)
198
199     always_ff @(posedge clk_i or negedge rst_ni) begin : p_cnt
200       if (!rst_ni) begin
201         gnt_cnt <= 0;
202       end else begin
203         gnt_cnt <= gnt_cnt + gnt_o[k];
204       end
205     end
206   end
207
208   // requests must stay asserted until they have been granted
209   `ASSUME(ReqStaysHighUntilGranted1_M, req_i[k] && !gnt_o[k] |
210 =>
211   req_i[k], clk_i, !rst_ni || !req_chk_i)
212 `endif
213 endmodule : prim_arbiter_ppc

```

7. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree.sv (291 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // N:1 arbiter module
7 //
8 // Verilog parameter
9 // N: Number of request ports
10 // DW: Data width
11 // DataPort: Set to 1 to enable the data port. Otherwise
12 that port will be ignored.
13 //
14 // This is a tree implementation of a round robin arbiter. It
15 has the same behavior as the PPC
16 // implementation in prim_arbiter_ppc, and also uses a prefix
17 summing approach to determine the next
18 // request to be granted. The main difference with respect to
19 the PPC arbiter is that the leading 1
20 // detection and the prefix summation are performed with a
21 binary tree instead of a sequential loop.
22 // Also, if the data port is enabled, the data is muxed based
23 on the local arbitration decisions at
24
25 // each node of the arbiter tree. This means that the data can
26 propagate through the tree
27 // simultaneously with the requests, instead of waiting for
28 the arbitration to determine the winner
29 // index first. As a result, this design has a shorter critic-
30 al path than other implementations,
31 // leading to better overall timing.
32
33 // Note that the currently winning request is held if the data
34 sink is not ready. This behavior is
35 // required by some interconnect protocols (AXI, TL). The mod-
36 ule contains an assertion that checks
37 // this behavior.
38 //
39 // Also, this module contains a request stability assertion
40 that checks that requests stay asserted
41
42 // until they have been served. This assertion can be gated by
43 driving the req_chk_i low. This is
44 // a non-functional input and does not affect the designs be-
45 havior.
46 //
47 // See also: prim_arbiter_ppc
48
49 `include "prim_assert.sv"
50
51 module prim_arbiter_tree #(
52   parameter int N = 8,
53   parameter int DW = 32,
54
55   // Configurations
56   // EnDataPort: {0, 1}, if 0, input data will be ignored
57   parameter bit EnDataPort = 1,
58
59   // Derived parameters
60   localparam int IdxW = $clog2(N)
61 ) (
62   input clk_i,
63   input rst_ni,

```



```

47
48 input req_chk_i, // Used for gating as-
sections. Drive to 1 during normal
49 // operation.
50 input [N-1:0] req_i,
51 input [DW-1:0] data_i [N],
52 output logic [N-1:0] gnt_o,
53 output logic [IdxW-1:0] idx_o,
54
55 output logic valid_o,
56 output logic [DW-1:0] data_o,
57 input ready_i
58 );
59
60 // req_chk_i is used for gating assertions only.
61 logic unused_req_chk;
62 assign unused_req_chk = req_chk_i;
63
64 `ASSERT_INIT(CheckNGreaterZero_A, N > 0)
65
66 // this case is basically just a bypass
67 if (N == 1) begin : gen_degenerate_case
68
69 assign valid_o = req_i[0];
70 assign data_o = data_i[0];
71 assign gnt_o[0] = valid_o & ready_i;
72 assign idx_o = '0;
73
74 end else begin : gen_normal_case
75
76 // align to powers of 2 for simplicity
77 // a full binary tree with N levels has 2**N + 2**N-1
nodes
78 logic [2*(IdxW+1)-2:0] req_tree;
79 logic [2*(IdxW+1)-2:0] prio_tree;
80 logic [2*(IdxW+1)-2:0] sel_tree;
81 logic [2*(IdxW+1)-2:0] mask_tree;
82 logic [2*(IdxW+1)-2:0][IdxW-1:0] idx_tree;
83 logic [2*(IdxW+1)-2:0][DW-1:0] data_tree;
84 logic [N-1:0] prio_mask_d,
prio_mask_q;
85
86 for (genvar level = 0; level < IdxW+1; level++) begin :
gen_tree
87 //
88 // level+1 C0 C1 <- "Base1" points to the first
node on "level+1",
89 // \ / these nodes are the children
of the nodes one level below
90 // level Pa <- "Base0", points to the first
node on "level",
91 // these nodes are the parents
of the nodes one level above
92 //
93 // hence we have the following indices for the Pa, C0,
C1 nodes:
94 // Pa = 2**level - 1 + offset = Base0 + offset
95 // C0 = 2**(level+1) - 1 + 2*offset = Base1 + 2*off-
set
96 // C1 = 2**(level+1) - 1 + 2*offset + 1 = Base1 + 2*off-
set + 1
97 //
98 localparam int Base0 = (2**level)-1;
99 localparam int Base1 = (2**(level+1))-1;
100
101 for (genvar offset = 0; offset < 2**level; offset++) be-
gin : gen_level
102 localparam int Pa = Base0 + offset;
103 localparam int C0 = Base1 + 2*offset;
104 localparam int C1 = Base1 + 2*offset + 1;
105
106 // this assigns the gated interrupt source signals,
their
107 // corresponding IDs and priorities to the tree leafs
108 if (level == IdxW) begin : gen_leafs
109 if (offset < N) begin : gen_assign
110 // forward path (requests and data)
111
112 // all requests inputs are assigned to the request
tree
112 assign req_tree[Pa] = req_i[offset];
113 // we basically split the incoming request vector
into two halves with the following
114 // priority assignment. the prio_mask_q register
contains a prefix sum that has been
115 // computed using the last winning index, and
hence masks out all requests at offsets
116 // lower or equal the previously granted index.
hence, all higher indices are considered
117 // first in the arbitration tree nodes below, be-
fore considering the lower indices.
118 assign prio_tree[Pa] = req_i[offset] &
prio_mask_q[offset];
119 // input for the index muxes (used to compute the
winner index)
120 assign idx_tree[Pa] = offset;
121 // input for the data muxes
122 assign data_tree[Pa] = data_i[offset];
123
124 // backward path (grants and prefix sum)
125 // grant if selected, ready and request asserted

```

```

126 assign gnt_o[offset] = req_i[offset] &
sel_tree[Pa] & ready_i;
127 // only update mask if there is a valid request
128 assign prio_mask_d[offset] = (req_i) ?
mask_tree[Pa] |
129 sel_tree[Pa] & ~ready_i :
prio_mask_q[offset];
130
131 end else begin : gen_tie_off
132 // forward path
133 assign req_tree[Pa] = '0;
134 assign prio_tree[Pa] = '0;
135 assign idx_tree[Pa] = '0;
136 assign data_tree[Pa] = '0;
137 logic unused_sigs;
138 assign unused_sigs = ^{mask_tree[Pa],
sel_tree[Pa]};
139
140 end
141 // this creates the node assignments
142 end else begin : gen_nodes
143 // local helper variable
144 logic sel;
145
146 // forward path (requests and data)
147 // each node looks at its two children, and selects
the one with higher priority
148 assign sel = ~req_tree[C0] | ~prio_tree[C0] &
prio_tree[C1];
149 // propagate requests
150 assign req_tree[Pa] = req_tree[C0] | req_tree[C1];
151 assign prio_tree[Pa] = prio_tree[C1] |
prio_tree[C0];
152 // data and index muxes
153 // Note: these ternaries have triggered a synthesis
bug in Vivado versions older
154 // than 2020.2. If the problem resurfaces again,
have a look at issue #1408.
155 assign idx_tree[Pa] = (sel) ? idx_tree[C1] :
idx_tree[C0];
156 assign data_tree[Pa] = (sel) ? data_tree[C1] :
data_tree[C0];
157
158 // backward path (grants and prefix sum)
159 // this propagates the selection index back and com-
putes a hot one mask
160 assign sel_tree[C0] = sel_tree[Pa] & ~sel;
161 assign sel_tree[C1] = sel_tree[Pa] & sel;
162 // this performs a prefix sum for masking the input
requests in the next cycle
163 assign mask_tree[C0] = mask_tree[Pa];
164 assign mask_tree[C1] = mask_tree[Pa] | sel_tree[C0];
165 end
166 end : gen_level
167 end : gen_tree
168
169 // the results can be found at the tree root
170 if (EnDataPort) begin : gen_data_port
171 assign data_o = data_tree[0];
172 end else begin : gen_no_dataport
173 logic [DW-1:0] unused_data;
174 assign unused_data = data_tree[0];
175 assign data_o = '1;
176 end
177
178 // This index is unused.
179 logic unused_prio_tree;
180 assign unused_prio_tree = prio_tree[0];
181
182 assign idx_o = idx_tree[0];
183 assign valid_o = req_tree[0];
184
185 // the select tree computes a hot one signal that indic-
ates which request is currently selected
186 assign sel_tree[0] = 1'b1;
187 // the mask tree is basically a prefix sum of the hot one
select signal computed above
188 assign mask_tree[0] = 1'b0;
189
190 always_ff @(posedge clk_i or negedge rst_ni) begin :
p_mask_reg
191 if (!rst_ni) begin
192 prio_mask_q <= '0;
193 end else begin
194 prio_mask_q <= prio_mask_d;
195 end
196 end
197 end
198
199 // assertions //
200 // assertions //
201 // assertions //
202
203 // KNOWN assertions on outputs, except for data as that may
be partially X in simulation
204 // e.g. when used on a BUS
205 `ASSERT_KNOWN(ValidKnown_A, valid_o)
206 `ASSERT_KNOWN(GrantKnown_A, gnt_o)
207 `ASSERT_KNOWN(IdxKnown_A, idx_o)
208
209 // grant index shall be higher index than previous index,
unless no higher requests exist.
210 `ASSERT(RoundRobin_A,

```

```

211      ##1 valid_o && ready_i && $past(ready_i) && $past(val-
id_o) &&
212      |(req_i & ~(N'(1) << $past(idx_o)+1) - 1)) |->
213      idx_o > $past(idx_o))
214      // we can only grant one requester at a time
215      `ASSERT(CheckHotOne_A, $onehot0(gnt_o))
216      // A grant implies that the sink is ready
217      `ASSERT(GntImpliesReady_A, |gnt_o |-> ready_i)
218      // A grant implies that the arbiter asserts valid as well
219      `ASSERT(GntImpliesValid_A, |gnt_o |-> valid_o)
220      // A request and a sink that is ready imply a grant
221      `ASSERT(ReqAndReadyImpliesGrant_A, |req_i && ready_i |-> |
gnt_o)
222      // A request and a sink that is ready imply a grant
223      `ASSERT(ReqImpliesValid_A, |req_i |-> valid_o)
224      // Both conditions above combined and reversed
225      `ASSERT(ReadyAndValidImpliesGrant_A, ready_i && valid_o |-> |
gnt_o)
226      // Both conditions above combined and reversed
227      `ASSERT(NoReadyValidNoGrant_A, !(ready_i || valid_o) |->
gnt_o == 0)
228      // check index / grant correspond
229      `ASSERT(IndexIsCorrect_A, ready_i && valid_o |->
gnt_o[idx_o] && req_i[idx_o])
230
231      if (EnDataPort) begin: gen_data_port_assertion
232      // data flow
233      `ASSERT(DataFlow_A, ready_i && valid_o |-> data_o ==
data_i[idx_o])
234      end
235
236      // requests must stay asserted until they have been granted
237      `ASSUME(ReqStaysHighUntilGranted0_M, |req_i && !ready_i |>=
238      (req_i & $past(req_i)) == $past(req_i), clk_i, !rst_ni
|| !req_chk_i)
239      // check that the arbitration decision is held if the sink
is not ready
240      `ASSERT(LockArbDecision_A, |req_i && !ready_i |>= idx_o ==
$past(idx_o),
241      clk_i, !rst_ni || !req_chk_i)
242
243      // FPV-only assertions with symbolic variables
244      `ifdef FPV_ON
245      // symbolic variables
246      int unsigned k;
247      bit ReadyIsStable;
248      bit ReqsAreStable;
249
250      // constraints for symbolic variables
251      `ASSUME(KStable_M, ##1 $stable(k))
252      `ASSUME(KRange_M, k < N)
253      // this is used enable checking for stable and unstable
ready_i
and req_i signals in the same run.
254      // the symbolic variables act like a switch that the solver
can turn on and off.
255      `ASSUME(ReadyIsStable_M, ##1 $stable(ReadyIsStable))
256      `ASSUME(ReqsAreStable_M, ##1 $stable(ReqsAreStable))
257      `ASSUME(ReadyStable_M, ##1 !ReadyIsStable ||
$stable(ready_i))
258      `ASSUME(ReqsStable_M, ##1 !ReqsAreStable || $stable(req_i))
259
260      // A grant implies a request
261      `ASSERT(GntImpliesReq_A, gnt_o[k] |-> req_i[k])
262
263      // if request and ready are constantly held at 1, we should
eventually
get a grant
264      `ASSERT(NoStarvation_A,
265      ReqsAreStable && ReadyIsStable && ready_i && req_i[k] |->
>
266      strong(##[0:$] gnt_o[k]))
267
268      // if N requests are constantly asserted and ready is con-
stant 1,
each request must
269      // be granted exactly once over a time window of N cycles
for the
arbiter to be fair.
270      for (genvar n = 1; n <= N; n++) begin : gen_fairness
271      integer gnt_cnt;
272      `ASSERT(Fairness_A,
273      ReqsAreStable && ReadyIsStable && ready_i && req_i[k]
&&
274      $countones(req_i) == n |->
275      ##n gnt_cnt == $past(gnt_cnt, n) + 1)
276
277      always_ff @(posedge clk_i or negedge rst_ni) begin : p_cnt
278      if (!rst_ni) begin
279      gnt_cnt <= 0;
280      end else begin
281      gnt_cnt <= gnt_cnt + gnt_o[k];
282      end
283      end
284      end
285
286      // requests must stay asserted until they have been granted
287      `ASSUME(ReqStaysHighUntilGranted1_M, req_i[k] && !gnt_o[k] |
=>
288      req_i[k], clk_i, !rst_ni || !req_chk_i)
289      `endif
290
291      endmodule : prim_arbiter_tree

```

8. lowRISC_opentitan/hw/ip/prim/rtl/prim_arbiter_tree_dup.sv (148 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This is a wrapper module that instantiates two prim_arbit-
er_tree modules.
6 // The reason for two is similar to modules such as
prim_count/prim_lfsr where
7 // we use spatial redundancy to ensure the arbitration results are
not altered.
8 //
9 // Note there are limits to this check, as the inputs to the
duplicated arbiter
10 // is still a single lane. So an upstream attack can defeat
this module. The
11 // duplication merely protects against attacks directly on the
arbiter.
12
13 `include "prim_assert.sv"
14
15 module prim_arbiter_tree_dup #(
16     parameter int N = 8,
17     parameter int DW = 32,
18
19     // Configurations
20     `EnDataPort: {0, 1}, if 0, input data will be ignored
parameter bit EnDataPort = 1,
21
22     // if arbiter has fixed priority
parameter bit FixedArb = 0,
23
24     // Derived parameters
25     localparam int IdxW = $clog2(N)
26 ) (
27     input clk_i,
28     input rst_ni,
29
30     input req_chk_i, // Used for gating as-
sions. Drive to 1 during normal
31
32     // operation.
33     input [N-1:0] req_i,
34     input [DW-1:0] data_i [N],
35     output logic [N-1:0] gnt_o,
36     output logic [IdxW-1:0] idx_o,
37
38     output logic valid_o,
39     output logic [DW-1:0] data_o,
40     input ready_i,
41     output logic err_o
42 );
43
44 localparam int ArbInstances = 2;
45
46 //typedef struct packed {
47 //    logic [N-1:0] req;
48 //    logic [N-1:0][DW-1:0] data;
49 //} arb_inputs_t;
50
51 typedef struct packed {
52     logic valid;
53     logic [N-1:0] gnt;
54     logic [IdxW-1:0] idx;
55     logic [DW-1:0] data;
56 } arb_outputs_t;
57
58 // buffer up the inputs separately for each instance
59 //arb_inputs_t arb_in;
60 //arb_inputs_t [ArbInstances-1:0] arb_input_buf;
61 arb_outputs_t [ArbInstances-1:0] arb_output_buf;
62
63 for (genvar i = 0; i < ArbInstances; i++) begin : gen_in-
put_bufs
64     logic [N-1:0] req_buf;
65     prim_buf #(
66         .Width(N)
67     ) u_req_buf (
68         .in_i(req_i),
69         .out_o(req_buf)
70     );
71
72     logic [DW-1:0] data_buf [N];
73     for (genvar j = 0; j < N; j++) begin : gen_data_bufs
74         prim_buf #(
75             .Width(DW)
76         ) u_dat_buf (
77             .in_i(data_i[j]),
78             .out_o(data_buf[j])
79         );
80     end
81 end
82
83 if (FixedArb) begin : gen_fixed_arbiter
84     prim_arbiter_fixed #(

```



```

85     .N(N),
86     .Dw(Dw),
87     .EnDataPort(EnDataPort)
88   ) u_arb (
89     .clk_i,
90     .rst_ni,
91     .req_i(req_buf),
92     .data_i(data_buf),
93     .gnt_o(arb_output_buf[i].gnt),
94     .idx_o(arb_output_buf[i].idx),
95     .valid_o(arb_output_buf[i].valid),
96     .data_o(arb_output_buf[i].data),
97     .ready_i
98   );
99   logic unused_req_chk;
100   assign unused_req_chk = req_chk_i;
101
102   end else begin : gen_rr_arbiter
103     prim_arbiter_tree #(
104       .N(N),
105       .Dw(Dw),
106       .EnDataPort(EnDataPort)
107     ) u_arb (
108       .clk_i,
109       .rst_ni,
110       .req_chk_i,
111       .req_i(req_buf),
112       .data_i(data_buf),
113       .gnt_o(arb_output_buf[i].gnt),
114       .idx_o(arb_output_buf[i].idx),
115       .valid_o(arb_output_buf[i].valid),
116       .data_o(arb_output_buf[i].data),
117       .ready_i
118     );
119   end
120 end
121
122 // the last buffered position is sent out
123 assign gnt_o = arb_output_buf[ArbInstances-1].gnt;
124 assign idx_o = arb_output_buf[ArbInstances-1].idx;
125 assign valid_o = arb_output_buf[ArbInstances-1].valid;
126 assign data_o = arb_output_buf[ArbInstances-1].data;
127
128 // Check the last buffer index against all other instances
129 logic [ArbInstances-2:0] output_delta;
130
131 for (genvar i = 0; i < ArbInstances-1; i++) begin :
gen_checks
132   assign output_delta[i] = arb_output_buf[ArbInstances-1] !=
arb_output_buf[i];
133 end
134
135 logic err_d, err_q;
136 // There is an error if anything ever disagrees
137 assign err_d = |output_delta;
138 always_ff @(posedge clk_i or negedge rst_ni) begin
139   if (!rst_ni) begin
140     err_q <= '0;
141   end else begin
142     err_q <= err_d | err_q;
143   end
144 end
145
146 assign err_o = err_q;
147
148 endmodule // prim_arbiter_tree

```

9. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_duplex.sv

(621 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Ascon duplex function implementation
6
7 // Ascon uses big endian encoding!
8
9 // Thus prim_ascon_duplex expects partial input blocks to fill up
from left to right.
10 // This means data_in_i[127:120] must always be set for non
empty inputs!
11
12 // TODO: Add countermeasures: mubi for control signals,
random values for state_to_round input, blinding for
output signals
13 // TODO: Add backpressure logic, if output is not ready
14 // TODO: Add check what to do, if data_in_valid_bytes_i is
greater than blocksize
15
16 module prim_ascon_duplex
17   import prim_ascon_pkg::*;
18 (
19   input logic clk_i,
20   input logic rst_ni,
21
22   input duplex_variant_e ascon_variant,

```

```

23   input duplex_op_e ascon_operation,
24
25   input logic start_i,
26   output logic done_o,
27
28   output prim_mubi_pkg::mubi4_t idle_o,
29
30   // It is assumed that no_ad, no_msg, key, and nonce are al-
ways
31   // valid and constant, when the cipher is triggered by the
start command
32   input prim_mubi_pkg::mubi4_t no_ad_i,
33   input prim_mubi_pkg::mubi4_t no_msg_i,
34
35   input logic [127:0] key_i,
36   input logic [127:0] nonce_i,
37
38   // Cipher Input Port
39   input logic [127:0] data_in_i,
40   input logic [4:0] data_in_valid_bytes_i,
41   input prim_mubi_pkg::mubi4_t last_block_ad_i,
42   input prim_mubi_pkg::mubi4_t last_block_msg_i,
43   input logic data_in_valid_i,
44   output logic data_in_ready_o,
45
46   // Cipher Output Port
47   output logic [127:0] data_out_o,
48   input logic data_out_ready_i,
49   output logic data_out_valid_o,
50
51   output logic [127:0] tag_out_o,
52   output logic tag_out_valid_o,
53
54   output duplex_fsm_state_e fsm_state_o,
55
56   output logic err_o
57 );
58
59 // TODO: Add backpressure check
60 logic unused_data_out_ready_i;
61 assign unused_data_out_ready_i = data_out_ready_i;
62
63 logic round_count_error;
64 logic sparse_fsm_error;
65
66 logic set_round_counter;
67 logic inc_round_counter;
68
69 logic [4:0][63:0] ascon_state_q, ascon_state_d;
70
71 // Ascon's 320 bit state
72 always_ff @(posedge clk_i or negedge rst_ni) begin : as-
con_state_reg
73   if (!rst_ni) begin
74     ascon_state_q <= '0;
75   end else begin
76     ascon_state_q <= ascon_state_d;
77   end
78 end
79
80 logic [319:0] state_to_round;
81 logic [319:0] state_from_round;
82
83 logic [4:0][63:0] round_to_mux;
84 assign round_to_mux = state_from_round;
85
86 duplex_fsm_state_e fsm_state_d, fsm_state_q;
87 perm_offset_e perm_offset;
88
89 assign fsm_state_o = fsm_state_q;
90
91 logic [63:0] iv;
92 assign iv = (ascon_variant == ASCON_128) ? IV_128 : IV_128A;
93
94 // internal combinatorial signals
95 prim_mubi_pkg::mubi4_t complete_block;
96
97 // TODO add a check what to do, if data_in_valid_bytes_i is
greater than blocksize
98 assign complete_block = (ascon_variant == ASCON_128 &&
data_in_valid_bytes_i == 8)
99   || (ascon_variant == ASCON_128A &&
data_in_valid_bytes_i == 16) ?
100     prim_mubi_pkg::MuBi4True :
prim_mubi_pkg::MuBi4False;
101
102 // Padding:
103 // 1) Associated Data
104 // a) empty AD: No padding, AD-processing phase is skipped!
105 // b) incomplete last block: A 10* padding is added to the
input data.
106 // The padded associated data is XORed to the state.
107 // c) complete last block: No padding to the input, but an
additional state in the FSM
108 // is used to perform the padding. A 10+ block is XORed
to the state.
109 // 2) Encryption:
110 // a) empty or incomplete last block: A 10* padding is ad-
ded to the input data.
111 // The padded associated data is XORed blockwise.
112 // b) complete last block: No padding to the input, but an

```

```

additional state in the FSM
113 // is used to perform the padding. A 10+ block is XORed
to the state.
114 // 3) Decryption:
115 // a) empty or incomplete last block: The padded (output)
PLAINTEXT is XORed to the state.
116 // This is equivalent to:
117 // The unpadded part of the Ciphertext replaces the cor-
responding part of S_r
118 // The remaining part of S_r is XORed with 10*.
119 // b) complete last block: No padding to the input, but an
additional state in the FSM
120 // is used to perform the padding. A 10+ block is XORed
to the state,
121
122 // Padding logic
123 logic [127:0] empty_padding;
124 assign empty_padding = get_padding_mask(5'b00000);
125
126 logic [127:0] valid_bytes_bit_mask;
127 assign valid_bytes_bit_mask =
bin2thermo(data_in_valid_bytes_i);
128
129 logic [127:0] padding_byte_bit_mask;
130 assign padding_byte_bit_mask = get_pad-
ding_mask(data_in_valid_bytes_i);
131
132 logic [127:0] data_in_valid_bytes;
133 assign data_in_valid_bytes = data_in_i & valid_bytes_bit_mask;
134
135 // data output
136 logic [127:0] data_out;
137 assign data_out = (data_in_i ^ {ascon_state_q[0], as-
con_state_q[1]}) & valid_bytes_bit_mask;
138
139 logic [127:0] data_in_padded;
140 logic [127:0] data_out_padded; // is only used internally for
decryption.
141
142 // For BOTH encryption AND decryption the PLAINTEXT is XORed
to the state!
143 // For encryption this is straight forward: S_r = S_r XOR P
144 // For decryption this means: S_r = S_r XOR P = S_r XOR (S_r
XOR C) = C
145 // Thus the ciphertext replaces the rate. However, we cannot
simply implement
146 // S_r = C, because this does not work for incomplete blocks,
where only a part
147 // of the rate is replaced by C and the other part remains:
148 // S_r = S_l || S_r(r-l) = C_l || S_r(r-l)
149 // Therefore, it is easier to always XOR the padded plaintext
150
151 always_comb begin
152 if (prim_mubi_pkg::mubi4_test_true_strict(complete_block))
begin
153 data_in_padded = data_in_valid_bytes;
154 data_out_padded = data_out;
155 end else begin
156 data_in_padded = data_in_valid_bytes | pad-
ding_byte_bit_mask;
157 data_out_padded = data_out | padding_byte_bit_mask;
158 end
159 end
160
161 // TODO add blinding
162 assign data_out_o = data_out;
163 assign tag_out_o = ({ascon_state_q[3], ascon_state_q[4]} ^
key_i);
164
165 // Due to Ascon's round constants the current_round
166 // contains an offset:
167 // for P12 we count: from 0 to 11 = 12 rounds,
168 // for P8 we count: from 4 to 11 = 8 rounds,
169 // for P6 we count: from 5 to 11 = 6 rounds
170 logic [AsconRoundCountW-1:0] current_round;
171
172 prim_count #(
173 .Width(AsconRoundCountW)
174 ) u_round_counter (
175 .clk_i,
176 .rst_ni,
177 .clr_i(1'b0),
178 .set_i(set_round_counter),
179 .set_cnt_i(perm_offset),
180 .incr_en_i(inc_round_counter),
181 .decr_en_i(1'b0),
182 .step_i(AsconRoundCountW'(1)),
183 .commit_i(1'b1),
184 .cnt_o(current_round),
185 .cnt_after_commit_o(),
186 .err_o(round_count_error)
187 );
188
189 // Duplexswitch muxes
190 // Selects data input source
191 padding_mux_e sel_padding;
192
193 // Main multiplexer per word
194 ascon_word_mux_e sel_mux_word0;
195 ascon_word_mux_e sel_mux_word1;
196 ascon_word_mux_e sel_mux_word2;

```

```

197 ascon_word_mux_e sel_mux_word3;
198 ascon_word_mux_e sel_mux_word4;
199
200 // Multiplexer before XOR
201 word_low_key_hi_mux_e sel_mux_key_word1;
202 key_hi_low_mux_e sel_mux_key_word2;
203 key_hi_low_mux_e sel_mux_key_word3;
204
205 // Selects round input
206 ascon_round_input_mux_e sel_round_input;
207
208 // Set domain separation
209 logic set_dom_sep;
210
211 // Intermediate signals
212 logic [4:0][63:0] xor_with_state;
213 logic [63:0] word4_dom_sep;
214 logic [127:0] data_to_duplex;
215
216 // Enc Dec Padding Mux
217 always_comb begin : Padding
218 unique case (sel_padding)
219 DATA_IN_PAD: data_to_duplex = data_in_padded;
220 DATA_OUT_PAD: data_to_duplex = data_out_padded;
221 EMPTY_PAD: data_to_duplex = empty_padding;
222 default: data_to_duplex = empty_padding;
223 endcase
224 end
225
226 // State input mux
227 always_comb begin : state_input
228 unique case (sel_round_input)
229 STATE: state_to_round = ascon_state_q;
230 BLINDING: state_to_round = '0;
231 default: state_to_round = '0;
232 endcase
233 end
234
235 // Main mux word0
236 assign xor_with_state[0] = data_to_duplex[127:64];
237 always_comb begin : state_word0
238 unique case (sel_mux_word0)
239 INIT: ascon_state_d[0] = iv;
240 ABSORB: ascon_state_d[0] = ascon_state_q[0] ^
xor_with_state[0];
241 KEEP: ascon_state_d[0] = ascon_state_q[0];
242 ROUND: ascon_state_d[0] = round_to_mux[0];
243 default: ascon_state_d[0] = ascon_state_q[0];
244 endcase
245 end
246
247 // Select key high or data low for XOR
248 always_comb begin : key_word1
249 unique case (sel_mux_key_word1)
250 WORD: xor_with_state[1] = data_to_duplex[63:0];
251 KEY: xor_with_state[1] = key_i[127:64];
252 default: xor_with_state[1] = key_i[127:64];
253 endcase
254 end
255
256 // Main mux word1
257 always_comb begin : state_word1
258 unique case (sel_mux_word1)
259 INIT: ascon_state_d[1] = key_i[127:64];
260 ABSORB: ascon_state_d[1] = ascon_state_q[1] ^
xor_with_state[1];
261 KEEP: ascon_state_d[1] = ascon_state_q[1];
262 ROUND: ascon_state_d[1] = round_to_mux[1];
263 default: ascon_state_d[1] = ascon_state_q[1];
264 endcase
265 end
266
267 // Select key high or low for XOR
268 always_comb begin : key_word2
269 unique case (sel_mux_key_word2)
270 KEY_LOW: xor_with_state[2] = key_i[63:0];
271 KEY_HI: xor_with_state[2] = key_i[127:64];
272 default: xor_with_state[2] = key_i[127:64];
273 endcase
274 end
275
276 // Main mux word2
277 always_comb begin : state_word2
278 unique case (sel_mux_word2)
279 INIT: ascon_state_d[2] = key_i[63:0];
280 ABSORB: ascon_state_d[2] = ascon_state_q[2] ^
xor_with_state[2];
281 KEEP: ascon_state_d[2] = ascon_state_q[2];
282 ROUND: ascon_state_d[2] = round_to_mux[2];
283 default: ascon_state_d[2] = ascon_state_q[2];
284 endcase
285 end
286
287 // Select key high or low for XOR
288 always_comb begin : key_word3
289 unique case (sel_mux_key_word3)
290 KEY_LOW: xor_with_state[3] = key_i[63:0];
291 KEY_HI: xor_with_state[3] = key_i[127:64];
292 default: xor_with_state[3] = key_i[127:64];
293 endcase
294 end

```

```

295
296 // Main mux word3
297 always_comb begin : state_word3
298     unique case (sel_mux_word3)
299         INIT: ascon_state_d[3] = nonce_i[127:64];
300         ABSORB: ascon_state_d[3] = ascon_state_q[3] ^
xor_with_state[3];
301         KEEP: ascon_state_d[3] = ascon_state_q[3];
302         ROUND: ascon_state_d[3] = round_to_mux[3];
303         default: ascon_state_d[3] = ascon_state_q[3];
304     endcase
305 end
306
307 // Set domain separation bit
308 assign word4_dom_sep = {ascon_state_q[4][63:1], as-
con_state_q[4][0] ^ set_dom_sep};
309 // For word 4 only the lower part of the key can be xored to
the state
310 assign xor_with_state[4] = key_i[63:0];
311
312 // Main mux word4
313 always_comb begin : state_word4
314     unique case (sel_mux_word4)
315         INIT: ascon_state_d[4] = nonce_i[63:0];
316         ABSORB: ascon_state_d[4] = word4_dom_sep ^
xor_with_state[4];
317         KEEP: ascon_state_d[4] = word4_dom_sep;
318         ROUND: ascon_state_d[4] = round_to_mux[4];
319         default: ascon_state_d[4] = word4_dom_sep;
320     endcase
321 end
322
323 always_comb begin : p_fsm
324     // Default assignments
325     fsm_state_d = fsm_state_q;
326     data_in_ready_o = 1'b0;
327     data_out_valid_o = 1'b0;
328     tag_out_valid_o = 1'b0;
329     sparse_fsm_error = 1'b0;
330     set_round_counter = 1'b0;
331     inc_round_counter = 1'b0;
332     perm_offset = P12;
333     done_o = 1'b0;
334     idle_o = prim_mubi_pkg::MuBi4False;
335
336     // Default: Don't update state
337     set_dom_sep = 1'b0;
338     sel_mux_word0 = KEEP;
339     sel_mux_word1 = KEEP;
340     sel_mux_word2 = KEEP;
341     sel_mux_word3 = KEEP;
342     sel_mux_word4 = KEEP;
343
344     sel_mux_key_word3 = KEY_HI;
345     sel_mux_key_word2 = KEY_LOW;
346     sel_mux_key_word1 = KEY;
347
348     sel_padding = EMPTY_PAD;
349
350     sel_round_input = BLINDING;
351
352     unique case (fsm_state_q)
353         Idle: begin
354             idle_o = prim_mubi_pkg::MuBi4True;
355             if (start_i) begin
356                 fsm_state_d = Init;
357             end
358         end
359         Init: begin
360             sel_mux_word0 = INIT;
361             sel_mux_word1 = INIT;
362             sel_mux_word2 = INIT;
363             sel_mux_word3 = INIT;
364             sel_mux_word4 = INIT;
365             fsm_state_d = PermInit;
366             perm_offset = P12;
367             set_round_counter = 1'b1;
368         end
369         PermInit: begin
370             sel_round_input = STATE;
371             sel_mux_word0 = ROUND;
372             sel_mux_word1 = ROUND;
373             sel_mux_word2 = ROUND;
374             sel_mux_word3 = ROUND;
375             sel_mux_word4 = ROUND;
376             if (current_round == ROUND_MAX) begin
377                 fsm_state_d = Xor0Key;
378             end else begin
379                 inc_round_counter = 1'b1;
380             end
381         end
382         Xor0Key: begin
383             sel_mux_word3 = ABSORB;
384             sel_mux_key_word3 = KEY_HI;
385             sel_mux_word4 = ABSORB;
386             if (prim_mubi_pkg::mubi4_test_true_strict(no_ad_i)) be-
gin
387                 fsm_state_d = XorDomSep;
388             end else begin
389                 fsm_state_d = AbsorbAD;
390             end
391         end
392     endcase
393 end

```

```

391 end
392 AbsorbAD: begin
393     // There will be AD, otherwise we wouldn't be in this
state/path
394     data_in_ready_o = 1'b1;
395     if (data_in_valid_i) begin
396         sel_mux_word0 = ABSORB;
397         sel_padding = DATA_IN_PAD;
398         if (ascon_variant == ASCON_128A) begin
399             sel_mux_word1 = ABSORB;
400             sel_mux_key_word1 = WORD;
401         end
402         if
(prim_mubi_pkg::mubi4_test_true_strict(last_block_ad_i)) begin
403             if
(prim_mubi_pkg::mubi4_test_true_strict(complete_block)) begin
404                 fsm_state_d = PermADEmpty;
405             end else begin
406                 fsm_state_d = PermADLast;
407             end
408         end else begin // there are more blocks to come
409             fsm_state_d = PermAD;
410         end
411     end
412     if (ascon_variant == ASCON_128) begin
413         perm_offset = P6;
414     end else begin // ASCON_128A
415         perm_offset = P8;
416     end
417     set_round_counter = 1'b1;
418 end
419 PermAD: begin
420     sel_round_input = STATE;
421     sel_mux_word0 = ROUND;
422     sel_mux_word1 = ROUND;
423     sel_mux_word2 = ROUND;
424     sel_mux_word3 = ROUND;
425     sel_mux_word4 = ROUND;
426     if (current_round == ROUND_MAX) begin
427         fsm_state_d = AbsorbAD;
428     end else begin
429         inc_round_counter = 1'b1;
430     end
431 end
432 PermADLast: begin
433     sel_round_input = STATE;
434     sel_mux_word0 = ROUND;
435     sel_mux_word1 = ROUND;
436     sel_mux_word2 = ROUND;
437     sel_mux_word3 = ROUND;
438     sel_mux_word4 = ROUND;
439     if (current_round == ROUND_MAX) begin
440         fsm_state_d = XorDomSep;
441     end else begin
442         inc_round_counter = 1'b1;
443     end
444 end
445 PermADEmpty: begin
446     sel_round_input = STATE;
447     sel_mux_word0 = ROUND;
448     sel_mux_word1 = ROUND;
449     sel_mux_word2 = ROUND;
450     sel_mux_word3 = ROUND;
451     sel_mux_word4 = ROUND;
452     if (current_round == ROUND_MAX) begin
453         fsm_state_d = AbsorbADEmpty;
454     end else begin
455         inc_round_counter = 1'b1;
456     end
457 end
458 AbsorbADEmpty: begin
459     sel_mux_word0 = ABSORB;
460     sel_padding = EMPTY_PAD;
461     fsm_state_d = PermADLast;
462     if (ascon_variant == ASCON_128) begin
463         perm_offset = P6;
464     end else begin // ASCON_128A
465         perm_offset = P8;
466         // This should be optimized by the tool.
467         // It is left here, so that the structure of the case
468         // is the same as AbsorbAD.
469         sel_mux_word1 = ABSORB;
470         sel_padding = EMPTY_PAD;
471     end
472     set_round_counter = 1'b1;
473 end
474 XorDomSep: begin
475     set_dom_sep = 1'b1;
476     sel_mux_word4 = KEEP;
477     if (prim_mubi_pkg::mubi4_test_true_strict(no_msg_i)) be-
gin
478         fsm_state_d = AbsorbMSGEmpty;
479     end else begin
480         fsm_state_d = AbsorbMSG;
481     end
482 end
483 AbsorbMSG: begin
484     data_in_ready_o = 1'b1;
485     if (data_in_valid_i) begin
486         data_out_valid_o = 1'b1;
487         if (ascon_operation == ASCON_ENC) begin

```

```

488     sel_mux_word0 = ABSORB;
489     sel_padding = DATA_IN_PAD;
490     if (ascon_variant == ASCON_128A) begin
491         sel_mux_word1 = ABSORB;
492     end
493 end else begin // ASCON_DEC
494     sel_mux_word0 = ABSORB;
495     sel_padding = DATA_OUT_PAD;
496     if (ascon_variant == ASCON_128A) begin
497         sel_mux_word1 = ABSORB;
498         sel_mux_key_word1 = WORD;
499     end
500 end
501 if
502 (prim_mubi_pkg::mubi4_test_true_strict(last_block_msg_i)) begin
503     if
504 (prim_mubi_pkg::mubi4_test_true_strict(complete_block)) begin // we
505 need extra padding
506         fsm_state_d = PermMSGEmpty;
507     end else begin // padding is done on the fly
508         fsm_state_d = XORKey0;
509     end
510 end else begin
511     fsm_state_d = PermMSG;
512 end
513 end
514 if (ascon_variant == ASCON_128) begin
515     perm_offset = P6;
516 end else begin //ASCON_128A
517     perm_offset = P8;
518 end
519 set_round_counter = 1'b1;
520 end
521 PermMSG: begin
522     sel_round_input = STATE;
523     sel_mux_word0 = ROUND;
524     sel_mux_word1 = ROUND;
525     sel_mux_word2 = ROUND;
526     sel_mux_word3 = ROUND;
527     sel_mux_word4 = ROUND;
528     if (current_round == ROUND_MAX) begin
529         fsm_state_d = AbsorbMSG;
530     end else begin
531         inc_round_counter = 1'b1;
532     end
533 end
534 PermMSGEmpty: begin
535     sel_round_input = STATE;
536     sel_mux_word0 = ROUND;
537     sel_mux_word1 = ROUND;
538     sel_mux_word2 = ROUND;
539     sel_mux_word3 = ROUND;
540     sel_mux_word4 = ROUND;
541     if (current_round == ROUND_MAX) begin
542         fsm_state_d = AbsorbMSGEmpty;
543     end else begin
544         inc_round_counter = 1'b1;
545     end
546 end
547 AbsorbMSGEmpty: begin
548     // The padding for an empty block is the same for en-
549 crypton and decryption.
550     sel_padding = EMPTY_PAD;
551     sel_mux_word0 = ABSORB;
552     if (ascon_variant == ASCON_128A) begin
553         // This should be optimized by the tool.
554         // It is left here, so that the structure of the case
555         // is the same as AbsorbAD.
556         sel_mux_word1 = ABSORB;
557         sel_mux_key_word1 = WORD;
558     end
559     fsm_state_d = XORKey0;
560 end
561 XORKey0: begin
562     if (ascon_variant == ASCON_128) begin
563         sel_mux_word1 = ABSORB;
564         sel_mux_key_word1 = KEY;
565         sel_mux_word2 = ABSORB;
566         sel_mux_key_word2 = KEY_LOW;
567     end else begin //ASCON_128a
568         sel_mux_word2 = ABSORB;
569         sel_mux_key_word2 = KEY_HI;
570         sel_mux_word3 = ABSORB;
571         sel_mux_key_word3 = KEY_LOW;
572     end
573     fsm_state_d = PermFinal;
574     set_round_counter = 1'b1;
575     perm_offset = P12;
576 end
577 PermFinal: begin
578     sel_round_input = STATE;
579     sel_mux_word0 = ROUND;
580     sel_mux_word1 = ROUND;
581     sel_mux_word2 = ROUND;
582     sel_mux_word3 = ROUND;
583     sel_mux_word4 = ROUND;
584     if (current_round == ROUND_MAX) begin
585         fsm_state_d = SqueezeTagXorKey;
586     end else begin
587         inc_round_counter = 1'b1;

```

```

585     end
586 end
587 SqueezeTagXorKey: begin
588     tag_out_valid_o = 1'b1;
589     fsm_state_d = Idle;
590     done_o = 1'b1;
591 end
592 Error: begin
593     fsm_state_d = Error;
594     sparse_fsm_error = 1'b1;
595 end
596 default: begin
597     fsm_state_d = Error;
598     sparse_fsm_error = 1'b1;
599 end
600 endcase
601 end
602
603 `PRIM_FLOP_SPARSE_FSM(u_state_regs, fsm_state_d, fsm_state_q,
604 duplex_fsm_state_e, Idle)
605
606 logic mubi_error;
607 assign mubi_error = prim_mubi_pkg::mubi4_test_invalid(no_ad_i)
608 |
609 prim_mubi_pkg::mubi4_test_invalid(no_msg_i)
610 |
611 prim_mubi_pkg::mubi4_test_invalid(complete_block)
612 |
613 prim_mubi_pkg::mubi4_test_invalid(last_block_ad_i)
614 |
615 prim_mubi_pkg::mubi4_test_invalid(last_block_msg_i);
616
617 assign err_o = round_count_error | sparse_fsm_error | mubi_er-
618 ror;
619
620 prim_ascon_round u_prim_ascon_round (
621     .state_o(state_from_round),
622     .state_i(state_to_round),
623     .rcon_i(get_ascon_rcon(current_round))
624 );
625
626 endmodule

```

10. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_pkg.sv (192 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5
6 package prim_ascon_pkg;
7
8 parameter int AsconRoundCountW = 4;
9
10 // Due to Ascon's round constants the current_round
11 // contains an offset:
12 // for P12 we count: from 0 to 11 = 12 rounds,
13 // for P8 we count: from 4 to 11 = 8 rounds,
14 // for P6 we count: from 5 to 11 = 6 rounds
15
16 typedef enum logic [3:0] {
17     P12 = 4'b0000,
18     P8 = 4'b0100,
19     P6 = 4'b0110
20 } perm_offset_e;
21
22 parameter int DUPLEX_OP_WIDTH = 3;
23
24 typedef enum logic [DUPLEX_OP_WIDTH-1:0] {
25     ASCON_ENC = 3'b001,
26     ASCON_DEC = 3'b010,
27     ASCON_HASH = 3'b100
28 } duplex_op_e;
29
30 parameter int DUPLEX_VARIANT_WIDTH = 2;
31
32 typedef enum logic [DUPLEX_VARIANT_WIDTH-1:0] {
33     ASCON_128 = 2'b01,
34     ASCON_128A = 2'b10
35 } duplex_variant_e;
36
37 localparam logic [63:0] IV_128 = 64'h80400c0600000000;
38 localparam logic [63:0] IV_128A = 64'h80800c0800000000;
39
40 localparam logic [3:0] ROUND_MAX = 4'b1011;
41
42 // Round constants
43 function automatic logic [7:0] get_ascon_rcon(logic [3:0]
44 round);
45     logic [7:0] result;
46     unique case (round)
47         4'b0000: result = 8'hf0;
48         4'b0001: result = 8'he1;
49         4'b0010: result = 8'hd2;
50         4'b0011: result = 8'hc3;
51         4'b0100: result = 8'hb4;
52         4'b0101: result = 8'ha5;

```

```

51 4'b0110: result = 8'h96;
52 4'b0111: result = 8'h87;
53 4'b1000: result = 8'h78;
54 4'b1001: result = 8'h69;
55 4'b1010: result = 8'h5a;
56 4'b1011: result = 8'h4b;
57 default: result = 8'h00;
58 endcase
59
60 return result;
61 endfunction
62
63 parameter int KEY_HI_LOW_MUX_WIDTH = 1;
64 typedef enum logic [KEY_HI_LOW_MUX_WIDTH-1:0] {
65   KEY_LOW = 1'b0,
66   KEY_HI  = 1'b1
67 } key_hi_low_mux_e;
68
69 parameter int WORD_LOW_KEY_HI_MUX_WIDTH = 1;
70 typedef enum logic [WORD_LOW_KEY_HI_MUX_WIDTH-1:0] {
71   WORD = 1'b0,
72   KEY  = 1'b1
73 } word_low_key_hi_mux_e;
74
75 parameter int ASCON_WORD_MUX_WIDTH = 2;
76 typedef enum logic [ASCON_WORD_MUX_WIDTH-1:0] {
77   INIT   = 2'b00,
78   ABSORB = 2'b01,
79   KEEP   = 2'b10,
80   ROUND  = 2'b11
81 } ascon_word_mux_e;
82
83 parameter int ROUND_INPUT_MUX_WIDTH = 1;
84 typedef enum logic [ROUND_INPUT_MUX_WIDTH-1:0] {
85   STATE = 1'b0,
86   BLINDING = 1'b1
87 } ascon_round_input_mux_e;
88
89 parameter int PADDING_MUX_WIDTH = 2;
90 typedef enum logic [PADDING_MUX_WIDTH-1:0] {
91   DATA_IN_PAD = 2'b00,
92   DATA_OUT_PAD = 2'b01,
93   EMPTY_PAD    = 2'b10
94 } padding_mux_e;
95
96 // Encoding generated with:
97 // $ ./util/design/sparse-fsm-encode.py -d 3 -m 18 -n 10 \
98 //   -s 3538518573 --language=sv
99 //
100 // Hamming distance histogram:
101 //
102 // 0: --
103 // 1: --
104 // 2: --
105 // 3: ||| (13.73%)
106 // 4: |||| (20.92%)
107 // 5: ||||| (24.84%)
108 // 6: ||||| (20.92%)
109 // 7: ||||| (14.38%)
110 // 8: ||| (5.23%)
111 // 9: --
112 // 10: --
113 //
114 // Minimum Hamming distance: 3
115 // Maximum Hamming distance: 8
116 // Minimum Hamming weight: 3
117 // Maximum Hamming weight: 8
118 //
119 localparam int AsconDuplexFSMStateWidth = 10;
120 typedef enum logic [AsconDuplexFSMStateWidth-1:0] {
121   Idle      = 10'b101010110,
122   Init      = 10'b0110000110,
123   PermInit  = 10'b1011110110,
124   Xor0Key   = 10'b0001010011,
125   AbsorbAD  = 10'b1110010000,
126   PermAD    = 10'b1101101101,
127   XorDomSep = 10'b0100110010,
128   AbsorbMSG = 10'b0011100011,
129   PermMSG   = 10'b0001001100,
130   AbsorbMSGEmpty = 10'b0100101100,
131   XorKey0   = 10'b1000011011,
132   PermFinal = 10'b0011010000,
133   SqueezeTagXorKey = 10'b1111010111,
134   PermADLast = 10'b1010000010,
135   AbsorbADEmpty = 10'b1101111011,
136   PermMSGEmpty = 10'b1011111000,
137   PermADEmpty = 10'b0010111100,
138   Error      = 10'b0100011110
139 } duplex_fsm_state_e;
140
141 function automatic logic [127:0] bin2thermo(logic [4:0] valid_bytes);
142   logic [127:0] valid_bytes_mask;
143   unique case (valid_bytes)
144     5'b00000 : valid_bytes_mask = {128{1'b0}};
145     5'b00001 : valid_bytes_mask = { 8{1'b1}}, {120{1'b0}};
146     5'b00010 : valid_bytes_mask = {16{1'b1}}, {112{1'b0}};
147     5'b00011 : valid_bytes_mask = {24{1'b1}}, {104{1'b0}};
148     5'b00100 : valid_bytes_mask = {32{1'b1}}, {96{1'b0}};
149     5'b00101 : valid_bytes_mask = {40{1'b1}}, {88{1'b0}};
150     5'b00110 : valid_bytes_mask = {48{1'b1}}, {80{1'b0}};

```

```

151 5'b00111 : valid_bytes_mask = {56{1'b1}}, {72{1'b0}};
152 5'b01000 : valid_bytes_mask = {64{1'b1}}, {64{1'b0}};
153 5'b01001 : valid_bytes_mask = {72{1'b1}}, {56{1'b0}};
154 5'b01010 : valid_bytes_mask = {80{1'b1}}, {48{1'b0}};
155 5'b01011 : valid_bytes_mask = {88{1'b1}}, {40{1'b0}};
156 5'b01100 : valid_bytes_mask = {96{1'b1}}, {32{1'b0}};
157 5'b01101 : valid_bytes_mask = {104{1'b1}}, {24{1'b0}};
158 5'b01110 : valid_bytes_mask = {112{1'b1}}, {16{1'b0}};
159 5'b01111 : valid_bytes_mask = {120{1'b1}}, {8{1'b0}};
160 default : valid_bytes_mask = {128{1'b1}};
161 endcase
162
163 return valid_bytes_mask;
164 endfunction
165
166 function automatic logic [127:0] get_padding_mask(logic [4:0]
valid_bytes);
167   logic [127:0] padding_byte_mask;
168   unique case (valid_bytes)
169     5'b00000 : padding_byte_mask = {8'h80,
170 {120{1'b0}}};
171     5'b00001 : padding_byte_mask = { 8{1'b0}}, 8'h80,
172 {112{1'b0}};
173     5'b00010 : padding_byte_mask = {16{1'b0}}, 8'h80,
174 {104{1'b0}};
175     5'b00011 : padding_byte_mask = {24{1'b0}}, 8'h80,
176 {96{1'b0}};
177     5'b00100 : padding_byte_mask = {32{1'b0}}, 8'h80,
178 {88{1'b0}};
179     5'b00101 : padding_byte_mask = {40{1'b0}}, 8'h80,
180 {80{1'b0}};
181     5'b00110 : padding_byte_mask = {48{1'b0}}, 8'h80,
182 {72{1'b0}};
183     5'b00111 : padding_byte_mask = {56{1'b0}}, 8'h80,
184 {64{1'b0}};
185     5'b01000 : padding_byte_mask = {64{1'b0}}, 8'h80,
186 {56{1'b0}};
187     5'b01001 : padding_byte_mask = {72{1'b0}}, 8'h80,
188 {48{1'b0}};
189     5'b01010 : padding_byte_mask = {80{1'b0}}, 8'h80,
190 {40{1'b0}};
191     5'b01011 : padding_byte_mask = {88{1'b0}}, 8'h80,
192 {32{1'b0}};
193     5'b01100 : padding_byte_mask = {96{1'b0}}, 8'h80,
194 {24{1'b0}};
195     5'b01101 : padding_byte_mask = {104{1'b0}}, 8'h80,
196 {16{1'b0}};
197     5'b01110 : padding_byte_mask = {112{1'b0}}, 8'h80,
198 {8{1'b0}};
199     5'b01111 : padding_byte_mask = {120{1'b0}},
200 8'h80 };
201     default : padding_byte_mask = {128{1'b0}};
202   endcase
203
204 return padding_byte_mask;
205 endfunction
206
207 endpackage : prim_ascon_pkg

```

11. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_round.sv (68 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Ascon round function implementation
6
7 module prim_ascon_round (
8   output logic [319:0] state_o,
9   input logic [319:0] state_i,
10  input logic [7:0] rcon_i
11 );
12
13 logic [319:0] ark_w, sbow_w;
14
15 // Add round constant
16 assign ark_w = state_i ^ {64'h0, 64'h0, 56'h0, rcon_i,
64'h0, 64'h0};
17
18 // Substitution layer
19 logic [63:0] x_w [5];
20 logic [4:0] xtranspose_w [64];
21 logic [4:0] ytranspose_w [64];
22 logic [63:0] y_w [5];
23
24 assign x_w[0] = ark_w[63:0];
25 assign x_w[1] = ark_w[127:64];
26 assign x_w[2] = ark_w[191:128];
27 assign x_w[3] = ark_w[255:192];
28 assign x_w[4] = ark_w[319:256];
29
30 for (genvar i = 0; i < 64; i = i + 1) begin : gen_s-
box_transpose
31   assign xtranspose_w[i] = {x_w[4][i],
32   x_w[3][i],

```



```

34         x_w[2][i],
35         x_w[1][i],
36         x_w[0][i]);
37     prim_ascon_sbox u_sbox (
38         .state_i(xtranspose_w[i]),
39         .state_o(ytranspose_w[i])
40     );
41     assign {y_w[4][i],
42            y_w[3][i],
43            y_w[2][i],
44            y_w[1][i],
45            y_w[0][i]} = ytranspose_w[i];
46 end
47
48 assign sbox_w = {y_w[4], y_w[3], y_w[2], y_w[1], y_w[0]};
49
50 // Linear layer
51 logic [63:0] xl_w [5];
52 logic [63:0] yl_w [5];
53
54 assign xl_w[0] = sbox_w[ 63: 0];
55 assign xl_w[1] = sbox_w[127: 64];
56 assign xl_w[2] = sbox_w[191:128];
57 assign xl_w[3] = sbox_w[255:192];
58 assign xl_w[4] = sbox_w[319:256];
59
60 assign yl_w[0] = xl_w[0] ^ {xl_w[0][18:0], xl_w[0][63:19]} ^
{xl_w[0][27:0], xl_w[0][63:28]};
61 assign yl_w[1] = xl_w[1] ^ {xl_w[1][60:0], xl_w[1][63:61]} ^
{xl_w[1][38:0], xl_w[1][63:39]};
62 assign yl_w[2] = xl_w[2] ^ {xl_w[2][ 0], xl_w[2][63: 1]} ^
{xl_w[2][ 5:0], xl_w[2][63: 6]};
63 assign yl_w[3] = xl_w[3] ^ {xl_w[3][ 9:0], xl_w[3][63:10]} ^
{xl_w[3][16:0], xl_w[3][63:17]};
64 assign yl_w[4] = xl_w[4] ^ {xl_w[4][ 6:0], xl_w[4][63: 7]} ^
{xl_w[4][40:0], xl_w[4][63:41]};
65
66 assign state_o = {yl_w[4], yl_w[3], yl_w[2], yl_w[1],
yl_w[0]};
67
68 endmodule

```

12. lowRISC_opentitan/hw/ip/prim/rtl/prim_ascon_sbox.sv (35 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Ascon SBox unprotected implementation
6
7 module prim_ascon_sbox (
8     output logic [4:0] state_o,
9     input  logic [4:0] state_i
10 );
11
12     logic [4:0] temp_w [4];
13
14     assign temp_w[0] = state_i;
15     assign temp_w[1] = {temp_w[0][3] ^ temp_w[0][4],
16                        temp_w[0][3],
17                        temp_w[0][1] ^ temp_w[0][2],
18                        temp_w[0][1],
19                        temp_w[0][0] ^ temp_w[0][4]
20                        };
21     assign temp_w[2] = {temp_w[1][4] ^ ((~temp_w[1][0]) &
temp_w[1][1]),
22                        temp_w[1][3] ^ ((~temp_w[1][4]) &
temp_w[1][0]),
23                        temp_w[1][2] ^ ((~temp_w[1][3]) &
temp_w[1][4]),
24                        temp_w[1][1] ^ ((~temp_w[1][2]) &
temp_w[1][3]),
25                        temp_w[1][0] ^ ((~temp_w[1][1]) &
temp_w[1][2])
26                        };
27     assign temp_w[3] = {temp_w[2][4],
28                        temp_w[2][3] ^ temp_w[2][2],
29                        ~temp_w[2][2],
30                        temp_w[2][1] ^ temp_w[2][0],
31                        temp_w[2][0] ^ temp_w[2][4]
32                        };
33     assign state_o = temp_w[3];
34
35 endmodule

```

13. lowRISC_opentitan/hw/ip/prim/rtl/prim_assert.sv (190 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.

```

```

3 // SPDX-License-Identifier: Apache-2.0
4
5 // Macros and helper code for using assertions.
6 // - Provides default clk and rst options to simplify code
7 // - Provides boiler plate template for common assertions
8
9 `ifndef PRIM_ASSERT_SV
10 `define PRIM_ASSERT_SV
11
12 // Helper macros //
13 // Helper macros //
14 // Helper macros //
15
16 // Default clk and reset signals used by assertion macros be-
low.
17 `define ASSERT_DEFAULT_CLK clk_i
18 `define ASSERT_DEFAULT_RST !rst_ni
19
20 // Converts an arbitrary block of code into a Verilog string
21 `define PRIM_STRINGIFY(x) "`x"
22
23 // ASSERT_ERROR logs an error message with either `uvm_error
or with $error.
24 //
25 // This somewhat duplicates `DV_ERROR macro defined in hw/dv/
sv/dv_utils/dv_macros.svh. The reason
26 // for redefining it here is to avoid creating a dependency.
27 `define AS-
SERT_ER-
ROR(____
name) \
28 `ifdef
UVM
\
29     uvm_pkg::uvm_report_error("ASSERT FAILED", `PRIM_STRINGI-
FY(__name), uvm_pkg::UVM_NONE, \
30         `__FILE__, `__LINE__, "",
31         1);
32 `else
\
33     $error("%0t: (%0s:%0d) [%m] [ASSERT FAILED] %0s", $time,
`__FILE__, `__LINE__, \
34         `PRIM_STRINGI-
FY(__name));
35 `endif
36
37 // This macro is suitable for conditionally triggering lint
errors, e.g., if a Sec parameter takes
38 // on a non-default value. This may be required for pre-silic-
on/FPGA evaluation but we don't want
39 // to allow this for tapeout.
40 `define ASSERT_STATIC_LINT_ERROR(__name, __prop) \
41     localparam int __name = (__prop) ? 1 : 2; \
42     always_comb begin \
43         logic unused_assert_static_lint_error; \
44         unused_assert_static_lint_error = __name'(1'b1); \
45     end
46
47 // Static assertions for checks inside SV packages. If the
conditions is not true, this will
48 // trigger an error during elaboration.
49 `define ASSERT_STATIC_IN_PACKAGE(__name, __prop)
\
49     function automatic bit assert_static_in_package_`__name();
\
50         bit unused_bit [((__prop) ? 1 : -1)];
\
51         unused_bit = '{default: 1'b0};
\
52         return unused_bit[0];
\
53     endfunction
54
55 // The basic helper macros are actually defined in "implement-
ation headers". The macros should do
56 // the same thing in each case (except for the dummy flavour),
but in a way that the respective
57 // tools support.
58 //
59 // If the tool supports assertions in some form, we also
define INC_ASSERT (which can be used to
60 // hide signal definitions that are only used for assertions).
61 //
62 // The list of basic macros supported is:
63 //
64 // ASSERT_I: Immediate assertion. Note that immediate as-
sertions are sensitive to simulation
65 // glitches.
66 //
67 // ASSERT_INIT: Assertion in initial block. Can be used for
things like parameter checking.
68 //
69 // ASSERT_INIT_NET: Assertion in initial block. Can be used
for initial value of a net.
70 //
71 // ASSERT_FINAL: Assertion in final block. Can be used for
things like queues being empty at end of
72 // sim, all credits returned at end of sim,
state machines in idle at end of sim.

```

```

73 //
74 // ASSERT_AT_RESET: Assertion just before reset. Can be used
to check sum-like properties that get
75 // cleared at reset.
76 // Note that unless your simulation ends
with a reset, the property does not get
77 // checked at end of simulation; use AS-
SERT_AT_RESET_AND_FINAL if the property
78 // should also get checked at end of simula-
tion.
79 //

80 // ASSERT_AT_RESET_AND_FINAL: Assertion just before reset and in
final block. Can be used to check
81 // sum-like properties before
every reset and at the end of simulation.
82 //
83 // ASSERT: Assert a concurrent property directly. It
can be called as a module (or
84 // interface) body item.
85 //
86 // Note: We use (__rst != '0') in the disable
iff statements instead of (__rst ==

87 // '1). This properly disables the assertion in
cases when reset is X at the
88 // beginning of a simulation. For that case,
(reset == '1) does not disable the
89 // assertion.
90 //
91 // ASSERT_NEVER: Assert a concurrent property NEVER happens
92 //
93 // ASSERT_KNOWN: Assert that signal has a known value (each
bit is either '0' or '1') after reset.
94 // It can be called as a module (or interface)
body item.
95 //
96 // COVER: Cover a concurrent property
97 //
98 // ASSUME: Assume a concurrent property
99 //
100 // ASSUME_I: Assume an immediate property
101 //
102 `ifdef VERILATOR
103 `include "prim_assert_dummy_macros.svh"
104 `elsif SYNTHESIS
105 `include "prim_assert_dummy_macros.svh"
106 `elsif YOSYS
107 `include "prim_assert_yosys_macros.svh"
108 `define INC_ASSERT
109 `else
110 `include "prim_assert_standard_macros.svh"
111 `define INC_ASSERT
112 `endif
113
114 //////////////////////////////////////////////////
115 // Complex assertion macros //
116 //////////////////////////////////////////////////
117
118 // Assert that signal is an active-high pulse with pulse
length of 1 clock cycle
119 `define ASSERT_PULSE(__name, __sig, __clk = `AS-
SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
120 `ASSERT(__name, $rose(__sig) | => !(__sig), __clk, __rst)
121
122 // Assert that a property is true only when an enable signal
is set. It can be called as a module
123 // (or interface) body item.
124 `define ASSERT_IF(__name, __prop, __enable, __clk = `AS-
SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
125 `ASSERT(__name, (__enable) | -> (__prop), __clk, __rst)
126
127 // Assert that signal has a known value (each bit is either
'0' or '1') after reset is known
128 // set. It can be called as a module (or interface) body
item.
129 `define ASSERT_KNOWN_IF(__name, __sig, __enable, __clk = `AS-
SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
130 `ifdef
FPV_ON
131 `ASSERT_KNOWN(__name`KnownEnable, __enable, __clk,
__rst)
132 `ASSERT_IF(__name, !$isunknown(__sig), __enable, __clk,
__rst)
133 `endif
134
135 //////////////////////////////////////////////////
136 // For formal verification only //
137 //////////////////////////////////////////////////
138
139 // Note that the existing set of ASSERT macros specified above
shall be used for FPV,
140 // thereby ensuring that the assertions are evaluated during
DV simulations as well.
141
142 // ASSUME_FPV
143 // Assume a concurrent property during formal verification
only.
144 `define ASSUME_FPV(__name, __prop, __clk = `AS-

```

```

SERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
145 `ifdef
FPV_ON
146 `ASSUME(__name, __prop, __clk,
__rst)
147 `endif
148
149 // ASSUME_I_FPV
150 // Assume a concurrent property during formal verification
only.
151 `define ASSUME_I_FPV(__name, __prop) \
152 `ifdef FPV_ON
153 `ASSUME_I(__name, __prop)
154 `endif
155
156 // COVER_FPV
157 // Cover a concurrent property during formal verification

158 `define COVER_FPV(__name, __prop, __clk = `ASSERT_DEFAULT_CLK,
__rst = `ASSERT_DEFAULT_RST) \
159 `ifdef
FPV_ON
160 `COVER(__name, __prop, __clk,
__rst)
161 `endif
162
163 // FPV assertion that proves that the FSM control flow is lin-
ear (no loops)

164 // The sequence triggers whenever the state changes and stores
the current state as "initial_state".
165 // Then thereafter we must never see that state again until
reset.
166 // It is possible for the reset to release ahead of the clock.
167 // Create a small "gray" window beyond the usual rst time to
avoid
168 // checking.

169 `define ASSERT_FPV_LINEAR_FSM(__name, __state, __type, __clk =
`ASSERT_DEFAULT_CLK, __rst = `ASSERT_DEFAULT_RST) \
170 `ifdef
INC_ASSERT
171 bit __-
name`__cond;
172
173 always_ff @(posedge __clk or posedge __rst) be-
gin
174 if (__rst) be-
gin
175 __name`__cond <=
0;
176
177 end else be-
gin
178 __name`__cond <=
1;
179
180 end
181
182 property __-
name`__p;
183
184 __type ini-
tial_state;
185
186 ($stable(__state) & __name`__cond, initial_state =
$past(__state)) | ->
187 (__state != initial_state) until !(__-
name`__cond);
188
189 endprop-
erty
190
191 `ASSERT(__name, __name`__p, __clk,
0)
192
193 `endif
194
195 `include "prim_assert_sec_cm.svh"
196 `include "prim_flop_macros.sv"
197
198 `endif // PRIM_ASSERT_SV

```

14. lowRISC_opentitan/hw/ip/prim/rtl/prim_blanker.sv (20 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 // Convenience module for wrapping prim_and2 for use in blank-

```



```

ing.
6 // When en_i == 1 the input is fed through to the output.
7 // When en_i == 0 the output is 0.
8 module prim_blanker #(
9     parameter int Width = 1
10 ) (
11     input logic [Width-1:0] in_i,
12     input logic en_i,
13     output logic [Width-1:0] out_o
14 );
15     prim_and2 #(.Width(Width)) u_blank_and (
16         .in0_i(in_i),
17         .in1_i({Width{en_i}}),
18         .out_o
19     );
20 endmodule

```

15. lowRISC_opentitan/hw/ip/prim/rtl/prim_cdc_rand_delay.sv (67 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module should be instantiated within flops of CDC syn-
  chronization primitives,
6 // and allows DV to model real CDC delays within simulations,
  especially useful at the chip level
7 // or in IPs that communicate across clock domains.
8 //
9 // The instrumentation is very simple: when this is enabled
  the input into the first
10 // synchronizer flop has a mux where the select is randomly
  set. One of the mux inputs is the input
11 // of this module, and the other is the output of the first
  flop: selecting the latter models the
12 // effect of the first flop missing the input transition.
13 //
14 // Notice the delay should cause the input to be skipped by at
  most a single cycle. As a perhaps
15 // unnecessary precaution, the select will only be allowed to
  be random when the input changes.
16
17 module prim_cdc_rand_delay #(
18     parameter int DataWidth = 1,
19     parameter bit Enable = 1
20 ) (
21     input logic clk_i,
22     input logic rst_ni,
23     input logic [DataWidth-1:0] prev_data_i,
24     input logic [DataWidth-1:0] src_data_i,
25     output logic [DataWidth-1:0] dst_data_o
26 );
27 `ifdef SIMULATION
28     if (Enable) begin : gen_enable
29
30         // This controls dst_data_o: any bit with its data_sel set
        uses prev_data_i, others use
31         // src_data_i.
32         bit [DataWidth-1:0] data_sel;
33         bit cdc_instrumentation_enabled;
34
35         function automatic bit [DataWidth-1:0] fast_randomize();
36         bit [DataWidth-1:0] data;
37         if (DataWidth <= 32) begin
38             data = $urandom();
39         end else begin
40             if (!std::randomize(data)) $fatal(1, "%t: [%m] Failed
        to randomize data", $time);
41             end
42             return data;
43         endfunction
44
45         initial begin
46             void'($value$plusargs("cdc_instrumentation_enabled=%d",
        cdc_instrumentation_enabled));
47             data_sel = '0;
48             end
49
50         // Set data_sel at random combinationally when the input
        changes.
51         always @(src_data_i) begin
52             data_sel = cdc_instrumentation_enabled ? fast_random-
        ize() : 0;
53         end
54
55         // Clear data_sel on any cycle start.
56         always @(posedge clk_i or negedge rst_ni) begin
57             data_sel <= 0;
58         end
59
60         always_comb dst_data_o = (prev_data_i & data_sel) |
        (src_data_i & ~data_sel);
61     end else begin : gen_no_enable
62         assign dst_data_o = src_data_i;
63     end

```

```

64 `else // SIMULATION
65     assign dst_data_o = src_data_i;
66 `endif // SIMULATION
67 endmodule

```

16. lowRISC_opentitan/hw/ip/prim/rtl/prim_cipher_pkg.sv (397 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This package holds common constants and functions for
  PRESENT- and
6 // PRINCE-based scrambling devices.
7 //
8 // See also: prim_present, prim_prince
9 //
10 // References: - https://en.wikipedia.org/wiki/PRESENT
11 //              - https://en.wikipedia.org/wiki/Prince_(cipher)
12 //              - http://www.lightweightcrypto.org/present/
  present_ches2007.pdf
13 //              - https://eprint.iacr.org/2012/529.pdf
14 //              - https://eprint.iacr.org/2015/372.pdf
15 //              - https://eprint.iacr.org/2014/656.pdf
16
17 package prim_cipher_pkg;
18
19 ///////////////////////////////////////////////////
20 // PRINCE Cipher //
21 ///////////////////////////////////////////////////
22
23     parameter logic [15:0][3:0] PRINCE_SBOX4 = {4'h4, 4'hD,
4'h5, 4'hE,
24
25         4'h0, 4'h8,
4'h7, 4'h6,
26
27         4'h1, 4'h9,
4'hC, 4'hA,
28
29         4'h2, 4'h3,
4'hF, 4'hB};
29
30     parameter logic [15:0][3:0] PRINCE_SBOX4_INV = {4'h1, 4'hC,
4'hE, 4'h5,
31
32         4'h0, 4'h4,
4'h6, 4'hA,
33
34         4'h9, 4'h8,
4'hD, 4'hF,
35
36         4'h2, 4'h3,
4'h7, 4'hB};
36
37     // nibble permutations
38     parameter logic [15:0][3:0] PRINCE_SHIFT_ROWS64 = '{4'hF,
4'hA, 4'h5, 4'h0,
39
40         4'hB,
4'h6, 4'h1, 4'hC,
41
42         4'h7,
4'h2, 4'hD, 4'h8,
43
44         4'h3,
4'hE, 4'h9, 4'h4};
44
45     parameter logic [15:0][3:0] PRINCE_SHIFT_ROWS64_INV =
        '{4'hF, 4'h2, 4'h5, 4'h8,
46
47         4'hB, 4'hE, 4'h1, 4'h4,
48
49         4'h7, 4'hA, 4'hD, 4'h0,
50
51         4'h3, 4'h6, 4'h9, 4'hC};
51
52     // these are the round constants
53     parameter logic [11:0][63:0] PRINCE_ROUND_CONST =
        {64'hC0AC29B7C97C50DD,
54
55         64'hD3B5A399CA0C2399,
56
57         64'h64A51195E0E3610D,
58
59         64'hC882D32F25323C54,
60
61         64'h85840851F1AC43AA,
62
63         64'h7E-
        F84F78FD955CB1,
64
65         64'hBE5466CF34E90C6C,
66
67         64'h452821E638D01377,
68
69         64'h082E-
        FA98EC4E6C89,
70
71         64'hA4093822299F31D0,
72
73         64'h13198A2E03707344,
74
75         64'h0000000000000000};
75
76     // tweak constant for key modification between enc/dec modes
77     parameter logic [63:0] PRINCE_ALPHA_CONST =
        64'hC0AC29B7C97C50DD;
78

```

```

60 // masking constants for shift rows function below
61 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST0 = 16'h7BDE;
62 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST1 = 16'hBDE7;
63 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST2 = 16'hDE7B;
64 parameter logic [15:0] PRINCE_SHIFT_ROWS_CONST3 = 16'hE7BD;
65
66 // nibble shifts
67
68 function automatic logic [31:0] prince_shiftrows_32bit(logic
[31:0] state_in,
69                                     logic
[15:0][3:0] shifts );
70     logic [31:0] state_out;
71     // note that if simulation performance becomes an issue,
this loop can be unrolled
72     for (int k = 0; k < 32/2; k++) begin
73         // operate on pairs of 2bit instead of nibbles
74         state_out[k*2 +: 2] = state_in[shifts[k]*2 +: 2];
75     end
76     return state_out;
77 endfunction : prince_shiftrows_32bit
78
79 function automatic logic [63:0] prince_shiftrows_64bit(logic
[63:0] state_in,
80                                     logic
[15:0][3:0] shifts );
81     logic [63:0] state_out;
82     // note that if simulation performance becomes an issue,
this loop can be unrolled
83     for (int k = 0; k < 64/4; k++) begin
84         state_out[k*4 +: 4] = state_in[shifts[k]*4 +: 4];
85     end
86     return state_out;
87 endfunction : prince_shiftrows_64bit
88
89 // XOR reduction of four nibbles in a 16bit subvector
90 function automatic logic [3:0] prince_nibble_red16(logic
[15:0] vect);
91     return vect[0 +: 4] ^ vect[4 +: 4] ^ vect[8 +: 4] ^
vect[12 +: 4];
92 endfunction : prince_nibble_red16
93
94 // M prime multiplication
95 function automatic logic [31:0]
prince_mult_prime_32bit(logic [31:0] state_in);
96     logic [31:0] state_out;
97     // M0
98     state_out[0 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
99     state_out[4 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
100    state_out[8 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
101    state_out[12 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
102    // M1
103    state_out[16 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
104    state_out[20 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
105    state_out[24 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
106    state_out[28 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
107    return state_out;
108 endfunction : prince_mult_prime_32bit
109
110 // M prime multiplication
111 function automatic logic [63:0]
prince_mult_prime_64bit(logic [63:0] state_in);
112     logic [63:0] state_out;
113     // M0
114     state_out[0 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
115     state_out[4 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
116     state_out[8 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
117     state_out[12 +: 4] = prince_nibble_red16(state_in[ 0 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
118    // M1
119    state_out[16 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
120    state_out[20 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
121    state_out[24 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
122    state_out[28 +: 4] = prince_nibble_red16(state_in[16 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
123    // M1
124    state_out[32 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
125    state_out[36 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
126    state_out[40 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
127    state_out[44 +: 4] = prince_nibble_red16(state_in[32 +:
16] & PRINCE_SHIFT_ROWS_CONST1);

```

```

127 // M0
128 state_out[48 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST3);
129 state_out[52 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST2);
130 state_out[56 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST1);
131 state_out[60 +: 4] = prince_nibble_red16(state_in[48 +:
16] & PRINCE_SHIFT_ROWS_CONST0);
132 return state_out;
133 endfunction : prince_mult_prime_64bit
134
135
136 // PRESENT Cipher
137
138 // this is the sbox from the present cipher
139 parameter logic [15:0][3:0] PRESENT_SBOX4 = {4'h2, 4'h1,
4'h7, 4'h4,
140                                     4'h8, 4'hF,
4'hE, 4'h3,
141                                     4'hD, 4'hA,
4'h0, 4'h9,
142                                     4'hB, 4'h6,
4'h5, 4'hC};
143
144 parameter logic [15:0][3:0] PRESENT_SBOX4_INV = {4'hA, 4'h9,
4'h7, 4'h0,
145                                     4'h3, 4'h6,
4'h4, 4'hB,
146                                     4'hD, 4'h2,
4'h1, 4'hC,
147                                     4'h8, 4'hF,
4'hE, 4'h5};
148
149 // these are modified permutation indices for a 32bit ver-
sion that
150 // follow the same pattern as for the 64bit version
151 parameter logic [31:0][4:0] PRESENT_PERM32 = {5'd31, 5'd23,
5'd15, 5'd07,
152                                     5'd30, 5'd22,
5'd14, 5'd06,
153                                     5'd29, 5'd21,
5'd13, 5'd05,
154                                     5'd28, 5'd20,
5'd12, 5'd04,
155                                     5'd27, 5'd19,
5'd11, 5'd03,
156                                     5'd26, 5'd18,
5'd10, 5'd02,
157                                     5'd25, 5'd17,
5'd09, 5'd01,
158                                     5'd24, 5'd16,
5'd08, 5'd00};
159
160 parameter logic [31:0][4:0] PRESENT_PERM32_INV = {5'd31,
5'd27, 5'd23, 5'd19,
161                                     5'd15,
5'd11, 5'd07, 5'd03,
162                                     5'd30,
5'd26, 5'd22, 5'd18,
163                                     5'd14,
5'd10, 5'd06, 5'd02,
164                                     5'd29,
5'd25, 5'd21, 5'd17,
165                                     5'd13,
5'd09, 5'd05, 5'd01,
166                                     5'd28,
5'd24, 5'd20, 5'd16,
167                                     5'd12,
5'd08, 5'd04, 5'd00};
168
169 // these are the permutation indices of the present cipher
170 parameter logic [63:0][5:0] PRESENT_PERM64 = {6'd63, 6'd47,
6'd31, 6'd15,
171                                     6'd62, 6'd46,
6'd30, 6'd14,
172                                     6'd61, 6'd45,
6'd29, 6'd13,
173                                     6'd60, 6'd44,
6'd28, 6'd12,
174                                     6'd59, 6'd43,
6'd27, 6'd11,
175                                     6'd58, 6'd42,
6'd26, 6'd10,
176                                     6'd57, 6'd41,
6'd25, 6'd09,
177                                     6'd56, 6'd40,
6'd24, 6'd08,
178                                     6'd55, 6'd39,
6'd23, 6'd07,
179                                     6'd54, 6'd38,
6'd22, 6'd06,
180                                     6'd53, 6'd37,
6'd21, 6'd05,
181                                     6'd52, 6'd36,
6'd20, 6'd04,
182                                     6'd51, 6'd35,
6'd19, 6'd11,
183                                     6'd50, 6'd34,
6'd18, 6'd10,
184                                     6'd49, 6'd33,
6'd17, 6'd09,
185                                     6'd48, 6'd32,
6'd16, 6'd08,
186                                     6'd47, 6'd31,
6'd15, 6'd07,
187                                     6'd46, 6'd30,
6'd14, 6'd06,
188                                     6'd45, 6'd29,
6'd13, 6'd05,
189                                     6'd44, 6'd28,
6'd12, 6'd04,
190                                     6'd43, 6'd27,
6'd11, 6'd03,
191                                     6'd42, 6'd26,
6'd10, 6'd02,
192                                     6'd41, 6'd25,
6'd09, 6'd01,
193                                     6'd40, 6'd24,
6'd08, 6'd00};

```

```

6'd20, 6'd04,
184
6'd19, 6'd03,
185
6'd18, 6'd02,
186
6'd17, 6'd01,
187
6'd16, 6'd00;
188
189 parameter logic [63:0][5:0] PRESENT_PERM64_INV = {6'd63,
6'd59, 6'd55, 6'd51,
190
6'd43, 6'd39, 6'd35,
191
6'd27, 6'd23, 6'd19,
192
6'd11, 6'd07, 6'd03,
193
6'd58, 6'd54, 6'd50,
194
6'd42, 6'd38, 6'd34,
195
6'd26, 6'd22, 6'd18,
196
6'd10, 6'd06, 6'd02,
197
6'd57, 6'd53, 6'd49,
198
6'd41, 6'd37, 6'd33,
199
6'd25, 6'd21, 6'd17,
200
6'd09, 6'd05, 6'd01,
201
6'd56, 6'd52, 6'd48,
202
6'd40, 6'd36, 6'd32,
203
6'd24, 6'd20, 6'd16,
204
6'd08, 6'd04, 6'd00};
205
206 // forward key schedule
207 function automatic logic [63:0] present_update_key64(logic
[63:0] key_in,
208
logic
[4:0] round_idx);
209 logic [63:0] key_out;
210 // rotate by 61 to the left
211 key_out = {key_in[63-61:0], key_in[63:64-61]};
212 // sbox on uppermost 4 bits
213 key_out[63 -: 4] = PRESENT_SBOX4[key_out[63 -: 4]];
214 // xor in round counter on bits 19 to 15
215 key_out[19:15] ^= round_idx;
216 return key_out;
217 endfunction : present_update_key64
218
219 function automatic logic [79:0] present_update_key80(logic
[79:0] key_in,
220
logic
[4:0] round_idx);
221 logic [79:0] key_out;
222 // rotate by 61 to the left
223 key_out = {key_in[79-61:0], key_in[79:80-61]};
224 // sbox on uppermost 4 bits
225 key_out[79 -: 4] = PRESENT_SBOX4[key_out[79 -: 4]];
226 // xor in round counter on bits 19 to 15
227 key_out[19:15] ^= round_idx;
228 return key_out;
229 endfunction : present_update_key80
230
231 function automatic logic [127:0] present_update_key128(logic
[127:0] key_in,
232
logic
[4:0] round_idx);
233 logic [127:0] key_out;
234 // rotate by 61 to the left
235 key_out = {key_in[127-61:0], key_in[127:128-61]};
236 // sbox on uppermost 4 bits
237 key_out[127 -: 4] = PRESENT_SBOX4[key_out[127 -: 4]];
238 // sbox on second nibble from top
239 key_out[123 -: 4] = PRESENT_SBOX4[key_out[123 -: 4]];
240 // xor in round counter on bits 66 to 62
241 key_out[66:62] ^= round_idx;
242 return key_out;
243 endfunction : present_update_key128
244
245
246 // inverse key schedule
247 function automatic logic [63:0]
present_inv_update_key64(logic [63:0] key_in,
248
lo-
gic [4:0] round_idx);
249 logic [63:0] key_out = key_in;
250 // xor in round counter on bits 19 to 15
251 key_out[19:15] ^= round_idx;
252 // sbox on uppermost 4 bits
253 key_out[63 -: 4] = PRESENT_SBOX4_INV[key_out[63 -: 4]];

```

```

254 // rotate by 61 to the right
255 key_out = {key_out[60:0], key_out[63:61]};
256 return key_out;
257 endfunction : present_inv_update_key64
258
259 function automatic logic [79:0]
present_inv_update_key80(logic [79:0] key_in,
260
lo-
gic [4:0] round_idx);
261 logic [79:0] key_out = key_in;
262 // xor in round counter on bits 19 to 15
263 key_out[19:15] ^= round_idx;
264 // sbox on uppermost 4 bits
265 key_out[79 -: 4] = PRESENT_SBOX4_INV[key_out[79 -: 4]];
266 // rotate by 61 to the right
267 key_out = {key_out[60:0], key_out[79:61]};
268 return key_out;
269 endfunction : present_inv_update_key80
270
271 function automatic logic [127:0]
present_inv_update_key128(logic [127:0] key_in,
272
logic [4:0] round_idx);
273 logic [127:0] key_out = key_in;
274 // xor in round counter on bits 66 to 62
275 key_out[66:62] ^= round_idx;
276 // sbox on second highest nibble
277 key_out[123 -: 4] = PRESENT_SBOX4_INV[key_out[123 -: 4]];
278 // sbox on uppermost 4 bits
279 key_out[127 -: 4] = PRESENT_SBOX4_INV[key_out[127 -: 4]];
280 // rotate by 61 to the right
281 key_out = {key_out[60:0], key_out[127:61]};
282 return key_out;
283 endfunction : present_inv_update_key128
284
285
286 // these functions can be used to derive the DEC key from
the ENC key by
287 // stepping the key by the correct number of rounds using
the keyschedule functions above.
288 function automatic logic [63:0] present_get_dec_key64(logic
[63:0] key_in,
289
//
total number of rounds employed
290
logic
[4:0] round_cnt);
291 logic [63:0] key_out;
292 key_out = key_in;
293 for (int unsigned k = 0; k < round_cnt; k++) begin
294 key_out = present_update_key64(key_out, 5'(k + 1));
295 end
296 return key_out;
297 endfunction : present_get_dec_key64
298
299 function automatic logic [79:0] present_get_dec_key80(logic
[79:0] key_in,
300
//
total number of rounds employed
301
logic
[4:0] round_cnt);
302 logic [79:0] key_out;
303 key_out = key_in;
304 for (int unsigned k = 0; k < round_cnt; k++) begin
305 key_out = present_update_key80(key_out, 5'(k + 1));
306 end
307 return key_out;
308 endfunction : present_get_dec_key80
309
310 function automatic logic [127:0]
present_get_dec_key128(logic [127:0] key_in,
311
//
total number of rounds employed
312
lo-
gic [4:0] round_cnt);
313 logic [127:0] key_out;
314 key_out = key_in;
315 for (int unsigned k = 0; k < round_cnt; k++) begin
316 key_out = present_update_key128(key_out, 5'(k + 1));
317 end
318 return key_out;
319 endfunction : present_get_dec_key128
320
321 ///////////////////////////////////////////////////////////////////
322 // Common Subfunctions //
323 ///////////////////////////////////////////////////////////////////
324
325 function automatic logic [7:0] sbox4_8bit(logic [7:0]
state_in, logic [15:0][3:0] sbox4);
326 logic [7:0] state_out;
327 // note that if simulation performance becomes an issue,
this loop can be unrolled
328 for (int k = 0; k < 8/4; k++) begin
329 state_out[k*4 +: 4] = sbox4[state_in[k*4 +: 4]];
330 end
331 return state_out;
332 endfunction : sbox4_8bit
333
334 function automatic logic [15:0] sbox4_16bit(logic [15:0]
state_in, logic [15:0][3:0] sbox4);
335 logic [15:0] state_out;
336 // note that if simulation performance becomes an issue,

```

```

this loop can be unrolled
337   for (int k = 0; k < 2; k++) begin
338       state_out[k*8 +: 8] = sbox4_8bit(state_in[k*8 +: 8],
sbox4);
339   end
340   return state_out;
341   endfunction : sbox4_16bit
342
343   function automatic logic [31:0] sbox4_32bit(logic [31:0]
state_in, logic [15:0][3:0] sbox4);
344       logic [31:0] state_out;
345       // note that if simulation performance becomes an issue,
this loop can be unrolled
346       for (int k = 0; k < 4; k++) begin
347           state_out[k*8 +: 8] = sbox4_8bit(state_in[k*8 +: 8],
sbox4);
348       end
349       return state_out;
350   endfunction : sbox4_32bit
351
352   function automatic logic [63:0] sbox4_64bit(logic [63:0]
state_in, logic [15:0][3:0] sbox4);
353       logic [63:0] state_out;
354       // note that if simulation performance becomes an issue,
this loop can be unrolled
355       for (int k = 0; k < 8; k++) begin
356           state_out[k*8 +: 8] = sbox4_8bit(state_in[k*8 +: 8],
sbox4);
357       end
358       return state_out;
359   endfunction : sbox4_64bit
360
361   function automatic logic [7:0] perm_8bit(logic [7:0]
state_in, logic [7:0][2:0] perm);
362       logic [7:0] state_out;
363       // note that if simulation performance becomes an issue,
this loop can be unrolled
364       for (int k = 0; k < 8; k++) begin
365           state_out[perm[k]] = state_in[k];
366       end
367       return state_out;
368   endfunction : perm_8bit
369
370   function automatic logic [15:0] perm_16bit(logic [15:0]
state_in, logic [15:0][3:0] perm);
371       logic [15:0] state_out;
372       // note that if simulation performance becomes an issue,
this loop can be unrolled
373       for (int k = 0; k < 16; k++) begin
374           state_out[perm[k]] = state_in[k];
375       end
376       return state_out;
377   endfunction : perm_16bit
378
379   function automatic logic [31:0] perm_32bit(logic [31:0]
state_in, logic [31:0][4:0] perm);
380       logic [31:0] state_out;
381       // note that if simulation performance becomes an issue,
this loop can be unrolled
382       for (int k = 0; k < 32; k++) begin
383           state_out[perm[k]] = state_in[k];
384       end
385       return state_out;
386   endfunction : perm_32bit
387
388   function automatic logic [63:0] perm_64bit(logic [63:0]
state_in, logic [63:0][5:0] perm);
389       logic [63:0] state_out;
390       // note that if simulation performance becomes an issue,
this loop can be unrolled
391       for (int k = 0; k < 64; k++) begin
392           state_out[perm[k]] = state_in[k];
393       end
394       return state_out;
395   endfunction : perm_64bit
396
397   endpackage : prim_cipher_pkg

```

17. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_gating_sync.sv (34 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Common Library: Clock Gating cell with synchronizer
6
7 module prim_clock_gating_sync (
8     input      clk_i,
9     input      rst_ni,
10    input      test_en_i,
11    input      async_en_i,
12    output logic en_o,
13    output logic clk_o
14 );
15
16
17    prim_flop_2sync #(

```

```

18        .Width(1)
19    ) i_sync (
20        .clk_i,
21        .rst_ni,
22        .d_i(async_en_i),
23        .q_o(en_o)
24    );
25
26    prim_clock_gating i_cg (
27        .clk_i,
28        .en_i(en_o),
29        .test_en_i,
30        .clk_o
31    );
32
33
34 endmodule

```

18. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_meas.sv (279 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_clk_meas is a generic module that measures two clocks
against each other.
6 // One clock is the reference, and another is the input.
7 // If the input clocks becomes too fast or too slow, an error
condition is created.
8
9 // The default parameters assume the reference clock is signi-
ficantly slower than
10 // the input clock
11
12 `include "prim_assert.sv"
13
14 module prim_clock_meas #(
15     // Maximum value of input clock counts over measurement
period
16     parameter int Cnt = 16,
17     // Maximum value of reference clock counts over measurement
period
18     parameter int RefCnt = 1,
19     parameter bit ClkTimeOutChkEn = 1,
20     parameter bit RefTimeOutChkEn = 1,
21     localparam int CntWidth = prim_util_pkg::vbits(Cnt),
22     localparam int RefCntWidth = prim_util_pkg::vbits(RefCnt)
23 ) (
24     input clk_i,
25     input rst_ni,
26     input clk_ref_i,
27     input rst_ref_ni,
28     input en_i,
29     input [CntWidth-1:0] max_cnt,
30     input [CntWidth-1:0] min_cnt,
31     // the output valid and fast/slow indications are all on the
// input clock domain
32     output logic valid_o,
33     output logic fast_o,
34     output logic slow_o,
35
36     // signal on clk_i domain that indicates clk_ref has timed
out
37     output logic timeout_clk_ref_o,
38
39     // signal on clk_ref_i domain that indicates clk has timed
out
40     output logic ref_timeout_clk_o
41 );
42
43
44
45 ///////////////////////////////////////////////////////////////////
46 // Reference clock logic
47 ///////////////////////////////////////////////////////////////////
48
49 logic ref_en;
50 prim_flop_2sync #(
51     .Width(1)
52 ) u_ref_meas_en_sync (
53     .d_i(en_i),
54     .clk_i(clk_ref_i),
55     .rst_ni(rst_ref_ni),
56     .q_o(ref_en)
57 );
58
59
60 ///////////////////////////////////////////////////////////////////
61 // Domain sequencing
62 ///////////////////////////////////////////////////////////////////
63
64 // the enable comes in on the clk_i domain, however, to en-
sure the
65 // clk and clk_ref domains remain in sync, the following se-
quencing is
66 // done to enable/disable the measurement feature
67
68 typedef enum logic [1:0] {

```

```

69     StDisable,
70     StEnabling,
71     StEnable,
72     StDisabling
73 } meas_chk_fsm_e;
74
75 // sync reference clock view of enable back into the source
domain
76 logic en_ref_sync;
77 prim_flop_2sync #(
78     .Width(1)
79 ) ack_sync (
80     .clk_i,
81     .rst_ni,
82     .d_i(ref_en),
83     .q_o(en_ref_sync)
84 );
85
86 meas_chk_fsm_e state_d, state_q;
87 always_ff @(posedge clk_i or negedge rst_ni) begin
88     if (!rst_ni) begin
89         state_q <= StDisable;
90     end else begin
91         state_q <= state_d;
92     end
93 end
94
95 // The following fsm sequence ensures that even if the
source
96 // side changes the enable too quickly, the measurement con-
trols
97 // remain consistent.
98 logic cnt_en;
99 always_comb begin
100     state_d = state_q;
101     cnt_en = '0;
102
103     unique case (state_q)
104
105         StDisable: begin
106             if (en_i) begin
107                 state_d = StEnabling;
108             end
109         end
110
111         StEnabling: begin
112             if (en_ref_sync) begin
113                 state_d = StEnable;
114             end
115         end
116
117         StEnable: begin
118             cnt_en = 1'b1;
119             if (!en_i) begin
120                 state_d = StDisabling;
121             end
122         end
123
124         StDisabling: begin
125             if (!en_ref_sync) begin
126                 state_d = StDisable;
127             end
128         end
129
130         //VCS coverage off
131         // pragma coverage off
132         default:;
133         //VCS coverage on
134         // pragma coverage on
135
136     endcase // unique case (state_q)
137 end
138
139 // Input Clock Logic
140 // Input Clock Logic
141
142 logic valid_ref;
143 logic valid;
144 // The valid pulse causes the count to reset and start
counting again
145 // for each reference cycle.
146 // The count obtained during the last reference cycle is
used
147 // to measure how fast/slow the input clock is.
148 prim_pulse_sync u_sync_ref (
149     .clk_src_i(clk_ref_i),
150     .rst_src_ni(rst_ref_ni),
151     .src_pulse_i(ref_en),
152     .clk_dst_i(clk_i),
153     .rst_dst_ni(rst_ni),
154     .dst_pulse_o(valid_ref)
155 );
156
157
158
159 if (RefCnt == 1) begin : gen_degenerate_case
160     // if reference count is one, cnt_ref is always 0.
161     // So there is no need to maintain a counter, and
162     // valid just becomes valid_ref
163     assign valid = valid_ref;
164 end else begin : gen_normal_case

```

```

165     logic [RefCntWidth-1:0] cnt_ref;
166     assign valid = valid_ref & (int'(cnt_ref) == RefCnt - 1);
167     always_ff @(posedge clk_i or negedge rst_ni) begin
168         if (!rst_ni) begin
169             cnt_ref <= '0;
170         end else if (!cnt_en && |cnt_ref) begin
171             cnt_ref <= '0;
172         end else if (cnt_en && valid) begin
173             cnt_ref <= '0;
174         end else if (cnt_en && valid_ref) begin
175             cnt_ref <= cnt_ref + 1'b1;
176         end
177     end
178 end
179
180 logic cnt_ovfl;
181 logic [CntWidth-1:0] cnt;
182 always_ff @(posedge clk_i or negedge rst_ni) begin
183     if (!rst_ni) begin
184         cnt <= '0;
185         cnt_ovfl <= '0;
186     end else if (!cnt_en && |cnt) begin
187         cnt <= '0;
188         cnt_ovfl <= '0;
189     end else if (valid_o) begin
190         cnt <= '0;
191         cnt_ovfl <= '0;
192     end else if (cnt_ovfl) begin
193         cnt <= '{default: '1};
194     end else if (cnt_en) begin
195         {cnt_ovfl, cnt} <= cnt + 1'b1;
196     end
197 end
198
199 assign valid_o = valid & |cnt;
200 assign fast_o = valid_o & ((cnt > max_cnt) | cnt_ovfl);
201 assign slow_o = valid_o & (cnt < min_cnt);
202
203 // Timeout Handling
204 // Timeout Handling
205
206 localparam bit TimeOutChkEn = ClkTimeOutChkEn | Ref-
TimeOutChkEn;
207
208 // determine ratio between
209 localparam int ClkRatio = Cnt / RefCnt;
210
211 // maximum cdc latency from the perspective of the measured
clock
212 // 1 cycle to output request
213 // 2 ref cycles for synchronization
214 // 1 ref cycle to send ack
215 // 2 cycles to sync ack
216 // Double for margin
217 localparam int MaxClkCdcLatency = (1 + 2*ClkRatio + 1*ClkRa-
tio + 2)*2;
218
219 // maximum cdc latency from the perspective of the reference
clock
220 // 1 ref cycle to output request
221 // 2 cycles to sync + 1 cycle to ack are less than 1 cycle
of ref based on assertion requirement
222 // 2 ref cycles to sync ack
223 // 2 extra ref cycles for margin
224 localparam int MaxRefCdcLatency = 1 + 1 + 2 + 2;
225
226 if (RefTimeOutChkEn) begin : gen_ref_timeout_chk
227     // check whether reference clock has timed out
228     prim_clock_timeout #(
229         .TimeOutCnt(MaxClkCdcLatency)
230     ) u_timeout_clk_to_ref (
231         .clk_chk_i(clk_ref_i),
232         .rst_chk_ni(rst_ref_ni),
233         .clk_i,
234         .rst_ni,
235         .en_i,
236         .timeout_o(timeout_clk_ref_o)
237     );
238 end else begin : gen_unused_ref_timeout
239     assign timeout_clk_ref_o = 1'b0;
240 end
241
242 if (ClkTimeOutChkEn) begin : gen_clk_timeout_chk
243     // check whether clock has timed out
244     prim_clock_timeout #(
245         .TimeOutCnt(MaxRefCdcLatency)
246     ) u_timeout_ref_to_clk (
247         .clk_chk_i(clk_i),
248         .rst_chk_ni(rst_ni),
249         .clk_i(clk_ref_i),
250         .rst_ni(rst_ref_ni),
251         .en_i(ref_en),
252         .timeout_o(ref_timeout_clk_o)
253     );
254 end else begin : gen_unused_clk_timeout
255     assign ref_timeout_clk_o = 1'b0;
256 end
257
258
259

```



```

260 //////////////////////////////////////////////////
261 // Assertions
262 //////////////////////////////////////////////////
263
264 if (TimeoutChkEn) begin : gen_timeout_assert
265     // the measured clock must be faster than the reference
clock
266     `ASSERT_INIT(ClkRatios_A, ClkRatio > 2)
267 end
268
269 // reference count has to be at least 1
270 `ASSERT_INIT(RefCntVal_A, RefCnt >= 1)
271
272 // if we've reached the max count, enable must be 0 next.
273 // Otherwise the width of the counter is too small to accom-
modate the use case
274 `ASSERT(MaxWidth_A, (cnt == Cnt-1) |>= !cnt_en )
275
276
277
278
279 endmodule // prim_clk_meas

```

19. lowRISC_opentitan/hw/ip/prim/rtl/prim_clock_timeout.sv (65 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_clk_timeout is a simple module that assesses whether
the input clock
6 // has stopped ticking as measured by the reference clock.
7 //
8 // If both clocks are stopped for whatever reason, this module is
effectively dead.
9
10 `include "prim_assert.sv"
11
12 module prim_clock_timeout #(
13     parameter int TimeoutCnt = 16,
14     localparam int CntWidth = prim_util_pkg::vbits(TimeoutCnt+1)
15 ) (
16     // clock to be checked
17     input clk_chk_i,
18     input rst_chk_ni,
19
20     // clock used to measure whether clk_chk has stopped ticking
21     input clk_i,
22     input rst_ni,
23     input en_i,
24     output logic timeout_o
25 );
26
27 logic [CntWidth-1:0] cnt;
28 logic ack;
29 logic timeout;
30 assign timeout = int'(cnt) >= TimeoutCnt;
31 always_ff @(posedge clk_i or negedge rst_ni) begin
32     if (!rst_ni) begin
33         cnt <= '0;
34     end else if (ack || !en_i) begin
35         cnt <= '0;
36     end else if (timeout) begin
37         cnt <= '{default: '1};
38     end else if (en_i) begin
39         cnt <= cnt + 1'b1;
40     end
41 end
42
43 logic chk_req;
44 prim_sync_reqack u_ref_timeout (
45     .clk_src_i(clk_i),
46     .rst_src_ni(rst_ni),
47     .clk_dst_i(clk_chk_i),
48     .rst_dst_ni(rst_chk_ni),
49     .req_chk_i('0),
50     .src_req_i(1'b1),
51     .src_ack_o(ack),
52     .dst_req_o(chk_req),
53     .dst_ack_i(chk_req)
54 );
55
56 prim_flop #(
57     .ResetValue('0)
58 ) u_out (
59     .clk_i,
60     .rst_ni,
61     .d_i(timeout),
62     .q_o(timeout_o)
63 );
64
65 endmodule // prim_clk_timeout

```

20. lowRISC_opentitan/hw/ip/prim/rtl/prim_count.sv (313 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Hardened counter primitive:
6 //
7 // This internally uses a cross counter scheme with primary
and a secondary counter, where the
8 // secondary counter counts in reverse direction. The sum of
both counters must remain constant and
9 // equal to 2**Width-1 or otherwise an err_o will be asserted.
10 //
11 // This counter supports a generic clear / set / increment /
decrement interface:
12 //
13 // clr_i: This clears the primary counter to ResetValue and
adjusts the secondary counter to match
14 // (i.e. 2**Width-1-ResetValue). Clear has priority
over set, increment and decrement.
15 // set_i: This sets the primary counter to set_cnt_i and ad-
justs the secondary counter to match
16 // (i.e. 2**Width-1-set_cnt_i). Set has priority over
increment and decrement.
17 // incr_en_i: Increments the primary counter by step_i, and
decrements the secondary by step_i.
18 // decr_en_i: Decrements the primary counter by step_i, and
increments the secondary by step_i.
19 // commit_i: Counter changes only take effect when `commit_i`
is set. This does not effect the
20 // `cnt_after_commit_o` output which gives the next
counter state if the change is
21 // committed.
22 //
23 // Note that if both incr_en_i and decr_en_i are asserted at
the same time, the counter remains
24 // unchanged. The counter is also protected against under- and
overflows.
25
26 `include "prim_assert.sv"
27
28 module prim_count
29     import prim_count_pkg::*;
30 (
31     parameter int Width = 2,
32     // Can be used to reset the counter to a different value
than 0, for example when
33     // the counter is used as a down-counter.
34     parameter logic [Width-1:0] ResetValue = '0,
35     // This should only be disabled in special circumstances,
for example
36     // in non-comportable IPs where an error does not trigger an
alert.
37     parameter bit EnableAlertTriggersVA = 1,
38
39     // We have some assertions below with preconditions that de-
pend on particular input actions
40     // (clear, set, incr, decr). If the design has instantiated
prim_count with one of these actions
41     // tied to zero, the preconditions for the associated asser-
tions will not be satisfiable. The
42     // result is an unreachable item, which we treat as a failed
assertion in the report.
43     //
44     // To avoid this, we the instantiation to specify the ac-
tions which might happen. If this is not
45     // '1, we will have an assertion which assert the corres-
ponding action is never triggered. We can
46     // then use this to avoid the unreachable assertions.
47     parameter action_mask_t PossibleActions = {$bits(ac-
tion_mask_t){1'b1}}
48 ) (
49     input clk_i,
50     input rst_ni,
51     input clr_i,
52     input set_i,
53     input [Width-1:0] set_cnt_i, // Set value
for the counter.
54     input incr_en_i,
55     input decr_en_i,
56     input [Width-1:0] step_i, // Increment/
decrement step when enabled.
57     input commit_i,
58     output logic [Width-1:0] cnt_o, // Current
counter state
59     output logic [Width-1:0] cnt_after_commit_o, // Next counter
state if committed
60     output logic err_o
61 );
62
63 //////////////////////////////////////////////////
64 // Counter logic //

```

```

65 //////////////////////////////////////////////////
66 // Reset Values for primary and secondary counters.
67 localparam int NumCnt = 2;
68 localparam logic [NumCnt-1:0][Width-1:0] ResetValues =
{{Width{1'b1}} - ResetValue, // secondary
setVal}; // primary
69 // primary
70 // secondary
71 // The fpv_force signal can be used in FPV runs to make the
72 // internal counters (cnt_q) jump
73 // unexpectedly. We only want to use this mechanism when
74 // we're doing FPV on prim_count itself. In
75 // that situation, we will have the PrimCountFpv define and
76 // wish to leave fpv_force undriven so
77 // that it becomes a free variable in FPV. In any other
78 // situation, we drive the signal with zero.
79 logic [NumCnt-1:0][Width-1:0] fpv_force;
80 `ifndef PrimCountFpv
81 assign fpv_force = '0;
82 `endif
83 for (genvar k = 0; k < NumCnt; k++) begin : gen_cnts
84 // Note that increments / decrements are reversed for the
85 // secondary counter.
86 logic incr_en, decr_en;
87 logic [Width-1:0] set_val;
88 if (k == 0) begin : gen_up_cnt
89 assign incr_en = incr_en_i;
90 assign decr_en = decr_en_i;
91 assign set_val = set_cnt_i;
92 end else begin : gen_dn_cnt
93 assign incr_en = decr_en_i;
94 assign decr_en = incr_en_i;
95 // The secondary value needs to be adjusted accordingly.
96 assign set_val = {Width{1'b1}} - set_cnt_i;
97 end
98 // Main counter logic
99 logic [Width:0] ext_cnt;
100 assign ext_cnt = (decr_en) ? {1'b0, cnt_q[k]} - {1'b0,
step_i} :
101 (incr_en) ? {1'b0, cnt_q[k]} + {1'b0,
step_i} : {1'b0, cnt_q[k]};
102 // Saturation logic
103 logic oflow, oflow;
104 assign oflow = incr_en && ext_cnt[Width];
105 assign oflow = decr_en && ext_cnt[Width];
106 logic [Width-1:0] cnt_sat;
107 assign cnt_sat = (oflow) ? '0 :
108 (oflow) ? {Width{1'b1}} :
109 ext_cnt[Width-1:0];
110 // Clock gate flops when in saturation, and do not
111 // count if both incr_en and decr_en are asserted.
112 logic cnt_en;
113 assign cnt_en = (incr_en ^ decr_en) &&
114 ((incr_en && !(cnt_q[k])) ||
115 (decr_en && !(cnt_q[k] == '0)));
116 // Counter muxes
117 assign cnt_d[k] = (clr_i) ? ResetValues[k] :
118 (set_i) ? set_val :
119 (cnt_en) ? cnt_sat : cnt_q[k];
120 assign cnt_d_committed[k] = commit_i ? cnt_d[k] :
cnt_q[k];
121 logic [Width-1:0] cnt_unforced_q;
122 prim_flop #(
123 .Width(Width),
124 .ResetValue(ResetValues[k])
125 ) u_cnt_flop (
126 .clk_i,
127 .rst_ni,
128 .d_i(cnt_d_committed[k]),
129 .q_o(cnt_unforced_q)
130 );
131 // fpv_force is only used during FPV.
132 assign cnt_q[k] = fpv_force[k] + cnt_unforced_q;
133 end
134 // The sum of both counters must always equal the counter
135 // maximum.
136 logic [Width:0] sum;
137 assign sum = (cnt_q[0] + cnt_q[1]);
138 // Register the error signal to avoid potential CDC issues
139 // downstream.
140 logic err_d, err_q;
141 assign err_d = (sum != {1'b0, {Width{1'b1}}});
142 always_ff @(posedge clk_i or negedge rst_ni) begin
143 if (!rst_ni) begin
144 err_q <= 'b0;
145 end else begin
146 err_q <= err_d;
147 end
148 end

```

```

153 end
154 assign err_o = err_q;
155 // Output count values
156 assign cnt_o = cnt_q[0];
157 assign cnt_after_commit_o = cnt_d[0];
158 //////////////////////////////////////////////////
159 // Assertions //
160 //////////////////////////////////////////////////
161 `ifdef INC_ASSERT
162 // VCS coverage off
163 // pragma coverage off
164 // We need to disable most assertions in that case using a
165 // helper signal.
166 // We can't rely on err_o since some error patterns cannot
167 // be detected (e.g. all error
168 // patterns that still fulfil the sum constraint).
169 logic fpv_err_present;
170 assign fpv_err_present = |fpv_force;
171 // Helper functions for assertions.
172
173 function automatic logic signed [Width+1:0] max(logic signed
[Width+1:0] a,
174 logic signed
[Width+1:0] b);
175 return (a > b) ? a : b;
176 endfunction
177
178 function automatic logic signed [Width+1:0] min(logic signed
[Width+1:0] a,
179 logic signed
[Width+1:0] b);
180 return (a < b) ? a : b;
181 endfunction
182 // VCS coverage on
183 // pragma coverage on
184 if (!(PossibleActions & Clr)) begin : g_check_no_clr
185 `ASSERT(ClrNeverTrue_A, clr_i !== 1'b1)
186 end
187 if (!(PossibleActions & Set)) begin : g_check_no_set
188 `ASSERT(SetNeverTrue_A, set_i !== 1'b1)
189 end
190 if (!(PossibleActions & Incr)) begin : g_check_no_incr
191 `ASSERT(IncrNeverTrue_A, incr_en_i !== 1'b1)
192 end
193 if (!(PossibleActions & Decr)) begin : g_check_no_decr
194 `ASSERT(DecrNeverTrue_A, decr_en_i !== 1'b1)
195 end
196 // Cnt next
197 `ASSERT(CntNext_A,
200 rst_ni
201 |>
202 $past(!commit_i) || (cnt_o ==
203 $past(cnt_after_commit_o)),
204 clk_i, err_d || fpv_err_present || !rst_ni)
205 // Clear
206 if (PossibleActions & Clr) begin : g_check_clr_fwd_a
207 `ASSERT(ClrFwd_A,
208 rst_ni && commit_i && clr_i
209 |>
210 (cnt_o == ResetValue) &&
211 (cnt_q[1] == ({Width{1'b1}} - ResetValue)),
212 clk_i, err_d || fpv_err_present || !rst_ni)
213 end
214 // Set
215 if (PossibleActions & Set) begin : g_check_set_fwd_a
216 `ASSERT(SetFwd_A,
217 rst_ni && commit_i && set_i && !clr_i
218 |>
219 (cnt_o == $past(set_cnt_i)) &&
220 (cnt_q[1] == ({Width{1'b1}} - $past(set_cnt_i))),
221 clk_i, err_d || fpv_err_present || !rst_ni)
222 end
223 // Do not count if both increment and decrement are asser-
224 // ted.
225 if ((PossibleActions & Incr) && (PossibleActions & Decr))
226 begin : g_check_inc_and_dec
227 `ASSERT(IncrDecrUpDnCnt_A,
228 rst_ni && incr_en_i && decr_en_i && !(clr_i || set_i)
229 |>
230 $stable(cnt_o) && $stable(cnt_q[1]),
231 clk_i, err_d || fpv_err_present || !rst_ni)
232 end
233 // Increment
234 if ((PossibleActions & Incr)) begin : g_check_incr
235 `ASSERT(IncrUpCnt_A,
236 rst_ni && incr_en_i && !(clr_i || set_i || decr_en_i)
237 && commit_i
238 |>
239

```



```

240 cnt_o == min($past(cnt_o) + $past({2'b0, step_i}),
{2'b0, {Width{1'b1}}}),
241 clk_i, err_d || fpv_err_present || !rst_ni)
242 `ASSERT(IncrDnCnt_A,
243 rst_ni && incr_en_i && !(clr_i || set_i || decr_en_i)
&& commit_i
244 |>=
245 cnt_q[1] == max($past(signed'({2'b0, cnt_q[1]})) -
$past({2'b0, step_i}), '0),
246 clk_i, err_d || fpv_err_present || !rst_ni)
247 `ASSERT(UpCntIncrStable_A,
248 incr_en_i && !(clr_i || set_i || decr_en_i) &&
249 cnt_o == {Width{1'b1}}
250 |>=
251 $stable(cnt_o),
252 clk_i, err_d || fpv_err_present || !rst_ni)
253 `ASSERT(DnCntIncrStable_A,
254 rst_ni && incr_en_i && !(clr_i || set_i || decr_en_i)
&&
255 cnt_q[1] == '0
256 |>=
257 $stable(cnt_q[1]),
258 clk_i, err_d || fpv_err_present || !rst_ni)
259 end
260
261 // Decrement
262 if ((PossibleActions & Decr)) begin : g_check_decr
263 `ASSERT(DecrDnCnt_A,
264 rst_ni && decr_en_i && !(clr_i || set_i || incr_en_i)
&& commit_i
265 |>=
266 cnt_o == max($past(signed'({2'b0, cnt_o})) -
$past({2'b0, step_i}), '0),
267 clk_i, err_d || fpv_err_present || !rst_ni)
268 `ASSERT(DecrDnCnt_A,
269 rst_ni && decr_en_i && !(clr_i || set_i || incr_en_i)
&& commit_i
270 |>=
271 cnt_q[1] == min($past(cnt_q[1]) + $past({2'b0,
step_i}), {2'b0, {Width{1'b1}}}),
272 clk_i, err_d || fpv_err_present || !rst_ni)
273 `ASSERT(UpCntDecrStable_A,
274 decr_en_i && !(clr_i || set_i || incr_en_i) &&
275 cnt_o == '0
276 |>=
277 $stable(cnt_o),
278 clk_i, err_d || fpv_err_present || !rst_ni)
279 `ASSERT(DnCntDecrStable_A,
280 rst_ni && decr_en_i && !(clr_i || set_i || incr_en_i)
&&
281 cnt_q[1] == {Width{1'b1}}
282 |>=
283 $stable(cnt_q[1]),
284 clk_i, err_d || fpv_err_present || !rst_ni)
285 end
286
287 // A backwards check for count changes. This asserts that
the count only changes if one of the
288 // inputs that should tell it to change (clear, set, incre-
ment, decrement) does so.
289 `ASSERT(ChangeBackward_A,
290 rst_ni ##1 $changed(cnt_o) && $changed(cnt_q[1])
291 |>->
292 $past(clr_i || set_i || (commit_i && (incr_en_i ||
decr_en_i))),
293 clk_i, err_d || fpv_err_present || !rst_ni)
294
295 // Check that count errors are reported properly in err_o
296 //
297 // This is essentially a "|>= implication", but is struc-
tured in a way to avoid generating a cover
298 // property for the left hand side if PrimCountFpv is not
defined (because we won't have a way to
299 // inject an error if not)
300 `ASSERT(CntErrReported_A, ##1 $past((cnt_q[1] + cnt_q[0]) !=
{Width{1'b1}}) == err_o)
301 `ifdef PrimCountFpv
302 `COVER(CntErr_C, err_o)
303 `endif
304
305 // This logic that will be assign to one, when user adds
macro
306 // ASSERT_PRIM_COUNT_ERROR_TRIGGER_ALERT to check the error
with alert, in case that prim_count
307 // is used in design without adding this assertion check.
308 logic unused_assert_connected;
309
310 `ASSERT_INIT_NET(AssertConnected_A, unused_assert_connected
=== '1'b1 || !EnableAlertTriggerSVA)
311 `endif
312
313 endmodule // prim_count

```

21. lowRISC_opentitan/hw/ip/prim/rtl/prim_count_pkg.sv (15 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for

```

details.

```

3 // SPDX-License-Identifier: Apache-2.0
4
5 package prim_count_pkg;
6
7 // An enum that names the possible actions that the inputs
might ask for. See the PossibleActions
8 // parameter in prim_count for how this is used.
9 typedef logic [3:0] action_mask_t;
10 typedef enum action_mask_t {Clr = 4'h1,
11 Set = 4'h2,
12 Incr = 4'h4,
13 Decr = 4'h8} action_e;
14
15 endpackage : prim_count_pkg

```

22. lowRISC_opentitan/hw/ip/prim/rtl/prim_crc32.sv (324 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // CRC32 calculator
6 //
7 // This module takes in n-bits data words (n defined by Byte-
PerWord parameter) and updates an
8 // internally stored CRC with each valid data word. The poly-
nomial used is the standard CRC32 IEEE
9 // one. An interface is provided to set the internal CRC to an ar-
bitrary value. The output CRC is an
10 // inverted version of the internally stored CRC and the input
CRC is inverted before being stored.
11 // This is done so results match existing widely used software
libraries (e.g. the crc32
12 // functionality available in Python). Note that a initial CRC of
0x0 (corresponding to an internal
13 // CRC of 0xffffffff) must be used to match results generated
elsewhere.
14
15 module prim_crc32 #(
16 parameter int unsigned BytesPerWord = 4
17 ) (
18 input logic clk_i,
19 input logic rst_ni,
20
21 input logic set_crc_i,
22 input logic [31:0] crc_in_i,
23
24 input logic data_valid_i,
25 input logic [BytesPerWord*8-1:0] data_i,
26
27 output logic [31:0] crc_out_o
28 );
29 // Generated using hw/ip/prim/util/prim_crc32_table_gen.py
30 function automatic logic [31:0] crc32_byte_calc(logic [7:0]
b);
31 unique case (b)
32 8'hff: crc32_byte_calc = 32'h2d02ef8d;
33 8'hfe: crc32_byte_calc = 32'h5a05df1b;
34 8'hfd: crc32_byte_calc = 32'hc30c8ea1;
35 8'hfc: crc32_byte_calc = 32'hb40bbe37;
36 8'hfb: crc32_byte_calc = 32'h2a6f2b94;
37 8'hfa: crc32_byte_calc = 32'h5d681b02;
38 8'hf9: crc32_byte_calc = 32'hc4614ab8;
39 8'hf8: crc32_byte_calc = 32'hb3667a2e;
40 8'hf7: crc32_byte_calc = 32'h23d967bf;
41 8'hf6: crc32_byte_calc = 32'h54de5729;
42 8'hf5: crc32_byte_calc = 32'hcd70693;
43 8'hf4: crc32_byte_calc = 32'hbad03605;
44 8'hf3: crc32_byte_calc = 32'h24b4a3a6;
45 8'hf2: crc32_byte_calc = 32'h53b39330;
46 8'hf1: crc32_byte_calc = 32'hcabac28a;
47 8'hf0: crc32_byte_calc = 32'hbdbdf21c;
48 8'hef: crc32_byte_calc = 32'h30b5ffe9;
49 8'hee: crc32_byte_calc = 32'h47b2cf7f;
50 8'hed: crc32_byte_calc = 32'hdebb9ec5;
51 8'hec: crc32_byte_calc = 32'ha9bcae53;
52 8'heb: crc32_byte_calc = 32'h37d83bf0;
53 8'hea: crc32_byte_calc = 32'h40df0b66;
54 8'he9: crc32_byte_calc = 32'hd9d65adc;
55 8'he8: crc32_byte_calc = 32'haed16a4a;
56 8'he7: crc32_byte_calc = 32'h3e6e77db;
57 8'he6: crc32_byte_calc = 32'h4969474d;
58 8'he5: crc32_byte_calc = 32'hd06016f7;
59 8'he4: crc32_byte_calc = 32'ha7672661;
60 8'he3: crc32_byte_calc = 32'h3903b3c2;
61 8'he2: crc32_byte_calc = 32'h4e048354;
62 8'he1: crc32_byte_calc = 32'hd70dd2ee;
63 8'he0: crc32_byte_calc = 32'ha00ae278;
64 8'hdf: crc32_byte_calc = 32'h166ccf45;
65 8'hde: crc32_byte_calc = 32'h616bffd3;
66 8'hdd: crc32_byte_calc = 32'hf862ae69;
67 8'hdc: crc32_byte_calc = 32'h8f659eff;

```

69	8'hdb:	crc32_byte_calc = 32'h11010b5c;	170	8'h76:	crc32_byte_calc = 32'hb966d409;
70	8'hda:	crc32_byte_calc = 32'h66063bca;	171	8'h75:	crc32_byte_calc = 32'h206f85b3;
71	8'hd9:	crc32_byte_calc = 32'hff0f6a70;	172	8'h74:	crc32_byte_calc = 32'h5768b525;
72	8'hd8:	crc32_byte_calc = 32'h80805ae6;	173	8'h73:	crc32_byte_calc = 32'hc902086;
73	8'hd7:	crc32_byte_calc = 32'h18b74777;	174	8'h72:	crc32_byte_calc = 32'hbe0b1010;
74	8'hd6:	crc32_byte_calc = 32'h6fb077e1;	175	8'h71:	crc32_byte_calc = 32'h270241aa;
75	8'hd5:	crc32_byte_calc = 32'hf6b9265b;	176	8'h70:	crc32_byte_calc = 32'h5005713c;
76	8'hd4:	crc32_byte_calc = 32'h81be1bcd;	177	8'h6f:	crc32_byte_calc = 32'hdd0d7cc9;
77	8'hd3:	crc32_byte_calc = 32'h1fda836e;	178	8'h6e:	crc32_byte_calc = 32'haa0a4c5f;
78	8'hd2:	crc32_byte_calc = 32'h68ddb3f8;	179	8'h6d:	crc32_byte_calc = 32'h33031de5;
79	8'hd1:	crc32_byte_calc = 32'hf1d4e242;	180	8'h6c:	crc32_byte_calc = 32'h44042d73;
80	8'hd0:	crc32_byte_calc = 32'h86d3d2d4;	181	8'h6b:	crc32_byte_calc = 32'hda60bd0;
81	8'hcf:	crc32_byte_calc = 32'h0bdbdf21;	182	8'h6a:	crc32_byte_calc = 32'had678846;
82	8'hce:	crc32_byte_calc = 32'h7cdcefb7;	183	8'h69:	crc32_byte_calc = 32'h346ed9fc;
83	8'hcd:	crc32_byte_calc = 32'he5d5be0d;	184	8'h68:	crc32_byte_calc = 32'h4369e96a;
84	8'hcc:	crc32_byte_calc = 32'h92d28e9b;	185	8'h67:	crc32_byte_calc = 32'hd3d6f4fb;
85	8'hcb:	crc32_byte_calc = 32'h0cb61b38;	186	8'h66:	crc32_byte_calc = 32'ha4d1c46d;
86	8'hca:	crc32_byte_calc = 32'h7bb12bae;	187	8'h65:	crc32_byte_calc = 32'h3dd895d7;
87	8'hc9:	crc32_byte_calc = 32'he2b87a14;	188	8'h64:	crc32_byte_calc = 32'h4adf5a51;
88	8'hc8:	crc32_byte_calc = 32'h95bf4a82;	189	8'h63:	crc32_byte_calc = 32'hd4bb30e2;
89	8'hc7:	crc32_byte_calc = 32'h05005713;	190	8'h62:	crc32_byte_calc = 32'ha3bc0074;
90	8'hc6:	crc32_byte_calc = 32'h72076785;	191	8'h61:	crc32_byte_calc = 32'h3ab551ce;
91	8'hc5:	crc32_byte_calc = 32'heb0e363f;	192	8'h60:	crc32_byte_calc = 32'h4db26158;
92	8'hc4:	crc32_byte_calc = 32'h9c0906a9;	193	8'h5f:	crc32_byte_calc = 32'hfbd44c65;
93	8'hc3:	crc32_byte_calc = 32'h026d930a;	194	8'h5e:	crc32_byte_calc = 32'h8cd37cf3;
94	8'hc2:	crc32_byte_calc = 32'h756aa39c;	195	8'h5d:	crc32_byte_calc = 32'h15da2d49;
95	8'hc1:	crc32_byte_calc = 32'h6c3f226;	196	8'h5c:	crc32_byte_calc = 32'h62dd1ddf;
96	8'hc0:	crc32_byte_calc = 32'h9b64c2b0;	197	8'h5b:	crc32_byte_calc = 32'hfc9887c;
97	8'hbf:	crc32_byte_calc = 32'h5bdeae1d;	198	8'h5a:	crc32_byte_calc = 32'h8bbe8ea;
98	8'hbe:	crc32_byte_calc = 32'h2cd99e8b;	199	8'h59:	crc32_byte_calc = 32'h12b7e950;
99	8'hbd:	crc32_byte_calc = 32'hb5d0cf31;	200	8'h58:	crc32_byte_calc = 32'h65bd09c6;
100	8'hbc:	crc32_byte_calc = 32'hc2d7ffa7;	201	8'h57:	crc32_byte_calc = 32'hf50fc457;
101	8'hbb:	crc32_byte_calc = 32'h5cb36a04;	202	8'h56:	crc32_byte_calc = 32'h8208f4c1;
102	8'hba:	crc32_byte_calc = 32'h2bb45a92;	203	8'h55:	crc32_byte_calc = 32'h1b01a57b;
103	8'hb9:	crc32_byte_calc = 32'hb2bd0b28;	204	8'h54:	crc32_byte_calc = 32'h6c0695ed;
104	8'hb8:	crc32_byte_calc = 32'hc5ba3bbe;	205	8'h53:	crc32_byte_calc = 32'hf22004e;
105	8'hb7:	crc32_byte_calc = 32'h5505262f;	206	8'h52:	crc32_byte_calc = 32'h856530d8;
106	8'hb6:	crc32_byte_calc = 32'h202016b9;	207	8'h51:	crc32_byte_calc = 32'h1c6c6162;
107	8'hb5:	crc32_byte_calc = 32'hbb0b4703;	208	8'h50:	crc32_byte_calc = 32'h6b6b51f4;
108	8'hb4:	crc32_byte_calc = 32'hcc0c7795;	209	8'h4f:	crc32_byte_calc = 32'he6635c01;
109	8'hb3:	crc32_byte_calc = 32'h5268e236;	210	8'h4e:	crc32_byte_calc = 32'h91646c97;
110	8'hb2:	crc32_byte_calc = 32'h256fd2a0;	211	8'h4d:	crc32_byte_calc = 32'h086d3d2d;
111	8'hb1:	crc32_byte_calc = 32'hbc66831a;	212	8'h4c:	crc32_byte_calc = 32'h7f6a0dbb;
112	8'hb0:	crc32_byte_calc = 32'hcb61b38c;	213	8'h4b:	crc32_byte_calc = 32'he10e9818;
113	8'haf:	crc32_byte_calc = 32'h4669be79;	214	8'h4a:	crc32_byte_calc = 32'h9609a88e;
114	8'hae:	crc32_byte_calc = 32'h316e8eef;	215	8'h49:	crc32_byte_calc = 32'h0f00f934;
115	8'had:	crc32_byte_calc = 32'ha867df55;	216	8'h48:	crc32_byte_calc = 32'h7807c9a2;
116	8'hac:	crc32_byte_calc = 32'hdf60efc3;	217	8'h47:	crc32_byte_calc = 32'he8b8d433;
117	8'hab:	crc32_byte_calc = 32'h41047a60;	218	8'h46:	crc32_byte_calc = 32'h9fbf4e4a;
118	8'haa:	crc32_byte_calc = 32'h36034af6;	219	8'h45:	crc32_byte_calc = 32'h066b51f;
119	8'ha9:	crc32_byte_calc = 32'haf0a1b4c;	220	8'h44:	crc32_byte_calc = 32'h71b18589;
120	8'ha8:	crc32_byte_calc = 32'hd80d2bda;	221	8'h43:	crc32_byte_calc = 32'h6fd5102a;
121	8'ha7:	crc32_byte_calc = 32'h48b2364b;	222	8'h42:	crc32_byte_calc = 32'h98d220bc;
122	8'ha6:	crc32_byte_calc = 32'h3fb506dd;	223	8'h41:	crc32_byte_calc = 32'h01db7106;
123	8'ha5:	crc32_byte_calc = 32'ha6bc5767;	224	8'h40:	crc32_byte_calc = 32'h76dc4190;
124	8'ha4:	crc32_byte_calc = 32'hd1bb67f1;	225	8'h3f:	crc32_byte_calc = 32'hb6662d3d;
125	8'ha3:	crc32_byte_calc = 32'h4fdff252;	226	8'h3e:	crc32_byte_calc = 32'hc1611dad;
126	8'ha2:	crc32_byte_calc = 32'h38d8c2c4;	227	8'h3d:	crc32_byte_calc = 32'h58684c11;
127	8'ha1:	crc32_byte_calc = 32'ha1d1937e;	228	8'h3c:	crc32_byte_calc = 32'h2f6f7c87;
128	8'ha0:	crc32_byte_calc = 32'hd6d6a3e8;	229	8'h3b:	crc32_byte_calc = 32'h10be924;
129	8'h9f:	crc32_byte_calc = 32'h0b080ed5;	230	8'h3a:	crc32_byte_calc = 32'hc60cd9b2;
130	8'h9e:	crc32_byte_calc = 32'h17b7be43;	231	8'h39:	crc32_byte_calc = 32'h5f058808;
131	8'h9d:	crc32_byte_calc = 32'h8ebeeef9;	232	8'h38:	crc32_byte_calc = 32'h2802b89e;
132	8'h9c:	crc32_byte_calc = 32'hf9b9d6f6;	233	8'h37:	crc32_byte_calc = 32'hb8bd450f;
133	8'h9b:	crc32_byte_calc = 32'h67dd4acc;	234	8'h36:	crc32_byte_calc = 32'hcfba9599;
134	8'h9a:	crc32_byte_calc = 32'h10da7a5a;	235	8'h35:	crc32_byte_calc = 32'h56b3c423;
135	8'h99:	crc32_byte_calc = 32'h89d32be0;	236	8'h34:	crc32_byte_calc = 32'h21b4f4b5;
136	8'h98:	crc32_byte_calc = 32'hfed41b76;	237	8'h33:	crc32_byte_calc = 32'hbfd06116;
137	8'h97:	crc32_byte_calc = 32'h66eb06e7;	238	8'h32:	crc32_byte_calc = 32'hc8d75180;
138	8'h96:	crc32_byte_calc = 32'h196c3671;	239	8'h31:	crc32_byte_calc = 32'h51de003a;
139	8'h95:	crc32_byte_calc = 32'h806567cb;	240	8'h30:	crc32_byte_calc = 32'h26d930ac;
140	8'h94:	crc32_byte_calc = 32'hf762575d;	241	8'h2f:	crc32_byte_calc = 32'habd13d59;
141	8'h93:	crc32_byte_calc = 32'h6906c2fe;	242	8'h2e:	crc32_byte_calc = 32'hdc60dcf;
142	8'h92:	crc32_byte_calc = 32'h1e01f268;	243	8'h2d:	crc32_byte_calc = 32'h45df5c75;
143	8'h91:	crc32_byte_calc = 32'h8708a3d2;	244	8'h2c:	crc32_byte_calc = 32'h32d86ce3;
144	8'h90:	crc32_byte_calc = 32'hf00f9344;	245	8'h2b:	crc32_byte_calc = 32'hacbcf940;
145	8'h8f:	crc32_byte_calc = 32'h7d079eb1;	246	8'h2a:	crc32_byte_calc = 32'hdbbbc9d6;
146	8'h8e:	crc32_byte_calc = 32'h0a00ae27;	247	8'h29:	crc32_byte_calc = 32'h42b2986c;
147	8'h8d:	crc32_byte_calc = 32'h9309ff9d;	248	8'h28:	crc32_byte_calc = 32'h35b5a8fa;
148	8'h8c:	crc32_byte_calc = 32'he40ecf0b;	249	8'h27:	crc32_byte_calc = 32'ha50ab56b;
149	8'h8b:	crc32_byte_calc = 32'h7a6a5aa8;	250	8'h26:	crc32_byte_calc = 32'hd20d85fd;
150	8'h8a:	crc32_byte_calc = 32'h0d6d6a3e;	251	8'h25:	crc32_byte_calc = 32'h4b04d447;
151	8'h89:	crc32_byte_calc = 32'h94643b84;	252	8'h24:	crc32_byte_calc = 32'h3c03e4d1;
152	8'h88:	crc32_byte_calc = 32'he3630b12;	253	8'h23:	crc32_byte_calc = 32'ha2677172;
153	8'h87:	crc32_byte_calc = 32'h73dc1683;	254	8'h22:	crc32_byte_calc = 32'hd56041e4;
154	8'h86:	crc32_byte_calc = 32'h04db2615;	255	8'h21:	crc32_byte_calc = 32'h4c69105e;
155	8'h85:	crc32_byte_calc = 32'h9dd277af;	256	8'h20:	crc32_byte_calc = 32'h3b6e20c8;
156	8'h84:	crc32_byte_calc = 32'head54739;	257	8'h1f:	crc32_byte_calc = 32'h8d080df5;
157	8'h83:	crc32_byte_calc = 32'h74b1d29a;	258	8'h1e:	crc32_byte_calc = 32'hfa0f3d63;
158	8'h82:	crc32_byte_calc = 32'h03b6e20c;	259	8'h1d:	crc32_byte_calc = 32'h63066cd9;
159	8'h81:	crc32_byte_calc = 32'h9abfb3b6;	260	8'h1c:	crc32_byte_calc = 32'h14015c4f;
160	8'h80:	crc32_byte_calc = 32'hedb88320;	261	8'h1b:	crc32_byte_calc = 32'h8a65c9ec;
161	8'h7f:	crc32_byte_calc = 32'h0ba6cad;	262	8'h1a:	crc32_byte_calc = 32'hfd62f97a;
162	8'h7e:	crc32_byte_calc = 32'hb7bd5c3b;	263	8'h19:	crc32_byte_calc = 32'h646ba8c0;
163	8'h7d:	crc32_byte_calc = 32'h2eb40d81;	264	8'h18:	crc32_byte_calc = 32'h136c9856;
164	8'h7c:	crc32_byte_calc = 32'h59b33d17;	265	8'h17:	crc32_byte_calc = 32'h83d38c57;
165	8'h7b:	crc32_byte_calc = 32'hc7d7a8b4;	266	8'h16:	crc32_byte_calc = 32'hf44db551;
166	8'h7a:	crc32_byte_calc = 32'hb0d09822;	267	8'h15:	crc32_byte_calc = 32'h6ddde4eb;
167	8'h79:	crc32_byte_calc = 32'h29d9c998;	268	8'h14:	crc32_byte_calc = 32'h1adad47d;
168	8'h78:	crc32_byte_calc = 32'h5edef90e;	269	8'h13:	crc32_byte_calc = 32'h84be41de;
169	8'h77:	crc32_byte_calc = 32'hce61e49f;	270	8'h12:	crc32_byte_calc = 32'hf3b97148;

```

271      8'h11:  crc32_byte_calc = 32'h6ab020f2;
272      8'h10:  crc32_byte_calc = 32'h1db71064;
273      8'h0f:  crc32_byte_calc = 32'h90bf1d91;
274      8'h0e:  crc32_byte_calc = 32'he7b82d07;
275      8'h0d:  crc32_byte_calc = 32'h7eb17cbd;
276      8'h0c:  crc32_byte_calc = 32'h09b64c2b;
277      8'h0b:  crc32_byte_calc = 32'h97d2d988;
278      8'h0a:  crc32_byte_calc = 32'he0d5e91e;
279      8'h09:  crc32_byte_calc = 32'h79dcb8a4;
280      8'h08:  crc32_byte_calc = 32'h0edb8832;
281      8'h07:  crc32_byte_calc = 32'h9e6495a3;
282      8'h06:  crc32_byte_calc = 32'he963a535;
283      8'h05:  crc32_byte_calc = 32'h706af48f;
284      8'h04:  crc32_byte_calc = 32'h076dc419;
285      8'h03:  crc32_byte_calc = 32'h990951ba;
286      8'h02:  crc32_byte_calc = 32'hee0e612c;
287      8'h01:  crc32_byte_calc = 32'h77073096;
288      8'h00:  crc32_byte_calc = 32'h00000000;
289      default: crc32_byte_calc = '0;
290    endcase
291  endfunction
292
293  logic [31:0] crc_d, crc_q;
294  logic        crc_en;
295  logic [31:0] crc_stages[BytesPerWord + 1];
296
297  assign crc_en = set_crc_i | data_valid_i;
298
299  assign crc_stages[0] = crc_q;
300
301  for (genvar i = 0; i < BytesPerWord; ++i) begin :
g_crc_stages
302    assign crc_stages[i + 1] =
303      {8'h00, crc_stages[i][31:8]} ^
304      crc32_byte_calc(crc_stages[i][7:0] ^ data_i[i * 8 +:
8]);
305  end
306
307  always_comb begin
308    if (set_crc_i) begin
309      crc_d = ~crc_in_i;
310    end else begin
311      crc_d = crc_stages[BytesPerWord];
312    end
313  end
314
315  always_ff @(posedge clk_i or negedge rst_ni) begin
316    if (!rst_ni) begin
317      crc_q <= 32'hFFFFFFF;
318    end else if (crc_en) begin
319      crc_q <= crc_d;
320    end
321  end
322
323  assign crc_out_o = ~crc_q;
324 endmodule

```

23. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_decode.sv (333 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module decodes a differentially encoded signal and de-
fects
6 // incorrectly encoded differential states.
7 //
8 // In case the differential pair crosses an asynchronous
boundary, it has
9 // to be re-synchronized to the local clock. This can be
achieved by
10 // setting the AsyncOn parameter to 1'b1. In that case, two
additional
11 // input registers are added (to counteract metastability),
and
12 // a pattern detector is instantiated that detects skewed
level changes on
13 // the differential pair (i.e., when level changes on the diff
pair are
14 // sampled one cycle apart due to a timing skew between the
two wires).
15 //
16 // See also: prim_alert_sender, prim_alert_receiver,
alert_handler
17
18 `include "prim_assert.sv"
19
20 module prim_diff_decode #(
21   // enables additional synchronization logic
22   parameter bit AsyncOn = 1'b0,
23   // Number of cycles a differential skew is tolerated before
a signal integrity issue is flagged.
24   // Only has an effect if AsyncOn = 1
25   // 0 means no skew is tolerated (any mismatch is an immedi-
ate signal integrity error).
26   // 1 means a one-cycle skew is tolerated.

```

```

27 // Values larger than 1 are also supported.
28 parameter int unsigned SkewCycles = 1
29 ) (
30   input        clk_i,
31   input        rst_ni,
32   // input diff pair
33   input        diff_pi,
34   input        diff_ni,
35   // logical level and
36   // detected edges
37   output logic level_o,
38   output logic rise_o,
39   output logic fall_o,
40   // either rise or fall
41   output logic event_o,
42   //signal integrity issue detected
43   output logic sigint_o
44 );
45
46 logic level_d, level_q;
47
48 //////////////////////////////////////////////////
49 // synchronization regs for incoming diff pair (if re-
quired) //
50 //////////////////////////////////////////////////
51 if (AsyncOn) begin : gen_async
52
53   typedef enum logic [1:0] {IsStd, IsSkewing, SigInt}
state_e;
54   state_e state_d, state_q;
55   logic diff_p_edge, diff_n_edge, diff_check_ok, level;
56
57   // 2 sync regs, one reg for edge detection
58   logic diff_pq, diff_nq, diff_pd, diff_nd;
59
60   // Counter for skew cycles tolerated before flagging an
issue
61   // The width needs to accommodate SkewCycles + 1 to count
up to SkewCycles.
62   logic [prim_util_pkg::vbits(SkewCycles + 1)-1:0]
skew_cnt_d, skew_cnt_q;
63
64   prim_flop_2sync #(
65     .Width(1),
66     .ResetValue('0)
67   ) i_sync_p (
68     .clk_i,
69     .rst_ni,
70     .d_i(diff_pi),
71     .q_o(diff_pd)
72   );
73
74   prim_flop_2sync #(
75     .Width(1),
76     .ResetValue(1'b1)
77   ) i_sync_n (
78     .clk_i,
79     .rst_ni,
80     .d_i(diff_ni),
81     .q_o(diff_nd)
82   );
83
84   // detect level transitions
85   assign diff_p_edge = diff_pq ^ diff_pd;
86   assign diff_n_edge = diff_nq ^ diff_nd;
87
88   // detect sigint issue
89   assign diff_check_ok = diff_pd ^ diff_nd;
90
91   // this is the current logical level
92   assign level = diff_pd;
93
94   // outputs
95   assign level_o = level_d;
96   assign event_o = rise_o | fall_o;
97
98   // sigint detection is a bit more involved in async case
since
99   // we might have skew on the diff pair, which can result
in a
100   // N cycle sampling delay between the two wires
101   // so we need a simple pattern matcher
102   // the following waves are legal
103   // clk | | | | | | |
104   //
105   // p _____ ... _____
106   // n _____ ... _____
107   //
108   // p _____ ... _____
109   // n _____ ... _____
110   //
111   // i.e., level changes may be off by N cycle - which is
permissible
112   // as long as this condition is only N cycle long.
113
114   always_comb begin : p_diff_fsm
115
116

```

```

118 // default
119 state_d = state_q;
120 level_d = level_q;
121 skew_cnt_d = skew_cnt_q;
122 rise_o = '1'b0;
123 fall_o = '1'b0;
124 sigint_o = '1'b0;
125
126 unique case (state_q)
127 // we remain here as long as
128 // the diff pair is correctly encoded
129 IsStd: begin
130   if (diff_check_ok) begin
131     level_d = level;
132     if (diff_p_edge && diff_n_edge) begin
133       if (level) begin
134         rise_o = '1'b1;
135       end else begin
136         fall_o = '1'b1;
137       end
138     end
139   end else begin
140     if (SkewCycles == 0) begin
141       // If no skew is tolerated, immediate signal in-
142       // tegrity error
143       state_d = SigInt;
144       sigint_o = '1'b1;
145     end else begin
146       // Mismatch with an edge: likely start of a tol-
147       // erated skew
148       state_d = IsSkewing;
149       skew_cnt_d = 1;
150     end
151   end
152 // diff pair must be correctly encoded, otherwise we
153 // got a sigint
154 IsSkewing: begin
155   if (diff_check_ok) begin
156     state_d = IsStd;
157     level_d = level;
158     // Reset the skew counter
159     skew_cnt_d = '0;
160     // Assert event that was delayed due to skew res-
161     // olution
162     if (level) rise_o = '1'b1;
163     else fall_o = '1'b1;
164   end else begin
165     if (skew_cnt_q < SkewCycles) begin
166       // Still within tolerated skew cycles
167       skew_cnt_d = skew_cnt_q + 1;
168     end else begin
169       // Maximum skew cycles exceeded, raise an integ-
170       // rity issue
171       state_d = SigInt;
172       sigint_o = '1'b1;
173       skew_cnt_d = '0;
174     end
175   end
176 // Signal integrity issue detected, remain here until
177 // resolved
178 SigInt: begin
179   sigint_o = '1'b1;
180   if (diff_check_ok) begin
181     state_d = IsStd;
182     sigint_o = '1'b0;
183     level_d = level;
184     // Assert any event that was pending while in
185     // SigInt
186     if (level) rise_o = '1'b1;
187     else fall_o = '1'b1;
188   end
189   default : ;
190   endcase
191 end
192 always_ff @(posedge clk_i or negedge rst_ni) begin :
193 p_sync_reg
194   if (!rst_ni) begin
195     state_q <= IsStd;
196     diff_pq <= '1'b0;
197     diff_nq <= '1'b1;
198     level_q <= '1'b0;
199     skew_cnt_q <= '0;
200   end else begin
201     state_q <= state_d;
202     diff_pq <= diff_pd;
203     diff_nq <= diff_nd;
204     level_q <= level_d;
205     skew_cnt_q <= skew_cnt_d;
206   end
207 end
208 ///////////////////////////////////////////////////////////////////
209 // fully synchronous case, no skew present in this case //
210 ///////////////////////////////////////////////////////////////////
211 end else begin : gen_no_async
212   logic diff_pq, diff_pd;
213 end

```

```

211 // one reg for edge detection
212 assign diff_pd = diff_pi;
213
214 // Raise a signal integrity error when the differential
215 // signals have equal values. This is
216 // implemented with a `prim_xnor2` instead of behavioral
217 // code to prevent the synthesis tool from
218 // optimizing away combinational logic on the complement-
219 // ary differential signals.
220 prim_xnor2 #(
221   .Width (1)
222 ) u_xnor2_sigint (
223   .in0_i (diff_pi),
224   .in1_i (diff_ni),
225   .out_o (sigint_o)
226 );
227
228 assign level_o = (sigint_o) ? level_q : diff_pi;
229 assign level_d = level_o;
230
231 // detect level transitions
232 assign rise_o = (~diff_pq & diff_pi) & ~sigint_o;
233 assign fall_o = (diff_pq & ~diff_pi) & ~sigint_o;
234 assign event_o = rise_o | fall_o;
235
236 always_ff @(posedge clk_i or negedge rst_ni) begin :
237 p_edge_reg
238   if (!rst_ni) begin
239     diff_pq <= '1'b0;
240     level_q <= '1'b0;
241   end else begin
242     diff_pq <= diff_pd;
243     level_q <= level_d;
244   end
245 end
246
247 // assertions //
248 ///////////////////////////////////////////////////////////////////
249 // shared assertions
250 // sigint -> level stays the same during sigint
251 // $isunknown is needed to avoid false assertion in first
252 // clock cycle
253 `ASSERT(SigintLevelCheck_A, ##1 sigint_o |>-
254 $stable(level_o))
255 // sigint -> no additional events asserted at output
256 `ASSERT(SigintEventCheck_A, sigint_o |>- !event_o)
257 `ASSERT(SigintRiseCheck_A, sigint_o |>- !rise_o)
258 `ASSERT(SigintFallCheck_A, sigint_o |>- !fall_o)
259
260 if (AsyncOn) begin : gen_async_assert
261 // assertions for asynchronous case
262 `ifdef INC_ASSERT
263 `ifndef FPV_ALERT_NO_SIGINT_ERR
264 // Correctly detect signal integrity issue:
265 // If diff_pd and diff_nd are equal for (SkewCycles + 1)
266 // consecutive cycles, sigint_o must be
267 // asserted.
268 `ASSERT(SigintCheck0_A,
269   gen_async.diff_pd == gen_async.diff_nd [* (Skew-
270 Cycles + 1)] |>- sigint_o)
271 // The following assertions (SigintCheck1_A to
272 SigintCheck4_A) describe specific
273 // 1-cycle skew patterns that should lead to an edge.
274 // These are highly
275 // specific to SkewCycles = 1. Therefore, they are only
276 // included when SkewCycles is 1.
277 // The synchronizer adds 2 cycles of latency with respect
278 // to input signals.
279 if (SkewCycles == 1) begin : gen_specific_skew_asserts
280   `ASSERT(SigintCheck1_A,
281     ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
282     $stable(gen_async.diff_pd) &&
283     $stable(gen_async.diff_nd) ##1
284     $rose(gen_async.diff_pd) &&
285     $stable(gen_async.diff_nd) ##1
286     $stable(gen_async.diff_pd) && $fell(gen_a-
287 sync.diff_nd)
288     |>- rise_o)
289   `ASSERT(SigintCheck2_A,
290     ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
291     $stable(gen_async.diff_pd) &&
292     $stable(gen_async.diff_nd) ##1
293     $fell(gen_async.diff_pd) &&
294     $stable(gen_async.diff_nd) ##1
295     $stable(gen_async.diff_pd) && $rose(gen_a-
296 sync.diff_nd)
297     |>- fall_o)
298   `ASSERT(SigintCheck3_A,
299     ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
300     $stable(gen_async.diff_pd) &&
301     $stable(gen_async.diff_nd) ##1
302     $rose(gen_async.diff_nd) &&
303     $stable(gen_async.diff_pd) ##1
304     $stable(gen_async.diff_nd) && $fell(gen_a-
305 sync.diff_pd)
306     |>- fall_o)
307   `ASSERT(SigintCheck4_A,

```

```

291     ##1 (gen_async.diff_pd ^ gen_async.diff_nd) &&
292     $stable(gen_async.diff_pd) &&
$stable(gen_async.diff_nd) ##1
293     $fell(gen_async.diff_nd) &&
$stable(gen_async.diff_pd) ##1
294     $stable(gen_async.diff_nd) && $rose(gen_a-
sync.diff_pd)
295     |-> rise_o)
296 end
297 `endif
298 // Correctly detect edges: an event should be asserted
within SkewCycles cycles after a valid
299 // transition
300 `ASSERT(RiseCheck_A,
301     !sigint_o ##1 $rose(gen_async.diff_pd) && (gen_a-
sync.diff_pd ^ gen_async.diff_nd) |->
302     ##[0:SkewCycles] rise_o, clk_i, !rst_ni || sigint_o)
303 `ASSERT(FallCheck_A,
304     !sigint_o ##1 $fell(gen_async.diff_pd) && (gen_a-
sync.diff_pd ^ gen_async.diff_nd) |->
305     ##[0:SkewCycles] fall_o, clk_i, !rst_ni || sigint_o)
306 `ASSERT(EventCheck_A,
307     !sigint_o ##1 $changed(gen_async.diff_pd) && (gen_a-
sync.diff_pd ^ gen_async.diff_nd) |->
308     ##[0:SkewCycles] event_o, clk_i, !rst_ni || sigint_o)
309 // Correctly detect level: the output level should match
diff_pd once the differential pair is
310 // stable
311 `ASSERT(LevelCheck0_A,
312     // Stable for SkewCycles + 1 cycles
313     !sigint_o && (gen_async.diff_pd ^ gen_async.diff_nd)
[* (SkewCycles + 1)] |->
314     gen_async.diff_pd == level_o,
315     clk_i, !rst_ni || sigint_o)
316 `endif
317 end else begin : gen_sync_assert
318     // assertions for synchronous case
319
320     `ifndef FPV_ALERT_NO_SIGINT_ERR
321     // correctly detect sigint issue
322     `ASSERT(SigintCheck_A, diff_pi == diff_ni |-> sigint_o)
323     `endif
324
325     // correctly detect edges
326     `ASSERT(RiseCheck_A, ##1 $rose(diff_pi) && (diff_pi ^
diff_ni) |-> rise_o)
327     `ASSERT(FallCheck_A, ##1 $fell(diff_pi) && (diff_pi ^
diff_ni) |-> fall_o)
328     `ASSERT(EventCheck_A, ##1 $changed(diff_pi) && (diff_pi ^
diff_ni) |-> event_o)
329     // correctly detect level
330     `ASSERT(LevelCheck_A, (diff_pi ^ diff_ni) |-> diff_pi ==
level_o)
331 end
332
333 endmodule : prim_diff_decode

```

24. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_encode.sv (61 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // This module encodes a single bit signal to a differentially en-
coded signal.
6 //
7 // In case the differential pair crosses an asynchronous
boundary, it has
8 // to be re-synchronized to the local clock.
9
10 module prim_diff_encode (
11     input logic clk_i,
12     input logic rst_ni,
13     // Single line input signal
14     input logic req_i,
15     // Output diff pair
16     output logic diff_po,
17     output logic diff_no
18 );
19 // Buff the input signal to avoid any optimization
logic req;
20 prim_sec_anchor_buf #(
21     .Width(1)
22 ) u_prim_buf_in_req (
23     .in_i(req_i),
24     .out_o(req)
25 );
26
27 // Differentially encode input
logic diff_p, diff_n;
28 assign diff_p = req;
29 assign diff_n = ~req;
30
31 // This prevents further tool optimizations of the differen-
tial signal.
32 prim_sec_anchor_buf #(

```

```

35     .Width(2)
36 ) u_prim_buf_ack (
37     .in_i({diff_n, diff_p}),
38     .out_o({diff_n_buf, diff_p_buf})
39 );
40
41 // Flop differentially encoded signal at the output
prim_flop #(
42     .Width(1),
43     .ResetValue(1'b0)
44 ) u_diff_p (
45     .clk_i,
46     .rst_ni,
47     .d_i ( diff_p_buf ),
48     .q_o ( diff_po )
49 );
50
51 prim_flop #(
52     .Width(1),
53     .ResetValue(1'b1)
54 ) u_diff_n (
55     .clk_i,
56     .rst_ni,
57     .d_i ( diff_n_buf ),
58     .q_o ( diff_no )
59 );
60
61 endmodule : prim_diff_encode

```

25. lowRISC_opentitan/hw/ip/prim/rtl/prim_diff_to_alert.sv (70 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // prim_diff_to_alert: This module receives a differentially
encoded alert request and translates
6 // that to an alert signal with the alert protocol.
7
8 module prim_diff_to_alert #(
9     // AsyncOn: Enables additional synchronization logic within
the alert receiver.
10     parameter bit AsyncOn = 1'b1,
11     // Number of cycles a differential skew is tolerated on the
alert signal
12     parameter int unsigned SkewCycles = 1,
13     // Alert sender will latch the incoming alert event perman-
ently and
14     // keep on sending alert events until the next reset.
15     parameter bit IsFatal = 1'b1
16 ) (
17     input logic clk_i,
18     input logic rst_ni,
19     // Input diff pair (differentially encoded alert signal)
20     input logic diff_pi,
21     input logic diff_ni,
22     // Alert pair (interface signals for the alert protocol)
23     input prim_alert_pkg::alert_rx_t alert_rx_i,
24     output prim_alert_pkg::alert_tx_t alert_tx_o
25 );
26 logic diff_p_sync, diff_n_sync;
27
28 if (AsyncOn) begin : gen_async
29     prim_flop_2sync #(
30         .Width(2),
31         .ResetValue(2'b10)
32     ) u_sync (
33         .clk_i,
34         .rst_ni,
35         .d_i ( {diff_ni, diff_pi} ),
36         .q_o ( {diff_n_sync, diff_p_sync} )
37     );
38 end else begin : gen_sync
39     assign diff_p_sync = diff_pi;
40     assign diff_n_sync = diff_ni;
41 end
42
43 // This prevents further tool optimizations of the differen-
tial signal.
44 logic diff_p_buf, diff_n_buf;
45 prim_sec_anchor_buf #(
46     .Width(2)
47 ) u_prim_buf_ack (
48     .in_i ( {diff_n_sync, diff_p_sync} ),
49     .out_o ( {diff_n_buf, diff_p_buf} )
50 );
51
52 // Treat any positive value on `diff_p_buf` and any negative
value on `diff_n_buf` as alert.
53 logic alert_req;
54 assign alert_req = diff_p_buf | ~diff_n_buf;
55
56 prim_alert_sender #(
57     .AsyncOn(AsyncOn),
58     .SkewCycles(SkewCycles),
59     .IsFatal(IsFatal)
60 ) u_prim_alert_sender (

```



```

61 .clk_i,
62 .rst_ni,
63 .alert_test_i ( 1'b0 ),
64 .alert_req_i ( alert_req ),
65 .alert_ack_o ( ),
66 .alert_state_o ( ),
67 .alert_rx_i,
68 .alert_tx_o
69 );
70 endmodule

```

26. lowRISC_opentitan/hw/ip/prim/rtl/prim_dom_and_2share.sv (173 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Domain-Oriented Masking GF(2) Multiplier with 2-shares
6 // ref: Higher-Order Side-Channel Protected Implementations of
  Keccak
7 // https://eprint.iacr.org/2017/395.pdf
8 //
9 // q0 = a0 & b0 + (a0 & b1 + z)
10 // q1 = a1 & b1 + (a1 & b0 + z)
11 // () ==> registered
12 //
13 // all input should be stable for two clocks
14 // as the output is valid after a clock
15 // For z, it can use other slice from the state
16 // as it is fairly random w.r.t the current inputs.
17 //
18 // General formula of Q in the paper
19 // Qi = t{i,i} + Sig(j>i,d)(t{i,j}+Z{i+j*(j-1)/2}) +
  Sig(j<i,d)(t{i,j}+Z{j+i*(i-1)/2})
20 // for d=1 (NumShare 2 for first order protection)
21 // Q0 = t{0,0} + Sig(j>0,1)(t{0,j}+Z{j*(j-1)/2}) + Sig(j<0,d)
  (...)
22 // = a0&b0 + (a0&b1 + z0 + 0)
23 // Q1 = t{1,1} + sig(j>1,1)(...) + sig(j<1,1)(t{1,j} + Z{j})
24 // = a1&b1 + (0 + a1&b0 + z0)
25
26 `include "prim_assert.sv"
27
28 module prim_dom_and_2share #(
29     parameter int DW = 64, // Input width
30     parameter bit Pipeline = 1'b0 // Enable full pipelining
31 ) (
32     input clk_i,
33     input rst_ni,
34
35     input [DW-1:0] a0_i, // share0 of a
36     input [DW-1:0] a1_i, // share1 of a
37     input [DW-1:0] b0_i, // share0 of b
38     input [DW-1:0] b1_i, // share1 of b
39     input z_valid_i, // random number input validity
40     input [DW-1:0] z_i, // random number
41
42     output logic [DW-1:0] q0_o, // share0 of q
43     output logic [DW-1:0] q1_o, // share1 of q
44     output logic [DW-1:0] prd_o // pseudo-random data for other
  instances
45 );
46
47 logic [DW-1:0] t0_d, t0_q, t1_d, t1_q;
48 logic [DW-1:0] t_a0b0, t_a1b1;
49 logic [DW-1:0] t_a0b0_d, t_a1b1_d;
50 logic [DW-1:0] t_a0b1, t_a1b0;
51
52 // Calculation //
53 // Inner-domain terms
54 // Inner-domain terms
55 assign t_a0b0_d = a0_i & b0_i;
56 assign t_a1b1_d = a1_i & b1_i;
57
58 // Cross-domain terms
59 assign t_a0b1 = a0_i & b1_i;
60 assign t_a1b0 = a1_i & b0_i;
61
62 // Resharing //
63 // Resharing of cross-domain terms
64 // Preserve the logic sequence for XOR not to proceed cross-
  domain AND.
65 prim_xor2 #(
66     .Width ( DW*2 )
67 ) u_prim_xor_t01 (
68     .in0_i ( {t_a0b1, t_a1b0} ),
69     .in1_i ( {z_i, z_i} ),
70     .out_o ( {t0_d, t1_d} )
71 );
72
73 // Register stage
74 prim_flop_en #(

```

```

75     .Width ( DW*2 ),
76     .ResetValue ( '0 )
77 ) u_prim_flop_t01 (
78     .clk_i ( clk_i ),
79     .rst_ni ( rst_ni ),
80     .en_i ( z_valid_i ),
81     .d_i ( {t0_d, t1_d} ),
82     .q_o ( {t0_q, t1_q} )
83 );
84
85 // Optional Pipelining //
86
87 if (Pipeline == 1'b1) begin : gen_inner_domain_regs
88     // Add pipeline registers on inner-domain terms prior to
  integration. This allows accepting new
89     // input data every clock cycle and prevents SCA leakage
  occurring due to the integration of
90     // reshared cross-domain terms with inner-domain terms de-
  rived from different input data.
91
92     logic [DW-1:0] t_a0b0_q, t_a1b1_q;
93     prim_flop_en #(
94         .Width ( DW*2 ),
95         .ResetValue ( '0 )
96     ) u_prim_flop_tab01 (
97         .clk_i ( clk_i ),
98         .rst_ni ( rst_ni ),
99         .en_i ( z_valid_i ),
100        .d_i ( {t_a0b0_d, t_a1b1_d} ),
101        .q_o ( {t_a0b0_q, t_a1b1_q} )
102    );
103    assign t_a0b0 = t_a0b0_q;
104    assign t_a1b1 = t_a1b1_q;
105
106    end else begin : gen_no_inner_domain_regs
107        // Do not add the optional pipeline registers on the in-
  ner-domain terms. This allows to save
108        // some area in case the multiplier does not need to ac-
  cept new data in every cycle. However,
109
110        // this can cause SCA leakage as during the clock cycle in
  which new data arrives, the new
111        // inner-domain terms are integrated with the previous,
  reshared cross-domain terms.
112
113        assign t_a0b0 = t_a0b0_d;
114        assign t_a1b1 = t_a1b1_d;
115    end
116
117 // Integration //
118
119 // Preserve the logic sequence for XOR not to proceed the
  inner-domain AND.
120 prim_xor2 #(
121     .Width ( DW*2 )
122 ) u_prim_xor_q01 (
123     .in0_i ( {t_a0b0, t_a1b1} ),
124     .in1_i ( {t0_q, t1_q} ),
125     .out_o ( {q0_o, q1_o} )
126 );
127
128 // Use intermediate results for remasking computations in
  another instance in the following
129 // clock cycle. Use one share only. Directly use output of
  flops updating with z_valid_i.
130 // t1_q is obtained by remasking t_a1b0 with z_i. Since z_i
  is uniformly distributed and
131 // independent of a1/b0_i, t1_q is also uniformly distrib-
  uted and independent of a1/b0_i.
132 // For details, see Lemma 1 in Canright, "A very compact
  'perfectly masked' S-box for AES
133 // (corrected)" available at https://eprint.iacr.org/
  2009/011.pdf
134 assign prd_o = t1_q;
135
136 // DOM AND should be same as unmasked computation
137 // The correct test sequence will be:
138 // 1. inputs are changed
139 // 2. check if z_valid_i,
140 // 3. at the next cycle, inputs are still stable (assump-
  tion) - only in case Pipeline = 0
141 // 4. and results Q == A & B (assertion)
142
143 // To speed up the FPV process, random value is ready in
  less than or
144 // equal to two cycles.
145 `ASSUME_FPV(RandomReadyInShortTime_A,
146     $changed(a0_i) || $changed(a1_i) || $changed(b0_i) ||
  $changed(b1_i)
147     |-> ##[0:2] z_valid_i,
148     clk_i, !rst_ni)
149
150 if (Pipeline == 0) begin: g_assert_stable
151     // If Pipeline is not set, the computation takes two
  cycles without flop

```

```

160 // crossing the domain. In this case, the signal should be
stable for at
161 // least two cycles.
162 `ASSUME(StableTwoCycles_M,
163 ($changed(a0_i) || $changed(a1_i) || $changed(b0_i) ||
$changed(b1_i)))
164 ##[0:$] z_valid_i |>=
165 $stable(a0_i) && $stable(a1_i) && $stable(b0_i) &&
$stable(b1_i))
166 end
167
168 `ASSERT(UnmaskedAndMatched_A,
169 z_valid_i |>= (q0_o ^ q1_o) ==
170 (($past(a0_i) ^ $past(a1_i)) & ($past(b0_i) ^
$past(b1_i))),
171 clk_i, !rst_ni)
172
173 endmodule

```

27. lowRISC_opentitan/hw/ip/prim/rtl/prim_double_lfsr.sv (113 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Hardened LFSR module that instantiates two LFSRs of the
same type.

6 // The state vector of both LFSRs is constantly checked and an er-
ror is asserted if the
7 // two states are inconsistent.
8
9 module prim_double_lfsr #(
10 // prim_lfsr parameters - refer to prim_lfsr for their mean-
ing/
11 parameter LfsrType = "GAL_XOR",
12 parameter int unsigned LfsrDw = 32,
13 localparam int unsigned LfsrIdxDw = $clog2(LfsrDw),
14 parameter int unsigned EntropyDw = 8,
15 parameter int unsigned StateOutDw = 8,
16 parameter logic [LfsrDw-1:0] DefaultSeed = LfsrDw'(1),
17 parameter logic [LfsrDw-1:0] CustomCoeffs = '0,
18 parameter bit StatePermEn = 1'b0,
19 parameter logic [LfsrDw-1:0] [LfsrIdxDw-1:0] StatePerm = '0,
20 parameter bit MaxLenSVA = 1'b1,
21 parameter bit LockupSVA = 1'b1,
22 parameter bit ExtSeedSVA = 1'b1,
23 parameter bit NonLinearOut = 1'b0,
24 // This should only be disabled in special circumstances,
for example

25 // in non-comportable IPs where an error does not trigger an
alert.
26 parameter bit EnableAlertTriggerSVA = 1
27 ) (
28 input clk_i,
29 input rst_ni,
30 input seed_en_i,
31 input [LfsrDw-1:0] seed_i,
32 input lfsr_en_i,
33 input [EntropyDw-1:0] entropy_i,
34 output logic [StateOutDw-1:0] state_o,
35 // Asserted if the parallel LFSR states are inconsistent.
36 output logic err_o
37 );
38
39
40 logic [1:0] [LfsrDw-1:0] lfsr_state;
41 // We employ redundant LFSRs to guard against FI attacks.
42 for (genvar k = 0; k < 2; k++) begin : gen_double_lfsr
43 // Instantiate size_only buffers to prevent
44 // optimization / merging of redundant logic.
45 logic lfsr_en_buf, seed_en_buf;
46 logic [EntropyDw-1:0] entropy_buf;
47 logic [LfsrDw-1:0] seed_buf, lfsr_state_unbuf;
48 prim_buf #(
49 .Width(EntropyDw + LfsrDw + 2)
50 ) u_prim_buf_input (
51 .in_i({seed_en_i, seed_i, lfsr_en_i, entropy_i}),
52 .out_o({seed_en_buf, seed_buf, lfsr_en_buf, en-
tropy_buf})
53 );
54
55 prim_lfsr #(
56 .LfsrType(LfsrType),
57 .LfsrDw(LfsrDw),
58 .EntropyDw(EntropyDw),
59 // output the full width so that the states can be cross
checked.
60 .StateOutDw(LfsrDw),
61 .DefaultSeed(DefaultSeed),
62 .CustomCoeffs(CustomCoeffs),
63 .StatePermEn(StatePermEn),
64 .StatePerm(StatePerm),
65 .MaxLenSVA(MaxLenSVA),
66 .LockupSVA(LockupSVA),

```

```

67 .ExtSeedSVA(ExtSeedSVA),
68 .NonLinearOut(NonLinearOut)
69 ) u_prim_lfsr (
70 .clk_i,
71 .rst_ni,
72 .seed_en_i ( seed_en_buf ),
73 .seed_i ( seed_buf ),
74 .lfsr_en_i ( lfsr_en_buf ),
75 .entropy_i ( entropy_buf ),
76 .state_o ( lfsr_state_unbuf )
77 );
78
79 prim_buf #(
80 .Width(LfsrDw)
81 ) u_prim_buf_output (
82 .in_i(lfsr_state_unbuf),
83 .out_o(lfsr_state[k])
84 );
85 end
86
87 `ifdef SIMULATION
88 `ifndef VERILATOR
89 // Ensure both LFSRs start off with the same default seed.
if randomized in simulations.
90 initial begin : p_sync_lfsr_default_seed
91 wait (!$isun-
known(gen_double_lfsr[0].u_prim_lfsr.DefaultSeedLocal));
92 wait (!$isun-
known(gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal));
93 gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal =
94 gen_double_lfsr[0].u_prim_lfsr.DefaultSeedLocal;
95 $display("%m: Updated
gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal = 0x%0h",
96 gen_double_lfsr[1].u_prim_lfsr.DefaultSeedLocal);
97 end
98 `endif
99 `endif
100
101 // Output the state from the first LFSR
102 assign state_o = lfsr_state[0][StateOutDw-1:0];
103 assign err_o = lfsr_state[0] != lfsr_state[1];
104
105 // This logic that will be assign to one, when user adds
macro
106 // ASSERT_PRIM_DOUBLE_LFSR_ERROR_TRIGGER_ALERT to check the
error with alert, in case that
107 // prim_double_lfsr is used in design without adding this
assertion check.
108 `ifdef INC_ASSERT
109 logic unused_assert_connected;
110
111 `ASSERT_INIT_NET(AssertConnected_A, unused_assert_connected
=== 1'b1 || !EnableAlertTriggerSVA)
112 `endif
113 endmodule : prim_double_lfsr

```

28. lowRISC_opentitan/hw/ip/prim/rtl/prim_edge_detector.sv (55 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).

2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // Edge Detector
6
7 module prim_edge_detector #(
8 parameter int unsigned Width = 1,
9
10 parameter logic [Width-1:0] ResetValue = '0,
11
12 // EnSync
13 //
14 // Enable Synchronizer to the input signal.
15 // It is assumed that the input signal is glitch free (re-
gistered input).
16 parameter bit EnSync = 1'b1
17 ) (
18 input clk_i,
19 input rst_ni,
20
21 input [Width-1:0] d_i,
22 output logic [Width-1:0] q_sync_o,
23
24 output logic [Width-1:0] q_posedge_pulse_o,
25 output logic [Width-1:0] q_negedge_pulse_o
26 );
27
28 logic [Width-1:0] q_sync_d, q_sync_q;
29
30 if (EnSync) begin : g_sync
31 prim_flop_2sync #(
32 .Width(Width),
33 .ResetValue(ResetValue)
34 ) u_sync (
35 .clk_i,
36 .rst_ni,
37 .d_i,
38 .q_o(q_sync_d)

```



```
39     );
40 end : g_sync
41 else begin : g_nosync
42     assign q_sync_d = d_i;
43 end : g_nosync
44
45 assign q_sync_o = q_sync_d;
46
47 always_ff @(posedge clk_i or negedge rst_ni) begin
```

```
48     if (!rst_ni) q_sync_q <= ResetValue;
49     else
50         q_sync_q <= q_sync_d;
51     end
52     assign q_posedge_pulse_o = q_sync_d & ~q_sync_q;
53     assign q_negedge_pulse_o = ~q_sync_d & q_sync_q;
54
55 endmodule : prim_edge_detector
```

Caliptra SHA-512

8 files • 4,374 lines

API — sha512.sv

Table 726. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — module / interface / package (1개)

이름	종류
sha512	module

Table 727. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH		32
DATA_WIDTH		64

Table 728. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — localparam (5개)

상수	형/폭	값
BLOCK_SIZE		1024
DIG_SIZE		512
BLOCK_NUM_DWORDS		BLOCK_SIZE / DATA_WIDTH
DIG_NUM_DWORDS		DIG_SIZE / DATA_WIDTH
NONCE_NUM_DWORDS		PV_SIZE_OF_NONCE / DATA_WIDTH

Table 729. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv — 포트 (17개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cs	input	wire
we	input	wire
address	input	wire [ADDR_WIDTH-1 : 0]
write_data	input	wire [DATA_WIDTH-1 : 0]
read_data	output	wire [DATA_WIDTH-1 : 0]
err	output	wire
pv_read	output	pv_read_t
pv_write	output	pv_write_t
pv_rd_resp	input	pv_rd_resp_t
pv_wr_resp	input	pv_wr_resp_t
pcr_signing_hash	output	logic [PCR_HASH_NUM_DWORDS-1:0][DATA_WIDTH-1:0]

포트	방향	형/폭
error_intr	output	wire
notif_intr	output	wire
debugUnlock_or_scan_mode_switch	input	logic

API — sha512_core.v

Table 730. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v — module / interface / package (1개)

이름	종류
sha512_core	module

Table 731. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v — localparam (4개)

상수	형/폭	값
SHA512_ROUNDS		79
CTRL_IDLE		2'h0
CTRL_ROUNDS		2'h1
CTRL_DONE		2'h2

Table 732. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v — 포트 (12개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
init_cmd	input	wire
next_cmd	input	wire
restore_cmd	input	wire
mode	input	wire [1 : 0]
block_msg	input	wire [1023 : 0]
restore_digest	input	wire [511 : 0]
ready	output	wire
digest	output	wire [511 : 0]
digest_valid	output	wire

API — sha512_ctrl.sv

Table 733. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv — module / interface / package (1개)

이름	종류
sha512_ctrl	module

Table 734. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv — parameter (2개)

파라미터	형/폭	기본값
AHB_DATA_WIDTH		32
AHB_ADDR_WIDTH		32

Table 735. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv — 포트 (21개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
haddr_i	input	logic [AHB_ADDR_WIDTH-1:0]
hdata_i	input	logic [AHB_DATA_WIDTH-1:0]
hsel_i	input	logic
hwrite_i	input	logic
hready_i	input	logic
htrans_i	input	logic [1:0]
hsize_i	input	logic [2:0]
hresp_o	output	logic
hreadyout_o	output	logic
hrdata_o	output	logic [AHB_DATA_WIDTH-1:0]
pv_read	output	pv_read_t
pv_write	output	pv_write_t
pv_rd_resp	input	pv_rd_resp_t
pv_wr_resp	input	pv_wr_resp_t
pcr_signing_hash	output	logic [PCR_HASH_NUM_DWORDS-1:0][31:0]
error_intr	output	logic
notif_intr	output	logic
debugUnlock_or_scan_mode_switch	input	logic

API — sha512_h_constants.v

Table 736. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_h_constants.v — module / interface / package (1개)

이름	종류
sha512_h_constants	module

Table 737. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_h_constants.v — 포트 (9개)

포트	방향	형/폭
mode	input	wire [1 : 0]
H0	output	wire [63 : 0]

포트	방향	형/폭
H1	output	wire [63 : 0]
H2	output	wire [63 : 0]
H3	output	wire [63 : 0]
H4	output	wire [63 : 0]
H5	output	wire [63 : 0]
H6	output	wire [63 : 0]
H7	output	wire [63 : 0]

API — sha512_k_constants.v

Table 738. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_k_constants.v — module / interface / package (1개)

이름	종류
sha512_k_constants	module

Table 739. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_k_constants.v — 포트 (2개)

포트	방향	형/폭
addr	input	wire [6 : 0]
K_val	output	wire [63 : 0]

API — sha512_params_pkg.sv

Table 740. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_params_pkg.sv — module / interface / package (1개)

이름	종류
sha512_params_pkg	package

Table 741. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_params_pkg.sv — localparam (8개)

상수	형/폭	값
SHA512_CORE_NAME0		32'h61327368
SHA512_CORE_NAME1		32'h31322d35
SHA512_CORE_VERSION0		32'h3830302e
SHA512_CORE_VERSION1		32'h00000000
MODE_SHA_512_224		2'h0
MODE_SHA_512_256		2'h1
MODE_SHA_384		2'h2
MODE_SHA_512		2'h3

API — sha512_reg.sv

Table 742. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg.sv — module / interface / package (1개)

이름	종류
sha512_reg	module

Table 743. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_cpuif_req	input	wire
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [11:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_biten	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire
hwif_in	input	sha512_reg_pkg::sha512_reg__in_t
hwif_out	output	sha512_reg_pkg::sha512_reg__out_t

API — sha512_reg_pkg.sv**Table 744. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg_pkg.sv — module / interface / package (1개)**

이름	종류
sha512_reg_pkg	package

Table 745. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg_pkg.sv — localparam (3개)

상수	형/폭	값
SHA512_REG_DATA_WIDTH		32
SHA512_REG_MIN_ADDR_WIDTH		12
SHA512_REG_ADDR_WIDTH		32'd12

1. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512.sv (546 lines)

```

1 //
=====
2 // Updated by Caliptra team to modify data access width
3 // and removing the work factor
4 //
5 // sha512.sv
6 // -----
7 // Top level wrapper for the SHA-512 hash function providing

```

```

8 // a simple memory like interface with 32 bit data access.
9 //
10 //
11 // Author: Joachim Strombergson
12 // Copyright (c) 2014, Secworks Sweden AB
13 // All rights reserved.
14 //
15 // Redistribution and use in source and binary forms, with or
16 // without modification, are permitted provided that the fol-
lowing
17 // conditions are met:
18 //

```

```

19 // 1. Redistributions of source code must retain the above
copyright
20 // notice, this list of conditions and the following dis-
claimer.
21 //
22 // 2. Redistributions in binary form must reproduce the above
copyright
23 // notice, this list of conditions and the following dis-
claimer in
24 // the documentation and/or other materials provided with
the
25 // distribution.
26 //
27 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
28 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
29 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
30 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
31 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
32 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
33 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
34 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
35 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
36 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
37 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
38 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
39 //
40 //
=====
==
41
42 module sha512
43     import sha512_reg_pkg::*;
44     import sha512_params_pkg::*;
45     import kv_defines_pkg::*;
46     import pv_defines_pkg::*;
47     #(
48         parameter ADDR_WIDTH = 32,
49         parameter DATA_WIDTH = 64
50     )(
51         // Clock and reset.
52         input wire          clk,
53         input wire          reset_n,
54         input wire          cptra_pwrgood,
55
56         // Control.
57         input wire          cs,
58         input wire          we,
59
60         // Data ports.
61         input wire [ADDR_WIDTH-1 : 0] address,
62         input wire [DATA_WIDTH-1 : 0] write_data,
63         output wire [DATA_WIDTH-1 : 0] read_data,
64         output wire          err,
65
66         // PV interface
67         output pv_read_t pv_read,
68         output pv_write_t pv_write,
69         input pv_rd_resp_t pv_rd_resp,
70         input pv_wr_resp_t pv_wr_resp,
71
72         output logic [PCR_HASH_NUM_DWORDS-1:0][DATA_WIDTH-1:0]
pcr_signing_hash,
73
74         // Interrupts
75         output wire error_intr,
76         output wire notif_intr,
77         input logic debugUnlock_or_scan_mode_switch
78     );
79     `include "caliptra_prim_assert.sv"
80     `include "kv_macros.svh"
81
82 //-----
83 // Registers including update variables and write enable.
84 //-----
85 reg init_reg;
86 reg next_reg;
87 reg restore_reg;
88 reg ready_reg;
89 reg [1 : 0] mode_reg;
90 logic zeroize_reg;
91 logic last_reg;
92
93 localparam BLOCK_SIZE = 1024;
94 localparam DIG_SIZE = 512;
95
96 localparam BLOCK_NUM_DWORDS = BLOCK_SIZE / DATA_WIDTH;
97 localparam DIG_NUM_DWORDS = DIG_SIZE / DATA_WIDTH;
98 localparam NONCE_NUM_DWORDS = PV_SIZE_OF_NONCE / DATA_WIDTH;
99

```

```

100 reg [DATA_WIDTH-1 : 0][BLOCK_NUM_DWORDS-1 : 0]
block_reg ;
101 reg [DIG_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] di-
gest_reg;
102 reg [PV_NUM_DWORDS -1 : 0][DATA_WIDTH-1 : 0] kv_reg;
103
104
105 reg [PCR_HASH_NUM_DWORDS -1 : 0][DATA_WIDTH-1 : 0]
pcr_sign_reg;
106 logic [PCR_HASH_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0]
pcr_sign;
107 logic
pcr_sign_we;
108 reg di-
gest_valid_reg;
109 logic di-
gest_we;
110
111 sha512_reg__in_t hwif_in;
112 sha512_reg__out_t hwif_out;
113 logic read_error, write_error;
114
115 //KV/PV Gasket
116 kv_read_t vault_read;
117 kv_write_t vault_write;
118 kv_rd_resp_t vault_rd_resp;
119 kv_wr_resp_t vault_wr_resp;
120
121 pv_read_t gen_hash_pv_read;
122
123 //interface with client
124 logic kv_src_write_en;
125 logic [4:0] kv_src_write_offset;
126 logic [31:0] kv_src_write_data;
127
128 kv_error_code_e kv_src_error, kv_dest_error;
129 logic kv_src_ready, kv_src_done;
130 logic kv_dest_ready, kv_dest_done;
131
132 logic dest_keyvault;
133 kv_write_ctrl_reg_t kv_write_ctrl_reg;
134 kv_write_ctrl_reg_t kv_write_ctrl_reg_q;
135 kv_read_ctrl_reg_t kv_read_ctrl_reg;
136 kv_read_filter_metrics_t kv_read_metrics;
137 kv_write_filter_metrics_t kv_write_metrics;
138
139 //KV Read Data Present
140 logic kv_read_data_present;
141 logic kv_read_data_present_set, kv_read_data_present_reset;
142 //PCR Hash Extend Function
143 logic pcr_hash_extend_ip;
144 logic pcr_hash_extend_set, pcr_hash_extend_reset;
145 logic [KV_ENTRY_ADDR_W-1:0] hash_extend_entry;
146 logic dest_data_avail;
147 logic [31:0] block_reg_lock, block_reg_lock_nxt;
148 //PCR Gen Hash Function
149 logic gen_hash_start;
150 logic gen_hash_ip;
151 logic gen_hash_init_reg;
152 logic gen_hash_next_reg;
153 logic gen_hash_last_reg;
154 logic gen_hash_block_write_en;
155 logic [4:0] gen_hash_block_write_offset;
156 logic [31:0] gen_hash_block_write_data;
157
158 logic [NONCE_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] pv_nonce;
159 //-----
160 // Wires.
161 //-----
162 wire core_ready;
163 wire [BLOCK_SIZE-1 : 0] core_block;
164 wire [DIG_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] core_digest;
165 wire core_di-
gest_valid;
166 logic [DIG_NUM_DWORDS-1 : 0][DATA_WIDTH-1 : 0] re-
store_digest;
167
168 //-----
169 // Concurrent connectivity for ports etc.
170 //-----
171 assign core_block = { block_reg[00], block_reg[01],
block_reg[02], block_reg[03],
block_reg[04], block_reg[05],
block_reg[06], block_reg[07],
block_reg[08], block_reg[09],
block_reg[10], block_reg[11],
block_reg[12], block_reg[13],
block_reg[14], block_reg[15],
block_reg[16], block_reg[17],
block_reg[18], block_reg[19],
block_reg[20], block_reg[21],
block_reg[22], block_reg[23],
block_reg[24], block_reg[25],
block_reg[26], block_reg[27],
block_reg[28], block_reg[29],
block_reg[30], block_reg[31]};
172

```



```

180 assign err = read_error | write_error;
181
182 always_comb begin // ecc_reg_writing
183     for (int dword=0; dword < PCR_HASH_NUM_DWORDS; dword+
+)begin
184         pcr_signing_hash[dword] =
pcr_sign_reg[(PCR_HASH_NUM_DWORDS-1)-dword];
185     end
186 end
187
188 //-----
189 // core instantiation.
190 //-----
191 sha512_core core(
192     .clk(clk),
193     .reset_n(reset_n),
194     .zeroize(zeroize_reg),
195
196     .init_cmd(init_reg),
197     .next_cmd(next_reg),
198     .restore_cmd(restore_reg),
199     .mode(mode_reg),
200
201     .block_msg(core_block),
202     .restore_digest(restore_digest),
203
204     .ready(core_ready),
205
206     .digest(core_digest),
207     .digest_valid(core_digest_valid)
208 );
209
210
211 //-----
212 // reg_update
213 //
214 // Update functionality for all registers in the core.
215 // All registers are positive edge triggered with asynchron-
ous
216 // active low reset. All registers have write enable.
217 //-----
218 always @ (posedge clk or negedge reset_n) begin : reg_update
219     if (!reset_n) begin
220         ready_reg         <= '0;
221         digest_reg        <= '0;
222         digest_valid_reg  <= '0;
223         kv_reg            <= '0;
224         pcr_sign_reg      <= '0;
225         block_reg_lock    <= '0;
226         pcr_hash_extend_ip <= '0;
227         hash_extend_entry <= '0;
228         kv_read_data_present <= '0;
229         digest_we         <= '0;
230     end
231     else if (zeroize_reg) begin
232         ready_reg         <= '0;
233         digest_reg        <= '0;
234         digest_valid_reg  <= '0;
235         kv_reg            <= '0;
236         pcr_sign_reg      <= '0;
237         kv_read_data_present <= '0;
238         digest_we         <= '0;
239         block_reg_lock    <= '0;
240     end
241     else begin
242         ready_reg         <= core_ready;
243         digest_valid_reg  <= core_digest_valid;
244
245         if (core_digest_valid & ~digest_valid_reg &
~(dest_keyvault | kv_read_data_present)) begin
246             digest_reg <= core_digest;
247             digest_we <= '1'b1;
248         end
249         else
250             digest_we <= '0;
251         if (core_digest_valid & ~digest_valid_reg &
(dest_keyvault | kv_read_data_present))
252             kv_reg <= core_digest[DI6_NUM_DWORDS-1:DI6_NUM_DWORDS-
PV_NUM_DWORDS];
253         if (pcr_sign_we)
254             pcr_sign_reg <= pcr_sign;
255
256         block_reg_lock <= block_reg_lock_nxt;
257         pcr_hash_extend_ip <= pcr_hash_extend_set ? '1 :
pcr_hash_extend_reset ? '0 :
258
pcr_hash_extend_ip;
259         hash_extend_entry <= pcr_hash_extend_set ?
kv_read_ctrl_reg.read_entry : hash_extend_entry;
260         kv_read_data_present <= kv_read_data_present_set ? '1 :
kv_read_data_present_reset ?
261 '0 : kv_read_data_present;
262     end
263 end // reg_update
264
265 always_comb begin
266     pcr_sign_we = (dest_data_avail & gen_hash_ip);
267     pcr_sign = core_digest[DI6_NUM_DWORDS-1:DI6_NUM_DWORDS-

```

```

PCR_HASH_NUM_DWORDS];
268 end
269
270 //register hw interface
271 always_comb begin
272
273     hwif_in.SHA512_NAME[0].NAME.next = SHA512_CORE_NAME0;
274     hwif_in.SHA512_NAME[1].NAME.next = SHA512_CORE_NAME1;
275
276     hwif_in.SHA512_VERSION[0].VERSION.next =
SHA512_CORE_VERSION0;
277     hwif_in.SHA512_VERSION[1].VERSION.next =
SHA512_CORE_VERSION1;
278
279     //Mask commands when keyvault is busy to prevent runs with
partial keys.
280     init_reg = gen_hash_ip ? gen_hash_init_reg :
(hwif_out.SHA512_CTRL.INIT.value & kv_src_ready);
281     next_reg = gen_hash_ip ? gen_hash_next_reg :
(hwif_out.SHA512_CTRL.NEXT.value & kv_src_ready);
282     restore_reg = gen_hash_ip ? '0 :
(hwif_out.SHA512_CTRL.RESTORE.value & kv_src_ready);
283     mode_reg = gen_hash_ip ? MODE_SHA_512 :
hwif_out.SHA512_CTRL.MODE.value;
284     zeroize_reg = hwif_out.SHA512_CTRL.ZEROIZE.value || debu-
gUnlock_or_scan_mode_switch;
285     last_reg = gen_hash_ip ? gen_hash_last_reg :
hwif_out.SHA512_CTRL.LAST.value;
286     hwif_in.SHA512_CTRL.LAST.hwclr = core_digest_valid & ~di-
gest_valid_reg;
287
288     hwif_in.SHA512_STATUS.READY.next = ready_reg;
289     hwif_in.SHA512_STATUS.VALID.next = digest_valid_reg;
290
291     //output comes in big endian
292     for (int dword=0; dword < DIG_NUM_DWORDS; dword++) begin
293         restore_digest[dword] =
hwif_out.SHA512_DIGEST[(DIG_NUM_DWORDS-1)-dword].DIGEST.value;
294         hwif_in.SHA512_DIGEST[dword].DIGEST.we = zeroize_reg?
0 : digest_we;
295         hwif_in.SHA512_DIGEST[dword].DIGEST.next = di-
gest_reg[(DIG_NUM_DWORDS-1)-dword];
296         hwif_in.SHA512_DIGEST[dword].DIGEST.hwclr = zeroize_reg;
297     end
298     //output comes in big endian
299     for (int dword=0; dword < PCR_HASH_NUM_DWORDS; dword++)
begin
300         hwif_in.SHA512_GEN_PCR_HASH_DIGEST[dword].DIGEST.next =
pcr_sign_reg[(PCR_HASH_NUM_DWORDS-1)-dword];
301         hwif_in.SHA512_GEN_PCR_HASH_DIGEST[dword].DIGEST.hwclr =
zeroize_reg;
302     end
303
304     for (int dword=0; dword < BLOCK_NUM_DWORDS; dword++) begin
305         block_reg[dword] =
hwif_out.SHA512_BLOCK[dword].BLOCK.value;
306         hwif_in.SHA512_BLOCK[dword].BLOCK.we = zeroize_reg ?
0 :
307         gen_hash_ip ?
gen_hash_block_write_en & (gen_hash_block_write_offset == dword) :
(kv_src_write_en
& (kv_src_write_offset == dword));
308         hwif_in.SHA512_BLOCK[dword].BLOCK.next = gen_hash_ip ?
gen_hash_block_write_data : kv_src_write_data;
309         hwif_in.SHA512_BLOCK[dword].BLOCK.hwclr = zeroize_reg |
kv_read_data_present_reset | (kv_src_error != KV_SUCCESS);
310         //lock the block during gen hash operation
311         hwif_in.SHA512_BLOCK[dword].BLOCK.swwel = gen_hash_ip ?
'1 : block_reg_lock[dword];
312     end
313
314     //Set valid when fsm is done
315     hwif_in.SHA512_VAULT_RD_STATUS.ERROR.next = kv_src_error;
316     hwif_in.SHA512_KV_WR_STATUS.ERROR.next = kv_dest_error;
317     //Set valid when fsm is done
318     hwif_in.SHA512_VAULT_RD_STATUS.READY.next = kv_src_ready;
319     hwif_in.SHA512_KV_WR_STATUS.READY.next = kv_dest_ready;
320     //Set valid when fsm is done
321     hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwset = kv_src_done;
322     hwif_in.SHA512_KV_WR_STATUS.VALID.hwset = kv_dest_done;
323     //reset valid when fsm is started
324     hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwclr =
kv_read_ctrl_reg.read_en;
325     hwif_in.SHA512_KV_WR_STATUS.VALID.hwclr =
kv_write_ctrl_reg.write_en;
326     //clear en when busy
327     hwif_in.SHA512_VAULT_RD_CTRL.read_en.hwclr =
~kv_src_ready;
328     hwif_in.SHA512_KV_WR_CTRL.write_en.hwclr = ~kv_dest_ready;
329
330     gen_hash_start =
hwif_out.SHA512_GEN_PCR_HASH_CTRL.START.value;
331     hwif_in.SHA512_GEN_PCR_HASH_STATUS.READY.next =
~gen_hash_ip & ~pcr_hash_extend_ip & ready_reg;
332     hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwset =
gen_hash_ip & dest_data_avail;
333     hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwclr =
gen_hash_start;
334
335 end

```

```

336
337 // Software write-enables to prevent KV reg manipulation
mid-operation
338 always_comb
hwif_in.SHA512_VAULT_RD_CTRL.read_en.swwe = !
kv_read_data_present && ready_reg && !gen_hash_ip;
339 always_comb
hwif_in.SHA512_VAULT_RD_CTRL.read_entry.swwe = !
kv_read_data_present && ready_reg && !gen_hash_ip;
340 always_comb
hwif_in.SHA512_VAULT_RD_CTRL.pcr_hash_extend.swwe = !
kv_read_data_present && ready_reg && !gen_hash_ip;
341 always_comb
hwif_in.SHA512_VAULT_RD_CTRL.rsvd.swwe = 0;
342
343 // KV write control must be written before SHA core opera-
tion begins, even though
344 // output isn't written to KV until the end of the opera-
tion.
345 // Prevent partial-key attacks by blocking register modi-
fications during core execution.
346 always_comb
hwif_in.SHA512_KV_WR_CTRL.write_en.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
347 always_comb
hwif_in.SHA512_KV_WR_CTRL.write_entry.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
348 always_comb
hwif_in.SHA512_KV_WR_CTRL.hmac_key_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
349 always_comb
hwif_in.SHA512_KV_WR_CTRL.hmac_block_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
350 always_comb
hwif_in.SHA512_KV_WR_CTRL.mldsa_seed_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
351 always_comb
hwif_in.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
352 always_comb
hwif_in.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
353 always_comb
hwif_in.SHA512_KV_WR_CTRL.aes_key_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
354 always_comb
hwif_in.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
355 always_comb
hwif_in.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
356 always_comb
hwif_in.SHA512_KV_WR_CTRL.dma_data_dest_valid.swwe = ready_reg &&
kv_dest_ready && !gen_hash_ip;
357 always_comb
hwif_in.SHA512_KV_WR_CTRL.rsvd.swwe = 0;
358
359 `CALIPTRA_KV_WRITE_CTRL_REG2STRUCT(kv_write_ctrl_reg,
SHA512_KV_WR_CTRL)
360 `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_read_ctrl_reg,
SHA512_VAULT_RD_CTRL)
361
362 //Force result into KV reg whenever source came from KV
363 always_comb kv_read_data_present_set =
kv_read_ctrl_reg.read_en & ~kv_read_ctrl_reg.pcr_hash_extend;
364 always_comb kv_read_data_present_reset =
kv_read_data_present & dest_data_avail;
365
366 //Hash extend logic
367 always_comb pcr_hash_extend_set = kv_read_ctrl_reg.read_en &
kv_read_ctrl_reg.pcr_hash_extend;
368 always_comb pcr_hash_extend_reset = pcr_hash_extend_ip &
kv_dest_done;
369
370 //set the lock for the part of the block being written by KV
logic during PCR hash extend
371 //release the lock once init has been seen
372 always_comb begin
373   for (int dword=0; dword< BLOCK_NUM_DWORDS; dword++) begin
374     if (init_reg | next_reg) begin
375       block_reg_lock_nxt[dword] = '0;
376     end
377   else begin
378     //Lock the block for any keyvault data
379     block_reg_lock_nxt[dword] = (kv_src_write_en &
(kv_src_write_offset == dword)) ? '1 : block_reg_lock[dword];
380   end
381 end
382 end
383
384 // Register Block
385 sha512_reg i_sha512_reg (
386   .clk(clk),
387   .rst(1'b0),
388
389   .s_cpuif_req      (cs),
390   .s_cpuif_req_is_wr (we),
391   .s_cpuif_addr     (address[SHA512_REG_ADDR_WIDTH-1:0]),
392   .s_cpuif_wr_data   (write_data),
393   .s_cpuif_wr_bitlen ('1),

```

```

394   .s_cpuif_req_stall_wr( ),
395   .s_cpuif_req_stall_rd( ),
396   .s_cpuif_rd_ack      ( ),
397   .s_cpuif_rd_err      (read_error),
398   .s_cpuif_rd_data     (read_data),
399   .s_cpuif_wr_ack      ( ),
400   .s_cpuif_wr_err      (write_error),
401
402   .hwif_in (hwif_in ),
403   .hwif_out(hwif_out)
404 );
405
406 //interrupt register hw interface
407 assign hwif_in.reset_b = reset_n;
408 assign hwif_in.error_reset_b = cptra_pwrgood;
409 assign hwif_in.sha512_ready = ready_reg;
410 assign hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset = core_di-
gest_valid & ~digest_valid_reg;
411 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset = 1'b0; // TODO
412 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset = 1'b0; // TODO
413 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset = 1'b0; // TODO
414 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset = 1'b0; // TODO
415
416 assign error_intr = hwif_out.intr_b-
lock_rf.error_global_intr_r.intr;
417 assign notif_intr = hwif_out.intr_b-
lock_rf.notif_global_intr_r.intr;
418
419 //Read Block
420 always_comb pv_read = gen_hash_ip ? gen_hash_pv_read :
pcr_hash_extend_ip ? vault_read : '0;
421
422 always_comb vault_rd_resp = pv_rd_resp;
423
424
425 kv_read_client #(
426   .DATA_WIDTH(BLOCK_SIZE),
427   .PAD(1),
428   .LEN_CHECK(0)
429 )
430 sha512_block_kv_read
431 (
432   .clk(clk),
433   .rst_b(reset_n),
434   .zeroize(zeroize_reg),
435
436   //client control register
437   .read_ctrl_reg(kv_read_ctrl_reg),
438   .read_metrics(kv_read_metrics),
439
440   //interface with kv
441   .kv_read(vault_read),
442   .kv_resp(vault_rd_resp),
443
444   //interface with client
445   .write_en(kv_src_write_en),
446   .write_offset(kv_src_write_offset),
447   .write_data(kv_src_write_data),
448
449   .error_code(kv_src_error),
450   .kv_ready(kv_src_ready),
451   .read_done(kv_src_done),
452   .check_key_size(1'b0),
453   .expected_key_size('0)
454 );
455
456
457 always_comb begin
458   pv_write.write_data = pcr_hash_extend_ip ?
vault_write.write_data : '0;
459   pv_write.write_en = pcr_hash_extend_ip ?
vault_write.write_en : '0;
460   pv_write.write_entry = pcr_hash_extend_ip ?
vault_write.write_entry : '0;
461   pv_write.write_offset = pcr_hash_extend_ip ?
vault_write.write_offset : '0;
462 end
463 always_comb vault_wr_resp = pv_wr_resp;
464
465 //during PCR hash extend overload write control
466 //force write enable and always write to the source address
that we read from
467 always_comb begin
468   kv_write_ctrl_reg.q = kv_write_ctrl_reg;
469   kv_write_ctrl_reg.q.write_en = ~pcr_hash_extend_ip ?
kv_write_ctrl_reg.write_en : '1;
470   kv_write_ctrl_reg.q.write_entry = ~pcr_hash_extend_ip ?
kv_write_ctrl_reg.write_entry : hash_extend_entry;
471 end
472
473 //write out the dest data to KV or PCR on last iteration of
SHA
474 always_comb dest_data_avail = core_digest_valid & ~di-
gest_valid_reg & last_reg;
475
476 always_comb begin
477   for (int dword=0; dword< NONCE_NUM_DWORDS; dword++) begin

```

```

478     pv_nonce[dword] =
hwif_out.SHA512_GEN_PCR_HASH_NONCE[dword].NONCE.value;
479     end
480 end
481 //No filtering rules for SHA512, goes to PCR vault
482 always_comb begin
483     kv_read_metrics = '{default: '0}';
484     kv_write_metrics = '{default: '0}';
485 end
486
487 kv_write_client #(
488     .DATA_WIDTH(PV_NUM_DWORDS*PV_DATA_W)
489 )
490 sha512_result_kv_write
491 (
492     .clk(clk),
493     .rst_b(reset_n),
494     .zeroize(zeroize_reg),
495     //client control register
496     .write_ctrl_reg(kv_write_ctrl_reg_q),
497     .num_dwords(PV_NUM_DWORDS[4:0]),
498     .write_metrics(kv_write_metrics),
499     //interface with kv
500     .kv_write(vault_write),
501     .kv_resp(vault_wr_resp),
502     //interface with client
503     .dest_keyvault(dest_keyvault),
504     .dest_data_avail(dest_data_avail),
505     .dest_data(kv_reg),
506     .error_code(kv_dest_error),
507     .kv_ready(kv_dest_ready),
508     .dest_done(kv_dest_done)
509 );
510
511 pv_gen_hash
512 pv_gen_hash1
513 (
514     .clk(clk),
515     .rst_b(reset_n),
516     .zeroize(zeroize_reg),
517     .core_ready(ready_reg),
518     .core_digest_valid(gen_hash_ip & dest_data_avail),
519     .start(gen_hash_start),
520     .nonce(pv_nonce),
521     .gen_hash_ip(gen_hash_ip),
522     .gen_hash_init_reg(gen_hash_init_reg),
523     .gen_hash_next_reg(gen_hash_next_reg),
524     .gen_hash_last_reg(gen_hash_last_reg),
525     .block_we(gen_hash_block_write_en),
526     .block_offset(gen_hash_block_write_offset),
527     .block_wr_data(gen_hash_block_write_data),
528     .pv_read(gen_hash_pv_read),
529     .pv_rd_resp(pv_rd_resp)
530 );
531
532 endmodule // sha512
533
534 //
535
536
537
538
539
540
541
542
543
544
545 // EOF sha512.sv
546 //

```

2. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_core.v (609 lines)

```

1 //
=====
2 //
3 // sha512_core.v
4 // -----
5 // Verilog 2001 implementation of the SHA-512 hash function.
6 // This is the internal core with wide interfaces.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2014 Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
15 // lowing conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
18 // copyright notice, this list of conditions and the following dis-

```

```

claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright notice, this list of conditions and the following dis-
22 // claimer in the documentation and/or other materials provided with
23 // the distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
26 // TRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
27 // BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
28 // FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
29 // THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
30 // INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
31 // (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
32 // VICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
33 // HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 module sha512_core(
41     // Clock and reset.
42     input wire clk,
43     input wire reset_n,
44     input wire zeroize,
45     // Control.
46     input wire init_cmd,
47     input wire next_cmd,
48     input wire restore_cmd,
49     input wire [1 : 0] mode,
50
51     // Data port.
52     input wire [1023 : 0] block_msg,
53     input wire [511 : 0] restore_digest,
54
55     output wire ready,
56     output wire [511 : 0] digest,
57     output wire digest_valid
58 );
59
60
61 //-----
62 // Internal constant and parameter definitions.
63 //-----
64
65 localparam SHA512_ROUNDS = 79;
66
67 localparam CTRL_IDLE = 2'h0;
68 localparam CTRL_ROUNDS = 2'h1;
69 localparam CTRL_DONE = 2'h2;
70
71 //-----
72 // Registers including update variables and write enable.
73 //-----
74
75 reg [63 : 0] a_reg;
76 reg [63 : 0] a_new;
77 reg [63 : 0] b_reg;
78 reg [63 : 0] b_new;
79 reg [63 : 0] c_reg;
80 reg [63 : 0] c_new;
81 reg [63 : 0] d_reg;
82 reg [63 : 0] d_new;
83 reg [63 : 0] e_reg;
84 reg [63 : 0] e_new;
85 reg [63 : 0] f_reg;
86 reg [63 : 0] f_new;
87 reg [63 : 0] g_reg;
88 reg [63 : 0] g_new;
89 reg [63 : 0] h_reg;
90 reg [63 : 0] h_new;
91 reg a_h_we;
92
93 reg [63 : 0] H0_reg;
94 reg [63 : 0] H0_new;
95 reg [63 : 0] H1_reg;
96 reg [63 : 0] H1_new;
97 reg [63 : 0] H2_reg;
98 reg [63 : 0] H2_new;
99 reg [63 : 0] H3_reg;
100 reg [63 : 0] H3_new;

```

```

101 reg [63 : 0] H4_new;
102 reg [63 : 0] H5_reg;
103 reg [63 : 0] H5_new;
104 reg [63 : 0] H6_reg;
105 reg [63 : 0] H6_new;
106 reg [63 : 0] H7_reg;
107 reg [63 : 0] H7_new;
108 reg      H_we;
109
110 reg [6 : 0] round_ctr_reg;
111 reg [6 : 0] round_ctr_new;
112 reg      round_ctr_we;
113 reg      round_ctr_inc;
114 reg      round_ctr_rst;
115
116 reg      ready_reg;
117 reg      ready_new;
118 reg      ready_we;
119
120 reg      digest_valid_reg;
121 reg      digest_valid_new;
122 reg      digest_valid_we;
123
124 reg [1 : 0] sha512_ctrl_reg;
125 reg [1 : 0] sha512_ctrl_new;
126 reg      sha512_ctrl_we;
127
128 //-----
129
130 // Wires.
131 //-----
132
133 reg digest_init;
134 reg digest_update;
135
136 reg state_init;
137 reg state_update;
138
139 reg first_block;
140 reg restore;
141
142 reg [63 : 0] t1;
143 reg [63 : 0] t2;
144
145 wire [63 : 0] k_data;
146
147 reg      w_init;
148 reg      w_next;
149 wire [63 : 0] w_data;
150
151 wire [63 : 0] H0_0;
152 wire [63 : 0] H0_1;
153 wire [63 : 0] H0_2;
154 wire [63 : 0] H0_3;
155 wire [63 : 0] H0_4;
156 wire [63 : 0] H0_5;
157 wire [63 : 0] H0_6;
158 wire [63 : 0] H0_7;
159
160 //-----
161 // Module instantiations.
162 //-----
163
164 sha512_k_constants k_constants_inst(
165     .addr(round_ctr_reg),
166     .K_val(k_data)
167 );
168
169 sha512_h_constants h_constants_inst(
170     .mode(mode),
171     .H0(H0_0),
172     .H1(H0_1),
173     .H2(H0_2),
174     .H3(H0_3),
175     .H4(H0_4),
176     .H5(H0_5),
177     .H6(H0_6),
178     .H7(H0_7)
179 );
180
181
182 sha512_w_mem w_mem_inst(
183     .clk(clk),
184     .reset_n(reset_n),
185     .zeroize(zeroize),
186     .block_msg(block_msg),
187     .init_cmd(w_init),
188     .next_cmd(w_next),
189     .w_val(w_data)
190 );
191
192 //-----
193
194
195
196
197 // Concurrent connectivity for ports etc.
198 //-----
199
200 assign ready = ready_reg;
201
202 assign digest = {H0_reg, H1_reg, H2_reg, H3_reg,
203                 H4_reg, H5_reg, H6_reg, H7_reg};
204
205 assign digest_valid = digest_valid_reg;
206
207 //-----
208
209 // reg_update
210 // Update functionality for all registers in the core.
211 // All registers are positive edge triggered with asynchron-
212 // active low reset. All registers have write enable.
213 //-----
214
215 always @ (posedge clk or negedge reset_n)
216 begin : reg_update
217     if (!reset_n)
218     begin
219         a_reg      <= 64'h0;
220         b_reg      <= 64'h0;
221         c_reg      <= 64'h0;
222         d_reg      <= 64'h0;
223         e_reg      <= 64'h0;
224         f_reg      <= 64'h0;
225         g_reg      <= 64'h0;
226         h_reg      <= 64'h0;
227         H0_reg     <= 64'h0;
228         H1_reg     <= 64'h0;
229         H2_reg     <= 64'h0;
230         H3_reg     <= 64'h0;
231         H4_reg     <= 64'h0;
232         H5_reg     <= 64'h0;
233         H6_reg     <= 64'h0;
234         H7_reg     <= 64'h0;
235         ready_reg  <= 1'b1;
236         digest_valid_reg <= 1'b0;
237         round_ctr_reg <= 7'h0;
238         sha512_ctrl_reg <= CTRL_IDLE;
239     end
240     else if (zeroize)
241     begin
242         a_reg      <= 64'h0;
243         b_reg      <= 64'h0;
244         c_reg      <= 64'h0;
245         d_reg      <= 64'h0;
246         e_reg      <= 64'h0;
247         f_reg      <= 64'h0;
248         g_reg      <= 64'h0;
249         h_reg      <= 64'h0;
250         H0_reg     <= 64'h0;
251         H1_reg     <= 64'h0;
252         H2_reg     <= 64'h0;
253         H3_reg     <= 64'h0;
254         H4_reg     <= 64'h0;
255         H5_reg     <= 64'h0;
256         H6_reg     <= 64'h0;
257         H7_reg     <= 64'h0;
258         ready_reg  <= 1'b1;
259         digest_valid_reg <= 1'b0;
260         round_ctr_reg <= 7'h0;
261         sha512_ctrl_reg <= CTRL_IDLE;
262     end
263     else
264     begin
265         if (a_h_we)
266         begin
267             a_reg <= a_new;
268             b_reg <= b_new;
269             c_reg <= c_new;
270             d_reg <= d_new;
271             e_reg <= e_new;
272             f_reg <= f_new;
273             g_reg <= g_new;
274             h_reg <= h_new;
275         end
276
277         if (H_we)
278         begin
279             H0_reg <= H0_new;
280             H1_reg <= H1_new;
281             H2_reg <= H2_new;
282             H3_reg <= H3_new;
283             H4_reg <= H4_new;
284             H5_reg <= H5_new;
285             H6_reg <= H6_new;
286             H7_reg <= H7_new;
287         end
288
289         if (round_ctr_we)
290             round_ctr_reg <= round_ctr_new;
291
292         if (ready_we)
293             ready_reg <= ready_new;

```

```

294
295     if (digest_valid_we)
296         digest_valid_reg <= digest_valid_new;
297
298     if (sha512_ctrl_we)
299         sha512_ctrl_reg <= sha512_ctrl_new;
300     end
301 end // reg_update
302
303
304 //-----
305 // digest_logic
306 //
307 // The logic needed to init as well as update the digest.
308 //-----
309
310 always @*
311 begin : digest_logic
312     H0_new = 64'h0;
313     H1_new = 64'h0;
314     H2_new = 64'h0;
315     H3_new = 64'h0;
316     H4_new = 64'h0;
317     H5_new = 64'h0;
318     H6_new = 64'h0;
319     H7_new = 64'h0;
320     H_we = 0;
321
322     if (digest_init)
323     begin
324         H0_new = H0_0;
325         H1_new = H0_1;
326         H2_new = H0_2;
327         H3_new = H0_3;
328         H4_new = H0_4;
329         H5_new = H0_5;
330         H6_new = H0_6;
331         H7_new = H0_7;
332         H_we = 1;
333     end
334     else if (restore)
335     begin
336         H0_new = restore_digest[511 : 448];
337         H1_new = restore_digest[447 : 384];
338         H2_new = restore_digest[383 : 320];
339         H3_new = restore_digest[319 : 256];
340         H4_new = restore_digest[255 : 192];
341         H5_new = restore_digest[191 : 128];
342         H6_new = restore_digest[127 : 64];
343         H7_new = restore_digest[63 : 0];
344         H_we = 1;
345     end
346
347     if (digest_update)
348     begin
349         H0_new = H0_reg + a_reg;
350         H1_new = H1_reg + b_reg;
351         H2_new = H2_reg + c_reg;
352         H3_new = H3_reg + d_reg;
353         H4_new = H4_reg + e_reg;
354         H5_new = H5_reg + f_reg;
355         H6_new = H6_reg + g_reg;
356         H7_new = H7_reg + h_reg;
357         H_we = 1;
358     end
359 end // digest_logic
360
361 //-----
362 // t1_logic
363 //
364 // The logic for the T1 function.
365 //-----
366
367 always @*
368 begin : t1_logic
369     reg [63 : 0] sum1;
370     reg [63 : 0] ch;
371
372     sum1 = {e_reg[13 : 0], e_reg[63 : 14]} ^
373           {e_reg[17 : 0], e_reg[63 : 18]} ^
374           {e_reg[40 : 0], e_reg[63 : 41]};
375
376     ch = (e_reg & f_reg) ^ ((~e_reg) & g_reg);
377
378     t1 = h_reg + sum1 + ch + k_data + w_data;
379 end // t1_logic
380
381 //-----
382 // t2_logic
383 //
384 // The logic for the T2 function
385 //-----
386
387 always @*
388 begin : t2_logic
389     reg [63 : 0] sum0;
389
390     reg [63 : 0] maj;
391
392     sum0 = {a_reg[27 : 0], a_reg[63 : 28]} ^
393           {a_reg[33 : 0], a_reg[63 : 34]} ^
394           {a_reg[38 : 0], a_reg[63 : 39]};
395
396     maj = (a_reg & b_reg) ^ (a_reg & c_reg) ^ (b_reg &
397 c_reg);
398
399     t2 = sum0 + maj;
400 end // t2_logic
401
402 //-----
403 // state_logic
404 //
405 // The logic needed to init as well as update the state dur-
406 ing
407 // round processing.
408 //-----
409
410 always @*
411 begin : state_logic
412     a_new = 64'h0;
413     b_new = 64'h0;
414     c_new = 64'h0;
415     d_new = 64'h0;
416     e_new = 64'h0;
417     f_new = 64'h0;
418     g_new = 64'h0;
419     h_new = 64'h0;
420     a_h_we = 0;
421
422     if (state_init)
423     begin
424         if (first_block)
425         begin
426             a_new = H0_0;
427             b_new = H0_1;
428             c_new = H0_2;
429             d_new = H0_3;
430             e_new = H0_4;
431             f_new = H0_5;
432             g_new = H0_6;
433             h_new = H0_7;
434             a_h_we = 1;
435         end
436         else if (restore)
437         begin
438             a_new = restore_digest[511 : 448];
439             b_new = restore_digest[447 : 384];
440             c_new = restore_digest[383 : 320];
441             d_new = restore_digest[319 : 256];
442             e_new = restore_digest[255 : 192];
443             f_new = restore_digest[191 : 128];
444             g_new = restore_digest[127 : 64];
445             h_new = restore_digest[63 : 0];
446             a_h_we = 1;
447         end
448         else
449         begin
450             a_new = H0_reg;
451             b_new = H1_reg;
452             c_new = H2_reg;
453             d_new = H3_reg;
454             e_new = H4_reg;
455             f_new = H5_reg;
456             g_new = H6_reg;
457             h_new = H7_reg;
458             a_h_we = 1;
459         end
460     end
461
462     if (state_update)
463     begin
464         a_new = t1 + t2;
465         b_new = a_reg;
466         c_new = b_reg;
467         d_new = c_reg;
468         e_new = d_reg + t1;
469         f_new = e_reg;
470         g_new = f_reg;
471         h_new = g_reg;
472         a_h_we = 1;
473     end
474 end // state_logic
475
476 //-----
477 // round_ctr
478 //
479 // Update logic for the round counter, a monotonically
480 // increasing counter with reset.
481 //-----
482
483 always @*
484 begin : round_ctr
485     round_ctr_new = 7'h00;
486     round_ctr_we = 0;

```

```

484
485   if (round_ctr_rst)
486   begin
487     round_ctr_new = 7'h00;
488     round_ctr_we = 1;
489   end
490
491   if (round_ctr_inc)
492   begin
493     round_ctr_new = round_ctr_reg + 1'b1;
494     round_ctr_we = 1;
495   end
496   end // round_ctr
497
498 //-----
499
500 // sha512_ctrl_fsm
501 //
502 // Logic for the state machine controlling the core beha-
503 //-----
504
505 always @*
506 begin : sha512_ctrl_fsm
507   digest_init      = 1'b0;
508   digest_update    = 1'b0;
509   state_init       = 1'b0;
510   state_update     = 1'b0;
511   first_block      = 1'b0;
512   w_init           = 1'b0;
513   w_next           = 1'b0;
514   round_ctr_inc    = 1'b0;
515   round_ctr_rst    = 1'b0;
516   digest_valid_new = 1'b0;
517   digest_valid_we  = 1'b0;
518   ready_new        = 1'b0;
519   ready_we         = 1'b0;
520   sha512_ctrl_new  = CTRL_IDLE;
521   sha512_ctrl_we   = 1'b0;
522   restore          = 1'b0;
523
524   unique case (sha512_ctrl_reg)
525     CTRL_IDLE:
526       begin
527         if (init_cmd)
528         begin
529           ready_new      = 1'b0;
530           ready_we       = 1'b1;
531           digest_init    = 1;
532           w_init         = 1;
533           state_init     = 1;
534           first_block    = 1;
535           round_ctr_rst  = 1;
536           digest_valid_new = 0;
537           digest_valid_we = 1;
538           sha512_ctrl_new = CTRL_ROUNDS;
539           sha512_ctrl_we = 1;
540         end
541
542         if (next_cmd)
543         begin
544           ready_new      = 1'b0;
545           ready_we       = 1'b1;
546           w_init         = 1;
547           state_init     = 1;
548           round_ctr_rst  = 1;
549           digest_valid_new = 0;
550           digest_valid_we = 1;
551           sha512_ctrl_new = CTRL_ROUNDS;
552           sha512_ctrl_we = 1;
553           restore        = restore_cmd;
554         end
555       end
556
557     CTRL_ROUNDS:
558       begin
559         w_next      = 1;
560         state_update = 1;
561         round_ctr_inc = 1;
562
563         if (round_ctr_reg == SHA512_ROUNDS)
564         begin
565           sha512_ctrl_new = CTRL_DONE;
566           sha512_ctrl_we = 1;
567         end
568       end
569
570     CTRL_DONE:
571       begin
572         ready_new      = 1'b1;
573         ready_we       = 1'b1;
574         digest_update  = 1'b1;
575         digest_valid_new = 1'b1;
576         digest_valid_we = 1'b1;
577         sha512_ctrl_new = CTRL_IDLE;
578         sha512_ctrl_we = 1'b1;
579       end
580   end
581

```

```

582
583   default:
584   begin
585     digest_init      = 1'b0;
586     digest_update    = 1'b0;
587     state_init       = 1'b0;
588     state_update     = 1'b0;
589     first_block      = 1'b0;
590     w_init           = 1'b0;
591     w_next           = 1'b0;
592     round_ctr_inc    = 1'b0;
593     round_ctr_rst    = 1'b0;
594     digest_valid_new = 1'b0;
595     digest_valid_we  = 1'b0;
596     ready_new        = 1'b0;
597     ready_we         = 1'b0;
598     sha512_ctrl_new  = CTRL_IDLE;
599     sha512_ctrl_we   = 1'b0;
600   end
601
602   endcase // case (sha512_ctrl_reg)
603 end // sha512_ctrl_fsm
604
605 endmodule // sha512_core
606
607 //
=====
==
608 // EOF sha512_core.v
609 //
=====
==

```

3. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_ctrl.sv (138 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // you may not use this file except in compliance with the Li-
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
12 // See the License for the specific language governing permis-
13 // limitations under the License.
14 //
15 //
=====
==
16 //
17 // sha512_ctrl.sv
18 // -----
19 // SHA512 controller for the AHB_lite interface.
20 //
21 //
22 //
=====
==
23
24 `include "kv_macros.svh"
25
26 module sha512_ctrl
27 import kv_defines_pkg::*;
28 import pv_defines_pkg::*;
29 #(
30   parameter AHB_DATA_WIDTH = 32,
31   parameter AHB_ADDR_WIDTH = 32
32 )
33 (
34   // Clock and reset.
35   input wire      clk,
36   input wire      reset_n,
37   input wire      cptra_pwrgood,
38
39   // from SLAVES PORT
40   input logic [AHB_ADDR_WIDTH-1:0] haddr_i,
41   input logic [AHB_DATA_WIDTH-1:0] hwddata_i,
42   input logic hsel_i,
43   input logic hwrite_i,
44   input logic hready_i,
45   input logic [1:0] htrans_i,
46   input logic [2:0] hsize_i,
47
48   output logic hresp_o,
49   output logic hreadyout_o,
50   output logic [AHB_DATA_WIDTH-1:0] hrdata_o,
51
52   // pcr vault interface
53   output pv_read_t pv_read,
54   output pv_write_t pv_write,

```



```

55 input pv_rd_resp_t pv_rd_resp,
56 input pv_wr_resp_t pv_wr_resp,
57 output logic [PCR_HASH_NUM_DWORDS-1:0][31:0] pcr_sign-
ing_hash,
58
59 // Interrupt
60 output logic error_intr,
61 output logic notif_intr,
62 input logic debugUnlock_or_scan_mode_switch
63 );
64
65 //-----
66 // sha512
67 //-----
68 logic sha512_cs;
69 logic sha512_we;
70 logic [AHB_ADDR_WIDTH - 1 : 0] sha512_address;
71 logic [31 : 0] sha512_write_data;
72 logic [31 : 0] sha512_read_data;
73 logic sha512_err;
74
75 sha512 #(
76 .ADDR_WIDTH(AHB_ADDR_WIDTH),
77 .DATA_WIDTH(32)
78 )
79 sha512_inst(
80 .clk(clk),
81 .reset_n(reset_n),
82 .cptra_pwrgood(cptra_pwrgood),
83 .cs(sha512_cs),
84 .we(sha512_we),
85 .address(sha512_address),
86 .write_data(sha512_write_data),
87 .read_data(sha512_read_data),
88 .err(sha512_err),
89 .pv_read(pv_read),
90 .pv_write(pv_write),
91 .pv_rd_resp(pv_rd_resp),
92 .pv_wr_resp(pv_wr_resp),
93 .pcr_signing_hash(pcr_signing_hash),
94 .error_intr(error_intr),
95 .notif_intr(notif_intr),
96 .debugUn-
lock_or_scan_mode_switch(debugUnlock_or_scan_mode_switch)
97 );
98
99 //instantiate ahb lite module
100 ahb_slv_sif #(
101 .AHB_ADDR_WIDTH(AHB_ADDR_WIDTH),
102 .AHB_DATA_WIDTH(AHB_DATA_WIDTH),
103 .CLIENT_DATA_WIDTH(32)
104 ) ahb_slv_sif_uut
105 (
106 //AMBA AHB Lite INF
107 .hclk(clk),
108 .hreset_n(hreset_n),
109 .haddr_i(haddr_i),
110 .hwddata_i(hwddata_i),
111 .hsel_i(hsel_i),
112 .hwrite_i(hwrite_i),
113 .hready_i(hready_i),
114 .htrans_i(htrans_i),
115 .hsize_i(hsize_i),
116
117 .hresp_o(hresp_o),
118 .hreadyout_o(hreadyout_o),
119 .hrdata_o(hrdata_o),
120
121 //COMPONENT INF
122 .dv(sha512_cs),
123 .hld(1'b0), //no holds from sha512
124 .err(1'b0),
125 .write(sha512_we),
126 .wdata(sha512_write_data),
127 .addr(sha512_address),
128 .rdata(sha512_read_data)
129 );
130
131 );
132
133 endmodule // sha512_ctrl
134
135
136 //
=====
137 // EOF sha512_ctrl.sv
138 //
=====

```

4. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_h_constants.v (156 lines)

```

1 //
=====
2 //

```

```

3 // sha512_h_constants.v
4 // -----
5 // The H initial constants for the different modes in SHA-512.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2014 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
copyright
17 // notice, this list of conditions and the following dis-
claimer.
18 //
19 // 2. Redistributions in binary form must reproduce the above
copyright
20 // notice, this list of conditions and the following dis-
claimer in
21 // the documentation and/or other materials provided with
the
22 // distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
26 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
27 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
28 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
29 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 module sha512_h_constants(
40     input wire [1 : 0] mode,
41
42     output wire [63 : 0] H0,
43     output wire [63 : 0] H1,
44     output wire [63 : 0] H2,
45     output wire [63 : 0] H3,
46     output wire [63 : 0] H4,
47     output wire [63 : 0] H5,
48     output wire [63 : 0] H6,
49     output wire [63 : 0] H7
50 );
51
52 //-----
53 // Wires.
54 //-----
55 reg [63 : 0] tmp_H0;
56 reg [63 : 0] tmp_H1;
57 reg [63 : 0] tmp_H2;
58 reg [63 : 0] tmp_H3;
59 reg [63 : 0] tmp_H4;
60 reg [63 : 0] tmp_H5;
61 reg [63 : 0] tmp_H6;
62 reg [63 : 0] tmp_H7;
63
64
65 //-----
66 // Concurrent connectivity for ports etc.
67 //-----
68 assign H0 = tmp_H0;
69 assign H1 = tmp_H1;
70 assign H2 = tmp_H2;
71 assign H3 = tmp_H3;
72 assign H4 = tmp_H4;
73 assign H5 = tmp_H5;
74 assign H6 = tmp_H6;
75 assign H7 = tmp_H7;
76
77
78 //-----
79 // mode_mux
80 //
81 // Based on the given mode, the correct H constants are se-

```



```

lected.
82 //-----
83 always @*
84   begin : mode_mux
85     unique case (mode)
86     0:
87       begin
88         // SHA-512/224
89         tmp_H0 = 64'h8c3d37c819544da2;
90         tmp_H1 = 64'h73e1996689dcd4d6;
91         tmp_H2 = 64'h1dfab7ae32ff9c82;
92         tmp_H3 = 64'h679dd514582f9fcf;
93         tmp_H4 = 64'h0f6d2b697bd44da8;
94         tmp_H5 = 64'h77e36f7304c48942;
95         tmp_H6 = 64'h3f9d85a86a1d36c8;
96         tmp_H7 = 64'h1112e6ad91d692a1;
97       end
98
99     1:
100    begin
101      // SHA-512/256
102      tmp_H0 = 64'h22312194fc2bf72c;
103      tmp_H1 = 64'h9f555fa3c84c64c2;
104      tmp_H2 = 64'h2393b86b6f53b151;
105      tmp_H3 = 64'h963877195940eabd;
106      tmp_H4 = 64'h96283ee2a88effe3;
107      tmp_H5 = 64'hbe5e1e2553863992;
108      tmp_H6 = 64'h2b0199fc2c85b8aa;
109      tmp_H7 = 64'h0eb72ddc81c52ca2;
110    end
111
112    2:
113    begin
114      // SHA-384
115      tmp_H0 = 64'hcbbb9d5dc1059ed8;
116      tmp_H1 = 64'h629a292a367cd507;
117      tmp_H2 = 64'h9159015a3070dd17;
118      tmp_H3 = 64'h152fec8f70e5939;
119      tmp_H4 = 64'h67332667ffc00b31;
120      tmp_H5 = 64'h8eb44a8768581511;
121      tmp_H6 = 64'hdb0c2e0d64f98fa7;
122      tmp_H7 = 64'h47b5481dbefa4fa4;
123    end
124
125    3:
126    begin
127      // SHA-512
128      tmp_H0 = 64'h6a09e667f3bcc908;
129      tmp_H1 = 64'hbb67ae8584caa73b;
130      tmp_H2 = 64'h3c6ef372fe94f82b;
131      tmp_H3 = 64'ha54ff53a5f1d36f1;
132      tmp_H4 = 64'h510e527fade682d1;
133      tmp_H5 = 64'h9b05688c2b3e6c1f;
134      tmp_H6 = 64'h1f83d9abfb41bd6b;
135      tmp_H7 = 64'h5be0cd19137e2179;
136    end
137
138    default:
139    begin
140      // SHA-384
141      tmp_H0 = 64'hcbbb9d5dc1059ed8;
142      tmp_H1 = 64'h629a292a367cd507;
143      tmp_H2 = 64'h9159015a3070dd17;
144      tmp_H3 = 64'h152fec8f70e5939;
145      tmp_H4 = 64'h67332667ffc00b31;
146      tmp_H5 = 64'h8eb44a8768581511;
147      tmp_H6 = 64'hdb0c2e0d64f98fa7;
148      tmp_H7 = 64'h47b5481dbefa4fa4;
149    end
150  endcase // case (addr)
151 end // block: mode_mux
152 endmodule // sha512_h_constants
153
154 //
=====
155 // sha512_h_constants.v
156 //
=====

```

5. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_k_constants.v (151 lines)

```

1 //
=====
2 //
3 // sha512_k_constants.v
4 // -----
5 // The table K with constants in the SHA-512 hash function.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2014 Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-

```

```

lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
copyright
17 // notice, this list of conditions and the following dis-
claimer.
18 //
19 // 2. Redistributions in binary form must reproduce the above
copyright
20 // notice, this list of conditions and the following dis-
claimer in
21 // the documentation and/or other materials provided with
the
22 // distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
26 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
27 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
28 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
29 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
==
38
39 module sha512_k_constants(
40     input wire [6 : 0] addr,
41     output wire [63 : 0] K_val
42 );
43
44 //-----
45 // Wires.
46 //-----
47 reg [63 : 0] tmp_K;
48
49
50 //-----
51 // Concurrent connectivity for ports etc.
52 //-----
53 assign K_val = tmp_K;
54
55 //-----
56 //-----
57 // addr_mux
58 //-----
59 always @*
60   begin : addr_mux
61     unique case(addr)
62     0: tmp_K = 64'h428a2f98d728ae22;
63     1: tmp_K = 64'h7137449123ef65cd;
64     2: tmp_K = 64'hb5c0fbcfec4d3b2f;
65     3: tmp_K = 64'he9b5dba58189dbbc;
66     4: tmp_K = 64'h3956c25bf348b538;
67     5: tmp_K = 64'h59f111f1b605d019;
68     6: tmp_K = 64'h923f82a4af194f9b;
69     7: tmp_K = 64'hab1c5ed5da6d8118;
70     8: tmp_K = 64'hd807aa98a3030242;
71     9: tmp_K = 64'h12835b0145706fbc;
72     10: tmp_K = 64'h243185be4ee4b28c;
73     11: tmp_K = 64'h550c7dc3d5fb4e2;
74     12: tmp_K = 64'h72be5d74f27b896f;
75     13: tmp_K = 64'h80deb1fe3b1696b1;
76     14: tmp_K = 64'h9bdc06a725c71235;
77     15: tmp_K = 64'hc19bf174cf692694;
78     16: tmp_K = 64'he49b69c19ef14ad2;
79     17: tmp_K = 64'hef6be4786384f25e3;
80     18: tmp_K = 64'h0fc19dc68b8cd5b5;
81     19: tmp_K = 64'h240ca1cc77ac9c65;
82     20: tmp_K = 64'h2de92c6f592b0275;
83     21: tmp_K = 64'h4a7484aa6e6a483;
84     22: tmp_K = 64'h5cb0a9dcdb41fbd4;
85     23: tmp_K = 64'h76f988da831153b5;
86     24: tmp_K = 64'h983e5152ee66dfab;
87     25: tmp_K = 64'ha831c66d2b43210;
88     26: tmp_K = 64'hb00327c898fb213f;
89     27: tmp_K = 64'hbf597fc7beef0ee4;
90     28: tmp_K = 64'hc6e00bf33da88fc2;
91     29: tmp_K = 64'hd5a79147930aa725;

```

```

92 30: tmp_K = 64'h06ca6351e003826f;
93 31: tmp_K = 64'h142929670a0e6e70;
94 32: tmp_K = 64'h27b70a8546d22ffc;
95 33: tmp_K = 64'h2e1b21385c26c926;
96 34: tmp_K = 64'h4d2c6dfc5ac42aed;
97 35: tmp_K = 64'h53380d139d95b3df;
98 36: tmp_K = 64'h650a73548baf63de;
99 37: tmp_K = 64'h76a0abb3c77b2a8;
100 38: tmp_K = 64'h81c2c92e47edae6;
101 39: tmp_K = 64'h92722c851482353b;
102 40: tmp_K = 64'ha2bfe8a14cf10364;
103 41: tmp_K = 64'ha81a664bbc423001;
104 42: tmp_K = 64'hc24b8b70d0f89791;
105 43: tmp_K = 64'hc76c51a30654be30;
106 44: tmp_K = 64'hd192e819d6ef5218;
107 45: tmp_K = 64'hd69906245565a910;
108 46: tmp_K = 64'hf40e35855771202a;
109 47: tmp_K = 64'h106aa07032bbd1b8;
110 48: tmp_K = 64'h19a4c116b8d20c8;
111 49: tmp_K = 64'h1e376c085141ab53;
112 50: tmp_K = 64'h2748774cdf8eeb99;
113 51: tmp_K = 64'h34b0bcb5e19b48a8;
114 52: tmp_K = 64'h391c0cb3c5c95a63;
115 53: tmp_K = 64'h4ed8aa4ae3418acb;
116 54: tmp_K = 64'h5b9cca4f7763e373;
117 55: tmp_K = 64'h682e6ff3d6b2b8a3;
118 56: tmp_K = 64'h748f82ee5defb2fc;
119 57: tmp_K = 64'h78a5636f43172f60;
120 58: tmp_K = 64'h84c87814a1f0ab72;
121 59: tmp_K = 64'h8cc702081a6439ec;
122 60: tmp_K = 64'h90befffa23631e28;
123 61: tmp_K = 64'ha4506cebd82bde9;
124 62: tmp_K = 64'hbef9a3f7b2c67915;
125 63: tmp_K = 64'hc67178f2e372532b;
126 64: tmp_K = 64'hca273cecea26619c;
127 65: tmp_K = 64'hd186b8c721c0c207;
128 66: tmp_K = 64'heada7d6dcde0eb1e;
129 67: tmp_K = 64'hf57d4f7fee6ed178;
130 68: tmp_K = 64'h06f067aa72176fba;
131 69: tmp_K = 64'h0a637dc5a2c898a6;
132 70: tmp_K = 64'h113f9804bef90dae;
133 71: tmp_K = 64'h1b710b35131c471b;
134 72: tmp_K = 64'h28db77f523047d84;
135 73: tmp_K = 64'h32caab7b40c72493;
136 74: tmp_K = 64'h3c9ebe0a15c9ebcb;
137 75: tmp_K = 64'h431d67c49c100d4c;
138 76: tmp_K = 64'h4cc5d4becb3e42b6;
139 77: tmp_K = 64'h597f299cfc657e2a;
140 78: tmp_K = 64'h5fcb6fab3ad6faec;
141 79: tmp_K = 64'h6c44198c4a475817;
142
143 default:
144     tmp_K = 64'h0;
145 endcase // case (addr)
146 end // block: addr_mux
147 endmodule // sha512_k_constants
148
149 //
=====
150 // sha512_k_constants.v
151 //
=====

```

6. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_params_pkg.sv (31 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0
10 //
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20 //
21 //
22 package sha512_params_pkg;
23
24     localparam SHA512_CORE_NAME0      = 32'h61327368; //
25 "sha2"
26     localparam SHA512_CORE_NAME1      = 32'h31322d35; //
27 "-512"
28     localparam SHA512_CORE_VERSION0    = 32'h3830302e; //
29 "0.80"
30     localparam SHA512_CORE_VERSION1    = 32'h00000000; //
31 "0"
32

```

```

23 localparam MODE_SHA_512_224      = 2'h0;
24 localparam MODE_SHA_512_256      = 2'h1;
25 localparam MODE_SHA_384          = 2'h2;
26 localparam MODE_SHA_512          = 2'h3;
27
28 endpackage
29 //

```

```

=====
30 // EOF sha512_params_pkg.sv
31 //
=====

```

7. chipsalliance_caliptra-rtl/src/sha512/rtl/sha512_reg.sv (2,297 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
2 temVerilog generator
3 // https://github.com/SystemRDL/PeakRDL-regblock
4
5 module sha512_reg (
6     input wire clk,
7     input wire rst,
8
9     input wire s_cpuif_req,
10    input wire s_cpuif_req_is_wr,
11    input wire [11:0] s_cpuif_addr,
12    input wire [31:0] s_cpuif_wr_data,
13    input wire [31:0] s_cpuif_wr_biten,
14    output wire s_cpuif_req_stall_wr,
15    output wire s_cpuif_req_stall_rd,
16    output wire s_cpuif_rd_ack,
17    output wire s_cpuif_rd_err,
18    output wire [31:0] s_cpuif_rd_data,
19    output wire s_cpuif_wr_ack,
20    output wire s_cpuif_wr_err,
21
22    input sha512_reg_pkg::sha512_reg_in_t hwif_in,
23    output sha512_reg_pkg::sha512_reg_out_t hwif_out
24 );
25 //-----
26 // CPU Bus interface logic
27 //-----
28 logic cpuif_req;
29 logic cpuif_req_is_wr;
30 logic [11:0] cpuif_addr;
31 logic [31:0] cpuif_wr_data;
32 logic [31:0] cpuif_wr_biten;
33 logic cpuif_req_stall_wr;
34 logic cpuif_req_stall_rd;
35
36 logic cpuif_rd_ack;
37 logic cpuif_rd_err;
38 logic [31:0] cpuif_rd_data;
39
40 logic cpuif_wr_ack;
41 logic cpuif_wr_err;
42
43 assign cpuif_req = s_cpuif_req;
44 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
45 assign cpuif_addr = s_cpuif_addr;
46 assign cpuif_wr_data = s_cpuif_wr_data;
47 assign cpuif_wr_biten = s_cpuif_wr_biten;
48 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
49 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
50 assign s_cpuif_rd_ack = cpuif_rd_ack;
51 assign s_cpuif_rd_err = cpuif_rd_err;
52 assign s_cpuif_rd_data = cpuif_rd_data;
53 assign s_cpuif_wr_ack = cpuif_wr_ack;
54 assign s_cpuif_wr_err = cpuif_wr_err;
55
56 logic cpuif_req_masked;
57
58 // Read & write latencies are balanced. Stalls not re-
59 quired
60 assign cpuif_req_stall_rd = '0;
61 assign cpuif_req_stall_wr = '0;
62 assign cpuif_req_masked = cpuif_req
63     & !(cpuif_req_is_wr &
64     cpuif_req_stall_rd)
65     & !(cpuif_req_is_wr &
66     cpuif_req_stall_wr);
67 //-----
68 // Address Decode
69 //-----
70 typedef struct packed{
71     logic [2-1:0]SHA512_NAME;
72     logic [2-1:0]SHA512_VERSION;
73     logic SHA512_CTRL;
74     logic SHA512_STATUS;
75     logic [32-1:0]SHA512_BLOCK;
76     logic [16-1:0]SHA512_DIGEST;
77     logic SHA512_VAULT_RD_CTRL;

```

```

76 logic SHA512_VAULT_RD_STATUS;
77 logic SHA512_KV_WR_CTRL;
78 logic SHA512_KV_WR_STATUS;
79 logic [8-1:0]SHA512_GEN_PCR_HASH_NONCE;
80 logic SHA512_GEN_PCR_HASH_CTRL;
81 logic SHA512_GEN_PCR_HASH_STATUS;
82 logic [16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
83 struct packed{
84     logic global_intr_en_r;
85     logic error_intr_en_r;
86     logic notif_intr_en_r;
87     logic error_global_intr_r;
88     logic notif_global_intr_r;
89     logic error_internal_intr_r;
90     logic notif_internal_intr_r;
91     logic error_intr_trig_r;
92     logic notif_intr_trig_r;
93     logic error0_intr_count_r;
94     logic error1_intr_count_r;
95     logic error2_intr_count_r;
96     logic error3_intr_count_r;
97     logic notif_cmd_done_intr_count_r;
98     logic error0_intr_count_incr_r;
99     logic error1_intr_count_incr_r;
100    logic error2_intr_count_incr_r;
101    logic error3_intr_count_incr_r;
102    logic notif_cmd_done_intr_count_incr_r;
103 } intr_block_rf;
104 } decoded_reg_strb_t;
105 decoded_reg_strb_t decoded_reg_strb;
106 logic decoded_req;
107 logic decoded_req_is_wr;
108 logic [31:0] decoded_wr_data;
109 logic [31:0] decoded_wr_bitlen;
110
111 always_comb begin
112     for(int i0=0; i0<2; i0++) begin
113         decoded_reg_strb.SHA512_NAME[i0] =
cpuif_req_masked & (cpuif_addr == 12'h0 + i0*12'h4);
114     end
115     for(int i0=0; i0<2; i0++) begin
116         decoded_reg_strb.SHA512_VERSION[i0] =
cpuif_req_masked & (cpuif_addr == 12'h8 + i0*12'h4);
117     end
118     decoded_reg_strb.SHA512_CTRL = cpuif_req_masked &
(cpuif_addr == 12'h10);
119     decoded_reg_strb.SHA512_STATUS = cpuif_req_masked &
(cpuif_addr == 12'h18);
120     for(int i0=0; i0<32; i0++) begin
121         decoded_reg_strb.SHA512_BLOCK[i0] =
cpuif_req_masked & (cpuif_addr == 12'h80 + i0*12'h4);
122     end
123     for(int i0=0; i0<16; i0++) begin
124         decoded_reg_strb.SHA512_DIGEST[i0] =
cpuif_req_masked & (cpuif_addr == 12'h100 + i0*12'h4);
125     end
126     decoded_reg_strb.SHA512_VAULT_RD_CTRL =
cpuif_req_masked & (cpuif_addr == 12'h600);
127     decoded_reg_strb.SHA512_VAULT_RD_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h604);
128     decoded_reg_strb.SHA512_KV_WR_CTRL = cpuif_req_masked
& (cpuif_addr == 12'h608);
129     decoded_reg_strb.SHA512_KV_WR_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h60c);
130     for(int i0=0; i0<8; i0++) begin
131         decoded_reg_strb.SHA512_GEN_PCR_HASH_NONCE[i0] =
cpuif_req_masked & (cpuif_addr == 12'h610 + i0*12'h4);
132     end
133     decoded_reg_strb.SHA512_GEN_PCR_HASH_CTRL =
cpuif_req_masked & (cpuif_addr == 12'h630);
134     decoded_reg_strb.SHA512_GEN_PCR_HASH_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h634);
135     for(int i0=0; i0<16; i0++) begin
136         decoded_reg_strb.SHA512_GEN_PCR_HASH_DIGEST[i0] =
cpuif_req_masked & (cpuif_addr == 12'h638 + i0*12'h4);
137     end
138     decoded_reg_strb.intr_block_rf.global_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h800);
139     decoded_reg_strb.intr_block_rf.error_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h804);
140     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h808);
141     decoded_reg_strb.intr_block_rf.error_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h80c);
142     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h810);
143     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h814);
144     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h818);
145     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h81c);
146     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h820);
147     decoded_reg_strb.intr_block_rf.error0_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h900);
148     decoded_reg_strb.intr_block_rf.error1_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h904);
149     decoded_reg_strb.intr_block_rf.error2_intr_count_r =

```

```

cpuif_req_masked & (cpuif_addr == 12'h908);
150     decoded_reg_strb.intr_block_rf.error3_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h90c);
151     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h980);
152     de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha00);
153     de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha04);
154     de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha08);
155     de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha0c);
156     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha10);
157     end
158
159     // Pass down signals to next stage
160     assign decoded_req = cpuif_req_masked;
161     assign decoded_req_is_wr = cpuif_req_is_wr;
162     assign decoded_wr_data = cpuif_wr_data;
163     assign decoded_wr_bitlen = cpuif_wr_bitlen;
164
165     //-----
166     // Field logic
167     //-----
168
169     typedef struct packed{
170         struct packed{
171             logic next;
172             logic load_next;
173         } INIT;
174         struct packed{
175             logic next;
176             logic load_next;
177         } NEXT;
178         struct packed{
179             logic [1:0] next;
180             logic load_next;
181         } MODE;
182         struct packed{
183             logic next;
184             logic load_next;
185         } ZEROIZE;
186         struct packed{
187             logic next;
188             logic load_next;
189         } LAST;
190         struct packed{
191             logic next;
192             logic load_next;
193         } RESTORE;
194     } SHA512_CTRL;
195     struct packed{
196         struct packed{
197             logic [31:0] next;
198             logic load_next;
199         } BLOCK;
200     } [32-1:0]SHA512_BLOCK;
201     struct packed{
202         struct packed{
203             logic [31:0] next;
204             logic load_next;
205         } DIGEST;
206     } [16-1:0]SHA512_DIGEST;
207     struct packed{
208         struct packed{
209             logic next;
210             logic load_next;
211         } read_en;
212         struct packed{
213             logic [4:0] next;
214             logic load_next;
215         } read_entry;
216         struct packed{
217             logic next;
218             logic load_next;
219         } pcr_hash_extend;
220         struct packed{
221             logic [24:0] next;
222             logic load_next;
223         } rsvd;
224     } SHA512_VAULT_RD_CTRL;
225     struct packed{
226         struct packed{
227             logic next;
228             logic load_next;
229         } VALID;
230     } SHA512_VAULT_RD_STATUS;
231     struct packed{
232         struct packed{
233             logic next;
234             logic load_next;

```

```

235     } write_en;
236     struct packed{
237         logic [4:0] next;
238         logic load_next;
239     } write_entry;
240     struct packed{
241         logic next;
242         logic load_next;
243     } hmac_key_dest_valid;
244     struct packed{
245         logic next;
246         logic load_next;
247     } hmac_block_dest_valid;
248     struct packed{
249         logic next;
250         logic load_next;
251     } mldsa_seed_dest_valid;
252     struct packed{
253         logic next;
254         logic load_next;
255     } ecc_pkey_dest_valid;
256     struct packed{
257         logic next;
258         logic load_next;
259     } ecc_seed_dest_valid;
260     struct packed{
261         logic next;
262         logic load_next;
263     } aes_key_dest_valid;
264     struct packed{
265         logic next;
266         logic load_next;
267     } mlkem_seed_dest_valid;
268     struct packed{
269         logic next;
270         logic load_next;
271     } mlkem_msg_dest_valid;
272     struct packed{
273         logic next;
274         logic load_next;
275     } dma_data_dest_valid;
276     struct packed{
277         logic [16:0] next;
278         logic load_next;
279     } rsvd;
280 } SHA512_KV_WR_CTRL;
281 struct packed{
282     struct packed{
283         logic next;
284         logic load_next;
285     } VALID;
286 } SHA512_KV_WR_STATUS;
287 struct packed{
288     struct packed{
289         logic [31:0] next;
290         logic load_next;
291     } NONCE;
292 } [8-1:0]SHA512_GEN_PCR_HASH_NONCE;
293 struct packed{
294     struct packed{
295         logic next;
296         logic load_next;
297     } START;
298 } SHA512_GEN_PCR_HASH_CTRL;
299 struct packed{
300     struct packed{
301         logic next;
302         logic load_next;
303     } VALID;
304 } SHA512_GEN_PCR_HASH_STATUS;
305 struct packed{
306     struct packed{
307         logic [31:0] next;
308         logic load_next;
309     } DIGEST;
310 } [16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
311 struct packed{
312     struct packed{
313         struct packed{
314             logic next;
315             logic load_next;
316         } error_en;
317         struct packed{
318             logic next;
319             logic load_next;
320         } notif_en;
321     } global_intr_en_r;
322     struct packed{
323         struct packed{
324             logic next;
325             logic load_next;
326         } error0_en;
327         struct packed{
328             logic next;
329             logic load_next;
330         } error1_en;
331         struct packed{
332             logic next;
333             logic load_next;
334         } error2_en;
335         struct packed{

```

```

336             logic next;
337             logic load_next;
338         } error3_en;
339     } error_intr_en_r;
340     struct packed{
341         struct packed{
342             logic next;
343             logic load_next;
344         } notif_cmd_done_en;
345     } notif_intr_en_r;
346     struct packed{
347         struct packed{
348             logic next;
349             logic load_next;
350         } agg_sts;
351     } error_global_intr_r;
352     struct packed{
353         struct packed{
354             logic next;
355             logic load_next;
356         } agg_sts;
357     } notif_global_intr_r;
358     struct packed{
359         struct packed{
360             logic next;
361             logic load_next;
362         } error0_sts;
363         struct packed{
364             logic next;
365             logic load_next;
366         } error1_sts;
367         struct packed{
368             logic next;
369             logic load_next;
370         } error2_sts;
371         struct packed{
372             logic next;
373             logic load_next;
374         } error3_sts;
375     } error_internal_intr_r;
376     struct packed{
377         struct packed{
378             logic next;
379             logic load_next;
380         } notif_cmd_done_sts;
381     } notif_internal_intr_r;
382     struct packed{
383         struct packed{
384             logic next;
385             logic load_next;
386         } error0_trig;
387         struct packed{
388             logic next;
389             logic load_next;
390         } error1_trig;
391         struct packed{
392             logic next;
393             logic load_next;
394         } error2_trig;
395         struct packed{
396             logic next;
397             logic load_next;
398         } error3_trig;
399     } error_intr_trig_r;
400     struct packed{
401         struct packed{
402             logic next;
403             logic load_next;
404         } notif_cmd_done_trig;
405     } notif_intr_trig_r;
406     struct packed{
407         struct packed{
408             logic [31:0] next;
409             logic load_next;
410             logic incrthreshold;
411             logic incrsaturate;
412         } cnt;
413     } error0_intr_count_r;
414     struct packed{
415         struct packed{
416             logic [31:0] next;
417             logic load_next;
418             logic incrthreshold;
419             logic incrsaturate;
420         } cnt;
421     } error1_intr_count_r;
422     struct packed{
423         struct packed{
424             logic [31:0] next;
425             logic load_next;
426             logic incrthreshold;
427             logic incrsaturate;
428         } cnt;
429     } error2_intr_count_r;
430     struct packed{
431         struct packed{
432             logic [31:0] next;
433             logic load_next;
434             logic incrthreshold;
435             logic incrsaturate;
436         } cnt;

```

```

437     } error3_intr_count_r;
438     struct packed{
439         struct packed{
440             logic [31:0] next;
441             logic load_next;
442             logic incrtreshold;
443             logic incrsaturate;
444         } cnt;
445     } notif_cmd_done_intr_count_r;
446     struct packed{
447         struct packed{
448             logic next;
449             logic load_next;
450             logic decrtreshold;
451             logic underflow;
452         } pulse;
453     } error0_intr_count_incr_r;
454     struct packed{
455         struct packed{
456             logic next;
457             logic load_next;
458             logic decrtreshold;
459             logic underflow;
460         } pulse;
461     } error1_intr_count_incr_r;
462     struct packed{
463         struct packed{
464             logic next;
465             logic load_next;
466             logic decrtreshold;
467             logic underflow;
468         } pulse;
469     } error2_intr_count_incr_r;
470     struct packed{
471         struct packed{
472             logic next;
473             logic load_next;
474             logic decrtreshold;
475             logic underflow;
476         } pulse;
477     } error3_intr_count_incr_r;
478     struct packed{
479         struct packed{
480             logic next;
481             logic load_next;
482             logic decrtreshold;
483             logic underflow;
484         } pulse;
485     } notif_cmd_done_intr_count_incr_r;
486     } intr_block_rf;
487 } field_combo_t;
488 field_combo_t field_combo;
489
490 typedef struct packed{
491     struct packed{
492         struct packed{
493             logic value;
494         } INIT;
495         struct packed{
496             logic value;
497         } NEXT;
498         struct packed{
499             logic [1:0] value;
500         } MODE;
501         struct packed{
502             logic value;
503         } ZEROIZE;
504         struct packed{
505             logic value;
506         } LAST;
507         struct packed{
508             logic value;
509         } RESTORE;
510     } SHA512_CTRL;
511     struct packed{
512         struct packed{
513             logic [31:0] value;
514         } BLOCK;
515     } [32-1:0]SHA512_BLOCK;
516     struct packed{
517         struct packed{
518             logic [31:0] value;
519         } DIGEST;
520     } [16-1:0]SHA512_DIGEST;
521     struct packed{
522         struct packed{
523             logic value;
524         } read_en;
525         struct packed{
526             logic [4:0] value;
527         } read_entry;
528         struct packed{
529             logic value;
530         } pcr_hash_extend;
531         struct packed{
532             logic [24:0] value;
533         } rsvd;
534     } SHA512_VAULT_RD_CTRL;
535     struct packed{
536         struct packed{
537             logic value;
538         } VALID;
539     } SHA512_VAULT_RD_STATUS;
540     struct packed{
541         struct packed{
542             logic value;
543         } write_en;
544         struct packed{
545             logic [4:0] value;
546         } write_entry;
547         struct packed{
548             logic value;
549         } hmac_key_dest_valid;
550         struct packed{
551             logic value;
552         } hmac_block_dest_valid;
553         struct packed{
554             logic value;
555         } mlds_seed_dest_valid;
556         struct packed{
557             logic value;
558         } ecc_pkey_dest_valid;
559         struct packed{
560             logic value;
561         } ecc_seed_dest_valid;
562         struct packed{
563             logic value;
564         } aes_key_dest_valid;
565         struct packed{
566             logic value;
567         } mlkem_seed_dest_valid;
568         struct packed{
569             logic value;
570         } mlkem_msg_dest_valid;
571         struct packed{
572             logic value;
573         } dma_data_dest_valid;
574         struct packed{
575             logic [16:0] value;
576         } rsvd;
577     } SHA512_KV_WR_CTRL;
578     struct packed{
579         struct packed{
580             logic value;
581         } VALID;
582     } SHA512_KV_WR_STATUS;
583     struct packed{
584         struct packed{
585             logic [31:0] value;
586         } NONCE;
587     } [8-1:0]SHA512_GEN_PCR_HASH_NONCE;
588     struct packed{
589         struct packed{
590             logic value;
591         } START;
592     } SHA512_GEN_PCR_HASH_CTRL;
593     struct packed{
594         struct packed{
595             logic value;
596         } VALID;
597     } SHA512_GEN_PCR_HASH_STATUS;
598     struct packed{
599         struct packed{
600             logic [31:0] value;
601         } DIGEST;
602     } [16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
603     struct packed{
604         struct packed{
605             struct packed{
606                 logic value;
607             } error_en;
608             struct packed{
609                 logic value;
610             } notif_en;
611         } global_intr_en_r;
612         struct packed{
613             struct packed{
614                 logic value;
615             } error0_en;
616             struct packed{
617                 logic value;
618             } error1_en;
619             struct packed{
620                 logic value;
621             } error2_en;
622             struct packed{
623                 logic value;
624             } error3_en;
625         } error_intr_en_r;
626         struct packed{
627             struct packed{
628                 logic value;
629             } notif_cmd_done_en;
630         } notif_intr_en_r;
631         struct packed{
632             struct packed{
633                 logic value;
634             } agg_sts;
635         } error_global_intr_r;
636         struct packed{
637             struct packed{
638                 logic value;

```



```

639     } agg_sts;
640 } notif_global_intr_r;
641 struct packed{
642     struct packed{
643         logic value;
644     } error0_sts;
645     struct packed{
646         logic value;
647     } error1_sts;
648     struct packed{
649         logic value;
650     } error2_sts;
651     struct packed{
652         logic value;
653     } error3_sts;
654 } error_internal_intr_r;
655 struct packed{
656     struct packed{
657         logic value;
658     } notif_cmd_done_sts;
659 } notif_internal_intr_r;
660 struct packed{
661     struct packed{
662         logic value;
663     } error0_trig;
664     struct packed{
665         logic value;
666     } error1_trig;
667     struct packed{
668         logic value;
669     } error2_trig;
670     struct packed{
671         logic value;
672     } error3_trig;
673 } error_intr_trig_r;
674 struct packed{
675     struct packed{
676         logic value;
677     } notif_cmd_done_trig;
678 } notif_intr_trig_r;
679 struct packed{
680     struct packed{
681         logic [31:0] value;
682     } cnt;
683 } error0_intr_count_r;
684 struct packed{
685     struct packed{
686         logic [31:0] value;
687     } cnt;
688 } error1_intr_count_r;
689 struct packed{
690     struct packed{
691         logic [31:0] value;
692     } cnt;
693 } error2_intr_count_r;
694 struct packed{
695     struct packed{
696         logic [31:0] value;
697     } cnt;
698 } error3_intr_count_r;
699 struct packed{
700     struct packed{
701         logic [31:0] value;
702     } cnt;
703 } notif_cmd_done_intr_count_r;
704 struct packed{
705     struct packed{
706         logic value;
707     } pulse;
708 } error0_intr_count_incr_r;
709 struct packed{
710     struct packed{
711         logic value;
712     } pulse;
713 } error1_intr_count_incr_r;
714 struct packed{
715     struct packed{
716         logic value;
717     } pulse;
718 } error2_intr_count_incr_r;
719 struct packed{
720     struct packed{
721         logic value;
722     } pulse;
723 } error3_intr_count_incr_r;
724 struct packed{
725     struct packed{
726         logic value;
727     } pulse;
728 } notif_cmd_done_intr_count_incr_r;
729 } intr_block_rf;
730 } field_storage_t;
731 field_storage_t field_storage;
732
733 // Field: sha512_reg.SHA512_CTRL.INIT
734 always_comb begin
735     automatic logic [0:0] next_c;
736     automatic logic load_next_c;
737     next_c = field_storage.SHA512_CTRL.INIT.value;
738     load_next_c = '0;
739     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr

```

```

&& hwif_in.sha512_ready) begin // SW write
740     next_c = (field_storage.SHA512_CTRL.INIT.value &
~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
741     load_next_c = '1;
742     end else begin // singlepulse clears back to 0
743         next_c = '0;
744         load_next_c = '1;
745     end
746     field_combo.SHA512_CTRL.INIT.next = next_c;
747     field_combo.SHA512_CTRL.INIT.load_next = load_next_c;
748 end
749 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
750     if(~hwif_in.reset_b) begin
751         field_storage.SHA512_CTRL.INIT.value <= 1'h0;
752     end else if(field_combo.SHA512_CTRL.INIT.load_next)
begin
753         field_storage.SHA512_CTRL.INIT.value <=
field_combo.SHA512_CTRL.INIT.next;
754     end
755 end
756 assign hwif_out.SHA512_CTRL.INIT.value =
field_storage.SHA512_CTRL.INIT.value;
757 // Field: sha512_reg.SHA512_CTRL.NEXT
758 always_comb begin
759     automatic logic [0:0] next_c;
760     automatic logic load_next_c;
761     next_c = field_storage.SHA512_CTRL.NEXT.value;
762     load_next_c = '0;
763     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
764         next_c = (field_storage.SHA512_CTRL.NEXT.value &
~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
765         load_next_c = '1;
766     end else begin // singlepulse clears back to 0
767         next_c = '0;
768         load_next_c = '1;
769     end
770     field_combo.SHA512_CTRL.NEXT.next = next_c;
771     field_combo.SHA512_CTRL.NEXT.load_next = load_next_c;
772 end
773 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
774     if(~hwif_in.reset_b) begin
775         field_storage.SHA512_CTRL.NEXT.value <= 1'h0;
776     end else if(field_combo.SHA512_CTRL.NEXT.load_next)
begin
777         field_storage.SHA512_CTRL.NEXT.value <=
field_combo.SHA512_CTRL.NEXT.next;
778     end
779 end
780 assign hwif_out.SHA512_CTRL.NEXT.value =
field_storage.SHA512_CTRL.NEXT.value;
781 // Field: sha512_reg.SHA512_CTRL.MODE
782 always_comb begin
783     automatic logic [1:0] next_c;
784     automatic logic load_next_c;
785     next_c = field_storage.SHA512_CTRL.MODE.value;
786     load_next_c = '0;
787     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
788         next_c = (field_storage.SHA512_CTRL.MODE.value &
~decoded_wr_biten[3:2]) | (decoded_wr_data[3:2] & de-
coded_wr_biten[3:2]);
789         load_next_c = '1;
790     end
791     field_combo.SHA512_CTRL.MODE.next = next_c;
792     field_combo.SHA512_CTRL.MODE.load_next = load_next_c;
793 end
794 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
795     if(~hwif_in.reset_b) begin
796         field_storage.SHA512_CTRL.MODE.value <= 2'h2;
797     end else if(field_combo.SHA512_CTRL.MODE.load_next)
begin
798         field_storage.SHA512_CTRL.MODE.value <=
field_combo.SHA512_CTRL.MODE.next;
799     end
800 end
801 assign hwif_out.SHA512_CTRL.MODE.value =
field_storage.SHA512_CTRL.MODE.value;
802 // Field: sha512_reg.SHA512_CTRL.ZEROIZE
803 always_comb begin
804     automatic logic [0:0] next_c;
805     automatic logic load_next_c;
806     next_c = field_storage.SHA512_CTRL.ZEROIZE.value;
807     load_next_c = '0;
808     if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr)
begin // SW write
809         next_c = (field_storage.SHA512_CTRL.ZEROIZE.value
& ~decoded_wr_biten[4:4]) | (decoded_wr_data[4:4] & de-
coded_wr_biten[4:4]);
810         load_next_c = '1;
811     end else begin // singlepulse clears back to 0
812         next_c = '0;
813         load_next_c = '1;
814     end
815     field_combo.SHA512_CTRL.ZEROIZE.next = next_c;
816     field_combo.SHA512_CTRL.ZEROIZE.load_next =
load_next_c;
817 end
818 always_ff @(posedge clk or negedge hwif_in.reset_b) begin

```

```

819         if(~hwif_in.reset_b) begin
820             field_storage.SHA512_CTRL.ZEROIZE.value <= 1'h0;
821         end else if(field_combo.SHA512_CTRL.ZEROIZE.load_next)
begin
822             field_storage.SHA512_CTRL.ZEROIZE.value <=
field_combo.SHA512_CTRL.ZEROIZE.next;
823         end
824     end
825     assign hwif_out.SHA512_CTRL.ZEROIZE.value =
field_storage.SHA512_CTRL.ZEROIZE.value;
826     // Field: sha512_reg.SHA512_CTRL.LAST
827     always_comb begin
828         automatic logic [0:0] next_c;
829         automatic logic load_next_c;
830         next_c = field_storage.SHA512_CTRL.LAST.value;
831         load_next_c = '0;
832         if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
833             next_c = (field_storage.SHA512_CTRL.LAST.value &
~decoded_wr_biten[5:5]) | (decoded_wr_data[5:5] & de-
coded_wr_biten[5:5]);
834             load_next_c = '1;
835         end else if(hwif_in.SHA512_CTRL.LAST.hwclr) begin //
HW Clear
836             next_c = '0;
837             load_next_c = '1;
838         end
839         field_combo.SHA512_CTRL.LAST.next = next_c;
840         field_combo.SHA512_CTRL.LAST.load_next = load_next_c;
841     end
842     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
843         if(~hwif_in.reset_b) begin
844             field_storage.SHA512_CTRL.LAST.value <= 1'h0;
845         end else if(field_combo.SHA512_CTRL.LAST.load_next)
begin
846             field_storage.SHA512_CTRL.LAST.value <=
field_combo.SHA512_CTRL.LAST.next;
847         end
848     end
849     assign hwif_out.SHA512_CTRL.LAST.value =
field_storage.SHA512_CTRL.LAST.value;
850     // Field: sha512_reg.SHA512_CTRL.RESTORE
851     always_comb begin
852         automatic logic [0:0] next_c;
853         automatic logic load_next_c;
854         next_c = field_storage.SHA512_CTRL.RESTORE.value;
855         load_next_c = '0;
856         if(decoded_reg_strb.SHA512_CTRL && decoded_req_is_wr
&& hwif_in.sha512_ready) begin // SW write
857             next_c = (field_storage.SHA512_CTRL.RESTORE.value
& ~decoded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
858             load_next_c = '1;
859         end else begin // singlepulse clears back to 0
860             next_c = '0;
861             load_next_c = '1;
862         end
863         field_combo.SHA512_CTRL.RESTORE.next = next_c;
864         field_combo.SHA512_CTRL.RESTORE.load_next =
load_next_c;
865     end
866     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
867         if(~hwif_in.reset_b) begin
868             field_storage.SHA512_CTRL.RESTORE.value <= 1'h0;
869         end else if(field_combo.SHA512_CTRL.RESTORE.load_next)
begin
870             field_storage.SHA512_CTRL.RESTORE.value <=
field_combo.SHA512_CTRL.RESTORE.next;
871         end
872     end
873     assign hwif_out.SHA512_CTRL.RESTORE.value =
field_storage.SHA512_CTRL.RESTORE.value;
874     for(genvar i0=0; i0<32; i0++) begin
875         // Field: sha512_reg.SHA512_BLOCK[i].BLOCK
876         always_comb begin
877             automatic logic [31:0] next_c;
878             automatic logic load_next_c;
879             next_c =
field_storage.SHA512_BLOCK[i0].BLOCK.value;
880             load_next_c = '0;
881             if(decoded_reg_strb.SHA512_BLOCK[i0] && de-
coded_req_is_wr && !(hwif_in.SHA512_BLOCK[i0].BLOCK.swel)) begin //
SW write
882                 next_c =
(field_storage.SHA512_BLOCK[i0].BLOCK.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
883                 load_next_c = '1;
884             end else if(hwif_in.SHA512_BLOCK[i0].BLOCK.we) be-
gin // HW Write - we
885                 next_c = hwif_in.SHA512_BLOCK[i0].BLOCK.next;
886                 load_next_c = '1;
887             end else if(hwif_in.SHA512_BLOCK[i0].BLOCK.hwclr)
begin // HW Clear
888                 next_c = '0;
889                 load_next_c = '1;
890             end
891             field_combo.SHA512_BLOCK[i0].BLOCK.next = next_c;
892             field_combo.SHA512_BLOCK[i0].BLOCK.load_next =

```

```

load_next_c;
893     end
894     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
895         if(~hwif_in.reset_b) begin
896             field_storage.SHA512_BLOCK[i0].BLOCK.value <=
32'h0;
897         end else
if(field_combo.SHA512_BLOCK[i0].BLOCK.load_next) begin
898             field_storage.SHA512_BLOCK[i0].BLOCK.value <=
field_combo.SHA512_BLOCK[i0].BLOCK.next;
899         end
900     end
901     assign hwif_out.SHA512_BLOCK[i0].BLOCK.value =
field_storage.SHA512_BLOCK[i0].BLOCK.value;
902     end
903     for(genvar i0=0; i0<16; i0++) begin
904         // Field: sha512_reg.SHA512_DIGEST[i].DIGEST
905         always_comb begin
906             automatic logic [31:0] next_c;
907             automatic logic load_next_c;
908             next_c =
field_storage.SHA512_DIGEST[i0].DIGEST.value;
909             load_next_c = '0;
910             if(decoded_reg_strb.SHA512_DIGEST[i0] && de-
coded_req_is_wr) begin // SW write
911                 next_c =
(field_storage.SHA512_DIGEST[i0].DIGEST.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
912                 load_next_c = '1;
913             end else if(hwif_in.SHA512_DIGEST[i0].DIGEST.we)
begin // HW Write - we
914                 next_c =
hwif_in.SHA512_DIGEST[i0].DIGEST.next;
915                 load_next_c = '1;
916             end else
if(hwif_in.SHA512_DIGEST[i0].DIGEST.hwclr) begin // HW Clear
917                 next_c = '0;
918                 load_next_c = '1;
919             end
920             field_combo.SHA512_DIGEST[i0].DIGEST.next =
next_c;
921             field_combo.SHA512_DIGEST[i0].DIGEST.load_next =
load_next_c;
922         end
923         always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
924             if(~hwif_in.reset_b) begin
925                 field_storage.SHA512_DIGEST[i0].DIGEST.value
<= 32'h0;
926             end else
if(field_combo.SHA512_DIGEST[i0].DIGEST.load_next) begin
927                 field_storage.SHA512_DIGEST[i0].DIGEST.value
<= field_combo.SHA512_DIGEST[i0].DIGEST.next;
928             end
929         end
930         assign hwif_out.SHA512_DIGEST[i0].DIGEST.value =
field_storage.SHA512_DIGEST[i0].DIGEST.value;
931     end
932     // Field: sha512_reg.SHA512_VAULT_RD_CTRL.read_en
933     always_comb begin
934         automatic logic [0:0] next_c;
935         automatic logic load_next_c;
936         next_c =
field_storage.SHA512_VAULT_RD_CTRL.read_en.value;
937         load_next_c = '0;
938         if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.read_en.swwe) be-
gin // SW write
939             next_c =
(field_storage.SHA512_VAULT_RD_CTRL.read_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
940             load_next_c = '1;
941         end else
if(hwif_in.SHA512_VAULT_RD_CTRL.read_en.hwclr) begin // HW Clear
942             next_c = '0;
943             load_next_c = '1;
944         end
945         field_combo.SHA512_VAULT_RD_CTRL.read_en.next =
next_c;
946         field_combo.SHA512_VAULT_RD_CTRL.read_en.load_next =
load_next_c;
947     end
948     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
949         if(~hwif_in.reset_b) begin
950             field_storage.SHA512_VAULT_RD_CTRL.read_en.value
<= 1'h0;
951         end else
if(field_combo.SHA512_VAULT_RD_CTRL.read_en.load_next) begin
952             field_storage.SHA512_VAULT_RD_CTRL.read_en.value
<= field_combo.SHA512_VAULT_RD_CTRL.read_en.next;
953         end
954     end
955     assign hwif_out.SHA512_VAULT_RD_CTRL.read_en.value =
field_storage.SHA512_VAULT_RD_CTRL.read_en.value;
956     // Field: sha512_reg.SHA512_VAULT_RD_CTRL.read_entry
957     always_comb begin
958         automatic logic [4:0] next_c;
959         automatic logic load_next_c;

```



```

960     next_c =
field_storage.SHA512_VAULT_RD_CTRL.read_entry.value;
961     load_next_c = '0';
962     if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.read_entry.swwe) be-
gin // SW write
963         next_c =
(field_storage.SHA512_VAULT_RD_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
964         load_next_c = '1';
965     end
966     field_combo.SHA512_VAULT_RD_CTRL.read_entry.next =
next_c;
967     field_combo.SHA512_VAULT_RD_CTRL.read_entry.load_next
= load_next_c;
968 end
969 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
970     if(~hwif_in.reset_b) begin
971         field_storage.SHA512_VAULT_RD_CTRL.read_entry.value <= 5'h0;
972     end else
if(field_combo.SHA512_VAULT_RD_CTRL.read_entry.load_next) begin
973         field_storage.SHA512_VAULT_RD_CTRL.read_entry.value <=
field_combo.SHA512_VAULT_RD_CTRL.read_entry.next;
974     end
975 end
976 assign hwif_out.SHA512_VAULT_RD_CTRL.read_entry.value =
field_storage.SHA512_VAULT_RD_CTRL.read_entry.value;
977 // Field: sha512_reg.SHA512_VAULT_RD_CTRL.pcr_hash_extend
978 always_comb begin
979     automatic logic [0:0] next_c;
980     automatic logic load_next_c;
981     next_c =
field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value;
982     load_next_c = '0';
983     if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.pcr_hash_extend.swwe) begin // SW write
984         next_c =
(field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
985         load_next_c = '1';
986     end
987     field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.next
= next_c;
988     field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.load_next =
load_next_c;
989 end
990 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
991     if(~hwif_in.reset_b) begin
992         field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value <= 1'h0;
993     end else
if(field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.load_next) begin
994         field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value <=
field_combo.SHA512_VAULT_RD_CTRL.pcr_hash_extend.next;
995     end
996 end
997 assign hwif_out.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value =
field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value;
998 // Field: sha512_reg.SHA512_VAULT_RD_CTRL.rsvd
999 always_comb begin
1000     automatic logic [24:0] next_c;
1001     automatic logic load_next_c;
1002     next_c =
field_storage.SHA512_VAULT_RD_CTRL.rsvd.value;
1003     load_next_c = '0';
1004     if(decoded_reg_strb.SHA512_VAULT_RD_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_VAULT_RD_CTRL.rsvd.swwe) begin //
SW write
1005         next_c =
(field_storage.SHA512_VAULT_RD_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
1006         load_next_c = '1';
1007     end
1008     field_combo.SHA512_VAULT_RD_CTRL.rsvd.next = next_c;
1009     field_combo.SHA512_VAULT_RD_CTRL.rsvd.load_next =
load_next_c;
1010 end
1011 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1012     if(~hwif_in.reset_b) begin
1013         field_storage.SHA512_VAULT_RD_CTRL.rsvd.value <=
25'h0;
1014     end else
if(field_combo.SHA512_VAULT_RD_CTRL.rsvd.load_next) begin
1015         field_storage.SHA512_VAULT_RD_CTRL.rsvd.value <=
field_combo.SHA512_VAULT_RD_CTRL.rsvd.next;
1016     end
1017 end
1018 assign hwif_out.SHA512_VAULT_RD_CTRL.rsvd.value =
field_storage.SHA512_VAULT_RD_CTRL.rsvd.value;
1019 // Field: sha512_reg.SHA512_VAULT_RD_STATUS.VALID
1020 always_comb begin
1021     automatic logic [0:0] next_c;

```

```

1022     automatic logic load_next_c;
1023     next_c =
field_storage.SHA512_VAULT_RD_STATUS.VALID.value;
1024     load_next_c = '0';
1025     if(hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwset) be-
gin // HW Set
1026         next_c = '1';
1027         load_next_c = '1';
1028     end else
if(hwif_in.SHA512_VAULT_RD_STATUS.VALID.hwclr) begin // HW Clear
1029         next_c = '0';
1030         load_next_c = '1';
1031     end
1032     field_combo.SHA512_VAULT_RD_STATUS.VALID.next =
next_c;
1033     field_combo.SHA512_VAULT_RD_STATUS.VALID.load_next =
load_next_c;
1034 end
1035 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1036     if(~hwif_in.reset_b) begin
1037         field_storage.SHA512_VAULT_RD_STATUS.VALID.value
<= 1'h0;
1038     end else
if(field_combo.SHA512_VAULT_RD_STATUS.VALID.load_next) begin
1039         field_storage.SHA512_VAULT_RD_STATUS.VALID.value
<= field_combo.SHA512_VAULT_RD_STATUS.VALID.next;
1040     end
1041 end
1042 // Field: sha512_reg.SHA512_KV_WR_CTRL.write_en
1043 always_comb begin
1044     automatic logic [0:0] next_c;
1045     automatic logic load_next_c;
1046     next_c =
field_storage.SHA512_KV_WR_CTRL.write_en.value;
1047     load_next_c = '0';
1048     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_KV_WR_CTRL.write_en.swwe) begin //
SW write
1049         next_c =
(field_storage.SHA512_KV_WR_CTRL.write_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1050         load_next_c = '1';
1051     end else if(hwif_in.SHA512_KV_WR_CTRL.write_en.hwclr)
begin // HW Clear
1052         next_c = '0';
1053         load_next_c = '1';
1054     end
1055     field_combo.SHA512_KV_WR_CTRL.write_en.next = next_c;
1056     field_combo.SHA512_KV_WR_CTRL.write_en.load_next =
load_next_c;
1057 end
1058 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1059     if(~hwif_in.reset_b) begin
1060         field_storage.SHA512_KV_WR_CTRL.write_en.value <=
1'h0;
1061     end else
if(field_combo.SHA512_KV_WR_CTRL.write_en.load_next) begin
1062         field_storage.SHA512_KV_WR_CTRL.write_en.value <=
field_combo.SHA512_KV_WR_CTRL.write_en.next;
1063     end
1064 end
1065 assign hwif_out.SHA512_KV_WR_CTRL.write_en.value =
field_storage.SHA512_KV_WR_CTRL.write_en.value;
1066 // Field: sha512_reg.SHA512_KV_WR_CTRL.write_entry
1067 always_comb begin
1068     automatic logic [4:0] next_c;
1069     automatic logic load_next_c;
1070     next_c =
field_storage.SHA512_KV_WR_CTRL.write_entry.value;
1071     load_next_c = '0';
1072     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_KV_WR_CTRL.write_entry.swwe) be-
gin // SW write
1073         next_c =
(field_storage.SHA512_KV_WR_CTRL.write_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1074         load_next_c = '1';
1075     end
1076     field_combo.SHA512_KV_WR_CTRL.write_entry.next =
next_c;
1077     field_combo.SHA512_KV_WR_CTRL.write_entry.load_next =
load_next_c;
1078 end
1079 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1080     if(~hwif_in.reset_b) begin
1081         field_storage.SHA512_KV_WR_CTRL.write_entry.value
<= 5'h0;
1082     end else
if(field_combo.SHA512_KV_WR_CTRL.write_entry.load_next) begin
1083         field_storage.SHA512_KV_WR_CTRL.write_entry.value
<= field_combo.SHA512_KV_WR_CTRL.write_entry.next;
1084     end
1085 end
1086 assign hwif_out.SHA512_KV_WR_CTRL.write_entry.value =
field_storage.SHA512_KV_WR_CTRL.write_entry.value;
1087 // Field: sha512_reg.SHA512_KV_WR_CTRL.hmac_key_dest_valid
1088 always_comb begin
1089     automatic logic [0:0] next_c;
1090     automatic logic load_next_c;

```

```

1091     next_c =
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value;
1092     load_next_c = '0';
1093     if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.hmac_key_dest_valid.swwe) begin // SW
write
1094         next_c =
(field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1095         load_next_c = '1';
1096     end

1097     field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.next
= next_c;
1098
field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.load_next =
load_next_c;
1099     end
1100     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1101         if(~hwif_in.reset_b) begin
1102
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value <= 1'h0;
1103         end else
if(field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.load_next) be-
gin
1104
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.hmac_key_dest_valid.next;
1105         end
1106     end
1107     assign
hwif_out.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value;
1108     // Field:
sha512_reg.SHA512_KV_WR_CTRL.hmac_block_dest_valid
1109     always_comb begin
1110         automatic logic [0:0] next_c;
1111         automatic logic load_next_c;
1112         next_c =
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value;
1113         load_next_c = '0';
1114         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.hmac_block_dest_valid.swwe) begin // SW
write
1115             next_c =
(field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value & ~de-
coded_wr_biten[7:7]) | (decoded_wr_data[7:7] & de-
coded_wr_biten[7:7]);
1116             load_next_c = '1';
1117         end
1118
field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.next = next_c;
1119
field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.load_next =
load_next_c;
1120     end
1121     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1122         if(~hwif_in.reset_b) begin
1123
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value <= 1'h0;
1124         end else
if(field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.load_next)
begin
1125
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.hmac_block_dest_valid.next;
1126         end
1127     end
1128     assign
hwif_out.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value;
1129     // Field:
sha512_reg.SHA512_KV_WR_CTRL.mlds_seed_dest_valid
1130     always_comb begin
1131         automatic logic [0:0] next_c;
1132         automatic logic load_next_c;
1133         next_c =
field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1134         load_next_c = '0';
1135         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.swwe) begin // SW
write
1136             next_c =
(field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value & ~de-
coded_wr_biten[8:8]) | (decoded_wr_data[8:8] & de-
coded_wr_biten[8:8]);
1137             load_next_c = '1';
1138         end
1139
field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.next = next_c;
1140
field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.load_next =
load_next_c;
1141     end
1142     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1143         if(~hwif_in.reset_b) begin
1144

```

```

field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value <= 1'h0;
1145     end else
if(field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.load_next)
begin
1146
field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.next;
1147     end
1148     end
1149     assign
hwif_out.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1150     // Field: sha512_reg.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid
1151     always_comb begin
1152         automatic logic [0:0] next_c;
1153         automatic logic load_next_c;
1154         next_c =
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value;
1155         load_next_c = '0';
1156         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe) begin // SW
write
1157             next_c =
(field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
1158             load_next_c = '1';
1159         end

1160     field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.next
= next_c;
1161
field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next =
load_next_c;
1162     end
1163     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1164         if(~hwif_in.reset_b) begin
1165
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value <= 1'h0;
1166         end else
if(field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next) be-
gin
1167
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.next;
1168         end
1169     end
1170     assign
hwif_out.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value;
1171     // Field: sha512_reg.SHA512_KV_WR_CTRL.ecc_seed_dest_valid
1172     always_comb begin
1173         automatic logic [0:0] next_c;
1174         automatic logic load_next_c;
1175         next_c =
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1176         load_next_c = '0';
1177         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.swwe) begin // SW
write
1178             next_c =
(field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value & ~de-
coded_wr_biten[10:10]) | (decoded_wr_data[10:10] & de-
coded_wr_biten[10:10]);
1179             load_next_c = '1';
1180         end

1181     field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.next
= next_c;
1182
field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.load_next =
load_next_c;
1183     end
1184     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1185         if(~hwif_in.reset_b) begin
1186
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value <= 1'h0;
1187         end else
if(field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.load_next) be-
gin
1188
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.next;
1189         end
1190     end
1191     assign
hwif_out.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1192     // Field: sha512_reg.SHA512_KV_WR_CTRL.aes_key_dest_valid
1193     always_comb begin
1194         automatic logic [0:0] next_c;
1195         automatic logic load_next_c;
1196         next_c =
field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value;
1197         load_next_c = '0';
1198         if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.aes_key_dest_valid.swwe) begin // SW write
1199             next_c =

```

```

(field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value & ~de-
coded_wr_biten[11:11]) | (decoded_wr_data[11:11] & de-
coded_wr_biten[11:11]);
1200     load_next_c = '1;
1201     end
1202     field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.next
= next_c;
1203
field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.load_next =
load_next_c;
1204     end
1205     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1206         if(~hwif_in.reset_b) begin
1207             field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value <= 1'h0;
1208             end else
1209             if(field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.load_next) begin
1210                 field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.aes_key_dest_valid.next;
1211                 end
1212             assign hwif_out.SHA512_KV_WR_CTRL.aes_key_dest_valid.value
= field_storage.SHA512_KV_WR_CTRL.aes_key_dest_valid.value;
1213             // Field:
sha512_reg.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid
1214             always_comb begin
1215                 automatic logic [0:0] next_c;
1216                 automatic logic load_next_c;
1217                 next_c =
field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1218                 load_next_c = '0;
1219                 if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.swwe) begin // SW
write
1220                     next_c =
(field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value & ~de-
coded_wr_biten[12:12]) | (decoded_wr_data[12:12] & de-
coded_wr_biten[12:12]);
1221                     load_next_c = '1;
1222                     end
1223                 end
1224                 field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.next = next_c;
1225                 field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next =
load_next_c;
1226                 end
1227                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1228                     if(~hwif_in.reset_b) begin
1229                         field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value <= 1'h0;
1230                         end else
1231                         if(field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next)
begin
1232                             field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.next;
1233                             end
1234                             assign
hwif_out.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1235                             // Field:
sha512_reg.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid
1236                             always_comb begin
1237                                 automatic logic [0:0] next_c;
1238                                 automatic logic load_next_c;
1239                                 next_c =
field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1240                                 load_next_c = '0;
1241                                 if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.swwe) begin // SW
write
1242                                     next_c =
(field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value & ~de-
coded_wr_biten[13:13]) | (decoded_wr_data[13:13] & de-
coded_wr_biten[13:13]);
1243                                     load_next_c = '1;
1244                                     end
1245                                     field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.next = next_c;
1246                                     field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next =
load_next_c;
1247                                     end
1248                                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1249                                         if(~hwif_in.reset_b) begin
1250                                             field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value <= 1'h0;
1251                                             end else
1252                                             if(field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next) be-
gin
1253                                                 field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.next;
1254                                                 end
1255                                                 assign

```

```

hwif_out.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1255         // Field: sha512_reg.SHA512_KV_WR_CTRL.dma_data_dest_valid
1256         always_comb begin
1257             automatic logic [0:0] next_c;
1258             automatic logic load_next_c;
1259             next_c =
field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value;
1260             load_next_c = '0;
1261             if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.SHA512_KV_WR_CTRL.dma_data_dest_valid.swwe) begin // SW
write
1262                 next_c =
(field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value & ~de-
coded_wr_biten[14:14]) | (decoded_wr_data[14:14] & de-
coded_wr_biten[14:14]);
1263                 load_next_c = '1;
1264                 end
1265                 field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.next
= next_c;
1266                 field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.load_next =
load_next_c;
1267                 end
1268                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1269                     if(~hwif_in.reset_b) begin
1270                         field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value <= 1'h0;
1271                         end else
1272                         if(field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.load_next) be-
gin
1273                             field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value <=
field_combo.SHA512_KV_WR_CTRL.dma_data_dest_valid.next;
1274                             end
1275                             assign
hwif_out.SHA512_KV_WR_CTRL.dma_data_dest_valid.value =
field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value;
1276                             // Field: sha512_reg.SHA512_KV_WR_CTRL.rsvd
1277                             always_comb begin
1278                                 automatic logic [16:0] next_c;
1279                                 automatic logic load_next_c;
1280                                 next_c = field_storage.SHA512_KV_WR_CTRL.rsvd.value;
1281                                 load_next_c = '0;
1282                                 if(decoded_reg_strb.SHA512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.SHA512_KV_WR_CTRL.rsvd.swwe) begin // SW
write
1283                                     next_c =
(field_storage.SHA512_KV_WR_CTRL.rsvd.value & ~de-
coded_wr_biten[31:15]) | (decoded_wr_data[31:15] & de-
coded_wr_biten[31:15]);
1284                                     load_next_c = '1;
1285                                     end
1286                                     field_combo.SHA512_KV_WR_CTRL.rsvd.next = next_c;
1287                                     field_combo.SHA512_KV_WR_CTRL.rsvd.load_next =
load_next_c;
1288                                     end
1289                                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1290                                         if(~hwif_in.reset_b) begin
1291                                             field_storage.SHA512_KV_WR_CTRL.rsvd.value <=
17'h0;
1292                                             end else
1293                                             if(field_combo.SHA512_KV_WR_CTRL.rsvd.load_next) begin
1294                                                 field_storage.SHA512_KV_WR_CTRL.rsvd.value <=
field_combo.SHA512_KV_WR_CTRL.rsvd.next;
1295                                                 end
1296                                                 assign hwif_out.SHA512_KV_WR_CTRL.rsvd.value =
field_storage.SHA512_KV_WR_CTRL.rsvd.value;
1297                                                 // Field: sha512_reg.SHA512_KV_WR_STATUS.VALID
1298                                                 always_comb begin
1299                                                     automatic logic [0:0] next_c;
1300                                                     automatic logic load_next_c;
1301                                                     next_c =
field_storage.SHA512_KV_WR_STATUS.VALID.value;
1302                                                     load_next_c = '0;
1303                                                     if(hwif_in.SHA512_KV_WR_STATUS.VALID.hwset) begin //
HW Set
1304                                                         next_c = '1;
1305                                                         load_next_c = '1;
1306                                                         end else if(hwif_in.SHA512_KV_WR_STATUS.VALID.hwclr)
begin // HW Clear
1307                                                             next_c = '0;
1308                                                             load_next_c = '1;
1309                                                             end
1310                                                             field_combo.SHA512_KV_WR_STATUS.VALID.next = next_c;
1311                                                             field_combo.SHA512_KV_WR_STATUS.VALID.load_next =
load_next_c;
1312                                                             end
1313                                                             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1314                                                                 if(~hwif_in.reset_b) begin
1315                                                                     field_storage.SHA512_KV_WR_STATUS.VALID.value <=
1'h0;
1316                                                                     end else
1317                                                                     if(field_combo.SHA512_KV_WR_STATUS.VALID.load_next) begin
1318                                                                         field_storage.SHA512_KV_WR_STATUS.VALID.value <=
field_combo.SHA512_KV_WR_STATUS.VALID.next;
1319                                                                         end

```

```

1319     end
1320     for(genvar i0=0; i0<8; i0++) begin
1321         // Field: sha512_reg.SHA512_GEN_PCR_HASH_NONCE[i].NONCE
1322         always_comb begin
1323             automatic logic [31:0] next_c;
1324             automatic logic load_next_c;
1325             next_c =
field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value;
1326             load_next_c = '0;
1327             if(decoded_reg_strb.SHA512_GEN_PCR_HASH_NONCE[i0]
&& decoded_req_is_wr) begin // SW write
1328                 next_c =
(field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1329                 load_next_c = '1;
1330             end
1331         end
field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.next = next_c;
1332     end
field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.load_next =
load_next_c;
1333     end
1334     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
1335         if(~hwif_in.reset_b) begin
1336             field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value <= 32'h0;
1337         end else
1338             if(field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.load_next) begin
1339                 field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value <=
field_combo.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.next;
1340             end
1341             assign
hwif_out.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value =
field_storage.SHA512_GEN_PCR_HASH_NONCE[i0].NONCE.value;
1342         end
1343         // Field: sha512_reg.SHA512_GEN_PCR_HASH_CTRL.START
1344         always_comb begin
1345             automatic logic [0:0] next_c;
1346             automatic logic load_next_c;
1347             next_c =
field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value;
1348             load_next_c = '0;
1349             if(decoded_reg_strb.SHA512_GEN_PCR_HASH_CTRL && de-
coded_req_is_wr && hwif_in.sha512_ready) begin // SW write
1350                 next_c =
(field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1351                 load_next_c = '1;
1352             end else begin // singlepulse clears back to 0
1353                 next_c = '0;
1354                 load_next_c = '1;
1355             end
1356             field_combo.SHA512_GEN_PCR_HASH_CTRL.START.next =
next_c;
1357         end
field_combo.SHA512_GEN_PCR_HASH_CTRL.START.load_next =
load_next_c;
1358     end
1359     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1360         if(~hwif_in.reset_b) begin
1361             field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value
<= 1'h0;
1362         end else
1363             if(field_combo.SHA512_GEN_PCR_HASH_CTRL.START.load_next) begin
1364                 field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value
<= field_combo.SHA512_GEN_PCR_HASH_CTRL.START.next;
1365             end
1366             assign hwif_out.SHA512_GEN_PCR_HASH_CTRL.START.value =
field_storage.SHA512_GEN_PCR_HASH_CTRL.START.value;
1367             // Field: sha512_reg.SHA512_GEN_PCR_HASH_STATUS.VALID
1368             always_comb begin
1369                 automatic logic [0:0] next_c;
1370                 automatic logic load_next_c;
1371                 next_c =
field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value;
1372                 load_next_c = '0;
1373                 if(hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwset) be-
gin // HW Set
1374                     next_c = '1;
1375                     load_next_c = '1;
1376                 end else
1377                     if(hwif_in.SHA512_GEN_PCR_HASH_STATUS.VALID.hwclr) begin // HW Clear
1378                         next_c = '0;
1379                         load_next_c = '1;
1380                     end
1381                     field_combo.SHA512_GEN_PCR_HASH_STATUS.VALID.next =
next_c;
1382                 end
1383                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1384                     if(~hwif_in.reset_b) begin

```

```

1385         field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value <= 1'h0;
1386         end else
1387             if(field_combo.SHA512_GEN_PCR_HASH_STATUS.VALID.load_next) begin
1388                 field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value <=
field_combo.SHA512_GEN_PCR_HASH_STATUS.VALID.next;
1389             end
1390         for(genvar i0=0; i0<16; i0++) begin
1391             // Field:
sha512_reg.SHA512_GEN_PCR_HASH_DIGEST[i].DIGEST
1392             always_comb begin
1393                 automatic logic [31:0] next_c;
1394                 automatic logic load_next_c;
1395                 next_c =
field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value;
1396                 load_next_c = '0;
1397             end
1398             if(hwif_in.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.hwclr) begin // HW
Clear
1399                 next_c = '0;
1400                 load_next_c = '1;
1401             end else begin // HW Write
1402                 next_c =
hwif_in.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.next;
1403                 load_next_c = '1;
1404             end
1405             field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.next = next_c;
1406             field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.load_next =
load_next_c;
1407             always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
1408                 if(~hwif_in.reset_b) begin
1409                     field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value <= 32'h0;
1410                 end else
1411                     if(field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.load_next) be-
gin
1412                         field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value <=
field_combo.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.next;
1413                     end
1414                     end
1415                     // Field:
sha512_reg.intr_block_rf.global_intr_en_r.error_en
1416                     always_comb begin
1417                         automatic logic [0:0] next_c;
1418                         automatic logic load_next_c;
1419                         next_c =
field_storage.intr_block_rf.global_intr_en_r.error_en.value;
1420                         load_next_c = '0;
1421                         if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1422                             next_c =
(field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1423                             load_next_c = '1;
1424                         end
1425                         field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;
1426                     end
1427                     field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
load_next_c;
1428                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1429                         if(~hwif_in.reset_b) begin
1430                             field_storage.intr_block_rf.global_intr_en_r.error_en.value <= 1'h0;
1431                         end else
1432                             if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
begin
1433                                 field_storage.intr_block_rf.global_intr_en_r.error_en.value <=
field_combo.intr_block_rf.global_intr_en_r.error_en.next;
1434                             end
1435                             end
1436                             // Field:
sha512_reg.intr_block_rf.global_intr_en_r.notif_en
1437                             always_comb begin
1438                                 automatic logic [0:0] next_c;
1439                                 automatic logic load_next_c;
1440                                 next_c =
field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
1441                                 load_next_c = '0;
1442                                 if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1443                                     next_c =
(field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1444                                     load_next_c = '1;
1445                                 end
1446                                 field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;

```



```

1442 field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
1443 load_next_c;
1444     end
1445     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1446         if(~hwif_in.reset_b) begin
1447             field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= '1'h0;
1448         end else
1449             if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
1450                 begin
1451                     field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
1452                     field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
1453                 end
1454             end
1455             // Field:
1456             sha512_reg.intr_block_rf.error_intr_en_r.error0_en
1457             always_comb begin
1458                 automatic logic [0:0] next_c;
1459                 automatic logic load_next_c;
1460                 next_c =
1461                 field_storage.intr_block_rf.error_intr_en_r.error0_en.value;
1462                 load_next_c = '0;
1463                 if(decoded_reg.strb.intr_block_rf.error_intr_en_r &&
1464                 decoded_req_is_wr) begin // SW write
1465                     next_c =
1466                     (field_storage.intr_block_rf.error_intr_en_r.error0_en.value & ~de-
1467                     coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
1468                     coded_wr_biten[0:0]);
1469                     load_next_c = '1;
1470                 end
1471             end
1472             field_combo.intr_block_rf.error_intr_en_r.error0_en.next = next_c;
1473             field_storage.intr_block_rf.error_intr_en_r.error0_en.load_next =
1474             load_next_c;
1475             end
1476             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1477                 if(~hwif_in.reset_b) begin
1478                     field_storage.intr_block_rf.error_intr_en_r.error0_en.value <= '1'h0;
1479                 end else
1480                     if(field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next)
1481                         begin
1482                             field_storage.intr_block_rf.error_intr_en_r.error0_en.value <=
1483                             field_combo.intr_block_rf.error_intr_en_r.error0_en.next;
1484                         end
1485                     end
1486                     // Field:
1487                     sha512_reg.intr_block_rf.error_intr_en_r.error1_en
1488                     always_comb begin
1489                         automatic logic [0:0] next_c;
1490                         automatic logic load_next_c;
1491                         next_c =
1492                         field_storage.intr_block_rf.error_intr_en_r.error1_en.value;
1493                         load_next_c = '0;
1494                         if(decoded_reg.strb.intr_block_rf.error_intr_en_r &&
1495                         decoded_req_is_wr) begin // SW write
1496                             next_c =
1497                             (field_storage.intr_block_rf.error_intr_en_r.error1_en.value & ~de-
1498                             coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
1499                             coded_wr_biten[1:1]);
1500                             load_next_c = '1;
1501                         end
1502                     end
1503                     field_combo.intr_block_rf.error_intr_en_r.error1_en.next = next_c;
1504                     field_storage.intr_block_rf.error_intr_en_r.error1_en.load_next =
1505                     load_next_c;
1506                     end
1507                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1508                         if(~hwif_in.reset_b) begin
1509                             field_storage.intr_block_rf.error_intr_en_r.error1_en.value <= '1'h0;
1510                         end else
1511                             if(field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next)
1512                                 begin
1513                                     field_storage.intr_block_rf.error_intr_en_r.error1_en.value <=
1514                                     field_combo.intr_block_rf.error_intr_en_r.error1_en.next;
1515                                 end
1516                             end
1517                             // Field:
1518                             sha512_reg.intr_block_rf.error_intr_en_r.error2_en
1519                             always_comb begin
1520                                 automatic logic [0:0] next_c;
1521                                 automatic logic load_next_c;
1522                                 next_c =
1523                                 field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
1524                                 load_next_c = '0;
1525                                 if(decoded_reg.strb.intr_block_rf.error_intr_en_r &&
1526                                 decoded_req_is_wr) begin // SW write
1527                                     next_c =
1528                                     (field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
1529                                     coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
1530                                     coded_wr_biten[2:2]);
1531                                     load_next_c = '1;
1532                                 end
1533                             end
1534                             field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;

```

```

1506 field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
1507 load_next_c;
1508 end
1509 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1510     if(~hwif_in.reset_b) begin
1511         field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= '1'h0;
1512     end else
1513         if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
1514             begin
1515                 field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
1516                 field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
1517             end
1518         end
1519         // Field:
1520 sha512_reg.intr_block_rf.error_intr_en_r.error3_en
1521         always_comb begin
1522             automatic logic [0:0] next_c;
1523             automatic logic load_next_c;
1524             next_c =
1525             field_storage.intr_block_rf.error_intr_en_r.error3_en.value;
1526             load_next_c = '0;
1527             if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
1528             decoded_req_is_wr) begin // SW write
1529                 next_c =
1530                 (field_storage.intr_block_rf.error_intr_en_r.error3_en.value & ~de-
1531                 coded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
1532                 coded_wr_biten[3:3]);
1533                 load_next_c = '1;
1534             end
1535             end
1536             field_combo.intr_block_rf.error_intr_en_r.error3_en.next = next_c;
1537             field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next =
1538             load_next_c;
1539             end
1540             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1541                 if(~hwif_in.reset_b) begin
1542                     field_storage.intr_block_rf.error_intr_en_r.error3_en.value <= '1'h0;
1543                 end else
1544                     if(field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next)
1545                         begin
1546                             field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=
1547                             field_combo.intr_block_rf.error_intr_en_r.error3_en.next;
1548                         end
1549                     end
1550                     // Field:
1551                     sha512_reg.intr_block_rf.notif_intr_en_r.notif_cmd_done_en
1552                     always_comb begin
1553                         automatic logic [0:0] next_c;
1554                         automatic logic load_next_c;
1555                         next_c =
1556                         field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value;
1557                         load_next_c = '0;
1558                         if(decoded_reg_strb.intr_block_rf.notif_intr_en_r &&
1559                         decoded_req_is_wr) begin // SW write
1560                             next_c =
1561                             (field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value & ~de-
1562                             coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
1563                             coded_wr_biten[0:0]);
1564                             load_next_c = '1;
1565                         end
1566                         end
1567                         field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next =
1568                         next_c;
1569                         field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_ne
1570                         xt = load_next_c;
1571                         end
1572                         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1573                             if(~hwif_in.reset_b) begin
1574                                 field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
1575                                 <= '1'h0;
1576                             end else
1577                                 if(field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_
1578                                 next) begin
1579                                     field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
1580                                     <= field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next;
1581                                 end
1582                                 end
1583                                 // Field:
1584                                 sha512_reg.intr_block_rf.error_global_intr_r.agg_sts
1585                                 always_comb begin
1586                                     automatic logic [0:0] next_c;
1587                                     automatic logic load_next_c;
1588                                     next_c =
1589                                     field_storage.intr_block_rf.error_global_intr_r.agg_sts.value;
1590                                     load_next_c = '0;
1591                                 end
1592                                 // HW Write
1593                                 next_c = hwif_out.intr_b-
1594                                 lock_rf.error_internal_intr_r.intr;
1595                                 load_next_c = '1;
1596                                 end
1597                                 field_combo.intr_block_rf.error_global_intr_r.agg_sts.next = next_c;

```

```

1566 field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next =
1567     load_next_c;
1568     end
1569     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1570         if(~hwif_in.reset_b) begin
1571             field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1572                 1'h0;
1573             end else
1574                 if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
1575                     begin
1576                         field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1577                             field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
1578                     end
1579                 end
1580                 assign hwif_out.intr_block_rf.error_global_intr_r.intr =
1581                     |
1582                     (field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
1583                     field_storage.intr_block_rf.global_intr_en_r.error_en.value);
1584             // Field:
1585             sha512_reg.intr_block_rf.notif_global_intr_r.agg_sts
1586             always_comb begin
1587                 automatic logic [0:0] next_c;
1588                 automatic logic load_next_c;
1589                 next_c =
1590                     field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
1591                 load_next_c = '0;
1592             end
1593             // HW Write
1594             next_c = hwif_out.intr_b-
1595             lock_rf.notif_internal_intr_r.intr;
1596             load_next_c = '1;
1597         end
1598         field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
1599     end
1600     field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
1601         load_next_c;
1602     end
1603     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1604         if(~hwif_in.reset_b) begin
1605             field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1606                 1'h0;
1607             end else
1608                 if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
1609                     begin
1610                         field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1611                             field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
1612                     end
1613                 end
1614                 assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
1615                     |
1616                     (field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
1617                     field_storage.intr_block_rf.global_intr_en_r.notif_en.value);
1618             // Field:
1619             sha512_reg.intr_block_rf.error_internal_intr_r.error0_sts
1620             always_comb begin
1621                 automatic logic [0:0] next_c;
1622                 automatic logic load_next_c;
1623                 next_c =
1624                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1625                 load_next_c = '0;
1626             end
1627             if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
1628                 '0) begin // stickybit
1629                 next_c =
1630                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
1631                     field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1632                 load_next_c = '1;
1633             end else if(hwif_in.intr_b-
1634             lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1635                 next_c = '1;
1636                 load_next_c = '1;
1637             end else if(de-
1638             coded_reg.strb.intr_block_rf.error_internal_intr_r && de-
1639             coded_req_is_wr) begin // SW write 1 clear
1640                 next_c =
1641                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1642                     ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1643                 load_next_c = '1;
1644             end
1645         end
1646         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
1647             next_c;
1648     end
1649     field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next =
1650         load_next_c;
1651     end
1652     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1653     begin
1654         if(~hwif_in.error_reset_b) begin
1655             field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1656             <= 1'h0;
1657             end else
1658                 if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
1659                 next) begin
1660                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value <=
1661                         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1662                 end
1663                 assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1664                     |
1665                     (field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1666                     field_storage.intr_block_rf.global_intr_en_r.error_en.value);
1667             // Field:
1668             sha512_reg.intr_block_rf.error_internal_intr_r.error0_sts
1669             always_comb begin
1670                 automatic logic [0:0] next_c;
1671                 automatic logic load_next_c;
1672                 next_c =
1673                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1674                 load_next_c = '0;
1675             end
1676             if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
1677                 '0) begin // stickybit
1678                 next_c =
1679                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
1680                     field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1681                 load_next_c = '1;
1682             end else if(hwif_in.intr_b-
1683             lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1684                 next_c = '1;
1685                 load_next_c = '1;
1686             end else if(de-
1687             coded_reg.strb.intr_block_rf.error_internal_intr_r && de-
1688             coded_req_is_wr) begin // SW write 1 clear
1689                 next_c =
1690                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1691                     ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1692                 load_next_c = '1;
1693             end
1694         end
1695         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
1696             next_c;
1697     end
1698     field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next =
1699         load_next_c;
1700     end
1701     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1702     begin
1703         if(~hwif_in.error_reset_b) begin
1704             field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1705             <= 1'h0;
1706             end else
1707                 if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
1708                 next) begin
1709                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value <=
1710                         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1711                 end
1712                 assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1713                     |
1714                     (field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1715                     field_storage.intr_block_rf.global_intr_en_r.error_en.value);
1716             // Field:
1717             sha512_reg.intr_block_rf.error_internal_intr_r.error0_sts
1718             always_comb begin
1719                 automatic logic [0:0] next_c;
1720                 automatic logic load_next_c;
1721                 next_c =
1722                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1723                 load_next_c = '0;
1724             end
1725             if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
1726                 '0) begin // stickybit
1727                 next_c =
1728                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
1729                     field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1730                 load_next_c = '1;
1731             end else if(hwif_in.intr_b-
1732             lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1733                 next_c = '1;
1734                 load_next_c = '1;
1735             end else if(de-
1736             coded_reg.strb.intr_block_rf.error_internal_intr_r && de-
1737             coded_req_is_wr) begin // SW write 1 clear
1738                 next_c =
1739                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1740                     ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1741                 load_next_c = '1;
1742             end
1743         end
1744         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
1745             next_c;
1746     end
1747     field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next =
1748         load_next_c;
1749     end
1750     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1751     begin
1752         if(~hwif_in.error_reset_b) begin
1753             field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1754             <= 1'h0;
1755             end else
1756                 if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
1757                 next) begin
1758                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value <=
1759                         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1760                 end
1761                 assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1762                     |
1763                     (field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1764                     field_storage.intr_block_rf.global_intr_en_r.error_en.value);
1765             // Field:
1766             sha512_reg.intr_block_rf.error_internal_intr_r.error0_sts
1767             always_comb begin
1768                 automatic logic [0:0] next_c;
1769                 automatic logic load_next_c;
1770                 next_c =
1771                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1772                 load_next_c = '0;
1773             end
1774             if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
1775                 '0) begin // stickybit
1776                 next_c =
1777                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
1778                     field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1779                 load_next_c = '1;
1780             end else if(hwif_in.intr_b-
1781             lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1782                 next_c = '1;
1783                 load_next_c = '1;
1784             end else if(de-
1785             coded_reg.strb.intr_block_rf.error_internal_intr_r && de-
1786             coded_req_is_wr) begin // SW write 1 clear
1787                 next_c =
1788                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1789                     ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1790                 load_next_c = '1;
1791             end
1792         end
1793         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
1794             next_c;
1795     end
1796     field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next =
1797         load_next_c;
1798     end
1799     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1800     begin
1801         if(~hwif_in.error_reset_b) begin
1802             field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
1803             <= 1'h0;
1804             end else
1805                 if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
1806                 next) begin
1807                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value <=
1808                         field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1809                 end
1810                 assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1811                     |
1812                     (field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
1813                     field_storage.intr_block_rf.global_intr_en_r.error_en.value);
1814             // Field:
1815             sha512_reg.intr_block_rf.error_internal_intr_r.error0_sts
1816             always_comb begin
1817                 automatic logic [0:0] next_c;
1818                 automatic logic load_next_c;
1819                 next_c =
1820                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1821                 load_next_c = '0;
1822             end
1823             if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
1824                 '0) begin // stickybit
1825                 next_c =
1826                     field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
1827                     field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1828                 load_next_c = '1;
1829             end else if(hwif_in.intr_b-
1830             lock_rf.error_internal_intr_r.error0_sts
```

```

field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1623     end
1624   end
1625   // Field:
sha512_reg.intr_block_rf.error_internal_intr_r.error1_sts
1626   always_comb begin
1627     automatic logic [0:0] next_c;
1628     automatic logic load_next_c;
1629     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value;
1630     load_next_c = '0;
1631
if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value != '0) begin // stickybit
1632     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1633     load_next_c = '1;
1634   end else if(hwif_in.intr_block_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1635     next_c = '1;
1636     load_next_c = '1;
1637   end else if(decoded_reg_strb.intr_block_rf.error_internal_intr_r
decoded_req_is_wr) begin // SW write 1 clear
1638     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value &
~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1639     load_next_c = '1;
1640   end
1641
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next =
next_c;
1642
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_next
= load_next_c;
1643   end
1644   always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1645     if(~hwif_in.error_reset_b) begin
1646       field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= 1'h0;
1647     end else
if(field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_next) begin
1648
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next;
1649     end
1650   end
1651   // Field:
sha512_reg.intr_block_rf.error_internal_intr_r.error2_sts
1652   always_comb begin
1653     automatic logic [0:0] next_c;
1654     automatic logic load_next_c;
1655     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
1656     load_next_c = '0;
1657
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value != '0) begin // stickybit
1658     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1659     load_next_c = '1;
1660   end else if(hwif_in.intr_block_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1661     next_c = '1;
1662     load_next_c = '1;
1663   end else if(decoded_reg_strb.intr_block_rf.error_internal_intr_r
decoded_req_is_wr) begin // SW write 1 clear
1664     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1665     load_next_c = '1;
1666   end
1667
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
1668
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next
= load_next_c;
1669   end
1670   always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1671     if(~hwif_in.error_reset_b) begin
1672       field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= 1'h0;
1673     end else
if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next) begin
1674
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1675     end
1676   end
1677   // Field:

```

```

sha512_reg.intr_block_rf.error_internal_intr_r.error3_sts
1678     always_comb begin
1679         automatic logic [0:0] next_c;
1680         automatic logic load_next_c;
1681         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1682         load_next_c = '0;
1683
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value != '0) begin // stickybit
1684     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1685     load_next_c = '1;
1686     end else if(hwif_in.intr_block_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1687         next_c = '1;
1688         load_next_c = '1;
1689     end else if(decoded_reg_strb.intr_block_rf.error_internal_intr_r && decoded_req_is_wr) begin // SW write 1 clear
1690         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &
~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1691         load_next_c = '1;
1692     end
1693
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
next_c;
1694
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next =
load_next_c;
1695     end
1696     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1697         if(~hwif_in.error_reset_b) begin
1698
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= 1'h0;
1699         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next)
begin
1700
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1701         end
1702     end
1703     assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1704     |
(field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error0_en.value)
1705     || |
(field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error1_en.value)
1706     || |
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1707     || |
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1708     // Field:
sha512_reg.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
1709     always_comb begin
1710         automatic logic [0:0] next_c;
1711         automatic logic load_next_c;
1712         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1713         load_next_c = '0;
1714
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0) begin // stickybit
1715     next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1716     load_next_c = '1;
1717     end else if(hwif_in.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset)
begin // HW Set
1718         next_c = '1;
1719         load_next_c = '1;
1720     end else if(decoded_reg_strb.intr_block_rf.notif_internal_intr_r &&
decoded_req_is_wr) begin // SW write 1 clear
1721         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1722         load_next_c = '1;
1723     end
1724
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.next = next_c;
1725
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.load_next = load_next_c;
1726     end
1727     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1728         if(~hwif_in.reset_b) begin
1729

```

```

field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= 1'h0;
1730     end else
if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.load_next) begin
1731
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.next;
1732     end
1733     end
1734     assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
1735     |
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1736     // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error0_trig
1737     always_comb begin
1738         automatic logic [0:0] next_c;
1739         automatic logic load_next_c;
1740         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1741         load_next_c = '0;
1742
if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1743     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value |
(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1744     load_next_c = '1;
1745     end else begin // singlepulse clears back to 0
1746         next_c = '0;
1747         load_next_c = '1;
1748     end
1749
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next =
next_c;
1750
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next =
load_next_c;
1751     end
1752     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1753         if(~hwif_in.reset_b) begin
1754
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
1'h0;
1755     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next
) begin
1756
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next;
1757     end
1758     end
1759     // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error1_trig
1760     always_comb begin
1761         automatic logic [0:0] next_c;
1762         automatic logic load_next_c;
1763         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1764         load_next_c = '0;
1765
if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1766     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value |
(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1767     load_next_c = '1;
1768     end else begin // singlepulse clears back to 0
1769         next_c = '0;
1770         load_next_c = '1;
1771     end
1772
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next =
next_c;
1773
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next =
load_next_c;
1774     end
1775     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1776         if(~hwif_in.reset_b) begin
1777
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
1'h0;
1778     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next
) begin
1779
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next;
1780     end
1781     end
1782     // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error2_trig
1783     always_comb begin
1784         automatic logic [0:0] next_c;
1785         automatic logic load_next_c;

```



```

1786     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1787     load_next_c = '0';

1788     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1789         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1790         load_next_c = '1';
1791     end else begin // singlepulse clears back to 0
1792         next_c = '0';
1793         load_next_c = '1';
1794     end
1795
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
next_c;
1796
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
load_next_c;
1797 end
1798 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1799     if(~hwif_in.reset_b) begin
1800
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1'h0;
1801     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
) begin
1802
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1803     end
1804 end
1805 // Field:
sha512_reg.intr_block_rf.error_intr_trig_r.error3_trig
1806 always_comb begin
1807     automatic logic [0:0] next_c;
1808     automatic logic load_next_c;
1809     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1810     load_next_c = '0';

1811     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1812         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1813         load_next_c = '1';
1814     end else begin // singlepulse clears back to 0
1815         next_c = '0';
1816         load_next_c = '1';
1817     end
1818
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
next_c;
1819
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
load_next_c;
1820 end
1821 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1822     if(~hwif_in.reset_b) begin
1823
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1'h0;
1824     end else
if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
) begin
1825
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1826     end
1827 end
1828 // Field:
sha512_reg.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
1829 always_comb begin
1830     automatic logic [0:0] next_c;
1831     automatic logic load_next_c;
1832     next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1833     load_next_c = '0';

1834     if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1835         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1836         load_next_c = '1';
1837     end else begin // singlepulse clears back to 0
1838         next_c = '0';
1839         load_next_c = '1';
1840     end
1841
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
= next_c;
1842
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig loa
d_next = load_next_c;
1843 end
1844 always_ff @(posedge clk or negedge hwif_in.reset_b) begin

```

```

1845     if(~hwif_in.reset_b) begin
1846
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <= 1'h0;
1847     end else
if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
load_next) begin
1848
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <=
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
;
1849     end
1850 end
1851 // Field: sha512_reg.intr_block_rf.error0_intr_count_r.cnt
1852 always_comb begin
1853     automatic logic [31:0] next_c;
1854     automatic logic load_next_c;
1855     next_c =
field_storage.intr_block_rf.error0_intr_count_r.cnt.value;
1856     load_next_c = '0';
1857     if(decoded_reg_strb.intr_block_rf.error0_intr_count_r
&& decoded_req_is_wr) begin // SW write
1858         next_c =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1859         load_next_c = '1';
1860     end
1861
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // increment
1862     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1863         next_c = 32'hffffffff;
1864     end else begin
1865         next_c = next_c + 32'h1;
1866     end
1867     load_next_c = '1';
1868 end
1869
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hffffffff);
1870
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hffffffff);
1871     if(next_c > 32'hffffffff) begin
1872         next_c = 32'hffffffff;
1873         load_next_c = '1';
1874     end

1875     field_combo.intr_block_rf.error0_intr_count_r.cnt.next
= next_c;
1876
field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next =
load_next_c;
1877 end
1878 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1879     if(~hwif_in.error_reset_b) begin
1880
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <= 32'h0;
1881     end else
if(field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next) be-
gin
1882
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error0_intr_count_r.cnt.next;
1883     end
1884 end
1885 // Field: sha512_reg.intr_block_rf.error1_intr_count_r.cnt
1886 always_comb begin
1887     automatic logic [31:0] next_c;
1888     automatic logic load_next_c;
1889     next_c =
field_storage.intr_block_rf.error1_intr_count_r.cnt.value;
1890     load_next_c = '0';
1891     if(decoded_reg_strb.intr_block_rf.error1_intr_count_r
&& decoded_req_is_wr) begin // SW write
1892         next_c =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1893         load_next_c = '1';
1894     end
1895
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // increment
1896     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1897         next_c = 32'hffffffff;
1898     end else begin
1899         next_c = next_c + 32'h1;
1900     end
1901     load_next_c = '1';
1902 end
1903
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=

```

```

32'hffffffff);
1904
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hffffffff);
1905     if(next_c > 32'hffffffff) begin
1906         next_c = 32'hffffffff;
1907         load_next_c = '1;
1908     end

1909     field_combo.intr_block_rf.error1_intr_count_r.cnt.next
= next_c;
1910
field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next =
load_next_c;
1911     end
1912     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1913         if(~hwif_in.error_reset_b) begin
1914             field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1915         end else
1916             if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
gin
1917                 field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1918             end
1919             // Field: sha512_reg.intr_block_rf.error2_intr_count_r.cnt
always_comb begin
1920                 automatic logic [31:0] next_c;
1921                 automatic logic load_next_c;
1922                 next_c =
field_storage.intr_block_rf.error2_intr_count_r.cnt.value;
1923                 load_next_c = '0;
1924                 if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
&& decoded_req_is_wr) begin // SW write
1925                     next_c =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1926                     load_next_c = '1;
1927                 end
1928             end
1929             if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // increment
1930                 if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1931                     next_c = 32'hffffffff;
1932                 end else begin
1933                     next_c = next_c + 32'h1;
1934                 end
1935                 load_next_c = '1;
1936             end
1937             field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
1938             field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hffffffff);
1939             if(next_c > 32'hffffffff) begin
1940                 next_c = 32'hffffffff;
1941                 load_next_c = '1;
1942             end

1943             field_combo.intr_block_rf.error2_intr_count_r.cnt.next
= next_c;
1944             field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
load_next_c;
1945         end
1946         always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1947             if(~hwif_in.error_reset_b) begin
1948                 field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;
1949             end else
1950                 if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
gin
1951                     field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
1952                 end
1953                 // Field: sha512_reg.intr_block_rf.error3_intr_count_r.cnt
always_comb begin
1954                     automatic logic [31:0] next_c;
1955                     automatic logic load_next_c;
1956                     next_c =
field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
1957                     load_next_c = '0;
1958                     if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
&& decoded_req_is_wr) begin // SW write
1959                         next_c =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1960                         load_next_c = '1;

```

```

1962         end
1963     end
1964     if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // increment
1965         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1966             next_c = 32'hffffffff;
1967         end else begin
1968             next_c = next_c + 32'h1;
1969         end
1970         load_next_c = '1;
1971     end
1972     field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hffffffff);
1973     field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hffffffff);
1974     if(next_c > 32'hffffffff) begin
1975         next_c = 32'hffffffff;
1976         load_next_c = '1;
1977     end

1978     field_combo.intr_block_rf.error3_intr_count_r.cnt.next
= next_c;
1979     field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
1980     end
1981     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1982         if(~hwif_in.error_reset_b) begin
1983             field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
1984         end else
1985             if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
1986                 field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
1987             end
1988             // Field:
sha512_reg.intr_block_rf.notif_cmd_done_intr_count_r.cnt
always_comb begin
1989                 automatic logic [31:0] next_c;
1990                 automatic logic load_next_c;
1991                 next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
1992                 load_next_c = '0;
1993                 if(de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_req_is_wr) begin // SW write
1994                     next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1995                     load_next_c = '1;
1996                 end
1997                 if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul-
se.value) begin // increment
1998                     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1999                         next_c = 32'hffffffff;
2000                     end else begin
2001                         next_c = next_c + 32'h1;
2002                     end
2003                     load_next_c = '1;
2004                 end
2005                 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres-
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
2006                 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur-
ate =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
2007                 if(next_c > 32'hffffffff) begin
2008                     next_c = 32'hffffffff;
2009                     load_next_c = '1;
2010                 end
2011                 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
2012                 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;
2013             end
2014             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2015                 if(~hwif_in.reset_b) begin
2016                     field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
32'h0;
2017                 end else
2018                     if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n-
ext) begin

```

```

2018
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
2019     end
2020   end
2021   // Field:
sha512_reg.intr_block_rf.error0_intr_count_incr_r.pulse
2022     always_comb begin
2023       automatic logic [0:0] next_c;
2024       automatic logic load_next_c;
2025       next_c =
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value;
2026       load_next_c = '0;
2027
if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value)
begin // HW Write - we
2028   next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
2029   load_next_c = '1;
2030   end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
2031     next_c = '1;
2032     load_next_c = '1;
2033   end
2034
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // decrement
2035   field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2036   next_c = next_c - 1'h1;
2037   load_next_c = '1;
2038   end else begin
2039
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
'0;
2040   end
2041
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'd0);
2042
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next =
next_c;
2043
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_next =
load_next_c;
2044   end
2045   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2046     if(~hwif_in.reset_b) begin
2047
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'h0;
2048     end else
if(field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_ne
xt) begin
2049
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next;
2050   end
2051   end
2052   // Field:
sha512_reg.intr_block_rf.error1_intr_count_incr_r.pulse
2053     always_comb begin
2054       automatic logic [0:0] next_c;
2055       automatic logic load_next_c;
2056       next_c =
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value;
2057       load_next_c = '0;
2058
if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value)
begin // HW Write - we
2059   next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
2060   load_next_c = '1;
2061   end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
2062     next_c = '1;
2063     load_next_c = '1;
2064   end
2065
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // decrement
2066   field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2067   next_c = next_c - 1'h1;
2068   load_next_c = '1;
2069   end else begin
2070
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
'0;
2071   end
2072
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'd0);
2073
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =

```

```

next_c;
2074
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
2075   end
2076   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2077     if(~hwif_in.reset_b) begin
2078
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
2079     end else
if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_ne
xt) begin
2080
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next;
2081   end
2082   end
2083   // Field:
sha512_reg.intr_block_rf.error2_intr_count_incr_r.pulse
2084     always_comb begin
2085       automatic logic [0:0] next_c;
2086       automatic logic load_next_c;
2087       next_c =
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
2088       load_next_c = '0;
2089
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
begin // HW Write - we
2090   next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
2091   load_next_c = '1;
2092   end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
2093     next_c = '1;
2094     load_next_c = '1;
2095   end
2096
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
2097   field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2098   next_c = next_c - 1'h1;
2099   load_next_c = '1;
2100   end else begin
2101
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
'0;
2102   end
2103
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'd0);
2104
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
next_c;
2105
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
load_next_c;
2106   end
2107   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2108     if(~hwif_in.reset_b) begin
2109
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'h0;
2110     end else
if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
xt) begin
2111
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
2112   end
2113   end
2114   // Field:
sha512_reg.intr_block_rf.error3_intr_count_incr_r.pulse
2115     always_comb begin
2116       automatic logic [0:0] next_c;
2117       automatic logic load_next_c;
2118       next_c =
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
2119       load_next_c = '0;
2120
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value)
begin // HW Write - we
2121   next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
2122   load_next_c = '1;
2123   end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
2124     next_c = '1;
2125     load_next_c = '1;
2126   end
2127
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // decrement
2128   field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
2129   next_c = next_c - 1'h1;

```

```

2130         load_next_c = '1;
2131     end else begin
2132         field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
'0;
2133     end
2134     field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'h0);
2135     field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next =
next_c;
2136     field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next =
load_next_c;
2137     end
2138     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2139         if(~hwif_in.reset_b) begin
2140             field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'h0;
2141             end else
2142             if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_ne
xt) begin
2143                 field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
2144             end
2145             // Field:
sha512_reg.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse
2146             always_comb begin
2147                 automatic logic [0:0] next_c;
2148                 automatic logic load_next_c;
2149                 next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value;
2150                 load_next_c = '0;
2151             if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value) begin // HW Write - we
2152                 next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
2153                 load_next_c = '1;
2154             end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
2155                 next_c = '1;
2156                 load_next_c = '1;
2157             end
2158             if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // decrement
2159                 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = (next_c < (1'h1));
2160                 next_c = next_c - 1'h1;
2161                 load_next_c = '1;
2162             end else begin
2163                 field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = '0;
2164             end
2165             field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
crthreshold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'h0);
2166             field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt = next_c;
2167             field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
ad_next = load_next_c;
2168             end
2169             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2170                 if(~hwif_in.reset_b) begin
2171                     field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'h0;
2172                     end else
2173                     if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
load_next) begin
2174                         field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt;
2175                     end
2176                     //-----
2177                     // Write response
2178                     //-----
2179                     assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;
2180                     // Writes are always granted with no error response

```

```

2182         assign cpuif_wr_err = '0;
2183         //-----
2184         // Readback
2185         //-----
2186         // Assign readback values to a flattened array
2187         logic [61:1:0][31:0] readback_array;
2188         logic readback_err;
2189         logic readback_done;
2190         logic [31:0] readback_data;
2191         // Assign readback values to a flattened array
2192         logic [61:1:0][31:0] readback_array;
2193         for(genvar i0=0; i0<2; i0++) begin
2194             assign readback_array[i0*1 + 0][31:0] = (de-
coded_reg_strb.SHA512_NAME[i0] && !decoded_req_is_wr) ?
hwif_in.SHA512_NAME[i0].NAME.next : '0;
2195         end
2196         for(genvar i0=0; i0<2; i0++) begin
2197             assign readback_array[i0*1 + 2][31:0] = (de-
coded_reg_strb.SHA512_VERSION[i0] && !decoded_req_is_wr) ?
hwif_in.SHA512_VERSION[i0].VERSION.next : '0;
2198         end
2199         assign readback_array[4][0:0] = (de-
coded_reg_strb.SHA512_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_STATUS.READY.next : '0;
2200         assign readback_array[4][1:1] = (de-
coded_reg_strb.SHA512_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_STATUS.VALID.next : '0;
2201         assign readback_array[4][31:2] = '0;
2202         for(genvar i0=0; i0<16; i0++) begin
2203             assign readback_array[i0*1 + 5][31:0] = (de-
coded_reg_strb.SHA512_DIGEST[i0] && !decoded_req_is_wr) ?
field_storage.SHA512_DIGEST[i0].DIGEST.value : '0;
2204         end
2205         assign readback_array[21][0:0] = (de-
coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_VAULT_RD_CTRL.read_en.value : '0;
2206         assign readback_array[21][5:1] = (de-
coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_VAULT_RD_CTRL.read_entry.value : '0;
2207         assign readback_array[21][6:6] = (de-
coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_VAULT_RD_CTRL.pcr_hash_extend.value : '0;
2208         assign readback_array[21][31:7] = (de-
coded_reg_strb.SHA512_VAULT_RD_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_VAULT_RD_CTRL.rsvd.value : '0;
2209         assign readback_array[22][0:0] = (de-
coded_reg_strb.SHA512_VAULT_RD_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_VAULT_RD_STATUS.READY.next : '0;
2210         assign readback_array[22][1:1] = (de-
coded_reg_strb.SHA512_VAULT_RD_STATUS && !decoded_req_is_wr) ?
field_storage.SHA512_VAULT_RD_STATUS.VALID.value : '0;
2211         assign readback_array[22][9:2] = (de-
coded_reg_strb.SHA512_VAULT_RD_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_VAULT_RD_STATUS.ERROR.next : '0;
2212         assign readback_array[22][31:10] = '0;
2213         assign readback_array[23][0:0] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.write_en.value : '0;
2214         assign readback_array[23][5:1] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.write_entry.value : '0;
2215         assign readback_array[23][6:6] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.hmac_key_dest_valid.value : '0;
2216         assign readback_array[23][7:7] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.hmac_block_dest_valid.value : '0;
2217         assign readback_array[23][8:8] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.mlds_a_seed_dest_valid.value : '0;
2218         assign readback_array[23][9:9] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.ecc_pkey_dest_valid.value : '0;
2219         assign readback_array[23][10:10] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.ecc_seed_dest_valid.value : '0;
2220         assign readback_array[23][13:13] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.mlkem_seed_dest_valid.value : '0;
2221         assign readback_array[23][14:14] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.mlkem_msg_dest_valid.value : '0;
2222         assign readback_array[23][12:12] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.dma_data_dest_valid.value : '0;
2223         assign readback_array[23][31:15] = (de-
coded_reg_strb.SHA512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_CTRL.rsvd.value : '0;
2224         assign readback_array[24][0:0] = (de-
coded_reg_strb.SHA512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_KV_WR_STATUS.READY.next : '0;
2225         assign readback_array[24][1:1] = (de-
coded_reg_strb.SHA512_KV_WR_STATUS && !decoded_req_is_wr) ?
field_storage.SHA512_KV_WR_STATUS.VALID.value : '0;
2226         assign readback_array[24][9:2] = (de-

```

```

coded_reg_strb.SHA512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_KV_WR_STATUS.ERROR.next : '0;
2229 assign readback_array[24][31:10] = '0;
2230 assign readback_array[25][0:0] = (de-
coded_reg_strb.SHA512_GEN_PCR_HASH_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA512_GEN_PCR_HASH_STATUS.READY.next : '0;
2231 assign readback_array[25][1:1] = (de-
coded_reg_strb.SHA512_GEN_PCR_HASH_STATUS && !decoded_req_is_wr) ?
field_storage.SHA512_GEN_PCR_HASH_STATUS.VALID.value : '0;
2232 assign readback_array[25][31:2] = '0;
2233 for(genvar i0=0; i0<16; i0++) begin
2234 assign readback_array[i0*1 + 26][31:0] = (de-
coded_reg_strb.SHA512_GEN_PCR_HASH_DIGEST[i0] && !de-
coded_req_is_wr) ?
field_storage.SHA512_GEN_PCR_HASH_DIGEST[i0].DIGEST.value : '0;
2235 end
2236 assign readback_array[42][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0;
2237 assign readback_array[42][1:1] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0;
2238 assign readback_array[42][31:2] = '0;
2239 assign readback_array[43][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error0_en.value : '0;
2240 assign readback_array[43][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error1_en.value : '0;
2241 assign readback_array[43][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0;
2242 assign readback_array[43][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0;
2243 assign readback_array[43][31:4] = '0;
2244 assign readback_array[44][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value : '0;
2245 assign readback_array[44][31:1] = '0;
2246 assign readback_array[45][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0;
2247 assign readback_array[45][31:1] = '0;
2248 assign readback_array[46][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0;
2249 assign readback_array[46][31:1] = '0;
2250 assign readback_array[47][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value : '0;
2251 assign readback_array[47][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value : '0;
2252 assign readback_array[47][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value : '0;
2253 assign readback_array[47][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value : '0;
2254 assign readback_array[47][31:4] = '0;
2255 assign readback_array[48][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value : '0;
2256 assign readback_array[48][31:1] = '0;
2257 assign readback_array[49][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value : '0;
2258 assign readback_array[49][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value : '0;
2259 assign readback_array[49][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value : '0;
2260 assign readback_array[49][3:3] = (de-

```

```

coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value : '0;
2261 assign readback_array[49][31:4] = '0;
2262 assign readback_array[50][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue : '0;
2263 assign readback_array[50][31:1] = '0;
2264 assign readback_array[51][31:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_r.cnt.value : '0;
2265 assign readback_array[52][31:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_r.cnt.value : '0;
2266 assign readback_array[53][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0;
2267 assign readback_array[54][31:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0;
2268 assign readback_array[55][31:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value : '0;
2269 assign readback_array[56][0:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value : '0;
2270 assign readback_array[56][31:1] = '0;
2271 assign readback_array[57][0:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value : '0;
2272 assign readback_array[57][31:1] = '0;
2273 assign readback_array[58][0:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value : '0;
2274 assign readback_array[58][31:1] = '0;
2275 assign readback_array[59][0:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value : '0;
2276 assign readback_array[59][31:1] = '0;
2277 assign readback_array[60][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse
.value : '0;
2278 assign readback_array[60][31:1] = '0;
2279
2280 // Reduce the array
2281 always_comb begin
2282 automatic logic [31:0] readback_data_var;
2283 readback_done = decoded_req & ~decoded_req_is_wr;
2284 readback_err = '0;
2285 readback_data_var = '0;
2286 for(int i=0; i<61; i++) readback_data_var |= read-
back_array[i];
2287 readback_data = readback_data_var;
2288 end
2289
2290 assign cpuif_rd_ack = readback_done;
2291 assign cpuif_rd_data = readback_data;
2292 assign cpuif_rd_err = readback_err;
2293
2294 `include "caliptra_prim_assert.sv"
2295 `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
2296
2297 endmodule

```

8. chipsalliance_caliptra-rtl/src/sha512rtl/sha512_reg_pkg.sv

(446 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 package sha512_reg_pkg;
5
6 localparam SHA512_REG_DATA_WIDTH = 32;
7 localparam SHA512_REG_MIN_ADDR_WIDTH = 12;
8
9 typedef struct packed{
10     logic [31:0] next;
11 } sha512_reg__SHA512_NAME__NAME__in_t;
12

```



```

13  typedef struct packed{
14      sha512_reg_SHA512_NAME_NAME_in_t NAME;
15  } sha512_reg_SHA512_NAME_in_t;
16
17  typedef struct packed{
18      logic [31:0] next;
19  } sha512_reg_SHA512_VERSION_VERSION_in_t;
20
21  typedef struct packed{
22      sha512_reg_SHA512_VERSION_VERSION_in_t VERSION;
23  } sha512_reg_SHA512_VERSION_in_t;
24
25  typedef struct packed{
26      logic hwclr;
27  } sha512_reg_SHA512_CTRL_LAST_in_t;
28
29  typedef struct packed{
30      sha512_reg_SHA512_CTRL_LAST_in_t LAST;
31  } sha512_reg_SHA512_CTRL_in_t;
32
33  typedef struct packed{
34      logic next;
35  } sha512_reg_SHA512_STATUS_READY_in_t;
36
37  typedef struct packed{
38      logic next;
39  } sha512_reg_SHA512_STATUS_VALID_in_t;
40
41  typedef struct packed{
42      sha512_reg_SHA512_STATUS_READY_in_t READY;
43      sha512_reg_SHA512_STATUS_VALID_in_t VALID;
44  } sha512_reg_SHA512_STATUS_in_t;
45
46  typedef struct packed{
47      logic [31:0] next;
48      logic we;
49      logic swwe1;
50      logic hwclr;
51  } sha512_reg_SHA512_BLOCK_BLOCK_in_t;
52
53  typedef struct packed{
54      sha512_reg_SHA512_BLOCK_BLOCK_in_t BLOCK;
55  } sha512_reg_SHA512_BLOCK_in_t;
56
57  typedef struct packed{
58      logic [31:0] next;
59      logic we;
60      logic hwclr;
61  } sha512_reg_SHA512_DIGEST_DIGEST_in_t;
62
63  typedef struct packed{
64      sha512_reg_SHA512_DIGEST_DIGEST_in_t DIGEST;
65  } sha512_reg_SHA512_DIGEST_in_t;
66
67  typedef struct packed{
68      logic swwe;
69      logic hwclr;
70  } kv_read_ctrl_reg_read_en_in_t;
71
72  typedef struct packed{
73      logic swwe;
74  } kv_read_ctrl_reg_read_entry_in_t;
75
76  typedef struct packed{
77      logic swwe;
78  } kv_read_ctrl_reg_pcr_hash_extend_in_t;
79
80  typedef struct packed{
81      logic swwe;
82  } kv_read_ctrl_reg_rsvd_in_t;
83
84  typedef struct packed{
85      kv_read_ctrl_reg_read_en_in_t read_en;
86      kv_read_ctrl_reg_read_entry_in_t read_entry;
87      kv_read_ctrl_reg_pcr_hash_extend_in_t
pcr_hash_extend;
88      kv_read_ctrl_reg_rsvd_in_t rsvd;
89  } kv_read_ctrl_reg_in_t;
90
91  typedef struct packed{
92      logic next;
93  } kv_status_reg_READY_in_t;
94
95  typedef struct packed{
96      logic hwclr;
97      logic hwset;
98  } kv_status_reg_VALID_in_t;
99
100  typedef struct packed{
101      logic [7:0] next;
102  } kv_status_reg_ERROR_in_t;
103
104  typedef struct packed{
105      kv_status_reg_READY_in_t READY;
106      kv_status_reg_VALID_in_t VALID;
107      kv_status_reg_ERROR_in_t ERROR;
108  } kv_status_reg_in_t;
109
110  typedef struct packed{
111      logic swwe;
112      logic hwclr;
113  } kv_write_ctrl_reg_write_en_in_t;
114
115  typedef struct packed{
116      logic swwe;
117  } kv_write_ctrl_reg_write_entry_in_t;
118
119  typedef struct packed{
120      logic swwe;
121  } kv_write_ctrl_reg_hmac_key_dest_valid_in_t;
122
123  typedef struct packed{
124      logic swwe;
125  } kv_write_ctrl_reg_hmac_block_dest_valid_in_t;
126
127  typedef struct packed{
128      logic swwe;
129  } kv_write_ctrl_reg_mldsa_seed_dest_valid_in_t;
130
131  typedef struct packed{
132      logic swwe;
133  } kv_write_ctrl_reg_ecc_pkey_dest_valid_in_t;
134
135  typedef struct packed{
136      logic swwe;
137  } kv_write_ctrl_reg_ecc_seed_dest_valid_in_t;
138
139  typedef struct packed{
140      logic swwe;
141  } kv_write_ctrl_reg_aes_key_dest_valid_in_t;
142
143  typedef struct packed{
144      logic swwe;
145  } kv_write_ctrl_reg_mlkem_seed_dest_valid_in_t;
146
147  typedef struct packed{
148      logic swwe;
149  } kv_write_ctrl_reg_mlkem_msg_dest_valid_in_t;
150
151  typedef struct packed{
152      logic swwe;
153  } kv_write_ctrl_reg_dma_data_dest_valid_in_t;
154
155  typedef struct packed{
156      logic swwe;
157  } kv_write_ctrl_reg_rsvd_in_t;
158
159  typedef struct packed{
160      kv_write_ctrl_reg_write_en_in_t write_en;
161      kv_write_ctrl_reg_write_entry_in_t write_entry;
162      kv_write_ctrl_reg_hmac_key_dest_valid_in_t
hmac_key_dest_valid;
163      kv_write_ctrl_reg_hmac_block_dest_valid_in_t hmac_b-
lock_dest_valid;
164      kv_write_ctrl_reg_mldsa_seed_dest_valid_in_t
mldsa_seed_dest_valid;
165      kv_write_ctrl_reg_ecc_pkey_dest_valid_in_t ecc_p-
key_dest_valid;
166      kv_write_ctrl_reg_ecc_seed_dest_valid_in_t
ecc_seed_dest_valid;
167      kv_write_ctrl_reg_aes_key_dest_valid_in_t aes_key_d-
est_valid;
168      kv_write_ctrl_reg_mlkem_seed_dest_valid_in_t
mlkem_seed_dest_valid;
169      kv_write_ctrl_reg_mlkem_msg_dest_valid_in_t
mlkem_msg_dest_valid;
170      kv_write_ctrl_reg_dma_data_dest_valid_in_t
dma_data_dest_valid;
171      kv_write_ctrl_reg_rsvd_in_t rsvd;
172  } kv_write_ctrl_reg_in_t;
173
174  typedef struct packed{
175      logic next;
176  } sha512_reg_SHA512_GEN_PCR_HASH_STATUS_READY_in_t;
177
178  typedef struct packed{
179      logic hwclr;
180      logic hwset;
181  } sha512_reg_SHA512_GEN_PCR_HASH_STATUS_VALID_in_t;
182
183  typedef struct packed{
184      sha512_reg_SHA512_GEN_PCR_HASH_STATUS_READY_in_t
READY;
185      sha512_reg_SHA512_GEN_PCR_HASH_STATUS_VALID_in_t
VALID;
186  } sha512_reg_SHA512_GEN_PCR_HASH_STATUS_in_t;
187
188  typedef struct packed{
189      logic [31:0] next;
190      logic hwclr;
191  } sha512_reg_SHA512_GEN_PCR_HASH_DIGEST_DIGEST_in_t;
192
193  typedef struct packed{
194      sha512_reg_SHA512_GEN_PCR_HASH_DIGEST_DIGEST_in_t
DIGEST;
195  } sha512_reg_SHA512_GEN_PCR_HASH_DIGEST_in_t;
196
197  typedef struct packed{
198      logic hwset;
199  }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error0_sts_enable_528ccada_n

```

```

ext_b1018582_resetsignal_939e99d4__in_t;
200
201     typedef struct packed{
202         logic hwset;
203     }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__error1_sts_enable_938cafef_n
ext_f460eb81_resetsignal_939e99d4__in_t;
204
205     typedef struct packed{
206         logic hwset;
207     }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__error2_sts_enable_0dacf7a6_n
ext_4b5b9e74_resetsignal_939e99d4__in_t;
208
209     typedef struct packed{
210         logic hwset;
211     }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__error3_sts_enable_fc3af94b_n
ext_c3125d40_resetsignal_939e99d4__in_t;
212
213     typedef struct packed{
214
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__error0_sts_enable_528ccada_n
ext_b1018582_resetsignal_939e99d4__in_t error0_sts;
215
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__error1_sts_enable_938cafef_n
ext_f460eb81_resetsignal_939e99d4__in_t error1_sts;
216
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__error2_sts_enable_0dacf7a6_n
ext_4b5b9e74_resetsignal_939e99d4__in_t error2_sts;
217
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__error3_sts_enable_fc3af94b_n
ext_c3125d40_resetsignal_939e99d4__in_t error3_sts;
218
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__in_t;
219
220     typedef struct packed{
221         logic hwset;
222     }
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e__notif_cmd_don
e_sts_enable_dabe0b8b_next_540fa3b7__in_t;
223
224     typedef struct packed{
225
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e__notif_cmd_don
e_sts_enable_dabe0b8b_next_540fa3b7__in_t notif_cmd_done_sts;
226
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e__in_t;
227
228     typedef struct packed{
229
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378__in_t error_internal_intr_r;
230
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e__in_t no-
tif_internal_intr_r;
231
232     } sha512_reg_intr_block_t_in_t;
233
234     typedef struct packed{
235         logic reset_b;
236         logic error_reset_b;
237         logic sha512_ready;
238         sha512_reg_SHA512_NAME__in_t [2-1:0]SHA512_NAME;
239         sha512_reg_SHA512_VERSION__in_t
[2-1:0]SHA512_VERSION;
240         sha512_reg_SHA512_CTRL__in_t SHA512_CTRL;
241         sha512_reg_SHA512_STATUS__in_t SHA512_STATUS;
242         sha512_reg_SHA512_BLOCK__in_t [32-1:0]SHA512_BLOCK;
243         sha512_reg_SHA512_DIGEST__in_t [16-1:0]SHA512_DIGEST;
244         kv_read_ctrl_reg__in_t SHA512_VAULT_RD_CTRL;
245         kv_status_reg__in_t SHA512_VAULT_RD_STATUS;
246         kv_write_ctrl_reg__in_t SHA512_KV_WR_CTRL;
247         kv_status_reg__in_t SHA512_KV_WR_STATUS;
248         sha512_reg_SHA512_GEN_PCR_HASH_STATUS__in_t
SHA512_GEN_PCR_HASH_STATUS;
249         sha512_reg_SHA512_GEN_PCR_HASH_DIGEST__in_t
[16-1:0]SHA512_GEN_PCR_HASH_DIGEST;
250         sha512_reg_intr_block_t_in_t intr_block_rf;
251     } sha512_reg__in_t;
252
253     typedef struct packed{
254         logic value;
255     } sha512_reg_SHA512_CTRL__INIT__out_t;
256
257     typedef struct packed{
258         logic value;
259     } sha512_reg_SHA512_CTRL__NEXT__out_t;
260
261     typedef struct packed{
262         logic [1:0] value;
263     } sha512_reg_SHA512_CTRL__MODE__out_t;
264
265     typedef struct packed{
266         logic value;
267     } sha512_reg_SHA512_CTRL__ZEROIZE__out_t;
268
269     typedef struct packed{
270         logic value;
271     } sha512_reg_SHA512_CTRL__LAST__out_t;
272
273     typedef struct packed{
274         logic value;
275     } sha512_reg_SHA512_CTRL__RESTORE__out_t;
276
277     typedef struct packed{
278         sha512_reg_SHA512_CTRL__INIT__out_t INIT;
279         sha512_reg_SHA512_CTRL__NEXT__out_t NEXT;
280         sha512_reg_SHA512_CTRL__MODE__out_t MODE;
281         sha512_reg_SHA512_CTRL__ZEROIZE__out_t ZEROIZE;
282         sha512_reg_SHA512_CTRL__LAST__out_t LAST;
283         sha512_reg_SHA512_CTRL__RESTORE__out_t RESTORE;
284     } sha512_reg_SHA512_CTRL__out_t;
285
286     typedef struct packed{
287         logic [31:0] value;
288     } sha512_reg_SHA512_BLOCK__BLOCK__out_t;
289
290     typedef struct packed{
291         sha512_reg_SHA512_BLOCK__BLOCK__out_t BLOCK;
292     } sha512_reg_SHA512_BLOCK__out_t;
293
294     typedef struct packed{
295         logic [31:0] value;
296     } sha512_reg_SHA512_DIGEST__DIGEST__out_t;
297
298     typedef struct packed{
299         sha512_reg_SHA512_DIGEST__DIGEST__out_t DIGEST;
300     } sha512_reg_SHA512_DIGEST__out_t;
301
302     typedef struct packed{
303         logic value;
304     } kv_read_ctrl_reg__read_en__out_t;
305
306     typedef struct packed{
307         logic [4:0] value;
308     } kv_read_ctrl_reg__read_entry__out_t;
309
310     typedef struct packed{
311         logic value;
312     } kv_read_ctrl_reg__pcr_hash_extend__out_t;
313
314     typedef struct packed{
315         logic [24:0] value;
316     } kv_read_ctrl_reg__rsvd__out_t;
317
318     typedef struct packed{
319         kv_read_ctrl_reg__read_en__out_t read_en;
320         kv_read_ctrl_reg__read_entry__out_t read_entry;
321         kv_read_ctrl_reg__pcr_hash_extend__out_t
pcr_hash_extend;
322         kv_read_ctrl_reg__rsvd__out_t rsvd;
323     } kv_read_ctrl_reg__out_t;
324
325     typedef struct packed{
326         logic value;
327     } kv_write_ctrl_reg__write_en__out_t;
328
329     typedef struct packed{
330         logic [4:0] value;
331     } kv_write_ctrl_reg__write_entry__out_t;
332
333     typedef struct packed{
334         logic value;
335     } kv_write_ctrl_reg__hmac_key_dest_valid__out_t;
336
337     typedef struct packed{
338         logic value;
339     } kv_write_ctrl_reg__hmac_block_dest_valid__out_t;
340
341     typedef struct packed{
342         logic value;
343     } kv_write_ctrl_reg__mldsa_seed_dest_valid__out_t;
344
345     typedef struct packed{
346         logic value;
347     } kv_write_ctrl_reg__ecc_pkey_dest_valid__out_t;
348
349     typedef struct packed{
350         logic value;
351     } kv_write_ctrl_reg__ecc_seed_dest_valid__out_t;
352
353     typedef struct packed{
354         logic value;
355     } kv_write_ctrl_reg__aes_key_dest_valid__out_t;
356
357     typedef struct packed{
358         logic value;
359     } kv_write_ctrl_reg__mlkem_seed_dest_valid__out_t;
360
361     typedef struct packed{
362         logic value;
363     } kv_write_ctrl_reg__mlkem_msg_dest_valid__out_t;
364
365     typedef struct packed{

```



```

365     logic value;
366 } kv_write_ctrl_reg_dma_data_dest_valid_out_t;
367
368 typedef struct packed{
369     logic [16:0] value;
370 } kv_write_ctrl_reg_rsvd_out_t;
371
372 typedef struct packed{
373     kv_write_ctrl_reg_write_en_out_t write_en;
374     kv_write_ctrl_reg_write_entry_out_t write_entry;
375     kv_write_ctrl_reg_hmac_key_dest_valid_out_t
hmac_key_dest_valid;
376     kv_write_ctrl_reg_hmac_block_dest_valid_out_t
hmac_block_dest_valid;
377     kv_write_ctrl_reg_mldsa_seed_dest_valid_out_t
mldsa_seed_dest_valid;
378     kv_write_ctrl_reg_ecc_pkey_dest_valid_out_t ecc_p-
key_dest_valid;
379     kv_write_ctrl_reg_ecc_seed_dest_valid_out_t
ecc_seed_dest_valid;
380     kv_write_ctrl_reg_aes_key_dest_valid_out_t
aes_key_dest_valid;
381     kv_write_ctrl_reg_mlkem_seed_dest_valid_out_t
mlkem_seed_dest_valid;
382     kv_write_ctrl_reg_mlkem_msg_dest_valid_out_t
mlkem_msg_dest_valid;
383     kv_write_ctrl_reg_dma_data_dest_valid_out_t
dma_data_dest_valid;
384     kv_write_ctrl_reg_rsvd_out_t rsvd;
385 } kv_write_ctrl_reg_out_t;
386
387 typedef struct packed{
388     logic [31:0] value;
389 } sha512_reg_SHA512_GEN_PCR_HASH_NONCE_NONCE_out_t;
390
391 typedef struct packed{
392     sha512_reg_SHA512_GEN_PCR_HASH_NONCE_NONCE_out_t
NONCE;
393 } sha512_reg_SHA512_GEN_PCR_HASH_NONCE_out_t;
394
395 typedef struct packed{
396     logic value;
397 } sha512_reg_SHA512_GEN_PCR_HASH_CTRL_START_out_t;
398
399 typedef struct packed{
400     sha512_reg_SHA512_GEN_PCR_HASH_CTRL_START_out_t
START;
401 } sha512_reg_SHA512_GEN_PCR_HASH_CTRL_out_t;
402
403 typedef struct packed{
404     logic intr;
405 } sha512_reg_global_intr_t_agg_sts_dd3dcf0a_out_t;
406

```

```

407 typedef struct packed{
408     logic intr;
409 } sha512_reg_global_intr_t_agg_sts_e6399b4a_out_t;
410
411 typedef struct packed{
412     logic intr;
413 }
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er-
ror2_sts_b1cf2205_error3_sts_74a35378_out_t;
414
415 typedef struct packed{
416     logic intr;
417 }
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t;
418
419 typedef struct packed{
420     sha512_reg_global_intr_t_agg_sts_dd3dcf0a_out_t er-
ror_global_intr_r;
421     sha512_reg_global_intr_t_agg_sts_e6399b4a_out_t no-
tif_global_intr_r;
422
sha512_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er-
ror2_sts_b1cf2205_error3_sts_74a35378_out_t error_internal_intr_r;
423
sha512_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t no-
tif_internal_intr_r;
424 } sha512_reg_intr_block_t_out_t;
425
426 typedef struct packed{
427     sha512_reg_SHA512_CTRL_out_t SHA512_CTRL;
428     sha512_reg_SHA512_BLOCK_out_t [32-1:0]SHA512_BLOCK;
429     sha512_reg_SHA512_DIGEST_out_t
[16-1:0]SHA512_DIGEST;
430     kv_read_ctrl_reg_out_t SHA512_VAULT_RD_CTRL;
431     kv_write_ctrl_reg_out_t SHA512_KV_WR_CTRL;
432     sha512_reg_SHA512_GEN_PCR_HASH_NONCE_out_t
[8-1:0]SHA512_GEN_PCR_HASH_NONCE;
433     sha512_reg_SHA512_GEN_PCR_HASH_CTRL_out_t
SHA512_GEN_PCR_HASH_CTRL;
434     sha512_reg_intr_block_t_out_t intr_block_rf;
435 } sha512_reg_out_t;
436
437 typedef enum logic [31:0] {
438     kv_status_reg_ERROR_kv_error_e_SUCCESS = 'h0,
439     kv_status_reg_ERROR_kv_error_e_KV_READ_FAIL = 'h1,
440     kv_status_reg_ERROR_kv_error_e_KV_WRITE_FAIL = 'h2,
441     kv_status_reg_ERROR_kv_error_e_KV_FSM_ERROR = 'h4
442 } kv_status_reg_ERROR_kv_error_e_e;
443
444 localparam SHA512_REG_ADDR_WIDTH = 32'd12;
445
446 endpackage

```

Caliptra SHA-256

9 files • 4,409 lines

API — sha256.sv

Table 746. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — module / interface / package (1개)

이름	종류
sha256	module

Table 747. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH		32
DATA_WIDTH		32

Table 748. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — localparam (1개)

상수	형/폭	값
BLOCK_NO		512 / DATA_WIDTH

Table 749. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — 포트 (12개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cs	input	wire
we	input	wire
address	input	wire [ADDR_WIDTH-1 : 0]
write_data	input	wire [DATA_WIDTH-1 : 0]
read_data	output	wire [DATA_WIDTH-1 : 0]
err	output	wire
error_intr	output	wire
notif_intr	output	wire
debugUnlock_or_scan_mode_switch	input	logic

Table 750. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv — typedef / using (1개)

새 이름	원래 형
wntz_fsm_t	enum logic [1:0] {WNTZ_IDLE, WNTZ_1ST, WNTZ_OTHERS}

API — sha256_core.v

Table 751. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v — module / interface / package (1개)

이름	종류
sha256_core	module

Table 752. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v — localparam (20개)

상수	형/폭	값
SHA224_H0_0		32'hc1059ed8
SHA224_H0_1		32'h367cd507
SHA224_H0_2		32'h3070dd17
SHA224_H0_3		32'hf70e5939
SHA224_H0_4		32'hffc00b31
SHA224_H0_5		32'h68581511
SHA224_H0_6		32'h64f98fa7
SHA224_H0_7		32'hbfa4fa4
SHA256_H0_0		32'h6a09e667
SHA256_H0_1		32'hbb67ae85
SHA256_H0_2		32'h3c6ef372
SHA256_H0_3		32'ha54ff53a
SHA256_H0_4		32'h510e527f
SHA256_H0_5		32'h9b05688c
SHA256_H0_6		32'h1f83d9ab
SHA256_H0_7		32'h5be0cd19
SHA256_ROUNDS		63
CTRL_IDLE	[1:0]	0
CTRL_ROUNDS	[1:0]	1
CTRL_DONE	[1:0]	2

Table 753. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v — 포트 (10개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
init_cmd	input	wire
next_cmd	input	wire
mode	input	wire
block_msg	input	wire [511 : 0]
ready	output	wire
digest	output	wire [255 : 0]

포트	방향	형/폭
digest_valid	output	wire

API — sha256_ctrl.sv

Table 754. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.sv — module / interface / package (1개)

이름	종류
sha256_ctrl	module

Table 755. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.sv — parameter (2개)

파라미터	형/폭	기본값
AHB_DATA_WIDTH		32
AHB_ADDR_WIDTH		32

Table 756. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
haddr_i	input	logic [AHB_ADDR_WIDTH-1:0]
hwd_data_i	input	logic [AHB_DATA_WIDTH-1:0]
hsel_i	input	logic
hwrite_i	input	logic
hready_i	input	logic
htrans_i	input	logic [1:0]
hsize_i	input	logic [2:0]
hresp_o	output	logic
hreadyout_o	output	logic
hrdata_o	output	logic [AHB_DATA_WIDTH-1:0]
error_intr	output	logic
notif_intr	output	logic
debugUnlock_or_scan_mode_switch	input	logic

API — sha256_k_constants.v

Table 757. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_k_constants.v — module / interface / package (1개)

이름	종류
sha256_k_constants	module

Table 758. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_k_constants.v — 포트 (2개)

포트	방향	형/폭
round	input	wire [5 : 0]
K_val	output	wire [31 : 0]

API — sha256_params_pkg.sv

Table 759. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_params_pkg.sv — module / interface / package (1개)

이름	종류
sha256_params_pkg	package

Table 760. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_params_pkg.sv — localparam (6개)

상수	형/폭	값
SHA256_CORE_NAME0		32'h61327368
SHA256_CORE_NAME1		32'h35362d32
SHA256_CORE_VERSION0		32'h3830312e
SHA256_CORE_VERSION1		32'h00000000
SHA256_MODE_SHA_224		1'h0
SHA256_MODE_SHA_256		1'h1

API — sha256_reg.sv

Table 761. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg.sv — module / interface / package (1개)

이름	종류
sha256_reg	module

Table 762. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_cpuif_req	input	wire
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [11:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_bitlen	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]

포트	방향	형/폭
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire
hwif_in	input	sha256_reg_pkg::sha256_reg__in_t
hwif_out	output	sha256_reg_pkg::sha256_reg__out_t

API — sha256_reg_pkg.sv

Table 763. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_pkg.sv — module / interface / package (1개)

이름	종류
sha256_reg_pkg	package

Table 764. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_pkg.sv — localparam (3개)

상수	형/폭	값
SHA256_REG_DATA_WIDTH		32
SHA256_REG_MIN_ADDR_WIDTH		12
SHA256_REG_ADDR_WIDTH		32'd12

API — sha256_reg_uvm.sv

Table 765. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_uvm.sv — module / interface / package (1개)

이름	종류
sha256_reg_uvm	package

API — sha256_w_mem.v

Table 766. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_w_mem.v — module / interface / package (1개)

이름	종류
sha256_w_mem	module

Table 767. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_w_mem.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
block_msg	input	wire [511 : 0]
init_cmd	input	wire
next_cmd	input	wire
w_val	output	wire [31 : 0]

1. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256.sv (446 lines)

```

1 //
=====
2 //
3 // Updated by Caliptra team to modify data access width
4 //
5 // sha256.sv
6 // -----
7 // Top level wrapper for the SHA-256 hash function providing
8 // a simple memory like interface with 32 bit data access.
9 //
10 //
11 // Author: Joachim Strombergson
12 // Copyright (c) 2013, 201, Secworks Sweden AB
13 // All rights reserved.
14 //
15 // Redistribution and use in source and binary forms, with or
16 // without modification, are permitted provided that the fol-
17 // lowing conditions are met:
18 //
19 // 1. Redistributions of source code must retain the above
20 // copyright notice, this list of conditions and the following dis-
21 // claimer.
22 // 2. Redistributions in binary form must reproduce the above
23 // copyright notice, this list of conditions and the following dis-
24 // claimer in the documentation and/or other materials provided with
25 // the distribution.
26 //
27 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
28 // TRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
29 // BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
30 // FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
31 // THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
32 // INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
33 // (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
34 // VICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
35 // HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
36 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
37 // WISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
38 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
39 //
40 //
=====
41
42 module sha256
43     import sha256_reg_pkg::*;
44     import sha256_params_pkg::*;
45     #(
46         parameter ADDR_WIDTH = 32,
47         parameter DATA_WIDTH = 32
48     )
49     // Clock and reset.
50     input wire clk,
51     input wire reset_n,
52     input wire cptra_pwrgood,
53
54     // Control.
55     input wire cs,
56     input wire we,
57
58     // Data ports.
59     input wire [ADDR_WIDTH-1 : 0] address,
60     input wire [DATA_WIDTH-1 : 0] write_data,
61     output wire [DATA_WIDTH-1 : 0] read_data,
62     output wire err,
63
64     // Interrupts
65     output wire error_intr,
66     output wire notif_intr,
67     input logic debugUnlock_or_scan_mode_switch
68 );
69
70 //-----
71
72 // Internal constant and parameter definitions.
73 //-----
74 // `include "sha256_param.sv"
75
76 //-----

```

```

77 // Registers including update variables and write enable.
78 //-----
79
80 reg init_reg;
81 reg next_reg;
82 reg mode_reg;
83 reg ready_reg;
84 reg zeroize_reg;
85 logic mode;
86
87 localparam BLOCK_NO = 512 / DATA_WIDTH;
88 reg [DATA_WIDTH-1 : 0] block_reg [BLOCK_NO-1 : 0];
89
90 reg [0 : 7][31 : 0] digest_reg;
91 reg [0 : 7][31 : 0] get_mask;
92 reg digest_valid_reg;
93
94 // Interrupts
95 sha256_reg_in_t hwif_in;
96 sha256_reg_out_t hwif_out;
97 logic read_error, write_error;
98
99 //-----
100 // Wires.
101 //-----
102
103 wire core_ready;
104 logic [511 : 0] core_block;
105 wire [0:7][31:0] core_digest; //Intentionally reverse
106 ordered to prepare block input in wntz mode
107 wire core_digest_valid;
108 logic core_digest_valid_reg;
109
110 logic ready_flag;
111 logic ready_flag_reg;
112 logic valid_flag;
113 logic valid_flag_reg;
114
115 logic wntz_enable;
116 logic wntz_j_inc;
117 logic [7:0] wntz_j_init, wntz_j_reg;
118 logic wntz_busy; // to regiser
119 logic wntz_mode; // from registers
120 logic wntz_n_mode, wntz_n_mode_reg;
121 logic [3:0] wntz_w, wntz_w_reg;
122 logic wntz_w_invalid, wntz_mode_invalid,
123 wntz_j_invalid;
124 logic invalid_sha_op;
125 logic core_init, core_next, core_mode;
126 logic wntz_init;
127 logic wntz_init_reg;
128 logic wntz_blk_done;
129 logic [7:0] wntz_iter, wntz_iter_reg;
130 logic [175 : 0] wntz_prefix;
131
132 typedef enum logic [1:0] {WNTZ_IDLE, WNTZ_1ST, WNTZ_OTHERS}
133 wntz_fsm_t;
134 wntz_fsm_t wntz_fsm, wntz_fsm_next;
135
136 //-----
137 // Concurrent connectivity for ports etc.
138 //-----
139
140 always_comb begin
141     if (wntz_busy) begin
142         core_block = wntz_n_mode_reg ?
143             //SHA256
144             {wntz_prefix, wntz_j_reg,
145             digest_reg[0], digest_reg[1],
146             digest_reg[2], digest_reg[3],
147             digest_reg[4], digest_reg[5],
148             digest_reg[6], digest_reg[7],
149             8'h80, // Padding
150             64'h01b8} // L = 440bits per SHA256
151             padding : //SHA192
152             {wntz_prefix, wntz_j_reg,
153             digest_reg[0], digest_reg[1],
154             digest_reg[2], digest_reg[3],
155             digest_reg[4], digest_reg[5],
156             8'h80, // Padding
157             64'h0178}; // L = 376bits per SHA256
158     end else begin
159         core_block = {block_reg[00], block_reg[01],

```



```

block_reg[02], block_reg[03],
161     block_reg[04], block_reg[05],
block_reg[06], block_reg[07],
162     block_reg[08], block_reg[09],
block_reg[10], block_reg[11],
163     block_reg[12], block_reg[13],
block_reg[14], block_reg[15]];
164     core_init = init_reg;
165     core_next = next_reg;
166     core_mode = mode;
167 end
168 end // always_comb
169
170 assign err = read_error | write_error;
171
172 //-----
173 // core instantiation.
174 //-----
175 sha256_core core(
176     .clk(clk),
177     .reset_n(reset_n),
178     .zeroize(zeroize_reg),
179     .init_cmd(core_init),
180     .next_cmd(core_next),
181     .mode(core_mode),
182     .block_msg(core_block),
183     .ready(core_ready),
184     .digest(core_digest),
185     .digest_valid(core_digest_valid)
186 );
187
188 //-----
189
190 assign wntz_busy      = (wntz_fsm != WNTZ_IDLE);
191 assign wntz_blk_done  = core_digest_valid & ~core_digest_valid_reg;
192 assign wntz_w_invalid = wntz_busy & ~(wntz_w_reg inside
193 { 'h1, 'h2, 'h4, 'h8 });
194 assign wntz_mode_invalid = wntz_busy & ~mode_reg;
195 assign wntz_j_invalid  = wntz_mode & (wntz_j_init >
196 wntz_iter);
197 assign invalid_sha_op  = init_reg & next_reg; //Trigger
198 an error when init and next are high in the same cycle
199
200 always_comb begin
201     unique case(wntz_w)
202     4'h1: wntz_iter = 'd0; //2*w - 1 (-1) (1st iteration is
203 considered separately)
204     4'h2: wntz_iter = 'd2;
205     4'h4: wntz_iter = 'd14;
206     4'h8: wntz_iter = 'd254;
207     default: wntz_iter = 'd0;
208     endcase
209 end
210 assign wntz_j_init = block_reg[5][15:8];
211
212 always_comb begin
213     wntz_init = 1'b0;
214     wntz_enable = 1'b0;
215     wntz_j_inc = 1'b0;
216     case (wntz_fsm)
217     WNTZ_IDLE:
218         begin
219             if (wntz_mode && init_reg && ready_reg &&
220 (wntz_j_init <= wntz_iter)) begin
221                 wntz_fsm_next = WNTZ_1ST;
222                 wntz_enable = 1'b1;
223             end else
224                 wntz_fsm_next = WNTZ_IDLE;
225         end
226     WNTZ_1ST:
227         begin
228             if (wntz_blk_done && (wntz_j_reg < wntz_iter_reg))
229 begin
230                 wntz_fsm_next = WNTZ_OTHERS;
231                 wntz_init = 1'b1;
232                 wntz_j_inc = 1'b1;
233             end else if (wntz_blk_done)
234                 wntz_fsm_next = WNTZ_IDLE;
235             else
236                 wntz_fsm_next = WNTZ_1ST;
237         end
238     WNTZ_OTHERS:
239         begin
240             if (wntz_blk_done && (wntz_j_reg < wntz_iter_reg))
241 begin
242                 wntz_fsm_next = WNTZ_OTHERS;
243                 wntz_init = 1'b1;
244                 wntz_j_inc = 1'b1;
245             end else if (wntz_blk_done)
246                 wntz_fsm_next = WNTZ_IDLE;
247             else
248                 wntz_fsm_next = WNTZ_OTHERS;
249             end
250         default:
251             wntz_fsm_next = WNTZ_IDLE;
252         endcase
253     end
254 end
255
256 always @ (posedge clk or negedge reset_n) begin
257     if (!reset_n)
258         wntz_fsm <= WNTZ_IDLE;
259     else if (zeroize_reg)
260         wntz_fsm <= WNTZ_IDLE;
261     else
262         wntz_fsm <= wntz_fsm_next;
263 end
264
265 always @ (posedge clk or negedge reset_n) begin
266     if (!reset_n) begin
267         wntz_j_reg <= '0;
268         wntz_prefix <= '0;
269         wntz_n_mode_reg <= '0;
270         wntz_w_reg <= '0;
271         wntz_iter_reg <= '0;
272     end else if (zeroize_reg) begin
273         wntz_j_reg <= '0;
274         wntz_prefix <= '0;
275         wntz_n_mode_reg <= '0;
276         wntz_w_reg <= '0;
277         wntz_iter_reg <= '0;
278     end else begin
279         if (wntz_enable) begin
280             wntz_j_reg <= wntz_j_init;
281             wntz_prefix <= {block_reg[00], block_reg[01],
282 block_reg[02], block_reg[03], // I: 16byte, q: 4bytes, i: 2bytes
283 block_reg[04], block_reg[05]
284 [31:16]};
285             wntz_n_mode_reg <= wntz_n_mode;
286             wntz_w_reg <= wntz_w;
287             wntz_iter_reg <= wntz_iter;
288         end else if (wntz_j_inc)
289             wntz_j_reg <= wntz_j_reg + 1;
290     end
291 end
292
293 always @ (posedge clk or negedge reset_n) begin
294     if (!reset_n)
295         wntz_init_reg <= 1'b0;
296     else if (zeroize_reg)
297         wntz_init_reg <= 1'b0;
298     else begin
299         if (wntz_init)
300             wntz_init_reg <= 1'b1;
301         else
302             wntz_init_reg <= 1'b0;
303     end
304 end
305
306 always @ (posedge clk or negedge reset_n) begin
307     if (!reset_n)
308         mode_reg <= '0;
309     else if (zeroize_reg)
310         mode_reg <= '0;
311     else begin
312         if (ready_reg)
313             mode_reg <= mode;
314     end
315 end
316
317 //-----
318
319 // reg_update
320 //
321 // Update functionality for all registers in the core.
322 // All registers are positive edge triggered with asynchronous
323 // active low reset. All registers have write enable.
324 //-----
325
326 always @ (posedge clk or negedge reset_n) begin
327     if (!reset_n) begin
328         get_mask <= {8{32'hffff_ffff}};
329     end
330     else if (wntz_busy) begin
331         unique case (wntz_n_mode_reg)
332         0: get_mask <= {{6{32'hffff_ffff}},
333 {2{32'h0000_0000}}};
334         default: get_mask <= {8{32'hffff_ffff}};
335         endcase
336     end
337     else begin
338         unique case (mode_reg)
339         0: get_mask <= {{7{32'hffff_ffff}},
340 {1{32'h0000_0000}}};
341         default: get_mask <= {8{32'hffff_ffff}};
342         endcase
343     end
344 end

```

```

339     endcase
340   end
341 end
342
343 assign ready_flag = core_ready & ~wntz_busy;
344 assign ready_reg = ready_flag & ready_flag_reg;
345 assign valid_flag = core_digest_valid & ~wntz_busy;
346 assign digest_valid_reg = valid_flag & valid_flag_reg;
347
348 always @ (posedge clk or negedge reset_n)
349   begin : reg_update
350     if (!reset_n) begin
351       ready_flag_reg <= '0;
352       digest_reg <= '0;
353       valid_flag_reg <= '0;
354       core_digest_valid_reg <= '0;
355     end
356     else if (zeroize_reg) begin
357       ready_flag_reg <= '0;
358       digest_reg <= '0;
359       valid_flag_reg <= '0;
360       core_digest_valid_reg <= '0;
361     end
362     else begin
363       ready_flag_reg <= ready_flag;
364       valid_flag_reg <= valid_flag;
365       core_digest_valid_reg <= core_digest_valid;
366
367       if (core_digest_valid & ~digest_valid_reg) begin
368         digest_reg <= core_digest & get_mask;
369       end
370     end
371   end // reg_update
372
373 //register hw interface
374 always_comb begin
375
376   hwif_in.SHA256_NAME[0].NAME.next = SHA256_CORE_NAME0;
377   hwif_in.SHA256_NAME[1].NAME.next = SHA256_CORE_NAME1;
378
379   hwif_in.SHA256_VERSION[0].VERSION.next =
SHA256_CORE_VERSION0;
381   hwif_in.SHA256_VERSION[1].VERSION.next =
SHA256_CORE_VERSION1;
382
383   init_reg = hwif_out.SHA256_CTRL.INIT.value;
384   next_reg = hwif_out.SHA256_CTRL.NEXT.value;
385   mode = hwif_out.SHA256_CTRL.MODE.value;
386   zeroize_reg = hwif_out.SHA256_CTRL.ZEROIZE.value || debu-
gUnlock_or_scan_mode_switch;
387   wntz_mode = hwif_out.SHA256_CTRL.WNTZ_MODE.value;
388   wntz_n_mode = hwif_out.SHA256_CTRL.WNTZ_N_MODE.value;
389   wntz_w = hwif_out.SHA256_CTRL.WNTZ_W.value;
390
391   hwif_in.SHA256_STATUS.READY.next = ready_reg;
392   hwif_in.SHA256_STATUS.VALID.next = digest_valid_reg;
393   hwif_in.SHA256_STATUS.WNTZ_BUSY.next = wntz_busy;
394
395   for (int dword=0; dword < 8; dword++) begin
396     // hwif_in.SHA256_DIGEST[dword].DIGEST.next = di-
gest_reg[7-dword];
397     hwif_in.SHA256_DIGEST[dword].DIGEST.next = di-
gest_reg[dword];
398     hwif_in.SHA256_DIGEST[dword].DIGEST.hwclr = zeroize_reg;
399   end
400
401   for (int dword=0; dword < BLOCK_NO; dword++) begin
402     block_reg[dword] =
hwif_out.SHA256_BLOCK[dword].BLOCK.value;
403     hwif_in.SHA256_BLOCK[dword].BLOCK.hwclr = zeroize_reg;
404   end
405 end
406
407 // Register Block
408 sha256_reg_i_sha256_reg (
409   .clk(clk),
410   .rst(1'b0),
411
412   .s_cpuif_req      (cs),
413   .s_cpuif_req_is_wr (we),
414   .s_cpuif_addr     (ad-
dress[SHA256_REG_ADDR_WIDTH-1:0]),
415   .s_cpuif_wr_data   (write_data),
416   .s_cpuif_wr_biten  ('1),
417   .s_cpuif_req_stall_wr( ),
418   .s_cpuif_req_stall_rd( ),
419   .s_cpuif_rd_ack    ( ),
420   .s_cpuif_rd_err    (read_error),
421   .s_cpuif_rd_data   (read_data),
422   .s_cpuif_wr_ack    ( ),
423   .s_cpuif_wr_err    (write_error),
424
425   .hwif_in (hwif_in ),
426   .hwif_out (hwif_out)
427 );
428
429 //interrupt register hw interface
430 assign hwif_in.sha256_ready = ready_reg;
431 assign hwif_in.reset_b = reset_n;

```

```

433   assign hwif_in.error_reset_b = cptra_pwrgood;
434   assign hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset = (~wntz_busy
& core_digest_valid & ~digest_valid_reg);
435   assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset = wntz_w_invalid |
wntz_mode_invalid | wntz_j_invalid;
436   assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset = invalid_sha_op;
437   assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset = 1'b0;
438   assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset = 1'b0;
439
440   assign error_intr = hwif_out.intr_b-
lock_rf.error_global_intr_r.intr;
441   assign notif_intr = hwif_out.intr_b-
lock_rf.notif_global_intr_r.intr;
442 endmodule // sha256
443
444 //

```

2. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_core.v (582 lines)

```

1 //
=====
2 //
3 // sha256_core.v
4 // -----
5 // Verilog 2001 implementation of the SHA-256 hash function.
6 // This is the internal core with wide interfaces.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
15 // lowing conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
18 // copyright notice, this list of conditions and the following dis-
19 // claimer.
20 // 2. Redistributions in binary form must reproduce the above
21 // copyright notice, this list of conditions and the following dis-
22 // claimer in the documentation and/or other materials provided with
23 // the distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
26 // TRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
27 // BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
28 // FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
29 // THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
30 // INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
31 // (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
32 // VICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
33 // HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
35 // WISE ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
39
40 module sha256_core(
41   // Clock and reset.
42   input wire      clk,
43   input wire      reset_n,
44   input wire      zeroize,
45
46   // Control.
47   input wire      init_cmd,
48   input wire      next_cmd,
49   input wire      mode,
50

```

```

51      // Data ports.
52      input wire [511 : 0] block_msg,
53
54      output wire ready,
55      output wire [255 : 0] digest,
56      output wire digest_valid
57  );
58
59  //-----
60  // Internal constant and parameter definitions.
61  //-----
62
63  localparam SHA224_H0_0 = 32'hc1059ed8;
64  localparam SHA224_H0_1 = 32'h367cd507;
65  localparam SHA224_H0_2 = 32'h3070dd17;
66  localparam SHA224_H0_3 = 32'hf70e5939;
67  localparam SHA224_H0_4 = 32'hffc00b31;
68  localparam SHA224_H0_5 = 32'h68581511;
69  localparam SHA224_H0_6 = 32'h64f98fa7;
70  localparam SHA224_H0_7 = 32'hbfa4fa4;
71
72  localparam SHA256_H0_0 = 32'h6a09e667;
73  localparam SHA256_H0_1 = 32'hbb67ae85;
74  localparam SHA256_H0_2 = 32'h3c6ef372;
75  localparam SHA256_H0_3 = 32'ha54ff53a;
76  localparam SHA256_H0_4 = 32'h510e527f;
77  localparam SHA256_H0_5 = 32'h9b05688c;
78  localparam SHA256_H0_6 = 32'h1f83d9ab;
79  localparam SHA256_H0_7 = 32'h5be0cd19;
80
81  localparam SHA256_ROUNDS = 63;
82
83  localparam [1:0] CTRL_IDLE = 0;
84  localparam [1:0] CTRL_ROUNDS = 1;
85  localparam [1:0] CTRL_DONE = 2;
86
87  //-----
88  // Registers including update variables and write enable.
89  //-----
90
91  reg [31 : 0] a_reg;
92  reg [31 : 0] a_new;
93  reg [31 : 0] b_reg;
94  reg [31 : 0] b_new;
95  reg [31 : 0] c_reg;
96  reg [31 : 0] c_new;
97  reg [31 : 0] d_reg;
98  reg [31 : 0] d_new;
99  reg [31 : 0] e_reg;
100 reg [31 : 0] e_new;
101 reg [31 : 0] f_reg;
102 reg [31 : 0] f_new;
103 reg [31 : 0] g_reg;
104 reg [31 : 0] g_new;
105 reg [31 : 0] h_reg;
106 reg [31 : 0] h_new;
107 reg a_h_we;
108
109 reg [31 : 0] H0_reg;
110 reg [31 : 0] H0_new;
111 reg [31 : 0] H1_reg;
112 reg [31 : 0] H1_new;
113 reg [31 : 0] H2_reg;
114 reg [31 : 0] H2_new;
115 reg [31 : 0] H3_reg;
116 reg [31 : 0] H3_new;
117 reg [31 : 0] H4_reg;
118 reg [31 : 0] H4_new;
119 reg [31 : 0] H5_reg;
120 reg [31 : 0] H5_new;
121 reg [31 : 0] H6_reg;
122 reg [31 : 0] H6_new;
123 reg [31 : 0] H7_reg;
124 reg [31 : 0] H7_new;
125 reg H_we;
126
127 reg [5 : 0] t_ctr_reg;
128 reg [5 : 0] t_ctr_new;
129 reg t_ctr_we;
130 reg t_ctr_inc;
131 reg t_ctr_rst;
132
133 reg digest_valid_reg;
134 reg digest_valid_new;
135 reg digest_valid_we;
136
137 reg [1 : 0] sha256_ctrl_reg;
138 reg [1 : 0] sha256_ctrl_new;
139 reg sha256_ctrl_we;
140
141 //-----
142 // Wires.
143 //-----
144
145 reg digest_init;
146
147 reg digest_update;
148
149 reg state_init;
150 reg state_update;
151
152 reg first_block;
153
154 reg ready_flag;
155
156 reg [31 : 0] t1;
157 reg [31 : 0] t2;
158
159 wire [31 : 0] k_data;
160
161 reg w_init;
162 reg w_next;
163 wire [31 : 0] w_data;
164
165 //-----
166 // Module instantiations.
167 //-----
168 sha256_k_constants k_constants_inst(
169     .round(t_ctr_reg),
170     .k_val(k_data)
171 );
172
173 sha256_w_mem w_mem_inst(
174     .clk(clk),
175     .reset_n(reset_n),
176     .zeroize(zeroize),
177     .block_msg(block_msg),
178     .init_cmd(w_init),
179     .next_cmd(w_next),
180     .w_val(w_data)
181 );
182
183 //-----
184 // Concurrent connectivity for ports etc.
185 //-----
186
187 assign ready = ready_flag;
188
189 assign digest = {H0_reg, H1_reg, H2_reg, H3_reg,
190     H4_reg, H5_reg, H6_reg, H7_reg};
191
192 assign digest_valid = digest_valid_reg;
193
194 //-----
195 // reg_update
196 // Update functionality for all registers in the core.
197 // All registers are positive edge triggered with asynchron-
198 // ous
199 // active low reset. All registers have write enable.
200 //-----
201
202 always @ (posedge clk or negedge reset_n)
203     begin : reg_update
204         if (!reset_n)
205             begin
206                 a_reg <= 32'h0;
207                 b_reg <= 32'h0;
208                 c_reg <= 32'h0;
209                 d_reg <= 32'h0;
210                 e_reg <= 32'h0;
211                 f_reg <= 32'h0;
212                 g_reg <= 32'h0;
213                 h_reg <= 32'h0;
214                 H0_reg <= 32'h0;
215                 H1_reg <= 32'h0;
216                 H2_reg <= 32'h0;
217                 H3_reg <= 32'h0;
218                 H4_reg <= 32'h0;
219                 H5_reg <= 32'h0;
220                 H6_reg <= 32'h0;
221                 H7_reg <= 32'h0;
222                 digest_valid_reg <= 0;
223                 t_ctr_reg <= 6'h0;
224                 sha256_ctrl_reg <= CTRL_IDLE;
225             end
226         else if (zeroize)
227             begin
228                 a_reg <= 32'h0;
229                 b_reg <= 32'h0;
230                 c_reg <= 32'h0;
231                 d_reg <= 32'h0;
232                 e_reg <= 32'h0;
233                 f_reg <= 32'h0;
234                 g_reg <= 32'h0;
235                 h_reg <= 32'h0;
236                 H0_reg <= 32'h0;
237                 H1_reg <= 32'h0;
238                 H2_reg <= 32'h0;
239                 H3_reg <= 32'h0;

```

```

240     H2_reg      <= 32'h0;
241     H3_reg      <= 32'h0;
242     H4_reg      <= 32'h0;
243     H5_reg      <= 32'h0;
244     H6_reg      <= 32'h0;
245     H7_reg      <= 32'h0;
246     digest_valid_reg <= 0;
247     t_ctr_reg    <= 6'h0;
248     sha256_ctrl_reg <= CTRL_IDLE;
249 end
250 else
251 begin
252
253     if (a_h_we)
254     begin
255         a_reg <= a_new;
256         b_reg <= b_new;
257         c_reg <= c_new;
258         d_reg <= d_new;
259         e_reg <= e_new;
260         f_reg <= f_new;
261         g_reg <= g_new;
262         h_reg <= h_new;
263     end
264
265     if (H_we)
266     begin
267         H0_reg <= H0_new;
268         H1_reg <= H1_new;
269         H2_reg <= H2_new;
270         H3_reg <= H3_new;
271         H4_reg <= H4_new;
272         H5_reg <= H5_new;
273         H6_reg <= H6_new;
274         H7_reg <= H7_new;
275     end
276
277     if (t_ctr_we)
278     t_ctr_reg <= t_ctr_new;
279
280     if (digest_valid_we)
281     digest_valid_reg <= digest_valid_new;
282
283     if (sha256_ctrl_reg)
284     sha256_ctrl_reg <= sha256_ctrl_new;
285 end
286 end // reg_update
287
288 //-----
289
290 // digest_logic
291 //
292 // The logic needed to init as well as update the digest.
293 //-----
294
295 always @*
296 begin : digest_logic
297     H0_new = 32'h0;
298     H1_new = 32'h0;
299     H2_new = 32'h0;
300     H3_new = 32'h0;
301     H4_new = 32'h0;
302     H5_new = 32'h0;
303     H6_new = 32'h0;
304     H7_new = 32'h0;
305     H_we = 0;
306
307     if (digest_init)
308     begin
309         H_we = 1;
310         if (mode)
311         begin
312             H0_new = SHA256_H0_0;
313             H1_new = SHA256_H0_1;
314             H2_new = SHA256_H0_2;
315             H3_new = SHA256_H0_3;
316             H4_new = SHA256_H0_4;
317             H5_new = SHA256_H0_5;
318             H6_new = SHA256_H0_6;
319             H7_new = SHA256_H0_7;
320         end
321     else
322     begin
323         H0_new = SHA224_H0_0;
324         H1_new = SHA224_H0_1;
325         H2_new = SHA224_H0_2;
326         H3_new = SHA224_H0_3;
327         H4_new = SHA224_H0_4;
328         H5_new = SHA224_H0_5;
329         H6_new = SHA224_H0_6;
330         H7_new = SHA224_H0_7;
331     end
332 end
333
334 if (digest_update)
335 begin
336     H0_new = H0_reg + a_reg;
337     H1_new = H1_reg + b_reg;
338     H2_new = H2_reg + c_reg;
339     H3_new = H3_reg + d_reg;

```

```

339     H4_new = H4_reg + e_reg;
340     H5_new = H5_reg + f_reg;
341     H6_new = H6_reg + g_reg;
342     H7_new = H7_reg + h_reg;
343     H_we = 1;
344 end
345 end // digest_logic
346
347 //-----
348
349 // t1_logic
350 //
351 // The logic for the T1 function.
352 //-----
353
354 always @*
355 begin : t1_logic
356     reg [31 : 0] sum1;
357     reg [31 : 0] ch;
358
359     sum1 = {e_reg[5 : 0], e_reg[31 : 6]} ^
360           {e_reg[10 : 0], e_reg[31 : 11]} ^
361           {e_reg[24 : 0], e_reg[31 : 25]};
362
363     ch = (e_reg & f_reg) ^ ((~e_reg) & g_reg);
364
365     t1 = h_reg + sum1 + ch + w_data + k_data;
366 end // t1_logic
367
368 //-----
369
370 // t2_logic
371 //
372 // The logic for the T2 function
373 //-----
374
375 always @*
376 begin : t2_logic
377     reg [31 : 0] sum0;
378     reg [31 : 0] maj;
379
380     sum0 = {a_reg[1 : 0], a_reg[31 : 2]} ^
381           {a_reg[12 : 0], a_reg[31 : 13]} ^
382           {a_reg[21 : 0], a_reg[31 : 22]};
383
384     maj = (a_reg & b_reg) ^ (a_reg & c_reg) ^ (b_reg &
385 c_reg);
386
387     t2 = sum0 + maj;
388 end // t2_logic
389
390 //-----
391
392 // state_logicerr
393 //
394 // The logic needed to init as well as update the state during
395 // round processing.
396 //-----
397
398 always @*
399 begin : state_logic
400     a_new = 32'h0;
401     b_new = 32'h0;
402     c_new = 32'h0;
403     d_new = 32'h0;
404     e_new = 32'h0;
405     f_new = 32'h0;
406     g_new = 32'h0;
407     h_new = 32'h0;
408     a_h_we = 0;
409
410     if (state_init)
411     begin
412         a_h_we = 1;
413         if (first_block)
414         begin
415             if (mode)
416             begin
417                 a_new = SHA256_H0_0;
418                 b_new = SHA256_H0_1;
419                 c_new = SHA256_H0_2;
420                 d_new = SHA256_H0_3;
421                 e_new = SHA256_H0_4;
422                 f_new = SHA256_H0_5;
423                 g_new = SHA256_H0_6;
424                 h_new = SHA256_H0_7;
425             end
426         else
427         begin
428             a_new = SHA224_H0_0;
429             b_new = SHA224_H0_1;
430             c_new = SHA224_H0_2;
431             d_new = SHA224_H0_3;
432             e_new = SHA224_H0_4;
433             f_new = SHA224_H0_5;
434             g_new = SHA224_H0_6;
435             h_new = SHA224_H0_7;

```

```

432         end
433     end
434     else
435         begin
436             a_new = H0_reg;
437             b_new = H1_reg;
438             c_new = H2_reg;
439             d_new = H3_reg;
440             e_new = H4_reg;
441             f_new = H5_reg;
442             g_new = H6_reg;
443             h_new = H7_reg;
444         end
445     end
446
447     if (state_update)
448         begin
449             a_new = t1 + t2;
450             b_new = a_reg;
451             c_new = b_reg;
452             d_new = c_reg;
453             e_new = d_reg + t1;
454             f_new = e_reg;
455             g_new = f_reg;
456             h_new = g_reg;
457             a_h_we = 1;
458         end
459     end // state_logic
460
461 //-----
462
463 // t_ctr
464 //
465 // Update logic for the round counter, a monotonically
466 // increasing counter with reset.
467 //-----
468
469 always @*
470 begin : t_ctr
471     t_ctr_new = 0;
472     t_ctr_we = 0;
473
474     if (t_ctr_rst)
475         begin
476             t_ctr_new = 0;
477             t_ctr_we = 1;
478         end
479
480     if (t_ctr_inc)
481         begin
482             t_ctr_new = t_ctr_reg + 1'b1;
483             t_ctr_we = 1;
484         end
485     end
486 end // t_ctr
487
488 //-----
489
490 // sha256_ctrl_fsm
491 //
492 // Logic for the state machine controlling the core beha-
493 // viour.
494 //-----
495
496 always @*
497 begin : sha256_ctrl_fsm
498     digest_init = 0;
499     digest_update = 0;
500
501     state_init = 0;
502     state_update = 0;
503
504     first_block = 0;
505     ready_flag = 0;
506
507     w_init = 0;
508     w_next = 0;
509
510     t_ctr_inc = 0;
511     t_ctr_rst = 0;
512
513     digest_valid_new = 0;
514     digest_valid_we = 0;
515
516     sha256_ctrl_new = CTRL_IDLE;
517     sha256_ctrl_we = 0;
518
519     case (sha256_ctrl_reg)
520     CTRL_IDLE:
521         begin
522             ready_flag = 1;
523
524             if (init_cmd)
525                 begin
526                     digest_init = 1;
527                     w_init = 1;
528                     state_init = 1;
529                     first_block = 1;
530                     t_ctr_rst = 1;

```

```

528         digest_valid_new = 0;
529         digest_valid_we = 1;
530         sha256_ctrl_new = CTRL_ROUNDS;
531         sha256_ctrl_we = 1;
532     end
533
534     if (next_cmd)
535         begin
536             t_ctr_rst = 1;
537             w_init = 1;
538             state_init = 1;
539             digest_valid_new = 0;
540             digest_valid_we = 1;
541             sha256_ctrl_new = CTRL_ROUNDS;
542             sha256_ctrl_we = 1;
543         end
544     end
545
546     CTRL_ROUNDS:
547         begin
548             w_next = 1;
549             state_update = 1;
550             t_ctr_inc = 1;
551
552             if (t_ctr_reg == SHA256_ROUNDS)
553                 begin
554                     sha256_ctrl_new = CTRL_DONE;
555                     sha256_ctrl_we = 1;
556                 end
557             end
558         end
559
560     CTRL_DONE:
561         begin
562             digest_update = 1;
563             digest_valid_new = 1;
564             digest_valid_we = 1;
565
566             sha256_ctrl_new = CTRL_IDLE;
567             sha256_ctrl_we = 1;
568         end
569     default:
570         begin
571             sha256_ctrl_new = sha256_ctrl_reg;
572             sha256_ctrl_we = 0;
573         end
574     endcase // case (sha256_ctrl_reg)
575 end // sha256_ctrl_fsm
576
577 endmodule // sha256_core
578
579
580 //

```

```

=====
581 // EOF sha256_core.v
582 //
=====

```

3. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_ctrl.sv (123 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 // http://www.apache.org/licenses/LICENSE-2.0
9 // Unless required by applicable law or agreed to in writing,
10 // software distributed under the License is distributed on an "AS IS"
11 // BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
13 // press or implied.
14 // See the License for the specific language governing permis-
15 // sions and
16 // limitations under the License.
17 //
18 //-----
19 // SHA256 controller for the AHB_lite interface.
20 //
21 //
22 //
23
24 module sha256_ctrl #(
25     parameter AHB_DATA_WIDTH = 32,
26     parameter AHB_ADDR_WIDTH = 32
27 )

```

```

28 (
29 // Clock and reset.
30 input wire      clk,
31 input wire      reset_n,
32 input wire      cptra_pwrgood,
33
34 // from SLAVES PORT
35 input logic [AHB_ADDR_WIDTH-1:0] haddr_i,
36 input logic [AHB_DATA_WIDTH-1:0] hwdata_i,
37 input logic hsel_i,
38 input logic hwrite_i,
39 input logic hready_i,
40 input logic [1:0] htrans_i,
41 input logic [2:0] hsize_i,
42
43 output logic hresp_o,
44 output logic hreadyout_o,
45 output logic [AHB_DATA_WIDTH-1:0] hrdata_o,
46
47 // Interrupt
48 output logic error_intr,
49 output logic notif_intr,
50 input logic debugUnlock_or_scan_mode_switch
51 );
52
53 //-----
54 // sha256
55 //-----
56 logic      sha256_cs;
57 logic      sha256_we;
58 logic [AHB_ADDR_WIDTH-1 : 0] sha256_address;
59 logic [31 : 0] sha256_write_data;
60 logic [31 : 0] sha256_read_data;
61 logic      sha256_err;
62
63 sha256 #(
64     .ADDR_WIDTH(AHB_ADDR_WIDTH),
65     .DATA_WIDTH(32)
66 ) sha256_inst(
67     .clk(clk),
68     .reset_n(reset_n),
69     .cptra_pwrgood(cptra_pwrgood),
70     .cs(sha256_cs),
71     .we(sha256_we),
72     .address(sha256_address),
73     .write_data(sha256_write_data),
74     .read_data(sha256_read_data),
75     .err(sha256_err),
76     .error_intr(error_intr),
77     .notif_intr(notif_intr),
78     .debugUn-
lock_or_scan_mode_switch(debugUnlock_or_scan_mode_switch)
79 );
80
81
82 //-----
83 // AHB Slave node
84 //-----
85
86 //instantiate ahb lite module
87 ahb_slv_sif #(
88     .AHB_ADDR_WIDTH(AHB_ADDR_WIDTH),
89     .AHB_DATA_WIDTH(AHB_DATA_WIDTH),
90     .CLIENT_DATA_WIDTH(32)
91 ) ahb_slv_sif_uut
92 (
93     //AMBA AHB Lite INF
94     .hclk(clk),
95     .hreset_n(reset_n),
96     .haddr_i(haddr_i),
97     .hwdata_i(hwdata_i),
98     .hsel_i(hsel_i),
99     .hwrite_i(hwrite_i),
100    .hready_i(hready_i),
101    .htrans_i(htrans_i),
102    .hsize_i(hsize_i),
103
104    .hresp_o(hresp_o),
105    .hreadyout_o(hreadyout_o),
106    .hrdata_o(hrdata_o),
107
108    //COMPONENT INF
109    .dv(sha256_cs),
110    .hld(1'b0), //no holds from sha256
111    .err(1'b0),
112    .write(sha256_we),
113    .wdata(sha256_write_data),
114    .addr(sha256_address),
115
116    .rdata(sha256_read_data)
117 );
118
119 endmodule //sha256_ctrl
120
121 //
=====
==

```

```
122 // EOF sha256_ctrl.sv
```

```
123 //
```

```

=====
4. chipsalliance_caliptra-rtl/src/sha256/rtl/
sha256_k_constants.v (132 lines)
1 //
=====
2 //
3 // sha256_k_constants.v
4 // -----
5 // The table K with constants in the SHA-256 hash function.
6 //
7 //
8 // Author: Joachim Strombergson
9 // Copyright (c) 2013, Secworks Sweden AB
10 // All rights reserved.
11 //
12 // Redistribution and use in source and binary forms, with or
13 // without modification, are permitted provided that the fol-
lowing
14 // conditions are met:
15 //
16 // 1. Redistributions of source code must retain the above
copyright
17 // notice, this list of conditions and the following dis-
claimer.
18 //
19 // 2. Redistributions in binary form must reproduce the above
copyright
20 // notice, this list of conditions and the following dis-
claimer in
21 // the documentation and/or other materials provided with
the
22 // distribution.
23 //
24 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
25 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
26 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
27 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL
THE
28 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
29 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
30 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
31 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
32 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
33 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
34 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
35 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
36 //
37 //
=====
38
39 module sha256_k_constants(
40     input wire [5 : 0] round,
41     output wire [31 : 0] K_val
42 );
43
44 //-----
45 // Wires.
46 //-----
47 reg [31 : 0] tmp_K;
48
49
50 //-----
51 // Concurrent connectivity for ports etc.
52 //-----
53 assign K_val = tmp_K;
54
55
56 //-----
57 // round_mux
58 //-----
59 always @*
60 begin : round_mux
61     case(round)
62         00: tmp_K = 32'h428a2f98;
63         01: tmp_K = 32'h71374491;
64         02: tmp_K = 32'hb5c0fbcf;
65         03: tmp_K = 32'he9b5dba5;
66         04: tmp_K = 32'h3956c25b;
67         05: tmp_K = 32'h59f111f1;

```



```

68      06: tmp_K = 32'h923f82a4;
69      07: tmp_K = 32'habc5ed5;
70      08: tmp_K = 32'hd807aa98;
71      09: tmp_K = 32'h12835b01;
72      10: tmp_K = 32'h243185be;
73      11: tmp_K = 32'h550c7dc3;
74      12: tmp_K = 32'h72be5d74;
75      13: tmp_K = 32'h80deb1fe;
76      14: tmp_K = 32'h9bdc06a7;
77      15: tmp_K = 32'hc19bf174;
78      16: tmp_K = 32'he49b69c1;
79      17: tmp_K = 32'hef6e4786;
80      18: tmp_K = 32'h0fc19dc6;
81      19: tmp_K = 32'h240ca1cc;
82      20: tmp_K = 32'h2de92c6f;
83      21: tmp_K = 32'h4a7484aa;
84      22: tmp_K = 32'h5cb0a9dc;
85      23: tmp_K = 32'h76f988da;
86      24: tmp_K = 32'h983e5152;
87      25: tmp_K = 32'ha831c66d;
88      26: tmp_K = 32'hb00327c8;
89      27: tmp_K = 32'hbf597fc7;
90      28: tmp_K = 32'hc6e00bf3;
91      29: tmp_K = 32'hd5a79147;
92      30: tmp_K = 32'h06ca6351;
93      31: tmp_K = 32'h14292967;
94      32: tmp_K = 32'h27b70a85;
95      33: tmp_K = 32'h2e1b2138;
96      34: tmp_K = 32'h4d2c6dfc;
97      35: tmp_K = 32'h53380d13;
98      36: tmp_K = 32'h650a7354;
99      37: tmp_K = 32'h766a0abb;
100     38: tmp_K = 32'h81c2c92e;
101     39: tmp_K = 32'h92722c85;
102     40: tmp_K = 32'ha2bfe8a1;
103     41: tmp_K = 32'ha81a664b;
104     42: tmp_K = 32'hc24b8b70;
105     43: tmp_K = 32'hc76c51a3;
106     44: tmp_K = 32'hd192e819;
107     45: tmp_K = 32'hd6990624;
108     46: tmp_K = 32'hf40e3585;
109     47: tmp_K = 32'h106aa070;
110     48: tmp_K = 32'h19a4c116;
111     49: tmp_K = 32'h1e376c08;
112     50: tmp_K = 32'h2748774c;
113     51: tmp_K = 32'h34b0bcb5;
114     52: tmp_K = 32'h391c0cb3;
115     53: tmp_K = 32'h4ed8aa4a;
116     54: tmp_K = 32'h5b9cca4f;
117     55: tmp_K = 32'h682e6ff3;
118     56: tmp_K = 32'h748f82ee;
119     57: tmp_K = 32'h78a5636f;
120     58: tmp_K = 32'h84c87814;
121     59: tmp_K = 32'h8cc70208;
122     60: tmp_K = 32'h90bafffa;
123     61: tmp_K = 32'ha4506ceb;
124     62: tmp_K = 32'hbef9a3f7;
125     63: tmp_K = 32'hc67178f2;
126     endcase // case (round)
127     end // block: round_mux
128 endmodule // sha256_k_constants
129 //
130 //
=====
131 // sha256_k_constants.v
132 //
=====

```

5. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_params_pkg.sv (30 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 //   cense");
5 // you may not use this file except in compliance with the Li-
6 //   cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0
10 // Unless required by applicable law or agreed to in writing,
11 // software
12 // distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 //   press or implied.
16 // See the License for the specific language governing permis-
17 //   sions and
18 //   limitations under the License.
19 //
20 package sha256_params_pkg;
21
22 localparam SHA256_CORE_NAME0 = 32'h61327368; //
23
24 "sha2"
25 localparam SHA256_CORE_NAME1 = 32'h3536d232; //

```

```

"-256"
21 localparam SHA256_CORE_VERSION0 = 32'h3830312e; //
"1.80"
22 localparam SHA256_CORE_VERSION1 = 32'h00000000; // "0"
23
24 localparam SHA256_MODE_SHA_224 = 1'h0;
25 localparam SHA256_MODE_SHA_256 = 1'h1;
26
27 endpackage
28 //
=====
29 // EOF sha256_params_pkg.sv
30 //
=====

```

6. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg.sv (1,597 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
2 //   temVerilog generator
3 // https://github.com/SystemRDL/PeakRDL-regblock
4
5 module sha256_reg (
6     input wire clk,
7     input wire rst,
8
9     input wire s_cpuif_req,
10    input wire s_cpuif_req_is_wr,
11    input wire [11:0] s_cpuif_addr,
12    input wire [31:0] s_cpuif_wr_data,
13    input wire [31:0] s_cpuif_wr_biten,
14    output wire s_cpuif_req_stall_wr,
15    output wire s_cpuif_req_stall_rd,
16    output wire s_cpuif_rd_ack,
17    output wire s_cpuif_rd_err,
18    output wire [31:0] s_cpuif_rd_data,
19    output wire s_cpuif_wr_ack,
20    output wire s_cpuif_wr_err,
21
22    input sha256_reg_pkg::sha256_reg_in_t hwif_in,
23    output sha256_reg_pkg::sha256_reg_out_t hwif_out
24);
25
26 //-----
27 // CPU Bus interface logic
28 //-----
29
30 logic cpuif_req;
31 logic cpuif_req_is_wr;
32 logic [11:0] cpuif_addr;
33 logic [31:0] cpuif_wr_data;
34 logic [31:0] cpuif_wr_biten;
35 logic cpuif_req_stall_wr;
36 logic cpuif_req_stall_rd;
37
38 logic cpuif_rd_ack;
39 logic cpuif_rd_err;
40 logic [31:0] cpuif_rd_data;
41
42 logic cpuif_wr_ack;
43 logic cpuif_wr_err;
44
45 assign cpuif_req = s_cpuif_req;
46 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
47 assign cpuif_addr = s_cpuif_addr;
48 assign cpuif_wr_data = s_cpuif_wr_data;
49 assign cpuif_wr_biten = s_cpuif_wr_biten;
50 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
51 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
52 assign s_cpuif_rd_ack = cpuif_rd_ack;
53 assign s_cpuif_rd_err = cpuif_rd_err;
54 assign s_cpuif_rd_data = cpuif_rd_data;
55 assign s_cpuif_wr_ack = cpuif_wr_ack;
56 assign s_cpuif_wr_err = cpuif_wr_err;
57
58 logic cpuif_req_masked;
59
60 // Read & write latencies are balanced. Stalls not re-
61 //   quired
62 assign cpuif_req_stall_rd = '0;
63 assign cpuif_req_stall_wr = '0;
64 assign cpuif_req_masked = cpuif_req
65    & !(cpuif_req_is_wr &
66    & !(cpuif_req_stall_wr));
67
68 //-----
69 // Address Decode
70 //-----
71
72 typedef struct packed{
73     logic [2-1:0]SHA256_NAME;
74     logic [2-1:0]SHA256_VERSION;
75     logic SHA256_CTRL;
76     logic SHA256_STATUS;
77 }

```



```

73     logic [16-1:0]SHA256_BLOCK;
74     logic [8-1:0]SHA256_DIGEST;
75     struct packed{
76         logic global_intr_en_r;
77         logic error_intr_en_r;
78         logic notif_intr_en_r;
79         logic error_global_intr_r;
80         logic notif_global_intr_r;
81         logic error_internal_intr_r;
82         logic notif_internal_intr_r;
83         logic error_intr_trig_r;
84         logic notif_intr_trig_r;
85         logic error0_intr_count_r;
86         logic error1_intr_count_r;
87         logic error2_intr_count_r;
88         logic error3_intr_count_r;
89         logic notif_cmd_done_intr_count_r;
90         logic error0_intr_count_incr_r;
91         logic error1_intr_count_incr_r;
92         logic error2_intr_count_incr_r;
93         logic error3_intr_count_incr_r;
94         logic notif_cmd_done_intr_count_incr_r;
95     } intr_block_rf;
96 } decoded_reg_strb_t;
97 decoded_reg_strb_t decoded_reg_strb;
98 logic decoded_req;
99 logic decoded_req_is_wr;
100 logic [31:0] decoded_wr_data;
101 logic [31:0] decoded_wr_bitlen;
102
103 always_comb begin
104     for(int i0=0; i0<2; i0++) begin
105         decoded_reg_strb.SHA256_NAME[i0] =
cpuif_req_masked & (cpuif_addr == 12'h0 + i0*12'h4);
106     end
107     for(int i0=0; i0<2; i0++) begin
108         decoded_reg_strb.SHA256_VERSION[i0] =
cpuif_req_masked & (cpuif_addr == 12'h8 + i0*12'h4);
109     end
110     decoded_reg_strb.SHA256_CTRL = cpuif_req_masked &
(cpuif_addr == 12'h10);
111     decoded_reg_strb.SHA256_STATUS = cpuif_req_masked &
(cpuif_addr == 12'h18);
112     for(int i0=0; i0<16; i0++) begin
113         decoded_reg_strb.SHA256_BLOCK[i0] =
cpuif_req_masked & (cpuif_addr == 12'h80 + i0*12'h4);
114     end
115     for(int i0=0; i0<8; i0++) begin
116         decoded_reg_strb.SHA256_DIGEST[i0] =
cpuif_req_masked & (cpuif_addr == 12'h100 + i0*12'h4);
117     end
118     decoded_reg_strb.intr_block_rf.global_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h800);
119     decoded_reg_strb.intr_block_rf.error_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h804);
120     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h808);
121     decoded_reg_strb.intr_block_rf.error_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h80c);
122     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h810);
123     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h814);
124     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h818);
125     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h81c);
126     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h820);
127     decoded_reg_strb.intr_block_rf.error0_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h900);
128     decoded_reg_strb.intr_block_rf.error1_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h904);
129     decoded_reg_strb.intr_block_rf.error2_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h908);
130     decoded_reg_strb.intr_block_rf.error3_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h90c);
131     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h980);
132     de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha00);
133     de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha04);
134     de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha08);
135     de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha0c);
136     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha10);
137     end
138
139 // Pass down signals to next stage
140 assign decoded_req = cpuif_req_masked;

```

```

141 assign decoded_req_is_wr = cpuif_req_is_wr;
142 assign decoded_wr_data = cpuif_wr_data;
143 assign decoded_wr_bitlen = cpuif_wr_bitlen;
144
145 //-----
146 // Field logic
147 //-----
148 typedef struct packed{
149     struct packed{
150         struct packed{
151             logic next;
152             logic load_next;
153         } INIT;
154         struct packed{
155             logic next;
156             logic load_next;
157         } NEXT;
158         struct packed{
159             logic next;
160             logic load_next;
161         } MODE;
162         struct packed{
163             logic next;
164             logic load_next;
165         } ZEROIZE;
166         struct packed{
167             logic next;
168             logic load_next;
169         } WNTZ_MODE;
170         struct packed{
171             logic [3:0] next;
172             logic load_next;
173         } WNTZ_W;
174         struct packed{
175             logic next;
176             logic load_next;
177         } WNTZ_N_MODE;
178     } SHA256_CTRL;
179     struct packed{
180         struct packed{
181             logic [31:0] next;
182             logic load_next;
183         } BLOCK;
184     } [16-1:0]SHA256_BLOCK;
185     struct packed{
186         struct packed{
187             logic [31:0] next;
188             logic load_next;
189         } DIGEST;
190     } [8-1:0]SHA256_DIGEST;
191     struct packed{
192         struct packed{
193             struct packed{
194                 logic next;
195                 logic load_next;
196             } error_en;
197             struct packed{
198                 logic next;
199                 logic load_next;
200             } notif_en;
201         } global_intr_en_r;
202         struct packed{
203             struct packed{
204                 logic next;
205                 logic load_next;
206             } error0_en;
207             struct packed{
208                 logic next;
209                 logic load_next;
210             } error1_en;
211             struct packed{
212                 logic next;
213                 logic load_next;
214             } error2_en;
215             struct packed{
216                 logic next;
217                 logic load_next;
218             } error3_en;
219         } error_intr_en_r;
220         struct packed{
221             struct packed{
222                 logic next;
223                 logic load_next;
224             } notif_cmd_done_en;
225             struct packed{
226                 struct packed{
227                     logic next;
228                     logic load_next;
229                 } agg_sts;
230             } error_global_intr_r;
231         } struct packed{
232             struct packed{
233                 logic next;
234                 logic load_next;
235             } agg_sts;
236         } notif_global_intr_r;
237     } struct packed{
238         struct packed{
239             struct packed{

```

```

240         logic next;
241         logic load_next;
242     } error0_sts;
243     struct packed{
244         logic next;
245         logic load_next;
246     } error1_sts;
247     struct packed{
248         logic next;
249         logic load_next;
250     } error2_sts;
251     struct packed{
252         logic next;
253         logic load_next;
254     } error3_sts;
255 } error_internal_intr_r;
256 struct packed{
257     struct packed{
258         logic next;
259         logic load_next;
260     } notif_cmd_done_sts;
261 } notif_internal_intr_r;
262 struct packed{
263     struct packed{
264         logic next;
265         logic load_next;
266     } error0_trig;
267     struct packed{
268         logic next;
269         logic load_next;
270     } error1_trig;
271     struct packed{
272         logic next;
273         logic load_next;
274     } error2_trig;
275     struct packed{
276         logic next;
277         logic load_next;
278     } error3_trig;
279 } error_intr_trig_r;
280 struct packed{
281     struct packed{
282         logic next;
283         logic load_next;
284     } notif_cmd_done_trig;
285 } notif_intr_trig_r;
286 struct packed{
287     struct packed{
288         logic [31:0] next;
289         logic load_next;
290         logic incrtthreshold;
291         logic incrsaturate;
292     } cnt;
293 } error0_intr_count_r;
294 struct packed{
295     struct packed{
296         logic [31:0] next;
297         logic load_next;
298         logic incrtthreshold;
299         logic incrsaturate;
300     } cnt;
301 } error1_intr_count_r;
302 struct packed{
303     struct packed{
304         logic [31:0] next;
305         logic load_next;
306         logic incrtthreshold;
307         logic incrsaturate;
308     } cnt;
309 } error2_intr_count_r;
310 struct packed{
311     struct packed{
312         logic [31:0] next;
313         logic load_next;
314         logic incrtthreshold;
315         logic incrsaturate;
316     } cnt;
317 } error3_intr_count_r;
318 struct packed{
319     struct packed{
320         logic [31:0] next;
321         logic load_next;
322         logic incrtthreshold;
323         logic incrsaturate;
324     } cnt;
325 } notif_cmd_done_intr_count_r;
326 struct packed{
327     struct packed{
328         logic next;
329         logic load_next;
330         logic decrthreshold;
331         logic underflow;
332     } pulse;
333 } error0_intr_count_incr_r;
334 struct packed{
335     struct packed{
336         logic next;
337         logic load_next;
338         logic decrthreshold;
339         logic underflow;
340     } pulse;
341 } error1_intr_count_incr_r;
342 struct packed{
343     struct packed{
344         logic next;
345         logic load_next;
346         logic decrthreshold;
347         logic underflow;
348     } pulse;
349 } error2_intr_count_incr_r;
350 struct packed{
351     struct packed{
352         logic next;
353         logic load_next;
354         logic decrthreshold;
355         logic underflow;
356     } pulse;
357 } error3_intr_count_incr_r;
358 struct packed{
359     struct packed{
360         logic next;
361         logic load_next;
362         logic decrthreshold;
363         logic underflow;
364     } pulse;
365 } notif_cmd_done_intr_count_incr_r;
366 } intr_block_rf;
367 } field_combo_t;
368 field_combo_t field_combo;
369
370 typedef struct packed{
371     struct packed{
372         struct packed{
373             logic value;
374         } INIT;
375         struct packed{
376             logic value;
377         } NEXT;
378         struct packed{
379             logic value;
380         } MODE;
381         struct packed{
382             logic value;
383         } ZEROIZE;
384         struct packed{
385             logic value;
386         } WNTZ_MODE;
387         struct packed{
388             logic [3:0] value;
389         } WNTZ_W;
390         struct packed{
391             logic value;
392         } WNTZ_N_MODE;
393     } SHA256_CTRL;
394     struct packed{
395         struct packed{
396             logic [31:0] value;
397         } BLOCK;
398     } [16-1:0]SHA256_BLOCK;
399     struct packed{
400         struct packed{
401             logic [31:0] value;
402         } DIGEST;
403     } [8-1:0]SHA256_DIGEST;
404     struct packed{
405         struct packed{
406             logic value;
407         } error_en;
408         struct packed{
409             logic value;
410         } notif_en;
411     } global_intr_en_r;
412     struct packed{
413         struct packed{
414             logic value;
415         } error0_en;
416         struct packed{
417             logic value;
418         } error1_en;
419         struct packed{
420             logic value;
421         } error2_en;
422         struct packed{
423             logic value;
424         } error3_en;
425     } error_intr_en_r;
426     struct packed{
427         struct packed{
428             logic value;
429         } notif_cmd_done_en;
430     } notif_intr_en_r;
431     struct packed{
432         struct packed{
433             logic value;
434         } agg_sts;
435     } error_global_intr_r;
436     struct packed{
437         struct packed{
438             logic value;
439         } agg_sts;
440     } notif_global_intr_r;
441 }

```

```

442 struct packed{
443     struct packed{
444         logic value;
445     } error0_sts;
446     struct packed{
447         logic value;
448     } error1_sts;
449     struct packed{
450         logic value;
451     } error2_sts;
452     struct packed{
453         logic value;
454     } error3_sts;
455 } error_internal_intr_r;
456 struct packed{
457     struct packed{
458         logic value;
459     } notif_cmd_done_sts;
460 } notif_internal_intr_r;
461 struct packed{
462     struct packed{
463         logic value;
464     } error0_trig;
465     struct packed{
466         logic value;
467     } error1_trig;
468     struct packed{
469         logic value;
470     } error2_trig;
471     struct packed{
472         logic value;
473     } error3_trig;
474 } error_intr_trig_r;
475 struct packed{
476     struct packed{
477         logic value;
478     } notif_cmd_done_trig;
479 } notif_intr_trig_r;
480 struct packed{
481     struct packed{
482         logic [31:0] value;
483     } cnt;
484 } error0_intr_count_r;
485 struct packed{
486     struct packed{
487         logic [31:0] value;
488     } cnt;
489 } error1_intr_count_r;
490 struct packed{
491     struct packed{
492         logic [31:0] value;
493     } cnt;
494 } error2_intr_count_r;
495 struct packed{
496     struct packed{
497         logic [31:0] value;
498     } cnt;
499 } error3_intr_count_r;
500 struct packed{
501     struct packed{
502         logic [31:0] value;
503     } cnt;
504 } notif_cmd_done_intr_count_r;
505 struct packed{
506     struct packed{
507         logic value;
508     } pulse;
509 } error0_intr_count_incr_r;
510 struct packed{
511     struct packed{
512         logic value;
513     } pulse;
514 } error1_intr_count_incr_r;
515 struct packed{
516     struct packed{
517         logic value;
518     } pulse;
519 } error2_intr_count_incr_r;
520 struct packed{
521     struct packed{
522         logic value;
523     } pulse;
524 } error3_intr_count_incr_r;
525 struct packed{
526     struct packed{
527         logic value;
528     } pulse;
529 } notif_cmd_done_intr_count_incr_r;
530 } intr_block_rf;
531 } field_storage_t;
532 field_storage_t field_storage;
533
534 // Field: sha256_reg.SHA256_CTRL.INIT
535 always_comb begin
536     automatic logic [0:0] next_c;
537     automatic logic load_next_c;
538     next_c = field_storage.SHA256_CTRL.INIT.value;
539     load_next_c = '0;
540     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
541         next_c = (field_storage.SHA256_CTRL.INIT.value &

```

```

~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
542         load_next_c = '1;
543     end else begin // singlepulse clears back to 0
544         next_c = '0;
545         load_next_c = '1;
546     end
547     field_combo.SHA256_CTRL.INIT.next = next_c;
548     field_combo.SHA256_CTRL.INIT.load_next = load_next_c;
549 end
550 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
551     if(~hwif_in.reset_b) begin
552         field_storage.SHA256_CTRL.INIT.value <= 1'h0;
553     end else if(field_combo.SHA256_CTRL.INIT.load_next)
begin
554         field_storage.SHA256_CTRL.INIT.value <=
field_combo.SHA256_CTRL.INIT.next;
555     end
556 end
557 assign hwif_out.SHA256_CTRL.INIT.value =
field_storage.SHA256_CTRL.INIT.value;
558 // Field: sha256_reg.SHA256_CTRL.NEXT
559 always_comb begin
560     automatic logic [0:0] next_c;
561     automatic logic load_next_c;
562     next_c = field_storage.SHA256_CTRL.NEXT.value;
563     load_next_c = '0;
564     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
565         next_c = (field_storage.SHA256_CTRL.NEXT.value &
~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
566         load_next_c = '1;
567     end else begin // singlepulse clears back to 0
568         next_c = '0;
569         load_next_c = '1;
570     end
571     field_combo.SHA256_CTRL.NEXT.next = next_c;
572     field_combo.SHA256_CTRL.NEXT.load_next = load_next_c;
573 end
574 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
575     if(~hwif_in.reset_b) begin
576         field_storage.SHA256_CTRL.NEXT.value <= 1'h0;
577     end else if(field_combo.SHA256_CTRL.NEXT.load_next)
begin
578         field_storage.SHA256_CTRL.NEXT.value <=
field_combo.SHA256_CTRL.NEXT.next;
579     end
580 end
581 assign hwif_out.SHA256_CTRL.NEXT.value =
field_storage.SHA256_CTRL.NEXT.value;
582 // Field: sha256_reg.SHA256_CTRL.MODE
583 always_comb begin
584     automatic logic [0:0] next_c;
585     automatic logic load_next_c;
586     next_c = field_storage.SHA256_CTRL.MODE.value;
587     load_next_c = '0;
588     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
589         next_c = (field_storage.SHA256_CTRL.MODE.value &
~decoded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
590         load_next_c = '1;
591     end
592     field_combo.SHA256_CTRL.MODE.next = next_c;
593     field_combo.SHA256_CTRL.MODE.load_next = load_next_c;
594 end
595 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
596     if(~hwif_in.reset_b) begin
597         field_storage.SHA256_CTRL.MODE.value <= 1'h1;
598     end else if(field_combo.SHA256_CTRL.MODE.load_next)
begin
599         field_storage.SHA256_CTRL.MODE.value <=
field_combo.SHA256_CTRL.MODE.next;
600     end
601 end
602 assign hwif_out.SHA256_CTRL.MODE.value =
field_storage.SHA256_CTRL.MODE.value;
603 // Field: sha256_reg.SHA256_CTRL.ZEROIZE
604 always_comb begin
605     automatic logic [0:0] next_c;
606     automatic logic load_next_c;
607     next_c = field_storage.SHA256_CTRL.ZEROIZE.value;
608     load_next_c = '0;
609     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr)
begin // SW write
610         next_c = (field_storage.SHA256_CTRL.ZEROIZE.value
& ~decoded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
611         load_next_c = '1;
612     end else begin // singlepulse clears back to 0
613         next_c = '0;
614         load_next_c = '1;
615     end
616     field_combo.SHA256_CTRL.ZEROIZE.next = next_c;
617     field_combo.SHA256_CTRL.ZEROIZE.load_next =
load_next_c;
618 end
619 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
620     if(~hwif_in.reset_b) begin
621         field_storage.SHA256_CTRL.ZEROIZE.value <= 1'h0;

```

```

622     end else if(field_combo.SHA256_CTRL.ZEROIZE.load_next)
begin
623         field_storage.SHA256_CTRL.ZEROIZE.value <=
field_combo.SHA256_CTRL.ZEROIZE.next;
624     end
625 end
626 assign hwif_out.SHA256_CTRL.ZEROIZE.value =
field_storage.SHA256_CTRL.ZEROIZE.value;
627 // Field: sha256_reg.SHA256_CTRL.WNTZ_MODE
628 always_comb begin
629     automatic logic [0:0] next_c;
630     automatic logic load_next_c;
631     next_c = field_storage.SHA256_CTRL.WNTZ_MODE.value;
632     load_next_c = '0;
633     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
634         next_c =
(field_storage.SHA256_CTRL.WNTZ_MODE.value & ~decoded_wr_biten[4:4])
| (decoded_wr_data[4:4] & decoded_wr_biten[4:4]);
635         load_next_c = '1;
636     end else begin // singlepulse clears back to 0
637         next_c = '0;
638         load_next_c = '1;
639     end
640     field_combo.SHA256_CTRL.WNTZ_MODE.next = next_c;
641     field_combo.SHA256_CTRL.WNTZ_MODE.load_next =
load_next_c;
642 end
643 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
644     if(~hwif_in.reset_b) begin
645         field_storage.SHA256_CTRL.WNTZ_MODE.value <= '1'h0;
646     end else
if(field_combo.SHA256_CTRL.WNTZ_MODE.load_next) begin
647         field_storage.SHA256_CTRL.WNTZ_MODE.value <=
field_combo.SHA256_CTRL.WNTZ_MODE.next;
648     end
649 end
650 assign hwif_out.SHA256_CTRL.WNTZ_MODE.value =
field_storage.SHA256_CTRL.WNTZ_MODE.value;
651 // Field: sha256_reg.SHA256_CTRL.WNTZ_W
652 always_comb begin
653     automatic logic [3:0] next_c;
654     automatic logic load_next_c;
655     next_c = field_storage.SHA256_CTRL.WNTZ_W.value;
656     load_next_c = '0;
657     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
658         next_c = (field_storage.SHA256_CTRL.WNTZ_W.value &
~decoded_wr_biten[8:5]) | (decoded_wr_data[8:5] & de-
coded_wr_biten[8:5]);
659         load_next_c = '1;
660     end
661     field_combo.SHA256_CTRL.WNTZ_W.next = next_c;
662     field_combo.SHA256_CTRL.WNTZ_W.load_next =
load_next_c;
663 end
664 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
665     if(~hwif_in.reset_b) begin
666         field_storage.SHA256_CTRL.WNTZ_W.value <= '4'h4;
667     end else if(field_combo.SHA256_CTRL.WNTZ_W.load_next)
begin
668         field_storage.SHA256_CTRL.WNTZ_W.value <=
field_combo.SHA256_CTRL.WNTZ_W.next;
669     end
670 end
671 assign hwif_out.SHA256_CTRL.WNTZ_W.value =
field_storage.SHA256_CTRL.WNTZ_W.value;
672 // Field: sha256_reg.SHA256_CTRL.WNTZ_N_MODE
673 always_comb begin
674     automatic logic [0:0] next_c;
675     automatic logic load_next_c;
676     next_c = field_storage.SHA256_CTRL.WNTZ_N_MODE.value;
677     load_next_c = '0;
678     if(decoded_reg_strb.SHA256_CTRL && decoded_req_is_wr
&& hwif_in.sha256_ready) begin // SW write
679         next_c =
(field_storage.SHA256_CTRL.WNTZ_N_MODE.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
680         load_next_c = '1;
681     end
682     field_combo.SHA256_CTRL.WNTZ_N_MODE.next = next_c;
683     field_combo.SHA256_CTRL.WNTZ_N_MODE.load_next =
load_next_c;
684 end
685 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
686     if(~hwif_in.reset_b) begin
687         field_storage.SHA256_CTRL.WNTZ_N_MODE.value <=
'1'h0;
688     end else
if(field_combo.SHA256_CTRL.WNTZ_N_MODE.load_next) begin
689         field_storage.SHA256_CTRL.WNTZ_N_MODE.value <=
field_combo.SHA256_CTRL.WNTZ_N_MODE.next;
690     end
691 end
692 assign hwif_out.SHA256_CTRL.WNTZ_N_MODE.value =
field_storage.SHA256_CTRL.WNTZ_N_MODE.value;
693 for(genvar i0=0; i0<16; i0++) begin
694     // Field: sha256_reg.SHA256_BLOCK[i].BLOCK

```

```

695     always_comb begin
696         automatic logic [31:0] next_c;
697         automatic logic load_next_c;
698         next_c =
field_storage.SHA256_BLOCK[i0].BLOCK.value;
699         load_next_c = '0;
700         if(decoded_reg_strb.SHA256_BLOCK[i0] && de-
coded_req_is_wr) begin // SW write
701             next_c =
(field_storage.SHA256_BLOCK[i0].BLOCK.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
702             load_next_c = '1;
703         end else if(hwif_in.SHA256_BLOCK[i0].BLOCK.hwclr)
begin // HW Clear
704             next_c = '0;
705             load_next_c = '1;
706         end
707         field_combo.SHA256_BLOCK[i0].BLOCK.next = next_c;
708         field_combo.SHA256_BLOCK[i0].BLOCK.load_next =
load_next_c;
709     end
710     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
711         if(~hwif_in.reset_b) begin
712             field_storage.SHA256_BLOCK[i0].BLOCK.value <=
32'h0;
713         end else
if(field_combo.SHA256_BLOCK[i0].BLOCK.load_next) begin
714             field_storage.SHA256_BLOCK[i0].BLOCK.value <=
field_combo.SHA256_BLOCK[i0].BLOCK.next;
715         end
716     end
717     assign hwif_out.SHA256_BLOCK[i0].BLOCK.value =
field_storage.SHA256_BLOCK[i0].BLOCK.value;
718 end
719 for(genvar i0=0; i0<8; i0++) begin
720     // Field: sha256_reg.SHA256_DIGEST[i].DIGEST
721     always_comb begin
722         automatic logic [31:0] next_c;
723         automatic logic load_next_c;
724         next_c =
field_storage.SHA256_DIGEST[i0].DIGEST.value;
725         load_next_c = '0;
726         if(hwif_in.SHA256_DIGEST[i0].DIGEST.hwclr) be-
gin // HW Clear
727             next_c = '0;
728             load_next_c = '1;
729         end else begin // HW Write
730             next_c =
hwif_in.SHA256_DIGEST[i0].DIGEST.next;
731             load_next_c = '1;
732         end
733         field_combo.SHA256_DIGEST[i0].DIGEST.next =
next_c;
734         field_combo.SHA256_DIGEST[i0].DIGEST.load_next =
load_next_c;
735     end
736     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
737         if(~hwif_in.reset_b) begin
738             field_storage.SHA256_DIGEST[i0].DIGEST.value
<= 32'h0;
739         end else
if(field_combo.SHA256_DIGEST[i0].DIGEST.load_next) begin
740             field_storage.SHA256_DIGEST[i0].DIGEST.value
<= field_combo.SHA256_DIGEST[i0].DIGEST.next;
741         end
742     end
743 end
744 // Field:
sha256_reg.intr_block_rf.global_intr_en_r.error_en
745 always_comb begin
746     automatic logic [0:0] next_c;
747     automatic logic load_next_c;
748     next_c =
field_storage.intr_block_rf.global_intr_en_r.error_en.value;
749     load_next_c = '0;
750     if(decoded_reg_strb.intr_block_rf.global_intr_en_r
&& decoded_req_is_wr) begin // SW write
751         next_c =
(field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
752         load_next_c = '1;
753     end
754     field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;
755     field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
load_next_c;
756 end
757     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
758         if(~hwif_in.reset_b) begin
759             field_storage.intr_block_rf.global_intr_en_r.error_en.value <= '1'h0;
760         end else
if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
begin
761             field_storage.intr_block_rf.global_intr_en_r.error_en.value <=

```

```

field_combo.intr_block_rf.global_intr_en_r.error_en.next;
762     end
763     end
764     // Field:
sha256_reg.intr_block_rf.global_intr_en_r.notif_en
765     always_comb begin
766         automatic logic [0:0] next_c;
767         automatic logic load_next_c;
768         next_c =
field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
769         load_next_c = '0;
770         if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
771             next_c =
(field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
772             load_next_c = '1;
773         end
774     end
field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;
775
field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
load_next_c;
776     end
777     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
778         if(~hwif_in.reset_b) begin
779
field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= '1'h0;
780         end else
if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
begin
781
field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
782         end
783     end
784     // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error0_en
785     always_comb begin
786         automatic logic [0:0] next_c;
787         automatic logic load_next_c;
788         next_c =
field_storage.intr_block_rf.error_intr_en_r.error0_en.value;
789         load_next_c = '0;
790         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
791             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error0_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
792             load_next_c = '1;
793         end
794     end
field_combo.intr_block_rf.error_intr_en_r.error0_en.next = next_c;
795
field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next =
load_next_c;
796     end
797     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
798         if(~hwif_in.reset_b) begin
799
field_storage.intr_block_rf.error_intr_en_r.error0_en.value <= '1'h0;
800         end else
if(field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next)
begin
801
field_storage.intr_block_rf.error_intr_en_r.error0_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error0_en.next;
802         end
803     end
804     // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error1_en
805     always_comb begin
806         automatic logic [0:0] next_c;
807         automatic logic load_next_c;
808         next_c =
field_storage.intr_block_rf.error_intr_en_r.error1_en.value;
809         load_next_c = '0;
810         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
811             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error1_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
812             load_next_c = '1;
813         end
814     end
field_combo.intr_block_rf.error_intr_en_r.error1_en.next = next_c;
815
field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next =
load_next_c;
816     end
817     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
818         if(~hwif_in.reset_b) begin
819
field_storage.intr_block_rf.error_intr_en_r.error1_en.value <= '1'h0;
820         end else
if(field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next)
begin
821

```

```

field_storage.intr_block_rf.error_intr_en_r.error1_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error1_en.next;
822     end
823     end
824     // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error2_en
825     always_comb begin
826         automatic logic [0:0] next_c;
827         automatic logic load_next_c;
828         next_c =
field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
829         load_next_c = '0;
830         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
831             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
832             load_next_c = '1;
833         end
834     end
field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;
835
field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
load_next_c;
836     end
837     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
838         if(~hwif_in.reset_b) begin
839
field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= '1'h0;
840         end else
if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
begin
841
field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
842         end
843     end
844     // Field:
sha256_reg.intr_block_rf.error_intr_en_r.error3_en
845     always_comb begin
846         automatic logic [0:0] next_c;
847         automatic logic load_next_c;
848         next_c =
field_storage.intr_block_rf.error_intr_en_r.error3_en.value;
849         load_next_c = '0;
850         if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
851             next_c =
(field_storage.intr_block_rf.error_intr_en_r.error3_en.value & ~de-
coded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
852             load_next_c = '1;
853         end
854     end
field_combo.intr_block_rf.error_intr_en_r.error3_en.next = next_c;
855
field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next =
load_next_c;
856     end
857     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
858         if(~hwif_in.reset_b) begin
859
field_storage.intr_block_rf.error_intr_en_r.error3_en.value <= '1'h0;
860         end else
if(field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next)
begin
861
field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error3_en.next;
862         end
863     end
864     // Field:
sha256_reg.intr_block_rf.notif_intr_en_r.notif_cmd_done_en
865     always_comb begin
866         automatic logic [0:0] next_c;
867         automatic logic load_next_c;
868         next_c =
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value;
869         load_next_c = '0;
870         if(decoded_reg_strb.intr_block_rf.notif_intr_en_r &&
decoded_req_is_wr) begin // SW write
871             next_c =
(field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
872             load_next_c = '1;
873         end
874     end
field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next =
next_c;
875
field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_ne
xt = load_next_c;
876     end
877     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
878         if(~hwif_in.reset_b) begin
879
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= '1'h0;
880         end else

```



```

if(field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_next) begin
881
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next;
882 end
883 end
884 // Field:
sha256_reg.intr_block_rf.error_global_intr_r.agg_sts
885 always_comb begin
886     automatic logic [0:0] next_c;
887     automatic logic load_next_c;
888     next_c =
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value;
889     load_next_c = '0;
890
891     // HW Write
892     next_c = hwif_out.intr_b-
lock_rf.error_internal_intr_r.intr;
893     load_next_c = '1;
894
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next = next_c;
895
field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next =
load_next_c;
896 end
897 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
898     if(~hwif_in.reset_b) begin
899
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1'h0;
900     end else
if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
begin
901
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
902     end
903 end
904 assign hwif_out.intr_block_rf.error_global_intr_r.intr =
905 |
(field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.global_intr_en_r.error_en.value);
906 // Field:
sha256_reg.intr_block_rf.notif_global_intr_r.agg_sts
907 always_comb begin
908     automatic logic [0:0] next_c;
909     automatic logic load_next_c;
910     next_c =
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
911     load_next_c = '0;
912
913     // HW Write
914     next_c = hwif_out.intr_b-
lock_rf.notif_internal_intr_r.intr;
915     load_next_c = '1;
916
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
917
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
load_next_c;
918 end
919 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
920     if(~hwif_in.reset_b) begin
921
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1'h0;
922     end else
if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
begin
923
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
924     end
925 end
926 assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
927 |
(field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.global_intr_en_r.notif_en.value);
928 // Field:
sha256_reg.intr_block_rf.error_internal_intr_r.error0_sts
929 always_comb begin
930     automatic logic [0:0] next_c;
931     automatic logic load_next_c;
932     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
933     load_next_c = '0;
934
if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
'0) begin // stickybit
935     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
936     load_next_c = '1;
937     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
938     next_c = '1;
939     load_next_c = '1;
940     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear

```

```

941     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
942     load_next_c = '1;
943 end
944
field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
next_c;
945
field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next
= load_next_c;
946 end
947 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
948     if(~hwif_in.error_reset_b) begin
949
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
<= 1'h0;
950     end else
if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
next) begin
951
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
952     end
953 end
954 // Field:
sha256_reg.intr_block_rf.error_internal_intr_r.error1_sts
955 always_comb begin
956     automatic logic [0:0] next_c;
957     automatic logic load_next_c;
958     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value;
959     load_next_c = '0;
960
if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value !=
'0) begin // stickybit
961     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
962     load_next_c = '1;
963     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
964     next_c = '1;
965     load_next_c = '1;
966     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
967     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value &
~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
968     load_next_c = '1;
969 end
970
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next =
next_c;
971
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_next
= load_next_c;
972 end
973 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
974     if(~hwif_in.error_reset_b) begin
975
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= 1'h0;
976     end else
if(field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_
next) begin
977
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next;
978     end
979 end
980 // Field:
sha256_reg.intr_block_rf.error_internal_intr_r.error2_sts
981 always_comb begin
982     automatic logic [0:0] next_c;
983     automatic logic load_next_c;
984     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
985     load_next_c = '0;
986
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !=
'0) begin // stickybit
987     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
988     load_next_c = '1;
989     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
990     next_c = '1;
991     load_next_c = '1;
992     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
993     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
994     load_next_c = '1;
995 end

```

```

996
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
997
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next
= load_next_c;
998     end
999     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1000         if(~hwif_in.error_reset_b) begin
1001
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= 1'h0;
1002         end else
1003         if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
next) begin
1004
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1005         end
1006         // Field:
sha256_reg.intr_block_rf.error_internal_intr_r.error3_sts
1007         always_comb begin
1008             automatic logic [0:0] next_c;
1009             automatic logic load_next_c;
1010             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1011             load_next_c = '0;
1012
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !
= '0) begin // stickybit
1013                 next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1014                 load_next_c = '1;
1015             end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1016                 next_c = '1;
1017                 load_next_c = '1;
1018             end else if(de-
coded_reg_stb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1019                 next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &
~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1020                 load_next_c = '1;
1021             end
1022
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
next_c;
1023
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next
= load_next_c;
1024     end
1025     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1026         if(~hwif_in.error_reset_b) begin
1027
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= 1'h0;
1028         end else
1029         if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_
next) begin
1030
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1031         end
1032         assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1033         |
(field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error0_en.value)
1034         || |
(field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error1_en.value)
1035         || |
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1036         || |
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1037         // Field:
sha256_reg.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
1038         always_comb begin
1039             automatic logic [0:0] next_c;
1040             automatic logic load_next_c;
1041             next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1042             load_next_c = '0;
1043
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0) begin // stickybit
1044                 next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1045                 load_next_c = '1;
1046             end else if(hwif_in.intr_b-

```

```

lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1047                 next_c = '1;
1048                 load_next_c = '1;
1049             end else if(de-
coded_reg_stb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1050                 next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1051                 load_next_c = '1;
1052             end
1053
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next = next_c;
1054
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
load_next = load_next_c;
1055     end
1056     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1057         if(~hwif_in.reset_b) begin
1058
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= 1'h0;
1059         end else
1060         if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
begin
1061
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next;
1062         end
1063         assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
1064         |
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1065         // Field:
sha256_reg.intr_block_rf.error_intr_trig_r.error0_trig
1066         always_comb begin
1067             automatic logic [0:0] next_c;
1068             automatic logic load_next_c;
1069             next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1070             load_next_c = '0;
1071
if(decoded_reg_stb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1072                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value |
(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1073                 load_next_c = '1;
1074             end else begin // singlepulse clears back to 0
1075                 next_c = '0;
1076                 load_next_c = '1;
1077             end
1078
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next =
next_c;
1079
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next =
load_next_c;
1080     end
1081     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1082         if(~hwif_in.reset_b) begin
1083
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
1'h0;
1084         end else
1085         if(field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next
) begin
1086
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next;
1087         end
1088         // Field:
sha256_reg.intr_block_rf.error_intr_trig_r.error1_trig
1089         always_comb begin
1090             automatic logic [0:0] next_c;
1091             automatic logic load_next_c;
1092             next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1093             load_next_c = '0;
1094
if(decoded_reg_stb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1095                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value |
(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1096                 load_next_c = '1;
1097             end else begin // singlepulse clears back to 0
1098                 next_c = '0;
1099                 load_next_c = '1;
1100             end
1101
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next =
next_c;

```



```

1102 field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next =
1103     load_next_c;
1104     end
1105     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1106         if(~hwif_in.reset_b) begin
1107             field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
1108                 1'h0;
1109             end else
1110                 if(field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next
1111                     ) begin
1112                         field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
1113                         field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next;
1114                     end
1115                     end
1116                     // Field:
1117 sha256_reg.intr_block_rf.error_intr_trig_r.error2_trig
1118     always_comb begin
1119         automatic logic [0:0] next_c;
1120         automatic logic load_next_c;
1121         next_c =
1122         field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1123         load_next_c = '0;
1124     end
1125     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
1126     decoded_req_is_wr) begin // SW write 1 set
1127         next_c =
1128         field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
1129         (decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1130         load_next_c = '1;
1131     end else begin // singlepulse clears back to 0
1132         next_c = '0;
1133         load_next_c = '1;
1134     end
1135     field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
1136     next_c;
1137     field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
1138     load_next_c;
1139     end
1140     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1141         if(~hwif_in.reset_b) begin
1142             field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1143                 1'h0;
1144             end else
1145                 if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
1146                     ) begin
1147                         field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1148                         field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1149                     end
1150                     end
1151                     // Field:
1152 sha256_reg.intr_block_rf.error_intr_trig_r.error3_trig
1153     always_comb begin
1154         automatic logic [0:0] next_c;
1155         automatic logic load_next_c;
1156         next_c =
1157         field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1158         load_next_c = '0;
1159     end
1160     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
1161     decoded_req_is_wr) begin // SW write 1 set
1162         next_c =
1163         field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
1164         (decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1165         load_next_c = '1;
1166     end else begin // singlepulse clears back to 0
1167         next_c = '0;
1168         load_next_c = '1;
1169     end
1170     field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
1171     next_c;
1172     field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
1173     load_next_c;
1174     end
1175     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1176         if(~hwif_in.reset_b) begin
1177             field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1178                 1'h0;
1179             end else
1180                 if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
1181                     ) begin
1182                         field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1183                         field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1184                     end
1185                     end
1186                     // Field:
1187 sha256_reg.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
1188     always_comb begin
1189         automatic logic [0:0] next_c;
1190         automatic logic load_next_c;
1191         next_c =

```

```

1162         load_next_c = '0';

1163         if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1164             next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1165             load_next_c = '1';
1166         end else begin // singlepulse clears back to 0
1167             next_c = '0';
1168             load_next_c = '1';
1169         end
1170     end
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
= next_c;
1171
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig loa
d_next = load_next_c;
1172     end
1173     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1174         if(~hwif_in.reset_b) begin
1175
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <= 1'h0;
1176         end else
if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
load_next) begin
1177
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <=
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
;
1178         end
1179     end
1180     // Field: sha256_reg.intr_block_rf.error0_intr_count_r.cnt
always_comb begin
1181         automatic logic [31:0] next_c;
1182         automatic logic load_next_c;
1183         next_c =
field_storage.intr_block_rf.error0_intr_count_r.cnt.value;
1184         load_next_c = '0';
1185         if(decoded_reg_strb.intr_block_rf.error0_intr_count_r
&& decoded_req_is_wr) begin // SW write
1186             next_c =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1187             load_next_c = '1';
1188         end
1189     end
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // increment
1190         if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1191             next_c = 32'hfffffff;
1192         end else begin
1193             next_c = next_c + 32'h1;
1194         end
1195         load_next_c = '1';
1196     end
1197
1198
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hfffffff);
1199
field_combo.intr_block_rf.error0_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
32'hfffffff);
1200     if(next_c > 32'hfffffff) begin
1201         next_c = 32'hfffffff;
1202         load_next_c = '1';
1203     end

1204     field_combo.intr_block_rf.error0_intr_count_r.cnt.next
= next_c;
1205
field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next =
load_next_c;
1206     end
1207     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1208         if(~hwif_in.error_reset_b) begin
1209
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <= 32'h0;
1210         end else
if(field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next) be-
gin
1211
field_storage.intr_block_rf.error0_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error0_intr_count_r.cnt.next;
1212         end
1213     end
1214     // Field: sha256_reg.intr_block_rf.error1_intr_count_r.cnt
always_comb begin
1215         automatic logic [31:0] next_c;
1216         automatic logic load_next_c;
1217         next_c =
field_storage.intr_block_rf.error1_intr_count_r.cnt.value;
1218         load_next_c = '0';

```

```

1220     if(decoded_reg_strb.intr_block_rf.error1_intr_count_r
&& decoded_req_is_wr) begin // SW write
1221         next_c =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1222         load_next_c = '1;
1223     end
1224
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // increment
1225     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1226         next_c = 32'hfffffff;
1227     end else begin
1228         next_c = next_c + 32'h1;
1229     end
1230     load_next_c = '1;
1231 end
1232
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hfffffff);
1233
field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
32'hfffffff);
1234     if(next_c > 32'hfffffff) begin
1235         next_c = 32'hfffffff;
1236         load_next_c = '1;
1237     end

1238     field_combo.intr_block_rf.error1_intr_count_r.cnt.next
= next_c;
1239
field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next =
load_next_c;
1240 end
1241     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1242         if(~hwif_in.error_reset_b) begin
1243
field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1244         end else
if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
gin
1245
field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1246         end
1247     end
1248     // Field: sha256_reg.intr_block_rf.error2_intr_count_r.cnt
1249     always_comb begin
1250         automatic logic [31:0] next_c;
1251         automatic logic load_next_c;
1252         next_c =
field_storage.intr_block_rf.error2_intr_count_r.cnt.value;
1253         load_next_c = '0;
1254         if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
&& decoded_req_is_wr) begin // SW write
1255             next_c =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1256             load_next_c = '1;
1257         end
1258
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // increment
1259     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1260         next_c = 32'hfffffff;
1261     end else begin
1262         next_c = next_c + 32'h1;
1263     end
1264     load_next_c = '1;
1265 end
1266
field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hfffffff);
1267
field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hfffffff);
1268     if(next_c > 32'hfffffff) begin
1269         next_c = 32'hfffffff;
1270         load_next_c = '1;
1271     end

1272     field_combo.intr_block_rf.error2_intr_count_r.cnt.next
= next_c;
1273
field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
load_next_c;
1274 end
1275     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1276         if(~hwif_in.error_reset_b) begin
1277
field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;

```

```

1278     end else
if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
gin
1279
field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
1280     end
1281
// Field: sha256_reg.intr_block_rf.error3_intr_count_r.cnt
1282     always_comb begin
1283         automatic logic [31:0] next_c;
1284         automatic logic load_next_c;
1285         next_c =
field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
1286         load_next_c = '0;
1287         if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
&& decoded_req_is_wr) begin // SW write
1288             next_c =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1289             load_next_c = '1;
1290         end
1291
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // increment
1292     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1293         next_c = 32'hfffffff;
1294     end else begin
1295         next_c = next_c + 32'h1;
1296     end
1297     load_next_c = '1;
1298 end
1299
field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hfffffff);
1301
field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hfffffff);
1302     if(next_c > 32'hfffffff) begin
1303         next_c = 32'hfffffff;
1304         load_next_c = '1;
1305     end

1306     field_combo.intr_block_rf.error3_intr_count_r.cnt.next
= next_c;
1307
field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
1308 end
1309     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1310         if(~hwif_in.error_reset_b) begin
1311
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
1312         end else
if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
1313
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
1314         end
1315     end
1316     // Field:
sha256_reg.intr_block_rf.notif_cmd_done_intr_count_r.cnt
1317     always_comb begin
1318         automatic logic [31:0] next_c;
1319         automatic logic load_next_c;
1320         next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
1321         load_next_c = '0;
1322         if(de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_req_is_wr) begin // SW write
1323             next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value &
~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1324             load_next_c = '1;
1325         end
1326
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul-
se.value) begin // increment
1327     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1328         next_c = 32'hfffffff;
1329     end else begin
1330         next_c = next_c + 32'h1;
1331     end
1332     load_next_c = '1;
1333 end
1334
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres-
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hfffffff);
1335
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur-

```

```

1336     if(next_c > 32'hffffffff) begin
1337         next_c = 32'hffffffff;
1338         load_next_c = '1;
1339     end
1340
1341     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
1342
1343     field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;
1344     end
1345     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1346         if(~hwif_in.reset_b) begin
1347             field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
32'h0;
1348         end else
1349             if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n
ext) begin
1350                 field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
1351             end
1352         end
1353         // Field:
1354         sha256_reg.intr_block_rf.error0_intr_count_incr_r.pulse
1355         always_comb begin
1356             automatic logic [0:0] next_c;
1357             automatic logic load_next_c;
1358             next_c =
1359             field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value;
1360             load_next_c = '0;
1361
1362             if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value)
begin // HW Write - we
1363                 next_c =
1364                 field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1365                 load_next_c = '1;
1366             end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1367                 next_c = '1;
1368                 load_next_c = '1;
1369             end
1370
1371             if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // decrement
1372                 next_c = next_c - 1'h1;
1373                 load_next_c = '1;
1374             end else begin
1375                 field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
'0;
1376                 end
1377             end
1378             field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.decrthresh
old =
1379             (field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'd0);
1380             field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next =
next_c;
1381             field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_next =
load_next_c;
1382             end
1383             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1384                 if(~hwif_in.reset_b) begin
1385                     field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'h0;
1386                     end else
1387                     if(field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_ne
xt) begin
1388                         field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next;
1389                     end
1390                     end
1391                     // Field:
1392                     sha256_reg.intr_block_rf.error1_intr_count_incr_r.pulse
1393                     always_comb begin
1394                         automatic logic [0:0] next_c;
1395                         automatic logic load_next_c;
1396                         next_c =
1397                         field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value;
1398                         load_next_c = '0;
1399
1400                         if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value)
begin // HW Write - we
1401                             next_c =
1402                             field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1403                             load_next_c = '1;
1404                         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1405                             next_c = '1;
1406                             load_next_c = '1;
1407                         end
1408
1409                         if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // decrement
1410                             next_c = next_c - 1'h1;
1411                             load_next_c = '1;
1412                         end else begin
1413                             field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
'0;
1414                             end
1415                         end
1416                         field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.decrthresh
old =
1417                         (field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'd0);
1418                         field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =
next_c;
1419                         field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
1420                         end
1421                         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1422                             if(~hwif_in.reset_b) begin
1423                                 field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
1424                                 end else
1425                                 if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_ne
xt) begin
1426                                     field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next;
1427                                 end
1428                                 // Field:
1429                                 sha256_reg.intr_block_rf.error2_intr_count_incr_r.pulse
1430                                 always_comb begin
1431                                     automatic logic [0:0] next_c;
1432                                     automatic logic load_next_c;
1433                                     next_c =
1434                                     field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
1435                                     load_next_c = '0;
1436
1437                                     if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
begin // HW Write - we
1438                                         next_c =
1439                                         field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1440                                         load_next_c = '1;
1441                                     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1442                                         next_c = '1;
1443                                         load_next_c = '1;
1444                                     end
1445
1446                                     if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
1447                                         next_c = next_c - 1'h1;
1448                                         load_next_c = '1;
1449                                     end else begin
1450                                         field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
'0;
1451                                         end
1452                                     end
1453                                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
old =
1454                                     (field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'd0);
1455                                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
next_c;
1456                                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
load_next_c;
1457                                     end
1458                                     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1459                                         if(~hwif_in.reset_b) begin
1460                                             field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'h0;
1461                                             end else
1462                                             if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
xt) begin
1463                                                 field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
1464                                             end
1465                                                 // Field:
1466                                                 sha256_reg.intr_block_rf.error3_intr_count_incr_r.pulse
1467                                                 always_comb begin
1468                                                     automatic logic [0:0] next_c;

```

```

1392         load_next_c = '1;
1393     end
1394
1395     if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // decrement
1396         field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1397         next_c = next_c - 1'h1;
1398         load_next_c = '1;
1399     end else begin
1400         field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
'0;
1401     end
1402
1403     field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.decrthresh
old =
1404     (field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'd0);
1405     field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =
next_c;
1406     field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
1407     end
1408     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1409         if(~hwif_in.reset_b) begin
1410             field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
1411             end else
1412             if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_ne
xt) begin
1413                 field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next;
1414             end
1415             // Field:
1416             sha256_reg.intr_block_rf.error2_intr_count_incr_r.pulse
1417             always_comb begin
1418                 automatic logic [0:0] next_c;
1419                 automatic logic load_next_c;
1420                 next_c =
1421                 field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
1422                 load_next_c = '0;
1423
1424                 if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
begin // HW Write - we
1425                     next_c =
1426                     field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1427                     load_next_c = '1;
1428                 end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1429                     next_c = '1;
1430                     load_next_c = '1;
1431                 end
1432
1433                 if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
1434                     next_c = next_c - 1'h1;
1435                     load_next_c = '1;
1436                 end else begin
1437                     field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
'0;
1438                     end
1439                 end
1440                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
old =
1441                 (field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'd0);
1442                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
next_c;
1443                 field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
load_next_c;
1444                 end
1445                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1446                     if(~hwif_in.reset_b) begin
1447                         field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'h0;
1448                         end else
1449                         if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
xt) begin
1450                             field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
1451                         end
1452                             // Field:
1453                             sha256_reg.intr_block_rf.error3_intr_count_incr_r.pulse
1454                             always_comb begin
1455                                 automatic logic [0:0] next_c;

```

```

1446     automatic logic load_next_c;
1447     next_c =
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
1448     load_next_c = '0';
1449
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value)
begin // HW Write - we
1450     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1451     load_next_c = '1';
1452     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1453         next_c = '1';
1454         load_next_c = '1';
1455     end
1456
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // decrement
1457
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1458     next_c = next_c - 1'h1;
1459     load_next_c = '1';
1460     end else begin
1461
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
'0';
1462     end
1463
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'd0);
1464
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next =
next_c;
1465
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next =
load_next_c;
1466     end
1467     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1468         if(~hwif_in.reset_b) begin
1469
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'h0;
1470         end else
if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_ne
xt) begin
1471
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
1472         end
1473     end
1474     // Field:
sha256_reg.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse
always_comb begin
1475
automatic logic [0:0] next_c;
1476     automatic logic load_next_c;
1477     next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value;
1478     load_next_c = '0';
1479
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value) begin // HW Write - we
1480         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1481         load_next_c = '1';
1482         end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1483             next_c = '1';
1484             load_next_c = '1';
1485         end
1486     end
1487
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // decrement
1488
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = (next_c < (1'h1));
1489     next_c = next_c - 1'h1;
1490     load_next_c = '1';
1491     end else begin
1492
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = '0';
1493     end
1494
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
crthreshold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'd0);
1495
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt = next_c;
1496
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
ad_next = load_next_c;
1497     end
1498     always_ff @(posedge clk or negedge hwif_in.reset_b) begin

```

```

1499     if(~hwif_in.reset_b) begin
1500
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'h0;
1501     end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
load_next) begin
1502
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt;
1503     end
1504     end
1505
-----
1506     // Write response
1507     //-----
1508
1509     assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;
1510     // Writes are always granted with no error response
1511     assign cpuif_wr_err = '0';
1512
1513     //-----
1514     // Readback
1515     //-----
1516
1517     logic readback_err;
1518     logic readback_done;
1519     logic [31:0] readback_data;
1520
1521     // Assign readback values to a flattened array
1522     logic [32:1:0][31:0] readback_array;
1523     for(genvar i0=0; i0<2; i0++) begin
1524         assign readback_array[i0*1 + 0][31:0] = (de-
coded_reg_strb.SHA256_NAME[i0] && !decoded_req_is_wr) ?
hwif_in.SHA256_NAME[i0].NAME.next : '0';
1525     end
1526     for(genvar i0=0; i0<2; i0++) begin
1527         assign readback_array[i0*1 + 2][31:0] = (de-
coded_reg_strb.SHA256_VERSION[i0] && !decoded_req_is_wr) ?
hwif_in.SHA256_VERSION[i0].VERSION.next : '0';
1528     end
1529     assign readback_array[4][0:0] = (de-
coded_reg_strb.SHA256_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA256_STATUS.READY.next : '0';
1530     assign readback_array[4][1:1] = (de-
coded_reg_strb.SHA256_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA256_STATUS.VALID.next : '0';
1531     assign readback_array[4][2:2] = (de-
coded_reg_strb.SHA256_STATUS && !decoded_req_is_wr) ?
hwif_in.SHA256_STATUS.WNTZ_BUSY.next : '0';
1532     assign readback_array[4][3:3] = '0';
1533     for(genvar i0=0; i0<8; i0++) begin
1534         assign readback_array[i0*1 + 5][31:0] = (de-
coded_reg_strb.SHA256_DIGEST[i0] && !decoded_req_is_wr) ?
field_storage.SHA256_DIGEST[i0].DIGEST.value : '0';
1535     end
1536     assign readback_array[13][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0';
1537     assign readback_array[13][1:1] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0';
1538     assign readback_array[13][31:2] = '0';
1539     assign readback_array[14][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error0_en.value : '0';
1540     assign readback_array[14][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error1_en.value : '0';
1541     assign readback_array[14][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0';
1542     assign readback_array[14][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0';
1543     assign readback_array[14][31:4] = '0';
1544     assign readback_array[15][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
: '0';
1545     assign readback_array[15][31:1] = '0';
1546     assign readback_array[16][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0';
1547     assign readback_array[16][31:1] = '0';
1548     assign readback_array[17][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?

```

```

field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0';
1549 assign readback_array[17][31:1] = '0';
1550 assign readback_array[18][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value :
'0';
1551 assign readback_array[18][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value :
'0';
1552 assign readback_array[18][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value :
'0';
1553 assign readback_array[18][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value :
'0';
1554 assign readback_array[18][31:4] = '0';
1555 assign readback_array[19][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value : '0';
1556 assign readback_array[19][31:1] = '0';
1557 assign readback_array[20][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value :
'0';
1558 assign readback_array[20][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value :
'0';
1559 assign readback_array[20][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value :
'0';
1560 assign readback_array[20][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value :
'0';
1561 assign readback_array[20][31:4] = '0';
1562 assign readback_array[21][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue : '0';
1563 assign readback_array[21][31:1] = '0';
1564 assign readback_array[22][31:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_r.cnt.value : '0';
1565 assign readback_array[23][31:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_r.cnt.value : '0';
1566 assign readback_array[24][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0';
1567 assign readback_array[25][31:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0';
1568 assign readback_array[26][31:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value :
'0';
1569 assign readback_array[27][0:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value :
'0';
1570 assign readback_array[27][31:1] = '0';
1571 assign readback_array[28][0:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value :
'0';
1572 assign readback_array[28][31:1] = '0';
1573 assign readback_array[29][0:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value :
'0';
1574 assign readback_array[29][31:1] = '0';
1575 assign readback_array[30][0:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value :
'0';
1576 assign readback_array[30][31:1] = '0';

```

```

1577 assign readback_array[31][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value : '0';
1578 assign readback_array[31][31:1] = '0';
1579
1580 // Reduce the array
1581 always_comb begin
1582     automatic logic [31:0] readback_data_var;
1583     readback_done = decoded_req & ~decoded_req_is_wr;
1584     readback_err = '0';
1585     readback_data_var = '0';
1586     for(int i=0; i<32; i++) readback_data_var |= read-
back_array[i];
1587     readback_data = readback_data_var;
1588 end
1589
1590 assign cpuif_rd_ack = readback_done;
1591 assign cpuif_rd_data = readback_data;
1592 assign cpuif_rd_err = readback_err;
1593
1594 `include "caliptra_prim_assert.sv"
1595 `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
1596
1597 endmodule

```

7. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_pkg.sv (185 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 package sha256_reg_pkg;
5
6     localparam SHA256_REG_DATA_WIDTH = 32;
7     localparam SHA256_REG_MIN_ADDR_WIDTH = 12;
8
9     typedef struct packed{
10         logic [31:0] next;
11     } sha256_reg_SHA256_NAME_NAME_in_t;
12
13     typedef struct packed{
14         sha256_reg_SHA256_NAME_NAME_in_t NAME;
15     } sha256_reg_SHA256_NAME_in_t;
16
17     typedef struct packed{
18         logic [31:0] next;
19     } sha256_reg_SHA256_VERSION_VERSION_in_t;
20
21     typedef struct packed{
22         sha256_reg_SHA256_VERSION_VERSION_in_t VERSION;
23     } sha256_reg_SHA256_VERSION_in_t;
24
25     typedef struct packed{
26         logic next;
27     } sha256_reg_SHA256_STATUS_READY_in_t;
28
29     typedef struct packed{
30         logic next;
31     } sha256_reg_SHA256_STATUS_VALID_in_t;
32
33     typedef struct packed{
34         logic next;
35     } sha256_reg_SHA256_STATUS_WNTZ_BUSY_in_t;
36
37     typedef struct packed{
38         sha256_reg_SHA256_STATUS_READY_in_t READY;
39         sha256_reg_SHA256_STATUS_VALID_in_t VALID;
40         sha256_reg_SHA256_STATUS_WNTZ_BUSY_in_t WNTZ_BUSY;
41     } sha256_reg_SHA256_STATUS_in_t;
42
43     typedef struct packed{
44         logic hwclr;
45     } sha256_reg_SHA256_BLOCK_BLOCK_in_t;
46
47     typedef struct packed{
48         sha256_reg_SHA256_BLOCK_BLOCK_in_t BLOCK;
49     } sha256_reg_SHA256_BLOCK_in_t;
50
51     typedef struct packed{
52         logic [31:0] next;
53         logic hwclr;
54     } sha256_reg_SHA256_DIGEST_DIGEST_in_t;
55
56     typedef struct packed{
57         sha256_reg_SHA256_DIGEST_DIGEST_in_t DIGEST;
58     } sha256_reg_SHA256_DIGEST_in_t;
59
60     typedef struct packed{
61         logic hwset;
62     }
63 sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error0_sts_enable_528ccada_n
ext_b1018582_resetsignal_939e99d4_in_t;
64
65     typedef struct packed{
66         logic hwset;
67     }

```



```

66     }
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error1_sts_enable_938cafe_n
ext_f460eb81_resetsignal_939e99d4_in_t;
67
68     typedef struct packed{
69         logic hwset;
70     }
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error2_sts_enable_0dacf7a6_n
ext_4b5b9e74_resetsignal_939e99d4_in_t;
71
72     typedef struct packed{
73         logic hwset;
74     }
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error3_sts_enable_fc3af94b_n
ext_c3125d40_resetsignal_939e99d4_in_t;
75
76     typedef struct packed{
77
78
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error0_sts_enable_528ccada_n
ext_b1018582_resetsignal_939e99d4_in_t error0_sts;
79
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error1_sts_enable_938cafe_n
ext_f460eb81_resetsignal_939e99d4_in_t error1_sts;
80
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error2_sts_enable_0dacf7a6_n
ext_4b5b9e74_resetsignal_939e99d4_in_t error2_sts;
81
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_error3_sts_enable_fc3af94b_n
ext_c3125d40_resetsignal_939e99d4_in_t error3_sts;
82
83     }
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_in_t;
84
85     typedef struct packed{
86         logic hwset;
87     }
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_notif_cmd_don
e_sts_enable_dabe0b8b_next_540fa3b7_in_t;
88
89     typedef struct packed{
90
91
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_notif_cmd_don
e_sts_enable_dabe0b8b_next_540fa3b7_in_t notif_cmd_done_sts;
92
93     }
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_in_t;
94
95     typedef struct packed{
96         logic reset_b;
97         logic error_reset_b;
98         logic sha256_ready;
99         sha256_reg_SHA256_NAME_in_t [2-1:0]SHA256_NAME;
100        sha256_reg_SHA256_VERSION_in_t
[2-1:0]SHA256_VERSION;
101
sha256_reg_SHA256_STATUS_in_t SHA256_STATUS;
102
sha256_reg_SHA256_BLOCK_in_t [16-1:0]SHA256_BLOCK;
103
sha256_reg_SHA256_DIGEST_in_t [8-1:0]SHA256_DIGEST;
104
sha256_reg_intr_block_t_in_t intr_block_rf;
105
} sha256_reg_in_t;
106
107
108     typedef struct packed{
109         logic value;
110     } sha256_reg_SHA256_CTRL_INIT_out_t;
111
112     typedef struct packed{
113         logic value;
114     } sha256_reg_SHA256_CTRL_NEXT_out_t;
115
116     typedef struct packed{
117         logic value;
118     } sha256_reg_SHA256_CTRL_MODE_out_t;
119
120     typedef struct packed{
121         logic value;
122     } sha256_reg_SHA256_CTRL_ZEROIZE_out_t;
123
124     typedef struct packed{
125         logic value;
126     } sha256_reg_SHA256_CTRL_WNTZ_MODE_out_t;
127
128     typedef struct packed{
129         logic [3:0] value;
130     } sha256_reg_SHA256_CTRL_WNTZ_W_out_t;
131
132     typedef struct packed{
133         logic value;

```

```

134     } sha256_reg_SHA256_CTRL_WNTZ_N_MODE_out_t;
135
136     typedef struct packed{
137         sha256_reg_SHA256_CTRL_INIT_out_t INIT;
138         sha256_reg_SHA256_CTRL_NEXT_out_t NEXT;
139         sha256_reg_SHA256_CTRL_MODE_out_t MODE;
140         sha256_reg_SHA256_CTRL_ZEROIZE_out_t ZEROIZE;
141         sha256_reg_SHA256_CTRL_WNTZ_MODE_out_t WNTZ_MODE;
142         sha256_reg_SHA256_CTRL_WNTZ_W_out_t WNTZ_W;
143         sha256_reg_SHA256_CTRL_WNTZ_N_MODE_out_t
WNTZ_N_MODE;
144     } sha256_reg_SHA256_CTRL_out_t;
145
146     typedef struct packed{
147         logic [31:0] value;
148     } sha256_reg_SHA256_BLOCK_BLOCK_out_t;
149
150     typedef struct packed{
151         sha256_reg_SHA256_BLOCK_BLOCK_out_t BLOCK;
152     } sha256_reg_SHA256_BLOCK_out_t;
153
154     typedef struct packed{
155         logic intr;
156     } sha256_reg_global_intr_t_agg_sts_dd3dcf0a_out_t;
157
158     typedef struct packed{
159         logic intr;
160     } sha256_reg_global_intr_t_agg_sts_e6399b4a_out_t;
161
162     typedef struct packed{
163         logic intr;
164     }
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_out_t;
165
166     typedef struct packed{
167         logic intr;
168     }
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t;
169
170     typedef struct packed{
171         sha256_reg_global_intr_t_agg_sts_dd3dcf0a_out_t er
ror_global_intr_r;
172         sha256_reg_global_intr_t_agg_sts_e6399b4a_out_t no
tif_global_intr_r;
173
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_out_t error_internal_intr_r;
174
sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_out_t no
tif_internal_intr_r;
175     } sha256_reg_intr_block_t_out_t;
176
177     typedef struct packed{
178         sha256_reg_SHA256_CTRL_out_t SHA256_CTRL;
179         sha256_reg_SHA256_BLOCK_out_t [16-1:0]SHA256_BLOCK;
180         sha256_reg_intr_block_t_out_t intr_block_rf;
181     } sha256_reg_out_t;
182
183     localparam SHA256_REG_ADDR_WIDTH = 32'd12;
184
185 endpackage

```

8. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_reg_uvm.sv (1,035 lines)

```

1
2 // This file was autogenerated by PeakRDL-uvm
3 package sha256_reg_uvm;
4 `include "uvm_macros.svh"
5 import uvm_pkg::*;
6 `include "sha256_reg_covergroups.svh"
7 // Reg - sha256_reg::SHA256_NAME
8 class sha256_reg_SHA256_NAME extends uvm_reg;
9     protected uvm_reg_data_t m_current;
10     protected uvm_reg_data_t m_data;
11     protected bit m_is_read;
12
13     sha256_reg_SHA256_NAME_bit_cg NAME_bit_cg[32];
14     sha256_reg_SHA256_NAME_fld_cg fld_cg;
15     rand uvm_reg_field NAME;
16
17     function new(string name = "sha256_reg_SHA256_NAME");
18         super.new(name, 32, build_coverage(UVM_CVR_ALL));
19     endfunction : new
20     extern virtual function void sample_values();
21     extern protected virtual function void
sample(uvm_reg_data_t data,
22
uvm_reg_data_t byte_en,
23
bit is_read,
24
uvm_reg_map map);
25
26     virtual function void build();
27         this.NAME = new("NAME");
28         this.NAME.configure(this, 32, 0, "R0", 1, 'h0, 0,
1, 0);
29         if (has_coverage(UVM_CVR_REG_BITS)) begin

```

```

30         foreach(NAME_bit_cg[bt]) NAME_bit_cg[bt] =
new();
31         end
32         if (has_coverage(UVM_CVR_FIELD_VALS))
33             fld_cg = new();
34         endfunction : build
35     endclass : sha256_reg__SHA256_NAME
36
37     // Reg - sha256_reg::SHA256_VERSION
38     class sha256_reg__SHA256_VERSION extends uvm_reg;
39     protected uvm_reg_data_t m_current;
40     protected uvm_reg_data_t m_data;
41     protected bit m_is_read;
42
43     sha256_reg__SHA256_VERSION_bit_cg VERSION_bit_cg[32];
44     sha256_reg__SHA256_VERSION_fld_cg fld_cg;
45     rand uvm_reg_field VERSION;
46
47     function new(string name =
"sha256_reg__SHA256_VERSION");
48         super.new(name, 32, build_coverage(UVM_CVR_ALL));
49     endfunction : new
50     extern virtual function void sample_values();
51     extern protected virtual function void
sample(uvm_reg_data_t data,
52
53     uvm_reg_data_t byte_en,
54
55     bit is_read,
56
57     uvm_reg_map map);
58
59     virtual function void build();
60     this.VERSION = new("VERSION");
61     this.VERSION.configure(this, 32, 0, "R0", 1, 'h0,
0, 1, 0);
62     if (has_coverage(UVM_CVR_REG_BITS)) begin
63         foreach(VERSION_bit_cg[bt]) VERSION_bit_cg[bt] =
new();
64     end
65     if (has_coverage(UVM_CVR_FIELD_VALS))
66         fld_cg = new();
67     endfunction : build
68     endclass : sha256_reg__SHA256_VERSION
69
70     // Reg - sha256_reg::SHA256_CTRL
71     class sha256_reg__SHA256_CTRL extends uvm_reg;
72     protected uvm_reg_data_t m_current;
73     protected uvm_reg_data_t m_data;
74     protected bit m_is_read;
75
76     sha256_reg__SHA256_CTRL_bit_cg INIT_bit_cg[1];
77     sha256_reg__SHA256_CTRL_bit_cg NEXT_bit_cg[1];
78     sha256_reg__SHA256_CTRL_bit_cg MODE_bit_cg[1];
79     sha256_reg__SHA256_CTRL_bit_cg ZEROIZE_bit_cg[1];
80     sha256_reg__SHA256_CTRL_bit_cg WNTZ_MODE_bit_cg[1];
81     sha256_reg__SHA256_CTRL_bit_cg WNTZ_W_bit_cg[4];
82     sha256_reg__SHA256_CTRL_bit_cg WNTZ_N_MODE_bit_cg[1];
83     sha256_reg__SHA256_CTRL_fld_cg fld_cg;
84     rand uvm_reg_field INIT;
85     rand uvm_reg_field NEXT;
86     rand uvm_reg_field MODE;
87     rand uvm_reg_field ZEROIZE;
88     rand uvm_reg_field WNTZ_MODE;
89     rand uvm_reg_field WNTZ_W;
90     rand uvm_reg_field WNTZ_N_MODE;
91
92     function new(string name = "sha256_reg__SHA256_CTRL");
93     super.new(name, 32, build_coverage(UVM_CVR_ALL));
94     endfunction : new
95     extern virtual function void sample_values();
96     extern protected virtual function void
sample(uvm_reg_data_t data,
97
98     uvm_reg_data_t byte_en,
99
100    bit is_read,
101
102    uvm_reg_map map);
103
104    virtual function void build();
105    this.INIT = new("INIT");
106    this.INIT.configure(this, 1, 0, "W0", 0, 'h0, 1,
1, 0);
107    this.NEXT = new("NEXT");
108    this.NEXT.configure(this, 1, 1, "W0", 0, 'h0, 1,
1, 0);
109    this.MODE = new("MODE");
110    this.MODE.configure(this, 1, 2, "W0", 0, 'h1, 1,
1, 0);
111    this.ZEROIZE = new("ZEROIZE");
112    this.ZEROIZE.configure(this, 1, 3, "W0", 0, 'h0,
1, 1, 0);
113    this.WNTZ_MODE = new("WNTZ_MODE");
114
115    this.WNTZ_MODE.configure(this, 1, 4, "W0", 0, 'h0,
1, 1, 0);
116    this.WNTZ_W = new("WNTZ_W");
117
118    this.WNTZ_W.configure(this, 4, 5, "W0", 0, 'h4, 1,

```

```

1, 0);
119    this.WNTZ_N_MODE = new("WNTZ_N_MODE");
120    this.WNTZ_N_MODE.configure(this, 1, 9, "W0", 0,
'h0, 1, 1, 0);
121    if (has_coverage(UVM_CVR_REG_BITS)) begin
122        foreach(INIT_bit_cg[bt]) INIT_bit_cg[bt] =
new();
123        foreach(NEXT_bit_cg[bt]) NEXT_bit_cg[bt] =
new();
124        foreach(MODE_bit_cg[bt]) MODE_bit_cg[bt] =
new();
125        foreach(ZEROIZE_bit_cg[bt]) ZEROIZE_bit_cg[bt] =
new();
126        foreach(WNTZ_MODE_bit_cg[bt])
WNTZ_MODE_bit_cg[bt] = new();
127        foreach(WNTZ_W_bit_cg[bt]) WNTZ_W_bit_cg[bt] =
new();
128        foreach(WNTZ_N_MODE_bit_cg[bt])
WNTZ_N_MODE_bit_cg[bt] = new();
129    end
130    if (has_coverage(UVM_CVR_FIELD_VALS))
131        fld_cg = new();
132    endfunction : build
133    endclass : sha256_reg__SHA256_CTRL
134
135    // Reg - sha256_reg::SHA256_STATUS
136    class sha256_reg__SHA256_STATUS extends uvm_reg;
137    protected uvm_reg_data_t m_current;
138    protected uvm_reg_data_t m_data;
139    protected bit m_is_read;
140
141    sha256_reg__SHA256_STATUS_bit_cg READY_bit_cg[1];
142    sha256_reg__SHA256_STATUS_bit_cg VALID_bit_cg[1];
143    sha256_reg__SHA256_STATUS_bit_cg WNTZ_BUSY_bit_cg[1];
144    sha256_reg__SHA256_STATUS_fld_cg fld_cg;
145    rand uvm_reg_field READY;
146    rand uvm_reg_field VALID;
147    rand uvm_reg_field WNTZ_BUSY;
148
149    function new(string name =
"sha256_reg__SHA256_STATUS");
150    super.new(name, 32, build_coverage(UVM_CVR_ALL));
151    endfunction : new
152    extern virtual function void sample_values();
153    extern protected virtual function void
sample(uvm_reg_data_t data,
154
155    uvm_reg_data_t byte_en,
156
157    bit is_read,
158
159    uvm_reg_map map);
160
161    virtual function void build();
162    this.READY = new("READY");
163    this.READY.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
164    this.VALID = new("VALID");
165    this.VALID.configure(this, 1, 1, "R0", 1, 'h0, 1,
1, 0);
166    this.WNTZ_BUSY = new("WNTZ_BUSY");
167
168    this.WNTZ_BUSY.configure(this, 1, 2, "R0", 1, 'h0,
1, 1, 0);
169    if (has_coverage(UVM_CVR_REG_BITS)) begin
170        foreach(READY_bit_cg[bt]) READY_bit_cg[bt] =
new();
171        foreach(VALID_bit_cg[bt]) VALID_bit_cg[bt] =
new();
172        foreach(WNTZ_BUSY_bit_cg[bt])
WNTZ_BUSY_bit_cg[bt] = new();
173    end
174    if (has_coverage(UVM_CVR_FIELD_VALS))
175        fld_cg = new();
176    endfunction : build
177    endclass : sha256_reg__SHA256_STATUS
178
179    // Reg - sha256_reg::SHA256_BLOCK
180    class sha256_reg__SHA256_BLOCK extends uvm_reg;
181    protected uvm_reg_data_t m_current;
182    protected uvm_reg_data_t m_data;
183    protected bit m_is_read;
184
185    sha256_reg__SHA256_BLOCK_bit_cg BLOCK_bit_cg[32];
186    sha256_reg__SHA256_BLOCK_fld_cg fld_cg;
187    rand uvm_reg_field BLOCK;
188
189    function new(string name =
"sha256_reg__SHA256_BLOCK");
190    super.new(name, 32, build_coverage(UVM_CVR_ALL));
191    endfunction : new
192    extern virtual function void sample_values();
193    extern protected virtual function void
sample(uvm_reg_data_t data,
194
195    uvm_reg_data_t byte_en,
196
197    bit is_read,

```



```

uvm_reg_map    map);
185
186     virtual function void build();
187         this.BLOCK = new("BLOCK");

188     this.BLOCK.configure(this, 32, 0, "W0", 1, 'h0, 1,
1, 0);
189         if (has_coverage(UVM_CVR_REG_BITS)) begin
190             foreach(BLOCK_bit_cg[bt]) BLOCK_bit_cg[bt] =
new();
191         end
192         if (has_coverage(UVM_CVR_FIELD_VALS))
193             fld_cg = new();
194         endfunction : build
195     endclass : sha256_reg__SHA256_BLOCK
196
197     // Reg - sha256_reg::SHA256_DIGEST
198     class sha256_reg__SHA256_DIGEST extends uvm_reg;
199     protected uvm_reg_data_t m_current;
200     protected uvm_reg_data_t m_data;
201     protected bit            m_is_read;
202
203     sha256_reg__SHA256_DIGEST_bit_cg DIGEST_bit_cg[32];
204     sha256_reg__SHA256_DIGEST_fld_cg fld_cg;
205     rand uvm_reg_field DIGEST;
206
207     function new(string name =
"sha256_reg__SHA256_DIGEST");
208         super.new(name, 32, build_coverage(UVM_CVR_ALL));
209     endfunction : new
210     extern virtual function void sample_values();
211     extern protected virtual function void
sample(uvm_reg_data_t data,
212
uvm_reg_data_t byte_en,
213
bit            is_read,
214
uvm_reg_map    map);
215
216     virtual function void build();
217         this.DIGEST = new("DIGEST");
218         this.DIGEST.configure(this, 32, 0, "R0", 1, 'h0,
1, 1, 0);
219         if (has_coverage(UVM_CVR_REG_BITS)) begin
220             foreach(DIGEST_bit_cg[bt]) DIGEST_bit_cg[bt] =
new();
221         end
222         if (has_coverage(UVM_CVR_FIELD_VALS))
223             fld_cg = new();
224         endfunction : build
225     endclass : sha256_reg__SHA256_DIGEST
226
227     // Reg - sha256_reg::global_intr_en_t
228     class sha256_reg__global_intr_en_t extends uvm_reg;
229     protected uvm_reg_data_t m_current;
230     protected uvm_reg_data_t m_data;
231     protected bit            m_is_read;
232
233     sha256_reg__global_intr_en_t_bit_cg er-
ror_en_bit_cg[1];
234     sha256_reg__global_intr_en_t_bit_cg no-
tif_en_bit_cg[1];
235     sha256_reg__global_intr_en_t_fld_cg fld_cg;
236     rand uvm_reg_field error_en;
237     rand uvm_reg_field notif_en;
238
239     function new(string name =
"sha256_reg__global_intr_en_t");
240         super.new(name, 32, build_coverage(UVM_CVR_ALL));
241     endfunction : new
242     extern virtual function void sample_values();
243     extern protected virtual function void
sample(uvm_reg_data_t data,
244
uvm_reg_data_t byte_en,
245
bit            is_read,
246
uvm_reg_map    map);
247
248     virtual function void build();
249         this.error_en = new("error_en");
250         this.error_en.configure(this, 1, 0, "RW", 0, 'h0,
1, 1, 0);
251         this.notif_en = new("notif_en");
252         this.notif_en.configure(this, 1, 1, "RW", 0, 'h0,
1, 1, 0);
253         if (has_coverage(UVM_CVR_REG_BITS)) begin
254             foreach(error_en_bit_cg[bt]) er-
ror_en_bit_cg[bt] = new();
255             foreach(notif_en_bit_cg[bt]) no-
tif_en_bit_cg[bt] = new();
256         end
257         if (has_coverage(UVM_CVR_FIELD_VALS))
258             fld_cg = new();
259         endfunction : build
260     endclass : sha256_reg__global_intr_en_t
261
262     // Reg - sha256_reg::error_intr_en_t

```

```

263     class sha256_reg__error_intr_en_t extends uvm_reg;
264     protected uvm_reg_data_t m_current;
265     protected uvm_reg_data_t m_data;
266     protected bit            m_is_read;
267
268     sha256_reg__error_intr_en_t_bit_cg er-
ror0_en_bit_cg[1];
269     sha256_reg__error_intr_en_t_bit_cg er-
ror1_en_bit_cg[1];
270     sha256_reg__error_intr_en_t_bit_cg er-
ror2_en_bit_cg[1];
271     sha256_reg__error_intr_en_t_bit_cg er-
ror3_en_bit_cg[1];
272     sha256_reg__error_intr_en_t_fld_cg fld_cg;
273     rand uvm_reg_field error0_en;
274     rand uvm_reg_field error1_en;
275     rand uvm_reg_field error2_en;
276     rand uvm_reg_field error3_en;
277
278     function new(string name =
"sha256_reg__error_intr_en_t");
279         super.new(name, 32, build_coverage(UVM_CVR_ALL));
280     endfunction : new
281     extern virtual function void sample_values();
282     extern protected virtual function void
sample(uvm_reg_data_t data,
283
uvm_reg_data_t byte_en,
284
bit            is_read,
285
uvm_reg_map    map);
286
287     virtual function void build();
288         this.error0_en = new("error0_en");
289
290         this.error0_en.configure(this, 1, 0, "RW", 0, 'h0,
1, 1, 0);
291         this.error1_en = new("error1_en");
292
293         this.error1_en.configure(this, 1, 1, "RW", 0, 'h0,
1, 1, 0);
294         this.error2_en = new("error2_en");
295
296         this.error2_en.configure(this, 1, 2, "RW", 0, 'h0,
1, 1, 0);
297         this.error3_en = new("error3_en");
298
299         this.error3_en.configure(this, 1, 3, "RW", 0, 'h0,
1, 1, 0);
300         if (has_coverage(UVM_CVR_REG_BITS)) begin
301             foreach(error0_en_bit_cg[bt]) er-
ror0_en_bit_cg[bt] = new();
302             foreach(error1_en_bit_cg[bt]) er-
ror1_en_bit_cg[bt] = new();
303             foreach(error2_en_bit_cg[bt]) er-
ror2_en_bit_cg[bt] = new();
304             foreach(error3_en_bit_cg[bt]) er-
ror3_en_bit_cg[bt] = new();
305         end
306         if (has_coverage(UVM_CVR_FIELD_VALS))
307             fld_cg = new();
308         endfunction : build
309     endclass : sha256_reg__error_intr_en_t
310
311     // Reg - sha256_reg::notif_intr_en_t
312     class sha256_reg__notif_intr_en_t extends uvm_reg;
313     protected uvm_reg_data_t m_current;
314     protected uvm_reg_data_t m_data;
315     protected bit            m_is_read;
316
317     sha256_reg__notif_intr_en_t_bit_cg no-
tif_cmd_done_en_bit_cg[1];
318     sha256_reg__notif_intr_en_t_fld_cg fld_cg;
319     rand uvm_reg_field notif_cmd_done_en;
320
321     function new(string name =
"sha256_reg__notif_intr_en_t");
322         super.new(name, 32, build_coverage(UVM_CVR_ALL));
323     endfunction : new
324     extern virtual function void sample_values();
325     extern protected virtual function void
sample(uvm_reg_data_t data,
326
uvm_reg_data_t byte_en,
327
bit            is_read,
328
uvm_reg_map    map);
329
330     virtual function void build();
331         this.notif_cmd_done_en = new("notif_cmd_done_en");
332
333         this.notif_cmd_done_en.configure(this, 1, 0, "RW",
0, 'h0, 1, 1, 0);
334         if (has_coverage(UVM_CVR_REG_BITS)) begin
335             foreach(notif_cmd_done_en_bit_cg[bt]) no-
tif_cmd_done_en_bit_cg[bt] = new();
336         end
337         if (has_coverage(UVM_CVR_FIELD_VALS))
338             fld_cg = new();

```

```

334     endfunction : build
335 endclass : sha256_reg_notif_intr_en_t
336
337 // Reg - sha256_reg::global_intr_t_agg_sts_dd3dcf0a
338 class sha256_reg_global_intr_t_agg_sts_dd3dcf0a extends
uvm_reg;
339     protected uvm_reg_data_t m_current;
340     protected uvm_reg_data_t m_data;
341     protected bit m_is_read;
342
343     sha256_reg_global_intr_t_agg_sts_dd3dcf0a_bit_cg
agg_sts_bit_cg[1];
344     sha256_reg_global_intr_t_agg_sts_dd3dcf0a_fld_cg
fld_cg;
345     rand uvm_reg_field agg_sts;
346
347     function new(string name =
"sha256_reg_global_intr_t_agg_sts_dd3dcf0a");
348         super.new(name, 32, build_coverage(UVM_CVR_ALL));
349     endfunction : new
350     extern virtual function void sample_values();
351     extern protected virtual function void
sample(uvm_reg_data_t data,
352
uvm_reg_data_t byte_en,
353
bit is_read,
354
uvm_reg_map map);
355
356     virtual function void build();
357     this.agg_sts = new("agg_sts");
358     this.agg_sts.configure(this, 1, 0, "R0", 1, 'h0,
1, 1, 0);
359     if (has_coverage(UVM_CVR_REG_BITS)) begin
360         foreach(agg_sts_bit_cg[bt]) agg_sts_bit_cg[bt] =
new();
361     end
362     if (has_coverage(UVM_CVR_FIELD_VALS))
363         fld_cg = new();
364     endfunction : build
365 endclass : sha256_reg_global_intr_t_agg_sts_dd3dcf0a
366
367 // Reg - sha256_reg::global_intr_t_agg_sts_e6399b4a
368 class sha256_reg_global_intr_t_agg_sts_e6399b4a extends
uvm_reg;
369     protected uvm_reg_data_t m_current;
370     protected uvm_reg_data_t m_data;
371     protected bit m_is_read;
372
373     sha256_reg_global_intr_t_agg_sts_e6399b4a_bit_cg
agg_sts_bit_cg[1];
374     sha256_reg_global_intr_t_agg_sts_e6399b4a_fld_cg
fld_cg;
375     rand uvm_reg_field agg_sts;
376
377     function new(string name =
"sha256_reg_global_intr_t_agg_sts_e6399b4a");
378         super.new(name, 32, build_coverage(UVM_CVR_ALL));
379     endfunction : new
380     extern virtual function void sample_values();
381     extern protected virtual function void
sample(uvm_reg_data_t data,
382
uvm_reg_data_t byte_en,
383
bit is_read,
384
uvm_reg_map map);
385
386     virtual function void build();
387     this.agg_sts = new("agg_sts");
388     this.agg_sts.configure(this, 1, 0, "R0", 1, 'h0,
1, 1, 0);
389     if (has_coverage(UVM_CVR_REG_BITS)) begin
390         foreach(agg_sts_bit_cg[bt]) agg_sts_bit_cg[bt] =
new();
391     end
392     if (has_coverage(UVM_CVR_FIELD_VALS))
393         fld_cg = new();
394     endfunction : build
395 endclass : sha256_reg_global_intr_t_agg_sts_e6399b4a
396
397 // Reg -
sha256_reg::error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378
398 class
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378 extends uvm_reg;
399     protected uvm_reg_data_t m_current;
400     protected uvm_reg_data_t m_data;
401     protected bit m_is_read;
402
403     sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_bit_cg error0_sts_bit_cg[1];
404
405     sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_bit_cg error1_sts_bit_cg[1];

```

```

405
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_bit_cg error2_sts_bit_cg[1];
406
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_bit_cg error3_sts_bit_cg[1];
407
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378_fld_cg fld_cg;
408     rand uvm_reg_field error0_sts;
409     rand uvm_reg_field error1_sts;
410     rand uvm_reg_field error2_sts;
411     rand uvm_reg_field error3_sts;
412
413     function new(string name =
"sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_e
ror2_sts_b1cf2205_error3_sts_74a35378");
414         super.new(name, 32, build_coverage(UVM_CVR_ALL));
415     endfunction : new
416     extern virtual function void sample_values();
417     extern protected virtual function void
sample(uvm_reg_data_t data,
418
uvm_reg_data_t byte_en,
419
bit is_read,
420
uvm_reg_map map);
421
422     virtual function void build();
423     this.error0_sts = new("error0_sts");
424     this.error0_sts.configure(this, 1, 0, "W1C", 1,
'h0, 1, 1, 0);
425     this.error1_sts = new("error1_sts");
426     this.error1_sts.configure(this, 1, 1, "W1C", 1,
'h0, 1, 1, 0);
427     this.error2_sts = new("error2_sts");
428     this.error2_sts.configure(this, 1, 2, "W1C", 1,
'h0, 1, 1, 0);
429     this.error3_sts = new("error3_sts");
430     this.error3_sts.configure(this, 1, 3, "W1C", 1,
'h0, 1, 1, 0);
431     if (has_coverage(UVM_CVR_REG_BITS)) begin
432         foreach(error0_sts_bit_cg[bt]) er
ror0_sts_bit_cg[bt] = new();
433         foreach(error1_sts_bit_cg[bt]) er
ror1_sts_bit_cg[bt] = new();
434         foreach(error2_sts_bit_cg[bt]) er
ror2_sts_bit_cg[bt] = new();
435         foreach(error3_sts_bit_cg[bt]) er
ror3_sts_bit_cg[bt] = new();
436     end
437     if (has_coverage(UVM_CVR_FIELD_VALS))
438         fld_cg = new();
439     endfunction : build
440 endclass :
sha256_reg_error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378
441
442 // Reg -
sha256_reg::notif_intr_t_notif_cmd_done_sts_1c68637e
443 class sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e
extends uvm_reg;
444     protected uvm_reg_data_t m_current;
445     protected uvm_reg_data_t m_data;
446     protected bit m_is_read;
447
448     sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_bit_cg no-
tif_cmd_done_sts_bit_cg[1];
449
450     sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e_fld_cg fld_cg;
451     rand uvm_reg_field notif_cmd_done_sts;
452
453     function new(string name =
"sha256_reg_notif_intr_t_notif_cmd_done_sts_1c68637e");
454         super.new(name, 32, build_coverage(UVM_CVR_ALL));
455     endfunction : new
456     extern virtual function void sample_values();
457     extern protected virtual function void
sample(uvm_reg_data_t data,
458
uvm_reg_data_t byte_en,
459
bit is_read,
460
uvm_reg_map map);
461
462     virtual function void build();
463     this.notif_cmd_done_sts = new("no-
tif_cmd_done_sts");
464     this.notif_cmd_done_sts.configure(this, 1, 0,
"W1C", 1, 'h0, 1, 1, 0);
465     if (has_coverage(UVM_CVR_REG_BITS)) begin
466         foreach(notif_cmd_done_sts_bit_cg[bt]) no-
tif_cmd_done_sts_bit_cg[bt] = new();
467     end
468     if (has_coverage(UVM_CVR_FIELD_VALS))
469         fld_cg = new();
470     endfunction : build

```

```

470     endclass :
sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e
471
472     // Reg - sha256_reg::error_intr_trig_t
473     class sha256_reg__error_intr_trig_t extends uvm_reg;
474     protected uvm_reg_data_t m_current;
475     protected uvm_reg_data_t m_data;
476     protected bit m_is_read;
477
478     sha256_reg__error_intr_trig_t_bit_cg error0_trig_bit_cg[1];
479     sha256_reg__error_intr_trig_t_bit_cg error1_trig_bit_cg[1];
480     sha256_reg__error_intr_trig_t_bit_cg error2_trig_bit_cg[1];
481     sha256_reg__error_intr_trig_t_bit_cg error3_trig_bit_cg[1];
482     sha256_reg__error_intr_trig_t_fld_cg fld_cg;
483     rand uvm_reg_field error0_trig;
484     rand uvm_reg_field error1_trig;
485     rand uvm_reg_field error2_trig;
486     rand uvm_reg_field error3_trig;
487
488     function new(string name =
"sha256_reg__error_intr_trig_t");
489     super.new(name, 32, build_coverage(UVM_CVR_ALL));
490     endfunction : new
491     extern virtual function void sample_values();
492     extern protected virtual function void
sample(uvm_reg_data_t data,
493
uvm_reg_data_t byte_en,
494
bit is_read,
495
uvm_reg_map map);
496
497     virtual function void build();
498     this.error0_trig = new("error0_trig");
499     this.error0_trig.configure(this, 1, 0, "W1S", 0,
'h0, 1, 1, 0);
500     this.error1_trig = new("error1_trig");
501     this.error1_trig.configure(this, 1, 1, "W1S", 0,
'h0, 1, 1, 0);
502     this.error2_trig = new("error2_trig");
503     this.error2_trig.configure(this, 1, 2, "W1S", 0,
'h0, 1, 1, 0);
504     this.error3_trig = new("error3_trig");
505     this.error3_trig.configure(this, 1, 3, "W1S", 0,
'h0, 1, 1, 0);
506     if (has_coverage(UVM_CVR_REG_BITS)) begin
507     foreach(error0_trig_bit_cg[bt]) error0_trig_bit_cg[bt] = new();
508     foreach(error1_trig_bit_cg[bt]) error1_trig_bit_cg[bt] = new();
509     foreach(error2_trig_bit_cg[bt]) error2_trig_bit_cg[bt] = new();
510     foreach(error3_trig_bit_cg[bt]) error3_trig_bit_cg[bt] = new();
511     end
512     if (has_coverage(UVM_CVR_FIELD_VALS))
513     fld_cg = new();
514     endfunction : build
515     endclass : sha256_reg__error_intr_trig_t
516
517     // Reg - sha256_reg::notif_intr_trig_t
518     class sha256_reg__notif_intr_trig_t extends uvm_reg;
519     protected uvm_reg_data_t m_current;
520     protected uvm_reg_data_t m_data;
521     protected bit m_is_read;
522
523     sha256_reg__notif_intr_trig_t_bit_cg notif_cmd_done_trig_bit_cg[1];
524     sha256_reg__notif_intr_trig_t_fld_cg fld_cg;
525     rand uvm_reg_field notif_cmd_done_trig;
526
527     function new(string name =
"sha256_reg__notif_intr_trig_t");
528     super.new(name, 32, build_coverage(UVM_CVR_ALL));
529     endfunction : new
530     extern virtual function void sample_values();
531     extern protected virtual function void
sample(uvm_reg_data_t data,
532
uvm_reg_data_t byte_en,
533
bit is_read,
534
uvm_reg_map map);
535
536     virtual function void build();
537     this.notif_cmd_done_trig = new("no-
tif_cmd_done_trig");
538     this.notif_cmd_done_trig.configure(this, 1, 0,
"W1S", 0, 'h0, 1, 1, 0);
539     if (has_coverage(UVM_CVR_REG_BITS)) begin
540     foreach(notif_cmd_done_trig_bit_cg[bt]) no-
tif_cmd_done_trig_bit_cg[bt] = new();
541     end
542     if (has_coverage(UVM_CVR_FIELD_VALS))
543     fld_cg = new();

```

```

544     endfunction : build
545     endclass : sha256_reg__notif_intr_trig_t
546
547     // Reg - sha256_reg::intr_count_t_cnt_35ace267
548     class sha256_reg__intr_count_t_cnt_35ace267 extends
uvm_reg;
549     protected uvm_reg_data_t m_current;
550     protected uvm_reg_data_t m_data;
551     protected bit m_is_read;
552
553     sha256_reg__intr_count_t_cnt_35ace267_bit_cg
cnt_bit_cg[32];
554     sha256_reg__intr_count_t_cnt_35ace267_fld_cg fld_cg;
555     rand uvm_reg_field cnt;
556
557     function new(string name =
"sha256_reg__intr_count_t_cnt_35ace267");
558     super.new(name, 32, build_coverage(UVM_CVR_ALL));
559     endfunction : new
560     extern virtual function void sample_values();
561     extern protected virtual function void
sample(uvm_reg_data_t data,
562
uvm_reg_data_t byte_en,
563
bit is_read,
564
uvm_reg_map map);
565
566     virtual function void build();
567     this.cnt = new("cnt");
568     this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
569     if (has_coverage(UVM_CVR_REG_BITS)) begin
570     foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
571     end
572     if (has_coverage(UVM_CVR_FIELD_VALS))
573     fld_cg = new();
574     endfunction : build
575     endclass : sha256_reg__intr_count_t_cnt_35ace267
576
577     // Reg - sha256_reg::intr_count_t_cnt_73c42c28
578     class sha256_reg__intr_count_t_cnt_73c42c28 extends
uvm_reg;
579     protected uvm_reg_data_t m_current;
580     protected uvm_reg_data_t m_data;
581     protected bit m_is_read;
582
583     sha256_reg__intr_count_t_cnt_73c42c28_bit_cg
cnt_bit_cg[32];
584     sha256_reg__intr_count_t_cnt_73c42c28_fld_cg fld_cg;
585     rand uvm_reg_field cnt;
586
587     function new(string name =
"sha256_reg__intr_count_t_cnt_73c42c28");
588     super.new(name, 32, build_coverage(UVM_CVR_ALL));
589     endfunction : new
590     extern virtual function void sample_values();
591     extern protected virtual function void
sample(uvm_reg_data_t data,
592
uvm_reg_data_t byte_en,
593
bit is_read,
594
uvm_reg_map map);
595
596     virtual function void build();
597     this.cnt = new("cnt");
598     this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
599     if (has_coverage(UVM_CVR_REG_BITS)) begin
600     foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
601     end
602     if (has_coverage(UVM_CVR_FIELD_VALS))
603     fld_cg = new();
604     endfunction : build
605     endclass : sha256_reg__intr_count_t_cnt_73c42c28
606
607     // Reg - sha256_reg::intr_count_t_cnt_d8af96ff
608     class sha256_reg__intr_count_t_cnt_d8af96ff extends
uvm_reg;
609     protected uvm_reg_data_t m_current;
610     protected uvm_reg_data_t m_data;
611     protected bit m_is_read;
612
613     sha256_reg__intr_count_t_cnt_d8af96ff_bit_cg
cnt_bit_cg[32];
614     sha256_reg__intr_count_t_cnt_d8af96ff_fld_cg fld_cg;
615     rand uvm_reg_field cnt;
616
617     function new(string name =
"sha256_reg__intr_count_t_cnt_d8af96ff");
618     super.new(name, 32, build_coverage(UVM_CVR_ALL));
619     endfunction : new
620     extern virtual function void sample_values();
621     extern protected virtual function void
sample(uvm_reg_data_t data,
622

```

```

623 uvm_reg_data_t byte_en,
624 bit is_read,
625 uvm_reg_map map);
626 virtual function void build();
627     this.cnt = new("cnt");
628     this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
629     if (has_coverage(UVM_CVR_REG_BITS)) begin
630         foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
631     end
632     if (has_coverage(UVM_CVR_FIELD_VALS))
633         fld_cg = new();
634     endfunction : build
635 endclass : sha256_reg_intr_count_t_cnt_d8af96ff
636
637 // Reg - sha256_reg::intr_count_t_cnt_9bd7f809
638 class sha256_reg_intr_count_t_cnt_9bd7f809 extends
uvm_reg;
639     protected uvm_reg_data_t m_current;
640     protected uvm_reg_data_t m_data;
641     protected bit m_is_read;
642
643     sha256_reg_intr_count_t_cnt_9bd7f809_bit_cg
cnt_bit_cg[32];
644     sha256_reg_intr_count_t_cnt_9bd7f809_fld_cg fld_cg;
645     rand uvm_reg_field cnt;
646
647     function new(string name =
"sha256_reg_intr_count_t_cnt_9bd7f809");
648         super.new(name, 32, build_coverage(UVM_CVR_ALL));
649     endfunction : new
650     extern virtual function void sample_values();
651     extern protected virtual function void
sample(uvm_reg_data_t data,
652 uvm_reg_data_t byte_en,
653 bit is_read,
654 uvm_reg_map map);
655
656     virtual function void build();
657         this.cnt = new("cnt");
658         this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
659         if (has_coverage(UVM_CVR_REG_BITS)) begin
660             foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
661         end
662         if (has_coverage(UVM_CVR_FIELD_VALS))
663             fld_cg = new();
664         endfunction : build
665     endclass : sha256_reg_intr_count_t_cnt_9bd7f809
666
667 // Reg - sha256_reg::intr_count_t_cnt_be67d6d5
668 class sha256_reg_intr_count_t_cnt_be67d6d5 extends
uvm_reg;
669     protected uvm_reg_data_t m_current;
670     protected uvm_reg_data_t m_data;
671     protected bit m_is_read;
672
673     sha256_reg_intr_count_t_cnt_be67d6d5_bit_cg
cnt_bit_cg[32];
674     sha256_reg_intr_count_t_cnt_be67d6d5_fld_cg fld_cg;
675     rand uvm_reg_field cnt;
676
677     function new(string name =
"sha256_reg_intr_count_t_cnt_be67d6d5");
678         super.new(name, 32, build_coverage(UVM_CVR_ALL));
679     endfunction : new
680     extern virtual function void sample_values();
681     extern protected virtual function void
sample(uvm_reg_data_t data,
682 uvm_reg_data_t byte_en,
683 bit is_read,
684 uvm_reg_map map);
685
686     virtual function void build();
687         this.cnt = new("cnt");
688         this.cnt.configure(this, 32, 0, "RW", 1, 'h0, 1,
1, 0);
689         if (has_coverage(UVM_CVR_REG_BITS)) begin
690             foreach(cnt_bit_cg[bt]) cnt_bit_cg[bt] =
new();
691         end
692         if (has_coverage(UVM_CVR_FIELD_VALS))
693             fld_cg = new();
694         endfunction : build
695     endclass : sha256_reg_intr_count_t_cnt_be67d6d5
696
697 // Reg - sha256_reg::intr_count_incr_t_pulse_37026c97
698 class sha256_reg_intr_count_incr_t_pulse_37026c97 extends
uvm_reg;

```

```

699     protected uvm_reg_data_t m_current;
700     protected uvm_reg_data_t m_data;
701     protected bit m_is_read;
702
703     sha256_reg_intr_count_incr_t_pulse_37026c97_bit_cg
pulse_bit_cg[1];
704     sha256_reg_intr_count_incr_t_pulse_37026c97_fld_cg
fld_cg;
705     rand uvm_reg_field pulse;
706
707     function new(string name =
"sha256_reg_intr_count_incr_t_pulse_37026c97");
708         super.new(name, 32, build_coverage(UVM_CVR_ALL));
709     endfunction : new
710     extern virtual function void sample_values();
711     extern protected virtual function void
sample(uvm_reg_data_t data,
712 uvm_reg_data_t byte_en,
713 bit is_read,
714 uvm_reg_map map);
715
716     virtual function void build();
717         this.pulse = new("pulse");
718         this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
719         if (has_coverage(UVM_CVR_REG_BITS)) begin
720             foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
721         end
722         if (has_coverage(UVM_CVR_FIELD_VALS))
723             fld_cg = new();
724         endfunction : build
725     endclass : sha256_reg_intr_count_incr_t_pulse_37026c97
726
727 // Reg - sha256_reg::intr_count_incr_t_pulse_d860d977
728 class sha256_reg_intr_count_incr_t_pulse_d860d977 extends
uvm_reg;
729     protected uvm_reg_data_t m_current;
730     protected uvm_reg_data_t m_data;
731     protected bit m_is_read;
732
733     sha256_reg_intr_count_incr_t_pulse_d860d977_bit_cg
pulse_bit_cg[1];
734     sha256_reg_intr_count_incr_t_pulse_d860d977_fld_cg
fld_cg;
735     rand uvm_reg_field pulse;
736
737     function new(string name =
"sha256_reg_intr_count_incr_t_pulse_d860d977");
738         super.new(name, 32, build_coverage(UVM_CVR_ALL));
739     endfunction : new
740     extern virtual function void sample_values();
741     extern protected virtual function void
sample(uvm_reg_data_t data,
742 uvm_reg_data_t byte_en,
743 bit is_read,
744 uvm_reg_map map);
745
746     virtual function void build();
747         this.pulse = new("pulse");
748         this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
749         if (has_coverage(UVM_CVR_REG_BITS)) begin
750             foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
751         end
752         if (has_coverage(UVM_CVR_FIELD_VALS))
753             fld_cg = new();
754         endfunction : build
755     endclass : sha256_reg_intr_count_incr_t_pulse_d860d977
756
757 // Reg - sha256_reg::intr_count_incr_t_pulse_87b45fe7
758 class sha256_reg_intr_count_incr_t_pulse_87b45fe7 extends
uvm_reg;
759     protected uvm_reg_data_t m_current;
760     protected uvm_reg_data_t m_data;
761     protected bit m_is_read;
762
763     sha256_reg_intr_count_incr_t_pulse_87b45fe7_bit_cg
pulse_bit_cg[1];
764     sha256_reg_intr_count_incr_t_pulse_87b45fe7_fld_cg
fld_cg;
765     rand uvm_reg_field pulse;
766
767     function new(string name =
"sha256_reg_intr_count_incr_t_pulse_87b45fe7");
768         super.new(name, 32, build_coverage(UVM_CVR_ALL));
769     endfunction : new
770     extern virtual function void sample_values();
771     extern protected virtual function void
sample(uvm_reg_data_t data,
772 uvm_reg_data_t byte_en,

```

```

773
bit          is_read,
774
uvm_reg_map  map);
775
776    virtual function void build();
777        this.pulse = new("pulse");
778        this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
779        if (has_coverage(UVM_CVR_REG_BITS)) begin
780            foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
781        end
782        if (has_coverage(UVM_CVR_FIELD_VALS))
783            fld_cg = new();
784        endfunction : build
785    endclass : sha256_reg__intr_count_incr_t_pulse_87b45fe7
786
787    // Reg - sha256_reg::intr_count_incr_t_pulse_c1689ee6
788
789    class sha256_reg__intr_count_incr_t_pulse_c1689ee6 extends
uvm_reg;
790        protected uvm_reg_data_t m_current;
791        protected uvm_reg_data_t m_data;
792        protected bit          m_is_read;
793
794        sha256_reg__intr_count_incr_t_pulse_c1689ee6_bit_cg
pulse_bit_cg[1];
795        sha256_reg__intr_count_incr_t_pulse_c1689ee6_fld_cg
fld_cg;
796        rand uvm_reg_field pulse;
797
798        function new(string name =
"sha256_reg__intr_count_incr_t_pulse_c1689ee6");
799            super.new(name, 32, build_coverage(UVM_CVR_ALL));
800        endfunction : new
801        extern virtual function void sample_values();
802        extern protected virtual function void
sample(uvm_reg_data_t data,
803
uvm_reg_data_t byte_en,
804
bit          is_read,
805
uvm_reg_map  map);
806
807    virtual function void build();
808        this.pulse = new("pulse");
809        this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
810        if (has_coverage(UVM_CVR_REG_BITS)) begin
811            foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
812        end
813        if (has_coverage(UVM_CVR_FIELD_VALS))
814            fld_cg = new();
815        endfunction : build
816    endclass : sha256_reg__intr_count_incr_t_pulse_c1689ee6
817
818    // Reg - sha256_reg::intr_count_incr_t_pulse_6173128e
819
820    class sha256_reg__intr_count_incr_t_pulse_6173128e extends
uvm_reg;
821        protected uvm_reg_data_t m_current;
822        protected uvm_reg_data_t m_data;
823        protected bit          m_is_read;
824
825        sha256_reg__intr_count_incr_t_pulse_6173128e_bit_cg
pulse_bit_cg[1];
826        sha256_reg__intr_count_incr_t_pulse_6173128e_fld_cg
fld_cg;
827        rand uvm_reg_field pulse;
828
829        function new(string name =
"sha256_reg__intr_count_incr_t_pulse_6173128e");
830            super.new(name, 32, build_coverage(UVM_CVR_ALL));
831        endfunction : new
832        extern virtual function void sample_values();
833        extern protected virtual function void
sample(uvm_reg_data_t data,
834
uvm_reg_data_t byte_en,
835
bit          is_read,
836
uvm_reg_map  map);
837
838    virtual function void build();
839        this.pulse = new("pulse");
840        this.pulse.configure(this, 1, 0, "R0", 1, 'h0, 1,
1, 0);
841        if (has_coverage(UVM_CVR_REG_BITS)) begin
842            foreach(pulse_bit_cg[bt]) pulse_bit_cg[bt] =
new();
843        end
844        if (has_coverage(UVM_CVR_FIELD_VALS))
845            fld_cg = new();
846        endfunction : build
847    endclass : sha256_reg__intr_count_incr_t_pulse_6173128e
848
849    // Regfile - sha256_reg::intr_block_t

```

```

848    class sha256_reg__intr_block_t extends uvm_reg_block;
849        rand sha256_reg__global_intr_en_t global_intr_en_r;
850        rand sha256_reg__error_intr_en_t error_intr_en_r;
851        rand sha256_reg__notif_intr_en_t notif_intr_en_r;
852        rand sha256_reg__global_intr_t_agg_sts_dd3dcf0a er-
ror_global_intr_r;
853        rand sha256_reg__global_intr_t_agg_sts_e6399b4a no-
tif_global_intr_r;
854        rand
sha256_reg__error_intr_t_error0_sts_28545624_error1_sts_40e0d3e1_er
ror2_sts_b1cf2205_error3_sts_74a35378 error_internal_intr_r;
855        rand
sha256_reg__notif_intr_t_notif_cmd_done_sts_1c68637e no-
tif_internal_intr_r;
856        rand sha256_reg__error_intr_trig_t error_intr_trig_r;
857        rand sha256_reg__notif_intr_trig_t notif_intr_trig_r;
858        rand sha256_reg__intr_count_t_cnt_35ace267 er-
ror0_intr_count_r;
859        rand sha256_reg__intr_count_t_cnt_73c42c28 er-
ror1_intr_count_r;
860        rand sha256_reg__intr_count_t_cnt_d8af96ff er-
ror2_intr_count_r;
861        rand sha256_reg__intr_count_t_cnt_9bd7f809 er-
ror3_intr_count_r;
862        rand sha256_reg__intr_count_t_cnt_be67d6d5 no-
tif_cmd_done_intr_count_r;
863        rand sha256_reg__intr_count_incr_t_pulse_37026c97 er-
ror0_intr_count_incr_r;
864        rand sha256_reg__intr_count_incr_t_pulse_d860d977 er-
ror1_intr_count_incr_r;
865        rand sha256_reg__intr_count_incr_t_pulse_87b45fe7 er-
ror2_intr_count_incr_r;
866        rand sha256_reg__intr_count_incr_t_pulse_c1689ee6 er-
ror3_intr_count_incr_r;
867        rand sha256_reg__intr_count_incr_t_pulse_6173128e no-
tif_cmd_done_intr_count_incr_r;
868
869        function new(string name =
"sha256_reg__intr_block_t");
870            super.new(name);
871        endfunction : new
872
873        virtual function void build();
874            this.default_map = create_map("reg_map", 0, 4,
UVM_NO_ENDIAN);
875            this.global_intr_en_r = new("global_intr_en_r");
876            this.global_intr_en_r.configure(this);
877
878            this.global_intr_en_r.build();
879            this.default_map.add_reg(this.global_intr_en_r,
'h0);
880            this.error_intr_en_r = new("error_intr_en_r");
881            this.error_intr_en_r.configure(this);
882
883            this.error_intr_en_r.build();
884            this.default_map.add_reg(this.error_intr_en_r,
'h4);
885            this.notif_intr_en_r = new("notif_intr_en_r");
886            this.notif_intr_en_r.configure(this);
887
888            this.notif_intr_en_r.build();
889            this.default_map.add_reg(this.notif_intr_en_r,
'h8);
890            this.error_global_intr_r = new("er-
ror_global_intr_r");
891            this.error_global_intr_r.configure(this);
892
893            this.error_global_intr_r.build();
894
895            this.default_map.add_reg(this.error_global_intr_r,
'hc);
896            this.notif_global_intr_r = new("no-
tif_global_intr_r");
897            this.notif_global_intr_r.configure(this);
898
899            this.notif_global_intr_r.build();
900            this.default_map.add_reg(this.notif_global_intr_r,
'h10);
901            this.error_internal_intr_r = new("er-
ror_internal_intr_r");
902            this.error_internal_intr_r.configure(this);
903
904            this.error_internal_intr_r.build();
905            this.de-
fault_map.add_reg(this.error_internal_intr_r, 'h14);
906            this.notif_internal_intr_r = new("no-
tif_internal_intr_r");
907            this.notif_internal_intr_r.configure(this);
908
909            this.notif_internal_intr_r.build();
910            this.de-
fault_map.add_reg(this.notif_internal_intr_r, 'h18);
911            this.error_intr_trig_r = new("error_intr_trig_r");
912            this.error_intr_trig_r.configure(this);
913
914            this.error_intr_trig_r.build();
915            this.default_map.add_reg(this.error_intr_trig_r,
'h1c);
916            this.notif_intr_trig_r = new("notif_intr_trig_r");
917            this.notif_intr_trig_r.configure(this);

```



```

917
918     this.notif_intr_trig_r.build();
919     this.default_map.add_reg(this.notif_intr_trig_r,
'h20');
920     this.error0_intr_count_r = new("er-
ror0_intr_count_r");
921     this.error0_intr_count_r.configure(this);
922
923     this.error0_intr_count_r.build();
924
925     this.default_map.add_reg(this.error0_intr_count_r,
'h100');
926     this.error1_intr_count_r = new("er-
ror1_intr_count_r");
927     this.error1_intr_count_r.configure(this);
928     this.error1_intr_count_r.build();
929
930     this.default_map.add_reg(this.error1_intr_count_r,
'h104');
931     this.error2_intr_count_r = new("er-
ror2_intr_count_r");
932     this.error2_intr_count_r.configure(this);
933     this.error2_intr_count_r.build();
934
935     this.default_map.add_reg(this.error2_intr_count_r,
'h108');
936     this.error3_intr_count_r = new("er-
ror3_intr_count_r");
937     this.error3_intr_count_r.configure(this);
938     this.error3_intr_count_r.build();
939
940     this.default_map.add_reg(this.error3_intr_count_r,
'h10c');
941     this.notif_cmd_done_intr_count_r = new("no-
tif_cmd_done_intr_count_r");
942     this.notif_cmd_done_intr_count_r.configure(this);
943
944     this.notif_cmd_done_intr_count_r.build();
945     this.de-
fault_map.add_reg(this.notif_cmd_done_intr_count_r, 'h180');
946     this.error0_intr_count_incr_r = new("er-
ror0_intr_count_incr_r");
947     this.error0_intr_count_incr_r.configure(this);
948
949     this.error0_intr_count_incr_r.build();
950     this.de-
fault_map.add_reg(this.error0_intr_count_incr_r, 'h200');
951     this.error1_intr_count_incr_r = new("er-
ror1_intr_count_incr_r");
952     this.error1_intr_count_incr_r.configure(this);
953
954     this.error1_intr_count_incr_r.build();
955     this.de-
fault_map.add_reg(this.error1_intr_count_incr_r, 'h204');
956     this.error2_intr_count_incr_r = new("er-
ror2_intr_count_incr_r");
957     this.error2_intr_count_incr_r.configure(this);
958
959     this.error2_intr_count_incr_r.build();
960     this.de-
fault_map.add_reg(this.error2_intr_count_incr_r, 'h208');
961     this.error3_intr_count_incr_r = new("er-
ror3_intr_count_incr_r");
962     this.error3_intr_count_incr_r.configure(this);
963
964     this.error3_intr_count_incr_r.build();
965     this.de-
fault_map.add_reg(this.error3_intr_count_incr_r, 'h20c');
966     this.notif_cmd_done_intr_count_incr_r = new("no-
tif_cmd_done_intr_count_incr_r");
967     this.no-
tif_cmd_done_intr_count_incr_r.configure(this);
968
969     this.notif_cmd_done_intr_count_incr_r.build();
970     this.de-
fault_map.add_reg(this.notif_cmd_done_intr_count_incr_r, 'h210');
971     endfunction : build
972 endclass : sha256_reg_intr_block_t
973
974 // Addrmap - sha256_reg
975 class sha256_reg extends uvm_reg_block;
976     rand sha256_reg__SHA256_NAME SHA256_NAME[2];
977     rand sha256_reg__SHA256_VERSION SHA256_VERSION[2];
978     rand sha256_reg__SHA256_CTRL SHA256_CTRL;
979     rand sha256_reg__SHA256_STATUS SHA256_STATUS;
980     rand sha256_reg__SHA256_BLOCK SHA256_BLOCK[16];
981     rand sha256_reg__SHA256_DIGEST SHA256_DIGEST[8];
982     rand sha256_reg__intr_block_t intr_block_rf;
983
984     function new(string name = "sha256_reg");
985         super.new(name);
986     endfunction : new
987
988     virtual function void build();
989         this.default_map = create_map("reg_map", 0, 4,
UVM_NO_ENDIAN);
990         foreach(this.SHA256_NAME[i0]) begin
991             this.SHA256_NAME[i0] = new($sform-

```

```

atf("SHA256_NAME[%0d]", i0));
991             this.SHA256_NAME[i0].configure(this);
992
993             this.SHA256_NAME[i0].build();
994
995             this.default_map.add_reg(this.SHA256_NAME[i0],
'h0 + i0*'h4);
996         end
997         foreach(this.SHA256_VERSION[i0]) begin
998             this.SHA256_VERSION[i0] = new($sform-
atf("SHA256_VERSION[%0d]", i0));
999             this.SHA256_VERSION[i0].configure(this);
1000
1001             this.SHA256_VERSION[i0].build();
1002             this.de-
fault_map.add_reg(this.SHA256_VERSION[i0], 'h8 + i0*'h4);
1003         end
1004         this.SHA256_CTRL = new("SHA256_CTRL");
1005         this.SHA256_CTRL.configure(this);
1006
1007         this.SHA256_CTRL.build();
1008         this.default_map.add_reg(this.SHA256_CTRL, 'h10);
1009         this.SHA256_STATUS = new("SHA256_STATUS");
1010         this.SHA256_STATUS.configure(this);
1011
1012         this.SHA256_STATUS.build();
1013         this.default_map.add_reg(this.SHA256_STATUS,
'h18);
1014         foreach(this.SHA256_BLOCK[i0]) begin
1015             this.SHA256_BLOCK[i0] = new($sform-
atf("SHA256_BLOCK[%0d]", i0));
1016             this.SHA256_BLOCK[i0].configure(this);
1017
1018             this.SHA256_BLOCK[i0].build();
1019             this.de-
fault_map.add_reg(this.SHA256_BLOCK[i0], 'h80 + i0*'h4);
1020         end
1021         foreach(this.SHA256_DIGEST[i0]) begin
1022             this.SHA256_DIGEST[i0] = new($sform-
atf("SHA256_DIGEST[%0d]", i0));
1023             this.SHA256_DIGEST[i0].configure(this);
1024
1025             this.SHA256_DIGEST[i0].build();
1026             this.de-
fault_map.add_reg(this.SHA256_DIGEST[i0], 'h100 + i0*'h4);
1027         end
1028         this.intr_block_rf = new("intr_block_rf");
1029         this.intr_block_rf.configure(this);
1030         this.intr_block_rf.build();
1031         this.de-
fault_map.add_submap(this.intr_block_rf.default_map, 'h800);
1032     endfunction : build
1033 endclass : sha256_reg
1034
1035 `include "sha256_reg_sample.svh"
1036 endpackage : sha256_reg_uvm

```

9. chipsalliance_caliptra-rtl/src/sha256/rtl/sha256_w_mem.v (279 lines)

```

1 //
=====
2 //
3 // sha256_w_mem_regs.v
4 // -----
5 // The W memory. This version uses 16 32-bit registers as a
sliding
6 // window to generate the 64 words.
7 //
8 //
9 // Author: Joachim Strombergson
10 // Copyright (c) 2013, Secworks Sweden AB
11 // All rights reserved.
12 //
13 // Redistribution and use in source and binary forms, with or
14 // without modification, are permitted provided that the fol-
lowing
15 // conditions are met:
16 //
17 // 1. Redistributions of source code must retain the above
copyright
18 // notice, this list of conditions and the following dis-
claimer.
19 //
20 // 2. Redistributions in binary form must reproduce the above
copyright
21 // notice, this list of conditions and the following dis-
claimer in
22 // the documentation and/or other materials provided with
the
23 // distribution.
24 //
25 // THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CON-
TRIBUTORS
26 // "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING,
BUT NOT
27 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND
FITNESS
28 // FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL

```

```

THE
29 // COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
INDIRECT,
30 // INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING,
31 // BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SER-
VICES;
32 // LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
HOWEVER
33 // CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT,
34 // STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHER-
WISE)
35 // ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF
36 // ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
37 //
38 //
=====
==
39 module sha256_w_mem(
40     // Clock and reset.
41     input wire          clk,
42     input wire          reset_n,
43     input wire          zeroize,
44     // Data port.
45     input wire [511 : 0] block_msg,
46     // Control.
47     input wire          init_cmd,
48     input wire          next_cmd,
49     output wire [31 : 0] w_val
50 );
51
52
53
54 //-----
55 // Registers including update variables and write enable.
56 //-----
57 reg [31 : 0] w_mem [0 : 15];
58 reg [31 : 0] w_mem00_new;
59 reg [31 : 0] w_mem01_new;
60 reg [31 : 0] w_mem02_new;
61 reg [31 : 0] w_mem03_new;
62 reg [31 : 0] w_mem04_new;
63 reg [31 : 0] w_mem05_new;
64 reg [31 : 0] w_mem06_new;
65 reg [31 : 0] w_mem07_new;
66 reg [31 : 0] w_mem08_new;
67 reg [31 : 0] w_mem09_new;
68 reg [31 : 0] w_mem10_new;
69 reg [31 : 0] w_mem11_new;
70 reg [31 : 0] w_mem12_new;
71 reg [31 : 0] w_mem13_new;
72 reg [31 : 0] w_mem14_new;
73 reg [31 : 0] w_mem15_new;
74 reg          w_mem_we;
75
76 reg [5 : 0] w_ctr_reg;
77 reg [5 : 0] w_ctr_new;
78 reg          w_ctr_we;
79
80
81 //-----
82 // Wires.
83 //-----
84 reg [31 : 0] w_tmp;
85 reg [31 : 0] w_new;
86
87
88 //-----
89 // Concurrent connectivity for ports etc.
90 //-----
91 assign w_val = w_tmp;
92
93
94 //-----
95 // reg_update
96 // Update functionality for all registers in the core.
97 // All registers are positive edge triggered with synchron-
ous
98 // active low reset. All registers have write enable.
99 //-----
100 always @ (posedge clk or negedge reset_n)
101     begin : reg_update
102         integer ii;
103
104         if (!reset_n)
105             begin
106                 for (ii = 0 ; ii < 16 ; ii = ii + 1)
107                     w_mem[ii] <= 32'h0;
108
109                 w_ctr_reg <= 6'h0;
110             end
111         else if (zeroize)
112             begin

```

```

113         for (ii = 0 ; ii < 16 ; ii = ii + 1)
114             w_mem[ii] <= 32'h0;
115
116         w_ctr_reg <= 6'h0;
117     end
118 else
119     begin
120         if (w_mem_we)
121             begin
122                 w_mem[00] <= w_mem00_new;
123                 w_mem[01] <= w_mem01_new;
124                 w_mem[02] <= w_mem02_new;
125                 w_mem[03] <= w_mem03_new;
126                 w_mem[04] <= w_mem04_new;
127                 w_mem[05] <= w_mem05_new;
128                 w_mem[06] <= w_mem06_new;
129                 w_mem[07] <= w_mem07_new;
130                 w_mem[08] <= w_mem08_new;
131                 w_mem[09] <= w_mem09_new;
132                 w_mem[10] <= w_mem10_new;
133                 w_mem[11] <= w_mem11_new;
134                 w_mem[12] <= w_mem12_new;
135                 w_mem[13] <= w_mem13_new;
136                 w_mem[14] <= w_mem14_new;
137                 w_mem[15] <= w_mem15_new;
138             end
139
140         if (w_ctr_we)
141             w_ctr_reg <= w_ctr_new;
142         end
143     end // reg_update
144
145
146 //-----
147 // select_w
148 //
149 // Mux for the external read operation. This is where we ex-
ract
150 // the W variable.
151 //-----
152 always @*
153     begin : select_w
154         if (w_ctr_reg < 16)
155             w_tmp = w_mem[w_ctr_reg[3 : 0]];
156         else
157             w_tmp = w_new;
158         end // select_w
159
160
161 //-----
162 // w_new_logic
163 //
164 // Logic that calculates the next value to be inserted into
165 // the sliding window of the memory.
166 //-----
167 always @*
168     begin : w_mem_update_logic
169         reg [31 : 0] w_0;
170         reg [31 : 0] w_1;
171         reg [31 : 0] w_9;
172         reg [31 : 0] w_14;
173         reg [31 : 0] d0;
174         reg [31 : 0] d1;
175
176         w_mem00_new = 32'h0;
177         w_mem01_new = 32'h0;
178         w_mem02_new = 32'h0;
179         w_mem03_new = 32'h0;
180         w_mem04_new = 32'h0;
181         w_mem05_new = 32'h0;
182         w_mem06_new = 32'h0;
183         w_mem07_new = 32'h0;
184         w_mem08_new = 32'h0;
185         w_mem09_new = 32'h0;
186         w_mem10_new = 32'h0;
187         w_mem11_new = 32'h0;
188         w_mem12_new = 32'h0;
189         w_mem13_new = 32'h0;
190         w_mem14_new = 32'h0;
191         w_mem15_new = 32'h0;
192         w_mem_we = 0;
193
194         w_0 = w_mem[0];
195         w_1 = w_mem[1];
196         w_9 = w_mem[9];
197         w_14 = w_mem[14];
198
199         d0 = {w_1[6 : 0], w_1[31 : 7]} ^
200             {w_1[17 : 0], w_1[31 : 18]} ^
201             {3'b0000, w_1[31 : 3]};
202
203         d1 = {w_14[16 : 0], w_14[31 : 17]} ^
204             {w_14[18 : 0], w_14[31 : 19]} ^
205             {10'b0000000000, w_14[31 : 10]};
206
207         w_new = d1 + w_9 + d0 + w_0;
208

```



```

209   if (init_cmd)
210     begin
211       w_mem00_new = block_msg[511 : 480];
212       w_mem01_new = block_msg[479 : 448];
213       w_mem02_new = block_msg[447 : 416];
214       w_mem03_new = block_msg[415 : 384];
215       w_mem04_new = block_msg[383 : 352];
216       w_mem05_new = block_msg[351 : 320];
217       w_mem06_new = block_msg[319 : 288];
218       w_mem07_new = block_msg[287 : 256];
219       w_mem08_new = block_msg[255 : 224];
220       w_mem09_new = block_msg[223 : 192];
221       w_mem10_new = block_msg[191 : 160];
222       w_mem11_new = block_msg[159 : 128];
223       w_mem12_new = block_msg[127 : 96];
224       w_mem13_new = block_msg[95 : 64];
225       w_mem14_new = block_msg[63 : 32];
226       w_mem15_new = block_msg[31 : 0];
227       w_mem_we = 1;
228     end
229
230   if (next_cmd && (w_ctr_reg > 15))
231     begin
232       w_mem00_new = w_mem[01];
233       w_mem01_new = w_mem[02];
234       w_mem02_new = w_mem[03];
235       w_mem03_new = w_mem[04];
236       w_mem04_new = w_mem[05];
237       w_mem05_new = w_mem[06];
238       w_mem06_new = w_mem[07];
239       w_mem07_new = w_mem[08];
240       w_mem08_new = w_mem[09];
241       w_mem09_new = w_mem[10];
242       w_mem10_new = w_mem[11];
243       w_mem11_new = w_mem[12];
244       w_mem12_new = w_mem[13];
245       w_mem13_new = w_mem[14];
246       w_mem14_new = w_mem[15];
247       w_mem15_new = w_new;

```

```

248       w_mem_we = 1;
249     end
250   end // w_mem_update_logic
251
252   //-----
253   // w_ctr
254   // W schedule adress counter. Counts from 0x10 to 0x3f and
255   // is used to expand the block into words.
256   //-----
257
258   always @*
259     begin : w_ctr
260       w_ctr_new = 6'h0;
261       w_ctr_we = 1'h0;
262
263       if (init_cmd)
264         begin
265           w_ctr_new = 6'h0;
266           w_ctr_we = 1'h1;
267         end
268
269       if (next_cmd)
270         begin
271           w_ctr_new = w_ctr_reg + 6'h01;
272           w_ctr_we = 1'h1;
273         end
274       end // w_ctr
275   endmodule // sha256_w_mem
276
277   //
=====
278   // sha256_w_mem.v
279   //
=====

```

Caliptra HMAC

6 files • 4,099 lines

API — hmac.sv

Table 768. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv — module / interface / package (1개)

이름	종류
hmac	module

Table 769. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv — localparam (4개)

상수	형/폭	값
BLOCK_NUM_DWORDS		BLOCK_SIZE / DATA_WIDTH
KEY_NUM_DWORDS		KEY_SIZE / DATA_WIDTH
TAG_NUM_DWORDS		TAG_SIZE / DATA_WIDTH
SEED_NUM_DWORDS		((LFSR_SEED_SIZE - 1) / DATA_WIDTH) + 1

Table 770. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv — 포트 (18개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cptra_csr_hmac_key	input	logic [CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
cs	input	wire
we	input	wire
address	input	wire [ADDR_WIDTH - 1 : 0]
write_data	input	wire [DATA_WIDTH - 1 : 0]
read_data	output	wire [DATA_WIDTH - 1 : 0]
kv_read	output	kv_read_t [1:0]
kv_write	output	kv_write_t
kv_rd_resp	input	kv_rd_resp_t [1:0]
kv_wr_resp	input	kv_wr_resp_t
busy_o	output	logic
error_intr	output	wire
notif_intr	output	wire
ocp_lock_in_progress	input	logic
debugUnlock_or_scan_mode_switch	input	logic

API — hmac_core.sv**Table 771. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv — module / interface / package (1개)**

이름	종류
hmac_core	module

Table 772. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv — localparam (7개)

상수	형/폭	값
CTRL_IDLE	[2 : 0]	3'd0
CTRL_ENTROPY	[2 : 0]	3'd1
CTRL_IPAD	[2 : 0]	3'd2
CTRL_MSG	[2 : 0]	3'd3
CTRL_OPAD	[2 : 0]	3'd4
CTRL_HMAC	[2 : 0]	3'd5
CTRL_DONE	[2 : 0]	3'd6

Table 773. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv — 포트 (13개)

포트	방향	형/폭
clk	input	logic
reset_n	input	logic
zeroize	input	logic
init_cmd	input	logic
next_cmd	input	logic
last_cmd	input	logic
mode_cmd	input	logic
ready	output	logic
tag_valid	output	logic
lfsr_seed	input	logic [LFSR_SEED_SIZE-1 : 0]
key	input	logic [KEY_SIZE-1 : 0]
block_msg	input	logic [BLOCK_SIZE-1 : 0]
tag	output	logic [TAG_SIZE-1 : 0]

API — hmac_ctrl.sv**Table 774. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv — module / interface / package (1개)**

이름	종류
hmac_ctrl	module

Table 775. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv — parameter (2개)

파라미터	형/폭	기본값
AHB_DATA_WIDTH		32
AHB_ADDR_WIDTH		32

Table 776. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv — 포트 (23개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
cptra_csr_hmac_key	input	logic [CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
haddr_i	input	logic [AHB_ADDR_WIDTH-1:0]
hdata_i	input	logic [AHB_DATA_WIDTH-1:0]
hsel_i	input	logic
hwrite_i	input	logic
hready_i	input	logic
htrans_i	input	logic [1:0]
hsize_i	input	logic [2:0]
hresp_o	output	logic
hreadyout_o	output	logic
hrdata_o	output	logic [AHB_DATA_WIDTH-1:0]
kv_read	output	kv_read_t [1:0]
kv_write	output	kv_write_t
kv_rd_resp	input	kv_rd_resp_t [1:0]
kv_wr_resp	input	kv_wr_resp_t
busy_o	output	logic
error_intr	output	logic
notif_intr	output	logic
ocp_lock_in_progress	input	logic
debugUnlock_or_scan_mode_switch	input	logic

API — hmac_param_pkg.sv

Table 777. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv — module / interface / package (1개)

이름	종류
hmac_param_pkg	package

Table 778. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv — localparam (20개)

[illegible]

Table 779. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
HMAC_PARAM_PKG		package hmac_param_pkg;

API — hmac_reg.sv

Table 780. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg.sv — module / interface / package (17/17)

이름	종류
hmac_reg	module

Table 781. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_cpuif_req	input	wire

포트	방향	형/폭
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [11:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_biten	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire
hwif_in	input	hmac_reg_pkg::hmac_reg__in_t
hwif_out	output	hmac_reg_pkg::hmac_reg__out_t

API — hmac_reg_pkg.sv

Table 782. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg_pkg.sv — module / interface / package (1개)

이름	종류
hmac_reg_pkg	package

Table 783. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg_pkg.sv — localparam (3개)

상수	형/폭	값
HMAC_REG_DATA_WIDTH		32
HMAC_REG_MIN_ADDR_WIDTH		12
HMAC_REG_ADDR_WIDTH		32'd12

1. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac.sv (583 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // hmac.sv
18 // -----
19 // HMAC-512/384 top-level wrapper with 32 bit data access.
20 //
21 //

```

```

=====
22 //`include "kv_defines.svh"
23 `include "caliptra_macros.svh"
24 `include "kv_macros.svh"
25 `include "caliptra_prim_assert.sv"
26
27 module hmac
28     import hmac_param_pkg::*;
29     import hmac_reg_pkg::*;
30     import kv_defines_pkg::*;
31     #(
32         ADDR_WIDTH = 32,
33         DATA_WIDTH = 32
34     )
35     // Clock and reset.
36     input wire          clk,
37     input wire          reset_n,
38     input wire          cptra_pwrgood,
39
40     //csr key
41     input logic [`CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
cptra_csr_hmac_key,
42
43     // Control.
44     input wire          cs,
45     input wire          we,
46
47     // Data ports.
48     input wire [ADDR_WIDTH - 1 : 0] address,
49     input wire [DATA_WIDTH - 1 : 0] write_data,
50     output wire [DATA_WIDTH - 1 : 0] read_data,

```

```

51
52 // KV interface
53 output kv_read_t [1:0] kv_read,
54 output kv_write_t kv_write,
55 input kv_rd_resp_t [1:0] kv_rd_resp,
56 input kv_wr_resp_t kv_wr_resp,
57
58 output logic busy_o,
59
60 output wire error_intr,
61 output wire notif_intr,
62 input logic ocp_lock_in_progress,
63 input logic debugUnlock_or_scan_mode_switch
64 );
65
66 //-----
67 // Registers including update variables and write enable.
68 //-----
69 reg init_reg;
70 reg next_reg;
71 reg last_reg;
72
73 reg mode_reg;
74
75 reg ready_reg;
76 reg tag_valid_reg;
77
78 localparam BLOCK_NUM_DWORDS = BLOCK_SIZE / DATA_WIDTH;
79 localparam KEY_NUM_DWORDS = KEY_SIZE / DATA_WIDTH;
80 localparam TAG_NUM_DWORDS = TAG_SIZE / DATA_WIDTH;
81 localparam SEED_NUM_DWORDS = ((LFSR_SEED_SIZE - 1) /
DATA_WIDTH) + 1;
82
83 reg [KEY_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0]
key_reg;
84 reg [BLOCK_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0]
block_reg;
85 reg [SEED_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0]
lfsr_seed_reg;
86
87 logic zeroize_reg;
88
89 //-----
90 // Wires.
91 //-----
92 wire [KEY_SIZE-1 : 0] core_key;
93 wire [BLOCK_SIZE-1 : 0] core_block;
94 wire core_ready;
95 wire [TAG_SIZE-1 : 0] core_tag;
96 wire core_tag_valid;
97 wire [LFSR_SEED_SIZE-1 : 0] core_lfsr_seed;
98 reg [TAG_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0] tag_reg;
99 reg [TAG_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0] kv_reg;
100
101 reg [TAG_NUM_DWORDS - 1 : 0][DATA_WIDTH - 1 : 0] get_mask;
102
103 hmac_reg_in_t hwif_in;
104 hmac_reg_out_t hwif_out;
105
106 //interface with kv client
107 logic kv_key_write_en;
108 logic [3:0] kv_key_write_offset;
109 logic [31:0] kv_key_write_data;
110 logic kv_block_write_en;
111 logic [4:0] kv_block_write_offset;
112 logic [31:0] kv_block_write_data;
113 //KV Read Data Present
114 logic kv_key_data_present, kv_key_data_present_set;
115 logic [BLOCK_NUM_DWORDS-1:0] block_reg_lock,
block_reg_lock_nxt;
116 logic kv_block_data_present, kv_block_data_present_set;
117 logic kv_data_present, kv_data_present_reset;
118
119 logic dest_keyvault;
120 kv_error_code_e kv_key_error, kv_block_error,
kv_write_error;
121 logic kv_key_ready, kv_key_done;
122 logic kv_block_ready, kv_block_done;
123 logic kv_write_ready, kv_write_done;
124 logic [KV_ENTRY_SIZE_W-1:0] hmac_expected_key_size;
125
126 kv_read_ctrl_reg_t kv_key_read_ctrl_reg;
127 kv_read_ctrl_reg_t kv_block_read_ctrl_reg;
128 kv_write_ctrl_reg_t kv_write_ctrl_reg;
129 kv_read_filter_metrics_t kv_key_read_metrics;
130 kv_read_filter_metrics_t kv_block_read_metrics;
131 kv_write_filter_metrics_t kv_write_metrics;
132 logic core_tag_we;
133
134 logic key_zero_error, key_zero_error_reg,
key_zero_error_edge;
135 logic key_mode_error, key_mode_error_reg,
key_mode_error_edge;
136 logic last_alone_error, last_alone_error_reg,
last_alone_error_edge;
137
138 logic error_flag;

139 logic error_flag_reg;
140 logic error_flag_edge;
141 //-----
142 // Concurrent connectivity for ports etc.
143 //-----
144 assign core_block = {block_reg[00], block_reg[01],
block_reg[02], block_reg[03],
145 block_reg[04], block_reg[05],
block_reg[06], block_reg[07],
146 block_reg[08], block_reg[09],
block_reg[10], block_reg[11],
147 block_reg[12], block_reg[13],
block_reg[14], block_reg[15],
148 block_reg[16], block_reg[17],
block_reg[18], block_reg[19],
149 block_reg[20], block_reg[21],
block_reg[22], block_reg[23],
150 block_reg[24], block_reg[25],
block_reg[26], block_reg[27],
151 block_reg[28], block_reg[29],
block_reg[30], block_reg[31]};
152
153 assign core_key = {key_reg[00], key_reg[01], key_reg[02],
key_reg[03], key_reg[04], key_reg[05],
154 key_reg[06], key_reg[07], key_reg[08],
key_reg[09], key_reg[10], key_reg[11],
155 key_reg[12], key_reg[13], key_reg[14],
key_reg[15]} & get_mask;
156
157 assign core_lfsr_seed = {lfsr_seed_reg[00],
lfsr_seed_reg[01], lfsr_seed_reg[02],
158 lfsr_seed_reg[03], lfsr_seed_reg[04], lfsr_seed_reg[05]};
159
160 //rising edge detect on core tag valid
161 assign core_tag_we = (core_tag_valid & ~tag_valid_reg) &
~error_flag_reg;
162
163 //-----
164 // core instantiation.
165 //-----
166 hmac_core core(
167 .clk(clk),
168 .reset_n(reset_n),
169 .zeroize(zeroize_reg),
170
171 .init_cmd(init_reg),
172 .next_cmd(next_reg),
173 .last_cmd(last_reg),
174 .mode_cmd(mode_reg),
175
176 .lfsr_seed(core_lfsr_seed),
177
178 .key(core_key),
179
180 .block_msg(core_block),
181
182 .ready(core_ready),
183 .tag(core_tag),
184 .tag_valid(core_tag_valid)
185 );
186
187 //-----
188 // reg_update
189 //
190 // Update functionality for all registers in the core.
191 // All registers are positive edge triggered with asynchron-
ous
192 // active low reset. All registers have write enable.
193 //-----
194
195 always @ (posedge clk or negedge reset_n)
196 begin : reg_update
197 integer ii;
198
199 if (!reset_n)
200 begin
201 kv_reg <= '0;
202 tag_reg <= '0;
203 tag_valid_reg <= '0;
204 ready_reg <= '0;
205 block_reg_lock <= '0;
206 kv_key_data_present <= '0;
207 kv_block_data_present <= '0;
208 end
209 else if (zeroize_reg)
210 begin
211 kv_reg <= '0;
212 tag_reg <= '0;
213 tag_valid_reg <= '0;
214 ready_reg <= '0;
215 block_reg_lock <= '0;
216 kv_key_data_present <= '0;
217 kv_block_data_present <= '0;
218 end
end

```



```

219     else
220     begin
221         tag_valid_reg <= core_tag_valid & ~error_flag_reg;
222         ready_reg <= core_ready;
223
224         //write to sw register
225         if (core_tag_we & ~(dest_keyvault |
kv_data_present))
226             tag_reg <= core_tag & get_mask;
227         if (core_tag_we & (dest_keyvault | kv_data_present))
228             kv_reg <= core_tag & get_mask;
229
230         block_reg_lock <= block_reg_lock_nxt;
231         kv_key_data_present <= kv_key_data_present_set ?
'1 :
232             kv_data_present_reset ? '0 :
kv_key_data_present;
233         kv_block_data_present <= kv_block_data_present_set ?
'1 :
234             kv_data_present_reset ?
'0 : kv_block_data_present;
235     end
236     end // reg_update
237
238 //HMAC register hardware interfaces
239 always_comb begin
240     //drive resets to register block
241     hwif_in.error_reset_b = cptra_pwrgood;
242     hwif_in.reset_b = reset_n;
243     //drive hardware writeable registers from hmac core
244     hwif_in.HMAC512_NAME[0].NAME.next = HMAC_CORE_NAME[31:0];
245     hwif_in.HMAC512_NAME[1].NAME.next = HMAC_CORE_NAME[63:32];
246     hwif_in.HMAC512_VERSION[0].VERSION.next =
HMAC_CORE_VERSION[31:0];
247     hwif_in.HMAC512_VERSION[1].VERSION.next =
HMAC_CORE_VERSION[63:32];
248     //Only allow setting CSR MODE when ready is set
249     hwif_in.HMAC512_CTRL.CSR_MODE.swwe = ready_reg;
250     hwif_in.HMAC512_CTRL.INIT.swwe = ready_reg;
251     hwif_in.HMAC512_CTRL.NEXT.swwe = ready_reg;
252     hwif_in.HMAC512_CTRL.LAST.swwe = ready_reg;
253     hwif_in.HMAC512_CTRL.MODE.swwe = ready_reg;
254
255     //assign hardware readable registers to drive hmac core
256     //mask the command until kv clients are idle
257     init_reg = hwif_out.HMAC512_CTRL.INIT.value & kv_key_ready &
kv_block_ready;
258     next_reg = hwif_out.HMAC512_CTRL.NEXT.value & kv_key_ready &
kv_block_ready;
259     last_reg = hwif_out.HMAC512_CTRL.LAST.value & kv_key_ready &
kv_block_ready;
260     zeroize_reg = hwif_out.HMAC512_CTRL.ZEROIZE.value || debu-
gunlock_or_scan_mode_switch;
261     mode_reg = hwif_out.HMAC512_CTRL.MODE.value;
262
263     //drive hardware writeable registers from hmac core
264     hwif_in.HMAC512_STATUS.READY.next = ready_reg;
265     hwif_in.HMAC512_STATUS.VALID.next = tag_valid_reg;
266     for (int dword=0; dword < TAG_NUM_DWORDS; dword++) begin
267         hwif_in.HMAC512_TAG[dword].TAG.next = tag_reg[(TAG_NUM_D-
WORDS - 1)-dword];
268         hwif_in.HMAC512_TAG[dword].TAG.hwclr = zeroize_reg;
269     end
270     //drive hardware writable registers from key vault
271     for (int dword=0; dword < BLOCK_NUM_DWORDS; dword++)begin
272         hwif_in.HMAC512_BLOCK[dword].BLOCK.we = (kv_block_write_en &
(kv_block_write_offset == dword)) & !(zeroize_reg |
kv_data_present_reset);
273         hwif_in.HMAC512_BLOCK[dword].BLOCK.next = kv_b-
lock_write_data;
274         hwif_in.HMAC512_BLOCK[dword].BLOCK.hwclr = zeroize_reg |
kv_data_present_reset | (kv_block_error != KV_SUCCESS);
275         hwif_in.HMAC512_BLOCK[dword].BLOCK.swwe1 =
block_reg_lock[dword];
276     end
277     for (int dword=0; dword < KEY_NUM_DWORDS; dword++)begin
278         hwif_in.HMAC512_KEY[dword].KEY.we = (kv_key_write_en &
(kv_key_write_offset == dword)) & !(zeroize_reg |
kv_data_present_reset);
279         hwif_in.HMAC512_KEY[dword].KEY.next = kv_key_write_data;
280         hwif_in.HMAC512_KEY[dword].KEY.hwclr = zeroize_reg |
kv_data_present_reset | (kv_key_error != KV_SUCCESS);
281         hwif_in.HMAC512_KEY[dword].KEY.swwe1 =
kv_key_data_present;
282     end
283     //set ready when keyvault isn't busy
284     hwif_in.HMAC512_KV_RD_KEY_STATUS.READY.next = kv_key_ready;
285     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.READY.next = kv_b-
lock_ready;
286     hwif_in.HMAC512_KV_WR_STATUS.READY.next = kv_write_ready;
287     //set error code
288     hwif_in.HMAC512_KV_RD_KEY_STATUS.ERROR.next = kv_key_error;
289     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.ERROR.next = kv_b-
lock_error;
290     hwif_in.HMAC512_KV_WR_STATUS.ERROR.next = kv_write_error;
291     //set valid when fsm is done

```

```

292     hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwset = kv_key_done;
293     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwset = kv_b-
lock_done;
294     hwif_in.HMAC512_KV_WR_STATUS.VALID.hwset = kv_write_done;
295     //clear valid when new request is made
296     hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwclr =
kv_key_read_ctrl_reg.read_en;
297     hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwclr = kv_b-
block_read_ctrl_reg.read_en;
298     hwif_in.HMAC512_KV_WR_STATUS.VALID.hwclr =
kv_write_ctrl_reg.write_en;
299     //clear enable when busy
300     hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.hwclr =
~kv_key_ready;
301     hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.hwclr = ~kv_b-
block_ready;
302     hwif_in.HMAC512_KV_WR_CTRL.write_en.hwclr = ~kv_write_ready;
303     //assign hardware readable registers to drive hmac core
304     for (int dword=0; dword < KEY_NUM_DWORDS; dword++) begin
305         key_reg[dword] = hwif_out.HMAC512_CTRL.CSR_MODE.value ?
cptra_csr_hmac_key[dword] : hwif_out.HMAC512_KEY[dword].KEY.value;
306     end
307     for (int dword=0; dword < BLOCK_NUM_DWORDS; dword++)begin
308         block_reg[dword] =
hwif_out.HMAC512_BLOCK[dword].BLOCK.value;
309     end
310
311     for (int dword=0; dword < SEED_NUM_DWORDS; dword++)begin
312         lfsr_seed_reg[dword] =
hwif_out.HMAC512_LFSR_SEED[dword].LFSR_SEED.value;
313     end
314 end
315
316 always_comb begin
317     unique case (mode_reg)
318         'b0 : get_mask = {(HMAC384_TAG_SIZE/DATA_WIDTH)
{ DATA_WIDTH{1'b1} } }},
319             {(HMAC384_TAG_PAD /DATA_WIDTH)
{ DATA_WIDTH{1'b0} } } }; //HMAC384
320         default : get_mask = {TAG_NUM_D-
WORDS{ DATA_WIDTH{1'b1} } } }; //HMAC512
321     endcase
322 end
323
324 //set the lock for the part of the block being written by KV
logic
325 //release the lock once init has been seen
326 always_comb begin
327     for (int dword=0; dword< BLOCK_NUM_DWORDS; dword++) begin
328         if (init_reg | next_reg) begin
329             block_reg_lock_nxt[dword] = '0;
330         end
331         else begin
332             block_reg_lock_nxt[dword] = (kv_block_write_en & (kv_b-
lock_write_offset == dword)) ? '1 : block_reg_lock[dword];
333         end
334     end
335 end
336
337 // Software write-enables to prevent KV reg manipulation mid-
operation
338 always_comb
339     hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.swwe = !
kv_key_data_present && !busy_o;
340 always_comb
341     hwif_in.HMAC512_KV_RD_KEY_CTRL.read_entry.swwe = !
kv_key_data_present && !busy_o;
342 always_comb
343     hwif_in.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.swwe = 0; //NA for
keyvault
344 always_comb
345     hwif_in.HMAC512_KV_RD_KEY_CTRL.rsvd.swwe = 0;
346 always_comb
347     hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.swwe = !kv_b-
block_data_present && !busy_o;
348 always_comb
349     hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_entry.swwe = !kv_b-
block_data_present && !busy_o;
350 always_comb
351     hwif_in.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.swwe = 0; //NA for
keyvault
352 always_comb
353     hwif_in.HMAC512_KV_RD_BLOCK_CTRL.rsvd.swwe = 0;
354
355 // KV write control must be written before HMAC core operation
begins, even though
356 // output isn't written to KV until the end of the operation.
357 // Prevent partial-key attacks by blocking register modifica-
tions during core execution.
358 always_comb
359     hwif_in.HMAC512_KV_WR_CTRL.write_en.swwe = !busy_o;
360 always_comb
361     hwif_in.HMAC512_KV_WR_CTRL.write_entry.swwe = !busy_o;
362 always_comb
363     hwif_in.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.swwe = !busy_o;
364 always_comb
365     hwif_in.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.swwe = !busy_o;
366 always_comb
367     hwif_in.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.swwe = !busy_o;

```

```

356 always_comb
hwif_in.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe = !busy_o;
357 always_comb
hwif_in.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.swwe = !busy_o;
358 always_comb
hwif_in.HMAC512_KV_WR_CTRL.aes_key_dest_valid.swwe = !busy_o;
359 always_comb
hwif_in.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.swwe = !busy_o;
360 always_comb
hwif_in.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.swwe = !busy_o;
361 always_comb
hwif_in.HMAC512_KV_WR_CTRL.dma_data_dest_valid.swwe = !busy_o;
362 always_comb
hwif_in.HMAC512_KV_WR_CTRL.rsvd.swwe = 0;
363
364 //keyvault control reg macros for assigning to struct
365 `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_key_read_ctrl_reg,
HMAC512_KV_RD_KEY_CTRL)
366 `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_block_read_ctrl_reg,
HMAC512_KV_RD_BLOCK_CTRL)
367 `CALIPTRA_KV_WRITE_CTRL_REG2STRUCT(kv_write_ctrl_reg,
HMAC512_KV_WR_CTRL)
368
369 //Force result into KV reg whenever source came from KV
370 always_comb kv_key_data_present_set =
kv_key_read_ctrl_reg.read_en;
371 always_comb kv_block_data_present_set = kv_b-
lock_read_ctrl_reg.read_en;
372 always_comb kv_data_present = kv_key_data_present | kv_b-
lock_data_present;
373 always_comb kv_data_present_reset = kv_data_present &
core_tag_we;
374
375 // Register block
376 hmac_reg i_hmac_reg (
377     .clk(clk),
378     .rst(1'b0),
379
380     .s_cpuif_req      (cs),
381     .s_cpuif_req_is_wr (we),
382     .s_cpuif_addr     (address[HMAC_REG_ADDR_WIDTH-1:0]),
383     .s_cpuif_wr_data   (write_data),
384     .s_cpuif_wr_bitlen ('1'),
385     .s_cpuif_req_stall_wr(),
386     .s_cpuif_req_stall_rd(),
387     .s_cpuif_rd_ack    (),
388     .s_cpuif_rd_err    (),
389     .s_cpuif_rd_data   (read_data),
390     .s_cpuif_wr_ack    (),
391     .s_cpuif_wr_err    (),
392
393     .hwif_in (hwif_in),
394     .hwif_out(hwif_out)
395 );
396
397 always_comb key_mode_error = kv_key_data_present & (init_reg |
next_reg) & (mode_reg == HMAC512_MODE) & (key_reg[KEY_NUM_DWORDS-1 :
HMAC384_KEY_SIZE/DATA_WIDTH] == '0);
398 always_comb key_zero_error = kv_key_data_present & (init_reg |
next_reg) & (key_reg == '0);
399 always_comb last_alone_error =
hwif_out.HMAC512_CTRL.LAST.value & ~hwif_out.HMAC512_CTRL.INIT.value
& ~hwif_out.HMAC512_CTRL.NEXT.value;
400
401 // last_alone_error is a soft notification: the engine never
started,
402 // so it must NOT join error_flag (which gates core_tag_we and
is
403 // sticky until zeroize). Keep it for the interrupt status bit
only.
404 always_comb error_flag = key_zero_error | key_mode_error |
(kv_key_error == KV_RD_LEN_MISMATCH);
405
406 always_ff @(posedge clk or negedge reset_n)
begin : error_detection
407     if(!reset_n) begin
408         error_flag_reg <= 1'b0;
409         key_mode_error_reg <= 1'b0;
410         key_zero_error_reg <= 1'b0;
411         last_alone_error_reg <= 1'b0;
412     end
413     else if(zeroize_reg) begin
414         error_flag_reg <= 1'b0;
415         key_mode_error_reg <= 1'b0;
416         key_zero_error_reg <= 1'b0;
417         last_alone_error_reg <= 1'b0;
418     end
419     else begin
420         if (error_flag)
421             error_flag_reg <= 1'b1;
422         key_mode_error_reg <= key_mode_error;
423         key_zero_error_reg <= key_zero_error;
424         last_alone_error_reg <= last_alone_error;
425     end
426 end // error_detection
427
428
429 always_comb error_flag_edge = error_flag & (!error_flag_reg);

```

```

430 always_comb key_mode_error_edge = key_mode_error & (!
key_mode_error_reg);
431 always_comb key_zero_error_edge = key_zero_error & (!
key_zero_error_reg);
432 always_comb last_alone_error_edge = last_alone_error & (!
last_alone_error_reg);
433
434 //Interrupts hardware interface
435 assign hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset =
core_tag_we;
436 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_mode_error_sts.hwset =
key_mode_error_edge;
437 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_zero_error_sts.hwset =
key_zero_error_edge;
438 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset =
last_alone_error_edge;
439 assign hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset = 1'b0; // TODO
440
441 assign error_intr = hwif_out.intr_b-
lock_rf.error_global_intr_r.intr;
442 assign notif_intr = hwif_out.intr_b-
block_rf.notif_global_intr_r.intr;
443
444 //Key Vault Control Modules
445 always_comb begin
446     kv_key_read_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
447     kv_key_read_metrics.kv_read_dest =
KV_NUM_READ'(1<<KV_DEST_IDX_HMAC_KEY);
448     kv_key_read_metrics.kv_key_entry =
kv_key_read_ctrl_reg.read_entry;
449 end
450
451 //Read Key
452 kv_read_client #(
453     .DATA_WIDTH(KEY_SIZE),
454     .PAD(0),
455     .LEN_CHECK_AT_KEY_USE(1) //HMAC mode_reg (- expected
length) may be
456                                     //programmed after the KV read com-
pletes.
457 )
458 hmac_key_kv_read
459 (
460     .clk(clk),
461     .rst_b(reset_n),
462     .zeroize(zeroize_reg),
463
464     //client control register
465     .read_ctrl_reg(kv_key_read_ctrl_reg),
466     .read_metrics (kv_key_read_metrics ),
467
468     //interface with kv
469     .kv_read(kv_read[0]),
470     .kv_resp(kv_rd_resp[0]),
471
472     //interface with client
473     .write_en(kv_key_write_en),
474     .write_offset(kv_key_write_offset),
475     .write_data(kv_key_write_data),
476
477     .error_code(kv_key_error),
478     .kv_ready(kv_key_ready),
479     .read_done(kv_key_done),
480     .check_key_size((init_reg | next_reg) &
kv_key_data_present),
481     .expected_key_size(hmac_expected_key_size)
482 );
483
484 //KV key-length-mismatch expected value (HMAC mode is runtime-
selectable).
485 always_comb hmac_expected_key_size = (mode_reg ==
HMAC512_MODE) ? KV_ENTRY_SIZE_W'(HMAC512_KV_LAST_DWORD)
486
487 : KV_ENTRY_SIZE_W'(HMAC384_KV_LAST_DWORD);
488
489 //Key Vault Control Modules
490 always_comb begin
491     kv_block_read_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
492     kv_block_read_metrics.kv_read_dest =
KV_NUM_READ'(1<<KV_DEST_IDX_HMAC_BLOCK);
493     kv_block_read_metrics.kv_key_entry = kv_b-
lock_read_ctrl_reg.read_entry;
494 end
495
496 //Read Block
497 // Length check disabled on the BLOCK path: the HMAC block is
a message chunk,
498 // not a security-sized key. Legitimate consumers (e.g., OCP
LOCK HEK seed,
499 // OCP_LOCK_HEK_NUM_DWORDS=8) may write KV entries shorter
than the mode's
500 // key size and rely on PAD=1 zero-extension.
501 kv_read_client #(

```

```

501 .DATA_WIDTH(BLOCK_SIZE),
502 .HMAC(1),
503 .PAD(1),
504 .LEN_CHECK(0)
505 )
506 hmac_block_kv_read
507 (
508     .clk(clk),
509     .rst_b(reset_n),
510     .zeroize(zeroize_reg),
511
512     //client control register
513     .read_ctrl_reg(kv_block_read_ctrl_reg),
514     .read_metrics(kv_block_read_metrics),
515
516     //interface with kv
517     .kv_read(kv_read[1]),
518     .kv_resp(kv_rd_resp[1]),
519
520     //interface with client
521     .write_en(kv_block_write_en),
522     .write_offset(kv_block_write_offset),
523     .write_data(kv_block_write_data),
524
525     .error_code(kv_block_error),
526     .kv_ready(kv_block_ready),
527     .read_done(kv_block_done),
528     .check_key_size(1'b0),
529     .expected_key_size('0)
530 );
531
532 //write 512 or 384 result based on mode bit
533 logic [$clog2(TAG_SIZE/32):0] num_dwords;
534 always_comb num_dwords = mode_reg ? 'd16 : 'd12;
535
536 //Write to keyvault
537 always_comb begin
538     kv_write_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
539     kv_write_metrics.kv_data0_present =
kv_key_data_present;
540     kv_write_metrics.kv_data0_entry =
kv_key_read_ctrl_reg.read_entry; // Stable after start-time due to
SWWE
541     kv_write_metrics.kv_data1_present = kv_b-
lock_data_present;
542     kv_write_metrics.kv_data1_entry = kv_b-
lock_read_ctrl_reg.read_entry; // Stable after start-time due to
SWWE
543     kv_write_metrics.kv_write_src = KV_NUM_WRITE'(1 <<
KV_WRITE_IDX_HMAC);
544     kv_write_metrics.kv_write_entry =
kv_write_ctrl_reg.write_entry;
545     kv_write_metrics.aes_decrypt_ecb_op = 1'b0;
546 end
547
548 kv_write_client #(
549     .DATA_WIDTH(TAG_SIZE)
550 )
551 hmac_result_kv_write
552 (
553     .clk(clk),
554     .rst_b(reset_n),
555     .zeroize(zeroize_reg),
556
557     //client control register
558     .write_ctrl_reg(kv_write_ctrl_reg),
559     .write_metrics(kv_write_metrics),
560     .num_dwords(num_dwords),
561
562     //interface with kv
563     .kv_write(kv_write),
564     .kv_resp(kv_wr_resp),
565
566     //interface with client
567     .dest_keyvault(dest_keyvault),
568     .dest_data_avail(core_tag_we),
569     .dest_data(kv_reg),
570
571     .error_code(kv_write_error),
572     .kv_ready(kv_write_ready),
573     .dest_done(kv_write_done)
574 );
575
576 always_comb busy_o = ~kv_write_ready | ~kv_block_ready |
~kv_key_ready | ~core_ready;
577
578
579 endmodule // hmac
580
581 //
=====
582 // EOF hmac.sv
583 //
=====

```

2. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_core.sv (447 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // hmac_core.sv
18 // -----
19 // HMAC-SHA-512/384 core with SCA countermeasures.
20 // Single SHA-512 masked core architecture for area optimiza-
tion.
21 //
22 // Commands:
23 //   init_cmd - SHA-init with (K xor ipad), then SHA-next
with first
24 //               message block. If issued together with
last_cmd,
25 //               continues directly to inner-digest capture,
OPAD, and
26 //               HMAC finalization.
27 //   next_cmd - SHA-next with one intermediate message block.
28 //   last_cmd - SHA-next with the final message block, cap-
tures the
29 //               inner digest, then SHA-init with (K xor opad)
and
30 //               SHA-next with {inner_digest, FINAL_PAD} to
produce
31 //               the HMAC tag.
32 //
33 //
=====
34
35 module hmac_core
36     import hmac_param_pkg::*;
37 (
38     // Clock and reset.
39     input logic clk,
40     input logic reset_n,
41     input logic zeroize,
42
43     // Control.
44     input logic init_cmd,
45     input logic next_cmd,
46     input logic last_cmd,
47     input logic mode_cmd,
48     output logic ready,
49     output logic tag_valid,
50
51     // Data ports.
52     input logic [LFSR_SEED_SIZE-1 : 0] lfsr_seed,
53
54     input logic [KEY_SIZE-1 : 0] key,
55     input logic [BLOCK_SIZE-1 : 0] block_msg,
56     output logic [TAG_SIZE-1 : 0] tag
57 );
58
59 //-----
60 // Internal constant and parameter definitions.
61 //-----
62
63 // IPAD/OPAD and SHA-512 family final-padding constants come
from
64 // hmac_param_pkg so the same widths/values are used every-
where.
65
66 localparam [2 : 0] CTRL_IDLE = 3'd0;
67 localparam [2 : 0] CTRL_ENTROPY = 3'd1;
68 localparam [2 : 0] CTRL_IPAD = 3'd2;
69 localparam [2 : 0] CTRL_MSG = 3'd3;
70 localparam [2 : 0] CTRL_OPAD = 3'd4;
71 localparam [2 : 0] CTRL_HMAC = 3'd5;
72 localparam [2 : 0] CTRL_DONE = 3'd6;
73

```

```

74
75 //-----
76 // Registers including update variables and write enable.
77 //-----
78 logic [2 : 0] hmac_ctrl_reg;
79 logic [2 : 0] hmac_ctrl_new;
80 logic         hmac_ctrl_we;
81 logic [2 : 0] hmac_ctrl_ff;
82
83 logic         ready_flag;
84 logic         digest_valid_reg;
85 logic         digest_valid_new;
86 logic         digest_valid_we;
87
88 logic [BLOCK_SIZE-1:0] key_ipadded;
89 logic [BLOCK_SIZE-1:0] key_opadded;
90 logic [BLOCK_SIZE-1:0] HMAC_padded;
91
92 logic         first_round;
93 logic         IPAD_ready;
94 logic         MSG_ready;
95 logic         OPAD_ready;
96 logic         HMAC_ready;
97 logic         ENTROPY_ready;
98 logic [1:0]    mode_reg;
99
100 // Latched on entry to MSG flow: 1 if the current message
101 // block is the final one (so MSG must capture inner_digest and continue
102 // to OPAD).
103 logic         is_last_block;
104
105 // Captured inner-hash digest at the final MSG -> OPAD
106 // transition; used
107 // as data into the HMAC finalization block.
108 logic [TAG_SIZE-1:0] inner_digest_reg;
109 logic         inner_digest_we;
110
111 logic         sha_init;
112 logic         sha_next;
113 logic [BLOCK_SIZE-1 : 0] sha_block;
114 logic         sha_ready;
115 logic [TAG_SIZE-1 : 0] sha_digest;
116 logic         sha_digest_valid;
117
118 // Entropy.
119 logic [LFSR_SEED_SIZE-1 : 0] entropy;
120 logic [LFSR_SEED_SIZE-1 : 0] lfsr_entropy;
121 logic [LFSR_SEED_SIZE-1 : 0] entropy_digest;
122 logic [63 : 0] counter_reg;
123 logic [BLOCK_SIZE-1 : 0] entropy_block;
124 logic         set_entropy;
125
126 //-----
127 // Concurrent connectivity for ports etc.
128 //-----
129 assign ready      = ready_flag;
130 assign tag        = digest_valid_reg ? sha_digest : {TAG_S-
131 IZE(1'b0)};
132 assign tag_valid  = digest_valid_reg;
133
134 //-----
135 // core instantiation.
136 //-----
137 sha512_masked_core u_sha512_core_h1
138 (
139     .clk(clk),
140     .reset_n(reset_n),
141     .zeroize(zeroize),
142     .init_cmd(sha_init),
143     .next_cmd(sha_next),
144     .mode(mode_reg),
145     .entropy(entropy),
146     .block_msg(sha_block),
147     .ready(sha_ready),
148     .digest(sha_digest),
149     .digest_valid(sha_digest_valid)
150 );
151
152 // One 32-bit LFSR per dword of entropy (LFSR_SEED_SIZE bits
153 // total).
154 genvar i;
155 generate
156     for (i = 0; i < LFSR_SEED_SIZE/32; i++) begin : gen_lfsr
157         caliptra_prim_lfsr
158             #(
159                 .LfsrType("FIB_XNOR"),
160                 .LfsrDw(32),
161                 .StateOutDw(32)
162
163             ) caliptra_prim_lfsr_inst_i
164         (
165             .clk_i(clk),
166             .rst_ni(reset_n),
167             .seed_en_i(init_cmd),
168             .seed_i(lfsr_entropy[i*32 +: 32]),
169             .lfsr_en_i(1'b1),
170             .entropy_i('0),
171             .state_o(entropy[i*32 +: 32])
172         );
173     end
174 endgenerate
175
176 //-----
177 // reg_update
178 //
179 // Update functionality for all registers in the core.
180 // All registers are positive edge triggered with asynchron-
181 // ous
182 // active low reset. All registers have write enable.
183 //-----
184 always_ff @(posedge clk or negedge reset_n)
185     begin : reg_update
186         if (!reset_n)
187             begin
188                 digest_valid_reg <= 1'b0;
189                 hmac_ctrl_reg <= CTRL_IDLE;
190                 hmac_ctrl_ff <= CTRL_IDLE;
191                 inner_digest_reg <= '0;
192                 is_last_block <= 1'b0;
193             end
194         else if (zeroize)
195             begin
196                 digest_valid_reg <= 1'b0;
197                 hmac_ctrl_reg <= CTRL_IDLE;
198                 hmac_ctrl_ff <= CTRL_IDLE;
199                 inner_digest_reg <= '0;
200                 is_last_block <= 1'b0;
201             end
202         else
203             begin
204                 hmac_ctrl_ff <= hmac_ctrl_reg;
205                 if (digest_valid_we)
206                     digest_valid_reg <= digest_valid_new;
207                 if (hmac_ctrl_we)
208                     hmac_ctrl_reg <= hmac_ctrl_new;
209                 if (inner_digest_we)
210                     inner_digest_reg <= sha_digest;
211             end
212         // Sample at IDLE: LAST is a modifier on INIT or
213         NEXT, so any
214         // command kicked off at IDLE latches is_last_block
215         from last_cmd.
216         // This handles all firmware patterns: INIT, INIT|
217         LAST, NEXT and NEXT|LAST.
218         if (hmac_ctrl_reg == CTRL_IDLE) begin
219             if (init_cmd | next_cmd)
220                 is_last_block <= last_cmd;
221         end
222     end // reg_update
223
224
225 always_ff @(posedge clk or negedge reset_n)
226     begin
227         if (!reset_n)
228             mode_reg <= '0;
229         else if (zeroize)
230             mode_reg <= '0;
231         else begin
232             if (hmac_ctrl_reg == CTRL_IDLE)
233                 mode_reg <= {1'b1, mode_cmd}; //hashing algorithm
234             //mode: 00 for SHA512/224, 01 for SHA512/256, 10 for SHA384, 11 for
235             //SHA512
236             end
237         end
238         // without zeroize to make it more complex
239         always_ff @(posedge clk or negedge reset_n) begin
240             if (!reset_n)
241                 entropy_digest <= '0;
242             else if (set_entropy)
243                 entropy_digest <= sha_digest[LFSR_SEED_SIZE-1 : 0];
244         end
245
246         //without zeroize to make it more complex
247         always_ff @(posedge clk or negedge reset_n) begin
248             if (!reset_n)
249                 counter_reg <= '0;
250             else
251                 counter_reg <= counter_reg + 1;
252         end
253
254         always_comb lfsr_entropy = entropy_digest ^ lfsr_seed;
255         always_comb entropy_block = {entropy_digest, lfsr_seed,

```

```

counter_reg, ENTROPY_PAD};
256
257 //-----
258 // sha_block_fsm
259 //
260 // Logic to select the appropriate SHA input block based on
the current state of the HMAC control FSM.
261 //-----
262 always_comb
263 begin : sha_block_fsm
264     unique case (hmac_ctrl_reg)
265         CTRL_ENTROPY: sha_block = entropy_block;
266         CTRL_IPAD:   sha_block = key_ipadded;
267         CTRL_MSG:    sha_block = block_msg;
268         CTRL_OPAD:   sha_block = key_opadded;
269         CTRL_HMAC:   sha_block = HMAC_padded;
270         default:     sha_block = '0';
271     endcase
272 end
273
274 //-----
275 // state_logic
276 //
277 // Mux and control for the single SHA core based on FSM
state.
278 //-----
279 always_comb
280 begin : state_logic
281     key_ipadded = {key, {(BLOCK_SIZE-KEY_SIZE){1'b0}}} ^
IPAD;
282     key_opadded = {key, {(BLOCK_SIZE-KEY_SIZE){1'b0}}} ^
OPAD;
283     HMAC_padded = mode_reg[0] ? {inner_digest_reg,
HMAC512_FINAL_PAD}
284                     : {in-
ner_digest_reg[TAG_SIZE-1 : HMAC384_TAG_PAD], HMAC384_FINAL_PAD};
285
286     // Defaults
287     sha_init = 1'b0;
288     sha_next = 1'b0;
289
290     first_round = (hmac_ctrl_reg == hmac_ctrl_ff) ? 1'b0 :
1'b1;
291     // IPAD/OPAD finish with sha_init only – digest is not
yet meaningful,
292     IPAD_ready = sha_ready;
293     MSG_ready = sha_ready & sha_digest_valid;
294     OPAD_ready = sha_ready;
295     HMAC_ready = sha_ready & sha_digest_valid;
296     ENTROPY_ready = sha_ready & sha_digest_valid;
297
298     unique case (hmac_ctrl_reg)
299         CTRL_ENTROPY:
300             if (first_round)
301                 begin
302                     sha_init = 1'b1;
303                     ENTROPY_ready = 1'b0;
304                 end
305             end
306         CTRL_IPAD:
307             if (first_round)
308                 begin
309                     sha_init = 1'b1;
310                     IPAD_ready = 1'b0;
311                 end
312             end
313         CTRL_MSG:
314             if (first_round)
315                 begin
316                     sha_next = 1'b1;
317                     MSG_ready = 1'b0;
318                 end
319             end
320         CTRL_OPAD:
321             if (first_round)
322                 begin
323                     sha_init = 1'b1;
324                     OPAD_ready = 1'b0;
325                 end
326             end
327         CTRL_HMAC:
328             if (first_round)
329                 begin
330                     sha_next = 1'b1;
331                     HMAC_ready = 1'b0;
332                 end
333             end
334         default: begin
335             sha_init = 1'b0;
336             sha_next = 1'b0;
337         end
338     endcase
339 end
340
341 //-----
342 // hmac_ctrl_fsm

```

```

343 //
344 // Logic for the state machine controlling the core beha-
viour.
345 //-----
346 always_comb
347 begin : hmac_ctrl_fsm
348     ready_flag = 1'b0;
349     digest_valid_new = 1'b0;
350     digest_valid_we = 1'b0;
351     hmac_ctrl_new = CTRL_IDLE;
352     hmac_ctrl_we = 1'b0;
353     inner_digest_we = 1'b0;
354     set_entropy = 1'b0;
355
356     unique case (hmac_ctrl_reg)
357         CTRL_IDLE:
358             begin
359                 ready_flag = 1'b1;
360
361                 if (init_cmd)
362                     begin
363                         digest_valid_new = 1'b0;
364                         digest_valid_we = 1'b1;
365                         hmac_ctrl_new = CTRL_ENTROPY;
366                         hmac_ctrl_we = 1'b1;
367                     end
368                 else if (next_cmd)
369                     begin
370                         digest_valid_new = 1'b0;
371                         digest_valid_we = 1'b1;
372                         hmac_ctrl_new = CTRL_MSG;
373                         hmac_ctrl_we = 1'b1;
374                     end
375                 end
376
377         CTRL_ENTROPY:
378             if (ENTROPY_ready)
379                 begin
380                     set_entropy = 1'b1;
381                     hmac_ctrl_new = CTRL_IPAD;
382                     hmac_ctrl_we = 1'b1;
383                 end
384             end
385         CTRL_IPAD:
386             if (IPAD_ready)
387                 begin
388                     hmac_ctrl_new = CTRL_MSG;
389                     hmac_ctrl_we = 1'b1;
390                 end
391             end
392         CTRL_MSG:
393             if (MSG_ready)
394                 begin
395                     if (is_last_block)
396                         begin
397                             inner_digest_we = 1'b1;
398                             hmac_ctrl_new = CTRL_OPAD;
399                         end
400                     else
401                         begin
402                             digest_valid_new = 1'b1;
403                             digest_valid_we = 1'b1;
404                             hmac_ctrl_new = CTRL_IDLE;
405                         end
406                     end
407                     hmac_ctrl_we = 1'b1;
408                 end
409             end
410         CTRL_OPAD:
411             if (OPAD_ready)
412                 begin
413                     hmac_ctrl_new = CTRL_HMAC;
414                     hmac_ctrl_we = 1'b1;
415                 end
416             end
417         CTRL_HMAC:
418             if (HMAC_ready)
419                 begin
420                     hmac_ctrl_new = CTRL_DONE;
421                     hmac_ctrl_we = 1'b1;
422                 end
423             end
424         CTRL_DONE:
425             begin
426                 digest_valid_new = 1'b1;
427                 digest_valid_we = 1'b1;
428                 hmac_ctrl_new = CTRL_IDLE;
429                 hmac_ctrl_we = 1'b1;
430             end
431         default:
432             begin
433                 ready_flag = 1'b0;
434                 digest_valid_new = 1'b0;
435                 digest_valid_we = 1'b0;
436                 hmac_ctrl_new = CTRL_IDLE;
437                 hmac_ctrl_we = 1'b0;
438             end
439         end
440     endcase // case (hmac_ctrl_reg)
441 end // hmac_ctrl_fsm

```



```

442
443 endmodule // hmac_core
444
445 //
=====
446 // EOF hmac_core.sv
447 //
=====

```

3. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_ctrl.sv (143 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // hmac_ctrl.sv
18 // -----
19 // HMAC controller for the AHB_lite interface.
20 //
21 //
22 //
23 //
=====
24
25
26 `include "caliptra_macros.svh"
27 `include "kv_macros.svh"
28
29 module hmac_ctrl
30     import kv_defines_pkg::*;
31     #(
32         parameter AHB_DATA_WIDTH = 32,
33         parameter AHB_ADDR_WIDTH = 32
34     )
35     (
36         // Clock and reset.
37         input wire      clk,
38         input wire      reset_n,
39         input wire      cptra_pwrgood,
40
41         //csr key
42         input logic [ `CLP_CSR_HMAC_KEY_DWORDS-1:0][31:0]
cptra_csr_hmac_key,
43
44         // from SLAVES PORT
45         input logic [AHB_ADDR_WIDTH-1:0] haddr_i,
46         input logic [AHB_DATA_WIDTH-1:0] hwdata_i,
47         input logic hsel_i,
48         input logic hwrite_i,
49         input logic hready_i,
50         input logic [1:0] htrans_i,
51         input logic [2:0] hsize_i,
52
53         output logic hresp_o,
54         output logic hreadyout_o,
55         output logic [AHB_DATA_WIDTH-1:0] hrdata_o,
56
57         // kv interface
58         output kv_read_t [1:0] kv_read,
59         output kv_write_t kv_write,
60         input kv_rd_resp_t [1:0] kv_rd_resp,
61         input kv_wr_resp_t kv_wr_resp,
62
63         output logic busy_o,
64
65         // Interrupt
66         output logic error_intr,
67         output logic notif_intr,
68         input  logic ocp_lock_in_progress,
69         input  logic debugUnlock_or_scan_mode_switch
70     );
71
72 //-----
73 // hmac
74 //-----

```

```

75     logic      hmac_cs;
76     logic      hmac_we;
77     logic [AHB_ADDR_WIDTH-1 : 0] hmac_address;
78     logic [31 : 0] hmac_write_data;
79     logic [31 : 0] hmac_read_data;
80
81     hmac #(
82         .ADDR_WIDTH (AHB_ADDR_WIDTH),
83         .DATA_WIDTH (32)
84     ) hmac_inst
85     (
86         .clk(clk),
87         .reset_n(reset_n),
88         .cptra_pwrgood(cptra_pwrgood),
89         .cptra_csr_hmac_key(cptra_csr_hmac_key),
90         .cs(hmac_cs),
91         .we(hmac_we),
92         .address(hmac_address),
93         .write_data(hmac_write_data),
94         .read_data(hmac_read_data),
95         .kv_read(kv_read),
96         .kv_write(kv_write),
97         .kv_rd_resp(kv_rd_resp),
98         .kv_wr_resp(kv_wr_resp),
99         .error_intr(error_intr),
100        .notif_intr(notif_intr),
101        .busy_o(busy_o),
102        .ocp_lock_in_progress(ocp_lock_in_progress),
103        .debugUn-
lock_or_scan_mode_switch(debugUnlock_or_scan_mode_switch)
104    );
105
106    //instantiate ahb lite module
107    ahb_slv_sif #(
108        .AHB_ADDR_WIDTH(AHB_ADDR_WIDTH),
109        .AHB_DATA_WIDTH(AHB_DATA_WIDTH),
110        .CLIENT_DATA_WIDTH(32)
111    ) ahb_slv_sif_uut
112    (
113        //AMBA AHB Lite INF
114        .hclk(clk),
115        .hreset_n(reset_n),
116        .haddr_i(haddr_i),
117        .hwdata_i(hwdata_i),
118        .hsel_i(hsel_i),
119        .hwrite_i(hwrite_i),
120        .hready_i(hready_i),
121        .htrans_i(htrans_i),
122        .hsize_i(hsize_i),
123
124        .hresp_o(hresp_o),
125        .hreadyout_o(hreadyout_o),
126        .hrdata_o(hrdata_o),
127
128        //COMPONENT INF
129        .dv(hmac_cs),
130        .hld(1'b0), //no holes from hmac
131        .err(1'b0), //no errors from hmac
132        .write(hmac_we),
133        .wdata(hmac_write_data),
134        .addr(hmac_address),
135
136        .rdata(hmac_read_data)
137    );
138
139
140
141
142
143 endmodule

```

4. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_param_pkg.sv (68 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // hmac_param_pkg.sv

```

[illegible]

5. **chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg.sv** (2,408 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 module hmac_reg (
5     input wire clk,
6     input wire rst,
7
8     input wire s_cpuif_req,
9     input wire s_cpuif_req_is_wr,
10    input wire [11:0] s_cpuif_addr,
11    input wire [31:0] s_cpuif_wr_data,
12    input wire [31:0] s_cpuif_wr_biten,
13    output wire s_cpuif_req_stall_wr,
14    output wire s_cpuif_req_stall_rd,
15    output wire s_cpuif_rd_ack,
16    output wire s_cpuif_rd_err,
17    output wire [31:0] s_cpuif_rd_data,
18    output wire s_cpuif_wr_ack,
19    output wire s_cpuif_wr_err,

```

```

20
21     input hmac_reg_pkg::hmac_reg_in_t hwif_in,
22     output hmac_reg_pkg::hmac_reg_out_t hwif_out
23 );
24
25 //-----
26 // CPU Bus interface logic
27 //-----
28
29 logic cpuif_req;
30 logic cpuif_req_is_wr;
31 logic [11:0] cpuif_addr;
32 logic [31:0] cpuif_wr_data;
33 logic [31:0] cpuif_wr_biten;
34 logic cpuif_req_stall_wr;
35 logic cpuif_req_stall_rd;
36
37 logic cpuif_rd_ack;
38 logic cpuif_rd_err;
39 logic [31:0] cpuif_rd_data;
40
41 logic cpuif_wr_ack;
42 logic cpuif_wr_err;
43
44 assign cpuif_req = s_cpuif_req;
45 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
46 assign cpuif_addr = s_cpuif_addr;
47 assign cpuif_wr_data = s_cpuif_wr_data;
48 assign cpuif_wr_biten = s_cpuif_wr_biten;
49 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
50 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
51 assign s_cpuif_rd_ack = cpuif_rd_ack;
52 assign s_cpuif_rd_err = cpuif_rd_err;
53 assign s_cpuif_rd_data = cpuif_rd_data;
54 assign s_cpuif_wr_ack = cpuif_wr_ack;
55 assign s_cpuif_wr_err = cpuif_wr_err;
56
57 logic cpuif_req_masked;
58
59 // Read & write latencies are balanced. Stalls not re-
60 quired
61 assign cpuif_req_stall_rd = '0;
62 assign cpuif_req_stall_wr = '0;
63 assign cpuif_req_masked = cpuif_req
64                        & !(cpuif_req_is_wr &
65 cpuif_req_stall_rd)
66                        & !(cpuif_req_is_wr &
67 cpuif_req_stall_wr);
68
69 //-----
70 // Address Decode
71 //-----
72
73 typedef struct packed{
74     logic [2-1:0]HMAC512_NAME;
75     logic [2-1:0]HMAC512_VERSION;
76     logic HMAC512_CTRL;
77     logic HMAC512_STATUS;
78     logic [16-1:0]HMAC512_KEY;
79     logic [32-1:0]HMAC512_BLOCK;
80     logic [16-1:0]HMAC512_TAG;
81     logic [6-1:0]HMAC512_LFSR_SEED;
82     logic HMAC512_KV_RD_KEY_CTRL;
83     logic HMAC512_KV_RD_KEY_STATUS;
84     logic HMAC512_KV_RD_BLOCK_CTRL;
85     logic HMAC512_KV_RD_BLOCK_STATUS;
86     logic HMAC512_KV_WR_CTRL;
87     logic HMAC512_KV_WR_STATUS;
88     struct packed{
89         logic global_intr_en_r;
90         logic error_intr_en_r;
91         logic notif_intr_en_r;
92         logic error_global_intr_r;
93         logic notif_global_intr_r;
94         logic error_internal_intr_r;
95         logic notif_internal_intr_r;
96         logic error_intr_trig_r;
97         logic notif_intr_trig_r;
98         logic key_mode_error_intr_count_r;
99         logic key_zero_error_intr_count_r;
100        logic error2_intr_count_r;
101        logic error3_intr_count_r;
102        logic notif_cmd_done_intr_count_r;
103        logic key_mode_error_intr_count_incr_r;
104        logic key_zero_error_intr_count_incr_r;
105        logic error2_intr_count_incr_r;
106        logic error3_intr_count_incr_r;
107        logic notif_cmd_done_intr_count_incr_r;
108    } intr_block_rf;
109 } decoded_reg_strb_t;
110 decoded_reg_strb_t decoded_reg_strb;
111 logic decoded_req;
112 logic decoded_req_is_wr;
113 logic [31:0] decoded_wr_data;
114 logic [31:0] decoded_wr_biten;
115
116 always_comb begin
117     for(int i0=0; i0<2; i0++) begin
118         decoded_reg_strb.HMAC512_NAME[i0] =

```



```

cpuif_req_masked & (cpuif_addr == 12'h0 + i0*12'h4);
114     end
115     for(int i0=0; i0<2; i0++) begin
116         decoded_reg_strb.HMAC512_VERSION[i0] =
cpuif_req_masked & (cpuif_addr == 12'h8 + i0*12'h4);
117     end
118     decoded_reg_strb.HMAC512_CTRL = cpuif_req_masked &
(cpuif_addr == 12'h10);
119     decoded_reg_strb.HMAC512_STATUS = cpuif_req_masked &
(cpuif_addr == 12'h18);
120     for(int i0=0; i0<16; i0++) begin
121         decoded_reg_strb.HMAC512_KEY[i0] =
cpuif_req_masked & (cpuif_addr == 12'h40 + i0*12'h4);
122     end
123     for(int i0=0; i0<32; i0++) begin
124         decoded_reg_strb.HMAC512_BLOCK[i0] =
cpuif_req_masked & (cpuif_addr == 12'h80 + i0*12'h4);
125     end
126     for(int i0=0; i0<16; i0++) begin
127         decoded_reg_strb.HMAC512_TAG[i0] =
cpuif_req_masked & (cpuif_addr == 12'h100 + i0*12'h4);
128     end
129     for(int i0=0; i0<6; i0++) begin
130         decoded_reg_strb.HMAC512_LFSR_SEED[i0] =
cpuif_req_masked & (cpuif_addr == 12'h140 + i0*12'h4);
131     end
132     decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL =
cpuif_req_masked & (cpuif_addr == 12'h600);
133     decoded_reg_strb.HMAC512_KV_RD_KEY_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h604);
134     decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL =
cpuif_req_masked & (cpuif_addr == 12'h608);
135     decoded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h60c);

136     decoded_reg_strb.HMAC512_KV_WR_CTRL = cpuif_req_masked &
(cpuif_addr == 12'h610);
137     decoded_reg_strb.HMAC512_KV_WR_STATUS =
cpuif_req_masked & (cpuif_addr == 12'h614);
138     decoded_reg_strb.intr_block_rf.global_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h800);
139     decoded_reg_strb.intr_block_rf.error_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h804);
140     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
cpuif_req_masked & (cpuif_addr == 12'h808);
141     decoded_reg_strb.intr_block_rf.error_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h80c);
142     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h810);

143     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h814);

144     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 12'h818);
145     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h81c);
146     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 12'h820);
147     de-
coded_reg_strb.intr_block_rf.key_mode_error_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h900);
148     de-
coded_reg_strb.intr_block_rf.key_zero_error_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h904);
149     decoded_reg_strb.intr_block_rf.error2_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h908);
150     decoded_reg_strb.intr_block_rf.error3_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h90c);
151     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
cpuif_req_masked & (cpuif_addr == 12'h980);
152     de-
coded_reg_strb.intr_block_rf.key_mode_error_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha00);
153     de-
coded_reg_strb.intr_block_rf.key_zero_error_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha04);
154     de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha08);
155     de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha0c);
156     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 12'ha10);
157     end
158     // Pass down signals to next stage
159     assign decoded_req = cpuif_req_masked;
160     assign decoded_req_is_wr = cpuif_req_is_wr;
161     assign decoded_wr_data = cpuif_wr_data;
162     assign decoded_wr_biten = cpuif_wr_biten;
163
164     //-----
165     // Field logic
166     //-----
167     //-----
168     typedef struct packed{
169
170         struct packed{
171             logic next;
172             logic load_next;
173         } INIT;
174         struct packed{
175             logic next;
176             logic load_next;
177         } NEXT;
178         struct packed{
179             logic next;
180             logic load_next;
181         } ZEROIZE;
182         struct packed{
183             logic next;
184             logic load_next;
185         } MODE;
186         struct packed{
187             logic next;
188             logic load_next;
189         } CSR_MODE;
190         struct packed{
191             logic next;
192             logic load_next;
193         } LAST;
194         struct packed{
195             logic next;
196             logic load_next;
197         } Reserved;
198     } HMAC512_CTRL;
199     struct packed{
200         struct packed{
201             logic [31:0] next;
202             logic load_next;
203         } KEY;
204     } [16:1:0]HMAC512_KEY;
205     struct packed{
206         struct packed{
207             logic [31:0] next;
208             logic load_next;
209         } BLOCK;
210     } [32:1:0]HMAC512_BLOCK;
211     struct packed{
212         struct packed{
213             logic [31:0] next;
214             logic load_next;
215         } TAG;
216     } [16:1:0]HMAC512_TAG;
217     struct packed{
218         struct packed{
219             logic [31:0] next;
220             logic load_next;
221         } LFSR_SEED;
222     } [6:1:0]HMAC512_LFSR_SEED;
223     struct packed{
224         struct packed{
225             logic next;
226             logic load_next;
227         } read_en;
228         struct packed{
229             logic [4:0] next;
230             logic load_next;
231         } read_entry;
232         struct packed{
233             logic next;
234             logic load_next;
235         } pcr_hash_extend;
236         struct packed{
237             logic [24:0] next;
238             logic load_next;
239         } rsvd;
240     } HMAC512_KV_RD_KEY_CTRL;
241     struct packed{
242         struct packed{
243             logic next;
244             logic load_next;
245         } VALID;
246     } HMAC512_KV_RD_KEY_STATUS;
247     struct packed{
248         struct packed{
249             logic next;
250             logic load_next;
251         } read_en;
252         struct packed{
253             logic [4:0] next;
254             logic load_next;
255         } read_entry;
256         struct packed{
257             logic next;
258             logic load_next;
259         } pcr_hash_extend;
260         struct packed{
261             logic [24:0] next;
262             logic load_next;
263         } rsvd;
264     } HMAC512_KV_RD_BLOCK_CTRL;
265     struct packed{
266         struct packed{
267             logic next;
268             logic load_next;
269         } VALID;

```

```

270 } HMAC512_KV_RD_BLOCK_STATUS;
271 struct packed{
272     struct packed{
273         logic next;
274         logic load_next;
275     } write_en;
276     struct packed{
277         logic [4:0] next;
278         logic load_next;
279     } write_entry;
280     struct packed{
281         logic next;
282         logic load_next;
283     } hmac_key_dest_valid;
284     struct packed{
285         logic next;
286         logic load_next;
287     } hmac_block_dest_valid;
288     struct packed{
289         logic next;
290         logic load_next;
291     } mldsa_seed_dest_valid;
292     struct packed{
293         logic next;
294         logic load_next;
295     } ecc_pkey_dest_valid;
296     struct packed{
297         logic next;
298         logic load_next;
299     } ecc_seed_dest_valid;
300     struct packed{
301         logic next;
302         logic load_next;
303     } aes_key_dest_valid;
304     struct packed{
305         logic next;
306         logic load_next;
307     } mlkem_seed_dest_valid;
308     struct packed{
309         logic next;
310         logic load_next;
311     } mlkem_msg_dest_valid;
312     struct packed{
313         logic next;
314         logic load_next;
315     } dma_data_dest_valid;
316     struct packed{
317         logic [16:0] next;
318         logic load_next;
319     } rsvd;
320 } HMAC512_KV_WR_CTRL;
321 struct packed{
322     struct packed{
323         logic next;
324         logic load_next;
325     } VALID;
326 } HMAC512_KV_WR_STATUS;
327 struct packed{
328     struct packed{
329         struct packed{
330             logic next;
331             logic load_next;
332         } error_en;
333         struct packed{
334             logic next;
335             logic load_next;
336         } notif_en;
337     } global_intr_en_r;
338     struct packed{
339         struct packed{
340             logic next;
341             logic load_next;
342         } key_mode_error_en;
343         struct packed{
344             logic next;
345             logic load_next;
346         } key_zero_error_en;
347         struct packed{
348             logic next;
349             logic load_next;
350         } error2_en;
351         struct packed{
352             logic next;
353             logic load_next;
354         } error3_en;
355     } error_intr_en_r;
356     struct packed{
357         struct packed{
358             logic next;
359             logic load_next;
360         } notif_cmd_done_en;
361     } notif_intr_en_r;
362     struct packed{
363         struct packed{
364             logic next;
365             logic load_next;
366         } agg_sts;
367     } error_global_intr_r;
368     struct packed{
369         struct packed{
370             logic next;
371             logic load_next;
372             logic agg_sts;
373         } notif_global_intr_r;
374     } struct packed{
375         struct packed{
376             logic next;
377             logic load_next;
378         } key_mode_error_sts;
379     } struct packed{
380         logic next;
381         logic load_next;
382     } key_zero_error_sts;
383     struct packed{
384         logic next;
385         logic load_next;
386     } error2_sts;
387     struct packed{
388         logic next;
389         logic load_next;
390     } error3_sts;
391 } error_internal_intr_r;
392 struct packed{
393     struct packed{
394         logic next;
395         logic load_next;
396     } notif_cmd_done_sts;
397 } notif_internal_intr_r;
398 struct packed{
399     struct packed{
400         logic next;
401         logic load_next;
402     } key_mode_error_trig;
403     struct packed{
404         logic next;
405         logic load_next;
406     } key_zero_error_trig;
407     struct packed{
408         logic next;
409         logic load_next;
410     } error2_trig;
411     struct packed{
412         logic next;
413         logic load_next;
414     } error3_trig;
415 } error_intr_trig_r;
416 struct packed{
417     struct packed{
418         logic next;
419         logic load_next;
420     } notif_cmd_done_trig;
421 } notif_intr_trig_r;
422 struct packed{
423     struct packed{
424         logic [31:0] next;
425         logic load_next;
426         logic incrthreshold;
427         logic incrsaturate;
428     } cnt;
429 } key_mode_error_intr_count_r;
430 struct packed{
431     struct packed{
432         logic [31:0] next;
433         logic load_next;
434         logic incrthreshold;
435         logic incrsaturate;
436     } cnt;
437 } key_zero_error_intr_count_r;
438 struct packed{
439     struct packed{
440         logic [31:0] next;
441         logic load_next;
442         logic incrthreshold;
443         logic incrsaturate;
444     } cnt;
445 } error2_intr_count_r;
446 struct packed{
447     struct packed{
448         logic [31:0] next;
449         logic load_next;
450         logic incrthreshold;
451         logic incrsaturate;
452     } cnt;
453 } error3_intr_count_r;
454 struct packed{
455     struct packed{
456         logic [31:0] next;
457         logic load_next;
458         logic incrthreshold;
459         logic incrsaturate;
460     } cnt;
461 } notif_cmd_done_intr_count_r;
462 struct packed{
463     struct packed{
464         logic next;
465         logic load_next;
466         logic decrthreshold;
467         logic underflow;
468     } pulse;
469 } key_mode_error_intr_count_incr_r;
470 struct packed{
471     struct packed{

```

```

472         logic next;
473         logic load_next;
474         logic decrthreshold;
475         logic underflow;
476     } pulse;
477 } key_zero_error_intr_count_incr_r;
478 struct packed{
479     struct packed{
480         logic next;
481         logic load_next;
482         logic decrthreshold;
483         logic underflow;
484     } pulse;
485 } error2_intr_count_incr_r;
486 struct packed{
487     struct packed{
488         logic next;
489         logic load_next;
490         logic decrthreshold;
491         logic underflow;
492     } pulse;
493 } error3_intr_count_incr_r;
494 struct packed{
495     struct packed{
496         logic next;
497         logic load_next;
498         logic decrthreshold;
499         logic underflow;
500     } pulse;
501 } notif_cmd_done_intr_count_incr_r;
502 } intr_block_rf;
503 } field_combo_t;
504 field_combo_t field_combo;
505
506 typedef struct packed{
507     struct packed{
508         struct packed{
509             logic value;
510         } INIT;
511         struct packed{
512             logic value;
513         } NEXT;
514         struct packed{
515             logic value;
516         } ZEROIZE;
517         struct packed{
518             logic value;
519         } MODE;
520         struct packed{
521             logic value;
522         } CSR_MODE;
523         struct packed{
524             logic value;
525         } LAST;
526         struct packed{
527             logic value;
528         } Reserved;
529     } HMAC512_CTRL;
530     struct packed{
531         struct packed{
532             logic [31:0] value;
533         } KEY;
534     } [16-1:0]HMAC512_KEY;
535     struct packed{
536         struct packed{
537             logic [31:0] value;
538         } BLOCK;
539     } [32-1:0]HMAC512_BLOCK;
540     struct packed{
541         struct packed{
542             logic [31:0] value;
543         } TAG;
544     } [16-1:0]HMAC512_TAG;
545     struct packed{
546         struct packed{
547             logic [31:0] value;
548         } LFSR_SEED;
549     } [6-1:0]HMAC512_LFSR_SEED;
550     struct packed{
551         struct packed{
552             logic value;
553         } read_en;
554         struct packed{
555             logic [4:0] value;
556         } read_entry;
557         struct packed{
558             logic value;
559         } pcr_hash_extend;
560         struct packed{
561             logic [24:0] value;
562         } rsvd;
563     } HMAC512_KV_RD_KEY_CTRL;
564     struct packed{
565         struct packed{
566             logic value;
567         } VALID;
568     } HMAC512_KV_RD_KEY_STATUS;
569     struct packed{
570         struct packed{
571             logic value;
572         } read_en;

```

```

573     struct packed{
574         logic [4:0] value;
575     } read_entry;
576     struct packed{
577         logic value;
578     } pcr_hash_extend;
579     struct packed{
580         logic [24:0] value;
581     } rsvd;
582 } HMAC512_KV_RD_BLOCK_CTRL;
583 struct packed{
584     struct packed{
585         logic value;
586     } VALID;
587 } HMAC512_KV_RD_BLOCK_STATUS;
588 struct packed{
589     struct packed{
590         logic value;
591     } write_en;
592     struct packed{
593         logic [4:0] value;
594     } write_entry;
595     struct packed{
596         logic value;
597     } hmac_key_dest_valid;
598     struct packed{
599         logic value;
600     } hmac_block_dest_valid;
601     struct packed{
602         logic value;
603     } mldsa_seed_dest_valid;
604     struct packed{
605         logic value;
606     } ecc_pkey_dest_valid;
607     struct packed{
608         logic value;
609     } ecc_seed_dest_valid;
610     struct packed{
611         logic value;
612     } aes_key_dest_valid;
613     struct packed{
614         logic value;
615     } mlkem_seed_dest_valid;
616     struct packed{
617         logic value;
618     } mlkem_msg_dest_valid;
619     struct packed{
620         logic value;
621     } dma_data_dest_valid;
622     struct packed{
623         logic [16:0] value;
624     } rsvd;
625 } HMAC512_KV_WR_CTRL;
626 struct packed{
627     struct packed{
628         logic value;
629     } VALID;
630 } HMAC512_KV_WR_STATUS;
631 struct packed{
632     struct packed{
633         struct packed{
634             logic value;
635         } error_en;
636         struct packed{
637             logic value;
638         } notif_en;
639     } global_intr_en_r;
640     struct packed{
641         struct packed{
642             logic value;
643         } key_mode_error_en;
644         struct packed{
645             logic value;
646         } key_zero_error_en;
647         struct packed{
648             logic value;
649         } error2_en;
650         struct packed{
651             logic value;
652         } error3_en;
653     } error_intr_en_r;
654     struct packed{
655         struct packed{
656             logic value;
657         } notif_cmd_done_en;
658     } notif_intr_en_r;
659     struct packed{
660         struct packed{
661             logic value;
662         } agg_sts;
663     } error_global_intr_r;
664     struct packed{
665         struct packed{
666             logic value;
667         } agg_sts;
668     } notif_global_intr_r;
669     struct packed{
670         struct packed{
671             logic value;
672         } key_mode_error_sts;
673     } struct packed{

```

```

674         logic value;
675     } key_zero_error_sts;
676     struct packed{
677         logic value;
678     } error2_sts;
679     struct packed{
680         logic value;
681     } error3_sts;
682 } error_internal_intr_r;
683 struct packed{
684     struct packed{
685         logic value;
686     } notif_cmd_done_sts;
687 } notif_internal_intr_r;
688 struct packed{
689     struct packed{
690         logic value;
691     } key_mode_error_trig;
692     struct packed{
693         logic value;
694     } key_zero_error_trig;
695     struct packed{
696         logic value;
697     } error2_trig;
698     struct packed{
699         logic value;
700     } error3_trig;
701 } error_intr_trig_r;
702 struct packed{
703     struct packed{
704         logic value;
705     } notif_cmd_done_trig;
706 } notif_intr_trig_r;
707 struct packed{
708     struct packed{
709         logic [31:0] value;
710     } cnt;
711 } key_mode_error_intr_count_r;
712 struct packed{
713     struct packed{
714         logic [31:0] value;
715     } cnt;
716 } key_zero_error_intr_count_r;
717 struct packed{
718     struct packed{
719         logic [31:0] value;
720     } cnt;
721 } error2_intr_count_r;
722 struct packed{
723     struct packed{
724         logic [31:0] value;
725     } cnt;
726 } error3_intr_count_r;
727 struct packed{
728     struct packed{
729         logic [31:0] value;
730     } cnt;
731 } notif_cmd_done_intr_count_r;
732 struct packed{
733     struct packed{
734         logic value;
735     } pulse;
736 } key_mode_error_intr_count_incr_r;
737 struct packed{
738     struct packed{
739         logic value;
740     } pulse;
741 } key_zero_error_intr_count_incr_r;
742 struct packed{
743     struct packed{
744         logic value;
745     } pulse;
746 } error2_intr_count_incr_r;
747 struct packed{
748     struct packed{
749         logic value;
750     } pulse;
751 } error3_intr_count_incr_r;
752 struct packed{
753     struct packed{
754         logic value;
755     } pulse;
756 } notif_cmd_done_intr_count_incr_r;
757 } intr_block_rf;
758 } field_storage_t;
759 field_storage_t field_storage;
760
761 // Field: hmac_reg.HMAC512_CTRL.INIT
762 always_comb begin
763     automatic logic [0:0] next_c;
764     automatic logic load_next_c;
765     next_c = field_storage.HMAC512_CTRL.INIT.value;
766     load_next_c = '0;
767     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.INIT.swwe) begin // SW write
768         next_c = (field_storage.HMAC512_CTRL.INIT.value &
~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
769         load_next_c = '1;
770     end else begin // singlepulse clears back to 0
771         next_c = '0;

```

```

772         load_next_c = '1;
773     end
774     field_combo.HMAC512_CTRL.INIT.next = next_c;
775     field_combo.HMAC512_CTRL.INIT.load_next = load_next_c;
776 end
777 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
778     if(~hwif_in.reset_b) begin
779         field_storage.HMAC512_CTRL.INIT.value <= 1'h0;
780     end else if(field_combo.HMAC512_CTRL.INIT.load_next)
begin
781         field_storage.HMAC512_CTRL.INIT.value <=
field_combo.HMAC512_CTRL.INIT.next;
782     end
783 end
784 assign hwif_out.HMAC512_CTRL.INIT.value =
field_storage.HMAC512_CTRL.INIT.value;
785 // Field: hmac_reg.HMAC512_CTRL.NEXT
786 always_comb begin
787     automatic logic [0:0] next_c;
788     automatic logic load_next_c;
789     next_c = field_storage.HMAC512_CTRL.NEXT.value;
790     load_next_c = '0;
791     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.NEXT.swwe) begin // SW write
792         next_c = (field_storage.HMAC512_CTRL.NEXT.value &
~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
793         load_next_c = '1;
794     end else begin // singlepulse clears back to 0
795         next_c = '0;
796         load_next_c = '1;
797     end
798     field_combo.HMAC512_CTRL.NEXT.next = next_c;
799     field_combo.HMAC512_CTRL.NEXT.load_next = load_next_c;
800 end
801 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
802     if(~hwif_in.reset_b) begin
803         field_storage.HMAC512_CTRL.NEXT.value <= 1'h0;
804     end else if(field_combo.HMAC512_CTRL.NEXT.load_next)
begin
805         field_storage.HMAC512_CTRL.NEXT.value <=
field_combo.HMAC512_CTRL.NEXT.next;
806     end
807 end
808 assign hwif_out.HMAC512_CTRL.NEXT.value =
field_storage.HMAC512_CTRL.NEXT.value;
809 // Field: hmac_reg.HMAC512_CTRL.ZEROIZE
810 always_comb begin
811     automatic logic [0:0] next_c;
812     automatic logic load_next_c;
813     next_c = field_storage.HMAC512_CTRL.ZEROIZE.value;
814     load_next_c = '0;
815     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr)
begin // SW write
816         next_c = (field_storage.HMAC512_CTRL.ZEROIZE.value &
~decoded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
817         load_next_c = '1;
818     end else begin // singlepulse clears back to 0
819         next_c = '0;
820         load_next_c = '1;
821     end
822     field_combo.HMAC512_CTRL.ZEROIZE.next = next_c;
823     field_combo.HMAC512_CTRL.ZEROIZE.load_next =
load_next_c;
824 end
825 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
826     if(~hwif_in.reset_b) begin
827         field_storage.HMAC512_CTRL.ZEROIZE.value <= 1'h0;
828     end else
if(field_combo.HMAC512_CTRL.ZEROIZE.load_next) begin
829         field_storage.HMAC512_CTRL.ZEROIZE.value <=
field_combo.HMAC512_CTRL.ZEROIZE.next;
830     end
831 end
832 assign hwif_out.HMAC512_CTRL.ZEROIZE.value =
field_storage.HMAC512_CTRL.ZEROIZE.value;
833 // Field: hmac_reg.HMAC512_CTRL.MODE
834 always_comb begin
835     automatic logic [0:0] next_c;
836     automatic logic load_next_c;
837     next_c = field_storage.HMAC512_CTRL.MODE.value;
838     load_next_c = '0;
839     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.MODE.swwe) begin // SW write
840         next_c = (field_storage.HMAC512_CTRL.MODE.value &
~decoded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
841         load_next_c = '1;
842     end
843     field_combo.HMAC512_CTRL.MODE.next = next_c;
844     field_combo.HMAC512_CTRL.MODE.load_next = load_next_c;
845 end
846 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
847     if(~hwif_in.reset_b) begin
848         field_storage.HMAC512_CTRL.MODE.value <= 1'h1;
849     end else if(field_combo.HMAC512_CTRL.MODE.load_next)
begin
850         field_storage.HMAC512_CTRL.MODE.value <=

```

```

field_combo.HMAC512_CTRL.MODE.next;
851     end
852   end
853   assign hwif_out.HMAC512_CTRL.MODE.value =
field_storage.HMAC512_CTRL.MODE.value;
854   // Field: hmac_reg.HMAC512_CTRL.CSR_MODE
855   always_comb begin
856     automatic logic [0:0] next_c;
857     automatic logic load_next_c;
858     next_c = field_storage.HMAC512_CTRL.CSR_MODE.value;
859     load_next_c = '0;
860     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.CSR_MODE.swwe) begin // SW write
861       next_c =
(field_storage.HMAC512_CTRL.CSR_MODE.value & ~decoded_wr_biten[4:4])
| (decoded_wr_data[4:4] & decoded_wr_biten[4:4]);
862       load_next_c = '1;
863     end
864     field_combo.HMAC512_CTRL.CSR_MODE.next = next_c;
865     field_combo.HMAC512_CTRL.CSR_MODE.load_next =
load_next_c;
866   end
867   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
868     if(~hwif_in.reset_b) begin
869       field_storage.HMAC512_CTRL.CSR_MODE.value <= '1'h0;
870     end else
if(field_combo.HMAC512_CTRL.CSR_MODE.load_next) begin
871       field_storage.HMAC512_CTRL.CSR_MODE.value <=
field_combo.HMAC512_CTRL.CSR_MODE.next;
872     end
873   end
874   assign hwif_out.HMAC512_CTRL.CSR_MODE.value =
field_storage.HMAC512_CTRL.CSR_MODE.value;
875   // Field: hmac_reg.HMAC512_CTRL.LAST
876   always_comb begin
877     automatic logic [0:0] next_c;
878     automatic logic load_next_c;
879     next_c = field_storage.HMAC512_CTRL.LAST.value;
880     load_next_c = '0;
881     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr
&& hwif_in.HMAC512_CTRL.LAST.swwe) begin // SW write
882       next_c = (field_storage.HMAC512_CTRL.LAST.value &
~decoded_wr_biten[5:5]) | (decoded_wr_data[5:5] & de-
coded_wr_biten[5:5]);
883       load_next_c = '1;
884     end else begin // singlepulse clears back to 0
885       next_c = '0;
886       load_next_c = '1;
887     end
888     field_combo.HMAC512_CTRL.LAST.next = next_c;
889     field_combo.HMAC512_CTRL.LAST.load_next = load_next_c;
890   end
891   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
892     if(~hwif_in.reset_b) begin
893       field_storage.HMAC512_CTRL.LAST.value <= '1'h0;
894     end else if(field_combo.HMAC512_CTRL.LAST.load_next)
begin
895       field_storage.HMAC512_CTRL.LAST.value <=
field_combo.HMAC512_CTRL.LAST.next;
896     end
897   end
898   assign hwif_out.HMAC512_CTRL.LAST.value =
field_storage.HMAC512_CTRL.LAST.value;
899   // Field: hmac_reg.HMAC512_CTRL.Reserved
900   always_comb begin
901     automatic logic [0:0] next_c;
902     automatic logic load_next_c;
903     next_c = field_storage.HMAC512_CTRL.Reserved.value;
904     load_next_c = '0;
905     if(decoded_reg_strb.HMAC512_CTRL && decoded_req_is_wr)
begin // SW write
906       next_c =
(field_storage.HMAC512_CTRL.Reserved.value & ~decoded_wr_biten[6:6])
| (decoded_wr_data[6:6] & decoded_wr_biten[6:6]);
907       load_next_c = '1;
908     end
909     field_combo.HMAC512_CTRL.Reserved.next = next_c;
910     field_combo.HMAC512_CTRL.Reserved.load_next =
load_next_c;
911   end
912   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
913     if(~hwif_in.reset_b) begin
914       field_storage.HMAC512_CTRL.Reserved.value <= '1'h0;
915     end else
if(field_combo.HMAC512_CTRL.Reserved.load_next) begin
916       field_storage.HMAC512_CTRL.Reserved.value <=
field_combo.HMAC512_CTRL.Reserved.next;
917     end
918   end
919   assign hwif_out.HMAC512_CTRL.Reserved.value =
field_storage.HMAC512_CTRL.Reserved.value;
920   for(genvar i0=0; i0<16; i0++) begin
921     // Field: hmac_reg.HMAC512_KEY[i].KEY
922     always_comb begin
923       automatic logic [31:0] next_c;
924       automatic logic load_next_c;
925       next_c = field_storage.HMAC512_KEY[i0].KEY.value;
926       load_next_c = '0;
927       if(decoded_reg_strb.HMAC512_KEY[i0] && de-
coded_req_is_wr && !(hwif_in.HMAC512_KEY[i0].KEY.swwe)) begin // SW

```

```

write
928       next_c =
(field_storage.HMAC512_KEY[i0].KEY.value & ~decoded_wr_biten[31:0])
| (decoded_wr_data[31:0] & decoded_wr_biten[31:0]);
929       load_next_c = '1;
930     end else if(hwif_in.HMAC512_KEY[i0].KEY.we) be-
gin // HW Write - we
931       next_c = hwif_in.HMAC512_KEY[i0].KEY.next;
932       load_next_c = '1;
933     end else if(hwif_in.HMAC512_KEY[i0].KEY.hwclr) be-
gin // HW Clear
934       next_c = '0;
935       load_next_c = '1;
936     end
937     field_combo.HMAC512_KEY[i0].KEY.next = next_c;
938     field_combo.HMAC512_KEY[i0].KEY.load_next =
load_next_c;
939   end
940   always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
941     if(~hwif_in.reset_b) begin
942       field_storage.HMAC512_KEY[i0].KEY.value <=
32'h0;
943     end else
if(field_combo.HMAC512_KEY[i0].KEY.load_next) begin
944       field_storage.HMAC512_KEY[i0].KEY.value <=
field_combo.HMAC512_KEY[i0].KEY.next;
945     end
946   end
947   assign hwif_out.HMAC512_KEY[i0].KEY.value =
field_storage.HMAC512_KEY[i0].KEY.value;
948   end
949   for(genvar i0=0; i0<32; i0++) begin
950     // Field: hmac_reg.HMAC512_BLOCK[i].BLOCK
951     always_comb begin
952       automatic logic [31:0] next_c;
953       automatic logic load_next_c;
954       next_c =
field_storage.HMAC512_BLOCK[i0].BLOCK.value;
955       load_next_c = '0;
956       if(decoded_reg_strb.HMAC512_BLOCK[i0] && de-
coded_req_is_wr && !(hwif_in.HMAC512_BLOCK[i0].BLOCK.swwe)) be-
gin // SW write
957       next_c =
(field_storage.HMAC512_BLOCK[i0].BLOCK.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
958       load_next_c = '1;
959     end else if(hwif_in.HMAC512_BLOCK[i0].BLOCK.we)
begin // HW Write - we
960       next_c = hwif_in.HMAC512_BLOCK[i0].BLOCK.next;
961       load_next_c = '1;
962     end else if(hwif_in.HMAC512_BLOCK[i0].BLOCK.hwclr)
begin // HW Clear
963       next_c = '0;
964       load_next_c = '1;
965     end
966     field_combo.HMAC512_BLOCK[i0].BLOCK.next = next_c;
967     field_combo.HMAC512_BLOCK[i0].BLOCK.load_next =
load_next_c;
968   end
969   always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
970     if(~hwif_in.reset_b) begin
971       field_storage.HMAC512_BLOCK[i0].BLOCK.value <=
32'h0;
972     end else
if(field_combo.HMAC512_BLOCK[i0].BLOCK.load_next) begin
973       field_storage.HMAC512_BLOCK[i0].BLOCK.value <=
field_combo.HMAC512_BLOCK[i0].BLOCK.next;
974     end
975   end
976   assign hwif_out.HMAC512_BLOCK[i0].BLOCK.value =
field_storage.HMAC512_BLOCK[i0].BLOCK.value;
977   end
978   for(genvar i0=0; i0<16; i0++) begin
979     // Field: hmac_reg.HMAC512_TAG[i].TAG
980     always_comb begin
981       automatic logic [31:0] next_c;
982       automatic logic load_next_c;
983       next_c = field_storage.HMAC512_TAG[i0].TAG.value;
984       load_next_c = '0;
985       if(hwif_in.HMAC512_TAG[i0].TAG.hwclr) begin // HW
Clear
986       next_c = '0;
987       load_next_c = '1;
988     end else begin // HW Write
989       next_c = hwif_in.HMAC512_TAG[i0].TAG.next;
990       load_next_c = '1;
991     end
992     field_combo.HMAC512_TAG[i0].TAG.next = next_c;
993     field_combo.HMAC512_TAG[i0].TAG.load_next =
load_next_c;
994   end
995   always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
996     if(~hwif_in.reset_b) begin
997       field_storage.HMAC512_TAG[i0].TAG.value <=

```



```

32'h0;
998     end else
if(field_combo.HMAC512_TAG[i0].TAG.load_next) begin
999     field_storage.HMAC512_TAG[i0].TAG.value <=
field_combo.HMAC512_TAG[i0].TAG.next;
1000     end
1001     end
1002     end
1003     for(genvar i0=0; i0<6; i0++) begin
1004     // Field: hmac_reg.HMAC512_LFSR_SEED[i].LFSR_SEED
1005     always_comb begin
1006     automatic logic [31:0] next_c;
1007     automatic logic load_next_c;
1008     next_c =
field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value;
1009     load_next_c = '0;
1010     if(decoded_reg_strb.HMAC512_LFSR_SEED[i0] && de-
coded_req_is_wr) begin // SW write
1011     next_c =
(field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1012     load_next_c = '1;
1013     end
1014     field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.next =
next_c;
1015     field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.load_next = load_next_c;
1016     end
1017     always_ff @(posedge clk or negedge hwif_in.reset_b)
begin
1018     if(~hwif_in.reset_b) begin
1019     field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value <= 32'h0;
1020     end else
if(field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.load_next) begin
1021     field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value <=
field_combo.HMAC512_LFSR_SEED[i0].LFSR_SEED.next;
1022     end
1023     end
1024     assign hwif_out.HMAC512_LFSR_SEED[i0].LFSR_SEED.value
= field_storage.HMAC512_LFSR_SEED[i0].LFSR_SEED.value;
1025     end
1026     // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.read_en
1027     always_comb begin
1028     automatic logic [0:0] next_c;
1029     automatic logic load_next_c;
1030     next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value;
1031     load_next_c = '0;
1032     if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.swve) be-
gin // SW write
1033     next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1034     load_next_c = '1;
1035     end else
if(hwif_in.HMAC512_KV_RD_KEY_CTRL.read_en.hwclr) begin // HW Clear
1036     next_c = '0;
1037     load_next_c = '1;
1038     end
1039     field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.next =
next_c;
1040     field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.load_next =
load_next_c;
1041     end
1042     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1043     if(~hwif_in.reset_b) begin
1044     field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value
<= 1'h0;
1045     end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.load_next) begin
1046     field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value
<= field_combo.HMAC512_KV_RD_KEY_CTRL.read_en.next;
1047     end
1048     end
1049     assign hwif_out.HMAC512_KV_RD_KEY_CTRL.read_en.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value;
1050     // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.read_entry
1051     always_comb begin
1052     automatic logic [4:0] next_c;
1053     automatic logic load_next_c;
1054     next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value;
1055     load_next_c = '0;
1056     if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_KEY_CTRL.read_entry.swve)
begin // SW write
1057     next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1058     load_next_c = '1;
1059     end

```

```

1060     field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.next =
next_c;
1061     field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.load_next =
load_next_c;
1062     end
1063     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1064     if(~hwif_in.reset_b) begin
1065     field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value <= 5'h0;
1066     end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.load_next) begin
1067     field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value <=
field_combo.HMAC512_KV_RD_KEY_CTRL.read_entry.next;
1068     end
1069     end
1070     assign hwif_out.HMAC512_KV_RD_KEY_CTRL.read_entry.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value;
1071     // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend
1072     always_comb begin
1073     automatic logic [0:0] next_c;
1074     automatic logic load_next_c;
1075     next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value;
1076     load_next_c = '0;
1077     if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.swve) begin // SW
write
1078     next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1079     load_next_c = '1;
1080     end
1081     field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.next = next_c;
1082     field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.load_next =
load_next_c;
1083     end
1084     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1085     if(~hwif_in.reset_b) begin
1086     field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value <= 1'h0;
1087     end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.load_next) be-
gin
1088     field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value <=
field_combo.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.next;
1089     end
1090     end
1091     assign
hwif_out.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value;
1092     // Field: hmac_reg.HMAC512_KV_RD_KEY_CTRL.rsvd
1093     always_comb begin
1094     automatic logic [24:0] next_c;
1095     automatic logic load_next_c;
1096     next_c =
field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value;
1097     load_next_c = '0;
1098     if(decoded_reg_strb.HMAC512_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_KEY_CTRL.rsvd.swve) be-
gin // SW write
1099     next_c =
(field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
1100     load_next_c = '1;
1101     end
1102     field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.next = next_c;
1103     field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.load_next =
load_next_c;
1104     end
1105     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1106     if(~hwif_in.reset_b) begin
1107     field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value <=
25'h0;
1108     end else
if(field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.load_next) begin
1109     field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value <=
field_combo.HMAC512_KV_RD_KEY_CTRL.rsvd.next;
1110     end
1111     end
1112     assign hwif_out.HMAC512_KV_RD_KEY_CTRL.rsvd.value =
field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value;
1113     // Field: hmac_reg.HMAC512_KV_RD_KEY_STATUS.VALID
1114     always_comb begin
1115     automatic logic [0:0] next_c;
1116     automatic logic load_next_c;
1117     next_c =
field_storage.HMAC512_KV_RD_KEY_STATUS.VALID.value;
1118     load_next_c = '0;
1119     if(hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwset) be-
gin // HW Set
1120     next_c = '1;

```

```

1121         load_next_c = '1';
1122     end else
1123 if(hwif_in.HMAC512_KV_RD_KEY_STATUS.VALID.hwc1r) begin // HW Clear
1124     next_c = '0';
1125     load_next_c = '1';
1126 end
1127 field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.next =
next_c;
1128
1129 field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.load_next =
load_next_c;
1130 end
1131 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1132 if(~hwif_in.reset_b) begin
1133     field_storage.HMAC512_KV_RD_KEY_STATUS.VALID.value
<= 1'h0;
1134 end else
1135 if(field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.load_next) begin
1136     field_storage.HMAC512_KV_RD_KEY_STATUS.VALID.value
<= field_combo.HMAC512_KV_RD_KEY_STATUS.VALID.next;
1137 end
1138 // Field: hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.read_en
1139 always_comb begin
1140     automatic logic [0:0] next_c;
1141     automatic logic load_next_c;
1142     next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value;
1143     load_next_c = '0';
1144     if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.swwe)
begin // SW write
1145         next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1146     end else
1147 if(hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_en.hwc1r) begin // HW Clear
1148     next_c = '0';
1149     load_next_c = '1';
1150 end
1151 field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.next =
next_c;
1152
1153 field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.load_next
= load_next_c;
1154 end
1155 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1156 if(~hwif_in.reset_b) begin
1157     field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value <= 1'h0;
1158 end else
1159 if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.load_next) begin
1160     field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value <=
field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_en.next;
1161 end
1162 end
1163 assign hwif_out.HMAC512_KV_RD_BLOCK_CTRL.read_en.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value;
1164 // Field: hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.read_entry
1165 always_comb begin
1166     automatic logic [4:0] next_c;
1167     automatic logic load_next_c;
1168     next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value;
1169     load_next_c = '0';
1170     if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_BLOCK_CTRL.read_entry.swwe)
begin // SW write
1171         next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1172     end
1173     load_next_c = '1';
1174 end
1175 field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.next =
next_c;
1176
1177 field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.load_next =
load_next_c;
1178 end
1179 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1180 if(~hwif_in.reset_b) begin
1181     field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value <= 5'h0;
1182 end else
1183 if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.load_next) begin
1184     field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value <=
field_combo.HMAC512_KV_RD_BLOCK_CTRL.read_entry.next;
1185 end
1186 end
1187 assign hwif_out.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value;
1188 // Field:
hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend

```

```

1182     always_comb begin
1183         automatic logic [0:0] next_c;
1184         automatic logic load_next_c;
1185         next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value;
1186         load_next_c = '0';
1187         if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.swwe) begin // SW
write
1188             next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1189         end
1190         load_next_c = '1';
1191     end
1192     field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.next = next_c;
1193
1194 field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.load_next =
load_next_c;
1195 end
1196 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1197 if(~hwif_in.reset_b) begin
1198     field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value <=
1'h0;
1199 end else
1200 if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.load_next)
begin
1201     field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value <=
field_combo.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.next;
1202 end
1203 end
1204 assign
hwif_out.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value;
1205 // Field: hmac_reg.HMAC512_KV_RD_BLOCK_CTRL.rsvd
1206 always_comb begin
1207     automatic logic [24:0] next_c;
1208     automatic logic load_next_c;
1209     next_c =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value;
1210     load_next_c = '0';
1211     if(decoded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_RD_BLOCK_CTRL.rsvd.swwe) be-
gin // SW write
1212         next_c =
(field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
1213     end
1214     load_next_c = '1';
1215 end
1216 field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.next =
next_c;
1217
1218 field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.load_next =
load_next_c;
1219 end
1220 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1221 if(~hwif_in.reset_b) begin
1222     field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value
<= 25'h0;
1223 end else
1224 if(field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.load_next) begin
1225     field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value <=
field_combo.HMAC512_KV_RD_BLOCK_CTRL.rsvd.next;
1226 end
1227 end
1228 assign hwif_out.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value =
field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value;
1229 // Field: hmac_reg.HMAC512_KV_RD_BLOCK_STATUS.VALID
1230 always_comb begin
1231     automatic logic [0:0] next_c;
1232     automatic logic load_next_c;
1233     next_c =
field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.value;
1234     load_next_c = '0';
1235     if(hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwset) be-
gin // HW Set
1236         next_c = '1';
1237     end else
1238 if(hwif_in.HMAC512_KV_RD_BLOCK_STATUS.VALID.hwc1r) begin // HW Clear
1239     next_c = '0';
1240     load_next_c = '1';
1241 end
1242 end
1243 field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.next =
next_c;
1244
1245 field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.load_next
= load_next_c;
1246 end
1247 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1248 if(~hwif_in.reset_b) begin
1249     field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.value <= 1'h0;
1250 end else
1251 if(field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.load_next) begin
1252     field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.value <=
field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.next

```



```

field_combo.HMAC512_KV_RD_BLOCK_STATUS.VALID.next;
1244     end
1245   end
1246   // Field: hmac_reg.HMAC512_KV_WR_CTRL.write_en
1247   always_comb begin
1248     automatic logic [0:0] next_c;
1249     automatic logic load_next_c;
1250     next_c =
field_storage.HMAC512_KV_WR_CTRL.write_en.value;
1251     load_next_c = '0;
1252     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_WR_CTRL.write_en.swwe) be-
gin // SW write
1253       next_c =
(field_storage.HMAC512_KV_WR_CTRL.write_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1254       load_next_c = '1;

1255     end else if(hwif_in.HMAC512_KV_WR_CTRL.write_en.hwclr)
begin // HW Clear
1256       next_c = '0;
1257       load_next_c = '1;
1258     end
1259     field_combo.HMAC512_KV_WR_CTRL.write_en.next = next_c;
1260     field_combo.HMAC512_KV_WR_CTRL.write_en.load_next =
load_next_c;
1261   end
1262   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1263     if(~hwif_in.reset_b) begin

1264       field_storage.HMAC512_KV_WR_CTRL.write_en.value <=
1'h0;
1265     end else
if(field_combo.HMAC512_KV_WR_CTRL.write_en.load_next) begin

1266       field_storage.HMAC512_KV_WR_CTRL.write_en.value <=
field_combo.HMAC512_KV_WR_CTRL.write_en.next;
1267     end
1268   end
1269   assign hwif_out.HMAC512_KV_WR_CTRL.write_en.value =
field_storage.HMAC512_KV_WR_CTRL.write_en.value;
1270   // Field: hmac_reg.HMAC512_KV_WR_CTRL.write_entry
1271   always_comb begin
1272     automatic logic [4:0] next_c;
1273     automatic logic load_next_c;
1274     next_c =
field_storage.HMAC512_KV_WR_CTRL.write_entry.value;
1275     load_next_c = '0;
1276     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.HMAC512_KV_WR_CTRL.write_entry.swwe) be-
gin // SW write
1277       next_c =
(field_storage.HMAC512_KV_WR_CTRL.write_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
1278     load_next_c = '1;
1279   end
1280   field_combo.HMAC512_KV_WR_CTRL.write_entry.next =
next_c;

1281   field_combo.HMAC512_KV_WR_CTRL.write_entry.load_next =
load_next_c;
1282   end
1283   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1284     if(~hwif_in.reset_b) begin

1285       field_storage.HMAC512_KV_WR_CTRL.write_entry.value
<= 5'h0;
1286     end else
if(field_combo.HMAC512_KV_WR_CTRL.write_entry.load_next) begin

1287       field_storage.HMAC512_KV_WR_CTRL.write_entry.value
<= field_combo.HMAC512_KV_WR_CTRL.write_entry.next;
1288     end
1289   end
1290   assign hwif_out.HMAC512_KV_WR_CTRL.write_entry.value =
field_storage.HMAC512_KV_WR_CTRL.write_entry.value;
1291   // Field: hmac_reg.HMAC512_KV_WR_CTRL.hmac_key_dest_valid
1292   always_comb begin
1293     automatic logic [0:0] next_c;
1294     automatic logic load_next_c;
1295     next_c =
field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value;
1296     load_next_c = '0;
1297     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.swwe) begin // SW
write
1298       next_c =
(field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
1299     load_next_c = '1;
1300   end
1301   field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.next = next_c;
1302   field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.load_next =
load_next_c;
1303   end

```

```

1304   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1305     if(~hwif_in.reset_b) begin

1306       field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value <= 1'h0;
1307     end else
if(field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.load_next) be-
gin
1308       field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.next;
1309     end
1310   end
1311   assign
hwif_out.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value;
1312   // Field:
hmac_reg.HMAC512_KV_WR_CTRL.hmac_block_dest_valid
1313   always_comb begin
1314     automatic logic [0:0] next_c;
1315     automatic logic load_next_c;
1316     next_c =
field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value;
1317     load_next_c = '0;
1318     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.swwe) begin // SW
write
1319       next_c =
(field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value & ~de-
coded_wr_biten[7:7]) | (decoded_wr_data[7:7] & de-
coded_wr_biten[7:7]);
1320     load_next_c = '1;
1321   end
1322   field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.next = next_c;
1323   field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.load_next =
load_next_c;
1324   end
1325   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1326     if(~hwif_in.reset_b) begin

1327       field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value <=
1'h0;
1328     end else
if(field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.load_next)
begin
1329       field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.next;
1330     end
1331   end
1332   assign
hwif_out.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value;
1333   // Field:
hmac_reg.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid
1334   always_comb begin
1335     automatic logic [0:0] next_c;
1336     automatic logic load_next_c;
1337     next_c =
field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1338     load_next_c = '0;
1339     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.swwe) begin // SW
write
1340       next_c =
(field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value & ~de-
coded_wr_biten[8:8]) | (decoded_wr_data[8:8] & de-
coded_wr_biten[8:8]);
1341     load_next_c = '1;
1342   end
1343   field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.next = next_c;
1344   field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.load_next =
load_next_c;
1345   end
1346   always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1347     if(~hwif_in.reset_b) begin

1348       field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value <=
1'h0;
1349     end else
if(field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.load_next)
begin
1350       field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.next;
1351     end
1352   end
1353   assign
hwif_out.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.mlds_seed_dest_valid.value;
1354   // Field: hmac_reg.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid
1355   always_comb begin
1356     automatic logic [0:0] next_c;
1357     automatic logic load_next_c;
1358     next_c =
field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value;

```

```

1359     load_next_c = '0';
1360     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.swwe) begin // SW
write
1361         next_c =
(field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
1362         load_next_c = '1';
1363     end
1364     field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.next = next_c;
1365     field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next =
load_next_c;
1366 end
1367 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1368     if(~hwif_in.reset_b) begin
1369         field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value <= '1'h0;
1370     end else
1371     if(field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.load_next) be-
gin
1372         field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.next;
1373     end
1374     assign
hwif_out.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value;
1375     // Field: hmac_reg.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid
1376     always_comb begin
1377         automatic logic [0:0] next_c;
1378         automatic logic load_next_c;
1379         next_c =
field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1380         load_next_c = '0';
1381         if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.swwe) begin // SW
write
1382             next_c =
(field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value & ~de-
coded_wr_biten[10:10]) | (decoded_wr_data[10:10] & de-
coded_wr_biten[10:10]);
1383             load_next_c = '1';
1384         end
1385         field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.next = next_c;
1386         field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.load_next =
load_next_c;
1387     end
1388     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1389         if(~hwif_in.reset_b) begin
1390             field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value <= '1'h0;
1391         end else
1392         if(field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.load_next) be-
gin
1393             field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.next;
1394         end
1395         assign
hwif_out.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value;
1396         // Field: hmac_reg.HMAC512_KV_WR_CTRL.aes_key_dest_valid
1397         always_comb begin
1398             automatic logic [0:0] next_c;
1399             automatic logic load_next_c;
1400             next_c =
field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value;
1401             load_next_c = '0';
1402             if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.aes_key_dest_valid.swwe) begin // SW
write
1403                 next_c =
(field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value & ~de-
coded_wr_biten[11:11]) | (decoded_wr_data[11:11] & de-
coded_wr_biten[11:11]);
1404                 load_next_c = '1';
1405             end
1406             field_combo.HMAC512_KV_WR_CTRL.aes_key_dest_valid.next =
next_c;
1407             field_combo.HMAC512_KV_WR_CTRL.aes_key_dest_valid.load_next =
load_next_c;
1408         end
1409         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1410             if(~hwif_in.reset_b) begin
1411                 field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value <= '1'h0;
1412             end else
1413             if(field_combo.HMAC512_KV_WR_CTRL.aes_key_dest_valid.load_next) be-
gin

```

```

1414                 end
1415             end
1416             assign
hwif_out.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value;
1417         // Field:
hmac_reg.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid
1418         always_comb begin
1419             automatic logic [0:0] next_c;
1420             automatic logic load_next_c;
1421             next_c =
field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1422             load_next_c = '0';
1423             if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.swwe) begin // SW
write
1424                 next_c =
(field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value & ~de-
coded_wr_biten[12:12]) | (decoded_wr_data[12:12] & de-
coded_wr_biten[12:12]);
1425                 load_next_c = '1';
1426             end
1427             field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.next = next_c;
1428             field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next =
load_next_c;
1429         end
1430         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1431             if(~hwif_in.reset_b) begin
1432                 field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
'1'h0;
1433             end else
1434             if(field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.load_next)
begin
1435                 field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.next;
1436             end
1437             assign
hwif_out.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value;
1438             // Field: hmac_reg.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid
1439             always_comb begin
1440                 automatic logic [0:0] next_c;
1441                 automatic logic load_next_c;
1442                 next_c =
field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1443                 load_next_c = '0';
1444                 if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.swwe) begin // SW
write
1445                     next_c =
(field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value & ~de-
coded_wr_biten[13:13]) | (decoded_wr_data[13:13] & de-
coded_wr_biten[13:13]);
1446                     load_next_c = '1';
1447                 end
1448                 field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.next = next_c;
1449                 field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next =
load_next_c;
1450             end
1451             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1452                 if(~hwif_in.reset_b) begin
1453                     field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value <= '1'h0;
1454                 end else
1455                 if(field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.load_next)
begin
1456                     field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value <=
field_combo.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.next;
1457                 end
1458                 assign
hwif_out.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value =
field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value;
1459                 // Field: hmac_reg.HMAC512_KV_WR_CTRL.dma_data_dest_valid
1460                 always_comb begin
1461                     automatic logic [0:0] next_c;
1462                     automatic logic load_next_c;
1463                     next_c =
field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value;
1464                     load_next_c = '0';
1465                     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
coded_req_is_wr &&
hwif_in.HMAC512_KV_WR_CTRL.dma_data_dest_valid.swwe) begin // SW
write
1466                         next_c =
(field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value & ~de-
coded_wr_biten[14:14]) | (decoded_wr_data[14:14] & de-
coded_wr_biten[14:14]);

```

```

1467         load_next_c = '1';
1468     end
1469 field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.next = next_c;
1470 field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.load_next =
1471 load_next_c;
1472     end
1473     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1474         if(~hwif_in.reset_b) begin
1475             field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value <= 1'h0;
1476         end else
1477             if(field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.load_next) be-
1478 gin
1479             field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value <=
1480 field_combo.HMAC512_KV_WR_CTRL.dma_data_dest_valid.next;
1481         end
1482     end
1483     assign
1484 hwif_out.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value =
1485 field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value;
1486 // Field: hmac_reg.HMAC512_KV_WR_CTRL.rsvd
1487 always_comb begin
1488     automatic logic [16:0] next_c;
1489     automatic logic load_next_c;
1490     next_c = field_storage.HMAC512_KV_WR_CTRL.rsvd.value;
1491     load_next_c = '0;
1492     if(decoded_reg_strb.HMAC512_KV_WR_CTRL && de-
1493 coded_req_is_wr && hwif_in.HMAC512_KV_WR_CTRL.rsvd.swwe) begin // SW
1494 write
1495         next_c =
1496 (field_storage.HMAC512_KV_WR_CTRL.rsvd.value & ~de-
1497 coded_wr_biten[31:15]) | (decoded_wr_data[31:15] & de-
1498 coded_wr_biten[31:15]);
1499     end
1500     load_next_c = '1;
1501 end
1502 field_combo.HMAC512_KV_WR_CTRL.rsvd.next = next_c;
1503 field_combo.HMAC512_KV_WR_CTRL.rsvd.load_next =
1504 load_next_c;
1505     end
1506     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1507         if(~hwif_in.reset_b) begin
1508             field_storage.HMAC512_KV_WR_CTRL.rsvd.value <=
1509 17'h0;
1510         end else
1511             if(field_combo.HMAC512_KV_WR_CTRL.rsvd.load_next) begin
1512                 field_storage.HMAC512_KV_WR_CTRL.rsvd.value <=
1513 field_combo.HMAC512_KV_WR_CTRL.rsvd.next;
1514             end
1515         end
1516     end
1517     assign hwif_out.HMAC512_KV_WR_CTRL.rsvd.value =
1518 field_storage.HMAC512_KV_WR_CTRL.rsvd.value;
1519 // Field: hmac_reg.HMAC512_KV_WR_STATUS.VALID
1520 always_comb begin
1521     automatic logic [0:0] next_c;
1522     automatic logic load_next_c;
1523     next_c =
1524 field_storage.HMAC512_KV_WR_STATUS.VALID.value;
1525     load_next_c = '0;
1526     if(hwif_in.HMAC512_KV_WR_STATUS.VALID.hwset) begin //
1527 HW Set
1528         next_c = '1;
1529         load_next_c = '1;
1530     end else if(hwif_in.HMAC512_KV_WR_STATUS.VALID.hwclr)
1531 begin // HW Clear
1532         next_c = '0;
1533         load_next_c = '1;
1534     end
1535     end
1536     field_combo.HMAC512_KV_WR_STATUS.VALID.next = next_c;
1537     field_combo.HMAC512_KV_WR_STATUS.VALID.load_next =
1538 load_next_c;
1539     end
1540     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1541         if(~hwif_in.reset_b) begin
1542             field_storage.HMAC512_KV_WR_STATUS.VALID.value <=
1543 1'h0;
1544         end else
1545             if(field_combo.HMAC512_KV_WR_STATUS.VALID.load_next) begin
1546                 field_storage.HMAC512_KV_WR_STATUS.VALID.value <=
1547 field_combo.HMAC512_KV_WR_STATUS.VALID.next;
1548             end
1549         end
1550     end
1551 // Field: hmac_reg.intr_block_rf.global_intr_en_r.error_en
1552 always_comb begin
1553     automatic logic [0:0] next_c;
1554     automatic logic load_next_c;
1555     next_c =
1556 field_storage.intr_block_rf.global_intr_en_r.error_en.value;
1557     load_next_c = '0;
1558     if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
1559 decoded_req_is_wr) begin // SW write
1560         next_c =
1561 (field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
1562 coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
1563 coded_wr_biten[0:0]);
1564     end
1565     load_next_c = '1;
1566 end
1567 field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;

```

```

1535
1536 field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
1537 load_next_c;
1538     end
1539     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1540         if(~hwif_in.reset_b) begin
1541             field_storage.intr_block_rf.global_intr_en_r.error_en.value <= 1'h0;
1542         end else
1543             if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
1544 begin
1545             field_storage.intr_block_rf.global_intr_en_r.error_en.value <=
1546 field_combo.intr_block_rf.global_intr_en_r.error_en.next;
1547         end
1548     end
1549 // Field: hmac_reg.intr_block_rf.global_intr_en_r.notif_en
1550 always_comb begin
1551     automatic logic [0:0] next_c;
1552     automatic logic load_next_c;
1553     next_c =
1554 field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
1555     load_next_c = '0;
1556     if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
1557 decoded_req_is_wr) begin // SW write
1558         next_c =
1559 (field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
1560 coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
1561 coded_wr_biten[1:1]);
1562     end
1563     load_next_c = '1;
1564 end
1565 field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;
1566     end
1567     field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
1568 load_next_c;
1569     end
1570     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1571         if(~hwif_in.reset_b) begin
1572             field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= 1'h0;
1573         end else
1574             if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
1575 begin
1576             field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
1577 field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
1578         end
1579     end
1580 // Field: hmac_reg.intr_b-
1581 lock_rf.error_intr_en_r.key_mode_error_en
1582 always_comb begin
1583     automatic logic [0:0] next_c;
1584     automatic logic load_next_c;
1585     next_c =
1586 field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value;
1587     load_next_c = '0;
1588     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
1589 decoded_req_is_wr) begin // SW write
1590         next_c =
1591 (field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value
1592 & ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
1593 coded_wr_biten[0:0]);
1594     end
1595     load_next_c = '1;
1596 end
1597 field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.next =
1598 next_c;
1599     end
1600     field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.load_ne
1601 xt = load_next_c;
1602     end
1603     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1604         if(~hwif_in.reset_b) begin
1605             field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value <= 1'h0;
1606         end else
1607             if(field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.load_
1608 next) begin
1609                 field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value
1610 <= field_combo.intr_block_rf.error_intr_en_r.key_mode_error_en.next;
1611             end
1612         end
1613     end
1614 // Field: hmac_reg.intr_b-
1615 lock_rf.error_intr_en_r.key_zero_error_en
1616 always_comb begin
1617     automatic logic [0:0] next_c;
1618     automatic logic load_next_c;
1619     next_c =
1620 field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value;
1621     load_next_c = '0;
1622     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
1623 decoded_req_is_wr) begin // SW write
1624         next_c =
1625 (field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value
1626 & ~decoded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
1627 coded_wr_biten[1:1]);
1628     end
1629     load_next_c = '1;
1630 end
1631 field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.next = next_c;

```

```

1594 field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.next =
1595     next_c;
1596 field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.load_ne
1597 xt = load_next_c;
1598     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1599         if(~hwif_in.reset_b) begin
1600             field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value
1601                 <= '1'h0;
1602             end else
1603             if(field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.load_
1604 next) begin
1605                 field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value
1606                 <= field_combo.intr_block_rf.error_intr_en_r.key_zero_error_en.next;
1607             end
1608         end
1609         // Field: hmac_reg.intr_block_rf.error_intr_en_r.error2_en
1610         always_comb begin
1611             automatic logic [0:0] next_c;
1612             automatic logic load_next_c;
1613             next_c =
1614 field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
1615             load_next_c = '0';
1616             if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
1617 decoded_req_is_wr) begin // SW write
1618                 next_c =
1619 (field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
1620 coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
1621 coded_wr_biten[2:2]);
1622                 load_next_c = '1';
1623             end
1624         end
1625         field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;
1626         field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
1627         load_next_c;
1628         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1629             if(~hwif_in.reset_b) begin
1630                 field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= '1'h0;
1631             end else
1632             if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
1633 begin
1634                 field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
1635 field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
1636             end
1637         end
1638         // Field: hmac_reg.intr_block_rf.error_intr_en_r.error3_en
1639         always_comb begin
1640             automatic logic [0:0] next_c;
1641             automatic logic load_next_c;
1642             next_c =
1643 field_storage.intr_block_rf.error_intr_en_r.error3_en.value;
1644             load_next_c = '0';
1645             if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
1646 decoded_req_is_wr) begin // SW write
1647                 next_c =
1648 (field_storage.intr_block_rf.error_intr_en_r.error3_en.value & ~de-
1649 coded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
1650 coded_wr_biten[3:3]);
1651                 load_next_c = '1';
1652             end
1653         end
1654         field_combo.intr_block_rf.error_intr_en_r.error3_en.next = next_c;
1655         field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next =
1656         load_next_c;
1657         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1658             if(~hwif_in.reset_b) begin
1659                 field_storage.intr_block_rf.error_intr_en_r.error3_en.value <= '1'h0;
1660             end else
1661             if(field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next)
1662 begin
1663                 field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=
1664 field_combo.intr_block_rf.error_intr_en_r.error3_en.next;
1665             end
1666         end
1667         // Field: hmac_reg.intr_b-
1668 lock_rf.notif_intr_en_r.notif_cmd_done_en
1669         always_comb begin
1670             automatic logic [0:0] next_c;
1671             automatic logic load_next_c;
1672             next_c =
1673 field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value;
1674             load_next_c = '0';
1675             if(decoded_reg_strb.intr_block_rf.notif_intr_en_r &&
1676 decoded_req_is_wr) begin // SW write
1677                 next_c =
1678 (field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value & ~de-
1679 coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
1680 coded_wr_biten[0:0]);
1681                 load_next_c = '1';

```

```

1653     end
1654
1655 field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next =
next_c;
1656
1657 field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_ne
xt = load_next_c;
1658     end
1659     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1660         if(~hwif_in.reset_b) begin
1661             field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= 1'h0;
1662         end else
1663             if(field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_
next) begin
1664                 field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next;
1665             end
1666         end
1667         // Field: hmac_reg.intr_b-
lock_rf.error_global_intr_r.agg_sts
1668         always_comb begin
1669             automatic logic [0:0] next_c;
1670             automatic logic load_next_c;
1671             next_c =
1672             field_storage.intr_block_rf.error_global_intr_r.agg_sts.value;
1673             load_next_c = '0;
1674
1675             // HW Write
1676             next_c = hwif_out.intr_b-
lock_rf.error_internal_intr_r.intr;
1677             load_next_c = '1;
1678         end
1679         field_combo.intr_block_rf.error_global_intr_r.agg_sts.next = next_c;
1680     end
1681     field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next =
load_next_c;
1682     end
1683     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1684         if(~hwif_in.reset_b) begin
1685             field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1'h0;
1686         end else
1687             if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
begin
1688                 field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
1689             end
1690         end
1691         assign hwif_out.intr_block_rf.error_global_intr_r.intr =
1692         |
1693         (field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.global_intr_en_r.error_en.value);
1694         // Field: hmac_reg.intr_b-
lock_rf.notif_global_intr_r.agg_sts
1695         always_comb begin
1696             automatic logic [0:0] next_c;
1697             automatic logic load_next_c;
1698             next_c =
1699             field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
1700             load_next_c = '0;
1701
1702             // HW Write
1703             next_c = hwif_out.intr_b-
lock_rf.notif_internal_intr_r.intr;
1704             load_next_c = '1;
1705         end
1706         field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
1707     end
1708     field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
load_next_c;
1709     end
1710     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1711         if(~hwif_in.reset_b) begin
1712             field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1'h0;
1713         end else
1714             if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
begin
1715                 field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
1716             end
1717         end
1718         assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
1719         |
1720         (field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.global_intr_en_r.notif_en.value);
1721         // Field: hmac_reg.intr_b-
lock_rf.error_internal_intr_r.key_mode_error_sts
1722         always_comb begin
1723             automatic logic [0:0] next_c;
1724             automatic logic load_next_c;
1725             next_c =
1726             field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
.value;

```

```

1713     load_next_c = '0';
1714
1715 if(field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig
1716 .value != '0') begin // stickybit
1717     next_c =
1718 field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
1719 .value |
1720 field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
1721 alue;
1722     load_next_c = '1';
1723 end else if(hwif_in.intr_b-
1724 lock_rf.error_internal_intr_r.key_mode_error_sts.hwset) begin // HW
1725 Set
1726     next_c = '1';
1727     load_next_c = '1';
1728 end else if(de-
1729 coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
1730 coded_req_is_wr) begin // SW write 1 clear
1731     next_c =
1732 field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
1733 .value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1734     load_next_c = '1';
1735 end
1736 field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_sts.
1737 next = next_c;
1738
1739 field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_sts.
1740 load_next = load_next_c;
1741 end
1742 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1743 begin
1744     if(~hwif_in.error_reset_b) begin
1745 field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
1746 .value <= 1'h0;
1747     end else
1748 if(field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_s
1749 ts.load_next) begin
1750     field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
1751 .value <=
1752 field_combo.intr_block_rf.error_internal_intr_r.key_mode_error_sts.
1753 next;
1754     end
1755 end
1756 // Field: hmac_reg.intr_b-
1757 lock_rf.error_internal_intr_r.key_zero_error_sts
1758 always_comb begin
1759     automatic logic [0:0] next_c;
1760     automatic logic load_next_c;
1761     next_c =
1762 field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
1763 .value;
1764     load_next_c = '0';
1765
1766 if(field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig
1767 .value != '0') begin // stickybit
1768     next_c =
1769 field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
1770 .value |
1771 field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
1772 alue;
1773     load_next_c = '1';
1774 end else if(hwif_in.intr_b-
1775 lock_rf.error_internal_intr_r.key_zero_error_sts.hwset) begin // HW
1776 Set
1777     next_c = '1';
1778     load_next_c = '1';
1779 end else if(de-
1780 coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
1781 coded_req_is_wr) begin // SW write 1 clear
1782     next_c =
1783 field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
1784 .value & ~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1785     load_next_c = '1';
1786 end
1787
1788 field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
1789 next = next_c;
1790
1791 field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
1792 load_next = load_next_c;
1793 end
1794 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1795 begin
1796     if(~hwif_in.error_reset_b) begin
1797 field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
1798 .value <= 1'h0;
1799     end else
1800 if(field_combo.intr_block_rf.error_internal_intr_r.key_zero_s
1801 ts.load_next) begin
1802     field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
1803 .value <=
1804 field_combo.intr_block_rf.error_internal_intr_r.key_zero_error_sts.
1805 next;
1806     end
1807 end
1808 // Field: hmac_reg.intr_b-

```

```

lock_rf.error_internal_intr_r.error2_sts
1761     always_comb begin
1762         automatic logic [0:0] next_c;
1763         automatic logic load_next_c;
1764         next_c =
1765 field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
1766         load_next_c = '0';
1767
1768 if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !=
1769 '0') begin // stickybit
1770     next_c =
1771 field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
1772 field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1773     load_next_c = '1';
1774 end else if(hwif_in.intr_b-
1775 lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1776     next_c = '1';
1777     load_next_c = '1';
1778 end else if(de-
1779 coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
1780 coded_req_is_wr) begin // SW write 1 clear
1781     next_c =
1782 field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
1783 ~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1784     load_next_c = '1';
1785 end
1786
1787 field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
1788 next_c;
1789
1790 field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next
1791 = load_next_c;
1792 end
1793 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1794 begin
1795     if(~hwif_in.error_reset_b) begin
1796 field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
1797 <= 1'h0;
1798     end else
1799 if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
1800 next) begin
1801     field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
1802 <= field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1803     end
1804 end
1805 // Field: hmac_reg.intr_b-
1806 lock_rf.error_internal_intr_r.error3_sts
1807 always_comb begin
1808     automatic logic [0:0] next_c;
1809     automatic logic load_next_c;
1810     next_c =
1811 field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1812     load_next_c = '0';
1813
1814 if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !=
1815 '0') begin // stickybit
1816     next_c =
1817 field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
1818 field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1819     load_next_c = '1';
1820 end else if(hwif_in.intr_b-
1821 lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1822     next_c = '1';
1823     load_next_c = '1';
1824 end else if(de-
1825 coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
1826 coded_req_is_wr) begin // SW write 1 clear
1827     next_c =
1828 field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &
1829 ~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1830     load_next_c = '1';
1831 end
1832
1833 field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
1834 next_c;
1835
1836 field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next
1837 = load_next_c;
1838 end
1839 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1840 begin
1841     if(~hwif_in.error_reset_b) begin
1842 field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
1843 <= 1'h0;
1844     end else
1845 if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_
1846 next) begin
1847     field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
1848 <= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1849     end
1850 end
1851
1852 assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
1853 |
1854 (field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_s
1855 ts.value &
1856 field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value)
1857 || |

```



```

(field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_s
ts.value &
field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value)
1815    || |
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1816    || |
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1817    // Field: hmac_reg.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts
1818    always_comb begin
1819        automatic logic [0:0] next_c;
1820        automatic logic load_next_c;
1821        next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1822        load_next_c = '0;
1823
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0) begin // stickybit
1824    next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1825        load_next_c = '1;
1826    end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1827        next_c = '1;
1828        load_next_c = '1;
1829    end else if(de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1830        next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1831        load_next_c = '1;
1832    end
1833
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next = next_c;
1834
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
load_next = load_next_c;
1835    end
1836    always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1837        if(~hwif_in.reset_b) begin
1838
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= 1'h0;
1839        end else
if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.load_next) begin
1840
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts.
next;
1841        end
1842    end
1843    assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
1844    |
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1845    // Field: hmac_reg.intr_b-
lock_rf.error_intr_trig_r.key_mode_error_trig
1846    always_comb begin
1847        automatic logic [0:0] next_c;
1848        automatic logic load_next_c;
1849        next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue;
1850        load_next_c = '0;

1851    if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1852        next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1853        load_next_c = '1;
1854    end else begin // singlepulse clears back to 0
1855        next_c = '0;
1856        load_next_c = '1;
1857    end
1858
field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.next
= next_c;
1859
field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.loa
d_next = load_next_c;
1860    end
1861    always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1862        if(~hwif_in.reset_b) begin
1863
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue <= 1'h0;
1864        end else

```

```

if(field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.
load_next) begin
1865
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue <=
field_combo.intr_block_rf.error_intr_trig_r.key_mode_error_trig.next
;
1866    end
1867    end
1868    // Field: hmac_reg.intr_b-
lock_rf.error_intr_trig_r.key_zero_error_trig
1869    always_comb begin
1870        automatic logic [0:0] next_c;
1871        automatic logic load_next_c;
1872        next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue;
1873        load_next_c = '0;

1874    if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1875        next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue | (decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1876        load_next_c = '1;
1877    end else begin // singlepulse clears back to 0
1878        next_c = '0;
1879        load_next_c = '1;
1880    end
1881
field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.next
= next_c;
1882
field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.loa
d_next = load_next_c;
1883    end
1884    always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1885        if(~hwif_in.reset_b) begin
1886
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue <= 1'h0;
1887    end else
if(field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.
load_next) begin
1888
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue <=
field_combo.intr_block_rf.error_intr_trig_r.key_zero_error_trig.next
;
1889    end
1890    end
1891    // Field: hmac_reg.intr_b-
lock_rf.error_intr_trig_r.error2_trig
1892    always_comb begin
1893        automatic logic [0:0] next_c;
1894        automatic logic load_next_c;
1895        next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1896        load_next_c = '0;

1897    if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1898        next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1899        load_next_c = '1;
1900    end else begin // singlepulse clears back to 0
1901        next_c = '0;
1902        load_next_c = '1;
1903    end
1904
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
next_c;
1905
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
load_next_c;
1906    end
1907    always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1908        if(~hwif_in.reset_b) begin
1909
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1'h0;
1910    end else
if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
) begin
1911
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1912    end
1913    end
1914    // Field: hmac_reg.intr_b-
lock_rf.error_intr_trig_r.error3_trig
1915    always_comb begin
1916        automatic logic [0:0] next_c;
1917        automatic logic load_next_c;
1918        next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1919        load_next_c = '0;

1920    if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set

```

```

1921     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1922     load_next_c = '1';
1923     end else begin // singlepulse clears back to 0
1924         next_c = '0';
1925         load_next_c = '1';
1926     end
1927
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
next_c;
1928
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
load_next_c;
1929     end
1930     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1931         if(~hwif_in.reset_b) begin
1932
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1'h0;
1933         end else
if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
) begin
1934
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1935         end
1936     end
1937     // Field: hmac_reg.intr_b-
lock_rf.notif_intr_trig_r.notif_cmd_done_trig
1938     always_comb begin
1939         automatic logic [0:0] next_c;
1940         automatic logic load_next_c;
1941         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1942         load_next_c = '0';
1943
if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1944         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1945         load_next_c = '1';
1946         end else begin // singlepulse clears back to 0
1947             next_c = '0';
1948             load_next_c = '1';
1949         end
1950
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
= next_c;
1951
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig loa
d_next = load_next_c;
1952     end
1953     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1954         if(~hwif_in.reset_b) begin
1955
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <= 1'h0;
1956         end else
if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
load_next) begin
1957
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <=
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
;
1958         end
1959     end
1960     // Field: hmac_reg.intr_b-
lock_rf.key_mode_error_intr_count_r.cnt
1961     always_comb begin
1962         automatic logic [31:0] next_c;
1963         automatic logic load_next_c;
1964         next_c =
field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value;
1965         load_next_c = '0';
1966         if(de-
coded_reg_strb.intr_block_rf.key_mode_error_intr_count_r && de-
coded_req_is_wr) begin // SW write
1967             next_c =
(field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value &
~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1968             load_next_c = '1';
1969         end
1970
if(field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pul
se.value) begin // increment
1971         if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
1972             next_c = 32'hfffffff;
1973         end else begin
1974             next_c = next_c + 32'h1;
1975         end
1976         load_next_c = '1';
1977     end
1978
field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.incrthres
hold =

```

```

(field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value
>= 32'hfffffff);
1979
field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.incrsatur
ate =
(field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value
>= 32'hfffffff);
1980     if(next_c > 32'hfffffff) begin
1981         next_c = 32'hfffffff;
1982         load_next_c = '1';
1983     end
1984
field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.next =
next_c;
1985
field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.load_next
= load_next_c;
1986     end
1987     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1988         if(~hwif_in.error_reset_b) begin
1989
field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value <=
32'h0;
1990         end else
if(field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.load_n
ext) begin
1991
field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value <=
field_combo.intr_block_rf.key_mode_error_intr_count_r.cnt.next;
1992         end
1993     end
1994     // Field: hmac_reg.intr_b-
lock_rf.key_zero_error_intr_count_r.cnt
1995     always_comb begin
1996         automatic logic [31:0] next_c;
1997         automatic logic load_next_c;
1998         next_c =
field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value;
1999         load_next_c = '0';
2000         if(de-
coded_reg_strb.intr_block_rf.key_zero_error_intr_count_r && de-
coded_req_is_wr) begin // SW write
2001             next_c =
(field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value &
~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
2002             load_next_c = '1';
2003         end
2004
if(field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pul
se.value) begin // increment
2005         if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
2006             next_c = 32'hfffffff;
2007         end else begin
2008             next_c = next_c + 32'h1;
2009         end
2010         load_next_c = '1';
2011     end
2012
field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.incrthres
hold =
(field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value
>= 32'hfffffff);
2013
field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.incrsatur
ate =
(field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value
>= 32'hfffffff);
2014     if(next_c > 32'hfffffff) begin
2015         next_c = 32'hfffffff;
2016         load_next_c = '1';
2017     end
2018
field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.next =
next_c;
2019
field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.load_next
= load_next_c;
2020     end
2021     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
2022         if(~hwif_in.error_reset_b) begin
2023
field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value <=
32'h0;
2024         end else
if(field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.load_n
ext) begin
2025
field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value <=
field_combo.intr_block_rf.key_zero_error_intr_count_r.cnt.next;
2026         end
2027     end
2028     // Field: hmac_reg.intr_block_rf.error2_intr_count_r.cnt
always_comb begin
2029         automatic logic [31:0] next_c;
2030         automatic logic load_next_c;
2031         next_c =
field_storage.intr_block_rf.error2_intr_count_r.cnt.value;

```



```

2033     load_next_c = '0;
2034     if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
&& decoded_req_is_wr) begin // SW write
2035         next_c =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
2036         load_next_c = '1;
2037     end
2038 if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // increment
2039     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
2040         next_c = 32'hfffffff;
2041     end else begin
2042         next_c = next_c + 32'h1;
2043     end
2044     load_next_c = '1;
2045 end
2046 field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hfffffff);
2047 field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
32'hfffffff);
2048 if(next_c > 32'hfffffff) begin
2049     next_c = 32'hfffffff;
2050     load_next_c = '1;
2051 end

2052 field_combo.intr_block_rf.error2_intr_count_r.cnt.next
= next_c;
2053 field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
load_next_c;
2054 end
2055 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
2056     if(~hwif_in.error_reset_b) begin
2057         field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;
2058     end else
2059 if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
gin
2060     field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
2061 end
2062 end
2063 // Field: hmac_reg.intr_block_rf.error3_intr_count_r.cnt
2064 always_comb begin
2065     automatic logic [31:0] next_c;
2066     automatic logic load_next_c;
2067     next_c =
field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
2068     load_next_c = '0;
2069 if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
&& decoded_req_is_wr) begin // SW write
2070     next_c =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
2071     load_next_c = '1;
2072 end
2073 if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // increment
2074     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
2075         next_c = 32'hfffffff;
2076     end else begin
2077         next_c = next_c + 32'h1;
2078     end
2079     load_next_c = '1;
2080 end
2081 field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hfffffff);
2082 field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
(field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
32'hfffffff);
2083 if(next_c > 32'hfffffff) begin
2084     next_c = 32'hfffffff;
2085     load_next_c = '1;
2086 end
2087 field_combo.intr_block_rf.error3_intr_count_r.cnt.next
= next_c;
2088 field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
2089 end
2090 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
2091     if(~hwif_in.error_reset_b) begin

```

```

field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
2092     end else
2093 if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
2094     field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
2095 end
2096 // Field: hmac_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_r.cnt
2097 always_comb begin
2098     automatic logic [31:0] next_c;
2099     automatic logic load_next_c;
2100     next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
2101     load_next_c = '0;
2102 if(de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_req_is_wr) begin // SW write
2103     next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value & ~de-
coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
2104     load_next_c = '1;
2105 end
2106 if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul-
se.value) begin // increment
2107     if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
gin // up-counter saturated
2108         next_c = 32'hfffffff;
2109     end else begin
2110         next_c = next_c + 32'h1;
2111     end
2112     load_next_c = '1;
2113 end
2114 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres-
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value >=
32'hfffffff);
2115 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur-
ate =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value >=
32'hfffffff);
2116 if(next_c > 32'hfffffff) begin
2117     next_c = 32'hfffffff;
2118     load_next_c = '1;
2119 end
2120 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
2121 field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;
2122 end
2123 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2124     if(~hwif_in.reset_b) begin
2125         field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
32'h0;
2126     end else
2127 if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n-
ext) begin
2128     field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
2129 end
2130 // Field: hmac_reg.intr_b-
lock_rf.key_mode_error_intr_count_incr_r.pulse
2131 always_comb begin
2132     automatic logic [0:0] next_c;
2133     automatic logic load_next_c;
2134     next_c =
field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse-
value;
2135     load_next_c = '0;
2136 if(field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig-
.value) begin // HW Write - we
2137     next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v-
alue;
2138     load_next_c = '1;
2139 end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.key_mode_error_sts.hwset) begin // HW
Set
2140     next_c = '1;
2141     load_next_c = '1;
2142 end
2143 if(field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pul-
se.value) begin // decrement
2144     field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.un-
derflow = (next_c < (1'h1));
2145     next_c = next_c - 1'h1;
2146     load_next_c = '1;

```

```

2147     end else begin
2148
field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.underflow = '0';
2149     end
2150
field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.decrthreshold =
(field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.value <= '1'd0);
2151
field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.next = next_c;
2152
field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.load_next = load_next_c;
2153     end
2154     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2155         if(~hwif_in.reset_b) begin
2156
field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.value <= '1'h0;
2157         end else
2158         if(field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.load_next) begin
2159
field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.next;
2159     end
2160     end
2161     // Field: hmac_reg.intr_block_rf.key_zero_error_intr_count_incr_r.pulse
2162     always_comb begin
2163         automatic logic [0:0] next_c;
2164         automatic logic load_next_c;
2165         next_c =
field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.value;
2166         load_next_c = '0;
2167
if(field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_intr_count_incr_r.pulse.value) begin // HW Write - we
2168     next_c =
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_intr_count_incr_r.pulse.value;
2169     load_next_c = '1;
2170     end else if(hwif_in.intr_block_rf.error_internal_intr_r.key_zero_error_intr_count_incr_r.pulse.value) begin // HW Set
2171         next_c = '1;
2172         load_next_c = '1;
2173     end
2174
if(field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.value) begin // decrement
2175
field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.underflow = (next_c < (1'h1));
2176     next_c = next_c - 1'h1;
2177     load_next_c = '1;
2178     end else begin
2179
field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.underflow = '0;
2180     end
2181
field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.decrthreshold =
(field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.value <= '1'd0);
2182
field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.next = next_c;
2183
field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.load_next = load_next_c;
2184     end
2185     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2186         if(~hwif_in.reset_b) begin
2187
field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.value <= '1'h0;
2188         end else
2189         if(field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.load_next) begin
2190
field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.next;
2190     end
2191     end
2192     // Field: hmac_reg.intr_block_rf.error2_intr_count_incr_r.pulse
2193     always_comb begin
2194         automatic logic [0:0] next_c;
2195         automatic logic load_next_c;
2196         next_c =
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;

```

```

2197         load_next_c = '0;
2198
if(field_storage.intr_block_rf.error_intr_trig_r.error2_intr_count_incr_r.pulse.value)
begin // HW Write - we
2199     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_intr_count_incr_r.pulse.value;
2200     load_next_c = '1;
2201     end else if(hwif_in.intr_block_rf.error_internal_intr_r.error2_intr_count_incr_r.pulse.value) begin // HW Set
2202         next_c = '1;
2203         load_next_c = '1;
2204     end
2205
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
2206
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow = (next_c < (1'h1));
2207     next_c = next_c - 1'h1;
2208     load_next_c = '1;
2209     end else begin
2210
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow = '0;
2211     end
2212
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthreshold =
(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <= '1'd0);
2213
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next = next_c;
2214
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next = load_next_c;
2215     end
2216     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2217         if(~hwif_in.reset_b) begin
2218
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <= '1'h0;
2219         end else
2220         if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next) begin
2221
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
2221     end
2222     end
2223     // Field: hmac_reg.intr_block_rf.error3_intr_count_incr_r.pulse
2224     always_comb begin
2225         automatic logic [0:0] next_c;
2226         automatic logic load_next_c;
2227         next_c =
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
2228         load_next_c = '0;
2229
if(field_storage.intr_block_rf.error_intr_trig_r.error3_intr_count_incr_r.pulse.value)
begin // HW Write - we
2230     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_intr_count_incr_r.pulse.value;
2231     load_next_c = '1;
2232     end else if(hwif_in.intr_block_rf.error_internal_intr_r.error3_intr_count_incr_r.pulse.value) begin // HW Set
2233         next_c = '1;
2234         load_next_c = '1;
2235     end
2236
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // decrement
2237
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow = (next_c < (1'h1));
2238     next_c = next_c - 1'h1;
2239     load_next_c = '1;
2240     end else begin
2241
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow = '0;
2242     end
2243
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthreshold =
(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <= '1'd0);
2244
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next = next_c;
2245
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next = load_next_c;
2246     end
2247     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2248         if(~hwif_in.reset_b) begin
2249
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <= '1'h0;
2250         end else
2251         if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next)

```

```

xt) begin
2251
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
2252     end
2253 end
2254 // Field: hmac_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_incr_r.pulse
2255     always_comb begin
2256         automatic logic [0:0] next_c;
2257         automatic logic load_next_c;
2258         next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value;
2259         load_next_c = '0;
2260
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value) begin // HW Write - we
2261     next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
2262     load_next_c = '1;
2263     end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
2264         next_c = '1;
2265         load_next_c = '1;
2266     end
2267
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // decrement
2268
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = (next_c < (1'h1));
2269     next_c = next_c - 1'h1;
2270     load_next_c = '1;
2271     end else begin
2272
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = '0;
2273     end
2274
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
crthreshold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'd0);
2275
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt = next_c;
2276
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
ad_next = load_next_c;
2277     end
2278     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
2279         if(~hwif_in.reset_b) begin
2280
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'h0;
2281         end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
load_next) begin
2282
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt;
2283     end
2284     end
2285
2286 //-----
2287 // Write response
2288 //-----
2289 assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;
2290 // Writes are always granted with no error response
2291 assign cpuif_wr_err = '0;
2292
2293 //-----
2294 // Readback
2295 //-----
2296
2297 logic readback_err;
2298 logic readback_done;
2299 logic [31:0] readback_data;
2300
2301 // Assign readback values to a flattened array
2302 logic [46-1:0][31:0] readback_array;
2303 for(genvar i0=0; i0<2; i0++) begin
2304     assign readback_array[i0*1 + 0][31:0] = (de-
coded_reg_strb.HMAC512_NAME[i0] && !decoded_req_is_wr) ?
hwif_in.HMAC512_NAME[i0].NAME.next : '0;
2305     end
2306     for(genvar i0=0; i0<2; i0++) begin
2307         assign readback_array[i0*1 + 2][31:0] = (de-
coded_reg_strb.HMAC512_VERSION[i0] && !decoded_req_is_wr) ?
hwif_in.HMAC512_VERSION[i0].VERSION.next : '0;
2308     end
2309     assign readback_array[4][0:0] = (de-

```

```

coded_reg_strb.HMAC512_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_STATUS.READY.next : '0;
2310     assign readback_array[4][1:1] = (de-
coded_reg_strb.HMAC512_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_STATUS.VALID.next : '0;
2311     assign readback_array[4][31:2] = '0;
2312     for(genvar i0=0; i0<16; i0++) begin
2313         assign readback_array[i0*1 + 5][31:0] = (de-
coded_reg_strb.HMAC512_TAG[i0] && !decoded_req_is_wr) ?
field_storage.HMAC512_TAG[i0].TAG.value : '0;
2314     end
2315     assign readback_array[21][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_CTRL.read_en.value : '0;
2316     assign readback_array[21][5:1] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_CTRL.read_entry.value : '0;
2317     assign readback_array[21][6:6] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_CTRL.pcr_hash_extend.value : '0;
2318     assign readback_array[21][31:7] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_CTRL.rsvd.value : '0;
2319     assign readback_array[22][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_RD_KEY_STATUS.READY.next : '0;
2320     assign readback_array[22][1:1] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_KEY_STATUS.VALID.value : '0;
2321     assign readback_array[22][9:2] = (de-
coded_reg_strb.HMAC512_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_RD_KEY_STATUS.ERROR.next : '0;
2322     assign readback_array[22][31:10] = '0;
2323     assign readback_array[23][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_en.value : '0;
2324     assign readback_array[23][5:1] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_CTRL.read_entry.value : '0;
2325     assign readback_array[23][6:6] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_CTRL.pcr_hash_extend.value : '0;
2326     assign readback_array[23][31:7] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_CTRL.rsvd.value : '0;
2327     assign readback_array[24][0:0] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_RD_BLOCK_STATUS.READY.next : '0;
2328     assign readback_array[24][1:1] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_RD_BLOCK_STATUS.VALID.value : '0;
2329     assign readback_array[24][9:2] = (de-
coded_reg_strb.HMAC512_KV_RD_BLOCK_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_RD_BLOCK_STATUS.ERROR.next : '0;
2330     assign readback_array[24][31:10] = '0;
2331     assign readback_array[25][0:0] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.write_en.value : '0;
2332     assign readback_array[25][5:1] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.write_entry.value : '0;
2333     assign readback_array[25][6:6] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.hmac_key_dest_valid.value : '0;
2334     assign readback_array[25][7:7] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.hmac_block_dest_valid.value : '0;
2335     assign readback_array[25][8:8] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.mldsa_seed_dest_valid.value : '0;
2336     assign readback_array[25][9:9] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.ecc_pkey_dest_valid.value : '0;
2337     assign readback_array[25][10:10] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.ecc_seed_dest_valid.value : '0;
2338     assign readback_array[25][11:11] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.aes_key_dest_valid.value : '0;
2339     assign readback_array[25][12:12] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.mlkem_seed_dest_valid.value : '0;
2340     assign readback_array[25][13:13] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.mlkem_msg_dest_valid.value : '0;
2341     assign readback_array[25][14:14] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.dma_data_dest_valid.value : '0;
2342     assign readback_array[25][31:15] = (de-
coded_reg_strb.HMAC512_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_CTRL.rsvd.value : '0;
2343     assign readback_array[26][0:0] = (de-
coded_reg_strb.HMAC512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_WR_STATUS.READY.next : '0;
2344     assign readback_array[26][1:1] = (de-
coded_reg_strb.HMAC512_KV_WR_STATUS && !decoded_req_is_wr) ?
field_storage.HMAC512_KV_WR_STATUS.VALID.value : '0;
2345     assign readback_array[26][9:2] = (de-
coded_reg_strb.HMAC512_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.HMAC512_KV_WR_STATUS.ERROR.next : '0;
2346     assign readback_array[26][31:10] = '0;

```

```

2347 assign readback_array[27][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0';
2348 assign readback_array[27][1:1] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0';
2349 assign readback_array[27][31:2] = '0';
2350 assign readback_array[28][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.key_mode_error_en.value
: '0';
2351 assign readback_array[28][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.key_zero_error_en.value
: '0';
2352 assign readback_array[28][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0';
2353 assign readback_array[28][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0';
2354 assign readback_array[28][31:4] = '0';
2355 assign readback_array[29][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
: '0';
2356 assign readback_array[29][31:1] = '0';
2357 assign readback_array[30][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0';
2358 assign readback_array[30][31:1] = '0';
2359 assign readback_array[31][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0';
2360 assign readback_array[31][31:1] = '0';
2361 assign readback_array[32][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.key_mode_error_sts
.value : '0';
2362 assign readback_array[32][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.key_zero_error_sts
.value : '0';
2363 assign readback_array[32][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value :
'0';
2364 assign readback_array[32][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value :
'0';
2365 assign readback_array[32][31:4] = '0';
2366 assign readback_array[33][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value : '0';
2367 assign readback_array[33][31:1] = '0';
2368 assign readback_array[34][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.key_mode_error_trig.v
alue : '0';
2369 assign readback_array[34][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.key_zero_error_trig.v
alue : '0';
2370 assign readback_array[34][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value :
'0';
2371 assign readback_array[34][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value :
'0';
2372 assign readback_array[34][31:4] = '0';
2373 assign readback_array[35][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue : '0';
2374 assign readback_array[35][31:1] = '0';
2375 assign readback_array[36][31:0] = (de-
coded_reg_strb.intr_block_rf.key_mode_error_intr_count_r && !de-
coded_req_is_wr) ?

```

```

field_storage.intr_block_rf.key_mode_error_intr_count_r.cnt.value :
'0';
2376 assign readback_array[37][31:0] = (de-
coded_reg_strb.intr_block_rf.key_zero_error_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.key_zero_error_intr_count_r.cnt.value :
'0';
2377 assign readback_array[38][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0';
2378 assign readback_array[39][31:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0';
2379 assign readback_array[40][31:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value :
'0';
2380 assign readback_array[41][0:0] = (de-
coded_reg_strb.intr_block_rf.key_mode_error_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.key_mode_error_intr_count_incr_r.pulse.
value : '0';
2381 assign readback_array[41][31:1] = '0';
2382 assign readback_array[42][0:0] = (de-
coded_reg_strb.intr_block_rf.key_zero_error_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.key_zero_error_intr_count_incr_r.pulse.
value : '0';
2383 assign readback_array[42][31:1] = '0';
2384 assign readback_array[43][0:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value :
'0';
2385 assign readback_array[43][31:1] = '0';
2386 assign readback_array[44][0:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value :
'0';
2387 assign readback_array[44][31:1] = '0';
2388 assign readback_array[45][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value : '0';
2389 assign readback_array[45][31:1] = '0';
2390
2391 // Reduce the array
2392 always_comb begin
2393     automatic logic [31:0] readback_data_var;
2394     readback_done = decoded_req & ~decoded_req_is_wr;
2395     readback_err = '0';
2396     readback_data_var = '0';
2397     for(int i=0; i<46; i++) readback_data_var |= read-
back_array[i];
2398     readback_data = readback_data_var;
2399 end
2400
2401 assign cpuif_rd_ack = readback_done;
2402 assign cpuif_rd_data = readback_data;
2403 assign cpuif_rd_err = readback_err;
2404
2405 `include "caliptra_prim_assert.sv"
2406 `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
2407
2408 endmodule

```

6. chipsalliance_caliptra-rtl/src/hmac/rtl/hmac_reg_pkg.sv (450 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 package hmac_reg_pkg;
5
6     localparam HMAC_REG_DATA_WIDTH = 32;
7     localparam HMAC_REG_MIN_ADDR_WIDTH = 12;
8
9     typedef struct packed{
10         logic [31:0] next;
11     } hmac_reg_HMAC512_NAME_NAME_in_t;
12
13     typedef struct packed{
14         hmac_reg_HMAC512_NAME_NAME_in_t NAME;
15     } hmac_reg_HMAC512_NAME_in_t;
16
17     typedef struct packed{
18         logic [31:0] next;
19     } hmac_reg_HMAC512_VERSION_VERSION_in_t;
20
21     typedef struct packed{
22         hmac_reg_HMAC512_VERSION_VERSION_in_t VERSION;
23     } hmac_reg_HMAC512_VERSION_in_t;
24

```

```

25  typedef struct packed{
26      logic swwe;
27  } hmac_reg_HMAC512_CTRL_INIT_in_t;
28
29  typedef struct packed{
30      logic swwe;
31  } hmac_reg_HMAC512_CTRL_NEXT_in_t;
32
33  typedef struct packed{
34      logic swwe;
35  } hmac_reg_HMAC512_CTRL_MODE_in_t;
36
37  typedef struct packed{
38      logic swwe;
39  } hmac_reg_HMAC512_CTRL_CSR_MODE_in_t;
40
41  typedef struct packed{
42      logic swwe;
43  } hmac_reg_HMAC512_CTRL_LAST_in_t;
44
45  typedef struct packed{
46      hmac_reg_HMAC512_CTRL_INIT_in_t INIT;
47      hmac_reg_HMAC512_CTRL_NEXT_in_t NEXT;
48      hmac_reg_HMAC512_CTRL_MODE_in_t MODE;
49      hmac_reg_HMAC512_CTRL_CSR_MODE_in_t CSR_MODE;
50      hmac_reg_HMAC512_CTRL_LAST_in_t LAST;
51  } hmac_reg_HMAC512_CTRL_in_t;
52
53  typedef struct packed{
54      logic next;
55  } hmac_reg_HMAC512_STATUS_READY_in_t;
56
57  typedef struct packed{
58      logic next;
59  } hmac_reg_HMAC512_STATUS_VALID_in_t;
60
61  typedef struct packed{
62      hmac_reg_HMAC512_STATUS_READY_in_t READY;
63      hmac_reg_HMAC512_STATUS_VALID_in_t VALID;
64  } hmac_reg_HMAC512_STATUS_in_t;
65
66  typedef struct packed{
67      logic [31:0] next;
68      logic we;
69      logic swwe1;
70      logic hwclr;
71  } hmac_reg_HMAC512_KEY_KEY_in_t;
72
73  typedef struct packed{
74      hmac_reg_HMAC512_KEY_KEY_in_t KEY;
75  } hmac_reg_HMAC512_KEY_in_t;
76
77  typedef struct packed{
78      logic [31:0] next;
79      logic we;
80      logic swwe1;
81      logic hwclr;
82  } hmac_reg_HMAC512_BLOCK_BLOCK_in_t;
83
84  typedef struct packed{
85      hmac_reg_HMAC512_BLOCK_BLOCK_in_t BLOCK;
86  } hmac_reg_HMAC512_BLOCK_in_t;
87
88  typedef struct packed{
89      logic [31:0] next;
90      logic hwclr;
91  } hmac_reg_HMAC512_TAG_TAG_in_t;
92
93  typedef struct packed{
94      hmac_reg_HMAC512_TAG_TAG_in_t TAG;
95  } hmac_reg_HMAC512_TAG_in_t;
96
97  typedef struct packed{
98      logic swwe;
99      logic hwclr;
100  } kv_read_ctrl_reg_read_en_in_t;
101
102  typedef struct packed{
103      logic swwe;
104  } kv_read_ctrl_reg_read_entry_in_t;
105
106  typedef struct packed{
107      logic swwe;
108  } kv_read_ctrl_reg_pcr_hash_extend_in_t;
109
110  typedef struct packed{
111      logic swwe;
112  } kv_read_ctrl_reg_rsvd_in_t;
113
114  typedef struct packed{
115      kv_read_ctrl_reg_read_en_in_t read_en;
116      kv_read_ctrl_reg_read_entry_in_t read_entry;
117      kv_read_ctrl_reg_pcr_hash_extend_in_t
pcr_hash_extend;
118      kv_read_ctrl_reg_rsvd_in_t rsvd;
119  } kv_read_ctrl_reg_in_t;
120
121  typedef struct packed{
122      logic next;
123  } kv_status_reg_READY_in_t;
124

```

```

125  typedef struct packed{
126      logic hwclr;
127      logic hwset;
128  } kv_status_reg_VALID_in_t;
129
130  typedef struct packed{
131      logic [7:0] next;
132  } kv_status_reg_ERROR_in_t;
133
134  typedef struct packed{
135      kv_status_reg_READY_in_t READY;
136      kv_status_reg_VALID_in_t VALID;
137      kv_status_reg_ERROR_in_t ERROR;
138  } kv_status_reg_in_t;
139
140  typedef struct packed{
141      logic swwe;
142      logic hwclr;
143  } kv_write_ctrl_reg_write_en_in_t;
144
145  typedef struct packed{
146      logic swwe;
147  } kv_write_ctrl_reg_write_entry_in_t;
148
149  typedef struct packed{
150      logic swwe;
151  } kv_write_ctrl_reg_hmac_key_dest_valid_in_t;
152
153  typedef struct packed{
154      logic swwe;
155  } kv_write_ctrl_reg_hmac_block_dest_valid_in_t;
156
157  typedef struct packed{
158      logic swwe;
159  } kv_write_ctrl_reg_mldsa_seed_dest_valid_in_t;
160
161  typedef struct packed{
162      logic swwe;
163  } kv_write_ctrl_reg_ecc_pkey_dest_valid_in_t;
164
165  typedef struct packed{
166      logic swwe;
167  } kv_write_ctrl_reg_ecc_seed_dest_valid_in_t;
168
169  typedef struct packed{
170      logic swwe;
171  } kv_write_ctrl_reg_aes_key_dest_valid_in_t;
172
173  typedef struct packed{
174      logic swwe;
175  } kv_write_ctrl_reg_mlkem_seed_dest_valid_in_t;
176
177  typedef struct packed{
178      logic swwe;
179  } kv_write_ctrl_reg_mlkem_msg_dest_valid_in_t;
180
181  typedef struct packed{
182      logic swwe;
183  } kv_write_ctrl_reg_dma_data_dest_valid_in_t;
184
185  typedef struct packed{
186      logic swwe;
187  } kv_write_ctrl_reg_rsvd_in_t;
188
189  typedef struct packed{
190      kv_write_ctrl_reg_write_en_in_t write_en;
191      kv_write_ctrl_reg_write_entry_in_t write_entry;
192      kv_write_ctrl_reg_hmac_key_dest_valid_in_t
hmac_key_dest_valid;
193      kv_write_ctrl_reg_hmac_block_dest_valid_in_t hmac_b-
lock_dest_valid;
194      kv_write_ctrl_reg_mldsa_seed_dest_valid_in_t
mldsa_seed_dest_valid;
195      kv_write_ctrl_reg_ecc_pkey_dest_valid_in_t ecc_p-
key_dest_valid;
196      kv_write_ctrl_reg_ecc_seed_dest_valid_in_t
ecc_seed_dest_valid;
197      kv_write_ctrl_reg_aes_key_dest_valid_in_t aes_key_d-
est_valid;
198      kv_write_ctrl_reg_mlkem_seed_dest_valid_in_t
mlkem_seed_dest_valid;
199      kv_write_ctrl_reg_mlkem_msg_dest_valid_in_t
mlkem_msg_dest_valid;
200      kv_write_ctrl_reg_dma_data_dest_valid_in_t
dma_data_dest_valid;
201      kv_write_ctrl_reg_rsvd_in_t rsvd;
202  } kv_write_ctrl_reg_in_t;
203
204  typedef struct packed{
205      logic hwset;
206  } hmac_reg_er-
ror_in-
tr_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_mode_error_sts_enable_28f0af4b_next-
7027a86d_resetsignal_939e99d4_in_t;
207
208  typedef struct packed{
209      logic hwset;
210  } hmac_reg_er-
ror_in-

```



```

tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_zero_error_sts_enable_8d7a01ba_next_
938a4efa_resetsignal_939e99d4_in_t;
211
212     typedef struct packed{
213         logic hwset;
214     } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error2_sts_enable_0dacf7a6_next_4b5b9e74
_resetsignal_939e99d4_in_t;
215
216     typedef struct packed{
217         logic hwset;
218     } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error3_sts_enable_fc3af94b_next_c3125d40
_resetsignal_939e99d4_in_t;
219
220     typedef struct packed{
221         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_mode_error_sts_enable_28f0af4b_next_
7027a86d_resetsignal_939e99d4_in_t key_mode_error_sts;
222         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_key_zero_error_sts_enable_8d7a01ba_next_
938a4efa_resetsignal_939e99d4_in_t key_zero_error_sts;
223         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error2_sts_enable_0dacf7a6_next_4b5b9e74
_resetsignal_939e99d4_in_t error2_sts;
224         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_error3_sts_enable_fc3af94b_next_c3125d40
_resetsignal_939e99d4_in_t error3_sts;
225     } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_in_t;
226
227     typedef struct packed{
228         logic hwset;
229     } hmac_reg_no-
tif_in-
tr_t_no-
tif_cmd_done_sts_1c68637e_notif_cmd_done_sts_enable_dabe0b8b_next_5
40fa3b7_in_t;
230
231     typedef struct packed{
232         hmac_reg_no-
tif_in-
tr_t_no-
tif_cmd_done_sts_1c68637e_notif_cmd_done_sts_enable_dabe0b8b_next_5
40fa3b7_in_t notif_cmd_done_sts;
233     } hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e_in_t;
234
235     typedef struct packed{
236         hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_in_t error_internal_intr_r;
237         hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e_in_t notif_internal_intr_r;
238     } hmac_reg_intr_block_t_in_t;
239
240     typedef struct packed{
241         logic reset_b;
242         logic error_reset_b;
243         hmac_reg_HMAC512_NAME_in_t [2-1:0]HMAC512_NAME;
244         hmac_reg_HMAC512_VERSION_in_t
[2-1:0]HMAC512_VERSION;
245         hmac_reg_HMAC512_CTRL_in_t HMAC512_CTRL;
246         hmac_reg_HMAC512_STATUS_in_t HMAC512_STATUS;
247         hmac_reg_HMAC512_KEY_in_t [16-1:0]HMAC512_KEY;
248         hmac_reg_HMAC512_BLOCK_in_t [32-1:0]HMAC512_BLOCK;
249         hmac_reg_HMAC512_TAG_in_t [16-1:0]HMAC512_TAG;
250         kv_read_ctrl_reg_in_t HMAC512_KV_RD_KEY_CTRL;
251         kv_status_reg_in_t HMAC512_KV_RD_KEY_STATUS;
252         kv_read_ctrl_reg_in_t HMAC512_KV_RD_BLOCK_CTRL;
253         kv_status_reg_in_t HMAC512_KV_RD_BLOCK_STATUS;
254         kv_write_ctrl_reg_in_t HMAC512_KV_WR_CTRL;
255         kv_status_reg_in_t HMAC512_KV_WR_STATUS;
256         hmac_reg_intr_block_t_in_t intr_block_rf;
257     } hmac_reg_in_t;
258

```

```

259     typedef struct packed{
260         logic value;
261     } hmac_reg_HMAC512_CTRL_INIT_out_t;
262
263     typedef struct packed{
264         logic value;
265     } hmac_reg_HMAC512_CTRL_NEXT_out_t;
266
267     typedef struct packed{
268         logic value;
269     } hmac_reg_HMAC512_CTRL_ZEROIZE_out_t;
270
271     typedef struct packed{
272         logic value;
273     } hmac_reg_HMAC512_CTRL_MODE_out_t;
274
275     typedef struct packed{
276         logic value;
277     } hmac_reg_HMAC512_CTRL_CSR_MODE_out_t;
278
279     typedef struct packed{
280         logic value;
281     } hmac_reg_HMAC512_CTRL_LAST_out_t;
282
283     typedef struct packed{
284         logic value;
285     } hmac_reg_HMAC512_CTRL_Reserved_out_t;
286
287     typedef struct packed{
288         hmac_reg_HMAC512_CTRL_INIT_out_t INIT;
289         hmac_reg_HMAC512_CTRL_NEXT_out_t NEXT;
290         hmac_reg_HMAC512_CTRL_ZEROIZE_out_t ZEROIZE;
291         hmac_reg_HMAC512_CTRL_MODE_out_t MODE;
292         hmac_reg_HMAC512_CTRL_CSR_MODE_out_t CSR_MODE;
293         hmac_reg_HMAC512_CTRL_LAST_out_t LAST;
294         hmac_reg_HMAC512_CTRL_Reserved_out_t Reserved;
295     } hmac_reg_HMAC512_CTRL_out_t;
296
297     typedef struct packed{
298         logic [31:0] value;
299     } hmac_reg_HMAC512_KEY_KEY_out_t;
300
301     typedef struct packed{
302         hmac_reg_HMAC512_KEY_KEY_out_t KEY;
303     } hmac_reg_HMAC512_KEY_out_t;
304
305     typedef struct packed{
306         logic [31:0] value;
307     } hmac_reg_HMAC512_BLOCK_BLOCK_out_t;
308
309     typedef struct packed{
310         hmac_reg_HMAC512_BLOCK_BLOCK_out_t BLOCK;
311     } hmac_reg_HMAC512_BLOCK_out_t;
312
313     typedef struct packed{
314         logic [31:0] value;
315     } hmac_reg_HMAC512_LFSR_SEED_LFSR_SEED_out_t;
316
317     typedef struct packed{
318         hmac_reg_HMAC512_LFSR_SEED_LFSR_SEED_out_t
LFSR_SEED;
319     } hmac_reg_HMAC512_LFSR_SEED_out_t;
320
321     typedef struct packed{
322         logic value;
323     } kv_read_ctrl_reg_read_en_out_t;
324
325     typedef struct packed{
326         logic [4:0] value;
327     } kv_read_ctrl_reg_read_entry_out_t;
328
329     typedef struct packed{
330         logic value;
331     } kv_read_ctrl_reg_pcr_hash_extend_out_t;
332
333     typedef struct packed{
334         logic [24:0] value;
335     } kv_read_ctrl_reg_rsvd_out_t;
336
337     typedef struct packed{
338         kv_read_ctrl_reg_read_en_out_t read_en;
339         kv_read_ctrl_reg_read_entry_out_t read_entry;
340         kv_read_ctrl_reg_pcr_hash_extend_out_t
pcr_hash_extend;
341         kv_read_ctrl_reg_rsvd_out_t rsvd;
342     } kv_read_ctrl_reg_out_t;
343
344     typedef struct packed{
345         logic value;
346     } kv_write_ctrl_reg_write_en_out_t;
347
348     typedef struct packed{
349         logic [4:0] value;
350     } kv_write_ctrl_reg_write_entry_out_t;
351
352     typedef struct packed{
353         logic value;
354     } kv_write_ctrl_reg_hmac_key_dest_valid_out_t;
355
356     typedef struct packed{
357         logic value;

```

```

358 } kv_write_ctrl_reg_hmac_block_dest_valid_out_t;
359
360 typedef struct packed{
361     logic value;
362 } kv_write_ctrl_reg_mldsa_seed_dest_valid_out_t;
363
364 typedef struct packed{
365     logic value;
366 } kv_write_ctrl_reg_ecc_pkey_dest_valid_out_t;
367
368 typedef struct packed{
369     logic value;
370 } kv_write_ctrl_reg_ecc_seed_dest_valid_out_t;
371
372 typedef struct packed{
373     logic value;
374 } kv_write_ctrl_reg_aes_key_dest_valid_out_t;
375
376 typedef struct packed{
377     logic value;
378 } kv_write_ctrl_reg_mlkem_seed_dest_valid_out_t;
379
380 typedef struct packed{
381     logic value;
382 } kv_write_ctrl_reg_mlkem_msg_dest_valid_out_t;
383
384 typedef struct packed{
385     logic value;
386 } kv_write_ctrl_reg_dma_data_dest_valid_out_t;
387
388 typedef struct packed{
389     logic [16:0] value;
390 } kv_write_ctrl_reg_rsvd_out_t;
391
392 typedef struct packed{
393     kv_write_ctrl_reg_write_en_out_t write_en;
394     kv_write_ctrl_reg_write_entry_out_t write_entry;
395     kv_write_ctrl_reg_hmac_key_dest_valid_out_t
hmac_key_dest_valid;
396     kv_write_ctrl_reg_hmac_block_dest_valid_out_t
hmac_block_dest_valid;
397     kv_write_ctrl_reg_mldsa_seed_dest_valid_out_t
mldsa_seed_dest_valid;
398     kv_write_ctrl_reg_ecc_pkey_dest_valid_out_t ecc_p-
key_dest_valid;
399     kv_write_ctrl_reg_ecc_seed_dest_valid_out_t
ecc_seed_dest_valid;
400     kv_write_ctrl_reg_aes_key_dest_valid_out_t
aes_key_dest_valid;
401     kv_write_ctrl_reg_mlkem_seed_dest_valid_out_t
mlkem_seed_dest_valid;
402     kv_write_ctrl_reg_mlkem_msg_dest_valid_out_t
mlkem_msg_dest_valid;
403     kv_write_ctrl_reg_dma_data_dest_valid_out_t
dma_data_dest_valid;
404     kv_write_ctrl_reg_rsvd_out_t rsvd;
405 } kv_write_ctrl_reg_out_t;
406

```

```

407 typedef struct packed{
408     logic intr;
409 } hmac_reg_global_intr_t_agg_sts_dd3dcf0a_out_t;
410
411 typedef struct packed{
412     logic intr;
413 } hmac_reg_global_intr_t_agg_sts_e6399b4a_out_t;
414
415 typedef struct packed{
416     logic intr;
417 } hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_out_t;
418
419 typedef struct packed{
420     logic intr;
421 } hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e_out_t;
422
423 typedef struct packed{
424     hmac_reg_global_intr_t_agg_sts_dd3dcf0a_out_t er-
ror_global_intr_r;
425     hmac_reg_global_intr_t_agg_sts_e6399b4a_out_t no-
tif_global_intr_r;
426     hmac_reg_er-
ror_in-
tr_t_er-
ror2_sts_b1cf2205_error3_sts_74a35378_key_mode_error_sts_f2304c86_k
ey_zero_error_sts_64a18183_out_t error_internal_intr_r;
427     hmac_reg_no-
tif_intr_t_notif_cmd_done_sts_1c68637e_out_t notif_internal_intr_r;
428 } hmac_reg_intr_block_t_out_t;
429
430 typedef struct packed{
431     hmac_reg_HMAC512_CTRL_out_t HMAC512_CTRL;
432     hmac_reg_HMAC512_KEY_out_t [16:1:0]HMAC512_KEY;
433     hmac_reg_HMAC512_BLOCK_out_t [32:1:0]HMAC512_BLOCK;
434     hmac_reg_HMAC512_LFSR_SEED_out_t
[6:1:0]HMAC512_LFSR_SEED;
435     kv_read_ctrl_reg_out_t HMAC512_KV_RD_KEY_CTRL;
436     kv_read_ctrl_reg_out_t HMAC512_KV_RD_BLOCK_CTRL;
437     kv_write_ctrl_reg_out_t HMAC512_KV_WR_CTRL;
438     hmac_reg_intr_block_t_out_t intr_block_rf;
439 } hmac_reg_out_t;
440
441 typedef enum logic [31:0] {
442     kv_status_reg_ERROR_kv_error_e_SUCCESS = 'h0,
443     kv_status_reg_ERROR_kv_error_e_KV_READ_FAIL = 'h1,
444     kv_status_reg_ERROR_kv_error_e_KV_WRITE_FAIL = 'h2,
445     kv_status_reg_ERROR_kv_error_e_KV_FSM_ERROR = 'h4
446 } kv_status_reg_ERROR_kv_error_e_e;
447
448 localparam HMAC_REG_ADDR_WIDTH = 32'd12;
449
450 endpackage

```


Caliptra ECC (P-384)

16 files • 3,663 lines

API — ecc_add_sub_mod_alter.sv

Table 784. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_add_sub_mod_alter.sv — module / interface / package (1개)

이름	종류
ecc_add_sub_mod_alter	module

Table 785. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_add_sub_mod_alter.sv — parameter (1개)

파라미터	형/폭	기본값
REG_SIZE		384

Table 786. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_add_sub_mod_alter.sv — 포트 (10개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
add_en_i	input	wire
sub_i	input	wire
opa_i	input	wire [REG_SIZE-1:0]
opb_i	input	wire [REG_SIZE-1:0]
prime_i	input	wire [REG_SIZE-1:0]
res_o	output	logic [REG_SIZE-1:0]
ready_o	output	logic

API — ecc_adder.sv

Table 787. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv — module / interface / package (1개)

이름	종류
ecc_adder	module

Table 788. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		8

Table 789. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv — 포트 (5개)

포트	방향	형/폭
a_i	input	wire [RADIX-1:0]

포트	방향	형/폭
b_i	input	wire [RADIX-1:0]
cin_i	input	wire
s_o	output	wire [RADIX-1:0]
cout_o	output	wire

API — ecc_arith_unit.sv

Table 790. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv — module / interface / package (1개)

이름	종류
ecc_arith_unit	module

Table 791. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv — parameter (8개)

파라미터	형/폭	기본값
REG_SIZE		384
RND_SIZE		192
RADIX		32
ADDR_WIDTH		6
p_prime	[REG_SIZE-1 : 0]	384'hff...
p_mu	[RADIX-1 : 0]	32'h00000001
q_grouporder	[REG_SIZE-1 : 0]	384'hfffffffffffffffffffffffffffffffffffffc7634d8...
q_mu	[RADIX-1 : 0]	32'he88fdc45

Table 792. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv — 포트 (11개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
ecc_cmd_i	input	wire [3 : 0]
addr_i	input	wire [ADDR_WIDTH-1 : 0]
wr_op_sel_i	input	wire
wr_en_i	input	wire
rd_reg_i	input	wire
RND_SIZE	input	wire [(REG_SIZE+
data_o	output	wire [REG_SIZE-1: 0]
busy_o	output	wire

API — ecc_defines_pkg.sv**Table 793. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv — module / interface / package (1개)**

이름	종류
ecc_defines_pkg	package

Table 794. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv — localparam (2개)

상수	형/폭	값
ECC_ADDR_W		32
ECC_DATA_W		32

Table 795. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
CALIPTRA_ECC_DEFINES		package ecc_defines_pkg;

API — ecc_dsa_ctrl.sv**Table 796. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv — module / interface / package (1개)**

이름	종류
ecc_dsa_ctrl	module

Table 797. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv — localparam (2개)

상수	형/폭	값
zero_pad	[RND_SIZE-1 : 0]	'0
REG_NUM_DWORDS		REG_SIZE / DATA_WIDTH

Table 798. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv — 포트 (15개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
cptra_pwrgood	input	wire
hwif_out	input	ecc_reg__out_t
hwif_in	output	ecc_reg__in_t
kv_read	output	kv_read_t [1:0]
kv_write	output	kv_write_t
kv_rd_resp	input	kv_rd_resp_t [1:0]
kv_wr_resp	input	kv_wr_resp_t
pcr_signing_data	input	pcr_signing_t
ocp_lock_in_progress	input	logic
busy_o	output	logic

포트	방향	형/폭
error_intr	output	logic
notif_intr	output	logic
debugUnlock_or_scan_mode_switch	input	logic

API — ecc_dsa_sequencer.sv

Table 799. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv — module / interface / package (1개)

이름	종류
ecc_dsa_sequencer	module

Table 800. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH		8
DATA_WIDTH		32

Table 801. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv — 포트 (6개)

포트	방향	형/폭
clka	input	wire
reset_n	input	wire
zeroize	input	wire
ena	input	wire
addra	input	wire [ADDR_WIDTH-1 : 0]
douta	output	logic [DATA_WIDTH-1 : 0]

API — ecc_dsa_uop_pkg.sv

Table 802. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv — module / interface / package (1개)

이름	종류
ecc_dsa_uop_pkg	package

Table 803. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv — localparam (54개)

상수	형/폭	값
DSA_UOP_ADDR_WIDTH	integer	9
DSA_OPR_ADDR_WIDTH	integer	6
DSA_PROG_ADDR_W		7
DSA_INSTRUCTION_LENGTH		DSA_UOP_ADDR_WIDTH + (2*DSA_OPR_ADDR_WIDTH)
DSA_UOP_NOP	opcode_t	{op_sel:1'b0
DSA_UOP_WR_CORE	opcode_t	{op_sel:1'b0

상수	형/폭	값
DSA_UOP_WR_SCALAR	opcode_t	{op_sel:1'b1
DSA_UOP_RD_CORE	opcode_t	{op_sel:1'b0
DSA_UOP_KEYGEN	opcode_t	{op_sel:1'b0
DSA_UOP_SIGN	opcode_t	{op_sel:1'b0
DSA_UOP_VERIFY0	opcode_t	{op_sel:1'b0
DSA_UOP_VERIFY1	opcode_t	{op_sel:1'b0
DSA_UOP_VERIFY2	opcode_t	{op_sel:1'b0
DSA_UOP_PK_CHK	opcode_t	{op_sel:1'b0
DSA_UOP_HMAC_DRBG	opcode_t	{op_sel:1'b0
DSA_UOP_SCALAR_SCA	opcode_t	{op_sel:1'b0
DH_UOP_SHAREDKEY	opcode_t	{op_sel:1'b0
NOP_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd0
CONST_ZERO_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd1
CONST_ONE_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd2
CONST_E_a_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd3
CONST_E_3b_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd4
CONST_ONE_p_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd5
CONST_R2_p_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd6
CONST_G_X_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd7
CONST_G_Y_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd8
CONST_R2_q_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd9
CONST_ONE_q_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd10
CONST_E_b_MONT_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd11
SEED_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd16
MSG_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd17
PRIVKEY_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd18
PUBKEYX_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd19
PUBKEYY_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd20
R_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd21
S_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd22
SCALAR_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd23
SCALAR_G_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd24
SCALAR_PK_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd25
VERIFY_R_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd26
LAMBDA_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd27

상수	형/폭	값
MASKING_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd28
PK_VALID_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd29
DH_SHAREDKEY_ID	[DSA_OPR_ADDR_WIDTH-1 : 0]	6'd30
ECC_RESET	[DSA_PROG_ADDR_W-1 : 0]	7'd0
ECC_NOP	[DSA_PROG_ADDR_W-1 : 0]	7'd12
DSA_KG_S	[DSA_PROG_ADDR_W-1 : 0]	ECC_NOP + 2
DSA_KG_E	[DSA_PROG_ADDR_W-1 : 0]	DSA_KG_S + 12
DSA_SGN_S	[DSA_PROG_ADDR_W-1 : 0]	DSA_KG_E + 2
DSA_SGN_E	[DSA_PROG_ADDR_W-1 : 0]	DSA_SGN_S + 14
DSA_VER_S	[DSA_PROG_ADDR_W-1 : 0]	DSA_SGN_E + 2
DSA_VER_E	[DSA_PROG_ADDR_W-1 : 0]	DSA_VER_S + 23
DH_SHARED_S	[DSA_PROG_ADDR_W-1 : 0]	DSA_VER_E + 2
DH_SHARED_E	[DSA_PROG_ADDR_W-1 : 0]	DH_SHARED_S + 17

Table 804. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
ECC_DSA_UOP_PKG		package ecc_dsa_uop_pkg;

API — ecc_fau.sv

Table 805. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv — module / interface / package (1개)

이름	종류
ecc_fau	module

Table 806. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv — parameter (2개)

파라미터	형/폭	기본값
REG_SIZE		384
RADIX		32

Table 807. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv — 포트 (12개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
add_en_i	input	wire
sub_i	input	wire
mult_en_i	input	wire
prime_i	input	wire [REG_SIZE-1:0]

포트	방향	형/폭
mult_mu_i	input	wire [RADIX-1 : 0]
opa_i	input	wire [REG_SIZE-1:0]
opb_i	input	wire [REG_SIZE-1:0]
add_res_o	output	wire [REG_SIZE-1:0]
mult_res_o	output	wire [REG_SIZE-1:0]

API — ecc_hmac_drbg_interface.sv

Table 808. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — module / interface / package (1개)

이름	종류
ecc_hmac_drbg_interface	module

Table 809. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — parameter (2개)

파라미터	형/폭	기본값
REG_SIZE		384
GROUP_ORDER	[REG_SIZE-1 : 0]	384'hffc7634d8...

Table 810. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — localparam (12개)

상수	형/폭	값
KEYGEN_CMD	[1 : 0]	2'b00
SIGN_CMD	[1 : 0]	2'b01
DH_SHARED_CMD	[1 : 0]	2'b10
IDLE_ST	[3 : 0]	4'd0
LFSR_ST	[3 : 0]	4'd1
LAMBDA_ST	[3 : 0]	4'd2
SCALAR_RND_ST	[3 : 0]	4'd3
RND_DONE_ST	[3 : 0]	4'd4
MASKING_RND_ST	[3 : 0]	4'd5
KEYGEN_ST	[3 : 0]	4'd6
SIGN_ST	[3 : 0]	4'd7
DONE_ST	[3 : 0]	4'd8

Table 811. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_hmac_drbg_interface.sv — 포트 (15개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire

포트	방향	형/폭
hmac_mode	input	wire [1 : 0]
en	input	wire
ready	output	wire
keygen_seed	input	wire [REG_SIZE-1 : 0]
keygen_nonce	input	wire [REG_SIZE-1 : 0]
privKey	input	wire [REG_SIZE-1 : 0]
hashed_msg	input	wire [REG_SIZE-1 : 0]
IV	input	wire [REG_SIZE-1 : 0]
lambda	output	wire [REG_SIZE-1 : 0]
scalar_rnd	output	wire [REG_SIZE-1 : 0]
masking_rnd	output	wire [REG_SIZE-1 : 0]
drbg	output	wire [REG_SIZE-1 : 0]

API — ecc_montgomerymultiplier.sv

Table 812. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — module / interface / package (1개)

이름	종류
ecc_montgomerymultiplier	module

Table 813. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — parameter (2개)

파라미터	형/폭	기본값
REG_SIZE		384
RADIX		32

Table 814. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — localparam (7개)

상수	형/폭	값
S_NUM	int unsigned	$((\text{REG_SIZE} + \text{RADIX} - 1) / \text{RADIX}) + 1$
FULL_REG_SIZE	int unsigned	$\text{S_NUM} * \text{RADIX}$
PE_UNITS		$((\text{S_NUM} - 1) / 2) - 1$
t0_WIDTH	int	$\lceil \log_2((2 * (\text{PE_UNITS} + 1)) + 1) \rceil$
t0_unsigned	bit [t0_WIDTH-1:0]	t0[t0_WIDTH-1:0]
t2_WIDTH	int	$\lceil \log_2((2 * (\text{PE_UNITS} + 1)) + 1) \rceil$
t2_unsigned	bit [t2_WIDTH-1:0]	t2[t2_WIDTH-1:0]

Table 815. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_montgomerymultiplier.sv — 포트 (10개)

포트	방향	형/폭
clk	input	wire

포트	방향	형/폭
reset_n	input	wire
zeroize	input	wire
start_i	input	wire
opa_i	input	wire [REG_SIZE-1:0]
opb_i	input	wire [REG_SIZE-1:0]
n_i	input	wire [REG_SIZE-1:0]
n_prime_i	input	wire [RADIX-1:0]
p_o	output	logic [REG_SIZE-1:0]
ready_o	output	logic

API — ecc_mult_dsp.sv

Table 816. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv — module / interface / package (1개)

이름	종류
ecc_mult_dsp	module

Table 817. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 818. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv — 포트 (3개)

포트	방향	형/폭
A_i	input	wire [RADIX-1 : 0]
B_i	input	wire [RADIX-1 : 0]
RADIX	output	logic [(2*

API — ecc_params_pkg.sv

Table 819. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv — module / interface / package (1개)

이름	종류
ecc_params_pkg	package

Table 820. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv — parameter (30개)

파라미터	형/폭	기본값
KEYGEN		3'b001
SIGN		3'b010
VERIFY		3'b011
SHARED_KEY		3'b100
ECC_CORE_NAME	[63 : 0]	64'h38342D33_63707365

파라미터	형/폭	기본값
ECC_CORE_VERSION	[63 : 0]	64'h00000000_3030312e
REG_SIZE	[9 : 0]	10'd384
RND_SIZE	[9 : 0]	10'd192
DATA_WIDTH		32
REG_NUM_DWORDS		REG_SIZE/DATA_WIDTH
REG_OFFSET_W		\$clog2(REG_NUM_DWORDS)
MULT_RADIX		48
SCALAR_BLIND_RADIX		32
ADD_NUM_ADDS		1
ADD_BASE_SZ		384
PRIME	[REG_SIZE-1 : 0]	384'hffffffffffffffffffffffffffffffff...
GROUP_ORDER	[REG_SIZE-1 : 0]	384'hffffffffffffffffffffffffffffffffc7634d8...
ZERO_CONST	[REG_SIZE-1 : 0]	384'h0
ONE_CONST	[REG_SIZE-1 : 0]	384'h1
E_a_MONT	[REG_SIZE-1 : 0]	384'hffffffffffffffffffffffffffffffffcfff...
E_b_MONT	[REG_SIZE-1 : 0]	384'hbfff9b62b21f41f022094e3374bee94938ae277f2209b1920022fc43...
E_3b_MONT	[REG_SIZE-1 : 0]	384'h3fed228165dc5d0661bea9a5e3cbbdbaa0a767d661d14b60068f4c9...
ONE_p_MONT	[REG_SIZE-1 : 0]	384'h100000000ffffffff00000001000000000000
R2_p_MONT	[REG_SIZE-1 : 0]	384'h10000000200000000fffffe0000000000000200000000ffff...
G_X_MONT	[REG_SIZE-1 : 0]	384'h1513812ff723614ede2b6454868459a30eff879c3afc541b4d6e6e1...
G_Y_MONT	[REG_SIZE-1 : 0]	384'hc5e9dd8002263969a840c6c3521968f4ffd98bade7562e83b050cd3...
PRIME_mu	[MULT_RADIX-1 : 0]	64'h100000001
R2_q_MONT	[REG_SIZE-1 : 0]	384'h28266895d40d49174aab1cc5bf030606de609f43cc9601f9ebbfed4...
ONE_q_MONT	[REG_SIZE-1 : 0]	384'h389cb27e0bc8d220a7e5f24db74f58851313e695333ad68d000000...
GROUP_ORDER_mu	[MULT_RADIX-1 : 0]	64'h6089e88fdc45

Table 821. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
CALIPTRA_ECC_PARAMS_PKG		package ecc_params_pkg;

API — ecc_pe.sv

Table 822. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv — module / interface / package (1개)

이름	종류
ecc_pe	module

Table 823. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 824. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv — 포트 (15개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
start_in	input	wire
a_in	input	wire [RADIX-1:0]
b_in	input	wire [RADIX-1:0]
p_in	input	wire [RADIX-1:0]
m_in	input	wire [RADIX-1:0]
s_in	input	wire [RADIX-1:0]
c_in	input	wire [RADIX :0]
odd	input	wire
a_out	output	logic [RADIX-1:0]
m_out	output	logic [RADIX-1:0]
s_out	output	logic [RADIX-1:0]
c_out	output	logic [RADIX :0]

API — ecc_pe_final.sv**Table 825. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv — module / interface / package (1개)**

이름	종류
ecc_pe_final	module

Table 826. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 827. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv — 포트 (13개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
start_in	input	wire
a_in	input	wire [RADIX-1:0]
b_in	input	wire [RADIX-1:0]

포트	방향	형/폭
p_in	input	wire [RADIX-1:0]
m_in	input	wire [RADIX-1:0]
s_in	input	wire [RADIX-1:0]
c_in	input	wire [RADIX :0]
odd	input	wire
s_out	output	logic [RADIX-1:0]
c_out	output	logic [RADIX :0]

API — ecc_pe_first.sv

Table 828. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.sv — module / interface / package (1개)

이름	종류
ecc_pe_first	module

Table 829. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.sv — parameter (1개)

파라미터	형/폭	기본값
RADIX		32

Table 830. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.sv — 포트 (13개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
start_in	input	wire
a_in	input	wire [RADIX-1:0]
b_in	input	wire [RADIX-1:0]
p_in	input	wire [RADIX-1:0]
s_in	input	wire [RADIX-1:0]
n_prime_in	input	wire [RADIX-1:0]
odd	input	wire
a_out	output	logic [RADIX-1:0]
m_out	output	logic [RADIX-1:0]
c_out	output	logic [RADIX :0]

API — ecc_pm_ctrl.sv

Table 831. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — module / interface / package (1개)

이름	종류
ecc_pm_ctrl	module

Table 832. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — parameter (3개)

파라미터	형/폭	기본값
REG_SIZE		384
RND_SIZE		192
INSTR_SIZE		24

Table 833. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — localparam (4개)

상수	형/폭	값
MULT_DELAY	[7 : 0]	8'd27
ADD_DELAY	[7 : 0]	8'd1
Secp384_SCA_MONT_COUNT	[9 : 0]	REG_SIZE[9 : 0] + RND_SIZE[9 : 0]
Secp384_MONT_COUNT	[9 : 0]	REG_SIZE[9 : 0]

Table 834. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv — 포트 (8개)

포트	방향	형/폭
clk	input	wire
reset_n	input	wire
zeroize	input	wire
ecc_cmd_i	input	wire [3 : 0]
digit_i	input	wire
instr_o	output	pm_instr_struct_t
req_digit_o	output	logic
busy_o	output	wire

```

1. chipsalliance_caliptra-rtl/src/ecc/rtl/
ecc_add_sub_mod_alter.sv (122 lines)

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_add_sub_mod_alter.sv
18 // -----
19 // modular addition/subtraction module to compute opa+opb %
prime
20 //
21 //
22 //
=====
23

```

```

24 module ecc_add_sub_mod_alter #(
25     parameter REG_SIZE      = 384
26 )
27 (
28     // Clock and reset.
29     input wire               clk,
30     input wire               reset_n,
31     input wire               zeroize,
32
33     // DATA PORT
34     input wire               add_en_i,
35     input wire               sub_i,
36     input wire [REG_SIZE-1:0] opa_i,
37     input wire [REG_SIZE-1:0] opb_i,
38     input wire [REG_SIZE-1:0] prime_i,
39     output logic [REG_SIZE-1:0] res_o,
40     output logic             ready_o
41 );
42
43 logic [REG_SIZE-1 : 0] opb0;
44 logic [REG_SIZE-1 : 0] opb1;
45 logic [REG_SIZE-1 : 0] r0;
46 logic [REG_SIZE-1 : 0] r1;
47 logic                 carry0;
48
49 logic                 sub_n;
50 logic [REG_SIZE-1 : 0] r0_reg;
51 logic                 carry0_reg;
52
53 logic                 carry1;
54 logic [1 : 0]         push_result_reg;
55
56 ecc_adder #(
57     .RADIX(REG_SIZE)
58 )
59 adder_inst_0(
60     .a_i(opa_i),
61     .b_i(opb0),
62     .cin_i(sub_i),

```

```

63     .s_o(r0),
64     .cout_o(carry0)
65 );
66
67 ecc_adder #(
68     .RADIX(REG_SIZE)
69 )
70     adder_inst_1(
71         .a_i(r0_reg),
72         .b_i(opb1),
73         .cin_i(sub_n),
74         .s_o(r1),
75         .cout_o(carry1)
76 );
77
78
79 assign opb0 = sub_i ? ~opb_i : opb_i;
80
81 always_ff @(posedge clk or negedge reset_n)
82 begin
83     if(!reset_n) begin
84         r0_reg <= '0;
85         carry0_reg <= '0;
86         sub_n <= '0;
87         opb1 <= '0;
88     end
89     else if (zeroize) begin
90         r0_reg <= '0;
91         carry0_reg <= '0;
92         sub_n <= '0;
93         opb1 <= '0;
94     end
95     else if (add_en_i) begin
96         r0_reg <= r0;
97         carry0_reg <= carry0;
98         sub_n <= !sub_i;
99         if (sub_i)
100             opb1 <= prime_i;
101         else
102             opb1 <= ~prime_i;
103     end
104 end
105
106 // Determines when results are ready
107 always_ff @(posedge clk or negedge reset_n) begin
108     if (!reset_n)
109         push_result_reg <= 2'b0;
110     else if (zeroize)
111         push_result_reg <= 2'b0;
112     else if (add_en_i)
113         push_result_reg <= 2'b10;
114     else // one shift to right
115         push_result_reg <= 2'(push_result_reg >> 1);
116 end
117
118 assign ready_o = push_result_reg[0];
119
120 assign res_o = sub_n ? (carry0_reg ^ carry1)? r1 :
r0_reg : (carry0_reg) ? r0_reg : r1;
121
122 endmodule

```

2. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_adder.sv (46 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
==
16 //
17 // ecc_adder.sv
18 // -----
19 // Full adder module to compute a + b in RADIX bits and output
add result
20 // with carry
21 //
22 //
23 //
=====
==
24

```

```

25 module ecc_adder #(
26     parameter RADIX          = 8
27 )
28 (
29     // DATA PORT
30     input wire [RADIX-1:0]  a_i,
31     input wire [RADIX-1:0]  b_i,
32     input wire               cin_i,
33     output wire [RADIX-1:0]  s_o,
34     output wire              cout_o
35 );
36
37     logic [RADIX : 0]    s_full;
38
39     always_comb begin
40         s_full = a_i + b_i + cin_i;
41     end
42
43     assign s_o = s_full[RADIX-1 : 0];
44     assign cout_o = s_full[RADIX];
45
46 endmodule

```

3. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_arith_unit.sv (224 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
==
16 //
17 // ecc_arith_unit.sv
18 // -----
19 // ECC arithmetic unit to perform point multiplication includ-
ing
20 // data memory, point mult (pm) controller, and field arith-
metic
21 // - The module controllol memory acces from outside and pm
controller
22 // - Secret key is stored in this madule's register instead of
data memory,
23 // and this module shifts and feeds one bit of secret key to pm
controller
24 // in each iteration corresponding with pm controller re-
quest (req_digit)
25 //
26 //
27 //
=====
==
28
29 module ecc_arith_unit
30 import ecc_pm_uop_pkg::*;
31
32 parameter REG_SIZE = 384,
33 parameter RND_SIZE = 192,
34 parameter RADIX = 32,
35 parameter ADDR_WIDTH = 6,
36 parameter [REG_SIZE-1 : 0] p_prime =
384'hffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff-
ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff,
37 parameter [RADIX-1 : 0] p_mu = 32'h00000001,
38 parameter [REG_SIZE-1 : 0] q_grouporder =
384'hfffffffffffffffffffffffffffffffffffffffffffffffffffffc7634d81f4372d
df581a0db248b0a77aecec196acc52973,
39 parameter [RADIX-1 : 0] q_mu = 32'h88fdc45
40 )
41 (
42 // Clock and reset.
43 input wire clk,
44 input wire reset_n,
45 input wire zeroize,
46
47 // DATA PORT
48 input wire [3 : 0] ecc_cmd_i,
49 input wire [ADDR_WIDTH-1 : 0] addr_i,
50 input wire wr_op_sel_i,
51 input wire wr_en_i,
52 input wire rd_req_i,

```



```

53 input wire [(REG_SIZE+RND_SIZE)-1 : 0] data_i,
54 output wire [REG_SIZE-1 : 0] data_o,
55 output wire busy_o
56 );
57
58 //-----
59 // Registers including update variables and write enable.
60 //-----
61
62 logic [REG_SIZE-1 : 0] opa_s;
63 logic [REG_SIZE-1 : 0] opb_s;
64 logic [REG_SIZE-1 : 0] add_res_s;
65 logic [REG_SIZE-1 : 0] mult_res_s;
66
67 reg digit_in;
68 logic req_digit;
69 logic ecc_busy_s;
70
71 pm_instr_struct_t ecc_instr_s;
72 logic [REG_SIZE-1 : 0] reg_dinb_r;
73 logic [REG_SIZE-1 : 0] dinb_mux_s;
74 logic [OPR_ADDR_WIDTH-1 : 0] reg_addr_r;
75 logic [OPR_ADDR_WIDTH-1 : 0] addrb_mux_s;
76 logic reg_web_r;
77 logic web_mux_s;
78
79 logic [REG_SIZE-1 : 0] d_o;
80
81 logic mod_p_q;
82 logic [REG_SIZE-1 : 0] adder_prime;
83 logic [RADIX-1 : 0] mult_mu;
84
85 reg [(REG_SIZE+RND_SIZE)-1 : 0] secret_key;
86
87 //-----
88 //
89 // ECC Control Logic
90 //
91 //-----
92
93 ecc_pm_ctrl #(
94     .REG_SIZE(REG_SIZE),
95     .RND_SIZE(RND_SIZE),
96     .INSTR_SIZE(INSTRUCTION_LENGTH)
97 )
98 ecc_pm_ctrl_i(
99     .clk(clk),
100    .reset_n(reset_n),
101    .zeroize(zeroize),
102    .ecc_cmd_i(ecc_cmd_i),
103    .digit_i(digit_in),
104    .instr_o(ecc_instr_s),
105    .req_digit_o(req_digit),
106    .busy_o(ecc_busy_s)
107 );
108
109 //-----
110 //
111 // Memory interface
112 //
113 //-----
114
115 ecc_ram_tdp_file #(
116     .ADDR_WIDTH(OPR_ADDR_WIDTH),
117     .DATA_WIDTH(REG_SIZE)
118 )
119 ram_tdp_file_i(
120     .clk(clk),
121     .reset_n(reset_n),
122     .zeroize(zeroize),
123     .ena(1'b1),
124     .wea(ecc_instr_s.opcode.add_we),
125     .addra(ecc_instr_s.opa_addr),
126     .dina(add_res_s),
127     .douta(opa_s),
128     .enb(1'b1),
129     .web(web_mux_s),
130     .addrb(addrb_mux_s),
131     .dinb(dinb_mux_s),
132     .doutb(opb_s)
133 );
134
135 //-----
136 //
137 // fau interface
138 //
139 //-----
140
141 assign mod_p_q = ecc_instr_s.opcode.mod_q_sel; //per-
142 forming mod_p if (mod_p_q == 0), else mod_q
143 assign adder_prime = (mod_p_q)? q_grouporder : p_prime;
144 assign mult_mu = (mod_p_q)? q_mu : p_mu;
145
146 ecc_fau #(

```

```

145     .REG_SIZE(REG_SIZE),
146     .RADIX(RADIX)
147 )
148 ecc_fau_i
149 (
150     // Clock and reset.
151     .clk(clk),
152     .reset_n(reset_n),
153     .zeroize(zeroize),
154
155     // DATA PORT
156     .add_en_i(ecc_instr_s.opcode.add_en),
157     .sub_i(ecc_instr_s.opcode.sub_sel),
158     .mult_en_i(ecc_instr_s.opcode.mult_en),
159     .prime_i(adder_prime),
160     .mult_mu_i(mult_mu),
161     .opa_i(opa_s),
162     .opb_i(opb_s),
163     .add_res_o(add_res_s),
164     .mult_res_o(mult_res_s)
165 );
166
167 //-----
168 //
169 // Register updates
170 //
171 //
172 //-----
173
174 always_ff @(posedge clk or negedge reset_n)
175 begin : reg_update
176     if (!reset_n) begin
177         reg_dinb_r <= '0;
178         reg_addr_r <= '0;
179         reg_web_r <= '0;
180         secret_key <= '0;
181         d_o <= '0;
182     end
183     else if (zeroize) begin
184         reg_dinb_r <= '0;
185         reg_addr_r <= '0;
186         reg_web_r <= '0;
187         secret_key <= '0;
188         d_o <= '0;
189     end
190     else begin
191         if (wr_en_i) begin
192             if (wr_op_sel_i == 1'b0) // Write new register
193                 reg_dinb_r <= data_i[REG_SIZE-1 : 0];
194             else // Write new key
195                 secret_key <= data_i;
196         end
197         else if (req_digit) begin
198             //Shift secret_key to the left
199             secret_key <=
200 {secret_key[(REG_SIZE+RND_SIZE)-2 : 0],
201 secret_key[(REG_SIZE+RND_SIZE)-1]};
202         end
203         reg_addr_r <= addr_i;
204         if (wr_op_sel_i == 1'b0)
205             reg_web_r <= wr_en_i;
206         else
207             reg_web_r <= '0;
208
209         // Read multiplexer
210         if (rd_reg_i)
211             d_o <= opb_s;
212         else
213             d_o <= '0;
214     end
215 end // reg_update
216
217 //Push key bit to ecc pm control
218 assign digit_in = secret_key[0];
219
220 assign addrb_mux_s = ecc_busy_s ? ecc_instr_s.opb_addr :
221 reg_addr_r;
222 assign web_mux_s = ecc_busy_s ? ecc_in-
223 str_s.opcode.mult_we : reg_web_r;
224 assign dinb_mux_s = ecc_busy_s ?
225 : reg_dinb_r;
226 assign busy_o = ecc_busy_s;
227 assign data_o = d_o;
228
229 endmodule

```

4. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_defines_pkg.sv (40 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 cense");
5 // you may not use this file except in compliance with the Li-
6 cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0

```

```

8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
=====
15 //
16 // ecc_defines_pkg.sv
17 // -----
18 // ECC interface parameters for the digital signature al-
gorithm (DSA).
19 //
20 //
21 //
=====
22
23 `ifndef CALIPTRA_ECC_DEFINES
24 `define CALIPTRA_ECC_DEFINES
25
26 package ecc_defines_pkg;
27
28 localparam ECC_ADDR_W = 32;
29 localparam ECC_DATA_W = 32;
30
31 //ECC REQ
32 typedef struct packed {
33     logic [ECC_ADDR_W-1:0] addr;
34     logic [ECC_DATA_W-1:0] wdata;
35     logic write;
36 } ecc_req_t;
37
38 endpackage
39
40 `endif

```

5. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_ctrl.sv (1,044 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
=====
15 //
16 //
17 // ecc_dsa_ctrl.sv
18 // -----
19 // Elliptic Curve Cryptography (ECC) digital signature al-
gorithm (DSA)
20 // controller to support deterministic ECDSA based on RFC
6979.
21 // The dsa architecture includes several countermeasures to
be protected
22 // against a subset of side-channel analysis (SCA) attacks.
23 //
24 // The architecture includes:
25 // 1) ecc_dsa_sequencer: including the required sequence to
perform
26 // keygen, signing, and verifying.
27 // 2) ecc_arith_unit: the arithmetic unit to perform point
multiplication
28 // and other required operations
29 // 3) hmac_drbg: the hmac384 drbg component to support RFC
6979
30 // 4) ecc_scalar_blinding: the SCA countermeasure to random-
ized scalar
31 // to avoid information leakage.
32 //
33 // The embedded countermeasures are:
34 // - point randomization:
35 // Randomized Projective Coordinates countermeasure based on
section 5.3
36 // "Resistance against Differential Power Analysis for
Elliptic Curve

```

```

37 // Cryptosystems" in CHES 1999 by Coron, J.
38 // This countermeasure uses a random value "lambda" in
point multiplication
39 // (Q = k * P) in keygen/signing to randomize base point
P = (X, Y, 1)
40 // as follows:
41 // P = (X * Lambda, Y * lambda, Lambda)
42 // - scalar blinding:
43 // Scalar blinding countermeasure based on section 5.1
44 // "Resistance against Differential Power Analysis for
Elliptic Curve
45 // Cryptosystems" in CHES 1999 by Coron, J.
46 // This countermeasure uses a random value "r" in the
point multiplication
47 // (Q = k * P) in keygen/signing to randomize scalar k as
follows:
48 // k = k + r*group_order
49 // - masking signature:
50 // Masking sign countermeasure uses a random value "d" in
the signing
51 // operation to generate signature proof s = (privkey * r +
h)*k_inv
52 // as follows:
53 // s = [((privkey - d) * r + (h - d)) * k_inv] + [(d * r
+ d) * k_inv]
54 //
=====
55
56 `include "kv_macros.svh"
57
58 module ecc_dsa_ctrl
59     import ecc_params_pkg::*;
60     import ecc_dsa_uop_pkg::*;
61     import ecc_reg_pkg::*;
62     import kv_defines_pkg::*;
63     (
64         // Clock and reset.
65         input wire clk,
66         input wire reset_n,
67         input wire cptra_pwrgood,
68
69         // Reg ports.
70         input ecc_reg_out_t hwif_out,
71         output ecc_reg_in_t hwif_in,
72
73         // KV interface
74         output kv_read_t [1:0] kv_read,
75         output kv_write_t kv_write,
76         input kv_rd_resp_t [1:0] kv_rd_resp,
77         input kv_wr_resp_t kv_wr_resp,
78
79         //PCR Signing
80         input pcr_signing_t pcr_signing_data,
81
82         input logic ocp_lock_in_progress,
83         output logic busy_o,
84
85         // Interrupts (from ecc_reg)
86         output logic error_intr,
87         output logic notif_intr,
88         input logic debugUnlock_or_scan_mode_switch
89     );
90
91     `include "caliptra_prim_assert.sv"
92
93     //-----
94     // Internal constant and parameter definitions.
95     //-----
96
97     localparam [RND_SIZE-1 : 0] zero_pad = '0;
98     localparam REG_NUM_DWORDS = REG_SIZE / DATA_WIDTH;
99     //-----
100     // Registers including update variables and write enable.
101     //-----
102
103     logic [DSA_PROG_ADDR_W-1 : 0] prog_cntr;
104
105     logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] read_reg;
106     logic [(REG_SIZE+RND_SIZE)-1 : 0] write_reg;
107     logic [1 : 0] cycle_cnt;
108
109     logic zeroize_reg;
110
111     logic dsa_busy;
112     logic subcomponent_busy;
113     logic pm_busy_o;
114
115     logic hw_privkey_we;
116     logic privkey_we_reg;
117     logic sharedkey_we_reg;
118     logic secretkey_we;
119     logic hw_pubkeyx_we;
120     logic hw_pubkeyy_we;
121     logic hw_r_we;

```

```

121 logic hw_s_we;
122 logic hw_scalar_G_we;
123 logic hw_scalar_PK_we;
124 logic hw_verify_r_we;
125 logic hw_pk_chk_we;
126 logic hw_sharedkey_we;
127 logic scalar_G_sel;
128
129 //Explicit hardware signature verification result
130 logic ecc_cmd_start;
131 logic ecc_verify_start;
132 logic ecc_verify_res_match;
133 logic ecc_verify_pass_reg;
134
135 logic ecc_valid_reg;
136 logic ecc_ready_reg;
137
138 logic ecc_status_done_d;
139 logic ecc_status_done_p;
140
141 logic [2 : 0] cmd_reg;
142 logic [3 : 0] pm_cmd_reg;
143 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] msg_reg;
144 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] msg_re-
duced_reg;
145 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] privkey_reg;
146 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] kv_reg;
147 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] pubkeyx_reg;
148 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] pubkeyy_reg;
149 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] seed_reg;
150 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] nonce_reg;
151 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] r_reg;
152 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] s_reg;
153 logic [REG_NUM_DWORDS-1 : 0][DATA_WIDTH-1:0] IV_reg;
154 logic [REG_SIZE-1 : 0] lambda_reg;
155 logic [REG_SIZE-1 : 0] masking_rnd_reg;
156 logic [REG_SIZE-1 : 0] pk_chk_reg;
157
158 logic [REG_SIZE-1 : 0] scalar_G_reg;
159 logic [REG_SIZE-1 : 0] scalar_PK_reg;
160
161 logic [REG_SIZE-1 : 0] scalar_in_reg;
162 logic [REG_SIZE-1 : 0] scalar_rnd_reg;
163 logic [(REG_SIZE+RND_SIZE)-1 : 0] scalar_out_reg;
164 logic scalar_sca_en;
165 logic scalar_sca_busy_o;
166
167 logic [1 : 0] hmac_mode;
168 logic hmac_init;
169 logic hmac_ready;
170 logic [REG_SIZE-1 : 0] hmac_drbg_result;
171 logic hmac_busy;
172
173 //interface with kv client
174 logic kv_privkey_write_en;
175 logic [REG_OFFSET_W-1:0] kv_privkey_write_offset;
176 logic [31:0] kv_privkey_write_data;
177 logic kv_seed_write_en;
178 logic [REG_OFFSET_W-1:0] kv_seed_write_offset;
179 logic [31:0] kv_seed_write_data;
180
181 logic dest_keyvault;
182 kv_error_code_e kv_privkey_error, kv_seed_error,
kv_nonce_error, kv_write_error;
183 logic kv_privkey_ready, kv_privkey_done;
184 logic kv_seed_ready, kv_seed_done;
185 logic kv_write_ready, kv_write_done;
186 //KV Seed Data Present
187 logic kv_seed_data_present;
188 logic kv_seed_data_present_set,
kv_seed_data_present_reset;
189 //KV Key Data Present
190 logic kv_key_data_present;
191 logic kv_key_data_present_set, kv_key_data_present_reset;
192
193 kv_read_ctrl_reg_t kv_privkey_read_ctrl_reg;
194 kv_read_ctrl_reg_t kv_seed_read_ctrl_reg;
195 kv_write_ctrl_reg_t kv_write_ctrl_reg;
196 kv_read_filter_metrics_t kv_privkey_read_metrics;
197 kv_read_filter_metrics_t kv_seed_read_metrics;
198 kv_write_filter_metrics_t kv_write_metrics;
199
200 logic pcr_sign_mode;
201
202 instr_struct_t prog_instr;
203
204 // ERROR
205 logic keygen_process;
206 logic signing_process;
207 logic verifying_process;
208 logic sharedkey_process;
209
210 logic privkey_input_outofrange;
211 logic r_output_outofrange;
212 logic s_output_outofrange;
213 logic r_input_outofrange;
214 logic s_input_outofrange;
215 logic pubkeyx_input_outofrange;
216 logic pubkeyy_input_outofrange;
217 logic pubkey_input_invalid;
218 logic pcr_sign_input_invalid;
219 logic privkey_output_outofrange, pub-
keyx_output_outofrange, pubkeyy_output_outofrange;
220 logic sharedkey_outofrange;
221
222 logic error_flag;
223 logic error_flag_reg;
224 logic error_flag_edge;
225
226 //-----
227 // Module instantiations.
228 //-----
229
230 ecc_dsa_sequencer #(
231 .ADDR_WIDTH(DSA_PROG_ADDR_W),
232 .DATA_WIDTH(DSA_INSTRUCTION_LENGTH)
233 )
234 ecc_dsa_sequencer_i(
235 .clk(clk),
236 .reset_n(reset_n),
237 .zeroize(zeroize_reg),
238 .ena(1'b1),
239 .addra(prog_cntr),
240 .douta(prog_instr)
241 );
242
243 ecc_arith_unit #(
244 .REG_SIZE(REG_SIZE),
245 .RND_SIZE(RND_SIZE),
246 .RADIX(MULT_RADIX),
247 .ADDR_WIDTH(DSA_OPR_ADDR_WIDTH),
248 .p_prime(PRIME),
249 .p_mu(PRIME_mu),
250 .q_grouporder(GROUP_ORDER),
251 .q_mu(GROUP_ORDER_mu)
252 )
253 ecc_arith_unit_i (
254 .clk(clk),
255 .reset_n(reset_n),
256 .zeroize(zeroize_reg),
257 .ecc_cmd_i(pm_cmd_reg),
258 .addr_i(prog_instr.mem_addr),
259 .wr_op_sel_i(prog_instr.opcode.op_sel),
260 .wr_en_i(prog_instr.opcode.wr_en),
261 .rd_reg_i(prog_instr.opcode.rd_en),
262 .data_i(write_reg),
263 .data_o(read_reg),
264 .busy_o(pm_busy_o)
265 );
266
267 ecc_hmac_drbg_interface #(
268 .REG_SIZE(REG_SIZE),
269 .GROUP_ORDER(GROUP_ORDER)
270 )
271 ecc_hmac_drbg_interface_i (
272 .clk(clk),
273 .reset_n(reset_n),
274 .zeroize(zeroize_reg),
275 .hmac_mode(hmac_mode),
276 .en(hmac_init),
277 .ready(hmac_ready),
278 .keygen_seed(seed_reg),
279 .keygen_nonce(nonce_reg),
280 .privkey(privkey_reg),
281 .hashed_msg(msg_reduced_reg),
282 .IV(IV_reg),
283 .lambda(lambda_reg),
284 .scalar_rnd(scalar_rnd_reg),
285 .masking_rnd(masking_rnd_reg),
286 .drbg(hmac_drbg_result)
287 );
288
289 ecc_scalar_blinding #(
290 .REG_SIZE(REG_SIZE),
291 .RND_SIZE(RND_SIZE),
292 .RADIX(SCALAR_BLIND_RADIX),
293 .GROUP_ORDER(GROUP_ORDER)
294 )
295 ecc_scalar_blinding_i(
296 .clk(clk),
297 .reset_n(reset_n),
298 .zeroize(zeroize_reg),
299 .en_i(scalar_sca_en),
300 .data_i(scalar_in_reg),
301 .rnd_i(scalar_rnd_reg[RND_SIZE-1 : 0]),
302 .data_o(scalar_out_reg),
303 .busy_o(scalar_sca_busy_o)
304 );
305
306 //-----
307
308 // ecc_reg_update
309
310 // Update functionality for all interface registers in the
311 core.
312 //-----
313
314 // read the registers written by sw

```

```

313 always_comb begin
314 //Mask the command if KV clients are not idle
315 cmd_reg = {hwif_out.ECC_CTRL.DH_SHAREDKEY.value,
hwif_out.ECC_CTRL.value} & {3{kv_seed_ready}} &
{3{kv_privkey_ready}};
316 zeroize_reg = hwif_out.ECC_CTRL.ZEROIZE.value || debu-
gUnlock_or_scan_mode_switch;
317 end
318
319 //there is a clk cycle memory read delay between
hw_privkey_we and read_reg
320 always_ff @(posedge clk or negedge reset_n)
321 begin : ecc_kv_reg
322 if (!reset_n) begin
323 privkey_we_reg <= '0;
324 sharedkey_we_reg <= '0;
325 kv_reg <= '0;
326 kv_seed_data_present <= '0;
327 kv_key_data_present <= '0;
328 end
329 else if (zeroize_reg) begin
330 privkey_we_reg <= '0;
331 sharedkey_we_reg <= '0;
332 kv_reg <= '0;
333 kv_seed_data_present <= '0;
334 kv_key_data_present <= '0;
335 end
336 //Store private key here before pushing to keyvault
337 else begin
338 privkey_we_reg <= hw_privkey_we;
339 sharedkey_we_reg <= hw_sharedkey_we;
340 if (secretkey_we & (dest_keyvault |
kv_seed_data_present))
341 kv_reg <= read_reg;
342
343 kv_seed_data_present <= kv_seed_data_present_set ?
'1 :
344 kv_seed_data_present_reset ? '0 : kv_seed_data_present;
345 kv_key_data_present <= kv_key_data_present_set ?
'1 :
346 kv_key_data_present_reset ? '0 : kv_key_data_present;
347 end
348 end
349
350 always_comb secretkey_we = (privkey_we_reg |
sharedkey_we_reg);
351
352 assign error_intr = hwif_out.intr_b-
lock_rf.error_global_intr_r.intr;
353 assign notif_intr = hwif_out.intr_b-
lock_rf.notif_global_intr_r.intr;
354
355 // write the registers by hw
356 always_comb hwif_in.reset_b = reset_n;
357 always_comb hwif_in.hard_reset_b = cptra_pwrgood;
358 always_comb hwif_in.ecc_ready = ecc_ready_reg;
359 always_comb hwif_in.ECC_NAME[0].NAME.next =
ECC_CORE_NAME[31 : 0];
360 always_comb hwif_in.ECC_NAME[1].NAME.next =
ECC_CORE_NAME[63 : 32];
361 always_comb hwif_in.ECC_VERSION[0].VERSION.next =
ECC_CORE_VERSION[31 : 0];
362 always_comb hwif_in.ECC_VERSION[1].VERSION.next =
ECC_CORE_VERSION[63 : 32];
363
364 always_comb hwif_in.ECC_STATUS.READY.next = ecc_ready_reg;
365 always_comb hwif_in.ECC_STATUS.VALID.next = ecc_valid_reg;
366 // Only expose the verification result once the engine is
idle again.
367 always_comb hwif_in.ECC_STATUS.VERIFY_PASS.next =
ecc_verify_pass_reg & ~error_flag_reg & ecc_ready_reg;
368
369
370 always_comb begin // ecc_reg_writing
371 for (int dword=0; dword < 12; dword++)begin
372 //Key Vault has priority if enabled to drive these
registers
373 //don't store the private key generated in sw ac-
cessible register
374 if it's going to keyvault
375 privkey_reg[dword] = hwif_out.ECC_PRIVKEY_IN[11-
dword].PRIVKEY_IN.value;
376 hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.we =
(pcr_sign_mode | (kv_privkey_write_en & (kv_privkey_write_offset ==
dword))) & !zeroize_reg;
377 hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.next =
pcr_sign_mode
? pcr_sign-
ing_data.pcr_ecc_signing_privkey[dword] :
kv_privkey_write_en ? kv_privkey_write_data :
378
379 read_reg[11-dword];
380 hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.hwclr =
zeroize_reg | kv_key_data_present_reset | (kv_privkey_error !=
KV_SUCCESS);
381 hwif_in.ECC_PRIVKEY_IN[dword].PRIVKEY_IN.swwe =
ecc_ready_reg & ~kv_key_data_present;

```

```

381 end
382
383 for (int dword=0; dword < 12; dword++)begin
384 //If keyvault is not enabled, grab the sw value as
usual
385 hwif_in.ECC_PRIVKEY_OUT[dword].PRIVKEY_OUT.we =
(privkey_we_reg & ~(dest_keyvault | kv_seed_data_present)) & !zero-
ize_reg;
386 hwif_in.ECC_PRIVKEY_OUT[dword].PRIVKEY_OUT.next =
read_reg[11-dword];
387 hwif_in.ECC_PRIVKEY_OUT[dword].PRIVKEY_OUT.hwclr =
zeroize_reg;
388 end
389
390 for (int dword=0; dword < 12; dword++)begin
391 seed_reg[dword] = hwif_out.ECC_SEED[11-
dword].SEED.value;
392 hwif_in.ECC_SEED[dword].SEED.we =
(kv_seed_write_en & (kv_seed_write_offset == dword)) & !zeroize_reg;
393 hwif_in.ECC_SEED[dword].SEED.next =
kv_seed_write_data;
394 hwif_in.ECC_SEED[dword].SEED.hwclr = zeroize_reg |
kv_seed_data_present_reset | (kv_seed_error != KV_SUCCESS);
395 hwif_in.ECC_SEED[dword].SEED.swwe = ecc_ready_reg &
~kv_seed_data_present;
396 end
397
398 for (int dword=0; dword < 12; dword++)begin
399 nonce_reg[dword] = hwif_out.ECC_NONCE[11-
dword].NONCE.value;
400 hwif_in.ECC_NONCE[dword].NONCE.hwclr = zero-
ize_reg;
401 end
402
403 for (int dword=0; dword < 12; dword++)begin
404 msg_reg[dword] = hwif_out.ECC_MSG[11-
dword].MSG.value;
405 hwif_in.ECC_MSG[dword].MSG.we = pcr_sign_mode & !
zeroize_reg;
406 hwif_in.ECC_MSG[dword].MSG.next = pcr_sign-
ing_data.pcr_hash[dword];
407 hwif_in.ECC_MSG[dword].MSG.hwclr = zeroize_reg;
408 end
409
410 for (int dword=0; dword < 12; dword++)begin
411 pubkeyx_reg[dword] = hwif_out.ECC_PUBKEY_X[11-
dword].PUBKEY_X.value;
412 hwif_in.ECC_PUBKEY_X[dword].PUBKEY_X.we = hw_pub-
keyx_we & !zeroize_reg;
413 hwif_in.ECC_PUBKEY_X[dword].PUBKEY_X.next =
read_reg[11-dword];
414 hwif_in.ECC_PUBKEY_X[dword].PUBKEY_X.hwclr = zero-
ize_reg;
415 end
416
417 for (int dword=0; dword < 12; dword++)begin
418 pubkeyy_reg[dword] = hwif_out.ECC_PUBKEY_Y[11-
dword].PUBKEY_Y.value;
419 hwif_in.ECC_PUBKEY_Y[dword].PUBKEY_Y.we = hw_pub-
keyy_we & !zeroize_reg;
420 hwif_in.ECC_PUBKEY_Y[dword].PUBKEY_Y.next =
read_reg[11-dword];
421 hwif_in.ECC_PUBKEY_Y[dword].PUBKEY_Y.hwclr = zero-
ize_reg;
422 end
423
424 for (int dword=0; dword < 12; dword++)begin
425 r_reg[dword] = hwif_out.ECC_SIGN_R[11-
dword].SIGN_R.value;
426 hwif_in.ECC_SIGN_R[dword].SIGN_R.we = hw_r_we & !
zeroize_reg;
427 hwif_in.ECC_SIGN_R[dword].SIGN_R.next =
read_reg[11-dword];
428 hwif_in.ECC_SIGN_R[dword].SIGN_R.hwclr = zero-
ize_reg;
429 end
430
431 for (int dword=0; dword < 12; dword++)begin
432 s_reg[dword] = hwif_out.ECC_SIGN_S[11-
dword].SIGN_S.value;
433 hwif_in.ECC_SIGN_S[dword].SIGN_S.we = hw_s_we & !
zeroize_reg;
434 hwif_in.ECC_SIGN_S[dword].SIGN_S.next =
read_reg[11-dword];
435 hwif_in.ECC_SIGN_S[dword].SIGN_S.hwclr = zero-
ize_reg;
436 end
437
438 for (int dword=0; dword < 12; dword++)begin
439 //Preload with the complement of the supplied sig-
nature R at the start of a
440 //VERIFY operation so an aborted verification can-
not leave a stale match
441 hwif_in.ECC_VERIFY_R[dword].VERIFY_R.we =
(hw_verify_r_we | ecc_verify_start) & !zeroize_reg;
442 hwif_in.ECC_VERIFY_R[dword].VERIFY_R.next =
ecc_verify_start ? ~r_reg[11-dword] : read_reg[11-dword];

```

```

443     hwif_in.ECC_VERIFY_R[dword].VERIFY_R.hwclr = zero-
ize_reg;
444     end
445
446     for (int dword=0; dword < 12; dword++)begin
447         IV_reg[dword] = hwif_out.ECC_IV[11-
dword].IV.value;
448     hwif_in.ECC_IV[dword].IV.hwclr = zeroize_reg;
449     end
450
451     for (int dword=0; dword < 12; dword++)begin
452         hwif_in.ECC_DH_SHARED_KEY[dword].DH_SHARED_KEY.we
= (sharedkey_we_reg & ~(dest_keyvault | kv_key_data_present)) & !
zeroize_reg;
453     hwif_in.ECC_DH_SHARED_KEY[dword].DH_SHARED_KEY.next = read_reg[11-
dword];
454     hwif_in.ECC_DH_SHARED_KEY[dword].DH_SHARED_KEY.hwclr = zeroize_reg;
455     end
456     end
457
458     //transformed msg into modulo q
459     always_ff @(posedge clk or negedge reset_n)
460     begin : reduced_msg
461         if (!reset_n)
462             msg_reduced_reg <= '0;
463         else if (zeroize_reg)
464             msg_reduced_reg <= '0;
465         else begin
466             if (msg_reg >= GROUP_ORDER)
467                 msg_reduced_reg <= msg_reg - GROUP_ORDER;
468             else
469                 msg_reduced_reg <= msg_reg;
470         end
471     end
472
473
474     always_comb hwif_in.ECC_CTRL.CTRL.hwclr = |cmd_reg;
475     always_comb hwif_in.ECC_CTRL.DH_SHAREDKEY.hwclr = |
cmd_reg;
476     always_comb hwif_in.ECC_CTRL.PCR_SIGN.hwclr =
hwif_out.ECC_CTRL.PCR_SIGN.value;
477
478     // TODO add other interrupt hwset signals (errors)
479     always_comb hwif_in.intr_b-
lock_rf.error_internal_intr_r.error_internal_sts.hwset = er-
ror_flag_edge;
480     always_comb hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset =
ecc_status_done_p;
481
482
483     always_comb begin: ecc_kv_ctrl_reg
484         //ready when fsm is not busy
485         hwif_in.ecc_kv_rd_pkey_status.ERROR.next =
kv_privkey_error;
486         hwif_in.ecc_kv_rd_seed_status.ERROR.next =
kv_seed_error;
487         hwif_in.ecc_kv_wr_pkey_status.ERROR.next =
kv_write_error;
488         //ready when fsm is not busy
489         hwif_in.ecc_kv_rd_pkey_status.READY.next =
kv_privkey_ready;
490         hwif_in.ecc_kv_rd_seed_status.READY.next =
kv_seed_ready;
491         hwif_in.ecc_kv_wr_pkey_status.READY.next =
kv_write_ready;
492         //set valid when fsm is done
493         hwif_in.ecc_kv_rd_pkey_status.VALID.hwset =
kv_privkey_done;
494         hwif_in.ecc_kv_rd_seed_status.VALID.hwset =
kv_seed_done;
495         hwif_in.ecc_kv_wr_pkey_status.VALID.hwset =
kv_write_done;
496         //clear valid when new request is made
497         hwif_in.ecc_kv_rd_pkey_status.VALID.hwclr =
kv_privkey_read_ctrl_reg.read_en;
498         hwif_in.ecc_kv_rd_seed_status.VALID.hwclr =
kv_seed_read_ctrl_reg.read_en;
499         hwif_in.ecc_kv_wr_pkey_status.VALID.hwclr =
kv_write_ctrl_reg.write_en;
500         //clear enable when busy
501         hwif_in.ecc_kv_rd_pkey_ctrl.read_en.hwclr =
~kv_privkey_ready;
502         hwif_in.ecc_kv_rd_seed_ctrl.read_en.hwclr =
~kv_seed_ready;
503         hwif_in.ecc_kv_wr_pkey_ctrl.write_en.hwclr =
~kv_write_ready;
504     end
505
506     // Software write-enables to prevent KV reg manipulation
mid-operation
507     always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.read_en.swwe = !kv_key_data_present && !busy_o;
508     always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.read_entry.swwe = !kv_key_data_present && !busy_o;
509     always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.pcr_hash_extend.swwe = 0; //NA for keyvault
510     always_comb hwif_in.ecc_kv_rd_p-
key_ctrl.rsvd.swwe = 0;

```

```

511
512     always_comb
hwif_in.ecc_kv_rd_seed_ctrl.read_en.swwe = !
kv_seed_data_present && !busy_o;
513     always_comb
hwif_in.ecc_kv_rd_seed_ctrl.read_entry.swwe = !
kv_seed_data_present && !busy_o;
514     always_comb
hwif_in.ecc_kv_rd_seed_ctrl.pcr_hash_extend.swwe = 0; //NA for
keyvault
515     always_comb
hwif_in.ecc_kv_rd_seed_ctrl.rsvd.swwe = 0;
516
517     // KV write control must be written before ECC core opera-
tion begins, even though
518     // output isn't written to KV until the end of the opera-
tion.
519     // Prevent partial-key attacks by blocking register modi-
fications during core execution.
520     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.write_en.swwe = !busy_o;
521     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.write_entry.swwe = !busy_o;
522     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.hmac_key_dest_valid.swwe = !busy_o;
523     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.hmac_block_dest_valid.swwe = !busy_o;
524     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.mlds_seed_dest_valid.swwe = !busy_o;
525     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.ecc_pkey_dest_valid.swwe = !busy_o;
526     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.ecc_seed_dest_valid.swwe = !busy_o;
527     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.aes_key_dest_valid.swwe = !busy_o;
528     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.mlkem_seed_dest_valid.swwe = !busy_o;
529     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.mlkem_msg_dest_valid.swwe = !busy_o;
530     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.dma_data_dest_valid.swwe = !busy_o;
531     always_comb hwif_in.ecc_kv_wr_p-
key_ctrl.rsvd.swwe = 0;
532
533     //keyvault control reg macros for assigning to struct
534     `CALIPT-
RA_KV_READ_CTRL_REG2STRUCT(kv_privkey_read_ctrl_reg, ecc_kv_rd_p-
key_ctrl)
535     `CALIPTRA_KV_READ_CTRL_REG2STRUCT(kv_seed_read_ctrl_reg,
ecc_kv_rd_seed_ctrl)
536     `CALIPTRA_KV_WRITE_CTRL_REG2STRUCT(kv_write_ctrl_reg,
ecc_kv_wr_pkey_ctrl)
537
538     //Detect keyvault data coming in to lock api registers and
protect outputs
539     always_comb kv_seed_data_present_set =
kv_seed_read_ctrl_reg.read_en;
540     always_comb kv_seed_data_present_reset =
kv_seed_data_present & ecc_status_done_p;
541
542     always_comb kv_key_data_present_set =
kv_privkey_read_ctrl_reg.read_en;
543     always_comb kv_key_data_present_reset =
kv_key_data_present & ecc_status_done_p;
544
545     always_comb pcr_sign_mode = hwif_out.ECC_C-
TRL.PCR_SIGN.value;
546
547     //-----
548     // register updates
549     //
550     // update the internal registers and their wr_en
551     //-----
552
553     always_ff @(posedge clk or negedge reset_n)
begin : SCALAR_REG
554         if (!reset_n) begin
555             scalar_G_reg <= '0;
556             scalar_PK_reg <= '0;
557         end
558         else if (zeroize_reg) begin
559             scalar_G_reg <= '0;
560             scalar_PK_reg <= '0;
561         end
562         else begin
563             if (!scalar_G_sel)
564                 scalar_G_reg <= hmac_drbg_result;
565             else if (hw_scalar_G_we)
566                 scalar_G_reg <= read_reg;
567             if (hw_scalar_PK_we)
568                 scalar_PK_reg <= read_reg;
569         end
570     end
571
572
573     // Set the write enable flag for different register based
on sequencer output of "DSA_UOP_RD_CORE"
574     // to read them from data memory inside ecc_arith_unit
575     always_comb

```



```

576 begin : wr_en_signals
577     hw_privkey_we = 0;
578     hw_pubkeyx_we = 0;
579     hw_pubkeyy_we = 0;
580     hw_r_we = 0;
581     hw_s_we = 0;
582     hw_scalar_G_we = 0;
583     hw_scalar_PK_we = 0;
584     hw_verify_r_we = 0;
585     hw_pk_chk_we = 0;
586     hw_sharedkey_we = 0;
587     if ((prog_instr.opcode == DSA_UOP_RD_CORE) &
(cycle_cnt == 0)) begin
588         unique case (prog_instr.reg_id)
589             PRIVKEY_ID : hw_privkey_we = 1;
590             PUBKEYX_ID : hw_pubkeyx_we = 1;
591             PUBKEYY_ID : hw_pubkeyy_we = 1;
592             R_ID : hw_r_we = 1;
593             S_ID : hw_s_we = 1;
594             SCALAR_G_ID : hw_scalar_G_we = 1;
595             SCALAR_PK_ID : hw_scalar_PK_we = 1;
596             VERIFY_R_ID : hw_verify_r_we = 1;
597             PK_VALID_ID : hw_pk_chk_we = 1;
598             DH_SHAREDKEY_ID : hw_sharedkey_we = 1;
599             default :
600                 begin
601                     hw_privkey_we = 0;
602                     hw_pubkeyx_we = 0;
603                     hw_pubkeyy_we = 0;
604                     hw_r_we = 0;
605                     hw_s_we = 0;
606                     hw_scalar_G_we = 0;
607                     hw_scalar_PK_we = 0;
608                     hw_verify_r_we = 0;
609                     hw_pk_chk_we = 0;
610                     hw_sharedkey_we = 0;
611                 end
612             endcase
613         end
614     end // wr_en_signals
615
616 // set the write register value based on sequencer output
of "DSA_UOP_WR_CORE"
617 // to write into data memory inside ecc_arith_unit
618 always_comb
619 begin : write_to_pm_core
620     write_reg = '0;
621     if (prog_instr.opcode == DSA_UOP_WR_CORE) begin
622         unique case (prog_instr.reg_id)
623
624             CONST_ZERO_ID : write_reg = {zero_pad,
ZERO_CONST};
625
626             CONST_ONE_ID : write_reg = {zero_pad,
ONE_CONST};
627
628             CONST_E_a_MONT_ID : write_reg = {zero_pad,
E_a_MONT};
629
630             CONST_E_b_MONT_ID : write_reg = {zero_pad,
E_b_MONT};
631
632             CONST_E_3b_MONT_ID : write_reg = {zero_pad,
E_3b_MONT};
633
634             CONST_ONE_p_MONT_ID : write_reg = {zero_pad,
ONE_p_MONT};
635
636             CONST_R2_p_MONT_ID : write_reg = {zero_pad,
R2_p_MONT};
637
638             CONST_G_X_MONT_ID : write_reg = {zero_pad,
G_X_MONT};
639
640             CONST_G_Y_MONT_ID : write_reg = {zero_pad,
G_Y_MONT};
641
642             CONST_R2_q_MONT_ID : write_reg = {zero_pad,
R2_q_MONT};
643
644             CONST_ONE_q_MONT_ID : write_reg = {zero_pad,
ONE_q_MONT};
645
646             MSG_ID : write_reg = {zero_pad,
msg_reduced_reg};
647
648             PRIVKEY_ID : write_reg = {zero_pad,
privkey_reg};
649
650             PUBKEYX_ID : write_reg = {zero_pad,
pubkeyx_reg};
651
652             PUBKEYY_ID : write_reg = {zero_pad,
pubkeyy_reg};
653
654             R_ID : write_reg = {zero_pad,
r_reg};
655
656             S_ID : write_reg = {zero_pad,
s_reg};

```

```

641         SCALAR_G_ID : write_reg = {zero_pad,
scalar_G_reg};
642
643         LAMBDA_ID : write_reg = {zero_pad,
lambda_reg};
644
645         MASKING_ID : write_reg = {zero_pad,
masking_rnd_reg};
646
647         default : write_reg = '0;
648     endcase
649 end
650 else if (prog_instr.opcode == DSA_UOP_WR_SCALAR) begin
651     unique case (prog_instr.reg_id)
652         SCALAR_PK_ID : write_reg = (REG_S-
IZE+RND_SIZE)'(scalar_PK_reg << RND_SIZE);
653         SCALAR_G_ID : write_reg = (REG_S-
IZE+RND_SIZE)'(scalar_G_reg << RND_SIZE);
654         SCALAR_ID : write_reg = scal-
ar_out_reg; // SCA
655         default : write_reg = '0;
656     endcase
657 end // write_to_pm_core
658
659 // Set the input scalar for scalar_blinding module
always_ff @(posedge clk or negedge reset_n)
660 begin : scalar_sca_ctrl
661     if(!reset_n) begin
662         scalar_in_reg <= '0;
663     end
664     else if(zeroize_reg) begin
665         scalar_in_reg <= '0;
666     end
667     else begin
668         if (prog_instr.opcode == DSA_UOP_SCALAR_SCA) begin
669             scalar_in_reg <= scalar_G_reg;
670         end
671     end
672 end // scalar_sca_ctrl
673
674 assign hmac_busy = ~hmac_ready;
675 assign subcomponent_busy = pm_busy_o | hmac_busy | scal-
ar_sca_busy_o;
676
677 //-----
678 // ERROR detection
679 //
680 // bound check for different input/output registers.
681 //-----
682
683 always_ff @(posedge clk or negedge reset_n)
684 begin : pk_chk
685     if(!reset_n) begin
686         pk_chk_reg <= '0;
687     end
688     else if(zeroize_reg) begin
689         pk_chk_reg <= '0;
690     end
691     else begin
692         if (hw_pk_chk_we) begin
693             pk_chk_reg <= read_reg;
694         end
695     end
696 end // pk_chk
697
698 always_ff @(posedge clk or negedge reset_n)
699 begin : error_detection
700     if(!reset_n)
701         error_flag_reg <= 1'b0;
702     else if(zeroize_reg)
703         error_flag_reg <= 1'b0;
704     else if (error_flag)
705         error_flag_reg <= 1'b1;
706 end // error_detection
707
708 assign error_flag_edge = error_flag & (!error_flag_reg);
709
710 assign privkey_input_outofrange = signing_process &
((privkey_reg == 0) | (privkey_reg >= GROUP_ORDER));
711 assign r_output_outofrange = signing_process &
(hw_r_we & ((read_reg == 0) | (read_reg >= GROUP_ORDER)));
712 assign s_output_outofrange = signing_process &
(hw_s_we & ((read_reg == 0) | (read_reg >= GROUP_ORDER)));
713
714 assign r_input_outofrange = verifying_process &
((r_reg == 0) | (r_reg >= GROUP_ORDER));
715 assign s_input_outofrange = verifying_process &
((s_reg == 0) | (s_reg >= GROUP_ORDER));
716 assign pubkeyx_input_outofrange = (verifying_process |
sharedkey_process) & (pubkeyx_reg >= PRIME);
717 assign pubkeyy_input_outofrange = (verifying_process |
sharedkey_process) & (pubkeyy_reg >= PRIME);
718 assign pubkey_input_invalid = (verifying_process |
sharedkey_process) & (pk_chk_reg != 0);
719
720 assign pcr_sign_input_invalid = ((cmd_reg == KEYGEN) |
(cmd_reg == VERIFY) | (cmd_reg == SHARED_KEY)) & pcr_sign_mode;

```

```

721 assign privkey_output_outofrange = keygen_process &
(hw_privkey_we & ((read_reg == 0) | (read_reg >= GROUP_ORDER)));
722 assign pubkeyx_output_outofrange = keygen_process &
(hw_pubkeyx_we & (read_reg >= PRIME));
723 assign pubkeyy_output_outofrange = keygen_process &
(hw_pubkeyy_we & (read_reg >= PRIME));
724
725 assign sharedkey_outofrange = sharedkey_process &
(hw_sharedkey_we & (read_reg >= PRIME));
726
727 assign error_flag = privkey_input_outofrange | r_out-
put_outofrange | s_output_outofrange |
728 r_input_outofrange | s_in-
put_outofrange | pubkeyx_input_outofrange | pubkeyy_input_outofrange
|
729 pubkey_input_invalid |
pcr_sign_input_invalid |
730 privkey_output_outofrange | pub-
keyx_output_outofrange | pubkeyy_output_outofrange |
731 sharedkey_outofrange;
732
733 //-----
734 // Explicit hardware signature verification result
735 //
736 // The recomputed R is compared against the supplied sig-
nature R by
737 // hardware, in the exact same cycle (and with the exact
same data)
738 // that populates ECC_VERIFY_R. This guarantees the repor-
ted pass/fail
739 // bit and the data returned to firmware can never dis-
agree. Firmware
740 // may still compare ECC_VERIFY_R against its own copy of
R as a
741 // fault-attack check on this bit.
742 //-----
743 always_comb ecc_cmd_start = (prog_cntr == ECC_NOP) & (|
cmd_reg);
744 always_comb ecc_verify_start = (prog_cntr == ECC_NOP) &
(cmd_reg == VERIFY);
745 always_comb ecc_verify_res_match = (read_reg == r_reg) &
(|r_reg);
746
747 always_ff @(posedge clk or negedge reset_n)
748 begin : ecc_verify_result
749 if (!reset_n)
750 ecc_verify_pass_reg <= 1'b0;
751 else if (zeroize_reg | error_flag | error_flag_reg |
ecc_cmd_start)
752 ecc_verify_pass_reg <= 1'b0;
753 else if (verifying_process & hw_verify_r_we)
754 ecc_verify_pass_reg <= ecc_verify_res_match;
755 end // ecc_verify_result
756
757 //-----
758 // ECDSA_FSM_flow
759 //
760 // This FSM starts with the keygen/signing/verifying com-
mand to
761 // perform different operations.
762 // Active low and async reset.
763 //-----
764 always_ff @(posedge clk or negedge reset_n)
765 begin : ECDSA_FSM
766 if (!reset_n) begin
767 prog_cntr <= ECC_RESET;
768 cycle_cnt <= '0;
769 pm_cmd_reg <= '0;
770 ecc_valid_reg <= 0;
771 scalar_G_sel <= 0;
772 hmac_mode <= '0;
773 hmac_init <= 0;
774 scalar_sca_en <= 0;
775 keygen_process <= 0;
776 signing_process <= 0;
777 verifying_process <= 0;
778 sharedkey_process <= 0;
779 end
780 else if (zeroize_reg) begin
781 prog_cntr <= ECC_RESET;
782 cycle_cnt <= '0;
783 pm_cmd_reg <= '0;
784 ecc_valid_reg <= 0;
785 scalar_G_sel <= 0;
786 hmac_mode <= '0;
787 hmac_init <= 0;
788 scalar_sca_en <= 0;
789 keygen_process <= 0;
790 signing_process <= 0;
791 verifying_process <= 0;
792 sharedkey_process <= 0;
793 end
794 else if (error_flag | error_flag_reg) begin
795 prog_cntr <= ECC_NOP;
796 cycle_cnt <= '0;
797 pm_cmd_reg <= '0;
798 ecc_valid_reg <= 0;

```

```

799 scalar_G_sel <= 0;
800 hmac_mode <= 0;
801 hmac_init <= 0;
802 scalar_sca_en <= 0;
803 keygen_process <= 0;
804 signing_process <= 0;
805 verifying_process <= 0;
806 sharedkey_process <= 0;
807
808 end
809 else begin
810 if (subcomponent_busy) begin //Stalled until sub-
component is done
811 prog_cntr <= prog_cntr;
812 cycle_cnt <= 2'd3;
813 pm_cmd_reg <= '0;
814 scalar_sca_en <= 0;
815 hmac_init <= 0;
816
817 end
818 else if (dsa_busy & (cycle_cnt != 2'd3)) begin
819 cycle_cnt <= cycle_cnt + 1;
820
821 end
822 else begin
823 cycle_cnt <= '0;
824 unique case (prog_cntr)
825 ECC_NOP : begin
826 keygen_process <= 0;
827 signing_process <= 0;
828 verifying_process <= 0;
829 sharedkey_process <= 0;
830 // Waiting for new valid command
831 unique case (cmd_reg)
832 KEYGEN : begin // keygen
833 prog_cntr <= DSA_KG_S;
834 ecc_valid_reg <= 0;
835 scalar_G_sel <= 0;
836 hmac_mode <= 2'b00;
837 keygen_process <= 1;
838
839 end
840
841 SIGN : begin // signing
842 prog_cntr <= DSA_SGN_S;
843 ecc_valid_reg <= 0;
844 scalar_G_sel <= 0;
845 hmac_mode <= 2'b01;
846 signing_process <= 1;
847
848 end
849
850 VERIFY : begin // verifying
851 prog_cntr <= DSA_VER_S;
852 ecc_valid_reg <= 0;
853 scalar_G_sel <= 1;
854 verifying_process <= 1;
855
856 end
857
858 SHARED_KEY : begin // DH
859 prog_cntr <= DH_SHARED_S;
860 ecc_valid_reg <= 0;
861 scalar_G_sel <= 1;
862 hmac_mode <= 2'b10;
863 sharedkey_process <= 1;
864
865 end
866
867 default : begin
868 prog_cntr <= ECC_NOP;
869 scalar_G_sel <= 0;
870
871 end
872 endcase
873 pm_cmd_reg <= '0;
874 hmac_init <= 0;
875
876 end
877
878 DSA_KG_E : begin // end of keygen
879 prog_cntr <= ECC_NOP;
880 ecc_valid_reg <= 1;
881
882 end
883
884 DSA_SGN_E : begin // end of signing
885 prog_cntr <= ECC_NOP;
886 ecc_valid_reg <= 1;
887
888 end
889
890 DSA_VER_E : begin // end of verifying
891 prog_cntr <= ECC_NOP;
892 ecc_valid_reg <= 1;
893
894 end
895
896 DH_SHARED_E : begin // end of DH shared key
897 prog_cntr <= ECC_NOP;
898 ecc_valid_reg <= 1;
899
900 end
901
902 ECC_RESET,
903 DSA_KG_S,
904 DSA_SGN_S,
905 DSA_VER_S,
906 DH_SHARED_S : begin
907 prog_cntr <= prog_cntr + 1;
908 pm_cmd_reg <= prog_in-

```

```
str.opcode.pm_cmd;
```



```

896 hmac_init      <= prog_in-
str.opcode.hmac_drbg_en;
897 scalar_sca_en  <= prog_in-
str.opcode.sca_en;
898 end
899
900 default : begin
901   prog_cntr      <= prog_cntr + 1;
902   pm_cmd_reg     <= prog_in-
str.opcode.pm_cmd;
903   hmac_init      <= prog_in-
str.opcode.hmac_drbg_en;
904   scalar_sca_en  <= prog_in-
str.opcode.sca_en;
905 end
906 endcase
907 end
908 end
909 end // ECDSA_FSM
910
911 // Generate a pulse to trig the interrupt after finishing
the operation
912 always_ff @(posedge clk or negedge reset_n)
913   if (!reset_n)
914     ecc_status_done_d <= 1'b0;
915   else if (zeroize_reg)
916     ecc_status_done_d <= 1'b0;
917   else
918     ecc_status_done_d <=
hwif_in.ECC_STATUS.VALID.next;
919   always_comb ecc_status_done_p =
hwif_in.ECC_STATUS.VALID.next && !ecc_status_done_d;
920
921 // Set the ready/busy flag of ECC
922 assign dsa_busy = (prog_cntr == ECC_NOP)? 1'b0 : 1'b1;
923 always_comb ecc_ready_reg = !(dsa_busy | pm_busy_o);
924
925 //Key Vault Control Modules
926 always_comb begin
927   kv_privkey_read_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
928   kv_privkey_read_metrics.kv_read_dest      =
KV_NUM_READ'(1<<KV_DEST_IDX_ECC_PKEY);
929   kv_privkey_read_metrics.kv_key_entry      =
kv_privkey_read_ctrl_reg.read_entry;
930 end
931
932 //Read PRIVKEY
933 kv_read_client #(
934   .DATA_WIDTH(REG_SIZE),
935   .PAD(0)
936 )
937 ecc_privkey_kv_read
938 (
939   .clk(clk),
940   .rst_b(reset_n),
941   .zeroize(zeroize_reg),
942
943   //client control register
944   .read_ctrl_reg(kv_privkey_read_ctrl_reg),
945   .read_metrics(kv_privkey_read_metrics),
946
947   //interface with kv
948   .kv_read(kv_read[0]),
949   .kv_resp(kv_rd_resp[0]),
950
951   //interface with client
952   .write_en(kv_privkey_write_en),
953   .write_offset(kv_privkey_write_offset),
954   .write_data(kv_privkey_write_data),
955
956   .error_code(kv_privkey_error),
957   .kv_ready(kv_privkey_ready),
958   .read_done(kv_privkey_done),
959   .check_key_size(1'b0),
960   .expected_key_size(KV_ENTRY_SIZE_W'((REG_SIZE/32)-1))
961 );
962
963 //Read SEED
964 always_comb begin
965   kv_seed_read_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
966   kv_seed_read_metrics.kv_read_dest      =
KV_NUM_READ'(1<<KV_DEST_IDX_ECC_SEED);
967   kv_seed_read_metrics.kv_key_entry      =
kv_seed_read_ctrl_reg.read_entry;
968 end
969
970 kv_read_client #(
971   .DATA_WIDTH(REG_SIZE),
972   .PAD(0)
973 )
974 ecc_seed_kv_read
975 (
976   .clk(clk),
977   .rst_b(reset_n),
978   .zeroize(zeroize_reg),
979
980   //client control register
981   .read_ctrl_reg(kv_seed_read_ctrl_reg),
982   .read_metrics(kv_seed_read_metrics),

```

```

983
984 //interface with kv
985 .kv_read(kv_read[1]),
986 .kv_resp(kv_rd_resp[1]),
987
988 //interface with client
989 .write_en(kv_seed_write_en),
990 .write_offset(kv_seed_write_offset),
991 .write_data(kv_seed_write_data),
992
993 .error_code(kv_seed_error),
994 .kv_ready(kv_seed_ready),
995 .read_done(kv_seed_done),
996 .check_key_size(1'b0),
997 .expected_key_size(KV_ENTRY_SIZE_W'((REG_SIZE/32)-1))
998 );
999
1000 //Write to keyvault
1001 always_comb begin
1002   kv_write_metrics.ocp_lock_in_progress =
ocp_lock_in_progress;
1003   kv_write_metrics.kv_data0_present    =
kv_seed_data_present;
1004   kv_write_metrics.kv_data0_entry      =
kv_seed_read_ctrl_reg.read_entry; // FIXME latch this at start-time
1005   kv_write_metrics.kv_data1_present    =
kv_key_data_present;
1006   kv_write_metrics.kv_data1_entry      =
kv_privkey_read_ctrl_reg.read_entry; // FIXME latch this at start-
time
1007   kv_write_metrics.kv_write_src         =
KV_NUM_WRITE'(1 << KV_WRITE_IDX_ECC);
1008   kv_write_metrics.kv_write_entry      =
kv_write_ctrl_reg.write_entry;
1009   kv_write_metrics.aes_decrypt_ecb_op   = 1'b0;
1010 end
1011
1012 kv_write_client #(
1013   .DATA_WIDTH(REG_SIZE)
1014 )
1015 ecc_privkey_kv_write
1016 (
1017   .clk(clk),
1018   .rst_b(reset_n),
1019   .zeroize(zeroize_reg),
1020
1021   //client control register
1022   .write_ctrl_reg(kv_write_ctrl_reg),
1023   .num_dwords(REG_NUM_DWORDS[4:0]),
1024   .write_metrics(kv_write_metrics),
1025
1026   //interface with kv
1027   .kv_write(kv_write),
1028   .kv_resp(kv_wr_resp),
1029
1030   //interface with client
1031   .dest_keyvault(dest_keyvault),
1032   .dest_data_avail(secretkey_we),
1033   .dest_data(kv_reg),
1034
1035   .error_code(kv_write_error),
1036   .kv_ready(kv_write_ready),
1037   .dest_done(kv_write_done)
1038 );
1039
1040 always_comb busy_o = ~ecc_ready_reg | ~kv_write_ready |
~kv_seed_ready | ~kv_privkey_ready;
1041 `CALIPTRA_ASSERT_MUTEX(ERR_ECC_PRIVKEY_WE_MUTEX,
{hw_privkey_we, privkey_we_reg}, clk, !reset_n)
1042 `CALIPTRA_ASSERT_MUTEX(ERR_ECC_SHAREDKEY_WE_MUTEX,
{hw_sharedkey_we, sharedkey_we_reg}, clk, !reset_n)
1043
1044 endmodule

```

6. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_sequencer.sv (167 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
==

```

```

16 //
17 // ecc_dsa_sequencer.sv
18 // -----
19 // This module contains the sequence of ECC operation. it in-
cludes 4 sequences
20 // of operation:

21 // 1) reset sequence: After reset, the core will initilze the
data memory based
22 // on the first lines of sequencer. Then,
it will stay on NOP
23 // line until getting the command.
24 // 2) keygen sequence: After getting the keygen command, it
will generate and
25 // randomize privkey and feed it into the
pm core and return
26 // key pairs of (privkey, pubkey)

27 // 3) sign sequence: After getting the sign command, it will
generate and
28 // randomize nonce and feed it into the pm
core and return
29 // the signature (R,
S).
30 // 4) verify sequence: After getting the verify command, it
will feed the signature
31 // into the pm core and return the com-
puted verify_R.
32 //
33 //
34 //
=====
35 module ecc_dsa_sequencer
36 import ecc_pm_uop_pkg::*;
37 import ecc_dsa_uop_pkg::*;
38 #(
39     parameter ADDR_WIDTH = 8,
40     parameter DATA_WIDTH = 32
41 )
42 (
43     input wire          clka,
44     input wire          reset_n,
45     input wire          zeroize,
46     input wire          ena,
47     input wire [ADDR_WIDTH-1 : 0] addra,
48     output logic [DATA_WIDTH-1 : 0] douta
49 );
50
51
52
53 //-----
54 // ROM content
55 //-----
56
57 always_ff @(posedge clka or negedge reset_n)
58 begin
59     if (!reset_n) begin
60         douta <= '0;
61     end
62     else if (zeroize) begin
63         douta <= '0;
64     end
65     else begin
66         if (ena) begin
67             unique case(addra)
68                 //PM CORE INIT
69                 0 : douta <= {DSA_UOP_NOP,
NOP_ID, UOP_OPR_DONTCARE};
70                 1 : douta <= {DSA_UOP_WR_CORE,
CONST_ZERO_ID, UOP_OPR_CONST_ZERO};
71                 2 : douta <= {DSA_UOP_WR_CORE,
CONST_ONE_ID, UOP_OPR_CONST_ONE};
72                 3 : douta <= {DSA_UOP_WR_CORE,
CONST_E_a_MONT_ID, UOP_OPR_CONST_E_a};
73                 4 : douta <= {DSA_UOP_WR_CORE,
CONST_E_3b_MONT_ID, UOP_OPR_CONST_E_3b};
74                 5 : douta <= {DSA_UOP_WR_CORE,
CONST_ONE_p_MONT_ID, UOP_OPR_CONST_ONE_MONT};
75                 6 : douta <= {DSA_UOP_WR_CORE,
CONST_R2_p_MONT_ID, UOP_OPR_CONST_R2_p};
76                 7 : douta <= {DSA_UOP_WR_CORE,
CONST_G_X_MONT_ID, UOP_OPR_CONST_GX_MONT};
77                 8 : douta <= {DSA_UOP_WR_CORE,
CONST_G_Y_MONT_ID, UOP_OPR_CONST_GY_MONT};
78                 9 : douta <= {DSA_UOP_WR_CORE,
CONST_R2_q_MONT_ID, UOP_OPR_CONST_R2_q};
79                 10 : douta <= {DSA_UOP_WR_CORE,
CONST_ONE_q_MONT_ID, UOP_OPR_CONST_ONE_q_MONT};
80                 11 : douta <= {DSA_UOP_NOP,
NOP_ID, UOP_OPR_DONTCARE};
81
82                 //KEYGEN
83                 DSA_KG_S : douta <= {DSA_UOP_H-
MAC_DRBG, NOP_ID, UOP_OPR_DONTCARE};
84                 DSA_KG_S+ 1 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
85                 DSA_KG_S+ 2 : douta <=

```

```

{DSA_UOP_WR_CORE, SCALAR_G_ID, UOP_OPR_SCALAR_G};
86     DSA_KG_S+ 3 : douta <=
{DSA_UOP_RD_CORE, PRIVKEY_ID, UOP_OPR_SCALAR_G};
87     DSA_KG_S+ 4 : douta <= {DSA_UOP_SCAL-
AR_SCA, SCALAR_G_ID, UOP_OPR_DONTCARE};
88     DSA_KG_S+ 5 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
89     DSA_KG_S+ 6 : douta <= {DSA_UOP_WR_S-
ALAR, SCALAR_ID, UOP_OPR_DONTCARE};
90     DSA_KG_S+ 7 : douta <=
{DSA_UOP_WR_CORE, LAMBDA_ID, UOP_OPR_LAMBDA};
91     DSA_KG_S+ 8 : douta <= {DSA_UOP_KEY-
GEN, NOP_ID, UOP_OPR_DONTCARE};
92     DSA_KG_S+ 9 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
93     DSA_KG_S+ 10 : douta <=
{DSA_UOP_RD_CORE, PUBKEYX_ID, UOP_OPR_Qx_AFFN};
94     DSA_KG_S+ 11 : douta <=
{DSA_UOP_RD_CORE, PUBKEYY_ID, UOP_OPR_Qy_AFFN};
95     DSA_KG_S+ 12 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
96
97     //SIGN
98     DSA_SGN_S : douta <=
{DSA_UOP_WR_CORE, MSG_ID, UOP_OPR_HASH_MSG};
99     DSA_SGN_S+ 1 : douta <=
{DSA_UOP_WR_CORE, PRIVKEY_ID, UOP_OPR_PRIVKEY};
100     DSA_SGN_S+ 2 : douta <= {DSA_UOP_H-
MAC_DRBG, NOP_ID, UOP_OPR_DONTCARE};
101     DSA_SGN_S+ 3 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
102     DSA_SGN_S+ 4 : douta <=
{DSA_UOP_WR_CORE, SCALAR_G_ID, UOP_OPR_SCALAR_G};
103     DSA_SGN_S+ 5 : douta <= {DSA_UOP_SCAL-
AR_SCA, SCALAR_G_ID, UOP_OPR_DONTCARE};
104     DSA_SGN_S+ 6 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
105     DSA_SGN_S+ 7 : douta <= {DSA_UOP_WR_S-
ALAR, SCALAR_ID, UOP_OPR_DONTCARE};
106     DSA_SGN_S+ 8 : douta <=
{DSA_UOP_WR_CORE, LAMBDA_ID, UOP_OPR_LAMBDA};
107     DSA_SGN_S+ 9 : douta <=
{DSA_UOP_WR_CORE, MASKING_ID, UOP_OPR_MASKING};
108     DSA_SGN_S+ 10 : douta <=
{DSA_UOP_SIGN, NOP_ID, UOP_OPR_DONTCARE};
109     DSA_SGN_S+ 11 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
110     DSA_SGN_S+ 12 : douta <=
{DSA_UOP_RD_CORE, R_ID, UOP_OPR_SIGN_R};
111     DSA_SGN_S+ 13 : douta <=
{DSA_UOP_RD_CORE, S_ID, UOP_OPR_SIGN_S};
112     DSA_SGN_S+ 14 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
113
114     //VERIFY
115     DSA_VER_S : douta <=
{DSA_UOP_WR_CORE, CONST_E_b_MONT_ID, UOP_OPR_CONST_E_b};
116     DSA_VER_S+ 1 : douta <=
{DSA_UOP_WR_CORE, PUBKEYX_ID, UOP_OPR_Qx_AFFN};
117     DSA_VER_S+ 2 : douta <=
{DSA_UOP_WR_CORE, PUBKEYY_ID, UOP_OPR_Qy_AFFN};
118     DSA_VER_S+ 3 : douta <=
{DSA_UOP_PK_CHK, NOP_ID, UOP_OPR_DONTCARE};
119     DSA_VER_S+ 4 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
120     DSA_VER_S+ 5 : douta <=
{DSA_UOP_RD_CORE, PK_VALID_ID, UOP_OPR_PK_VALID};
121     DSA_VER_S+ 6 : douta <=
{DSA_UOP_WR_CORE, MSG_ID, UOP_OPR_HASH_MSG};
122     DSA_VER_S+ 7 : douta <=
{DSA_UOP_WR_CORE, R_ID, UOP_OPR_SIGN_R};
123     DSA_VER_S+ 8 : douta <=
{DSA_UOP_WR_CORE, S_ID, UOP_OPR_SIGN_S};
124     DSA_VER_S+ 9 : douta <=
{DSA_UOP_WR_CORE, CONST_ONE_ID, UOP_OPR_LAMBDA};
125     DSA_VER_S+ 10 : douta <= {DSA_UOP_VERI-
FY0, NOP_ID, UOP_OPR_DONTCARE};
126     DSA_VER_S+ 11 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
127     DSA_VER_S+ 12 : douta <=
{DSA_UOP_RD_CORE, SCALAR_G_ID, UOP_OPR_SCALAR_G};
128     DSA_VER_S+ 13 : douta <=
{DSA_UOP_RD_CORE, SCALAR_PK_ID, UOP_OPR_SCALAR_PK};
129     DSA_VER_S+ 14 : douta <= {DSA_UOP_WR_SCAL-
AR, SCALAR_G_ID, UOP_OPR_DONTCARE};
130     DSA_VER_S+ 15 : douta <= {DSA_UOP_VERI-
FY1, NOP_ID, UOP_OPR_DONTCARE};
131     DSA_VER_S+ 16 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
132     DSA_VER_S+ 17 : douta <=
{DSA_UOP_WR_CORE, PUBKEYX_ID, UOP_OPR_Qx_AFFN};
133     DSA_VER_S+ 18 : douta <=
{DSA_UOP_WR_CORE, PUBKEYY_ID, UOP_OPR_Qy_AFFN};
134     DSA_VER_S+ 19 : douta <= {DSA_UOP_WR_SCAL-
AR, SCALAR_PK_ID, UOP_OPR_DONTCARE};
135     DSA_VER_S+ 20 : douta <= {DSA_UOP_VERI-
FY2, NOP_ID, UOP_OPR_DONTCARE};
136     DSA_VER_S+ 21 : douta <=
{DSA_UOP_NOP, NOP_ID, UOP_OPR_DONTCARE};
137     DSA_VER_S+ 22 : douta <=
{DSA_UOP_RD_CORE, VERIFY_R_ID, UOP_OPR_Qx_AFFN};

```

```

138      DSA_VER_S+ 23 : douta <=
{DSA_UOP_NOP,      NOP_ID,      UOP_OPR_DONTCARE};
139
140
141      //DH_SHARED_KEY
142      DH_SHARED_S : douta <=
{DSA_UOP_WR_CORE,  CONST_E_b_MONT_ID, UOP_OPR_CONST_E_b};
143
144      DH_SHARED_S+ 1 : douta <=
{DSA_UOP_WR_CORE,  PUBKEYX_ID,      UOP_OPR_Qx_AFFN};
145
146      DH_SHARED_S+ 2 : douta <=
{DSA_UOP_WR_CORE,  PUBKEYY_ID,      UOP_OPR_Qy_AFFN};
147
148      DH_SHARED_S+ 3 : douta <=
{DSA_UOP_PK_CHK,   NOP_ID,      UOP_OPR_DONTCARE};
149
150      DH_SHARED_S+ 4 : douta <=
{DSA_UOP_NOP,      NOP_ID,      UOP_OPR_DONTCARE};
151
152      DH_SHARED_S+ 5 : douta <=
{DSA_UOP_RD_CORE,  PK_VALID_ID,   UOP_OPR_PK_VALID};
153
154      DH_SHARED_S+ 6 : douta <= {DSA_UOP_H-
MAC_DRBG,          NOP_ID,      UOP_OPR_DONTCARE};
155
156      DH_SHARED_S+ 7 : douta <=
{DSA_UOP_NOP,      NOP_ID,      UOP_OPR_DONTCARE};
157
158      DH_SHARED_S+ 8 : douta <=
{DSA_UOP_WR_CORE,  PRIVKEY_ID,   UOP_OPR_SCALAR_PK};
159
160      DH_SHARED_S+ 9 : douta <=
{DSA_UOP_RD_CORE,  SCALAR_G_ID,   UOP_OPR_SCALAR_PK};
161
162      DH_SHARED_S+ 10 : douta <= {DSA_UOP_SCAL-
AR_SCA,            SCALAR_G_ID,   UOP_OPR_DONTCARE};
163
164      DH_SHARED_S+ 11 : douta <=
{DSA_UOP_NOP,      NOP_ID,      UOP_OPR_DONTCARE};
165
166      DH_SHARED_S+ 12 : douta <= {DSA_UOP_WR_S-
CALAR,             SCALAR_ID,     UOP_OPR_DONTCARE};
167
168      DH_SHARED_S+ 13 : douta <=
{DSA_UOP_WR_CORE,  LAMBDA_ID,    UOP_OPR_LAMBDA};
169
170      DH_SHARED_S+ 14 : douta <=
{DH_UOP_SHAREDKEY, NOP_ID,      UOP_OPR_DONTCARE};
171
172      DH_SHARED_S+ 15 : douta <=
{DSA_UOP_NOP,      NOP_ID,      UOP_OPR_DONTCARE};
173
174      DH_SHARED_S+ 16 : douta <=
{DSA_UOP_RD_CORE,  DH_SHAREDKEY_ID, UOP_OPR_Qx_AFFN};
175
176      DH_SHARED_S+ 17 : douta <=
{DSA_UOP_NOP,      NOP_ID,      UOP_OPR_DONTCARE};
177
178      default :      douta <=
{DSA_UOP_NOP,      NOP_ID,      UOP_OPR_DONTCARE};
179
180      endcase
181    end
182  end
183 endmodule

```

7. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_dsa_uop_pkg.sv (126 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // cense");
5 // you may not use this file except in compliance with the Li-
6 // cense.
7 // You may obtain a copy of the License at
8 //
9 // http://www.apache.org/licenses/LICENSE-2.0
10 //
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20 //
21 //
22 //
23 =====
24 //
25 // ecc_dsa_uop_pkg.sv
26 // -----
27 // ECC instructin for the digital signature algorithm (DSA).
28 //
29 //
30 //
31 =====
32
33 `ifndef ECC_DSA_UOP_PKG
34 `define ECC_DSA_UOP_PKG
35
36 package ecc_dsa_uop_pkg;
37
38
39 localparam integer DSA_UOP_ADDR_WIDTH = 9;
40 localparam integer DSA_OPR_ADDR_WIDTH = 6;
41
42
43 localparam DSA_PROG_ADDR_W = 7; // $clo-
44 g2(DSA_VER_E+2);
45 localparam DSA_INSTRUCTION_LENGTH = DSA_UOP_AD-
46 DR_WIDTH + (2*DSA_OPR_ADDR_WIDTH); // opcode + 2 * operand

```

```

35
36 typedef enum logic[3 : 0]
37 {
38     no_cmd      = 4'b0000,
39     keygen_cmd  = 4'b0001,
40     sign_cmd    = 4'b0010,
41     verify0_cmd = 4'b0100,
42     verify1_cmd = 4'b0101,
43     verify2_cmd = 4'b0110,
44     pk_chk_cmd  = 4'b0111,
45     shared_key_cmd = 4'b1000
46 } cmd_t;
47
48 typedef struct packed
49 {
50     logic op_sel;
51     logic wr_en;
52     logic rd_en;
53     cmd_t pm_cmd;
54     logic hmac_drbg_en;
55     logic sca_en;
56 } opcode_t;
57
58 typedef struct packed
59 {
60     opcode_t opcode;
61     logic [DSA_OPR_ADDR_WIDTH-1 : 0] reg_id;
62     logic [DSA_OPR_ADDR_WIDTH-1 : 0] mem_addr;
63 } instr_struct_t;
64
65 // DSA INSTRUCTIONS LIST
66 localparam opcode_t DSA_UOP_NOP = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0000;
67 localparam opcode_t DSA_UOP_WR_CORE = '{op_sel:1'b0,
wr_en:1'b1, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0010;
68 localparam opcode_t DSA_UOP_WR_SCALAR = '{op_sel:1'b1,
wr_en:1'b1, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0011;
69 localparam opcode_t DSA_UOP_RD_CORE = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b1, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_0100;
70 localparam opcode_t DSA_UOP_KEYGEN = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:keygen_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_1000;
71 localparam opcode_t DSA_UOP_SIGN = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:sign_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0001_0000;
72 localparam opcode_t DSA_UOP_VERIFY0 = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:verify0_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0010_0000;
73 localparam opcode_t DSA_UOP_VERIFY1 = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:verify1_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0010_1000;
74 localparam opcode_t DSA_UOP_VERIFY2 = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:verify2_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0011_0000;
75 localparam opcode_t DSA_UOP_PK_CHK = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:pk_chk_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0011_1000;
76 localparam opcode_t DSA_UOP_HMAC_DRBG = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b1,
sca_en:1'b0}; // = 8'b0100_0000;
77 localparam opcode_t DSA_UOP_SCALAR_SCA = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:no_cmd, hmac_drbg_en:1'b0,
sca_en:1'b1}; // = 8'b1000_0000;
78 localparam opcode_t DH_UOP_SHAREDKEY = '{op_sel:1'b0,
wr_en:1'b0, rd_en:1'b0, pm_cmd:shared_key_cmd, hmac_drbg_en:1'b0,
sca_en:1'b0}; // = 8'b0000_1000;
79
80 // DSA REGISTERS ID Listing
81 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] NOP_ID =
6'd0;
82 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ZERO_ID =
6'd1;
83 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ONE_ID =
6'd2;
84 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_E_a_MONT_ID =
6'd3;
85 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_E_3b_MONT_ID =
6'd4;
86 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ONE_p_MONT_ID =
6'd5;
87 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_R2_p_MONT_ID =
6'd6;
88 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_G_X_MONT_ID =
6'd7;
89 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_G_Y_MONT_ID =
6'd8;
90 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_R2_q_MONT_ID =

```

```

6'd9;
91 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_ONE_q_MONT_ID =
6'd10;

92 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] CONST_E_b_MONT_ID =
6'd11;
93

94 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SEED_ID =
6'd16;

95 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] MSG_ID =
6'd17;

96 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PRIVKEY_ID =
6'd18;

97 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PUBKEYX_ID =
6'd19;

98 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PUBKEYY_ID =
6'd20;

99 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] R_ID =
6'd21;

100 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] S_ID =
6'd22;

101 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SCALAR_ID =
6'd23;

102 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SCALAR_G_ID =
6'd24;

103 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] SCALAR_PK_ID =
6'd25;

104 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] VERIFY_R_ID =
6'd26;

105 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] LAMBDA_ID =
6'd27;

106 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] MASKING_ID =
6'd28;

107 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] PK_VALID_ID =
6'd29;

108 localparam [DSA_OPR_ADDR_WIDTH-1 : 0] DH_SHAREDKEY_ID =
6'd30;
109
110
111 // DSA Subroutine listing
112 localparam [DSA_PROG_ADDR_W-1 : 0] ECC_RE-
SET = 7'd0;
113 localparam [DSA_PROG_ADDR_W-1 : 0]
ECC_NOP = 7'd12;
114 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_KG_S = ECC_NOP + 2;
115 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_KG_E = DSA_KG_S + 12;
116 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_SGN_S = DSA_KG_E + 2;
117 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_SGN_E = DSA_SGN_S + 14;
118 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_VER_S = DSA_SGN_E + 2;
119 localparam [DSA_PROG_ADDR_W-1 : 0]
DSA_VER_E = DSA_VER_S + 23;
120 localparam [DSA_PROG_ADDR_W-1 : 0]
DH_SHARED_S = DSA_VER_E + 2;
121 localparam [DSA_PROG_ADDR_W-1 : 0]
DH_SHARED_E = DH_SHARED_S + 17;
122
123
124 endpackage
125
126 `endif

```

8. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_fau.sv (161 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.

```

```

12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_fau.sv
18 // -----
19 // ECC Finite field Arithmetic Unit including Montgomery Mul-
tiplier and Adder
20 //
21 //
22 //
=====
23
24 module ecc_fau #(
25     parameter REG_SIZE      = 384,
26     parameter RADIX         = 32
27 )
28 (
29     // Clock and reset.
30     input wire      clk,
31     input wire      reset_n,
32     input wire      zeroize,
33
34     // DATA PORT
35     input wire      add_en_i,
36     input wire      sub_i,
37     input wire      mult_en_i,
38     input wire [REG_SIZE-1:0] prime_i,
39     input wire [RADIX-1 : 0] mult_mu_i,
40     input wire [REG_SIZE-1:0] opa_i,
41     input wire [REG_SIZE-1:0] opb_i,
42     output wire [REG_SIZE-1:0] add_res_o,
43     output wire [REG_SIZE-1:0] mult_res_o
44 );
45
46 logic [REG_SIZE-1 : 0] mult_res_s;
47 logic [REG_SIZE-1 : 0] mult_opa;
48 logic [REG_SIZE-1 : 0] mult_opb;
49 logic [REG_SIZE-1 : 0] add_res_s;
50
51 reg      mult_start;
52 reg      mult_start_dly;
53 wire     mult_start_edge;
54 logic    add_start;
55 logic    add_start_dly;
56 logic    add_start_edge;
57 reg      sub;
58
59 logic    add_ready_o;
60 logic    mult_ready_o;
61
62 logic    ready_garbage_bit;
63
64 assign mult_opa = opa_i;
65 assign mult_opb = opb_i;
66
67 //-----
68 // MULTIPLIER
69 //-----
70
71 ecc_montgomerymultiplier #(
72     .REG_SIZE(REG_SIZE),
73     .RADIX(RADIX)
74 )
75 i_MULTIPLIER (
76     // Clock and reset.
77     .clk(clk),
78     .reset_n(reset_n),
79     .zeroize(zeroize),
80
81     // DATA PORT
82     .start_i(mult_start_edge),
83     .opa_i(mult_opa),
84     .opb_i(mult_opb),
85     .n_i(prime_i),
86     .n_prime_i(mult_mu_i), // only need the last few bits
87     .p_o(mult_res_s),
88     .ready_o(mult_ready_o)
89 );
90
91 //-----
92 // ADDER/SUBTRACTOR
93 //-----
94
95 ecc_add_sub_mod_alter #(
96     .REG_SIZE(REG_SIZE)
97 )
98 i_ADDER_SUBTRACTOR (
99     .clk(clk),
100     .reset_n(reset_n),
101     .zeroize(zeroize),
102     .add_en_i(add_start_edge),

```

```

103     .sub_i(sub),
104     .opa_i(opa_i),
105     .opb_i(opb_i),
106     .prime_i(prime_i),
107     .res_o(add_res_s),
108     .ready_o(add_ready_o)
109 );
110
111 //-----
112 // Registers including update enables.
113 //-----
114
115 always_ff @(posedge clk or negedge reset_n)
116 begin
117     if (!reset_n) begin
118         mult_start <= '0;
119         add_start <= '0;
120         sub <= '0;
121     end
122     else if (zeroize) begin
123         mult_start <= '0;
124         add_start <= '0;
125         sub <= '0;
126     end
127     else begin
128         mult_start <= mult_en_i;
129         add_start <= add_en_i;
130         sub <= sub_i;
131     end
132 end
133
134 always_ff @(posedge clk or negedge reset_n) begin
135     if (!reset_n) begin
136         mult_start_dly <= '0;
137         add_start_dly <= '0;
138     end
139     else if (zeroize) begin
140         mult_start_dly <= '0;
141         add_start_dly <= '0;
142     end
143     else begin
144         mult_start_dly <= mult_start;
145         add_start_dly <= add_start;
146     end
147 end
148
149 assign ready_garbage_bit = add_ready_o & mult_ready_o;
150
151 // generate a pulse enable for mult
152 assign mult_start_edge = mult_start & ~mult_start_dly;
153 assign add_start_edge = add_start & ~add_start_dly;
154
155 //-----
156 // Concurrent connectivity for ports etc.
157 //-----
158
159 assign mult_res_o = mult_res_s;
160 assign add_res_o = add_res_s;
161 endmodule

```

9. chipsalliance_caliptra-rtl/src/ecc/rtl/ ecc_hmac_drbg_interface.sv (344 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // you may not use this file except in compliance with the Li-
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
12 // press or implied.
13 // See the License for the specific language governing permis-
14 // sions and
15 // limitations under the License.
16 //
17 //-----
18 //
19 // interface with hmac_drbg component to generate the required
20 // values:
21 // 1) in keygen mode:
22 // 1.1. generate lambda from IV for point randomization
23 // 1.2. generate scalar_rnd from IV for scalar blinding

```

```

SCA countermeasure
23 // 1.3. generate privkey from seed and nonce for key gen-
24 // 2) in sign mode:
25 // 2.1. generate lambda from IV for point randomization
SCA countermeasure
26 // 2.2. generate scalar_rnd from IV for scalar blinding
SCA countermeasure
27 // 2.3. generate masking_rnd from IV for masking signa-
28 // 2.4. generate k from privkey and hashed_msg for sign-
29 //
30 // To generate random values using IV, the hmac_drbg is con-
31 // tinued by triggering
32 // next command (instead of init).
33 //
34 // - INIT: Instantiates HMAC-DRBG with entropy,
35 // nonce, (personalization string is empty).
36 // Then, it calls
37 // HMAC_DRBG_Generate_algorithm (additional input string is empty)
38 // to generate a 384-bit random value while
39 // it's in (0, prime) range.
40 // if the random value is zero or >= prime,
41 // it is rejected and a new random value
42 // is generated by calling
43 // HMAC_DRBG_Generate_algorithm.
44 // There is NO RESEED process.
45 // - NEXT: it calls HMAC_DRBG_Generate_algorithm (ad-
46 // ditional input string is empty)
47 // to generate a 384-bit random value while
48 // it's in (0, prime) range.
49 // if the random value is zero or >= prime,
50 // it is rejected and a new random value
51 // is generated by calling
52 // HMAC_DRBG_Generate_algorithm.
53 // There is NO RESEED process.
54 //
55 //-----
56 ==
57 import hmac_param_pkg::*;
58
59 module ecc_hmac_drbg_interface#(
60     parameter REG_SIZE = 384,
61     parameter [REG_SIZE-1 : 0] GROUP_ORDER =
62     384'hfffffffffffffffffffffffffffffffffffffffffffffffffffffc7634d81f4372d
63     df581a0db248b0a77aecec196acc52973
64 )
65 (
66     // Clock and reset.
67     input wire clk,
68     input wire reset_n,
69     input wire zeroize,
70     input wire [1 : 0] hmac_mode,
71     input wire en,
72     output wire ready,
73
74     //Data
75     input wire [REG_SIZE-1 : 0] keygen_seed,
76     input wire [REG_SIZE-1 : 0] keygen_nonce,
77     input wire [REG_SIZE-1 : 0] privKey,
78     input wire [REG_SIZE-1 : 0] hashed_msg,
79     input wire [REG_SIZE-1 : 0] IV,
80
81     output wire [REG_SIZE-1 : 0] lambda,
82     output wire [REG_SIZE-1 : 0] scalar_rnd,
83     output wire [REG_SIZE-1 : 0] masking_rnd,
84     output wire [REG_SIZE-1 : 0] drbg
85 );
86
87 //-----
88 // Registers including update variables and write enable.
89 //-----
90
91 logic [REG_SIZE-1 : 0] lfsr_seed_reg;
92 logic [LFSR_SEED_SIZE-1 : 0] hmac_lfsr_seed;
93
94 logic hmac_drbg_init;
95 logic hmac_drbg_next;
96 logic hmac_drbg_ready;
97 logic hmac_drbg_valid;
98 logic [REG_SIZE-1 : 0] hmac_drbg_entropy;
99 logic [REG_SIZE-1 : 0] hmac_drbg_nonce;
100 logic [REG_SIZE-1 : 0] hmac_drbg_result;
101
102 logic first_round;
103 logic [REG_SIZE-1 : 0] lambda_reg;
104 logic [REG_SIZE-1 : 0] scalar_rnd_reg;
105 logic [REG_SIZE-1 : 0] masking_rnd_reg;
106 logic [REG_SIZE-1 : 0] drbg_reg;
107 logic hmac_drbg_valid_last;
108 logic hmac_done_edge;
109
110 logic [63 : 0] counter_reg;
111 logic [REG_SIZE-1 : 0] counter_nonce;
112 logic [REG_SIZE-1 : 0] counter_nonce_reg;
113 logic [REG_SIZE-1 : 0] sca_entropy, sca_entropy_reg;
114
115 localparam [1 : 0] KEYGEN_CMD = 2'b00;

```



```

102 localparam [1 : 0] SIGN_CMD      = 2'b01;
103 localparam [1 : 0] DH_SHARED_CMD = 2'b10;
104
105 /*State register*/
106 reg [3 : 0] state_reg;
107 reg [3 : 0] state_next;
108 reg [3 : 0] state_reg_last;
109
110 /*STATES*/
111 localparam [3 : 0] IDLE_ST      = 4'd0;
112 localparam [3 : 0] LFSR_ST     = 4'd1;
113 localparam [3 : 0] LAMBDA_ST   = 4'd2;
114 localparam [3 : 0] SCALAR_RND_ST = 4'd3;
115 localparam [3 : 0] RND_DONE_ST = 4'd4;
116 localparam [3 : 0] MASKING_RND_ST = 4'd5;
117 localparam [3 : 0] KEYGEN_ST   = 4'd6;
118 localparam [3 : 0] SIGN_ST     = 4'd7;
119 localparam [3 : 0] DONE_ST     = 4'd8;
120
121 //-----
122 // Module instantiations.
123 //-----
124
125 hmac_drbg #(
126     .REG_SIZE(REG_SIZE),
127     .HMAC_DRBG_PRIME(GROUP_ORDER)
128 )
129 hmac_drbg_i (
130     .clk(clk),
131     .reset_n(reset_n),
132     .zeroize(zeroize),
133     .init_cmd(hmac_drbg_init),
134     .next_cmd(hmac_drbg_next),
135     .ready(hmac_drbg_ready),
136     .valid(hmac_drbg_valid),
137     .lfsr_seed(hmac_lfsr_seed),
138     .entropy(hmac_drbg_entropy),
139     .nonce(hmac_drbg_nonce),
140     .drbg(hmac_drbg_result)
141 );
142
143 genvar i;
144 generate
145     for (i=0; i < LFSR_SEED_SIZE/32; i++) begin : gen_lfsr
146         caliptra_prim_lfsr
147         #(
148             .LfsrType("FIB_XNOR"),
149             .LfsrDw(32),
150             .StateOutDw(32)
151         ) caliptra_prim_lfsr_inst_i
152         (
153             .clk_i(clk),
154             .rst_ni(reset_n),
155             .seed_en_i(hmac_drbg_init),
156             .seed_i(sca_entropy[i*32 +: 32]),
157             .lfsr_en_i(1'b1),
158             .entropy_i('0),
159             .state_o(hmac_lfsr_seed[i*32 +: 32])
160         );
161     end
162 endgenerate
163
164 //-----
165 // hmac_drbg_interface_logic
166 //
167 // The logic needed to init as well as update the
168 // hmac_drbg commands.
169 //-----
170
171 always_comb first_round = (state_reg == state_reg_last)?
172 1'b0 : 1'b1;
173 always_comb hmac_done_edge = hmac_drbg_valid & (!
174 hmac_drbg_valid_last);
175
176 always_comb
177 begin : hmac_drbg_entropy_input
178     unique case (state_reg)
179         LFSR_ST:      hmac_drbg_entropy = sca_en-
180 tropy_reg;
181         LAMBDA_ST:    hmac_drbg_entropy = sca_en-
182 tropy_reg;
183         SCALAR_RND_ST: hmac_drbg_entropy = sca_en-
184 tropy_reg;
185         MASKING_RND_ST: hmac_drbg_entropy = sca_en-
186 tropy_reg;
187         KEYGEN_ST:    hmac_drbg_entropy = keygen_seed;
188         SIGN_ST:      hmac_drbg_entropy = privKey;
189         default:      hmac_drbg_entropy = sca_en-
190 tropy_reg;
191     endcase
192 end // hmac_drbg_entropy_input
193
194 always_comb
195 begin : hmac_drbg_nonce_input
196     unique case (state_reg)
197         LFSR_ST:      hmac_drbg_nonce =
198 counter_nonce_reg;
199         LAMBDA_ST:    hmac_drbg_nonce =
200 counter_nonce_reg;
201         SCALAR_RND_ST: hmac_drbg_nonce =
202 counter_nonce_reg;
203         MASKING_RND_ST: hmac_drbg_nonce =
204 counter_nonce_reg;
205         KEYGEN_ST:    hmac_drbg_nonce = keygen_nonce;
206         SIGN_ST:      hmac_drbg_nonce = hashed_msg;
207         default:      hmac_drbg_nonce =
208 counter_nonce_reg;
209     endcase
210 end // hmac_drbg_nonce_input
211
212 always_comb
213 begin : hmac_trigger
214     hmac_drbg_init = 0;
215     hmac_drbg_next = 0;
216     if (first_round) begin
217         unique case (state_reg)
218             LFSR_ST:      hmac_drbg_init = 1;
219             LAMBDA_ST:    hmac_drbg_next = 1;
220             SCALAR_RND_ST: hmac_drbg_next = 1;
221             MASKING_RND_ST: hmac_drbg_next = 1;
222             KEYGEN_ST:    hmac_drbg_init = 1;
223             SIGN_ST:      hmac_drbg_init = 1;
224             default:      hmac_drbg_init = 0;
225             hmac_drbg_next = 0;
226         end
227     end
228 end // hmac_trigger
229
230 //-----
231 // register updates
232 //
233 // update the internal registers
234 //-----
235
236 always_ff @(posedge clk or negedge reset_n)
237 begin //reg_update
238     if (!reset_n) begin
239         lambda_reg <= '0;
240         scalar_rnd_reg <= '0;
241         masking_rnd_reg <= '0;
242         drbg_reg <= '0;
243         lfsr_seed_reg <= '0;
244     end
245     else if (zeroize) begin
246         lambda_reg <= '0;
247         scalar_rnd_reg <= '0;
248         masking_rnd_reg <= '0;
249         drbg_reg <= '0;
250         //lfsr_seed_reg <= '0; // without zeroize to make
251 it more complex
252     end
253     else
254         if (hmac_done_edge) begin
255             unique case (state_reg) inside
256                 LFSR_ST:      lfsr_seed_reg <=
257 hmac_drbg_result;
258                 LAMBDA_ST:    lambda_reg <=
259 hmac_drbg_result;
260                 SCALAR_RND_ST: scalar_rnd_reg <=
261 hmac_drbg_result;
262                 MASKING_RND_ST: masking_rnd_reg <=
263 hmac_drbg_result;
264                 KEYGEN_ST:    drbg_reg <=
265 hmac_drbg_result;
266                 SIGN_ST:      drbg_reg <=
267 hmac_drbg_result;
268                 default:      begin
269                     lambda_reg <= '0;
270                     scalar_rnd_reg <= '0;
271                     masking_rnd_reg <= '0;
272                     drbg_reg <= '0;
273                 end
274             endcase
275         end
276     end //reg_update
277
278 always_ff @(posedge clk or negedge reset_n)
279 begin : state_reg_update
280     if (!reset_n)
281         state_reg <= IDLE_ST;
282     else if (zeroize)
283         state_reg <= IDLE_ST;
284     else
285         state_reg <= state_next;
286     end // state_reg_update
287
288 always_ff @(posedge clk or negedge reset_n)
289 begin : ff_state_reg
290     if (!reset_n)
291         state_reg_last <= IDLE_ST;
292     else if (zeroize)
293         state_reg_last <= IDLE_ST;
294     else
295         state_reg_last <= state_reg;
296     end // ff_state_reg

```

```

278
279 always_ff @(posedge clk or negedge reset_n)
280 begin : ff_hamc_valid
281     if (!reset_n)
282         hmac_drbg_valid_last <= '0;
283     else if (zeroize)
284         hmac_drbg_valid_last <= '0;
285     else
286         hmac_drbg_valid_last <= hmac_drbg_valid;
287 end //ff_hamc_valid
288
289 // without zeroize to make it more complex
290 always_ff @(posedge clk or negedge reset_n)
291 begin : counter_reg_update
292     if (!reset_n)
293         counter_reg <= '0;
294     else
295         counter_reg <= counter_reg + 1;
296 end // counter_reg_update
297
298 // without zeroize to make it more complex
299 always_ff @(posedge clk or negedge reset_n)
300 begin
301     if (!reset_n) begin
302         counter_nonce_reg <= '0;
303         sca_entropy_reg <= '0;
304     end else if (en && (state_reg == IDLE_ST)) begin
305         counter_nonce_reg <= counter_nonce;
306         sca_entropy_reg <= sca_entropy;
307     end
308 end
309
310 always_comb counter_nonce = {counter_reg, counter_reg,
counter_reg, counter_reg, counter_reg, counter_reg};
311 always_comb sca_entropy = IV ^ lfsr_seed_reg ^
counter_nonce;
312 //-----
313 // FSM_flow
314 //
315 // This FSM starts with the en command to perform HMAC-
DRBG.
316 // Active low and async reset.
317 //-----
318 always_comb
319 begin : interface_fsm
320     state_next = IDLE_ST;
321     unique case(state_reg)
322         IDLE_ST: state_next = (en &
hmac_drbg_ready)? LFSR_ST : IDLE_ST;
323         LFSR_ST: state_next = (hmac_done_edge)?
LAMBDA_ST : LFSR_ST;
324         LAMBDA_ST: state_next = (hmac_done_edge)?
SCALAR_RND_ST : LAMBDA_ST;
325         SCALAR_RND_ST: state_next = (hmac_done_edge)?
RND_DONE_ST : SCALAR_RND_ST;
326         RND_DONE_ST: state_next = (hmac_mode ==
SIGN_CMD)? MASKING_RND_ST : (hmac_mode == KEYGEN_CMD)? KEYGEN_ST :
DONE_ST;
327         MASKING_RND_ST: state_next = (hmac_done_edge)?
SIGN_ST : MASKING_RND_ST;
328         KEYGEN_ST: state_next = (hmac_done_edge)?
DONE_ST : KEYGEN_ST;
329         SIGN_ST: state_next = (hmac_done_edge)?
DONE_ST : SIGN_ST;
330         DONE_ST: state_next = IDLE_ST;
331         default: state_next = IDLE_ST;
332     endcase
333 end // interface_fsm
334
335 //-----
336 // Concurrent connectivity for ports etc.
337 //-----
338 assign lambda = lambda_reg;
339 assign scalar_rnd = scalar_rnd_reg;
340 assign masking_rnd = masking_rnd_reg;
341 assign drbg = drbg_reg;
342 assign ready = (state_reg == IDLE_ST);
343
344 endmodule

```

10. chipsalliance_caliptra-rtl/src/ecc/rtl/ ecc_montgomerymultiplier.sv (375 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
sense");
4 // you may not use this file except in compliance with the Li-
sense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"

```

```

BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 // Module:
17 // ecc_montgomerymultiplier.sv
18 //
19 // Montgomery Multiplier based on
20 // Optimized Algorithms and Architectures for Montgomery Mul-
tiplication
21 // for Post-quantum Cryptography by Rami Elkhatab et. al.
22 //
23 // Description:
24 // The module produces a Montgomery product of input op-
erands with the following constraints:
25 // Input operands must be less than n_i, i.e. opa_i <
n_i and opb_i < n_i.
26 // RADIX > 2
27 // R = 2^((ceil($bits(n_i) / RADIX) + 1) * RADIX)
28 // R > n_i.
29 // (1) T = opa_i * opb_i < n_i * n_i = n_i^2
30 // (2) p_internal = (T + (T*n_prime mod R) * n_i) / R
31 // (3) p_subtracted_internal = p_internal - n_i
32 // (4) p_o = (p_internal >= ni)? p_subtracted_internal :
p_internal
33 // From (1) and (2) given R > n_i, and opa_i, opb_i <
n_i:
34 // p_internal < (n_i^2 + R * n_i) / R = n_i + n_i^2 / R
< n_i + n_i^2 / n_i = n_i + n_i = 2*n_i
35 // From (3) and (4):
36 // p_subtracted_internal < 2*n_i - n_i = n_i
37 // p_o < n_i
38 //
=====
39 module ecc_montgomerymultiplier #(
40     parameter REG_SIZE = 384,
41     parameter RADIX = 32
42 )
43 (
44     // Clock and reset.
45     input wire clk,
46     input wire reset_n,
47     input wire zeroize,
48
49     // DATA PORT
50     input wire start_i,
51     input wire [REG_SIZE-1:0] opa_i,
52     input wire [REG_SIZE-1:0] opb_i,
53     input wire [REG_SIZE-1:0] n_i,
54     input wire [RADIX-1:0] n_prime_i, // only need the
last few bits
55     output logic [REG_SIZE-1:0] p_o,
56     output logic ready_o
57 );
58 //-----
59 // Local Parameters
60 //-----
61
62 // Equivalent to $ceil(real'(REG_SIZE) / RADIX) + 1;
63 localparam int unsigned S_NUM = ((REG_SIZE + RADIX -
1) / RADIX) + 1;
64 localparam int unsigned FULL_REG_SIZE = S_NUM * RADIX;
65 // Each PE performs two iterations out of S_NUM - 1
66 // See section 4.2 of Elkhatab_2019 paper.
67 // PE_UNITS does not include the last PE.
68 // The last PE is instantiated separately.
69 localparam PE_UNITS = ((S_NUM - 1) / 2) - 1;
70 localparam [(FULL_REG_SIZE-REG_SIZE)-1 : 0]
zero_pad = '0;
71
72 //-----
73 // Registers
74 //-----
75
76 logic [RADIX-1 : 0] a_array[0:PE_UNITS+1];
77 logic [RADIX-1 : 0] b_array[0:PE_UNITS+1];
78 logic [RADIX-1 : 0] p_array[0:PE_UNITS+1];
79 logic [RADIX-1 : 0] m_array[0:PE_UNITS];
80 logic [RADIX : 0] c_array[0:PE_UNITS+1];
81 logic [RADIX-1 : 0] s_array[0:PE_UNITS+1];
82
83 logic [RADIX-1 : 0] p_neg_array[0:2*(PE_UNITS+1)];
84 logic [RADIX-1 : 0] t_reg[0:2*(PE_UNITS+1)];
85 logic [RADIX-1 : 0] t_subtracted_reg[0:2*(PE_UNITS+1)];
86 logic [RADIX : 0] sub_res[0:2*(PE_UNITS+1)];
87
88 logic sub_b_i[0:2*(PE_UNITS+1)];
89 logic sub_b_o[0:2*(PE_UNITS+1)];

```



```

90
91 logic [FULL_REG_SIZE-1 : 0] a_reg; //extended with
zero
92 logic [FULL_REG_SIZE-1 : 0] b_reg; //extended with
zero
93 logic [FULL_REG_SIZE-1 : 0] p_reg; //extended with
zero
94 logic [FULL_REG_SIZE-1 : 0] p_neg_reg; //extended
with one
95 logic [RADIX-1:0] n_prime_reg;
96 logic [(3*S_NUM)-1 : 0] push_reg;
97 logic push_reg_eq_zero;
98 logic push_reg_eq_zero_ff;
99 logic odd;
100 logic [RADIX-1 : 0] last_s_reg;
101 logic [FULL_REG_SIZE-1:0] p_internal;
102 logic [FULL_REG_SIZE-1:0] p_subtracted_internal;
103 logic last_reduction;
104
105 //-----
106 // Processing elements (PEs)
107 //-----
108 ecc_pe_first #(
109     .RADIX(RADIX)
110 )
111 first_box
112 (
113     .clk(clk),
114     .reset_n(reset_n),
115     .zeroize(zeroize),
116     .start_in(start_i),
117     .a_in(a_array[0]),
118     .b_in(b_array[0]),
119     .p_in(p_array[0]),
120     .s_in(s_array[0]),
121     .n_prime_in(n_prime_reg),
122     .odd(odd),
123
124     .a_out(a_array[1]),
125     .m_out(m_array[0]),
126     .c_out(c_array[0])
127 );
128
129 genvar i0;
130 generate
131     for (i0=0; i0 < PE_UNITS; i0++) begin : gen_PE //PE
general boxes
132         ecc_pe #(
133             .RADIX(RADIX)
134         )
135         box_i
136         (
137             .clk(clk),
138             .reset_n(reset_n),
139             .zeroize(zeroize),
140             .start_in(start_i),
141             .a_in(a_array[i0+1]),
142             .b_in(b_array[i0+1]),
143             .p_in(p_array[i0+1]),
144             .m_in(m_array[i0]),
145             .s_in(s_array[i0+1]),
146             .c_in(c_array[i0]),
147             .odd(odd),
148
149             .a_out(a_array[i0+2]),
150             .m_out(m_array[i0+1]),
151             .s_out(s_array[i0]),
152             .c_out(c_array[i0+1])
153         );
154     end
155 endgenerate
156
157 ecc_pe_final #(
158     .RADIX(RADIX)
159 )
160 final_box
161 (
162     .clk(clk),
163     .reset_n(reset_n),
164     .zeroize(zeroize),
165     .start_in(start_i),
166     .a_in(a_array[PE_UNITS+1]),
167     .b_in(b_array[PE_UNITS+1]),
168     .p_in(p_array[PE_UNITS+1]),
169     .m_in(m_array[PE_UNITS]),
170     .s_in(s_array[PE_UNITS+1]),
171     .c_in(c_array[PE_UNITS]),
172     .odd(odd),
173
174     .s_out(s_array[PE_UNITS]),
175     .c_out(c_array[PE_UNITS+1])
176 );
177
178 // Determines the input s for the final PE
179 assign s_array[PE_UNITS+1] = last_s_reg;
180 always_ff @(posedge clk or negedge reset_n)
181 begin
182     if (~reset_n)
183         last_s_reg <= 'b0;

```

```

184     else if (zeroize)
185         last_s_reg <= 'b0;
186     else if (start_i)
187         last_s_reg <= 'b0;
188     else
189         last_s_reg <= c_array[PE_UNITS+1][RADIX-1:0];
190 end
191
192 // Set odd flag
193 always_ff @(posedge clk or negedge reset_n)
194 begin
195     if (~reset_n)
196         odd <= 0;
197     else if (zeroize)
198         odd <= 0;
199     else if (start_i)
200         odd <= 1'b1;
201     else
202         odd <= ~odd;
203 end
204
205 always_ff @(posedge clk or negedge reset_n)
206 begin : input_reg
207     if (~reset_n) begin
208         a_reg <= 'b0;
209         b_reg <= 'b0;
210         p_reg <= 'b0;
211         n_prime_reg <= 'b0;
212     end
213     else if (zeroize) begin
214         a_reg <= 'b0;
215         b_reg <= 'b0;
216         p_reg <= 'b0;
217         n_prime_reg <= 'b0;
218     end
219     else if (start_i) begin
220         a_reg <= {zero_pad, opa_i};
221         b_reg <= {zero_pad, opb_i};
222         p_reg <= {zero_pad, n_i};
223         n_prime_reg <= n_prime_i;
224     end else begin
225         if (odd) begin
226             a_reg[(FULL_REG_SIZE-RADIX)-1 : 0] <=
a_reg[FULL_REG_SIZE-1 : RADIX];
227             a_reg[FULL_REG_SIZE-1 : FULL_REG_SIZE-RADIX]
<= 'b0; //Shift
228         end
229     end
230 end // input_reg
231
232 assign a_array[0] = a_reg[RADIX-1 : 0];
233
234 // Determines which b and p is pushed through the system
235 assign b_array[0] = b_reg[RADIX-1 : 0];
236 assign p_array[0] = p_reg[RADIX-1 : 0];
237 genvar j0;
238 generate
239     for (j0=1; j0 < (PE_UNITS+2); j0++) begin :
gen_b_p_array
240         assign b_array[j0] = odd ?
b_reg[(((2*j0)+1)*RADIX)-1 : (2*j0)*RADIX] :
b_reg[((2*j0)*RADIX)-1 : ((2*j0)-1)*RADIX];
241         assign p_array[j0] = odd ?
p_reg[(((2*j0)+1)*RADIX)-1 : (2*j0)*RADIX] :
p_reg[((2*j0)*RADIX)-1 : ((2*j0)-1)*RADIX];
242     end
243 endgenerate
244
245 assign p_neg_reg = ~p_reg;
246
247 genvar i1;
248 generate
249     for (i1=0; i1 < ((2*(PE_UNITS+1))+1); i1++) begin :
gen_n_neg_array
250         assign p_neg_array[i1] = p_neg_reg[i1*RADIX + :
RADIX];
251     end
252 endgenerate
253
254 // Storing the partial results from the system
255 genvar t0;
256 generate
257     for (t0=0; t0 < ((2*(PE_UNITS+1))+1); t0++) begin :
gen_t_reg
258         if ((t0 == 0) | (t0 == 1)) begin
259             always_ff @(posedge clk or negedge reset_n)
260             begin
261                 if (~reset_n)
262                     t_reg[t0] <= '0;
263                 else if (zeroize)
264                     t_reg[t0] <= '0;
265                 else if (push_reg[(2*(PE_UNITS+1)) - t0])
266                     t_reg[t0] <= s_array[0];
267                 else
268                     t_reg[t0] <= t_reg[t0];
269             end
270         end
271     end begin
272         localparam int t0_WIDTH = $clog2((2*(PE_UNITS+1))+1);

```

```

273      localparam bit [t0_WIDTH-1:0] t0_unsigned =
t0[t0_WIDTH-1:0];
274      always_ff @(posedge clk or negedge reset_n)
begin
275          if (~reset_n)
276              t_reg[t0] <= '0;
277          else if (zeroize)
278              t_reg[t0] <= '0;
279          else if (push_reg[(2*(PE_UNITS+1)) - t0])
280              t_reg[t0] <= s_array[t0_unsigned >>
1];
281          else
282              t_reg[t0] <= t_reg[t0];
283          end
284      end
285      end // gen_t_reg
286      endgenerate
287
288      genvar t1;
289      generate
290          for (t1=0; t1 < ((2*(PE_UNITS+1))+1); t1++) begin :
gen_sub_t
291              if (t1 == 0)
292                  always_comb sub_b_i[t1] = 1;
293              else
294                  always_comb sub_b_i[t1] = sub_b_o[t1 - 1];
295
296              always_ff @(posedge clk or negedge reset_n) begin
297                  if (~reset_n)
298                      sub_b_o[t1] <= '0;
299                  else if (zeroize)
300                      sub_b_o[t1] <= '0;
301                  else
302                      sub_b_o[t1] <= sub_res[t1][RADIX];
303              end
304          end
305      endgenerate
306
307      genvar t2;
308      generate
309          for (t2=0; t2 < ((2*(PE_UNITS+1))+1); t2++)
310              begin : gen_sub_res_t
311                  //if (~reset_n)
312                  //    sub_res[t2] = 0;
313                  //else if (push_reg[2*(PE_UNITS+1) - t2])
314                  localparam int t2_WIDTH = $clog2((2*(PE_UNITS+1))
+1);
315                  localparam bit [t2_WIDTH-1:0] t2_unsigned =
t2[t2_WIDTH-1:0];
316                  if ((t2 == 0) | (t2 == 1))
317                      always_comb sub_res[t2] = s_array[0] +
p_neg_array[t2] + sub_b_i[t2];
318                  else
319                      always_comb sub_res[t2] = s_array[t2_unsigned
>> 1] + p_neg_array[t2] + sub_b_i[t2];
320                  end // gen_sub_res_t
321              endgenerate
322
323              genvar t3;
324              generate
325                  for (t3=0; t3 < ((2*(PE_UNITS+1))+1); t3++) begin :
gen_t_sub_reg
326                      always_ff @(posedge clk or negedge reset_n) begin
327                          if (~reset_n)
328                              t_subtracted_reg[t3] <= '0;
329                          else if (zeroize)
330                              t_subtracted_reg[t3] <= '0;
331                          else if (push_reg[(2*(PE_UNITS+1)) - t3])
332                              t_subtracted_reg[t3] <= sub_res[t3]
[RADIX-1 : 0];
333                          else
334                              t_subtracted_reg[t3] <= t_subtrac-
ted_reg[t3];
335                      end
336                  end // gen_t_sub_reg
337              endgenerate
338
339              // Storing the final results from the system
340              genvar k0;
341              generate
342                  for (k0=0; k0 < ((2*(PE_UNITS+1))+1); k0++) begin :
gen_p_o
343                      assign p_internal[k0*RADIX +: RADIX] = t_reg[k0];
344                      assign p_subtracted_internal[k0*RADIX +: RADIX] =
t_subtracted_reg[k0];
345                  end
346              endgenerate
347
348              always_comb last_reduction = (sub_b_o[2*(PE_UNITS+1)]);
349              assign p_o = (last_reduction)? p_subtrac-
ted_internal[REG_SIZE-1:0] : p_internal[REG_SIZE-1:0];
350
351              // Determines when results are ready based on S_NUM
352              always_ff @(posedge clk or negedge reset_n) begin
353                  if (!reset_n)
354                      push_reg <= 'b0;
355                  else if (zeroize)
356                      push_reg <= 'b0;
357                  else if (start_i)
358                      push_reg[(3*S_NUM)-1] <= 'b1;
359                  else // one shift to right

```

```

360          push_reg <= {1'b0, push_reg[(3*S_NUM)-1 : 1]};
361      end
362
363      always_ff @(posedge clk or negedge reset_n) begin
364          if (!reset_n)
365              push_reg_eq_zero_ff <= '0;
366          else if (zeroize)
367              push_reg_eq_zero_ff <= '0;
368          else
369              push_reg_eq_zero_ff <= push_reg_eq_zero;
370      end
371
372      assign push_reg_eq_zero = push_reg == 'b0;
373      assign ready_o = push_reg_eq_zero & ~push_reg_eq_zero_ff;
374
375      endmodule

```

11. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_mult_dsp.sv (39 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_mult_dsp.sv
18 // -----
19 // General multiplier to perform P = A*B.
20 //
21 //
22 //
=====
23
24 module ecc_mult_dsp #(
25     parameter RADIX = 32
26 )
27 (
28     // DATA PORT
29     input wire [RADIX-1 : 0]    A_i,
30     input wire [RADIX-1 : 0]    B_i,
31
32     output logic [(2*RADIX)-1 : 0]    P_o
33 );
34
35     always_comb begin
36         P_o = A_i * B_i;
37     end
38
39 endmodule

```

12. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_params_pkg.sv (77 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_params_pkg.sv
18 // -----

```

[illegible]

13. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe.sv (162 lines)

```
1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
```

```

cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
==
16 //
17 // ecc_pe.sv
18 // -----
19 // processing element for Montgomery Multiplier based on
20 // Optimized Algorithms and Architectures for Montgomery Mul-
tiplication
21 // for Post-quantum Cryptography by Rami Elkhatib et. al.
22 //
23 //
=====
==
24
25 module ecc_pe #(
26     parameter RADIX = 32
27 )
28 (
29     // Clock and reset.
30     input wire          clk,
31     input wire          reset_n,
32     input wire          zeroize,
33
34     input wire          start_in,
35     // DATA PORT
36     input wire [RADIX-1:0] a_in,
37     input wire [RADIX-1:0] b_in,
38     input wire [RADIX-1:0] p_in,
39     input wire [RADIX-1:0] m_in,
40     input wire [RADIX-1:0] s_in,
41     input wire [RADIX :0] c_in,
42     input wire          odd,
43
44     output logic [RADIX-1:0] a_out,
45     output logic [RADIX-1:0] m_out,
46     output logic [RADIX-1:0] s_out,
47     output logic [RADIX :0] c_out
48 );
49
50
51 //-----
52
53 // ecc_pe
54 //-----
55
56 logic [(2*RADIX)-1 : 0] mult0_out;
57 logic [(2*RADIX)-1 : 0] mult1_out;
58
59 logic [RADIX : 0] c_mux;
60 logic [RADIX-1 : 0] s_mux;
61
62 logic [2*RADIX : 0] res;
63 logic [RADIX-1 : 0] res_LSW;
64 logic [RADIX : 0] res_MSW;
65
66 logic carry_garbage_bit;
67
68 //-----
69
70 // Module instantiations.
71 //-----
72
73 ecc_mult_dsp #(
74     .RADIX(RADIX)
75 ) MULT1 (
76     .A_i(a_in),
77     .B_i(b_in),
78     .P_o(mult0_out)
79 );
80
81 ecc_mult_dsp #(
82     .RADIX(RADIX)
83 ) MULT2 (
84     .A_i(p_in),
85     .B_i(m_in),
86     .P_o(mult1_out)
87 );
88
89 //-----
90
91 // pe_logic
92 //
93 // The logic needed to update the multiplier inputs.

```

```

89 //-----
90 assign c_mux = odd ? c_out : c_in;
91 assign s_mux = odd ? s_in : s_out;
92
93 assign res_MSW = res[2*RADIX : RADIX];
94 assign res_LSW = res[RADIX-1 : 0];
95
96 always_comb {carry_garbage_bit, res} = mult0_out + mul-
t1_out + c_mux + s_mux;
97
98 //-----
99 // register updates
100 //
101 //-----
102 always_ff @(posedge clk or negedge reset_n) begin
103     if (~reset_n) begin
104         c_out <= 'b0;
105     end
106     else if (zeroize) begin
107         c_out <= 'b0;
108     end
109     else if (start_in) begin
110         c_out <= 'b0;
111     end
112     else begin
113         c_out <= res_MSW;
114     end
115 end
116 always_ff @(posedge clk or negedge reset_n) begin
117     if (~reset_n) begin
118         s_out <= 'b0;
119     end
120     else if (zeroize) begin
121         s_out <= 'b0;
122     end
123     else if (start_in) begin
124         s_out <= 'b0;
125     end
126     else begin
127         s_out <= res_LSW;
128     end
129 end
130 always_ff @(posedge clk or negedge reset_n) begin
131     if (~reset_n) begin
132         a_out <= 'b0;
133     end
134     else if (zeroize) begin
135         a_out <= 'b0;
136     end
137     else if (start_in) begin
138         a_out <= 'b0;
139     end
140     else begin
141         if (odd) begin
142             a_out <= a_in;
143         end
144     end
145 end
146 always_ff @(posedge clk or negedge reset_n) begin
147     if (~reset_n) begin
148         m_out <= 'b0;
149     end
150     else if (zeroize) begin
151         m_out <= 'b0;
152     end
153     else if (start_in) begin
154         m_out <= 'b0;
155     end
156     else begin
157         if (odd) begin
158             m_out <= m_in;
159         end
160     end
161 end
162 endmodule

```

14. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_final.sv (128 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and

```

```

13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_pe_final.sv
18 // -----
19 // final processing element for Montgomery Multiplier based
on
20 // Optimized Algorithms and Architectures for Montgomery Mul-
tiplication
21 // for Post-quantum Cryptography by Rami Elkhatab et. al.
22 //
23 //
=====
24
25 module ecc_pe_final #(
26     parameter RADIX = 32
27 )
28 (
29     // Clock and reset.
30     input wire          clk,
31     input wire          reset_n,
32     input wire          zeroize,
33
34     input wire          start_in,
35     // DATA PORT
36     input wire [RADIX-1:0] a_in,
37     input wire [RADIX-1:0] b_in,
38     input wire [RADIX-1:0] p_in,
39     input wire [RADIX-1:0] m_in,
40     input wire [RADIX-1:0] s_in,
41     input wire [RADIX :0] c_in,
42     input wire          odd,
43
44     output logic [RADIX-1:0] s_out,
45     output logic [RADIX :0] c_out
46 );
47
48
49 //-----
50 // ecc_pe
51 //-----
52 logic [(2*RADIX)-1 : 0] mult0_out;
53 logic [(2*RADIX)-1 : 0] mult1_out;
54
55 logic [RADIX : 0] c_mux;
56 logic [RADIX-1 : 0] s_mux;
57
58 logic [2*RADIX : 0] res;
59 logic [RADIX-1 : 0] res_LSW;
60 logic [RADIX : 0] res_MSW;
61
62 logic          carry_garbage_bit;
63
64 //-----
65 // Module instantiations.
66 //-----
67 ecc_mult_dsp #(
68     .RADIX(RADIX)
69 ) MULT1 (
70     .A_i(a_in),
71     .B_i(b_in),
72     .P_o(mult0_out)
73 );
74
75 ecc_mult_dsp #(
76     .RADIX(RADIX)
77 ) MULT2 (
78     .A_i(p_in),
79     .B_i(m_in),
80     .P_o(mult1_out)
81 );
82
83 //-----
84 // pe_logic
85 //
86 // The logic needed to update the multiplier inputs.
87 //-----
88 assign c_mux = odd ? c_out : c_in;
89 assign s_mux = odd ? s_in : s_out;
90
91 assign res_MSW = res[2*RADIX : RADIX];
92 assign res_LSW = res[RADIX-1 : 0];
93
94 always_comb {carry_garbage_bit, res} = mult0_out + mul-
t1_out + c_mux + s_mux;
95
96 //-----
97 // register updates
98 //
99 //-----

```

```

-----
100 always_ff @(posedge clk or negedge reset_n) begin
101     if (~reset_n) begin
102         c_out <= 'b0;
103     end
104     else if (zeroize) begin
105         c_out <= 'b0;
106     end
107     else if (start_in) begin
108         c_out <= 'b0;
109     end
110     else begin
111         c_out <= res_MSW;
112     end
113 end
114 always_ff @(posedge clk or negedge reset_n) begin
115     if (~reset_n) begin
116         s_out <= 'b0;
117     end
118     else if (zeroize) begin
119         s_out <= 'b0;
120     end
121     else if (start_in) begin
122         s_out <= 'b0;
123     end
124     else begin
125         s_out <= res_LSW;
126     end
127 end
128 endmodule

```

15. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pe_first.v (204 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
cense");
4 // you may not use this file except in compliance with the Li-
cense.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
software
10 // distributed under the License is distributed on an "AS IS"
BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
12 // See the License for the specific language governing permis-
sions and
13 // limitations under the License.
14 //
15 //
=====
16 //
17 // ecc_pe_first.v
18 // -----
19 // first processing element for Montgomery Multiplier based
on
20 // Optimized Algorithms and Architectures for Montgomery Mul-
tiplication
21 // for Post-quantum Cryptography by Rami Elkhatib et. al.
22 //
23 //
=====
24
25 module ecc_pe_first #(
26     parameter RADIX = 32
27 )
28 (
29     // Clock and reset.
30     input wire clk,
31     input wire reset_n,
32     input wire zeroize,
33
34     input wire start_in,
35     // DATA PORT
36     input wire [RADIX-1:0] a_in,
37     input wire [RADIX-1:0] b_in,
38     input wire [RADIX-1:0] p_in,
39     input wire [RADIX-1:0] s_in,
40     input wire [RADIX-1:0] n_prime_in,
41     input wire odd,
42
43     output logic [RADIX-1:0] a_out,
44     output logic [RADIX-1:0] m_out,
45     output logic [RADIX :0] c_out
46 );
47
48
49 //-----
50 // ecc_pe_first
51 //-----
52 logic [RADIX-1:0] s_reg;

```

```

53 logic [RADIX-1:0] s_val;
54 logic [RADIX-1:0] a_reg;
55 logic [RADIX-1:0] a_val;
56
57 logic [(2*RADIX)-1 : 0] mult_out_0;
58 logic [RADIX-1 : 0] mult_out_0_MSW;
59 logic [RADIX-1 : 0] mult_out_0_LSW;
60
61 logic [RADIX : 0] res_0;
62 logic [RADIX-1 : 0] sum_0;
63 logic carry_0;
64 logic [RADIX-1 : 0] sum_0_reg;
65
66 logic [RADIX : 0] res_1;
67 logic carry_1;
68
69 logic [RADIX : 0] c_0;
70 logic [RADIX : 0] c_0_reg;
71 logic [RADIX : 0] c_1;
72
73 logic [(2*RADIX)-1 : 0] mult_out_1;
74 logic [RADIX-1 : 0] m_temp;
75 logic [RADIX-1 : 0] m_temp_reg;
76
77 logic [(2*RADIX)-1 : 0] mult_out_2;
78 logic [RADIX-1 : 0] mult_out_2_MSW;
79 logic [RADIX-1 : 0] mult_out_2_LSW;
80
81 logic carry_garbage_bit;
82
83 //-----

```

```

84 // Module instantiations.
85 //-----

```

```

86 ecc_mult_dsp #(
87     .RADIX(RADIX)
88 ) MULT0 (
89     .A_i(a_val),
90     .B_i(b_in),
91     .P_o(mult_out_0)
92 );
93
94 ecc_mult_dsp #(
95     .RADIX(RADIX)
96 ) MULT1 (
97     .A_i(n_prime_in),
98     .B_i(sum_0),
99     .P_o(mult_out_1)
100 );
101
102 ecc_mult_dsp #(
103     .RADIX(RADIX)
104 ) MULT2 (
105     .A_i(m_temp_reg),
106     .B_i(p_in),
107     .P_o(mult_out_2)
108 );
109
110 //-----
111 // pe_logic
112 //
113 // The logic needed to update the multiplier inputs.
114 //-----

```

```

115 assign m_temp = mult_out_1[RADIX-1:0];
116
117 assign mult_out_0_MSW = mult_out_0[(2*RADIX)-1 : RADIX];
118 assign mult_out_0_LSW = mult_out_0[RADIX-1 : 0];
119 assign mult_out_2_MSW = mult_out_2[(2*RADIX)-1 : RADIX];
120 assign mult_out_2_LSW = mult_out_2[RADIX-1 : 0];
121
122 assign s_val = odd ? s_in : s_reg;
123 assign a_val = odd ? a_in : a_reg;
124
125 always_comb begin
126     res_0 = s_val + mult_out_0_LSW;
127     sum_0 = res_0[RADIX-1 : 0];
128     carry_0 = res_0[RADIX];
129 end
130
131 always_comb begin
132     res_1 = sum_0_reg + mult_out_2_LSW;
133     carry_1 = res_1[RADIX];
134 end
135
136 always_comb c_0 = mult_out_0_MSW + carry_0;
137
138 always_comb c_1 = mult_out_2_MSW + carry_1;
139
140 assign {carry_garbage_bit, c_out} = c_0_reg + c_1;
141
142 //-----

```

```

143 // register updates
144 //
145 //-----

```

```

146 always_ff @(posedge clk or negedge reset_n) begin
147     if(~reset_n) begin

```

```

148     c_0_reg <= 'b0;
149     sum_0_reg <= 'b0;
150     m_temp_reg <= 'b0;
151     s_reg <= 'b0;
152     a_reg <= 'b0;
153 end
154 else if (zeroize) begin
155     c_0_reg <= 'b0;
156     sum_0_reg <= 'b0;
157     m_temp_reg <= 'b0;
158     s_reg <= 'b0;
159     a_reg <= 'b0;
160 end
161 else if (start_in) begin
162     c_0_reg <= 'b0;
163     sum_0_reg <= 'b0;
164     m_temp_reg <= 'b0;
165     s_reg <= 'b0;
166     a_reg <= 'b0;
167 end else begin
168     s_reg <= s_in;
169     a_reg <= a_in;
170     m_temp_reg <= m_temp;
171     c_0_reg <= c_0;
172     sum_0_reg <= sum_0;
173 end
174 end
175 always_ff @(posedge clk or negedge reset_n) begin
176     if (~reset_n) begin
177         a_out <= 'b0;
178     end
179     else if (zeroize) begin
180         a_out <= 'b0;
181     end
182     else if (start_in) begin
183         a_out <= 'b0;
184     end else begin
185         a_out <= a_val;
186     end
187 end
188 end
189 always_ff @(posedge clk or negedge reset_n) begin
190     if (~reset_n) begin
191         m_out <= 'b0;
192     end
193     else if (zeroize) begin
194         m_out <= 'b0;
195     end
196     else if (start_in) begin
197         m_out <= 'b0;
198     end else begin
199         m_out <= m_temp;
200     end
201 end
202 end
203
204 endmodule

```

16. chipsalliance_caliptra-rtl/src/ecc/rtl/ecc_pm_ctrl.sv (404 lines)

```

1 // SPDX-License-Identifier: Apache-2.0
2 //
3 // Licensed under the Apache License, Version 2.0 (the "Li-
4 // you may not use this file except in compliance with the Li-
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing,
10 // distributed under the License is distributed on an "AS IS"
11 // BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
13 // press or implied.
14 // See the License for the specific language governing permis-
15 // sions and
16 // limitations under the License.
17 //
18 //
19 //
20 //
21 // The architecture includes ecc_pm_sequencer including the
22 // required
23 // sequence to perform:
24 // - point multiplication (pm) based on Montgomery ladder
25 // and Fermat's little
26 // theorem (FLT) inversion
27 // - Montgomery ladder initialization with base point G or
28 // public key PK in
29 // keygen, signing, or verifying operations.
30 // - Montgomery ladder steps: including the iterative point
31 // addition (PA)

```

```

28 // and point doubling (PD) operations in Montgomery domain
29 // using projective
30 // coordinates. The number of iteration is equal to REG_S-
31 // IZE of scalar in
32 //
33 // unprotected point multiplication, and REG_SIZE+RND_SIZE of
34 // scalar in the
35 // SCA protected operation.
36 // - Modular Inversion (mod p, mod q) is based on Fermat's
37 // little theorem (FLT)
38 // inversion.
39 // - Conversion from projective coordinates in Montgomery
40 // domain (X,Y,Z) to
41 // affine coordinates in the normal domain (x,y).
42 // - Modular arithmetic operations such as (h + r*privKey)
43 // in signing
44 // or (h*s_inv) and (r*s_inv) in verifying
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //

```



```

109     always_ff @(posedge clk or negedge reset_n)
110     begin : ff_stalled_reg
111         if(!reset_n)
112             stalled_pipe1 <= '0;
113         else if (zeroize)
114             stalled_pipe1 <= '0;
115         else
116             stalled_pipe1 <= stalled;
117     end // ff_stalled_reg
118
119     //-----
120
121     // Point multiplication FSM flow
122     //
123     // This FSM starts with the keygen/signing/verifying com-
124     // mand to
125     // perform different operations.
126     // Active low and async reset.
127     //-----
128
129     always_ff @(posedge clk or negedge reset_n)
130     begin : pm_fsm
131         if(!reset_n) begin
132             prog_cntr <= '0;
133             mont_cntr <= '0;
134             stall_cntr <= '0;
135             stalled <= '0;
136             req_digit_o <= '0;
137             mont_ladder <= '0;
138             ecc_cmd_reg <= '0;
139         end
140         else if (zeroize) begin
141             prog_cntr <= '0;
142             mont_cntr <= '0;
143             stall_cntr <= '0;
144             stalled <= '0;
145             req_digit_o <= '0;
146             mont_ladder <= '0;
147             ecc_cmd_reg <= '0;
148         end
149         else begin
150             if (stalled & (stall_cntr > 0)) begin
151                 stall_cntr <= stall_cntr - 1;
152             end
153             else if (stall_flag & (!stalled) & (!
154                 stalled_pipe1)) begin
155                 unique case (prog_instr.opcode)
156                     UOP_DO_ADD_p : begin stalled <= '1'b1;
157                 stall_cntr <= ADD_DELAY; end // ADD
158                     UOP_DO_SUB_p : begin stalled <= '1'b1;
159                 stall_cntr <= ADD_DELAY; end // SUB
160                     UOP_DO_MUL_p : begin stalled <= '1'b1;
161                 stall_cntr <= MULT_DELAY; end // MULT
162                     UOP_DO_ADD_q : begin stalled <= '1'b1;
163                 stall_cntr <= ADD_DELAY; end // ADD
164                     UOP_DO_SUB_q : begin stalled <= '1'b1;
165                 stall_cntr <= ADD_DELAY; end // SUB
166                     UOP_DO_MUL_q : begin stalled <= '1'b1;
167                 stall_cntr <= MULT_DELAY; end // MULT
168                     default : begin stalled <= '1'b0;
169                 stall_cntr <= '0; end
170             endcase
171             else begin
172                 stalled <= 0;
173                 unique case (prog_cntr)
174                     NOP : begin // Waiting for new valid
175                         command
176                             ecc_cmd_reg <= ecc_cmd_i;
177                             unique case (ecc_cmd_i)
178                                 KEYGEN_CMD : begin // keygen
179                                     mont_cntr <=
180                                     Secp384_SCA_MONT_COUNT;
181                                     prog_cntr <= PM_INIT_G_S;
182                                     end
183                                     SIGN_CMD : begin // signing
184                                     mont_cntr <=
185                                     Secp384_SCA_MONT_COUNT;
186                                     prog_cntr <= PM_INIT_G_S;
187                                     end
188                                     VER_PART0_CMD : begin // verify-
189                                     ing part0
190                                     prog_cntr <= VER0_P0_S;
191                                     end
192                                     VER_PART1_CMD : begin // verify-
193                                     ing part1
194                                     mont_cntr <=
195                                     Secp384_MONT_COUNT;
196                                     prog_cntr <= PM_INIT_G_S;
197                                     end
198                                     VER_PART2_CMD : begin // verify-
199                                     ing part2
200                                     prog_cntr <= VER1_ST_S;
201                                     end
202                                     end
203                                     end
204                                     end
205                                     end
206                                     end
207                                     end
208                                     end
209                                     end
210                                     end
211                                     end
212                                     end
213                                     end
214                                     end
215                                     end
216                                     end
217                                     end
218                                     end
219                                     end
220                                     end
221                                     end
222                                     end
223                                     end
224                                     end
225                                     end
226                                     end
227                                     end
228                                     end
229                                     end
230                                     end
231                                     end
232                                     end
233                                     end
234                                     end
235                                     end
236                                     end
237                                     end
238                                     end
239                                     end
240                                     end
241                                     end
242                                     end
243                                     end
244                                     end
245                                     end
246                                     end
247                                     end
248                                     end
249                                     end
250                                     end
251                                     end
252                                     end
253                                     end
254                                     end
255                                     end
256                                     end
257                                     end
258                                     end
259                                     end
260                                     end
261                                     end
262                                     end
263                                     end
264                                     end
265                                     end
266                                     end
267                                     end
268                                     end
269                                     end
270                                     end
271                                     end
272                                     end
273                                     end
274                                     end
275                                     end
276                                     end
277                                     end
278                                     end
279                                     end
280                                     end
281                                     end
282                                     end
283                                     end
284                                     end
285                                     end
286                                     end
287                                     end
288                                     end
289                                     end
290                                     end
291                                     end
292                                     end
293                                     end
294                                     end
295                                     end
296                                     end
297                                     end
298                                     end
299                                     end
300                                     end
301                                     end
302                                     end
303                                     end
304                                     end
305                                     end
306                                     end
307                                     end
308                                     end
309                                     end
310                                     end
311                                     end
312                                     end
313                                     end
314                                     end
315                                     end
316                                     end
317                                     end
318                                     end
319                                     end
320                                     end
321                                     end
322                                     end
323                                     end
324                                     end
325                                     end
326                                     end
327                                     end
328                                     end
329                                     end
330                                     end
331                                     end
332                                     end
333                                     end
334                                     end
335                                     end
336                                     end
337                                     end
338                                     end
339                                     end
340                                     end
341                                     end
342                                     end
343                                     end
344                                     end
345                                     end
346                                     end
347                                     end
348                                     end
349                                     end
350                                     end
351                                     end
352                                     end
353                                     end
354                                     end
355                                     end
356                                     end
357                                     end
358                                     end
359                                     end
360                                     end
361                                     end
362                                     end
363                                     end
364                                     end
365                                     end
366                                     end
367                                     end
368                                     end
369                                     end
370                                     end
371                                     end
372                                     end
373                                     end
374                                     end
375                                     end
376                                     end
377                                     end
378                                     end
379                                     end
380                                     end
381                                     end
382                                     end
383                                     end
384                                     end
385                                     end
386                                     end
387                                     end
388                                     end
389                                     end
390                                     end
391                                     end
392                                     end
393                                     end
394                                     end
395                                     end
396                                     end
397                                     end
398                                     end
399                                     end
400                                     end
401                                     end
402                                     end
403                                     end
404                                     end
405                                     end
406                                     end
407                                     end
408                                     end
409                                     end
410                                     end
411                                     end
412                                     end
413                                     end
414                                     end
415                                     end
416                                     end
417                                     end
418                                     end
419                                     end
420                                     end
421                                     end
422                                     end
423                                     end
424                                     end
425                                     end
426                                     end
427                                     end
428                                     end
429                                     end
430                                     end
431                                     end
432                                     end
433                                     end
434                                     end
435                                     end
436                                     end
437                                     end
438                                     end
439                                     end
440                                     end
441                                     end
442                                     end
443                                     end
444                                     end
445                                     end
446                                     end
447                                     end
448                                     end
449                                     end
450                                     end
451                                     end
452                                     end
453                                     end
454                                     end
455                                     end
456                                     end
457                                     end
458                                     end
459                                     end
460                                     end
461                                     end
462                                     end
463                                     end
464                                     end
465                                     end
466                                     end
467                                     end
468                                     end
469                                     end
470                                     end
471                                     end
472                                     end
473                                     end
474                                     end
475                                     end
476                                     end
477                                     end
478                                     end
479                                     end
480                                     end
481                                     end
482                                     end
483                                     end
484                                     end
485                                     end
486                                     end
487                                     end
488                                     end
489                                     end
490                                     end
491                                     end
492                                     end
493                                     end
494                                     end
495                                     end
496                                     end
497                                     end
498                                     end
499                                     end
500                                     end
501                                     end
502                                     end
503                                     end
504                                     end
505                                     end
506                                     end
507                                     end
508                                     end
509                                     end
510                                     end
511                                     end
512                                     end
513                                     end
514                                     end
515                                     end
516                                     end
517                                     end
518                                     end
519                                     end
520                                     end
521                                     end
522                                     end
523                                     end
524                                     end
525                                     end
526                                     end
527                                     end
528                                     end
529                                     end
530                                     end
531                                     end
532                                     end
533                                     end
534                                     end
535                                     end
536                                     end
537                                     end
538                                     end
539                                     end
540                                     end
541                                     end
542                                     end
543                                     end
544                                     end
545                                     end
546                                     end
547                                     end
548                                     end
549                                     end
550                                     end
551                                     end
552                                     end
553                                     end
554                                     end
555                                     end
556                                     end
557                                     end
558                                     end
559                                     end
560                                     end
561                                     end
562                                     end
563                                     end
564                                     end
565                                     end
566                                     end
567                                     end
568                                     end
569                                     end
570                                     end
571                                     end
572                                     end
573                                     end
574                                     end
575                                     end
576                                     end
577                                     end
578                                     end
579                                     end
580                                     end
581                                     end
582                                     end
583                                     end
584                                     end
585                                     end
586                                     end
587                                     end
588                                     end
589                                     end
590                                     end
591                                     end
592                                     end
593                                     end
594                                     end
595                                     end
596                                     end
597                                     end
598                                     end
599                                     end
600                                     end
601                                     end
602                                     end
603                                     end
604                                     end
605                                     end
606                                     end
607                                     end
608                                     end
609                                     end
610                                     end
611                                     end
612                                     end
613                                     end
614                                     end
615                                     end
616                                     end
617                                     end
618                                     end
619                                     end
620                                     end
621                                     end
622                                     end
623                                     end
624                                     end
625                                     end
626                                     end
627                                     end
628                                     end
629                                     end
630                                     end
631                                     end
632                                     end
633                                     end
634                                     end
635                                     end
636                                     end
637                                     end
638                                     end
639                                     end
640                                     end
641                                     end
642                                     end
643                                     end
644                                     end
645                                     end
646                                     end
647                                     end
648                                     end
649                                     end
650                                     end
651                                     end
652                                     end
653                                     end
654                                     end
655                                     end
656                                     end
657                                     end
658                                     end
659                                     end
660                                     end
661                                     end
662                                     end
663                                     end
664                                     end
665                                     end
666                                     end
667                                     end
668                                     end
669                                     end
670                                     end
671                                     end
672                                     end
673                                     end
674                                     end
675                                     end
676                                     end
677                                     end
678                                     end
679                                     end
680                                     end
681                                     end
682                                     end
683                                     end
684                                     end
685                                     end
686                                     end
687                                     end
688                                     end
689                                     end
690                                     end
691                                     end
692                                     end
693                                     end
694                                     end
695                                     end
696                                     end
697                                     end
698                                     end
699                                     end
700                                     end
701                                     end
702                                     end
703                                     end
704                                     end
705                                     end
706                                     end
707                                     end
708                                     end
709                                     end
710                                     end
711                                     end
712                                     end
713                                     end
714                                     end
715                                     end
716                                     end
717                                     end
718                                     end
719                                     end
720                                     end
721                                     end
722                                     end
723                                     end
724                                     end
725                                     end
726                                     end
727                                     end
728                                     end
729                                     end
730                                     end
731                                     end
732                                     end
733                                     end
734                                     end
735                                     end
736                                     end
737                                     end
738                                     end
739                                     end
740                                     end
741                                     end
742                                     end
743                                     end
744                                     end
745                                     end
746                                     end
747                                     end
748                                     end
749                                     end
750                                     end
751                                     end
752                                     end
753                                     end
754                                     end
755                                     end
756                                     end
757                                     end
758                                     end
759                                     end
760                                     end
761                                     end
762                                     end
763                                     end
764                                     end
765                                     end
766                                     end
767                                     end
768                                     end
769                                     end
770                                     end
771                                     end
772                                     end
773                                     end
774                                     end
775                                     end
776                                     end
777                                     end
778                                     end
779                                     end
780                                     end
781                                     end
782                                     end
783                                     end
784                                     end
785                                     end
786                                     end
787                                     end
788                                     end
789                                     end
790                                     end
791                                     end
792                                     end
793                                     end
794                                     end
795                                     end
796                                     end
797                                     end
798                                     end
799                                     end
800                                     end
801                                     end
802                                     end
803                                     end
804                                     end
805                                     end
806                                     end
807                                     end
808                                     end
809                                     end
810                                     end
811                                     end
812                                     end
813                                     end
814                                     end
815                                     end
816                                     end
817                                     end
818                                     end
819                                     end
820                                     end
821                                     end
822                                     end
823                                     end
824                                     end
825                                     end
826                                     end
827                                     end
828                                     end
829                                     end
830                                     end
831                                     end
832                                     end
833                                     end
834                                     end
835                                     end
836                                     end
837                                     end
838                                     end
839                                     end
840                                     end
841                                     end
842                                     end
843                                     end
844                                     end
845                                     end
846                                     end
847                                     end
848                                     end
849                                     end
850                                     end
851                                     end
852                                     end
853                                     end
854                                     end
855                                     end
856                                     end
857                                     end
858                                     end
859                                     end
860                                     end
861                                     end
862                                     end
863                                     end
864                                     end
865                                     end
866                                     end
867                                     end
868                                     end
869                                     end
870                                     end
871                                     end
872                                     end
873                                     end
874                                     end
875                                     end
876                                     end
877                                     end
878                                     end
879                                     end
880                                     end
881                                     end
882                                     end
883                                     end
884                                     end
885                                     end
886                                     end
887                                     end
888                                     end
889                                     end
890                                     end
891                                     end
892                                     end
893                                     end
894                                     end
895                                     end
896                                     end
897                                     end
898                                     end
899                                     end
900                                     end
901                                     end
902                                     end
903                                     end
904                                     end
905                                     end
906                                     end
907                                     end
908                                     end
909                                     end
910                                     end
911                                     end
912                                     end
913                                     end
914                                     end
915                                     end
916                                     end
917                                     end
918                                     end
919                                     end
920                                     end
921                                     end
922                                     end
923                                     end
924                                     end
925                                     end
926                                     end
927                                     end
928                                     end
929                                     end
930                                     end
931                                     end
932                                     end
933                                     end
934                                     end
935                                     end
936                                     end
937                                     end
938                                     end
939                                     end
940                                     end
941                                     end
942                                     end
943                                     end
944                                     end
945                                     end
946                                     end
947                                     end
948                                     end
949                                     end
950                                     end
951                                     end
952                                     end
953                                     end
954                                     end
955                                     end
956                                     end
957                                     end
958                                     end
959                                     end
960                                     end
961                                     end
962                                     end
963                                     end
964                                     end
965                                     end
966                                     end
967                                     end
968                                     end
969                                     end
970                                     end
971                                     end
972                                     end
973                                     end
974                                     end
975                                     end
976                                     end
977                                     end
978                                     end
979                                     end
980                                     end
981                                     end
982                                     end
983                                     end
984                                     end
985                                     end
986                                     end
987                                     end
988                                     end
989                                     end
990                                     end
991                                     end
992                                     end
993                                     end
994                                     end
995                                     end
996                                     end
997                                     end
998                                     end
999                                     end
1000                                    end

```

```

191     CHK_PK_CMD : begin
192         prog_cntr <= CHK_PK_S;
193     end
194
195     DH_SHARED_CMD : begin // DH
196         mont_cntr <=
197         Secp384_SCA_MONT_COUNT;
198         prog_cntr <= PM_INIT_DH_S;
199     end
200
201     default :
202         prog_cntr <= NOP;
203     endcase
204     req_digit_o <= 0;
205     end
206
207     VER1_ST_E : begin // Storing the VER1 MONT
208     results before starting VER2
209         mont_cntr <= Secp384_MONT_COUNT;
210         prog_cntr <= PM_INIT_PK_S;
211     end
212
213     // Montgoemry Ladder
214     PM_INIT_6_E : begin //End of initilize R0
215         prog_cntr <= PM_INIT_S;
216     end
217
218     PM_INIT_PK_E : begin //End of initilize R0 =
219     PK
220         prog_cntr <= PM_INIT_S;
221     end
222
223     PM_INIT_DH_E : begin //End of initilize R0 =
224     Randomized PK
225         prog_cntr <= PM_INIT_S;
226     end
227
228     PM_INIT_E : begin // End of initilaze R1 = 0
229         mont_cntr <= mont_cntr - 1;
230         req_digit_o <= 1;
231         prog_cntr <= PA_S;
232         mont_ladder <= 1;
233     end
234
235     PA_S : begin //Start of point addition
236         req_digit_o <= 0;
237         prog_cntr <= prog_cntr + 1;
238     end
239
240     PA_E : begin //End of point addition
241         prog_cntr <= PD_S;
242     end
243
244     PD_E : begin //End of point doubling
245         if (mont_cntr == 0) begin // Mont-
246         gomery ladder is done
247             unique case (ecc_cmd_reg)
248                 VER_PART1_CMD : prog_cntr <=
249                 NOP;
250                 VER_PART2_CMD : prog_cntr <=
251                 VER2_PA_S;
252                 default : prog_cntr <=
253                 INV_S;
254             endcase
255             mont_ladder <= 0;
256             req_digit_o <= 0;
257         end
258         else begin
259             mont_cntr <= mont_cntr - 1;
260             req_digit_o <= 1;
261             prog_cntr <= PA_S;
262         end
263     end
264
265     INV_E : begin // End of inversion mod p
266         prog_cntr <= CONV_S;
267     end
268
269     CONV_E : begin // End of conversion from
270     projective Mont (X,Y,Z) to affine normanl (x,y)
271         unique case (ecc_cmd_reg)
272             SIGN_CMD : prog_cntr <= SIGN0_S;
273             default : prog_cntr <= NOP;
274         endcase
275     end
276
277     SIGN0_E : begin // End of signing part 0
278     to convert inputs to Mont domain & compute (h + r*privKey)
279         prog_cntr <= INVq_S;
280     end
281
282     INVq_E : begin // End of inversion mod q
283         unique case (ecc_cmd_reg)
284             SIGN_CMD : prog_cntr <=
285             SIGN1_S;
286         end
287     end

```



```

275          VER_PART0_CMD : prog_cntr <=
VER0_P1_S;
276          default      : prog_cntr <= NOP;
277          endcase
278        end
279      end
280      SIGN1_E : begin // End of signing part 1
to compute s
281          prog_cntr <= NOP;
282        end
283      end
284      CHK_PK_E: begin // End of public key val-
idation before verifying process
285          prog_cntr <= NOP;
286        end
287      end
288      VER0_P0_E : begin // End of verifying part
0 to convert inputs to Mont domain
289          prog_cntr <= INVq_S;
290        end
291      end
292      VER0_P1_E : begin // End of verifying part
1 to compute (h*s_inv) and (r*s_inv)
293          prog_cntr <= NOP;
294        end
295      end

296      VER2_PA_E : begin // End of point addition
of PA((h*s_inv)*G, (r*s_inv)*PK)
297          prog_cntr <= INV_S;
298        end
299      end
300      PM_INIT_G_S,
301      PM_INIT_PK_S,
302      PM_INIT_S,
303      PD_S,
304      INV_S,
305      INVq_S,
306      CONV_S,
307      SIGN0_S,
308      SIGN1_S,
309      VER0_P0_S,
310      VER0_P1_S,
311      VER1_ST_S,
312      VER2_PA_S,
313      PM_INIT_DH_S: begin
314          prog_cntr <= prog_cntr + 1;
315        end
316      end
317      default : begin
318          prog_cntr <= prog_cntr + 1;
319        end
320      endcase
321    end
322  end
323  end // pm_fsm
324
325  always_ff @(posedge clk or negedge reset_n)
326  begin : sequencer_pipeline1
327      if (!reset_n)
328          prog_instr_pipe1 <= '{UOP_NOP, '0, '0};
329      else if (zeroize)
330          prog_instr_pipe1 <= '{UOP_NOP, '0, '0};
331      else begin
332          if (stalled_pipe1)
333              prog_instr_pipe1 <= prog_instr_pipe1;
334          else
335              prog_instr_pipe1 <= prog_instr;
336        end
337      end // sequencer_pipeline1
338
339  always_ff @(posedge clk or negedge reset_n)
340  begin : sequencer_pipeline2
341      if (!reset_n)
342          prog_instr_pipe2 <= '{UOP_NOP, '0, '0};

```

```

343      else if (zeroize)
344          prog_instr_pipe2 <= '{UOP_NOP, '0, '0};
345      else begin
346          if (stalled_pipe1)
347              prog_instr_pipe2 <= prog_instr_pipe2;
348          else
349              prog_instr_pipe2 <= prog_instr_pipe1;
350        end
351      end // sequencer_pipeline2
352
353  /*
354  always_comb
355  begin : delay_flag
356      unique case (prog_instr.opcode)
357          UOP_DO_ADD_p : stall_flag = 1;
358          UOP_DO_SUB_p : stall_flag = 1;
359          UOP_DO_MUL_p : stall_flag = 1;
360          UOP_DO_ADD_q : stall_flag = 1;
361          UOP_DO_SUB_q : stall_flag = 1;
362          UOP_DO_MUL_q : stall_flag = 1;
363          default      : stall_flag = 0;
364        endcase
365      end // delay_flag
366  */
367
368  always_comb stall_flag = (prog_instr.opcode.add_en ||
prog_instr.opcode.mult_en);
369
370  //Instruction out to arithmetic units and memory
371  always_ff @(posedge clk or negedge reset_n)
372  begin : instruction_out
373      if (!reset_n)
374          instr_o <= '{UOP_NOP, '0, '0};
375      else if (zeroize)
376          instr_o <= '{UOP_NOP, '0, '0};
377      else begin
378          instr_o.opcode <= prog_instr_pipe2.opcode;
379
380          if (mont_ladder) begin
381              // constant-time conditional swaps depending
on the value of the currently processed bit of secret key
382
383              //Addr A for ADD/SUB result
384              if (prog_instr_pipe2.opa_addr[3 +: OPR_AD-
DR_WIDTH-3] == 3'b001)
385                  instr_o.opa_addr <= {prog_in-
str_pipe2.opa_addr[3 +: OPR_ADDR_WIDTH-3], digit_i ^ prog_in-
str_pipe2.opa_addr[2], prog_instr_pipe2.opa_addr[1 : 0]};
386              else
387                  instr_o.opa_addr <= prog_in-
str_pipe2.opa_addr;
388
389              //Addr B for MULT result
390              if (prog_instr_pipe2.opb_addr[3 +: OPR_AD-
DR_WIDTH-3] == 3'b001)
391                  instr_o.opb_addr <= {prog_in-
str_pipe2.opb_addr[3 +: OPR_ADDR_WIDTH-3], digit_i ^ prog_in-
str_pipe2.opb_addr[2], prog_instr_pipe2.opb_addr[1 : 0]};
392              else
393                  instr_o.opb_addr <= prog_in-
str_pipe2.opb_addr;
394          end
395          else begin
396              instr_o.opa_addr <= prog_in-
str_pipe2.opa_addr; //Addr A for ADD/SUB result
397              instr_o.opb_addr <= prog_in-
str_pipe2.opb_addr; //Addr B for MULT result
398          end
399        end
400      end // instruction_out
401
402      assign busy_o = ~(prog_cntr == NOP);
403  end
404  endmodule

```

Caliptra AES

7 files • 5,137 lines

API — aes.sv

Table 835. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — module / interface / package (1개)

이름	종류
aes	module

Table 836. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — parameter (13개)

파라미터	형/폭	기본값
AES192Enable	bit	1
AESGCMEnable	bit	1
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecStartTriggerDelay	int unsigned	0
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
AlertAsyncOn	logic [NumAlerts-1:0]	{NumAlerts{1'b1}}
RndCnstClearingLfsrSeed	clearing_lfsr_seed_t	RndCnstClearingLfsrSeedDefault
RndCnstClearingLfsrPerm	clearing_lfsr_perm_t	RndCnstClearingLfsrPermDefault
RndCnstClearingSharePerm	clearing_lfsr_perm_t	RndCnstClearingSharePermDefault
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 837. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — localparam (2개)

상수	형/폭	값
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH
CLP_AES_KV_CHUNK_SIZE		NumRegsData*32

Table 838. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv — 포트 (18개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
rst_shadowed_ni	input	logic
idle_o	output	caliptra_prim_mubi_pkg::mubi4_t
lc_escalate_en_i	input	lc_ctrl_pkg::lc_tx_t
clk_edn_i	input	logic

포트	방향	형/폭
rst_edn_ni	input	logic
edn_o	output	edn_pkg::edn_req_t
edn_i	input	edn_pkg::edn_rsp_t
input_ready_o	output	logic
output_valid_o	output	logic
caliptra2aes	input	caliptra2aes_t
aes2caliptra	output	aes2caliptra_t
keymgr_key_i	input	keymgr_pkg::hw_key_req_t
tl_i	input	caliptra_tlul_pkg::tl_h2d_t
tl_o	output	caliptra_tlul_pkg::tl_d2h_t
alert_rx_i	input	caliptra_prim_alert_pkg::alert_rx_t [NumAlerts-1:0...
alert_tx_o	output	caliptra_prim_alert_pkg::alert_tx_t [NumAlerts-1:0...

API — aes_cipher_control.sv

Table 839. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — module / interface / package (1개)

이름	종류
aes_cipher_control	module

Table 840. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — parameter (3개)

파라미터	형/폭	기본값
CiphOpFwdOnly	bit	0
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 841. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — localparam (2개)

상수	형/폭	값
NumSp2VSig	int unsigned	8
Sp2VEnSecBuf	bit [NumSp2VSig-1:0]	8'b1100_0000

Table 842. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv — 포트 (45개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e

포트	방향	형/폭
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic
key_clear_i	input	logic
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	sp2v_e
sub_bytes_en_o	output	sp2v_e
sub_bytes_out_req_i	input	sp2v_e
sub_bytes_out_ack_o	output	sp2v_e
add_rk_sel_o	output	add_rk_sel_e
key_expand_op_o	output	ciph_op_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	sp2v_e
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	sp2v_e
key_expand_en_o	output	sp2v_e
key_expand_out_req_i	input	sp2v_e

포트	방향	형/폭
key_expand_out_ack_o	output	sp2v_e
key_expand_clear_o	output	logic
key_expand_round_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e

API — aes_cipher_control_fsm.sv

Table 843. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm.sv — module / interface / package (1개)

이름	종류
aes_cipher_control_fsm	module

Table 844. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 845. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic

포트	방향	형/폭
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_n.sv**Table 846. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — module / interface / package (1개)**

이름	종류
aes_cipher_control_fsm_n	module

Table 847. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 848. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 849. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_n.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_ni	input	logic
in_ready_no	output	logic
out_valid_no	output	logic
out_ready_ni	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_ni	input	logic
dec_key_gen_ni	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic

포트	방향	형/폭
prng_update_o	output	logic
prng_reseed_req_o	output	logic
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_no	output	logic
sub_bytes_en_no	output	logic
sub_bytes_out_req_ni	input	logic
sub_bytes_out_ack_no	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_no	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_no	output	logic
key_expand_en_no	output	logic
key_expand_out_req_ni	input	logic
key_expand_out_ack_no	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_ni	input	logic
crypt_d_no	output	logic
dec_key_gen_q_ni	input	logic
dec_key_gen_d_no	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_control_fsm_p.sv

Table 850. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — module / interface / package (17¶)

이름	종류
aes_cipher_control_fsm_p	module

Table 851. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — parameter (2개)

파라미터	형/폭	기본값
SecMasking	bit	0
SecSBoxImpl	sbox_impl_e	SBoxImplDom

Table 852. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — localparam (2개)

상수	형/폭	값
NumInBufBits	int	\$bits({
NumOutBufBits	int	\$bits({

Table 853. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv — 포트 (50개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	logic
in_ready_o	output	logic
out_valid_o	output	logic
out_ready_i	input	logic
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	logic
dec_key_gen_i	input	logic
prng_reseed_i	input	logic
key_clear_i	input	logic
data_out_clear_i	input	logic
mux_sel_err_i	input	logic
sp_enc_err_i	input	logic
rnd_ctr_err_i	input	logic
op_err_i	input	logic
alert_fatal_i	input	logic
alert_o	output	logic
prng_update_o	output	logic
prng_reseed_req_o	output	logic

포트	방향	형/폭
prng_reseed_ack_i	input	logic
state_sel_o	output	state_sel_e
state_we_o	output	logic
sub_bytes_en_o	output	logic
sub_bytes_out_req_i	input	logic
sub_bytes_out_ack_o	output	logic
add_rk_sel_o	output	add_rk_sel_e
key_full_sel_o	output	key_full_sel_e
key_full_we_o	output	logic
key_dec_sel_o	output	key_dec_sel_e
key_dec_we_o	output	logic
key_expand_en_o	output	logic
key_expand_out_req_i	input	logic
key_expand_out_ack_o	output	logic
key_expand_clear_o	output	logic
rnd_ctr_o	output	logic [3:0]
key_words_sel_o	output	key_words_sel_e
round_key_sel_o	output	round_key_sel_e
crypt_q_i	input	logic
crypt_d_o	output	logic
dec_key_gen_q_i	input	logic
dec_key_gen_d_o	output	logic
prng_reseed_q_i	input	logic
prng_reseed_d_o	output	logic
key_clear_q_i	input	logic
key_clear_d_o	output	logic
data_out_clear_q_i	input	logic
data_out_clear_d_o	output	logic

API — aes_cipher_core.sv

Table 854. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — module / interface / package (1개)

이름	종류
aes_cipher_core	module

Table 855. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — parameter (9개)

파라미터	형/폭	기본값
AES192Enable	bit	1
CiphOpFwdOnly	bit	0
SecMasking	bit	1
SecSBoxImpl	sbox_impl_e	SBoxImplDom
SecAllowForcingMasks	bit	0
SecSkipPRNGReseeding	bit	0
EntropyWidth	int unsigned	edn_pkg::ENDPOINT_BUS_WIDTH
RndCnstMaskingLfsrSeed	masking_lfsr_seed_t	RndCnstMaskingLfsrSeedDefault
RndCnstMaskingLfsrPerm	masking_lfsr_perm_t	RndCnstMaskingLfsrPermDefault

Table 856. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — localparam (5개)

상수	형/폭	값
NumShares	int	SecMasking ? 2 : 1
WidthPRDRow	int unsigned	4*WidthPRDSBox
NumSp2VSig	int unsigned	3
Sp2VEnSecBuf	bit [NumSp2VSig-1:0]	{NumSp2VSig{1'b1}}
NumCyclesPerRound	int unsigned	(SecSBoxImpl == SBoxImplDom) ? 5 : 1

Table 857. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
in_valid_i	input	sp2v_e
in_ready_o	output	sp2v_e
out_valid_o	output	sp2v_e
out_ready_i	input	sp2v_e
cfg_valid_i	input	logic
op_i	input	ciph_op_e
key_len_i	input	key_len_e
crypt_i	input	sp2v_e
crypt_o	output	sp2v_e
dec_key_gen_i	input	sp2v_e
dec_key_gen_o	output	sp2v_e
prng_reseed_i	input	logic
prng_reseed_o	output	logic
key_clear_i	input	logic

포트	방향	형/폭
key_clear_o	output	logic
data_out_clear_i	input	logic
data_out_clear_o	output	logic
alert_fatal_i	input	logic
alert_o	output	logic
force_masks_i	input	logic
data_in_mask_o	output	logic [3:0][3:0][7:0]
entropy_req_o	output	logic
entropy_ack_i	input	logic
entropy_i	input	logic [EntropyWidth-1:0]

API — aes_clp_reg.sv

Table 858. chipsalliance_caliptra-rtl/src/aes/rtl/aes_clp_reg.sv — module / interface / package (1개)

이름	종류
aes_clp_reg	module

Table 859. chipsalliance_caliptra-rtl/src/aes/rtl/aes_clp_reg.sv — 포트 (16개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_cpuif_req	input	wire
s_cpuif_req_is_wr	input	wire
s_cpuif_addr	input	wire [10:0]
s_cpuif_wr_data	input	wire [31:0]
s_cpuif_wr_bitlen	input	wire [31:0]
s_cpuif_req_stall_wr	output	wire
s_cpuif_req_stall_rd	output	wire
s_cpuif_rd_ack	output	wire
s_cpuif_rd_err	output	wire
s_cpuif_rd_data	output	wire [31:0]
s_cpuif_wr_ack	output	wire
s_cpuif_wr_err	output	wire
hwif_in	input	aes_clp_reg_pkg::aes_clp_reg__in_t
hwif_out	output	aes_clp_reg_pkg::aes_clp_reg__out_t

1. chipsalliance_caliptra-rtl/src/aes/rtl/aes.sv (486 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES top-level wrapper
6
7 `include "caliptra_prim_assert.sv"
8
9 module aes
10     import aes_pkg::*;
11     import aes_reg_pkg::*;
12     #(
13         parameter bit AES192Enable = 1, // Can be
0 (disable), or 1 (enable).
14         parameter bit AESGCMEEnable = 1, // Can be
0 (disable), or 1 (enable).
15         parameter bit SecMasking = 1, // Can be
0 (no masking), or
16         // 1
17         // core.
18         // masked
19         parameter sbbox_impl_e SecSBoxImpl = SBoxImpl-
20         Dom, // See aes_pkg.sv
21         parameter int unsigned SecStartTriggerDelay = 0, // Manual
22         start trigger delay, useful for
23         // SCA
24         measurements. A value of e.g. 40
25         // allows
26         the processor to go into sleep
27         // before
28         AES starts operation.
29         parameter bit SecAllowForcingMasks = 0, // Allow
30         forcing masks to constant values using
31         //
32         FORCE_MASKS bit in Auxiliary Control
33         // Re-
34         gister. Useful for SCA only.
35         parameter bit SecSkipPRNGReseeding = 0, // The
36         current SCA setup doesn't provide enough
37         // re-
38         sources to implement the infrastucture
39         // re-
40         quired for PRNG reseeding (CSRNG, EDN).
41         // To en-
42         able SCA resistance evaluations, we
43
44         // need to
45         skip reseeding requests.
46         // Useful
47         for SCA only.
48         parameter logic [NumAlerts-1:0] AlertAsyncOn = {Nu-
49         mAlerts{1'b1}},
50         parameter clearing_lfsr_seed_t RndCnstClearingLfsrSeed =
51         RndCnstClearingLfsrSeedDefault,
52         parameter clearing_lfsr_perm_t RndCnstClearingLfsrPerm =
53         RndCnstClearingLfsrPermDefault,
54         parameter clearing_lfsr_perm_t RndCnstClearingSharePerm =
55         RndCnstClearingSharePermDefault,
56         parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed =
57         RndCnstMaskingLfsrSeedDefault,
58         parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm =
59         RndCnstMaskingLfsrPermDefault
60     ) (
61         input logic clk_i,
62         input logic rst_ni,
63         input logic rst_shad-
64         owed_ni,
65         // Idle indicator for clock manager
66         output caliptra_prim_mubi_pkg::mubi4_t
67         idle_o,
68         // Life cycle
69         input lc_ctrl_pkg::lc_tx_t lc_escal-
70         ate_en_i,
71         // Entropy distribution network (EDN) interface
72         input logic clk_edn_i,
73         input logic rst_edn_ni,
74         output edn_pkg::edn_req_t edn_o,
75         input edn_pkg::edn_rsp_t edn_i,
76         // status signals
77         output logic input_ready_o,
78         output logic output_valid_o,
79
80         // Caliptra interface
81         input caliptra2aes_t caliptra2aes,
82         output aes2caliptra_t aes2caliptra,

```

```

65 // Key manager (keymgr) key sideload interface
66 input keymgr_pkg::hw_key_req_t keym-
67 gr_key_i,
68 // Bus interface
69 input caliptra_tlul_pkg::tl_h2d_t
70 tl_i,
71 output caliptra_tlul_pkg::tl_d2h_t
72 tl_o,
73 // Alerts
74 input caliptra_prim_alert_pkg::alert_rx_t [NumAlerts-1:0]
75 alert_rx_i,
76 output caliptra_prim_alert_pkg::alert_tx_t [NumAlerts-1:0]
77 alert_tx_o
78 );
79 localparam int unsigned EntropyWidth = edn_pkg::END-
80 POINT_BUS_WIDTH;
81 // Signals
82 aes_reg2hw_t reg2hw;
83 aes_hw2reg_t hw2reg;
84 logic [NumAlerts-1:0] alert;
85 lc_ctrl_pkg::lc_tx_t lc_escalate_en;
86 logic edn_req_int;
87 logic edn_req_hold_d, edn_req_hold_q;
88 logic edn_req;
89 logic edn_ack;
90 logic [EntropyWidth-1:0] edn_data;
91 logic unused_edn_fips;
92 logic entropy_clearing_req, en-
93 trophy_masking_req;
94 logic entropy_clearing_ack, en-
95 trophy_masking_ack;
96 // Inputs //
97 //////////////////////////////////////////////////
98 // SEC_CM: BUS.INTEGRITY
99 // SEC_CM: AUX.CONFIG.SHADOW
100 // SEC_CM: AUX.CONFIG.REGWEN
101 // SEC_CM: KEY.SW_UNREADABLE
102 // SEC_CM: DATA_REG.SW_UNREADABLE
103 // Register interface
104 logic intg_err_alert;
105 logic shadowed_storage_err, shadowed_update_err;
106 aes_reg_top u_reg (
107     .clk_i,
108     .rst_ni,
109     .rst_shadowed_ni,
110     .tl_i,
111     .tl_o,
112     .input_ready_o,
113     .output_valid_o,
114     .reg2hw,
115     .hw2reg,
116     .shadowed_storage_err_o(shadowed_storage_err),
117     .shadowed_update_err_o(shadowed_update_err),
118     .intg_err_o(intg_err_alert)
119 );
120 // SEC_CM: LC_ESCALATE.EN.INTERSIG.MUBI
121 // Synchronize life cycle input
122 caliptra_prim_lc_sync #(
123     .NumCopies (1)
124 ) u_caliptra_prim_lc_sync (
125     .clk_i,
126     .rst_ni,
127     .lc_en_i ( lc_escalate_en_i ),
128     .lc_en_o ( {lc_escalate_en} )
129 );
130 //////////////////////////////////////////////////
131 // Caliptra Control Intercept //
132 //////////////////////////////////////////////////
133 localparam CLP_AES_KV_CHUNK_SIZE = NumRegsData*32; // Size
134 of data that can be stored at once, atomically, at end of AES op
135 genvar kv_ii;
136 aes_hw2reg_t hw2reg_caliptra;
137 aes_reg2hw_t reg2hw_caliptra;
138 logic aes2caliptra_kv_data_out_valid;
139 logic [CLP_AES_KV_WR_DW-1:0] aes2caliptra_kv_data_out;
140 logic kv_data_intercept;
141 logic kv_data_intercept_end;
142 logic [$log2(CLP_AES_KV_WR_DW/CLP_AES_KV_CHUNK_SIZE)-1:0]
143 kv_data_counter; // This will peg at the key_size value if decrypted
144 plaintext is larger than CLP_AES_KV_WR_DW
145 logic [$log2(CLP_AES_KV_WR_DW/CLP_AES_KV_CHUNK_SIZE)-1:0]
146 kv_data_thresh; // Comparison value for kv_data_counter to saturate,
147 i.e. the key_size normalized to data chunk size
148 logic incr_kv_data_counter;
149 logic hw2reg_data_out_mask_en;
150 logic [$bits(aes_hw2reg_data_out_mreg_t)-1:0]
151 hw2reg_data_out_mask;
152 logic output_blocked_de;
153 logic output_vald_r;

```

```

152
153 // Mask to conceal data_out from reg API (when dest is KV)
154 assign hw2reg_data_out_mask = {$bits(aes_h-
hw2reg_data_out_mreg_t){hw2reg_data_out_mask_en}};
155
156 // Enable signal to drive an update to the
status.output_lost register field
157 always_comb output_blocked_de =
hw2reg_caliptra.status.output_valid.de &&
158 hw2reg_caliptra.status.output_valid.d &&
159 calipt-
ra2aes.block_reg_output;
160
161 always_comb begin
162 // Passthrough
163 hw2reg
hw2reg_caliptra;
164 // Augmented
165 // For OCP LOCK use-cases, add a condition to set out-
put_lost if output data is required to go to KV.
166 // Output Lost is usually only used for Manual mode.
TODO if we support manual mode with OCP LOCK flows (and KV WR) this
may
167 // need to be updated.
168 hw2reg.status.output_lost.d = out-
put_blocked_de ? 1'b1 :
169
hw2reg_caliptra.status.output_lost.d;
170 hw2reg.status.output_lost.de =
hw2reg_caliptra.status.output_lost.de ||
171 out-
put_blocked_de;
172 // Concealed
173 foreach (hw2reg.data_out[idx]) begin
174 hw2reg.data_out[idx].d =
hw2reg_caliptra.data_out[idx].d & hw2reg_data_out_mask;
175 end
176 end
177 always_comb begin
178 // Passthrough
179 reg2hw_caliptra = reg2hw;
180 // RE intercept
181 foreach (reg2hw.data_out[idx]) begin
182 reg2hw_caliptra.data_out[idx].q =
reg2hw.data_out[idx].q ;
183 reg2hw_caliptra.data_out[idx].re = kv_data_intercept ?
output_valid_r :
184
reg2hw.data_out[idx].re;
185 end
186 end
187
188 // Flag to detect when data out shall be routed to Caliptra
KeyVault
189 always_ff @(posedge clk_i or negedge rst_ni) begin:
kv_data_intercept_reg
190 if (!rst_ni) begin
191 kv_data_intercept <= 1'b0;
192 hw2reg_data_out_mask_en <= 1'b1;
193 end
194 // FW must arm the KV write prior to starting AES opera-
tion
195 else if ((kv_data_counter == 0) &&
reg2hw_caliptra.data_in[0].qe) begin
196 kv_data_intercept <= caliptra2aes.kv_en;
197 hw2reg_data_out_mask_en <= ~caliptra2aes.kv_en &&
~caliptra2aes.block_reg_output; // This signal winds up being ef-
fectively (~kv_data_intercept && ~caliptra2aes.block_reg_output)
198 end
199 // TODO support for Manual operation mode with trig-
ger.start.q?
200 else if (kv_data_intercept_end) begin
201 kv_data_intercept <= 1'b0;
202 hw2reg_data_out_mask_en <= 1'b1;
203 end
204 end
205
206 // NOTE if key size is not an integer multiple of
CLP_AES_KV_CHUNK_SIZE, this is a rounded down value
207 always_comb kv_data_thresh = ($clog2(CLP_AES_KV_WR_DW/
CLP_AES_KV_CHUNK_SIZE)) * (caliptra2aes.key_release_key_size/
(CLP_AES_KV_CHUNK_SIZE/8)-1);
208 // NOTE: This assumes that output_valid will always assert
prior to entering idle state, which should be true.
209 // If this doesn't hold, then kv_data_intercept will
deassert before the final data beat is captured, and
210 // the KV write won't be issued
211
212 always_comb kv_data_intercept_end = hw2reg.status.idle.de &&
hw2reg.status.idle.d && !reg2hw_caliptra.status.idle.q &&
(kv_data_counter == kv_data_thresh);
213
214 // Latch when data_out is valid, used to generate read-en-
able and signal data capture
215 always_ff @(posedge clk_i or negedge rst_ni) begin: out-
put_valid_dd_reg
216 if (!rst_ni) begin
217 output_valid_r <= 1'b0;
218 end

```

```

218 else if (hw2reg_caliptra.status.output_valid.de) begin
219 output_valid_r <=
hw2reg_caliptra.status.output_valid.d;
220 end
221 end
222
223 // Index into the KV output data based on number of AES
rounds observed
224 always_comb incr_kv_data_counter = kv_data_intercept && out-
put_valid_r;
225 always_ff @(posedge clk_i or negedge rst_ni) begin: aes2c-
aliptra_kv_data_counter_reg
226 if (!rst_ni) begin
227 kv_data_counter <= '0;
228 end
229 else if (!kv_data_intercept) begin
230 kv_data_counter <= '0;
231 end
232 else if (incr_kv_data_counter && (kv_data_counter ==
kv_data_thresh)) begin
233 kv_data_counter <= kv_data_counter;
234 end
235 else if (incr_kv_data_counter) begin
236 kv_data_counter <= kv_data_counter + 1;
237 end
238 end
239
240 // Signal data is valid for KV write client once full AES
operation is done
241 always_ff @(posedge clk_i or negedge rst_ni) begin: aes2c-
aliptra_kv_data_valid_reg
242 if (!rst_ni) begin
243 aes2caliptra_kv_data_out_valid <= 1'b0;
244 end
245 else if (caliptra2aes.clear_secrets ||
reg2hw_caliptra.trigger.data_out_clear.q) begin
246 aes2caliptra_kv_data_out_valid <= 1'b0;
247 end
248 else if (kv_data_intercept_end) begin
249 aes2caliptra_kv_data_out_valid <= 1'b1;
250 end
251 else if (caliptra2aes.kv_write_done) begin
252 aes2caliptra_kv_data_out_valid <= 1'b0;
253 end
254 end
255 assign aes2caliptra.kv_data_out_valid = aes2calipt-
ra_kv_data_out_valid;
256 assign aes2caliptra.kv_key_in_use =
hw2reg.ctrl_shadowed.sideload.d;
257 assign aes2caliptra.key_len =
key_len_e'(hw2reg.ctrl_shadowed.key_len.d);
258 // TODO: Qualify this with anything?
259
260 // Probably not needed. Timing of the kv write request
is tightly controlled, and that's the only
261 // place this signal is used.
262 always_comb aes2caliptra.aes_operation_is_ecb_decrypt =
( aes_op_e'(hw2reg.ctrl_shadowed.operation.d) == AES_DEC) &&
(aes_mode_e'(hw2reg.ctrl_shadowed.mode.d) == AES_ECB);
263
264 // Capture data_out until operation is done
265 generate
266 for (kv_ii=0; kv_ii < CLP_AES_KV_WR_DW/
CLP_AES_KV_CHUNK_SIZE; kv_ii++) begin
267 always_ff @(posedge clk_i or negedge rst_ni) begin:
aes2caliptra_kv_data_reg
268 if (!rst_ni) begin
269 aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]
<= CLP_AES_KV_CHUNK_SIZE'(0);
270 end
271 else if ((caliptra2aes.clear_secrets) ||
reg2hw_caliptra.trigger.data_out_clear.q) begin
272 aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]
<= CLP_AES_KV_CHUNK_SIZE'(0);
273 end
274 else if (incr_kv_data_counter &&
(kv_data_counter == kv_ii)) begin
275 aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]
<= {hw2reg_caliptra.data_out[3].d,
276
hw2reg_caliptra.data_out[2].d,
277
hw2reg_caliptra.data_out[1].d,
278
hw2reg_caliptra.data_out[0].d};
279 end
280 else if (caliptra2aes.kv_write_done) begin
281 aes2calipt-
ra_kv_data_out[(kv_ii)*CLP_AES_KV_CHUNK_SIZE+:CLP_AES_KV_CHUNK_SIZE]
<= CLP_AES_KV_CHUNK_SIZE'(0);
282 end
283 end
284 end
285 endgenerate

```



```

286 assign aes2caliptra_kv_data_out = aes2caliptra_kv_data_out;
287
288 //////////////////////////////////////////////////
289 // EDN Interface //
290 //////////////////////////////////////////////////
291
292 // Internally, we have up to two PRNGs that share the EDN
interface for reseeding. Here, we just
293 // arbitrate the requests. Upsizing of the entropy to the
correct width is performed inside the
294 // PRNGs.
295 // Reseed operations for the clearing PRNG are initiated by
software. Reseed operations for the
296 // masking PRNG can also be automatically initiated.
297 assign edn_req_int = entropy_clearing_req | en-
tropy_masking_req;
298 // Only forward ACK to PRNG currently requesting entropy.
Give higher priority to clearing PRNG.
299 assign entropy_clearing_ack = entropy_clearing_req &
edn_ack;
300 assign entropy_masking_ack = ~entropy_clearing_req & en-
tropy_masking_req & edn_ack;
301
302 // Upon escalation or detection of a fatal alert, an EDN re-
quest signal can be dropped before
303 // getting acknowledged. This is okay with respect to AES as
the module will need to be reset
304 // anyway. However, to not leave EDN in a strange state, we
hold the request until it's actually
305 // acknowledged.
306 assign edn_req = edn_req_int | edn_req_hold_q;
307 assign edn_req_hold_d = (edn_req_hold_q | edn_req) &
~edn_ack;
308 always_ff @(posedge clk_i or negedge rst_ni) begin :
edn_req_reg
309   if (!rst_ni) begin
310     edn_req_hold_q <= '0;
311   end else begin
312     edn_req_hold_q <= edn_req_hold_d;
313   end
314 end
315
316 // Synchronize EDN interface
317 caliptra_prim_sync_reqack_data #(
318   .Width(EntropyWidth),
319   .DataSrc2Dst(1'b0),
320   .DataReg(1'b0)
321 ) u_caliptra_prim_sync_reqack_data (
322   .clk_src_i ( clk_i ),
323   .rst_src_ni ( rst_ni ),
324   .clk_dst_i ( clk_edn_i ),
325   .rst_dst_ni ( rst_edn_ni ),
326   .req_chk_i ( 1'b1 ),
327   .src_req_i ( edn_req ),
328   .src_ack_o ( edn_ack ),
329   .dst_req_o ( edn_o.edn_req ),
330   .dst_ack_i ( edn_i.edn_ack ),
331   .data_i ( edn_i.edn_bus ),
332   .data_o ( edn_data )
333 );
334 // We don't track whether the entropy is pre-FIPS or not in-
side AES.
335 assign unused_edn_fips = edn_i.edn_fips;
336
337 //////////////////////////////////////////////////
338 // Core //
339 //////////////////////////////////////////////////
340
341 // AES core
342 aes_core #(
343   .AES192Enable ( AES192Enable ),
344   .AESGCMEnable ( AESGCMEnable ),
345   .SecMasking ( SecMasking ),
346   .SecSBoxImpl ( SecSBoxImpl ),
347   .SecStartTriggerDelay ( SecStartTriggerDelay ),
348   .SecAllowForcingMasks ( SecAllowForcingMasks ),
349   .SecSkipPRNGReseeding ( SecSkipPRNGReseeding ),
350   .EntropyWidth ( EntropyWidth ),
351   .RndCnstClearingLfsrSeed ( RndCnstClearingLfsrSeed ),
352   .RndCnstClearingLfsrPerm ( RndCnstClearingLfsrPerm ),
353   .RndCnstClearingSharePerm ( RndCnstClearingSharePerm ),
354   .RndCnstMaskingLfsrSeed ( RndCnstMaskingLfsrSeed ),
355   .RndCnstMaskingLfsrPerm ( RndCnstMaskingLfsrPerm )
356 ) u_aes_core (
357   .clk_i ( clk_i ),
358   .rst_ni ( rst_ni ),
359   .rst_shadowed_ni ( rst_shadowed_ni ),
360   .entropy_clearing_req_o ( entropy_clearing_req ),
361   .entropy_clearing_ack_i ( entropy_clearing_ack ),
362   .entropy_clearing_i ( edn_data ),
363   .entropy_masking_req_o ( entropy_masking_req ),
364   .entropy_masking_ack_i ( entropy_masking_ack ),
365   .entropy_masking_i ( edn_data ),
366
367   .keymgr_key_i ( keymgr_key_i ),
368
369   .lc_escalate_en_i ( lc_escalate_en ),
370
371   .shadowed_storage_err_i ( shadowed_storage_err ),
372   .shadowed_update_err_i ( shadowed_update_err ),

```

```

373   .intg_err_alert_i ( intg_err_alert ),
374   .alert_recov_o ( alert[0] ),
375   .alert_fatal_o ( alert[1] ),
376
377   .reg2hw ( reg2hw_caliptra ),
378   .hw2reg ( hw2reg_caliptra )
379 );
380
381 assign idle_o = calipt-
ra_prim_mubi_pkg::mubi4_bool_to_mubi(reg2hw.status.idle.q);
382
383 //////////////////////////////////////////////////
384 // Alerts //
385 //////////////////////////////////////////////////
386
387 logic [NumAlerts-1:0] alert_test;
388 assign alert_test = {
389   reg2hw.alert_test.fatal_fault.q &
390   reg2hw.alert_test.fatal_fault.qe,
391   reg2hw.alert_test.recov_ctrl_update_err.q &
392   reg2hw.alert_test.recov_ctrl_update_err.qe
393 };
394
395 for (genvar i = 0; i < NumAlerts; i++) begin : gen_alert_tx
396   caliptra_prim_alert_sender #(
397     .AsyncOn(AlertAsyncOn[i]),
398     .IsFatal(i)
399   ) u_caliptra_prim_alert_sender (
400     .clk_i,
401     .rst_ni,
402     .alert_test_i ( alert_test[i] ),
403     .alert_req_i ( alert[i] ),
404     .alert_ack_o ( ),
405     .alert_state_o ( ),
406     .alert_rx_i ( alert_rx_i[i] ),
407     .alert_tx_o ( alert_tx_o[i] )
408   );
409 end
410
411 //////////////////////////////////////////////////
412 // Assertions //
413 //////////////////////////////////////////////////
414
415 // All outputs should have a known value after reset
416 `CALIPTRA_ASSERT_KNOWN(Tl0ValidKnown, tl_o.d_valid)
417 `CALIPTRA_ASSERT_KNOWN(Tl0AReadyKnown, tl_o.a_ready)
418 `CALIPTRA_ASSERT_KNOWN(IdleKnown, idle_o)
419 `CALIPTRA_ASSERT_KNOWN(EdnReqKnown, edn_o)
420 `CALIPTRA_ASSERT_KNOWN(AlertTxKnown, alert_tx_o)
421
422 // Alert assertions for sparse FSMs.
423 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_con-
trol_fsm_svas
424   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_con-
trol_fsm_svas_p
425     `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesControlFsmCheck_A,
426       u_aes_core.u_aes_control.gen_fsm[i].gen_fsm.p.
427       u_aes_con-
trol_fsm_i.u_aes_control_fsm.u_state_regs,
428       alert_tx_o[1])
429   end else begin : gen_control_fsm_svas_n
430     `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesControlFsmCheck_A,
431       u_aes_core.u_aes_control.gen_fsm[i].gen_fsm.n.
432       u_aes_con-
trol_fsm_i.u_aes_control_fsm.u_state_regs,
433       alert_tx_o[1])
434   end
435 end
436
437 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_ctr_fsm_svas
438   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_ctr_fsm_svas_p
439     `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrFsmCheck_A,
440       u_aes_core.u_aes_ctr.gen_fsm[i].gen_fsm.p.
441       u_aes_ctr_fsm_i.u_aes_ctr_fsm.u_state_regs,
442       alert_tx_o[1])
443   end else begin : gen_ctr_fsm_svas_n
444     `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCtrFsmCheck_A,
445       u_aes_core.u_aes_ctr.gen_fsm[i].gen_fsm.n.
446       u_aes_ctr_fsm_i.u_aes_ctr_fsm.u_state_regs,
447       alert_tx_o[1])
448   end
449 end
450
451 for (genvar i = 0; i < Sp2VWidth; i++) begin :
gen_cipher_control_fsm_svas
452   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin :
gen_cipher_control_fsm_svas_p
453     `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
454       u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fs
455       m.p.
456       u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
457       alert_tx_o[1])

```

```

458     end else begin : gen_cipher_control_fsm_svas_n
459     `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesCipherControlFsmCheck_A,
460
u_aes_core.u_aes_cipher_core.u_aes_cipher_control.gen_fsm[i].gen_fsm_n.
461
u_aes_cipher_control_fsm_i.u_aes_cipher_control_fsm.u_state_regs,
462     alert_tx_o[1])
463     end
464     end
465     if (AESGCMEEnable) begin : gen_ghash_fsm_sva
466     `CALIPT-
RA_ASSERT_PRIM_FSM_ERROR_TRIGGER_ALERT(AesGhashFsmCheck_A,
467
u_aes_core.gen_ghash.u_aes_ghash.u_state_regs,
468     alert_tx_o[1])
469     end
470     if (AESGCMEEnable && SecMasking) begin : gen_ghash_onehot_sva
471     for (genvar s = 0; s < 2; s++) begin :
gen_ghash_onehot_add_in_sva
472     `CALIPT-
RA_ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashAadOnehotCheck_A,
473
u_aes_core.gen_ghash.u_aes_ghash.gen_masked_add.gen_add_in_muxes[s].
474     u_caliptra_prim_onehot_check_add_in_sel,
475     alert_tx_o[1])
476     end
477     `CALIPT-
RA_ASSERT_PRIM_ONEHOT_ERROR_TRIGGER_ALERT(GhashMultOnehotCheck_A,
478
u_aes_core.gen_ghash.u_aes_ghash.gen_gf_mult1_mux.u_caliptra_prim_o
479     nehot_check_gf_mult1_in_sel,
480     alert_tx_o[1])
481     end
482     // Alert assertions for reg_we onehot check
483     `CALIPT-
RA_ASSERT_PRIM_REG_WE_ONEHOT_ERROR_TRIGGER_ALERT(RegWeOnehotCheck_A,
484     u_reg, alert_tx_o[1])
485     endmodule

```

2. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control.sv

(485 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control
6 //
7 // This module controls the AES cipher core including the key
  expand module.
8
9 `include "caliptra_prim_assert.sv"
10
11 module aes_cipher_control import aes_pkg::*;
12 #(
13     parameter bit CiphOpFwdOnly = 0,
14     parameter bit SecMasking = 0,
15     parameter sbbox_impl_e SecSBBoxImpl = SBBoxImplDom
16 ) (
17     input logic clk_i,
18     input logic rst_ni,
19
20     // Input handshake signals
21     input sp2v_e in_valid_i,
22     output sp2v_e in_ready_o,
23
24     // Output handshake signals
25     output sp2v_e out_valid_o,
26     input sp2v_e out_ready_i,
27
28     // Control and sync signals
29     input logic cfg_valid_i,
30     input logic op_i,
31     input logic ciph_op_e,
32     input logic key_len_e,
33     input sp2v_e crypt_i,
34     output sp2v_e crypt_o,
35     input sp2v_e dec_key_gen_i,
36     output sp2v_e dec_key_gen_o,
37     input logic prng_reseed_i,
38     output logic prng_reseed_o,
39     input logic key_clear_i,
40     output logic key_clear_o,
41     input logic data_out_clear_i,
42     output logic data_out_clear_o,
43     input logic mux_sel_err_i,
44     input logic op_err_i,
45     input logic alert_fatal_i,
46     output logic alert_o,
47
48     // Control signals for masking PRNG
49     output logic prng_update_o,
50     output logic prng_reseed_req_o,
51     input logic prng_reseed_ack_i,

```

```

52
53 // Control and sync signals for cipher data path
54 output state_sel_e state_sel_o,
55 output sp2v_e state_we_o,
56 output sp2v_e sub_bytes_en_o,
57 input sp2v_e sub_bytes_out_req_i,
58 output sp2v_e sub_bytes_out_ack_o,
59 output add_rk_sel_e add_rk_sel_o,
60
61 // Control and sync signals for key expand data path
62 output ciph_op_e key_expand_op_o,
63 output key_full_sel_e key_full_sel_o,
64 output sp2v_e key_full_we_o,
65 output key_dec_sel_e key_dec_sel_o,
66 output sp2v_e key_dec_we_o,
67 output sp2v_e key_expand_en_o,
68 input sp2v_e key_expand_out_req_i,
69 output sp2v_e key_expand_out_ack_o,
70 output logic key_expand_clear_o,
71 output logic [3:0] key_expand_round_o,
72 output key_words_sel_e key_words_sel_o,
73 output round_key_sel_e round_key_sel_o,
74 );
75
76 // Signals
77 logic [3:0] rnd_ctr;
78 sp2v_e crypt_d, crypt_q;
79 sp2v_e dec_key_gen_d, dec_key_gen_q;
80 logic prng_reseed_d, prng_reseed_q;
81 logic key_clear_d, key_clear_q;
82 logic data_out_clear_d, data_out_clear_q;
83 sp2v_e sub_bytes_out_req;
84 sp2v_e key_expand_out_req;
85 sp2v_e in_valid;
86 sp2v_e out_ready;
87 sp2v_e crypt;
88 sp2v_e dec_key_gen;
89 logic mux_sel_err;
90 logic mr_err;
91 logic sp_enc_err;
92 logic rnd_ctr_err;
93
94 // Sparsified FSM signals. These are needed for connecting
  the individual bits of the Sp2V
95 // signals to the single-rail FSMs.
96 logic [Sp2VWidth-1:0] sp_in_valid;
97 logic [Sp2VWidth-1:0] sp_in_ready;
98 logic [Sp2VWidth-1:0] sp_out_valid;
99 logic [Sp2VWidth-1:0] sp_out_ready;
100 logic [Sp2VWidth-1:0] sp_crypt;
101 logic [Sp2VWidth-1:0] sp_dec_key_gen;
102 logic [Sp2VWidth-1:0] sp_state_we;
103 logic [Sp2VWidth-1:0] sp_sub_bytes_en;
104 logic [Sp2VWidth-1:0] sp_sub_bytes_out_req;
105 logic [Sp2VWidth-1:0] sp_sub_bytes_out_ack;
106 logic [Sp2VWidth-1:0] sp_key_full_we;
107 logic [Sp2VWidth-1:0] sp_key_dec_we;
108 logic [Sp2VWidth-1:0] sp_key_expand_en;
109 logic [Sp2VWidth-1:0] sp_key_expand_out_req;
110 logic [Sp2VWidth-1:0] sp_key_expand_out_ack;
111 logic [Sp2VWidth-1:0] sp_crypt_d;
112 logic [Sp2VWidth-1:0] sp_crypt_q;
113 logic [Sp2VWidth-1:0] sp_dec_key_gen_d;
114 logic [Sp2VWidth-1:0] sp_dec_key_gen_q;
115
116 // Multi-rail signals. These are outputs of the single-rail
  FSMs and need combining.
117 logic [Sp2VWidth-1:0] mr_alert;
118 logic [Sp2VWidth-1:0] mr_prng_update;
119 logic [Sp2VWidth-1:0] mr_prng_reseed_req;
120 logic [Sp2VWidth-1:0] mr_key_expand_clear;
121 logic [Sp2VWidth-1:0] mr_prng_reseed_d;
122 logic [Sp2VWidth-1:0] mr_key_clear_d;
123 logic [Sp2VWidth-1:0] mr_data_out_clear_d;
124
125 state_sel_e [Sp2VWidth-1:0] mr_state_sel;
126 add_rk_sel_e [Sp2VWidth-1:0] mr_add_rk_sel;
127 key_full_sel_e [Sp2VWidth-1:0] mr_key_full_sel;
128 key_dec_sel_e [Sp2VWidth-1:0] mr_key_dec_sel;
129 key_words_sel_e [Sp2VWidth-1:0] mr_key_words_sel;
130 round_key_sel_e [Sp2VWidth-1:0] mr_round_key_sel;
131
132 logic [Sp2VWidth-1:0][3:0] mr_rnd_ctr;
133
134 // FSM
135 //
136 //
137
138 // Convert sp2v_e signals to sparsified inputs.
139 assign sp_in_valid = {in_valid};
140 assign sp_out_ready = {out_ready};
141 assign sp_crypt = {crypt};
142 assign sp_dec_key_gen = {dec_key_gen};
143 assign sp_sub_bytes_out_req = {sub_bytes_out_req};
144 assign sp_key_expand_out_req = {key_expand_out_req};
145 assign sp_crypt_q = {crypt_q};
146 assign sp_dec_key_gen_q = {dec_key_gen_q};

```

```

147
148 // SEC_CM: CIPHER.FSM.REDUN
149 // SEC_CM: CIPHER.CTR.REDUN
150 // For every bit in the Sp2V signals, one separate rail is
instantiated. The inputs and outputs
151 // of every rail are buffered to prevent aggressive syntheses
is optimizations.
152 for (genvar i = 0; i < Sp2VWidth; i++) begin : gen_fsm
153   if (SP2V_LOGIC_HIGH[i] == 1'b1) begin : gen_fsm_p
154     aes_cipher_control_fsm_p #(
155       .SecMasking ( SecMasking ),
156       .SecSBoxImpl ( SecSBoxImpl )
157     ) u_aes_cipher_control_fsm_i (
158       .clk_i      ( clk_i ),
159       .rst_ni     ( rst_ni ),
160
161       .in_valid_i  ( sp_in_val-
id[i] ), // Sparsified
162       .in_ready_o  ( sp_in_ready[i] ), // Sparsified
163
164       .out_valid_o  ( sp_out_val-
id[i] ), // Sparsified
165       .out_ready_i  ( sp_out_ready[i] ), // Sparsified
166
167       .cfg_valid_i  ( cfg_valid_i ),
168       .op_i         ( op_i ),
169       .key_len_i    ( key_len_i ),
170       .crypt_i      ( sp_crypt[i] ), // Sparsified
171       .dec_key_gen_i ( sp_dec_key_gen[i] ), // Sparsified
172       .prng_reseed_i ( prng_reseed_i ),
173       .key_clear_i   ( key_clear_i ),
174       .data_out_clear_i ( data_out_clear_i ),
175       .mux_sel_err_i ( mux_sel_err ),
176       .sp_enc_err_i  ( sp_enc_err ),
177       .rnd_ctr_err_i ( rnd_ctr_err ),
178       .op_err_i      ( op_err_i ),
179       .alert_fatal_i ( alert_fatal_i ),
180       .alert_o       ( mr_alert[i] ), // OR-combine
181
182       .prng_update_o ( mr_prng_update[i] ), // OR-combine
183       .prng_reseed_req_o ( mr_prng_reseed_req[i] ), // OR-combine
184       .prng_reseed_ack_i ( prng_reseed_ack_i ),
185
186       .state_sel_o   ( mr_state_sel[i] ), // OR-combine
187       .state_we_o    ( sp_state_we[i] ), // Sparsified
188       .sub_bytes_en_o ( sp_sub_bytes_en[i] ), // Sparsified
189       .sub_bytes_out_req_i ( sp_sub_bytes_out_req[i] ), // Sparsified
190       .sub_bytes_out_ack_o ( sp_sub_bytes_out_ack[i] ), // Sparsified
191       .add_rk_sel_o  ( mr_add_rk_sel[i] ), // OR-combine
192
193       .key_full_sel_o ( mr_key_full_sel[i] ), // OR-combine
194       .key_full_we_o  ( sp_key_full_we[i] ), // Sparsified
195       .key_dec_sel_o  ( mr_key_dec_sel[i] ), // OR-combine
196       .key_dec_we_o   ( sp_key_dec_we[i] ), // Sparsified
197       .key_expand_en_o ( sp_key_expand_en[i] ), // Sparsified
198       .key_expand_out_req_i ( sp_key_expand_out_req[i] ), // Sparsified
199       .key_expand_out_ack_o ( sp_key_expand_out_ack[i] ), // Sparsified
200       .key_expand_clear_o ( mr_key_expand_clear[i] ), // OR-combine
201       .rnd_ctr_o      ( mr_rnd_ctr[i] ), // OR-combine
202       .key_words_sel_o ( mr_key_words_sel[i] ), // OR-combine
203       .round_key_sel_o ( mr_round_key_sel[i] ), // OR-combine
204
205       .crypt_q_i      ( sp_crypt_q[i] ), // Sparsified
206       .crypt_d_o      ( sp_crypt_d[i] ), // Sparsified
207       .dec_key_gen_q_i ( sp_dec_key_gen_q[i] ), // Sparsified
208       .dec_key_gen_d_o ( sp_dec_key_gen_d[i] ), // Sparsified
209       .prng_reseed_q_i ( prng_reseed_q ),
210       .prng_reseed_d_o ( prng_reseed_d ), // AND-combine
211       .key_clear_q_i   ( key_clear_q ),
212       .key_clear_d_o   ( mr_key_clear_d[i] ), // AND-combine
213       .data_out_clear_q_i ( data_out_clear_q ),

```

```

214       .data_out_clear_d_o
( mr_data_out_clear_d[i] ) // AND-combine
215     );
216   end else begin : gen_fsm_n
217     aes_cipher_control_fsm_n #(
218       .SecMasking ( SecMasking ),
219       .SecSBoxImpl ( SecSBoxImpl )
220     ) u_aes_cipher_control_fsm_i (
221       .clk_i      ( clk_i ),
222       .rst_ni     ( rst_ni ),
223
224       .in_valid_ni  ( sp_in_val-
id[i] ), // Sparsified
225       .in_ready_no  ( sp_in_ready[i] ), // Sparsified
226
227       .out_valid_no  ( sp_out_val-
id[i] ), // Sparsified
228       .out_ready_ni  ( sp_out_ready[i] ), // Sparsified
229
230       .cfg_valid_i  ( cfg_valid_i ),
231       .op_i         ( op_i ),
232       .key_len_i    ( key_len_i ),
233       .crypt_ni     ( sp_crypt[i] ), // Sparsified
234       .dec_key_gen_ni ( sp_dec_key_gen[i] ), // Sparsified
235       .prng_reseed_i ( prng_reseed_i ),
236       .key_clear_i   ( key_clear_i ),
237       .data_out_clear_i ( data_out_clear_i ),
238       .mux_sel_err_i ( mux_sel_err ),
239       .sp_enc_err_i  ( sp_enc_err ),
240       .rnd_ctr_err_i ( rnd_ctr_err ),
241       .op_err_i      ( op_err_i ),
242       .alert_fatal_i ( alert_fatal_i ),
243       .alert_o       ( mr_alert[i] ), // OR-combine
244
245       .prng_update_o ( mr_prng_update[i] ), // OR-combine
246       .prng_reseed_req_o ( mr_prng_reseed_req[i] ), // OR-combine
247       .prng_reseed_ack_i ( prng_reseed_ack_i ),
248
249       .state_sel_o   ( mr_state_sel[i] ), // OR-combine
250       .state_we_no   ( sp_state_we[i] ), // Sparsified
251       .sub_bytes_en_no ( sp_sub_bytes_en[i] ), // Sparsified
252       .sub_bytes_out_req_ni ( sp_sub_bytes_out_req[i] ), // Sparsified
253       .sub_bytes_out_ack_no ( sp_sub_bytes_out_ack[i] ), // Sparsified
254       .add_rk_sel_o  ( mr_add_rk_sel[i] ), // OR-combine
255
256       .key_full_sel_o ( mr_key_full_sel[i] ), // OR-combine
257       .key_full_we_no ( sp_key_full_we[i] ), // Sparsified
258       .key_dec_sel_o  ( mr_key_dec_sel[i] ), // OR-combine
259       .key_dec_we_no  ( sp_key_dec_we[i] ), // Sparsified
260       .key_expand_en_no ( sp_key_expand_en[i] ), // Sparsified
261       .key_expand_out_req_ni ( sp_key_expand_out_req[i] ), // Sparsified
262       .key_expand_out_ack_no ( sp_key_expand_out_ack[i] ), // Sparsified
263       .key_expand_clear_o ( mr_key_expand_clear[i] ), // OR-combine
264       .rnd_ctr_o      ( mr_rnd_ctr[i] ), // OR-combine
265       .key_words_sel_o ( mr_key_words_sel[i] ), // OR-combine
266       .round_key_sel_o ( mr_round_key_sel[i] ), // OR-combine
267
268       .crypt_q_ni     ( sp_crypt_q[i] ), // Sparsified
269       .crypt_d_no     ( sp_crypt_d[i] ), // Sparsified
270       .dec_key_gen_q_ni ( sp_dec_key_gen_q[i] ), // Sparsified
271       .dec_key_gen_d_no ( sp_dec_key_gen_d[i] ), // Sparsified
272       .prng_reseed_q_i ( prng_reseed_q ),
273       .prng_reseed_d_o ( prng_reseed_d ), // AND-combine
274       .key_clear_q_i   ( key_clear_q ),
275       .key_clear_d_o   ( mr_key_clear_d[i] ), // AND-combine
276       .data_out_clear_q_i ( data_out_clear_q ),
277       .data_out_clear_d_o ( mr_data_out_clear_d[i] ), // AND-combine
278     );
279   end
280 end

```

```

281
282 // Convert sparsified outputs to sp2v_e type.
283 assign in_ready_o    = sp2v_e'(sp_in_ready);
284 assign out_valid_o   = sp2v_e'(sp_out_valid);
285 assign state_we_o    = sp2v_e'(sp_state_we);
286 assign sub_bytes_en_o = sp2v_e'(sp_sub_bytes_en);
287 assign sub_bytes_out_ack_o = sp2v_e'(sp_sub_bytes_out_ack);
288 assign key_full_we_o  = sp2v_e'(sp_key_full_we);
289 assign key_dec_we_o   = sp2v_e'(sp_key_dec_we);
290 assign key_expand_en_o = sp2v_e'(sp_key_expand_en);
291 assign key_expand_out_ack_o = sp2v_e'(sp_key_ex-
pand_out_ack);
292 assign crypt_d        = sp2v_e'(sp_crypt_d);
293 assign dec_key_gen_d  = sp2v_e'(sp_dec_key_gen_d);
294
295 // Combine single-bit FSM outputs.
296 // OR: One bit is sufficient to drive the corresponding out-
put bit high.
297 assign alert_o        = |mr_alert;
298 assign prng_update_o  = |mr_prng_update;
299 assign prng_reseed_req_o = |mr_prng_reseed_req;
300 assign key_expand_clear_o = |mr_key_expand_clear;
301 // AND: Only if all bits are high, the corresponding status
is signaled which will lead to
302 // the clearing of these trigger bits.
303 assign prng_reseed_d  = &mr_prng_reseed_d;
304 assign key_clear_d    = &mr_key_clear_d;
305 assign data_out_clear_d = &mr_data_out_clear_d;
306
307 // Combine multi-bit, sparse FSM outputs. We simply OR them
together. If the FSMs don't provide
308 // the same outputs, two cases are possible:
309 // - An invalid encoding results: A downstream checker will
fire, see mux_sel_err_i.
310 // - A valid encoding results: The outputs are compared be-
low to cover this case, see mr_err;
311 always_comb begin : combine_sparse_signals
312   state_sel_o  = state_sel_e'({StateSelWidth{1'b0}});
313   add_rk_sel_o  = add_rk_sel_e'({AddRkSelWidth{1'b0}});
314   key_full_sel_o =
key_full_sel_e'({KeyFullSelWidth{1'b0}});
315   key_dec_sel_o = key_dec_sel_e'({KeyDecSelWidth{1'b0}});
316   key_words_sel_o =
key_words_sel_e'({KeyWordsSelWidth{1'b0}});
317   round_key_sel_o =
round_key_sel_e'({RoundKeySelWidth{1'b0}});
318   mr_err       = 1'b0;
319
320   for (int i = 0; i < Sp2VWidth; i++) begin
321     state_sel_o  = state_sel_e'({state_sel_o} |
{mr_state_sel[i]});
322     add_rk_sel_o  = add_rk_sel_e'({add_rk_sel_o} |
{mr_add_rk_sel[i]});
323     key_full_sel_o = key_full_sel_e'({key_full_sel_o} |
{mr_key_full_sel[i]});
324     key_dec_sel_o = key_dec_sel_e'({key_dec_sel_o} |
{mr_key_dec_sel[i]});
325     key_words_sel_o = key_words_sel_e'({key_words_sel_o} |
{mr_key_words_sel[i]});
326     round_key_sel_o = round_key_sel_e'({round_key_sel_o} |
{mr_round_key_sel[i]});
327   end
328
329   for (int i = 0; i < Sp2VWidth; i++) begin
330     if (state_sel_o  != mr_state_sel[i] ||
331         add_rk_sel_o  != mr_add_rk_sel[i] ||
332         key_full_sel_o != mr_key_full_sel[i] ||
333         key_dec_sel_o != mr_key_dec_sel[i] ||
334         key_words_sel_o != mr_key_words_sel[i] ||
335         round_key_sel_o != mr_round_key_sel[i]) begin
336       mr_err = 1'b1;
337     end
338   end
339 end
340
341 // Collect errors in mux selector signals.
342 assign mux_sel_err = mux_sel_err_i | mr_err;
343
344 // Combine counter signals. We simply OR them together. If
the FSMs don't provide the same
345 // outputs, rnd_ctr_err will be set.
346 always_comb begin : combine_counter_signals
347   rnd_ctr      = '0;
348   rnd_ctr_err  = 1'b0;
349   for (int i = 0; i < Sp2VWidth; i++) begin
350     rnd_ctr |= mr_rnd_ctr[i];
351   end
352
353   for (int i = 0; i < Sp2VWidth; i++) begin
354     if (rnd_ctr != mr_rnd_ctr[i]) begin
355       rnd_ctr_err = 1'b1;
356     end
357   end
358 end
359
360 always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
361   if (!rst_ni) begin
362     prng_reseed_q <= 1'b0;
363     key_clear_q   <= 1'b0;
364     data_out_clear_q <= 1'b0;
365   end else begin
366     prng_reseed_q <= prng_reseed_d;
367     key_clear_q   <= key_clear_d;
368     data_out_clear_q <= data_out_clear_d;
369   end
370 end
371
372 // Use separate signal for key expand operation, forward
round.
373 assign key_expand_op_o = (dec_key_gen_d == SP2V_HIGH ||
dec_key_gen_q == SP2V_HIGH) ||
CiphOpFwdOnly ? CIPH_FWD : op_i;
375 assign key_expand_round_o = rnd_ctr;
376
377 // Let the main controller know what we are doing.
378 assign crypt_o          = crypt_q;
379 assign dec_key_gen_o    = dec_key_gen_q;
380 assign prng_reseed_o    = prng_reseed_q;
381 assign key_clear_o      = key_clear_q;
382 assign data_out_clear_o = data_out_clear_q;
383
384 ///////////////////////////////////////////////////
385 // Sparsely Encoded Signals //
386 ///////////////////////////////////////////////////
387
388 // SEC_CM: CTRL.SPARSE
389 // We use sparse encodings for various critical signals and
must ensure that:
391 // 1. The synthesis tool doesn't optimize away the sparse
encoding.
392 // 2. The sparsely encoded signal is always valid. More pre-
cisely, an alert or SVA is triggered
393 //    if a sparse signal takes on an invalid value.
394 // 3. The alert signal remains asserted until reset even if
the sparse signal becomes valid again
395 //    This is achieved by driving the control FSM into the
terminal error state whenever any
396 //    sparsely encoded signal becomes invalid.
397 //
398 // If any sparsely encoded signal becomes invalid, the
cipher core further immediately de-asserts
399 // the out_valid_o signal to prevent any data from being re-
leased.
400
401 // The following primitives are used to place a size-only
constraint on the
402 // flops in order to prevent optimizations on these status
signals.
403 logic [Sp2VWidth-1:0] crypt_q_raw;
404 caliptra_prim_flop #(
405   .Width      ( Sp2VWidth ),
406   .ResetValue ( Sp2VWidth'(SP2V_LOW) )
407 ) u_crypt_regs (
408   .clk_i ( clk_i ),
409   .rst_ni ( rst_ni ),
410   .d_i   ( crypt_d ),
411   .q_o   ( crypt_q_raw )
412 );
413
414 logic [Sp2VWidth-1:0] dec_key_gen_q_raw;
415 caliptra_prim_flop #(
416   .Width      ( Sp2VWidth ),
417   .ResetValue ( Sp2VWidth'(SP2V_LOW) )
418 ) u_dec_key_gen_regs (
419   .clk_i ( clk_i ),
420   .rst_ni ( rst_ni ),
421   .d_i   ( dec_key_gen_d ),
422   .q_o   ( dec_key_gen_q_raw )
423 );
424
425 // We use vectors of sparsely encoded signals to reduce code
duplication.
426 localparam int unsigned NumSp2VSig = 8;
427 sp2v_e [NumSp2VSig-1:0] sp2v_sig;
428 sp2v_e [NumSp2VSig-1:0] sp2v_sig_chk;
429 logic [NumSp2VSig-1:0][Sp2VWidth-1:0] sp2v_sig_chk_raw;
430 logic [NumSp2VSig-1:0] sp2v_sig_err;
431
432 assign sp2v_sig[0] = in_valid_i;
433 assign sp2v_sig[1] = out_ready_i;
434 assign sp2v_sig[2] = crypt_i;
435 assign sp2v_sig[3] = dec_key_gen_i;
436 assign sp2v_sig[4] = sp2v_e'(crypt_q_raw);
437 assign sp2v_sig[5] = sp2v_e'(dec_key_gen_q_raw);
438 assign sp2v_sig[6] = sub_bytes_out_req_i;
439 assign sp2v_sig[7] = key_expand_out_req_i;
440
441 // All signals inside sp2v_sig except for sub_bytes/key_ex-
pand_out_req_i are driven and consumed
442 // by multi-rail FSMs.
443 localparam bit [NumSp2VSig-1:0] Sp2VEnSecBuf = 8'b1100_0000;
444
445 // Individually check sparsely encoded signals.
446 for (genvar i = 0; i < NumSp2VSig; i++) begin :
gen_sel_buf_chk
447   aes_sel_buf_chk #(
448     .Num      ( Sp2VNum ),
449     .Width    ( Sp2VWidth ),
450     .EnSecBuf ( Sp2VEnSecBuf[i] )
451   ) u_aes_sp2v_sig_buf_chk_i (

```

```

452     .clk_i ( clk_i          ),
453     .rst_ni ( rst_ni        ),
454     .sel_i  ( sp2v_sig[i]    ),
455     .sel_o  ( sp2v_sig_chk_raw[i] ),
456     .err_o  ( sp2v_sig_err[i] )
457 );
458 assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
459 end
460
461 assign in_valid      = sp2v_sig_chk[0];
462 assign out_ready     = sp2v_sig_chk[1];
463 assign crypt         = sp2v_sig_chk[2];
464 assign dec_key_gen   = sp2v_sig_chk[3];
465 assign crypt_q       = sp2v_sig_chk[4];
466 assign dec_key_gen_q = sp2v_sig_chk[5];
467 assign sub_bytes_out_req = sp2v_sig_chk[6];
468 assign key_expand_out_req = sp2v_sig_chk[7];
469
470 // Collect encoding errors.
471 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
472 // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
473 assign sp_enc_err = |sp2v_sig_err | sp_enc_err_i;
474
475 // Assertions //
476 // Assertions //
477 // Assertions //
478
479 // Selectors must be known/valid
480 `CALIPTRA_ASSERT(AesCiphOpValid, cfg_valid_i |-> op_i inside {
481     CIPH_FWD,
482     CIPH_INV
483 })
484
485 endmodule

```

3. chipsalliance_caliptra-rtl/src/aes/rtl/ aes_cipher_control_fsm.sv (515 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM.
8
9 `include "caliptra_prim_assert.sv"
10
11 module aes_cipher_control_fsm import aes_pkg::*;
12 #(
13     parameter bit SecMasking = 0,
14     parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
15 ) (
16     input logic clk_i,
17     input logic rst_ni,
18
19     // Input handshake signals
20     input logic in_valid_i, // Sparsify
using multi-rail.
21     output logic in_ready_o, // Sparsify
using multi-rail.
22
23     // Output handshake signals
24     output logic out_valid_o, // Sparsify
using multi-rail.
25     input logic out_ready_i, // Sparsify
using multi-rail.
26
27     // Control and sync signals
28     input logic cfg_valid_i, // Used for
SVAs only.
29     input ciph_op_e op_i,
30     input key_len_e key_len_i,
31     input logic crypt_i, // Sparsify
using multi-rail.
32     input logic dec_key_gen_i, // Sparsify
using multi-rail.
33     input logic prng_reseed_i,
34     input logic key_clear_i,
35     input logic data_out_clear_i,
36     input logic mux_sel_err_i,
37     input logic sp_enc_err_i,
38     input logic rnd_ctr_err_i,
39     input logic op_err_i,
40     input logic alert_fatal_i,
41     output logic alert_o,
42
43     // Control signals for masking PRNG
44     output logic prng_update_o,
45     output logic prng_reseed_req_o,
46     input logic prng_reseed_ack_i,
47
48     // Control and sync signals for cipher data path
49     output state_sel_e state_sel_o,
50     output logic state_we_o, // Sparsify

```

```

using multi-rail.
51     output logic sub_bytes_en_o, // Sparsify
using multi-rail.
52     input logic sub_bytes_out_req_i, // Sparsify
using multi-rail.
53     output logic sub_bytes_out_ack_o, // Sparsify
using multi-rail.
54     output add_rk_sel_e add_rk_sel_o,
55
56     // Control and sync signals for key expand data path
57     output key_full_sel_e key_full_sel_o,
58     output logic key_full_we_o, // Sparsify
using multi-rail.
59     output key_dec_sel_e key_dec_sel_o,
60     output logic key_dec_we_o, // Sparsify
using multi-rail.
61     output logic key_expand_en_o, // Sparsify
using multi-rail.
62     input logic key_expand_out_req_i, // Sparsify
using multi-rail.
63     output logic key_expand_out_ack_o, // Sparsify
using multi-rail.
64     output logic key_expand_clear_o,
65     output logic [3:0] rnd_ctr_o,
66     output key_words_sel_e key_words_sel_o,
67     output round_key_sel_e round_key_sel_o,
68
69     // Register signals
70     input logic crypt_q_i, // Sparsify
using multi-rail.
71     output logic crypt_d_o, // Sparsify
using multi-rail.
72     input logic dec_key_gen_q_i, // Sparsify
using multi-rail.
73     output logic dec_key_gen_d_o, // Sparsify
using multi-rail.
74     input logic prng_reseed_q_i,
75     output logic prng_reseed_d_o,
76     input logic key_clear_q_i,
77     output logic key_clear_d_o,
78     input logic data_out_clear_q_i,
79     output logic data_out_clear_d_o
80 );
81
82 // cfg_valid_i is used for SVAs only.
83 logic unused_cfg_valid;
84 assign unused_cfg_valid = cfg_valid_i;
85
86 // Tie off unused inputs.
87 if (!SecMasking) begin : gen_unused_prng_reseed
88     logic unused_prng_reseed;
89     assign unused_prng_reseed = prng_reseed_i;
90 end
91
92 // Signals
93 aes_cipher_ctrl_e aes_cipher_ctrl_ns, aes_cipher_ctrl_cs;
94 logic advance;
95 logic [2:0] cyc_ctr_d, cyc_ctr_q;
96 logic cyc_ctr_expr;
97 logic prng_reseed_done_d, prng_reseed_done_q;
98 logic [3:0] rnd_ctr_d, rnd_ctr_q;
99 logic [3:0] num_rounds_d, num_rounds_q;
100 logic [3:0] num_rounds_regular;
101
102 // Use separate signal for number of regular rounds.
103 assign num_rounds_regular = num_rounds_q - 4'd1;
104
105 // FSM
106 always_comb begin : aes_cipher_ctrl_fsm
107
108     // Handshake signals
109     in_ready_o = 1'b0;
110     out_valid_o = 1'b0;
111
112     // Masking PRNG signals
113     prng_update_o = 1'b0;
114     prng_reseed_req_o = 1'b0;
115
116     // Cipher data path
117     state_sel_o = STATE_ROUND;
118     state_we_o = 1'b0;
119     add_rk_sel_o = ADD_RK_ROUND;
120     sub_bytes_en_o = 1'b0;
121     sub_bytes_out_ack_o = 1'b0;
122
123     // Key expand data path
124     key_full_sel_o = KEY_FULL_ROUND;
125     key_full_we_o = 1'b0;
126     key_dec_sel_o = KEY_DEC_EXPAND;
127     key_dec_we_o = 1'b0;
128     key_expand_en_o = 1'b0;
129     key_expand_out_ack_o = 1'b0;
130     key_expand_clear_o = 1'b0;
131     key_words_sel_o = KEY_WORDS_ZERO;
132     round_key_sel_o = ROUND_KEY_DIRECT;
133
134     // FSM
135     aes_cipher_ctrl_ns = aes_cipher_ctrl_cs;
136     num_rounds_d = num_rounds_q;
137     rnd_ctr_d = rnd_ctr_q;
138     crypt_d_o = crypt_q_i;

```



```

139 dec_key_gen_d_o = dec_key_gen_q_i;
140 prng_reseed_d_o = prng_reseed_q_i;
141 key_clear_d_o = key_clear_q_i;
142 data_out_clear_d_o = data_out_clear_q_i;
143 prng_reseed_done_d = prng_reseed_done_q | prng_re-
seed_ack_i;
144 advance = 1'b0;
145 cyc_ctr_d = (SecSBoxImpl == SBoxImplDom) ?
cyc_ctr_q + 3'd1 : 3'd0;
146
147 // Alert
148 alert_o = 1'b0;
149
150 unique case (aes_cipher_ctrl_cs)
151
152 CIPHER_CTRL_IDLE: begin
153     cyc_ctr_d = 3'd0;
154
155     // Signal that we are ready, wait for handshake.
156     in_ready_o = 1'b1;
157     if (in_valid_i) begin
158         if (SecMasking && prng_reseed_i && !dec_key_gen_i
&& !crypt_i) begin
159             // Reseed the masking PRNG without starting en-
cryption/decryption or generation of the
160             // start key for decryption.
161             prng_reseed_d_o = 1'b1;
162             prng_reseed_done_d = 1'b0;
163             aes_cipher_ctrl_ns = CIPHER_CTRL_PRNG_RESEED;
164
165         end else if (key_clear_i || data_out_clear_i) begin
166             // Clear internal key registers. The cipher core
muxes are used to
167             // clear the data
168             // output registers.
169             key_clear_d_o = key_clear_i;
170             data_out_clear_d_o = data_out_clear_i;
171
172             // To clear the data output registers, we must
first clear the state.
173             aes_cipher_ctrl_ns = data_out_clear_i ?
CIPHER_CTRL_CLEAR_S : CIPHER_CTRL_CLEAR_KD;
174
175             end else if (dec_key_gen_i || crypt_i) begin
176                 // Start encryption/decryption or generation of
start key for decryption.
177                 crypt_d_o = ~dec_key_gen_i & crypt_i;
178                 dec_key_gen_d_o = dec_key_gen_i;
179
180                 // Latch whether we shall reseed the masking PRNG.
181                 prng_reseed_d_o = SecMasking & prng_reseed_i;
182
183                 // Load input data to state
184                 state_sel_o = dec_key_gen_i ? STATE_CLEAR :
STATE_INIT;
185
186                 state_we_o = 1'b1;
187
188                 // Make the masking PRNG advance. The current
pseudo-random data
189                 // is used to mask the
190                 // input data.
191                 prng_update_o = SecMasking;
192
193                 // Init key expand
194                 key_expand_clear_o = 1'b1;
195
196                 // Load full key
197                 key_full_sel_o = dec_key_gen_i ?
KEY_FULL_ENC_INIT :
198                 (op_i == CIPH_FWD) ?
KEY_FULL_ENC_INIT :
199                 (op_i == CIPH_INV) ?
KEY_FULL_DEC_INIT :
200                 (op_i == CIPH_INV) ?
KEY_FULL_ENC_INIT;
201
202                 key_full_we_o = 1'b1;
203
204                 // Load num_rounds, initialize round counter.
205                 num_rounds_d = (key_len_i == AES_128) ? 4'd10 :
206                 (key_len_i == AES_192) ? 4'd12 :
207                 (key_len_i == AES_256) ? 4'd14;
208
209                 rnd_ctr_d = 0;
210                 aes_cipher_ctrl_ns = CIPHER_CTRL_INIT;
211
212             end else begin
213                 // Handshake without a valid command. We should
never get here. If we do (e.g. via a
214                 // malicious glitch), error out immediately.
215                 aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
216             end
217         end
218     end
219
220 CIPHER_CTRL_INIT: begin
221     // Initial round: just add key to state
222     add_rk_sel_o = ADD_RK_INIT;
223
224     // Select key words for initial add_round_key
225     key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
226     (key_len_i == AES_128) ?
KEY_WORDS_0123 :
227     (key_len_i == AES_192 && op_i == CIPH_FWD) ?

```

```

KEY_WORDS_0123 :
228     (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_2345 :
229     (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_0123 :
230     (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_4567 : KEY_WORDS_ZERO;
231
232 // Clear masking PRNG reseed status.
233 prng_reseed_done_d = 1'b0;
234
235 // AES-256 has two round keys available right from be-
ginning. Pseudo-random data is
236 // required by KeyExpand only.
237 if (key_len_i != AES_256) begin
238     // Advance in sync with KeyExpand. Based on the S-
Box implementation, it can take
239     // multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
240     // only in the first clock cycle and that PRD is
taken from the buffer stage updated
241     // based on key_full_we_o. The PRNG itself is up-
dated in every clock cycle to increase
242     // the noise.
243     advance = key_expand_out_req_i & cyc_ctr_ex-
pr;
244
245     prng_update_o = SecMasking;
246     key_expand_en_o = 1'b1;
247     if (advance) begin
248         key_expand_out_ack_o = 1'b1;
249         state_we_o = ~dec_key_gen_q_i;
250         key_full_we_o = 1'b1;
251         rnd_ctr_d = rnd_ctr_q + 4'b0001;
252         cyc_ctr_d = 3'd0;
253         aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
254     end
255 end else begin
256     prng_update_o = SecMasking;
257     state_we_o = ~dec_key_gen_q_i;
258     rnd_ctr_d = rnd_ctr_q + 4'b0001;
259     cyc_ctr_d = 3'd0;
260     aes_cipher_ctrl_ns = CIPHER_CTRL_ROUND;
261 end
262
263 CIPHER_CTRL_ROUND: begin
264     // Normal rounds
265
266     // Select key words for add_round_key
267     key_words_sel_o = (dec_key_gen_q_i) ?
KEY_WORDS_ZERO :
268     (key_len_i == AES_128) ?
KEY_WORDS_0123 :
269     (key_len_i == AES_192 && op_i == CIPH_FWD) ?
KEY_WORDS_2345 :
270     (key_len_i == AES_192 && op_i == CIPH_INV) ?
KEY_WORDS_0123 :
271     (key_len_i == AES_256 && op_i == CIPH_FWD) ?
KEY_WORDS_4567 :
272     (key_len_i == AES_256 && op_i == CIPH_INV) ?
KEY_WORDS_0123 : KEY_WORDS_ZERO;
273
274 // Keep requesting PRNG reseeding until it is acknow-
ledged.
275 prng_reseed_req_o = SecMasking & prng_reseed_q_i &
~prng_reseed_done_q;
276
277 // Select round key: direct or mixed (equivalent in-
verse cipher)
278 round_key_sel_o = (op_i == CIPH_FWD) ?
ROUND_KEY_DIRECT :
279     (op_i == CIPH_INV) ?
ROUND_KEY_MIXED : ROUND_KEY_DIRECT;
280
281 // Advance in sync with SubBytes and KeyExpand. Based
on the S-Box implementation, both can
282 // take multiple cycles to finish. Wait for handshake.
The DOM S-Boxes consume fresh PRD
283 // only in the first clock cycle and that PRD is taken
from the buffer stages updated
284 // with state_we_o / based on key_full_we_o. The PRNG
itself is updated in every clock
285 // cycle to increase the noise.
286 advance = key_expand_out_req_i & cyc_ctr_expr &
(dec_key_gen_q_i | sub_bytes_out_req_i);
287 prng_update_o = SecMasking;
288 sub_bytes_en_o = ~dec_key_gen_q_i;
289 key_expand_en_o = 1'b1;
290
291 if (advance) begin
292     sub_bytes_out_ack_o = ~dec_key_gen_q_i;
293     key_expand_out_ack_o = 1'b1;
294
295     state_we_o = ~dec_key_gen_q_i;
296     key_full_we_o = 1'b1;
297
298     // Update round
299     rnd_ctr_d = rnd_ctr_q + 4'b0001;
300     cyc_ctr_d = 3'd0;

```

```

296
297 // Are we doing the last regular round?
298 if (rnd_ctr_q >= num_rounds_regular) begin
299     aes_cipher_ctrl_ns = CIPHER_CTRL_FINISH;
300
301     if (dec_key_gen_q_i) begin
302         // Write decryption key.
303         key_dec_we_o = 1'b1;
304     end
305
306 // Indicate that we are done, try to perform the
307 handshake. But we don't wait here.
308 // If we don't get the handshake now, we will
309 wait in the finish state. When using
310 // masking, we only finish if the masking PRNG
311 has been reseeded.
312 out_valid_o = SecMasking ? (prng_reseed_q_i ?
313 prng_reseed_done_q : 1'b1) : 1'b1;
314 if (out_valid_o && out_ready_i) begin
315     // Go to idle state directly.
316     dec_key_gen_d_o = 1'b0;
317     prng_reseed_d_o = 1'b0;
318     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
319 end
320 end
321 end // rnd_ctr_q
322 end // SubBytes/KeyExpand REQ/ACK
323
324 CIPHER_CTRL_FINISH: begin
325     // Final round
326     // Select key words for add_round_key
327     key_words_sel_o = (dec_key_gen_q_i) ?
328 KEY_WORDS_ZERO :
329 (key_len_i == AES_128) ?
330 KEY_WORDS_0123 :
331 (key_len_i == AES_192 && op_i == CIPH_FWD) ?
332 KEY_WORDS_2345 :
333 (key_len_i == AES_192 && op_i == CIPH_INV) ?
334 KEY_WORDS_0123 :
335 (key_len_i == AES_256 && op_i == CIPH_FWD) ?
336 KEY_WORDS_4567 :
337 (key_len_i == AES_256 && op_i == CIPH_INV) ?
338 KEY_WORDS_0123 : KEY_WORDS_ZERO;
339
340 // Skip mix_columns
341 add_rk_sel_o = ADD_RK_FINAL;
342
343 // Keep requesting PRNG reseeding until it is acknow-
344 ledged.
345 prng_reseed_req_o = SecMasking & prng_reseed_q_i &
346 ~prng_reseed_done_q;
347
348 // SEC_CM: DATA_REG.KEY.SCA
349 // Once we're done, we won't need the state anymore.
350 We actually clear it when progressing
351 // to the next state.
352 state_sel_o = STATE_CLEAR;
353
354 // SEC_CM: DATA_REG.LOCAL_ESC
355 // Advance in sync with SubBytes. Based on the S-Box
356 implementation, it can take multiple
357 // cycles to finish. Only indicate that we are done
358 if:
359 // - we have valid output (SubBytes finished),
360
361 // - the masking PRNG has been reseeded (if masking is
362 used),
363
364 // - all mux selector signals are valid (don't release
365 data in case of errors), and
366 // - all sparsely encoded signals are valid (don't re-
367 lease data in case of errors).
368 // Perform both handshakes simultaneously.
369 advance = (sub_bytes_out_req_i & cyc_ctr_expr)
370 | dec_key_gen_q_i;
371 sub_bytes_en_o = ~dec_key_gen_q_i;
372 out_valid_o = (mux_sel_err_i || sp_enc_err_i ||
373 op_err_i) ? 1'b0 :
374
375 SecMasking ? (prng_reseed_q_i ? prng_reseed_done_q &
376 advance : advance) : advance;
377
378 // Stop updating the cycle counter once we have valid
379 output.
380 cyc_ctr_d =
381 (SecSBoxImpl == SBoxImplDom) ? (!advance ?
382 cyc_ctr_q + 3'd1 : cyc_ctr_q) : 3'd0;
383
384
385 // The DOM S-Boxes consume fresh PRD only in the first
386 clock cycle and that PRD is taken
387 // from the buffer stages updated with state_we_o /
388 based on key_full_we_o. The PRNG itself
389 // is updated in every but the last processing clock
390 cycle to increase the noise. Once the
391 // processing is all done (e.g. if we're just waiting
392 for the PRNG reseeding to finish or
393 // if we're waiting for out_ready_i), the PRNG is no
394 longer updated to save power. In the
395
396 // very last clock cycle, we update the PRNG again to
397 get ready for the next block.
398 prng_update_o =
399
400 ((SecSBoxImpl == SBoxImplDom) ? !advance : 1'b0) |
401 (out_valid_o & out_ready_i);
402
403 if (out_valid_o && out_ready_i) begin
404     sub_bytes_out_ack_o = ~dec_key_gen_q_i;
405
406 // Clear the state.
407 state_we_o = 1'b1;
408 crypt_d_o = 1'b0;
409 cyc_ctr_d = 3'd0;
410 // If we were generating the decryption key and
411 didn't get the handshake in the last
412 // regular round, we should clear dec_key_gen now.
413 dec_key_gen_d_o = 1'b0;
414 prng_reseed_d_o = 1'b0;
415 aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
416 end
417 end
418
419 CIPHER_CTRL_PRNG_RESEED: begin
420 // Keep requesting PRNG reseeding until it is acknow-
421 ledged.
422 prng_reseed_req_o = prng_reseed_q_i & ~prng_re-
423 seed_done_q;
424
425 // Don't update the cycle counter as we don't need it.
426 cyc_ctr_d = 3'd0;
427
428 // Once we're done, wait for handshake.
429 out_valid_o = prng_reseed_done_q;
430 if (out_valid_o && out_ready_i) begin
431     prng_reseed_d_o = 1'b0;
432     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
433 end
434 end
435
436 CIPHER_CTRL_CLEAR_S: begin
437 // Clear the state with pseudo-random data.
438 state_we_o = 1'b1;
439 state_sel_o = STATE_CLEAR;
440 aes_cipher_ctrl_ns = CIPHER_CTRL_CLEAR_KD;
441 end
442
443 CIPHER_CTRL_CLEAR_KD: begin
444 // Clear internal key registers and/or external data
445 output registers.
446 if (key_clear_q_i) begin
447     key_full_sel_o = KEY_FULL_CLEAR;
448     key_full_we_o = 1'b1;
449     key_dec_sel_o = KEY_DEC_CLEAR;
450     key_dec_we_o = 1'b1;
451 end
452 if (data_out_clear_q_i) begin
453     // Forward the state (previously cleared with
454 psuedo-random data).
455     // SEC_CM: DATA_REG.SEC_WIPE
456     add_rk_sel_o = ADD_RK_INIT;
457     key_words_sel_o = KEY_WORDS_ZERO;
458     round_key_sel_o = ROUND_KEY_DIRECT;
459 end
460 // Indicate that we are done, wait for handshake.
461 out_valid_o = 1'b1;
462 if (out_ready_i) begin
463     key_clear_d_o = 1'b0;
464     data_out_clear_d_o = 1'b0;
465     aes_cipher_ctrl_ns = CIPHER_CTRL_IDLE;
466 end
467 end
468
469 CIPHER_CTRL_ERROR: begin
470 // SEC_CM: CIPHER.FSM.LOCAL_ESC
471 // Terminal error state
472 alert_o = 1'b1;
473 end
474
475 // We should never get here. If we do (e.g. via a mali-
476 cious glitch), error out immediately.
477 default: begin
478     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
479     alert_o = 1'b1;
480 end
481 endcase
482
483 // Unconditionally jump into the terminal error state in
484 case a mux selector or a sparsely
485 // encoded signal becomes invalid, in case we have detec-
486 ted a fault in the round counter,
487 // or if a fatal alert has been triggered.
488 if (mux_sel_err_i || sp_enc_err_i || rnd_ctr_err_i ||
489 op_err_i || alert_fatal_i) begin
490     aes_cipher_ctrl_ns = CIPHER_CTRL_ERROR;
491 end
492 end
493
494 // SEC_CM: CIPHER.FSM.SPARSE
495 `CALIPTRA_PRIM_FLOP_SPARSE_FSM(u_state_regs,
496 aes_cipher_ctrl_ns,

```



```

452     aes_cipher_ctrl_cs, aes_cipher_ctrl_e, CIPHER_CTRL_IDLE)
453
454     always_ff @(posedge clk_i or negedge rst_ni) begin : reg_fsm
455         if (!rst_ni) begin
456             prng_reseed_done_q <= 1'b0;
457             rnd_ctr_q <= '0;
458             num_rounds_q <= '0;
459         end else begin
460             prng_reseed_done_q <= prng_reseed_done_d;
461             rnd_ctr_q <= rnd_ctr_d;
462             num_rounds_q <= num_rounds_d;
463         end
464     end
465
466     assign rnd_ctr_o = rnd_ctr_q;
467
468     if (SecSBoxImpl == SBoxImplDom) begin : gen_cyc_ctr
469         always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_cyc_ctr
470             if (!rst_ni) begin
471                 cyc_ctr_q <= 3'd0;
472             end else begin
473                 cyc_ctr_q <= cyc_ctr_d;
474             end
475         end
476         assign cyc_ctr_expr = cyc_ctr_q >= 3'd4;
477     end else begin : gen_no_cyc_ctr
478         logic [2:0] unused_cyc_ctr;
479         assign cyc_ctr_q = cyc_ctr_d;
480         assign unused_cyc_ctr = cyc_ctr_q;
481         assign cyc_ctr_expr = 1'b1;
482     end
483
484     //////////////////////////////////////////////////
485     // Assertions //
486     //////////////////////////////////////////////////
487
488     // Create a lint error to reduce the risk of accidentally
489     // disabling the masking.
490     `CALIPT-
RA_ASSERT_STATIC_LINT_ERROR(AesCipherControlFsmSecMaskingNonDefault,
SecMasking == 1)
491
492     // Create a lint error to reduce the risk of accidentally
493     // using a less secure SBox
494     // implementation.
495     `CALIPT-
RA_ASSERT_STATIC_LINT_ERROR(AesCipherControlFsmSecSBoxImplNonDefault
, SecSBoxImpl == SBoxImplDom)
496
497     // Selectors must be known/valid
498
499     `CALIPTRA_ASSERT(AesCiphOpValid, cfg_valid_i |-> op_i inside {
500         CIPH_FWD,
501         CIPH_INV
502     })
503     `CALIPTRA_ASSERT(AesKeyLenValid, cfg_valid_i |-> key_len_i
inside {
504         AES_128,
505         AES_192,
506         AES_256
507     })
508     `CALIPTRA_ASSERT(AesCipherControlStateValid, !alert_o |->
aes_cipher_ctrl_cs inside {
509         CIPHER_CTRL_IDLE,
510         CIPHER_CTRL_INIT,
511         CIPHER_CTRL_ROUND,
512         CIPHER_CTRL_FINISH,
513         CIPHER_CTRL_PRNG_RESEED,
514         CIPHER_CTRL_CLEAR_S,
515         CIPHER_CTRL_CLEAR_KD
516     })
517 endmodule

```

4. chipsalliance_caliptra-rtl/src/aes/rtl/ aes_cipher_control_fsm_n.sv (400 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: Apache-2.0
5 //
6 // AES cipher core control FSM
7 //
8 // This module contains the AES cipher core control FSM oper-
9 // ating on and producing the negated
10 // values of important control signals. This is achieved by:
11 // - instantiating the regular AES cipher core control FSM op-
12 // erating on and producing the positive
13 // values of these signals, and
14 // - inverting these signals between the regular FSM and the
15 // caliptra_prim_buf synthesis barriers.
16 // Synthesis tools will then push the inverters into the actu-
17 // al FSM.
18
19 module aes_cipher_control_fsm_n import aes_pkg::*;
20 #(
21     parameter bit SecMasking = 0,

```

```

17     parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
18 ) (
19     input logic clk_i,
20     input logic rst_ni,
21
22     // Input handshake signals
23     input logic in_valid_ni, // Sparsify
24     using multi-rail.
25     output logic in_ready_no, // Sparsify
26     using multi-rail.
27
28     // Output handshake signals
29     output logic out_valid_no, // Sparsify
30     using multi-rail.
31     input logic out_ready_ni, // Sparsify
32     using multi-rail.
33
34     // Control and sync signals
35     input logic cfg_valid_i, // Used for
36     SVAs only.
37     input ciph_op_e op_i,
38     input key_len_e key_len_i,
39     input logic crypt_ni, // Sparsify
40     using multi-rail.
41     input logic dec_key_gen_ni, // Sparsify
42     using multi-rail.
43
44     input logic prng_reseed_i,
45     input logic key_clear_i,
46     input logic data_out_clear_i,
47     input logic mux_sel_err_i,
48     input logic sp_enc_err_i,
49     input logic rnd_ctr_err_i,
50     input logic op_err_i,
51     input logic alert_fatal_i,
52     output logic alert_o,
53
54     // Control signals for masking PRNG
55     output logic prng_update_o,
56     output logic prng_reseed_req_o,
57     input logic prng_reseed_ack_i,
58
59     // Control and sync signals for cipher data path
60     output state_sel_e state_sel_o,
61     output logic state_we_no, // Sparsify
62     using multi-rail.
63     output logic sub_bytes_en_no, // Sparsify
64     using multi-rail.
65     input logic sub_bytes_out_req_ni, // Sparsify
66     using multi-rail.
67     output logic sub_bytes_out_ack_no, // Sparsify
68     using multi-rail.
69     output add_rk_sel_e add_rk_sel_o,
70
71     // Control and sync signals for key expand data path
72     output key_full_sel_e key_full_sel_o,
73     output logic key_full_we_no, // Sparsify
74     using multi-rail.
75     output key_dec_sel_e key_dec_sel_o,
76     output logic key_dec_we_no, // Sparsify
77     using multi-rail.
78     output logic key_expand_en_no, // Sparsify
79     using multi-rail.
80     input logic key_expand_out_req_ni, // Sparsify
81     using multi-rail.
82     output logic key_expand_out_ack_no, // Sparsify
83     using multi-rail.
84     output logic key_expand_clear_o,
85     output logic [3:0] rnd_ctr_o,
86     output key_words_sel_e key_words_sel_o,
87     output round_key_sel_e round_key_sel_o,
88
89     // Register signals
90     input logic crypt_q_ni, // Sparsify
91     using multi-rail.
92     output logic crypt_d_no, // Sparsify
93     using multi-rail.
94     input logic dec_key_gen_q_ni, // Sparsify
95     using multi-rail.
96     output logic dec_key_gen_d_no, // Sparsify
97     using multi-rail.
98     input logic prng_reseed_q_i,
99     output logic prng_reseed_d_o,
100     input logic key_clear_q_i,
101     output logic key_clear_d_o,
102     input logic data_out_clear_q_i,
103     output logic data_out_clear_d_o
104 );
105
106 //////////////////////////////////////////////////
107 // Input Buffering //
108 //////////////////////////////////////////////////
109
110 localparam int NumInBufBits = $bits({
111     in_valid_ni,
112     out_ready_ni,
113     cfg_valid_i,
114     op_i,
115     key_len_i,
116     crypt_ni,
117     dec_key_gen_ni,
118     prng_reseed_i,

```

```

98   key_clear_i,
99   data_out_clear_i,
100  mux_sel_err_i,
101  sp_enc_err_i,
102  rnd_ctr_err_i,
103  op_err_i,
104  alert_fatal_i,
105  prng_reseed_ack_i,
106  sub_bytes_out_req_ni,
107  key_expand_out_req_ni,
108  crypt_q_ni,
109  dec_key_gen_q_ni,
110  prng_reseed_q_i,
111  key_clear_q_i,
112  data_out_clear_q_i
113  });
114
115  logic [NumInBufBits-1:0] in, in_buf;
116
117  assign in = {
118    in_valid_ni,
119    out_ready_ni,
120    cfg_valid_i,
121    op_i,
122    key_len_i,
123    crypt_ni,
124    dec_key_gen_ni,
125    prng_reseed_i,
126    key_clear_i,
127    data_out_clear_i,
128    mux_sel_err_i,
129    sp_enc_err_i,
130    rnd_ctr_err_i,
131    op_err_i,
132    alert_fatal_i,
133    prng_reseed_ack_i,
134    sub_bytes_out_req_ni,
135    key_expand_out_req_ni,
136    crypt_q_ni,
137    dec_key_gen_q_ni,
138    prng_reseed_q_i,
139    key_clear_q_i,
140    data_out_clear_q_i
141  };
142
143  // This primitive is used to place a size-only constraint on
the
144  // buffers to act as a synthesis optimization barrier.
145  caliptra_prim_buf #(
146    .Width(NumInBufBits)
147  ) u_caliptra_prim_buf_in (
148    .in_i(in),
149    .out_o(in_buf)
150  );
151
152  logic          in_valid_n;
153  logic          out_ready_n;
154  logic          cfg_valid;
155  ciph_op_e      op;
156  logic [$bits(op)-1:0] op_raw;
157  key_len_e      key_len;
158  logic          crypt_n;
159  logic          dec_key_gen_n;
160  logic          prng_reseed;
161  logic          key_clear;
162  logic          data_out_clear;
163  logic          mux_sel_err;
164  logic          sp_enc_err;
165  logic          rnd_ctr_err;
166  logic          op_err;
167  logic          alert_fatal;
168  logic          prng_reseed_ack;
169  logic          sub_bytes_out_req_n;
170  logic          key_expand_out_req_n;
171  logic          crypt_q_n;
172  logic          dec_key_gen_q_n;
173  logic          prng_reseed_q;
174  logic          key_clear_q;
175  logic          data_out_clear_q;
176
177  assign {in_valid_n,
178    out_ready_n,
179    cfg_valid,
180    op_raw,
181    key_len,
182    crypt_n,
183    dec_key_gen_n,
184    prng_reseed,
185    key_clear,
186    data_out_clear,
187    mux_sel_err,
188    sp_enc_err,
189    rnd_ctr_err,
190    op_err,
191    alert_fatal,
192    prng_reseed_ack,
193    sub_bytes_out_req_n,
194    key_expand_out_req_n,
195    crypt_q_n,
196    dec_key_gen_q_n,
197    prng_reseed_q,
198    key_clear_q,
199    data_out_clear_q} = in_buf;
200
201  assign op = ciph_op_e'(op_raw);
202
203  // Intermediate output signals
204  logic          in_ready;
205  logic          out_valid;
206  logic          alert;
207  logic          prng_update;
208  logic          prng_reseed_req;
209  state_sel_e    state_sel;
210  logic          state_we;
211  logic          sub_bytes_en;
212  logic          sub_bytes_out_ack;
213  add_rk_sel_e   add_rk_sel;
214  key_full_sel_e key_full_sel;
215  logic          key_full_we;
216  key_dec_sel_e  key_dec_sel;
217  logic          key_dec_we;
218  logic          key_expand_en;
219  logic          key_expand_out_ack;
220  logic          key_expand_clear;
221  key_words_sel_e key_words_sel;
222  round_key_sel_e round_key_sel;
223  logic [3:0]    rnd_ctr;
224  logic          crypt_d;
225  logic          dec_key_gen_d;
226  logic          prng_reseed_d;
227  logic          key_clear_d;
228  logic          data_out_clear_d;
229
230  ///////////////////////////////////////////////////
231  // Regular FSM //
232  ///////////////////////////////////////////////////
233
234  // The regular FSM operates on and produces the positive
values of important control signals.
235  // Invert *_n input signals here to get the positive values
for the regular FSM. To obtain the
236  // negated outputs, important output signals are inverted
further below. Thanks to the caliptra_prim_buf
237  // synthesis optimization barriers, tools will push the in-
verters into the regular FSM.
238  aes_cipher_control_fsm #(
239    .SecMasking ( SecMasking ),
240    .SecSBoxImpl ( SecSBoxImpl )
241  ) u_aes_cipher_control_fsm (
242    .clk_i          ( clk_i          ),
243    .rst_ni         ( rst_ni         ),
244    .in_valid_i     ( ~in_valid_n    ), // In-
vert for regular FSM.
245    .in_ready_o     ( in_ready       ), // In-
vert below for negated output.
246    .out_valid_o    ( out_valid      ), // In-
vert below for negated output.
247    .out_ready_i    ( ~out_ready_n   ), // In-
vert for regular FSM.
248    .cfg_valid_i    ( cfg_valid      ),
249    .op_i           ( op             ),
250    .key_len_i      ( key_len        ),
251    .crypt_i        ( ~crypt_n       ), // In-
vert for regular FSM.
252    .dec_key_gen_i  ( ~dec_key_gen_n ), // In-
vert for regular FSM.
253    .prng_reseed_i  ( prng_reseed    ),
254    .key_clear_i    ( key_clear      ),
255    .data_out_clear_i ( data_out_clear ),
256    .mux_sel_err_i  ( mux_sel_err    ),
257    .sp_enc_err_i   ( sp_enc_err     ),
258    .rnd_ctr_err_i  ( rnd_ctr_err    ),
259    .op_err_i       ( op_err         ),
260    .alert_fatal_i  ( alert_fatal    ),
261    .alert_o        ( alert          ),
262    .prng_update_o  ( prng_update    ),
263    .prng_reseed_req_o ( prng_reseed_req ),
264    .prng_reseed_ack_i ( prng_reseed_ack ),
265    .state_sel_o    ( state_sel      ),
266    .state_we_o     ( state_we       ), // In-
vert below for negated output.
267    .sub_bytes_en_o ( sub_bytes_en   ), // In-
vert below for negated output.
268    .sub_bytes_out_req_i ( ~sub_bytes_out_req_n ), // In-
vert for regular FSM.
269    .sub_bytes_out_ack_o ( sub_bytes_out_ack ), // In-
vert below for negated output.
270    .add_rk_sel_o   ( add_rk_sel     ),
271    .key_full_sel_o ( key_full_sel    ),
272    .key_full_we_o  ( key_full_we    ), // In-
vert below for negated output.
273    .key_dec_sel_o  ( key_dec_sel     ),
274    .key_dec_we_o   ( key_dec_we     ), // In-
vert below for negated output.
275    .key_expand_en_o ( key_expand_en  ), // In-

```

```

vert below for negated output.
282 .key_expand_out_req_i ( ~key_expand_out_req_n ), // In-
vert for regular FSM.
283 .key_expand_out_ack_o ( key_expand_out_ack ), // In-
vert below for negated output.
284 .key_expand_clear_o ( key_expand_clear ),
285 .rnd_ctr_o ( rnd_ctr ),
286 .key_words_sel_o ( key_words_sel ),
287 .round_key_sel_o ( round_key_sel ),
288
289 .crypt_q_i ( ~crypt_q_n ), // In-
vert for regular FSM.
290 .crypt_d_o ( crypt_d ), // In-
vert below for negated output.
291 .dec_key_gen_q_i ( ~dec_key_gen_q_n ), // In-
vert for regular FSM.
292 .dec_key_gen_d_o ( dec_key_gen_d ), // In-
vert below for negated output.
293 .key_clear_q_i ( key_clear_q ),
294 .key_clear_d_o ( key_clear_d ),
295 .prng_reseed_q_i ( prng_reseed_q ),
296 .prng_reseed_d_o ( prng_reseed_d ),
297 .data_out_clear_q_i ( data_out_clear_q ),
298 .data_out_clear_d_o ( data_out_clear_d );
299
300
301 //////////////////////////////////////////////////
302 // Output Buffering //
303 //////////////////////////////////////////////////
304
305 localparam int NumOutBufBits = $bits({
306   in_ready_no,
307   out_valid_no,
308   alert_o,
309   prng_update_o,
310   prng_reseed_req_o,
311   state_sel_o,
312   state_we_no,
313   sub_bytes_en_no,
314   sub_bytes_out_ack_no,
315   add_rk_sel_o,
316   key_full_sel_o,
317   key_full_we_no,
318   key_dec_sel_o,
319   key_dec_we_no,
320   key_expand_en_no,
321   key_expand_out_ack_no,
322   key_expand_clear_o,
323   rnd_ctr_o,
324   key_words_sel_o,
325   round_key_sel_o,
326   crypt_d_no,
327   dec_key_gen_d_no,
328   key_clear_d_o,
329   prng_reseed_d_o,
330   data_out_clear_d_o
331 });
332
333 logic [NumOutBufBits-1:0] out, out_buf;
334
335 // Important output control signals need to be inverted
336 here. Synthesis tools will push the
337 // inverters back into the regular FSM.
338 assign out = {
339   ~in_ready,
340   ~out_valid,
341   alert,
342   prng_update,
343   prng_reseed_req,
344   state_sel,
345   ~state_we,
346   ~sub_bytes_en,
347   ~sub_bytes_out_ack,
348   add_rk_sel,
349   key_full_sel,
350   ~key_full_we,
351   key_dec_sel,
352   ~key_dec_we,
353   ~key_expand_en,
354   ~key_expand_out_ack,
355   key_expand_clear,
356   rnd_ctr,
357   key_words_sel,
358   round_key_sel,
359   ~crypt_d,
360   ~dec_key_gen_d,
361   key_clear_d,
362   prng_reseed_d,
363   data_out_clear_d
364 };
365
366 // This primitive is used to place a size-only constraint on
367 the
368 // buffers to act as a synthesis optimization barrier.
369 caliptra_prim_buf #(
370   .Width(NumOutBufBits)
371 ) u_caliptra_prim_buf_out (
372   .in_i(out),
373   .out_o(out_buf)
374 );

```

```

373
374 assign {in_ready_no,
375   out_valid_no,
376   alert_o,
377   prng_update_o,
378   prng_reseed_req_o,
379   state_sel_o,
380   state_we_no,
381   sub_bytes_en_no,
382   sub_bytes_out_ack_no,
383   add_rk_sel_o,
384   key_full_sel_o,
385   key_full_we_no,
386   key_dec_sel_o,
387   key_dec_we_no,
388   key_expand_en_no,
389   key_expand_out_ack_no,
390   key_expand_clear_o,
391   rnd_ctr_o,
392   key_words_sel_o,
393   round_key_sel_o,
394   crypt_d_no,
395   dec_key_gen_d_no,
396   key_clear_d_o,
397   prng_reseed_d_o,
398   data_out_clear_d_o} = out_buf;
399
400 endmodule

```

5. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_control_fsm_p.sv (390 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core control FSM
6 //
7 // This module contains the AES cipher core control FSM oper-
  ating on
8 // and producing the positive values of important control sig-
  nals.
9
10 module aes_cipher_control_fsm_p import aes_pkg::*;
11 #(
12   parameter bit      SecMasking = 0,
13   parameter sbox_impl_e SecSBoxImpl = SBoxImplDom
14 ) (
15   input logic         clk_i,
16   input logic         rst_ni,
17
18   // Input handshake signals
19   input logic         in_valid_i,           // Sparsify
  using multi-rail.
20   output logic        in_ready_o,          // Sparsify
  using multi-rail.
21
22   // Output handshake signals
23   output logic        out_valid_o,         // Sparsify
  using multi-rail.
24   input logic         out_ready_i,         // Sparsify
  using multi-rail.
25
26   // Control and sync signals
27   input logic         cfg_valid_i,         // Used for
  SVAs only.
28   input ciph_op_e     op_i,
29   input key_len_e     key_len_i,
30   input logic         crypt_i,            // Sparsify
  using multi-rail.
31   input logic         dec_key_gen_i,      // Sparsify
  using multi-rail.
32   input logic         prng_reseed_i,
33   input logic         key_clear_i,
34   input logic         data_out_clear_i,
35   input logic         mux_sel_err_i,
36   input logic         sp_enc_err_i,
37   input logic         rnd_ctr_err_i,
38   input logic         op_err_i,
39   input logic         alert_fatal_i,
40   output logic        alert_o,
41
42   // Control signals for masking PRNG
43   output logic        prng_update_o,
44   output logic        prng_reseed_req_o,
45   input logic         prng_reseed_ack_i,
46
47   // Control and sync signals for cipher data path
48   output state_sel_e  state_sel_o,
49   output logic        state_we_o,         // Sparsify
  using multi-rail.
50   output logic        sub_bytes_en_o,     // Sparsify
  using multi-rail.
51   input logic         sub_bytes_out_req_i, // Sparsify
  using multi-rail.
52   output logic        sub_bytes_out_ack_o, // Sparsify
  using multi-rail.
53   output add_rk_sel_e add_rk_sel_o,

```

```

54
55 // Control and sync signals for key expand data path
56 output key_full_sel_e key_full_sel_o,
57 output logic          key_full_we_o, // Sparsify
using multi-rail.
58 output key_dec_sel_e key_dec_sel_o,
59 output logic          key_dec_we_o, // Sparsify
using multi-rail.
60 output logic          key_expand_en_o, // Sparsify
using multi-rail.
61 input logic          key_expand_out_req_i, // Sparsify
using multi-rail.
62 output logic          key_expand_out_ack_o, // Sparsify
using multi-rail.
63 output logic          key_expand_clear_o,
64 output logic [3:0]    rnd_ctr_o,
65 output key_words_sel_e key_words_sel_o,
66 output round_key_sel_e round_key_sel_o,
67
68 // Register signals
69 input logic          crypt_q_i, // Sparsify
using multi-rail.
70 output logic          crypt_d_o, // Sparsify
using multi-rail.
71 input logic          dec_key_gen_q_i, // Sparsify
using multi-rail.
72 output logic          dec_key_gen_d_o, // Sparsify
using multi-rail.
73 input logic          prng_reseed_q_i,
74 output logic          prng_reseed_d_o,
75 input logic          key_clear_q_i,
76 output logic          key_clear_d_o,
77 input logic          data_out_clear_q_i,
78 output logic          data_out_clear_d_o
79 );
80
81 //////////////////////////////////////////////////
82 // Input Buffering //
83 //////////////////////////////////////////////////
84
85 localparam int NumInBufBits = $bits({
86   in_valid_i,
87   out_ready_i,
88   cfg_valid_i,
89   op_i,
90   key_len_i,
91   crypt_i,
92   dec_key_gen_i,
93   prng_reseed_i,
94   key_clear_i,
95   data_out_clear_i,
96   mux_sel_err_i,
97   sp_enc_err_i,
98   rnd_ctr_err_i,
99   op_err_i,
100  alert_fatal_i,
101  prng_reseed_ack_i,
102  sub_bytes_out_req_i,
103  key_expand_out_req_i,
104  crypt_q_i,
105  dec_key_gen_q_i,
106  prng_reseed_q_i,
107  key_clear_q_i,
108  data_out_clear_q_i
109 });
110
111 logic [NumInBufBits-1:0] in, in_buf;
112
113 assign in = {
114   in_valid_i,
115   out_ready_i,
116   cfg_valid_i,
117   op_i,
118   key_len_i,
119   crypt_i,
120   dec_key_gen_i,
121   prng_reseed_i,
122   key_clear_i,
123   data_out_clear_i,
124   mux_sel_err_i,
125   sp_enc_err_i,
126   rnd_ctr_err_i,
127   op_err_i,
128   alert_fatal_i,
129   prng_reseed_ack_i,
130   sub_bytes_out_req_i,
131   key_expand_out_req_i,
132   crypt_q_i,
133   dec_key_gen_q_i,
134   prng_reseed_q_i,
135   key_clear_q_i,
136   data_out_clear_q_i
137 };
138
139 // This primitive is used to place a size-only constraint on
the
140 // buffers to act as a synthesis optimization barrier.
141 caliptra_prim_buf #(
142   .Width(NumInBufBits)
143 ) u_caliptra_prim_buf_in (
144   .in_i(in),
145   .out_o(in_buf)
146 );
147
148 logic          in_valid;
149 logic          out_ready;
150 logic          cfg_valid;
151 ciph_op_e      op;
152 logic [$bits(op)-1:0] op_raw;
153 key_len_e      key_len;
154 logic          crypt;
155 logic          dec_key_gen;
156 logic          prng_reseed;
157 logic          key_clear;
158 logic          data_out_clear;
159 logic          mux_sel_err;
160 logic          sp_enc_err;
161 logic          rnd_ctr_err;
162 logic          op_err;
163 logic          alert_fatal;
164 logic          prng_reseed_ack;
165 logic          sub_bytes_out_req;
166 logic          key_expand_out_req;
167 logic          crypt_q;
168 logic          dec_key_gen_q;
169 logic          prng_reseed_q;
170 logic          key_clear_q;
171 logic          data_out_clear_q;
172
173 assign {in_valid,
174        out_ready,
175        cfg_valid,
176        op_raw,
177        key_len,
178        crypt,
179        dec_key_gen,
180        prng_reseed,
181        key_clear,
182        data_out_clear,
183        mux_sel_err,
184        sp_enc_err,
185        rnd_ctr_err,
186        op_err,
187        alert_fatal,
188        prng_reseed_ack,
189        sub_bytes_out_req,
190        key_expand_out_req,
191        crypt_q,
192        dec_key_gen_q,
193        prng_reseed_q,
194        key_clear_q,
195        data_out_clear_q} = in_buf;
196
197 assign op = ciph_op_e'(op_raw);
198
199 // Intermediate output signals
200 logic          in_ready;
201 logic          out_valid;
202 logic          alert;
203 logic          prng_update;
204 logic          prng_reseed_req;
205 state_sel_e    state_sel;
206 logic          state_we;
207 logic          sub_bytes_en;
208 logic          sub_bytes_out_ack;
209 add_rk_sel_e   add_rk_sel;
210 key_full_sel_e key_full_sel;
211 logic          key_full_we;
212 key_dec_sel_e  key_dec_sel;
213 logic          key_dec_we;
214 logic          key_expand_en;
215 logic          key_expand_out_ack;
216 logic          key_expand_clear;
217 logic [3:0]    rnd_ctr;
218 key_words_sel_e key_words_sel;
219 round_key_sel_e round_key_sel;
220 logic          crypt_d;
221 logic          dec_key_gen_d;
222 logic          prng_reseed_d;
223 logic          key_clear_d;
224 logic          data_out_clear_d;
225
226 //////////////////////////////////////////////////
227 // Regular FSM //
228 //////////////////////////////////////////////////
229
230 aes_cipher_control_fsm #(
231   .SecMasking ( SecMasking ),
232   .SecSBoxImpl ( SecSBoxImpl )
233 ) u_aes_cipher_control_fsm (
234   .clk_i      ( clk_i ),
235   .rst_ni     ( rst_ni ),
236
237   .in_valid_i ( in_valid ),
238   .in_ready_o ( in_ready ),
239
240   .out_valid_o ( out_valid ),
241   .out_ready_i ( out_ready ),
242
243   .cfg_valid_i ( cfg_valid ),
244   .op_i        ( op ),

```

```

245 .key_len_i      ( key_len      ),
246 .crypt_i       ( crypt       ),
247 .dec_key_gen_i  ( dec_key_gen  ),
248 .prng_reseed_i  ( prng_reseed  ),
249 .key_clear_i    ( key_clear    ),
250 .data_out_clear_i ( data_out_clear ),
251 .mux_sel_err_i  ( mux_sel_err  ),
252 .sp_enc_err_i   ( sp_enc_err   ),
253 .rnd_ctr_err_i  ( rnd_ctr_err  ),
254 .op_err_i       ( op_err       ),
255 .alert_fatal_i  ( alert_fatal  ),
256 .alert_o        ( alert        ),
257
258 .prng_update_o   ( prng_update  ),
259 .prng_reseed_req_o ( prng_reseed_req ),
260 .prng_reseed_ack_i ( prng_reseed_ack ),
261
262 .state_sel_o     ( state_sel    ),
263 .state_we_o      ( state_we     ),
264 .sub_bytes_en_o  ( sub_bytes_en ),
265 .sub_bytes_out_req_i ( sub_bytes_out_req ),
266 .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
267 .add_rk_sel_o    ( add_rk_sel   ),
268
269 .key_full_sel_o  ( key_full_sel  ),
270 .key_full_we_o   ( key_full_we   ),
271 .key_dec_sel_o   ( key_dec_sel   ),
272 .key_dec_we_o    ( key_dec_we    ),
273 .key_expand_en_o ( key_expand_en ),
274 .key_expand_out_req_i ( key_expand_out_req ),
275 .key_expand_out_ack_o ( key_expand_out_ack ),
276 .key_expand_clear_o ( key_expand_clear ),
277 .rnd_ctr_o       ( rnd_ctr      ),
278 .key_words_sel_o ( key_words_sel ),
279 .round_key_sel_o ( round_key_sel ),
280
281 .crypt_q_i       ( crypt_q      ),
282 .crypt_d_o       ( crypt_d      ),
283 .dec_key_gen_q_i ( dec_key_gen_q ),
284 .dec_key_gen_d_o ( dec_key_gen_d ),
285 .key_clear_q_i   ( key_clear_q   ),
286 .key_clear_d_o   ( key_clear_d   ),
287 .prng_reseed_q_i ( prng_reseed_q ),
288 .prng_reseed_d_o ( prng_reseed_d ),
289 .data_out_clear_q_i ( data_out_clear_q ),
290 .data_out_clear_d_o ( data_out_clear_d ),
291 );
292
293 //////////////////////////////////////////////////
294 // Output Buffering //
295 //////////////////////////////////////////////////
296
297 localparam int NumOutBufBits = $bits({
298   in_ready_o,
299   out_valid_o,
300   alert_o,
301   prng_update_o,
302   prng_reseed_req_o,
303   state_sel_o,
304   state_we_o,
305   sub_bytes_en_o,
306   sub_bytes_out_ack_o,
307   add_rk_sel_o,
308   key_full_sel_o,
309   key_full_we_o,
310   key_dec_sel_o,
311   key_dec_we_o,
312   key_expand_en_o,
313   key_expand_out_ack_o,
314   key_expand_clear_o,
315   rnd_ctr_o,
316   key_words_sel_o,
317   round_key_sel_o,
318   crypt_d_o,
319   dec_key_gen_d_o,
320   key_clear_d_o,
321   prng_reseed_d_o,
322   data_out_clear_d_o
323 });
324
325 logic [NumOutBufBits-1:0] out, out_buf;
326
327 assign out = {
328   in_ready,
329   out_valid,
330   alert,
331   prng_update,
332   prng_reseed_req,
333   state_sel,
334   state_we,
335   sub_bytes_en,
336   sub_bytes_out_ack,
337   add_rk_sel,
338   key_full_sel,
339   key_full_we,
340   key_dec_sel,
341   key_dec_we,
342   key_expand_en,
343   key_expand_out_ack,
344   key_expand_clear,
345   rnd_ctr,

```

```

346   key_words_sel,
347   round_key_sel,
348   crypt_d,
349   dec_key_gen_d,
350   key_clear_d,
351   prng_reseed_d,
352   data_out_clear_d
353   };
354
355 // This primitive is used to place a size-only constraint on
the
356 // buffers to act as a synthesis optimization barrier.
357 caliptra_prim_buf #(
358   .Width(NumOutBufBits)
359 ) u_caliptra_prim_buf_out (
360   .in_i(out),
361   .out_o(out_buf)
362 );
363
364 assign {in_ready_o,
365   out_valid_o,
366   alert_o,
367   prng_update_o,
368   prng_reseed_req_o,
369   state_sel_o,
370   state_we_o,
371   sub_bytes_en_o,
372   sub_bytes_out_ack_o,
373   add_rk_sel_o,
374   key_full_sel_o,
375   key_full_we_o,
376   key_dec_sel_o,
377   key_dec_we_o,
378   key_expand_en_o,
379   key_expand_out_ack_o,
380   key_expand_clear_o,
381   rnd_ctr_o,
382   key_words_sel_o,
383   round_key_sel_o,
384   crypt_d_o,
385   dec_key_gen_d_o,
386   key_clear_d_o,
387   prng_reseed_d_o,
388   data_out_clear_d_o} = out_buf;
389
390 endmodule

```

6. chipsalliance_caliptra-rtl/src/aes/rtl/aes_cipher_core.sv

(932 lines)

```

1 // Copyright lowRISC contributors (OpenTitan project).
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
details.
3 // SPDX-License-Identifier: Apache-2.0
4 //
5 // AES cipher core implementation
6 //
7 // This module contains the AES cipher core including, state
register, full key and decryption key
8 // registers as well as key expand module and control unit.
9 //
10 //
11 // Masking
12 // -----
13 //
14 // If the parameter "Masking" is set to one, first-order mask-
ing is applied to the entire
15 // cipher core including key expand module. For details, see
Rivain et al., "Provably secure
16 // higher-order masking of AES" available at https://
eprint.iacr.org/2010/441.pdf .
17 //
18 //
19 // Details on the data formats
20 // -----
21 //
22 // This implementation uses 4-dimensional SystemVerilog arrays to
represent the AES state:
23 //
24 //   logic [3:0][3:0][7:0] state_q [NumShares];
25 //
26 // The fourth dimension (unpacked) corresponds to the differ-
ent shares. The first element holds the
27 // (masked) data share whereas the other elements hold the
masks (masked implementation only).
28 //
29 // The three packed dimensions correspond to the 128-bit state
matrix per share. This
30 // implementation uses the same encoding as the Advanced En-
cryption Standard (AES) FIPS Publication
31 // 197 available at https://www.nist.gov/publications/ad-
vanced-encryption-standard-aes (see Section
32 // 3.4). An input sequence of 16 bytes (128-bit, left most
byte is the first one)
33 //
34 //   b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14 b15
35 //

```

```

35 // is mapped to the state matrix as
36 //
37 // [ b0 b4 b8 b12 ]
38 // [ b1 b5 b9 b13 ]
39 // [ b2 b6 b10 b14 ]
40 // [ b3 b7 b11 b15 ] .
41 //
42 // This is mapped to three packed dimensions of SystemVerilog
array as follows:
43 // - The first dimension corresponds to the rows. Thus,
state_q[0] gives
44 // - The first row of the state matrix [ b0 b4 b8
b12 ], or
45 // - A 32-bit packed SystemVerilog array 32h'{ b12, b8, b4,
b0 }.
46 //
47 // - The second dimension corresponds to the columns. To access
complete columns, the state matrix
48 // must be transposed first. Thus state_transposed =
aes_pkg::aes_transpose(state_q) and then
49 // state_transposed[1] gives
50 // - The second column of the state matrix [ b4 b5 b6
b7 ], or
51 // - A 32-bit packed SystemVerilog array 32h'{ b7, b6, b5,
b4 }.
52 //
53 // - The third dimension corresponds to the bytes.
54 //
55 // Note that the CSRs are little-endian. The input sequence
above is provided to 32-bit DATA_IN_0 -
56 // DATA_IN_3 registers as
57 // MSB LSB
58 // - DATA_IN_0 32h'{ b3, b2, b1, b0 }
59 // - DATA_IN_1 32h'{ b7, b6, b4, b4 }
60 // - DATA_IN_2 32h'{ b11, b10, b9, b8 }
61 // - DATA_IN_3 32h'{ b15, b14, b13, b12 } .
62 //
63 // The input state can thus be obtained by transposing the
content of the DATA_IN_0 - DATA_IN_3
64 // registers.
65 //
66 // Similarly, the implementation uses a 3-dimensional array to
represent the AES keys:
67 //
68 // logic [7:0][31:0] key_full_q [NumShares]
69 //
70 // The third dimension (unpacked) corresponds to the different
shares. The first element holds the
71 // (masked) key share whereas the other elements hold the
masks (masked implementation only).
72 // The two packed dimensions correspond to the 256-bit key per
share. This implementation uses
73 // the same encoding as the Advanced Encryption Standard (AES)
FIPS Publication
74 // 197 available at https://www.nist.gov/publications/advanced-encryption-standard-aes .
75 //
76 // The first packed dimension corresponds to the 8 key words.
The second packed dimension
77 // corresponds to the 32 bits per key word. A key sequence of
32 bytes (256-bit, left most byte is
78 // the first one)
79 //
80 // b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14
b15 ... b28 b29 b30 b31
81 //
82 // is mapped to the key words and registers (little-endian) as
83 // MSB LSB
84 // - KEY_SHARE0_0 32h'{ b3, b2, b1, b0 }
85 // - KEY_SHARE0_1 32h'{ b7, b6, b4, b4 }
86 // - KEY_SHARE0_2 32h'{ b11, b10, b9, b8 }
87 // - KEY_SHARE0_3 32h'{ b15, b14, b13, b12 }
88 // - KEY_SHARE0_4 32h'{ . . . }
89 // - KEY_SHARE0_5 32h'{ . . . }
90 // - KEY_SHARE0_6 32h'{ . . . }
91 // - KEY_SHARE0_7 32h'{ b31, b30, b29, b28 } .
92 //
93 `include "caliptra_prim_assert.sv"
94
95 module aes_cipher_core import aes_pkg::*;
96 #(
97     parameter bit AES192Enable = 1,
98     parameter bit CiphOpFwdOnly = 0,
99     parameter bit SecMasking = 1,
100    parameter sbox_impl_e SecSBoxImpl = SBoxImplDom,
101    parameter bit SecAllowForcingMasks = 0,
102    parameter bit SecSkipPRNGReseeding = 0,
103    parameter int unsigned EntropyWidth = edn_pkg::END-
POINT_BUS_WIDTH,
104
105    localparam int NumShares = SecMasking ?
2 : 1, // derived parameter
106
107    parameter masking_lfsr_seed_t RndCnstMaskingLfsrSeed = Rnd-
CnstMaskingLfsrSeedDefault,
108    parameter masking_lfsr_perm_t RndCnstMaskingLfsrPerm = Rnd-
CnstMaskingLfsrPermDefault

```

```

109 ) (
110     input logic clk_i,
111     input logic rst_ni,
112
113     // Input handshake signals
114     input sp2v_e in_valid_i,
115     output sp2v_e in_ready_o,
116
117     // Output handshake signals
118     output sp2v_e out_valid_o,
119     input sp2v_e out_ready_i,
120
121     // Control and sync signals
122     input logic cfg_valid_i, // Used for
gating assertions only.
123     input ciph_op_e op_i,
124     input key_len_e key_len_i,
125     input sp2v_e crypt_i,
126     output sp2v_e crypt_o,
127     input sp2v_e dec_key_gen_i,
128     output sp2v_e dec_key_gen_o,
129     input logic prng_reseed_i,
130     output logic prng_reseed_o,
131     input logic key_clear_i,
132     output logic key_clear_o,
133     input logic data_out_clear_i, // Re-
use the cipher core muxes.
134     output logic data_out_clear_o,
135     input logic alert_fatal_i,
136     output logic alert_o,
137
138     // Pseudo-random data for register clearing
139     input logic [3:0][3:0][7:0] prd_clearing_state_i
[NumShares],
140     input logic [7:0][31:0] prd_clearing_key_i [Num-
Shares],
141
142     // Masking PRNG
143     input logic force_masks_i, // Useful
for SCA only.
144     output logic [3:0][3:0][7:0] data_in_mask_o,
145     output logic entropy_req_o,
146     input logic entropy_ack_i,
147     input logic [EntropyWidth-1:0] entropy_i,
148
149     // I/O data & initial key
150     input logic [3:0][3:0][7:0] state_init_i [Num-
Shares],
151     input logic [7:0][31:0] key_init_i [NumShares],
152     output logic [3:0][3:0][7:0] state_o [NumShares]
153 );
154
155 // Signals
156 logic [3:0][3:0][7:0] state_d [NumShares];
157 logic [3:0][3:0][7:0] state_q [NumShares];
158 sp2v_e state_we_ctrl;
159 sp2v_e state_we;
160 logic [StateSelWidth-1:0] state_sel_raw;
161 state_sel_e state_sel_ctrl;
162 state_sel_e state_sel;
163 logic state_sel_err;
164
165 sp2v_e sub_bytes_en;
166 sp2v_e sub_bytes_out_req;
167 sp2v_e sub_bytes_out_ack;
168 logic sub_bytes_err;
169 logic [3:0][3:0][7:0] sub_bytes_out;
170 logic [3:0][3:0][7:0] sb_in_mask;
171 logic [3:0][3:0][7:0] sb_out_mask;
172 logic [3:0][3:0][7:0] shift_rows_in [Num-
Shares];
173 logic [3:0][3:0][7:0] shift_rows_out [Num-
Shares];
174 logic [3:0][3:0][7:0] mix_columns_out [Num-
Shares];
175 logic [3:0][3:0][7:0] add_round_key_in [Num-
Shares];
176 logic [3:0][3:0][7:0] add_round_key_out [Num-
Shares];
177 logic [AddrRkSelWidth-1:0] add_rk_sel_raw;
178 add_rk_sel_e add_rk_sel_ctrl;
179 add_rk_sel_e add_rk_sel;
180 logic add_rk_sel_err;
181
182 logic [7:0][31:0] key_full_d [NumShares];
183 logic [7:0][31:0] key_full_q [NumShares];
184 sp2v_e key_full_we_ctrl;
185 sp2v_e key_full_we;
186 logic [KeyFullSelWidth-1:0] key_full_sel_raw;
187 key_full_sel_e key_full_sel_ctrl;
188 key_full_sel_e key_full_sel;
189 logic key_full_sel_err;
190 logic [7:0][31:0] key_dec_d [NumShares];
191 logic [7:0][31:0] key_dec_q [NumShares];
192 sp2v_e key_dec_we_ctrl;
193 sp2v_e key_dec_we;
194 logic [KeyDecSelWidth-1:0] key_dec_sel_raw;
195 key_dec_sel_e key_dec_sel_ctrl;
196 key_dec_sel_e key_dec_sel;

```



```

197 logic                                key_dec_sel_err;
198 logic                                [7:0][31:0] key_expand_out [Num-
Shares];
199 ciph_op_e                            key_expand_op;
200 sp2v_e                               key_expand_en;
201 logic                                key_expand_prd_we;
202 sp2v_e                               key_expand_out_req;
203 sp2v_e                               key_expand_out_ack;
204 logic                                key_expand_err;
205 logic                                key_expand_clear;
206 logic                                [3:0] key_expand_round;
207 logic                                [KeywordsSelWidth-1:0] key_words_sel_raw;
208 key_words_sel_e                      key_words_sel_ctrl;
209 key_words_sel_e                      key_words_sel;
210 logic                                key_words_sel_err;
211 logic                                [3:0][31:0] key_words [NumShares];
212 logic                                [3:0][3:0][7:0] key_bytes [NumShares];
213 logic                                [3:0][3:0][7:0] key_mix_columns_out
[NumShares];
214 logic                                [3:0][3:0][7:0] round_key [NumShares];
215 logic                                [RoundKeySelWidth-1:0] round_key_sel_raw;
216 round_key_sel_e                     round_key_sel_ctrl;
217 round_key_sel_e                     round_key_sel;
218 logic                                round_key_sel_err;
219
220 logic                                cfg_valid;
221 logic                                mux_sel_err;
222 logic                                sp_enc_err_d,
sp_enc_err_q;
223 logic                                op_err;
224
225 // Pseudo-random data for masking purposes
226 logic                                [WidthPRDMasking-1:0] prd_masking;
227 logic                                [3:0][3:0][WidthPRDBox-1:0] prd_sub_bytes_d;
228 logic                                [3:0][3:0][WidthPRDBox-1:0] prd_sub_bytes_q;
229 logic                                [WidthPRDKey-1:0] prd_key_expand;
230 logic                                prd_masking_upd;
231 logic                                prd_masking_rsd_req;
232 logic                                prd_masking_rsd_ack;
233
234 logic                                [3:0][3:0][7:0] data_in_mask;
235
236 // op_i is one-hot encoded. Check the provided value and
trigger an alert upon detecting invalid
237 // encodings.
238 assign op_err = ~(op_i == CIPH_FWD || op_i == CIPH_INV);
239 assign cfg_valid = cfg_valid_i & ~op_err;
240
241 //////////////////////////////////////////////////
242 // Data //
243 //////////////////////////////////////////////////
244
245 // SEC_CM: DATA_REG.SEC_WIPE
246 // State registers
247 always_comb begin : state_mux
248     unique case (state_sel)
249         STATE_INIT: state_d = state_init_i;
250         STATE_ROUND: state_d = add_round_key_out;
251         STATE_CLEAR: state_d = prd_clearing_state_i;
252         default: state_d = prd_clearing_state_i;
253     endcase
254 end
255
256 always_ff @(posedge clk_i or negedge rst_ni) begin :
state_reg
257     if (!rst_ni) begin
258         state_q <= '{default: '0};
259     end else if (state_we == SP2V_HIGH) begin
260         state_q <= state_d;
261     end
262 end
263
264 // Masking
265 if (!SecMasking) begin : gen_no_masks
266     // The masks are ignored anyway, they can be 0.
267     assign sb_in_mask = '0;
268     assign prd_masking = '0;
269
270     // Tie-off unused signals.
271     logic unused_entropy_ack;
272     logic [EntropyWidth-1:0] unused_entropy;
273     assign unused_entropy_ack = entropy_ack_i;
274     assign unused_entropy = entropy_i;
275     assign entropy_req_o = 1'b0;
276
277     logic unused_force_masks;
278     logic unused_prd_masking_upd;
279     logic unused_prd_masking_rsd_req;
280     assign unused_force_masks = force_masks_i;
281     assign unused_prd_masking_upd = prd_masking_upd;
282     assign unused_prd_masking_rsd_req = prd_masking_rsd_req;
283     assign prd_masking_rsd_ack = 1'b0;
284
285     logic [3:0][3:0][7:0] unused_sb_out_mask;
286     assign unused_sb_out_mask = sb_out_mask;
287
288 end else begin : gen_masks
289     // The input mask is the mask share of the state.
290     assign sb_in_mask = state_q[1];
291
292     // The masking PRNG generates:

```

```

293 // - the pseudo-random data (PRD) required by SubBytes,
294 // - the PRD required by the key expand module (has 4 S-
Boxes internally).
295 aes_prng_masking #(
296     .Width (WidthPRDMasking),
297     .EntropyWidth (EntropyWidth),
298     .SecAllowForcingMasks (SecAllowForcingMasks),
299     .SecSkipPRNGReseeding (SecSkipPRNGReseeding),
300     .RndCnstLfSrSeed (RndCnstMaskingLfSrSeed),
301     .RndCnstLfSrPerm (RndCnstMaskingLfSrPerm)
302 ) u_aes_prng_masking (
303     .clk_i (clk_i),
304     .rst_ni (rst_ni),
305     .force_masks_i (force_masks_i),
306     .data_update_i (prd_masking_upd),
307     .data_o (prd_masking_rsd_req),
308     .reseed_req_i (prd_masking_rsd_req),
309     .reseed_ack_o (prd_masking_rsd_ack),
310     .entropy_req_o (entropy_req_o),
311     .entropy_ack_i (entropy_ack_i),
312     .entropy_i (entropy_i)
313 );
314 end
315
316 // Extract randomness for key expand module and SubBytes.
317 //
318 // The masking PRNG output has the following shape:
319 // prd_masking = { prd_key_expand, prd_sub_bytes_d }
320 assign prd_key_expand = prd_masking[WidthPRDMasking-1 -:
WidthPRDKey];
321 assign prd_sub_bytes_d = prd_masking[WidthPRDData-1 -:
WidthPRDData];
322
323 // PRD buffering
324 if (!SecMasking) begin : gen_no_prd_buffer
325     // The masks are ignored anyway.
326     assign prd_sub_bytes_q = prd_sub_bytes_d;
327
328 end else begin : gen_prd_buffer
329     // PRD buffer stage to:
330     // 1. Make sure the S-Boxes get always presented new data/
mask inputs together with fresh PRD
331     // for remasking.
332
333     // 2. Prevent glitches originating from inside the masking
PRNG from propagating into the
334     // masked S-Boxes.
335     always_ff @(posedge clk_i or negedge rst_ni) begin :
prd_sub_bytes_reg
336         if (!rst_ni) begin
337             prd_sub_bytes_q <= '0;
338         end else if (state_we == SP2V_HIGH) begin
339             prd_sub_bytes_q <= prd_sub_bytes_d;
340         end
341     end
342
343     // Convert the 3-dimensional prd_sub_bytes_q array to a 1-
dimensional packed array for the
344     // aes_prd_get_lsbs() function used below.
345     logic [WidthPRDData-1:0] prd_sub_bytes;
346     assign prd_sub_bytes = prd_sub_bytes_q;
347
348     // Extract randomness for masking the input data.
349     //
350     // The masking PRNG is used for generating both the PRD for
the S-Boxes/SubBytes operation as
351     // well as for the input data masks. When using any of the
masked Canright S-Box implementations,
352     // it is important that the SubBytes input masks (generated
by the PRNG in Round X-1) and the
353     // SubBytes output masks (generated by the PRNG in Round X)
are independent. This can be achieved
354     // by using e.g. an unrolled Bivium stream cipher primitive
inside the PRNG. Since the input data
355     // masks become the SubBytes input masks in the first round,
we select the same 8 bit lanes for
356     // the input data masks which are also used to form the Sub-
Bytes output mask for the masked
357     // Canright S-Box implementations, i.e., the 8 LSBs of the
per S-Box PRD. In particular, we have:
358     //
359     // prd_masking = { prd_key_expand, ... , sb_prd[4],
sb_out_mask[4], sb_prd[0], sb_out_mask[0] }
360     //
361     // Where sb_out_mask[x] contains the SubBytes output mask
for byte x (when using a masked
362     // Canright S-Box implementation) and sb_prd[x] contains ad-
ditional PRD consumed by SubBytes for
363     // byte x.
364     //
365     // When using a masked S-Box implementation other than Can-
right, we still select the 8 LSBs of
366     // the per-S-Box PRD to form the input data mask of the cor-
responding byte. We do this to
367     // distribute the input data masks over all output bits the
masking PRNG. We do the extraction on
368     // a row basis.
369     localparam int unsigned WidthPRDRow = 4*WidthPRDBox;
370     for (genvar i = 0; i < 4; i++) begin : gen_in_mask

```



```

371 assign data_in_mask[i] = aes_prd_get_lsbs(prd_sub_bytes[i
* WidthPRDRow +: WidthPRDRow]);
372 end
373
374 // Rotate the data input masks by 64 bits to ensure the data
input masks are independent
375 // from the PRD fed to the S-Boxes/SubBytes operation.
376 assign data_in_mask_o = {data_in_mask[1], data_in_mask[0],
data_in_mask[3], data_in_mask[2]};
377
378 // Cipher data path
379 aes_sub_bytes #(
380   .SecSBoxImpl ( SecSBoxImpl )
381 ) u_aes_sub_bytes (
382   .clk_i      ( clk_i      ),
383   .rst_ni     ( rst_ni     ),
384   .en_i       ( sub_bytes_en ),
385   .out_req_o   ( sub_bytes_out_req ),
386   .out_ack_i   ( sub_bytes_out_ack ),
387   .op_i       ( op_i       ),
388   .data_i      ( state_q[0] ),
389   .mask_i      ( sb_in_mask ),
390   .prd_i       ( prd_sub_bytes_q ),
391   .data_o      ( sub_bytes_out ),
392   .mask_o      ( sb_out_mask ),
393   .err_o       ( sub_bytes_err )
394 );
395
396 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_shift_mux
397   if (s == 0) begin : gen_shift_in_data
398     // The (masked) data share
399     assign shift_rows_in[s] = sub_bytes_out;
400   end else begin : gen_shift_in_mask
401     // The mask share
402     assign shift_rows_in[s] = sb_out_mask;
403   end
404
405   aes_shift_rows u_aes_shift_rows (
406     .op_i ( op_i ),
407     .data_i ( shift_rows_in[s] ),
408     .data_o ( shift_rows_out[s] )
409   );
410
411   aes_mix_columns u_aes_mix_columns (
412     .op_i ( op_i ),
413     .data_i ( shift_rows_out[s] ),
414     .data_o ( mix_columns_out[s] )
415   );
416 end
417
418 always_comb begin : add_round_key_in_mux
419   unique case (add_rk_sel)
420     ADD_RK_INIT: add_round_key_in = state_q;
421     ADD_RK_ROUND: add_round_key_in = mix_columns_out;
422     ADD_RK_FINAL: add_round_key_in = shift_rows_out;
423     default: add_round_key_in = state_q;
424   endcase
425 end
426
427 for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_add_round_key
428   assign add_round_key_out[s] = add_round_key_in[s] ^
round_key[s];
429 end
430
431 // Key
432 // Key
433 // Key
434
435 // SEC_CM: KEY.SEC_WIPE
436 // Full Key registers
437 always_comb begin : key_full_mux
438   unique case (key_full_sel)
439     KEY_FULL_ENC_INIT: key_full_d = key_init_i;
440     KEY_FULL_DEC_INIT: key_full_d = !CiphOpFwdOnly ?
key_dec_q : prd_clearing_key_i;
441     KEY_FULL_ROUND: key_full_d = key_expand_out;
442     KEY_FULL_CLEAR: key_full_d = prd_clearing_key_i;
443     default: key_full_d = prd_clearing_key_i;
444   endcase
445 end
446
447 always_ff @(posedge clk_i or negedge rst_ni) begin :
key_full_reg
448   if (!rst_ni) begin
449     key_full_q <= '{default: '0};
450   end else if (key_full_we == SP2V_HIGH) begin
451     key_full_q <= key_full_d;
452   end
453 end
454
455 if (!CiphOpFwdOnly) begin : gen_key_dec
456   // SEC_CM: KEY.SEC_WIPE
457   // Decryption Key registers
458   always_comb begin : key_dec_mux
459     unique case (key_dec_sel)
460       KEY_DEC_EXPAND: key_dec_d = key_expand_out;
461       KEY_DEC_CLEAR: key_dec_d = prd_clearing_key_i;
462       default: key_dec_d = prd_clearing_key_i;

```

```

463   endcase
464 end
465
466 always_ff @(posedge clk_i or negedge rst_ni) begin :
key_dec_reg
467   if (!rst_ni) begin
468     key_dec_q <= '{default: '0};
469   end else if (key_dec_we == SP2V_HIGH) begin
470     key_dec_q <= key_dec_d;
471   end
472 end
473
474 end else begin : gen_no_key_dec
475   // No Decryption Key registers
476   assign key_dec_q = '{default: '0};
477   assign key_dec_d = key_dec_q;
478
479   // Tie-off unused signals.
480   logic unused_key_dec;
481   always_comb begin
482     unused_key_dec = ^{key_dec_sel, key_dec_we};
483     for (int s = 0; s < NumShares; s++) begin
484       unused_key_dec ^= ^{key_dec_d[s]};
485     end
486   end
487
488   // Make sure that whenever the data/mask inputs of the S-
Boxes update, the internally buffered
489   // PRD is updated in sync.
490   assign key_expand_prd_we = (key_full_we == SP2V_HIGH) ?
1'b1 : 1'b0;
491
492   // Key expand data path
493   aes_key_expand #(
494     .AES192Enable ( AES192Enable ),
495     .SecMasking ( SecMasking ),
496     .SecSBoxImpl ( SecSBoxImpl )
497   ) u_aes_key_expand (
498     .clk_i      ( clk_i      ),
499     .rst_ni     ( rst_ni     ),
500     .cfg_valid_i ( cfg_valid ),
501     .op_i       ( key_expand_op ),
502     .en_i       ( key_expand_en ),
503     .prd_we_i   ( key_expand_prd_we ),
504     .out_req_o   ( key_expand_out_req ),
505     .out_ack_i   ( key_expand_out_ack ),
506     .clear_i     ( key_expand_clear ),
507     .round_i     ( key_expand_round ),
508     .key_len_i   ( key_len_i ),
509     .key_i       ( key_full_q ),
510     .key_o       ( key_expand_out ),
511     .prd_i       ( prd_key_expand ),
512     .err_o       ( key_expand_err )
513   );
514
515   for (genvar s = 0; s < NumShares; s++) begin :
gen_shares_round_key
516     always_comb begin : key_words_mux
517       unique case (key_words_sel)
518         KEY_WORDS_0123: key_words[s] = key_full_q[s][3:0];
519         KEY_WORDS_2345: key_words[s] = AES192Enable ?
key_full_q[s][5:2] : '0;
520         KEY_WORDS_4567: key_words[s] = key_full_q[s][7:4];
521         KEY_WORDS_ZERO: key_words[s] = '0;
522         default: key_words[s] = '0;
523       endcase
524     end
525
526     // Convert words to bytes (every key word contains one
column).
527     assign key_bytes[s] = aes_transpose(key_words[s]);
528
529     aes_mix_columns u_aes_key_mix_columns (
530       .op_i ( CIPH_INV ),
531       .data_i ( key_bytes[s] ),
532       .data_o ( key_mix_columns_out[s] )
533     );
534   end
535
536   always_comb begin : round_key_mux
537     unique case (round_key_sel)
538       ROUND_KEY_DIRECT: round_key = key_bytes;
539       ROUND_KEY_MIXED: round_key = !CiphOpFwdOnly ?
key_mix_columns_out : key_bytes;
540       default: round_key = key_bytes;
541     endcase
542   end
543
544   if (CiphOpFwdOnly) begin : gen_unused_key_mix_columns_out
545     // Tie-off unused signals.
546     logic unused_key_mix_columns_out;
547     always_comb begin
548       unused_key_mix_columns_out = 1'b0;
549       for (int s = 0; s < NumShares; s++) begin
550         unused_key_mix_columns_out ^=
^key_mix_columns_out[s];
551       end
552     end
553   end
554
555   // Key

```

```

556 // Control //
557 ///////////////
558
559 // Control
560 aes_cipher_control #(
561   .CiphOpFwdOnly ( CiphOpFwdOnly ),
562   .SecMasking     ( SecMasking     ),
563   .SecSBoxImpl    ( SecSBoxImpl    )
564 ) u_aes_cipher_control (
565   .clk_i          ( clk_i           ),
566   .rst_ni         ( rst_ni          ),
567
568   .in_valid_i     ( in_valid_i      ),
569   .in_ready_o     ( in_ready_o      ),
570
571   .out_valid_o    ( out_valid_o     ),
572   .out_ready_i    ( out_ready_i     ),
573
574   .cfg_valid_i    ( cfg_valid       ),
575   .op_i           ( op_i            ),
576   .key_len_i      ( key_len_i       ),
577   .crypt_i        ( crypt_i         ),
578   .crypt_o        ( crypt_o         ),
579   .dec_key_gen_i  ( dec_key_gen_i   ),
580   .dec_key_gen_o  ( dec_key_gen_o   ),
581   .prng_reseed_i  ( prng_reseed_i   ),
582   .prng_reseed_o  ( prng_reseed_o   ),
583   .key_clear_i    ( key_clear_i     ),
584   .key_clear_o    ( key_clear_o     ),
585   .data_out_clear_i ( data_out_clear_i ),
586   .data_out_clear_o ( data_out_clear_o ),
587   .mux_sel_err_i  ( mux_sel_err     ),
588   .sp_enc_err_i   ( sp_enc_err_q    ),
589   .op_err_i       ( op_err          ),
590   .alert_fatal_i  ( alert_fatal_i   ),
591   .alert_o        ( alert_o         ),
592
593   .prng_update_o  ( prd_masking_upd ),
594   .prng_reseed_req_o ( prd_masking_rsd_req ),
595   .prng_reseed_ack_i ( prd_masking_rsd_ack ),
596
597   .state_sel_o    ( state_sel_ctrl  ),
598   .state_we_o     ( state_we_ctrl   ),
599   .sub_bytes_en_o ( sub_bytes_en    ),
600   .sub_bytes_out_req_i ( sub_bytes_out_req ),
601   .sub_bytes_out_ack_o ( sub_bytes_out_ack ),
602   .add_rk_sel_o   ( add_rk_sel_ctrl ),
603
604   .key_expand_op_o ( key_expand_op   ),
605   .key_full_sel_o  ( key_full_sel_ctrl ),
606   .key_full_we_o   ( key_full_we_ctrl ),
607   .key_dec_sel_o   ( key_dec_sel_ctrl ),
608   .key_dec_we_o    ( key_dec_we_ctrl ),
609   .key_expand_en_o ( key_expand_en   ),
610   .key_expand_out_req_i ( key_expand_out_req ),
611   .key_expand_out_ack_o ( key_expand_out_ack ),
612   .key_expand_clear_o ( key_expand_clear ),
613   .key_expand_round_o ( key_expand_round ),
614   .key_words_sel_o ( key_words_sel_ctrl ),
615   .round_key_sel_o ( round_key_sel_ctrl )
616 );
617
618 // Selectors //
619 ///////////////
620
621 // We use sparse encodings for these mux selector signals
622 // and must ensure that:
623 // 1. The synthesis tool doesn't optimize away the sparse
624 // encoding.
625 // 2. The selector signal is always valid. More precisely,
626 // an alert or SVA is triggered if a
627 // selector signal takes on an invalid value.
628 // 3. The alert signal remains asserted until reset even if
629 // the selector signal becomes valid
630 // again. This is achieved by driving the control FSM
631 // into the terminal error state whenever
632 // any mux selector signal becomes invalid.
633 // If any mux selector signal becomes invalid, the cipher
634 // core further immediately de-asserts
635 // the out_valid_o signal to prevent any data from being re-
636 // leased.
637
638 aes_sel_buf_chk #(
639   .Num      ( StateSelNum ),
640   .Width    ( StateSelWidth ),
641   .EnSecBuf ( 1'b1 )
642 ) u_aes_state_sel_buf_chk (
643   .clk_i ( clk_i ),
644   .rst_ni ( rst_ni ),
645   .sel_i ( state_sel_ctrl ),
646   .sel_o ( state_sel_raw ),
647   .err_o ( state_sel_err )
648 );
649 assign state_sel = state_sel_e'(state_sel_raw);
650
651 aes_sel_buf_chk #(
652   .Num      ( AddRKSelNum ),
653   .Width    ( AddRKSelWidth ),
654   .EnSecBuf ( 1'b1 )
655 ) u_aes_add_rk_sel_buf_chk (
656   .clk_i ( clk_i ),
657   .rst_ni ( rst_ni ),
658   .sel_i ( add_rk_sel_ctrl ),
659   .sel_o ( add_rk_sel_raw ),
660   .err_o ( add_rk_sel_err )
661 );
662 assign add_rk_sel = add_rk_sel_e'(add_rk_sel_raw);
663
664 aes_sel_buf_chk #(
665   .Num      ( KeyFullSelNum ),
666   .Width    ( KeyFullSelWidth ),
667   .EnSecBuf ( 1'b1 )
668 ) u_aes_key_full_sel_buf_chk (
669   .clk_i ( clk_i ),
670   .rst_ni ( rst_ni ),
671   .sel_i ( key_full_sel_ctrl ),
672   .sel_o ( key_full_sel_raw ),
673   .err_o ( key_full_sel_err )
674 );
675 assign key_full_sel = key_full_sel_e'(key_full_sel_raw);
676
677 aes_sel_buf_chk #(
678   .Num      ( KeyDecSelNum ),
679   .Width    ( KeyDecSelWidth ),
680   .EnSecBuf ( 1'b1 )
681 ) u_aes_key_dec_sel_buf_chk (
682   .clk_i ( clk_i ),
683   .rst_ni ( rst_ni ),
684   .sel_i ( key_dec_sel_ctrl ),
685   .sel_o ( key_dec_sel_raw ),
686   .err_o ( key_dec_sel_err )
687 );
688 assign key_dec_sel = key_dec_sel_e'(key_dec_sel_raw);
689
690 aes_sel_buf_chk #(
691   .Num      ( KeyWordsSelNum ),
692   .Width    ( KeyWordsSelWidth ),
693   .EnSecBuf ( 1'b1 )
694 ) u_aes_key_words_sel_buf_chk (
695   .clk_i ( clk_i ),
696   .rst_ni ( rst_ni ),
697   .sel_i ( key_words_sel_ctrl ),
698   .sel_o ( key_words_sel_raw ),
699   .err_o ( key_words_sel_err )
700 );
701 assign key_words_sel = key_words_sel_e'(key_words_sel_raw);
702
703 aes_sel_buf_chk #(
704   .Num      ( RoundKeySelNum ),
705   .Width    ( RoundKeySelWidth ),
706   .EnSecBuf ( 1'b1 )
707 ) u_aes_round_key_sel_buf_chk (
708   .clk_i ( clk_i ),
709   .rst_ni ( rst_ni ),
710   .sel_i ( round_key_sel_ctrl ),
711   .sel_o ( round_key_sel_raw ),
712   .err_o ( round_key_sel_err )
713 );
714 assign round_key_sel = round_key_sel_e'(round_key_sel_raw);
715
716 // Signal invalid mux selector signals to control FSM which
717 // will lock up and trigger an alert.
718 assign mux_sel_err = state_sel_err | add_rk_sel_err |
719 key_full_sel_err |
720 key_dec_sel_err | key_words_sel_err | round_key_sel_err;
721
722 // Sparsely Encoded Signals //
723 ///////////////
724
725 // We use sparse encodings for various critical signals and
726 // must ensure that:
727 // 1. The synthesis tool doesn't optimize away the sparse
728 // encoding.
729 // 2. The sparsely encoded signal is always valid. More pre-
730 // cisely, an alert or SVA is triggered
731 // if a sparse signal takes on an invalid value.
732 // 3. The alert signal remains asserted until reset even if
733 // the sparse signal becomes valid again
734 // This is achieved by driving the control FSM into the
735 // terminal error state whenever any
736 // sparsely encoded signal becomes invalid.
737 // If any sparsely encoded signal becomes invalid, the
738 // cipher core further immediately de-asserts
739 // the out_valid_o signal to prevent any data from being re-
740 // leased.
741
742 // We use vectors of sparsely encoded signals to reduce code
743 // duplication.
744 localparam int unsigned NumSp2Vsig = 3;
745 sp2v_e [NumSp2Vsig-1:0] sp2v_sig;
746 sp2v_e [NumSp2Vsig-1:0] sp2v_sig_chk;
747 logic [NumSp2Vsig-1:0][Sp2Vwidth-1:0] sp2v_sig_chk_raw;
748 logic [NumSp2Vsig-1:0] sp2v_sig_err;
749
750 assign sp2v_sig[0] = state_we_ctrl;
751 assign sp2v_sig[1] = key_full_we_ctrl;
752 assign sp2v_sig[2] = key_dec_we_ctrl;

```

```

650 ) u_aes_add_rk_sel_buf_chk (
651   .clk_i ( clk_i ),
652   .rst_ni ( rst_ni ),
653   .sel_i ( add_rk_sel_ctrl ),
654   .sel_o ( add_rk_sel_raw ),
655   .err_o ( add_rk_sel_err )
656 );
657 assign add_rk_sel = add_rk_sel_e'(add_rk_sel_raw);
658
659 aes_sel_buf_chk #(
660   .Num      ( KeyFullSelNum ),
661   .Width    ( KeyFullSelWidth ),
662   .EnSecBuf ( 1'b1 )
663 ) u_aes_key_full_sel_buf_chk (
664   .clk_i ( clk_i ),
665   .rst_ni ( rst_ni ),
666   .sel_i ( key_full_sel_ctrl ),
667   .sel_o ( key_full_sel_raw ),
668   .err_o ( key_full_sel_err )
669 );
670 assign key_full_sel = key_full_sel_e'(key_full_sel_raw);
671
672 aes_sel_buf_chk #(
673   .Num      ( KeyDecSelNum ),
674   .Width    ( KeyDecSelWidth ),
675   .EnSecBuf ( 1'b1 )
676 ) u_aes_key_dec_sel_buf_chk (
677   .clk_i ( clk_i ),
678   .rst_ni ( rst_ni ),
679   .sel_i ( key_dec_sel_ctrl ),
680   .sel_o ( key_dec_sel_raw ),
681   .err_o ( key_dec_sel_err )
682 );
683 assign key_dec_sel = key_dec_sel_e'(key_dec_sel_raw);
684
685 aes_sel_buf_chk #(
686   .Num      ( KeyWordsSelNum ),
687   .Width    ( KeyWordsSelWidth ),
688   .EnSecBuf ( 1'b1 )
689 ) u_aes_key_words_sel_buf_chk (
690   .clk_i ( clk_i ),
691   .rst_ni ( rst_ni ),
692   .sel_i ( key_words_sel_ctrl ),
693   .sel_o ( key_words_sel_raw ),
694   .err_o ( key_words_sel_err )
695 );
696 assign key_words_sel = key_words_sel_e'(key_words_sel_raw);
697
698 aes_sel_buf_chk #(
699   .Num      ( RoundKeySelNum ),
700   .Width    ( RoundKeySelWidth ),
701   .EnSecBuf ( 1'b1 )
702 ) u_aes_round_key_sel_buf_chk (
703   .clk_i ( clk_i ),
704   .rst_ni ( rst_ni ),
705   .sel_i ( round_key_sel_ctrl ),
706   .sel_o ( round_key_sel_raw ),
707   .err_o ( round_key_sel_err )
708 );
709 assign round_key_sel = round_key_sel_e'(round_key_sel_raw);
710
711 // Signal invalid mux selector signals to control FSM which
712 // will lock up and trigger an alert.
713 assign mux_sel_err = state_sel_err | add_rk_sel_err |
714 key_full_sel_err |
715 key_dec_sel_err | key_words_sel_err | round_key_sel_err;
716
717 // Sparsely Encoded Signals //
718 ///////////////
719
720 // We use sparse encodings for various critical signals and
721 // must ensure that:
722 // 1. The synthesis tool doesn't optimize away the sparse
723 // encoding.
724 // 2. The sparsely encoded signal is always valid. More pre-
725 // cisely, an alert or SVA is triggered
726 // if a sparse signal takes on an invalid value.
727 // 3. The alert signal remains asserted until reset even if
728 // the sparse signal becomes valid again
729 // This is achieved by driving the control FSM into the
730 // terminal error state whenever any
731 // sparsely encoded signal becomes invalid.
732 // If any sparsely encoded signal becomes invalid, the
733 // cipher core further immediately de-asserts
734 // the out_valid_o signal to prevent any data from being re-
735 // leased.
736
737 // We use vectors of sparsely encoded signals to reduce code
738 // duplication.
739 localparam int unsigned NumSp2Vsig = 3;
740 sp2v_e [NumSp2Vsig-1:0] sp2v_sig;
741 sp2v_e [NumSp2Vsig-1:0] sp2v_sig_chk;
742 logic [NumSp2Vsig-1:0][Sp2Vwidth-1:0] sp2v_sig_chk_raw;
743 logic [NumSp2Vsig-1:0] sp2v_sig_err;
744
745 assign sp2v_sig[0] = state_we_ctrl;
746 assign sp2v_sig[1] = key_full_we_ctrl;
747 assign sp2v_sig[2] = key_dec_we_ctrl;

```

```

740
741 // All signals inside sp2v_sig are eventually converted to
single-rail signals.
742 localparam bit [NumSp2VSig-1:0] Sp2VEnSecBuf = {Num-
Sp2VSig{1'b1}};
743
744 // Individually check sparsely encoded signals.
745 for (genvar i = 0; i < NumSp2VSig; i++) begin :
gen_sel_buf_chk
746   aes_sel_buf_chk #(
747     .Num      ( Sp2VNum      ),
748     .Width    ( Sp2VWidth    ),
749     .EnSecBuf ( Sp2VEnSecBuf[i] )
750   ) u_aes_sp2v_sig_buf_chk_i (
751     .clk_i ( clk_i ),
752     .rst_ni ( rst_ni ),
753     .sel_i ( sp2v_sig[i] ),
754     .sel_o ( sp2v_sig_chk_raw[i] ),
755     .err_o ( sp2v_sig_err[i] )
756   );
757   assign sp2v_sig_chk[i] = sp2v_e'(sp2v_sig_chk_raw[i]);
758 end
759
760 assign state_we = sp2v_sig_chk[0];
761 assign key_full_we = sp2v_sig_chk[1];
762 assign key_dec_we = sp2v_sig_chk[2];
763
764 // Collect encoding errors.
765 // We instantiate the checker modules as close as possible
to where the sparsely encoded signals
766 // are used. Here, we collect also encoding errors detected
in other places of the cipher core.
767 assign sp_enc_err_d = |sp2v_sig_err | sub_bytes_err |
key_expand_err;
768
769 // We need to register the collected error signal to avoid
circular loops in the cipher core
770
771 // controller related to out_valid_o and detecting errors in
state_we_o and sub_bytes_out_ack.
772 always_ff @(posedge clk_i or negedge rst_ni) begin :
reg_sp_enc_err
773   if (!rst_ni) begin
774     sp_enc_err_q <= 1'b0;
775   end else if (sp_enc_err_d) begin
776     sp_enc_err_q <= 1'b1;
777   end
778 end
779
780 // Outputs //
781 //////////////////////////////////////////////////////////////////
782
783 // The output of the last round is not stored into the state
register but forwarded directly.
784 assign state_o = add_round_key_out;
785
786 //////////////////////////////////////////////////////////////////
787 // Assertions //
788 //////////////////////////////////////////////////////////////////
789
790 // Typically assertions already contain this macro, which en-
sures that assertions are only compiled
791 // in simulation and FPV. However, we wrap the entire asser-
tion section with CALIPTRA_INC_ASSERT so that the
792 // helper logic below is not synthesized either, since that
could cause issues in DC.
793 `ifdef CALIPTRA_INC_ASSERT
794   //VCS coverage off
795   // pragma coverage off
796
797   // Create a lint error to reduce the risk of accidentally
disabling the masking.
798   CALIPTRA_ASSERT_STATIC_LINT_ERROR(AesSecMaskingNonDefault,
SecMasking == 1)
799
800   // Cipher core masking requires a masked SBox and vice
versa.
801   `CALIPTRA_ASSERT_INIT(AesMaskedCoreAndSBox,
802     (SecMasking &&
803     (SecSBoxImpl == SBoxImplCanrightMasked ||
804     SecSBoxImpl == SBoxImplCanrightMaskedNoreuse ||
805     SecSBoxImpl == SBoxImplDom)) ||
806     (!SecMasking &&
807     (SecSBoxImpl == SBoxImplLut ||
808     SecSBoxImpl == SBoxImplCanright)))
809
810   // Signals used for assertions only.
811   logic prd_clearing_equals_output, un-
used_prd_clearing_equals_output;
812
813   assign prd_clearing_equals_output = (prd_clearing_state_i ==
add_round_key_out);
814   assign unused_prd_clearing_equals_output = prd_clear-
ing_equals_output;
815
816   // Ensure that the state register gets cleared with pseudo-
random data at the end of the last
817   // round. The following two scenarios are unlikely but not
illegal:

```

```

817 // 1. The newly loaded initial state matches the previous
output (the round counter is only
818 // cleared upon loading the new initial state).
819 // 2. The previous pseudo-random data is equal to the previ-
ous output.
820 // Otherwise, we must see an alert e.g. because the state
multiplexer got glitched.
821 `CALIPTRA_ASSERT(AesSecCmDataRegKeySca, (state_we ==
SP2V_HIGH) &&
822   ((key_len_i == AES_128 && u_aes_cipher_control.rnd_ctr
== 4'd10) ||
823   (key_len_i == AES_192 && u_aes_cipher_control.rnd_ctr
== 4'd12) ||
824   (key_len_i == AES_256 && u_aes_cipher_control.rnd_ctr
== 4'd14)) |>=
825   (state_q != $past(add_round_key_out)) ||
826   (state_q == $past(state_init_i)) ||
827   $past(prd_clearing_equals_output) || alert_o)
828
829   if (SecMasking) begin : gen_sec_cm_key_masking_svas
830     // The number of clock cycles a regular AES round takes
- only used for assertions.
831
832     localparam int unsigned NumCyclesPerRound = (SecSBoxImpl
== SBoxImplDom) ? 5 : 1;
833     logic unused_param;
834     assign unused_param = (NumCyclesPerRound == 1);
835     // Ensure that SubBytes gets fresh PRD input for every
evaluation unless mask forcing is
836     // enabled. We effectively check that the PRNG has been
updated at least once within the
837     // last NumCyclesPerRound cycles. This also holds for
the very first round, as the PRNG
838
839     // is always updated in the last cycle of the IDLE state
and/or the first cycle of the
840     // INIT state.
841     `CALIPTRA_ASSERT(AesSecCmKeyMaskingPrdSubBytes,
842       sub_bytes_en == SP2V_HIGH && ($past(sub_bytes_en) ==
SP2V_LOW ||
843       ($past(sub_bytes_out_req) == SP2V_HIGH &&
844       $past(sub_bytes_out_ack) == SP2V_HIGH)) |>=
845       $past(prd_sub_bytes_q) != $past(prd_sub_bytes_q,
NumCyclesPerRound + 1) ||
846       SecAllowForcingMasks && force_masks_i)
847
848     // Ensure that the PRNG has been updated between masking
the input and starting the first
849     // SubBytes evaluation/KeyExpand operation unless mask
forcing is enabled. For AES-256,
850
851     // we just spend 1 cycle in the INIT state and KeyExpand
isn't evaluating its S-Boxes,
852     // i.e., no fresh randomness is required. For the other
key lengths, KeyExpand evaluates
853     // its S-Boxes which takes NumCyclesPerRound cycles.
When computing the start key for
854     // decryption, the input isn't loaded and the PRNG is
thus not advanced.
855     `CALIPT-
RA_ASSERT(AesSecCmKeyMaskingInitialPrngUpdateSubBytes,
856       sub_bytes_en == SP2V_HIGH && $past(sub_bytes_en) ==
SP2V_LOW |>=
857       (key_len_i == AES_256 &&
858       $past(prd_masking) != $past(prd_masking, 3)) ||
859       ((key_len_i == AES_128 || key_len_i == AES_192) &&
860       $past(prd_masking) != $past(prd_masking, Num-
CyclesPerRound + 2)) ||
861       (SecAllowForcingMasks && force_masks_i))
862     `CALIPT-
RA_ASSERT(AesSecCmKeyMaskingInitialPrngUpdateKeyExpand,
863       key_expand_en == SP2V_HIGH && $past(key_expand_en)
== SP2V_LOW |>=
864       (key_len_i == AES_256 &&
865       $past(prd_masking) != $past(prd_masking, 3)) ||
866       ((key_len_i == AES_128 || key_len_i == AES_192) &&
867       $past(prd_masking) != $past(prd_masking, 2)) ||
868       (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
869
870     // Ensure none of the state shares keeps being constant
during encryption/decryption
871     // unless mask forcing is enabled. Even though unlikely
it's not impossible that one
872
873     // share remains constant throughout one round. The SVAS
thus only fire if a share
874     // remains constant across two rounds.
875     for (genvar s = 0; s < NumShares; s++) begin :
gen_sec_cm_key_masking_share_svas
876       `CALIPTRA_ASSERT(AesSecCmKeyMaskingStateShare,
state_we == SP2V_HIGH &&
877         (crypt_i == SP2V_HIGH || crypt_o == SP2V_HIGH) |>=
878         state_q[s] != $past(state_q[s], NumCyclesPerRound)
||
879         $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
880         (SecAllowForcingMasks && force_masks_i) ||

```

```

dec_key_gen_o == SP2V_HIGH)
877 `CALIPTRA_ASSERT(AesSecCmKeyMaskingOutputShare,
878 (out_valid_o == SP2V_HIGH && $past(out_valid_o) ==
SP2V_LOW) &&
879 (crypt_o == SP2V_HIGH) | =>
880 $past(state_o[s]) != $past(state_q[s], NumCycles-
PerRound) ||
881 $past(state_q[s], NumCyclesPerRound) !=
$past(state_q[s], 2*NumCyclesPerRound) ||
882 (SecAllowForcingMasks && force_masks_i) ||
dec_key_gen_o == SP2V_HIGH)
883 end
884 end
885
886 // Make sure the output of the masking PRNG is properly ex-
tracted without creating overlaps
887 // in the data input masks, or between the PRD fed to the
key expand module and SubBytes.
888 if (WidthPRDSBox > 8) begin : gen_prd_extract_assert
889 // For one row of the state matrix, extract the WidthPRDS-
Box-8 MSBs of the per-S-Box PRD from
890 // the PRNG output.
891 function automatic logic [3:0][(WidthPRDSBox-8)-1:0]
aes_prd_get_msbs(
892 logic [(4*WidthPRDSBox)-1:0] in
893 );
894 logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs;
895 for (int i = 0; i < 4; i++) begin
896 prd_msbs[i] = in[(i*WidthPRDSBox) + 8 +: (WidthPRDS-
Box-8)];
897 end
898 return prd_msbs;
899 endfunction
900
901 // For one row of the state matrix, undo the extraction of
LSBs and MSBs of the per-S-Box PRD
902 // from the PRNG output. This can be used to verify proper
extraction (no overlap of output
903 // masks and PRD for masked Canright S-Box implementa-
tions, no unused PRNG output).
904 function automatic logic [4*WidthPRDSBox-1:0] aes_prd_con-
cat_bits(
905 logic [3:0] [7:0] prd_lsb,
906 logic [3:0][(WidthPRDSBox-8)-1:0] prd_msbs
907 );
908 logic [(4*WidthPRDSBox)-1:0] prd;
909 for (int i = 0; i < 4; i++) begin
910 prd[(i*WidthPRDSBox) +: WidthPRDSBox] = {prd_msbs[i],
prd_lsb[i]};
911 end
912 return prd;
913 endfunction
914
915 // Check for correct extraction of masking PRNG output
without overlaps.
916 logic [WidthPRDMasking-1:0] unused_prd_masking;
917 logic [3:0][3:0][(WidthPRDSBox-8)-1:0] unused_prd_msbs;
918 for (genvar i = 0; i < 4; i++) begin : gen_unused_prd_msbs
919 assign unused_prd_msbs[i] =
aes_prd_get_msbs(prd_masking[i] * WidthPRDRow +: Width-
PRDRow);
920 end
921 for (genvar i = 0; i < 4; i++) begin : gen_un-
used_prd_masking
922 assign unused_prd_masking[i] * WidthPRDRow +: Width-
PRDRow] =
aes_prd_concat_bits(data_in_mask[i], un-
used_prd_msbs[i]);
923 end
924 assign unused_prd_masking[WidthPRDMasking-1 -: Width-
PRDKey] = prd_key_expand;
925 `CALIPTRA_ASSERT(AesMskgPrdExtraction, prd_masking == un-
used_prd_masking)
926 end
927 //VCS coverage on
928 // pragma coverage on
929 `endif
930
931 endmodule

```

7. chipsalliance_caliptra-rtl/src/aes/rtl/aes_clp_reg.sv (1,929 lines)

```

1 // Generated by PeakRDL-regblock - A free and open-source Sys-
temVerilog generator
2 // https://github.com/SystemRDL/PeakRDL-regblock
3
4 module aes_clp_reg (
5     input wire clk,
6     input wire rst,
7
8     input wire s_cpuif_req,
9     input wire s_cpuif_req_is_wr,
10    input wire [10:0] s_cpuif_addr,
11    input wire [31:0] s_cpuif_wr_data,
12    input wire [31:0] s_cpuif_wr_biten,
13    output wire s_cpuif_req_stall_wr,
14    output wire s_cpuif_req_stall_rd,

```

```

15    output wire s_cpuif_rd_ack,
16    output wire s_cpuif_rd_err,
17    output wire [31:0] s_cpuif_rd_data,
18    output wire s_cpuif_wr_ack,
19    output wire s_cpuif_wr_err,
20
21    input aes_clp_reg_pkg::aes_clp_reg__in_t hwif_in,
22    output aes_clp_reg_pkg::aes_clp_reg__out_t hwif_out
23 );
24
25 //-----
26 // CPU Bus interface logic
27 //-----
28
29 logic cpuif_req;
30 logic cpuif_req_is_wr;
31 logic [10:0] cpuif_addr;
32 logic [31:0] cpuif_wr_data;
33 logic [31:0] cpuif_wr_biten;
34 logic cpuif_req_stall_wr;
35 logic cpuif_req_stall_rd;
36
37 logic cpuif_rd_ack;
38 logic cpuif_rd_err;
39 logic [31:0] cpuif_rd_data;
40
41 logic cpuif_wr_ack;
42 logic cpuif_wr_err;
43
44 assign cpuif_req = s_cpuif_req;
45 assign cpuif_req_is_wr = s_cpuif_req_is_wr;
46 assign cpuif_addr = s_cpuif_addr;
47 assign cpuif_wr_data = s_cpuif_wr_data;
48 assign cpuif_wr_biten = s_cpuif_wr_biten;
49 assign s_cpuif_req_stall_wr = cpuif_req_stall_wr;
50 assign s_cpuif_req_stall_rd = cpuif_req_stall_rd;
51 assign s_cpuif_rd_ack = cpuif_rd_ack;
52 assign s_cpuif_rd_err = cpuif_rd_err;
53 assign s_cpuif_rd_data = cpuif_rd_data;
54 assign s_cpuif_wr_ack = cpuif_wr_ack;
55 assign s_cpuif_wr_err = cpuif_wr_err;
56
57 logic cpuif_req_masked;
58
59 // Read & write latencies are balanced. Stalls not re-
quired
60 assign cpuif_req_stall_rd = '0;
61 assign cpuif_req_stall_wr = '0;
62 assign cpuif_req_masked = cpuif_req
& !(cpuif_req_is_wr &
cpuif_req_stall_rd)
& !(cpuif_req_is_wr &
cpuif_req_stall_wr);
63
64 //-----
65 // Address Decode
66 //-----
67
68 typedef struct packed{
69     logic [2-1:0]AES_NAME;
70     logic [2-1:0]AES_VERSION;
71     logic [9-1:0]ENTROPY_IF_SEED;
72     logic CTRL0;
73     logic AES_KV_RD_KEY_CTRL;
74     logic AES_KV_RD_KEY_STATUS;
75     logic AES_KV_WR_CTRL;
76     logic AES_KV_WR_STATUS;
77     struct packed{
78         logic global_intr_en_r;
79         logic error_intr_en_r;
80         logic notif_intr_en_r;
81         logic error_global_intr_r;
82         logic notif_global_intr_r;
83         logic error_internal_intr_r;
84         logic notif_internal_intr_r;
85         logic error_intr_trig_r;
86         logic notif_intr_trig_r;
87         logic error0_intr_count_r;
88         logic error1_intr_count_r;
89         logic error2_intr_count_r;
90         logic error3_intr_count_r;
91         logic notif_cmd_done_intr_count_r;
92         logic error0_intr_count_incr_r;
93         logic error1_intr_count_incr_r;
94         logic error2_intr_count_incr_r;
95         logic error3_intr_count_incr_r;
96         logic notif_cmd_done_intr_count_incr_r;
97     } intr_block_rf;
98 } decoded_reg_strb_t;
99 decoded_reg_strb_t decoded_reg_strb;
100 logic decoded_req;
101 logic decoded_req_is_wr;
102 logic [31:0] decoded_wr_data;
103 logic [31:0] decoded_wr_biten;
104
105 always_comb begin
106     for(int i0=0; i0<2; i0++) begin
107         decoded_reg_strb.AES_NAME[i0] = cpuif_req_masked &

```

```

(cpuif_addr == 11'h0 + i0*11'h4);
108     end
109     for(int i0=0; i0<2; i0++) begin
110         decoded_reg_strb.AES_VERSION[i0] =
cpuif_req_masked & (cpuif_addr == 11'h8 + i0*11'h4);
111     end
112     for(int i0=0; i0<9; i0++) begin
113         decoded_reg_strb.ENTROPY_IF_SEED[i0] =
cpuif_req_masked & (cpuif_addr == 11'h10 + i0*11'h4);
114     end
115     decoded_reg_strb.CTRL0 = cpuif_req_masked &
(cpuif_addr == 11'h134);

116     decoded_reg_strb.AES_KV_RD_KEY_CTRL = cpuif_req_masked &
(cpuif_addr == 11'h200);
117     decoded_reg_strb.AES_KV_RD_KEY_STATUS =
cpuif_req_masked & (cpuif_addr == 11'h204);
118     decoded_reg_strb.AES_KV_WR_CTRL = cpuif_req_masked &
(cpuif_addr == 11'h208);

119     decoded_reg_strb.AES_KV_WR_STATUS = cpuif_req_masked &
(cpuif_addr == 11'h20c);
120     decoded_reg_strb.intr_block_rf.global_intr_en_r =
cpuif_req_masked & (cpuif_addr == 11'h400);
121     decoded_reg_strb.intr_block_rf.error_intr_en_r =
cpuif_req_masked & (cpuif_addr == 11'h404);
122     decoded_reg_strb.intr_block_rf.notif_intr_en_r =
cpuif_req_masked & (cpuif_addr == 11'h408);
123     decoded_reg_strb.intr_block_rf.error_global_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h40c);
124     decoded_reg_strb.intr_block_rf.notif_global_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h410);

125     decoded_reg_strb.intr_block_rf.error_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h414);

126     decoded_reg_strb.intr_block_rf.notif_internal_intr_r =
cpuif_req_masked & (cpuif_addr == 11'h418);
127     decoded_reg_strb.intr_block_rf.error_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 11'h41c);
128     decoded_reg_strb.intr_block_rf.notif_intr_trig_r =
cpuif_req_masked & (cpuif_addr == 11'h420);
129     decoded_reg_strb.intr_block_rf.error0_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h500);
130     decoded_reg_strb.intr_block_rf.error1_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h504);
131     decoded_reg_strb.intr_block_rf.error2_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h508);
132     decoded_reg_strb.intr_block_rf.error3_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h50c);
133     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r =
cpuif_req_masked & (cpuif_addr == 11'h580);
134     de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h600);
135     de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h604);
136     de-
coded_reg_strb.intr_block_rf.error2_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h608);
137     de-
coded_reg_strb.intr_block_rf.error3_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h60c);
138     de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r =
cpuif_req_masked & (cpuif_addr == 11'h610);
139     end
140
141     // Pass down signals to next stage
142     assign decoded_req = cpuif_req_masked;
143     assign decoded_req_is_wr = cpuif_req_is_wr;
144     assign decoded_wr_data = cpuif_wr_data;
145     assign decoded_wr_biten = cpuif_wr_biten;
146
147     //-----
148     // Field logic
149     //-----
150     typedef struct packed{
151         struct packed{
152             struct packed{
153                 logic [31:0] next;
154                 logic load_next;
155             } ENTROPY_IF_SEED;
156         } [9:1:0]ENTROPY_IF_SEED;
157         struct packed{
158             struct packed{
159                 logic next;
160                 logic load_next;
161             } ENDIAN_SWAP;
162         } CTRL0;
163         struct packed{
164             struct packed{
165                 logic next;
166                 logic load_next;
167             } read_en;
168             struct packed{
169                 logic [4:0] next;
170                 logic load_next;
171             } read_entry;
172             struct packed{
173                 logic next;
174                 logic load_next;
175             } pcr_hash_extend;
176             struct packed{
177                 logic [24:0] next;
178                 logic load_next;
179             } rsvd;
180         } AES_KV_RD_KEY_CTRL;
181         struct packed{
182             struct packed{
183                 logic next;
184                 logic load_next;
185             } VALID;
186         } AES_KV_RD_KEY_STATUS;
187         struct packed{
188             struct packed{
189                 logic next;
190                 logic load_next;
191             } write_en;
192             struct packed{
193                 logic [4:0] next;
194                 logic load_next;
195             } write_entry;
196             struct packed{
197                 logic next;
198                 logic load_next;
199             } hmac_key_dest_valid;
200             struct packed{
201                 logic next;
202                 logic load_next;
203             } hmac_block_dest_valid;
204             struct packed{
205                 logic next;
206                 logic load_next;
207             } mldsa_seed_dest_valid;
208             struct packed{
209                 logic next;
210                 logic load_next;
211             } ecc_pkey_dest_valid;
212             struct packed{
213                 logic next;
214                 logic load_next;
215             } ecc_seed_dest_valid;
216             struct packed{
217                 logic next;
218                 logic load_next;
219             } aes_key_dest_valid;
220             struct packed{
221                 logic next;
222                 logic load_next;
223             } mlkem_seed_dest_valid;
224             struct packed{
225                 logic next;
226                 logic load_next;
227             } mlkem_msg_dest_valid;
228             struct packed{
229                 logic next;
230                 logic load_next;
231             } dma_data_dest_valid;
232             struct packed{
233                 logic [16:0] next;
234                 logic load_next;
235             } rsvd;
236         } AES_KV_WR_CTRL;
237         struct packed{
238             struct packed{
239                 logic next;
240                 logic load_next;
241             } VALID;
242         } AES_KV_WR_STATUS;
243         struct packed{
244             struct packed{
245                 struct packed{
246                     logic next;
247                     logic load_next;
248                 } error_en;
249                 struct packed{
250                     logic next;
251                     logic load_next;
252                 } notif_en;
253             } global_intr_en_r;
254             struct packed{
255                 struct packed{
256                     logic next;
257                     logic load_next;
258                 } error0_en;
259                 struct packed{
260                     logic next;
261                     logic load_next;
262                 } error1_en;
263                 struct packed{
264                     logic next;
265                     logic load_next;
266                 } error2_en;
267                 struct packed{
268                     logic next;
269                     logic load_next;
270                 } error3_en;

```



```

271     } error_intr_en_r;
272     struct packed{
273         struct packed{
274             logic next;
275             logic load_next;
276         } notif_cmd_done_en;
277     } notif_intr_en_r;
278     struct packed{
279         struct packed{
280             logic next;
281             logic load_next;
282         } agg_sts;
283     } error_global_intr_r;
284     struct packed{
285         struct packed{
286             logic next;
287             logic load_next;
288         } agg_sts;
289     } notif_global_intr_r;
290     struct packed{
291         struct packed{
292             logic next;
293             logic load_next;
294         } error0_sts;
295         struct packed{
296             logic next;
297             logic load_next;
298         } error1_sts;
299         struct packed{
300             logic next;
301             logic load_next;
302         } error2_sts;
303         struct packed{
304             logic next;
305             logic load_next;
306         } error3_sts;
307     } error_internal_intr_r;
308     struct packed{
309         struct packed{
310             logic next;
311             logic load_next;
312         } notif_cmd_done_sts;
313     } notif_internal_intr_r;
314     struct packed{
315         struct packed{
316             logic next;
317             logic load_next;
318         } error0_trig;
319         struct packed{
320             logic next;
321             logic load_next;
322         } error1_trig;
323         struct packed{
324             logic next;
325             logic load_next;
326         } error2_trig;
327         struct packed{
328             logic next;
329             logic load_next;
330         } error3_trig;
331     } error_intr_trig_r;
332     struct packed{
333         struct packed{
334             logic next;
335             logic load_next;
336         } notif_cmd_done_trig;
337     } notif_intr_trig_r;
338     struct packed{
339         struct packed{
340             logic [31:0] next;
341             logic load_next;
342             logic incrthreshold;
343             logic incrsaturate;
344         } cnt;
345     } error0_intr_count_r;
346     struct packed{
347         struct packed{
348             logic [31:0] next;
349             logic load_next;
350             logic incrthreshold;
351             logic incrsaturate;
352         } cnt;
353     } error1_intr_count_r;
354     struct packed{
355         struct packed{
356             logic [31:0] next;
357             logic load_next;
358             logic incrthreshold;
359             logic incrsaturate;
360         } cnt;
361     } error2_intr_count_r;
362     struct packed{
363         struct packed{
364             logic [31:0] next;
365             logic load_next;
366             logic incrthreshold;
367             logic incrsaturate;
368         } cnt;
369     } error3_intr_count_r;
370     struct packed{
371         struct packed{
372             logic [31:0] next;
373             logic load_next;
374             logic incrthreshold;
375             logic incrsaturate;
376         } cnt;
377     } notif_cmd_done_intr_count_r;
378     struct packed{
379         struct packed{
380             logic next;
381             logic load_next;
382             logic decrthreshold;
383             logic underflow;
384         } pulse;
385     } error0_intr_count_incr_r;
386     struct packed{
387         struct packed{
388             logic next;
389             logic load_next;
390             logic decrthreshold;
391             logic underflow;
392         } pulse;
393     } error1_intr_count_incr_r;
394     struct packed{
395         struct packed{
396             logic next;
397             logic load_next;
398             logic decrthreshold;
399             logic underflow;
400         } pulse;
401     } error2_intr_count_incr_r;
402     struct packed{
403         struct packed{
404             logic next;
405             logic load_next;
406             logic decrthreshold;
407             logic underflow;
408         } pulse;
409     } error3_intr_count_incr_r;
410     struct packed{
411         struct packed{
412             logic next;
413             logic load_next;
414             logic decrthreshold;
415             logic underflow;
416         } pulse;
417     } notif_cmd_done_intr_count_incr_r;
418     } intr_block_rf;
419     } field_combo_t;
420     field_combo_t field_combo;
421
422     typedef struct packed{
423         struct packed{
424             struct packed{
425                 logic [31:0] value;
426             } ENTROPY_IF_SEED;
427             } [9-1:0]ENTROPY_IF_SEED;
428         struct packed{
429             struct packed{
430                 logic value;
431             } ENDIAN_SWAP;
432         } CTRL0;
433         struct packed{
434             struct packed{
435                 logic value;
436             } read_en;
437             struct packed{
438                 logic [4:0] value;
439             } read_entry;
440             struct packed{
441                 logic value;
442             } pcr_hash_extend;
443             struct packed{
444                 logic [24:0] value;
445             } rsvd;
446         } AES_KV_RD_KEY_CTRL;
447         struct packed{
448             struct packed{
449                 logic value;
450             } VALID;
451         } AES_KV_RD_KEY_STATUS;
452         struct packed{
453             logic value;
454         } write_en;
455         struct packed{
456             logic [4:0] value;
457         } write_entry;
458         struct packed{
459             logic value;
460         } hmac_key_dest_valid;
461         struct packed{
462             logic value;
463         } hmac_block_dest_valid;
464         struct packed{
465             logic value;
466         } mlds_a_seed_dest_valid;
467         struct packed{
468             logic value;
469         } ecc_pkey_dest_valid;
470         struct packed{
471             logic value;
472         }

```

```

473     } ecc_seed_dest_valid;
474     struct packed{
475         logic value;
476     } aes_key_dest_valid;
477     struct packed{
478         logic value;
479     } mlkem_seed_dest_valid;
480     struct packed{
481         logic value;
482     } mlkem_msg_dest_valid;
483     struct packed{
484         logic value;
485     } dma_data_dest_valid;
486     struct packed{
487         logic [16:0] value;
488     } rsvd;
489 } AES_KV_WR_CTRL;
490 struct packed{
491     struct packed{
492         logic value;
493     } VALID;
494 } AES_KV_STATUS;
495 struct packed{
496     struct packed{
497         struct packed{
498             logic value;
499         } error_en;
500         struct packed{
501             logic value;
502         } notif_en;
503     } global_intr_en_r;
504     struct packed{
505         struct packed{
506             logic value;
507         } error0_en;
508         struct packed{
509             logic value;
510         } error1_en;
511         struct packed{
512             logic value;
513         } error2_en;
514         struct packed{
515             logic value;
516         } error3_en;
517     } error_intr_en_r;
518     struct packed{
519         struct packed{
520             logic value;
521         } notif_cmd_done_en;
522     } notif_intr_en_r;
523     struct packed{
524         struct packed{
525             logic value;
526         } agg_sts;
527     } error_global_intr_r;
528     struct packed{
529         struct packed{
530             logic value;
531         } agg_sts;
532     } notif_global_intr_r;
533     struct packed{
534         struct packed{
535             logic value;
536         } error0_sts;
537         struct packed{
538             logic value;
539         } error1_sts;
540         struct packed{
541             logic value;
542         } error2_sts;
543         struct packed{
544             logic value;
545         } error3_sts;
546     } error_internal_intr_r;
547     struct packed{
548         struct packed{
549             logic value;
550         } notif_cmd_done_sts;
551     } notif_internal_intr_r;
552     struct packed{
553         struct packed{
554             logic value;
555         } error0_trig;
556         struct packed{
557             logic value;
558         } error1_trig;
559         struct packed{
560             logic value;
561         } error2_trig;
562         struct packed{
563             logic value;
564         } error3_trig;
565     } error_intr_trig_r;
566     struct packed{
567         struct packed{
568             logic value;
569         } notif_cmd_done_trig;
570     } notif_intr_trig_r;
571     struct packed{
572         struct packed{
573             logic [31:0] value;

```

```

574         } cnt;
575     } error0_intr_count_r;
576     struct packed{
577         struct packed{
578             logic [31:0] value;
579         } cnt;
580     } error1_intr_count_r;
581     struct packed{
582         struct packed{
583             logic [31:0] value;
584         } cnt;
585     } error2_intr_count_r;
586     struct packed{
587         struct packed{
588             logic [31:0] value;
589         } cnt;
590     } error3_intr_count_r;
591     struct packed{
592         struct packed{
593             logic [31:0] value;
594         } cnt;
595     } notif_cmd_done_intr_count_r;
596     struct packed{
597         struct packed{
598             logic value;
599         } pulse;
600     } error0_intr_count_incr_r;
601     struct packed{
602         struct packed{
603             logic value;
604         } pulse;
605     } error1_intr_count_incr_r;
606     struct packed{
607         struct packed{
608             logic value;
609         } pulse;
610     } error2_intr_count_incr_r;
611     struct packed{
612         struct packed{
613             logic value;
614         } pulse;
615     } error3_intr_count_incr_r;
616     struct packed{
617         struct packed{
618             logic value;
619         } pulse;
620     } notif_cmd_done_intr_count_incr_r;
621     } intr_block_rf;
622 } field_storage_t;
623 field_storage_t field_storage;
624
625 for(genvar i0=0; i0<9; i0++) begin
626     // Field: aes_clp_reg.EN-
627     TROPY_IF_SEED[i].ENTROPY_IF_SEED
628     always_comb begin
629         automatic logic [31:0] next_c;
630         automatic logic load_next_c;
631         next_c =
632         field_storage.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.value;
633         load_next_c = '0;
634         if(decoded_reg_strb.ENTROPY_IF_SEED[i0] && de-
635         coded_req_is_wr) begin // SW write
636             next_c =
637             (field_storage.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.value & ~de-
638             coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
639             coded_wr_biten[31:0]);
640             load_next_c = '1;
641         end
642         field_combo.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.next = next_c;
643     end
644     field_combo.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.load_next =
645     load_next_c;
646 end
647 always_ff @(posedge clk) begin
648     if(field_combo.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.load_next) begin
649         field_storage.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.value <=
650         field_combo.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.next;
651         end
652         end
653         assign hwif_out.EN-
654         TROPY_IF_SEED[i0].ENTROPY_IF_SEED.value =
655         field_storage.ENTROPY_IF_SEED[i0].ENTROPY_IF_SEED.value;
656         assign hwif_out.EN-
657         TROPY_IF_SEED[i0].ENTROPY_IF_SEED.swmod = de-
658         coded_reg_strb.ENTROPY_IF_SEED[i0] && decoded_req_is_wr;
659         end
660         // Field: aes_clp_reg.CTRL0.ENDIAN_SWAP
661         always_comb begin
662             automatic logic [0:0] next_c;
663             automatic logic load_next_c;
664             next_c = field_storage.CTRL0.ENDIAN_SWAP.value;
665             load_next_c = '0;
666             if(decoded_reg_strb.CTRL0 && decoded_req_is_wr) be-
667             gin // SW write
668                 next_c = (field_storage.CTRL0.ENDIAN_SWAP.value &
669                 ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
670                 coded_wr_biten[0:0]);

```



```

656         load_next_c = '1';
657     end
658     field_combo.CTRL0.ENDIAN_SWAP.next = next_c;
659     field_combo.CTRL0.ENDIAN_SWAP.load_next = load_next_c;
660 end
661 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
662     if(~hwif_in.reset_b) begin
663         field_storage.CTRL0.ENDIAN_SWAP.value <= 1'h0;
664     end else if(field_combo.CTRL0.ENDIAN_SWAP.load_next)
begin
665         field_storage.CTRL0.ENDIAN_SWAP.value <=
field_combo.CTRL0.ENDIAN_SWAP.next;
666     end
667 end
668 assign hwif_out.CTRL0.ENDIAN_SWAP.value =
field_storage.CTRL0.ENDIAN_SWAP.value;
669 // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.read_en
670 always_comb begin
671     automatic logic [0:0] next_c;
672     automatic logic load_next_c;
673     next_c =
field_storage.AES_KV_RD_KEY_CTRL.read_en.value;
674     load_next_c = '0;
675     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.read_en.swve) begin //
SW write
676         next_c =
(field_storage.AES_KV_RD_KEY_CTRL.read_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
677         load_next_c = '1;
678     end else if(hwif_in.AES_KV_RD_KEY_CTRL.read_en.hwclr)
begin // HW Clear
679         next_c = '0;
680         load_next_c = '1;
681     end
682     field_combo.AES_KV_RD_KEY_CTRL.read_en.next = next_c;
683     field_combo.AES_KV_RD_KEY_CTRL.read_en.load_next =
load_next_c;
684 end
685 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
686     if(~hwif_in.reset_b) begin
687         field_storage.AES_KV_RD_KEY_CTRL.read_en.value <=
1'h0;
688     end else
if(field_combo.AES_KV_RD_KEY_CTRL.read_en.load_next) begin
689         field_storage.AES_KV_RD_KEY_CTRL.read_en.value <=
field_combo.AES_KV_RD_KEY_CTRL.read_en.next;
690     end
691 end
692 assign hwif_out.AES_KV_RD_KEY_CTRL.read_en.value =
field_storage.AES_KV_RD_KEY_CTRL.read_en.value;
693 // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.read_entry
694 always_comb begin
695     automatic logic [4:0] next_c;
696     automatic logic load_next_c;
697     next_c =
field_storage.AES_KV_RD_KEY_CTRL.read_entry.value;
698     load_next_c = '0;
699     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.read_entry.swve) be-
gin // SW write
700         next_c =
(field_storage.AES_KV_RD_KEY_CTRL.read_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
701         load_next_c = '1;
702     end
703     field_combo.AES_KV_RD_KEY_CTRL.read_entry.next =
next_c;
704     field_combo.AES_KV_RD_KEY_CTRL.read_entry.load_next =
load_next_c;
705 end
706 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
707     if(~hwif_in.reset_b) begin
708         field_storage.AES_KV_RD_KEY_CTRL.read_entry.value
<= 5'h0;
709     end else
if(field_combo.AES_KV_RD_KEY_CTRL.read_entry.load_next) begin
710         field_storage.AES_KV_RD_KEY_CTRL.read_entry.value
<= field_combo.AES_KV_RD_KEY_CTRL.read_entry.next;
711     end
712 end
713 assign hwif_out.AES_KV_RD_KEY_CTRL.read_entry.value =
field_storage.AES_KV_RD_KEY_CTRL.read_entry.value;
714 // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.pcr_hash_extend
715 always_comb begin
716     automatic logic [0:0] next_c;
717     automatic logic load_next_c;
718     next_c =
field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value;
719     load_next_c = '0;
720     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.pcr_hash_extend.swve)
begin // SW write
721         next_c =
(field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
722         load_next_c = '1;
723     end

```

```

724         field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.next =
next_c;
725     field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.load_next =
load_next_c;
726 end
727 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
728     if(~hwif_in.reset_b) begin
729         field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value <= 1'h0;
730     end else
if(field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.load_next) begin
731         field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value <=
field_combo.AES_KV_RD_KEY_CTRL.pcr_hash_extend.next;
732     end
733 end
734 assign hwif_out.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value =
field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value;
735 // Field: aes_clp_reg.AES_KV_RD_KEY_CTRL.rsvd
736 always_comb begin
737     automatic logic [24:0] next_c;
738     automatic logic load_next_c;
739     next_c = field_storage.AES_KV_RD_KEY_CTRL.rsvd.value;
740     load_next_c = '0;
741     if(decoded_reg_strb.AES_KV_RD_KEY_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_RD_KEY_CTRL.rsvd.swve) begin // SW
write
742         next_c =
(field_storage.AES_KV_RD_KEY_CTRL.rsvd.value & ~de-
coded_wr_biten[31:7]) | (decoded_wr_data[31:7] & de-
coded_wr_biten[31:7]);
743         load_next_c = '1;
744     end
745     field_combo.AES_KV_RD_KEY_CTRL.rsvd.next = next_c;
746     field_combo.AES_KV_RD_KEY_CTRL.rsvd.load_next =
load_next_c;
747 end
748 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
749     if(~hwif_in.reset_b) begin
750         field_storage.AES_KV_RD_KEY_CTRL.rsvd.value <=
25'h0;
751     end else
if(field_combo.AES_KV_RD_KEY_CTRL.rsvd.load_next) begin
752         field_storage.AES_KV_RD_KEY_CTRL.rsvd.value <=
field_combo.AES_KV_RD_KEY_CTRL.rsvd.next;
753     end
754 end
755 assign hwif_out.AES_KV_RD_KEY_CTRL.rsvd.value =
field_storage.AES_KV_RD_KEY_CTRL.rsvd.value;
756 // Field: aes_clp_reg.AES_KV_RD_KEY_STATUS.VALID
757 always_comb begin
758     automatic logic [0:0] next_c;
759     automatic logic load_next_c;
760     next_c =
field_storage.AES_KV_RD_KEY_STATUS.VALID.value;
761     load_next_c = '0;
762     if(hwif_in.AES_KV_RD_KEY_STATUS.VALID.hwset) begin //
HW Set
763         next_c = '1;
764         load_next_c = '1;
765     end else if(hwif_in.AES_KV_RD_KEY_STATUS.VALID.hwclr)
begin // HW Clear
766         next_c = '0;
767         load_next_c = '1;
768     end
769     field_combo.AES_KV_RD_KEY_STATUS.VALID.next = next_c;
770     field_combo.AES_KV_RD_KEY_STATUS.VALID.load_next =
load_next_c;
771 end
772 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
773     if(~hwif_in.reset_b) begin
774         field_storage.AES_KV_RD_KEY_STATUS.VALID.value <=
1'h0;
775     end else
if(field_combo.AES_KV_RD_KEY_STATUS.VALID.load_next) begin
776         field_storage.AES_KV_RD_KEY_STATUS.VALID.value <=
field_combo.AES_KV_RD_KEY_STATUS.VALID.next;
777     end
778 end
779 // Field: aes_clp_reg.AES_KV_WR_CTRL.write_en
780 always_comb begin
781     automatic logic [0:0] next_c;
782     automatic logic load_next_c;
783     next_c = field_storage.AES_KV_WR_CTRL.write_en.value;
784     load_next_c = '0;
785     if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.write_en.swve) begin // SW
write
786         next_c =
(field_storage.AES_KV_WR_CTRL.write_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
787         load_next_c = '1;
788     end else if(hwif_in.AES_KV_WR_CTRL.write_en.hwclr) be-
gin // HW Clear
789         next_c = '0;
790         load_next_c = '1;
791     end
792     field_combo.AES_KV_WR_CTRL.write_en.next = next_c;

```

```

793     field_combo.AES_KV_WR_CTRL.write_en.load_next =
load_next_c;
794     end
795     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
796         if(~hwif_in.reset_b) begin
797             field_storage.AES_KV_WR_CTRL.write_en.value <=
1'h0;
798         end else
if(field_combo.AES_KV_WR_CTRL.write_en.load_next) begin
799             field_storage.AES_KV_WR_CTRL.write_en.value <=
field_combo.AES_KV_WR_CTRL.write_en.next;
800         end
801     end
802     assign hwif_out.AES_KV_WR_CTRL.write_en.value =
field_storage.AES_KV_WR_CTRL.write_en.value;
803     // Field: aes_clp_reg.AES_KV_WR_CTRL.write_entry
804     always_comb begin
805         automatic logic [4:0] next_c;
806         automatic logic load_next_c;
807         next_c =
field_storage.AES_KV_WR_CTRL.write_entry.value;
808         load_next_c = '0;
809         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.write_entry.swwe) begin //
SW write
810             next_c =
(field_storage.AES_KV_WR_CTRL.write_entry.value & ~de-
coded_wr_biten[5:1]) | (decoded_wr_data[5:1] & de-
coded_wr_biten[5:1]);
811             load_next_c = '1;
812         end
813         field_combo.AES_KV_WR_CTRL.write_entry.next = next_c;
814         field_combo.AES_KV_WR_CTRL.write_entry.load_next =
load_next_c;
815     end
816     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
817         if(~hwif_in.reset_b) begin
818             field_storage.AES_KV_WR_CTRL.write_entry.value <=
5'h0;
819         end else
if(field_combo.AES_KV_WR_CTRL.write_entry.load_next) begin
820             field_storage.AES_KV_WR_CTRL.write_entry.value <=
field_combo.AES_KV_WR_CTRL.write_entry.next;
821         end
822     end
823     assign hwif_out.AES_KV_WR_CTRL.write_entry.value =
field_storage.AES_KV_WR_CTRL.write_entry.value;
824     // Field: aes_clp_reg.AES_KV_WR_CTRL.hmac_key_dest_valid
825     always_comb begin
826         automatic logic [0:0] next_c;
827         automatic logic load_next_c;
828         next_c =
field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value;
829         load_next_c = '0;
830         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.hmac_key_dest_valid.swwe)
begin // SW write
831             next_c =
(field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value & ~de-
coded_wr_biten[6:6]) | (decoded_wr_data[6:6] & de-
coded_wr_biten[6:6]);
832             load_next_c = '1;
833         end
834         field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.next =
next_c;
835     end
836     field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.load_next =
load_next_c;
837     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
838         if(~hwif_in.reset_b) begin
839             field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value <=
1'h0;
840         end else
if(field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.load_next) begin
841             field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.hmac_key_dest_valid.next;
842         end
843     end
844     assign hwif_out.AES_KV_WR_CTRL.hmac_key_dest_valid.value =
field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value;
845     // Field: aes_clp_reg.AES_KV_WR_CTRL.hmac_block_dest_valid
846     always_comb begin
847         automatic logic [0:0] next_c;
848         automatic logic load_next_c;
849         next_c =
field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value;
850         load_next_c = '0;
851         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_C-
TRL.hmac_block_dest_valid.swwe) begin // SW write
852             next_c =
(field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value & ~de-
coded_wr_biten[7:7]) | (decoded_wr_data[7:7] & de-
coded_wr_biten[7:7]);
853             load_next_c = '1;
854         end
855         field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.next
= next_c;

```

```

856     field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.load_next =
load_next_c;
857     end
858     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
859         if(~hwif_in.reset_b) begin
860             field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value <=
1'h0;
861         end else
if(field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.load_next) begin
862             field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.hmac_block_dest_valid.next;
863         end
864     end
865     assign hwif_out.AES_KV_WR_CTRL.hmac_block_dest_valid.value =
field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value;
866     // Field: aes_clp_reg.AES_KV_WR_CTRL.mlds_seed_dest_valid
867     always_comb begin
868         automatic logic [0:0] next_c;
869         automatic logic load_next_c;
870         next_c =
field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value;
871         load_next_c = '0;
872         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_C-
TRL.mlds_seed_dest_valid.swwe) begin // SW write
873             next_c =
(field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value & ~de-
coded_wr_biten[8:8]) | (decoded_wr_data[8:8] & de-
coded_wr_biten[8:8]);
874             load_next_c = '1;
875         end
876         field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.next
= next_c;
877     end
878     field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.load_next =
load_next_c;
879     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
880         if(~hwif_in.reset_b) begin
881             field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value <=
1'h0;
882         end else
if(field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.load_next) begin
883             field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.mlds_seed_dest_valid.next;
884         end
885     end
886     assign hwif_out.AES_KV_WR_CTRL.mlds_seed_dest_valid.value =
field_storage.AES_KV_WR_CTRL.mlds_seed_dest_valid.value;
887     // Field: aes_clp_reg.AES_KV_WR_CTRL.ecc_pkey_dest_valid
888     always_comb begin
889         automatic logic [0:0] next_c;
890         automatic logic load_next_c;
891         next_c =
field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value;
892         load_next_c = '0;
893         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.ecc_pkey_dest_valid.swwe)
begin // SW write
894             next_c =
(field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value & ~de-
coded_wr_biten[9:9]) | (decoded_wr_data[9:9] & de-
coded_wr_biten[9:9]);
895             load_next_c = '1;
896         end
897         field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.next =
next_c;
898     end
899     field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.load_next =
load_next_c;
900     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
901         if(~hwif_in.reset_b) begin
902             field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
1'h0;
903         end else
if(field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.load_next) begin
904             field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.ecc_pkey_dest_valid.next;
905         end
906     end
907     assign hwif_out.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value =
field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value;
908     // Field: aes_clp_reg.AES_KV_WR_CTRL.ecc_seed_dest_valid
909     always_comb begin
910         automatic logic [0:0] next_c;
911         automatic logic load_next_c;
912         next_c =
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value;
913         load_next_c = '0;
914         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.ecc_seed_dest_valid.swwe)
begin // SW write
915             next_c =

```

```

(field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value & ~de-
coded_wr_biten[10:10]) | (decoded_wr_data[10:10] & de-
coded_wr_biten[10:10]);
916     load_next_c = '1;
917     end
918     field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.next =
next_c;
919
field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.load_next =
load_next_c;
920     end
921     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
922         if(~hwif_in.reset_b) begin
923
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value <= 1'h0;
924         end else
925         if(field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.load_next) begin
926             field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.ecc_seed_dest_valid.next;
927         end
928     end
929     assign hwif_out.AES_KV_WR_CTRL.ecc_seed_dest_valid.value =
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value;
930     // Field: aes_clp_reg.AES_KV_WR_CTRL.aes_key_dest_valid
931     always_comb begin
932         automatic logic [0:0] next_c;
933         automatic logic load_next_c;
934         next_c =
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value;
935         load_next_c = '0;
936         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.aes_key_dest_valid.swwe)
begin // SW write
937             next_c =
(field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value & ~de-
coded_wr_biten[11:11]) | (decoded_wr_data[11:11] & de-
coded_wr_biten[11:11]);
938             load_next_c = '1;
939         end
940         field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.next =
next_c;
941     end
942     field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.load_next =
load_next_c;
943     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
944         if(~hwif_in.reset_b) begin
945
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value <= 1'h0;
946         end else
947         if(field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.load_next) begin
948             field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.aes_key_dest_valid.next;
949         end
950         assign hwif_out.AES_KV_WR_CTRL.aes_key_dest_valid.value =
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value;
951         // Field: aes_clp_reg.AES_KV_WR_CTRL.mlkem_seed_dest_valid
952         always_comb begin
953             automatic logic [0:0] next_c;
954             automatic logic load_next_c;
955             next_c =
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value;
956             load_next_c = '0;
957             if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.mlkem_seed_dest_valid.swwe)
begin // SW write
958                 next_c =
(field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value & ~de-
coded_wr_biten[12:12]) | (decoded_wr_data[12:12] & de-
coded_wr_biten[12:12]);
959                 load_next_c = '1;
960             end
961             field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.next =
next_c;
962         end
963         field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.load_next =
load_next_c;
964         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
965             if(~hwif_in.reset_b) begin
966
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value <= 1'h0;
967             end else
968             if(field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.load_next) begin
969                 field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.mlkem_seed_dest_valid.next;
970             end
971         end
972         assign hwif_out.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value =
field_storage.AES_KV_WR_CTRL.mlkem_seed_dest_valid.value;
973         // Field: aes_clp_reg.AES_KV_WR_CTRL.mlkem_msg_dest_valid
974         always_comb begin
975             automatic logic [0:0] next_c;
976             automatic logic load_next_c;
977             next_c =

```

```

field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value;
976         load_next_c = '0;
977         if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.mlkem_msg_dest_valid.swwe)
begin // SW write
978             next_c =
(field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value & ~de-
coded_wr_biten[13:13]) | (decoded_wr_data[13:13] & de-
coded_wr_biten[13:13]);
979             load_next_c = '1;
980         end
981     end
982     field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.next =
next_c;
983     field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.load_next =
load_next_c;
984     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
985         if(~hwif_in.reset_b) begin
986
field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value <= 1'h0;
987         end else
988         if(field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.load_next) begin
989             field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.mlkem_msg_dest_valid.next;
990         end
991         assign hwif_out.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value =
field_storage.AES_KV_WR_CTRL.mlkem_msg_dest_valid.value;
992         // Field: aes_clp_reg.AES_KV_WR_CTRL.dma_data_dest_valid
993         always_comb begin
994             automatic logic [0:0] next_c;
995             automatic logic load_next_c;
996             next_c =
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value;
997             load_next_c = '0;
998             if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.dma_data_dest_valid.swwe)
begin // SW write
999                 next_c =
(field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value & ~de-
coded_wr_biten[14:14]) | (decoded_wr_data[14:14] & de-
coded_wr_biten[14:14]);
1000                 load_next_c = '1;
1001             end
1002             field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.next =
next_c;
1003         end
1004         field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.load_next =
load_next_c;
1005         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1006             if(~hwif_in.reset_b) begin
1007
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value <= 1'h0;
1008             end else
1009             if(field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.load_next) begin
1010                 field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value <=
field_combo.AES_KV_WR_CTRL.dma_data_dest_valid.next;
1011             end
1012         end
1013         assign hwif_out.AES_KV_WR_CTRL.dma_data_dest_valid.value =
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value;
1014         // Field: aes_clp_reg.AES_KV_WR_CTRL.rsvd
1015         always_comb begin
1016             automatic logic [16:0] next_c;
1017             automatic logic load_next_c;
1018             next_c = field_storage.AES_KV_WR_CTRL.rsvd.value;
1019             load_next_c = '0;
1020             if(decoded_reg_strb.AES_KV_WR_CTRL && de-
coded_req_is_wr && hwif_in.AES_KV_WR_CTRL.rsvd.swwe) begin // SW
write
1021                 next_c = (field_storage.AES_KV_WR_CTRL.rsvd.value
& ~decoded_wr_biten[31:15]) | (decoded_wr_data[31:15] & de-
coded_wr_biten[31:15]);
1022                 load_next_c = '1;
1023             end
1024             field_combo.AES_KV_WR_CTRL.rsvd.next = next_c;
1025             field_combo.AES_KV_WR_CTRL.rsvd.load_next =
load_next_c;
1026         end
1027         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1028             if(~hwif_in.reset_b) begin
1029                 field_storage.AES_KV_WR_CTRL.rsvd.value <= 17'h0;
1030             end else if(field_combo.AES_KV_WR_CTRL.rsvd.load_next)
begin
1031                 field_storage.AES_KV_WR_CTRL.rsvd.value <=
field_combo.AES_KV_WR_CTRL.rsvd.next;
1032             end
1033         end
1034         assign hwif_out.AES_KV_WR_CTRL.rsvd.value =
field_storage.AES_KV_WR_CTRL.rsvd.value;
1035         // Field: aes_clp_reg.AES_KV_WR_STATUS.VALID
1036         always_comb begin
1037             automatic logic [0:0] next_c;
1038             automatic logic load_next_c;

```

```

1038     next_c = field_storage.AES_KV_WR_STATUS.VALID.value;
1039     load_next_c = '0';
1040     if(hwif_in.AES_KV_WR_STATUS.VALID.hwset) begin // HW
Set
1041         next_c = '1';
1042         load_next_c = '1';
1043     end else if(hwif_in.AES_KV_WR_STATUS.VALID.hwclr) be-
gin // HW Clear
1044         next_c = '0';
1045         load_next_c = '1';
1046     end
1047     field_combo.AES_KV_WR_STATUS.VALID.next = next_c;
1048     field_combo.AES_KV_WR_STATUS.VALID.load_next =
load_next_c;
1049 end
1050 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1051     if(~hwif_in.reset_b) begin
1052         field_storage.AES_KV_WR_STATUS.VALID.value <=
1'h0;
1053     end else
1054     if(field_combo.AES_KV_WR_STATUS.VALID.load_next) begin
1055         field_storage.AES_KV_WR_STATUS.VALID.value <=
field_combo.AES_KV_WR_STATUS.VALID.next;
1056     end
1057     // Field: aes_clp_reg.intr_b-
lock_rf.global_intr_en_r.error_en
1058     always_comb begin
1059         automatic logic [0:0] next_c;
1060         automatic logic load_next_c;
1061         next_c =
field_storage.intr_block_rf.global_intr_en_r.error_en.value;
1062         load_next_c = '0';
1063         if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1064             next_c =
(field_storage.intr_block_rf.global_intr_en_r.error_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1065             load_next_c = '1';
1066         end
1067         field_combo.intr_block_rf.global_intr_en_r.error_en.next = next_c;
1068         field_combo.intr_block_rf.global_intr_en_r.error_en.load_next =
load_next_c;
1069     end
1070     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1071         if(~hwif_in.reset_b) begin
1072             field_storage.intr_block_rf.global_intr_en_r.error_en.value <= 1'h0;
1073         end else
1074         if(field_combo.intr_block_rf.global_intr_en_r.error_en.load_next)
begin
1075             field_storage.intr_block_rf.global_intr_en_r.error_en.value <=
field_combo.intr_block_rf.global_intr_en_r.error_en.next;
1076         end
1077         // Field: aes_clp_reg.intr_b-
lock_rf.global_intr_en_r.notif_en
1078         always_comb begin
1079             automatic logic [0:0] next_c;
1080             automatic logic load_next_c;
1081             next_c =
field_storage.intr_block_rf.global_intr_en_r.notif_en.value;
1082             load_next_c = '0';
1083             if(decoded_reg_strb.intr_block_rf.global_intr_en_r &&
decoded_req_is_wr) begin // SW write
1084                 next_c =
(field_storage.intr_block_rf.global_intr_en_r.notif_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1085                 load_next_c = '1';
1086             end
1087             field_combo.intr_block_rf.global_intr_en_r.notif_en.next = next_c;
1088             field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next =
load_next_c;
1089         end
1090         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1091             if(~hwif_in.reset_b) begin
1092                 field_storage.intr_block_rf.global_intr_en_r.notif_en.value <= 1'h0;
1093             end else
1094             if(field_combo.intr_block_rf.global_intr_en_r.notif_en.load_next)
begin
1095                 field_storage.intr_block_rf.global_intr_en_r.notif_en.value <=
field_combo.intr_block_rf.global_intr_en_r.notif_en.next;
1096             end
1097             // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_en_r.error0_en
1098             always_comb begin
1099                 automatic logic [0:0] next_c;
1100                 automatic logic load_next_c;
1101                 next_c =
field_storage.intr_block_rf.error_intr_en_r.error0_en.value;
1102                 load_next_c = '0';

```

```

1103             if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1104                 next_c =
(field_storage.intr_block_rf.error_intr_en_r.error0_en.value & ~de-
coded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1105                 load_next_c = '1';
1106             end
1107             field_combo.intr_block_rf.error_intr_en_r.error0_en.next = next_c;
1108             field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next =
load_next_c;
1109         end
1110         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1111             if(~hwif_in.reset_b) begin
1112                 field_storage.intr_block_rf.error_intr_en_r.error0_en.value <= 1'h0;
1113             end else
1114             if(field_combo.intr_block_rf.error_intr_en_r.error0_en.load_next)
begin
1115                 field_storage.intr_block_rf.error_intr_en_r.error0_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error0_en.next;
1116             end
1117             // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_en_r.error1_en
1118             always_comb begin
1119                 automatic logic [0:0] next_c;
1120                 automatic logic load_next_c;
1121                 next_c =
field_storage.intr_block_rf.error_intr_en_r.error1_en.value;
1122                 load_next_c = '0';
1123                 if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1124                     next_c =
(field_storage.intr_block_rf.error_intr_en_r.error1_en.value & ~de-
coded_wr_biten[1:1]) | (decoded_wr_data[1:1] & de-
coded_wr_biten[1:1]);
1125                     load_next_c = '1';
1126                 end
1127                 field_combo.intr_block_rf.error_intr_en_r.error1_en.next = next_c;
1128                 field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next =
load_next_c;
1129             end
1130             always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1131                 if(~hwif_in.reset_b) begin
1132                     field_storage.intr_block_rf.error_intr_en_r.error1_en.value <= 1'h0;
1133                 end else
1134                 if(field_combo.intr_block_rf.error_intr_en_r.error1_en.load_next)
begin
1135                     field_storage.intr_block_rf.error_intr_en_r.error1_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error1_en.next;
1136                 end
1137                 // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_en_r.error2_en
1138                 always_comb begin
1139                     automatic logic [0:0] next_c;
1140                     automatic logic load_next_c;
1141                     next_c =
field_storage.intr_block_rf.error_intr_en_r.error2_en.value;
1142                     load_next_c = '0';
1143                     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1144                         next_c =
(field_storage.intr_block_rf.error_intr_en_r.error2_en.value & ~de-
coded_wr_biten[2:2]) | (decoded_wr_data[2:2] & de-
coded_wr_biten[2:2]);
1145                         load_next_c = '1';
1146                     end
1147                     field_combo.intr_block_rf.error_intr_en_r.error2_en.next = next_c;
1148                     field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next =
load_next_c;
1149                 end
1150                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1151                     if(~hwif_in.reset_b) begin
1152                         field_storage.intr_block_rf.error_intr_en_r.error2_en.value <= 1'h0;
1153                     end else
1154                     if(field_combo.intr_block_rf.error_intr_en_r.error2_en.load_next)
begin
1155                         field_storage.intr_block_rf.error_intr_en_r.error2_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error2_en.next;
1156                     end
1157                     // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_en_r.error3_en
1158                     always_comb begin
1159                         automatic logic [0:0] next_c;
1160                         automatic logic load_next_c;
1161                         next_c =
field_storage.intr_block_rf.error_intr_en_r.error3_en.value;

```

```

1162     load_next_c = '0';
1163     if(decoded_reg_strb.intr_block_rf.error_intr_en_r &&
decoded_req_is_wr) begin // SW write
1164         next_c =
(field_storage.intr_block_rf.error_intr_en_r.error3_en.value & ~de-
coded_wr_biten[3:3]) | (decoded_wr_data[3:3] & de-
coded_wr_biten[3:3]);
1165         load_next_c = '1';
1166     end
1167 field_combo.intr_block_rf.error_intr_en_r.error3_en.next = next_c;
1168 field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next =
load_next_c;
1169 end
1170 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1171     if(~hwif_in.reset_b) begin
1172         field_storage.intr_block_rf.error_intr_en_r.error3_en.value <= 1'h0;
1173     end else
if(field_combo.intr_block_rf.error_intr_en_r.error3_en.load_next)
begin
1174         field_storage.intr_block_rf.error_intr_en_r.error3_en.value <=
field_combo.intr_block_rf.error_intr_en_r.error3_en.next;
1175     end
1176 end
1177 // Field: aes_clp_reg.intr_b-
lock_rf.notif_intr_en_r.notif_cmd_done_en
1178 always_comb begin
1179     automatic logic [0:0] next_c;
1180     automatic logic load_next_c;
1181     next_c =
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value;
1182     load_next_c = '0';
1183     if(decoded_reg_strb.intr_block_rf.notif_intr_en_r &&
decoded_req_is_wr) begin // SW write
1184         next_c =
(field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
& ~decoded_wr_biten[0:0]) | (decoded_wr_data[0:0] & de-
coded_wr_biten[0:0]);
1185         load_next_c = '1';
1186     end
1187 field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next =
next_c;
1188 field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_ne
xt = load_next_c;
1189 end
1190 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1191     if(~hwif_in.reset_b) begin
1192         field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= 1'h0;
1193     end else
if(field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.load_
next) begin
1194         field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
<= field_combo.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.next;
1195     end
1196 end
1197 // Field: aes_clp_reg.intr_b-
lock_rf.error_global_intr_r.agg_sts
1198 always_comb begin
1199     automatic logic [0:0] next_c;
1200     automatic logic load_next_c;
1201     next_c =
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value;
1202     load_next_c = '0';
1203 // HW Write
1204 next_c = hwif_out.intr_b-
lock_rf.error_internal_intr_r.intr;
1205 load_next_c = '1';
1206 field_combo.intr_block_rf.error_global_intr_r.agg_sts.next = next_c;
1207 field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next =
load_next_c;
1208 end
1209 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1210     if(~hwif_in.reset_b) begin
1211         field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
1'h0;
1212     end else
if(field_combo.intr_block_rf.error_global_intr_r.agg_sts.load_next)
begin
1213         field_storage.intr_block_rf.error_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.error_global_intr_r.agg_sts.next;
1214     end
1215 end
1216 assign hwif_out.intr_block_rf.error_global_intr_r.intr =
1217 |
(field_storage.intr_block_rf.error_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.error_global_intr_en_r.error_en.value);
1218 // Field: aes_clp_reg.intr_b-
lock_rf.notif_global_intr_r.agg_sts

```

```

1220     always_comb begin
1221         automatic logic [0:0] next_c;
1222         automatic logic load_next_c;
1223         next_c =
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value;
1224         load_next_c = '0';
1225 // HW Write
1226 next_c = hwif_out.intr_b-
lock_rf.notif_internal_intr_r.intr;
1227 load_next_c = '1';
1228 field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next = next_c;
1229 field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next =
load_next_c;
1230 end
1231 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1232     if(~hwif_in.reset_b) begin
1233         field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
1'h0;
1234     end else
if(field_combo.intr_block_rf.notif_global_intr_r.agg_sts.load_next)
begin
1235         field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value <=
field_combo.intr_block_rf.notif_global_intr_r.agg_sts.next;
1236     end
1237 assign hwif_out.intr_block_rf.notif_global_intr_r.intr =
1238 |
(field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value &
field_storage.intr_block_rf.error_global_intr_en_r.error_en.value);
1239 // Field: aes_clp_reg.intr_b-
lock_rf.error_internal_intr_r.error0_sts
1240 always_comb begin
1241     automatic logic [0:0] next_c;
1242     automatic logic load_next_c;
1243     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value;
1244     load_next_c = '0';
1245 if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value !=
'0) begin // stickybit
1246     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1247     load_next_c = '1';
1248 end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1249     next_c = '1';
1250     load_next_c = '1';
1251 end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1252     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value &
~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1253     load_next_c = '1';
1254 end
1255 field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next =
next_c;
1256 field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_next
= load_next_c;
1257 end
1258 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1259     if(~hwif_in.error_reset_b) begin
1260         field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
<= 1'h0;
1261     end else
if(field_combo.intr_block_rf.error_internal_intr_r.error0_sts.load_
next) begin
1262         field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error0_sts.next;
1263     end
1264 // Field: aes_clp_reg.intr_b-
lock_rf.error_internal_intr_r.error1_sts
1265 always_comb begin
1266     automatic logic [0:0] next_c;
1267     automatic logic load_next_c;
1268     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value;
1269     load_next_c = '0';
1270 if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value !=
'0) begin // stickybit
1271     next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1272     load_next_c = '1';
1273 end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1274     next_c = '1';
1275     load_next_c = '1';
1276 end

```



```

1279     end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1280         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value &
~(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1281         load_next_c = '1';
1282     end
1283
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next =
next_c;
1284
field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_next
= load_next_c;
1285     end
1286     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1287         if(~hwif_in.error_reset_b) begin
1288
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= 1'h0;
1289         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error1_sts.load_
next) begin
1290
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error1_sts.next;
1291         end
1292     end
1293     // Field: aes_clp_reg.intr_b-
lock_rf.error_internal_intr_r.error2_sts
1294     always_comb begin
1295         automatic logic [0:0] next_c;
1296         automatic logic load_next_c;
1297         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value;
1298         load_next_c = '0';
1299
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value !=
'0') begin // stickybit
1300             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1301             load_next_c = '1';
1302         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1303             next_c = '1';
1304             load_next_c = '1';
1305         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1306             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value &
~(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1307             load_next_c = '1';
1308         end
1309
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next =
next_c;
1310
field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_next
= load_next_c;
1311     end
1312     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1313         if(~hwif_in.error_reset_b) begin
1314
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= 1'h0;
1315         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error2_sts.load_
next) begin
1316
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error2_sts.next;
1317         end
1318     end
1319     // Field: aes_clp_reg.intr_b-
lock_rf.error_internal_intr_r.error3_sts
1320     always_comb begin
1321         automatic logic [0:0] next_c;
1322         automatic logic load_next_c;
1323         next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value;
1324         load_next_c = '0';
1325
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value !=
'0') begin // stickybit
1326             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value |
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1327             load_next_c = '1';
1328         end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1329             next_c = '1';
1330             load_next_c = '1';
1331         end else if(de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1332             next_c =
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value &

```

```

~(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1333         load_next_c = '1';
1334     end
1335
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next =
next_c;
1336
field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_next
= load_next_c;
1337     end
1338     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1339         if(~hwif_in.error_reset_b) begin
1340
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= 1'h0;
1341         end else
if(field_combo.intr_block_rf.error_internal_intr_r.error3_sts.load_
next) begin
1342
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
<= field_combo.intr_block_rf.error_internal_intr_r.error3_sts.next;
1343         end
1344     end
1345     assign hwif_out.intr_block_rf.error_internal_intr_r.intr =
|
(field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error0_en.value)
1347     ||
(field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error1_en.value)
1348     ||
(field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error2_en.value)
1349     ||
(field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value
& field_storage.intr_block_rf.error_intr_en_r.error3_en.value);
1350     // Field: aes_clp_reg.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts
1351     always_comb begin
1352         automatic logic [0:0] next_c;
1353         automatic logic load_next_c;
1354         next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value;
1355         load_next_c = '0';
1356
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value != '0') begin // stickybit
1357             next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value |
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1358             load_next_c = '1';
1359         end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1360             next_c = '1';
1361             load_next_c = '1';
1362         end else if(de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && de-
coded_req_is_wr) begin // SW write 1 clear
1363             next_c =
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value & ~(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1364             load_next_c = '1';
1365         end
1366
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.next = next_c;
1367
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.load_next = load_next_c;
1368     end
1369     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1370         if(~hwif_in.reset_b) begin
1371
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <= 1'h0;
1372         end else
if(field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.load_next) begin
1373
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value <=
field_combo.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.next;
1374         end
1375     end
1376     assign hwif_out.intr_block_rf.notif_internal_intr_r.intr =
|
(field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_s
ts.value &
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value)
;
1378     // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error0_trig
1379     always_comb begin
1380         automatic logic [0:0] next_c;
1381         automatic logic load_next_c;

```

```

1382     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1383     load_next_c = '0';

1384     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1385         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value |
(decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1386         load_next_c = '1';
1387     end else begin // singlepulse clears back to 0
1388         next_c = '0';
1389         load_next_c = '1';
1390     end
1391 end
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next =
next_c;
1392 field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next =
load_next_c;
1393 end
1394 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1395     if(~hwif_in.reset_b) begin
1396         field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
1'h0;
1397     end else
1398         if(field_combo.intr_block_rf.error_intr_trig_r.error0_trig.load_next
) begin
1399             field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error0_trig.next;
1400         end
1401         // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error1_trig
1402         always_comb begin
1403             automatic logic [0:0] next_c;
1404             automatic logic load_next_c;
1405             next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1406             load_next_c = '0';

1407             if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1408                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value |
(decoded_wr_data[1:1] & decoded_wr_biten[1:1]);
1409                 load_next_c = '1';
1410             end else begin // singlepulse clears back to 0
1411                 next_c = '0';
1412                 load_next_c = '1';
1413             end
1414         end
1415         field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next =
next_c;
1416         field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next =
load_next_c;
1417         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1418             if(~hwif_in.reset_b) begin
1419                 field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
1'h0;
1420             end else
1421                 if(field_combo.intr_block_rf.error_intr_trig_r.error1_trig.load_next
) begin
1422                     field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error1_trig.next;
1423                 end
1424                 // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error2_trig
1425                 always_comb begin
1426                     automatic logic [0:0] next_c;
1427                     automatic logic load_next_c;
1428                     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1429                     load_next_c = '0';

1430                     if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1431                         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value |
(decoded_wr_data[2:2] & decoded_wr_biten[2:2]);
1432                         load_next_c = '1';
1433                     end else begin // singlepulse clears back to 0
1434                         next_c = '0';
1435                         load_next_c = '1';
1436                     end
1437                 end
1438                 field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next =
next_c;
1439                 field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next =
load_next_c;
1440                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1441                     if(~hwif_in.reset_b) begin

```

```

1442     field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
1'h0;
1443     end else
1444         if(field_combo.intr_block_rf.error_intr_trig_r.error2_trig.load_next
) begin
1445             field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error2_trig.next;
1446         end
1447         // Field: aes_clp_reg.intr_b-
lock_rf.error_intr_trig_r.error3_trig
1448         always_comb begin
1449             automatic logic [0:0] next_c;
1450             automatic logic load_next_c;
1451             next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1452             load_next_c = '0';

1453             if(decoded_reg_strb.intr_block_rf.error_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1454                 next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value |
(decoded_wr_data[3:3] & decoded_wr_biten[3:3]);
1455                 load_next_c = '1';
1456             end else begin // singlepulse clears back to 0
1457                 next_c = '0';
1458                 load_next_c = '1';
1459             end
1460         end
1461         field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next =
next_c;
1462         field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next =
load_next_c;
1463         always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1464             if(~hwif_in.reset_b) begin
1465                 field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
1'h0;
1466             end else
1467                 if(field_combo.intr_block_rf.error_intr_trig_r.error3_trig.load_next
) begin
1468                     field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value <=
field_combo.intr_block_rf.error_intr_trig_r.error3_trig.next;
1469                 end
1470                 // Field: aes_clp_reg.intr_b-
lock_rf.notif_intr_trig_r.notif_cmd_done_trig
1471                 always_comb begin
1472                     automatic logic [0:0] next_c;
1473                     automatic logic load_next_c;
1474                     next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1475                     load_next_c = '0';

1476                     if(decoded_reg_strb.intr_block_rf.notif_intr_trig_r &&
decoded_req_is_wr) begin // SW write 1 set
1477                         next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue | (decoded_wr_data[0:0] & decoded_wr_biten[0:0]);
1478                         load_next_c = '1';
1479                     end else begin // singlepulse clears back to 0
1480                         next_c = '0';
1481                         load_next_c = '1';
1482                     end
1483                 end
1484                 field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
= next_c;
1485                 field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.loa
d_next = load_next_c;
1486                 always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1487                     if(~hwif_in.reset_b) begin
1488                         field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <= 1'h0;
1489                     end else
1490                         if(field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.
load_next) begin
1491                             field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue <=
field_combo.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.next
;
1492                         end
1493                         // Field: aes_clp_reg.intr_b-
lock_rf.error0_intr_count_r.cnt
1494                         always_comb begin
1495                             automatic logic [31:0] next_c;
1496                             automatic logic load_next_c;
1497                             next_c =
field_storage.intr_block_rf.error0_intr_count_r.cnt.value;
1498                             load_next_c = '0';
1499                             if(decoded_reg_strb.intr_block_rf.error0_intr_count_r

```



```

1500         next_c =
1501         (field_storage.intr_block_rf.error0_intr_count_r.cnt.value & ~de-
1502         coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1503         coded_wr_biten[31:0]);
1504         load_next_c = '1';
1505     end
1506 if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
1507 begin // increment
1508     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1509 gin // up-counter saturated
1510         next_c = 32'hffffffff;
1511     end else begin
1512         next_c = next_c + 32'h1;
1513     end
1514     load_next_c = '1';
1515 end
1516
1517 field_combo.intr_block_rf.error0_intr_count_r.cnt.incrthreshold =
1518 (field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
1519 32'hffffffff);
1520
1521 field_combo.intr_block_rf.error0_intr_count_r.cnt.incrsaturate =
1522 (field_storage.intr_block_rf.error0_intr_count_r.cnt.value >=
1523 32'hffffffff);
1524
1525 if(next_c > 32'hffffffff) begin
1526     next_c = 32'hffffffff;
1527     load_next_c = '1';
1528 end
1529
1530 field_combo.intr_block_rf.error0_intr_count_r.cnt.next
1531 = next_c;
1532
1533 field_storage.intr_block_rf.error0_intr_count_r.cnt.load_next =
1534 load_next_c;
1535 end
1536 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1537 begin
1538     if(~hwif_in.error_reset_b) begin
1539         if(~hwif_in.error_reset_b) begin
1540             field_storage.intr_block_rf.error0_intr_count_r.cnt.value <= 32'h0;
1541             end else
1542             if(field_combo.intr_block_rf.error0_intr_count_r.cnt.load_next) be-
1543 gin
1544                 field_storage.intr_block_rf.error0_intr_count_r.cnt.value <=
1545                 field_combo.intr_block_rf.error0_intr_count_r.cnt.next;
1546             end
1547         end
1548         // Field: aes_clp_reg.intr_b-
1549         lock_rf.error1_intr_count_r.cnt
1550         always_comb begin
1551             automatic logic [31:0] next_c;
1552             automatic logic load_next_c;
1553             next_c =
1554             (field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
1555             coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1556             coded_wr_biten[31:0]);
1557             load_next_c = '1';
1558         end
1559
1560 if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
1561 begin // increment
1562     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1563 gin // up-counter saturated
1564         next_c = 32'hffffffff;
1565     end else begin
1566         next_c = next_c + 32'h1;
1567     end
1568     load_next_c = '1';
1569 end
1570
1571 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
1572 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1573 32'hffffffff);
1574
1575 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
1576 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1577 32'hffffffff);
1578
1579 if(next_c > 32'hffffffff) begin
1580     next_c = 32'hffffffff;
1581     load_next_c = '1';
1582 end
1583
1584 field_combo.intr_block_rf.error1_intr_count_r.cnt.next
1585 = next_c;
1586
1587 field_storage.intr_block_rf.error1_intr_count_r.cnt.load_next =
1588 load_next_c;
1589 end
1590 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1591 begin
1592     if(~hwif_in.error_reset_b) begin
1593         field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1594         end else
1595         if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
1596 gin
1597             field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
1598             field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1599         end
1600     end
1601     // Field: aes_clp_reg.intr_b-
1602     lock_rf.error1_intr_count_r.cnt
1603     always_comb begin
1604         automatic logic [31:0] next_c;
1605         automatic logic load_next_c;
1606         next_c =
1607         (field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
1608         coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1609         coded_wr_biten[31:0]);
1610         load_next_c = '1';
1611     end
1612
1613 if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
1614 begin // increment
1615     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1616 gin // up-counter saturated
1617         next_c = 32'hffffffff;
1618     end else begin
1619         next_c = next_c + 32'h1;
1620     end
1621     load_next_c = '1';
1622 end
1623
1624 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
1625 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1626 32'hffffffff);
1627
1628 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
1629 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1630 32'hffffffff);
1631
1632 if(next_c > 32'hffffffff) begin
1633     next_c = 32'hffffffff;
1634     load_next_c = '1';
1635 end
1636
1637 field_combo.intr_block_rf.error1_intr_count_r.cnt.next
1638 = next_c;
1639
1640 field_storage.intr_block_rf.error1_intr_count_r.cnt.load_next =
1641 load_next_c;
1642 end
1643 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1644 begin
1645     if(~hwif_in.error_reset_b) begin
1646         field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1647         end else
1648         if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
1649 gin
1650             field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
1651             field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1652         end
1653     end
1654     // Field: aes_clp_reg.intr_b-
1655     lock_rf.error1_intr_count_r.cnt
1656     always_comb begin
1657         automatic logic [31:0] next_c;
1658         automatic logic load_next_c;
1659         next_c =
1660         (field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
1661         coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1662         coded_wr_biten[31:0]);
1663         load_next_c = '1';
1664     end
1665
1666 if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
1667 begin // increment
1668     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1669 gin // up-counter saturated
1670         next_c = 32'hffffffff;
1671     end else begin
1672         next_c = next_c + 32'h1;
1673     end
1674     load_next_c = '1';
1675 end
1676
1677 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
1678 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1679 32'hffffffff);
1680
1681 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
1682 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1683 32'hffffffff);
1684
1685 if(next_c > 32'hffffffff) begin
1686     next_c = 32'hffffffff;
1687     load_next_c = '1';
1688 end
1689
1690 field_combo.intr_block_rf.error1_intr_count_r.cnt.next
1691 = next_c;
1692
1693 field_storage.intr_block_rf.error1_intr_count_r.cnt.load_next =
1694 load_next_c;
1695 end
1696 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1697 begin
1698     if(~hwif_in.error_reset_b) begin
1699         field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1700         end else
1701         if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
1702 gin
1703             field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
1704             field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1705         end
1706     end
1707     // Field: aes_clp_reg.intr_b-
1708     lock_rf.error1_intr_count_r.cnt
1709     always_comb begin
1710         automatic logic [31:0] next_c;
1711         automatic logic load_next_c;
1712         next_c =
1713         (field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
1714         coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1715         coded_wr_biten[31:0]);
1716         load_next_c = '1';
1717     end
1718
1719 if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
1720 begin // increment
1721     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1722 gin // up-counter saturated
1723         next_c = 32'hffffffff;
1724     end else begin
1725         next_c = next_c + 32'h1;
1726     end
1727     load_next_c = '1';
1728 end
1729
1730 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
1731 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1732 32'hffffffff);
1733
1734 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrsaturate =
1735 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1736 32'hffffffff);
1737
1738 if(next_c > 32'hffffffff) begin
1739     next_c = 32'hffffffff;
1740     load_next_c = '1';
1741 end
1742
1743 field_combo.intr_block_rf.error1_intr_count_r.cnt.next
1744 = next_c;
1745
1746 field_storage.intr_block_rf.error1_intr_count_r.cnt.load_next =
1747 load_next_c;
1748 end
1749 always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1750 begin
1751     if(~hwif_in.error_reset_b) begin
1752         field_storage.intr_block_rf.error1_intr_count_r.cnt.value <= 32'h0;
1753         end else
1754         if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
1755 gin
1756             field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
1757             field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1758         end
1759     end
1760     // Field: aes_clp_reg.intr_b-
1761     lock_rf.error1_intr_count_r.cnt
1762     always_comb begin
1763         automatic logic [31:0] next_c;
1764         automatic logic load_next_c;
1765         next_c =
1766         (field_storage.intr_block_rf.error1_intr_count_r.cnt.value & ~de-
1767         coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1768         coded_wr_biten[31:0]);
1769         load_next_c = '1';
1770     end
1771
1772 if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
1773 begin // increment
1774     if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
1775 gin // up-counter saturated
1776         next_c = 32'hffffffff;
1777     end else begin
1778         next_c = next_c + 32'h1;
1779     end
1780     load_next_c = '1';
1781 end
1782
1783 field_combo.intr_block_rf.error1_intr_count_r.cnt.incrthreshold =
1784 (field_storage.intr_block_rf.error1_intr_count_r.cnt.value >=
1785 32'hffffffff
```

```

1557         end else
1558         if(field_combo.intr_block_rf.error1_intr_count_r.cnt.load_next) be-
1559         gin
1560         field_storage.intr_block_rf.error1_intr_count_r.cnt.value <=
1561         field_combo.intr_block_rf.error1_intr_count_r.cnt.next;
1562         end
1563         end
1564         // Field: aes_clp_reg.intr_b-
1565         lock_rf.error2_intr_count_r.cnt
1566         always_comb begin
1567             automatic logic [31:0] next_c;
1568             automatic logic load_next_c;
1569             next_c =
1570             field_storage.intr_block_rf.error2_intr_count_r.cnt.value;
1571             load_next_c = '0;
1572             if(decoded_reg_strb.intr_block_rf.error2_intr_count_r
1573             && decoded_req_is_wr) begin // SW write
1574                 next_c =
1575                 (field_storage.intr_block_rf.error2_intr_count_r.cnt.value & ~de-
1576                 coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1577                 coded_wr_biten[31:0]);
1578                 load_next_c = '1;
1579             end
1580             end
1581             if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
1582             begin // increment
1583                 if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
1584                 gin // up-counter saturated
1585                     next_c = 32'hfffffff;
1586                 end else begin
1587                     next_c = next_c + 32'h1;
1588                 end
1589                 load_next_c = '1;
1590             end
1591             end
1592             field_combo.intr_block_rf.error2_intr_count_r.cnt.incrthreshold =
1593             (field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
1594             32'hfffffff);
1595             field_combo.intr_block_rf.error2_intr_count_r.cnt.incrsaturate =
1596             (field_storage.intr_block_rf.error2_intr_count_r.cnt.value >=
1597             32'hfffffff);
1598             if(next_c > 32'hfffffff) begin
1599                 next_c = 32'hfffffff;
1600                 load_next_c = '1;
1601             end
1602             end
1603             field_combo.intr_block_rf.error2_intr_count_r.cnt.next
1604             = next_c;
1605             field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next =
1606             load_next_c;
1607             end
1608             always_ff @(posedge clk or negedge hwif_in.error_reset_b)
1609             begin
1610                 if(~hwif_in.error_reset_b) begin
1611                     field_storage.intr_block_rf.error2_intr_count_r.cnt.value <= 32'h0;
1612                     end else
1613                     if(field_combo.intr_block_rf.error2_intr_count_r.cnt.load_next) be-
1614                     gin
1615                         field_storage.intr_block_rf.error2_intr_count_r.cnt.value <=
1616                         field_combo.intr_block_rf.error2_intr_count_r.cnt.next;
1617                     end
1618                     end
1619                     // Field: aes_clp_reg.intr_b-
1620                     lock_rf.error3_intr_count_r.cnt
1621                     always_comb begin
1622                         automatic logic [31:0] next_c;
1623                         automatic logic load_next_c;
1624                         next_c =
1625                         field_storage.intr_block_rf.error3_intr_count_r.cnt.value;
1626                         load_next_c = '0;
1627                         if(decoded_reg_strb.intr_block_rf.error3_intr_count_r
1628                         && decoded_req_is_wr) begin // SW write
1629                             next_c =
1630                             (field_storage.intr_block_rf.error3_intr_count_r.cnt.value & ~de-
1631                             coded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
1632                             coded_wr_biten[31:0]);
1633                             load_next_c = '1;
1634                         end
1635                         end
1636                         if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
1637                         begin // increment
1638                             if(((33)'(next_c) + 32'h1) > 32'hfffffff) be-
1639                             gin // up-counter saturated
1640                                 next_c = 32'hfffffff;
1641                             end else begin
1642                                 next_c = next_c + 32'h1;
1643                             end
1644                             load_next_c = '1;
1645                         end
1646                         end
1647                         field_combo.intr_block_rf.error3_intr_count_r.cnt.incrthreshold =
1648                         (field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
1649                         32'hfffffff);
1650                         field_combo.intr_block_rf.error3_intr_count_r.cnt.incrsaturate =
1651                         (field_storage.intr_block_rf.error3_intr_count_r.cnt.value >=
1652                         32'hfffffff);
1653                     end
1654                     end

```

```

32'hffffffff);
1615     if(next_c > 32'hffffffff) begin
1616         next_c = 32'hffffffff;
1617         load_next_c = '1;
1618     end

1619     field_combo.intr_block_rf.error3_intr_count_r.cnt.next
= next_c;
1620
field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next =
load_next_c;
1621     end
1622     always_ff @(posedge clk or negedge hwif_in.error_reset_b)
begin
1623         if(~hwif_in.error_reset_b) begin
1624
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <= 32'h0;
1625         end else
if(field_combo.intr_block_rf.error3_intr_count_r.cnt.load_next) be-
gin
1626
field_storage.intr_block_rf.error3_intr_count_r.cnt.value <=
field_combo.intr_block_rf.error3_intr_count_r.cnt.next;
1627         end
1628     end
1629     // Field: aes_clp_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_r.cnt
1630     always_comb begin
1631         automatic logic [31:0] next_c;
1632         automatic logic load_next_c;
1633         next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value;
1634         load_next_c = '0;
1635         if(de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && de-
coded_req_is_wr) begin // SW write
1636             next_c =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value &
~decoded_wr_biten[31:0]) | (decoded_wr_data[31:0] & de-
coded_wr_biten[31:0]);
1637             load_next_c = '1;
1638         end
1639     end
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul-
se.value) begin // increment
1640         if(((33)'(next_c) + 32'h1) > 32'hffffffff) be-
gin // up-counter saturated
1641             next_c = 32'hffffffff;
1642         end else begin
1643             next_c = next_c + 32'h1;
1644         end
1645         load_next_c = '1;
1646     end
1647
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrthres
hold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
1648
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.incrsatur
ate =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value
>= 32'hffffffff);
1649     if(next_c > 32'hffffffff) begin
1650         next_c = 32'hffffffff;
1651         load_next_c = '1;
1652     end
1653
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next =
next_c;
1654
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_next
= load_next_c;
1655     end
1656     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1657         if(~hwif_in.reset_b) begin
1658
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
32'h0;
1659         end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.load_n
ext) begin
1660
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_r.cnt.next;
1661         end
1662     end
1663     // Field: aes_clp_reg.intr_b-
lock_rf.error0_intr_count_incr_r.pulse
1664     always_comb begin
1665         automatic logic [0:0] next_c;
1666         automatic logic load_next_c;
1667         next_c =
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value;
1668         load_next_c = '0;
1669     end
if(field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value)
begin // HW Write - we
1670         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value;
1671         load_next_c = '1;

```

```

1672     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error0_sts.hwset) begin // HW Set
1673         next_c = '1;
1674         load_next_c = '1;
1675     end
1676
if(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value)
begin // decrement
1677
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1678         next_c = next_c - 1'h1;
1679         load_next_c = '1;
1680     end else begin
1681
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.underflow =
'0;
1682     end
1683
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'd0);
1684
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next =
next_c;
1685
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_next =
load_next_c;
1686     end
1687     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1688         if(~hwif_in.reset_b) begin
1689
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
1'h0;
1690         end else
if(field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.load_ne
xt) begin
1691
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error0_intr_count_incr_r.pulse.next;
1692         end
1693     end
1694     // Field: aes_clp_reg.intr_b-
lock_rf.error1_intr_count_incr_r.pulse
1695     always_comb begin
1696         automatic logic [0:0] next_c;
1697         automatic logic load_next_c;
1698         next_c =
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value;
1699         load_next_c = '0;
1700     end
if(field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value)
begin // HW Write - we
1701         next_c =
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value;
1702         load_next_c = '1;
1703     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error1_sts.hwset) begin // HW Set
1704         next_c = '1;
1705         load_next_c = '1;
1706     end
1707
if(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value)
begin // decrement
1708
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1709         next_c = next_c - 1'h1;
1710         load_next_c = '1;
1711     end else begin
1712
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.underflow =
'0;
1713     end
1714
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'd0);
1715
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next =
next_c;
1716
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_next =
load_next_c;
1717     end
1718     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1719         if(~hwif_in.reset_b) begin
1720
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
1'h0;
1721         end else
if(field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.load_ne
xt) begin
1722
field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error1_intr_count_incr_r.pulse.next;
1723         end
1724     end
1725     // Field: aes_clp_reg.intr_b-

```

```

lock_rf.error2_intr_count_incr_r.pulse
1726     always_comb begin
1727         automatic logic [0:0] next_c;
1728         automatic logic load_next_c;
1729         next_c =
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value;
1730         load_next_c = '0;
1731
if(field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value)
begin // HW Write - we
1732     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value;
1733     load_next_c = '1;
1734     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error2_sts.hwset) begin // HW Set
1735         next_c = '1;
1736         load_next_c = '1;
1737     end
1738
if(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value)
begin // decrement
1739
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1740     next_c = next_c - 1'h1;
1741     load_next_c = '1;
1742     end else begin
1743
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.underflow =
'0;
1744     end
1745
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'd0);
1746
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next =
next_c;
1747
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_next =
load_next_c;
1748     end
1749     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1750         if(~hwif_in.reset_b) begin
1751
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
1'h0;
1752         end else
if(field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.load_ne
xt) begin
1753
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error2_intr_count_incr_r.pulse.next;
1754     end
1755     end
1756     // Field: aes_clp_reg.intr_b-
lock_rf.error3_intr_count_incr_r.pulse
1757     always_comb begin
1758         automatic logic [0:0] next_c;
1759         automatic logic load_next_c;
1760         next_c =
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value;
1761         load_next_c = '0;
1762
if(field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value)
begin // HW Write - we
1763     next_c =
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value;
1764     load_next_c = '1;
1765     end else if(hwif_in.intr_b-
lock_rf.error_internal_intr_r.error3_sts.hwset) begin // HW Set
1766         next_c = '1;
1767         load_next_c = '1;
1768     end
1769
if(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value)
begin // decrement
1770
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
(next_c < (1'h1));
1771     next_c = next_c - 1'h1;
1772     load_next_c = '1;
1773     end else begin
1774
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.underflow =
'0;
1775     end
1776
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.decrthresh
old =
(field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'd0);
1777
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next =
next_c;
1778
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_next =
load_next_c;
1779     end
1780     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1781         if(~hwif_in.reset_b) begin
1782
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
1'h0;
1783         end else
if(field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.load_ne
xt) begin
1784
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value <=
field_combo.intr_block_rf.error3_intr_count_incr_r.pulse.next;
1785     end
1786     end
1787     // Field: aes_clp_reg.intr_b-
lock_rf.notif_cmd_done_intr_count_incr_r.pulse
1788     always_comb begin
1789         automatic logic [0:0] next_c;
1790         automatic logic load_next_c;
1791         next_c =
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value;
1792         load_next_c = '0;
1793
if(field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig
.value) begin // HW Write - we
1794     next_c =
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue;
1795     load_next_c = '1;
1796     end else if(hwif_in.intr_b-
lock_rf.notif_internal_intr_r.notif_cmd_done_sts.hwset) begin // HW
Set
1797         next_c = '1;
1798         load_next_c = '1;
1799     end
1800
if(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pul
se.value) begin // decrement
1801
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = (next_c < (1'h1));
1802     next_c = next_c - 1'h1;
1803     load_next_c = '1;
1804     end else begin
1805
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.un
derflow = '0;
1806     end
1807
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.de
crthreshold =
(field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'd0);
1808
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt = next_c;
1809
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.lo
ad_next = load_next_c;
1810     end
1811     always_ff @(posedge clk or negedge hwif_in.reset_b) begin
1812         if(~hwif_in.reset_b) begin
1813
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <= 1'h0;
1814         end else
if(field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
load_next) begin
1815
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value <=
field_combo.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.ne
xt;
1816     end
1817     end
1818
1819     //-----
1820     // Write response
1821     //-----
1822     assign cpuif_wr_ack = decoded_req & decoded_req_is_wr;
1823     // Writes are always granted with no error response
1824     assign cpuif_wr_err = '0;
1825
1826     //-----
1827     // Readback
1828     //-----
1829
1830     logic readback_err;
1831     logic readback_done;
1832     logic [31:0] readback_data;
1833
1834     // Assign readback values to a flattened array
1835     logic [28:1:0][31:0] readback_array;
1836     for(genvar i0=0; i0<2; i0++) begin
1837         assign readback_array[i0*1 + 0][31:0] = (de-
coded_reg_strb.AES_NAME[i0] && !decoded_req_is_wr) ?
hwif_in.AES_NAME[i0].NAME.next : '0;
1838     end

```

```

1839   for(genvar i0=0; i0<2; i0++) begin
1840       assign readback_array[i0*1 + 2][31:0] = (de-
coded_reg_strb.AES_VERSION[i0] && !decoded_req_is_wr) ?
hwif_in.AES_VERSION[i0].VERSION.next : '0';
1841   end
1842   assign readback_array[4][0:0] = (decoded_reg_strb.CTRL0
&& !decoded_req_is_wr) ? field_storage.CTRL0.ENDIAN_SWAP.value : '0';
1843   assign readback_array[4][31:1] = '0';
1844   assign readback_array[5][0:0] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.read_en.value : '0';
1845   assign readback_array[5][5:1] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.read_entry.value : '0';
1846   assign readback_array[5][6:6] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.pcr_hash_extend.value : '0';
1847   assign readback_array[5][31:7] = (de-
coded_reg_strb.AES_KV_RD_KEY_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_CTRL.rsvd.value : '0';
1848   assign readback_array[6][0:0] = (de-
coded_reg_strb.AES_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_RD_KEY_STATUS.READY.next : '0';
1849   assign readback_array[6][1:1] = (de-
coded_reg_strb.AES_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
field_storage.AES_KV_RD_KEY_STATUS.VALID.value : '0';
1850   assign readback_array[6][9:2] = (de-
coded_reg_strb.AES_KV_RD_KEY_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_RD_KEY_STATUS.ERROR.next : '0';
1851   assign readback_array[6][31:10] = '0';
1852   assign readback_array[7][0:0] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.write_en.value : '0';
1853   assign readback_array[7][5:1] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.write_entry.value : '0';
1854   assign readback_array[7][6:6] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.hmac_key_dest_valid.value : '0';
1855   assign readback_array[7][7:7] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.hmac_block_dest_valid.value : '0';
1856   assign readback_array[7][8:8] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.mldsa_seed_dest_valid.value : '0';
1857   assign readback_array[7][9:9] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.ecc_pkey_dest_valid.value : '0';
1858   assign readback_array[7][10:10] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.ecc_seed_dest_valid.value : '0';
1859   assign readback_array[7][11:11] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.aes_key_dest_valid.value : '0';
1860   assign readback_array[7][12:12] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.mtkem_seed_dest_valid.value : '0';
1861   assign readback_array[7][13:13] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.mtkem_msg_dest_valid.value : '0';
1862   assign readback_array[7][14:14] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.dma_data_dest_valid.value : '0';
1863   assign readback_array[7][31:15] = (de-
coded_reg_strb.AES_KV_WR_CTRL && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_CTRL.rsvd.value : '0';
1864   assign readback_array[8][0:0] = (de-
coded_reg_strb.AES_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_WR_STATUS.READY.next : '0';
1865   assign readback_array[8][1:1] = (de-
coded_reg_strb.AES_KV_WR_STATUS && !decoded_req_is_wr) ?
field_storage.AES_KV_WR_STATUS.VALID.value : '0';
1866   assign readback_array[8][9:2] = (de-
coded_reg_strb.AES_KV_WR_STATUS && !decoded_req_is_wr) ?
hwif_in.AES_KV_WR_STATUS.ERROR.next : '0';
1867   assign readback_array[8][31:10] = '0';
1868   assign readback_array[9][0:0] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.error_en.value : '0';
1869   assign readback_array[9][1:1] = (de-
coded_reg_strb.intr_block_rf.global_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.global_intr_en_r.notif_en.value : '0';
1870   assign readback_array[9][31:2] = '0';
1871   assign readback_array[10][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error0_en.value : '0';
1872   assign readback_array[10][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error1_en.value : '0';
1873   assign readback_array[10][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error2_en.value : '0';
1874   assign readback_array[10][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_en_r.error3_en.value : '0';

```

```

1875   assign readback_array[10][31:4] = '0';
1876   assign readback_array[11][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_en_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_en_r.notif_cmd_done_en.value
: '0';
1877   assign readback_array[11][31:1] = '0';
1878   assign readback_array[12][0:0] = (de-
coded_reg_strb.intr_block_rf.error_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_global_intr_r.agg_sts.value : '0';
1879   assign readback_array[12][31:1] = '0';
1880   assign readback_array[13][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_global_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_global_intr_r.agg_sts.value : '0';
1881   assign readback_array[13][31:1] = '0';
1882   assign readback_array[14][0:0] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error0_sts.value :
'0';
1883   assign readback_array[14][1:1] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error1_sts.value :
'0';
1884   assign readback_array[14][2:2] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error2_sts.value :
'0';
1885   assign readback_array[14][3:3] = (de-
coded_reg_strb.intr_block_rf.error_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_internal_intr_r.error3_sts.value :
'0';
1886   assign readback_array[14][31:4] = '0';
1887   assign readback_array[15][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_internal_intr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_internal_intr_r.notif_cmd_done_sts
.value : '0';
1888   assign readback_array[15][31:1] = '0';
1889   assign readback_array[16][0:0] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error0_trig.value :
'0';
1890   assign readback_array[16][1:1] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error1_trig.value :
'0';
1891   assign readback_array[16][2:2] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error2_trig.value :
'0';
1892   assign readback_array[16][3:3] = (de-
coded_reg_strb.intr_block_rf.error_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error_intr_trig_r.error3_trig.value :
'0';
1893   assign readback_array[16][31:4] = '0';
1894   assign readback_array[17][0:0] = (de-
coded_reg_strb.intr_block_rf.notif_intr_trig_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_intr_trig_r.notif_cmd_done_trig.v
alue : '0';
1895   assign readback_array[17][31:1] = '0';
1896   assign readback_array[18][31:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_r.cnt.value : '0';
1897   assign readback_array[19][31:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error1_intr_count_r.cnt.value : '0';
1898   assign readback_array[20][31:0] = (de-
coded_reg_strb.intr_block_rf.error2_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_r.cnt.value : '0';
1899   assign readback_array[21][31:0] = (de-
coded_reg_strb.intr_block_rf.error3_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_r.cnt.value : '0';
1900   assign readback_array[22][31:0] = (de-
coded_reg_strb.intr_block_rf.notif_cmd_done_intr_count_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_r.cnt.value :
'0';
1901   assign readback_array[23][0:0] = (de-
coded_reg_strb.intr_block_rf.error0_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error0_intr_count_incr_r.pulse.value :
'0';
1902   assign readback_array[23][31:1] = '0';
1903   assign readback_array[24][0:0] = (de-
coded_reg_strb.intr_block_rf.error1_intr_count_incr_r && !de-
coded_req_is_wr) ?

```

```

field_storage.intr_block_rf.error1_intr_count_incr_r.pulse.value :
'0;
1904     assign readback_array[24][31:1] = '0;
1905     assign readback_array[25][0:0] = (de-
coded_req_strb.intr_block_rf.error2_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error2_intr_count_incr_r.pulse.value :
'0;
1906     assign readback_array[25][31:1] = '0;
1907     assign readback_array[26][0:0] = (de-
coded_req_strb.intr_block_rf.error3_intr_count_incr_r && !de-
coded_req_is_wr) ?
field_storage.intr_block_rf.error3_intr_count_incr_r.pulse.value :
'0;
1908     assign readback_array[26][31:1] = '0;
1909     assign readback_array[27][0:0] = (de-
coded_req_strb.intr_block_rf.notif_cmd_done_intr_count_incr_r && !
decoded_req_is_wr) ?
field_storage.intr_block_rf.notif_cmd_done_intr_count_incr_r.pulse.
value : '0;
1910     assign readback_array[27][31:1] = '0;

```

```

1911
1912 // Reduce the array
1913 always_comb begin
1914     automatic logic [31:0] readback_data_var;
1915     readback_done = decoded_req & ~decoded_req_is_wr;
1916     readback_err = '0;
1917     readback_data_var = '0;
1918     for(int i=0; i<28; i++) readback_data_var |= read-
back_array[i];
1919     readback_data = readback_data_var;
1920 end
1921
1922 assign cpuif_rd_ack = readback_done;
1923 assign cpuif_rd_data = readback_data;
1924 assign cpuif_rd_err = readback_err;
1925
1926 `include "caliptra_prim_assert.sv"
1927 `CALIPTRA_ASSERT_KNOWN(ERR_HWIF_IN, hwif_in, clk, !hwif_in.er-
ror_reset_b)
1928
1929 endmodule

```

PULP 기본 셀 (FIFO·CDC·아비터)

28 files • 5,148 lines

API — assert_rpt_pkg.sv

Table 860. pulp-platform-common-cells/src/assert_rpt_pkg.sv — module / interface / package (1개)

이름	종류
assert_rpt_pkg	package

API — cc_addr_decode.sv

Table 861. pulp-platform-common-cells/src/cc_addr_decode.sv — module / interface / package (1개)

이름	종류
cc_addr_decode	module

Table 862. pulp-platform-common-cells/src/cc_addr_decode.sv — parameter (7개)

파라미터	형/폭	기본값
NoIndices	int unsigned	32'd0
NoRules	int unsigned	32'd1
addr_t		logic
Napot	bit	0
IdxWidth	int unsigned	cc_pkg::idx_width(NoIndices)
idx_t		logic [IdxWidth-1:0]
rule_t		struct packed {

Table 863. pulp-platform-common-cells/src/cc_addr_decode.sv — 포트 (7개)

포트	방향	형/폭
addr_i	input	addr_t
addr_map_i	input	rule_t [NoRules-1:0]
idx_o	output	idx_t
dec_valid_o	output	logic
dec_error_o	output	logic
en_default_idx_i	input	logic
default_idx_i	input	idx_t

API — cc_addr_decode_dync.sv

Table 864. pulp-platform-common-cells/src/cc_addr_decode_dync.sv — module / interface / package (1개)

이름	종류
cc_addr_decode_dync	module

Table 865. pulp-platform_common_cells/src/cc_addr_decode_dync.sv — parameter (7개)

파라미터	형/폭	기본값
NoIndices	int unsigned	32'd0
NoRules	int unsigned	32'd1
addr_t		logic
Napot	bit	0
IdxWidth	int unsigned	cc_pkg::idx_width(NoIndices)
idx_t		logic [IdxWidth-1:0]
rule_t		struct packed {

Table 866. pulp-platform_common_cells/src/cc_addr_decode_dync.sv — 포트 (8개)

포트	방향	형/폭
addr_i	input	addr_t
addr_map_i	input	rule_t [NoRules-1:0]
idx_o	output	idx_t
dec_valid_o	output	logic
dec_error_o	output	logic
en_default_idx_i	input	logic
default_idx_i	input	idx_t
config_ongoing_i	input	logic

API — cc_addr_decode_napot.sv**Table 867. pulp-platform_common_cells/src/cc_addr_decode_napot.sv — module / interface / package (1개)**

이름	종류
cc_addr_decode_napot	module

Table 868. pulp-platform_common_cells/src/cc_addr_decode_napot.sv — parameter (6개)

파라미터	형/폭	기본값
NoIndices	int unsigned	32'd0
NoRules	int unsigned	32'd1
addr_t		logic
IdxWidth	int unsigned	cc_pkg::idx_width(NoIndices)
idx_t		logic [IdxWidth-1:0]
rule_t		struct packed {

Table 869. pulp-platform_common_cells/src/cc_addr_decode_napot.sv — 포트 (7개)

포트	방향	형/폭
addr_i	input	addr_t

포트	방향	형/폭
addr_map_i	input	rule_t [NoRules-1:0]
idx_o	output	idx_t
dec_valid_o	output	logic
dec_error_o	output	logic
en_default_idx_i	input	logic
default_idx_i	input	idx_t

API — cc_binary_to_gray.sv

Table 870. pulp-platform_common_cells/src/cc_binary_to_gray.sv — module / interface / package (1개)

이름	종류
cc_binary_to_gray	module

Table 871. pulp-platform_common_cells/src/cc_binary_to_gray.sv — parameter (1개)

파라미터	형/폭	기본값
Width	int unsigned	1

Table 872. pulp-platform_common_cells/src/cc_binary_to_gray.sv — 포트 (2개)

포트	방향	형/폭
a_i	input	logic [Width-1:0]
z_o	output	logic [Width-1:0]

API — cc_boxcar.sv

Table 873. pulp-platform_common_cells/src/cc_boxcar.sv — module / interface / package (1개)

이름	종류
cc_boxcar	module

Table 874. pulp-platform_common_cells/src/cc_boxcar.sv — parameter (1개)

파라미터	형/폭	기본값
Width	int unsigned	32

Table 875. pulp-platform_common_cells/src/cc_boxcar.sv — localparam (3개)

상수	형/폭	값
IdxWidth	int unsigned	cc_pkg::idx_width(Width)
idx_t		logic [IdxWidth-1:0]
mask_t		logic [Width-1:0]

Table 876. pulp-platform_common_cells/src/cc_boxcar.sv — 포트 (3개)

포트	방향	형/폭
lsb_i	input	idx_t

포트	방향	형/폭
msb_i	input	idx_t
mask_o	output	mask_t

API — cc_cb_filter.sv

Table 877. pulp-platform_common_cells/src/cc_cb_filter.sv — module / interface / package (2개)

이름	종류
cc_cb_filter	module
cc_hash_block	module

Table 878. pulp-platform_common_cells/src/cc_cb_filter.sv — parameter (11개)

파라미터	형/폭	기본값
KHashes	int unsigned	32'd3
HashWidth	int unsigned	32'd4
HashRounds	int unsigned	32'd1
InpWidth	int unsigned	32'd32
BucketWidth	int unsigned	32'd4
Seeds	cc_pkg::cb_seed_t [KHashes-1:0]	cc_pkg::CbEgSeeds
NoHashes	int unsigned	32'd3
InpWidth	int unsigned	32'd11
HashWidth	int unsigned	32'd5
NoRounds	int unsigned	32'd1
Seeds	cc_pkg::cb_seed_t [NoHashes-1:0]	cc_pkg::CbEgSeeds

Table 879. pulp-platform_common_cells/src/cc_cb_filter.sv — localparam (1개)

상수	형/폭	값
NoCounters	int unsigned	2**HashWidth

Table 880. pulp-platform_common_cells/src/cc_cb_filter.sv — 포트 (15개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
look_data_i	input	logic [InpWidth-1:0]
look_valid_o	output	logic
incr_data_i	input	logic [InpWidth-1:0]
incr_valid_i	input	logic
decr_data_i	input	logic [InpWidth-1:0]

포트	방향	형/폭
decr_valid_i	input	logic
filter_usage_o	output	logic [HashWidth-1:0]
filter_full_o	output	logic
filter_empty_o	output	logic
filter_error_o	output	logic
data_i	input	logic [InpWidth-1:0]
indicator_o	output	logic [2**HashWidth-1:0]

API — cc_cdc_2phase.sv

Table 881. pulp-platform_common_cells/src/cc_cdc_2phase.sv — module / interface / package (3개)

이름	종류
cc_cdc_2phase	module
cc_cdc_2phase_src	module
cc_cdc_2phase_dst	module

Table 882. pulp-platform_common_cells/src/cc_cdc_2phase.sv — parameter (6개)

파라미터	형/폭	기본값
data_t		logic
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2

Table 883. pulp-platform_common_cells/src/cc_cdc_2phase.sv — 포트 (26개)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic

포트	방향	형/폭
rst_ni	input	logic
clk_i	input	logic
data_i	input	data_t
valid_i	input	logic
ready_o	output	logic
async_req_o	output	logic
async_ack_i	input	logic
async_data_o	output	data_t
rst_ni	input	logic
clk_i	input	logic
data_o	output	data_t
valid_o	output	logic
ready_i	input	logic
async_req_i	input	logic
async_ack_o	output	logic
async_data_i	input	data_t

API — cc_cdc_2phase_clearable.sv

Table 884. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — module / interface / package (5개)

이름	종류
cc_cdc_2phase_clearable	module
cc_cdc_2phase_clearable_dst	module
cc_cdc_2phase_clearable_src	module
cc_cdc_2phase_src_clearable	module
cc_cdc_2phase_dst_clearable	module

Table 885. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — parameter (13개)

파라미터	형/폭	기본값
data_t		logic
SyncStages	int unsigned	3
ClearOnAsyncReset	bit	1
data_t		logic
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic

파라미터	형/폭	기본값
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2

Table 886. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — localparam (2개)

상수	형/폭	값
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2

Table 887. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv — 포트 (64개) — 앞 60개만 표시(전체 64개, 본문 소스 참조)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic

포트	방향	형/폭
async_payload_req_i	input	logic
async_payload_ack_o	output	logic
async_payload_data_i	input	data_t
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_payload_req_o	output	logic
async_payload_ack_i	input	logic
async_payload_data_o	output	data_t
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
rst_ni	input	logic
clk_i	input	logic
clear_i	input	logic
data_i	input	data_t
valid_i	input	logic
ready_o	output	logic
async_req_o	output	logic
async_ack_i	input	logic
async_data_o	output	data_t
rst_ni	input	logic

포트	방향	형/폭
clk_i	input	logic
clear_i	input	logic
data_o	output	data_t
valid_o	output	logic

API — cc_cdc_4phase.sv

Table 888. pulp-platform_common_cells/src/cc_cdc_4phase.sv — module / interface / package (3개)

이름	종류
cc_cdc_4phase	module
cc_cdc_4phase_src	module
cc_cdc_4phase_dst	module

Table 889. pulp-platform_common_cells/src/cc_cdc_4phase.sv — parameter (13개)

파라미터	형/폭	기본값
data_t		logic
Decoupled	bit	1'b1
SendResetMsg	bit	1'b0
ResetMsg	data_t	data_t('0)
SyncStages	int unsigned	2
data_t		logic
SyncStages	int unsigned	2
Decoupled	bit	1'b1
SendResetMsg	bit	1'b0
ResetMsg	data_t	data_t('0)
data_t		logic
SyncStages	int unsigned	2
Decoupled	bit	1'b1

Table 890. pulp-platform_common_cells/src/cc_cdc_4phase.sv — localparam (2개)

상수	형/폭	값
ReqSrcResetValue	logic	SendResetMsg ? 1'b1 : 1'b0
DataSrcResetValue	data_t	SendResetMsg ? ResetMsg : data_t('0)

Table 891. pulp-platform_common_cells/src/cc_cdc_4phase.sv — 포트 (26개)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic

포트	방향	형/폭
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
rst_ni	input	logic
clk_i	input	logic
data_i	input	data_t
valid_i	input	logic
ready_o	output	logic
async_req_o	output	logic
async_ack_i	input	logic
async_data_o	output	data_t
rst_ni	input	logic
clk_i	input	logic
data_o	output	data_t
valid_o	output	logic
ready_i	input	logic
async_req_i	input	logic
async_ack_o	output	logic
async_data_i	input	data_t

Table 892. pulp-platform_common_cells/src/cc_cdc_4phase.sv — typedef / using (2개)

새 이름	원래 형
state_e	enum logic[1:0] {IDLE, WAIT_ACK_ASSERT, WAIT_ACK_DEASSERT}
state_e	enum logic[1:0] {IDLE, WAIT_DOWNSTREAM_ACK, WAIT_REQ_DEASSERT}

API — cc_cdc_fifo_2phase.sv

Table 893. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — module / interface / package (3개)

이름	종류
cc_cdc_fifo_2phase	module
cc_cdc_fifo_2phase_src	module
cc_cdc_fifo_2phase_dst	module

Table 894. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — parameter (9개)

파라미터	형/폭	기본값
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2

Table 895. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — localparam (5개)

상수	형/폭	값
PtrWidth	int unsigned	LogDepth+1
PtrWidth	int unsigned	LogDepth+1
PtrFull	pointer_t	(1 << LogDepth)
PtrWidth	int unsigned	LogDepth+1
PtrEmpty	pointer_t	'0

Table 896. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — 포트 (34개)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic

포트	방향	형/폭
async_data_o	output	data_t [2**LogDepth-1:0]
async_wptr_req_o	output	logic
async_wptr_ack_i	input	logic
async_wptr_data_o	output	logic [LogDepth:0]
async_rptr_req_i	input	logic
async_rptr_ack_o	output	logic
async_rptr_data_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
async_data_i	input	data_t [2**LogDepth-1:0]
async_wptr_req_i	input	logic
async_wptr_ack_o	output	logic
async_wptr_data_i	input	logic [LogDepth:0]
async_rptr_req_o	output	logic
async_rptr_ack_i	input	logic
async_rptr_data_o	output	logic [LogDepth:0]

Table 897. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv — typedef / using (5개)

새 이름	원래 형
pointer_t	logic [PtrWidth-1:0]
pointer_t	logic [PtrWidth-1:0]
index_t	logic [LogDepth-1:0]
pointer_t	logic [PtrWidth-1:0]
index_t	logic [LogDepth-1:0]

API — cc_cdc_fifo_gray.sv

Table 898. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — module / interface / package (7개)

이름	종류
cc_cdc_fifo_gray	module
cc_cdc_fifo_gray_src	module
cc_cdc_fifo_gray_dst	module
cc_cdc_fifo_gray_src_data	module
cc_cdc_fifo_gray_src_ptr	module

이름	종류
cc_cdc_fifo_gray_dst_ptr	module
cc_cdc_fifo_gray_dst_data	module

Table 899. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — parameter (18개)

파라미터	형/폭	기본값
Width	int unsigned	1
data_t		logic [Width-1:0]
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
LogDepth	int unsigned	3
SyncStages	int unsigned	2
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3

Table 900. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — localparam (4개)

상수	형/폭	값
PtrWidth	int unsigned	LogDepth+1
PtrFull	logic [PtrWidth-1:0]	(1 << LogDepth)
PtrWidth	int unsigned	LogDepth+1
PtrEmpty	logic [PtrWidth-1:0]	'0

Table 901. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv — 포트 (55개)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t

포트	방향	형/폭
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_data_o	output	data_t [2**LogDepth-1:0]
async_wptr_o	output	logic [LogDepth:0]
async_rptr_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
async_data_i	input	data_t [2**LogDepth-1:0]
async_wptr_i	input	logic [LogDepth:0]
async_rptr_o	output	logic [LogDepth:0]
src_rst_ni	input	logic
src_clk_i	input	logic
src_data_i	input	data_t
src_widx_i	input	logic [LogDepth-1:0]
src_write_i	input	logic
async_data_o	output	data_t [2**LogDepth-1:0]
src_rst_ni	input	logic
src_clk_i	input	logic
src_valid_i	input	logic
src_ready_o	output	logic
src_widx_o	output	logic [LogDepth-1:0]
async_wptr_o	output	logic [LogDepth:0]

포트	방향	형/폭
async_rptr_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_ready_i	input	logic
dst_valid_o	output	logic
dst_ridx_o	output	logic [LogDepth-1:0]
async_wptra_i	input	logic [LogDepth:0]
async_rptr_o	output	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
dst_valid_i	input	logic
dst_ready_o	output	logic
dst_ridx_i	input	logic [LogDepth-1:0]
async_data_i	input	data_t [2**LogDepth-1:0]

API — cc_cdc_fifo_gray_clearable.sv

Table 902. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — module / interface / package (9개)

이름	종류
cc_cdc_fifo_gray_clearable	module
cc_cdc_fifo_gray_clearable_src	module
cc_cdc_fifo_gray_clearable_dst	module
cc_cdc_fifo_gray_src_clearable	module
cc_cdc_fifo_gray_dst_clearable	module
cc_cdc_fifo_gray_src_data_clearable	module
cc_cdc_fifo_gray_src_ptr_clearable	module
cc_cdc_fifo_gray_dst_ptr_clearable	module
cc_cdc_fifo_gray_dst_data_clearable	module

Table 903. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — parameter (27개)

파라미터	형/폭	기본값
Width	int unsigned	1
data_t		logic [Width-1:0]

파라미터	형/폭	기본값
LogDepth	int unsigned	3
SyncStages	int unsigned	3
ClearOnAsyncReset	bit	1
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
ClearOnAsyncReset	bit	1
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3
LogDepth	int unsigned	3
SyncStages	int unsigned	2
LogDepth	int unsigned	3
SyncStages	int unsigned	2
data_t		logic
LogDepth	int unsigned	3

Table 904. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — localparam (6개)

상수	형/폭	값
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages - 1 : 2
PtrWidth	int unsigned	LogDepth+1
PtrFull	logic [PtrWidth-1:0]	(1 << LogDepth)
PtrWidth	int unsigned	LogDepth+1
PtrEmpty	logic [PtrWidth-1:0]	'0

Table 905. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv — 포트 (96개) — 앞 60개만 표시
(전체 96개, 본문 소스 참조)

포트	방향	형/폭
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_clear_pending_o	output	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_data_o	output	data_t [2**LogDepth-1:0]
async_wptr_o	output	logic [LogDepth:0]
async_rptr_i	input	logic [LogDepth:0]
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic

포트	방향	형/폭
dst_clear_pending_o	output	logic
dst_data_o	output	data_t
dst_valid_o	output	logic
dst_ready_i	input	logic
async_data_i	input	data_t [2**LogDepth-1:0]
async_wptr_i	input	logic [LogDepth:0]
async_rptr_o	output	logic [LogDepth:0]
async_reset_next_phase_o	output	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_o	output	logic
async_reset_ack_i	input	logic
async_reset_next_phase_i	input	cc_pkg::cdc_clear_seq_phase_e
async_reset_req_i	input	logic
async_reset_ack_o	output	logic
src_rst_ni	input	logic
src_clk_i	input	logic
src_clear_i	input	logic
src_data_i	input	data_t
src_valid_i	input	logic
src_ready_o	output	logic
async_data_o	output	data_t [2**LogDepth-1:0]
async_wptr_o	output	logic [LogDepth:0]
async_rptr_i	input	logic [LogDepth:0]
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_clear_i	input	logic
dst_data_o	output	data_t
dst_valid_o	output	logic

API — cc_cdc_reset_ctrlr.sv

Table 906. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — module / interface / package (2개)

이름	종류
cc_cdc_reset_ctrlr	module
cc_cdc_reset_ctrlr_half	module

Table 907. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — parameter (4개)

파라미터	형/폭	기본값
SyncStages	int unsigned	2
ClearOnAsyncReset	logic	1'b1
SyncStages	int unsigned	2
ClearOnAsyncReset	logic	1'b1

Table 908. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — localparam (2개)

상수	형/폭	값
InitiatorStateResetValue	initiator_state_e	ClearOnAsyncReset ? ISOLATE : IDLE
ResetSyncStages	int unsigned	(SyncStages > 2) ? SyncStages : 2

Table 909. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.sv — 포트 (27개)

포트	방향	형/폭
a_clk_i	input	logic
a_rst_ni	input	logic
a_clear_i	input	logic
a_clear_o	output	logic
a_clear_ack_i	input	logic
a_isolate_o	output	logic
a_isolate_ack_i	input	logic
b_clk_i	input	logic
b_rst_ni	input	logic
b_clear_i	input	logic
b_clear_o	output	logic
b_clear_ack_i	input	logic
b_isolate_o	output	logic
b_isolate_ack_i	input	logic
clk_i	input	logic
rst_ni	input	logic
clear_i	input	logic
isolate_o	output	logic
isolate_ack_i	input	logic
clear_o	output	logic
clear_ack_i	input	logic
async_next_phase_o	output	cdc_clear_seq_phase_e
async_req_o	output	logic
async_ack_i	input	logic

포트	방향	형/폭
async_next_phase_i	input	cdc_clear_seq_phase_e
async_req_i	input	logic
async_ack_o	output	logic

API — cc_clk_int_div.sv

Table 910. pulp-platform_common_cells/src/cc_clk_int_div.sv — module / interface / package (1개)

이름	종류
cc_clk_int_div	module

Table 911. pulp-platform_common_cells/src/cc_clk_int_div.sv — parameter (3개)

파라미터	형/폭	기본값
DivValueWidth	int unsigned	4
DefaultDivValue	int unsigned	0
EnableClockInReset	bit	1'b0

Table 912. pulp-platform_common_cells/src/cc_clk_int_div.sv — localparam (3개)

상수	형/폭	값
DivResetValue	int unsigned	(DefaultDivValue != 0)? DefaultDivValue: 1
UseOddDivisionResetValue	logic	DefaultDivValue[0]
ClkDivBypassEnResetValue	logic	(DefaultDivValue < 2)? 1'b1: 1'b0

Table 913. pulp-platform_common_cells/src/cc_clk_int_div.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
en_i	input	logic
test_mode_en_i	input	logic
div_i	input	logic [DivValueWidth-1:0]
div_valid_i	input	logic
div_ready_o	output	logic
clk_o	output	logic
cycl_count_o	output	logic [DivValueWidth-1:0]

Table 914. pulp-platform_common_cells/src/cc_clk_int_div.sv — typedef / using (1개)

새 이름	원래 형
clk_gate_state_e	enum logic[1:0] {IDLE, LOAD_DIV, WAIT_END_PERIOD}

API — cc_clk_int_div_static.sv**Table 915. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — module / interface / package (1개)**

이름	종류
cc_clk_int_div_static	module

Table 916. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — parameter (2개)

파라미터	형/폭	기본값
DivValue	int unsigned	1
EnableClockInReset	bit	1'b1

Table 917. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — localparam (1개)

상수	형/폭	값
DivValueWidth	int unsigned	\$clog2(DivValue+1)

Table 918. pulp-platform_common_cells/src/cc_clk_int_div_static.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
en_i	input	logic
test_mode_en_i	input	logic
clk_o	output	logic

API — cc_clk_mux_glitch_free.sv**Table 919. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — module / interface / package (2개)**

이름	종류
cc_clk_mux_glitch_free	module
cc_clk_or_tree	module

Table 920. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — parameter (3개)

파라미터	형/폭	기본값
NumInputs	int unsigned	2
NumSyncStages	int unsigned	2
ClockDuringReset	bit	1'b1

Table 921. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — localparam (1개)

상수	형/폭	값
SelWidth	int unsigned	\$clog2(NumInputs)

Table 922. pulp-platform_common_cells/src/cc_clk_mux_glitch_free.sv — 포트 (8개)

포트	방향	형/폭
clks_i	input	logic [NumInputs-1:0]
test_clk_i	input	logic
test_en_i	input	logic
async_rstn_i	input	logic
async_sel_i	input	logic [SelWidth-1:0]
clk_o	output	logic
clks_i	input	logic [NumInputs-1:0]
clk_o	output	logic

API — cc_counter.sv

Table 923. pulp-platform_common_cells/src/cc_counter.sv — module / interface / package (1개)

이름	종류
cc_counter	module

Table 924. pulp-platform_common_cells/src/cc_counter.sv — parameter (2개)

파라미터	형/폭	기본값
Width	int unsigned	4
StickyOverflow	bit	1'b0

Table 925. pulp-platform_common_cells/src/cc_counter.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
en_i	input	logic
load_i	input	logic
down_i	input	logic
d_i	input	logic [Width-1:0]
q_o	output	logic [Width-1:0]
overflow_o	output	logic

API — cc_credit_counter.sv

Table 926. pulp-platform_common_cells/src/cc_credit_counter.sv — module / interface / package (1개)

이름	종류
cc_credit_counter	module

Table 927. pulp-platform_common_cells/src/cc_credit_counter.sv — parameter (2개)

파라미터	형/폭	기본값
NumCredits	int unsigned	0
InitCreditEmpty	bit	1'b0

Table 928. pulp-platform_common_cells/src/cc_credit_counter.sv — localparam (2개)

상수	형/폭	값
InitNumCredits	int unsigned	InitCreditEmpty ? '0 : NumCredits
credit_cnt_t		logic [\$clog2(NumCredits):0]

Table 929. pulp-platform_common_cells/src/cc_credit_counter.sv — 포트 (9개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
credit_o	output	credit_cnt_t
credit_give_i	input	logic
credit_take_i	input	logic
credit_left_o	output	logic
credit_crit_o	output	logic
credit_full_o	output	logic

API — cc_delta_counter.sv

Table 930. pulp-platform_common_cells/src/cc_delta_counter.sv — module / interface / package (1개)

이름	종류
cc_delta_counter	module

Table 931. pulp-platform_common_cells/src/cc_delta_counter.sv — parameter (2개)

파라미터	형/폭	기본값
Width	int unsigned	4
StickyOverflow	bit	1'b0

Table 932. pulp-platform_common_cells/src/cc_delta_counter.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
en_i	input	logic
load_i	input	logic
down_i	input	logic

포트	방향	형/폭
delta_i	input	logic [Width-1:0]
d_i	input	logic [Width-1:0]
q_o	output	logic [Width-1:0]
overflow_o	output	logic

API — cc_ecc_decode.sv

Table 933. pulp-platform_common_cells/src/cc_ecc_decode.sv — module / interface / package (1개)

이름	종류
cc_ecc_decode	module

Table 934. pulp-platform_common_cells/src/cc_ecc_decode.sv — parameter (1개)

파라미터	형/폭	기본값
DataWidth	int unsigned	64

Table 935. pulp-platform_common_cells/src/cc_ecc_decode.sv — localparam (4개)

상수	형/폭	값
data_t		logic [DataWidth-1:0]
parity_t		logic [ecc_get_parity_width(DataWidth)-1:0]
code_word_t		logic [ecc_get_cw_width(DataWidth)-1:0]
encoded_data_t		struct packed {

Table 936. pulp-platform_common_cells/src/cc_ecc_decode.sv — 포트 (6개)

포트	방향	형/폭
data_i	input	encoded_data_t
data_o	output	data_t
syndrome_o	output	parity_t
single_error_o	output	logic
parity_error_o	output	logic
double_error_o	output	logic

API — cc_ecc_encode.sv

Table 937. pulp-platform_common_cells/src/cc_ecc_encode.sv — module / interface / package (1개)

이름	종류
cc_ecc_encode	module

Table 938. pulp-platform_common_cells/src/cc_ecc_encode.sv — parameter (1개)

파라미터	형/폭	기본값
DataWidth	int unsigned	64

Table 939. pulp-platform_common_cells/src/cc_ecc_encode.sv — localparam (4개)

상수	형/폭	값
data_t		logic [DataWidth-1:0]
parity_t		logic [ecc_get_parity_width(DataWidth)-1:0]
code_word_t		logic [ecc_get_cw_width(DataWidth)-1:0]
encoded_data_t		struct packed {

Table 940. pulp-platform_common_cells/src/cc_ecc_encode.sv — 포트 (2개)

포트	방향	형/폭
data_i	input	data_t
data_o	output	encoded_data_t

API — cc_edge_detect.sv**Table 941. pulp-platform_common_cells/src/cc_edge_detect.sv — module / interface / package (1개)**

이름	종류
cc_edge_detect	module

Table 942. pulp-platform_common_cells/src/cc_edge_detect.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
d_i	input	logic
re_o	output	logic
detected	output	logic fe_o // Falling edge

API — cc_edge_propagator.sv**Table 943. pulp-platform_common_cells/src/cc_edge_propagator.sv — module / interface / package (1개)**

이름	종류
cc_edge_propagator	module

Table 944. pulp-platform_common_cells/src/cc_edge_propagator.sv — 포트 (6개)

포트	방향	형/폭
clk_tx_i	input	logic
rst_tx_ni	input	logic
edge_i	input	logic
clk_rx_i	input	logic
rst_rx_ni	input	logic
edge_o	output	logic

API — cc_edge_propagator_ack.sv**Table 945. pulp-platform_common_cells/src/cc_edge_propagator_ack.sv — module / interface / package (1개)**

이름	종류
cc_edge_propagator_ack	module

Table 946. pulp-platform_common_cells/src/cc_edge_propagator_ack.sv — 포트 (7개)

포트	방향	형/폭
clk_tx_i	input	logic
rst_tx_ni	input	logic
edge_i	input	logic
ack_tx_o	output	logic
clk_rx_i	input	logic
rst_rx_ni	input	logic
edge_o	output	logic

API — cc_edge_propagator_rx.sv**Table 947. pulp-platform_common_cells/src/cc_edge_propagator_rx.sv — module / interface / package (1개)**

이름	종류
cc_edge_propagator_rx	module

Table 948. pulp-platform_common_cells/src/cc_edge_propagator_rx.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
valid_i	input	logic
ack_o	output	logic
valid_o	output	logic

API — cc_edge_propagator_tx.sv**Table 949. pulp-platform_common_cells/src/cc_edge_propagator_tx.sv — module / interface / package (1개)**

이름	종류
cc_edge_propagator_tx	module

Table 950. pulp-platform_common_cells/src/cc_edge_propagator_tx.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
valid_i	input	logic

포트	방향	형/폭
ack_i	input	logic
valid_o	output	logic

API — cc_exp_backoff.sv

Table 951. pulp-platform-common-cells/src/cc_exp_backoff.sv — module / interface / package (1개)

이름	종류
cc_exp_backoff	module

Table 952. pulp-platform-common-cells/src/cc_exp_backoff.sv — parameter (2개)

파라미터	형/폭	기본값
Seed	int unsigned	'hffff
MaxExp	int unsigned	16

Table 953. pulp-platform-common-cells/src/cc_exp_backoff.sv — localparam (1개)

상수	형/폭	값
WIDTH	int unsigned	16

Table 954. pulp-platform-common-cells/src/cc_exp_backoff.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
clr_i	input	logic
set_cnt_i	input	logic
clr_cnt_i	input	logic
is_zero_o	output	logic

1. pulp-platform-common-cells/src/assert_rpt_pkg.sv (17 lines)

```

1 // Copyright lowRISC contributors.
2 // Licensed under the Apache License, Version 2.0, see LICENSE for
  details.
3 // SPDX-License-Identifier: Apache-2.0
4
5 `ifndef UVM
6 package assert_rpt_pkg;
7
8 import uvm_pkg::*;
9
10 `include "uvm_macros.svh"
11
12 function automatic void assert_rpt(string msg);
13     `uvm_error("ASSERT FAILED", msg)
14 endfunction
15
16 endpackage
17 `endif

```

2. pulp-platform-common-cells/src/cc_addr_decode.sv (114 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware

```

```

3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations under
  the License.
10
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@ethz.ch>
13 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
14
15 /// Address Decoder: Maps the input address combinatorially to an
  index.
16 /// The address map `addr_map_i` is a packed array of rule_t
  structs.
17 /// The ranges of any two rules may overlap. If so, the rule
  at the higher (more significant)
18 /// position in `addr_map_i` prevails.
19 ///
20 /// There can be an arbitrary number of address rules. There
  can be multiple
21 /// ranges defined for the same index. The start address has
  to be less than the end address.
22 ///

```

```

23 /// There is the possibility to add a default mapping:
24 /// `en_default_idx_i`: Driving this port to `1'b1` maps all
input addresses
25 /// for which no rule in `addr_map_i` exists to the default
index specified by
26 /// `default_idx_i`. In this case, `dec_error_o` is always
`1'b0`.
27 ///

28 /// The `Napot` parameter allows using naturally-aligned power of
two (NAPOT) regions,
29 /// using base addresses and masks instead of address ranges
to specify rules.
30 ///

31 /// Assertions: The module checks every time there is a change in
the address mapping
32 /// if the resulting map is valid. It fatals if `start_addr`
is higher than `end_addr` (non-NAPOT

33 /// only) or if a mapping targets an index that is outside the
number of allowed indices.
34 /// It issues warnings if the address regions of any two map-
pings overlap (non-NAPOT only).
35 module cc_addr_decode_dync #(
36     /// Highest index which can happen in a rule.
37     parameter int unsigned NoIndices = 32'd0,
38     /// Total number of rules.
39     parameter int unsigned NoRules = 32'd1,
40     /// Address type inside the rules and to decode.
41     parameter type addr_t = logic,
42     /// Whether this is a NAPOT (base and mask) or regular range
decoder
43     parameter bit Napot = 0,
44     /// The output index type `idx_t` can be specified either
with the width `IdxWidth`
45     /// or directly with the type `idx_t`. By default, it will
use the maximum index
46     /// `NoIndices` to calculate the required width.
47     parameter int unsigned IdxWidth =
cc_pkg::idx_width(NoIndices),
48     parameter type idx_t = logic [IdxWidth-1:0],
49     /// Rule packed struct type.
50     /// The address decoder expects three fields in `rule_t`:
51     ///
52     /// typedef struct packed {
53     ///     idx_t idx;
54     ///     addr_t start_addr;
55     ///     addr_t end_addr;
56     /// } rule_t;
57     ///

58     /// - `idx`: The index to be returned for a matching
rule. Usually an integer but can
59     /// be any type of data.
60     /// - `start_addr`: start address of the range the rule de-
scribes, value is included in range
61     /// - `end_addr`: end address of the range the rule de-
scribes, value is NOT included in range
62     /// if `end_addr == '0` end of address
space is assumed
63     ///
64     /// If `Napot` is 1, The field names remain the same, but
the rule describes a naturally-aligned

65     /// power of two (NAPOT) region instead of an address range:
`start_addr` becomes the base address
66     /// and `end_addr` the mask. See the wrapping module `cc_ad-
dr_decode_napot` for details.
67     // verilog_lint: waive typedef-structs-unions
68     parameter type rule_t = struct packed {
69         idx_t idx;
70         addr_t start_addr;
71         addr_t end_addr;
72     }
73 ) (
74     /// Address to decode.
75     input addr_t addr_i,
76     /// Address map: rule with the highest array position wins
on collision
77     input rule_t [NoRules-1:0] addr_map_i,
78     /// Decoded index.
79     output idx_t idx_o,
80     /// Decode is valid.
81     output logic dec_valid_o,
82     /// Decode is not valid, no matching rule found.
83     output logic dec_error_o,
84     /// Enable default port mapping.
85     ///
86     /// When not used, tie to `0`.
87     input logic en_default_idx_i,
88     /// Default port index.
89     ///
90     /// When `en_default_idx_i` is `1`, this will be the index
when no rule matches.
91     ///
92     /// When not used, tie to `0`.
93     input idx_t default_idx_i
94 );
95
96 /// wraps the dynamic configuration version of the address

```

```

decoder
97     cc_addr_decode_dync #(
98         .NoRules ( NoRules ),
99         .addr_t ( addr_t ),
100        .rule_t ( rule_t ),
101        .Napot ( Napot ),
102        .idx_t ( idx_t )
103    ) i_addr_decode_dync (
104        .addr_i,
105        .addr_map_i,
106        .idx_o,
107        .dec_valid_o,
108        .dec_error_o,
109        .en_default_idx_i,
110        .default_idx_i,
111        .config_ongoing_i ( 1'b0 )
112    );
113
114 endmodule

```

3. pulp-platform_common_cells/src/ cc_addr_decode_dync.sv (182 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware

3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@ethz.ch>
13 // - Paul Scheffler <paulsc@iis.ee.ethz.ch>
14 // - Michael Rogenmoser <michaelro@iis.ee.ethz.ch>
15 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
16
17 `include "common_cells/assertions.svh"
18

19 /// Address Decoder: Maps the input address combinatorially to an
index.
20 /// DYNAMIC Configuration (DYNC) version
21 /// The address map `addr_map_i` is a packed array of rule_t
structs.
22 /// The ranges of any two rules may overlap. If so, the rule
at the higher (more significant)
23 /// position in `addr_map_i` prevails.
24 ///
25 /// There can be an arbitrary number of address rules. There
can be multiple
26 /// ranges defined for the same index. The start address has
to be less than the end address.
27 ///
28 /// There is the possibility to add a default mapping:
29 /// `en_default_idx_i`: Driving this port to `1'b1` maps all
input addresses
30 /// for which no rule in `addr_map_i` exists to the default
index specified by
31 /// `default_idx_i`. In this case, `dec_error_o` is always
`1'b0`.
32 ///

33 /// The `Napot` parameter allows using naturally-aligned power of
two (NAPOT) regions,
34 /// using base addresses and masks instead of address ranges
to specify rules.
35 ///

36 /// Assertions: The module checks every time there is a change in
the address mapping
37 /// if the resulting map is valid. It fatals if `start_addr`
is higher than `end_addr` (non-NAPOT

38 /// only) or if a mapping targets an index that is outside the
number of allowed indices.
39 /// It issues warnings if the address regions of any two map-
pings overlap (non-NAPOT only).
40 module cc_addr_decode_dync #(
41     /// Highest index which can happen in a rule.
42     parameter int unsigned NoIndices = 32'd0,
43     /// Total number of rules.
44     parameter int unsigned NoRules = 32'd1,
45     /// Address type inside the rules and to decode.
46     parameter type addr_t = logic,
47     /// Whether this is a NAPOT (base and mask) or regular range
decoder
48     parameter bit Napot = 0,

```

```

49  /// The output index type `idx_t` can be specified either
with the width `IdxWidth`
50  /// or directly with the type `idx_t`. By default, it will
use the maximum index
51  /// `NoIndices` to calculate the required width.
52  parameter int unsigned IdxWidth =
cc_pkg::idx_width(NoIndices),
53  parameter type      idx_t      = logic [IdxWidth-1:0],
54  /// Rule packed struct type.
55  /// The address decoder expects three fields in `rule_t`:
56  ///
57  /// typedef struct packed {
58  ///     idx_t idx;
59  ///     addr_t start_addr;
60  ///     addr_t end_addr;
61  /// } rule_t;
62  ///

63  /// - `idx`: The index to be returned for a matching
rule. Usually an integer but can
64  /// be any type of data.
65  /// - `start_addr`: start address of the range the rule de-
scribes, value is included in range
66  /// - `end_addr`: end address of the range the rule de-
scribes, value is NOT included in range
67  /// if `end_addr == '0` end of address
space is assumed
68  ///
69  /// If `Napot` is 1, The field names remain the same, but
the rule describes a naturally-aligned

70  /// power of two (NAPOT) region instead of an address range:
`start_addr` becomes the base address
71  /// and `end_addr` the mask. See the wrapping module `cc_ad-
dr_decode_napot` for details.
72  // verilog_lint: waive typedef-structs-unions
73  parameter type      rule_t      = struct packed {
74  idx_t idx;
75  addr_t start_addr;
76  addr_t end_addr;
77  }
78  ) (
79  /// Address to decode.
80  input  addr_t      addr_i,
81  /// Address map: rule with the highest array position wins
on collision
82  input  rule_t [NoRules-1:0] addr_map_i,
83  /// Decoded index.
84  output idx_t      idx_o,
85  /// Decode is valid.
86  output logic      dec_valid_o,
87  /// Decode is not valid, no matching rule found.
88  output logic      dec_error_o,
89  /// Enable default port mapping.
90  ///
91  /// When not used, tie to `0`.
92  input  logic      en_default_idx_i,
93  /// Default port index.
94  ///
95  /// When `en_default_idx_i` is `1`, this will be the index
when no rule matches.
96  ///
97  /// When not used, tie to `0`.
98  input  idx_t      default_idx_i,
99  /// The module is dynamically configured, this deactivates
its output and the integrated
100  /// assertions.
101  input  logic      config_ongoing_i
102  );
103

104  logic [NoRules-1:0] matched_rules; // purely for address map
debugging
105
106  always_comb begin
107  // default assignments
108  matched_rules = '0;
109  dec_valid_o   = 'b0;
110  dec_error_o   = (en_default_idx_i) ? 1'b0 : 1'b1;
111  idx_o         = (en_default_idx_i) ? default_idx_i : '0;
112
113  // match the rules
114  for (int unsigned i = 0; i < NoRules; i++) begin
115  if (
116  !Napot && (addr_i >= addr_map_i[i].start_addr) &&
117  ((addr_i < addr_map_i[i].end_addr) ||
(addr_map_i[i].end_addr == '0)) ||
118  Napot && (addr_map_i[i].start_addr &
addr_map_i[i].end_addr) ==
119  (addr_i & addr_map_i[i].end_addr)
120  ) begin
121  matched_rules[i] = ~config_ongoing_i;
122  dec_valid_o      = ~config_ongoing_i;
123  dec_error_o      = 1'b0;
124  idx_o            = config_ongoing_i ? default_idx_i :
idx_t'(addr_map_i[i].idx);
125  end
126  end
127  end
128
129  // Assumptions and assertions

```

```

130  `ifndef COMMON_CELLS_ASSERTS_OFF
131  `ifndef SYNTHESIS
132  initial begin : proc_check_parameters
133  `ASSUME_I(addr_width_mismatch, $bits(addr_i) ==
$bits(addr_map_i[0].start_addr),
134  $formatf("Input address has %d bits and address
map has %d bits.",
135  $bits(addr_i),
136  $bits(addr_map_i[0].start_addr)))
137  `ASSUME_I(norules_0, NoRules > 0, $formatf("At least one
rule needed"))
138  end
139  `ASSERT_FINAL(more_than_1_bit_set, $onehot0(matched_rules)
|| config_ongoing_i,
140  "More than one bit set in the one-hot signal,
matched_rules")
141
142  // These following assumptions check the validity of the ad-
dress map.
143  // The assumptions gets generated for each distinct pair of
rules.
144  // Each assumption is present two times, as they rely on one
rules being
145  // effectively ordered. Only one of the rules with the same
function is
146  // active at a time for a given pair.
147  // check_start: Enforces a smaller start than end ad-
dress.
148  // check_idx: Enforces a valid index in the rule.
149  // check_overlap: Warns if there are overlapping ad-
dress regions.
150  always_comb begin : proc_check_addr_map
151  if (!$isunknown(addr_map_i) && ~config_ongoing_i) begin
152  for (int unsigned i = 0; i < NoRules; i++) begin
153  `ASSUME_I(check_start, Napot ||
addr_map_i[i].start_addr < addr_map_i[i].end_addr ||
154  addr_map_i[i].end_addr == '0,
155  $formatf("This rule has a higher start than end ad-
dress!!!\n\
156  Violating rule %d.\n\
157  Rule> IDX: %h START: %h END: %h\n\
158  #####",
159  i, addr_map_i[i].idx, addr_map_i[i].start_addr,
160  addr_map_i[i].end_addr))
161  for (int unsigned j = i + 1; j < NoRules; j++) begin
162  // overlap check
163  `ASSUME_I(check_overlap, Napot ||
!((addr_map_i[j].start_addr
164  < addr_map_i[i].end_addr) &&
(addr_map_i[j].end_addr >
165  addr_map_i[i].start_addr)) ||
166  !((addr_map_i[i].end_addr ==
'0) &&
(addr_map_i[j].end_addr >
167  addr_map_i[i].start_addr)) ||
168  !((addr_map_i[j].start_addr
< addr_map_i[i].end_addr) &&
(addr_map_i[j].end_addr ==
'0))),
169  $formatf("Overlapping address region found!!!
\n\
170  Rule %d: IDX: %h START: %h END: %h\n\
171  Rule %d: IDX: %h START: %h END: %h\n\
172  #####",
173  i, addr_map_i[i].idx, addr_map_i[i].start_addr,
174  addr_map_i[i].end_addr,
175  j, addr_map_i[j].idx, addr_map_i[j].start_addr,
176  addr_map_i[j].end_addr))
177  end
178  end
179  `endif
180  `endif
181
182  endmodule

```

4. pulp-platform_common_cells/src/ cc_addr_decode_napot.sv (102 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR

```

```

8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
12 // Author: Paul Scheffler <paulsc@iis.ee.ethz.ch>
13
14 /// This module wraps `cc_addr_decode_dync` in its naturally-
aligned power of two (NAPOT) variant,
15 /// alleviating the need to set the `Napot` parameter and us-
ing more descriptive `rule_t`
16 /// field names. See the `cc_addr_decode` documentation for de-
tails.
17 module cc_addr_decode_napot #(
18     /// Highest index which can happen in a rule.
19     parameter int unsigned NoIndices = 32'd0,
20     /// Total number of rules.
21     parameter int unsigned NoRules = 32'd1,
22     /// Address type inside the rules and to decode.
23     parameter type addr_t = logic,
24     /// The output index type `idx_t` can be specified either
with the width `IdxWidth`
25     /// or directly with the type `idx_t`. By default, it will
use the maximum index
26     /// `NoIndices` to calculate the required width.
27     parameter int unsigned IdxWidth =
cc_pkg::idx_width(NoIndices),
28     parameter type idx_t = logic [IdxWidth-1:0],
29     /// Rule packed struct type.
30     /// The NAPOT address decoder expects three fields in
`rule_t`:
31     ///
32     /// typedef struct packed {
33     ///     idx_t    idx;
34     ///     addr_t   base;
35     ///     addr_t   mask;
36     /// } rule_t;
37     ///
38     /// - `idx`: index of the rule, has to be < `NoIndices`.
39     /// - `base`: base address whose specified bits should
match `addr_i`.
40     /// - `mask`: set for bits which are to be checked to de-
termine a match.
41     // verilog_lint: waive typedef-structs-unions
42     parameter type rule_t = struct packed {
43         idx_t    idx;
44         addr_t   base;
45         addr_t   mask;
46     }
47 ) (
48     /// Address to decode.
49     input  addr_t      addr_i,
50     /// Address map: rule with the highest array position wins
on collision
51     input  rule_t [NoRules-1:0] addr_map_i,
52     /// Decoded index.
53     output idx_t      idx_o,
54     /// Decode is valid.
55     output logic      dec_valid_o,
56     /// Decode is not valid, no matching rule found.
57     output logic      dec_error_o,
58     /// Enable default port mapping.
59     ///
60     /// When not used, tie to `0`.
61     input  logic      en_default_idx_i,
62     /// Default port index.
63     ///
64     /// When `en_default_idx_i` is `1`, this will be the index
when no rule matches.
65     ///
66     /// When not used, tie to `0`.
67     input  idx_t      default_idx_i
68 );
69
70 // Rename struct field names to those expected by `cc_ad-
dr_decode`
71 typedef struct packed {
72     idx_t    idx;
73     addr_t   start_addr;
74     addr_t   end_addr;
75 } rule_range_t;
76
77 rule_range_t [NoRules-1:0] range_addr_map;
78
79 for (genvar i = 0; i < NoRules; i++) begin : gen_rule_assign
80     assign range_addr_map[i].idx      = addr_map_i[i].idx;
81     assign range_addr_map[i].start_addr = addr_map_i[i].base;
82     assign range_addr_map[i].end_addr  = addr_map_i[i].mask;
83 end
84
85 cc_addr_decode_dync #(
86     .NoIndices ( NoIndices ),
87     .NoRules   ( NoRules ),
88     .addr_t    ( addr_t ),
89     .rule_t    ( rule_range_t ),
90     .Napot     ( 1 )
91 ) i_addr_decode_dync (
92     .addr_i,
93     .addr_map_i ( range_addr_map ),
94     .idx_o,

```

```

95     .dec_valid_o,
96     .dec_error_o,
97     .en_default_idx_i,
98     .default_idx_i,
99     .config_ongoing_i ( 1'b0 )
100 );
101
102 endmodule

```

5. pulp-platform_common_cells/src/cc_binary_to_gray.sv (25 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13
14 /// A binary to gray code converter.
15 module cc_binary_to_gray #(
16     parameter int unsigned Width = 1
17 ) (
18     input  logic [Width-1:0] a_i,
19     output logic [Width-1:0] z_o
20 );
21     logic [Width:0] a_ext;
22
23     assign a_ext = {1'b0, a_i};
24     assign z_o   = a_i ^ a_ext[Width:1];
25 endmodule

```

6. pulp-platform_common_cells/src/cc_boxcar.sv (31 lines)

```

1 // Copyright 2025 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
details.
3 // SPDX-License-Identifier: SHL-0.51
4 //
5 // Author: Luca Colagrande <colluca@iis.ee.ethz.ch>
6 //
7 // This module can be used to generate any mask that can be
obtained
8 // by a boxcar function (https://en.wikipedia.org/wiki/Boxcar\_function).
9 // Specifically, it generates a mask with all and only the
bits in
10 // the [lsb_i, msb_i] interval asserted.
11
12 module cc_boxcar #(
13     parameter int unsigned Width = 32,
14     /// Derived parameter *Do not override*
15     localparam int unsigned IdxWidth =
cc_pkg::idx_width(Width),
16     localparam type idx_t = logic [IdxWidth-1:0],
17     localparam type mask_t = logic [Width-1:0]
18 ) (
19     input idx_t lsb_i,
20     input idx_t msb_i,
21     output mask_t mask_o
22 );
23
24     mask_t low_mask, high_mask_n;
25
26     cc_heaviside #(Width(Width)) i_lo
(.x_i(lsb_i), .mask_o(low_mask));
27     cc_heaviside #(Width(Width)) i_hi
(.x_i(msb_i), .mask_o(high_mask_n));
28
29     assign mask_o = ~low_mask & high_mask_n;
30
31 endmodule

```

7. pulp-platform_common_cells/src/cc_cb_filter.sv (248 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by

```



```

applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 // Author: Wolfgang Roenninger <wroennin@ethz.ch>
12
13 // `cc_cb_filter`: This module implements a counting bloom
filter with parameterizable hash functions.
14 //
15 // Functionality: A counting bloom filter is a data structure
to efficiently implement
16 // set lookups. It does so by hashing its data
inputs onto multiple pointers
17 // which serve as indicators for an array of
buckets. For lookups can be
18 // false positives, but no false negatives.
19 // - Seeding: The pseudo random generators need seeds at
elaboration time to generate
20 // different hashes. In principle any combina-
tion of seeds can be used.
21 // But one should look that the hash outputs
give sufficient different patterns,
22 // such that the resulting collision rate is
low. The package `cc`
23 // contains the struct for seeding the PRG's in
the hash functions.
24 // - Lookup:
25 //   - Ports: `look_data_i`, `look_valid_o`
26 //   - Description: Lookup combinational, `look_valid_o` is
high, when `look_data_i` was
27 //   previously put into the filter.
28 // - Increment:
29 //   - Ports: `incr_data_i`, `incr_valid_i`
30 //   - Description: Put data into the counting bloom filter,
when valid is high.
31 // - Decrement:
32 //   - Ports: `decr_data_i`, `decr_valid_i`
33 //   - Description: Remove data from the counting bloom fil-
ter. Only remove data that was
34 //   previously put in, otherwise will go in a
wrong state.
35 // - Status:
36 //   - `clr_i`: Clears the filter and sets all coun-
ters to 0.
37 //   - `filter_usage_o`: How many data items are currently in
the filter.
38 //   - `filter_full_o`: Filter is full, can no longer hold
more items.
39 //   - `filter_empty_o`: Filter is empty.
40 //   - `filter_error_o`: One of the internal counters or
buckets overflowed.
41
42 `include "common_cells/assertions.svh"
43
44 /// This is a counting bloom filter
45 module cc_cb_filter #(
46     parameter int unsigned KHashes = 32'd3, // Number of
hash functions
47     parameter int unsigned HashWidth = 32'd4, // Number of
counters is 2**HashWidth
48     parameter int unsigned HashRounds = 32'd1, // Number of
permutation substitution rounds
49     parameter int unsigned InpWidth = 32'd32, // Input data
width
50     parameter int unsigned BucketWidth = 32'd4, // Width of
Bucket counters
51     // the seeds used for seeding the PRG's inside each hash,
one `cb_seed_t` per hash function.
52     parameter cc_pkg::cb_seed_t [KHashes-1:0] Seeds =
cc_pkg::CbEgSeeds
53 ) (
54     input logic clk_i, // Clock
55     input logic rst_ni, // Asynchronous reset
active low
56     input logic clr_i, // Synchronous clear
active high
57     // data lookup
58     input logic [InpWidth-1:0] look_data_i,
59     output logic look_valid_o,
60     // data increment
61     input logic [InpWidth-1:0] incr_data_i,
62     input logic incr_valid_i,
63     // data decrement
64     input logic [InpWidth-1:0] decr_data_i,
65     input logic decr_valid_i,
66     // status signals
67     output logic [HashWidth-1:0] filter_usage_o,
68     output logic filter_full_o,
69     output logic filter_empty_o,
70     output logic filter_error_o
71 );
72
73 localparam int unsigned NoCounters = 2**HashWidth;

```

```

74
75 // signal declarations
76 logic [NoCounters-1:0] look_ind; // hash function pointers
77 logic [NoCounters-1:0] incr_ind; // hash function pointers
78 logic [NoCounters-1:0] decr_ind; // hash function pointers
79 // bucket (counter signals)
80 logic [NoCounters-1:0] bucket_en;
81 logic [NoCounters-1:0] bucket_down;
82 logic [NoCounters-1:0] bucket_occupied;
83 logic [NoCounters-1:0] bucket_overflow;
84 logic [NoCounters-1:0] bucket_full;
85 logic [NoCounters-1:0] bucket_empty;
86 // membership lookup signals
87 logic [NoCounters-1:0] data_in_bucket;
88 // tot count signals (filter usage)
89 logic cnt_en;
90 logic cnt_down;
91 logic cnt_overflow;
92
93 // -----
94 // Lookup Hash - Membership Detection
95 // -----
96 cc_hash_block #(
97     .NoHashes ( KHashes ),
98     .InpWidth ( InpWidth ),
99     .HashWidth ( HashWidth ),
100     .NoRounds ( HashRounds ),
101     .Seeds ( Seeds )
102 ) i_look_hashes (
103     .data_i ( look_data_i ),
104     .indicator_o ( look_ind )
105 );
106 assign data_in_bucket = look_ind & bucket_occupied;
107 assign look_valid_o = (data_in_bucket == look_ind) ?
1'b1 : 1'b0;
108
109 // -----
110 // Increment Hash - Add Member to Set
111 // -----
112 cc_hash_block #(
113     .NoHashes ( KHashes ),
114     .InpWidth ( InpWidth ),
115     .HashWidth ( HashWidth ),
116     .NoRounds ( HashRounds ),
117     .Seeds ( Seeds )
118 ) i_incr_hashes (
119     .data_i ( incr_data_i ),
120     .indicator_o ( incr_ind )
121 );
122
123 // -----
124 // Decrement Hash - Remove Member from Set
125 // -----
126 cc_hash_block #(
127     .NoHashes ( KHashes ),
128     .InpWidth ( InpWidth ),
129     .HashWidth ( HashWidth ),
130     .NoRounds ( HashRounds ),
131     .Seeds ( Seeds )
132 ) i_decr_hashes (
133     .data_i ( decr_data_i ),
134     .indicator_o ( decr_ind )
135 );
136
137 // -----
138 // Control the incr/decr of buckets
139 // -----
140 assign bucket_down = decr_valid_i ? decr_ind : '0;
141
142 always_comb begin : proc_bucket_control
143     case ({incr_valid_i, decr_valid_i})
144         2'b00 : bucket_en = '0;
145         2'b10 : bucket_en = incr_ind;
146         2'b01 : bucket_en = decr_ind;
147         2'b11 : bucket_en = incr_ind ^ decr_ind;
148         default: bucket_en = '0; // unreachable
149     endcase
150 end
151
152 // -----
153 // Counters
154 // -----
155 for (genvar i = 0; i < NoCounters; i++) begin : gen_buckets
156     logic [BucketWidth-1:0] bucket_content;
157     cc_counter #(
158         .Width( BucketWidth )
159     ) i_bucket (
160         .clk_i ( clk_i ),
161         .rst_ni ( rst_ni ),
162         .clr_i ( clr_i ),
163         .en_i ( bucket_en[i] ),
164         .load_i ( '0 ),
165         .down_i ( bucket_down[i] ),
166         .d_i ( '0 ),
167         .q_o ( bucket_content ),
168         .overflow_o ( bucket_overflow[i] )
169     );
170     assign bucket_full[i] = bucket_overflow[i] | (&buck-
et_content);
171     assign bucket_occupied[i] = |bucket_content;
172     assign bucket_empty[i] = ~bucket_occupied[i];

```

```

173 end
174
175 // -----
176 // Filter tot item counter
177 // -----
178 assign cnt_en = incr_valid_i ^ decr_valid_i;
179 assign cnt_down = decr_valid_i;
180 cc_counter #(
181     .Width ( HashWidth )
182 ) i_tot_count (
183     .clk_i      ( clk_i      ),
184     .rst_ni     ( rst_ni     ),
185     .clr_i      ( clr_i      ),
186     .en_i       ( cnt_en     ),
187     .load_i     ( '0         ),
188     .down_i     ( cnt_down   ),
189     .d_i        ( '0         ),
190     .q_o        ( filter_usage_o ),
191     .overflow_o ( cnt_overflow )
192 );
193
194 // -----
195 // Filter Output Flags
196 // -----
197 assign filter_full_o = |bucket_full;
198 assign filter_empty_o = &bucket_empty;
199 assign filter_error_o = |bucket_overflow | cnt_overflow;
200 endmodule
201
202 // gives out the or 'onehots' of all hash functions
203 module cc_hash_block #(
204     parameter int unsigned NoHashes
205     =
32'd3,
206     parameter int unsigned InpWidth
207     =
32'd11,
208     parameter int unsigned HashWidth
209     =
32'd5,
210     parameter int unsigned NoRounds
211     =
32'd1,
212     parameter cc_pkg::cb_seed_t [NoHashes-1:0] Seeds =
cc_pkg::CbEgSeeds
213 ) (
214     input logic [InpWidth-1:0] data_i,
215     output logic [2*HashWidth-1:0] indicator_o
216 );
217
218 logic [NoHashes-1:0][2*HashWidth-1:0] hashes;
219
220 for (genvar i = 0; i < NoHashes; i++) begin : gen_hashes
221     cc_sub_per_hash #(
222         .InpWidth ( InpWidth ),
223         .HashWidth ( HashWidth ),
224         .NoRounds ( NoRounds ),
225         .PermuteKey ( Seeds[i].PermuteSeed ),
226         .XorKey ( Seeds[i].XorSeed )
227     ) i_hash (
228         .data_i ( data_i ),
229         .hash_o ( ), // not used, because we
want the onehot
230         .hash_onehot_o ( hashes[i] )
231     );
232 end
233
234 // output assignment
235 always_comb begin : proc_hash_or
236     indicator_o = '0;
237     for (int unsigned j = 0; j < NoHashes; j++) begin
238         indicator_o = indicator_o | hashes[j];
239     end
240 end
241
242 `ifndef COMMON_CELLS_ASSERTS_OFF
243 `ifndef SYNTHESIS
244 // assertions
245 initial begin
246     `ASSUME_I(hash_conf, InpWidth > HashWidth,
247 $formatf("%m:\nA Hash Function reduces the width of the
input>\nInpWidth: %s\nOUT_WIDTH: %s",
248     InpWidth, HashWidth))
249 end
250 `endif
251 `endif
252 endmodule

```

8. pulp-platform_common_cells/src/cc_cdc_2phase.sv (206 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials

```

```

distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13
14 /// A two-phase clock domain crossing.
15 ///
16 /// # Reset Behavior!!
17 ///
18 /// This module must not be used if warm reset capabilty is a
requirement. The
19 /// only exception is if you consistently use a reset control-
ler that sequences
20 /// the resets while gating both clock domains (be very care-
ful if you follow
21 /// this strategy!). If you need warm reset/clear/flush capab-
ilities, use (AND
22 /// CAREFULLY READ THE DESCRIPTION) the
cc_cdc_2phase_clearable module.
23 ///
24 /// After this disclaimer, here is how you connect the
src_rst_ni and the
25 /// dst_rst_ni of this module for power-on-reset (POR). The
src_rst_ni and
26 /// dst_rst_ni signal must be asserted SIMULTANEOUSLY (i.e.
asynchronous
27 /// assertion). Otherwise, spurious transactions could occur
in the domain
28 /// where the reset arrives later than the other. The de-as-
sertion of both reset
29 /// must be synchronized to their respective clock domain
(i.e. src_rst_ni must
30 /// be deasserted synchronously to the src_clk_i and
dst_rst_ni must be
31 /// deasserted synchronously to dst_clk_i.) You can use the
cc_rstgen cell in the
32 /// common_cells library to achieve this (synchronization of
only the
33 /// de-assertion). However, be careful about reset domain
crossings; If you
34 /// reset both domain asynchronously in their entirety (i.e.
POR) you are fine.
35 /// However, if you use this strategy for warm resets (some
parts of the circuit
36 /// are not reset) you might introduce metastability in this
separate
37 /// reset-domain when you assert the reset (the deassertion
synchronizer doen't
38 /// help here).
39 ///
40 /// CONSTRAINT: Requires max_delay of min_period(src_clk_i,
dst_clk_i) through
41 /// the paths async_req, async_ack, async_data.
42 `include "common_cells/registers.svh"
43
44 module cc_cdc_2phase #(
45     parameter type data_t = logic,
46     parameter int unsigned SyncStages = 2
47 ) (
48     input logic src_rst_ni,
49     input logic src_clk_i,
50     input data_t src_data_i,
51     input logic src_valid_i,
52     output logic src_ready_o,
53
54     input logic dst_rst_ni,
55     input logic dst_clk_i,
56     output data_t dst_data_o,
57     output logic dst_valid_o,
58     input logic dst_ready_i
59 );
60
61 // Asynchronous handshake signals.
62 (* dont_touch = "true" *) logic async_req;
63 (* dont_touch = "true" *) logic async_ack;
64 (* dont_touch = "true" *) data_t async_data;
65
66 if (SyncStages < 2) begin : gen_sync_stage_assertion
67     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
68 end
69
70 // The sender in the source domain.
71 cc_cdc_2phase_src #(
72     .data_t ( data_t ),
73     .SyncStages ( SyncStages )
74 ) i_src (
75     .rst_ni ( src_rst_ni ),
76     .clk_i ( src_clk_i ),
77     .data_i ( src_data_i ),
78     .valid_i ( src_valid_i ),
79     .ready_o ( src_ready_o ),
80     .async_req_o ( async_req ),
81     .async_ack_i ( async_ack ),
82     .async_data_o ( async_data )

```

```

83 );
84 // The receiver in the destination domain.
85 cc_cdc_2phase_dst #(
86   .data_t ( data_t ),
87   .SyncStages ( SyncStages )
88 ) i_dst (
89   .rst_ni ( dst_rst_ni ),
90   .clk_i ( dst_clk_i ),
91   .data_o ( dst_data_o ),
92   .valid_o ( dst_valid_o ),
93   .ready_i ( dst_ready_i ),
94   .async_req_i ( async_req ),
95   .async_ack_o ( async_ack ),
96   .async_data_i ( async_data )
97 );
98 endmodule
99
100 // Half of the two-phase clock domain crossing located in the
101 source domain.
102
103 module cc_cdc_2phase_src #(
104   parameter type data_t = logic,
105   parameter int unsigned SyncStages = 2
106 ) (
107   input logic rst_ni,
108   input logic clk_i,
109   input data_t data_i,
110   input logic valid_i,
111   output logic ready_o,
112   output logic async_req_o,
113   input logic async_ack_i,
114   output data_t async_data_o
115 );
116
117 (* dont_touch = "true" *)
118 logic req_src_q, ack_synced;
119 (* dont_touch = "true" *)
120 data_t data_src_q;
121 logic src_accept;
122
123 assign src_accept = valid_i && ready_o;
124
125 if (SyncStages < 2) begin : gen_sync_stage_assertion
126
127   $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
128   end
129
130 // Synchronize the async ACK.
131 tc_sync #(
132   .Stages ( SyncStages )
133 ) i_sync (
134   .clk_i,
135   .rst_ni,
136   .serial_i ( async_ack_i ),
137   .serial_o ( ack_synced )
138 );
139
140 // The req_src and data_src registers change when a new data
141 item is accepted.
142 `FFL(req_src_q, ~req_src_q, src_accept, 1'b0, clk_i,
rst_ni)
143
144 // Output assignments.
145 assign ready_o = (req_src_q == ack_synced);
146 assign async_req_o = req_src_q;
147 assign async_data_o = data_src_q;
148
149 endmodule
150
151 // Half of the two-phase clock domain crossing located in the
152 destination
153 domain.
154 module cc_cdc_2phase_dst #(
155   parameter type data_t = logic,
156   parameter int unsigned SyncStages = 2
157 ) (
158   input logic rst_ni,
159   input logic clk_i,
160   output data_t data_o,
161   output logic valid_o,
162   input logic ready_i,
163   input logic async_req_i,
164   output logic async_ack_o,
165   input data_t async_data_i
166 );
167
168 (* dont_touch = "true" *)
169 logic req_synced, req_synced_q1, ack_dst_q;
170 (* dont_touch = "true" *)
171 data_t data_dst_q;
172 logic dst_accept, dst_data_load;

```

```

173
174 assign dst_accept = valid_o && ready_i;
175 assign dst_data_load = req_synced != req_synced_q1 && !val-
id_o;
176
177 if (SyncStages < 2) begin : gen_sync_stage_assertion
178
179   $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
180   end
181
182 // Synchronize the async request.
183 tc_sync #(
184   .Stages ( SyncStages )
185 ) i_sync (
186   .clk_i,
187   .rst_ni,
188   .serial_i ( async_req_i ),
189   .serial_o ( req_synced )
190 );
191
192 // The ack_dst register changes when a new data item is ac-
cepted.
193 `FFL(ack_dst_q, ~ack_dst_q, dst_accept, 1'b0, clk_i, rst_ni)
194
195 // The data_dst register changes when a new data item is
presented. This is
196 // indicated by the async_req line changing levels.
197 `FFL(data_dst_q, async_data_i, dst_data_load, data_t('0),
clk_i, rst_ni)
198
199 // Delayed version of the synchronized request for trans-
ition detection.
200 `FF(req_synced_q1, req_synced, 1'b0, clk_i, rst_ni)
201
202 // Output assignments.
203 assign valid_o = (ack_dst_q != req_synced_q1);
204 assign data_o = data_dst_q;
205 assign async_ack_o = ack_dst_q;
206 endmodule

```

9. pulp-platform_common_cells/src/cc_cdc_2phase_clearable.sv (485 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
4 pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 file except in
8 // compliance with the License. You may obtain a copy of the
9 License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 applicable law
12 // or agreed to in writing, software, hardware and materials
13 distributed under
14 // this license is distributed on an "AS IS" BASIS, WITHOUT
15 WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 License for the
18 // specific language governing permissions and limitations un-
19 der the License.
20
21 // Authors:
22 // - Fabian Schuiki <fschuiki@iis.ee.ethz.ch> (original CDC)
23 // - Manuel Eggimann <meggimann@iis.ee.ethz.ch> (clearability
24 feature)
25 // - Philippe Sauter <phsauter@iis.ee.ethz.ch> (source/destin-
26 ation split)
27
28 // Description: Clearable Two-Phase CDC
29 // Transfer payloads with a two-phase handshake while coordin-
30 ating synchronous
31 // and optional asynchronous one-sided clears across source
32 and destination.
33
34 // A two-phase clock domain crossing.
35
36 // CONSTRAINT: Requires max_delay of min_period(src_clk_i,
37 dst_clk_i) through
38 // all asynchronous paths between the domains. The payload
39 CDC uses
40
41 // async_payload_req, async_payload_ack, and
42 async_payload_data for the actual
43 // data handshake. The clear sequencer CDCs use
44 async_reset_src2dst_req,
45 // async_reset_dst2src_ack, async_reset_src2dst_next_phase,
46 // async_reset_dst2src_req, async_reset_src2dst_ack, and
47 // async_reset_dst2src_next_phase to handshake the clearing
48 state machines.
49
50 //
51 //
52 // Reset Behavior:
53
54 // In contrast to the cc_cdc_2phase version without clear
55 signal, this module
56 // supports one-sided warm resets (asynchronously and syn-
57 chronously). The way

```

```

36 /// this is implemented is described in more detail in the
cc_cdc_reset_ctrlr
37 /// module. To summarize a synchronous clear request i.e. src/
dst_clear_i will
38 /// cause the respective other clock domain to reset as well
without introducing
39 /// any spurious transactions. This is accomplished by an in-
ternal module
40 /// (cc_cdc_reset_ctrlr) that starts a reset sequence on both
sides of the CDC in
41 /// lock-step that first isolates the CDC from the outside
world and then resets
42 /// it. The reset sequencer provides the following behavior:
43 /// 1. There are no spurious invalid or duplicated transac-
tions regardless how

44 /// the individual sides are reset (can also happen roughly
simultaneously)
45 /// 2. The CDC becomes unready at the src side in the next
cycle after
46 /// synchronous reset request until the reset sequence is
completed. A currently
47 /// pending transactions might still complete (if the dst
accepts at the
48 /// exact time the reset is request on the src die).
49 /// 3. During the reset sequence the dst might withdraw the
valid signal. This
50 /// might violate higher level protocols. If you need this
feature you would
51 /// have to path the existing implementation to wait with
the isolate_ack
52 /// assertion until all open handshakes were acknowledged.

53 /// 4. If the parameter ClearOnAsyncReset is enabled, the same
behavior as
54 /// above is also valid for asynchronous resets on either
side. However, this
55 /// increases the minimum number of synchronization stages
(SyncStages
56 /// parameter) from 2 to 3 (read the cc_cdc_reset_ctrlr
header to figure out
57 /// why).
58 ///
59 ///
60 `include "common_cells/registers.svh"
61 `include "common_cells/assertions.svh"
62
63 module cc_cdc_2phase_clearable #(
64     parameter type data_t = logic,
65     parameter int unsigned SyncStages = 3,
66     parameter bit ClearOnAsyncReset = 1
67 )()
68     input logic src_rst_ni,
69     input logic src_clk_i,
70     input logic src_clear_i,
71     output logic src_clear_pending_o,
72     input data_t src_data_i,
73     input logic src_valid_i,
74     output logic src_ready_o,
75
76     input logic dst_rst_ni,
77     input logic dst_clk_i,
78     input logic dst_clear_i,
79     output logic dst_clear_pending_o,
80     output data_t dst_data_o,
81     output logic dst_valid_o,
82     input logic dst_ready_i
83 );

84 // Asynchronous payload handshake signals between the source
and destination.
85 (* dont_touch = "true" *) logic async_payload_req;
86 (* dont_touch = "true" *) logic async_payload_ack;
87 (* dont_touch = "true" *) data_t async_payload_data;
88
89 // Asynchronous clear-controller phase handshakes between
the domains.
90 (* dont_touch = "true" *) logic async_reset_src2dst_req;
91 (* dont_touch = "true" *) logic async_reset_dst2src_ack;
92 (* dont_touch = "true" *) cc_pkg::cdc_clear_seq_phase_e
async_reset_src2dst_next_phase;
93 (* dont_touch = "true" *) logic async_reset_dst2src_req;
94 (* dont_touch = "true" *) logic async_reset_src2dst_ack;
95 (* dont_touch = "true" *) cc_pkg::cdc_clear_seq_phase_e
async_reset_dst2src_next_phase;
96
97 // Clear/isolate contract:
98 // - src/dst_clear_pending_o are the local isolate requests.
99 // - local isolate and clear acknowledgements are one-cycle
delayed copies of
100 // the corresponding local requests.
101 // - isolate gates the external valid/ready interface on
both sides.

102 // - transactions accepted before or during a clear sequence
may complete or
103 // be dropped, but fresh post-clear traffic must not be
duplicated,
104 // reordered, or corrupted.
105
106 if (ClearOnAsyncReset) begin : gen_elaboration_assertion

```

```

107     if (SyncStages < 3)
108         $error("The clearable 2-phase CDC with async reset",
109             "synchronization requires at least 3 synchronizer
stages for the FIFO.");
110     end else begin : gen_elaboration_assertion
111         if (SyncStages < 2) begin : gen_elaboration_assertion
112             $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
113         end
114     end
115
116 // The source side owns the payload CDC half, local clear/
isolate
117 // acknowledgements, and the local reset-controller half.
118 cc_cdc_2phase_clearable_src #(
119     .data_t (data_t),
120     .SyncStages (SyncStages),
121     .ClearOnAsyncReset (ClearOnAsyncReset)
122 ) i_src (
123     .src_rst_ni
124     .src_clk_i
125     .src_clear_i
126     .src_clear_pending_o
127     .src_data_i
128     .src_valid_i (src_val-
id_i),
129     .src_ready_o
130     (* async *) .async_payload_req_o (async_payload_req),
131     (* async *) .async_payload_ack_i (async_payload_ack),
132     (* async *) .async_payload_data_o (async_payload_data),
133     (* async *) .async_reset_next_phase_o
134     (* async *) .async_reset_req_o
135     (* async *) .async_reset_ack_i
136     (* async *) .async_reset_dst2src_ack
137     (* async *) .async_reset_next_phase_i
138     (* async *) .async_reset_req_i
139     (* async *) .async_reset_ack_o
140 );
141
142 // The destination side owns the payload CDC half, local
clear/isolate acknowledgements, and the local reset-con-
troller half.
143 cc_cdc_2phase_clearable_dst #(
144     .data_t (data_t),
145     .SyncStages (SyncStages),
146     .ClearOnAsyncReset (ClearOnAsyncReset)
147 ) i_dst (
148     .dst_rst_ni
149     .dst_clk_i
150     .dst_clear_i
151     .dst_clear_pending_o
152     .dst_data_o
153     .dst_valid_o (dst_val-
id_o),
154     .dst_ready_i
155     (* async *) .async_payload_req_i (async_payload_req),
156     (* async *) .async_payload_ack_o (async_payload_ack),
157     (* async *) .async_payload_data_i (async_payload_data),
158     (* async *) .async_reset_next_phase_o
159     (* async *) .async_reset_req_o
160     (* async *) .async_reset_ack_i
161     (* async *) .async_reset_next_phase_i
162     (* async *) .async_reset_req_i
163     (* async *) .async_reset_ack_o
164 );
165
166 `ifndef COMMON_CELLS_ASSERTS_OFF
167     `ASSERT(no_valid_i_during_clear_i, src_clear_i |>= src_val-
id_i, src_clk_i, !src_rst_ni)
168 `endif
169
170 endmodule
171
172
173
174
175

```

```

176
177 /// Destination side for the clearable two-phase CDC.
178 module cc_cdc_2phase_clearable_dst #(
179     parameter type data_t = logic,
180     parameter int unsigned SyncStages = 2,
181     parameter bit ClearOnAsyncReset = 1
182 ) (
183     input logic dst_rst_ni,
184     input logic dst_clk_i,
185     input logic dst_clear_i,
186     output logic dst_clear_pending_o,
187     output data_t dst_data_o,
188     output logic dst_valid_o,
189     input logic dst_ready_i,
190     input logic async_payload_req_i,
191     output logic async_payload_ack_o,
192     input data_t async_payload_data_i,
193     output cc_pkg::cdc_clear_seq_phase_e
194     async_reset_next_phase_o,
195     output logic async_reset_req_o,
196     input logic async_reset_ack_i,
197     input cc_pkg::cdc_clear_seq_phase_e
198     async_reset_next_phase_i,
199 );
200
201 logic dst_clear_req;
202 logic dst_clear_ack_q;
203 logic dst_isolate_req;
204 logic dst_isolate_ack_q;
205 logic dst_valid;
206
207 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
208     if (SyncStages < 3)
209         $error("The clearable 2-phase CDC with async reset",
210
211             "synchronization requires at least 3 synchronizer
212             stages for the FIFO.");
213     end else begin : gen_elaboration_assertion
214         if (SyncStages < 2) begin : gen_sync_stage_assertion
215             $error("A minimum of 2 synchronizer stages is required
216             for proper functionality.");
217         end
218     end
219
220 // Keep the clear-control path one synchronizer stage ahead
221 // of the payload CDC
222 // when async reset propagation is enabled, but never below
223 // two stages.
224
225 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
226 SyncStages - 1 : 2;
227
228 cc_cdc_reset_ctrlr_half #(
229     .SyncStages ( ResetSyncStages ),
230     .ClearOnAsyncReset ( ClearOnAsyncReset )
231 ) i_cdc_reset_ctrlr_half (
232     .clk_i ( dst_clk_i ),
233     .rst_ni ( dst_rst_ni ),
234     .clear_i ( dst_clear_i ),
235     .clear_o ( dst_clear_req ),
236     .clear_ack_i ( dst_clear_ack_q ),
237     .isolate_o ( dst_isolate_req ),
238     .isolate_ack_i ( dst_isolate_ack_q ),
239     (* async *) .async_next_phase_o
240     ( async_reset_next_phase_o ),
241     (* async *) .async_req_o
242     ( async_reset_req_o ),
243     (* async *) .async_ack_i
244     ( async_reset_ack_i ),
245     (* async *) .async_next_phase_i
246     ( async_reset_next_phase_i ),
247     (* async *) .async_req_i
248     ( async_reset_req_i ),
249     (* async *) .async_ack_o
250     ( async_reset_ack_o ),
251 );
252
253 cc_cdc_2phase_dst_clearable #(
254     .data_t ( data_t ),
255     .SyncStages ( SyncStages )
256 ) i_dst (
257     .rst_ni ( dst_rst_ni ),
258     .clk_i ( dst_clk_i ),
259     .clear_i ( dst_clear_req ),
260     .data_o ( dst_data_o ),
261     .valid_o ( dst_valid ),
262     .ready_i ( dst_ready_i & !dst_isolate_req ),
263     (* async *) .async_req_i ( async_payload_req_i ),
264     (* async *) .async_ack_o ( async_payload_ack_o ),
265     (* async *) .async_data_i ( async_payload_data_i )
266 );
267
268 assign dst_valid_o = dst_valid & !dst_isolate_req;
269
270 // Isolation and clear are acknowledged one cycle after the
271 local request.
272 `FF(dst_isolate_ack_q, dst_isolate_req, 1'b0, dst_clk_i,
273 dst_rst_ni)
274 `FF(dst_clear_ack_q, dst_clear_req, 1'b0, dst_clk_i,

```

```

dst_rst_ni)
260
261 assign dst_clear_pending_o = dst_isolate_req;
262
263 endmodule
264
265
266 /// Source side for the clearable two-phase CDC.
267 module cc_cdc_2phase_clearable_src #(
268     parameter type data_t = logic,
269     parameter int unsigned SyncStages = 2,
270     parameter bit ClearOnAsyncReset = 1
271 ) (
272     input logic src_rst_ni,
273     input logic src_clk_i,
274     input logic src_clear_i,
275     output logic src_clear_pending_o,
276     input data_t src_data_i,
277     input logic src_valid_i,
278     output logic src_ready_o,
279     output logic async_payload_req_o,
280     input logic async_payload_ack_i,
281     output data_t async_payload_data_o,
282     output cc_pkg::cdc_clear_seq_phase_e
283     async_reset_next_phase_o,
284     output logic async_reset_req_o,
285     input logic async_reset_ack_i,
286     input cc_pkg::cdc_clear_seq_phase_e
287     async_reset_next_phase_i,
288 );
289
290 logic src_clear_req;
291 logic src_clear_ack_q;
292 logic src_ready;
293 logic src_isolate_req;
294 logic src_isolate_ack_q;
295
296 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
297     if (SyncStages < 3)
298         $error("The clearable 2-phase CDC with async reset",
299
300             "synchronization requires at least 3 synchronizer
301             stages for the FIFO.");
302     end else begin : gen_elaboration_assertion
303         if (SyncStages < 2) begin : gen_sync_stage_assertion
304             $error("A minimum of 2 synchronizer stages is required
305             for proper functionality.");
306         end
307     end
308
309 // Keep the clear-control path one synchronizer stage ahead
310 // of the payload CDC
311 // when async reset propagation is enabled, but never below
312 // two stages.
313
314 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
315 SyncStages - 1 : 2;
316
317 cc_cdc_reset_ctrlr_half #(
318     .SyncStages ( ResetSyncStages ),
319     .ClearOnAsyncReset ( ClearOnAsyncReset )
320 ) i_cdc_reset_ctrlr_half (
321     .clk_i ( src_clk_i ),
322     .rst_ni ( src_rst_ni ),
323     .clear_i ( src_clear_i ),
324     .clear_o ( src_clear_req ),
325     .clear_ack_i ( src_clear_ack_q ),
326     .isolate_o ( src_isolate_req ),
327     .isolate_ack_i ( src_isolate_ack_q ),
328     (* async *) .async_next_phase_o
329     ( async_reset_next_phase_o ),
330     (* async *) .async_req_o
331     ( async_reset_req_o ),
332     (* async *) .async_ack_i
333     ( async_reset_ack_i ),
334     (* async *) .async_next_phase_i
335     ( async_reset_next_phase_i ),
336     (* async *) .async_req_i
337     ( async_reset_req_i ),
338     (* async *) .async_ack_o
339     ( async_reset_ack_o ),
340 );
341
342 cc_cdc_2phase_src_clearable #(
343     .data_t ( data_t ),
344     .SyncStages ( SyncStages )
345 ) i_src (
346     .rst_ni ( src_rst_ni ),
347     .clk_i ( src_clk_i ),
348     .clear_i ( src_clear_req ),
349     .data_o ( src_data_i ),
350     .valid_i ( src_valid_i & !src_isolate_req ),
351     .ready_o ( src_ready ),
352     (* async *) .async_req_o ( async_payload_req_o ),
353     (* async *) .async_ack_i ( async_payload_ack_i ),
354     (* async *) .async_data_o ( async_payload_data_o )
355 );
356
357 assign src_ready_o = src_ready & !src_isolate_req;

```



```

345
346 // Isolation and clear are acknowledged one cycle after the
local request.
347 `FF(src_isolate_ack_q, src_isolate_req, 1'b0, src_clk_i,
src_rst_ni)
348 `FF(src_clear_ack_q, src_clear_req, 1'b0, src_clk_i,
src_rst_ni)
349
350 assign src_clear_pending_o = src_isolate_req; // The isolate
signal stays
351 // asserted
during the whole
352 // clear se-
quence.
353
354 endmodule
355
356
357 /// Half of the two-phase clock domain crossing located in the
source domain.
358 module cc_cdc_2phase_src_clearable #(
359     parameter type data_t = logic,
360     parameter int unsigned SyncStages = 2
361 ) (
362     input logic rst_ni,
363     input logic clk_i,
364     input logic clear_i,
365     input data_t data_i,
366     input logic valid_i,
367     output logic ready_o,
368     output logic async_req_o,
369     input logic async_ack_i,
370     output data_t async_data_o
371 );
372
373 (* dont_touch = "true" *)
374 logic req_src_d, req_src_q, ack_synced;
375 (* dont_touch = "true" *)
376 data_t data_src_d, data_src_q;
377
378 // Synchronize the async ACK
379 tc_sync #(
380     .Stages(SyncStages)
381 ) i_sync(
382     .clk_i,
383     .rst_ni,
384     .serial_i( async_ack_i ),
385     .serial_o( ack_synced )
386 );
387
388 // If we receive the clear signal clear the content of the
request flip-flop
389 // and the data register
390 always_comb begin
391     data_src_d = data_src_q;
392     req_src_d = req_src_q;
393     if (clear_i) begin
394         req_src_d = 1'b0;
395         // The req_src and data_src registers change when a new
data item is accepted.
396     end else if (valid_i && ready_o) begin
397         req_src_d = ~req_src_q;
398         data_src_d = data_i;
399     end
400 end
401
402 `FFNR(data_src_q, data_src_d, clk_i)
403
404 `FF(req_src_q, req_src_d, 1'b0, clk_i, rst_ni)
405
406 // Output assignments.
407 assign ready_o = (req_src_q == ack_synced);
408 assign async_req_o = req_src_q;
409 assign async_data_o = data_src_q;
410
411 // Assertions
412 `ifndef COMMON_CELLS_ASSERTS_OFF
413 `ASSUME(no_clear_and_request, clear_i |> ~valid_i, clk_i, !
rst_ni,
414     "No request allowed while clear_i is asserted.")
415 `endif
416
417 endmodule
418
419
420 /// Half of the two-phase clock domain crossing located in the
destination
421 /// domain.
422 module cc_cdc_2phase_dst_clearable #(
423     parameter type data_t = logic,
424     parameter int unsigned SyncStages = 2
425 ) (
426     input logic rst_ni,
427     input logic clk_i,
428     input logic clear_i,
429     output data_t data_o,
430     output logic valid_o,
431     input logic ready_i,

```

```

432     input logic async_req_i,
433     output logic async_ack_o,
434     input data_t async_data_i
435 );
436
437 (* dont_touch = "true" *)
438 (* async_req = "true" *)
439 logic ack_dst_d, ack_dst_q, req_synced, req_synced_q1;
440 (* dont_touch = "true" *)
441 data_t data_dst_d, data_dst_q;
442
443 //Synchronize the request
444 tc_sync #(
445     .Stages(SyncStages)
446 ) i_sync(
447     .clk_i,
448     .rst_ni,
449     .serial_i( async_req_i ),
450     .serial_o( req_synced )
451 );
452
453 // The ack_dst register changes when a new data item is ac-
cepted.
454 always_comb begin
455     ack_dst_d = ack_dst_q;
456     if (clear_i) begin
457         ack_dst_d = 1'b0;
458     end else if (valid_o && ready_i) begin
459         ack_dst_d = ~ack_dst_q;
460     end
461 end
462
463 // The data_dst register samples when a new data item is
presented. This is
464 // indicated by a transition in the req_synced line.
465 always_comb begin
466     data_dst_d = data_dst_q;
467     if (req_synced != req_synced_q1 && !valid_o) begin
468         data_dst_d = async_data_i;
469     end
470 end
471
472 `FFNR(data_dst_q, data_dst_d, clk_i)
473
474 `FF(ack_dst_q, ack_dst_d, 1'b0, clk_i, rst_ni)
475 // The req_synced_q1 is the delayed version of the synchron-
ized req_synced
476 // used to detect transitions in the request.
477 `FF(req_synced_q1, req_synced, 1'b0, clk_i, rst_ni)
478
479 // Output assignments.
480 assign valid_o = (ack_dst_q != req_synced_q1);
481 assign data_o = data_dst_q;
482 assign async_ack_o = ack_dst_q;
483
484 endmodule
485

```

10. pulp-platform_common_cells/src/cc_cdc_4phase.sv (338 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Manuel Eggimann <meggimann@iis.ee.ethz.ch>
13
14 /// A 4-phase clock domain crossing. While this is less effi-
cient than a 2-phase
15 /// CDC, it doesn't suffer from the same issues during one
sided resets since
16 /// the IDLE state doesn't alternate with every transaction.
17 ///
18 /// Parameters: data_t - The type of the data to transmit
through the CDC.
19 ///
20 /// Decoupled - If decoupled is disabled, the 4phase cdc will
not consume the
21 /// src item until the handshake with the other side is com-
pleted. This
22 /// increases the latency of the first transaction but has no
effect on
23 /// throughput. However, critical paths might be slightly
longer. Use this mode

```

```

24 /// if you want to ensure that there are no in-flight transac-
tions within the
25 /// CDC.
26 ///
27 /// SendResetMsg - If send reset msg is enabled, the 4phase
cdc starts sending
28 /// the ResetMsg within its' asynchronous reset state. This
can be usefull if
29 /// we need to transmit a message to the other side of the CDC
immediately
30 /// during an async reset even if there is no clock available.
This mode is
31 /// required for proper functionality of the cc_cdc_re-
set_ctrlr module.
32 ///
33 /// SyncStages - Number of synchronization stages for
async_req and async_ack.
34 ///
35 /// CONSTRAINT: Requires max_delay of min_period(src_clk_i,
dst_clk_i) through
36 /// the paths async_req, async_ack, async_data.
37 `include "common_cells/registers.svh"
38
39 module cc_cdc_4phase #(
40     parameter type data_t = logic,
41     parameter bit Decoupled = 1'b1,
42     parameter bit SendResetMsg = 1'b0,
43     parameter data_t ResetMsg = data_t'('0),
44     parameter int unsigned SyncStages = 2
45 ) (
46     input logic src_rst_ni,
47     input logic src_clk_i,
48     input data_t src_data_i,
49     input logic src_valid_i,
50     output logic src_ready_o,
51
52     input logic dst_rst_ni,
53     input logic dst_clk_i,
54     output data_t dst_data_o,
55     output logic dst_valid_o,
56     input logic dst_ready_i
57 );
58
59 // Asynchronous handshake signals.
60 (* dont_touch = "true" *) logic async_req;
61 (* dont_touch = "true" *) logic async_ack;
62 (* dont_touch = "true" *) data_t async_data;
63
64 if (SyncStages < 2) begin : gen_sync_stage_assertion
65     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
66 end
67
68 // The sender in the source domain.
69 cc_cdc_4phase_src #(
70     .data_t(data_t),
71     .SyncStages(SyncStages),
72     .Decoupled(Decoupled),
73     .SendResetMsg(SendResetMsg),
74     .ResetMsg(ResetMsg)
75 ) i_src (
76     .rst_ni ( src_rst_ni ),
77     .clk_i ( src_clk_i ),
78     .data_i ( src_data_i ),
79     .valid_i ( src_valid_i ),
80     .ready_o ( src_ready_o ),
81     .async_req_o ( async_req ),
82     .async_ack_i ( async_ack ),
83     .async_data_o ( async_data )
84 );
85
86 // The receiver in the destination domain.
87 cc_cdc_4phase_dst #(
88     .data_t(data_t),
89     .SyncStages(SyncStages),
90     .Decoupled(Decoupled)
91 ) i_dst (
92     .rst_ni ( dst_rst_ni ),
93     .clk_i ( dst_clk_i ),
94     .data_o ( dst_data_o ),
95     .valid_o ( dst_valid_o ),
96     .ready_i ( dst_ready_i ),
97     .async_req_i ( async_req ),
98     .async_ack_o ( async_ack ),
99     .async_data_i ( async_data )
100 );
101 endmodule
102
103
104 /// Half of the 4-phase clock domain crossing located in the
source domain.
105 module cc_cdc_4phase_src #(
106     parameter type data_t = logic,
107     parameter int unsigned SyncStages = 2,
108     parameter bit Decoupled = 1'b1,
109     parameter bit SendResetMsg = 1'b0,
110     parameter data_t ResetMsg = data_t'('0)
111 ) (

```

```

112     input logic rst_ni,
113     input logic clk_i,
114     input data_t data_i,
115     input logic valid_i,
116     output logic ready_o,
117     output logic async_req_o,
118     input logic async_ack_i,
119     output data_t async_data_o
120 );
121
122 (* dont_touch = "true" *)
123 logic req_src_d, req_src_q;
124 (* dont_touch = "true" *)
125 data_t data_src_d, data_src_q;
126 (* dont_touch = "true" *)
127 logic ack_synced;
128
129 typedef enum logic[1:0] {IDLE, WAIT_ACK_ASSERT, WAIT_ACK_DE-
ASSERT} state_e;
130 state_e state_d, state_q;
131 localparam logic ReqSrcResetValue = SendResetMsg ? 1'b1 :
1'b0;
132 localparam data_t DataSrcResetValue = SendResetMsg ? Re-
setMsg : data_t'('0);
133
134 if (SyncStages < 2) begin : gen_sync_stage_assertion
135     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
136 end
137
138 // Synchronize the async ACK
139 tc_sync #(
140     .Stages(SyncStages)
141 ) i_sync (
142     .clk_i,
143     .rst_ni,
144     .serial_i( async_ack_i ),
145     .serial_o( ack_synced )
146 );
147
148 // Output logic derived from registered local state only.
149 always_comb begin
150     ready_o = 1'b0;
151     case (state_q)
152         IDLE: begin
153             ready_o = Decoupled;
154         end
155         WAIT_ACK_DEASSERT: begin
156             ready_o = !Decoupled && (ack_synced == 1'b0);
157         end
158         default: begin
159             ready_o = 1'b0;
160         end
161     endcase
162 end
163
164 // FSM for the 4-phase handshake; calculates next state from
inputs and local state.
165 always_comb begin
166     state_d = state_q;
167     req_src_d = 1'b0;
168     data_src_d = data_src_q;
169     case (state_q)
170         IDLE: begin
171             // Sample a new item when the valid signal is asser-
ted.
172             if (valid_i) begin
173                 data_src_d = data_i;
174                 req_src_d = 1'b1;
175                 state_d = WAIT_ACK_ASSERT;
176             end
177         end
178         WAIT_ACK_ASSERT: begin
179             req_src_d = 1'b1;
180             if (ack_synced == 1'b1) begin
181                 req_src_d = 1'b0;
182                 state_d = WAIT_ACK_DEASSERT;
183             end
184         end
185         WAIT_ACK_DEASSERT: begin
186             if (ack_synced == 1'b0) begin
187                 state_d = IDLE;
188             end
189         end
190         default: begin
191             state_d = IDLE;
192         end
193     endcase
194 end
195
196 `FF(state_q, state_d, IDLE, clk_i, rst_ni)
197
198 // Sample the data and the request signal to filter combina-
tional glitches
199 `FF(req_src_q, req_src_d, ReqSrcResetValue, clk_i,
rst_ni)
200
201 `FF(data_src_q, data_src_d, DataSrcResetValue, clk_i,
rst_ni)
201

```



```

202 // Async output assignments.
203 assign async_req_o = req_src_q;
204 assign async_data_o = data_src_q;
205
206 endmodule
207
208
209 /// Half of the 4-phase clock domain crossing located in the
destination
210 /// domain.
211 module cc_cdc_4phase_dst #(
212     parameter type data_t = logic,
213     parameter int unsigned SyncStages = 2,
214     parameter bit Decoupled = 1'b1
215 ) (
216     input logic rst_ni,
217     input logic clk_i,
218     output data_t data_o,
219     output logic valid_o,
220     input logic ready_i,
221     input logic async_req_i,
222     output logic async_ack_o,
223     input data_t async_data_i
224 );
225
226 (* dont_touch = "true" *)
227 logic ack_dst_d, ack_dst_q;
228 (* dont_touch = "true" *)
229 logic req_synced;
230
231 logic data_valid;
232
233 logic output_ready;
234
235
236 typedef enum logic[1:0] {IDLE, WAIT_DOWNSTREAM_ACK,
WAIT_REQ_DEASSERT} state_e;
237 state_e state_d, state_q;
238
239 if (SyncStages < 2) begin : gen_sync_stage_assertion
240     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
241 end
242
243 //Synchronize the request
244 tc_sync #(
245     .Stages(SyncStages)
246 ) i_sync(
247     .clk_i,
248     .rst_ni,
249     .serial_i( async_req_i ),
250     .serial_o( req_synced )
251 );
252
253 // Output logic derived from registered local state only.
254 always_comb begin
255     data_valid = 1'b0;
256     case (state_q)
257         IDLE: begin
258             data_valid = req_synced;
259         end
260         WAIT_DOWNSTREAM_ACK: begin
261             data_valid = 1'b1;
262         end
263         default: begin
264             data_valid = 1'b0;
265         end
266     endcase
267 end
268
269 // FSM for the 4-phase handshake; calculates next state from
inputs and local state.
270 always_comb begin
271     state_d = state_q;
272     ack_dst_d = 1'b0;
273
274     case (state_q)
275         IDLE: begin
276             // Sample the data upon a new request and transition
to the next state
277             if (req_synced == 1'b1) begin
278                 if (output_ready == 1'b1) begin
279                     state_d = WAIT_REQ_DEASSERT;
280                 end else begin
281                     state_d = WAIT_DOWNSTREAM_ACK;
282                 end
283             end
284         end
285
286         WAIT_DOWNSTREAM_ACK: begin
287             if (output_ready == 1'b1) begin
288                 state_d = WAIT_REQ_DEASSERT;
289                 ack_dst_d = 1'b1;
290             end
291         end
292
293         WAIT_REQ_DEASSERT: begin
294             ack_dst_d = 1'b1;
295             if (req_synced == 1'b0) begin

```

```

296         ack_dst_d = 1'b0;
297         state_d = IDLE;
298     end
299 end
300
301 default: begin
302     state_d = IDLE;
303 end
304 endcase
305 end
306
307 `FF(state_q, state_d, IDLE, clk_i, rst_ni)
308
309 // Filter glitches on ack signal before sending it through
the asynchronous channel
310 `FF(ack_dst_q, ack_dst_d, 1'b0, clk_i, rst_ni)
311
312 if (Decoupled) begin : gen_decoupled
313     // Decouple the output from the asynchronous data bus
without introducing
314     // additional latency by inserting a spill register
315     cc_spill_register #(
316         .data_t(data_t),
317         .Bypass(1'b0)
318     ) i_spill_register (
319         .clk_i,
320         .rst_ni,
321         .clr_i('0),
322         .valid_i(data_valid),
323         .ready_o(output_ready),
324         .data_i(async_data_i),
325         .valid_o,
326         .ready_i,
327         .data_o
328     );
329 end else begin : gen_not_decoupled
330     assign valid_o = data_valid;
331     assign output_ready = ready_i;
332     assign data_o = async_data_i;
333 end
334
335 // Output assignments.
336 assign async_ack_o = ack_dst_q;
337
338 endmodule

```

11. pulp-platform_common_cells/src/cc_cdc_fifo_2phase.sv (312 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13
14 `include "common_cells/registers.svh"
15 `include "common_cells/assertions.svh"
16
17 /// A clock domain crossing FIFO, using 2-phase hand shakes.
18 ///
19 /// This FIFO has its push and pop ports in two separate clock
domains. Its size
20 /// can only be powers of two, which is why its depth is given as
2**LogDepth.
21 /// LogDepth must be at least 1.
22 ///
23 /// # Reset Behavior!!
24 ///
25 /// This module must not be used if warm reset capabilty is a
requirement. The
26 /// only exception is if you consistently use a reset control-
ler that sequences
27 /// the resets while gating both clock domains (be very care-
ful if you follow
28 /// this strategy!).
29 ///
30 /// After this disclaimer, here is how you connect the
src_rst_ni and the
31 /// dst_rst_ni of this module for power-on-reset (POR). The
src_rst_ni and
32 /// dst_rst_ni signal must be asserted SIMULTANEOUSLY (i.e.
asynchronous

```

```

33 /// assertion). Otherwise, spurious transactions could occur
in the domain
34 /// where the reset arrives later than the other. The de-assertion
of both reset
35 /// must be synchronized to their respective clock domain
(i.e. src_rst_ni must
36 /// be deasserted synchronously to the src_clk_i and
dst_rst_ni must be
37 /// deasserted synchronously to dst_clk_i.) You can use the
cc_stgen cell in the
38 /// common_cells library to achieve this (synchronization of
only the
39 /// de-assertion). However, be careful about reset domain
crossings; If you
40 /// reset both domain asynchronously in their entirety (i.e.
POR) you are fine.
41 /// However, if you use this strategy for warm resets (some
parts of the circuit
42 /// are not reset) you might introduce metastability in this
separate
43 /// reset-domain when you assert the reset (the deassertion
synchronizer doesn't
44 /// help here).
45 ///
46 /// CONSTRAINT: Requires max_delay of min_period(src_clk_i,
dst_clk_i) through
47 /// async_wptr_req, async_wptr_ack, async_wptr_data,
async_rptr_req,
48 /// async_rptr_ack, async_rptr_data, and async_data. The
async_wptr_* and
49 /// async_rptr_* signals are the actual pointer handshakes,
while async_data is
50 /// the FIFO storage exposed from the source domain to the
destination domain.
51 module cc_cdc_fifo_2phase #(
52     /// The data type of the payload transported by the FIFO.
53     parameter type data_t = logic,
54     /// The FIFO's depth given as 2**LogDepth.
55     parameter int unsigned LogDepth = 3,
56     /// The number of synchronization registers to insert on the
async pointers.
57     parameter int unsigned SyncStages = 2
58 ) (
59     input logic src_rst_ni,
60     input logic src_clk_i,
61     input data_t src_data_i,
62     input logic src_valid_i,
63     output logic src_ready_o,
64
65     input logic dst_rst_ni,
66     input logic dst_clk_i,
67     output data_t dst_data_o,
68     output logic dst_valid_o,
69     input logic dst_ready_i
70 );
71
72 localparam int unsigned PtrWidth = LogDepth+1;
73 typedef logic [PtrWidth-1:0] pointer_t;
74
75 data_t [2**LogDepth-1:0] async_data;
76 logic async_wptr_req;
77 logic async_wptr_ack;
78 pointer_t async_wptr_data;
79 logic async_rptr_req;
80 logic async_rptr_ack;
81 pointer_t async_rptr_data;
82
83 cc_cdc_fifo_2phase_src #(
84     .data_t ( data_t ),
85     .LogDepth ( LogDepth ),
86     .SyncStages ( SyncStages )
87 ) i_src (
88     .src_rst_ni ( src_rst_ni ),
89     .src_clk_i ( src_clk_i ),
90     .src_data_i ( src_data_i ),
91     .src_valid_i ( src_valid_i ),
92     .src_ready_o ( src_ready_o ),
93
94     (* async *) .async_data_o ( async_data ),
95     (* async *) .async_wptr_req_o ( async_wptr_req ),
96     (* async *) .async_wptr_ack_i ( async_wptr_ack ),
97     (* async *) .async_wptr_data_o ( async_wptr_data ),
98     (* async *) .async_rptr_req_i ( async_rptr_req ),
99     (* async *) .async_rptr_ack_o ( async_rptr_ack ),
100     (* async *) .async_rptr_data_i ( async_rptr_data )
101 );
102
103 cc_cdc_fifo_2phase_dst #(
104     .data_t ( data_t ),
105     .LogDepth ( LogDepth ),
106     .SyncStages ( SyncStages )
107 ) i_dst (
108     .dst_rst_ni ( dst_rst_ni ),
109     .dst_clk_i ( dst_clk_i ),
110     .dst_data_o ( dst_data_o ),
111     .dst_valid_o ( dst_valid_o ),
112     .dst_ready_i ( dst_ready_i ),
113
114     (* async *) .async_data_i ( async_data ),
115     (* async *) .async_wptr_req_i ( async_wptr_req ),

```

```

116     (* async *) .async_wptr_ack_o ( async_wptr_ack ),
117     (* async *) .async_wptr_data_i ( async_wptr_data ),
118     (* async *) .async_rptr_req_o ( async_rptr_req ),
119     (* async *) .async_rptr_ack_i ( async_rptr_ack ),
120     (* async *) .async_rptr_data_o ( async_rptr_data )
121 );
122
123 // Check the invariants.
124 `ifndef COMMON_CELLS_ASSERTS_OFF
125 `ASSERT_INIT(log_depth_0, LogDepth > 0)
126 `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
127 `endif
128
129 endmodule
130
131
132 /// Source side for the 2-phase CDC FIFO.
133 module cc_cdc_fifo_2phase_src #(
134     parameter type data_t = logic,
135     parameter int unsigned LogDepth = 3,
136     parameter int unsigned SyncStages = 2
137 ) (
138     input logic src_rst_ni,
139     input logic src_clk_i,
140     input data_t src_data_i,
141     input logic src_valid_i,
142     output logic src_ready_o,
143
144     output data_t [2**LogDepth-1:0] async_data_o,
145     output logic async_wptr_req_o,
146     input logic async_wptr_ack_i,
147     output logic [LogDepth:0] async_wptr_data_o,
148     input logic async_rptr_req_i,
149     output logic async_rptr_ack_o,
150     input logic [LogDepth:0] async_rptr_data_i
151 );
152
153 localparam int unsigned PtrWidth = LogDepth+1;
154 typedef logic [PtrWidth-1:0] pointer_t;
155 typedef logic [LogDepth-1:0] index_t;
156
157 localparam pointer_t PtrFull = (1 << LogDepth);
158
159 index_t fifo_widx;
160 logic fifo_write;
161 data_t [2**LogDepth-1:0] fifo_data_q;
162
163 pointer_t src_wptr_q;
164 pointer_t src_rptr;
165
166 if (SyncStages < 2) begin : gen_sync_stage_assertion
167     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
168 end
169
170 assign fifo_widx = src_wptr_q[LogDepth-1:0];
171 assign fifo_write = src_valid_i && src_ready_o;
172 assign async_data_o = fifo_data_q;
173
174 for (genvar i = 0; i < 2**LogDepth; i++) begin : gen_word
175     always_ff @(posedge src_clk_i, negedge src_rst_ni) begin
176         if (!src_rst_ni) begin
177             fifo_data_q[i] <= data_t'('0);
178         end else if (fifo_write && (fifo_widx == i)) begin
179             fifo_data_q[i] <= src_data_i;
180         end
181     end
182 end
183
184 `FFL(src_wptr_q, src_wptr_q + 1, fifo_write, '0, src_clk_i,
src_rst_ni)
185
186 // The pointers into the FIFO are one bit wider than the ac-
tual address into
187 // the FIFO. This makes detecting critical states very
simple: if all but the
188 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
189 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
190 assign src_ready_o = ((src_wptr_q ^ src_rptr) != PtrFull);
191
192 cc_cdc_2phase_src #(
193     .data_t ( pointer_t ),
194     .SyncStages ( SyncStages )
195 ) i_cdc_wptr_src (
196     .rst_ni ( src_rst_ni ),
197     .clk_i ( src_clk_i ),
198     .data_i ( src_wptr_q ),
199     .valid_i ( '1'b1 ),
200     .ready_o ( ),
201     .async_req_o ( async_wptr_req_o ),
202     .async_ack_i ( async_wptr_ack_i ),
203     .async_data_o ( async_wptr_data_o )
204 );
205
206 cc_cdc_2phase_dst #(
207     .data_t ( pointer_t ),
208     .SyncStages ( SyncStages )
209 ) i_cdc_rptr_dst (

```

```

210 .rst_ni      ( src_rst_ni      ),
211 .clk_i       ( src_clk_i       ),
212 .data_o      ( src_rptr       ),
213 .valid_o     (                ),
214 .ready_i     ( 1'b1           ),
215 .async_req_i ( async_rptr_req_i ),
216 .async_ack_o ( async_rptr_ack_o ),
217 .async_data_i ( async_rptr_data_i )
218 );
219
220 // Check the invariants.
221 `ifndef COMMON_CELLS_ASSERTS_OFF
222 `ASSERT_INIT(log_depth_0, LogDepth > 0)
223 `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
224 `endif
225
226 endmodule
227
228
229 /// Destination side for the 2-phase CDC FIFO.
230 module cc_cdc_fifo_2phase_dst #(
231     parameter type data_t = logic,
232     parameter int unsigned LogDepth = 3,
233     parameter int unsigned SyncStages = 2
234 ) (
235     input  logic dst_rst_ni,
236     input  logic dst_clk_i,
237     output data_t dst_data_o,
238     output logic dst_valid_o,
239     input  logic dst_ready_i,
240
241     input  data_t [2**LogDepth-1:0] async_data_i,
242     input  logic async_wptr_req_i,
243     output logic async_wptr_ack_o,
244     input  logic [LogDepth:0] async_wptr_data_i,
245     output logic async_rptr_req_o,
246     input  logic async_rptr_ack_i,
247     output logic [LogDepth:0] async_rptr_data_o
248 );
249
250 localparam int unsigned PtrWidth = LogDepth+1;
251 typedef logic [PtrWidth-1:0] pointer_t;
252 typedef logic [LogDepth-1:0] index_t;
253
254 localparam pointer_t PtrEmpty = '0;
255
256 index_t fifo_ridx;
257 data_t fifo_rdata;
258
259 pointer_t dst_wptr;
260 pointer_t dst_rptr_q;
261
262 if (SyncStages < 2) begin : gen_sync_stage_assertion
263     $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
264 end
265
266 assign fifo_ridx = dst_rptr_q[LogDepth-1:0];
267 assign fifo_rdata = async_data_i[fifo_ridx];
268 assign dst_data_o = fifo_rdata;
269
270 `FLL(dst_rptr_q, dst_rptr_q + 1, dst_valid_o && dst_ready_i,
'0, dst_clk_i, dst_rst_ni)
271
272 // The pointers into the FIFO are one bit wider than the ac-
tual address into
273 // the FIFO. This makes detecting critical states very
simple: if all but the
274 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
275 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
276 assign dst_valid_o = ((dst_rptr_q ^ dst_wptr) != PtrEmpty);
277
278 cc_cdc_2phase_dst #(
279     .data_t      ( pointer_t ),
280     .SyncStages ( SyncStages )
281 ) i_cdc_wptr_dst (
282     .rst_ni      ( dst_rst_ni      ),
283     .clk_i       ( dst_clk_i       ),
284     .data_o      ( dst_wptr       ),
285     .valid_o     (                ),
286     .ready_i     ( 1'b1           ),
287     .async_req_i ( async_wptr_req_i ),
288     .async_ack_o ( async_wptr_ack_o ),
289     .async_data_i ( async_wptr_data_i )
290 );
291
292 cc_cdc_2phase_src #(
293     .data_t      ( pointer_t ),
294     .SyncStages ( SyncStages )
295 ) i_cdc_rptr_src (
296     .rst_ni      ( dst_rst_ni      ),
297     .clk_i       ( dst_clk_i       ),
298     .data_i      ( dst_rptr_q      ),
299     .valid_i     ( 1'b1           ),
300     .ready_o     (                ),
301     .async_req_o ( async_rptr_req_o ),
302     .async_ack_i ( async_rptr_ack_i ),

```

```

303     .async_data_o ( async_rptr_data_o )
304 );
305
306 // Check the invariants.
307 `ifndef COMMON_CELLS_ASSERTS_OFF
308 `ASSERT_INIT(log_depth_0, LogDepth > 0)
309 `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
310 `endif
311
312 endmodule

```

12. pulp-platform_common_cells/src/cc_cdc_fifo_gray.sv (447 lines)

```

1 // Copyright 2018-2019 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13 // Florian Zaruba <zarubaf@iis.ee.ethz.ch>
14 // Philippe Sauter <phsauter@iis.ee.ethz.ch> (source/destina-
tion split)
15
16 /// A clock domain crossing FIFO, using gray counters.
17 ///
18 /// # Architecture
19 ///
20 /// The design is split into two parts, each one being clocked
and reset
21 /// separately.
22 /// 1. The data to be transferred over the clock domain
boundary is
23 /// is stored in a FIFO. The corresponding write pointer is
managed
24 /// (incremented) in the source clock domain.
25 /// 2. The entire FIFO content is exposed over the
`async_data` port.
26 /// The destination clock domain increments its read point-
er
27 /// in its destination clock domain.
28 ///
29 /// Read and write pointers are then gray coded, communicated
30 /// and synchronized using a classic multi-stage FF synchron-
izer
31 /// in the other clock domain. The gray coding ensures that
only
32 /// one bit changes at each pointer increment, preventing the
33 /// synchronizer to accidentally latch an inconsistent state
34 /// on a multi-bit bus.
35 ///
36 /// The not full signal e.g. `src_ready_o` (on the sending
side)
37 /// is generated using the local write pointer and the pessim-
istic
38 /// read pointer from the destination clock domain (pessimist-
ic
39 /// because it is delayed at least two cycles because of the
synchronizer
40 /// stages). This prevents the FIFO from overflowing.
41 ///
42 /// The not empty signal e.g. `dst_valid_o` is generated using
43 /// the pessimistic write pointer and the local read pointer
in
44 /// the destination clock domain. This means the FIFO content
45 /// does not need to be synchronized as we are sure we are
reading
46 /// data which has been written at least two cycles earlier.
47 /// Furthermore, the read select logic into the FIFO is com-
pletely
48 /// clocked by the destination clock domain which avoids
49 /// inefficient data synchronization.
50 ///
51 /// The FIFO size must be powers of two, which is why its
depth is
52 /// given as 2**LogDepth. LogDepth must be at least 1.
53 ///
54 /// # Reset Behavior!!
55 ///
56 /// This module must not be used if warm reset capabilty is a
requirement. The
57 /// only exception is if you consistently use a reset control-
ler that sequences

```

```

58 /// the resets while gating both clock domains (be very care-
ful if you follow
59 /// this strategy!). If you need warm reset/clear/flush capab-
ilities, use (AND
60 /// CAREFULLY READ THE DESCRIPTION) the
cc_cdc_fifo_gray_clearable module.
61 ///
62 /// After this disclaimer, here is how you connect the
src_rst_ni and the
63 /// dst_rst_ni of this module for power-on-reset (POR). The
src_rst_ni and
64 /// dst_rst_ni signal must be asserted SIMULTANEOUSLY (i.e.
asynchronous
65 /// assertion). Otherwise, spurious transactions could occur
in the domain
66 /// where the reset arrives later than the other. The de-as-
sertion of both reset
67 /// must be synchronized to their respective clock domain
(i.e. src_rst_ni must
68 /// be deasserted synchronously to the src_clk_i and
dst_rst_ni must be
69 /// deasserted synchronously to dst_clk_i.) You can use the
cc_rstgen cell in the
70 /// common_cells library to achieve this (synchronization of
only the
71 /// de-assertion). However be carefull about reset domain
crossings; If you
72 /// reset both domain asynchronously in their entirety (i.e.
POR) you are fine.
73 /// However, if you use this strategy for warm resets (some
parts of the circuit
74 /// are not reset) you might introduce metastability in this
separate
75 /// reset-domain when you assert the reset (the deassertion
synchronizer doesn't
76 /// help here).
77 ///
78 /// # Constraints
79 ///
80 /// We need to make sure that the propagation delay of the
81 /// data, read and write pointer is bound to the minimum of
82 /// either the sending or receiving clock period to prevent
83 /// an inconsistent state to be latched (if for example the
one
84 /// bit of the read/write pointer have an excessive delay).
85 /// Furthermore, we should deactivate setup and hold checks on
86 /// the asynchronous signals.
87 ///
88 /// ...
89 /// set_ungroup [get_designs cc_cdc_fifo_gray*] false
90 /// set_boundary_optimization [get_designs cc_cdc_fifo_gray*]
false
91 /// set_max_delay min(T_src, T_dst) \
92 /// -through [get_pins -hierarchical -filter async] \
93 /// -through [get_pins -hierarchical -filter async]
94 /// set_false_path -hold \
95 /// -through [get_pins -hierarchical -filter async] \
96 /// -through [get_pins -hierarchical -filter async]
97 /// ...
98
99 `include "common_cells/registers.svh"
100 `include "common_cells/assertions.svh"
101
102 (* no_ungroup *)
103 (* no_boundary_optimization *)
104 module cc_cdc_fifo_gray #(
105     /// The width of the default logic type.
106     parameter int unsigned Width = 1,
107     /// The data type of the payload transported by the FIFO.
108     parameter type data_t = logic [Width-1:0],
109     /// The FIFO's depth given as 2**LogDepth.
110     parameter int unsigned LogDepth = 3,
111     /// The number of synchronization registers to insert on the
async pointers.
112     parameter int unsigned SyncStages = 2
113 ) (
114     input logic src_rst_ni,
115     input logic src_clk_i,
116     input data_t src_data_i,
117     input logic src_valid_i,
118     output logic src_ready_o,
119
120     input logic dst_rst_ni,
121     input logic dst_clk_i,
122     output data_t dst_data_o,
123     output logic dst_valid_o,
124     input logic dst_ready_i
125 );
126
127 data_t [2**LogDepth-1:0] async_data;
128 logic [LogDepth:0] async_wptr;
129 logic [LogDepth:0] async_rptr;
130
131 cc_cdc_fifo_gray_src #(
132     .data_t ( data_t ),
133     .LogDepth ( LogDepth ),
134     .SyncStages ( SyncStages )
135 ) i_src (
136     .src_rst_ni,
137     .src_clk_i,

```

```

138     .src_data_i,
139     .src_valid_i,
140     .src_ready_o,
141
142     (* async *) .async_data_o ( async_data ),
143     (* async *) .async_wptr_o ( async_wptr ),
144     (* async *) .async_rptr_i ( async_rptr )
145 );
146
147 cc_cdc_fifo_gray_dst #(
148     .data_t ( data_t ),
149     .LogDepth ( LogDepth ),
150     .SyncStages ( SyncStages )
151 ) i_dst (
152     .dst_rst_ni,
153     .dst_clk_i,
154     .dst_data_o,
155     .dst_valid_o,
156     .dst_ready_i,
157
158     (* async *) .async_data_i ( async_data ),
159     (* async *) .async_wptr_i ( async_wptr ),
160     (* async *) .async_rptr_o ( async_rptr )
161 );
162
163 // Check the invariants.
164 `ifndef COMMON_CELLS_ASSERTS_OFF
165 `ASSERT_INIT(log_depth_0, LogDepth > 0)
166 `ASSERT_INIT(sync_stages_gt_2, SyncStages >= 2)
167 `endif
168
169 endmodule
170
171
172 (* no_ungroup *)
173 (* no_boundary_optimization *)
174 module cc_cdc_fifo_gray_src #(
175     parameter type data_t = logic,
176     parameter int unsigned LogDepth = 3,
177     parameter int unsigned SyncStages = 2
178 ) (
179     input logic src_rst_ni,
180     input logic src_clk_i,
181     input data_t src_data_i,
182     input logic src_valid_i,
183     output logic src_ready_o,
184
185     output data_t [2**LogDepth-1:0] async_data_o,
186     output logic [LogDepth:0] async_wptr_o,
187     input logic [LogDepth:0] async_rptr_i
188 );
189
190 logic [LogDepth-1:0] src_widx;
191 logic src_write;
192
193 assign src_write = src_valid_i & src_ready_o;
194
195 cc_cdc_fifo_gray_src_data #(
196     .data_t ( data_t ),
197     .LogDepth ( LogDepth )
198 ) i_src_data (
199     .src_rst_ni,
200     .src_clk_i,
201     .src_data_i,
202     .src_widx_i ( src_widx ),
203     .src_write_i ( src_write ),
204     .async_data_o
205 );
206
207 cc_cdc_fifo_gray_src_ptr #(
208     .LogDepth ( LogDepth ),
209     .SyncStages ( SyncStages )
210 ) i_src_ptr (
211     .src_rst_ni,
212     .src_clk_i,
213     .src_valid_i,
214     .src_ready_o,
215     .src_widx_o ( src_widx ),
216     .async_wptr_o,
217     .async_rptr_i
218 );
219
220 endmodule
221
222
223 (* no_ungroup *)
224 (* no_boundary_optimization *)
225 module cc_cdc_fifo_gray_dst #(
226     parameter type data_t = logic,
227     parameter int unsigned LogDepth = 3,
228     parameter int unsigned SyncStages = 2
229 ) (
230     input logic dst_rst_ni,
231     input logic dst_clk_i,
232     output data_t dst_data_o,
233     output logic dst_valid_o,
234     input logic dst_ready_i,
235
236     input data_t [2**LogDepth-1:0] async_data_i,
237     input logic [LogDepth:0] async_wptr_i,
238     output logic [LogDepth:0] async_rptr_o

```

```

239 );
240 logic dst_valid, dst_ready;
241 logic [LogDepth-1:0] dst_ridx;
242
243 cc_cdc_fifo_gray_dst_ptr #(
244   .LogDepth ( LogDepth ),
245   .SyncStages ( SyncStages )
246 ) i_dst_ptr (
247   .dst_rst_ni,
248   .dst_clk_i,
249   .dst_ready_i ( dst_ready ),
250   .dst_valid_o ( dst_valid ),
251   .dst_ridx_o ( dst_ridx ),
252   .async_wptr_i,
253   .async_rptr_o
254 );
255
256 cc_cdc_fifo_gray_dst_data #(
257   .data_t ( data_t ),
258   .LogDepth ( LogDepth )
259 ) i_dst_data (
260   .dst_rst_ni,
261   .dst_clk_i,
262   .dst_data_o,
263   .dst_valid_o,
264   .dst_ready_i,
265   .dst_valid_i ( dst_valid ),
266   .dst_ready_o ( dst_ready ),
267   .dst_ridx_i ( dst_ridx ),
268   .async_data_i
269 );
270
271 endmodule
272
273 (* no_ungroup *)
274 (* no_boundary_optimization *)
275 module cc_cdc_fifo_gray_src_data #(
276   parameter type data_t = logic,
277   parameter int unsigned LogDepth = 3
278 )(
279   input logic src_rst_ni,
280   input logic src_clk_i,
281   input data_t src_data_i,
282   input logic [LogDepth-1:0] src_widx_i,
283   input logic src_write_i,
284   output data_t [2**LogDepth-1:0] async_data_o
285 );
286
287 data_t [2**LogDepth-1:0] data_d, data_q;
288
289 always_comb begin
290   data_d = data_q;
291   data_d[src_widx_i] = src_data_i;
292 end
293
294 `FFL(data_q, data_d, src_write_i, '0, src_clk_i, src_rst_ni)
295
296 assign async_data_o = data_q;
297
298 endmodule
299
300 (* no_ungroup *)
301 (* no_boundary_optimization *)
302 module cc_cdc_fifo_gray_src_ptr #(
303   parameter int unsigned LogDepth = 3,
304   parameter int unsigned SyncStages = 2
305 )(
306   input logic src_rst_ni,
307   input logic src_clk_i,
308   input logic src_valid_i,
309   output logic src_ready_o,
310   output logic [LogDepth-1:0] src_widx_o,
311   output logic [LogDepth:0] async_wptr_o,
312   input logic [LogDepth:0] async_rptr_i
313 );
314
315 localparam int unsigned PtrWidth = LogDepth+1;
316 localparam logic [PtrWidth-1:0] PtrFull = (1 << LogDepth);
317
318 logic [PtrWidth-1:0] wptr_q, wptr_d, wptr_bin, wptr_next,
rptr, rptr_bin;
319
320 if (SyncStages < 2) begin : gen_sync_stage_assertion
321   $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
322 end
323
324 // Read pointer from the destination domain, synchronized
into the source
325 // domain for the full check.
326 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
327   tc_sync #(.Stages(SyncStages)) i_sync (
328     .clk_i ( src_clk_i ),
329     .rst_ni ( src_rst_ni ),
330     .serial_i ( async_rptr_i[i] ),
331     .serial_o ( rptr[i] )
332   );
333 end
334
335

```

```

336 end
337 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
(.a_i(rptr), .z_o(rptr_bin));
338
339 // Local write pointer.
340 assign wptr_next = wptr_bin+1;
341 cc_gray_to_binary #(PtrWidth) i_wptr_g2b
(.a_i(wptr_q), .z_o(wptr_bin));
342 cc_binary_to_gray #(PtrWidth) i_wptr_b2g
(.a_i(wptr_next), .z_o(wptr_d));
343 `FFL(wptr_q, wptr_d, src_valid_i & src_ready_o, '0,
src_clk_i, src_rst_ni)
344
345 assign src_widx_o = wptr_bin[LogDepth-1:0];
346 assign async_wptr_o = wptr_q;
347
348 // The pointers into the FIFO are one bit wider than the ac-
tual address into
349 // the FIFO. This makes detecting critical states very
simple: if all but the
350 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
351 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
352 assign src_ready_o = ((wptr_bin ^ rptr_bin) != PtrFull);
353
354 endmodule
355
356 (* no_ungroup *)
357 (* no_boundary_optimization *)
358 module cc_cdc_fifo_gray_dst_ptr #(
359   parameter int unsigned LogDepth = 3,
360   parameter int unsigned SyncStages = 2
361 )(
362   input logic dst_rst_ni,
363   input logic dst_clk_i,
364   input logic dst_ready_i,
365   output logic dst_valid_o,
366   output logic [LogDepth-1:0] dst_ridx_o,
367   input logic [LogDepth:0] async_wptr_i,
368   output logic [LogDepth:0] async_rptr_o
369 );
370
371 localparam int unsigned PtrWidth = LogDepth+1;
372 localparam logic [PtrWidth-1:0] PtrEmpty = '0;
373
374 logic [PtrWidth-1:0] rptr_q, rptr_d, rptr_bin, rptr_next,
wptr, wptr_bin;
375
376 if (SyncStages < 2) begin : gen_sync_stage_assertion
377   $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
378 end
379
380 // Local read pointer.
381 assign rptr_next = rptr_bin+1;
382 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
(.a_i(rptr_q), .z_o(rptr_bin));
383 cc_binary_to_gray #(PtrWidth) i_rptr_b2g
(.a_i(rptr_next), .z_o(rptr_d));
384 `FFL(rptr_q, rptr_d, dst_valid_o & dst_ready_i, '0,
dst_clk_i, dst_rst_ni)
385
386 assign dst_ridx_o = rptr_bin[LogDepth-1:0];
387 assign async_rptr_o = rptr_q;
388
389 // Write pointer from the source domain, synchronized into
the destination
390 // domain for the empty check.
391 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
392   tc_sync #(.Stages(SyncStages)) i_sync (
393     .clk_i ( dst_clk_i ),
394     .rst_ni ( dst_rst_ni ),
395     .serial_i ( async_wptr_i[i] ),
396     .serial_o ( wptr[i] )
397   );
398 end
399
400 cc_gray_to_binary #(PtrWidth) i_wptr_g2b
(.a_i(wptr), .z_o(wptr_bin));
401
402 // The pointers into the FIFO are one bit wider than the ac-
tual address into
403 // the FIFO. This makes detecting critical states very
simple: if all but the
404 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
405 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
406 assign dst_valid_o = ((wptr_bin ^ rptr_bin) != PtrEmpty);
407
408 endmodule
409
410 (* no_ungroup *)
411 (* no_boundary_optimization *)
412 module cc_cdc_fifo_gray_dst_data #(
413   parameter type data_t = logic,
414   parameter int unsigned LogDepth = 3
415 )(
416

```



```

417 input logic dst_rst_ni,
418 input logic dst_clk_i,
419 output data_t dst_data_o,
420 output logic dst_valid_o,
421 input logic dst_ready_i,
422 input logic dst_valid_i,
423 output logic dst_ready_o,
424 input logic [LogDepth-1:0] dst_ridx_i,
425 input data_t [2**LogDepth-1:0] async_data_i
426 );
427
428 data_t dst_data;
429
430 assign dst_data = async_data_i[dst_ridx_i];
431
432 // Cut the combinatorial path with a spill register.
433 cc_spill_register #(
434   .data_t ( data_t )
435 ) i_spill_register (
436   .clk_i ( dst_clk_i ),
437   .rst_ni ( dst_rst_ni ),
438   .clr_i ( '0 ),
439   .valid_i ( dst_valid_i ),
440   .ready_o ( dst_ready_o ),
441   .data_i ( dst_data ),
442   .valid_o ( dst_valid_o ),
443   .ready_i ( dst_ready_i ),
444   .data_o ( dst_data_o )
445 );
446
447 endmodule

```

13. pulp-platform_common_cells/src/cc_cdc_fifo_gray_clearable.sv (698 lines)

```

1 // Copyright 2018-2019 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
  pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
  file except in
5 // compliance with the License. You may obtain a copy of the
  License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
7 // or agreed to in writing, software, hardware and materials
  distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
10 // specific language governing permissions and limitations un-
  der the License.
11 //
12 // Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
13 // Florian Zaruba <zarubaf@iis.ee.ethz.ch>
14 // Manuel Eggimann <meggimann@iis.ee.ethz.ch> (clearability
  feature)
15 // Philippe Sauter <phsauter@iis.ee.ethz.ch> (source/destina-
  tion split)
16
17 /// A clock domain crossing FIFO, using gray counters.
18 ///
19 /// # Architecture
20 ///
21 /// The design is split into two parts, each one being clocked
  and reset
22 /// separately.
23 /// 1. The data to be transferred over the clock domain
  boundary is
24 /// is stored in a FIFO. The corresponding write pointer is
  managed
25 /// (incremented) in the source clock domain.
26 /// 2. The entire FIFO content is exposed over the
  `async_data` port.
27 /// The destination clock domain increments its read point-
  er
28 /// in its destination clock domain.
29 ///
30 /// Read and write pointers are then gray coded, communicated
31 /// and synchronized using a classic multi-stage FF synchron-
  izer
32 /// in the other clock domain. The gray coding ensures that
  only
33 /// one bit changes at each pointer increment, preventing the
34 /// synchronizer to accidentally latch an inconsistent state
35 /// on a multi-bit bus.
36 ///
37 /// The not full signal e.g. `src_ready_o` (on the sending
  side)
38 /// is generated using the local write pointer and the pessim-
  istic
39 /// read pointer from the destination clock domain (pessimist-
  ic
40 /// because it is delayed at least two cycles because of the
  synchronizer
41 /// stages). This prevents the FIFO from overflowing.

```

```

42 ///
43 /// The not empty signal e.g. `dst_valid_o` is generated using
44 /// the pessimistic write pointer and the local read pointer
  in
45 /// the destination clock domain. This means the FIFO content
46 /// does not need to be synchronized as we are sure we are
  reading
47 /// data which has been written at least two cycles earlier.
48 /// Furthermore, the read select logic into the FIFO is com-
  pletely
49 /// clocked by the destination clock domain which avoids
50 /// inefficient data synchronization.
51 ///
52 /// The FIFO size must be powers of two, which is why its
  depth is
53 /// given as 2**LogDepth. LogDepth must be at least 1.
54
55 /// Reset Behavior:
56 ///
57 /// In contrast to the cc_cdc_fifo_gray version without clear
  signal, this module
58 /// supports one-sided warm resets (asynchronously and syn-
  chronously). The way
59 /// this is implemented is described in more detail in the
  cc_cdc_reset_ctrlr
60 /// module. To summarize a synchronous clear request i.e. src/
  dst_clear_i will
61 /// cause the respective other clock domain to reset as well
  without introducing
62 /// any spurious transactions. This is accomplished by an in-
  ternal module
63 /// (cc_cdc_reset_ctrlr) the starts a reset sequence on both
  sides of the CDC in
64 /// lock-step that first isolates the CDC from the outside
  world and then resets
65 /// it. The reset sequencer provides the following behavior:
66 /// 1. There are no spurious invalid or duplicated transac-
  tions regardless how
67 /// the individual sides are reset (can also happen roughly
  simultaneously)
68 /// 2. The FIFO becomes unready at the src side in the next
  cycle after
69 /// synchronous reset request until the reset sequence is
  completed. Some of
70 /// the pending transactions might still complete (if the
  dst accepts at the
71 /// exact time the reset is request on the src die), some
  of them will be
72 /// dropped (of course still guaranteeing FIFO order).
73 /// 3. During the reset sequence the dst might withdraw the
  valid signal. This
74 /// might violate higher level protocols. If you need this
  feature you would
75 /// have to path the existing implementation to wait with
  the isolate_ack
76 /// assertion until all open handshakes were acknowledged.
77 /// 4. If the parameter ClearOnAsyncReset is enabled, the same
  behavior as
78 /// above is also valid for asynchronous resets on either
  side. However, this
79 /// increases the minimum number of synchronization stages
  (SyncStages
80 /// parameter) from 2 to 3 (read the cc_cdc_reset_ctrlr
  header to figure out
81 /// why).
82 ///
83 ///
84 /// # Constraints
85 ///
86 /// We need to make sure that the propagation delay of the
  data, read and write
87 /// pointer is bound to the minimum of either the sending or
  receiving clock
88 /// period to prevent an inconsistent state to be latched (if
  for example the
89 /// one bit of the read/write pointer have an excessive
  delay). Furthermore, we
90 /// should deactivate setup and hold checks on the asynchron-
  ous signals.
91 ///
92 /// async_data, async_wptr, and async_rptr are the FIFO stor-
  age and pointer CDCs
93 /// for the actual data path. async_reset_src2dst_req,
94 /// async_reset_dst2src_ack, async_reset_src2dst_next_phase,
95 /// async_reset_dst2src_req, async_reset_src2dst_ack, and
96 /// async_reset_dst2src_next_phase are the CDCs for the clear-
  ing state machines.
97 ///
98 /// `` set_ungroup [get_designs cc_cdc_fifo_gray*] false
  set_boundary_optimization
99 /// [get_designs cc_cdc_fifo_gray*] false set_max_delay
  min(T_src, T_dst) \
100 /// -through [get_pins -hierarchical -filter async] \ -through
  [get_pins
101 /// -hierarchical -filter async] set_false_path -hold \ -
  through [get_pins
102 /// -hierarchical -filter async] \ -through [get_pins -hier-
  archical -filter

```

```

103 /// async] ```
104
105 `include "common_cells/registers.svh"
106 `include "common_cells/assertions.svh"
107
108 (* no_ungroup *)
109 (* no_boundary_optimization *)
110 module cc_cdc_fifo_gray_clearable #(
111     /// The width of the default logic type.
112     parameter int unsigned Width = 1,
113     /// The data type of the payload transported by the FIFO.
114     parameter type data_t = logic [Width-1:0],
115     /// The FIFO's depth given as 2**LogDepth.
116     parameter int unsigned LogDepth = 3,
117     /// The number of synchronization registers to insert on the
118     async pointers
119     /// between the FIFOs. If ClearOnAsyncReset is enabled, we
120     need at least 3
121     /// synchronizer stages to provide the clear synchronizer
122     lower latency than
123     /// the async reset. I.e. if ClearOnAsyncReset==1 -> Sync-
124     Stages >= 3 else
125     /// SyncStages >= 2.
126     parameter int unsigned SyncStages = 3,
127     parameter bit ClearOnAsyncReset = 1
128 ) (
129     input logic src_rst_ni,
130     input logic src_clk_i,
131     input logic src_clear_i,
132     output logic src_clear_pending_o,
133     input data_t src_data_i,
134     input logic src_valid_i,
135     output logic src_ready_o,
136
137     input logic dst_rst_ni,
138     input logic dst_clk_i,
139     input logic dst_clear_i,
140     output logic dst_clear_pending_o,
141     output data_t dst_data_o,
142     output logic dst_valid_o,
143     input logic dst_ready_i
144 );
145
146 data_t [2**LogDepth-1:0] async_data;
147 logic [LogDepth:0] async_wptr;
148 logic [LogDepth:0] async_rptr;
149 cc_pkg::cdc_clear_seq_phase_e
150 async_reset_src2dst_next_phase;
151 logic async_reset_src2dst_req;
152 logic async_reset_dst2src_ack;
153 cc_pkg::cdc_clear_seq_phase_e
154 async_reset_dst2src_next_phase;
155 logic async_reset_dst2src_req;
156 logic async_reset_src2dst_ack;
157
158 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
159     if (SyncStages < 3)
160         $error("The clearable CDC FIFO with async reset syn-
161             chronization requires at least",
162             "3 synchronizer stages for the FIFO.");
163     end else begin : gen_elaboration_assertion
164         if (SyncStages < 2) begin : gen_elaboration_assertion
165             $error("A minimum of 2 synchronizer stages is required
166                 for proper functionality.");
167         end
168     end
169
170 if (2*SyncStages > 2**LogDepth) begin : gen_elabora-
171     tion_assertion2
172         $warning("The FIFOs depth of %0d is insufficient to com-
173             pletely hide the latency of",
174             "%0d SyncStages. The FIFO will stall in the case
175             where f_src ~= f_dst.",
176             "It is recommended to increase the FIFO's log
177             depth to at least %0d.",
178             2**LogDepth, SyncStages, $clog2(2*SyncStages));
179     end
180
181 cc_cdc_fifo_gray_clearable_src #(
182     .data_t ( data_t ),
183     .LogDepth ( LogDepth ),
184     .SyncStages ( SyncStages ),
185     .ClearOnAsyncReset ( ClearOnAsyncReset )
186 ) i_src (
187     .src_rst_ni,
188     .src_clk_i,
189     .src_clear_i,
190     .src_clear_pending_o,
191     .src_data_i,
192     .src_valid_i,
193     .src_ready_o,
194
195     (* async *) .async_data_o
196 );
197
198 (* async *) .async_wptr_o
199 );
200
201 (* async *) .async_rptr_i

```

```

202     (* async *) .async_reset_next_phase_o
203 );
204
205 (* async *) .async_reset_req_o
206 );
207
208 (* async *) .async_reset_ack_i
209 );
210
211 (* async *) .async_reset_dst2src_ack_i
212 );
213
214 (* async *) .async_reset_next_phase_i
215 );
216
217 (* async *) .async_reset_req_i
218 );
219
220 (* async *) .async_reset_ack_o
221 );
222
223 cc_cdc_fifo_gray_clearable_dst #(
224     .data_t ( data_t ),
225     .LogDepth ( LogDepth ),
226     .SyncStages ( SyncStages ),
227     .ClearOnAsyncReset ( ClearOnAsyncReset )
228 ) i_dst (
229     .dst_rst_ni,
230     .dst_clk_i,
231     .dst_clear_i,
232     .dst_clear_pending_o,
233     .dst_data_o,
234     .dst_valid_o,
235     .dst_ready_i,
236
237     (* async *) .async_data_i
238 );
239
240 (* async *) .async_wptr_i
241 );
242
243 (* async *) .async_rptr_o
244 );
245
246 (* async *) .async_reset_next_phase_o
247 );
248
249 (* async *) .async_reset_req_o
250 );
251
252 (* async *) .async_reset_ack_i
253 );
254
255 (* async *) .async_reset_next_phase_i
256 );
257
258 (* async *) .async_reset_req_i
259 );
260
261 (* async *) .async_reset_ack_o
262 );
263
264 // Check the invariants.
265 `ifndef COMMON_CELLS_ASSERTS_OFF
266     `ASSERT_INIT(log_depth_0, LogDepth > 0)
267     `ASSERT_INIT(sync_stages_lt_2, SyncStages >= 2)
268 `endif
269
270 endmodule
271
272
273 (* no_ungroup *)
274 (* no_boundary_optimization *)
275 module cc_cdc_fifo_gray_clearable_src #(
276     parameter type data_t = logic,
277     parameter int unsigned LogDepth = 3,
278     parameter int unsigned SyncStages = 2,
279     parameter bit ClearOnAsyncReset = 1
280 ) (
281     input logic src_rst_ni,
282     input logic src_clk_i,
283     input logic src_clear_i,
284     output logic src_clear_pending_o,
285     input data_t src_data_i,
286     input logic src_valid_i,
287     output logic src_ready_o,
288
289     output data_t [2**LogDepth-1:0] async_data_o,
290     output logic [LogDepth:0] async_wptr_o,
291     input logic [LogDepth:0] async_rptr_i,
292     output cc_pkg::cdc_clear_seq_phase_e
293     async_reset_next_phase_o,
294     output logic async_reset_req_o,
295     input logic async_reset_ack_i,
296     input cc_pkg::cdc_clear_seq_phase_e
297     async_reset_next_phase_i,
298     input logic async_reset_req_i,
299     output logic async_reset_ack_o
300 );
301
302 logic src_clear_req;
303 logic src_clear_ack_q;
304 logic src_ready;
305 logic src_isolate_req;
306 logic src_isolate_ack_q;
307
308 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
309     if (SyncStages < 3)
310         $error("The clearable CDC FIFO with async reset syn-
311             chronization requires at least",
312             "3 synchronizer stages for the FIFO.");
313     end else begin : gen_elaboration_assertion
314         if (SyncStages < 2) begin : gen_sync_stage_assertion
315             $error("A minimum of 2 synchronizer stages is required

```



```

for proper functionality.");
270     end
271 end
272
273 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
SyncStages - 1 : 2;
274
275 cc_cdc_reset_ctrlr_half #(
276     .SyncStages      ( ResetSyncStages ),
277     .ClearOnAsyncReset ( ClearOnAsyncReset )
278 ) i_cdc_reset_ctrlr_half (
279     .clk_i           ( src_clk_i           ),
280     .rst_ni          ( src_rst_ni          ),
281     .clear_i         ( src_clear_i         ),
282     .clear_o         ( src_clear_req       ),
283     .clear_ack_i     ( src_clear_ack_q     ),
284     .isolate_o       ( src_isolate_req     ),
285     .isolate_ack_i   ( src_isolate_ack_q   ),
286     (* async *) .async_next_phase_o
( async_reset_next_phase_o ),
287     (* async *) .async_req_o
( async_reset_req_o ),
288     (* async *) .async_ack_i
( async_reset_ack_i ),
289     (* async *) .async_next_phase_i
( async_reset_next_phase_i ),
290     (* async *) .async_req_i
( async_reset_req_i ),
291     (* async *) .async_ack_o
( async_reset_ack_o )
292 );
293
294 cc_cdc_fifo_gray_src_clearable #(
295     .data_t      ( data_t ),
296     .LogDepth    ( LogDepth ),
297     .SyncStages  ( SyncStages )
298 ) i_src (
299     .src_rst_ni,
300     .src_clk_i,
301     .src_clear_i ( src_clear_req ),
302     .src_data_i,
303     .src_valid_i ( src_valid_i & !src_isolate_req ),
304     .src_ready_o ( src_ready ),
305
306     (* async *) .async_data_o ( async_data_o ),
307     (* async *) .async_wptr_o ( async_wptr_o ),
308     (* async *) .async_rptr_i ( async_rptr_i )
309 );
310
311 assign src_ready_o = src_ready & !src_isolate_req;
312
313 // Isolation and clear are acknowledged one cycle after the
local request.
314 `FF(src_isolate_ack_q, src_isolate_req, 1'b0, src_clk_i,
src_rst_ni)
315 `FF(src_clear_ack_q, src_clear_req, 1'b0, src_clk_i,
src_rst_ni)
316
317 assign src_clear_pending_o = src_isolate_req;
318
319 endmodule
320
321
322 (* no_ungroup *)
323 (* no_boundary_optimization *)
324 module cc_cdc_fifo_gray_clearable_dst #(
325     parameter type data_t = logic,
326     parameter int unsigned LogDepth = 3,
327     parameter int unsigned SyncStages = 2,
328     parameter bit ClearOnAsyncReset = 1
329 )(
330     input  logic dst_rst_ni,
331     input  logic dst_clk_i,
332     input  logic dst_clear_i,
333     output logic dst_clear_pending_o,
334     output data_t dst_data_o,
335     output logic dst_valid_o,
336     input  logic dst_ready_i,
337
338     input data_t [2**LogDepth-1:0] async_data_i,
339     input logic [LogDepth:0] async_wptr_i,
340     output logic [LogDepth:0] async_rptr_o,
341     output cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_o,
342     output logic async_reset_req_o,
343     input  logic async_reset_ack_i,
344     input  cc_pkg::cdc_clear_seq_phase_e
async_reset_next_phase_i,
345     input logic async_reset_req_i,
346     output logic async_reset_ack_o
347 );
348
349 logic dst_clear_req;
350 logic dst_clear_ack_q;
351 logic dst_valid;
352 logic dst_isolate_req;
353 logic dst_isolate_ack_q;
354
355 if (ClearOnAsyncReset) begin : gen_elaboration_assertion
356     if (SyncStages < 3)

```

```

357     $error("The clearable CDC FIFO with async reset syn-
chronization requires at least",
358         "3 synchronizer stages for the FIFO.");
359 end else begin : gen_elaboration_assertion
360     if (SyncStages < 2) begin : gen_sync_stage_assertion
361         $error("A minimum of 2 synchronizer stages is required
for proper functionality.");
362     end
363 end
364
365 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
SyncStages - 1 : 2;
366
367 cc_cdc_reset_ctrlr_half #(
368     .SyncStages      ( ResetSyncStages ),
369     .ClearOnAsyncReset ( ClearOnAsyncReset )
370 ) i_cdc_reset_ctrlr_half (
371     .clk_i           ( dst_clk_i           ),
372     .rst_ni          ( dst_rst_ni          ),
373     .clear_i         ( dst_clear_i         ),
374     .clear_o         ( dst_clear_req       ),
375     .clear_ack_i     ( dst_clear_ack_q     ),
376     .isolate_o       ( dst_isolate_req     ),
377     .isolate_ack_i   ( dst_isolate_ack_q   ),
378     (* async *) .async_next_phase_o
( async_reset_next_phase_o ),
379     (* async *) .async_req_o
( async_reset_req_o ),
380     (* async *) .async_ack_i
( async_reset_ack_i ),
381     (* async *) .async_next_phase_i
( async_reset_next_phase_i ),
382     (* async *) .async_req_i
( async_reset_req_i ),
383     (* async *) .async_ack_o
( async_reset_ack_o )
384 );
385
386 cc_cdc_fifo_gray_dst_clearable #(
387     .data_t      ( data_t ),
388     .LogDepth    ( LogDepth ),
389     .SyncStages  ( SyncStages )
390 ) i_dst (
391     .dst_rst_ni,
392     .dst_clk_i,
393     .dst_clear_i ( dst_clear_req ),
394     .dst_data_o,
395     .dst_valid_o ( dst_valid ),
396     .dst_ready_i ( dst_ready_i & !dst_isolate_req ),
397
398     (* async *) .async_data_i ( async_data_i ),
399     (* async *) .async_wptr_i ( async_wptr_i ),
400     (* async *) .async_rptr_o ( async_rptr_o )
401 );
402
403 assign dst_valid_o = dst_valid & !dst_isolate_req;
404
405 // Isolation and clear are acknowledged one cycle after the
local request.
406 `FF(dst_isolate_ack_q, dst_isolate_req, 1'b0, dst_clk_i,
dst_rst_ni)
407 `FF(dst_clear_ack_q, dst_clear_req, 1'b0, dst_clk_i,
dst_rst_ni)
408
409 assign dst_clear_pending_o = dst_isolate_req;
410
411 endmodule
412
413
414 (* no_ungroup *)
415 (* no_boundary_optimization *)
416 module cc_cdc_fifo_gray_src_clearable #(
417     parameter type data_t = logic,
418     parameter int unsigned LogDepth = 3,
419     parameter int unsigned SyncStages = 2
420 )(
421     input logic src_rst_ni,
422     input logic src_clk_i,
423     input logic src_clear_i,
424     input data_t src_data_i,
425     input logic src_valid_i,
426     output logic src_ready_o,
427
428     output data_t [2**LogDepth-1:0] async_data_o,
429     output logic [LogDepth:0] async_wptr_o,
430     input logic [LogDepth:0] async_rptr_i
431 );
432
433 logic [LogDepth-1:0] src_widx;
434 logic src_write;
435
436 assign src_write = src_valid_i & src_ready_o;
437
438 cc_cdc_fifo_gray_src_data_clearable #(
439     .data_t      ( data_t ),
440     .LogDepth    ( LogDepth )
441 ) i_src_data (
442     .src_rst_ni,
443     .src_clk_i,
444     .src_data_i,

```

```

445 .src_widx_i ( src_widx ),
446 .src_write_i ( src_write ),
447 .async_data_o
448 );
449
450 cc_cdc_fifo_gray_src_ptr_clearable #(
451 .LogDepth ( LogDepth ),
452 .SyncStages ( SyncStages )
453 ) i_src_ptr (
454 .src_rst_ni,
455 .src_clk_i,
456 .src_clear_i,
457 .src_valid_i,
458 .src_ready_o,
459 .src_widx_o ( src_widx ),
460 .async_wptra_o,
461 .async_rptr_i
462 );
463
464 endmodule
465
466 (* no_ungroup *)
467 (* no_boundary_optimization *)
468 module cc_cdc_fifo_gray_dst_clearable #(
469 parameter type data_t = logic,
470 parameter int unsigned LogDepth = 3,
471 parameter int unsigned SyncStages = 2
472 )
473 (
474 input logic dst_rst_ni,
475 input logic dst_clk_i,
476 input logic dst_clear_i,
477 output data_t dst_data_o,
478 output logic dst_valid_o,
479 input logic dst_ready_i,
480
481 input data_t [2**LogDepth-1:0] async_data_i,
482 input logic [LogDepth:0] async_wptra_i,
483 output logic [LogDepth:0] async_rptr_o
484 );
485
486 logic dst_valid, dst_ready;
487 logic [LogDepth-1:0] dst_ridx;
488
489 cc_cdc_fifo_gray_dst_ptr_clearable #(
490 .LogDepth ( LogDepth ),
491 .SyncStages ( SyncStages )
492 ) i_dst_ptr (
493 .dst_rst_ni,
494 .dst_clk_i,
495 .dst_clear_i,
496 .dst_ready_i ( dst_ready ),
497 .dst_valid_o ( dst_valid ),
498 .dst_ridx_o ( dst_ridx ),
499 .async_wptra_i,
500 .async_rptr_o
501 );
502
503 cc_cdc_fifo_gray_dst_data_clearable #(
504 .data_t ( data_t ),
505 .LogDepth ( LogDepth )
506 ) i_dst_data (
507 .dst_rst_ni,
508 .dst_clk_i,
509 .dst_clear_i,
510 .dst_data_o,
511 .dst_valid_o,
512 .dst_ready_i,
513 .dst_valid_i ( dst_valid ),
514 .dst_ready_o ( dst_ready ),
515 .dst_ridx_i ( dst_ridx ),
516 .async_data_i
517 );
518
519 endmodule
520
521 (* no_ungroup *)
522 (* no_boundary_optimization *)
523 module cc_cdc_fifo_gray_src_data_clearable #(
524 parameter type data_t = logic,
525 parameter int unsigned LogDepth = 3
526 )
527 (
528 input logic src_rst_ni,
529 input logic src_clk_i,
530 input data_t src_data_i,
531 input logic [LogDepth-1:0] src_widx_i,
532 input logic src_write_i,
533 output data_t [2**LogDepth-1:0] async_data_o
534 );
535
536 data_t [2**LogDepth-1:0] data_d, data_q;
537
538 always_comb begin
539 data_d = data_q;
540 data_d[src_widx_i] = src_data_i;
541 end
542
543 `FLL(data_q, data_d, src_write_i, '0, src_clk_i, src_rst_ni)
544
545 assign async_data_o = data_q;

```

```

546
547 endmodule
548
549 (* no_ungroup *)
550 (* no_boundary_optimization *)
551 module cc_cdc_fifo_gray_src_ptr_clearable #(
552 parameter int unsigned LogDepth = 3,
553 parameter int unsigned SyncStages = 2
554 )
555 (
556 input logic src_rst_ni,
557 input logic src_clk_i,
558 input logic src_clear_i,
559 input logic src_valid_i,
560 output logic src_ready_o,
561 output logic [LogDepth-1:0] src_widx_o,
562 output logic [LogDepth:0] async_wptra_o,
563 input logic [LogDepth:0] async_rptr_i
564 );
565
566 localparam int unsigned PtrWidth = LogDepth+1;
567 localparam logic [PtrWidth-1:0] PtrFull = (1 << LogDepth);
568
569 logic [PtrWidth-1:0] wptra_q, wptra_d, wptra_bin, wptra_next,
rptr, rptr_bin;
570
571 if (SyncStages < 2) begin : gen_sync_stage_assertion
572 $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
573 end
574
575 // Read pointer from the destination domain, synchronized
into the source
576 // domain for the full check.
577 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
578 tc_sync #(.Stages(SyncStages)) i_sync (
579 .clk_i ( src_clk_i ),
580 .rst_ni ( src_rst_ni ),
581 .serial_i ( async_rptr_i[i] ),
582 .serial_o ( rptr[i] )
583 );
584 end
585 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
(.a_i(rptr), .z_o(rptr_bin));
586
587 // Local write pointer.
588 assign wptra_next = wptra_bin+1;
589 cc_gray_to_binary #(PtrWidth) i_wptra_g2b
(.a_i(wptra_q), .z_o(wptra_bin));
590 cc_binary_to_gray #(PtrWidth) i_wptra_b2g
(.a_i(wptra_next), .z_o(wptra_d));
591 `FLLRNC(wptra_q, wptra_d, src_valid_i & src_ready_o,
src_clear_i, '0, src_clk_i, src_rst_ni)
592
593 assign src_widx_o = wptra_bin[LogDepth-1:0];
594 assign async_wptra_o = wptra_q;
595
596 // The pointers into the FIFO are one bit wider than the ac-
tual address into
597 // the FIFO. This makes detecting critical states very
simple: if all but the
598 // topmost bit of rptr and wptra agree, the FIFO is in a
critical state. If the
599 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
600 assign src_ready_o = ((wptra_bin ^ rptr_bin) != PtrFull);
601
602 endmodule
603
604 (* no_ungroup *)
605 (* no_boundary_optimization *)
606 module cc_cdc_fifo_gray_dst_ptr_clearable #(
607 parameter int unsigned LogDepth = 3,
608 parameter int unsigned SyncStages = 2
609 )
610 (
611 input logic dst_rst_ni,
612 input logic dst_clk_i,
613 input logic dst_clear_i,
614 input logic dst_ready_i,
615 output logic dst_valid_o,
616 output logic [LogDepth-1:0] dst_ridx_o,
617 input logic [LogDepth:0] async_wptra_i,
618 output logic [LogDepth:0] async_rptr_o
619 );
620
621 localparam int unsigned PtrWidth = LogDepth+1;
622 localparam logic [PtrWidth-1:0] PtrEmpty = '0;
623
624 logic [PtrWidth-1:0] rptr_q, rptr_d, rptr_bin, rptr_next,
wptra, wptra_bin;
625
626 if (SyncStages < 2) begin : gen_sync_stage_assertion
627 $error("A minimum of 2 synchronizer stages is required for
proper functionality.");
628 end
629
630 // Local read pointer.
631 assign rptr_next = rptr_bin+1;

```

```

632 cc_gray_to_binary #(PtrWidth) i_rptr_g2b
(.a_i(rptr_q), .z_o(rptr_bin));
633 cc_binary_to_gray #(PtrWidth) i_rptr_b2g
(.a_i(rptr_next), .z_o(rptr_d));
634 `FLLARNC(rptr_q, rptr_d, dst_valid_o & dst_ready_i,
dst_clear_i, '0, dst_clk_i, dst_rst_ni)
635
636 assign dst_ridx_o = rptr_bin[LogDepth-1:0];
637 assign async_rptr_o = rptr_q;
638
639 // Write pointer from the source domain, synchronized into
the destination
640 // domain for the empty check.
641 for (genvar i = 0; i < PtrWidth; i++) begin : gen_sync
642   tc_sync #(.Stages(SyncStages)) i_sync (
643     .clk_i ( dst_clk_i ),
644     .rst_ni ( dst_rst_ni ),
645     .serial_i ( async_wptr_i[i] ),
646     .serial_o ( wptr[i] )
647   );
648 end
649 cc_gray_to_binary #(PtrWidth) i_wptr_g2b
(.a_i(wptr), .z_o(wptr_bin));
650
651 // The pointers into the FIFO are one bit wider than the ac-
tual address into
652 // the FIFO. This makes detecting critical states very
simple: if all but the
653 // topmost bit of rptr and wptr agree, the FIFO is in a
critical state. If the
654 // topmost bit is equal, the FIFO is empty, otherwise it is
full.
655 assign dst_valid_o = (wptr_bin ^ rptr_bin) != PtrEmpty;
656
657 endmodule
658
659
660 (* no_ungroup *)
661 (* no_boundary_optimization *)
662 module cc_cdc_fifo_gray_dst_data_clearable #(
663   parameter type data_t = logic,
664   parameter int unsigned LogDepth = 3
665 )
666   input logic dst_rst_ni,
667   input logic dst_clk_i,
668   input logic dst_clear_i,
669   output data_t dst_data_o,
670   output logic dst_valid_o,
671   input logic dst_ready_i,
672   input logic dst_valid_i,
673   output logic dst_ready_o,
674   input logic [LogDepth-1:0] dst_ridx_i,
675   input data_t [2*LogDepth-1:0] async_data_i
676 ;
677
678   data_t dst_data;
679
680   assign dst_data = async_data_i[dst_ridx_i];
681
682   // Cut the combinatorial path with a spill register.
683   cc_spill_register_flushable #(
684     .data_t ( data_t )
685   ) i_spill_register (
686     .clk_i ( dst_clk_i ),
687     .rst_ni ( dst_rst_ni ),
688     .clr_i ( '0 ),
689     .flush_i ( dst_clear_i ),
690     .valid_i ( dst_valid_i & !dst_clear_i ),
691     .ready_o ( dst_ready_o ),
692     .data_i ( dst_data ),
693     .valid_o ( dst_valid_o ),
694     .ready_i ( dst_ready_i ),
695     .data_o ( dst_data_o )
696   );
697
698 endmodule

```

14. pulp-platform_common_cells/src/cc_cdc_reset_ctrlr.vv (522 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //

```

```

12 // Authors:
13 // - Manuel Eggimann <megegimann@iis.ee.ethz.ch>
14 // - Philippe Sauter <phsauter@iis.ee.ethz.ch> (formalization
and stage cleanup)
15 //
16 // Description: CDC Clear Signaling Synchronization
17 // Coordinate clear and isolate sequencing between two CDC
clock domains so
18 // one-sided reset or clear requests do not break the protec-
ted CDC protocol.
19 //
20 // Detailed Description:
21 // This module is mainly used internally to synchronize the
clear requests
22 // between both sides of a CDC module. It aims to solve the
problem of
23 // initiating a CDC clear, reset one-sidedly without running
into
24 // reset-domain-crossing issues and breaking CDC protocol as-
sumption.
25 //
26 // Problem Formulation:
27 //
28 // CDC implementations usually face the issue that one side of
the CDC must not
29 // be cleared without clearing the other side. E.g. clearing
the write-pointer
30 // without clearing the read-pointer in a gray-counting CDC
FIFO results in an
31 // invalid fill-state and may cause spurious transactions of
invalid data to be
32 // propagated across the CDC. A similar effect is caused in
2-phase CDC
33 // implementations.
34 //
35 // A naive mitigation technique would be to reset both domains
asynchronously
36 // with the same reset signal. This will cause intra-clock do-
main RDC issues
37 // since the asynchronous clear event (assertion of the reset
signal) might
38 // happen close to the active edge of the CDC's periphery and
thus might induce
39 // metastability. A better, but still flawed approach would be to
naively
40 // synchronize assertion AND deassertion (the usual rst sync
only synchronize
41 // deassertion) of the resets into the respective other do-
main. However, this
42 // might cause the classic issue of fast-to-slow clock domain
crossing where the
43 // clear signal is not asserted long enough to be captured by
the receiving
44 // side. The common mitigation strategy is to use a feedback
acknowledge signal
45 // to handshake the reset request into the other domain. One
even more peculiar
46 // corner case this approach might suffer is the scenario
where the synchronized
47 // clear signal arrives at the other side of the CDC within or
even worse after
48 // the same clock cycle that the other domain crossing signals
(e.g. read/write
49 // pointers) are cleared. In this scenario, multiple signals
change within the
50 // same clock cycle and due to metastability we cannot be
sure, that the other
51 // side of the CDC sees the reset assertion before the first
bits of e.g. the
52 // write/read pointer start to switch to their reset state.
Care must also be

```

```

53 // taken to handle the corner cases where both sides are reset
simultaneously or
54 // the case where one side leaves reset earlier than the oth-
er.
55 //
56 // How this Module Works
57 //
58 // This module has two interfaces, the 'a' side and the 'b'
side. Each side can
59 // be triggered using the a/b_clear_i signal or (optionally)
by the asynchronous
60 // a/b_rst_ni. Once e.g. 'a' is triggered it will initiate a
clear sequence that
61 // first asserts an 'a_isolate_o' signal, waits until the ex-
ternal circuitry
62 // acknowledges isolation using the 'a_isolate_ack_i'. Then
the module asserts
63 // the 'a_clear_o' signals before some cycles later, the isol-
ate signal is
64 // deasserted. This sequence ensures that no transactions can
arrive to the CDC
65 // while the state is cleared. Now the important part is, that
those four phases

```

```

66 // (asser isolate, assert clear, deassert clear, deassert
isolate) are mirrored
67 // on the other side ('b') in lock-step. The cc_cdc_re-
set_ctrlr module uses a
68 // dedicated 4-phase handshaking CDC to transmit the current
phase of the clear
69 // sequence to the other domain. We use a 4-phase rather than
a 2-phase CDC to
70 // avoid the issues of one-sided async reset that might trig-
ger spurious
71 // transactions. Furthermore, the 4-phase CDC within this mod-
ule is operated in
72 // a special mode: Decoupled=0 ensures that there are no in-
flight transactions.
73 // The src side only consumes the item once the destination
side acknowledged
74 // the reception. This property is required to transition
through the phases in

75 // lock-step. Furthermore, (SendResetMsg=1) will cause the src
side of the
76 // 4-phase CDC to immediately initiate the isolation phase in
the dst domain

77 // upon asynchronous reset regardless how long the async reset
stays asserted or
78 // whether the source clock is gated. Both sides of this mod-
ule independently

79 // generate the sequence signals as an initiator (triggered by
the clear_i or
80 // rst_ni signal) or receiver (triggered for the other side).
The or-ed version
81 // of initiator and receiver are used to generate the actual
a/b_isolate_o and

82 // a/b_clear_o signal. That way, it doesn't matter wheter both
sides
83 // simulatenously trigger a clear sequence, proper sequencing
is still
84 // guaranteed.
85 //
86 // The time it takes to complete an entire clear sequence can
be bounded as follows:
87 //
88 //  $t_{clear} \leq 20 \cdot T + 16 \cdot SyncStages \cdot T$ , with  $T = \max(T_a, T_b)$ 
(clock periods of src and dst)
89 //
90 // How to Use the Module
91 //
92 // Instantiate the module within your CDC and connect a/
b_clk_i, the
93 // asynchronous a/b_rst_ni and the synchronous a/b_clear_i
signals. The 'a' and

94 // 'b' port are entirely symetric so it doesn't matter whether
you connect src
95 // to 'a' or 'b'. If you enable support for async reset
96 // (ClearOnAsyncReset==1), parametrize the number of synchron-
ization stages

97 // (for metastability resolution) to be strictly less than the
latency of the
98 // CDC. E.g. if your CDC uses 3 (the minimum) sync stages,
parametrize this
99 // module with SyncStages < 2! Your CDC must implement a src/
dst_clear_i port
100 // that SYNCHRONOUSLY clears all FFs on the respective side.
Connect the CDC's

101 // src/dst_clear ports to this module's a/b_clear_o port. Once
the a/b_isolate_o
102 // signal is asserted, the respective CDC side (src/dst) must
be isolated from
103 // the outside world (i.e. must no longer accept any transac-
tion on the src side
104 // and cease presenting or even withdrawing data on the dst
side). Once your CDC
105 // side is isolated (depending on protocol this might take
several cycles),
106 // assert the a/b_isolate_ack_i signal.
107 //
108 // include "common_cells/registers.svh"
109
110 module cc_cdc_reset_ctrlr
111     import cc_pkg::*;
112     #(
113         /// The number of synchronization stages to use for the
114         /// clear signal request/acknowledge. Must be less than the
115         /// number of sync stages used in the CDC.
116         parameter int unsigned SyncStages = 2,
117         /// Whether an asynchronous reset shall cause a clear
118         /// request to be sent to the other side.
119         parameter logic ClearOnAsyncReset = 1'b1
120     ) (
121         // Side A (both sides are symmetric)
122         input logic a_clk_i,
123         input logic a_rst_ni,
124         input logic a_clear_i,
125         output logic a_clear_o,
126         input logic a_clear_ack_i,

```

```

127         output logic a_isolate_o,
128         input logic a_isolate_ack_i,
129         // Side B (both sides are symmetric)
130         input logic b_clk_i,
131         input logic b_rst_ni,
132         input logic b_clear_i,
133         output logic b_clear_o,
134         input logic b_clear_ack_i,
135         output logic b_isolate_o,
136         input logic b_isolate_ack_i
137     );

138
139     (* dont_touch = "true" *)
140     logic async_a2b_req, async_b2a_ack;
141     (* dont_touch = "true" *)
142     cdc_clear_seq_phase_e async_a2b_next_phase;
143     (* dont_touch = "true" *)
144     logic async_b2a_req, async_a2b_ack;
145     (* dont_touch = "true" *)
146     cdc_clear_seq_phase_e async_b2a_next_phase;
147
148     cc_cdc_reset_ctrlr_half #(
149         .SyncStages ( SyncStages ),
150         .ClearOnAsyncReset ( ClearOnAsyncReset )
151     ) i_cdc_reset_ctrlr_half_a (
152         .clk_i ( a_clk_i ),
153         .rst_ni ( a_rst_ni ),
154         .clear_i ( a_clear_i ),
155         .clear_o ( a_clear_o ),
156         .clear_ack_i ( a_clear_ack_i ),
157         .isolate_o ( a_isolate_o ),
158         .isolate_ack_i ( a_isolate_ack_i ),
159         (* async *) .async_next_phase_o ( async_a2b_next_phase ),
160         (* async *) .async_req_o ( async_a2b_req ),
161         (* async *) .async_ack_i ( async_b2a_ack ),
162         (* async *) .async_next_phase_i ( async_b2a_next_phase ),
163         (* async *) .async_req_i ( async_b2a_req ),
164         (* async *) .async_ack_o ( async_a2b_ack )
165     );
166
167     cc_cdc_reset_ctrlr_half #(
168         .SyncStages ( SyncStages ),
169         .ClearOnAsyncReset ( ClearOnAsyncReset )
170     ) i_cdc_reset_ctrlr_half_b (
171         .clk_i ( b_clk_i ),
172         .rst_ni ( b_rst_ni ),
173         .clear_i ( b_clear_i ),
174         .clear_o ( b_clear_o ),
175         .clear_ack_i ( b_clear_ack_i ),
176         .isolate_o ( b_isolate_o ),
177         .isolate_ack_i ( b_isolate_ack_i ),
178         (* async *) .async_next_phase_o ( async_b2a_next_phase ),
179         (* async *) .async_req_o ( async_b2a_req ),
180         (* async *) .async_ack_i ( async_a2b_ack ),
181         (* async *) .async_next_phase_i ( async_a2b_next_phase ),
182         (* async *) .async_req_i ( async_a2b_req ),
183         (* async *) .async_ack_o ( async_b2a_ack )
184     );
185 endmodule
186
187
188 module cc_cdc_reset_ctrlr_half
189     import cc_pkg::*;
190     #(
191         /// The number of synchronization stages to use for the
192         /// clear signal request/acknowledge. Must be less than
193         /// the number of sync stages used in the CDC
194         parameter int unsigned SyncStages = 2,
195         /// Whether an asynchronous reset shall cause a clear
196         /// request to be sent to the other side.
197         parameter logic ClearOnAsyncReset = 1'b1
198     ) (
199         // Synchronous side
200         input logic clk_i,
201         input logic rst_ni,
202         input logic clear_i,
203         output logic isolate_o,
204         input logic isolate_ack_i,
205         output logic clear_o,
206         input logic clear_ack_i,
207         // Asynchronous clear sequence handshaking
208         output cdc_clear_seq_phase_e async_next_phase_o,
209         output logic async_req_o,
210         input logic async_ack_i,
211         input cdc_clear_seq_phase_e async_next_phase_i,
212         input logic async_req_i,
213         output logic async_ack_o
214     );
215
216
217     // How this module works:
218
219     // The module is split into two parts. The initiator part
220     // consists of an FSM
221     // that is triggered by the clear_i signal and transitions
222     // through reset
223     // sequence. During those transitions, the `initiat-
224     // or_isolate_out` and
225     // `initiator_clear_out` signals are asserted appropriately.

```

```

224 // The receiver part receives the state transitions from the
other clock
225 // domain (initiator part of the `cc_cdc_reset_ctrlr_half`
instance in the other
226 // clock domain) and asserts the `receiver_isolate_out` and
227 // `receiver_clear_out` appropriately (considering the
`isolate_ack_i`
228 // signal).
229
230 // In both, the initiator and the receiver part, the re-
spective FSM
231 // transitions through 4 phases. In the ISOLATE phase, the
isolate signal is
232 // asserted and the connected CDCs are expected to block all
further
233 // interactions with the outside world and acknowledge the
isolation with the
234 // isolate_ack_i signal. In the CLEAR phase, the clear sig-
nal is asserted
235 // which resets the internal state of the CDC while keeping
the isolate signal
236 // asserted. In the POST_CLEAR phase, the clear signal is
deasserted. Finally,
237 // when returning to the IDLE phase, the isolate signal is
deasserted to
238 // continue normal operation. The FSM uses a dedicated 4-
phase handshaking CDC
239 // to transition between the phases in lock-step and trans-
mits the current
240 // state to the other domain to avoid issues if the other
domain is reset
241 // asynchronously while a clear procedure is pending.
242
243 //----- Initiator Side
244 // Sends clear sequence state transitions to the other side.
245 typedef enum logic[3:0] {
246     IDLE,
247     ISOLATE,
248     WAIT_ISOLATE_PHASE_ACK,
249     WAIT_ISOLATE_ACK,
250     CLEAR,
251     WAIT_CLEAR_PHASE_ACK,
252     WAIT_CLEAR_ACK,
253     POST_CLEAR,
254     FINISHED
255 } initiator_state_e;
256 initiator_state_e initiator_state_d, initiator_state_q;
257 localparam initiator_state_e InitiatorStateResetValue =
ClearOnAsyncReset ? ISOLATE : IDLE;
258
259 // The current phase of the clear sequence, sent to the oth-
er side
using a
260 // 4-phase CDC
261 cdc_clear_seq_phase_e initiator_clear_seq_phase;
262 logic initiator_phase_transition_req;
263 logic initiator_phase_transition_ack;
264 logic initiator_isolate_out;
265 logic initiator_clear_out;
266
267 // The phase CDC is a real CDC even when callers pass a re-
duced latency for
268 // synchronous-clear-only configurations.
269 localparam int unsigned ResetSyncStages = (SyncStages > 2) ?
SyncStages : 2;
270
271 always_comb begin
272     initiator_state_d = initiator_state_q;
273     initiator_phase_transition_req = 1'b0;
274     initiator_isolate_out = 1'b0;
275     initiator_clear_out = 1'b0;
276     initiator_clear_seq_phase = CDC_CLEAR_PHASE_IDLE;
277
278     case (initiator_state_q)
279     IDLE: begin
280         if (clear_i) begin
281             initiator_state_d = ISOLATE;
282         end
283     end
284
285     ISOLATE: begin
286         initiator_phase_transition_req = 1'b1;
287         initiator_clear_seq_phase =
CDC_CLEAR_PHASE_ISOLATE;
288         initiator_isolate_out = 1'b1;
289         initiator_clear_out = 1'b0;
290         if (initiator_phase_transition_ack && isolate_ack_i)
begin
291             initiator_state_d = CLEAR;
292         end else if (initiator_phase_transition_ack) begin
293             initiator_state_d = WAIT_ISOLATE_ACK;
294         end else if (isolate_ack_i) begin
295             initiator_state_d = WAIT_ISOLATE_PHASE_ACK;
296         end
297     end
298
299     WAIT_ISOLATE_ACK: begin
300         initiator_isolate_out = 1'b1;
301         initiator_clear_out = 1'b0;

```

```

302     initiator_clear_seq_phase = CDC_CLEAR_PHASE_ISOLATE;
303     if (isolate_ack_i) begin
304         initiator_state_d = CLEAR;
305     end
306 end
307
308 WAIT_ISOLATE_PHASE_ACK: begin
309     initiator_phase_transition_req = 1'b1;
310     initiator_clear_seq_phase =
CDC_CLEAR_PHASE_ISOLATE;
311     initiator_isolate_out = 1'b1;
312     initiator_clear_out = 1'b0;
313     if (initiator_phase_transition_ack) begin
314         initiator_state_d = CLEAR;
315     end
316 end
317
318 CLEAR: begin
319     initiator_isolate_out = 1'b1;
320     initiator_clear_out = 1'b1;
321     initiator_phase_transition_req = 1'b1;
322     initiator_clear_seq_phase =
CDC_CLEAR_PHASE_CLEAR;
323     if (initiator_phase_transition_ack && clear_ack_i) be-
gin
324         initiator_state_d = POST_CLEAR;
325     end else if (initiator_phase_transition_ack) begin
326         initiator_state_d = WAIT_CLEAR_ACK;
327     end else if (clear_ack_i) begin
328         initiator_state_d = WAIT_CLEAR_PHASE_ACK;
329     end
330 end
331
332 WAIT_CLEAR_ACK: begin
333     initiator_isolate_out = 1'b1;
334     initiator_clear_out = 1'b1;
335     initiator_clear_seq_phase = CDC_CLEAR_PHASE_CLEAR;
336     if (clear_ack_i) begin
337         initiator_state_d = POST_CLEAR;
338     end
339 end
340
341 WAIT_CLEAR_PHASE_ACK: begin
342     initiator_phase_transition_req = 1'b1;
343     initiator_clear_seq_phase =
CDC_CLEAR_PHASE_CLEAR;
344     initiator_isolate_out = 1'b1;
345     initiator_clear_out = 1'b1;
346     if (initiator_phase_transition_ack) begin
347         initiator_state_d = POST_CLEAR;
348     end
349 end
350
351 POST_CLEAR: begin
352     initiator_isolate_out = 1'b1;
353     initiator_clear_out = 1'b0;
354     initiator_phase_transition_req = 1'b1;
355     initiator_clear_seq_phase =
CDC_CLEAR_PHASE_POST_CLEAR;
356     if (initiator_phase_transition_ack) begin
357         initiator_state_d = FINISHED;
358     end
359 end
360
361 FINISHED: begin
362     initiator_isolate_out = 1'b1;
363     initiator_clear_out = 1'b0;
364     initiator_phase_transition_req = 1'b1;
365     initiator_clear_seq_phase = CDC_CLEAR_PHASE_IDLE;
366     if (initiator_phase_transition_ack) begin
367         initiator_state_d = IDLE;
368     end
369 end
370
371 default: begin
372     initiator_state_d = ISOLATE;
373 end
374 endcase
375 end
376
377 // Start in the ISOLATE state if asynchronous reset should
launch a clear sequence.
378 `FF(initiator_state_q, initiator_state_d, InitiatorStateR-
esetValue, clk_i, rst_ni)
379
380 // Initiator CDC SRC
381
382 // We use 4 phase handshaking. That way it doesn't matter if
one side is
383 // suddenly reset asynchronously. With a 2phase CDC, one-
sided async resets might
384 // introduce spurious transactions.
385 cc_cdc_4phase_src #(
386     .data_t(cdc_clear_seq_phase_e),
387     .SyncStages(ResetSyncStages),
388     .Decoupled(0), // Important! The CDC must not be in de-
coupled mode.
389     // Otherwise we will proceed to the next
state without
390     // waiting for the new state to arrive on

```



```

the other side.
391   .SendResetMsg(ClearOnAsyncReset), // Send the ISOLATE
phase request immediately on async
392                                     // reset if async reset
synchronization is enabled.
393   .ResetMsg(CDC_CLEAR_PHASE_ISOLATE)
394 ) i_state_transition_cdc_src(
395   .clk_i,
396   .rst_ni,
397   .data_i(initiator_clear_seq_phase),
398   .valid_i(initiator_phase_transition_req),
399   .ready_o(initiator_phase_transition_ack),
400   .async_req_o,
401   .async_ack_i,
402   .async_data_o(async_next_phase_o)
403 );
404
405 //----- Receiver Side
406 //-----
407 // This part of the circuit receives clear sequence state
transitions from the
408 // other side.
409
410 cdc_clear_seq_phase_e receiver_phase_d, receiver_phase_q;
411 cdc_clear_seq_phase_e receiver_effective_phase;
412 cdc_clear_seq_phase_e receiver_next_phase;
413
414 logic receiver_phase_req;
415 logic receiver_phase_ack;
416 logic receiver_phase_pending_d, receiver_phase_pending_q;
417 logic receiver_phase_done_d, receiver_phase_done_q;
418
419 logic receiver_capture_phase;
420
421 logic receiver_isolate_out;
422 logic receiver_clear_out;
423
424 cc_cdc_4phase_dst #(
425   .data_t(cdc_clear_seq_phase_e),
426   .SyncStages(ResetSyncStages),
427   .Decoupled(0) // Important! The CDC must not be in de-
coupled mode. Otherwise
428
429   // we will proceed to the next state without
waiting for the
430   // new state to arrive on the other side.
431 ) i_state_transition_cdc_dst(
432   .clk_i,
433   .rst_ni,
434   .data_o(receiver_next_phase),
435   .valid_o(receiver_phase_req),
436   .ready_i(receiver_phase_ack),
437   .async_req_i,
438   .async_ack_o,
439   .async_data_i(async_next_phase_i)
440 );
441
442 // Capture async phase payload after synchronized request
443 // Do not decode receiver_next_phase into clear/isolate
assign receiver_capture_phase =
444   receiver_phase_req
445   && !receiver_phase_pending_q
446   && !receiver_phase_done_q;
447
448 always_comb begin
449   receiver_phase_d = receiver_phase_q;
450   receiver_phase_pending_d = receiver_phase_pending_q;
451   receiver_phase_done_d = receiver_phase_done_q;
452
453   if (!receiver_phase_req) begin
454     receiver_phase_done_d = 1'b0;
455   end
456
457   if (receiver_capture_phase) begin
458     receiver_phase_d = receiver_next_phase;
459     receiver_phase_pending_d = 1'b1;
460   end
461
462   if (receiver_phase_ack) begin
463     receiver_phase_pending_d = 1'b0;
464     receiver_phase_done_d = 1'b1;
465   end
466 end
467
468 `FF(receiver_phase_q, receiver_phase_d,
CDC_CLEAR_PHASE_IDLE, clk_i, rst_ni)
469 `FF(receiver_phase_pending_q, receiver_phase_pending_d,
1'b0, clk_i, rst_ni)
470 `FF(receiver_phase_done_q, receiver_phase_done_d,
1'b0, clk_i, rst_ni)
471
472 assign receiver_effective_phase = receiver_phase_q;
473
474 always_comb begin
475   receiver_isolate_out = 1'b0;
476   receiver_clear_out = 1'b0;
477   receiver_phase_ack = 1'b0;
478
479   unique case (receiver_effective_phase)
480     CDC_CLEAR_PHASE_IDLE: begin

```

```

481     receiver_clear_out = 1'b0;
482     receiver_isolate_out = 1'b0;
483     receiver_phase_ack = receiver_phase_req && receiv-
er_phase_pending_q;
484   end
485   CDC_CLEAR_PHASE_ISOLATE: begin
486     receiver_clear_out = 1'b0;
487     receiver_isolate_out = 1'b1;
488     receiver_phase_ack = receiver_phase_req && receiv-
er_phase_pending_q && isolate_ack_i;
489   end
490   CDC_CLEAR_PHASE_CLEAR: begin
491     receiver_clear_out = 1'b1;
492     receiver_isolate_out = 1'b1;
493     receiver_phase_ack = receiver_phase_req && receiv-
er_phase_pending_q && clear_ack_i;
494   end
495   CDC_CLEAR_PHASE_POST_CLEAR: begin
496     receiver_clear_out = 1'b0;
497     receiver_isolate_out = 1'b1;
498     receiver_phase_ack = receiver_phase_req && receiv-
er_phase_pending_q;
499   end
500   default: begin
501     receiver_clear_out = 1'b0;
502     receiver_isolate_out = 1'b0;
503     receiver_phase_ack = 1'b0;
504   end
505 endcase
506
507 // Preserve async-reset ordering margin without
async_next_phase_i decode
508 if (receiver_capture_phase) begin
509   receiver_isolate_out = 1'b1;
510 end
511
512 // Output Assignment
513
514 // The clear and isolate signal are the OR combination of
the receiver and
515 // initiator's clear/isolate signal. This ensures that the
correct sequence is
516 // followed even if both sides are cleared independently at
roughly the same
517 // time.
518 assign clear_o = initiator_clear_out || receiver_clear_out;
519 assign isolate_o = initiator_isolate_out || receiv-
er_isolate_out;
520
521
522 endmodule : cc_cdc_reset_ctrl_half

```

15. pulp-platform common_cells/src/cc_clk_int_div.sv (383 lines)

```

1 //-----
2 // Title : Configurable Integer Clock Divider
3 //-----
4 // File : cc_clk_int_div.sv Author : Manuel Eggimann <meggim-
mann@iis.ee.ethz.ch>
5 // Created : 17.03.2022
6 //-----
7 // Description :
8 //
9 // This module implements an at runtime configurable integer
clock divider that
10 // always generates clean 50% duty cycle output clock. Clock
divider setting
11 // changes are handshaked and during the transitioning phase
between cc_clk_div
12 // value changes, the output clock is gated to prevent clock
glitches and no
13 // other cc_clk_div change request is accepted. clk_o remains
gated for at most
14 // 3x<new clk period> clk_i cycles. If the new div_i value
equals the currently
15 // configured value, the clock is not gated and the handshake
is immediately
16 // granted. It is thus safe to statically tie the valid signal to
logic high if
17 // we can guarantee, that the div_i value remains stable long
enough (upper
18 // limit 2 output clock cycles).
19 //
20 // The `en_i` signal can be used to enable or disable the out-
put clock in a safe
21 // manner.
22 //
23 // If a div value of 0 or 1 is requested, the input clock is
fed through to the
24 // output clock. However, the same gating rules apply (again
to prevent
25 // glitches).
26 //

```

```

27 // If test_mode_en_i is asserted, the output clock gate is by-
passed entirely and
28 // clk_o will always be directly driven by clk_i. Use this
mode for DFT of the
29 // downstream logic.
30 //
31 // Parameters:
32 //
33 // DivValueWidth: The number of bits to use for the internal
34 // counter. Defines the maximum division factor.
35 //
36 // DefaultDivValue: The default division factor to use after
reset. Use this
37 // parameter and tie div_valid_i to zero if you don't need at
runtime
38 // configurability. An elaboration time error will be issued
if the supplied
39 // default div value is not representable with DivValueWidth
bits.
40 //

41 // EnableClockInReset: If 1'b1, the clock gate will be enabled
during reset
42 // which allows the cc_clk_int_div instance to bypass the
clock during reset, IFF
43 // the DefaultDivValue is 1. For all other DefaultDivValues,
the output
44 // clock will not be available until rst_ni is deasserted!
45 //
46 // IMPORTANT!!!
47 //
48 // All clock gating/logic within this design is performed by
dedicated clock logic
49 // tech cells. By default the common_cell library uses the be-
havioral models in

50 // the `tech_cells_generic` repository. However, for synthesis
these cells need to be
51 // mapped to dedicated cells from your standard cell library,
preferably ones
52 // that are designed for clock logic (they have balanced rise
and fall time).
53 // During synthesis you furthermore have to properly set
`dont_touch` or
54 // `size_only` attributes to prevent the logic synthesizer
from replacing those
55 // cells with regular logic gates which could end up being
glitchy!
56 //
57 //-----
58 // Copyright (C) 2022 ETH Zurich, University of Bologna Copy-
right and related
59 // rights are licensed under the Solderpad Hardware License,
Version 0.51 (the
60 // "License"); you may not use this file except in compliance
with the License.
61 // You may obtain a copy of the License at
62 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law or
63 // agreed to in writing, software, hardware and materials dis-
tributed under this
64 // License is distributed on an "AS IS" BASIS, WITHOUT WAR-
RANTIES OR CONDITIONS

65 // OF ANY KIND, either express or implied. See the License for
the specific
66 // language governing permissions and limitations under the
License.
67 // SPDX-License-Identifier: SHL-0.51
68 //
69 //-----
70 `include "common_cells/registers.svh"
71
72 module cc_clk_int_div #(
73     /// The with
74     parameter int unsigned DivValueWidth = 4,
75     /// The default divider value which is used right after re-
set
76     parameter int unsigned DefaultDivValue = 0,
77     /// If 1'b1, the output clock is enabled during async reset
assertion
78     parameter bit          EnableClockInReset = 1'b0
79 ) (
80     /// Clock.
81     input logic            clk_i,
82     /// Asynchronous reset active low.
83     input logic            rst_ni,
84     /// Active-high output clock enable. Controls a glitch-free
clock gate so the
85     /// enable signal may be driven by combinational logic
without introducing
86     /// glitches.
87     input logic            en_i,
88     /// If asserted (active-high) bypass the clock divider and
drive clk_o
89     /// directly with clk_i.
90     input logic            test_mode_en_i,
91     /// Divider select value. The output clock has a frequency

```

```

of f_clk_i/div_i.
92     /// For div_i == 0 or div_i == 1, the output clock has the
same frequency as
93     /// the input clock.
94     input logic [DivValueWidth-1:0] div_i,
95     /// Valid handshake signal. Must not combinationally depend
on `div_ready_o`.
96     /// Once asserted, the valid signal must not be deasserted
until it is
97     /// accepted with `div_ready_o`.
98     input logic            div_valid_i,
99     output logic           div_ready_o,

100     /// Generated output clock. Given a glitch free input clock,
the output clock
101     /// is guaranteed to be glitch free with 50% duty cycle, re-
gardless the timing
102     /// of reconfiguration requests or en_i de/assetion. During
the

103     /// reconfiguration, the output clock is gated with its next
falling edge and
104     /// remains gated (idle-low) for at least one period of the
new target output
105     /// period to filter out any glitches during the config
transition.
106     output logic           clk_o,
107     /// Current value of the internal cycle counter. Might be
usefull if you need
108     /// to do some phase shifting relative to the generated
clock.
109     output logic [DivValueWidth-1:0] cycl_count_o
110 );
111
112 if ($clog2(DefaultDivValue+1) > DivValueWidth) begin :
gen_elab_error
113     $error("Default divider value %0d is not representable
with the configured",
114           "div value width of %0d bits.",
115           DefaultDivValue, DivValueWidth);
116 end
117
118 // We have to preset the div_q register with a value larger
than one to avoid
119 // an infinite loop in the WAIT_END_PERIOD state. If the de-
fault state of the

120 // clock divider should be bypass, we thus always preset the
div_q with 1
121 // rather than 1 or zero.
122 localparam int unsigned DivResetValue = (DefaultDivValue !=
0)? DefaultDivValue: 1;
123
124 logic [DivValueWidth-1:0] div_i_normalized;
125 logic [DivValueWidth-1:0] div_d, div_q;
126 logic toggle_ffs_en;
127 logic t_ff1_d, t_ff1_q;
128 logic t_ff1_en;
129
130 logic t_ff2_d, t_ff2_q;
131 logic t_ff2_en;
132
133 logic [DivValueWidth-1:0] cycle_cntr_d, cycle_cntr_q;
134 logic cycle_counter_en;
135 logic clk_div_bypass_en_d, clk_div_bypass_en_q;
136 logic odd_clk;
137 logic even_clk;
138 logic generated_clock;
139 logic ungated_output_clock;
140
141 logic use_odd_division_d, use_odd_division_q;
142 logic gate_en_d, gate_en_q;
143 logic gate_is_open_q;
144 logic clear_cycle_counter;
145 logic clear_toggle_flops;
146
147 typedef enum logic[1:0] {IDLE, LOAD_DIV, WAIT_END_PERIOD}
clk_gate_state_e;
148 clk_gate_state_e clk_gate_state_d, clk_gate_state_q;
149

150 // Normalize the div_i value. div_i == 0 and div_i == 1 have
the same meaning
151 // but actually loading 0 causes issues in the FSM. We thus
always load 1 for
152 // both cases
153 assign div_i_normalized = (div_i != 0)? div_i : 1;
154
155 //----- Divider Load FSM -----
156 always_comb begin
157     div_d          = div_q;
158     div_ready_o    = 1'b0;
159     clk_div_bypass_en_d = clk_div_bypass_en_q;
160     use_odd_division_d = use_odd_division_q;
161     clk_gate_state_d = clk_gate_state_q;
162     cycle_counter_en = 1'b1;
163     clear_cycle_counter = 1'b0;
164     clear_toggle_flops = 1'b0;
165     toggle_ffs_en    = 1'b1;
166     gate_en_d        = 1'b0;
167

```



```

168
169     case (clk_gate_state_q)
170     IDLE: begin
171         gate_en_d = 1'b1;
172         toggle_ffs_en = 1'b1;
173         if (div_valid_i) begin
174             if (div_i_normalized == div_q) begin
175                 div_ready_o = 1'b1;
176             end else begin
177                 clk_gate_state_d = LOAD_DIV;
178                 gate_en_d = 1'b0;
179             end
180             // If we disabled the clock output, stop the cycle
counter and the
181             // toggle flip-flops at the start of the next period
to safe energy.
182         end else if (!en_i && gate_is_open_q == 1'b0) begin
183             cycle_counter_en = 1'b0;
184             toggle_ffs_en = 1'b0;
185         end
186     end
187
188     LOAD_DIV: begin
189         gate_en_d = 1'b0;
190         toggle_ffs_en = 1'b1;
191         // Wait until the ouptut clock is currently deasserted. This ensures
192         // that the clock gate disable (gate_en) was latched
and changing the
193         // div_q and the cycle_counter_q value cannot affect
the output clock
194         // any longer.
195         if ((gate_is_open_q == 1'b0) || clk_div_bypass_en_q)
begin
196             // Now clear the cycle counter and the toggle flip
flops to have a
197             // deterministic output phase (rising edge of output
clock always
198             // coincides with first rising edge of input clock
when cycl_count_o
199             // == 0).
200             toggle_ffs_en = 1'b0;
201             div_d = div_i_normalized;
202             div_ready_o = 1'b1;
203             clear_cycle_counter = 1'b1;
204             clear_toggle_flops = 1'b1;
205             use_odd_division_d = div_i_normalized[0];
206             clk_div_bypass_en_d = div_i_normalized == 1;
207             clk_gate_state_d = WAIT_END_PERIOD;
208         end
209     end
210
211     WAIT_END_PERIOD: begin
212         gate_en_d = 1'b0;
213         // Keep the toggle flip-flops disabled until we reach
idle state.
214         // Otherwise, the start state of the t-ffs depends on
the number of wait
215         // cycles which would yield different output clock
phase depending on
216         // the number of wait cycles.
217         toggle_ffs_en = 1'b0;
218         if (cycle_cntr_q == div_q - 1) begin
219             clk_gate_state_d = IDLE;
220         end
221     end
222
223     default: begin
224         clk_gate_state_d = IDLE;
225     end
226 endcase
227 end
228
229 localparam logic UseOddDivisionResetValue = DefaultDiv-
Value[0];
230 localparam logic ClkDivBypassEnResetValue = (DefaultDivValue <
2)? 1'b1: 1'b0;
231
232 `FF(use_odd_division_q, use_odd_division_d, UseOddDivi-
sionResetValue, clk_i, rst_ni)
233 `FF(clk_div_bypass_en_q, clk_div_bypass_en_d, ClkDivBy-
passEnResetValue, clk_i, rst_ni)
234 `FF(div_q, div_d, DivReset-
Value, clk_i, rst_ni)
235 `FF(clk_gate_state_q, clk_gate_state_d,
IDLE, clk_i, rst_ni)
236 `FF(gate_en_q, gate_en_d, EnableClockIn-
Reset, clk_i, rst_ni)
237
238 //----- Cycle Counter
239
240 // Cycle Counter
241 always_comb begin
242     cycle_cntr_d = cycle_cntr_q;
243     // Reset the counter if we load a new divider value.
244     if (clear_cycle_counter) begin
245         cycle_cntr_d = '0;
246     end else begin

```

```

247         if (cycle_counter_en) begin
248             // During normal operation, reset the counter whenever
it reaches
249             // <target_count>-1. In bypass mode (div == 0 or div
== 1) disable the
250             // counter to save power.
251             if (clk_div_bypass_en_q || (cycle_cntr_q == div_q-1))
begin
252                 cycle_cntr_d = '0;
253             end else begin
254                 cycle_cntr_d = cycle_cntr_q + 1;
255             end
256         end
257     end
258 end
259
260 `FF(cycle_cntr_q, cycle_cntr_d, '0, clk_i, rst_ni)
261
262 assign cycl_count_o = cycle_cntr_q;
263
264 //----- T-Flip-Flops
265
266 // These T-Flip-Flop intentionally use blocking assignments!
If we were to use
267 // non-blocking assignment like we normally do for flip-
flops, we would create
268 // a race condition when sampling data from the fast clock
domain into
269 // flip-flops clocked by t_ff1_q and t_ff2_q. To avoid this,
we use blocking assignments
270 // which is the recommended method according to:
271 // S. Sutherland and D. Mills,
272 // Verilog and System Verilog gotchas: 101 common coding er-
rors and how to
273 // avoid them. New York: Springer, 2007. page 64.
274
275 always_ff @(posedge clk_i, negedge rst_ni) begin
276     if (!rst_ni) begin
277         // verilog_lint: waive always-ff-non-blocking
278         t_ff1_q = '0; // Intentional blocking assignment! Do not
replace!
279     end else begin
280         if (t_ff1_en) begin
281             // verilog_lint: waive always-ff-non-blocking
282             t_ff1_q = t_ff1_d; // Intentional blocking assignment!
Do not replace!
283         end
284     end
285 end
286
287 // The second flip-flop is required for odd integer division
and needs to
288 // negative edge triggered.
289 always_ff @(negedge clk_i, negedge rst_ni) begin
290     if (!rst_ni) begin
291         // verilog_lint: waive always-ff-non-blocking
292         t_ff2_q = '0; // Intentional blocking assignment! Do not
replace!
293     end else begin
294         if (t_ff2_en) begin
295             // verilog_lint: waive always-ff-non-blocking
296             t_ff2_q = t_ff2_d; // Intentional blocking assignment!
Do not replace!
297         end
298     end
299 end
300
301 always_comb begin
302     if (clear_toggle_flops) begin
303         t_ff1_d = '0;
304         t_ff2_d = '0;
305     end else begin
306         t_ff1_d = t_ff1_en? !t_ff1_q: t_ff1_q;
307         t_ff2_d = t_ff2_en? !t_ff2_q: t_ff2_q;
308     end
309 end
310
311 //----- FSM to control T-FF enable and clk_div_bypass_en
312
313 always_comb begin
314     t_ff1_en = 1'b0;
315     t_ff2_en = 1'b0;
316     if (!clk_div_bypass_en_q && toggle_ffs_en) begin
317         if (use_odd_division_q) begin
318             t_ff1_en = (cycle_cntr_q == 0)? 1'b1: 1'b0;
319             t_ff2_en = (cycle_cntr_q == (div_q+1)/2)? 1'b1: 1'b0;
320         end else begin
321             t_ff1_en = (cycle_cntr_q == 0 || cycle_cntr_q ==
div_q/2)? 1'b1: 1'b0;
322         end
323     end
324 end

```

```

325 end
326
327 assign even_clk = t_ff1_q;
328
329 //----- Clock XOR for the odd division logic
330 tc_clk_xor2 i_odd_clk_xor (
331   .clk0_i ( t_ff1_q ),
332   .clk1_i ( t_ff2_q ),
333   .clk_o ( odd_clk )
334 );
335
336 //---- Clock MUX to select between odd and even div logic
337 tc_clk_mux2 i_clk_mux (
338   .clk0_i ( even_clk ),
339   .clk1_i ( odd_clk ),
340   .clk_sel_i ( use_odd_division_q ),
341   .clk_o ( generated_clock )
342 );
343
344 //----- clock mux to bypass clock if divide-
by-1
345 tc_clk_mux2 i_clk_bypass_mux (
346   .clk0_i ( generated_clock ),
347   .clk1_i ( clk_i ),
348   .clk_sel_i ( clk_div_bypass_en_q || test_mode_en_i ),
349   .clk_o ( ungated_output_clock )
350 );
351
352 //----- Clock gate logic
353 // During the transitioning phase, we gate the clock to pre-
vent clock glitches
354
355 // The gate_is_open_q signal is used by the FSM to determine
whether the
356 // gate_en signal has been latched by the clock gate cell,
i.e. if this signal
357 // is 1'b0, it means not only that the clock gate should be
disabled but also that
358 // clk_o is currently LOW and thus the enable signal has
been latched and the
359 // output clock will remain LOW until we enable the clock
again.
360 // The FSM needs to know this to not disable the T-FFs and
the cycle counter
361 // to early. Otherwise the clock might get stuck active high
or we deassert a
362 // clock to early
363 always_ff @(posedge ungated_output_clock, negedge rst_ni)
begin
364   if (!rst_ni) begin
365     gate_is_open_q <= 1'b0;
366   end else begin
367     gate_is_open_q <= gate_en_q & en_i;
368   end
369 end
370
371 tc_clk_gating #(
372   .IS_FUNCTIONAL(1) // The gate is required to prevent
glitches during
373   // transitioning. Target specific imple-
mentations must not
374   // remove it to save ICGs (e.g. in FP-
GAs).
375 ) i_clk_gate (
376   .clk_i ( ungated_output_clock ),
377   .en_i ( gate_en_q & en_i ),
378   .test_en_i ( test_mode_en_i ),
379   .clk_o
380 );
381
382
383 endmodule

```

16. pulp-platform_common_cells/src/cc_clk_int_div_static.sv (95 lines)

```

1 //-----
2 // Title : Static Integer Clock Divider
3 //-----
4 // File : cc_clk_int_div_static.sv
5 // Author : Manuel Eggmann <meggmann@iis.ee.ethz.ch>
6 // Created : 08.05.2023
7 //-----
8 // Description :
9 //
10 // This module implements a static arbitrary integer divider.
Static in this
11 // case means, the divider value is constant at elaboration
time (SV parameter).
12 // It supports arbitrary integer division with a guaranteed
50% duty cycle for
13 // odd and even division.

```

```

14
15 // Internally, this module is wrapper around the "at-runtime"
configurable
16 // `cc_clk_int_div` module. If you need to change the division
factor at-runtime you
17 // should directly use `cc_clk_int_div` instead. However if all
you need is a single
18 // division factor this module provides a convenience wrapper
for you with a
19 // simplified interface.
20 //
21 // The `en_i` signal can be used to enable or disable the out-
put clock in a safe
22 // manner (there is an internal, glitch-free clock gate).
23 //
24 // Parameters:
25 //
26 // DivValue: The integer value by which the clock shall be di-
vided. Must be
27 // non-zero integer smaller than 2^32-1.
28 //
29 // EnableClockInReset: If 1'b1, the clock gate will be enabled
during reset
30 // which allows the cc_clk_int_div instance to bypass the
clock during reset, IFF
31 // the DefaultDivValue is 1. For all other DefaultDivValues,
the output
32 // clock will not be available until rst_ni is deasserted!
33 //
34 // IMPORTANT!!!
35 //
36 // All clock gating/logic within this design is performed by
dedicated clock
37 // logic tech cells. By default the common_cell library uses
the behavioral
38 // models in the `tech_cells_generic` repository. However, for
synthesis these
39 // cells need to be mapped to dedicated cells from your stand-
ard cell library,
40 // preferably ones that are designed for clock logic (they
have balanced rise
41 // and fall time). During synthesis you furthermore have to
properly set
42 // `dont_touch` or `size_only` attributes to prevent the logic
synthesizer from
43 // replacing those cells with regular logic gates which could
end up being
44 // glitchy!
45 //
46 //-----
47 // Copyright (C) 2023 ETH Zurich, University of Bologna Copy-
right and related
48 // rights are licensed under the Solderpad Hardware License,
Version 0.51 (the
49 // "License"); you may not use this file except in compliance
with the License.
50 // You may obtain a copy of the License at
51 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law or
52 // agreed to in writing, software, hardware and materials dis-
tributed under this
53 // License is distributed on an "AS IS" BASIS, WITHOUT WAR-
RANTIES OR CONDITIONS
54 // OF ANY KIND, either express or implied. See the License for
the specific
55 // language governing permissions and limitations under the
License.
56 // SPDX-License-Identifier: SHL-0.51
57 //

```

```

58
59
60 module cc_clk_int_div_static #(
61   parameter int unsigned DivValue = 1,
62   parameter bit EnableClockInReset = 1'b1
63 ) (
64   input logic clk_i, // Clock
65   input logic rst_ni, // Asynchronous reset active
low
66   input logic en_i,
67   input logic test_mode_en_i,
68   output logic clk_o
69 );
70 if (DivValue == 0) begin : gen_elab_error
71   $error("DivValue must be strictly larger than 0.");
72 end
73
74 localparam int unsigned DivValueWidth = $clog2(DivValue+1);
75
76 logic [DivValueWidth-1:0] div_value;
77 assign div_value = DivValue;
78
79 cc_clk_int_div #(
80   .DivValueWidth ( DivValueWidth ),

```

```

81 .DefaultDivValue ( DivValue ),
82 .EnableClockInReset ( EnableClockInReset )
83 ) i_clk_int_div (
84   .clk_i,
85   .rst_ni,
86   .en_i,
87   .test_mode_en_i,
88   .div_i ( div_value ),
89   .div_valid_i ( 1'b0 ),
90   .div_ready_o ( ),
91   .clk_o,
92   .cycl_count_o ( )
93 );
94
95 endmodule

```

17. pulp-platform_common_cells/src/ cc_clk_mux_glitch_free.sv (291 lines)

```

1 //-----
2 // Title      : Glitch-free Clock Multiplexer
3 //-----
4 // File       : cc_clk_mux_glitch_free.sv
5 // Author     : Manuel Eggimann <meggimann@iis.ee.ethz.ch>
6 // Created    : 10.12.2022
7 //-----
8 // Description :
9 //
10 // This module allows glitch free clock multiplexing between N
11 // arbitrary input
12 // clock with completely unknown phase relations. The
13 // module will make
14 // sure to first synchronize the clock multiplexer signal to
15 // the relevant clock
16 // domains and ensures glitch free switching between the
17 // source clock and the
18 // new target clock by silencing the output at appropriate
19 // times. The clock
20 // signals themselves only pass through: 1 clock-gate, 1 N-in-
21 // put clock-OR Gate,
22 // 1 2-input clock mux. All these cells are referenced from
23 // the tech_cells
24 // repository and thus no conventional logic gate is directly
25 // in the clock path.
26 //
27 // The correctness of this module is based on the following
28 // assumptions:
29 // 1. The select signal stays stable for a duration of at
30 // least min(clks_i
31 // period)
32 // 2. Glitches on the select signal are shorter than
33 // min(clks_i) - t_setup
34 //
35 // 3. During a transition from clock input a to clock input b,
36 // both clocks have
37 // a stable period.
38 //
39 // A clock switching procedure from clock a to clock b has the
40 // following timing behavior:
41 // 1. After at most NumSyncStages clock cycle of clock a, the
42 // output clock is
43 // disabled with its next falling edge.
44 // 2. After clock cycle of clock a and another NumSyncStages
45 // clock cycles of clock b, the output is
46 // enabled with the next rising edge of clock B.
47 //
48 // So in total, an upper bound for the worst case clock
49 // switching delay is 2x
50 // NumSyncStages x max(clock_periods)
51 //
52 // The design has a parameter (ClockDuringReset) that allows
53 // the clock
54 // multiplexer to propagate the selected clock even during re-
55 // set assertion.
56 // However, during reset assertion the glitch filtering and
57 // the synchronization
58 // registers are bypassed (since they are frozen in reset
59 // state). Thus no glitch
60 // filtering is performed during reset. This is ok if the
61 // async_sel_i signal
62 // stays constant during reset assertion. Once you deassert
63 // the reset, regular
64 // glitch filtering and synchronization will kick in. However,
65 // you must wait for
66 // at least 1x max(input clock periods) before changing the
67 // async_sel_i after a
68 // reset to be sure the switch to regular operation has com-
69 // pleted. During the
70 // transition from async_reset operation to regular operation
71 // there will be a
72 // short phase where the clock is gated (similar to what hap-
73 // pens when you switch
74 // from one clock to the other).
75 //
76 //

```

```

48 // IMPORTANT!!!
49 //
50 // All clock gating/logic within this design is performed by
51 // dedicated clock
52 // logic tech cells. By default the common_cell library uses
53 // the behavioral
54 //
55 // models in the `tech_cells_generic` repository. However, for
56 // synthesis these
57 // cells need to be mapped to dedicated cells from your stand-
58 // ard cell library,
59 // preferably ones that are designed for clock logic (they
60 // have balanced rise
61 // and fall time). During synthesis you furthermore have to
62 // properly set
63 //
64 // `dont_touch` or `size_only` attributes to prevent the logic
65 // synthesizer from
66 // replacing those cells with regular logic gates which could
67 // end up being
68 // glitchy!
69 //-----
70 // Copyright (C) 2013-2022 ETH Zurich, University of Bologna
71 // Copyright and related rights are licensed under the Solder-
72 // pad Hardware
73 //
74 // License, Version 0.51 (the "License"); you may not use this
75 // file except in
76 // compliance with the License. You may obtain a copy of the
77 // License at
78 // http://solderpad.org/licenses/SHL-0.51. Unless required by
79 // applicable law
80 // or agreed to in writing, software, hardware and materials
81 // distributed under
82 // this license is distributed on an "AS IS" BASIS, WITHOUT
83 // WARRANTIES OR
84 // CONDITIONS OF ANY KIND, either express or implied. See the
85 // License for the
86 // specific language governing permissions and limitations un-
87 // der the License.
88 //-----
89
90 `include "common_cells/registers.svh"
91
92 module cc_clk_mux_glitch_free #(
93   parameter int unsigned NumInputs = 2,
94   parameter int unsigned NumSyncStages = 2,
95   parameter bit ClockDuringReset = 1'b1, //< If 1, allow the
96   selected clock to
97   //propagate even dur-
98   ing reset
99   //assertion.
100   localparam int unsigned SelWidth = $clog2(NumInputs)
101 ) (
102   input logic [NumInputs-1:0] clks_i,
103   input logic test_clk_i,
104   input logic test_en_i,
105   input logic async_rstn_i,
106   input logic [SelWidth-1:0] async_sel_i,
107   output logic clk_o
108 );
109
110 if (NumInputs < 2)
111   $error("Num inputs must be parametrized to a value >=
112 2.");
113
114 // For each input, we generate an enable signal that enables
115 // the clock
116 //
117 // propagation through an N-input clock OR gate. The crucial
118 // and most critical
119 // part is to make sure that these clock enable signal
120 // transitions are
121 // non-overlapping and have enough timing separation to pre-
122 // vent any glitches
123 //
124 // on the clock output during the transition. We ensure this as
125 // follows:
126 //
127 // 1. Decode the sel_i input to a onehot signal.
128 // 2. For each clock input, take the correspondi onehot sig-
129 // nal. For each clock
130 // input we also have a corresponding output clock enable
131 // signal that
132 // controls the corresponding clock's clk gate. We thus and-
133 // gate the one-hot
134 // signal of the current clock with the inverse of every
135 // other clocks enable
136 // signal. In other words, we only allow the propagation of
137 // the onehot enable
138 // signal if the clock is currently disabled.
139 // 3. Filter any glitches on this and-gated signal by
140 // passing it through a
141 // flip-flop clocked by the current clock and and-gating
142 // both the output and

```

```

110 // the input. I.e. the output is only becomes active if the
signal stays
111 // stable HIGH for at least one clock period.
112 // 4. Synchronize this glitch-filtered signal into the cur-
rent clock domain
113 // with an M-stage synchronizer.
114 // 5. Use this synchronized signal as the enable signal for
a glitch-free
115 // clock gate.
116 // 7. Feed the output of the clock gate to an N-input clock-
AND gate.
117 // 8. Latch the gate enable signal with an active low latch
before using the
118 // signal as a gating signal for the other clock input's
onehot signal.
119
120 // Internal signals
121 logic [NumInputs-1:0] s_sel_onehot;
122 (*dont_touch*)
123 (*async_reg*)
124 logic [NumInputs-1:0][1:0] glitch_filter_arr_d,
glitch_filter_arr_q;
125 logic [NumInputs-1:0] s_gate_enable_unfiltered_async;
126 logic [NumInputs-1:0] s_glitch_filter_output_async;
127 logic [NumInputs-1:0] s_gate_enable_sync;
128 logic [NumInputs-1:0] s_gate_enable;
129 logic [NumInputs-1:0] clock_has_been_disabled_arr_q;
130 logic [NumInputs-1:0] s_gated_clock;
131 logic s_output_clock;
132
133 logic [NumInputs-1:0] s_reset_synced;
134 logic [NumInputs-1:0] async_reset_bypass_active_q;
135
136 // Onehot decoder
137 always_comb begin
138   s_sel_onehot = '0;
139   s_sel_onehot[async_sel_i] = 1'b1;
140 end
141
142 // Input stages
143 for (genvar i = 0; i < NumInputs; i++) begin : gen_in-
put_stages
144   // Slice the glitch_filter and clock_has_been_disabled ar-
rays
145   // to avoid Verilator MULTIDRIVEN warnings.
146   (*dont_touch*)
147   (*async_reg*)
148   logic [1:0] glitch_filter_d, glitch_filter_q;
149   logic clock_has_been_disabled_q;
150   assign glitch_filter_arr_d[i] = glitch_filter_d;
151   assign glitch_filter_arr_q[i] = glitch_filter_q;
152   assign clock_has_been_disabled_arr_q[i] =
clock_has_been_disabled_q;
153
154   // Synchronize the reset into each clock domain
155   cc_rstgen i_rstgen(
156     .clk_i ( clk_i[i] ),
157     .rst_ni ( async_rstn_i ),
158     .test_mode_i ( test_en_i ),
159     .rst_no ( s_reset_synced[i] ),
160     .init_no ( )
161   );
162
163   // Gate onehot signal with other clocks' output gate en-
able
164   always_comb begin
165     s_gate_enable_unfiltered_async[i] = 1'b1;
166     for (int j = 0; j < NumInputs; j++) begin
167       if (i==j) begin
168         s_gate_enable_unfiltered_async[i] &= s_sel_one-
hot[j];
169       end else begin
170         s_gate_enable_unfiltered_async[i] &=
clock_has_been_disabled_arr_q[j];
171       end
172     end
173   end
174   assign glitch_filter_d[0] = s_g-
ate_enable_unfiltered_async[i];
175   assign glitch_filter_d[1] = glitch_filter_q[0];
176
177   // Filter HIGH-pulse glitches
178   `FF(glitch_filter_q, glitch_filter_d, '0, clk_i[i], s_re-
set_synced[i])
179   assign s_glitch_filter_output_async[i] =
glitch_filter_q[1] &
180   glitch_filter_q[0] &
181   s_g-
ate_enable_unfiltered_async[i];
182
183   // Synchronize to current clock
184   tc_sync #(Stages(NumSyncStages)) i_sync_en(
185     .clk_i ( clk_i[i] ),
186     .rst_ni ( s_reset_synced[i] ),
187     .serial_i ( s_glitch_filter_output_async[i] ),
188     .serial_o ( s_gate_enable_sync[i] )
189   );
190
191   // If the design is parametrized to propagate a clock dur-

```

```

ing asserted reset,
194 // we have to provide a bypass path that directly connects
the unfiltered
195 // gate enable signal to the clock gate for as long as the
reset is active.
196
197 if (ClockDuringReset) begin : gen_a-
sync_reset_clock_bypass_logic
198   always_ff @(posedge clk_i[i], negedge s_re-
set_synced[i]) begin
199     if (!s_reset_synced[i]) begin
200       async_reset_bypass_active_q[i] <= 1'b1;
201     end else begin
202       async_reset_bypass_active_q[i] <= 1'b0;
203     end
204   end
205
206   assign s_gate_enable[i] =
async_reset_bypass_active_q[i]?
207     s_g-
ate_enable_unfiltered_async[i]
208     : s_gate_enable_sync[i];
209 end else begin : gen_no_async_reset_bypass_logic
210   assign s_gate_enable[i] = s_gate_enable_sync[i];
211 end
212
213 // Gate the input clock with the synced enable signal
214 tc_clk_gating #(
215   .IS_FUNCTIONAL(1'b1)
216 ) i_clk_gate (
217   .clk_i ( clk_i[i] ),
218   .en_i ( s_gate_enable[i] ),
219   .test_en_i ( 1'b0 ),
220   .clk_o ( s_gated_clock[i] )
221 );
222
223 // Latch the enable signal with the next rising edge of
the input clock and
224 // feed the output back to the other stage's input. If we
were to directly
225 // use the clock gate enable signal to determine whether it
is save to enable
226 // another clock (i.e. the signal becomes low) we would
risk enabling the
227 // other clock too early. This is because the glitch free
clock gate will
228 // only really disable the clock with the next falling
edge. By delaying the
229 // enable signal one more cycle, we ensure that the clock
stays low for at
230 // least one clock period of the original clock input be-
fore any other clock
231 // even has the chance to become active.
232 `FF(clock_has_been_disabled_q, ~s_gate_enable[i], 1'b1,
clk_i[i], s_reset_synced[i])
233 end
234
235 // Output OR-gate. At this stage, we should be already sure
that the clocks
236 // are enabled/disabled at the proper time to prevent any
glitches from
237 // escaping.
238
239 cc_clk_or_tree #(NumInputs) i_clk_or_tree (
240   .clk_i(s_gated_clock),
241   .clk_o(s_output_clock)
242 );
243
244 // Mux between the regular muxed clock and the test_clk_i
used for DFT.
245 tc_clk_mux2 i_test_clk_mux(
246   .clk0_i(s_output_clock),
247   .clk1_i(test_clk_i),
248   .clk_sel_i(test_en_i),
249   .clk_o
250 );
251
252 endmodule
253
254 // Helper Module to generate an N-input clock OR-gate from a
tree of tc_clk_or2 cells.
255 module cc_clk_or_tree #(
256   parameter int unsigned NumInputs
257 ) (
258   input logic [NumInputs-1:0] clk_i,
259   output logic clk_o
260 );
261
262 if (NumInputs < 1) begin : gen_error
263   $error("Cannot parametrize clk_or with less than 1 input
but was %0d", NumInputs);
264 end else if (NumInputs == 1) begin : gen_leaf
265   assign clk_o = clk_i[0];
266 end else if (NumInputs == 2) begin : gen_leaf
267   tc_clk_or2 i_clk_or2 (
268     .clk0_i(clk_i[0]),
269     .clk1_i(clk_i[1]),
270     .clk_o

```

```

271 );
272 end else begin : gen_recursive
273   logic branch_a, branch_b;
274   cc_clk_or_tree #(NumInputs/2) i_or_branch_a (
275     .clks_i(clks_i[0+:NumInputs/2]),
276     .clk_o(branch_a)
277   );
278
279   cc_clk_or_tree #(NumInputs/2 + NumInputs%2) i_or_branch_b
280   (
281     .clks_i(clks_i[NumInputs-1:NumInputs/2]),
282     .clk_o(branch_b)
283   );
284   tc_clk_or2 i_clk_or2 (
285     .clk0_i(branch_a),
286     .clk1_i(branch_b),
287     .clk_o
288   );
289 end
290
291 endmodule

```

18. pulp-platform_common_cells/src/cc_counter.sv (43 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Author: Florian Zaruba
12 // Description: Generic up/down counter
13
14 module cc_counter #(
15   parameter int unsigned Width = 4,
16   parameter bit StickyOverflow = 1'b0
17 )(
18   input logic          clk_i,    // Clock
19   input logic          rst_ni,   // Asynchronous reset
20   active low
21   input logic          clr_i,    // Synchronous clear
22   active high
23   input logic          en_i,     // Enable the counter
24   input logic          load_i,   // Load a new value
25
26   input logic          down_i,   // Downcount, default is up
27   input logic [Width-1:0] d_i,
28   output logic [Width-1:0] q_o,
29   output logic         overflow_o
30 );
31
32 cc_delta_counter #(
33   .Width (Width),
34   .StickyOverflow (StickyOverflow)
35 ) i_counter (
36   .clk_i,
37   .rst_ni,
38   .clr_i,
39   .en_i,
40   .load_i,
41   .down_i,
42   .delta_i({Width-1{1'b0}}, 1'b1),
43   .d_i,
44   .q_o,
45   .overflow_o
46 );
47
48 endmodule

```

19. pulp-platform_common_cells/src/cc_credit_counter.sv (57 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
  details.
3 // SPDX-License-Identifier: SHL-0.51
4
5 // Author: Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
6 // Author: Paul Scheffler <paulsc@iis.ee.ethz.ch>
7
8 `include "common_cells/registers.svh"
9 `include "common_cells/assertions.svh"
10
11 module cc_credit_counter #(
12   parameter int unsigned NumCredits = 0,
13   /// Whether credit is full or empty on reset
14   parameter bit InitCreditEmpty = 1'b0,

```

```

15   /// Derived parameters *Do not override*
16   localparam int unsigned InitNumCredits = InitCreditEmpty ?
  '0 : NumCredits,
17   localparam type          credit_cnt_t = logic [$clo-
  g2(NumCredits):0]
18 ) (
19   input logic clk_i, // Clock
20   input logic rst_ni, // Asynchronous reset active low
21   input logic clr_i, // Synchronous clear active high; rein-
  itializes credit, takes priority
22
23   output credit_cnt_t credit_o,
24
25   input logic credit_give_i,
26   input logic credit_take_i,
27
28   output logic credit_left_o,
29   output logic credit_crit_o, // Giving one more credit will
  fill the credits
30   output logic credit_full_o
31 );
32
33 credit_cnt_t credit_d, credit_q;
34 logic increment, decrement;
35
36 assign decrement = credit_take_i & ~credit_give_i;
37 assign increment = ~credit_take_i & credit_give_i;
38
39 always_comb begin
40   credit_d = credit_q;
41   if (decrement) credit_d = credit_q - 1;
42   else if (increment) credit_d = credit_q + 1;
43 end
44
45 `FFARNC(credit_q, credit_d, clr_i, InitNumCredits, clk_i,
  rst_ni)
46
47 assign credit_o = credit_q;
48 assign credit_left_o = (credit_q != '0);
49 assign credit_crit_o = (credit_q == NumCredits-1);
50 assign credit_full_o = (credit_q == NumCredits);
51
52 `ifndef COMMON_CELLS_ASSERTS_OFF
53 `ASSERT_NEVER(CreditUnderflow, credit_o == '0 && decrement)
54 `ASSERT_NEVER(CreditOverflow, credit_o == NumCredits && in-
  crement)
55 `endif
56
57 endmodule

```

20. pulp-platform_common_cells/src/cc_delta_counter.sv (70 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Up/down counter with variable delta
12
13 `include "common_cells/registers.svh"
14
15 module cc_delta_counter #(
16   parameter int unsigned Width = 4,
17   parameter bit StickyOverflow = 1'b0
18 )(
19   input logic          clk_i,    // Clock
20   input logic          rst_ni,   // Asynchronous reset
21   active low
22   input logic          clr_i,    // Synchronous clear
23   active high
24   input logic          en_i,     // Enable the counter
25   input logic          load_i,   // Load a new value
26
27   input logic          down_i,   // Downcount, default is up
28   input logic [Width-1:0] delta_i,
29   input logic [Width-1:0] d_i,
30   output logic [Width-1:0] q_o,
31   output logic         overflow_o
32 );
33
34 logic [Width:0] counter_q, counter_d;
35 if (StickyOverflow) begin : gen_sticky_overflow
36   logic overflow_d, overflow_q;
37   logic overflow_clr;
38
39   assign overflow_clr = clr_i || load_i;

```



```

36 `FFARNC(overflow_q, overflow_d, overflow_clr, 1'b0,
clk_i, rst_ni)
37
38 always_comb begin
39     overflow_d = overflow_q;
40     if (!overflow_q && en_i) begin
41         if (down_i) begin
42             overflow_d = delta_i >
counter_q[Width-1:0];
43         end else begin
44             overflow_d = counter_q[Width-1:0] >
({Width{1'b1}} - delta_i);
45         end
46     end
47 end
48 assign overflow_o = overflow_q;
49 end else begin : gen_transient_overflow
50     // counter overflowed if the MSB is set
51     assign overflow_o = counter_q[Width];
52 end
53 assign q_o = counter_q[Width-1:0];
54
55 always_comb begin
56     counter_d = counter_q;
57
58     if (load_i) begin
59         counter_d = {1'b0, d_i};
60     end else if (en_i) begin
61         if (down_i) begin
62             counter_d = counter_q - delta_i;
63         end else begin
64             counter_d = counter_q + delta_i;
65         end
66     end
67 end
68
69 `FFARNC(counter_q, counter_d, clr_i, '0, clk_i, rst_ni)
70 endmodule

```

21. pulp-platform_common_cells/src/cc_ecc_decode.sv (129 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10 //
11 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
12 //
13 /// # ECC Decoder
14 ///
15 /// Implements SECDED (Single Error Correction, Double Error
Detection) Hamming Code
16 /// with extended parity bit [1].
17 /// The module receives a data word including parity bit and
decodes it according to the
18 /// number of data and parity bit.
19 ///
20 /// 1. If no error has been detected, the syndrome will be
zero and all flags will be zero.
21 /// 2. If a single error has been detected, the syndrome is
non-zero, and `single_error_o` will be
22 /// asserted. The output word contains the corrected data.
23 /// 3. If the parity bit contained an error, the module will
assert `parity_error_o`.
24 /// 4. In case of a double fault the syndrome is non-zero,
`double_error_o` will be asserted.
25 /// All other status flags will be de-asserted.
26 ///
27 /// [1] https://en.wikipedia.org/wiki/Hamming_code
28
29 module cc_ecc_decode import cc_pkg::*; #(
30     /// Data width of unencoded word.
31     parameter int unsigned DataWidth = 64,
32     /// Do not change
33     localparam type data_t = logic [DataWidth-1:0],
34     localparam type parity_t = logic [ecc_get_par-
ity_width(DataWidth)-1:0],
35     localparam type code_word_t = logic
[ecc_get_cw_width(DataWidth)-1:0],
36     localparam type encoded_data_t = struct packed {
37         logic parity;
38         code_word_t code_word;
39     }
40 ) (
41     /// Encoded data in

```

```

42 input encoded_data_t data_i,
43 /// Corrected data out
44 output data_t data_o,
45 /// Error syndrome indicates the erroneous bit position
46 output parity_t syndrome_o,
47 /// A single error occurred
48 output logic single_error_o,
49 /// Error received in parity bit (MSB)
50 output logic parity_error_o,
51 /// A double error occurred
52 output logic double_error_o
53 );
54
55 logic parity;
56 data_t data_wo_parity;
57 parity_t syndrome;
58 logic syndrome_not_zero;
59 code_word_t correct_data;
60
61 // Check parity bit. 0 = parity equal, 1 = different parity
62 assign parity = data_i.parity ^ (^data_i.code_word);
63
64 ///! | 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
65 ///! | p1 p2 d1 p4 d2 d3 d4 p8 d5 d6 d7 d8 d9 d10 d11
66 ///! ---|-----
67 ///! p1 | x x x x x x x x x x x x x
68 ///! p2 | x x x x x x x x x x x x x
69 ///! p4 | x x x x x x x x x x x x x
70 ///! p8 | x x x x x x x x x x x x x
71
72 ///! 1. Parity bit 1 covers all bit positions which have the
least significant bit
73 ///! set: bit 1 (the parity bit itself), 3, 5, 7, 9, etc.
74
75 ///! 2. Parity bit 2 covers all bit positions which have the
second least
76 ///! significant bit set: bit 2 (the parity bit itself),
3, 6, 7, 10, 11, etc.
77
78 ///! 3. Parity bit 4 covers all bit positions which have the
third least
79 ///! significant bit set: bits 4-7, 12-15, 20-23, etc.
80
81 ///! 4. Parity bit 8 covers all bit positions which have the
fourth least
82 ///! significant bit set: bits 8-15, 24-31, 40-47, etc.
83
84 ///! 5. In general each parity bit covers all bits where the
bitwise AND of the
85 ///! parity position and the bit position is non-zero.
86 always_comb begin : calculate_syndrome
87     syndrome = 0;
88     for (int unsigned i = 0; i < unsigned('$bits(parity_t));
i++) begin
89         for (int unsigned j = 0; j < un-
signed('$bits(code_word_t)); j++) begin
90             if ((unsigned'(2**i) & (j + 1))) syndrome[i] = syn-
drome[i] ^ data_i.code_word[j];
91         end
92     end
93 end
94
95 assign syndrome_not_zero = |syndrome;
96 assign syndrome_o = syndrome;
97
98 // correct the data word if the syndrome is non-zero
99 always_comb begin
100     correct_data = data_i.code_word;
101     if (syndrome_not_zero) begin
102         correct_data[syndrome - 1] = ~data_i.code_word[syndrome
- 1];
103     end
104 end
105
106 ///! Syndrome | Overall Parity (MSB) | Error Type | Notes
107 ///!
108 ///! 0 | 0 | No Error |
109 ///! !=0 | 1 | Single Error | Cor-
rectable. Syndrome holds incorrect bit position.
110 ///! 0 | 1 | Parity Error | Over-
all parity, MSB is in error and can be corrected.
111 ///! !=0 | 0 | Double Error | Not
correctable.
112 assign single_error_o = parity & syndrome_not_zero;
113 assign parity_error_o = parity & ~syndrome_not_zero;
114 assign double_error_o = ~parity & syndrome_not_zero;
115
116 // Extract data vector
117 always_comb begin
118     automatic int unsigned idx; // bit index
119     data_wo_parity = '0;
120     idx = 0;
121
122 for (int unsigned i = 1; i < unsigned('$bits(code_word_t)) +
1; i++) begin
123     // if i is a power of two we are indexing a parity bit
124     if (unsigned'(2**$clog2(i)) != i) begin
125         data_wo_parity[idx] = correct_data[i - 1];
126     end
127     idx++;
128 end

```

```

122         idx++;
123     end
124 end
125 end
126
127 assign data_o = data_wo_parity;
128
129 endmodule

```

22. pulp-platform_common_cells/src/cc_ecc_encode.sv (78 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware

3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the license.
10 //
11 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
12 /// # ECC Encoder
13 ///
14 /// Implements SECDED (Single Error Correction, Double Error
  Detection) Hamming Code
15 /// with extended parity bit [1].

16 /// The module receives a data word and encodes it using above
  mentioned error
17 /// detection and correction code. The corresponding decode
  module
18 /// can be found in `cc_ecc_decode.sv`
19 ///
20 /// [1] https://en.wikipedia.org/wiki/Hamming_code
21
22 module cc_ecc_encode import cc_pkg::*; #(
23     /// Data width of unencoded word.
24     parameter int unsigned DataWidth = 64,
25     /// Do not change
26     localparam type data_t = logic [DataWidth-1:0],
27     localparam type parity_t = logic [ecc_get_par-
  ity_width(DataWidth)-1:0],
28     localparam type code_word_t = logic
  [ecc_get_cw_width(DataWidth)-1:0],
29     localparam type encoded_data_t = struct packed {
30         logic parity;
31         code_word_t code_word;
32     }
33 ) (
34     /// Unencoded data in
35     input data_t data_i,
36     /// Encoded data out
37     output encoded_data_t data_o
38 );
39
40 parity_t parity_code_word;
41 code_word_t data, codeword;
42
43 // Expand incoming data to codeword width
44 always_comb begin : expand_data
45     automatic int unsigned idx;
46     data = '0;
47     idx = 0;
48
49 for (int unsigned i = 1; i < unsigned'($bits(code_word_t)) +
  1; i++) begin
50     // if it is not a power of two word it is a normal data
51     if (unsigned'(2**$clog2(i)) != i) begin
52         data[i - 1] = data_i[idx];
53         idx++;
54     end
55 end
56
57 // calculate code word
58 always_comb begin : calculate_syndrome
59     parity_code_word = 0;
60     for (int unsigned i = 0; i < unsigned'($bits(parity_t));
  i++) begin
61         for (int unsigned j = 1; j < un-
  signed'($bits(code_word_t)) + 1; j++) begin
62             if (((unsigned'(2**i) & j)) parity_code_word[i] = par-
  ity_code_word[i] ^ data[j - 1];
63         end
64     end
65 end
66
67 // fuse the final codeword

```

```

68 always_comb begin : generate_codeword
69     codeword = data;
70
71 for (int unsigned i = 0; i < unsigned'($bits(parity_t)); i+
  +) begin
72         codeword[2**i-1] = parity_code_word[i];
73     end
74
75 assign data_o.code_word = codeword;
76 assign data_o.parity = ^codeword;
77
78 endmodule

```

23. pulp-platform_common_cells/src/cc_edge_detect.sv (31 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware

3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10 //
11 // Author: Florian Zaruba
12 // Description: Edge detector, clock needs to oversample for
  proper edge detection
13
14 module cc_edge_detect (
15     input logic clk_i, // Clock
16     input logic rst_ni, // Asynchronous reset active low
17     input logic d_i, // Data stream in
18     output logic re_o, // Rising edge detected
19     output logic fe_o // Falling edge detected
20 );
21
22 cc_sync_wedge i_sync_wedge (
23     .clk_i ( clk_i ),
24     .rst_ni ( rst_ni ),
25     .serial_i ( d_i ),
26     .r_edge_o ( re_o ),
27     .f_edge_o ( fe_o ),
28     .serial_o ( )
29 );
30
31 endmodule

```

24. pulp-platform_common_cells/src/cc_edge_propagator.sv (32 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware

3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10 //
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator (
14     input logic clk_tx_i,
15     input logic rst_tx_ni,
16     input logic edge_i,
17     input logic clk_rx_i,
18     input logic rst_rx_ni,
19     output logic edge_o
20 );
21
22 cc_edge_propagator_ack i_edge_propagator_ack (
23     .clk_tx_i,
24     .rst_tx_ni,
25     .edge_i,
26     .ack_tx_o (* unused *),
27     .clk_rx_i,
28     .rst_rx_ni,

```



```

29     .edge_o
30 );
31
32 endmodule

```

25. pulp-platform_common_cells/src/cc_edge_propagator_ack.sv (52 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator_ack (
14     input  logic clk_tx_i,
15     input  logic rst_tx_ni,
16     input  logic edge_i,
17     output logic ack_tx_o,
18     input  logic clk_rx_i,
19     input  logic rst_rx_ni,
20     output logic edge_o
21 );
22
23     logic [1:0] sync_a;
24     logic      sync_b;
25
26     logic r_input_reg;
27     logic s_input_reg_next;
28
29     assign ack_tx_o = sync_a[0];
30
31     assign s_input_reg_next = edge_i | (r_input_reg &
  ~sync_a[0]);
32
33     always @(negedge rst_tx_ni or posedge clk_tx_i) begin
34         if (~rst_tx_ni) begin
35             r_input_reg <= 1'b0;
36             sync_a <= 2'b00;
37         end else begin
38             r_input_reg <= s_input_reg_next;
39             sync_a <= {sync_b, sync_a[1]};
40         end
41     end
42
43     cc_sync_wedge u_sync_clkb (
44         .clk_i ( clk_rx_i ),
45         .rst_ni ( rst_rx_ni ),
46         .serial_i ( r_input_reg ),
47         .r_edge_o ( edge_o ),
48         .f_edge_o (
49             .serial_o ( sync_b
50         );
51
52 endmodule

```

26. pulp-platform_common_cells/src/cc_edge_propagator_rx.sv (30 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator_rx (
14     input  logic clk_i,
15     input  logic rst_ni,
16     input  logic valid_i,

```

```

17     output logic ack_o,
18     output logic valid_o
19 );
20
21     cc_sync_wedge i_sync_clkb (
22         .clk_i ( clk_i ),
23         .rst_ni ( rst_ni ),
24         .serial_i ( valid_i ),
25         .r_edge_o ( valid_o ),
26         .f_edge_o (
27             .serial_o ( ack_o
28         );
29
30 endmodule

```

27. pulp-platform_common_cells/src/cc_edge_propagator_tx.sv (40 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Antonio Pullini <pullinia@iis.ee.ethz.ch>
12
13 module cc_edge_propagator_tx (
14     input  logic clk_i,
15     input  logic rst_ni,
16     input  logic valid_i,
17     input  logic ack_i,
18     output logic valid_o
19 );
20
21     logic [1:0] sync_a;
22
23     logic r_input_reg;
24     logic s_input_reg_next;
25
26     assign s_input_reg_next = valid_i | (r_input_reg &
  ~sync_a[0]);
27
28     always @(negedge rst_ni or posedge clk_i) begin
29         if (~rst_ni) begin
30             r_input_reg <= 1'b0;
31             sync_a <= 2'b00;
32         end else begin
33             r_input_reg <= s_input_reg_next;
34             sync_a <= {ack_i, sync_a[1]};
35         end
36     end
37
38     assign valid_o = r_input_reg;
39
40 endmodule

```

28. pulp-platform_common_cells/src/cc_exp_backoff.sv (90 lines)

```

1 // Copyright 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // Author: Michael Schaffner <schaffner@iis.ee.ethz.ch>, ETH
  Zurich
12 // Date: 10.04.2019
13 // Description: exponential backoff counter with randomiza-
  tion.
14
15 // For each failed trial (set_cnt_i pulsed), this unit expo-
  nentially increases the

```

```

16 // (average) backoff time by masking an LFSR with a shifted
mask in order to
17 // create the backoff counter initial value.
18 //
19 // The shift register mask and the counter value are both re-
set to '0 in case of
20 // a successful trial (clr_cnt_i).
21 //
22
23 `include "common_cells/assertions.svh"
24 `include "common_cells/registers.svh"
25
26 module cc_exp_backoff #(
27     /// Seed for 16bit LFSR
28     parameter int unsigned Seed = 'hffff,
29     /// 2*MaxExp-1 determines the maximum range from which ran-
dom wait counts are drawn
30     parameter int unsigned MaxExp = 16
31 ) (
32     /// Clock
33     input logic clk_i,
34     /// Asynchronous reset active low
35     input logic rst_ni,
36     /// Synchronous clear active high
37     input logic clr_i,
38     /// Sets the backoff counter (pulse) -> use when trial did
not succeed
39     input logic set_cnt_i,
40     /// Clears the backoff counter (pulse) -> use when trial
succeeded
41     input logic clr_cnt_i,
42     /// Indicates whether the backoff counter is equal to zero
and a new trial can be launched
43     output logic is_zero_o
44 );
45
46 // leave this constant
47 localparam int unsigned WIDTH = 16;
48
49 logic [WIDTH-1:0] lfsr_d, lfsr_q, cnt_d, cnt_q, mask_d,
mask_q;
50 logic lfsr;
51
52 // generate random wait counts
53 // note: we use a flipped lfsr here to

```

```

54 // avoid strange correlation effects between
55 // the (left-shifted) mask and the lfsr
56 assign lfsr = lfsr_q[15:15] ^
57             lfsr_q[15:13] ^
58             lfsr_q[15:12] ^
59             lfsr_q[15:10];
60
61 assign lfsr_d = (set_cnt_i) ? {lfsr, lfsr_q[$high(lfsr_q):
1]} :
62             lfsr_q;
63
64 // mask the wait counts with exponentially increasing mask
(shift reg)
65 assign mask_d = (clr_cnt_i) ?
66             (set_cnt_i) ? {{(WIDTH-MaxExp)
{1'b0}},mask_q[MaxExp-2:0], 1'b1} :
67             mask_q;
68
69 assign cnt_d = (clr_cnt_i) ? '0 :
70             (set_cnt_i) ? (mask_q & lfsr_q) :
71             (!is_zero_o) ? cnt_q - 1'b1 : '0;
72
73 assign is_zero_o = (cnt_q=='0);
74
75 `FFARNC(lfsr_q, lfsr_d, clr_i, WIDTH'(Seed), clk_i, rst_ni)
76 `FFARNC(mask_q, mask_d, clr_i, '0, clk_i, rst_ni)
77 `FFARNC(cnt_q, cnt_d, clr_i, '0, clk_i, rst_ni)
78
79 // assertions
80 // assertions
81 // assertions
82
83 `ifndef COMMON_CELLS_ASSERTS_OFF
84 // assert wrong parameterizations
85 `ASSERT_INIT(max_exp_0, MaxExp>0, "MaxExp must be greater
than 0")
86 `ASSERT_INIT(max_exp_gt_16, MaxExp<=16, "MaxExp cannot be
greater than 16")
87 `ASSERT_INIT(seed_0, Seed>0, "Zero seed is not allowed for
LFSR")
88 `endif
89
90 endmodule // cc_exp_backoff

```

AXI4 상호연결

13 files • 4,754 lines

API — axi_atop_filter.sv

Table 955. pulp-platform_axi/src/axi_atop_filter.sv — module / interface / package (2개)

이름	종류
axi_atop_filter	module
axi_atop_filter_intf	module

Table 956. pulp-platform_axi/src/axi_atop_filter.sv — parameter (9개)

파라미터	형/폭	기본값
AxiIdWidth	int unsigned	0
AxiMaxWriteTxns	int unsigned	0
axi_req_t		logic
axi_resp_t		logic
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
AXI_MAX_WRITE_TXNS	int unsigned	0

Table 957. pulp-platform_axi/src/axi_atop_filter.sv — localparam (1개)

상수	형/폭	값
COUNTER_WIDTH	int unsigned	(AxiMaxWriteTxns == 1) ? 2 : \$clog2(AxiMaxWriteTxns+1)

Table 958. pulp-platform_axi/src/axi_atop_filter.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic

Table 959. pulp-platform_axi/src/axi_atop_filter.sv — typedef / using (8개)

새 이름	원래 형
r_state_e	enum logic [1:0] { R_RESET, R_FEEDTHROUGH, INJECT_R, R_HOLD }
id_t	logic [AxIdWidth-1:0]
len_t	logic [7:0]
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]

API — axi_burst_splitter.sv

Table 960. pulp-platform_axi/src/axi_burst_splitter.sv — module / interface / package (1개)

이름	종류
axi_burst_splitter	module

Table 961. pulp-platform_axi/src/axi_burst_splitter.sv — parameter (10개)

파라미터	형/폭	기본값
MaxReadTxns	int unsigned	32'd0
MaxWriteTxns	int unsigned	32'd0
FullBW	bit	0
AddrWidth	int unsigned	32'd0
DataWidth	int unsigned	32'd0
IdWidth	int unsigned	32'd0
AxiLookBits	int unsigned	IdWidth
UserWidth	int unsigned	32'd0
axi_req_t		logic
axi_resp_t		logic

Table 962. pulp-platform_axi/src/axi_burst_splitter.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t

Table 963. pulp-platform_axi/src/axi_burst_splitter.sv — typedef / using (5개)

새 이름	원래 형
addr_t	logic [AddrWidth-1:0]
data_t	logic [DataWidth-1:0]
strb_t	logic [DataWidth/8-1:0]
id_t	logic [IdWidth-1:0]
user_t	logic [UserWidth-1:0]

API — axi_burst_splitter_gran.sv

Table 964. pulp-platform_axi/src/axi_burst_splitter_gran.sv — module / interface / package (3개)

이름	종류
axi_burst_splitter_gran	module
axi_burst_splitter_gran_ax_chan	module
axi_burst_splitter_gran_counters	module

Table 965. pulp-platform_axi/src/axi_burst_splitter_gran.sv — parameter (29개)

파라미터	형/폭	기본값
MaxReadTxns	int unsigned	32'd0
MaxWriteTxns	int unsigned	32'd0
FullBW	bit	1'b0
CutPath	bit	1'b0
DisableChecks	bit	1'b0
AddrWidth	int unsigned	32'd0
DataWidth	int unsigned	32'd0
IdWidth	int unsigned	32'd0
AxiLookBits	int unsigned	IdWidth
UserWidth	int unsigned	32'd0
axi_req_t		logic
axi_resp_t		logic
axi_aw_chan_t		logic
axi_w_chan_t		logic
axi_b_chan_t		logic
axi_ar_chan_t		logic
axi_r_chan_t		logic
chan_t		logic
IdWidth	int unsigned	32'd0
MaxTxns	int unsigned	32'd0
CutPath	bit	1'b0

파라미터	형/폭	기본값
FullBW	bit	1'b0
id_t		logic [IdWidth-1:0]
MaxTxns	int unsigned	32'd0
IdWidth	int unsigned	32'd0
CutPath	bit	1'b0
FullBW	bit	1'b0
id_t		logic [IdWidth-1:0]
cnt_t		logic [axi_pkg::LenWidth:0]

Table 966. pulp-platform_axi/src/axi_burst_splitter_gran.sv — localparam (2개)

상수	형/폭	값
MaxTxns	int unsigned	(MaxReadTxns > MaxWriteTxns) ? MaxReadTxns : MaxWriteTxns
CntIdxWidth	int unsigned	(MaxTxns > 1) ? \$clog2(MaxTxns) : 32'd1

Table 967. pulp-platform_axi/src/axi_burst_splitter_gran.sv — 포트 (37개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
len_limit_i	input	axi_pkg::len_t
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic
len_limit_i	input	axi_pkg::len_t
ax_i	input	chan_t
ax_valid_i	input	logic
ax_ready_o	output	logic
ax_o	output	chan_t
ax_valid_o	output	logic
ax_ready_i	input	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t
cnt_set_err_i	input	logic
cnt_err_o	output	logic

포트	방향	형/폭
cnt_dec_i	input	logic
cnt_req_i	input	logic
cnt_gnt_o	output	logic
clk_i	input	logic
rst_ni	input	logic
alloc_id_i	input	id_t
alloc_len_i	input	axi_pkg::len_t
alloc_req_i	input	logic
alloc_gnt_o	output	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t
cnt_set_err_i	input	logic
cnt_err_o	output	logic
cnt_dec_i	input	logic
cnt_delta_i	input	cnt_t
cnt_req_i	input	logic
cnt_gnt_o	output	logic

Table 968. pulp-platform_axi/src/axi_burst_splitter_gran.sv — typedef / using (3개)

새 이름	원래 형
cnt_id_t	logic[IdWidth-1:0]
num_beats_t	logic[axi_pkg::LenWidth:0]
cnt_idx_t	logic [CntIdxWidth-1:0]

API — axi_burst_unwrap.sv

Table 969. pulp-platform_axi/src/axi_burst_unwrap.sv — module / interface / package (3개)

이름	종류
axi_burst_unwrap	module
axi_burst_unwrap_ax_chan	module
axi_burst_counters	module

Table 970. pulp-platform_axi/src/axi_burst_unwrap.sv — parameter (18개)

파라미터	형/폭	기본값
MaxReadTxns	int unsigned	32'd0
MaxWriteTxns	int unsigned	32'd0
AddrWidth	int unsigned	32'd0

파라미터	형/폭	기본값
DataWidth	int unsigned	32'd0
IdWidth	int unsigned	32'd0
AxiLookBits	int unsigned	IdWidth
UserWidth	int unsigned	32'd0
axi_req_t		logic
axi_resp_t		logic
AwChan	bit	0
chan_t		logic
AddrWidth	int unsigned	0
IdWidth	int unsigned	0
MaxTxns	int unsigned	0
id_t		logic[IdWidth-1:0]
MaxTxns	int unsigned	0
IdWidth	int unsigned	0
id_t		logic [IdWidth-1:0]

Table 971. pulp-platform_axi/src/axi_burst_unwrap.sv — localparam (2개)

상수	형/폭	값
MaxTxns	int unsigned	(MaxReadTxns > MaxWriteTxns) ? MaxReadTxns : MaxWriteTxns
CntIdxWidth	int unsigned	(MaxTxns > 1) ? \$clog2(MaxTxns) : 32'd1

Table 972. pulp-platform_axi/src/axi_burst_unwrap.sv — 포트 (34개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic
ax_i	input	chan_t
ax_valid_i	input	logic
ax_ready_o	output	logic
ax_o	output	chan_t
ax_valid_o	output	logic

포트	방향	형/폭
ax_ready_i	input	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t [1:0]
cnt_set_err_i	input	logic
cnt_err_o	output	logic
cnt_dec_i	input	logic [1:0]
cnt_req_i	input	logic [1:0]
cnt_gnt_o	output	logic [1:0]
clk_i	input	logic
rst_ni	input	logic
alloc_id_i	input	id_t
alloc_len_i	input	axi_pkg::len_t
alloc_req_i	input	logic
alloc_gnt_o	output	logic
cnt_id_i	input	id_t
cnt_len_o	output	axi_pkg::len_t
cnt_set_err_i	input	logic
cnt_err_o	output	logic
cnt_dec_i	input	logic
cnt_req_i	input	logic
cnt_gnt_o	output	logic

Table 973. `pulp-platform_axi/src/axi_burst_unwrap.sv` — typedef / using (8개)

새 이름	원래 형
addr_t	logic [AddrWidth-1:0]
data_t	logic [DataWidth-1:0]
id_t	logic [IdWidth-1:0]
strb_t	logic [DataWidth/8-1:0]
user_t	logic [UserWidth-1:0]
cnt_id_t	logic [IdWidth-1:0]
cnt_idx_t	logic [CntIdxWidth-1:0]
cnt_t	logic [\$bits(axi_pkg::len_t):0]

API — axi_bus_compare.svTable 974. `pulp-platform_axi/src/axi_bus_compare.sv` — module / interface / package (1개)

이름	종류
axi_bus_compare	module

Table 975. pulp-platform_axi/src/axi_bus_compare.sv — parameter (12개)

파라미터	형/폭	기본값
AxiIdWidth	int unsigned	32'd0
FifoDepth	int unsigned	32'd0
UseSize	bit	1'b0
DataWidth	int unsigned	32'd8
axi_aw_chan_t		logic
axi_w_chan_t		logic
axi_b_chan_t		logic
axi_ar_chan_t		logic
axi_r_chan_t		logic
axi_req_t		logic
axi_rsp_t		logic
id_t		logic [2**AxiIdWidth-1:0]

Table 976. pulp-platform_axi/src/axi_bus_compare.sv — 포트 (17개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
axi_a_req_i	input	axi_req_t
axi_a_rsp_o	output	axi_rsp_t
axi_a_req_o	output	axi_req_t
axi_a_rsp_i	input	axi_rsp_t
axi_b_req_i	input	axi_req_t
axi_b_rsp_o	output	axi_rsp_t
axi_b_req_o	output	axi_req_t
axi_b_rsp_i	input	axi_rsp_t
aw_mismatch_o	output	id_t
w_mismatch_o	output	logic
b_mismatch_o	output	id_t
ar_mismatch_o	output	id_t
r_mismatch_o	output	id_t
mismatch_o	output	logic
busy_o	output	logic

API — axi_cdc.sv**Table 977. pulp-platform_axi/src/axi_cdc.sv — module / interface / package (3개)**

이름	종류
axi_cdc	module
axi_cdc_intf	module
axi_lite_cdc_intf	module

Table 978. pulp-platform_axi/src/axi_cdc.sv — parameter (19개)

파라미터	형/폭	기본값
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
LogDepth	int unsigned	1
SyncStages	int unsigned	2
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2

Table 979. pulp-platform_axi/src/axi_cdc.sv — 포트 (16개)

포트	방향	형/폭
src_clk_i	input	logic
src_rst_ni	input	logic
src_req_i	input	axi_req_t
src_resp_o	output	axi_resp_t
dst_clk_i	input	logic
dst_rst_ni	input	logic

포트	방향	형/폭
dst_req_o	output	axi_req_t
dst_resp_i	input	axi_resp_t
src_clk_i	input	logic
src_rst_ni	input	logic
dst_clk_i	input	logic
dst_rst_ni	input	logic
src_clk_i	input	logic
src_rst_ni	input	logic
dst_clk_i	input	logic
dst_rst_ni	input	logic

Table 980. pulp-platform_axi/src/axi_cdc.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]

API — axi_cdc_dst.sv

Table 981. pulp-platform_axi/src/axi_cdc_dst.sv — module / interface / package (3개)

이름	종류
axi_cdc_dst	module
axi_cdc_dst_intf	module
axi_lite_cdc_dst_intf	module

Table 982. pulp-platform_axi/src/axi_cdc_dst.sv — parameter (19개)

파라미터	형/폭	기본값
LogDepth	int unsigned	1
SyncStages	int unsigned	2
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic

파라미터	형/폭	기본값
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2

Table 983. `pulp-platform_axi/src/axi_cdc_dst.sv` — 포트 (23개)

포트	방향	형/폭
async_data_slave_aw_data_i	input	aw_chan_t [2**LogDepth-1:0]
async_data_slave_aw_wptr_i	input	logic [LogDepth:0]
async_data_slave_aw_rptr_o	output	logic [LogDepth:0]
async_data_slave_w_data_i	input	w_chan_t [2**LogDepth-1:0]
async_data_slave_w_wptr_i	input	logic [LogDepth:0]
async_data_slave_w_rptr_o	output	logic [LogDepth:0]
async_data_slave_b_data_o	output	b_chan_t [2**LogDepth-1:0]
async_data_slave_b_wptr_o	output	logic [LogDepth:0]
async_data_slave_b_rptr_i	input	logic [LogDepth:0]
async_data_slave_ar_data_i	input	ar_chan_t [2**LogDepth-1:0]
async_data_slave_ar_wptr_i	input	logic [LogDepth:0]
async_data_slave_ar_rptr_o	output	logic [LogDepth:0]
async_data_slave_r_data_o	output	r_chan_t [2**LogDepth-1:0]
async_data_slave_r_wptr_o	output	logic [LogDepth:0]
async_data_slave_r_rptr_i	input	logic [LogDepth:0]
dst_clk_i	input	logic
dst_rst_ni	input	logic
dst_req_o	output	axi_req_t

포트	방향	형/폭
dst_resp_i	input	axi_resp_t
dst_clk_i	input	logic
dst_rst_ni	input	logic
dst_clk_i	input	logic
dst_rst_ni	input	logic

Table 984. pulp-platform_axi/src/axi_cdc_dst.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]

API — axi_cdc_src.sv

Table 985. pulp-platform_axi/src/axi_cdc_src.sv — module / interface / package (3개)

이름	종류
axi_cdc_src	module
axi_cdc_src_intf	module
axi_lite_cdc_src_intf	module

Table 986. pulp-platform_axi/src/axi_cdc_src.sv — parameter (19개)

파라미터	형/폭	기본값
LogDepth	int unsigned	1
SyncStages	int unsigned	2
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
AXI_ID_WIDTH	int unsigned	0

파라미터	형/폭	기본값
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1
SYNC_STAGES	int unsigned	2

Table 987. `pulp-platform_axi/src/axi_cdc_src.sv` — 포트 (23개)

포트	방향	형/폭
<code>src_clk_i</code>	input	logic
<code>src_rst_ni</code>	input	logic
<code>src_req_i</code>	input	<code>axi_req_t</code>
<code>src_resp_o</code>	output	<code>axi_resp_t</code>
<code>async_data_master_aw_data_o</code>	output	<code>aw_chan_t [2**LogDepth-1:0]</code>
<code>async_data_master_aw_wptr_o</code>	output	logic [LogDepth:0]
<code>async_data_master_aw_rptr_i</code>	input	logic [LogDepth:0]
<code>async_data_master_w_data_o</code>	output	<code>w_chan_t [2**LogDepth-1:0]</code>
<code>async_data_master_w_wptr_o</code>	output	logic [LogDepth:0]
<code>async_data_master_w_rptr_i</code>	input	logic [LogDepth:0]
<code>async_data_master_b_data_i</code>	input	<code>b_chan_t [2**LogDepth-1:0]</code>
<code>async_data_master_b_wptr_i</code>	input	logic [LogDepth:0]
<code>async_data_master_b_rptr_o</code>	output	logic [LogDepth:0]
<code>async_data_master_ar_data_o</code>	output	<code>ar_chan_t [2**LogDepth-1:0]</code>
<code>async_data_master_ar_wptr_o</code>	output	logic [LogDepth:0]
<code>async_data_master_ar_rptr_i</code>	input	logic [LogDepth:0]
<code>async_data_master_r_data_i</code>	input	<code>r_chan_t [2**LogDepth-1:0]</code>
<code>async_data_master_r_wptr_i</code>	input	logic [LogDepth:0]
<code>async_data_master_r_rptr_o</code>	output	logic [LogDepth:0]
<code>src_clk_i</code>	input	logic
<code>src_rst_ni</code>	input	logic
<code>src_clk_i</code>	input	logic
<code>src_rst_ni</code>	input	logic

Table 988. pulp-platform_axi/src/axi_cdc_src.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]

API — axi_chan_compare.sv**Table 989. pulp-platform_axi/src/axi_chan_compare.sv — module / interface / package (1개)**

이름	종류
axi_chan_compare	module

Table 990. pulp-platform_axi/src/axi_chan_compare.sv — parameter (10개)

파라미터	형/폭	기본값
IgnoreId	bit	1'b0
AllowReordering	bit	1'b0
IdWidth	int unsigned	1
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
req_t		logic
resp_t		logic

Table 991. pulp-platform_axi/src/axi_chan_compare.sv — localparam (1개)

상수	형/폭	값
NumIds		(AllowReordering)? 2**IdWidth : 1

Table 992. pulp-platform_axi/src/axi_chan_compare.sv — 포트 (16개)

포트	방향	형/폭
clk_a_i	input	logic
clk_b_i	input	logic
axi_a_req	input	req_t

포트	방향	형/폭
axi_a_res	input	resp_t
axi_b_req	input	req_t
axi_b_res	input	resp_t
aw_expected	input	aw_chan_t
aw_received	input	aw_chan_t
ar_expected	input	ar_chan_t
ar_received	input	ar_chan_t
w_expected	input	w_chan_t
w_received	input	w_chan_t
b_expected	input	b_chan_t
b_received	input	b_chan_t
r_expected	input	r_chan_t
r_received	input	r_chan_t

API — axi_cut.sv

Table 993. pulp-platform_axi/src/axi_cut.sv — module / interface / package (3개)

이름	종류
axi_cut	module
axi_cut_intf	module
axi_lite_cut_intf	module

Table 994. pulp-platform_axi/src/axi_cut.sv — parameter (31개)

파라미터	형/폭	기본값
Bypass	bit	1'b0
BypassAw	bit	Bypass
BypassW	bit	Bypass
BypassB	bit	Bypass
BypassAr	bit	Bypass
BypassR	bit	Bypass
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic

파라미터	형/폭	기본값
BYPASS	bit	1'b0
BYPASS_AW	bit	BYPASS
BYPASS_W	bit	BYPASS
BYPASS_B	bit	BYPASS
BYPASS_AR	bit	BYPASS
BYPASS_R	bit	BYPASS
ADDR_WIDTH	int unsigned	0
DATA_WIDTH	int unsigned	0
ID_WIDTH	int unsigned	0
USER_WIDTH	int unsigned	0
BYPASS	bit	1'b0
BYPASS_AW	bit	BYPASS
BYPASS_W	bit	BYPASS
BYPASS_B	bit	BYPASS
BYPASS_AR	bit	BYPASS
BYPASS_R	bit	BYPASS
ADDR_WIDTH	int unsigned	0
DATA_WIDTH	int unsigned	0

Table 995. pulp-platform_axi/src/axi_cut.sv — 포트 (10개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic
clk_i	input	logic
rst_ni	input	logic

Table 996. pulp-platform_axi/src/axi_cut.sv — typedef / using (8개)

새 이름	원래 형
id_t	logic [ID_WIDTH-1:0]
addr_t	logic [ADDR_WIDTH-1:0]

새 이름	원래 형
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [DATA_WIDTH/8-1:0]
user_t	logic [USER_WIDTH-1:0]
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [DATA_WIDTH/8-1:0]

API — axi_delayer.sv

Table 997. pulp-platform_axi/src/axi_delayer.sv — module / interface / package (2개)

이름	종류
axi_delayer	module
axi_delayer_intf	module

Table 998. pulp-platform_axi/src/axi_delayer.sv — parameter (19개)

파라미터	형/폭	기본값
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
StallRandomInput	bit	0
StallRandomOutput	bit	0
FixedDelayInput	int unsigned	1
FixedDelayOutput	int unsigned	1
AXI_ID_WIDTH	int unsigned	0
AXI_ADDR_WIDTH	int unsigned	0
AXI_DATA_WIDTH	int unsigned	0
AXI_USER_WIDTH	int unsigned	0
STALL_RANDOM_INPUT	bit	0
STALL_RANDOM_OUTPUT	bit	0
FIXED_DELAY_INPUT	int unsigned	1
FIXED_DELAY_OUTPUT	int unsigned	1

Table 999. pulp-platform_axi/src/axi_delayer.sv — 포트 (8개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_resp_o	output	axi_resp_t
mst_req_o	output	axi_req_t
mst_resp_i	input	axi_resp_t
clk_i	input	logic
rst_ni	input	logic

Table 1000. pulp-platform_axi/src/axi_delayer.sv — typedef / using (5개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]

API — axi_demux.sv

Table 1001. pulp-platform_axi/src/axi_demux.sv — module / interface / package (2개)

이름	종류
axi_demux	module
axi_demux_intf	module

Table 1002. pulp-platform_axi/src/axi_demux.sv — parameter (36개)

파라미터	형/폭	기본값
AxiIdWidth	int unsigned	32'd0
AtopSupport	bit	1'b1
aw_chan_t		logic
w_chan_t		logic
b_chan_t		logic
ar_chan_t		logic
r_chan_t		logic
axi_req_t		logic
axi_resp_t		logic
NoMstPorts	int unsigned	32'd0
MaxTrans	int unsigned	32'd8

파라미터	형/폭	기본값
AxiLookBits	int unsigned	32'd3
UniqueIds	bit	1'b0
SpillAw	bit	1'b1
SpillW	bit	1'b0
SpillB	bit	1'b0
SpillAr	bit	1'b1
SpillR	bit	1'b0
SelectWidth	int unsigned	(NoMstPorts > 32'd1) ? \$clog2(NoMstPorts) : 32'd1
select_t		logic [SelectWidth-1:0]
AXI_ID_WIDTH	int unsigned	32'd0
ATOP_SUPPORT	bit	1'b1
AXI_ADDR_WIDTH	int unsigned	32'd0
AXI_DATA_WIDTH	int unsigned	32'd0
AXI_USER_WIDTH	int unsigned	32'd0
NO_MST_PORTS	int unsigned	32'd3
MAX_TRANS	int unsigned	32'd8
AXI_LOOK_BITS	int unsigned	32'd3
UNIQUE_IDS	bit	1'b0
SPIII_AW	bit	1'b1
SPIII_W	bit	1'b0
SPIII_B	bit	1'b0
SPIII_AR	bit	1'b1
SPIII_R	bit	1'b0
SELECT_WIDTH	int unsigned	(NO_MST_PORTS > 32'd1) ? \$clog2(NO_MST_PORTS) : 32'd1
select_t		logic [SELECT_WIDTH-1:0] // MST port select type

Table 1003. pulp-platform_axi/src/axi_demux.sv — 포트 (12개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
slv_req_i	input	axi_req_t
slv_aw_select_i	input	select_t
slv_ar_select_i	input	select_t
slv_resp_o	output	axi_resp_t
mst_reqs_o	output	axi_req_t [NoMstPorts-1:0]

포트	방향	형/폭
mst_resps_i	input	axi_resp_t [NoMstPorts-1:0]
clk_i	input	logic
rst_ni	input	logic
slv_aw_select_i	input	select_t
slv_ar_select_i	input	select_t

Table 1004. pulp-platform_axi/src/axi_demux.sv — typedef / using (5개)

새 이름	원래 형
id_t	logic [AXI_ID_WIDTH-1:0]
addr_t	logic [AXI_ADDR_WIDTH-1:0]
data_t	logic [AXI_DATA_WIDTH-1:0]
strb_t	logic [AXI_DATA_WIDTH/8-1:0]
user_t	logic [AXI_USER_WIDTH-1:0]

Table 1005. pulp-platform_axi/src/axi_demux.sv — #define / `define 매크로 (1개)

매크로	인자	정의
TARGET_VSIM		`endif

API — axi_demux_id_counters.sv

Table 1006. pulp-platform_axi/src/axi_demux_id_counters.sv — module / interface / package (1개)

이름	종류
axi_demux_id_counters	module

Table 1007. pulp-platform_axi/src/axi_demux_id_counters.sv — parameter (4개)

파라미터	형/폭	기본값
AxiIdBits	int unsigned	2
CounterWidth	int unsigned	4
MaxTrans	int unsigned	2**CounterWidth - 1
mst_port_select_t		logic

Table 1008. pulp-platform_axi/src/axi_demux_id_counters.sv — localparam (1개)

상수	형/폭	값
NoCounters	int unsigned	2**AxiIdBits

Table 1009. pulp-platform_axi/src/axi_demux_id_counters.sv — 포트 (14개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic

포트	방향	형/폭
lookup_axi_id_i	input	logic [AxildBits-1:0]
lookup_mst_select_o	output	mst_port_select_t
lookup_mst_select_occupied_o	output	logic
full_o	output	logic
push_axi_id_i	input	logic [AxildBits-1:0]
push_mst_select_i	input	mst_port_select_t
push_i	input	logic
inject_axi_id_i	input	logic [AxildBits-1:0]
inject_i	input	logic
pop_axi_id_i	input	logic [AxildBits-1:0]
pop_i	input	logic
any_outstanding_trx_o	output	logic

Table 1010. `pulp-platform_axi/src/axi_demux_id_counters.sv` — `typedef / using` (17||)

새 이름	원래 형
cnt_t	logic [CounterWidth-1:0]

1. `pulp-platform_axi/src/axi_atop_filter.sv` (447 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the license.
10 //
11 // Authors:
12 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14
15 /// Filter atomic operations (ATOPs) in a protocol-compliant
  manner.
16 ///
17 /// This module filters atomic operations (ATOPs), i.e., write
  transactions that have a non-zero
18 /// `aw_atop` value, from its `slv` to its `mst` port. This
  module guarantees that:
19 ///
20 /// 1) `aw_atop` is always zero on the `mst` port;
21 ///
22 /// 2) write transactions with non-zero `aw_atop` on the `slv`
  port are handled in conformance with
23 /// the AXI standard by replying to such write transactions
  with the proper B and R responses.
24 /// The response code on atomic operations that reach this
  module is always SLVERR
25 /// (implementation-specific, not defined in the AXI stand-
  ard).
26 ///
27 /// ## Intended usage
28 /// This module is intended to be placed between masters that
  may issue ATOPs and slaves that do not
29 /// support ATOPs. That way, this module ensures that the AXI
  protocol remains in a defined state on

```

```

30 /// systems with mixed ATOP capabilities.
31 ///
32 /// ## Specification reminder
33 /// The AXI standard specifies that there may be no ordering
  requirements between different atomic
34 /// bursts (i.e., a burst started by an AW with ATOP other
  than 0) and none between atomic bursts
35 /// and non-atomic bursts [E2.1.4]. That is, **an atomic burst
  may never have the same ID as any
36 /// other write or read burst that is in-flight at the same
  time**.
37 module axi_atop_filter #(
38     /// AXI ID width
39     parameter int unsigned AxidWidth = 0,
40     /// Maximum number of in-flight AXI write transactions
41     parameter int unsigned AxiMaxWriteTxns = 0,
42     /// AXI request type
43     parameter type axi_req_t = logic,
44     /// AXI response type
45     parameter type axi_resp_t = logic
46 ) (
47     /// Rising-edge clock of both ports
48     input logic clk_i,
49     /// Asynchronous reset, active low
50     input logic rst_ni,
51     /// Slave port request
52     input axi_req_t slv_req_i,
53     /// Slave port response
54     output axi_resp_t slv_resp_o,
55     /// Master port request
56     output axi_req_t mst_req_o,
57     /// Master port response
58     input axi_resp_t mst_resp_i
59 );
60
61 // Minimum counter width is 2 to detect underflows.
62 localparam int unsigned COUNTER_WIDTH = (AxiMaxWriteTxns ==
1) ? 2 : $clog2(AxiMaxWriteTxns+1);
63 typedef struct packed {
64     logic underflow;
65     logic [COUNTER_WIDTH-1:0] cnt;
66 } cnt_t;
67 cnt_t w_cnt_d, w_cnt_q;
68
69 typedef enum logic [2:0] {
70     W_RESET, W_FEEDTHROUGH, BLOCK_AW, ABSORB_W, HOLD_B, IN-
  JECT_B, WAIT_R
71 } w_state_e;
72 w_state_e w_state_d, w_state_q;
73
74 typedef enum logic [1:0] { R_RESET, R_FEEDTHROUGH, INJECT_R,

```

```

R_HOLD } r_state_e;
75  r_state_e  r_state_d, r_state_q;
76
77  typedef logic [AxiIdWidth-1:0] id_t;
78  id_t id_d, id_q;
79
80  typedef logic [7:0] len_t;
81  len_t r_beats_d, r_beats_q;
82
83  typedef struct packed {
84    len_t len;
85  } r_resp_cmd_t;
86  r_resp_cmd_t r_resp_cmd_push, r_resp_cmd_pop;
87
88  logic aw_without_complete_w_downstream,
89        complete_w_without_aw_downstream,
90        r_resp_cmd_push_valid, r_resp_cmd_push_ready,
91        r_resp_cmd_pop_valid, r_resp_cmd_pop_ready;
92
93  // An AW without a complete W burst is in-flight downstream
if the W counter is > 0 and not
94  // underflowed.

95  assign aw_without_complete_w_downstream = !w_cnt_q.underflow &&
(w_cnt_q.cnt > 0);
96  // A complete W burst without AW is in-flight downstream if
the W counter is -1.
97  assign complete_w_without_aw_downstream = w_cnt_q.underflow
&& (w_cnt_q.cnt);
98
99  // Manage AW, W, and B channels.
100 always_comb begin
101  // Defaults:
102  // Disable AW and W handshakes.
103  mst_req_o.aw_valid = 1'b0;
104  slv_resp_o.aw_ready = 1'b0;
105  mst_req_o.w_valid = 1'b0;
106  slv_resp_o.w_ready = 1'b0;
107  // Feed write responses through.
108  mst_req_o.b_ready = slv_req_i.b_ready;
109  slv_resp_o.b_valid = mst_resp_i.b_valid;
110  slv_resp_o.b = mst_resp_i.b;
111  // Keep ID stored for B and R response.
112  id_d = id_q;
113  // Do not push R response commands.
114  r_resp_cmd_push_valid = 1'b0;
115  // Keep the current state.
116  w_state_d = w_state_q;
117
118  unique case (w_state_q)
119    W_RESET: w_state_d = W_FEEDTHROUGH;
120
121    W_FEEDTHROUGH: begin
122      // Feed AW channel through if the maximum number of
outstanding bursts is not reached.

123      if (complete_w_without_aw_downstream || (w_cnt_q.cnt <
AxiMaxWriteTxns)) begin
124        mst_req_o.aw_valid = slv_req_i.aw_valid;
125        slv_resp_o.aw_ready = mst_resp_i.aw_ready;
126      end
127      // Feed W channel through if ..
128      if (aw_without_complete_w_downstream // .. downstream
is missing W bursts ..
129          // .. or a new non-ATOP AW is being applied and
there is not already a complete W burst
130          // downstream (to prevent underflows of w_cnt).
131          || ((slv_req_i.aw_valid && slv_req_i.aw.atop[5:4]
== axi_pkg::ATOP_NONE)
&& !complete_w_without_aw_downstream)
132          ) begin
133        mst_req_o.w_valid = slv_req_i.w_valid;
134        slv_resp_o.w_ready = mst_resp_i.w_ready;
135      end
136      // Filter out AWs that are atomic operations.
137      if (slv_req_i.aw_valid && slv_req_i.aw.atop[5:4] !=
axi_pkg::ATOP_NONE) begin
138
139        mst_req_o.aw_valid = 1'b0; // Do not let AW pass to
master port.
140        slv_resp_o.aw_ready = 1'b1; // Absorb AW on slave
port.
141        id_d = slv_req_i.aw.id; // Store ID for B response.
142        // Some atomic operations require a response on the
R channel.
143        if (slv_req_i.aw.atop[axi_pkg::ATOP_R_RESP]) begin
144          // Push R response command. We do not have to
wait for the ready of the register
145          // because we know it is ready: we are its only
master and will wait for the register to
146          // be emptied before going back to the `W_FEED-
THROUGH` state.
147          r_resp_cmd_push_valid = 1'b1;
148        end
149        // If downstream is missing W beats, block the AW
channel and let the W bursts complete.
150        if (aw_without_complete_w_downstream) begin
151          w_state_d = BLOCK_AW;
152          // If downstream is not missing W beats, absorb the
W beats for this atomic AW.
153          end else begin

```

```

154        mst_req_o.w_valid = 1'b0; // Do not let W beats
pass to master port.
155        slv_resp_o.w_ready = 1'b1; // Absorb W beats on
slave port.
156        if (slv_req_i.w_valid && slv_req_i.w.last) begin
157          // If the W beat is valid and the last, proceed
by injecting the B response.
158          // However, if there is a non-handshaked B on
our response port, we must let that
159          // complete first.
160          if (slv_resp_o.b_valid && !slv_req_i.b_ready)
begin
161            w_state_d = HOLD_B;
162          end else begin
163            w_state_d = INJECT_B;
164          end
165        end else begin
166          // Otherwise continue with absorbing W beats.
167          w_state_d = ABSORB_W;
168        end
169      end
170    end
171  end
172
173  BLOCK_AW: begin
174    // Feed W channel through to let outstanding bursts
complete.
175    if (aw_without_complete_w_downstream) begin
176      mst_req_o.w_valid = slv_req_i.w_valid;
177      slv_resp_o.w_ready = mst_resp_i.w_ready;
178    end else begin
179      // If there are no more outstanding W bursts, start
absorbing the next W burst.
180      slv_resp_o.w_ready = 1'b1;
181      if (slv_req_i.w_valid && slv_req_i.w.last) begin
182        // If the W beat is valid and the last, proceed by
injecting the B response.
183        if (slv_resp_o.b_valid && !slv_req_i.b_ready) be-
gin
184          w_state_d = HOLD_B;
185        end else begin
186          w_state_d = INJECT_B;
187        end
188      end else begin
189        // Otherwise continue with absorbing W beats.
190        w_state_d = ABSORB_W;
191      end
192    end
193  end
194
195  ABSORB_W: begin
196    // Absorb all W beats of the current burst.
197    slv_resp_o.w_ready = 1'b1;
198    if (slv_req_i.w_valid && slv_req_i.w.last) begin
199      if (slv_resp_o.b_valid && !slv_req_i.b_ready) begin
200        w_state_d = HOLD_B;
201      end else begin
202        w_state_d = INJECT_B;
203      end
204    end
205  end
206
207  HOLD_B: begin
208    // Proceed with injection of B response upon hand-
shake.
209    if (slv_resp_o.b_valid && slv_req_i.b_ready) begin
210      w_state_d = INJECT_B;
211    end
212  end
213
214  INJECT_B: begin
215    // Pause forwarding of B response.
216    mst_req_o.b_ready = 1'b0;
217
218    // Inject error response instead. Since the B channel
has an ID and the atomic burst we are
219    // replying to is guaranteed to be the only burst with
this ID in flight, we do not have to
220    // observe any ordering and can immediately inject on
the B channel.
221    slv_resp_o.b = '0;
222    slv_resp_o.b.id = id_q;
223    slv_resp_o.b.resp = axi_pkg::RESP_SLVERR;
224    slv_resp_o.b_valid = 1'b1;
225    if (slv_req_i.b_ready) begin
226      // If not all beats of the R response have been in-
jected, wait for them. Otherwise, return
227      // to `W_FEEDTHROUGH`.
228      if (r_resp_cmd_pop_valid && !r_resp_cmd_pop_ready)
begin
229        w_state_d = WAIT_R;
230      end else begin
231        w_state_d = W_FEEDTHROUGH;
232      end
233    end
234
235    WAIT_R: begin
236      // Wait with returning to `W_FEEDTHROUGH` until all

```

```

beats of the R response have been
237 // injected.
238 if (!r_resp_cmd_pop_valid) begin
239     w_state_d = W_FEEDTHROUGH;
240 end
241 end
242
243 default: w_state_d = W_RESET;
244 endcase
245 end
246 // Connect signals on AW and W channel that are not managed
by the control FSM from slave port to
247 // master port.
248 // Feed-through of the AW and W vectors, make sure that
downstream aw.atop is always zero
249 always_comb begin
250     // overwrite the atop signal
251     mst_req_o.aw = slv_req_i.aw;
252     mst_req_o.aw.atop = '0;
253 end
254 assign mst_req_o.w = slv_req_i.w;
255
256 // Manage R channel.
257 always_comb begin
258     // Defaults:
259     // Feed read responses through.
260     slv_resp_o.r = mst_resp_i.r;
261     slv_resp_o.r_valid = mst_resp_i.r_valid;
262     mst_req_o.r_ready = slv_req_i.r_ready;
263     // Do not pop R response command.
264     r_resp_cmd_pop_ready = 1'b0;
265     // Keep the current value of the beats counter.
266     r_beats_d = r_beats_q;
267     // Keep the current state.
268     r_state_d = r_state_q;
269
270     unique case (r_state_q)
271         R_RESET: r_state_d = R_FEEDTHROUGH;
272
273         R_FEEDTHROUGH: begin
274             if (mst_resp_i.r_valid && !slv_req_i.r_ready) begin
275                 r_state_d = R_HOLD;
276             end else if (r_resp_cmd_pop_valid) begin
277                 // Upon a command to inject an R response, immedi-
ately proceed with doing so because there
278                 // are no ordering requirements with other bursts
that may be ongoing on the R channel at
279                 // this moment.
280                 r_beats_d = r_resp_cmd_pop.len;
281                 r_state_d = INJECT_R;
282             end
283         end
284
285         INJECT_R: begin
286             mst_req_o.r_ready = 1'b0;
287             slv_resp_o.r = '0;
288             slv_resp_o.r.id = id_q;
289             slv_resp_o.r.resp = axi_pkg::RESP_SLVERR;
290             slv_resp_o.r.last = (r_beats_q == '0);
291             slv_resp_o.r_valid = 1'b1;
292             if (slv_req_i.r_ready) begin
293                 if (slv_resp_o.r.last) begin
294                     r_resp_cmd_pop_ready = 1'b1;
295                     r_state_d = R_FEEDTHROUGH;
296                 end else begin
297                     r_beats_d -= 1;
298                 end
299             end
300         end
301
302         R_HOLD: begin
303             if (mst_resp_i.r_valid && slv_req_i.r_ready) begin
304                 r_state_d = R_FEEDTHROUGH;
305             end
306         end
307
308         default: r_state_d = R_RESET;
309     endcase
310 end
311 // Feed all signals on AR through.
312 assign mst_req_o.ar = slv_req_i.ar;
313 assign mst_req_o.ar_valid = slv_req_i.ar_valid;
314 assign slv_resp_o.ar_ready = mst_resp_i.ar_ready;
315
316 // Keep track of outstanding downstream write bursts and re-
sponses.
317 always_comb begin
318     w_cnt_d = w_cnt_q;
319     if (mst_req_o.aw_valid && mst_resp_i.aw_ready) begin
320         w_cnt_d.cnt += 1;
321     end
322     if (mst_req_o.w_valid && mst_resp_i.w_ready &&
mst_req_o.w.last) begin
323         w_cnt_d.cnt -= 1;
324     end
325     if (w_cnt_q.underflow && (w_cnt_d.cnt == '0)) begin
326         w_cnt_d.underflow = 1'b0;
327     end else if (w_cnt_q.cnt == '0 && (w_cnt_d.cnt)) begin
328         w_cnt_d.underflow = 1'b1;
329     end
330 end

```

```

331
332 always_ff @(posedge clk_i, negedge rst_ni) begin
333     if (!rst_ni) begin
334         id_q <= '0;
335         r_beats_q <= '0;
336         r_state_q <= R_RESET;
337         w_cnt_q <= '{default: '0};
338         w_state_q <= W_RESET;
339     end else begin
340         id_q <= id_d;
341         r_beats_q <= r_beats_d;
342         r_state_q <= r_state_d;
343         w_cnt_q <= w_cnt_d;
344         w_state_q <= w_state_d;
345     end
346 end
347
348 cc_stream_register #(
349     .data_t (r_resp_cmd_t)
350 ) r_resp_cmd (
351     .clk_i (clk_i),
352     .rst_ni (rst_ni),
353     .clr_i (1'b0),
354     .valid_i (r_resp_cmd_push_valid),
355     .ready_o (r_resp_cmd_push_ready),
356     .data_i (r_resp_cmd_push),
357     .valid_o (r_resp_cmd_pop_valid),
358     .ready_i (r_resp_cmd_pop_ready),
359     .data_o (r_resp_cmd_pop)
360 );
361 assign r_resp_cmd_push.len = slv_req_i.aw.len;
362
363 // pragma translate_off
364 `ifndef VERILATOR
365     initial begin: p_assertions
366         assert (AxiIdWidth >= 1) else $fatal(1, "AXI ID width must
be at least 1!");
367         assert (AxiMaxWriteTxns >= 1)
368         else $fatal(1, "Maximum number of outstanding write
transactions must be at least 1!");
369     end
370 `endif
371 // pragma translate_on
372 endmodule
373
374 `include "axi/assign.svh"
375 `include "axi/typedef.svh"
376
377 /// Interface variant of ['axi_atop_filter'](mod-
ule.axi_atop_filter).
378 module axi_atop_filter_intf #(
379     /// AXI ID width
380     parameter int unsigned AXI_ID_WIDTH = 0,
381     /// AXI address width
382     parameter int unsigned AXI_ADDR_WIDTH = 0,
383     /// AXI data width
384     parameter int unsigned AXI_DATA_WIDTH = 0,
385     /// AXI user signal width
386     parameter int unsigned AXI_USER_WIDTH = 0,
387     /// Maximum number of in-flight AXI write transactions
388     parameter int unsigned AXI_MAX_WRITE_TXNS = 0
389 ) (
390     /// Rising-edge clock of both ports
391     input logic clk_i,
392     /// Asynchronous reset, active low
393     input logic rst_ni,
394     /// Slave interface port
395     AXI_BUS.Slave slv,
396     /// Master interface port
397     AXI_BUS.Master mst
398 );
399
400 typedef logic [AXI_ID_WIDTH-1:0] id_t;
401 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
402 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
403 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
404 typedef logic [AXI_USER_WIDTH-1:0] user_t;
405
406 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
407 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
408 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
409 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
410 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
411 `AXI_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
412 `AXI_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
413
414 axi_req_t slv_req, mst_req;
415 axi_resp_t slv_resp, mst_resp;
416
417 `AXI_ASSIGN_TO_REQ(slv_req, slv)
418 `AXI_ASSIGN_FROM_RESP(slv, slv_resp)
419
420 `AXI_ASSIGN_FROM_REQ(mst, mst_req)
421 `AXI_ASSIGN_TO_RESP(mst_resp, mst)
422
423 axi_atop_filter #(
424     .AxiIdWidth (AXI_ID_WIDTH),
425     // Maximum number of AXI write bursts outstanding at the
same time

```

```

426     .AxiMaxWriteTxns ( AXI_MAX_WRITE_TXNS ),
427     // AXI request & response type
428     .axi_req_t      ( axi_req_t      ),
429     .axi_resp_t     ( axi_resp_t     )
430 ) i_axi_atop_filter (
431     .clk_i,
432     .rst_ni,
433     .slv_req_i ( slv_req ),
434     .slv_resp_o ( slv_resp ),
435     .mst_req_o ( mst_req ),
436     .mst_resp_i ( mst_resp )
437 );
438 // pragma translate_off
439 `ifndef VERILATOR
440     initial begin: p_assertions
441         assert (AXI_ADDR_WIDTH >= 1) else $fatal(1, "AXI ADDR
width must be at least 1!");
442         assert (AXI_DATA_WIDTH >= 1) else $fatal(1, "AXI DATA
width must be at least 1!");
443         assert (AXI_USER_WIDTH >= 1) else $fatal(1, "AXI USER
width must be at least 1!");
444     end
445 `endif
446 // pragma translate_on
447 endmodule

```

2. pulp-platform_axi/src/axi_burst_splitter.sv (97 lines)

```

1 // Copyright (c) 2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Authors:
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 `include "axi/typedef.svh"
17
18 /// Split AXI4 bursts into single-beat transactions.
19 ///
20 /// ## Limitations
21 ///
22 /// - This module does not support wrapping
23 /// - This module does not support atomic operations (ATOPs)
24 /// - This module does not support atomic operations (ATOPs)
and responds to ATOPs with a slave
25 /// error. Place an ['axi_atop_filter'](mod-
ule.axi_atop_filter) before this module if upstream
26 /// modules can generate ATOPs.
27 module axi_burst_splitter #(
28     // Maximum number of AXI read bursts outstanding at the same
time
29     parameter int unsigned MaxReadTxns = 32'd0,
30     // Maximum number of AXI write bursts outstanding at the
same time
31     parameter int unsigned MaxWriteTxns = 32'd0,
32     // Internal ID queue can work in two bandwidth modes: see
id_queue.sv for details
33     parameter bit FullBW = 0,
34     // AXI Bus Types
35     parameter int unsigned AddrWidth = 32'd0,
36     parameter int unsigned DataWidth = 32'd0,
37     parameter int unsigned IdWidth = 32'd0,
38     // Number of least-significant ID bits used by the internal
demux to track in-flight
39     // transactions (0 < AxiLookBits <= IdWidth), see doc/
axi_demux.md for the trade-off.
40     parameter int unsigned AxiLookBits = IdWidth,
41     parameter int unsigned UserWidth = 32'd0,
42     parameter type axi_req_t = logic,
43     parameter type axi_resp_t = logic
44 ) (
45     input logic clk_i,
46     input logic rst_ni,
47
48     // Input / Slave Port
49     input axi_req_t slv_req_i,
50     output axi_resp_t slv_resp_o,
51
52     // Output / Master Port
53     output axi_req_t mst_req_o,
54     input axi_resp_t mst_resp_i

```

```

55 );
56
57 typedef logic [AddrWidth-1:0] addr_t;
58 typedef logic [DataWidth-1:0] data_t;
59 typedef logic [DataWidth/8-1:0] strb_t;
60 typedef logic [IdWidth-1:0] id_t;
61 typedef logic [UserWidth-1:0] user_t;
62
63 `AXI_TYPERDEF_AXI_CHAN_T(axi_aw_chan_t, addr_t, id_t, user_t)
64 `AXI_TYPERDEF_W_CHAN_T(axi_w_chan_t, data_t, strb_t, user_t)
65 `AXI_TYPERDEF_B_CHAN_T(axi_b_chan_t, id_t, user_t)
66 `AXI_TYPERDEF_AR_CHAN_T(axi_ar_chan_t, addr_t, id_t, user_t)
67 `AXI_TYPERDEF_R_CHAN_T(axi_r_chan_t, data_t, id_t, user_t)
68
69 axi_burst_splitter_gran #(
70     .MaxReadTxns ( MaxReadTxns ),
71     .MaxWriteTxns ( MaxWriteTxns ),
72     .CutPath ( 1'b0 ),
73     .DisableChecks ( 1'b0 ),
74     .FullBW ( FullBW ),
75     .AddrWidth ( AddrWidth ),
76     .DataWidth ( DataWidth ),
77     .IdWidth ( IdWidth ),
78     .AxiLookBits ( AxiLookBits ),
79     .UserWidth ( UserWidth ),
80     .axi_req_t ( axi_req_t ),
81     .axi_resp_t ( axi_resp_t ),
82     .axi_aw_chan_t ( axi_aw_chan_t ),
83     .axi_w_chan_t ( axi_w_chan_t ),
84     .axi_b_chan_t ( axi_b_chan_t ),
85     .axi_ar_chan_t ( axi_ar_chan_t ),
86     .axi_r_chan_t ( axi_r_chan_t )
87 ) i_axi_burst_splitter_gran (
88     .clk_i,
89     .rst_ni,
90     .len_limit_i ( 8'h00 ),
91     .slv_req_i,
92     .slv_resp_o,
93     .mst_req_o,
94     .mst_resp_i
95 );
96
97 endmodule

```

3. pulp-platform_axi/src/axi_burst_splitter_gran.sv (751 lines)

```

1 // Copyright 2023 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
details.
3 // SPDX-License-Identifier: SHL-0.51
4 //
5 // Authors:
6 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
7 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
8 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
9
10 `include "axi/typedef.svh"
11 `include "common_cells/registers.svh"
12
13 /// Split AXI4 bursts into single-beat transactions.
14 ///
15 /// ## Limitations
16 ///
17 /// - This module does not support wrapping
18 /// - This module does not support atomic operations (ATOPs)
19 /// - This module does not support atomic operations (ATOPs)
and responds to ATOPs with a slave
20 /// error. Place an ['axi_atop_filter'](mod-
ule.axi_atop_filter) before this module if upstream
21 /// modules can generate ATOPs.
22 module axi_burst_splitter_gran #(
23     // Maximum number of AXI read bursts outstanding at the
same time
24     parameter int unsigned MaxReadTxns = 32'd0,
25     // Maximum number of AXI write bursts outstanding at the
same time
26     parameter int unsigned MaxWriteTxns = 32'd0,
27     // Internal ID queue can work in two bandwidth modes: see
id_queue.sv for details
28     parameter bit FullBW = 1'b0,
29     // Cut paths through the IP
30     parameter bit CutPath = 1'b0,
31     // Disable checks, handle unsupported transfers as bypass
instead of reporting an error
32     parameter bit DisableChecks = 1'b0,
33     // AXI Bus Types
34     parameter int unsigned AddrWidth = 32'd0,
35     parameter int unsigned DataWidth = 32'd0,
36     parameter int unsigned IdWidth = 32'd0,
37
38     // Number of least-significant ID bits used by the internal
demux to track in-flight
39     // transactions (0 < AxiLookBits <= IdWidth), see doc/
axi_demux.md for the trade-off.
40     parameter int unsigned AxiLookBits = IdWidth,
41     parameter type axi_req_t = logic,
42     parameter type axi_resp_t = logic,
43     parameter type axi_aw_chan_t = logic,

```

```

44 parameter type      axi_w_chan_t = logic,
45 parameter type      axi_b_chan_t = logic,
46 parameter type      axi_ar_chan_t = logic,
47 parameter type      axi_r_chan_t = logic
48 ) (
49   input logic clk_i,
50   input logic rst_ni,
51
52   // length
53   input axi_pkg::len_t len_limit_i,
54
55   // Input / Slave Port
56   input axi_req_t slv_req_i,
57   output axi_resp_t slv_resp_o,
58
59   // Output / Master Port
60   output axi_req_t mst_req_o,
61   input axi_resp_t mst_resp_i
62 );
63
64 // Demultiplex between supported and unsupported transactions.
65 axi_req_t slv_req, act_req, unsupported_req;
66 axi_resp_t slv_resp, act_resp, unsupported_resp;
67
68 axi_multicut #(
69   .NoCuts ( CutPath ),
70   .aw_chan_t ( axi_aw_chan_t ),
71   .w_chan_t ( axi_w_chan_t ),
72   .b_chan_t ( axi_b_chan_t ),
73   .ar_chan_t ( axi_ar_chan_t ),
74   .r_chan_t ( axi_r_chan_t ),
75   .axi_req_t ( axi_req_t ),
76   .axi_resp_t ( axi_resp_t )
77 ) i_axi_multicut (
78   .clk_i,
79   .rst_ni,
80   .slv_req_i,
81   .slv_resp_o,
82   .mst_req_o ( slv_req ),
83   .mst_resp_i ( slv_resp )
84 );
85
86 logic sel_aw_unsupported, sel_ar_unsupported;
87 localparam int unsigned MaxTxns = (MaxReadTxns > MaxWriteTxns) ? MaxReadTxns : MaxWriteTxns;
88 axi_demux_simple #(
89   .AxiIdWidth ( Idwidth ),
90   .axi_req_t ( axi_req_t ),
91   .axi_resp_t ( axi_resp_t ),
92   .NoMstPorts ( 2 ),
93   .MaxTrans ( MaxTxns ),
94   .AxiLookBits ( AxiLookBits )
95 ) i_demux_supported_vs_unsupported (
96   .clk_i,
97   .rst_ni,
98   .slv_req_i ( slv_req ),
99   .slv_aw_select_i ( sel_aw_unsupported ),
100  .slv_ar_select_i ( sel_ar_unsupported ),
101  .slv_resp_o ( slv_resp ),
102  .mst_reqs_o ( {unsupported_req, act_req} ),
103  .mst_resps_i ( {unsupported_resp, act_resp} )
104 );
105
106 // Define supported transactions.
107 function bit txn_supported(axi_pkg::atop_t atop,
108 axi_pkg::burst_t burst, axi_pkg::cache_t cache,
109 axi_pkg::len_t len, axi_pkg::len_t len_limit);
110
111 // if the splitter does not touch the transaction: allow
112 if (len <= len_limit) begin
113   return 1'b1;
114
115 //
116 end else begin
117
118 // Wrapping bursts are currently not supported to be
119 split
120   if (burst == axi_pkg::BURST_WRAP) return 1'b0;
121
122 // ATOP bursts are not supported.
123   if (atop != '0 & len > 0) return 1'b0;
124
125 // The AXI Spec (A3.4.1) only allows splitting non-modifiable transactions ..
126   if (!axi_pkg::modifiable(cache)) begin
127     // .. if they are INCR bursts and longer than 16
128     // beats.
129     return (burst == axi_pkg::BURST_INCR) & (len > 16);
130   end
131
132 // All other transactions are supported for splitting.
133   return 1'b1;
134 end
135 endfunction
136
137 assign sel_aw_unsupported = DisableChecks ? 1'b0 : ~txn_supported(slv_req.aw.atop,
138 slv_req.aw.burst, slv_req.aw.cache, slv_req.aw.len,
139 len_limit_i);
140
141 assign sel_ar_unsupported = DisableChecks ? 1'b0 : ~txn_supported('0, slv_req.ar.burst,
142 slv_req.ar.cache, slv_req.ar.len, len_limit_i);
143
144 // Respond to unsupported transactions with slave errors.
145 axi_err_slv #(
146   .AxiIdWidth ( Idwidth ),
147   .axi_req_t ( axi_req_t ),
148   .axi_resp_t ( axi_resp_t ),
149   .Resp ( axi_pkg::RESP_SLVERR ),
150   .ATOPs ( 1'b0 ), // The burst
151   .MaxTrans ( 1 ) // Splitting
152 ) i_err_slv (
153   .clk_i,
154   .rst_ni,
155   .slv_req_i ( unsupported_req ),
156   .slv_resp_o ( unsupported_resp )
157 );
158
159 // -----
160 // AW Channel
161 // -----
162 logic w_cnt_dec, w_cnt_req, w_cnt_gnt, w_cnt_err;
163 axi_pkg::len_t w_cnt_len;
164 axi_burst_splitter_gran_ax_chan #(
165   .chan_t ( axi_aw_chan_t ),
166   .IdWidth ( Idwidth ),
167   .MaxTxns ( MaxWriteTxns ),
168   .CutPath ( CutPath ),
169   .FullBW ( FullBW )
170 ) i_axi_burst_splitter_gran_aw_chan (
171   .clk_i,
172   .rst_ni,
173   .len_limit_i,
174   .ax_i ( act_req.aw ),
175   .ax_valid_i ( act_req.aw_valid ),
176   .ax_ready_o ( act_resp.aw_ready ),
177   .ax_o ( mst_req_o.aw ),
178   .ax_valid_o ( mst_req_o.aw_valid ),
179   .ax_ready_i ( mst_resp_i.aw_ready ),
180   .cnt_id_i ( mst_resp_i.b.id ),
181   .cnt_len_o ( w_cnt_len ),
182   .cnt_set_err_i ( mst_resp_i.b.resp[1] ),
183   .cnt_err_o ( w_cnt_err ),
184   .cnt_dec_i ( w_cnt_dec ),
185   .cnt_req_i ( w_cnt_req ),
186   .cnt_gnt_o ( w_cnt_gnt )
187 );
188
189 // -----
190 // W Channel
191 // -----
192 // keep a state where we are in the fragmentation of the w
193 axi_pkg::len_t w_len_d, w_len_q;
194 logic w_len_vld_d, w_len_vld_q;
195
196 // Feed through, except `last`, which needs to be modified
197 always_comb begin : proc_w_frag
198   mst_req_o.w = act_req.w;
199   w_len_d = w_len_q;
200   w_len_vld_d = w_len_vld_q;
201   // the entire detection machine is only required if
202   len_limit > 0
203   if (len_limit_i != 8'h00) begin
204     // In the case we are in the granular mode: last is from
205     req or when w_len valid and '0.
206     mst_req_o.w.last = (w_len_vld_q & (w_len_q == 8'h00)) |
207     act_req.w.last;
208     // only advance the machine if w ready and valid
209     if (act_resp.w_ready & act_req.w_valid) begin
210       // the counter is not yet valid, set it to valid and
211       initialize
212       if (!w_len_vld_q) begin
213         w_len_vld_d = 1'b1;
214         w_len_d = len_limit_i - 8'h01;
215       end else begin
216         w_len_d = w_len_q - 8'h01;
217         // in the last case, reinitialize the counter
218         if (w_len_q == 8'h00) begin
219           w_len_d = len_limit_i;
220         end
221       end
222     end
223     // final overwrite. if a downstream last comes, the
224     counter is invalid and set to 0
225     if (act_req.w.last) begin
226       w_len_vld_d = 1'b0;
227       w_len_d = 8'h00;
228     end
229   end
230   end else begin
231     // we operate in the legacy mode -> every w is last
232     mst_req_o.w.last = 1'b1;
233   end
234 end

```



```

227     end
228 end
229
230 assign mst_req_o.w_valid = act_req.w_valid;
231 assign act_resp.w_ready = mst_resp_i.w_ready;
232
233 // -----
234 // B Channel
235 // -----
236 // Filter B response, except for the last one
237 enum logic {BReady, BWait} b_state_d, b_state_q;
238 logic b_err_d, b_err_q;
239 always_comb begin
240     mst_req_o.b_ready = 1'b0;
241     act_resp.b       = '0;
242     act_resp.b_valid = 1'b0;
243     w_cnt_dec        = 1'b0;
244     w_cnt_req        = 1'b0;
245     b_err_d          = b_err_q;
246     b_state_d        = b_state_q;
247
248     unique case (b_state_q)
249     BReady: begin
250         if (mst_resp_i.b_valid) begin
251             w_cnt_req = 1'b1;
252             if (w_cnt_gnt) begin
253                 if (w_cnt_len < ({1'b0, len_limit_i} + 9'h001))
254                     act_resp.b = mst_resp_i.b;
255                 if (w_cnt_err) begin
256                     act_resp.b.resp = axi_pkg::RESP_SLVERR;
257                 end
258                 act_resp.b_valid = 1'b1;
259                 w_cnt_dec        = 1'b1;
260                 if (act_req.b_ready) begin
261                     mst_req_o.b_ready = 1'b1;
262                 end else begin
263                     b_state_d = BWait;
264                     b_err_d   = w_cnt_err;
265                 end
266             end else begin
267                 mst_req_o.b_ready = 1'b1;
268                 w_cnt_dec        = 1'b1;
269             end
270         end
271     end
272     BWait: begin
273         act_resp.b = mst_resp_i.b;
274         if (b_err_q) begin
275             act_resp.b.resp = axi_pkg::RESP_SLVERR;
276         end
277         act_resp.b_valid = 1'b1;
278         if (mst_resp_i.b_valid && act_req.b_ready) begin
279             mst_req_o.b_ready = 1'b1;
280             b_state_d        = BReady;
281         end
282     end
283     default: /*do nothing*/;
284 endcase
285 end
286
287 // -----
288 // AR Channel
289 // -----
290 // See description of `axi_chan` module.
291 logic r_cnt_dec, r_cnt_req, r_cnt_gnt;
292 axi_pkg::len_t r_cnt_len;
293 axi_burst_splitter_gran_axi_chan #(
294     .chan_t ( axi_ar_chan_t ),
295     .IdWidth ( IdWidth ),
296     .MaxTxns ( MaxReadTxns ),
297     .CutPath ( CutPath ),
298     .FullBW ( FullBW )
299 ) i_axi_burst_splitter_gran_ar_chan (
300     .clk_i,
301     .rst_ni,
302     .len_limit_i,
303     .ax_i ( act_req.ar ),
304     .ax_valid_i ( act_req.ar_valid ),
305     .ax_ready_o ( act_resp.ar_ready ),
306     .ax_o ( mst_req_o.ar ),
307     .ax_valid_o ( mst_req_o.ar_valid ),
308     .ax_ready_i ( mst_resp_i.ar_ready ),
309     .cnt_id_i ( mst_resp_i.r.id ),
310     .cnt_len_o ( r_cnt_len ),
311     .cnt_set_err_i ( 1'b0 ),
312     .cnt_err_o ( ),
313     .cnt_dec_i ( r_cnt_dec ),
314     .cnt_req_i ( r_cnt_req ),
315     .cnt_gnt_o ( r_cnt_gnt )
316 );
317
318 // -----
319 // R Channel
320 // -----
321 // Reconstruct `last`, feed rest through.
322 logic r_last_d, r_last_q;
323 enum logic {RFeedthrough, RWait} r_state_d, r_state_q;
324 always_comb begin
325     r_cnt_dec = 1'b0;

```

```

327     r_cnt_req = 1'b0;
328     r_last_d = r_last_q;
329     r_state_d = r_state_q;
330     mst_req_o.r_ready = 1'b0;
331     act_resp.r       = mst_resp_i.r;
332     act_resp.r.last = 1'b0;
333     act_resp.r_valid = 1'b0;
334
335     unique case (r_state_q)
336     RFeedthrough: begin
337         // If downstream has an R beat and the R counters can
338         // give us the remaining length of
339         // that burst, ...
340         if (mst_resp_i.r_valid) begin
341             // if downstream is last
342             if (mst_resp_i.r.last) begin
343                 r_cnt_req = 1'b1;
344                 if (r_cnt_gnt) begin
345                     r_last_d = (r_cnt_len < ({1'b0, len_limit_i} +
346                     9'h001));
347                     act_resp.r.last = r_last_d;
348                     // Decrement the counter.
349                     r_cnt_dec = 1'b1;
350                     // Try to forward the beat upstream.
351                     act_resp.r_valid = 1'b1;
352                     if (act_req.r_ready) begin
353                         // Acknowledge downstream.
354                         mst_req_o.r_ready = 1'b1;
355                     end else begin
356                         // Wait for upstream to become ready.
357                         r_state_d = RWait;
358                     end
359                 end else begin
360                     // downstream was not last, just a normal read to
361                     // pass through
362                     r_last_d = 1'b0;
363                     act_resp.r.last = r_last_d;
364                     // Try to forward the beat upstream.
365                     act_resp.r_valid = 1'b1;
366                     if (act_req.r_ready) begin
367                         // Acknowledge downstream.
368                         mst_req_o.r_ready = 1'b1;
369                     end else begin
370                         // Wait for upstream to become ready.
371                         r_state_d = RWait;
372                     end
373                 end
374             end
375             RWait: begin
376                 act_resp.r.last = r_last_q;
377                 act_resp.r_valid = mst_resp_i.r_valid;
378                 if (mst_resp_i.r_valid && act_req.r_ready) begin
379                     mst_req_o.r_ready = 1'b1;
380                     r_state_d = RFeedthrough;
381                 end
382             end
383             default: /*do nothing*/;
384         endcase
385     end
386
387 // -----
388 // Flip-Flops
389 // -----
390 `FF(b_err_q, b_err_d, 1'b0, clk_i, rst_ni)
391 `FF(b_state_q, b_state_d, BReady, clk_i, rst_ni)
392 `FF(r_last_q, r_last_d, 1'b0, clk_i, rst_ni)
393 `FF(r_state_q, r_state_d, RFeedthrough, clk_i, rst_ni)
394 `FF(w_len_q, w_len_d, 8'h00, clk_i, rst_ni)
395 `FF(w_len_vld_q, w_len_vld_d, 1'b0, clk_i, rst_ni)
396
397 // -----
398 // Assumptions and assertions
399 // -----
400 `ifndef VERILATOR
401 `ifndef XILINX_SIMULATOR
402 // pragma translate_off
403 default disable iff (!rst_ni);
404 // Inputs
405 assume property (@(posedge clk_i) slv_req_i.aw_valid |->
406     txn_supported(slv_req_i.aw.atop, slv_req_i.aw.burst,
407     slv_req_i.aw.cache, slv_req_i.aw.len,
408     len_limit_i)
409 ) else $warning("Unsupported AW transaction received, re-
410 turning slave error!");
411 assume property (@(posedge clk_i) slv_req_i.ar_valid |->
412     txn_supported('0, slv_req_i.ar.burst,
413     slv_req_i.ar.cache, slv_req_i.ar.len, len_limit_i)
414 ) else $warning("Unsupported AR transaction received, re-
415 turning slave error!");
416 // Outputs
417 assert property (@(posedge clk_i) mst_req_o.aw_valid |->
418     mst_req_o.aw.len <= len_limit_i)
419 else $fatal(1, "AW burst longer than a single beat emit-
420 ted!");
421 assert property (@(posedge clk_i) mst_req_o.ar_valid |->
422     mst_req_o.ar.len <= len_limit_i)
423 else $fatal(1, "AR burst longer than a single beat emit-
424 ted!");
425 initial begin

```



```

417     assume (AxilookBits > 0 && AxilookBits <= IdWidth) else
418     $fatal(1, "AxilookBits (%0d) must be in ]0, IdWidth
(00d)]", AxilookBits, IdWidth);
419 end
420 // pragma translate_on
421 `endif
422 `endif
423
424 endmodule
425
426
427
428 /// Internal module of ['axi_burst_splitter_gran'](mod-
ule.axi_burst_splitter_gran) to control
429 /// Ax channels.
430 ///
431 /// Store burst lengths in counters, which are associated to
AXI IDs through ID queues (to allow
432 /// reordering of responses w.r.t. requests).
433 module axi_burst_splitter_gran_ax_chan #(
434     parameter type      chan_t = logic,
435     parameter int unsigned IdWidth = 32'd0,
436     parameter int unsigned MaxTxns = 32'd0,
437     parameter bit        CutPath = 1'b0,
438     parameter bit        FullBW = 1'b0,
439     parameter type      id_t = logic[IdWidth-1:0]
440 ) (
441     input logic      clk_i,
442     input logic      rst_ni,
443
444     // length
445     input axi_pkg::len_t len_limit_i,
446
447     input chan_t      ax_i,
448     input logic      ax_valid_i,
449     output logic      ax_ready_o,
450     output chan_t      ax_o,
451     output logic      ax_valid_o,
452     input logic      ax_ready_i,
453
454     input id_t      cnt_id_i,
455     output axi_pkg::len_t cnt_len_o,
456     input logic      cnt_set_err_i,
457     output logic      cnt_err_o,
458     input logic      cnt_dec_i,
459     input logic      cnt_req_i,
460     output logic      cnt_gnt_o
461 );
462
463 typedef logic[IdWidth-1:0] cnt_id_t;
464 typedef logic[axi_pkg::LenWidth:0] num_beats_t;
465
466 chan_t ax_d, ax_q;
467 // keep the number of remaining beats. != len
468 num_beats_t num_beats_d, num_beats_q;
469 // maximum number of beats to subtract in one go
470 num_beats_t max_beats;
471
472
473 logic cnt_alloc_req, cnt_alloc_gnt;
474 axi_burst_splitter_gran_counters #(
475     .MaxTxns ( MaxTxns ),
476     .IdWidth ( IdWidth ),
477     .CutPath ( CutPath ),
478     .FullBW ( FullBW )
479 ) i_axi_burst_splitter_gran_counters (
480     .clk_i,
481     .rst_ni,
482     .alloc_id_i ( ax_i.id ),
483     .alloc_len_i ( ax_i.len ),
484     .alloc_req_i ( cnt_alloc_req ),
485     .alloc_gnt_o ( cnt_alloc_gnt ),
486     .cnt_id_i ( cnt_id_i ),
487     .cnt_len_o ( cnt_len_o ),
488     .cnt_set_err_i ( cnt_set_err_i ),
489     .cnt_err_o ( cnt_err_o ),
490     .cnt_dec_i ( cnt_dec_i ),
491     .cnt_delta_i ( max_beats ),
492     .cnt_req_i ( cnt_req_i ),
493     .cnt_gnt_o ( cnt_gnt_o )
494 );
495
496 // assign the max_beats depending on the limit value. Limit
value is given as an AXI len.
497 // limit = 0 means one beat each AX
498 assign max_beats = {1'b0, len_limit_i} + 9'h001;
499
500 enum logic {Idle, Busy} state_d, state_q;
501 always_comb begin
502     cnt_alloc_req = 1'b0;
503     ax_d = ax_q;
504     state_d = state_q;
505     num_beats_d = num_beats_q;
506     ax_o = '0;
507     ax_valid_o = 1'b0;
508     ax_ready_o = 1'b0;
509     unique case (state_q)
510         Idle: begin
511             if (ax_valid_i && cnt_alloc_gnt) begin
512
513                 // No splitting required -> feed through.

```

```

514             if (ax_i.len <= len_limit_i) begin
515                 ax_o = ax_i;
516                 ax_valid_o = 1'b1;
517                 // As soon as downstream is ready, allocate a
counter and acknowledge upstream.
518                 if (ax_ready_i) begin
519                     cnt_alloc_req = 1'b1;
520                     ax_ready_o = 1'b1;
521                 end
522
523                 // Splitting required.
524             end else begin
525                 // Store Ax, allocate a counter, and acknowledge
upstream.
526                 ax_d = ax_i;
527                 cnt_alloc_req = 1'b1;
528                 ax_ready_o = 1'b1;
529                 // As burst is too long, we will need to send mul-
tiple
530                 state_d = Busy;
531                 num_beats_d = ({1'b0, ax_i.len} + 9'h001);
532                 // Try to feed first burst through.
533                 ax_o = ax_d;
534                 // if we are here we can send the length limit
once for sure
535                 ax_o.len = len_limit_i;
536                 ax_valid_o = 1'b1;
537                 if (ax_ready_i) begin
538                     // Reduce number of bursts still to be sent by
one and increment address.
539                     num_beats_d = ({1'b0, ax_i.len} + 9'h001) -
max_beats;
540                     if (ax_d.burst == axi_pkg::BURST_INCR) begin
541                         // Align
542                         ax_d.addr =
axi_pkg::aligned_addr(axi_pkg::largest_addr_t'(ax_d.addr),
ax_d.size);
543                         // modify the address
544                         ax_d.addr += (1 << ax_d.size) * max_beats;
545                     end
546                 end
547             end
548         end
549     end
550     Busy: begin
551         // Sent next burst from split.
552         ax_o = ax_q;
553         ax_valid_o = 1'b1;
554         // emit the proper length
555         if (num_beats_q <= max_beats) begin
556             // this is the remainder
557             ax_o.len = axi_pkg::len_t'(num_beats_q - 9'h001);
558         end else begin
559             ax_o.len = len_limit_i;
560         end
561         // next state
562         if (ax_ready_i) begin
563             if (num_beats_q <= max_beats) begin
564                 // If this was the last burst, go back to idle.
565                 state_d = Idle;
566             end else begin
567                 // Otherwise, continue with the next burst.
568                 num_beats_d = num_beats_q - max_beats;
569                 if (ax_q.burst == axi_pkg::BURST_INCR) begin
570                     ax_d.addr =
axi_pkg::aligned_addr(axi_pkg::largest_addr_t'(ax_q.addr),
ax_q.size);
571                     ax_d.addr += (1 << ax_q.size) * max_beats;
572                 end
573             end
574         end
575     end
576     default: /*do nothing*/;
577 endcase
578 end
579
580 // registers
581 `FF(ax_q, ax_d, '0, clk_i, rst_ni)
582 `FF(state_q, state_d, Idle, clk_i, rst_ni)
583 `FF(num_beats_q, num_beats_d, 9'h000, clk_i, rst_ni)
584 endmodule
585
586
587
588 /// Internal module of ['axi_burst_splitter_gran'](mod-
ule.axi_burst_splitter_gran) to order
589 /// transactions.
590 module axi_burst_splitter_gran_counters #(
591     parameter int unsigned MaxTxns = 32'd0,
592     parameter int unsigned IdWidth = 32'd0,
593     parameter bit        CutPath = 1'b0,
594     parameter bit        FullBW = 1'b0,
595     parameter type      id_t = logic [IdWidth-1:0],
596     parameter type      cnt_t = logic [axi_pkg::LenWidth:0]
597 ) (
598     input logic      clk_i,
599     input logic      rst_ni,
600
601     input id_t      alloc_id_i,
602     input axi_pkg::len_t alloc_len_i,
603     input logic      alloc_req_i,

```

```

604 output logic      alloc_gnt_o,
605
606 input  id_t        cnt_id_i,
607 output axi_pkg::len_t cnt_len_o,
608 input  logic        cnt_set_err_i,
609 output logic        cnt_err_o,
610 input  logic        cnt_dec_i,
611 input  cnt_t        cnt_delta_i,
612 input  logic        cnt_req_i,
613 output logic        cnt_gnt_o
614 );
615
616 // the allocation interface can be cut
617 typedef struct packed {
618     id_t        id;
619     axi_pkg::len_t len;
620 } alloc_pld_t;
621
622 alloc_pld_t alloc_pld_in, alloc_pld_out;
623 logic      alloc_req;
624 logic      alloc_gnt;
625
626 assign alloc_pld_in.id = alloc_id_i;
627 assign alloc_pld_in.len = alloc_len_i;
628
629 if (CutPath) begin : gen_spill
630     cc_spill_register #(
631         .data_t ( alloc_pld_t ),
632         .Bypass ( 1'b0 )
633     ) i_spill_register_alloc (
634         .clk_i,
635         .rst_ni,
636         .clr_i ( 1'b0 ),
637         .valid_i ( alloc_req_i ),
638         .ready_o ( alloc_gnt_o ),
639         .data_i ( alloc_pld_in ),
640         .valid_o ( alloc_req ),
641         .ready_i ( alloc_gnt ),
642         .data_o ( alloc_pld_out )
643     );
644 end else begin : gen_no_spill
645     assign alloc_req = alloc_req_i;
646     assign alloc_gnt_o = alloc_gnt;
647     assign alloc_pld_out = alloc_pld_in;
648 end
649
650 localparam int unsigned CntIdxWidth = (MaxTxns > 1) ? $clog2(MaxTxns) : 32'd1;
651 typedef logic [CntIdxWidth-1:0] cnt_idx_t;
652 logic [MaxTxns-1:0] cnt_dec, cnt_free, cnt_set, err_d,
err_q, cnt_clr;
653 cnt_t cnt_inp;
654 cnt_t [MaxTxns-1:0] cnt_outp;
655 cnt_idx_t cnt_free_idx, cnt_r_idx;
656 for (genvar i = 0; i < MaxTxns; i++) begin : gen_cnt
657     cc_delta_counter #(
658         .Width ( $bits(cnt_t) )
659     ) i_cnt (
660         .clk_i,
661         .rst_ni,
662         .clr_i ( cnt_clr[i] ),
663         .en_i ( cnt_dec[i] ),
664         .load_i ( cnt_set[i] ),
665         .down_i ( 1'b1 ),
666         .delta_i ( cnt_delta_i ),
667         .d_i ( cnt_inp ),
668         .q_o ( cnt_outp[i] ),
669         .overflow_o ( cnt_clr[i] )
670     );
671     assign cnt_free[i] = (cnt_outp[i] == '0');
672 end
673 assign cnt_inp = {1'b0, alloc_pld_out.len} + 1;
674
675 cc_lzc #(
676     .Width ( MaxTxns ),
677     .Mode ( cc_pkg::LZC_TRAILING_ZERO_CNT )
678 ) i_lzc (
679     .in_i ( cnt_free ),
680     .cnt_o ( cnt_free_idx ),
681     .empty_o ( )
682 );
683
684 logic idq_inp_req, idq_inp_gnt,
685 idq_outp_gnt, idq_outp_valid, idq_outp_pop;
686 cc_id_queue #(
687     .IdWidth ( $bits(id_t) ),
688     .Capacity ( MaxTxns ),
689     .FullBw ( FullBW ),
690     .data_t ( cnt_idx_t )
691 ) i_idq (
692     .clk_i,
693     .rst_ni,
694     .clr_i ( 1'b0 ),
695     .inp_id_i ( alloc_pld_out.id ),
696     .inp_data_i ( cnt_free_idx ),
697     .inp_req_i ( idq_inp_req ),
698     .inp_gnt_o ( idq_inp_gnt ),
699     .exists_data_i ( '0 ),
700     .exists_mask_i ( '0 ),
701     .exists_req_i ( 1'b0 ),
702     .exists_o ( /* keep open */,

```

```

703     .exists_gnt_o ( /* keep open */,
704     .outp_id_i ( cnt_id_i ),
705     .outp_pop_i ( idq_outp_pop ),
706     .outp_req_i ( cnt_req_i ),
707     .outp_data_o ( cnt_r_idx ),
708     .outp_data_valid_o ( idq_outp_valid ),
709     .outp_gnt_o ( idq_outp_gnt ),
710     .full_o ( /* keep open */,
711     .empty_o ( /* keep open */,
712 );
713 assign idq_inp_req = alloc_req & alloc_gnt;
714 assign alloc_gnt = idq_inp_gnt & !(cnt_free);
715 assign cnt_gnt_o = idq_outp_gnt & idq_outp_valid;
716 logic [8:0] read_len;
717 assign read_len = cnt_outp[cnt_r_idx] - 1;
718 assign cnt_len_o = read_len[7:0];
719
720 assign idq_outp_pop = cnt_req_i & cnt_gnt_o & cnt_dec_i &
(cnt_len_o < cnt_delta_i);
721 always_comb begin
722     cnt_dec = '0;
723     cnt_dec[cnt_r_idx] = cnt_req_i & cnt_gnt_o & cnt_dec_i;
724 end
725 always_comb begin
726     cnt_set = '0;
727     cnt_set[cnt_free_idx] = alloc_req & alloc_gnt;
728 end
729 always_comb begin
730     err_d = err_q;
731     cnt_err_o = err_q[cnt_r_idx];
732     if (cnt_req_i && cnt_gnt_o && cnt_set_err_i) begin
733         err_d[cnt_r_idx] = 1'b1;
734         cnt_err_o = 1'b1;
735     end
736     if (alloc_req && alloc_gnt) begin
737         err_d[cnt_free_idx] = 1'b0;
738     end
739 end
740
741 // registers
742 `FF(err_q, err_d, '0, clk_i, rst_ni)
743
744 `ifndef VERILATOR
745 // pragma translate_off
746 assume property (@(posedge clk_i) idq_outp_gnt |->
747 idq_outp_valid)
748     else $warning("Invalid output at ID queue, read not granted!");
749 // pragma translate_on
750 `endif
751 endmodule

```

4. pulp-platform_axi/src/axi_burst_unwrap.sv (656 lines)

```

1 // Copyright (c) 2023 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
4 // pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 // file except in
8 // compliance with the License. You may obtain a copy of the
9 // License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 // applicable law
12 // or agreed to in writing, software, hardware and materials
13 // distributed under
14 // this license is distributed on an "AS IS" BASIS, WITHOUT
15 // WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 // License for the
18 // specific language governing permissions and limitations under
19 // the License.
20 //
21 // Authors:
22 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
23 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
24 // - Nils Wistoff <nwistoff@iis.ee.ethz.ch>
25
26 `include "axi/typedef.svh"
27 `include "common_cells/registers.svh"
28
29 /// Splits wrapping AXI4 bursts into incremental bursts.
30 ///
31 /// ## Limitations
32 ///
33 /// - This module does not support atomic operations (ATOPs)
34 // and responds to ATOPs with a slave
35 // error. Place an [axi_atop_filter](mod-
36 // ule.axi_atop_filter) before this module if upstream
37 // modules can generate ATOPs.
38 module axi_burst_unwrap #(
39     // Maximum number of AXI read bursts outstanding at the same
40     // time
41     parameter int unsigned MaxReadTxns = 32'd0,
42     // Maximum number of AXI write bursts outstanding at the
43     // same time
44     parameter int unsigned MaxWriteTxns = 32'd0,

```

```

32 // AXI Bus Types
33 parameter int unsigned AddrWidth  = 32'd0,
34 parameter int unsigned DataWidth  = 32'd0,
35 parameter int unsigned IdWidth    = 32'd0,
36 // Number of least-significant ID bits used by the internal
demux to track in-flight
37 // transactions (0 < AxiLookBits <= IdWidth), see doc/
axi_demux.md for the trade-off.
38 parameter int unsigned AxiLookBits = IdWidth,
39 parameter int unsigned UserWidth   = 32'd0,
40 parameter type      axi_req_t      = logic,
41 parameter type      axi_resp_t     = logic
42 ) (
43   input logic  clk_i,
44   input logic  rst_ni,
45
46   // Input / Slave Port
47   input axi_req_t slv_req_i,
48   output axi_resp_t slv_resp_o,
49
50   // Output / Master Port
51   output axi_req_t mst_req_o,
52   input axi_resp_t mst_resp_i
53 );
54
55 typedef logic [AddrWidth-1:0] addr_t;
56 typedef logic [DataWidth-1:0] data_t;
57 typedef logic [IdWidth-1:0] id_t;
58 typedef logic [DataWidth/8-1:0] strb_t;
59 typedef logic [UserWidth-1:0] user_t;
60 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
61 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
62 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
63 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
64 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
65
66 // Demultiplex between supported and unsupported transactions.
67 axi_req_t act_req, unsupported_req;
68 axi_resp_t act_resp, unsupported_resp;
69 logic sel_aw_unsupported, sel_ar_unsupported;
70 localparam int unsigned MaxTxns = (MaxReadTxns > MaxWriteTxns) ? MaxReadTxns : MaxWriteTxns;
71 axi_demux #(
72   .AxiIdWidth ( IdWidth ),
73   .aw_chan_t ( aw_chan_t ),
74   .w_chan_t ( w_chan_t ),
75   .b_chan_t ( b_chan_t ),
76   .ar_chan_t ( ar_chan_t ),
77   .r_chan_t ( r_chan_t ),
78   .axi_req_t ( axi_req_t ),
79   .axi_resp_t ( axi_resp_t ),
80   .NoMstPorts ( 2 ),
81   .MaxTrans ( MaxTxns ),
82   .AxiLookBits ( AxiLookBits ),
83   .SpillAw ( 1'b0 ),
84   .SpillW ( 1'b0 ),
85   .SpillB ( 1'b0 ),
86   .SpillAr ( 1'b0 ),
87   .SpillR ( 1'b0 )
88 ) i_demux_supported_vs_unsupported (
89   .clk_i,
90   .rst_ni,
91   .slv_req_i,
92   .slv_aw_select_i ( sel_aw_unsupported ),
93   .slv_ar_select_i ( sel_ar_unsupported ),
94   .slv_resp_o,
95   .mst_reqs_o ( {unsupported_req, act_req} ),
96   .mst_resps_i ( {unsupported_resp, act_resp} )
97 );
98 // Define supported transactions.
99 function bit txn_supported(axi_pkg::atop_t atop,
axi_pkg::burst_t burst, axi_pkg::cache_t cache,
100   axi_pkg::len_t len);
101 // Single-beat transactions do not need splitting, so all
are supported.
102 if (len == '0) return 1'b1;
103 // ATOPs are not supported.
104 if (atop != '0) return 1'b0;
105 // The AXI Spec (A3.4.1) only allows splitting non-modifiable transactions ..
106 // ... but this module only splits WRAP bursts, so ignore
all others
107 if (!axi_pkg::modifiable(cache) && (burst ==
axi_pkg::BURST_WRAP)) begin
108   return 1'b0;
109 end
110 // All other transactions are supported.
111 return 1'b1;
112 endfunction
113 assign sel_aw_unsupported = ~txn_supported(
slv_req_i.aw.atop, slv_req_i.aw.burst,
114   slv_req_i.aw.cache, slv_req_i.aw.len);
115 assign sel_ar_unsupported = ~txn_supported('0,
slv_req_i.ar.burst,
116   slv_req_i.ar.cache, slv_req_i.ar.len);
117 // Respond to unsupported transactions with slave errors.
118 axi_err_slv #(
119   .AxiIdWidth ( IdWidth ),

```

```

120   .axi_req_t ( axi_req_t ),
121   .axi_resp_t ( axi_resp_t ),
122   .Resp ( axi_pkg::RESP_SLVERR ),
123   .ATOPs ( 1'b0 ), // The burst
splitter does not support ATOPs.
124   .MaxTrans ( 1 ) // Splitting
bursts implies a low-performance bus.
125 ) i_err_slv (
126   .clk_i,
127   .rst_ni,
128   .slv_req_i ( unsupported_req ),
129   .slv_resp_o ( unsupported_resp )
130 );
131
132 // -----
133 // AW Channel
134 // -----
135 logic b_cnt_dec, b_cnt_req, b_cnt_gnt, b_cnt_err;
136 logic w_cnt_dec, w_cnt_req, w_cnt_gnt;
137 axi_pkg::len_t b_cnt_len, w_cnt_len;
138 axi_burst_unwrap_ax_chan #(
139   .AwChan ( 1'b1 ),
140   .chan_t ( aw_chan_t ),
141   .AddrWidth ( AddrWidth ),
142   .IdWidth ( IdWidth ),
143   .MaxTxns ( MaxWriteTxns )
144 ) i_axi_burst_unwrap_aw_chan (
145   .clk_i,
146   .rst_ni,
147   .ax_i ( act_req.aw ),
148   .ax_valid_i ( act_req.aw_valid ),
149   .ax_ready_o ( act_resp.aw_ready ),
150   .ax_o ( mst_req_o.aw ),
151   .ax_valid_o ( mst_req_o.aw_valid ),
152   .ax_ready_i ( mst_resp_i.aw_ready ),
153   .cnt_id_i ( mst_resp_i.b.id ),
154   .cnt_len_o ( {w_cnt_len, b_cnt_len} ),
155   .cnt_set_err_i ( mst_resp_i.b.resp[i] ),
156   .cnt_err_o ( b_cnt_err ),
157   .cnt_dec_i ( {w_cnt_dec, b_cnt_dec} ),
158   .cnt_req_i ( {w_cnt_req, b_cnt_req} ),
159   .cnt_gnt_o ( {w_cnt_gnt, b_cnt_gnt} )
160 );
161
162 // -----
163 // W Channel
164 // -----
165 // Set 'last' flag in last beat of downstream burst
166 logic w_last_d, w_last_q;
167 enum logic [1:0] {WReady, WWait, WFeedthrough} w_state_d,
w_state_q;
168 always_comb begin
169   w_cnt_dec = 1'b0;
170   w_cnt_req = 1'b0;
171   w_last_d = w_last_q;
172   w_state_d = w_state_q;
173   mst_req_o.w_valid = 1'b0;
174   mst_req_o.w = act_req.w;
175   act_resp.w_ready = 1'b0;
176
177   unique case (w_state_q)
178     WReady: begin
179       if (act_req.w_valid) begin
180         w_cnt_req = 1'b1;
181         if (w_cnt_gnt) begin
182           w_last_d = act_req.w.last | (w_cnt_len == 8'd0);
183           mst_req_o.w.last = w_last_d;
184           w_cnt_dec = 1'b1;
185           // Try to forward the beat downstream.
186           mst_req_o.w_valid = 1'b1;
187           if (mst_resp_i.w_ready) begin
188             act_resp.w_ready = 1'b1;
189             if (w_last_d && !act_req.w.last) begin
190               w_state_d = WFeedthrough;
191             end
192           end else begin
193             w_state_d = WWait;
194           end
195         end // if (w_cnt_gnt)
196       end // if (act_req.w_valid)
197     end // case: WReady
198     WWait: begin
199       mst_req_o.w.last = w_last_q;
200       mst_req_o.w_valid = 1'b1;
201       if (mst_resp_i.w_ready) begin
202         act_resp.w_ready = 1'b1;
203         w_state_d = (!w_last_q || act_req.w.last) ?
WReady : WFeedthrough;
204       end
205     end
206     WFeedthrough: begin
207       // Feed through second incremental burst.
208       mst_req_o.w_valid = act_req.w_valid;
209       act_resp.w_ready = mst_resp_i.w_ready;
210       if (act_req.w_valid && mst_resp_i.w_ready &&
act_req.w.last) begin
211         w_state_d = WReady;
212       end
213     end
214     default: /*do nothing*/;
215   endcase // unique case (w_state_q)

```

```

216 end
217
218 // -----
219 // B Channel
220 // -----
221 // Filter B response, except for the last one
222 enum logic {BReady, BWait} b_state_d, b_state_q;
223 logic b_err_d, b_err_q;
224 always_comb begin
225     mst_req_o.b_ready = 1'b0;
226     act_resp.b        = '0;
227     act_resp.b_valid   = 1'b0;
228     b_cnt_dec          = 1'b0;
229     b_cnt_req          = 1'b0;
230     b_err_d            = b_err_q;
231     b_state_d          = b_state_q;
232
233     unique case (b_state_q)
234     BReady: begin
235         if (mst_resp.i.b_valid) begin
236             b_cnt_req = 1'b1;
237             if (b_cnt_gnt) begin
238                 if (b_cnt_len == 8'd0) begin
239                     act_resp.b = mst_resp.i.b;
240                     if (b_cnt_err) begin
241                         act_resp.b.resp = axi_pkg::RESP_SLVERR;
242                     end
243                     act_resp.b_valid = 1'b1;
244                     b_cnt_dec = 1'b1;
245                     if (act_req.b_ready) begin
246                         mst_req_o.b_ready = 1'b1;
247                     end else begin
248                         b_state_d = BWait;
249                         b_err_d = b_cnt_err;
250                     end
251                 end else begin
252                     mst_req_o.b_ready = 1'b1;
253                     b_cnt_dec = 1'b1;
254                 end
255             end
256         end
257     end
258     BWait: begin
259         act_resp.b = mst_resp.i.b;
260         if (b_err_q) begin
261             act_resp.b.resp = axi_pkg::RESP_SLVERR;
262         end
263         act_resp.b_valid = 1'b1;
264         if (mst_resp.i.b_valid && act_req.b_ready) begin
265             mst_req_o.b_ready = 1'b1;
266             b_state_d = BReady;
267         end
268     end
269     default: /*do nothing*/;
270 endcase
271 end
272
273 // -----
274 // AR Channel
275 // -----
276 // See description of `axi_chan` module.
277 logic r_cnt_dec, r_cnt_req, r_cnt_gnt, unc0;
278 axi_pkg::len_t r_cnt_len, unc1;
279 axi_burst_unwrap_ax_chan #(
280     .AwChan ( 0 ),
281     .chan_t ( ar_chan_t ),
282     .AddrWidth ( AddrWidth ),
283     .IdWidth ( IdWidth ),
284     .MaxTxns ( MaxReadTxns )
285 ) i_axi_burst_unwrap_ar_chan (
286     .clk_i,
287     .rst_ni,
288     .ax_i ( act_req.ar ),
289     .ax_valid_i ( act_req.ar_valid ),
290     .ax_ready_o ( act_resp.ar_ready ),
291     .ax_o ( mst_req_o.ar ),
292     .ax_valid_o ( mst_req_o.ar_valid ),
293     .ax_ready_i ( mst_resp.i.ar_ready ),
294     .cnt_id_i ( mst_resp.i.r.id ),
295     .cnt_len_o ( { unc1, r_cnt_len } ),
296     .cnt_set_err_i ( 1'b0 ),
297     .cnt_err_o ( ),
298     .cnt_dec_i ( { 1'b0, r_cnt_dec } ),
299     .cnt_req_i ( { 1'b0, r_cnt_req } ),
300     .cnt_gnt_o ( { unc0, r_cnt_gnt } )
301 );
302
303 // -----
304 // R Channel
305 // -----
306 // Reconstruct `last`, feed rest through.
307 logic r_last_d, r_last_q;
308 enum logic {RFeedthrough, RWait} r_state_d, r_state_q;
309 always_comb begin
310     r_cnt_dec = 1'b0;
311     r_cnt_req = 1'b0;
312     r_last_d = r_last_q;
313     r_state_d = r_state_q;
314     mst_req_o.r_ready = 1'b0;
315     act_resp.r        = mst_resp.i.r;
316     act_resp.r.last   = 1'b0;
317
318     act_resp.r_valid = 1'b0;
319
320     unique case (r_state_q)
321     RFeedthrough: begin
322         // If downstream has an R beat and the R counters can
323         // give us the remaining length of
324         // that burst, ...
325         if (mst_resp.i.r_valid) begin
326             r_cnt_req = 1'b1;
327             if (r_cnt_gnt) begin
328                 r_last_d = (r_cnt_len == 8'd0);
329                 act_resp.r.last = r_last_d;
330                 // Decrement the counter.
331                 r_cnt_dec = 1'b1;
332                 // Try to forward the beat upstream.
333                 act_resp.r_valid = 1'b1;
334                 if (act_req.r_ready) begin
335                     // Acknowledge downstream.
336                     mst_req_o.r_ready = 1'b1;
337                 end else begin
338                     // Wait for upstream to become ready.
339                     r_state_d = RWait;
340                 end
341             end
342         end
343     end
344     RWait: begin
345         act_resp.r.last = r_last_q;
346         act_resp.r_valid = mst_resp.i.r_valid;
347         if (mst_resp.i.r_valid && act_req.r_ready) begin
348             mst_req_o.r_ready = 1'b1;
349             r_state_d = RFeedthrough;
350         end
351     end
352     default: /*do nothing*/;
353 endcase
354 end
355
356 // -----
357 // Flip-Flops
358 // -----
359 `FF(b_err_q, b_err_d, 1'b0, clk_i, rst_ni)
360 `FF(b_state_q, b_state_d, BReady, clk_i, rst_ni)
361 `FF(r_last_q, r_last_d, 1'b0, clk_i, rst_ni)
362 `FF(r_state_q, r_state_d, RFeedthrough, clk_i, rst_ni)
363 `FF(w_last_q, w_last_d, 1'b0, clk_i, rst_ni)
364 `FF(w_state_q, w_state_d, WReady, clk_i, rst_ni)
365
366 // -----
367 // Assumptions and assertions
368 // -----
369 `ifndef VERILATOR
370 `ifndef XILINX_SIMULATOR
371 // pragma translate_off
372 // pragma translate_on
373 // Inputs
374 assume property (@(posedge clk_i) slv_req.i.aw_valid |->
375     txn_supported(slv_req.i.aw.atop, slv_req.i.aw.burst,
376     slv_req.i.aw.cache, slv_req.i.aw.len)
377 ) else $warning("Unsupported AW transaction received, re-
378 turning slave error!");
379 assume property (@(posedge clk_i) slv_req.i.ar_valid |->
380     txn_supported('0, slv_req.i.ar.burst,
381     slv_req.i.ar.cache, slv_req.i.ar.len)
382 ) else $warning("Unsupported AR transaction received, re-
383 turning slave error!");
384 assume property (@(posedge clk_i) slv_req.i.aw_valid |->
385     slv_req.i.aw.atop == '0 || slv_req.i.aw.atop[5:4] ==
386     axi_pkg::ATOP_ATOMICSTORE
387 ) else $fatal(1, "Unsupported ATOP that gives rise to a R
388 response received,\
389 cannot respond in protocol-compliant
390 manner!");
391 // pragma translate_on
392 `endif
393 `endif
394 endmodule
395
396 /// Internal module of [ `axi_burst_splitter` ] (mod-
397 ule.axi_burst_splitter) to control Ax channels.
398 ///
399 /// Store burst lengths in counters, which are associated to
400 AXI IDs through ID queues (to allow
401 reordering of responses w.r.t. requests).
402 module axi_burst_unwrap_ax_chan #(
403     parameter bit AwChan = 0,
404     parameter type chan_t = logic,
405     parameter int unsigned AddrWidth = 0,
406     parameter int unsigned IdWidth = 0,
407     parameter int unsigned MaxTxns = 0,
408     parameter type id_t = logic[IdWidth-1:0]
409 ) (
410     input logic clk_i,
411     input logic rst_ni,
412     input chan_t ax_i,
413     input logic ax_valid_i,
414     output logic ax_ready_o,
415     output chan_t ax_o,
416     output logic ax_valid_o,

```

```

408 input logic ax_ready_i,
409
410 input id_t cnt_id_i,
411 output axi_pkg::len_t [1:0] cnt_len_o,
412 input logic cnt_set_err_i,
413 output logic cnt_err_o,
414 input logic [1:0] cnt_dec_i,
415 input logic [1:0] cnt_req_i,
416 output logic [1:0] cnt_gnt_o
417 );
418 typedef logic[IdWidth-1:0] cnt_id_t;
419
420 logic [1:0] cnt_alloc_req, cnt_alloc_gnt;
421 axi_pkg::len_t split_len, alloc_len_0;
422
423 // Count down from upstream burst len to generate upstream
last flag.
424 axi_burst_counters #(
425     .MaxTxns ( MaxTxns ),
426     .IdWidth ( IdWidth )
427 ) i_axi_burst_counters0 (
428     .clk_i,
429     .rst_ni,
430     .alloc_id_i ( ax_i.id ),
431     .alloc_len_i ( alloc_len_0 ),
432     .alloc_req_i ( cnt_alloc_req[0] ),
433     .alloc_gnt_o ( cnt_alloc_gnt[0] ),
434     .cnt_id_i ( cnt_id_i ),
435     .cnt_len_o ( cnt_len_o[0] ),
436     .cnt_set_err_i ( cnt_set_err_i ),
437     .cnt_err_o ( cnt_err_o ),
438     .cnt_dec_i ( cnt_dec_i[0] ),
439     .cnt_req_i ( cnt_req_i[0] ),
440     .cnt_gnt_o ( cnt_gnt_o[0] )
441 );
442
443 // For AR we need to count the r responses, for AW we need
to count downstream b responses
444 assign alloc_len_0 = AwChan ? split_len : ax_i.len;
445
446 // For AwChan, count down W beats until end of first incr
burst to set last flag.
447 // Since the W channel is ordered and has no IDs, just use
the same ID for all transactions to
448 // implement a FIFO. TODO: replace internal ID queue with
FIFO
449 axi_burst_counters #(
450     .MaxTxns ( MaxTxns ),
451     .IdWidth ( 1 )
452 ) i_axi_burst_counters1 (
453     .clk_i,
454     .rst_ni,
455     .alloc_id_i ( 1'b0 ),
456     .alloc_len_i ( ax_o.len ),
457     .alloc_req_i ( cnt_alloc_req[1] ),
458     .alloc_gnt_o ( cnt_alloc_gnt[1] ),
459     .cnt_id_i ( 1'b0 ),
460     .cnt_len_o ( cnt_len_o[1] ),
461     .cnt_set_err_i ( 1'b0 ),
462     .cnt_err_o ( ),
463     .cnt_dec_i ( cnt_dec_i[1] ),
464     .cnt_req_i ( cnt_req_i[1] ),
465     .cnt_gnt_o ( cnt_gnt_o[1] )
466 );
467
468 chan_t ax_d, ax_q;
469
470 logic [10:0] container_size;
471 logic [AddrWidth-1:0] wrap_boundary;
472
473 // The total size of this burst (beat_size * burst_length)
assign container_size = (ax_i.len + 1) << ax_i.size;
474
475 // For wrapping bursts, this returns the wrap boundary (con-
tainer size is power of two according to A.3.4.1)
476 assign wrap_boundary = ax_i.addr & ~(Ad-
drWidth'(container_size) - 1);
477
478 enum logic {Idle, Busy} state_d, state_q;
479 always_comb begin
480     cnt_alloc_req = '0;
481     ax_d = ax_q;
482     state_d = state_q;
483     ax_o = '0;
484     ax_valid_o = 1'b0;
485     ax_ready_o = 1'b0;
486     split_len = 8'd0;
487     unique case (state_q)
488         Idle: begin
489             if (ax_valid_i && &cnt_alloc_gnt) begin
490                 if (ax_i.burst == axi_pkg::BURST_WRAP && ax_i.addr !
= wrap_boundary) begin // Splitting required.
491                     // Store Ax, allocate a counter, and acknowledge
upstream.
492                         ax_d = ax_i;
493                         ax_d.burst = axi_pkg::BURST_INCR;
494                         split_len = 8'd1;
495                         // Try to feed first burst through.
496                         ax_o = ax_d;
497                         // First (this) incr burst from addr to wrap
boundary + container size
498                         ax_o.len = ((wrap_boundary + container_size -

```

```

ax_i.addr) >> ax_i.size) - 1;
499 // Next incr burst from wrap boundary to addr
500 ax_d.len = ((ax_i.addr - wrap_boundary) >>
ax_i.size) - 1;
501 ax_d.addr = wrap_boundary;
502 ax_valid_o = 1'b1;
503 if (ax_ready_i) begin
504     ax_ready_o = 1'b1;
505     state_d = Busy;
506     // Allocate second counter only for AwChan
cnt_alloc_req = { AwChan, 1'b1 };
507 end
508 end else begin // No splitting required -> feed
through.
509     ax_o = ax_i;
510     // A wrapping burst starting on the wrap boundary
maps directly to an incrementing burst.
511     if (ax_i.burst == axi_pkg::BURST_WRAP) begin
512         ax_o.burst = axi_pkg::BURST_INCR;
513     end
514     ax_valid_o = 1'b1;
515     // As soon as downstream is ready, allocate a
counter and acknowledge upstream.
516     if (ax_ready_i) begin
517         cnt_alloc_req = { AwChan, 1'b1 };
518         ax_ready_o = 1'b1;
519     end
520 end
521 end
522 end
523 end
524 Busy: begin
525     // Sent next burst from split.
526     ax_o = ax_q;
527     ax_valid_o = 1'b1;
528     if (ax_ready_i) begin
529         state_d = Idle;
530     end
531 end
532 default: /*do nothing*/;
533 endcase
534 end
535
536 // registers
537 `FF(ax_q, ax_d, '0, clk_i, rst_ni)
538 `FF(state_q, state_d, Idle, clk_i, rst_ni)
539 endmodule
540
541 /// Internal module of [`axi_burst_splitter`](mod-
ule.axi_burst_splitter) to order transactions.
542 module axi_burst_counters #(
543     parameter int unsigned MaxTxns = 0,
544     parameter int unsigned IdWidth = 0,
545     parameter type id_t = logic [IdWidth-1:0]
546 ) (
547     input logic clk_i,
548     input logic rst_ni,
549
550     input id_t alloc_id_i,
551     input axi_pkg::len_t alloc_len_i,
552     input logic alloc_req_i,
553     output logic alloc_gnt_o,
554
555     input id_t cnt_id_i,
556     output axi_pkg::len_t cnt_len_o,
557     input logic cnt_set_err_i,
558     output logic cnt_err_o,
559     input logic cnt_dec_i,
560     input logic cnt_req_i,
561     output logic cnt_gnt_o
562 );
563 localparam int unsigned CntIdxWidth = (MaxTxns > 1) ? $clo-
g2(MaxTxns) : 32'd1;
564 typedef logic [CntIdxWidth-1:0] cnt_idx_t;
565 typedef logic [$bits(axi_pkg::len_t):0] cnt_t;
566 logic [MaxTxns-1:0] cnt_dec, cnt_free, cnt_set, err_d,
err_q;
567 cnt_t cnt_inp;
568 cnt_t [MaxTxns-1:0] cnt_outp;
569 cnt_idx_t cnt_free_idx, cnt_r_idx;
570 for (genvar i = 0; i < MaxTxns; i++) begin : gen_cnt
571     cc_counter #(
572         .Width ( $bits(cnt_t) )
573     ) i_cnt (
574         .clk_i,
575         .rst_ni,
576         .clr_i ( 1'b0 ),
577         .en_i ( cnt_dec[i] ),
578         .load_i ( cnt_set[i] ),
579         .down_i ( 1'b1 ),
580         .d_i ( cnt_inp ),
581         .q_o ( cnt_outp[i] ),
582         .overflow_o ( ) // not used
583     );
584     assign cnt_free[i] = (cnt_outp[i] == '0);
585 end
586 assign cnt_inp = {1'b0, alloc_len_i} + 1;
587
588 cc_lzc #(
589     .Width ( MaxTxns ),
590     .Mode ( cc_pkg::LZC_TRAILING_ZERO_CNT ) // start count-
ing at index 0

```



```

591 ) i_lzc (
592   .in_i   ( cnt_free   ),
593   .cnt_o   ( cnt_free_idx ),
594   .empty_o (           )
595 );
596
597 logic idq_inp_req, idq_inp_gnt,
598       idq_oup_gnt, idq_oup_valid, idq_oup_pop;
599 cc_idqueue #(
600   .Idwidth ( $bits(idt) ),
601   .Capacity ( MaxTxns ),
602   .data_t   ( cnt_idx_t )
603 ) i_idq (
604   .clk_i,
605   .rst_ni,
606   .clr_i   ( 1'b0 ),
607   .inp_id_i ( alloc_id_i ),
608   .inp_data_i ( cnt_free_idx ),
609   .inp_req_i ( idq_inp_req ),
610   .inp_gnt_o ( idq_inp_gnt ),
611   .exists_data_i ( '0 ),
612   .exists_mask_i ( '0 ),
613   .exists_req_i ( 1'b0 ),
614   .exists_o   ( /* keep open */ ),
615   .exists_gnt_o ( /* keep open */ ),
616   .oup_id_i   ( cnt_id_i ),
617   .oup_pop_i  ( idq_oup_pop ),
618   .oup_req_i  ( cnt_req_i ),
619   .oup_data_o  ( cnt_r_idx ),
620   .oup_data_valid_o ( idq_oup_valid ),
621   .oup_gnt_o  ( idq_oup_gnt ),
622   .full_o    ( /* keep open */ ),
623   .empty_o   ( /* keep open */ )
624 );
625 assign idq_inp_req = alloc_req_i & alloc_gnt_o;
626 assign alloc_gnt_o = idq_inp_gnt & !(cnt_free);
627 assign cnt_gnt_o = idq_oup_gnt & idq_oup_valid;
628 logic [8:0] read_len;
629 assign read_len = cnt_oup[cnt_r_idx] - 1;
630 assign cnt_len_o = read_len[7:0];
631
632 assign idq_oup_pop = cnt_req_i & cnt_gnt_o & cnt_dec_i &
(cnt_len_o == 8'd0);
633 always_comb begin
634   cnt_dec = '0;
635   cnt_dec[cnt_r_idx] = cnt_req_i & cnt_gnt_o & cnt_dec_i;
636 end
637 always_comb begin
638   cnt_set = '0;
639   cnt_set[cnt_free_idx] = alloc_req_i & alloc_gnt_o;
640 end
641 always_comb begin
642   err_d = err_q;
643   cnt_err_o = err_q[cnt_r_idx];
644   if (cnt_req_i && cnt_gnt_o && cnt_set_err_i) begin
645     err_d[cnt_r_idx] = 1'b1;
646     cnt_err_o = 1'b1;
647   end
648   if (alloc_req_i && alloc_gnt_o) begin
649     err_d[cnt_free_idx] = 1'b0;
650   end
651 end
652
653 // registers
654 `FF(err_q, err_d, '0, clk_i, rst_ni)
655
656 endmodule

```

5. pulp-platform_axi/src/axi_bus_compare.sv (730 lines)

```

1 // Copyright (c) 2019-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
4 // pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 // file except in
8 // compliance with the License. You may obtain a copy of the
9 // License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 // applicable law
12 // or agreed to in writing, software, hardware and materials
13 // distributed under
14 // this License is distributed on an "AS IS" BASIS, WITHOUT
15 // WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 // License for the
18 // specific language governing permissions and limitations under
19 // the License.
20 //
21 // Authors:
22 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
23
24 `include "axi/assign.svh"
25
26 /// Synthesizable test module comparing two AXI channels of
27 the same type
28
29 module axi_bus_compare #(
30   /// ID width of the AXI4+ATOP interface

```

```

20 parameter int unsigned AxiIdWidth = 32'd0,
21 /// FIFO depth
22 parameter int unsigned FifoDepth = 32'd0,
23 /// Consider size field in comparison
24 parameter bit          UseSize = 1'b0,
25 /// Data width of the AXI4+ATOP interface
26 parameter int unsigned DataWidth = 32'd8,
27 /// AW channel type of the AXI4+ATOP interface
28 parameter type axi_aw_chan_t = logic,
29 /// W channel type of the AXI4+ATOP interface
30 parameter type axi_w_chan_t = logic,
31 /// B channel type of the AXI4+ATOP interface
32 parameter type axi_b_chan_t = logic,
33 /// AR channel type of the AXI4+ATOP interface
34 parameter type axi_ar_chan_t = logic,
35 /// R channel type of the AXI4+ATOP interface
36 parameter type axi_r_chan_t = logic,
37 /// Request struct type of the AXI4+ATOP slave port
38 parameter type axi_req_t = logic,
39 /// Response struct type of the AXI4+ATOP slave port
40 parameter type axi_rsp_t = logic,
41 /// ID type (*do not overwrite*)
42 parameter type id_t = logic [2*AxiIdWidth-1:0]
43 )(
44   /// Clock
45   input logic clk_i,
46   /// Asynchronous reset, active low
47   input logic rst_ni,
48   /// AXI4+ATOP A channel request in
49   input axi_req_t axi_a_req_i,
50   /// AXI4+ATOP A channel response out
51   output axi_rsp_t axi_a_rsp_o,
52   /// AXI4+ATOP A channel request out
53   output axi_req_t axi_a_req_o,
54   /// AXI4+ATOP A channel response in
55   input axi_rsp_t axi_a_rsp_i,
56   /// AXI4+ATOP B channel request in
57   input axi_req_t axi_b_req_i,
58   /// AXI4+ATOP B channel response out
59   output axi_rsp_t axi_b_rsp_o,
60   /// AXI4+ATOP B channel request out
61   output axi_req_t axi_b_req_o,
62   /// AXI4+ATOP B channel response in
63   input axi_rsp_t axi_b_rsp_i,
64   /// AW mismatch
65   output id_t aw_mismatch_o,
66   /// W mismatch
67   output logic w_mismatch_o,
68   /// B mismatch
69   output id_t b_mismatch_o,
70   /// AR mismatch
71   output id_t ar_mismatch_o,
72   /// R mismatch
73   output id_t r_mismatch_o,
74   /// General mismatch
75   output logic mismatch_o,
76   /// Unit is busy
77   output logic busy_o
78 );
79
80
81 //-----
82 // Channel Signals
83 //-----
84 // assign request payload A
85
86 `AXI_ASSIGN_AW_STRUCT(axi_a_req_o.aw, axi_a_req_i.aw)
87 `AXI_ASSIGN_W_STRUCT(axi_a_req_o.w, axi_a_req_i.w)
88 `AXI_ASSIGN_AR_STRUCT(axi_a_req_o.ar, axi_a_req_i.ar)
89
90 // assign response payload A
91 `AXI_ASSIGN_R_STRUCT(axi_a_rsp_o.r, axi_a_rsp_i.r)
92 `AXI_ASSIGN_B_STRUCT(axi_a_rsp_o.b, axi_a_rsp_i.b)
93
94 // assign request payload B
95 `AXI_ASSIGN_AW_STRUCT(axi_b_req_o.aw, axi_b_req_i.aw)
96 `AXI_ASSIGN_W_STRUCT(axi_b_req_o.w, axi_b_req_i.w)
97 `AXI_ASSIGN_AR_STRUCT(axi_b_req_o.ar, axi_b_req_i.ar)
98
99 // assign response payload B
100 `AXI_ASSIGN_R_STRUCT(axi_b_rsp_o.r, axi_b_rsp_i.r)
101 `AXI_ASSIGN_B_STRUCT(axi_b_rsp_o.b, axi_b_rsp_i.b)
102
103 // fifo handshaking signals A
104 id_t fifo_valid_aw_a, fifo_ready_aw_a;
105 id_t fifo_valid_b_a, fifo_ready_b_a;
106 id_t fifo_valid_ar_a, fifo_ready_ar_a;
107 id_t fifo_valid_r_a, fifo_ready_r_a;
108
109 logic fifo_sel_valid_aw_a, fifo_sel_ready_aw_a;
110 logic fifo_sel_valid_w_a, fifo_sel_ready_w_a;
111 logic fifo_sel_valid_b_a, fifo_sel_ready_b_a;
112 logic fifo_sel_valid_ar_a, fifo_sel_ready_ar_a;
113 logic fifo_sel_valid_r_a, fifo_sel_ready_r_a;
114
115 // fifo handshaking signals B
116 id_t fifo_valid_aw_b, fifo_ready_aw_b;
117 id_t fifo_valid_b_b, fifo_ready_b_b;
118 id_t fifo_valid_ar_b, fifo_ready_ar_b;
119 id_t fifo_valid_r_b, fifo_ready_r_b;
120

```

```

121 logic fifo_sel_valid_aw_b, fifo_sel_ready_aw_b;
122 logic fifo_sel_valid_w_b, fifo_sel_ready_w_b;
123 logic fifo_sel_valid_b_b, fifo_sel_ready_b_b;
124 logic fifo_sel_valid_ar_b, fifo_sel_ready_ar_b;
125 logic fifo_sel_valid_r_b, fifo_sel_ready_r_b;
126
127 //-----
128 // FIFO Output Signals
129 //-----
130 id_t fifo_cmp_valid_aw_a;
131 logic fifo_cmp_valid_w_a;
132 id_t fifo_cmp_valid_b_a;
133 id_t fifo_cmp_valid_ar_a;
134 id_t fifo_cmp_valid_r_a;
135
136 id_t fifo_cmp_valid_aw_b;
137 logic fifo_cmp_valid_w_b;
138 id_t fifo_cmp_valid_b_b;
139 id_t fifo_cmp_valid_ar_b;
140 id_t fifo_cmp_valid_r_b;
141
142 axi_aw_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_aw_a;
143 axi_w_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_w_a;
144 axi_b_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_b_a;
145 axi_ar_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_ar_a;
146 axi_r_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_r_a;
147
148 axi_aw_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_aw_b;
149 axi_w_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_w_b;
150 axi_b_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_b_b;
151 axi_ar_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_ar_b;
152 axi_r_chan_t [2**AxiIdWidth-1:0] fifo_cmp_data_r_b;
153
154 // Size alignment signals
155 logic [2:0] w_size;
156 logic [$clog2(DataWidth/8)-1:0] w_lower, w_offset, w_in-
crement;
157 logic [2**AxiIdWidth-1:0][2:0] r_size;
158 logic [2**AxiIdWidth-1:0][$clog2(DataWidth/8)-1:0]
r_lower, r_offset, r_increment;
159
160 //-----
161 // Channel A stream forks
162 //-----
163 cc_stream_fork #(
164     .NumOut ( 32'd2 )
165 ) i_stream_fork_aw_a (
166     .clk_i,
167     .rst_ni,
168     .clr_i
169 );
170
171 ( 1'b0
172     .valid_i ( axi_a_req_i.aw_val-
173     id
174     .ready_o
175     ( axi_a_rsp_o.aw_ready
176     .valid_o ( {fifo_sel_valid_aw_a, axi_a_req_o.aw_val-
177     id} ),
178     .ready_i ( {fifo_sel_ready_aw_a,
179     axi_a_rsp_i.aw_ready} )
180     );
181     cc_stream_fork #(
182     .NumOut ( 32'd2 )
183     ) i_stream_fork_w_a (
184     .clk_i,
185     .rst_ni,
186     .clr_i
187 );
188
189 ( 1'b0
190     .valid_i ( axi_a_req_i.w_val-
191     id
192     .ready_o
193     ( axi_a_rsp_o.w_ready
194     .valid_o ( {fifo_sel_valid_w_a, axi_a_req_o.w_val-
195     id} ),
196     .ready_i ( {fifo_sel_ready_w_a, axi_a_rsp_i.w_ready} )
197     );
198     cc_stream_fork #(
199     .NumOut ( 32'd2 )
200     ) i_stream_fork_b_a (
201     .clk_i,
202     .rst_ni,
203     .clr_i
204 );
205
206 ( 1'b0
207     .valid_i ( axi_a_req_i.b_val-
208     id
209     .ready_o
210     ( axi_a_rsp_o.b_ready
211     .valid_o ( {fifo_sel_valid_b_a, axi_a_req_o.b_val-
212     id} ),
213     .ready_i ( {fifo_sel_ready_b_a, axi_a_req_i.b_ready} )
214     );
215     cc_stream_fork #(
216     .NumOut ( 32'd2 )
217     ) i_stream_fork_ar_a (
218     .clk_i,
219     .rst_ni,
220     .clr_i
221 );
222
223 ( 1'b0
224     .valid_i ( axi_a_req_i.r_val-
225     id
226     .ready_o
227     ( axi_a_rsp_o.r_ready
228     .valid_o ( {fifo_sel_valid_r_a, axi_a_rsp_o.r_val-
229     id} ),
230     .ready_i ( {fifo_sel_ready_r_a, axi_a_req_i.r_ready} )
231     );
232     cc_stream_fork #(
233     .NumOut ( 32'd2 )
234     ) i_stream_fork_r_a (
235     .clk_i,
236     .rst_ni,
237     .clr_i
238 );
239
240 ( 1'b0
241     .flush_i
242     .usage_o ( /* NOT CONNECTED
243     */
244     .data_i
245     ( axi_a_req_i.aw
246     .valid_i ( fifo_valid_aw_a
247     [id] ),
248     .ready_o ( fifo_ready_aw_a
249     [id] ),
250     .data_o ( fifo_cmp_data_aw_a
251     [id] ),
252     .valid_o ( fifo_cmp_valid_aw_a
253     [id] ),
254     .ready_i ( fifo_cmp_valid_aw_a [id] &
255     fifo_cmp_valid_b_b [id] )
256     );
257     cc_stream_fork #(
258     .FallThrough ( 1'b0 ),
259     .DataWidth ( 1'b0 ),
260     .Depth ( FifoDepth ),
261     .data_t ( axi_b_chan_t )
262     ) i_stream_fork_b_a (
263     .clk_i,
264     .rst_ni,
265     .clr_i
266 );
267
268 ( 1'b0
269     .flush_i
270     .usage_o ( /* NOT CONNECTED
271     */
272     .data_i
273     ( axi_a_rsp_i.b
274     .valid_i ( fifo_valid_b_a
275     [id] ),
276     .ready_o ( fifo_ready_b_a
277     [id] ),
278     .data_o ( fifo_cmp_data_b_a
279     [id] ),
280     .valid_o ( fifo_cmp_valid_b_a
281     [id] ),
282     .ready_i ( fifo_cmp_valid_b_a [id] &
283     fifo_cmp_valid_b_b [id] )
284     );
285     cc_stream_fork #(
286     .FallThrough ( 1'b0 ),
287     .DataWidth ( 1'b0 ),
288     .Depth ( FifoDepth ),
289     .data_t ( axi_ar_chan_t )
290     ) i_stream_fork_ar_a (
291     .clk_i,
292     .rst_ni,
293     .clr_i
294 );
295
296 ( 1'b0
297     .flush_i

```



```

( 1'b0
279      .usage_o    ( /* NOT CONNECTED
*/
280      .data_i
( axi_a_req_i.ar
281      .valid_i    ( fifo_valid_ar_a
[id]
282      .ready_o    ( fifo_ready_ar_a
[id]
283      .data_o     ( fifo_cmp_data_ar_a
[id]
284      .valid_o    ( fifo_cmp_valid_ar_a
[id]
285      .ready_i    ( fifo_cmp_valid_ar_a [id] &
fifo_cmp_valid_ar_b [id] )
286      );
287
288      if (UseSize) begin : gen_r_size
289          cc_stream_fifo #(
290              .FallThrough ( 1'b0 ),
291              .DataWidth   ( $clog2(DataWidth/8)+3 ),
292              .Depth       ( 2*FifoDepth )
293          ) i_stream_fifo_w_size (
294              .clk_i,
295              .rst_ni,
296              .clr_i
( 1'b0
297              ),
298              .flush_i    ( 1'b0 ),
299              .usage_o    ( ),
300              .data_i     ( {axi_a_req_i.ar.addr[$clog2(DataWidth/8)-1:0], axi_a_req_i.ar.size} ),
301              .valid_i    ( fifo_valid_ar_a [id] &
302              .ready_o    ( ),
303              .data_o     ( {r_offset[id], r_size[id]} ),
304              .valid_o    ( ),
305              .ready_i    ( fifo_cmp_valid_r_a[id] &
306              .ready_i    ( fifo_cmp_data_r_a[id].last )
307              );
308              always_ff @(posedge clk_i or negedge rst_ni) be-
309              gin : proc_r_increment
310                  if(!rst_ni) begin
311                      r_increment[id] <= '0;
312                  end else begin
313                      if (fifo_cmp_valid_r_a[id] &&
314                      if (fifo_cmp_data_r_a[id].last) begin
315                          r_increment[id] <= '0;
316                      end else begin
317                          r_increment[id] <= r_increment[id] +
318                          2**r_size[id] - ((r_offset[id]+r_increment[id])%(2**r_size[id]));
319                      end
320                  end
321                  end
322                  assign r_lower[id] = r_offset[id] + r_incre-
323                  ment[id];
324              end else begin : gen_no_size
325                  assign r_offset[id] = '0;
326                  assign r_size[id] = '1;
327                  assign r_lower[id] = '0;
328              end
329
330              cc_stream_fifo #(
331                  .FallThrough ( 1'b0 ),
332                  .DataWidth   ( 1'b0 ),
333                  .Depth       ( FifoDepth ),
334                  .data_t      ( axi_r_chan_t )
335              ) i_stream_fifo_r_a (
336                  .clk_i,
337                  .rst_ni,
338                  .clr_i
( 1'b0
339                  .flush_i
( 1'b0
340                  .usage_o    ( /* NOT CONNECTED
*/
341                  .data_i
( axi_a_rsp_i.r
342                  .valid_i    ( fifo_valid_r_a
[id]
343                  .ready_o    ( fifo_ready_r_a
[id]
344                  .data_o     ( fifo_cmp_data_r_a
[id]
345                  .valid_o    ( fifo_cmp_valid_r_a
[id]
346                  .ready_i    ( fifo_cmp_valid_r_a [id] &
347                  fifo_cmp_valid_r_b [id] )
348                  );
349                  end
350
351                  if (UseSize) begin : gen_w_size
352                      cc_stream_fifo #(
353                          .FallThrough ( 1'b0 ),
354                          .DataWidth   ( $clog2(DataWidth/8)+3 ),
355                          .Depth       ( FifoDepth )
356                      ) i_stream_fifo_w_size (

```

```

353      .clk_i,
354      .rst_ni,
355      .clr_i
( 1'b0
356      ),
357      .flush_i    ( 1'b0 ),
358      .usage_o    ( ),
359      .data_i     ( {axi_a_req_i.aw.addr[$clog2(DataWidth/8)-1:0], axi_a_req_i.aw.size} ),
360      .valid_i    ( axi_a_req_i.aw_valid &
361      .ready_o    ( ),
362      .data_o     ( {w_offset, w_size} ),
363      .valid_o    ( ),
364      .ready_i    ( fifo_cmp_valid_w_a & fifo_cmp_val-
365      id_w_b & fifo_cmp_data_w_a.last )
366      );
367      always_ff @(posedge clk_i or negedge rst_ni) begin :
368      proc_w_increment
369          if(!rst_ni) begin
370              w_increment <= '0;
371          end else begin
372              if (fifo_cmp_valid_w_a && fifo_cmp_valid_w_b)
373              begin
374                  if (fifo_cmp_data_w_a.last) begin
375                      w_increment <= '0;
376                  end else begin
377                      w_increment <= (w_increment + 2**w_s-
378                      ize) - ((w_offset+w_increment)%(2**w_size));
379                  end
380              end
381              end
382              assign w_lower = w_offset + w_increment;
383          end else begin : gen_no_size
384              assign w_offset = '0;
385              assign w_size = '1;
386              assign w_lower = '0;
387          end
388
389          cc_stream_fifo #(
390              .FallThrough ( 1'b0 ),
391              .DataWidth   ( 1'b0 ),
392              .Depth       ( FifoDepth ),
393              .data_t      ( axi_w_chan_t )
394          ) i_stream_fifo_w_a (
395              .clk_i,
396              .rst_ni,
397              .clr_i
( 1'b0
398              .flush_i
( 1'b0
399              .usage_o    ( /* NOT CONNECTED
*/
400              .data_i
( axi_a_req_i.w
401              .valid_i    ( fifo_sel_val-
402              id_w_a
403              .ready_o
404              ( fifo_sel_ready_w_a
405              .data_o
406              ( fifo_cmp_data_w_a
407              .valid_o    ( fifo_cmp_val-
408              id_w_a
409              .ready_i    ( fifo_cmp_valid_w_a & fifo_cmp_val-
410              id_w_b )
411              );
412
413              //-----
414              // Input Handshaking A
415              //-----
416              always_comb begin : gen_handshaking_a
417                  // aw
418                  // defaults
419                  fifo_valid_aw_a = '0;
420                  fifo_sel_ready_aw_a = '0;
421                  // assign according id
422                  fifo_valid_aw_a [axi_a_req_i.aw.id] = fifo_sel_val-
423                  id_aw_a;
424                  fifo_sel_ready_aw_a =
425                  fifo_ready_aw_a[axi_a_req_i.aw.id];
426
427                  // b
428                  // defaults
429                  fifo_valid_b_a = '0;
430                  fifo_sel_ready_b_a = '0;
431                  // assign according id
432                  fifo_valid_b_a [axi_a_rsp_i.b.id] = fifo_sel_val-
433                  id_b_a;
434                  fifo_sel_ready_b_a =
435                  fifo_ready_b_a[axi_a_rsp_i.b.id];
436
437                  // ar
438                  // defaults
439                  fifo_valid_ar_a = '0;
440                  fifo_sel_ready_ar_a = '0;
441                  // assign according id
442                  fifo_valid_ar_a [axi_a_req_i.ar.id] = fifo_sel_val-
443                  id_ar_a;

```

```

432     fifo_sel_ready_ar_a           =
fifo_ready_ar_a[axi_a_req_i.ar.id];
433
434     // b
435     // defaults
436     fifo_valid_r_a = '0;
437     fifo_sel_ready_r_a = '0;
438     // assign according id
439     fifo_valid_r_a [axi_a_rsp_i.r.id] = fifo_sel_val-
id_r_a;
440     fifo_sel_ready_r_a           =
fifo_ready_r_a[axi_a_rsp_i.r.id];
441     end
442
443     //-----
444     // Channel B stream forks
445     //-----
446     cc_stream_fork #(
447         .NumOut ( 32'd2 )
448     ) i_stream_fork_aw_b (
449         .clk_i,
450         .rst_ni,
451         .clr_i
452     ( 1'b0
453         .valid_i ( axi_b_req_i.aw_val-
id
454         .ready_o
455         .valid_o ( {fifo_sel_valid_aw_b, axi_b_req_o.aw_val-
id} ),
456         .ready_i ( {fifo_sel_ready_aw_b,
axi_b_rsp_i.aw_ready} )
457     );
458     cc_stream_fork #(
459         .NumOut ( 32'd2 )
460     ) i_stream_fork_w_b (
461         .clk_i,
462         .rst_ni,
463         .clr_i
464     ( 1'b0
465         .valid_i ( axi_b_req_i.w_val-
id
466         .ready_o
467         .valid_o ( {fifo_sel_valid_w_b, axi_b_req_o.w_val-
id} ),
468         .ready_i ( {fifo_sel_ready_w_b, axi_b_rsp_i.w_ready} )
469     );
470     cc_stream_fork #(
471         .NumOut ( 32'd2 )
472     ) i_stream_fork_b_b (
473         .clk_i,
474         .rst_ni,
475         .clr_i
476     ( 1'b0
477         .valid_i ( axi_b_rsp_i.b_val-
id
478         .ready_o
479         .valid_o ( {fifo_sel_valid_b_b, axi_b_rsp_o.b_val-
id} ),
480         .ready_i ( {fifo_sel_ready_b_b, axi_b_req_i.b_ready} )
481     );
482     cc_stream_fork #(
483         .NumOut ( 32'd2 )
484     ) i_stream_fork_ar_b (
485         .clk_i,
486         .rst_ni,
487         .clr_i
488     ( 1'b0
489         .valid_i ( axi_b_req_i.ar_val-
id
490         .ready_o
491         .valid_o ( {fifo_sel_valid_ar_b, axi_b_req_o.ar_val-
id} ),
492         .ready_i ( {fifo_sel_ready_ar_b,
axi_b_rsp_i.ar_ready} )
493     );
494     cc_stream_fork #(
495         .NumOut ( 32'd2 )
496     ) i_stream_fork_r_b (
497         .clk_i,
498         .rst_ni,
499         .clr_i
500     ( 1'b0
501         .valid_i ( axi_b_rsp_i.r_val-
id
502         .ready_o
503         .valid_o ( {fifo_sel_valid_r_b, axi_b_rsp_o.r_val-
id} ),
504         .ready_i ( {fifo_sel_ready_r_b, axi_b_req_i.r_ready} )
505     );
506
507

```

```

508     //-----
509     // Channel B FIFOs
510     //-----
511     for (genvar id = 0; id < 2**AxiIdWidth; id++) begin :
gen_fifos_b
512
513         cc_stream_fifo #(
514             .FallThrough ( 1'b0 ),
515             .DataWidth ( 1'b0 ),
516             .Depth ( FifoDepth ),
517             .data_t ( axi_aw_chan_t )
518         ) i_stream_fifo_aw_b (
519             .clk_i,
520             .rst_ni,
521             .clr_i
522     ( 1'b0
523         .flush_i
524     ( 1'b0
525         .usage_o ( /* NOT CONNECTED
*/
526         .data_i
527     ( axi_b_req_i.aw
528         .valid_i ( fifo_valid_aw_b
[id]
529         .ready_o ( fifo_ready_aw_b
[id]
530         .data_o ( fifo_cmp_data_aw_b
[id]
531         .valid_o ( fifo_cmp_valid_aw_b
[id]
532         .ready_i ( fifo_cmp_valid_aw_a [id] &
fifo_cmp_valid_aw_b [id] )
533     );
534     cc_stream_fifo #(
535         .FallThrough ( 1'b0 ),
536         .DataWidth ( 1'b0 ),
537         .Depth ( FifoDepth ),
538         .data_t ( axi_b_chan_t )
539     ) i_stream_fifo_b_b (
540         .clk_i,
541         .rst_ni,
542         .clr_i
543     ( 1'b0
544         .flush_i
545     ( 1'b0
546         .usage_o ( /* NOT CONNECTED
*/
547         .data_i
548     ( axi_b_rsp_i.b
549         .valid_i ( fifo_valid_b_b
[id]
550         .ready_o ( fifo_ready_b_b
[id]
551         .data_o ( fifo_cmp_data_b_b
[id]
552         .valid_o ( fifo_cmp_valid_b_b
[id]
553         .ready_i ( fifo_cmp_valid_b_a [id] &
fifo_cmp_valid_b_b [id] )
554     );
555     cc_stream_fifo #(
556         .FallThrough ( 1'b0 ),
557         .DataWidth ( 1'b0 ),
558         .Depth ( FifoDepth ),
559         .data_t ( axi_ar_chan_t )
560     ) i_stream_fifo_ar_b (
561         .clk_i,
562         .rst_ni,
563         .clr_i
564     ( 1'b0
565         .flush_i
566     ( 1'b0
567         .usage_o ( /* NOT CONNECTED
*/
568         .data_i
569     ( axi_b_req_i.ar
570         .valid_i ( fifo_valid_ar_b
[id]
571         .ready_o ( fifo_ready_ar_b
[id]
572         .data_o ( fifo_cmp_data_ar_b
[id]
573         .valid_o ( fifo_cmp_valid_ar_b
[id]
574         .ready_i ( fifo_cmp_valid_ar_a [id] &
fifo_cmp_valid_ar_b [id] )
575     );
576     cc_stream_fifo #(
577         .FallThrough ( 1'b0 ),
578         .DataWidth ( 1'b0 ),
579         .Depth ( FifoDepth ),
580         .data_t ( axi_r_chan_t )
581     ) i_stream_fifo_r_b (
582         .clk_i,
583         .rst_ni,
584         .clr_i
585     ( 1'b0
586         .flush_i
587

```

```

( 1'b0
580      .usage_o    ( /* NOT CONNECTED
*/
581      .data_i
( axi_b_rsp_i.r
582      .valid_i    ( fifo_valid_r_b
[id]
583      .ready_o    ( fifo_ready_r_b
[id]
584      .data_o    ( fifo_cmp_data_r_b
[id]
585      .valid_o    ( fifo_cmp_valid_r_b
[id]
586      .ready_i    ( fifo_cmp_valid_r_a [id] &
fifo_cmp_valid_r_b [id] )
587      );
588      end
589
590      cc_stream_fifo #(
591          .FallThrough ( 1'b0
592          .DataWidth    ( 1'b0
593          .Depth        ( FifoDepth
594          .data_t        ( axi_w_chan_t
595      ) i_stream_fifo_w_b (
596          .clk_i,
597          .rst_ni,
598          .clr_i
( 1'b0
599          .flush_i
( 1'b0
600          .usage_o    ( /* NOT CONNECTED
*/
601          .data_i
( axi_b_req_i.w
602          .valid_i    ( fifo_sel_val-
id_w_b
603          .ready_o
( fifo_sel_ready_w_b
604          .data_o
( fifo_cmp_data_w_b
605          .valid_o    ( fifo_cmp_val-
id_w_b
606          .ready_i    ( fifo_cmp_valid_w_a & fifo_cmp_val-
id_w_b )
607      );
608
609      //-----
610      // Input Handshaking B
611      //-----
612      always_comb begin : gen_handshaking_b
613          // aw
614          // defaults
615          fifo_valid_aw_b = '0;
616          fifo_sel_ready_aw_b = '0;
617          // assign according id
618          fifo_valid_aw_b [axi_b_req_i.aw.id] = fifo_sel_val-
id_aw_b;
619          fifo_sel_ready_aw_b =
fifo_ready_aw_b[axi_b_req_i.aw.id];
620
621          // b
622          // defaults
623          fifo_valid_b_b = '0;
624          fifo_sel_ready_b_b = '0;
625          // assign according id
626          fifo_valid_b_b [axi_b_rsp_i.b.id] = fifo_sel_val-
id_b_b;
627          fifo_sel_ready_b_b =
fifo_ready_b_b[axi_b_rsp_i.b.id];
628
629          // ar
630          // defaults
631          fifo_valid_ar_b = '0;
632          fifo_sel_ready_ar_b = '0;
633          // assign according id
634          fifo_valid_ar_b [axi_b_req_i.ar.id] = fifo_sel_val-
id_ar_b;
635          fifo_sel_ready_ar_b =
fifo_ready_ar_b[axi_b_req_i.ar.id];
636
637          // b
638          // defaults
639          fifo_valid_r_b = '0;
640          fifo_sel_ready_r_b = '0;
641          // assign according id
642          fifo_valid_r_b [axi_b_rsp_i.r.id] = fifo_sel_val-
id_r_b;
643          fifo_sel_ready_r_b =
fifo_ready_r_b[axi_b_rsp_i.r.id];
644      end
645
646      //-----
647      // Comparison
648      //-----
649      for (genvar id = 0; id < 2**AxiIdWidth; id++) begin :
gen_cmp
650          logic [DataWidth/8-1:0] r_data_partial_mismatch;
651          logic r_data_mismatch;

```

```

655
656          if (UseSize) begin : gen_r_mismatch_sized
657              for (genvar j = 0; j < DataWidth/8; j++) begin :
gen_r_partial_mismatch
658                  assign r_data_partial_mismatch[j] =
fifo_cmp_data_r_a[id].data[8*j+:8] !=
fifo_cmp_data_r_b[id].data[8*j+:8];
659              end
660          end
661          always_comb begin : proc_r_data_mismatch
662              r_data_mismatch = '0;
663              for (int unsigned j = 0; j < DataWidth/8; j++)
begin
664                  if (j >= r_lower[id] && j < (r_lower[id] +
2**r_size[id])-((r_lower[id] + 2**r_size[id])%(2**r_size[id]))) be-
gin
665                      r_data_mismatch |=
r_data_partial_mismatch[j];
666                  end
667              end
668          end else begin : gen_r_mismatch_unsized
669              assign r_data_mismatch =
fifo_cmp_data_r_a[id].data != fifo_cmp_data_r_b[id].data;
670          end
671          assign aw_mismatch_o [id] = (fifo_cmp_valid_aw_a [id]
& fifo_cmp_valid_aw_b [id]) ?
672              fifo_cmp_data_aw_a
[id] != fifo_cmp_data_aw_b [id] : '0;
673          assign b_mismatch_o [id] = (fifo_cmp_valid_b_a [id]
& fifo_cmp_valid_b_b [id]) ?
674              fifo_cmp_data_b_a
[id] != fifo_cmp_data_b_b [id] : '0;
675          assign ar_mismatch_o [id] = (fifo_cmp_valid_ar_a [id]
& fifo_cmp_valid_ar_b [id]) ?
676              fifo_cmp_data_ar_a
[id] != fifo_cmp_data_ar_b [id] : '0;
677          assign r_mismatch_o [id] = (fifo_cmp_valid_r_a [id]
& fifo_cmp_valid_r_b [id]) ?
678              fifo_cmp_data_r_a[id].id != fifo_cmp_data_r_b[id].id |
679              r_data_mismatch |
680              fifo_cmp_data_r_a[id].resp != fifo_cmp_data_r_b[id].resp |
681              fifo_cmp_data_r_a[id].last != fifo_cmp_data_r_b[id].last |
682              fifo_cmp_data_r_a[id].user != fifo_cmp_data_r_b[id].user )
683              : '0;
684          end
685          logic [DataWidth/8-1:0] w_data_partial_mismatch;
686          logic w_data_mismatch;
687          if (UseSize) begin : gen_w_mismatch_sized
688              for (genvar j = 0; j < DataWidth/8; j++) begin :
gen_w_partial_mismatch
689                  assign w_data_partial_mismatch[j] =
fifo_cmp_data_w_a.data[8*j+:8] != fifo_cmp_data_w_b.data[8*j+:8] |
690                  fifo_cmp_data_w_a.strb[ j ] != fifo_cmp_data_w_b.strb[ j ];
691              end
692          end else begin : gen_w_mismatch_unsized
693              assign w_data_mismatch = fifo_cmp_data_w_a.data !=
fifo_cmp_data_w_b.data |
694              fifo_cmp_data_w_a.strb !=
695              fifo_cmp_data_w_b.strb;
696          end
697          assign w_mismatch_o = (fifo_cmp_valid_w_a &
fifo_cmp_valid_w_b ) ?
698              ( w_data_mismatch
699              | fifo_cmp_data_w_a.last != fifo_cmp_data_w_b.last |
700              | fifo_cmp_data_w_a.user != fifo_cmp_data_w_b.user )
701              : '0;
702          //-----
703          // Outputs
704          //-----
705          assign busy_o = (|fifo_cmp_valid_aw_a) | (|fifo_cmp_val-
id_aw_b) |
706              (|fifo_cmp_valid_w_a) | (|fifo_cmp_val-
id_w_b) |
707              (|fifo_cmp_valid_b_a) | (|fifo_cmp_val-
id_b_b) |

```

```

723             (|fifo_cmp_valid_ar_a) | (|fifo_cmp_val-
id_ar_b) |
724             (|fifo_cmp_valid_r_a) | (|fifo_cmp_val-
id_r_b);
725
726
727     assign mismatch_o = (|aw_mismatch_o) | (|w_mismatch_o) |
(|b_mismatch_o) |
728             (|ar_mismatch_o) | (|r_mismatch_o);
729
730 endmodule

```

6. pulp-platform_axi/src/axi_cdc.sv (254 lines)

```

1 // Copyright (c) 2019-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Authors:
13 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
14 // - Luca Valente <luca.valente@unibo.it>
15 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
16
17 `include "axi/assign.svh"
18
19 /// A clock domain crossing on an AXI interface.
20 ///
21 /// For each of the five AXI channels, this module instanti-
ates a CDC FIFO, whose push and pop
22 /// ports are in separate clock domains. IMPORTANT: For each
AXI channel, you MUST properly
23 /// constrain three paths through the FIFO; see the header of
`cdc_fifo_gray` for instructions.
24 module axi_cdc #(
25     parameter type aw_chan_t = logic, // AW Channel Type
26     parameter type w_chan_t = logic, // W Channel Type
27     parameter type b_chan_t = logic, // B Channel Type
28     parameter type ar_chan_t = logic, // AR Channel Type
29     parameter type r_chan_t = logic, // R Channel Type
30     parameter type axi_req_t = logic, // encapsulates request
channels
31     parameter type axi_resp_t = logic, // encapsulates request
channels
32     /// Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
33     parameter int unsigned LogDepth = 1,
34     /// Number of synchronization registers to insert on the
async pointers
35     parameter int unsigned SyncStages = 2
36 ) (
37     // slave side - clocked by `src_clk_i`
38     input logic src_clk_i,
39     input logic src_rst_ni,
40     input axi_req_t src_req_i,
41     output axi_resp_t src_resp_o,
42     // master side - clocked by `dst_clk_i`
43     input logic dst_clk_i,
44     input logic dst_rst_ni,
45     output axi_req_t dst_req_o,
46     input axi_resp_t dst_resp_i
47 );
48
49 aw_chan_t [2**LogDepth-1:0] async_data_aw_data;
50 w_chan_t [2**LogDepth-1:0] async_data_w_data;
51 b_chan_t [2**LogDepth-1:0] async_data_b_data;
52 ar_chan_t [2**LogDepth-1:0] async_data_ar_data;
53 r_chan_t [2**LogDepth-1:0] async_data_r_data;
54 logic [LogDepth:0] async_data_aw_wptr,
async_data_aw_rptr,
55 async_data_w_wptr,
56 async_data_w_rptr,
57 async_data_b_wptr,
58 async_data_b_rptr,
59 async_data_ar_wptr,
60 async_data_ar_rptr,
61 async_data_r_wptr,
62 async_data_r_rptr;
63
64 axi_cdc_src #(
65     .aw_chan_t ( aw_chan_t ),
66     .w_chan_t ( w_chan_t ),
67     .b_chan_t ( b_chan_t ),
68     .ar_chan_t ( ar_chan_t ),
69     .r_chan_t ( r_chan_t ),

```

```

66     .axi_req_t ( axi_req_t ),
67     .axi_resp_t ( axi_resp_t ),
68     .LogDepth ( LogDepth ),
69     .SyncStages ( SyncStages )
70 ) i_axi_cdc_src (
71     .src_clk_i,
72     .src_rst_ni,
73     .src_req_i,
74     .src_resp_o,
75     (* async *) .async_data_master_aw_data_o
( async_data_aw_data ),
76     (* async *) .async_data_master_aw_wptr_o
( async_data_aw_wptr ),
77     (* async *) .async_data_master_aw_rptr_i
( async_data_aw_rptr ),
78     (* async *) .async_data_master_w_data_o
( async_data_w_data ),
79     (* async *) .async_data_master_w_wptr_o
( async_data_w_wptr ),
80     (* async *) .async_data_master_w_rptr_i
( async_data_w_rptr ),
81     (* async *) .async_data_master_b_data_i
( async_data_b_data ),
82     (* async *) .async_data_master_b_wptr_i
( async_data_b_wptr ),
83     (* async *) .async_data_master_b_rptr_o
( async_data_b_rptr ),
84     (* async *) .async_data_master_ar_data_o
( async_data_ar_data ),
85     (* async *) .async_data_master_ar_wptr_o
( async_data_ar_wptr ),
86     (* async *) .async_data_master_ar_rptr_i
( async_data_ar_rptr ),
87     (* async *) .async_data_master_r_data_i
( async_data_r_data ),
88     (* async *) .async_data_master_r_wptr_i
( async_data_r_wptr ),
89     (* async *) .async_data_master_r_rptr_o
( async_data_r_rptr )
90 );
91
92 axi_cdc_dst #(
93     .aw_chan_t ( aw_chan_t ),
94     .w_chan_t ( w_chan_t ),
95     .b_chan_t ( b_chan_t ),
96     .ar_chan_t ( ar_chan_t ),
97     .r_chan_t ( r_chan_t ),
98     .axi_req_t ( axi_req_t ),
99     .axi_resp_t ( axi_resp_t ),
100     .LogDepth ( LogDepth ),
101     .SyncStages ( SyncStages )
102 ) i_axi_cdc_dst (
103     .dst_clk_i,
104     .dst_rst_ni,
105     .dst_req_o,
106     .dst_resp_i,
107     (* async *) .async_data_slave_aw_wptr_i
( async_data_aw_wptr ),
108     (* async *) .async_data_slave_aw_rptr_o
( async_data_aw_rptr ),
109     (* async *) .async_data_slave_aw_data_i
( async_data_aw_data ),
110     (* async *) .async_data_slave_w_wptr_i
( async_data_w_wptr ),
111     (* async *) .async_data_slave_w_rptr_o
( async_data_w_rptr ),
112     (* async *) .async_data_slave_w_data_i
( async_data_w_data ),
113     (* async *) .async_data_slave_b_wptr_o
( async_data_b_wptr ),
114     (* async *) .async_data_slave_b_rptr_i
( async_data_b_rptr ),
115     (* async *) .async_data_slave_b_data_o
( async_data_b_data ),
116     (* async *) .async_data_slave_ar_wptr_i
( async_data_ar_wptr ),
117     (* async *) .async_data_slave_ar_rptr_o
( async_data_ar_rptr ),
118     (* async *) .async_data_slave_ar_data_i
( async_data_ar_data ),
119     (* async *) .async_data_slave_r_wptr_o
( async_data_r_wptr ),
120     (* async *) .async_data_slave_r_rptr_i
( async_data_r_rptr ),
121     (* async *) .async_data_slave_r_data_o
( async_data_r_data )
122 );
123
124 endmodule
125
126 `include "axi/assign.svh"
127 `include "axi/typedef.svh"
128
129 // interface wrapper
130 module axi_cdc_intf #(
131     parameter int unsigned AXI_ID_WIDTH = 0,
132     parameter int unsigned AXI_ADDR_WIDTH = 0,
133     parameter int unsigned AXI_DATA_WIDTH = 0,
134     parameter int unsigned AXI_USER_WIDTH = 0,
135     /// Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.

```

```

136 parameter int unsigned LOG_DEPTH = 1,
137 /// Number of synchronization registers to insert on the
async pointers
138 parameter int unsigned SYNC_STAGES = 2
139 ) (
140 // slave side - clocked by `src_clk_i`
141 input logic src_clk_i,
142 input logic src_rst_ni,
143 AXI_BUS.Slave src,
144 // master side - clocked by `dst_clk_i`
145 input logic dst_clk_i,
146 input logic dst_rst_ni,
147 AXI_BUS.Master dst
148 );
149
150 typedef logic [AXI_ID_WIDTH-1:0] id_t;
151 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
152 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
153 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
154 typedef logic [AXI_USER_WIDTH-1:0] user_t;
155 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
156 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
157 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
158 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
159 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
160 `AXI_TYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t, ar_chan_t)
161 `AXI_TYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
162
163 req_t src_req, dst_req;
164 resp_t src_resp, dst_resp;
165
166 `AXI_ASSIGN_TO_REQ(src_req, src)
167 `AXI_ASSIGN_FROM_RESP(src, src_resp)
168
169 `AXI_ASSIGN_FROM_REQ(dst, dst_req)
170 `AXI_ASSIGN_TO_RESP(dst_resp, dst)
171
172 axi_cdc #(
173 .aw_chan_t ( aw_chan_t ),
174 .w_chan_t ( w_chan_t ),
175 .b_chan_t ( b_chan_t ),
176 .ar_chan_t ( ar_chan_t ),
177 .r_chan_t ( r_chan_t ),
178 .axi_req_t ( req_t ),
179 .axi_resp_t ( resp_t ),
180 .LogDepth ( LOG_DEPTH ),
181 .SyncStages ( SYNC_STAGES )
182 ) i_axi_cdc (
183 .src_clk_i,
184 .src_rst_ni,
185 .src_req_i ( src_req ),
186 .src_resp_o ( src_resp ),
187 .dst_clk_i,
188 .dst_rst_ni,
189 .dst_req_o ( dst_req ),
190 .dst_resp_i ( dst_resp )
191 );
192
193 endmodule
194
195 module axi_lite_cdc_intf #(
196 parameter int unsigned AXI_ADDR_WIDTH = 0,
197 parameter int unsigned AXI_DATA_WIDTH = 0,
198 /// Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
199 parameter int unsigned LOG_DEPTH = 1,
200 /// Number of synchronization registers to insert on the
async pointers
201 parameter int unsigned SYNC_STAGES = 2
202 ) (
203 // slave side - clocked by `src_clk_i`
204 input logic src_clk_i,
205 input logic src_rst_ni,
206 AXI_LITE.Slave src,
207 // master side - clocked by `dst_clk_i`
208 input logic dst_clk_i,
209 input logic dst_rst_ni,
210 AXI_LITE.Master dst
211 );
212
213 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
214 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
215 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
216 `AXI_LITE_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t)
217 `AXI_LITE_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
218 `AXI_LITE_TYPEDEF_B_CHAN_T(b_chan_t)
219 `AXI_LITE_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
220 `AXI_LITE_TYPEDEF_R_CHAN_T(r_chan_t, data_t)
221 `AXI_LITE_TYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t,
ar_chan_t)
222 `AXI_LITE_TYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
223
224 req_t src_req, dst_req;
225 resp_t src_resp, dst_resp;
226
227 `AXI_LITE_ASSIGN_TO_REQ(src_req, src)
228 `AXI_LITE_ASSIGN_FROM_RESP(src, src_resp)
229
230 `AXI_LITE_ASSIGN_FROM_REQ(dst, dst_req)
231 `AXI_LITE_ASSIGN_TO_RESP(dst_resp, dst)
232

```

```

233 axi_cdc #(
234 .aw_chan_t ( aw_chan_t ),
235 .w_chan_t ( w_chan_t ),
236 .b_chan_t ( b_chan_t ),
237 .ar_chan_t ( ar_chan_t ),
238 .r_chan_t ( r_chan_t ),
239 .axi_req_t ( req_t ),
240 .axi_resp_t ( resp_t ),
241 .LogDepth ( LOG_DEPTH ),
242 .SyncStages ( SYNC_STAGES )
243 ) i_axi_cdc (
244 .src_clk_i,
245 .src_rst_ni,
246 .src_req_i ( src_req ),
247 .src_resp_o ( src_resp ),
248 .dst_clk_i,
249 .dst_rst_ni,
250 .dst_req_o ( dst_req ),
251 .dst_resp_i ( dst_resp )
252 );
253
254 endmodule

```

7. pulp-platform_axi/src/axi_cdc_dst.sv (297 lines)

```

1 // Copyright (c) 2019-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11 //
12 // Authors:
13 // - Luca Valente <luca.valente@unibo.it>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 `include "axi/assign.svh"
17 `include "axi/typedef.svh"
18
19 /// Destination-clock-domain half of the AXI CDC crossing.
20 ///
21 /// For each of the five AXI channels, this module instanti-
ates the source or destination half of
22 /// a CDC FIFO. IMPORTANT: For each AXI channel, you MUST
properly constrain three paths through
23 /// the FIFO; see the header of `cdc_fifo_gray` for instruc-
tions.
24 module axi_cdc_dst #(
25 /// Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
26 parameter int unsigned LogDepth = 1,
27 /// Number of synchronization registers to insert on the
async pointers
28 parameter int unsigned SyncStages = 2,
29 parameter type aw_chan_t = logic,
30 parameter type w_chan_t = logic,
31 parameter type b_chan_t = logic,
32 parameter type ar_chan_t = logic,
33 parameter type r_chan_t = logic,
34 parameter type axi_req_t = logic,
35 parameter type axi_resp_t = logic
36 ) (
37 // asynchronous slave port
38 input aw_chan_t [2**LogDepth-1:0]
async_data_slave_aw_data_i,
39 input logic [LogDepth:0]
async_data_slave_aw_wptr_i,
40 output logic [LogDepth:0]
async_data_slave_aw_rptr_o,
41 input w_chan_t [2**LogDepth-1:0]
async_data_slave_w_data_i,
42 input logic [LogDepth:0]
async_data_slave_w_wptr_i,
43 output logic [LogDepth:0]
async_data_slave_w_rptr_o,
44 output b_chan_t [2**LogDepth-1:0]
async_data_slave_b_data_o,
45 output logic [LogDepth:0]
async_data_slave_b_wptr_o,
46 input logic [LogDepth:0]
async_data_slave_b_rptr_i,
47 input ar_chan_t [2**LogDepth-1:0]
async_data_slave_ar_data_i,
48 input logic [LogDepth:0]
async_data_slave_ar_wptr_i,
49 output logic [LogDepth:0]
async_data_slave_ar_rptr_o,

```



```

50  output r_chan_t  [2*LogDepth-1:0]
async_data_slave_r_data_o,
51  output logic      [LogDepth:0]
async_data_slave_r_wptr_o,
52  input  logic      [LogDepth:0]
async_data_slave_r_rptr_i,
53  // synchronous master port - clocked by `dst_clk_i`
54  input  logic      dst_clk_i,
55  input  logic      dst_rst_ni,
56  output axi_req_t  dst_req_o,
57  input  axi_resp_t dst_resp_i
58  );
59
60  cc_cdc_fifo_gray_dst #(
61  `ifdef QUESTA
62  // Workaround for a bug in Questa: Pass flat logic vector
instead of struct to type parameter.
63  .data_t  ( logic [$bits(aw_chan_t)-1:0] ),
64  `else
65  // Other tools, such as VCS, have problems with type para-
meters constructed through `$bits()`.
66  .data_t  ( aw_chan_t ),
67  `endif
68  .LogDepth ( LogDepth ),
69  .SyncStages ( SyncStages )
70  ) i_cdc_fifo_gray_dst_aw (
71  .async_data_i ( async_data_slave_aw_data_i ),
72  .async_wptr_i ( async_data_slave_aw_wptr_i ),
73  .async_rptr_o ( async_data_slave_aw_rptr_o ),
74  .dst_clk_i,
75  .dst_rst_ni,
76  .dst_data_o ( dst_req_o.aw ),
77  .dst_valid_o ( dst_req_o.aw_valid ),
78  .dst_ready_i ( dst_resp_i.aw_ready )
79  );
80
81  cc_cdc_fifo_gray_dst #(
82  `ifdef QUESTA
83  .data_t  ( logic [$bits(w_chan_t)-1:0] ),
84  `else
85  .data_t  ( w_chan_t ),
86  `endif
87  .LogDepth ( LogDepth ),
88  .SyncStages ( SyncStages )
89  ) i_cdc_fifo_gray_dst_w (
90  .async_data_i ( async_data_slave_w_data_i ),
91  .async_wptr_i ( async_data_slave_w_wptr_i ),
92  .async_rptr_o ( async_data_slave_w_rptr_o ),
93  .dst_clk_i,
94  .dst_rst_ni,
95  .dst_data_o ( dst_req_o.w ),
96  .dst_valid_o ( dst_req_o.w_valid ),
97  .dst_ready_i ( dst_resp_i.w_ready )
98  );
99
100  cc_cdc_fifo_gray_src #(
101  `ifdef QUESTA
102  .data_t  ( logic [$bits(b_chan_t)-1:0] ),
103  `else
104  .data_t  ( b_chan_t ),
105  `endif
106  .LogDepth ( LogDepth ),
107  .SyncStages ( SyncStages )
108  ) i_cdc_fifo_gray_src_b (
109  .src_clk_i ( dst_clk_i ),
110  .src_rst_ni ( dst_rst_ni ),
111  .src_data_i ( dst_resp_i.b ),
112  .src_valid_i ( dst_resp_i.b_valid ),
113  .src_ready_o ( dst_req_o.b_ready ),
114  .async_data_o ( async_data_slave_b_data_o ),
115  .async_wptr_o ( async_data_slave_b_wptr_o ),
116  .async_rptr_i ( async_data_slave_b_rptr_i )
117  );
118
119  cc_cdc_fifo_gray_dst #(
120  `ifdef QUESTA
121  .data_t  ( logic [$bits(ar_chan_t)-1:0] ),
122  `else
123  .data_t  ( ar_chan_t ),
124  `endif
125  .LogDepth ( LogDepth ),
126  .SyncStages ( SyncStages )
127  ) i_cdc_fifo_gray_dst_ar (
128  .dst_clk_i,
129  .dst_rst_ni,
130  .dst_data_o ( dst_req_o.ar ),
131  .dst_valid_o ( dst_req_o.ar_valid ),
132  .dst_ready_i ( dst_resp_i.ar_ready ),
133  .async_data_i ( async_data_slave_ar_data_i ),
134  .async_wptr_i ( async_data_slave_ar_wptr_i ),
135  .async_rptr_o ( async_data_slave_ar_rptr_o )
136  );
137
138  cc_cdc_fifo_gray_src #(
139  `ifdef QUESTA
140  .data_t  ( logic [$bits(r_chan_t)-1:0] ),
141  `else
142  .data_t  ( r_chan_t ),
143  `endif
144  .LogDepth ( LogDepth ),
145  .SyncStages ( SyncStages )

```

```

146  ) i_cdc_fifo_gray_src_r (
147  .src_clk_i ( dst_clk_i ),
148  .src_rst_ni ( dst_rst_ni ),
149  .src_data_i ( dst_resp_i.r ),
150  .src_valid_i ( dst_resp_i.r_valid ),
151  .src_ready_o ( dst_req_o.r_ready ),
152  .async_data_o ( async_data_slave_r_data_o ),
153  .async_wptr_o ( async_data_slave_r_wptr_o ),
154  .async_rptr_i ( async_data_slave_r_rptr_i )
155  );
156
157  endmodule
158
159  module axi_cdc_dst_intf #(
160  parameter int unsigned AXI_ID_WIDTH = 0,
161  parameter int unsigned AXI_ADDR_WIDTH = 0,
162  parameter int unsigned AXI_DATA_WIDTH = 0,
163  parameter int unsigned AXI_USER_WIDTH = 0,
164  // Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
165  parameter int unsigned LOG_DEPTH = 1,
166  // Number of synchronization registers to insert on the
async pointers
167  parameter int unsigned SYNC_STAGES = 2
168  ) (
169  // asynchronous slave port
170  AXI_BUS_ASYNC_GRAY.Slave src,
171  // synchronous master port - clocked by `dst_clk_i`
172  input logic dst_clk_i,
173  input logic dst_rst_ni,
174  AXI_BUS.Master dst
175  );
176
177  typedef logic [AXI_ID_WIDTH-1:0] id_t;
178  typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
179  typedef logic [AXI_DATA_WIDTH-1:0] data_t;
180  typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
181  typedef logic [AXI_USER_WIDTH-1:0] user_t;
182  `AXI_TYPERDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
183  `AXI_TYPERDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
184  `AXI_TYPERDEF_B_CHAN_T(b_chan_t, id_t, user_t)
185  `AXI_TYPERDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
186  `AXI_TYPERDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
187  `AXI_TYPERDEF_REQ_T(req_t, aw_chan_t, w_chan_t, ar_chan_t)
188  `AXI_TYPERDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
189
190  req_t dst_req;
191  resp_t dst_resp;
192
193  axi_cdc_dst #(
194  .aw_chan_t ( aw_chan_t ),
195  .w_chan_t ( w_chan_t ),
196  .b_chan_t ( b_chan_t ),
197  .ar_chan_t ( ar_chan_t ),
198  .r_chan_t ( r_chan_t ),
199  .axi_req_t ( req_t ),
200  .axi_resp_t ( resp_t ),
201  .LogDepth ( LOG_DEPTH ),
202  .SyncStages ( SYNC_STAGES )
203  ) i_axi_cdc_dst (
204  .async_data_slave_aw_data_i ( src.aw_data ),
205  .async_data_slave_aw_wptr_i ( src.aw_wptr ),
206  .async_data_slave_aw_rptr_o ( src.aw_rptr ),
207  .async_data_slave_w_data_i ( src.w_data ),
208  .async_data_slave_w_wptr_i ( src.w_wptr ),
209  .async_data_slave_w_rptr_o ( src.w_rptr ),
210  .async_data_slave_b_data_o ( src.b_data ),
211  .async_data_slave_b_wptr_o ( src.b_wptr ),
212  .async_data_slave_b_rptr_i ( src.b_rptr ),
213  .async_data_slave_ar_data_i ( src.ar_data ),
214  .async_data_slave_ar_wptr_i ( src.ar_wptr ),
215  .async_data_slave_ar_rptr_o ( src.ar_rptr ),
216  .async_data_slave_r_data_o ( src.r_data ),
217  .async_data_slave_r_wptr_o ( src.r_wptr ),
218  .async_data_slave_r_rptr_i ( src.r_rptr ),
219  .dst_clk_i,
220  .dst_rst_ni,
221  .dst_req_o ( dst_req ),
222  .dst_resp_i ( dst_resp )
223  );
224
225  `AXI_ASSIGN_FROM_REQ(dst, dst_req)
226  `AXI_ASSIGN_TO_RESP(dst_resp, dst)
227
228  endmodule
229
230  module axi_lite_cdc_dst_intf #(
231  parameter int unsigned AXI_ADDR_WIDTH = 0,
232  parameter int unsigned AXI_DATA_WIDTH = 0,
233  // Depth of the FIFO crossing the clock domain, given as
2**LOG_DEPTH.
234  parameter int unsigned LOG_DEPTH = 1,
235  // Number of synchronization registers to insert on the
async pointers
236  parameter int unsigned SYNC_STAGES = 2
237  ) (
238  // asynchronous slave port
239  AXI_LITE_ASYNC_GRAY.Slave src,
240  // synchronous master port - clocked by `dst_clk_i`

```

```

243 input logic          dst_clk_i,
244 input logic          dst_rst_ni,
245 AXI_LITE.Master      dst
246 );
247
248 typedef logic [AXI_ADDR_WIDTH-1:0]  addr_t;
249 typedef logic [AXI_DATA_WIDTH-1:0]  data_t;
250 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
251 `AXI_LITE_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t)
252 `AXI_LITE_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
253 `AXI_LITE_TYPEDEF_B_CHAN_T(b_chan_t)
254 `AXI_LITE_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
255 `AXI_LITE_TYPEDEF_R_CHAN_T(r_chan_t, data_t)
256 `AXI_LITE_TYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t,
ar_chan_t)
257 `AXI_LITE_TYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
258
259 req_t dst_req;
260 resp_t dst_resp;
261
262 axi_cdc_dst #(
263     .aw_chan_t ( aw_chan_t ),
264     .w_chan_t ( w_chan_t ),
265     .b_chan_t ( b_chan_t ),
266     .ar_chan_t ( ar_chan_t ),
267     .r_chan_t ( r_chan_t ),
268     .axi_req_t ( req_t ),
269     .axi_resp_t ( resp_t ),
270     .LogDepth ( LOG_DEPTH ),
271     .SyncStages ( SYNC_STAGES )
272 ) i_axi_cdc_dst (
273     .async_data_slave_aw_data_i ( src.aw_data ),
274     .async_data_slave_aw_wptr_i ( src.aw_wptr ),
275     .async_data_slave_aw_rptr_o ( src.aw_rptr ),
276     .async_data_slave_w_data_i ( src.w_data ),
277     .async_data_slave_w_wptr_i ( src.w_wptr ),
278     .async_data_slave_w_rptr_o ( src.w_rptr ),
279     .async_data_slave_b_data_o ( src.b_data ),
280     .async_data_slave_b_wptr_o ( src.b_wptr ),
281     .async_data_slave_b_rptr_i ( src.b_rptr ),
282     .async_data_slave_ar_data_i ( src.ar_data ),
283     .async_data_slave_ar_wptr_i ( src.ar_wptr ),
284     .async_data_slave_ar_rptr_o ( src.ar_rptr ),
285     .async_data_slave_r_data_o ( src.r_data ),
286     .async_data_slave_r_wptr_o ( src.r_wptr ),
287     .async_data_slave_r_rptr_i ( src.r_rptr ),
288     .dst_clk_i,
289     .dst_rst_ni,
290     .dst_req_o ( dst_req ),
291     .dst_resp_i ( dst_resp )
292 );
293
294 `AXI_LITE_ASSIGN_FROM_REQ(dst, dst_req)
295 `AXI_LITE_ASSIGN_TO_RESP(dst_resp, dst)
296
297 endmodule

```

8. pulp-platform_axi/src/axi_cdc_src.sv (296 lines)

```

1 // Copyright (c) 2019-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the license.
11 //
12 // Authors:
13 // - Luca Valente <luca.valente@unibo.it>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 `include "axi/assign.svh"
17 `include "axi/typedef.svh"
18
19 /// Source-clock-domain half of the AXI CDC crossing.
20 ///
21 /// For each of the five AXI channels, this module instanti-
ates the source or destination half of
22 /// a CDC FIFO. IMPORTANT: For each AXI channel, you MUST
properly constrain three paths through
23 /// the FIFO; see the header of `cdc_fifo_gray` for instruc-
tions.
24 module axi_cdc_src #(
25     /// Depth of the FIFO crossing the clock domain, given as
2*LOG_DEPTH.
26     parameter int unsigned LogDepth = 1,
27     /// Number of synchronization registers to insert on the
async pointers

```

```

28     parameter int unsigned SyncStages = 2,
29     parameter type aw_chan_t = logic,
30     parameter type w_chan_t = logic,
31     parameter type b_chan_t = logic,
32     parameter type ar_chan_t = logic,
33     parameter type r_chan_t = logic,
34     parameter type axi_req_t = logic,
35     parameter type axi_resp_t = logic
36 ) (
37     // synchronous slave port - clocked by `src_clk_i`
38     input logic          src_clk_i,
39     input logic          src_rst_ni,
40     input axi_req_t      src_req_i,
41     output axi_resp_t    src_resp_o,
42     // asynchronous master port
43     output aw_chan_t [2**LogDepth-1:0]
async_data_master_aw_data_o,
44     output logic [LogDepth:0]
async_data_master_aw_wptr_o,
45     input logic [LogDepth:0]
async_data_master_aw_rptr_i,
46     output w_chan_t [2**LogDepth-1:0]
async_data_master_w_data_o,
47     output logic [LogDepth:0]
async_data_master_w_wptr_o,
48     input logic [LogDepth:0]
async_data_master_w_rptr_i,
49     input b_chan_t [2**LogDepth-1:0]
async_data_master_b_data_i,
50     input logic [LogDepth:0]
async_data_master_b_wptr_i,
51     output logic [LogDepth:0]
async_data_master_b_rptr_o,
52     output ar_chan_t [2**LogDepth-1:0]
async_data_master_ar_data_o,
53     output logic [LogDepth:0]
async_data_master_ar_wptr_o,
54     input logic [LogDepth:0]
async_data_master_ar_rptr_i,
55     input r_chan_t [2**LogDepth-1:0]
async_data_master_r_data_i,
56     input logic [LogDepth:0]
async_data_master_r_wptr_i,
57     output logic [LogDepth:0]
async_data_master_r_rptr_o
58 );
59
60 cc_cdc_fifo_gray_src #(
61     // Workaround for a bug in Questa (see comment in
`axi_cdc_dst` for details).
62     `ifdef QUESTA
63         .data_t ( logic [$bits(aw_chan_t)-1:0] ),
64     `else
65         .data_t ( aw_chan_t ),
66     `endif
67     .LogDepth ( LogDepth ),
68     .SyncStages ( SyncStages )
69 ) i_cdc_fifo_gray_src_aw (
70     .src_clk_i,
71     .src_rst_ni,
72     .src_data_i ( src_req_i.aw ),
73     .src_valid_i ( src_req_i.aw_valid ),
74     .src_ready_o ( src_resp_o.aw_ready ),
75     .async_data_o ( async_data_master_aw_data_o ),
76     .async_wptr_o ( async_data_master_aw_wptr_o ),
77     .async_rptr_i ( async_data_master_aw_rptr_i )
78 );
79
80 cc_cdc_fifo_gray_dst #(
81     `ifdef QUESTA
82         .data_t ( logic [$bits(w_chan_t)-1:0] ),
83     `else
84         .data_t ( w_chan_t ),
85     `endif
86     .LogDepth ( LogDepth ),
87     .SyncStages ( SyncStages )
88 ) i_cdc_fifo_gray_src_w (
89     .src_clk_i,
90     .src_rst_ni,
91     .src_data_i ( src_req_i.w ),
92     .src_valid_i ( src_req_i.w_valid ),
93     .src_ready_o ( src_resp_o.w_ready ),
94     .async_data_o ( async_data_master_w_data_o ),
95     .async_wptr_o ( async_data_master_w_wptr_o ),
96     .async_rptr_i ( async_data_master_w_rptr_i )
97 );
98
99 cc_cdc_fifo_gray_dst #(
100     `ifdef QUESTA
101         .data_t ( logic [$bits(b_chan_t)-1:0] ),
102     `else
103         .data_t ( b_chan_t ),
104     `endif
105     .LogDepth ( LogDepth ),
106     .SyncStages ( SyncStages )
107 ) i_cdc_fifo_gray_dst_b (
108     .dst_clk_i ( src_clk_i ),
109     .dst_rst_ni ( src_rst_ni ),
110     .dst_data_o ( src_resp_o.b ),
111     .dst_valid_o ( src_resp_o.b_valid ),
112     .dst_ready_i ( src_req_i.b_ready ),

```



```

113 .async_data_i ( async_data_master_b_data_i ),
114 .async_wptr_i ( async_data_master_b_wptr_i ),
115 .async_rptr_o ( async_data_master_b_rptr_o )
116 );
117
118 cc_cdc_fifo_gray_src #(
119 `ifdef QUESTA
120   .data_t ( logic [$bits(ar_chan_t)-1:0] ),
121 `else
122   .data_t ( ar_chan_t ),
123 `endif
124 .LogDepth ( LogDepth ),
125 .SyncStages ( SyncStages )
126 ) i_cdc_fifo_gray_src_ar (
127   .src_clk_i,
128   .src_rst_ni,
129   .src_data_i ( src_req_i.ar ),
130   .src_valid_i ( src_req_i.ar_valid ),
131   .src_ready_o ( src_resp_o.ar_ready ),
132   .async_data_o ( async_data_master_ar_data_o ),
133   .async_wptr_o ( async_data_master_ar_wptr_o ),
134   .async_rptr_i ( async_data_master_ar_rptr_i )
135 );
136
137 cc_cdc_fifo_gray_dst #(
138 `ifdef QUESTA
139   .data_t ( logic [$bits(r_chan_t)-1:0] ),
140 `else
141   .data_t ( r_chan_t ),
142 `endif
143 .LogDepth ( LogDepth ),
144 .SyncStages ( SyncStages )
145 ) i_cdc_fifo_gray_dst_r (
146   .dst_clk_i ( src_clk_i ),
147   .dst_rst_ni ( src_rst_ni ),
148   .dst_data_o ( src_resp_o.r ),
149   .dst_valid_o ( src_resp_o.r_valid ),
150   .dst_ready_i ( src_req_i.r_ready ),
151   .async_data_i ( async_data_master_r_data_i ),
152   .async_wptr_i ( async_data_master_r_wptr_i ),
153   .async_rptr_o ( async_data_master_r_rptr_o )
154 );
155
156 endmodule
157
158 module axi_cdc_src_intf #(
159   parameter int unsigned AXI_ID_WIDTH = 0,
160   parameter int unsigned AXI_ADDR_WIDTH = 0,
161   parameter int unsigned AXI_DATA_WIDTH = 0,
162   parameter int unsigned AXI_USER_WIDTH = 0,
163   parameter int unsigned AXI_USER_WIDTH = 0,
164   /// Depth of the FIFO crossing the clock domain, given as
165   2**LOG_DEPTH.
166   parameter int unsigned LOG_DEPTH = 1,
167   /// Number of synchronization registers to insert on the
168   async pointers
169   parameter int unsigned SYNC_STAGES = 2
170 ) (
171   // synchronous slave port - clocked by `src_clk_i`
172   input logic src_clk_i,
173   input logic src_rst_ni,
174   AXI_BUS.Slave src,
175   // asynchronous master port
176   AXI_BUS.ASYNC_GRAY.Master dst
177 );
178
179 typedef logic [AXI_ID_WIDTH-1:0] id_t;
180 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
181 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
182 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
183 typedef logic [AXI_USER_WIDTH-1:0] user_t;
184 `AXI_TTYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, user_t)
185 `AXI_TTYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
186 `AXI_TTYPEDEF_B_CHAN_T(b_chan_t, data_t, strb_t, user_t)
187 `AXI_TTYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
188 `AXI_TTYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
189 `AXI_TTYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t, ar_chan_t)
190 `AXI_TTYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
191
192 req_t src_req;
193 resp_t src_resp;
194
195 `AXI_ASSIGN_TO_REQ(src_req, src)
196 `AXI_ASSIGN_FROM_RESP(src, src_resp)
197
198 axi_cdc_src #(
199   .aw_chan_t ( aw_chan_t ),
200   .w_chan_t ( w_chan_t ),
201   .b_chan_t ( b_chan_t ),
202   .ar_chan_t ( ar_chan_t ),
203   .r_chan_t ( r_chan_t ),
204   .axi_req_t ( req_t ),
205   .axi_resp_t ( resp_t ),
206   .LogDepth ( LOG_DEPTH ),
207   .SyncStages ( SYNC_STAGES )
208 ) i_axi_cdc_src (
209   .src_clk_i,
210   .src_rst_ni,
211   .src_req_i ( src_req ),
212   .src_resp_o ( src_resp ),
213   .async_data_master_aw_data_o ( dst.aw_data ),

```

```

212 .async_data_master_aw_wptr_o ( dst.aw_wptr ),
213 .async_data_master_aw_rptr_i ( dst.aw_rptr ),
214 .async_data_master_w_data_o ( dst.w_data ),
215 .async_data_master_w_wptr_o ( dst.w_wptr ),
216 .async_data_master_w_rptr_i ( dst.w_rptr ),
217 .async_data_master_b_data_i ( dst.b_data ),
218 .async_data_master_b_wptr_i ( dst.b_wptr ),
219 .async_data_master_b_rptr_o ( dst.b_rptr ),
220 .async_data_master_ar_data_o ( dst.ar_data ),
221 .async_data_master_ar_wptr_o ( dst.ar_wptr ),
222 .async_data_master_ar_rptr_i ( dst.ar_rptr ),
223 .async_data_master_r_data_i ( dst.r_data ),
224 .async_data_master_r_wptr_i ( dst.r_wptr ),
225 .async_data_master_r_rptr_o ( dst.r_rptr )
226 );
227
228 endmodule
229
230
231 module axi_lite_cdc_src_intf #(
232   parameter int unsigned AXI_ADDR_WIDTH = 0,
233   parameter int unsigned AXI_DATA_WIDTH = 0,
234   /// Depth of the FIFO crossing the clock domain, given as
235   2**LOG_DEPTH.
236   parameter int unsigned LOG_DEPTH = 1,
237   /// Number of synchronization registers to insert on the
238   async pointers
239   parameter int unsigned SYNC_STAGES = 2
240 ) (
241   // synchronous slave port - clocked by `src_clk_i`
242   input logic src_clk_i,
243   input logic src_rst_ni,
244   AXI_BUS.Slave src,
245   // asynchronous master port
246   AXI_LITE_ASYNC_GRAY.Master dst
247 );
248
249 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
250 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
251 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
252 `AXI_LITE_TTYPEDEF_AW_CHAN_T(aw_chan_t, addr_t)
253 `AXI_LITE_TTYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
254 `AXI_LITE_TTYPEDEF_B_CHAN_T(b_chan_t, data_t, strb_t)
255 `AXI_LITE_TTYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
256 `AXI_LITE_TTYPEDEF_R_CHAN_T(r_chan_t, data_t)
257 `AXI_LITE_TTYPEDEF_REQ_T(req_t, aw_chan_t, w_chan_t,
258   ar_chan_t)
259 `AXI_LITE_TTYPEDEF_RESP_T(resp_t, b_chan_t, r_chan_t)
260
261 req_t src_req;
262 resp_t src_resp;
263
264 `AXI_LITE_ASSIGN_TO_REQ(src_req, src)
265 `AXI_LITE_ASSIGN_FROM_RESP(src, src_resp)
266
267 axi_lite_cdc_src #(
268   .aw_chan_t ( aw_chan_t ),
269   .w_chan_t ( w_chan_t ),
270   .b_chan_t ( b_chan_t ),
271   .ar_chan_t ( ar_chan_t ),
272   .r_chan_t ( r_chan_t ),
273   .axi_req_t ( req_t ),
274   .axi_resp_t ( resp_t ),
275   .LogDepth ( LOG_DEPTH ),
276   .SyncStages ( SYNC_STAGES )
277 ) i_axi_lite_cdc_src (
278   .src_clk_i,
279   .src_rst_ni,
280   .src_req_i ( src_req ),
281   .src_resp_o ( src_resp ),
282   .async_data_master_aw_data_o ( dst.aw_data ),
283   .async_data_master_aw_wptr_o ( dst.aw_wptr ),
284   .async_data_master_w_data_o ( dst.w_data ),
285   .async_data_master_w_wptr_o ( dst.w_wptr ),
286   .async_data_master_w_rptr_i ( dst.w_rptr ),
287   .async_data_master_b_data_i ( dst.b_data ),
288   .async_data_master_b_wptr_i ( dst.b_wptr ),
289   .async_data_master_b_rptr_o ( dst.b_rptr ),
290   .async_data_master_ar_data_o ( dst.ar_data ),
291   .async_data_master_ar_wptr_o ( dst.ar_wptr ),
292   .async_data_master_ar_rptr_i ( dst.ar_rptr ),
293   .async_data_master_r_data_i ( dst.r_data ),
294   .async_data_master_r_wptr_i ( dst.r_wptr ),
295   .async_data_master_r_rptr_o ( dst.r_rptr )
296 );
297
298 endmodule

```

9. pulp-platform_axi/src/axi_chan_compare.sv (266 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
3 // pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
5 // file except in
6 // compliance with the License. You may obtain a copy of the
7 // License at
8 // http://solderpad.org/licenses/SHL-0.51. Unless required by

```

```

applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the license.
10 //
11 // Authors:
12 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
13 // - Paul Scheffler <paulsc@iis.ee.ethz.ch>
14 // - Tim Fischer <fischeti@iis.ee.ethz.ch>
15
16 /// Non-synthesizable module comparing two AXI channels of the
same type
17 module axi_chan_compare #(
18     /// Ignore ID field if it was remapped
19     parameter bit IgnoreId = 1'b0,
20     /// Allow reordered responses of different IDs,
21     /// not compatible with `IgnoreId`
22     parameter bit AllowReordering = 1'b0,
23     /// AXI ID Width
24     parameter int unsigned IdWidth = 1,
25     parameter type aw_chan_t = logic,
26     parameter type w_chan_t = logic,
27     parameter type b_chan_t = logic,
28     parameter type ar_chan_t = logic,
29     parameter type r_chan_t = logic,
30     parameter type req_t = logic,
31     parameter type resp_t = logic
32 ) (
33     input logic clk_a_i,
34     input logic clk_b_i,
35     input req_t axi_a_req,
36     input resp_t axi_a_res,
37     input req_t axi_b_req,
38     input resp_t axi_b_res
39 );
40
41     function automatic void print_aw (
42         input aw_chan_t aw_expected,
43         input aw_chan_t aw_received
44     );
45         // verilog_lint: waive-start line-length
46         $display("AW      | expec-
ted
ceived
47         $dis-
play("-----|-----
-----");
48         $display("id:      | %64d | %64d", aw_expected.id,
aw_received.id);
49         $display("addr:    | %64x | %64x", aw_expected.addr,
aw_received.addr);
50         $display("len:     | %64d | %64d", aw_expected.len,
aw_received.len);
51         $display("size:    | %64d | %64d", aw_expected.size,
aw_received.size);
52         $display("burst:   | %64d | %64d", aw_expected.burst,
aw_received.burst);
53         $display("lock:    | %64d | %64d", aw_expected.lock,
aw_received.lock);
54         $display("cache:   | %64d | %64d", aw_expected.cache,
aw_received.cache);
55         $display("prot:    | %64d | %64d", aw_expected.prot,
aw_received.prot);
56         $display("qos:     | %64d | %64d", aw_expected.qos,
aw_received.qos);
57         $display("region:  | %64d | %64d", aw_expected.region,
aw_received.region);
58         $display("user:    | %64d | %64d", aw_expected.user,
aw_received.user);
59         $display("atop:    | %64d | %64d", aw_expected.atop,
aw_received.atop);
60         $dis-
play("-----|-----
-----");
61         // verilog_lint: waive-stop line-length
62     endfunction
63
64     function automatic void print_ar (
65         input ar_chan_t ar_expected,
66         input ar_chan_t ar_received
67     );
68         // verilog_lint: waive-start line-length
69         $display("AR      | expec-
ted
ceived
70         $dis-
play("-----|-----
-----");
71         $display("id:      | %64d | %64d", ar_expected.id,
ar_received.id);
72         $display("addr:    | %64x | %64x", ar_expected.addr,
ar_received.addr);

```

```

73         $display("len:     | %64d | %64d", ar_expected.len,
ar_received.len);
74         $display("size:    | %64d | %64d", ar_expected.size,
ar_received.size);
75         $display("burst:   | %64d | %64d", ar_expected.burst,
ar_received.burst);
76         $display("lock:    | %64d | %64d", ar_expected.lock,
ar_received.lock);
77         $display("cache:   | %64d | %64d", ar_expected.cache,
ar_received.cache);
78         $display("prot:    | %64d | %64d", ar_expected.prot,
ar_received.prot);
79         $display("qos:     | %64d | %64d", ar_expected.qos,
ar_received.qos);
80         $display("region:  | %64d | %64d", ar_expected.region,
ar_received.region);
81         $display("user:    | %64d | %64d", ar_expected.user,
ar_received.user);
82         $dis-
play("-----|-----
-----");
83         // verilog_lint: waive-stop line-length
84     endfunction
85
86     function automatic void print_w (
87         input w_chan_t w_expected,
88         input w_chan_t w_received
89     );
90         // verilog_lint: waive-start line-length
91         $display("W      | expec-
ted
ceived
92         $dis-
play("-----|-----
-----");
93         $display("data:    | %64x | %64x", w_expected.data,
w_received.data);
94         $display("strb:    | %64d | %64d", w_expected.strb,
w_received.strb);
95         $display("last:    | %64d | %64d", w_expected.last,
w_received.last);
96         $display("user:    | %64d | %64d", w_expected.user,
w_received.user);
97         $dis-
play("-----|-----
-----");
98         // verilog_lint: waive-stop line-length
99     endfunction
100
101     function automatic void print_b (
102         input b_chan_t b_expected,
103         input b_chan_t b_received
104     );
105         // verilog_lint: waive-start line-length
106         $display("B      | expec-
ted
ceived
107         $dis-
play("-----|-----
-----");
108         $display("id:      | %64d | %64d", b_expected.id, b_re-
ceived.id);
109         $display("resp:    | %64d | %64d", b_expected.resp,
b_received.resp);
110         $display("user:    | %64d | %64d", b_expected.user,
b_received.user);
111         $dis-
play("-----|-----
-----");
112         // verilog_lint: waive-stop line-length
113     endfunction
114
115     function automatic void print_r (
116         input r_chan_t r_expected,
117         input r_chan_t r_received
118     );
119         // verilog_lint: waive-start line-length
120         $display("R      | expec-
ted
ceived
121         $dis-
play("-----|-----
-----");
122         $display("id:      | %64d | %64d", r_expected.id, r_re-
ceived.id);
123         $display("data:    | %64x | %64x", r_expected.data,
r_received.data);
124         $display("resp:    | %64d | %64d", r_expected.resp,
r_received.resp);
125         $display("last:    | %64d | %64d", r_expected.last,
r_received.last);
126         $display("user:    | %64d | %64d", r_expected.user,
r_received.user);
127         $dis-
play("-----|-----
-----");

```

```

-----|-----
-----");
128 // verilog_lint: waive-stop line-length
129 endfunction
130
131 localparam NumIds = (AllowReordering)? 2**IdWidth : 1;
132
133 // queues
134 aw_chan_t aw_queue [NumIds-1:0][$];
135 w_chan_t w_queue [NumIds-1:0][$];
136 b_chan_t b_queue [NumIds-1:0][$];
137 ar_chan_t ar_queue [NumIds-1:0][$];
138 r_chan_t r_queue [NumIds-1:0][$];
139
140 // requests generated at axi A: enqueue elements
141 always_ff @(posedge clk_a_i) begin : proc_enqueue_a
142 // aw
143 if (axi_a_req.aw_valid & axi_a_res.aw_ready)
144 if (AllowReordering)
145 aw_queue[axi_a_req.aw.id].push_back(axi_a_req.aw);
146 else aw_queue[0].push_back(axi_a_req.aw);
147 // w
148 if (axi_a_req.w_valid & axi_a_res.w_ready)
149 w_queue.push_back(axi_a_req.w);
150 // ar
151 if (axi_a_req.ar_valid & axi_a_res.ar_ready)
152 if (AllowReordering)
153 ar_queue[axi_a_req.ar.id].push_back(axi_a_req.ar);
154 else ar_queue[0].push_back(axi_a_req.ar);
155 end
156 // responses generated at axi B: enqueue elements
157 always_ff @(posedge clk_b_i) begin : proc_enqueue_b
158 // b
159 if (axi_b_res.b_valid & axi_b_req.b_ready)
160 if (AllowReordering)
161 b_queue[axi_b_res.b.id].push_back(axi_b_res.b);
162 else b_queue[0].push_back(axi_b_res.b);
163 // r
164 if (axi_b_res.r_valid & axi_b_req.r_ready)
165 if (AllowReordering)
166 r_queue[axi_b_res.r.id].push_back(axi_b_res.r);
167 else r_queue[0].push_back(axi_b_res.r);
168 end
169
170 // requests arriving at axi B from A: dequeue elements and
171 check
172 always_ff @(posedge clk_b_i) begin : proc_d-
173 equeue_and_check_b
174 // aw
175 if (axi_b_req.aw_valid & axi_b_res.aw_ready) begin
176 automatic aw_chan_t aw_exp, aw_rcv;
177 if (AllowReordering) begin
178 if (aw_queue[axi_b_req.aw.id].size() == 0)
179 $error("AW queue is empty!");
180 aw_exp =
181 aw_queue[axi_b_req.aw.id].pop_front(); // verilog_lint: waive al-
182 ways-ff-non-blocking
183 end else begin
184 if (aw_queue[0].size() == 0) $error("AW queue
185 is empty!");
186 aw_exp = aw_queue[0].pop_front(); // veri-
187 log_lint: waive always-ff-non-blocking
188 end
189 aw_rcv = axi_b_req.aw;
190 if (IgnoreId) begin
191 aw_exp.id = 'X;
192 aw_rcv.id = 'X;
193 end
194 if (aw_exp !== aw_rcv) begin
195 $error("AW mismatch!");
196 print_aw(aw_exp, aw_rcv);
197 end
198 end
199 // w
200 if (axi_b_req.w_valid & axi_b_res.w_ready) begin
201 automatic w_chan_t w_exp, w_rcv;
202 if (w_queue.size() == 0) $error("W queue is
203 empty!");
204 w_exp = w_queue.pop_front(); // verilog_lint:
205 waive always-ff-non-blocking
206 w_rcv = axi_b_req.w;
207 if (w_exp !== w_rcv) begin
208 $error("W mismatch!");
209 print_w(w_exp, w_rcv);
210 end
211 end
212 // ar
213 if (axi_b_req.ar_valid & axi_b_res.ar_ready) begin
214 automatic ar_chan_t ar_exp, ar_rcv;
215 if (AllowReordering) begin
216 if (ar_queue[axi_b_req.ar.id].size() == 0)
217 $error("AR queue is empty!");
218 ar_exp =
219 ar_queue[axi_b_req.ar.id].pop_front(); // verilog_lint: waive al-
220 ways-ff-non-blocking
221 end else begin
222 if (ar_queue[0].size() == 0) $error("AR queue
223 is empty!");
224 ar_exp = ar_queue[0].pop_front(); // veri-

```

```

log_lint: waive always-ff-non-blocking
209 end
210 ar_rcv = axi_b_req.ar;
211 if (IgnoreId) begin
212 ar_exp.id = 'X;
213 ar_rcv.id = 'X;
214 end
215 if (ar_exp !== ar_rcv) begin
216 $error("AR mismatch!");
217 print_ar(ar_exp, ar_rcv);
218 end
219 end
220 end
221
222 // responses arriving at axi A from B: dequeue elements
223 and check
224 always_ff @(posedge clk_a_i) begin : proc_d-
225 equeue_and_check_a
226 // b
227 if (axi_a_res.b_valid & axi_a_req.b_ready) begin
228 automatic b_chan_t b_exp, b_rcv;
229 if (AllowReordering) begin
230 if (b_queue[axi_a_res.b.id].size() == 0) $er-
231 ror("B queue is empty!");
232 b_exp =
233 b_queue[axi_a_res.b.id].pop_front(); // verilog_lint: waive always-
234 ff-non-blocking
235 end else begin
236 if (b_queue[0].size() == 0) $error("B queue is
237 empty!");
238 b_exp = b_queue[0].pop_front(); // veri-
239 log_lint: waive always-ff-non-blocking
240 end
241 b_rcv = axi_a_res.b;
242 if (IgnoreId) begin
243 b_exp.id = 'X;
244 b_rcv.id = 'X;
245 end
246 if (b_exp !== b_rcv) begin
247 $error("B mismatch!");
248 print_b(b_exp, b_rcv);
249 end
250 end
251 // r
252 if (axi_a_res.r_valid & axi_a_req.r_ready) begin
253 automatic r_chan_t r_exp, r_rcv;
254 if (AllowReordering) begin
255 if (r_queue[axi_a_res.r.id].size() == 0) $er-
256 ror("R queue is empty!");
257 r_exp =
258 r_queue[axi_a_res.r.id].pop_front(); // verilog_lint: waive always-
259 ff-non-blocking
260 end else begin
261 if (r_queue[0].size() == 0) $error("R queue is
262 empty!");
263 r_exp = r_queue[0].pop_front(); // veri-
264 log_lint: waive always-ff-non-blocking
265 end
266 r_rcv = axi_a_res.r;
267 if (IgnoreId) begin
268 r_exp.id = 'X;
269 r_rcv.id = 'X;
270 end
271 if (r_exp !== r_rcv) begin
272 $error("R mismatch!");
273 print_r(r_exp, r_rcv);
274 end
275 end
276 end
277 endmodule : axi_chan_compare

```

10. pulp-platform_axi/src/axi_cut.sv (295 lines)

```

1 // Copyright (c) 2014-2018 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
4 pad Hardware
5
6 // License, Version 0.51 (the "License"); you may not use this
7 file except in
8 // compliance with the License. You may obtain a copy of the
9 License at
10 // http://solderpad.org/licenses/SHL-0.51. Unless required by
11 applicable law
12 // or agreed to in writing, software, hardware and materials
13 distributed under
14 // this license is distributed on an "AS IS" BASIS, WITHOUT
15 WARRANTIES OR
16 // CONDITIONS OF ANY KIND, either express or implied. See the
17 License for the
18 // specific language governing permissions and limitations un-
19 der the License.
20
21 //
22 // Authors:
23 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
24 // - Fabian Schuiki <fschuiki@iis.ee.ethz.ch>
25 // - Andreas Kurth <akurth@iis.ee.ethz.ch>

```

```

16
17 /// An AXI4 cut.
18 ///
19 /// Breaks all combinatorial paths between its input and out-
put.
20 module axi_cut #(
21 // bypass enable
22 parameter bit Bypass    = 1'b0,
23 parameter bit BypassAw  = Bypass,
24 parameter bit BypassW   = Bypass,
25 parameter bit BypassB   = Bypass,
26 parameter bit BypassAr  = Bypass,
27 parameter bit BypassR   = Bypass,
28 // AXI channel structs
29 parameter type aw_chan_t = logic,
30 parameter type w_chan_t  = logic,
31 parameter type b_chan_t  = logic,
32 parameter type ar_chan_t = logic,
33 parameter type r_chan_t  = logic,
34 // AXI request & response structs
35 parameter type axi_req_t = logic,
36 parameter type axi_resp_t = logic
37 ) (
38 input logic      clk_i,
39 input logic      rst_ni,
40 // slave port
41 input axi_req_t  slv_req_i,
42 output axi_resp_t slv_resp_o,
43 // master port
44 output axi_req_t  mst_req_o,
45 input axi_resp_t  mst_resp_i
46 );
47
48 // a spill register for each channel
49 cc_spill_register #(
50 .data_t ( aw_chan_t ),
51 .Bypass ( BypassAw )
52 ) i_reg_aw (
53 .clk_i ( clk_i ),
54 .rst_ni ( rst_ni ),
55 .clr_i ( 1'b0 ),
56 .valid_i ( slv_req_i.aw_valid ),
57 .ready_o ( slv_resp_o.aw_ready ),
58 .data_i ( slv_req_i.aw ),
59 .valid_o ( mst_req_o.aw_valid ),
60 .ready_i ( mst_resp_i.aw_ready ),
61 .data_o ( mst_req_o.aw )
62 );
63
64 cc_spill_register #(
65 .data_t ( w_chan_t ),
66 .Bypass ( BypassW )
67 ) i_reg_w (
68 .clk_i ( clk_i ),
69 .rst_ni ( rst_ni ),
70 .clr_i ( 1'b0 ),
71 .valid_i ( slv_req_i.w_valid ),
72 .ready_o ( slv_resp_o.w_ready ),
73 .data_i ( slv_req_i.w ),
74 .valid_o ( mst_req_o.w_valid ),
75 .ready_i ( mst_resp_i.w_ready ),
76 .data_o ( mst_req_o.w )
77 );
78
79 cc_spill_register #(
80 .data_t ( b_chan_t ),
81 .Bypass ( BypassB )
82 ) i_reg_b (
83 .clk_i ( clk_i ),
84 .rst_ni ( rst_ni ),
85 .clr_i ( 1'b0 ),
86 .valid_i ( mst_resp_i.b_valid ),
87 .ready_o ( mst_req_o.b_ready ),
88 .data_i ( mst_resp_i.b ),
89 .valid_o ( slv_resp_o.b_valid ),
90 .ready_i ( slv_req_i.b_ready ),
91 .data_o ( slv_resp_o.b )
92 );
93
94 cc_spill_register #(
95 .data_t ( ar_chan_t ),
96 .Bypass ( BypassAr )
97 ) i_reg_ar (
98 .clk_i ( clk_i ),
99 .rst_ni ( rst_ni ),
100 .clr_i ( 1'b0 ),
101 .valid_i ( slv_req_i.ar_valid ),
102 .ready_o ( slv_resp_o.ar_ready ),
103 .data_i ( slv_req_i.ar ),
104 .valid_o ( mst_req_o.ar_valid ),
105 .ready_i ( mst_resp_i.ar_ready ),
106 .data_o ( mst_req_o.ar )
107 );
108
109 cc_spill_register #(
110 .data_t ( r_chan_t ),
111 .Bypass ( BypassR )
112 ) i_reg_r (
113 .clk_i ( clk_i ),
114 .rst_ni ( rst_ni ),
115 .clr_i ( 1'b0 ),

```

```

116 .valid_i ( mst_resp_i.r_valid ),
117 .ready_o ( mst_req_o.r_ready ),
118 .data_i ( mst_resp_i.r ),
119 .valid_o ( slv_resp_o.r_valid ),
120 .ready_i ( slv_req_i.r_ready ),
121 .data_o ( slv_resp_o.r )
122 );
123 endmodule
124
125 `include "axi/assign.svh"
126 `include "axi/typedef.svh"
127
128 // interface wrapper
129 module axi_cut_intf #(
130 // Bypass enable
131 parameter bit BYPASS    = 1'b0,
132 parameter bit BYPASS_AW = BYPASS,
133 parameter bit BYPASS_W  = BYPASS,
134 parameter bit BYPASS_B  = BYPASS,
135 parameter bit BYPASS_AR = BYPASS,
136 parameter bit BYPASS_R  = BYPASS,
137 // The address width.
138 parameter int unsigned ADDR_WIDTH = 0,
139 // The data width.
140 parameter int unsigned DATA_WIDTH = 0,
141 // The ID width.
142 parameter int unsigned ID_WIDTH  = 0,
143 // The user data width.
144 parameter int unsigned USER_WIDTH = 0
145 ) (
146 input logic      clk_i,
147 input logic      rst_ni,
148 AXI_BUS.Slave  in,
149 AXI_BUS.Master out
150 );
151
152 typedef logic [ID_WIDTH-1:0] id_t;
153 typedef logic [ADDR_WIDTH-1:0] addr_t;
154 typedef logic [DATA_WIDTH-1:0] data_t;
155 typedef logic [DATA_WIDTH/8-1:0] strb_t;
156 typedef logic [USER_WIDTH-1:0] user_t;
157
158 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
159 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
160 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
161 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
162 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
163 `AXI_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
164 `AXI_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
165
166 axi_req_t slv_req, mst_req;
167 axi_resp_t slv_resp, mst_resp;
168
169 `AXI_ASSIGN_TO_REQ(slv_req, in)
170 `AXI_ASSIGN_FROM_RESP(in, slv_resp)
171
172 `AXI_ASSIGN_FROM_REQ(out, mst_req)
173 `AXI_ASSIGN_TO_RESP(mst_resp, out)
174
175 axi_cut #(
176 .Bypass ( BYPASS ),
177 .BypassAw ( BYPASS_AW ),
178 .BypassW ( BYPASS_W ),
179 .BypassB ( BYPASS_B ),
180 .BypassAr ( BYPASS_AR ),
181 .BypassR ( BYPASS_R ),
182 .aw_chan_t ( aw_chan_t ),
183 .w_chan_t ( w_chan_t ),
184 .b_chan_t ( b_chan_t ),
185 .ar_chan_t ( ar_chan_t ),
186 .r_chan_t ( r_chan_t ),
187 .axi_req_t ( axi_req_t ),
188 .axi_resp_t ( axi_resp_t )
189 ) i_axi_cut (
190 .clk_i,
191 .rst_ni,
192 .slv_req_i ( slv_req ),
193 .slv_resp_o ( slv_resp ),
194 .mst_req_o ( mst_req ),
195 .mst_resp_i ( mst_resp )
196 );
197
198 // Check the invariants.
199 // pragma translate_off
200 `ifndef VERILATOR
201 initial begin
202 assert (ADDR_WIDTH > 0) else $fatal(1, "Wrong addr width
parameter");
203 assert (DATA_WIDTH > 0) else $fatal(1, "Wrong data width
parameter");
204 assert (ID_WIDTH > 0) else $fatal(1, "Wrong id width
parameter");
205 assert (USER_WIDTH > 0) else $fatal(1, "Wrong user width
parameter");
206 assert (in.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
207 assert (in.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
208 assert (in.AXI_ID_WIDTH == ID_WIDTH) else $fatal(1,
"Wrong interface definition");

```

```

209 assert (in.AXI_USER_WIDTH == USER_WIDTH) else $fatal(1,
"Wrong interface definition");
210 assert (out.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
211 assert (out.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
212 assert (out.AXI_ID_WIDTH == ID_WIDTH) else $fatal(1,
"Wrong interface definition");
213 assert (out.AXI_USER_WIDTH == USER_WIDTH) else $fatal(1,
"Wrong interface definition");
214 end
215 `endif
216 // pragma translate_on
217 endmodule
218
219 module axi_lite_cut_intf #(
220 // bypass enable
221 parameter bit BYPASS = 1'b0,
222 parameter bit BYPASS_AW = BYPASS,
223 parameter bit BYPASS_W = BYPASS,
224 parameter bit BYPASS_B = BYPASS,
225 parameter bit BYPASS_AR = BYPASS,
226 parameter bit BYPASS_R = BYPASS,
227 /// The address width.
228 parameter int unsigned ADDR_WIDTH = 0,
229 /// The data width.
230 parameter int unsigned DATA_WIDTH = 0
231 ) (
232 input logic clk_i ,
233 input logic rst_ni ,
234 AXI_LITE.Slave in ,
235 AXI_LITE.Master out
236 );
237
238 typedef logic [ADDR_WIDTH-1:0] addr_t;
239 typedef logic [DATA_WIDTH-1:0] data_t;
240 typedef logic [DATA_WIDTH/8-1:0] strb_t;
241
242 `AXI_LITE_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t)
243 `AXI_LITE_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t)
244 `AXI_LITE_TYPEDEF_B_CHAN_T(b_chan_t)
245 `AXI_LITE_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t)
246 `AXI_LITE_TYPEDEF_R_CHAN_T(r_chan_t, data_t)
247 `AXI_LITE_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
248 `AXI_LITE_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
249
250 axi_req_t slv_req, mst_req;
251 axi_resp_t slv_resp, mst_resp;
252
253 `AXI_LITE_ASSIGN_TO_REQ(slv_req, in)
254 `AXI_LITE_ASSIGN_FROM_RESP(in, slv_resp)
255
256 `AXI_LITE_ASSIGN_FROM_REQ(out, mst_req)
257 `AXI_LITE_ASSIGN_TO_RESP(mst_resp, out)
258
259 axi_cut #(
260 .Bypass ( BYPASS ),
261 .BypassAw ( BYPASS_AW ),
262 .BypassW ( BYPASS_W ),
263 .BypassB ( BYPASS_B ),
264 .BypassAr ( BYPASS_AR ),
265 .BypassR ( BYPASS_R ),
266 .aw_chan_t ( aw_chan_t ),
267 .w_chan_t ( w_chan_t ),
268 .b_chan_t ( b_chan_t ),
269 .ar_chan_t ( ar_chan_t ),
270 .r_chan_t ( r_chan_t ),
271 .axi_req_t ( axi_req_t ),
272 .axi_resp_t ( axi_resp_t )
273 ) i_axi_cut (
274 .clk_i,
275 .rst_ni,
276 .slv_req_i ( slv_req ),
277 .slv_resp_o ( slv_resp ),
278 .mst_req_o ( mst_req ),
279 .mst_resp_i ( mst_resp )
280 );
281
282 // Check the invariants.
283 // pragma translate_off
284 `ifndef VERILATOR
285 initial begin
286 assert (ADDR_WIDTH > 0) else $fatal(1, "Wrong addr width
parameter");
287 assert (DATA_WIDTH > 0) else $fatal(1, "Wrong data width
parameter");
288 assert (in.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
289 assert (in.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
290 assert (out.AXI_ADDR_WIDTH == ADDR_WIDTH) else $fatal(1,
"Wrong interface definition");
291 assert (out.AXI_DATA_WIDTH == DATA_WIDTH) else $fatal(1,
"Wrong interface definition");
292 end
293 `endif
294 // pragma translate_on
295 endmodule

```

11. pulp-platform_axi/src/axi_delayer.sv (203 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10 //
11 // Authors:
12 // - Wolfgang Roenniger <wroennin@iis.ee.ethz.ch>
13 // - Florian Zaruba <zarubaf@iis.ee.ethz.ch>
14 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
15
16 // Synthesizable module that (randomly) delays AXI channels.
17 module axi_delayer #(
18 // AXI channel types
19 parameter type aw_chan_t = logic,
20 parameter type w_chan_t = logic,
21 parameter type b_chan_t = logic,
22 parameter type ar_chan_t = logic,
23 parameter type r_chan_t = logic,
24 // AXI request & response types
25 parameter type axi_req_t = logic,
26 parameter type axi_resp_t = logic,
27 // delay parameters
28 parameter bit StallRandomInput = 0,
29 parameter bit StallRandomOutput = 0,
30 parameter int unsigned FixedDelayInput = 1,
31 parameter int unsigned FixedDelayOutput = 1
32 ) (
33 input logic clk_i, // Clock
34 input logic rst_ni, // Asynchronous reset active
35
36 // slave port
37 input axi_req_t slv_req_i,
38 output axi_resp_t slv_resp_o,
39 // master port
40 output axi_req_t mst_req_o,
41 input axi_resp_t mst_resp_i
42 );
43 // AW
44 cc_stream_delay #(
45 .StallRandom ( StallRandomInput ),
46 .FixedDelay ( FixedDelayInput ),
47 .payload_t ( aw_chan_t )
48 ) i_stream_delay_aw (
49 .clk_i,
50 .rst_ni,
51 .clr_i ( 1'b0 ),
52 .payload_i ( slv_req_i.aw ),
53 .ready_o ( slv_resp_o.aw_ready ),
54 .valid_i ( slv_req_i.aw_valid ),
55 .payload_o ( mst_req_o.aw ),
56 .ready_i ( mst_resp_i.aw_ready ),
57 .valid_o ( mst_req_o.aw_valid )
58 );
59 // AR
60 cc_stream_delay #(
61 .StallRandom ( StallRandomInput ),
62 .FixedDelay ( FixedDelayInput ),
63 .payload_t ( ar_chan_t )
64 ) i_stream_delay_ar (
65 .clk_i,
66 .rst_ni,
67 .clr_i ( 1'b0 ),
68 .payload_i ( slv_req_i.ar ),
69 .ready_o ( slv_resp_o.ar_ready ),
70 .valid_i ( slv_req_i.ar_valid ),
71 .payload_o ( mst_req_o.ar ),
72 .ready_i ( mst_resp_i.ar_ready ),
73 .valid_o ( mst_req_o.ar_valid )
74 );
75 // W
76 cc_stream_delay #(
77 .StallRandom ( StallRandomInput ),
78 .FixedDelay ( FixedDelayInput ),
79 .payload_t ( w_chan_t )
80 ) i_stream_delay_w (
81 .clk_i,
82 .rst_ni,
83 .clr_i ( 1'b0 ),
84 .payload_i ( slv_req_i.w ),
85 .ready_o ( slv_resp_o.w_ready ),
86 .valid_i ( slv_req_i.w_valid ),
87

```



```

88     .payload_o ( mst_req_o.w      ),
89     .ready_i   ( mst_resp_i.w_ready ),
90     .valid_o   ( mst_req_o.w_valid )
91 );
92
93 // B
94 cc_stream_delay #(
95     .StallRandom ( StallRandomOutput ),
96     .FixedDelay  ( FixedDelayOutput ),
97     .payload_t   ( b_chan_t )
98 ) i_stream_delay_b (
99     .clk_i,
100    .rst_ni,
101    .clr_i   ( 1'b0 ),
102    .payload_i ( mst_resp_i.b ),
103    .ready_o  ( mst_req_o.b_ready ),
104    .valid_i  ( mst_resp_i.b_valid ),
105    .payload_o ( slv_resp_o.b ),
106    .ready_i  ( slv_req_i.b_ready ),
107    .valid_o  ( slv_resp_o.b_valid )
108 );
109
110 // R
111 cc_stream_delay #(
112     .StallRandom ( StallRandomOutput ),
113     .FixedDelay  ( FixedDelayOutput ),
114     .payload_t   ( r_chan_t )
115 ) i_stream_delay_r (
116     .clk_i,
117     .rst_ni,
118     .clr_i   ( 1'b0 ),
119     .payload_i ( mst_resp_i.r ),
120     .ready_o  ( mst_req_o.r_ready ),
121     .valid_i  ( mst_resp_i.r_valid ),
122     .payload_o ( slv_resp_o.r ),
123     .ready_i  ( slv_req_i.r_ready ),
124     .valid_o  ( slv_resp_o.r_valid )
125 );
126 endmodule
127
128 `include "axi/typedef.svh"
129 `include "axi/assign.svh"
130
131 // interface wrapper
132 module axi_delayer_intf #(
133     // Synopsys DC requires a default value for parameters.
134     parameter int unsigned AXI_ID_WIDTH      = 0,
135     parameter int unsigned AXI_ADDR_WIDTH    = 0,
136     parameter int unsigned AXI_DATA_WIDTH    = 0,
137     parameter int unsigned AXI_USER_WIDTH    = 0,
138     parameter bit        STALL_RANDOM_INPUT  = 0,
139     parameter bit        STALL_RANDOM_OUTPUT = 0,
140     parameter int unsigned FIXED_DELAY_INPUT  = 1,
141     parameter int unsigned FIXED_DELAY_OUTPUT = 1
142 ) (
143     input logic clk_i,
144     input logic rst_ni,
145     AXI_BUS.Slave slv,
146     AXI_BUS.Master mst
147 );
148
149 typedef logic [AXI_ID_WIDTH-1:0] id_t;
150 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
151 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
152 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
153 typedef logic [AXI_USER_WIDTH-1:0] user_t;
154
155 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
156 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
157 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
158 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
159 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
160 `AXI_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
axi_delayer
161 `AXI_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
162
163 axi_req_t slv_req, mst_req;
164 axi_resp_t slv_resp, mst_resp;
165
166 `AXI_ASSIGN_TO_REQ(slv_req, slv)
167 `AXI_ASSIGN_FROM_RESP(slv, slv_resp)
168
169 `AXI_ASSIGN_FROM_REQ(mst, mst_req)
170 `AXI_ASSIGN_TO_RESP(mst_resp, mst)
171
172 axi_delayer #(
173     .aw_chan_t ( aw_chan_t ),
174     .w_chan_t  ( w_chan_t ),
175     .b_chan_t  ( b_chan_t ),
176     .ar_chan_t ( ar_chan_t ),
177     .r_chan_t  ( r_chan_t ),
178     .axi_req_t ( axi_req_t ),
179     .axi_resp_t ( axi_resp_t ),
180     .StallRandomInput ( STALL_RANDOM_INPUT ),
181     .StallRandomOutput ( STALL_RANDOM_OUTPUT ),
182     .FixedDelayInput ( FIXED_DELAY_INPUT ),
183     .FixedDelayOutput ( FIXED_DELAY_OUTPUT )
184 ) i_axi_delayer (
185     .clk_i, // Clock
186     .rst_ni, // Asynchronous reset active low
187     .slv_req_i ( slv_req ),

```

```

188     .slv_resp_o ( slv_resp ),
189     .mst_req_o  ( mst_req ),
190     .mst_resp_i ( mst_resp )
191 );
192
193 // pragma translate_off
194 `ifndef VERILATOR
195     initial begin: p_assertions
196         assert (AXI_ID_WIDTH >= 1) else $fatal(1, "AXI ID width
must be at least 1!");
197         assert (AXI_ADDR_WIDTH >= 1) else $fatal(1, "AXI ADDR
width must be at least 1!");
198         assert (AXI_DATA_WIDTH >= 1) else $fatal(1, "AXI DATA
width must be at least 1!");
199         assert (AXI_USER_WIDTH >= 1) else $fatal(1, "AXI USER
width must be at least 1!");
200     end
201 `endif
202 // pragma translate_on
203 endmodule

```

12. pulp-platform_axi/src/axi_demux.sv (305 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations under
the License.
10 //
11 // Authors:
12 // - Michael Rogenmoser <michaero@iis.ee.ethz.ch>
13 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
14 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
15 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
16
17 `include "common_cells/assertions.svh"
18 `include "common_cells/registers.svh"
19
20 `ifdef QUESTA
21 // Derive `TARGET_VSIM`, which is used for tool-specific work-
arounds in this file, from `QUESTA`,
22 // which is automatically set in Questa.
23 `define TARGET_VSIM
24 `endif
25
26 /// Demultiplex one AXI4+ATOP slave port to multiple AXI4+ATOP
master ports.
27 ///
28 /// The AW and AR slave channels each have a `select` input to
determine to which master port the
29 /// current request is sent. The `select` can, for example,
be driven by an address decoding module
30 /// to map address ranges to different AXI slaves.
31 ///
32 /// ## Design overview
33 ///
34 /// ![Block diagram](module.axi_demux.png "Block diagram")
35 ///
36 /// Beats on the W channel are routed by demultiplexer accord-
ing to the selection for the
37 /// corresponding AW beat. This relies on the AXI property
that W bursts must be sent in the same
38 /// order as AW beats and beats from different W bursts may
not be interleaved.
39 ///
40 /// Beats on the B and R channel are multiplexed from the mas-
ter ports to the slave port with
41 /// a round-robin arbitration tree.
42 module axi_demux #(
43     parameter int unsigned AxiIdWidth      = 32'd0,
44     parameter bit        AtopSupport      = 1'b1,
45     parameter type       aw_chan_t       = logic,
46     parameter type       w_chan_t        = logic,
47     parameter type       b_chan_t        = logic,
48     parameter type       ar_chan_t       = logic,
49     parameter type       r_chan_t        = logic,
50     parameter type       axi_req_t       = logic,
51     parameter type       axi_resp_t      = logic,
52     parameter int unsigned NoMstPorts     = 32'd0,
53     parameter int unsigned MaxTrans      = 32'd8,
54     parameter int unsigned AxiLookBits   = 32'd3,
55     parameter bit        UniqueIds       = 1'b0,
56     parameter bit        SpillAW         = 1'b1,
57     parameter bit        SpillW         = 1'b0,
58     parameter bit        SpillB         = 1'b0,

```

```

59 parameter bit      SpillAr      = 1'b1,
60 parameter bit      SpillR      = 1'b0,
61 // Dependent parameters, DO NOT OVERRIDE!
62 parameter int unsigned SelectWidth = (NoMstPorts >
32'd1) ? $clog2(NoMstPorts) : 32'd1,
63 parameter type      select_t      = logic [Se-
lectWidth-1:0]
64 ) (
65   input logic      clk_i,
66   input logic      rst_ni,
67   // Slave Port
68   input axi_req_t  slv_req_i,
69   input select_t    slv_aw_select_i,
70   input select_t    slv_ar_select_i,
71   output axi_resp_t slv_resp_o,
72   // Master Ports
73   output axi_req_t  [NoMstPorts-1:0] mst_reqs_o,
74   input axi_resp_t  [NoMstPorts-1:0] mst_resps_i
75 );
76
77 axi_req_t slv_req_cut;
78 axi_resp_t slv_resp_cut;
79
80 logic slv_aw_ready_chan, slv_aw_ready_sel;
81 logic slv_aw_valid_chan, slv_aw_valid_sel;
82
83 logic slv_ar_ready_chan, slv_ar_ready_sel;
84 logic slv_ar_valid_chan, slv_ar_valid_sel;
85
86 select_t slv_aw_select, slv_ar_select;
87
88 cc_spill_register #(
89   .data_t ( aw_chan_t ),
90   .Bypass ( ~SpillAw )
91 ) i_aw_spill_reg (
92   .clk_i,
93   .rst_ni,
94   .clr_i ( 1'b0 ),
95   .valid_i ( slv_req_i.aw_valid ),
96   .ready_o ( slv_aw_ready_chan ),
97   .data_i ( slv_req_i.aw ),
98   .valid_o ( slv_aw_valid_chan ),
99   .ready_i ( slv_resp_cut.aw_ready ),
100  .data_o ( slv_req_cut.aw );
101 );
102 cc_spill_register #(
103   .data_t ( select_t ),
104   .Bypass ( ~SpillAr )
105 ) i_aw_select_spill_reg (
106   .clk_i,
107   .rst_ni,
108   .clr_i ( 1'b0 ),
109   .valid_i ( slv_req_i.ar_valid ),
110   .ready_o ( slv_ar_ready_chan ),
111   .data_i ( slv_ar_select_i ),
112   .valid_o ( slv_ar_valid_sel ),
113   .ready_i ( slv_resp_cut.ar_ready ),
114   .data_o ( slv_ar_select );
115 );
116
117 assign slv_resp_o.aw_ready = slv_aw_ready_chan &
slv_aw_ready_sel;
118 assign slv_req_cut.aw_valid = slv_aw_valid_chan &
slv_aw_valid_sel;
119
120 cc_spill_register #(
121   .data_t ( w_chan_t ),
122   .Bypass ( ~SpillW )
123 ) i_w_spill_reg (
124   .clk_i,
125   .rst_ni,
126   .clr_i ( 1'b0 ),
127   .valid_i ( slv_req_i.w_valid ),
128   .ready_o ( slv_resp_o.w_ready ),
129   .data_i ( slv_req_i.w ),
130   .valid_o ( slv_req_cut.w_valid ),
131   .ready_i ( slv_resp_cut.w_ready ),
132   .data_o ( slv_req_cut.w );
133 );
134 cc_spill_register #(
135   .data_t ( ar_chan_t ),
136   .Bypass ( ~SpillAr )
137 ) i_ar_spill_reg (
138   .clk_i,
139   .rst_ni,
140   .clr_i ( 1'b0 ),
141   .valid_i ( slv_req_i.ar_valid ),
142   .ready_o ( slv_ar_ready_chan ),
143   .data_i ( slv_req_i.ar ),
144   .valid_o ( slv_ar_valid_chan ),
145   .ready_i ( slv_resp_cut.ar_ready ),
146   .data_o ( slv_req_cut.ar );
147 );
148 cc_spill_register #(
149   .data_t ( select_t ),
150   .Bypass ( ~SpillAr )
151 ) i_ar_sel_spill_reg (
152   .clk_i,
153   .rst_ni,
154   .clr_i ( 1'b0 ),
155   .valid_i ( slv_req_i.ar_valid ),

```

```

156   .ready_o ( slv_ar_ready_sel ),
157   .data_i ( slv_ar_select_i ),
158   .valid_o ( slv_ar_valid_sel ),
159   .ready_i ( slv_resp_cut.ar_ready ),
160   .data_o ( slv_ar_select );
161 );
162
163 assign slv_resp_o.ar_ready = slv_ar_ready_chan &
slv_ar_ready_sel;
164 assign slv_req_cut.ar_valid = slv_ar_valid_chan &
slv_ar_valid_sel;
165
166 cc_spill_register #(
167   .data_t ( b_chan_t ),
168   .Bypass ( ~SpillB )
169 ) i_b_spill_reg (
170   .clk_i,
171   .rst_ni,
172   .clr_i ( 1'b0 ),
173   .valid_i ( slv_resp_cut.b_valid ),
174   .ready_o ( slv_req_cut.b_ready ),
175   .data_i ( slv_resp_cut.b ),
176   .valid_o ( slv_resp_o.b_valid ),
177   .ready_i ( slv_req_i.b_ready ),
178   .data_o ( slv_resp_o.b );
179 );
180 cc_spill_register #(
181   .data_t ( r_chan_t ),
182   .Bypass ( ~SpillR )
183 ) i_r_spill_reg (
184   .clk_i,
185   .rst_ni,
186   .clr_i ( 1'b0 ),
187   .valid_i ( slv_resp_cut.r_valid ),
188   .ready_o ( slv_req_cut.r_ready ),
189   .data_i ( slv_resp_cut.r ),
190   .valid_o ( slv_resp_o.r_valid ),
191   .ready_i ( slv_req_i.r_ready ),
192   .data_o ( slv_resp_o.r );
193 );
194
195 axi_demux_simple #(
196   .AxiIdWidth ( AxiIdWidth ),
197   .AtopSupport ( AtopSupport ),
198   .axi_req_t ( axi_req_t ),
199   .axi_resp_t ( axi_resp_t ),
200   .NoMstPorts ( NoMstPorts ),
201   .MaxTrans ( MaxTrans ),
202   .AxiLookBits ( AxiLookBits ),
203   .UniqueIds ( UniqueIds )
204 ) i_demux_simple (
205   .clk_i,
206   .rst_ni,
207
208   .slv_req_i ( slv_req_cut ),
209   .slv_aw_select_i ( slv_aw_select ),
210   .slv_ar_select_i ( slv_ar_select ),
211   .slv_resp_o ( slv_resp_cut ),
212   .mst_reqs_o ( mst_reqs_o ),
213   .mst_resps_i ( mst_resps_i );
214 );
215
216 endmodule
217
218 // interface wrapper
219 `include "axi/assign.svh"
220 `include "axi/typedef.svh"
221 module axi_demux_intf #(
222   parameter int unsigned AXI_ID_WIDTH = 32'd0, // Synopsys
DC requires default value for params
223   parameter bit      ATOP_SUPPORT = 1'b1,
224   parameter int unsigned AXI_ADDR_WIDTH = 32'd0,
225   parameter int unsigned AXI_DATA_WIDTH = 32'd0,
226   parameter int unsigned AXI_USER_WIDTH = 32'd0,
227   parameter int unsigned NO_MST_PORTS = 32'd3,
228   parameter int unsigned MAX_TRANS = 32'd8,
229   parameter int unsigned AXI_LOOK_BITS = 32'd3,
230   parameter bit      UNIQUE_IDS = 1'b0,
231   parameter bit      SPILL_AW = 1'b1,
232   parameter bit      SPILL_W = 1'b0,
233   parameter bit      SPILL_B = 1'b0,
234   parameter bit      SPILL_AR = 1'b1,
235   parameter bit      SPILL_R = 1'b0,
236   // Dependent parameters, DO NOT OVERRIDE!
237   parameter int unsigned SELECT_WIDTH = (NO_MST_PORTS >
32'd1) ? $clog2(NO_MST_PORTS) : 32'd1,
238   parameter type      select_t      = logic [SE-
LECT_WIDTH-1:0] // MST port select type
239 ) (
240   input logic      clk_i, // Clock
241
242   input logic      rst_ni, // Asynchronous reset
active low
243   input select_t    slv_aw_select_i, // has to be stable,
when aw_valid
244   input select_t    slv_ar_select_i, // has to be stable,
when ar_valid
245   AXI_BUS.Slave slv, // slave port
246   AXI_BUS.Master mst [NO_MST_PORTS-1:0] // master ports
247 );

```



```

247
248 typedef logic [AXI_ID_WIDTH-1:0] id_t;
249 typedef logic [AXI_ADDR_WIDTH-1:0] addr_t;
250 typedef logic [AXI_DATA_WIDTH-1:0] data_t;
251 typedef logic [AXI_DATA_WIDTH/8-1:0] strb_t;
252 typedef logic [AXI_USER_WIDTH-1:0] user_t;
253 `AXI_TYPEDEF_AW_CHAN_T(aw_chan_t, addr_t, id_t, user_t)
254 `AXI_TYPEDEF_W_CHAN_T(w_chan_t, data_t, strb_t, user_t)
255 `AXI_TYPEDEF_B_CHAN_T(b_chan_t, id_t, user_t)
256 `AXI_TYPEDEF_AR_CHAN_T(ar_chan_t, addr_t, id_t, user_t)
257 `AXI_TYPEDEF_R_CHAN_T(r_chan_t, data_t, id_t, user_t)
258 `AXI_TYPEDEF_REQ_T(axi_req_t, aw_chan_t, w_chan_t,
ar_chan_t)
259 `AXI_TYPEDEF_RESP_T(axi_resp_t, b_chan_t, r_chan_t)
260
261 axi_req_t          slv_req;
262 axi_resp_t         slv_resp;
263 axi_req_t [NO_MST_PORTS-1:0] mst_req;
264 axi_resp_t [NO_MST_PORTS-1:0] mst_resp;
265
266 `AXI_ASSIGN_TO_REQ(slv_req, slv)
267 `AXI_ASSIGN_FROM_RESP(slv, slv_resp)
268
269 for (genvar i = 0; i < NO_MST_PORTS; i++) begin :
gen_assign_mst_ports
270     `AXI_ASSIGN_FROM_REQ(mst[i], mst_req[i])
271     `AXI_ASSIGN_TO_RESP(mst_resp[i], mst[i])
272 end
273
274 axi_demux #(
275     .AxiIdWidth      ( AXI_ID_WIDTH ), // ID Width
276     .AtopSupport      ( ATOP_SUPPORT ),
277     .aw_chan_t        ( aw_chan_t ), // AW Channel Type
278     .w_chan_t         ( w_chan_t ), // W Channel Type
279     .b_chan_t         ( b_chan_t ), // B Channel Type
280     .ar_chan_t        ( ar_chan_t ), // AR Channel Type
281     .r_chan_t         ( r_chan_t ), // R Channel Type
282     .axi_req_t        ( axi_req_t ),
283     .axi_resp_t       ( axi_resp_t ),
284     .NoMstPorts       ( NO_MST_PORTS ),
285     .MaxTrans         ( MAX_TRANS ),
286     .AxiLookBits      ( AXI_LOOK_BITS ),
287     .UniqueIds        ( UNIQUE_IDS ),
288     .SpillAw          ( SPILL_AW ),
289     .SpillW           ( SPILL_W ),
290     .SpillB           ( SPILL_B ),
291     .SpillAr          ( SPILL_AR ),
292     .SpillR           ( SPILL_R )
293 ) i_axi_demux (
294     .clk_i, // Clock
295     .rst_ni, // Asynchronous reset active low
296     // slave port
297     .slv_req_i ( slv_req ),
298     .slv_aw_select_i ( slv_aw_select_i ),
299     .slv_ar_select_i ( slv_ar_select_i ),
300     .slv_resp_o ( slv_resp ),
301     // master port
302     .mst_reqs_o ( mst_req ),
303     .mst_resps_i ( mst_resp )
304 );
305 endmodule

```

13. pulp-platform_axi/src/axi_demux_id_counters.v (157 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10 //
11 // Authors:
12 // - Wolfgang Roenninger <wroennin@iis.ee.ethz.ch>
13 // - Michael Rogenmoser <michaelro@iis.ee.ethz.ch>
14 // - Thomas Benz <tbenz@iis.ee.ethz.ch>
15 // - Andreas Kurth <akurth@iis.ee.ethz.ch>
16
17 `include "common_cells/registers.svh"
18
19 module axi_demux_id_counters #(
20     // the lower bits of the AXI ID that should be considered,
results in 2**AXI_ID_BITS counters
21     parameter int unsigned AxiIdBits = 2,
22     parameter int unsigned CounterWidth = 4,
23     // Maximum number of in-flight transactions per counter; a
counter reports full once it

```

```

24 // reaches this value. Defaults to the counter's saturation
value, which preserves the
25 // legacy behavior for instantiations that do not set this
parameter.
26 parameter int unsigned MaxTrans = 2**CounterWidth - 1,
27 parameter type mst_port_select_t = logic
28 ) (
29     input logic clk_i, // Clock
30     input logic rst_ni, // Asynchronous reset
active low
31     // lookup
32     input logic [AxiIdBits-1:0] lookup_axi_id_i,
33     output mst_port_select_t lookup_mst_select_o,
34     output logic lookup_mst_select_occupied_o,
35     // push
36     output logic full_o,
37     input logic [AxiIdBits-1:0] push_axi_id_i,
38     input mst_port_select_t push_mst_select_i,
39     input logic push_i,
40     // inject ATOPs in AR channel
41     input logic [AxiIdBits-1:0] inject_axi_id_i,
42     input logic inject_i,
43     // pop
44     input logic [AxiIdBits-1:0] pop_axi_id_i,
45     input logic pop_i,
46     // outstanding transactions
47     output logic any_outstanding_trx_o
48 );
49 localparam int unsigned NoCounters = 2**AxiIdBits;
50 typedef logic [CounterWidth-1:0] cnt_t;
51
52 // registers, each gets loaded when push_en[i]
53 mst_port_select_t [NoCounters-1:0] mst_select_q;
54
55 // counter signals
56 logic [NoCounters-1:0] push_en, inject_en, pop_en, occupied,
cnt_full;
57
58 //-----
59 // Lookup
60 //-----
61 assign lookup_mst_select_o = mst_se-
lect_q[lookup_axi_id_i];
62 assign lookup_mst_select_occupied_o = occu-
pied[lookup_axi_id_i];
63 //-----
64 // Push and Pop
65 //-----
66 assign push_en = (push_i) ? (1 << push_axi_id_i) : '0;
67 assign inject_en = (inject_i) ? (1 << inject_axi_id_i) : '0;
68 assign pop_en = (pop_i) ? (1 << pop_axi_id_i) : '0;
69 assign full_o = |cnt_full;
70 //-----
71 // Status
72 //-----
73 assign any_outstanding_trx_o = |occupied;
74
75 // counters
76 for (genvar i = 0; i < NoCounters; i++) begin : gen_counters
77     logic cnt_en, cnt_down, overflow;
78     cnt_t cnt_delta, in_flight;
79     always_comb begin
80         unique case ({push_en[i], inject_en[i], pop_en[i]})
81             3'b001 : begin // pop_i = -1
82                 cnt_en = 1'b1;
83                 cnt_down = 1'b1;
84                 cnt_delta = cnt_t'(1);
85             end
86             3'b010 : begin // inject_i = +1
87                 cnt_en = 1'b1;
88                 cnt_down = 1'b0;
89                 cnt_delta = cnt_t'(1);
90             end
91             // 3'b011, inject_i & pop_i = 0 --> use default
92             3'b100 : begin // push_i = +1
93                 cnt_en = 1'b1;
94                 cnt_down = 1'b0;
95                 cnt_delta = cnt_t'(1);
96             end
97             // 3'b101, push_i & pop_i = 0 --> use default
98             3'b110 : begin // push_i & inject_i = +2
99                 cnt_en = 1'b1;
100                 cnt_down = 1'b0;
101                 cnt_delta = cnt_t'(2);
102             end
103             3'b111 : begin // push_i & inject_i & pop_i = +1
104                 cnt_en = 1'b1;
105                 cnt_down = 1'b0;
106                 cnt_delta = cnt_t'(1);
107             end
108             default : begin // do nothing to the counters
109                 cnt_en = 1'b0;
110                 cnt_down = 1'b0;
111                 cnt_delta = cnt_t'(0);
112             end
113         endcase
114     end
115 end
116 cc_delta_counter #(

```

```

117     .Width          ( CounterWidth ),
118     .StickyOverflow ( 1'b0
119 ) i_in_flight_cnt (
120     .clk_i      ( clk_i      ),
121     .rst_ni     ( rst_ni     ),
122     .clr_i      ( 1'b0      ),
123     .en_i       ( cnt_en     ),
124     .load_i     ( 1'b0      ),
125     .down_i     ( cnt_down   ),
126     .delta_i    ( cnt_delta  ),
127     .d_i        ( '0        ),
128     .q_o        ( in_flight  ),
129     .overflow_o  ( overflow  )
130 );
131 assign occupied[i] = |{overflow, in_flight};
132 // `>=` rather than `==`: a simultaneous push and ATOP in-
jection advances the counter
133 // by 2, which could step over an exact-equality threshold
from `MaxTrans - 1`.
134 assign cnt_full[i] = overflow | (in_flight >= cnt_t'(Max-
Trans));
135
136 // holds the selection signal for this id
137 `FFL(mst_select_q[i], push_mst_select_i, push_en[i], '0,
clk_i, rst_ni)

```

```

138
139 // pragma translate_off
140 `ifndef VERILATOR
141 `ifndef XILINX_SIMULATOR
142     // Validate parameters.
143     cnt_underflow: assert property(
144         @(posedge clk_i) disable iff (~rst_ni) (pop_en[i] |>= !
overflow)) else
145         $fatal(1, "axi_demux_id_counters > Counter: %0d under-
flowed.\
146             The reason is probably a faulty AXI re-
sponse.", i);
147     cnt_full_means_limit: assert property(
148         @(posedge clk_i) disable iff (~rst_ni)
149         (cnt_full[i] |>= ({overflow, in_flight} >= MaxTrans)))
else
150         $fatal(1, "axi_demux_id_counters > Counter %0d reports
full at %0d < MaxTrans = %0d.",
151             i, {overflow, in_flight}, MaxTrans);
152 `endif
153 `endif
154 // pragma translate_on
155 end
156 endmodule
157

```

APB

7 files • 1,178 lines

API — apb_cdc.sv

Table 1011. pulp-platform_apb/src/apb_cdc.sv — module / interface / package (2개)

이름	종류
apb_cdc	module
apb_cdc_intf	module

Table 1012. pulp-platform_apb/src/apb_cdc.sv — parameter (9개)

파라미터	형/폭	기본값
LogDepth		1
req_t		logic
resp_t		logic
addr_t		logic
data_t		logic
strb_t		logic
APB_ADDR_WIDTH	int unsigned	0
APB_DATA_WIDTH	int unsigned	0
LOG_DEPTH	int unsigned	1

Table 1013. pulp-platform_apb/src/apb_cdc.sv — 포트 (12개)

포트	방향	형/폭
src_pclk_i	input	logic
src_preset_ni	input	logic
src_req_i	input	req_t
src_resp_o	output	resp_t
dst_pclk_i	input	logic
dst_preset_ni	input	logic
dst_req_o	output	req_t
dst_resp_i	input	resp_t
src_pclk_i	input	logic
src_preset_ni	input	logic
dst_pclk_i	input	logic
dst_preset_ni	input	logic

Table 1014. pulp-platform_apb/src/apb_cdc.sv — typedef / using (5개)

새 이름	원래 형
src_fsm_state_e	enum logic {Src_Idle, Src_Busy}
dst_fsm_state_e	enum logic[1:0] {Dst_Idle, Dst_Access, Dst_Busy}
addr_t	logic [APB_ADDR_WIDTH-1:0]
data_t	logic [APB_DATA_WIDTH-1:0]
strb_t	logic [APB_DATA_WIDTH/8-1:0]

API — apb_demux.sv

Table 1015. pulp-platform_apb/src/apb_demux.sv — module / interface / package (2개)

이름	종류
apb_demux	module
apb_demux_intf	module

Table 1016. pulp-platform_apb/src/apb_demux.sv — parameter (10개)

파라미터	형/폭	기본값
NoMstPorts	int unsigned	32'd2
req_t		logic
resp_t		logic
SelectWidth	int unsigned	(NoMstPorts > 32'd1)? \$clog2(NoMstPorts) : 32'd1
select_t		logic [SelectWidth-1:0]
APB_ADDR_WIDTH	int unsigned	0
APB_DATA_WIDTH	int unsigned	0
NoMstPorts	int unsigned	32'd2
SelectWidth	int unsigned	(NoMstPorts > 32'd1)? \$clog2(NoMstPorts) : 32'd1
select_t		logic [SelectWidth-1:0]

Table 1017. pulp-platform_apb/src/apb_demux.sv — 포트 (6개)

포트	방향	형/폭
slv_req_i	input	req_t
slv_resp_o	output	resp_t
mst_req_o	output	req_t [NoMstPorts-1:0]
mst_resp_i	input	resp_t [NoMstPorts-1:0]
select_i	input	select_t
select_i	input	select_t

Table 1018. pulp-platform_apb/src/apb_demux.sv — typedef / using (3개)

새 이름	원래 형
addr_t	logic [APB_ADDR_WIDTH-1:0]

새 이름	원래 형
data_t	logic [APB_DATA_WIDTH-1:0]
strb_t	logic [APB_DATA_WIDTH/8-1:0]

API — apb_err_slv.sv

Table 1019. pulp-platform_apb/src/apb_err_slv.sv — module / interface / package (2개)

이름	종류
apb_err_slv	module
apb_err_slv_intf	module

Table 1020. pulp-platform_apb/src/apb_err_slv.sv — parameter (7개)

파라미터	형/폭	기본값
req_t		logic
resp_t		logic
RespWidth	int unsigned	32'd32
RespData	logic [RespWidth-1:0]	32'hBADCAB1E // Hexvalue for the data to return on error.
APB_ADDR_WIDTH	int unsigned	0
APB_DATA_WIDTH	int unsigned	0
RespData	logic [APB_DATA_WIDTH-1:0]	32'hBADCAB1E

Table 1021. pulp-platform_apb/src/apb_err_slv.sv — 포트 (2개)

포트	방향	형/폭
slv_req_i	input	req_t
slv_resp_o	output	resp_t

Table 1022. pulp-platform_apb/src/apb_err_slv.sv — typedef / using (3개)

새 이름	원래 형
addr_t	logic [APB_ADDR_WIDTH-1:0]
data_t	logic [APB_DATA_WIDTH-1:0]
strb_t	logic [APB_DATA_WIDTH/8-1:0]

API — apb_intf.sv

Table 1023. pulp-platform_apb/src/apb_intf.sv — module / interface / package (2개)

이름	종류
APB	interface
APB_DV	interface

Table 1024. pulp-platform_apb/src/apb_intf.sv — parameter (4개)

파라미터	형/폭	기본값
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32

Table 1025. pulp-platform_apb/src/apb_intf.sv — localparam (2개)

상수	형/폭	값
STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(DATA_WIDTH
STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(DATA_WIDTH

Table 1026. pulp-platform_apb/src/apb_intf.sv — 포트 (9개)

포트	방향	형/폭
paddr	output	
pready	input	
paddr	input	
pready	output	
clk_i	input	logic
paddr	output	
pready	input	
paddr	input	
pready	output	

Table 1027. pulp-platform_apb/src/apb_intf.sv — typedef / using (6개)

새 이름	원래 형
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [STRB_WIDTH-1:0]
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [STRB_WIDTH-1:0]

API — apb_pkg.sv**Table 1028. pulp-platform_apb/src/apb_pkg.sv — module / interface / package (1개)**

이름	종류
apb_pkg	package

Table 1029. pulp-platform_apb/src/apb_pkg.sv — localparam (2개)

상수	형/폭	값
RESP_OKAY		1'b0
RESP_SLVERR		1'b1

Table 1030. pulp-platform_apb/src/apb_pkg.sv — typedef / using (1개)

새 이름	원래 형
prot_t	logic [2:0]

API — apb_regs.sv**Table 1031. pulp-platform_apb/src/apb_regs.sv — module / interface / package (2개)**

이름	종류
apb_regs	module
apb_regs_intf	module

Table 1032. pulp-platform_apb/src/apb_regs.sv — parameter (18개)

파라미터	형/폭	기본값
NoApbRegs	int unsigned	32'd0
ApbAddrWidth	int unsigned	32'd0
AddrOffset	int unsigned	32'd4
ApbDataWidth	int unsigned	32'd0
RegDataWidth	int unsigned	32'd0
ReadOnly	bit [NoApbRegs-1:0]	32'h0
req_t		logic
resp_t		logic
apb_addr_t		logic[ApbAddrWidth-1:0]
reg_data_t		logic[RegDataWidth-1:0]
NO_APB_REGS	int unsigned	32'd0
APB_ADDR_WIDTH	int unsigned	32'd0
ADDR_OFFSET	int unsigned	32'd4
APB_DATA_WIDTH	int unsigned	32'd0
REG_DATA_WIDTH	int unsigned	32'd0
READ_ONLY	bit [NO_APB_REGS-1:0]	32'h0
apb_addr_t		logic[APB_ADDR_WIDTH-1:0]
reg_data_t		logic[REG_DATA_WIDTH-1:0]

Table 1033. pulp-platform_apb/src/apb_regs.sv — localparam (2개)

상수	형/폭	값
IdxWidth	int unsigned	(NoApbRegs > 32'd1) ? \$clog2(NoApbRegs) : 32'd1

상수	형/폭	값
APB_STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(APB_DATA_WIDTH

Table 1034. pulp-platform_apb/src/apb_regs.sv — 포트 (12개)

포트	방향	형/폭
pclk_i	input	logic
preset_ni	input	logic
req_i	input	req_t
resp_o	output	resp_t
base_addr_i	input	apb_addr_t
reg_init_i	input	reg_data_t [NoApbRegs-1:0]
reg_q_o	output	reg_data_t [NoApbRegs-1:0]
pclk_i	input	logic
preset_ni	input	logic
base_addr_i	input	apb_addr_t
reg_init_i	input	reg_data_t [NO_APB_REGS-1:0]
reg_q_o	output	reg_data_t [NO_APB_REGS-1:0]

Table 1035. pulp-platform_apb/src/apb_regs.sv — typedef / using (4개)

새 이름	원래 형
idx_t	logic [IdxWidth-1:0]
apb_data_t	logic [ApbAddrWidth-1:0]
apb_data_t	logic [APB_DATA_WIDTH-1:0]
apb_strb_t	logic [APB_STRB_WIDTH-1:0]

API — apb_test.sv**Table 1036. pulp-platform_apb/src/apb_test.sv — module / interface / package (1개)**

이름	종류
apb_test	package

Table 1037. pulp-platform_apb/src/apb_test.sv — parameter (19개)

파라미터	형/폭	기본값
ADDR_WIDTH		32'd32
DATA_WIDTH		32'd32
DATA_WIDTH		32'd32
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
TA	time	0ns

파라미터	형/폭	기본값
TT	time	0ns // test time
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
TA	time	0ps
TT	time	0ps
RESP_MIN_WAIT_CYCLES	int	0
RESP_MAX_WAIT_CYCLES	int	20
ADDR_WIDTH	int unsigned	32'd32
DATA_WIDTH	int unsigned	32'd32
TA	time	0ps
TT	time	0ps
REQ_MIN_WAIT_CYCLES	int	0
REQ_MAX_WAIT_CYCLES	int	20

Table 1038. pulp-platform_apb/src/apb_test.sv — localparam (2개)

상수	형/폭	값
STRB_WIDTH		cf_math_pkg::ceil_div(DATA_WIDTH
STRB_WIDTH	int unsigned	cf_math_pkg::ceil_div(DATA_WIDTH

Table 1039. pulp-platform_apb/src/apb_test.sv — 포트 (9개)

포트	방향	형/폭
addr	input	addr_t
data	output	data_t
err	output	logic
addr	input	addr_t
data	input	data_t
strb	input	strb_t
err	output	logic
request	output	apb_request_t
response	input	apb_response_t

Table 1040. pulp-platform_apb/src/apb_test.sv — typedef / using (15개)

새 이름	원래 형
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [DATA_WIDTH-1:0]
strb_t	logic [STRB_WIDTH-1:0]

새 이름	원래 형
apb_request_t	apb_request #(.ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH))
apb_response_t	apb_response #(.DATA_WIDTH(DATA_WIDTH))
addr_t	apb_driver_t::addr_t
data_t	apb_driver_t::data_t
strb_t	apb_driver_t::strb_t
apb_request_t	apb_driver_t::apb_request_t
apb_response_t	apb_driver_t::apb_response_t
addr_t	apb_driver_t::addr_t
data_t	apb_driver_t::data_t
strb_t	apb_driver_t::strb_t
apb_request_t	apb_driver_t::apb_request_t
apb_response_t	apb_driver_t::apb_response_t

1. pulp-platform_apb/src/apb_cdc.sv (285 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware

3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // APB Clock Domain Crossing
12 // Author: Manuel Eggimann <eggimann@iis.ee.ethz.ch>
13 // Description: A clock domain crossing module on an APB in-
  terface. The module uses gray-counting
14 // CDC FIFOs in both directions to synchronize source side
  with destination side.
15 // Parameters:
16 // - `LogDepth`: The FIFO crossing the clock domain has
  `2^LogDepth` entries

17 // - `req_t`: APB4 request struct. See macro definition in
  `include/typedef.svh`
18 // - `resp_t`: APB4 response struct. See macro defini-
  tion in `include/typedef.svh`
19 //
20 // Ports:
21 //
22 // - `src_pclk_i`: Source side clock input signal (1-bit).

23 // - `src_preset_ni`: Source side asynchronous active low reset
  signal (1-bit).
24 // - `src_req_i`: Source side APB4 request struct, bundles
  all APB4 signals from the master (req_t).

25 // - `src_resp_o`: Source side APB4 response struct, bundles
  all APB4 signals to the master (resp_t).
26 // - `dst_pclk_i`: Destination side clock input signal (1-
  bit).
27 // - `dst_preset_ni`: Destination side asynchronous active low
  reset signal (1-bit).
28 // - `dst_req_o`: Destination side APB4 request struct,
  bundles all APB4 signals to the slave (req_t).
29 // - `dst_resp_i`: Destination side APB4 response struct,
  bundles all APB4 signals from the slave (resp_t).
30 //
31 // This file also features the module `apb_cdc_intf`. The dif-
  ference is that instead of the
32 // request and response structs it uses a `APB` interfaces.
  The parameters have the same
33 // function, however are defined in `ALL_CAPS`.

```

```

34
35 (* no_ungroup *)
36 (* no_boundary_optimization *)
37 module apb_cdc #(
38     parameter LogDepth = 1,
39     parameter type req_t = logic,
40     parameter type resp_t = logic,
41     parameter type addr_t = logic,
42     parameter type data_t = logic,
43     parameter type strb_t = logic
44 ) (
45     // synchronous slave port - clocked by `src_pclk_i`
46     input logic src_pclk_i,
47     input logic src_preset_ni,
48     input req_t src_req_i,
49     output resp_t src_resp_o,
50     // synchronous master port - clocked by `dst_pclk_i`
51     input logic dst_pclk_i,
52     input logic dst_preset_ni,
53     output req_t dst_req_o,
54     input resp_t dst_resp_i
55 );
56
57 typedef struct packed {
58     addr_t paddr;
59     apb_pkg::prot_t pprot;
60     logic pwrite;
61     data_t pwrite;
62     strb_t pstrb;
63 } apb_async_req_data_t;
64
65 typedef struct packed {
66     data_t prdata;
67     logic pslverr;
68 } apb_async_resp_data_t;
69
70 typedef enum logic {Src_Idle, Src_Busy} src_fsm_state_e;
71 typedef enum logic [1:0] {Dst_Idle, Dst_Access, Dst_Busy}
  dst_fsm_state_e;
72 src_fsm_state_e src_state_d, src_state_q;
73 dst_fsm_state_e dst_state_d, dst_state_q;
74 logic src_req_valid, src_req_ready, src_resp_valid,
  src_resp_ready;
75 logic dst_req_valid, dst_req_ready, dst_resp_valid,
  dst_resp_ready;
76
77 apb_async_req_data_t src_req_data, dst_req_data;
78
79 apb_async_resp_data_t dst_resp_data_d, dst_resp_data_q,
  src_resp_data;
80
81 assign src_req_data.paddr = src_req_i.paddr;
82 assign src_req_data.pprot = src_req_i.pprot;
83 assign src_req_data.pwrite = src_req_i.pwrite;
84 assign src_req_data.pwrite = src_req_i.pwrite;
85 assign src_req_data.pstrb = src_req_i.pstrb;
86
87 assign dst_req_o.paddr = dst_req_data.paddr;
88 assign dst_req_o.pprot = dst_req_data.pprot;
89 assign dst_req_o.pwrite = dst_req_data.pwrite;
90 assign dst_req_o.pwrite = dst_req_data.pwrite;
91 assign dst_req_o.pstrb = dst_req_data.pstrb;
92

```

```

93 assign src_resp_o.prdata = src_resp_data.prdata;
94 assign src_resp_o.pslvrr = src_resp_data.pslvrr;
95
96 ///////////////////////////////////////////////////
97 // SRC DOMAIN HANDSHAKE //
98 ///////////////////////////////////////////////////
99
100 // In the source domain we translate simultaneous assertion
of psel and penable into a transaction
101 // on the CDC. The FSM then transitions into a busy state
where it waits for a response coming
102 // back on the other CDC FIFO. Once this response appears
the pready signal is asserted to finish
103 // the APB transaction.
104
105 always_comb begin
106     src_state_d = src_state_q;
107     src_req_valid = 1'b0;
108     src_resp_ready = 1'b0;
109     src_resp_o.pready = 1'b0;
110     case (src_state_q)
111     Src_Idle: begin
112         if (src_req_i.psel & src_req_i.penable) begin
113             src_req_valid = 1'b1;
114             if (src_req_ready) src_state_d = Src_Busy;
115         end
116     end
117     Src_Busy: begin
118         src_resp_ready = 1'b1;
119         if (src_resp_valid) begin
120             src_resp_o.pready = 1'b1;
121             src_state_d = Src_Idle;
122         end
123     end
124     default:;
125     endcase
126 end
127
128 always_ff @(posedge src_pclk_i, negedge src_preset_ni) begin
129     if (!src_preset_ni)
130         src_state_q <= Src_Idle;
131     else
132         src_state_q <= src_state_d;
133     end
134
135 ///////////////////////////////////////////////////
136 // DST DOMAIN HANDSHAKE //
137 ///////////////////////////////////////////////////
138
139
140 // In the destination domain we need to perform a proper APB
handshake with setup and access
141 // phase. Once the destination slave asserts the pready sig-
nal we store the response data and
142 // transition into a busy state. In the busy state we send
the response data back to the src
143 // domain and wait for the transaction to complete.
144
145 always_comb begin
146     dst_state_d = dst_state_q;
147     dst_req_ready = 1'b0;
148     dst_resp_valid = 1'b0;
149     dst_req_o.psel = 1'b0;
150     dst_req_o.penable = 1'b0;
151     dst_resp_data_d = dst_resp_data_q;
152     case (dst_state_q)
153     Dst_Idle: begin
154         if (dst_req_valid) begin
155             dst_req_o.psel = 1'b1;
156             dst_state_d = Dst_Access;
157         end
158     end
159     Dst_Access: begin
160         dst_req_o.psel = 1'b1;
161         dst_req_o.penable = 1'b1;
162         if (dst_resp_i.pready) begin
163             dst_req_ready = 1'b1;
164             dst_resp_data_d.prdata = dst_resp_i.prdata;
165             dst_resp_data_d.pslvrr = dst_resp_i.pslvrr;
166             dst_state_d = Dst_Busy;
167         end
168     end
169     Dst_Busy: begin
170         dst_resp_valid = 1'b1;
171         if (dst_resp_ready) begin
172             dst_state_d = Dst_Idle;
173         end
174     end
175     default: begin
176         dst_state_d = Dst_Idle;
177     end
178     endcase
179 end
180
181 always_ff @(posedge dst_pclk_i, negedge dst_preset_ni) begin
182     if (!dst_preset_ni)
183         dst_state_q <= Dst_Idle;

```

```

187     dst_resp_data_q <= '0;
188 end else begin
189     dst_state_q <= dst_state_d;
190     dst_resp_data_q <= dst_resp_data_d;
191 end
192 end
193
194 ///////////////////////////////////////////////////
195 // CDC FIFOs //
196 ///////////////////////////////////////////////////
197
198 cdc_fifo_gray #(
199     .T ( apb_async_req_data_t ),
200     .LOG_DEPTH ( LogDepth )
201 ) i_cdc_fifo_gray_req (
202     .src_clk_i ( src_pclk_i ),
203     .src_rst_ni ( src_preset_ni ),
204     .src_data_i ( src_req_data ),
205     .src_valid_i ( src_req_valid ),
206     .src_ready_o ( src_req_ready ),
207     .dst_clk_i ( dst_pclk_i ),
208     .dst_rst_ni ( dst_preset_ni ),
209     .dst_data_o ( dst_req_data ),
210     .dst_valid_o ( dst_req_valid ),
211     .dst_ready_i ( dst_req_ready )
212 );
213
214 cdc_fifo_gray #(
215     .T ( apb_async_resp_data_t ),
216     .LOG_DEPTH ( LogDepth )
217 ) i_cdc_fifo_gray_resp (
218     .src_clk_i ( dst_pclk_i ),
219     .src_rst_ni ( dst_preset_ni ),
220     .src_data_i ( dst_resp_data_q ),
221     .src_valid_i ( dst_resp_valid ),
222     .src_ready_o ( dst_resp_ready ),
223     .dst_clk_i ( src_pclk_i ),
224     .dst_rst_ni ( src_preset_ni ),
225     .dst_data_o ( src_resp_data ),
226     .dst_valid_o ( src_resp_valid ),
227     .dst_ready_i ( src_resp_ready )
228 );
229
230 endmodule // apb_cdc
231
232
233 `include "apb/typedef.svh"
234 `include "apb/assign.svh"
235
236 module apb_cdc_intf #(
237     parameter int unsigned APB_ADDR_WIDTH = 0,
238     parameter int unsigned APB_DATA_WIDTH = 0,
239     /// Depth of the FIFO crossing the clock domain, given as
240     2**LOG_DEPTH.
241     parameter int unsigned LOG_DEPTH = 1
242 ) (
243     input logic src_pclk_i,
244     input logic src_preset_ni,
245     APB_Slave src,
246     input logic dst_pclk_i,
247     input logic dst_preset_ni,
248     APB_Master dst
249 );
250
251 typedef logic [APB_ADDR_WIDTH-1:0] addr_t;
252 typedef logic [APB_DATA_WIDTH-1:0] data_t;
253 typedef logic [APB_DATA_WIDTH/8-1:0] strb_t;
254
255 `APB_TYPEDEF_REQ_T(apb_req_t, addr_t, data_t, strb_t)
256 `APB_TYPEDEF_RESP_T(apb_resp_t, data_t)
257
258 apb_req_t src_req, dst_req;
259 apb_resp_t dst_resp, src_resp;
260
261 `APB_ASSIGN_TO_REQ(src_req, src)
262 `APB_ASSIGN_FROM_REQ(dst, dst_req)
263 `APB_ASSIGN_FROM_RESP(src, src_resp)
264 `APB_ASSIGN_TO_RESP(dst_resp, dst)
265
266 apb_cdc #(
267     .LogDepth ( LOG_DEPTH ),
268     .req_t ( apb_req_t ),
269     .resp_t ( apb_resp_t ),
270     .addr_t ( addr_t ),
271     .data_t ( data_t ),
272     .strb_t ( strb_t )
273 ) i_apb_cdc (
274     .src_pclk_i,
275     .src_preset_ni,
276     .src_req_i ( src_req ),
277     .src_resp_o ( src_resp ),
278     .dst_pclk_i,
279     .dst_preset_ni,
280     .dst_req_o ( dst_req ),
281     .dst_resp_i ( dst_resp )
282 );
283
284 endmodule

```

2. pulp-platform_apb/src/apb_demux.sv (108 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 // APB Demultiplexer Description:
12 // This module demultiplexes one APB slave port to several mas-
  ter ports using
13 // the select_i signal.
14
15 module apb_demux #(
16     parameter int unsigned NoMstPorts = 32'd2, // The number of
  demux ports. If
17                                     // 1, degener-
  ates gracefully to a
18                                     // feedthrough.
19     parameter type req_t           = logic, // APB request
20     parameter type resp_t          = logic, // APB response
21     // DEPENDENT PARAMETERS DO NOT OVERWRITE!
22     parameter int unsigned SelectWidth = (NoMstPorts > 32'd1)?
  $clog2(NoMstPorts) : 32'd1,
23     parameter type select_t         = logic [SelectWidth-1:0]
24 ) (
25     input req_t          slv_req_i,
26     output resp_t        slv_resp_o,
27     output req_t [NoMstPorts-1:0] mst_req_o,
28     input resp_t [NoMstPorts-1:0] mst_resp_i,
29     input select_t       select_i
30 );
31
32 if (NoMstPorts == 32'd1) begin
33     assign mst_req_o[0] = slv_req_i;
34     assign slv_resp_o = mst_resp_i[0];
35 end else begin
36     for (genvar idx = 0; idx < NoMstPorts; idx++) begin
37         assign mst_req_o[idx].paddr = slv_req_i.paddr;
38         assign mst_req_o[idx].pprot = slv_req_i.pprot;
39         assign mst_req_o[idx].psel = (select_i == idx)?
  slv_req_i.psel : 1'b0;
40         assign mst_req_o[idx].penable = (select_i == idx)?
  slv_req_i.penable : 1'b0;
41         assign mst_req_o[idx].pwrite = slv_req_i.pwrite;
42         assign mst_req_o[idx].pwrdata = slv_req_i.pwrdata;
43         assign mst_req_o[idx].pstrb = slv_req_i.pstrb;
44     end
45
46     always_comb begin
47         if (select_i < NoMstPorts) begin
48             slv_resp_o.pready = mst_resp_i[select_i].pready;
49             slv_resp_o.prdata = mst_resp_i[select_i].prdata;
50             slv_resp_o.pslverr = mst_resp_i[select_i].pslverr;
51         end else begin
52             slv_resp_o.pready = 1'b1;
53             slv_resp_o.prdata = '0;
54             slv_resp_o.pslverr = 1'b1;
55         end
56     end
57 end
58
59 endmodule
60
61 `include "apb/typedef.svh"
62 `include "apb/assign.svh"
63
64 module apb_demux_intf #(
65     parameter int unsigned APB_ADDR_WIDTH = 0,
66     parameter int unsigned APB_DATA_WIDTH = 0,
67     parameter int unsigned NoMstPorts = 32'd2,
68     // DEPENDENT PARAMETERS DO NOT OVERWRITE!
69     parameter int unsigned SelectWidth = (NoMstPorts > 32'd1)?
  $clog2(NoMstPorts) : 32'd1,
70     parameter type select_t = logic [SelectWidth-1:0]
71 ) (
72     APB.Slave slv,
73     APB.Master mst [NoMstPorts-1:0],
74     input select_t select_i
75 );
76
77 typedef logic [APB_ADDR_WIDTH-1:0] addr_t;
78 typedef logic [APB_DATA_WIDTH-1:0] data_t;

```

```

79 typedef logic [APB_DATA_WIDTH/8-1:0] strb_t;
80
81 `APB_TYPEDEF_REQ_T(apb_req_t, addr_t, data_t, strb_t)
82 `APB_TYPEDEF_RESP_T(apb_resp_t, data_t)
83
84 apb_req_t slv_req;
85 apb_resp_t slv_resp;
86
87 apb_req_t [NoMstPorts-1:0] mst_req;
88 apb_resp_t [NoMstPorts-1:0] mst_resp;
89
90 for (genvar idx = 0; idx < NoMstPorts; idx++) begin
91     `APB_ASSIGN_FROM_REQ(mst[idx], mst_req[idx])
92     `APB_ASSIGN_TO_RESP(mst_resp[idx], mst[idx])
93 end
94 `APB_ASSIGN_TO_REQ(slv_req, slv)
95 `APB_ASSIGN_FROM_RESP(slv, slv_resp)
96
97 apb_demux #(
98     .NoMstPorts ( NoMstPorts ),
99     .req_t      ( apb_req_t ),
100     .resp_t     ( apb_resp_t )
101 ) i_apb_cdc (
102     .slv_req_i ( slv_req ),
103     .slv_resp_o ( slv_resp ),
104     .mst_req_o  ( mst_req ),
105     .mst_resp_i ( mst_resp ),
106     .select_i   ( select_i )
107 );
108 endmodule

```

3. pulp-platform_apb/src/apb_err_slv.sv (79 lines)

```

1 //-----
2 // Title           : APB Error Slave
3 //-----
4 // File            : apb_err_slv.sv
5 // Author           : Manuel Eggimann <meggimann@iis.ee.ethz.ch>
6 // Created          : 05.12.2022
7 //-----
8 // Description :
9 // This module always responds with an error response on any
  request of its APB
10 // slave port. Use this module e.g. as the default port in a
  APB demultiplexer
11 // to handle illegal access. Following the recommendation of
  the APB 2.0
12 // specification, the error response will only be asserted on
  a valid
13 // transaction.
14 //-----
15 // Copyright (C) 2013-2022 ETH Zurich, University of Bologna
16 // Copyright and related rights are licensed under the Solder-
  pad Hardware
17 // License, Version 0.51 (the "License"); you may not use this
  file except in
18 // compliance with the License. You may obtain a copy of the
  License at
19 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
20 // or agreed to in writing, software, hardware and materials
  distributed under
21 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
22 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
23 // specific language governing permissions and limitations un-
  der the License.
24 //-----
25 module apb_err_slv #(
26     parameter type req_t = logic, //
  APB request struct
27     parameter type resp_t = logic, //
  APB response struct
28     parameter int unsigned RespWidth = 32'd32, //
  Data width of the response. Gets zero extended or truncated to
  r.data.
29     parameter logic [RespWidth-1:0] RespData = 32'hBADCAB1E //
  Hexvalue for the data to return on error.
30 ) (
31     input req_t slv_req_i,
32     output resp_t slv_resp_o
33 );
34
35 assign slv_resp_o.prdata = RespData;
36 assign slv_resp_o.pready = 1'b1;
37 // Following the APB recommendations, we only assert the er-
  ror signal if there is actually a
38 // request.
39 assign slv_resp_o.pslverr = (slv_req_i.psel & slv_req_i.penable)?
  apb_pkg::RESP_SLVERR :
  apb_pkg::RESP_OKAY;
41

```

```

42 endmodule
43
44
45 `include "apb/typedef.svh"
46 `include "apb/assign.svh"
47
48 module apb_err_slv_intf #(
49     parameter int unsigned          APB_ADDR_WIDTH = 0,
50     parameter int unsigned          APB_DATA_WIDTH = 0,
51     parameter logic [APB_DATA_WIDTH-1:0] RespData = 32'hBAD-
CAB1E
52 ) (
53     APB.Slave slv
54 );
55
56 typedef logic [APB_ADDR_WIDTH-1:0] addr_t;
57 typedef logic [APB_DATA_WIDTH-1:0] data_t;
58 typedef logic [APB_DATA_WIDTH/8-1:0] strb_t;
59
60 `APB_TYPEDEF_REQ_T(apb_req_t, addr_t, data_t, strb_t)
61 `APB_TYPEDEF_RESP_T(apb_resp_t, data_t)
62
63 apb_req_t slv_req;
64 apb_resp_t slv_resp;
65
66 `APB_ASSIGN_TO_REQ(slv_req, slv)
67 `APB_ASSIGN_FROM_RESP(slv, slv_resp)
68
69 apb_err_slv #(
70     .req_t      ( apb_req_t      ),
71     .resp_t     ( apb_resp_t     ),
72     .RespWidth  ( APB_DATA_WIDTH ),
73     .RespData   ( RespData       )
74 ) i_apb_err_slv (
75     .slv_req_i  ( slv_req ),
76     .slv_resp_o ( slv_resp )
77 );
78
79 endmodule

```

4. pulp-platform_apb/src/apb_intf.sv (78 lines)

```

1 // Copyright (c) 2014-2020 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations under
the License.
11
12 // An APB4 (v2.0) interface
13 interface APB #(
14     parameter int unsigned ADDR_WIDTH = 32'd32,
15     parameter int unsigned DATA_WIDTH = 32'd32
16 );
17     localparam int unsigned STRB_WIDTH =
cf_math_pkg::ceil_div(DATA_WIDTH, 8);
18     typedef logic [ADDR_WIDTH-1:0] addr_t;
19     typedef logic [DATA_WIDTH-1:0] data_t;
20     typedef logic [STRB_WIDTH-1:0] strb_t;
21
22     addr_t      paddr;
23     apb_pkg::prot_t pprot;
24     logic       psel;
25     logic       penable;
26     logic       pwrite;
27     data_t      pwrite;
28     strb_t      pstrb;
29     logic       pready;
30     data_t      prdata;
31     logic       pslverr;
32
33     modport Master (
34         output paddr, pprot, psel, penable, pwrite, pwrite, pstrb,
35         input pready, prdata, pslverr
36     );
37
38     modport Slave (
39         input paddr, pprot, psel, penable, pwrite, pwrite, pstrb,
40         output pready, prdata, pslverr
41     );
42
43 endinterface
44
45 // A clocked APB4 (v2.0) interface for use in design verifica-
tion
46 interface APB_DV #(
47     parameter int unsigned ADDR_WIDTH = 32'd32,

```

```

48     parameter int unsigned DATA_WIDTH = 32'd32
49 ) (
50     input logic clk_i
51 );
52     localparam int unsigned STRB_WIDTH =
cf_math_pkg::ceil_div(DATA_WIDTH, 8);
53     typedef logic [ADDR_WIDTH-1:0] addr_t;
54     typedef logic [DATA_WIDTH-1:0] data_t;
55     typedef logic [STRB_WIDTH-1:0] strb_t;
56
57     addr_t      paddr;
58     apb_pkg::prot_t pprot;
59     logic       psel;
60     logic       penable;
61     logic       pwrite;
62     data_t      pwrite;
63     strb_t      pstrb;
64     logic       pready;
65     data_t      prdata;
66     logic       pslverr;
67
68     modport Master (
69         output paddr, pprot, psel, penable, pwrite, pwrite, pstrb,
70         input pready, prdata, pslverr
71     );
72
73     modport Slave (
74         input paddr, pprot, psel, penable, pwrite, pwrite, pstrb,
75         output pready, prdata, pslverr
76     );
77
78 endinterface

```

5. pulp-platform_apb/src/apb_pkg.sv (22 lines)

```

1 // Copyright (c) 2019 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations under
the License.
10
11 // Author: Wolfgang Roenninger <wroennin@ethz.ch>
12
13 // Description: Package with constant APB v2.0 constants
14
15 package apb_pkg;
16
17     typedef logic [2:0] prot_t;
18
19     localparam RESP_OKAY = 1'b0;
20     localparam RESP_SLVERR = 1'b1;
21
22 endpackage

```

6. pulp-platform_apb/src/apb_regs.sv (264 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations under
the License.
10
11 // APB Read-Write Registers
12 // Description: This module exposes a number of registers on
an APB interface.
13 // It responds to not mapped accesses with a
slave error.
14 // Some of the registers can be configured to be
read only.
15 // Parameters:
16 // - `NoApbRegs`: Number of registers.
17 // - `ApbAddrWidth`: Address width of `req_i.paddr`, is used
to generate internal address map.
18 // - `AddrOffset`: The address offset in bytes between the

```

```

registers. Is asserted to be a least
19 //      '32'd4', the reason is to prevent overlap in
the address map.
20 //      Each register is mapped to a 4 byte wide
address range. When this parameter
21 //      is bigger than '32'd4' there will be
holes in the address map and the module
22 //      answers with `apb_pkg::RESP_SLVERR`. It
is recommended that this value is a
23 //      power of 2, to prevent data alignment is-
sues on upstream buses.
24 // - `ApbDataWidth`: The data width of the APB4 bus, this
value can be up to '32'd32' (bit).
25 // - `RegDataWidth`: The data width of the registers, this
value has to be less or equal to
26 //      `ApbDataWidth`. If it is less the re-
gister gets zero extended for reads and
27 //      higher bits on writes get ignored.
28 // - `ReadOnly`: This flag can specify a read only re-
gister at the given register index of the
29 //      array. When in the array the correspond-
ing bit is set, the `reg_init_i` signal
30 //      at given index can be read out. Writes
are ignored if the flag is set.

31 // - `req_t`: APB4 request struct. See macro definition in
`include/typedef.svh`
32 // - `resp_t`: APB4 response struct. See macro defini-
tion in `include/typedef.svh`
33 //
34 // Ports:
35 //
36 // - `pclk_i`: Clock input signal (1-bit).
37 // - `preset_ni`: Asynchronous active low reset signal (1-
bit).
38 // - `req_i`: APB4 request struct, bundles all APB4
signals from the master (req_t).
39 // - `resp_o`: APB4 response struct, bundles all APB4
signals to the master (resp_t).
40 // - `base_addr_i`: Base address of this module, from here
the registers '0' are mapped, starting
41 //      with index '0'. All subsequent register
with higher indices have their bases
42 //      mapped with reg_index * `AddrOffset` from
this value (ApbAddrWidth-bit).
43 // - `reg_init_i`: Initialization value for each register,
when the register index is configured
44 //      as `ReadOnly[reg_index] == 1'b1` this
value is passed through directly to
45 //      the APB4 bus (Array size `NoApbRegs` *
RegDataWidth-bit).

46 // - `reg_q_o`: The current value of the register. If the
register at the array index is
47 //      read only, the `reg_init_i` value is
passed through to the respective index
48 //      (Array size `NoApbRegs` * RegDataWidth-
bit).
49 //
50 // This file also features the module `apb_regs_intf`. The
difference is that instead of the
51 // request and response structs it uses an `APB.Slave` inter-
face. The parameters have the same
52 // Function, however are defined in `ALL_CAPS`.
53 //
54 //
55 // `include "common_cells/registers.svh"
56 //
57 module apb_regs #(
58     parameter int unsigned      NoApbRegs      = 32'd0,
59     parameter int unsigned      ApbAddrWidth   = 32'd0,
60     parameter int unsigned      AddrOffset     = 32'd4,
61     parameter int unsigned      ApbDataWidth   = 32'd0,
62     parameter int unsigned      RegDataWidth   = 32'd0,
63     parameter bit [NoApbRegs-1:0] ReadOnly     = 32'h0,
64     parameter type              req_t          = logic,
65     parameter type              resp_t         = logic,
66     // DEPENDENT PARAMETERS DO NOT OVERWRITE!
67     parameter type apb_addr_t      = lo-
gic[ApbAddrWidth-1:0],
68     parameter type reg_data_t      = lo-
gic[RegDataWidth-1:0]
69 ) (
70     // APB Interface
71     input  logic                pclk_i,
72     input  logic                preset_ni,
73     input  req_t                req_i,
74     output resp_t               resp_o,
75     // Register Interface
76     input  apb_addr_t           base_addr_i, // base ad-
dress of the read/write registers
77     input  reg_data_t [NoApbRegs-1:0] reg_init_i,
78     output reg_data_t [NoApbRegs-1:0] reg_q_o
79 );
80 localparam int unsigned IdxWidth = (NoApbRegs > 32'd1) ?
$log2(NoApbRegs) : 32'd1;
81 typedef logic [IdxWidth-1:0] idx_t;
82 typedef logic [ApbAddrWidth-1:0] apb_data_t;
83 typedef struct packed {
84     int unsigned idx;
85     apb_addr_t   start_addr;
86     apb_addr_t   end_addr;
87 } rule_t;
88
89 logic has_reset_d, has_reset_q;
90 `FFARN(has_reset_q, has_reset_d, 1'b0, pclk_i, preset_ni)
91 assign has_reset_d = 1'b1;
92
93 // signal declarations
94 rule_t [NoApbRegs-1:0] addr_map;
95 idx_t [NoApbRegs-1:0] reg_idx;
96 logic [NoApbRegs-1:0] decode_valid;
97 // register signals
98 reg_data_t [NoApbRegs-1:0] reg_d, reg_q;
99 logic [NoApbRegs-1:0] reg_update;
100
101 // generate address map for the registers
102 for (genvar i = 0; i < NoApbRegs; i++) begin: gen_reg_ad-
dr_map
103     assign addr_map[i] = rule_t'{
104         idx:      unsigned'(i),
105         start_addr: base_addr_i + apb_addr_t'( i * Ad-
drOffset),
106         end_addr:   base_addr_i + apb_addr_t'((i+32'd1) * Ad-
drOffset)
107     };
108 end
109
110 always_comb begin
111     // default assignments
112     reg_d      = has_reset_q ? reg_q : reg_init_i;
113     reg_update = '0;
114     resp_o     = '{
115         pready: req_i.psel & req_i.penable,
116         prdata: apb_data_t'(32'h0BAD_B10C),
117         pslverr: apb_pkg::RESP_OKAY
118     };
119     if (req_i.psel) begin
120         if (!decode_valid) begin
121             // Error response on decode errors
122             resp_o.pslverr = apb_pkg::RESP_SLVERR;
123         end else begin
124             if (req_i.pwrite) begin
125                 if (!ReadOnly[reg_idx]) begin
126                     for (int unsigned i = 0; i < RegDataWidth; i++)
127                         if (req_i.pstrb[i/8]) begin
128                             reg_d[reg_idx][i] = req_i.pwdata[i];
129                         end
130                     end
131                     reg_update[reg_idx] = |req_i.pstrb;
132                 end else begin
133                     // this register is read only
134                     resp_o.pslverr = apb_pkg::RESP_SLVERR;
135                 end
136             end else begin
137                 if (!ReadOnly[reg_idx]) begin
138                     resp_o.prdata = apb_data_t'(reg_q[reg_idx]);
139                 end else begin
140                     // for read only register the directly connect the
init signal
141                     resp_o.prdata = apb_data_t'(reg_init_i[reg_idx]);
142                 end
143             end
144         end
145     end
146 end
147
148 // output assignment and registers
149 for (genvar i = 0; i < NoApbRegs; i++) begin: gen_rw_regs
150     assign reg_q_o[i] = ReadOnly[i] ? reg_init_i[i] :
reg_q[i];
151     `FFLARN(reg_q[i], reg_d[i], reg_update[i], '0, pclk_i,
preset_ni)
152 end
153
154 addr_decode #(
155     .NoIndices ( NoApbRegs ),
156     .NoRules ( NoApbRegs ),
157     .addr_t ( apb_addr_t ),
158     .rule_t ( rule_t )
159 ) i_addr_decode (
160     .addr_i ( req_i.paddr ),
161     .addr_map_i ( addr_map ),
162     .idx_o ( reg_idx ),
163     .dec_valid_o ( decode_valid ),
164     .dec_error_o ( /*not used*/ ),
165     .en_default_idx_i ( '0 ),
166     .default_idx_i ( '0 )
167 );
168
169 // Validate parameters.
170 // pragma translate_off
171 `ifndef VERILATOR
172     initial begin: p_assertions
173         assert (NoApbRegs > 32'd0)
174             else $fatal(1, "The number of registers must be at
least 1!");
175         assert (ApbAddrWidth > 32'd2)

```



```

176     else $fatal(1, "ApbAddrWidth is not wide enough, has
to be at least 3 bit wide!");
177     assert (AddrOffset > 32'd3)
178     else $fatal(1, "AddrOffset has to be at least 4 and
is recommended to be a power of 2!");
179     assert ($bits(req_i.paddr) == ApbAddrWidth)
180     else $fatal(1, "AddrWidth does not match req_i.padr-
dr!");
181     assert (ApbDataWidth == $bits(resp_o.pdata))

182     else $fatal(1, "ApbDataWidth has to be: ApbDataWidth
== $bits(req_i.pdata)!");
183     assert (ApbDataWidth > 32'd0 && ApbDataWidth <= 32'd32)
184     else $fatal(1, "ApbDataWidth has to be: 32'd32 >=
RegDataWidth > 0!");
185     assert ($bits(resp_o.pdata) == $bits(req_i.pdata))
186     else $fatal(1, "req_i.pdata has to match
resp_o.pdata in width!");
187     assert (RegDataWidth > 32'd0 && RegDataWidth <= 32'd32)
188     else $fatal(1, "RegDataWidth has to be: 32'd32 >=
RegDataWidth > 0!");
189     assert (RegDataWidth <= $bits(resp_o.pdata))

190     else $fatal(1, "RegDataWidth has to be: RegDataWidth
<= $bits(req_i.pdata)!");
191     assert (NoApbRegs == $bits(ReadOnly))
192     else $fatal(1, "Each register need a 'ReadOnly'
flag!");
193     end
194     `endif
195     // pragma translate_on
196 endmodule
197
198 `include "apb/assign.svh"
199 `include "apb/typedef.svh"
200
201 module apb_regs_intf #(
202     parameter int unsigned          NO_APB_REGS    = 32'd0, //
number of read only registers
203     parameter int unsigned          APB_ADDR_WIDTH = 32'd0, //
address width of `paddr`
204     parameter int unsigned          ADDR_OFFSET    = 32'd4, //
address offset in bytes
205     parameter int unsigned          APB_DATA_WIDTH = 32'd0, //
data width of the registers
206     parameter int unsigned          REG_DATA_WIDTH = 32'd0,
207     parameter bit [NO_APB_REGS-1:0] READ_ONLY    = 32'h0,
208     // DEPENDENT PARAMETERS DO NOT OVERTWRITE!
209     parameter type                  apb_addr_t     = lo-
gic[APB_ADDR_WIDTH-1:0],
210     parameter type                  reg_data_t      = lo-
gic[REG_DATA_WIDTH-1:0]
211 ) (
212     // APB Interface
213     input logic                     pclk_i,
214     input logic                     preset_ni,
215     APB_Slave                       slv,
216     // Register Interface
217     input apb_addr_t                base_addr_i, // base ad-
dress of the read only registers
218     input reg_data_t [NO_APB_REGS-1:0] reg_init_i, // initial-
isation value for the registers
219     output reg_data_t [NO_APB_REGS-1:0] reg_q_o
220 );
221     localparam int unsigned APB_STRB_WIDTH =
cf_math_pkg::ceil_div(APB_DATA_WIDTH, 8);
222     typedef logic [APB_DATA_WIDTH-1:0] apb_data_t;
223     typedef logic [APB_STRB_WIDTH-1:0] apb_strb_t;
224
225     `APB_TYPEDEF_REQ_T(apb_req_t, apb_addr_t, apb_data_t,
apb_strb_t)
226     `APB_TYPEDEF_RESP_T(apb_resp_t, apb_data_t)
227
228     apb_req_t  apb_req;
229     apb_resp_t apb_resp;
230
231     `APB_ASSIGN_TO_REQ(apb_req, slv)
232     `APB_ASSIGN_FROM_RESP(slv, apb_resp )
233
234     apb_regs #(
235         .NoApbRegs    ( NO_APB_REGS ),
236         .ApbAddrWidth ( APB_ADDR_WIDTH ),
237         .AddrOffset    ( ADDR_OFFSET ),
238         .ApbDataWidth  ( APB_DATA_WIDTH ),
239         .RegDataWidth  ( REG_DATA_WIDTH ),
240         .ReadOnly      ( READ_ONLY ),
241         .req_t          ( apb_req_t ),
242         .resp_t         ( apb_resp_t )
243     ) i_apb_regs (
244         .pclk_i,
245         .preset_ni,
246         .req_i    ( apb_req ),
247         .resp_o   ( apb_resp ),
248         .base_addr_i,
249         .reg_init_i,
250         .reg_q_o
251     );
252
253     // Validate parameters.
254     // pragma translate_off

```

```

255 `ifndef VERILATOR
256     initial begin: p_assertions
257         assert (APB_ADDR_WIDTH == $bits(slv.paddr))
258         else $fatal(1, "APB_ADDR_WIDTH does not match slv
interface!");
259         assert (APB_DATA_WIDTH == $bits(slv.pdata))
260         else $fatal(1, "APB_DATA_WIDTH does not match slv
interface!");
261     end
262 `endif
263 // pragma translate_on
264 endmodule

```

7. pulp-platform_apb/src/apb_test.sv (342 lines)

```

1 // Copyright (c) 2018 ETH Zurich, University of Bologna
2 //
3 // Copyright and related rights are licensed under the Solder-
pad Hardware
4 // License, Version 0.51 (the "License"); you may not use this
file except in
5 // compliance with the License. You may obtain a copy of the
License at
6 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
7 // or agreed to in writing, software, hardware and materials
distributed under
8 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
9 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
10 // specific language governing permissions and limitations un-
der the License.
11
12 // Test infrastructure for APB interfaces
13 package apb_test;
14
15 class apb_request #(
16     parameter ADDR_WIDTH = 32'd32,
17     parameter DATA_WIDTH = 32'd32
18 );
19     localparam STRB_WIDTH = cf_math_pkg::ceil_div(DATA_WIDTH,
8);
20
21     rand logic [ADDR_WIDTH-1:0] paddr = '0;
22     rand logic [DATA_WIDTH-1:0] pdata = '0;
23     rand logic [STRB_WIDTH-1:0] pstrb = '0;
24     rand logic                  pwrite = 1'b1;
25 endclass;
26
27 class apb_response #(
28     parameter DATA_WIDTH = 32'd32
29 );
30     rand logic [DATA_WIDTH-1:0] prdata = '0;
31     rand logic                  pslverr = 1'b0;
32 endclass;
33
34 // Simple APB driver with thread-safe read and write func-
tions
35 class apb_driver #(
36     parameter int unsigned ADDR_WIDTH = 32'd32, // APB4 ad-
dress width
37     parameter int unsigned DATA_WIDTH = 32'd32, // APB4 data
width
38     parameter time          TA          = 0ns, // application
time
39     parameter time          TT          = 0ns // test time
40 );
41     localparam int unsigned STRB_WIDTH =
cf_math_pkg::ceil_div(DATA_WIDTH, 8);
42     typedef logic [ADDR_WIDTH-1:0] addr_t;
43     typedef logic [DATA_WIDTH-1:0] data_t;
44     typedef logic [STRB_WIDTH-1:0] strb_t;
45     typedef apb_request
#(ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH)) apb_request_t;
46     typedef apb_response #(DATA_WIDTH(DATA_WIDTH)) apb_re-
sponse_t;
47     virtual APB_DV #(
48         .ADDR_WIDTH(ADDR_WIDTH),
49         .DATA_WIDTH(DATA_WIDTH)
50     ) apb;
51     std::semaphore lock;
52
53     function new(virtual APB_DV
#(ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH)) apb);
54         this.apb = apb;
55         this.lock = new(1);
56     endfunction
57
58     function void reset_master();
59         apb.paddr <= '0;
60         apb.pprot <= '0;
61         apb.psel <= 1'b0;
62         apb.penable <= 1'b0;
63         apb.pwrite <= 1'b0;
64         apb.pdata <= '0;
65         apb.pstrb <= '0;
66     endfunction

```

```

67
68 function void reset_slave();
69     apb.pready <= 1'b0;
70     apb.prdata <= '0;
71     apb.pslverr <= 1'b0;
72 endfunction
73
74 task cycle_start;
75     #TT;
76 endtask
77
78 task cycle_end;
79     @(posedge apb.clk_i);
80 endtask
81
82 // this task reads from an APB4 slave, acts as master
83 task read(
84     input addr_t addr,
85     output data_t data,
86     output logic err
87 );
88     while (!lock.try_get()) begin
89         cycle_end();
90     end
91     apb.paddr <= #TA addr;
92     apb.pwrite <= #TA 1'b0;
93     apb.psel <= #TA 1'b1;
94     cycle_end();
95     apb.penable <= #TA 1'b1;
96     cycle_start();
97     while (!apb.pready) begin
98         cycle_end();
99         cycle_start();
100    end
101    data = apb.prdata;
102    err = apb.pslverr;
103    cycle_end();
104    apb.paddr <= #TA '0;
105    apb.psel <= #TA 1'b0;
106    apb.penable <= #TA 1'b0;
107    lock.put();
108 endtask
109
110 // this task writes to an APB4 slave, acts as master
111 task write(
112     input addr_t addr,
113     input data_t data,
114     input strb_t strb,
115     output logic err
116 );
117     while (!lock.try_get()) begin
118         cycle_end();
119     end
120     apb.paddr <= #TA addr;
121     apb.pwdata <= #TA data;
122     apb.pstrb <= #TA strb;
123     apb.pwrite <= #TA 1'b1;
124     apb.psel <= #TA 1'b1;
125     cycle_end();
126     apb.penable <= #TA 1'b1;
127     cycle_start();
128     while (!apb.pready) begin
129         cycle_end();
130         cycle_start();
131     end
132     err = apb.pslverr;
133     cycle_end();
134     apb.paddr <= #TA '0;
135     apb.pwdata <= #TA '0;
136     apb.pstrb <= #TA '0;
137     apb.pwrite <= #TA 1'b0;
138     apb.psel <= #TA 1'b0;
139     apb.penable <= #TA 1'b0;
140     lock.put();
141 endtask
142
143 // Wait for incoming apb request
144 task rcv_request (
145     output apb_request_t request
146 );
147     cycle_start();
148     while (!apb.psel) begin cycle_end(); cycle_start(); end
149     while (!apb.penable) begin cycle_end(); cycle_start(); end
150     request = new;
151     request.paddr = apb.paddr;
152     request.pwdata = apb.pwdata;
153     request.pwrite = apb.pwrite;
154     request.pstrb = apb.pstrb;
155     cycle_end();
156 endtask
157
158 // Acknowledge incoming APB write request (assert pready).
159 task send_write_ack();
160     apb.pready <= #TA 1;
161     cycle_end();
162     apb.pready <= #TA 0;
163 endtask
164
165 // Send response for incoming APB read request acknow-
166     ledging the request (assert pready).

```

```

166 task send_read_response (
167     input apb_response_t response
168 );
169     apb.pready <= #TA 1;
170     apb.prdata <= #TA response.prdata;
171     apb.pslverr <= #TA response.pslverr;
172     cycle_end();
173     apb.pready <= #TA 0;
174     apb.prdata <= #TA '0;
175     apb.pslverr <= #TA 0;
176 endtask
177
178 endclass
179
180 class apb_rand_slave #(
181     // APB interface parameters
182     parameter int unsigned ADDR_WIDTH = 32'd32,
183     parameter int unsigned DATA_WIDTH = 32'd32,
184     // Stimuli application and test time
185     parameter time TA = 0ps,
186     parameter time TT = 0ps,
187     // Upper and lower bounds on wait states on Response
188     parameter int RESP_MIN_WAIT_CYCLES = 0,
189     parameter int RESP_MAX_WAIT_CYCLES = 20
190 );
191
192 typedef apb_test::apb_driver #(
193     .ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH), .TA(TA
194 ), .TT(TT)
195 ) apb_driver_t;
196
197 typedef apb_driver_t::addr_t addr_t;
198 typedef apb_driver_t::data_t data_t;
199 typedef apb_driver_t::strb_t strb_t;
200
201 typedef apb_driver_t::apb_request_t apb_request_t;
202 typedef apb_driver_t::apb_response_t apb_response_t;
203
204 apb_driver_t drv;
205 std::mailbox #(apb_request_t) request_queue;
206 std::mailbox #(apb_response_t) response_queue;
207
208 function new(
209     virtual APB_DV #(
210         .ADDR_WIDTH(ADDR_WIDTH),
211         .DATA_WIDTH(DATA_WIDTH)
212     ) apb
213 );
214     this.drv = new(apb);
215     this.request_queue = new;
216     this.response_queue = new;
217     this.reset();
218 endfunction
219
220 function reset();
221     drv.reset_slave();
222 endfunction
223
224 // TODO: The `rand_wait` task exists in `rand_verif_pkg`,
225 but that task cannot be called with
226 // `this.drv.apb.clk_i` as `clk` argument. What is the
227 syntax getting an assignable reference?
228 task automatic rand_wait(input int unsigned min, max);
229     int unsigned rand_success, cycles;
230     rand_success = std::randomize(cycles) with {
231         cycles >= min;
232         cycles <= max;
233     };
234     assert (rand_success) else $error("Failed to randomize
235 wait cycles!");
236     repeat (cycles) @(posedge this.drv.apb.clk_i);
237 endtask : rand_wait
238
239 task handle_requests();
240     forever begin
241         automatic apb_request_t request;
242         automatic apb_response_t read_response = new;
243         automatic logic rand_success;
244         drv.rcv_request(request);
245         request_queue.put(request);
246         if (request.pwrite) begin
247             rand_wait(RESP_MIN_WAIT_CYCLES,
248             RESP_MAX_WAIT_CYCLES);
249             drv.send_write_ack();
250         end else begin
251             rand_success = read_response.randomize(); as-
252             sert(rand_success);
253             response_queue.put(read_response);
254             rand_wait(RESP_MIN_WAIT_CYCLES,
255             RESP_MAX_WAIT_CYCLES);
256             drv.send_read_response(read_response);
257         end
258     end
259 endtask : handle_requests // rcv_requests
260
261 task run();
262     handle_requests();
263 endtask
264
265 endclass

```

```

259
260 class apb_rand_master #(
261     // APB interface parameters
262     parameter int unsigned ADDR_WIDTH = 32'd32,
263     parameter int unsigned DATA_WIDTH = 32'd32,
264     // Stimuli application and test time
265     parameter time TA = 0ps,
266     parameter time TT = 0ps,
267     // Upper and lower bounds on wait cycles on request chan-
nel
268     parameter int REQ_MIN_WAIT_CYCLES = 0,
269     parameter int REQ_MAX_WAIT_CYCLES = 20
270 );
271
272     typedef apb_test::apb_driver #(
273         .ADDR_WIDTH(ADDR_WIDTH), .DATA_WIDTH(DATA_WIDTH), .TA(TA
), .TT(TT)
274     ) apb_driver_t;
275
276     typedef apb_driver_t::addr_t addr_t;
277     typedef apb_driver_t::data_t data_t;
278     typedef apb_driver_t::strb_t strb_t;
279
280     typedef apb_driver_t::apb_request_t apb_request_t;
281     typedef apb_driver_t::apb_response_t apb_response_t;
282
283     apb_driver_t drv;
284     std::mailbox #(apb_request_t) request_queue;
285     std::mailbox #(apb_response_t) response_queue;
286
287
288     function new(
289         virtual APB_DV #(
290             .ADDR_WIDTH(ADDR_WIDTH),
291             .DATA_WIDTH(DATA_WIDTH)
292         ) apb
293     );
294         this.drv = new(apb);
295         this.request_queue = new;
296         this.response_queue = new;
297         this.reset();
298     endfunction
299
300     function reset();
301         drv.reset_master();
302     endfunction
303
304     // TODO: The `rand_wait` task exists in `rand_verif_pkg`,

```

```

but that task cannot be called with
305     // `this.drv.apb.clk_i` as `clk` argument. What is the
syntax getting an assignable reference?
306     task automatic rand_wait(input int unsigned min, max);
307         int unsigned rand_success, cycles;
308         rand_success = std::randomize(cycles) with {
309             cycles >= min;
310             cycles <= max;
311         };
312         assert (rand_success) else $error("Failed to randomize
wait cycles!");
313         repeat (cycles) @(posedge this.drv.apb.clk_i);
314     endtask
315
316     task automatic send_requests(input int unsigned n_re-
quests);
317         automatic apb_request_t request = new;
318         automatic apb_response_t response = new;
319         automatic logic write_err;
320         automatic logic rand_success;
321         repeat (n_requests) begin
322             rand_wait(REQ_MIN_WAIT_CYCLES, REQ_MAX_WAIT_CYCLES);
323             rand_success = request.randomize(); as-
sert(rand_success);
324             request_queue.put(request);
325             if (request.pwrite) begin
326                 $info("Sending write request with addr: %h, pstrb:
%b and pwdata: %h", request.paddr, request.pstrb, request.pwdata);
327                 drv.write(request.paddr, request.pwdata, re-
quest.pstrb, write_err);
328             end else begin
329                 $info("Sending read request with addr: %h...", re-
quest.paddr);
330                 drv.read(request.paddr, response.prdata, re-
sponse.pslverr);
331                 response_queue.put(response);
332             end
333         end
334     endtask : send_requests
335
336     task automatic run(input int unsigned n_request);
337         $info("Run apb_rand_master for %0d transactions", n_re-
quest);
338         send_requests(n_request);
339     endtask
340
341 endclass
342 endpackage

```

RISC-V 디버거

12 files • 3,665 lines

API — dm_csrs.sv

Table 1041. pulp-platform_riscv-dbg/src/dm_csrs.sv — module / interface / package (1개)

이름	종류
dm_csrs	module

Table 1042. pulp-platform_riscv-dbg/src/dm_csrs.sv — parameter (3개)

파라미터	형/폭	기본값
NrHarts	int unsigned	1
BusWidth	int unsigned	32
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}

Table 1043. pulp-platform_riscv-dbg/src/dm_csrs.sv — localparam (6개)

상수	형/폭	값
DataIndexWidth	int unsigned	$\$clog2(dm::DataCount)$
HartSelLen	int unsigned	$(NrHarts == 1) ? 1 : \$clog2(NrHarts)$
NrHartsAligned	int unsigned	$2^{HartSelLen}$
HartSelMask	logic [19:0]	$20'((2^{HartSelLen} - 1))$
DataEnd	dm::dm_csr_e	$dm::dm_csr_e'(dm::Data0 + \{4'h0$
ProgBufEnd	dm::dm_csr_e	$dm::dm_csr_e'(dm::ProgBuf0 + \{4'h0$

Table 1044. pulp-platform_riscv-dbg/src/dm_csrs.sv — 포트 (46개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
next_dm_addr_i	input	logic [31:0]
testmode_i	input	logic
dmi_rst_ni	input	logic
dmi_req_valid_i	input	logic
dmi_req_ready_o	output	logic
dmi_req_i	input	dm::dmi_req_t
dmi_resp_valid_o	output	logic
dmi_resp_ready_i	input	logic
dmi_resp_o	output	dm::dmi_resp_t
ndmreset_o	output	logic

포트	방향	형/폭
ndmreset_ack_i	input	logic
dmactive_o	output	logic
hartinfo_i	input	dm::hartinfo_t [NrHarts-1:0]
halted_i	input	logic [NrHarts-1:0]
unavailable_i	input	logic [NrHarts-1:0]
resumeack_i	input	logic [NrHarts-1:0]
hartsel_o	output	logic [19:0]
haltreq_o	output	logic [NrHarts-1:0]
resumereq_o	output	logic [NrHarts-1:0]
clear_resumeack_o	output	logic [NrHarts-1:0]
cmd_valid_o	output	logic
cmd_o	output	dm::command_t
cmderror_valid_i	input	logic
cmderror_i	input	dm::cmderr_e
cmdbusy_i	input	logic
progbuf_o	output	logic [dm::ProgBufSize-1:0][31:0]
data_o	output	logic [dm::DataCount-1:0][31:0]
data_i	input	logic [dm::DataCount-1:0][31:0]
data_valid_i	input	logic
sbaddress_o	output	logic [BusWidth-1:0]
sbaddress_i	input	logic [BusWidth-1:0]
sbaddress_write_valid_o	output	logic
sbreadonaddr_o	output	logic
sbautoincrement_o	output	logic
sbaccess_o	output	logic [2:0]
sbreadondata_o	output	logic
sbdata_o	output	logic [BusWidth-1:0]
sbdata_read_valid_o	output	logic
sbdata_write_valid_o	output	logic
sbdata_i	input	logic [BusWidth-1:0]
sbdata_valid_i	input	logic
sbbusy_i	input	logic
sberror_valid_i	input	logic
occurred	input	logic [2:0] sberror_i // bus error

API — dm_mem.sv**Table 1045. pulp-platform_riscv-dbg/src/dm_mem.sv — module / interface / package (1개)**

이름	종류
dm_mem	module

Table 1046. pulp-platform_riscv-dbg/src/dm_mem.sv — parameter (5개)

파라미터	형/폭	기본값
NrHarts	int unsigned	1
BusWidth	int unsigned	32
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
DmBaseAddress	int unsigned	'0
MaxRegisterAccessWidth	int unsigned	BusWidth

Table 1047. pulp-platform_riscv-dbg/src/dm_mem.sv — localparam (20개)

상수	형/폭	값
DbgAddressBits	int unsigned	12
DataIndexWidth	int unsigned	\$clog2(dm::DataCount)
HartSelLen	int unsigned	(NrHarts == 1) ? 1 : \$clog2(NrHarts)
NrHartsAligned	int unsigned	2**HartSelLen
MaxAar	int unsigned	(MaxRegisterAccessWidth == 64) ? 4 : 3
HasSndScratch	bit	(DmBaseAddress != 0)
LoadBaseAddr	logic [4:0]	(DmBaseAddress == 0) ? 5'd0 : 5'd10
DataBaseAddr	logic [DbgAddressBits-1:0]	(dm::DataAddr)
DataEndAddr	logic [DbgAddressBits-1:0]	(dm::DataAddr + 4*dm::DataCount - 1)
ProgBufBaseAddr	logic [DbgAddressBits-1:0]	(dm::DataAddr - 4*dm::ProgBufSize)
ProgBufEndAddr	logic [DbgAddressBits-1:0]	(dm::DataAddr - 1)
AbstractCmdBaseAddr	logic [DbgAddressBits-1:0]	(ProgBufBaseAddr - 4*10)
AbstractCmdEndAddr	logic [DbgAddressBits-1:0]	(ProgBufBaseAddr - 1)
WhereToAddr	logic [DbgAddressBits-1:0]	'h300
FlagsBaseAddr	logic [DbgAddressBits-1:0]	'h400
FlagsEndAddr	logic [DbgAddressBits-1:0]	'h7FF
HaltedAddr	logic [DbgAddressBits-1:0]	'h100
GoingAddr	logic [DbgAddressBits-1:0]	'h108
ResumingAddr	logic [DbgAddressBits-1:0]	'h110
ExceptionAddr	logic [DbgAddressBits-1:0]	'h118

Table 1048. pulp-platform_riscv-dbg/src/dm_mem.sv — 포트 (26개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
dmactive_i	input	logic
debug_req_o	output	logic [NrHarts-1:0]
ndmreset_i	input	logic
hartsel_i	input	logic [19:0]
haltreq_i	input	logic [NrHarts-1:0]
resumereq_i	input	logic [NrHarts-1:0]
clear_resumeack_i	input	logic [NrHarts-1:0]
halted_o	output	logic [NrHarts-1:0]
resuming_o	output	logic [NrHarts-1:0]
progbuf_i	input	logic [dm::ProgBufSize-1:0][31:0]
data_i	input	logic [dm::DataCount-1:0][31:0]
data_o	output	logic [dm::DataCount-1:0][31:0]
data_valid_o	output	logic
cmd_valid_i	input	logic
cmd_i	input	dm::command_t
cmderror_valid_o	output	logic
cmderror_o	output	dm::cmderr_e
cmdbusy_o	output	logic
req_i	input	logic
we_i	input	logic
addr_i	input	logic [BusWidth-1:0]
wdata_i	input	logic [BusWidth-1:0]
be_i	input	logic [BusWidth/8-1:0]
rdata_o	output	logic [BusWidth-1:0]

Table 1049. `pulp-platform_riscv-dbg/src/dm_mem.sv` — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [1:0] { Idle, Go, CmdExecuting }

API — dm_obi_top.svTable 1050. `pulp-platform_riscv-dbg/src/dm_obi_top.sv` — module / interface / package (1개)

이름	종류
dm_obi_top	module

Table 1051. pulp-platform_riscv-dbg/src/dm_obi_top.sv — parameter (6개)

파라미터	형/폭	기본값
IdWidth	int unsigned	1
NrHarts	int unsigned	1
BusWidth	int unsigned	32
DmBaseAddress	int unsigned	'h1000
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
MaxRegisterAccessWidth	int unsigned	BusWidth

Table 1052. pulp-platform_riscv-dbg/src/dm_obi_top.sv — 포트 (35개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
testmode_i	input	logic
ndmreset_o	output	logic
dmactive_o	output	logic
debug_req_o	output	logic [NrHarts-1:0]
unavailable_i	input	logic [NrHarts-1:0]
hartinfo_i	input	dm::hartinfo_t [NrHarts-1:0]
slave_req_i	input	logic
slave_gnt_o	output	logic
slave_we_i	input	logic
slave_addr_i	input	logic [BusWidth-1:0]
slave_be_i	input	logic [BusWidth/8-1:0]
slave_wdata_i	input	logic [BusWidth-1:0]
slave_aid_i	input	logic [IdWidth-1:0]
slave_rvalid_o	output	logic
slave_rdata_o	output	logic [BusWidth-1:0]
slave_rid_o	output	logic [IdWidth-1:0]
master_req_o	output	logic
master_addr_o	output	logic [BusWidth-1:0]
master_we_o	output	logic
master_wdata_o	output	logic [BusWidth-1:0]
master_be_o	output	logic [BusWidth/8-1:0]
master_gnt_i	input	logic
master_rvalid_i	input	logic

포트	방향	형/폭
master_err_i	input	logic
master_other_err_i	input	logic
master_rdata_i	input	logic [BusWidth-1:0]
dmi_rst_ni	input	logic
dmi_req_valid_i	input	logic
dmi_req_ready_o	output	logic
dmi_req_i	input	dm::dmi_req_t
dmi_resp_valid_o	output	logic
dmi_resp_ready_i	input	logic
dmi_resp_o	output	dm::dmi_resp_t

API — dm_pkg.sv

Table 1053. pulp-platform_riscv-dbg/src/dm_pkg.sv — module / interface / package (1개)

이름	종류
dm	package

Table 1054. pulp-platform_riscv-dbg/src/dm_pkg.sv — localparam (11개)

상수	형/폭	값
DbgVersion013	logic [3:0]	4'h2
ProgBufSize	logic [4:0]	5'h8
DataCount	logic [3:0]	4'h2
HaltAddress	logic [63:0]	64'h800
ResumeAddress	logic [63:0]	HaltAddress + 8
ExceptionAddress	logic [63:0]	HaltAddress + 16
DataAddr	logic [11:0]	12'h380
CauseBreakpoint	logic [2:0]	3'h1
CauseTrigger	logic [2:0]	3'h2
CauseRequest	logic [2:0]	3'h3
CauseSingleStep	logic [2:0]	3'h4

API — dm_sba.sv

Table 1055. pulp-platform_riscv-dbg/src/dm_sba.sv — module / interface / package (1개)

이름	종류
dm_sba	module

Table 1056. pulp-platform_riscv-dbg/src/dm_sba.sv — parameter (2개)

파라미터	형/폭	기본값
BusWidth	int unsigned	32
ReadByteEnable	bit	1

Table 1057. pulp-platform_riscv-dbg/src/dm_sba.sv — localparam (1개)

상수	형/폭	값
BeIdxWidth	int	\$clog2(BusWidth/8)

Table 1058. pulp-platform_riscv-dbg/src/dm_sba.sv — 포트 (28개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
dmactive_i	input	logic
master_req_o	output	logic
master_add_o	output	logic [BusWidth-1:0]
master_we_o	output	logic
master_wdata_o	output	logic [BusWidth-1:0]
master_be_o	output	logic [BusWidth/8-1:0]
master_gnt_i	input	logic
master_r_valid_i	input	logic
master_r_err_i	input	logic
master_r_other_err_i	input	logic
master_r_rdata_i	input	logic [BusWidth-1:0]
sbaddress_i	input	logic [BusWidth-1:0]
sbaddress_write_valid_i	input	logic
sbreadonaddr_i	input	logic
sbaddress_o	output	logic [BusWidth-1:0]
sbautoincrement_i	input	logic
sbaccess_i	input	logic [2:0]
sbreadondata_i	input	logic
sbdata_i	input	logic [BusWidth-1:0]
sbdata_read_valid_i	input	logic
sbdata_write_valid_i	input	logic
sbdata_o	output	logic [BusWidth-1:0]
sbdata_valid_o	output	logic
sbbusy_o	output	logic
sberror_valid_o	output	logic

포트	방향	형/폭
occurred	output	logic [2:0] sberror_o // bus error

API — dm_top.sv

Table 1059. pulp-platform_riscv-dbg/src/dm_top.sv — module / interface / package (1개)

이름	종류
dm_top	module

Table 1060. pulp-platform_riscv-dbg/src/dm_top.sv — parameter (6개)

파라미터	형/폭	기본값
NrHarts	int unsigned	1
BusWidth	int unsigned	32
DmBaseAddress	int unsigned	'h1000
SelectableHarts	logic [NrHarts-1:0]	{NrHarts{1'b1}}
ReadByteEnable	bit	1
MaxRegisterAccessWidth	int unsigned	BusWidth

Table 1061. pulp-platform_riscv-dbg/src/dm_top.sv — 포트 (33개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
next_dm_addr_i	input	logic [31:0]
testmode_i	input	logic
ndmreset_o	output	logic
ndmreset_ack_i	input	logic
dmactive_o	output	logic
debug_req_o	output	logic [NrHarts-1:0]
unavailable_i	input	logic [NrHarts-1:0]
hartinfo_i	input	dm::hartinfo_t [NrHarts-1:0]
slave_req_i	input	logic
slave_we_i	input	logic
slave_addr_i	input	logic [BusWidth-1:0]
slave_be_i	input	logic [BusWidth/8-1:0]
slave_wdata_i	input	logic [BusWidth-1:0]
slave_rdata_o	output	logic [BusWidth-1:0]
master_req_o	output	logic
master_add_o	output	logic [BusWidth-1:0]

포트	방향	형/폭
master_we_o	output	logic
master_wdata_o	output	logic [BusWidth-1:0]
master_be_o	output	logic [BusWidth/8-1:0]
master_gnt_i	input	logic
master_r_valid_i	input	logic
master_r_err_i	input	logic
master_r_other_err_i	input	logic
master_r_rdata_i	input	logic [BusWidth-1:0]
dmi_rst_ni	input	logic
dmi_req_valid_i	input	logic
dmi_req_ready_o	output	logic
dmi_req_i	input	dm::dmi_req_t
dmi_resp_valid_o	output	logic
dmi_resp_ready_i	input	logic
dmi_resp_o	output	dm::dmi_resp_t

API — dmi_bscane_tap.sv

Table 1062. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv — module / interface / package (1개)

이름	종류
dmi_jtag_tap	module

Table 1063. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv — parameter (2개)

파라미터	형/폭	기본값
IrLength	int unsigned	5
IdcodeValue	logic [31:0]	32'h00000001

Table 1064. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv — 포트 (17개)

포트	방향	형/폭
tck_i	input	logic
tms_i	input	logic
trst_ni	input	logic
td_i	input	logic
td_o	output	logic
tdo_oe_o	output	logic
testmode_i	input	logic
tck_o	output	logic

포트	방향	형/폭
dmi_clear_o	output	logic
update_o	output	logic
capture_o	output	logic
shift_o	output	logic
tdi_o	output	logic
dtmcs_select_o	output	logic
dtmcs_tdo_i	input	logic
dmi_select_o	output	logic
dmi_tdo_i	input	logic

API — dmi_cdc.sv

Table 1065. pulp-platform_riscv-dbg/src/dmi_cdc.sv — module / interface / package (1개)

이름	종류
dmi_cdc	module

Table 1066. pulp-platform_riscv-dbg/src/dmi_cdc.sv — 포트 (18개)

포트	방향	형/폭
tck_i	input	logic
trst_ni	input	logic
jtag_dmi_req_i	input	dm::dmi_req_t
jtag_dmi_ready_o	output	logic
jtag_dmi_valid_i	input	logic
jtag_dmi_cdc_clear_i	input	logic
jtag_dmi_resp_o	output	dm::dmi_resp_t
jtag_dmi_valid_o	output	logic
jtag_dmi_ready_i	input	logic
clk_i	input	logic
rst_ni	input	logic
core_dmi_rst_no	output	logic
core_dmi_req_o	output	dm::dmi_req_t
core_dmi_valid_o	output	logic
core_dmi_ready_i	input	logic
core_dmi_resp_i	input	dm::dmi_resp_t
core_dmi_ready_o	output	logic
core_dmi_valid_i	input	logic

API — dmi_intf.sv**Table 1067. pulp-platform_riscv-dbg/src/dmi_intf.sv — module / interface / package (1개)**

이름	종류
DMI_BUS_DV	interface

Table 1068. pulp-platform_riscv-dbg/src/dmi_intf.sv — parameter (1개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	-1

Table 1069. pulp-platform_riscv-dbg/src/dmi_intf.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
q_addr	input	
q_ready	output	
q_addr	output	
q_ready	input	
q_addr	input	

Table 1070. pulp-platform_riscv-dbg/src/dmi_intf.sv — typedef / using (2개)

새 이름	원래 형
addr_t	logic [ADDR_WIDTH-1:0]
data_t	logic [31:0]

API — dmi_jtag.sv**Table 1071. pulp-platform_riscv-dbg/src/dmi_jtag.sv — module / interface / package (1개)**

이름	종류
dmi_jtag	module

Table 1072. pulp-platform_riscv-dbg/src/dmi_jtag.sv — parameter (1개)

파라미터	형/폭	기본값
IdcodeValue	logic [31:0]	32'h00000DB3

Table 1073. pulp-platform_riscv-dbg/src/dmi_jtag.sv — 포트 (16개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
testmode_i	input	logic
dmi_rst_no	output	logic
dmi_req_o	output	dm::dmi_req_t

포트	방향	형/폭
dmi_req_valid_o	output	logic
dmi_req_ready_i	input	logic
dmi_resp_i	input	dm::dmi_resp_t
dmi_resp_ready_o	output	logic
dmi_resp_valid_i	input	logic
tck_i	input	logic
tms_i	input	logic
trst_ni	input	logic
td_i	input	logic
td_o	output	logic
enable	output	logic tdo_oe_o // Data out output

Table 1074. pulp-platform_riscv-dbg/src/dmi_jtag.sv — typedef / using (1개)

새 이름	원래 형
state_e	enum logic [2:0] { Idle, Read, WaitReadValid, Write, WaitWriteValid }

API — dmi_jtag_tap.sv

Table 1075. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv — module / interface / package (1개)

이름	종류
dmi_jtag_tap	module

Table 1076. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv — parameter (2개)

파라미터	형/폭	기본값
IrLength	int unsigned	5
IdcodeValue	logic [31:0]	32'h00000001

Table 1077. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv — 포트 (17개)

포트	방향	형/폭
tck_i	input	logic
tms_i	input	logic
trst_ni	input	logic
td_i	input	logic
td_o	output	logic
tdo_oe_o	output	logic
testmode_i	input	logic
tck_o	output	logic
dmi_clear_o	output	logic

포트	방향	형/폭
update_o	output	logic
capture_o	output	logic
shift_o	output	logic
tdi_o	output	logic
dtmcs_select_o	output	logic
dtmcs_tdo_i	input	logic
dmi_select_o	output	logic
dmi_tdo_i	input	logic

API — dmi_test.sv

Table 1078. pulp-platform_riscv-dbg/src/dmi_test.sv — module / interface / package (1개)

이름	종류
dmi_test	package

Table 1079. pulp-platform_riscv-dbg/src/dmi_test.sv — parameter (24개)

파라미터	형/폭	기본값
AW	int	7
AW	int	-1
TA	realtime	0
TT	realtime	0 // stimuli test time
AW	int	32
TA	realtime	0ps
TT	realtime	0ps
AW	int	32
TA	realtime	0ps
TT	realtime	0ps
REQ_MIN_WAIT_CYCLES	int unsigned	1
REQ_MAX_WAIT_CYCLES	int unsigned	20
RSP_MIN_WAIT_CYCLES	int unsigned	1
RSP_MAX_WAIT_CYCLES	int unsigned	20
AW	int	32
TA	realtime	0ps
TT	realtime	0ps
REQ_MIN_WAIT_CYCLES	int unsigned	0
REQ_MAX_WAIT_CYCLES	int unsigned	10
RSP_MIN_WAIT_CYCLES	int unsigned	0

파라미터	형/폭	기본값
RSP_MAX_WAIT_CYCLES	int unsigned	10
AW	int	32
TA	realtime	Ops
TT	realtime	Ops

1. pulp-platform_riscv-dbg/src/dm_csrs.sv (722 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2  * Copyright and related rights are licensed under the Solder-
3  * pad Hardware
4  *
5  * License, Version 0.51 (the "License"); you may not use this
6  * file except in
7  * compliance with the License. You may obtain a copy of the
8  * License at
9  * http://solderpad.org/licenses/SHL-0.51. Unless required by
10 * applicable law
11 * or agreed to in writing, software, hardware and materials
12 * distributed under
13 * this License is distributed on an "AS IS" BASIS, WITHOUT
14 * WARRANTIES OR
15 * CONDITIONS OF ANY KIND, either express or implied. See the
16 * License for the
17 * specific language governing permissions and limitations under
18 * the License.
19 *
20 * File: dm_csrs.sv
21 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
22 * Date: 30.6.2018
23 *
24 * Description: Debug CSRs. Communication over Debug Transport
25 * Module (DTM)
26 */
27
28 module dm_csrs #(
29     parameter int unsigned      NrHarts      = 1,
30     parameter int unsigned      BusWidth     = 32,
31     parameter logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}}
32 ) (
33     input logic clk_i, // Clock
34     input logic rst_ni, // Asynchronous reset active low
35     input logic [31:0] dr_i, // Static next_dm word address.
36     input logic testmode_i,
37     input logic dmi_rst_ni, // sync. DTM reset,
38
39     // active-low
40     input logic dmi_req_valid_i,
41     output logic dmi_req_ready_o,
42     input logic dmi_req_t,
43     // every request needs a response one cycle later
44     output logic dmi_resp_valid_o,
45     input logic dmi_resp_ready_i,
46     output logic dmi_resp_o,
47     // global ctrl
48     output logic ndmre-,
49     set_o, // non-debug module reset active-high
50     input logic ndmre-,
51     set_ack_i, // non-debug module reset ack pulse
52     output logic dmact-,
53     ive_o, // 1 -> debug-module is active,
54
55     // 0 -> synchronous re-set
56     input logic [NrHarts-1:0] hartinfo_t,
57     input logic [NrHarts-1:0] hartinfo_i, // static hartinfo
58     input logic [NrHarts-1:0] hal-,
59     ted_i, // hart is halted
60     input logic [NrHarts-1:0] unavail-,
61     able_i, // e.g.: powered down
62     input logic [NrHarts-1:0] re-,
63     sumeack_i, // hart acknowledged resume request
64     // hart control
65     output logic [19:0] hartsel_o, // hartselect to ctrl module
66     output logic [NrHarts-1:0] hal-,
67     tree_o, // request to halt a hart
68     output logic [NrHarts-1:0] resume-,
69     req_o, // request hart to resume
70     // Clears stale acknowledgements before issuing new resume
71     requests
72     output logic [NrHarts-1:0] clear_resumeack_o,
73     output logic cmd_val-

```

```

id_o, // debugger writing to cmd field
54 output dm::command_t
cmd_o, // abstract command
55 input logic cmderr-,
ror_valid_i, // an error occurred
56 input dm::cmderr_e cmderr-,
ror_i, // this error occurred
57 input logic cmd-,
busy_i, // cmd is currently busy executing
58
59 output logic [dm::ProgBufSize-1:0][31:0] progbuf_o, // to
system bus
60 output logic [dm::DataCount-1:0][31:0] data_o,
61
62 input logic [dm::DataCount-1:0][31:0] data_i,
63 input logic data_valid_i,
64 // system bus access module (SBA)
65 output logic [BusWidth-1:0] sbaddress_o,
66 input logic [BusWidth-1:0] sbaddress_i,
67 output logic sbad-,
dress_write_valid_o,
68 // control signals in
69 output logic sbreadonaddr_o,
70 output logic sbautoincrement_o,
71 output logic [2:0] sbaccess_o,
72 // data out
73 output logic sbreadondata_o,
74 output logic [BusWidth-1:0] sbdata_o,
75 output logic sb-,
data_read_valid_o,
76 output logic sb-,
data_write_valid_o,
77 // read data in
78 input logic [BusWidth-1:0] sbdata_i,
79 input logic sbdata_valid_i,
80 // control signals
81 input logic sbbusy_i,
82 input logic sber-,
ror_valid_i, // bus error occurred
83 input logic [2:0] sberror_i // bus
error occurred
84 );
85 // the amount of bits we need to represent all harts
86 localparam int unsigned DataIndexWidth = $clog2-
g2(dm::DataCount);
87 localparam int unsigned HartSelLen = (NrHarts == 1) ? 1 :
$clog2(NrHarts);
88 localparam int unsigned NrHartsAligned = 2**HartSelLen;
89 // Keep only the implemented WARL hart-select bits
90 localparam logic [19:0] HartSelMask = 20'((2**$clog2-
g2(NrHarts)) - 1);
91
92 dm::dtm_op_e dtm_op;
93 assign dtm_op = dm::dtm_op_e'(dmi_req_i.op);
94
95 logic resp_queue_full;
96 logic resp_queue_empty;
97 logic resp_queue_push;
98 logic resp_queue_pop;
99
100 localparam dm::dm_csr_e DataEnd = dm::dm_csr_e'(dm::Data0 +
{4'h0, dm::DataCount} - 8'h1);
101 localparam dm::dm_csr_e ProgBufEnd = dm::dm_csr_e'(dm::Prog-
Buf0 + {4'h0, dm::ProgBufSize} - 8'h1);
102
103 logic [31:0] haltsum0, haltsum1, haltsum2, haltsum3;
104 logic [(NrHarts-1)/2**5 + 1] * 32 - 1 : 0] halted;
105 logic [(NrHarts-1)/2**5:0][31:0] halted_resaped0;
106 logic [(NrHarts-1)/2**10:0][31:0] halted_resaped1;
107 logic [(NrHarts-1)/2**15:0][31:0] halted_resaped2;
108 logic [(NrHarts-1)/2**10+1]*32-1:0] halted_flat1;
109 logic [(NrHarts-1)/2**15+1]*32-1:0] halted_flat2;
110 logic [31:0] halted_flat3;
111
112 // haltsum0
113 logic [14:0] hartsel_idx0;
114 always_comb begin : p_haltsum0
115     halted = '0;
116     haltsum0 = '0;
117     hartsel_idx0 = hartsel_o[19:5];
118     halted[NrHarts-1:0] = halted_i;
119     halted_resaped0 = halted;
120     if (hartsel_idx0 < 15'((NrHarts-1)/2**5+1)) begin
121         haltsum0 = halted_resaped0[hartsel_idx0];
122     end

```

```

123 end
124
125 // haltsum1
126 logic [9:0] hartsel_idx1;
127 always_comb begin : p_reduction1
128     halted_flat1 = '0;
129     haltsum1 = '0;
130     hartsel_idx1 = hartsel_o[19:10];
131
132     for (int unsigned k = 0; k < (NrHarts-1)/2**5+1; k++) be-
gin
133         halted_flat1[k] = |halted_resaped0[k];
134     end
135     halted_resaped1 = halted_flat1;
136
137     if (hartsel_idx1 < 10'(((NrHarts-1)/2**10+1))) begin
138         haltsum1 = halted_resaped1[hartsel_idx1];
139     end
140 end
141
142 // haltsum2
143 logic [4:0] hartsel_idx2;
144 always_comb begin : p_reduction2
145     halted_flat2 = '0;
146     haltsum2 = '0;
147     hartsel_idx2 = hartsel_o[19:15];
148
149     for (int unsigned k = 0; k < (NrHarts-1)/2**10+1; k++) be-
gin
150         halted_flat2[k] = |halted_resaped1[k];
151     end
152     halted_resaped2 = halted_flat2;
153
154     if (hartsel_idx2 < 5'(((NrHarts-1)/2**15+1))) begin
155         haltsum2 = halted_resaped2[hartsel_idx2];
156     end
157 end
158
159 // haltsum3
160 always_comb begin : p_reduction3
161     halted_flat3 = '0;
162     for (int unsigned k = 0; k < NrHarts/2**15+1; k++) begin
163         halted_flat3[k] = |halted_resaped2[k];
164     end
165     haltsum3 = halted_flat3;
166 end
167
168
169 dm::dmstatus_t dmstatus;
170 dm::dmcontrol_t dmcontrol_d, dmcontrol_q, dmcon-
trol_read;
171 dm::abstractcs_t abstractcs;
172 dm::cmderr_e cmderr_d, cmderr_q;
173 dm::command_t command_d, command_q;
174 logic cmd_valid_d, cmd_valid_q;
175 dm::abstractauto_t abstractauto_d, abstractauto_q;
176 dm::sbcs_t sbcs_d, sbcs_q;
177 logic [63:0] sbaddr_d, sbaddr_q;
178 logic [63:0] sbdata_d, sbdata_q;
179
180 logic [NrHarts-1:0] havereset_d, havereset_q;
181 logic [NrHarts-1:0] resume_pending_d, resume_pending_q;
182 // program buffer
183 logic [dm::ProgBufSize-1:0][31:0] progbuf_d, progbuf_q;
184 logic [dm::DataCount-1:0][31:0] data_d, data_d_dmi, data_q;
185
186 logic [HartSelLen-1:0] selected_hart;
187 logic selected_hart_valid;
188
189 dm::dmi_resp_t resp_queue_inp;
190
191 assign dmi_resp_valid_o = ~resp_queue_empty;
192 assign dmi_req_ready_o = ~resp_queue_full;
193 assign resp_queue_push = dmi_req_valid_i &
dmi_req_ready_o;
194 // SBA
195 assign sbautoincrement_o = sbcs_q.sbautoincrement;
196 assign sbreadonaddr_o = sbcs_q.sbreadonaddr;
197 assign sbreadondata_o = sbcs_q.sbreadondata;
198 assign sbaccess_o = sbcs_q.sbaccess;
199 assign sbdata_o = sbdata_q[BusWidth-1:0];
200 assign sbaddress_o = sbaddr_q[BusWidth-1:0];
201
202 assign hartsel_o = {dmcontrol_q.hartselhi, dmcon-
trol_q.hartsello};
203 assign selected_hart = hartsel_o[HartSelLen-1:0];
204 assign selected_hart_valid = selected_hart <= HartSel-
Len'(NrHarts - 1);
205
206 logic [NrHartsAligned-1:0] havereset_q_aligned, re-
sumeack_aligned, unavailable_aligned,
207 halted_aligned;
208 assign resumeack_aligned = NrHartsAligned'(resumeack_i);
209 assign unavailable_aligned = NrHartsAligned'(unavailable_i);
210 assign halted_aligned = NrHartsAligned'(halted_i);
211
212 assign havereset_q_aligned = NrHartsAligned'(havereset_q);
213
214 dm::hartinfo_t [NrHartsAligned-1:0] hartinfo_aligned;
215 always_comb begin : p_hartinfo_align
216     hartinfo_aligned = '0;

```

```

217     hartinfo_aligned[NrHarts-1:0] = hartinfo_i;
218 end
219
220 // helper variables
221 dm::dm_csr_e dm_csr_addr;
222 dm::sbcs_t sbcs;
223 logic [3:0] autoexecdata_idx; // 0 == Data0 ... 11 == Data11
224
225 // Decode immediate write effects independently of stored
CSR state
226 logic [19:0] dmcontrol_write_hartsel;
227 logic [HartSelLen-1:0] dmcontrol_write_hart;
228 logic dmcontrol_write_hart_valid;
229 logic dmcontrol_write_accepted;
230 logic dmcontrol_write_resumereq;
231 logic dmcontrol_write_haltreq;
232 logic dmcontrol_write_dmtime;
233
234 // Get the data index, i.e. 0 for dm::Data0 up to 11 for
dm::Data11
235 assign dm_csr_addr = dm::dm_csr_e'({1'b0, dmi_req_i.addr});
236 // Xilinx Vivado 2020.1 does not allow subtraction of two
enums; do the subtraction with logic
237 // types instead.
238 assign autoexecdata_idx = 4'({dm_csr_addr} - {dm::Data0});
239
240 assign dmcontrol_write_hartsel = {dmi_req_i.data[15:6],
dmi_req_i.data[25:16]}
241 & HartSelMask;
242 assign dmcontrol_write_hart = dmcon-
trol_write_hartsel[HartSelLen-1:0];
243 assign dmcontrol_write_hart_valid = dmcontrol_write_hartsel
<= 20'(NrHarts - 1);
244 assign dmcontrol_write_accepted = dmi_req_ready_o &
dmi_req_valid_i &
245 dtm_op == dm::DTM_WRITE &&
246 dm_csr_addr == dm::DMCon-
trol;
247 assign dmcontrol_write_resumereq = dmcontrol_write_accepted
&& !cmdbusy_i && dmi_req_i.data[30];
248 assign dmcontrol_write_haltreq = dmi_req_i.data[31];
249 assign dmcontrol_write_dmtime = dmi_req_i.data[0];
250
251 always_comb begin : p_clear_resumeack
252     clear_resumeack_o = '0;
253     if (dmcontrol_write_resumereq && !dmcontrol_write_haltreq
&&
254 dmcontrol_write_hart_valid && Selectable-
Harts[dmcontrol_write_hart]) begin
255         clear_resumeack_o[dmcontrol_write_hart] = 1'b1;
256     end
257 end
258
259 always_comb (*xprop_off *) begin : csr_read_write
260     // -----
261     // Static Values (R/O)
262     // -----
263     // dmstatus
264     dmstatus = '0;
265     dmstatus.version = dm::DbgVersion013;
266     // no authentication implemented
267     dmstatus.authenticated = 1'b1;
268     // we do not support halt-on-reset sequence
269     dmstatus.hasreset_haltreq = 1'b0;
270     // TODO(zarubaf) things need to change here if we imple-
ment the array mask
271     dmstatus.allhavereset = haver-
eset_q_aligned[selected_hart];
272     dmstatus.anyhavereset = haver-
eset_q_aligned[selected_hart];
273
274     dmstatus.allresumeack = resumeack_aligned[selected_hart];
275     dmstatus.anyresumeack = resumeack_aligned[selected_hart];
276
277     dmstatus.allunavail = unavail-
able_aligned[selected_hart];
278     dmstatus.anyunavail = unavail-
able_aligned[selected_hart];
279
280 // as soon as we are out of the legal Hart region tell the
debugger
281 // that there are only non-existent harts
282 dmstatus.allnonexistent = logic'(32'(hartsel_o) > (NrHarts -
1));
283 dmstatus.anynonexistent = logic'(32'(hartsel_o) > (NrHarts -
1));
284
285 // We are not allowed to be in multiple states at once.
This is a
286 // make the running/halted and unavailable states exclus-
ive.
287 dmstatus.allhalted = halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];
288 dmstatus.anyhalted = halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];
289
290 dmstatus.allrunning = ~halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];

```

```

291 dmstatus.anyrunning = ~halted_aligned[selected_hart] &
~unavailable_aligned[selected_hart];
292
293 // abstractcs
294 abstractcs = '0';
295 abstractcs.datacount = dm::DataCount;
296 abstractcs.progbufsize = dm::ProgBufSize;
297 abstractcs.busy = cmdbusy_i;
298 abstractcs.cmderr = cmderr_q;
299
300 // abstractautoexec
301 abstractauto_d = abstractauto_q;
302 abstractauto_d.zero0 = '0';
303
304 // default assignments
305 havereset_d = havereset_q;
306 dmcontrol_d = dmcontrol_q;
307 dmcontrol_read = dmcontrol_q;
308 dmcontrol_read.haltreq = 1'b0;
309 dmcontrol_read.resumereq = 1'b0;
310 resume_pending_d = resume_pending_q;
311 // Acknowledgements retire only the corresponding pending
actions
312 resume_pending_d &= ~resumeack_i;
313 cmderr_d = cmderr_q;
314 command_d = command_q;
315 progbuf_d = progbuf_q;
316 data_d_dmi = data_q;
317 sbcs_d = sbcs_q;
318 sbaddr_d = 64'(sbaddress_i);
319 sbdata_d = sbdata_q;
320
321 resp_queue_inp.data = 32'h0;
322 resp_queue_inp.resp = dm::DTM_SUCCESS;
323 cmd_valid_d = 1'b0;
324 sbaddress_write_valid_o = 1'b0;
325 sbdata_read_valid_o = 1'b0;
326 sbdata_write_valid_o = 1'b0;
327
328 // helper variables
329 sbcs = '0';
330
331 // reads
332 if (dmi_req_ready_o && dmi_req_valid_i && dtm_op ==
dm::DTM_READ) begin
333   unique case (dm_csr_addr) inside
334     [(dm::Data0):DataEnd]: begin
335       resp_queue_inp.data = data_q[DataIndex-
Width'(autoexecdata_idx)];
336       if (!cmdbusy_i) begin
337         // check whether we need to re-execute the command
(just give a cmd_valid)
338         if (cmderr_q == dm::CmdErrNone)
339           cmd_valid_d = abstractau-
to_q.autoexecdata[autoexecdata_idx];
340         // An abstract command was executing while one of
the data registers was read
341       end else begin
342         resp_queue_inp.resp = dm::DTM_BUSY;
343         if (cmderr_q == dm::CmdErrNone) begin
344           cmderr_d = dm::CmdErrBusy;
345         end
346       end
347     end
348   dm::DMControl: resp_queue_inp.data = dmcon-
trol_read;
349   dm::DMStatus: resp_queue_inp.data = dmstatus;
350   dm::Hartinfo: resp_queue_inp.data =
hartinfo_aligned[selected_hart];
351   dm::AbstractCS: resp_queue_inp.data = abstractcs;
352   dm::AbstractAuto: resp_queue_inp.data = abstractau-
to_q;
353   dm::Command: resp_queue_inp.data = '0';
354   dm::NextDM: resp_queue_inp.data = next_dm_ad-
dr_i;
355   [(dm::ProgBuf0):ProgBufEnd]: begin
356     resp_queue_inp.data = prog-
buf_q[dmi_req_i.addr[$clog2(dm::ProgBufSize)-1:0]];
357     if (!cmdbusy_i) begin
358       // check whether we need to re-execute the command
(just give a cmd_valid)
359       // range of autoexecprogbuf is 31:16
360       if (cmderr_q == dm::CmdErrNone)
361         cmd_valid_d = abstractau-
to_q.autoexecprogbuf[1'b1, dmi_req_i.addr[3:0]];
362       // An abstract command was executing while one of
the progbuf registers was read
363     end else begin
364       resp_queue_inp.resp = dm::DTM_BUSY;
365       if (cmderr_q == dm::CmdErrNone) begin
366         cmderr_d = dm::CmdErrBusy;
367       end
368     end
369   end
370   dm::HaltSum0: resp_queue_inp.data = haltsum0;
371   dm::HaltSum1: resp_queue_inp.data = haltsum1;
372   dm::HaltSum2: resp_queue_inp.data = haltsum2;
373   dm::HaltSum3: resp_queue_inp.data = haltsum3;

```

```

375 dm::SBSCS: begin
376   resp_queue_inp.data = sbcs_q;
377 end
378 dm::SBAddress0: begin
379   resp_queue_inp.data = sbaddr_q[31:0];
380 end
381 dm::SBAddress1: begin
382   resp_queue_inp.data = sbaddr_q[63:32];
383 end
384 dm::SBData0: begin
385   // access while the SBA was busy
386   if (sbbusy_i || sbcs_q.sbbusyerror) begin
387     sbcs_d.sbbusyerror = 1'b1;
388     resp_queue_inp.resp = dm::DTM_BUSY;
389   end else begin
390     sbdata_read_valid_o = (sbcs_q.sbbusyerror == '0');
391     resp_queue_inp.data = sbdata_q[31:0];
392   end
393 end
394 dm::SBData1: begin
395   // access while the SBA was busy
396   if (sbbusy_i || sbcs_q.sbbusyerror) begin
397     sbcs_d.sbbusyerror = 1'b1;
398     resp_queue_inp.resp = dm::DTM_BUSY;
399   end else begin
400     resp_queue_inp.data = sbdata_q[63:32];
401   end
402 end
403 default;;
404 endcase
405 end
406
407 // write
408 if (dmi_req_ready_o && dmi_req_valid_i && dtm_op ==
dm::DTM_WRITE) begin
409   unique case (dm_csr_addr) inside
410     [(dm::Data0):DataEnd]: begin
411       if (dm::DataCount > 0) begin
412         // attempts to write them while busy is set does
not change their value
413         if (!cmdbusy_i) begin
414           data_d_dmi[dmi_req_i.addr[DataIndexWidth-1:0]] =
dmi_req_i.data;
415           // check whether we need to re-execute the com-
mand (just give a cmd_valid)
416           if (cmderr_q == dm::CmdErrNone)
417             cmd_valid_d = abstractau-
to_q.autoexecdata[autoexecdata_idx];
418           //An abstract command was executing while one of
the data registers was written
419         end else begin
420           resp_queue_inp.resp = dm::DTM_BUSY;
421           if (cmderr_q == dm::CmdErrNone) begin
422             cmderr_d = dm::CmdErrBusy;
423           end
424         end
425       end
426     end
427   dm::DMControl: begin
428     if (cmdbusy_i) begin
429       dmcontrol_d.ndmreset = dmi_req_i.data[1];
430       dmcontrol_d.dmactive = dmi_req_i.data[0];
431     end else begin
432       dmcontrol_d = dmi_req_i.data;
433     end
434     // clear the havereset of the selected hart
435   if (dmcontrol_d.ackhavereset && selected_hart_valid)
begin
436     havereset_d[selected_hart] = 1'b0;
437   end
438 end
439 dm::DMStatus;; // write are ignored to R/O register
440 dm::Hartinfo;; // hartinfo is R/O
441 // only command error is write-able
442 dm::AbstractCS: begin // W1C
443   // Gets set if an abstract command fails. The bits
in this
444   // field remain set until they are cleared by writ-
ing 1 to
445   // them. No abstract command is started until the
value is
446   // reset to 0.
447   // reads during abstract command execution are not
allowed
448   if (!cmdbusy_i) begin
449     cmderr_d = dm::cmderr_e'(~dmi_req_i.data[10:8] &
cmderr_q);
450   end else begin
451     resp_queue_inp.resp = dm::DTM_BUSY;
452     if (cmderr_q == dm::CmdErrNone) begin
453       cmderr_d = dm::CmdErrBusy;
454     end
455   end
456 end
457 dm::Command: begin
458   // writes are ignored if a command is already busy
or if cmderr is non-zero
459   if (!cmdbusy_i && cmderr_q == dm::CmdErrNone) begin
460     cmd_valid_d = 1'b1;

```

```

461         command_d = dm::command_t'(dmi_req_i.data);
462         // if there was an attempted to write during a busy
execution
463         // and the cmderror field is zero set the busy error
464         end else if (cmdbusy_i) begin
465             resp_queue_inp.resp = dm::DTM_BUSY;
466             if (cmderr_q == dm::CmdErrNone) begin
467                 cmderr_d = dm::CmdErrBusy;
468             end
469         end
470     end
471     dm::NextDM; // nextdm is R/O
472     dm::AbstractAuto: begin
473         // this field can only be written legally when there
is no command executing
474         if (!cmdbusy_i) begin
475             abstractauto_d = 32'h0;
476             abstractauto_d.autoexecdata =
12'(dmi_req_i.data[dm::DataCount-1:0]);
477             abstractauto_d.autoexecprogbuf =
16'(dmi_req_i.data[dm::ProgBufSize-1+16:16]);
478         end else begin
479             resp_queue_inp.resp = dm::DTM_BUSY;
480             if (cmderr_q == dm::CmdErrNone) begin
481                 cmderr_d = dm::CmdErrBusy;
482             end
483         end
484     end
485     [(dm::ProgBuf0):ProgBufEnd]: begin
486         // attempts to write them while busy is set does not
change their value
487         if (!cmdbusy_i) begin
488             prog-
buf_d[dmi_req_i.addr[$clog2(dm::ProgBufSize)-1:0]] = dmi_req_i.data;
489         // check whether we need to re-execute the command
(just give a cmd_valid)
490         // this should probably throw an error if executed
during another command
491         // was busy
492         // range of autoexecprogbuf is 31:16
493         if (cmderr_q == dm::CmdErrNone)
494             cmd_valid_d = abstractau-
to_q.autoexecprogbuf[{1'b1, dmi_req_i.addr[3:0]}];
495         //An abstract command was executing while one of the
progbuf registers was written
496         end else begin
497             resp_queue_inp.resp = dm::DTM_BUSY;
498             if (cmderr_q == dm::CmdErrNone) begin
499                 cmderr_d = dm::CmdErrBusy;
500             end
501         end
502     end
503     dm::SBCS: begin
504         // access while the SBA was busy
505         if (sbbusy_i) begin
506             sbcs_d.sbbusyserror = 1'b1;
507             resp_queue_inp.resp = dm::DTM_BUSY;
508         end else begin
509             sbcs = dm::sbcs_t'(dmi_req_i.data);
510             sbcs_d = sbcs;
511             // R/W1C
512             sbcs_d.sbbusyserror = sbcs.q.sbbusyserror &
(~sbcs.sbbusyserror);
513             sbcs_d.sbberror = (sbcs.sbberror) ? 3'b0 :
sbcs.q.sbberror;
514         end
515     end
516     dm::SBAddress0: begin
517         // access while the SBA was busy
518         if (sbbusy_i || sbcs.q.sbbusyserror) begin
519             sbcs_d.sbbusyserror = 1'b1;
520             resp_queue_inp.resp = dm::DTM_BUSY;
521         end else begin
522             sbaddr_d[31:0] = dmi_req_i.data;
523             sbaddress_write_valid_o = (sbcs.q.sbberror == '0);
524         end
525     end
526     dm::SBAddress1: begin
527         // access while the SBA was busy
528         if (sbbusy_i || sbcs.q.sbbusyserror) begin
529             sbcs_d.sbbusyserror = 1'b1;
530             resp_queue_inp.resp = dm::DTM_BUSY;
531         end else begin
532             sbaddr_d[63:32] = dmi_req_i.data;
533         end
534     end
535     dm::SBData0: begin
536         // access while the SBA was busy
537         if (sbbusy_i || sbcs.q.sbbusyserror) begin
538             sbcs_d.sbbusyserror = 1'b1;
539             resp_queue_inp.resp = dm::DTM_BUSY;
540         end else begin
541             sbdata_d[31:0] = dmi_req_i.data;
542             sbdata_write_valid_o = (sbcs.q.sbberror == '0);
543         end
544     end

```

```

545     dm::SBData1: begin
546         // access while the SBA was busy
547         if (sbbusy_i || sbcs.q.sbbusyserror) begin
548             sbcs_d.sbbusyserror = 1'b1;
549             resp_queue_inp.resp = dm::DTM_BUSY;
550         end else begin
551             sbdata_d[63:32] = dmi_req_i.data;
552         end
553     end
554     default::;
555 endcase
556 end
557 // hart threw a command error and has precedence over bus
writes
558 if (cmderror_valid_i) begin
559     cmderr_d = cmderror_i;
560 end
561 // set the havereset flag when the ndmreset completed
562 if (ndmreset_ack_i) begin
563     havereset_d = 1;
564 end
565 // -----
566 // System Bus
567 // -----
568 // set bus error
569 if (sberror_valid_i) begin
570     sbcs_d.sbberror = sberror_i;
571 end
572 // update read data
573 if (sbdata_valid_i) begin
574     sbdata_d = 64'(sbdata_i);
575 end
576 // dmcontrol
577 // TODO(zarubaf) we currently do not implement the har-
tarry mask
578 dmcontrol_d.hasel = 1'b0;
579 // we do not support resetting an individual hart
580 dmcontrol_d.hartreset = 1'b0;
581 dmcontrol_d.setresethaltreq = 1'b0;
582 dmcontrol_d.clresethaltreq = 1'b0;
583 dmcontrol_d.zero1 = '0;
584 dmcontrol_d.zero0 = '0;
585 // Non-writeable, clear only
586 dmcontrol_d.ackhavereset = 1'b0;
587 // resumereq is W1/read-zero and is represented by the
per-hart pending vector
588 dmcontrol_d.resumereq = 1'b0;
589 // Set a request only for an effective, selectable hart
that is halted when
590 // the write is accepted. A new request wins over a same-
cycle acknowledgement
591 if (dmcontrol_write_resumereq && dmcon-
trol_write_hart_valid &&
592     SelectableHarts[dmcontrol_write_hart] &&
593     halted_aligned[dmcontrol_write_hart] &&
594     !unavailable_aligned[dmcontrol_write_hart] &&
595     !dmcontrol_write_haltreq) begin
596         resume_pending_d[dmcontrol_write_hart] = 1'b1;
597     end
598 // Deactivating the debug module clears all pending ac-
tions
599 if (dmcontrol_write_accepted && !dmcontrol_write_dmacrive)
begin
600     resume_pending_d = '0;
601 end
602 // WARL behavior of hartsel, depending on NrHarts.
603 // If NrHarts = 1 this is just masked to all-zeros.
604 {dmcontrol_d.hartselhi, dmcontrol_d.hartsello} &= HartSel-
Mask;
605 // static values for dcsr
606 sbcs_d.sbverson = 3'd1;
607 sbcs_d.sbbusy = sbbusy_i;
608 sbcs_d.sbasize = $bits(sbcs_d.sbas-
ize)'(BusWidth);
609 sbcs_d.sbaccess128 = logic'(BusWidth >= 32'd128);
610 sbcs_d.sbaccess64 = logic'(BusWidth >= 32'd64);
611 sbcs_d.sbaccess32 = logic'(BusWidth >= 32'd32);
612 sbcs_d.sbaccess16 = logic'(BusWidth >= 32'd16);
613 sbcs_d.sbaccess8 = logic'(BusWidth >= 32'd8);
614 end
615 // Hart writes take precedence over DMI writes to the ab-
stract data registers.
616 assign data_d = data_valid_i ? data_i : data_d_dmi;
617 // output multiplexer
618 always_comb begin : p_outmux
619     // default assignment
620     haltreq_o = '0;
621     resumereq_o = resume_pending_q;
622     if (selected_hart_valid) begin
623         haltreq_o[selected_hart] = dmcontrol_q.haltreq;
624     end
625 end
626 assign dmacrive_o = dmcontrol_q.dmacrive;

```



```

634 assign cmd_o      = command_q;
635 assign cmd_valid_o = cmd_valid_q;
636 assign progbuf_o   = progbuf_q;
637 assign data_o      = data_q;
638
639 assign resp_queue_pop = dmi_resp_ready_i &
~resp_queue_empty;
640
641 assign ndmreset_o = dmcontrol_q.ndmreset;
642
643 // response FIFO
644 fifo_v3 #(
645     .dtype      ( logic [$bits(dmi_resp_o)-1:0] ),
646     .DEPTH      ( 2 )
647 ) i_fifo (
648     .clk_i,
649     .rst_ni,
650     .flush_i    ( ~dmi_rst_ni ), // Flush the
queue if the DTM is
651     // reset
652     .testmode_i ( testmode_i ),
653     .full_o      ( resp_queue_full ),
654     .empty_o     ( resp_queue_empty ),
655     .usage_o     ( ),
656     .data_i      ( resp_queue_inp ),
657     .push_i      ( resp_queue_push ),
658     .data_o      ( dmi_resp_o ),
659     .pop_i       ( resp_queue_pop )
660 );
661
662 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
663 // PoR
664 if (!rst_ni) begin
665     dmcontrol_q <= '0;
666     // this is the only write-able bit during reset
667     cmderr_q <= dm::CmdErrNone;
668     command_q <= '0;
669     cmd_valid_q <= '0;
670     abstractauto_q <= '0;
671     progbuf_q <= '0;
672     data_q <= '0;
673     sbcs_q <= '{default: '0, sbaccess: 3'd2};
674     sbaddr_q <= '0;
675     sbdata_q <= '0;
676     havereset_q <= '1;
677     resume_pending_q <= '0;
678 end else begin
679     havereset_q <= SelectableHarts & havereset_d;
680 // synchronous re-set of debug module, active-low, ex-
cept for dmactive
681 if (!dmcontrol_q.dmactive) begin
682     dmcontrol_q.haltreq <= '0;
683     dmcontrol_q.resumeeq <= '0;
684     dmcontrol_q.hartreset <= '0;
685     dmcontrol_q.ackhavereset <= '0;
686     dmcontrol_q.zero1 <= '0;
687     dmcontrol_q.hasel <= '0;
688     dmcontrol_q.hartsello <= '0;
689     dmcontrol_q.hartselhi <= '0;
690     dmcontrol_q.zero0 <= '0;
691     dmcontrol_q.setresethaltreq <= '0;
692     dmcontrol_q.clrresethaltreq <= '0;
693     dmcontrol_q.ndmreset <= '0;
694     resume_pending_q <= '0;
695     // this is the only write-able bit during reset
696     dmcontrol_q.dmactive <= dmcontrol_d.dmactive;
697     cmderr_q <= dm::CmdErrNone;
698     command_q <= '0;
699     cmd_valid_q <= '0;
700     abstractauto_q <= '0;
701     progbuf_q <= '0;
702     data_q <= '0;
703     sbcs_q <= '{default: '0, sbac-
cess: 3'd2};
704     sbaddr_q <= '0;
705     sbdata_q <= '0;
706 end else begin
707     dmcontrol_q <= dmcontrol_d;
708     resume_pending_q <= SelectableHarts & re-
sume_pending_d;
709     cmderr_q <= cmderr_d;
710     command_q <= command_d;
711     cmd_valid_q <= cmd_valid_d;
712     abstractauto_q <= abstractauto_d;
713     progbuf_q <= progbuf_d;
714     data_q <= data_d;
715     sbcs_q <= sbcs_d;
716     sbaddr_q <= sbaddr_d;
717     sbdata_q <= sbdata_d;
718 end
719 end
720 end
721 end
722 endmodule : dm_csrs

```

2. pulp-platform_riscv-dbg/src/dm_mem.sv (571 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
pad Hardware

```

```

3 * License, Version 0.51 (the "License"); you may not use this
file except in
4 * compliance with the License. You may obtain a copy of the
License at
5 * http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 * or agreed to in writing, software, hardware and materials
distributed under
7 * this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 * CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 * specific language governing permissions and limitations un-
der the License.
10 *
11 * File: dm_mem.sv
12 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
13 * Date: 11.7.2018
14 *
15 * Description: Memory module for execution-based debug clients
16 *
17 */
18
19 module dm_mem #(
20     parameter int unsigned NrHarts = 1,
21     parameter int unsigned BusWidth = 32,
22     parameter logic [NrHarts-1:0] SelectableHarts =
{NrHarts{1'b1}},
23     parameter int unsigned DmBaseAddress = '0,
24 // Maximum width supported by Access Register commands; must be
32 or 64.
25     parameter int unsigned MaxRegisterAccessWidth =
BusWidth
26 ) (
27     input logic clk_i, //
28     input logic rst_ni, // de-
bug module reset
29     input logic dmactive_i, // de-
bug module is active
30
31     output logic [NrHarts-1:0] debug_req_o,
32     input logic ndmreset_i,
33     input logic [19:0] hartsel_i,
34 // from Ctrl and Status register
35     input logic [NrHarts-1:0] haltreq_i,
36     input logic [NrHarts-1:0] resumereq_i,
37 // Clears stale acknowledgements before new resume requests
38     input logic [NrHarts-1:0] clear_resumeack_i,
39
40 // state bits
41     output logic [NrHarts-1:0] halted_o, //
hart acknowledge halt
42     output logic [NrHarts-1:0] resuming_o, //
hart is resuming
43
44     input logic [dm::ProgBufSize-1:0][31:0] progbuf_i, //
program buffer to expose
45
46     input logic [dm::DataCount-1:0][31:0] data_i, //
data in
47     output logic [dm::DataCount-1:0][31:0] data_o, //
data out
48     output logic data_valid_o, //
data out is valid
49 // abstract command interface
50     input logic cmd_valid_i,
51     input dm::command_t cmd_i,
52     output logic cmderror_valid_o,
53     output dm::cmderr_e cmderror_o,
54     output logic cmbusy_o,
55 // data interface
56
57 // SRAM interface
58     input logic req_i,
59     input logic we_i,
60     input logic [BusWidth-1:0] addr_i,
61     input logic [BusWidth-1:0] wdata_i,
62     input logic [BusWidth-1:0] be_i,
63     output logic [BusWidth-1:0] rdata_o
64 );
65 localparam int unsigned DbgAddressBits = 12;
66 localparam int unsigned DataIndexWidth = $clog2(
g2(dm::DataCount));
67 localparam int unsigned HartSellen = (NrHarts == 1) ?
1 : $clog2(NrHarts);
68 localparam int unsigned NrHartsAligned = 2**HartSellen;
69 localparam int unsigned MaxAar = (MaxRegisterAccess-
Width == 64) ? 4 : 3;
70 localparam bit HasSndScratch = (DmBaseAddress !=
0);
71 // Depending on whether we are at the zero page or not we
either use 'x0' or 'x10/a0'
72 localparam logic [4:0] LoadBaseAddr = (DmBaseAddress ==
0) ? 5'd0 : 5'd10;
73
74 localparam logic [DbgAddressBits-1:0] DataBaseAddr =
(dm::DataAddr);
75 localparam logic [DbgAddressBits-1:0] DataEndAddr =
(dm::DataAddr + 4*dm::DataCount - 1);

```



```

76 localparam logic [DbgAddressBits-1:0] ProgBufBaseAddr =
(dm::DataAddr - 4*dm::ProgBufSize);
77 localparam logic [DbgAddressBits-1:0] ProgBufEndAddr =
(dm::DataAddr - 1);
78 localparam logic [DbgAddressBits-1:0] AbstractCmdBaseAddr =
(ProgBufBaseAddr - 4*10);
79 localparam logic [DbgAddressBits-1:0] AbstractCmdEndAddr =
(ProgBufBaseAddr - 1);
80
81 localparam logic [DbgAddressBits-1:0] WhereToAddr = 'h300;
82 localparam logic [DbgAddressBits-1:0] FlagsBaseAddr = 'h400;
83 localparam logic [DbgAddressBits-1:0] FlagsEndAddr = 'h7FF;
84
85 localparam logic [DbgAddressBits-1:0] HaltedAddr = 'h100;
86 localparam logic [DbgAddressBits-1:0] GoingAddr = 'h108;
87 localparam logic [DbgAddressBits-1:0] ResumingAddr = 'h110;
88 localparam logic [DbgAddressBits-1:0] ExceptionAddr = 'h118;
89
90 initial begin : p_validate_max_register_access_width
91   assert (MaxRegisterAccessWidth == 32 || MaxRegisterAccess-
Width == 64) else
92     $fatal(1, "MaxRegisterAccessWidth must be 32 or 64.");
93   end
94
95   logic [dm::ProgBufSize/2-1:0][63:0] progbuf;
96   logic [7:0][63:0] abstract_cmd;
97   logic [NrHarts-1:0] halted_d, halted_q;
98   logic [NrHarts-1:0] resum_d, resum_q;
99   logic go, going;
100
101   logic exception;
102   logic unsupported_command;
103
104   logic [63:0] rom_rdata;
105   logic [63:0] rdata_d, rdata_q;
106   logic word_enable32_q;
107
108   logic [HartSellen-1:0] hartsel, wdata_hartsel;
109   logic [HartSellen-1:0] cmd_hartsel_d, cmd_hartsel_q;
110   logic wdata_hartsel_valid;
111   logic [DbgAddressBits-1:0] flags_hart_idx;
112   logic [HartSellen-1:0] flags_hart;
113
114   assign hartsel = hartsel_i[HartSellen-1:0];
115   assign wdata_hartsel = wdata_i[HartSellen-1:0];
116   assign wdata_hartsel_valid = wdata_i[31:0] <= 32'(NrHarts -
1);
117   assign flags_hart_idx = addr_i[DbgAddressBits-1:0] -
FlagsBaseAddr;
118   assign flags_hart = HartSellen'(flags_hart_idx);
119
120   logic [NrHartsAligned-1:0] resumereq_aligned, hal-
ted_q_aligned, halted_aligned;
121
122   assign resumereq_aligned = NrHarts-
Aligned'(resumereq_i);
123
124   assign halted_q_aligned = NrHartsAligned'(halted_q);
125
126   // distinguish whether we need to forward data from the ROM
or the FSM
127   // latch the address for this
128   logic fwd_rom_d, fwd_rom_q;
129   dm::ac_ar_cmd_t ac_ar;
130
131   // Abstract Command Access Register
132   assign ac_ar = dm::ac_ar_cmd_t'(cmd_i.control);
133   assign debug_req_o = halted_q;
134   assign halted_o = halted_q;
135   assign resum_o = resum_q;
136
137   // reshape progbuf
138   assign progbuf = progbuf_i;
139
140   typedef enum logic [1:0] { Idle, Go, CmdExecuting } state_e;
141   state_e state_d, state_q;
142
143   // hart ctrl queue
144   always_comb begin : p_hart_ctrl_queue
145     cmderror_valid_o = 1'b0;
146     cmderror_o = dm::CmdErrNone;
147     state_d = state_q;
148     cmd_hartsel_d = cmd_hartsel_q;
149     go = 1'b0;
150     cmdbusy_o = 1'b1;
151
152     unique case (state_q)
153       Idle: begin
154         cmdbusy_o = 1'b0;
155         if (cmd_valid_i && halted_q_aligned[hartsel] &&
!resumereq_aligned[hartsel] && !unsuppor-
ted_command) begin
156           // give the go signal
157           state_d = Go;
158           cmd_hartsel_d = hartsel;
159         end else if (cmd_valid_i) begin
160           // hart must be halted for all requests
161           cmderror_valid_o = 1'b1;
162           cmderror_o = dm::CmdErrorHaltResume;
163         end
164       end
165     endcase
166
167     Go: begin
168       // we are already busy here since we scheduled the ex-
ecution of a program
169       cmdbusy_o = 1'b1;
170       go = 1'b1;
171       // the thread is now executing the command, track its
state
172       if (going) begin
173         state_d = CmdExecuting;
174       end
175     end
176
177     CmdExecuting: begin
178       cmdbusy_o = 1'b1;
179       go = 1'b0;
180       // wait until the hart has halted again
181       if (halted_aligned[cmd_hartsel_q]) begin
182         state_d = Idle;
183       end
184     end
185
186     default: ;
187   endcase
188
189   // only signal once that cmd is unsupported so that we can
clear cmderr
190   // in subsequent writes to abstractcs
191   if (unsupported_command && cmd_valid_i) begin
192     cmderror_valid_o = 1'b1;
193     cmderror_o = dm::CmdErrNotSupported;
194   end
195
196   if (exception) begin
197     cmderror_valid_o = 1'b1;
198     cmderror_o = dm::CmdErrorException;
199   end
200
201   if (ndmreset_i) begin
202     // Clear state of hart and its control signals when it
is being reset.
203     state_d = Idle;
204     go = 1'b0;
205   end
206 end
207
208 // word mux for 32bit and 64bit buses
209 logic [63:0] word_mux;
210 assign word_mux = (fwd_rom_q) ? rom_rdata : rdata_q;
211
212 if (BusWidth == 64) begin : gen_word_mux64
213   assign rdata_o = word_mux;
214 end else begin : gen_word_mux32
215   assign rdata_o = (word_enable32_q) ? word_mux[32 +: 32] :
word_mux[0 +: 32];
216 end
217
218 // read/write logic
219 logic [dm::DataCount-1:0][31:0] data_bits;
220 logic [DataIndexWidth-1:0] data_reg_idx;
221 logic [7:0][7:0] rdata;
222
223 // Forward writes from the hart to the abstract data re-
gisters.
224 // VCS Xprop cannot instrument case-inside range items (see
PR #150).
225 always_comb (* xprop_off *) begin : p_data_write
226   data_bits = data_i;
227   data_reg_idx = DataIndexWidth'(
addr_i[DbgAddressBits-1:2] - DataBaseAd-
dr[DbgAddressBits-1:2]
228 );
229   data_valid_o = 1'b0;
230
231   if (req_i && we_i) begin
232     unique case (addr_i[DbgAddressBits-1:0]) inside
233       [DataBaseAddr:DataEndAddr]: begin
234         data_valid_o = 1'b1;
235         for (int unsigned byte_idx = 0; byte_idx <
$bits(be_i); byte_idx++) begin
236           if (be_i[byte_idx] &&
237               (data_reg_idx + byte_idx / 4) < dm::DataCount)
begin
238             data_bits[data_reg_idx + byte_idx / 4][(byte_idx %
4) * 8 +: 8] =
239               wdata_i[byte_idx * 8 +: 8];
240           end
241         end
242       end
243     endcase
244     default: ;
245   endcase
246 end
247
248 data_o = data_bits;
249 end
250
251 // Track resume acknowledgements with hart writes taking
priority over clear requests

```

```

252 always_comb begin : p_resuming
253     resuming_d = resuming_q;
254
255     resuming_d &= ~clear_resumeack_i;
256
257     if (req_i && we_i && addr_i[DbgAddressBits-1:0] == Re-
sumingAddr &&
258         wdata_hartsel_valid) begin
259         resuming_d[wdata_hartsel] = 1'b1;
260     end
261
262     if (!dmactive_i) begin
263         resuming_d = '0;
264     end
265 end
266
267 // VCS Xprop cannot instrument case-inside range items (see
PR #150).
268 always_comb (* xprop_off *) begin : p_rw_logic
269
270     halted_d = halted_q;
271     rdata_d = rdata_q;
272     rdata = '0;
273
274     exception = 1'b0;
275     halted_aligned = '0;
276     going = 1'b0;
277
278     // we've got a new request
279     if (req_i) begin
280         // this is a write
281         if (we_i) begin
282             unique case (addr_i[DbgAddressBits-1:0]) inside
283             HaltedAddr: begin
284                 if (wdata_hartsel_valid) begin
285                     halted_aligned[wdata_hartsel] = 1'b1;
286                     halted_d[wdata_hartsel] = 1'b1;
287                 end
288             end
289             GoingAddr: begin
290                 going = 1'b1;
291             end
292             ResumingAddr: begin
293                 // clear the halted flag as the hart resumed exe-
cution
294                 if (wdata_hartsel_valid) begin
295                     halted_d[wdata_hartsel] = 1'b0;
296                 end
297             end
298             // an exception occurred during execution
299             ExceptionAddr: exception = 1'b1;
300             default ;
301         endcase
302
303         // this is a read
304         end else begin
305             unique case (addr_i[DbgAddressBits-1:0]) inside
306             // variable ROM content
307             WhereToAddr: begin
308                 // variable jump to abstract cmd, program_buffer
309             or resume
310                 if (wdata_hartsel_valid && resume-
req_aligned[wdata_hartsel]) begin
311                     rdata_d = {32'b0, dm::jal('0, 21'(dm::ResumeAd-
dress[11:0])-21'(WhereToAddr))};
312                 end
313
314                 // there is a command active so jump there
315                 if (cmdbusy_o) begin
316                     // transfer not set is shortcut to the program
buffer if postexec is set
317                     // keep this statement narrow to not catch in-
valid commands
318                     if (cmd_i.cmdtype == dm::AccessRegister &&
!ac_ar.transfer && ac_ar.postexec) begin
319                         rdata_d = {32'b0, dm::jal('0, 21'(ProgBufBase-
Addr)-21'(WhereToAddr))};
320                     // this is a legit abstract cmd -> execute it
321                     end else begin
322                         rdata_d = {32'b0, dm::jal('0, 21'(AbstractCmd-
BaseAddr)-21'(WhereToAddr))};
323                     end
324                 end
325             end
326
327             [DataBaseAddr:DataEndAddr]: begin
328                 rdata_d = {
329                     data_i[DataIndexWidth'(
((addr_i[DbgAddressBits-1:3] -
DataBaseAddr[DbgAddressBits-1:3])
<< 1) + 1'b1)],
330                     data_i[DataIndexWidth'(
((addr_i[DbgAddressBits-1:3] -
DataBaseAddr[DbgAddressBits-1:3])
<< 1))];
331                 };
332             end
333
334             [ProgBufBaseAddr:ProgBufEndAddr]: begin
335                 rdata_d = prog-
buf[$clog2(dm::ProgBufSize)'(addr_i[DbgAddressBits-1:3] -

```

```

340 ProgBufBaseAd-
dr[DbgAddressBits-1:3]]];
341     end
342
343     // two slots for abstract command
344     [AbstractCmdBaseAddr:AbstractCmdEndAddr]: begin
345         // return the correct address index
346         rdata_d = ab-
stract_cmd[3'(addr_i[DbgAddressBits-1:3] -
347             AbstractCmdBaseAd-
dr[DbgAddressBits-1:3])];
348     end
349     // harts are polling for flags here
350     [FlagsBaseAddr:FlagsEndAddr]: begin
351         // Route resume by flag address and go to the act-
ive command hart
352         if (20'(flags_hart_idx) <= 20'(NrHarts - 1)) begin
353             rdata[addr_i[2:0]] = {
354                 6'b0,
355                 resumereq_aligned[flags_hart] && dmactive_i
&& !ndmreset_i,
356                 go && dmactive_i && flags_hart ==
cmd_hartsel_q
357             };
358         end
359         rdata_d = rdata;
360     end
361     default : ;
362 endcase
363 end
364 end
365
366 if (ndmreset_i) begin
367     // When harts are reset, they are not halted.
368     halted_d = '0;
369 end
370 end
371
372 always_comb begin : p_abstract_cmd_rom
373     // this abstract command is currently unsupported
374     unsupported_command = 1'b0;
375     // default memory
376     // if ac_ar.transfer is not set then we can take a short-
cut to the program buffer
377     abstract_cmd[0][31:0] = dm::illegal();
378     // load debug module base address into a0, this is shared
among all commands
379     abstract_cmd[0][63:32] = HasSndScratch ? dm::auipc(5'd10,
'0) : dm::nop();
380     // clr lowest 12b -> DM base offset
381     abstract_cmd[1][31:0] = HasSndScratch ? dm::srli(5'd10,
5'd10, 6'd12) : dm::nop();
382     abstract_cmd[1][63:32] = HasSndScratch ? dm::slli(5'd10,
5'd10, 6'd12) : dm::nop();
383     abstract_cmd[2][31:0] = dm::nop();
384     abstract_cmd[2][63:32] = dm::nop();
385     abstract_cmd[3][31:0] = dm::nop();
386     abstract_cmd[3][63:32] = dm::nop();
387     abstract_cmd[4][31:0] = HasSndScratch ?
dm::csrr(dm::CSR_DSCRATCH1, 5'd10) : dm::nop();
388     abstract_cmd[4][63:32] = dm::ebreak();
389     abstract_cmd[7:5] = '0;
390
391     // this depends on the command being executed
392     unique case (cmd_i.cmdtype)
393     // -----
394     // Access Register
395     // -----
396     dm::AccessRegister: begin
397         if (32'(ac_ar.aarsize) < MaxAar && ac_ar.transfer &&
ac_ar.write) begin
398             // store a0 in dscratch1
399             abstract_cmd[0][31:0] = HasSndScratch ?
dm::csrw(dm::CSR_DSCRATCH1, 5'd10) : dm::nop();
400             // this range is reserved
401             if (ac_ar.regno[15:14] != '0) begin
402                 abstract_cmd[0][31:0] = dm::ebreak(); // we leave
asap
403                 unsupported_command = 1'b1;
404                 // A0 access needs to be handled separately, as we
use A0 to load
405
406                 // the DM address offset need to access DSCRATCH1 in
this case
407                 end else if (HasSndScratch && ac_ar.regno[12] && (!
ac_ar.regno[5]) &&
408                     (ac_ar.regno[4:0] == 5'd10)) begin
409                     // store s0 in dscratch
410                     abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
411                     // load from data register
412                     abstract_cmd[2][63:32] = dm::load(ac_ar.aarsize,
5'd8, LoadBaseAddr, dm::DataAddr);
413                     // and store it in the corresponding CSR
414                     abstract_cmd[3][31:0] = dm::csrw(dm::CSR_D-
SCRATCH1, 5'd8);
415                     // restore s0 again from dscratch
416                     abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
417                     // GPR/FPR access
418                     end else if (ac_ar.regno[12]) begin

```

```

418 // determine whether we want to access the float-
ing point register or not
419 if (ac_ar.regno[5]) begin
420     abstract_cmd[2][31:0] =
421         dm::float_load(ac_ar.aarsize,
ac_ar.regno[4:0], LoadBaseAddr, dm::DataAddr);
422 end else begin
423     abstract_cmd[2][31:0] =
424         dm::load(ac_ar.aarsize, ac_ar.regno[4:0],
LoadBaseAddr, dm::DataAddr);
425 end
426 // CSR access
427 end else begin
428     // data register to CSR
429     // store s0 in dscratch
430     abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
431     // load from data register
432     abstract_cmd[2][63:32] = dm::load(ac_ar.aarsize,
5'd8, LoadBaseAddr, dm::DataAddr);
433     // and store it in the corresponding CSR
434     abstract_cmd[3][31:0] =
dm::csrw(dm::csr_reg_t'(ac_ar.regno[11:0]), 5'd8);
435     // restore s0 again from dscratch
436     abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
437 end
438 end else if (32'(ac_ar.aarsize) < MaxAar &&
ac_ar.transfer && !ac_ar.write) begin
439     // store a0 in dscratch1
440     abstract_cmd[0][31:0] = HasSndScratch ?

441         dm::csrw(dm::CSR_DSCRATCH1,
LoadBaseAddr) :
442         dm::nop();
443     // this range is reserved
444     if (ac_ar.regno[15:14] != '0) begin
445         abstract_cmd[0][31:0] = dm::ebreak(); // we
leave asap
446         unsupported_command = 1'b1;
447     // A0 access needs to be handled separately, as we
use A0 to load

448     // the DM address offset need to access DSCRATCH1 in
this case
449     end else if (HasSndScratch && ac_ar.regno[12] && (!
ac_ar.regno[5]) &&
450         (ac_ar.regno[4:0] == 5'd10)) begin
451         // store s0 in dscratch
452         abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
453         // read value from CSR into s0
454         abstract_cmd[2][63:32] = dm::csrr(dm::CSR_D-
SCRATCH1, 5'd8);
455         // and store s0 into data section
456         abstract_cmd[3][31:0] = dm::store(ac_ar.aarsize,
5'd8, LoadBaseAddr, dm::DataAddr);
457         // restore s0 again from dscratch
458         abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
459         // GPR/FPR access
460     end else if (ac_ar.regno[12]) begin
461         // determine whether we want to access the float-
ing point register or not
462         if (ac_ar.regno[5]) begin
463             abstract_cmd[2][31:0] =
464                 dm::float_store(ac_ar.aarsize,
ac_ar.regno[4:0], LoadBaseAddr, dm::DataAddr);
465         end else begin
466             abstract_cmd[2][31:0] =
467                 dm::store(ac_ar.aarsize, ac_ar.regno[4:0],
LoadBaseAddr, dm::DataAddr);
468         end
469         // CSR access
470     end else begin
471         // CSR register to data
472         // store s0 in dscratch
473         abstract_cmd[2][31:0] = dm::csrw(dm::CSR_D-
SCRATCH0, 5'd8);
474         // read value from CSR into s0
475         abstract_cmd[2][63:32] =
dm::csrr(dm::csr_reg_t'(ac_ar.regno[11:0]), 5'd8);
476         // and store s0 into data section
477         abstract_cmd[3][31:0] = dm::store(ac_ar.aarsize,
5'd8, LoadBaseAddr, dm::DataAddr);
478         // restore s0 again from dscratch
479         abstract_cmd[3][63:32] = dm::csrr(dm::CSR_D-
SCRATCH0, 5'd8);
480     end
481     end else if (32'(ac_ar.aarsize) >= MaxAar ||
ac_ar.aarpostincrement == 1'b1) begin
482         // this should happend when e.g. ac_ar.aarsize >=
MaxAar
483         // Openocd will try to do an access with aarsize=64
bits
484         // first before falling back to 32 bits.
485         abstract_cmd[0][31:0] = dm::ebreak(); // we leave
asap
486         unsupported_command = 1'b1;
487     end
488

```

```

489 // Check whether we need to execute the program buf-
fer. When we
490 // get an unsupported command we really should abort
instead of
491 // still trying to execute the program buffer, makes
it easier
492 // for the debugger to recover
493 if (ac_ar.postexec && !unsupported_command) begin
494     // issue a nop, we will automatically run into the
program buffer
495     abstract_cmd[4][63:32] = dm::nop();
496 end
497 end
498 // not supported at the moment
499 // dm::QuickAccess;;
500 // dm::AccessMemory;;
501 default: begin
502     abstract_cmd[0][31:0] = dm::ebreak();
503     unsupported_command = 1'b1;
504 end
505 endcase
506 end
507
508 logic [63:0] rom_addr;
509 assign rom_addr = 64'(addr_i);
510
511 // Depending on whether the debug module is located
512 // at the zero page we can instantiate a simplified version
513 // which only requires one scratch register per hart.
514 // For all other cases we need to set aside
515 // two registers per hart, hence we also need
516 // two scratch registers.
517 if (HasSndScratch) begin : gen_rom_snd_scratch
518     debug_rom_i_debug_rom (
519         .clk_i,
520         .rst_ni,
521         .req_i,
522         .addr_i ( rom_addr ),
523         .rdata_o ( rom_rdata )
524     );
525 end else begin : gen_rom_one_scratch
526     // It uses the zero register ('x0') as the base
527     // for its loads. The zero register does not need to
528     // be saved.
529     debug_rom_one_scratch_i_debug_rom (
530         .clk_i,
531         .rst_ni,
532         .req_i,
533         .addr_i ( rom_addr ),
534         .rdata_o ( rom_rdata )
535     );
536 end
537
538 // ROM starts at the HaltAddress of the core e.g.: it imme-
diately jumps to
539 // the ROM base address
540 assign fwd_rom_d = logic'(addr_i[DbgAddressBits-1:0] >=
dm::HaltAddress[DbgAddressBits-1:0]);
541
542 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
543     if (!rst_ni) begin
544         cmd_hartsel_q <= '0;
545     end else begin
546         cmd_hartsel_q <= cmd_hartsel_d;
547     end
548     if (!rst_ni) begin
549         fwd_rom_q <= 1'b0;
550         rdata_q <= '0;
551         state_q <= Idle;
552         word_enable32_q <= 1'b0;
553     end else begin
554         fwd_rom_q <= fwd_rom_d;
555         rdata_q <= rdata_d;
556         state_q <= state_d;
557         word_enable32_q <= addr_i[2];
558     end
559 end
560
561 always_ff @(posedge clk_i or negedge rst_ni) begin
562     if (!rst_ni) begin
563         halted_q <= '0;
564         resuming_q <= '0;
565     end else begin
566         halted_q <= SelectableHarts & halted_d;
567         resuming_q <= SelectableHarts & resuming_d;
568     end
569 end
570
571 endmodule : dm_mem

```

3. pulp-platform_riscv-dbg/src/dm_obi_top.sv (196 lines)

```

1 // Copyright 2020 Silicon Labs, Inc.
2 //
3 // This file, and derivatives thereof are licensed under the
4 // Solderpad License, Version 2.0 (the "License").
5 //
6 // Use of this file means you agree to the terms and condi-
7 // tions of the license and are in full compliance with the License.

```

```

8 //
9 // You may obtain a copy of the License at:
10 //
11 //     https://solderpad.org/licenses/SHL-2.0/
12 //
13 // Unless required by applicable law or agreed to in writing,
14 // software and hardware implementations thereof distributed under the
15 // License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
16 // CONDITIONS OF ANY KIND, EITHER EXPRESSED OR IMPLIED.
17 //
18 // See the License for the specific language governing permis-
19 // sions and limitations under the License.
20 //
21 ///////////////////////////////////////////////////////////////////
22 // Engineer:      Arjan Bink - ar-
23 // jan.bink@silabs.com
24 //
25 // Design Name:   OBI Wrapper for Debug Module
26 // (dm_top)
27 // Project Name:  CV32E40P
28 // Language:      SystemVeri-
29 // log
30 //
31 // Description:   Wrapper for the Debug Module (dm_top) which
32 // gives it an //
33 // OBI (Open Bus Interface) compatible inter-
34 // faces so that it //
35 // can be integrated without further glue lo-
36 // gic (other than //
37 // tie offs in an OBI compliant sys-
38 // tem.
39 //
40 // This wrapper is only intended for OBI com-
41 // pliant systems; //
42 // in other systems the existing dm_top can be
43 // used as before //
44 // and this wrapper can be ig-
45 // nored.
46 //
47 // The OBI spec is available
48 // at:
49 //
50 // - https://github.com/openhwgroup/core-v-
51 // docs/blob/master/ //
52 // cores/
53 // cv32e40p/
54 //
55 // Compared to 'logint' interfaces of dm_top
56 // the following //
57 // signals are ad-
58 // ded:
59 //
60 // - slave_* OBI inter-
61 // face:
62 //
63 // -
64 // slave_gnt_o
65 // slave_rvalid_o
66 // slave_aid_i
67 // slave_rid_o
68 //
69 // Compared to 'logint' interfaces of dm_top
70 // the following //
71 // signals have been re-
72 // named:
73 //
74 // - master_* OBI inter-
75 // face:
76 //

```

```

57 // - Renamed master_add_o to mas-
58 // ter_addr_o
59 // - Renamed master_r_valid_i to mas-
60 // ter_rvalid_i
61 // - Renamed master_r_rdata_i to mas-
62 // ter_rdata_i
63 //
64 ///////////////////////////////////////////////////////////////////
65 module dm_obi_top #(
66     parameter int unsigned IdWidth = 1, //
67     Width of aid/rid
68     parameter int unsigned NrHarts = 1,
69     parameter int unsigned BusWidth = 32,
70     parameter int unsigned DmBaseAddress = 'h1000, //
71     default to non-zero page
72     // Bitmask to select physically available harts for systems
73     // that don't use hart numbers in a contiguous fashion.
74     parameter logic [NrHarts-1:0] SelectableHarts =
75     {NrHarts{1'b1}},
76     71 // Maximum width supported by Access Register commands; must be
77     32 or 64.
78     parameter int unsigned MaxRegisterAccessWidth =
79     BusWidth
80 ) (
81     input logic clk_i, // clock
82     // asynchronous reset active low, connect PoR here, not the
83     system reset
84     input logic rst_ni,
85     input logic testmode_i,
86     output logic ndmreset_o, // non-debug
87     module reset
88     output logic dmactive_o, // debug mod-
89     ule is active
90     output logic [NrHarts-1:0] debug_req_o, // async de-
91     bug request
92     // communicate whether the hart is unavailable (e.g.: power
93     down)
94     input logic [NrHarts-1:0] unavailable_i,
95     input dm::hartinfo_t [NrHarts-1:0] hartinfo_i,
96     input logic slave_req_i,
97     // OBI grant for slave_req_i (not present on dm_top)
98     output logic slave_gnt_o,
99     input logic slave_we_i,
100    input logic [BusWidth-1:0] slave_addr_i,
101    input logic [BusWidth/8-1:0] slave_be_i,
102    input logic [BusWidth-1:0] slave_wdata_i,
103    // Address phase transaction identifier (not present on
104    dm_top)
105    input logic [IdWidth-1:0] slave_aid_i,
106    // OBI rvalid signal (end of response phase for reads/
107    writes) (not present on dm_top)
108    output logic slave_rvalid_o,
109    output logic [BusWidth-1:0] slave_rdata_o,
110    // Response phase transaction identifier (not present on
111    dm_top)
112    output logic [IdWidth-1:0] slave_rid_o,
113    output logic master_req_o,
114    output logic [BusWidth-1:0] master_addr_o, // Renamed
115    according to OBI spec
116    output logic master_we_o,
117    output logic [BusWidth-1:0] master_wdata_o,
118    output logic [BusWidth/8-1:0] master_be_o,
119    input logic master_gnt_i,
120    input logic master_rvalid_i, // Renamed
121    according to OBI spec
122    input logic master_err_i,
123    input logic master_other_err_i, // *oth-
124    er_err_i has priority over *err_i
125    input logic [BusWidth-1:0] master_rdata_i, // Renamed
126    according to OBI spec
127    //
128    // Connection to DTM - compatible to RocketChip Debug Module
129    input logic dmi_rst_ni,
130    input logic dmi_req_valid_i,
131    output logic dmi_req_ready_o,
132    input dm::dmi_req_t dmi_req_i,
133    output logic dmi_resp_valid_o,
134    input logic dmi_resp_ready_i,
135    output dm::dmi_resp_t dmi_resp_o
136 );
137 // Slave response phase (rvalid and identifier)
138 logic slave_rvalid_q;
139 logic [IdWidth-1:0] slave_rid_q;
140 // dm_top instance
141 dm_top #(
142     .NrHarts (NrHarts),
143     .BusWidth (BusWidth),
144     .DmBaseAddress (DmBaseAddress),

```

```

131 .SelectableHarts      ( SelectableHarts      ),
132 .MaxRegisterAccessWidth ( MaxRegisterAccessWidth ),
133 ) i_dm_top (
134     .clk_i             ( clk_i             ),
135     .rst_ni            ( rst_ni            ),
136     .next_dm_addr_i    ( '0' ), //
Note exposed at the top yet.
137     .testmode_i        ( testmode_i        ),
138     .ndmreset_o         ( ndmreset_o         ),
139     .ndmreset_ack_i     ( ndmreset_o         ), //
This is currently not exposed yet.
140     .dmactive_o         ( dmactive_o         ),
141     .debug_req_o        ( debug_req_o        ),
142     .unavailable_i      ( unavailable_i      ),
143     .hartinfo_i         ( hartinfo_i         ),
144
145     .slave_req_i        ( slave_req_i        ),
146     .slave_we_i         ( slave_we_i         ),
147     .slave_addr_i       ( slave_addr_i       ),
148     .slave_be_i         ( slave_be_i         ),
149     .slave_wdata_i      ( slave_wdata_i      ),
150     .slave_rdata_o      ( slave_rdata_o      ),
151
152     .master_req_o       ( master_req_o       ),
153     .master_add_o       ( master_addr_o      ), // Re-
named according to OBI spec
154     .master_we_o        ( master_we_o        ),
155     .master_wdata_o     ( master_wdata_o     ),
156     .master_be_o        ( master_be_o        ),
157     .master_gnt_i       ( master_gnt_i       ),
158     .master_rvalid_i    ( master_rvalid_i    ), // Re-
named according to OBI spec
159     .master_r_err_i     ( master_err_i       ),
160     .master_r_other_err_i ( master_other_err_i ), //
*other_err_i has priority over *err_i
161     .master_r_rdata_i    ( master_rdata_i    ), // Re-
named according to OBI spec
162
163     .dmi_rst_ni         ( dmi_rst_ni         ),
164     .dmi_req_valid_i     ( dmi_req_valid_i     ),
165     .dmi_req_ready_o     ( dmi_req_ready_o     ),
166     .dmi_req_i          ( dmi_req_i          ),
167
168     .dmi_resp_valid_o    ( dmi_resp_valid_o    ),
169     .dmi_resp_ready_i    ( dmi_resp_ready_i    ),
170     .dmi_resp_o         ( dmi_resp_o         ),
171 );
172
173 // Extension to wrap dm_top as an OBI-compliant module
174 //
175 // dm_top has an implied rvalid pulse the cycle after its
granted request.
176
177 // Registers
178 always_ff @(posedge clk_i or negedge rst_ni) begin :
obi_regs
179     if (!rst_ni) begin
180         slave_rvalid_q <= 'b0;
181         slave_rid_q    <= 'b0;
182     end else begin
183         if (slave_req_i && slave_gnt_o) begin // 1 cycle pulse
on rvalid for every granted request
184             slave_rvalid_q <= 'b1;
185             slave_rid_q    <= slave_aid_i; // Mirror aid to
rid
186         end else begin
187             slave_rvalid_q <= 'b0; // rid is don't
care if rvalid = 0
188         end
189     end
190 end
191
192 assign slave_gnt_o = 'b1; // Always recept-
ive to request (slave_req_i)
193 assign slave_rvalid_o = slave_rvalid_q;
194 assign slave_rid_o = slave_rid_q;
195
196 endmodule : dm_obi_top

```

4. pulp-platform_riscv-dbg/src/dm_pkg.sv (451 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
pad Hardware
3 * License, Version 0.51 (the "License"); you may not use this
file except in
4 * compliance with the License. You may obtain a copy of the
License at
5 * http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 * or agreed to in writing, software, hardware and materials
distributed under
7 * this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 * CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 * specific language governing permissions and limitations un-
der the License.
10 *

```

```

11 * File: dm_pkg.sv
12 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
13 * Date: 30.6.2018
14 *
15 * Description: Debug-module package, contains common system
definitions.
16 *
17 */
18
19 package dm;
20     localparam logic [3:0] DbgVersion013 = 4'h2;
21     // size of program buffer in junks of 32-bit words
22     localparam logic [4:0] ProgBufSize = 5'h8;
23
24     // amount of data count registers implemented
25     localparam logic [3:0] DataCount = 4'h2;
26
27 // address to which a hart should jump when it was requested to
halt
28     localparam logic [63:0] HaltAddress = 64'h800;
29     localparam logic [63:0] ResumeAddress = HaltAddress + 8;
30     localparam logic [63:0] ExceptionAddress = HaltAddress + 16;
31
32 // address where data0-15 is shadowed or if shadowed in a
CSR
33 // address of the first CSR used for shadowing the data
34     localparam logic [11:0] DataAddr = 12'h380; // we are
aligned with Rocket here
35
36 // debug registers
37 typedef enum logic [7:0] {
38     Data0 = 8'h04,
39     Data1 = 8'h05,
40     Data2 = 8'h06,
41     Data3 = 8'h07,
42     Data4 = 8'h08,
43     Data5 = 8'h09,
44     Data6 = 8'h0A,
45     Data7 = 8'h0B,
46     Data8 = 8'h0C,
47     Data9 = 8'h0D,
48     Data10 = 8'h0E,
49     Data11 = 8'h0F,
50     DMControl = 8'h10,
51     DMStatus = 8'h11, // r/o
52     Hartinfo = 8'h12,
53     HaltSum1 = 8'h13,
54     HAWindowSel = 8'h14,
55     HAWindow = 8'h15,
56     AbstractCS = 8'h16,
57     Command = 8'h17,
58     AbstractAuto = 8'h18,
59     DevTreeAddr0 = 8'h19,
60     DevTreeAddr1 = 8'h1A,
61     DevTreeAddr2 = 8'h1B,
62     DevTreeAddr3 = 8'h1C,
63     NextDM = 8'h1D,
64     ProgBuf0 = 8'h20,
65     ProgBuf1 = 8'h21,
66     ProgBuf2 = 8'h22,
67     ProgBuf3 = 8'h23,
68     ProgBuf4 = 8'h24,
69     ProgBuf5 = 8'h25,
70     ProgBuf6 = 8'h26,
71     ProgBuf7 = 8'h27,
72     ProgBuf8 = 8'h28,
73     ProgBuf9 = 8'h29,
74     ProgBuf10 = 8'h2A,
75     ProgBuf11 = 8'h2B,
76     ProgBuf12 = 8'h2C,
77     ProgBuf13 = 8'h2D,
78     ProgBuf14 = 8'h2E,
79     ProgBuf15 = 8'h2F,
80     AuthData = 8'h30,
81     HaltSum2 = 8'h34,
82     HaltSum3 = 8'h35,
83     SBAddress3 = 8'h37,
84     SBCS = 8'h38,
85     SBAddress0 = 8'h39,
86     SBAddress1 = 8'h3A,
87     SBAddress2 = 8'h3B,
88     SBData0 = 8'h3C,
89     SBData1 = 8'h3D,
90     SBData2 = 8'h3E,
91     SBData3 = 8'h3F,
92     HaltSum0 = 8'h40
93 } dm_csr_e;
94
95 // debug causes
96     localparam logic [2:0] CauseBreakpoint = 3'h1;
97     localparam logic [2:0] CauseTrigger = 3'h2;
98     localparam logic [2:0] CauseRequest = 3'h3;
99     localparam logic [2:0] CauseSingleStep = 3'h4;
100
101 typedef struct packed {
102     logic [31:23] zero1;
103     logic impebreak;
104     logic [21:20] zero0;
105     logic allhavereset;
106     logic anyhavereset;

```



```

107 logic      allresumeack;
108 logic      anyresumeack;
109 logic      allnonexistent;
110 logic      anynonexistent;
111 logic      allunavail;
112 logic      anyunavail;
113 logic      allrunning;
114 logic      anyrunning;
115 logic      allhalted;
116 logic      anyhalted;
117 logic      authenticated;
118 logic      authbusy;
119 logic      hasresethaltreq;
120 logic      devtreerevalid;
121 logic [3:0] version;
122 } dmstatus_t;
123
124 typedef struct packed {
125     logic      haltreq;
126     logic      resumereq;
127     logic      hartreset;
128     logic      ackhavereset;
129     logic      zero1;
130     logic      hasel;
131     logic [25:16] hartsello;
132     logic [15:6] hartselhi;
133     logic [5:4] zero0;
134     logic      setresethaltreq;
135     logic      clrresethaltreq;
136     logic      ndmreset;
137     logic      dmactive;
138 } dmcontrol_t;
139
140 typedef struct packed {
141     logic [31:24] zero1;
142     logic [23:20] nscratch;
143     logic [19:17] zero0;
144     logic      dataaccess;
145     logic [15:12] datasize;
146     logic [11:0] dataaddr;
147 } hartinfo_t;
148
149 typedef enum logic [2:0] {
150     CmdErrNone, CmdErrBusy, CmdErrNotSupported,
151     CmdErrorException, CmdErrorHaltResume,
152     CmdErrorBus, CmdErrorOther = 7
153 } cmderr_e;
154
155 typedef struct packed {
156     logic [31:29] zero3;
157     logic [28:24] progbufsize;
158     logic [23:13] zero2;
159     logic      busy;
160     logic      zero1;
161     cmderr_e   cmderr;
162     logic [7:4] zero0;
163     logic [3:0] datacount;
164 } abstractcs_t;
165
166 typedef enum logic [7:0] {
167     AccessRegister = 8'h0,
168     QuickAccess    = 8'h1,
169     AccessMemory   = 8'h2
170 } cmd_e;
171
172 typedef struct packed {
173     cmd_e   cmdtype;
174     logic [23:0] control;
175 } command_t;
176
177 typedef struct packed {
178     logic [31:16] autoexecprogbuf;
179     logic [15:12] zero0;
180     logic [11:0] autoexecdata;
181 } abstractauto_t;
182
183 typedef struct packed {
184     logic      zero1;
185     logic [22:20] aarsize;
186     logic      aarpostincrement;
187     logic      postexec;
188     logic      transfer;
189     logic      write;
190     logic [15:0] regno;
191 } ac_ar_cmd_t;
192
193 // DTM
194 typedef enum logic [1:0] {
195     DTM_NOP = 2'h0,
196     DTM_READ = 2'h1,
197     DTM_WRITE = 2'h2
198 } dtm_op_e;
199
200 typedef enum logic [1:0] {
201     DTM_SUCCESS = 2'h0,
202     DTM_ERR = 2'h2,
203     DTM_BUSY = 2'h3
204 } dtm_op_status_e;
205
206 typedef struct packed {
207     logic [31:29] sbversion;
208     logic [28:23] zero0;
209     logic      sbbusyerror;
210     logic      sbbusy;
211     logic      sbreadonaddr;
212     logic [19:17] sbaccess;
213     logic      sbautoincrement;
214     logic      sbreadondata;
215     logic [14:12] sberror;
216     logic [11:5] sbasize;
217     logic      sbaccess128;
218     logic      sbaccess64;
219     logic      sbaccess32;
220     logic      sbaccess16;
221     logic      sbaccess8;
222 } sbcs_t;
223
224 typedef struct packed {
225     logic [6:0] addr;
226     dtm_op_e   op;
227     logic [31:0] data;
228 } dmi_req_t;
229
230 typedef struct packed {
231     logic [31:0] data;
232     logic [1:0] resp;
233 } dmi_resp_t;
234
235 typedef struct packed {
236     logic [31:18] zero1;
237     logic      dmihardreset;
238     logic      dmireset;
239     logic      zero0;
240     logic [14:12] idle;
241     logic [11:10] dmistat;
242     logic [9:4] abits;
243     logic [3:0] version;
244 } dtmcs_t;
245
246 // privilege levels
247 typedef enum logic [1:0] {
248     PRIV_LVL_M = 2'b11,
249     PRIV_LVL_S = 2'b01,
250     PRIV_LVL_U = 2'b00
251 } priv_lvl_t;
252
253 // debugregs in core
254 typedef struct packed {
255     logic [31:28] xdebugver;
256     logic [27:16] zero2;
257     logic      ebreakm;
258     logic      zero1;
259     logic      ebreaks;
260     logic      ebreaku;
261     logic      stepie;
262     logic      stopcount;
263     logic      stoptime;
264     logic [8:6] cause;
265     logic      zero0;
266     logic      mprven;
267     logic      nmip;
268     logic      step;
269     priv_lvl_t priv;
270 } dcsr_t;
271
272 // CSRs
273 typedef enum logic [11:0] {
274     // Floating-Point CSRs
275     CSR_FFLAGS = 12'h001,
276     CSR_FRM = 12'h002,
277     CSR_FCSR = 12'h003,
278     CSR_FTRAN = 12'h800,
279     // Supervisor Mode CSRs
280     CSR_SSTATUS = 12'h100,
281     CSR_SIE = 12'h104,
282     CSR_STVEC = 12'h105,
283     CSR_SCOUNTEREN = 12'h106,
284     CSR_SSCRATCH = 12'h140,
285     CSR_SEPC = 12'h141,
286     CSR_SCAUSE = 12'h142,
287     CSR_STVAL = 12'h143,
288     CSR_SIP = 12'h144,
289     CSR_SATP = 12'h180,
290     // Machine Mode CSRs
291     CSR_MSTATUS = 12'h300,
292     CSR_MISA = 12'h301,
293     CSR_MEDELEG = 12'h302,
294     CSR_MIDELEG = 12'h303,
295     CSR_MIE = 12'h304,
296     CSR_MTVEC = 12'h305,
297     CSR_MCOUNTEREN = 12'h306,
298     CSR_MSCRATCH = 12'h340,
299     CSR_MEPC = 12'h341,
300     CSR_MCAUSE = 12'h342,
301     CSR_MTVAL = 12'h343,
302     CSR_MIP = 12'h344,
303     CSR_PMPCFG0 = 12'h3A0,
304     CSR_PMPADDR0 = 12'h3B0,
305     CSR_MVENDORID = 12'hF11,
306     CSR_MARCHID = 12'hF12,
307     CSR_MIMPID = 12'hF13,
308     CSR_MHARTID = 12'hF14,

```

```

309 CSR_MCYCLE      = 12'hb00,
310 CSR_MINSTRET    = 12'hb02,
311 CSR_DCACHE      = 12'h701,
312 CSR_ICACHE      = 12'h700,
313
314 CSR_TSELECT      = 12'h7A0,
315 CSR_TDATA1       = 12'h7A1,
316 CSR_TDATA2       = 12'h7A2,
317 CSR_TDATA3       = 12'h7A3,
318 CSR_TINFO        = 12'h7A4,
319
320 // Debug CSR
321 CSR_DCSR          = 12'h7b0,
322 CSR_DPC           = 12'h7b1,
323 CSR_DSCRATCH0     = 12'h7b2, // optional
324 CSR_DSCRATCH1     = 12'h7b3, // optional
325
326 // Counters and Timers
327 CSR_CYCLE         = 12'hc00,
328 CSR_TIME          = 12'hc01,
329 CSR_INSTRET       = 12'hc02
330 } csr_reg_t;
331
332 // SBA state
333 typedef enum logic [2:0] {
334     Idle,
335     Read,
336     Write,
337     WaitRead,
338     WaitWrite
339 } sba_state_e;
340
341 // Instruction Generation Helpers
342 function automatic logic [31:0] jal (logic [4:0] rd,
343                                     logic [20:0] imm);
344     // OpCode Jal
345     return {imm[20], imm[10:1], imm[11], imm[19:12], rd,
346             7'h6f};
347 endfunction
348
349 function automatic logic [31:0] jalr (logic [4:0] rd,
350                                     logic [4:0] rs1,
351                                     logic [11:0] offset);
352     // OpCode Jalr
353     return {offset[11:0], rs1, 3'b0, rd, 7'h67};
354 endfunction
355
356 function automatic logic [31:0] andi (logic [4:0] rd,
357                                     logic [4:0] rs1,
358                                     logic [11:0] imm);
359     // OpCode andi
360     return {imm[11:0], rs1, 3'h7, rd, 7'h13};
361 endfunction
362
363 function automatic logic [31:0] slli (logic [4:0] rd,
364                                     logic [4:0] rs1,
365                                     logic [5:0] shamt);
366     // OpCode slli
367     return {6'b0, shamt[5:0], rs1, 3'h1, rd, 7'h13};
368 endfunction
369
370 function automatic logic [31:0] srli (logic [4:0] rd,
371                                     logic [4:0] rs1,
372                                     logic [5:0] shamt);
373     // OpCode srli
374     return {6'b0, shamt[5:0], rs1, 3'h5, rd, 7'h13};
375 endfunction
376
377 function automatic logic [31:0] load (logic [2:0] size,
378                                     logic [4:0] dest,
379                                     logic [4:0] base,
380                                     logic [11:0] offset);
381     // OpCode Load
382     return {offset[11:0], base, size, dest, 7'h03};
383 endfunction
384
385 function automatic logic [31:0] auipc (logic [4:0] rd,
386                                     logic [20:0] imm);
387     // OpCode Auipc
388     return {imm[20], imm[10:1], imm[11], imm[19:12], rd,
389             7'h17};
390 endfunction
391
392 function automatic logic [31:0] store (logic [2:0] size,
393                                     logic [4:0] src,
394                                     logic [4:0] base,
395                                     logic [11:0] offset);
396     // OpCode Store
397     return {offset[11:5], src, base, size, offset[4:0],
398             7'h23};
399 endfunction
400
401 function automatic logic [31:0] float_load (logic [2:0]
402 size,
403                                     logic [4:0]
404 dest,
405                                     logic [4:0]
406 base,
407                                     logic [11:0]
408 offset);
409     // OpCode Load

```

```

403     return {offset[11:0], base, size, dest, 7'b00_001_11};
404 endfunction
405
406 function automatic logic [31:0] float_store (logic [2:0]
407 size,
408                                     logic [4:0]
409 src,
410                                     logic [4:0]
411 base,
412                                     logic [11:0]
413 offset);
414     // OpCode Store
415     return {offset[11:5], src, base, size, offset[4:0],
416             7'b01_001_11};
417 endfunction
418
419 function automatic logic [31:0] csrwr (csr_reg_t csr,
420                                     logic [4:0] rs1);
421     // CSRRW, rd, OpCode System
422     return {csr, rs1, 3'h1, 5'h0, 7'h73};
423 endfunction
424
425 function automatic logic [31:0] csrr (csr_reg_t csr,
426                                     logic [4:0] dest);
427     // rs1, CSRRS, rd, OpCode System
428     return {csr, 5'h0, 3'h2, dest, 7'h73};
429 endfunction
430
431 function automatic logic [31:0] branch (logic [4:0] src2,
432                                     logic [4:0] src1,
433                                     logic [2:0] funct3,
434                                     logic [11:0] offset);
435     // OpCode Branch
436     return {offset[11], offset[9:4], src2, src1, funct3,
437             offset[3:0], offset[10], 7'b11_000_11};
438 endfunction
439
440 function automatic logic [31:0] ebreak ();
441     return 32'h00100073;
442 endfunction
443
444 function automatic logic [31:0] wfi ();
445     return 32'h10500073;
446 endfunction
447
448 function automatic logic [31:0] nop ();
449     return 32'h00000013;
450 endfunction
451
452 function automatic logic [31:0] illegal ();
453     return 32'h00000000;
454 endfunction
455
456 endpackage : dm

```

5. pulp-platform_riscv-dbg/src/dm_sba.sv (211 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
3 pad Hardware
4 * License, Version 0.51 (the "License"); you may not use this
5 file except in
6 * compliance with the License. You may obtain a copy of the
7 License at
8 * http://solderpad.org/licenses/SHL-0.51. Unless required by
9 applicable law
10 * or agreed to in writing, software, hardware and materials
11 distributed under
12 * this License is distributed on an "AS IS" BASIS, WITHOUT
13 WARRANTIES OR
14 * CONDITIONS OF ANY KIND, either express or implied. See the
15 License for the
16 * specific language governing permissions and limitations under
17 the License.
18 */
19 * File: dm_sba.sv
20 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
21 * Date: 1.8.2018
22 *
23 * Description: System Bus Access Module
24 */
25
26 module dm_sba #(
27     parameter int unsigned BusWidth = 32,
28     parameter bit ReadByteEnable = 1
29 ) (
30     input logic clk_i, // Clock
31     input logic rst_ni,
32     input logic dmactive_i, // synchronous
33     reset active low
34
35     output logic master_req_o,
36     output logic [BusWidth-1:0] master_add_o,
37     output logic master_we_o,
38     output logic [BusWidth-1:0] master_wdata_o,
39     output logic [BusWidth/8-1:0] master_be_o,
40     input logic master_gnt_i,
41     input logic master_r_valid_i,
42     input logic master_r_err_i,
43     input logic master_r_other_err_i, //

```



```

*other_err_i has priority over *err_i
35 input logic [BusWidth-1:0] master_r_rdata_i,
36
37 input logic [BusWidth-1:0] sbaddress_i,
38 input logic sbaddress_write_valid_i,
39 // control signals in
40 input logic sbreadonaddr_i,
41 output logic [BusWidth-1:0] sbaddress_o,
42 input logic sbautoincrement_i,
43 input logic [2:0] sbaccess_i,
44 // data in
45 input logic sbreadondata_i,
46 input logic [BusWidth-1:0] sbdata_i,
47 input logic sbdata_read_valid_i,
48 input logic sbdata_write_valid_i,
49 // read data out
50 output logic [BusWidth-1:0] sbdata_o,
51 output logic sbdata_valid_o,
52 // control signals
53 output logic sbbusy_o,

54 output logic sberror_valid_o, // bus error
occurred
55 output logic [2:0] sberror_o // bus error oc-
curred
56 );
57
58 localparam int BeIdxWidth = $clog2(BusWidth/8);
59 dm::sba_state_e state_d, state_q;
60
61 logic [BusWidth-1:0] address;
62 logic req;
63 logic gnt;
64 logic we;
65 logic [BusWidth/8-1:0] be;
66 logic [BusWidth/8-1:0] be_mask;
67 logic [BeIdxWidth-1:0] be_idx;
68
69 assign sbbusy_o = logic'(state_q != dm::Idle);
70
71 always_comb begin : p_be_mask
72     be_mask = '0;
73
74     // generate byte enable mask
75     unique case (sbaccess_i)
76         3'b000: begin
77             be_mask[be_idx] = '1;
78         end
79         3'b001: begin
80             be_mask[int'({be_idx[$high(be_idx):1], 1'b0}) +: 2] =
'1;
81         end
82         3'b010: begin
83             if (BusWidth == 32'd64)
84                 be_mask[int'({be_idx[$high(be_idx)], 2'h0}) +: 4] = '1;
85             else
86                 be_mask = '1;
87         end
88         default: ;
89     endcase
90 end
91
92 logic [BusWidth-1:0] sbaccess_mask;
93 assign sbaccess_mask = {BusWidth{1'b1}} << sbaccess_i;
94
95 logic addr_incr_en;
96 logic [BusWidth-1:0] addr_incr;
97 assign addr_incr = (addr_incr_en) ? (BusWidth{1'b1} <<
sbaccess_i) : '0;
98 assign sbaddress_o = sbaddress_i + addr_incr;
99
100 always_comb begin : p_fsm
101     req = 1'b0;
102     address = sbaddress_i;
103     we = 1'b0;
104     be = '0;
105     be_idx = sbaddress_i[BeIdxWidth-1:0];
106
107     sberror_o = '0;
108     sberror_valid_o = 1'b0;
109
110     addr_incr_en = 1'b0;
111
112     state_d = state_q;
113
114     unique case (state_q)
115         dm::Idle: begin
116             // debugger requested a read
117             if (sbaddress_write_valid_i && sbreadonaddr_i)
118                 state_d = dm::Read;
119             // debugger requested a write
120             if (sbdata_write_valid_i) state_d = dm::Write;
121             // perform another read
122             if (sbdata_read_valid_i && sbreadondata_i) state_d =
dm::Read;
123         end
124         dm::Read: begin
125             req = 1'b1;
126             if (ReadByteEnable) be = be_mask;

```

```

127         if (gnt) state_d = dm::WaitRead;
128     end
129
130     dm::Write: begin
131         req = 1'b1;
132         we = 1'b1;
133         be = be_mask;
134         if (gnt) state_d = dm::WaitWrite;
135     end
136
137     dm::WaitRead: begin
138         if (sbdata_valid_o) begin
139             state_d = dm::Idle;
140             // auto-increment address
141             addr_incr_en = sbautoincrement_i;
142             // check whether an "other" error has been en-
countered.
143             if (master_r_other_err_i) begin
144                 sberror_valid_o = 1'b1;
145                 sberror_o = 3'd7;
146                 // check whether there was a bus error (== bad ad-
dress).
147             end else if (master_r_err_i) begin
148                 sberror_valid_o = 1'b1;
149                 sberror_o = 3'd2;
150             end
151         end
152     end
153
154     dm::WaitWrite: begin
155         if (sbdata_valid_o) begin
156             state_d = dm::Idle;
157             // auto-increment address
158             addr_incr_en = sbautoincrement_i;
159             // check whether an "other" error has been en-
countered.
160             if (master_r_other_err_i) begin
161                 sberror_valid_o = 1'b1;
162                 sberror_o = 3'd7;
163                 // check whether there was a bus error (== bad ad-
dress).
164             end else if (master_r_err_i) begin
165                 sberror_valid_o = 1'b1;
166                 sberror_o = 3'd2;
167             end
168         end
169     end
170
171     default: state_d = dm::Idle; // catch parasitic state
endcase
172
173 // handle error case
174 if (32'(sbaccess_i) > BeIdxWidth && state_q != dm::Idle)
begin
175     req = 1'b0;
176     state_d = dm::Idle;
177     sberror_valid_o = 1'b1;
178     sberror_o = 3'd4; // unsupported size was reques-
ted
179 end
180
181 //if sbaccess_i lsb of address is not 0 - report mis-
alignment error
182 if (!(sbaddress_i & ~sbaccess_mask) && state_q !=
dm::Idle) begin
183     req = 1'b0;
184     state_d = dm::Idle;
185     sberror_valid_o = 1'b1;
186     sberror_o = 3'd3; // alignment error
187 end
188 // further error handling should go here ...
189 end
190
191 always_ff @(posedge clk_i or negedge rst_ni) begin : p_regs
192     if (!rst_ni) begin
193         state_q <= dm::Idle;
194     end else begin
195         state_q <= state_d;
196     end
197 end
198
199 logic [BeIdxWidth-1:0] be_idx_masked;
200 assign be_idx_masked = be_idx & BeIdx-
width'(sbaccess_mask);
201 assign master_req_o = req;
202 assign master_add_o = address[BusWidth-1:0];
203 assign master_we_o = we;
204 assign master_wdata_o = sbdata_i[BusWidth-1:0] << (8 *
be_idx_masked);
205 assign master_be_o = be[BusWidth/8-1:0];
206 assign gnt = master_gnt_i;
207 assign sbdata_valid_o = master_r_valid_i;
208 assign sbdata_o = master_r_rdata_i[BusWidth-1:0] >>
(8 * be_idx_masked);
209
210 endmodule : dm_sba

```

6. pulp-platform_riscv-dbg/src/dm_top.sv (243 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
3 * pad Hardware
4 * License, Version 0.51 (the "License"); you may not use this
5 * file except in
6 * compliance with the License. You may obtain a copy of the
7 * License at
8 * http://solderpad.org/licenses/SHL-0.51. Unless required by
9 * applicable law
10 * or agreed to in writing, software, hardware and materials
11 * distributed under
12 * this License is distributed on an "AS IS" BASIS, WITHOUT
13 * WARRANTIES OR
14 * CONDITIONS OF ANY KIND, either express or implied. See the
15 * License for the
16 * specific language governing permissions and limitations under
17 * the License.
18 *
19 * File: dm_top.sv
20 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
21 * Date: 30.6.2018
22 *
23 * Description: Top-level of debug module (DM). This is an AXI-
24 * Slave.
25
26 * DTM protocol is equal to SiFives debug protocol to
27 * leverage
28 * SW infrastructure re-use. As of version 0.13
29 */
30
31 module dm_top #(
32     parameter int unsigned      NrHarts      = 1,
33     parameter int unsigned      BusWidth     = 32,
34     parameter int unsigned      DmBaseAddress = 'h1000, //
35     default to non-zero page
36     // Bitmask to select physically available harts for systems
37     // that don't use hart numbers in a contiguous fashion.
38     parameter logic [NrHarts-1:0] SelectableHarts =
39     {NrHarts[1'b1]},
40     // toggle new behavior to drive master_be_o during a read
41     parameter bit               ReadByteEnable = 1,
42
43     // Maximum width supported by Access Register commands; must be
44     // 32 or 64.
45     parameter int unsigned      MaxRegisterAccessWidth =
46     BusWidth
47 ) (
48     input logic                 clk_i,           // clock
49     // asynchronous reset active low, connect PoR here, not the
50     system reset
51     input logic                 rst_ni,
52     // Subsequent debug modules can be chained by setting the
53     nextdm register value to the offset of
54     // the next debug module. The RISC-V debug spec mandates
55     that the first debug module located at
56
57     // 0x0, and that the last debug module in the chain sets the
58     nextdm register to 0x0. The nextdm
59     // register is a word address and not a byte address. This
60     value is passed in as a static signal
61     // so that it becomes possible to assign this value with
62     chiplet tie-offs or straps, if needed.
63     input logic [31:0]          next_dm_addr_i,
64     input logic                 testmode_i,
65     output logic                ndmreset_o, // non-debug mod-
66     ule reset
67     input logic                 ndmreset_ack_i, // non-debug
68     module reset acknowledgement pulse
69     output logic                dmactive_o, // debug module
70     is active
71     output logic [NrHarts-1:0]  debug_req_o, // async debug
72     request
73     // communicate whether the hart is unavailable (e.g.: power
74     down)
75     input logic [NrHarts-1:0]  unavailable_i,
76     input dm::hartinfo_t [NrHarts-1:0] hartinfo_i,
77
78     input logic                 slave_req_i,
79     input logic                 slave_we_i,
80     input logic [BusWidth-1:0] slave_addr_i,
81     input logic [BusWidth/8-1:0] slave_be_i,
82     input logic [BusWidth-1:0] slave_wdata_i,
83     output logic [BusWidth-1:0] slave_rdata_o,
84
85     output logic                master_req_o,
86     output logic [BusWidth-1:0] master_add_o,
87     output logic                master_we_o,
88     output logic [BusWidth-1:0] master_wdata_o,
89     output logic [BusWidth/8-1:0] master_be_o,
90     input logic                master_gnt_i,
91     input logic                master_r_valid_i,
92     input logic                master_r_err_i, // *oth-
93     er_err_i has priority over *err_i
94     input logic [BusWidth-1:0] master_r_rdata_i,
95
96     // Connection to DTM - compatible to RocketChip Debug Module

```

```

69     input logic                 dmi_rst_ni, // Synchronous
70     clear request from
71
72     // the DTM to
73     clear the DMI response
74
75     // FIFO.
76     input logic                 dmi_req_valid_i,
77     output logic                dmi_req_ready_o,
78     input dm::dmi_req_t         dmi_req_i,
79
80     output logic                dmi_resp_valid_o,
81     input logic                 dmi_resp_ready_i,
82     output dm::dmi_resp_t       dmi_resp_o
83 );
84
85 // Debug CSRs
86 logic [NrHarts-1:0]          halted;
87 // logic [NrHarts-1:0]
88 logic [NrHarts-1:0]          running;
89 logic [NrHarts-1:0]          resumeack;
90 logic [NrHarts-1:0]          haltreq;
91 logic [NrHarts-1:0]          resumereq;
92 logic [NrHarts-1:0]          clear_resumeack;
93 logic                         cmd_valid;
94 dm::command_t                 cmd;
95
96 cmderror_valid;
97 dm::cmderr_e                 cmderror;
98 logic                         cmdbusy;
99 logic [dm::ProgBufSize-1:0][31:0] progbuf;
100 logic [dm::DataCount-1:0][31:0] data_csrs_mem;
101 logic [dm::DataCount-1:0][31:0] data_mem_csrs;
102 logic                         data_valid;
103 logic                         ndmreset;
104 logic [19:0]                  hartsel;
105
106 // System Bus Access Module
107 logic [BusWidth-1:0]          sbaddress_csrs_sba;
108 logic [BusWidth-1:0]          sbaddress_sba_csrs;
109 logic                         sbaddress_write_valid;
110 logic                         sbreadonaddr;
111 logic                         sbautoincrement;
112 logic                         sbaccess;
113 logic                         sbreadondata;
114 logic                         sbdata_write;
115 logic                         sbdata_read_valid;
116 logic                         sbdata_write_valid;
117 logic                         sbdata_read;
118 logic                         sbdata_valid;
119 logic                         sbbusy;
120 logic                         sberror_valid;
121 logic [2:0]                   sberror;
122
123 assign ndmreset_o = ndmreset;
124
125 dm_csrs #(
126     .NrHarts(NrHarts),
127     .BusWidth(BusWidth),
128     .SelectableHarts(SelectableHarts)
129 ) i_dm_csrs (
130     .clk_i,
131     .rst_ni,
132     .next_dm_addr_i,
133     .testmode_i,
134     .dmi_rst_ni,
135     .dmi_req_valid_i,
136     .dmi_req_ready_o,
137     .dmi_req_i,
138     .dmi_resp_valid_o,
139     .dmi_resp_ready_i,
140     .dmi_resp_o,
141     .ndmreset_o,
142     .ndmreset_ack_i,
143     .dmactive_o,
144     .hartsel_o,
145     .hartinfo_i,
146     .halted_i,
147     .unavailable_i,
148     .resumeack_i,
149     .haltreq_o,
150     .resumereq_o,
151     .clear_resumeack_o,
152     .cmd_valid_o,
153     .cmd_o,
154     .cmderror_valid_i,
155     .cmderror_i,
156     .cmdbusy_i,
157     .progbuf_o,
158     .data_i,
159     .data_valid_i,
160     .data_o,
161     .sbaddress_o,
162     .sbaddress_i,
163     .sbaddress_write_valid_o,
164     .sbreadonaddr_o,
165     .sbautoincrement_o,
166     .sbaccess_o,
167     .sbreadondata_o,
168     .sbdata_o,
169     .sbdata_read_valid_o,
170     .sbdata_write_valid_o,
171     .sbdata_i,
172     .sbdata_valid_i,
173     .sbbusy_i,
174     ( ndmreset ),
175     ( ndmreset_ack_i ),
176     ( hartsel ),
177     ( halted ),
178     ( resumeack ),
179     ( haltreq ),
180     ( resumereq ),
181     ( clear_resumeack ),
182     ( cmd_valid ),
183     ( cmd ),
184     ( cmderror_valid ),
185     ( cmderror ),
186     ( cmdbusy ),
187     ( progbuf ),
188     ( data_mem_csrs ),
189     ( data_valid ),
190     ( data_csrs_mem ),
191     ( sbaddress_csrs_sba ),
192     ( sbaddress_sba_csrs ),
193     ( sbaddress_write_valid ),
194     ( sbreadonaddr ),
195     ( sbautoincrement ),
196     ( sbaccess ),
197     ( sbreadondata ),
198     ( sbdata_write ),
199     ( sbdata_read_valid ),
200     ( sbdata_write_valid ),
201     ( sbdata_read ),
202     ( sbdata_valid ),
203     ( sbbusy ),

```

```

168 .sberror_valid_i      ( sberror_valid      ),
169 .sberror_i            ( sberror            ),
170 );
171
172 dm_sba #(
173   .BusWidth(BusWidth),
174   .ReadByteEnable(ReadByteEnable)
175 ) i_dm_sba (
176   .clk_i,
177   .rst_ni,
178   .dmactive_i          ( dmactive_o          ),
179
180   .master_req_o,
181   .master_add_o,
182   .master_we_o,
183   .master_wdata_o,
184   .master_be_o,
185   .master_gnt_i,
186   .master_r_valid_i,
187   .master_r_err_i,
188   .master_r_other_err_i,
189   .master_r_rdata_i,
190
191   .sbaddress_i         ( sbaddress_csrs_sba ),
192   .sbaddress_o         ( sbaddress_sba_csrs ),
193   .sbaddress_write_valid_i ( sbaddress_write_valid ),
194   .sbreadonaddr_i      ( sbreadonaddr      ),
195   .sbautoincrement_i   ( sbautoincrement   ),
196   .sbaccess_i          ( sbaccess          ),
197   .sbreadondata_i      ( sbreadondata      ),
198   .sbdata_i            ( sbdata_write      ),
199   .sbdata_read_valid_i ( sbdata_read_valid ),
200   .sbdata_write_valid_i ( sbdata_write_valid ),
201   .sbdata_o            ( sbdata_read      ),
202   .sbdata_valid_o      ( sbdata_valid      ),
203   .sbbusy_o            ( sbbusy           ),
204   .sberror_valid_o     ( sberror_valid     ),
205   .sberror_o           ( sberror          ),
206 );
207
208 dm_mem #(
209   .NrHarts              ( NrHarts          ),
210   .BusWidth             ( BusWidth         ),
211   .SelectableHarts      ( SelectableHarts   ),
212   .DmBaseAddress        ( DmBaseAddress    ),
213   .MaxRegisterAccessWidth ( MaxRegisterAccessWidth ),
214 ) i_dm_mem (
215   .clk_i,
216   .rst_ni,
217   .dmactive_i           ( dmactive_o       ),
218   .debug_req_o,
219   .ndmreset_i          ( ndmreset         ),
220   .hartsel_i           ( hartsel          ),
221   .haltreq_i           ( haltreq          ),
222   .resumereq_i         ( resumereq        ),
223   .clear_resumeack_i   ( clear_resumeack   ),
224   .halted_o            ( halted           ),
225   .resuming_o          ( resumeack        ),
226   .cmd_valid_i         ( cmd_valid         ),
227   .cmd_i               ( cmd              ),
228   .cmderror_valid_o    ( cmderror_valid    ),
229   .cmderror_o          ( cmderror         ),
230   .cmdbusy_o           ( cmdbusy          ),
231   .progbuf_i           ( progbuf          ),
232   .data_i              ( data_csrs_mem     ),
233   .data_o              ( data_mem_csrs     ),
234   .data_valid_o        ( data_valid        ),
235   .req_i               ( slave_req_i      ),
236   .we_i                ( slave_we_i       ),
237   .addr_i              ( slave_addr_i     ),
238   .wdata_i             ( slave_wdata_i    ),
239   .be_i                ( slave_be_i       ),
240   .rdata_o             ( slave_rdata_o    ),
241 );
242
243 endmodule : dm_top

```

7. pulp-platform_riscv-dbg/src/dmi_bscane_tap.sv (80 lines)

```

1 // Copyright 2020 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
3 // details.
4 // SPDX-License-Identifier: SHL-0.51
5
6 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
7
8 /// Replacement for the full JTAG tap with `BSCANE2` Xilinx
9 /// elements which hook
10 /// into the FPGA native scan chain. Meant for FPGA boards
11 /// which do not expose a
12 /// usable pin-header or a separate, programmable FTDI chip.
13
14 /// They replace the functionality of `dmi_jtag_tap.sv`. The
15 /// file is
16 /// pin-compatible so that by selecting the appropriate file
17 /// for the target it
18 /// can be transparently managed without relying on tick
19 /// defines.
20
21 module dmi_jtag_tap #(
22   // Ignored, defined by the FPGA model.

```

```

16   parameter int unsigned IrLength = 5,
17   // JTAG IDCODE Value
18   parameter logic [31:0] IdcodeValue = 32'h00000001
19   // xxxx version
20   // xxxxxxxxxxxxxxxx part number
21   // xxxxxxxxxxxx manufacturer id
22   // 1 required by standard
23 ) (
24   /// Unused. Here to maintain pin compatibility with
25   `dmi_jtag_tap` so that it
26   /// can be used as a drop-in replacement.
27   input logic tck_i,
28   input logic tms_i,
29   input logic trst_ni,
30   input logic tdi_i,
31   output logic tdo_o,
32   output logic tdo_oe_o,
33   input logic testmode_i,
34
35   output logic tck_o,
36   output logic dmi_clear_o,
37   output logic update_o,
38   output logic capture_o,
39   output logic shift_o,
40   output logic tdi_o,
41   output logic dtmcs_select_o,
42   input logic dtmcs_tdo_i,
43   // we want to access DMI register
44   output logic dmi_select_o,
45   input logic dmi_tdo_i,
46 );
47
48 BSCANE2 #(
49   .JTAG_CHAIN (3)
50 ) i_tap_dtmcs (
51   .CAPTURE (capture_o),
52   .DRCK (),
53   .RESET (dmi_clear_o),
54   .RUNTEST (),
55   .SEL (dtmcs_select_o),
56   .SHIFT (shift_o),
57   .TCK (tck_o),
58   .TDI (tdi_o),
59   .TMS (),
60   .TDO (dtmcs_tdo_i),
61   .UPDATE (update_o)
62 );
63
64 /// DMI Register
65 BSCANE2 #(
66   .JTAG_CHAIN (4)
67 ) i_tap_dmi (
68   .CAPTURE (),
69   .DRCK (),
70   .RESET (),
71   .RUNTEST (),
72   .SEL (dmi_select_o),
73   .SHIFT (),
74   .TCK (),
75   .TDI (),
76   .TMS (),
77   .TDO (dmi_tdo_i),
78   .UPDATE ()
79 );
80 endmodule

```

8. pulp-platform_riscv-dbg/src/dmi_cdc.sv (114 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
3 * pad Hardware
4 * License, Version 0.51 (the "License"); you may not use this
5 * file except in
6 * compliance with the License. You may obtain a copy of the
7 * License at
8 * http://solderpad.org/licenses/SHL-0.51. Unless required by
9 * applicable law
10 * or agreed to in writing, software, hardware and materials
11 * distributed under
12 * this License is distributed on an "AS IS" BASIS, WITHOUT
13 * WARRANTIES OR
14 * CONDITIONS OF ANY KIND, either express or implied. See the
15 * License for the
16 * specific language governing permissions and limitations un-
17 * der the License.
18
19 * File: axi_riscv_debug_module.sv
20 * Author: Andreas Traber <atraber@iis.ee.ethz.ch>
21 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
22
23 * Description: Clock domain crossings for JTAG to DMI very
24 * heavily based
25
26 * on previous work by Andreas Traber for the PULP
27 * project.
28
29 * This is mainly a wrapper around the existing
30 * CDCs.
31
32 */
33 module dmi_cdc (

```

```

20 // JTAG side (master side)
21 input logic tck_i,
22 input logic trst_ni,
23 input dm::dmi_req_t jtag_dmi_req_i,
24 output logic jtag_dmi_ready_o,
25 input logic jtag_dmi_valid_i,
26 input logic jtag_dmi_cdc_clear_i, // Synchron-
ous clear signal.
27 // Triggers
reset sequencing
28 // accross
CDC
29
30 output dm::dmi_resp_t jtag_dmi_resp_o,
31 output logic jtag_dmi_valid_o,
32 input logic jtag_dmi_ready_i,
33
34 // core side (slave side)
35 input logic clk_i,
36 input logic rst_ni,
37
38 output logic core_dmi_rst_no,
39 output dm::dmi_req_t core_dmi_req_o,
40 output logic core_dmi_valid_o,
41 input logic core_dmi_ready_i,
42
43 input dm::dmi_resp_t core_dmi_resp_i,
44 output logic core_dmi_ready_o,
45 input logic core_dmi_valid_i
46 );
47
48 logic core_clear_pending;
49
50 cdc_2phase_clearable #(T(dm::dmi_req_t)) i_cdc_req (
51 .src_rst_ni ( trst_ni ),
52 .src_clear_i ( jtag_dmi_cdc_clear_i ),
53 .src_clk_i ( tck_i ),
54 .src_clear_pending_o(), // Not used
55 .src_data_i ( jtag_dmi_req_i ),
56 .src_valid_i ( jtag_dmi_valid_i ),
57 .src_ready_o ( jtag_dmi_ready_o ),
58
59 .dst_rst_ni ( rst_ni ),
60 .dst_clear_i ( 1'b0 ), // No functional
reset from core side
61 // used (only
async).
62 .dst_clear_pending_o( core_clear_pending ), // use the
clear pending signal
63 // to syn-
chronously clear the
64 // response
FIFO in the dm_top
65 // csrs
66 .dst_clk_i ( clk_i ),
67 .dst_data_o ( core_dmi_req_o ),
68 .dst_valid_o ( core_dmi_valid_o ),
69 .dst_ready_i ( core_dmi_ready_i )
70 );
71
72 cdc_2phase_clearable #(T(dm::dmi_resp_t)) i_cdc_resp (
73 .src_rst_ni ( rst_ni ),
74 .src_clear_i ( 1'b0 ), // No functional
reset from core side
75 // used (only
async).
76 .src_clear_pending_o(), // Not used
77 .src_clk_i ( clk_i ),
78 .src_data_i ( core_dmi_resp_i ),
79 .src_valid_i ( core_dmi_valid_i ),
80 .src_ready_o ( core_dmi_ready_o ),
81
82 .dst_rst_ni ( trst_ni ),
83 .dst_clear_i ( jtag_dmi_cdc_clear_i ),
84 .dst_clear_pending_o(), //Not used
85 .dst_clk_i ( tck_i ),
86 .dst_data_o ( jtag_dmi_resp_o ),
87 .dst_valid_o ( jtag_dmi_valid_o ),
88 .dst_ready_i ( jtag_dmi_ready_i )
89 );
90
91 // We need to flush the DMI response FIFO in DM top using
the core clock
92 // synchronous clear signal core_dmi_rst_no. We repurpose
the clear
93 // pending signal in the core clock domain by generating a 1
cycle pulse from
94 // it.
95
96 logic core_clear_pending_q;
97 logic core_dmi_rst_nq;
98 logic clear_pending_rise_edge_detect;
99
100 assign clear_pending_rise_edge_detect = !
core_clear_pending_q && core_clear_pending;
101
102 always_ff @(posedge clk_i, negedge rst_ni) begin
103 if (!rst_ni) begin
104 core_dmi_rst_nq <= 1'b1;
105 core_clear_pending_q <= 1'b0;

```

```

106 end else begin
107 core_dmi_rst_nq <=
~clear_pending_rise_edge_detect; // active-low!
108 core_clear_pending_q <= core_clear_pending;
109 end
110 end
111
112 assign core_dmi_rst_no = core_dmi_rst_nq;
113
114 endmodule : dmi_cdc

```

9. pulp-platform_riscv-dbg/src/dmi_intf.sv (59 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
details.
3 // SPDX-License-Identifier: SHL-0.51
4
5 // Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
6
7
8 // The DV interface additionally carries a clock signal.
9 interface DMI_BUS_DV #(
10 /// The width of the address.
11 parameter int ADDR_WIDTH = -1
12 ) (
13 input logic clk_i
14 );
15
16 import dm::*;
17
18 typedef logic [ADDR_WIDTH-1:0] addr_t;
19 typedef logic [31:0] data_t;
20 /// The request channel (Q).
21 addr_t q_addr;
22 dtm_op_e q_op;
23 data_t q_data;
24 logic q_valid;
25 logic q_ready;
26
27 /// The response channel (P).
28 data_t p_data;
29 logic [1:0] p_resp;
30 logic p_valid;
31 logic p_ready;
32
33 modport in (
34 input q_addr, q_op, q_data, q_valid, p_ready,
35 output q_ready, p_data, p_resp, p_valid
36 );
37 modport out (
38 output q_addr, q_op, q_data, q_valid, p_ready,
39 input q_ready, p_data, p_resp, p_valid
40 );
41 modport monitor (
42 input q_addr, q_op, q_data, q_valid, p_ready,
43 q_ready, p_data, p_resp, p_valid
44 );
45
46 // pragma translate_off
47 `ifndef VERILATOR
48 assert property (@(posedge clk_i) (q_valid && !q_ready |>
$stable(q_addr)));
49 assert property (@(posedge clk_i) (q_valid && !q_ready |>
$stable(q_op)));
50 assert property (@(posedge clk_i) (q_valid && !q_ready |>
$stable(q_data)));
51 assert property (@(posedge clk_i) (q_valid && !q_ready |>
q_valid));
52
53 assert property (@(posedge clk_i) (p_valid && !p_ready |>
$stable(p_data)));
54 assert property (@(posedge clk_i) (p_valid && !p_ready |>
$stable(p_resp)));
55 assert property (@(posedge clk_i) (p_valid && !p_ready |>
p_valid));
56 `endif
57 // pragma translate_on
58
59 endinterface

```

10. pulp-platform_riscv-dbg/src/dmi_jtag.sv (353 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
pad Hardware
3 * License, Version 0.51 (the "License"); you may not use this
file except in
4 * compliance with the License. You may obtain a copy of the
License at
5 * http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 * or agreed to in writing, software, hardware and materials
distributed under
7 * this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 * CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 * specific language governing permissions and limitations un-
der the License.

```

```

10 *
11 * File: axi_riscv_debug_module.sv
12 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
13 * Date: 19.7.2018
14 *
15 * Description: JTAG DMI (debug module interface)
16 *
17 */
18
19 module dmi_jtag #(
20     parameter logic [31:0] IdcodeValue = 32'h00000DB3
21 ) (
22     input logic clk_i, // DMI Clock
23     input logic rst_ni, // Asynchronous reset active low
24     input logic testmode_i,
25
26     // active-low glitch free reset signal. Is asserted for one
27     // dmi clock cycle
28     // (clk_i) whenever the dmi_jtag is reset (POR or functional
29     // reset).
30     output logic dmi_rst_no,
31     output logic dmi_req_o,
32     output logic dmi_req_valid_o,
33     input logic dmi_req_ready_i,
34
35     input logic dmi_resp_i,
36     output logic dmi_resp_ready_o,
37     input logic dmi_resp_valid_i,
38
39     input logic tck_i, // JTAG test clock pad
40     input logic tms_i, // JTAG test mode select pad
41     input logic trst_ni, // JTAG test reset pad
42     input logic tdi_i, // JTAG test data input pad
43     output logic tdo_o, // JTAG test data output pad
44     output logic tdo_oe_o // Data out output enable
45 );
46
47 typedef enum logic [1:0] {
48     DMInoError = 2'h0, DMIReservedError = 2'h1,
49     DMIOPFailed = 2'h2, DMIBusy = 2'h3
50 } dmi_error_e;
51 dmi_error_e error_d, error_q;
52
53 logic tck;
54 logic jtag_dmi_clear; // Synchronous reset of DMI triggered
55 by TestLogicReset in
56 // jtag TAP
57 logic dmi_clear; // Functional (warm) reset of the entire
58 DMI
59
60 logic update;
61 logic capture;
62 logic shift;
63 logic tdi;
64
65 logic dtmcs_select;
66
67 dtmcs_t dtmcs_d, dtmcs_q;
68
69 assign dmi_clear = jtag_dmi_clear || (dtmcs_select && update &&
70 dtmcs_q.dmihardreset);
71
72 // -----
73 // Debug Module Control and Status
74 // -----
75
76 always_comb begin
77     dtmcs_d = dtmcs_q;
78     if (capture) begin
79         if (dtmcs_select) begin
80             dtmcs_d = '{
81                 zero1 : '0,
82                 dmihardreset : 1'b0,
83                 dmireset : 1'b0,
84                 zero0 : '0,
85                 idle : 3'd1, // 1: Enter Run-
86                 Test/Idle and leave it immediately
87                 dmistat : error_q, // 0: No error,
88                 2: Op failed, 3: too fast
89                 abits : 6'd7, // The size of ad-
90                 dress in dmi
91                 version : 4'd1 // Version de-
92                 scribed in spec version 0.13 (and later?)
93             };
94         end
95     end
96
97     if (shift) begin
98         if (dtmcs_select) dtmcs_d = {tdi, 31'(dtmcs_q >> 1)};
99     end
100
101     always_ff @(posedge tck or negedge trst_ni) begin
102         if (!trst_ni) begin
103             dtmcs_q <= '0;
104         end else begin
105             dtmcs_q <= dtmcs_d;
106         end
107     end
108 end

```

```

99
100 // -----
101 // DMI (Debug Module Interface)
102 // -----
103
104 logic dmi_select;
105 logic dmi_tdo;
106
107 dm::dmi_req_t dmi_req;
108 logic dmi_req_ready;
109 logic dmi_req_valid;
110
111 dm::dmi_resp_t dmi_resp;
112 logic dmi_resp_valid;
113 logic dmi_resp_ready;
114
115 typedef struct packed {
116     logic [6:0] address;
117     logic [31:0] data;
118     logic [1:0] op;
119 } dmi_t;
120
121 typedef enum logic [2:0] { Idle, Read, WaitReadValid, Write,
122 WaitWriteValid } state_e;
123 state_e state_d, state_q;
124
125 logic [$bits(dmi_t)-1:0] dr_d, dr_q;
126 logic [6:0] address_d, address_q;
127 logic [31:0] data_d, data_q;
128
129 dmi_t dmi;
130 assign dmi = dmi_t'(dr_q);
131 assign dmi_req.addr = address_q;
132 assign dmi_req.data = data_q;
133 assign dmi_req.op = (state_q == Write) ? dm::DTM_WRITE :
134 dm::DTM_READ;
135
136 // We will always be ready to accept the data we requested.
137 assign dmi_resp_ready = 1'b1;
138
139 logic error_dmi_busy;
140 logic error_dmi_op_failed;
141
142 always_comb begin : p_fsm
143     error_dmi_busy = 1'b0;
144     error_dmi_op_failed = 1'b0;
145     // default assignments
146     state_d = state_q;
147     address_d = address_q;
148     data_d = data_q;
149     error_d = error_q;
150
151     dmi_req_valid = 1'b0;
152
153     if (dmi_clear) begin
154         state_d = Idle;
155         data_d = '0;
156         error_d = DMInoError;
157         address_d = '0;
158     end else begin
159         unique case (state_q)
160             Idle: begin
161                 // make sure that no error is sticky
162                 if (dmi_select && update && (error_q == DMInoError))
163                     begin
164                         // save address and value
165                         address_d = dmi.address;
166                         data_d = dmi.data;
167                         if (dm::dtm_op_e'(dmi.op) == dm::DTM_READ) begin
168                             state_d = Read;
169                         end else if (dm::dtm_op_e'(dmi.op) ==
170 dm::DTM_WRITE) begin
171                             state_d = Write;
172                         end
173                         // else this is a nop and we can stay here
174                     end
175                     end
176
177     Read: begin
178         dmi_req_valid = 1'b1;
179         if (dmi_req_ready) begin
180             state_d = WaitReadValid;
181         end
182     end
183
184     WaitReadValid: begin
185         // load data into register and shift out
186         if (dmi_resp_valid) begin
187             unique case (dmi_resp.resp)
188                 dm::DTM_SUCCESS: begin
189                     data_d = dmi_resp.data;
190                 end
191                 dm::DTM_ERR: begin
192                     data_d = 32'hDEAD_BEEF;
193                     error_dmi_op_failed = 1'b1;
194                 end
195                 dm::DTM_BUSY: begin
196                     data_d = 32'hB051_B051;
197                     error_dmi_busy = 1'b1;
198                 end
199             end
200         end
201     end
202 end

```



```

194         default: begin
195             data_d = 32'hBAAD_CODE;
196         end
197     endcase
198     state_d = Idle;
199 end
200 end
201
202 Write: begin
203     dmi_req_valid = 1'b1;
204
205 // request sent, wait for response before going back
206 to idle
207     if (dmi_req_ready) begin
208         state_d = WaitWriteValid;
209     end
210 end
211
212 WaitWriteValid: begin
213     // got a valid answer go back to idle
214     if (dmi_resp_valid) begin
215         unique case (dmi_resp.resp)
216             dmi::DTM_ERR: error_dmi_op_failed = 1'b1;
217             dmi::DTM_BUSY: error_dmi_busy = 1'b1;
218             default: ;
219         endcase
220         state_d = Idle;
221     end
222 end
223
224 default: begin
225     // just wait for idle here
226     if (dmi_resp_valid) begin
227         state_d = Idle;
228     end
229 end
230 endcase
231
232 // update means we got another request but we didn't
233 finish
234 // the one in progress, this state is sticky
235 if (update && state_q != Idle) begin
236     error_dmi_busy = 1'b1;
237 end
238
239 // if capture goes high while we are in the read state
240 // or in the corresponding wait state we are not giving
241 back a valid word
242 // -> throw an error
243 if (capture && state_q inside {Read, WaitReadValid}) be-
244 gin
245     error_dmi_busy = 1'b1;
246 end
247
248 if (error_dmi_busy && error_q == DMINoError) begin
249     error_d = DMIBusy;
250 end
251
252 if (error_dmi_op_failed && error_q == DMINoError) begin
253     error_d = DMIOPFailed;
254 end
255
256 // clear sticky error flag
257 if (update && dtmcs_q.dmireset && dtmcs_select) begin
258     error_d = DMINoError;
259 end
260
261 end
262 end
263
264 // shift register
265 assign dmi_tdo = dr_q[0];
266
267 always_comb begin : p_shift
268     dr_d = dr_q;
269     if (dmi_clear) begin
270         dr_d = '0;
271     end else begin
272         if (capture) begin
273             if (dmi_select) begin
274                 if (error_q == DMINoError && !error_dmi_busy) begin
275                     dr_d = {address_q, data_q, DMINoError};
276                     // DMI was busy, report an error
277                 end else if (error_q == DMIBusy || error_dmi_busy)
278                 begin
279                     dr_d = {address_q, data_q, DMIBusy};
280                 end
281             end
282         end
283     end
284 end
285
286 if (shift) begin
287     if (dmi_select) begin
288         dr_d = {tdi, dr_q[$bits(dr_q)-1:1]};
289     end
290 end
291 end
292
293 always_ff @(posedge tck or negedge trst_ni) begin
294     if (!trst_ni) begin
295         dr_q <= '0;
296         state_q <= Idle;

```

```

289     address_q <= '0;
290     data_q <= '0;
291     error_q <= DMINoError;
292 end else begin
293     dr_q <= dr_d;
294     state_q <= state_d;
295     address_q <= address_d;
296     data_q <= data_d;
297     error_q <= error_d;
298 end
299 end
300
301 // -----
302 // TAP
303 // -----
304 dmi_jtag_tap #(
305     .IrLength (5),
306     .IdcodeValue(IdcodeValue)
307 ) i_dmi_jtag_tap (
308     .tck_i,
309     .tms_i,
310     .trst_ni,
311     .tdi_i,
312     .tdo_o,
313     .tdo_oe_o,
314     .testmode_i,
315     .tck_o ( tck ),
316     .dmi_clear_o ( jtag_dmi_clear ),
317     .update_o ( update ),
318     .capture_o ( capture ),
319     .shift_o ( shift ),
320     .tdi_o ( tdi ),
321     .dtmcs_select_o ( dtmcs_select ),
322     .dtmcs_tdo_i ( dtmcs_q[0] ),
323     .dmi_select_o ( dmi_select ),
324     .dmi_tdo_i ( dmi_tdo )
325 );
326
327 // -----
328 // CDC
329 // -----
330 dmi_cdc i_dmi_cdc (
331     // JTAG side (master side)
332     .tck_i ( tck ),
333     .trst_ni ( trst_ni ),
334     .jtag_dmi_cdc_clear_i ( dmi_clear ),
335     .jtag_dmi_req_i ( dmi_req ),
336     .jtag_dmi_ready_o ( dmi_req_ready ),
337     .jtag_dmi_valid_i ( dmi_req_valid ),
338     .jtag_dmi_resp_o ( dmi_resp ),
339     .jtag_dmi_valid_o ( dmi_resp_valid ),
340     .jtag_dmi_ready_i ( dmi_resp_ready ),
341     // core side
342     .clk_i,
343     .rst_ni,
344     .core_dmi_rst_no ( dmi_rst_no ),
345     .core_dmi_req_o ( dmi_req_o ),
346     .core_dmi_valid_o ( dmi_req_valid_o ),
347     .core_dmi_ready_i ( dmi_req_ready_i ),
348     .core_dmi_resp_i ( dmi_resp_i ),
349     .core_dmi_ready_o ( dmi_resp_ready_o ),
350     .core_dmi_valid_i ( dmi_resp_valid_i )
351 );
352
353 endmodule : dmi_jtag

```

11. pulp-platform_riscv-dbg/src/dmi_jtag_tap.sv (326 lines)

```

1 /* Copyright 2018 ETH Zurich and University of Bologna.
2 * Copyright and related rights are licensed under the Solder-
3 * pad Hardware
4 * License, Version 0.51 (the "License"); you may not use this
5 * file except in
6 * compliance with the License. You may obtain a copy of the
7 * License at
8 * http://solderpad.org/licenses/SHL-0.51. Unless required by
9 * applicable law
10 * or agreed to in writing, software, hardware and materials
11 * distributed under
12 * this License is distributed on an "AS IS" BASIS, WITHOUT
13 * WARRANTIES OR
14 * CONDITIONS OF ANY KIND, either express or implied. See the
15 * License for the
16 * specific language governing permissions and limitations un-
17 * der the License.
18 *
19 * File: dmi_jtag_tap.sv
20 * Author: Florian Zaruba <zarubaf@iis.ee.ethz.ch>
21 * Date: 19.7.2018
22 *
23 * Description: JTAG TAP for DMI (according to debug spec
24 * 0.13)
25 *
26 */
27
28 module dmi_jtag_tap #(
29     parameter int unsigned IrLength = 5,
30     // JTAG IDCODE Value
31     parameter logic [31:0] IdcodeValue = 32'h000000001

```

```

23 // xxxx version
24 // xxxxxxxxxxxxxxxx part number
25 // xxxxxxxxxxxxxxxx manufacturer id
26 // 1 required by standard
27 ) (
28 input logic tck_i, // JTAG test clock pad
29 input logic tms_i, // JTAG test mode select pad
30 input logic trst_ni, // JTAG test reset pad
31 input logic td_i, // JTAG test data input pad
32 output logic td_o, // JTAG test data output pad
33 output logic tdo_oe_o, // Data out output enable
34 input logic testmode_i,
35 // JTAG is interested in writing the DTM CSR register
36 output logic tck_o,
37 // Synchronous reset of the dmi module triggered by JTAG TAP
38 output logic dmi_clear_o,
39 output logic update_o,
40 output logic capture_o,
41 output logic shift_o,
42 output logic tdi_o,
43 output logic dtmcs_select_o,
44 input logic dtmcs_tdo_i,
45 // we want to access DMI register
46 output logic dmi_select_o,
47 input logic dmi_tdo_i
48 );
49
50 typedef enum logic [3:0] {
51     TestLogicReset, RunTestIdle, SelectDrScan,
52     CaptureDr, ShiftDr, Exit1Dr, PauseDr, Exit2Dr,
53     UpdateDr, SelectIrScan, CaptureIr, ShiftIr,
54     Exit1Ir, PauseIr, Exit2Ir, UpdateIr
55 } tap_state_e;
56
57 tap_state_e tap_state_q, tap_state_d;
58 logic update_dr, shift_dr, capture_dr;
59
60 typedef enum logic [IrLength-1:0] {
61     BYPASS0 = 'h0,
62     IDCODE = 'h1,
63     DTMCSR = 'h10,
64     DMIACCESS = 'h11,
65     BYPASS1 = 'h1f
66 } ir_reg_e;
67
68 // -----
69 // IR logic
70 // -----
71
72 // shift register
73 logic [IrLength-1:0] jtag_ir_shift_d, jtag_ir_shift_q;
74 // IR register -> this gets captured from shift register
75
76 upon update_ir
77     ir_reg_e jtag_ir_d, jtag_ir_q;
78     logic capture_ir, shift_ir, update_ir, test_logic_reset; //
79
80 pause_ir
81
82 always_comb begin : p_jtag
83     jtag_ir_shift_d = jtag_ir_shift_q;
84     jtag_ir_d = jtag_ir_q;
85
86     // IR shift register
87     if (shift_ir) begin
88         jtag_ir_shift_d = {td_i, jtag_ir_shift_q[IrLength-1:1]};
89     end
90
91     // capture IR register
92     if (capture_ir) begin
93         jtag_ir_shift_d = IrLength'(4'b0101);
94     end
95
96     // update IR register
97     if (update_ir) begin
98         jtag_ir_d = ir_reg_e'(jtag_ir_shift_q);
99     end
100
101     if (test_logic_reset) begin
102         // Bring all TAP state to the initial value.
103         jtag_ir_shift_d = '0;
104         jtag_ir_d = IDCODE;
105     end
106 end
107
108 p_jtag_ir_reg
109     if (!trst_ni) begin
110         jtag_ir_shift_q <= '0;
111         jtag_ir_q <= IDCODE;
112     end else begin
113         jtag_ir_shift_q <= jtag_ir_shift_d;
114         jtag_ir_q <= jtag_ir_d;
115     end
116 end
117
118 // -----
119 // TAP DR Regs
120 // -----
121 // - Bypass
122 // - IDCODE
123 // - DTM CS
124 logic [31:0] idcode_d, idcode_q;

```

```

121 logic idcode_select;
122 logic bypass_select;
123
124 logic bypass_d, bypass_q; // this is a 1-bit re-
125 gister
126
127 always_comb begin
128     idcode_d = idcode_q;
129     bypass_d = bypass_q;
130
131     if (capture_dr) begin
132         if (idcode_select) idcode_d = IdcodeValue;
133         if (bypass_select) bypass_d = 1'b0;
134     end
135
136     if (shift_dr) begin
137         if (idcode_select) idcode_d = {td_i, 31'(idcode_q >>
138 1)};
139         if (bypass_select) bypass_d = td_i;
140     end
141
142     if (test_logic_reset) begin
143         // Bring all TAP state to the initial value.
144         idcode_d = IdcodeValue;
145         bypass_d = 1'b0;
146     end
147 end
148
149 // -----
150 // Data reg select
151 // -----
152 always_comb begin : p_data_reg_sel
153     dmi_select_o = 1'b0;
154     dtmcs_select_o = 1'b0;
155     idcode_select = 1'b0;
156     bypass_select = 1'b0;
157     unique case (jtag_ir_q)
158         BYPASS0: bypass_select = 1'b1;
159         IDCODE: idcode_select = 1'b1;
160         DTMCSR: dtmcs_select_o = 1'b1;
161         DMIACCESS: dmi_select_o = 1'b1;
162         BYPASS1: bypass_select = 1'b1;
163         default: bypass_select = 1'b1;
164     endcase
165 end
166
167 // -----
168 // Output select
169 // -----
170 logic tdo_mux;
171
172 always_comb begin : p_out_sel
173     // we are shifting out the IR register
174     if (shift_ir) begin
175         tdo_mux = jtag_ir_shift_q[0];
176     // here we are shifting the DR register
177     end else begin
178         unique case (jtag_ir_q)
179             IDCODE: tdo_mux = idcode_q[0]; // Reading ID
180             DTMCSR: tdo_mux = dtmcs_tdo_i; // Read from
181             DMIACCESS: tdo_mux = dmi_tdo_i; // Read from
182             default: tdo_mux = bypass_q; // BYPASS in-
183             struction
184         endcase
185     end
186 end
187
188 // -----
189 // DFT
190 // -----
191 logic tck_n, tck_ni;
192
193 tc_clk_inverter i_tck_inv (
194     .clk_i ( tck_i ),
195     .clk_o ( tck_ni )
196 );
197
198 tc_clk_mux2 i_dft_tck_mux (
199     .clk0_i ( tck_ni ),
200     .clk1_i ( tck_i ), // bypass the inverted clock
201 );
202
203 for testing
204     .clk_sel_i ( testmode_i ),
205     .clk_o ( tck_n )
206 );
207
208 // TDO changes state at negative edge of TCK
209 always_ff @(posedge tck_n, negedge trst_ni) begin :
210 p_tdo_regs
211     if (!trst_ni) begin
212         td_o <= 1'b0;
213         tdo_oe_o <= 1'b0;
214     end else begin
215         td_o <= tdo_mux;
216         tdo_oe_o <= (shift_ir | shift_dr);
217     end
218 end
219
220 // -----

```



```

213 // TAP FSM
214 // -----
215 // Determination of next state; purely combinatorial
216 always_comb begin : p_tap_fsm
217     test_logic_reset = 1'b0;
218
219     capture_dr       = 1'b0;
220     shift_dr         = 1'b0;
221     update_dr        = 1'b0;
222
223     capture_ir       = 1'b0;
224     shift_ir         = 1'b0;
225     // pause_ir      = 1'b0; unused
226     update_ir        = 1'b0;
227
228     unique case (tap_state_q)
229     TestLogicReset: begin
230         tap_state_d = (tms_i) ? TestLogicReset : RunTestIdle;
231         test_logic_reset = 1'b1;
232     end
233     RunTestIdle: begin
234         tap_state_d = (tms_i) ? SelectDrScan : RunTestIdle;
235     end
236     // DR Path
237     SelectDrScan: begin
238         tap_state_d = (tms_i) ? SelectIrScan : CaptureDr;
239     end
240     CaptureDr: begin
241         capture_dr = 1'b1;
242         tap_state_d = (tms_i) ? Exit1Dr : ShiftDr;
243     end
244     ShiftDr: begin
245         shift_dr = 1'b1;
246         tap_state_d = (tms_i) ? Exit1Dr : ShiftDr;
247     end
248     Exit1Dr: begin
249         tap_state_d = (tms_i) ? UpdateDr : PauseDr;
250     end
251     PauseDr: begin
252         tap_state_d = (tms_i) ? Exit2Dr : PauseDr;
253     end
254     Exit2Dr: begin
255         tap_state_d = (tms_i) ? UpdateDr : ShiftDr;
256     end
257     UpdateDr: begin
258         update_dr = 1'b1;
259         tap_state_d = (tms_i) ? SelectDrScan : RunTestIdle;
260     end
261     // IR Path
262     SelectIrScan: begin
263         tap_state_d = (tms_i) ? TestLogicReset : CaptureIr;
264     end
265     // In this controller state, the shift register bank in
266     the
267     // Instruction Register parallel loads a pattern of
268     // the rising edge of TCK. The last two significant bits
269     // must always
270     // be "01".
271     CaptureIr: begin
272         capture_ir = 1'b1;
273         tap_state_d = (tms_i) ? Exit1Ir : ShiftIr;
274     end
275     // In this controller state, the instruction register
276     // gets connected
277     // between TDI and TD0, and the captured pattern gets
278     // shifted on
279     // each rising edge of TCK. The instruction available on
280     // the TDI
281     // pin is also shifted in to the instruction register.
282     ShiftIr: begin
283         shift_ir = 1'b1;
284         tap_state_d = (tms_i) ? Exit1Ir : ShiftIr;
285     end
286     Exit1Ir: begin
287         tap_state_d = (tms_i) ? UpdateIr : PauseIr;
288     end
289     PauseIr: begin
290         // pause_ir = 1'b1; // unused
291         tap_state_d = (tms_i) ? Exit2Ir : PauseIr;
292     end
293     Exit2Ir: begin
294         tap_state_d = (tms_i) ? UpdateIr : ShiftIr;
295     end
296     // In this controller state, the instruction in the in-
297     // struction
298     // shift register is latched to the latch bank of the
299     // Instruction
300     // Register on every falling edge of TCK. This instruc-
301     // tion becomes
302     // the current instruction once it is latched.
303     UpdateIr: begin
304         update_ir = 1'b1;
305         tap_state_d = (tms_i) ? SelectDrScan : RunTestIdle;
306     end
307     default: ; // can't actually happen since case is full
308 endcase
309 end

```

```

303
304 always_ff @(posedge tck_i or negedge trst_ni) begin : p_regs
305     if (!trst_ni) begin
306         tap_state_q <= TestLogicReset;
307         idcode_q <= IdcodeValue;
308         bypass_q <= 1'b0;
309     end else begin
310         tap_state_q <= tap_state_d;
311         idcode_q <= idcode_d;
312         bypass_q <= bypass_d;
313     end
314 end
315
316 // Pass through JTAG signals to debug custom DR logic.
317 // In case of a single TAP those are just feed-through.
318 assign tck_o = tck_i;
319 assign tdi_o = tdi_i;
320 assign update_o = update_dr;
321 assign shift_o = shift_dr;
322 assign capture_o = capture_dr;
323 assign dmi_clear_o = test_logic_reset;
324
325 endmodule : dmi_jtag_tap

```

12. pulp-platform_riscv-dbg/src/dmi_test.sv (339 lines)

```

1 // Copyright 2021 ETH Zurich and University of Bologna.
2 // Solderpad Hardware License, Version 0.51, see LICENSE for
3 // details.
4 // SPD-License-Identifier: SHL-0.51
5 // Florian Zaruba <zarubaf@iis.ee.ethz.ch>
6
7 /// A set of testbench utilities for the DMI interfaces.
8 package dmi_test;
9
10 import dm::*;
11
12 class req_t #(
13     parameter int AW = 7
14 );
15     rand logic [AW-1:0] addr;
16     rand dtm_op_e op;
17     rand logic [31:0] data;
18
19     /// Compare objects of same type.
20     function do_compare(req_t rhs);
21         return addr == rhs.addr &
22             op == rhs.op &
23             data == rhs.data;
24     endfunction
25
26 endclass
27
28 class rsp_t;
29     rand logic [31:0] data;
30     rand logic [1:0] resp;
31
32     /// Compare objects of same type.
33     function do_compare(rsp_t rhs);
34         return data == rhs.data &
35             resp == rhs.resp;
36     endfunction
37
38 endclass
39
40 /// A driver for the DMI interface.
41 class dmi_driver #(
42     parameter int AW = -1,
43     parameter realtime TA = 0, // stimuli application time
44     parameter realtime TT = 0 // stimuli test time
45 );
46     virtual DMI_BUS_DV #(
47         .ADDR_WIDTH(AW)
48     ) bus;
49
50     function new(
51         virtual DMI_BUS_DV #(
52             .ADDR_WIDTH(AW)
53         ) bus
54     );
55         this.bus = bus;
56     endfunction
57
58     task reset_master;
59         bus.q_addr <= '0;
60         bus.q_op <= DTM_NOP;
61         bus.q_data <= '0;
62         bus.q_valid <= '0;
63         bus.p_ready <= '0;
64     endtask
65
66     task reset_slave;
67         bus.q_ready <= '0;
68         bus.p_data <= '0;
69         bus.p_resp <= '0;
70         bus.p_valid <= '0;
71     endtask
72

```

```

73 task cycle_start;
74 #TT;
75 endtask
76
77 task cycle_end;
78 @(posedge bus.clk_i);
79 endtask
80
81 /// Send a request.
82 task send_req (input req_t req);
83 bus.q_addr <= #TA req.addr;
84 bus.q_op <= #TA req.op;
85 bus.q_data <= #TA req.data;
86 bus.q_valid <= #TA 1;
87 cycle_start();
88 while (bus.q_ready != 1) begin cycle_end();
89 cycle_start(); end
89 cycle_end();
90 bus.q_addr <= #TA '0;
91 bus.q_op <= #TA DTM_NOP;
92 bus.q_data <= #TA '0;
93 bus.q_valid <= #TA 0;
94 endtask
95
96 /// Send a response.
97 task send_rsp (input rsp_t rsp);
98 bus.p_data <= #TA rsp.data;
99 bus.p_resp <= #TA rsp.resp;
100 bus.p_valid <= #TA 1;
101 cycle_start();
102 while (bus.p_ready != 1) begin cycle_end();
103 cycle_start(); end
103 cycle_end();
104 bus.p_data <= #TA '0;
105 bus.p_resp <= #TA '0;
106 bus.p_valid <= #TA 0;
107 endtask
108
109 /// Receive a request.
110 task rcv_req (output req_t req);
111 bus.q_ready <= #TA 1;
112 cycle_start();
113 while (bus.q_valid != 1) begin cycle_end();
114 cycle_start(); end
114 req = new;
115 req.addr = bus.q_addr;
116 req.op = bus.q_op;
117 req.data = bus.q_data;
118 cycle_end();
119 bus.q_ready <= #TA 0;
120 endtask
121
122 /// Receive a response.
123 task rcv_rsp (output rsp_t rsp);
124 bus.p_ready <= #TA 1;
125 cycle_start();
126 while (bus.p_valid != 1) begin cycle_end();
127 cycle_start(); end
127 rsp = new;
128 rsp.data = bus.p_data;
129 rsp.resp = bus.p_resp;
130 cycle_end();
131 bus.p_ready <= #TA 0;
132 endtask
133
134 /// Monitor request.
135 task mon_req (output req_t req);
136 cycle_start();
137 while (!(bus.q_valid && bus.q_ready)) begin cycle_end();
138 cycle_start(); end
138 req = new;
139 req.addr = bus.q_addr;
140 req.op = bus.q_op;
141 req.data = bus.q_data;
142 cycle_end();
143 endtask
144
145 /// Monitor response.
146 task mon_rsp (output rsp_t rsp);
147 cycle_start();
148 while (!(bus.p_valid && bus.p_ready)) begin cycle_end();
149 cycle_start(); end
149 rsp = new;
150 rsp.data = bus.p_data;
151 rsp.resp = bus.p_resp;
152 cycle_end();
153 endtask
154
155 endclass
156
157 // Super class for random dmi drivers.
158 virtual class rand_dmi #(
159 // dmi interface parameters
160 parameter int AW = 32,
161 // Stimuli application and test time
162 parameter realtime TA = 0ps,
163 parameter realtime TT = 0ps
164 );
165

```

```

166 typedef dmi_test::dmi_driver #(
167 // dmi bus interface parameters;
168 .AW ( AW ),
169 // Stimuli application and test time
170 .TA ( TA ),
171 .TT ( TT )
172 ) dmi_driver_t;
173
174 dmi_driver_t drv;
175
176 function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (AW)) bus);
177 this.drv = new (bus);
178 endfunction
179
180 task automatic rand_wait(input int unsigned min, input int
181 unsigned max);
182 int unsigned cycles, rand_success, selection;
183 assert (min <= max) else $fatal(1, "Invalid random wait
184 range.");
185 // Give the minimum ten times the weight of every other
186 value. Expressing
187 // this directly avoids simulator-dependent support for
188 'dist' constraints.
189 rand_success = std::randomize(selection) with {
190 selection <= max - min + 9;
191 };
192 assert (rand_success == 1) else $fatal(1, "Failed to
193 randomize wait cycles.");
194 cycles = (selection < 10) ? min : min + selection - 9;
195 repeat (cycles) @(posedge this.drv.bus.clk_i);
196 endtask
197
198 endclass
199
200 /// Generate random requests as a master device.
201 class rand_dmi_master #(
202 // dmi interface parameters
203 parameter int AW = 32,
204 // Stimuli application and test time
205 parameter realtime TA = 0ps,
206 parameter realtime TT = 0ps,
207 parameter int unsigned REQ_MIN_WAIT_CYCLES = 1,
208 parameter int unsigned REQ_MAX_WAIT_CYCLES = 20,
209 parameter int unsigned RSP_MIN_WAIT_CYCLES = 1,
210 parameter int unsigned RSP_MAX_WAIT_CYCLES = 20
211 ) extends rand_dmi #( .AW(AW), .TA(TA), .TT(TT));
212
213 int unsigned cnt = 0;
214 bit req_done = 0;
215
216 /// Reset the driver.
217 task reset();
218 drv.reset_master();
219 endtask
220
221 /// Constructor.
222 function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (AW)) bus);
223 super.new(bus);
224 endfunction
225
226 task run(input int n);
227 fork
228 send_requests(n);
229 rcv_response();
230 join
231 endtask
232
233 /// Send random requests.
234 task send_requests (input int n);
235 automatic req_t r = new;
236
237 repeat (n) begin
238 this.cnt++;
239 assert(r.randomize());
240 rand_wait(REQ_MIN_WAIT_CYCLES, REQ_MAX_WAIT_CYCLES);
241 this.drv.send_req(r);
242 end
243 this.req_done = 1;
244 endtask
245
246 /// Receive random responses.
247 task rcv_response;
248 while (!this.req_done || this.cnt > 0) begin
249 automatic rsp_t rsp;
250 this.cnt--;
251 rand_wait(RSP_MIN_WAIT_CYCLES, RSP_MAX_WAIT_CYCLES);
252 this.drv.rcv_rsp(rsp);
253 end
254 endtask
255
256 endclass
257
258 class rand_dmi_slave #(
259 // dmi interface parameters
260 parameter int AW = 32,
261 // Stimuli application and test time
262 parameter realtime TA = 0ps,
263 parameter realtime TT = 0ps,
264 parameter int unsigned REQ_MIN_WAIT_CYCLES = 0,
265 parameter int unsigned REQ_MAX_WAIT_CYCLES = 10,
266 parameter int unsigned RSP_MIN_WAIT_CYCLES = 0,

```

```

261     parameter int unsigned RSP_MAX_WAIT_CYCLES = 10
262 ) extends rand_dmi #(.Aw(Aw), .TA(TA), .TT(TT));
263
264     mailbox #(req_t) req_mbx = new();
265
266     /// Reset the driver.
267     task reset();
268     drv.reset_slave();
269     endtask
270
271     task run();
272     fork
273         recv_requests();
274         send_responses();
275     join
276     endtask
277
278     /// Constructor.
279     function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (Aw)) bus);
280     super.new(bus);
281     endfunction
282
283     task recv_requests();
284     forever begin
285         automatic req_t req;
286         rand_wait(REQ_MIN_WAIT_CYCLES, REQ_MAX_WAIT_CYCLES);
287         this.drv.recv_req(req);
288         req_mbx.put(req);
289     end
290     endtask
291
292     task send_responses();
293     automatic rsp_t rsp = new;
294     automatic req_t req;
295     forever begin
296         req_mbx.get(req);
297         rsp.data = $urandom;
298         rsp.resp = DTM_SUCCESS;
299         @(posedge this.drv.bus.clk_i);
300         rand_wait(RSP_MIN_WAIT_CYCLES, RSP_MAX_WAIT_CYCLES);

```

```

301         this.drv.send_rsp(rsp);
302     end
303     endtask
304 endclass
305
306 class dmi_monitor #(
307     // dmi interface parameters
308     parameter int Aw = 32,
309     // Stimuli application and test time
310     parameter realtime TA = 0ps,
311     parameter realtime TT = 0ps
312 ) extends rand_dmi #(.Aw(Aw), .TA(TA), .TT(TT));
313
314     mailbox #(req_t) req_mbx = new;
315     mailbox #(rsp_t) rsp_mbx = new;
316
317     /// Constructor.
318     function new(virtual DMI_BUS_DV #( .ADDR_WIDTH (Aw)) bus);
319     super.new(bus);
320     endfunction
321
322     // dmi Monitor.
323     task monitor;
324     fork
325         forever begin
326             automatic dmi_test::req_t req;
327             this.drv.mon_req(req);
328             req_mbx.put(req);
329         end
330         forever begin
331             automatic dmi_test::rsp_t rsp;
332             this.drv.mon_rsp(rsp);
333             rsp_mbx.put(rsp);
334         end
335     join
336     endtask
337 endclass
338
339 endpackage

```

CV32E40P RISC-V 코어

6 files • 3,930 lines

API — cv32e40p_aligner.sv

Table 1080. openhwgroup_cv32e40p/rtl/cv32e40p_aligner.sv — module / interface / package (1개)

이름	종류
cv32e40p_aligner	module

Table 1081. openhwgroup_cv32e40p/rtl/cv32e40p_aligner.sv — 포트 (13개)

포트	방향	형/폭
clk	input	logic
rst_n	input	logic
fetch_valid_i	input	logic
aligner_ready_o	output	logic
if_valid_i	input	logic
fetch_rdata_i	input	logic [31:0]
instr_aligned_o	output	logic [31:0]
instr_valid_o	output	logic
branch_addr_i	input	logic [31:0]
branch_i	input	logic
hwlp_addr_i	input	logic [31:0]
hwlp_update_pc_i	input	logic
pc_o	output	logic [31:0]

API — cv32e40p_alu.sv

Table 1082. openhwgroup_cv32e40p/rtl/cv32e40p_alu.sv — module / interface / package (1개)

이름	종류
cv32e40p_alu	module

Table 1083. openhwgroup_cv32e40p/rtl/cv32e40p_alu.sv — 포트 (18개)

포트	방향	형/폭
clk	input	logic
rst_n	input	logic
enable_i	input	logic
operator_i	input	alu_opcode_e
operand_a_i	input	logic [31:0]

포트	방향	형/폭
operand_b_i	input	logic [31:0]
operand_c_i	input	logic [31:0]
vector_mode_i	input	logic [1:0]
bmask_a_i	input	logic [4:0]
bmask_b_i	input	logic [4:0]
imm_vec_ext_i	input	logic [1:0]
is_clpx_i	input	logic
is_subrot_i	input	logic
clpx_shift_i	input	logic [1:0]
result_o	output	logic [31:0]
comparison_result_o	output	logic
ready_o	output	logic
ex_ready_i	input	logic

API — cv32e40p_alu_div.sv

Table 1084. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv — module / interface / package (1개)

이름	종류
cv32e40p_alu_div	module

Table 1085. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv — parameter (2개)

파라미터	형/폭	기본값
C_WIDTH		32
C_LOG_WIDTH		6

Table 1086. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv — 포트 (12개)

포트	방향	형/폭
Clk_CI	input	logic
Rst_RBI	input	logic
OpA_DI	input	logic [C_WIDTH-1:0]
OpB_DI	input	logic [C_WIDTH-1:0]
OpBShift_DI	input	logic [C_LOG_WIDTH-1:0]
OpBIsZero_SI	input	logic
OpBSign_SI	input	logic
OpCode_SI	input	logic [1:0]
InVld_SI	input	logic
OutRdy_SI	input	logic

포트	방향	형/폭
OutVld_S0	output	logic
Res_D0	output	logic [C_WIDTH-1:0]

API — cv32e40p_apu_disp.sv

Table 1087. openhwgroup_cv32e40p/rtl/cv32e40p_apu_disp.sv — module / interface / package (1개)

이름	종류
cv32e40p_apu_disp	module

Table 1088. openhwgroup_cv32e40p/rtl/cv32e40p_apu_disp.sv — 포트 (23개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
enable_i	input	logic
apu_lat_i	input	logic [1:0]
apu_waddr_i	input	logic [5:0]
apu_waddr_o	output	logic [5:0]
apu_multicycle_o	output	logic
apu_singlecycle_o	output	logic
active_o	output	logic
stall_o	output	logic
is_decoding_i	input	logic
read_regs_i	input	logic [2:0][5:0]
read_regs_valid_i	input	logic [2:0]
read_dep_o	output	logic
read_dep_for_jalr_o	output	logic
write_regs_i	input	logic [1:0][5:0]
write_regs_valid_i	input	logic [1:0]
write_dep_o	output	logic
perf_type_o	output	logic
perf_cont_o	output	logic
apu_req_o	output	logic
apu_gnt_i	input	logic
apu_rvalid_i	input	logic

API — cv32e40p_compressed_decoder.sv

Table 1089. openhwgroup_cv32e40p/rtl/cv32e40p_compressed_decoder.sv — module / interface / package (1개)

이름	종류
cv32e40p_compressed_decoder	module

Table 1090. openhwgroup_cv32e40p/rtl/cv32e40p_compressed_decoder.sv — parameter (2개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0

Table 1091. openhwgroup_cv32e40p/rtl/cv32e40p_compressed_decoder.sv — 포트 (4개)

포트	방향	형/폭
instr_i	input	logic [31:0]
instr_o	output	logic [31:0]
is_compressed_o	output	logic
illegal_instr_o	output	logic

API — cv32e40p_controller.sv**Table 1092. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv — module / interface / package (1개)**

이름	종류
cv32e40p_controller	module

Table 1093. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv — parameter (3개)

파라미터	형/폭	기본값
COREV_CLUSTER		0
COREV_PULP		0
FPU		0

Table 1094. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv — 포트 (108개) — 앞 60개만 표시(전체 108개, 본문 소스 참조)

포트	방향	형/폭
clk	input	logic
clk_ungated_i	input	logic
rst_n	input	logic
fetch_enable_i	input	logic
ctrl_busy_o	output	logic
is_decoding_o	output	logic
is_fetch_failed_i	input	logic
deassert_we_o	output	logic
illegal_insn_i	input	logic
ecall_insn_i	input	logic

포트	방향	형/폭
mret_insn_i	input	logic
uret_insn_i	input	logic
dret_insn_i	input	logic
mret_dec_i	input	logic
uret_dec_i	input	logic
dret_dec_i	input	logic
wfi_i	input	logic
ebrk_insn_i	input	logic
fencei_insn_i	input	logic
csr_status_i	input	logic
hwlp_mask_o	output	logic
instr_valid_i	input	logic
instr_req_o	output	logic
pc_set_o	output	logic
pc_mux_o	output	logic [3:0]
exc_pc_mux_o	output	logic [2:0]
trap_addr_mux_o	output	logic [1:0]
pc_id_i	input	logic [31:0]
hwlp_start_addr_i	input	logic [1:0] [31:0]
hwlp_end_addr_i	input	logic [1:0] [31:0]
hwlp_counter_i	input	logic [1:0] [31:0]
hwlp_dec_cnt_o	output	logic [1:0]
hwlp_jump_o	output	logic
hwlp_targ_addr_o	output	logic [31:0]
data_req_ex_i	input	logic
data_we_ex_i	input	logic
data_misaligned_i	input	logic
data_load_event_i	input	logic
data_err_i	input	logic
data_err_ack_o	output	logic
mult_multicycle_i	input	logic
apu_en_i	input	logic
apu_read_dep_i	input	logic
apu_read_dep_for_jalr_i	input	logic
apu_write_dep_i	input	logic

포트	방향	형/폭
apu_stall_o	output	logic
branch_taken_ex_i	input	logic
ctrl_transfer_insn_in_id_i	input	logic [1:0]
ctrl_transfer_insn_in_dec_i	input	logic [1:0]
irq_req_ctrl_i	input	logic
irq_sec_ctrl_i	input	logic
irq_id_ctrl_i	input	logic [4:0]
irq_wu_ctrl_i	input	logic
current_priv_lvl_i	input	PrivLvl_t
irq_ack_o	output	logic
irq_id_o	output	logic [4:0]
exc_cause_o	output	logic [4:0]
debug_mode_o	output	logic
debug_cause_o	output	logic [2:0]
debug_csr_save_o	output	logic

1. openhwgroup_cv32e40p/rtl/cv32e40p_aligner.sv (252 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // Engineer:      Pasquale Davide Schiavone - ps-
  chiavo@iis.ee.ethz.ch
13 //
14 // Additional contributions
15 //
16 //
17 // Design Name:   Instrctuan Align-
  er
18 // Project Name:
19 // Language:      SystemVeri-
  log
20 //
21 ///////////////////////////////////////////////////////////////////
22
23 module cv32e40p_aligner (
24     input logic clk,
25     input logic rst_n,
26
27     input logic fetch_valid_i,
28     output logic aligner_ready_o, //prevents overwriting the

```

fethced instruction

```

29
30     input logic if_valid_i,
31
32     input logic [31:0] fetch_rdata_i,
33     output logic [31:0] instr_aligned_o,
34     output logic
35         instr_valid_o,
36
37     input logic [31:0] branch_addr_i,
38     input logic
39         branch_i, // Asserted if we are
  branching/jumping now
40
41     input logic [31:0] hwlp_addr_i,
42     input logic
43         hwlp_update_pc_i,
44
45     output logic [31:0] pc_o
46 );
47
48 enum logic [2:0] {
49     ALIGNED32,
50     MISALIGNED32,
51     MISALIGNED16,
52     BRANCH_MISALIGNED,
53     WAIT_VALID_BRANCH
54 }
55
56 state, next_state;
57
58 logic [15:0] r_instr_h;
59 logic [31:0] hwlp_addr_q;
60 logic [31:0] pc_q, pc_n;
61 logic update_state;
62 logic [31:0] pc_plus4, pc_plus2;
63 logic aligner_ready_q, hwlp_update_pc_q;
64
65 assign pc_o = pc_q;
66
67 assign pc_plus2 = pc_q + 2;
68 assign pc_plus4 = pc_q + 4;
69
70 always_ff @(posedge clk or negedge rst_n) begin :
  proc_SEQ_FSM
71     if (~rst_n) begin
72         state <= ALIGNED32;
73         r_instr_h <= '0;
74         hwlp_addr_q <= '0;
75         pc_q <= '0;
76         aligner_ready_q <= 1'b0;
77         hwlp_update_pc_q <= 1'b0;
78     end else begin
79         if (update_state) begin
80             pc_q <= pc_n;
81             state <= next_state;
82             r_instr_h <= fetch_rdata_i[31:16];
83             aligner_ready_q <= aligner_ready_o;

```

```

80     hwl原因_update_pc_q <= 1'b0;
81   end else begin
82     if (hwl原因_update_pc_i) begin
83       hwl原因_addr_q <= hwl原因_addr_i; // Save the JUMP
target address to keep pc_n up to date during the stall
84       hwl原因_update_pc_q <= 1'b1;
85     end
86   end
87 end
88 end
89 end
90
91 always_comb begin
92   //default outputs
93   pc_n = pc_q;
94   instr_valid_o = fetch_valid_i;
95   instr_aligned_o = fetch_rdata_i;
96   aligner_ready_o = 1'b1;
97   update_state = 1'b0;
98   next_state = state;
99
100
101   case (state)
102     ALIGNED32: begin
103       if (fetch_rdata_i[1:0] == 2'b11) begin
104         /*
105          * Before we fetched a 32bit aligned instruction
106          * Therefore, now the address is aligned too
107          * and it is 32bits
108          */
109         next_state = ALIGNED32;
110         pc_n = pc_plus4;
111         instr_aligned_o = fetch_rdata_i;
112         //gate id_valid with fetch_valid as the next state
113         update_state = fetch_valid_i & if_valid_i;
114         if (hwl原因_update_pc_i || hwl原因_update_pc_q)
115           pc_n = hwl原因_update_pc_i ? hwl原因_addr_i : hwl原因_ad-
dr_q;
116       end else begin
117         /*
118          * Before we fetched a 32bit aligned instruction
119          * Therefore, now the address is aligned too
120          * and it is 16bits
121          */
122         next_state = MISALIGNED32;
123         pc_n = pc_plus2;
124         instr_aligned_o = fetch_rdata_i; //only the first
16b are used
125         //gate id_valid with fetch_valid as the next state
126         update_state = fetch_valid_i & if_valid_i;
127       end
128     end
129
130     MISALIGNED32: begin
131       if (r_instr_h[1:0] == 2'b11) begin
132         /*
133          * Before we fetched a 32bit misaligned in-
struction
134          * So now the beginning of the next instruction
135          * is the stored one
136          * The instruction is 32bits so it is misaligned
137          * again
138          */
139         next_state = MISALIGNED32;
140         pc_n = pc_plus4;
141         instr_aligned_o = {fetch_rdata_i[15:0], r_in-
str_h[15:0]};
142         //gate id_valid with fetch_valid as the next state
143         update_state = fetch_valid_i & if_valid_i;
144       end else begin
145         /*
146          * Before we fetched a 32bit misaligned in-
struction
147          * So now the beginning of the next instruction
148          * is the stored one
149          * The instruction is 16bits misaligned
150          */
151         instr_aligned_o = {fetch_rdata_i[31:16], r_in-
str_h[15:0]}; //only the first 16b are used
152         next_state = MISALIGNED16;
153         instr_valid_o = 1'b1;
154         pc_n = pc_plus2;
155         //we cannot overwrite the 32bit instruction just
156         fetched
157         //so tell the IF stage to stall, the coming instruc-
tion goes to the FIFO
158         aligner_ready_o = !fetch_valid_i;
159         //not need to gate id_valid with fetch_valid as the
160         next state depends only on r_instr_h
161         update_state = if_valid_i;
162       end
163     end
164   end
165 end

```

```

158   end
159
160   MISALIGNED16: begin
161     //this is 1 as we holded the value before with raw_in-
str_hold_o
162     instr_valid_o = !aligner_ready_q || fetch_valid_i;
163     if (fetch_rdata_i[1:0] == 2'b11) begin
164       /*
165        * Before we fetched a 16bit misaligned in-
struction
166        * So now the beginning of the next instruction
167        * is the new one
168        * The instruction is 32bits so it is aligned
169        */
170     next_state = ALIGNED32;
171     pc_n = pc_plus4;
172     instr_aligned_o = fetch_rdata_i;
173     //no gate id_valid with fetch_valid as the next
174     state sdepends only on mem content that has be held the previous
175     cycle with raw_instr_hold_o
176   end else begin
177     /*
178      * Before we fetched a 16bit misaligned in-
struction
179      * So now the beginning of the next instruction
180      * is the new one
181      * The instruction is 16bit aligned
182      */
183     next_state = MISALIGNED32;
184     pc_n = pc_plus2;
185     instr_aligned_o = fetch_rdata_i; //only the first
16b are used
186     //no gate id_valid with fetch_valid as the next
187     state sdepends only on mem content that has be held the previous
188     cycle with raw_instr_hold_o
189     update_state = (!aligner_ready_q | fetch_valid_i) &
if_valid_i;
190   end
191 end
192
193   BRANCH_MISALIGNED: begin
194     //we jumped to a misaligned location, so now we re-
ceived {TARGET, XXXX}
195     if (fetch_rdata_i[17:16] == 2'b11) begin
196       /*
197        * We jumped to a misaligned location that con-
tains 32bits instruction
198        */
199     next_state = MISALIGNED32;
200     instr_valid_o = 1'b0;
201     pc_n = pc_q;
202     instr_aligned_o = fetch_rdata_i;
203     //gate id_valid with fetch_valid as the next state
204     update_state = fetch_valid_i & if_valid_i;
205   end else begin
206     /*
207      * We jumped to a misaligned location that con-
tains 16bits instruction, as we consumed the whole word, we can
208      prete to start again from ALIGNED32
209      */
210     next_state = ALIGNED32;
211     pc_n = pc_plus2;
212     instr_aligned_o = {
213       fetch_rdata_i[31:16], fetch_rdata_i[15:0]
214     }; //only the first 16b are used
215     //gate id_valid with fetch_valid as the next state
216     update_state = fetch_valid_i & if_valid_i;
217   end
218 end
219
220   endcase // state
221
222   // JUMP, BRANCH, SPECIAL JUMP control
223   if (branch_i) begin
224     update_state = 1'b1;
225     pc_n = branch_addr_i;
226     next_state = branch_addr_i[1] ? BRANCH_MISALIGNED :
ALIGNED32;
227   end
228   /*
229   When a branch is taken in EX, if_valid_i is asserted because
230   the BRANCH is resolved also in
231   case of stalls. This is because the branch information is
232   stored in the IF stage (in the prefetcher)
233   when branch_i is asserted. We introduced here an apparently
234   unuseful special case for
235   the JUMPS for a cleaner and more robust HW: theoretically,

```

```

we don't need to save the instruction
233   after a taken branch in EX, thus we will not do it.
234   */
235
236   //////////////////////////////////////
237   // Assertions
238   //////////////////////////////////////
239   //////////////////////////////////////
240   `ifdef CV32E40P_ASSERT_ON
241
242   // Hardware Loop check
243   property p_hwlp_update_pc;
244   @(posedge clk) disable iff (!rst_n) (1'b1) |> (! (hwlp_up-
date_pc_i && hwlp_update_pc_q));
245   endproperty
246
247   a_hwlp_update_pc :
248   assert property (p_hwlp_update_pc);
249
250   `endif
251
252 endmodule

```

2. openhwgroup_cv32e40p/rtl/cv32e40p_alu.sv (986 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the license.
10
11 //////////////////////////////////////
12 // Engineer:      Matthias Baer - baer-
matt@student.ethz.ch
13 //
14 // Additional contributions
by:
15 //      Igor Loi -
igor.loi@unibo.it
16 //      Andreas Traber - at-
raber@student.ethz.ch
17 //      Michael Gautschi - gauts-
chi@iis.ee.ethz.ch
18 //      Davide Schiavone - ps-
chiavo@iis.ee.ethz.ch
19 //
20 // Design Name:
ALU
21 // Project Name:
RISCY
22 // Language:      SystemVeri-
log
23 //
24 // Description:    Arithmetic logic unit of the pipelined pro-
cessor
25 //
26 //////////////////////////////////////
27
28 module cv32e40p_alu
29   import cv32e40p_pkg::*;
30 (
31   input logic      clk,
32   input logic      rst_n,
33   input logic      enable_i,
34   input alu_opcode_e operator_i,
35   input logic [31:0] operand_a_i,
36   input logic [31:0] operand_b_i,
37   input logic [31:0] operand_c_i,
38
39   input logic [1:0] vector_mode_i,
40   input logic [4:0] bmask_a_i,
41   input logic [4:0] bmask_b_i,
42   input logic [1:0] imm_vec_ext_i,
43
44   input logic      is_clpx_i,

```

```

45   input logic      is_subrot_i,
46   input logic [1:0] clpx_shift_i,
47
48   output logic [31:0] result_o,
49   output logic      comparison_result_o,
50
51   output logic ready_o,
52   input  logic ex_ready_i
53 );
54
55 logic [31:0] operand_a_rev;
56 logic [31:0] operand_a_neg;
57 logic [31:0] operand_a_neg_rev;
58
59 assign operand_a_neg = ~operand_a_i;
60
61 // bit reverse operand_a for left shifts and bit counting
62 generate
63   genvar k;
64   for (k = 0; k < 32; k++) begin : gen_operand_a_rev
65     assign operand_a_rev[k] = operand_a_i[31-k];
66   end
67 endgenerate
68
69 // bit reverse operand_a_neg for left shifts and bit count-
ing
70 generate
71   genvar m;
72   for (m = 0; m < 32; m++) begin : gen_operand_a_neg_rev
73     assign operand_a_neg_rev[m] = operand_a_neg[31-m];
74   end
75 endgenerate
76
77 logic [31:0] operand_b_neg;
78
79 assign operand_b_neg = ~operand_b_i;
80
81 logic [ 5:0] div_shift;
82 logic      div_valid;
83 logic [31:0] bmask;
84
85 //////////////////////////////////////
86 //////////////////////////////////////
87 //
88 //
89 //
90 //
91 //
92 //
93 //////////////////////////////////////
94
95 logic      adder_op_b_negate;
96 logic [31:0] adder_op_a, adder_op_b;
97 logic [35:0] adder_in_a, adder_in_b;
98 logic [31:0] adder_result;
99 logic [36:0] adder_result_expanded;
100
101
102 assign adder_op_b_negate = (operator_i == ALU_SUB) || (oper-
ator_i == ALU_SUBR) ||
103 (operator_i == ALU_SUBU) || (op-
erator_i == ALU_SUBUR) || is_subrot_i;
104
105 // prepare operand a
106 assign adder_op_a = (operator_i == ALU_ABS) ? oper-
and_a_neg : (is_subrot_i ? {
107   operand_b_i[15:0], operand_a_i[31:16]
108 } : operand_a_i);
109
110 // prepare operand b
111 assign adder_op_b = adder_op_b_negate ? (is_subrot_i ? ~{
112   operand_a_i[15:0], operand_b_i[31:16]
113 } : operand_b_neg) : operand_b_i;
114
115 // prepare carry
116 always_comb begin
117   adder_in_a[0] = 1'b1;
118   adder_in_a[8:1] = adder_op_a[7:0];
119   adder_in_a[9] = 1'b1;
120   adder_in_a[17:10] = adder_op_a[15:8];
121   adder_in_a[18] = 1'b1;
122   adder_in_a[26:19] = adder_op_a[23:16];
123   adder_in_a[27] = 1'b1;
124   adder_in_a[35:28] = adder_op_a[31:24];
125
126   adder_in_b[0] = 1'b0;
127   adder_in_b[8:1] = adder_op_b[7:0];
128   adder_in_b[9] = 1'b0;
129   adder_in_b[17:10] = adder_op_b[15:8];
130   adder_in_b[18] = 1'b0;

```

```

131 adder_in_b[26:19] = adder_op_b[23:16];
132 adder_in_b[27] = 1'b0;
133 adder_in_b[35:28] = adder_op_b[31:24];
134
135 if (adder_op_b_negate || (operator_i == ALU_ABS || operat-
or_i == ALU_CLIP)) begin
136 // special case for subtractions and absolute number
calculations
137 adder_in_b[0] = 1'b1;
138
139 case (vector_mode_i)
140 VEC_MODE16: begin
141 adder_in_b[18] = 1'b1;
142 end
143
144 VEC_MODE8: begin
145 adder_in_b[9] = 1'b1;
146 adder_in_b[18] = 1'b1;
147 adder_in_b[27] = 1'b1;
148 end
149 endcase
150
151 end else begin
152 // take care of partitioning the adder for the addition
case
153 case (vector_mode_i)
154 VEC_MODE16: begin
155 adder_in_a[18] = 1'b0;
156 end
157
158 VEC_MODE8: begin
159 adder_in_a[9] = 1'b0;
160 adder_in_a[18] = 1'b0;
161 adder_in_a[27] = 1'b0;
162 end
163 endcase
164 end
165 end
166
167 // actual adder
168 assign adder_result_expanded = $signed(adder_in_a) +
$signed(adder_in_b);
169 assign adder_result = {
170 adder_result_expanded[35:28],
171 adder_result_expanded[26:19],
172 adder_result_expanded[17:10],
173 adder_result_expanded[8:1]
174 };
175
176 // normalization stage
177 logic [31:0] adder_round_value;
178 logic [31:0] adder_round_result;
179
180
181 assign adder_round_value = ((operator_i == ALU_ADDR) ||
(operator_i == ALU_SUBR) ||
(operator_i == ALU_ADDUR) ||
(operator_i == ALU_SUBUR)) ?
182 {
183 1'b0, bmask[31:1]
184 } : '0;
185
186 assign adder_round_result = adder_result + ad-
der_round_value;
187
188
189 ////////////////////////////////////////////////////
190 //
191 // /---|---|---|---|---|---|
192 // \---|---|---|---|---|---|
193 // /---|---|---|---|---|---|
194 // \---|---|---|---|---|---|
195 //
196 ////////////////////////////////////////////////////
197
198 logic shift_left; // should we shift left
199 logic shift_use_round;
200 logic shift_arithmetic;
201
202 logic [31:0] shift_amt_left; // amount of shift, if to the
left
203 logic [31:0] shift_amt; // amount of shift, to the right
204 logic [31:0] shift_amt_int; // amount of shift, used for
the actual shifters
205 logic [31:0] shift_amt_norm; // amount of shift, used for
normalization
206 logic [31:0] shift_op_a; // input of the shifter
207 logic [31:0] shift_result;
208 logic [31:0] shift_right_result;
209 logic [31:0] shift_left_result;
210 logic [15:0] clpx_shift_ex;
211
212 // shifter is also used for preparing operand for division
213 assign shift_amt = div_valid ? div_shift : operand_b_i;
214
215 // by reversing the bits of the input, we also have to re-
verse the order of shift amounts
216 always_comb begin
217 case (vector_mode_i)
218 VEC_MODE16: begin
219 shift_amt_left[15:0] = shift_amt[31:16];
220 shift_amt_left[31:16] = shift_amt[15:0];

```

```

221 end
222
223 VEC_MODE8: begin
224 shift_amt_left[7:0] = shift_amt[31:24];
225 shift_amt_left[15:8] = shift_amt[23:16];
226 shift_amt_left[23:16] = shift_amt[15:8];
227 shift_amt_left[31:24] = shift_amt[7:0];
228 end
229
230 default: // VEC_MODE32
begin
231 shift_amt_left[31:0] = shift_amt[31:0];
232 end
233 endcase
234 end
235
236 // ALU_FL1 and ALU_CBL are used for the bit counting ops
later
237 assign shift_left = (operator_i == ALU_SLL) || (operator_i
== ALU_BINS) ||
238 (operator_i == ALU_FL1) || (operator_i
== ALU_CLB) ||
239 (operator_i == ALU_DIV) || (operator_i
== ALU_DIVU) ||
240 (operator_i == ALU_REM) || (operator_i
== ALU_REMU) ||
241 (operator_i == ALU_BREV);
242
243 assign shift_use_round = (operator_i == ALU_ADD) || (oper-
ator_i == ALU_SUB) ||
244 (operator_i == ALU_ADDR) || (oper-
ator_i == ALU_SUBR) ||
245 (operator_i == ALU_ADDU) || (oper-
ator_i == ALU_SUBU) ||
246 (operator_i == ALU_ADDUR) || (oper-
ator_i == ALU_SUBUR);
247
248 assign shift_arithmetic = (operator_i == ALU_SRA) || (oper-
ator_i == ALU_BEXT) ||
249 (operator_i == ALU_ADD) || (oper-
ator_i == ALU_SUB) ||
250 (operator_i == ALU_ADDR) || (oper-
ator_i == ALU_SUBR);
251
252 // choose the bit reversed or the normal input for shift op-
erand a
253 assign shift_op_a = shift_left ? operand_a_rev :
(shift_use_round ? ad-
der_round_result : operand_a_i);
254 assign shift_amt_int = shift_use_round ? shift_amt_norm :
(shift_left ? shift_amt_left :
shift_amt);
255
256 assign shift_amt_norm = is_clpx_i ? {clpx_shift_ex,
clpx_shift_ex : {4{3'b000, bmask_b_i}}};
257
258 assign clpx_shift_ex = $unsigned(clpx_shift_i);
259
260 // right shifts, we let the synthesizer optimize this
261 logic [63:0] shift_op_a_32;
262
263 assign shift_op_a_32 = (operator_i == ALU_ROR) ? {
shift_op_a, shift_op_a
264 } : $signed(
265 {32{shift_arithmetic & shift_op_a[31]}},
shift_op_a
266 );
267
268 always_comb begin
269 case (vector_mode_i)
270 VEC_MODE16: begin
271 shift_right_result[31:16] = $signed(
272 {shift_arithmetic & shift_op_a[31],
shift_op_a[31:16]}
273 ) >>> shift_amt_int[19:16];
274 shift_right_result[15:0] = $signed(
275 {shift_arithmetic & shift_op_a[15],
shift_op_a[15:0]}
276 ) >>> shift_amt_int[3:0];
277 end
278
279 VEC_MODE8: begin
280 shift_right_result[31:24] = $signed(
281 {shift_arithmetic & shift_op_a[31],
shift_op_a[31:24]}
282 ) >>> shift_amt_int[26:24];
283 shift_right_result[23:16] = $signed(
284 {shift_arithmetic & shift_op_a[23],
shift_op_a[23:16]}
285 ) >>> shift_amt_int[18:16];
286 shift_right_result[15:8] = $signed(
287 {shift_arithmetic & shift_op_a[15],
shift_op_a[15:8]}
288 ) >>> shift_amt_int[10:8];
289 shift_right_result[7:0] = $signed(
290 {shift_arithmetic & shift_op_a[7],
shift_op_a[7:0]}
291 ) >>> shift_amt_int[2:0];
292 end
293
294 default: // VEC_MODE32

```

```

299     begin
300         shift_right_result = shift_op_a_32 >>
shift_amt_int[4:0];
301     end
302 endcase
303 ; // case (vec_mode_i)
304 end
305
306 // bit reverse the shift_right_result for left shifts
307 genvar j;
308 generate
309     for (j = 0; j < 32; j++) begin : gen_shift_left_result
310         assign shift_left_result[j] = shift_right_result[31-j];
311     end
312 endgenerate
313
314 assign shift_result = shift_left ? shift_left_result :
shift_right_result;
315
316
317 ///////////////////////////////////////////////////////////////////
318 // _____ - _____ -
319 // / _/ _ \ \ / | _ \ \ / | _ \ _/ _ | / _ \ \ |
| //
320 // | | | | | | | \ / | | _ ) / _ \ | | _ ) | | \ _ \ | | | | \ |
| //
321 // | | | | | | | _/ _ \ | _ < | | _ ) | | | | |
| //
322 // \ _ \ _/ | | | | / \ \ \ \ _ \ | _ \ \ \ \ |
\ | //
323 //
324 ///////////////////////////////////////////////////////////////////
325
326 logic [ 3:0] is_equal;
327 logic [ 3:0] is_greater; // handles both signed and un-
signed forms
328
329 // 8-bit vector comparisons, basic building blocks
330 logic [ 3:0] cmp_signed;
331 logic [ 3:0] is_equal_vec;
332 logic [ 3:0] is_greater_vec;
333 logic [31:0] operand_b_eq;
334 logic
is_equal_clip;
335
336
337 //second == comparator for CLIP instructions
338 always_comb begin
339     operand_b_eq = operand_b_neg;
340     if (operator_i == ALU_CLIPU) operand_b_eq = '0;
341     else operand_b_eq = operand_b_neg;
342 end
343 assign is_equal_clip = operand_a_i == operand_b_eq;
344
345 always_comb begin
346     cmp_signed = 4'b0;
347
348     unique case (operator_i)
349         ALU_GTS,
350         ALU_GES,
351         ALU_LTS,
352         ALU_LES,
353         ALU_SLTS,
354         ALU_SLETS,
355         ALU_MIN,
356         ALU_MAX,
357         ALU_ABS,
358         ALU_CLIP,
359         ALU_CLIPU: begin
360             case (vector_mode_i)
361                 VEC_MODE8: cmp_signed[3:0] = 4'b1111;
362                 VEC_MODE16: cmp_signed[3:0] = 4'b1010;
363                 default: cmp_signed[3:0] = 4'b1000;
364             endcase
365         end
366
367         default: ;
368     endcase
369 end
370
371 // generate vector equal and greater than signals,
cmp_signed decides if the
372 // comparison is done signed or unsigned
373 genvar i;
374 generate
375     for (i = 0; i < 4; i++) begin : gen_is_vec
376         assign is_equal_vec[i] = (operand_a_i[8*i+7:8*i] == op-
erand_b_i[8*i+7:8*i]);
377         assign is_greater_vec[i] = $signed(
378             {operand_a_i[8*i+7] & cmp_signed[i], oper-
and_a_i[8*i+7:8*i]}
379             ) > $signed(
380             {operand_b_i[8*i+7] & cmp_signed[i], oper-
and_b_i[8*i+7:8*i]}

```

```

381     );
382     end
383     endgenerate
384
385     // generate the real equal and greater than signals that
take the vector
386     // mode into account
387     always_comb begin
388         // 32-bit mode
389         is_equal[3:0] = {4{is_equal_vec[3] & is_equal_vec[2] &
is_equal_vec[1] & is_equal_vec[0]}};
390         is_greater[3:0] = {4{is_greater_vec[3] | (is_equal_vec[3]
& (is_greater_vec[2]
391                                     | (is_equal_vec[2] &
392                                     |
(is_equal_vec[1] & (is_greater_vec[0]))))}});
393
394         case (vector_mode_i)
395             VEC_MODE16: begin
396                 is_equal[1:0] = {2{is_equal_vec[0] &
is_equal_vec[1]}};
397                 is_equal[3:2] = {2{is_equal_vec[2] &
is_equal_vec[3]}};
398                 is_greater[1:0] = {2{is_greater_vec[1] |
(is_equal_vec[1] & is_greater_vec[0])}};
399                 is_greater[3:2] = {2{is_greater_vec[3] |
(is_equal_vec[3] & is_greater_vec[2])}};
400             end
401
402             VEC_MODE8: begin
403                 is_equal[3:0] = is_equal_vec[3:0];
404                 is_greater[3:0] = is_greater_vec[3:0];
405             end
406
407             default: ; // see default assignment
408         endcase
409     end
410
411     // generate comparison result
412     logic [3:0] cmp_result;
413
414     always_comb begin
415         cmp_result = is_equal;
416         unique case (operator_i)
417             ALU_EQ: cmp_result =
is_equal;
418             ALU_NE: cmp_result =
~is_equal;
419             ALU_GTS, ALU_GTU: cmp_result =
is_greater;
420             ALU_GES, ALU_GEU: cmp_result =
is_greater | is_equal;
421             ALU_LTS, ALU_SLTS, ALU_LTU, ALU_SLTU: cmp_result =
~(is_greater | is_equal);
422             ALU_SLETS, ALU_SLETU, ALU_LES, ALU_LEU: cmp_result =
~is_greater;
423             default: ;
424         endcase
425     end
426
427     assign comparison_result_o = cmp_result[3];
428
429     // min/max/abs handling
430     logic [31:0] result_minmax;
431     logic [ 3:0] sel_minmax;
432     logic        do_min;
433     logic [31:0] minmax_b;
434
435     assign minmax_b = (operator_i == ALU_ABS) ? adder_result :
operand_b_i;
436
437
438     assign do_min      = (operator_i == ALU_MIN) || (operator_i ==
ALU_MINU) ||
439
440                                     (operator_i == ALU_CLIP) || (operator_i ==
ALU_CLIPU);
441
442     assign sel_minmax[3:0] = is_greater ^ {4{do_min}};
443
444     assign result_minmax[31:24] = (sel_minmax[3] == 1'b1) ? oper-
and_a_i[31:24] : minmax_b[31:24];
445     assign result_minmax[23:16] = (sel_minmax[2] == 1'b1) ? oper-
and_a_i[23:16] : minmax_b[23:16];
446     assign result_minmax[15:8] = (sel_minmax[1] == 1'b1) ? oper-
and_a_i[15:8] : minmax_b[15:8];
447     assign result_minmax[7:0] = (sel_minmax[0] == 1'b1) ? oper-
and_a_i[7:0] : minmax_b[7:0];
448
449     ////////////////////////////////////////////
450     // Clip
451     ////////////////////////////////////////////
452     logic [31:0] clip_result; // result of clip and clip
453
454     always_comb begin
455         clip_result = result_minmax;
456         if (operator_i == ALU_CLIPU) begin
457             if (operand_a_i[31] || is_equal_clip) begin

```

```

457     clip_result = '0';
458   end else begin
459     clip_result = result_minmax;
460   end
461 end else begin
462   //CLIP
463   if (adder_result_expanded[36] || is_equal_clip) begin
464     clip_result = operand_b_neg;
465   end else begin
466     clip_result = result_minmax;
467   end
468 end
469 end
470 end
471
472 ////////////////////////////////////////////////////
473 //
474 //
475 //
476 //
477 //
478 //
479 ////////////////////////////////////////////////////
480
481 logic [3:0][1:0] shuffle_byte_sel; // select byte in re-
gister: 31:24, 23:16, 15:8, 7:0
482 logic [3:0] shuffle_reg_sel; // select register: rD/
rS2 or rS1
483 logic [1:0] shuffle_reg1_sel; // select register rD or
rS2 for next stage
484 logic [1:0] shuffle_reg0_sel;
485 logic [3:0] shuffle_through;
486
487 logic [31:0] shuffle_r1, shuffle_r0;
488 logic [31:0] shuffle_r1_in, shuffle_r0_in;
489 logic [31:0] shuffle_result;
490 logic [31:0] pack_result;
491
492
493 always_comb begin
494   shuffle_reg_sel = '0;
495   shuffle_reg1_sel = 2'b01;
496   shuffle_reg0_sel = 2'b01;
497   shuffle_through = '1;
498
499   unique case (operator_i)
500     ALU_EXT, ALU_EXTs: begin
501       if (operator_i == ALU_EXTs) shuffle_reg1_sel = 2'b11;
502
503       if (vector_mode_i == VEC_MODE8) begin
504         shuffle_reg_sel[3:1] = 3'b111;
505         shuffle_reg_sel[0] = 1'b0;
506       end else begin
507         shuffle_reg_sel[3:2] = 2'b11;
508         shuffle_reg_sel[1:0] = 2'b00;
509       end
510     end
511
512     ALU_PCKL0: begin
513       shuffle_reg1_sel = 2'b00;
514
515       if (vector_mode_i == VEC_MODE8) begin
516         shuffle_through = 4'b0011;
517         shuffle_reg_sel = 4'b0001;
518       end else begin
519         shuffle_reg_sel = 4'b0011;
520       end
521     end
522
523     ALU_PCKHI: begin
524       shuffle_reg1_sel = 2'b00;
525
526       if (vector_mode_i == VEC_MODE8) begin
527         shuffle_through = 4'b1100;
528         shuffle_reg_sel = 4'b0100;
529       end else begin
530         shuffle_reg_sel = 4'b0011;
531       end
532     end
533
534     ALU_SHUF2: begin
535       unique case (vector_mode_i)
536         VEC_MODE8: begin
537           shuffle_reg_sel[3] = ~operand_b_i[26];
538           shuffle_reg_sel[2] = ~operand_b_i[18];
539           shuffle_reg_sel[1] = ~operand_b_i[10];
540           shuffle_reg_sel[0] = ~operand_b_i[2];
541         end
542
543         VEC_MODE16: begin
544           shuffle_reg_sel[3] = ~operand_b_i[17];
545           shuffle_reg_sel[2] = ~operand_b_i[17];
546           shuffle_reg_sel[1] = ~operand_b_i[1];
547           shuffle_reg_sel[0] = ~operand_b_i[1];
548         end
549       default: ;
550     endcase
551   end
552
553   ALU_INS: begin

```

```

554   unique case (vector_mode_i)
555     VEC_MODE8: begin
556       shuffle_reg0_sel = 2'b00;
557       unique case (imm_vec_ext_i)
558         2'b00: begin
559           shuffle_reg_sel[3:0] = 4'b1110;
560         end
561         2'b01: begin
562           shuffle_reg_sel[3:0] = 4'b1101;
563         end
564         2'b10: begin
565           shuffle_reg_sel[3:0] = 4'b1011;
566         end
567         2'b11: begin
568           shuffle_reg_sel[3:0] = 4'b0111;
569         end
570       endcase
571     end
572     VEC_MODE16: begin
573       shuffle_reg0_sel = 2'b01;
574       shuffle_reg_sel[3] = ~imm_vec_ext_i[0];
575       shuffle_reg_sel[2] = ~imm_vec_ext_i[0];
576       shuffle_reg_sel[1] = imm_vec_ext_i[0];
577       shuffle_reg_sel[0] = imm_vec_ext_i[0];
578     end
579   default: ;
580 endcase
581 end
582
583 default: ;
584 endcase
585 end
586
587 always_comb begin
588   shuffle_byte_sel = '0;
589
590   // byte selector
591   unique case (operator_i)
592     ALU_EXTs, ALU_EXT: begin
593       unique case (vector_mode_i)
594         VEC_MODE8: begin
595           shuffle_byte_sel[3] = imm_vec_ext_i[1:0];
596           shuffle_byte_sel[2] = imm_vec_ext_i[1:0];
597           shuffle_byte_sel[1] = imm_vec_ext_i[1:0];
598           shuffle_byte_sel[0] = imm_vec_ext_i[1:0];
599         end
600
601         VEC_MODE16: begin
602           shuffle_byte_sel[3] = {imm_vec_ext_i[0], 1'b1};
603           shuffle_byte_sel[2] = {imm_vec_ext_i[0], 1'b1};
604           shuffle_byte_sel[1] = {imm_vec_ext_i[0], 1'b1};
605           shuffle_byte_sel[0] = {imm_vec_ext_i[0], 1'b0};
606         end
607
608       default: ;
609     endcase
610   end
611
612   ALU_PCKL0: begin
613     unique case (vector_mode_i)
614       VEC_MODE8: begin
615         shuffle_byte_sel[3] = 2'b00;
616         shuffle_byte_sel[2] = 2'b00;
617         shuffle_byte_sel[1] = 2'b00;
618         shuffle_byte_sel[0] = 2'b00;
619       end
620
621       VEC_MODE16: begin
622         shuffle_byte_sel[3] = 2'b01;
623         shuffle_byte_sel[2] = 2'b00;
624         shuffle_byte_sel[1] = 2'b01;
625         shuffle_byte_sel[0] = 2'b00;
626       end
627
628     default: ;
629   endcase
630 end
631
632   ALU_PCKHI: begin
633     unique case (vector_mode_i)
634       VEC_MODE8: begin
635         shuffle_byte_sel[3] = 2'b00;
636         shuffle_byte_sel[2] = 2'b00;
637         shuffle_byte_sel[1] = 2'b00;
638         shuffle_byte_sel[0] = 2'b00;
639       end
640
641       VEC_MODE16: begin
642         shuffle_byte_sel[3] = 2'b11;
643         shuffle_byte_sel[2] = 2'b10;
644         shuffle_byte_sel[1] = 2'b11;
645         shuffle_byte_sel[0] = 2'b10;
646       end
647
648     default: ;
649   endcase
650 end
651
652   ALU_SHUF2, ALU_SHUF: begin
653     unique case (vector_mode_i)
654       VEC_MODE8: begin

```



```

655 shuffle_byte_sel[3] = operand_b_i[25:24];
656 shuffle_byte_sel[2] = operand_b_i[17:16];
657 shuffle_byte_sel[1] = operand_b_i[9:8];
658 shuffle_byte_sel[0] = operand_b_i[1:0];
659 end
660
661 VEC_MODE16: begin
662   shuffle_byte_sel[3] = {operand_b_i[16], 1'b1};
663   shuffle_byte_sel[2] = {operand_b_i[16], 1'b0};
664   shuffle_byte_sel[1] = {operand_b_i[0], 1'b1};
665   shuffle_byte_sel[0] = {operand_b_i[0], 1'b0};
666 end
667 default: ;
668 endcase
669 end
670
671 ALU_INS: begin
672   shuffle_byte_sel[3] = 2'b11;
673   shuffle_byte_sel[2] = 2'b10;
674   shuffle_byte_sel[1] = 2'b01;
675   shuffle_byte_sel[0] = 2'b00;
676 end
677
678 default: ;
679 endcase
680 end
681
682 assign shuffle_r0_in = shuffle_reg0_sel[1] ?
683   operand_a_i :
684   (shuffle_reg0_sel[0] ? {2{oper-
and_a_i[15:0]}} : {4{operand_a_i[7:0]}});
685
686 assign shuffle_r1_in = shuffle_reg1_sel[1] ? {
687   {8{operand_a_i[31]}} , {8{operand_a_i[23]}} , {8{oper-
and_a_i[15]}} , {8{operand_a_i[7]}}
688 } : (shuffle_reg1_sel[0] ? operand_c_i : operand_b_i);
689
690 assign shuffle_r0[31:24] = shuffle_byte_sel[3][1] ?
691   (shuffle_byte_sel[3][0] ?
shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
692   (shuffle_byte_sel[3][0] ?
shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
693 assign shuffle_r0[23:16] = shuffle_byte_sel[2][1] ?
694   (shuffle_byte_sel[2][0] ?
shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
695   (shuffle_byte_sel[2][0] ?
shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
696 assign shuffle_r0[15: 8] = shuffle_byte_sel[1][1] ?
697   (shuffle_byte_sel[1][0] ?
shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
698   (shuffle_byte_sel[1][0] ?
shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
699 assign shuffle_r0[ 7: 0] = shuffle_byte_sel[0][1] ?
700   (shuffle_byte_sel[0][0] ?
shuffle_r0_in[31:24] : shuffle_r0_in[23:16]) :
701   (shuffle_byte_sel[0][0] ?
shuffle_r0_in[15: 8] : shuffle_r0_in[ 7: 0]);
702
703 assign shuffle_r1[31:24] = shuffle_byte_sel[3][1] ?
704   (shuffle_byte_sel[3][0] ?
shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
705   (shuffle_byte_sel[3][0] ?
shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
706 assign shuffle_r1[23:16] = shuffle_byte_sel[2][1] ?
707   (shuffle_byte_sel[2][0] ?
shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
708   (shuffle_byte_sel[2][0] ?
shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
709 assign shuffle_r1[15: 8] = shuffle_byte_sel[1][1] ?
710   (shuffle_byte_sel[1][0] ?
shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
711   (shuffle_byte_sel[1][0] ?
shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
712 assign shuffle_r1[ 7: 0] = shuffle_byte_sel[0][1] ?
713   (shuffle_byte_sel[0][0] ?
shuffle_r1_in[31:24] : shuffle_r1_in[23:16]) :
714   (shuffle_byte_sel[0][0] ?
shuffle_r1_in[15: 8] : shuffle_r1_in[ 7: 0]);
715
716 assign shuffle_result[31:24] = shuffle_reg_sel[3] ?
shuffle_r1[31:24] : shuffle_r0[31:24];
717 assign shuffle_result[23:16] = shuffle_reg_sel[2] ?
shuffle_r1[23:16] : shuffle_r0[23:16];
718 assign shuffle_result[15:8] = shuffle_reg_sel[1] ?
shuffle_r1[15:8] : shuffle_r0[15:8];
719 assign shuffle_result[7:0] = shuffle_reg_sel[0] ?
shuffle_r1[7:0] : shuffle_r0[7:0];
720
721 assign pack_result[31:24] = shuffle_through[3] ?
shuffle_result[31:24] : operand_c_i[31:24];
722 assign pack_result[23:16] = shuffle_through[2] ?
shuffle_result[23:16] : operand_c_i[23:16];
723 assign pack_result[15:8] = shuffle_through[1] ?
shuffle_result[15:8] : operand_c_i[15:8];
724 assign pack_result[7:0] = shuffle_through[0] ?
shuffle_result[7:0] : operand_c_i[7:0];
725
726
727 //////////////////////////////////////
728 //

```

```

729 // | _ ) ( _ | _ / _ | _ _ _ _ | _ / _ \ _ _
730 // | _ \ | _ | | | / _ \ | | | ' _ \ | | | | ' _ \
731 // | | _ | | | | | | ( _ | | | | | | | | | | | |
732 // | _ / | _ \ | _ \ _ \ / _ \ _ | | _ \ | _ \ / | _ /
733 // |
734 //
735 //
736 logic [31:0] ff_input; // either op_a_i or its bit reversed
version
737 logic [ 5:0] cnt_result; // population count
738 logic [ 5:0] clb_result; // count leading bits
739 logic [ 4:0] ff1_result; // holds the index of the first
'1'
740 logic ff_no_one; // if no ones are found
741 logic [ 4:0] fl1_result; // holds the index of the last '1'
742 logic [ 5:0] bitop_result; // result of all bitop opera-
tions muxed together
743
744 cv32e40p_popcnt popcnt_i (
745     .in_i (operand_a_i),
746     .result_o(cnt_result)
747 );
748
749 always_comb begin
750     ff_input = '0;
751
752     case (operator_i)
753         ALU_FF1: ff_input = operand_a_i;
754
755         ALU_DIVU, ALU_REMU, ALU_FL1: ff_input = operand_a_rev;
756
757         ALU_DIV, ALU_REM, ALU_CLB: begin
758             if (operand_a_i[31]) ff_input = operand_a_neg_rev;
759             else ff_input = operand_a_rev;
760         end
761     endcase
762 end
763
764 cv32e40p_ff_one ff_one_i (
765     .in_i (ff_input),
766     .first_one_o(ff1_result),
767     .no_ones_o (ff_no_one)
768 );
769
770 // special case if ff1_res is 0 (no 1 found), then we keep
the 0
771 // this is done in the result mux
772 assign fl1_result = 5'd31 - ff1_result;
773 assign clb_result = ff1_result - 5'd1;
774
775 always_comb begin
776     bitop_result = '0;
777     case (operator_i)
778         ALU_FF1: bitop_result = ff_no_one ? 6'd32 : {1'b0,
ff1_result};
779         ALU_FL1: bitop_result = ff_no_one ? 6'd32 : {1'b0,
fl1_result};
780         ALU_CNT: bitop_result = cnt_result;
781         ALU_CLB: begin
782             if (ff_no_one) begin
783                 if (operand_a_i[31]) bitop_result = 6'd31;
784                 else bitop_result = '0;
785             end else begin
786                 bitop_result = clb_result;
787             end
788         end
789         default: ;
790     endcase
791 end
792
793
794 //
795 // | _ ) ( _ | _ \ _ | _ | _ \ _ | _ | _ \ _ | _ | _ \ _
796 // | | _ | | | | | | | | | | | | | | | | | | | | | |
797 // | | _ | | | | | | | | | | | | | | | | | | | | | |
798 // | | _ | | | | | | | | | | | | | | | | | | | | | |
799 // | | _ | | | | | | | | | | | | | | | | | | | | | |
800 // | | _ | | | | | | | | | | | | | | | | | | | | | |
801 // | | _ | | | | | | | | | | | | | | | | | | | | | |
802 //
803 logic extract_is_signed;
804 logic extract_sign;
805 logic [31:0] bmask_first, bmask_inv;
806 logic [31:0] bextins_and;
807 logic [31:0] bextins_result, bclr_result, bset_result;
808
809
810 // construct bit mask for insert/extract/bclr/bset
811 // bmask looks like this 00..0011..1100..00
812 assign bmask_first = {32'hFFFFFFFFE} << bmask_a_i;
813 assign bmask = (~bmask_first) << bmask_b_i;
814 assign bmask_inv = ~bmask;

```

[illegible]

```

ALU_REMU));
896
897 // inputs A and B are swapped
898 cv32e40p_alu_div alu_div_i (
899     .Clk_CI (clk),
900     .Rst_RBI(rst_n),
901
902     // input IF
903     .OpA_DI      (operand_b_i),
904     .OpB_DI      (shift_left_result),
905     .OpBShift_DI (div_shift),
906     .OpBIsZero_SI(cnt_result == 0)),
907
908     .OpBSign_SI(div_op_a_signed),
909     .OpCode_SI  (operator_i[1:0]),
910
911     .Res_D0(result_div),
912
913     // Hand-Shake
914     .Invld_SI (div_valid),
915     .OutRdy_SI(ex_ready_i),
916     .OutVld_S0(div_ready)
917 );
918
919 ////////////////////////////////////////
920 //                                     //
921 //          |-----|               |-----|               //
922 //          |  I  \_ /  X  -   |  L  |   | M  |   | X  \_/  //
923 //          |  <  ^  ^  ^  ^  |  L  |   | L  |   | >  ^  //
924 //          |  L  ^  ^  ^  ^  |  L  |   | L  |   | >  ^  //
925 //          |-----|               |-----|               //
926 ////////////////////////////////////////
927
928 always_comb begin
929     result_o = '0;
930
931     unique case (operator_i)
932         // Standard Operations
933         ALU_AND: result_o = operand_a_i & operand_b_i;
934         ALU_OR : result_o = operand_a_i | operand_b_i;
935         ALU_XOR: result_o = operand_a_i ^ operand_b_i;
936
937         // Shift Operations
938         ALU_ADD, ALU_ADDR, ALU_ADDU, ALU_ADDUR,
939         ALU_SUB, ALU_SUBR, ALU_SUBU, ALU_SUBUR,
940         ALU_SLL,
941         ALU_SRL, ALU_SRA,
942         ALU_ROR:
943             result_o = shift_result;
944
945         // bit manipulation instructions
946         ALU_BINS, ALU_BEXT, ALU_BXTU: result_o =
bextins_result;
947
948         ALU_BCLR: result_o = bclr_result;
949         ALU_BSET: result_o = bset_result;
950
951         // Bit reverse instruction
952         ALU_BREV: result_o = reverse_result;
953
954         // pack and shuffle operations
955         ALU_SHUF, ALU_SHUF2, ALU_PCKLO, ALU_PCKHI, ALU_EXT,
ALU_EXTS, ALU_INS: result_o = pack_result;
956
957         // Min/Max/Ins
958         ALU_MIN, ALU_MINU, ALU_MAX, ALU_MAXU: result_o = res-
ult_minmax;
959
960         //Abs/Cplxconj , ABS is used to do 0 - A for Cplxconj
961         ALU_ABS: result_o = is_clpx_i ? {adder_result[31:16],
operand_a_i[15:0]} : result_minmax;
962
963         ALU_CLIP, ALU_CLIPU: result_o = clip_result;
964
965         // Comparison Operations
966         ALU_EQ, ALU_NE, ALU_GTU, ALU_GEU, ALU_LTU, ALU_LEU,
ALU_GTS, ALU_GES, ALU_LTS, ALU_LES: begin
967             result_o[31:24] = {8{cmp_result[3]}};
968             result_o[23:16] = {8{cmp_result[2]}};
969             result_o[15:8]  = {8{cmp_result[1]}};
970             result_o[7:0]   = {8{cmp_result[0]}};
971         end
972         // Non-vector comparisons
973         ALU_SLTS, ALU_SLTU, ALU_SLETS, ALU_SLETU: result_o =
{31'b0, comparison_result_o};
974
975         ALU_FF1, ALU_FL1, ALU_CLB, ALU_CNT: result_o = {26'h0,
bitop_result[5:0]};
976
977         // Division Unit Commands
978         ALU_DIV, ALU_DIVU, ALU_REM, ALU_REMU: result_o = res-
ult_div;
979
980         default: ; // default case to suppress unique warning
981     endcase
982 end
983
984 assign ready_o = div_ready;
985
986 endmodule

```

3. openhwgroup_cv32e40p/rtl/cv32e40p_alu_div.sv (226 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 //////////////////////////////////////////////////
12 // File      : Simple Serial Divider
13 // Ver       : 1.0
14 // Date      : 15.03.2016
15 //////////////////////////////////////////////////
16 //
17 // Description: this is a simple serial divider for signed in-
  tegers (int32).
18 //
19 //////////////////////////////////////////////////
20 //
21 // Authors   : Michael Schaffner (schaffner@iis.ee.ethz.ch)
22 //             Andreas Traber    (atraber@iis.ee.ethz.ch)
23 //
24 //////////////////////////////////////////////////
25
26 module cv32e40p_alu_div #(
27     parameter C_WIDTH      = 32,
28     parameter C_LOG_WIDTH  = 6
29 ) (
30     input logic             Clk_CI,
31     input logic             Rst_RBI,
32     // input IF
33     input logic [C_WIDTH-1:0] OpA_DI,
34     input logic [C_WIDTH-1:0] OpB_DI,
35     input logic [C_LOG_WIDTH-1:0] OpBShift_DI,
36     input logic             OpBIsZero_SI,
37     //
38     input logic             OpBSign_SI, // gate this
  to 0 in case of unsigned ops
39     input logic [1:0] OpCode_SI, // 0: udiv, 2:
  urem, 1: div, 3: rem
40     // handshake
41     input logic             InVld_SI,
42     // output IF
43     input logic             OutRdy_SI,
44     output logic            OutVld_SO,
45     output logic [C_WIDTH-1:0] Res_DO
46 );
47
48 //////////////////////////////////////////////////
49 // signal declarations
50 //////////////////////////////////////////////////
51
52 logic [C_WIDTH-1:0] ResReg_DP, ResReg_DN;
53 logic [C_WIDTH-1:0] ResReg_DP_rev;
54 logic [C_WIDTH-1:0] AReg_DP, AReg_DN;
55 logic [C_WIDTH-1:0] BReg_DP, BReg_DN;
56
57 logic RemSel_SN, RemSel_SP;
58 logic CompInv_SN, CompInv_SP;
59 logic ResInv_SN, ResInv_SP;
60
61 logic [C_WIDTH-1:0] AddMux_D;
62 logic [C_WIDTH-1:0] AddOut_D;
63 logic [C_WIDTH-1:0] AddTmp_D;
64 logic [C_WIDTH-1:0] BMux_D;
65 logic [C_WIDTH-1:0] OutMux_D;
66
67 logic [C_LOG_WIDTH-1:0] Cnt_DP, Cnt_DN;
68 logic CntZero_S;
69
70 logic ARegEn_S, BRegEn_S, ResRegEn_S, ABComp_S, PmSel_S,
  LoadEn_S;
71
72 enum logic [1:0] {
73     IDLE,
74     DIVIDE,
75     FINISH
76 }
77     State_SN, State_SP;
78

```

```

79
80 //////////////////////////////////////////////////
81 // datapath
82 //////////////////////////////////////////////////
83
84 assign PmSel_S = LoadEn_S & ~(OpCode_SI[0] &
  (OpA_DI[high(OpA_DI)] ^ OpBSign_SI));
85
86 // muxes
87 assign AddMux_D = (LoadEn_S) ? OpA_DI : BReg_DP;
88
89 // attention: logical shift in case of negative operand B!
90 assign BMux_D = (LoadEn_S) ? OpB_DI : {CompInv_SP,
  (BReg_DP[high(BReg_DP):1])};
91
92 genvar index;
93 generate
94     for (index = 0; index < C_WIDTH; index++) begin :
  gen_bit_swapping
95         assign ResReg_DP_rev[index] = ResReg_DP[C_WIDTH-1-in-
  dex];
96     end
97 endgenerate
98
99 assign OutMux_D = (RemSel_SP) ? AReg_DP : ResReg_DP_rev;
100
101 // invert if necessary
102 assign Res_DO = (ResInv_SP) ? ~$signed(OutMux_D) : OutMux_D;
103
104 // main comparator
105 assign ABComp_S = ((AReg_DP == BReg_DP) | ((AReg_DP >
  BReg_DP) ^ CompInv_SP)) & ((|AReg_DP) | OpBIsZero_SI);
106
107 // main adder
108 assign AddTmp_D = (LoadEn_S) ? 0 : AReg_DP;
109
110 assign AddOut_D = (PmSel_S) ? AddTmp_D + AddMux_D : AddTmp_D -
  $signed(AddMux_D);
111
112 //////////////////////////////////////////////////
113 // counter
114 //////////////////////////////////////////////////
115
116 assign Cnt_DN = (LoadEn_S) ? OpBShift_DI : (~CntZero_S) ?
  Cnt_DP - 1 : Cnt_DP;
117
118 assign CntZero_S = ~(|Cnt_DP);
119
120 // FSM
121 //////////////////////////////////////////////////
122
123 always_comb begin : p_fsm
124     // default
125     State_SN = State_SP;
126
127     OutVld_SO = 1'b0;
128
129     LoadEn_S = 1'b0;
130
131     ARegEn_S = 1'b0;
132     BRegEn_S = 1'b0;
133     ResRegEn_S = 1'b0;
134
135     case (State_SP)
136         //////////////////////////////////////////////////
137         IDLE: begin
138             OutVld_SO = 1'b1;
139
140             if (InVld_SI) begin
141                 OutVld_SO = 1'b0;
142                 ARegEn_S = 1'b1;
143                 BRegEn_S = 1'b1;
144                 LoadEn_S = 1'b1;
145                 State_SN = DIVIDE;
146             end
147         end
148         //////////////////////////////////////////////////
149         DIVIDE: begin
150
151             ARegEn_S = ABComp_S;
152             BRegEn_S = 1'b1;
153             ResRegEn_S = 1'b1;
154
155             // calculation finished
156             // one more divide cycle (32nd divide cycle)
157             if (CntZero_S) begin
158                 State_SN = FINISH;
159             end
160         end
161         //////////////////////////////////////////////////
162         FINISH: begin
163             OutVld_SO = 1'b1;
164
165             if (OutRdy_SI) begin

```

```

166         State_SN = IDLE;
167     end
168 end
169 //////////////////////////////////////////////////
170 default: /* default */;
171 //////////////////////////////////////////////////
172 endcase
173 end
174
175 //////////////////////////////////////////////////
176 //////////////////////////////////////////////////
177 // regs
178 //////////////////////////////////////////////////
179 //////////////////////////////////////////////////
180 // get flags
181 assign RemSel_SN = (LoadEn_S) ? OpCode_SI[1] : RemSel_SP;
182 assign CompInv_SN = (LoadEn_S) ? OpBSign_SI : CompInv_SP;
183 assign ResInv_SN = (LoadEn_S) ? (~OpBisZero_SI | Op-
Code_SI[1]) & OpCode_SI[0] & (OpA_DI[Shigh(
184     OpA_DI
185 )) ^ OpBSign_SI : ResInv_SP;
186
187 assign AReg_DN = (ARegEn_S) ? AddOut_D : AReg_DP;
188 assign BReg_DN = (BRegEn_S) ? BMux_D : BReg_DP;
189 assign ResReg_DN = (LoadEn_S) ? '0 : (ResRegEn_S) ? {
190     ABComp_S, ResReg_DP[Shigh(ResReg_DP):1]
191 } : ResReg_DP;
192
193 always_ff @(posedge Clk_CI or negedge Rst_RBI) begin :
p_regs
194     if (~Rst_RBI) begin
195         State_SP <= IDLE;
196         AReg_DP <= '0;
197         BReg_DP <= '0;
198         ResReg_DP <= '0;
199         Cnt_DP <= '0;
200         RemSel_SP <= '1'b0;
201         CompInv_SP <= '1'b0;
202         ResInv_SP <= '1'b0;
203     end else begin
204         State_SP <= State_SN;
205         AReg_DP <= AReg_DN;
206         BReg_DP <= BReg_DN;
207         ResReg_DP <= ResReg_DN;
208         Cnt_DP <= Cnt_DN;
209         RemSel_SP <= RemSel_SN;
210         CompInv_SP <= CompInv_SN;
211         ResInv_SP <= ResInv_SN;
212     end
213 end
214
215 //////////////////////////////////////////////////
216 // assertions
217 //////////////////////////////////////////////////
218
219 `ifdef CV32E40P_ASSERT_ON
220     initial begin : p_assertions
221         assert (C_LOG_WIDTH == $clog2(C_WIDTH + 1))
222         else $error("C_LOG_WIDTH must be $clog2(C_WIDTH+1)");
223     end
224 `endif
225
226 endmodule // serDiv

```

4. openhwgroup_cv32e40p/rtl/cv32e40p_apu_disp.sv (242 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 //////////////////////////////////////////////////
12 // Engineer:         Lukas Mueller - lukas-
mue@student.ethz.ch
13 //
14 // Additional contributions
15 //
by:
16 //
Michael Gautschi - gauts-

```

```

chi@iis.ee.ethz.ch //
16 //
//
17 // Design Name:      Simple APU dispatch-
er
18 // Project Name:
PULP
19 // Language:         SystemVeri-
log
20 //
//
21 // Description:      Dispatcher for sending instructions to the
Marx
22 //
intercon-
nect.
23 //////////////////////////////////////////////////
24
25 module cv32e40p_apu_disp (
26     input logic clk_i,
27     input logic rst_ni,
28
29     // request input
30     input logic enable_i,
31     input logic [1:0] apu_lat_i,
32     input logic [5:0] apu_waddr_i,
33
34     // response output
35     output logic [5:0] apu_waddr_o,
36     output logic apu_multicycle_o,
37     output logic apu_singlecycle_o,
38
39     // active signal
40     output logic active_o,
41
42     // stall signals
43     output logic stall_o,
44
45     // dependency checks
46     input logic is_decoding_i,
47     input logic [2:0][5:0] read_regs_i,
48     input logic [2:0] read_regs_valid_i,
49     output logic read_dep_o,
50     output logic read_dep_for_jalr_o,
51
52     input logic [1:0][5:0] write_regs_i,
53     input logic [1:0] write_regs_valid_i,
54     output logic write_dep_o,
55
56     // perf counter stuff
57     output logic perf_type_o,
58     output logic perf_cont_o,
59
60     // apu-interconnect
61     // handshake signals
62     output logic apu_req_o,
63     input logic apu_gnt_i,
64     // response channel
65     input logic apu_rvalid_i
66 );
67
68
69 logic [5:0] addr_req, addr_inflight, addr_waiting;
70 logic [5:0] addr_inflight_dn, addr_waiting_dn;
71 logic valid_req, valid_inflight, valid_waiting;
72 logic valid_inflight_dn, valid_waiting_dn;
73 logic returned_req, returned_inflight, returned_waiting;
74
75 logic req_accepted;
76 logic active;
77 logic [1:0] apu_lat;
78
79
80 logic [2:0] read_deps_req, read_deps_inflight,
read_deps_waiting;
81 logic [1:0] write_deps_req, write_deps_inflight,
write_deps_waiting;
82 logic read_dep_req, read_dep_inflight, read_dep_waiting;
83 logic write_dep_req, write_dep_inflight, write_dep_waiting;
84
85 logic stall_full, stall_type, stall_nack;
86
87 // Generate request signal; do not generate request if
stalled unless it's a nack stall
88 assign valid_req = enable_i & !(stall_full |
stall_type);
89 assign addr_req = apu_waddr_i;
90
91 assign req_accepted = valid_req & apu_gnt_i;
92
93 //
94 // In-flight instructions
95 //
96 // Check whether the instructions have returned
97 assign returned_req = valid_req & apu_rvalid_i & !val-
id_inflight & !valid_waiting;
98 assign returned_inflight = valid_inflight & (apu_rvalid_i)
& !valid_waiting;
99 assign returned_waiting = valid_waiting & (apu_rvalid_i);

```

```

100
101 // Inflight and waiting registers
102 always_ff @(posedge clk_i or negedge rst_ni) begin
103     if (~rst_ni) begin
104         valid_inflight <= 1'b0;
105         valid_waiting <= 1'b0;
106         addr_inflight <= '0;
107         addr_waiting <= '0;
108     end else begin
109         valid_inflight <= valid_inflight_dn;
110         valid_waiting <= valid_waiting_dn;
111         addr_inflight <= addr_inflight_dn;
112         addr_waiting <= addr_waiting_dn;
113     end
114 end
115
116 always_comb begin
117     valid_inflight_dn = valid_inflight;
118     valid_waiting_dn = valid_waiting;
119     addr_inflight_dn = addr_inflight;
120     addr_waiting_dn = addr_waiting;
121
122     if (req_accepted & !returned_req) begin // this is a mul-
ticycle request
123         valid_inflight_dn = 1'b1;
124         addr_inflight_dn = addr_req;
125         if (valid_inflight & !(returned_inflight)) begin // we
already have an inflight instruction!
126             valid_waiting_dn = 1'b1;
127             addr_waiting_dn = addr_inflight;
128         end
129         if (returned_waiting) begin // we have received a new
request and waiting goes out of the pipe but will be refilled
130             valid_waiting_dn = 1'b1;
131             addr_waiting_dn = addr_inflight;
132         end
133     end // no new request
134     else if (returned_inflight) begin // multicycle request
has returned
135         valid_inflight_dn = '0;
136         valid_waiting_dn = '0;
137         addr_inflight_dn = '0;
138         addr_waiting_dn = '0;
139     end else if (returned_waiting) begin // multicycle re-
quest has returned
140         valid_waiting_dn = '0;
141         addr_waiting_dn = '0;
142     end
143 end
144
145 //
146 // Active type
147 //
148 // Dispatcher is active when there is an unreturned instruc-
tion
149 assign active = valid_inflight | valid_waiting;
150
151 // Store the latency type whenever there is a request
152 always_ff @(posedge clk_i or negedge rst_ni) begin
153     if (~rst_ni) begin
154         apu_lat <= '0;
155     end else begin
156         if (valid_req) begin
157             apu_lat <= apu_lat_i;
158         end
159     end
160 end
161
162 //
163 // Dependency checks
164 //
165 // There is a dependency if the register is equal to one of
the instructions
166 generate
167     for (genvar i = 0; i < 3; i++) begin : gen_read_deps
168         assign read_deps_req[i] = (read_regs_i[i] ==
addr_req) & read_regs_valid_i[i];
169         assign read_deps_inflight[i] = (read_regs_i[i] ==
addr_inflight) & read_regs_valid_i[i];
170         assign read_deps_waiting[i] = (read_regs_i[i] ==
addr_waiting) & read_regs_valid_i[i];
171     end
172 endgenerate
173
174 generate
175     for (genvar i = 0; i < 2; i++) begin : gen_write_deps
176         assign write_deps_req[i] = (write_regs_i[i] ==
addr_req) & write_regs_valid_i[i];
177         assign write_deps_inflight[i] = (write_regs_i[i] ==
addr_inflight) & write_regs_valid_i[i];
178         assign write_deps_waiting[i] = (write_regs_i[i] ==
addr_waiting) & write_regs_valid_i[i];
179     end
180 endgenerate
181
182 // Reduce the individual dependency signals into one read
and one write dependency
183 assign read_dep_req = |read_deps_req & valid_req & !re-
turned_req;
184 assign read_dep_inflight = |read_deps_inflight & val-
id_inflight & !returned_inflight;

```

```

185 assign read_dep_waiting = |read_deps_waiting & valid_waiting
& !returned_waiting;
186 assign write_dep_req = |write_deps_req & valid_req & !re-
turned_req;
187 assign write_dep_inflight = |write_deps_inflight & val-
id_inflight & !returned_inflight;
188 assign write_dep_waiting = |write_deps_waiting & val-
id_waiting & !returned_waiting;
189
190 assign read_dep_o = (read_dep_req | read_dep_inflight |
read_dep_waiting) & is_decoding_i;
191 assign write_dep_o = (write_dep_req | write_dep_inflight |
write_dep_waiting) & is_decoding_i;
192
193 assign read_dep_for_jalr_o = is_decoding_i & ((|
read_deps_req & enable_i) |
194
195 read_deps_inflight & valid_inflight) |
196
197 read_deps_waiting & valid_waiting));
198
199 // Stall signals
200 // Stall if we cannot store any more outstanding requests
201 assign stall_full = valid_inflight & valid_waiting;
202 // Stall if there is a type conflict. If apu is active we
can only issue requests with a larger or equal latency
203 // than the latency of the inflight operation
(apu_lat_i >= apu_lat). otherwise operations would overtake each oth-
er!
204 // so we stall if: (apu_lat_i = 1 & apu_lat = 2/3) |
(apu_lat_i = 2 & apu_lat = 3) | (apu_lat_i = 3 (multicycle))
205 assign stall_type = enable_i & active &
((apu_lat_i == 2'h1) | ((apu_lat_i == 2'h2) & (apu_lat == 2'h3)) |
(apu_lat_i == 2'h3));
206 assign stall_nack = valid_req & !apu_gnt_i;
207 assign stall_o = stall_full | stall_type | stall_nack;
208
209 //
210 // Generate Apu_master request
211 //
212 assign apu_req_o = valid_req;
213
214 // Determine write register based on where the instruction
returned.
215 always_comb begin
216     apu_waddr_o = '0;
217     if (returned_req) apu_waddr_o = addr_req;
218     if (returned_inflight) apu_waddr_o = addr_inflight;
219     if (returned_waiting) apu_waddr_o = addr_waiting;
220 end
221
222 // Output active signal
223 assign active_o = active;
224
225 // Performance counter signals
226 assign perf_type_o = stall_type;
227 assign perf_cont_o = stall_nack;
228
229 assign apu_multicycle_o = (apu_lat == 2'h3);
230 assign apu_singlecycle_o = ~(valid_inflight | val-
id_waiting);
231
232 //
233 // Assertions
234 //
235 //
236
237 `ifdef CV32E40P_ASSERT_ON
238     assert property (@(posedge clk_i) (apu_rvalid_i) |-> (val-
id_req | valid_inflight | valid_waiting))
239     else $warning("APU Dispatcher] instruction returned while
no instruction is in-flight");
240 `endif
241
242 endmodule

```

5.

openhwgroup_cv32e40p/rtl/
cv32e40p_compressed_decoder.sv (601 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.

```

[illegible]

```

77 // instr_i[12:10] -> offset[5:3], instr_i[6:5] ->
offset[7:6]
78     instr_o = {
79         4'b0,
80         instr_i[6:5],
81         instr_i[12:10],
82         3'b000,
83         2'b01,
84         instr_i[9:7],
85         3'b011,
86         2'b01,
87         instr_i[4:2],
88         OPCODE_LOAD_FP
89     };
90     else illegal_instr_o = 1'b1;
91 end
92
93 3'b010: begin
94     // c.lw -> lw rd', imm(rs1')
95     instr_o = {
96         5'b0,
97         instr_i[5],
98         instr_i[12:10],
99         instr_i[6],
100        2'b00,
101        2'b01,
102        instr_i[9:7],
103        3'b010,
104        2'b01,
105        instr_i[4:2],
106        OPCODE_LOAD
107    };
108 end
109
110 3'b011: begin
111     // c.flw -> flw rd', imm(rs1')
112     if (FPU == 1 && ZFINX == 0)
113         instr_o = {
114             5'b0,
115             instr_i[5],
116             instr_i[12:10],
117             instr_i[6],
118             2'b00,
119             2'b01,
120             instr_i[9:7],
121             3'b010,
122             2'b01,
123             instr_i[4:2],
124             OPCODE_LOAD_FP
125         };
126         else illegal_instr_o = 1'b1;
127     end
128
129 3'b101: begin
130     // c.fsd -> fsd rs2', imm(rs1')
131     if (FPU == 1 && ZFINX == 0)
132         // instr_i[12:10] -> offset[5:3], instr_i[6:5] -
> offset[7:6]
133         instr_o = {
134             4'b0,
135             instr_i[6:5],
136             instr_i[12],
137             2'b01,
138             instr_i[4:2],
139             2'b01,
140             instr_i[9:7],
141             3'b011,
142             instr_i[11:10],
143             3'b000,
144             OPCODE_STORE_FP
145         };
146         else illegal_instr_o = 1'b1;
147     end
148
149 3'b110: begin
150     // c.sw -> sw rs2', imm(rs1')
151     instr_o = {
152         5'b0,
153         instr_i[5],
154         instr_i[12],
155         2'b01,
156         instr_i[4:2],
157         2'b01,
158         instr_i[9:7],
159         3'b010,
160         instr_i[11:10],
161         instr_i[6],
162         2'b00,
163         OPCODE_STORE
164     };
165 end
166
167 3'b111: begin
168     // c.fsw -> fsw rs2', imm(rs1')
169     if (FPU == 1 && ZFINX == 0)
170         instr_o = {
171             5'b0,
172             instr_i[5],
173             instr_i[12],
174             2'b01,
175             instr_i[4:2].

```



```

176         2'b01,
177         instr_i[9:7],
178         3'b010,
179         instr_i[11:10],
180         instr_i[6],
181         2'b00,
182         OPCODE_STORE_FP
183     };
184     else illegal_instr_o = 1'b1;
185 end
186 default: begin
187     illegal_instr_o = 1'b1;
188 end
189 endcase
190 end
191
192 // C1
193 2'b01: begin
194     unique case (instr_i[15:13])
195     3'b000: begin
196         // c.addi -> addi rd, rd, nzimm
197         // c.nop
198         instr_o = {
199             {6{instr_i[12]}},
200             instr_i[12],
201             instr_i[6:2],
202             instr_i[11:7],
203             3'b0,
204             instr_i[11:7],
205             OPCODE_OPIMM
206         };
207     end
208
209     3'b001, 3'b101: begin
210         // 001: c.jal -> jal x1, imm
211         // 101: c.j -> jal x0, imm
212         instr_o = {
213             instr_i[12],
214             instr_i[8],
215             instr_i[10:9],
216             instr_i[6],
217             instr_i[7],
218             instr_i[2],
219             instr_i[11],
220             instr_i[5:3],
221             {9{instr_i[12]}},
222             4'b0,
223             ~instr_i[15],
224             OPCODE_JAL
225         };
226     end
227
228     3'b010: begin
229         if (instr_i[11:7] == 5'b0) begin
230             // Hint -> addi x0, x0, nzimm
231             instr_o = {
232                 {6{instr_i[12]}}, instr_i[12], instr_i[6:2],
233                 5'b0, 3'b0, instr_i[11:7], OPCODE_OPIMM
234             };
235         end else begin
236             // c.li -> addi rd, x0, nzimm
237             instr_o = {
238                 {6{instr_i[12]}}, instr_i[12], instr_i[6:2],
239                 5'b0, 3'b0, instr_i[11:7], OPCODE_OPIMM
240             };
241         end
242
243     3'b011: begin
244         if ({instr_i[12], instr_i[6:2]} == 6'b0) begin
245             illegal_instr_o = 1'b1;
246         end else begin
247             if (instr_i[11:7] == 5'h02) begin
248                 // c.addi16sp -> addi x2, x2, nzimm
249                 instr_o = {
250                     {3{instr_i[12]}},
251                     instr_i[4:3],
252                     instr_i[5],
253                     instr_i[2],
254                     instr_i[6],
255                     4'b0,
256                     5'h02,
257                     3'b000,
258                     5'h02,
259                     OPCODE_OPIMM
260                 };
261             end else if (instr_i[11:7] == 5'b0) begin
262                 // Hint -> lui x0, imm
263                 instr_o = {{15{instr_i[12]}}, instr_i[6:2],
264                     instr_i[11:7], OPCODE_LUI};
265             end else begin
266                 // c.lui -> lui rd, imm
267                 instr_o = {{15{instr_i[12]}}, instr_i[6:2],
268                     instr_i[11:7], OPCODE_LUI};
269             end
270         end
271
272     3'b100: begin
273         unique case (instr_i[11:10])

```

```

273     2'b00, 2'b01: begin
274         // 00: c.srli -> srli rd, rd, shamt
275         // 01: c.srai -> srai rd, rd, shamt
276         if (instr_i[12] == 1'b1) begin
277             // Reserved for future custom extensions
278             (instr_o don't care)
279         end
280
281         instr_o = {
282             1'b0,
283             instr_i[10],
284             5'b0,
285             instr_i[6:2],
286             2'b01,
287             instr_i[9:7],
288             3'b101,
289             2'b01,
290             instr_i[9:7],
291             OPCODE_OPIMM
292         };
293     end else begin
294         if (instr_i[6:2] == 5'b0) begin
295             // Hint
296             instr_o = {
297                 1'b0,
298                 instr_i[10],
299                 5'b0,
300                 instr_i[6:2],
301                 2'b01,
302                 instr_i[9:7],
303                 3'b101,
304                 2'b01,
305                 instr_i[9:7],
306                 OPCODE_OPIMM
307             };
308         end else begin
309             instr_o = {
310                 1'b0,
311                 instr_i[10],
312                 5'b0,
313                 instr_i[6:2],
314                 2'b01,
315                 instr_i[9:7],
316                 3'b101,
317                 2'b01,
318                 instr_i[9:7],
319                 OPCODE_OPIMM
320             };
321         end
322     end
323
324     2'b10: begin
325         // c.andi -> andi rd, rd, imm
326         instr_o = {
327             {6{instr_i[12]}},
328             instr_i[12],
329             instr_i[6:2],
330             2'b01,
331             instr_i[9:7],
332             3'b111,
333             2'b01,
334             instr_i[9:7],
335             OPCODE_OPIMM
336         };
337     end
338
339     2'b11: begin
340         unique case ({
341             instr_i[12], instr_i[6:5]
342         })
343         3'b000: begin
344             // c.sub -> sub rd', rd', rs2'
345             instr_o = {
346                 2'b01,
347                 5'b0,
348                 2'b01,
349                 instr_i[4:2],
350                 2'b01,
351                 instr_i[9:7],
352                 3'b000,
353                 2'b01,
354                 instr_i[9:7],
355                 OPCODE_OP
356             };
357         end
358
359         3'b001: begin
360             // c.xor -> xor rd', rd', rs2'
361             instr_o = {
362                 7'b0,
363                 2'b01,
364                 instr_i[4:2],
365                 2'b01,
366                 instr_i[9:7],
367                 3'b100,
368                 2'b01,
369                 instr_i[9:7],
370                 OPCODE_OP
371             };
372         end

```



```

373         3'b010: begin
374             // c.or -> or rd', rd', rs2'
375             instr_o = {
376                 7'b0,
377                 2'b01,
378                 instr_i[4:2],
379                 2'b01,
380                 instr_i[9:7],
381                 3'b110,
382                 2'b01,
383                 instr_i[9:7],
384                 OPCODE_OP
385             };
386         end
387     end
388     3'b011: begin
389         // c.and -> and rd', rd', rs2'
390         instr_o = {
391             7'b0,
392             2'b01,
393             instr_i[4:2],
394             2'b01,
395             instr_i[9:7],
396             3'b111,
397             2'b01,
398             instr_i[9:7],
399             OPCODE_OP
400         };
401     end
402     3'b100, 3'b101, 3'b110, 3'b111: begin
403         // 100: c.subw
404         // 101: c.addw
405         illegal_instr_o = 1'b1;
406     end
407 endcase
408 end
409 endcase
410 end
411 3'b110, 3'b111: begin
412     // 0: c.beqz -> beq rs1', x0, imm
413     // 1: c.bnez -> bne rs1', x0, imm
414     instr_o = {
415         {4{instr_i[12]}},
416         instr_i[6:5],
417         instr_i[2],
418         5'b0,
419         2'b01,
420         instr_i[9:7],
421         2'b00,
422         instr_i[13],
423         instr_i[11:10],
424         instr_i[4:3],
425         instr_i[12],
426         OPCODE_BRANCH
427     };
428 end
429 endcase
430 end
431 // C2
432 2'b10: begin
433     unique case (instr_i[15:13])
434     3'b000: begin
435         if (instr_i[12] == 1'b1) begin
436             // Reserved for future extensions (instr_o don't
437             care)
438             instr_o = {7'b0, instr_i[6:2], instr_i[11:7],
439             3'b001, instr_i[11:7], OPCODE_OPIMM};
440             illegal_instr_o = 1'b1;
441         end else begin
442             if ((instr_i[6:2] == 5'b0) || (instr_i[11:7] ==
443             5'b0)) begin
444                 // Hint -> slli rd, rd, shamt
445                 instr_o = {7'b0, instr_i[6:2], instr_i[11:7],
446                 3'b001, instr_i[11:7], OPCODE_OPIMM};
447             end else begin
448                 // c.slli -> slli rd, rd, shamt
449                 instr_o = {7'b0, instr_i[6:2], instr_i[11:7],
450                 3'b001, instr_i[11:7], OPCODE_OPIMM};
451             end
452         end
453     end
454     3'b001: begin
455         // c.fldsp -> fld rd, imm(x2)
456         if (FPU == 1 && ZFINX == 0)
457             // instr_i[6:5] -> offset[4:3], instr_i[4:2] ->
458             offset[8:6], instr_i[12] -> offset[5]
459             instr_o = {
460                 3'b0,
461                 instr_i[4:2],
462                 instr_i[12],
463                 instr_i[6:5],
464                 3'b000,
465                 5'h02,
466                 3'b011,
467                 instr_i[11:7],
468                 OPCODE_LOAD_FP

```

```

467             };
468         else illegal_instr_o = 1'b1;
469     end
470     3'b010: begin
471         // c.lwsp -> lw rd, imm(x2)
472         instr_o = {
473             4'b0,
474             instr_i[3:2],
475             instr_i[12],
476             instr_i[6:4],
477             2'b00,
478             5'h02,
479             3'b010,
480             instr_i[11:7],
481             OPCODE_LOAD
482         };
483     end
484     if (instr_i[11:7] == 5'b0) illegal_instr_o = 1'b1;
485 end
486     3'b011: begin
487         // c.flwsp -> flw rd, imm(x2)
488         if (FPU == 1 && ZFINX == 0)
489             instr_o = {
490                 4'b0,
491                 instr_i[3:2],
492                 instr_i[12],
493                 instr_i[6:4],
494                 2'b00,
495                 5'h02,
496                 3'b010,
497                 instr_i[11:7],
498                 OPCODE_LOAD_FP
499             };
500         else illegal_instr_o = 1'b1;
501     end
502     3'b100: begin
503         if (instr_i[12] == 1'b0) begin
504             if (instr_i[6:2] == 5'b0) begin
505                 // c.jr -> jalr x0, rd/rs1, 0
506                 instr_o = {12'b0, instr_i[11:7], 3'b0, 5'b0,
507                 OPCODE_JALR};
508                 // c.jr with rs1 = 0 is reserved
509                 if (instr_i[11:7] == 5'b0) illegal_instr_o =
510                 1'b1;
511             end else begin
512                 if (instr_i[11:7] == 5'b0) begin
513                     // Hint -> add x0, x0, rs2
514                     instr_o = {7'b0, instr_i[6:2], 5'b0, 3'b0,
515                     instr_i[11:7], OPCODE_OP};
516                 end else begin
517                     // c.mv -> add rd, x0, rs2
518                     instr_o = {7'b0, instr_i[6:2], 5'b0, 3'b0,
519                     instr_i[11:7], OPCODE_OP};
520                 end
521             end else begin
522                 if (instr_i[6:2] == 5'b0) begin
523                     if (instr_i[11:7] == 5'b0) begin
524                         // c.ebreak -> ebreak
525                         instr_o = {32'h00_10_00_73};
526                     end else begin
527                         // c.jalr -> jalr x1, rs1, 0
528                         instr_o = {12'b0, instr_i[11:7], 3'b000,
529                         5'b00001, OPCODE_JALR};
530                     end
531                 end else begin
532                     if (instr_i[11:7] == 5'b0) begin
533                         // Hint -> add x0, x0, rs2
534                         instr_o = {7'b0, instr_i[6:2], in-
535                         str_i[11:7], 3'b0, instr_i[11:7], OPCODE_OP};
536                     end else begin
537                         // c.add -> add rd, rd, rs2
538                         instr_o = {7'b0, instr_i[6:2], in-
539                         str_i[11:7], 3'b0, instr_i[11:7], OPCODE_OP};
540                     end
541                 end
542             end
543         end
544     3'b101: begin
545         // c.fsdsp -> fsd rs2, imm(x2)
546         if (FPU == 1 && ZFINX == 0)
547             // instr_i[12:10] -> offset[5:3], instr_i[9:7] -
548             > offset[8:6]
549             instr_o = {
550                 3'b0,
551                 instr_i[9:7],
552                 instr_i[12],
553                 instr_i[6:2],
554                 5'h02,
555                 3'b011,
556                 instr_i[11:10],
557                 3'b000,
558                 OPCODE_STORE_FP
559             };
560         else illegal_instr_o = 1'b1;
561     end
562     3'b110: begin
563         // c.swsp -> sw rs2, imm(x2)

```

```

560     instr_o = {
561         4'b0,
562         instr_i[8:7],
563         instr_i[12],
564         instr_i[6:2],
565         5'h02,
566         3'b010,
567         instr_i[11:9],
568         2'b00,
569         OPCODE_STORE
570     };
571 end
572
573 3'b111: begin
574     // c.fswsp -> fsw rs2, imm(x2)
575     if (FPU == 1 && ZFINX == 0)
576         instr_o = {
577             4'b0,
578             instr_i[8:7],
579             instr_i[12],
580             instr_i[6:2],
581             5'h02,
582             3'b010,
583             instr_i[11:9],
584             2'b00,
585             OPCODE_STORE_FP
586         };
587     else illegal_instr_o = 1'b1;
588 end
589 endcase
590 end
591
592 default: begin
593     // 32 bit (or more) instruction
594     instr_o = instr_i;
595 end
596 endcase
597 end
598
599 assign is_compressed_o = (instr_i[1:0] != 2'b11);
600
601 endmodule

```

6. openhwgroup_cv32e40p/rtl/cv32e40p_controller.sv (1,623 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
  pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
  file except in
4 // compliance with the License. You may obtain a copy of the
  License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
  applicable law
6 // or agreed to in writing, software, hardware and materials
  distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
  WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
  License for the
9 // specific language governing permissions and limitations un-
  der the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // Engineer:      Matthias Baer - baer-
  matt@student.ethz.ch
13 //
14 //
15 // Additional contributions
  by:
16 //      Igor Loi -
  igor.loi@unibo.it
17 //      Andreas Traber - at-
  raber@student.ethz.ch
18 //      Sven Stucki - svs-
  tucki@student.ethz.ch
19 //      Michael Gautschi - gauts-
  chi@iis.ee.ethz.ch
20 //      Davide Schiavone - ps-
  chiamo@iis.ee.ethz.ch
21 //      Robert Balas -
  balasr@iis.ee.ethz.ch
22 //      Andrea Bettati - an-
  drea.bettati@studenti.unipr.it
23 //
24 //
25 // Design Name:    Main control-
  ler
26 // Project Name:   RISCY
27 // Language:       SystemVeri-
  log
28 //

```

```

27 // Description:    Main CPU controller of the pro-
  cessor
28 //
29 ///////////////////////////////////////////////////////////////////
30
31 module cv32e40p_controller import cv32e40p_pkg::*;
32 #(
33     parameter COREV_CLUSTER = 0,
34     parameter COREV_PULP    = 0,
35     parameter FPU            = 0
36 )
37 (
38     input logic      clk,                // Gated
39     input logic      clk_ungated_i,      // Ungated
40     input logic      rst_n,
41
42     input logic      fetch_enable_i,      // Start the
  decoding
43     output logic     ctrl_busy_o,         // Core is
  busy processing instructions
44     output logic     is_decoding_o,       // Core is
  in decoding state
45     input logic      is_fetch_failed_i,
46
47     // decoder related signals
48     output logic     deassert_we_o,      // deassert
  write enable for next instruction
49
50     input logic      illegal_insn_i,      // decoder
  encountered an invalid instruction
51     input logic      ecall_insn_i,       // decoder
  encountered an ecall instruction
52     input logic      mret_insn_i,        // decoder
  encountered an mret instruction
53     input logic      uret_insn_i,        // decoder
  encountered an uret instruction
54
55     input logic      dret_insn_i,         // decoder
  encountered an dret instruction
56
57     input logic      mret_dec_i,
58     input logic      uret_dec_i,
59     input logic      dret_dec_i,
60
61     input logic      wfi_i,              // decoder
  wants to execute a WFI
62     input logic      ebrk_insn_i,        // decoder
  encountered an ebreak instruction
63     input logic      fencei_insn_i,      // decoder
  encountered an fence.i instruction
64     input logic      csr_status_i,       // decoder
  encountered an csr status instruction
65
66     output logic     hwlp_mask_o,         // prevent
  writes on the hwloop instructions in case interrupt are taken
67
68     // from IF/ID pipeline
69     input logic      instr_valid_i,       // instruc-
  tion coming from IF/ID pipeline is valid
70
71     // from prefetcher
72     output logic     instr_req_o,         // Start
  fetching instructions
73
74     // to prefetcher
75     output logic     pc_set_o,            // jump to
  address set by pc_mux
76     output logic [3:0] pc_mux_o,         // Selector
  in the Fetch stage to select the right PC (normal, jump ...)
77     output logic [2:0] exc_pc_mux_o,     // Selects
  target PC for exception
78     output logic [1:0] trap_addr_mux_o,  // Selects
  trap address base
79
80     // HWLoop signals
81     input logic [31:0] pc_id_i,
82
83     // from hwloop_regs
84     input logic [1:0] [31:0] hwlp_start_addr_i,
85     input logic [1:0] [31:0] hwlp_end_addr_i,
86     input logic [1:0] [31:0] hwlp_counter_i,
87
88     // to hwloop_regs
89     output logic [1:0] hwlp_dec_cnt_o,
90
91     output logic     hwlp_jump_o,
92     output logic [31:0] hwlp_targ_addr_o,
93
94     // LSU
95     input logic      data_req_ex_i,      // data
  memory access is currently performed in EX stage
96     input logic      data_we_ex_i,
97     input logic      data_misaligned_i,
98     input logic      data_load_event_i,
99     input logic      data_err_i,

```

```

100 output logic data_err_ack_o,
101
102 // from ALU
103 input logic mult_multicycle_i, // multiplier is taken multiple cycles and uses op c as storage
104
105 // APU dependency checks
106 input logic apu_en_i,
107 input logic apu_read_dep_i,
108 input logic apu_read_dep_for_jalr_i,
109 input logic apu_write_dep_i,
110
111 output logic apu_stall_o,
112
113 // jump/branch signals
114 input logic branch_taken_ex_i, // branch taken signal from EX ALU
115 input logic [1:0] ctrl_trans- // jump is being calculated in ALU
fer_insn_in_id_i,
116 input logic [1:0] ctrl_trans- // jump is being calculated in ALU
fer_insn_in_dec_i,
117
118 // Interrupt Controller Signals
119 input logic irq_req_ctrl_i,
120 input logic irq_sec_ctrl_i,
121 input logic [4:0] irq_id_ctrl_i,
122 input logic irq_wu_ctrl_i,
123 input PrivLvl_t current_priv_lvl_i,
124
125 output logic irq_ack_o,
126 output logic [4:0] irq_id_o,
127
128 output logic [4:0] exc_cause_o,
129
130 // Debug Signal
131 output logic debug_mode_o,
132 output logic [2:0] debug_cause_o,
133 output logic debug_csr_save_o,
134 input logic debug_req_i,
135 input logic debug_single_step_i,
136 input logic debug_ebreakm_i,
137 input logic debug_ebreaku_i,
138 input logic trigger_match_i,
139 output logic debug_p_elw_no_sleep_o,
140 output logic debug_wfi_no_sleep_o,
141 output logic debug_havereset_o,
142 output logic debug_running_o,
143 output logic debug_halted_o,
144
145 // Wakeup Signal
146 output logic wake_from_sleep_o,
147
148 output logic csr_save_if_o,
149 output logic csr_save_id_o,
150 output logic csr_save_ex_o,
151 output logic [5:0] csr_cause_o,
152 output logic csr_irq_sec_o,
153 output logic csr_restore_mret_id_o,
154 output logic csr_restore_uret_id_o,
155
156 output logic csr_restore_dret_id_o,
157
158 output logic csr_save_cause_o,
159
160
161 // Regfile target
162 input logic regfile_we_id_i, // currently decoded we enable
163 input logic [5:0] regfile_alu_waddr_id_i, // currently decoded target address
164
165 // Forwarding signals from regfile
166 input logic regfile_we_ex_i, // FW: write enable from EX stage
167 input logic [5:0] regfile_waddr_ex_i, // FW: write address from EX stage
168 input logic regfile_we_wb_i, // FW: write enable from WB stage
169 input logic regfile_alu_we_fw_i, // FW: ALU/MUL write enable from EX stage
170
171 // forwarding signals
172 output logic [1:0] operand_a_fw_mux_sel_o, // regfile ra data selector form ID stage
173 output logic [1:0] operand_b_fw_mux_sel_o, // regfile rb data selector form ID stage
174 output logic [1:0] operand_c_fw_mux_sel_o, // regfile rc data selector form ID stage
175
176 // forwarding detection signals
177 input logic reg_d_ex_is_reg_a_i,
178 input logic reg_d_ex_is_reg_b_i,
179 input logic reg_d_ex_is_reg_c_i,
180 input logic reg_d_wb_is_reg_a_i,
181 input logic reg_d_wb_is_reg_b_i,
182 input logic reg_d_wb_is_reg_c_i,

```

```

183 input logic reg_d_alu_is_reg_a_i,
184 input logic reg_d_alu_is_reg_b_i,
185 input logic reg_d_alu_is_reg_c_i,
186
187 // stall signals
188 output logic halt_if_o,
189 output logic halt_id_o,
190
191 output logic misaligned_stall_o,
192 output logic jr_stall_o,
193 output logic load_stall_o,
194
195 input logic id_ready_i, // ID stage
196 input logic id_valid_i, // ID stage
197
198 input logic ex_valid_i, // EX stage
199
200 input logic wb_ready_i, // WB stage
201
202 // Performance Counters
203
204 output logic perf_pipeline_stall_o // stall due
205 to cv.elw extra cycles
206 );
207
208 // FSM state encoding
209 ctrl_state_e ctrl_fsm_cs, ctrl_fsm_ns;
210
211 // Debug state
212 debug_state_e debug_fsm_cs, debug_fsm_ns;
213
214 logic jump_done, jump_done_q, jump_in_dec, branch_in_id;
215
216 logic data_err_q;
217
218 logic debug_mode_q, debug_mode_n;
219 logic ebrk_force_debug_mode;
220 logic is_hwlp_body;
221 logic illegal_insn_q, illegal_insn_n;
222 logic debug_req_entry_q, debug_req_entry_n;
223 logic debug_force_wakeup_q, debug_force_wakeup_n;
224
225 logic hwlp_end0_eq_pc;
226 logic hwlp_end1_eq_pc;
227 logic hwlp_counter0_gt_1;
228 logic hwlp_counter1_gt_1;
229 logic hwlp_counter0_eq_1;
230 logic hwlp_counter1_eq_1;
231 logic hwlp_counter0_eq_0;
232 logic hwlp_counter1_eq_0;
233 logic hwlp_end0_eq_pc_plus4;
234 logic hwlp_end1_eq_pc_plus4;
235 logic hwlp_start0_leq_pc;
236 logic hwlp_start1_leq_pc;
237 logic hwlp_end0_geq_pc;
238 logic hwlp_end1_geq_pc;
239
240 // Auxiliary signals to make hwlp_jump_o last only one cycle
241 (converting it into a pulse)
242 logic hwlp_end_4_id_d, hwlp_end_4_id_q;
243
244 logic debug_req_q;
245 logic debug_req_pending;
246
247 // qualify wfi vs nosleep locally
248 logic wfi_active;
249
250
251
252
253 //
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

266  csr_save_ex_o      = 1'b0;
267  csr_restore_mret_id_o = 1'b0;
268  csr_restore_uret_id_o = 1'b0;
269
270  csr_restore_dret_id_o = 1'b0;
271
272  csr_save_cause_o     = 1'b0;
273
274  exc_cause_o          = '0;
275  exc_pc_mux_o         = EXC_PC_IRQ;
276  trap_addr_mux_o      = TRAP_MACHINE;
277
278  csr_cause_o          = '0;
279  csr_irq_sec_o        = 1'b0;
280
281  pc_mux_o             = PC_BOOT;
282  pc_set_o             = 1'b0;
283  jump_done            = jump_done_q;
284
285  ctrl_fsm_ns          = ctrl_fsm_cs;
286
287  ctrl_busy_o          = 1'b1;
288
289  halt_if_o            = 1'b0;
290  halt_id_o            = 1'b0;
291  is_decoding_o        = 1'b0;
292  irq_ack_o            = 1'b0;
293  irq_id_o             = 5'b0;
294
295  jump_in_dec          = ctrl_transfer_insn_in_dec_i ==
BRANCH_JALR || ctrl_transfer_insn_in_dec_i == BRANCH_JAL;
296
297  branch_in_id         = ctrl_transfer_insn_in_id_i ==
BRANCH_COND;
298
299  ebrk_force_debug_mode = (debug_ebreakm_i && current_priv_lvl_i == PRIV_LVL_M) ||
300  (debug_ebreaku_i && current_priv_lvl_i == PRIV_LVL_U);
301  debug_csr_save_o     = 1'b0;
302  debug_cause_o        = DBG_CAUSE_EBREAK;
303  debug_mode_n         = debug_mode_q;
304
305  illegal_insn_n       = illegal_insn_q;
306  // a trap towards the debug unit is generated when one of
the
307  // following conditions are true:
308  // - ebreak instruction encountered
309  // - single-stepping mode enabled
310  // - illegal instruction exception and IIE bit is set
311  // - IRQ and INTE bit is set and no exception is currently
running
312  // - Debugger requests halt
313
314  debug_req_entry_n    = debug_req_entry_q;
315
316  debug_force_wakeup_n = debug_force_wakeup_q;
317
318  perf_pipeline_stall_o = 1'b0;
319
320  hwlp_mask_o          = 1'b0;
321
322  hwlp_dec_cnt_o       = '0;
323  hwlp_end_4_id_d      = 1'b0;
324
325  // When the controller tells to hwlp-jump, the prefetcher
does not always jump immediately,
326  // but the aligner immediately modifies pc_id to HWLP_BE-
GIN. This condition on hwlp_targ_addr_o
327  // ensures that the target is kept constant even if pc_id
is no more HWLP_END
328  hwlp_targ_addr_o     = ((hwlp_start1_leq_pc &&
hwlp_end1_geq_pc) && !(hwlp_start0_leq_pc && hwlp_end0_geq_pc)) ?
hwlp_start_addr_i[1] : hwlp_start_addr_i[0];
329
330  unique case (ctrl_fsm_cs)
331  // We were just reset, wait for fetch_enable
332  RESET:
333  begin
334  is_decoding_o = 1'b0;
335  instr_req_o   = 1'b0;
336  if (fetch_enable_i == 1'b1)
337  begin
338  ctrl_fsm_ns = BOOT_SET;
339  end
340  end
341
342  // copy boot address to instr fetch address
343  BOOT_SET:
344  begin
345  is_decoding_o = 1'b0;
346  instr_req_o   = 1'b1;
347  pc_mux_o      = PC_BOOT;
348  pc_set_o      = 1'b1;
349  if (debug_req_pending) begin
350  ctrl_fsm_ns = DBG_TAKEN_IF;
351  debug_force_wakeup_n = 1'b1;
352  end
353  else begin
354  ctrl_fsm_ns = FIRST_FETCH;
355  end

```

```

355  end
356
357  WAIT_SLEEP:
358  begin
359  is_decoding_o = 1'b0;
360  ctrl_busy_o   = 1'b0;
361  instr_req_o   = 1'b0;
362  halt_if_o     = 1'b1;
363  halt_id_o     = 1'b1;
364  ctrl_fsm_ns   = SLEEP;
365  end
366
367  // instruction in if_stage is already valid
368  SLEEP:
369  begin
370  // we begin execution when an
371  // interrupt has arrived
372  is_decoding_o = 1'b0;
373  instr_req_o   = 1'b0;
374  halt_if_o     = 1'b1;
375  halt_id_o     = 1'b1;
376
377  // normal execution flow
378  // in debug mode or single step mode we leave immedi-
ately (wfi=nop)
379  if (wake_from_sleep_o) begin
380  if (debug_req_pending) begin
381  ctrl_fsm_ns = DBG_TAKEN_IF;
382  debug_force_wakeup_n = 1'b1;
383  end else begin
384  ctrl_fsm_ns = FIRST_FETCH;
385  end
386  end else begin
387  ctrl_busy_o = 1'b0;
388  end
389  end
390
391  FIRST_FETCH:
392  begin
393  is_decoding_o = 1'b0;
394
395  // ID stage is always ready
396  ctrl_fsm_ns = DECODE;
397
398  // handle interrupts
399  if (irq_req_ctrl_i && ~(debug_req_pending || de-
bug_mode_q)) begin
400  // This assumes that the pipeline is always flushed
before
401  // going to sleep.
402  // Debug mode takes precedence over irq (see DE-
CODE:)
403
404  // Taken IRQ
405  halt_if_o   = 1'b1;
406  halt_id_o   = 1'b1;
407
408  pc_set_o    = 1'b1;
409  pc_mux_o    = PC_EXCEPTION;
410  exc_pc_mux_o = EXC_PC_IRQ;
411  exc_cause_o = irq_id_ctrl_i;
412  csr_irq_sec_o = irq_sec_ctrl_i;
413
414  // IRQ interface
415  irq_ack_o   = 1'b1;
416  irq_id_o    = irq_id_ctrl_i;
417
418  if (irq_sec_ctrl_i)
419  trap_addr_mux_o = TRAP_MACHINE;
420  else
421  trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
422
423  csr_save_cause_o = 1'b1;
424  csr_cause_o      = {1'b1, irq_id_ctrl_i};
425  csr_save_if_o    = 1'b1;
426  end
427  end
428
429  DECODE:
430  begin
431
432  if (branch_taken_ex_i)
433  begin //taken branch
434  // there is a branch in the EX stage that is taken
435
436  is_decoding_o = 1'b0;
437
438  pc_mux_o      = PC_BRANCH;
439  pc_set_o      = 1'b1;
440
441  // if we want to debug, flush the pipeline
442  // the current_pc_if will take the value of the
next instruction to
443  // be executed (NPC)
444
445  end //taken branch
446
447  else if (data_err_i)
448  begin //data error
449  // the current LW or SW have been blocked by the

```

```

PMP
450
451     is_decoding_o    = 1'b0;
452     halt_if_o        = 1'b1;
453     halt_id_o        = 1'b1;
454     csr_save_ex_o    = 1'b1;
455     csr_save_cause_o = 1'b1;
456     data_err_ack_o   = 1'b1;
457     //no jump in this stage as we have to wait one
cycle to go to Machine Mode
458
459     csr_cause_o       = {1'b0, data_we_ex_i ?
EXC_CAUSE_STORE_FAULT : EXC_CAUSE_LOAD_FAULT};
460     ctrl_fsm_ns       = FLUSH_WB;
461
462     end //data error
463
464     else if (is_fetch_failed_i)
465     begin
466
467         // the current instruction has been blocked by the
PMP
468
469         is_decoding_o    = 1'b0;
470         halt_id_o        = 1'b1;
471         halt_if_o        = 1'b1;
472         csr_save_if_o    = 1'b1;
473         csr_save_cause_o = !debug_mode_q;
474
475         //no jump in this stage as we have to wait one
cycle to go to Machine Mode
476
477         csr_cause_o       = {1'b0, EXC_CAUSE_INSTR_FAULT};
478         ctrl_fsm_ns       = FLUSH_WB;
479
480     end
481     // decode and execute instructions only if the cur-
rent conditional
482     // branch in the EX stage is either not taken, or
there is no
483     // conditional branch in the EX stage
484     else if (instr_valid_i) //valid block
485     begin: blk_decode_level1 // now analyze the current
instruction in the ID stage
486
487         is_decoding_o = 1'b1;
488         illegal_insn_n = 1'b0;
489
490         if ( (debug_req_pending || trigger_match_i) & ~de-
bug_mode_q )
491         begin
492             //Serving the debug
493             is_decoding_o    = COREV_PULP ? 1'b0 : 1'b1;
494             halt_if_o        = 1'b1;
495             halt_id_o        = 1'b1;
496             ctrl_fsm_ns      = DBG_FLUSH;
497             debug_req_entry_n = 1'b1;
498         end
499     else if (irq_req_ctrl_i && ~debug_mode_q)
500     begin
501         // Taken IRQ
502         hwlp_mask_o         = COREV_PULP ? 1'b1 : 1'b0;
503
504         is_decoding_o       = 1'b0;
505         halt_if_o           = 1'b1;
506         halt_id_o           = 1'b1;
507
508         pc_set_o            = 1'b1;
509         pc_mux_o            = PC_EXCEPTION;
510         exc_pc_mux_o        = EXC_PC_IRQ;
511         exc_cause_o         = irq_id_ctrl_i;
512         csr_irq_sec_o       = irq_sec_ctrl_i;
513
514         // IRQ interface
515         irq_ack_o           = 1'b1;
516         irq_id_o            = irq_id_ctrl_i;
517
518         if (irq_sec_ctrl_i)
519             trap_addr_mux_o = TRAP_MACHINE;
520         else
521             trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
522
523         csr_save_cause_o    = 1'b1;
524         csr_cause_o         = {1'b1, irq_id_ctrl_i};
525         csr_save_id_o       = 1'b1;
526     end
527     else
528     begin
529         if (illegal_insn_i) begin
530
531             halt_if_o        = 1'b1;
532             halt_id_o        = 1'b0;
533             ctrl_fsm_ns      = id_ready_i ? FLUSH_EX :
534             illegal_insn_n    = 1'b1;
535         end else begin
536
537
538
539
540         //decoding block
541         unique case (1'b1)
542
543             jump_in_dec: begin
544                 // handle unconditional jumps
545                 // we can jump directly since we know the
address already
546                 // we don't need to worry about condition-
al branches here as they
547                 // will be evaluated in the EX stage
548                 pc_mux_o = PC_JUMP;
549                 // if there is a jr stall, wait for it
550                 if ((~jr_stall_o) && (~jump_done_q)) be-
gin
551                     pc_set_o    = 1'b1;
552                     jump_done    = 1'b1;
553                 end
554             end
555
556             ebrk_insn_i: begin
557                 halt_if_o        = 1'b1;
558                 halt_id_o        = 1'b0;
559
560                 if (debug_mode_q)
561                     // we got back to the park loop in the
debug rom
562                     ctrl_fsm_ns = DBG_FLUSH;
563                 else if (ebrk_force_debug_mode) begin
564                     // debug module commands us to enter
debug mode anyway
565                     ctrl_fsm_ns = DBG_FLUSH;
566                 end else begin
567                     // otherwise just a normal ebreak ex-
ception
568                     ctrl_fsm_ns = id_ready_i ? FLUSH_EX :
569                     DECODE;
570                 end
571             end
572
573             wfi_active: begin
574                 halt_if_o        = 1'b1;
575                 halt_id_o        = 1'b0;
576                 ctrl_fsm_ns      = id_ready_i ?
577             FLUSH_EX : DECODE;
578         end
579
580             ecall_insn_i: begin
581                 halt_if_o        = 1'b1;
582                 halt_id_o        = 1'b0;
583                 ctrl_fsm_ns      = id_ready_i ?
584             FLUSH_EX : DECODE;
585         end
586
587             fencei_insn_i: begin
588                 halt_if_o        = 1'b1;
589                 halt_id_o        = 1'b0;
590                 ctrl_fsm_ns      = id_ready_i ?
591             FLUSH_EX : DECODE;
592         end
593
594             mret_insn_i | uret_insn_i | dret_insn_i:
595                 halt_if_o        = 1'b1;
596                 halt_id_o        = 1'b0;
597                 ctrl_fsm_ns      = id_ready_i ?
598             FLUSH_EX : DECODE;
599         end
600
601             csr_status_i: begin
602                 halt_if_o        = 1'b1;
603                 if (~id_ready_i) begin
604                     ctrl_fsm_ns = DECODE;
605                 end else begin
606                     ctrl_fsm_ns = FLUSH_EX;
607                     if (hwlp_end0_eq_pc) begin
608                         hwlp_dec_cnt_o[0] = 1'b1;
609                     end
610                     if (hwlp_end1_eq_pc) begin
611                         hwlp_dec_cnt_o[1] = 1'b1;
612                     end
613                 end
614             end
615
616             data_load_event_i: begin
617                 ctrl_fsm_ns      = id_ready_i ? ELW_EXE :
618                 halt_if_o        = 1'b1;
619             end
620
621             default: begin
622                 if(is_hwlp_body) begin
623                     //we are at the inside of an HWloop,
thus change state
624
625                     //We stay here in case we returned

```

```

from the second last instruction, otherwise the next cycle
624 //in DECODE_HWLOOP we miss to jump, we
jump at PC_END.
625 //This way looses a cycle but it's a
corner case of returning from exceptions or interrupts
626
627 ctrl_fsm_ns = hwlp_end0_eq_pc_plus4
|| hwlp_end1_eq_pc_plus4 ? DECODE : DECODE_HWLOOP;
628
629 // we can be at the end of Hwloop due
to a return from interrupt or ecall or ebreak or exceptions
630 if (hwlp_end0_eq_pc && hwlp_coun-
ter0_gt_1) begin
631     pc_mux_o = PC_HWLOOP;
632     if (~jump_done_q) begin
633         pc_set_o = 1'b1;
634         // Keep the instruction and the
related address in the Aligner if
635         // ID is stalled during a jump
636         jump_done = 1'b1;
637         hwlp_dec_cnt_o[0] = 1'b1;
638     end
639 end
640 if (hwlp_end1_eq_pc && hwlp_coun-
ter1_gt_1) begin
641     pc_mux_o = PC_HWLOOP;
642     if (~jump_done_q) begin
643         pc_set_o = 1'b1;
644         // Keep the instruction and the
related address in the Aligner if
645         // ID is stalled during a jump
646         jump_done = 1'b1;
647         hwlp_dec_cnt_o[1] = 1'b1;
648     end
649 end
650 end
651 if (hwlp_end0_eq_pc && hwlp_coun-
ter0_eq_1) begin
652     hwlp_dec_cnt_o[0] = 1'b1;
653 end
654 if (hwlp_end1_eq_pc && hwlp_coun-
ter1_eq_1) begin
655     hwlp_dec_cnt_o[1] = 1'b1;
656 end
657 end
658 endcase // unique case (1'b1)
659 end
660 if (debug_single_step_i & ~debug_mode_q) begin
661     // prevent any more instructions from ex-
ecuting
662     halt_if_o = 1'b1;
663     // we don't handle dret here because its
should be illegal
664     // anyway in this context
665
666 // illegal, ecall, ebrk and xrettransition
to later to a DBG
667 // state since we need the return address
which is
668 // determined later
669 if (id_ready_i) begin
670     // make sure the current instruction has
been executed
671     unique case(1'b1)
672         illegal_insn_i | ecall_insn_i:
673         begin
674             ctrl_fsm_ns = FLUSH_EX;
675         end
676         (~ebrk_force_debug_mode & ebrk_in-
sn_i):
677         begin
678             ctrl_fsm_ns = FLUSH_EX;
679         end
680         mret_insn_i | uret_insn_i:
681         begin
682             ctrl_fsm_ns = FLUSH_EX;
683         end
684         branch_in_id:
685         begin
686             ctrl_fsm_ns = DBG_WAIT_BRANCH;
687         end
688         default:
689         // regular instruction or ebrk
force debug
690         ctrl_fsm_ns = DBG_FLUSH;
691     endcase // unique case (1'b1)
692 end
693 end
694 end
695 end

```

```

705
706 end // else: !if (irq_req_ctrl_i && ~de-
bug_mode_q)
707
708 end //valid block
709 else begin
710     is_decoding_o = 1'b0;
711     perf_pipeline_stall_o = data_load_event_i;
712 end
713
714 DECODE_HWLOOP:
715 begin
716     if (COREV_PULP) begin
717         if (instr_valid_i) // valid block
718         begin // now analyze the current instruction in the
ID stage
719             is_decoding_o = 1'b1;
720
721             if ( (debug_req_pending || trigger_match_i) & ~de-
bug_mode_q )
722             begin
723                 //Serving the debug
724                 is_decoding_o = COREV_PULP ? 1'b0 : 1'b1;
725                 halt_if_o = 1'b1;
726                 halt_id_o = 1'b1;
727                 ctrl_fsm_ns = DBG_FLUSH;
728                 debug_req_entry_n = 1'b1;
729             end
730             else if (irq_req_ctrl_i && ~debug_mode_q)
731             begin
732                 // Taken IRQ
733                 hwlp_mask_o = COREV_PULP ? 1'b1 : 1'b0;
734
735                 is_decoding_o = 1'b0;
736                 halt_if_o = 1'b1;
737                 halt_id_o = 1'b1;
738
739                 pc_set_o = 1'b1;
740                 pc_mux_o = PC_EXCEPTION;
741                 exc_pc_mux_o = EXC_PC_IRQ;
742                 exc_cause_o = irq_id_ctrl_i;
743                 csr_irq_sec_o = irq_sec_ctrl_i;
744
745                 // IRQ interface
746                 irq_ack_o = 1'b1;
747                 irq_id_o = irq_id_ctrl_i;
748
749                 if (irq_sec_ctrl_i)
750                 trap_addr_mux_o = TRAP_MACHINE;
751             else
752                 trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
753
754                 csr_save_cause_o = 1'b1;
755                 csr_cause_o = {1'b1, irq_id_ctrl_i};
756                 csr_save_id_o = 1'b1;
757
758                 ctrl_fsm_ns = DECODE;
759             end
760             else
761             begin
762                 if (illegal_insn_i) begin
763                     halt_if_o = 1'b1;
764                     halt_id_o = 1'b1;
765                     ctrl_fsm_ns = FLUSH_EX;
766                     illegal_insn_n = 1'b1;
767                 end
768                 else begin
769                     //decoding block
770                     unique case (1'b1)
771                         ebrk_insn_i: begin
772                             halt_if_o = 1'b1;
773                             halt_id_o = 1'b0;
774                         end
775                         if (debug_mode_q)
776                         // we got back to the park loop in the
debug rom
777                         ctrl_fsm_ns = DBG_FLUSH;
778                     else if (ebrk_force_debug_mode)
779                     // debug module commands us to enter
debug mode anyway
780                     ctrl_fsm_ns = DBG_FLUSH;
781                     else begin
782                         // otherwise just a normal ebreak ex-
ception
783                         ctrl_fsm_ns = id_ready_i ? FLUSH_EX :
DECODE_HWLOOP;
784                     end
785                     end
786                     ecall_insn_i: begin
787
788
789
790
791
792
793
794
795
796

```



```

797     halt_if_o    = 1'b1;
798     halt_id_o    = 1'b0;
799     ctrl_fsm_ns   = id_ready_i ? FLUSH_EX :
DECODE_HWLOOP;
800     end
801
802     csr_status_i: begin
803         halt_if_o    = 1'b1;
804         if (~id_ready_i) begin
805             ctrl_fsm_ns = DECODE_HWLOOP;
806         end else begin
807             ctrl_fsm_ns = FLUSH_EX;
808             if (hwlp_end0_eq_pc) begin
809                 hwlp_dec_cnt_o[0] = 1'b1;
810             end
811             if (hwlp_end1_eq_pc) begin
812                 hwlp_dec_cnt_o[1] = 1'b1;
813             end
814         end
815     end
816
817     data_load_event_i: begin
818         ctrl_fsm_ns   = id_ready_i ? ELW_EXE :
DECODE_HWLOOP;
819         halt_if_o    = 1'b1;
820     end
821
822     default: begin
823         // we can be at the end of HWloop due
824         to a return from interrupt or ecall or ebreak or exceptions
825         if(hwlp_end1_eq_pc_plus4) begin
826             if(hwlp_counter1_gt_1) begin
827                 hwlp_end_4_id_d = 1'b1;
828                 hwlp_targ_addr_o =
hwlp_start_addr_i[1];
829                 ctrl_fsm_ns   = DECODE_HWLOOP;
830             end else
831                 ctrl_fsm_ns   = is_hwlp_body ?
DECODE_HWLOOP : DECODE;
832             end
833         if(hwlp_end0_eq_pc_plus4) begin
834             if(hwlp_counter0_gt_1) begin
835                 hwlp_end_4_id_d = 1'b1;
836                 hwlp_targ_addr_o =
hwlp_start_addr_i[0];
837             end
838             ctrl_fsm_ns   = DECODE_HWLOOP;
839         end else
840             ctrl_fsm_ns   = is_hwlp_body ?
DECODE_HWLOOP : DECODE;
841         end
842         hwlp_dec_cnt_o[0] = hwlp_end0_eq_pc && !
hwlp_counter0_eq_0;
843         hwlp_dec_cnt_o[1] = hwlp_end1_eq_pc && !
hwlp_counter1_eq_0;
844     end
845
846     endcase // unique case (1'b1)
847     end // else: !if(illegal_insn_i)
848
849     if (debug_single_step_i & ~debug_mode_q) begin
850         // prevent any more instructions from ex-
851         cuting
852         halt_if_o = 1'b1;
853
854         // we don't handle dret here because its
855         should be illegal
856         // anyway in this context
857
858         // illegal, ecall, ebrk and xrettransition
859         to later to a DBG
860         // state since we need the return address
861         which is
862         // determined later
863
864         if (id_ready_i) begin
865             // make sure the current instruction has
866             been executed
867             unique case(1'b1)
868             illegal_insn_i | ecall_insn_i:
869             begin
870                 ctrl_fsm_ns = FLUSH_EX;
871             end
872             (~ebrk_force_debug_mode & ebrk_in-
873             sn_i):
874             begin
875                 ctrl_fsm_ns = FLUSH_EX;
876             end
877             mret_insn_i | uret_insn_i:
878             begin
879                 ctrl_fsm_ns = FLUSH_EX;
880             end
881             branch_in_id:
882             begin

```

```

882         ctrl_fsm_ns = DBG_WAIT_BRANCH;
883     end
884
885     default:
886         // regular instruction or ebrk
887
888     force debug
889         ctrl_fsm_ns = DBG_FLUSH;
890         endcase // unique case (1'b1)
891     end
892     end // if (debug_single_step_i & ~de-
893     bug_mode_q)
894     end // else: !if (irq_req_ctrl_i && ~de-
895     bug_mode_q)
896     end // block: blk_decode_level1 : valid block
897     else begin
898         is_decoding_o    = 1'b0;
899         perf_pipeline_stall_o = data_load_event_i;
900     end
901     end
902     end
903
904     // flush the pipeline, insert NOP into EX stage
905     FLUSH_EX:
906     begin
907         is_decoding_o = 1'b0;
908         halt_if_o = 1'b1;
909         halt_id_o = 1'b1;
910
911         if (data_err_i)
912             begin //data error
913                 // the current LW or SW have been blocked by the
914                 PMP
915                 csr_save_ex_o    = 1'b1;
916                 csr_save_cause_o = 1'b1;
917                 data_err_ack_o   = 1'b1;
918                 //no jump in this stage as we have to wait one
919                 cycle to go to Machine Mode
920                 csr_cause_o      = {1'b0, data_we_ex_i ?
921                 EXC_CAUSE_STORE_FAULT : EXC_CAUSE_LOAD_FAULT};
922                 ctrl_fsm_ns      = FLUSH_WB;
923
924                 //putting illegal to 0 as if it was 1, the core is
925                 going to jump to the exception of the EX stage,
926                 //so the illegal was never executed
927                 illegal_insn_n   = 1'b0;
928                 end //data erro
929                 else if (ex_valid_i) begin
930                     //check done to prevent data harzard in the CSR re-
931                     gisters
932                     ctrl_fsm_ns = FLUSH_WB;
933
934                     if(illegal_insn_q) begin
935                         csr_save_id_o    = 1'b1;
936                         csr_save_cause_o = !debug_mode_q;
937                         csr_cause_o      = {1'b0,
938                         EXC_CAUSE_ILLEGAL_INSN};
939                     end else begin
940                         unique case (1'b1)
941                         ebrk_insn_i: begin
942                             csr_save_id_o    = 1'b1;
943                             csr_save_cause_o = 1'b1;
944                             csr_cause_o      = {1'b0,
945                             EXC_CAUSE_BREAKPOINT};
946                         end
947                         ecall_insn_i: begin
948                             csr_save_id_o    = 1'b1;
949                             csr_save_cause_o = !debug_mode_q;
950                             csr_cause_o      = {1'b0, current_priv_lvl_i
951                             == PRIV_LVL_U ? EXC_CAUSE_ECALL_UMODE : EXC_CAUSE_ECALL_MMODE};
952                         end
953                         default;;
954                     endcase // unique case (1'b1)
955                 end
956                 end
957                 end
958                 end
959                 end
960                 end
961                 end
962                 end
963                 end
964                 end
965                 end
966                 end
967                 end
968                 end
969                 end
970                 end
971                 end
972                 end
973                 end
974                 end
975                 end
976                 end
977                 end
978                 end
979                 end
980                 end
981                 end
982                 end
983                 end
984                 end
985                 end
986                 end
987                 end
988                 end
989                 end
990                 end
991                 end
992                 end
993                 end
994                 end
995                 end
996                 end
997                 end
998                 end
999                 end
1000                end
1001                end
1002                end
1003                end
1004                end
1005                end
1006                end
1007                end
1008                end
1009                end
1010                end
1011                end
1012                end
1013                end
1014                end
1015                end
1016                end
1017                end
1018                end
1019                end
1020                end
1021                end
1022                end
1023                end
1024                end
1025                end
1026                end
1027                end
1028                end
1029                end
1030                end
1031                end
1032                end
1033                end
1034                end
1035                end
1036                end
1037                end
1038                end
1039                end
1040                end
1041                end
1042                end
1043                end
1044                end
1045                end
1046                end
1047                end
1048                end
1049                end
1050                end
1051                end
1052                end
1053                end
1054                end
1055                end
1056                end
1057                end
1058                end
1059                end
1060                end
1061                end
1062                end
1063                end
1064                end
1065                end
1066                end
1067                end
1068                end
1069                end
1070                end
1071                end
1072                end
1073                end
1074                end
1075                end
1076                end
1077                end
1078                end
1079                end
1080                end
1081                end
1082                end
1083                end
1084                end
1085                end
1086                end
1087                end
1088                end
1089                end
1090                end
1091                end
1092                end
1093                end
1094                end
1095                end
1096                end
1097                end
1098                end
1099                end
1100                end
1101                end
1102                end
1103                end
1104                end
1105                end
1106                end
1107                end
1108                end
1109                end
1110                end
1111                end
1112                end
1113                end
1114                end
1115                end
1116                end
1117                end
1118                end
1119                end
1120                end
1121                end
1122                end
1123                end
1124                end
1125                end
1126                end
1127                end
1128                end
1129                end
1130                end
1131                end
1132                end
1133                end
1134                end
1135                end
1136                end
1137                end
1138                end
1139                end
1140                end
1141                end
1142                end
1143                end
1144                end
1145                end
1146                end
1147                end
1148                end
1149                end
1150                end
1151                end
1152                end
1153                end
1154                end
1155                end
1156                end
1157                end
1158                end
1159                end
1160                end
1161                end
1162                end
1163                end
1164                end
1165                end
1166                end
1167                end
1168                end
1169                end
1170                end
1171                end
1172                end
1173                end
1174                end
1175                end
1176                end
1177                end
1178                end
1179                end
1180                end
1181                end
1182                end
1183                end
1184                end
1185                end
1186                end
1187                end
1188                end
1189                end
1190                end
1191                end
1192                end
1193                end
1194                end
1195                end
1196                end
1197                end
1198                end
1199                end
1200                end
1201                end
1202                end
1203                end
1204                end
1205                end
1206                end
1207                end
1208                end
1209                end
1210                end
1211                end
1212                end
1213                end
1214                end
1215                end
1216                end
1217                end
1218                end
1219                end
1220                end
1221                end
1222                end
1223                end
1224                end
1225                end
1226                end
1227                end
1228                end
1229                end
1230                end
1231                end
1232                end
1233                end
1234                end
1235                end
1236                end
1237                end
1238                end
1239                end
1240                end
1241                end
1242                end
1243                end
1244                end
1245                end
1246                end
1247                end
1248                end
1249                end
1250                end
1251                end
1252                end
1253                end
1254                end
1255                end
1256                end
1257                end
1258                end
1259                end
1260                end
1261                end
1262                end
1263                end
1264                end
1265                end
1266                end
1267                end
1268                end
1269                end
1270                end
1271                end
1272                end
1273                end
1274                end
1275                end
1276                end
1277                end
1278                end
1279                end
1280                end
1281                end
1282                end
1283                end
1284                end
1285                end
1286                end
1287                end
1288                end
1289                end
1290                end
1291                end
1292                end
1293                end
1294                end
1295                end
1296                end
1297                end
1298                end
1299                end
1300                end
1301                end
1302                end
1303                end
1304                end
1305                end
1306                end
1307                end
1308                end
1309                end
1310                end
1311                end
1312                end
1313                end
1314                end
1315                end
1316                end
1317                end
1318                end
1319                end
1320                end
1321                end
1322                end
1323                end
1324                end
1325                end
1326                end
1327                end
1328                end
1329                end
1330                end
1331                end
1332                end
1333                end
1334                end
1335                end
1336                end
1337                end
1338                end
1339                end
1340                end
1341                end
1342                end
1343                end
1344                end
1345                end
1346                end
1347                end
1348                end
1349                end
1350                end
1351                end
1352                end
1353                end
1354                end
1355                end
1356                end
1357                end
1358                end
1359                end
1360                end
1361                end
1362                end
1363                end
1364                end
1365                end
1366                end
1367                end
1368                end
1369                end
1370                end
1371                end
1372                end
1373                end
1374                end
1375                end
1376                end
1377                end
1378                end
1379                end
1380                end
1381                end
1382                end
1383                end
1384                end
1385                end
1386                end
1387                end
1388                end
1389                end
1390                end
1391                end
1392                end
1393                end
1394                end
1395                end
1396                end
1397                end
1398                end
1399                end
1400                end
1401                end
1402                end
1403                end
1404                end
1405                end
1406                end
1407                end
1408                end
1409                end
1410                end
1411                end
1412                end
1413                end
1414                end
1415                end
1416                end
1417                end
1418                end
1419                end
1420                end
1421                end
1422                end
1423                end
1424                end
1425                end
1426                end
1427                end
1428                end
1429                end
1430                end
1431                end
1432                end
1433                end
1434                end
1435                end
1436                end
1437                end
1438                end
1439                end
1440                end
1441                end
1442                end
1443                end
1444                end
1445                end
1446                end
1447                end
1448                end
1449                end
1450                end
1451                end
1452                end
1453                end
1454                end
1455                end
1456                end
1457                end
1458                end
1459                end
1460                end
1461                end
1462                end
1463                end
1464                end
1465                end
1466                end
1467                end
1468                end
1469                end
1470                end
1471                end
1472                end
1473                end
1474                end
1475                end
1476                end
1477                end
1478                end
1479                end
1480                end
1481                end
1482                end
1483                end
1484                end
1485                end
1486                end
1487                end
1488                end
1489                end
1490                end
1491                end
1492                end
1493                end
1494                end
1495                end
1496                end
1497                end
1498                end
1499                end
1500                end
1501                end
1502                end
1503                end
1504                end
1505                end
1506                end
1507                end
1508                end
1509                end
1510                end
1511                end
1512                end
1513                end
1514                end
1515                end
1516                end
1517                end
1518                end
1519                end
1520                end
1521                end
1522                end
1523                end
1524                end
1525                end
1526                end
1527                end
1528                end
1529                end
1530                end
1531                end
1532                end
1533                end
1534                end
1535                end
1536                end
1537                end
1538                end
1539                end
1540                end
1541                end
1542                end
1543                end
1544                end
1545                end
1546                end
1547                end
1548                end
1549                end
1550                end
1551                end
1552                end
1553                end
1554                end
1555                end
1556                end
1557                end
1558                end
1559                end
1560                end
1561                end
1562                end
1563                end
1564                end
1565                end
1566                end
1567                end
1568                end
1569                end
1570                end
1571                end
1572                end
1573                end
1574                end
1575                end
1576                end
1577                end
1578                end
1579                end
1580                end
1581                end
1582                end
1583                end
1584                end
1585                end
1586                end
1587                end
1588                end
1589                end
1590                end
1591                end
1592                end
1593                end
1594                end
1595                end
1596                end
1597                end
1598                end
1599                end
1600                end
1601                end
1602                end
1603                end
1604                end
1605                end
1606                end
1607                end
1608                end
1609                end
1610                end
1611                end
1612                end
1613                end
1614                end
1615                end
1616                end
1617                end
1618                end
1619                end
1620                end
1621                end
1622                end
1623                end
1624                end
1625                end
1626                end
1627                end
1628                end
1629                end
1630                end
1631                end
1632                end
1633                end
1634                end
1635                end
1636                end
1637                end
1638                end
1639                end
1640                end
1641                end
1642                end
1643                end
1644                end
1645                end
1646                end
1647                end
1648                end
1649                end
1650                end
1651                end
1652                end
1653                end
1654                end
1655                end
1656                end
1657                end
1658                end
1659                end
1660                end
1661                end
1662                end
1663                end
1664                end
1665                end
1666                end
1667                end
1668                end
1669                end
1670                end
1671                end
1672                end
1673                end
1674                end
1675                end
1676                end
1677                end
1678                end
1679                end
1680                end
1681                end
1682                end
1683                end
1684                end
1685                end
1686                end
1687                end
1688                end
1689                end
1690                end
1691                end
1692                end
1693                end
1694                end
1695                end
1696                end
1697                end
1698                end
1699                end
1700                end
1701                end
1702                end
1703                end
1704                end
1705                end
1706                end
1707                end
1708                end
1709                end
1710                end
1711                end
1712                end
1713                end
1714                end
1715                end
1716                end
1717                end
1718                end
1719                end
1720                end
1721                end
1722                end
1723                end
1724                end
1725                end
1726                end
1727                end
1728                end
1729                end
1730                end
1731                end
1732                end
1733                end
1734                end
1735                end
1736                end
1737                end
1738                end
1739                end
1740                end
1741                end
1742                end
1743                end
1744                end
1745                end
1746                end
1747                end
1748                end
1749                end
1750                end
1751                end
1752                end
1753                end
1754                end
1755                end
1756                end
1757                end
1758                end
1759                end
1760                end
1761                end
1762                end
1763                end
1764                end
1765                end
1766                end
1767                end
1768                end
1769                end
1770                end
1771                end
1772                end
1773                end
1774                end
1775                end
1776                end
1777                end
1778                end
1779                end
1780                end
1781                end
1782                end
1783                end
1784                end
1785                end
1786                end
1787                end
1788                end
1789                end
1790                end
1791                end
1792                end
1793                end
1794                end
1795                end
1796                end
1797                end
1798                end
1799                end
1800                end
1801                end
1802                end
1803                end
1804                end
1805                end
1806                end
1807                end
1808                end
1809                end
1810                end
1811                end
1812                end
1813                end
1814                end
1815                end
1816                end
1817                end
1818                end
1819                end
1820                end
1821                end
1822                end
1823                end
1824                end
1825                end
1826                end
1827                end
1828                end
1829                end
1830                end
1831                end
1832                end
1833                end
1834                end
1835                end
1836                end
1837                end
1838                end
1839                end
1840                end
1841                end
1842                end
1843                end
1844                end
1845                end
1846                end
1847                end
1848                end
1849                end
1850                end
1851                end
1852                end
1853                end
1854                end
1855                end
1856                end
1857                end
1858                end
1859                end
1860                end
1861                end
1862                end
1863                end
1864                end
1865                end
1866                end
1867                end
1868                end
1869                end
1870                end
1871                end
1872                end
1873                end
1874                end
1875                end
1876                end
1877                end
1878                end
1879                end
1880                end
1881                end
1882                end
1883                end
1884                end
1885                end
1886                end
1887                end
1888                end
1889                end
1890                end
1891                end
1892                end
1893                end
1894                end
1895                end
1896                end
1897                end
1898                end
1899                end
1900                end
1901                end
1902                end
1903                end
1904                end
1905                end
1906                end
1907                end
1908                end
1909                end
1910                end
1911                end
1912                end
1913                end
1914                end
1915                end
1916                end
1917                end
1918                end
1919                end
1920                end
1921                end
1922                end
1923                end
1924                end
1925                end
1926                end
1927                end
1928                end
1929                end
1930                end
1931                end
1932                end
1933                end
1934                end
1935                end
1936                end
1937                end
1938                end
1939                end
1940                end
1941                end
1942                end
1943                end
1944                end
1945                end
1946                end
1947                end
1948                end
1949                end
1950                end
1951                end
1952                end
1953                end
1954                end
1955                end
1956                end
1957                end
1958                end
1959                end
1960                end
1961                end
1962                end
1963                end
1964                end
1965                end
1966                end
1967                end
1968                end
1969                end
1970                end
1971                end
1972                end
1973                end
1974                end
1975                end
1976                end
1977                end
1978                end
1979                end
1980                end
1981                end
1982                end
1983                end
1984                end
1985                end
1986                end
1987                end
1988                end
1989                end
1990                end
1991                end
1992                end
1993                end
1994                end
1995                end
1996                end
1997                end
1998                end
1999                end
2000                end
2001                end
2002                end
2003                end
2004                end
2005                end
2006                end
2007                end
2008                end
2009                end
2010                end
2011                end
2012                end
2013                end
2014                end
2015                end
2016                end
2017                end
2018                end
2019                end
2020                end
2021                end
2022                end
2023                end
2024                end
2025                end
2026                end
2027                end
2028                end
2029                end
2030                end
2031                end
2032                end
2033                end
2034                end
2035                end
2036                end
2037                end
2038                end
2039                end
2040                end
2041                end
2042                end
2043                end
2044                end
2045                end
2046                end
2047                end
2048                end
2049                end
2050                end
2051                end
2052                end
2053                end
2054                end
2055                end
2056                end
2057                end
2058                end
2059                end
2060                end
2061                end
2062                end
2063                end
2064                end
2065                end
2066                end
2067                end
2068                end
2069                end
2070                end
2071                end
2072                end
2073                end
2074                end
2075                end
2076                end
2077                end
2078                end
2079                end
2080                end
2081                end
2082                end
2083                end
2084                end
2085                end
2086                end
2087                end
2088                end
2089                end
2090                end
2091                end
2092                end
2093                end
2094                end
2095                end
2096                end
2097                end
2098                end
2099                end
2100                end
2101                end
2102                end
2103                end
2104                end
2105                end
2106                end
2107                end
2108                end
2109                end
2110                end
2111                end
2112                end
2113                end
2114                end
2115                end
2116                end
2117                end
2118                end
2119                end
2120                end
2121                end
2122                end
2123                end
2124                end
2125                end
2126                end
2127                end
2128                end
2129                end
2130                end
2131                end
2132                end
2133                end
2134                end
2135                end
2136                end
2137                end
2138                end
2139                end
2140                end
2141                end
2142                end
2143                end
2144                end
2145                end
2146                end
2147                end
2148                end
2149                end
2150                end
2151                end
2152                end
2153                end
2154                end
2155                end
2156                end
2157                end
2158                end
2159                end
2160                end
2161                end
2162                end
2163                end
2164                end
2165                end
2166                end
2167                end
2168                end
2169                end
2170                end
2171                end
2172                end
2173                end
2174                end
2175                end
2176                end
2177                end
2178                end
2179                end
2180                end
2181                end
2182                end
2183                end
2184                end
2185                end
2186                end
2187                end
2188                end
2189                end
2190                end
2191                end
2192                end
2193                end
2194                end
2195                end
2196                end
2197                end
2198                end
2199                end
2200                end
2201                end
2202                end
2203                end
2204                end
2205                end
2206                end
2207                end
2208                end
2209                end
2210                end
2211                end
2212                end
2213                end
2214                end
2215                end
2216                end
2217                end
2218                end
2219                end
2220                end
2221                end
2222                end
2223                end
2224                end
2225                end
2226                end
2227                end
2228                end
2229                end
2230                end
2231                end
2232                end
2233                end
2234                end
2235                end
2236                end
2237                end
2238                end
2239                end
2240                end
2241                end
2242                end
2243                end
2244                end
2245                end
2246                end
2247                end
2248                end
2249                end
2250                end
2251                end
2252                end
2253                end
2254                end
2255                end
2256                end
2257                end
2258                end
2259                end
2260                end
2261                end
2262                end
2263                end
2264                end
2265                end
2266                end
2267                end
2268                end
2269                end
2270                end
2271                end
2272                end
2273                end
2274                end
2275                end
2276                end
2277                end
2278                end
2279                end
2280                end
2281                end
2282                end
2283                end
2284                end
2285                end
2286                end
2287                end
2288                end
2289                end
2290                end
2291                end
2292                end
2293                end
2294                end
2295                end
2296                end
2297                end
2298                end
2299                end
2300                end
2301                end
2302                end
2303                end
2304                end
2305                end
2306                end
2307                end
2308                end
2309                end
2310                end
2311                end
2312                end
2313                end
2314                end
2315                end
2316                end
2317                end
2318                end
2319                end
2320                end
2321                end
2322                end
2323                end
2324                end
2325                end
2326                end
2327                end
2328                end
2329                end
2330                end
2331                end
2332                end
2333                end
2334                end
2335                end
2336                end
2337                end
2338                end
2339                end
2340                end
2341                end
2342                end
2343                end
2344                end
2345                end
2346                end
2347                end
2348                end
2349                end
2350                end
2351                end
2352                end
2353                end
2354                end
2355                end
2356                end
2357                end
2358                end
2359                end
2360                end
2361                end
2362                end
2363                end
2364                end
2365                end
2366                end
2367                end
2368                end
2369                end
2370                end
2371                end
2372                end
2373                end
2374                end
2375                end
2376                end
2377                end
2378                end
2379                end
2380                end
2381                end
2382                end
2383                end
2384                end
2385                end
2386                end
2387                end
2388                end
2389                end
2390                end
2391                end
2392                end
2393                end
2394                end
2395                end
2396                end
2397                end
2398                end
2399                end
2400                end
2401                end
2402                end
2403                end
2404                end
2405                end
2406                end
2407                end
2408                end
2409                end
2410                end
2411                end
2412                end
2413                end
2414                end
2415                end
2416                end
2417                end
2418                end
2419                end
2420                end
2421                end
2422                end
2423                end
2424                end
2425                end
2426                end
2427                end
2428                end
2429                end
2430                end
2431                end

```



```

970     exc_pc_mux_o      = EXC_PC_IRQ;
971     exc_cause_o       = irq_id_ctrl_i;
972     csr_irq_sec_o     = irq_sec_ctrl_i;
973
974     // IRQ interface
975     irq_ack_o         = 1'b1;
976     irq_id_o          = irq_id_ctrl_i;
977
978     if (irq_sec_ctrl_i)
979         trap_addr_mux_o = TRAP_MACHINE;
980     else
981         trap_addr_mux_o = current_priv_lvl_i ==
PRIV_LVL_U ? TRAP_USER : TRAP_MACHINE;
982
983     csr_save_cause_o   = 1'b1;
984     csr_cause_o        = {1'b1, irq_id_ctrl_i};
985     csr_save_id_o      = 1'b1;
986 end
987 end
988 end
989
990 ELW_EXE:
991 begin
992     if (COREV_CLUSTER == 1'b1) begin
993         is_decoding_o = 1'b0;
994
995         halt_if_o      = 1'b1;
996         halt_id_o      = 1'b1;
997
998         //if we are here, a cv.elw is executing now in the
EX stage
999         //or if an interrupt has been received
1000         //the ID stage contains the PC_ID of the cv.elw,
therefore halt_id is set to invalid the instruction
1001         //If an interrupt occurs, we replay the ELW
1002         //No needs to check irq_int_req_i since in the EX
stage there is only the cv.elw, no CSR pendings
1003         if(id_ready_i)
1004             ctrl_fsm_ns = ((debug_req_pending || trig-
ger_match_i) & ~debug_mode_q) ? DBG_FLUSH : IRQ_FLUSH_ELW;
1005         // if from the ELW EXE we go to IRQ_FLUSH_ELW, it
is assumed that if there was an IRQ req together with the grant and
IE was valid, then
1006         // there must be no hazard due to xIE
1007         else
1008             ctrl_fsm_ns = ELW_EXE;
1009
1010         perf_pipeline_stall_o = data_load_event_i;
1011     end
1012 end
1013
1014 // flush the pipeline, insert NOP into EX and WB stage
1015 FLUSH_WB:
1016 begin
1017     is_decoding_o = 1'b0;
1018
1019     halt_if_o      = 1'b1;
1020     halt_id_o      = 1'b1;
1021
1022     ctrl_fsm_ns = DECODE;
1023
1024     if(data_err_q) begin
1025         //PMP data_error
1026         pc_mux_o      = PC_EXCEPTION;
1027         pc_set_o       = 1'b1;
1028         trap_addr_mux_o = TRAP_MACHINE;
1029         //little hack during testing
1030         exc_pc_mux_o   = EXC_PC_EXCEPTION;
1031         exc_cause_o    = data_we_ex_i ?
EXC_CAUSE_LOAD_FAULT : EXC_CAUSE_STORE_FAULT;
1032     end
1033     else if (is_fetch_failed_i) begin
1034         //instruction fetch error
1035         pc_mux_o      = PC_EXCEPTION;
1036         pc_set_o       = 1'b1;
1037         trap_addr_mux_o = TRAP_MACHINE;
1038         exc_pc_mux_o   = debug_mode_q ?
EXC_PC_DBE : EXC_PC_EXCEPTION;
1039         exc_cause_o    = EXC_CAUSE_INSTR_FAULT;
1040     end
1041     else begin
1042         if(illegal_insn_q) begin
1043             //exceptions
1044             pc_mux_o      = PC_EXCEPTION;
1045             pc_set_o       = 1'b1;
1046             trap_addr_mux_o = TRAP_MACHINE;
1047             exc_pc_mux_o   = debug_mode_q ?
EXC_PC_DBE : EXC_PC_EXCEPTION;
1048             illegal_insn_n = 1'b0;
1049             if (debug_single_step_i && ~debug_mode_q)
1050                 ctrl_fsm_ns = DBG_TAKEN_IF;
1051             end else begin
1052                 unique case(1'b1)
1053                     ebrk_insn_i: begin
1054                         //ebreak
1055                         pc_mux_o      = PC_EXCEPTION;
1056                         pc_set_o       = 1'b1;
1057                         trap_addr_mux_o = TRAP_MACHINE;
1058                         exc_pc_mux_o   = EXC_PC_EXCEPTION;
1059                     end
1060                 end

```

```

1061
1062         if (debug_single_step_i && ~debug_mode_q)
1063             ctrl_fsm_ns = DBG_TAKEN_IF;
1064     end
1065     ecall_insn_i: begin
1066         //ecall
1067         pc_mux_o      = PC_EXCEPTION;
1068         pc_set_o       = 1'b1;
1069         trap_addr_mux_o = TRAP_MACHINE;
1070         exc_pc_mux_o   = debug_mode_q ?
EXC_PC_DBE : EXC_PC_EXCEPTION;
1071
1072         if (debug_single_step_i && ~debug_mode_q)
1073             ctrl_fsm_ns = DBG_TAKEN_IF;
1074     end
1075
1076     mret_insn_i: begin
1077         csr_restore_mret_id_o = !debug_mode_q;
1078         ctrl_fsm_ns          = XRET_JUMP;
1079     end
1080
1081     uret_insn_i: begin
1082         csr_restore_uret_id_o = !debug_mode_q;
1083         ctrl_fsm_ns          = XRET_JUMP;
1084     end
1085
1086     dret_insn_i: begin
1087         csr_restore_dret_id_o = 1'b1;
1088         ctrl_fsm_ns          = XRET_JUMP;
1089     end
1090
1091     csr_status_i: begin
1092         if ((hwlp_end0_eq_pc && !hwlp_counter0_eq_0)
||
1093             (hwlp_end1_eq_pc && !hwlp_counter1_eq_0))
1094         begin
1095             pc_mux_o = PC_HWLOOP;
1096             pc_set_o = 1'b1;
1097         end
1098     end
1099
1100     wfi_i: begin
1101         if ( debug_req_pending) begin
1102             ctrl_fsm_ns = DBG_TAKEN_IF;
1103             debug_force_wakeup_n = 1'b1;
1104         end else begin
1105             ctrl_fsm_ns = WAIT_SLEEP;
1106         end
1107     end
1108
1109     fencei_insn_i: begin
1110         // we just jump to instruction after the
fence.i since that
1111         // forces the instruction cache to refetch
1112         pc_mux_o      = PC_FENCEI;
1113         pc_set_o       = 1'b1;
1114     end
1115
1116     default;;
1117 endcase
1118 end
1119
1120 XRET_JUMP:
1121 begin
1122     is_decoding_o = 1'b0;
1123     ctrl_fsm_ns   = DECODE;
1124     unique case(1'b1)
1125         mret_dec_i: begin
1126             //mret
1127             pc_mux_o      = debug_mode_q ? PC_EXCEP-
TION : PC_MRET;
1128             pc_set_o       = 1'b1;
1129             exc_pc_mux_o   = EXC_PC_DBE; // only used
if in debug_mode
1130         end
1131         uret_dec_i: begin
1132             //uret
1133             pc_mux_o      = debug_mode_q ? PC_EXCEP-
TION : PC_URET;
1134             pc_set_o       = 1'b1;
1135             exc_pc_mux_o   = EXC_PC_DBE; // only used
if in debug_mode
1136         end
1137         dret_dec_i: begin
1138             //dret
1139             // this case is only reachable while in de-
bug_mode
1140             pc_mux_o      = PC_DRET;
1141             pc_set_o       = 1'b1;
1142             debug_mode_n   = 1'b0;
1143         end
1144         default;;
1145     endcase
1146
1147     if (debug_single_step_i && ~debug_mode_q) begin
1148         ctrl_fsm_ns = DBG_TAKEN_IF;
1149     end
1150
1151     // a branch was in ID when trying to go to debug rom.

```

```

Wait until we can
1151 // determine branch target address (for saving into dpc)
before proceeding
1152 DBG_WAIT_BRANCH:
1153 begin
1154   is_decoding_o = 1'b0;
1155   halt_if_o = 1'b1;
1156
1157   if (branch_taken_ex_i) begin
1158     // there is a branch in the EX stage that is taken
1159     pc_mux_o = PC_BRANCH;
1160     pc_set_o = 1'b1;
1161   end
1162
1163   ctrl_fsm_ns = DBG_FLUSH;
1164 end
1165
1166 // We enter this state when we encounter
1167 // 1. ebreak during debug mode
1168 // 2. trigger match
1169 // 3. ebreak with forced entry into debug mode (ebreaku
or ebreaku set).
1170 // 4. halt request during decode
1171 // Regular ebreak's go through FLUSH_EX and FLUSH_WB.
1172 // For 1. we don't update dcsr and dpc while for 2., 3.,
& 4. we do
1173 // dpc is set to the address of ebreak and trigger match
1174 // not to the next instruction's (which is why we save
the pc in id).
1175 DBG_TAKEN_ID:
1176 begin
1177   is_decoding_o = 1'b0;
1178   pc_set_o = 1'b1;
1179   pc_mux_o = PC_EXCEPTION;
1180   exc_pc_mux_o = EXC_PC_DBT;
1181   // If not in debug mode then save cause and dpc csrs
1182   // else it was an ebreak in debug mode, so don't up-
date csrs
1183   if (~debug_mode_q) begin
1184     csr_save_cause_o = 1'b1;
1185     csr_save_id_o = 1'b1;
1186     debug_csr_save_o = 1'b1;
1187     if (trigger_match_i)
1188       debug_cause_o = DBG_CAUSE_TRIGGER; // pri 4
(highest)
1189   else if (ebrk_force_debug_mode & ebrk_insn_i)
1190     debug_cause_o = DBG_CAUSE_EBREAK; // pri 3
1191   else if (debug_req_entry_q)
1192     debug_cause_o = DBG_CAUSE_HALTREQ; // pri 2 and
1
1193   end
1194   debug_req_entry_n = 1'b0;
1195   ctrl_fsm_ns = DECODE;
1196   debug_mode_n = 1'b1;
1197 end
1198
1199 // We enter this state for single stepping
1200 // DPC is set the next instruction to be executed/
fetched
1201
1202 DBG_TAKEN_IF:
1203 begin
1204   is_decoding_o = 1'b0;
1205   pc_set_o = 1'b1;
1206   pc_mux_o = PC_EXCEPTION;
1207   exc_pc_mux_o = EXC_PC_DBT;
1208   csr_save_cause_o = 1'b1;
1209   debug_csr_save_o = 1'b1;
1210   if (debug_force_wakeup_q)
1211     debug_cause_o = DBG_CAUSE_HALTREQ;
1212   else if (debug_single_step_i)
1213     debug_cause_o = DBG_CAUSE_STEP; // pri 0
1214   csr_save_if_o = 1'b1;
1215   ctrl_fsm_ns = DECODE;
1216   debug_mode_n = 1'b1;
1217   debug_force_wakeup_n = 1'b0;
1218 end
1219
1220
1221 DBG_FLUSH:
1222 begin
1223   is_decoding_o = 1'b0;
1224
1225   halt_if_o = 1'b1;
1226   halt_id_o = 1'b1;
1227
1228   perf_pipeline_stall_o = data_load_event_i;
1229
1230   if (data_err_i)
1231     begin //data error
1232       // the current LW or SW have been blocked by the
PMP
1233       csr_save_ex_o = 1'b1;
1234       csr_save_cause_o = 1'b1;
1235       data_err_ack_o = 1'b1;
1236       //no jump in this stage as we have to wait one
cycle to go to Machine Mode
1237       csr_cause_o = {1'b0, data_we_ex_i ?

```

```

EXC_CAUSE_STORE_FAULT : EXC_CAUSE_LOAD_FAULT};
1238   ctrl_fsm_ns = FLUSH_WB;
1239 end //data error
1240 else begin
1241   if(debug_mode_q
1242     trigger_match_i
1243     (ebrk_force_debug_mode & ebrk_insn_i)
1244     data_load_event_i
1245     debug_req_entry_q
1246   )
1247   begin
1248     ctrl_fsm_ns = DBG_TAKEN_ID;
1249   end else
1250   begin
1251     // else must be debug_single_step_i
1252     ctrl_fsm_ns = DBG_TAKEN_IF;
1253   end
1254 end
1255 // Debug end
1256
1257 default: begin
1258   is_decoding_o = 1'b0;
1259   instr_req_o = 1'b0;
1260   ctrl_fsm_ns = RESET;
1261 end
1262 endcase
1263 end
1264
1265 generate
1266   if(COREV_PULP) begin : gen_hwlp
1267     // Convert hwlp_jump_o to a pulse
1270     // hwlp_jump_o should last one cycle only, as the
prefetcher
1271     // reacts immediately. If it last more cycles, the
prefetcher
1272     // goes on requesting HWLP_BEGIN more than one time
(wrong!).
1273     // This signal is not controlled by id_ready because oth-
erwise,
1274     // in case of stall, the jump would happen at the end of
the stall.
1275     // Make hwlp_jump_o last only one cycle
1276     assign hwlp_jump_o = (hwlp_end_4_id_d && !
hwlp_end_4_id_q) ? 1'b1 : 1'b0;
1277     always_ff @(posedge clk or negedge rst_n) begin
1278       if(!rst_n) begin
1279         hwlp_end_4_id_q <= 1'b0;
1280       end else begin
1281         hwlp_end_4_id_q <= hwlp_end_4_id_d;
1282       end
1283     end
1284
1285     assign hwlp_end0_eq_pc = hwlp_end_addr_i[0] ==
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[0] - 4 == pc_id_i
1286     assign hwlp_end1_eq_pc = hwlp_end_addr_i[1] ==
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[1] - 4 == pc_id_i
1287     assign hwlp_counter0_gt_1 = hwlp_counter_i[0] > 1;
1288     assign hwlp_counter1_gt_1 = hwlp_counter_i[1] > 1;
1289     assign hwlp_counter0_eq_1 = hwlp_counter_i[0] == 1;
1290     assign hwlp_counter1_eq_1 = hwlp_counter_i[1] == 1;
1291     assign hwlp_counter0_eq_0 = hwlp_counter_i[0] == 0;
1292     assign hwlp_counter1_eq_0 = hwlp_counter_i[1] == 0;
1293     assign hwlp_end0_eq_pc_plus4 = hwlp_end_addr_i[0] ==
pc_id_i + 8; // Equivalent to hwlp_end_addr_i[0] - 4 == pc_id_i +
4
1294     assign hwlp_end1_eq_pc_plus4 = hwlp_end_addr_i[1] ==
pc_id_i + 8; // Equivalent to hwlp_end_addr_i[1] - 4 == pc_id_i +
4
1295     assign hwlp_start0_leq_pc = hwlp_start_addr_i[0] <=
pc_id_i;
1296     assign hwlp_start1_leq_pc = hwlp_start_addr_i[1] <=
pc_id_i;
1297     assign hwlp_end0_geq_pc = hwlp_end_addr_i[0] >=
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[0] - 4 >= pc_id_i
1298     assign hwlp_end1_geq_pc = hwlp_end_addr_i[1] >=
pc_id_i + 4; // Equivalent to hwlp_end_addr_i[1] - 4 >= pc_id_i;
1299     assign is_hwlp_body = ((hwlp_start0_leq_pc &&
hwlp_end0_geq_pc) && hwlp_counter0_gt_1) || ((hwlp_start1_leq_pc &&
hwlp_end1_geq_pc) && hwlp_counter1_gt_1);
1300   end else begin : gen_no_hwlp
1301     assign hwlp_jump_o = 1'b0;
1302     assign hwlp_end_4_id_q = 1'b0;
1303     assign hwlp_end0_eq_pc = 1'b0;
1304     assign hwlp_end1_eq_pc = 1'b0;
1305     assign hwlp_counter0_gt_1 = 1'b0;
1306     assign hwlp_counter1_gt_1 = 1'b0;
1307     assign hwlp_counter0_eq_1 = 1'b0;
1308     assign hwlp_counter1_eq_1 = 1'b0;
1309     assign hwlp_counter0_eq_0 = 1'b0;
1310     assign hwlp_counter1_eq_0 = 1'b0;

```

```

1318 assign hwlp_end0_eq_pc_plus4 = 1'b0;
1319 assign hwlp_end1_eq_pc_plus4 = 1'b0;
1320 assign hwlp_start0_leq_pc = 1'b0;
1321 assign hwlp_start1_leq_pc = 1'b0;
1322 assign hwlp_end0_geq_pc = 1'b0;
1323 assign hwlp_end1_geq_pc = 1'b0;
1324 assign is_hwlp_body = 1'b0;
1325
1326 end
1327
1328 endgenerate
1329
1330 ///////////////////////////////////////////////////
1331 // ____ - - - - -
1332 // / _|| | _ _ | | / _| _ - _ | _ _ _ _ |
1333 // \ _ \ | / _ ' | | | | / _ \ ' _ \ | _ / _ \
1334 // _ ) | | ( | | | | | ( ) | | | | | ( ) |
1335 // | _ / \ \ _ , | | | \ \ _ / | | \ \ _ | \ _ /
1336 //
1337 ///////////////////////////////////////////////////
1338 always_comb
1339 begin
1340   load_stall_o = 1'b0;
1341   deassert_we_o = 1'b0;
1342
1343   // deassert WE when the core is not decoding instructions
1344   if (~is_decoding_o)
1345     deassert_we_o = 1'b1;
1346
1347   // deassert WE in case of illegal instruction
1348   if (illegal_insn_i)
1349     deassert_we_o = 1'b1;
1350
1351   // Stall because of load operation
1352   if (
1353     (data_req_ex_i == 1'b1) && (regfile_we_ex_i ==
1'b1) ||
1354     (wb_ready_i == 1'b0) && (regfile_we_wb_i == 1'b1)
1355     ) &&
1356     (reg_d_ex_is_reg_a_i == 1'b1) ||
1357     (is_decoding_o && (regfile_we_id_i && !data_mis-
aligned_i) && (regfile_waddr_ex_i == regfile_alu_waddr_id_i))
1358   )
1359     begin
1360       deassert_we_o = 1'b1;
1361       load_stall_o = 1'b1;
1362     end
1363
1364   // Stall because of jr path
1365   // - always stall if a result is to be forwarded to the PC
1366   // we don't care about in which state the ctrl_fsm is as
we deassert_we
1367   // anyway when we are not in DECODE
1368   if ((ctrl_transfer_insn_in_dec_i == BRANCH_JALR) &&
1369     (((regfile_we_wb_i == 1'b1) && (reg_d_wb_is_reg_a_i ==
1'b1)) ||
1370     ((regfile_we_ex_i == 1'b1) && (reg_d_ex_is_reg_a_i ==
1'b1)) ||
1371     ((regfile_alu_we_fw_i == 1'b1) &&
1372     (reg_d_alu_is_reg_a_i == 1'b1)) ||
1373     (FPU && (apu_read_dep_for_jalr_i == 1'b1)))
1374   )
1375     begin
1376       jr_stall_o = 1'b1;
1377       deassert_we_o = 1'b1;
1378     end
1379   else
1380     begin
1381       jr_stall_o = 1'b0;
1382     end
1383   end
1384
1385   // stall because of misaligned data access
1386   assign misaligned_stall_o = data_misaligned_i;
1387
1388   // APU dependency stalls (data hazards)
1389   assign apu_stall_o = apu_read_dep_i | (apu_write_dep_i &
~apu_en_i);
1390
1391   // Forwarding control unit
1392   always_comb
1393   begin
1394     // default assignments
1395     operand_a_fw_mux_sel_o = SEL_REGFILE;
1396     operand_b_fw_mux_sel_o = SEL_REGFILE;
1397     operand_c_fw_mux_sel_o = SEL_REGFILE;
1398
1399     // Forwarding WB -> ID

```

```

1401   if (regfile_we_wb_i == 1'b1)
1402   begin
1403     if (reg_d_wb_is_reg_a_i == 1'b1)
1404       operand_a_fw_mux_sel_o = SEL_FW_WB;
1405     if (reg_d_wb_is_reg_b_i == 1'b1)
1406       operand_b_fw_mux_sel_o = SEL_FW_WB;
1407     if (reg_d_wb_is_reg_c_i == 1'b1)
1408       operand_c_fw_mux_sel_o = SEL_FW_WB;
1409   end
1410
1411   // Forwarding EX -> ID
1412   if (regfile_alu_we_fw_i == 1'b1)
1413   begin
1414     if (reg_d_alu_is_reg_a_i == 1'b1)
1415       operand_a_fw_mux_sel_o = SEL_FW_EX;
1416     if (reg_d_alu_is_reg_b_i == 1'b1)
1417       operand_b_fw_mux_sel_o = SEL_FW_EX;
1418     if (reg_d_alu_is_reg_c_i == 1'b1)
1419       operand_c_fw_mux_sel_o = SEL_FW_EX;
1420   end
1421
1422   // for misaligned memory accesses
1423   if (data_misaligned_i)
1424   begin
1425     operand_a_fw_mux_sel_o = SEL_FW_EX;
1426     operand_b_fw_mux_sel_o = SEL_REGFILE;
1427   end else if (mult_multicycle_i) begin
1428     operand_c_fw_mux_sel_o = SEL_FW_EX;
1429   end
1430 end
1431
1432 // update registers
1433 always_ff @(posedge clk , negedge rst_n)
1434 begin : UPDATE_REGS
1435   if (rst_n == 1'b0 )
1436   begin
1437     ctrl_fsm_cs <= RESET;
1438     jump_done_q <= 1'b0;
1439     data_err_q <= 1'b0;
1440
1441     debug_mode_q <= 1'b0;
1442     illegal_insn_q <= 1'b0;
1443
1444     debug_req_entry_q <= 1'b0;
1445     debug_force_wakeup_q <= 1'b0;
1446   end
1447   else
1448   begin
1449     ctrl_fsm_cs <= ctrl_fsm_ns;
1450
1451     // clear when id is valid (no instruction incoming)
1452     jump_done_q <= jump_done & (~id_ready_i);
1453
1454     data_err_q <= data_err_i;
1455
1456     debug_mode_q <= debug_mode_n;
1457
1458     illegal_insn_q <= illegal_insn_n;
1459
1460     debug_req_entry_q <= debug_req_entry_n;
1461     debug_force_wakeup_q <= debug_force_wakeup_n;
1462   end
1463 end
1464
1465 // wakeup from sleep conditions
1466 assign wake_from_sleep_o = irq_wu_ctrl_i || de-
bug_req_pending || debug_mode_q;
1467
1468 // debug mode
1469 assign debug_mode_o = debug_mode_q;
1470 assign debug_req_pending = debug_req_i || debug_req_q;
1471
1472 // Do not let cv.elw cause core_sleep_o during debug
1473 assign debug_p_elw_no_sleep_o = debug_mode_q || debug_req_q
|| debug_single_step_i || trigger_match_i;
1474
1475 // Do not let WFI cause core_sleep_o (but treat as NOP):
1476 //
1477 // - During debug
1478 // - For PULP Cluster (only cv.elw can trigger sleep)
1479
1480 assign debug_wfi_no_sleep_o = debug_mode_q || de-
bug_req_pending || debug_single_step_i || trigger_match_i ||
COREV_CLUSTER;
1481
1482 // Gate off wfi
1483 assign wfi_active = wfi_i & ~debug_wfi_no_sleep_o;
1484
1485 // sticky version of debug_req (must be on clk_ungated_i
such that incoming pulse before core is enabled is not missed)
1486 always_ff @(posedge clk_ungated_i, negedge rst_n)
1487 if (!rst_n)
1488   debug_req_q <= 1'b0;
1489 else
1490   if( debug_req_i )
1491     debug_req_q <= 1'b1;
1492   else if( debug_mode_q )
1493     debug_req_q <= 1'b0;
1494
1495 // Debug state FSM
1496 always_ff @(posedge clk , negedge rst_n)

```

```

1497 begin
1498   if ( rst_n == 1'b0 )
1499     begin
1500       debug_fsm_cs <= HAVERESET;
1501     end
1502   else
1503     begin
1504       debug_fsm_cs <= debug_fsm_ns;
1505     end
1506   end
1507
1508   always_comb
1509   begin
1510     debug_fsm_ns = debug_fsm_cs;
1511
1512     case (debug_fsm_cs)
1513       HAVERESET:
1514         begin
1515           if (debug_mode_n || (ctrl_fsm_ns == FIRST_FETCH)) be-
gin
1516             if (debug_mode_n) begin
1517               debug_fsm_ns = HALTED;
1518             end else begin
1519               debug_fsm_ns = RUNNING;
1520             end
1521           end
1522         end
1523       RUNNING:
1524         begin
1525           if (debug_mode_n) begin
1526             debug_fsm_ns = HALTED;
1527           end
1528         end
1529       end
1530       HALTED:
1531         begin
1532           if (!debug_mode_n) begin
1533             debug_fsm_ns = RUNNING;
1534           end
1535         end
1536       default: begin
1537         debug_fsm_ns = HAVERESET;
1538       end
1539     endcase
1540   end
1541
1542   assign debug_havereset_o = debug_fsm_cs[HAVERESET_INDEX];
1543   assign debug_running_o = debug_fsm_cs[RUNNING_INDEX];
1544   assign debug_halted_o = debug_fsm_cs[HALTED_INDEX];
1545
1546   //-----
1547   // Assertions
1548   //-----
1549
1550   `ifdef CV32E40P_ASSERT_ON
1551
1552   // make sure that taken branches do not happen back-to-back,
1553   // as this is not
1554   // possible without branch prediction in the IF stage
1555   assert property (
1556     @(posedge clk) (branch_taken_ex_i) |>
1557     (~branch_taken_ex_i) ) else $warning("Two branches back-to-back are
taken");
1558
1559   // ELW_EXE and IRQ_FLUSH_ELW states are only used for
1560   COREV_CLUSTER = 1
1561   property p_pulp_cluster_only_states;
1562   @(posedge clk) (1'b1) |> ( !((COREV_CLUSTER == 1'b0) &&
((ctrl_fsm_cs == ELW_EXE) || (ctrl_fsm_cs == IRQ_FLUSH_ELW))) );
1563   endproperty
1564
1565   a_pulp_cluster_only_states : assert prop-
erty(p_pulp_cluster_only_states);
1566
1567   // WAIT_SLEEP and SLEEP states are never used for
1568   COREV_CLUSTER = 1
1569   property p_pulp_cluster_excluded_states;
1570   @(posedge clk) (1'b1) |> ( !((COREV_CLUSTER == 1'b1) &&
((ctrl_fsm_cs == SLEEP) || (ctrl_fsm_cs == WAIT_SLEEP))) );
1571   endproperty
1572

```

```

1571   a_pulp_cluster_excluded_states : assert prop-
erty(p_pulp_cluster_excluded_states);
1572
1573   generate
1574   if (COREV_PULP) begin : gen_pulp_xpulp_assertions
1575
1576     // HWLoop 0 and 1 having target address constraints
1577     property p_hwlp_same_target_address;
1578     @(posedge clk) (hwlp_counter_i[1] > 1 &&
hwlp_counter_i[0] > 1 && pc_id_i >= hwlp_start_addr_i[0] && pc_id_i
<= hwlp_end_addr_i[0] - 4) |> ( hwlp_end_addr_i[1] - 4 >=
hwlp_end_addr_i[0] - 4 + 8 );
1579     endproperty
1580
1581     a_hwlp_same_target_address : assert prop-
erty(p_hwlp_same_target_address) else $warning("%t, HWLoops target
address do not respect constraints", $time);
1582
1583     end else begin : gen_no_pulp_xpulp_assertions
1584
1585       property p_no_hwlp;
1586       @(posedge clk) (1'b1) |> ((pc_mux_o != PC_HWLOOP) &&
(ctrl_fsm_cs != DECODE_HWLOOP) &&
(hwlp_mask_o == 1'b0) &&
(is_hwlp_body == 'b0) &&
(hwlp_start_addr_i == 'b0)
&& (hwlp_end_addr_i == 'b0) && (hwlp_counter_i[1] == 32'b0) &&
(hwlp_counter_i[0] == 32'b0) &&
(hwlp_dec_cnt_o == 2'b0) &&
(hwlp_jump_o == 1'b0) && (hwlp_targ_addr_o == 32'b0) &&
(hwlp_end0_eq_pc == 1'b0) && (hwlp_end0_eq_pc == 1'b0) &&
(hwlp_end1_eq_pc == 1'b0) && (hwlp_counter0_gt_1 == 1'b0) &&
(hwlp_counter1_gt_1 == 1'b0) &&
(hwlp_counter0_eq_1 == 1'b0)
&& (hwlp_counter1_eq_1 == 1'b0) &&
(hwlp_end0_eq_pc_plus4 ==
1'b0) && (hwlp_end1_eq_pc_plus4 == 1'b0) && (hwlp_start0_leq_pc ==
0) && (hwlp_start1_leq_pc == 0) &&
(hwlp_end0_geq_pc == 1'b0)
&& (hwlp_end1_geq_pc == 1'b0) && (hwlp_end4_id_d == 1'b0) &&
(hwlp_end4_id_q == 1'b0));
1590       endproperty
1591
1592       a_no_hwlp : assert property(p_no_hwlp);
1593
1594     end
1595   endgenerate
1596
1597   // Ensure DBG_TAKEN_IF can only be entered if in single
1598   step mode or woken
1599   up from sleep by debug_req_i
1600
1601   a_single_step_dbg_taken_if : assert property (@(posedge
clk) disable iff (!rst_n) (ctrl_fsm_ns==DBG_TAKEN_IF) |> ((~de-
bug_mode_q && debug_single_step_i) || debug_force_wakeup_n));
1602
1603   // Ensure DBG_FLUSH state is only one cycle. This implies
1604   that cause is either trigger, debug_req_entry, or ebreak
1605   a_dbg_flush : assert property (@(posedge clk) disable iff
(!rst_n) (ctrl_fsm_cs==DBG_FLUSH) |> (ctrl_fsm_ns!=DBG_FLUSH) );
1606
1607   // Ensure that debug state outputs are one-hot
1608   a_debug_state_onehot : assert property (@(posedge clk) $one-
hot({debug_havereset_o, debug_running_o, debug_halted_o}));
1609
1610   // Ensure that debug_halted_o equals debug_mode_q
1611   a_debug_halted_equals_debug_mode : assert property
(@(posedge clk) disable iff (!rst_n) (1'b1) |> (debug_mode_q == de-
bug_halted_o));
1612
1613   // Ensure ID always ready in FIRST_FETCH state
1614   a_first_fetch_id_ready : assert property (@(posedge clk)
disable iff (!rst_n) (ctrl_fsm_cs == FIRST_FETCH) |> (id_ready_i ==
1'b1));
1615
1616   // Ensure that the only way to get to DBG_TAKEN_IF from
1617   DBG_FLUSH is if debug_single_step_i is asserted
1618   a_dbg_flush_to_taken_if : assert property (@(posedge clk)
disable iff (!rst_n) (ctrl_fsm_cs == DBG_FLUSH) && (ctrl_fsm_ns ==
DBG_TAKEN_IF) |> debug_single_step_i);
1619
1620   `endif
1621
1622   endmodule // cv32e40p_controller

```

CV32E40P RVFI 트레이스

8 files • 4,745 lines

API — cv32e40p_apu_tracer.sv

Table 1095. openhwgroup_cv32e40p/bhv/cv32e40p_apu_tracer.sv — module / interface / package (1개)

이름	종류
cv32e40p_apu_tracer	module

Table 1096. openhwgroup_cv32e40p/bhv/cv32e40p_apu_tracer.sv — 포트 (6개)

포트	방향	형/폭
clk_i	input	logic
rst_n	input	logic
hart_id_i	input	logic [31:0]
apu_valid_i	input	logic
apu_waddr_i	input	logic [5:0]
apu_result_i	input	logic [31:0]

API — cv32e40p_core_log.sv

Table 1097. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv — module / interface / package (1개)

이름	종류
cv32e40p_core_log	module

Table 1098. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv — parameter (5개)

파라미터	형/폭	기본값
COREV_PULP		1
COREV_CLUSTER		0
FPU		0
ZFINX		0
NUM_MHPMCOUNTERS		1

Table 1099. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv — 포트 (5개)

포트	방향	형/폭
clk_i	input	logic
is_decoding_i	input	logic
illegal_insn_dec_i	input	logic
hart_id_i	input	logic [31:0]
pc_id_i	input	logic [31:0]

API — cv32e40p_rvfi.sv**Table 1100. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — module / interface / package (1개)**

이름	종류
cv32e40p_rvfi	module

Table 1101. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — parameter (3개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0
NUM_MHPMCOUNTERS		1

Table 1102. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — localparam (4개)

상수	형/폭	값
MSTATUS_WRITE_MASK	logic [31:0]	32'h0000_6088
MCOUNTINHIBIT_WRITE_MASK	logic [31:0]	{{(29-NUM_MHPMCOUNTERS){1'b0}}}
NUM_HPM_EVENTS		16
MHPMEVENT_WRITE_MASK	logic [31:0]	{{(31-NUM_HPM_EVENTS){1'b0}}}

Table 1103. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — 포트 (505개) — 앞 60개만 표시(전체 505개, 본문 소스 참조)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
is_decoding_i	input	logic
is_illegal_i	input	logic
trigger_match_i	input	logic
data_misaligned_i	input	logic
lsu_data_we_ex_i	input	logic
debug_mode_i	input	logic
debug_cause_i	input	logic [2:0]
instr_req_i	input	logic
instr_grant_i	input	logic
instr_rvalid_i	input	logic
prefetch_req_i	input	logic
pc_set_i	input	logic
if_valid_i	input	logic
if_ready_i	input	logic
instr_valid_if_i	input	logic

포트	방향	형/폭
instr_if_i	input	logic [31:0]
pc_if_i	input	logic [31:0]
instr_pmp_err_if_i	input	logic
instr_valid_id_i	input	logic
instr_rdata_id_i	input	logic [31:0]
is_fetch_failed_id_i	input	logic
instr_req_int_i	input	logic
clear_instr_valid_i	input	logic
pc_id_i	input	logic [31:0]
id_valid_i	input	logic
id_ready_i	input	logic
rf_re_id_i	input	logic [1:0]
sys_en_id_i	input	logic
sys_mret_insn_id_i	input	logic
jump_in_id_i	input	logic
jump_target_id_i	input	logic [31:0]
is_compressed_id_i	input	logic
ebrk_insn_dec_i	input	logic
ecall_insn_dec_i	input	logic
mret_insn_dec_i	input	logic
mret_dec_i	input	logic
csr_cause_i	input	logic [5:0]
debug_csr_save_i	input	logic
hwlp_start_q_i	input	logic [1:0][31:0]
hwlp_end_q_i	input	logic [1:0][31:0]
hwlp_counter_q_i	input	logic [1:0][31:0]
hwlp_counter_n_i	input	logic [1:0][31:0]
minstret_i	input	logic
lsu_en_id_i	input	logic
lsu_we_id_i	input	logic
lsu_size_id_i	input	logic [1:0]
rs1_addr_id_i	input	logic [5:0]
rs2_addr_id_i	input	logic [5:0]
rs3_addr_id_i	input	logic [5:0]
operand_a_fw_id_i	input	logic [31:0]

포트	방향	형/폭
operand_b_fw_id_i	input	logic [31:0]
operand_c_fw_id_i	input	logic [31:0]
ex_ready_i	input	logic
ex_valid_i	input	logic
ex_reg_we_i	input	logic
ex_reg_addr_i	input	logic [5:0]
ex_reg_wdata_i	input	logic [31:0]
apu_en_ex_i	input	logic

Table 1104. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv — #define / `define 매크로 (2개)

매크로	인자	정의
SET_RVFI_CSR_FROM_INSN	(CSR_NAME)	\
CSR_FROM_PIPE	(TRACE_NAME, CSR_NAME)	\

API — cv32e40p_rvfi_trace.sv

Table 1105. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv — module / interface / package (1개)

이름	종류
cv32e40p_rvfi_trace	module

Table 1106. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv — parameter (2개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0

Table 1107. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv — 포트 (33개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
hart_id_i	input	logic [31:0]
imm_s3_type	input	logic [31:0]
rvfi_valid	input	logic
rvfi_insn	input	logic [31:0]
rvfi_start_cycle	input	integer
rvfi_start_time	input	time
rvfi_stop_cycle	input	integer
rvfi_stop_time	input	time
rvfi_pc_rdata	input	logic [31:0]

포트	방향	형/폭
rvfi_trap	input	rvfi_trap_t
rvfi_2_rd	input	logic
rvfi_rs1_addr	input	logic [4:0]
rvfi_rs2_addr	input	logic [4:0]
rvfi_rs3_addr	input	logic [4:0]
rvfi_rs1_rdata	input	logic [31:0]
rvfi_rs2_rdata	input	logic [31:0]
rvfi_rs3_rdata	input	logic [31:0]
rvfi_frs1_addr	input	logic [4:0]
rvfi_frs2_addr	input	logic [4:0]
rvfi_frs3_addr	input	logic [4:0]
rvfi_frs1_rvalid	input	logic
rvfi_frs2_rvalid	input	logic
rvfi_frs3_rvalid	input	logic
rvfi_frs1_rdata	input	logic [31:0]
rvfi_frs2_rdata	input	logic [31:0]
rvfi_frs3_rdata	input	logic [31:0]
rvfi_mem_addr	input	logic [31:0]
rvfi_mem_rmask	input	logic [31:0]
rvfi_mem_wmask	input	logic [31:0]
rvfi_mem_rdata	input	logic [31:0]
rvfi_mem_wdata	input	logic [31:0]

API — cv32e40p_sim_clock_gate.sv

Table 1108. openhwgroup_cv32e40p/bhv/cv32e40p_sim_clock_gate.sv — module / interface / package (1개)

이름	종류
cv32e40p_clock_gate	module

Table 1109. openhwgroup_cv32e40p/bhv/cv32e40p_sim_clock_gate.sv — 포트 (4개)

포트	방향	형/폭
clk_i	input	logic
en_i	input	logic
scan_cg_en_i	input	logic
clk_o	output	logic

API — cv32e40p_tb_wrapper.sv**Table 1110. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv — module / interface / package (1개)**

이름	종류
cv32e40p_tb_wrapper	module

Table 1111. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv — parameter (7개)

파라미터	형/폭	기본값
COREV_PULP		0
COREV_CLUSTER		0
FPU		0
FPU_ADDMUL_LAT		0
FPU_OTHERS_LAT		0
ZFINX		0
NUM_MHPMCOUNTERS		1

Table 1112. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv — 포트 (31개)

포트	방향	형/폭
clk_i	input	logic
rst_ni	input	logic
pulp_clock_en_i	input	logic
scan_cg_en_i	input	logic
boot_addr_i	input	logic [31:0]
mtvec_addr_i	input	logic [31:0]
dm_halt_addr_i	input	logic [31:0]
hart_id_i	input	logic [31:0]
dm_exception_addr_i	input	logic [31:0]
instr_req_o	output	logic
instr_gnt_i	input	logic
instr_rvalid_i	input	logic
instr_addr_o	output	logic [31:0]
instr_rdata_i	input	logic [31:0]
data_req_o	output	logic
data_gnt_i	input	logic
data_rvalid_i	input	logic
data_we_o	output	logic
data_be_o	output	logic [3:0]
data_addr_o	output	logic [31:0]

포트	방향	형/폭
data_wdata_o	output	logic [31:0]
data_rdata_i	input	logic [31:0]
irq_i	input	logic [31:0]
irq_ack_o	output	logic
irq_id_o	output	logic [4:0]
debug_req_i	input	logic
debug_havereset_o	output	logic
debug_running_o	output	logic
debug_halted_o	output	logic
fetch_enable_i	input	logic
core_sleep_o	output	logic

API — cv32e40p_tracer.sv

Table 1113. openhwgroup_cv32e40p/bhv/cv32e40p_tracer.sv — module / interface / package (1개)

이름	종류
cv32e40p_tracer	module

Table 1114. openhwgroup_cv32e40p/bhv/cv32e40p_tracer.sv — parameter (2개)

파라미터	형/폭	기본값
FPU		0
ZFINX		0

Table 1115. openhwgroup_cv32e40p/bhv/cv32e40p_tracer.sv — 포트 (54개)

포트	방향	형/폭
clk_i	input	logic
rst_n	input	logic
hart_id_i	input	logic [31:0]
pc	input	logic [31:0]
instr	input	logic [31:0]
controller_state_i	input	ctrl_state_e
compressed	input	logic
id_valid	input	logic
is_decoding	input	logic
is_illegal	input	logic
trigger_match	input	logic
rs1_value	input	logic [31:0]

포트	방향	형/폭
rs2_value	input	logic [31:0]
rs3_value	input	logic [31:0]
rs2_value_vec	input	logic [31:0]
rd_is_fp	input	logic
rs1_is_fp	input	logic
rs2_is_fp	input	logic
rs3_is_fp	input	logic
ex_valid	input	logic
ex_reg_addr	input	logic [5:0]
ex_reg_we	input	logic
ex_reg_wdata	input	logic [31:0]
ex_data_req	input	logic
ex_data_gnt	input	logic
ex_data_we	input	logic
ex_data_addr	input	logic [31:0]
ex_data_wdata	input	logic [31:0]
data_misaligned	input	logic
ebrk_insn	input	logic
debug_mode	input	logic
ebrk_force_debug_mode	input	logic
wb_bypass	input	logic
wb_valid	input	logic
wb_reg_addr	input	logic [5:0]
wb_reg_we	input	logic
wb_reg_wdata	input	logic [31:0]
imm_u_type	input	logic [31:0]
imm_uj_type	input	logic [31:0]
imm_i_type	input	logic [31:0]
imm_iz_type	input	logic [11:0]
imm_z_type	input	logic [31:0]
imm_s_type	input	logic [31:0]
imm_sb_type	input	logic [31:0]
imm_s2_type	input	logic [31:0]
imm_s3_type	input	logic [31:0]
imm_vs_type	input	logic [31:0]

포트	방향	형/폭
imm_vu_type	input	logic [31:0]
imm_shuffle_type	input	logic [31:0]
imm_clip_type	input	logic [4:0]
apu_en_i	input	logic
apu_singlecycle_i	input	logic
apu_multicycle_i	input	logic
apu_rvalid_i	input	logic

API — insn_trace.sv

Table 1116. openhwgroup_cv32e40p/bhv/insn_trace.sv — #define / `define 매크로 (3개)

매크로	인자	정의
DEFINE_CSR	(CSR_NAME)	\
ASSIGN_CSR	(CSR_NAME)	\
INIT_CSR	(CSR_NAME)	\

```

1. openhwgroup_cv32e40p/bhv/cv32e40p_apu_tracer.sv (79
lines)
1 // Copyright 2020 Silicon Labs, Inc.
2 //
3 // This file, and derivatives thereof are licensed under the
4 // Solderpad License, Version 2.0 (the "License").
5 //
6 // Use of this file means you agree to the terms and condi-
7 // tions of the license and are in full compliance with the License.
8 //
9 // You may obtain a copy of the License at:
10 //
11 //     https://solderpad.org/licenses/SHL-2.0/
12 //
13 // Unless required by applicable law or agreed to in writing,
14 // software and hardware implementations thereof distributed under the
15 // License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
16 // CONDITIONS
17 // OF ANY KIND, EITHER EXPRESSED OR IMPLIED.
18 // See the License for the specific language governing permis-
19 // sions and
20 // limitations under the License.
21 //
22 // Engineer:      Arjan Bink - ar-
23 // jan.bink@silabs.com
24 //
25 // Design Name:   cv32e40p_apu_tracer.sv (APU
26 // trace)
27 // Project Name:  CV32E40P
28 // Language:      SystemVeri-
29 // log
30 //
31 // Description:   Logs the follow-
32 // ing:
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //

```

```

33 // Note:          This code was here from cv32e40p_core.sv in
34 // order to        //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //

```

```

78
79 `endif // CV32E40P_APU_TRACE

```

2. openhwgroup_cv32e40p/bhv/cv32e40p_core_log.sv (68 lines)

```

1 // Copyright 2020 Silicon Labs, Inc.
2 //
3 // This file, and derivatives thereof are licensed under the
4 // Solderpad License, Version 2.0 (the "License").
5 //
6 // Use of this file means you agree to the terms and condi-
7 // tions of the license and are in full compliance with the License.
8 //
9 // You may obtain a copy of the License at:
10 // https://solderpad.org/licenses/SHL-2.0/
11 //
12 // Unless required by applicable law or agreed to in writing,
13 // software and hardware implementations thereof distributed under the
14 // License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
15 // CONDITIONS OF ANY KIND, EITHER EXPRESSED OR IMPLIED.
16 // See the License for the specific language governing permis-
17 // sions and limitations under the License.
18 //
19 //
20 //
21 //
22 // Engineer: Arjan Bink - ar-
23 // jan.bink@silabs.com
24 //
25 // Design Name: cv32e40p_core_log.sv (cv32e40p_core simula-
26 // tion log)
27 // Project Name: CV32E40P
28 // Language: SystemVeri-
29 // log
30 //
31 // Description: Logs the follow-
32 // ing:
33 //
34 // - top level parameter set-
35 // tings
36 // - illegal instruc-
37 // tions
38 //
39 // Note: This code was here from cv32e40p_core.sv
40 // and cv32e40p_controller.sv in order to remove
41 // the use of global defines in the RTL
42 // code.
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
111 //
112 //
113 //
114 //
115 //
116 //
117 //
118 //
119 //
120 //
121 //
122 //
123 //
124 //
125 //
126 //
127 //
128 //
129 //
130 //
131 //
132 //
133 //
134 //
135 //
136 //
137 //
138 //
139 //
140 //
141 //
142 //
143 //
144 //
145 //
146 //
147 //
148 //
149 //
150 //
151 //
152 //
153 //
154 //
155 //
156 //
157 //
158 //
159 //
160 //
161 //
162 //
163 //
164 //
165 //
166 //
167 //
168 //
169 //
170 //
171 //
172 //
173 //
174 //
175 //
176 //
177 //
178 //
179 //
180 //
181 //
182 //
183 //
184 //
185 //
186 //
187 //
188 //
189 //
190 //
191 //
192 //
193 //
194 //
195 //
196 //
197 //
198 //
199 //
200 //
201 //
202 //
203 //
204 //
205 //
206 //
207 //
208 //
209 //
210 //
211 //
212 //
213 //
214 //
215 //
216 //
217 //
218 //
219 //
220 //
221 //
222 //
223 //
224 //
225 //
226 //
227 //
228 //
229 //
230 //
231 //
232 //
233 //
234 //
235 //
236 //
237 //
238 //
239 //
240 //
241 //
242 //
243 //
244 //
245 //
246 //
247 //
248 //
249 //
250 //
251 //
252 //
253 //
254 //
255 //
256 //
257 //
258 //
259 //
260 //
261 //
262 //
263 //
264 //
265 //
266 //
267 //
268 //
269 //
270 //
271 //
272 //
273 //
274 //
275 //
276 //
277 //
278 //
279 //
280 //
281 //
282 //
283 //
284 //
285 //
286 //
287 //
288 //
289 //
290 //
291 //
292 //
293 //
294 //
295 //
296 //
297 //
298 //
299 //
300 //
301 //
302 //
303 //
304 //
305 //
306 //
307 //
308 //
309 //
310 //
311 //
312 //
313 //
314 //
315 //
316 //
317 //
318 //
319 //
320 //
321 //
322 //
323 //
324 //
325 //
326 //
327 //
328 //
329 //
330 //
331 //
332 //
333 //
334 //
335 //
336 //
337 //
338 //
339 //
340 //
341 //
342 //
343 //
344 //
345 //
346 //
347 //
348 //
349 //
350 //
351 //
352 //
353 //
354 //
355 //
356 //
357 //
358 //
359 //
360 //
361 //
362 //
363 //
364 //
365 //
366 //
367 //
368 //
369 //
370 //
371 //
372 //
373 //
374 //
375 //
376 //
377 //
378 //
379 //
380 //
381 //
382 //
383 //
384 //
385 //
386 //
387 //
388 //
389 //
390 //
391 //
392 //
393 //
394 //
395 //
396 //
397 //
398 //
399 //
400 //
401 //
402 //
403 //
404 //
405 //
406 //
407 //
408 //
409 //
410 //
411 //
412 //
413 //
414 //
415 //
416 //
417 //
418 //
419 //
420 //
421 //
422 //
423 //
424 //
425 //
426 //
427 //
428 //
429 //
430 //
431 //
432 //
433 //
434 //
435 //
436 //
437 //
438 //
439 //
440 //
441 //
442 //
443 //
444 //
445 //
446 //
447 //
448 //
449 //
450 //
451 //
452 //
453 //
454 //
455 //
456 //
457 //
458 //
459 //
460 //
461 //
462 //
463 //
464 //
465 //
466 //
467 //
468 //
469 //
470 //
471 //
472 //
473 //
474 //
475 //
476 //
477 //
478 //
479 //
480 //
481 //
482 //
483 //
484 //
485 //
486 //
487 //
488 //
489 //
490 //
491 //
492 //
493 //
494 //
495 //
496 //
497 //
498 //
499 //
500 //
501 //
502 //
503 //
504 //
505 //
506 //
507 //
508 //
509 //
510 //
511 //
512 //
513 //
514 //
515 //
516 //
517 //
518 //
519 //
520 //
521 //
522 //
523 //
524 //
525 //
526 //
527 //
528 //
529 //
530 //
531 //
532 //
533 //
534 //
535 //
536 //
537 //
538 //
539 //
540 //
541 //
542 //
543 //
544 //
545 //
546 //
547 //
548 //
549 //
550 //
551 //
552 //
553 //
554 //
555 //
556 //
557 //
558 //
559 //
560 //
561 //
562 //
563 //
564 //
565 //
566 //
567 //
568 //
569 //
570 //
571 //
572 //
573 //
574 //
575 //
576 //
577 //
578 //
579 //
580 //
581 //
582 //
583 //
584 //
585 //
586 //
587 //
588 //
589 //
590 //
591 //
592 //
593 //
594 //
595 //
596 //
597 //
598 //
599 //
600 //
601 //
602 //
603 //
604 //
605 //
606 //
607 //
608 //
609 //
610 //
611 //
612 //
613 //
614 //
615 //
616 //
617 //
618 //
619 //
620 //
621 //
622 //
623 //
624 //
625 //
626 //
627 //
628 //
629 //
630 //
631 //
632 //
633 //
634 //
635 //
636 //
637 //
638 //
639 //
640 //
641 //
642 //
643 //
644 //
645 //
646 //
647 //
648 //
649 //
650 //
651 //
652 //
653 //
654 //
655 //
656 //
657 //
658 //
659 //
660 //
661 //
662 //
663 //
664 //
665 //
666 //
667 //
668 //
669 //
670 //
671 //
672 //
673 //
674 //
675 //
676 //
677 //
678 //
679 //
680 //
681 //
682 //
683 //
684 //
685 //
686 //
687 //
688 //
689 //
690 //
691 //
692 //
693 //
694 //
695 //
696 //
697 //
698 //
699 //
700 //
701 //
702 //
703 //
704 //
705 //
706 //
707 //
708 //
709 //
710 //
711 //
712 //
713 //
714 //
715 //
716 //
717 //
718 //
719 //
720 //
721 //
722 //
723 //
724 //
725 //
726 //
727 //
728 //
729 //
730 //
731 //
732 //
733 //
734 //
735 //
736 //
737 //
738 //
739 //
740 //
741 //
742 //
743 //
744 //
745 //
746 //
747 //
748 //
749 //
750 //
751 //
752 //
753 //
754 //
755 //
756 //
757 //
758 //
759 //
760 //
761 //
762 //
763 //
764 //
765 //
766 //
767 //
768 //
769 //
770 //
771 //
772 //
773 //
774 //
775 //
776 //
777 //
778 //
779 //
780 //
781 //
782 //
783 //
784 //
785 //
786 //
787 //
788 //
789 //
790 //
791 //
792 //
793 //
794 //
795 //
796 //
797 //
798 //
799 //
800 //
801 //
802 //
803 //
804 //
805 //
806 //
807 //
808 //
809 //
810 //
811 //
812 //
813 //
814 //
815 //
816 //
817 //
818 //
819 //
820 //
821 //
822 //
823 //
824 //
825 //
826 //
827 //
828 //
829 //
830 //
831 //
832 //
833 //
834 //
835 //
836 //
837 //
838 //
839 //
840 //
841 //
842 //
843 //
844 //
845 //
846 //
847 //
848 //
849 //
850 //
851 //
852 //
853 //
854 //
855 //
856 //
857 //
858 //
859 //
860 //
861 //
862 //
863 //
864 //
865 //
866 //
867 //
868 //
869 //
870 //
871 //
872 //
873 //
874 //
875 //
876 //
877 //
878 //
879 //
880 //
881 //
882 //
883 //
884 //
885 //
886 //
887 //
888 //
889 //
890 //
891 //
892 //
893 //
894 //
895 //
896 //
897 //
898 //
899 //
900 //
901 //
902 //
903 //
904 //
905 //
906 //
907 //
908 //
909 //
910 //
911 //
912 //
913 //
914 //
915 //
916 //
917 //
918 //
919 //
920 //
921 //
922 //
923 //
924 //
925 //
926 //
927 //
928 //
929 //
930 //
931 //
932 //
933 //
934 //
935 //
936 //
937 //
938 //
939 //
940 //
941 //
942 //
943 //
944 //
945 //
946 //
947 //
948 //
949 //
950 //
951 //
952 //
953 //
954 //
955 //
956 //
957 //
958 //
959 //
960 //
961 //
962 //
963 //
964 //
965 //
966 //
967 //
968 //
969 //
970 //
971 //
972 //
973 //
974 //
975 //
976 //
977 //
978 //
979 //
980 //
981 //
982 //
983 //
984 //
985 //
986 //
987 //
988 //
989 //
990 //
991 //
992 //
993 //
994 //
995 //
996 //
997 //
998 //
999 //
1000 //

```

```

60 // Log illegal instructions
61 always_ff @(negedge clk_i) begin
62 // print warning in case of decoding errors
63 if (is_decoding_i && illegal_insn_dec_i) begin
64 $display("%t: Illegal instruction (core %0d) at PC
0x%h:", $time, hart_id_i[3:0], pc_id_i);
65 end
66 end
67
68 endmodule // cv32e40p_core_log

```

3. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi.sv (2,062 lines)

```

1 // Copyright 2024 Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
5 // "License");
6 // you may not use this file except in compliance with the Li-
7 // cense, or,
8 // at your option, the Apache License version 2.0.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/SHL-2.1/
11 // Unless required by applicable law or agreed to in writing,
12 // any work distributed under the License is distributed on an "AS IS"
13 // BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
15 // press or implied.
16 // See the License for the specific language governing permis-
17 // sions and limitations under the License.
18 //
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
111 //
112 //
113 //
114 //
115 //
116 //
117 //
118 //
119 //
120 //
121 //
122 //
123 //
124 //
125 //
126 //
127 //
128 //
129 //
130 //
131 //
132 //
133 //
134 //
135 //
136 //
137 //
138 //
139 //
140 //
141 //
142 //
143 //
144 //
145 //
146 //
147 //
148 //
149 //
150 //
151 //
152 //
153 //
154 //
155 //
156 //
157 //
158 //
159 //
160 //
161 //
162 //
163 //
164 //
165 //
166 //
167 //
168 //
169 //
170 //
171 //
172 //
173 //
174 //
175 //
176 //
177 //
178 //
179 //
180 //
181 //
182 //
183 //
184 //
185 //
186 //
187 //
188 //
189 //
190 //
191 //
192 //
193 //
194 //
195 //
196 //
197 //
198 //
199 //
200 //
201 //
202 //
203 //
204 //
205 //
206 //
207 //
208 //
209 //
210 //
211 //
212 //
213 //
214 //
215 //
216 //
217 //
218 //
219 //
220 //
221 //
222 //
223 //
224 //
225 //
226 //
227 //
228 //
229 //
230 //
231 //
232 //
233 //
234 //
235 //
236 //
237 //
238 //
239 //
240 //
241 //
242 //
243 //
244 //
245 //
246 //
247 //
248 //
249 //
250 //
251 //
252 //
253 //
254 //
255 //
256 //
257 //
258 //
259 //
260 //
261 //
262 //
263 //
264 //
265 //
266 //
267 //
268 //
269 //
270 //
271 //
272 //
273 //
274 //
275 //
276 //
277 //
278 //
279 //
280 //
281 //
282 //
283 //
284 //
285 //
286 //
287 //
288 //
289 //
290 //
291 //
292 //
293 //
294 //
295 //
296 //
297 //
298 //
299 //
300 //
301 //
302 //
303 //
304 //
305 //
306 //
307 //
308 //
309 //
310 //
311 //
312 //
313 //
314 //
315 //
316 //
317 //
318 //
319 //
320 //
321 //
322 //
323 //
324 //
325 //
326 //
327 //
328 //
329 //
330 //
331 //
332 //
333 //
334 //
335 //
336 //
337 //
338 //
339 //
340 //
341 //
342 //
343 //
344 //
345 //
346 //
347 //
348 //
349 //
350 //
351 //
352 //
353 //
354 //
355 //
356 //
357 //
358 //
359 //
360 //
361 //
362 //
363 //
364 //
365 //
366 //
367 //
368 //
369 //
370 //
371 //
372 //
373 //
374 //
375 //
376 //
377 //
378 //
379 //
380 //
381 //
382 //
383 //
384 //
385 //
386 //
387 //
388 //
389 //
390 //
391 //
392 //
393 //
394 //
395 //
396 //
397 //
398 //
399 //
400 //
401 //
402 //
403 //
404 //
405 //
406 //
407 //
408 //
409 //
410 //
411 //
412 //
413 //
414 //
415 //
416 //
417 //
418 //
419 //
420 //
421 //
422 //
423 //
424 //
425 //
426 //
427 //
428 //
429 //
430 //
431 //
432 //
433 //
434 //
435 //
436 //
437 //
438 //
439 //
440 //
441 //
442 //
443 //
444 //
445 //
446 //
447 //
448 //
449 //
450 //
451 //
452 //
453 //
454 //
455 //
456 //
457 //
458 //
459 //
460 //
461 //
462 //
463 //
464 //
465 //
466 //
467 //
468 //
469 //
470 //
471 //
472 //
473 //
474 //
475 //
476 //
477 //
478 //
479 //
480 //
481 //
482 //
483 //
484 //
485 //
486 //
487 //
488 //
489 //
490 //
491 //
492 //
493 //
494 //
495 //
496 //
497 //
498 //
499 //
500 //
501 //
502 //
503 //
504 //
505 //
506 //
507 //
508 //
509 //
510 //
511 //
512 //
513 //
514 //
515 //
516 //
517 //
518 //
519 //
520 //
521 //
522 //
523 //
524 //
525 //
526 //
527 //
528 //
529 //
530 //
531 //
532 //
533 //
534 //
535 //
536 //
537 //
538 //
539 //
540 //
541 //
542 //
543 //
544 //
545 //
546 //
547 //
548 //
549 //
550 //
551 //
552 //
553 //
554 //
555 //
556 //
557 //
558 //
559 //
560 //
561 //
562 //
563 //
564 //
565 //
566 //
567 //
568 //
569 //
570 //
571 //
572 //
573 //
574 //
575 //
576 //
577 //
578 //
579 //
580 //
581 //
582 //
583 //
584 //
585 //
586 //
587 //
588 //
589 //
590 //
591 //
592 //
593 //
594 //
595 //
596 //
597 //
598 //
599 //
600 //
601 //
602 //
603 //
604 //
605 //
606 //
607 //
608 //
609 //
610 //
611 //
612 //
613 //
614 //
615 //
616 //
617 //
618 //
619 //
620 //
621 //
622 //
623 //
624 //
625 //
626 //
627 //
628 //
629 //
630 //
631 //
632 //
633 //
634 //
635 //
636 //
637 //
638 //
639 //
640 //
641 //
642 //
643 //
644 //
645 //
646 //
647 //
648 //
649 //
650 //
651 //
652 //
653 //
654 //
655 //
656 //
657 //
658 //
659 //
660 //
661 //
662 //
663 //
664 //
665 //
666 //
667 //
668 //
669 //
670 //
671 //
672 //
673 //
674 //
675 //
676 //
677 //
678 //
679 //
680 //
681 //
682 //
683 //
684 //
685 //
686 //
687 //
688 //
689 //
690 //
691 //
692 //
693 //
694 //
695 //
696 //
697 //
698 //
699 //
700 //
701 //
702 //
703 //
704 //
705 //
706 //
707 //
708 //
709 //
710 //
711 //
712 //
713 //
714 //
715 //
716 //
717 //
718 //
719 //
720 //
721 //
722 //
723 //
724 //
725 //
726 //
727 //
728 //
729 //
730 //
731 //
732 //
733 //
734 //
735 //
736 //
737 //
738 //
739 //
740 //
741 //
742 //
743 //
744 //
745 //
746 //
747 //
748 //
749 //
750 //
751 //
752 //
753 //
754 //
755 //
756 //
757 //
758 //
759 //
760 //
761 //
762 //
763 //
764 //
765 //
766 //
767 //
768 //
769 //
770 //
771 //
772 //
773 //
774 //
775 //
776 //
777 //
778 //
779 //
780 //
781 //
782 //
783 //
784 //
785 //
786 //
787 //
788 //
789 //
790 //
791 //
792 //
793 //
794 //
795 //
796 //
797 //
798 //
799 //
800 //
801 //
802 //
803 //
804 //
805 //
806 //
807 //
808 //
809 //
810 //
811 //
812 //
813 //
814 //
815 //
816 //
817 //
818 //
819 //
820 //
821 //
822 //
823 //
824 //
825 //
826 //
827 //
828 //
829 //
830 //
831 //
832 //
833 //
834 //
835 //
836 //
837 //
838 //
839 //
840 //
841 //
842 //
843 //
844 //
845 //
846 //
847 //
848 //
849 //
850 //
851 //
852 //
853 //
854 //
855 //
856 //
857 //
858 //
859 //
860 //
861 //
862 //
863 //
864 //
865 //
866 //
867 //
868 //
869 //
870 //
871 //
872 //
873 //
874 //
875 //
876 //
877 //
878 //
879 //
880 //
881 //
882 //
883 //
884 //
885 //
886 //
887 //
888 //
889 //
890 //
891 //
892 //
893 //
894 //
895 //
896 //
897 //
898 //
899 //
900 //
901 //
902 //
903 //
904 //
905 //
906 //
907 //
908 //
909 //
910 //
911 //
912 //
913 //
914 //
915 //
916 //
917 //
918 //
919 //
920 //
921 //
922 //
923 //
924 //
925 //
926 //
927 //
928 //
929 //
930 //
931 //
932 //
933 //
934 //
935 //
936 //
937 //
938 //
939 //
940 //
941 //
942 //
943 //
944 //
945 //
946 //
947 //
948 //
949 //
950 //
951 //
952 //
953 //
954 //
955 //
956 //
957 //
958 //
959 //
960 //
961 //
962 //
963 //
964 //
965 //
966 //
967 //
968 //
969 //
970 //
971 //
972 //
973 //
974 //
975 //
976 //
977 //
978 //
979 //
980 //
981 //
982 //
983 //
984 //
985 //
986 //
987 //
988 //
989 //
990 //
991 //
992 //
993 //
994 //
995 //
996 //
997 //
998 //
999 //
1000 //

```



```

70  input logic      id_valid_i,
71
72  input logic      id_ready_i,
73  input logic [ 1:0] rf_re_id_i,
74  input logic      sys_en_id_i,
75  input logic      sys_mret_insn_id_i,
76  input logic      jump_in_id_i,
77  input logic [31:0] jump_target_id_i,
78  input logic      is_compressed_id_i,
79
80  input logic      ebrk_insn_dec_i,
81  input logic      ecall_insn_dec_i,
82
83  input logic      mret_insn_dec_i,
84  input logic      mret_dec_i,
85
86  input logic [5:0] csr_cause_i,
87
88  input logic      debug_csr_save_i,
89  // HWLOOP regs
90  input logic [ 1:0][31:0] hwl_start_q_i,
91  input logic [ 1:0][31:0] hwl_end_q_i,
92  input logic [ 1:0][31:0] hwl_counter_q_i,
93  input logic [ 1:0][31:0] hwl_counter_n_i,
94  input logic      minstret_i,
95  // LSU
96  input logic      lsu_en_id_i,
97  input logic      lsu_we_id_i,
98  input logic [ 1:0] lsu_size_id_i,
99  // Register reads
100 input logic [ 5:0] rs1_addr_id_i,
101 input logic [ 5:0] rs2_addr_id_i,
102 input logic [ 5:0] rs3_addr_id_i,
103 input logic [31:0] operand_a_fw_id_i,
104 input logic [31:0] operand_b_fw_id_i,
105 input logic [31:0] operand_c_fw_id_i,
106
107  /// EX probes ///
108
109  // Register writes in EX
110  input logic      ex_ready_i,
111  input logic      ex_valid_i,
112
113  input logic      ex_reg_we_i,
114  input logic [ 5:0] ex_reg_addr_i,
115  input logic [31:0] ex_reg_wdata_i,
116
117  input logic      apu_en_ex_i,
118  input logic      apu_singlecycle_i,
119  input logic      apu_multicycle_i,
120  input logic      wb_contention_lsu_i,
121  input logic      wb_contention_i,
122  input logic      regfile_we_lsu_i,
123
124  input logic      branch_in_ex_i,
125  input logic      branch_decision_ex_i,
126  input logic      dret_in_ex_i,
127  // LSU
128  input logic      lsu_en_ex_i,
129  input logic      lsu_pmp_err_ex_i,
130  input logic      lsu_pma_err_atomic_ex_i,
131
132  input logic [31:0] branch_target_ex_i,
133
134  input logic [31:0] data_addr_ex_i,
135  input logic [31:0] data_wdata_ex_i,
136  input logic      lsu_split_q_ex_i,
137
138  input logic      mult_ready_i,
139  input logic      alu_ready_i,
140
141  /// WB probes ///
142  input logic [31:0] pc_wb_i,
143  input logic      wb_ready_i,
144  input logic      wb_valid_i,
145  input logic      ebreak_in_wb_i,
146  input logic [31:0] instr_rdata_wb_i,
147  input logic      csr_en_wb_i,
148  input logic      sys_wfi_insn_wb_i,
149  // Register writes
150  input logic      rf_we_wb_i,
151  input logic [ 5:0] rf_addr_wb_i,
152  input logic [31:0] rf_wdata_wb_i,
153  input logic      regfile_alu_we_ex_i,
154  // LSU
155  input logic [31:0] lsu_rdata_wb_i,
156
157  input logic      data_we_ex_i,
158  input logic [ 5:0] data_atop_ex_i,
159  input logic [ 1:0] data_type_ex_i,
160  input logic [31:0] alu_operand_c_ex_i,
161  input logic [ 1:0] data_reg_offset_ex_i,
162  input logic      data_load_event_ex_i,
163  input logic [ 1:0] data_sign_ext_ex_i,
164  input logic [31:0] lsu_rdata_i,
165  input logic      data_req_ex_i,
166  input logic [31:0] alu_operand_a_ex_i,
167  input logic [31:0] alu_operand_b_ex_i,
168  input logic      useincr_addr_ex_i,
169  input logic      data_misaligned_ex_i,
170  input logic      p_elw_start_i,
171
172  input logic      p_elw_finish_i,
173  input logic      lsu_ready_ex_i,
174  input logic      lsu_ready_wb_i,
175
176  input logic [3:0] lsu_data_be_i,
177
178  input logic      data_req_pmp_i,
179  input logic      data_gnt_pmp_i,
180  input logic      data_rvalid_i,
181  input logic      data_err_pmp_i,
182  input logic [31:0] data_addr_pmp_i,
183  input logic      data_we_i,
184  input logic [ 5:0] data_atop_i,
185  input logic [ 3:0] data_be_i,
186  input logic [31:0] data_wdata_i,
187  input logic [31:0] data_rdata_i,
188
189  /// APU ///
190  input logic      apu_req_i,
191  input logic      apu_gnt_i,
192  input logic      apu_rvalid_i,
193
194  // PC //
195  input logic [31:0] branch_addr_n_i,
196
197  // Controller FSM probes
198  input ctrl_state_e ctrl_fsm_cs_i,
199  input ctrl_state_e ctrl_fsm_ns_i,
200  input logic      pending_single_step_i,
201  input logic      single_step_allowed_i,
202  input logic      pending_debug_i,
203  input logic      debug_mode_q_i,
204  input logic [3:0] pc_mux_i,
205  input logic [2:0] exc_pc_mux_i,
206
207  // Interrupt Controller probes
208  input logic [31:0] irq_i,
209  input logic      irq_wu_ctrl_i,
210  input logic [ 4:0] irq_id_ctrl_i,
211
212  /// CSR Probes ///
213  input csr_num_e   csr_addr_i,
214  input logic      csr_we_i,
215  input logic [31:0] csr_wdata_int_i,
216
217  input logic      csr_fregs_we_i,
218  input logic      csr_jvt_we_i,
219  input Status_t   csr_mstatus_ns_i,
220  input Status_t   csr_mstatus_q_i,
221  input FS_t       csr_mstatus_fs_n_i,
222  input FS_t       csr_mstatus_fs_q_i,
223
224  input logic      csr_mstathsh_we_i,
225  input logic [31:0] csr_misa_n_i,
226  input logic [31:0] csr_misa_q_i,
227
228  input logic [31:0] csr_mie_n_i,
229  input logic [31:0] csr_mie_q_i,
230  input logic      csr_mie_we_i,
231  input logic [23:0] csr_mtvec_n_i,
232  input logic [23:0] csr_mtvec_q_i,
233  input logic [ 1:0] csr_mtvec_mode_n_i,
234  input logic [ 1:0] csr_mtvec_mode_q_i,
235
236  input logic      csr_mtv_t_we_i,
237  input logic [31:0] csr_mcountinhbit_n_i,
238  input logic [31:0] csr_mcountinhbit_q_i,
239  input logic      csr_mcountinhbit_we_i,
240  input logic [31:0][31:0] csr_mhpmevent_n_i,
241  input logic [31:0][31:0] csr_mhpmevent_q_i,
242  input logic [31:0] csr_mhpmevent_we_i,
243  input logic [31:0] csr_mscratch_n_i,
244  input logic [31:0] csr_mscratch_q_i,
245  input logic [31:0] csr_mepc_n_i,
246  input logic [31:0] csr_mepc_q_i,
247
248  input logic [ 5:0] csr_mcause_n_i,
249  input logic [ 5:0] csr_mcause_q_i,
250  input logic [31:0] csr_mip_n_i,
251  input logic [31:0] csr_mip_q_i,
252  input logic      csr_mip_we_i,
253  input logic [31:0] csr_mnxti_n_i,
254  input logic [31:0] csr_mnxti_q_i,
255  input logic      csr_mnxti_we_i,
256
257  input logic      csr_mintstatus_we_i,
258  input logic [31:0] csr_mintthresh_n_i,
259  input logic [31:0] csr_mintthresh_q_i,
260  input logic      csr_mintthresh_we_i,
261  input logic [31:0] csr_mscratchcs_n_i,
262  input logic [31:0] csr_mscratchcs_q_i,
263  input logic      csr_mscratchcs_we_i,
264  input logic [31:0] csr_mscratchcs_n_i,
265  input logic [31:0] csr_mscratchcs_q_i,
266  input logic      csr_mscratchcs_we_i,
267  input logic [31:0] csr_mclibbase_n_i,
268  input logic [31:0] csr_mclibbase_q_i,
269  input logic      csr_mclibbase_we_i,
270  input logic [31:0] csr_tdata1_n_i,
271  input logic [31:0] csr_tdata1_q_i,

```

```

272 input logic      csr_tdata1_we_i,
273 input logic [31:0] csr_tdata2_n_i,
274 input logic [31:0] csr_tdata2_q_i,
275 input logic      csr_tdata2_we_i,
276 input logic [31:0] csr_tinfo_n_i,
277 input logic [31:0] csr_tinfo_q_i,
278
279 input Dcsr_t csr_dcsr_n_i,
280 input Dcsr_t csr_dcsr_q_i,
281
282 input logic [31:0] csr_dpc_n_i,
283 input logic [31:0] csr_dpc_q_i,
284 input logic      csr_dpc_we_i,
285 input logic [31:0] csr_dscratch0_n_i,
286 input logic [31:0] csr_dscratch0_q_i,
287 input logic      csr_dscratch0_we_i,
288 input logic [31:0] csr_dscratch1_n_i,
289 input logic [31:0] csr_dscratch1_q_i,
290 input logic      csr_dscratch1_we_i,
291 input logic [31:0] csr_mconfigptr_n_i,
292 input logic [31:0] csr_mconfigptr_q_i,
293 input logic      csr_mconfigptr_we_i,
294
295 // performance counters
296 // cycle, instret, hpcounter, cycleh, instreth, hp-
counterh
297 // mcycle, minstret, mhpcounter, mcycleh, minstreth, mhp-
counterh
298 input logic [63:0][MHPMCOUNTER_WIDTH-1:0] csr_mhpm-
counter_q_i,
299 input logic [31:0]      csr_mhpm-
counter_write_lower_i,
300 input logic [31:0]      csr_mhpm-
counter_write_upper_i,
301
302 input logic [31:0] csr_mvendid_i,
303 input logic [31:0] csr_marchid_i,
304 input logic [31:0] csr_mhartid_i,
305 input logic [31:0] csr_mimpid_i,
306
307 input logic [31:0] csr_mcounteren_n_i,
308 input logic [31:0] csr_mcounteren_q_i,
309 input logic      csr_mcounteren_we_i,
310
311 input logic [ 7:0] csr_pmpcfg_n_i [16],
312 input logic [ 7:0] csr_pmpcfg_q_i [16],
313 input logic [15:0] csr_pmpcfg_we_i,
314 input logic [31:0] csr_pmpaddr_n_i, // PMP address input
shared for all pmpaddr registers
315 input logic [31:0] csr_pmpaddr_q_i [16],
316 input logic [15:0] csr_pmpaddr_we_i,
317 input logic [31:0] csr_mseccfg_n_i,
318 input logic [31:0] csr_mseccfg_q_i,
319 input logic      csr_mseccfg_we_i,
320 input logic [31:0] csr_mseccfg_n_i,
321 input logic [31:0] csr_mseccfg_q_i,
322 input logic      csr_mseccfg_we_i,
323
324 input logic [4:0] csr_fcsr_fflags_n_i,
325 input logic [4:0] csr_fcsr_fflags_q_i,
326 input logic      csr_fcsr_fflags_we_i,
327 input logic [2:0] csr_fcsr_frm_n_i,
328 input logic [2:0] csr_fcsr_frm_q_i,
329
330 // RISC-V Formal Interface
331 // Does not comply with the coding standards of _i/_o suf-
fixes, but follow,
332 // the convention of RISC-V Formal Interface Specifica-
tion.
333 output logic [ 0:0] rvfi_valid,
334 output logic [63:0] rvfi_order,
335 output integer rvfi_start_cycle,
336 output time rvfi_start_time,
337 output integer rvfi_stop_cycle,
338 output time rvfi_stop_time,
339 output logic [31:0] rvfi_insn,
340 output rvfi_trap_t rvfi_trap,
341 output logic [ 0:0] rvfi_halt,
342 output rvfi_intr_t rvfi_intr,
343 output logic [ 1:0] rvfi_mode,
344 output logic [ 1:0] rvfi_ixl,
345 output logic [ 1:0] rvfi_nmip,
346
347 output logic [2:0] rvfi_dbg,
348 output logic [0:0] rvfi_dbg_mode,
349
350 output rvfi_wu_t rvfi_wu,
351 output logic [0:0] rvfi_sleep,
352
353 output logic [ 4:0] rvfi_rd_addr [1:0],
354 output logic [31:0] rvfi_rd_wdata [1:0],
355 output logic      rvfi_frđ_wvalid [1:0],
356 output logic [ 4:0] rvfi_frđ_addr [1:0],
357 output logic [31:0] rvfi_frđ_wdata [1:0],
358 output logic      rvfi_2_rd,
359 output logic [ 4:0] rvfi_rs1_addr,
360 output logic [ 4:0] rvfi_rs2_addr,
361 output logic [ 4:0] rvfi_rs3_addr,
362 output logic [31:0] rvfi_rs1_rdata,
363 output logic [31:0] rvfi_rs2_rdata,
364 output logic [31:0] rvfi_rs3_rdata,

```

```

365 output logic [ 4:0] rvfi_frs1_addr,
366 output logic [ 4:0] rvfi_frs2_addr,
367 output logic [ 4:0] rvfi_frs3_addr,
368 output logic      rvfi_frs1_rvalid,
369 output logic      rvfi_frs2_rvalid,
370 output logic      rvfi_frs3_rvalid,
371 output logic [31:0] rvfi_frs1_rdata,
372 output logic [31:0] rvfi_frs2_rdata,
373 output logic [31:0] rvfi_frs3_rdata,
374
375 output logic [31:0] rvfi_pc_rdata,
376 output logic [31:0] rvfi_pc_wdata,
377
378 output logic [31:0] rvfi_mem_addr,
379 output logic [31:0] rvfi_mem_rmask,
380 output logic [31:0] rvfi_mem_wmask,
381 output logic [31:0] rvfi_mem_rdata,
382 output logic [31:0] rvfi_mem_wdata,
383
384 // CSRs
385 output logic [31:0] rvfi_csr_fflags_rmask,
386 output logic [31:0] rvfi_csr_fflags_wmask,
387 output logic [31:0] rvfi_csr_fflags_rdata,
388 output logic [31:0] rvfi_csr_fflags_wdata,
389 output logic [31:0] rvfi_csr_frm_rmask,
390 output logic [31:0] rvfi_csr_frm_wmask,
391 output logic [31:0] rvfi_csr_frm_rdata,
392 output logic [31:0] rvfi_csr_frm_wdata,
393 output logic [31:0] rvfi_csr_fcsr_rmask,
394 output logic [31:0] rvfi_csr_fcsr_wmask,
395 output logic [31:0] rvfi_csr_fcsr_rdata,
396 output logic [31:0] rvfi_csr_fcsr_wdata,
397 output logic [31:0] rvfi_csr_jvt_rmask,
398 output logic [31:0] rvfi_csr_jvt_wmask,
399 output logic [31:0] rvfi_csr_jvt_rdata,
400 output logic [31:0] rvfi_csr_jvt_wdata,
401 output logic [31:0] rvfi_csr_mstatus_rmask,
402 output logic [31:0] rvfi_csr_mstatus_wmask,
403 output logic [31:0] rvfi_csr_mstatus_rdata,
404 output logic [31:0] rvfi_csr_mstatus_wdata,
405 output logic [31:0] rvfi_csr_mstatush_rmask,
406 output logic [31:0] rvfi_csr_mstatush_wmask,
407 output logic [31:0] rvfi_csr_mstatush_rdata,
408 output logic [31:0] rvfi_csr_mstatush_wdata,
409 output logic [31:0] rvfi_csr_misa_rmask,
410 output logic [31:0] rvfi_csr_misa_wmask,
411 output logic [31:0] rvfi_csr_misa_rdata,
412 output logic [31:0] rvfi_csr_misa_wdata,
413 output logic [31:0] rvfi_csr_mie_rmask,
414 output logic [31:0] rvfi_csr_mie_wmask,
415 output logic [31:0] rvfi_csr_mie_rdata,
416 output logic [31:0] rvfi_csr_mie_wdata,
417 output logic [31:0] rvfi_csr_mtvec_rmask,
418 output logic [31:0] rvfi_csr_mtvec_wmask,
419 output logic [31:0] rvfi_csr_mtvec_rdata,
420 output logic [31:0] rvfi_csr_mtvec_wdata,
421 output logic [31:0] rvfi_csr_mtvrt_rmask,
422 output logic [31:0] rvfi_csr_mtvrt_wmask,
423 output logic [31:0] rvfi_csr_mtvrt_rdata,
424 output logic [31:0] rvfi_csr_mtvrt_wdata,
425 output logic [31:0] rvfi_csr_mcountinhibit_rmask,
426 output logic [31:0] rvfi_csr_mcountinhibit_wmask,
427 output logic [31:0] rvfi_csr_mcountinhibit_rdata,
428 output logic [31:0] rvfi_csr_mcountinhibit_wdata,
429 output logic [31:0][31:0] rvfi_csr_mhpmevent_rmask, //
3-31 implemented
430 output logic [31:0][31:0] rvfi_csr_mhpmevent_wmask,
431 output logic [31:0][31:0] rvfi_csr_mhpmevent_rdata,
432 output logic [31:0][31:0] rvfi_csr_mhpmevent_wdata,
433 output logic [31:0] rvfi_csr_mscratch_rmask,
434 output logic [31:0] rvfi_csr_mscratch_wmask,
435 output logic [31:0] rvfi_csr_mscratch_rdata,
436 output logic [31:0] rvfi_csr_mscratch_wdata,
437 output logic [31:0] rvfi_csr_mepc_rmask,
438 output logic [31:0] rvfi_csr_mepc_wmask,
439 output logic [31:0] rvfi_csr_mepc_rdata,
440 output logic [31:0] rvfi_csr_mepc_wdata,
441 output logic [31:0] rvfi_csr_mcause_rmask,
442 output logic [31:0] rvfi_csr_mcause_wmask,
443 output logic [31:0] rvfi_csr_mcause_rdata,
444 output logic [31:0] rvfi_csr_mcause_wdata,
445 output logic [31:0] rvfi_csr_mtval_rmask,
446 output logic [31:0] rvfi_csr_mtval_wmask,
447 output logic [31:0] rvfi_csr_mtval_rdata,
448 output logic [31:0] rvfi_csr_mtval_wdata,
449 output logic [31:0] rvfi_csr_mip_rmask,
450 output logic [31:0] rvfi_csr_mip_wmask,
451 output logic [31:0] rvfi_csr_mip_rdata,
452 output logic [31:0] rvfi_csr_mip_wdata,
453 output logic [31:0] rvfi_csr_mnxti_rmask,
454 output logic [31:0] rvfi_csr_mnxti_wmask,
455 output logic [31:0] rvfi_csr_mnxti_rdata,
456 output logic [31:0] rvfi_csr_mnxti_wdata,
457 output logic [31:0] rvfi_csr_mintstatus_rmask,
458 output logic [31:0] rvfi_csr_mintstatus_wmask,
459 output logic [31:0] rvfi_csr_mintstatus_rdata,
460 output logic [31:0] rvfi_csr_mintstatus_wdata,
461 output logic [31:0] rvfi_csr_mintthresh_rmask,
462 output logic [31:0] rvfi_csr_mintthresh_wmask,
463 output logic [31:0] rvfi_csr_mintthresh_rdata,
464 output logic [31:0] rvfi_csr_mintthresh_wdata,

```

```

465 output logic [31:0] rvfi_csr_mscratchcsw_rmask,
466 output logic [31:0] rvfi_csr_mscratchcsw_wmask,
467 output logic [31:0] rvfi_csr_mscratchcsw_rdata,
468 output logic [31:0] rvfi_csr_mscratchcsw_wdata,
469 output logic [31:0] rvfi_csr_mscratchcswl_rmask,
470 output logic [31:0] rvfi_csr_mscratchcswl_wmask,
471 output logic [31:0] rvfi_csr_mscratchcswl_rdata,
472 output logic [31:0] rvfi_csr_mscratchcswl_wdata,
473 output logic [31:0] rvfi_csr_mclibase_rmask,
474 output logic [31:0] rvfi_csr_mclibase_wmask,
475 output logic [31:0] rvfi_csr_mclibase_rdata,
476 output logic [31:0] rvfi_csr_mclibase_wdata,
477 output logic [31:0] rvfi_csr_tselect_rmask,
478 output logic [31:0] rvfi_csr_tselect_wmask,
479 output logic [31:0] rvfi_csr_tselect_rdata,
480 output logic [31:0] rvfi_csr_tselect_wdata,
481 output logic [ 3:0][31:0] rvfi_csr_tdata_rmask, // 1-3
implemented
482 output logic [ 3:0][31:0] rvfi_csr_tdata_wmask,
483 output logic [ 3:0][31:0] rvfi_csr_tdata_rdata,
484 output logic [ 3:0][31:0] rvfi_csr_tdata_wdata,
485 output logic [31:0] rvfi_csr_tinfo_rmask,
486 output logic [31:0] rvfi_csr_tinfo_wmask,
487 output logic [31:0] rvfi_csr_tinfo_rdata,
488 output logic [31:0] rvfi_csr_tinfo_wdata,
489 output logic [31:0] rvfi_csr_mcontext_rmask,
490 output logic [31:0] rvfi_csr_mcontext_wmask,
491 output logic [31:0] rvfi_csr_mcontext_rdata,
492 output logic [31:0] rvfi_csr_mcontext_wdata,
493 output logic [31:0] rvfi_csr_scontext_rmask,
494 output logic [31:0] rvfi_csr_scontext_wmask,
495 output logic [31:0] rvfi_csr_scontext_rdata,
496 output logic [31:0] rvfi_csr_scontext_wdata,
497 output logic [31:0] rvfi_csr_dcsr_rmask,
498 output logic [31:0] rvfi_csr_dcsr_wmask,
499 output logic [31:0] rvfi_csr_dcsr_rdata,
500 output logic [31:0] rvfi_csr_dcsr_wdata,
501 output logic [31:0] rvfi_csr_dpc_rmask,
502 output logic [31:0] rvfi_csr_dpc_wmask,
503 output logic [31:0] rvfi_csr_dpc_rdata,
504 output logic [31:0] rvfi_csr_dpc_wdata,

505 output logic [ 1:0][31:0] rvfi_csr_dscratch_rmask, // 0-1
implemented
506 output logic [ 1:0][31:0] rvfi_csr_dscratch_wmask,
507 output logic [ 1:0][31:0] rvfi_csr_dscratch_rdata,
508 output logic [ 1:0][31:0] rvfi_csr_dscratch_wdata,
509 output logic [31:0] rvfi_csr_mcycle_rmask,
510 output logic [31:0] rvfi_csr_mcycle_wmask,
511 output logic [31:0] rvfi_csr_mcycle_rdata,
512 output logic [31:0] rvfi_csr_mcycle_wdata,
513 output logic [31:0] rvfi_csr_minstret_rmask,
514 output logic [31:0] rvfi_csr_minstret_wmask,
515 output logic [31:0] rvfi_csr_minstret_rdata,
516 output logic [31:0] rvfi_csr_minstret_wdata,
517 output logic [31:0][31:0] rvfi_csr_mhpmcounter_rmask, //
3-31 implemented
518 output logic [31:0][31:0] rvfi_csr_mhpmcounter_wmask,
519 output logic [31:0][31:0] rvfi_csr_mhpmcounter_rdata,
520 output logic [31:0][31:0] rvfi_csr_mhpmcounter_wdata,
521 output logic [31:0] rvfi_csr_mcycleh_rmask,
522 output logic [31:0] rvfi_csr_mcycleh_wmask,
523 output logic [31:0] rvfi_csr_mcycleh_rdata,
524 output logic [31:0] rvfi_csr_mcycleh_wdata,
525 output logic [31:0] rvfi_csr_minstreth_rmask,
526 output logic [31:0] rvfi_csr_minstreth_wmask,
527 output logic [31:0] rvfi_csr_minstreth_rdata,
528 output logic [31:0] rvfi_csr_minstreth_wdata,

529 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_rmask, //
3-31 implemented
530 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_wmask,
531 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_rdata,
532 output logic [31:0][31:0] rvfi_csr_mhpmcounterh_wdata,
533 output logic [31:0] rvfi_csr_cycle_rmask,
534 output logic [31:0] rvfi_csr_cycle_wmask,
535 output logic [31:0] rvfi_csr_cycle_rdata,
536 output logic [31:0] rvfi_csr_cycle_wdata,
537 output logic [31:0] rvfi_csr_instret_rmask,
538 output logic [31:0] rvfi_csr_instret_wmask,
539 output logic [31:0] rvfi_csr_instret_rdata,
540 output logic [31:0] rvfi_csr_instret_wdata,
541 output logic [31:0][31:0] rvfi_csr_hpmcounter_rmask, //
3-31 implemented
542 output logic [31:0][31:0] rvfi_csr_hpmcounter_wmask,
543 output logic [31:0][31:0] rvfi_csr_hpmcounter_rdata,
544 output logic [31:0][31:0] rvfi_csr_hpmcounter_wdata,
545 output logic [31:0] rvfi_csr_cycleh_rmask,
546 output logic [31:0] rvfi_csr_cycleh_wmask,
547 output logic [31:0] rvfi_csr_cycleh_rdata,
548 output logic [31:0] rvfi_csr_cycleh_wdata,
549 output logic [31:0] rvfi_csr_instreth_rmask,
550 output logic [31:0] rvfi_csr_instreth_wmask,
551 output logic [31:0] rvfi_csr_instreth_rdata,
552 output logic [31:0] rvfi_csr_instreth_wdata,
553 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rmask, //
3-31 implemented
554 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wmask,
555 output logic [31:0][31:0] rvfi_csr_hpmcounterh_rdata,
556 output logic [31:0][31:0] rvfi_csr_hpmcounterh_wdata,
557 output logic [31:0] rvfi_csr_mvendordid_rmask,

558 output logic [31:0] rvfi_csr_mvendordid_wmask,
559 output logic [31:0] rvfi_csr_mvendordid_rdata,
560 output logic [31:0] rvfi_csr_mvendordid_wdata,
561 output logic [31:0] rvfi_csr_marchid_rmask,
562 output logic [31:0] rvfi_csr_marchid_wmask,
563 output logic [31:0] rvfi_csr_marchid_rdata,
564 output logic [31:0] rvfi_csr_marchid_wdata,
565 output logic [31:0] rvfi_csr_mimpid_rmask,
566 output logic [31:0] rvfi_csr_mimpid_wmask,
567 output logic [31:0] rvfi_csr_mimpid_rdata,
568 output logic [31:0] rvfi_csr_mimpid_wdata,
569 output logic [31:0] rvfi_csr_mhartid_rmask,
570 output logic [31:0] rvfi_csr_mhartid_wmask,
571 output logic [31:0] rvfi_csr_mhartid_rdata,
572 output logic [31:0] rvfi_csr_mhartid_wdata,
573

574 output logic [31:0] rvfi_csr_mcounteren_rmask,
575 output logic [31:0] rvfi_csr_mcounteren_wmask,
576 output logic [31:0] rvfi_csr_mcounteren_rdata,
577 output logic [31:0] rvfi_csr_mcounteren_wdata,
578

579 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_rmask,
580 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_wmask,
581 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_rdata,
582 output logic [ 3:0][31:0] rvfi_csr_pmpcfg_wdata,
583 output logic [15:0][31:0] rvfi_csr_pmpaddr_rmask,
584 output logic [15:0][31:0] rvfi_csr_pmpaddr_wmask,
585 output logic [15:0][31:0] rvfi_csr_pmpaddr_rdata,
586 output logic [15:0][31:0] rvfi_csr_pmpaddr_wdata,
587 output logic [31:0] rvfi_csr_mseccfg_rmask,
588 output logic [31:0] rvfi_csr_mseccfg_wmask,
589 output logic [31:0] rvfi_csr_mseccfg_rdata,
590 output logic [31:0] rvfi_csr_mseccfg_wdata,
591 output logic [31:0] rvfi_csr_mseccfg_rmask,
592 output logic [31:0] rvfi_csr_mseccfg_wmask,
593 output logic [31:0] rvfi_csr_mseccfg_rdata,
594 output logic [31:0] rvfi_csr_mseccfg_wdata,
595

596 output logic [31:0] rvfi_csr_mconfigptr_rmask,
597 output logic [31:0] rvfi_csr_mconfigptr_wmask,
598 output logic [31:0] rvfi_csr_mconfigptr_rdata,
599 output logic [31:0] rvfi_csr_mconfigptr_wdata,
600

601 output logic [31:0] rvfi_csr_lptest0_rmask,
602 output logic [31:0] rvfi_csr_lptest0_wmask,
603 output logic [31:0] rvfi_csr_lptest0_rdata,
604 output logic [31:0] rvfi_csr_lptest0_wdata,
605 output logic [31:0] rvfi_csr_lpend0_rmask,
606 output logic [31:0] rvfi_csr_lpend0_wmask,
607 output logic [31:0] rvfi_csr_lpend0_rdata,
608 output logic [31:0] rvfi_csr_lpend0_wdata,
609 output logic [31:0] rvfi_csr_lpcount0_rmask,
610 output logic [31:0] rvfi_csr_lpcount0_wmask,
611 output logic [31:0] rvfi_csr_lpcount0_rdata,
612 output logic [31:0] rvfi_csr_lpcount0_wdata,
613 output logic [31:0] rvfi_csr_lptest1_rmask,
614 output logic [31:0] rvfi_csr_lptest1_wmask,
615 output logic [31:0] rvfi_csr_lptest1_rdata,
616 output logic [31:0] rvfi_csr_lptest1_wdata,
617 output logic [31:0] rvfi_csr_lpend1_rmask,
618 output logic [31:0] rvfi_csr_lpend1_wmask,
619 output logic [31:0] rvfi_csr_lpend1_rdata,
620 output logic [31:0] rvfi_csr_lpend1_wdata,
621 output logic [31:0] rvfi_csr_lpcount1_rmask,
622 output logic [31:0] rvfi_csr_lpcount1_wmask,
623 output logic [31:0] rvfi_csr_lpcount1_rdata,
624 output logic [31:0] rvfi_csr_lpcount1_wdata
625 );
626
627
628 bit clk_i_d;
629 assign #0.01 clk_i_d = clk_i;
630
631 integer cycles;
632 // cycle counter
633 always_ff @(posedge clk_i_d, negedge rst_ni) begin
634   if (rst_ni == 1'b0) cycles <= 0;
635   else cycles <= cycles + 1;
636 end
637
638 logic pc_mux_debug;
639 logic pc_mux_dret;
640 logic pc_mux_exception;
641 logic pc_mux_debug_exception;
642 logic pc_mux_interrupt;
643 logic pc_mux_nmi;
644
645 localparam logic [31:0] MSTATUS_WRITE_MASK = 32'h0000_6088;
646 localparam logic [31:0] MCOUNTERINHIBIT_WRITE_MASK = {(29-
NUM_MHPMCOUNTERS){1'b0}}, {(NUM_MHPMCOUNTERS){1'b1}}, 3'b101;
647 localparam NUM_HPM_EVENTS = 16;
648 localparam logic [31:0] MHPMEVENT_WRITE_MASK = {(31-
NUM_HPM_EVENTS){1'b0}}, {(NUM_HPM_EVENTS){1'b1}};
649
650 `include "pipe_freeze_trace.sv"
651
652 `include "insn_trace.sv"
653
654 insn_trace_t trace_if, trace_id, trace_ex, trace_ex_next,
trace_wb;
655 insn_trace_t tmp_trace_wb;

```

```

656 insn_trace_t rvfi_trace_q[$], wb_bypass_trace_q[$];
657
658 bit          clear_trace_ex; // = 1'b1;
659 bit          clear_trace_wb;
660
661 logic        csr_is_irq;
662 logic        is_dbg_taken;
663 logic [2:0]   saved_debug_cause;
664 integer       next_send;
665
666 event        e_empty_queue;
667 function void empty_fifo();
668     integer i, trace_q_size;
669     trace_q_size = wb_bypass_trace_q.size();
670     if (trace_q_size > 50) begin
671         $fatal("Reorder queue too long\n");
672     end
673     if (trace_q_size != 0) begin
674         for (i = 0; i < trace_q_size; i++) begin
675             insn_trace_t new_rvfi_trace;
676             new_rvfi_trace = new();
677             new_r-
678             vfi_trace.copy_full(wb_bypass_trace_q.pop_front());
679             if (next_send == new_rvfi_trace.m_order) begin
680                 new_rvfi_trace.m_csr.mstatus_fs_rdata =
681                 r_pipe_freeze_trace.csr.mstatus_fs_n;
682                 rvfi_trace_q.push_back(new_rvfi_trace);
683                 next_send = next_send + 1;
684                 ->e_empty_queue;
685             end else begin
686                 wb_bypass_trace_q.push_back(new_rvfi_trace);
687             end
688         end
689     end
690     trace_q_size = wb_bypass_trace_q.size(); //Re-calculate
691 here for accurate status
692 endfunction
693 /*
694 * Function used to allocate a new insn and send it to the
695 rvfi driver
696 */
697 event e_add_to_bypass;
698 function void send_rvfi(insn_trace_t m_wb_insn);
699     insn_trace_t new_rvfi_trace;
700     new_rvfi_trace = new();
701     new_rvfi_trace.copy_full(m_wb_insn);
702     if (next_send == new_rvfi_trace.m_order) begin
703         rvfi_trace_q.push_back(new_rvfi_trace);
704         next_send = next_send + 1;
705     end else begin
706         wb_bypass_trace_q.push_back(new_rvfi_trace);
707         ->e_add_to_bypass;
708     end
709     empty_fifo();
710 endfunction
711 /*
712 * Assing rvfi signals once the instruction is completed
713 */
714 `define SET_RVFI_CSR_FROM_INSN(CSR_NAME) \
715     rvfi_csr_`CSR_NAME`_rdata = new_r-
716 vfi_trace.m_csr_`CSR_NAME`_rdata; \
717     rvfi_csr_`CSR_NAME`_rmask = new_r-
718 vfi_trace.m_csr_`CSR_NAME`_rmask; \
719     rvfi_csr_`CSR_NAME`_wdata = new_r-
720 vfi_trace.m_csr_`CSR_NAME`_wdata; \
721     rvfi_csr_`CSR_NAME`_wmask = new_r-
722 vfi_trace.m_csr_`CSR_NAME`_wmask;
723
724 logic [31:0] s_fflags_mirror;
725 logic [31:0] s_frm_mirror;
726 logic [31:0] s_fcsr_mirror;
727
728 function void set_rvfi();
729     insn_trace_t new_rvfi_trace;
730     new_rvfi_trace = rvfi_trace_q.pop_front();
731
732     if (new_rvfi_trace.m_is_apu) begin
733         if (new_rvfi_trace.m_csr.fflags_we) begin
734             s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
735         end else begin
736             s_fflags_mirror = new_rvfi_trace.m_csr.fflags_rdata;
737         end
738         if (new_rvfi_trace.m_csr.frm_we) begin
739             s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
740         end else begin
741             s_frm_mirror = new_rvfi_trace.m_csr.frm_rdata;
742         end
743         if (new_rvfi_trace.m_csr.fcsr_we) begin
744             s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
745         end else begin
746             s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_rdata;
747         end
748     end else begin
749         new_rvfi_trace.m_csr.fflags_rdata = s_fflags_mirror;
750         new_rvfi_trace.m_csr.frm_rdata = s_frm_mirror;
751         new_rvfi_trace.m_csr.fcsr_rdata = s_fcsr_mirror;
752
753         if (new_rvfi_trace.m_fflags_we_non_apu) begin
754             s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;

```

```

749     s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
750     new_rvfi_trace.m_csr.fflags_wmask = 32'hFFFF_FFFF;
751     new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
752 end
753 if (new_rvfi_trace.m_frm_we_non_apu) begin
754     s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
755     s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
756     new_rvfi_trace.m_csr.frm_wmask = 32'hFFFF_FFFF;
757     new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
758 end
759 if (new_rvfi_trace.m_fcsr_we_non_apu) begin
760     s_fcsr_mirror = new_rvfi_trace.m_csr.fcsr_wdata;
761     s_fflags_mirror = new_rvfi_trace.m_csr.fflags_wdata;
762     s_frm_mirror = new_rvfi_trace.m_csr.frm_wdata;
763     new_rvfi_trace.m_csr.fcsr_wmask = 32'hFFFF_FFFF;
764     new_rvfi_trace.m_csr.fflags_wmask = 32'hFFFF_FFFF;
765     new_rvfi_trace.m_csr.frm_wmask = 32'hFFFF_FFFF;
766 end
767 end
768
769 rvfi_order      = new_rvfi_trace.m_order;
770 rvfi_start_cycle = new_rvfi_trace.m_start_cycle;
771 rvfi_start_time  = new_rvfi_trace.m_start_time;
772 rvfi_stop_cycle  = new_rvfi_trace.m_stop_cycle;
773 rvfi_stop_time   = new_rvfi_trace.m_stop_time;
774 rvfi_pc_rdata    = new_rvfi_trace.m_pc_rdata;
775 rvfi_insn        = new_rvfi_trace.m_insn;
776
777 rvfi_rs1_addr    = '0;
778 rvfi_rs1_rdata   = '0;
779 rvfi_rs2_addr    = '0;
780 rvfi_rs2_rdata   = '0;
781 rvfi_rs3_addr    = '0;
782 rvfi_rs3_rdata   = '0;
783 rvfi_frs1_rvalid = '0;
784 rvfi_frs1_addr   = '0;
785 rvfi_frs1_rdata  = '0;
786 rvfi_frs2_rvalid = '0;
787 rvfi_frs2_addr   = '0;
788 rvfi_frs2_rdata  = '0;
789 rvfi_frs3_rvalid = '0;
790 rvfi_frs3_addr   = '0;
791 rvfi_frs3_rdata  = '0;
792
793 if (new_rvfi_trace.m_rs1_addr[5]) begin
794     rvfi_frs1_rvalid = 1'b1;
795     rvfi_frs1_addr   = new_rvfi_trace.m_rs1_addr[4:0];
796     rvfi_frs1_rdata  = new_rvfi_trace.m_rs1_rdata;
797 end else begin
798     rvfi_rs1_addr    = new_rvfi_trace.m_rs1_addr[4:0];
799     rvfi_rs1_rdata   = new_rvfi_trace.m_rs1_rdata;
800 end
801
802 if (new_rvfi_trace.m_rs2_addr[5]) begin
803     rvfi_frs2_rvalid = 1'b1;
804     rvfi_frs2_addr   = new_rvfi_trace.m_rs2_addr[4:0];
805     rvfi_frs2_rdata  = new_rvfi_trace.m_rs2_rdata;
806 end else begin
807     rvfi_rs2_addr    = new_rvfi_trace.m_rs2_addr[4:0];
808     rvfi_rs2_rdata   = new_rvfi_trace.m_rs2_rdata;
809 end
810
811 if (new_rvfi_trace.m_rs3_addr[5]) begin
812     rvfi_frs3_rvalid = 1'b1;
813     rvfi_frs3_addr   = new_rvfi_trace.m_rs3_addr[4:0];
814     rvfi_frs3_rdata  = new_rvfi_trace.m_rs3_rdata;
815 end else begin
816     rvfi_rs3_addr    = new_rvfi_trace.m_rs3_addr[4:0];
817     rvfi_rs3_rdata   = new_rvfi_trace.m_rs3_rdata;
818 end
819
820 rvfi_frd_wvalid[0] = '0;
821 rvfi_frd_addr[0]   = '0;
822 rvfi_frd_wdata[0]  = '0;
823
824 rvfi_frd_wvalid[1] = '0;
825 rvfi_frd_addr[1]   = '0;
826 rvfi_frd_wdata[1]  = '0;
827
828 rvfi_2_rd = new_rvfi_trace.m_2_rd_insn;
829 if (new_rvfi_trace.m_rd_addr[0][5] == 1'b0) begin
830     rvfi_rd_addr[0] = new_rvfi_trace.m_rd_addr[0][4:0];
831     rvfi_rd_wdata[0] = new_rvfi_trace.m_rd_wdata[0];
832 end else begin
833     rvfi_rd_addr[0] = '0;
834     rvfi_rd_wdata[0] = '0;
835
836     rvfi_frd_wvalid[0] = 1'b1;
837     rvfi_frd_addr[0]   = new_rvfi_trace.m_rd_addr[0][4:0];
838     rvfi_frd_wdata[0]  = new_rvfi_trace.m_rd_wdata[0];
839 end
840
841 if (new_rvfi_trace.m_2_rd_insn) begin
842     if (new_rvfi_trace.m_rd_addr[1][5] == 1'b0) begin
843         rvfi_rd_addr[1] = new_rvfi_trace.m_rd_addr[1][4:0];
844         rvfi_rd_wdata[1] = new_rvfi_trace.m_rd_wdata[1];
845     end else begin
846         rvfi_frd_wvalid[1] = 1'b1;
847         rvfi_frd_addr[1]   = new_rvfi_trace.m_rd_addr[1][4:0];
848         rvfi_frd_wdata[1]  = new_rvfi_trace.m_rd_wdata[1];

```



```

850
851     rvfi_rd_addr[1] = '0';
852     rvfi_rd_wdata[1] = '0';
853 end
854 end else begin
855     rvfi_rd_addr[1] = '0';
856     rvfi_rd_wdata[1] = '0';
857 end
858
859 rvfi_mem_addr = new_rvfi_trace.m_mem.addr;
860 rvfi_mem_rmask = new_rvfi_trace.m_mem.rmask;
861 rvfi_mem_wmask = new_rvfi_trace.m_mem.wmask;
862 rvfi_mem_rdata = new_rvfi_trace.m_mem.rdata;
863 rvfi_mem_wdata = new_rvfi_trace.m_mem.wdata;
864
865 rvfi_dbg = new_rvfi_trace.m_dbg.cause;
866 rvfi_dbg_mode = new_rvfi_trace.m_dbg.taken;
867
868 rvfi_trap.trap = 0;
869 if (new_rvfi_trace.m_is_illegal) begin
870     rvfi_trap.trap = '1'b1;
871 end
872
873 if (new_rvfi_trace.m_trap) begin
874     rvfi_trap.trap = '1'b1;
875 end
876
877 //CSR
878 rvfi_csr_mstatus_rmask = new_r-
vfi_trace.m_csr.mstatus_rmask | new_r-
vfi_trace.m_csr.mstatus_fs_rmask;
879 rvfi_csr_mstatus_wmask = new_r-
vfi_trace.m_csr.mstatus_wmask & MSTATUS_WRITE_MASK;
880 rvfi_csr_mstatus_wmask[31] = new_r-
vfi_trace.m_csr.mstatus_fs_wmask[31];
881 rvfi_csr_mstatus_wmask[14:13] = new_r-
vfi_trace.m_csr.mstatus_fs_wmask[14:13];
882
883 if (FPU == 1 && ZFINX == 0) begin
884     rvfi_csr_mstatus_rdata[31] = (new_r-
vfi_trace.m_csr.mstatus_fs_rdata == FS_DIRTY) ? '1'b1 : '1'b0;
885 end else begin
886     rvfi_csr_mstatus_rdata[31] = '0';
887 end
888 rvfi_csr_mstatus_rdata[30:18] = '0';
889 rvfi_csr_mstatus_rdata[17] = new_r-
vfi_trace.m_csr.mstatus_rdata.mprv;
890 rvfi_csr_mstatus_rdata[16:15] = '0';
891 if (FPU == 1 && ZFINX == 0) begin
892     rvfi_csr_mstatus_rdata[14:13] = new_r-
vfi_trace.m_csr.mstatus_fs_rdata;
893 end else begin
894     rvfi_csr_mstatus_rdata[14:13] = '0';
895 end
896 rvfi_csr_mstatus_rdata[12:11] = new_r-
vfi_trace.m_csr.mstatus_rdata.mpp;
897 rvfi_csr_mstatus_rdata[10:8] = '0';
898 rvfi_csr_mstatus_rdata[7] = new_r-
vfi_trace.m_csr.mstatus_rdata.mpie;
899 rvfi_csr_mstatus_rdata[6:5] = '0';
900 rvfi_csr_mstatus_rdata[4] = new_r-
vfi_trace.m_csr.mstatus_rdata.upie;
901 rvfi_csr_mstatus_rdata[3] = new_r-
vfi_trace.m_csr.mstatus_rdata.mie;
902 rvfi_csr_mstatus_rdata[2:1] = '0';
903 rvfi_csr_mstatus_rdata[0] = new_r-
vfi_trace.m_csr.mstatus_rdata.uie;
904
905 if (FPU == 1 && ZFINX == 0) begin
906     rvfi_csr_mstatus_wdata[31] = (new_r-
vfi_trace.m_csr.mstatus_fs_wdata == FS_DIRTY) ? '1'b1 : '1'b0;
907 end else begin
908     rvfi_csr_mstatus_wdata[31] = '0';
909 end
910 rvfi_csr_mstatus_wdata[30:18] = '0';
911 // MPRV is not implemented in the target configuration,
writes to it are ignored
912 rvfi_csr_mstatus_wdata[17] = '1'b0; //new_r-
vfi_trace.m_csr.mstatus_wdata.mprv;
913 rvfi_csr_mstatus_wdata[16:15] = '0';
914 if (FPU == 1 && ZFINX == 0) begin
915     rvfi_csr_mstatus_wdata[14:13] = new_r-
vfi_trace.m_csr.mstatus_fs_wdata;
916 end else begin
917     rvfi_csr_mstatus_wdata[14:13] = '0';
918 end
919 rvfi_csr_mstatus_wdata[12:11] = new_r-
vfi_trace.m_csr.mstatus_wdata.mpp;
920 rvfi_csr_mstatus_wdata[10:8] = '0';
921 rvfi_csr_mstatus_wdata[7] = new_r-
vfi_trace.m_csr.mstatus_wdata.mpie;
922 rvfi_csr_mstatus_wdata[6:5] = '0';
923 // UPIE is not implemented in the target configuration,
writes to it are ignored
924 rvfi_csr_mstatus_wdata[4] = '1'b0; //new_r-
vfi_trace.m_csr.mstatus_wdata.upie;
925 rvfi_csr_mstatus_wdata[3] = new_r-
vfi_trace.m_csr.mstatus_wdata.mie;
926 rvfi_csr_mstatus_wdata[2:1] = '0';
927 // UIE is not implemented in the target configuration,
writes to it are ignored

```

```

928     rvfi_csr_mstatus_wdata[0] = '1'b0; //new_r-
vfi_trace.m_csr.mstatus_wdata.uie;
929
930 `SET_RVFI_CSR_FROM_INSN(misa)
931 `SET_RVFI_CSR_FROM_INSN(mie)
932 `SET_RVFI_CSR_FROM_INSN(mvec)
933
934 rvfi_csr_mcountinhibit_rdata = new_r-
vfi_trace.m_csr.mcountinhibit_rdata;
935 rvfi_csr_mcountinhibit_rmask = new_r-
vfi_trace.m_csr.mcountinhibit_rmask;
936 rvfi_csr_mcountinhibit_wdata = new_r-
vfi_trace.m_csr.mcountinhibit_wdata;
937 rvfi_csr_mcountinhibit_wmask = new_r-
vfi_trace.m_csr.mcountinhibit_wmask & MCOUNTINHIBIT_WRITE_MASK;
938
939 `SET_RVFI_CSR_FROM_INSN(mscratch)
940 `SET_RVFI_CSR_FROM_INSN(mepc)
941 `SET_RVFI_CSR_FROM_INSN(mcause)
942 `SET_RVFI_CSR_FROM_INSN(mcycle)
943 `SET_RVFI_CSR_FROM_INSN(minstret)
944 `SET_RVFI_CSR_FROM_INSN(minstreth)
945
946 // `SET_RVFI_CSR_FROM_INSN(cycle)
947 // `SET_RVFI_CSR_FROM_INSN(instret)
948 rvfi_csr_instret_rdata = new_r-
vfi_trace.m_csr.minstret_rdata;
949 rvfi_csr_instret_rmask = new_r-
vfi_trace.m_csr.minstret_rmask;
950 rvfi_csr_instret_wdata = new_r-
vfi_trace.m_csr.minstret_wdata;
951 rvfi_csr_instret_wmask = new_r-
vfi_trace.m_csr.minstret_wmask;
952
953 for(int idx=3; idx<32; idx++) begin
954     rvfi_csr_mhpmcounter_rmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_rmask[idx][31:0];
955     rvfi_csr_mhpmcounter_wmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_wmask[idx][31:0];
956     rvfi_csr_mhpmcounter_rdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_rdata[idx][31:0];
957     rvfi_csr_mhpmcounter_wdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounter_wdata[idx][31:0];
958
959     rvfi_csr_mhpmcounterh_rmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_rmask[idx][63:32];
960     rvfi_csr_mhpmcounterh_wmask[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_wmask[idx][63:32];
961     rvfi_csr_mhpmcounterh_rdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_rdata[idx][63:32];
962     rvfi_csr_mhpmcounterh_wdata[idx] = new_r-
vfi_trace.m_csr.mhpmcounterh_wdata[idx][63:32];
963
964     rvfi_csr_mhpmevent_rmask[idx] = new_r-
vfi_trace.m_csr.mhpmevent_rmask[idx];
965     rvfi_csr_mhpmevent_wmask[idx] = new_r-
vfi_trace.m_csr.mhpmevent_wmask[idx] & MHPMEVENT_WRITE_MASK;
966     rvfi_csr_mhpmevent_rdata[idx] = new_r-
vfi_trace.m_csr.mhpmevent_rdata[idx];
967     rvfi_csr_mhpmevent_wdata[idx] = new_r-
vfi_trace.m_csr.mhpmevent_wdata[idx];
968 end
969 // `SET_RVFI_CSR_FROM_INSN(instreth)
970 rvfi_csr_instreth_rdata = new_r-
vfi_trace.m_csr.minstreth_rdata;
971 rvfi_csr_instreth_rmask = new_r-
vfi_trace.m_csr.minstreth_rmask;
972 rvfi_csr_instreth_wdata = new_r-
vfi_trace.m_csr.minstreth_wdata;
973 rvfi_csr_instreth_wmask = new_r-
vfi_trace.m_csr.minstreth_wmask;
974
975 `SET_RVFI_CSR_FROM_INSN(mip)
976
977 rvfi_csr_tdata_rdata[0] = 'Z;
978 rvfi_csr_tdata_rmask[0] = '0; // Does not exist
979 rvfi_csr_tdata_wdata[0] = 'Z; // Does not exist
980 rvfi_csr_tdata_wmask[0] = '0;
981
982 rvfi_csr_tdata_rdata[1] = new_r-
vfi_trace.m_csr.tdata1_rdata;
983 rvfi_csr_tdata_rmask[1] = new_r-
vfi_trace.m_csr.tdata1_rmask; // '1
984 rvfi_csr_tdata_wdata[1] = new_r-
vfi_trace.m_csr.tdata1_wdata;
985 rvfi_csr_tdata_wmask[1] = new_r-
vfi_trace.m_csr.tdata1_wmask;
986
987 rvfi_csr_tdata_rdata[2] = new_r-
vfi_trace.m_csr.tdata2_rdata;
988 rvfi_csr_tdata_rmask[2] = new_r-
vfi_trace.m_csr.tdata2_rmask; // '1
989 rvfi_csr_tdata_wdata[2] = new_r-
vfi_trace.m_csr.tdata2_wdata;
990 rvfi_csr_tdata_wmask[2] = new_r-
vfi_trace.m_csr.tdata2_wmask;
991
992 rvfi_csr_tdata_rmask[3] = '1;
993
994 `SET_RVFI_CSR_FROM_INSN(tinfo)
995 `SET_RVFI_CSR_FROM_INSN(dcsr)

```

```

996     `SET_RVFI_CSR_FROM_INSN(dpc)
997     rvfi_csr_dscratch_rmask[0] = new_r-
vfi_trace.m_csr.dscratch0_rmask;
998     rvfi_csr_dscratch_wmask[0] = new_r-
vfi_trace.m_csr.dscratch0_wmask;
999     rvfi_csr_dscratch_rdata[0] = new_r-
vfi_trace.m_csr.dscratch0_rdata;
1000    rvfi_csr_dscratch_wdata[0] = new_r-
vfi_trace.m_csr.dscratch0_wdata;
1001    rvfi_csr_dscratch_rmask[1] = new_r-
vfi_trace.m_csr.dscratch1_rmask;
1002    rvfi_csr_dscratch_wmask[1] = new_r-
vfi_trace.m_csr.dscratch1_wmask;
1003    rvfi_csr_dscratch_rdata[1] = new_r-
vfi_trace.m_csr.dscratch1_rdata;
1004    rvfi_csr_dscratch_wdata[1] = new_r-
vfi_trace.m_csr.dscratch1_wdata;
1005    // `SET_RVFI_CSR_FROM_INSN(dscratch0)
1006    `SET_RVFI_CSR_FROM_INSN(mvendorid)
1007    `SET_RVFI_CSR_FROM_INSN(marchid)
1008
1009    `SET_RVFI_CSR_FROM_INSN(fflags)
1010    `SET_RVFI_CSR_FROM_INSN(frm)
1011    `SET_RVFI_CSR_FROM_INSN(fcsr)
1012
1013    `SET_RVFI_CSR_FROM_INSN(lpstart0)
1014    `SET_RVFI_CSR_FROM_INSN(lpend0)
1015    `SET_RVFI_CSR_FROM_INSN(lpcount0)
1016    `SET_RVFI_CSR_FROM_INSN(lpstart1)
1017    `SET_RVFI_CSR_FROM_INSN(lpend1)
1018    `SET_RVFI_CSR_FROM_INSN(lpcount1)
1019
1020    endfunction // set_rvfi
1021
1022    function void sample_perf_counter_to_id(int idx);
1023        trace_id.m_csr.mhpmcounter_rdata[idx][31:0] =
r_pipe_freeze_trace.csr.mhpmcounter_q[idx][31:0];
1024        trace_id.m_csr.mhpmcounter_rmask[idx][31:0] = '1;
1025    endfunction
1026
1027    function void perf_counter_to_id(int idx);
1028        if(!trace_id.m_csr.mhpmcounter_we[idx][0]) begin
1029            trace_id.m_csr.mhpmcounter_wdata[idx][31:0] =
r_pipe_freeze_trace.csr.wdata_int;
1030        end
1031        if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx])
begin
1032            trace_id.m_csr.mhpmcounter_we[idx][0] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx];
1033            trace_id.m_csr.mhpmcounter_wdata[idx][31:0] =
r_pipe_freeze_trace.csr.wdata_int;
1034            trace_id.m_csr.mhpmcounter_wmask[idx][31:0] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx] ? '1 : '0;
1035        end
1036        sample_perf_counter_to_id(idx);
1037    endfunction
1038
1039    function void sample_perf_event_to_trace(int idx,
insn_trace_t m_trace);
1040        m_trace.m_csr.mhpmevent_rdata[idx] =
r_pipe_freeze_trace.csr.mhpmevent_q[idx];
1041        m_trace.m_csr.mhpmevent_rmask[idx] = '1;
1042    endfunction
1043
1044    function void perf_event_to_trace(int idx, insn_trace_t
m_trace);
1045        if(!m_trace.m_csr.mhpmevent_we[idx]) begin
1046            m_trace.m_csr.mhpmevent_wdata[idx] =
r_pipe_freeze_trace.csr.wdata_int;
1047        end
1048        if(r_pipe_freeze_trace.csr.mhpmevent_we[idx]) begin
1049            m_trace.m_csr.mhpmevent_we[idx] =
r_pipe_freeze_trace.csr.mhpmevent_we[idx];
1050            m_trace.m_csr.mhpmevent_wdata[idx] =
r_pipe_freeze_trace.csr.wdata_int;
1051            m_trace.m_csr.mhpmevent_wmask[idx] =
r_pipe_freeze_trace.csr.mhpmevent_we[idx] ? '1 : '0;
1052        end
1053        sample_perf_event_to_trace(idx, m_trace);
1054    endfunction
1055
1056    function void sample_minstret_to_trace(insn_trace_t
m_trace);
1057        m_trace.m_csr.minstret_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2][31:0];
1058        m_trace.m_csr.minstret_rmask = '1;
1059    endfunction
1060
1061    function void minstret_to_trace(insn_trace_t m_trace);
1062        if(!m_trace.m_csr.minstret_we) begin
1063            m_trace.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1064        end
1065        if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2]) be-
gin
1066            m_trace.m_csr.minstret_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2];
1067            m_trace.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1068            m_trace.m_csr.minstret_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2] ? '1 : '0;

```

```

1069    end
1070    sample_minstret_to_trace(m_trace);
1071    endfunction
1072
1073    function void sample_perf_counter_h_to_id(int idx);
1074        trace_id.m_csr.mhpmcounter_rdata[idx][63:32] =
r_pipe_freeze_trace.csr.mhpmcounter_q[idx][63:0];
1075        trace_id.m_csr.mhpmcounter_rmask[idx][63:32] = '1;
1076    endfunction
1077
1078    function void perf_counter_h_to_id(int idx);
1079        if(!trace_id.m_csr.mhpmcounter_we[idx][1]) begin
1080            trace_id.m_csr.mhpmcounter_wdata[idx][63:32] =
r_pipe_freeze_trace.csr.wdata_int;
1081        end
1082        if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx])
begin
1083            trace_id.m_csr.mhpmcounter_we[idx][1] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx];
1084            trace_id.m_csr.mhpmcounter_wdata[idx][63:32] =
r_pipe_freeze_trace.csr.wdata_int;
1085            trace_id.m_csr.mhpmcounter_wmask[idx][63:32] =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx] ? '1 : '0;
1086        end
1087        sample_perf_counter_h_to_id(idx);
1088    endfunction
1089
1090    function void sample_minstreth_to_trace(insn_trace_t
m_trace);
1091        m_trace.m_csr.minstreth_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2][63:32];
1092        m_trace.m_csr.minstreth_rmask = '1;
1093    endfunction
1094
1095    function void sample_mcycle_to_trace(insn_trace_t m_trace);
1096        m_trace.m_csr.mcycle_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[0];
1097        m_trace.m_csr.mcycle_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[0][31:0];
1098        m_trace.m_csr.mcycle_rmask = '1;
1099        m_trace.m_csr.mcycle_wdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[31:0];
1100        m_trace.m_csr.mcycle_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[0] ? '1 : '0;
1101    endfunction
1102
1103    function void minstreth_to_trace(insn_trace_t m_trace);
1104        if(!m_trace.m_csr.minstreth_we) begin
1105            m_trace.m_csr.minstreth_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1106        end
1107        if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2]) be-
gin
1108            m_trace.m_csr.minstreth_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2];
1109            m_trace.m_csr.minstreth_wdata =
r_pipe_freeze_trace.csr.wdata_int;
1110            m_trace.m_csr.minstreth_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2] ? '1 : '0;
1111        end
1112        sample_minstreth_to_trace(m_trace);
1113    endfunction
1114
1115    function void sample_perf_counter_to_trace(insn_trace_t
m_trace);
1116        sample_minstret_to_trace(m_trace);
1117        sample_minstreth_to_trace(m_trace);
1118        sample_mcycle_to_trace(m_trace);
1119        for(int idx=3; idx<32; idx++)begin
1120            sample_perf_event_to_trace(idx, m_trace); //TO CHANGE
1121        end
1122    endfunction
1123
1124    function void perf_counter_to_trace(insn_trace_t m_trace);
1125        if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2])
begin
1126            minstret_to_trace(m_trace);
1127        end
1128        if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[2])
begin
1129            minstreth_to_trace(m_trace);
1130        end
1131        for(int idx=3; idx<32; idx++) begin
1132            if(r_pipe_freeze_trace.csr.mhpmcounter_write_lower[idx]) begin
1133                perf_counter_to_id(idx);
1134            end
1135        end
1136        if(r_pipe_freeze_trace.csr.mhpmcounter_write_upper[idx]) begin
1137            perf_counter_h_to_id(idx);
1138        end
1139        if(r_pipe_freeze_trace.csr.mhpmevent_we[idx]) begin
1140            perf_event_to_trace(idx, m_trace);
1141        end
1142    endfunction
1143
1144    function void tinfo_to_trace(insn_trace_t m_trace);
1145        m_trace.m_csr.tinfo_we = '0; // READ ONLY csr_t-
info_we_i;
1146        m_trace.m_csr.tinfo_rdata =

```

```

r_pipe_freeze_trace.csr.tinfo_q;
1147     m_trace.m_csr.tinfo_rmask = '1';
1148     m_trace.m_csr.tinfo_wdata =
r_pipe_freeze_trace.csr.tinfo_n;
1149     m_trace.m_csr.tinfo_wmask = '0';
1150     endfunction
1151
1152     function void mtvec_to_id();
1153         trace_id.m_csr.mtvec_we =
r_pipe_freeze_trace.csr.mtvec_we;
1154         trace_id.m_csr.mtvec_rdata = {
1155             r_pipe_freeze_trace.csr.mtvec_q, 6'h0,
r_pipe_freeze_trace.csr.mtvec_mode_q
1156         };
1157         trace_id.m_csr.mtvec_rmask = '1';
1158         trace_id.m_csr.mtvec_wdata = {
1159             r_pipe_freeze_trace.csr.mtvec_n, 6'h0,
r_pipe_freeze_trace.csr.mtvec_mode_n
1160         };
1161         ;
1162         trace_id.m_csr.mtvec_wmask =
r_pipe_freeze_trace.csr.mtvec_we ? '1' : '0';
1163     endfunction
1164
1165     function void dcsr_to_id();
1166         trace_id.m_csr.dcsr_wdata = trace_id.m_csr.dcsr_we ?
trace_id.m_csr.dcsr_wdata : r_pipe_freeze_trace.csr.dcsr_n;
1167         trace_id.m_csr.dcsr_we = r_pipe_freeze_trace.csr.dcsr_we |
trace_id.m_csr.dcsr_we;
1168         trace_id.m_csr.dcsr_rdata =
r_pipe_freeze_trace.csr.dcsr_q;
1169         trace_id.m_csr.dcsr_rmask = '1';
1170         trace_id.m_csr.dcsr_wdata =
r_pipe_freeze_trace.csr.dcsr_n;
1171         trace_id.m_csr.dcsr_wmask =
r_pipe_freeze_trace.csr.dcsr_we ? '1' : '0';
1172     endfunction
1173
1174     function void csrId_to_id();
1175         trace_id.m_csr.mvendorid_we = '0';
1176         trace_id.m_csr.mvendorid_rdata =
r_pipe_freeze_trace.csr.mvendorid;
1177         trace_id.m_csr.mvendorid_rmask = '1';
1178         trace_id.m_csr.mvendorid_wdata = '0';
1179         trace_id.m_csr.mvendorid_wmask = '0';
1180
1181         trace_id.m_csr.marchid_we = '0';
1182         trace_id.m_csr.marchid_rdata =
r_pipe_freeze_trace.csr.marchid;
1183         trace_id.m_csr.marchid_rmask = '0';
1184         trace_id.m_csr.marchid_wdata = '0';
1185         trace_id.m_csr.marchid_wmask = '0';
1186     endfunction
1187
1188     function void lpcount1_to_id();
1189         trace_id.m_csr.lpcount1_we = '0';
1190         trace_id.m_csr.lpcount1_rdata =
r_pipe_freeze_trace.hwloop.counter_q[1];
1191         trace_id.m_csr.lpcount1_rmask = '1';
1192         trace_id.m_csr.lpcount1_wdata = '0';
1193         trace_id.m_csr.lpcount1_wmask = '0';
1194     endfunction
1195
1196     function void lpcount0_to_id();
1197         trace_id.m_csr.lpcount0_we = '0';
1198         trace_id.m_csr.lpcount0_rdata =
r_pipe_freeze_trace.hwloop.counter_q[0];
1199         trace_id.m_csr.lpcount0_rmask = '1';
1200         trace_id.m_csr.lpcount0_wdata = '0';
1201         trace_id.m_csr.lpcount0_wmask = '0';
1202     endfunction
1203
1204     function void lpend0_to_id();
1205         trace_id.m_csr.lpend0_we = '0';
1206         trace_id.m_csr.lpend0_rdata =
r_pipe_freeze_trace.hwloop.end_q[0];
1207         trace_id.m_csr.lpend0_rmask = '1';
1208         trace_id.m_csr.lpend0_wdata = '0';
1209         trace_id.m_csr.lpend0_wmask = '0';
1210     endfunction
1211
1212     function void lpend1_to_id();
1213         trace_id.m_csr.lpend1_we = '0';
1214         trace_id.m_csr.lpend1_rdata =
r_pipe_freeze_trace.hwloop.end_q[1];
1215         trace_id.m_csr.lpend1_rmask = '1';
1216         trace_id.m_csr.lpend1_wdata = '0';
1217         trace_id.m_csr.lpend1_wmask = '0';
1218     endfunction
1219
1220     function void lpstart0_to_id();
1221         trace_id.m_csr.lpstart0_we = '0';
1222         trace_id.m_csr.lpstart0_rdata =
r_pipe_freeze_trace.hwloop.start_q[0];
1223         trace_id.m_csr.lpstart0_rmask = '1';
1224         trace_id.m_csr.lpstart0_wdata = '0';
1225         trace_id.m_csr.lpstart0_wmask = '0';
1226     endfunction
1227
1228     function void lpstart1_to_id();
1229         trace_id.m_csr.lpstart1_we = '0';
1230         trace_id.m_csr.lpstart1_rdata =
r_pipe_freeze_trace.hwloop.start_q[1];
1231         trace_id.m_csr.lpstart1_rmask = '1';
1232         trace_id.m_csr.lpstart1_wdata = '0';
1233         trace_id.m_csr.lpstart1_wmask = '0';
1234     endfunction
1235
1236     function void hwloop_to_id();
1237         lpcount0_to_id();
1238         lpend0_to_id();
1239         lpstart0_to_id();
1240
1241         lpcount1_to_id();
1242         lpend1_to_id();
1243         lpstart1_to_id();
1244     endfunction
1245
1246     bit s_was_flush; //debug exception is flagged as trap only
if preceed by a flush
1247     //work around until I find the coreect way to distinguish
trap
1248     function void check_trap();
1249         bit s_dbg_exception, s_exception, s_irq;
1250         s_dbg_exception = '1'b0;
1251         s_exception = '1'b0;
1252         s_irq = '1'b0;
1253
1254         if (r_pipe_freeze_trace.pc_mux == PC_EXCEPTION) begin
1255             if (r_pipe_freeze_trace.exc_pc_mux[1] == '1'b1) begin
1256                 if (r_pipe_freeze_trace.ctr_fsm_cs != XRET_JUMP) be-
gin
1257                     s_dbg_exception = '1'b1;
1258                 end
1259             end
1260             if (r_pipe_freeze_trace.exc_pc_mux == EXC_PC_EXCEPTION)
begin
1261                 s_exception = '1'b1;
1262             end
1263             if (r_pipe_freeze_trace.exc_pc_mux == EXC_PC_IRQ) begin
1264                 s_irq = '1'b1;
1265                 trace_if.m_is_irq = '1'b1;
1266             end
1267         end
1268
1269         if (s_was_flush == '1'b0) begin
1270             s_dbg_exception = '1'b0;
1271         end
1272     endfunction
1273
1274     /*
1275     * This tracer works with three process,
1276     *
1277     * The first one simply stores information from the pipeline
1278     * every clock cycles and sends those information to the second process
1279     * The second process uses those information to fill the
1280     * rvfi interface with the correct values
1281     * The third updates the rvfi interface
1282     */
1283     `define CSR_FROM_PIPE(TRACE_NAME, CSR_NAME) \
1284         if(!trace``TRACE_NAME``.m_csr``CSR_NAME``_we) begin \
1285             trace``TRACE_NAME``.m_csr``CSR_NAME``_wdata =
r_pipe_freeze_trace.csr``CSR_NAME``_n; \
1286         end \
1287         if (r_pipe_freeze_trace.csr``CSR_NAME``_we) begin \
1288             trace``TRACE_NAME``.m_csr``CSR_NAME``_we =
r_pipe_freeze_trace.csr``CSR_NAME``_we; \
1289             trace``TRACE_NAME``.m_csr``CSR_NAME``_wdata =
r_pipe_freeze_trace.csr``CSR_NAME``_n; \
1290             trace``TRACE_NAME``.m_csr``CSR_NAME``_wmask = '1'; \
1291         end \
1292         trace``TRACE_NAME``.m_csr``CSR_NAME``_rdata =
r_pipe_freeze_trace.csr``CSR_NAME``_q; \
1293         trace``TRACE_NAME``.m_csr``CSR_NAME``_rmask = '1';
1294
1295     event e_mstatus_to_id;
1296     event e_fregs_dirty_1, e_fregs_dirty_2, e_fregs_dirty_3;
1297     function void mstatus_to_id();
1298         `CSR_FROM_PIPE(id, mstatus)
1299         `CSR_FROM_PIPE(id, mstatus_fs)
1300         if(r_pipe_freeze_trace.csr.fregs_we && !
r_pipe_freeze_trace.csr.mstatus_fs_we && !
(r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we)) begin //writes happening in
ex that needs to be reported to id
1301             trace_id.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1302             ->e_fregs_dirty_2;
1303         end
1304         ->e_mstatus_to_id;
1305     endfunction
1306     //those event are for debug purpose
1307     event e_dev_send_wb_1, e_dev_send_wb_2;
1308     event
1309         e_dev_commit_rf_to_ex_1,
1310         e_dev_commit_rf_to_ex_2,
1311         e_dev_commit_rf_to_ex_3,
1312         e_dev_commit_rf_to_ex_4,

```



```

1313     e_dev_commit_rf_to_ex_5;
1314 event e_if_2_id_1, e_if_2_id_2, e_if_2_id_3, e_if_2_id_4;
1315 event e_ex_to_wb_1, e_ex_to_wb_2;
1316 event e_id_to_ex_1, e_id_to_ex_2;
1317 event e_commit_dpc;
1318 event e_csr_in_ex, e_csr_irq;
1319
1320 event e_send_rvfi_trace_apu_resp;
1321 event
1322     e_send_rvfi_trace_ex_1,
1323     e_send_rvfi_trace_ex_2,
1324     e_send_rvfi_trace_ex_3,
1325     e_send_rvfi_trace_ex_4,
1326     e_send_rvfi_trace_ex_5,
1327     e_send_rvfi_trace_ex_6;
1328 event e_send_rvfi_trace_wb_1, e_send_rvfi_trace_wb_2,
e_send_rvfi_trace_wb_3;
1329 event e_send_rvfi_trace_id_1;
1330
1331 //used to match memory response to memory request and cor-
responding instruction
1332 integer cnt_data_req, cnt_data_resp;
1333 integer cnt_apu_req, cnt_apu_resp;
1334
1335 insn_trace_t apu_trace_q[$];
1336 insn_trace_t trace_apu_req, trace_apu_resp;
1337
1338 function void csr_to_apu_resp();
1339     `CSR_FROM_PIPE(apu_resp, fcsr)
1340     `CSR_FROM_PIPE(apu_resp, fflags)
1341
1342     `CSR_FROM_PIPE(apu_resp, mstatus_fs)
1343
1344     if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_id.m_order > trace_apu_resp.m_order)) begin
1345         trace_id.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1346     end
1347     if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_ex.m_order > trace_apu_resp.m_order)) begin
1348         trace_ex.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1349     end
1350     if (r_pipe_freeze_trace.csr.mstatus_fs_we &&
(trace_wb.m_order > trace_apu_resp.m_order)) begin
1351         trace_wb.m_csr.mstatus_fs_rdata =
r_pipe_freeze_trace.csr.mstatus_fs_n;
1352     end
1353 endfunction
1354
1355 function void csr_to_apu_req();
1356     `CSR_FROM_PIPE(apu_req, misa)
1357     `CSR_FROM_PIPE(apu_req, tdata1)
1358     `CSR_FROM_PIPE(apu_req, tdata2)
1359     trace_apu_req.m_csr.tinfo_we = '0; // READ ONLY csr-t-
info_we_i;
1360     trace_apu_req.m_csr.tinfo_rdata =
r_pipe_freeze_trace.csr.tinfo_q;
1361     trace_apu_req.m_csr.tinfo_rmask = '1;
1362     trace_apu_req.m_csr.tinfo_wdata =
r_pipe_freeze_trace.csr.tinfo_n;
1363     trace_apu_req.m_csr.tinfo_wmask = '0;
1364
1365     trace_apu_req.m_csr.minstret_we =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2];
1366     trace_apu_req.m_csr.minstret_rdata =
r_pipe_freeze_trace.csr.mhpmcounter_q[2];
1367     trace_apu_req.m_csr.minstret_rmask = '1;
1368     trace_apu_req.m_csr.minstret_wdata =
r_pipe_freeze_trace.csr.mhpmcounter_q;
1369     trace_apu_req.m_csr.minstret_wmask =
r_pipe_freeze_trace.csr.mhpmcounter_write_lower[2] ? '1 : '0;
1370
1371     trace_apu_req.m_csr.lpcount0_we = '0;
1372     trace_apu_req.m_csr.lpcount0_rdata =
r_pipe_freeze_trace.hwloop.counter_q[0];
1373     trace_apu_req.m_csr.lpcount0_rmask = '1;
1374     trace_apu_req.m_csr.lpcount0_wdata = '0;
1375     trace_apu_req.m_csr.lpcount0_wmask = '0;
1376
1377     trace_apu_req.m_csr.lpcount1_we = '0;
1378     trace_apu_req.m_csr.lpcount1_rdata =
r_pipe_freeze_trace.hwloop.counter_q[1];
1379     trace_apu_req.m_csr.lpcount1_rmask = '1;
1380     trace_apu_req.m_csr.lpcount1_wdata = '0;
1381     trace_apu_req.m_csr.lpcount1_wmask = '0;
1382
1383     `CSR_FROM_PIPE(apu_req, frm)
1384
1385 endfunction
1386
1387
1388 function void fcsr_to_wb();
1389     `CSR_FROM_PIPE(wb, fflags)
1390     `CSR_FROM_PIPE(wb, frm)
1391     `CSR_FROM_PIPE(wb, fcsr)
1392     trace_wb.m_csr.fflags_wmask = '0;
1393     trace_wb.m_csr.frm_wmask = '0;
1394     trace_wb.m_csr.fcsr_wmask = '0;
1395 endfunction
1396
1397 bit s_apu_to_alu_port;
1398 bit s_apu_to_lsu_port;
1399
1400 function void apu_resp();
1401     if (!r_pipe_freeze_trace.wb_contention_lsu) begin
1402         if (apu_trace_q.size() > 0) begin
1403             trace_apu_resp = apu_trace_q.pop_front();
1404             if (s_apu_to_alu_port) begin
1405                 if (r_pipe_freeze_trace.ex_reg_we) begin
1406                     trace_apu_resp.m_rd_addr[0] =
r_pipe_freeze_trace.ex_reg_addr;
1407                     trace_apu_resp.m_rd_wdata[0] =
r_pipe_freeze_trace.ex_reg_wdata;
1408                 end
1409                 end else if (s_apu_to_lsu_port) begin
1410                     if (r_pipe_freeze_trace.rf_we_wb) begin
1411                         trace_apu_resp.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1412                         trace_apu_resp.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1413                     end
1414                 end
1415                 csr_to_apu_resp();
1416
1417                 trace_apu_resp.m_stop_cycle = cycles;
1418                 trace_apu_resp.m_stop_time = $time;
1419                 send_rvfi(trace_apu_resp);
1420                 ->e_send_rvfi_trace_apu_resp;
1421             end
1422         end
1423     endfunction
1424
1425 /*
1426 * Decoding is complete and instruction enters execute stage
1427 * If at that time, minstret is not asserted it means we
have a trap
1428 */
1429 bit s_is_pc_set; //If pc_set, wait until next trace_id to
commit csr changes
1430 bit s_is_irq_start;
1431 bit s_id_done;
1432 function void if_to_id();
1433     if (trace_id.m_valid) begin
1434         `CSR_FROM_PIPE(id, misa)
1435         `CSR_FROM_PIPE(id, tdata1)
1436         `CSR_FROM_PIPE(id, tdata2)
1437         tinfo_to_trace(trace_id);
1438         `CSR_FROM_PIPE(id, mip)
1439         send_rvfi(trace_id);
1440     end
1441     trace_id.init(trace_if);
1442     trace_id.m_trap = ~r_pipe_freeze_trace.minstret;
1443     trace_id.m_is_illegal = trace_id.m_is_illegal |
r_pipe_freeze_trace.is_illegal;
1444     `CSR_FROM_PIPE(id, dpc)
1445     s_is_pc_set = 1'b0;
1446     s_is_irq_start = 1'b0;
1447     trace_if.m_valid = 1'b0;
1448     s_id_done = 1'b0;
1449 endfunction
1450
1451 function logic [31:0] be_to_mask(logic [3:0] be);
1452     logic [31:0] mask;
1453     mask[7:0] = (be[0] == 1'b1) ? 8'hFF : 8'h00;
1454     mask[15:8] = (be[1] == 1'b1) ? 8'hFF : 8'h00;
1455     mask[23:16] = (be[2] == 1'b1) ? 8'hFF : 8'h00;
1456     mask[31:24] = (be[3] == 1'b1) ? 8'hFF : 8'h00;
1457
1458     be_to_mask = mask;
1459     return mask;
1460 endfunction
1461
1462 function void commit_rf_to_trace(insn_trace_t m_trace);
1463     if (m_trace.m_got_ex_reg) begin
1464         m_trace.m_rd_addr[1] = r_pipe_freeze_trace.rf_addr_wb;
1465         m_trace.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1466         m_trace.m_2_rd_insn = 1'b1;
1467         m_trace.m_got_first_data = 1'b1;
1468     end else begin
1469         m_trace.m_rd_addr[0] = r_pipe_freeze_trace.rf_addr_wb;
1470         m_trace.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1471         m_trace.m_got_first_data = 1'b1;
1472     end
1473 endfunction
1474
1475 task compute_pipeline();
1476     bit s_new_valid_insn;
1477     bit s_ex_valid_adjusted;
1478     bit s_wb_valid_adjusted;
1479
1480     bit s_apu_wb_ok;
1481     bit s_apu_0_cycle_reps;
1482
1483     bit s_fflags_we_non_apu;
1484     bit s_frm_we_non_apu;
1485     bit s_fcsr_we_non_apu;
1486
1487     bit s_skip_wb; // used to skip wb monitoring when apu
resp and not lsu

```

```

1488
1489 bit s_core_is_decoding; // For readability, ctrl_fsm is
DECODE or DECODE_HWLOOP
1490
1491 bit s_ex_reg_we_adjusted; //ex_reg_we
1492 bit s_rf_we_wb_adjusted; //
1493
1494 bit s_dont_override_mstatus_fs_id;
1495
1496 trace_if = new();
1497 trace_id = new();
1498 trace_ex = new();
1499 trace_wb = new();
1500 s_new_valid_insn = 1'b0;
1501 s_ex_valid_adjusted = 1'b0;
1502
1503 s_id_done = 1'b0;
1504 s_apu_wb_ok = 1'b0;
1505 s_apu_0_cycle_reps = 1'b0;
1506
1507 next_send = 1;
1508 cnt_data_req = 0;
1509 cnt_data_resp = 0;
1510 cnt_apu_req = 0;
1511 cnt_apu_resp = 0;
1512 csr_is_irq = 0;
1513 is_dbg_taken = 0;
1514 s_was_flush = 1'b0;
1515
1516 s_is_pc_set = 1'b0;
1517 s_is_irq_start = 1'b0;
1518
1519 s_is_pc_set = 1'b0;
1520 s_is_irq_start = 1'b0;
1521 s_skip_wb = 1'b0;
1522
1523 s_core_is_decoding = 1'b0;
1524
1525 s_ex_reg_we_adjusted = 1'b0;
1526 s_rf_we_wb_adjusted = 1'b0;
1527
1528 s_dont_override_mstatus_fs_id = 1'b0;
1529
1530 forever begin
1531 wait(e_pipe_monitor_ok.triggered); // event triggered
1532 #1;
1533
1534 s_core_is_decoding = (r_pipe_freeze_trace.ctrl_fsm_cs ==
DECODE) || (r_pipe_freeze_trace.ctrl_fsm_cs == DECODE_HWLOOP);
1535 check_trap();
1536
1537 pc_mux_interrupt = 1'b0;
1538 if (r_pipe_freeze_trace.pc_mux == 4'b0100) begin
1539 if (r_pipe_freeze_trace.exc_pc_mux == 3'b001) begin
1540 pc_mux_interrupt = 1'b1;
1541 s_is_irq_start = 1'b1;
1542 end
1543 end
1544
1545 if (r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID &&
r_pipe_freeze_trace.ebrk_insn_dec) begin
1546 if (trace_id.m_valid) begin
1547 `CSR_FROM_PIPE(id, misa)
1548 `CSR_FROM_PIPE(id, tdata1)
1549 `CSR_FROM_PIPE(id, tdata2)
1550 tinfo_to_trace(trace_id);
1551 `CSR_FROM_PIPE(id, mip)
1552 end
1553 end
1554
1555 if (r_pipe_freeze_trace.data_rvalid) begin
1556 cnt_data_resp = cnt_data_resp + 1;
1557 end
1558
1559 if (r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt) begin
1560 cnt_apu_req = cnt_apu_req + 1;
1561 end
1562 if (r_pipe_freeze_trace.apu_rvalid) begin
1563 cnt_apu_resp = cnt_apu_resp + 1;
1564 if (r_pipe_freeze_trace.apu_singlecycle |
r_pipe_freeze_trace.apu_multicycle) begin
1565 s_apu_to_alu_port = 1'b1;
1566 s_apu_to_lsu_port = 1'b0;
1567 end else begin
1568 s_apu_to_lsu_port = 1'b1;
1569 s_apu_to_alu_port = 1'b0;
1570 end
1571 end else begin
1572 s_apu_to_lsu_port = 1'b0;
1573 s_apu_to_alu_port = 1'b0;
1574 end
1575
1576 if (r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt && r_pipe_freeze_trace.apu_rvalid &&
(cnt_apu_resp == cnt_apu_req)) begin
1577 s_apu_0_cycle_reps = 1'b1;
1578 end else begin
1579 s_apu_0_cycle_reps = 1'b0;
1580 end

```

```

1581
1582 if (trace_ex.m_valid & s_wb_valid_adjusted) begin
1583 // Used flopped values in case write happened before
wb_valid
1584 sample_perf_counter_to_trace(trace_ex);
1585 trace_ex.m_csr.got_minstret = '1;
1586 end
1587
1588 if (trace_wb.m_valid & trace_wb.m_is_apu &
(trace_wb.m_apu_req_id == cnt_apu_resp)) begin
1589 s_apu_wb_ok = 1'b1;
1590 end else begin
1591 s_apu_wb_ok = 1'b0;
1592 end
1593
1594 s_new_valid_insn = r_pipe_freeze_trace.id_valid &&
r_pipe_freeze_trace.is_decoding; // && !
r_pipe_freeze_trace.apu_rvalid;
1595
1596 s_wb_valid_adjusted = r_pipe_freeze_trace.wb_valid &&
(s_core_is_decoding || (r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX)
|| (r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_WB) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF)); // && !
r_pipe_freeze_trace.apu_rvalid;
1597 s_ex_reg_we_adjusted = r_pipe_freeze_trace.ex_reg_we &&
r_pipe_freeze_trace.mult_ready && r_pipe_freeze_trace.alu_ready &&
r_pipe_freeze_trace.lsu_ready_ex && !s_apu_to_alu_port;
1598 s_rf_we_wb_adjusted = r_pipe_freeze_trace.rf_we_wb &&
(~r_pipe_freeze_trace.data_misaligned_ex &&
r_pipe_freeze_trace.wb_ready) && (!s_apu_to_lsu_port ||
r_pipe_freeze_trace.wb_contention_lsu);
1599
1600 s_fflags_we_non_apu = 1'b0;
1601 if (r_pipe_freeze_trace.csr.fflags_we) begin
1602 if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1603 s_fflags_we_non_apu = 1'b1;
1604 end
1605 end
1606
1607 s_frm_we_non_apu = 1'b0;
1608 if (r_pipe_freeze_trace.csr.frm_we) begin
1609 if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1610 s_frm_we_non_apu = 1'b1;
1611 end
1612 end
1613
1614 s_fcsr_we_non_apu = 1'b0;
1615 if (r_pipe_freeze_trace.csr.fcsr_we) begin
1616 if (cnt_apu_resp == cnt_apu_req) begin //No ongoing
apu instruction
1617 s_fcsr_we_non_apu = 1'b1;
1618 end
1619 end
1620
1621 //WB_STAGE
1622 s_skip_wb = 1'b0;
1623 if (r_pipe_freeze_trace.apu_rvalid &&
(apu_trace.q.size() > 0)) begin
1624 apu_resp();
1625 if (!r_pipe_freeze_trace.data_rvalid) begin
1626 s_skip_wb = 1'b1;
1627 end
1628 end
1629
1630 if (trace_wb.m_valid && !s_skip_wb && s_rf_we_wb_adju-
ted) begin
1631 if (trace_wb.m_2_rd_insn) begin
1632 trace_wb.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1633 trace_wb.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1634 end else if (trace_wb.m_ex_fw) begin
1635 trace_wb.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1636 trace_wb.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1637 trace_wb.m_2_rd_insn = 1'b1;
1638 end else begin
1639 trace_wb.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1640 trace_wb.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1641 end
1642
1643 if (r_pipe_freeze_trace.csr.fregs_we &&
r_pipe_freeze_trace.rf_we_wb && r_pipe_freeze_trace.rf_addr_wb[5])
begin //Catching mstatus updates caused by flw
1644 `CSR_FROM_PIPE(wb, mstatus_fs)
1645 trace_wb.m_csr.mstatus_fs_we = 1'b1;
1646 trace_wb.m_csr.mstatus_fs_wmask = '1;
1647 if (r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we) begin //In this specific
case, two writes to mstatus_fs happen at the same time. We need to
recreate the writes caused by fregs_we
1648 trace_wb.m_csr.mstatus_fs_wdata = FS_DIRTY;
1649 end
1650 ->e_fregs_dirty_1;

```

```

1651     end
1652
1653     send_rvfi(trace_wb);
1654     ->e_dev_send_wb_1; ->e_send_rvfi_trace_wb_2;
1655     trace_wb.m_valid = 1'b0;
1656
1657 end
1658
1659 if (trace_ex.m_valid) begin
1660   if (trace_ex.m_instret_smample_trigger == 1) begin //
time to sample instret
1661     sample_perf_counter_to_trace(trace_ex);
1662   end
1663   trace_ex.m_instret_smample_trigger =
trace_ex.m_instret_smample_trigger + 1;
1664
1665   `CSR_FROM_PIPE(ex, misa)
1666   `CSR_FROM_PIPE(ex, tdata1)
1667   `CSR_FROM_PIPE(ex, tdata2)
1668   tinfo_to_trace(trace_ex);
1669
1670   if (s_rf_we_wb_adjusted) begin
1671     ->e_dev_commit_rf_to_ex_4;
1672     if (!(trace_ex.m_got_ex_reg) &&
trace_ex.m_mem_req_id_valid[0]) begin
1673       trace_ex.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1674       trace_ex.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1675       trace_ex.m_got_first_data = 1'b1;
1676       trace_ex.m_mem_req_id_valid[0] = 1'b0;
1677     end else if (trace_ex.m_mem_req_id_valid[1]) begin
1678       trace_ex.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1679       trace_ex.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1680       trace_ex.m_got_first_data = 1'b1;
1681       trace_ex.m_mem_req_id_valid[1] = 1'b0;
1682     end
1683   end
1684
1685   if (s_wb_valid_adjusted) begin
1686     if (trace_wb.m_valid) begin
1687       send_rvfi(trace_ex);
1688       trace_ex.m_valid = 1'b0;
1689       ->e_send_rvfi_trace_ex_2;
1690     end else begin
1691
1692       if (s_rf_we_wb_adjusted) begin
1693         ->e_dev_commit_rf_to_ex_1;
1694         commit_rf_to_trace(trace_ex);
1695
1696         if (r_pipe_freeze_trace.csr.fregs_we &&
(r_pipe_freeze_trace.rf_we_wb && r_pipe_freeze_trace.rf_addr_wb[5]))
begin //Catching mstatus_fs updates caused by flw
1697           `CSR_FROM_PIPE(ex, mstatus_fs)
1698           trace_ex.m_csr.mstatus_fs_we = 1'b1;
1699           trace_ex.m_csr.mstatus_fs_wmask = '1;
1700           if (r_pipe_freeze_trace.csr.we &&
r_pipe_freeze_trace.csr.mstatus_fs_we) begin //In this specific
case, two writes to mstatus_fs happen at the same time. We need to
recreate the writes caused by fregs_we
1701             trace_ex.m_csr.mstatus_fs_wdata = FS_DIRTY;
1702           end else begin
1703             trace_id.m_csr.mstatus_fs_rdata =
trace_ex.m_csr.mstatus_fs_wdata;
1704             s_dont_override_mstatus_fs_id = 1'b1;
1705           end
1706           ->e_fregs_dirty_3;
1707         end
1708
1709         send_rvfi(trace_ex);
1710         trace_ex.m_valid = 1'b0;
1711       end
1712     end else begin
1713       if (trace_ex.m_is_load) begin // only move rel-
evant instr in wb stage
1714         ->e_ex_to_wb_1;
1715         trace_wb.move_down_pipe(trace_ex);
1716       end else begin
1717         send_rvfi(trace_ex);
1718         ->e_send_rvfi_trace_ex_6;
1719       end
1720       trace_ex.m_valid = 1'b0;
1721     end
1722   end
1723   end else if (s_rf_we_wb_adjusted && !s_was_flush) be-
gin
1724     ->e_dev_commit_rf_to_ex_2;
1725     commit_rf_to_trace(trace_ex);
1726   end
1727 end
1728
1729 // If mret, we need to keep the instruction in Id during
flush_ex because mstatus update happens at that time
1730 s_ex_valid_adjusted = (r_pipe_freeze_trace.ex_valid &&
r_pipe_freeze_trace.ex_ready) && (s_core_is_decoding ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_ID) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||

```

```

((r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX) && !
r_pipe_freeze_trace.mret_insn_dec));
1731 //EX_STAGE
1732
1733   if (trace_id.m_valid) begin
1734     if (trace_id.m_instret_smample_trigger == 1) begin //
time to sample instret
1735       sample_perf_counter_to_trace(trace_id);
1736       for(int idx=3; idx<32; idx++) begin
1737         sample_perf_counter_to_id(idx);
1738         sample_perf_counter_h_to_id(idx);
1739         sample_perf_event_to_trace(idx, trace_id);
1740       end
1741     end
1742     trace_id.m_instret_smample_trigger =
trace_id.m_instret_smample_trigger + 1;
1743
1744     if (trace_id.m_sample_csr_write_in_ex && !csr_is_irq
&& !s_is_irq_start) begin //First cycle after id_ready, csr write is
asserted in this cycle
1745       `CSR_FROM_PIPE(id, mstatus)
1746       if (!s_dont_override_mstatus_fs_id) begin
1747         `CSR_FROM_PIPE(id, mstatus_fs)
1748       end
1749       `CSR_FROM_PIPE(id, mepc)
1750       `CSR_FROM_PIPE(id, mcause)
1751       `CSR_FROM_PIPE(id, dscratch0)
1752       `CSR_FROM_PIPE(id, dscratch1)
1753       if (r_pipe_freeze_trace.csr.we &&
(r_pipe_freeze_trace.csr.addr == CSR_DPC)) begin
1754         `CSR_FROM_PIPE(id, dpc)
1755       end
1756
1757       `CSR_FROM_PIPE(id, mcountinhibit)
1758
1759       perf_counter_to_trace(trace_id);
1760       ->e_csr_in_ex;
1761     end
1762
1763     if (r_pipe_freeze_trace.is_decoding) begin
1764       trace_id.m_sample_csr_write_in_ex = 1'b0;
1765     end
1766     mtvec_to_id();
1767
1768     `CSR_FROM_PIPE(id, mip)
1769     `CSR_FROM_PIPE(id, misa)
1770
1771     `CSR_FROM_PIPE(id, mcountinhibit)
1772     `CSR_FROM_PIPE(id, mscratch)
1773     `CSR_FROM_PIPE(id, mie)
1774
1775     `CSR_FROM_PIPE(id, fflags)
1776     `CSR_FROM_PIPE(id, frm)
1777     `CSR_FROM_PIPE(id, fcsr)
1778
1779     if (r_pipe_freeze_trace.csr.dcsr_we) begin
1780       dcsr_to_id();
1781     end
1782
1783     if (s_fflags_we_non_apu) begin
1784       trace_id.m_fflags_we_non_apu = 1'b1;
1785     end
1786
1787     if (s_frm_we_non_apu) begin
1788       trace_id.m_frm_we_non_apu = 1'b1;
1789     end
1790
1791     if (s_fcsr_we_non_apu) begin
1792       trace_id.m_fcsr_we_non_apu = 1'b1;
1793     end
1794
1795     trace_ex.m_csr.fflags_wmask = '0;
1796     trace_ex.m_csr.frm_wmask = '0;
1797     trace_ex.m_csr.fcsr_wmask = '0;
1798
1799     if (r_pipe_freeze_trace.ctrl_fsm_cs == XRET_JUMP) be-
gin //xret exit pipeline
1800       tinfo_to_trace(trace_id);
1801       `CSR_FROM_PIPE(id, tdata1)
1802       `CSR_FROM_PIPE(id, tdata2)
1803       send_rvfi(trace_id);
1804       trace_id.m_valid = 1'b0;
1805       s_dont_override_mstatus_fs_id = 1'b0;
1806     end
1807
1808     if (r_pipe_freeze_trace.apu_req &&
r_pipe_freeze_trace.apu_gnt) begin
1809       trace_id.m_is_apu = 1'b1;
1810       trace_id.m_apu_req_id = cnt_apu_req;
1811       trace_apu_req = new();
1812       trace_apu_req.copy_full(trace_id);
1813       csr_to_apu_req();
1814       trace_apu_req.set_to_apu();
1815       apu_trace_q.push_back(trace_apu_req);
1816       trace_id.m_valid = 1'b0;
1817       s_dont_override_mstatus_fs_id = 1'b0;
1818
1819       if (r_pipe_freeze_trace.apu_rvalid && (cnt_apu_req ==
cnt_apu_resp)) begin//APU return in the same cycle
1820         apu_resp();

```

```

1821     end
1822   end
1823
1824   if (s_ex_valid_adjusted) begin //A valid instruction
goes from ID to EX
1825     if (trace_ex.m_valid) begin //We need to bypass wb
1826       send_rvfi(trace_ex);
1827       trace_ex.m_valid = 1'b0;
1828       ->e_send_rvfi_trace_ex_3;
1829     end
1830     if (r_pipe_freeze_trace.ex_reg_we && !
s_apu_to_alu_port) begin
1831       trace_id.m_ex_fw = 1'b1;
1832       trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.ex_reg_addr;
1833       trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.ex_reg_wdata;
1834       trace_id.m_got_ex_reg = 1'b1;
1835     end else if (!trace_ex.m_valid & s_rf_we_wb_adjusted
& !trace_id.m_ex_fw) begin
1836       trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1837       trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1838     end else if (s_rf_we_wb_adjusted) begin
1839       trace_id.m_rd_addr[1] =
r_pipe_freeze_trace.rf_addr_wb;
1840       trace_id.m_rd_wdata[1] =
r_pipe_freeze_trace.rf_wdata_wb;
1841       trace_id.m_2_rd_insn = 1'b1;
1842     end
1843
1844     if (r_pipe_freeze_trace.data_req_ex) begin
1845       cnt_data_req = cnt_data_req + 1;
1846       trace_id.m_is_memory = 1'b1;
1847       trace_id.m_mem_req_id[0] = cnt_data_req;
1848       trace_id.m_mem_req_id_valid[0] = 1'b1;
1849
1850       trace_id.m_mem_addr =
r_pipe_freeze_trace.data_addr_pmp;
1851       if (r_pipe_freeze_trace.data_misaligned) begin
1852         cnt_data_req = cnt_data_req + 1;
1853         trace_id.m_mem_req_id[0] = cnt_data_req;
1854       end
1855
1856       if (!r_pipe_freeze_trace.data_we_ex) begin
1857         trace_id.m_is_load = 1'b1;
1858         trace_id.m_mem.wmask =
be_to_mask(r_pipe_freeze_trace.lsu_data_be); //1;
1859       end else begin
1860         trace_id.m_mem.rmask =
be_to_mask(r_pipe_freeze_trace.lsu_data_be); //1;
1861       end
1862     end
1863
1864     if (trace_id.m_got_ex_reg) begin // Shift index 0
to 1
1865       trace_id.m_mem_req_id[1] =
trace_id.m_mem_req_id[0];
1866       trace_id.m_mem_req_id[0] = 0;
1867       trace_id.m_mem_req_id_valid[1] = 1'b1;
1868       trace_id.m_mem_req_id_valid[0] = 1'b0;
1869     end
1870     hwlloop_to_id();
1871     trace_ex.move_down_pipe(trace_id); // The instruc-
tion moves forward from ID to EX
1872     trace_id.m_valid = 1'b0;
1873     s_dont_override_mstatus_fs_id = 1'b0;
1874     ->e_id_to_ex_1;
1875
1876     end else if (r_pipe_freeze_trace.ex_reg_we &&
r_pipe_freeze_trace.rf_alu_we_ex) begin
1877       trace_id.m_ex_fw = 1'b1;
1878       trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.ex_reg_addr;
1879       trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.ex_reg_wdata;
1880       trace_id.m_got_ex_reg = 1'b1;
1881       trace_id.m_got_regs_write = 1'b1;
1882       // mem_req_id[0] already set here indicates req_id
was set before rf write from EX
1883       // Hence adjust the req_id again here for such cases
1884       if (trace_id.m_mem_req_id[0] != 0) begin
1885         trace_id.m_mem_req_id[1] =
trace_id.m_mem_req_id[0];
1886         trace_id.m_mem_req_id[0] = 0;
1887         trace_id.m_mem_req_id_valid[0] = 1'b0;
1888         trace_id.m_mem_req_id_valid[1] = 1'b1;
1889       end
1890     end
1891   end //trace_if.m_valid
1892
1893   //ID_STAGE
1894   if (s_new_valid_insn) begin // There is a new valid in-
struction
1895     if (trace_id.m_valid) begin
1896       if (trace_ex.m_valid) begin
1897         if (trace_wb.m_valid) begin
1898           send_rvfi(trace_ex);

```

```

1899       ->e_send_rvfi_trace_ex_4;
1900     end else begin
1901       if (trace_ex.m_is_load) begin // only move rel-
evant instr in wb stage
1902         ->e_ex_to_wb_2;
1903         trace_wb.move_down_pipe(trace_ex);
1904       end else begin
1905         send_rvfi(trace_ex);
1906         ->e_send_rvfi_trace_ex_5;
1907       end
1908     end
1909     trace_ex.m_valid = 1'b0;
1910   end
1911   if (r_pipe_freeze_trace.data_req_ex) begin
1912     cnt_data_req = cnt_data_req + 1;
1913     trace_id.m_is_memory = 1'b1;
1914     trace_id.m_mem_req_id[0] = cnt_data_req;
1915     trace_id.m_mem_req_id_valid[0] = 1'b1;
1916     trace_id.m_mem_addr =
r_pipe_freeze_trace.data_addr_pmp;
1917     if (r_pipe_freeze_trace.data_misaligned) begin
1918       cnt_data_req = cnt_data_req + 1;
1919       trace_id.m_mem_req_id[0] = cnt_data_req;
1920     end
1921     if (!r_pipe_freeze_trace.data_we_ex) begin
1922       trace_id.m_is_load = 1'b1;
1923     end
1924
1925     if (trace_id.m_got_ex_reg) begin // Shift index 0
to 1
1926       trace_id.m_mem_req_id[1] =
trace_id.m_mem_req_id[0];
1927       trace_id.m_mem_req_id[0] = 0;
1928       trace_id.m_mem_req_id_valid[0] = 1'b0;
1929       trace_id.m_mem_req_id_valid[1] = 1'b1;
1930     end
1931     end else if (s_rf_we_wb_adjusted && !
r_pipe_freeze_trace.ex_reg_we) begin
1932       trace_id.m_rd_addr[0] =
r_pipe_freeze_trace.rf_addr_wb;
1933       trace_id.m_rd_wdata[0] =
r_pipe_freeze_trace.rf_wdata_wb;
1934     end
1935     hwlloop_to_id();
1936     trace_ex.move_down_pipe(trace_id);
1937     trace_id.m_valid = 1'b0;
1938     s_dont_override_mstatus_fs_id = 1'b0;
1939     ->e_id_to_ex_2;
1940   end
1941   if_to_id();
1942   trace_id.m_is_ebreak = trace_if.m_is_ebreak;
1943   csrId_to_id();
1944   `CSR_FROM_PIPE(id, dscratch0)
1945   `CSR_FROM_PIPE(id, dscratch1)
1946   mstatus_to_id();
1947   ->e_if_2_id_1;
1948   end else begin
1949     if (trace_id.m_valid) begin
1950       `CSR_FROM_PIPE(id, dscratch0)
1951       `CSR_FROM_PIPE(id, dscratch1)
1952     end
1953   end
1954
1955   //IF_STAGE
1956   if (trace_if.m_valid) begin
1957     if (r_pipe_freeze_trace.is_illegal &&
r_pipe_freeze_trace.is_decoding) begin
1958       trace_if.m_is_illegal = 1'b1;
1959     end
1960   end
1961
1962   if (r_pipe_freeze_trace.if_valid &&
r_pipe_freeze_trace.if_ready && r_pipe_freeze_trace.instr_valid_if)
begin
1963     if (trace_if.m_valid) begin
1964       if (r_pipe_freeze_trace.id_valid &&
r_pipe_freeze_trace.id_ready && !trace_id.m_valid &&
r_pipe_freeze_trace.ebrk_insn_dec) begin
1965         if_to_id();
1966         trace_id.m_is_ebreak = '1; //
trace_if.m_is_ebreak;
1967         ->e_if_2_id_2;
1968       end else if (trace_if.m_is_illegal) begin
1969         if_to_id();
1970         ->e_if_2_id_3;
1971       end else if (r_pipe_freeze_trace.ecall_insn_dec) be-
gin
1972         if_to_id();
1973         ->e_if_2_id_4;
1974       end
1975     end
1976
1977     trace_if.m_insn = r_pipe_freeze_trace.instr_if; //In-
str comes from if, buffer for one cycle
1978     trace_if.m_pc_rdata = r_pipe_freeze_trace.pc_if;
1979     trace_if.m_dbg_taken = is_dbg_taken;
1980     trace_if.m_dbg_cause = saved_debug_cause;
1981     trace_if.m_is_ebreak = '0;
1982     trace_if.m_trap = 1'b0;

```

```

1984
1985     trace_if.m_valid = 1'b1;
1986 end
1987
1988 if (csr_is_irq && !s_is_pc_set) begin
1989     mstatus_to_id();
1990     `CSR_FROM_PIPE(id, mepc)
1991     `CSR_FROM_PIPE(id, mcause)
1992     ->e_csr_irq;
1993 end
1994
1995 if (!s_id_done && r_pipe_freeze_trace.is_decoding) begin
1996     dcsr_to_id();
1997     ->e_commit_dpc;
1998 end
1999
2000 if (r_pipe_freeze_trace.pc_set) begin
2001     s_is_pc_set = 1'b1;
2002 end
2003
2004 csr_is_irq = r_pipe_freeze_trace.csr_cause[5];
2005
2006 is_dbg_taken = ((r_pipe_freeze_trace.ctrl_fsm_cs ==
DBG_TAKEN_ID) | (r_pipe_freeze_trace.ctrl_fsm_cs == DBG_TAKEN_IF)) ?
1'b1 : 1'b0;
2006     saved_debug_cause = r_pipe_freeze_trace.debug_cause;
2007     s_id_done = r_pipe_freeze_trace.id_valid;
2008     if((r_pipe_freeze_trace.ctrl_fsm_cs == DBG_FLUSH) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_EX) ||
(r_pipe_freeze_trace.ctrl_fsm_cs == FLUSH_WB)) begin
2009         s_was_flush = 1'b1;
2010     end else begin
2011         s_was_flush = 1'b0;
2012     end
2013     #1;
2014 end
2015 endtask
2016
2017 task update_rvfi();
2018     rvfi_valid = 1'b0;
2019     rvfi_order = '0;
2020     rvfi_insn = '0;
2021     rvfi_pc_rdata = '0;
2022     rvfi_rd_addr[0] = '0;
2023     rvfi_rd_wdata[0] = '0;
2024     rvfi_rd_addr[1] = '0;
2025     rvfi_rd_wdata[1] = '0;
2026     rvfi_rs1_addr = '0;
2027     rvfi_rs2_addr = '0;
2028     rvfi_rs3_addr = '0;
2029     rvfi_rs1_rdata = '0;
2030     rvfi_rs2_rdata = '0;
2031     rvfi_rs3_rdata = '0;
2032     rvfi_mem_addr = '0;
2033     rvfi_mem_rmask = '0;
2034     rvfi_mem_wmask = '0;
2035     rvfi_mem_rdata = '0;
2036     rvfi_mem_wdata = '0;
2037
2038     rvfi_mode = 2'b11; //priv_lvl_i; //TODO: correct
this if needed
2039     $display("*****Starting update rvfi task*****\n");
2040     wait(clk_i_d == 1'b1);
2041     forever begin
2042         wait(clk_i_d == 1'b1);
2043         if (rvfi_trace.q.size() != 0) begin
2044             set_rvfi();
2045             rvfi_valid = 1'b1;
2046         end else begin
2047             rvfi_valid = 1'b0;
2048         end
2049         wait(clk_i_d == 1'b0);
2050     end
2051 end
2052 endtask
2053
2054 initial begin
2055     fork
2056         monitor_pipeline();
2057         compute_pipeline();
2058         update_rvfi();
2059     join
2060 end
2061
2062 endmodule // cv32e40p_rvfi

```

4. openhwgroup_cv32e40p/bhv/cv32e40p_rvfi_trace.sv (262 lines)

```

1 // Copyright 2024 Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
"License");
5 // you may not use this file except in compliance with the Li-
cense, or,
6 // at your option, the Apache License version 2.0.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/SHL-2.1/

```

```

10 //
11 // Unless required by applicable law or agreed to in writing,
any work
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17 ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
18 //
19 // Contributors: Halfdan Bechmann, Silicon Labs <half-
dan.bechmann@silabs.com>
20 // Yoann Pruvost, Dolphin Design
<yoann.pruvost@dolphin.fr>
21 //
22 // Description: CV32E40P RVFI
tracer
23 //
24 ///////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
25
26 module cv32e40p_rvfi_trace
27     import cv32e40p_pkg::*;
28     import cv32e40p_rvfi_pkg::*;
29     #(
30         parameter FPU = 0,
31         parameter ZFINX = 0
32     ) (
33         input logic clk_i,
34         input logic rst_ni,
35
36         input logic [31:0] hart_id_i,
37
38         input logic [31:0] imm_s3_type,
39
40         input logic rvfi_valid,
41         input logic [31:0] rvfi_insn,
42         input integer rvfi_start_cycle,
43         input time rvfi_start_time,
44         input integer rvfi_stop_cycle,
45         input time rvfi_stop_time,
46         input logic [31:0] rvfi_pc_rdata,
47         input rvfi_trap_t rvfi_trap,
48
49         input logic [ 4:0] rvfi_rd_addr [1:0],
50         input logic [31:0] rvfi_rd_wdata[1:0],
51
52         input logic rvfi_fr_d_wvalid[1:0],
53         input logic [ 4:0] rvfi_fr_d_addr [1:0],
54         input logic [31:0] rvfi_fr_d_wdata [1:0],
55         input logic rvfi_2_rd,
56
57         input logic [ 4:0] rvfi_rs1_addr,
58         input logic [ 4:0] rvfi_rs2_addr,
59         input logic [ 4:0] rvfi_rs3_addr,
60         input logic [31:0] rvfi_rs1_rdata,
61         input logic [31:0] rvfi_rs2_rdata,
62         input logic [31:0] rvfi_rs3_rdata,
63
64         input logic [ 4:0] rvfi_frs1_addr,
65         input logic [ 4:0] rvfi_frs2_addr,
66         input logic [ 4:0] rvfi_frs3_addr,
67         input logic rvfi_frs1_rvalid,
68         input logic rvfi_frs2_rvalid,
69         input logic rvfi_frs3_rvalid,
70         input logic [31:0] rvfi_frs1_rdata,
71         input logic [31:0] rvfi_frs2_rdata,
72         input logic [31:0] rvfi_frs3_rdata,
73
74         input logic [31:0] rvfi_mem_addr,
75         input logic [31:0] rvfi_mem_rmask,
76         input logic [31:0] rvfi_mem_wmask,
77         input logic [31:0] rvfi_mem_rdata,
78         input logic [31:0] rvfi_mem_wdata
79     );
80
81     import cv32e40p_tracer_pkg::*;
82
83     logic rst_n;
84     assign rst_n = rst_ni;
85
86     integer f; //file pointer
87     string fn;
88     // integer cycles;
89     string info_tag;
90
91     logic is_compressed;
92     logic [31:0] decomp_insn;
93
94     logic [5:0] rd, rs1, rs2, rs3, rs4;
95     //TODO get from rvfi

```



```

96 logic [31:0] rs1_value;
97 logic [31:0] rs2_value;
98 logic [31:0] rs3_value;
99
100 logic [31:0] rs2_value_vec;
101
102 logic [31:0] imm_u_type;
103 logic [31:0] imm_uj_type;
104 logic [31:0] imm_i_type;
105 logic [11:0] imm_iz_type;
106 logic [31:0] imm_z_type;
107 logic [31:0] imm_s_type;
108 logic [31:0] imm_sb_type;
109 logic [31:0] imm_s2_type;
110 logic [31:0] imm_vs_type;
111 logic [31:0] imm_vu_type;
112 logic [31:0] imm_shuffle_type;
113 logic [4:0] imm_clip_type;
114
115 always_comb begin
116   if (rvfi_frs1_rvalid) begin
117     rs1 = {1'b1, rvfi_frs1_addr};
118     rs1_value = rvfi_frs1_rdata;
119   end else begin
120     rs1 = {1'b0, rvfi_rs1_addr};
121     rs1_value = rvfi_rs1_rdata;
122   end
123   if (rvfi_frs2_rvalid) begin
124     rs2 = {1'b1, rvfi_frs2_addr};
125     rs2_value = rvfi_frs2_rdata;
126   end else begin
127     rs2 = {1'b0, rvfi_rs2_addr};
128     rs2_value = rvfi_rs2_rdata;
129   end
130
131   if (rvfi_frs3_rvalid) begin
132     rs3 = {1'b1, rvfi_frs3_addr};
133     rs3_value = rvfi_frs3_rdata;
134   end else begin
135     rs3 = {1'b0, rvfi_rs3_addr};
136     rs3_value = rvfi_rs3_rdata;
137   end
138
139   if (rvfi_2_rd) begin
140     if (rvfi_frd_wvalid[1]) begin
141       rd = {1'b1, rvfi_frd_addr[1]};
142     end else begin
143       rd = {1'b0, rvfi_rd_addr[1]};
144     end
145   end else if (rvfi_frd_wvalid[0]) begin
146     rd = {1'b1, rvfi_frd_addr[0]};
147   end else begin
148     rd = {1'b0, rvfi_rd_addr[0]};
149   end
150 end
151
152 assign rs4 = rs3;
153
154 cv32e40p_compressed_decoder #(
155   .FPU(FPU)
156 ) rvfi_trace_decompress_i (
157   .instr_i(rvfi_instr),
158   .instr_o(decomp_instr),
159   .is_compressed_o(is_compressed)
160 );
161
162 assign imm_i_type = {{20{decomp_instr[31]}}, de-
comp_instr[31:20]};
163 assign imm_iz_type = {20'b0, decomp_instr[31:20]};
164 assign imm_s_type = {{20{decomp_instr[31]}}, de-
comp_instr[31:25], decomp_instr[11:7]};
165 assign imm_sb_type = {
166   {19{decomp_instr[31]}},
167   decomp_instr[31],
168   decomp_instr[7],
169   decomp_instr[30:25],
170   decomp_instr[11:8],
171   1'b0
172 };
173 assign imm_u_type = {decomp_instr[31:12], 12'b0};
174 assign imm_uj_type = {
175   {12{decomp_instr[31]}}, decomp_instr[19:12], de-
comp_instr[20], decomp_instr[30:21], 1'b0
176 };
177
178 assign imm_z_type = '0; //{27'b0, de-
comp_instr[REG_S1_MSB:REG_S1_LSB]};
179
180 assign imm_s2_type = {27'b0, decomp_instr[24:20]};
181 assign imm_vs_type = '0;
182 assign imm_vu_type = '0;
183 assign imm_shuffle_type = '0;
184 assign imm_clip_type = '0;
185
186 `include "cv32e40p_instr_trace.svh"
187 instr_trace_t trace_retire;
188
189 function instr_trace_t trace_new_instr();
190   instr_trace_t trace;
191   trace = new();
192   trace.external_time = 1;

```

```

193   trace.simtime = rvfi_start_time - 1ns;
194   trace.stoptime = rvfi_stop_time;
195   trace.stopcycles = rvfi_stop_cycle;
196   trace.ctx = (rvfi_trap.trap) ? "(C)" : "";
197
198   trace.init(.cycles(rvfi_start_cycle), .pc(rvfi_pc_rdata), .compress-
ed(is_compressed),
199     .instr(decomp_instr));
200   return trace;
201 endfunction : trace_new_instr
202
203 function void apply_reg_write();
204   foreach (trace_retire.regs_write[i]) begin
205     if (rvfi_frd_wvalid[1] &&
206         (trace_retire.regs_write[i].addr == {1'b1, rvfi_frd_addr[1]})) begin
207       trace_retire.regs_write[i].value = rvfi_frd_wdata[1];
208     end else if (trace_retire.regs_write[i].addr ==
rvfi_rd_addr[1]) begin
209       trace_retire.regs_write[i].value = rvfi_rd_wdata[1];
210     end
211   end
212   foreach (trace_retire.regs_write[i]) begin
213     if (rvfi_frd_wvalid[0] &&
214         (trace_retire.regs_write[i].addr == {1'b1, rvfi_frd_addr[0]})) begin
215       trace_retire.regs_write[i].value = rvfi_frd_wdata[0];
216     end else if (trace_retire.regs_write[i].addr ==
rvfi_rd_addr[0]) begin
217       trace_retire.regs_write[i].value = rvfi_rd_wdata[0];
218     end
219   end
220 endfunction : apply_reg_write
221
222 function void apply_mem_access();
223   mem_acc_t mem_acc;
224
225   mem_acc.addr = rvfi_mem_addr;
226   if (rvfi_mem_wmask) begin
227     mem_acc.we = 1'b1;
228     mem_acc.wdata = rvfi_mem_wdata;
229     trace_retire.mem_access.push_back(mem_acc);
230   end else if (rvfi_mem_rmask) begin
231     mem_acc.we = 1'b0;
232     mem_acc.wdata = 'x;
233     trace_retire.mem_access.push_back(mem_acc);
234   end
235 endfunction : apply_mem_access
236
237 string      insn_disas;
238 logic [31:0] insn_pc;
239 logic [31:0] insn_val;
240
241 always @(posedge clk_i) begin
242   if (rvfi_valid) begin
243     trace_retire = trace_new_instr();
244     apply_reg_write();
245     apply_mem_access();
246     trace_retire.printInstrTrace();
247     insn_disas = trace_retire.str;
248     insn_pc = trace_retire.pc;
249     insn_val = trace_retire.instr;
250   end
251 end
252
253 initial begin
254   wait(rst_n == 1'b1);
255   $format(fn, "trace_core.log");
256   $format(info_tag, "CORE_TRACER %2d", hart_id_i);
257   $display("[%s] Output filename is: %s", info_tag, fn);
258   f = $fopen(fn, "w");
259   $fwrite(f,
260     "          Time          Cycle PC      In-
str   Ctx Decoded instruction Register and memory con-
261     tents          Stop cycle Stop time\n");
262 endmodule

```

5. openhwgroup_cv32e40p/bhv/cv32e40p_sim_clock_gate.sv (30 lines)

```

1 // Copyright 2017 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in
4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this license is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.

```

```

10
11 // !!! cv32e40p_sim_clock_gate file is meant for simulation
only !!!
12 // !!! It must not be used for ASIC syntheses-
is      !!!
13 // !!! It must not be used for FPGA syntheses-
is      !!!
14
15 module cv32e40p_clock_gate (
16     input  logic clk_i,
17     input  logic en_i,
18     input  logic scan_cg_en_i,
19     output logic clk_o
20 );
21
22     logic clk_en;
23
24     always_latch begin
25         if (clk_i == 1'b0) clk_en <= en_i | scan_cg_en_i;
26     end
27
28     assign clk_o = clk_i & clk_en;
29
30 endmodule // cv32e40p_clock_gate

```

6. openhwgroup_cv32e40p/bhv/cv32e40p_tb_wrapper.sv (549 lines)

```

1 // Copyright 2024 Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
"License");
5 // you may not use this file except in compliance with the Li-
cense, or,
6 // at your option, the Apache License version 2.0.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/SHL-2.1/
10 //
11 // Unless required by applicable law or agreed to in writing,
any work
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17 ///////////////////////////////////////////////////////////////////
18 //
19 // Contributors: Davide Schiavone, OpenHW Group <dav-
ide@openhwgroup.org>
20 // Yoann Pruvost, Dolphin Design
<yoann.pruvost@dolphin.fr>
21 //
22 // Description: Test-bench wrapper for cv32e40p_top, tracer
and and rvfi_tracer //
23 //
24 ///////////////////////////////////////////////////////////////////
25
26 `ifndef CV32E40P_ASSERT_ON
27 `include "cv32e40p_prefetch_controller_sva.sv"
28 `endif
29
30 `ifndef CV32E40P_CORE_LOG
31 `include "cv32e40p_core_log.sv"
32 `endif
33
34 `ifndef CV32E40P_APU_TRACE
35 `include "cv32e40p_apu_tracer.sv"
36 `endif
37
38 `ifndef CV32E40P_TRACE_EXECUTION
39 `include "cv32e40p_tracer.sv"
40 `endif
41
42 `ifndef CV32E40P_RVFI
43 `include "cv32e40p_rvfi.sv"
44 `endif
45
46 `ifndef CV32E40P_RVFI_TRACE_EXECUTION
47 `include "cv32e40p_rvfi_trace.sv"
48 `endif
49
50 module cv32e40p_tb_wrapper
51 import cv32e40p_pkg::*;
52 #(
53     parameter COREV_PULP = 0, // PULP ISA Extension (incl.
custom CSRs and hardware loop, excl. cv.elw)
54     parameter COREV_CLUSTER = 0, // PULP Cluster interface

```

```

(incl. cv.elw)
55     parameter FPU = 0, // Floating Point Unit (interfaced via
APU interface)
56     parameter FPU_ADDMUL_LAT = 0, // Floating-Point Addition/
MULTiplication computing lane pipeline registers number
57     parameter FPU_OTHERS_LAT = 0, // Floating-Point COMPari-
on/CONVersion computing lanes pipeline registers number
58     parameter ZFINX = 0, // Float-in-General Purpose re-
gisters
59     parameter NUM_MHPMCOUNTERS = 1
60 ) (
61     // Clock and Reset
62     input logic clk_i,
63     input logic rst_ni,
64
65     input logic pulp_clock_en_i, // PULP clock enable (only
used if COREV_CLUSTER = 1)
66     input logic scan_cg_en_i, // Enable all clock gates for
testing
67
68     // Core ID, Cluster ID, debug mode halt address and boot
address are considered more or less static
69     input logic [31:0] boot_addr_i,
70     input logic [31:0] mtvec_addr_i,
71     input logic [31:0] dm_halt_addr_i,
72     input logic [31:0] hart_id_i,
73     input logic [31:0] dm_exception_addr_i,
74
75     // Instruction memory interface
76     output logic instr_req_o,
77     input logic instr_gnt_i,
78     input logic instr_rvalid_i,
79     output logic [31:0] instr_addr_o,
80     input logic [31:0] instr_rdata_i,
81
82     // Data memory interface
83     output logic data_req_o,
84     input logic data_gnt_i,
85     input logic data_rvalid_i,
86     output logic data_we_o,
87     output logic [3:0] data_be_o,
88     output logic [31:0] data_addr_o,
89     output logic [31:0] data_wdata_o,
90     input logic [31:0] data_rdata_i,
91
92     // Interrupt inputs
93     input logic [31:0] irq_i, // CLINT interrupts + CLINT
extension
94     output logic irq_ack_o,
95     output logic [4:0] irq_id_o,
96
97     // Debug Interface
98     input logic debug_req_i,
99     output logic debug_havereset_o,
100    output logic debug_running_o,
101    output logic debug_halted_o,
102
103    // CPU Control Signals
104    input logic fetch_enable_i,
105    output logic core_sleep_o
106 );
107
108 `ifndef CV32E40P_ASSERT_ON
109
110 // RTL Assertions
111 bind cv32e40p_prefetch_controller:
112
113 cv32e40p_top_i.core_i.if_stage_i.prefetch_buffer_i.prefetch_control
ler_i
114     cv32e40p_prefetch_controller_sva
115     #(
116         .DEPTH (DEPTH),
117         .COREV_PULP (COREV_PULP),
118         .PULP_OBI (PULP_OBI),
119         .FIFO_ADDR_DEPTH(FIFO_ADDR_DEPTH)
120     ) prefetch_controller_sva (.*)
121 `endif // CV32E40P_ASSERT_ON
122
123 `ifndef CV32E40P_CORE_LOG
124 cv32e40p_core_log #(
125     .COREV_PULP (COREV_PULP),
126     .COREV_CLUSTER (COREV_CLUSTER),
127     .FPU (FPU),
128     .ZFINX (ZFINX),
129     .NUM_MHPMCOUNTERS(NUM_MHPMCOUNTERS)
130 ) core_log_i (
131     .clk_i
(cv32e40p_top_i.core_i.id_stage_i.clk),
132     .is_decoding_i
(cv32e40p_top_i.core_i.id_stage_i.is_decoding_o),
133     .illeg-
al_insn_dec_i(cv32e40p_top_i.core_i.id_stage_i.illegal_insn_dec),
134     .hart_id_i (cv32e40p_top_i.core_i.hart_id_i),
135     .pc_id_i (cv32e40p_top_i.core_i.pc_id)
136 );
137 `endif // CV32E40P_CORE_LOG
138
139 `ifndef CV32E40P_APU_TRACE
140 cv32e40p_apu_tracer apu_tracer_i (

```



```

141 .clk_i (cv32e40p_top_i.core_i.rst_ni),
142 .rst_n (cv32e40p_top_i.core_i.clk_i),
143 .hart_id_i (cv32e40p_top_i.core_i.hart_id_i),
144 .apu_valid_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_valid),
145 .apu_waddr_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_waddr),
146 .apu_res-
ult_i(cv32e40p_top_i.core_i.ex_stage_i.apu_result)
147 );
148 `endif
149
150 `ifdef CV32E40P_TRACE_EXECUTION
151 cv32e40p_tracer #(
152 .FPU (FPU),
153 .ZFINX(ZFINX)
154 ) tracer_i (
155 .clk_i(cv32e40p_top_i.core_i.clk_i), // always-running
clock for tracing
156 .rst_n(cv32e40p_top_i.core_i.rst_ni),
157
158 .hart_id_i(cv32e40p_top_i.core_i.hart_id_i),
159
160 .pc
(cv32e40p_top_i.core_i.id_stage_i.pc_id_i),
161 .instr
(cv32e40p_top_i.core_i.id_stage_i.instr),
162 .control-
ler_state_i(cv32e40p_top_i.core_i.id_stage_i.controller_i.ctrl_fsm_
cs),
163 .compressed
(cv32e40p_top_i.core_i.id_stage_i.is_compressed_i),
164 .id_valid
(cv32e40p_top_i.core_i.id_stage_i.id_valid_o),
165 .is_decoding
(cv32e40p_top_i.core_i.id_stage_i.is_decoding_o),
166 .is_illegal
(cv32e40p_top_i.core_i.id_stage_i.is_illegal_insn_dec),
167 .trigger_match
(cv32e40p_top_i.core_i.id_stage_i.trigger_match_i),
168 .rs1_value
(cv32e40p_top_i.core_i.id_stage_i.operand_a_fw_id),
169 .rs2_value
(cv32e40p_top_i.core_i.id_stage_i.operand_b_fw_id),
170 .rs3_value
(cv32e40p_top_i.core_i.id_stage_i.alu_operand_c),
171 .rs2_value_vec
(cv32e40p_top_i.core_i.id_stage_i.alu_operand_b),
172
173 .rs1_is_fp(cv32e40p_top_i.core_i.id_stage_i.regfile_fp_a
),
174 .rs2_is_fp(cv32e40p_top_i.core_i.id_stage_i.regfile_fp_b
),
175 .rs3_is_fp(cv32e40p_top_i.core_i.id_stage_i.regfile_fp_c
),
176 .rd_is_fp
(cv32e40p_top_i.core_i.id_stage_i.regfile_fp_d),
177
178 .ex_valid (cv32e40p_top_i.core_i.ex_valid),
179 .ex_reg_addr
(cv32e40p_top_i.core_i.regfile_alu_waddr_fw),
180 .ex_reg_we (cv32e40p_top_i.core_i.regfile_alu_we_fw),
181 .ex_reg_wdata(cv32e40p_top_i.core_i.regfile_alu_wdata_fw
),
182
183 .ex_data_addr (cv32e40p_top_i.core_i.data_addr_o),
184 .ex_data_req (cv32e40p_top_i.core_i.data_req_o),
185 .ex_data_gnt (cv32e40p_top_i.core_i.data_gnt_i),
186 .ex_data_we (cv32e40p_top_i.core_i.data_we_o),
187 .ex_data_wdata (cv32e40p_top_i.core_i.data_wdata_o),
188 .data_misaligned(cv32e40p_top_i.core_i.data_misaligned),
189
190 .ebrk_in-
sn(cv32e40p_top_i.core_i.id_stage_i.ebrk_insn_dec),
191 .debug_mode(cv32e40p_top_i.core_i.debug_mode),
192 .ebrk_force_debug_mode(cv32e40p_top_i.core_i.id_stage_i.
controller_i.ebrk_force_debug_mode),
193
194 .wb_by-
pass(cv32e40p_top_i.core_i.ex_stage_i.branch_in_ex_i),
195
196 .wb_valid (cv32e40p_top_i.core_i.wb_valid),
197 .wb_reg_addr
(cv32e40p_top_i.core_i.regfile_waddr_fw_wb_o),
198 .wb_reg_we (cv32e40p_top_i.core_i.regfile_we_wb),
199 .wb_reg_wdata(cv32e40p_top_i.core_i.regfile_wdata),
200
201 .imm_u_type
(cv32e40p_top_i.core_i.id_stage_i.imm_u_type),
202 .imm_uj_type
(cv32e40p_top_i.core_i.id_stage_i.imm_uj_type),
203 .imm_i_type
(cv32e40p_top_i.core_i.id_stage_i.imm_i_type),
204 .imm_iz_type
(cv32e40p_top_i.core_i.id_stage_i.imm_iz_type[11:0]),
205 .imm_z_type
(cv32e40p_top_i.core_i.id_stage_i.imm_z_type),
206 .imm_s_type
(cv32e40p_top_i.core_i.id_stage_i.imm_s_type),
207 .imm_sb_type
(cv32e40p_top_i.core_i.id_stage_i.imm_sb_type),

```

```

208 .imm_s2_type
(cv32e40p_top_i.core_i.id_stage_i.imm_s2_type),
209 .imm_s3_type
(cv32e40p_top_i.core_i.id_stage_i.imm_s3_type),
210 .imm_vs_type
(cv32e40p_top_i.core_i.id_stage_i.imm_vs_type),
211 .imm_vu_type
(cv32e40p_top_i.core_i.id_stage_i.imm_vu_type),
212 .imm_shuffle_type
(cv32e40p_top_i.core_i.id_stage_i.imm_shuffle_type),
213 .imm_clip_type
(cv32e40p_top_i.core_i.id_stage_i.instr[11:7]),
214 .apu_en_i (cv32e40p_top_i.apu_req),
215 .apu_single-
cycle_i(cv32e40p_top_i.core_i.ex_stage_i.apu_singlecycle),
216 .apu_multicycle_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_multicycle),
217 .apu_rvalid_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_valid)
218 );
219 `endif
220
221 `ifdef CV32E40P_RVFI
222 logic [1:0][31:0] hwlp_start_q;
223 logic [1:0][31:0] hwlp_end_q;
224 logic [1:0][31:0] hwlp_counter_q;
225 logic [1:0][31:0] hwlp_counter_n;
226 generate
227 if (COREV_PULP) begin
228 assign hwlp_start_q =
cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlp_
start_q ;
229 assign hwlp_end_q =
cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlp_
end_q;
230 assign hwlp_counter_q =
cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlp_
counter_q;
231 assign hwlp_counter_n =
cv32e40p_top_i.core_i.id_stage_i.gen_hwloop_regs.hwloop_regs_i.hwlp_
counter_n;
232 end else begin
233 assign hwlp_start_q = '0;
234 assign hwlp_end_q = '0;
235 assign hwlp_counter_q = '0;
236 assign hwlp_counter_n = '0;
237 end
238 endgenerate
239
240 cv32e40p_rvfi #(
241 .FPU(FPU),
242 .ZFINX(ZFINX),
243 .NUM_MHPMCOUNTERS(NUM_MHPMCOUNTERS)
244 ) rvfi_i (
245 .clk_i (cv32e40p_top_i.core_i.clk_i),
246 .rst_ni(cv32e40p_top_i.core_i.rst_ni),
247
248 .is_decoding_i
(cv32e40p_top_i.core_i.id_stage_i.is_decoding_o),
249 .is_illegal_i
(cv32e40p_top_i.core_i.id_stage_i.is_illegal_insn_dec),
250 .trigger_match_i
(cv32e40p_top_i.core_i.id_stage_i.trigger_match_i),
251 .data_mis-
aligned_i(cv32e40p_top_i.core_i.data_misaligned),
252 .lsu_data_we_ex_i (cv32e40p_top_i.core_i.data_we_ex),
253 .debug_mode_i (cv32e40p_top_i.core_i.debug_mode),
254 .debug_cause_i (cv32e40p_top_i.core_i.debug_cause),
255 /// IF probes ///
256 .instr_req_i (cv32e40p_top_i.core_i.instr_req_o),
257 .instr_grant_i (cv32e40p_top_i.core_i.instr_gnt_i),
258 .instr_rvalid_i
(cv32e40p_top_i.core_i.instr_rvalid_i),
259 .prefetch_req_i (cv32e40p_top_i.core_i.instr_req_int),
260 .pc_set_i (cv32e40p_top_i.core_i.pc_set),
261
262 .instr_valid_id_i
(cv32e40p_top_i.core_i.instr_valid_id),
263 .instr_rdata_id_i
(cv32e40p_top_i.core_i.instr_rdata_id),
264 .is_fetch_failed_id_i(cv32e40p_top_i.core_i.is_fetch_fa
iled_id),
265 .instr_req_int_i
(cv32e40p_top_i.core_i.instr_req_int),
266 .clear_instr_valid_i
(cv32e40p_top_i.core_i.clear_instr_valid),
267 /// IF probes ///
268 .instr_valid_if_i
(cv32e40p_top_i.core_i.if_stage_i.instr_valid),
269 .if_valid_i
(cv32e40p_top_i.core_i.if_stage_i.if_valid),
270 .if_ready_i
(cv32e40p_top_i.core_i.if_stage_i.if_ready),
271 .instr_if_i
(cv32e40p_top_i.core_i.if_stage_i.instr_aligned),
272 .pc_if_i (cv32e40p_top_i.core_i.pc_if),
273 /// ID probes ///
274 .pc_id_i
(cv32e40p_top_i.core_i.id_stage_i.pc_id_i),
275 .id_valid_i
(cv32e40p_top_i.core_i.id_stage_i.id_valid_o),

```

```

276 .id_ready_i
(cv32e40p_top_i.core_i.id_stage_i.id_ready_o),
277
278 .rs1_addr_id_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_addr_ra_id),
279 .rs2_addr_id_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_addr_rb_id),
280 .rs3_addr_id_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_addr_rc_id),
281 .operand_a_fw_id_i
(cv32e40p_top_i.core_i.id_stage_i.operand_a_fw_id),
282 .operand_b_fw_id_i
(cv32e40p_top_i.core_i.id_stage_i.operand_b_fw_id),
283 .operand_c_fw_id_i
(cv32e40p_top_i.core_i.id_stage_i.operand_c_fw_id),
284 // .instr
(cv32e40p_top_i.core_i.id_stage_i.instr ),
285 .is_com-
pressed_id_i(cv32e40p_top_i.core_i.id_stage_i.is_compressed_i),
286 .ebrk_insn_dec_i
(cv32e40p_top_i.core_i.id_stage_i.ebrk_insn_dec),
287 .ecall_insn_dec_i
(cv32e40p_top_i.core_i.id_stage_i.ecall_insn_dec),
288 .mret_insn_dec_i
(cv32e40p_top_i.core_i.id_stage_i.mret_insn_dec),
289 .mret_dec_i
(cv32e40p_top_i.core_i.id_stage_i.mret_dec),
290
291 .csr_cause_i (cv32e40p_top_i.core_i.csr_cause),
292 .debug_csr_save_i(cv32e40p_top_i.core_i.debug_csr_save),
293
294 // HWL00P regs
295 .hwlp_start_q_i (hwlp_start_q),
296 .hwlp_end_q_i (hwlp_end_q),
297 .hwlp_counter_q_i(hwlp_counter_q),
298 .hwlp_counter_n_i(hwlp_counter_n),
299
300 .minstret_i
(cv32e40p_top_i.core_i.id_stage_i.minstret),
301 //// EX probes ////
302 .ex_valid_i (cv32e40p_top_i.core_i.ex_valid),
303 .ex_ready_i (cv32e40p_top_i.core_i.ex_ready),
304 .ex_reg_addr_i
(cv32e40p_top_i.core_i.regfile_alu_waddr_fw),
305 .ex_reg_we_i
(cv32e40p_top_i.core_i.regfile_alu_we_fw),
306 .ex_reg_wdata_i
(cv32e40p_top_i.core_i.regfile_alu_wdata_fw),
307 .apu_en_ex_i (cv32e40p_top_i.core_i.apu_en_ex),
308 .apu_singlecycle_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_singlecycle),
309 .apu_multicycle_i
(cv32e40p_top_i.core_i.ex_stage_i.apu_multicycle),
310 .wb_content-
tion_lsu_i(cv32e40p_top_i.core_i.ex_stage_i.wb_contention_lsu),
311 .wb_contention_i
(cv32e40p_top_i.core_i.ex_stage_i.wb_contention),
312 .regfile_we_lsu_i
(cv32e40p_top_i.core_i.ex_stage_i.regfile_we_lsu),
313 // .rf_we_alu_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_we_fw_i),
314 // .rf_addr_alu_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_waddr_fw_i),
315 // .rf_wdata_alu_i
(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_wdata_fw_i),
316
317 .mult_ready_i
(cv32e40p_top_i.core_i.ex_stage_i.mult_ready),
318 .alu_ready_i
(cv32e40p_top_i.core_i.ex_stage_i.alu_ready),
319 //// WB probes ////
320 .wb_valid_i (cv32e40p_top_i.core_i.wb_valid),
321 .wb_ready_i
(cv32e40p_top_i.core_i.lsu_ready_wb),
322 //// LSU probes ////
323 .data_we_ex_i (cv32e40p_top_i.core_i.data_we_ex),
324 .data_atop_ex_i
(cv32e40p_top_i.core_i.data_atop_ex),
325 .data_type_ex_i
(cv32e40p_top_i.core_i.data_type_ex),
326 .alu_operand_c_ex_i
(cv32e40p_top_i.core_i.alu_operand_c_ex),
327 .data_reg_off-
set_ex_i(cv32e40p_top_i.core_i.data_reg_offset_ex),
328 .data_load_event_ex_i(cv32e40p_top_i.core_i.data_load_e-
vent_ex),
329 .data_sign_ext_ex_i
(cv32e40p_top_i.core_i.data_sign_ext_ex),
330 .lsu_rdata_i (cv32e40p_top_i.core_i.lsu_rdata),
331 .data_req_ex_i
(cv32e40p_top_i.core_i.data_req_ex),
332 .alu_operand_a_ex_i
(cv32e40p_top_i.core_i.alu_operand_a_ex),
333 .alu_operand_b_ex_i
(cv32e40p_top_i.core_i.alu_operand_b_ex),
334 .useincr_addr_ex_i
(cv32e40p_top_i.core_i.useincr_addr_ex),
335 .data_mis-
aligned_ex_i(cv32e40p_top_i.core_i.data_misaligned_ex),
336 .p_elw_start_i
(cv32e40p_top_i.core_i.p_elw_start),

```

```

337 .p_elw_finish_i
(cv32e40p_top_i.core_i.p_elw_finish),
338 .lsu_ready_ex_i
(cv32e40p_top_i.core_i.lsu_ready_ex),
339 .lsu_ready_wb_i
(cv32e40p_top_i.core_i.lsu_ready_wb),
340
341 .lsu_data_be_i(cv32e40p_top_i.core_i.load_store_unit_i.
data_be),
342
343 .data_req_pmp_i(cv32e40p_top_i.core_i.data_req_pmp),
344 .data_gnt_pmp_i(cv32e40p_top_i.core_i.data_gnt_pmp),
345 .data_rvalid_i(cv32e40p_top_i.core_i.data_rvalid_i),
346 .data_err_pmp_i(cv32e40p_top_i.core_i.data_err_pmp),
347 .data_addr_pmp_i(cv32e40p_top_i.core_i.data_addr_pmp),
348 .data_we_i(cv32e40p_top_i.core_i.data_we_o),
349 .data_atop_i(cv32e40p_top_i.core_i.data_atop_o),
350 .data_be_i(cv32e40p_top_i.core_i.data_be_o),
351 .data_wdata_i(cv32e40p_top_i.core_i.data_wdata_o),
352 .data_rdata_i(cv32e40p_top_i.core_i.data_rdata_i),
353 // Register writes
354 .rf_we_wb_i(cv32e40p_top_i.core_i.id_stage_i.regfile_we-
wb_i),
355 .rf_ad-
dr_wb_i(cv32e40p_top_i.core_i.id_stage_i.regfile_waddr_wb_i),
356 .rf_wdata_wb_i(cv32e40p_top_i.core_i.id_stage_i.regfile-
wdata_wb_i),
357 .reg-
file_alu_we_ex_i(cv32e40p_top_i.core_i.id_stage_i.regfile_alu_we_ex-
o),
358
359 // APU
360 .apu_req_i (cv32e40p_top_i.core_i.apu_req_o),
361 .apu_gnt_i (cv32e40p_top_i.core_i.apu_gnt_i),
362 .apu_rval-
id_i(cv32e40p_top_i.core_i.ex_stage_i.apu_valid),
363
364 // Controller FSM probes
365 .ctrl_fsm_cs_i(cv32e40p_top_i.core_i.id_stage_i.control-
ler_i.ctrl_fsm_cs),
366 .pc_mux_i
(cv32e40p_top_i.core_i.id_stage_i.controller_i.pc_mux_o),
367 .exc_pc_mux_i
(cv32e40p_top_i.core_i.id_stage_i.controller_i.exc_pc_mux_o),
368
369 //CSR
370 .csr_addr_i
(cv32e40p_top_i.core_i.cs_registers_i.csr_addr_i),
371 .csr_we_i
(cv32e40p_top_i.core_i.cs_registers_i.csr_we_int),
372 .csr_wdata_int_i(cv32e40p_top_i.core_i.cs_registers_i.c-
sr_wdata_int),
373
374 .csr_fregs_we_i(cv32e40p_top_i.core_i.cs_registers_i.fr-
egs_we_i),
375
376 .csr_mstatus_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mstatus_n),
377 .csr_mstatus_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mstatus_q),
378 .csr_m-
status_fs_n_i(cv32e40p_top_i.core_i.cs_registers_i.mstatus_fs_n),
379 .csr_m-
status_fs_q_i(cv32e40p_top_i.core_i.cs_registers_i.mstatus_fs_q),
380
381 .csr_misa_n_i(cv32e40p_top_i.core_i.cs_registers_i.MISA-
VALUE), // WARL
382 .csr_misa_q_i(cv32e40p_top_i.core_i.cs_registers_i.MISA-
VALUE),
383
384 .csr_tdata1_n_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_control_rdata),//
csr_wdata_int
),
385 .csr_tdata1_q_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_control_rdata),//
gen_trigger_regs.tmatch_control_exec_q
),
386 .csr_t-
data1_we_i(cv32e40p_top_i.core_i.cs_registers_i.gen_trigger_regs.tm-
atch_control_we),
387
388 .csr_tdata2_n_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_value_rdata),//
csr_wdata_int
),
389 .csr_tdata2_q_i
(cv32e40p_top_i.core_i.cs_registers_i.tmatch_value_rdata),//
gen_trigger_regs.tmatch_control_exec_q
),
390 .csr_t-
data2_we_i(cv32e40p_top_i.core_i.cs_registers_i.gen_trigger_regs.tm-
atch_value_we),
391
392 .csr_tinfo_n_i({16'h0,
cv32e40p_top_i.core_i.cs_registers_i.tinfo_types}),
393 .csr_tinfo_q_i({16'h0,
cv32e40p_top_i.core_i.cs_registers_i.tinfo_types}),
394
395 .csr_mie_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mie_n),
396 .csr_mie_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mie_q),
397 .csr_mie_we_i
(cv32e40p_top_i.core_i.cs_registers_i.csr_mie_we),

```

```

398     .csr_mtvec_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mtvec_n),
399     .csr_mtvec_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mtvec_q),
400     .csr_mtvec_mode_n_i(cv32e40p_top_i.core_i.cs_registers_i
.mtvec_mode_n),
401     .csr_mtvec_mode_q_i(cv32e40p_top_i.core_i.cs_registers_i
.mtvec_mode_q),
402
403     .csr_mcountinhibit_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mcountinhibit_q),
404     .csr_mcountinhibit_n_i
(cv32e40p_top_i.core_i.cs_registers_i.mcountinhibit_n),
405     .csr_mcountinhib-
it_we_i(cv32e40p_top_i.core_i.cs_registers_i.mcountinhibit_we),
406
407     .csr_mhp-
mevent_n_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmevent_n),
408     .csr_mhp-
mevent_q_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmevent_q),
409     .csr_mhp-
mevent_we_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmevent_we),
410     .csr_m-
scratch_q_i(cv32e40p_top_i.core_i.cs_registers_i.mscratch_q),
411     .csr_m-
scratch_n_i(cv32e40p_top_i.core_i.cs_registers_i.mscratch_n),
412     .csr_mepc_q_i(cv32e40p_top_i.core_i.cs_registers_i.mepc-
q),
413     .csr_mepc_n_i(cv32e40p_top_i.core_i.cs_registers_i.mepc-
n),
414     .csr_m-
cause_q_i(cv32e40p_top_i.core_i.cs_registers_i.mcause_q),
415     .csr_m-
cause_n_i(cv32e40p_top_i.core_i.cs_registers_i.mcause_n),
416     .csr_mip_n_i(cv32e40p_top_i.core_i.cs_registers_i.mip),
417     .csr_mip_q_i(cv32e40p_top_i.core_i.cs_registers_i.mip),
418     .csr_mip_we_i('0'), //
(cv32e40p_top_i.core_i.cs_registers_i.mip)
419
420
421     .csr_dc-
sr_q_i(cv32e40p_top_i.core_i.cs_registers_i.dcsr_q),
422     .csr_dc-
sr_n_i(cv32e40p_top_i.core_i.cs_registers_i.dcsr_n),
423
424     .csr_dpc_n_i(cv32e40p_top_i.core_i.cs_registers_i.depc_n
),
425     .csr_dpc_q_i(cv32e40p_top_i.core_i.cs_registers_i.depc_q
),
426     .csr_dpc_we_i('0'), //
cv32e40p_top_i.core_i.cs_registers_i.),
427     .csr_d-
scratch0_n_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch0_n),
428     .csr_d-
scratch0_q_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch0_q),
429     .csr_dscratch0_we_i('0'), //
cv32e40p_top_i.core_i.cs_registers_i.),
430
431     .csr_d-
scratch1_n_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch1_n),
432     .csr_d-
scratch1_q_i(cv32e40p_top_i.core_i.cs_registers_i.dscratch1_q),
433     .csr_dscratch1_we_i('0'), //
cv32e40p_top_i.core_i.cs_registers_i.),
434
435     .csr_mhpmcounter_q_i
(cv32e40p_top_i.core_i.cs_registers_i.mhpmcounter_q),
436     .csr_mhpm-
counter_write_lower_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmcoun
ter_write_lower),
437     .csr_mhpm-
counter_write_upper_i(cv32e40p_top_i.core_i.cs_registers_i.mhpmcoun
ter_write_upper),
438
439     .csr_mvendroid_i({
440         MVENDORID_BANK, MVENDORID_OFFSET
441     }), //TODO: get this from the design instead of the pkg
442     .csr_marchid_i(MARCHID), //TODO: get this from the
design instead of the pkg
443
444     .csr_fcsr_fflags_n_i
(cv32e40p_top_i.core_i.cs_registers_i.fflags_n),
445     .csr_fcsr_fflags_q_i
(cv32e40p_top_i.core_i.cs_registers_i.fflags_q),
446     .csr_fc-
sr_fflags_we_i(cv32e40p_top_i.core_i.cs_registers_i.fflags_we_i),
447     .csr_fcsr_frm_n_i
(cv32e40p_top_i.core_i.cs_registers_i.frm_n),
448     .csr_fcsr_frm_q_i
(cv32e40p_top_i.core_i.cs_registers_i.frm_q)
449 );
450 `endif
451
452 `ifdef CV32E40P_RVFI_TRACE_EXECUTION
453     bind cv32e40p_rvfi: rvfi_i cv32e40p_rvfi_trace #(
454         .FPU (FPU),
455         .ZFIDX (ZFIDX)
456     ) cv32e40p_tracer_i (
457         .clk_i(clk_i),
458         .rst_ni(rst_ni),
459         .hart_id_i(cv32e40p_top_i.core_i.hart_id_i),

```

```

460
461     .imm_s3_type(cv32e40p_top_i.core_i.id_stage_i.imm_s3_ty
pe),
462
463     .rvfi_valid(rvfi_valid),
464     .rvfi_insn(rvfi_insn),
465     .rvfi_start_cycle(rvfi_start_cycle),
466     .rvfi_start_time(rvfi_start_time),
467     .rvfi_stop_cycle(rvfi_stop_cycle),
468     .rvfi_stop_time(rvfi_stop_time),
469     .rvfi_pc_rdata(rvfi_pc_rdata),
470     .rvfi_trap(rvfi_trap),
471     .rvfi_rd_addr(rvfi_rd_addr),
472     .rvfi_rd_wdata(rvfi_rd_wdata),
473     .rvfi_frd_wvalid(rvfi_frd_wvalid),
474     .rvfi_frd_addr(rvfi_frd_addr),
475     .rvfi_frd_wdata(rvfi_frd_wdata),
476     .rvfi_2_rd(rvfi_2_rd),
477     .rvfi_rs1_addr(rvfi_rs1_addr),
478     .rvfi_rs2_addr(rvfi_rs2_addr),
479     .rvfi_rs3_addr(rvfi_rs3_addr),
480     .rvfi_rs1_rdata(rvfi_rs1_rdata),
481     .rvfi_rs2_rdata(rvfi_rs2_rdata),
482     .rvfi_rs3_rdata(rvfi_rs3_rdata),
483     .rvfi_frs1_addr(rvfi_frs1_addr),
484     .rvfi_frs2_addr(rvfi_frs2_addr),
485     .rvfi_frs3_addr(rvfi_frs3_addr),
486     .rvfi_frs1_rvalid(rvfi_frs1_rvalid),
487     .rvfi_frs2_rvalid(rvfi_frs2_rvalid),
488     .rvfi_frs3_rvalid(rvfi_frs3_rvalid),
489     .rvfi_frs1_rdata(rvfi_frs1_rdata),
490     .rvfi_frs2_rdata(rvfi_frs2_rdata),
491     .rvfi_frs3_rdata(rvfi_frs3_rdata),
492     .rvfi_mem_addr(rvfi_mem_addr),
493     .rvfi_mem_rmask(rvfi_mem_rmask),
494     .rvfi_mem_wmask(rvfi_mem_wmask),
495     .rvfi_mem_rdata(rvfi_mem_rdata),
496     .rvfi_mem_wdata(rvfi_mem_wdata)
497 );
498 `endif
499 // Instantiate the Core and the optional FPU
500 cv32e40p_top #(
501     .COREV_PULP      (COREV_PULP),
502     .COREV_CLUSTER  (COREV_CLUSTER),
503     .FPU             (FPU),
504     .FPU_ADDMUL_LAT  (FPU_ADDMUL_LAT),
505     .FPU_OTHERS_LAT  (FPU_OTHERS_LAT),
506     .ZFIDX           (ZFIDX),
507     .NUM_MHPMCOUNTERS(NUM_MHPMCOUNTERS)
508 ) cv32e40p_top_i (
509     .clk_i (clk_i),
510     .rst_ni(rst_ni),
511
512     .pulp_clock_en_i(pulp_clock_en_i),
513     .scan_cg_en_i   (scan_cg_en_i),
514
515     .boot_addr_i     (boot_addr_i),
516     .mtvec_addr_i    (mtvec_addr_i),
517     .dm_halt_addr_i   (dm_halt_addr_i),
518     .hart_id_i       (hart_id_i),
519     .dm_exception_addr_i(dm_exception_addr_i),
520
521     .instr_req_o      (instr_req_o),
522     .instr_gnt_i      (instr_gnt_i),
523     .instr_rvalid_i   (instr_rvalid_i),
524     .instr_addr_o     (instr_addr_o),
525     .instr_rdata_i    (instr_rdata_i),
526
527     .data_req_o       (data_req_o),
528     .data_gnt_i       (data_gnt_i),
529     .data_rvalid_i    (data_rvalid_i),
530     .data_we_o        (data_we_o),
531     .data_be_o        (data_be_o),
532     .data_addr_o      (data_addr_o),
533     .data_wdata_o     (data_wdata_o),
534     .data_rdata_i     (data_rdata_i),
535
536     .irq_i            (irq_i),
537     .irq_ack_o(irq_ack_o),
538     .irq_id_o         (irq_id_o),
539
540     .debug_req_i      (debug_req_i),
541     .debug_havereset_o(debug_havereset_o),
542     .debug_running_o  (debug_running_o),
543     .debug_halted_o   (debug_halted_o),
544
545     .fetch_enable_i   (fetch_enable_i),
546     .core_sleep_o     (core_sleep_o)
547 );
548
549 endmodule

```

7. openhwgroup_cv32e40p/bhvh/cv32e40p_tracer.sv (563 lines)

```

1 // Copyright 2018 ETH Zurich and University of Bologna.
2 // Copyright and related rights are licensed under the Solder-
pad Hardware
3 // License, Version 0.51 (the "License"); you may not use this
file except in

```

```

4 // compliance with the License. You may obtain a copy of the
License at
5 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
6 // or agreed to in writing, software, hardware and materials
distributed under
7 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
8 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
9 // specific language governing permissions and limitations un-
der the License.
10
11 ///////////////////////////////////////////////////////////////////
12 // Engineer:      Andreas Traber - at-
raber@iis.ee.ethz.ch      //
13 //
14 // Additional contributions
by:
15 //      Davide Schiavone - ps-
chiavo@iis.ee.ethz.ch      //
16 //
17 // Design Name:    RISC-V
Tracer
18 // Project Name:
RISCV
19 // Language:      SystemVeri-
log
20 //
21 // Description:    Traces the executed instruc-
tions
22 //
23 ///////////////////////////////////////////////////////////////////
24
25 `ifndef CV32E40P_TRACE_EXECUTION
26
27 `include "uvm_macros.svh"
28
29 module cv32e40p_tracer
30 import cv32e40p_pkg::*;
31 import uvm_pkg::*;
32 #(
33     parameter FPU    = 0,
34     parameter ZFINX  = 0
35 ) (
36     // Clock and Reset
37     input logic clk_i,
38     input logic rst_n,
39
40     input logic [31:0] hart_id_i,
41
42
43     input logic [31:0] pc,
44     input logic [31:0] instr,
45     input ctrl_state_e controller_state_i,
46
47     input logic compressed,
48     input logic id_valid,
49     input logic is_decoding,
50     input logic is_illegal,
51     input logic trigger_match,
52
53     input logic [31:0] rs1_value,
54     input logic [31:0] rs2_value,
55     input logic [31:0] rs3_value,
56
57     input logic [31:0] rs2_value_vec,
58
59     input logic rd_is_fp,
60     input logic rs1_is_fp,
61     input logic rs2_is_fp,
62     input logic rs3_is_fp,
63
64     input logic      ex_valid,
65     input logic [ 5:0] ex_reg_addr,
66     input logic      ex_reg_we,
67     input logic [31:0] ex_reg_wdata,
68
69     input logic      ex_data_req,
70     input logic      ex_data_gnt,
71     input logic      ex_data_we,
72     input logic [31:0] ex_data_addr,
73     input logic [31:0] ex_data_wdata,
74     input logic      data_misaligned,
75
76     input logic ebrk_insn,
77     input logic debug_mode,
78     input logic ebrk_force_debug_mode,
79
80     input logic wb_bypass,
81

```

```

82     input logic      wb_valid,
83     input logic [ 5:0] wb_reg_addr,
84     input logic      wb_reg_we,
85     input logic [31:0] wb_reg_wdata,
86
87     input logic [31:0] imm_u_type,
88     input logic [31:0] imm_uj_type,
89     input logic [31:0] imm_i_type,
90     input logic [11:0] imm_iz_type,
91     input logic [31:0] imm_z_type,
92     input logic [31:0] imm_s_type,
93     input logic [31:0] imm_sb_type,
94     input logic [31:0] imm_s2_type,
95     input logic [31:0] imm_s3_type,
96     input logic [31:0] imm_vs_type,
97     input logic [31:0] imm_vu_type,
98     input logic [31:0] imm_shuffle_type,
99     input logic [ 4:0] imm_clip_type,
100
101     input logic apu_en_i,
102     input logic apu_singlecycle_i,
103     input logic apu_multicycle_i,
104     input logic apu_rvalid_i,
105
106 );
107
108 import cv32e40p_tracer_pkg::*;
109
110 // Make clock a bit to avoid x->0 transitions in tracer lo-
gic
111 bit clk_i_d, eval_comb;
112 assign #0.01 clk_i_d = clk_i;
113 assign eval_comb = ~(clk_i & ~clk_i_d);
114
115 //event      ovp_retire;
116 //bit        use_iss;
117
118 integer f;
119 string fn;
120 integer cycles;
121 logic [5:0] rd, rs1, rs2, rs3, rs4;
122
123 logic [31:0] pc_ex_stage;
124 logic [31:0] pc_ex_delay_stage;
125 logic [31:0] pc_wb_stage;
126 logic [31:0] pc_wb_delay_stage;
127 logic [31:0] pc_retire_head_q;
128
129 `include "cv32e40p_instr_trace.svh"
130
131 string      info_tag;
132
133 event        retire;
134
135 instr_trace_t      trace_ex;
136 instr_trace_t      trace_wb;
137 instr_trace_t      trace_wb_delay;
138 instr_trace_t      trace_ex_delay;
139
140 bit      clear_trace_ex = 0;
141 bit      move_trace_ex_to_trace_wb = 0;
142 bit      clear_trace_wb = 0;
143 bit      move_trace_wb_to_trace_wb_delay = 0;
144 bit      clear_trace_wb_delay = 0;
145 bit      move_trace_ex_to_trace_ex_delay = 0;
146 bit      clear_trace_ex_delay = 0;
147 bit      move_trace_ex_delay_to_trace_wb = 0;
148
149 bit      trace_new = 0;
150 bit      trace_new_ebreak = 0;
151 bit      trace_ex_misaligned = 0;
152 bit      trace_ex_retire = 0;
153 bit      trace_ex_wb_bypass = 0;
154 bit      trace_wb_retire = 0;
155 bit      trace_wb_delay_retire = 0;
156
157 bit      trace_ex_is_null = 1;
158 bit      trace_ex_delay_is_null = 1;
159 bit      trace_wb_is_null = 1;
160 bit      trace_wb_delay_is_null = 1;
161
162 bit      trace_ex_is_delay_instr = 0;
163 bit      trace_wb_is_delay_instr = 0;
164
165 string      insn_disas;
166 logic        insn_compressed;
167 logic        insn_wb_bypass;
168 logic        insn_ebreak;
169 logic [31:0] insn_pc;
170 logic [31:0] insn_val;
171 reg_t        insn_regs_write
172
173 instr_trace_t      trace_q
174
175 instr_trace_t      trace_retire;
176
177 // cycle counter
178 always_ff @(posedge clk_i, negedge rst_n) begin
179     if (rst_n == 1'b0) cycles <= 0;
180     else cycles <= cycles + 1;

```

```

180 end
181
182 initial begin
183     wait(rst_n == 1'b1);
184     $sformat(fn, "trace_core.%h.log", hart_id_i);
185     $sformat(info_tag, "CORE_TRACER %2d", hart_id_i);
186     $display("[%s] Output filename is: %s", info_tag, fn);
187     f = $fopen(fn, "w");
188     $fwrite(f,
189         "
190         Time          Cycle PC      In-
str Ctx Decoded instruction Register and memory contents\n");
191 end
192 //initial begin
193 // use_iss = 0;
194 // if ($test$plusargs("USE_ISS")) use_iss = 1;
195 //end
196
197 always @(trace_ex or trace_ex_delay or trace_wb or
trace_wb_delay or trace_retire) begin
198     pc_ex_stage = (trace_ex != null) ? trace_ex.pc : 'x;
199     pc_ex_delay_stage = (trace_ex_delay != null) ?
trace_ex_delay.pc : 'x;
200     pc_wb_stage = (trace_wb != null) ? trace_wb.pc : 'x;
201     pc_wb_delay_stage = (trace_wb_delay != null) ?
trace_wb_delay.pc : 'x;
202     pc_retire_head_q = (trace_retire != null) ?
trace_retire.pc : 'x;
203 end
204
205 always @(trace_wb)
206     trace_wb_is_delay_instr = (trace_wb != null &&
is_wb_delay_instr(
207         trace_wb
208     )) ? 1 : 0;
209
210 assign rd = {rd_is_fp, instr[11:07]};
211 assign rs1 = {rs1_is_fp, instr[19:15]};
212 assign rs2 = {rs2_is_fp, instr[24:20]};
213 assign rs3 = {rs3_is_fp, instr[29:25]};
214 assign rs4 = {rs3_is_fp, instr[31:27]};
215
216 function void apply_reg_write(instr_trace_t trace, int un-
signed reg_addr, int unsigned wdata);
217     foreach (trace.regs_write[i])
218         if (trace.regs_write[i].addr == reg_addr) begin
219             trace.regs_write[i].value = wdata;
220             `uvm_info(info_tag, $sformatf(
221                 "Write mapped %0d, %0d:0x%08x pc:0x%08x", i,
reg_addr, wdata, trace.pc),
222                 UVM_DEBUG)
223         end else begin
224             `uvm_info(info_tag, $sformatf(
225                 "Unmapped write to %0d:0x%08x, expected
write to %0d",
226                 reg_addr,
227                 wdata,
228                 trace.regs_write[i].addr
229             ), UVM_DEBUG)
230         end
231     endfunction : apply_reg_write
232
233 function void apply_mem_access(instr_trace_t trace, bit we,
int unsigned addr,
234     int unsigned wdata);
235     mem_acc_t mem_acc;
236     mem_acc.addr = addr;
237     mem_acc.we = we;
238     if (we) mem_acc.wdata = wdata;
239     else mem_acc.wdata = 'x;
240     trace.mem_access.push_back(mem_acc);
241 endfunction : apply_mem_access
242
243 function instr_trace_t trace_new_instr();
244     instr_trace_t trace;
245     trace = new();
246     trace.init(.cycles(cycles), .pc(pc), .compressed(compressed), .instr
(instr));
247     return trace;
248 endfunction : trace_new_instr
249
250 // Funnel all handoffs to the ISS here, note that this must
be automatic
251 // as multiple retire events may occur at a time (wb_bypass)
252 always begin
253     wait(trace_q.size() != 0);
254     trace_retire = trace_q.pop_front();
255     wait(trace_retire.retire != 0);
256
257     if (trace_retire.ebreak) wait(debug_mode == 1);
258
259     // Write signals and data structures used by step-and-com-
pare
260     insn_regs_write = trace_retire.regs_write;

```

```

266     insn_disas      = trace_retire.str;
267     insn_compressed = trace_retire.compressed;
268     insn_pc         = trace_retire.pc;
269     insn_val        = trace_retire.instr;
270     insn_wb_bypass  = trace_retire.wb_bypass;
271
272     trace_retire.printInstrTrace();
273
274     ->retire;
275     // if (use_iss) @(ovp_retire);
276     #0.1ns;
277 end
278
279 // EX stage
280 always @(negedge clk_i_d or negedge rst_n) begin
281     if (!rst_n) begin
282         trace_ex <= null;
283         trace_ex_is_null <= 1;
284     end else begin
285         if (trace_ex_retire) trace_ex.retire = 1;
286         if (trace_ex_wb_bypass) trace_ex.wb_bypass = 1;
287         if (trace_ex_misaligned) trace_ex.misaligned = 1;
288
289         if (trace_new_ebreak) begin
290             instr_trace_t new_instr;
291             new_instr = trace_new_instr();
292
293             // The EBREAK bypasses the pipeline so it retirable
immediately upon debug_mode entry
294             new_instr.ebreak = 1;
295             new_instr.retire = 1;
296             trace_q.push_back(new_instr);
297         end
298
299         if (trace_new) begin
300             instr_trace_t new_instr;
301             new_instr = trace_new_instr();
302             trace_q.push_back(new_instr);
303             trace_ex <= new_instr;
304             trace_ex_is_null <= 0;
305         end else if (clear_trace_ex) begin
306             trace_ex <= null;
307             trace_ex_is_null <= 1;
308         end
309     end
310 end
311
312 // EX delay stage
313 always @(negedge clk_i_d or negedge rst_n) begin
314     if (!rst_n) begin
315         trace_ex_delay <= null;
316         trace_ex_delay_is_null <= 1;
317     end else begin
318         if (move_trace_ex_to_trace_ex_delay) begin
319             trace_ex_delay <= trace_ex;
320             trace_ex_delay_is_null <= (trace_ex == null) ? 1 : 0;
321         end else if (clear_trace_ex_delay) begin
322             trace_ex_delay <= null;
323             trace_ex_delay_is_null <= 1;
324         end
325     end
326 end
327
328 // WB stage
329 always @(negedge clk_i_d or negedge rst_n) begin
330     if (rst_n) begin
331         if (trace_wb_retire) begin
332             if (trace_wb != null) begin
333                 trace_wb.retire = 1;
334             end else begin
335                 `uvm_fatal(info_tag, "Received retire signal with no
valid trace_wb instruction");
336             end
337         end
338     end
339 end
340
341 always @(negedge clk_i_d or negedge rst_n) begin
342     if (!rst_n) begin
343         trace_wb <= null;
344         trace_wb_is_null <= 1;
345     end else begin
346         if (move_trace_ex_to_trace_wb) begin
347             trace_wb <= trace_ex;
348             trace_wb_is_null <= (trace_ex == null) ? 1 : 0;
349         end else if (move_trace_ex_delay_to_trace_wb) begin
350             trace_wb <= trace_ex_delay;
351             trace_wb_is_null <= (trace_ex_delay == null) ? 1 : 0;
352         end else if (clear_trace_wb) begin
353             trace_wb <= null;
354             trace_wb_is_null <= 1;
355         end
356     end
357 end
358
359 // WB delay stage
360 always @(negedge clk_i_d or negedge rst_n) begin
361     if (!rst_n) begin
362         trace_wb_delay <= null;
363         trace_wb_delay_is_null <= 1;

```



```

364     end else begin
365         if (trace_wb_delay_retire) trace_wb_delay.retire = 1;
366     end
367     if (move_trace_wb_to_trace_wb_delay) begin
368         trace_wb_delay <= trace_wb;
369         trace_wb_delay_is_null <= (trace_wb == null) ? 1 : 0;
370     end else if (clear_trace_wb_delay) begin
371         trace_wb_delay <= null;
372         trace_wb_delay_is_null <= 1;
373     end
374 end
375 end
376
377 function bit is_wb_delay_instr(instr_trace_t trace_wb);
378 if (trace_wb.str == "mret" || trace_wb.str == "uret" ||
trace_wb.str == "ebreak" || trace_wb.str == "c.ebreak")
379     return 1;
380
381     return 0;
382 endfunction : is_wb_delay_instr
383
384 always @* begin
385     trace_new = 0;
386     trace_new_ebreak = 0;
387     trace_ex_misaligned = 0;
388     trace_ex_retire = 0;
389     trace_ex_wb_bypass = 0;
390     trace_wb_retire = 0;
391     trace_wb_delay_retire = 0;
392
393     clear_trace_ex = 0;
394
395     move_trace_ex_to_trace_wb = 0;
396     clear_trace_wb = 0;
397
398     move_trace_wb_to_trace_wb_delay = 0;
399     clear_trace_wb_delay = 0;
400
401     move_trace_ex_to_trace_ex_delay = 0;
402     move_trace_ex_delay_to_trace_wb = 0;
403     clear_trace_ex_delay = 0;
404
405     // -----
406     // WB Delay logic
407     // -----
408     if (!trace_wb_delay_is_null && eval_comb) begin
409         // Always retire
410         trace_wb_delay_retire = 1;
411         clear_trace_wb_delay = 1;
412     end
413
414     // -----
415     // WB logic
416     // -----
417     if (!trace_wb_is_null && eval_comb) begin
418         if (wb_valid) begin
419             // Some instructions get an extra cycle to retire
420             if (trace_wb_is_delay_instr) begin
421                 move_trace_wb_to_trace_wb_delay = 1;
422             end else begin
423                 trace_wb_retire = 1;
424                 clear_trace_wb = 1;
425             end
426         end
427     end
428
429     // -----
430     // EX Delay logic
431     // -----
432     // apu_rvalid_i for variable latency apu offloading
433     // non apu instructions will always go to wb as fast as
possible (i.e. next cycle)
434     if (!trace_ex_delay_is_null && eval_comb) begin
435         if (apu_rvalid_i || !trace_ex_delay.is_apu) begin
436             move_trace_ex_delay_to_trace_wb = 1;
437             clear_trace_ex_delay = 1;
438         end
439     end
440
441     // -----
442     // EX logic
443     // -----
444
445     // New instruction created if is a legal decoded instruc-
tion
446     if (id_valid && is_decoding && !is_illegal) begin
447         trace_new = 1;
448         // Create a new EBREAK instruction (will bypass
pipeline execution)
449     end else if (is_decoding && !trigger_match && ebrk_insn &&
(ebrk_force_debug_mode || debug_mode)) begin
450         trace_new_ebreak = 1;
451     end
452
453     // Instruction remains in the pipeline for one more cycle
if misaligned
454     if (!trace_ex_is_null && eval_comb && ex_valid &&
data_misaligned) begin
455         trace_ex_misaligned = 1;
456         // Instruction bypasses WB - mark as retireable and do

```

```

not advance
457     end else if (wb_bypass) begin
458         trace_ex_retire = 1;
459         trace_ex_wb_bypass = 1;
460         clear_trace_ex = 1;
461
462         // Some instructions get an extra cycle to retire
463         // offload to (potentially) multicyle APU/FPU
464     end else if (!trace_ex_is_null && eval_comb && apu_en_i
&& !apu_rvalid_i) begin
465         move_trace_ex_to_trace_ex_delay = 1;
466         clear_trace_ex = 1;
467
468         // Instruction leaves EX
469     end else if (!trace_ex_is_null && eval_comb && ex_valid
&& !data_misaligned) begin
470         // If ex delay is already writing to wb then we also
delay this one
471         if (move_trace_ex_delay_to_trace_wb) begin
472             move_trace_ex_to_trace_ex_delay = 1;
473             clear_trace_ex = 1;
474         end else begin
475             move_trace_ex_to_trace_wb = 1;
476             clear_trace_ex = 1;
477         end
478     end
479
480     if (move_trace_ex_to_trace_wb &&
move_trace_ex_delay_to_trace_wb) begin
481         `uvm_info(info_tag, "ex delay stage and ex stage col-
lide", UVM_DEBUG);
482     end
483 end
484
485     // Monitors for memory access and register writeback
486     always @(negedge clk_i or negedge rst_n) begin
487         if (!rst_n) begin
488             end else begin
489                 // Register updates in EX
490                 // !wb_valid necessary for CSR to GPR write when OBI
Data stalled
491                 if (ex_reg_we && (ex_valid || !wb_valid || apu_rval-
id_i)) begin
492                     `uvm_info(info_tag, $sformatf("EX: Reg WR %02d =
0x%08x", ex_reg_addr, ex_reg_wdata),
493                         UVM_DEBUG);
494                     if (!trace_ex_delay_is_null && !
trace_ex_delay.got_regs_write && !trace_ex_delay.is_load &&
495                         (!trace_ex_delay.is_apu && (ex_valid || !wb_val-
id_i))) ||
496                         (trace_ex_delay.is_apu && apu_rvalid_i &&
(apu_singlecycle_i || apu_multicycle_i))
497                     ) begin
498                         apply_reg_write(trace_ex_delay, ex_reg_addr,
ex_reg_wdata);
499                         trace_ex_delay.got_regs_write = 1;
500                     end else if (!trace_ex_is_null) begin
501                         apply_reg_write(trace_ex, ex_reg_addr,
ex_reg_wdata);
502                     if (trace_ex.got_regs_write) begin
503                         `uvm_info(info_tag, $sformatf(
504                             "EX: Multiple Reg WR %02d = 0x%08x",
ex_reg_addr, ex_reg_wdata), UVM_DEBUG);
505                         end
506                         trace_ex.got_regs_write = 1;
507                     end else begin
508                         `uvm_info(info_tag, $sformatf(
509                             "EX: Reg WR %02d:0x%08x but no active EX
instruction", ex_reg_addr, ex_reg_wdata
510                             ), UVM_DEBUG);
511                     end
512                 end
513             end
514
515             // Register updates in WB
516             if (wb_reg_we) begin
517                 `uvm_info(info_tag, $sformatf("WB: Reg WR %02d =
0x%08x", wb_reg_addr, wb_reg_wdata),
518                     UVM_DEBUG);
519                 if (!trace_ex_delay_is_null &&
((trace_ex_delay.is_load) ||
520                     (trace_ex_delay.is_apu && apu_rvalid_i && !
apu_singlecycle_i && !apu_multicycle_i))
521                 ) begin
522                     ) begin
523                         apply_reg_write(trace_ex_delay, wb_reg_addr,
wb_reg_wdata);
524                         trace_ex_delay.got_regs_write = 1;
525                     end else if (!trace_wb_is_null && !
trace_wb.got_regs_write) begin
526                         if (!trace_wb.is_load || (trace_wb.is_load &&
wb_valid)) begin
527                             apply_reg_write(trace_wb, wb_reg_addr,
wb_reg_wdata);
528                             trace_wb.got_regs_write = 1;
529                         end
530                     end else if (!trace_ex_is_null && !
trace_ex.got_regs_write && trace_ex.misaligned) begin
531                         // Do nothing as double load concatenation will be
managed by trace_wb
532                     end else begin
533

```

```

534         `uvm_info(info_tag, $sformatf(
535             "WB: Reg WR %02d:0x%08x but no active WB
instruction", wb_reg_addr, wb_reg_wdata
536             ), UVM_DEBUG);
537     end
538 end
539
540 // Memory access in EX
541 if (ex_data_req && ex_data_gnt) begin
542     if (ex_data_we) begin
543         `uvm_info(info_tag, $sformatf("EX: Mem WR 0x%08x =
0x%08x", ex_data_addr, ex_data_wdata),
544             UVM_DEBUG);
545     end else begin
546         `uvm_info(info_tag, $sformatf("EX: Mem RD 0x%08x",
ex_data_addr, UVM_DEBUG);
547     end
548     if (trace_ex_is_null) begin
549         `uvm_info(info_tag, $sformatf(
550             "EX: Mem %s 0x%08x:0x%08x but no active EX
instruction",
551             ex_data_we ? "WR" : "RD",
552             ex_data_addr,
553             ex_reg_wdata
554             ), UVM_DEBUG);
555     end else apply_mem_access(trace_ex, ex_data_we,
ex_data_addr, ex_data_wdata);
556 end
557 end
558 end
559
560 endmodule : cv32e40p_tracer
561
562
563 `endif // CV32E40P_TRACE_EXECUTION

```

8. openhwgroup_cv32e40p/bhv/insn_trace.sv (1,132 lines)

```

1 // Copyright 2024 OpenHW Group and Dolphin Design
2 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.1
3 //
4 // Licensed under the Solderpad Hardware License v 2.1 (the
"License");
5 // you may not use this file except in compliance with the Li-
cense, or,
6 // at your option, the Apache License version 2.0.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/SHL-2.1/
10 //
11 // Unless required by applicable law or agreed to in writing,
any work
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17 ///////////////////////////////////////////////////////////////////
18 //
19 // Contributors: Yoann Pruvost, Dolphin Design
<yoann.pruvost@dolphin.fr> //
20 //
21 // Description: Macros and Functions to print information on
RVFI interface //
22 //
23 ///////////////////////////////////////////////////////////////////
24
25 `define DEFINE_CSR(CSR_NAME) \
26     logic `CSR_NAME`_we; \
27     logic [31:0] `CSR_NAME`_rdata; \
28     logic [31:0] `CSR_NAME`_rmask; \
29     logic [31:0] `CSR_NAME`_wdata; \
30     logic [31:0] `CSR_NAME`_wmask;
31
32 `define ASSIGN_CSR(CSR_NAME) \
33     this.m_csr.`CSR_NAME`_we =
m_source.m_csr.`CSR_NAME`_we; \
34     this.m_csr.`CSR_NAME`_rdata =
m_source.m_csr.`CSR_NAME`_rdata; \
35     this.m_csr.`CSR_NAME`_rmask =
m_source.m_csr.`CSR_NAME`_rmask; \
36     this.m_csr.`CSR_NAME`_wdata =
m_source.m_csr.`CSR_NAME`_wdata; \
37     this.m_csr.`CSR_NAME`_wmask =
m_source.m_csr.`CSR_NAME`_wmask;
38
39 `define INIT_CSR(CSR_NAME) \
40     this.m_csr.`CSR_NAME`_we = '0; \

```

```

41     this.m_csr.`CSR_NAME`_wmask = '0;
42
43 import cv32e40p_tracer_pkg::*;
44 class insn_trace_t;
45     bit m_valid;
46     logic [63:0] m_order;
47     integer m_start_cycle;
48     integer m_stop_cycle;
49     time m_start_time;
50     time m_stop_time;
51     bit m_skip_order; //next order was used by trap;
52     logic [31:0] m_pc_rdata;
53     logic [31:0] m_insn;
54     string m_mnemonic;
55     logic m_is_ebreak;
56     logic m_is_illegal;
57     logic m_is_irq;
58     logic m_is_memory;
59     logic m_is_load;
60     logic m_is_apu;
61     logic m_is_apu_ok;
62     integer m_apu_req_id;
63     integer m_mem_req_id[1:0];
64     logic [1:0] m_mem_req_id_valid;
65     logic m_data_missaligned;
66     logic m_got_first_data;
67     logic m_got_ex_reg;
68     logic m_dbg_taken;
69     logic [2:0] m_dbg_cause;
70
71     logic m_fflags_we_non_apu;
72     logic m_frm_we_non_apu;
73     logic m_fcsr_we_non_apu;
74     logic [5:0] m_rs1_addr;
75     logic [5:0] m_rs2_addr;
76     logic [5:0] m_rs3_addr;
77     logic [31:0] m_rs1_rdata;
78     logic [31:0] m_rs2_rdata;
79     logic [31:0] m_rs3_rdata;
80
81     bit m_trap;
82
83     bit m_got_regs_write;
84     bit m_ex_fw;
85     logic [5:0] m_rd_addr [1:0];
86     logic [31:0] m_rd_wdata[1:0];
87     logic m_2_rd_insn; //this instruction uses 2 des-
tination registers
88     rvfi_intr_t m_intr;
89
90     bit m_move_down_pipe;
91
92     int m_instret_cnt;
93
94     int m_instret_smample_trigger; //We need to sample minstret
from csr 2 cycle after id is doen
95
96     bit m_sample_csr_write_in_ex;
97
98     struct {
99         logic [31:0] addr ;
100         logic [31:0] rmask;
101         logic [31:0] rdata;
102         logic [31:0] wmask;
103         logic [31:0] wdata;
104     } m_mem;
105
106     struct {
107         logic mstatus_we;
108         logic [31:0] mstatus_rmask;
109         Status_t mstatus_wdata;
110         logic [31:0] mstatus_wmask;
111         Status_t mstatus_rdata;
112
113         logic mstatus_fs_we;
114         FS_t mstatus_fs_rdata;
115         logic [31:0] mstatus_fs_rmask;
116         FS_t mstatus_fs_wdata;
117         logic [31:0] mstatus_fs_wmask;
118
119         // mstatush
120         `DEFINE_CSR(misa)
121         `DEFINE_CSR(mie)
122         `DEFINE_CSR(mtvec)
123         //mtvec_mode
124         //mtvt
125         `DEFINE_CSR(mcountinhibit)
126         //mhpmevent
127
128         `DEFINE_CSR(mscratch)
129         `DEFINE_CSR(mepc)
130         `DEFINE_CSR(mcause)
131         `DEFINE_CSR(mcycle)
132         `DEFINE_CSR(minstret)
133         bit got_minstret;
134         `DEFINE_CSR(mcycleh)
135         `DEFINE_CSR(minstreth)
136         `DEFINE_CSR(cycle)
137         `DEFINE_CSR(instret)
138         // bit got_minstret;

```



```

139 `DEFINE_CSR(cycleh)
140 `DEFINE_CSR(instreth)
141
142 logic [31:0][ 1:0] mhpcounter_we;
143 logic [31:0][63:0] mhpcounter_rdata;
144 logic [31:0][63:0] mhpcounter_rmask;
145 logic [31:0][63:0] mhpcounter_wdata;
146 logic [31:0][63:0] mhpcounter_wmask;
147
148 logic [31:0] mhpmevent_we;
149 logic [31:0][31:0] mhpmevent_rdata;
150 logic [31:0][31:0] mhpmevent_rmask;
151 logic [31:0][31:0] mhpmevent_wdata;
152 logic [31:0][31:0] mhpmevent_wmask;
153 `DEFINE_CSR(mip)
154 //mnxti
155 //mintstatus
156 //mintthresh
157 //mscratchcsw
158 //mscratchcswl
159 //mclicbase
160
161 `DEFINE_CSR(tdata1)
162 `DEFINE_CSR(tdata2)
163 `DEFINE_CSR(tinfo)
164 `DEFINE_CSR(dcsr)
165
166 `DEFINE_CSR(dpc)
167 `DEFINE_CSR(dscratch0)
168 `DEFINE_CSR(dscratch1)
169 //dscratch1
170 //mconfigptr
171 //mhpcounter
172
173 `DEFINE_CSR(mvendorid)
174 `DEFINE_CSR(marchid)
175
176 `DEFINE_CSR(fflags)
177 `DEFINE_CSR(frm )
178 `DEFINE_CSR(fcsr )
179
180 `DEFINE_CSR(lpstart0 )
181 `DEFINE_CSR(lpend0 )
182 `DEFINE_CSR(lpcount0 )
183 `DEFINE_CSR(lpstart1 )
184 `DEFINE_CSR(lpend1 )
185 `DEFINE_CSR(lpcount1 )
186
187 } m_csr;
188
189 enum logic[2:0] {
190 IF, ID, EX, WB, WB_2, APU
191 } m_stage;
192
193 function new();
194 this.m_order = 0;
195 this.m_start_cycle = 0;
196 this.m_stop_cycle = 0;
197 this.m_start_time = 0;
198 this.m_stop_time = 0;
199 this.m_skip_order = 1'b0;
200 this.m_valid = 1'b0;
201 this.m_move_down_pipe = 1'b0;
202 this.m_data_misaligned = 1'b0;
203 this.m_got_first_data = 1'b0;
204 this.m_got_ex_reg = 1'b0;
205 this.m_intr = 1'b0;
206 this.m_intr = 1'b0;
207 this.m_dbg_taken = 1'b0;
208 this.m_dbg_cause = 1'b0;
209 this.m_is_ebreak = 1'b0;
210 this.m_is_illegal = 1'b0;
211 this.m_is_irq = 1'b0;
212 this.m_is_memory = 1'b0;
213 this.m_is_load = 1'b0;
214 this.m_is_apu = 1'b0;
215 this.m_is_apu_ok = 1'b0;
216 this.m_apu_req_id = 0;
217 this.m_mem_req_id[0] = 0;
218 this.m_mem_req_id[1] = 0;
219 this.m_mem_req_id_valid = 1'b0;
220 this.m_trap = 1'b0;
221 this.m_fflags_we_non_apu = 1'b0;
222 this.m_frm_we_non_apu = 1'b0;
223 this.m_fcsr_we_non_apu = 1'b0;
224 this.m_instret_cnt = 0;
225 this.m_instret_smample_trigger = 0;
226 this.m_sample_csr_write_in_ex = 1'b1;
227 endfunction
228
229 function void get_mnemonic();
230
231 this.m_mnemonic = "INVALID";
232
233 if(!is_compressed_id_i) begin
234 // use casex instead of case inside due to ModelSim
235 casex (this.m_insn)
236 // Aliases
237 32'h00_00_00_13: this.m_mnemonic = "nop";
238 // Regular opcodes
239
240 INSTR_LUI: this.m_mnemonic = "lui";
241 INSTR_AUIPC: this.m_mnemonic = "auipc";
242 INSTR_JAL: this.m_mnemonic = "jal";
243 INSTR_JALR: this.m_mnemonic = "jalr";
244 // BRANCH
245 INSTR_BEQ: this.m_mnemonic = "beq";
246 INSTR_BNE: this.m_mnemonic = "bne";
247 INSTR_BLT: this.m_mnemonic = "blt";
248 INSTR_BGE: this.m_mnemonic = "bge";
249 INSTR_BLTU: this.m_mnemonic = "bltu";
250 INSTR_BGEU: this.m_mnemonic = "bgeu";
251 INSTR_BEQIMM: this.m_mnemonic = "cv.beqimm";
252 INSTR_BNEIMM: this.m_mnemonic = "cv.bneimm";
253 // OPIMM
254 INSTR_ADDI: this.m_mnemonic = "addi";
255 INSTR_SLTI: this.m_mnemonic = "slti";
256 INSTR_SLTIU: this.m_mnemonic = "sltiu";
257 INSTR_XORI: this.m_mnemonic = "xori";
258 INSTR_ORI: this.m_mnemonic = "ori";
259 INSTR_ANDI: this.m_mnemonic = "andi";
260 INSTR_SLLI: this.m_mnemonic = "slli";
261 INSTR_SRLI: this.m_mnemonic = "srli";
262 INSTR_SRAI: this.m_mnemonic = "srai";
263 // OP
264 INSTR_ADD: this.m_mnemonic = "add";
265 INSTR_SUB: this.m_mnemonic = "sub";
266 INSTR_SLL: this.m_mnemonic = "sll";
267 INSTR_SLT: this.m_mnemonic = "slt";
268 INSTR_SLTU: this.m_mnemonic = "sltu";
269 INSTR_XOR: this.m_mnemonic = "xor";
270 INSTR_SRL: this.m_mnemonic = "srl";
271 INSTR_SRA: this.m_mnemonic = "sra";
272 INSTR_OR: this.m_mnemonic = "or";
273 INSTR_AND: this.m_mnemonic = "and";
274 INSTR_EXTHS: this.m_mnemonic = "cv.exths";
275 INSTR_EXTHZ: this.m_mnemonic = "cv.exthz";
276 INSTR_EXTBS: this.m_mnemonic = "cv.extbs";
277 INSTR_EXTBZ: this.m_mnemonic = "cv.extbz";
278 INSTR_PAVG: this.m_mnemonic = "cv.avg";
279 INSTR_PAVGU: this.m_mnemonic = "cv.avgu";
280
281 INSTR_PADDN: this.m_mnemonic = "cv.addn";
282 INSTR_PADDUN: this.m_mnemonic = "cv.addun";
283 INSTR_PADDRN: this.m_mnemonic = "cv.addrN";
284 INSTR_PADDURN: this.m_mnemonic = "cv.addrNr";
285 INSTR_PSUBN: this.m_mnemonic = "cv.subN";
286 INSTR_PSUBUN: this.m_mnemonic = "cv.subun";
287 INSTR_PSUBRN: this.m_mnemonic = "cv.subrN";
288 INSTR_PSUBURN: this.m_mnemonic = "cv.subrNr";
289
290 INSTR_PADDNR: this.m_mnemonic = "cv.addNr";
291 INSTR_PADDUNR: this.m_mnemonic = "cv.addunr";
292 INSTR_PADDRNR: this.m_mnemonic = "cv.addrNr";
293 INSTR_PADDURNR: this.m_mnemonic = "cv.addrNr";
294 INSTR_PSUBNR: this.m_mnemonic = "cv.subNr";
295 INSTR_PSUBUNR: this.m_mnemonic = "cv.subunr";
296 INSTR_PSUBRNR: this.m_mnemonic = "cv.subrNr";
297 INSTR_PSUBURNR: this.m_mnemonic = "cv.subrNr";
298
299 INSTR_PSLET: this.m_mnemonic = "cv.slet";
300 INSTR_PSLETU: this.m_mnemonic = "cv.sletu";
301 INSTR_PMIN: this.m_mnemonic = "cv.min";
302 INSTR_PMINU: this.m_mnemonic = "cv.minu";
303 INSTR_PMAX: this.m_mnemonic = "cv.max";
304 INSTR_PMAXU: this.m_mnemonic = "cv.maxu";
305 INSTR_PABS: this.m_mnemonic = "cv.abs";
306 INSTR_PCLIP: this.m_mnemonic = "cv.clip";
307 INSTR_PCLIPU: this.m_mnemonic = "cv.clipu";
308 INSTR_PBEXT: this.m_mnemonic = "cv.extract";
309 INSTR_PBEXTU: this.m_mnemonic = "cv.extractu";
310 INSTR_PBINS: this.m_mnemonic = "cv.insert";
311 INSTR_PBCLR: this.m_mnemonic = "cv.bclr";
312 INSTR_PBSET: this.m_mnemonic = "cv.bset";
313 INSTR_PBREV: this.m_mnemonic = "cv.bitrev";
314
315 INSTR_PCLIPR: this.m_mnemonic = "cv.clipr";
316 INSTR_PCLIPUR: this.m_mnemonic = "cv.clipur";
317 INSTR_PBEXTR: this.m_mnemonic = "cv.extractr";
318 INSTR_PBEXTUR: this.m_mnemonic = "cv.extractur";
319 INSTR_PBINSR: this.m_mnemonic = "cv.insertr";
320 INSTR_PBCLRR: this.m_mnemonic = "cv.bclrr";
321 INSTR_PBSETR: this.m_mnemonic = "cv.bsetr";
322
323 INSTR_FF1: this.m_mnemonic = "cv.ff1";
324 INSTR_FL1: this.m_mnemonic = "cv.fl1";
325 INSTR_CLB: this.m_mnemonic = "cv.clb";
326 INSTR_CNT: this.m_mnemonic = "cv.cnt";
327 INSTR_ROR: this.m_mnemonic = "cv.ror";
328
329 // FENCE
330 INSTR_FENCE: this.m_mnemonic = "fence";
331 INSTR_FENCEI: this.m_mnemonic = "fencei";
332 // SYSTEM (CSR manipulation)
333 INSTR_CSRRW: this.m_mnemonic = "csrrw";
334 INSTR_CSRRS: this.m_mnemonic = "csrrs";
335 INSTR_CSRRC: this.m_mnemonic = "csrrc";
336 INSTR_CSRRWI: this.m_mnemonic = "csrrwi";
337 INSTR_CSRRSI: this.m_mnemonic = "csrrsi";
338 INSTR_CSRRCI: this.m_mnemonic = "csrrci";
339 // SYSTEM (others)

```

```

340 INSTR_ECALL: this.m_mnemonic = "ecall";
341 INSTR_EBREAK: this.m_mnemonic = "ebreak";
342 INSTR_URET: this.m_mnemonic = "uret";
343 INSTR_MRET: this.m_mnemonic = "mret";
344 INSTR_WFI: this.m_mnemonic = "wfi";
345
346 INSTR_DRET: this.m_mnemonic = "dret";
347
348 // RV32M
349 INSTR_PMUL: this.m_mnemonic = "mul";
350 INSTR_PMUH: this.m_mnemonic = "mulh";
351 INSTR_PMULHSU: this.m_mnemonic = "mulhsu";
352 INSTR_PMULHU: this.m_mnemonic = "mulhu";
353 INSTR_DIV: this.m_mnemonic = "div";
354 INSTR_DIVU: this.m_mnemonic = "divu";
355 INSTR_REM: this.m_mnemonic = "rem";
356 INSTR_REMU: this.m_mnemonic = "remu";
357
358 // PULP MULTIPLIER
359 INSTR_PMAC: this.m_mnemonic = "cv.mac";
360 INSTR_PMSU: this.m_mnemonic = "cv.msu";
361 INSTR_PMULSN: this.m_mnemonic = "cv.mulsN";
362 INSTR_PMULHSN: this.m_mnemonic = "cv.mulhsN";
363 INSTR_PMULSRN: this.m_mnemonic = "cv.mulsRN";
364 INSTR_PMULHSRN: this.m_mnemonic = "cv.mulhsRN";
365 INSTR_PMULUN: this.m_mnemonic = "cv.muluN";
366 INSTR_PMULHUHUN: this.m_mnemonic = "cv.mulhuN";
367 INSTR_PMULHURN: this.m_mnemonic = "cv.mulhuRN";
368 INSTR_PMULHURN: this.m_mnemonic = "cv.mulhuRN";
369 INSTR_PMACHSN: this.m_mnemonic = "cv.machsN";
370 INSTR_PMACSRN: this.m_mnemonic = "cv.machsRN";
371 INSTR_PMACHSRN: this.m_mnemonic = "cv.machsRN";
372 INSTR_PMACUN: this.m_mnemonic = "cv.macuN";
373 INSTR_PMACHUN: this.m_mnemonic = "cv.machuN";
374 INSTR_PMACURN: this.m_mnemonic = "cv.macuRN";
375 INSTR_PMACHURN: this.m_mnemonic = "cv.machuRN";
376
377 // FP-OP
378 INSTR_FMAADD: this.m_mnemonic = "fmadd.s";
379 INSTR_FMSUB: this.m_mnemonic = "fmsub.s";
380 INSTR_FNMADD: this.m_mnemonic = "fnmadd.s";
381 INSTR_FNMSUB: this.m_mnemonic = "fnmsub.s";
382 INSTR_FADD: this.m_mnemonic = "fadd.s";
383 INSTR_FSUB: this.m_mnemonic = "fsub.s";
384 INSTR_FMUL: this.m_mnemonic = "fmul.s";
385 INSTR_FDIV: this.m_mnemonic = "fddiv.s";
386 INSTR_FSQRT: this.m_mnemonic = "fsqrt.s";
387 INSTR_FSGNJS: this.m_mnemonic = "fsgnj.s";
388 INSTR_FSGNJNS: this.m_mnemonic = "fsgnjN.s";
389 INSTR_FSGNJXS: this.m_mnemonic = "fsgnjx.s";
390 INSTR_FMIN: this.m_mnemonic = "fmin.s";
391 INSTR_FMAX: this.m_mnemonic = "fmax.s";
392 INSTR_FCVTWS: this.m_mnemonic = "fcvt.w.s";
393 INSTR_FCVTWSU: this.m_mnemonic = "fcvt.wu.s";
394 INSTR_FMVXS: this.m_mnemonic = "fmv.x.s";
395 INSTR_FEQS: this.m_mnemonic = "feq.s";
396 INSTR_FLTS: this.m_mnemonic = "flt.s";
397 INSTR_FLES: this.m_mnemonic = "fle.s";
398 INSTR_FCLASS: this.m_mnemonic = "fclass.s";
399 INSTR_FCVTSW: this.m_mnemonic = "fcvt.s.w";
400 INSTR_FCVTSWU: this.m_mnemonic = "fcvt.s.wu";
401 INSTR_FMVXSX: this.m_mnemonic = "fmv.s.x";
402
403 // RV32A
404 INSTR_LR: this.m_mnemonic = "lr.w";
405 INSTR_SC: this.m_mnemonic = "sc.w";
406 INSTR_AMOSWAP: this.m_mnemonic = "amoswap.w";
407 INSTR_AMOADD: this.m_mnemonic = "amoadd.w";
408 INSTR_AMOXOR: this.m_mnemonic = "amoxor.w";
409 INSTR_AMOAND: this.m_mnemonic = "amoand.w";
410 INSTR_AMOOR: this.m_mnemonic = "amoor.w";
411 INSTR_AMOIN: this.m_mnemonic = "amomin.w";
412 INSTR_AMOMAX: this.m_mnemonic = "amomax.w";
413 INSTR_AMOMINU: this.m_mnemonic = "amominu.w";
414 INSTR_AMOMAXU: this.m_mnemonic = "amomaxu.w";
415
416 // LOAD STORE
417 INSTR_LB: this.m_mnemonic = "lb";
418 INSTR_LH: this.m_mnemonic = "lh";
419 INSTR_LW: this.m_mnemonic = "lw";
420 INSTR_LBU: this.m_mnemonic = "lbu";
421 INSTR_LHU: this.m_mnemonic = "lhu";
422 INSTR_SB: this.m_mnemonic = "sb";
423 INSTR_SH: this.m_mnemonic = "sh";
424 INSTR_SW: this.m_mnemonic = "sw";
425
426 // CUSTOM 0
427 // Post-Increment Register-Immediate Load
428 INSTR_CVLBI: this.m_mnemonic = "cv.lb";
429 INSTR_CVLBU: this.m_mnemonic = "cv.lbu";
430 INSTR_CVLHI: this.m_mnemonic = "cv.lh";
431 INSTR_CVLHUI: this.m_mnemonic = "cv.lhu";
432 INSTR_CVLWI: this.m_mnemonic = "cv.lw";
433
434 // Event Load
435 INSTR_CVELW: this.m_mnemonic = "cv.elw";
436
437 // CUSTOM_1
438 // Post-Increment Register-Register Load
439 INSTR_CVLBR: this.m_mnemonic = "cv.lb";
440 INSTR_CVLBUR: this.m_mnemonic = "cv.lbu";
441
442 INSTR_CVLHR: this.m_mnemonic = "cv.lh";
443 INSTR_CVLHUR: this.m_mnemonic = "cv.lhu";
444 INSTR_CVLWR: this.m_mnemonic = "cv.lw";
445
446 // Register-Register Load
447 INSTR_CVLBR: this.m_mnemonic = "cv.lb";
448 INSTR_CVLBUR: this.m_mnemonic = "cv.lbu";
449 INSTR_CVLHR: this.m_mnemonic = "cv.lh";
450 INSTR_CVLHUR: this.m_mnemonic = "cv.lhu";
451 INSTR_CVLWR: this.m_mnemonic = "cv.lw";
452
453 // Post-Increment Register-Immediate Store
454 INSTR_CVSB: this.m_mnemonic = "cv.sb";
455 INSTR_CVSH: this.m_mnemonic = "cv.sh";
456 INSTR_CVSW: this.m_mnemonic = "cv.sw";
457
458 // Post-Increment Register-Register Store operations
459 INSTR_CVSB: this.m_mnemonic = "cv.sb";
460 INSTR_CVSH: this.m_mnemonic = "cv.sh";
461 INSTR_CVSW: this.m_mnemonic = "cv.sw";
462
463 // Register-Register Store operations
464 INSTR_CVSB: this.m_mnemonic = "cv.sb";
465 INSTR_CVSH: this.m_mnemonic = "cv.sh";
466 INSTR_CVSW: this.m_mnemonic = "cv.sw";
467
468 // Hardware Loops
469 INSTR_CVSTART0: this.m_mnemonic = "cv.starti 0";
470 INSTR_CVSTART0: this.m_mnemonic = "cv.start 0";
471 INSTR_CVSENDI0: this.m_mnemonic = "cv.endi 0";
472 INSTR_CVEND0: this.m_mnemonic = "cv.end 0";
473 INSTR_CVCOUNTI0: this.m_mnemonic = "cv.counti 0";
474 INSTR_CVCOUNT0: this.m_mnemonic = "cv.count 0";
475 INSTR_CVSETUPI0: this.m_mnemonic = "cv.setupi 0";
476 INSTR_CVSETUP0: this.m_mnemonic = "cv.setup 0";
477 INSTR_CVSTART1: this.m_mnemonic = "cv.starti 1";
478 INSTR_CVSTART1: this.m_mnemonic = "cv.start 1";
479 INSTR_CVSENDI1: this.m_mnemonic = "cv.endi 1";
480 INSTR_CVEND1: this.m_mnemonic = "cv.end 1";
481 INSTR_CVCOUNTI1: this.m_mnemonic = "cv.counti 1";
482 INSTR_CVCOUNT1: this.m_mnemonic = "cv.count 1";
483 INSTR_CVSETUPI1: this.m_mnemonic = "cv.setupi 1";
484 INSTR_CVSETUP1: this.m_mnemonic = "cv.setup 1";
485
486 // CUSTOM 3
487 // SIMD ALU
488 INSTR_CVADDH: this.m_mnemonic = "cv.add.h";
489 INSTR_CVADDSC: this.m_mnemonic = "cv.add.sc.h";
490 INSTR_CVADDSCIH: this.m_mnemonic = "cv.add.sci.h";
491 INSTR_CVADDB: this.m_mnemonic = "cv.add.b";
492 INSTR_CVADDSCB: this.m_mnemonic = "cv.add.sc.b";
493 INSTR_CVADDSCIB: this.m_mnemonic = "cv.add.sci.b";
494 INSTR_CVSUBH: this.m_mnemonic = "cv.sub.h";
495 INSTR_CVSUBSCH: this.m_mnemonic = "cv.sub.sc.h";
496 INSTR_CVSUBSCIH: this.m_mnemonic = "cv.sub.sci.h";
497 INSTR_CVSUBB: this.m_mnemonic = "cv.sub.b";
498 INSTR_CVSUBSCB: this.m_mnemonic = "cv.sub.sc.b";
499 INSTR_CVSUBSCIB: this.m_mnemonic = "cv.sub.sci.b";
500 INSTR_CVAUGH: this.m_mnemonic = "cv.avg.h";
501 INSTR_CVAVGSC: this.m_mnemonic = "cv.avg.sc.h";
502 INSTR_CVAVGSCIH: this.m_mnemonic = "cv.avg.sci.h";
503 INSTR_CVAVGB: this.m_mnemonic = "cv.avg.b";
504 INSTR_CVAVGSCB: this.m_mnemonic = "cv.avg.sc.b";
505 INSTR_CVAVGSCIB: this.m_mnemonic = "cv.avg.sci.b";
506 INSTR_CVAVGU: this.m_mnemonic = "cv.avgu.h";
507 INSTR_CVAVGUSC: this.m_mnemonic = "cv.avgu.sc.h";
508 INSTR_CVAVGUSCIH: this.m_mnemonic = "cv.avgu.sci.h";
509 INSTR_CVAVGUB: this.m_mnemonic = "cv.avgu.b";
510 INSTR_CVAVGUSCB: this.m_mnemonic = "cv.avgu.sc.b";
511 INSTR_CVAVGUSCIB: this.m_mnemonic = "cv.avgu.sci.b";
512 INSTR_CVMINH: this.m_mnemonic = "cv.min.h";
513 INSTR_CVMINSCH: this.m_mnemonic = "cv.min.sc.h";
514 INSTR_CVMINSCIH: this.m_mnemonic = "cv.min.sci.h";

```

"cv.min.sci.h";			565	INSTR_CVANDH	: this.m_mnemonic =
514	INSTR_CVMINB	: this.m_mnemonic =	"cv.and.h";	INSTR_CVANDSCH	: this.m_mnemonic =
"cv.min.b";			566		
515	INSTR_CVMINSCB	: this.m_mnemonic =	"cv.and.sc.h";	INSTR_CVANDSCIH	: this.m_mnemonic =
"cv.min.sc.b";			567		
516	INSTR_CVMINSCIB	: this.m_mnemonic =	"cv.and.sci.h";	INSTR_CVANDB	: this.m_mnemonic =
"cv.min.sci.b";			568		
517	INSTR_CVMINUH	: this.m_mnemonic =	"cv.and.b";	INSTR_CVANDSCB	: this.m_mnemonic =
"cv.minu.h";			569		
518	INSTR_CVMINUSCH	: this.m_mnemonic =	"cv.and.sc.b";	INSTR_CVANDSCIB	: this.m_mnemonic =
"cv.minu.sc.h";			570		
519	INSTR_CVMINUSCIH	: this.m_mnemonic =	"cv.and.sci.b";	INSTR_CVABSH	: this.m_mnemonic =
"cv.minu.sci.h";			571		
520	INSTR_CVMINUB	: this.m_mnemonic =	"cv.abs.h";	INSTR_CVABSB	: this.m_mnemonic =
"cv.minu.b";			572		
521	INSTR_CVMINUSCB	: this.m_mnemonic =	"cv.abs.b";	INSTR_CVEXTRACTH	: this.m_mnemonic = "cv.ex-
"cv.minu.sc.b";			573		
522	INSTR_CVMINUSCIB	: this.m_mnemonic =	tract.h";	INSTR_CVEXTRACTB	: this.m_mnemonic = "cv.ex-
"cv.minu.sci.b";			574		
523	INSTR_CVMAXH	: this.m_mnemonic =	tract.b";	INSTR_CVEXTRACTUH	: this.m_mnemonic = "cv.ex-
"cv.max.h";			575		
524	INSTR_CVMAXSCH	: this.m_mnemonic =	tractu.h";	INSTR_CVEXTRACTUB	: this.m_mnemonic = "cv.ex-
"cv.max.sc.h";			576		
525	INSTR_CVMAXSCIH	: this.m_mnemonic =	tractu.b";	INSTR_CVINERTH	: this.m_mnemonic = "cv.in-
"cv.max.sci.h";			577		
526	INSTR_CVMAXB	: this.m_mnemonic =	sert.h";	INSTR_CVINERTB	: this.m_mnemonic = "cv.in-
"cv.max.b";			578		
527	INSTR_CVMAXSCB	: this.m_mnemonic =	sert.b";	INSTR_CVDOTUPH	: this.m_mnemonic =
"cv.max.sc.b";			579		
528	INSTR_CVMAXSCIB	: this.m_mnemonic =	"cv.dotup.h";	INSTR_CVDOTUPSCH	: this.m_mnemonic =
"cv.max.sci.b";			580		
529	INSTR_CVMAXUH	: this.m_mnemonic =	"cv.dotup.sc.h";	INSTR_CVDOTUPSCIH	: this.m_mnemonic =
"cv.maxu.h";			581		
530	INSTR_CVMAXUSCH	: this.m_mnemonic =	"cv.dotup.sci.h";	INSTR_CVDOTUPB	: this.m_mnemonic =
"cv.maxu.sc.h";			582		
531	INSTR_CVMAXUSCIH	: this.m_mnemonic =	"cv.dotup.b";	INSTR_CVDOTUPSCB	: this.m_mnemonic =
"cv.maxu.sci.h";			583		
532	INSTR_CVMAXUB	: this.m_mnemonic =	"cv.dotup.sc.b";	INSTR_CVDOTUPSCIB	: this.m_mnemonic =
"cv.maxu.b";			584		
533	INSTR_CVMAXUSCB	: this.m_mnemonic =	"cv.dotup.sci.b";	INSTR_CVDOTUSPH	: this.m_mnemonic =
"cv.maxu.sc.b";			585		
534	INSTR_CVMAXUSCIB	: this.m_mnemonic =	"cv.dotusp.h.h";	INSTR_CVDOTUSPSCH	: this.m_mnemonic =
"cv.maxu.sci.b";			586		
535	INSTR_CVSR LH	: this.m_mnemonic =	"cv.dotusp.sc.h.sc.h";	INSTR_CVDOTUSPSCIH	: this.m_mnemonic =
"cv.srl.h";			587		
536	INSTR_CVSR LSCH	: this.m_mnemonic =	"cv.dotusp.sci.h.sci.h";	INSTR_CVDOTUSPB	: this.m_mnemonic =
"cv.srl.sc.h";			588		
537	INSTR_CVSR LSCIH	: this.m_mnemonic =	"cv.dotusp.b.b";	INSTR_CVDOTUSPSCB	: this.m_mnemonic =
"cv.srl.sci.h";			589		
538	INSTR_CVSR LB	: this.m_mnemonic =	"cv.dotusp.sc.b.sc.b";	INSTR_CVDOTUSPSCIB	: this.m_mnemonic =
"cv.srl.b";			590		
539	INSTR_CVSR LSCB	: this.m_mnemonic =	"cv.dotusp.sci.b.sci.b";	INSTR_CVDOTSPH	: this.m_mnemonic = "cv.dot-
"cv.srl.sc.b";			591		
540	INSTR_CVSR LSCIB	: this.m_mnemonic =	sp.h";	INSTR_CVDOTSPSCH	: this.m_mnemonic = "cv.dot-
"cv.srl.sci.b";			592		
541	INSTR_CVSR AH	: this.m_mnemonic =	sp.sc.h";	INSTR_CVDOTSPSCIH	: this.m_mnemonic = "cv.dot-
"cv.sra.h";			593		
542	INSTR_CVSR ASCH	: this.m_mnemonic =	sp.sci.h";	INSTR_CVDOTSPB	: this.m_mnemonic = "cv.dot-
"cv.sra.sc.h";			594		
543	INSTR_CVSR ASCIH	: this.m_mnemonic =	sp.b";	INSTR_CVDOTSPSCB	: this.m_mnemonic = "cv.dot-
"cv.sra.sci.h";			595		
544	INSTR_CVSR AB	: this.m_mnemonic =	sp.sc.b";	INSTR_CVDOTSPSCIB	: this.m_mnemonic = "cv.dot-
"cv.sra.b";			596		
545	INSTR_CVSR ASCB	: this.m_mnemonic =	sp.sci.b";	INSTR_CVSDOTUPH	: this.m_mnemonic =
"cv.sra.sc.b";			597		
546	INSTR_CVSR ASCIB	: this.m_mnemonic =	"cv.sdotup.h";	INSTR_CVSDOTUPSCH	: this.m_mnemonic =
"cv.sra.sci.b";			598		
547	INSTR_CVSL LH	: this.m_mnemonic =	"cv.sdotup.sc.h";	INSTR_CVSDOTUPSCIH	: this.m_mnemonic =
"cv.sll.h";			599		
548	INSTR_CVSL LSCH	: this.m_mnemonic =	"cv.sdotup.sci.h";	INSTR_CVSDOTUPB	: this.m_mnemonic =
"cv.sll.sc.h";			600		
549	INSTR_CVSL LSCIH	: this.m_mnemonic =	"cv.sdotup.b";	INSTR_CVSDOTUPSCB	: this.m_mnemonic =
"cv.sll.sci.h";			601		
550	INSTR_CVSL LB	: this.m_mnemonic =	"cv.sdotup.sc.b";	INSTR_CVSDOTUPSCIB	: this.m_mnemonic =
"cv.sll.b";			602		
551	INSTR_CVSL LSCB	: this.m_mnemonic =	"cv.sdotup.sci.b";	INSTR_CVSDOTUSPH	: this.m_mnemonic =
"cv.sll.sc.b";			603		
552	INSTR_CVSL LSCIB	: this.m_mnemonic =	"cv.sdotusp.h";	INSTR_CVSDOTUSPSCH	: this.m_mnemonic =
"cv.sll.sci.b";			604		
553	INSTR_CVORH	: this.m_mnemonic = "cv.or.h";	"cv.sdotusp.sc.h";	INSTR_CVSDOTUSPSCIH	: this.m_mnemonic =
554	INSTR_CVOR SCH	: this.m_mnemonic =	605		
"cv.or.sc.h";			"cv.sdotusp.sci.h";	INSTR_CVSDOTUSPB	: this.m_mnemonic =
555	INSTR_CVOR SCIH	: this.m_mnemonic =	606		
"cv.or.sci.h";			"cv.sdotusp.b";	INSTR_CVSDOTUSPSCB	: this.m_mnemonic =
556	INSTR_CVORB	: this.m_mnemonic = "cv.or.b";	607		
557	INSTR_CVOR SCB	: this.m_mnemonic =	"cv.sdotusp.sc.b";	INSTR_CVSDOTUSPSCIB	: this.m_mnemonic =
"cv.or.sc.b";			608		
558	INSTR_CVOR SCIB	: this.m_mnemonic =	"cv.sdotusp.sci.b";	INSTR_CVSDOTSPH	: this.m_mnemonic = "cv.sdot-
"cv.or.sci.b";			609		
559	INSTR_CVXORH	: this.m_mnemonic =	sp.h";	INSTR_CVSDOTSPSCH	: this.m_mnemonic = "cv.sdot-
"cv.xor.h";			610		
560	INSTR_CVXOR SCH	: this.m_mnemonic =	sp.sc.h";	INSTR_CVSDOTSPSCIH	: this.m_mnemonic = "cv.sdot-
"cv.xor.sc.h";			611		
561	INSTR_CVXOR SCIH	: this.m_mnemonic =	sp.sci.h";	INSTR_CVSDOTSPB	: this.m_mnemonic = "cv.sdot-
"cv.xor.sci.h";			612		
562	INSTR_CVXORB	: this.m_mnemonic =	sp.b";	INSTR_CVSDOTSPSCB	: this.m_mnemonic = "cv.sdot-
"cv.xor.b";			613		
563	INSTR_CVXOR SCB	: this.m_mnemonic =	sp.sc.b";	INSTR_CVSDOTSPSCIB	: this.m_mnemonic = "cv.sdot-
"cv.xor.sc.b";			614		
564	INSTR_CVXOR SCIB	: this.m_mnemonic =	sp.sci.b";	INSTR_CVSHUFFLEH	: this.m_mnemonic =
"cv.xor.sci.b";			615		

```

"cv.shuffle.h";
616 INSTR_CVSHUFFLESCIH : this.m_mnemonic =
"cv.shuffle.sci.h";
617 INSTR_CVSHUFFLEB : this.m_mnemonic =
"cv.shuffle.b";
618 INSTR_CVSHUFFLEL0SCIB : this.m_mnemonic =
"cv.shufflel0.sci.b";
619 INSTR_CVSHUFFLEL1SCIB : this.m_mnemonic =
"cv.shufflel1.sci.b";
620 INSTR_CVSHUFFLEL2SCIB : this.m_mnemonic =
"cv.shufflel2.sci.b";
621 INSTR_CVSHUFFLEL3SCIB : this.m_mnemonic =
"cv.shufflel3.sci.b";
622 INSTR_CVSHUFFLE2H : this.m_mnemonic =
"cv.shuffle2.h";
623 INSTR_CVSHUFFLE2B : this.m_mnemonic =
"cv.shuffle2.b";
624 INSTR_CVPACK : this.m_mnemonic = "cv.pack";
625 INSTR_CVPACKH : this.m_mnemonic =
"cv.pack.h";
626 INSTR_CVPACKHIB : this.m_mnemonic =
"cv.packhi.b";
627 INSTR_CVPACKLOB : this.m_mnemonic = "cv.pack-
lo.b";
628
629 // SIMD COMPARISON
630 INSTR_CVCMPEQH : this.m_mnemonic = "cv.cmp-
631 INSTR_CVCMPEQSCH : this.m_mnemonic = "cv.cm-
peq.sc.h";
632 INSTR_CVCMPEQSCIH : this.m_mnemonic = "cv.cm-
peq.sci.h";
633 INSTR_CVCMPEQB : this.m_mnemonic = "cv.cmp-
634 INSTR_CVCMPEQSCB : this.m_mnemonic = "cv.cm-
peq.sc.b";
635 INSTR_CVCMPEQSCIB : this.m_mnemonic = "cv.cm-
peq.sci.b";
636 INSTR_CVCMPEH : this.m_mnemonic = "cv.cmpne.h";
637 INSTR_CVCMPEHSC : this.m_mnemonic =
"cv.cmpne.sc.h";
638 INSTR_CVCMPEHSCIH : this.m_mnemonic =
"cv.cmpne.sci.h";
639 INSTR_CVCMPEB : this.m_mnemonic = "cv.cmpne.b";
640 INSTR_CVCMPEBSC : this.m_mnemonic =
"cv.cmpne.sc.b";
641 INSTR_CVCMPEBSCIB : this.m_mnemonic =
"cv.cmpne.sci.b";
642 INSTR_CVCMPTGTH : this.m_mnemonic = "cv.cmp-
643 INSTR_CVCMPTGTSCH : this.m_mnemonic = "cv.cmp-
gt.sc.h";
644 INSTR_CVCMPTGTSCIH : this.m_mnemonic = "cv.cmp-
gt.sci.h";
645 INSTR_CVCMPTGTB : this.m_mnemonic = "cv.cmp-
646 INSTR_CVCMPTGTSCB : this.m_mnemonic = "cv.cmp-
gt.sc.b";
647 INSTR_CVCMPTGTSCIB : this.m_mnemonic = "cv.cmp-
gt.sci.b";
648 INSTR_CVCMPEGH : this.m_mnemonic = "cv.cmp-
649 INSTR_CVCMPEGSCH : this.m_mnemonic = "cv.cmp-
ge.sc.h";
650 INSTR_CVCMPEGSCIH : this.m_mnemonic = "cv.cmp-
ge.sci.h";
651 INSTR_CVCMPEGB : this.m_mnemonic = "cv.cmp-
652 INSTR_CVCMPEGSCB : this.m_mnemonic = "cv.cmp-
ge.sc.b";
653 INSTR_CVCMPEGSCIB : this.m_mnemonic = "cv.cmp-
ge.sci.b";
654 INSTR_CVCMPLTH : this.m_mnemonic = "cv.cmp-
655 INSTR_CVCMPLTSC : this.m_mnemonic = "cv.cm-
plt.sc.h";
656 INSTR_CVCMPLTSCIH : this.m_mnemonic = "cv.cm-
plt.sci.h";
657 INSTR_CVCMPLTB : this.m_mnemonic = "cv.cmp-
658 INSTR_CVCMPLTSCB : this.m_mnemonic = "cv.cm-
plt.sc.b";
659 INSTR_CVCMPLTSCIB : this.m_mnemonic = "cv.cm-
plt.sci.b";
660 INSTR_CVCMPLH : this.m_mnemonic = "cv.cmp-
661 INSTR_CVCMPLESCH : this.m_mnemonic =
"cv.cmp-
662 INSTR_CVCMPLESCIH : this.m_mnemonic =
"cv.cmp-
663 INSTR_CVCMPLB : this.m_mnemonic = "cv.cmp-
664 INSTR_CVCMPLESCB : this.m_mnemonic =
"cv.cmp-
665 INSTR_CVCMPLESCIB : this.m_mnemonic =
"cv.cmp-
666 INSTR_CVCMPLTUH : this.m_mnemonic = "cv.cmp-
gtu.h";
667 INSTR_CVCMPLTUSCH : this.m_mnemonic = "cv.cmp-
gtu.sc.h";
668 INSTR_CVCMPLTUSCIH : this.m_mnemonic = "cv.cmp-
gtu.sci.h";
669 INSTR_CVCMPLGTUB : this.m_mnemonic = "cv.cmp-
gtu.b";
670 INSTR_CVCMPLGTUSCB : this.m_mnemonic = "cv.cmp-
gtu.sc.b";
671 INSTR_CVCMPLGTUSCIB : this.m_mnemonic = "cv.cmp-
gtu.sci.b";
672 INSTR_CVCMPLGEUH : this.m_mnemonic = "cv.cmp-
geu.h";
673 INSTR_CVCMPLGEUSCH : this.m_mnemonic = "cv.cmp-

```

```

geu.sc.h";
674 INSTR_CVCMPLGEUSCIH : this.m_mnemonic = "cv.cmp-
geu.sci.h";
675 INSTR_CVCMPLGEUB : this.m_mnemonic = "cv.cmp-
geu.b";
676 INSTR_CVCMPLGEUSCB : this.m_mnemonic = "cv.cmp-
geu.sc.b";
677 INSTR_CVCMPLGEUSCIB : this.m_mnemonic = "cv.cmp-
geu.sci.b";
678 INSTR_CVCMPLTUH : this.m_mnemonic = "cv.cm-
pltu.h";
679 INSTR_CVCMPLTUSCH : this.m_mnemonic = "cv.cm-
pltu.sc.h";
680 INSTR_CVCMPLTUSCIH : this.m_mnemonic = "cv.cm-
pltu.sci.h";
681 INSTR_CVCMPLTUB : this.m_mnemonic = "cv.cm-
pltu.b";
682 INSTR_CVCMPLTUSCB : this.m_mnemonic = "cv.cm-
pltu.sc.b";
683 INSTR_CVCMPLTUSCIB : this.m_mnemonic = "cv.cm-
pltu.sci.b";
684 INSTR_CVCMPLLEUH : this.m_mnemonic = "cv.cm-
pleu.h";
685 INSTR_CVCMPLLEUSCH : this.m_mnemonic = "cv.cm-
pleu.sc.h";
686 INSTR_CVCMPLLEUSCIH : this.m_mnemonic = "cv.cm-
pleu.sci.h";
687 INSTR_CVCMPLLEUB : this.m_mnemonic = "cv.cm-
pleu.b";
688 INSTR_CVCMPLLEUSCB : this.m_mnemonic = "cv.cm-
pleu.sc.b";
689 INSTR_CVCMPLLEUSCIB : this.m_mnemonic = "cv.cm-
pleu.sci.b";
690
691 // SIMD CPLX
692 INSTR_CVCPXLMULR : this.m_mnemonic =
"cv.cplxmul.r";
693 INSTR_CVCPXLMULRDIV2 : this.m_mnemonic =
"cv.cplxmul.r.div2";
694 INSTR_CVCPXLMULRDIV4 : this.m_mnemonic =
"cv.cplxmul.r.div4";
695 INSTR_CVCPXLMULRDIV8 : this.m_mnemonic =
"cv.cplxmul.r.div8";
696 INSTR_CVCPXLMULI : this.m_mnemonic =
"cv.cplxmul.i";
697 INSTR_CVCPXLMULIDIV2 : this.m_mnemonic =
"cv.cplxmul.i.div2";
698 INSTR_CVCPXLMULIDIV4 : this.m_mnemonic =
"cv.cplxmul.i.div4";
699 INSTR_CVCPXLMULIDIV8 : this.m_mnemonic =
"cv.cplxmul.i.div8";
700 INSTR_CVCPXCONJ : this.m_mnemonic =
"cv.cplxconj";
701 INSTR_CVSUBROTMJ : this.m_mnemonic = "cv.subrot-
mj";
702 INSTR_CVSUBROTMJDIV2 : this.m_mnemonic = "cv.subrot-
mj.div2";
703 INSTR_CVSUBROTMJDIV4 : this.m_mnemonic = "cv.subrot-
mj.div4";
704 INSTR_CVSUBROTMJDIV8 : this.m_mnemonic = "cv.subrot-
mj.div8";
705 INSTR_CVADDIV2 : this.m_mnemonic =
"cv.add.div2";
706 INSTR_CVADDIV4 : this.m_mnemonic =
"cv.add.div4";
707 INSTR_CVADDIV8 : this.m_mnemonic =
"cv.add.div8";
708 INSTR_CVSUBIV2 : this.m_mnemonic =
"cv.sub.div2";
709 INSTR_CVSUBIV4 : this.m_mnemonic =
"cv.sub.div4";
710 INSTR_CVSUBIV8 : this.m_mnemonic =
"cv.sub.div8";
711
712 default : this.m_mnemonic = "INVALID";
713 endcase // unique case (instr)
714 end else begin //Compressed instruction
715 unique case (this.m_insn[1:0])
716 // C0
717 2'b00: begin
718 unique case (this.m_insn[15:13])
719 3'b000: begin
720 // c.addi4spn -> addi rd, x2, imm
721 this.m_mnemonic = "c.addi4spn";
722 end
723
724 3'b001: begin this.m_mnemonic = "c.fld"; end
725 3'b010: begin this.m_mnemonic = "c.lw"; end
726 3'b011: begin this.m_mnemonic = "c.flw"; end
727 3'b101: begin this.m_mnemonic = "c.fsd"; end
728 3'b110: begin this.m_mnemonic = "c.sw"; end
729 3'b111: begin this.m_mnemonic = "c.fsw"; end
730 default: begin this.m_mnemonic = "INVALID"; end
731 endcase
732 end
733
734 // C1
735 2'b01: begin
736 unique case (this.m_insn[15:13])
737 3'b000: begin
738 // c.addi -> addi rd, rd, nzimm

```

```

739         // c.nop
740         if(this.m_insn[11:7] == '0') begin
741             this.m_mnemonic = "c.nop";
742         end else begin
743             this.m_mnemonic = "c.addi";
744         end
745     end
746     3'b001: this.m_mnemonic = "c.jal";
747     3'b101: this.m_mnemonic = "c.j";
748
749     3'b010: begin
750         if (this.m_insn[11:7] == 5'b0) begin
751             // Hint -> addi x0, x0, nzimm
752             this.m_mnemonic = "HINT";
753         end else begin
754             this.m_mnemonic = "c.li";
755         end
756     end
757
758     3'b011: begin
759         if ({this.m_insn[12], this.m_insn[6:2]} ==
6'b0) begin
760             this.m_mnemonic = "INVALID";
761         end else begin
762             if (this.m_insn[11:7] == 5'h02) begin
763                 // c.addi16sp -> addi x2, x2, nzimm
764                 this.m_mnemonic = "c.addi16sp";
765             end else if (this.m_insn[11:7] == 5'b0) be-
gin
766                 // Hint -> lui x0, imm
767                 this.m_mnemonic = "HINT";
768             end else begin
769                 this.m_mnemonic = "c.lui";
770             end
771         end
772     end
773
774     3'b100: begin
775         unique case (this.m_insn[11:10])
776             2'b00 : begin
777                 // 00: c.srli -> srli rd, rd, shamt
778                 // 01: c.srai -> srai rd, rd, shamt
779                 if (this.m_insn[12] == 1'b1) begin
780
781                     // Reserved for future custom extensions
782                     this.m_mnemonic = "INVALID";
783                 end else begin
784                     if (this.m_insn[6:2] == 5'b0) begin
785                         // Hint
786                         this.m_mnemonic = "HINT";
787                     end else begin
788                         this.m_mnemonic = "c.srli";
789                     end
790                 end
791             end
792             2'b01 : begin
793                 if (this.m_insn[12] == 1'b1) begin
794
795                     // Reserved for future custom extensions
796                     this.m_mnemonic = "INVALID";
797                 end else begin
798                     if (this.m_insn[6:2] == 5'b0) begin
799                         // Hint
800                         this.m_mnemonic = "HINT";
801                     end else begin
802                         this.m_mnemonic = "c.srai";
803                     end
804                 end
805             end
806             2'b10: begin this.m_mnemonic = "c.andi"; end
807             2'b11: begin
808                 unique case ({ this.m_insn[12], this.m_in-
sn[6:5] })
809
810                     3'b000: begin this.m_mnemonic = "c.sub";
811                     end
812                     3'b001: begin this.m_mnemonic = "c.xor";
813                     end
814                     3'b010: begin this.m_mnemonic = "c.or";
815                     end
816                     3'b011: begin this.m_mnemonic = "c.and";
817                     end
818                     3'b100 : this.m_mnemonic = "c.subw";
819                     3'b101 : this.m_mnemonic = "c.addw";
820
821                     3'b110, 3'b111: begin
822                         this.m_mnemonic = "INVALID";
823                     end
824                 endcase
825             end
826         endcase
827     end
828
829     3'b110: begin this.m_mnemonic = "c.beqz"; end
830     3'b111: begin this.m_mnemonic = "c.bnez"; end
831 endcase
832 end
833
834 // C2
835 2'b10: begin
836     unique case (this.m_insn[15:13])
837         3'b000: begin
838             if (this.m_insn[12] == 1'b1) begin
839                 // Reserved for future extensions (instr_o
don't care)
840                 this.m_mnemonic = "TODO";
841             end else begin
842                 if ((this.m_insn[6:2] == 5'b0) ||
(this.m_insn[11:7] == 5'b0)) begin
843                     // Hint -> slli rd, rd, shamt
844                     this.m_mnemonic = "HINT";
845                 end else begin
846                     this.m_mnemonic = "c.slli";
847                 end
848             end
849         end
850         3'b001: begin this.m_mnemonic = "c.fldsp"; end
851         3'b010: begin this.m_mnemonic = "c.lwsp"; end
852         3'b011: begin this.m_mnemonic = "c.flwsp"; end
853
854         3'b100: begin
855             if (this.m_insn[12] == 1'b0) begin
856                 if (this.m_insn[6:2] == 5'b0) begin
857                     this.m_mnemonic = "c.jr";
858                 end else begin
859                     if (this.m_insn[11:7] == 5'b0) begin
860                         // Hint -> add x0, x0, rs2
861                         this.m_mnemonic = "HINT";
862                     end else begin
863                         this.m_mnemonic = "c.mv";
864                     end
865                 end
866             end else begin
867                 if (this.m_insn[6:2] == 5'b0) begin
868                     if (this.m_insn[11:7] == 5'b0) begin
869                         this.m_mnemonic = "c.ebreak";
870                     end else begin
871                         this.m_mnemonic = "c.jalr";
872                     end
873                 end else begin
874                     if (this.m_insn[11:7] == 5'b0) begin
875                         // Hint -> add x0, x0, rs2
876                         this.m_mnemonic = "HINT";
877                     end else begin
878                         this.m_mnemonic = "c.add";
879                     end
880                 end
881             end
882         end
883     endcase
884 end
885
886 3'b101: begin this.m_mnemonic = "c.fsdsp"; end
887 3'b110: begin this.m_mnemonic = "c.swsp"; end
888 3'b111: begin this.m_mnemonic = "c.fswsp"; end
889 endcase
890 end
891
892 endfunction
893
894 function void init_csr();
895     `INIT_CSR(mstatus)
896     `INIT_CSR(mstatus_fs)
897     `INIT_CSR(misa)
898     `INIT_CSR(mie)
899     `INIT_CSR(mtvec)
900     `INIT_CSR(mcountinhibit)
901     `INIT_CSR(mscratch)
902     `INIT_CSR(mepc)
903     `INIT_CSR(mcause)
904     `INIT_CSR(mcycle)
905     `INIT_CSR(minstret)
906     `INIT_CSR(mcycleh)
907     `INIT_CSR(minstreth)
908     `INIT_CSR(cycle)
909     `INIT_CSR(instret)
910     `INIT_CSR(cycleh)
911     `INIT_CSR(instreth)
912     this.m_csr.mhpmcounter_we = '0;
913     this.m_csr.mhpmcounter_wmask = '0;
914     this.m_csr.mhpmevent_we = '0;
915     this.m_csr.mhpmevent_wmask = '0;
916     `INIT_CSR(mip)
917     `INIT_CSR(tdata1)
918     `INIT_CSR(tdata2)
919     `INIT_CSR(tinfo)
920     `INIT_CSR(dcsr)
921     `INIT_CSR(dpc)
922     `INIT_CSR(dscratch0)
923     `INIT_CSR(dscratch1)
924     `INIT_CSR(mvendorid)
925     `INIT_CSR(marchid)
926     `INIT_CSR(fflags)
927     `INIT_CSR(frm)

```



```

924 `INIT_CSR(fcsr )
925 `INIT_CSR(lpstart0 )
926 `INIT_CSR(lpend0 )
927 `INIT_CSR(lpcount0 )
928 `INIT_CSR(lpstart1 )
929 `INIT_CSR(lpend1 )
930 `INIT_CSR(lpcount1 )
931 endfunction
932 /*
933 *
934 */
935 function void init( insn_trace_t m_source); //logic[31:0]
instr_id );
936 this.m_valid = 1'b1;
937 this.m_stage = ID;
938 this.m_order = this.m_order + 64'h1;
939 this.m_start_cycle = cycles;
940 this.m_stop_cycle = 0;
941 this.m_start_time = $time;
942 this.m_stop_time = 0;
943 if(this.m_skip_order) begin
944 this.m_order = this.m_order + 64'h1;
945 end
946 this.m_skip_order = 1'b0;
947 this.m_pc_rdata =
r_pipe_freeze_trace.pc_id;
948 this.m_is_illegal = 1'b0;
949 this.m_is_irq = 1'b0;
950 this.m_is_memory = 1'b0;
951 this.m_is_load = 1'b0;
952 this.m_is_apu = 1'b0;
953 this.m_is_apu_ok = 1'b0;
954 this.m_apu_req_id = 0;
955 this.m_mem_req_id[0] = 0;
956 this.m_mem_req_id[1] = 0;
957 this.m_mem_req_id_valid = 0;
958 this.m_data_missaligned = 1'b0;
959 this.m_got_first_data = 1'b0;
960 this.m_got_ex_reg = 1'b0;
961 this.m_got_regs_write = 1'b0;
962 this.m_move_down_pipe = 1'b0;
963 this.m_instret_cnt = 0;
964 this.m_instret_smample_trigger = 0;
965 this.m_sample_csr_write_in_ex = 1'b1;
966 this.m_rd_addr[0] = 0;
967 this.m_rd_addr[1] = 0;
968 this.m_2_rd_insn = 1'b0;
969 this.m_rs1_addr = 0;
970 this.m_rs2_addr = 0;
971 this.m_rs3_addr = 0;
972 this.m_ex_fw = 0;
973 this.m_csr_got_minstret = 0;
974 this.m_dbg_taken = 0;
975 this.m_trap = 1'b0;
976 this.m_fflags_we_non_apu = 1'b0;
977 this.m_frm_we_non_apu = 1'b0;
978 this.m_fcsr_we_non_apu = 1'b0;
979 this.m_csr_mcause_we = 0;
980 if (is_compressed_id_i) begin
981 this.m_insn[31:16] = 0;
982 this.m_insn[15:0] = m_source.m_insn[15:0];
983 end else begin
984 this.m_insn = m_source.m_insn;
985 end
986 this.get_mnemonic();
987
988 this.m_intr = m_source.m_intr;
989 this.m_dbg_taken = m_source.m_dbg_taken;
990 this.m_dbg_cause = m_source.m_dbg_cause;
991 this.m_trap = m_source.m_trap;
992
993
994 this.m_rs1_addr = r_pipe_freeze_trace.rs1_addr_id;
995 this.m_rs2_addr = r_pipe_freeze_trace.rs2_addr_id;
996 this.m_rs3_addr = r_pipe_freeze_trace.rs3_addr_id;
997 this.m_rs1_rdata = r_pipe_freeze_trace.operand_a_fw_id;
998 this.m_rs2_rdata = r_pipe_freeze_trace.operand_b_fw_id;
999 this.m_rs3_rdata = r_pipe_freeze_trace.operand_c_fw_id;
1000
1001 this.m_mem.addr = 0;
1002 this.m_mem.rmask = 0;
1003 this.m_mem.wmask = 0;
1004 this.m_mem.rdata = 0;
1005 this.m_mem.wdata = 0;
1006
1007 init_csr();
1008 endfunction
1009
1010 function logic [63:0] get_order_for_trap();
1011 // this.m_order = this.m_order + 64'h1;
1012 this.m_skip_order = 1'b1;
1013 return (this.m_order + 64'h1);
1014 endfunction
1015
1016 function void copy_full(insn_trace_t m_source);
1017 this.m_valid = m_source.m_valid;
1018 this.m_stage = m_source.m_stage;
1019 this.m_order = m_source.m_order;
1020 this.m_start_cycle = m_source.m_start_cycle;
1021 this.m_stop_cycle = m_source.m_stop_cycle;
1022 this.m_start_time = m_source.m_start_time;

```

```

1023 this.m_stop_time = m_source.m_stop_time;
1024 this.m_pc_rdata = m_source.m_pc_rdata;
1025 this.m_insn = m_source.m_insn;
1026 this.m_mnemonic = m_source.m_mnemonic;
1027 this.m_is_memory = m_source.m_is_memory;
1028 this.m_is_load = m_source.m_is_load;
1029 this.m_is_apu = m_source.m_is_apu;
1030 this.m_is_apu_ok = m_source.m_is_apu_ok;
1031 this.m_apu_req_id = m_source.m_apu_req_id;
1032 this.m_mem_req_id = m_source.m_mem_req_id;
1033 this.m_mem_req_id_valid =
m_source.m_mem_req_id_valid;
1034 this.m_data_missaligned =
m_source.m_data_missaligned;
1035 this.m_got_first_data =
m_source.m_got_first_data;
1036 this.m_got_ex_reg = m_source.m_got_ex_reg;
1037 this.m_dbg_taken = m_source.m_dbg_taken;
1038 this.m_dbg_cause = m_source.m_dbg_cause;
1039 this.m_is_ebreak = m_source.m_is_ebreak;
1040 this.m_is_illegal = m_source.m_is_illegal;
1041 this.m_is_irq = m_source.m_is_irq;
1042 this.m_instret_cnt = m_source.m_instret_cnt;
1043 this.m_instret_smample_trigger =
m_source.m_instret_smample_trigger;
1044 this.m_sample_csr_write_in_ex =
m_source.m_sample_csr_write_in_ex;
1045 this.m_rs1_addr = m_source.m_rs1_addr;
1046 this.m_rs2_addr = m_source.m_rs2_addr;
1047 this.m_rs3_addr = m_source.m_rs3_addr;
1048 this.m_rs1_rdata = m_source.m_rs1_rdata;
1049 this.m_rs2_rdata = m_source.m_rs2_rdata;
1050 this.m_rs3_rdata = m_source.m_rs3_rdata;
1051
1052 this.m_ex_fw = m_source.m_ex_fw;
1053 this.m_rd_addr = m_source.m_rd_addr;
1054 this.m_2_rd_insn = m_source.m_2_rd_insn;
1055 this.m_rd_wdata = m_source.m_rd_wdata;
1056
1057 this.m_intr = m_source.m_intr;
1058 this.m_trap = m_source.m_trap;
1059 this.m_fflags_we_non_apu =
m_source.m_fflags_we_non_apu;
1060 this.m_frm_we_non_apu =
m_source.m_frm_we_non_apu;
1061 this.m_fcsr_we_non_apu =
m_source.m_fcsr_we_non_apu;
1062
1063 this.m_mem = m_source.m_mem;
1064 //CRS
1065 `ASSIGN_CSR(mstatus)
1066 `ASSIGN_CSR(mstatus_fs)
1067 `ASSIGN_CSR(misa)
1068 `ASSIGN_CSR(mie)
1069 `ASSIGN_CSR(mtvec)
1070 `ASSIGN_CSR(mcountinhibit)
1071 `ASSIGN_CSR(mscratch)
1072 `ASSIGN_CSR(mepc)
1073 `ASSIGN_CSR(mcause)
1074 `ASSIGN_CSR(mcycle)
1075 `ASSIGN_CSR(minstret)
1076 this.m_csr.got_minstret = m_source.m_csr.got_minstret;
1077 `ASSIGN_CSR(mcycleh)
1078 `ASSIGN_CSR(minstreth)
1079 `ASSIGN_CSR(cycle)
1080 `ASSIGN_CSR(instret)
1081 // this.m_csr.got_minstret =
m_source.m_csr.got_minstret;
1082 `ASSIGN_CSR(cycleh)
1083 `ASSIGN_CSR(instreth)
1084 this.m_csr.mhpmcounter_we =
m_source.m_csr.mhpmcounter_we;
1085 this.m_csr.mhpmcounter_rdata =
m_source.m_csr.mhpmcounter_rdata;
1086 this.m_csr.mhpmcounter_rmask =
m_source.m_csr.mhpmcounter_rmask;
1087 this.m_csr.mhpmcounter_wdata =
m_source.m_csr.mhpmcounter_wdata;
1088 this.m_csr.mhpmcounter_wmask =
m_source.m_csr.mhpmcounter_wmask;
1089 this.m_csr.mhpmevent_we = m_source.m_csr.mhpmevent_we;
1090 this.m_csr.mhpmevent_rdata =
m_source.m_csr.mhpmevent_rdata;
1091 this.m_csr.mhpmevent_rmask =
m_source.m_csr.mhpmevent_rmask;
1092 this.m_csr.mhpmevent_wdata =
m_source.m_csr.mhpmevent_wdata;
1093 this.m_csr.mhpmevent_wmask =
m_source.m_csr.mhpmevent_wmask;
1094 `ASSIGN_CSR(mip)
1095 `ASSIGN_CSR(tdata1)
1096 `ASSIGN_CSR(tdata2)
1097 `ASSIGN_CSR(tinfo)
1098 `ASSIGN_CSR(dcsr)
1099 `ASSIGN_CSR(dpc)
1100 `ASSIGN_CSR(dscratch0)
1101 `ASSIGN_CSR(dscratch1)
1102 `ASSIGN_CSR(mvendorid)
1103 `ASSIGN_CSR(marchid)
1104
1105 `ASSIGN_CSR(fflags)

```

```

1106     `ASSIGN_CSR(frm    )
1107     `ASSIGN_CSR(fcsr   )
1108
1109     `ASSIGN_CSR(lpstart0)
1110     `ASSIGN_CSR(lpend0  )
1111     `ASSIGN_CSR(lpcount0)
1112     `ASSIGN_CSR(lpstart1)
1113     `ASSIGN_CSR(lpend1  )
1114     `ASSIGN_CSR(lpcount1)
1115
1116 endfunction
1117
1118 function void move_down_pipe(insn_trace_t m_source);
1119     this.copy_full(m_source);

```

```

1120     case(this.m_stage)
1121         IF : this.m_stage = ID;
1122         ID : this.m_stage = EX;
1123         EX : this.m_stage = WB;
1124         WB : this.m_stage = WB_2;
1125         APU: this.m_stage = APU;
1126     endcase
1127 endfunction
1128
1129 function void set_to_apu();
1130     this.m_stage = APU;
1131 endfunction
1132 endclass

```


PicoRV32 -- 초소형 RISC-V

13 files • 5,076 lines

API — picorv32.v

Table 1117. YosysHQ_picorv32/picorv32.v — module / interface / package (8개)

이름	종류
picorv32	module
picorv32_regs	module
picorv32_pcpi_mul	module
picorv32_pcpi_fast_mul	module
picorv32_pcpi_div	module
picorv32_axi	module
picorv32_axi_adapter	module
picorv32_wb	module

Table 1118. YosysHQ_picorv32/picorv32.v — parameter (81개) — 앞 60개만 표시(전체 81개, 본문 소스 참조)

파라미터	형/폭	기본값
ENABLE_COUNTERS	[0:0]	1
ENABLE_COUNTERS64	[0:0]	1
ENABLE_REGS_16_31	[0:0]	1
ENABLE_REGS_DUALPORT	[0:0]	1
LATCHED_MEM_RDATA	[0:0]	0
TWO_STAGE_SHIFT	[0:0]	1
BARREL_SHIFTER	[0:0]	0
TWO_CYCLE_COMPARE	[0:0]	0
TWO_CYCLE_ALU	[0:0]	0
COMPRESSED_ISA	[0:0]	0
CATCH_MISALIGN	[0:0]	1
CATCH_ILLINSN	[0:0]	1
ENABLE_PCPI	[0:0]	0
ENABLE_MUL	[0:0]	0
ENABLE_FAST_MUL	[0:0]	0
ENABLE_DIV	[0:0]	0
ENABLE_IRQ	[0:0]	0
ENABLE_IRQ_QREGS	[0:0]	1

파라미터	형/폭	기본값
ENABLE_IRQ_TIMER	[0:0]	1
ENABLE_TRACE	[0:0]	0
REGS_INIT_ZERO	[0:0]	0
MASKED_IRQ	[31:0]	32'h 0000_0000
LATCHED_IRQ	[31:0]	32'h ffff_ffff
PROGADDR_RESET	[31:0]	32'h 0000_0000
PROGADDR_IRQ	[31:0]	32'h 0000_0010
STACKADDR	[31:0]	32'h ffff_ffff
STEPS_AT_ONCE		1
CARRY_CHAIN		4
EXTRA_MUL_FFS		0
EXTRA_INSN_FFS		0
MUL_CLKGATE		0
ENABLE_COUNTERS	[0:0]	1
ENABLE_COUNTERS64	[0:0]	1
ENABLE_REGS_16_31	[0:0]	1
ENABLE_REGS_DUALPORT	[0:0]	1
TWO_STAGE_SHIFT	[0:0]	1
BARREL_SHIFTER	[0:0]	0
TWO_CYCLE_COMPARE	[0:0]	0
TWO_CYCLE_ALU	[0:0]	0
COMPRESSED_ISA	[0:0]	0
CATCH_MISALIGN	[0:0]	1
CATCH_ILLINSN	[0:0]	1
ENABLE_PCPI	[0:0]	0
ENABLE_MUL	[0:0]	0
ENABLE_FAST_MUL	[0:0]	0
ENABLE_DIV	[0:0]	0
ENABLE_IRQ	[0:0]	0
ENABLE_IRQ_QREGS	[0:0]	1
ENABLE_IRQ_TIMER	[0:0]	1
ENABLE_TRACE	[0:0]	0
REGS_INIT_ZERO	[0:0]	0
MASKED_IRQ	[31:0]	32'h 0000_0000
LATCHED_IRQ	[31:0]	32'h ffff_ffff

파라미터	형/폭	기본값
PROGADDR_RESET	[31:0]	32'h 0000_0000
PROGADDR_IRQ	[31:0]	32'h 0000_0010
STACKADDR	[31:0]	32'h ffff_fff
ENABLE_COUNTERS	[0:0]	1
ENABLE_COUNTERS64	[0:0]	1
ENABLE_REGS_16_31	[0:0]	1
ENABLE_REGS_DUALPORT	[0:0]	1

Table 1119. YosysHQ_picorv32/picorv32.v — localparam (21개)

상수	형/폭	값
irq_timer	integer	0
irq_ebreak	integer	1
irq_buserror	integer	2
irqregs_offset	integer	ENABLE_REGS_16_31 ? 32 : 16
regfile_size	integer	(ENABLE_REGS_16_31 ? 32 : 16) + 4*ENABLE_IRQ*ENABLE_IRQ_QREG...
regindex_bits	integer	(ENABLE_REGS_16_31 ? 5 : 4) + ENABLE_IRQ*ENABLE_IRQ_QREGS
WITH_PCPI		ENABLE_PCPI ENABLE_MUL ENABLE_FAST_MUL ENABLE_DIV
TRACE_BRANCH	[35:0]	{4'b 0001
TRACE_ADDR	[35:0]	{4'b 0010
TRACE_IRQ	[35:0]	{4'b 1000
cpu_state_trap		8'b10000000
cpu_state_fetch		8'b01000000
cpu_state_ld_rs1		8'b00100000
cpu_state_ld_rs2		8'b00010000
cpu_state_exec		8'b00001000
cpu_state_shift		8'b00000100
cpu_state_stmem		8'b00000010
cpu_state_ldmem		8'b00000001
IDLE		2'b00
WBSTART		2'b01
WBEND		2'b10

Table 1120. YosysHQ_picorv32/picorv32.v — 포트 (207개) — 앞 60개만 표시(전체 207개, 본문 소스 참조)

포트	방향	형/폭
clk	input	
trap	output	reg

포트	방향	형/폭
mem_valid	output	reg
mem_instr	output	reg
mem_ready	input	
mem_addr	output	reg [31:0]
mem_wdata	output	reg [31:0]
mem_wstrb	output	reg [3:0]
mem_rdata	input	[31:0]
mem_la_read	output	
mem_la_write	output	
mem_la_addr	output	[31:0]
mem_la_wdata	output	reg [31:0]
mem_la_wstrb	output	reg [3:0]
pcpi_valid	output	reg
pcpi_insn	output	reg [31:0]
pcpi_rs1	output	[31:0]
pcpi_rs2	output	[31:0]
pcpi_wr	input	
pcpi_rd	input	[31:0]
pcpi_wait	input	
pcpi_ready	input	
irq	input	[31:0]
eoi	output	reg [31:0]
rvfi_valid	output	reg
rvfi_order	output	reg [63:0]
rvfi_insn	output	reg [31:0]
rvfi_trap	output	reg
rvfi_halt	output	reg
rvfi_intr	output	reg
rvfi_mode	output	reg [1:0]
rvfi_ixl	output	reg [1:0]
rvfi_rs1_addr	output	reg [4:0]
rvfi_rs2_addr	output	reg [4:0]
rvfi_rs1_rdata	output	reg [31:0]
rvfi_rs2_rdata	output	reg [31:0]
rvfi_rd_addr	output	reg [4:0]

포트	방향	형/폭
rvfi_rd_wdata	output	reg [31:0]
rvfi_pc_rdata	output	reg [31:0]
rvfi_pc_wdata	output	reg [31:0]
rvfi_mem_addr	output	reg [31:0]
rvfi_mem_rmask	output	reg [3:0]
rvfi_mem_wmask	output	reg [3:0]
rvfi_mem_rdata	output	reg [31:0]
rvfi_mem_wdata	output	reg [31:0]
rvfi_csr_mcycle_rmask	output	reg [63:0]
rvfi_csr_mcycle_wmask	output	reg [63:0]
rvfi_csr_mcycle_rdata	output	reg [63:0]
rvfi_csr_mcycle_wdata	output	reg [63:0]
rvfi_csr_minstret_rmask	output	reg [63:0]
rvfi_csr_minstret_wmask	output	reg [63:0]
rvfi_csr_minstret_rdata	output	reg [63:0]
rvfi_csr_minstret_wdata	output	reg [63:0]
trace_valid	output	reg
trace_data	output	reg [35:0]
clk	input	
waddr	input	[5:0]
raddr1	input	[5:0]
raddr2	input	[5:0]
wdata	input	[31:0]

Table 1121. YosysHQ_picorv32/picorv32.v — #define / `define 매크로 (8개)

매크로	인자	정의
debug	(debug_command)	debug_command
debug	(debug_command)	`endif
FORMAL_KEEP		(* keep *)
assert	(assert_expr)	assert(assert_expr)
FORMAL_KEEP		(* keep *)
FORMAL_KEEP		`endif
assert	(assert_expr)	empty_statement
PICORV32_V		/*****

API — testbench.v**Table 1122. YosysHQ_picorv32/testbench.v — module / interface / package (3개)**

이름	종류
testbench	module
picorv32_wrapper	module
axi4_memory	module

Table 1123. YosysHQ_picorv32/testbench.v — parameter (6개)

파라미터	형/폭	기본값
AXI_TEST		0
VERBOSE		0
AXI_TEST		0
VERBOSE		0
AXI_TEST		0
VERBOSE		0

Table 1124. YosysHQ_picorv32/testbench.v — 포트 (24개)

포트	방향	형/폭
clk	input	
resetsn	input	
trap	output	
trace_valid	output	
trace_data	output	[35:0]
clk	input	
mem_axi_awvalid	input	
mem_axi_awready	output	reg
mem_axi_awaddr	input	[31:0]
mem_axi_awprot	input	[2:0]
mem_axi_wvalid	input	
mem_axi_wready	output	reg
mem_axi_wdata	input	[31:0]
mem_axi_wstrb	input	[3:0]
mem_axi_bvalid	output	reg
mem_axi_bready	input	
mem_axi_arvalid	input	
mem_axi_arready	output	reg
mem_axi_araddr	input	[31:0]

포트	방향	형/폭
mem_axi_arprot	input	[2:0]
mem_axi_rvalid	output	reg
mem_axi_rready	input	
mem_axi_rdata	output	reg [31:0]
tests_passed	output	reg

API — testbench_ez.v

Table 1125. YosysHQ_picorv32/testbench_ez.v — module / interface / package (1개)

이름	종류
testbench	module

API — testbench_wb.v

Table 1126. YosysHQ_picorv32/testbench_wb.v — module / interface / package (3개)

이름	종류
testbench	module
picorv32_wrapper	module
wb_ram	module

Table 1127. YosysHQ_picorv32/testbench_wb.v — parameter (5개)

파라미터	형/폭	기본값
VERBOSE		0
VERBOSE		0
depth		256
memfile		""
VERBOSE		0

Table 1128. YosysHQ_picorv32/testbench_wb.v — 포트 (17개)

포트	방향	형/폭
wb_clk	input	
wb_rst	input	
trap	output	
trace_valid	output	
trace_data	output	[35:0]
wb_clk_i	input	
wb_rst_i	input	
wb_adr_i	input	[31:0]
wb_dat_i	input	[31:0]

포트	방향	형/폭
wb_sel_i	input	[3:0]
wb_we_i	input	
wb_cyc_i	input	
wb_stb_i	input	
wb_ack_o	output	reg
wb_dat_o	output	reg [31:0]
mem_instr	input	
tests_passed	output	reg

API — axicheck.v

Table 1129. YosysHQ_picorv32/scripts/smtbmc/axicheck.v — module / interface / package (1개)

이름	종류
testbench	module

Table 1130. YosysHQ_picorv32/scripts/smtbmc/axicheck.v — 포트 (19개)

포트	방향	형/폭
clk	input	
trap	output	
mem_axi_awvalid	output	
mem_axi_awready	input	
mem_axi_awaddr	output	[31:0]
mem_axi_awprot	output	[2:0]
mem_axi_wvalid	output	
mem_axi_wready	input	
mem_axi_wdata	output	[31:0]
mem_axi_wstrb	output	[3:0]
mem_axi_bvalid	input	
mem_axi_bready	output	
mem_axi_arvalid	output	
mem_axi_arready	input	
mem_axi_araddr	output	[31:0]
mem_axi_arprot	output	[2:0]
mem_axi_rvalid	input	
mem_axi_rready	output	
mem_axi_rdata	input	[31:0]

API — axicheck2.v**Table 1131. YosysHQ_picorv32/scripts/smtbmc/axicheck2.v — module / interface / package (1개)**

이름	종류
testbench	module

Table 1132. YosysHQ_picorv32/scripts/smtbmc/axicheck2.v — 포트 (35개)

포트	방향	형/폭
clk	input	
resetsn	input	
trap_0	output	
trap_1	output	
mem_axi_awvalid_0	output	
mem_axi_awready_0	input	
mem_axi_awaddr_0	output	[31:0]
mem_axi_awprot_0	output	[2:0]
mem_axi_awvalid_1	output	
mem_axi_awready_1	input	
mem_axi_awaddr_1	output	[31:0]
mem_axi_awprot_1	output	[2:0]
mem_axi_wvalid_0	output	
mem_axi_wready_0	input	
mem_axi_wdata_0	output	[31:0]
mem_axi_wstrb_0	output	[3:0]
mem_axi_wvalid_1	output	
mem_axi_wready_1	input	
mem_axi_wdata_1	output	[31:0]
mem_axi_wstrb_1	output	[3:0]
mem_axi_bvalid	input	
mem_axi_bready_0	output	
mem_axi_bready_1	output	
mem_axi_arvalid_0	output	
mem_axi_arready_0	input	
mem_axi_araddr_0	output	[31:0]
mem_axi_arprot_0	output	[2:0]
mem_axi_arvalid_1	output	
mem_axi_arready_1	input	

포트	방향	형/폭
mem_axi_araddr_1	output	[31:0]
mem_axi_arprot_1	output	[2:0]
mem_axi_rvalid	input	
mem_axi_rready_0	output	
mem_axi_rready_1	output	
mem_axi_rdata	input	[31:0]

API — mulcmp.v

Table 1133. YosysHQ_picorv32/scripts/smtbmc/mulcmp.v — module / interface / package (1개)

이름	종류
testbench	module

API — notrap_validop.v

Table 1134. YosysHQ_picorv32/scripts/smtbmc/notrap_validop.v — module / interface / package (1개)

이름	종류
testbench	module

API — opcode.v

Table 1135. YosysHQ_picorv32/scripts/smtbmc/opcode.v — 포트 (8개)

포트	방향	형/폭
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]
opcode	input	[31:0]

API — tracecmp.v

Table 1136. YosysHQ_picorv32/scripts/smtbmc/tracecmp.v — module / interface / package (1개)

이름	종류
testbench	module

Table 1137. YosysHQ_picorv32/scripts/smtbmc/tracecmp.v — localparam (1개)

상수	형/폭	값
ENABLE_COUNTERS		0

API — tracecmp2.v**Table 1138. YosysHQ_picorv32/scripts/smtbmc/tracecmp2.v — module / interface / package (1개)**

이름	종류
testbench	module

Table 1139. YosysHQ_picorv32/scripts/smtbmc/tracecmp2.v — localparam (1개)

상수	형/폭	값
ENABLE_COUNTERS		0

Table 1140. YosysHQ_picorv32/scripts/smtbmc/tracecmp2.v — 포트 (2개)

포트	방향	형/폭
clk	input	
mem_rdata	input	[31:0]

API — tracecmp3.v**Table 1141. YosysHQ_picorv32/scripts/smtbmc/tracecmp3.v — module / interface / package (1개)**

이름	종류
testbench	module

Table 1142. YosysHQ_picorv32/scripts/smtbmc/tracecmp3.v — 포트 (6개)

포트	방향	형/폭
clk	input	
mem_rdata_in	input	[31:0]
pcpi_wr	input	
pcpi_rd	input	[31:0]
pcpi_wait	input	
pcpi_ready	input	

API — testbench.v**Table 1143. YosysHQ_picorv32/scripts/cxxdemo/testbench.v — module / interface / package (1개)**

이름	종류
testbench	module

Table 1144. YosysHQ_picorv32/scripts/cxxdemo/testbench.v — localparam (1개)

상수	형/폭	값
MEM_SIZE		4*1024*1024

1. YosysHQ_picorv32/picorv32.v (3,049 lines)

```

1 /*
2  * PicoRV32 -- A Small RISC-V (RV32I) Processor Core
3  *
4  * Copyright (C) 2015 Claire Xenia Wolf <claire@yosyshq.com>
5  *

```

```

6  * Permission to use, copy, modify, and/or distribute this
7  * software for any purpose with or without fee is hereby granted, provided
8  * that the above copyright notice and this permission notice appear in all
9  * copies.
10 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS

```

```

ALL WARRANTIES
11 * WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WAR-
RANTIES OF
12 * MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR
BE LIABLE FOR

13 * ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR
ANY DAMAGES
14 * WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN
15 * ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION,
ARISING OUT OF
16 * OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFT-
WARE.
17 *
18 */
19
20 /* verilator lint_off WIDTH */
21 /* verilator lint_off PINMISSING */
22 /* verilator lint_off CASEOVERLAP */
23 /* verilator lint_off CASEINCOMPLETE */
24
25 `timescale 1 ns / 1 ps
26 // `default_nettype none
27 // `define DEBUGNETS
28 // `define DEBUGREGS
29 // `define DEBUGASM
30 // `define DEBUG
31
32 `ifdef DEBUG
33 `define debug(debug_command) debug_command
34 `else
35 `define debug(debug_command)
36 `endif
37
38 `ifdef FORMAL
39 `define FORMAL_KEEP (* keep *)
40 `define assert(assert_expr) assert(assert_expr)
41 `else
42 `ifdef DEBUGNETS
43 `define FORMAL_KEEP (* keep *)
44 `else
45 `define FORMAL_KEEP
46 `endif
47 `define assert(assert_expr) empty_statement
48 `endif
49
50 // uncomment this for register file in extra module
51 // `define PICORV32_REGS picorv32_regs
52
53 // this macro can be used to check if the verilog files in
your
54 // design are read in the correct order.
55 `define PICORV32_V
56
57 /
58 /
*****
59 * picorv32
60
*****/
61
62 module picorv32 #(
63     parameter [0:0] ENABLE_COUNTERS = 1,
64     parameter [0:0] ENABLE_COUNTERS64 = 1,
65     parameter [0:0] ENABLE_REGS_16_31 = 1,
66     parameter [0:0] ENABLE_REGS_DUALPORT = 1,
67     parameter [0:0] LATCHED_MEM_RDATA = 0,
68     parameter [0:0] TWO_STAGE_SHIFT = 1,
69     parameter [0:0] BARREL_SHIFTER = 0,
70     parameter [0:0] TWO_CYCLE_COMPARE = 0,
71     parameter [0:0] TWO_CYCLE_ALU = 0,
72     parameter [0:0] COMPRESSED_ISA = 0,
73     parameter [0:0] CATCH_MISALIGN = 1,
74     parameter [0:0] CATCH_ILINSN = 1,
75     parameter [0:0] ENABLE_PCPPI = 0,
76     parameter [0:0] ENABLE_MUL = 0,
77     parameter [0:0] ENABLE_FAST_MUL = 0,
78     parameter [0:0] ENABLE_DIV = 0,
79     parameter [0:0] ENABLE_IRQ = 0,
80     parameter [0:0] ENABLE_IRQ_QREGS = 1,
81     parameter [0:0] ENABLE_IRQ_TIMER = 1,
82     parameter [0:0] ENABLE_TRACE = 0,
83     parameter [0:0] REGS_INIT_ZERO = 0,
84     parameter [31:0] MASKED_IRQ = 32'h 0000_0000,
85     parameter [31:0] LATCHED_IRQ = 32'h ffff_ffff,
86     parameter [31:0] PROGADDR_RESET = 32'h 0000_0000,
87     parameter [31:0] PROGADDR_IRQ = 32'h 0000_0010,
88     parameter [31:0] STACKADDR = 32'h ffff_ffff
89 ) (
90     input clk, resetn,
91     output reg trap,
92
93     output reg mem_valid,
94     output reg mem_instr,
95     input mem_ready,
96
97     output reg [31:0] mem_addr,
98     output reg [31:0] mem_wdata,
99     output reg [3:0] mem_wstrb,
100     input [31:0] mem_rdata,

```

```

101
102 // Look-Ahead Interface
103 output mem_la_read,
104 output mem_la_write,
105 output [31:0] mem_la_addr,
106 output reg [31:0] mem_la_wdata,
107 output reg [3:0] mem_la_wstrb,
108
109 // Pico Co-Processor Interface (PCPI)
110 output reg pcpi_valid,
111 output reg [31:0] pcpi_insn,
112 output [31:0] pcpi_rs1,
113 output [31:0] pcpi_rs2,
114 input pcpi_wr,
115 input [31:0] pcpi_rd,
116 input pcpi_wait,
117 input pcpi_ready,
118
119 // IRQ Interface
120 input [31:0] irq,
121 output reg [31:0] eoi,
122
123 `ifdef RISC_V_FORMAL
124     output reg rvfi_valid,
125     output reg [63:0] rvfi_order,
126     output reg [31:0] rvfi_insn,
127     output reg rvfi_trap,
128     output reg rvfi_halt,
129     output reg rvfi_intr,
130     output reg [1:0] rvfi_mode,
131     output reg [1:0] rvfi_ixl,
132     output reg [4:0] rvfi_rs1_addr,
133     output reg [4:0] rvfi_rs2_addr,
134     output reg [31:0] rvfi_rs1_rdata,
135     output reg [31:0] rvfi_rs2_rdata,
136     output reg [4:0] rvfi_rd_addr,
137     output reg [31:0] rvfi_rd_wdata,
138     output reg [31:0] rvfi_pc_rdata,
139     output reg [31:0] rvfi_pc_wdata,
140     output reg [31:0] rvfi_mem_addr,
141     output reg [3:0] rvfi_mem_rmask,
142     output reg [3:0] rvfi_mem_wmask,
143     output reg [31:0] rvfi_mem_rdata,
144     output reg [31:0] rvfi_mem_wdata,
145
146     output reg [63:0] rvfi_csr_mcycle_rmask,
147     output reg [63:0] rvfi_csr_mcycle_wmask,
148     output reg [63:0] rvfi_csr_mcycle_rdata,
149     output reg [63:0] rvfi_csr_mcycle_wdata,
150
151     output reg [63:0] rvfi_csr_minstret_rmask,
152     output reg [63:0] rvfi_csr_minstret_wmask,
153     output reg [63:0] rvfi_csr_minstret_rdata,
154     output reg [63:0] rvfi_csr_minstret_wdata,
155 `endif
156
157 // Trace Interface
158 output reg trace_valid,
159 output reg [35:0] trace_data
160 );
161
162 localparam integer irq_timer = 0;
163 localparam integer irq_ebreak = 1;
164 localparam integer irq_buserror = 2;
165
166 localparam integer irqregs_offset = ENABLE_REGS_16_31 ?
32 : 16;
167
168 localparam integer regfile_size = (ENABLE_REGS_16_31 ?
32 : 16) + 4*ENABLE_IRQ*ENABLE_IRQ_QREGS;
169
170 localparam integer regindex_bits = (ENABLE_REGS_16_31 ?
5 : 4) + ENABLE_IRQ*ENABLE_IRQ_QREGS;
171
172 localparam WITH_PCPPI = ENABLE_PCPPI || ENABLE_MUL || EN-
ABLE_FAST_MUL || ENABLE_DIV;
173
174 localparam [35:0] TRACE_BRANCH = {4'b 0001, 32'b 0};
175 localparam [35:0] TRACE_ADDR = {4'b 0010, 32'b 0};
176 localparam [35:0] TRACE_IRQ = {4'b 1000, 32'b 0};
177
178 reg [63:0] count_cycle, count_instr;
179 reg [31:0] reg_pc, reg_next_pc, reg_op1, reg_op2,
reg_out;
180 reg [4:0] reg_sh;
181
182 reg [31:0] next_insn_opcode;
183 reg [31:0] dbg_insn_opcode;
184 reg [31:0] dbg_insn_addr;
185
186 wire dbg_mem_valid = mem_valid;
187 wire dbg_mem_instr = mem_instr;
188 wire dbg_mem_ready = mem_ready;
189 wire [31:0] dbg_mem_addr = mem_addr;
190 wire [31:0] dbg_mem_wdata = mem_wdata;
191 wire [3:0] dbg_mem_wstrb = mem_wstrb;
192 wire [31:0] dbg_mem_rdata = mem_rdata;
193
194 assign pcpi_rs1 = reg_op1;
195 assign pcpi_rs2 = reg_op2;
196
197 wire [31:0] next_pc;

```

```

195
196     reg irq_delay;
197     reg irq_active;
198     reg [31:0] irq_mask;
199     reg [31:0] irq_pending;
200     reg [31:0] timer;
201
202 `ifndef PICORV32_REGS
203     reg [31:0] cpuregs [0:regfile_size-1];
204
205     integer i;
206     initial begin
207         if (REGS_INIT_ZERO) begin
208             for (i = 0; i < regfile_size; i = i+1)
209                 cpuregs[i] = 0;
210         end
211     end
212 `endif
213
214     task empty_statement;
215         // This task is used by the `assert directive
216         // avoid empty statement (which are unsupported
217         // by plain Verilog syntax).
218     endtask
219
220 `ifndef DEBUGREGS
221     wire [31:0] dbg_reg_x0 = 0;
222     wire [31:0] dbg_reg_x1 = cpuregs[1];
223     wire [31:0] dbg_reg_x2 = cpuregs[2];
224     wire [31:0] dbg_reg_x3 = cpuregs[3];
225     wire [31:0] dbg_reg_x4 = cpuregs[4];
226     wire [31:0] dbg_reg_x5 = cpuregs[5];
227     wire [31:0] dbg_reg_x6 = cpuregs[6];
228     wire [31:0] dbg_reg_x7 = cpuregs[7];
229     wire [31:0] dbg_reg_x8 = cpuregs[8];
230     wire [31:0] dbg_reg_x9 = cpuregs[9];
231     wire [31:0] dbg_reg_x10 = cpuregs[10];
232     wire [31:0] dbg_reg_x11 = cpuregs[11];
233     wire [31:0] dbg_reg_x12 = cpuregs[12];
234     wire [31:0] dbg_reg_x13 = cpuregs[13];
235     wire [31:0] dbg_reg_x14 = cpuregs[14];
236     wire [31:0] dbg_reg_x15 = cpuregs[15];
237     wire [31:0] dbg_reg_x16 = cpuregs[16];
238     wire [31:0] dbg_reg_x17 = cpuregs[17];
239     wire [31:0] dbg_reg_x18 = cpuregs[18];
240     wire [31:0] dbg_reg_x19 = cpuregs[19];
241     wire [31:0] dbg_reg_x20 = cpuregs[20];
242     wire [31:0] dbg_reg_x21 = cpuregs[21];
243     wire [31:0] dbg_reg_x22 = cpuregs[22];
244     wire [31:0] dbg_reg_x23 = cpuregs[23];
245     wire [31:0] dbg_reg_x24 = cpuregs[24];
246     wire [31:0] dbg_reg_x25 = cpuregs[25];
247     wire [31:0] dbg_reg_x26 = cpuregs[26];
248     wire [31:0] dbg_reg_x27 = cpuregs[27];
249     wire [31:0] dbg_reg_x28 = cpuregs[28];
250     wire [31:0] dbg_reg_x29 = cpuregs[29];
251     wire [31:0] dbg_reg_x30 = cpuregs[30];
252     wire [31:0] dbg_reg_x31 = cpuregs[31];
253 `endif
254
255     // Internal PCPI Cores
256
257     wire pcpi_mul_wr;
258     wire [31:0] pcpi_mul_rd;
259     wire pcpi_mul_wait;
260     wire pcpi_mul_ready;
261
262     wire pcpi_div_wr;
263     wire [31:0] pcpi_div_rd;
264     wire pcpi_div_wait;
265     wire pcpi_div_ready;
266
267     reg pcpi_int_wr;
268     reg [31:0] pcpi_int_rd;
269     reg pcpi_int_wait;
270     reg pcpi_int_ready;
271
272     generate if (ENABLE_FAST_MUL) begin
273         picorv32_pcpi_fast_mul pcpi_mul (
274             .clk (clk),
275             .resetn (resetn),
276             .pcpi_valid (pcpi_valid),
277             .pcpi_insn (pcpi_insn),
278             .pcpi_rs1 (pcpi_rs1),
279             .pcpi_rs2 (pcpi_rs2),
280             .pcpi_wr (pcpi_mul_wr),
281             .pcpi_rd (pcpi_mul_rd),
282             .pcpi_wait (pcpi_mul_wait),
283             .pcpi_ready (pcpi_mul_ready)
284         );
285     end else if (ENABLE_MUL) begin
286         picorv32_pcpi_mul pcpi_mul (
287             .clk (clk),
288             .resetn (resetn),
289             .pcpi_valid (pcpi_valid),
290             .pcpi_insn (pcpi_insn),
291             .pcpi_rs1 (pcpi_rs1),
292             .pcpi_rs2 (pcpi_rs2),

```

```

293             .pcpi_wr (pcpi_mul_wr),
294             .pcpi_rd (pcpi_mul_rd),
295             .pcpi_wait (pcpi_mul_wait),
296             .pcpi_ready (pcpi_mul_ready)
297         );
298     end else begin
299         assign pcpi_mul_wr = 0;
300         assign pcpi_mul_rd = 32'bx;
301         assign pcpi_mul_wait = 0;
302         assign pcpi_mul_ready = 0;
303     end endgenerate
304
305     generate if (ENABLE_DIV) begin
306         picorv32_pcpi_div pcpi_div (
307             .clk (clk),
308             .resetn (resetn),
309             .pcpi_valid (pcpi_valid),
310             .pcpi_insn (pcpi_insn),
311             .pcpi_rs1 (pcpi_rs1),
312             .pcpi_rs2 (pcpi_rs2),
313             .pcpi_wr (pcpi_div_wr),
314             .pcpi_rd (pcpi_div_rd),
315             .pcpi_wait (pcpi_div_wait),
316             .pcpi_ready (pcpi_div_ready)
317         );
318     end else begin
319         assign pcpi_div_wr = 0;
320         assign pcpi_div_rd = 32'bx;
321         assign pcpi_div_wait = 0;
322         assign pcpi_div_ready = 0;
323     end endgenerate
324
325     always @* begin
326         pcpi_int_wr = 0;
327         pcpi_int_rd = 32'bx;
328         pcpi_int_wait = |(ENABLE_PCPI && pcpi_wait,
329             (ENABLE_MUL || ENABLE_FAST_MUL) && pcpi_mul_wait, ENABLE_DIV &&
330             pcpi_div_wait);
331         pcpi_int_ready = |(ENABLE_PCPI && pcpi_ready,
332             (ENABLE_MUL || ENABLE_FAST_MUL) && pcpi_mul_ready, ENABLE_DIV &&
333             pcpi_div_ready);
334
335         (* parallel_case *)
336         case (1'b1)
337             ENABLE_PCPI && pcpi_ready: begin
338                 pcpi_int_wr = ENABLE_PCPI ?
339                 pcpi_int_rd = ENABLE_PCPI ?
340             end
341             (ENABLE_MUL || ENABLE_FAST_MUL) &&
342             pcpi_mul_ready: begin
343                 pcpi_int_wr = pcpi_mul_wr;
344                 pcpi_int_rd = pcpi_mul_rd;
345             end
346             ENABLE_DIV && pcpi_div_ready: begin
347                 pcpi_int_wr = pcpi_div_wr;
348                 pcpi_int_rd = pcpi_div_rd;
349             end
350         endcase
351     end
352
353     // Memory Interface
354
355     reg [1:0] mem_state;
356     reg [1:0] mem_wordsize;
357     reg [31:0] mem_rdata_word;
358     reg [31:0] mem_rdata_q;
359     reg mem_do_prefetch;
360     reg mem_do_rinst;
361     reg mem_do_rdata;
362     reg mem_do_wdata;
363
364     wire mem_xfer;
365     reg mem_la_secondword, mem_la_firstword_reg,
366     last_mem_valid;
367     wire mem_la_firstword = COMPRESSED_ISA &&
368     (mem_do_prefetch || mem_do_rinst) && next_pc[1] && !mem_la_second-
369     word;
370
371     wire mem_la_firstword_xfer = COMPRESSED_ISA && mem_xfer
372     && (!last_mem_valid ? mem_la_firstword : mem_la_firstword_reg);
373
374     reg prefetched_high_word;
375     reg clear_prefetched_high_word;
376     reg [15:0] mem_16bit_buffer;
377
378     wire [31:0] mem_rdata_latched_noshuffle;
379     wire [31:0] mem_rdata_latched;
380
381     wire mem_la_use_prefetched_high_word = COMPRESSED_ISA
382     && mem_la_firstword && prefetched_high_word && !
383     clear_prefetched_high_word;
384     assign mem_xfer = (mem_valid && mem_ready) ||
385     (mem_la_use_prefetched_high_word && mem_do_rinst);
386
387     wire mem_busy = |(mem_do_prefetch, mem_do_rinst,
388     mem_do_rdata, mem_do_wdata);
389     wire mem_done = resetn && ((mem_xfer && !mem_state &&
390     (mem_do_rinst || mem_do_rdata || mem_do_wdata)) || (&mem_state &&

```

```

mem_do_rinst)) &&
377      (!mem_la_firstword ||
378      (~mem_rdata_latched[1:0] && mem_xfer));
379      assign mem_la_write = resethn && !mem_state &&
mem_do_wdata;
380      assign mem_la_read = resethn && ((!
mem_la_use_prefetched_high_word && !mem_state && (mem_do_rinst ||
mem_do_prefetch || mem_do_rdata)) ||
381      (COMPRESSED_ISA && mem_xfer && (!
last_mem_valid ? mem_la_firstword : mem_la_firstword_reg) && !
mem_la_secondword && mem_rdata_latched[1:0]));
382      assign mem_la_addr = (mem_do_prefetch ||
mem_do_rinst) ? {next_pc[31:2] + mem_la_firstword_xfer, 2'b00} :
{reg_op1[31:2], 2'b00};
383
384      assign mem_rdata_latched_noshuffle = (mem_xfer ||
LATCHED_MEM_RDATA) ? mem_rdata : mem_rdata_q;
385
386      assign mem_rdata_latched = COMPRESSED_ISA &&
mem_la_use_prefetched_high_word ? {16'bx, mem_16bit_buffer} :
387      COMPRESSED_ISA && mem_la_secondword ?
{mem_rdata_latched_noshuffle[15:0], mem_16bit_buffer} :
388      COMPRESSED_ISA && mem_la_firstword ?
{16'bx, mem_rdata_latched_noshuffle[31:16]} :
mem_rdata_latched_noshuffle;
389
390      always @(posedge clk) begin
391          if (!resethn) begin
392              mem_la_firstword_reg <= 0;
393              last_mem_valid <= 0;
394          end else begin
395              if (!last_mem_valid)
396                  mem_la_firstword_reg <=
mem_la_firstword;
397              last_mem_valid <= mem_valid && !
mem_ready;
398          end
399      end
400
401      always @* begin
402          (* full_case *)
403          case (mem_wordsize)
404              0: begin
405                  mem_la_wdata = reg_op2;
406                  mem_la_wstrb = 4'b1111;
407                  mem_rdata_word = mem_rdata;
408              end
409              1: begin
410                  mem_la_wdata =
{2{reg_op2[15:0]}};
411                  mem_la_wstrb = reg_op1[1] ?
4'b1100 : 4'b0011;
412                  case (reg_op1[1])
413                      1'b0: mem_rdata_word =
{16'b0, mem_rdata[15: 0]};
414                      1'b1: mem_rdata_word =
{16'b0, mem_rdata[31:16]};
415                  endcase
416              end
417              2: begin
418                  mem_la_wdata =
{4{reg_op2[7:0]}};
419                  mem_la_wstrb = 4'b0001 <<
reg_op1[1:0];
420                  case (reg_op1[1:0])
421                      2'b00: mem_rdata_word =
{24'b0, mem_rdata[ 7: 0]};
422                      2'b01: mem_rdata_word =
{24'b0, mem_rdata[15: 8]};
423                      2'b10: mem_rdata_word =
{24'b0, mem_rdata[23:16]};
424                      2'b11: mem_rdata_word =
{24'b0, mem_rdata[31:24]};
425                  endcase
426              end
427          endcase
428      end
429
430      always @(posedge clk) begin
431          if (mem_xfer) begin
432              mem_rdata_q <= COMPRESSED_ISA ?
mem_rdata_latched : mem_rdata;
433              next_insn_opcode <= COMPRESSED_ISA ?
mem_rdata_latched : mem_rdata;
434          end
435
436          if (COMPRESSED_ISA && mem_done &&
(mem_do_prefetch || mem_do_rinst)) begin
437              case (mem_rdata_latched[1:0])
438                  2'b00: begin // Quadrant 0
439                      (mem_rdata_latched[15:13])
440                      3'b000: be-
gin // C.ADDI4SPN
441                      mem_rdata_q[14:12] <= 3'b000;

```

```

442
mem_rdata_q[31:20] <= {2'b0, mem_rdata_latched[10:7],
mem_rdata_latched[12:11], mem_rdata_latched[5],
mem_rdata_latched[6], 2'b00};
443
444      end
445      3'b010: be-
gin // C.LW
446
mem_rdata_q[31:20] <= {5'b0, mem_rdata_latched[5],
mem_rdata_latched[12:10], mem_rdata_latched[6], 2'b00};
447
448      end
449      3'b 110: be-
gin // C.SW
450
{mem_rdata_q[31:25], mem_rdata_q[11:7]} <= {5'b0,
mem_rdata_latched[5], mem_rdata_latched[12:10],
mem_rdata_latched[6], 2'b00};
451
452      end
453      endcase
454      2'b01: begin // Quadrant 1
455          case
(mem_rdata_latched[15:13])
456          3'b 000: be-
gin // C.ADDI
457
mem_rdata_q[14:12] <= 3'b000;
458
mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[6:2]});
459
460      end
461      3'b 010: be-
gin // C.LI
462
mem_rdata_q[14:12] <= 3'b000;
463
mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[6:2]});
464
465      end
466      3'b 011: begin
467          if
(mem_rdata_latched[11:7] == 2) begin // C.ADDI16SP
468
mem_rdata_q[14:12] <= 3'b000;
469
mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[4:3],
470
mem_rdata_latched[5], mem_rdata_latched[2], mem_rdata_latched[6],
4'b 0000});
471
472      end
473      else begin // C.LUI
474
mem_rdata_q[31:12] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[6:2]});
475
476      end
477      3'b100: begin
478          if
(mem_rdata_latched[11:10] == 2'b00) begin // C.SRLI
479
mem_rdata_q[31:25] <= 7'b00000000;
480
mem_rdata_q[14:12] <= 3'b 101;
481
482      end
483      (mem_rdata_latched[11:10] == 2'b01) begin // C.SRAI
484
mem_rdata_q[31:25] <= 7'b01000000;
485
mem_rdata_q[14:12] <= 3'b 101;
486
487      end
488      (mem_rdata_latched[11:10] == 2'b10) begin // C.ANDI
489
mem_rdata_q[14:12] <= 3'b111;
490
491      end
492      mem_rdata_q[31:20] <= $signed({mem_rdata_latched[12],
mem_rdata_latched[6:2]});
493
494      end
495      if
(mem_rdata_latched[12:10] == 3'b011) begin // C.SUB, C.XOR, C.OR,
C.AND
496
497      end
498      if
(mem_rdata_latched[6:5] == 2'b00) mem_rdata_q[14:12] <= 3'b000;

```



```

488 if (mem_rdata_latched[6:5] == 2'b01) mem_rdata_q[14:12] <= 3'b100;
489 if (mem_rdata_latched[6:5] == 2'b10) mem_rdata_q[14:12] <= 3'b110;
490 if (mem_rdata_latched[6:5] == 2'b11) mem_rdata_q[14:12] <= 3'b111;
491 mem_rdata_q[31:25] <= mem_rdata_latched[6:5] == 2'b00 ? 7'b01000000 :
7'b00000000;
492 end
493 end
494 3'b 110: be-
gin // C.BEQZ
495 mem_rdata_q[14:12] <= 3'b000;
496 { mem_rdata_q[31], mem_rdata_q[7], mem_rdata_q[30:25],
mem_rdata_q[11:8] } <=
497 $signed({mem_rdata_latched[12], mem_rdata_latched[6:5],
mem_rdata_latched[2],
498 mem_rdata_latched[11:10], mem_rdata_latched[4:3]});
499 end
500 3'b 111: be-
gin // C.BNEZ
501 mem_rdata_q[14:12] <= 3'b001;
502 { mem_rdata_q[31], mem_rdata_q[7], mem_rdata_q[30:25],
mem_rdata_q[11:8] } <=
503 $signed({mem_rdata_latched[12], mem_rdata_latched[6:5],
mem_rdata_latched[2],
504 mem_rdata_latched[11:10], mem_rdata_latched[4:3]});
505 end
506 endcase
507 end
508 2'b10: begin // Quadrant 2
509 case
(mem_rdata_latched[15:13])
510 3'b000: be-
gin // C.SLLI
511 mem_rdata_q[31:25] <= 7'b00000000;
512 mem_rdata_q[14:12] <= 3'b 001;
513 end
514 3'b010: be-
gin // C.LWSP
515 mem_rdata_q[31:20] <= {4'b0, mem_rdata_latched[3:2],
mem_rdata_latched[12], mem_rdata_latched[6:4], 2'b00};
516 mem_rdata_q[14:12] <= 3'b 010;
517 end
518 3'b100: begin
519 if
(mem_rdata_latched[12] == 0 && mem_rdata_latched[6:2] == 0) begin //
C.JR
520 mem_rdata_q[14:12] <= 3'b000;
521 mem_rdata_q[31:20] <= 12'b0;
522 end
523 if
(mem_rdata_latched[12] == 0 && mem_rdata_latched[6:2] != 0) begin //
C.MV
524 mem_rdata_q[14:12] <= 3'b000;
525 mem_rdata_q[31:25] <= 7'b00000000;
526 end
527 if
(mem_rdata_latched[12] != 0 && mem_rdata_latched[11:7] != 0 &&
mem_rdata_latched[6:2] == 0) begin // C.JALR
528 mem_rdata_q[14:12] <= 3'b000;
529 mem_rdata_q[31:20] <= 12'b0;
530 end
531 if
(mem_rdata_latched[12] != 0 && mem_rdata_latched[6:2] != 0) begin //
C.ADD
532 mem_rdata_q[14:12] <= 3'b000;

```

```

533 mem_rdata_q[31:25] <= 7'b00000000;
534 end
535 end
536 3'b110: be-
gin // C.SWSP
537 {mem_rdata_q[31:25], mem_rdata_q[11:7]} <= {4'b0,
mem_rdata_latched[8:7], mem_rdata_latched[12:9], 2'b00};
538 mem_rdata_q[14:12] <= 3'b 010;
539 end
540 endcase
541 end
542 endcase
543 end
544 end
545 always @(posedge clk) begin
546 if (resetsn && !trap) begin
547 if (mem_do_prefetch || mem_do_rinst ||
mem_do_rdata)
548 `assert(!mem_do_wdata);
549 if (mem_do_prefetch || mem_do_rinst)
550 `assert(!mem_do_rdata);
551 if (mem_do_rdata)
552 `assert(!mem_do_prefetch && !
mem_do_rinst);
553 if (mem_do_wdata)
554 `assert(!(mem_do_prefetch ||
mem_do_rinst || mem_do_rdata));
555 if (mem_state == 2 || mem_state == 3)
556 `assert(mem_valid ||
mem_do_prefetch);
557 end
558 end
559 always @(posedge clk) begin
560 if (!resetsn || trap) begin
561 if (!resetsn)
562 mem_state <= 0;
563 if (!resetsn || mem_ready)
564 mem_valid <= 0;
565 mem_la_secondword <= 0;
566 prefetched_high_word <= 0;
567 end else begin
568 if (mem_la_read || mem_la_write) begin
569 mem_addr <= mem_la_addr;
570 mem_wstrb <= mem_la_wstrb &
{4{mem_la_write}};
571 end
572 if (mem_la_write) begin
573 mem_wdata <= mem_la_wdata;
574 end
575 case (mem_state)
576 0: begin
577 if (mem_do_prefetch ||
mem_do_rinst || mem_do_rdata) begin
578 mem_valid <= !
579 mem_la_use_prefetched_high_word;
580 mem_instr <=
581 mem_do_prefetch || mem_do_rinst;
582 mem_wstrb <= 0;
583 mem_state <= 1;
584 end
585 if (mem_do_wdata) begin
586 mem_valid <= 1;
587 mem_instr <= 0;
588 mem_state <= 2;
589 end
590 end
591 end
592 1: begin
593 `assert(mem_wstrb ==
0);
594 `assert(mem_do_prefetch
|| mem_do_rinst || mem_do_rdata);
595 `assert(mem_valid == !
596 mem_la_use_prefetched_high_word);
597 `assert(mem_instr ==
(mem_do_prefetch || mem_do_rinst));
598 if (mem_xfer) begin
599 if (COM-
PRESSED_ISA && mem_la_read) begin
600 mem_valid <= 1;
601 mem_la_secondword <= 1;
602 mem_la_use_prefetched_high_word)
603 if (!
604 mem_16bit_buffer <= mem_rdata[31:16];
605 end else begin
606 mem_valid <= 0;

```

```

608 mem_la_secondword <= 0;
609 (COMPRESSED_ISA && !mem_do_rdata) begin if
610 if (~&mem_rdata[1:0] || mem_la_secondword) begin
611 mem_16bit_buffer <= mem_rdata[31:16];
612 prefetched_high_word <= 1;
613 end else begin
614 prefetched_high_word <= 0;
615 end
616 end
617 mem_state <= mem_do_rinst || mem_do_rdata ? 0 : 3;
618 end
619 end
620 end
621 2: begin
622 `assert(mem_wstrb !=
0);
623 `assert(mem_do_wdata);
624 if (mem_xfer) begin
625 mem_valid <= 0;
626 mem_state <= 0;
627 end
628 end
629 3: begin
630 `assert(mem_wstrb ==
0);
631 `as-
sert(mem_do_prefetch);
632 if (mem_do_rinst) begin
633 mem_state <= 0;
634 end
635 end
636 endcase
637 end
638 if (clear_prefetched_high_word)
639 prefetched_high_word <= 0;
640 end
641 // Instruction Decoder
642 reg instr_lui, instr_auipc, instr_jal, instr_jalr;
643 reg instr_beq, instr_bne, instr_blt, instr_bge, in-
str_bltu, instr_bgeu;
644 reg instr_lb, instr_lh, instr_lw, instr_lbu, instr_lhu,
instr_sb, instr_sh, instr_sw;
645 reg instr_addi, instr_slti, instr_sltiu, instr_xori,
instr_ori, instr_andi, instr_slli, instr_srli, instr_srai;
646 reg instr_add, instr_sub, instr_sll, instr_slt, in-
str_sltu, instr_xor, instr_srl, instr_sra, instr_or, instr_and;
647 reg instr_rdcycle, instr_rdcycleh, instr_rdinstr, in-
str_rdinstrh, instr_ecall, instr_fence;
648 reg instr_getq, instr_setq, instr_retirq, in-
str_maskirq, instr_waitirq, instr_timer;
649 wire instr_trap;
650 reg [regindex_bits-1:0] decoded_rd, decoded_rs1;
651 reg [4:0] decoded_rs2;
652 reg [31:0] decoded_imm, decoded_imm_j;
653 reg decoder_trigger;
654 reg decoder_trigger_q;
655 reg decoder_pseudo_trigger;
656 reg decoder_pseudo_trigger_q;
657 reg compressed_instr;
658 reg is_lui_auipc_jal;
659 reg is_lb_lh_lw_lbu_lhu;
660 reg is_slli_srli_srai;
661 reg is_jalr_addi_slti_sltiu_xori_ori_andi;
662 reg is_sb_sh_sw;
663 reg is_sll_srl_sra;
664 reg is_lui_auipc_jal_jalr_addi_add_sub;
665 reg is_slti_blt_slt;
666 reg is_sltiu_bltu_sltu;
667 reg is_beq_bne_blt_bge_bltu_bgeu;
668 reg is_lbu_lhu_lw;
669 reg is_alu_reg_imm;
670 reg is_alu_reg_reg;
671 reg is_compare;
672 assign instr_trap = (CATCH_ILLINSN || WITH_PCPI) && !
{instr_lui, instr_auipc, instr_jal, instr_jalr,
673 instr_beq, instr_bne, instr_blt, in-
str_bge, instr_bltu, instr_bgeu,
674 instr_lb, instr_lh, instr_lw, in-

```

```

str_lbu, instr_lhu, instr_sb, instr_sh, instr_sw,
675 instr_xori, instr_ori, instr_andi, instr_slli, instr_srli, in-
str_srai,
676 instr_add, instr_sub, instr_sll, in-
str_slt, instr_sltu, instr_xor, instr_srl, instr_sra, instr_or, in-
str_and,
677 instr_rdcycle, instr_rdcycleh, in-
str_rdinstr, instr_rdinstrh, instr_fence,
678 instr_getq, instr_setq, instr_retirq,
instr_maskirq, instr_waitirq, instr_timer};
679 wire is_rdcycle_rdcycleh_rdinstr_rdinstrh;
680 assign is_rdcycle_rdcycleh_rdinstr_rdinstrh = |{in-
str_rdcycle, instr_rdcycleh, instr_rdinstr, instr_rdinstrh};
681 reg [63:0] new_ascii_instr;
682 `FORMAL_KEEP reg [63:0] dbg_ascii_instr;
683 `FORMAL_KEEP reg [31:0] dbg_insn_imm;
684 `FORMAL_KEEP reg [4:0] dbg_insn_rs1;
685 `FORMAL_KEEP reg [4:0] dbg_insn_rs2;
686 `FORMAL_KEEP reg [4:0] dbg_insn_rd;
687 `FORMAL_KEEP reg [31:0] dbg_rs1val;
688 `FORMAL_KEEP reg [31:0] dbg_rs2val;
689 `FORMAL_KEEP reg dbg_rs1val_valid;
690 `FORMAL_KEEP reg dbg_rs2val_valid;
691 always @* begin
692 new_ascii_instr = "";
693 if (instr_lui) new_ascii_instr = "lui";
694 if (instr_auipc) new_ascii_instr = "auipc";
695 if (instr_jal) new_ascii_instr = "jal";
696 if (instr_jalr) new_ascii_instr = "jalr";
697 if (instr_beq) new_ascii_instr = "beq";
698 if (instr_bne) new_ascii_instr = "bne";
699 if (instr_blt) new_ascii_instr = "blt";
700 if (instr_bge) new_ascii_instr = "bge";
701 if (instr_bltu) new_ascii_instr = "bltu";
702 if (instr_bgeu) new_ascii_instr = "bgeu";
703 if (instr_lb) new_ascii_instr = "lb";
704 if (instr_lh) new_ascii_instr = "lh";
705 if (instr_lw) new_ascii_instr = "lw";
706 if (instr_lbu) new_ascii_instr = "lbu";
707 if (instr_lhu) new_ascii_instr = "lhu";
708 if (instr_sb) new_ascii_instr = "sb";
709 if (instr_sh) new_ascii_instr = "sh";
710 if (instr_sw) new_ascii_instr = "sw";
711 if (instr_addi) new_ascii_instr = "addi";
712 if (instr_slti) new_ascii_instr = "slti";
713 if (instr_sltiu) new_ascii_instr = "sltiu";
714 if (instr_xori) new_ascii_instr = "xori";
715 if (instr_ori) new_ascii_instr = "ori";
716 if (instr_andi) new_ascii_instr = "andi";
717 if (instr_slli) new_ascii_instr = "slli";
718 if (instr_srli) new_ascii_instr = "srli";
719 if (instr_srai) new_ascii_instr = "srai";
720 if (instr_add) new_ascii_instr = "add";
721 if (instr_sub) new_ascii_instr = "sub";
722 if (instr_sll) new_ascii_instr = "sll";
723 if (instr_slt) new_ascii_instr = "slt";
724 if (instr_sltu) new_ascii_instr = "sltu";
725 if (instr_xor) new_ascii_instr = "xor";
726 if (instr_srl) new_ascii_instr = "srl";
727 if (instr_sra) new_ascii_instr = "sra";
728 if (instr_or) new_ascii_instr = "or";
729 if (instr_and) new_ascii_instr = "and";
730 if (instr_rdcycle) new_ascii_instr = "rd-
cycle";
731 if (instr_rdcycleh) new_ascii_instr = "rd-
cycleh";
732 if (instr_rdinstr) new_ascii_instr = "rdin-
str";
733 if (instr_rdinstrh) new_ascii_instr = "rdin-
strh";
734 if (instr_fence) new_ascii_instr = "fence";
735 if (instr_getq) new_ascii_instr = "getq";
736 if (instr_setq) new_ascii_instr = "setq";
737 if (instr_retirq) new_ascii_instr = "retirq";
738 if (instr_maskirq) new_ascii_instr =
"maskirq";
739 if (instr_waitirq) new_ascii_instr =
"waitirq";
740 if (instr_timer) new_ascii_instr = "timer";
741 end
742 reg [63:0] q_ascii_instr;
743 reg [31:0] q_insn_imm;
744 reg [31:0] q_insn_opcode;
745 reg [4:0] q_insn_rs1;
746 reg [4:0] q_insn_rs2;
747 reg [4:0] q_insn_rd;
748 reg dbg_next;
749 wire launch_next_insn;

```

```

769     reg dbg_valid_insn;
770
771     reg [63:0] cached_ascii_instr;
772     reg [31:0] cached_insn_imm;
773     reg [31:0] cached_insn_opcode;
774     reg [4:0] cached_insn_rs1;
775     reg [4:0] cached_insn_rs2;
776     reg [4:0] cached_insn_rd;
777
778     always @(posedge clk) begin
779         q_ascii_instr <= dbg_ascii_instr;
780         q_insn_imm <= dbg_insn_imm;
781         q_insn_opcode <= dbg_insn_opcode;
782         q_insn_rs1 <= dbg_insn_rs1;
783         q_insn_rs2 <= dbg_insn_rs2;
784         q_insn_rd <= dbg_insn_rd;
785         dbg_next <= launch_next_insn;
786
787         if (!resetn || trap)
788             dbg_valid_insn <= 0;
789         else if (launch_next_insn)
790             dbg_valid_insn <= 1;
791
792         if (decoder_trigger_q) begin
793             cached_ascii_instr <= new_ascii_instr;
794             cached_insn_imm <= decoded_imm;
795             if (&next_insn_opcode[1:0])
796                 cached_insn_opcode <= next_in-
sn_opcode;
797             else
798                 cached_insn_opcode <= {16'b0,
next_insn_opcode[15:0]};
799             cached_insn_rs1 <= decoded_rs1;
800             cached_insn_rs2 <= decoded_rs2;
801             cached_insn_rd <= decoded_rd;
802         end
803
804         if (launch_next_insn) begin
805             dbg_insn_addr <= next_pc;
806         end
807     end
808
809     always @* begin
810         dbg_ascii_instr = q_ascii_instr;
811         dbg_insn_imm = q_insn_imm;
812         dbg_insn_opcode = q_insn_opcode;
813         dbg_insn_rs1 = q_insn_rs1;
814         dbg_insn_rs2 = q_insn_rs2;
815         dbg_insn_rd = q_insn_rd;
816
817         if (dbg_next) begin
818             if (decoder_pseudo_trigger_q) begin
819                 dbg_ascii_instr =
cached_ascii_instr;
820                 dbg_insn_imm = cached_insn_imm;
821                 dbg_insn_opcode =
cached_insn_opcode;
822                 dbg_insn_rs1 = cached_insn_rs1;
823                 dbg_insn_rs2 = cached_insn_rs2;
824                 dbg_insn_rd = cached_insn_rd;
825             end else begin
826                 dbg_ascii_instr = new_as-
cii_instr;
827                 if (&next_insn_opcode[1:0])
828                     dbg_insn_opcode =
next_insn_opcode;
829                 else
830                     dbg_insn_opcode =
{16'b0, next_insn_opcode[15:0]};
831                 dbg_insn_imm = decoded_imm;
832                 dbg_insn_rs1 = decoded_rs1;
833                 dbg_insn_rs2 = decoded_rs2;
834                 dbg_insn_rd = decoded_rd;
835             end
836         end
837     end
838
839     `ifdef DEBUGASM
840     always @(posedge clk) begin
841         if (dbg_next) begin
842             $display("debugasm %x %x %s", dbg_in-
sn_addr, dbg_insn_opcode, dbg_ascii_instr ? dbg_as-
cii_instr : "UNKNOWN");
843         end
844     end
845     `endif
846
847     `ifdef DEBUG
848     always @(posedge clk) begin
849         if (dbg_next) begin
850             if (&dbg_insn_opcode[1:0])
851                 $display("DECODE: 0x%08x 0x%08x
%-0s", dbg_insn_addr, dbg_insn_opcode, dbg_ascii_in-
str ? dbg_as-
cii_instr : "UNKNOWN");
852             else
853                 $display("DECODE: 0x%08x
0x%04x %-0s", dbg_insn_addr, dbg_insn_opcode[15:0], dbg_as-
cii_instr ? dbg_ascii_instr : "UNKNOWN");
854             end
855         end
856     `endif

```

```

857
858     always @(posedge clk) begin
859         is_lui_auipc_jal <= |{instr_lui, instr_auipc,
instr_jal};
860         is_lui_auipc_jal_jalr_addi_add_sub <= |{in-
str_lui, instr_auipc, instr_jalr, instr_addi, instr_add,
instr_sub};
861         is_slti_blt_slt <= |{instr_slti, instr_blt, in-
str_slt};
862         is_sltiu_bltu_sltu <= |{instr_sltiu, in-
str_bltu, instr_sltu};
863         is_lbu_lhu_lw <= |{instr_lbu, instr_lhu, in-
str_lw};
864         is_compare <= |{is_beq_bne_blt_bge_bltu_bgeu,
instr_slti, instr_slt, instr_sltiu, instr_sltu};
865
866         if (mem_do_rinst && mem_done) begin
867             instr_lui <= mem_rdata_latched[6:0]
== 7'b0110111;
868             instr_auipc <= mem_rdata_latched[6:0]
== 7'b0010111;
869             instr_jal <= mem_rdata_latched[6:0]
== 7'b1101111;
870             instr_jalr <= mem_rdata_latched[6:0]
== 7'b1100111 && mem_rdata_latched[14:12] == 3'b000;
871             instr_retirq <= mem_rdata_latched[6:0]
== 7'b0001011 && mem_rdata_latched[31:25] == 7'b0000010 && EN-
ABLE_IRQ;
872             instr_waitirq <= mem_rdata_latched[6:0]
== 7'b0001011 && mem_rdata_latched[31:25] == 7'b0000010 && EN-
ABLE_IRQ;
873             is_beq_bne_blt_bge_bltu_bgeu <=
mem_rdata_latched[6:0] == 7'b1100011;
874             is_lb_lh_lw_lbu_lhu <=
mem_rdata_latched[6:0] == 7'b0000011;
875             is_sb_sh_sw <=
mem_rdata_latched[6:0] == 7'b0100011;
876             is_alu_reg_imm <=
mem_rdata_latched[6:0] == 7'b0010011;
877             is_alu_reg_reg <=
mem_rdata_latched[6:0] == 7'b0110011;
878             { decoded_imm_j[31:20], de-
coded_imm_j[10:1], decoded_imm_j[11], decoded_imm_j[19:12], de-
coded_imm_j[0] } <= $signed({mem_rdata_latched[31:12], 1'b0});
879             decoded_rd <= mem_rdata_latched[11:7];
880             decoded_rs1 <=
mem_rdata_latched[19:15];
881             decoded_rs2 <=
mem_rdata_latched[24:20];
882             if (mem_rdata_latched[6:0] ==
7'b0001011 && mem_rdata_latched[31:25] == 7'b0000000 && ENABLE_IRQ
&& ENABLE_IRQ_QREGS)
883                 decoded_rs1[regindex_bits-1] <=
1; // instr_getq
884             if (mem_rdata_latched[6:0] ==
7'b0001011 && mem_rdata_latched[31:25] == 7'b0000010 && ENABLE_IRQ)
885                 decoded_rs1 <= EN-
ABLE_IRQ_QREGS ? irqregs_offset : 3; // instr_retirq
886             compressed_instr <= 0;
887             if (COMPRESSED_ISA &&
mem_rdata_latched[1:0] != 2'b11) begin
888                 compressed_instr <= 1;
889                 decoded_rd <= 0;
890                 decoded_rs1 <= 0;
891                 decoded_rs2 <= 0;
892                 { decoded_imm_j[31:11], de-
coded_imm_j[4], decoded_imm_j[9:8], decoded_imm_j[10], de-
coded_imm_j[6],
893                 decoded_imm_j[7], de-
coded_imm_j[3:1], decoded_imm_j[5], decoded_imm_j[0] } <=
$signed({mem_rdata_latched[12:2], 1'b0});
894                 case (mem_rdata_latched[1:0])
895                     2'b00: begin // Quad-
rant 0
896                     case
(mem_rdata_latched[15:13])
897                     3'b000: begin // C.ADDI4SPN
898
899                     is_alu_reg_imm <= |mem_rdata_latched[12:5];
900                     decoded_rs1 <= 2;
901
902                     case

```

```

909 decoded_rd <= 8 + mem_rdata_latched[4:2];
910
911 3'b010: begin // C.LW
912
913 is_lb_lh_lw_lbu_lhu <= 1;
914
915 decoded_rs1 <= 8 + mem_rdata_latched[9:7];
916
917 decoded_rd <= 8 + mem_rdata_latched[4:2];
918
919 decoded_rs2 <= 8 + mem_rdata_latched[4:2];
920
921
922
923 rant 1
924 (mem_rdata_latched[15:13])
925 3'b000: begin // C.NOP / C.ADDI
926
927 is_alu_reg_imm <= 1;
928
929 decoded_rd <= mem_rdata_latched[11:7];
930
931 decoded_rs1 <= mem_rdata_latched[11:7];
932
933 3'b001: begin // C.JAL
934
935 instr_jal <= 1;
936
937 decoded_rd <= 1;
938
939 010: begin // C.LI
940
941 is_alu_reg_imm <= 1;
942
943 decoded_rd <= mem_rdata_latched[11:7];
944
945 decoded_rs1 <= 0;
946
947 011: begin
948
949 if (mem_rdata_latched[12] || mem_rdata_latched[6:2]) begin
950
951 if (mem_rdata_latched[11:7] == 2) begin // C.ADDI16SP
952
953 is_alu_reg_imm <= 1;
954
955 decoded_rd <= mem_rdata_latched[11:7];
956
957 decoded_rs1 <= mem_rdata_latched[11:7];
958
959 end else begin // C.LUI
960
961 instr_lui <= 1;
962
963 decoded_rd <= mem_rdata_latched[11:7];
964
965 decoded_rs1 <= 0;
966
967 010: begin // C.SLLI
968
969 if (!mem_rdata_latched[12]) begin
970
971 is_alu_reg_imm <= 1;
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

990 decoded_rd <= mem_rdata_latched[11:7];
991 decoded_rs1 <= mem_rdata_latched[11:7];
992 decoded_rs2 <= {mem_rdata_latched[12], mem_rdata_latched[6:2]};
993 end
994                                     end
995
3'b010: begin // C.LWSP
996 if (mem_rdata_latched[11:7]) begin
997 is_lb_lh_lw_lbu_lhu <= 1;
998 decoded_rd <= mem_rdata_latched[11:7];
999 decoded_rs1 <= 2;
1000 end
1001                                     end
1002
3'b100: begin
1003 if (mem_rdata_latched[12] == 0 && mem_rdata_latched[11:7] != 0 &&
mem_rdata_latched[6:2] == 0) begin // C.JR
1004 instr_jalr <= 1;
1005 decoded_rd <= 0;
1006 decoded_rs1 <= mem_rdata_latched[11:7];
1007 end
1008 if (mem_rdata_latched[12] == 0 && mem_rdata_latched[6:2] != 0) be-
gin // C.MV
1009 is_alu_reg_reg <= 1;
1010 decoded_rd <= mem_rdata_latched[11:7];
1011 decoded_rs1 <= 0;
1012 decoded_rs2 <= mem_rdata_latched[6:2];
1013 end
1014 if (mem_rdata_latched[12] != 0 && mem_rdata_latched[11:7] != 0 &&
mem_rdata_latched[6:2] == 0) begin // C.JALR
1015 instr_jalr <= 1;
1016 decoded_rd <= 1;
1017 decoded_rs1 <= mem_rdata_latched[11:7];
1018 end
1019 if (mem_rdata_latched[12] != 0 && mem_rdata_latched[6:2] != 0) be-
gin // C.ADD
1020 is_alu_reg_reg <= 1;
1021 decoded_rd <= mem_rdata_latched[11:7];
1022 decoded_rs1 <= mem_rdata_latched[11:7];
1023 decoded_rs2 <= mem_rdata_latched[6:2];
1024

```

```

end
1025                                     end
1026
3'b110: begin // C.SWSP
1027 is_sb_sh_sw <= 1;
1028 decoded_rs1 <= 2;
1029 decoded_rs2 <= mem_rdata_latched[6:2];
1030                                     end
1031                                     endcase
1032                                     end
1033                                     endcase
1034                                     end
1035                                     end
1036
1037 if (decoder_trigger && !decoder_pseudo_trigger)
begin
1038 pcpi_insn <= WITH_PCPI ? mem_rdata_q :
'bx;
1039
1040 instr_beq <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b000;
1041 instr_bne <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b001;
1042 instr_blt <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b100;
1043 instr_bge <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b101;
1044 instr_bltu <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b110;
1045 instr_bgeu <=
is_beq_bne_blt_bge_bltu_bgeu && mem_rdata_q[14:12] == 3'b111;
1046
1047 instr_lb <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b000;
1048 instr_lh <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b001;
1049 instr_lw <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b010;
1050 instr_lbu <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b100;
1051 instr_lhu <= is_lb_lh_lw_lbu_lhu &&
mem_rdata_q[14:12] == 3'b101;
1052
1053 instr_sb <= is_sb_sh_sw &&
mem_rdata_q[14:12] == 3'b000;
1054 instr_sh <= is_sb_sh_sw &&
mem_rdata_q[14:12] == 3'b001;
1055 instr_sw <= is_sb_sh_sw &&
mem_rdata_q[14:12] == 3'b010;
1056
1057 instr_addi <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b000;
1058 instr_slti <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b010;
1059 instr_sltiu <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b011;
1060 instr_xori <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b100;
1061 instr_ori <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b110;
1062 instr_andi <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b111;
1063
1064 instr_slli <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b001 && mem_rdata_q[31:25] == 7'b00000000;
1065 instr_srli <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b00000000;
1066 instr_srai <= is_alu_reg_imm &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b01000000;
1067
1068 instr_add <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b000 && mem_rdata_q[31:25] == 7'b00000000;
1069 instr_sub <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b000 && mem_rdata_q[31:25] == 7'b01000000;
1070 instr_sll <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b001 && mem_rdata_q[31:25] == 7'b00000000;
1071 instr_slt <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b010 && mem_rdata_q[31:25] == 7'b00000000;
1072 instr_sltu <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b011 && mem_rdata_q[31:25] == 7'b00000000;
1073 instr_xor <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b100 && mem_rdata_q[31:25] == 7'b00000000;
1074 instr_srl <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b00000000;
1075 instr_sra <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b101 && mem_rdata_q[31:25] == 7'b01000000;
1076 instr_or <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b110 && mem_rdata_q[31:25] == 7'b00000000;
1077 instr_and <= is_alu_reg_reg &&
mem_rdata_q[14:12] == 3'b111 && mem_rdata_q[31:25] == 7'b00000000;
1078
1079 instr_rdcycle <= ((mem_rdata_q[6:0] ==
7'b1110011 && mem_rdata_q[31:12] == 'b11000000000000000010) ||

```

```

1080      (mem_rdata_q[6:0] ==
1081      7'b1110011 && mem_rdata_q[31:12] == 'b1100000000100000010)) && EN-
1082      ABLE_COUNTERS;

1081      instr_rdcycleh <= ((mem_rdata_q[6:0] ==
1082      7'b1110011 && mem_rdata_q[31:12] == 'b1100100000000000010) ||

1083      (mem_rdata_q[6:0] ==
1084      7'b1110011 && mem_rdata_q[31:12] == 'b11001000000100000010)) && EN-
1085      ABLE_COUNTERS && ENABLE_COUNTERS64;

1086      instr_rdinstr <= (mem_rdata_q[6:0] ==
1087      7'b1110011 && mem_rdata_q[31:12] == 'b1100000000100000010) && EN-
1088      ABLE_COUNTERS;

1089      instr_rdinstrh <= (mem_rdata_q[6:0] ==
1090      7'b1110011 && mem_rdata_q[31:12] == 'b11001000000100000010) && EN-
1091      ABLE_COUNTERS && ENABLE_COUNTERS64;

1092      instr_eread_ebreak <=
1093      ((mem_rdata_q[6:0] == 7'b1110011 && !mem_rdata_q[31:21] && !
1094      mem_rdata_q[19:7]) ||
1095      (COMPRESSED_ISA &&
1096      mem_rdata_q[15:0] == 16'h9002));

1097      instr_fence <= (mem_rdata_q[6:0] ==
1098      7'b0001111 && !mem_rdata_q[14:12]);

1099      instr_getq <= mem_rdata_q[6:0] ==
1100      7'b0001011 && mem_rdata_q[31:25] == 7'b0000000 && ENABLE_IRQ && EN-
1101      ABLE_IRQ_QREGS;

1102      instr_setq <= mem_rdata_q[6:0] ==
1103      7'b0001011 && mem_rdata_q[31:25] == 7'b0000001 && ENABLE_IRQ && EN-
1104      ABLE_IRQ_QREGS;

1105      instr_maskirq <= mem_rdata_q[6:0] ==
1106      7'b0001011 && mem_rdata_q[31:25] == 7'b0000011 && ENABLE_IRQ;

1107      instr_timer <= mem_rdata_q[6:0] ==
1108      7'b0001011 && mem_rdata_q[31:25] == 7'b0000101 && ENABLE_IRQ && EN-
1109      ABLE_IRQ_TIMER;

1110      is_slli_srli_srai <= is_alu_reg_imm &&
1111      {

1112      mem_rdata_q[14:12] == 3'b001 &&
1113      mem_rdata_q[31:25] == 7'b0000000,

1114      mem_rdata_q[14:12] == 3'b101 &&
1115      mem_rdata_q[31:25] == 7'b0000000,

1116      mem_rdata_q[14:12] == 3'b101 &&
1117      mem_rdata_q[31:25] == 7'b0100000
1118      };

1119      is_jalr_addi_slti_sltiu_xori_ori_andi
1120      <= instr_jalr || is_alu_reg_imm && {
1121      mem_rdata_q[14:12] == 3'b000,
1122      mem_rdata_q[14:12] == 3'b010,
1123      mem_rdata_q[14:12] == 3'b011,
1124      mem_rdata_q[14:12] == 3'b100,
1125      mem_rdata_q[14:12] == 3'b110,
1126      mem_rdata_q[14:12] == 3'b111
1127      };

1128      is_sll_srl_sra <= is_alu_reg_reg && {
1129      mem_rdata_q[14:12] == 3'b001 &&
1130      mem_rdata_q[31:25] == 7'b0000000,

1131      mem_rdata_q[14:12] == 3'b101 &&
1132      mem_rdata_q[31:25] == 7'b0000000,

1133      mem_rdata_q[14:12] == 3'b101 &&
1134      mem_rdata_q[31:25] == 7'b0100000
1135      };

1136      is_lui_auiopc_jal_jalr_addi_add_sub <=
1137      0;

1138      is_compare <= 0;

1139      (* parallel_case *)
1140      case (1'b1)
1141      instr_jal:
1142      decoded_imm <= de-
1143      coded_imm_j;
1144      {instr_lui, instr_auiopc}:
1145      decoded_imm <=
1146      {instr_jalr,
1147      decoded_imm <=
1148      $signed(mem_rdata_q[31:20]);
1149      is_beq_bne_blt_bge_bltu_bgeu:
1150      decoded_imm <=
1151      $signed({mem_rdata_q[31], mem_rdata_q[7], mem_rdata_q[30:25],
1152      mem_rdata_q[11:8], 1'b0});
1153      is_sb_sh_sw:
1154      decoded_imm <=
1155      $signed({mem_rdata_q[31:25], mem_rdata_q[11:7]});
1156      default:
1157      decoded_imm <= 1'bx;
1158      endcase

```

```

1134      end
1135
1136      if (!resetn) begin
1137      is_beq_bne_blt_bge_bltu_bgeu <= 0;
1138      is_compare <= 0;
1139
1140      instr_beq <= 0;
1141      instr_bne <= 0;
1142      instr_blt <= 0;
1143      instr_bge <= 0;
1144      instr_bltu <= 0;
1145      instr_bgeu <= 0;
1146
1147      instr_addi <= 0;
1148      instr_slti <= 0;
1149      instr_sltiu <= 0;
1150      instr_xori <= 0;
1151      instr_ori <= 0;
1152      instr_andi <= 0;
1153
1154      instr_add <= 0;
1155      instr_sub <= 0;
1156      instr_sll <= 0;
1157      instr_slt <= 0;
1158      instr_sltu <= 0;
1159      instr_xor <= 0;
1160      instr_srl <= 0;
1161      instr_sra <= 0;
1162      instr_or <= 0;
1163      instr_and <= 0;
1164
1165      instr_fence <= 0;
1166      end
1167      end
1168
1169      // Main State Machine
1170
1171      localparam cpu_state_trap = 8'b10000000;
1172      localparam cpu_state_fetch = 8'b01000000;
1173      localparam cpu_state_ld_rs1 = 8'b00100000;
1174      localparam cpu_state_ld_rs2 = 8'b00010000;
1175      localparam cpu_state_exec = 8'b00001000;
1176      localparam cpu_state_shift = 8'b00000100;
1177      localparam cpu_state_stmem = 8'b00000010;
1178      localparam cpu_state_ldmem = 8'b00000001;
1179
1180      reg [7:0] cpu_state;
1181      reg [1:0] irq_state;
1182
1183      `FORMAL_KEEP reg [127:0] dbg_ascii_state;
1184
1185      always @* begin
1186      dbg_ascii_state = "";
1187      if (cpu_state == cpu_state_trap) dbg_as-
1188      cii_state = "trap";
1189      if (cpu_state == cpu_state_fetch) dbg_as-
1190      cii_state = "fetch";
1191      if (cpu_state == cpu_state_ld_rs1) dbg_as-
1192      cii_state = "ld_rs1";
1193      if (cpu_state == cpu_state_ld_rs2) dbg_as-
1194      cii_state = "ld_rs2";
1195      if (cpu_state == cpu_state_exec) dbg_as-
1196      cii_state = "exec";
1197      if (cpu_state == cpu_state_shift) dbg_as-
1198      cii_state = "shift";
1199      if (cpu_state == cpu_state_stmem) dbg_as-
1200      cii_state = "stmem";
1201      if (cpu_state == cpu_state_ldmem) dbg_as-
1202      cii_state = "ldmem";
1203      end
1204
1205      reg set_mem_do_rinst;
1206      reg set_mem_do_rdata;
1207      reg set_mem_do_wdata;
1208
1209      reg latched_store;
1210      reg latched_stalu;
1211      reg latched_branch;
1212      reg latched_compr;
1213      reg latched_trace;
1214      reg latched_is_lu;
1215      reg latched_is_lh;
1216      reg latched_is_lb;
1217      reg [regindex_bits-1:0] latched_rd;
1218
1219      reg [31:0] current_pc;
1220      assign next_pc = latched_store && latched_branch ?
1221      reg_out & ~1 : reg_next_pc;
1222
1223      reg [3:0] pcpi_timeout_counter;
1224      reg pcpi_timeout;
1225
1226      reg [31:0] next_irq_pending;
1227      reg do_waitirq;
1228
1229      reg [31:0] alu_out, alu_out_q;
1230      reg alu_out_0, alu_out_0_q;
1231      reg alu_wait, alu_wait_2;
1232
1233      reg [31:0] alu_add_sub;

```



```

1226     reg [31:0] alu_shl, alu_shr;
1227     reg alu_eq, alu_ltu, alu_lts;
1228
1229     generate if (TWO_CYCLE_ALU) begin
1230         always @(posedge clk) begin
1231             alu_add_sub <= instr_sub ? reg_op1 -
reg_op2 : reg_op1 + reg_op2;
1232             alu_eq <= reg_op1 == reg_op2;
1233             alu_lts <= $signed(reg_op1) <
$signed(reg_op2);
1234             alu_ltu <= reg_op1 < reg_op2;
1235             alu_shl <= reg_op1 << reg_op2[4:0];
1236             alu_shr <= $signed({instr_sra || in-
str_srai ? reg_op1[31] : 1'b0, reg_op1}) >>> reg_op2[4:0];
1237         end
1238     end else begin
1239         always @* begin
1240             alu_add_sub = instr_sub ? reg_op1 -
reg_op2 : reg_op1 + reg_op2;
1241             alu_eq = reg_op1 == reg_op2;
1242             alu_lts = $signed(reg_op1) <
$signed(reg_op2);
1243             alu_ltu = reg_op1 < reg_op2;
1244             alu_shl = reg_op1 << reg_op2[4:0];
1245             alu_shr = $signed({instr_sra || in-
str_srai ? reg_op1[31] : 1'b0, reg_op1}) >>> reg_op2[4:0];
1246         end
1247     end endgenerate
1248
1249     always @* begin
1250         alu_out_0 = 'bx;
1251         (* parallel_case, full_case *)
1252         case (1'b1)
1253             instr_beq:
1254                 alu_out_0 = alu_eq;
1255             instr_bne:
1256                 alu_out_0 = !alu_eq;
1257             instr_bge:
1258                 alu_out_0 = !alu_lts;
1259             instr_bgeu:
1260                 alu_out_0 = !alu_ltu;
1261             is_slti_blt_slt && (!TWO_CYCLE_COMPARE
|| !{instr_beq, instr_bne, instr_bge, instr_bgeu}):
1262                 alu_out_0 = alu_lts;
1263             is_sltiu_bltu_sltu && (!
TWO_CYCLE_COMPARE || !{instr_beq, instr_bne, instr_bge, instr_bgeu}):
1264                 alu_out_0 = alu_ltu;
1265         endcase
1266
1267         alu_out = 'bx;
1268         (* parallel_case, full_case *)
1269         case (1'b1)
1270             is_lui_auiopc_jal_jalr_addi_add_sub:
1271                 alu_out = alu_add_sub;
1272             is_compare:
1273                 alu_out = alu_out_0;
1274             instr_xori || instr_xor:
1275                 alu_out = reg_op1 ^ reg_op2;
1276             instr_ori || instr_or:
1277                 alu_out = reg_op1 | reg_op2;
1278             instr_andi || instr_and:
1279                 alu_out = reg_op1 & reg_op2;
1280             BARREL_SHIFTER && (instr_sll || in-
str_slli):
1281                 alu_out = alu_shl;
1282             BARREL_SHIFTER && (instr_srl || in-
str_srli || instr_sra || instr_srai):
1283                 alu_out = alu_shr;
1284         endcase
1285
1286         `ifndef RISC_V_FORMAL_BLACKBOX_ALU
1287             alu_out_0 = $anyseq;
1288             alu_out = $anyseq;
1289         `endif
1290     end
1291
1292     reg clear_prefetched_high_word_q;
1293     always @(posedge clk) clear_prefetched_high_word_q <=
clear_prefetched_high_word;
1294
1295     always @* begin
1296         clear_prefetched_high_word =
clear_prefetched_high_word_q;
1297         if (!prefetched_high_word)
1298             clear_prefetched_high_word = 0;
1299         if (latched_branch || irq_state || !resets)
1300             clear_prefetched_high_word = COM-
PRESSED_ISA;
1301     end
1302
1303     reg cpuregs_write;
1304     reg [31:0] cpuregs_wrddata;
1305     reg [31:0] cpuregs_rs1;
1306     reg [31:0] cpuregs_rs2;
1307     reg [regindex_bits-1:0] decoded_rs;
1308
1309     always @* begin
1310         cpuregs_write = 0;
1311         cpuregs_wrddata = 'bx;
1312
1313         if (cpu_state == cpu_state_fetch) begin

```

```

1314             (* parallel_case *)
1315             case (1'b1)
1316                 latched_branch: begin
1317                     cpuregs_wrddata = reg_pc
+ (latched_compr ? 2 : 4);
1318                     cpuregs_write = 1;
1319                 end
1320                 latched_store && !
latched_branch: begin
1321                     cpuregs_wrddata =
latched_stalu ? alu_out_q : reg_out;
1322                     cpuregs_write = 1;
1323                 end
1324                 ENABLE_IRQ && irq_state[0]: be-
gin
1325                     cpuregs_wrddata =
reg_next_pc | latched_compr;
1326                     cpuregs_write = 1;
1327                 end
1328                 ENABLE_IRQ && irq_state[1]: be-
gin
1329                     cpuregs_wrddata =
irq_pending & ~irq_mask;
1330                     cpuregs_write = 1;
1331                 end
1332             endcase
1333         end
1334     end
1335
1336     `ifndef PICORV32_REGS
1337         always @(posedge clk) begin
1338             if (resets && cpuregs_write && latched_rd)
1339                 `ifndef PICORV32_TESTBUG_001
1340                     cpuregs[latched_rd ^ 1] <=
cpuregs_wrddata;
1341                 `elsif PICORV32_TESTBUG_002
1342                     cpuregs[latched_rd] <= cpuregs_wrddata ^
1;
1343             `else
1344                 cpuregs[latched_rd] <= cpuregs_wrddata;
1345             `endif
1346         end
1347
1348         always @* begin
1349             decoded_rs = 'bx;
1350             if (ENABLE_REGS_DUALPORT) begin
1351                 `ifndef RISC_V_FORMAL_BLACKBOX_REGS
1352                     cpuregs_rs1 = decoded_rs1 ?
cpuregs[decoded_rs1] : 0;
1353                     cpuregs_rs2 = decoded_rs2 ?
cpuregs[decoded_rs2] : 0;
1354                 `else
1355                     cpuregs_rs1 = decoded_rs1 ? $anyseq :
0;
1356                     cpuregs_rs2 = decoded_rs2 ? $anyseq :
0;
1357                 `endif
1358             end else begin
1359                 decoded_rs = (cpu_state ==
cpu_state_ld_rs2) ? decoded_rs2 : decoded_rs1;
1360                 `ifndef RISC_V_FORMAL_BLACKBOX_REGS
1361                     cpuregs_rs1 = decoded_rs ?
cpuregs[decoded_rs] : 0;
1362                 `else
1363                     cpuregs_rs1 = decoded_rs ? $anyseq : 0;
1364                 `endif
1365                 cpuregs_rs2 = cpuregs_rs1;
1366             end
1367         end
1368     `else
1369         wire [31:0] cpuregs_rdata1;
1370         wire [31:0] cpuregs_rdata2;
1371
1372         wire [5:0] cpuregs_waddr = latched_rd;
1373         wire [5:0] cpuregs_raddr1 = ENABLE_REGS_DUALPORT ? de-
coded_rs1 : decoded_rs;
1374         wire [5:0] cpuregs_raddr2 = ENABLE_REGS_DUALPORT ? de-
coded_rs2 : 0;
1375
1376         `PICORV32_REGS cpuregs (
1377             .clk(clk),
1378             .wen(resets && cpuregs_write && latched_rd),
1379             .waddr(cpuregs_waddr),
1380             .raddr1(cpuregs_raddr1),
1381             .raddr2(cpuregs_raddr2),
1382             .wdata(cpuregs_wrddata),
1383             .rdata1(cpuregs_rdata1),
1384             .rdata2(cpuregs_rdata2)
1385         );
1386
1387         always @* begin
1388             decoded_rs = 'bx;
1389             if (ENABLE_REGS_DUALPORT) begin
1390                 cpuregs_rs1 = decoded_rs1 ?
cpuregs_rdata1 : 0;
1391                 cpuregs_rs2 = decoded_rs2 ?
cpuregs_rdata2 : 0;
1392             end else begin
1393                 decoded_rs = (cpu_state ==

```



```

cpu_state_ld_rs2) ? decoded_rs2 : decoded_rs1;
1394         cpuregs_rs1 = decoded_rs ?
cpuregs_rdata1 : 0;
1395         cpuregs_rs2 = cpuregs_rs1;
1396     end
1397 end
1398 `endif
1399
1400     assign launch_next_insn = cpu_state == cpu_state_fetch
&& decoder_trigger && (!ENABLE_IRQ || irq_delay || irq_active || !
(irq_pending & ~irq_mask));
1401
1402     always @(posedge clk) begin
1403         trap <= 0;
1404         reg_sh <= 'bx;
1405         reg_out <= 'bx;
1406         set_mem_do_rinst = 0;
1407         set_mem_do_rdata = 0;
1408         set_mem_do_wdata = 0;
1409
1410         alu_out_0_q <= alu_out_0;
1411         alu_out_q <= alu_out;
1412
1413         alu_wait <= 0;
1414         alu_wait_2 <= 0;
1415
1416         if (launch_next_insn) begin
1417             dbg_rs1val <= 'bx;
1418             dbg_rs2val <= 'bx;
1419             dbg_rs1val_valid <= 0;
1420             dbg_rs2val_valid <= 0;
1421         end
1422
1423         if (WITH_PCPI && CATCH_ILLINSN) begin
1424             if (resetrn && pcpi_valid && !
pcpi_int_wait) begin
1425                 if (pcpi_timeout_counter)
1426                     pcpi_timeout_counter <=
pcpi_timeout_counter - 1;
1427             end else
1428                 pcpi_timeout_counter <= ~0;
1429             pcpi_timeout <= !pcpi_timeout_counter;
1430         end
1431
1432         if (ENABLE_COUNTERS) begin
1433             count_cycle <= resetrn ? count_cycle +
1 : 0;
1434             if (!ENABLE_COUNTERS64)
1435                 count_cycle[63:32] <= 0;
1436             end else begin
1437                 count_cycle <= 'bx;
1438                 count_instr <= 'bx;
1439             end
1440             next_irq_pending = ENABLE_IRQ ? irq_pending &
LATCHED_IRQ : 'bx;
1441
1442             if (ENABLE_IRQ && ENABLE_IRQ_TIMER && timer)
1443                 timer <= timer - 1;
1444             end
1445
1446             decoder_trigger <= mem_do_rinst && mem_done;
1447             decoder_trigger_q <= decoder_trigger;
1448             decoder_pseudo_trigger <= 0;
1449             decoder_pseudo_trigger_q <= de-
coder_pseudo_trigger;
1450             do_waitirq <= 0;
1451
1452             trace_valid <= 0;
1453
1454             if (!ENABLE_TRACE)
1455                 trace_data <= 'bx;
1456
1457             if (!resetrn) begin
1458                 reg_pc <= PROGADDR_RESET;
1459                 reg_next_pc <= PROGADDR_RESET;
1460                 if (ENABLE_COUNTERS)
1461                     count_instr <= 0;
1462                 latched_store <= 0;
1463                 latched_stalu <= 0;
1464                 latched_branch <= 0;
1465                 latched_trace <= 0;
1466                 latched_is_lu <= 0;
1467                 latched_is_lh <= 0;
1468                 latched_is_lb <= 0;
1469                 pcpi_valid <= 0;
1470                 pcpi_timeout <= 0;
1471                 irq_active <= 0;
1472                 irq_delay <= 0;
1473                 irq_mask <= ~0;
1474                 next_irq_pending = 0;
1475                 irq_state <= 0;
1476                 eoi <= 0;
1477                 timer <= 0;
1478                 if (~STACKADDR) begin
1479                     latched_store <= 1;
1480                     latched_rd <= 2;
1481                     reg_out <= STACKADDR;
1482                 end

```

```

1483         cpu_state <= cpu_state_fetch;
1484     end else
1485         (* parallel_case, full_case *)
1486         case (cpu_state)
1487             cpu_state_trap: begin
1488                 trap <= 1;
1489             end
1490
1491             cpu_state_fetch: begin
1492                 mem_do_rinst <= !de-
coder_trigger && !do_waitirq;
1493                 mem_wordsize <= 0;
1494                 current_pc = reg_next_pc;
1495                 (* parallel_case *)
1496                 case (1'b1)
1497                     latched_branch: begin
1498                         current_pc =
latched_store ? (latched_stalu ? alu_out_q : reg_out) & ~1 :
reg_next_pc;
1499                     `de-
bug($display("ST_RD: %2d 0x%08x", latched_rd, reg_pc
+ (latched_compr ? 2 : 4), current_pc));
1500                 end
1501                 latched_store && !
latched_branch: begin
1502                     `de-
bug($display("ST_RD: %2d 0x%08x", latched_rd, latched_stalu ?
alu_out_q : reg_out));
1503                 end
1504                 ENABLE_IRQ &&
irq_state[0]: begin
1505                     current_pc =
PROGADDR_IRQ;
1506                     irq_active <=
1;
1507
1508                     mem_do_rinst <=
1;
1509                     end
1510                     ENABLE_IRQ &&
irq_state[1]: begin
1511                         eoi <=
irq_pending & ~irq_mask;
1512                         next_irq_pend-
ing = next_irq_pending & irq_mask;
1513                     end
1514                     endcase
1515                     if (ENABLE_TRACE &&
latched_trace) begin
1516                         latched_trace <= 0;
1517                         trace_valid <= 1;
1518                         if (latched_branch)
1519                             trace_data <=
(irq_active ? TRACE_IRQ : 0) | TRACE_BRANCH | (current_pc &
32'hfffffffe);
1520                         else
1521                             trace_data <=
(irq_active ? TRACE_IRQ : 0) | (latched_stalu ? alu_out_q :
reg_out);
1522                     end
1523                     reg_pc <= current_pc;
1524                     reg_next_pc <= current_pc;
1525                     latched_store <= 0;
1526                     latched_stalu <= 0;
1527                     latched_branch <= 0;
1528                     latched_is_lu <= 0;
1529                     latched_is_lh <= 0;
1530                     latched_is_lb <= 0;
1531                     latched_rd <= decoded_rd;
1532                     latched_compr <= com-
pressed_instr;
1533                     if (ENABLE_IRQ && ((de-
coder_trigger && !irq_active && !irq_delay && !(irq_pending &
~irq_mask)) || irq_state)) begin
1534                         irq_state <=
2'b00 ? 2'b01 :
2'b01 ? 2'b10 : 2'b00;
1535                         latched_compr <=
latched_compr;
1536                         if (ENABLE_IRQ_QREGS)
1537                             latched_rd <=
1538                             else
1539                                 latched_rd <=
1540                     end else
1541                         if (ENABLE_IRQ && (de-
coder_trigger || do_waitirq) && instr_waitirq) begin
1542                             if (irq_pending) begin
1543                                 latched_store
<= 1;
1544                                 reg_out <=
irq_pending;

```

```

1552                                     reg_next_pc <=
current_pc + (compressed_instr ? 2 : 4);

1553                                     mem_do_rinst <=
1;
1554                                     end else
1555                                     do_waitirq <=
1;
1556                                     end else
1557                                     if (decoder_trigger) begin
1558                                     `debug($display("--
%-0t", $time));
1559                                     irq_delay <= irq_act-
ive;
1560                                     reg_next_pc <= cur-
rent_pc + (compressed_instr ? 2 : 4);
1561                                     if (ENABLE_TRACE)
1562                                     latched_trace
<= 1;
1563                                     if (ENABLE_COUNTERS)
begin
1564                                     count_instr <=
count_instr + 1;
1565                                     if (!EN-
ABLE_COUNTERS64) count_instr[63:32] <= 0;
1566                                     end
1567                                     if (instr_jal) begin

1568                                     mem_do_rinst <=
1;
1569                                     reg_next_pc <=
current_pc + decoded_imm_j;
1570                                     latched_branch
<= 1;
1571                                     end else begin

1572                                     mem_do_rinst <=
0;

1573                                     mem_do_prefetch
<= !instr_jalr && !instr_retirq;
1574                                     cpu_state <=
cpu_state_ld_rs1;
1575                                     end
1576                                     end
1577                                     end
1578                                     end
1579                                     cpu_state_ld_rs1: begin
1580                                     reg_op1 <= 'bx;
1581                                     reg_op2 <= 'bx;

1582                                     (* parallel_case *)
1583                                     case (1'b1)
1584                                     case (1'b1)
1585                                     (CATCH_ILLINSN ||
WITH_PCPI) && instr_trap: begin
1586                                     if (WITH_PCPI)
begin
1587                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1588                                     reg_op1 <= cpuregs_rs1;
1589                                     dbg_r-
s1val <= cpuregs_rs1;
1590                                     dbg_r-
s1val_valid <= 1;
1591                                     if
(ENABLE_REGS_DUALPORT) begin
1592                                     pcpi_valid <= 1;

1593                                     `debug($display("LD_RS2: %2d 0x%08x", decoded_rs2, cpuregs_rs2));

1594                                     reg_sh <= cpuregs_rs2;

1595                                     reg_op2 <= cpuregs_rs2;

1596                                     dbg_rs2val <= cpuregs_rs2;

1597                                     dbg_rs2val_valid <= 1;

1598                                     if (pcpi_int_ready) begin

1599                                     mem_do_rinst <= 1;

1600                                     pcpi_valid <= 0;

1601                                     reg_out <= pcpi_int_rd;

1602                                     latched_store <= pcpi_int_wr;

```

```

1603                                     cpu_state <= cpu_state_fetch;

1604                                     end else

1605                                     if (CATCH_ILLINSN && (pcpi_timeout || instr_ecall_ebreak)) begin

1606                                     pcpi_valid <= 0;

1607                                     `debug($display("EBREAK OR UNSUPPORTED INSN AT 0x%08x", reg_pc));

1608                                     if (ENABLE_IRQ && !irq_mask[irq_ebreak] && !irq_active) begin

1609                                     next_irq_pending[irq_ebreak] = 1;

1610                                     cpu_state <= cpu_state_fetch;

1611                                     end else

1612                                     cpu_state <= cpu_state_trap;

1613                                     end

1614                                     else begin

1615                                     cpu_state <= cpu_state_ld_rs2;

1616                                     end

1617                                     end else begin

1618                                     `de-
bug($display("EBREAK OR UNSUPPORTED INSN AT 0x%08x", reg_pc));

1619                                     if
(ENABLE_IRQ && !irq_mask[irq_ebreak] && !irq_active) begin

1620                                     next_irq_pending[irq_ebreak] = 1;

1621                                     cpu_state <= cpu_state_fetch;

1622                                     end

else

1623                                     cpu_state <= cpu_state_trap;

1624                                     end

1625                                     end

1626                                     is_rdcycle_rdcycleh_rdinstr_rdinstrh: begin

1627                                     (* paral-
lel_case, full_case *)

1628                                     case (1'b1)

1629                                     in-
str_rdcycle:

1630                                     reg_out <= count_cycle[31:0];

1631                                     in-
str_rdcycleh && ENABLE_COUNTERS64:

1632                                     reg_out <= count_cycle[63:32];

1633                                     in-
str_rdinstr:

1634                                     reg_out <= count_instr[31:0];

1635                                     in-
str_rdinstrh && ENABLE_COUNTERS64:

1636                                     reg_out <= count_instr[63:32];

1637                                     endcase

1638                                     latched_store

1639                                     cpu_state <=
cpu_state_fetch;

1640                                     end

1641                                     is_lui_auiopc_jal: begin

1642                                     reg_op1 <= in-
str_lui ? 0 : reg_pc;

1643                                     reg_op2 <= de-
coded_imm;

1644                                     if
(TWO_CYCLE_ALU)

1645                                     alu_wait <= 1;

1646                                     else

1647                                     mem_do_rinst <= mem_do_prefetch;

1648                                     cpu_state <=
cpu_state_exec;

```

```

1649                                     end
1650                                     ENABLE_IRQ && EN-
ABLE_IRQ_QREGS && instr_getq: begin
1651                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1652                                     reg_out <=
cpuregs_rs1;
1653                                     dbg_rs1val <=
cpuregs_rs1;
1654                                     dbg_rs1v-
al_valid <= 1;
1655                                     latched_store
<= 1;
1656                                     cpu_state <=
cpu_state_fetch;
1657                                     end
1658                                     ENABLE_IRQ && EN-
ABLE_IRQ_QREGS && instr_setq: begin
1659                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1660                                     reg_out <=
cpuregs_rs1;
1661                                     dbg_rs1val <=
cpuregs_rs1;
1662                                     dbg_rs1v-
al_valid <= 1;
1663                                     latched_rd <=
latched_rd | irqregs_offset;
1664                                     latched_store
<= 1;
1665                                     cpu_state <=
cpu_state_fetch;
1666                                     end
1667                                     ENABLE_IRQ && in-
str_retirq: begin
1668                                     eoi <= 0;
1669                                     irq_active <=
0;
1670                                     latched_branch
<= 1;
1671                                     latched_store
<= 1;
1672                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1673                                     reg_out <=
CATCH_MISALIGN ? (cpuregs_rs1 & 32'h ffffffff) : cpuregs_rs1;
1674                                     dbg_rs1val <=
cpuregs_rs1;
1675                                     dbg_rs1v-
al_valid <= 1;
1676                                     cpu_state <=
cpu_state_fetch;
1677                                     end
1678                                     ENABLE_IRQ && in-
str_maskirq: begin
1679                                     latched_store
<= 1;
1680                                     reg_out <=
irq_mask;
1681                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1682                                     irq_mask <=
cpuregs_rs1 | MASKED_IRQ;
1683                                     dbg_rs1val <=
cpuregs_rs1;
1684                                     dbg_rs1v-
al_valid <= 1;
1685                                     cpu_state <=
cpu_state_fetch;
1686                                     end
1687                                     ENABLE_IRQ && EN-
ABLE_IRQ_TIMER && instr_timer: begin
1688                                     latched_store
<= 1;
1689                                     reg_out <=
timer;
1690                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1691                                     timer <=
cpuregs_rs1;
1692                                     dbg_rs1val <=
cpuregs_rs1;
1693                                     dbg_rs1v-
al_valid <= 1;
1694                                     cpu_state <=
cpu_state_fetch;
1695                                     end
1696                                     is_lb_lh_lw_lbu_lhu
&& !instr_trap: begin
1697                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1698                                     reg_op1 <=
cpuregs_rs1;
1699                                     dbg_rs1val <=
cpuregs_rs1;
1700                                     dbg_rs1v-
al_valid <= 1;
1701                                     cpu_state <=
cpu_state_ldmem;
1702                                     mem_do_rinst <=

```

```

1;
1703                                     end
1704                                     is_slli_srli_srai && !
BARREL_SHIFTER: begin
1705                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1706                                     reg_op1 <=
cpuregs_rs1;
1707                                     dbg_rs1val <=
cpuregs_rs1;
1708                                     dbg_rs1v-
al_valid <= 1;
1709                                     reg_sh <= de-
coded_rs2;
1710                                     cpu_state <=
cpu_state_shift;
1711                                     end
1712                                     is_jalr_addi_slti_sltiu_xori_ori_andi, is_slli_srli_srai && BAR-
REL_SHIFTER: begin
1713                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1714                                     reg_op1 <=
cpuregs_rs1;
1715                                     dbg_rs1val <=
cpuregs_rs1;
1716                                     dbg_rs1v-
al_valid <= 1;
1717                                     reg_op2 <=
is_slli_srli_srai && BARREL_SHIFTER ? decoded_rs2 : decoded_imm;
1718                                     if
(TWO_CYCLE_ALU)
1719                                     else
1720                                     mem_do_rinst <= mem_do_prefetch;
1721                                     cpu_state <=
cpu_state_exec;
1722                                     end
1723                                     default: begin
1724                                     `de-
bug($display("LD_RS1: %2d 0x%08x", decoded_rs1, cpuregs_rs1));
1725                                     reg_op1 <=
cpuregs_rs1;
1726                                     dbg_rs1val <=
cpuregs_rs1;
1727                                     dbg_rs1v-
al_valid <= 1;
1728                                     if (EN-
ABLE_REGS_DUALPORT) begin
1729                                     `de-
bug($display("LD_RS2: %2d 0x%08x", decoded_rs2, cpuregs_rs2));
1730                                     reg_sh
<= cpuregs_rs2;
1731                                     reg_op2 <= cpuregs_rs2;
1732                                     dbg_r-
s2val <= cpuregs_rs2;
1733                                     dbg_r-
s2val_valid <= 1;
1734                                     (*
parallel_case *)
1735                                     case
1736                                     (1'b1)
1737                                     is_sb_sh_sw: begin
1738                                     cpu_state <= cpu_state_stmem;
1739                                     mem_do_rinst <= 1;
1740                                     end
1741                                     is_sll_srl_sra && !BARREL_SHIFTER: begin
1742                                     cpu_state <= cpu_state_shift;
1743                                     end
1744                                     default: begin
1745                                     if (TWO_CYCLE_ALU || (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu)) begin
1746                                     alu_wait_2 <= TWO_CYCLE_ALU && (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu);
1747                                     alu_wait <= 1;

```

```

1748
end else
1749
mem_do_rinst <= mem_do_prefetch;

1750
cpu_state <= cpu_state_exec;

1751
end
1752
case
1753
end else
1754
cpu_state <= cpu_state_ld_rs2;
1755
end
1756
endcase
1757
end
1758
cpu_state_ld_rs2: begin
1759
`debug($display("LD_RS2: %2d
0x%08x", decoded_rs2, cpuregs_rs2));
1760
reg_sh <= cpuregs_rs2;
1761
reg_op2 <= cpuregs_rs2;
1762
dbg_rs2val <= cpuregs_rs2;
1763
dbg_rs2val_valid <= 1;
1764
(* parallel_case *)
1765
case (1'b1)
1766
WITH_PCPI && in-
1767
str_trap: begin
1768
pcpi_valid <=
1769
1;
1770
if
1771
(pcpi_int_ready) begin
1772
mem_do_rinst <= 1;
1773
pcpi_valid <= 0;
1774
reg_out <= pcpi_int_rd;
1775
latched_store <= pcpi_int_wr;
1776
cpu_state <= cpu_state_fetch;
1777
end else
1778
if
1779
(CATCH_ILLINSN && (pcpi_timeout || instr_ecall_ebreak)) begin
1780
pcpi_valid <= 0;
1781
`de-
bug($display("EBREAK OR UNSUPPORTED INSN AT 0x%08x", reg_pc));
1782
if
1783
(ENABLE_IRQ && !irq_mask[irq_ebreak] && !irq_active) begin
1784
next_irq_pending[irq_ebreak] = 1;
1785
1786
cpu_state <= cpu_state_fetch;
1787
end
1788
else
1789
end
1790
cpu_state <= cpu_state_trap;
1791
end
1792
is_sb_sh_sw: begin
1793
cpu_state <=
1794
end
1795
default: begin
1796
if
1797
(TWO_CYCLE_ALU || (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu)) begin
1798
alu_wait_2 <= TWO_CYCLE_ALU && (TWO_CYCLE_COMPARE &&
is_beq_bne_blt_bge_bltu_bgeu);
1799
alu_wait <= 1;
1800
end else
1801
end
1802
mem_do_rinst <= mem_do_prefetch;
1803
cpu_state <=
1804
end
1805
cpu_state_exec: begin
1806
reg_out <= reg_pc + de-

```

```

coded_imm;
1807
if ((TWO_CYCLE_ALU ||
TWO_CYCLE_COMPARE) && (alu_wait || alu_wait_2)) begin
1808
mem_do_rinst <=
1809
mem_do_prefetch && !alu_wait_2;
1810
alu_wait <= alu_wait_2;
1811
end else
1812
if
1813
(is_beq_bne_blt_bge_bltu_bgeu) begin
1814
latched_rd <= 0;
1815
latched_store <=
1816
TWO_CYCLE_COMPARE ? alu_out_0_q : alu_out_0;
1817
latched_branch <=
1818
TWO_CYCLE_COMPARE ? alu_out_0_q : alu_out_0;
1819
if (mem_done)
1820
cpu_state <=
1821
cpu_state_fetch;
1822
if (TWO_CYCLE_COMPARE ?
1823
alu_out_0_q : alu_out_0) begin
1824
decoder_trigger
1825
<= 0;
1826
1827
set_mem_do_rinst = 1;
1828
end
1829
end else begin
1830
latched_branch <= in-
1831
str_jalr;
1832
latched_store <= 1;
1833
latched_stalu <= 1;
1834
cpu_state <=
1835
cpu_state_fetch;
1836
end
1837
end
1838
cpu_state_shift: begin
1839
latched_store <= 1;
1840
if (reg_sh == 0) begin
1841
reg_out <= reg_op1;
1842
mem_do_rinst <=
1843
cpu_state <=
1844
end else if (TWO_STAGE_SHIFT &&
1845
(* parallel_case,
1846
case (1'b1)
1847
instr_slli ||
1848
instr_srli ||
1849
instr_srai ||
1850
instr_sll: reg_op1 <= reg_op1 << 4;
1851
instr_srl: reg_op1 <= reg_op1 >> 4;
1852
instr_sra: reg_op1 <= $signed(reg_op1) >>> 4;
1853
endcase
1854
reg_sh <= reg_sh - 4;
1855
end else begin
1856
(* parallel_case,
1857
case (1'b1)
1858
instr_slli ||
1859
instr_srli ||
1860
instr_srai ||
1861
instr_sll: reg_op1 <= reg_op1 << 1;
1862
instr_srl: reg_op1 <= reg_op1 >> 1;
1863
instr_sra: reg_op1 <= $signed(reg_op1) >>> 1;
1864
endcase
1865
reg_sh <= reg_sh - 1;
1866
end
1867
end
1868
cpu_state_stmem: begin
1869
if (ENABLE_TRACE)
1870
reg_out <= reg_op2;
1871
if (!mem_do_prefetch ||
1872
if (!mem_do_wdata) be-
1873
gin
1874
(* paral-
1875
lel_case, full_case *)
1876
case (1'b1)
1877
in-
1878
str_sb: mem_wordsize <= 2;
1879
in-
1880
str_sh: mem_wordsize <= 1;
1881
in-
1882
str_sw: mem_wordsize <= 0;
1883
endcase
1884
if (EN-
1885
ABLE_TRACE) begin
1886
trace_valid <= 1;
1887
trace_data <= (irq_active ? TRACE_IRQ : 0) | TRACE_ADDR | ((reg_op1
+ decoded_imm) & 32'hfffffff);
1888
end
1889
reg_op1 <=
1890
reg_op1 + decoded_imm;
1891

```

```

1871 set_mem_do_wdata = 1;
1872                                     end
1873                                     if (!mem_do_prefetch &&
mem_done) begin
1874                                     cpu_state <=
1875                                     decoder_trigger
1876                                     de-
1877                                     end
1878                                     end
1879                                     end
1880                                     cpu_state_ldmem: begin
1881                                     latched_store <= 1;
1882                                     if (!mem_do_prefetch ||
mem_done) begin
1883                                     if (!mem_do_rdata) be-
1884                                     (* paral-
1885                                     case (1'b1)
1886                                     in-
1887                                     in-
1888                                     in-
1889                                     endcase
1890                                     latched_is_lu
1891                                     latched_is_lh
1892                                     latched_is_lb
1893                                     if (EN-
ABLE_TRACE) begin
1894                                     trace_valid <= 1;
1895                                     trace_data <= (irq_active ? TRACE_IRQ : 0) | TRACE_ADDR | ((reg_op1
+ decoded_imm) & 32'hffffffff);
1896                                     end
1897                                     reg_op1 <=
1898                                     reg_op1 + decoded_imm;
1899                                     set_mem_do_rdata = 1;
1900                                     end
1901                                     if (!mem_do_prefetch &&
mem_done) begin
1902                                     (* paral-
1903                                     case (1'b1)
1904                                     latched_is_lu: reg_out <= mem_rdata_word;
1905                                     latched_is_lh: reg_out <= $signed(mem_rdata_word[15:0]);
1906                                     latched_is_lb: reg_out <= $signed(mem_rdata_word[7:0]);
1907                                     endcase
1908                                     decoder_trigger
1909                                     de-
1910                                     cpu_state <=
1911                                     end
1912                                     end
1913                                     endcase
1914                                     if (ENABLE_IRQ) begin
1915                                     next_irq_pending = next_irq_pending |
1916                                     irq;
1917                                     if(ENABLE_IRQ_TIMER && timer)
1918                                     if (timer - 1 == 0)
1919                                     next_irq_pend-
1920                                     ing[irq_timer] = 1;
1921                                     end
1922                                     if (CATCH_MISALIGN && resetn && (mem_do_rdata
|| mem_do_wdata)) begin
1923                                     if (mem_wordsize == 0 && reg_op1[1:0] !=
0) begin
1924                                     `debug($display("MISALIGNED
WORD: 0x%08x", reg_op1));
1925                                     if (ENABLE_IRQ && !
irq_mask[irq_buserror] && !irq_active) begin
1926                                     next_irq_pend-
1927                                     ing[irq_buserror] = 1;
1928                                     end else
1929                                     cpu_state <=
1930                                     end
1931                                     if (mem_wordsize == 1 && reg_op1[0] !=
0) begin

```

```

1932                                     `debug($display("MISALIGNED
HALFWORD: 0x%08x", reg_op1));
1933                                     if (ENABLE_IRQ && !
irq_mask[irq_buserror] && !irq_active) begin
1934                                     next_irq_pend-
1935                                     ing[irq_buserror] = 1;
1936                                     end else
1937                                     cpu_state <=
1938                                     end
1939                                     if (CATCH_MISALIGN && resetn && mem_do_rinst &&
(COMPRESSED_ISA ? reg_pc[0] : |reg_pc[1:0])) begin
1940                                     `debug($display("MISALIGNED INSTRU-
TION: 0x%08x", reg_pc));
1941                                     if (ENABLE_IRQ && !
irq_mask[irq_buserror] && !irq_active) begin
1942                                     next_irq_pending[irq_buserror]
= 1;
1943                                     end else
1944                                     cpu_state <= cpu_state_trap;
1945                                     end
1946                                     if (!CATCH_ILLEGAL && decoder_trigger_q && !de-
coder_pseudo_trigger_q && instr_ecall_ebreak) begin
1947                                     cpu_state <= cpu_state_trap;
1948                                     end
1949                                     if (!resetn || mem_done) begin
1950                                     mem_do_prefetch <= 0;
1951                                     mem_do_rinst <= 0;
1952                                     mem_do_rdata <= 0;
1953                                     mem_do_wdata <= 0;
1954                                     end
1955                                     if (set_mem_do_rinst)
1956                                     mem_do_rinst <= 1;
1957                                     if (set_mem_do_rdata)
1958                                     mem_do_rdata <= 1;
1959                                     if (set_mem_do_wdata)
1960                                     mem_do_wdata <= 1;
1961                                     irq_pending <= next_irq_pending & ~MASKED_IRQ;
1962                                     if (!CATCH_MISALIGN) begin
1963                                     if (COMPRESSED_ISA) begin
1964                                     reg_pc[0] <= 0;
1965                                     reg_next_pc[0] <= 0;
1966                                     end else begin
1967                                     reg_pc[1:0] <= 0;
1968                                     reg_next_pc[1:0] <= 0;
1969                                     end
1970                                     end
1971                                     current_pc = 'bx;
1972                                     end
1973                                     end
1974                                     end
1975                                     end
1976                                     end
1977                                     `ifdef RISC_V_FORMAL
1978                                     reg dbg_irq_call;
1979                                     reg dbg_irq_enter;
1980                                     reg [31:0] dbg_irq_ret;
1981                                     always @(posedge clk) begin
1982                                     rvfi_valid <= resetn && (launch_next_insn ||
trap) && dbg_valid_insn;
1983                                     rvfi_order <= resetn ? rvfi_order + rvfi_val-
id : 0;
1984                                     rvfi_insn <= dbg_insn_opcode;
1985                                     rvfi_rs1_addr <= dbg_rs1val_valid ? dbg_in-
sn_rs1 : 0;
1986                                     rvfi_rs2_addr <= dbg_rs2val_valid ? dbg_in-
sn_rs2 : 0;
1987                                     rvfi_pc_rdata <= dbg_insn_addr;
1988                                     rvfi_rs1_rdata <= dbg_rs1val_valid ? dbg_rs1v-
al : 0;
1989                                     rvfi_rs2_rdata <= dbg_rs2val_valid ? dbg_rs2v-
al : 0;
1990                                     rvfi_trap <= trap;
1991                                     rvfi_halt <= trap;
1992                                     rvfi_intr <= dbg_irq_enter;
1993                                     rvfi_mode <= 3;
1994                                     rvfi_ixl <= 1;
1995                                     if (!resetn) begin
1996                                     dbg_irq_call <= 0;
1997                                     dbg_irq_enter <= 0;
1998                                     end else
1999                                     if (rvfi_valid) begin
2000                                     dbg_irq_call <= 0;
2001                                     dbg_irq_enter <= dbg_irq_call;
2002                                     end else
2003                                     if (irq_state == 1) begin
2004                                     dbg_irq_call <= 1;
2005                                     dbg_irq_ret <= next_pc;
2006                                     end
2007                                     if (!resetn) begin
2008                                     rvfi_rd_addr <= 0;
2009                                     rvfi_rd_wdata <= 0;
2010                                     end else
2011                                     if (cpuregs.write && !irq_state) begin
2012                                     `ifdef PICORV32_TESTBUG_003

```

```

2016          rvfi_rd_addr <= latched_rd ^ 1;
2017 `else
2018          rvfi_rd_addr <= latched_rd;
2019 `endif
2020 `ifdef PICORV32_TESTBUG_004
2021          rvfi_rd_wdata <= latched_rd ?
cpuregs_wdata ^ 1 : 0;
2022 `else
2023          rvfi_rd_wdata <= latched_rd ?
cpuregs_wdata : 0;
2024 `endif
2025          end else
2026          if (rvfi_valid) begin
2027              rvfi_rd_addr <= 0;
2028              rvfi_rd_wdata <= 0;
2029          end
2030
2031          casez (dbg_insn_opcode)
2032              32'b 0000000_????_000??_???_????
_0001011: begin // getq
2033              rvfi_rs1_addr <= 0;
2034              rvfi_rs1_rdata <= 0;
2035          end
2036              32'b 0000001_????_????_???_000??
_0001011: begin // setq
2037              rvfi_rd_addr <= 0;
2038              rvfi_rd_wdata <= 0;
2039          end
2040              32'b 0000010_????_00000_???
_00000_0001011: begin // retirq
2041              rvfi_rs1_addr <= 0;
2042              rvfi_rs1_rdata <= 0;
2043          end
2044          endcase
2045          if (!dbg_irq_call) begin
2046              if (dbg_mem_instr) begin
2047                  rvfi_mem_addr <= 0;
2048                  rvfi_mem_rmask <= 0;
2049                  rvfi_mem_wmask <= 0;
2050                  rvfi_mem_rdata <= 0;
2051                  rvfi_mem_wdata <= 0;
2052              end else
2053                  if (dbg_mem_valid && dbg_mem_ready) be-
gin
2054                      rvfi_mem_addr <= dbg_mem_addr;
2055                      rvfi_mem_rmask <=
2056                      rvfi_mem_wmask <=
2057                      rvfi_mem_rdata <=
2058                      rvfi_mem_wdata <=
2059                      end
2060                  end
2061              end
2062          end
2063          always @* begin
2064              `ifdef PICORV32_TESTBUG_005
2065                  rvfi_pc_wdata = (dbg_irq_call ? dbg_irq_ret :
2066                  rvfi_pc_wdata = dbg_irq_call ? dbg_irq_ret :
2067                  `endif
2068                  rvfi_pc_wdata = dbg_irq_call ? dbg_irq_ret :
2069                  `endif
2070
2071          rvfi_csr_mcycle_rmask = 0;
2072          rvfi_csr_mcycle_wmask = 0;
2073          rvfi_csr_mcycle_rdata = 0;
2074          rvfi_csr_mcycle_wdata = 0;
2075
2076          rvfi_csr_minstret_rmask = 0;
2077          rvfi_csr_minstret_wmask = 0;
2078          rvfi_csr_minstret_rdata = 0;
2079          rvfi_csr_minstret_wdata = 0;
2080
2081          if (rvfi_valid && rvfi_insn[6:0] == 7'b 1110011
&& rvfi_insn[13:12] == 3'b 010) begin
2082              if (rvfi_insn[31:20] == 12'h C00) begin
2083                  rvfi_csr_mcycle_rmask = 64'h
2084                  rvfi_csr_mcycle_rdata = {32'h
0000_0000_FFFF_FFFF;
2085                  rvfi_csr_mcycle_rdata = {32'h
0000_0000, rvfi_rd_wdata};
2086              end
2087              if (rvfi_insn[31:20] == 12'h C80) begin
2088                  rvfi_csr_mcycle_rmask = 64'h
2089                  rvfi_csr_mcycle_rdata =
2090                  rvfi_csr_mcycle_rdata =
2091                  rvfi_csr_minstret_rmask = 64'h
2092                  rvfi_csr_minstret_rdata = {32'h
0000_0000, rvfi_rd_wdata};
2093              end
2094              if (rvfi_insn[31:20] == 12'h C82) begin
2095                  rvfi_csr_minstret_rmask = 64'h

```

```

FFFF_FFFF_0000_0000;
2096          rvfi_csr_minstret_rdata =
{rvfi_rd_wdata, 32'h 0000_0000};
2097          end
2098          end
2099          end
2100          `endif
2101
2102          // Formal Verification
2103          `ifdef FORMAL
2104              reg [3:0] last_mem_nowait;
2105              always @(posedge clk)
2106                  last_mem_nowait <= {last_mem_nowait, mem_ready
|| !mem_valid};
2107
2108          // stall the memory interface for max 4 cycles
2109          restrict property (!last_mem_nowait || mem_ready || !
mem_valid);
2110
2111          // resetn low in first cycle, after that resetn high
2112          restrict property (resetn != $initstate);
2113
2114          // this just makes it much easier to read traces. un-
comment as needed.
2115          // assume property (mem_valid || !mem_ready);
2116
2117          reg ok;
2118          always @* begin
2119              if (resetn) begin
2120                  // instruction fetches are read-only
2121                  if (mem_valid && mem_instr)
2122                      assert (mem_wstrb == 0);
2123
2124                  // cpu_state must be valid
2125                  ok = 0;
2126
2127                  if (cpu_state == cpu_state_trap) ok =
1;
2128
2129                  if (cpu_state == cpu_state_fetch) ok =
1;
2130
2131                  if (cpu_state == cpu_state_ld_rs1) ok =
1;
2132                  if (cpu_state == cpu_state_ld_rs2) ok
= 1;
2133                  if (cpu_state == cpu_state_exec) ok =
1;
2134                  if (cpu_state == cpu_state_shift) ok =
1;
2135                  if (cpu_state == cpu_state_stmem) ok =
1;
2136                  if (cpu_state == cpu_state_ldmem) ok =
1;
2137                  assert (ok);
2138              end
2139              reg last_mem_la_read = 0;
2140              reg last_mem_la_write = 0;
2141              reg [31:0] last_mem_la_addr;
2142              reg [31:0] last_mem_la_wdata;
2143              reg [3:0] last_mem_la_wstrb = 0;
2144
2145              always @(posedge clk) begin
2146                  last_mem_la_read <= mem_la_read;
2147                  last_mem_la_write <= mem_la_write;
2148                  last_mem_la_addr <= mem_la_addr;
2149                  last_mem_la_wdata <= mem_la_wdata;
2150                  last_mem_la_wstrb <= mem_la_wstrb;
2151
2152                  if (last_mem_la_read) begin
2153                      assert(mem_valid);
2154                      assert(mem_addr == last_mem_la_addr);
2155                      assert(mem_wstrb == 0);
2156                  end
2157                  if (last_mem_la_write) begin
2158                      assert(mem_valid);
2159                      assert(mem_addr == last_mem_la_addr);
2160                      assert(mem_wdata == last_mem_la_wdata);
2161                      assert(mem_wstrb == last_mem_la_wstrb);
2162                  end
2163                  if (mem_la_read || mem_la_write) begin
2164                      assert(!mem_valid || mem_ready);
2165                  end
2166              end
2167          `endif
2168          endmodule
2169
2170          // This is a simple example implementation of PICORV32_REGS.
2171          // Use the PICORV32_REGS mechanism if you want to use custom
2172          // memory resources to implement the processor register file.
2173          // Note that your implementation must match the requirements
2174          of
2175          // the PicoRV32 configuration. (e.g. QREGS, etc)
2176          module picorv32_regs (
2177              input clk, wen,

```

```

2176     input [5:0] waddr,
2177     input [5:0] raddr1,
2178     input [5:0] raddr2,
2179     input [31:0] wdata,
2180     output [31:0] rdata1,
2181     output [31:0] rdata2
2182 );
2183     reg [31:0] regs [0:30];
2184
2185     always @(posedge clk)
2186         if (wen) regs[~waddr[4:0]] <= wdata;
2187
2188     assign rdata1 = regs[~raddr1[4:0]];
2189     assign rdata2 = regs[~raddr2[4:0]];
2190 endmodule
2191
2192 /
2193 *****
2194 * picorv32_pcpi_mul
2195 *****/
2196 module picorv32_pcpi_mul #(
2197     parameter STEPS_AT_ONCE = 1,
2198     parameter CARRY_CHAIN = 4
2199 ) (
2200     input clk, resetn,
2201
2202     input          pcpi_valid,
2203     input [31:0]   pcpi_insn,
2204     input [31:0]   pcpi_rs1,
2205     input [31:0]   pcpi_rs2,
2206     output reg     pcpi_wr,
2207     output reg [31:0] pcpi_rd,
2208     output reg     pcpi_wait,
2209     output reg     pcpi_ready
2210 );
2211     reg instr_mul, instr_mulh, instr_mulhsu, instr_mulhu;
2212     wire instr_any_mul = |{instr_mul, instr_mulh, in-
str_mulhsu, instr_mulhu};
2213     wire instr_any_mulh = |{instr_mulh, instr_mulhsu, in-
str_mulhu};
2214     wire instr_rs1_signed = |{instr_mulh, instr_mulhsu};
2215     wire instr_rs2_signed = |{instr_mulh};
2216
2217     reg pcpi_wait_q;
2218     wire mul_start = pcpi_wait && !pcpi_wait_q;
2219
2220     always @(posedge clk) begin
2221         instr_mul <= 0;
2222         instr_mulh <= 0;
2223         instr_mulhsu <= 0;
2224         instr_mulhu <= 0;
2225
2226         if (resetn && pcpi_valid && pcpi_insn[6:0] ==
7'b0110011 && pcpi_insn[31:25] == 7'b0000001) begin
2227             case (pcpi_insn[14:12])
2228                 3'b000: instr_mul <= 1;
2229                 3'b001: instr_mulh <= 1;
2230                 3'b010: instr_mulhsu <= 1;
2231                 3'b011: instr_mulhu <= 1;
2232             endcase
2233         end
2234
2235         pcpi_wait <= instr_any_mul;
2236         pcpi_wait_q <= pcpi_wait;
2237     end
2238
2239     reg [63:0] rs1, rs2, rd, rdx;
2240     reg [63:0] next_rs1, next_rs2, this_rs2;
2241     reg [63:0] next_rd, next_rdx, next_rdt;
2242     reg [6:0] mul_counter;
2243     reg mul_waiting;
2244     reg mul_finish;
2245     integer i, j;
2246
2247     // carry save accumulator
2248     always @* begin
2249         next_rd = rd;
2250         next_rdx = rdx;
2251         next_rs1 = rs1;
2252         next_rs2 = rs2;
2253
2254         for (i = 0; i < STEPS_AT_ONCE; i=i+1) begin
2255             this_rs2 = next_rs1[0] ? next_rs2 : 0;
2256             if (CARRY_CHAIN == 0) begin
2257                 next_rdt = next_rd ^ next_rdx ^
this_rs2;
2258                 next_rdx = ((next_rd &
next_rdx) | (next_rd & this_rs2) | (next_rdx & this_rs2)) << 1;
2259                 next_rd = next_rdt;
2260             end else begin
2261                 next_rdt = 0;
2262                 for (j = 0; j < 64; j = j +
CARRY_CHAIN)
2263                     {next_rdt[j+CARRY_CHAIN-1], next_rd[j +: CARRY_CHAIN]} =
{next_rdt[j+CARRY_CHAIN-1], next_rd[j +: CARRY_CHAIN] + this_rs2[j

```

```

+: CARRY_CHAIN];
2266         next_rdx = next_rdt << 1;
2267     end
2268     next_rs1 = next_rs1 >> 1;
2269     next_rs2 = next_rs2 << 1;
2270 end
2271 end
2272
2273     always @(posedge clk) begin
2274         mul_finish <= 0;
2275         if (!resetn) begin
2276             mul_waiting <= 1;
2277         end else
2278             if (mul_waiting) begin
2279                 if (instr_rs1_signed)
2280                     rs1 <= $signed(pcpi_rs1);
2281                 else
2282                     rs1 <= $unsigned(pcpi_rs1);
2283
2284                 if (instr_rs2_signed)
2285                     rs2 <= $signed(pcpi_rs2);
2286                 else
2287                     rs2 <= $unsigned(pcpi_rs2);
2288
2289                 rd <= 0;
2290                 rdx <= 0;
2291                 mul_counter <= (instr_any_mulh ? 63 -
STEPS_AT_ONCE : 31 - STEPS_AT_ONCE);
2292                 mul_waiting <= !mul_start;
2293             end else begin
2294                 rd <= next_rd;
2295                 rdx <= next_rdx;
2296                 rs1 <= next_rs1;
2297                 rs2 <= next_rs2;
2298
2299                 mul_counter <= mul_counter -
STEPS_AT_ONCE;
2300                 if (mul_counter[6]) begin
2301                     mul_finish <= 1;
2302                     mul_waiting <= 1;
2303                 end
2304             end
2305         end
2306
2307         always @(posedge clk) begin
2308             pcpi_wr <= 0;
2309             pcpi_ready <= 0;
2310             if (mul_finish && resetn) begin
2311                 pcpi_wr <= 1;
2312                 pcpi_ready <= 1;
2313                 pcpi_rd <= instr_any_mulh ? rd >> 32 :
rd;
2314             end
2315         end
2316     endmodule
2317
2318 module picorv32_pcpi_fast_mul #(
2319     parameter EXTRA_MUL_FFS = 0,
2320     parameter EXTRA_INSN_FFS = 0,
2321     parameter MUL_CLKGATE = 0
2322 ) (
2323     input clk, resetn,
2324
2325     input          pcpi_valid,
2326     input [31:0]   pcpi_insn,
2327     input [31:0]   pcpi_rs1,
2328     input [31:0]   pcpi_rs2,
2329     output reg     pcpi_wr,
2330     output reg [31:0] pcpi_rd,
2331     output reg     pcpi_wait,
2332     output reg     pcpi_ready
2333 );
2334     reg instr_mul, instr_mulh, instr_mulhsu, instr_mulhu;
2335     wire instr_any_mul = |{instr_mul, instr_mulh, in-
str_mulhsu, instr_mulhu};
2336     wire instr_any_mulh = |{instr_mulh, instr_mulhsu, in-
str_mulhu};
2337     wire instr_rs1_signed = |{instr_mulh, instr_mulhsu};
2338     wire instr_rs2_signed = |{instr_mulh};
2339
2340     reg shift_out;
2341     reg [3:0] active;
2342     reg [32:0] rs1, rs2, rs1_q, rs2_q;
2343     reg [63:0] rd, rd_q;
2344
2345     wire pcpi_insn_valid = pcpi_valid && pcpi_insn[6:0] ==
7'b0110011 && pcpi_insn[31:25] == 7'b0000001;
2346     reg pcpi_insn_valid_q;
2347
2348     always @* begin
2349         instr_mul = 0;
2350         instr_mulh = 0;
2351         instr_mulhsu = 0;
2352         instr_mulhu = 0;
2353
2354         if (resetn && (EXTRA_INSN_FFS ? pcpi_in-
sn_valid_q : pcpi_insn_valid)) begin
2355             case (pcpi_insn[14:12])
2356                 3'b000: instr_mul = 1;
2357                 3'b001: instr_mulh = 1;
2358                 3'b010: instr_mulhsu = 1;

```



```

2359             3'b011: instr_mulhu = 1;
2360         endcase
2361     end
2362 end
2363
2364 always @(posedge clk) begin
2365     pcpi_insn_valid_q <= pcpi_insn_valid;
2366     if (!MUL_CLKGATE || active[0]) begin
2367         rs1_q <= rs1;
2368         rs2_q <= rs2;
2369     end
2370     if (!MUL_CLKGATE || active[1]) begin
2371         rd <= $signed(EXTRA_MUL_FFS ? rs1_q :
2372 rs1) * $signed(EXTRA_MUL_FFS ? rs2_q : rs2);
2373     end
2374     if (!MUL_CLKGATE || active[2]) begin
2375         rd_q <= rd;
2376     end
2377 end
2378
2379 always @(posedge clk) begin
2380     if (instr_any_mul && !(EXTRA_MUL_FFS ? act-
2381 ive[3:0] : active[1:0])) begin
2382         if (instr_rs1_signed)
2383             rs1 <= $signed(pcpi_rs1);
2384         else
2385             rs1 <= $unsigned(pcpi_rs1);
2386
2387         if (instr_rs2_signed)
2388             rs2 <= $signed(pcpi_rs2);
2389         else
2390             rs2 <= $unsigned(pcpi_rs2);
2391         active[0] <= 1;
2392     end else begin
2393         active[0] <= 0;
2394     end
2395
2396     active[3:1] <= active;
2397     shift_out <= instr_any_mulh;
2398
2399     if (!resetsn)
2400         active <= 0;
2401 end
2402
2403 assign pcpi_wr = active[EXTRA_MUL_FFS ? 3 : 1];
2404 assign pcpi_wait = 0;
2405 assign pcpi_ready = active[EXTRA_MUL_FFS ? 3 : 1];
2406 `ifdef RISCV_FORMAL_ALTOPS
2407 assign pcpi_rd =
2408     instr_mul ? (pcpi_rs1 + pcpi_rs2) ^
2409 32'h5876063e :
2410     instr_mulh ? (pcpi_rs1 + pcpi_rs2) ^
2411 32'hf6583fb7 :
2412     instr_mulhsu ? (pcpi_rs1 - pcpi_rs2) ^
2413 32'hecfe137 :
2414     instr_mulhu ? (pcpi_rs1 + pcpi_rs2) ^
2415 32'h949ce5e8 : 1'bx;
2416 `else
2417     assign pcpi_rd = shift_out ? (EXTRA_MUL_FFS ? rd_q :
2418 rd) >> 32 : (EXTRA_MUL_FFS ? rd_q : rd);
2419 `endif
2420 endmodule
2421 /
2422 *****
2423 * picorv32_pcpi_div
2424 *****/
2425
2426 module picorv32_pcpi_div (
2427     input clk, resetsn,
2428     input pcpi_valid,
2429     input [31:0] pcpi_insn,
2430     input [31:0] pcpi_rs1,
2431     input [31:0] pcpi_rs2,
2432     output reg pcpi_wr,
2433     output reg [31:0] pcpi_rd,
2434     output reg pcpi_wait,
2435     output reg pcpi_ready
2436 );
2437     reg instr_div, instr_divu, instr_rem, instr_remu;
2438     wire instr_any_div_rem = |{instr_div, instr_divu, in-
2439 str_rem, instr_remu};
2440
2441     reg pcpi_wait_q;
2442     wire start = pcpi_wait && !pcpi_wait_q;
2443
2444     always @(posedge clk) begin
2445         instr_div <= 0;
2446         instr_divu <= 0;
2447         instr_rem <= 0;
2448         instr_remu <= 0;
2449
2450         if (resetsn && pcpi_valid && !pcpi_ready &&
2451 pcpi_insn[6:0] == 7'b0110011 && pcpi_insn[31:25] == 7'b0000001) be-
2452 gin
2453             case (pcpi_insn[14:12])
2454                 3'b100: instr_div <= 1;
2455                 3'b101: instr_divu <= 1;

```

```

2448             3'b110: instr_rem <= 1;
2449             3'b111: instr_remu <= 1;
2450         endcase
2451     end
2452
2453     pcpi_wait <= instr_any_div_rem && resetsn;
2454     pcpi_wait_q <= pcpi_wait && resetsn;
2455 end
2456
2457     reg [31:0] dividend;
2458     reg [62:0] divisor;
2459     reg [31:0] quotient;
2460     reg [31:0] quotient_msk;
2461     reg running;
2462     reg outsign;
2463
2464     always @(posedge clk) begin
2465         pcpi_ready <= 0;
2466         pcpi_wr <= 0;
2467         pcpi_rd <= 'bx;
2468
2469         if (!resetsn) begin
2470             running <= 0;
2471         end else
2472             if (start) begin
2473                 running <= 1;
2474
2475                 dividend <= (instr_div || instr_rem) &&
2476 pcpi_rs1[31] ? -pcpi_rs1 : pcpi_rs1;
2477
2478                 divisor <= ((instr_div || instr_rem) &&
2479 pcpi_rs2[31] ? -pcpi_rs2 : pcpi_rs2) << 31;
2480                 outsign <= (instr_div &&
2481 (pcpi_rs1[31] != pcpi_rs2[31]) && |pcpi_rs2) || (instr_rem &&
2482 pcpi_rs1[31]);
2483                 quotient <= 0;
2484                 quotient_msk <= 1 << 31;
2485             end else
2486                 if (!quotient_msk && running) begin
2487                     running <= 0;
2488                     pcpi_ready <= 1;
2489                     pcpi_wr <= 1;
2490                     `ifdef RISCV_FORMAL_ALTOPS
2491                     case (1)
2492                         instr_div: pcpi_rd <=
2493 (pcpi_rs1 - pcpi_rs2) ^ 32'h7f8529ec;
2494                         instr_divu: pcpi_rd <=
2495 (pcpi_rs1 - pcpi_rs2) ^ 32'h10e8fd70;
2496                         instr_rem: pcpi_rd <=
2497 (pcpi_rs1 - pcpi_rs2) ^ 32'h8da68fa5;
2498                         instr_remu: pcpi_rd <=
2499 (pcpi_rs1 - pcpi_rs2) ^ 32'h3138d0e1;
2500                     endcase
2501                     `else
2502                         if (instr_div || instr_divu)
2503                             pcpi_rd <= outsign ? -quo-
2504 tient : quotient;
2505                         else
2506                             pcpi_rd <= outsign ? -di-
2507 vidend : dividend;
2508                     `endif
2509                 end else begin
2510                     if (divisor <= dividend) begin
2511                         dividend <= dividend - divisor;
2512                         quotient <= quotient | quo-
2513 tient_msk;
2514                     end
2515                     divisor <= divisor >> 1;
2516                 `endif RISCV_FORMAL_ALTOPS
2517                 quotient_msk <= quotient_msk >> 5;
2518             `else
2519                 quotient_msk <= quotient_msk >> 1;
2520             `endif
2521         end
2522     end
2523 endmodule
2524 /
2525 *****
2526 * picorv32_axi
2527 *****/
2528
2529 module picorv32_axi #(
2530     parameter [0:0] ENABLE_COUNTERS = 1,
2531     parameter [0:0] ENABLE_COUNTERS64 = 1,
2532     parameter [0:0] ENABLE_REGS_16_31 = 1,
2533     parameter [0:0] ENABLE_REGS_DUALPORT = 1,
2534     parameter [0:0] TWO_STAGE_SHIFT = 1,
2535     parameter [0:0] BARREL_SHIFTER = 0,
2536     parameter [0:0] TWO_CYCLE_COMPARE = 0,
2537     parameter [0:0] TWO_CYCLE_ALU = 0,
2538     parameter [0:0] COMPRESSED_ISA = 0,
2539     parameter [0:0] CATCH_MISALIGN = 1,
2540     parameter [0:0] CATCH_ILL_INSN = 1,
2541     parameter [0:0] ENABLE_PCPI = 0,
2542     parameter [0:0] ENABLE_MUL = 0,
2543     parameter [0:0] ENABLE_FAST_MUL = 0,
2544     parameter [0:0] ENABLE_DIV = 0,
2545     parameter [0:0] ENABLE_IRQ = 0,

```

```

2534 parameter [ 0:0] ENABLE_IRQ_QREGS = 1,
2535 parameter [ 0:0] ENABLE_IRQ_TIMER = 1,
2536 parameter [ 0:0] ENABLE_TRACE = 0,
2537 parameter [ 0:0] REGS_INIT_ZERO = 0,
2538 parameter [31:0] MASKED_IRQ = 32'h 0000_0000,
2539 parameter [31:0] LATCHED_IRQ = 32'h ffff_ffff,
2540 parameter [31:0] PROGADDR_RESET = 32'h 0000_0000,
2541 parameter [31:0] PROGADDR_IRQ = 32'h 0000_0010,
2542 parameter [31:0] STACKADDR = 32'h ffff_ffff
2543 ) (
2544     input clk, resetn,
2545     output trap,
2546
2547     // AXI4-lite master memory interface
2548
2549     output mem_axi_awvalid,
2550     input mem_axi_awready,
2551     output [31:0] mem_axi_awaddr,
2552     output [ 2:0] mem_axi_awprot,
2553
2554     output mem_axi_wvalid,
2555     input mem_axi_wready,
2556     output [31:0] mem_axi_wdata,
2557     output [ 3:0] mem_axi_wstrb,
2558
2559     input mem_axi_bvalid,
2560     output mem_axi_bready,
2561
2562     output mem_axi_rvalid,
2563     input mem_axi_rready,
2564     output [31:0] mem_axi_raddr,
2565     output [ 2:0] mem_axi_rprot,
2566
2567     input mem_axi_rvalid,
2568     output mem_axi_rready,
2569     input [31:0] mem_axi_rdata,
2570
2571     // Pico Co-Processor Interface (PCPI)
2572     output pcpi_valid,
2573     output [31:0] pcpi_insn,
2574     output [31:0] pcpi_rs1,
2575     output [31:0] pcpi_rs2,
2576     input pcpi_wr,
2577     input [31:0] pcpi_rd,
2578     input pcpi_wait,
2579     input pcpi_ready,
2580
2581     // IRQ interface
2582     input [31:0] irq,
2583     output [31:0] eoi,
2584
2585 `ifdef RISC_V Formal
2586     output rvfi_valid,
2587     output [63:0] rvfi_order,
2588     output [31:0] rvfi_insn,
2589     output rvfi_trap,
2590     output rvfi_halt,
2591     output rvfi_intr,
2592     output [ 4:0] rvfi_rs1_addr,
2593     output [ 4:0] rvfi_rs2_addr,
2594     output [31:0] rvfi_rs1_rdata,
2595     output [31:0] rvfi_rs2_rdata,
2596     output [ 4:0] rvfi_rd_addr,
2597     output [31:0] rvfi_rd_wdata,
2598     output [31:0] rvfi_pc_rdata,
2599     output [31:0] rvfi_pc_wdata,
2600     output [31:0] rvfi_mem_addr,
2601     output [ 3:0] rvfi_mem_rmask,
2602     output [ 3:0] rvfi_mem_wmask,
2603     output [31:0] rvfi_mem_rdata,
2604     output [31:0] rvfi_mem_wdata,
2605 `endif
2606
2607 // Trace Interface
2608 output trace_valid,
2609 output [35:0] trace_data
2610 );
2611
2612 wire mem_valid;
2613 wire [31:0] mem_addr;
2614 wire [31:0] mem_wdata;
2615 wire [ 3:0] mem_wstrb;
2616 wire mem_instr;
2617 wire mem_ready;
2618 wire [31:0] mem_rdata;
2619
2620 picorv32_axi_adapter axi_adapter (
2621     .clk (clk),
2622     .resetn (resetn),
2623     .mem_axi_awvalid (mem_axi_awvalid),
2624     .mem_axi_awready (mem_axi_awready),
2625     .mem_axi_awaddr (mem_axi_awaddr),
2626     .mem_axi_awprot (mem_axi_awprot),
2627     .mem_axi_wvalid (mem_axi_wvalid),
2628     .mem_axi_wready (mem_axi_wready),
2629     .mem_axi_wdata (mem_axi_wdata),
2630     .mem_axi_wstrb (mem_axi_wstrb),
2631     .mem_axi_bvalid (mem_axi_bvalid),
2632     .mem_axi_bready (mem_axi_bready),
2633     .mem_axi_rvalid (mem_axi_rvalid),
2634     .mem_axi_rready (mem_axi_rready),
2635     .mem_axi_raddr (mem_axi_raddr),

```

```

2635     .mem_axi_arprot (mem_axi_arprot),
2636     .mem_axi_rvalid (mem_axi_rvalid),
2637     .mem_axi_rready (mem_axi_rready),
2638     .mem_axi_rdata (mem_axi_rdata),
2639     .mem_valid (mem_valid),
2640     .mem_instr (mem_instr),
2641     .mem_ready (mem_ready),
2642     .mem_addr (mem_addr),
2643     .mem_wdata (mem_wdata),
2644     .mem_wstrb (mem_wstrb),
2645     .mem_rdata (mem_rdata)
2646 );
2647
2648 picorv32 #(
2649     .ENABLE_COUNTERS (ENABLE_COUNTERS),
2650     .ENABLE_COUNTERS64 (ENABLE_COUNTERS64),
2651     .ENABLE_REGS_16_31 (ENABLE_REGS_16_31),
2652     .ENABLE_REGS_DUALPORT (ENABLE_REGS_DUALPORT),
2653     .TWO_STAGE_SHIFT (TWO_STAGE_SHIFT),
2654     .BARREL_SHIFTER (BARREL_SHIFTER),
2655     .TWO_CYCLE_COMPARE (TWO_CYCLE_COMPARE),
2656     .TWO_CYCLE_ALU (TWO_CYCLE_ALU),
2657     .COMPRESSED_ISA (COMPRESSED_ISA),
2658     .CATCH_MISALIGN (CATCH_MISALIGN),
2659     .CATCH_ILLLNSN (CATCH_ILLLNSN),
2660     .ENABLE_PCPI (ENABLE_PCPI),
2661     .ENABLE_MUL (ENABLE_MUL),
2662     .ENABLE_FAST_MUL (ENABLE_FAST_MUL),
2663     .ENABLE_DIV (ENABLE_DIV),
2664     .ENABLE_IRQ (ENABLE_IRQ),
2665     .ENABLE_IRQ_QREGS (ENABLE_IRQ_QREGS),
2666     .ENABLE_IRQ_TIMER (ENABLE_IRQ_TIMER),
2667     .ENABLE_TRACE (ENABLE_TRACE),
2668     .REGS_INIT_ZERO (REGS_INIT_ZERO),
2669     .MASKED_IRQ (MASKED_IRQ),
2670     .LATCHED_IRQ (LATCHED_IRQ),
2671     .PROGADDR_RESET (PROGADDR_RESET),
2672     .PROGADDR_IRQ (PROGADDR_IRQ),
2673     .STACKADDR (STACKADDR)
2674 ) picorv32_core (
2675     .clk (clk),
2676     .resetn (resetn),
2677     .trap (trap),
2678
2679     .mem_valid (mem_valid),
2680     .mem_addr (mem_addr),
2681     .mem_wdata (mem_wdata),
2682     .mem_wstrb (mem_wstrb),
2683     .mem_instr (mem_instr),
2684     .mem_ready (mem_ready),
2685     .mem_rdata (mem_rdata),
2686
2687     .pcpi_valid (pcpi_valid),
2688     .pcpi_insn (pcpi_insn),
2689     .pcpi_rs1 (pcpi_rs1),
2690     .pcpi_rs2 (pcpi_rs2),
2691     .pcpi_wr (pcpi_wr),
2692     .pcpi_rd (pcpi_rd),
2693     .pcpi_wait (pcpi_wait),
2694     .pcpi_ready (pcpi_ready),
2695
2696     .irq (irq),
2697     .eoi (eoi),
2698
2699 `ifdef RISC_V Formal
2700     .rvfi_valid (rvfi_valid),
2701     .rvfi_order (rvfi_order),
2702     .rvfi_insn (rvfi_insn),
2703     .rvfi_trap (rvfi_trap),
2704     .rvfi_halt (rvfi_halt),
2705     .rvfi_intr (rvfi_intr),
2706     .rvfi_rs1_addr (rvfi_rs1_addr),
2707     .rvfi_rs2_addr (rvfi_rs2_addr),
2708     .rvfi_rs1_rdata (rvfi_rs1_rdata),
2709     .rvfi_rs2_rdata (rvfi_rs2_rdata),
2710     .rvfi_rd_addr (rvfi_rd_addr),
2711     .rvfi_rd_wdata (rvfi_rd_wdata),
2712     .rvfi_pc_rdata (rvfi_pc_rdata),
2713     .rvfi_pc_wdata (rvfi_pc_wdata),
2714     .rvfi_mem_addr (rvfi_mem_addr),
2715     .rvfi_mem_rmask (rvfi_mem_rmask),
2716     .rvfi_mem_wmask (rvfi_mem_wmask),
2717     .rvfi_mem_rdata (rvfi_mem_rdata),
2718     .rvfi_mem_wdata (rvfi_mem_wdata),
2719 `endif
2720
2721     .trace_valid (trace_valid),
2722     .trace_data (trace_data)
2723 );
2724 endmodule
2725
2726
2727 /
2728 * picorv32_axi_adapter
2729
2730
2731 module picorv32_axi_adapter (
2732     input clk, resetn,
2733

```

```

2734 // AXI4-lite master memory interface
2735
2736 output      mem_axi_avalid,
2737 input       mem_axi_awready,
2738 output [31:0] mem_axi_awaddr,
2739 output [ 2:0] mem_axi_awprot,
2740
2741 output      mem_axi_wvalid,
2742 input       mem_axi_wready,
2743 output [31:0] mem_axi_wdata,
2744 output [ 3:0] mem_axi_wstrb,
2745
2746 input       mem_axi_bvalid,
2747 output      mem_axi_bready,
2748
2749 output      mem_axi_arvalid,
2750 input       mem_axi_arready,
2751 output [31:0] mem_axi_araddr,
2752 output [ 2:0] mem_axi_arprot,
2753
2754 input       mem_axi_rvalid,
2755 output      mem_axi_rready,
2756 input [31:0] mem_axi_rdata,
2757
2758 // Native PicoRV32 memory interface
2759
2760 input       mem_valid,
2761 input       mem_instr,
2762 output      mem_ready,
2763 input [31:0] mem_addr,
2764 input [31:0] mem_wdata,
2765 input [ 3:0] mem_wstrb,
2766 output [31:0] mem_rdata
2767 );
2768 reg ack_avalid;
2769 reg ack_arvalid;
2770 reg ack_wvalid;
2771 reg xfer_done;
2772
2773 assign mem_axi_avalid = mem_valid && !mem_wstrb && !
ack_avalid;
2774 assign mem_axi_awaddr = mem_addr;
2775 assign mem_axi_awprot = 0;
2776
2777 assign mem_axi_arvalid = mem_valid && !mem_wstrb && !
ack_arvalid;
2778 assign mem_axi_araddr = mem_addr;
2779 assign mem_axi_arprot = mem_instr ? 3'b100 : 3'b000;
2780
2781 assign mem_axi_wvalid = mem_valid && !mem_wstrb && !
ack_wvalid;
2782 assign mem_axi_wdata = mem_wdata;
2783 assign mem_axi_wstrb = mem_wstrb;
2784
2785 assign mem_ready = mem_axi_bvalid || mem_axi_rvalid;
2786 assign mem_axi_bready = mem_valid && !mem_wstrb;
2787 assign mem_axi_rready = mem_valid && !mem_wstrb;
2788 assign mem_rdata = mem_axi_rdata;
2789
2790 always @(posedge clk) begin
2791     if (!resetsn) begin
2792         ack_avalid <= 0;
2793     end else begin
2794         xfer_done <= mem_valid && mem_ready;
2795         if (mem_axi_awready && mem_axi_avalid)
2796             ack_avalid <= 1;
2797         if (mem_axi_arready && mem_axi_arvalid)
2798             ack_arvalid <= 1;
2799         if (mem_axi_wready && mem_axi_wvalid)
2800             ack_wvalid <= 1;
2801         if (xfer_done || !mem_valid) begin
2802             ack_avalid <= 0;
2803             ack_arvalid <= 0;
2804             ack_wvalid <= 0;
2805         end
2806     end
2807 end
2808 endmodule
2809
2810
2811 /
2812 * picorv32_wb
2813
2814 *****
2815 module picorv32_wb #(
2816     parameter [ 0:0] ENABLE_COUNTERS = 1,
2817     parameter [ 0:0] ENABLE_COUNTERS64 = 1,
2818     parameter [ 0:0] ENABLE_REGS_16_31 = 1,
2819     parameter [ 0:0] ENABLE_REGS_DUALPORT = 1,
2820     parameter [ 0:0] TWO_STAGE_SHIFT = 1,
2821     parameter [ 0:0] BARREL_SHIFTER = 0,
2822     parameter [ 0:0] TWO_CYCLE_COMPARE = 0,
2823     parameter [ 0:0] TWO_CYCLE_ALU = 0,
2824     parameter [ 0:0] COMPRESSED_ISA = 0,
2825     parameter [ 0:0] CATCH_MISALIGN = 1,
2826     parameter [ 0:0] CATCH_ILLINSN = 1,
2827     parameter [ 0:0] ENABLE_PCPI = 0,
2828     parameter [ 0:0] ENABLE_MUL = 0,
2829     parameter [ 0:0] ENABLE_FAST_MUL = 0,

```

```

2830     parameter [ 0:0] ENABLE_DIV = 0,
2831     parameter [ 0:0] ENABLE_IRQ = 0,
2832     parameter [ 0:0] ENABLE_IRQ_QREGS = 1,
2833     parameter [ 0:0] ENABLE_IRQ_TIMER = 1,
2834     parameter [ 0:0] ENABLE_TRACE = 0,
2835     parameter [ 0:0] REGS_INIT_ZERO = 0,
2836     parameter [31:0] MASKED_IRQ = 32'h 0000_0000,
2837     parameter [31:0] LATCHED_IRQ = 32'h ffff_ffff,
2838     parameter [31:0] PROGADDR_RESET = 32'h 0000_0000,
2839     parameter [31:0] PROGADDR_IRQ = 32'h 0000_0010,
2840     parameter [31:0] STACKADDR = 32'h ffff_ffff
2841 ) (
2842     output trap,
2843
2844     // Wishbone interfaces
2845     input wb_rst_i,
2846     input wb_clk_i,
2847
2848     output reg [31:0] wbm_adr_o,
2849     output reg [31:0] wbm_dat_o,
2850     input [31:0] wbm_dat_i,
2851     output reg wbm_we_o,
2852     output reg [3:0] wbm_sel_o,
2853     output reg wbm_stb_o,
2854     input wbm_ack_i,
2855     output reg wbm_cyc_o,
2856
2857     // Pico Co-Processor Interface (PCPI)
2858     output      pcpi_valid,
2859     output [31:0] pcpi_insn,
2860     output [31:0] pcpi_rst1,
2861     output [31:0] pcpi_rst2,
2862     input       pcpi_wr,
2863     input [31:0] pcpi_rd,
2864     input       pcpi_wait,
2865     input       pcpi_ready,
2866
2867     // IRQ interface
2868     input [31:0] irq,
2869     output [31:0] eoi,
2870
2871 `ifdef RISC_V_FORMAL
2872     output      rvfi_valid,
2873     output [63:0] rvfi_order,
2874     output [31:0] rvfi_insn,
2875     output      rvfi_trap,
2876     output      rvfi_halt,
2877     output      rvfi_intr,
2878     output [ 4:0] rvfi_rst1_addr,
2879     output [ 4:0] rvfi_rst2_addr,
2880     output [31:0] rvfi_rst1_rdata,
2881     output [31:0] rvfi_rst2_rdata,
2882     output [ 4:0] rvfi_rd_addr,
2883     output [31:0] rvfi_rd_wdata,
2884     output [31:0] rvfi_pc_rdata,
2885     output [31:0] rvfi_pc_wdata,
2886     output [31:0] rvfi_mem_addr,
2887     output [ 3:0] rvfi_mem_rmask,
2888     output [ 3:0] rvfi_mem_wmask,
2889     output [31:0] rvfi_mem_rdata,
2890     output [31:0] rvfi_mem_wdata,
2891 `endif
2892
2893     // Trace Interface
2894     output      trace_valid,
2895     output [35:0] trace_data,
2896
2897     output mem_instr
2898 );
2899
2900 wire      mem_valid;
2901 wire [31:0] mem_addr;
2902 wire [31:0] mem_wdata;
2903 wire [ 3:0] mem_wstrb;
2904 reg      mem_ready;
2905 reg [31:0] mem_rdata;
2906
2907 wire clk;
2908 wire resetsn;
2909
2910 assign clk = wb_clk_i;
2911 assign resetsn = ~wb_rst_i;
2912
2913 picorv32 #(
2914     .ENABLE_COUNTERS (ENABLE_COUNTERS),
2915     .ENABLE_COUNTERS64 (ENABLE_COUNTERS64),
2916     .ENABLE_REGS_16_31 (ENABLE_REGS_16_31),
2917     .ENABLE_REGS_DUALPORT (ENABLE_REGS_DUALPORT),
2918     .TWO_STAGE_SHIFT (TWO_STAGE_SHIFT),
2919     .BARREL_SHIFTER (BARREL_SHIFTER),
2920     .TWO_CYCLE_COMPARE (TWO_CYCLE_COMPARE),
2921     .TWO_CYCLE_ALU (TWO_CYCLE_ALU),
2922     .COMPRESSED_ISA (COMPRESSED_ISA),
2923     .CATCH_MISALIGN (CATCH_MISALIGN),
2924     .CATCH_ILLINSN (CATCH_ILLINSN),
2925     .ENABLE_PCPI (ENABLE_PCPI),
2926     .ENABLE_MUL (ENABLE_MUL),
2927     .ENABLE_FAST_MUL (ENABLE_FAST_MUL),
2928     .ENABLE_DIV (ENABLE_DIV),
2929     .ENABLE_IRQ (ENABLE_IRQ),
2930     .ENABLE_IRQ_QREGS (ENABLE_IRQ_QREGS),
2931     .ENABLE_IRQ_TIMER (ENABLE_IRQ_TIMER),

```

```

2931         .ENABLE_TRACE      (ENABLE_TRACE      ),
2932         .REGS_INIT_ZERO    (REGS_INIT_ZERO    ),
2933         .MASKED_IRQ        (MASKED_IRQ        ),
2934         .LATCHED_IRQ       (LATCHED_IRQ       ),
2935         .PROGADDR_RESET    (PROGADDR_RESET    ),
2936         .PROGADDR_IRQ      (PROGADDR_IRQ      ),
2937         .STACKADDR         (STACKADDR         )
2938     ) picorv32_core (
2939         .clk      (clk      ),
2940         .resetn   (resetn   ),
2941         .trap     (trap     ),
2942
2943         .mem_valid(mem_valid),
2944         .mem_addr  (mem_addr ),
2945         .mem_wdata (mem_wdata),
2946         .mem_wstrb (mem_wstrb),
2947         .mem_instr (mem_instr),
2948         .mem_ready (mem_ready),
2949         .mem_rdata (mem_rdata),
2950
2951         .pcpi_valid(pcpi_valid),
2952         .pcpi_insn  (pcpi_insn ),
2953         .pcpi_rs1   (pcpi_rs1  ),
2954         .pcpi_rs2   (pcpi_rs2  ),
2955         .pcpi_wr     (pcpi_wr   ),
2956         .pcpi_rd     (pcpi_rd   ),
2957         .pcpi_wait   (pcpi_wait ),
2958         .pcpi_ready  (pcpi_ready),
2959
2960         .irq(irq),
2961         .eoi(eoi),
2962
2963     `ifdef RISC_V_FORMAL
2964         .rvfi_valid  (rvfi_valid  ),
2965         .rvfi_order  (rvfi_order  ),
2966         .rvfi_insn   (rvfi_insn   ),
2967         .rvfi_trap   (rvfi_trap   ),
2968         .rvfi_halt   (rvfi_halt   ),
2969         .rvfi_intr   (rvfi_intr   ),
2970         .rvfi_rs1_addr (rvfi_rs1_addr ),
2971         .rvfi_rs2_addr (rvfi_rs2_addr ),
2972         .rvfi_rs1_rdata (rvfi_rs1_rdata),
2973         .rvfi_rs2_rdata (rvfi_rs2_rdata),
2974         .rvfi_rd_addr  (rvfi_rd_addr ),
2975         .rvfi_rd_wdata (rvfi_rd_wdata ),
2976         .rvfi_pc_rdata (rvfi_pc_rdata ),
2977         .rvfi_pc_wdata (rvfi_pc_wdata ),
2978         .rvfi_mem_addr (rvfi_mem_addr ),
2979         .rvfi_mem_rmask (rvfi_mem_rmask),
2980         .rvfi_mem_wmask (rvfi_mem_wmask),
2981         .rvfi_mem_rdata (rvfi_mem_rdata),
2982         .rvfi_mem_wdata (rvfi_mem_wdata),
2983     `endif
2984
2985         .trace_valid(trace_valid),
2986         .trace_data  (trace_data)
2987     );
2988
2989     localparam IDLE = 2'b00;
2990     localparam WBSTART = 2'b01;
2991     localparam WBEND = 2'b10;
2992
2993     reg [1:0] state;
2994
2995     wire we;
2996
2997     assign we = (mem_wstrb[0] | mem_wstrb[1] | mem_wstrb[2]
2998 | mem_wstrb[3]);
2999
3000     always @(posedge wb_clk_i) begin
3001         if (wb_rst_i) begin
3002             wbm_adr_o <= 0;
3003             wbm_dat_o <= 0;
3004             wbm_we_o <= 0;
3005             wbm_sel_o <= 0;
3006             wbm_stb_o <= 0;
3007             wbm_cyc_o <= 0;
3008             state <= IDLE;
3009         end else begin
3010             case (state)
3011                 IDLE: begin
3012                     if (mem_valid) begin
3013                         wbm_adr_o <=
3014                             mem_addr;
3015                         wbm_dat_o <=
3016                             mem_wdata;
3017                         wbm_we_o <= we;
3018                         wbm_sel_o <=
3019                             mem_wstrb;
3020                         wbm_stb_o <=
3021                             1'b1;
3022                         wbm_cyc_o <=
3023                             1'b1;
3024                         state <=
3025                             WBSTART;
3026                     end else begin
3027                         mem_ready <=
3028                             1'b0;
3029                         state <=
3030                             IDLE;
3031                     end
3032                 end
3033             endcase
3034         end
3035     end

```

```

1'b0;
3023             wbm_cyc_o <=
1'b0;
3024             wbm_we_o <=
1'b0;
3025             end
3026         end
3027     end
3028     WBSTART: begin
3029         if (wbm_ack_i) begin
3030             mem_rdata <=
3031                 mem_ready <=
3032                 state <= WBEND;
3033             wbm_stb_o <=
3034             wbm_cyc_o <=
3035             wbm_we_o <=
3036             end
3037         end
3038     end
3039     WBEND: begin
3040         mem_ready <= 1'b0;
3041         state <= IDLE;
3042     end
3043     default:
3044         state <= IDLE;
3045     endcase
3046 end
3047 end
3048 end
3049 endmodule

```

2. YosysHQ_picorv32/testbench.v (479 lines)

```

1 // This is free and unencumbered software released into the
2 public domain.
3 // Anyone is free to copy, modify, publish, use, compile,
4 // distribute this software, either in source code form or as
5 // a compiled binary, for any purpose, commercial or non-commercial, and
6 // by any means.
7
8 `timescale 1 ns / 1 ps
9
10 `ifndef VERILATOR
11 module testbench #(
12     parameter AXI_TEST = 0,
13     parameter VERBOSE = 0
14 );
15     reg clk = 1;
16     reg resetn = 0;
17     wire trap;
18
19     always #5 clk = ~clk;
20
21     initial begin
22         repeat (100) @(posedge clk);
23         resetn <= 1;
24     end
25
26     initial begin
27         if ($test$plusargs("vcd")) begin
28             $dumpfile("testbench.vcd");
29             $dumpvars(0, testbench);
30         end
31         repeat (1000000) @(posedge clk);
32         $display("TIMEOUT");
33         $finish;
34     end
35
36     wire trace_valid;
37     wire [35:0] trace_data;
38     integer trace_file;
39
40     initial begin
41         if ($test$plusargs("trace")) begin
42             trace_file = $fopen("testbench.trace",
43 "w");
44             repeat (10) @(posedge clk);
45             while (!trap) begin
46                 @(posedge clk);
47                 if (trace_valid)
48                     $fwrite(trace_file,
49 "%x\n", trace_data);
50             end
51             $fclose(trace_file);
52             $display("Finished writing test-
53 bench.trace.");
54         end
55     end
56
57     picorv32_wrapper #(
58         .AXI_TEST (AXI_TEST),

```

```

56         .VERBOSE (VERBOSE)
57     ) top (
58         .clk(clk),
59         .resetn(resetn),
60         .trap(trap),
61         .trace_valid(trace_valid),
62         .trace_data(trace_data)
63     );
64 endmodule
65 `endif
66
67 module picorv32_wrapper #(
68     parameter AXI_TEST = 0,
69     parameter VERBOSE = 0
70 ) (
71     input clk,
72     input resetn,
73     output trap,
74     output trace_valid,
75     output [35:0] trace_data
76 );
77     wire tests_passed;
78     reg [31:0] irq = 0;
79
80     reg [15:0] count_cycle = 0;
81     always @(posedge clk) count_cycle <= resetn ?
count_cycle + 1 : 0;
82
83     always @* begin
84         irq = 0;
85         irq[4] = &count_cycle[12:0];
86         irq[5] = &count_cycle[15:0];
87     end
88
89     wire      mem_axi_awvalid;
90     wire      mem_axi_awready;
91     wire [31:0] mem_axi_awaddr;
92     wire [ 2:0] mem_axi_awprot;
93
94     wire      mem_axi_wvalid;
95     wire      mem_axi_wready;
96     wire [31:0] mem_axi_wdata;
97     wire [ 3:0] mem_axi_wstrb;
98
99     wire      mem_axi_bvalid;
100    wire      mem_axi_bready;
101
102    wire      mem_axi_rvalid;
103    wire      mem_axi_rready;
104    wire [31:0] mem_axi_raddr;
105    wire [ 2:0] mem_axi_rprot;
106
107    wire      mem_axi_rvalid;
108    wire      mem_axi_rready;
109    wire [31:0] mem_axi_rdata;
110
111    axi4_memory #(
112        .AXI_TEST (AXI_TEST),
113        .VERBOSE (VERBOSE)
114    ) mem (
115        .clk (clk),
116        .mem_axi_awvalid (mem_axi_awvalid),
117        .mem_axi_awready (mem_axi_awready),
118        .mem_axi_awaddr (mem_axi_awaddr),
119        .mem_axi_awprot (mem_axi_awprot),
120
121        .mem_axi_wvalid (mem_axi_wvalid),
122        .mem_axi_wready (mem_axi_wready),
123        .mem_axi_wdata (mem_axi_wdata),
124        .mem_axi_wstrb (mem_axi_wstrb),
125
126        .mem_axi_bvalid (mem_axi_bvalid),
127        .mem_axi_bready (mem_axi_bready),
128
129        .mem_axi_rvalid (mem_axi_rvalid),
130        .mem_axi_rready (mem_axi_rready),
131        .mem_axi_raddr (mem_axi_raddr),
132        .mem_axi_rprot (mem_axi_rprot),
133
134        .mem_axi_rvalid (mem_axi_rvalid),
135        .mem_axi_rready (mem_axi_rready),
136        .mem_axi_rdata (mem_axi_rdata),
137
138        .tests_passed (tests_passed)
139    );
140
141 `ifdef RISC_V_FORMAL
142     wire      rvfi_valid;
143     wire [63:0] rvfi_order;
144     wire [31:0] rvfi_insn;
145     wire      rvfi_trap;
146     wire      rvfi_halt;
147     wire      rvfi_intr;
148     wire [4:0] rvfi_rs1_addr;
149     wire [4:0] rvfi_rs2_addr;
150     wire [31:0] rvfi_rs1_rdata;
151     wire [31:0] rvfi_rs2_rdata;
152     wire [4:0] rvfi_rd_addr;
153     wire [31:0] rvfi_rd_wdata;
154     wire [31:0] rvfi_pc_rdata;
155     wire [31:0] rvfi_pc_wdata;

```

```

156     wire [31:0] rvfi_mem_addr;
157     wire [3:0]  rvfi_mem_rmask;
158     wire [3:0]  rvfi_mem_wmask;
159     wire [31:0] rvfi_mem_rdata;
160     wire [31:0] rvfi_mem_wdata;
161 `endif
162
163     picorv32_axi #(
164 `ifndef SYNTH_TEST
165 `ifdef SP_TEST
166         .ENABLE_REGS_DUALPORT(0),
167 `endif
168 `ifdef COMPRESSED_ISA
169         .COMPRESSED_ISA(1),
170 `endif
171         .ENABLE_MUL(1),
172         .ENABLE_DIV(1),
173         .ENABLE_IRQ(1),
174         .ENABLE_TRACE(1)
175 `endif
176     ) uut (
177         .clk (clk),
178         .resetn (resetn),
179         .trap (trap),
180         .mem_axi_awvalid(mem_axi_awvalid),
181         .mem_axi_awready(mem_axi_awready),
182         .mem_axi_awaddr (mem_axi_awaddr),
183         .mem_axi_awprot (mem_axi_awprot),
184         .mem_axi_wvalid (mem_axi_wvalid),
185         .mem_axi_wready (mem_axi_wready),
186         .mem_axi_wdata (mem_axi_wdata),
187         .mem_axi_wstrb (mem_axi_wstrb),
188         .mem_axi_bvalid (mem_axi_bvalid),
189         .mem_axi_bready (mem_axi_bready),
190         .mem_axi_rvalid(mem_axi_rvalid),
191         .mem_axi_rready(mem_axi_rready),
192         .mem_axi_raddr (mem_axi_raddr),
193         .mem_axi_rprot (mem_axi_rprot),
194         .mem_axi_rvalid (mem_axi_rvalid),
195         .mem_axi_rready (mem_axi_rready),
196         .mem_axi_rdata (mem_axi_rdata),
197         .irq (irq),
198 `ifdef RISC_V_FORMAL
199         .rvfi_valid (rvfi_valid),
200         .rvfi_order (rvfi_order),
201         .rvfi_insn (rvfi_insn),
202         .rvfi_trap (rvfi_trap),
203         .rvfi_halt (rvfi_halt),
204         .rvfi_intr (rvfi_intr),
205         .rvfi_rs1_addr (rvfi_rs1_addr),
206         .rvfi_rs2_addr (rvfi_rs2_addr),
207         .rvfi_rs1_rdata (rvfi_rs1_rdata),
208         .rvfi_rs2_rdata (rvfi_rs2_rdata),
209         .rvfi_rd_addr (rvfi_rd_addr),
210         .rvfi_rd_wdata (rvfi_rd_wdata),
211         .rvfi_pc_rdata (rvfi_pc_rdata),
212         .rvfi_pc_wdata (rvfi_pc_wdata),
213         .rvfi_mem_addr (rvfi_mem_addr),
214         .rvfi_mem_rmask (rvfi_mem_rmask),
215         .rvfi_mem_wmask (rvfi_mem_wmask),
216         .rvfi_mem_rdata (rvfi_mem_rdata),
217         .rvfi_mem_wdata (rvfi_mem_wdata),
218 `endif
219         .trace_valid (trace_valid),
220         .trace_data (trace_data)
221     );
222
223 `ifdef RISC_V_FORMAL
224     picorv32_rvfimon rvfimon (
225         .clock (clk),
226         .reset (!resetn),
227         .rvfi_valid (rvfi_valid),
228         .rvfi_order (rvfi_order),
229         .rvfi_insn (rvfi_insn),
230         .rvfi_trap (rvfi_trap),
231         .rvfi_halt (rvfi_halt),
232         .rvfi_intr (rvfi_intr),
233         .rvfi_rs1_addr (rvfi_rs1_addr),
234         .rvfi_rs2_addr (rvfi_rs2_addr),
235         .rvfi_rs1_rdata (rvfi_rs1_rdata),
236         .rvfi_rs2_rdata (rvfi_rs2_rdata),
237         .rvfi_rd_addr (rvfi_rd_addr),
238         .rvfi_rd_wdata (rvfi_rd_wdata),
239         .rvfi_pc_rdata (rvfi_pc_rdata),
240         .rvfi_pc_wdata (rvfi_pc_wdata),
241         .rvfi_mem_addr (rvfi_mem_addr),
242         .rvfi_mem_rmask (rvfi_mem_rmask),
243         .rvfi_mem_wmask (rvfi_mem_wmask),
244         .rvfi_mem_rdata (rvfi_mem_rdata),
245         .rvfi_mem_wdata (rvfi_mem_wdata)
246     );
247 `endif
248
249     reg [1023:0] firmware_file;
250     initial begin
251         if (!$value$plusargs("firmware=%s", firm-
ware_file))
252             firmware_file = "firmware/firm-
ware.hex";
253         $readmemh(firmware_file, mem.memory);
254     end

```



```

255
256 integer cycle_counter;
257 always @(posedge clk) begin
258     cycle_counter <= resetn ? cycle_counter + 1 :
0;
259     if (resetn && trap) begin
260         `ifndef VERILATOR
261             repeat (10) @(posedge clk);
262         `endif
263         $display("TRAP after %1d clock cycles",
cycle_counter);
264         if (tests_passed) begin
265             $display("ALL TESTS PASSED.");
266             $finish;
267         end else begin
268             $display("ERROR!");
269             if ($test$plusargs("noerror"))
270                 $finish;
271             $stop;
272         end
273     end
274 end
275 endmodule
276
277 module axi4_memory #(
278     parameter AXI_TEST = 0,
279     parameter VERBOSE = 0
280 ) (
281     /* verilator lint_off MULTIDRIVEN */
282
283     input          clk,
284     input          mem_axi_avalid,
285     output reg     mem_axi_awready,
286     input [31:0]   mem_axi_awaddr,
287     input [ 2:0]   mem_axi_awprot,
288
289     input          mem_axi_wvalid,
290     output reg     mem_axi_wready,
291     input [31:0]   mem_axi_wdata,
292     input [ 3:0]   mem_axi_wstrb,
293
294     output reg     mem_axi_bvalid,
295     input          mem_axi_bready,
296
297     input          mem_axi_rvalid,
298     output reg     mem_axi_rready,
299     input [31:0]   mem_axi_raddr,
300     input [ 2:0]   mem_axi_rprot,
301
302     output reg     mem_axi_rvalid,
303     input          mem_axi_rready,
304     output reg [31:0] mem_axi_rdata,
305
306     output reg     tests_passed
307 );
308 reg [31:0] memory [0:128*1024/4-1] /* verilator pub-
lic */;
309 reg verbose;
310 initial verbose = $test$plusargs("verbose") || VERBOSE;
311
312 reg axi_test;
313 initial axi_test = $test$plusargs("axi_test") ||
AXI_TEST;
314
315 initial begin
316     mem_axi_awready = 0;
317     mem_axi_wready = 0;
318     mem_axi_bvalid = 0;
319     mem_axi_rready = 0;
320     mem_axi_rvalid = 0;
321     tests_passed = 0;
322 end
323
324 reg [63:0] xorshift64_state = 64'd88172645463325252;
325
326 task xorshift64_next;
327 begin
328     // see page 4 of Marsaglia, George
(July 2003). "Xorshift RNGs". Journal of Statistical Software 8
(14).
329     xorshift64_state = xorshift64_state ^
(xorshift64_state << 13);
330     xorshift64_state = xorshift64_state ^
(xorshift64_state >> 7);
331     xorshift64_state = xorshift64_state ^
(xorshift64_state << 17);
332 end
333 endtask
334
335 reg [2:0] fast_axi_transaction = ~0;
336 reg [4:0] async_axi_transaction = ~0;
337 reg [4:0] delay_axi_transaction = 0;
338
339 always @(posedge clk) begin
340     if (axi_test) begin
341         xorshift64_next;
342         {fast_axi_transaction,
async_axi_transaction, delay_axi_transaction} <= xorshift64_state;
343     end
344 end

```

```

345
346 reg latched_raddr_en = 0;
347 reg latched_waddr_en = 0;
348 reg latched_wdata_en = 0;
349
350 reg fast_raddr = 0;
351 reg fast_waddr = 0;
352 reg fast_wdata = 0;
353
354 reg [31:0] latched_raddr;
355 reg [31:0] latched_waddr;
356 reg [31:0] latched_wdata;
357 reg [ 3:0] latched_wstrb;
358 reg        latched_rinsn;
359
360 task handle_axi_arvalid; begin
361     mem_axi_arready <= 1;
362     latched_raddr = mem_axi_araddr;
363     latched_rinsn = mem_axi_arprot[2];
364     latched_raddr_en = 1;
365     fast_raddr <= 1;
366 end endtask
367
368 task handle_axi_awvalid; begin
369     mem_axi_awready <= 1;
370     latched_waddr = mem_axi_awaddr;
371     latched_waddr_en = 1;
372     fast_waddr <= 1;
373 end endtask
374
375 task handle_axi_wvalid; begin
376     mem_axi_wready <= 1;
377     latched_wdata = mem_axi_wdata;
378     latched_wstrb = mem_axi_wstrb;
379     latched_wdata_en = 1;
380     fast_wdata <= 1;
381 end endtask
382
383 task handle_axi_rvalid; begin
384     if (verbose)
385         $display("RD: ADDR=%08x DATA=%08x%s",
latched_raddr, memory[latched_raddr >> 2], latched_rinsn ? " INSN" :
""");
386     if (latched_raddr < 128*1024) begin
387         mem_axi_rdata <= memory[latched_raddr
>> 2];
388         mem_axi_rvalid <= 1;
389         latched_raddr_en = 0;
390     end else begin
391         $display("OUT-OF-BOUNDS MEMORY READ
FROM %08x", latched_raddr);
392         $finish;
393     end
394 end endtask
395
396 task handle_axi_bvalid; begin
397     if (verbose)
398         $display("WR: ADDR=%08x DATA=%08x
STRB=%04b", latched_waddr, latched_wdata, latched_wstrb);
399     if (latched_waddr < 128*1024) begin
400         if (latched_wstrb[0])
memory[latched_waddr >> 2][ 7: 0] <= latched_wdata[ 7: 0];
401         if (latched_wstrb[1])
memory[latched_waddr >> 2][15: 8] <= latched_wdata[15: 8];
402         if (latched_wstrb[2])
memory[latched_waddr >> 2][23:16] <= latched_wdata[23:16];
403         if (latched_wstrb[3])
memory[latched_waddr >> 2][31:24] <= latched_wdata[31:24];
404     end else
405         if (latched_waddr == 32'h1000_0000) begin
406             if (verbose) begin
407                 if (32 <= latched_wdata &&
latched_wdata < 128)
408                     $display("OUT: '%c'",
latched_wdata[7:0]);
409                 else
410                     $display("OUT: '%3d",
latched_wdata);
411             end else begin
412                 $write("%c",
latched_wdata[7:0]);
413             `ifndef VERILATOR
414                 $fflush();
415             `endif
416         end
417     end else
418         if (latched_waddr == 32'h2000_0000) begin
419             if (latched_wdata == 123456789)
420                 tests_passed = 1;
421         end else begin
422             $display("OUT-OF-BOUNDS MEMORY WRITE TO
%08x", latched_waddr);
423             $finish;
424         end
425     mem_axi_bvalid <= 1;
426     latched_waddr_en = 0;
427     latched_wdata_en = 0;
428 end endtask
429
430 always @(negedge clk) begin

```

```

431         if (mem_axi_rvalid && !(latched_raddr_en ||
fast_raddr) && async_axi_transaction[0]) handle_axi_rvalid;
432         if (mem_axi_wvalid && !(latched_waddr_en ||
fast_waddr) && async_axi_transaction[1]) handle_axi_wvalid;
433         if (mem_axi_vvalid && !(latched_wdata_en ||
fast_wdata) && async_axi_transaction[2]) handle_axi_vvalid;
434         if (!mem_axi_rvalid && latched_raddr_en &&
async_axi_transaction[3]) handle_axi_rvalid;
435         if (!mem_axi_bvalid && latched_waddr_en &&
latched_wdata_en && async_axi_transaction[4]) handle_axi_bvalid;
436     end
437
438     always @(posedge clk) begin
439         mem_axi_arready <= 0;
440         mem_axi_awready <= 0;
441         mem_axi_wready <= 0;
442
443         fast_raddr <= 0;
444         fast_waddr <= 0;
445         fast_wdata <= 0;
446
447         if (mem_axi_rvalid && mem_axi_rready) begin
448             mem_axi_rvalid <= 0;
449         end
450
451         if (mem_axi_bvalid && mem_axi_bready) begin
452             mem_axi_bvalid <= 0;
453         end
454
455         if (mem_axi_arvalid && mem_axi_arready && !
fast_raddr) begin
456             latched_raddr = mem_axi_araddr;
457             latched_rinsn = mem_axi_arprot[2];
458             latched_raddr_en = 1;
459         end
460
461         if (mem_axi_awvalid && mem_axi_awready && !
fast_waddr) begin
462             latched_waddr = mem_axi_awaddr;
463             latched_waddr_en = 1;
464         end
465
466         if (mem_axi_vvalid && mem_axi_wready && !
fast_wdata) begin
467             latched_wdata = mem_axi_wdata;
468             latched_wstrb = mem_axi_wstrb;
469             latched_wdata_en = 1;
470         end
471
472         if (mem_axi_arvalid && !(latched_raddr_en ||
fast_raddr) && !delay_axi_transaction[0]) handle_axi_arvalid;
473         if (mem_axi_awvalid && !(latched_waddr_en ||
fast_waddr) && !delay_axi_transaction[1]) handle_axi_awvalid;
474         if (mem_axi_vvalid && !(latched_wdata_en ||
fast_wdata) && !delay_axi_transaction[2]) handle_axi_vvalid;
475
476         if (!mem_axi_rvalid && latched_raddr_en && !
delay_axi_transaction[3]) handle_axi_rvalid;
477         if (!mem_axi_bvalid && latched_waddr_en &&
latched_wdata_en && !delay_axi_transaction[4]) handle_axi_bvalid;
478     end
479 endmodule

```

3. YosysHQ_picorv32/testbench_ez.v (86 lines)

```

1 // This is free and unencumbered software released into the
public domain.
2 //
3 // Anyone is free to copy, modify, publish, use, compile,
sell, or
4 // distribute this software, either in source code form or as
a compiled
5 // binary, for any purpose, commercial or non-commercial, and
by any
6 // means.
7
8 `timescale 1 ns / 1 ps
9
10 module testbench;
11     reg clk = 1;
12     reg resetn = 0;
13     wire trap;
14
15     always #5 clk = ~clk;
16
17     initial begin
18         if ($test$plusargs("vcd")) begin
19             $dumpfile("testbench.vcd");
20             $dumpvars(0, testbench);
21         end
22         repeat (100) @(posedge clk);
23         resetn <= 1;
24         repeat (1000) @(posedge clk);
25         $finish;
26     end
27
28     wire mem_valid;
29     wire mem_instr;
30     reg mem_ready;
31     wire [31:0] mem_addr;

```

```

32     wire [31:0] mem_wdata;
33     wire [3:0] mem_wstrb;
34     reg [31:0] mem_rdata;
35
36     always @(posedge clk) begin
37         if (mem_valid && mem_ready) begin
38             if (mem_instr)
39                 $display("ifetch 0x%08x:
0x%08x", mem_addr, mem_rdata);
40             else if (mem_wstrb)
41                 $display("write 0x%08x: 0x%08x
(wstrb=%b)", mem_addr, mem_wdata, mem_wstrb);
42             else
43                 $display("read 0x%08x:
0x%08x", mem_addr, mem_rdata);
44         end
45     end
46
47     picorv32 #(
48     ) uut (
49         .clk          (clk          ),
50         .resetn       (resetn       ),
51         .trap         (trap         ),
52         .mem_valid    (mem_valid    ),
53         .mem_instr    (mem_instr    ),
54         .mem_ready    (mem_ready    ),
55         .mem_addr     (mem_addr     ),
56         .mem_wdata    (mem_wdata    ),
57         .mem_wstrb    (mem_wstrb    ),
58         .mem_rdata    (mem_rdata    )
59     );
60
61     reg [31:0] memory [0:255];
62
63     initial begin
64         memory[0] = 32'h 3fc00093; //      li
x1,1020
65         memory[1] = 32'h 0000a023; //      sw
x0,0(x1)
66         memory[2] = 32'h 0000a103; // loop: lw
x2,0(x1)
67         memory[3] = 32'h 00110113; //      addi
x2,x2,1
68         memory[4] = 32'h 0020a023; //      sw
x2,0(x1)
69         memory[5] = 32'h ff5ff06f; //      j
<loop>
70
71     end
72
73     always @(posedge clk) begin
74         mem_ready <= 0;
75         if (mem_valid && !mem_ready) begin
76             if (mem_addr < 1024) begin
77                 mem_rdata <= memory[mem_addr >>
2];
78                 if (mem_wstrb[0])
79                     memory[mem_addr >> 2][ 7: 0] <= mem_wdata[ 7: 0];
80                 if (mem_wstrb[1])
81                     memory[mem_addr >> 2][15: 8] <= mem_wdata[15: 8];
82                 if (mem_wstrb[2])
83                     memory[mem_addr >> 2][23:16] <= mem_wdata[23:16];
84                 if (mem_wstrb[3])
85                     memory[mem_addr >> 2][31:24] <= mem_wdata[31:24];
86             end
87             /* add memory-mapped IO here */
88         end
89     end
90 endmodule

```

4. YosysHQ_picorv32/testbench_wb.v (293 lines)

```

1 `timescale 1 ns / 1 ps
2
3 `ifndef VERILATOR
4 module testbench #(
5     parameter VERBOSE = 0
6 );
7
8     reg clk = 1;
9     reg resetn = 1;
10    wire trap;
11
12    always #5 clk = ~clk;
13
14    initial begin
15        repeat (100) @(posedge clk);
16        resetn <= 0;
17    end
18
19    initial begin
20        if ($test$plusargs("vcd")) begin
21            $dumpfile("testbench.vcd");
22            $dumpvars(0, testbench);
23        end
24        repeat (1000000) @(posedge clk);
25        $display("TIMEOUT");
26        $finish;
27    end
28 end

```



```

27
28     wire trace_valid;
29     wire [35:0] trace_data;
30     integer trace_file;
31
32     initial begin
33         if ($test$plusargs("trace")) begin
34             trace_file = $fopen("testbench.trace",
"w");
35             repeat (10) @(posedge clk);
36             while (!trap) begin
37                 @(posedge clk);
38                 if (trace_valid)
39                     $fwrite(trace_file,
"%x\n", trace_data);
40             end
41             $fclose(trace_file);
42             $display("Finished writing test-
bench.trace.");
43         end
44     end
45
46     picorv32_wrapper #(
47         .VERBOSE (VERBOSE)
48     ) top (
49         .wb_clk(clk),
50         .wb_rst(resetn),
51         .trap(trap),
52         .trace_valid(trace_valid),
53         .trace_data(trace_data)
54     );
55 endmodule
56 `endif
57
58 module picorv32_wrapper #(
59     parameter VERBOSE = 0
60 ) (
61     input wb_clk,
62     input wb_rst,
63     output trap,
64     output trace_valid,
65     output [35:0] trace_data
66 );
67     wire tests_passed;
68     reg [31:0] irq = 0;
69     wire mem_instr;
70
71     reg [15:0] count_cycle = 0;
72     always @(posedge wb_clk) count_cycle <= !wb_rst ?
count_cycle + 1 : 0;
73
74     always @* begin
75         irq = 0;
76         irq[4] = &count_cycle[12:0];
77         irq[5] = &count_cycle[15:0];
78     end
79
80     wire [31:0] wb_m2s_adr;
81     wire [31:0] wb_m2s_dat;
82     wire [3:0] wb_m2s_sel;
83     wire wb_m2s_we;
84     wire wb_m2s_cyc;
85     wire wb_m2s_stb;
86     wire [31:0] wb_s2m_dat;
87     wire wb_s2m_ack;
88
89     wb_ram #(
90         .depth (128*1024),
91         .VERBOSE (VERBOSE)
92     ) ram ( // Wishbone interface
93         .wb_clk_i(wb_clk),
94         .wb_rst_i(wb_rst),
95
96         .wb_adr_i(wb_m2s_adr),
97         .wb_dat_i(wb_m2s_dat),
98         .wb_stb_i(wb_m2s_stb),
99         .wb_cyc_i(wb_m2s_cyc),
100        .wb_dat_o(wb_s2m_dat),
101        .wb_ack_o(wb_s2m_ack),
102        .wb_sel_i(wb_m2s_sel),
103        .wb_we_i(wb_m2s_we),
104
105        .mem_instr(mem_instr),
106        .tests_passed(tests_passed)
107    );
108
109    picorv32_wb #(
110        `ifndef SYNTH_TEST
111        `ifdef SP_TEST
112            .ENABLE_REGS_DUALPORT(0),
113        `endif
114        `ifdef COMPRESSED_ISA
115            .COMPRESSED_ISA(1),
116        `endif
117            .ENABLE_MUL(1),
118            .ENABLE_DIV(1),
119            .ENABLE_IRQ(1),
120            .ENABLE_TRACE(1)
121        `endif
122    ) uut (
123        .trap (trap),

```

```

124        .irq (irq),
125        .trace_valid (trace_valid),
126        .trace_data (trace_data),
127        .mem_instr(mem_instr),
128
129        .wb_clk_i(wb_clk),
130        .wb_rst_i(wb_rst),
131
132        .wbm_adr_o(wb_m2s_adr),
133        .wbm_dat_i(wb_s2m_dat),
134        .wbm_stb_o(wb_m2s_stb),
135        .wbm_ack_i(wb_s2m_ack),
136        .wbm_cyc_o(wb_m2s_cyc),
137        .wbm_dat_o(wb_m2s_dat),
138        .wbm_we_o(wb_m2s_we),
139        .wbm_sel_o(wb_m2s_sel)
140    );
141
142    reg [1023:0] firmware_file;
143    initial begin
144        if (!$value$plusargs("firmware=%s", firm-
ware_file))
145            firmware_file = "firmware/firm-
ware.hex";
146        $readmemh(firmware_file, ram.mem);
147    end
148
149    integer cycle_counter;
150    always @(posedge wb_clk) begin
151        cycle_counter <= !wb_rst ? cycle_counter + 1 :
0;
152        if (!wb_rst && trap) begin
153            `ifndef VERILATOR
154                repeat (10) @(posedge wb_clk);
155            `endif
156            $display("TRAP after %1d clock cycles",
cycle_counter);
157            if (tests_passed) begin
158                $display("ALL TESTS PASSED.");
159                $finish;
160            end else begin
161                $display("ERROR!");
162                if ($test$plusargs("noerror"))
163                    $finish;
164                $stop;
165            end
166        end
167    end
168 endmodule
169
170 /* ISC License
171 *
172 * Verilog on-chip RAM with Wishbone interface
173 *
174 * Copyright (C) 2014, 2016 Olof Kindgren <olof.kind-
gren@gmail.com>
175 *
176 * Permission to use, copy, modify, and/or distribute this
software for any
177 * purpose with or without fee is hereby granted, provided
that the above
178 * copyright notice and this permission notice appear in all
copies.
179 *
180 * THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS
ALL WARRANTIES
181 * WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WAR-
RANTIES OF
182 * MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR
BE LIABLE FOR
183 * ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR
ANY DAMAGES
184 * WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN
185 * ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION,
ARISING OUT OF
186 * OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFT-
WARE.
187 */
188
189 module wb_ram #(
190     parameter depth = 256,
191     parameter memfile = "",
192     parameter VERBOSE = 0
193 ) (
194     input wb_clk_i,
195     input wb_rst_i,
196
197     input [31:0] wb_adr_i,
198     input [31:0] wb_dat_i,
199     input [3:0] wb_sel_i,
200     input wb_we_i,
201     input wb_cyc_i,
202     input wb_stb_i,
203
204     output reg wb_ack_o,
205     output reg [31:0] wb_dat_o,
206
207     input mem_instr,
208     output reg tests_passed

```

```

209 );
210
211 reg verbose;
212 initial verbose = $test$plusargs("verbose") || VERBOSE;
213
214 initial tests_passed = 0;
215
216 reg [31:0] adr_r;
217 wire valid = wb_cyc_i & wb_stb_i;
218
219 always @(posedge wb_clk_i) begin
220     adr_r <= wb_adr_i;
221     // Ack generation
222     wb_ack_o <= valid & !wb_ack_o;
223     if (wb_rst_i)
224         begin
225             adr_r <= {32{1'b0}};
226             wb_ack_o <= 1'b0;
227         end
228     end
229
230 wire ram_we = wb_we_i & valid & wb_ack_o;
231
232 wire [31:0] waddr = adr_r[31:2];
233 wire [31:0] raddr = wb_adr_i[31:2];
234 wire [3:0] we = {4{ram_we}} & wb_sel_i;
235
236 wire [$clog2(depth/4)-1:0] raddr2 = raddr[$clog2(depth/
4)-1:0];
237 wire [$clog2(depth/4)-1:0] waddr2 = waddr[$clog2(depth/
4)-1:0];
238
239 reg [31:0] mem [0:depth/4-1] /* verilator public */;
240
241 always @(posedge wb_clk_i) begin
242     if (ram_we) begin
243         if (verbose)
244             $display("WR: ADDR=%08x
DATA=%08x STRB=%04b",
245                     adr_r, wb_dat_i, we);
246         if (adr_r[31:0] == 32'h1000_0000)
247             if (verbose) begin
248                 if (32 <= wb_dat_i[7:0]
&& wb_dat_i[7:0] < 128)
249                     $display("OUT:
'%c'", wb_dat_i[7:0]);
250                 else
251                     $display("OUT:
%3d", wb_dat_i[7:0]);
252             end else begin
253                 $write("%c",
254                       wb_dat_i[7:0]);
255             `ifndef VERILATOR
256                 $fflush();
257             `endif
258         end
259     else
260         if (adr_r[31:0] == 32'h2000_0000)
261             if (wb_dat_i[31:0] ==
123456789)
262                 tests_passed = 1;
263     end
264 end
265
266 always @(posedge wb_clk_i) begin
267     if (waddr2 < 128 * 1024 / 4) begin
268         if (we[0])
269             mem[waddr2][7:0] <=
wb_dat_i[7:0];
270         if (we[1])
271             mem[waddr2][15:8] <=
wb_dat_i[15:8];
272         if (we[2])
273             mem[waddr2][23:16] <=
wb_dat_i[23:16];
274         if (we[3])
275             mem[waddr2][31:24] <=
wb_dat_i[31:24];
276     end
277
278     if (valid & wb_ack_o & !ram_we)
279         if (verbose)
280             $display("RD: ADDR=%08x
DATA=%08x%s", adr_r, mem[raddr2], mem_instr ? " INSN" : "");
281
282     wb_dat_o <= mem[raddr2];
283
284     initial begin
285         if (memfile != "")
286             $readmemh(memfile, mem);
287     end
288 end
289
290 endmodule

```

5. YosysHQ_picorv32/scripts/smtbmc/axicheck.v (210 lines)

```

1 module testbench (
2     input      clk,
3     output     trap,
4
5     output     mem_axi_awvalid,
6     input      mem_axi_awready,
7     output [31:0] mem_axi_awaddr,
8     output [ 2:0] mem_axi_awprot,
9
10    output     mem_axi_wvalid,
11    input      mem_axi_wready,
12    output [31:0] mem_axi_wdata,
13    output [ 3:0] mem_axi_wstrb,
14
15    input      mem_axi_bvalid,
16    output     mem_axi_bready,
17
18    output     mem_axi_arvalid,
19    input      mem_axi_arready,
20    output [31:0] mem_axi_araddr,
21    output [ 2:0] mem_axi_arprot,
22
23    input      mem_axi_rvalid,
24    output     mem_axi_rready,
25    input [31:0] mem_axi_rdata
26 );
27
28 reg resetn = 0;
29
30 always @(posedge clk)
31     resetn <= 1;
32
33 picorv32_axi #(
34     .ENABLE_COUNTERS(1),
35     .ENABLE_COUNTERS64(1),
36     .ENABLE_REGS_16_31(1),
37     .ENABLE_REGS_DUALPORT(1),
38     .BARREL_SHIFTER(1),
39     .TWO_CYCLE_COMPARE(0),
40     .TWO_CYCLE_ALU(0),
41     .COMPRESSED_ISA(0),
42     .CATCH_MISALIGN(1),
43     .CATCH_ILLSN(1)
44 ) uut (
45     .clk          (clk          ),
46     .resetn       (resetn       ),
47     .trap         (trap         ),
48     .mem_axi_awvalid (mem_axi_awvalid),
49     .mem_axi_awready (mem_axi_awready),
50     .mem_axi_awaddr (mem_axi_awaddr ),
51     .mem_axi_awprot (mem_axi_awprot ),
52     .mem_axi_wvalid (mem_axi_wvalid ),
53     .mem_axi_wready (mem_axi_wready ),
54     .mem_axi_wdata  (mem_axi_wdata  ),
55     .mem_axi_wstrb  (mem_axi_wstrb  ),
56     .mem_axi_bvalid (mem_axi_bvalid ),
57     .mem_axi_bready (mem_axi_bready ),
58     .mem_axi_arvalid (mem_axi_arvalid),
59     .mem_axi_arready (mem_axi_arready),
60     .mem_axi_araddr (mem_axi_araddr ),
61     .mem_axi_arprot (mem_axi_arprot ),
62     .mem_axi_rvalid (mem_axi_rvalid ),
63     .mem_axi_rready (mem_axi_rready ),
64     .mem_axi_rdata  (mem_axi_rdata  )
65 );
66
67 reg expect_bvalid_aw = 0;
68 reg expect_bvalid_w  = 0;
69 reg expect_rvalid    = 0;
70
71 reg [3:0] timeout_aw = 0;
72 reg [3:0] timeout_w  = 0;
73 reg [3:0] timeout_b  = 0;
74 reg [3:0] timeout_ar = 0;
75 reg [3:0] timeout_r  = 0;
76 reg [3:0] timeout_ex = 0;
77
78 always @(posedge clk) begin
79     timeout_aw <= !mem_axi_awvalid ||
mem_axi_awready ? 0 : timeout_aw + 1;
80     timeout_w  <= !mem_axi_wvalid ||
mem_axi_wready ? 0 : timeout_w  + 1;
81     timeout_b  <= !mem_axi_bvalid ||
mem_axi_bready ? 0 : timeout_b  + 1;
82     timeout_ar <= !mem_axi_arvalid || mem_axi_ar-
ready ? 0 : timeout_ar + 1;
83     timeout_r  <= !mem_axi_rvalid || mem_axi_r-
ready ? 0 : timeout_r  + 1;
84     timeout_ex <= !{expect_bvalid_aw, ex-
pect_bvalid_w, expect_rvalid} ? 0 : timeout_ex + 1;
85     restrict(timeout_aw != 15);
86     restrict(timeout_w  != 15);
87     restrict(timeout_b  != 15);
88     restrict(timeout_ar != 15);
89     restrict(timeout_r  != 15);
90     restrict(timeout_ex != 15);
91     restrict(!trap);

```

```

92     end
93     always @(posedge clk) begin
94         if (resetn) begin
95             if (!$past(resetn)) begin
96                 assert(!mem_axi_awvalid);
97                 assert(!mem_axi_wvalid);
98                 assume(!mem_axi_bvalid);
99                 assert(!mem_axi_rvalid);
100                 assume(!mem_axi_rvalid);
101             end else begin
102                 // Only one read/write transaction in flight at each point in time
103                 if (expect_bvalid_aw) begin
104                     assert(!mem_axi_awvalid);
105                 end
106                 if (expect_bvalid_w) begin
107                     assert(!mem_axi_wvalid);
108                 end
109                 if (expect_rvalid) begin
110                     assert(!mem_axi_rvalid);
111                 end
112                 if (expect_rvalid) begin
113                     assert(!mem_axi_rvalid);
114                 end
115                 expect_bvalid_aw = expect_bvalid_w = expect_rvalid = 0;
116                 expect_bvalid_w = expect_rvalid = 0;
117                 expect_rvalid = 0;
118                 if (expect_bvalid_aw || expect_bvalid_w || expect_rvalid) begin
119                     assert(!expect_rvalid);
120                 end
121                 if (expect_rvalid) begin
122                     assert(!expect_rvalid);
123                 end
124                 if (!expect_bvalid_aw || !expect_bvalid_w || !expect_rvalid) begin
125                     assume(!mem_axi_bvalid);
126                 end
127                 if (!expect_rvalid) begin
128                     assume(!mem_axi_rvalid);
129                 end
130                 if (mem_axi_bvalid && expect_bvalid_aw = 0; expect_bvalid_w = 0; end
131                 if (mem_axi_rvalid && expect_rvalid = 0; end
132                 end
133                 // Check AXI Master Streams
134                 if ($past(mem_axi_awvalid && !mem_axi_arready)) begin
135                     as-
136                     sert(mem_axi_awvalid);
137                     as-
138                     sert($stable(mem_axi_awaddr));
139                     as-
140                     sert($stable(mem_axi_awprot));
141                 end
142                 if ($fell(mem_axi_awvalid)) begin
143                     as-
144                     sert($past(mem_axi_arready));
145                 end
146                 if ($fell(mem_axi_arready)) begin
147                     as-
148                     sume($past(mem_axi_arvalid));
149                 end
150                 if ($past(mem_axi_arvalid && !mem_axi_rready)) begin
151                     as-
152                     sert(mem_axi_rvalid);
153                     as-
154                     sert($stable(mem_axi_raddr));
155                     as-
156                     sert($stable(mem_axi_rprot));
157                 end
158                 end

```

```

166         if ($fell(mem_axi_arvalid)) begin
167             as-
168             sert($past(mem_axi_arready));
169         end
170         if ($fell(mem_axi_arready)) begin
171             as-
172             sume($past(mem_axi_arvalid));
173         end
174         if ($past(mem_axi_wvalid && !mem_axi_wready)) begin
175             assert(mem_axi_wvalid);
176             as-
177             sert($stable(mem_axi_wdata));
178             as-
179             sert($stable(mem_axi_wstrb));
180         end
181         if ($fell(mem_axi_wvalid)) begin
182             as-
183             sert($past(mem_axi_wready));
184         end
185         if ($fell(mem_axi_wready)) begin
186             as-
187             sert($past(mem_axi_bready));
188             as-
189             sume($past(mem_axi_bready));
190         end
191         if ($fell(mem_axi_bready)) begin
192             as-
193             sert($past(mem_axi_bvalid));
194         end
195         if ($past(mem_axi_rvalid && !mem_axi_rready)) begin
196             assume(mem_axi_rvalid);
197             as-
198             sume($stable(mem_axi_rdata));
199         end
200         if ($fell(mem_axi_rvalid)) begin
201             as-
202             sert($past(mem_axi_rready));
203         end
204         if ($fell(mem_axi_rready)) begin
205             as-
206             sert($past(mem_axi_rvalid));
207         end
208         end
209         end
210     endmodule

```

6. YosysHQ_picorv32/scripts/smtbmc/axicheck2.v (147 lines)

```

1 module testbench (
2     input      clk,
3     input      resetn,
4     output     trap_0,
5     output     trap_1,
6
7     output     mem_axi_awvalid_0,
8     input      mem_axi_awready_0,
9     output [31:0] mem_axi_awaddr_0,
10    output [ 2:0] mem_axi_awprot_0,
11
12    output     mem_axi_awvalid_1,
13    input      mem_axi_awready_1,
14    output [31:0] mem_axi_awaddr_1,
15    output [ 2:0] mem_axi_awprot_1,
16
17    output     mem_axi_wvalid_0,
18    input      mem_axi_wready_0,
19    output [31:0] mem_axi_wdata_0,
20    output [ 3:0] mem_axi_wstrb_0,
21
22    output     mem_axi_wvalid_1,
23    input      mem_axi_wready_1,
24    output [31:0] mem_axi_wdata_1,
25    output [ 3:0] mem_axi_wstrb_1,
26
27    input      mem_axi_bvalid,
28    output     mem_axi_bready_0,
29    output     mem_axi_bready_1,
30

```

```

31 output      mem_axi_arvalid_0,
32 input       mem_axi_arready_0,
33 output [31:0] mem_axi_araddr_0,
34 output [ 2:0] mem_axi_arprot_0,
35
36 output      mem_axi_arvalid_1,
37 input       mem_axi_arready_1,
38 output [31:0] mem_axi_araddr_1,
39 output [ 2:0] mem_axi_arprot_1,
40
41 input       mem_axi_rvalid,
42 output      mem_axi_rready_0,
43 output      mem_axi_rready_1,
44 input [31:0] mem_axi_rdata
45 );
46
47 picorv32_axi #(
48     .ENABLE_COUNTERS(1),
49     .ENABLE_COUNTERS64(1),
50     .ENABLE_REGS_16_31(1),
51     .ENABLE_REGS_DUALPORT(1),
52     .BARREL_SHIFTER(1),
53     .TWO_CYCLE_COMPARE(0),
54     .TWO_CYCLE_ALU(0),
55     .COMPRESSED_ISA(0),
56     .CATCH_MISALIGN(1),
57     .CATCH_ILLSN(1)
58 ) uut_0 (
59     .clk          (clk          ),
60     .resetsn      (resetsn      ),
61     .trap         (trap         ),
62     .mem_axi_arvalid (mem_axi_arvalid_0),
63     .mem_axi_arready (mem_axi_arready_0),
64     .mem_axi_araddr  (mem_axi_araddr_0 ),
65     .mem_axi_arprot  (mem_axi_arprot_0 ),
66     .mem_axi_wvalid  (mem_axi_wvalid_0 ),
67     .mem_axi_wready  (mem_axi_wready_0 ),
68     .mem_axi_wdata   (mem_axi_wdata_0  ),
69     .mem_axi_wstrb   (mem_axi_wstrb_0  ),
70     .mem_axi_bvalid  (mem_axi_bvalid_0 ),
71     .mem_axi_bready  (mem_axi_bready_0 ),
72     .mem_axi_arvalid (mem_axi_arvalid_0),
73     .mem_axi_arready (mem_axi_arready_0),
74     .mem_axi_araddr  (mem_axi_araddr_0 ),
75     .mem_axi_arprot  (mem_axi_arprot_0 ),
76     .mem_axi_rvalid  (mem_axi_rvalid_0 ),
77     .mem_axi_rready  (mem_axi_rready_0 ),
78     .mem_axi_rdata   (mem_axi_rdata_0  )
79 );
80
81 picorv32_axi #(
82     .ENABLE_COUNTERS(1),
83     .ENABLE_COUNTERS64(1),
84     .ENABLE_REGS_16_31(1),
85     .ENABLE_REGS_DUALPORT(1),
86     .BARREL_SHIFTER(1),
87     .TWO_CYCLE_COMPARE(0),
88     .TWO_CYCLE_ALU(0),
89     .COMPRESSED_ISA(0),
90     .CATCH_MISALIGN(1),
91     .CATCH_ILLSN(1)
92 ) uut_1 (
93     .clk          (clk          ),
94     .resetsn      (resetsn      ),
95     .trap         (trap         ),
96     .mem_axi_arvalid (mem_axi_arvalid_1),
97     .mem_axi_arready (mem_axi_arready_1),
98     .mem_axi_araddr  (mem_axi_araddr_1 ),
99     .mem_axi_arprot  (mem_axi_arprot_1 ),
100    .mem_axi_wvalid  (mem_axi_wvalid_1 ),
101    .mem_axi_wready  (mem_axi_wready_1 ),
102    .mem_axi_wdata   (mem_axi_wdata_1  ),
103    .mem_axi_wstrb   (mem_axi_wstrb_1  ),
104    .mem_axi_bvalid  (mem_axi_bvalid_1 ),
105    .mem_axi_bready  (mem_axi_bready_1 ),
106    .mem_axi_arvalid (mem_axi_arvalid_1),
107    .mem_axi_arready (mem_axi_arready_1),
108    .mem_axi_araddr  (mem_axi_araddr_1 ),
109    .mem_axi_arprot  (mem_axi_arprot_1 ),
110    .mem_axi_rvalid  (mem_axi_rvalid_1 ),
111    .mem_axi_rready  (mem_axi_rready_1 ),
112    .mem_axi_rdata   (mem_axi_rdata_1  )
113 );
114
115 always @(posedge clk) begin
116     if (resetsn && $past(resetsn)) begin
117         assert(trap_0 ==
118 trap_1 );
119         assert(mem_axi_arvalid_0 ==
120 mem_axi_arvalid_1);
121         assert(mem_axi_araddr_0 ==
122 mem_axi_araddr_1 );
123         assert(mem_axi_arprot_0 == mem_axi_ar-
124 prot_1 );
125         assert(mem_axi_wvalid_0 ==
126 mem_axi_wvalid_1 );
127         assert(mem_axi_wdata_0 ==
128 mem_axi_wdata_1 );
129         assert(mem_axi_wstrb_0 ==
130 mem_axi_wstrb_1 );
131         assert(mem_axi_bready_0 ==
132 mem_axi_bready_1 );
133     end
134 end

```

```

124         assert(mem_axi_arvalid_0 == mem_axi_ar-
125 valid_1);
126         assert(mem_axi_araddr_0 ==
127 mem_axi_araddr_1 );
128         assert(mem_axi_arprot_0 == mem_axi_ar-
129 prot_1 );
130         assert(mem_axi_rready_0 == mem_axi_r-
131 ready_1 );
132
133         if (mem_axi_arvalid_0) as-
134 sume(mem_axi_arready_0 == mem_axi_arready_1);
135         if (mem_axi_wvalid_0 ) as-
136 sume(mem_axi_wready_0 == mem_axi_wready_1 );
137         if (mem_axi_arvalid_0) as-
138 sume(mem_axi_arready_0 == mem_axi_arready_1);
139         if ($fell(mem_axi_arready_0)) as-
140 sume($past(mem_axi_arvalid_0));
141         if ($fell(mem_axi_wready_0 )) as-
142 sume($past(mem_axi_wvalid_0 ));
143         if ($fell(mem_axi_arready_0)) as-
144 sume($past(mem_axi_arvalid_0));
145         if ($fell(mem_axi_arready_1)) as-
146 sume($past(mem_axi_arvalid_1));
147         if ($fell(mem_axi_wready_1 )) as-
148 sume($past(mem_axi_wvalid_1 ));
149         if ($fell(mem_axi_arready_1)) as-
150 sume($past(mem_axi_arvalid_1));
151         if ($fell(mem_axi_bvalid)) as-
152 sume($past(mem_axi_bready_0));
153         if ($fell(mem_axi_rvalid)) as-
154 sume($past(mem_axi_rready_0));
155         if (mem_axi_rvalid && $past(mem_axi_r-
156 valid)) assume($stable(mem_axi_rdata));
157     end
158 end
159 endmodule

```

7. YosysHQ_picorv32/scripts/smtbmc/mulcmp.v (87 lines)

```

1 module testbench(input clk, mem_ready_0, mem_ready_1);
2
3     reg resetsn = 0;
4
5     always @(posedge clk)
6         resetsn <= 1;
7
8     reg pcpi_valid_0 = 1;
9     reg pcpi_valid_1 = 1;
10
11     wire [31:0] pcpi_insn = $anyconst;
12     wire [31:0] pcpi_rs1 = $anyconst;
13     wire [31:0] pcpi_rs2 = $anyconst;
14
15     wire pcpi_wr_0;
16     wire [31:0] pcpi_rd_0;
17     wire pcpi_wait_0;
18     wire pcpi_ready_0;
19
20     wire pcpi_wr_1;
21     wire [31:0] pcpi_rd_1;
22     wire pcpi_wait_1;
23     wire pcpi_ready_1;
24
25     reg pcpi_wr_ref;
26     reg [31:0] pcpi_rd_ref;
27     reg pcpi_ready_ref = 0;
28
29     picorv32_pcpi_mul mul_0 (
30         .clk          (clk          ),
31         .resetsn      (resetsn      ),
32         .pcpi_valid   (pcpi_valid_0),
33         .pcpi_insn    (pcpi_insn   ),
34         .pcpi_rs1     (pcpi_rs1    ),
35         .pcpi_rs2     (pcpi_rs2    ),
36         .pcpi_wr      (pcpi_wr_0   ),
37         .pcpi_rd      (pcpi_rd_0   ),
38         .pcpi_wait    (pcpi_wait_0 ),
39         .pcpi_ready   (pcpi_ready_0),
40     );
41
42     picorv32_pcpi_fast_mul mul_1 (
43         .clk          (clk          ),
44         .resetsn      (resetsn      ),
45         .pcpi_valid   (pcpi_valid_1),
46         .pcpi_insn    (pcpi_insn   ),
47         .pcpi_rs1     (pcpi_rs1    ),
48         .pcpi_rs2     (pcpi_rs2    ),
49         .pcpi_wr      (pcpi_wr_1   ),
50         .pcpi_rd      (pcpi_rd_1   ),
51         .pcpi_wait    (pcpi_wait_1 ),
52         .pcpi_ready   (pcpi_ready_1),
53     );
54
55     );
56
57     always @(posedge clk) begin

```

```

58     if (resetsn) begin
59         if (pcpi_ready_0 && pcpi_ready_1) begin
60             assert(pcpi_wr_0 == pcpi_wr_1);
61             assert(pcpi_rd_0 == pcpi_rd_1);
62         end
63     end
64     if (pcpi_ready_0) begin
65         pcpi_valid_0 <= 0;
66         pcpi_wr_ref <= pcpi_wr_0;
67         pcpi_rd_ref <= pcpi_rd_0;
68         pcpi_ready_ref <= 1;
69         if (pcpi_ready_ref) begin
70             assert(pcpi_wr_0 ==
71                 pcpi_wr_ref);
72             assert(pcpi_rd_0 ==
73                 pcpi_rd_ref);
74         end
75     end
76     if (pcpi_ready_1) begin
77         pcpi_valid_1 <= 0;
78         pcpi_wr_ref <= pcpi_wr_1;
79         pcpi_rd_ref <= pcpi_rd_1;
80         pcpi_ready_ref <= 1;
81         if (pcpi_ready_ref) begin
82             assert(pcpi_wr_1 ==
83                 pcpi_wr_ref);
84             assert(pcpi_rd_1 ==
85                 pcpi_rd_ref);
86         end
87     end
88 endmodule

```

8. YosysHQ_picorv32/scripts/smtbmc/notrap_validop.v (67 lines)

```

1 module testbench(input clk, mem_ready);
2     `include "opcode.v"
3
4     reg resetsn = 0;
5     always @(posedge clk) resetsn <= 1;
6
7     (* keep *) wire trap, mem_valid, mem_instr;
8     (* keep *) wire [3:0] mem_wstrb;
9     (* keep *) wire [31:0] mem_addr, mem_wdata, mem_rdata;
10    (* keep *) wire [35:0] trace_data;
11
12    reg [31:0] mem [0:2**30-1];
13
14    assign mem_rdata = mem[mem_addr >> 2];
15
16    always @(posedge clk) begin
17        if (resetsn && mem_valid && mem_ready) begin
18            if (mem_wstrb[3]) mem[mem_addr >> 2]
19                [31:24] <= mem_wdata[31:24];
20            if (mem_wstrb[2]) mem[mem_addr >> 2]
21                [23:16] <= mem_wdata[23:16];
22            if (mem_wstrb[1]) mem[mem_addr >> 2]
23                [15: 8] <= mem_wdata[15: 8];
24            if (mem_wstrb[0]) mem[mem_addr >> 2]
25                [ 7: 0] <= mem_wdata[ 7: 0];
26        end
27    end
28    reg [1:0] mem_ready_stall = 0;
29    always @(posedge clk) begin
30        mem_ready_stall <= {mem_ready_stall, mem_valid
31            && !mem_ready};
32        restrict(&mem_ready_stall == 0);
33    end
34    if (mem_instr && mem_ready && mem_valid) begin
35        assume(opcode_valid(mem_rdata));
36        assume(!opcode_branch(mem_rdata));
37        assume(!opcode_load(mem_rdata));
38        assume(!opcode_store(mem_rdata));
39    end
40    if (!mem_valid)
41        assume(!mem_ready);
42    if (resetsn)
43        assert(!trap);
44 end
45 picorv32 #(
46     // change this settings as you like
47     .ENABLE_REGS_DUALPORT(1),
48     .TWO_STAGE_SHIFT(1),
49     .BARREL_SHIFTER(0),
50     .TWO_CYCLE_COMPARE(0),
51     .TWO_CYCLE_ALU(0),
52     .COMPRESSED_ISA(0),
53     .ENABLE_MUL(0),
54     .ENABLE_DIV(0)
55 ) cpu (
56     .clk          (clk          ),
57     .resetsn      (resetsn      ),

```

```

58     .trap         (trap         ),
59     .mem_valid    (mem_valid    ),
60     .mem_instr    (mem_instr    ),
61     .mem_ready    (mem_ready    ),
62     .mem_addr     (mem_addr     ),
63     .mem_wdata    (mem_wdata    ),
64     .mem_wstrb    (mem_wstrb    ),
65     .mem_rdata    (mem_rdata    )
66 );
67 endmodule

```

9. YosysHQ_picorv32/scripts/smtbmc/opcode.v (104 lines)

```

1 function opcode_jump;
2     input [31:0] opcode;
3     begin
4         opcode_jump = 0;
5         if (opcode[6:0] == 7'b1101111) opcode_jump =
6             1; // JAL
7         if (opcode[14:12] == 3'b000 && opcode[6:0] ==
8             7'b1100111) opcode_jump = 1; // JALR
9     end
10 endfunction
11 function opcode_branch;
12     input [31:0] opcode;
13     begin
14         opcode_branch = 0;
15         if (opcode[14:12] == 3'b000 && opcode[6:0] ==
16             7'b1100011) opcode_branch = 1; // BEQ
17         if (opcode[14:12] == 3'b001 && opcode[6:0] ==
18             7'b1100011) opcode_branch = 1; // BNE
19         if (opcode[14:12] == 3'b100 && opcode[6:0] ==
20             7'b1100011) opcode_branch = 1; // BLT
21         if (opcode[14:12] == 3'b101 && opcode[6:0] ==
22             7'b1100011) opcode_branch = 1; // BGE
23         if (opcode[14:12] == 3'b110 && opcode[6:0] ==
24             7'b1100011) opcode_branch = 1; // BLTU
25         if (opcode[14:12] == 3'b111 && opcode[6:0] ==
26             7'b1100011) opcode_branch = 1; // BGEU
27     end
28 endfunction
29 function opcode_load;
30     input [31:0] opcode;
31     begin
32         opcode_load = 0;
33         if (opcode[14:12] == 3'b000 && opcode[6:0] ==
34             7'b0000011) opcode_load = 1; // LB
35         if (opcode[14:12] == 3'b001 && opcode[6:0] ==
36             7'b0000011) opcode_load = 1; // LH
37         if (opcode[14:12] == 3'b010 && opcode[6:0] ==
38             7'b0000011) opcode_load = 1; // LW
39         if (opcode[14:12] == 3'b100 && opcode[6:0] ==
40             7'b0000011) opcode_load = 1; // LBU
41         if (opcode[14:12] == 3'b101 && opcode[6:0] ==
42             7'b0000011) opcode_load = 1; // LHU
43     end
44 endfunction
45 function opcode_store;
46     input [31:0] opcode;
47     begin
48         opcode_store = 0;
49         if (opcode[14:12] == 3'b000 && opcode[6:0] ==
50             7'b0100011) opcode_store = 1; // SB
51         if (opcode[14:12] == 3'b001 && opcode[6:0] ==
52             7'b0100011) opcode_store = 1; // SH
53         if (opcode[14:12] == 3'b010 && opcode[6:0] ==
54             7'b0100011) opcode_store = 1; // SW
55     end
56 endfunction
57 function opcode_alui;
58     input [31:0] opcode;
59     begin
60         opcode_alui = 0;
61         if (opcode[14:12] == 3'b000 && opcode[6:0] ==
62             7'b0010011) opcode_alui = 1; // ADDI
63         if (opcode[14:12] == 3'b010 && opcode[6:0] ==
64             7'b0010011) opcode_alui = 1; // SLTI
65         if (opcode[14:12] == 3'b011 && opcode[6:0] ==
66             7'b0010011) opcode_alui = 1; // SLTIU
67         if (opcode[14:12] == 3'b100 && opcode[6:0] ==
68             7'b0010011) opcode_alui = 1; // XORI
69         if (opcode[14:12] == 3'b110 && opcode[6:0] ==
70             7'b0010011) opcode_alui = 1; // ORI
71         if (opcode[14:12] == 3'b111 && opcode[6:0] ==
72             7'b0010011) opcode_alui = 1; // ANDI
73         if (opcode[31:25] == 7'b0000000 && op-
74             code[14:12] == 3'b001 && opcode[6:0] == 7'b0010011) opcode_alui =
75             1; // SLLI
76         if (opcode[31:25] == 7'b0000000 && op-
77             code[14:12] == 3'b101 && opcode[6:0] == 7'b0010011) opcode_alui =
78             1; // SRLI
79         if (opcode[31:25] == 7'b0100000 && op-
80             code[14:12] == 3'b101 && opcode[6:0] == 7'b0010011) opcode_alui =
81             1; // SRAI
82     end
83 endfunction

```



```

60
61 function opcode_alu;
62     input [31:0] opcode;
63     begin
64         opcode_alu = 0;
65         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b000 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // ADD
66         if (opcode[31:25] == 7'b01000000 && op-
code[14:12] == 3'b000 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SUB
67         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b001 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SLL
68         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b010 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SLT
69         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b011 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SLTU
70         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b100 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // XOR
71         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b101 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SRL
72         if (opcode[31:25] == 7'b01000000 && op-
code[14:12] == 3'b101 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // SRA
73         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b110 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // OR
74         if (opcode[31:25] == 7'b00000000 && op-
code[14:12] == 3'b111 && opcode[6:0] == 7'b0110011) opcode_alu =
1; // AND
75     end
76 endfunction
77
78 function opcode_sys;
79     input [31:0] opcode;
80     begin
81         opcode_sys = 0;
82         if (opcode[31:20] == 12'hC00 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDCYCLE
83         if (opcode[31:20] == 12'hC01 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDTIME
84         if (opcode[31:20] == 12'hC02 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDINSTRET
85         if (opcode[31:20] == 12'hC80 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDCYCLEH
86         if (opcode[31:20] == 12'hC81 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDTIMEH
87         if (opcode[31:20] == 12'hC82 && opcode[19:12]
== 3'b010 && opcode[6:0] == 7'b1110011) opcode_sys = 1; // RDIN-
STRETH
88     end
89 endfunction
90
91 function opcode_valid;
92     input [31:0] opcode;
93     begin
94         opcode_valid = 0;
95         if (opcode_jump (opcode)) opcode_valid = 1;
96         if (opcode_branch(opcode)) opcode_valid = 1;
97         if (opcode_load (opcode)) opcode_valid = 1;
98         if (opcode_store (opcode)) opcode_valid = 1;
99         if (opcode_alui (opcode)) opcode_valid = 1;
100        if (opcode_alu (opcode)) opcode_valid = 1;
101        if (opcode_sys (opcode)) opcode_valid = 1;
102    end
103 endfunction
104

```

10. YosysHQ_picorv32/scripts/smtbmc/tracecmp.v (109 lines)

```

1 module testbench(input clk, mem_ready_0, mem_ready_1);
2     // set this to 1 to test generation of counter examples
3     localparam ENABLE_COUNTERS = 0;
4
5     reg resetn = 0;
6     always @(posedge clk) resetn <= 1;
7
8     (* keep *) wire trap_0, trace_valid_0, mem_valid_0,
mem_instr_0;
9     (* keep *) wire [3:0] mem_wstrb_0;
10    (* keep *) wire [31:0] mem_addr_0, mem_wdata_0,
mem_rdata_0;
11    (* keep *) wire [35:0] trace_data_0;
12
13    (* keep *) wire trap_1, trace_valid_1, mem_valid_1,
mem_instr_1;
14    (* keep *) wire [3:0] mem_wstrb_1;
15    (* keep *) wire [31:0] mem_addr_1, mem_wdata_1,
mem_rdata_1;
16    (* keep *) wire [35:0] trace_data_1;
17
18    reg [31:0] mem_0 [0:2**30-1];
19    reg [31:0] mem_1 [0:2**30-1];
20
21    assign mem_rdata_0 = mem_0[mem_addr_0 >> 2];

```

```

22    assign mem_rdata_1 = mem_1[mem_addr_1 >> 2];
23
24    always @(posedge clk) begin
25        if (resetn && mem_valid_0 && mem_ready_0) begin
26            if (mem_wstrb_0[3]) mem_0[mem_addr_0 >> 2]
[31:24] <= mem_wdata_0[31:24];
27            if (mem_wstrb_0[2]) mem_0[mem_addr_0 >> 2]
[23:16] <= mem_wdata_0[23:16];
28            if (mem_wstrb_0[1]) mem_0[mem_addr_0 >> 2]
[15: 8] <= mem_wdata_0[15: 8];
29            if (mem_wstrb_0[0]) mem_0[mem_addr_0 >> 2]
[ 7: 0] <= mem_wdata_0[ 7: 0];
30        end
31        if (resetn && mem_valid_1 && mem_ready_1) begin
32            if (mem_wstrb_1[3]) mem_1[mem_addr_1 >> 2]
[31:24] <= mem_wdata_1[31:24];
33            if (mem_wstrb_1[2]) mem_1[mem_addr_1 >> 2]
[23:16] <= mem_wdata_1[23:16];
34            if (mem_wstrb_1[1]) mem_1[mem_addr_1 >> 2]
[15: 8] <= mem_wdata_1[15: 8];
35            if (mem_wstrb_1[0]) mem_1[mem_addr_1 >> 2]
[ 7: 0] <= mem_wdata_1[ 7: 0];
36        end
37    end
38
39    (* keep *) reg [7:0] trace_balance;
40    reg [7:0] trace_balance_q;
41
42    always @* begin
43        trace_balance = trace_balance_q;
44        if (trace_valid_0) trace_balance =
trace_balance + 1;
45        if (trace_valid_1) trace_balance =
trace_balance - 1;
46    end
47
48    always @(posedge clk) begin
49        trace_balance_q <= resetn ? trace_balance : 0;
50    end
51
52    picorv32 #(
53        // do not change this settings
54        .ENABLE_COUNTERS(ENABLE_COUNTERS),
55        .ENABLE_TRACE(1),
56
57        // change this settings as you like
58        .ENABLE_REGS_DUALPORT(1),
59        .TWO_STAGE_SHIFT(1),
60        .BARREL_SHIFTER(0),
61        .TWO_CYCLE_COMPARE(0),
62        .TWO_CYCLE_ALU(0),
63        .COMPRESSED_ISA(0),
64        .ENABLE_MUL(0),
65        .ENABLE_DIV(0)
66    ) cpu_0 (
67        .clk (clk),
68        .resetn (resetn),
69        .trap (trap_0),
70        .mem_valid (mem_valid_0),
71        .mem_instr (mem_instr_0),
72        .mem_ready (mem_ready_0),
73        .mem_addr (mem_addr_0),
74        .mem_wdata (mem_wdata_0),
75        .mem_wstrb (mem_wstrb_0),
76        .mem_rdata (mem_rdata_0),
77        .trace_valid (trace_valid_0),
78        .trace_data (trace_data_0)
79    );
80
81    picorv32 #(
82        // do not change this settings
83        .ENABLE_COUNTERS(ENABLE_COUNTERS),
84        .ENABLE_TRACE(1),
85
86        // change this settings as you like
87        .ENABLE_REGS_DUALPORT(1),
88        .TWO_STAGE_SHIFT(1),
89        .BARREL_SHIFTER(0),
90        .TWO_CYCLE_COMPARE(0),
91        .TWO_CYCLE_ALU(0),
92        .COMPRESSED_ISA(0),
93        .ENABLE_MUL(0),
94        .ENABLE_DIV(0)
95    ) cpu_1 (
96        .clk (clk),
97        .resetn (resetn),
98        .trap (trap_1),
99        .mem_valid (mem_valid_1),
100       .mem_instr (mem_instr_1),
101       .mem_ready (mem_ready_1),
102       .mem_addr (mem_addr_1),
103       .mem_wdata (mem_wdata_1),
104       .mem_wstrb (mem_wstrb_1),

```

```

105         .mem_rdata    (mem_rdata_1 ),
106         .trace_valid  (trace_valid_1),
107         .trace_data   (trace_data_1 )
108     );
109 endmodule

```

11. YosysHQ_picorv32/scripts/smtbmc/tracecmp2.v (196 lines)

```

1 module testbench(
2     input clk, mem_ready_0, mem_ready_1,
3     input [31:0] mem_rdata
4 );
5     // set this to 1 to test generation of counterexamples
6     localparam ENABLE_COUNTERS = 0;
7
8     reg resetn = 0;
9     always @(posedge clk) resetn <= 1;
10
11     (* keep *) wire trap_0, trace_valid_0, mem_valid_0,
mem_instr_0;
12     (* keep *) wire [3:0] mem_wstrb_0;
13     (* keep *) wire [31:0] mem_addr_0, mem_wdata_0,
mem_rdata_0;
14     (* keep *) wire [35:0] trace_data_0;
15
16     (* keep *) wire trap_1, trace_valid_1, mem_valid_1,
mem_instr_1;
17     (* keep *) wire [3:0] mem_wstrb_1;
18     (* keep *) wire [31:0] mem_addr_1, mem_wdata_1,
mem_rdata_1;
19     (* keep *) wire [35:0] trace_data_1;
20
21     reg [31:0] last_mem_rdata;
22
23     assign mem_rdata_0 = mem_rdata;
24     assign mem_rdata_1 = mem_rdata;
25
26     wire mem_xfer_0 = resetn && mem_valid_0 && mem_ready_0;
27     wire mem_xfer_1 = resetn && mem_valid_1 && mem_ready_1;
28
29     reg [1:0] cmp_mem_state = 0;
30     reg [31:0] cmp_mem_addr;
31     reg [31:0] cmp_mem_wdata;
32     reg [3:0] cmp_mem_wstrb;
33
34     always @* begin
35         if (mem_valid_0 == 0)
36             assume(!mem_ready_0 == 0);
37         if (mem_valid_1 == 0)
38             assume(mem_ready_1 == 0);
39     end
40
41     always @(posedge clk) begin
42         if (cmp_mem_state)
43             assume(last_mem_rdata == mem_rdata);
44         last_mem_rdata <= mem_rdata;
45     end
46
47     always @(posedge clk) begin
48         case (cmp_mem_state)
49             2'b 00: begin
50                 case ({mem_xfer_1, mem_xfer_0})
51                     2'b 11: begin
52                         as-
53                         srt(mem_addr_0 == mem_addr_1);
54                         as-
55                         srt(mem_wstrb_0 == mem_wstrb_1);
56                         if
57                             (mem_wstrb_0[3]) assert(mem_wdata_0[31:24] == mem_wdata_1[31:24]);
58                         if
59                             (mem_wstrb_0[2]) assert(mem_wdata_0[23:16] == mem_wdata_1[23:16]);
60                         if
61                             (mem_wstrb_0[1]) assert(mem_wdata_0[15: 8] == mem_wdata_1[15: 8]);
62                         if
63                             (mem_wstrb_0[0]) assert(mem_wdata_0[ 7: 0] == mem_wdata_1[ 7: 0]);
64                         end
65                         2'b 01: begin
66                             cmp_mem_state
67                             <= 2'b 01;
68
69                             cmp_mem_addr <=
mem_addr_0;
70                             cmp_mem_wdata
71                             <= mem_wdata_0;
72                             cmp_mem_wstrb
73                             <= mem_wstrb_0;
74                         end
75                     end
76                 end
77             end
78             2'b 10: begin
79                 cmp_mem_state
80                 <= 2'b 10;
81
82                 cmp_mem_addr <=
mem_addr_1;
83                 cmp_mem_wdata
84                 <= mem_wdata_1;
85                 cmp_mem_wstrb
86                 <= mem_wstrb_1;
87             end
88         endcase
89     end
90 end

```

```

73     2'b 01: begin
74         assume(!mem_xfer_0);
75         if (mem_xfer_1) begin
76             cmp_mem_state <= 2'b
00;
77             assert(cmp_mem_addr ==
mem_addr_1);
78
79             assert(cmp_mem_wstrb ==
mem_wstrb_1);
80             if (cmp_mem_wstrb[3])
81                 assert(cmp_mem_wdata[31:24] == mem_wdata_1[31:24]);
82             if (cmp_mem_wstrb[2])
83                 assert(cmp_mem_wdata[23:16] == mem_wdata_1[23:16]);
84             if (cmp_mem_wstrb[1])
85                 assert(cmp_mem_wdata[15: 8] == mem_wdata_1[15: 8]);
86             if (cmp_mem_wstrb[0])
87                 assert(cmp_mem_wdata[ 7: 0] == mem_wdata_1[ 7: 0]);
88             end
89         end
90     end
91     2'b 10: begin
92         assume(!mem_xfer_1);
93         if (mem_xfer_0) begin
94             cmp_mem_state <= 2'b
00;
95             assert(cmp_mem_addr ==
mem_addr_0);
96
97             assert(cmp_mem_wstrb ==
mem_wstrb_0);
98             if (cmp_mem_wstrb[3])
99                 assert(cmp_mem_wdata[31:24] == mem_wdata_0[31:24]);
100             if (cmp_mem_wstrb[2])
101                 assert(cmp_mem_wdata[23:16] == mem_wdata_0[23:16]);
102             if (cmp_mem_wstrb[1])
103                 assert(cmp_mem_wdata[15: 8] == mem_wdata_0[15: 8]);
104             if (cmp_mem_wstrb[0])
105                 assert(cmp_mem_wdata[ 7: 0] == mem_wdata_0[ 7: 0]);
106             end
107         end
108     endcase
109 end
110     reg [1:0] cmp_trace_state = 0;
111     reg [35:0] cmp_trace_data;
112
113     always @(posedge clk) begin
114         if (resetn) begin
115             case (cmp_trace_state)
116                 2'b 00: begin
117                     case ({trace_valid_1,
trace_valid_0})
118                         2'b 11: begin
119                             as-
120                             srt(trace_data_0 == trace_data_1);
121                             end
122                         2'b 01: begin
123                             cmp_trace_state <= 2'b 01;
124                             cmp_trace_data <= trace_data_0;
125                             end
126                         2'b 10: begin
127                             cmp_trace_state <= 2'b 10;
128                             cmp_trace_data <= trace_data_1;
129                             end
130                     endcase
131                 end
132             end
133         end
134     end
135     2'b 01: begin
136         assume(!trace_valid_0);
137         if (trace_valid_1) be-
gin
138             cmp_trace_state
139             <= 2'b 00;
140             as-
141             srt(cmp_trace_data == trace_data_1);
142             end
143         end
144     end
145     2'b 10: begin
146         assume(!trace_valid_1);
147         if (trace_valid_0) be-
gin
148             cmp_trace_state
149             <= 2'b 00;
150             as-
151             srt(cmp_trace_data == trace_data_0);
152             end
153         end
154     end
155 endcase
156 end
157
158 picorv32 #(
159     // do not change this settings
160     .ENABLE_COUNTERS(ENABLE_COUNTERS),
161     .ENABLE_TRACE(1),
162 )

```



```

144         // change this settings as you like
145         .ENABLE_REGS_DUALPORT(1),
146         .TWO_STAGE_SHIFT(0),
147         .BARREL_SHIFTER(0),
148         .TWO_CYCLE_COMPARE(0),
149         .TWO_CYCLE_ALU(0),
150         .COMPRESSED_ISA(1),
151         .ENABLE_MUL(0),
152         .ENABLE_DIV(0)
153     ) cpu_0 (
154         .clk          (clk          ),
155         .resetn       (resetn       ),
156         .trap         (trap_0       ),
157         .mem_valid    (mem_valid_0   ),
158         .mem_instr     (mem_instr_0  ),
159         .mem_ready     (mem_ready_0  ),
160         .mem_addr      (mem_addr_0   ),
161         .mem_wdata     (mem_wdata_0  ),
162         .mem_wstrb     (mem_wstrb_0  ),
163         .mem_rdata     (mem_rdata_0  ),
164         .trace_valid   (trace_valid_0),
165         .trace_data    (trace_data_0 )
166     );
167
168     picorv32 #(
169         // do not change this settings
170         .ENABLE_COUNTERS(ENABLE_COUNTERS),
171         .ENABLE_TRACE(1),
172
173         // change this settings as you like
174         .ENABLE_REGS_DUALPORT(1),
175         .TWO_STAGE_SHIFT(1),
176         .BARREL_SHIFTER(0),
177         .TWO_CYCLE_COMPARE(0),
178         .TWO_CYCLE_ALU(0),
179         .COMPRESSED_ISA(1),
180         .ENABLE_MUL(0),
181         .ENABLE_DIV(0)
182     ) cpu_1 (
183         .clk          (clk          ),
184         .resetn       (resetn       ),
185         .trap         (trap_1       ),
186         .mem_valid    (mem_valid_1  ),
187         .mem_instr     (mem_instr_1  ),
188         .mem_ready     (mem_ready_1  ),
189         .mem_addr      (mem_addr_1   ),
190         .mem_wdata     (mem_wdata_1  ),
191         .mem_wstrb     (mem_wstrb_1  ),
192         .mem_rdata     (mem_rdata_1  ),
193         .trace_valid   (trace_valid_1),
194         .trace_data    (trace_data_1 )
195     );
196 endmodule

```

12. YosysHQ_picorv32/scripts/smtbmc/tracecmp3.v (135 lines)

1 // Based on the benchmark from 2016 Yosys-SMTBMC presentation,
which in turn is

2 // based on the tracecmp2 test from this directory.

```

3
4 module testbench (
5     input clk,
6     input [31:0] mem_rdata_in,
7
8     input          pcpi_wr,
9     input [31:0]  pcpi_rd,
10    input          pcpi_wait,
11    input          pcpi_ready
12 );
13
14    reg resetn = 0;
15
16    always @(posedge clk)
17        resetn <= 1;
18
19    wire      cpu0_trap;
20    wire      cpu0_mem_valid;
21    wire      cpu0_mem_instr;
22    wire      cpu0_mem_ready;
23    wire [31:0] cpu0_mem_addr;
24    wire [31:0] cpu0_mem_wdata;
25    wire [3:0]  cpu0_mem_wstrb;
26    wire [31:0] cpu0_mem_rdata;
27    wire      cpu0_trace_valid;
28    wire [35:0] cpu0_trace_data;
29
30    wire      cpu1_trap;
31    wire      cpu1_mem_valid;
32    wire      cpu1_mem_instr;
33    wire      cpu1_mem_ready;
34    wire [31:0] cpu1_mem_addr;
35    wire [31:0] cpu1_mem_wdata;
36    wire [3:0]  cpu1_mem_wstrb;
37    wire [31:0] cpu1_mem_rdata;
38    wire      cpu1_trace_valid;
39    wire [35:0] cpu1_trace_data;
40
41    wire      mem_ready;
42    wire [31:0] mem_rdata;

```

```

43    assign mem_ready = cpu0_mem_valid && cpu1_mem_valid;
44    assign mem_rdata = mem_rdata_in;
45
46    assign cpu0_mem_ready = mem_ready;
47    assign cpu0_mem_rdata = mem_rdata;
48
49    assign cpu1_mem_ready = mem_ready;
50    assign cpu1_mem_rdata = mem_rdata;
51
52    reg [2:0] trace_balance = 3'b010;
53    reg [35:0] trace_buffer_cpu0 = 0, trace_buffer_cpu1 =
0;
54
55    always @(posedge clk) begin
56        if (resetn) begin
57            if (cpu0_trace_valid)
58                trace_buffer_cpu0 <=
cpu0_trace_data;
59
60            if (cpu1_trace_valid)
61                trace_buffer_cpu1 <=
cpu1_trace_data;
62
63            if (cpu0_trace_valid && !
cpu1_trace_valid)
64                trace_balance <= trace_balance
<< 1;
65
66            if (!cpu0_trace_valid &&
cpu1_trace_valid)
67                trace_balance <= trace_balance
>> 1;
68        end
69    end
70
71    always @* begin
72        if (resetn && cpu0_mem_ready) begin
73            assert(cpu0_mem_addr == cpu1_mem_addr);
74            assert(cpu0_mem_wstrb ==
cpu1_mem_wstrb);
75            if (cpu0_mem_wstrb[3]) as-
sert(cpu0_mem_wdata[31:24] == cpu1_mem_wdata[31:24]);
76            if (cpu0_mem_wstrb[2]) as-
sert(cpu0_mem_wdata[23:16] == cpu1_mem_wdata[23:16]);
77            if (cpu0_mem_wstrb[1]) as-
sert(cpu0_mem_wdata[15: 8] == cpu1_mem_wdata[15: 8]);
78            if (cpu0_mem_wstrb[0]) as-
sert(cpu0_mem_wdata[ 7: 0] == cpu1_mem_wdata[ 7: 0]);
79        end
80        if (trace_balance == 3'b010) begin
81            assert(trace_buffer_cpu0 ==
trace_buffer_cpu1);
82        end
83    end
84
85    picorv32 #(
86        .ENABLE_COUNTERS(0),
87        .REGS_INIT_ZERO(1),
88        .COMPRESSED_ISA(1),
89        .ENABLE_TRACE(1),
90
91        .TWO_STAGE_SHIFT(0),
92        .ENABLE_PCPI(1)
93    ) cpu0 (
94        .clk          (clk          ),
95        .resetn       (resetn       ),
96        .trap         (cpu0_trap    ),
97        .mem_valid    (cpu0_mem_valid),
98        .mem_instr     (cpu0_mem_instr),
99        .mem_ready     (cpu0_mem_ready),
100        .mem_addr      (cpu0_mem_addr),
101        .mem_wdata     (cpu0_mem_wdata),
102        .mem_wstrb     (cpu0_mem_wstrb),
103        .mem_rdata     (cpu0_mem_rdata),
104        .pcpi_wr       (pcpi_wr     ),
105        .pcpi_rd       (pcpi_rd     ),
106        .pcpi_wait     (pcpi_wait   ),
107        .pcpi_ready    (pcpi_ready  ),
108        .trace_valid   (cpu0_trace_valid),
109        .trace_data    (cpu0_trace_data)
110    );
111
112    picorv32 #(
113        .ENABLE_COUNTERS(0),
114        .REGS_INIT_ZERO(1),
115        .COMPRESSED_ISA(1),
116        .ENABLE_TRACE(1),
117
118        .TWO_STAGE_SHIFT(1),
119        .TWO_CYCLE_COMPARE(1),
120        .TWO_CYCLE_ALU(1)
121    ) cpu1 (
122        .clk          (clk          ),
123        .resetn       (resetn       ),
124        .trap         (cpu1_trap    ),
125        .mem_valid    (cpu1_mem_valid),
126        .mem_instr     (cpu1_mem_instr),
127        .mem_ready     (cpu1_mem_ready),
128        .mem_addr      (cpu1_mem_addr),
129        .mem_wdata     (cpu1_mem_wdata),
130        .mem_wstrb     (cpu1_mem_wstrb),

```

```

131         .mem_rdata    (cpu1_mem_rdata ),
132         .trace_valid  (cpu1_trace_valid),
133         .trace_data   (cpu1_trace_data )
134     );
135 endmodule

```

13. YosysHQ_picorv32/scripts/cxxdemo/testbench.v (114 lines)

```

1  `timescale 1 ns / 1 ps
2  `undef VERBOSE_MEM
3  `undef WRITE_VCD
4  `undef MEM8BIT
5
6  module testbench;
7      reg clk = 1;
8      reg resetn = 0;
9      wire trap;
10
11     always #5 clk = ~clk;
12
13     initial begin
14         repeat (100) @(posedge clk);
15         resetn <= 1;
16     end
17
18     wire mem_valid;
19     wire mem_instr;
20     reg mem_ready;
21     wire [31:0] mem_addr;
22     wire [31:0] mem_wdata;
23     wire [3:0] mem_wstrb;
24     reg [31:0] mem_rdata;
25
26     picorv32 #(
27         .COMPRESSED_ISA(1)
28     ) uut (
29         .clk          (clk          ),
30         .resetn       (resetn       ),
31         .trap         (trap         ),
32         .mem_valid    (mem_valid    ),
33         .mem_instr    (mem_instr    ),
34         .mem_ready    (mem_ready    ),
35         .mem_addr     (mem_addr     ),
36         .mem_wdata    (mem_wdata    ),
37         .mem_wstrb    (mem_wstrb    ),
38         .mem_rdata    (mem_rdata    )
39     );
40
41     localparam MEM_SIZE = 4*1024*1024;
42     `ifdef MEM8BIT
43         reg [7:0] memory [0:MEM_SIZE-1];
44         initial $readmemh("firmware.hex", memory);
45     `else
46         reg [31:0] memory [0:MEM_SIZE/4-1];
47         initial $readmemh("firmware32.hex", memory);
48     `endif
49
50     always @(posedge clk) begin
51         mem_ready <= 0;
52         if (mem_valid && !mem_ready) begin
53             mem_ready <= 1;
54             mem_rdata <= 'bx;
55             case (1)
56                 mem_addr < MEM_SIZE: begin
57                     `ifdef MEM8BIT
58                         if (!mem_wstrb) begin
59                             if
60                                 (mem_wstrb[0]) memory[mem_addr + 0] <= mem_wdata[ 7: 0];
61                             if
62                                 (mem_wstrb[1]) memory[mem_addr + 1] <= mem_wdata[15: 8];
63                             if
64                                 (mem_wstrb[2]) memory[mem_addr + 2] <= mem_wdata[23:16];
65                             if
66                                 (mem_wstrb[3]) memory[mem_addr + 3] <= mem_wdata[31:24];
67                             end else begin
68                                 mem_rdata <=
69                                     {memory[mem_addr+3], memory[mem_addr+2], memory[mem_addr+1],
70                                     memory[mem_addr]};
71                             end
72                         `else
73                             if (!mem_wstrb) begin
74                                 if
75                                     (mem_wstrb[0]) memory[mem_addr >> 2][ 7: 0] <= mem_wdata[ 7: 0];
76                                 if
77                                     (mem_wstrb[1]) memory[mem_addr >> 2][15: 8] <= mem_wdata[15: 8];
78                                 if
79                                     (mem_wstrb[2]) memory[mem_addr >> 2][23:16] <= mem_wdata[23:16];
80                                 if
81                                     (mem_wstrb[3]) memory[mem_addr >> 2][31:24] <= mem_wdata[31:24];
82                                 end else begin
83                                     mem_rdata <=
84                                         memory[mem_addr >> 2];
85                                 end
86                             `endif
87                             end
88                             mem_addr == 32'h 1000_0000: be-
89                                 gin
90                                     $write("%c",
91                                     mem_wdata[7:0]);
92                                 end
93                                 endcase
94                                 end
95                                 if (mem_valid && mem_ready) begin
96                                     `ifdef VERBOSE_MEM
97                                         if (!mem_wstrb)
98                                             $display("WR: ADDR=%x DATA=%x
99                                             MASK=%b", mem_addr, mem_wdata, mem_wstrb);
100                                         else
101                                             $display("RD: ADDR=%x
102                                             DATA=%x%s", mem_addr, mem_rdata, mem_instr ? " INSN" : "");
103                                     `endif
104                                     if (^mem_addr == 1'bx ||
105                                         ^mem_wdata[ 7: 0] == 1'bx ||
106                                         ^mem_wdata[15: 8] == 1'bx ||
107                                         ^mem_wdata[23:16] == 1'bx ||
108                                         ^mem_wdata[31:24] == 1'bx) begin
109                                         $display("CRITICAL UNDEF MEM
110                                         TRANSACTION");
111                                         $finish;
112                                     end
113                                     end
114                                     end
115                                     `ifdef WRITE_VCD
116                                         initial begin
117                                             $dumpfile("testbench.vcd");
118                                             $dumpvars(0, testbench);
119                                         end
120                                     `endif
121                                     always @(posedge clk) begin
122                                         if (resetn && trap) begin
123                                             repeat (10) @(posedge clk);
124                                             $display("TRAP");
125                                             $finish;
126                                         end
127                                     end
128                                 endmodule

```

SERV -- 비트직렬 RISC-V

18 files • 3,562 lines

API — serv_aligner.v

Table 1145. olofk_serv/rtl/serv_aligner.v — module / interface / package (1개)

이름	종류
serv_aligner	module

Table 1146. olofk_serv/rtl/serv_aligner.v — 포트 (10개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
i_ibus_adr	input	wire [31:0]
i_ibus_cyc	input	wire
o_ibus_rdt	output	wire [31:0]
o_ibus_ack	output	wire
o_wb_ibus_adr	output	wire [31:0]
o_wb_ibus_cyc	output	wire
i_wb_ibus_rdt	input	wire [31:0]
i_wb_ibus_ack	input	wire

API — serv_alu.v

Table 1147. olofk_serv/rtl/serv_alu.v — module / interface / package (1개)

이름	종류
serv_alu	module

Table 1148. olofk_serv/rtl/serv_alu.v — parameter (2개)

파라미터	형/폭	기본값
W		1
B		W-1

Table 1149. olofk_serv/rtl/serv_alu.v — 포트 (13개)

포트	방향	형/폭
clk	input	wire
i_en	input	wire
i_cnt0	input	wire
o_cmp	output	wire

포트	방향	형/폭
i_sub	input	wire
i_bool_op	input	wire [1:0]
i_cmp_eq	input	wire
i_cmp_sig	input	wire
i_rd_sel	input	wire [2:0]
i_rs1	input	wire [B:0]
i_op_b	input	wire [B:0]
i_buf	input	wire [B:0]
o_rd	output	wire [B:0]

API — serv_bufreg.v

Table 1150. olofk_serv/rtl/serv_bufreg.v — module / interface / package (1개)

이름	종류
serv_bufreg	module

Table 1151. olofk_serv/rtl/serv_bufreg.v — parameter (3개)

파라미터	형/폭	기본값
MDU	[0:0]	0
W		1
B		W-1

Table 1152. olofk_serv/rtl/serv_bufreg.v — 포트 (20개)

포트	방향	형/폭
i_clk	input	wire
i_cnt0	input	wire
i_cnt1	input	wire
i_cnt_done	input	wire
i_en	input	wire
i_init	input	wire
i_mdu_op	input	wire
o_lsb	output	wire [1:0]
i_rs1_en	input	wire
i_imm_en	input	wire
i_clr_lsb	input	wire
i_shift_op	input	wire
i_right_shift_op	input	wire

포트	방향	형/폭
i_shamt	input	wire [2:0]
i_sh_signed	input	wire
i_rs1	input	wire [B:0]
i_imm	input	wire [B:0]
o_q	output	wire [B:0]
o_dbus_adr	output	wire [31:0]
o_ext_rs1	output	wire [31:0]

API — serv_bufreg2.v

Table 1153. olofk_serv/rtl/serv_bufreg2.v — module / interface / package (1개)

이름	종류
serv_bufreg2	module

Table 1154. olofk_serv/rtl/serv_bufreg2.v — parameter (1개)

파라미터	형/폭	기본값
B		W-1)

Table 1155. olofk_serv/rtl/serv_bufreg2.v — 포트 (18개)

포트	방향	형/폭
i_clk	input	wire
i_en	input	wire
i_init	input	wire
i_cnt7	input	wire
i_cnt_done	input	wire
i_sh_right	input	wire
i_lsb	input	wire [1:0]
i_bytecnt	input	wire [1:0]
o_sh_done	output	wire
i_op_b_sel	input	wire
i_shift_op	input	wire
i_rs2	input	wire [B:0]
i_imm	input	wire [B:0]
o_op_b	output	wire [B:0]
o_q	output	wire [B:0]
o_dat	output	wire [31:0]
i_load	input	wire

포트	방향	형/폭
i_dat	input	wire [31:0]

API — serv_compdec.v

Table 1156. olofk_serv/rtl/serv_compdec.v — module / interface / package (1개)

이름	종류
serv_compdec	module

Table 1157. olofk_serv/rtl/serv_compdec.v — localparam (8개)

상수	형/폭	값
OPCODE_LOAD		7'h03
OPCODE_OP_IMM		7'h13
OPCODE_STORE		7'h23
OPCODE_OP		7'h33
OPCODE_LUI		7'h37
OPCODE_BRANCH		7'h63
OPCODE_JALR		7'h67
OPCODE_JAL		7'h6f

Table 1158. olofk_serv/rtl/serv_compdec.v — 포트 (5개)

포트	방향	형/폭
i_clk	input	wire
i_instr	input	wire [31:0]
i_ack	input	wire
o_instr	output	wire [31:0]
o_iscomp	output	reg

API — serv_csr.v

Table 1159. olofk_serv/rtl/serv_csr.v — module / interface / package (1개)

이름	종류
serv_csr	module

Table 1160. olofk_serv/rtl/serv_csr.v — parameter (3개)

파라미터	형/폭	기본값
RESET_STRATEGY		"MINI"
W		1
B		W-1

Table 1161. olofk_serv/rtl/serv_csr.v — localparam (1개)

상수	형/폭	값
CSR_SOURCE_CSR	[1:0]	2'b00

Table 1162. olofk_serv/rtl/serv_csr.v — 포트 (28개)

포트	방향	형/폭
i_clk	input	wire
i_rst	input	wire
i_trig_irq	input	wire
i_en	input	wire
i_cnt0to3	input	wire
i_cnt3	input	wire
i_cnt7	input	wire
i_cnt11	input	wire
i_cnt12	input	wire
i_cnt_done	input	wire
i_mem_op	input	wire
i_mtip	input	wire
i_trap	input	wire
o_new_irq	output	reg
i_e_op	input	wire
i_ebreak	input	wire
i_mem_cmd	input	wire
i_mstatus_en	input	wire
i_mie_en	input	wire
i_mcause_en	input	wire
i_csr_source	input	wire [1:0]
i_mret	input	wire
i_csr_d_sel	input	wire
i_rf_csr_out	input	wire [B:0]
o_csr_in	output	wire [B:0]
i_csr_imm	input	wire [B:0]
i_rs1	input	wire [B:0]
o_q	output	wire [B:0]

API — serv_ctrl.v**Table 1163. olofk_serv/rtl/serv_ctrl.v — module / interface / package (1개)**

이름	종류
serv_ctrl	module

Table 1164. olofk_serv/rtl/serv_ctrl.v — parameter (4개)

파라미터	형/폭	기본값
RESET_PC		32'd0
WITH_CSR		1
W		1
B		W-1

Table 1165. olofk_serv/rtl/serv_ctrl.v — 포트 (19개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_pc_en	input	wire
i_cnt12to31	input	wire
i_cnt0	input	wire
i_cnt1	input	wire
i_cnt2	input	wire
i_jump	input	wire
i_jal_or_jalr	input	wire
i_ctype	input	wire
i_pc_rel	input	wire
i_trap	input	wire
i_iscomp	input	wire
i_imm	input	wire [B:0]
i_buf	input	wire [B:0]
i_csr_pc	input	wire [B:0]
o_rd	output	wire [B:0]
o_bad_pc	output	wire [B:0]
o_ibus_adr	output	reg [31:0]

API — serv_debug.v**Table 1166. olofk_serv/rtl/serv_debug.v — module / interface / package (1개)**

이름	종류
serv_debug	module

Table 1167. olofk_serv/rtl/serv_debug.v — parameter (2개)

파라미터	형/폭	기본값
RESET_PC		0
B		W-1)

Table 1168. olofk_serv/rtl/serv_debug.v — 포트 (55개)

포트	방향	형/폭
b0	output	reg rvfi_valid = 1'
d0	output	reg [63:0] rvfi_order = 64'
d0	output	reg [31:0] rvfi_insn = 32'
b0	output	reg rvfi_trap = 1'
b0	output	reg rvfi_halt = 1'
b0	output	reg rvfi_intr = 1'
b11	output	reg [1:0] rvfi_mode = 2'
b01	output	reg [1:0] rvfi_ixl = 2'
rvfi_rs1_addr	output	reg [4:0]
rvfi_rs2_addr	output	reg [4:0]
rvfi_rs1_rdata	output	reg [31:0]
rvfi_rs2_rdata	output	reg [31:0]
rvfi_rd_addr	output	reg [4:0]
rvfi_rd_wdata	output	wire [31:0]
rvfi_pc_rdata	output	reg [31:0]
rvfi_pc_wdata	output	wire [31:0]
rvfi_mem_addr	output	reg [31:0]
rvfi_mem_rmask	output	reg [3:0]
rvfi_mem_wmask	output	reg [3:0]
rvfi_mem_rdata	output	reg [31:0]
rvfi_mem_wdata	output	reg [31:0]
i_dbus_adr	input	wire [31:0]
i_dbus_dat	input	wire [31:0]
i_dbus_sel	input	wire [3:0]
i_dbus_we	input	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
i_ctrl_pc_en	input	wire
rs1	input	wire [B:0]
rs2	input	wire [B:0]

포트	방향	형/폭
rs1_addr	input	wire [4:0]
rs2_addr	input	wire [4:0]
immdec_en	input	wire [3:0]
rd_en	input	wire
trap	input	wire
i_rf_ready	input	wire
i_ibus_cyc	input	wire
two_stage_op	input	wire
init	input	wire
i_ibus_adr	input	wire [31:0]
i_clk	input	wire
i_rst	input	wire
i_ibus_rdt	input	wire [31:0]
i_ibus_ack	input	wire
i_rd_addr	input	wire [4:0]
i_cnt_en	input	wire
i_csr_in	input	wire [B:0]
i_csr_mstatus_en	input	wire
i_csr_mie_en	input	wire
i_csr_mcause_en	input	wire
i_csr_en	input	wire
i_csr_addr	input	wire [1:0]
i_wen0	input	wire
i_wdata0	input	wire [B:0]
i_cnt_done	input	wire

API — serv_decode.v

Table 1169. olofk_serv/rtl/serv_decode.v — module / interface / package (1개)

이름	종류
serv_decode	module

Table 1170. olofk_serv/rtl/serv_decode.v — parameter (1개)

파라미터	형/폭	기본값
MDU	[0:0]	0)

Table 1171. olofk_serv/rtl/serv_decode.v — 포트 (47개)

포트	방향	형/폭
clk	input	wire
i_wb_rdt	input	wire [31:2]
i_wb_en	input	wire
o_sh_right	output	reg
o_bne_or_bge	output	reg
o_cond_branch	output	reg
o_e_op	output	reg
o_ebreak	output	reg
o_branch_op	output	reg
o_shift_op	output	reg
o_rd_op	output	reg
o_two_stage_op	output	reg
o_dbus_en	output	reg
o_mdu_op	output	reg
o_ext_funct3	output	reg [2:0]
o_bufreg_rs1_en	output	reg
o_bufreg_imm_en	output	reg
o_bufreg_clr_lsb	output	reg
o_bufreg_sh_signed	output	reg
o_ctrl_jal_or_jalr	output	reg
o_ctrl_ctype	output	reg
o_ctrl_pc_rel	output	reg
o_ctrl_mret	output	reg
o_alu_sub	output	reg
o_alu_bool_op	output	reg [1:0]
o_alu_cmp_eq	output	reg
o_alu_cmp_sig	output	reg
o_alu_rd_sel	output	reg [2:0]
o_mem_signed	output	reg
o_mem_word	output	reg
o_mem_half	output	reg
o_mem_cmd	output	reg
o_csr_en	output	reg
o_csr_addr	output	reg [1:0]
o_csr_mstatus_en	output	reg

포트	방향	형/폭
o_csr_mie_en	output	reg
o_csr_mcause_en	output	reg
o_csr_source	output	reg [1:0]
o_csr_d_sel	output	reg
o_csr_imm_en	output	reg
o_mtval_pc	output	reg
o_immdec_ctrl	output	reg [3:0]
o_immdec_en	output	reg [3:0]
o_op_b_source	output	reg
o_rd_mem_en	output	reg
o_rd_csr_en	output	reg
o_rd_alu_en	output	reg

API — serv_immdec.v

Table 1172. olofk_serv/rtl/serv_immdec.v — module / interface / package (1개)

이름	종류
serv_immdec	module

Table 1173. olofk_serv/rtl/serv_immdec.v — parameter (1개)

파라미터	형/폭	기본값
W		1)

Table 1174. olofk_serv/rtl/serv_immdec.v — 포트 (13개)

포트	방향	형/폭
i_clk	input	wire
i_cnt_en	input	wire
i_cnt_done	input	wire
i_immdec_en	input	wire [3:0]
i_csr_imm_en	input	wire
i_ctrl	input	wire [3:0]
o_rd_addr	output	wire [4:0]
o_rs1_addr	output	wire [4:0]
o_rs2_addr	output	wire [4:0]
o_csr_imm	output	wire [W-1:0]
o_imm	output	wire [W-1:0]
i_wb_en	input	wire

포트	방향	형/폭
i_wb_rdt	input	wire [31:7]

API — serv_mem_if.v

Table 1175. olofk_serv/rtl/serv_mem_if.v — module / interface / package (1개)

이름	종류
serv_mem_if	module

Table 1176. olofk_serv/rtl/serv_mem_if.v — parameter (3개)

파라미터	형/폭	기본값
WITH_CSR	[0:0]	1
W		1
B		W-1

Table 1177. olofk_serv/rtl/serv_mem_if.v — 포트 (11개)

포트	방향	형/폭
i_clk	input	wire
i_bytecnt	input	wire [1:0]
i_lsb	input	wire [1:0]
o_misalign	output	wire
i_signed	input	wire
i_word	input	wire
i_half	input	wire
i_mdu_op	input	wire
i_bufreg2_q	input	wire [B:0]
o_rd	output	wire [B:0]
o_wb_sel	output	wire [3:0]

API — serv_rf_if.v

Table 1178. olofk_serv/rtl/serv_rf_if.v — module / interface / package (1개)

이름	종류
serv_rf_if	module

Table 1179. olofk_serv/rtl/serv_rf_if.v — parameter (2개)

파라미터	형/폭	기본값
W		1
B		W-1

Table 1180. olofk_serv/rtl/serv_rf_if.v — 포트 (35개)

포트	방향	형/폭
i_cnt_en	input	wire
o_wreg0	output	wire [4+WITH_CSR:0]
o_wreg1	output	wire [4+WITH_CSR:0]
o_wen0	output	wire
o_wen1	output	wire
o_wdata0	output	wire [B:0]
o_wdata1	output	wire [B:0]
o_rreg0	output	wire [4+WITH_CSR:0]
o_rreg1	output	wire [4+WITH_CSR:0]
i_rdata0	input	wire [B:0]
i_rdata1	input	wire [B:0]
i_trap	input	wire
i_mret	input	wire
i_mepc	input	wire [B:0]
i_mtval_pc	input	wire
i_bufreg_q	input	wire [B:0]
i_bad_pc	input	wire [B:0]
o_csr_pc	output	wire [B:0]
i_csr_en	input	wire
i_csr_addr	input	wire [1:0]
i_csr	input	wire [B:0]
o_csr	output	wire [B:0]
i_rd_wen	input	wire
i_rd_waddr	input	wire [4:0]
i_ctrl_rd	input	wire [B:0]
i_alu_rd	input	wire [B:0]
i_rd_alu_en	input	wire
i_csr_rd	input	wire [B:0]
i_rd_csr_en	input	wire
i_mem_rd	input	wire [B:0]
i_rd_mem_en	input	wire
i_rs1_raddr	input	wire [4:0]
o_rs1	output	wire [B:0]
i_rs2_raddr	input	wire [4:0]
o_rs2	output	wire [B:0]

API — serv_rf_ram.v**Table 1181. olofk_serv/rtl/serv_rf_ram.v — module / interface / package (1개)**

이름	종류
serv_rf_ram	module

Table 1182. olofk_serv/rtl/serv_rf_ram.v — parameter (2개)

파라미터	형/폭	기본값
csr_regs		4
depth		32*(32+csr_regs)/width

Table 1183. olofk_serv/rtl/serv_rf_ram.v — 포트 (6개)

포트	방향	형/폭
depth	input	wire [\$clog2(
i_wdata	input	wire [width-1:0]
i_wen	input	wire
depth	input	wire [\$clog2(
i_ren	input	wire
o_rdata	output	wire [width-1:0]

API — serv_rf_ram_if.v**Table 1184. olofk_serv/rtl/serv_rf_ram_if.v — module / interface / package (1개)**

이름	종류
serv_rf_ram_if	module

Table 1185. olofk_serv/rtl/serv_rf_ram_if.v — parameter (8개)

파라미터	형/폭	기본값
width		8
W		1
reset_strategy		"MINI"
csr_regs		4
B		W-1
raw		\$clog2(32+csr_regs)
l2w		\$clog2(width)
aw		5+raw-l2w) //Address width

Table 1186. olofk_serv/rtl/serv_rf_ram_if.v — localparam (3개)

상수	형/폭	값
ratio		width/W
CMSB		4-\$clog2(W)

상수	형/폭	값
<code>l2r</code>		<code>\$clog2(ratio)</code>

Table 1187. olofk_serv/rtl/serv_rf_ram_if.v — 포트 (21개)

포트	방향	형/폭
<code>i_clk</code>	input	wire
<code>i_rst</code>	input	wire
<code>i_wreq</code>	input	wire
<code>i_rreq</code>	input	wire
<code>o_ready</code>	output	wire
<code>i_wreg0</code>	input	wire [raw-1:0]
<code>i_wreg1</code>	input	wire [raw-1:0]
<code>i_wen0</code>	input	wire
<code>i_wen1</code>	input	wire
<code>i_wdata0</code>	input	wire [B:0]
<code>i_wdata1</code>	input	wire [B:0]
<code>i_rreg0</code>	input	wire [raw-1:0]
<code>i_rreg1</code>	input	wire [raw-1:0]
<code>o_rdata0</code>	output	wire [B:0]
<code>o_rdata1</code>	output	wire [B:0]
<code>o_waddr</code>	output	wire [aw-1:0]
<code>o_wdata</code>	output	wire [width-1:0]
<code>o_wen</code>	output	wire
<code>o_raddr</code>	output	wire [aw-1:0]
<code>o_ren</code>	output	wire
<code>i_rdata</code>	input	wire [width-1:0]

API — serv_rf_top.v**Table 1188. olofk_serv/rtl/serv_rf_top.v — module / interface / package (1개)**

이름	종류
<code>serv_rf_top</code>	module

Table 1189. olofk_serv/rtl/serv_rf_top.v — parameter (10개)

파라미터	형/폭	기본값
<code>COMPRESSED</code>	[0:0]	0
<code>ALIGN</code>	[0:0]	<code>COMPRESSED</code>
<code>MDU</code>	[0:0]	0

파라미터	형/폭	기본값
PRE_REGISTER		1
RESET_STRATEGY		"MINI"
DEBUG	[0:0]	1'b0
WITH_CSR		1
W		1
RF_WIDTH		$W * 2$
RF_L2D		$\$clog2((32+(WITH_CSR*4))*32/RF_WIDTH))$

Table 1190. olofk_serv/rtl/serv_rf_top.v — localparam (1개)

상수	형/폭	값
CSR_REGS		WITH_CSR*4

Table 1191. olofk_serv/rtl/serv_rf_top.v — 포트 (41개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_timer_irq	input	wire
rvfi_valid	output	wire
rvfi_order	output	wire [63:0]
rvfi_insn	output	wire [31:0]
rvfi_trap	output	wire
rvfi_halt	output	wire
rvfi_intr	output	wire
rvfi_mode	output	wire [1:0]
rvfi_ixl	output	wire [1:0]
rvfi_rs1_addr	output	wire [4:0]
rvfi_rs2_addr	output	wire [4:0]
rvfi_rs1_rdata	output	wire [31:0]
rvfi_rs2_rdata	output	wire [31:0]
rvfi_rd_addr	output	wire [4:0]
rvfi_rd_wdata	output	wire [31:0]
rvfi_pc_rdata	output	wire [31:0]
rvfi_pc_wdata	output	wire [31:0]
rvfi_mem_addr	output	wire [31:0]
rvfi_mem_rmask	output	wire [3:0]
rvfi_mem_wmask	output	wire [3:0]

포트	방향	형/폭
rvfi_mem_rdata	output	wire [31:0]
rvfi_mem_wdata	output	wire [31:0]
o_ibus_adr	output	wire [31:0]
o_ibus_cyc	output	wire
i_ibus_rdt	input	wire [31:0]
i_ibus_ack	input	wire
o_dbus_adr	output	wire [31:0]
o_dbus_dat	output	wire [31:0]
o_dbus_sel	output	wire [3:0]
o_dbus_we	output	wire
o_dbus_cyc	output	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
o_ext_rs1	output	wire [31:0]
o_ext_rs2	output	wire [31:0]
o_ext_funct3	output	wire [2:0]
i_ext_rd	input	wire [31:0]
i_ext_ready	input	wire
o_mdu_valid	output	wire

API — serv_state.v

Table 1192. olofk_serv/rtl/serv_state.v — module / interface / package (1개)

이름	종류
serv_state	module

Table 1193. olofk_serv/rtl/serv_state.v — parameter (4개)

파라미터	형/폭	기본값
WITH_CSR	[0:0]	1
ALIGN	[0:0]	0
MDU	[0:0]	0
W		1

Table 1194. olofk_serv/rtl/serv_state.v — 포트 (46개)

포트	방향	형/폭
i_clk	input	wire
i_rst	input	wire

포트	방향	형/폭
i_new_irq	input	wire
i_alu_cmp	input	wire
o_init	output	wire
o_cnt_en	output	wire
o_cnt0to3	output	wire
o_cnt12to31	output	wire
o_cnt0	output	wire
o_cnt1	output	wire
o_cnt2	output	wire
o_cnt3	output	wire
o_cnt7	output	wire
o_cnt11	output	wire
o_cnt12	output	wire
o_cnt_done	output	wire
o_bufreg_en	output	wire
o_ctrl_pc_en	output	wire
o_ctrl_jump	output	reg
o_ctrl_trap	output	wire
i_ctrl_misalign	input	wire
i_sh_done	input	wire
o_mem_bytecnt	output	wire [1:0]
i_mem_misalign	input	wire
i_bne_or_bge	input	wire
i_cond_branch	input	wire
i_dbus_en	input	wire
i_two_stage_op	input	wire
i_branch_op	input	wire
i_shift_op	input	wire
i_sh_right	input	wire
i_alu_rd_sel1	input	wire
i_rd_alu_en	input	wire
i_e_op	input	wire
i_rd_op	input	wire
i_mdu_op	input	wire
o_mdu_valid	output	wire

포트	방향	형/폭
i_mdu_ready	input	wire
o_dbus_cyc	output	wire
i_dbus_ack	input	wire
o_ibus_cyc	output	wire
i_ibus_ack	input	wire
o_rf_rreq	output	wire
o_rf_wreq	output	wire
i_rf_ready	input	wire
o_rf_rd_en	output	wire

API — serv_synth_wrapper.v

Table 1195. olofk_serv/rtl/serv_synth_wrapper.v — module / interface / package (1개)

이름	종류
serv_synth_wrapper	module

Table 1196. olofk_serv/rtl/serv_synth_wrapper.v — parameter (5개)

파라미터	형/폭	기본값
PRE_REGISTER		1
RESET_STRATEGY		"MINI"
WITH_CSR		1
RF_WIDTH		2
RF_L2D		$\$clog2((32+(WITH_CSR*4))*32/RF_WIDTH)$

Table 1197. olofk_serv/rtl/serv_synth_wrapper.v — localparam (1개)

상수	형/폭	값
CSR_REGS		WITH_CSR*4

Table 1198. olofk_serv/rtl/serv_synth_wrapper.v — 포트 (19개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_timer_irq	input	wire
o_ibus_adr	output	wire [31:0]
o_ibus_cyc	output	wire
i_ibus_rdt	input	wire [31:0]
i_ibus_ack	input	wire
o_dbus_adr	output	wire [31:0]

포트	방향	형/폭
o_dbus_dat	output	wire [31:0]
o_dbus_sel	output	wire [3:0]
o_dbus_we	output	wire
o_dbus_cyc	output	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
o_waddr	output	wire [RF_L2D-1:0]
o_wdata	output	wire [RF_WIDTH-1:0]
o_wen	output	wire
o_raddr	output	wire [RF_L2D-1:0]
i_rdata	input	wire [RF_WIDTH-1:0]

API — serv_top.v

Table 1199. olofk_serv/rtl/serv_top.v — module / interface / package (1개)

이름	종류
serv_top	module

Table 1200. olofk_serv/rtl/serv_top.v — parameter (9개)

파라미터	형/폭	기본값
W		1
B		W-1
PRE_REGISTER		1
RESET_STRATEGY		"MINI"
RESET_PC		32'd0
DEBUG	[0:0]	1'b0
MDU	[0:0]	1'b0
COMPRESSED	[0:0]	0
ALIGN	[0:0]	COMPRESSED)

Table 1201. olofk_serv/rtl/serv_top.v — 포트 (54개)

포트	방향	형/폭
clk	input	wire
i_rst	input	wire
i_timer_irq	input	wire
rvfi_valid	output	wire
rvfi_order	output	wire [63:0]

포트	방향	형/폭
rvfi_insn	output	wire [31:0]
rvfi_trap	output	wire
rvfi_halt	output	wire
rvfi_intr	output	wire
rvfi_mode	output	wire [1:0]
rvfi_ixl	output	wire [1:0]
rvfi_rs1_addr	output	wire [4:0]
rvfi_rs2_addr	output	wire [4:0]
rvfi_rs1_rdata	output	wire [31:0]
rvfi_rs2_rdata	output	wire [31:0]
rvfi_rd_addr	output	wire [4:0]
rvfi_rd_wdata	output	wire [31:0]
rvfi_pc_rdata	output	wire [31:0]
rvfi_pc_wdata	output	wire [31:0]
rvfi_mem_addr	output	wire [31:0]
rvfi_mem_rmask	output	wire [3:0]
rvfi_mem_wmask	output	wire [3:0]
rvfi_mem_rdata	output	wire [31:0]
rvfi_mem_wdata	output	wire [31:0]
o_rf_rreq	output	wire
o_rf_wreq	output	wire
i_rf_ready	input	wire
o_wreg0	output	wire [4+WITH_CSR:0]
o_wreg1	output	wire [4+WITH_CSR:0]
o_wen0	output	wire
o_wen1	output	wire
o_wdata0	output	wire [B:0]
o_wdata1	output	wire [B:0]
o_rreg0	output	wire [4+WITH_CSR:0]
o_rreg1	output	wire [4+WITH_CSR:0]
i_rdata0	input	wire [B:0]
i_rdata1	input	wire [B:0]
o_ibus_adr	output	wire [31:0]
o_ibus_cyc	output	wire
i_ibus_rdt	input	wire [31:0]

포트	방향	형/폭
i_ibus_ack	input	wire
o_dbus_adr	output	wire [31:0]
o_dbus_dat	output	wire [31:0]
o_dbus_sel	output	wire [3:0]
o_dbus_we	output	wire
o_dbus_cyc	output	wire
i_dbus_rdt	input	wire [31:0]
i_dbus_ack	input	wire
o_ext_func3	output	wire [2:0]
i_ext_ready	input	wire
i_ext_rd	input	wire [31:0]
o_ext_rs1	output	wire [31:0]
o_ext_rs2	output	wire [31:0]
o_mdu_valid	output	wire

1. olofk_serv/rtl/serv_aligner.v (73 lines)

```

1 /*
2  * serv_aligner.v : Realign a misaligned 32-bit word fetched
from memory
3  */
4  * SPDX-FileCopyrightText: 2022 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 module serv_aligner
8 (
9     input wire clk,
10    input wire rst,
11    // serv_top
12    input wire [31:0] i_ibus_adr,
13    input wire      i_ibus_cyc,
14    output wire [31:0] o_ibus_rdt,
15    output wire      o_ibus_ack,
16    // serv_rf_top
17    output wire [31:0] o_wb_ibus_adr,
18    output wire      o_wb_ibus_cyc,
19    input wire [31:0] i_wb_ibus_rdt,
20    input wire      i_wb_ibus_ack);
21
22    wire [31:0] ibus_rdt_concat;
23    wire      ack_en;
24
25    reg [15:0] lower_hw;
26    reg      ctrl_misal ;
27
28    /* From SERV core to Memory
29
30    o_wb_ibus_adr: Carries address of instruction to memory.
In case of misaligned access,
31    which is caused by pc+2 due to compressed instruction,
next instruction is fetched
32    by pc+4 and concatenation is done to make the instruction
aligned.
33
34    o_wb_ibus_cyc: Simply forwarded from SERV to Memory and is
only altered by memory or SERV core.
35    */
36    assign o_wb_ibus_adr = ctrl_misal ? (i_ibus_adr+32'b100) :
i_ibus_adr;
37    assign o_wb_ibus_cyc = i_ibus_cyc;
38
39    /* From Memory to SERV core
40
41    o_ibus_ack: Instruction bus acknowledge is send to
SERV only when the aligned instruction,
42    either compressed or un-compressed, is ready to dis-
patch.
43

```

```

44    o_ibus_rdt: Carries the instruction from memory to
SERV core. It can be either aligned
45    instruction coming from memory or made aligned by two
bus transactions and concatenation.
46    */
47    assign o_ibus_ack = i_wb_ibus_ack & ack_en;
48    assign o_ibus_rdt = ctrl_misal ? ibus_rdt_concat :
i_wb_ibus_rdt;
49
50    /* 16-bit register used to hold the upper half word of the
current instruction in-case
51
52    concatenation will be required with the upper half word of
upcoming instruction
53    */
54    always @(posedge clk) begin
55        if(i_wb_ibus_ack)begin
56            lower_hw <= i_wb_ibus_rdt[31:16];
57        end
58    end
59    assign ibus_rdt_concat = {i_wb_ibus_rdt[15:0], lower_hw};
60
61    /* Two control signals: ack_en, ctrl_misal are set to con-
trol the bus transactions between
62    SERV core and the memory
63    */
64    assign ack_en    = !(i_ibus_adr[1] & !ctrl_misal);
65
66    always @(posedge clk ) begin
67        if(rst)
68            ctrl_misal <= 0;
69        else if(i_wb_ibus_ack & i_ibus_adr[1])
70            ctrl_misal <= !ctrl_misal;
71    end
72
73 endmodule

```

2. olofk_serv/rtl/serv_alu.v (87 lines)

```

1 /*
2  * serv_alu.v : SERV Arithmetic Logic Unit
3  */
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8 module serv_alu
9 #(
10     parameter W = 1,
11     parameter B = W-1
12 )
13 (
14     input wire      clk,
15     //State

```

```

16 input wire    i_en,
17 input wire    i_cnt0,
18 output wire    o_cmp,
19 //Control
20 input wire    i_sub,
21 input wire [1:0] i_bool_op,
22 input wire    i_cmp_eq,
23 input wire    i_cmp_sig,
24 input wire [2:0] i_rd_sel,
25 //Data
26 input wire [B:0] i_rs1,
27 input wire [B:0] i_op_b,
28 input wire [B:0] i_buf,
29 output wire [B:0] o_rd;
30
31 wire [B:0] result_add;
32 wire [B:0] result_slt;
33
34 reg        cmp_r;
35
36 wire    add_cy;
37 reg [B:0] add_cy_r;
38
39 //Sign-extended operands
40 wire rs1_sx = i_rs1[B] & i_cmp_sig;
41 wire op_b_sx = i_op_b[B] & i_cmp_sig;
42
43 wire [B:0] add_b = i_op_b^W{i_sub}};
44
45 assign {add_cy,result_add} = i_rs1+add_b+add_cy_r;
46
47 wire result_lt = rs1_sx + ~op_b_sx + add_cy;
48
49 wire result_eq = !(result_add & (cmp_r | i_cnt0));
50
51 assign o_cmp = i_cmp_eq ? result_eq : result_lt;
52
53 /*
54 The result_bool expression implements the following opera-
55 tions between
56 i_rs1 and i_op_b depending on the value of i_bool_op
57 00 xor
58 01 0
59 10 or
60 11 and
61
62 i_bool_op will be 01 during shift operations, so by out-
63 putting zero under
64 this condition we can safely or result_bool with i_buf
65 */
66 wire [B:0] result_bool = ((i_rs1 ^ i_op_b) &
67 ~W{i_bool_op[0]}}) | (W{i_bool_op[1]}} & i_op_b & i_rs1);
68
69 assign result_slt[0] = cmp_r & i_cnt0;
70 generate
71 if (W>1) begin : gen_w_gt_1
72 assign result_slt[B:1] = {B{1'b0}};
73 end
74 endgenerate
75
76 assign o_rd = i_buf |
77 ({W{i_rd_sel[0]}} & result_add) |
78 ({W{i_rd_sel[1]}} & result_slt) |
79 ({W{i_rd_sel[2]}} & result_bool);
80
81 always @(posedge clk) begin
82 add_cy_r <= W{1'b0}};
83 add_cy_r[0] <= i_en ? add_cy : i_sub;
84
85 if (i_en)
86 cmp_r <= o_cmp;
87 end
88 endmodule

```

3. olofk_serv/rtl/serv_bufreg.v (101 lines)

```

1 /*
2 * serv_bufreg.v : SERV buffer register for load/store address and
3 shift data
4 *
5 * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
6 ning.me>
7 *
8 * SPDX-License-Identifier: ISC
9 */
10 module serv_bufreg #(
11     parameter [0:0] MDU = 0,
12     parameter W = 1,
13     parameter B = W-1
14 )
15 input wire    i_clk,
16 //State
17 input wire    i_cnt0,
18 input wire    i_cnt1,
19 input wire    i_cnt_done,
20 input wire    i_en,
21 input wire    i_init,
22 input wire    i_mdu_op,

```

```

23 output wire [1:0] o_lsb,
24 //Control
25 input wire    i_rs1_en,
26 input wire    i_imm_en,
27 input wire    i_clr_lsb,
28 input wire    i_shift_op,
29 input wire    i_right_shift_op,
30 input wire [2:0] i_shamt,
31 input wire    i_sh_signed,
32 //Data
33 input wire [B:0] i_rs1,
34 input wire [B:0] i_imm,
35 output wire [B:0] o_q,
36 //External
37 output wire [31:0] o_dbus_adr,
38 //Extension
39 output wire [31:0] o_ext_rs1;
40
41 wire    c;
42 wire [B:0] q;
43 reg [B:0] c_r;
44 reg [31:0] data;
45 wire [B:0] clr_lsb;
46
47 assign clr_lsb[0] = i_cnt0 & i_clr_lsb;
48
49 generate
50 if (W > 1) begin : gen_clr_lsb_w_gt_1
51 assign clr_lsb[B:1] = {B{1'b0}};
52 end
53 endgenerate
54
55 assign {c,q} = {1'b0,(i_rs1 & W{i_rs1_en}}) + {1'b0,
56 (i_imm & W{i_imm_en}} & ~clr_lsb} + c_r;
57
58 always @(posedge i_clk) begin
59 //Make sure carry is cleared before loading new data
60 c_r <= W{1'b0}};
61 c_r[0] <= c & i_en;
62 end
63
64 generate
65 if (W == 1) begin : gen_w_eq_1
66 always @(posedge i_clk) begin
67 if (i_en)
68 data[31:2] <= {i_init ? q : W{data[31] &
69 i_sh_signed}}, data[31:3];
70
71 if (i_init ? (i_cnt0 | i_cnt1) : i_en)
72 data[1:0] <= {i_init ? q : data[2], data[1]};
73 end
74
75 assign o_lsb = (MDU & i_mdu_op) ? 2'b00 : data[1:0];
76 assign o_q = data[0] & W{i_en};
77 end else if (W == 4) begin : gen_lsb_w_4
78 reg [1:0] lsb;
79 reg [W-2:0] data_tail;
80
81 wire [2:0] shift_amount
82 = !i_shift_op ? 3'd3 :
83 i_right_shift_op ? (3'd3+{1'b0,i_shamt[1:0]}) :
84 ({1'b0,~i_shamt[1:0]});
85
86 always @(posedge i_clk) begin
87 if (i_en)
88 if (i_cnt0) lsb <= q[1:0];
89 if (i_en)
90 data <= {i_init ? q : W{i_sh_signed &
91 data[31]}} , data[31:W];
92 if (i_en)
93 data_tail <= data[B:1] & {B{~i_cnt_done}};
94 end
95
96 wire [2*W+B-2:0] muxdata = {data[W+B-1:0],data_tail};
97 wire [B:0] muxout = muxdata[{1'b0,shift_amount}
98 +:W];
99
100 assign o_lsb = (MDU & i_mdu_op) ? 2'b00 : lsb;
101 assign o_q = i_en ? muxout : W{1'b0}};
102 end
103 endgenerate
104
105 assign o_dbus_adr = {data[31:2], 2'b00};
106 assign o_ext_rs1 = data;
107 endmodule

```

4. olofk_serv/rtl/serv_bufreg2.v (105 lines)

```

1 /*
2 * serv_bufreg2.v : SERV buffer register for load/store data
3 and shift amount
4 *
5 * SPDX-FileCopyrightText: 2022 Olof Kindgren <olof@award-win-
6 ning.me>
7 *
8 * SPDX-License-Identifier: ISC
9 */
10 module serv_bufreg2
11     #(parameter W = 1,
12       //Internally calculated. Do not touch

```

```

10   parameter B=W-1)
11   (
12   input wire      i_clk,
13   //State
14   input wire      i_en,
15   input wire      i_init,
16   input wire      i_cnt7,
17   input wire      i_cnt_done,
18   input wire      i_sh_right,
19   input wire [1:0] i_lsb,
20   input wire [1:0] i_bytecnt,
21   output wire     o_sh_done,
22   //Control
23   input wire      i_op_b_sel,
24   input wire      i_shift_op,
25   //Data
26   input wire [B:0] i_rs2,
27   input wire [B:0] i_imm,
28   output wire [B:0] o_op_b,
29   output wire [B:0] o_q,
30   //External
31   output wire [31:0] o_dat,
32   input wire      i_load,
33   input wire [31:0] i_dat);
34
35   // High and low data words form a 32-bit word
36   reg [7:0] dhi;
37   reg [23:0] dlo;
38
39   /*
40    Before a store operation, the data to be written needs to
41    be shifted into place. Depending on the address alignment, we need to
42    shift different amounts. One formula for calculating this is to say that
43    we shift when i_lsb + i_bytecnt < 4. Unfortunately, the synthesis tools
44    don't seem to be clever enough so the hideous expression below is used to
45    achieve the same thing in a more optimal way.
46    */
47   wire byte_valid
48   = (!i_lsb[0] & !i_lsb[1]) |
49     (!i_bytecnt[0] & !i_bytecnt[1]) |
50     (!i_bytecnt[1] & !i_lsb[1]) |
51     (!i_bytecnt[1] & !i_lsb[0]) |
52     (!i_bytecnt[0] & !i_lsb[1]);
53
54   assign o_op_b = i_op_b_sel ? i_rs2 : i_imm;
55
56   wire shift_en = i_shift_op ? (i_en & i_init &
57   (i_bytecnt == 2'b00)) : (i_en & byte_valid);
58   wire cnt_en = (i_shift_op & (!i_init | (i_cnt_done
59   & i_sh_right)));
60   /* The dat register has three different use cases for
61   store, load and shift operations.
62   store : Data to be written is shifted to the correct posi-
63   tion in dat during init by shift_en and is presented on the data bus
64   as o_wb_dat
65   load : Data from the bus gets latched into dat during
66   i_wb_ack and is then shifted out at the appropriate time to end up in
67   the correct position in rd
68   shift : Data is shifted in during init. After that, the
69   six LSB are used as a downcounter (with bit 5 initially set to 0) that
70   trigger
71   o_sh_done when they wrap around to indicate that
72   the requested number of shifts have been performed
73   */
74   wire [7:0] cnt_next;
75   generate
76   if (W == 1) begin : gen_cnt_w_eq_1
77     assign cnt_next = {o_op_b, dhi[7], dhi[5:0]-6'd1};
78   end else if (W == 4) begin : gen_cnt_w_eq_4
79     assign cnt_next = {o_op_b[3:2], dhi[5:0]-6'd4};
80   end
81   endgenerate
82   wire [7:0] dat_shamt = cnt_en ?
83   //Down counter mode
84   cnt_next :
85   //Shift reg mode
86   {o_op_b, dhi[7:W]};
87
88   assign o_sh_done = dat_shamt[5];
89
90   assign o_q =
91   ({W{(!i_lsb == 2'd3)}} & o_dat[W+23:24]) |
92   ({W{(!i_lsb == 2'd2)}} & o_dat[W+15:16]) |
93   ({W{(!i_lsb == 2'd1)}} & o_dat[W+7:8]) |
94   ({W{(!i_lsb == 2'd0)}} & o_dat[W-1:0]);
95

```

```

96   assign o_dat = {dhi,dlo};
97
98   always @(posedge i_clk) begin
99     if (shift_en | cnt_en | i_load)
100       dhi <= i_load ? i_dat[31:24] : dat_shamt & {2'b11, !
101       (i_shift_op & i_cnt7 & !cnt_en), 5'b11111};
102     if (shift_en | i_load)
103       dlo <= i_load ? i_dat[23:0] : {dhi[B:0], dlo[23:W]};
104   end
105 endmodule

```

5. olofk_serv/rtl/serv_compdec.v (232 lines)

```

1  /* Copyright lowRISC contributors.
2  Copyright 2018 ETH Zurich and University of Bologna, see also
3  CREDITS.md.
4  Licensed under the Apache License, Version 2.0, see LICENSE
5  for details.
6  SPDX-License-Identifier: Apache-2.0
7
8  * Adapted to SERV by @Abdulwadoodd as part of the project un-
9  der spring '22 LFX Mentorship program */
10
11 /* Decodes RISC-V compressed instructions into their RV32i
12 equivalent. */
13
14 module serv_compdec
15 (
16   input wire i_clk,
17   input wire [31:0] i_instr,
18   input wire i_ack,
19   output wire [31:0] o_instr,
20   output reg o_iscomp);
21
22 localparam OPCODE_LOAD = 7'h03;
23 localparam OPCODE_OP_IMM = 7'h13;
24 localparam OPCODE_STORE = 7'h23;
25 localparam OPCODE_OP = 7'h33;
26 localparam OPCODE_LUI = 7'h37;
27 localparam OPCODE_BRANCH = 7'h63;
28 localparam OPCODE_JALR = 7'h67;
29 localparam OPCODE_JAL = 7'h6f;
30
31 reg [31:0] comp_instr;
32 reg illegal_instr;
33
34 assign o_instr = illegal_instr ? i_instr : comp_instr;
35
36 always @(posedge i_clk) begin
37   if(i_ack)
38     o_iscomp <= !illegal_instr;
39 end
40
41 always @ (*) begin
42   // By default, forward incoming instruction, mark it as
43   legal.
44   comp_instr = i_instr;
45   illegal_instr = 1'b0;
46
47   // Check if incoming instruction is compressed.
48   case (i_instr[1:0])
49     // C0
50     2'b00: begin
51       case (i_instr[15:14])
52         2'b00: begin
53           // c.addi4spn -> addi rd', x2, imm
54           comp_instr = {2'b0, i_instr[10:7], i_instr[12:11],
55           i_instr[5],
56           i_instr[6], 2'b00, 5'h02, 3'b000, 2'b01,
57           i_instr[4:2], {OPCODE_OP_IMM}};
58         end
59       2'b01: begin
60         // c.lw -> lw rd', imm(rs1')
61         comp_instr = {5'b0, i_instr[5], i_instr[12:10],
62           i_instr[6],
63           2'b00, 2'b01, i_instr[9:7], 3'b010,
64           2'b01, i_instr[4:2], {OPCODE_LOAD}};
65       end
66       2'b11: begin
67         // c.sw -> sw rs2', imm(rs1')
68         comp_instr = {5'b0, i_instr[5], i_instr[12],
69           2'b01, i_instr[4:2],
70           2'b01, i_instr[9:7], 3'b010, i_in-
71           str[11:10], i_instr[6],
72           2'b00, {OPCODE_STORE}};
73       end
74       2'b10: begin
75         illegal_instr = 1'b1;
76       end
77     endcase
78   end
79   // C1

```

```

74
75 // Register address checks for RV32E are performed in
the regular instruction decoder.
76 // If this check fails, an illegal instruction exception is
triggered and the controller
77 // writes the actual faulting instruction to mtval.
78 2'b01: begin
79   case (i_instr[15:13])
80     3'b000: begin
81       // c.addi -> addi rd, rd, nzimm
82       // c.nop
83       comp_instr = {{6 {i_instr[12]}}, i_instr[12],
i_instr[6:2],
84         i_instr[11:7], 3'b0, i_instr[11:7], {OP-
CODE_OP_IMM}};
85     end
86   3'b001, 3'b101: begin
87     // 001: c.jal -> jal x1, imm
88     // 101: c.j -> jal x0, imm
89     comp_instr = {i_instr[12], i_instr[8], i_in-
str[10:9], i_instr[6],
90       i_instr[7], i_instr[2], i_instr[11],
i_instr[5:3],
91       {9 {i_instr[12]}}, 4'b0, ~i_instr[15],
{OPCODE_JAL}};
92   end
93   3'b010: begin
94     // c.li -> addi rd, x0, nzimm
95     // (c.li hints are translated into an addi hint)
96     comp_instr = {{6 {i_instr[12]}}, i_instr[12],
i_instr[6:2], 5'b0,
97       3'b0, i_instr[11:7], {OPCODE_OP_IMM}};
98   end
99   3'b011: begin
100    // c.lui -> lui rd, imm
101    // (c.lui hints are translated into a lui hint)
102    comp_instr = {{15 {i_instr[12]}}, i_instr[6:2],
i_instr[11:7], {OPCODE_LUI}};
103    if (i_instr[11:7] == 5'h02) begin
104      // c.addi16sp -> addi x2, x2, nzimm
105      comp_instr = {{3 {i_instr[12]}}, i_instr[4:3],
i_instr[5], i_instr[2],
106        i_instr[6], 4'b0, 5'h02, 3'b000,
5'h02, {OPCODE_OP_IMM}};
107    end
108    end
109    3'b100: begin
110      case (i_instr[11:10])
111        2'b00,
112        2'b01: begin
113          // 00: c.srli -> srli rd, rd, shamt
114          // 01: c.srai -> srai rd, rd, shamt
115          // (c.srli/c.srai hints are translated into a
srli/srai hint)
116          comp_instr = {1'b0, i_instr[10], 5'b0, i_in-
str[6:2], 2'b01, i_instr[9:7],
117            3'b101, 2'b01, i_instr[9:7], {OP-
CODE_OP_IMM}};
118        end
119        2'b10: begin
120          // c.andi -> andi rd, rd, imm
121          comp_instr = {{6 {i_instr[12]}}, i_instr[12],
i_instr[6:2], 2'b01, i_instr[9:7],
122            3'b111, 2'b01, i_instr[9:7], {OP-
CODE_OP_IMM}};
123        end
124        2'b11: begin
125          case (i_instr[6:5])
126            2'b00: begin
127              // c.sub -> sub rd', rd', rs2'
128              comp_instr = {2'b01, 5'b0, 2'b01, i_in-
str[4:2], 2'b01, i_instr[9:7],
129                3'b000, 2'b01, i_instr[9:7],
{OPCODE_OP}};
130            end
131            2'b01: begin
132              // c.xor -> xor rd', rd', rs2'
133              comp_instr = {7'b0, 2'b01, i_instr[4:2],
2'b01, i_instr[9:7], 3'b100,
134                2'b01, i_instr[9:7], {OP-
CODE_OP}};
135            end
136            2'b10: begin
137              // c.or -> or rd', rd', rs2'
138              comp_instr = {7'b0, 2'b01, i_instr[4:2],
2'b01, i_instr[9:7], 3'b110,
139                2'b01, i_instr[9:7], {OP-
CODE_OP}};
140            end
141            2'b11: begin
142              // c.and -> and rd', rd', rs2'
143              comp_instr = {7'b0, 2'b01, i_instr[4:2],
2'b01, i_instr[9:7], 3'b111,
144                2'b01, i_instr[9:7], {OP-
CODE_OP}};
145            end
146          end
147        end
148      end
149    end
150  end

```

```

151
152 2'b11: begin
153   // c.and -> and rd', rd', rs2'
154   comp_instr = {7'b0, 2'b01, i_instr[4:2],
2'b01, i_instr[9:7], 3'b111,
155     2'b01, i_instr[9:7], {OP-
CODE_OP}};
156   end
157   endcase
158   end
159   endcase
160   end
161   3'b110, 3'b111: begin
162     // 0: c.beqz -> beq rs1', x0, imm
163     // 1: c.bnez -> bne rs1', x0, imm
164     comp_instr = {{4 {i_instr[12]}}, i_instr[6:5],
i_instr[2], 5'b0, 2'b01,
165       i_instr[9:7], 2'b00, i_instr[13], i_in-
str[11:10], i_instr[4:3],
166       i_instr[12], {OPCODE_BRANCH}};
167   end
168   endcase
169   end
170   // C2
171   // Register address checks for RV32E are performed in
the regular instruction decoder.
172
173 // If this check fails, an illegal instruction exception
is triggered and the controller
174 // writes the actual faulting instruction to mtval.
175 2'b10: begin
176   case (i_instr[15:14])
177     2'b00: begin
178       // c.slli -> slli rd, rd, shamt
179       // (c.slli hints are translated into a slli hint)
180       comp_instr = {7'b0, i_instr[6:2], i_instr[11:7],
3'b001, i_instr[11:7], {OPCODE_OP_IMM}};
181     end
182     2'b01: begin
183       // c.lwsp -> lw rd, imm(x2)
184       comp_instr = {4'b0, i_instr[3:2], i_instr[12],
i_instr[6:4], 2'b00, 5'h02,
185         3'b010, i_instr[11:7], OPCODE_LOAD};
186     end
187     2'b10: begin
188       if (i_instr[12] == 1'b0) begin
189         if (i_instr[6:2] != 5'b0) begin
190           // c.mv -> add rd/rs1, x0, rs2
191           // (c.mv hints are translated into an add
hint)
192           comp_instr = {7'b0, i_instr[6:2], 5'b0, 3'b0,
i_instr[11:7], {OPCODE_OP}};
193         end else begin
194           // c.jr -> jalr x0, rd/rs1, 0
195           comp_instr = {12'b0, i_instr[11:7], 3'b0,
5'b0, {OPCODE_JALR}};
196         end
197       end else begin
198         if (i_instr[6:2] != 5'b0) begin
199           // c.add -> add rd, rd, rs2
200           // (c.add hints are translated into an add
hint)
201           comp_instr = {7'b0, i_instr[6:2], i_in-
str[11:7], 3'b0, i_instr[11:7], {OPCODE_OP}};
202         end else begin
203           if (i_instr[11:7] == 5'b0) begin
204             // c.ebreak -> ebreak
205             comp_instr = {32'h00_10_00_73};
206           end else begin
207             // c.jalr -> jalr x1, rs1, 0
208             comp_instr = {12'b0, i_instr[11:7], 3'b000,
5'b00001, {OPCODE_JALR}};
209           end
210         end
211       end
212     end
213   end
214   end
215   end
216   end
217   2'b11: begin
218     // c.swsp -> sw rs2, imm(x2)
219     comp_instr = {4'b0, i_instr[8:7], i_instr[12],
i_instr[6:2], 5'h02, 3'b010,
220       i_instr[11:9], 2'b00, {OPCODE_STORE}};
221   end
222   endcase
223   end
224   // Incoming instruction is not compressed.
225   2'b11: illegal_instr = 1'b1;
226   endcase
227   end
228   endmodule

```

6. olofk_serv/rtl/serv_csr.v (164 lines)

```

1 /*
2  * serv_csr.v : SERV module for handling CSR registers
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
5  * ning.me>
6  * SPDX-License-Identifier: ISC
7  */
8 `default_nettype none
9 module serv_csr
10     #(
11         parameter RESET_STRATEGY = "MINI",
12         parameter W = 1,
13         parameter B = W-1
14     )
15     (
16         input wire i_clk,
17         input wire i_rst,
18         //State
19         input wire i_trig_irq,
20         input wire i_en,
21         input wire i_cnt0to3,
22         input wire i_cnt3,
23         input wire i_cnt7,
24         input wire i_cnt11,
25         input wire i_cnt12,
26         input wire i_cnt_done,
27         input wire i_mem_op,
28         input wire i_mtip,
29         input wire i_trap,
30         output reg o_new_irq,
31         //Control
32         input wire i_e_op,
33         input wire i_ebreak,
34         input wire i_mem_cmd,
35         input wire i_mstatus_en,
36         input wire i_mie_en,
37         input wire i_mcause_en,
38         input wire [1:0] i_csr_source,
39         input wire i_mret,
40         input wire i_csr_d_sel,
41         //Data
42         input wire [B:0] i_rf_csr_out,
43         output wire [B:0] o_csr_in,
44         input wire [B:0] i_csr_imm,
45         input wire [B:0] i_rst1,
46         output wire [B:0] o_q;
47
48         localparam [1:0]
49             CSR_SOURCE_CSR = 2'b00,
50             CSR_SOURCE_EXT = 2'b01,
51             CSR_SOURCE_SET = 2'b10,
52             CSR_SOURCE_CLR = 2'b11;
53
54         reg mstatus_mie;
55         reg mstatus_mpie;
56         reg mie_mtie;
57
58         reg mcause31;
59         reg [3:0] mcause3_0;
60         wire [B:0] mcause;
61
62         wire [B:0] csr_in;
63         wire [B:0] csr_out;
64
65         reg timer_irq_r;
66
67         wire [B:0] d = i_csr_d_sel ? i_csr_imm : i_rst1;
68         assign csr_in = (i_csr_source == CSR_SOURCE_EXT) ? d :
69
70             (i_csr_source == CSR_SOURCE_SET) ? csr_out |
71
72             (i_csr_source == CSR_SOURCE_CLR) ? csr_out &
73
74             (i_csr_source == CSR_SOURCE_CSR) ? csr_out :
75             {W{1'bx}};
76
77         wire [B:0] mstatus;
78
79         generate
80             if (W==1) begin : gen_mstatus_w1
81                 assign mstatus = ((mstatus_mie & i_cnt3) | (i_cnt11 |
82 i_cnt12));
83             end else if (W==4) begin : gen_mstatus_w4
84                 assign mstatus = {i_cnt11 | (mstatus_mie & i_cnt3),
85 2'b00, i_cnt12};
86             end
87         endgenerate
88
89         assign csr_out = ({W{i_mstatus_en & i_en}} & mstatus) |
90             i_rf_csr_out |
91             ({W{i_mcause_en & i_en}} & mcause);
92
93         assign o_q = csr_out;
94
95         wire timer_irq = i_mtip & mstatus_mie & mie_mtie;
96
97         assign mcause = i_cnt0to3 ? mcause3_0[B:0] : {[3:0]
98             i_cnt_done ? {mcause31, {B{1'b0}}} : {[31]
99             : {W{1'b0}}};
100
101         assign o_csr_in = csr_in;
102
103         always @(posedge i_clk) begin
104             if (i_trig_irq) begin
105                 timer_irq_r <= timer_irq;
106                 o_new_irq <= timer_irq & !timer_irq_r;
107             end
108
109             if (i_mie_en & i_cnt7)
110                 mie_mtie <= csr_in[B];
111
112             /*
113              * The mie bit in mstatus gets updated under three condi-
114              tions
115              *
116              * When a trap is taken, the bit is cleared
117              * During an mret instruction, the bit is restored from
118              mpie
119              * During a mstatus CSR access instruction it's assigned
120              when
121              * bit 3 gets updated
122              *
123              * These conditions are all mutually exclusive
124              */
125
126             if ((i_trap & i_cnt_done) | i_mstatus_en & i_cnt3 & i_en |
127 i_mret)
128                 mstatus_mie <= !i_trap & (i_mret ? mstatus_mpie :
129 csr_in[B]);
130
131             /*
132              * Note: To save resources mstatus_mpie (mstatus bit 7) is
133              not
134              * readable or writable from sw
135              */
136             if (i_trap & i_cnt_done)
137                 mstatus_mpie <= mstatus_mie;
138
139             /*
140              * The four lowest bits in mcause hold the exception code
141              *
142              * These bits get updated under three conditions
143              *
144              * During an mcause CSR access function, they are assigned
145              when
146              * bits 0 to 3 gets updated
147              *
148              * During an external interrupt the exception code is set
149              to
150              * 7, since SERV only support timer interrupts
151              *
152              * During an exception, the exception code is assigned to
153              indicate
154              * if it was caused by an ebreak instruction (3),
155              *
156              * ecall instruction (11), misaligned load (4), misaligned
157              store (6)
158              * or misaligned jump (0)
159              *
160              * The expressions below are derived from the following
161              truth table
162              *
163              * irq => 0111 (timer=7)
164              * e_op => x011 (ebreak=3, ecall=11)
165              * mem => 01x0 (store=6, load=4)
166              * ctrl => 0000 (jump=0)
167              */
168             if (i_mcause_en & i_en & i_cnt0to3 | (i_trap &
169 i_cnt_done)) begin
170                 mcause3_0[3] <= (i_e_op & !i_ebreak) | (!i_trap &
171 csr_in[B]);
172                 mcause3_0[2] <= o_new_irq | i_mem_op | (!i_trap & ((W
173 == 1) ? mcause3_0[3] : csr_in[(W == 1) ? 0 : 2]));
174                 mcause3_0[1] <= o_new_irq | i_e_op | (i_mem_op &
175 i_mem_cmd) | (!i_trap & ((W == 1) ? mcause3_0[2] : csr_in[(W == 1) ?
176 0 : 1]));
177                 mcause3_0[0] <= o_new_irq | i_e_op | (!i_trap & ((W ==
178 1) ? mcause3_0[1] : csr_in[0]));
179             end
180
181             if (i_mcause_en & i_cnt_done | i_trap)
182                 mcause31 <= i_trap ? o_new_irq : csr_in[B];
183
184             if (i_rst)
185                 if (RESET_STRATEGY != "NONE") begin
186                     o_new_irq <= 1'b0;
187                     mie_mtie <= 1'b0;
188                 end
189             end
190
191         end
192     endmodule

```

7. olofk_serv/rtl/serv_ctrl.v (115 lines)

```

1 /*
2  * serv_ctrl.v : SERV module for updating program counter
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
5  *   ning.me>
6  * SPDX-License-Identifier: ISC
7  */
8 `default_nettype none
9 module serv_ctrl
10   #(parameter RESET_STRATEGY = "MINI",
11     parameter RESET_PC = 32'd0,
12     parameter WITH_CSR = 1,
13     parameter W = 1,
14     parameter B = W-1
15   )
16   (
17     input wire      clk,
18     input wire      i_rst,
19     //State
20     input wire      i_pc_en,
21     input wire      i_cnt12to31,
22     input wire      i_cnt0,
23     input wire      i_cnt1,
24     input wire      i_cnt2,
25     //Control
26     input wire      i_jump,
27     input wire      i_jal_or_jalr,
28     input wire      i_ctype,
29     input wire      i_pc_rel,
30     input wire      i_trap,
31     input wire      i_iscomp,
32     //Data
33     input wire [B:0] i_imm,
34     input wire [B:0] i_buf,
35     input wire [B:0] i_csr_pc,
36     output wire [B:0] o_rd,
37     output wire [B:0] o_bad_pc,
38     //External
39     output reg [31:0] o_ibus_adr;
40
41     wire [B:0] pc_plus_4;
42     wire      pc_plus_4_cy;
43     reg       pc_plus_4_cy_r;
44     wire [B:0] pc_plus_4_cy_r_w;
45     wire [B:0] pc_plus_offset;
46     wire      pc_plus_offset_cy;
47     reg       pc_plus_offset_cy_r;
48     wire [B:0] pc_plus_offset_cy_r_w;
49     wire [B:0] pc_plus_offset_aligned;
50     wire [B:0] plus_4;
51
52     wire [B:0] pc = o_ibus_adr[B:0];
53
54     wire [B:0] new_pc;
55
56     wire [B:0] offset_a;
57     wire [B:0] offset_b;
58
59     /* If i_iscomp=1: increment pc by 2 else increment pc by 4 */
60     generate
61       if (W == 1) begin : gen_plus_4_w_eq_1
62         assign plus_4 = i_iscomp ? i_cnt1 : i_cnt2;
63       end else if (W == 4) begin : gen_plus_4_w_eq_4
64         assign plus_4 = (i_cnt0 | i_cnt1) ? (i_iscomp ? 2 :
65         4) : 0;
66       end
67     endgenerate
68     assign o_bad_pc = pc_plus_offset_aligned;
69
70     assign {pc_plus_4_cy, pc_plus_4} =
71     pc_plus_4 + pc_plus_4_cy_r_w;
72
73     generate
74       if (!WITH_CSR) begin : gen_csr
75         if (W == 1) begin : gen_new_pc_w_eq_1
76           assign new_pc = i_trap ? (i_csr_pc & !(i_cnt0 ||
77           i_cnt1)) : i_jump ? pc_plus_offset_aligned : pc_plus_4;
78         end else if (W == 4) begin : gen_new_pc_w_eq_4
79           assign new_pc = i_trap ? (i_csr_pc & ((i_cnt0 ||
80           i_cnt1) ? 4'b1100 : 4'b1111)) : i_jump ? pc_plus_offset_aligned :
81           pc_plus_4;
82         end
83       end else begin : gen_no_csr
84         assign new_pc = i_jump ? pc_plus_offset_aligned :
85         pc_plus_4;
86       end
87     endgenerate
88     assign o_rd = ({i_ctype} & pc_plus_offset_aligned) |
89     (pc_plus_4 & {i_jal_or_jalr});
90
91     assign offset_a = {i_ctype} & pc;
92     assign offset_b = i_ctype ? i_imm & {i_cnt12to31} :
93     i_buf;
94     assign {pc_plus_offset_cy, pc_plus_offset} = off-

```

```

set_a + offset_b + pc_plus_offset_cy_r_w;
88
89   generate
90     if (W > 1) begin : gen_w_gt_1
91       assign pc_plus_offset_aligned[B:1] = pc_plus_offset[B:
92       1];
93       assign pc_plus_offset_cy_r_w[B:1] = {B{1'b0}};
94       assign pc_plus_4_cy_r_w[B:1] = {B{1'b0}};
95     end
96   endgenerate
97   assign pc_plus_offset_aligned[0] = pc_plus_offset[0] & !
98   i_cnt0;
99   assign pc_plus_offset_cy_r_w[0] = pc_plus_offset_cy_r;
100  assign pc_plus_4_cy_r_w[0] = pc_plus_4_cy_r;
101
102  initial if (RESET_STRATEGY == "NONE") o_ibus_adr = RE-
103  SET_PC;
104
105  always @(posedge clk) begin
106    pc_plus_4_cy_r <= i_pc_en & pc_plus_4_cy;
107    pc_plus_offset_cy_r <= i_pc_en & pc_plus_offset_cy;
108
109    if (RESET_STRATEGY == "NONE") begin
110      if (i_pc_en)
111        o_ibus_adr <= {new_pc, o_ibus_adr[31:W]};
112    end else begin
113      if (i_pc_en | i_rst)
114        o_ibus_adr <= i_rst ? RESET_PC : {new_pc,
115        o_ibus_adr[31:W]};
116    end
117  end
118 endmodule

```

8. olofk_serv/rtl/serv_debug.v (351 lines)

```

1 /*
2  * serv_debug.v : SERV module for introspecting CPU/RF state
3  *   during simulations
4  *
5  * SPDX-FileCopyrightText: 2024 Olof Kindgren <olof@award-win-
6  *   ning.me>
7  * SPDX-License-Identifier: ISC
8  */
9 module serv_debug
10   #(parameter W = 1,
11     parameter RESET_PC = 0,
12     //Internally calculated. Do not touch
13     parameter B=W-1
14   )
15   `ifdef RISC_V_FORMAL
16     output reg      rvfi_valid = 1'b0,
17     output reg [63:0] rvfi_order = 64'd0,
18     output reg [31:0] rvfi_insn = 32'd0,
19     output reg      rvfi_trap = 1'b0,
20     output reg      rvfi_halt = 1'b0, // Not used
21     output reg      rvfi_intr = 1'b0, // Not used
22     output reg [1:0] rvfi_mode = 2'b11, // Not used
23     output reg [1:0] rvfi_ixl = 2'b01, // Not used
24     output reg [4:0] rvfi_rs1_addr,
25     output reg [4:0] rvfi_rs2_addr,
26     output reg [31:0] rvfi_rs1_rdata,
27     output reg [31:0] rvfi_rs2_rdata,
28     output reg [4:0] rvfi_rd_addr,
29     output reg [31:0] rvfi_rd_wdata,
30     output reg [31:0] rvfi_pc_rdata,
31     output reg [31:0] rvfi_pc_wdata,
32     output reg [31:0] rvfi_mem_addr,
33     output reg [3:0] rvfi_mem_rmask,
34     output reg [3:0] rvfi_mem_wmask,
35     output reg [31:0] rvfi_mem_rdata,
36     output reg [31:0] rvfi_mem_wdata,
37     input wire [31:0] i_dbus_adr,
38     input wire [31:0] i_dbus_dat,
39     input wire [3:0] i_dbus_sel,
40     input wire      i_dbus_we,
41     input wire [31:0] i_dbus_rdt,
42     input wire      i_dbus_ack,
43     input wire      i_ctrl_pc_en,
44     input wire [B:0] rs1,
45     input wire [B:0] rs2,
46     input wire [4:0] rs1_addr,
47     input wire [4:0] rs2_addr,
48     input wire [3:0] immedc_en,
49     input wire      rd_en,
50     input wire      trap,
51     input wire      i_rf_ready,
52     input wire      i_ibus_cyc,
53     input wire      two_stage_op,
54     input wire      init,
55     input wire [31:0] i_ibus_adr,
56   `endif
57   (
58     input wire      i_clk,
59     input wire      i_rst,
60     input wire [31:0] i_ibus_rdt,
61     input wire      i_ibus_ack,
62     input wire [4:0] i_rd_addr,
63     input wire      i_cnt_en,
64     input wire [B:0] i_csr_in,
65     input wire      i_csr_mstatus_en,

```



```

63  input wire      i_csr_mie_en,
64  input wire      i_csr_mcause_en,
65  input wire      i_csr_en,
66  input wire [1:0] i_csr_addr,
67  input wire      i_wen0,
68  input wire [B:0] i_wdata0,
69  input wire      i_cnt_done);
70
71  reg              update_rd = 1'b0;
72  reg              update_mscratch;
73  reg              update_mtvec;
74  reg              update_mepc;
75  reg              update_mtval;
76  reg              update_mstatus;
77  reg              update_mie;
78  reg              update_mcause;
79
80  reg [31:0]       dbg_rd = 32'hxxxxxxxx;
81  reg [31:0]       dbg_csr = 32'hxxxxxxxx;
82  reg [31:0]       dbg_mstatus = 32'hxxxxxxxx;
83  reg [31:0]       dbg_mie = 32'hxxxxxxxx;
84  reg [31:0]       dbg_mcause = 32'hxxxxxxxx;
85  reg [31:0]       dbg_mscratch = 32'hxxxxxxxx;
86  reg [31:0]       dbg_mtvec = 32'hxxxxxxxx;
87  reg [31:0]       dbg_mepc = 32'hxxxxxxxx;
88  reg [31:0]       dbg_mtval = 32'hxxxxxxxx;
89  reg [31:0]       x1 = 32'hxxxxxxxx;
90  reg [31:0]       x2 = 32'hxxxxxxxx;
91  reg [31:0]       x3 = 32'hxxxxxxxx;
92  reg [31:0]       x4 = 32'hxxxxxxxx;
93  reg [31:0]       x5 = 32'hxxxxxxxx;
94  reg [31:0]       x6 = 32'hxxxxxxxx;
95  reg [31:0]       x7 = 32'hxxxxxxxx;
96  reg [31:0]       x8 = 32'hxxxxxxxx;
97  reg [31:0]       x9 = 32'hxxxxxxxx;
98  reg [31:0]       x10 = 32'hxxxxxxxx;
99  reg [31:0]       x11 = 32'hxxxxxxxx;
100 reg [31:0]       x12 = 32'hxxxxxxxx;
101 reg [31:0]       x13 = 32'hxxxxxxxx;
102 reg [31:0]       x14 = 32'hxxxxxxxx;
103 reg [31:0]       x15 = 32'hxxxxxxxx;
104 reg [31:0]       x16 = 32'hxxxxxxxx;
105 reg [31:0]       x17 = 32'hxxxxxxxx;
106 reg [31:0]       x18 = 32'hxxxxxxxx;
107 reg [31:0]       x19 = 32'hxxxxxxxx;
108 reg [31:0]       x20 = 32'hxxxxxxxx;
109 reg [31:0]       x21 = 32'hxxxxxxxx;
110 reg [31:0]       x22 = 32'hxxxxxxxx;
111 reg [31:0]       x23 = 32'hxxxxxxxx;
112 reg [31:0]       x24 = 32'hxxxxxxxx;
113 reg [31:0]       x25 = 32'hxxxxxxxx;
114 reg [31:0]       x26 = 32'hxxxxxxxx;
115 reg [31:0]       x27 = 32'hxxxxxxxx;
116 reg [31:0]       x28 = 32'hxxxxxxxx;
117 reg [31:0]       x29 = 32'hxxxxxxxx;
118 reg [31:0]       x30 = 32'hxxxxxxxx;
119 reg [31:0]       x31 = 32'hxxxxxxxx;
120
121 always @(posedge i_clk) begin
122   update_rd <= i_cnt_done & i_wen0;
123
124   if (i_wen0)
125     dbg_rd <= {i_wdata0, dbg_rd[31:W]};
126
127   //End of instruction that writes to RF
128   if (update_rd) begin
129     case (i_rd_addr)
130       5'd1 : x1 <= dbg_rd;
131       5'd2 : x2 <= dbg_rd;
132       5'd3 : x3 <= dbg_rd;
133       5'd4 : x4 <= dbg_rd;
134       5'd5 : x5 <= dbg_rd;
135       5'd6 : x6 <= dbg_rd;
136       5'd7 : x7 <= dbg_rd;
137       5'd8 : x8 <= dbg_rd;
138       5'd9 : x9 <= dbg_rd;
139       5'd10 : x10 <= dbg_rd;
140       5'd11 : x11 <= dbg_rd;
141       5'd12 : x12 <= dbg_rd;
142       5'd13 : x13 <= dbg_rd;
143       5'd14 : x14 <= dbg_rd;
144       5'd15 : x15 <= dbg_rd;
145       5'd16 : x16 <= dbg_rd;
146       5'd17 : x17 <= dbg_rd;
147       5'd18 : x18 <= dbg_rd;
148       5'd19 : x19 <= dbg_rd;
149       5'd20 : x20 <= dbg_rd;
150       5'd21 : x21 <= dbg_rd;
151       5'd22 : x22 <= dbg_rd;
152       5'd23 : x23 <= dbg_rd;
153       5'd24 : x24 <= dbg_rd;
154       5'd25 : x25 <= dbg_rd;
155       5'd26 : x26 <= dbg_rd;
156       5'd27 : x27 <= dbg_rd;
157       5'd28 : x28 <= dbg_rd;
158       5'd29 : x29 <= dbg_rd;
159       5'd30 : x30 <= dbg_rd;
160       5'd31 : x31 <= dbg_rd;
161       default : ;
162     endcase
163   end

```

```

164
165   update_mscratch <= i_cnt_done & i_csr_en & (i_csr_addr
== 2'b00);
166   update_mtvec <= i_cnt_done & i_csr_en & (i_csr_addr
== 2'b01);
167   update_mepc <= i_cnt_done & i_csr_en & (i_csr_addr
== 2'b10);
168   update_mtval <= i_cnt_done & i_csr_en & (i_csr_addr
== 2'b11);
169   update_mstatus <= i_cnt_done & i_csr_mstatus_en;
170   update_mie <= i_cnt_done & i_csr_mie_en;
171   update_mcause <= i_cnt_done & i_csr_mcause_en;
172
173   if (i_cnt_en)
174     dbg_csr <= {i_csr_in, dbg_csr[31:W]};
175
176   if (update_mscratch) dbg_mscratch <= dbg_csr;
177   if (update_mtvec) dbg_mtvec <= dbg_csr;
178   if (update_mepc) dbg_mepc <= dbg_csr;
179   if (update_mtval) dbg_mtval <= dbg_csr;
180   if (update_mstatus) dbg_mstatus <= dbg_csr;
181   if (update_mie) dbg_mie <= dbg_csr;
182   if (update_mcause) dbg_mcause <= dbg_csr;
183 end
184
185 reg LUI, AUIPC, JAL, JALR, BEQ, BNE, BLT, BGE, BLTU, BGEU,
LB, LH, LW, LBU, LHU, SB, SH, SW, ADDI, SLTI, SLTIU, XORI, ORI,
ANDI, SLLI, SRLI, SRAI, ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR,
AND, FENCE, ECALL, EBREAK;
186 reg CSRRW, CSRRS, CSRRC, CSRRWI, CSRRSI, CSRRCI;
187 reg OTHER;
188
189 always @(posedge i_clk) begin
190   if (i_ibus_ack) begin
191     LUI <= 1'b0;
192     AUIPC <= 1'b0;
193     JAL <= 1'b0;
194     JALR <= 1'b0;
195     BEQ <= 1'b0;
196     BNE <= 1'b0;
197     BLT <= 1'b0;
198     BGE <= 1'b0;
199     BLTU <= 1'b0;
200     BGEU <= 1'b0;
201     LB <= 1'b0;
202     LH <= 1'b0;
203     LW <= 1'b0;
204     LBU <= 1'b0;
205     LHU <= 1'b0;
206     SB <= 1'b0;
207     SH <= 1'b0;
208     SW <= 1'b0;
209     ADDI <= 1'b0;
210     SLTI <= 1'b0;
211     SLTIU <= 1'b0;
212     XORI <= 1'b0;
213     ORI <= 1'b0;
214     ANDI <= 1'b0;
215     SLLI <= 1'b0;
216     SRLI <= 1'b0;
217     SRAI <= 1'b0;
218     ADD <= 1'b0;
219     SUB <= 1'b0;
220     SLL <= 1'b0;
221     SLT <= 1'b0;
222     SLTU <= 1'b0;
223     XOR <= 1'b0;
224     SRL <= 1'b0;
225     SRA <= 1'b0;
226     OR <= 1'b0;
227     AND <= 1'b0;
228     FENCE <= 1'b0;
229     ECALL <= 1'b0;
230     EBREAK <= 1'b0;
231     CSRRW <= 1'b0;
232     CSRRS <= 1'b0;
233     CSRRC <= 1'b0;
234     CSRRWI <= 1'b0;
235     CSRRSI <= 1'b0;
236     CSRRCI <= 1'b0;
237     OTHER <= 1'b0;
238
239     casez(i_ibus_rdt)
240       // 3322222_22222 11111_111 11
241       // 1098765_43210 98765_432 10987_65432_10
242       32'b???????_?????_?????_???_?????_01101_11 : LUI
<= 1'b1;
243       32'b???????_?????_?????_???_?????_00101_11 : AUIPC
<= 1'b1;
244       32'b???????_?????_?????_???_?????_11011_11 : JAL
<= 1'b1;
245       32'b???????_?????_?????_000_?????_11001_11 : JALR
<= 1'b1;
246       32'b???????_?????_?????_000_?????_11000_11 : BEQ
<= 1'b1;
247       32'b???????_?????_?????_001_?????_11000_11 : BNE
<= 1'b1;
248       32'b???????_?????_?????_100_?????_11000_11 : BLT
<= 1'b1;
249       32'b???????_?????_?????_101_?????_11000_11 : BGE
<= 1'b1;

```

```

250      32'b???????_????_????_110_????_11000_11 : BLTU
<= 1'b1;
251      32'b???????_????_????_111_????_11000_11 : BGEU
<= 1'b1;
252      32'b???????_????_????_000_????_00000_11 : LB
<= 1'b1;
253      32'b???????_????_????_001_????_00000_11 : LH
<= 1'b1;
254      32'b???????_????_????_010_????_00000_11 : LW
<= 1'b1;
255      32'b???????_????_????_100_????_00000_11 : LBU
<= 1'b1;
256      32'b???????_????_????_101_????_00000_11 : LHU
<= 1'b1;
257      32'b???????_????_????_000_????_01000_11 : SB
<= 1'b1;
258      32'b???????_????_????_001_????_01000_11 : SH
<= 1'b1;
259      32'b???????_????_????_010_????_01000_11 : SW
<= 1'b1;
260      32'b???????_????_????_000_????_00100_11 : ADDI
<= 1'b1;
261      32'b???????_????_????_010_????_00100_11 : SLTI
<= 1'b1;
262      32'b???????_????_????_011_????_00100_11 : SLTIU
<= 1'b1;
263      32'b???????_????_????_100_????_00100_11 : XORI
<= 1'b1;
264      32'b???????_????_????_110_????_00100_11 : ORI
<= 1'b1;
265      32'b???????_????_????_111_????_00100_11 : ANDI
<= 1'b1;
266      32'b0000000_????_????_001_????_00100_11 : SLLI
<= 1'b1;
267      32'b0000000_????_????_101_????_00100_11 : SRLI
<= 1'b1;
268      32'b0100000_????_????_101_????_00100_11 : SRAI
<= 1'b1;
269      32'b0000000_????_????_000_????_01100_11 : ADD
<= 1'b1;
270      32'b0100000_????_????_000_????_01100_11 : SUB
<= 1'b1;
271      32'b0000000_????_????_001_????_01100_11 : SLL
<= 1'b1;
272      32'b0000000_????_????_010_????_01100_11 : SLT
<= 1'b1;
273      32'b0000000_????_????_011_????_01100_11 : SLTU
<= 1'b1;
274      32'b???????_????_????_100_????_01100_11 : XOR
<= 1'b1;
275      32'b0000000_????_????_101_????_01100_11 : SRL
<= 1'b1;
276      32'b0100000_????_????_101_????_01100_11 : SRA
<= 1'b1;
277      32'b???????_????_????_110_????_01100_11 : OR
<= 1'b1;
278      32'b???????_????_????_111_????_01100_11 : AND
<= 1'b1;
279      32'b???????_????_????_000_????_00011_11 : FENCE
<= 1'b1;
280      32'b0000000_00000_00000_000_00000_11100_11 : ECALL
<= 1'b1;
281      32'b0000000_00001_00000_000_00000_11100_11 : EBREAK
<= 1'b1;
282      32'b???????_????_????_001_????_11100_11 : CSRRW
<= 1'b1;
283      32'b???????_????_????_010_????_11100_11 : CSRRS
<= 1'b1;
284      32'b???????_????_????_011_????_11100_11 : CSRRC
<= 1'b1;
285      32'b???????_????_????_101_????_11100_11 : CSRRWI
<= 1'b1;
286      32'b???????_????_????_110_????_11100_11 : CSRRSI
<= 1'b1;
287      32'b???????_????_????_111_????_11100_11 : CSRRCI
<= 1'b1;
288      default : OTHER <= 1'b1;
289      endcase
290    end
291  end
292
293 `ifdef RISC_V_FORMAL
294    reg [31:0] pc = RESET_PC;
295
296    wire rs_en = two_stage_op ? init : i_ctrl_pc_en;
297
298    assign rvfi_rd_wdata = update_rd ? dbg_rd : 32'd0;
299
300    always @(posedge i_clk) begin
301      /* End of instruction */
302      rvfi_valid <= i_cnt_done & i_ctrl_pc_en & !i_rst;
303      rvfi_order <= rvfi_order + {63'd0,rvfi_valid};
304
305      /* Get instruction word when it's fetched from ibus */
306      if (i_ibus_cyc & i_ibus_ack)
307        rvfi_insn <= i_ibus_rdt;
308
309
310      if (i_cnt_done & i_ctrl_pc_en) begin
311        rvfi_pc_rdata <= pc;
312        if (!(rd_en & (i_rd_addr))) begin

```

```

313          rvfi_rd_addr <= 5'd0;
314        end
315      end
316      rvfi_trap <= trap;
317      if (rvfi_valid) begin
318        rvfi_trap <= 1'b0;
319        pc <= rvfi_pc_wdata;
320      end
321
322      /* RS1 not valid during J, U instructions (immdec_en[1])
323      */
324      /* RS2 not valid during I, J, U instructions (imm-
325      dec_en[2]) */
326      if (i_rf_ready) begin
327        rvfi_rs1_addr <= !immdec_en[1] ? rs1_addr : 5'd0;
328        rvfi_rs2_addr <= !immdec_en[2] /*rs2_valid*/ ?
329        rs2_addr : 5'd0;
330        rvfi_rd_addr <= i_rd_addr;
331      end
332      if (rs_en) begin
333        rvfi_rs1_rdata <= {(immdec_en[1] ? rs1 :
334        {W(1'b0)}),rvfi_rs1_rdata[31:W]};
335        rvfi_rs2_rdata <= {(immdec_en[2] ? rs2 :
336        {W(1'b0)}),rvfi_rs2_rdata[31:W]};
337      end
338      if (i_dbus_ack) begin
339        rvfi_mem_addr <= i_dbus_addr;
340        rvfi_mem_rmask <= i_dbus_we ? 4'b0000 : i_dbus_sel;
341        rvfi_mem_wmask <= i_dbus_we ? i_dbus_sel : 4'b0000;
342        rvfi_mem_rdata <= i_dbus_rdt;
343        rvfi_mem_wdata <= i_dbus_dat;
344      end
345      if (i_ibus_ack) begin
346        rvfi_mem_rmask <= 4'b0000;
347        rvfi_mem_wmask <= 4'b0000;
348      end
349      assign rvfi_pc_wdata = i_ibus_addr;
350    `endif
351  endmodule

```

9. olofk_serv/rtl/serv_decode.v (367 lines)

```

1 /*
2  * serv_decode.v : SERV module decoding instruction word into
3  * control signals
4  *
5  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
6  * ning.me>
7  *
8  * SPDX-License-Identifier: ISC
9  */
10 `default_nettype none
11 module serv_decode
12   #(parameter [0:0] PRE_REGISTER = 1,
13     parameter [0:0] MDU = 0)
14 (
15   input wire      clk,
16   //Input
17   input wire [31:2] i_wb_rdt,
18   input wire      i_wb_en,
19   //To state
20   output reg      o_sh_right,
21   output reg      o_bne_or_bge,
22   output reg      o_cond_branch,
23   output reg      o_e_op,
24   output reg      o_ebreak,
25   output reg      o_branch_op,
26   output reg      o_shift_op,
27   output reg      o_rd_op,
28   output reg      o_two_stage_op,
29   output reg      o_dbus_en,
30   //MDU
31   output reg      o_mdu_op,
32   //Extension
33   output reg [2:0] o_ext_funct3,
34   //To bufreg
35   output reg      o_bufreg_rs1_en,
36   output reg      o_bufreg_imm_en,
37   output reg      o_bufreg_clr_lsb,
38   output reg      o_bufreg_sh_signed,
39   //To ctrl
40   output reg      o_ctrl_jal_or_jalr,
41   output reg      o_ctrl_lutype,
42   output reg      o_ctrl_pc_rel,
43   output reg      o_ctrl_mret,
44   //To alu
45   output reg      o_alu_sub,
46   output reg [1:0] o_alu_bool_op,
47   output reg      o_alu_cmp_eq,
48   output reg      o_alu_cmp_sgt,
49   output reg [2:0] o_alu_rd_sel,
50   //To mem IF
51   output reg      o_mem_signed,
52   output reg      o_mem_word,
53   output reg      o_mem_half,

```

```

51 output reg      o_mem_cmd,
52 //To CSR
53 output reg      o_csr_en,
54 output reg [1:0] o_csr_addr,
55 output reg      o_csr_mstatus_en,
56 output reg      o_csr_mie_en,
57 output reg      o_csr_mcause_en,
58 output reg [1:0] o_csr_source,
59 output reg      o_csr_d_sel,
60 output reg      o_csr_imm_en,
61 output reg      o_mtval_pc,
62 //To top
63 output reg [3:0] o_immdec_ctrl,
64 output reg [3:0] o_immdec_en,
65 output reg      o_op_b_source,
66 //To RF IF
67 output reg      o_rd_mem_en,
68 output reg      o_rd_csr_en,
69 output reg      o_rd_alu_en;
70
71 reg [4:0] opcode;
72 reg [2:0] funct3;
73 reg      op20;
74 reg      op21;
75 reg      op22;
76 reg      op26;
77
78 reg      imm25;
79 reg      imm30;
80
81 wire co_mdu_op      = MDU & (opcode == 5'b01100) & imm25;
82
83 wire co_two_stage_op =
84   ~opcode[2] | (funct3[0] & ~funct3[1] & ~opcode[0] &
85   ~opcode[4]) |
86   (funct3[1] & ~funct3[2] & ~opcode[0] & ~opcode[4]) |
87   co_mdu_op;
88 wire co_shift_op = (opcode[2] & ~funct3[1]) & !co_mdu_op;
89 wire co_branch_op = opcode[4];
90 wire co_dbus_en   = ~opcode[2] & ~opcode[4];
91 wire co_mtval_pc  = opcode[4];
92 wire co_mem_word  = funct3[1];
93 wire co_rd_alu_en = !opcode[0] & opcode[2] & !opcode[4]
94 & !co_mdu_op;
95 wire co_rd_mem_en = (!opcode[2] & !opcode[0]) | co_mdu_op;
96 wire [2:0] co_ext_funct3 = funct3;
97
98 //jal,branch = imm
99 //jalr      = rs1+imm
100 //mem       = rs1+imm
101 //shift     = rs1
102 wire co_bufreg_rs1_en = !opcode[4] | (!opcode[1] & op-
103 code[0]);
104 wire co_bufreg_imm_en = !opcode[2];
105
106 //Clear LSB of immediate for BRANCH and JAL ops
107 //True for BRANCH and JAL
108 //False for JALR/LOAD/STORE/OP/OPIMM?
109 wire co_bufreg_clr_lsb = opcode[4] & ((opcode[1:0] ==
110 2'b00) | (opcode[1:0] == 2'b11));
111
112 //Conditional branch
113 //True for BRANCH
114 //False for JAL/JALR
115 wire co_cond_branch = !opcode[0];
116
117 wire co_ctrl_ctype = !opcode[4] & opcode[2] & op-
118 code[0];
119 wire co_ctrl_jal_or_jalr = opcode[4] & opcode[0];
120
121 //PC-relative operations
122 //True for jal, b* auipc, ebreak
123 //False for jalr, lui
124 wire co_ctrl_pc_rel = (opcode[2:0] == 3'b000) |
125   (opcode[1:0] == 2'b11) |
126   (opcode[4] & opcode[2]) & op20 |
127   (opcode[4:3] == 2'b00);
128
129 //Write to RD
130 //True for OP-IMM, AUIPC, OP, LUI, SYSTEM, JALR, JAL, LOAD
131 //False for STORE, BRANCH, MISC-MEM
132 wire co_rd_op = (opcode[2] |
133   (!opcode[2] & opcode[4] & opcode[0]) |
134   (!opcode[2] & !opcode[3] & !opcode[0]));
135
136 //
137 //funct3
138 //
139
140 wire co_sh_right = funct3[2];
141 wire co_bne_or_bge = funct3[0];
142
143 //Matches system ops except ecall/ebreak/mret
144 wire csr_op = opcode[4] & opcode[2] & (!funct3);
145
146 //op20
147 wire co_ebreak = op20;
148
149 //opcode & funct3 & op21
150

```

```

146 wire co_ctrl_mret = opcode[4] & opcode[2] & op21 & !(|func-
147 t3);
148 //Matches system opcodes except CSR accesses (funct3 == 0)
149 //and mret (!op21)
150 wire co_e_op = opcode[4] & opcode[2] & !op21 & !(|funct3);
151
152 //opcode & funct3 & imm30
153
154 wire co_bufreg_sh_signed = imm30;
155
156 /*
157 True for sub, b*, slt*
158 False for add*
159 op      opcode f3 i30
160 b*      11000 xxx x t
161 addi    00100 000 x f
162 slt*    0x100 01x x t
163 add     01100 000 0 f
164 sub     01100 000 1 t
165 */
166 wire co_alu_sub = funct3[1] | funct3[0] | (opcode[3] & im-
167 m30) | opcode[4];
168
169 /*
170 Bits 26, 22, 21 and 20 are enough to uniquely identify the
171 eight supported CSR regs
172 mtime, mtimecmp, mscratch, mepc and mtval are stored externally
173 (normally in the RF) and are
174
175 treated differently from mstatus, mie and mcause which are
176 stored in serv_csr.
177
178 The former get a 2-bit address as seen below while the
179 latter get a
180 one-hot enable signal each.
181
182 Hex|2 222|Reg      |csr
183 adr|6 210|name     |addr
184 ---|-----|-----|----
185 300|0_000|mstatus   |xx
186 304|0_100|mie      |xx
187 305|0_101|mtimecmp  |01
188 340|1_000|mscratch  |00
189 341|1_001|mepc      |10
190 342|1_010|mcause    |xx
191 343|1_011|mtval     |11
192
193 */
194
195 //true for mtimecmp, mscratch, mepc and mtval
196 //false for mstatus, mie, mcause
197 wire csr_valid = op20 | (op26 & !op21);
198
199 wire co_rd_csr_en = csr_op;
200
201 wire co_csr_en      = csr_op & csr_valid;
202 wire co_csr_mstatus_en = csr_op & !op26 & !op22 & !op20;
203 wire co_csr_mie_en   = csr_op & !op26 & op22 & !op20;
204 wire co_csr_mcause_en = csr_op      & op21 & !op20;
205
206 wire [1:0] co_csr_source = funct3[1:0];
207 wire co_csr_d_sel = funct3[2];
208 wire co_csr_imm_en = opcode[4] & opcode[2] & funct3[2];
209 wire [1:0] co_csr_addr = {op26 & op20, !op26 | op21};
210
211 wire co_alu_cmp_eq = funct3[2:1] == 2'b00;
212
213 wire co_alu_cmp_sig = ~((funct3[0] & funct3[1]) | (func-
214 t3[1] & funct3[2]));
215
216 wire co_mem_cmd = opcode[3];
217 wire co_mem_signed = ~funct3[2];
218 wire co_mem_half = funct3[0];
219
220 wire [1:0] co_alu_bool_op = funct3[1:0];
221
222 wire [3:0] co_immdec_ctrl;
223 //True for S (STORE) or B (BRANCH) type instructions
224 //False for J type instructions
225 assign co_immdec_ctrl[0] = opcode[3:0] == 4'b1000;
226 //True for OP-IMM, LOAD, STORE, JALR (I S)
227 //False for LUI, AUIPC, JAL (U J)
228 assign co_immdec_ctrl[1] = (opcode[1:0] == 2'b00) | (op-
229 code[2:1] == 2'b00);
230 assign co_immdec_ctrl[2] = opcode[4] & !opcode[0];
231 assign co_immdec_ctrl[3] = opcode[4];
232
233 wire [3:0] co_immdec_en;
234 assign co_immdec_en[3] = opcode[4] | opcode[3] | opcode[2]
235 | !opcode[0];
236 //B I J S U
237 assign co_immdec_en[2] = (opcode[4] & opcode[2]) | !op-
238 code[3] | opcode[0];
239 //I J U
240 assign co_immdec_en[1] = (opcode[2:1] == 2'b01) | (op-
241 code[2] & opcode[0]) | co_csr_imm_en;
242 //J U
243 assign co_immdec_en[0] =
244 ~co_rd_op;
245 B S
246
247 wire [2:0] co_alu_rd_sel;
248 assign co_alu_rd_sel[0] = (funct3 == 3'b000); // Add/sub

```

```

232 assign co_alu_rd_sel[1] = (funct3[2:1] == 2'b01); //SLT*
233 assign co_alu_rd_sel[2] = funct3[2]; //Bool
234
235 //0 (OP_B_SOURCE_IMM) when OPIMM
236 //1 (OP_B_SOURCE_RS2) when BRANCH or OP
237 wire co_op_b_source = opcode[3];
238
239 generate
240 if (PRE_REGISTER) begin : gen_pre_register
241
242     always @(posedge clk) begin
243         if (i_wb_en) begin
244             funct3 <= i_wb_rdt[14:12];
245             imm30 <= i_wb_rdt[30];
246             imm25 <= i_wb_rdt[25];
247             opcode <= i_wb_rdt[6:2];
248             op20 <= i_wb_rdt[20];
249             op21 <= i_wb_rdt[21];
250             op22 <= i_wb_rdt[22];
251             op26 <= i_wb_rdt[26];
252         end
253     end
254
255     always @(*) begin
256         o_sh_right = co_sh_right;
257         o_bne_or_bge = co_bne_or_bge;
258         o_cond_branch = co_cond_branch;
259         o_dbus_en = co_dbus_en;
260         o_mtval_pc = co_mtval_pc;
261         o_two_stage_op = co_two_stage_op;
262         o_e_op = co_e_op;
263         o_ebreak = co_ebreak;
264         o_branch_op = co_branch_op;
265         o_shift_op = co_shift_op;
266         o_rd_op = co_rd_op;
267         o_mdu_op = co_mdu_op;
268         o_ext_funct3 = co_ext_funct3;
269         o_bufreg_rs1_en = co_bufreg_rs1_en;
270         o_bufreg_imm_en = co_bufreg_imm_en;
271         o_bufreg_clr_lsb = co_bufreg_clr_lsb;
272         o_bufreg_sh_signed = co_bufreg_sh_signed;
273         o_ctrl_jal_or_jalr = co_ctrl_jal_or_jalr;
274         o_ctrl_utyte = co_ctrl_utyte;
275         o_ctrl_pc_rel = co_ctrl_pc_rel;
276         o_ctrl_mret = co_ctrl_mret;
277         o_alu_sub = co_alu_sub;
278         o_alu_bool_op = co_alu_bool_op;
279         o_alu_cmp_eq = co_alu_cmp_eq;
280         o_alu_cmp_sig = co_alu_cmp_sig;
281         o_alu_rd_sel = co_alu_rd_sel;
282         o_mem_signed = co_mem_signed;
283         o_mem_word = co_mem_word;
284         o_mem_half = co_mem_half;
285         o_mem_cmd = co_mem_cmd;
286         o_csr_en = co_csr_en;
287         o_csr_addr = co_csr_addr;
288         o_csr_mstatus_en = co_csr_mstatus_en;
289         o_csr_mie_en = co_csr_mie_en;
290         o_csr_mcause_en = co_csr_mcause_en;
291         o_csr_source = co_csr_source;
292         o_csr_d_sel = co_csr_d_sel;
293         o_csr_imm_en = co_csr_imm_en;
294         o_immdec_ctrl = co_immdec_ctrl;
295         o_immdec_en = co_immdec_en;
296         o_op_b_source = co_op_b_source;
297         o_rd_csr_en = co_rd_csr_en;
298         o_rd_alu_en = co_rd_alu_en;
299         o_rd_mem_en = co_rd_mem_en;
300     end
301 end
302
303 end else begin : gen_post_register
304
305     always @(*) begin
306         funct3 = i_wb_rdt[14:12];
307         imm30 = i_wb_rdt[30];
308         imm25 = i_wb_rdt[25];
309         opcode = i_wb_rdt[6:2];
310         op20 = i_wb_rdt[20];
311         op21 = i_wb_rdt[21];
312         op22 = i_wb_rdt[22];
313         op26 = i_wb_rdt[26];
314     end
315
316     always @(posedge clk) begin
317         if (i_wb_en) begin
318             o_sh_right <= co_sh_right;
319             o_bne_or_bge <= co_bne_or_bge;
320             o_cond_branch <= co_cond_branch;
321             o_e_op <= co_e_op;
322             o_ebreak <= co_ebreak;
323             o_two_stage_op <= co_two_stage_op;
324             o_dbus_en <= co_dbus_en;
325             o_mtval_pc <= co_mtval_pc;
326             o_branch_op <= co_branch_op;
327             o_shift_op <= co_shift_op;
328             o_rd_op <= co_rd_op;
329             o_mdu_op <= co_mdu_op;
330             o_ext_funct3 <= co_ext_funct3;
331             o_bufreg_rs1_en <= co_bufreg_rs1_en;
332             o_bufreg_imm_en <= co_bufreg_imm_en;
333             o_bufreg_clr_lsb <= co_bufreg_clr_lsb;

```

```

333         o_bufreg_sh_signed <= co_bufreg_sh_signed;
334         o_ctrl_jal_or_jalr <= co_ctrl_jal_or_jalr;
335         o_ctrl_utyte <= co_ctrl_utyte;
336         o_ctrl_pc_rel <= co_ctrl_pc_rel;
337         o_ctrl_mret <= co_ctrl_mret;
338         o_alu_sub <= co_alu_sub;
339         o_alu_bool_op <= co_alu_bool_op;
340         o_alu_cmp_eq <= co_alu_cmp_eq;
341         o_alu_cmp_sig <= co_alu_cmp_sig;
342         o_alu_rd_sel <= co_alu_rd_sel;
343         o_mem_signed <= co_mem_signed;
344         o_mem_word <= co_mem_word;
345         o_mem_half <= co_mem_half;
346         o_mem_cmd <= co_mem_cmd;
347         o_csr_en <= co_csr_en;
348         o_csr_addr <= co_csr_addr;
349         o_csr_mstatus_en <= co_csr_mstatus_en;
350         o_csr_mie_en <= co_csr_mie_en;
351         o_csr_mcause_en <= co_csr_mcause_en;
352         o_csr_source <= co_csr_source;
353         o_csr_d_sel <= co_csr_d_sel;
354         o_csr_imm_en <= co_csr_imm_en;
355         o_immdec_ctrl <= co_immdec_ctrl;
356         o_immdec_en <= co_immdec_en;
357         o_op_b_source <= co_op_b_source;
358         o_rd_csr_en <= co_rd_csr_en;
359         o_rd_alu_en <= co_rd_alu_en;
360         o_rd_mem_en <= co_rd_mem_en;
361     end
362 end
363
364 end
365 endgenerate
366
367 endmodule

```

10. olofk_serv/rtl/serv_immdec.v (234 lines)

```

1 /*
2  * serv_immdec.v : SERV module for decoding immediates from
3  * instruction words
4  *
5  * SPDX-FileCopyrightText: 2020 Olof Kindgren <olof@award-win-
6  * ning.me>
7  *
8  * SPDX-License-Identifier: ISC
9  */
10
11 `default_nettype none
12 module serv_immdec
13     #(parameter SHARED_RFADDR_IMM_REGS = 1,
14       parameter W = 1)
15 (
16     input wire i_clk,
17     //State
18     input wire i_cnt_en,
19     input wire i_cnt_done,
20     //Control
21     input wire [3:0] i_immdec_en,
22     input wire i_csr_imm_en,
23     input wire [3:0] i_ctrl,
24     output wire [4:0] o_rd_addr,
25     output wire [4:0] o_rs1_addr,
26     output wire [4:0] o_rs2_addr,
27     //Data
28     output wire [W-1:0] o_csr_imm,
29     output wire [W-1:0] o_imm,
30     //External
31     input wire i_wb_en,
32     input wire [31:7] i_wb_rdt);
33
34 generate
35 if (W == 1) begin : gen_immdec_w_eq_1
36     reg
37         imm31;
38
39     reg [8:0] imm19_12_20;
40     reg imm7;
41     reg [5:0] imm30_25;
42     reg [4:0] imm24_20;
43     reg [4:0] imm11_7;
44
45     assign o_csr_imm = imm19_12_20[4];
46
47     wire
48         signbit = imm31 & !i_csr_imm_en;
49
50     if (SHARED_RFADDR_IMM_REGS) begin : gen_shared_imm_regs
51         assign o_rs1_addr = imm19_12_20[8:4];
52         assign o_rs2_addr = imm24_20;
53         assign o_rd_addr = imm11_7;
54
55         always @(posedge i_clk) begin
56             if (i_wb_en) begin
57                 /* CSR immediates are always zero-extended,
58                 hence clear the signbit */
59                 imm31 <= i_wb_rdt[31];
60             end
61             if (i_wb_en | (i_cnt_en & i_immdec_en[1]))
62                 imm19_12_20 <= i_wb_en ?
63                 {i_wb_rdt[19:12], i_wb_rdt[20]} : {i_ctrl[3] ? signbit : imm24_20[0],
64                 imm19_12_20[8:1]};
65             if (i_wb_en | (i_cnt_en))
66                 imm7 <= i_wb_en ?

```

```

i_wb_rdt[7] : signbit;
58
59 if (i_wb_en | (i_cnt_en & i_immdec_en[3]))
60   imm30_25 <= i_wb_en ? i_wb_rdt[30:25] : {i_c-
trl[2] ? imm7 : i_ctrl[1] ? signbit : imm19_12_20[0], im-
m30_25[5:1]};
61
62 if (i_wb_en | (i_cnt_en & i_immdec_en[2]))
63   imm24_20 <= i_wb_en ? i_wb_rdt[24:20] : {im-
m30_25[0], imm24_20[4:1]};
64
65 if (i_wb_en | (i_cnt_en & i_immdec_en[0]))
66   imm11_7 <= i_wb_en ? i_wb_rdt[11:7] : {im-
m30_25[0], imm11_7[4:1]};
67 end
68 end else begin : gen_separate_imm_regs
69   reg [4:0] rd_addr;
70   reg [4:0] rs1_addr;
71   reg [4:0] rs2_addr;
72
73   assign o_rd_addr = rd_addr;
74   assign o_rs1_addr = rs1_addr;
75   assign o_rs2_addr = rs2_addr;
76   always @(posedge i_clk) begin
77     if (i_wb_en) begin
78       /* CSR immediates are always zero-extended,
hence clear the signbit */
79       imm31 <= i_wb_rdt[31];
80       imm19_12_20 <= {i_wb_rdt[19:12], i_wb_rdt[20]};
81       imm7 <= i_wb_rdt[7];
82       imm30_25 <= i_wb_rdt[30:25];
83       imm24_20 <= i_wb_rdt[24:20];
84       imm11_7 <= i_wb_rdt[11:7];
85
86       rd_addr <= i_wb_rdt[11:7];
87       rs1_addr <= i_wb_rdt[19:15];
88       rs2_addr <= i_wb_rdt[24:20];
89     end
90     if (i_cnt_en) begin
91       imm19_12_20 <= {i_ctrl[3] ? signbit : im-
m24_20[0], imm19_12_20[8:1]};
92       imm7 <= signbit;
93       imm30_25 <= {i_ctrl[2] ? imm7 : i_ctrl[1] ?
signbit : imm19_12_20[0], imm30_25[5:1]};
94       imm24_20 <= {imm30_25[0], imm24_20[4:1]};
95       imm11_7 <= {imm30_25[0], imm11_7[4:1]};
96     end
97   end
98 end
99
100 assign o_imm = i_cnt_done ? signbit : i_ctrl[0] ? im-
m11_7[0] : imm24_20[0];
101 end else begin : gen_immdec_w_eq_4
102   reg [4:0] rd_addr;
103   reg [4:0] rs1_addr;
104   reg [4:0] rs2_addr;
105
106   reg i31;
107   reg i30;
108   reg i29;
109   reg i28;
110   reg i27;
111   reg i26;
112   reg i25;
113   reg i24;
114   reg i23;
115   reg i22;
116   reg i21;
117   reg i20;
118   reg i19;
119   reg i18;
120   reg i17;
121   reg i16;
122   reg i15;
123   reg i14;
124   reg i13;
125   reg i12;
126   reg i11;
127   reg i10;
128   reg i9;
129   reg i8;
130   reg i7;
131
132   reg i7_2;
133   reg i20_2;
134
135   wire signbit = i31 & !i_csr_imm_en;
136
137   assign o_csr_imm[3] = i18;
138   assign o_csr_imm[2] = i17;
139   assign o_csr_imm[1] = i16;
140   assign o_csr_imm[0] = i15;
141
142   assign o_rd_addr = rd_addr;
143   assign o_rs1_addr = rs1_addr;
144   assign o_rs2_addr = rs2_addr;
145   always @(posedge i_clk) begin
146     if (i_wb_en) begin
147       //Common
148       i31 <= i_wb_rdt[31];
149

```

```

150 //Bit lane 3
151 i19 <= i_wb_rdt[19];
152 i15 <= i_wb_rdt[15];
153 i20 <= i_wb_rdt[20];
154 i7 <= i_wb_rdt[7];
155 i27 <= i_wb_rdt[27];
156 i23 <= i_wb_rdt[23];
157 i10 <= i_wb_rdt[10];
158
159 //Bit lane 2
160 i22 <= i_wb_rdt[22];
161 i9 <= i_wb_rdt[9];
162 i26 <= i_wb_rdt[26];
163 i30 <= i_wb_rdt[30];
164 i14 <= i_wb_rdt[14];
165 i18 <= i_wb_rdt[18];
166
167 //Bit lane 1
168 i21 <= i_wb_rdt[21];
169 i8 <= i_wb_rdt[8];
170 i25 <= i_wb_rdt[25];
171 i29 <= i_wb_rdt[29];
172 i13 <= i_wb_rdt[13];
173 i17 <= i_wb_rdt[17];
174
175 //Bit lane 0
176 i11 <= i_wb_rdt[11];
177 i7_2 <= i_wb_rdt[7];
178 i20_2 <= i_wb_rdt[20];
179 i24 <= i_wb_rdt[24];
180 i28 <= i_wb_rdt[28];
181 i12 <= i_wb_rdt[12];
182 i16 <= i_wb_rdt[16];
183
184 rd_addr <= i_wb_rdt[11:7];
185 rs1_addr <= i_wb_rdt[19:15];
186 rs2_addr <= i_wb_rdt[24:20];
187 end
188 if (i_cnt_en) begin
189   //Bit lane 3
190   i10 <= i27;
191   i23 <= i27;
192   i27 <= i_ctrl[2] ? i7 : i_ctrl[1] ? signbit : i20;
193   i7 <= signbit;
194   i20 <= i15;
195   i15 <= i19;
196   i19 <= i_ctrl[3] ? signbit : i23;
197
198   //Bit lane 2
199   i22 <= i26;
200   i9 <= i26;
201   i26 <= i30;
202   i30 <= (i_ctrl[1] | i_ctrl[2]) ? signbit : i14;
203   i14 <= i18;
204   i18 <= i_ctrl[3] ? signbit : i22;
205
206   //Bit lane 1
207   i21 <= i25;
208   i8 <= i25;
209   i25 <= i29;
210   i29 <= (i_ctrl[1] | i_ctrl[2]) ? signbit : i13;
211   i13 <= i17;
212   i17 <= i_ctrl[3] ? signbit : i21;
213
214   //Bit lane 0
215   i7_2 <= i11;
216   i11 <= i28;
217   i20_2 <= i24;
218   i24 <= i28;
219   i28 <= (i_ctrl[1] | i_ctrl[2]) ? signbit : i12;
220   i12 <= i16;
221   i16 <= i_ctrl[3] ? signbit : i20_2;
222
223 end
224 end
225
226 assign o_imm[3] = (i_cnt_done ? signbit : (i_ctrl[0] ?
i10 : i23));
227 assign o_imm[2] = i_ctrl[0] ? i9 : i22;
228 assign o_imm[1] = i_ctrl[0] ? i8 : i21;
229 assign o_imm[0] = i_ctrl[0] ? i7_2 : i20_2;
230
231 end
232 endgenerate
233
234 endmodule

```

11. olofk_serv/rtl/serv_mem_if.v (59 lines)

```

1 /*
2  * serv_mem_if.v : SERV memory interface
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8 module serv_mem_if
9   #(
10     parameter [0:0] WITH_CSR = 1,

```

```

11     parameter      W = 1,
12     parameter      B = W-1
13 )
14 (
15     input wire      i_clk,
16     //State
17     input wire [1:0] i_bytecnt,
18     input wire [1:0] i_lsb,
19     output wire      o_misalign,
20     //Control
21     input wire      i_signed,
22     input wire      i_word,
23     input wire      i_half,
24     //MDU
25     input wire      i_mdu_op,
26     //Data
27     input wire [B:0] i_bufreg2_q,
28     output wire [B:0] o_rd,
29     //External interface
30     output wire [3:0] o_wb_sel);
31
32     reg signbit;
33
34     wire dat_valid =
35         i_mdu_op |
36         i_word |
37         (i_bytecnt == 2'b00) |
38         (i_half & !i_bytecnt[1]);
39
40     assign o_rd = dat_valid ? i_bufreg2_q : {W{i_signed & sign-
bit}};
41
42     assign o_wb_sel[3] = (i_lsb == 2'b11) | i_word | (i_half &
i_lsb[1]);
43     assign o_wb_sel[2] = (i_lsb == 2'b10) | i_word;
44     assign o_wb_sel[1] = (i_lsb == 2'b01) | i_word | (i_half
& !i_lsb[1]);
45     assign o_wb_sel[0] = (i_lsb == 2'b00);
46
47     always @(posedge i_clk) begin
48         if (dat_valid)
49             signbit <= i_bufreg2_q[B];
50     end
51
52     /*
53     mem_misalign is checked after the init stage to decide
whether to do a data
54     bus transaction or go to the trap state. It is only guar-
anteed to be correct
55     at this time
56     */
57     assign o_misalign = WITH_CSR & ((i_lsb[0] & (i_word |
i_half)) | (i_lsb[1] & i_word));
58
59 endmodule

```

12. olof_serv/rtl/serv_rf_if.v (159 lines)

```

1  /*
2  * serv_rf_if.v : SERV register file interface
3  *
4  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7  `default_nettype none
8  module serv_rf_if
9      #(parameter WITH_CSR = 1,
10         parameter W = 1,
11         parameter B = W-1
12     )
13     //RF Interface
14     input wire      i_cnt_en,
15     output wire [4+WITH_CSR:0] o_wreg0,
16     output wire [4+WITH_CSR:0] o_wreg1,
17     output wire      o_wen0,
18     output wire      o_wen1,
19     output wire [B:0] o_wdata0,
20     output wire [B:0] o_wdata1,
21     output wire [4+WITH_CSR:0] o_rreg0,
22     output wire [4+WITH_CSR:0] o_rreg1,
23     input wire [B:0] i_rdata0,
24     input wire [B:0] i_rdata1,
25
26     //Trap interface
27     input wire      i_trap,
28     input wire      i_mret,
29     input wire [B:0] i_mepc,
30     input wire      i_mtval_pc,
31     input wire [B:0] i_bufreg_q,
32     input wire [B:0] i_bad_pc,
33     output wire [B:0] o_csr_pc,
34     //CSR interface
35     input wire      i_csr_en,
36     input wire [1:0] i_csr_addr,
37     input wire [B:0] i_csr,
38     output wire [B:0] o_csr,
39     //RD write port
40     input wire      i_rd_wen,
41     input wire [4:0] i_rd_waddr,

```

```

42     input wire [B:0] i_ctrl_rd,
43     input wire [B:0] i_alu_rd,
44     input wire      i_rd_alu_en,
45     input wire [B:0] i_csr_rd,
46     input wire      i_rd_csr_en,
47     input wire [B:0] i_mem_rd,
48     input wire      i_rd_mem_en,
49
50     //RS1 read port
51     input wire [4:0] i_rs1_raddr,
52     output wire [B:0] o_rs1,
53     //RS2 read port
54     input wire [4:0] i_rs2_raddr,
55     output wire [B:0] o_rs2;
56
57     /*
58     ***** Write side *****
59     */
60
61     wire      rd_wen = i_rd_wen & (!i_rd_waddr);
62
63     generate
64     if (!WITH_CSR) begin : gen_csr
65     wire [B:0] rd =
66         {W{i_rd_alu_en}} & i_alu_rd |
67         {W{i_rd_csr_en}} & i_csr_rd |
68         {W{i_rd_mem_en}} & i_mem_rd |
69         i_ctrl_rd;
70
71     wire [B:0] mtval = i_mtval_pc ? i_bad_pc : i_bufreg_q;
72
73     assign      o_wdata0 = i_trap ? mtval : rd;
74     assign      o_wdata1 = i_trap ? i_mepc : i_csr;
75
76     /* Port 0 handles writes to mtval during traps and rd oth-
erwise
77     * Port 1 handles writes to mepc during traps and csr ac-
cesses otherwise
78     */
79     * GPR registers are mapped to address 0-31 (bits 0xxxxx).
80     * Following that are four CSR registers
81     * mscratch 100000
82     * mtvec    100001
83     * mepc     100010
84     * mtval    100011
85     */
86
87     assign o_wreg0 = i_trap ? {6'b100011} : {1'b0, i_rd_waddr};
88     assign o_wreg1 = i_trap ? {6'b100010} :
{4'b1000, i_csr_addr};
89
90     assign      o_wen0 = i_cnt_en & (i_trap | rd_wen);
91     assign      o_wen1 = i_cnt_en & (i_trap | i_csr_en);
92
93     /*
94     ***** Read side *****
95     */
96
97     //0 : RS1
98     //1 : RS2 / CSR
99
100     assign o_rreg0 = {1'b0, i_rs1_raddr};
101
102     /*
103     The address of the second read port (o_rreg1) can get as-
signed from four
104     different sources
105
106     Normal operations : i_rs2_raddr
107     CSR access        : i_csr_addr
108     trap              : MTVEC
109     mret              : MEPC
110
111     Address 0-31 in the RF are assigned to the GPRs. After
that follows the four
112     CSRs on addresses 32-35
113
114     32 MSCRATCH
115     33 MTVEC
116     34 MEPC
117     35 MTVAL
118
119     The expression below is an optimized version of this logic
120     */
121     wire sel_rs2 = !(i_trap | i_mret | i_csr_en);
122     assign o_rreg1 = {~sel_rs2,
i_rs2_raddr[4:2] & {3{sel_rs2}},
123         {1'b0, i_trap} | {i_mret, 1'b0} |
124         ({2{i_csr_en}} & i_csr_addr) | ({2{sel_rs2}} & i_rs2_raddr[1:0])};
125
126     assign o_rs1 = i_rdata0;
127     assign o_rs2 = i_rdata1;
128     assign o_csr = i_rdata1 & {W{i_csr_en}};
129     assign o_csr_pc = i_rdata1;
130
131     end else begin : gen_no_csr
132     wire [B:0] rd = (i_ctrl_rd) |
133         i_alu_rd & {W{i_rd_alu_en}} |
134         i_mem_rd & {W{i_rd_mem_en}};
135
136

```



```

137 assign o_wdata0 = rd;
138 assign o_wdata1 = {W{1'b0}};
139
140 assign o_wreg0 = i_rd_waddr;
141 assign o_wreg1 = 5'd0;
142
143 assign o_wen0 = i_cnt_en & rd_wen;
144 assign o_wen1 = 1'b0;
145
146 /*
147 ***** Read side *****
148 */
149
150 assign o_rreg0 = i_rs1_raddr;
151 assign o_rreg1 = i_rs2_raddr;
152
153 assign o_rs1 = i_rdata0;
154 assign o_rs2 = i_rdata1;
155 assign o_csr = {W{1'b0}};
156 assign o_csr_pc = {W{1'b0}};
157 end // else: !if(WITH_CSR)
158 endgenerate
159 endmodule

```

13. olof_serv/rtl/serv_rf_ram.v (51 lines)

```

1 /*
2  * serv_rf_ram.v : SRAM-based RF storage for SERV
3  *
4  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
5  *   ning.me>
6  * SPDX-License-Identifier: ISC
7  */
8 module serv_rf_ram
9 # (parameter width=0,
10   parameter csr_regs=4,
11   parameter depth=32*(32+csr_regs)/width)
12 (input wire i_clk,
13  input wire [$clog2(depth)-1:0] i_waddr,
14  input wire [width-1:0] i_wdata,
15  input wire i_wen,
16  input wire [$clog2(depth)-1:0] i_raddr,
17  input wire i_ren,
18  output wire [width-1:0] o_rdata);
19
20 reg [width-1:0] memory [0:depth-1];
21 reg [width-1:0] rdata;
22
23 always @(posedge i_clk) begin
24   if (i_wen)
25     memory[i_waddr] <= i_wdata;
26   rdata <= i_ren ? memory[i_raddr] : {width{1'bx}};
27 end
28
29 /* Reads from reg x0 needs to return 0
30 Check that the part of the read address corresponding to
31 the register
32 is zero and gate the output
33 width LSB of reg index $clog2(width)
34 2 4 1
35 4 3 2
36 8 2 3
37 16 1 4
38 32 0 5
39 */
40 reg regzero;
41
42 always @(posedge i_clk)
43   regzero <= !([i_raddr[$clog2(depth)-1:5-$clog2(width)]]);
44
45 assign o_rdata = rdata & ~{width{regzero}};
46
47 `ifdef SERV_CLEAR_RAM
48   integer i;
49   initial
50     for (i=0;i<depth;i=i+1)
51       memory[i] = {width{1'd0}};
52 `endif
53 endmodule

```

14. olof_serv/rtl/serv_rf_ram_if.v (186 lines)

```

1 /*
2  * serv_rf_ram_if.v : Interface between SERV and SRAM-based RF
3  *   storage
4  *
5  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
6  *   ning.me>
7  * SPDX-License-Identifier: ISC
8  */
9 `default_nettype none
10 module serv_rf_ram_if
11 # (//Data width. Adjust to preferred width of SRAM data in-
12   parameter width=8,
13   parameter W = 1,
14   //Select reset strategy.
15   // "MINI" for resetting minimally required FFs

```

```

15 // "NONE" for relying on FFs having a defined value on
16 startup
17 parameter reset_strategy="MINI",
18 //Number of CSR registers. These are allocated after the
19 normal
20 // GPR registers in the RAM.
21 parameter csr_regs=4,
22
23 //Internal parameters calculated from above values. Do not
24 change
25 parameter B=W-1,
26 parameter raw=$clog2(32+csr_regs), //Register address
27 width
28 parameter l2w=$clog2(width), //log2 of width
29 parameter aw=5+raw-l2w //Address width
30 (
31 //SERV side
32 input wire i_clk,
33 input wire i_rst,
34 input wire i_wreg,
35 input wire i_rreq,
36 input wire i_rreg,
37 output wire o_ready,
38 input wire [raw-1:0] i_wreg0,
39 input wire [raw-1:0] i_wreg1,
40 input wire i_wen0,
41 input wire i_wen1,
42 input wire [B:0] i_wdata0,
43 input wire [B:0] i_wdata1,
44 input wire [raw-1:0] i_rreg0,
45 input wire [raw-1:0] i_rreg1,
46 output wire [B:0] o_rdata0,
47 output wire [B:0] o_rdata1,
48 //RAM side
49 output wire [aw-1:0] o_waddr,
50 output wire [width-1:0] o_wdata,
51 output wire o_wen,
52 output wire [aw-1:0] o_raddr,
53 output wire o_ren,
54 input wire [width-1:0] i_rdata);
55
56 localparam ratio = width/W;
57 localparam CMSB = 4-$clog2(W); //Counter MSB
58 localparam l2r = $clog2(ratio);
59
60 reg rcnt;
61 assign o_ready = rcnt | i_wreg;
62 reg [CMSB:0] rcnt;
63
64 reg rtrig1;
65 /*
66 ***** Write side *****
67 */
68 wire [CMSB:0] wcnt;
69
70 reg [width-1:0] wdata0_r;
71 reg [width+W-1:0] wdata1_r;
72
73 reg wen0_r;
74 reg wen1_r;
75 wire wtrig0;
76 wire wtrig1;
77
78 assign wtrig0 = rtrig1;
79
80 generate if (ratio == 2) begin : gen_wtrig_ratio_eq_2
81   assign wtrig1 = wcnt[0];
82 end else begin : gen_wtrig_ratio_neq_2
83   reg wtrig0_r;
84   always @(posedge i_clk) wtrig0_r <= wtrig0;
85   assign wtrig1 = wtrig0_r;
86 end
87 endgenerate
88
89 assign
90   o_wdata = wtrig1 ?
91     wdata1_r[width-1:0] :
92     wdata0_r;
93
94 wire [raw-1:0] wreg = wtrig1 ? i_wreg1 : i_wreg0;
95 generate if (width == 32) begin : gen_w_eq_32
96   assign o_waddr = wreg;
97 end else begin : gen_w_neq_32
98   assign o_waddr = {wreg, wcnt[CMSB:l2r]};
99 end
100 endgenerate
101
102 assign o_wen = (wtrig0 & wen0_r) | (wtrig1 & wen1_r);
103
104 assign rcnt = rcnt-4;
105
106 always @(posedge i_clk) begin
107   if (wcnt[0]) begin
108     wen0_r <= i_wen0;
109     wen1_r <= i_wen1;
110   end
111
112   wdata0_r <= {i_wdata0, wdata0_r[width-1:W]};
113   wdata1_r <= {i_wdata1, wdata1_r[width+W-1:W]};
114 end

```



```

111 end
112 /*
113 ***** Read side *****
114 */
115
116 wire          rtrig0;
117
118 wire [raw-1:0] rreg = rtrig0 ? i_rreg1 : i_rreg0;
119 generate if (width == 32) begin : gen_rreg_eq_32
120     assign o_raddr = rreg;
121 end else begin : gen_rreg_neq_32
122     assign o_raddr = {rreg, rcnt[CMSB:l2r]};
123 end
124 endgenerate
125
126 reg [width-1:0] rdata0;
127 reg [width-1-W:0] rdata1;
128
129 reg          rgate;
130
131 assign o_rdata0 = rdata0[B:0];
132 assign o_rdata1 = rtrig1 ? i_rdata[B:0] : rdata1[B:0];
133
134 assign rtrig0 = (rcnt[l2r-1:0] == 1);
135
136 generate if (ratio == 2) begin : gen_ren_w_eq_2
137     assign o_ren = rgate;
138 end else begin : gen_ren_w_neq_2
139     assign o_ren = rgate & (rcnt[l2r-1:1] == 0);
140 end
141 endgenerate
142
143 reg          rreq_r;
144
145 generate if (ratio > 2) begin : gen_rdata1_w_neq_2
146     always @(posedge i_clk) begin
147         rdata1 <= {{W{1'b0}}, rdata1[width-W-1:W]};
148         if (rtrig1)
149             rdata1[width-W-1:0] <= i_rdata[width-1:W];
150     end
151 end else begin : gen_rdata1_w_eq_2
152     always @(posedge i_clk) if (rtrig1) rdata1 <=
153         i_rdata[W*2-1:W];
154 end
155 endgenerate
156
157 always @(posedge i_clk) begin
158     if (&rcnt | i_rreq)
159         rgate <= i_rreq;
160
161     rtrig1 <= rtrig0;
162     rcnt <= rcnt+{{CMSB{1'b0}}, 1'b1};
163     if (i_rreq | i_wreq)
164         rcnt <= {{CMSB-1{1'b0}}, i_wreq, 1'b0};
165
166     rreq_r <= i_rreq;
167     rgnt <= rreq_r;
168
169     rdata0 <= {{W{1'b0}}, rdata0[width-1:W]};
170     if (rtrig0)
171         rdata0 <= i_rdata;
172
173     if (i_rst) begin
174         if (reset_strategy != "NONE") begin
175             rgate <= 1'b0;
176             rgnt <= 1'b0;
177             rreq_r <= 1'b0;
178             rcnt <= {CMSB+1{1'b0}};
179         end
180     end
181 end
182 end
183
184
185
186 endmodule

```

15. olofk_serv/rtl/serv_rf_top.v (227 lines)

```

1 /*
2  * serv_rf_top.v : Toplevel including SERV and SRAM-based RF
3  storage
4  *
5  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
6  ning.me>
7  * SPDX-License-Identifier: ISC
8  */
9 `default_nettype none
10
11 module serv_rf_top
12     #(parameter RESET_PC = 32'd0,
13       /* COMPRESSED=1: Enable the compressed decoder and al-
14        lowed misaligned jump of pc
15        COMPRESSED=0: Disable the compressed decoder and does
16        not allow the misaligned jump of pc
17       */
18       parameter [0:0] COMPRESSED = 0,
19       /*
20        ALIGN = 1: Fetch the aligned instruction by making two

```

```

bus transactions if the misaligned address
17     is given to the instruction bus.
18     */
19     parameter [0:0] ALIGN = COMPRESSED,
20     /* Multiplication and Division Unit
21     This parameter enables the interface for connecting
22     SERV and MDU
23     */
24     parameter [0:0] MDU = 0,
25     /* Register signals before or after the decoder
26     0 : Register after the decoder. Faster but uses more
27     resources
28     1 : (default) Register before the decoder. Slower but
29     uses less resources
30     */
31     parameter PRE_REGISTER = 1,
32     /* Amount of reset applied to design
33     "NONE" : No reset at all. Relies on a POR to set cor-
34     rect initialization
35     values and that core isn't reset during
36     runtime
37
38     "MINI" : Standard setting. Resets the minimal amount of
39     FFs needed to
40     restart execution from the instruction at RE-
41     SET_PC
42     */
43     parameter RESET_STRATEGY = "MINI",
44     parameter [0:0] DEBUG = 1'b0,
45     parameter WITH_CSR = 1,
46     parameter W = 1,
47     parameter RF_WIDTH = W * 2,
48     parameter RF_L2D = $clog2((32+(WITH_CSR*4))*32/
49     RF_WIDTH))
50     (
51         input wire          clk,
52         input wire          i_rst,
53         input wire          i_timer_irq,
54         `ifdef RISC_V_FORMAL
55         output wire          rvfi_valid,
56         output wire [63:0] rvfi_order,
57         output wire [31:0] rvfi_insn,
58         output wire          rvfi_trap,
59         output wire          rvfi_halt,
60         output wire          rvfi_intr,
61         output wire [1:0] rvfi_mode,
62         output wire [1:0] rvfi_ixl,
63         output wire [4:0] rvfi_rs1_addr,
64         output wire [4:0] rvfi_rs2_addr,
65         output wire [31:0] rvfi_rs1_rdata,
66         output wire [31:0] rvfi_rs2_rdata,
67         output wire [4:0] rvfi_rd_addr,
68         output wire [31:0] rvfi_rd_wdata,
69         output wire [31:0] rvfi_pc_rdata,
70         output wire [31:0] rvfi_pc_wdata,
71         output wire [31:0] rvfi_mem_addr,
72         output wire [3:0] rvfi_mem_rmask,
73         output wire [3:0] rvfi_mem_wmask,
74         output wire [31:0] rvfi_mem_rdata,
75         output wire [31:0] rvfi_mem_wdata,
76         `endif
77         output wire [31:0] o_ibus_adr,
78         output wire          o_ibus_cyc,
79         input wire [31:0] i_ibus_rdt,
80         input wire          i_ibus_ack,
81         output wire [31:0] o_dbus_adr,
82         output wire [31:0] o_dbus_dat,
83         output wire [3:0] o_dbus_sel,
84         output wire          o_dbus_we,
85         output wire          o_dbus_cyc,
86         input wire [31:0] i_dbus_rdt,
87         input wire          i_dbus_ack,
88
89         // Extension
90         output wire [31:0] o_ext_rs1,
91         output wire [31:0] o_ext_rs2,
92         output wire [ 2:0] o_ext_func3,
93         input wire [31:0] i_ext_rd,
94         input wire          i_ext_ready,
95         // MDU
96         output wire          o_mdu_valid);
97
98     localparam CSR_REGS = WITH_CSR*4;
99
100     wire          rf_wreq;
101     wire          rf_rreq;
102     wire [4+WITH_CSR:0] wreg0;
103     wire [4+WITH_CSR:0] wreg1;
104     wire          wen0;
105     wire          wen1;
106     wire [W-1:0] wdata0;
107     wire [W-1:0] wdata1;
108     wire [4+WITH_CSR:0] rreg0;
109     wire [4+WITH_CSR:0] rreg1;
110     wire          rf_ready;
111     wire [W-1:0] rdata0;
112     wire [W-1:0] rdata1;
113
114     wire [RF_L2D-1:0] waddr;
115     wire [RF_WIDTH-1:0] wdata;
116     wire          wen;

```

```

108 wire [RF_L2D-1:0] raddr;
109 wire ren;
110 wire [RF_WIDTH-1:0] rdata;
111
112 serv_rf_ram_if
113   #(.width (RF_WIDTH),
114     .reset_strategy (RESET_STRATEGY),
115     .csr_regs (CSR_REGS),
116     .W(W))
117   rf_ram_if
118     (.i_clk (clk),
119      .i_rst (i_rst),
120      .i_wreq (rf_wreq),
121      .i_rreq (rf_rreq),
122      .o_ready (rf_ready),
123      .i_wreg0 (wreg0),
124      .i_wreg1 (wreg1),
125      .i_wen0 (wen0),
126      .i_wen1 (wen1),
127      .i_wdata0 (wdata0),
128      .i_wdata1 (wdata1),
129      .i_rreg0 (rreg0),
130      .i_rreg1 (rreg1),
131      .o_rdata0 (rdata0),
132      .o_rdata1 (rdata1),
133      .o_waddr (waddr),
134      .o_wdata (wdata),
135      .o_wen (wen),
136      .o_raddr (raddr),
137      .o_ren (ren),
138      .i_rdata (rdata));
139
140 serv_rf_ram
141   #(.width (RF_WIDTH),
142     .csr_regs (CSR_REGS))
143   rf_ram
144     (.i_clk (clk),
145      .i_waddr (waddr),
146      .i_wdata (wdata),
147      .i_wen (wen),
148      .i_raddr (raddr),
149      .i_ren (ren),
150      .o_rdata (rdata));
151
152 serv_top
153   #(.RESET_PC (RESET_PC),
154     .PRE_REGISTER (PRE_REGISTER),
155     .RESET_STRATEGY (RESET_STRATEGY),
156     .WITH_CSR (WITH_CSR),
157     .DEBUG (DEBUG),
158     .MDU (MDU),
159     .COMPRESSED (COMPRESSED),
160     .ALIGN (ALIGN),
161     .W(W))
162   cpu
163     (
164       .clk (clk),
165       .i_rst (i_rst),
166       .i_timer_irq (i_timer_irq),
167       `ifdef RISC_V_FORMAL
168         .rvfi_valid (rvfi_valid),
169         .rvfi_order (rvfi_order),
170         .rvfi_insn (rvfi_insn),
171         .rvfi_trap (rvfi_trap),
172         .rvfi_halt (rvfi_halt),
173         .rvfi_intr (rvfi_intr),
174         .rvfi_mode (rvfi_mode),
175         .rvfi_ixl (rvfi_ixl),
176         .rvfi_rs1_addr (rvfi_rs1_addr),
177         .rvfi_rs2_addr (rvfi_rs2_addr),
178         .rvfi_rs1_rdata (rvfi_rs1_rdata),
179         .rvfi_rs2_rdata (rvfi_rs2_rdata),
180         .rvfi_rd_addr (rvfi_rd_addr),
181         .rvfi_rd_wdata (rvfi_rd_wdata),
182         .rvfi_pc_rdata (rvfi_pc_rdata),
183         .rvfi_pc_wdata (rvfi_pc_wdata),
184         .rvfi_mem_addr (rvfi_mem_addr),
185         .rvfi_mem_rmask (rvfi_mem_rmask),
186         .rvfi_mem_wmask (rvfi_mem_wmask),
187         .rvfi_mem_rdata (rvfi_mem_rdata),
188         .rvfi_mem_wdata (rvfi_mem_wdata),
189       `endif
190       .o_rf_rreq (rf_rreq),
191       .o_rf_wreq (rf_wreq),
192       .i_rf_ready (rf_ready),
193       .o_wreg0 (wreg0),
194       .o_wreg1 (wreg1),
195       .o_wen0 (wen0),
196       .o_wen1 (wen1),
197       .o_wdata0 (wdata0),
198       .o_wdata1 (wdata1),
199       .o_rreg0 (rreg0),
200       .o_rreg1 (rreg1),
201       .i_rdata0 (rdata0),
202       .i_rdata1 (rdata1),
203
204       .o_ibus_adr (o_ibus_adr),
205       .o_ibus_cyc (o_ibus_cyc),
206       .i_ibus_rdt (i_ibus_rdt),
207       .i_ibus_ack (i_ibus_ack),
208

```

```

209     .o_dbus_adr (o_dbus_adr),
210     .o_dbus_dat (o_dbus_dat),
211     .o_dbus_sel (o_dbus_sel),
212     .o_dbus_we (o_dbus_we),
213     .o_dbus_cyc (o_dbus_cyc),
214     .i_dbus_rdt (i_dbus_rdt),
215     .i_dbus_ack (i_dbus_ack),
216
217     //Extension
218     .o_ext_func3 (o_ext_func3),
219     .i_ext_ready (i_ext_ready),
220     .i_ext_rd (i_ext_rd),
221     .o_ext_rs1 (o_ext_rs1),
222     .o_ext_rs2 (o_ext_rs2),
223     //MDU
224     .o_mdu_valid (o_mdu_valid));
225
226 endmodule
227 default_nettype wire

```

16. olof_serv/rtl/serv_state.v (239 lines)

```

1 /*
2  * serv_state.v : SERV module for handling internal state dur-
3  * ing instructions
4  *
5  * SPDX-FileCopyrightText: 2019 Olof Kindgren <olof@award-win-
6  * ning.me>
7  *
8  * SPDX-License-Identifier: ISC
9  */
10 module serv_state
11   #(.parameter RESET_STRATEGY = "MINI",
12     .parameter [0:0] WITH_CSR = 1,
13     .parameter [0:0] ALIGN = 0,
14     .parameter [0:0] MDU = 0,
15     .parameter W = 1)
16   (
17     input wire i_clk,
18     input wire i_rst,
19     //State
20     input wire i_new_irq,
21     input wire i_alu_cmp,
22     output wire o_init,
23     output wire o_cnt_en,
24     output wire o_cnt0to3,
25     output wire o_cnt12to31,
26     output wire o_cnt0,
27     output wire o_cnt1,
28     output wire o_cnt2,
29     output wire o_cnt3,
30     output wire o_cnt7,
31     output wire o_cnt11,
32     output wire o_cnt12,
33     output wire o_cnt_done,
34     output wire o_bufreg_en,
35     output wire o_ctrl_pc_en,
36     output reg o_ctrl_jump,
37     output wire o_ctrl_trap,
38     input wire i_ctrl_misalign,
39     input wire i_sh_done,
40     output wire [1:0] o_mem_bytecnt,
41     input wire i_mem_misalign,
42     //Control
43     input wire i_bne_or_bge,
44     input wire i_cond_branch,
45     input wire i_dbus_en,
46     input wire i_two_stage_op,
47     input wire i_branch_op,
48     input wire i_shift_op,
49     input wire i_sh_right,
50     input wire i_alu_rd_sel1,
51     input wire i_rd_alu_en,
52     input wire i_e_op,
53     input wire i_rd_op,
54     //MDU
55     input wire i_mdu_op,
56     output wire o_mdu_valid,
57     //Extension
58     input wire i_mdu_ready,
59     //External
60     output wire o_dbus_cyc,
61     input wire i_dbus_ack,
62     output wire o_ibus_cyc,
63     input wire i_ibus_ack,
64     //RF Interface
65     output wire o_rf_rreq,
66     output wire o_rf_wreq,
67     input wire i_rf_ready,
68     output wire o_rf_rd_en;
69
70     reg init_done;
71     wire misalign_trap_sync;
72
73     reg [4:2] o_cnt;
74     wire [3:0] cnt_r;
75
76     reg ibus_cyc;
77     //Update PC in RUN or TRAP states
78     assign o_ctrl_pc_en = o_cnt_en & !o_init;

```

```

77
78 assign o_mem_bytecnt = o_cnt[4:3];
79
80 assign o_cnt0to3 = (o_cnt[4:2] == 3'd0);
81 assign o_cnt12to31 = (o_cnt[4] | (o_cnt[3:2] == 2'b11));
82 assign o_cnt0 = (o_cnt[4:2] == 3'd0) & cnt_r[0];
83 assign o_cnt1 = (o_cnt[4:2] == 3'd0) & cnt_r[1];
84 assign o_cnt2 = (o_cnt[4:2] == 3'd0) & cnt_r[2];
85 assign o_cnt3 = (o_cnt[4:2] == 3'd0) & cnt_r[3];
86 assign o_cnt7 = (o_cnt[4:2] == 3'd1) & cnt_r[3];
87 assign o_cnt11 = (o_cnt[4:2] == 3'd2) & cnt_r[3];
88 assign o_cnt12 = (o_cnt[4:2] == 3'd3) & cnt_r[0];
89
90 //Take branch for jump or branch instructions (opcode ==
1x0xx) if
91 //a) It's an unconditional branch (opcode[0] == 1)
92 //b) It's a conditional branch (opcode[0] == 0) of type
beq,blt,bltu (funct3[0] == 0) and ALU compare is true
93 //c) It's a conditional branch (opcode[0] == 0) of type
bne,bge,bgeu (funct3[0] == 1) and ALU compare is false
94 //Only valid during the last cycle of INIT, when the branch
condition has
95 //been calculated.
96 wire take_branch = i_branch_op & (!i_cond_branch |
(i_alu_cmp^i_bne_or_bge));
97
98 wire last_init = o_cnt_done & o_init;
99
100 //valid signal for mdu
101 assign o_mdu_valid = MDU & !o_cnt_en & init_done &
i_mdu_op;
102
103 //trap_pending is only guaranteed to have correct value
during the
104 // last cycle of the init stage
105 wiretrap_pending = WITH_CSR & ((take_branch & i_c-
trl_misalign & !ALIGN) |
106                                     (i_dbus_en &
i_mem_misalign));
107
108 //Prepare RF for writes when everything is ready to enter
stage two
109 // and the first stage didn't cause a misalign exception
110 //Left shifts, SLT & Branch ops. First cycle after init
111 //Right shift. o_sh_done
112 //Mem ops. i_dbus_ack
113 //MDU ops. i_mdu_ready
114 assign o_rf_wreq = (i_shift_op & (i_sh_right ? (i_sh_done &
(last_init | !o_cnt_en & init_done)) : last_init)) |
115 i_dbus_ack | (MDU & i_mdu_ready) |
116 (i_branch_op & (last_init & !trap_pend-
ing)) |
117 (i_rd_alu_en & i_alu_rd_sel1 & last_in-
it);
118
119 assign o_dbus_cyc = !o_cnt_en & init_done & i_dbus_en & !
i_mem_misalign;
120
121 //Prepare RF for reads when a new instruction is fetched
122 // or when stage one caused an exception (rreq implies a
write request too)
123 assign o_rf_rreq = i_ibus_ack | (trap_pending & last_init);
124
125 assign o_rf_rd_en = i_rd_op & !o_init;
126
127 /*
128 bufreg is used during mem, branch, and shift operations
129
130 mem : bufreg is used for dbus address. Shift in data dur-
ing phase 1.
131
132 Shift out during phase 2 if there was a misalignment
exception.
133
134 branch : Shift in during phase 1. Shift out during phase 2
135
136 shift : Shift in during phase 1. Continue shifting between
phases (except
137 for the first cycle after init). Shift out during
phase 2
138 */
139 assign o_bufreg_en = (o_cnt_en & (o_init | ((o_ctrl_trap |
i_branch_op) & i_two_stage_op))) | (i_shift_op & init_done &
(i_sh_right | i_sh_done));
140
141 assign o_ibus_cyc = ibus_cyc & !i_rst;
142
143 assign o_init = i_two_stage_op & !i_new_irq & !init_done;
144
145 assign o_cnt_done = (o_cnt[4:2] == 3'b111) & cnt_r[3];
146
147 always @(posedge i_clk) begin
148 //ibus_cyc changes on three conditions.
149 //1. i_rst is asserted. Together with the async gating
above, o_ibus_cyc
150 // will be asserted as soon as the reset is released.
This is how the

```

```

151 // first instruction is fetched
152 //2. o_cnt_done and o_ctrl_pc_en are asserted. This
means that SERV just
153 // finished updating the PC, is done with the current
instruction and
154 // o_ibus_cyc gets asserted to fetch a new instruction
155 //3. When i_ibus_ack, a new instruction is fetched and
o_ibus_cyc gets
156 // deasserted to finish the transaction
157 if (i_ibus_ack | o_cnt_done | i_rst)
158     ibus_cyc <= o_ctrl_pc_en | i_rst;
159
160 if (o_cnt_done) begin
161     init_done <= o_init & !init_done;
162     o_ctrl_jump <= o_init & take_branch;
163 end
164
165 if (i_rst) begin
166     if (RESET_STRATEGY != "NONE") begin
167         init_done <= 1'b0;
168         o_ctrl_jump <= 1'b0;
169     end
170 end
171 end
172
173 generate
174 /*
175     Because SERV is 32-bit bit-serial we need a counter
than can count 0-31
176
177 to keep track of which bit we are currently processing.
o_cnt and cnt_r
178 are used together to create such a counter.
179 The top three bits (o_cnt) are implemented as a normal
counter, but
180
181 instead of the two LSB, cnt_r is a 4-bit shift register
which loops 0-3
182 When cnt_r[3] is 1, o_cnt will be increased.
183
184 The counting starts when the core is idle and the
i_rf_ready signal
185 comes in from the RF module by shifting in the
i_rf_ready bit as LSB of
186 the shift register. Counting is stopped by using
o_cnt_done to block the
187 bit that was supposed to be shifted into bit 0 of
cnt_r.
188
189 There are two benefit of doing the counter this way
190
191 1. We only need to check four bits instead of five when
we want to check
192 if the counter is at a certain value. For 4-LUT archi-
tectures this means
193 we only need one LUT instead of two for each comparis-
on.
194 2. We don't need a separate enable signal to turn on
and off the counter
195 between stages, which saves an extra FF and a unique
control signal. We
196 just need to check if cnt_r is not zero to see if the
counter is
197 currently running
198 */
199 if (W == 1) begin : gen_cnt_w_eq_1
200     reg [3:0] cnt_lsb;
201     always @(posedge i_clk) begin
202         o_cnt <= o_cnt + {2'd0, cnt_r[3]};
203         cnt_lsb <= {cnt_lsb[2:0], cnt_lsb[3] & !
o_cnt_done} | i_rf_ready;
204         if (i_rst & (RESET_STRATEGY != "NONE")) begin
205             o_cnt <= 3'd0;
206             cnt_lsb <= 4'b0000;
207         end
208     end
209     assign cnt_r = cnt_lsb;
210     assign o_cnt_en = !cnt_lsb;
211 end else if (W == 4) begin : gen_cnt_w_eq_4
212     reg cnt_en;
213     always @(posedge i_clk) begin
214         if (i_rf_ready) cnt_en <= 1; else
215         if (o_cnt_done) cnt_en <= 0;
216         o_cnt <= o_cnt + {2'd0, cnt_en};
217         if (i_rst & (RESET_STRATEGY != "NONE")) begin
218             o_cnt <= 3'd0;
219             cnt_en <= 1'b0;
220         end
221     end
222     assign cnt_r = 4'b1111;
223     assign o_cnt_en = cnt_en;
224 end
225 endgenerate
226
227 assign o_ctrl_trap = WITH_CSR & (i_e_op | i_new_irq | mis-
align_trap_sync);
228
229 generate
230 if (WITH_CSR) begin : gen_csr
231     reg misalign_trap_sync_r;
232
233

```

```

230     always @(posedge i_clk) begin
231         if (i_ibus_ack | o_cnt_done | i_rst)
232             misalign_trap_sync_r <= !(i_ibus_ack | i_rst) &
((trap_pending & o_init) | misalign_trap_sync_r);
233     end
234     assign misalign_trap_sync = misalign_trap_sync_r;
235 end else begin : gen_no_csr
236     assign misalign_trap_sync = 1'b0;
237 end
238 endgenerate
239 endmodule

```

17. olofk_serv/rtl/serv_synth_wrapper.v (138 lines)

```

1 /*
2  * serv_synth_wrapper.v : SERV wrapper for synthesis
3  *
4  * SPDX-FileCopyrightText: 2021 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8
9 module serv_synth_wrapper
10 #()
11 /* Register signals before or after the decoder
12 0 : Register after the decoder. Faster but uses more re-
sources
13 1 : (default) Register before the decoder. Slower but
uses less resources
14 */
15 parameter PRE_REGISTER = 1,
16 /* Amount of reset applied to design
17 "NONE" : No reset at all. Relies on a POR to set cor-
rect initialization
18 values and that core isn't reset during
runtime
19 "MINI" : Standard setting. Resets the minimal amount of
FFs needed to
20 restart execution from the instruction at RE-
SET_PC
21 */
22 parameter RESET_STRATEGY = "MINI",
23 parameter WITH_CSR = 1,
24 parameter RF_WIDTH = 2,
25 parameter RF_L2D = $clog2((32+(WITH_CSR*4))*32/
RF_WIDTH))
26 (
27 input wire clk,
28 input wire i_rst,
29 input wire i_timer_irq,
30 output wire [31:0] o_ibus_adr,
31 output wire o_ibus_cyc,
32 input wire [31:0] i_ibus_rdt,
33 input wire i_ibus_ack,
34 output wire [31:0] o_dbus_adr,
35 output wire [31:0] o_dbus_dat,
36 output wire [3:0] o_dbus_sel,
37 output wire o_dbus_we,
38 output wire o_dbus_cyc,
39 input wire [31:0] i_dbus_rdt,
40 input wire i_dbus_ack,
41
42 output wire [RF_L2D-1:0] o_waddr,
43 output wire [RF_WIDTH-1:0] o_wdata,
44 output wire o_wen,
45 output wire [RF_L2D-1:0] o_raddr,
46 input wire [RF_WIDTH-1:0] i_rdata);
47
48 localparam CSR_REGS = WITH_CSR*4;
49
50 wire rf_wreq;
51 wire rf_rreq;
52 wire [4+WITH_CSR:0] wreg0;
53 wire [4+WITH_CSR:0] wreg1;
54 wire wen0;
55 wire wen1;
56 wire wdata0;
57 wire wdata1;
58 wire [4+WITH_CSR:0] rreg0;
59 wire [4+WITH_CSR:0] rreg1;
60 wire rf_ready;
61 wire rdata0;
62 wire rdata1;
63
64 serv_rf_ram_if
65 #(.width (RF_WIDTH),
66 .reset_strategy (RESET_STRATEGY),
67 .csr_regs (CSR_REGS))
68 rf_ram_if
69 (.i_clk (clk),
70 .i_rst (i_rst),
71 .i_wreq (rf_wreq),
72 .i_rreq (rf_rreq),
73 .o_ready (rf_ready),
74 .i_wreg0 (wreg0),
75 .i_wreg1 (wreg1),
76 .i_wen0 (wen0),
77 .i_wen1 (wen1),

```

```

78 .i_wdata0 (wdata0),
79 .i_wdata1 (wdata1),
80 .i_rreg0 (rreg0),
81 .i_rreg1 (rreg1),
82 .o_rdata0 (rdata0),
83 .o_rdata1 (rdata1),
84 .o_waddr (o_waddr),
85 .o_wdata (o_wdata),
86 .o_wen (o_wen),
87 .o_raddr (o_raddr),
88 .i_rdata (i_rdata));
89
90 serv_top
91 #(.RESET_PC (32'd0),
92 .PRE_REGISTER (PRE_REGISTER),
93 .RESET_STRATEGY (RESET_STRATEGY),
94 .WITH_CSR (WITH_CSR),
95 .MDU(1'b0))
96 cpu
97 (
98 .clk (clk),
99 .i_rst (i_rst),
100 .i_timer_irq (i_timer_irq),
101 .o_rf_rreq (rf_rreq),
102 .o_rf_wreq (rf_wreq),
103 .i_rf_ready (rf_ready),
104 .o_wreg0 (wreg0),
105 .o_wreg1 (wreg1),
106 .o_wen0 (wen0),
107 .o_wen1 (wen1),
108 .o_wdata0 (wdata0),
109 .o_wdata1 (wdata1),
110 .o_rreg0 (rreg0),
111 .o_rreg1 (rreg1),
112 .i_rdata0 (rdata0),
113 .i_rdata1 (rdata1),
114
115 .o_ibus_adr (o_ibus_adr),
116 .o_ibus_cyc (o_ibus_cyc),
117 .i_ibus_rdt (i_ibus_rdt),
118 .i_ibus_ack (i_ibus_ack),
119
120 .o_dbus_adr (o_dbus_adr),
121 .o_dbus_dat (o_dbus_dat),
122 .o_dbus_sel (o_dbus_sel),
123 .o_dbus_we (o_dbus_we),
124 .o_dbus_cyc (o_dbus_cyc),
125 .i_dbus_rdt (i_dbus_rdt),
126 .i_dbus_ack (i_dbus_ack),
127
128 //Extension
129 .o_ext_func3 (),
130 .i_ext_ready (1'b0),
131 .i_ext_rd (32'd0),
132 .o_ext_rs1 (),
133 .o_ext_rs2 (),
134 //MDU
135 .o_mdu_valid ());
136
137 endmodule
138 `default_nettype wire

```

18. olofk_serv/rtl/serv_top.v (674 lines)

```

1 /*
2  * serv_top.v : SERV toplevel
3  *
4  * SPDX-FileCopyrightText: 2018 Olof Kindgren <olof@award-win-
ning.me>
5  * SPDX-License-Identifier: ISC
6  */
7 `default_nettype none
8
9 module serv_top
10 #(parameter WITH_CSR = 1,
11 parameter W = 1,
12 parameter B = W-1,
13 parameter PRE_REGISTER = 1,
14 parameter RESET_STRATEGY = "MINI",
15 parameter RESET_PC = 32'd0,
16 parameter [0:0] DEBUG = 1'b0,
17 parameter [0:0] MDU = 1'b0,
18 parameter [0:0] COMPRESSED=0,
19 parameter [0:0] ALIGN = COMPRESSED)
20 (
21 input wire clk,
22 input wire i_rst,
23 input wire i_timer_irq,
24 `ifdef RISC_V_FORMAL
25 output wire rvfi_valid,
26 output wire [63:0] rvfi_order,
27 output wire [31:0] rvfi_insn,
28 output wire rvfi_trap,
29 output wire rvfi_halt,
30 output wire rvfi_intr,
31 output wire [1:0] rvfi_mode,
32 output wire [1:0] rvfi_ixl,
33 output wire [4:0] rvfi_rs1_addr,
34 output wire [4:0] rvfi_rs2_addr,
35 output wire [31:0] rvfi_rs1_rdata,

```

```

36 output wire [31:0] rvfi_rs2_rdata,
37 output wire [4:0] rvfi_rd_addr,
38 output wire [31:0] rvfi_rd_wdata,
39 output wire [31:0] rvfi_pc_rdata,
40 output wire [31:0] rvfi_pc_wdata,
41 output wire [31:0] rvfi_mem_addr,
42 output wire [3:0] rvfi_mem_rmask,
43 output wire [3:0] rvfi_mem_wmask,
44 output wire [31:0] rvfi_mem_rdata,
45 output wire [31:0] rvfi_mem_wdata,
46 `endif
47 //RF Interface
48 output wire o_rf_rreq,
49 output wire o_rf_wreq,
50 input wire i_rf_ready,
51 output wire [4+WITH_CSR:0] o_wreg0,
52 output wire [4+WITH_CSR:0] o_wreg1,
53 output wire o_wen0,
54 output wire o_wen1,
55 output wire [B:0] o_wdata0,
56 output wire [B:0] o_wdata1,
57 output wire [4+WITH_CSR:0] o_rreg0,
58 output wire [4+WITH_CSR:0] o_rreg1,
59 input wire [B:0] i_rdata0,
60 input wire [B:0] i_rdata1,
61
62 output wire [31:0] o_ibus_adr,
63 output wire o_ibus_cyc,
64 input wire [31:0] i_ibus_rdt,
65 input wire i_ibus_ack,
66 output wire [31:0] o_dbus_adr,
67 output wire [31:0] o_dbus_dat,
68 output wire [3:0] o_dbus_sel,
69 output wire o_dbus_we,
70 output wire o_dbus_cyc,
71 input wire [31:0] i_dbus_rdt,
72 input wire i_dbus_ack,
73 //Extension
74 output wire [2:0] o_ext_funct3,
75 input wire i_ext_ready,
76 input wire [31:0] i_ext_rd,
77 output wire [31:0] o_ext_rs1,
78 output wire [31:0] o_ext_rs2,
79 //MDU
80 output wire o_mdu_valid);
81
82 wire [4:0] rd_addr;
83 wire [4:0] rs1_addr;
84 wire [4:0] rs2_addr;
85
86 wire [3:0] immdec_ctrl;
87 wire [3:0] immdec_en;
88
89 wire sh_right;
90 wire bne_or_bge;
91 wire cond_branch;
92 wire two_stage_op;
93 wire e_op;
94 wire ebreak;
95 wire branch_op;
96 wire shift_op;
97 wire rd_op;
98 wire mdu_op;
99
100 wire rd_alu_en;
101 wire rd_csr_en;
102 wire rd_mem_en;
103 wire [B:0] ctrl_rd;
104 wire [B:0] alu_rd;
105 wire [B:0] mem_rd;
106 wire [B:0] csr_rd;
107 wire mtval_pc;
108
109 wire ctrl_pc_en;
110 wire jump;
111 wire jal_or_jalr;
112 wire utype;
113 wire mret;
114 wire [B:0] imm;
115 wire trap;
116 wire pc_rel;
117 wire iscomp;
118
119 wire init;
120 wire cnt_en;
121 wire cnt0to3;
122 wire cnt12to31;
123 wire cnt0;
124 wire cnt1;
125 wire cnt2;
126 wire cnt3;
127 wire cnt7;
128 wire cnt11;
129 wire cnt12;
130
131 wire cnt_done;
132
133 wire bufreg_en;
134 wire bufreg_sh_signed;
135 wire bufreg_rs1_en;
136 wire bufreg_imm_en;
137 wire bufreg_clr_lsb;
138 wire [B:0] bufreg_q;
139 wire [B:0] bufreg2_q;
140 wire [31:0] dbus_rdt;
141 wire dbus_ack;
142
143 wire alu_sub;
144 wire [1:0] alu_bool_op;
145 wire alu_cmp_eq;
146 wire alu_cmp_sig;
147 wire alu_cmp;
148 wire [2:0] alu_rd_sel;
149
150 wire [B:0] rs1;
151 wire [B:0] rs2;
152 wire rd_en;
153
154 wire [B:0] op_b;
155 wire op_b_sel;
156
157 wire mem_signed;
158 wire mem_word;
159 wire mem_half;
160 wire [1:0] mem_bytecnt;
161 wire sh_done;
162
163 wire mem_misalign;
164
165 wire [B:0] bad_pc;
166
167 wire csr_mstatus_en;
168 wire csr_mie_en;
169 wire csr_mcause_en;
170 wire [1:0] csr_source;
171 wire [B:0] csr_imm;
172 wire csr_d_sel;
173 wire csr_en;
174 wire [1:0] csr_addr;
175 wire [B:0] csr_pc;
176 wire csr_imm_en;
177 wire [B:0] csr_in;
178 wire [B:0] rf_csr_out;
179 wire dbus_en;
180
181 wire new_irq;
182
183 wire [1:0] lsb;
184
185 wire [31:0] i_wb_rdt;
186
187 wire [31:0] wb_ibus_adr;
188 wire wb_ibus_cyc;
189 wire [31:0] wb_ibus_rdt;
190 wire wb_ibus_ack;
191
192 generate
193 if (ALIGN) begin : gen_align
194 serv_aligner align
195 (
196 .clk(clk),
197 .rst(i_rst),
198 // serv_rf_top
199 .i_ibus_adr(wb_ibus_adr),
200 .i_ibus_cyc(wb_ibus_cyc),
201 .o_ibus_rdt(wb_ibus_rdt),
202 .o_ibus_ack(wb_ibus_ack),
203 // servant_arbiter
204 .o_wb_ibus_adr(o_ibus_adr),
205 .o_wb_ibus_cyc(o_ibus_cyc),
206 .i_wb_ibus_rdt(i_ibus_rdt),
207 .i_wb_ibus_ack(i_ibus_ack));
208 end else begin : gen_no_align
209 assign o_ibus_adr = wb_ibus_adr;
210 assign o_ibus_cyc = wb_ibus_cyc;
211 assign wb_ibus_rdt = i_ibus_rdt;
212 assign wb_ibus_ack = i_ibus_ack;
213 end
214 endgenerate
215
216 generate
217 if (COMPRESSED) begin : gen_compressed
218 serv_compdec compdec
219 (
220 .i_clk(clk),
221 .i_instr(wb_ibus_rdt),
222 .i_ack(wb_ibus_ack),
223 .o_instr(i_wb_rdt),
224 .o_iscomp(iscomp));
225 end else begin : gen_no_compressed
226 assign i_wb_rdt = wb_ibus_rdt;
227 assign iscomp = 1'b0;
228 end
229 endgenerate
230
231 serv_state
232 #(.RESET_STRATEGY (RESET_STRATEGY),
233 .WITH_CSR (WITH_CSR[0:0]),
234 .MDU(MDU),
235 .ALIGN(ALIGN),
236 .W(W))
237 state

```

```

238 (
239   .i_clk (clk),
240   .i_rst      (i_rst),
241   //State
242   .i_new_irq  (new_irq),
243   .i_alu_cmp  (alu_cmp),
244   .o_init     (init),
245   .o_cnt_en   (cnt_en),
246   .o_cnt0to3  (cnt0to3),
247   .o_cnt12to31 (cnt12to31),
248   .o_cnt0     (cnt0),
249   .o_cnt1     (cnt1),
250   .o_cnt2     (cnt2),
251   .o_cnt3     (cnt3),
252   .o_cnt7     (cnt7),
253   .o_cnt11    (cnt11),
254   .o_cnt12    (cnt12),
255   .o_cnt_done  (cnt_done),
256   .o_bufreg_en (bufreg_en),
257   .o_ctrl_pc_en (ctrl_pc_en),
258   .o_ctrl_jump (jump),
259   .o_ctrl_trap (trap),
260   .i_ctrl_misalign (lsb[1]),
261   .i_sh_done      (sh_done),
262   .o_mem_bytecnt  (mem_bytecnt),
263   .i_mem_misalign (mem_misalign),
264   //Control
265   .i_bne_or_bge  (bne_or_bge),
266   .i_cond_branch (cond_branch),
267   .i_dbus_en      (dbus_en),
268   .i_two_stage_op (two_stage_op),
269   .i_branch_op    (branch_op),
270   .i_shift_op     (shift_op),
271   .i_sh_right     (sh_right),
272   .i_alu_rd_sel1  (alu_rd_sel[1]),
273   .i_rd_alu_en    (rd_alu_en),
274   .i_e_op         (e_op),
275   .i_rd_op        (rd_op),
276   //MDU
277   .i_mdu_op       (mdu_op),
278   .o_mdu_valid    (o_mdu_valid),
279   //Extension
280   .i_mdu_ready    (i_ext_ready),
281   //External
282   .o_dbus_cyc     (o_dbus_cyc),
283   .i_dbus_ack     (i_dbus_ack),
284   .o_ibus_cyc     (wb_ibus_cyc),
285   .i_ibus_ack     (wb_ibus_ack),
286   //RF Interface
287   .o_rf_rreq      (o_rf_rreq),
288   .o_rf_wreq      (o_rf_wreq),
289   .i_rf_ready     (i_rf_ready),
290   .o_rf_rd_en     (rd_en));
291
292 serv_decode
293   #(.PRE_REGISTER (PRE_REGISTER),
294     .MDU(MDU))
295 decode
296 (
297   .clk (clk),
298   //Input
299   .i_wb_rdt      (i_wb_rdt[31:2]),
300   .i_wb_en       (wb_ibus_ack),
301   //To state
302   .o_bne_or_bge  (bne_or_bge),
303   .o_cond_branch (cond_branch),
304   .o_dbus_en      (dbus_en),
305   .o_e_op         (e_op),
306   .o_ebreak      (ebreak),
307   .o_branch_op   (branch_op),
308   .o_shift_op    (shift_op),
309   .o_rd_op       (rd_op),
310   .o_sh_right    (sh_right),
311   .o_mdu_op      (mdu_op),
312   .o_two_stage_op (two_stage_op),
313   //Extension
314   .o_ext_funct3  (o_ext_funct3),
315
316   //To bufreg
317   .o_bufreg_rs1_en (bufreg_rs1_en),
318   .o_bufreg_imm_en (bufreg_imm_en),
319   .o_bufreg_clr_lsb (bufreg_clr_lsb),
320   .o_bufreg_sh_signed (bufreg_sh_signed),
321   //To bufreg2
322   .o_op_b_source  (op_b_sel),
323   //To ctrl
324   .o_ctrl_jal_or_jalr (jal_or_jalr),
325   .o_ctrl_utyype      (utyype),
326   .o_ctrl_pc_rel      (pc_rel),
327   .o_ctrl_mret        (mret),
328   //To alu
329   .o_alu_sub      (alu_sub),
330   .o_alu_bool_op  (alu_bool_op),
331   .o_alu_cmp_eq    (alu_cmp_eq),
332   .o_alu_cmp_sig    (alu_cmp_sig),
333   .o_alu_rd_sel    (alu_rd_sel),
334   //To mem IF
335   .o_mem_cmd       (o_dbus_we),
336   .o_mem_signed    (mem_signed),
337   .o_mem_word      (mem_word),
338   .o_mem_half      (mem_half),

```

```

339 //To CSR
340 .o_csr_en      (csr_en),
341 .o_csr_addr    (csr_addr),
342 .o_csr_mstatus_en (csr_mstatus_en),
343 .o_csr_mie_en  (csr_mie_en),
344 .o_csr_mcause_en (csr_mcause_en),
345 .o_csr_source  (csr_source),
346 .o_csr_d_sel   (csr_d_sel),
347 .o_csr_imm_en  (csr_imm_en),
348 .o_mtval_pc    (mtval_pc),
349 //To top
350 .o_immdec_ctrl (immdec_ctrl),
351 .o_immdec_en   (immdec_en),
352 //To RF IF
353 .o_rd_mem_en   (rd_mem_en),
354 .o_rd_csr_en   (rd_csr_en),
355 .o_rd_alu_en   (rd_alu_en));
356
357 serv_immdec #(.W (W)) immdec
358 (
359   .i_clk      (clk),
360   //State
361   .i_cnt_en   (cnt_en),
362   .i_cnt_done (cnt_done),
363   //Control
364   .i_immdec_en (immdec_en),
365   .i_csr_imm_en (csr_imm_en),
366   .i_ctrl      (immdec_ctrl),
367   .o_rd_addr   (rd_addr),
368   .o_rs1_addr  (rs1_addr),
369   .o_rs2_addr  (rs2_addr),
370   //Data
371   .o_csr_imm   (csr_imm),
372   .o_imm       (imm),
373   //External
374   .i_wb_en     (wb_ibus_ack),
375   .i_wb_rdt    (i_wb_rdt[31:7]));
376
377 serv_bufreg
378   #(.MDU(MDU),
379     .W(W))
380 bufreg
381 (
382   .i_clk      (clk),
383   //State
384   .i_cnt0     (cnt0),
385   .i_cnt1     (cnt1),
386   .i_cnt_done (cnt_done),
387   .i_en       (bufreg_en),
388   .i_init     (init),
389   .i_mdu_op   (mdu_op),
390   .o_lsb      (lsb),
391   //Control
392   .i_sh_signed (bufreg_sh_signed),
393   .i_rs1_en   (bufreg_rs1_en),
394   .i_imm_en   (bufreg_imm_en),
395   .i_clr_lsb  (bufreg_clr_lsb),
396   .i_shift_op (shift_op),
397   .i_right_shift_op (sh_right),
398   .i_shamt    (o_dbus_dat[26:24]),
399   //Data
400   .i_rs1      (rs1),
401   .i_imm      (imm),
402   .o_q        (bufreg_q),
403   //External
404   .o_dbus_adr (o_dbus_adr),
405   .o_ext_rs1  (o_ext_rs1));
406
407 serv_bufreg2 #(.W(W)) bufreg2
408 (
409   .i_clk      (clk),
410   //State
411   .i_en       (cnt_en),
412   .i_init     (init),
413   .i_cnt7     (cnt7),
414   .i_cnt_done (cnt_done),
415   .i_sh_right (sh_right),
416   .i_lsb      (lsb),
417   .i_bytecnt  (mem_bytecnt),
418   .o_sh_done  (sh_done),
419   //Control
420   .i_op_b_sel (op_b_sel),
421   .i_shift_op (shift_op),
422   //Data
423   .i_rs2      (rs2),
424   .i_imm      (imm),
425   .o_op_b     (op_b),
426   .o_q        (bufreg2_q),
427   //External
428   .o_dat      (o_dbus_dat),
429   .i_load     (dbus_ack),
430   .i_dat      (dbus_rdt));
431
432 serv_ctrl
433   #(.RESET_PC (RESET_PC),
434     .RESET_STRATEGY (RESET_STRATEGY),
435     .WITH_CSR (WITH_CSR),
436     .W (W))
437 ctrl
438 (
439   .clk      (clk),

```



```

440     .i_rst      (i_rst),
441     //State
442     .i_pc_en    (ctrl_pc_en),
443     .i_cnt12to31 (cnt12to31),
444     .i_cnt0     (cnt0),
445     .i_cnt1     (cnt1),
446     .i_cnt2     (cnt2),
447     //Control
448     .i_jump     (jump),
449     .i_jal_or_jalr (jal_or_jalr),
450     .i_utyype    (utyype),
451     .i_pc_rel    (pc_rel),
452     .i_trap     (trap | mret),
453     .i_iscomp    (iscomp),
454     //Data
455     .i_imm       (imm),
456     .i_buf       (bufreg_q),
457     .i_csr_pc    (csr_pc),
458     .o_rd        (ctrl_rd),
459     .o_bad_pc    (bad_pc),
460     //External
461     .o_ibus_adr  (wb_ibus_adr));
462
463     serv_alu #(.W (W)) alu
464     (
465         .clk      (clk),
466         //State
467         .i_en     (cnt_en),
468         .i_cnt0   (cnt0),
469         .o_cmp    (alu_cmp),
470         //Control
471         .i_sub    (alu_sub),
472         .i_bool_op (alu_bool_op),
473         .i_cmp_eq  (alu_cmp_eq),
474         .i_cmp_sig (alu_cmp_sig),
475         .i_rd_sel  (alu_rd_sel),
476         //Data
477         .i_rs1    (rs1),
478         .i_op_b   (op_b),
479         .i_buf    (bufreg_q),
480         .o_rd     (alu_rd));
481
482     serv_rf_if
483     #(.WITH_CSR (WITH_CSR), .W(W))
484     rf_if
485     (//RF interface
486         .i_cnt_en (cnt_en),
487         .o_wreg0  (o_wreg0),
488         .o_wreg1  (o_wreg1),
489         .o_wen0   (o_wen0),
490         .o_wen1   (o_wen1),
491         .o_wdata0 (o_wdata0),
492         .o_wdata1 (o_wdata1),
493         .o_rreg0  (o_rreg0),
494         .o_rreg1  (o_rreg1),
495         .i_rdata0 (i_rdata0),
496         .i_rdata1 (i_rdata1),
497
498         //Trap interface
499         .i_trap (trap),
500         .i_mret (mret),
501         .i_mepc (wb_ibus_adr[B:0]),
502         .i_mtval_pc (mtval_pc),
503         .i_bufreg_q (bufreg_q),
504         .i_bad_pc (bad_pc),
505         .o_csr_pc (csr_pc),
506         //CSR write port
507         .i_csr_en (csr_en),
508         .i_csr_addr (csr_addr),
509         .i_csr (csr_in),
510         //RD write port
511         .i_rd_wen (rd_en),
512         .i_rd_waddr (rd_addr),
513         .i_ctrl_rd (ctrl_rd),
514         .i_alu_rd (alu_rd),
515         .i_rd_alu_en (rd_alu_en),
516         .i_csr_rd (csr_rd),
517         .i_rd_csr_en (rd_csr_en),
518         .i_mem_rd (mem_rd),
519         .i_rd_mem_en (rd_mem_en),
520
521         //RS1 read port
522         .i_rs1_raddr (rs1_addr),
523         .o_rs1 (rs1),
524         //RS2 read port
525         .i_rs2_raddr (rs2_addr),
526         .o_rs2 (rs2),
527
528         //CSR read port
529         .o_csr (rf_csr_out));
530
531     serv_mem_if
532     #(.WITH_CSR (WITH_CSR[0:0]),
533         .W (W))
534     mem_if
535     (
536         .i_clk      (clk),
537         //State
538         .i_bytecnt  (mem_bytecnt),
539         .i_lsb      (lsb),
540         .o_misalign (mem_misalign),
541
542         //Control
543         .i_mdu_op    (mdu_op),
544         .i_signed    (mem_signed),
545         .i_word      (mem_word),
546         .i_half      (mem_half),
547         //Data
548         .i_bufreg2_q (bufreg2_q),
549         .o_rd        (mem_rd),
550         //External interface
551         .o_wb_sel    (o_dbus_sel));
552
553     generate
554     if (!WITH_CSR) begin : gen_csr
555         serv_csr
556         #(.RESET_STRATEGY (RESET_STRATEGY),
557             .W(W))
558         csr
559         (
560             .i_clk      (clk),
561             .i_rst      (i_rst),
562             //State
563             .i_trig_irq (wb_ibus_ack),
564             .i_en       (cnt_en),
565             .i_cnt0to3  (cnt0to3),
566             .i_cnt3     (cnt3),
567             .i_cnt7     (cnt7),
568             .i_cnt11    (cnt11),
569             .i_cnt12    (cnt12),
570             .i_cnt_done (cnt_done),
571             .i_mem_op   (!mtval_pc),
572             .i_mtip     (i_timer_irq),
573             .i_trap     (trap),
574             .o_new_irq  (new_irq),
575             //Control
576             .i_e_op     (e_op),
577             .i_ebreak   (ebreak),
578             .i_mem_cmd  (o_dbus_we),
579             .i_mstatus_en (csr_mstatus_en),
580             .i_mie_en   (csr_mie_en),
581             .i_mcause_en (csr_mcause_en),
582             .i_csr_source (csr_source),
583             .i_mret     (mret),
584             .i_csr_d_sel (csr_d_sel),
585             //Data
586             .i_rf_csr_out (rf_csr_out),
587             .o_csr_in    (csr_in),
588             .i_csr_imm   (csr_imm),
589             .i_rs1       (rs1),
590             .o_q         (csr_rd));
591         end else begin : gen_no_csr
592             assign csr_in = {W[1'b0]};
593             assign csr_rd = {W[1'b0]};
594             assign new_irq = 1'b0;
595         end
596     endgenerate
597
598     generate
599     if (DEBUG) begin : gen_debug
600         serv_debug #(.W (W), .RESET_PC (RESET_PC)) debug
601         (
602             //RISC-V FORMAL
603             .rvfi_valid (rvfi_valid),
604             .rvfi_order (rvfi_order),
605             .rvfi_insn  (rvfi_insn),
606             .rvfi_trap  (rvfi_trap),
607             .rvfi_halt   (rvfi_halt),
608             .rvfi_intr   (rvfi_intr),
609             .rvfi_mode   (rvfi_mode),
610             .rvfi_ixl    (rvfi_ixl),
611             .rvfi_rs1_addr (rvfi_rs1_addr),
612             .rvfi_rs2_addr (rvfi_rs2_addr),
613             .rvfi_rs1_rdata (rvfi_rs1_rdata),
614             .rvfi_rs2_rdata (rvfi_rs2_rdata),
615             .rvfi_rd_addr  (rvfi_rd_addr),
616             .rvfi_rd_wdata (rvfi_rd_wdata),
617             .rvfi_pc_rdata (rvfi_pc_rdata),
618             .rvfi_pc_wdata (rvfi_pc_wdata),
619             .rvfi_mem_addr (rvfi_mem_addr),
620             .rvfi_mem_rmask (rvfi_mem_rmask),
621             .rvfi_mem_wmask (rvfi_mem_wmask),
622             .rvfi_mem_rdata (rvfi_mem_rdata),
623             .rvfi_mem_wdata (rvfi_mem_wdata),
624             .i_dbus_adr     (o_dbus_adr),
625             .i_dbus_dat     (o_dbus_dat),
626             .i_dbus_sel     (o_dbus_sel),
627             .i_dbus_we      (o_dbus_we),
628             .i_dbus_rdt     (i_dbus_rdt),
629             .i_dbus_ack     (i_dbus_ack),
630             .i_ctrl_pc_en   (ctrl_pc_en),
631             .rs1            (rs1),
632             .rs2            (rs2),
633             .rs1_addr       (rs1_addr),
634             .rs2_addr       (rs2_addr),
635             .immdec_en      (immdec_en),
636             .rd_en          (rd_en),
637             .trap           (trap),
638             .i_rf_ready     (i_rf_ready),
639             .i_ibus_cyc     (o_ibus_cyc),
640             .two_stage_op   (two_stage_op),
641             .init           (init),
642             .i_ibus_adr     (o_ibus_adr),

```



```

642 `endif
643     .i_clk          (clk),
644     .i_rst          (i_rst),
645     .i_ibus_rdt     (i_ibus_rdt),
646     .i_ibus_ack     (i_ibus_ack),
647     .i_rd_addr      (rd_addr      ),
648     .i_cnt_en       (cnt_en       ),
649     .i_csr_in       (csr_in       ),
650     .i_csr_mstatus_en (csr_mstatus_en),
651     .i_csr_mie_en   (csr_mie_en   ),
652     .i_csr_mcause_en (csr_mcause_en ),
653     .i_csr_en       (csr_en       ),
654     .i_csr_addr     (csr_addr     ),
655     .i_wen0         (o_wen0),
656     .i_wdata0       (o_wdata0),
657     .i_cnt_done     (cnt_done));
658 end

```

```

659     endgenerate
660
661
662 generate
663     if (MDU) begin: gen_mdu
664         assign dbus_rdt = i_ext_ready ? i_ext_rd:i_dbus_rdt;
665         assign dbus_ack = i_dbus_ack | i_ext_ready;
666     end else begin : gen_no_mdu
667         assign dbus_rdt = i_dbus_rdt;
668         assign dbus_ack = i_dbus_ack;
669     end
670     assign o_ext_rs2 = o_dbus_dat;
671 endgenerate
672
673 endmodule
674 `default_nettype wire

```

형식검증된 AXI/Wishbone 브리지

6 files • 4,175 lines

API — addrdecode.v

Table 1202. ZipCPU_wb2axip/rtl/addrdecode.v — module / interface / package (1개)

이름	종류
addrdecode	module

Table 1203. ZipCPU_wb2axip/rtl/addrdecode.v — parameter (5개)

파라미터	형/폭	기본값
NS		8
AW		32
ACCESS_ALLOWED	[NS-1:0]	-1
OPT_REGISTERED	[0:0]	0
OPT_LOWPOWER	[0:0]	0

Table 1204. ZipCPU_wb2axip/rtl/addrdecode.v — localparam (1개)

상수	형/폭	값
OPT_NONESEL	[0:0]	(!ACCESS_ALLOWED[0])

Table 1205. ZipCPU_wb2axip/rtl/addrdecode.v — 포트 (10개)

포트	방향	형/폭
i_clk	input	wire
i_valid	input	wire
o_stall	output	reg
i_addr	input	wire [AW-1:0]
i_data	input	wire [DW-1:0]
o_valid	output	reg
i_stall	input	wire
o_decode	output	reg [NS:0]
o_addr	output	reg [AW-1:0]
o_data	output	reg [DW-1:0]

API — afifo.v

Table 1206. ZipCPU_wb2axip/rtl/afifo.v — module / interface / package (1개)

이름	종류
afifo	module

Table 1207. ZipCPU_wb2axip/rtl/afifo.v — parameter (5개)

파라미터	형/폭	기본값
LGFIFO		3
WIDTH		16
NFF		2
WRITE_ON_POSEDGE	[0:0]	1'b1
OPT_REGISTER_READS	[0:0]	1'b1

Table 1208. ZipCPU_wb2axip/rtl/afifo.v — localparam (1개)

상수	형/폭	값
MSB		LGFIFO

Table 1209. ZipCPU_wb2axip/rtl/afifo.v — 포트 (6개)

포트	방향	형/폭
i_wclk	input	wire
i_wr_data	input	wire [WIDTH-1:0]
o_wr_full	output	reg
i_rclk	input	wire
o_rd_data	output	reg [WIDTH-1:0]
o_rd_empty	output	reg

Table 1210. ZipCPU_wb2axip/rtl/afifo.v — #define / `define 매크로 (4개)

매크로	인자	정의
ASSERT		assert
ASSUME		assume
ASSERT		assert
ASSUME		assert

API — apbslave.v**Table 1211. ZipCPU_wb2axip/rtl/apbslave.v — module / interface / package (1개)**

이름	종류
apbslave	module

Table 1212. ZipCPU_wb2axip/rtl/apbslave.v — parameter (2개)

파라미터	형/폭	기본값
C_APB_ADDR_WIDTH		12
C_APB_DATA_WIDTH		32

Table 1213. ZipCPU_wb2axip/rtl/apbslave.v — localparam (3개)

상수	형/폭	값
AW		C_APB_ADDR_WIDTH
DW		C_APB_DATA_WIDTH
APBLSB		$\$clog2(C_APB_DATA_WIDTH)-3$

Table 1214. ZipCPU_wb2axip/rtl/apbslave.v — 포트 (11개)

포트	방향	형/폭
PCLK	input	wire
PSEL	input	wire
PENABLE	input	wire
PREADY	output	reg
PADDR	input	wire [AW-1:0]
PWRITE	input	wire
PWDATA	input	wire [DW-1:0]
PWSTRB	input	wire [DW/8-1:0]
PPROT	input	wire [2:0]
PRDATA	output	reg [DW-1:0]
PSLVERR	output	wire

API — apbxclock.v**Table 1215. ZipCPU_wb2axip/rtl/apbxclock.v — module / interface / package (1개)**

이름	종류
apbxclock	module

Table 1216. ZipCPU_wb2axip/rtl/apbxclock.v — parameter (3개)

파라미터	형/폭	기본값
C_APB_ADDR_WIDTH		12
C_APB_DATA_WIDTH		32
OPT_REGISTERED	[0:0]	1'b0

Table 1217. ZipCPU_wb2axip/rtl/apbxclock.v — localparam (3개)

상수	형/폭	값
AW		C_APB_ADDR_WIDTH
DW		C_APB_DATA_WIDTH
F_NCLK		5

Table 1218. ZipCPU_wb2axip/rtl/apbxclock.v — 포트 (23개)

포트	방향	형/폭
S_APB_PCLK	input	wire

포트	방향	형/폭
S_APB_PSEL	input	wire
S_APB_PENABLE	input	wire
S_APB_PREADY	output	reg
S_APB_PADDR	input	wire [AW-1:0]
S_APB_PWRITE	input	wire
S_APB_PWDATA	input	wire [DW-1:0]
S_APB_PWSTRB	input	wire [DW/8-1:0]
S_APB_PPROT	input	wire [2:0]
S_APB_PRDATA	output	wire [DW-1:0]
S_APB_PSLVERR	output	wire
M_APB_PCLK	input	wire
M_PRESETn	output	reg
M_APB_PSEL	output	reg
M_APB_PENABLE	output	reg
M_APB_PREADY	input	wire
M_APB_PADDR	output	wire [AW-1:0]
M_APB_PWRITE	output	wire
M_APB_PWDATA	output	wire [DW-1:0]
M_APB_PWSTRB	output	wire [DW/8-1:0]
M_APB_PPROT	output	wire [2:0]
M_APB_PRDATA	input	wire [DW-1:0]
M_APB_PSLVERR	input	wire

API — axi2axi3.v

Table 1219. ZipCPU_wb2axip/rtl/axi2axi3.v — module / interface / package (1개)

이름	종류
axi2axi3	module

Table 1220. ZipCPU_wb2axip/rtl/axi2axi3.v — parameter (4개)

파라미터	형/폭	기본값
C_AXI_ID_WIDTH		1
C_AXI_ADDR_WIDTH		32
C_AXI_DATA_WIDTH		32
LGIFIFO		8

Table 1221. ZipCPU_wb2axip/rtl/axi2axi3.v — localparam (4개)

상수	형/폭	값
ADDRLSB		$\$clog2(C_AXI_DATA_WIDTH)-3$
OPT_LOWPOWER	[0:0]	1'b0
LGWFIFO		4
NID		$(1 << C_AXI_ID_WIDTH)$

Table 1222. ZipCPU_wb2axip/rtl/axi2axi3.v — 포트 (77개) — 앞 60개만 표시(전체 77개, 본문 소스 참조)

포트	방향	형/폭
S_AXI_ACLK	input	wire
S_AXI_ARESETN	input	wire
S_AXI_AWVALID	input	reg
S_AXI_AWREADY	output	wire
S_AXI_AWID	input	reg [C_AXI_ID_WIDTH-1:0]
S_AXI_AWADDR	input	reg [C_AXI_ADDR_WIDTH-1:0]
S_AXI_AWLEN	input	reg [7:0]
S_AXI_AWSIZE	input	reg [2:0]
S_AXI_AWBURST	input	reg [1:0]
S_AXI_AWLOCK	input	reg
S_AXI_AWCACHE	input	reg [3:0]
S_AXI_AWPROT	input	reg [2:0]
S_AXI_AWQOS	input	reg [3:0]
S_AXI_WVALID	input	wire
S_AXI_WREADY	output	wire
S_AXI_WDATA	input	wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_WSTRB	input	wire [C_AXI_DATA_WIDTH/8-1:0]
S_AXI_WLAST	input	wire
S_AXI_BVALID	output	wire
S_AXI_BREADY	input	wire
S_AXI_BID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_BRESP	output	wire [1:0]
S_AXI_ARVALID	input	wire
S_AXI_ARREADY	output	wire
S_AXI_ARID	input	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_ARADDR	input	wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_ARLEN	input	wire [7:0]
S_AXI_ARSIZE	input	wire [2:0]

포트	방향	형/폭
S_AXI_ARBURST	input	wire [1:0]
S_AXI_ARLOCK	input	wire
S_AXI_ARCACHE	input	wire [3:0]
S_AXI_ARPROT	input	wire [2:0]
S_AXI_ARQOS	input	wire [3:0]
S_AXI_RVALID	output	wire
S_AXI_RREADY	input	wire
S_AXI_RID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_RDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_RLAST	output	wire
S_AXI_RRESP	output	wire [1:0]
M_AXI_AWVALID	output	wire
M_AXI_AWREADY	input	wire
M_AXI_AWID	output	wire [C_AXI_ID_WIDTH-1:0]
M_AXI_AWADDR	output	wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWLEN	output	wire [3:0]
M_AXI_AWSIZE	output	wire [2:0]
M_AXI_AWBURST	output	wire [1:0]
M_AXI_AWLOCK	output	wire [1:0]
M_AXI_AWCACHE	output	wire [3:0]
M_AXI_AWPROT	output	wire [2:0]
M_AXI_AWQOS	output	wire [3:0]
M_AXI_WVALID	output	wire
M_AXI_WREADY	input	wire
M_AXI_WID	output	wire [C_AXI_ID_WIDTH-1:0]
M_AXI_WDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
M_AXI_WSTRB	output	wire [C_AXI_DATA_WIDTH/8-1:0]
M_AXI_WLAST	output	wire
M_AXI_BVALID	input	wire
M_AXI_BREADY	output	wire
M_AXI_BID	input	wire [C_AXI_ID_WIDTH-1:0]
M_AXI_BRESP	input	wire [1:0]

API — axi2axilite.v**Table 1223. ZipCPU_wb2axip/rtl/axi2axilite.v — module / interface / package (17#)**

이름	종류
axi2axilite	module

Table 1224. ZipCPU_wb2axip/rtl/axi2axilite.v — parameter (7개)

파라미터	형/폭	기본값
C_AXI_ID_WIDTH	integer	2
C_AXI_DATA_WIDTH	integer	32
C_AXI_ADDR_WIDTH	integer	6
OPT_WRITES	[0:0]	1
OPT_READS	[0:0]	1
OPT_LOWPPOWER	[0:0]	0
LGFIFO		4

Table 1225. ZipCPU_wb2axip/rtl/axi2axilite.v — localparam (5개)

상수	형/폭	값
SLVERR	[1:0]	2'b10
AW		C_AXI_ADDR_WIDTH
DW		C_AXI_DATA_WIDTH
IW		C_AXI_ID_WIDTH
F_LGDEPTH		LGFIFO+1+8

Table 1226. ZipCPU_wb2axip/rtl/axi2axilite.v — 포트 (58개)

포트	방향	형/폭
S_AXI_ACLK	input	wire
S_AXI_ARESETN	input	wire
S_AXI_AWVALID	input	wire
S_AXI_AWREADY	output	wire
S_AXI_AWID	input	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_AWADDR	input	wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_AWLEN	input	wire [7:0]
S_AXI_AWSIZE	input	wire [2:0]
S_AXI_AWBURST	input	wire [1:0]
S_AXI_AWLOCK	input	wire
S_AXI_AWCACHE	input	wire [3:0]
S_AXI_AWPROT	input	wire [2:0]
S_AXI_AWQOS	input	wire [3:0]
S_AXI_WVALID	input	wire

포트	방향	형/폭
S_AXI_WREADY	output	wire
S_AXI_WDATA	input	wire [C_AXI_DATA_WIDTH-1:0]
8	input	wire [(C_AXI_DATA_WIDTH/
S_AXI_WLAST	input	wire
S_AXI_BVALID	output	wire
S_AXI_BREADY	input	wire
S_AXI_BID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_BRESP	output	wire [1:0]
S_AXI_ARVALID	input	wire
S_AXI_ARREADY	output	wire
S_AXI_ARID	input	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_ARADDR	input	wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_ARLEN	input	wire [7:0]
S_AXI_ARSIZE	input	wire [2:0]
S_AXI_ARBURST	input	wire [1:0]
S_AXI_ARLOCK	input	wire
S_AXI_ARCACHE	input	wire [3:0]
S_AXI_ARPROT	input	wire [2:0]
S_AXI_ARQOS	input	wire [3:0]
S_AXI_RVALID	output	wire
S_AXI_RREADY	input	wire
S_AXI_RID	output	wire [C_AXI_ID_WIDTH-1:0]
S_AXI_RDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_RRESP	output	wire [1:0]
S_AXI_RLAST	output	wire
M_AXI_AWADDR	output	wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWPROT	output	wire [2 : 0]
M_AXI_AWVALID	output	wire
M_AXI_AWREADY	input	wire
M_AXI_WDATA	output	wire [C_AXI_DATA_WIDTH-1:0]
8	output	wire [(C_AXI_DATA_WIDTH/
M_AXI_WVALID	output	wire
M_AXI_WREADY	input	wire
M_AXI_BRESP	input	wire [1 : 0]
M_AXI_BVALID	input	wire

포트	방향	형/폭
M_AXI_BREADY	output	wire
M_AXI_ARADDR	output	wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_ARPROT	output	wire [2:0]
M_AXI_ARVALID	output	wire
M_AXI_ARREADY	input	wire
M_AXI_RVALID	input	wire
M_AXI_RREADY	output	wire
M_AXI_RDATA	input	wire [C_AXI_DATA_WIDTH-1 : 0]
M_AXI_RRESP	input	wire [1 : 0]

1. ZipCPU_wb2axip/rtl/addrdecode.v (467 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:   rtl/addrdecode.v
4 // {{{
5 // Project:    WB2AXISP: bus bridges and other odds and ends
6 //
7 // Purpose:    Supports bus crossbars by answering the ques-
8 //             tion, which slave
9 //             does the current address need to be routed to? Re-
10 //             quests are
11 //             pipelined using valid!/stall handshaking. For those
12 //             familiar with
13 //             AXI, READY!=STALL. The outgoing stream is identical to
14 //             the incoming
15 //             one, save for the (new) o_decode field. This is a bit-
16 //             mask containing
17 //             one bit for each slave that the request might be routed
18 //             to, and one
19 //             extra bit to indicate no slaves matched.
20 //             The keys to the operation of this module are found in
21 //             the two
22 //             parameters, SLAVE_ADDR and SLAVE_MASK.
23 //             SLAVE_ADDR specifies the address of the slave in ques-
24 //             tion. It's a large
25 //             array, with one address (i.e. one set of AW
26 //             bits) for each
27 //             potential slave address region.
28 //             SLAVE_MASK specifies which of the bits in SLAVE_ADDR
29 //             need to match in
30 //             order to route a request to a that slave.
31 //             It's important to guarantee that no two slaves will
32 //             ever map to the
33 //             same address, and likewise any given slave may only map to
34 //             a single
35 //             address region.
36 //             Incidentally, the algorithm forces all slaves to have
37 //             an address
38 //             aligned with their memory size. Hence a 2GB memory
39 //             must have an
40 //             address (range) of either 0-2GB, 2GB-4GB, 4GB-6GB,
41 //             etc. However, a
42 //             second slave having only 8kB of memory may be placed at
43 //             0-8kB,
44 //             8-16kB, 16-24kB, etc. For logic minimization purposes, it
45 //             is often to
46 //             the advantage of the bus compositor to minimize the
47 //             number of mask
48 //             bits, and hence 8kB slaves may be aliased to many
49 //             places in memory.
50 //             Bus composition and address assignment, however, are
51 //             both outside of
52 //             the scope of the operation of this module.
53 //
54 // Creator:     Dan Gisselquist, Ph.D.
55 //              Gisselquist Technology, LLC

```

```

42 //
43 ///////////////////////////////////////////////////////////////////
44 // }}}
45 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
46 // {{{
47 // This file is part of the WB2AXIP project.
48 //
49 // The WB2AXIP project contains free software and gateway,
50 // licensed under the
51 // Apache License, Version 2.0 (the "License"). You may not
52 // use this project,
53 // or this file, except in compliance with the License. You
54 // may obtain a copy
55 // of the License at
56 // http://www.apache.org/licenses/LICENSE-2.0
57 // {{{
58 // Unless required by applicable law or agreed to in writing,
59 // software
60 // distributed under the License is distributed on an "AS IS"
61 // BASIS, WITHOUT
62 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
63 // plied. See the
64 // License for the specific language governing permissions and
65 // limitations
66 // under the License.
67 //
68 ///////////////////////////////////////////////////////////////////
69 //
70 // default_nettype      none
71 // }}}
72 // module addrdecode #(
73 //     parameter          NS=8,
74 //     parameter          AW = 32, DW=32+32/8+1+1,
75 //     //
76 //     // SLAVE_ADDR contains address assignments for
77 //     each of the
78 //     // various slaves we are adjudicating between.
79 //     parameter          [NS*AW-1:0] SLAVE_ADDR = {
80 //         { 3'b111, {(AW-3){1'b0}} },
81 //         { 3'b110, {(AW-3){1'b0}} },
82 //         { 3'b101, {(AW-3){1'b0}} },
83 //         { 3'b100, {(AW-3){1'b0}} },
84 //         { 3'b011, {(AW-3){1'b0}} },
85 //         { 3'b010, {(AW-3){1'b0}} },
86 //         { 4'b0010, {(AW-4){1'b0}} },
87 //         { 4'b0000, {(AW-4){1'b0}} },
88 //     },
89 //     //
90 //     // SLAVE_MASK contains a mask of those address
91 //     bits in
92 //     // SLAVE_ADDR which are relevant. It shall be
93 //     true that if
94 //     // !SLAVE_MASK[k] then !SLAVE_ADDR[k], for any
95 //     bits of k
96 //     parameter          [NS*AW-1:0] SLAVE_MASK =
97 //     (NS <= 1) ? 0
98 //         : { {(NS-2){ 3'b111, {(AW-3){1'b0}} }},
99 //             {(2){ 4'b1111, {(AW-4)
100 // }}}
101 //
102 // ACCESS_ALLOWED is a bit-wise mask indicating
103 // which slaves
104 // may get access to the bus. If AC-
105 // CESS_ALLOWED[slave] is true,
106 // then a master can connect to the slave via
107 // this method. This

```

```

93          // parameter is primarily here to support AXI
(or other similar
94          // buses) which may have separate accesses for
both read and

95          // write. By using this, a read-only slave can be
connected,
96          // which would also naturally create an error
on any attempt to
97          // write to it.
98          parameter [NS-1:0]ACCESS_ALLOWED = -1,
99          //
100          // If OPT_REGISTERED is set, address decoding
will take an extra
101          // clock, and will register the results of the
decoding
102          // operation.
103          parameter [0:0] OPT_REGISTERED = 0,
104          //
105          // If OPT_LOWPPOWER is set, then whenever the
output is not
106          // valid, any respective data linsse will also
be forced to zero
107          // in an effort to minimize power.
108          parameter [0:0] OPT_LOWPPOWER = 0
109          // }}}
110      ) (
111          // {{{
112          input      wire          i_clk, i_reset,
113          //
114          input      wire          i_valid,
115          output     reg           o_stall,
116          input      wire          [AW-1:0]i_addr,
117          input      wire          [DW-1:0]i_data,
118          //
119          output     reg           o_valid,
120          input      wire          i_stall,
121          output     reg           [NS:0]o_decode,
122          output     reg           [AW-1:0]o_addr,
123          output     reg           [DW-1:0]o_data
124          // }}}
125      );
126      // Local declarations
127      // {{{
128      // OPT_NONESEL controls whether or not the address
lines are fully
129      // proscribed, or whether or not a "no-slave identi-
fied" slave should
130      // be created. To avoid a "no-slave selected" output,
slave zero must
131      // have no mask bits set (and therefore no address bits
set), and it
132      // must also allow access.
133      localparam [0:0] OPT_NONESEL = (!ACCESS_ALLOWED[0])
134      //
135      (SLAVE_MASK[AW-1:0] != 0);
136      //
137      wire [NS:0] request;
138      reg [NS-1:0]prerequisite;
139      integer iM;
140      // }}}
141      // prerequisite
142      // {{{
143      always @(*)
144      for(iM=0; iM<NS; iM=iM+1)
145      prerequisite[iM] = (((i_addr ^ SLAVE_ADDR[iM*AW
+: AW])
146      &SLAVE_MASK[iM*AW +: AW])==0)
147      &&(ACCESS_ALLOWED[iM]);
148      // }}}
149      // request
150      // {{{
151      generate if (OPT_NONESEL)
152      begin : NO_DEFAULT_REQUEST
153      // {{{
154      reg [NS-1:0]r_request;
155      // Need to create a slave to describe when
nothing is selected
156      //
157      always @(*)
158      begin
159      for(iM=0; iM<NS; iM=iM+1)
160      r_request[iM] = i_valid && pre-
request[iM];
161      if (!OPT_NONESEL && (NS > 1 && |pre-
request[NS-1:1]))
162      r_request[0] = 1'b0;
163      end
164      assign request[NS-1:0] = r_request;
165      // }}}
166      end else if (NS == 1)
167      begin : SINGLE_SLAVE
168      // {{{
169      assign request[0] = i_valid;

```

```

175          // Verilator lint_off UNUSED
176          wire unused;
177          assign unused = &{ 1'b0, prerequisite };
178          // Verilator lint_on UNUSED
179          // }}}
180      end else begin : GENERAL_CASE
181      // {{{
182      reg [NS-1:0]r_request;
183      always @(*)
184      begin
185      for(iM=0; iM<NS; iM=iM+1)
186      r_request[iM] = i_valid && pre-
request[iM];
187      if (!OPT_NONESEL && (NS > 1 && |pre-
request[NS-1:1]))
188      r_request[0] = 1'b0;
189      end
190      assign request[NS-1:0] = r_request;
191      // }}}
192      end endgenerate
193      // }}}
194      // request[NS]
195      // {{{
196      generate if (OPT_NONESEL)
197      begin : GENERATE_NONSEL_SLAVE
198      reg r_request_NS, r_none_sel;
199      always @(*)
200      begin
201      // Let's assume nothing's been selec-
ted, and then check
202      // to prove ourselves wrong.
203      //
204      // Note that none_sel will be con-
sidered an error
205      // condition in the follow-on pro-
cessing. Therefore
206      // it's important to clear it if no re-
quest is pending.
207      r_none_sel = i_valid && (prerequisite ==
0);
208      //
209      // request[NS] indicates a request for
a non-existent
210      // slave. A request that should (even-
tually) return a
211      // bus error
212      //
213      r_request_NS = r_none_sel;
214      end
215      assign request[NS] = r_request_NS;
216      end else begin : NO_NONESEL_SLAVE
217      assign request[NS] = 1'b0;
218      end endgenerate
219      // }}}
220      // o_valid, o_addr, o_data, o_decode, o_stall
221      // {{{
222      generate if (OPT_REGISTERED)
223      begin : GEN_REG_OUTPUTS
224      // o_valid
225      // {{{
226      initial o_valid = 0;
227      always @(posedge i_clk)
228      if (i_reset)
229      o_valid <= 0;
230      else if (!o_stall)
231      o_valid <= i_valid;
232      // }}}
233      // o_addr, o_data
234      // {{{
235      initial o_addr = 0;
236      initial o_data = 0;
237      always @(posedge i_clk)
238      if (i_reset && OPT_LOWPPOWER)
239      begin
240      o_addr <= 0;
241      o_data <= 0;
242      end else if (!o_valid || !i_stall)
243      && (i_valid || !OPT_LOWPPOWER))
244      begin
245      o_addr <= i_addr;
246      o_data <= i_data;
247      end else if (OPT_LOWPPOWER && !i_stall)
248      begin
249      o_addr <= 0;
250      o_data <= 0;
251      end
252      // }}}
253      // o_decode
254      // {{{
255      initial o_decode = 0;
256      always @(posedge i_clk)

```

```

267         if (i_reset)
268             o_decode <= 0;
269         else if ((!o_valid || !i_stall)
270             && (i_valid || !OPT_LOWPPOWER))
271             o_decode <= request;
272         else if (OPT_LOWPPOWER && !i_stall)
273             o_decode <= 0;
274         // }}}
275
276         // o_stall
277         // {{{
278         always @(*)
279             o_stall = (o_valid && i_stall);
280         // }}}
281     end else begin : GEN_COMBINATORIAL_OUTPUTS
282
283         always @(*)
284         begin
285             o_valid = i_valid;
286             o_stall = i_stall;
287             o_addr = i_addr;
288             o_data = i_data;
289
290             o_decode = request;
291         end
292
293         // Make Verilator happy
294         // {{{
295         // verilator coverage_off
296         // verilator lint_off UNUSED
297         wire unused;
298         assign unused = &{ 1'b0,
299 `ifdef VERILATOR
300             // Can't declare the clock as
301             unused for formal,
302             // lest it not be recognized as
303             i_clk,
304             i_reset };
305         // verilator lint_on UNUSED
306         // verilator coverage_on
307         // }}}
308     end endgenerate
309     // }}}
310 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
311 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
312 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
313 //
314 // Formal properties
315 // {{{
316 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
317 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
318 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
319 `ifdef FORMAL
320     reg f_past_valid;
321     initial f_past_valid = 0;
322     always @(posedge i_clk)
323         f_past_valid <= 1;
324
325     reg [AW+DW-1:0] f_idata;
326     always @(*)
327         f_idata = { i_addr, i_data };
328 `endif
329 `ifdef ADDRDECODE
330     always @(posedge i_clk)
331         if (!f_past_valid)
332             assume(i_reset);
333 `else
334     always @(posedge i_clk)
335         if (!f_past_valid)
336             assert(i_reset);
337 `endif // ADDRDECODE
338 always @(posedge i_clk)
339     if (OPT_REGISTERED && (!f_past_valid || $past(i_re-
340 set)))
341     begin
342         assert(!o_valid);
343         assert(o_decode == 0);
344     end else if ($past(o_valid && i_stall) && OPT_RE-
345 GISTERED)
346     begin
347         assert($stable(o_addr));
348         assert($stable(o_decode));
349         assert($stable(o_data));
350     end
351
352     // If the output is ever valid, there must be at least
353     one
354     // decoded output
355     always @(*)
356         assert(o_valid == (o_decode != 0));

```

```

356     always @(*)
357     for(im=0; im<NS; im=im+1)
358     if (o_decode[im])
359     begin
360         // The address must match
361         assert((((o_addr ^ SLAVE_ADDR[im*AW +: AW])
362             & SLAVE_MASK[im*AW +: AW]) == 0)
363             && ACCESS_ALLOWED[im]);
364
365         //
366         // And nothing else must match
367         assert(o_decode == (1<<im));
368     end
369
370     always @(*)
371     for(im=0; im<NS; im=im+1)
372     if (!ACCESS_ALLOWED[im])
373         assert(!o_decode[im]);
374
375     // LOWPOWER check
376     // {{{
377     generate if (OPT_LOWPPOWER && OPT_REGISTERED)
378     begin
379         always @(*)
380         if (!o_valid)
381         begin
382             assert(o_addr == 0);
383             assert(o_decode == 0);
384             assert(o_data == 0);
385         end
386     end endgenerate
387     // }}}
388
389     //
390     // The output decoded value may only ever have one
391     value high,
392     // never more--i.e. $onehot0
393     // {{{
394     `ifdef VERIFIC
395     always @(*)
396         assert($onehot0(request));
397     `else
398     reg onehot_request;
399     always @(*)
400     begin
401         onehot_request = 0;
402         for(im=0; im<NS+1; im=im+1)
403             if ((request ^ (1<<im)) == 0)
404                 onehot_request = 1;
405     end
406
407     always @(*)
408     if (request != 0)
409         assert(onehot_request);
410     `endif
411     // }}}
412 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
413 // Cover properties
414 // {{{
415 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
416 //
417 //
418 //
419 //
420 // Make sure all addresses are reachable
421 //
422 reg [NS:0] f_reached;
423
424 always @(posedge i_clk)
425     cover(i_valid);
426
427 always @(posedge i_clk)
428     cover(o_valid);
429
430 always @(posedge i_clk)
431     cover(o_valid && !i_stall);
432
433 initial f_reached = 0;
434 always @(posedge i_clk)
435     if (i_reset)
436         f_reached = 0;
437     else if (o_valid)
438         f_reached = f_reached | o_decode;
439
440 generate if (!OPT_NONESSEL && ACCESS_ALLOWED[0]
441     && SLAVE_MASK == 0 && NS == 1)
442     begin
443         always @(*)
444             cover(f_reached[0]);
445
446         always @(posedge i_clk)
447             if (f_past_valid && $stable(o_valid))
448                 assert($stable(o_decode));
449     end else begin
450         always @(*)

```

```

454         cover(&f_reached);
455     always @(posedge i_clk)
456     if (f_past_valid && $stable(o_valid))
457         cover($changed(o_decode));
458
459 endendgenerate
460 // }}}
461 `endif // FORMAL
462 // }}}
463 endmodule
464 `ifndef YOSYS
465 `default_nettype wire
466 `endif

```

2. ZipCPU_wb2axip/rtl/afifo.v (804 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:    rtl/afifo.v
4 // {{{
5 // Project:    WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:    A basic asynchronous FIFO.
8 //
9 // Creator:    Dan Gisselquist, Ph.D.
10 //             Gisselquist Technology, LLC
11 //
12 ///////////////////////////////////////////////////////////////////
13 // }}}
14 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
15 // {{{
16 // This file is part of the WB2AXIP project.
17 //
18 // The WB2AXIP project contains free software and gateware,
19 // licensed under the
20 // Apache License, Version 2.0 (the "License"). You may not
21 // use this project,
22 // or this file, except in compliance with the License. You
23 // may obtain a copy
24 // of the License at
25 // http://www.apache.org/licenses/LICENSE-2.0
26 // {{{
27 // Unless required by applicable law or agreed to in writing,
28 // software
29 // distributed under the License is distributed on an "AS IS"
30 // BASIS, WITHOUT
31 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
32 // plied. See the
33 //
34 // License for the specific language governing permissions and
35 // limitations
36 // under the License.
37 //
38 ///////////////////////////////////////////////////////////////////
39 // }}}
40 //
41 // default_nettype none
42 // }}}
43 module afifo #(
44     // {{{
45     // LGFIFO is the log based-two of the number of
46     // entries
47     // in the FIFO, log_2(fifo size)
48     // parameter LGFIFO = 3,
49     //
50     // WIDTH is the number of data bits in each
51     // entry
52     // parameter WIDTH = 16,
53     //
54     // NFF is the number of flip flops used to
55     // cross clock domains.
56     // 2 is a minimum. Some applications appreci-
57     // ate the better
58     // parameter NFF = 2,
59     //
60     // This core can either write on the positive
61     // edge of the clock
62     // or the negative edge. Set WRITE_ON_POSEDGE
63     // (the default)
64     // to write on the positive edge of the clock.
65     // parameter [0:0] WRITE_ON_POSEDGE = 1'b1,
66     //
67     // Many logic elements can read from memory
68     // asynchronously.
69     // This capability is sometimes known as "Dis-
70     // tributed RAM" or
71     // "LUT RAM". Doing so, however, burdens the
72     // timing of any
73     // logic to follow. By setting OPT_RE-
74     // GISTER_READS, we force
75     // all reads to be synchronous and not burdened by
76     // any logic.
77     // This tends to work with more hardware. If

```

```

your hardware does
60 // support "Distributed RAM", then you may be
able to spare a
61 // clock of latency by clearing this parameter.
62 // parameter [0:0] OPT_REGISTER_READS = 1'b1
63 `ifndef FORMAL
64 // F_OPT_DATA_STB
65 // {{{
66 // In the formal proof, F_OPT_DATA_STB includes a
67 // series of
68 // assumptions associated with a data strobe I/
69 // 0 pin--things
70 // like a discontinuous clock--just to make
71 // sure the core still
72 // works in those circumstances
73 // , parameter [0:0] F_OPT_DATA_STB = 1'b1
74 // }}}
75 `endif
76 // }}}
77 ) (
78 // {{{
79 // The (incoming) write data interface
80 // input wire i_wclk,
81 // i_wr_reset_n, i_wr,
82 // input wire [WIDTH-1:0] i_wr_data,
83 // output reg o_wr_full,
84 //
85 // The (incoming) write data interface
86 // input wire i_rclk,
87 // i_rd_reset_n, i_rd,
88 // output reg [WIDTH-1:0] o_rd_data,
89 // output reg o_rd_empty
90 // , output reg [LGFIFO:0] f_fill
91 // }}}
92 );
93 // Register/net declarations
94 // {{{
95 // MSB = most significant bit of the FIFO address vec-
96 // tor. It's
97 // just short-hand for LGFIFO, and won't work any other
98 // way.
99 localparam MSB = LGFIFO;
100 //
101 reg [WIDTH-1:0] mem
102 [(1<<LGFIFO)-1:0];
103 reg [LGFIFO:0] rd_addr, wr_addr,
104 rd_wgray, wr_rgray;
105 next_rd_addr,
106 next_wr_addr;
107 reg [LGFIFO:0] rgray, wgray;
108 (* ASYNC_REG = "TRUE" *) reg
109 [(LGFIFO+1)*(NFF-1)-1:0]
110 rgray_cross,
111 wgray_cross;
112 wire [WIDTH-1:0] wclk;
113 reg lcl_rd_data;
114 reg lcl_read, lcl_rd_empty;
115 // }}}
116 // wclk - Write clock generation
117 // {{{
118 generate if (WRITE_ON_POSEDGE)
119 begin : GEN_POSEDGE_WRITES
120 assign wclk = i_wclk;
121 end else begin : GEN_NEGEDGE_WRITES
122 assign wclk = !i_wclk;
123 end endgenerate
124 // }}}
125 //
126 // Write to and read from memory
127 // {{{
128 ///////////////////////////////////////////////////////////////////
129 //
130 //
131 // wr_addr, wgray
132 // {{{
133 assign next_wr_addr = wr_addr + 1;
134 always @(posedge wclk or negedge i_wr_reset_n)
135 if (!i_wr_reset_n)
136 begin
137 wr_addr <= 0;
138 wgray <= 0;
139 end else if (i_wr && !o_wr_full)
140 begin
141 wr_addr <= next_wr_addr;
142 wgray <= next_wr_addr ^ (next_wr_addr >> 1);

```

```

144     end
145     // }}}
146
147     // Write to memory
148     // {{{
149     always @(posedge wclk)
150     if (i_wr && !o_wr_full)
151         mem[wr_addr[LGFI0-1:0]] <= i_wr_data;
152     // }}}
153
154     // rd_addr, rgray
155     // {{{
156     assign next_rd_addr = rd_addr + 1;
157     always @(posedge i_rclk or negedge i_rd_reset_n)
158     if (!i_rd_reset_n)
159     begin
160         rd_addr <= 0;
161         rgray <= 0;
162     end else if (lcl_read && !lcl_rd_empty)
163     begin
164         rd_addr <= next_rd_addr;
165         rgray <= next_rd_addr ^ (next_rd_addr >> 1);
166     end
167     // }}}
168
169     // Read from memory
170     // {{{
171     always @(*)
172         lcl_rd_data = mem[rd_addr[LGFI0-1:0]];
173     // }}}
174     // }}}
175     ///////////////////////////////////////////////////////////////////
176     //
177     // Cross clock domains
178     // {{{
179     ///////////////////////////////////////////////////////////////////
180     //
181     //
182
183     // read pointer -> wr_rgray
184     // {{{
185     always @(posedge wclk or negedge i_wr_reset_n)
186     if (!i_wr_reset_n)
187         { wr_rgray, rgray_cross } <= 0;
188     else
189         { wr_rgray, rgray_cross } <= { rgray_cross,
rgray };
190     // }}}
191
192     // write pointer -> rd_wgray
193     // {{{
194     always @(posedge i_rclk or negedge i_rd_reset_n)
195     if (!i_rd_reset_n)
196         { rd_wgray, wgray_cross } <= 0;
197     else
198         { rd_wgray, wgray_cross } <= { wgray_cross,
wgray };
199     // }}}
200
201     // }}}
202     ///////////////////////////////////////////////////////////////////
203     //
204     // Flag generation
205     // {{{
206     ///////////////////////////////////////////////////////////////////
207     //
208     //
209
210     always @(*)
211         o_wr_full = (wr_rgray == { ~wgray[MSB:MSB-1],
wgray[MSB-2:0] });
212
213     always @(*)
214         lcl_rd_empty = (rd_wgray == rgray);
215
216     // o_rd_empty, o_rd_data
217     // {{{
218     generate if (OPT_REGISTER_READS)
219     begin : GEN_REGISTERED_READ
220         // {{{
221         always @(*)
222             lcl_read = (o_rd_empty || i_rd);
223
224         always @(posedge i_rclk or negedge i_rd_re-
set_n)
225         if (!i_rd_reset_n)
226             o_rd_empty <= 1'b1;
227         else if (lcl_read)
228             o_rd_empty <= lcl_rd_empty;
229
230         always @(posedge i_rclk)
231         if (lcl_read)
232             o_rd_data <= lcl_rd_data;
233         // }}}
234     end else begin : GEN_COMBINATORIAL_FLAGS
235         // {{{
236         always @(*)

```

```

237         lcl_read = i_rd;
238
239         always @(*)
240             o_rd_empty = lcl_rd_empty;
241
242         always @(*)
243             o_rd_data = lcl_rd_data;
244         // }}}
245     end endgenerate
246     // }}}
247     // }}}
248     ///////////////////////////////////////////////////////////////////
249     ///////////////////////////////////////////////////////////////////
250     ///////////////////////////////////////////////////////////////////
251     //
252     // Formal properties
253     // {{{
254     ///////////////////////////////////////////////////////////////////
255     ///////////////////////////////////////////////////////////////////
256     ///////////////////////////////////////////////////////////////////
257     `ifdef FORMAL
258
259     // Start out with some register/net/macro declarations,
260     f_past_valid,etc
261     // {{{
262     `ifndef AFIFO
263     `define ASSERT assert
264     `define ASSUME assume
265     `else
266     `define ASSERT assert
267     `define ASSUME assume
268     `endif
269
270     (* gclk *)      reg      gbl_clk;
271     reg             f_past_valid_gbl,
f_past_valid_rd,
272     reg             f_rd_in_reset, f_wr_in_reset;
273     reg             [WIDTH-1:0] past_rd_data, past_wr_data;
274     reg             past_wr_reset_n, past_rd_re-
set_n,
275     reg             past_rd_empty, past_wclk,
past_rclk, past_rd;
276     reg             [(LGFI0+1)*(NFF-1)-1:0] f_wcross, f_rcross;
277     reg             [LGFI0:0] f_rd_waddr, f_wr_raddr;
278     reg             [LGFI0:0] f_rdcross_fill [NFF-1:0];
279     reg             [LGFI0:0] f_wrcross_fill [NFF-1:0];
280
281     initial f_past_valid_gbl = 1'b0;
282     always @(posedge gbl_clk)
283         f_past_valid_gbl <= 1'b1;
284
285     initial f_past_valid_rd = 1'b0;
286     always @(posedge i_rclk)
287         f_past_valid_rd <= 1'b1;
288     // }}}
289     ///////////////////////////////////////////////////////////////////
290     //
291     // Reset checks
292     // {{{
293     ///////////////////////////////////////////////////////////////////
294     //
295     initial f_wr_in_reset = 1'b1;
296     always @(posedge wclk or negedge i_wr_reset_n)
297     if (!i_wr_reset_n)
298         f_wr_in_reset <= 1'b1;
299     else
300         f_wr_in_reset <= 1'b0;
301
302     initial f_rd_in_reset = 1'b1;
303     always @(posedge i_rclk or negedge i_rd_reset_n)
304     if (!i_rd_reset_n)
305         f_rd_in_reset <= 1'b1;
306     else
307         f_rd_in_reset <= 1'b0;
308
309     //
310     // Resets are ...
311     // 1. Asserted always initially, and ...
312     always @(*)
313     if (!f_past_valid_gbl)
314     begin
315         `ASSUME(!i_wr_reset_n);
316         `ASSUME(!i_rd_reset_n);
317     end
318
319     // 2. They only ever become active together
320     always @(*)
321     if (past_wr_reset_n && !i_wr_reset_n)
322         `ASSUME(!i_rd_reset_n);
323
324     always @(*)

```



```

325     if (past_rd_reset_n && !i_rd_reset_n)
326         `ASSUME(!i_wr_reset_n);
327
328         // }}}
329         ///////////////////////////////////////////////////////////////////
330         //
331         // Synchronous signal assumptions
332         // {{{
333         ///////////////////////////////////////////////////////////////////
334         //
335         //
336
337         always @(posedge gbl_clk)
338         begin
339             past_wr_reset_n <= i_wr_reset_n;
340             past_rd_reset_n <= i_rd_reset_n;
341
342             past_wclk <= wclk;
343             past_rclk <= i_rclk;
344
345             past_rd <= i_rd;
346             past_rd_data <= lcl_rd_data;
347             past_wr_data <= i_wr_data;
348
349             past_rd_empty <= lcl_rd_empty;
350         end
351
352         //
353         // Read side may be assumed to be synchronous
354         always @(*)
355         if (f_past_valid_gbl && i_rd_reset_n && (past_rclk || !
i_rclk))
356             // i.e. if (!$rose(i_rclk))
357             `ASSUME(i_rd == past_rd);
358
359         always @(*)
360         if (f_past_valid_rd && !f_rd_in_reset && !lcl_rd_empty
&& (past_rclk || !i_rclk))
361         begin
362             `ASSERT(lcl_rd_data == past_rd_data);
363             `ASSERT(lcl_rd_empty == past_rd_empty);
364         end
365
366         generate if (F_OPT_DATA_STB)
367         begin
368             always @(posedge gbl_clk)
369                 `ASSUME(!o_wr_full);
370
371             always @(posedge gbl_clk)
372                 if (!i_wr_reset_n)
373                     `ASSUME(!i_wclk);
374
375             always @(posedge gbl_clk)
376                 `ASSUME(i_wr == i_wr_reset_n);
377
378             always @(posedge gbl_clk)
379                 if ($changed(i_wr_reset_n))
380                     `ASSUME($stable(wclk));
381
382             end endgenerate
383             // }}}
384             ///////////////////////////////////////////////////////////////////
385             //
386             // Fill checks
387             // {{{
388             ///////////////////////////////////////////////////////////////////
389             //
390             //
391             //
392             //
393             //
394             always @(*)
395                 f_fill = wr_addr - rd_addr;
396
397             always @(*)
398                 if (!f_wr_in_reset)
399                     `ASSERT(f_fill <= { 1'b1, {(MSB){1'b0}} });
400
401             always @(*)
402                 if (wr_addr == rd_addr)
403                     `ASSERT(lcl_rd_empty);
404
405             always @(*)
406                 if ((!f_wr_in_reset && !f_rd_in_reset)
&& wr_addr == { ~rd_addr[MSB], rd_ad-
dr[MSB-1:0] })
407                     `ASSERT(o_wr_full);
408
409             // }}}
410             ///////////////////////////////////////////////////////////////////
411             //
412             //
413             // Induction checks
414             // {{{
415             ///////////////////////////////////////////////////////////////////
416             //
417             //

```

```

418
419         // f_wr_in_reset -- write logic is in its reset state
420         // {{{
421         always @(*)
422         if (f_wr_in_reset)
423         begin
424             `ASSERT(wr_addr == 0);
425             `ASSERT(wgray_cross == 0);
426
427             `ASSERT(rd_addr == 0);
428             `ASSERT(rgray_cross == 0);
429             `ASSERT(rd_wgray == 0);
430
431             `ASSERT(lcl_rd_empty);
432             `ASSERT(!o_wr_full);
433         end
434         // }}}
435
436         // f_rd_in_reset -- read logic is in its reset state
437         // {{{
438         always @(*)
439         if (f_rd_in_reset)
440         begin
441             `ASSERT(rd_addr == 0);
442             `ASSERT(rgray_cross == 0);
443             `ASSERT(rd_wgray == 0);
444
445             `ASSERT(lcl_rd_empty);
446         end
447         // }}}
448
449         // f_wr_raddr -- a read address to match the gray val-
ues
450         // {{{
451         always @(posedge wclk or negedge i_wr_reset_n)
452         if (!i_wr_reset_n)
453             { f_wr_raddr, f_rcross } <= 0;
454         else
455             { f_wr_raddr, f_rcross } <= { f_rcross, rd_ad-
dr };
456         // }}}
457
458         // f_rd_waddr -- a write address to match the gray val-
ues
459         // {{{
460         always @(posedge i_rclk or negedge i_rd_reset_n)
461         if (!i_rd_reset_n)
462             { f_rd_waddr, f_wcross } <= 0;
463         else
464             { f_rd_waddr, f_wcross } <= { f_wcross, wr_ad-
dr };
465         // }}}
466
467         integer k;
468
469         // wgray check
470         // {{{
471         always @(*)
472             `ASSERT((wr_addr ^ (wr_addr >> 1)) == wgray);
473         // }}}
474
475         // wgray_cross check
476         // {{{
477         always @(*)
478         for(k=0; k<NFF-1; k=k+1)
479             `ASSERT((f_wcross[k*(LGFI0+1) +: LGFI0+1]
^ (f_wcross[k*(LGFI0+1) +:
LGFI0+1]>>1))
480                 == wgray_cross[k*(LGFI0+1) +:
LGFI0+1]);
481         // }}}
482
483         // rgray check
484         // {{{
485         always @(*)
486             `ASSERT((rd_addr ^ (rd_addr >> 1)) == rgray);
487         // }}}
488
489         // rgray_cross check
490         // {{{
491         always @(*)
492         for(k=0; k<NFF-1; k=k+1)
493             `ASSERT((f_rcross[k*(LGFI0+1) +: LGFI0+1]
^ (f_rcross[k*(LGFI0+1) +:
LGFI0+1]>>1))
494                 == rgray_cross[k*(LGFI0+1) +:
LGFI0+1]);
495         // }}}
496
497         // wr_rgray
498         // {{{
499         always @(*)
500             `ASSERT((f_wr_raddr ^ (f_wr_raddr >> 1)) ==
wr_rgray);
501         // }}}
502
503         // rd_wgray
504         // {{{
505         always @(*)
506             `ASSERT((f_rd_waddr ^ (f_rd_waddr >> 1)) ==
rd_wgray);
507         // }}}
508

```

```

509 // }}}
510 // f_rdcross_fill
511 // {{{
512 always @(*)
513 for(k=0; k<NFF-1; k=k+1)
514   f_rdcross_fill[k] = f_wcross[k*(LGFI0+1) +:
LGFI0+1]
515   - rd_addr;
516
517 always @(*)
518   f_rdcross_fill[NFF-1] = f_rd_waddr - rd_addr;
519
520 always @(*)
521 for(k=0; k<NFF; k=k+1)
522   `ASSERT(f_rdcross_fill[k] <= { 1'b1, {(LGFI0)
{1'b0}} });
523
524 always @(*)
525 for(k=1; k<NFF; k=k+1)
526   `ASSERT(f_rdcross_fill[k] <= f_rd-
cross_fill[k-1]);
527
528 always @(*)
529   `ASSERT(f_rdcross_fill[0] <= f_fill);
530 // }}}
531
532 // f_wrcross_fill
533 // {{{
534 always @(*)
535 for(k=0; k<NFF-1; k=k+1)
536   f_wrcross_fill[k] = wr_addr -
f_rcross[k*(LGFI0+1) +: LGFI0+1];
537
538 always @(*)
539   f_wrcross_fill[NFF-1] = wr_addr - f_wr_raddr;
540
541 always @(*)
542 for(k=0; k<NFF; k=k+1)
543   `ASSERT(f_wrcross_fill[k] <= { 1'b1, {(LGFI0)
{1'b0}} });
544
545 always @(*)
546 for(k=1; k<NFF; k=k+1)
547   `ASSERT(f_wrcross_fill[k] >= f_wr-
cross_fill[k-1]);
548
549 always @(*)
550   `ASSERT(f_wrcross_fill[0] >= f_fill);
551 // }}}
552 // }}}
553 // }}}
554 ///////////////////////////////////////////////////
555 //
556 // Clock generation
557 // {{{
558 ///////////////////////////////////////////////////
559 //
560 //
561 // Here's the challenge: if we use $past with any of
our clocks, such
562 // as to determine stability or any such, the proof
takes forever, and
563
564 // we need to guarantee a minimum number of transitions
within the
565
566 // depth of the proof. If, on the other hand, we build
our own $past
567 // primitives--we can then finish much faster and be
successful on
568 // any depth of proof.
569
570 // pre_xclk is what the clock will become on the next
global clock
edge.
571 // By using it here, we can check things @(*) instead
of
572 // @posedge gbl_clk). Further, we can check
$rose(pre_xclk) (or $fell)
573 // and essentially check things @(*) but while using
@(global_clk).
574 // In other words, we can transition on @posedge
gbl_clk), but stay
575
576 // in sync with the data--rather than being behind by a
clock.
577 // now_xclk is what the clock is currently.
578 //
579 (* anyseq *) reg pre_wclk, pre_rclk;
580 reg now_wclk, now_rclk;
581
582 always @posedge gbl_clk
583 begin
584   now_wclk <= pre_wclk;
585   now_rclk <= pre_rclk;
586 end
587
588 always @(*)
589 begin
590   assume(i_wclk == now_wclk);
591   assume(i_rclk == now_rclk);

```

```

589 end
590
591 always @posedge gbl_clk
592   assume(i_rclk == $past(pre_rclk));
593
594 // Assume both clocks start idle
595 // {{{
596 always @(*)
597 if (!f_past_valid_gbl)
598 begin
599   assume(!pre_wclk && !wclk);
600   assume(!pre_rclk && !i_rclk);
601 end
602 // }}}
603
604 // }}}
605 ///////////////////////////////////////////////////
606 //
607 // Formal contract check --- the twin write test
608 // {{{
609 ///////////////////////////////////////////////////
610 //
611 //
612 // Tracking register declarations
613 // {{{
614
615 reg [WIDTH-1:0] f_first,
f_next;
616 (* anyconst *) reg [LGFI0:0] f_addr;
617 reg [LGFI0:0] f_next_addr;
618 reg f_first_in_fifo,
f_next_in_fifo;
619 reg [LGFI0:0] f_to_first, f_to_next;
620 reg [1:0] f_state;
621
622 always @(*)
623   f_next_addr = f_addr + 1;
624 // }}}
625
626 // distance_to*, *_in_fifo
627 // {{{
628 always @(*)
629 begin
630   f_to_first = f_addr - rd_addr;
631
632   f_first_in_fifo = 1'b1;
633   if ((f_to_first >= f_fill) || (f_fill == 0))
634     f_first_in_fifo = 1'b0;
635
636   if (mem[f_addr] != f_first)
637     f_first_in_fifo = 1'b0;
638
639 //
640 // Check the second item
641 //
642
643   f_to_next = f_next_addr - rd_addr;
644   f_next_in_fifo = 1'b1;
645   if ((f_to_next >= f_fill) || (f_fill == 0))
646     f_next_in_fifo = 1'b0;
647
648   if (mem[f_next_addr] != f_next)
649     f_next_in_fifo = 1'b0;
650 end
651 // }}}
652
653 // f_state -- generate our state variable
654 // {{{
655 initial f_state = 0;
656 always @posedge gbl_clk
657 if (!i_wr_reset_n)
658   f_state <= 0;
659 else case(f_state)
660 2'b00: if (($rose(pre_wclk)) && i_wr && !o_wr_full
&& (wr_addr == f_addr))
661   begin
662     f_state <= 2'b01;
663     f_first <= i_wr_data;
664   end
665 2'b01: if ($rose(pre_rclk) && lcl_read && rd_addr ==
f_addr)
666   f_state <= 2'b00;
667   else if ($rose(pre_wclk) && i_wr && !
o_wr_full )
668   begin
669     f_state <= 2'b10;
670     f_next <= i_wr_data;
671   end
672 2'b10: if ($rose(pre_rclk) && lcl_read && !lcl_rd_empty
&& rd_addr == f_addr)
673   f_state <= 2'b11;
674 2'b11: if ($rose(pre_rclk) && lcl_read && !lcl_rd_empty
&& rd_addr == f_next_addr)
675   f_state <= 2'b00;
676 endcase
677 // }}}
678

```

```

679 // f_state invariants
680 // {{{
681 always @(*)
682 if (i_wr_reset_n) case(f_state)
683 2'b00: begin end
684 2'b01: begin
685     `ASSERT(f_first_in_fifo);
686     `ASSERT(wr_addr == f_next_addr);
687     `ASSERT(f_fill >= 1);
688 end
689 2'b10: begin
690     `ASSERT(f_first_in_fifo);
691     `ASSERT(f_next_in_fifo);
692     if (!lcl_rd_empty && (rd_addr == f_addr))
693         `ASSERT(lcl_rd_data == f_first);
694     `ASSERT(f_fill >= 2);
695 end
696 2'b11: begin
697     `ASSERT(rd_addr == f_next_addr);
698     `ASSERT(f_next_in_fifo);
699     `ASSERT(f_fill >= 1);
700     if (!lcl_rd_empty)
701         `ASSERT(lcl_rd_data == f_next);
702 end
703 endcase
704 // }}}
705 generate if (OPT_REGISTER_READS)
706 begin
707     reg    past_o_rd_empty;
708
709     always @(posedge gbl_clk)
710         past_o_rd_empty <= o_rd_empty;
711
712     always @(posedge gbl_clk)
713         if (f_past_valid_gbl && i_rd_reset_n)
714             begin
715                 if ($past(!o_rd_empty && !i_rd &&
716 i_rd_reset_n))
717                     `ASSERT($stable(o_rd_data));
718             end
719
720     always @(posedge gbl_clk)
721         if (!f_rd_in_reset && i_rd_reset_n && i_rclk
722 && !past_rclk)
723             begin
724                 if (past_o_rd_empty)
725                     `ASSERT(o_rd_data ==
726 past_rd_data);
727                 if (past_rd)
728                     `ASSERT(o_rd_data ==
729 past_rd_data);
730             end
731         end
732     end endgenerate
733 // }}}
734 // {{{
735 // Cover checks
736 // {{{
737 // {{{
738 // Prove that we can read and write the FIFO
739 // {{{
740 always @(*)
741 if (i_wr_reset_n && i_rd_reset_n)
742 begin
743     cover(o_rd_empty);
744     cover(!o_rd_empty);
745     cover(f_state == 2'b01);
746     cover(f_state == 2'b10);
747     cover(f_state == 2'b11);
748     cover(&f_fill[MSB-1:0]);
749
750     cover(i_rd);
751     cover(i_rd && !o_rd_empty);
752 end
753 // }}}
754 // }}}
755 `ifdef AFIFO
756 generate if (!F_OPT_DATA_STB)
757 begin : COVER_FULL
758     // {{{
759     reg    cvr_full;
760
761     initial cvr_full = 1'b0;
762     always @(posedge gbl_clk)
763         if (!i_wr_reset_n)
764             cvr_full <= 1'b0;
765         else if (o_wr_full)
766             cvr_full <= 1'b1;
767
768     always @(*)
769         if (f_past_valid_gbl && i_wr_reset_n)
770             begin
771                 cover(o_wr_full);

```

```

774         cover(o_rd_empty && cvr_full);
775         cover(o_rd_empty && f_fill == 0 &&
776 cvr_full);
777     end
778     // }}}
779 end else begin : COVER_NEARLY_FULL
780     // {{{
781     reg    cvr_nearly_full;
782
783     initial cvr_nearly_full = 1'b0;
784     always @(posedge gbl_clk)
785         if (!i_wr_reset_n)
786             cvr_nearly_full <= 1'b0;
787         else if (f_fill == { 1'b0, {(LGFIFO){1'b1} }})
788             cvr_nearly_full <= 1'b1;
789
790     always @(*)
791         if (f_past_valid_gbl && i_wr_reset_n)
792             begin
793                 cover(f_fill == { 1'b0, {(LGFIFO)
794 {1'b1} }});
795                 cover(cvr_nearly_full && i_wr_reset_n);
796                 cover(o_rd_empty && cvr_nearly_full);
797                 cover(o_rd_empty && f_fill == 0 &&
798 cvr_nearly_full);
799             end
800         end endgenerate
801     // }}}
802     // }}}
803 // }}}
804 endmodule

```

3. ZipCPU_wb2axip/rtl/apbslave.v (228 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:    rtl/apbslave.v
4 // {{{
5 // Project:    WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:    Just a simple demonstration APB slave
8 //
9 // Creator:    Dan Gisselquist, Ph.D.
10 //             Gisselquist Technology, LLC
11 //
12 ///////////////////////////////////////////////////////////////////
13 //
14 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
15 //
16 // This file is part of the WB2AXIP project.
17 //
18 // The WB2AXIP project contains free software and gateware,
19 // licensed under the
20 // Apache License, Version 2.0 (the "License"). You may not
21 // use this project,
22 // or this file, except in compliance with the License. You
23 // may obtain a copy
24 // of the License at
25 // http://www.apache.org/licenses/LICENSE-2.0
26 //
27 // Unless required by applicable law or agreed to in writing,
28 // software
29 // distributed under the License is distributed on an "AS IS"
30 // BASIS, WITHOUT
31 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
32 // plied. See the
33 //
34 // License for the specific language governing permissions and
35 // limitations
36 // under the License.
37 //
38 ///////////////////////////////////////////////////////////////////
39 //
40 // default_nettype    none
41 // {{{
42 module apbslave #(
43     // {{{
44     parameter          C_APB_ADDR_WIDTH = 12,
45     parameter          C_APB_DATA_WIDTH = 32,
46     localparam         AW = C_APB_ADDR_WIDTH,
47     localparam         DW = C_APB_DATA_WIDTH,
48     localparam         APBLSB = $clog2(C_APB_DATA_WIDTH)-3
49 ) (
50     // {{{
51     input  wire          PCLK, PRESETn,
52     input  wire          PSEL,
53     input  wire          PENABLE,
54     output reg           PREADY,
55     input  wire          [AW-1:0]PADDR,
56     input  wire          PWRITE,
57     input  wire          [DW-1:0]PDATA,

```

```

52      input wire [DW/8-1:0] PWSTRB,
53      input wire [2:0] PPROT,
54      output reg [DW-1:0] PRDATA,
55      output wire PSLVERR
56      // }}}
57  );
58
59  // Register declarations
60  // {{{
61  // Just our demonstration "memory" here
62  reg [DW-1:0] mem [0:(1<<(AW-APBLSB))-1];
63  integer ik;
64  // }}}
65
66  // PREADY
67  // {{{
68  initial PREADY = 1'b0;
69  always @(posedge PCLK)
70  if (!PRESETn)
71      PREADY <= 1'b0;
72  else if (PSEL && !PENABLE)
73      PREADY <= 1'b1;
74  else
75      PREADY <= 1'b0;
76  // }}}
77
78  // mem writes
79  // {{{
80  always @(posedge PCLK)
81  if (PRESETn && PSEL && !PENABLE && PWRITE)
82  begin
83      for(ik=0; ik<DW/8; ik=ik+1)
84      if (PWSTRB[ik])
85          mem[PADDR[AW-1:APBLSB]][8*ik+: 8] <=
86          PWDATA[8*ik+: 8];
87      end
88      // }}}
89
90  // PRDATA, memory reads
91  // {{{
92  always @(posedge PCLK)
93  if (PSEL && !PENABLE && !PWRITE)
94      PRDATA <= mem[PADDR[AW-1:APBLSB]];
95  // }}}
96
97  // PSLVERR -- unused in this design, and so kept at
98  // {{{
99  assign PSLVERR = 1'b0;
100  // }}}
101
102  // Make Verilator happy
103  // {{{
104  wire unused;
105  assign unused = &{ 1'b0, PADDR[APBLSB-1:0], PPROT };
106  // Verilator lint_on UNUSED
107  // }}}
108  ///////////////////////////////////////////////////
109  ///////////////////////////////////////////////////
110  ///////////////////////////////////////////////////
111  //
112  // Formal properties
113  // {{{
114  ///////////////////////////////////////////////////
115  ///////////////////////////////////////////////////
116  ///////////////////////////////////////////////////
117  `ifdef FORMAL
118  ///////////////////////////////////////////////////
119  ///////////////////////////////////////////////////
120  //
121  // Interface properties
122  // {{{
123  ///////////////////////////////////////////////////
124  //
125  //
126
127  fapb_slave #(.AW(C_APB_AD-
128  DR_WIDTH), .DW(C_APB_DATA_WIDTH),
129  .F_OPT_MAXSTALL(1)
130  ) fapb (PCLK, PRESETn,
131  PSEL, PENABLE, PREADY, PADDR, PWRITE, PWDATA,
132  PWSTRB,
133  PPROT, PRDATA, PSLVERR);
134
135  always @(*)
136  if (PRESETn && PSEL && PENABLE)
137      assert(PREADY);
138  // }}}
139  ///////////////////////////////////////////////////
140  //
141  // Contract check

```

```

140  // {{{
141  ///////////////////////////////////////////////////
142  //
143  //
144  (* anyconst *) reg [AW-1:0] f_addr;
145  reg [DW-1:0] f_data;
146
147  `ifndef FORMAL
148  initial for(ik=0; ik<(1<<AW); ik=ik+1)
149      mem[ik] = 0;
150  `endif
151
152  always @(*)
153  if (!PRESETn)
154      assume(mem[f_addr[AW-1:APBLSB]] == f_data);
155
156  initial f_data = 0;
157  always @(posedge PCLK)
158  if (PRESETn && PSEL && !PENABLE && PWRITE && PAD-
159  DR[AW-1:APBLSB] == f_addr[AW-1:APBLSB])
160  begin
161      for(ik=0; ik<DW/8; ik=ik+1)
162      if (PWSTRB[ik])
163          f_data[ik*8+: 8] <= PWDATA[ik*8+: 8];
164      end
165
166  always @(posedge PCLK)
167  if (PSEL && PENABLE && PREADY && !PWRITE && PAD-
168  DR[AW-1:APBLSB] == f_addr[AW-1:APBLSB])
169      assert(PRDATA == f_data);
170
171  always @(*)
172      assert(f_data == mem[f_addr[AW-1:APBLSB]]);
173  // }}}
174  ///////////////////////////////////////////////////
175  //
176  //
177  //
178
179  reg [2:0] cvr_reads, cvr_writes, cvr_seq;
180
181  initial cvr_writes = 0;
182  always @(posedge PCLK)
183  if (!PRESETn)
184      cvr_writes <= 0;
185  else if (PSEL && PENABLE && PREADY && PWRITE && !
186  cvr_writes[2])
187      cvr_writes <= cvr_writes + 1;
188
189  initial cvr_reads = 0;
190  always @(posedge PCLK)
191  if (!PRESETn)
192      cvr_reads <= 0;
193  else if (PSEL && PENABLE && PREADY && !PWRITE && !
194  cvr_reads[2])
195      cvr_reads <= cvr_reads + 1;
196
197  always @(*)
198      cover(cvr_writes[2]);
199
200  always @(*)
201      cover(cvr_reads[2]);
202
203  initial cvr_seq = 0;
204  always @(posedge PCLK)
205  if (!PRESETn)
206      cvr_seq <= 0;
207  else if (PSEL && PENABLE && PREADY && !PWRITE && PADDR
208  == f_addr)
209  begin
210      if (cvr_seq == 0 && PRDATA == 32'h12345678)
211          cvr_seq[0] <= 1'b1;
212      if (cvr_seq[0] && PRDATA == 32'h87654321)
213          cvr_seq[1] <= 1'b1;
214      if (cvr_seq[1] && PRDATA == 32'h0)
215          cvr_seq[2] <= 1'b1;
216      end
217
218  always @(*)
219  if (PRESETn)
220  begin
221      cover(cvr_seq[0]);
222      cover(cvr_seq[1]);
223      cover(cvr_seq[2]);
224      end
225
226  always @(*)
227      cover(PRESETn && !PSEL && !PENABLE &&
228  cvr_seq[2]);
229  // }}}
230  `endif
231  // }}}
232  endmodule

```

4. ZipCPU_wb2axip/rtl/apbxclock.v (609 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:    rtl/apbxclock.v
4 // {{{
5 // Project:    WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:
8 //
9 // Creator:    Dan Gisselquist, Ph.D.
10 //             Gisselquist Technology, LLC
11 //
12 ///////////////////////////////////////////////////////////////////
13 // }}}
14 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
15 // {{{
16 // This file is part of the WB2AXIP project.
17 //
18 // The WB2AXIP project contains free software and gateway,
19 // licensed under the
20 // Apache License, Version 2.0 (the "License"). You may not
21 // use this project,
22 // or this file, except in compliance with the License. You
23 // may obtain a copy
24 // of the License at
25 // http://www.apache.org/licenses/LICENSE-2.0
26 // Unless required by applicable law or agreed to in writing,
27 // software
28 // distributed under the License is distributed on an "AS IS"
29 // BASIS, WITHOUT
30 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
31 // plied. See the
32 // License for the specific language governing permissions and
33 // limitations
34 // under the License.
35 //
36 ///////////////////////////////////////////////////////////////////
37 // }}}
38 //
39 // default_nettype    none
40 // {{{
41 // module apbxclock #(
42 //     parameter        C_APB_ADDR_WIDTH = 12,
43 //     parameter        C_APB_DATA_WIDTH = 32,
44 //     parameter [0:0]  OPT_REGISTERED = 1'b0,
45 //     localparam       AW = C_APB_ADDR_WIDTH,
46 //     localparam       DW = C_APB_DATA_WIDTH
47 // ) (
48 //     // {{{
49 //     input            wire          S_APB_PCLK,
50 //     S_PRESETn,
51 //     input            wire          S_APB_PSEL,
52 //     input            wire          S_APB_PENABLE,
53 //     output           reg          S_APB_READY,
54 //     input            wire          [AW-1:0]S_APB_PADDR,
55 //     input            wire          S_APB_PWRITE,
56 //     input            wire          [DW-1:0]S_APB_PWDATA,
57 //     input            wire          [DW/8-1:0] S_APB_PWSTRB,
58 //     input            wire          [2:0] S_APB_PPROT,
59 //     output           wire          [DW-1:0]S_APB_PRDATA,
60 //     output           wire          S_APB_PSLVERR,
61 //     //
62 //     input            wire          M_APB_PCLK,
63 //     output           reg          M_PRESETn,
64 //     output           reg          M_APB_PSEL,
65 //     output           reg          M_APB_PENABLE,
66 //     input            wire          M_APB_READY,
67 //     output           wire          [AW-1:0]M_APB_PADDR,
68 //     output           wire          M_APB_PWRITE,
69 //     output           wire          [DW-1:0]M_APB_PWDATA,
70 //     output           wire          [DW/8-1:0] M_APB_PWSTRB,
71 //     output           wire          [2:0] M_APB_PPROT,
72 //     input            wire          [DW-1:0]M_APB_PRDATA,
73 //     input            wire          M_APB_PSLVERR
74 //     // }}}
75 // );
76 //
77 // Local declarations
78 // {{{
79 // reg          reset_pipe, full_reset_pipe,
80 // full_reset;
81 //
82 // reg          s_request, s_tfr_request;
83 // reg          m_request, m_request_pipe;
84 //
85 // reg          m_ack;
86 // reg          s_ack, s_ack_pipe;
87 // reg          [DW-1:0]m_prdata;
88 // reg          m_pslverr;
89 // }}}
90
91 ///////////////////////////////////////////////////////////////////
92 //
93 // Synchronize resets
94 // {{{
95 // always @(posedge M_APB_PCLK or negedge S_PRESETn)
96 // if (!S_PRESETn)
97 //     { M_PRESETn, reset_pipe } <= 0;
98 // else
99 //     { M_PRESETn, reset_pipe } <= { reset_pipe,
100 // 1'b1 };
101 //
102 // always @(posedge S_APB_PCLK or negedge M_PRESETn)
103 // if (!M_PRESETn)
104 //     { full_reset, full_reset_pipe } <= 0;
105 // else
106 //     { full_reset, full_reset_pipe } <= { full_re-
107 // set_pipe, 1'b1 };
108 // }}}
109 //
110 // Move the request across clock domains
111 // {{{
112 // s_request
113 // {{{
114 // Step 1: Register the request
115 // Always register anything before moving cross
116 // clock domains
117 // always @(posedge S_APB_PCLK or negedge S_PRESETn)
118 // if (!S_PRESETn)
119 //     s_request <= 0;
120 // else if (S_APB_READY)
121 //     s_request <= 0;
122 // else if (S_APB_PSEL && !S_APB_PENABLE)
123 //     s_request <= 1;
124 // }}}
125 //
126 // s_tfr_request
127 // {{{
128 // Step 2: Forward requests--but only when the down-
129 // stream handshake
130 // is ready to accept them
131 // always @(posedge S_APB_PCLK or negedge S_PRESETn)
132 // if (!S_PRESETn)
133 //     s_tfr_request <= 0;
134 // else if (S_APB_READY)
135 //     s_tfr_request <= 0;
136 // else if ((S_APB_PSEL && !S_APB_PENABLE) || s_request)
137 //     s_tfr_request <= (full_reset && !s_ack);
138 // }}}
139 //
140 // m_request, m_request_pipe -- 3. Capture the request
141 // in new clk domain
142 // {{{
143 // always @(posedge M_APB_PCLK or negedge M_PRESETn)
144 // if (!M_PRESETn)
145 //     { m_request, m_request_pipe } <= 0;
146 // else
147 //     { m_request, m_request_pipe }
148 //     <= { m_request_pipe, s_tfr_re-
149 // quest };
150 // }}}
151 //
152 // M_APB_PSEL, M_APB_PENABLE -- 4. Forward the request
153 // downstream
154 // {{{
155 // always @(posedge M_APB_PCLK or negedge M_PRESETn)
156 // if (!M_PRESETn)
157 //     begin
158 //         M_APB_PSEL <= 0;
159 //         M_APB_PENABLE <= 1'b0;
160 //     end else begin
161 //         M_APB_PSEL <= m_request && !m_ack
162 //         && (!M_APB_PSEL || !M_APB_PEN-
163 // ABLE || !M_APB_READY);
164 //         M_APB_PENABLE <= M_APB_PSEL && (!M_APB_PSEL
165 // || !M_APB_PENABLE || !M_APB_READY);
166 //     end
167 // }}}
168 //
169 // m_ack
170 // {{{
171 // always @(posedge M_APB_PCLK or negedge M_PRESETn)
172 // if (!M_PRESETn)
173 //     m_ack <= 0;
174 // else if (m_request && !m_ack && M_APB_PSEL &&
175 // M_APB_READY && M_APB_PENABLE)
176 //     m_ack <= 1'b1;
177 // else if (!m_request)
178 //     m_ack <= 1'b0;
179 // }}}
180 //
181 // s_ack, s_ack_pipe

```

```

173 // {{{
174 always @(posedge S_APB_PCLK or negedge S_PRESETn)
175 if (!S_PRESETn)
176     { s_ack, s_ack_pipe } <= 0;
177 else
178     { s_ack, s_ack_pipe } <= { s_ack_pipe, m_ack };
179 // }}}
180
181 // S_APB_READY
182 // {{{
183 always @(posedge S_APB_PCLK or negedge S_PRESETn)
184 if (!S_PRESETn)
185     S_APB_READY <= 0;
186 else
187     S_APB_READY <= !S_APB_READY && s_tfr_request
&& s_ack;
188 // }}}
189
190 // m_prdata
191 // {{{
192 always @(posedge M_APB_PCLK)
193 if (M_APB_PSEL && M_APB_PREADY)
194     m_prdata <= M_APB_PRDATA;
195 // }}}
196
197 // m_pslverr
198 // {{{
199 always @(posedge M_APB_PCLK or negedge M_PRESETn)
200 if (!M_PRESETn)
201     m_pslverr <= 0;
202 else if (M_APB_PSEL && M_APB_PREADY)
203     m_pslverr <= M_APB_PSLVERR;
204 // }}}
205 // }}}
206 ///////////////////////////////////////////////////////////////////
207 //
208 // Handle the ancillary (data bearing) signals
209 // {{{
210
211 generate if (OPT_REGISTERED)
212 begin : GEN_REGISTERS
213     // {{{
214     // Register all results into the new domain
215 when
216     // crossing clock domains
217     reg [AW-1:0]m_paddr;
218     reg m_pwrite;
219     reg [DW-1:0]m_pwdata;
220     reg [DW/8-1:0] m_pwstrb;
221     reg [2:0] m_pprot;
222
223     reg [DW-1:0]s_prdata;
224     reg s_pslverr;
225
226     always @(posedge M_APB_PCLK)
227     if (m_request && !m_ack)
228     begin
229         m_paddr <= S_APB_PADDR;
230         m_pwrite <= S_APB_PWRITE;
231         m_pwdata <= S_APB_PWDATA;
232         m_pwstrb <= S_APB_PWSTRB;
233         m_pprot <= S_APB_PPROT;
234     end
235
236     assign M_APB_PADDR = m_paddr;
237     assign M_APB_PWRITE = m_pwrite;
238     assign M_APB_PWDATA = m_pwdata;
239     assign M_APB_PWSTRB = m_pwstrb;
240     assign M_APB_PPROT = m_pprot;
241
242     always @(posedge S_APB_PCLK or negedge S_PRE-
243 SETn)
244     if (!S_PRESETn)
245         s_pslverr <= 1'b0;
246     else if (s_tfr_request && s_ack
247 && (!S_APB_READY || !
248 S_APB_PENABLE))
249         s_pslverr <= m_pslverr;
250     else
251         s_pslverr <= 0;
252
253     always @(posedge S_APB_PCLK)
254     if (s_request && s_ack)
255     begin
256         s_prdata <= m_prdata;
257     end else begin
258         s_prdata <= 0;
259     end
260
261     assign S_APB_PRDATA = s_prdata;
262     assign S_APB_PSLVERR = s_pslverr;
263 // }}}
264 end else begin : NO_REGISTERING
265     // {{{
266     // Results will be stable whenever PSEL is
267 true. There's
268 // really no *need* to register them
269
270     assign M_APB_PADDR = S_APB_PADDR;
271     assign M_APB_PWRITE = S_APB_PWRITE;

```

```

268     assign M_APB_PWDATA = S_APB_PWDATA;
269     assign M_APB_PWSTRB = S_APB_PWSTRB;
270     assign M_APB_PPROT = S_APB_PPROT;
271
272     assign S_APB_PRDATA = m_prdata;
273     assign S_APB_PSLVERR = m_pslverr &&
S_APB_READY;
274 // }}}
275 end endgenerate
276 // }}}
277
278 // Make Verilator happy
279 // {{{
280 // Verilator lint_off UNUSED
281 wire unused;
282 assign unused = &{ 1'b0 };
283 // Verilator lint_on UNUSED
284 // }}}
285 ///////////////////////////////////////////////////////////////////
286 ///////////////////////////////////////////////////////////////////
287 ///////////////////////////////////////////////////////////////////
288 //
289 // Formal properties
290 // {{{
291 ///////////////////////////////////////////////////////////////////
292 ///////////////////////////////////////////////////////////////////
293 ///////////////////////////////////////////////////////////////////
294 `ifdef FORMAL
295     (* gclk *) reg gbl_clk;
296
297 ///////////////////////////////////////////////////////////////////
298 //
299 // Interface properties
300 // {{{
301 ///////////////////////////////////////////////////////////////////
302 //
303 //
304
305 fapb_slave #(
306     // {{{
307     .AW(C_APB_ADDR_WIDTH), .DW(C_APB_DATA_WIDTH),
308     .F_OPT_MAXSTALL(0), .F_OPT_ASYNC_RESET(1'b1),
309     .F_OPT_SLVERR(1'b1)
310     // }}}
311 ) fslv (
312     // {{{
313     .PCLK(S_APB_PCLK), .PRESETn(S_PRESETn),
314     .PSEL(S_APB_PSEL),
315     .PENABLE(S_APB_PENABLE),
316     .PREREADY(S_APB_PREADY),
317     .PADDR(S_APB_PADDR),
318     .PWRITE(S_APB_PWRITE), .PWDATA(S_APB_PWDATA),
319     .PWSTRB(S_APB_PWSTRB),
320     .PPROT(S_APB_PPROT),
321     .PRDATA(S_APB_PRDATA), .PSLVERR(S_APB_PSLVERR)
322     // }}}
323 );
324
325 fapb_master #(
326     // {{{
327     .AW(C_APB_ADDR_WIDTH), .DW(C_APB_DATA_WIDTH),
328     .F_OPT_MAXSTALL(0), .F_OPT_ASYNC_RESET(1'b1),
329     .F_OPT_SLVERR(1'b1)
330     // }}}
331 ) fmas (
332     // {{{
333     .PCLK(M_APB_PCLK), .PRESETn(M_PRESETn),
334     .PSEL(M_APB_PSEL), .PENABLE(M_APB_PENABLE),
335     .PREREADY(M_APB_PREADY), .PADDR(M_APB_PA
DDR),
336     .PWRITE(M_APB_PWRITE), .PWDATA(M_APB_PWDATA),
337     .PWSTRB(M_APB_PWSTRB),
338     .PPROT(M_APB_PPROT),
339     .PRDATA(M_APB_PRDATA), .PSLVERR(M_APB_PSLVERR)
340     // }}}
341 );
342
343 // }}}
344 ///////////////////////////////////////////////////////////////////
345 //
346 // Clock stability
347 // {{{
348 ///////////////////////////////////////////////////////////////////
349 //
350 //
351 reg f_past_valid_gbl;
352
353 initial f_past_valid_gbl <= 1'b0;
354 always @(posedge gbl_clk)
355     f_past_valid_gbl <= 1'b1;
356

```



```

357 always @(*)
358 if (!f_past_valid_gbl)
359     assume(!S_PRESETn);
360
361 always @(posedge gbl_clk)
362 if (!$rose(S_APB_PCLK))
363 begin
364     assume(!$rose(S_PRESETn));
365     assume($stable(S_APB_PSEL));
366     assume($stable(S_APB_PENABLE));
367     assume($stable(S_APB_PADDR));
368     assume($stable(S_APB_PWRITE));
369     assume($stable(S_APB_PWDATA));
370     assume($stable(S_APB_PWSTRB));
371     assume($stable(S_APB_PPROT));
372     //
373     if (f_past_valid_gbl && S_PRESETn)
374     begin
375         assert($stable(S_APB_PREADY));
376         if (OPT_REGISTERED)
377         begin
378             assert($stable(S_APB_PRDATA));
379             assert($stable(S_APB_PSLVERR));
380         end else if ($past(S_APB_PREADY) &&
S_APB_PREADY)
381         begin
382             assert($stable(S_APB_PRDATA));
383             assert($stable(S_APB_PSLVERR));
384         end
385     end
386 end
387
388 always @(posedge gbl_clk)
389 if (!$rose(M_APB_PCLK))
390 begin
391     if (f_past_valid_gbl)
392     begin
393         assert(!$rose(M_PRESETn));
394     end
395
396     if (f_past_valid_gbl && M_PRESETn)
397     begin
398         assert($stable(M_APB_PSEL));
399         assert($stable(M_APB_PENABLE));
400         if (OPT_REGISTERED)
401         begin
402             assert($stable(M_APB_PADDR));
403             assert($stable(M_APB_PWRITE));
404             assert($stable(M_APB_PWDATA));
405             assert($stable(M_APB_PWSTRB));
406             assert($stable(M_APB_PPROT));
407         end else if ($past(M_APB_PSEL) &&
M_APB_PSEL)
408         begin
409             assert($stable(M_APB_PADDR));
410             assert($stable(M_APB_PWRITE));
411             assert($stable(M_APB_PWDATA));
412             assert($stable(M_APB_PWSTRB));
413             assert($stable(M_APB_PPROT));
414         end
415     end
416     //
417     assume($stable(M_APB_PREADY));
418     assume($stable(M_APB_PRDATA));
419     assume($stable(M_APB_PSLVERR));
420 end
421
422 always @(posedge gbl_clk)
423 if ($past(S_APB_PSEL && !S_APB_PREADY) && $past(S_PRE-
SETn) && S_PRESETn)
424 begin
425     assume($stable(S_APB_PSEL));
426     assume($stable(S_APB_PADDR));
427     assume($stable(S_APB_PWRITE));
428     assume($stable(S_APB_PWDATA));
429     assume($stable(S_APB_PWSTRB));
430     assume($stable(S_APB_PPROT));
431 end
432
433 // }}}
434 // Induction invariants
435 // {{{
436 //
437 //
438 //
439 //
440 //
441 //
442
443 always @(*)
444 case({ S_PRESETn, reset_pipe, M_PRESETn, full_re-
set_pipe, full_reset })
445 5'b0_00_00: begin end
446 5'b1_00_00: begin end
447 5'b1_10_00: begin end
448 5'b1_11_00: begin end
449 5'b1_11_10: begin end
450 5'b1_11_11: begin end
451 default: assert(0);

```

```

452 endcase
453
454 always @(*)
455 casez({ s_request, s_tfr_request, m_request_pipe, m_re-
quest, m_ack, s_ack_pipe, s_ack })
456 7'b00_00_000: begin end
457 7'b10_00_000: begin end
458 7'b11_00_000: begin end
459 7'b11_10_000: begin end
460 7'b11_11_000: begin end
461 7'b11_11_100: begin end
462 7'b11_11_110: begin end
463 7'b11_11_111: begin end
464 7'b?0_11_111: begin end
465 7'b?0_01_111: begin end
466 7'b?0_00_111: begin end
467 7'b?0_00_011: begin end
468 7'b?0_00_001: begin end
469 7'b?0_00_000: begin end
470 default: assert(0);
471 endcase
472
473 always @(posedge S_APB_PCLK)
474 if (s_request && S_PRESETn)
475     assert(S_APB_PSEL && S_APB_PENABLE);
476
477 always @(posedge gbl_clk)
478 if (!s_tfr_request && S_PRESETn)
479     assert(!M_APB_PSEL);
480
481 always @(posedge gbl_clk)
482 if (M_APB_PSEL)
483     assert(m_request && !m_ack);
484
485 always @(posedge gbl_clk)
486 if (S_PRESETn && (m_ack || s_ack_pipe || s_ack ||
S_APB_PREADY))
487     assert(!M_APB_PSEL);
488
489 // }}}
490 // Contract check
491 // {{{
492 //
493 //
494 //
495 //
496 //
497
498 (* anyconst *) reg          fnvr_check;
499 (* anyconst *) reg          [3+DW/8+DW:0] fnvr_write;
500 (* anyconst *) reg          [AW-1:0]fnvr_addr;
501 (* anyconst *) reg          [1+DW-1:0]   fnvr_return;
502
503 always @(posedge gbl_clk)
504 if (S_APB_PSEL && fnvr_check)
505 begin
506     assume({ S_APB_PPROT, S_APB_PWSTRB, S_APB_P-
WDATA } != fnvr_write);
507     assume(S_APB_PADDR != fnvr_addr);
508 end
509
510 always @(posedge gbl_clk)
511 if (M_APB_PSEL && fnvr_check && M_PRESETn)
512 begin
513     assert({ M_APB_PPROT, M_APB_PWSTRB, M_APB_P-
WDATA } != fnvr_write);
514     assert(M_APB_PADDR != fnvr_addr);
515 end
516
517 always @(posedge gbl_clk)
518 if (M_APB_PSEL && M_PRESETn)
519 begin
520     assert(M_APB_PADDR == S_APB_PADDR);
521     assert(M_APB_PWRITE == S_APB_PWRITE);
522     assert(M_APB_PWDATA == S_APB_PWDATA);
523     assert(M_APB_PWSTRB == S_APB_PWSTRB);
524 end
525
526 always @(posedge gbl_clk)
527 if (fnvr_check && M_APB_PSEL && M_APB_PENABLE &&
M_APB_PREADY)
528     assume({ M_APB_PSLVERR, M_APB_PRDATA } !=
fnvr_return);
529
530 always @(posedge gbl_clk)
531 if (fnvr_check && m_ack)
532     assert({ m_pslverr, m_prdata } != fnvr_return);
533
534 always @(posedge gbl_clk)
535 if (fnvr_check && S_APB_PSEL && S_APB_PENABLE &&
S_APB_PREADY)
536     assert({ S_APB_PSLVERR, S_APB_PRDATA } !=
fnvr_return);
537
538 // }}}
539 //
540 //
541 //

```



```

542 // Cover checks
543 // {{{
544 //////////////////////////////////////////////////
545 //
546 //
547 reg [2:0] cvr_reads, cvr_writes;
548
549 initial cvr_writes = 0;
550 always @(posedge S_APB_PCLK)
551 if (!S_PRESETn)
552 cvr_writes <= 0;
553
554 else if (S_APB_PSEL && S_APB_PENABLE && S_APB_PREADY &&
S_APB_PWRITE
&& !cvr_writes[2])
555 cvr_writes <= cvr_writes + 1;
556
557 initial cvr_reads = 0;
558 always @(posedge S_APB_PCLK)
559 if (!S_PRESETn)
560 cvr_reads <= 0;
561 else if (S_APB_PSEL && S_APB_PENABLE && S_APB_PREADY
&& !S_APB_PWRITE
&& !cvr_reads[2])
562 cvr_reads <= cvr_reads + 1;
563
564 always @(*)
565 if (S_PRESETn)
566 begin
567 cover(cvr_writes >= 2);
568 cover(cvr_reads >= 2);
569
570 cover(cvr_writes >= 3);
571 cover(cvr_reads >= 3);
572
573 end
574
575 // }}}
576 //////////////////////////////////////////////////
577 //
578 // "Careless" assumptions
579 // {{{
580 //////////////////////////////////////////////////
581 //
582 //
583 //
584 localparam F_NCLK = 5;
585 reg [F_NCLK-1:0] gbl_sclk, gbl_mclk;
586
587 always @(posedge gbl_clk)
588 gbl_sclk <= { gbl_sclk[F_NCLK-2:0], S_APB_P-
CLK };
589
590 always @(posedge gbl_clk)
591 gbl_mclk <= { gbl_mclk[F_NCLK-2:0], M_APB_P-
CLK };
592
593 always @(posedge gbl_clk)
594 if (gbl_sclk == 0)
595 assume(S_APB_PCLK);
596 else if (&gbl_sclk)
597 assume(!S_APB_PCLK);
598
599 always @(posedge gbl_clk)
600 if (gbl_mclk == 0)
601 assume(M_APB_PCLK);
602 else if (&gbl_mclk)
603 assume(!M_APB_PCLK);
604
605 // }}}
606 `endif
607 // }}}
608 // }}}
609 endmodule

```

5. ZipCPU_wb2axip/rtl/axi2axi3.v (895 lines)

```

1 //////////////////////////////////////////////////
2 //
3 // Filename: rtl/axi2axi3.v
4 // {{{
5 // Project: WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose: Bridge from an AXI4 slave to an AXI3 master
8 //
9 // The goal here is to not lose bus resolution, capacity
or capability,
10 // while bridging from AXI4 to AXI3. The biggest problem
with such
11 // a bridge is that we'll need to break large requests
(AxLEN>15) up
12 // into smaller packets. After that, everything should
work as normal
13 // with only minor modifications for AxCACHE.
14 //
15 // Opportunity:
16 // The cost of this core is very much determined by the

```

```

number of ID's
17 // supported. It should be possible, with little extra
cost or effort,
18 // to reduce the ID space herein. This should be a topic
for future
19 // exploration.
20 //
21 // Status:
22 // This core is an unverified first draft. It has past
neither formal
23 // nor simulation testing. Therefore, it is almost cer-
tain to have bugs
24 // within it. Use it at your own risk.
25 //
26 // Creator: Dan Gisselquist, Ph.D.
27 // Gisselquist Technology, LLC
28 //
29 //////////////////////////////////////////////////
30 // }}}
31 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
32 // {{{
33 // This file is part of the WB2AXIP project.
34 //
35 // The WB2AXIP project contains free software and gateway,
licensed under the
36 // Apache License, Version 2.0 (the "License"). You may not
use this project,
37 // or this file, except in compliance with the License. You
may obtain a copy
38 // of the License at
39 // }}}
40 // http://www.apache.org/licenses/LICENSE-2.0
41 // {{{
42 // Unless required by applicable law or agreed to in writing,
software
43 // distributed under the License is distributed on an "AS IS"
BASIS, WITHOUT
44 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
plied. See the
45 // License for the specific language governing permissions and
limitations
46 // under the License.
47 //
48 //////////////////////////////////////////////////
49 //
50 `default_nettype none
51 // }}}
52 module axi2axi3 #(
53 // {{{
54 parameter C_AXI_ID_WIDTH = 1,
55 parameter C_AXI_ADDR_WIDTH = 32,
56 parameter C_AXI_DATA_WIDTH = 32
57 // }}}
58 ) (
59 // {{{
60 input wire
S_AXI_ACLK,
61 input wire
S_AXI_ARESETN,
62 //
63 // The AXI4 incoming/slave interface
64 input reg
S_AXI_AWVALID,
65 output wire
S_AXI_AWREADY,
66 input reg [C_AXI_ID_WIDTH-1:0]
S_AXI_AWID,
67 input reg [C_AXI_ADDR_WIDTH-1:0]
S_AXI_AWADDR,
68 input reg [7:0]
S_AXI_AWLEN,
69 input reg [2:0]
S_AXI_AWSIZE,
70 input reg [1:0]
S_AXI_AWBURST,
71 input reg
S_AXI_AWLOCK,
72 input reg [3:0]
S_AXI_AWCACHE,
73 input reg [2:0]
S_AXI_AWPROT,
74 input reg [3:0]
S_AXI_AWQOS,
75 //
76 //
77 input wire
S_AXI_WVALID,
78 output wire
S_AXI_WREADY,
79 input wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_WDATA,
80 input wire [C_AXI_DATA_WIDTH/8-1:0]
S_AXI_WSTRB,
81 input wire
S_AXI_WLAST,
82 //
83 //
84 output wire

```

```

S_AXI_BVALID,      85      input  wire
S_AXI_BREADY,      86      output wire  [C_AXI_ID_WIDTH-1:0]
S_AXI_BID,          87      output wire  [1:0]
S_AXI_BRESP,        88      //
89      //
90      input  wire
S_AXI_ARVALID,      91      output wire
S_AXI_ARREADY,      92      input  wire  [C_AXI_ID_WIDTH-1:0]
S_AXI_ARID,          93      input  wire  [C_AXI_ADDR_WIDTH-1:0]
S_AXI_ARADDR,       94      input  wire  [7:0]
S_AXI_ARLEN,        95      input  wire  [2:0]
S_AXI_ARSIZE,       96      input  wire  [1:0]
S_AXI_ARBURST,      97      input  wire
S_AXI_ARLOCK,       98      input  wire  [3:0]
S_AXI_ARCACHE,      99      input  wire  [2:0]
S_AXI_ARPROT,       100     input  wire  [3:0]
S_AXI_ARQOS,        101     //
102     output wire
S_AXI_RVALID,       103     input  wire
S_AXI_RREADY,       104     output wire  [C_AXI_ID_WIDTH-1:0]
S_AXI_RID,          105     output wire  [C_AXI_DATA_WIDTH-1:0]
S_AXI_RDATA,        106     output wire
S_AXI_RLAST,        107     output wire  [1:0]
S_AXI_RRESP,        108     //
109     //
110     // The AXI3 Master (outgoing) interface
111     output wire
M_AXI_AWVALID,      112     input  wire
M_AXI_AWREADY,      113     output wire  [C_AXI_ID_WIDTH-1:0]
M_AXI_AWID,         114     output wire  [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWADDR,       115     output wire  [3:0]
M_AXI_AWLEN,        116     output wire  [2:0]
M_AXI_AWSIZE,       117     output wire  [1:0]
M_AXI_AWBURST,      118     output wire  [1:0]
M_AXI_AWLOCK,       119     output wire  [3:0]
M_AXI_AWCACHE,      120     output wire  [2:0]
M_AXI_AWPROT,       121     output wire  [3:0]
M_AXI_AWQOS,        122     //
123     //
124     output wire
M_AXI_WVALID,       125     input  wire
M_AXI_WREADY,       126     output wire  [C_AXI_ID_WIDTH-1:0]
M_AXI_WID,          127     output wire  [C_AXI_DATA_WIDTH-1:0]
M_AXI_WDATA,        128     output wire  [C_AXI_DATA_WIDTH/8-1:0]
M_AXI_WSTRB,        129     output wire
M_AXI_WLAST,        130     //
131     //
132     input  wire
M_AXI_BVALID,       133     output wire
M_AXI_BREADY,       134     input  wire  [C_AXI_ID_WIDTH-1:0]
M_AXI_BID,          135     input  wire  [1:0]
M_AXI_BRESP,        136     //
137     //
138     output wire
M_AXI_ARVALID,      139     input  wire
M_AXI_ARREADY,      140     output wire  [C_AXI_ID_WIDTH-1:0]
M_AXI_ARID,

```

```

141     output wire  [C_AXI_ADDR_WIDTH-1:0]
M_AXI_ARADDR,       142     output wire  [3:0]
M_AXI_ARLEN,        143     output wire  [2:0]
M_AXI_ARSIZE,       144     output wire  [1:0]
M_AXI_ARBURST,      145     output wire  [1:0]
M_AXI_ARLOCK,       146     output wire  [3:0]
M_AXI_ARCACHE,      147     output wire  [2:0]
M_AXI_ARPROT,       148     output wire  [3:0]
M_AXI_ARQOS,        149     //
150     input  wire
M_AXI_RVALID,       151     output wire
M_AXI_RREADY,       152     input  wire  [C_AXI_ID_WIDTH-1:0]
M_AXI_RID,          153     input  wire  [C_AXI_DATA_WIDTH-1:0]
M_AXI_RDATA,        154     input  wire
M_AXI_RLAST,        155     input  wire  [1:0]
M_AXI_RRESP,        156     // }}}
157     );
158     //
159     // localparam
160     localparam ADDR_LSB = $clog2(C_AXI_DATA_WIDTH)-3;
161     localparam [0:0] OPT_LOWPPOWER = 1'b0;
162     localparam LGWFIFO = 4;
163     localparam NID = (1<<C_AXI_ID_WIDTH);
164     parameter LGFIFO = 8;
165     genvar ID;
166     //
167     //
168     //
169     //
170     //
171     // WRITE SIDE
172     // {{{
173     //
174     //
175     //
176     //
177     //
178     // Write address channel
179     // {{{
180     //
181     //
182     //
183     reg [7:0] r_awlen;
184     reg [3:0] next_wlen;
185     wire awskd_valid;
186     reg awskd_ready;
187     wire [C_AXI_ID_WIDTH-1:0] awskd_id;
188     wire [C_AXI_ADDR_WIDTH-1:0] awskd_addr;
189     wire [7:0] awskd_len;
190     wire [2:0] awskd_size;
191     wire [1:0] awskd_burst;
192     wire awskd_lock;
193     wire [3:0] awskd_cache;
194     wire [2:0] awskd_prot;
195     wire [3:0] awskd_qos;
196     reg axi_awvalid;
197     reg [C_AXI_ID_WIDTH-1:0] axi_awid;
198     reg [C_AXI_ADDR_WIDTH-1:0] axi_awaddr;
199     reg [3:0] axi_awlen;
200     reg [2:0] axi_awsz;
201     reg [1:0] axi_awburst;
202     reg [1:0] axi_awlock;
203     reg [3:0] axi_awcache;
204     reg [2:0] axi_awprot;
205     reg [3:0] axi_awqos;
206     //
207     skidbuffer #(
208         .DW(C_AXI_ADDR_WIDTH + C_AXI_ID_WIDTH + 8 + 3 + 2
209         + 1*4+3*4),
210         .OPT_OUTREG(1'b0)
211     ) awskid (
212         .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
213         .i_val-
214         id(S_AXI_AWVALID), .o_ready(S_AXI_AWREADY),
215         .i_data({ S_AXI_AWID, S_AXI_AWADDR,
S_AXI_AWLEN, S_AXI_AWSIZE, S_AXI_AWBURST,
S_AXI_AWLOCK, S_AXI_AWCACHE, S_AXI_AWPROT,

```

```

S_AXI_AWQOS }},
216         .o_valid(awskd_valid), .i_ready(awskd_ready),
217         .o_data({ awskd_id, awskd_addr,
awskd_len,
218                 awskd_size, awskd_burst,
awskd_lock,
219                 awskd_cache, awskd_prot,
awskd_qos })
220     );
221
222     always @(*)
223     begin
224         awskd_ready = 1;
225         if (r_awlen > 0)
226             awskd_ready = 0;
227         if (M_AXI_AWVALID && M_AXI_AWREADY)
228             awskd_ready = 0;
229         if (!wfifo_fill[LGWFIFO:LGWFIFO-1])
230             awskd_ready = 0;
231     end
232
233     initial axi_awvalid = 1'b0;
234     always @(posedge S_AXI_ACLK)
235     if (!S_AXI_ARESETN)
236         axi_awvalid <= 0;
237     else if (awskd_valid || r_awlen > 0)
238         axi_awvalid <= 1;
239     else if (M_AXI_AWREADY)
240         axi_awvalid <= 0;
241
242     initial r_awlen = 0;
243     initial axi_awlen = 0;
244     always @(posedge S_AXI_ACLK)
245     if (!S_AXI_ARESETN)
246     begin
247         r_awlen <= 0;
248         axi_awlen <= 0;
249     end else if (!M_AXI_AWVALID || M_AXI_AWREADY)
250     begin
251         if (r_awlen > 15)
252         begin
253             axi_awlen <= 4'd15;
254             r_awlen <= r_awlen - 8'd16;
255         end else if (r_awlen > 0)
256         begin
257             axi_awlen[3:0] <= r_awlen[3:0] - 1;
258             r_awlen <= 0;
259         end else if (awskd_valid)
260         begin
261             if (awskd_len > 15)
262             begin
263                 r_awlen <= awskd_len + 1'b1 -
264                 8'd16;
265                 axi_awlen <= 4'd15;
266             end else begin
267                 r_awlen <= 0;
268                 axi_awlen <= awskd_len[3:0];
269             end
270         end else begin
271             r_awlen <= 0;
272             axi_awlen <= 0;
273         end
274     end
275
276     always @(posedge S_AXI_ACLK)
277     if (awskd_valid && awskd_ready)
278         axi_awaddr <= awskd_addr;
279     else if (M_AXI_AWVALID && M_AXI_AWREADY)
280     begin
281         // Verilator lint_off WIDTH
282         axi_awaddr <= axi_awaddr + ((M_AXI_AWLEN + 1)
<< M_AXI_AWSIZE);
283         // Verilator lint_on WIDTH
284
285         case(M_AXI_AWSIZE)
286         3'b000: begin end
287         3'b001: axi_awaddr[0] <= 0;
288         3'b010: axi_awaddr[1:0] <= 0;
289         3'b011: axi_awaddr[2:0] <= 0;
290         3'b100: axi_awaddr[3:0] <= 0;
291         3'b101: axi_awaddr[4:0] <= 0;
292         3'b110: axi_awaddr[5:0] <= 0;
293         3'b111: axi_awaddr[6:0] <= 0;
294         endcase
295     end
296
297     initial M_AXI_AWSIZE = ADDR_LSB[2:0];
298     always @(posedge S_AXI_ACLK)
299     begin
300         if (awskd_valid && awskd_ready)
301         begin
302             axi_awid <= awskd_id;
303             axi_awsz <= awskd_size;
304             axi_awburst <= awskd_burst;
305             axi_awlock[0] <= awskd_lock;
306             axi_awcache <= awskd_cache;
307             axi_awprot <= awskd_prot;
308             axi_awqos <= awskd_qos;
309         end
310     end

```

```

311
312     if (C_AXI_DATA_WIDTH < 16)
313         axi_awsz <= 3'b000;
314     else if (C_AXI_DATA_WIDTH < 32)
315         axi_awsz[2:1] <= 2'b00;
316     else if (C_AXI_DATA_WIDTH < 128)
317         axi_awsz[2] <= 1'b0;
318
319     axi_awlock[1] <= 1'b0;
320
321     end
322
323     assign M_AXI_AWVALID = axi_awvalid;
324     assign M_AXI_AWID = axi_awid;
325     assign M_AXI_AWADDR = axi_awaddr;
326     assign M_AXI_AWLEN = axi_awlen;
327     assign M_AXI_AWSIZE = axi_awsz;
328     assign M_AXI_AWBURST = axi_awburst;
329     assign M_AXI_AWLOCK = axi_awlock;
330     assign M_AXI_AWCACHE = axi_awcache;
331     assign M_AXI_AWPROT = axi_awprot;
332     assign M_AXI_AWQOS = axi_awqos;
333     // }}}
334     //
335     // Write data channel
336     // {{{
337     //
338     //
339     //
340     wire wskd_valid;
341     reg wskd_ready, write_idle;
342     wire [C_AXI_DATA_WIDTH-1:0] wskd_data;
343     wire [C_AXI_DATA_WIDTH/8-1:0] wskd_strb;
344     wire wskd_last;
345     reg [4:0] r_wlen;
346     wire [C_AXI_ID_WIDTH-1:0] wfifo_id, raw_w-
347     fifo_id;
348     wire [3:0] wfifo_wlen, raw_w-
349     fifo_wlen;
350     reg next_wlast, r_wlast;
351     wire raw_wfifo_last;
352     wire wfifo_empty;
353     wire [LGWFIFO:0] wfifo_fill;
354     reg axi_wvalid;
355     reg [C_AXI_DATA_WIDTH-1:0] axi_wdata;
356     reg [C_AXI_DATA_WIDTH/8-1:0] axi_wstrb;
357     reg [C_AXI_ID_WIDTH-1:0] axi_wid;
358     reg axi_wlast;
359
360     skidbuffer #(
361         .DW(C_AXI_DATA_WIDTH + (C_AXI_DATA_WIDTH/8) +
362         1),
363         .OPT_OUTREG(1'b0)
364     ) wskid (
365         .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
366         .i_valid(S_AXI_WVALID), .o_ready(S_AXI_WREADY),
367         .i_data({ S_AXI_WDATA, S_AXI_WSTRB,
S_AXI_WLAST }},
368         .o_valid(wskd_valid), .i_ready(wskd_ready),
369         .o_data({ wskd_data, wskd_strb,
wskd_last })
370     );
371
372     always @(*)
373     wskd_ready = !M_AXI_WVALID || M_AXI_WREADY;
374
375     initial axi_wvalid = 0;
376     always @(posedge S_AXI_ACLK)
377     if (!S_AXI_ARESETN)
378         axi_wvalid <= 0;
379     else if (wskd_valid)
380         axi_wvalid <= 1;
381     else if (M_AXI_WREADY)
382         axi_wvalid <= 0;
383
384     initial write_idle = 0;
385     always @(posedge S_AXI_ACLK)
386     if (!S_AXI_ARESETN)
387         write_idle <= 1;
388     else if (M_AXI_WVALID && M_AXI_WREADY && M_AXI_WLAST
&& !wskd_valid)
389         write_idle <= 1;
390     else if (wskd_valid && wskd_ready)
391         write_idle <= 0;
392
393     always @(posedge S_AXI_ACLK)
394     if (!S_AXI_ARESETN && OPT_LOWPOWER)
395     begin
396         axi_wdata <= 0;
397         axi_wstrb <= 0;
398     end else if (!M_AXI_WVALID || M_AXI_WREADY)
399     begin
400         axi_wdata <= wskd_data;
401         axi_wstrb <= wskd_strb;
402         if (OPT_LOWPOWER && !wskd_valid)
403         begin
404             axi_wdata <= 0;

```

```

403         axi_wstrb <= 0;
404     end
405 end
406
407 always @(*)
408 begin
409     if (r_awlen[7:4] != 0)
410         next_wlen = 4'd15;
411     else if (r_awlen[3:0] != 0)
412         next_wlen = r_awlen[3:0];
413     else if (awskd_len[7:4] != 0)
414         next_wlen = 4'd15;
415     else
416         next_wlen = awskd_len[3:0];
417
418     if (r_awlen != 0)
419         next_wlast = (r_awlen[7:4] == 0);
420     else
421         next_wlast = (awskd_len[7:4] == 0);
422 end
423
424 sfifo #(.BW(C_AXI_ID_WIDTH+4+1), .LGFLN(LGWFI0))
425 wfifo(S_AXI_ACLK, !S_AXI_ARESETN,
426       (!M_AXI_AWVALID ||
M_AXI_AWREADY)&&(awskd_valid||(r_awlen > 0))
427       && !write_idle,
428       // The length of the next burst
429       { ((r_awlen != 0) ? M_AXI_AWID : awskd_id),
430         next_wlen,
next_wlast },
431       wfifo_full, wfifo_fill,
432       //
433       (!M_AXI_WVALID || M_AXI_WREADY),
434       { raw_wfifo_id, raw_wfifo_wlen, raw_w-
fifo_last },
435       wfifo_empty);
436
437 assign wfifo_id = (wfifo_empty) ? awskd_id : raw_w-
fifo_id;
438 assign wfifo_wlen = (wfifo_empty) ? next_wlen : raw_w-
fifo_wlen;
439
440 initial r_wlen = 0;
441 always @(posedge S_AXI_ACLK)
442 if (!S_AXI_ARESETN && OPT_LOWPOWER)
443     r_wlen <= 0;
444 else if (!M_AXI_WVALID || M_AXI_WREADY)
445 begin
446     if (r_wlen > 1)
447         r_wlen <= r_wlen - 1;
448     else if (!wfifo_empty)
449         r_wlen <= raw_wfifo_wlen + 1;
450     else if (awskd_valid && awskd_ready)
451         r_wlen <= next_wlen + 1;
452 end
453
454 initial r_wlast = 0;
455 always @(posedge S_AXI_ACLK)
456 if (!S_AXI_ARESETN && OPT_LOWPOWER)
457     r_wlast <= 0;
458 else if (!M_AXI_WVALID || M_AXI_WREADY)
459 begin
460     if (r_wlen > 0)
461         r_wlast <= (r_wlen <= 1);
462     else
463         r_wlast <= raw_wfifo_last;
464 end
465
466 always @(posedge S_AXI_ACLK)
467 if (!S_AXI_ARESETN && OPT_LOWPOWER)
468 begin
469     axi_wid <= 0;
470     axi_wlast <= 0;
471 end else if (!M_AXI_WVALID || M_AXI_WREADY)
472 begin
473     axi_wid <= wfifo_id;
474     axi_wlast <= r_wlast;
475
476     if (OPT_LOWPOWER && !wskd_valid)
477     begin
478         axi_wid <= 0;
479         axi_wlast <= 0;
480     end
481 end
482
483 assign M_AXI_WVALID = axi_wvalid;
484 assign M_AXI_WDATA = axi_wdata;
485 assign M_AXI_WSTRB = axi_wstrb;
486 assign M_AXI_WID = axi_wid;
487 assign M_AXI_WLAST = axi_wlast;
488 // }}}
489 ///////////////////////////////////////////////////////////////////
490 //
491 // Write return channel
492 // {{{
493 ///////////////////////////////////////////////////////////////////
494 //
495 //
496 wire                                     bskd_valid;

```

```

497 reg                                     bskd_ready;
498 wire [C_AXI_ID_WIDTH-1:0] bskd_id;
499 wire [1:0] bskd_resp;
500 reg [1:0] bburst_err
[0:NID-1];
501 reg [NID-1:0] wbfifo_valid;
502 reg [NID-1:0] wbfifo_last;
503 reg axi_bvalid;
504 reg [C_AXI_ID_WIDTH-1:0] axi_bid;
505 reg [1:0] axi_bresp;
506
507 skidbuffer #(.DW(C_AXI_ID_WIDTH + 2), .OPT_OUTREG(1'b0))
508 ) bskid (
509     .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
510     .i_valid(M_AXI_BVALID), .o_ready(M_AXI_BREADY),
511     .i_data({ M_AXI_BID, M_AXI_BRESP }),
512     .o_valid(bskd_valid), .i_ready(bskd_ready),
513     .o_data({ bskd_id, bskd_resp })
514 );
515
516 generate for(ID=0; ID < NID; ID = ID + 1)
517 begin : WRITE_BURST_TRACKING
518     wire wbfifo_empty,
wbfifo_full;
519     wire [LGFI0:0] wbfifo_fill;
520
521     initial wbfifo_valid[ID] = 0;
522     always @(posedge S_AXI_ACLK)
523     if (!S_AXI_ARESETN)
524         wbfifo_valid[ID] <= 0;
525     else if (!wbfifo_valid[ID] && !wbfifo_empty)
526         wbfifo_valid[ID] <= 1;
527     else if (bskd_valid && bskd_ready &&
wbfifo_valid[ID] <= !wbfifo_empty;
528
529     sfifo #(.BW(1), .LGFLN(LGFI0))
530     wfifo(S_AXI_ACLK, !S_AXI_ARESETN,
531           (M_AXI_WVALID && M_AXI_WREADY &&
&& (M_AXI_WID == ID)),
532           (r_wlen == 0) ? 1'b1 : 1'b0,
533           wbfifo_full, wbfifo_fill,
534           //
535           (!wbfifo_valid[ID] || (M_AXI_BVALID &&
&& M_AXI_BID == ID))),
536           wbfifo_last[ID], wbfifo_empty
537 );
538
539 // Verilator lint_off UNUSED
540 wire unused;
541 assign unused = &{ 1'b0, wbfifo_full,
wbfifo_fill };
542 // Verilator lint_on UNUSED
543
544 end endgenerate
545
546 always @(*)
547     bskd_ready = (!S_AXI_BVALID || S_AXI_BREADY);
548
549 initial axi_bvalid = 1'b0;
550 always @(posedge S_AXI_ACLK)
551 if (!S_AXI_ARESETN)
552     axi_bvalid <= 1'b0;
553 else if (bskd_valid && bskd_ready)
554     axi_bvalid <= wbfifo_last[bskd_id];
555 else if (M_AXI_BREADY)
556     axi_bvalid <= 1'b0;
557
558 generate for(ID=0; ID < NID; ID = ID + 1)
559 begin : WRITE_ERR_BY_ID
560
561     initial bburst_err[ID] = 2'b0;
562     always @(posedge S_AXI_ACLK)
563     if (!S_AXI_ARESETN)
564         bburst_err[ID] <= 2'b0;
565     else if (S_AXI_BVALID && S_AXI_BREADY &&
&& wbfifo_last[ID])
566         bburst_err[ID] <= 2'b0;
567     else if (bskd_valid && bskd_id == ID)
568         bburst_err[ID] <= bburst_err[ID] |
bskd_resp;
569
570 end endgenerate
571
572 initial axi_bid = 0;
573 initial axi_bresp = 0;
574 always @(posedge S_AXI_ACLK)
575 if (OPT_LOWPOWER && !S_AXI_ARESETN)
576 begin
577     axi_bid <= 0;
578     axi_bresp <= 0;
579 end else if (!S_AXI_BVALID || S_AXI_BREADY)
580 begin
581     axi_bid <= bskd_id;
582     axi_bresp <= bskd_resp | bburst_err[bskd_id];
583
584     if (OPT_LOWPOWER && !bskd_valid)
585     begin

```

```

590             axi_bid  <= 0;
591             axi_bresp <= 0;
592         end
593     end
594
595     assign S_AXI_BVALID = axi_bvalid;
596     assign S_AXI_BID   = axi_bid;
597     assign S_AXI_BRESP = axi_bresp;
598     // }}}
599     // }}}
600     ///////////////////////////////////////////////////////////////////
601     ///////////////////////////////////////////////////////////////////
602     //
603     // READ SIDE
604     // {{{
605     ///////////////////////////////////////////////////////////////////
606     ///////////////////////////////////////////////////////////////////
607     //
608     ///////////////////////////////////////////////////////////////////
609     //
610     // Read address channel
611     // {{{
612     ///////////////////////////////////////////////////////////////////
613     //
614     //
615     reg [7:0] r_arlen;
616     wire arskd_valid;
617     reg arskd_ready;
618     wire [C_AXI_ID_WIDTH-1:0] arskd_id;
619     wire [C_AXI_ADDR_WIDTH-1:0] arskd_addr;
620     wire [7:0] arskd_len;
621     wire [2:0] arskd_size;
622     wire [1:0] arskd_burst;
623     wire arskd_lock;
624     wire [3:0] arskd_cache;
625     wire [2:0] arskd_prot;
626     wire [3:0] arskd_qos;
627     reg axi_arvalid;
628     reg [C_AXI_ID_WIDTH-1:0] axi_arid;
629     reg [C_AXI_ADDR_WIDTH-1:0] axi_araddr;
630     reg [3:0] axi_arlen;
631     reg [2:0] axi_arsize;
632     reg [1:0] axi_arburst;
633     reg [1:0] axi_arlock;
634     reg [3:0] axi_arcache;
635     reg [2:0] axi_arprot;
636     reg [3:0] axi_arqos;
637
638     skidbuffer #(
639         .DW(C_AXI_ADDR_WIDTH + C_AXI_ID_WIDTH + 8 + 3 + 2
640         + 1+4+3+4),
641         .OPT_OUTREG(1'b0)
642     ) arskid (
643         .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
644         .i_val-
645         id(S_AXI_ARVALID), .o_ready(S_AXI_ARREADY),
646         .i_data({ S_AXI_ARID, S_AXI_ARADDR,
647         S_AXI_ARLEN,
648         S_AXI_ARLOCK,
649         S_AXI_ARCACHE, S_AXI_ARPROT,
650         S_AXI_ARQOS }},
651         .o_valid(arskd_valid), .i_ready(arskd_ready),
652         .o_data({ arskd_id, arskd_addr,
653         arskd_len,
654         arskd_lock,
655         arskd_size, arskd_burst,
656         arskd_cache, arskd_prot,
657         arskd_qos })
658     );
659
660     always @(*)
661     begin
662         arskd_ready = 1;
663         if (r_arlen > 0)
664             arskd_ready = 0;
665         if (M_AXI_ARVALID && M_AXI_ARREADY)
666             arskd_ready = 0;
667     end
668
669     initial axi_arvalid = 1'b0;
670     always @(posedge S_AXI_ACLK)
671     if (!S_AXI_ARESETN)
672         axi_arvalid <= 1'b0;
673     else if (r_arlen > 0)
674         axi_arvalid <= 1'b1;
675     else if (arskd_ready)
676         axi_arvalid <= 1'b1;
677     else if (M_AXI_ARREADY)
678         axi_arvalid <= 1'b0;
679
680     initial r_arlen = 0;
681     initial M_AXI_ARLEN = 0;
682     always @(posedge S_AXI_ACLK)

```

```

676     if (!S_AXI_ARESETN)
677     begin
678         r_arlen <= 0;
679         axi_arlen <= 0;
680     end else if (!M_AXI_ARVALID || M_AXI_ARREADY)
681     begin
682         if (r_arlen[7:4] != 0)
683         begin
684             axi_arlen <= 4'd15;
685             r_arlen <= r_arlen - 16;
686         end else if (r_arlen[3:0] != 0)
687         begin
688             axi_arlen <= r_arlen[3:0] - 1;
689             r_arlen <= 0;
690         end else if (arskd_valid)
691         begin
692             if (arskd_len[7:4] != 0)
693             begin
694                 r_arlen <= arskd_len + 1 - 16;
695                 axi_arlen <= 4'd15;
696             end else begin
697                 r_arlen <= 0;
698                 axi_arlen <= arskd_len[3:0];
699             end
700         end else begin
701             r_arlen <= 0;
702             axi_arlen <= 0;
703         end
704     end
705
706     always @(posedge S_AXI_ACLK)
707     if (arskd_valid && arskd_ready)
708         axi_araddr <= arskd_addr;
709     else if (M_AXI_ARVALID && M_AXI_ARREADY)
710     begin
711         // Verilator lint_off WIDTH
712         axi_araddr <= axi_araddr + ((M_AXI_ARLEN + 1)
713         << M_AXI_ARSIZE);
714         // Verilator lint_on WIDTH
715
716         case(M_AXI_AWSIZE)
717         3'b000: begin end
718         3'b001: axi_araddr[ 0] <= 0;
719         3'b010: axi_araddr[1:0] <= 0;
720         3'b011: axi_araddr[2:0] <= 0;
721         3'b100: axi_araddr[3:0] <= 0;
722         3'b101: axi_araddr[4:0] <= 0;
723         3'b110: axi_araddr[5:0] <= 0;
724         3'b111: axi_araddr[6:0] <= 0;
725         endcase
726
727     end
728
729     initial axi_arsize = ADDR_LSB[2:0];
730     always @(posedge S_AXI_ACLK)
731     begin
732         if (arskd_valid && arskd_ready)
733         begin
734             axi_arid <= arskd_id;
735             axi_arsize <= arskd_size;
736             axi_arburst <= arskd_burst;
737             axi_arlock[0] <= arskd_lock;
738             axi_arcache <= arskd_cache;
739             axi_arprot <= arskd_prot;
740             axi_arqos <= arskd_qos;
741         end
742
743         // Propagate constants, to help the optimizer
744         out out a touch
745         axi_arlock[1] <= 1'b0;
746
747         if (C_AXI_DATA_WIDTH < 16)
748             axi_arsize <= 3'b000;
749         else if (C_AXI_DATA_WIDTH < 32)
750             axi_arsize[2:1] <= 2'b00;
751         else if (C_AXI_DATA_WIDTH < 128)
752             axi_arsize[2] <= 1'b0;
753     end
754
755     generate for(ID=0; ID < NID; ID = ID + 1)
756     begin : READ_BURST_TRACKING
757         wire rbfifo_empty, rb-
758         fifo_full;
759         wire [LGFIFO:0] rbfifo_fill;
760
761         initial rbfifo_valid[ID] = 0;
762         always @(posedge S_AXI_ACLK)
763         if (!S_AXI_ARESETN)
764             rbfifo_valid[ID] <= 0;
765         else if (!rbfifo_valid[ID] && !rbfifo_empty)
766             rbfifo_valid[ID] <= 1;
767         else if (rskd_valid && rskd_ready && rskd_last
768         && rskd_id==ID)
769             rbfifo_valid[ID] <= !rbfifo_empty;
770
771         sfifo #(.BW(1), .LGFIFO(LGFIFO))
772         rbfifo(S_AXI_ACLK, !S_AXI_ARESETN,
773         (!M_AXI_ARVALID || M_AXI_ARREADY)
774         && ((r_ar-
775         len>0)&&(M_AXI_ARID==ID))
776         || (arskd_valid &&

```

```

rskd_id == ID)),
772                ((rskd_valid && rskd_ready &&
rskd_len <= 15)
773                || (r_arlen <= 15)) ? 1'b1 :
1'b0,
774                rbfifo_full, rbfifo_fill,
775                //
776                (!rbfifo_valid[ID] || (M_AXI_RVALID &&
M_AXI_RREADY
777                && M_AXI_RID == ID &&
M_AXI_RLAST))),
778                rid_last[ID], rbfifo_empty
779            );
780            // Verilator lint_off UNUSED
781            wire unused;
782            assign unused = &{ 1'b0, rbfifo_full, rb-
fifo_fill };
783            // Verilator lint_on UNUSED
784            end endgenerate
785
786            assign M_AXI_ARVALID= axi_arvalid;
787            assign M_AXI_ARID   = axi_arid;
788            assign M_AXI_ARADDR = axi_araddr;
789            assign M_AXI_ARLEN  = axi_arlen;
790            assign M_AXI_ARSIZE = axi_arsize;
791            assign M_AXI_ARBURST= axi_arburst;
792            assign M_AXI_ARLOCK = axi_arlock;
793            assign M_AXI_ARCACHE= axi_arcache;
794            assign M_AXI_ARPROT = axi_arprot;
795            assign M_AXI_ARQOS  = axi_arqos;
796            // }}}
797            ///////////////////////////////////////////////////////////////////
798            ///////////////////////////////////////////////////////////////////
799            //
800            // Read data channel
801            // {{{
802            ///////////////////////////////////////////////////////////////////
803            //
804            //
805            wire                rskd_valid;
806            reg                 rskd_ready;
807            wire [C_AXI_ID_WIDTH-1:0] rskd_id;
808            wire [C_AXI_DATA_WIDTH-1:0] rskd_data;
809            wire                rskd_last;
810            wire                rskd_resp;
811            reg                 rburst_err
812
813            [0:NID-1];
814            reg [NID-1:0] rbfifo_valid;
815            reg [NID-1:0] rid_last;
816            reg axi_rvalid;
817            reg [C_AXI_ID_WIDTH-1:0] axi_rid;
818            reg [C_AXI_DATA_WIDTH-1:0] axi_rdata;
819            reg axi_rlast;
820            reg [1:0] axi_rresp;
821
822            skidbuffer #(
823                .DW(C_AXI_DATA_WIDTH + C_AXI_ID_WIDTH + 3),
824                .OPT_OUTREG(1'b0)
825            ) rskid (
826                .i_clk(S_AXI_ACLK), .i_reset(!S_AXI_ARESETN),
827                .i_valid(M_AXI_RVALID), .o_ready(M_AXI_RREADY),
828                .i_data({ M_AXI_RID, M_AXI_RDATA,
M_AXI_RLAST,
829                M_AXI_RRESP }),
830                .o_valid(rskd_valid), .i_ready(rskd_ready),
831                .o_data({ rskd_id, rskd_data,
rskd_last, rskd_resp })
832            );
833            always @(*)
834                rskd_ready = !S_AXI_RVALID || S_AXI_RREADY;
835
836            initial axi_rvalid = 1'b0;
837            always @(posedge S_AXI_ACLK)
838                if (!S_AXI_ARESETN)
839                    axi_rvalid <= 1'b0;
840            else if (rskd_valid && rskd_ready)
841                axi_rvalid <= 1'b1;
842            else if (S_AXI_RREADY)
843                axi_rvalid <= 1'b0;
844
845            generate for(ID=0; ID < NID; ID = ID + 1)
846                begin : READ_ERR_BY_ID
847
848                    initial rburst_err[ID] = 2'b0;
849                    always @(posedge S_AXI_ACLK)
850                        if (!S_AXI_ARESETN)
851                            rburst_err[ID] <= 2'b0;
852                    else if (S_AXI_RVALID && S_AXI_RREADY &&
S_AXI_RLAST
853                            && S_AXI_RID == ID)
854                        rburst_err[ID] <= 2'b0;
855                    else if (rskd_valid && rskd_id == ID)
856                        rburst_err[ID] <= rburst_err[ID] |
rskd_resp;
857
858                    end endgenerate
859
860                    always @(posedge S_AXI_ACLK)

```

```

860            if (!S_AXI_RVALID || S_AXI_RREADY)
861                begin
862                    axi_rid <= rskd_id;
863                    axi_rdata <= rskd_data;
864                    axi_rresp <= rskd_resp | rburst_err[rskd_id];
865                    axi_rlast <= rid_last[rskd_id] && rskd_last;
866                end
867
868            assign S_AXI_RVALID= axi_rvalid;
869            assign S_AXI_RID   = axi_rid;
870            assign S_AXI_RDATA = axi_rdata;
871            assign S_AXI_RRESP = axi_rresp;
872            assign S_AXI_RLAST = axi_rlast;
873            // }}}
874            // }}}
875            // Make Verilator happy
876            // {{{
877            // Verilator lint_off UNUSED
878            wire unused;
879            assign unused = &{ 1'b0, wfifo_fill[LGWFIPO-2:0],
wskd_last,
880                wfifo_wlen, wfifo_full };
881            // Verilator lint_on UNUSED
882            // }}}
883            ///////////////////////////////////////////////////////////////////
884            ///////////////////////////////////////////////////////////////////
885            //
886            // Formal property section
887            //
888            ///////////////////////////////////////////////////////////////////
889            ///////////////////////////////////////////////////////////////////
890            `ifdef FORMAL
891            //
892            // This design has not been formally verified.
893            //
894            `endif
895        endmodule

```

6. ZipCPU_wb2axip/rtl/axi2axilite.v (1,172 lines)

```

1 ///////////////////////////////////////////////////////////////////
2 //
3 // Filename:    rtl/axi2axilite.v
4 // {{{
5 // Project:    WB2AXIPSP: bus bridges and other odds and ends
6 //
7 // Purpose:    Convert from AXI to AXI-lite with no perform-
ance loss.
8 //
9 // Performance: The goal of this converter is to convert from
AXI to AXI-lite
10 // while still maintaining the one-clock per
transaction speed
11 // of AXI. It currently achieves this goal. The design
needs very little
12 // configuration to be useful, but you might wish to res-
ize the FIFOs
13 // within depending upon the length of your slave's data
path. The current
14 // FIFO length, LGWFIPO=4, is sufficient to maintain full
speed. If the
15 // slave, however, can maintain full speed but requires a
longer
16 // processing cycle, then you may need longer FIFOs.
17 //
18 // The AXI specification does require an additional 2
clocks per
19 // transaction when using this core, so your latency will
go up.
20 //
21 // Related:    There's a related axidouple.v core in the same
repository as
22 // well. That can be used to convert the AXI pro-
tocol to something
23 // simpler (even simpler than AXI-lite), but it can also
do so for multiple
24 // downstream slaves at the same time.
25 //
26 // Creator:    Dan Gisselquist, Ph.D.
Gisselquist Technology, LLC
27 //
28 //
29 ///////////////////////////////////////////////////////////////////
30 // }}}
31 // Copyright (C) 2019-2025, Gisselquist Technology, LLC
32 // {{{
33 // This file is part of the WB2AXIP project.
34 //
35 // The WB2AXIP project contains free software and gateware,
licensed under the
36 // Apache License, Version 2.0 (the "License"). You may not
use this project,
37 // or this file, except in compliance with the License. You
may obtain a copy
38 // of the License at

```



```

39 // }}}
40 // http://www.apache.org/licenses/LICENSE-2.0
41 // {{{
42 // Unless required by applicable law or agreed to in writing,
software
43 // distributed under the License is distributed on an "AS IS"
BASIS, WITHOUT
44 // WARRANTIES OR CONDITIONS OF ANY KIND, either express or im-
plied. See the
45 // License for the specific language governing permissions and
limitations
46 // under the License.
47 //
48 ///////////////////////////////////////////////////////////////////
49 //
50 `default_nettype      none
51 // }}}
52 module axi2axilite #(
53     // {{{
54     parameter integer C_AXI_ID_WIDTH= 2,
55     parameter integer C_AXI_DATA_WIDTH  = 32,
56     parameter integer C_AXI_ADDR_WIDTH  = 6,
57     parameter [0:0] OPT_WRITES  = 1,
58     parameter [0:0] OPT_READS   = 1,
59     parameter [0:0] OPT_LOWPPOWER = 0,
60     // Log (based two) of the maximum number of
outstanding AXI
61     // (not AXI-lite) transactions. If you mul-
tiply 2^LGIFIFO * 256,
62     // you'll get the maximum number of outstanding
AXI-lite
63     // transactions
64     parameter LGIFIFO = 4
65     // }}}
66 ) (
67     // {{{
68     input wire S_AXI_ACLK,
69     input wire S_AXI_ARESETN,
70     // AXI4 slave interface
71     // {{{
72     // Write address channel
73     // {{{
74     input wire
S_AXI_AWVALID,
75     output wire
S_AXI_AWREADY,
76     input wire [C_AXI_ID_WIDTH-1:0]
S_AXI_AWID,
77     input wire [C_AXI_ADDR_WIDTH-1:0]
S_AXI_AWADDR,
78     input wire [7:0]
S_AXI_AWLEN,
79     input wire [2:0]
S_AXI_AWSIZE,
80     input wire [1:0]
S_AXI_AWBURST,
81     input wire
S_AXI_AWLOCK,
82     input wire [3:0]
S_AXI_AWCACHE,
83     input wire [2:0]
S_AXI_AWPROT,
84     input wire [3:0]
S_AXI_AWQOS,
85     // }}}
86     // Write data channel
87     // {{{
88     input wire
S_AXI_WVALID,
89     output wire
S_AXI_WREADY,
90     input wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_WDATA,
91     input wire [(C_AXI_DATA_WIDTH/8)-1:0]
S_AXI_WSTRB,
92     input wire
S_AXI_WLAST,
93     // }}}
94     // Write return channel
95     // {{{
96     output wire
S_AXI_BVALID,
97     input wire
S_AXI_BREADY,
98     output wire [C_AXI_ID_WIDTH-1:0]
S_AXI_BID,
99     output wire [1:0]
S_AXI_BRESP,
100     // }}}
101     // Read address channel
102     // {{{
103     input wire
S_AXI_ARVALID,
104     output wire
S_AXI_ARREADY,
105     input wire [C_AXI_ID_WIDTH-1:0]
S_AXI_ARID,
106     input wire [C_AXI_ADDR_WIDTH-1:0]

```

```

S_AXI_ARADDR,
107     input wire [7:0]
S_AXI_ARLEN,
108     input wire [2:0]
S_AXI_ARSIZE,
109     input wire [1:0]
S_AXI_ARBURST,
110     input wire
S_AXI_ARLOCK,
111     input wire [3:0]
S_AXI_ARCACHE,
112     input wire [2:0]
S_AXI_ARPROT,
113     input wire [3:0]
S_AXI_ARQOS,
114     // }}}
115     // Read data channel
116     // {{{
117     output wire
S_AXI_RVALID,
118     input wire
S_AXI_RREADY,
119     output wire [C_AXI_ID_WIDTH-1:0] S_AXI_RID,
120     output wire [C_AXI_DATA_WIDTH-1:0]
S_AXI_RDATA,
121     output wire [1:0]
S_AXI_RRESP,
122     output wire
S_AXI_RLAST,
123     // }}}
124     // }}}
125     // AXI-lite master interface
126     // {{{
127     // AXI-lite Write interface
128     // {{{
129     output wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_AWADDR,
130     output wire [2 : 0]
M_AXI_AWPROT,
131     output wire
M_AXI_AWVALID,
132     input wire
M_AXI_AWREADY,
133     output wire [C_AXI_DATA_WIDTH-1:0]
M_AXI_WDATA,
134     output wire [(C_AXI_DATA_WIDTH/8)-1:0]
M_AXI_WSTRB,
135     output wire
M_AXI_WVALID,
136     input wire
M_AXI_WREADY,
137     input wire [1 : 0]
M_AXI_BRESP,
138     input wire
M_AXI_BVALID,
139     output wire
M_AXI_BREADY,
140     // }}}
141     // AXI-lite read interface
142     // {{{
143     output wire [C_AXI_ADDR_WIDTH-1:0]
M_AXI_ARADDR,
144     output wire [2:0]
M_AXI_ARPROT,
145     output wire
M_AXI_ARVALID,
146     input wire
M_AXI_ARREADY,
147     //
148     input wire
M_AXI_RVALID,
149     output wire
M_AXI_RREADY,
150     input wire [C_AXI_DATA_WIDTH-1 : 0]
M_AXI_RDATA,
151     input wire [1 : 0]
M_AXI_RRESP
152     // }}}
153     // }}}
154     // }}}
155 );
156
157 // Local parameters, register, and net declarations
158 // {{{
159 localparam [1:0]SLVERR = 2'b10;
160 // localparam [1:0] OKAY = 2'b00,
161 // EXOKAY = 2'b01,
162 // DECERR = 2'b10;
163 localparam AW = C_AXI_ADDR_WIDTH;
164 localparam DW = C_AXI_DATA_WIDTH;
165 localparam IW = C_AXI_ID_WIDTH;
166 // }}}
167 // Register declarations
168 // {{{
169 //
170 // Write registers
171 reg m_axi_awvalid,
s_axi_wready;
172 reg [C_AXI_ADDR_WIDTH-1:0] axi_awaddr;
173 reg [7:0] axi_awlen, axi_blen;
174 reg [1:0] axi_awburst;

```



```

175 reg [2:0] axi_awsz;
176 wire [C_AXI_ADDR_WIDTH-1:0] next_write_addr;
177 wire [4:0] wfifo_count;
178 wire wfifo_full;
179 wire wfifo_empty;
180 wire [7:0] wfifo_bcount;
181 wire [IW-1:0] wfifo_bid;
182 reg [8:0] bcounts;
183 reg [C_AXI_ID_WIDTH-1:0] axi_bid, bid;
184 reg [1:0] axi_bresp;
185 reg s_axi_bvalid;
186 wire read_from_wrfifo;
187 //
188 // Read register
189 reg m_axi_arvalid;
190 wire [4:0] rfifo_count;
191 wire rfifo_full;
192 wire rfifo_empty;
193 wire [7:0] rfifo_rcount;
194 reg s_axi_rvalid;
195 reg [1:0] s_axi_rresp;
196 reg [8:0] rcounts;
197 reg [C_AXI_ADDR_WIDTH-1:0] axi_araddr;
198 reg [7:0] axi_arlen, axi_rlen;
199 reg [1:0] axi_arburst;
200 reg [2:0] axi_arsize;
201 wire [C_AXI_ADDR_WIDTH-1:0] next_read_addr;
202 reg [C_AXI_ID_WIDTH-1:0] s_axi_rid;
203 wire [C_AXI_ID_WIDTH-1:0] rfifo_rid;
204 reg [C_AXI_DATA_WIDTH-1:0] s_axi_rdata;
205 reg s_axi_rlast;
206 reg [IW-1:0] rid;
207 wire read_from_rdfifo;
208 //
209 // S_AXI_AW* skid buffer
210 wire skids_awvalid, skids_awready;
211 wire [IW-1:0] skids_awid;
212 wire [AW-1:0] skids_awaddr;
213 wire [7:0] skids_awlen;
214 wire [2:0] skids_awsz;
215 wire [1:0] skids_awburst;
216 //
217 // S_AXI_W* skid buffer
218 wire skids_wvalid, skids_wready,
skids_wlast;
219 wire [DW-1:0] skids_wdata;
220 wire [DW/8-1:0] skids_wstrb;
221 //
222 // S_AXI_B* skid buffer isn't needed
223 //
224 // M_AXI_AW* skid buffer isn't needed
225 //
226 // M_AXI_W* skid buffer
227 wire skidm_wvalid, skidm_wready;
228 wire [DW-1:0] skidm_wdata;
229 wire [DW/8-1:0] skidm_wstrb;
230 //
231 // M_AXI_B* skid buffer
232 wire skidm_bvalid, skidm_bready;
233 wire [1:0] skidm_bresp;
234 //
235 //
236 //
237 //
238 // S_AXI_AR* skid buffer
239 wire skids_arvalid, skids_arready;
240 wire [IW-1:0] skids_arid;
241 wire [AW-1:0] skids_araddr;
242 wire [7:0] skids_arlen;
243 wire [2:0] skids_arsize;
244 wire [1:0] skids_arburst;
245 //
246 // S_AXI_R* skid buffer isn't needed
247 //
248 // M_AXI_AR* skid buffer isn't needed
249 // M_AXI_R* skid buffer
250 wire skidm_rvalid, skidm_rready;
251 wire [DW-1:0] skidm_rdata;
252 wire [1:0] skidm_rresp;
253 // }}}
254 // }}}
255 //
256 // Write logic
257 // {{{
258 // }}}
259 //
260 //
261 generate if (OPT_WRITES)
262 begin : IMPLEMENT_WRITES
263 // {{{
264 // The write address channel's skid buffer
265 // {{{
266 skidbuffer #(
267 // {{{
268 .DW(IW+AW+8+3+2), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
269 // }}}
270 ) awskid(
271 // {{{

```

```

272 S_AXI_ACLK, !S_AXI_ARESETN,
273 S_AXI_AWVALID, S_AXI_AWREADY,
274 { S_AXI_AWID, S_AXI_AWADDR,
S_AXI_AWLEN, S_AXI_AWSIZE,
275 S_AXI_AWBURST },
276 skids_awvalid, skids_awready,
277 { skids_awid, skids_awaddr,
skids_awlen, skids_awsz,
278 skids_awburst }
279 // }}}
280 );
281 // }}}
282 //
283 // The write data channel's skid buffer
(S_AXI_W*)
284 // {{{
285 skidbuffer #(
286 // {{{
287 .DW(DW+DW/8+1), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
288 // }}}
289 ) wskid(
290 // {{{
291 S_AXI_ACLK, !S_AXI_ARESETN,
292 S_AXI_WVALID, S_AXI_WREADY,
293 { S_AXI_WDATA, S_AXI_WSTRB, S_AXI_W-
LAST },
294 skids_wvalid, skids_wready,
295 { skids_wdata, skids_wstrb,
skids_wlast }
296 // }}}
297 );
298 // }}}
299 //
300 // The downstream AXI-lite write data
(M_AXI_W*) skid buffer
301 // {{{
302 skidbuffer #(
303 // {{{
304 .DW(DW+DW/8), .OPT_LOW-
POWER(0), .OPT_OUTREG(1)
305 // }}}
306 ) mwskid(
307 // {{{
308 S_AXI_ACLK, !S_AXI_ARESETN,
309 skidm_wvalid, skidm_wready,
{ skidm_wdata, skidm_wstrb },
310 M_AXI_WVALID, M_AXI_WREADY,
{ M_AXI_WDATA, M_AXI_WSTRB }
311 // }}}
312 );
313 // }}}
314 //
315 // The downstream AXI-lite response (M_AXI_B*)
skid buffer
316 // {{{
317 skidbuffer #(
318 // {{{
319 .DW(2), .OPT_LOW-
POWER(0), .OPT_OUTREG(0)
320 // }}}
321 ) bskid(
322 // {{{
323 S_AXI_ACLK, !S_AXI_ARESETN,
324 M_AXI_BVALID, M_AXI_BREADY,
{ M_AXI_BRESP },
325 skidm_bvalid, skidm_bready,
{ skidm_bresp }
326 // }}}
327 );
328 // }}}
329 //
330 // m_axi_awvalid
331 // {{{
332 initial m_axi_awvalid = 0;
333 always @(posedge S_AXI_ACLK)
334 if (!S_AXI_ARESETN)
335 m_axi_awvalid <= 0;
336 else if (skids_awvalid & skids_awready)
337 m_axi_awvalid <= 1;
338 else if (M_AXI_AWREADY && axi_awlen == 0)
339 m_axi_awvalid <= 0;
340 //
341 assign M_AXI_AWVALID = m_axi_awvalid;
342 // }}}
343 //
344 // skids_awready
345 // {{{
346 assign skids_awready = (!M_AXI_AWVALID
347 || ((axi_awlen ==
0)&&M_AXI_AWREADY))
348 && !wfifo_full
349 && (!s_axi_wready ||
(skids_wvalid && skids_wlast && skids_wready));
350 // }}}
351 //
352 // Address processing
353 // {{{
354 always @(posedge S_AXI_ACLK)
355 if (skids_awvalid && skids_awready)
356 begin

```

```

357         axi_awaddr <= skids_awaddr;
358         axi_blen <= skids_awlen;
359         axi_awburst <= skids_awburst;
360         axi_awsz <= skids_awsz;
361     end else if (M_AXI_AWVALID && M_AXI_AWREADY)
362         axi_awaddr <= next_write_addr;
363     // }}}
364
365     // axi_awlen
366     // {{{
367     initial axi_awlen = 0;
368     always @(posedge S_AXI_ACLK)
369     if (!S_AXI_ARESETN)
370         axi_awlen <= 0;
371     else if (skids_awvalid && skids_awready)
372         axi_awlen <= skids_awlen;
373     else if (M_AXI_AWVALID && M_AXI_AWREADY &&
374         axi_awlen > 0)
375         axi_awlen <= axi_awlen - 1;
376     // }}}
377
378     // axi_addr
379     // {{{
380     axi_addr #(
381         // {{{
382         .AW(C_AXI_AD-
383             DR_WIDTH), .DW(C_AXI_DATA_WIDTH)
384         ) calcwaddr(
385             // {{{
386             axi_awaddr, axi_awsz, axi_awburst,
387             axi_blen, next_write_addr
388             // }}}
389         );
390     // }}}
391
392     // s_axi_wready
393     // {{{
394     // We really don't need to do anything special
395     // channel.
396     initial s_axi_wready = 0;
397     always @(posedge S_AXI_ACLK)
398     if (!S_AXI_ARESETN)
399         s_axi_wready <= 0;
400     else if (skids_awvalid && skids_awready)
401         s_axi_wready <= 1;
402     else if (skids_wvalid && skids_wready &&
403         skids_wlast)
404         s_axi_wready <= 0;
405     // }}}
406
407     // skidm*, and read_from_wrfifo
408     // {{{
409     assign skidm_wdata = skids_wdata;
410     assign skidm_wstrb = skids_wstrb;
411     assign skidm_wvalid = skids_wvalid &&
412         s_axi_wready;
413     assign skids_wready = s_axi_wready &&
414         skidm_wready;
415
416     assign read_from_wrfifo = (bcounts <= 1)&&(!
417         wfifo_empty)
418         &&(skidm_bvalid && skidm_bready);
419     // }}}
420
421     // BFIFO
422     // {{{
423     sfifo #(
424         .BW(C_AXI_ID_WIDTH+8), .LGFLN(LGFIFO)
425     ) bidlnfifo(
426         S_AXI_ACLK, !S_AXI_ARESETN,
427         skids_awvalid && skids_awready,
428         { skids_awid, skids_awlen },
429         wfifo_full, wfifo_count,
430         read_from_wrfifo,
431         { wfifo_bid, wfifo_bcount },
432         wfifo_empty);
433     // }}}
434
435     // bcounts
436     // {{{
437     // Return counts
438     initial bcounts = 0;
439     always @(posedge S_AXI_ACLK)
440     if (!S_AXI_ARESETN)
441         bcounts <= 0;
442     else if (read_from_wrfifo)
443         begin
444             bcounts <= wfifo_bcount + bcounts;
445         end else if (skidm_bvalid && skidm_bready)
446             bcounts <= bcounts - 1;
447     // }}}
448
449     // bid
450     // {{{
451     always @(posedge S_AXI_ACLK)
452     if (read_from_wrfifo)
453         bid <= wfifo_bid;
454     // }}}
455
456     always @(posedge S_AXI_ACLK)
457         if (!S_AXI_BVALID || S_AXI_BREADY)
458             axi_bid <= (read_from_wrfifo &&
459                 bcounts==0) ? wfifo_bid : bid;
460         // }}}
461
462         // s_axi_bvalid
463         // {{{
464         initial s_axi_bvalid = 0;
465         always @(posedge S_AXI_ACLK)
466         if (!S_AXI_ARESETN)
467             s_axi_bvalid <= 0;
468         else if (skidm_bvalid && skidm_bready)
469             s_axi_bvalid <= (bcounts == 1)
470                 ||(bcounts == 0) && (!
471                 wfifo_empty) && (wfifo_bcount == 0));
472         else if (S_AXI_BREADY)
473             s_axi_bvalid <= 0;
474         // }}}
475
476         // axi_bresp
477         // {{{
478         initial axi_bresp = 0;
479         always @(posedge S_AXI_ACLK)
480         if (!S_AXI_ARESETN)
481             axi_bresp <= 0;
482         else if (S_AXI_BVALID && S_AXI_BREADY)
483             begin
484                 if (skidm_bvalid && skidm_bready)
485                     axi_bresp <= skidm_bresp;
486                 else
487                     axi_bresp <= 0;
488             end else if (skidm_bvalid && skidm_bready)
489             begin
490                 // Let SLVERR take priority over DECERR
491                 casez({ S_AXI_BRESP, skidm_bresp })
492                 4'b???0: axi_bresp <= S_AXI_BRESP;
493                 4'b0?1?: axi_bresp <= skidm_bresp;
494                 4'b1?10: axi_bresp <= SLVERR;
495                 4'b1011: axi_bresp <= SLVERR;
496                 4'b1111: axi_bresp <= skidm_bresp;
497                 endcase
498             end
499         // }}}
500
501         // M_AXI_AW*
502         // {{{
503         assign M_AXI_AWVALID= m_axi_awvalid;
504         assign M_AXI_AWADDR = axi_awaddr;
505         assign M_AXI_AWPROT = 0;
506         // }}}
507
508         // skidm_bready, S_AXI_B*
509         // {{{
510         assign skidm_bready = ((bcounts > 0)||(!
511             wfifo_empty))&&(!S_AXI_BVALID | S_AXI_BREADY);
512         assign S_AXI_BID = axi_bid;
513         assign S_AXI_BRESP = axi_bresp;
514         assign S_AXI_BVALID = s_axi_bvalid;
515         // }}}
516
517         end else begin : NO_WRITE_SUPPORT
518             // {{{
519             assign S_AXI_AWREADY = 0;
520             assign S_AXI_WREADY = 0;
521             assign S_AXI_BID = 0;
522             assign S_AXI_BRESP = 2'b11;
523             assign S_AXI_BVALID = 0;
524             assign S_AXI_BID = 0;
525
526             //
527             assign M_AXI_AWVALID = 0;
528             assign M_AXI_AWADDR = 0;
529             assign M_AXI_AWPROT = 0;
530
531             //
532             assign M_AXI_WVALID = 0;
533             assign M_AXI_WDATA = 0;
534             assign M_AXI_WSTRB = 0;
535
536             //
537             assign M_AXI_BREADY = 0;
538
539             //
540             // S_AXI_AW* skid buffer
541             assign skids_awvalid = 0;
542             assign skids_awready = 0;
543             assign skids_awid = 0;
544             assign skids_awaddr = 0;
545             assign skids_awlen = 0;
546             assign skids_awsz = 0;
547             assign skids_awburst = 0;
548
549             //
550             // S_AXI_W* skid buffer
551             assign skids_wvalid = S_AXI_WVALID;
552             assign skids_wready = S_AXI_WREADY;
553             assign skids_wdata = S_AXI_WDATA;
554             assign skids_wstrb = S_AXI_WSTRB;
555             assign skids_wlast = S_AXI_WLAST;
556
557             //
558             // S_AXI_B* skid buffer isn't needed
559             //
560             // M_AXI_AW* skid buffer isn't needed
561             //

```

```

548 // M_AXI_W* skid buffer
549 assign skidm_wvalid = M_AXI_WVALID;
550 assign skidm_wready = M_AXI_WREADY;
551 assign skidm_wdata = M_AXI_WDATA;
552 assign skidm_wstrb = M_AXI_WSTRB;
553 //
554 // M_AXI_B* skid buffer
555 assign skidm_bvalid = M_AXI_BVALID;
556 assign skidm_bready = M_AXI_BREADY;
557 assign skidm_bresp = M_AXI_BRESP;
558 //
559 //
560 always @(*)
561 begin
562     s_axi_wready = 0;
563
564     axi_awlen = 0;
565     bcounts = 0;
566     bid = 0;
567     axi_bresp = 0;
568     axi_bid = 0;
569
570 end
571
572 assign wfifo_full = 0;
573 assign wfifo_empty = 1;
574 assign wfifo_count = 0;
575 assign read_from_wrfifo = 0;
576
577 // Make Verilator happy
578 // {{{
579 // Verilator lint_off UNUSED
580 wire unused_write_signals;
581 assign unused_write_signals = { 1'b0,
M_AXI_AWREADY,
582                                     M_AXI_WREADY, S_AXI_BREADY };
583 // Verilator lint_on UNUSED
584 // }}}
585
586 // }}}
587 end endgenerate
588 // }}}
589 //////////////////////////////////////////////////
590 //
591 // Read logic
592 // {{{
593 //////////////////////////////////////////////////
594 //
595 //
596 generate if (OPT_READS)
597 begin : IMPLEMENT_READS
598     // {{{
599     // S_AXI_AR* skid buffer
600     // {{{
601     skidbuffer #(
602         // {{{
603         .DW(IW+AW+8+3+2), .OPT_LOW-
604         // }}}
605     ) arskid(
606         // {{{
607         S_AXI_ACLK, !S_AXI_ARESETN,
608         S_AXI_ARVALID, S_AXI_ARREADY,
609         { S_AXI_ARID, S_AXI_ARADDR, S_AXI_AR-
610             S_AXI_ARBURST },
611         skids_arvalid, skids_arready,
612         { skids_arid, skids_araddr,
613             skids_arburst }
614         // }}}
615     );
616     // }}}
617     // M_AXI_R* skid buffer
618     // {{{
619     skidbuffer #(
620         // {{{
621         .DW(DW+2), .OPT_LOW-
622         // }}}
623     ) rskid(
624         // {{{
625         S_AXI_ACLK, !S_AXI_ARESETN,
626         M_AXI_RVALID, M_AXI_RREADY,
627         { M_AXI_RDATA, M_AXI_RRESP },
628         skidm_rvalid, skidm_rready,
629         { skidm_rdata, skidm_rresp }
630         // }}}
631     );
632     // }}}
633     // m_axi_arvalid
634     // {{{
635     initial m_axi_arvalid = 0;
636     always @(posedge S_AXI_ACLK)
637     if (!S_AXI_ARESETN)
638         m_axi_arvalid <= 0;
639     else if (skids_arvalid && skids_arready)
640         m_axi_arvalid <= 1;
641     else if (M_AXI_ARREADY && axi_arlen == 0)

```

```

640     m_axi_arvalid <= 0;
641 // }}}
642
643 // Read address processing
644 // {{{
645 always @(posedge S_AXI_ACLK)
646 if (skids_arvalid && skids_arready)
647 begin
648     axi_araddr <= skids_araddr;
649     axi_arburst <= skids_arburst;
650     axi_arsize <= skids_arsize;
651     axi_rlen <= skids_arlen;
652 end else if (M_AXI_ARREADY)
653 begin
654     axi_araddr <= next_read_addr;
655     if (OPT_LOWPOWER && axi_arlen == 0)
656         axi_araddr <= 0;
657 end
658
659 axi_addr #(
660     // {{{
661     .AW(C_AXI_AD-
662     DR_WIDTH), .DW(C_AXI_DATA_WIDTH)
663     // }}}
664 ) calcrdaddr(
665     // {{{
666     axi_araddr, axi_arsize, axi_arburst,
667     axi_rlen, next_read_addr
668     // }}}
669 );
670 // }}}
671
672 // axi_arlen, Read length processing
673 // {{{
674 initial axi_arlen = 0;
675 always @(posedge S_AXI_ACLK)
676 if (!S_AXI_ARESETN)
677     axi_arlen <= 0;
678 else if (skids_arvalid && skids_arready)
679     axi_arlen <= skids_arlen;
680 else if (M_AXI_ARVALID && M_AXI_ARREADY &&
681     axi_arlen > 0)
682     axi_arlen <= axi_arlen - 1;
683 // }}}
684
685 assign skids_arready = (!M_AXI_ARVALID ||
686     ((axi_arlen == 0) && M_AXI_AR-
687     READY))
688     && !rfifo_full;
689
690 assign read_from_rdfifo = skidm_rvalid &&
691     skidm_rready
692     && (rcounts <= 1) && !
693     rfifo_empty;
694
695 // Read ID FIFO
696 // {{{
697 sfifo #(
698     // {{{
699     .BW(C_AXI_ID_WIDTH+8), .LGFLN(LGFIFO)
700     // }}}
701 ) ridlnfif(
702     // {{{
703     S_AXI_ACLK, !S_AXI_ARESETN,
704     skids_arvalid && skids_arready,
705     { skids_arid, skids_arlen },
706     rfifo_full, rfifo_count,
707     read_from_rdfifo,
708     { rfifo_rid, rfifo_rcount },
709     // }}}
710 );
711 // }}}
712
713 assign skidm_rready = (!S_AXI_RVALID ||
714     S_AXI_RREADY);
715
716 // s_axi_rvalid
717 // {{{
718 initial s_axi_rvalid = 0;
719 always @(posedge S_AXI_ACLK)
720 if (!S_AXI_ARESETN)
721     s_axi_rvalid <= 0;
722 else if (skidm_rvalid && skidm_rready)
723     s_axi_rvalid <= 1;
724 else if (S_AXI_RREADY)
725     s_axi_rvalid <= 0;
726 // }}}
727
728 // s_axi_rresp, s_axi_rdata
729 // {{{
730 always @(posedge S_AXI_ACLK)
731 if (skidm_rvalid && skidm_rready)
732 begin
733     s_axi_rresp <= skidm_rresp;
734     s_axi_rdata <= skidm_rdata;
735 end else if (S_AXI_RREADY)
736 begin
737     s_axi_rresp <= 0;
738     s_axi_rdata <= 0;
739 end

```

```

734 // }}
735
736 // rcounts, Return counts
737 // {{{
738 initial rcounts = 0;
739 always @(posedge S_AXI_ACLK)
740 if (!S_AXI_ARESETN)
741     rcounts <= 0;
742 else if (read_from_rdfifo)
743     rcounts <= rfifo_rcount + rcounts;
744 else if (skidm_rvalid && skidm_rready)
745     rcounts <= rcounts - 1;
746 // }}}
747
748 // rid
749 // {{{
750 initial rid = 0;
751 always @(posedge S_AXI_ACLK)
752 if (read_from_rdfifo)
753     rid <= rfifo_rid;
754 // }}}
755
756 // s_axi_rlast
757 // {{{
758 always @(posedge S_AXI_ACLK)
759 if (!S_AXI_RVALID || S_AXI_RREADY)
760 begin
761     // if (rcounts == 1) s_axi_rlast <= 1;
762     if (read_from_rdfifo)
763         s_axi_rlast <= (rfifo_rcount ==
0);
764     else
765         s_axi_rlast <= 0;
766     if (rcounts == 1)
767         s_axi_rlast <= 1;
768 end
769 // }}}
770
771 // s_axi_rid
772 // {{{
773 initial s_axi_rid = 0;
774 always @(posedge S_AXI_ACLK)
775 if ((S_AXI_RVALID && S_AXI_RREADY && S_AXI_R-
LAST)
776     || (!S_AXI_RVALID && rcounts ==
0))
777     s_axi_rid <=
(read_from_rdfifo)&&(rcounts == 0)?rfifo_rid : rid;
778 // }}}
779
780 // M_AXI_AR*
781 // {{{
782 assign M_AXI_ARVALID= m_axi_arvalid;
783 assign M_AXI_ARADDR = axi_araddr;
784 assign M_AXI_ARPROT = 0;
785 // }}}
786 // S_AXI_R*
787 // {{{
788 assign S_AXI_RVALID = s_axi_rvalid;
789 assign S_AXI_RDATA = s_axi_rdata;
790 assign S_AXI_RRESP = s_axi_rresp;
791 assign S_AXI_RLAST = s_axi_rlast;
792 assign S_AXI_RID = s_axi_rid;
793 // }}}
794 // }}}
795
796 end else begin : NO_READ_SUPPORT // if (!OPT_READS)
797 // {{{
798 assign M_AXI_ARVALID= 0;
799 assign M_AXI_ARADDR = 0;
800 assign M_AXI_ARPROT = 0;
801 assign M_AXI_RREADY = 0;
802 //
803 assign S_AXI_ARREADY= 0;
804 assign S_AXI_RVALID = 0;
805 assign S_AXI_RDATA = 0;
806 assign S_AXI_RRESP = 0;
807 assign S_AXI_RLAST = 0;
808 assign S_AXI_RID = 0;
809
810 //
811 assign skids_arvalid = S_AXI_ARVALID;
812 assign skids_arready = S_AXI_ARREADY;
813 assign skids_arid = S_AXI_ARID;
814 assign skids_araddr = S_AXI_ARADDR;
815 assign skids_arlen = S_AXI_ARLEN;
816 assign skids_arsize = S_AXI_ARSIZE;
817 assign skids_arburst = S_AXI_ARBURST;
818 //
819 assign skidm_rvalid = M_AXI_RVALID;
820 assign skidm_rready = M_AXI_RREADY;
821 assign skidm_rdata = M_AXI_RDATA;
822 assign skidm_rresp = M_AXI_RRESP;
823 //
824 //
825 always @(*)
826 begin
827     axi_arlen = 0;
828

```

```

829 rcounts = 0;
830 rid = 0;
831
832 end
833 assign rfifo_empty = 1;
834 assign rfifo_full = 0;
835 assign rfifo_count = 0;
836
837 // Make Verilator happy
838 // {{{
839 // Verilator lint_off UNUSED
840 wire unused_read_signals;
841 assign unused_read_signals = &{ 1'b0,
M_AXI_ARREADY,
842 S_AXI_RREADY };
843 // Verilator lint_on UNUSED
844 // }}}
845
846 // }}}
847 end endgenerate
848 // }}}
849 // Make Verilator happy
850 // {{{
851 // Verilator lint_off UNUSED
852 wire [35:1:0]unused;
853 assign unused = {
854 S_AXI_AWLOCK, S_AXI_AWCACHE, S_AXI_AWPROT,
S_AXI_AWQOS,
855 skids_wlast, wfifo_count,
856 S_AXI_ARLOCK, S_AXI_ARCACHE, S_AXI_ARPROT,
S_AXI_ARQOS,
857 rfifo_count };
858 // Verilator lint_on UNUSED
859 // }}}
860 ///////////////////////////////////////////////////////////////////
861 ///////////////////////////////////////////////////////////////////
862 ///////////////////////////////////////////////////////////////////
863 //
864 // Formal properties
865 // {{{
866 // The following are a subset of the formal properties used to
verify this core
867 ///////////////////////////////////////////////////////////////////
868 ///////////////////////////////////////////////////////////////////
869 ///////////////////////////////////////////////////////////////////
870 `ifdef FORMAL
871 localparam F_LGDEPTH = LGFIFO+1+8;
872
873 //
874 // ...
875 //
876 ///////////////////////////////////////////////////////////////////
877 ///////////////////////////////////////////////////////////////////
878 //
879 // AXI channel properties
880 // {{{
881 ///////////////////////////////////////////////////////////////////
882 //
883 //
884 faxi_slave #(C_AXI_ID_WIDTH(IW),
885 C_AXI_DATA_WIDTH(DW),
886 C_AXI_ADDR_WIDTH(AW),
887 //
888 )
889 faxi(.i_clk(S_AXI_ACLK),
890 .i_axi_reset_n(S_AXI_ARESETN),
891 // Write address
892 .i_axi_awready(skids_awready),
893 .i_axi_awid( skids_awid),
894 .i_axi_awaddr( skids_awaddr),
895 .i_axi_awlen( skids_awlen),
896 .i_axi_awsz( skids_awsz),
897 .i_axi_awburst(skids_awburst),
898 .i_axi_awlock( 0),
899 .i_axi_awcache(0),
900 .i_axi_awprot( 0),
901 .i_axi_awqos( 0),
902 .i_axi_awvalid(skids_awvalid),
903 // Write data
904 .i_axi_wready( skids_wready),
905 .i_axi_wdata( skids_wdata),
906 .i_axi_wstrb( skids_wstrb),
907 .i_axi_wlast( skids_wlast),
908 .i_axi_wvalid( skids_wvalid),
909 // Write return response
910 .i_axi_bid( S_AXI_BID),
911 .i_axi_bresp( S_AXI_BRESP),
912 .i_axi_bvalid( S_AXI_BVALID),
913 .i_axi_bready( S_AXI_BREADY),
914 // Read address
915 .i_axi_arready(skids_arready),
916 .i_axi_arid( skids_arid),

```

```

917         .i_axi_araddr( skids_araddr),
918         .i_axi_arlen( skids_arlen),
919         .i_axi_arsize( skids_arsize),
920         .i_axi_arburst(skids_arburst),
921         .i_axi_arlock( 0),
922         .i_axi_arcache(0),
923         .i_axi_arprot( 0),
924         .i_axi_arqos( 0),
925         .i_axi_arvalid(skids_arvalid),
926         // Read response
927         .i_axi_rid( S_AXI_RID),
928         .i_axi_rresp( S_AXI_RRESP),
929         .i_axi_rvalid( S_AXI_RVALID),
930         .i_axi_rdata( S_AXI_RDATA),
931         .i_axi_rlast( S_AXI_RLAST),
932         .i_axi_rready( S_AXI_RREADY),
933         //
934         // Formal property data
935         .f_axi_awn-
bursts( faxi_awn_bursts),
936         .f_axi_wr_pending( faxi_wr_pending),
937         .f_axi_rd_nbursts( faxi_rd_nbursts),
938         .f_axi_rd_outstand-
ing(faxi_rd_outstanding),
939         //
940         // ...
941         );
942         // }}}
943         ///////////////////////////////////////////////////////////////////
944         //
945         // AXI-lite properties
946         // {{{
947         ///////////////////////////////////////////////////////////////////
948         //
949         //
950         faxil_master
#(.C_AXI_DATA_WIDTH(DW), .C_AXI_ADDR_WIDTH(AW),
951         .F_OPT_NO_RESET(1),
952         .F_AXI_MAXWAIT(5),
953         .F_AXI_MAXDELAY(4),
954         .F_AXI_MAXSTALL(0),
955         .F_OPT_WRITE_ONLY(OPT_WRITES && !
OPT_READS),
956         .F_OPT_READ_ONLY(!OPT_WRITES &&
OPT_READS),
957         .F_LGDEPTH(F_AXIL_LGDEPTH))
958         faxil(.i_clk(S_AXI_ACLK),
959         .i_axi_reset_n(S_AXI_ARESETN),
960         // Write address channel
961         .i_axi_awvalid(M_AXI_AWVALID),
962         .i_axi_awready(M_AXI_AWREADY),
963         .i_axi_awaddr( M_AXI_AWADDR),
964         .i_axi_awprot( M_AXI_AWPROT),
965         // Write data
966         .i_axi_wready( skidm_wready),
967         .i_axi_wdata( skidm_wdata),
968         .i_axi_wstrb( skidm_wstrb),
969         .i_axi_wvalid( skidm_wvalid),
970         // Write response
971         .i_axi_bresp( skidm_bresp),
972         .i_axi_bvalid( skidm_bvalid),
973         .i_axi_bready( skidm_bready),
974         // Read address
975         .i_axi_arvalid(M_AXI_ARVALID),
976         .i_axi_arready(M_AXI_ARREADY),
977         .i_axi_araddr( M_AXI_ARADDR),
978         .i_axi_arprot( M_AXI_ARPROT),
979         // Read data return
980         .i_axi_rvalid( skidm_rvalid),
981         .i_axi_rready( skidm_rready),
982         .i_axi_rdata( skidm_rdata),
983         .i_axi_rresp( skidm_rresp),
984         //
985         // Formal check variables
986         .f_axi_rd_outstand-
ing(faxi_rd_outstanding),
987         .f_axi_wr_outstand-
ing(faxi_wr_outstanding),
988         .f_axi_awn_outstand-
ing(faxi_awn_outstanding));
989         // }}}
990         ///////////////////////////////////////////////////////////////////
991         //
992         // Assume that the two write channels stay within an
appropriate
993         // distance of each other. This is to make certain
that the property
994         // file features are not violated, although not neces-
sary true for
995         // actual operation
996         //
997         always @(*)
998             assert(s_axi_wready == (OPT_WRITES &&
faxi_wr_pending > 0));
999
1000         ///////////////////////////////////////////////////////////////////
1001         //

```

```

1002         // Write induction properties
1003         // {{{
1004         ///////////////////////////////////////////////////////////////////
1005         //
1006         // These are extra properties necessary to pass write
induction
1007         //
1008
1009         always @(*)
1010             if ((bcounts == 0)&&(!read_from_wrfifo))
1011                 assert(!skidm_bvalid || !skidm_bready);
1012
1013         always @(*)
1014             if (axi_awn > 0)
1015                 begin
1016                     assert(m_axi_awnvalid);
1017                     if (axi_awn > 1)
1018                         begin
1019                             assert(!skids_awnready);
1020                         end
1021                     else if (wfifo_full)
1022                         begin
1023                             assert(!skids_awnready);
1024                         end
1025                     else if (M_AXI_AWVALID && !M_AXI_AWREADY)
1026                         assert(!skids_awnready);
1027                 end
1028
1029         always @(*)
1030             assert(axi_bresp != EXOKAY);
1031
1032         reg [F_LGDEPTH-1:0] f_wfifo_bursts, f_w-
fifo_bursts_minus_one,
1033             f_wfifo_within,
1034             f_wfiid_bursts, f_w-
fiid_bursts_minus_one;
1035         reg [IW-1:0] f_aid;
1036         always @(posedge S_AXI_ACLK)
1037             if (skids_awnvalid && skids_awnready)
1038                 f_aid = skids_aid;
1039         // }}}
1040         ///////////////////////////////////////////////////////////////////
1041         //
1042         // Read induction properties
1043         // {{{
1044         ///////////////////////////////////////////////////////////////////
1045         //
1046         always @(*)
1047             if (!S_AXI_RVALID && rcounts > 0)
1048                 assert(rid == S_AXI_RID);
1049
1050         always @(*)
1051             if (S_AXI_RVALID && !S_AXI_RLAST)
1052                 assert(rid == S_AXI_RID);
1053
1054         always @(*)
1055             if ((rcounts == 0)&&(!read_from_rdfifo))
1056                 assert(!skidm_rvalid || !skidm_rready);
1057
1058         always @(*)
1059             if (axi_arlen > 0)
1060                 begin
1061                     assert(m_axi_arvalid);
1062                     assert(!skids_arready);
1063                 end
1064
1065         //
1066         // ...
1067         //
1068         // }}}
1069         ///////////////////////////////////////////////////////////////////
1070         //
1071         //
1072         // Select only write or only read operation
1073         // {{{
1074         ///////////////////////////////////////////////////////////////////
1075         //
1076         //
1077         generate if (!OPT_WRITES)
1078             begin
1079                 always @(*)
1080                     begin
1081                         assume(!skids_awnvalid);
1082                         assume(!skids_wvalid);
1083                         assert(M_AXI_AWVALID == 0);
1084                         assert(faxi_awn_outstanding == 0);
1085                         assert(faxi_wr_outstanding == 0);
1086                         assert(!skidm_bvalid);
1087                         assert(!S_AXI_BVALID);
1088                     end
1089             end
1090         end generate
1091
1092         generate if (!OPT_READS)
1093             begin
1094                 always @(*)

```

```

1095         begin
1096             assume(!S_AXI_ARVALID);
1097             assert(M_AXI_ARVALID == 0);
1098             assert(faxil_rd_outstanding == 0);
1099         end
1100     end endgenerate
1101     // }}}
1102     // }}}
1103     // }}}
1104     //
1105     // Cover statements, to show performance
1106     // {{{
1107     // }}}
1108     //
1109     //
1110     generate if (OPT_WRITES)
1111     begin
1112         // {{{
1113         reg [3:0]   cvr_write_count,
1114         cvr_write_count_simple;
1115         initial cvr_write_count = 0;
1116         always @(posedge S_AXI_ACLK)
1117         if (!S_AXI_ARESETN)
1118             cvr_write_count_simple <= 0;
1119         else if (S_AXI_AWVALID && S_AXI_AWREADY &&
1120 S_AXI_AWLEN == 0)
1121             cvr_write_count_simple <=
1122 cvr_write_count_simple + 1;
1123         initial cvr_write_count = 0;
1124         always @(posedge S_AXI_ACLK)
1125         if (!S_AXI_ARESETN)
1126             cvr_write_count <= 0;
1127         else if (S_AXI_AWVALID && S_AXI_AWREADY &&
1128 S_AXI_AWLEN > 2)
1129             cvr_write_count <= cvr_write_count + 1;
1130         always @(*)
1131             cover(cvr_write_count_simple > 6 && /
1132 * ... */ !S_AXI_BVALID);
1133         always @(*)
1134             cover(cvr_write_count > 2 && /* ...
1135 */ !S_AXI_BVALID);
1136     // }}}

```

```

1134     end endgenerate
1135     generate if (OPT_READS)
1136     begin
1137         // {{{
1138         reg [3:0]   cvr_read_count,
1139         cvr_read_count_simple;
1140         initial cvr_read_count_simple = 0;
1141         always @(posedge S_AXI_ACLK)
1142         if (!S_AXI_ARESETN)
1143             cvr_read_count_simple <= 0;
1144         else if (S_AXI_ARVALID && S_AXI_ARREADY &&
1145 S_AXI_ARLEN == 0)
1146             cvr_read_count_simple <=
1147 cvr_read_count_simple + 1;
1148         initial cvr_read_count = 0;
1149         always @(posedge S_AXI_ACLK)
1150         if (!S_AXI_ARESETN)
1151             cvr_read_count <= 0;
1152         else if (S_AXI_ARVALID && S_AXI_ARREADY &&
1153 S_AXI_ARLEN > 2)
1154             cvr_read_count <= cvr_read_count + 1;
1155         always @(*)
1156             cover(cvr_read_count_simple > 6 && /
1157 * ... */ !S_AXI_RVALID);
1158         always @(*)
1159             cover(cvr_read_count > 2 && /* ... */ !
1160 S_AXI_RVALID);
1161         // }}}
1162     end endgenerate
1163     // }}}
1164     // ...
1165     // {{{
1166     // }}}
1167     //
1168     // }}}
1169     `undef BMC_ASSERT
1170 `endif
1171 // }}}
1172 endmodule

```

PCIe 코어

9 files • 5,198 lines

API — arbiter.v

Table 1227. alexforencich_verilog-pcie/rtl/arbiter.v — module / interface / package (1개)

이름	종류
arbiter	module

Table 1228. alexforencich_verilog-pcie/rtl/arbiter.v — parameter (5개)

파라미터	형/폭	기본값
PORTS		4
ARB_TYPE_ROUND_ROBIN		0
ARB_BLOCK		0
ARB_BLOCK_ACK		1
ARB_LSB_HIGH_PRIORITY		0

Table 1229. alexforencich_verilog-pcie/rtl/arbiter.v — 포트 (7개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
request	input	wire [PORTS-1:0]
acknowledge	input	wire [PORTS-1:0]
grant	output	wire [PORTS-1:0]
grant_valid	output	wire
PORTS	output	wire [\$clog2(

API — axis_arb_mux.v

Table 1230. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — module / interface / package (1개)

이름	종류
axis_arb_mux	module

Table 1231. alexforencich_verilog-pcie/rtl/axis_arb_mux.v — parameter (17개)

파라미터	형/폭	기본값
S_COUNT		4
DATA_WIDTH		8
KEEP_ENABLE		(DATA_WIDTH>8)
KEEP_WIDTH		((DATA_WIDTH+7)/8)

파라미터	형/폭	기본값
ID_ENABLE		0
S_ID_WIDTH		8
M_ID_WIDTH		$S_ID_WIDTH + \lceil \log_2(S_COUNT) \rceil$
DEST_ENABLE		0
DEST_WIDTH		8
USER_ENABLE		1
USER_WIDTH		1
LAST_ENABLE		1
UPDATE_TID		0
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1
CL_S_COUNT		$\lceil \log_2(S_COUNT) \rceil$
S_ID_WIDTH_INT		$S_ID_WIDTH > 0 ? S_ID_WIDTH : 1$

Table 1232. alexforenrich_verilog-pcie/rtl/axis_arb_mux.v — 포트 (18개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_tdata	input	wire [S_COUNT*DATA_WIDTH-1:0]
s_axis_tkeep	input	wire [S_COUNT*KEEP_WIDTH-1:0]
s_axis_tvalid	input	wire [S_COUNT-1:0]
s_axis_tready	output	wire [S_COUNT-1:0]
s_axis_tlast	input	wire [S_COUNT-1:0]
s_axis_tid	input	wire [S_COUNT*S_ID_WIDTH-1:0]
s_axis_tdest	input	wire [S_COUNT*DEST_WIDTH-1:0]
s_axis_tuser	input	wire [S_COUNT*USER_WIDTH-1:0]
m_axis_tdata	output	wire [DATA_WIDTH-1:0]
m_axis_tkeep	output	wire [KEEP_WIDTH-1:0]
m_axis_tvalid	output	wire
m_axis_tready	input	wire
m_axis_tlast	output	wire
m_axis_tid	output	wire [M_ID_WIDTH-1:0]
m_axis_tdest	output	wire [DEST_WIDTH-1:0]
m_axis_tuser	output	wire [USER_WIDTH-1:0]

API — dma_client_axis_sink.v**Table 1233. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — module / interface / package (1개)**

이름	종류
dma_client_axis_sink	module

Table 1234. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — parameter (33개)

파라미터	형/폭	기본값
RAM_ADDR_WIDTH		16
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(SEG_COUNT*SEG_BE_WIDTH)
AXIS_DATA_WIDTH		SEG_DATA_WIDTH*SEG_COUNT/2
AXIS_KEEP_ENABLE		(AXIS_DATA_WIDTH>8)
AXIS_KEEP_WIDTH		(AXIS_DATA_WIDTH/8)
AXIS_LAST_ENABLE		1
AXIS_ID_ENABLE		0
AXIS_ID_WIDTH		8
AXIS_DEST_ENABLE		0
AXIS_DEST_WIDTH		8
AXIS_USER_ENABLE		1
AXIS_USER_WIDTH		1
LEN_WIDTH		16
TAG_WIDTH		8
RAM_WORD_WIDTH		SEG_BE_WIDTH
RAM_WORD_SIZE		SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXIS_KEEP_WIDTH_INT		AXIS_KEEP_ENABLE ? AXIS_KEEP_WIDTH : 1
AXIS_WORD_WIDTH		AXIS_KEEP_WIDTH_INT
AXIS_WORD_SIZE		AXIS_DATA_WIDTH/AXIS_WORD_WIDTH
PART_COUNT		SEG_COUNT*SEG_BE_WIDTH / AXIS_KEEP_WIDTH_INT
PART_COUNT_WIDTH		PART_COUNT > 1 ? \$clog2(PART_COUNT) : 1
PART_OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
PARTS_PER_SEG		(SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT - 1) / AXIS_KEEP_WIDTH_I...
SEGS_PER_PART		(AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH - 1) / SEG_BE_WIDTH
OFFSET_WIDTH		AXIS_KEEP_WIDTH_INT > 1 ? \$clog2(AXIS_KEEP_WIDTH_INT) : 1
OFFSET_MASK		AXIS_KEEP_WIDTH_INT > 1 ? {OFFSET_WIDTH{1'b1}} : 0

파라미터	형/폭	기본값
ADDR_MASK		{RAM_ADDR_WIDTH{1'b1}} << \$clog2(Axis_Keep_Width_Int)
Cycle_Count_Width		Len_Width - \$clog2(Axis_Keep_Width_Int) + 1
Status_Fifo_Addr_Width		5
Output_Fifo_Addr_Width		5

Table 1235. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — localparam (1개)

상수	형/폭	값
STATE_IDLE	[1:0]	2'd0

Table 1236. alexforencich_verilog-pcie/rtl/dma_client_axis_sink.v — 포트 (30개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_len	output	wire [LEN_WIDTH-1:0]
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_id	output	wire [AXIS_ID_WIDTH-1:0]
m_axis_write_desc_status_dest	output	wire [AXIS_DEST_WIDTH-1:0]
m_axis_write_desc_status_user	output	wire [AXIS_USER_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire
s_axis_write_data_tdata	input	wire [AXIS_DATA_WIDTH-1:0]
s_axis_write_data_tkeep	input	wire [AXIS_Keep_Width-1:0]
s_axis_write_data_tvalid	input	wire
s_axis_write_data_tready	output	wire
s_axis_write_data_tlast	input	wire
s_axis_write_data_tid	input	wire [AXIS_ID_WIDTH-1:0]
s_axis_write_data_tdest	input	wire [AXIS_DEST_WIDTH-1:0]
s_axis_write_data_tuser	input	wire [AXIS_USER_WIDTH-1:0]
ram_wr_cmd_be	output	wire [SEG_Count*SEG_BE_Width-1:0]
ram_wr_cmd_addr	output	wire [SEG_Count*SEG_ADDR_Width-1:0]
ram_wr_cmd_data	output	wire [SEG_Count*SEG_DATA_Width-1:0]

포트	방향	형/폭
ram_wr_cmd_valid	output	wire [SEG_COUNT-1:0]
ram_wr_cmd_ready	input	wire [SEG_COUNT-1:0]
ram_wr_done	input	wire [SEG_COUNT-1:0]
enable	input	wire
abort	input	wire

API — dma_client_axis_source.v

Table 1237. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — module / interface / package (1개)

이름	종류
dma_client_axis_source	module

Table 1238. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — parameter (32개)

파라미터	형/폭	기본값
RAM_ADDR_WIDTH		16
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(SEG_COUNT*SEG_BE_WIDTH)
AXIS_DATA_WIDTH		SEG_DATA_WIDTH*SEG_COUNT/2
AXIS_KEEP_ENABLE		(AXIS_DATA_WIDTH>8)
AXIS_KEEP_WIDTH		(AXIS_DATA_WIDTH/8)
AXIS_LAST_ENABLE		1
AXIS_ID_ENABLE		0
AXIS_ID_WIDTH		8
AXIS_DEST_ENABLE		0
AXIS_DEST_WIDTH		8
AXIS_USER_ENABLE		1
AXIS_USER_WIDTH		1
LEN_WIDTH		16
TAG_WIDTH		8
RAM_WORD_WIDTH		SEG_BE_WIDTH
RAM_WORD_SIZE		SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXIS_KEEP_WIDTH_INT		AXIS_KEEP_ENABLE ? AXIS_KEEP_WIDTH : 1
AXIS_WORD_WIDTH		AXIS_KEEP_WIDTH_INT
AXIS_WORD_SIZE		AXIS_DATA_WIDTH/AXIS_WORD_WIDTH
PART_COUNT		SEG_COUNT*SEG_BE_WIDTH / AXIS_KEEP_WIDTH_INT

파라미터	형/폭	기본값
PART_COUNT_WIDTH		$\text{PART_COUNT} > 1 ? \$\text{clog2}(\text{PART_COUNT}) : 1$
PART_OFFSET_WIDTH		$\text{AXIS_KEEP_WIDTH_INT} > 1 ? \$\text{clog2}(\text{AXIS_KEEP_WIDTH_INT}) : 1$
PARTS_PER_SEG		$(\text{SEG_BE_WIDTH} + \text{AXIS_KEEP_WIDTH_INT} - 1) / \text{AXIS_KEEP_WIDTH_INT}$
SEGS_PER_PART		$(\text{AXIS_KEEP_WIDTH_INT} + \text{SEG_BE_WIDTH} - 1) / \text{SEG_BE_WIDTH}$
OFFSET_WIDTH		$\text{AXIS_KEEP_WIDTH_INT} > 1 ? \$\text{clog2}(\text{AXIS_KEEP_WIDTH_INT}) : 1$
OFFSET_MASK		$\text{AXIS_KEEP_WIDTH_INT} > 1 ? \{\text{OFFSET_WIDTH}\{1'b1\}\} : 0$
ADDR_MASK		$\{\text{RAM_ADDR_WIDTH}\{1'b1\}\} \ll \$\text{clog2}(\text{AXIS_KEEP_WIDTH_INT})$
CYCLE_COUNT_WIDTH		$\text{LEN_WIDTH} - \$\text{clog2}(\text{AXIS_KEEP_WIDTH_INT}) + 1$
OUTPUT_FIFO_ADDR_WIDTH		5

Table 1239. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — localparam (2개)

상수	형/폭	값
READ_STATE_IDLE	[0:0]	1'd0
AXIS_STATE_IDLE	[0:0]	1'd0

Table 1240. alexforencich_verilog-pcie/rtl/dma_client_axis_source.v — 포트 (28개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_id	input	wire [AXIS_ID_WIDTH-1:0]
s_axis_read_desc_dest	input	wire [AXIS_DEST_WIDTH-1:0]
s_axis_read_desc_user	input	wire [AXIS_USER_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]
m_axis_read_desc_status_valid	output	wire
m_axis_read_data_tdata	output	wire [AXIS_DATA_WIDTH-1:0]
m_axis_read_data_tkeep	output	wire [AXIS_KEEP_WIDTH-1:0]
m_axis_read_data_tvalid	output	wire
m_axis_read_data_tready	input	wire
m_axis_read_data_tlast	output	wire
m_axis_read_data_tid	output	wire [AXIS_ID_WIDTH-1:0]

포트	방향	형/폭
m_axis_read_data_tdest	output	wire [AXIS_DEST_WIDTH-1:0]
m_axis_read_data_tuser	output	wire [AXIS_USER_WIDTH-1:0]
ram_rd_cmd_addr	output	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_valid	output	wire [SEG_COUNT-1:0]
ram_rd_cmd_ready	input	wire [SEG_COUNT-1:0]
ram_rd_resp_data	input	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_rd_resp_valid	input	wire [SEG_COUNT-1:0]
ram_rd_resp_ready	output	wire [SEG_COUNT-1:0]
enable	input	wire

API — dma_if_axi.v

Table 1241. alexforencich_verilog-pcie/rtl/dma_if_axi.v — module / interface / package (1개)

이름	종류
dma_if_axi	module

Table 1242. alexforencich_verilog-pcie/rtl/dma_if_axi.v — parameter (19개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
TAG_WIDTH		8
READ_OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
WRITE_OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
READ_USE_AXI_ID		0
WRITE_USE_AXI_ID		1

Table 1243. alexforencich_verilog-pcie/rtl/dma_if_axi.v — 포트 (105개) — 앞 60개만 표시(전체 105개, 본문 소스 참조)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_awid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_awaddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_awlen	output	wire [7:0]
m_axi_awsz	output	wire [2:0]
m_axi_awburst	output	wire [1:0]
m_axi_awlock	output	wire
m_axi_awcache	output	wire [3:0]
m_axi_awprot	output	wire [2:0]
m_axi_awvalid	output	wire
m_axi_awready	input	wire
m_axi_wdata	output	wire [AXI_DATA_WIDTH-1:0]
m_axi_wstrb	output	wire [AXI_STRB_WIDTH-1:0]
m_axi_wlast	output	wire
m_axi_wvalid	output	wire
m_axi_wready	input	wire
m_axi_bid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_bresp	input	wire [1:0]
m_axi_bvalid	input	wire
m_axi_bready	output	wire
m_axi_arid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_araddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_arlen	output	wire [7:0]
m_axi_arsz	output	wire [2:0]
m_axi_arburst	output	wire [1:0]
m_axi_arlock	output	wire
m_axi_arcache	output	wire [3:0]
m_axi_arprot	output	wire [2:0]
m_axi_arvalid	output	wire
m_axi_arready	input	wire
m_axi_rid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_rdata	input	wire [AXI_DATA_WIDTH-1:0]

포트	방향	형/폭
m_axi_rresp	input	wire [1:0]
m_axi_rlast	input	wire
m_axi_rvalid	input	wire
m_axi_rready	output	wire
s_axis_read_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]
m_axis_read_desc_status_valid	output	wire
s_axis_write_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_write_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire
ram_wr_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]

API — dma_if_axi_rd.v

Table 1244. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — module / interface / package (1개)

이름	종류
dma_if_axi_rd	module

Table 1245. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — parameter (32개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32

파라미터	형/폭	기본값
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
LEN_WIDTH		16
TAG_WIDTH		8
OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
USE_AXI_ID		0
RAM_DATA_WIDTH		RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH
RAM_WORD_WIDTH		RAM_SEG_BE_WIDTH
RAM_WORD_SIZE		RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXI_WORD_WIDTH		AXI_STRB_WIDTH
AXI_WORD_SIZE		AXI_DATA_WIDTH/AXI_WORD_WIDTH
AXI_BURST_SIZE		\$clog2(AXI_STRB_WIDTH)
AXI_MAX_BURST_SIZE		AXI_MAX_BURST_LEN << AXI_BURST_SIZE
OFFSET_WIDTH		AXI_STRB_WIDTH > 1 ? \$clog2(AXI_STRB_WIDTH) : 1
OFFSET_MASK		AXI_STRB_WIDTH > 1 ? {OFFSET_WIDTH{1'b1}} : 0
RAM_OFFSET_WIDTH		\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
ADDR_MASK		{AXI_ADDR_WIDTH{1'b1}} << \$clog2(AXI_STRB_WIDTH)
CYCLE_COUNT_WIDTH		LEN_WIDTH - AXI_BURST_SIZE + 1
OP_TAG_WIDTH		\$clog2(OP_TABLE_SIZE)
OP_TABLE_READ_COUNT_WIDTH		AXI_ID_WIDTH+1
OP_TABLE_WRITE_COUNT_WIDTH		LEN_WIDTH
STATUS_FIFO_ADDR_WIDTH		5
OUTPUT_FIFO_ADDR_WIDTH		5

Table 1246. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — localparam (4개)

상수	형/폭	값
AXI_RESP_OKAY	[1:0]	2'b00

상수	형/폭	값
DMA_ERROR_NONE	[3:0]	4'd0
REQ_STATE_IDLE	[0:0]	1'd0
AXI_STATE_IDLE	[0:0]	1'd0

Table 1247. alexforencich_verilog-pcie/rtl/dma_if_axi_rd.v — 포트 (51개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_arid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_araddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_arlen	output	wire [7:0]
m_axi_arsize	output	wire [2:0]
m_axi_arburst	output	wire [1:0]
m_axi_arlock	output	wire
m_axi_arcache	output	wire [3:0]
m_axi_arprot	output	wire [2:0]
m_axi_arvalid	output	wire
m_axi_arready	input	wire
m_axi_rid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_rdata	input	wire [AXI_DATA_WIDTH-1:0]
m_axi_rresp	input	wire [1:0]
m_axi_rlast	input	wire
m_axi_rvalid	input	wire
m_axi_rready	output	wire
s_axis_read_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire
s_axis_read_desc_ready	output	wire
m_axis_read_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [3:0]
m_axis_read_desc_status_valid	output	wire
ram_wr_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]

포트	방향	형/폭
ram_wr_cmd_be	output	wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0]
ram_wr_cmd_addr	output	wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_data	output	wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_valid	output	wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready	input	wire [RAM_SEG_COUNT-1:0]
ram_wr_done	input	wire [RAM_SEG_COUNT-1:0]
enable	input	wire
status_busy	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_op_start_len	output	wire [LEN_WIDTH-1:0]
stat_rd_op_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_op_finish_status	output	wire [3:0]
stat_rd_op_finish_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_req_start_len	output	wire [12:0]
stat_rd_req_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_rd_req_finish_status	output	wire [3:0]
stat_rd_req_finish_valid	output	wire
stat_rd_op_table_full	output	wire
stat_rd_tx_stall	output	wire

API — dma_if_axi_wr.v

Table 1248. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — module / interface / package (1개)

이름	종류
dma_if_axi_wr	module

Table 1249. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — parameter (32개)

파라미터	형/폭	기본값
AXI_DATA_WIDTH		32
AXI_ADDR_WIDTH		16
AXI_STRB_WIDTH		(AXI_DATA_WIDTH/8)
AXI_ID_WIDTH		8
AXI_MAX_BURST_LEN		256
RAM_SEL_WIDTH		2

파라미터	형/폭	기본값
RAM_ADDR_WIDTH		16
RAM_SEG_COUNT		2
RAM_SEG_DATA_WIDTH		AXI_DATA_WIDTH*2/RAM_SEG_COUNT
RAM_SEG_BE_WIDTH		RAM_SEG_DATA_WIDTH/8
RAM_SEG_ADDR_WIDTH		RAM_ADDR_WIDTH-\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
TAG_WIDTH		8
OP_TABLE_SIZE		2**(AXI_ID_WIDTH)
USE_AXI_ID		1
RAM_DATA_WIDTH		RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH
RAM_WORD_WIDTH		RAM_SEG_BE_WIDTH
RAM_WORD_SIZE		RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH
AXI_WORD_WIDTH		AXI_STRB_WIDTH
AXI_WORD_SIZE		AXI_DATA_WIDTH/AXI_WORD_WIDTH
AXI_BURST_SIZE		\$clog2(AXI_STRB_WIDTH)
AXI_MAX_BURST_SIZE		AXI_MAX_BURST_LEN << AXI_BURST_SIZE
OFFSET_WIDTH		AXI_STRB_WIDTH > 1 ? \$clog2(AXI_STRB_WIDTH) : 1
OFFSET_MASK		AXI_STRB_WIDTH > 1 ? {OFFSET_WIDTH{1'b1}} : 0
RAM_OFFSET_WIDTH		\$clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH)
ADDR_MASK		{AXI_ADDR_WIDTH{1'b1}} << \$clog2(AXI_STRB_WIDTH)
CYCLE_COUNT_WIDTH		LEN_WIDTH - AXI_BURST_SIZE + 1
MASK_FIFO_ADDR_WIDTH		\$clog2(OP_TABLE_SIZE)+1
OP_TAG_WIDTH		\$clog2(OP_TABLE_SIZE)
OUTPUT_FIFO_ADDR_WIDTH		5

Table 1250. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — localparam (5개)

상수	형/폭	값
AXI_RESP_OKAY	[1:0]	2'b00
DMA_ERROR_NONE	[3:0]	4'd0
REQ_STATE_IDLE	[0:0]	1'd0
READ_STATE_IDLE	[0:0]	1'd0
AXI_STATE_IDLE	[0:0]	1'd0

Table 1251. alexforencich_verilog-pcie/rtl/dma_if_axi_wr.v — 포트 (56개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axi_awid	output	wire [AXI_ID_WIDTH-1:0]
m_axi_awaddr	output	wire [AXI_ADDR_WIDTH-1:0]
m_axi_awlen	output	wire [7:0]
m_axi_awsz	output	wire [2:0]
m_axi_awburst	output	wire [1:0]
m_axi_awlock	output	wire
m_axi_awcache	output	wire [3:0]
m_axi_awprot	output	wire [2:0]
m_axi_awvalid	output	wire
m_axi_awready	input	wire
m_axi_wdata	output	wire [AXI_DATA_WIDTH-1:0]
m_axi_wstrb	output	wire [AXI_STRB_WIDTH-1:0]
m_axi_wlast	output	wire
m_axi_wvalid	output	wire
m_axi_wready	input	wire
m_axi_bid	input	wire [AXI_ID_WIDTH-1:0]
m_axi_bresp	input	wire [1:0]
m_axi_bvalid	input	wire
m_axi_bready	output	wire
s_axis_write_desc_axi_addr	input	wire [AXI_ADDR_WIDTH-1:0]
s_axis_write_desc_ram_sel	input	wire [RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire
s_axis_write_desc_len	input	wire [LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [TAG_WIDTH-1:0]
s_axis_write_desc_valid	input	wire
s_axis_write_desc_ready	output	wire
m_axis_write_desc_status_tag	output	wire [TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [3:0]
m_axis_write_desc_status_valid	output	wire
ram_rd_cmd_sel	output	wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_rd_cmd_addr	output	wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]

포트	방향	형/폭
ram_rd_cmd_valid	output	wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_ready	input	wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_data	input	wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_rd_resp_valid	input	wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_ready	output	wire [RAM_SEG_COUNT-1:0]
enable	input	wire
status_busy	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_op_start_len	output	wire [LEN_WIDTH-1:0]
stat_wr_op_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_op_finish_status	output	wire [3:0]
stat_wr_op_finish_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_req_start_len	output	wire [12:0]
stat_wr_req_start_valid	output	wire
OP_TABLE_SIZE	output	wire [\$clog2(
stat_wr_req_finish_status	output	wire [3:0]
stat_wr_req_finish_valid	output	wire
stat_wr_op_table_full	output	wire
stat_wr_tx_stall	output	wire

API — dma_if_desc_mux.v

Table 1252. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — module / interface / package (1개)

이름	종류
dma_if_desc_mux	module

Table 1253. alexforencich_verilog-pcie/rtl/dma_if_desc_mux.v — parameter (16개)

파라미터	형/폭	기본값
PORTS		2
EXTEND_RAM_SEL		0
S_RAM_SEL_WIDTH		2
M_RAM_SEL_WIDTH		S_RAM_SEL_WIDTH+(EXTEND_RAM_SEL ? \$clog2(PORTS) : 0)
RAM_ADDR_WIDTH		16
DMA_ADDR_WIDTH		64
IMM_ENABLE		0

파라미터	형/폭	기본값
IMM_WIDTH		32
LEN_WIDTH		16
S_TAG_WIDTH		8
M_TAG_WIDTH		S_TAG_WIDTH+\$clog2(PORTS)
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1
CL_PORTS		\$clog2(PORTS)
S_RAM_SEL_WIDTH_INT		S_RAM_SEL_WIDTH > 0 ? S_RAM_SEL_WIDTH : 1
FIFO_ADDR_WIDTH		5

Table 1254. alexforenrich_verilog-pcie/rtl/dma_if_desc_mux.v — 포트 (26개)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axis_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_desc_imm	output	wire [IMM_WIDTH-1:0]
m_axis_desc_imm_en	output	wire
m_axis_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_desc_valid	output	wire
m_axis_desc_ready	input	wire
s_axis_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_desc_status_error	input	wire [3:0]
s_axis_desc_status_valid	input	wire
s_axis_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]
s_axis_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]
s_axis_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_desc_imm	input	wire [PORTS*IMM_WIDTH-1:0]
s_axis_desc_imm_en	input	wire [PORTS-1:0]
s_axis_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_desc_valid	input	wire [PORTS-1:0]
s_axis_desc_ready	output	wire [PORTS-1:0]

포트	방향	형/폭
m_axis_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_desc_status_valid	output	wire [PORTS-1:0]

API — dma_if_mux.v

Table 1255. alexforencich_verilog-pcie/rtl/dma_if_mux.v — module / interface / package (1개)

이름	종류
dma_if_mux	module

Table 1256. alexforencich_verilog-pcie/rtl/dma_if_mux.v — parameter (16개)

파라미터	형/폭	기본값
PORTS		2
SEG_COUNT		2
SEG_DATA_WIDTH		64
SEG_ADDR_WIDTH		8
SEG_BE_WIDTH		SEG_DATA_WIDTH/8
S_RAM_SEL_WIDTH		2
M_RAM_SEL_WIDTH		S_RAM_SEL_WIDTH+\$clog2(PORTS)
RAM_ADDR_WIDTH		SEG_ADDR_WIDTH+\$clog2(SEG_COUNT)+\$clog2(SEG_BE_WIDTH)
DMA_ADDR_WIDTH		64
IMM_ENABLE		0
IMM_WIDTH		32
LEN_WIDTH		16
S_TAG_WIDTH		8
M_TAG_WIDTH		S_TAG_WIDTH+\$clog2(PORTS)
ARB_TYPE_ROUND_ROBIN		0
ARB_LSB_HIGH_PRIORITY		1

Table 1257. alexforencich_verilog-pcie/rtl/dma_if_mux.v — 포트 (74개) — 앞 60개만 표시(전체 74개, 본문 소스 참조)

포트	방향	형/폭
clk	input	wire
rst	input	wire
m_axis_read_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_read_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_read_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_read_desc_len	output	wire [LEN_WIDTH-1:0]

포트	방향	형/폭
m_axis_read_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_read_desc_valid	output	wire
m_axis_read_desc_ready	input	wire
s_axis_read_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_read_desc_status_error	input	wire [3:0]
s_axis_read_desc_status_valid	input	wire
m_axis_write_desc_dma_addr	output	wire [DMA_ADDR_WIDTH-1:0]
m_axis_write_desc_ram_sel	output	wire [M_RAM_SEL_WIDTH-1:0]
m_axis_write_desc_ram_addr	output	wire [RAM_ADDR_WIDTH-1:0]
m_axis_write_desc_imm	output	wire [IMM_WIDTH-1:0]
m_axis_write_desc_imm_en	output	wire
m_axis_write_desc_len	output	wire [LEN_WIDTH-1:0]
m_axis_write_desc_tag	output	wire [M_TAG_WIDTH-1:0]
m_axis_write_desc_valid	output	wire
m_axis_write_desc_ready	input	wire
s_axis_write_desc_status_tag	input	wire [M_TAG_WIDTH-1:0]
s_axis_write_desc_status_error	input	wire [3:0]
s_axis_write_desc_status_valid	input	wire
s_axis_read_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]
s_axis_read_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]
s_axis_read_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_read_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_read_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]
s_axis_read_desc_valid	input	wire [PORTS-1:0]
s_axis_read_desc_ready	output	wire [PORTS-1:0]
m_axis_read_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_read_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_read_desc_status_valid	output	wire [PORTS-1:0]
s_axis_write_desc_dma_addr	input	wire [PORTS*DMA_ADDR_WIDTH-1:0]
s_axis_write_desc_ram_sel	input	wire [PORTS*S_RAM_SEL_WIDTH-1:0]
s_axis_write_desc_ram_addr	input	wire [PORTS*RAM_ADDR_WIDTH-1:0]
s_axis_write_desc_imm	input	wire [PORTS*IMM_WIDTH-1:0]
s_axis_write_desc_imm_en	input	wire [PORTS-1:0]
s_axis_write_desc_len	input	wire [PORTS*LEN_WIDTH-1:0]
s_axis_write_desc_tag	input	wire [PORTS*S_TAG_WIDTH-1:0]

포트	방향	형/폭
s_axis_write_desc_valid	input	wire [PORTS-1:0]
s_axis_write_desc_ready	output	wire [PORTS-1:0]
m_axis_write_desc_status_tag	output	wire [PORTS*S_TAG_WIDTH-1:0]
m_axis_write_desc_status_error	output	wire [PORTS*4-1:0]
m_axis_write_desc_status_valid	output	wire [PORTS-1:0]
if_ram_wr_cmd_sel	input	wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_wr_cmd_be	input	wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
if_ram_wr_cmd_addr	input	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_wr_cmd_data	input	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_wr_cmd_valid	input	wire [SEG_COUNT-1:0]
if_ram_wr_cmd_ready	output	wire [SEG_COUNT-1:0]
if_ram_wr_done	output	wire [SEG_COUNT-1:0]
if_ram_rd_cmd_sel	input	wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_rd_cmd_addr	input	wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_rd_cmd_valid	input	wire [SEG_COUNT-1:0]
if_ram_rd_cmd_ready	output	wire [SEG_COUNT-1:0]
if_ram_rd_resp_data	output	wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_rd_resp_valid	output	wire [SEG_COUNT-1:0]
if_ram_rd_resp_ready	input	wire [SEG_COUNT-1:0]

1. alexforenrich_verilog-pcie/rtl/arbiter.v (159 lines)

```

1 /*
2
3 Copyright (c) 2014-2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall

```

```

28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * Arbiter module
33 */
34 module arbiter #
35 (
36     parameter PORTS = 4,
37     // select round robin arbitration
38     parameter ARB_TYPE_ROUND_ROBIN = 0,
39     // blocking arbiter enable
40     parameter ARB_BLOCK = 0,
41     // block on acknowledge assert when nonzero, request deas-
sert when 0
42     parameter ARB_BLOCK_ACK = 1,
43     // LSB priority selection
44     parameter ARB_LSB_HIGH_PRIORITY = 0
45 )
46 (
47     input wire clk,
48     input wire rst,
49
50     input wire [PORTS-1:0] request,
51     input wire [PORTS-1:0] acknowledge,
52
53     output wire [PORTS-1:0] grant,
54     output wire grant_valid,
55     output wire [$clog2(PORTS)-1:0] grant_encoded
56 );
57
58 reg [PORTS-1:0] grant_reg = 0, grant_next;
59 reg grant_valid_reg = 0, grant_valid_next;
60 reg [$clog2(PORTS)-1:0] grant_encoded_reg = 0,
grant_encoded_next;
61
62 assign grant_valid = grant_valid_reg;
63 assign grant = grant_reg;
64 assign grant_encoded = grant_encoded_reg;
65
66 wire request_valid;
67 wire [$clog2(PORTS)-1:0] request_index;
68 wire [PORTS-1:0] request_mask;

```

```

69
70 priority_encoder #(
71     .WIDTH(PORTS),
72     .LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
73 )
74 priority_encoder_inst (
75     .input_unencoded(request),
76     .output_valid(request_valid),
77     .output_encoded(request_index),
78     .output_unencoded(request_mask)
79 );
80
81 reg [PORTS-1:0] mask_reg = 0, mask_next;
82
83 wire masked_request_valid;
84 wire [$clog2(PORTS)-1:0] masked_request_index;
85 wire [PORTS-1:0] masked_request_mask;
86
87 priority_encoder #(
88     .WIDTH(PORTS),
89     .LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
90 )
91 priority_encoder_masked (
92     .input_unencoded(request & mask_reg),
93     .output_valid(masked_request_valid),
94     .output_encoded(masked_request_index),
95     .output_unencoded(masked_request_mask)
96 );
97
98 always @* begin
99     grant_next = 0;
100     grant_valid_next = 0;
101     grant_encoded_next = 0;
102     mask_next = mask_reg;
103
104     if (ARB_BLOCK && !ARB_BLOCK_ACK && grant_reg & request)
105     begin
106         // granted request still asserted; hold it
107         grant_valid_next = grant_valid_reg;
108         grant_next = grant_reg;
109         grant_encoded_next = grant_encoded_reg;
110     end else if (ARB_BLOCK && ARB_BLOCK_ACK && grant_valid
111 && !(grant_reg & acknowledge)) begin
112         // granted request not yet acknowledged; hold it
113         grant_valid_next = grant_valid_reg;
114         grant_next = grant_reg;
115         grant_encoded_next = grant_encoded_reg;
116     end else if (request_valid) begin
117         if (ARB_TYPE_ROUND_ROBIN) begin
118             if (masked_request_valid) begin
119                 grant_valid_next = 1;
120                 grant_next = masked_request_mask;
121                 grant_encoded_next = masked_request_index;
122                 if (ARB_LSB_HIGH_PRIORITY) begin
123                     mask_next = {PORTS{1'b1}} <<
124 (masked_request_index + 1);
125                 end else begin
126                     mask_next = {PORTS{1'b1}} >> (PORTS -
127 masked_request_index);
128                 end
129             end else begin
130                 grant_valid_next = 1;
131                 grant_next = request_mask;
132                 grant_encoded_next = request_index;
133                 if (ARB_LSB_HIGH_PRIORITY) begin
134                     mask_next = {PORTS{1'b1}} << (re-
135 quest_index + 1);
136                 end else begin
137                     mask_next = {PORTS{1'b1}} >> (PORTS - re-
138 quest_index);
139                 end
140             end
141         end
142     end
143 always @(posedge clk) begin
144     if (rst) begin
145         grant_reg <= 0;
146         grant_valid_reg <= 0;
147         grant_encoded_reg <= 0;
148         mask_reg <= 0;
149     end else begin
150         grant_reg <= grant_next;
151         grant_valid_reg <= grant_valid_next;
152         grant_encoded_reg <= grant_encoded_next;
153         mask_reg <= mask_next;
154     end
155 end
156 endmodule
157 `resetall
158
159

```

2. alexforenrich_verilog-pcie/rtl/axis_arb_mux.v (310 lines)

```

1 /*
2
3 Copyright (c) 2014-2018 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
6 obtaining a copy
7 of this software and associated documentation files (the
8 "Software"), to deal
9 in the Software without restriction, including without limita-
10 tion the rights
11 to use, copy, modify, merge, publish, distribute, sublicense,
12 and/or sell
13 copies of the Software, and to permit persons to whom the
14 Software is
15 furnished to do so, subject to the following conditions:
16
17 The above copyright notice and this permission notice shall be
18 included in
19 all copies or substantial portions of the Software.
20
21 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
22 KIND, EXPRESS OR
23 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
24 CHANTABILITY
25 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
26 EVENT SHALL THE
27 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
28 OR OTHER
29 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
30 WISE, ARISING FROM,
31 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
32 DEALINGS IN
33 THE SOFTWARE.
34
35 */
36 // Language: Verilog 2001
37
38 `resetall
39 `timescale 1ns / 1ps
40 `default_nettype none
41
42 /*
43 * AXI4-Stream arbitrated multiplexer
44 */
45 module axis_arb_mux #
46 (
47     // Number of AXI stream inputs
48     parameter S_COUNT = 4,
49     // Width of AXI stream interfaces in bits
50     parameter DATA_WIDTH = 8,
51     // Propagate tkeep signal
52     parameter KEEP_ENABLE = (DATA_WIDTH>8),
53     // tkeep signal width (words per cycle)
54     parameter KEEP_WIDTH = ((DATA_WIDTH+7)/8),
55     // Propagate tid signal
56     parameter ID_ENABLE = 0,
57     // input tid signal width
58     parameter S_ID_WIDTH = 8,
59     // output tid signal width
60     parameter M_ID_WIDTH = S_ID_WIDTH+$clog2(S_COUNT),
61     // Propagate tdest signal
62     parameter DEST_ENABLE = 0,
63     // tdest signal width
64     parameter DEST_WIDTH = 8,
65     // Propagate tuser signal
66     parameter USER_ENABLE = 1,
67     // tuser signal width
68     parameter USER_WIDTH = 1,
69     // Propagate tlast signal
70     parameter LAST_ENABLE = 1,
71     // Update tid with routing information
72     parameter UPDATE_TID = 0,
73     // select round robin arbitration
74     parameter ARB_TYPE_ROUND_ROBIN = 0,
75     // LSB priority selection
76     parameter ARB_LSB_HIGH_PRIORITY = 1
77 )
78 (
79     input wire clk,
80     input wire rst,
81
82     /*
83     * AXI Stream inputs
84     */
85     input wire [S_COUNT*DATA_WIDTH-1:0] s_axis_tdata,
86     input wire [S_COUNT*KEEP_WIDTH-1:0] s_axis_tkeep,
87     input wire [S_COUNT-1:0] s_axis_tvalid,
88     output wire [S_COUNT-1:0] s_axis_tready,
89     input wire [S_COUNT-1:0] s_axis_tlast,
90     input wire [S_COUNT*S_ID_WIDTH-1:0] s_axis_tid,
91     input wire [S_COUNT*DEST_WIDTH-1:0] s_axis_tdest,
92     input wire [S_COUNT*USER_WIDTH-1:0] s_axis_tuser,
93
94     /*
95     * AXI Stream output
96     */
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

85  */
86  output wire [DATA_WIDTH-1:0] m_axis_tdata,
87  output wire [KEEP_WIDTH-1:0] m_axis_tkeep,
88  output wire m_axis_tvalid,
89  input wire m_axis_tready,
90  output wire m_axis_tlast,
91  output wire [M_ID_WIDTH-1:0] m_axis_tid,
92  output wire [DEST_WIDTH-1:0] m_axis_tdest,
93  output wire [USER_WIDTH-1:0] m_axis_tuser
94 );
95
96 parameter CL_S_COUNT = $clog2(S_COUNT);
97
98 parameter S_ID_WIDTH_INT = S_ID_WIDTH > 0 ? S_ID_WIDTH : 1;
99
100 // check configuration
101 initial begin
102     if (UPDATE_TID) begin
103         if (!ID_ENABLE) begin
104             $error("Error: UPDATE_TID set requires ID_ENABLE
set (instance %m)");
105         $finish;
106     end
107
108     if (M_ID_WIDTH < CL_S_COUNT) begin
109         $error("Error: M_ID_WIDTH too small for port count
(instance %m)");
110     end
111     $finish;
112 end
113
114 wire [S_COUNT-1:0] request;
115 wire [S_COUNT-1:0] acknowledge;
116 wire [S_COUNT-1:0] grant;
117 wire grant_valid;
118 wire [CL_S_COUNT-1:0] grant_encoded;
119
120 // input registers to pipeline arbitration delay
121 reg [S_COUNT*DATA_WIDTH-1:0] s_axis_tdata_reg = 0;
122 reg [S_COUNT*KEEP_WIDTH-1:0] s_axis_tkeep_reg = 0;
123 reg [S_COUNT-1:0] s_axis_tvalid_reg = 0;
124 reg [S_COUNT-1:0] s_axis_tlast_reg = 0;
125 reg [S_COUNT*S_ID_WIDTH-1:0] s_axis_tid_reg = 0;
126 reg [S_COUNT*DEST_WIDTH-1:0] s_axis_tdest_reg = 0;
127 reg [S_COUNT*USER_WIDTH-1:0] s_axis_tuser_reg = 0;
128
129 // internal datapath
130 reg [DATA_WIDTH-1:0] m_axis_tdata_int;
131 reg [KEEP_WIDTH-1:0] m_axis_tkeep_int;
132 reg m_axis_tvalid_int;
133 reg m_axis_tlast_int;
134 reg m_axis_tready_int_reg = 1'b0;
135 reg m_axis_tlast_reg;
136 reg [M_ID_WIDTH-1:0] m_axis_tid_int;
137 reg [DEST_WIDTH-1:0] m_axis_tdest_int;
138 reg [USER_WIDTH-1:0] m_axis_tuser_int;
139 wire m_axis_tready_int_early;
140
141 assign s_axis_tready = ~s_axis_tvalid_reg |
({S_COUNT{m_axis_tready_int_reg}} & grant);
142
143 // mux for incoming packet
144 wire [DATA_WIDTH-1:0] current_s_tdata = s_ax-
is_tdata_reg[grant_encoded*DATA_WIDTH+: DATA_WIDTH];
145 wire [KEEP_WIDTH-1:0] current_s_tkeep = s_ax-
is_tkeep_reg[grant_encoded*KEEP_WIDTH+: KEEP_WIDTH];
146 wire current_s_tvalid = s_ax-
is_tvalid_reg[grant_encoded];
147 wire current_s_tready = s_ax-
is_tready[grant_encoded];
148 wire current_s_tlast = s_ax-
is_tlast_reg[grant_encoded];
149 wire [S_ID_WIDTH-1:0] current_s_tid = s_ax-
is_tid_reg[grant_encoded*S_ID_WIDTH+: S_ID_WIDTH_INT];
150 wire [DEST_WIDTH-1:0] current_s_tdest = s_ax-
is_tdest_reg[grant_encoded*DEST_WIDTH+: DEST_WIDTH];
151 wire [USER_WIDTH-1:0] current_s_tuser = s_ax-
is_tuser_reg[grant_encoded*USER_WIDTH+: USER_WIDTH];
152
153 // arbiter instance
154 arbiter #(
155     .PORTS(S_COUNT),
156     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
157     .ARB_BLOCK(1),
158     .ARB_BLOCK_ACK(1),
159     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
160 )
161 arb_inst (
162     .clk(clk),
163     .rst(rst),
164     .request(request),
165     .acknowledge(acknowledge),
166     .grant(grant),
167     .grant_valid(grant_valid),
168     .grant_encoded(grant_encoded)
169 );
170
171 assign request = (s_axis_tvalid_reg & ~grant) | (s_axis_tvalid &
grant);
172 assign acknowledge = grant & s_axis_tvalid_reg &
{S_COUNT{m_axis_tready_int_reg}} & (LAST_ENABLE ? s_axis_tlast_reg :
{S_COUNT{1'b1}});
173
174 always @* begin
175     // pass through selected packet data
176     m_axis_tdata_int = current_s_tdata;
177     m_axis_tkeep_int = current_s_tkeep;
178     m_axis_tvalid_int = current_s_tvalid && m_ax-
is_tready_int_reg && grant_valid;
179     m_axis_tlast_int = current_s_tlast;
180     m_axis_tid_int = current_s_tid;
181     if (UPDATE_TID && S_COUNT > 1) begin
182         m_axis_tid_int[M_ID_WIDTH-1:M_ID_WIDTH-CL_S_COUNT] =
grant_encoded;
183     end
184     m_axis_tdest_int = current_s_tdest;
185     m_axis_tuser_int = current_s_tuser;
186 end
187
188 integer i;
189
190 always @(posedge clk) begin
191     // register inputs
192     for (i = 0; i < S_COUNT; i = i + 1) begin
193         if (s_axis_tready[i]) begin
194             s_axis_tdata_reg[i*DATA_WIDTH+: DATA_WIDTH] <=
s_axis_tdata[i*DATA_WIDTH+: DATA_WIDTH];
195             s_axis_tkeep_reg[i*KEEP_WIDTH+: KEEP_WIDTH] <=
s_axis_tkeep[i*KEEP_WIDTH+: KEEP_WIDTH];
196             s_axis_tvalid_reg[i] <= s_axis_tvalid[i];
197             s_axis_tlast_reg[i] <= s_axis_tlast[i];
198             s_axis_tid_reg[i*S_ID_WIDTH+: S_ID_WIDTH_INT] <=
s_axis_tid[i*S_ID_WIDTH+: S_ID_WIDTH_INT];
199             s_axis_tdest_reg[i*DEST_WIDTH+: DEST_WIDTH] <=
s_axis_tdest[i*DEST_WIDTH+: DEST_WIDTH];
200             s_axis_tuser_reg[i*USER_WIDTH+: USER_WIDTH] <=
s_axis_tuser[i*USER_WIDTH+: USER_WIDTH];
201         end
202     end
203
204     if (rst) begin
205         s_axis_tvalid_reg <= 0;
206     end
207 end
208
209 // output datapath logic
210 reg [DATA_WIDTH-1:0] m_axis_tdata_reg = {DATA_WIDTH{1'b0}};
211 reg [KEEP_WIDTH-1:0] m_axis_tkeep_reg = {KEEP_WIDTH{1'b0}};
212 reg m_axis_tvalid_reg = 1'b0, m_ax-
is_tvalid_next;
213 reg m_axis_tlast_reg = 1'b0;
214 reg [M_ID_WIDTH-1:0] m_axis_tid_reg = {M_ID_WIDTH{1'b0}};
215 reg [DEST_WIDTH-1:0] m_axis_tdest_reg = {DEST_WIDTH{1'b0}};
216 reg [USER_WIDTH-1:0] m_axis_tuser_reg = {USER_WIDTH{1'b0}};
217
218 reg [DATA_WIDTH-1:0] temp_m_axis_tdata_reg =
{DATA_WIDTH{1'b0}};
219 reg [KEEP_WIDTH-1:0] temp_m_axis_tkeep_reg =
{KEEP_WIDTH{1'b0}};
220 reg temp_m_axis_tvalid_reg = 1'b0, temp_m_ax-
is_tvalid_next;
221 reg temp_m_axis_tlast_reg = 1'b0;
222 reg [M_ID_WIDTH-1:0] temp_m_axis_tid_reg =
{M_ID_WIDTH{1'b0}};
223 reg [DEST_WIDTH-1:0] temp_m_axis_tdest_reg =
{DEST_WIDTH{1'b0}};
224 reg [USER_WIDTH-1:0] temp_m_axis_tuser_reg =
{USER_WIDTH{1'b0}};
225
226 // datapath control
227 reg store_axis_int_to_output;
228 reg store_axis_int_to_temp;
229 reg store_axis_temp_to_output;
230
231 assign m_axis_tdata = m_axis_tdata_reg;
232 assign m_axis_tkeep = KEEP_ENABLE ? m_axis_tkeep_reg :
{KEEP_WIDTH{1'b1}};
233 assign m_axis_tvalid = m_axis_tvalid_reg;
234 assign m_axis_tlast = LAST_ENABLE ? m_axis_tlast_reg : 1'b1;
235 assign m_axis_tid = ID_ENABLE ? m_axis_tid_reg :
{M_ID_WIDTH{1'b0}};
236 assign m_axis_tdest = DEST_ENABLE ? m_axis_tdest_reg :
{DEST_WIDTH{1'b0}};
237 assign m_axis_tuser = USER_ENABLE ? m_axis_tuser_reg :
{USER_WIDTH{1'b0}};
238
239 // enable ready input next cycle if output is ready or if both
output registers are empty
240 assign m_axis_tready_int_early = m_axis_tready || (!temp_m_ax-
is_tvalid_reg && !m_axis_tvalid_reg);
241
242 always @* begin
243     // transfer sink ready state to source
244     m_axis_tvalid_next = m_axis_tvalid_reg;
245     temp_m_axis_tvalid_next = temp_m_axis_tvalid_reg;
246
247     store_axis_int_to_output = 1'b0;
248     store_axis_int_to_temp = 1'b0;
249     store_axis_temp_to_output = 1'b0;

```

```

250
251   if (m_axis_tready_int_reg) begin
252       // input is ready
253       if (m_axis_tready || !m_axis_tvalid_reg) begin
254           // output is ready or currently not valid, transfer data to output
255           m_axis_tvalid_next = m_axis_tvalid_int;
256           store_axis_int_to_output = 1'b1;
257       end else begin
258           // output is not ready, store input in temp
259           temp_m_axis_tvalid_next = m_axis_tvalid_int;
260           store_axis_int_to_temp = 1'b1;
261       end
262   end else if (m_axis_tready) begin
263       // input is not ready, but output is ready
264       m_axis_tvalid_next = temp_m_axis_tvalid_reg;
265       temp_m_axis_tvalid_next = 1'b0;
266       store_axis_temp_to_output = 1'b1;
267   end
268 end
269
270 always @(posedge clk) begin
271     m_axis_tvalid_reg <= m_axis_tvalid_next;
272     m_axis_tready_int_reg <= m_axis_tready_int_early;
273     temp_m_axis_tvalid_reg <= temp_m_axis_tvalid_next;
274
275     // datapath
276     if (store_axis_int_to_output) begin
277         m_axis_tdata_reg <= m_axis_tdata_int;
278         m_axis_tkeep_reg <= m_axis_tkeep_int;
279         m_axis_tlast_reg <= m_axis_tlast_int;
280         m_axis_tid_reg <= m_axis_tid_int;
281         m_axis_tdest_reg <= m_axis_tdest_int;
282         m_axis_tuser_reg <= m_axis_tuser_int;
283     end else if (store_axis_temp_to_output) begin
284         m_axis_tdata_reg <= temp_m_axis_tdata_reg;
285         m_axis_tkeep_reg <= temp_m_axis_tkeep_reg;
286         m_axis_tlast_reg <= temp_m_axis_tlast_reg;
287         m_axis_tid_reg <= temp_m_axis_tid_reg;
288         m_axis_tdest_reg <= temp_m_axis_tdest_reg;
289         m_axis_tuser_reg <= temp_m_axis_tuser_reg;
290     end
291
292     if (store_axis_int_to_temp) begin
293         temp_m_axis_tdata_reg <= m_axis_tdata_int;
294         temp_m_axis_tkeep_reg <= m_axis_tkeep_int;
295         temp_m_axis_tlast_reg <= m_axis_tlast_int;
296         temp_m_axis_tid_reg <= m_axis_tid_int;
297         temp_m_axis_tdest_reg <= m_axis_tdest_int;
298         temp_m_axis_tuser_reg <= m_axis_tuser_int;
299     end
300
301     if (rst) begin
302         m_axis_tvalid_reg <= 1'b0;
303         m_axis_tready_int_reg <= 1'b0;
304         temp_m_axis_tvalid_reg <= 1'b0;
305     end
306 end
307
308 endmodule
309
310 `resetall

```

3. alexforench_verilog-pcie/rtl/dma_client_axis_sink.v (635 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenchich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limitation the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.

```

```

22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32  * AXI stream sink DMA client
33  */
34 module dma_client_axis_sink #
35 (
36     // RAM address width
37     parameter RAM_ADDR_WIDTH = 16,
38     // RAM segment count
39     parameter SEG_COUNT = 2,
40     // RAM segment data width
41     parameter SEG_DATA_WIDTH = 64,
42     // RAM segment byte enable width
43     parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
44     // RAM segment address width
45     parameter SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clog2(SEG_COUNT*SEG_BE_WIDTH),
46     // Width of AXI stream interfaces in bits
47     parameter AXIS_DATA_WIDTH = SEG_DATA_WIDTH*SEG_COUNT/2,
48     // Use AXI stream tkeep signal
49     parameter AXIS_KEEP_ENABLE = (AXIS_DATA_WIDTH>8),
50     // AXI stream tkeep signal width (words per cycle)
51     parameter AXIS_KEEP_WIDTH = (AXIS_DATA_WIDTH/8),
52     // Use AXI stream tlast signal
53     parameter AXIS_LAST_ENABLE = 1,
54     // Propagate AXI stream tid signal
55     parameter AXIS_ID_ENABLE = 0,
56     // AXI stream tid signal width
57     parameter AXIS_ID_WIDTH = 8,
58     // Propagate AXI stream tdest signal
59     parameter AXIS_DEST_ENABLE = 0,
60     // AXI stream tdest signal width
61     parameter AXIS_DEST_WIDTH = 8,
62     // Propagate AXI stream tuser signal
63     parameter AXIS_USER_ENABLE = 1,
64     // AXI stream tuser signal width
65     parameter AXIS_USER_WIDTH = 1,
66     // Width of length field
67     parameter LEN_WIDTH = 16,
68     // Width of tag field
69     parameter TAG_WIDTH = 8
70 )
71 (
72     input wire clk,
73     input wire rst,
74
75     /*
76      * AXI write descriptor input
77      */
78     input wire [RAM_ADDR_WIDTH-1:0] s_ax-
is_write_desc_ram_addr,
79     input wire [LEN_WIDTH-1:0] s_ax-
is_write_desc_len,
80     input wire [TAG_WIDTH-1:0] s_ax-
is_write_desc_tag,
81     input wire s_ax-
is_write_desc_valid,
82     output wire s_ax-
is_write_desc_ready,
83
84     /*
85      * AXI write descriptor status output
86      */
87     output wire [LEN_WIDTH-1:0] m_ax-
is_write_desc_status_len,
88     output wire [TAG_WIDTH-1:0] m_ax-
is_write_desc_status_tag,
89     output wire [AXIS_ID_WIDTH-1:0] m_ax-
is_write_desc_status_id,
90     output wire [AXIS_DEST_WIDTH-1:0] m_ax-
is_write_desc_status_dest,
91     output wire [AXIS_USER_WIDTH-1:0] m_ax-
is_write_desc_status_user,
92     output wire [3:0] m_ax-
is_write_desc_status_error,
93     output wire m_ax-
is_write_desc_status_valid,
94
95     /*
96      * AXI stream write data input
97      */
98     input wire [AXIS_DATA_WIDTH-1:0] s_ax-
is_write_data_tdata,
99     input wire [AXIS_KEEP_WIDTH-1:0] s_ax-
is_write_data_tkeep,
100    input wire s_ax-
is_write_data_tvalid,
101    output wire s_ax-
is_write_data_tready,
102    input wire s_ax-
is_write_data_tlast,
103    input wire [AXIS_ID_WIDTH-1:0] s_ax-
is_write_data_tid,

```



```

104 input wire [AXIS_DEST_WIDTH-1:0] s_ax-
is_write_data_tdest,
105 input wire [AXIS_USER_WIDTH-1:0] s_ax-
is_write_data_tuser,
106
107 /*
108 * RAM interface
109 */
110 output wire [SEG_COUNT*SEG_BE_WIDTH-1:0] ram_wr_cmd_be,
111 output wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_wr_cmd_ad-
dr,
112 output wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data,
113 output wire [SEG_COUNT-1:0]
ram_wr_cmd_valid,
114 input wire [SEG_COUNT-1:0]
ram_wr_cmd_ready,
115 input wire [SEG_COUNT-1:0] ram_wr_done,
116
117 /*
118 * Configuration
119 */
120 input wire enable,
121 input wire abort
122 );
123
124 parameter RAM_WORD_WIDTH = SEG_BE_WIDTH;
125 parameter RAM_WORD_SIZE = SEG_DATA_WIDTH/RAM_WORD_WIDTH;
126
127 parameter AXIS_KEEP_WIDTH_INT = AXIS_KEEP_ENABLE ?
AXIS_KEEP_WIDTH : 1;
128 parameter AXIS_WORD_WIDTH = AXIS_KEEP_WIDTH_INT;
129 parameter AXIS_WORD_SIZE = AXIS_DATA_WIDTH/AXIS_WORD_WIDTH;
130
131 parameter PART_COUNT = SEG_COUNT*SEG_BE_WIDTH /
AXIS_KEEP_WIDTH_INT;
132 parameter PART_COUNT_WIDTH = PART_COUNT > 1 ? $clog-
2(PART_COUNT) : 1;
133 parameter PART_OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
134 parameter PARTS_PER_SEG = (SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT
- 1) / AXIS_KEEP_WIDTH_INT;
135 parameter SEGS_PER_PART = (AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH
- 1) / SEG_BE_WIDTH;
136
137 parameter OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
138 parameter OFFSET_MASK = AXIS_KEEP_WIDTH_INT > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
139 parameter ADDR_MASK = {RAM_ADDR_WIDTH{1'b1}} << $clog-
2(AXIS_KEEP_WIDTH_INT);
140 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - $clog-
2(AXIS_KEEP_WIDTH_INT) + 1;
141
142 parameter STATUS_FIFO_ADDR_WIDTH = 5;
143 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
144
145 // bus width assertions
146 initial begin
147 if (RAM_WORD_SIZE * SEG_BE_WIDTH != SEG_DATA_WIDTH) begin
148 $error("Error: RAM data width not evenly divisible (in-
stance %m)");
149 $finish;
150 end
151
152 if (AXIS_WORD_SIZE * AXIS_KEEP_WIDTH_INT !=
AXIS_DATA_WIDTH) begin
153 $error("Error: AXI stream data width not evenly divis-
ible (instance %m)");
154 $finish;
155 end
156
157 if (RAM_WORD_SIZE != AXIS_WORD_SIZE) begin
158 $error("Error: word size mismatch (instance %m)");
159 $finish;
160 end
161
162 if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
163 $error("Error: RAM word width must be even power of
two (instance %m)");
164 $finish;
165 end
166
167 if (RAM_ADDR_WIDTH != SEG_ADDR_WIDTH+$clog2(SEG_COUNT)+
$clog2(SEG_BE_WIDTH)) begin
168 $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
169 $finish;
170 end
171
172 if (AXIS_DATA_WIDTH > SEG_COUNT*SEG_DATA_WIDTH) begin
173 $error("Error: AXI stream interface width must not be
wider than RAM interface width (instance %m)");
174 $finish;
175 end
176
177 if (AXIS_DATA_WIDTH*2**$clog2(PART_COUNT) !=
SEG_COUNT*SEG_DATA_WIDTH) begin
178 $error("Error: AXI stream interface width must be a
power of two fraction of RAM interface width (instance %m)");
179 $finish;

```

```

180 end
181 end
182
183 localparam [1:0]
184 STATE_IDLE = 2'd0,
185 STATE_WRITE = 2'd1,
186 STATE_DROP_DATA = 2'd2;
187
188 reg [1:0] state_reg = STATE_IDLE, state_next;
189
190 integer i;
191 reg [OFFSET_WIDTH:0] cycle_size;
192
193 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
194 reg [AXIS_KEEP_WIDTH_INT-1:0] keep_mask_reg =
{AXIS_KEEP_WIDTH_INT{1'b0}}, keep_mask_next;
195 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, last_cycle_offset_next;
196 reg [LEN_WIDTH-1:0] length_reg = {LEN_WIDTH{1'b0}},
length_next;
197 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
198 reg last_cycle_reg = 1'b0, last_cycle_next;
199
200 reg [TAG_WIDTH-1:0] tag_reg = {TAG_WIDTH{1'b0}}, tag_next;
201
202 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_wr_ptr_reg = 0;
203 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_rd_ptr_reg = 0,
status_fifo_rd_ptr_next;
204 reg [LEN_WIDTH-1:0]
status_fifo_len[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
205 reg [TAG_WIDTH-1:0]
status_fifo_tag[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
206 reg [AXIS_ID_WIDTH-1:0]
status_fifo_id[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
207 reg [AXIS_DEST_WIDTH-1:0]
status_fifo_dest[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
208 reg [AXIS_USER_WIDTH-1:0]
status_fifo_user[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
209 reg [SEG_COUNT-1:0]
status_fifo_mask[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
210 reg status_fifo_last[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
211 reg [LEN_WIDTH-1:0] status_fifo_wr_len;
212 reg [TAG_WIDTH-1:0] status_fifo_wr_tag;
213 reg [AXIS_ID_WIDTH-1:0] status_fifo_wr_id;
214 reg [AXIS_DEST_WIDTH-1:0] status_fifo_wr_dest;
215 reg [AXIS_USER_WIDTH-1:0] status_fifo_wr_user;
216 reg [SEG_COUNT-1:0] status_fifo_wr_mask;
217 reg status_fifo_wr_last;
218 reg status_fifo_we = 1'b0;
219 reg status_fifo_half_full_reg = 1'b0;
220
221 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] active_count_reg = 0;
222 reg active_count_av_reg = 1'b1;
223 reg inc_active;
224 reg dec_active;
225
226 reg s_axis_write_desc_ready_reg = 1'b0, s_ax-
is_write_desc_ready_next;
227
228 reg [LEN_WIDTH-1:0] m_axis_write_desc_status_len_reg =
{LEN_WIDTH{1'b0}}, m_axis_write_desc_status_len_next;
229 reg [TAG_WIDTH-1:0] m_axis_write_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_write_desc_status_tag_next;
230 reg [AXIS_ID_WIDTH-1:0] m_axis_write_desc_status_id_reg =
{AXIS_ID_WIDTH{1'b0}}, m_axis_write_desc_status_id_next;
231 reg [AXIS_DEST_WIDTH-1:0] m_axis_write_desc_status_dest_reg =
{AXIS_DEST_WIDTH{1'b0}}, m_axis_write_desc_status_dest_next;
232 reg [AXIS_USER_WIDTH-1:0] m_axis_write_desc_status_user_reg =
{AXIS_USER_WIDTH{1'b0}}, m_axis_write_desc_status_user_next;
233 reg m_axis_write_desc_status_valid_reg = 1'b0, m_ax-
is_write_desc_status_valid_next;
234
235 reg s_axis_write_data_tready_reg = 1'b0, s_ax-
is_write_data_tready_next;
236
237 // internal datapath
238 reg [SEG_COUNT*SEG_BE_WIDTH-1:0] ram_wr_cmd_be_int;
239 reg [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_int;
240 reg [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_int;
241 reg [SEG_COUNT-1:0] ram_wr_cmd_valid_int;
242 wire [SEG_COUNT-1:0] ram_wr_cmd_ready_int;
243
244 reg [SEG_COUNT-1:0] ram_wr_cmd_mask;
245
246 wire [SEG_COUNT-1:0] out_done;
247 reg [SEG_COUNT-1:0] out_done_ack;
248
249 assign s_axis_write_desc_ready = s_axis_write_desc_ready_reg;
250
251 assign m_axis_write_desc_status_len = m_ax-
is_write_desc_status_len_reg;
252 assign m_axis_write_desc_status_tag = m_ax-
is_write_desc_status_tag_reg;
253 assign m_axis_write_desc_status_id = m_ax-
is_write_desc_status_id_reg;
254 assign m_axis_write_desc_status_dest = m_ax-
is_write_desc_status_dest_reg;
255 assign m_axis_write_desc_status_user = m_ax-

```

```

is_write_desc_status_user_reg;
256 assign m_axis_write_desc_status_error = 4'd0;
257 assign m_axis_write_desc_status_valid = m_ax-
is_write_desc_status_valid_reg;
258
259 assign s_axis_write_data_tready = s_ax-
is_write_data_tready_reg;
260
261 always @* begin
262     state_next = STATE_IDLE;
263
264     s_axis_write_desc_ready_next = 1'b0;
265
266     m_axis_write_desc_status_len_next = m_ax-
is_write_desc_status_len_reg;
267     m_axis_write_desc_status_tag_next = m_ax-
is_write_desc_status_tag_reg;
268     m_axis_write_desc_status_id_next = m_ax-
is_write_desc_status_id_reg;
269     m_axis_write_desc_status_dest_next = m_ax-
is_write_desc_status_dest_reg;
270     m_axis_write_desc_status_user_next = m_ax-
is_write_desc_status_user_reg;
271     m_axis_write_desc_status_valid_next = 1'b0;
272
273     s_axis_write_data_tready_next = 1'b0;
274
275     if (PART_COUNT > 1) begin
276         ram_wr_cmd_be_int = (s_axis_write_data_tkeep &
keep_mask_reg) << (addr_reg & ({PART_COUNT_WIDTH{1'b1}} << PART_OFF-
SET_WIDTH));
277     end else begin
278         ram_wr_cmd_be_int = s_axis_write_data_tkeep &
keep_mask_reg;
279     end
280     ram_wr_cmd_addr_int =
{PART_COUNT{addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-SEG_AD-
DR_WIDTH]}};
281     ram_wr_cmd_data_int =
{PART_COUNT{s_axis_write_data_tdata}};
282     ram_wr_cmd_valid_int = {SEG_COUNT{1'b0}};
283     for (i = 0; i < SEG_COUNT; i = i + 1) begin
284         ram_wr_cmd_mask[i] = ram_wr_cmd_be_int[i*SEG_BE_WIDTH
+ SEG_BE_WIDTH] != 0;
285     end
286
287     cycle_size = AXIS_KEEP_WIDTH_INT;
288
289     addr_next = addr_reg;
290     keep_mask_next = keep_mask_reg;
291     last_cycle_offset_next = last_cycle_offset_reg;
292     length_next = length_reg;
293     cycle_count_next = cycle_count_reg;
294     last_cycle_next = last_cycle_reg;
295
296     tag_next = tag_reg;
297
298     status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg;
299
300     status_fifo_wr_len = 0;
301     status_fifo_wr_tag = tag_reg;
302     status_fifo_wr_id = s_axis_write_data_tid;
303     status_fifo_wr_dest = s_axis_write_data_tdest;
304     status_fifo_wr_user = s_axis_write_data_tuser;
305     status_fifo_wr_mask = ram_wr_cmd_mask;
306     status_fifo_wr_last = 1'b0;
307     status_fifo_we = 1'b0;
308
309     inc_active = 1'b0;
310     dec_active = 1'b0;
311
312     out_done_ack = {SEG_COUNT{1'b0}};
313
314     case (state_reg)
315     STATE_IDLE: begin
316         // idle state - load new descriptor to start oper-
ation
317         s_axis_write_desc_ready_next = enable && act-
ive_count_av_reg;
318
319         addr_next = s_axis_write_desc_ram_addr &
ADDR_MASK;
320         last_cycle_offset_next = s_axis_write_desc_len &
OFFSET_MASK;
321
322         tag_next = s_axis_write_desc_tag;
323
324         length_next = 0;
325
326         cycle_count_next = (s_axis_write_desc_len - 1) >>
$log2(AXIS_KEEP_WIDTH_INT);
327         last_cycle_next = cycle_count_next == 0;
328         if (cycle_count_next == 0 &&
last_cycle_offset_next != 0) begin
329             keep_mask_next = {AXIS_KEEP_WIDTH_INT{1'b1}}
>> (AXIS_KEEP_WIDTH_INT - last_cycle_offset_next);
330         end else begin
331             keep_mask_next = {AXIS_KEEP_WIDTH_INT{1'b1}};
332         end
333
334         if (s_axis_write_desc_ready && s_ax-

```

```

is_write_desc_valid) begin
335     s_axis_write_desc_ready_next = 1'b0;
336     s_axis_write_data_tready_next =
&ram_wr_cmd_ready_int && !status_fifo_half_full_reg;
337
338     inc_active = 1'b1;
339
340     state_next = STATE_WRITE;
341 end else begin
342     state_next = STATE_IDLE;
343 end
344 end
345 STATE_WRITE: begin
346     // write state - generate write operations
347     s_axis_write_data_tready_next =
&ram_wr_cmd_ready_int && !status_fifo_half_full_reg;
348
349     if (s_axis_write_data_tready && s_ax-
is_write_data_tvalid) begin
350
351         // update counters
352         addr_next = addr_reg + AXIS_KEEP_WIDTH_INT;
353         length_next = length_reg +
AXIS_KEEP_WIDTH_INT;
354         cycle_count_next = cycle_count_reg - 1;
355         last_cycle_next = cycle_count_next == 0;
356         if (cycle_count_next == 0 &&
last_cycle_offset_reg != 0) begin
357             keep_mask_next =
{AXIS_KEEP_WIDTH_INT{1'b1}} >> (AXIS_KEEP_WIDTH_INT -
last_cycle_offset_reg);
358         end else begin
359             keep_mask_next =
{AXIS_KEEP_WIDTH_INT{1'b1}};
360         end
361
362         if (PART_COUNT > 1) begin
363             ram_wr_cmd_be_int = (s_ax-
is_write_data_tkeep & keep_mask_reg) << (addr_reg &
({PART_COUNT_WIDTH{1'b1}} << PART_OFFSET_WIDTH));
364         end else begin
365             ram_wr_cmd_be_int = s_ax-
is_write_data_tkeep & keep_mask_reg;
366         end
367         ram_wr_cmd_addr_int =
{SEG_COUNT{addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-SEG_AD-
DR_WIDTH]}};
368         ram_wr_cmd_data_int =
{PART_COUNT{s_axis_write_data_tdata}};
369         ram_wr_cmd_valid_int = ram_wr_cmd_mask;
370
371         // enqueue status FIFO entry for write comple-
tion
372         status_fifo_wr_len = length_next;
373         status_fifo_wr_tag = tag_reg;
374         status_fifo_wr_id = s_axis_write_data_tid;
375         status_fifo_wr_dest = s_axis_write_data_tdest;
376         status_fifo_wr_user = s_axis_write_data_tuser;
377         status_fifo_wr_mask = ram_wr_cmd_mask;
378         status_fifo_wr_last = 1'b0;
379         status_fifo_we = 1'b1;
380
381         if (AXIS_LAST_ENABLE && s_ax-
is_write_data_tlast) begin
382             if (AXIS_KEEP_ENABLE) begin
383                 cycle_size = AXIS_KEEP_WIDTH_INT;
384                 for (i = AXIS_KEEP_WIDTH_INT-1; i >=
0; i = i - 1) begin
385                     if (~(s_axis_write_data_tkeep &
keep_mask_reg) & (1 << i)) begin
386                         cycle_size = i;
387                     end
388                 end
389             end else begin
390                 cycle_size = AXIS_KEEP_WIDTH_INT;
391             end
392         end
393         // no more data to transfer, finish opera-
tion
394         if (last_cycle_reg &&
last_cycle_offset_reg > 0) begin
395             if (AXIS_KEEP_ENABLE && !(s_ax-
is_write_data_tkeep & keep_mask_reg & ~(AXIS_KEEP_WIDTH_INT{1'b1}}
>> (AXIS_KEEP_WIDTH_INT - last_cycle_offset_reg)))) begin
396                 length_next = length_reg +
cycle_size;
397             end else begin
398                 length_next = length_reg +
last_cycle_offset_reg;
399             end
400         end else begin
401             if (AXIS_KEEP_ENABLE) begin
402                 length_next = length_reg +
cycle_size;
403             end
404         end
405
406         // enqueue status FIFO entry for write
completion
407         status_fifo_wr_len = length_next;
408         status_fifo_wr_tag = tag_reg;

```

```

409         status_fifo_wr_id = s_axis_write_data_tid;
410         status_fifo_wr_dest = s_ax-
is_write_data_tdest;
411         status_fifo_wr_user = s_ax-
is_write_data_tuser;
412         status_fifo_wr_mask = ram_wr_cmd_mask;
413         status_fifo_wr_last = 1'b1;
414         status_fifo_we = 1'b1;
415
416         s_axis_write_data_tready_next = 1'b0;
417         s_axis_write_desc_ready_next = enable &&
active_count_av_reg;
418         state_next = STATE_IDLE;
419     end else if (last_cycle_reg) begin
420         if (last_cycle_offset_reg > 0) begin
421             length_next = length_reg +
last_cycle_offset_reg;
422         end
423
424         // enqueue status FIFO entry for write
completion
425         status_fifo_wr_len = length_next;
426         status_fifo_wr_tag = tag_reg;
427         status_fifo_wr_id = s_axis_write_data_tid;
428         status_fifo_wr_dest = s_ax-
is_write_data_tdest;
429         status_fifo_wr_user = s_ax-
is_write_data_tuser;
430         status_fifo_wr_mask = ram_wr_cmd_mask;
431         status_fifo_wr_last = 1'b1;
432         status_fifo_we = 1'b1;
433
434         if (AXIS_LAST_ENABLE) begin
435             s_axis_write_data_tready_next = 1'b1;
436             state_next = STATE_DROP_DATA;
437         end else begin
438             s_axis_write_data_tready_next = 1'b0;
439             s_axis_write_desc_ready_next = enable
&& active_count_av_reg;
440             state_next = STATE_IDLE;
441         end
442     end else begin
443         state_next = STATE_WRITE;
444     end
445 end else begin
446     state_next = STATE_WRITE;
447 end
448 end
449 STATE_DROP_DATA: begin
450     // drop excess AXI stream data
451     s_axis_write_data_tready_next = 1'b1;
452
453     if (s_axis_write_data_tready && s_ax-
is_write_data_tvalid) begin
454         if (s_axis_write_data_tlast) begin
455             s_axis_write_data_tready_next = 1'b0;
456             s_axis_write_desc_ready_next = enable &&
active_count_av_reg;
457             state_next = STATE_IDLE;
458         end else begin
459             state_next = STATE_DROP_DATA;
460         end
461     end else begin
462         state_next = STATE_DROP_DATA;
463     end
464 end
465 endcase
466
467     m_axis_write_desc_status_len_next =
status_fifo_len[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
468     m_axis_write_desc_status_tag_next =
status_fifo_tag[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
469     m_axis_write_desc_status_id_next =
status_fifo_id[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
470     m_axis_write_desc_status_dest_next =
status_fifo_dest[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
471     m_axis_write_desc_status_user_next =
status_fifo_user[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]];
472     m_axis_write_desc_status_valid_next = 1'b0;
473
474     if (status_fifo_rd_ptr_reg != status_fifo_wr_ptr_reg) be-
gin
475         // status FIFO not empty
476         if
((status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
& ~out_done) == 0) begin
477             // got write completion, pop and return status
478             status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg +
1;
479
480             out_done_ack =
status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
481
482             if
(status_fifo_last[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
) begin
483                 m_axis_write_desc_status_valid_next = 1'b1;

```

```

484
485         dec_active = 1'b1;
486     end
487 end
488 end
489 end
490
491 always @(posedge clk) begin
492     state_reg <= state_next;
493
494     s_axis_write_desc_ready_reg <= s_ax-
is_write_desc_ready_next;
495
496     m_axis_write_desc_status_len_reg <= m_ax-
is_write_desc_status_len_next;
497     m_axis_write_desc_status_tag_reg <= m_ax-
is_write_desc_status_tag_next;
498     m_axis_write_desc_status_id_reg <= m_ax-
is_write_desc_status_id_next;
499     m_axis_write_desc_status_dest_reg <= m_ax-
is_write_desc_status_dest_next;
500     m_axis_write_desc_status_user_reg <= m_ax-
is_write_desc_status_user_next;
501     m_axis_write_desc_status_valid_reg <= m_ax-
is_write_desc_status_valid_next;
502
503     s_axis_write_data_tready_reg <= s_ax-
is_write_data_tready_next;
504
505     addr_reg <= addr_next;
506     keep_mask_reg <= keep_mask_next;
507     last_cycle_offset_reg <= last_cycle_offset_next;
508     length_reg <= length_next;
509     cycle_count_reg <= cycle_count_next;
510     last_cycle_reg <= last_cycle_next;
511
512     tag_reg <= tag_next;
513
514     if (status_fifo_we) begin
515
516         status_fifo_len[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_len;
517
518         status_fifo_tag[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_tag;
519
520         status_fifo_id[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_id;
521
522         status_fifo_dest[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_dest;
523
524         status_fifo_user[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_user;
525
526         status_fifo_mask[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_mask;
527
528         status_fifo_last[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_last;
529
530         status_fifo_wr_ptr_reg <= status_fifo_wr_ptr_reg + 1;
531     end
532
533     status_fifo_rd_ptr_reg <= status_fifo_rd_ptr_next;
534
535     status_fifo_half_full_reg <= $un-
signed(status_fifo_wr_ptr_reg - status_fifo_rd_ptr_reg) >=
2**(STATUS_FIFO_ADDR_WIDTH-1);
536
537     if (active_count_reg < 2**STATUS_FIFO_ADDR_WIDTH &&
inc_active && !dec_active) begin
538         active_count_reg <= active_count_reg + 1;
539         active_count_av_reg <= active_count_reg <
(2**STATUS_FIFO_ADDR_WIDTH-1);
540     end else if (active_count_reg > 0 && !inc_active &&
dec_active) begin
541         active_count_reg <= active_count_reg - 1;
542         active_count_av_reg <= 1'b1;
543     end else begin
544         active_count_av_reg <= active_count_reg <
2**STATUS_FIFO_ADDR_WIDTH;
545     end
546
547     if (rst) begin
548         state_reg <= STATE_IDLE;
549
550         s_axis_write_desc_ready_reg <= 1'b0;
551         m_axis_write_desc_status_valid_reg <= 1'b0;
552
553         s_axis_write_data_tready_reg <= 1'b0;
554
555         status_fifo_wr_ptr_reg <= 0;
556         status_fifo_rd_ptr_reg <= 0;
557
558         active_count_reg <= 0;
559         active_count_av_reg <= 1'b1;
560     end
561 end
562
563 // output datapath logic (write data)
564 generate

```

```

557 genvar n;
558
559 for (n = 0; n < SEG_COUNT; n = n + 1) begin
560
561     reg [SEG_BE_WIDTH-1:0] ram_wr_cmd_be_reg =
562     {SEG_BE_WIDTH{1'b0}};
563     reg [SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_reg = {SEG_AD-
564     DR_WIDTH{1'b0}};
565     reg [SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_reg =
566     {SEG_DATA_WIDTH{1'b0}};
567     reg ram_wr_cmd_valid_reg = 1'b0;
568
569     reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg =
570     0;
571     reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg =
572     0;
573     reg out_fifo_half_full_reg = 1'b0;
574
575     wire out_fifo_full = out_fifo_wr_ptr_reg ==
576     (out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
577     wire out_fifo_empty = out_fifo_wr_ptr_reg ==
578     out_fifo_rd_ptr_reg;
579
580     (* ram_style = "distributed", ramstyle = "no_rw_check,
581     mlab" *)
582     reg [SEG_BE_WIDTH-1:0]
583     out_fifo_wr_cmd_be[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
584     (* ram_style = "distributed", ramstyle = "no_rw_check,
585     mlab" *)
586     reg [SEG_ADDR_WIDTH-1:0]
587     out_fifo_wr_cmd_addr[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
588     (* ram_style = "distributed", ramstyle = "no_rw_check,
589     mlab" *)
590     reg [SEG_DATA_WIDTH-1:0]
591     out_fifo_wr_cmd_data[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
592
593     reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] done_count_reg = 0;
594     reg done_reg = 1'b0;
595
596     assign ram_wr_cmd_ready_int[n +: 1] = !
597     out_fifo_half_full_reg;
598
599     assign ram_wr_cmd_be[n*SEG_BE_WIDTH +: SEG_BE_WIDTH] =
600     ram_wr_cmd_be_reg;
601
602     assign ram_wr_cmd_addr[n*SEG_ADDR_WIDTH +: SEG_ADDR_WIDTH] =
603     ram_wr_cmd_addr_reg;
604
605     assign ram_wr_cmd_data[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH] =
606     ram_wr_cmd_data_reg;
607     assign ram_wr_cmd_valid[n +: 1] = ram_wr_cmd_valid_reg;
608     assign out_done[n] = done_reg;
609
610     always @(posedge clk) begin
611         ram_wr_cmd_valid_reg <= ram_wr_cmd_valid_reg && !
612         ram_wr_cmd_ready[n +: 1];
613
614         out_fifo_half_full_reg <= $un-
615         signed(out_fifo_wr_ptr_reg - out_fifo_rd_ptr_reg) >= 2** (OUT-
616         PUT_FIFO_ADDR_WIDTH-1);
617
618         if (!out_fifo_full && ram_wr_cmd_valid_int[n +: 1])
619         begin
620             out_fifo_wr_cmd_be[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
621             <= ram_wr_cmd_be_int[n*SEG_BE_WIDTH +: SEG_BE_WIDTH];
622
623             out_fifo_wr_cmd_addr[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
624             <= ram_wr_cmd_addr_int[n*SEG_ADDR_WIDTH +: SEG_ADDR_WIDTH];
625
626             out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
627             <= ram_wr_cmd_data_int[n*SEG_DATA_WIDTH +: SEG_DATA_WIDTH];
628
629             out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
630
631             if (!out_fifo_empty && (!ram_wr_cmd_valid_reg ||
632             ram_wr_cmd_ready[n +: 1])) begin
633                 ram_wr_cmd_be_reg <=
634                 out_fifo_wr_cmd_be[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
635                 ram_wr_cmd_addr_reg <=
636                 out_fifo_wr_cmd_addr[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
637             ];
638                 ram_wr_cmd_data_reg <=
639                 out_fifo_wr_cmd_data[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
640             ];
641                 ram_wr_cmd_valid_reg <= 1'b1;
642                 out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
643             end
644
645             if (done_count_reg < 2**OUTPUT_FIFO_ADDR_WIDTH &&
646             ram_wr_done[n] && !out_done_ack[n]) begin
647                 done_count_reg <= done_count_reg + 1;
648                 done_reg <= 1;
649             end else if (done_count_reg > 0 && !ram_wr_done[n] &&
650             out_done_ack[n]) begin
651                 done_count_reg <= done_count_reg - 1;
652                 done_reg <= done_count_reg > 1;
653             end
654
655             if (rst) begin

```

```

621         out_fifo_wr_ptr_reg <= 0;
622         out_fifo_rd_ptr_reg <= 0;
623         ram_wr_cmd_valid_reg <= 1'b0;
624         done_count_reg <= 0;
625         done_reg <= 1'b0;
626     end
627 end
628
629 end
630
631 endgenerate
632
633 endmodule
634
635 `resetall

```

4. alexforench_verilog-pcie/rtl/dma_client_axis_source.v (568 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forench
4
5 Permission is hereby granted, free of charge, to any person
6 obtaining a copy
7 of this software and associated documentation files (the
8 "Software"), to deal
9 in the Software without restriction, including without limita-
10 tion the rights
11 to use, copy, modify, merge, publish, distribute, sublicense,
12 and/or sell
13 copies of the Software, and to permit persons to whom the
14 Software is
15 furnished to do so, subject to the following conditions:
16
17 1. The above copyright notice and this permission notice shall be
18 included in
19 all copies or substantial portions of the Software.
20
21 2. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
22 KIND, EXPRESS OR
23 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
24 CHANTABILITY
25 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
26 EVENT SHALL THE
27 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
28 OR OTHER
29 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
30 WISE, ARISING FROM,
31 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
32 DEALINGS IN
33 THE SOFTWARE.
34
35 // Language: Verilog 2001
36
37 `resetall
38 `timescale 1ns / 1ps
39 `default_nettype none
40
41 /*
42 * AXI stream source DMA client
43 */
44 module dma_client_axis_source #
45 (
46     // RAM address width
47     parameter RAM_ADDR_WIDTH = 16,
48     // RAM segment count
49     parameter SEG_COUNT = 2,
50     // RAM segment data width
51     parameter SEG_DATA_WIDTH = 64,
52     // RAM segment byte enable width
53     parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
54     // RAM segment address width
55     parameter SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
56     g2(SEG_COUNT*SEG_BE_WIDTH),
57     // Width of AXI stream interfaces in bits
58     parameter AXIS_DATA_WIDTH = SEG_DATA_WIDTH*SEG_COUNT/2,
59     // Use AXI stream tkeep signal
60     parameter AXIS_KEEP_ENABLE = (AXIS_DATA_WIDTH>8),
61     // AXI stream tkeep signal width (words per cycle)
62     parameter AXIS_KEEP_WIDTH = (AXIS_DATA_WIDTH/8),
63     // Use AXI stream tlast signal
64     parameter AXIS_LAST_ENABLE = 1,
65     // Propagate AXI stream tid signal
66     parameter AXIS_ID_ENABLE = 0,
67     // AXI stream tid signal width
68     parameter AXIS_ID_WIDTH = 8,
69     // Propagate AXI stream tdest signal
70     parameter AXIS_DEST_ENABLE = 0,
71     // AXI stream tdest signal width
72     parameter AXIS_DEST_WIDTH = 8,
73     // Propagate AXI stream tuser signal
74     parameter AXIS_USER_ENABLE = 1,
75     // AXI stream tuser signal width
76     parameter AXIS_USER_WIDTH = 1,
77     // Width of length field
78     parameter LEN_WIDTH = 16,

```

```

68 // Width of tag field
69 parameter TAG_WIDTH = 8
70 )
71 (
72   input wire clk,
73   input wire rst,
74
75   /*
76    * AXI read descriptor input
77    */
78   input wire [RAM_ADDR_WIDTH-1:0] s_ax-
is_read_desc_ram_addr,
79   input wire [LEN_WIDTH-1:0] s_ax-
is_read_desc_len,
80   input wire [TAG_WIDTH-1:0] s_ax-
is_read_desc_tag,
81   input wire [AXIS_ID_WIDTH-1:0] s_ax-
is_read_desc_id,
82   input wire [AXIS_DEST_WIDTH-1:0] s_ax-
is_read_desc_dest,
83   input wire [AXIS_USER_WIDTH-1:0] s_ax-
is_read_desc_user,
84   input wire s_ax-
is_read_desc_valid,
85   output wire s_ax-
is_read_desc_ready,
86
87   /*
88    * AXI read descriptor status output
89    */
90   output wire [TAG_WIDTH-1:0] m_ax-
is_read_desc_status_tag,
91   output wire [3:0] m_ax-
is_read_desc_status_error,
92   output wire m_ax-
is_read_desc_status_valid,
93
94   /*
95    * AXI stream read data output
96    */
97   output wire [AXIS_DATA_WIDTH-1:0] m_ax-
is_read_data_tdata,
98   output wire [AXIS_KEEP_WIDTH-1:0] m_ax-
is_read_data_tkeep,
99   output wire m_ax-
is_read_data_tvalid,
100  input wire m_ax-
is_read_data_tready,
101  output wire m_ax-
is_read_data_tlast,
102  output wire [AXIS_ID_WIDTH-1:0] m_ax-
is_read_data_tid,
103  output wire [AXIS_DEST_WIDTH-1:0] m_ax-
is_read_data_tdest,
104  output wire [AXIS_USER_WIDTH-1:0] m_ax-
is_read_data_tuser,
105
106  /*
107   * RAM interface
108   */
109  output wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_rd_cmd_ad-
dr,
110  output wire [SEG_COUNT-1:0] ram_rd_cmd_valid,
111  input wire [SEG_COUNT-1:0] ram_rd_cmd_ready,
112  input wire [SEG_COUNT*SEG_DATA_WIDTH-1:0] ram_rd_resp_data,
113  input wire [SEG_COUNT-1:0] ram_rd_resp_valid,
114  output wire [SEG_COUNT-1:0] ram_rd_resp_ready,
115
116  /*
117   * Configuration
118   */
119  input wire enable
120 );
121
122 parameter RAM_WORD_WIDTH = SEG_BE_WIDTH;
123 parameter RAM_WORD_SIZE = SEG_DATA_WIDTH/RAM_WORD_WIDTH;
124
125 parameter AXIS_KEEP_WIDTH_INT = AXIS_KEEP_ENABLE ?
AXIS_KEEP_WIDTH : 1;
126 parameter AXIS_WORD_WIDTH = AXIS_KEEP_WIDTH_INT;
127 parameter AXIS_WORD_SIZE = AXIS_DATA_WIDTH/AXIS_WORD_WIDTH;
128
129 parameter PART_COUNT = SEG_COUNT*SEG_BE_WIDTH /
AXIS_KEEP_WIDTH_INT;
130 parameter PART_COUNT_WIDTH = PART_COUNT > 1 ? $clog-
2(PART_COUNT) : 1;
131 parameter PART_OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
132 parameter PARTS_PER_SEG = (SEG_BE_WIDTH + AXIS_KEEP_WIDTH_INT
- 1) / AXIS_KEEP_WIDTH_INT;
133 parameter SEGS_PER_PART = (AXIS_KEEP_WIDTH_INT + SEG_BE_WIDTH
- 1) / SEG_BE_WIDTH;
134
135 parameter OFFSET_WIDTH = AXIS_KEEP_WIDTH_INT > 1 ? $clog-
2(AXIS_KEEP_WIDTH_INT) : 1;
136 parameter OFFSET_MASK = AXIS_KEEP_WIDTH_INT > 1 ? {OFF-

```

```

SET_WIDTH{1'b1}} : 0;
137 parameter ADDR_MASK = {RAM_ADDR_WIDTH{1'b1}} << $clog-
2(AXIS_KEEP_WIDTH_INT);
138 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - $clog-
2(AXIS_KEEP_WIDTH_INT) + 1;
139
140 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
141
142 // bus width assertions
143 initial begin
144   if (RAM_WORD_SIZE * SEG_BE_WIDTH != SEG_DATA_WIDTH) begin
145     $error("Error: RAM data width not evenly divisble (in-
stance %m)");
146   $finish;
147   end
148
149   if (AXIS_WORD_SIZE * AXIS_KEEP_WIDTH_INT !=
AXIS_DATA_WIDTH) begin
150     $error("Error: AXI stream data width not evenly divis-
ble (instance %m)");
151   $finish;
152   end
153
154   if (RAM_WORD_SIZE != AXIS_WORD_SIZE) begin
155     $error("Error: word size mismatch (instance %m)");
156   $finish;
157   end
158
159   if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
160     $error("Error: RAM word width must be even power of
two (instance %m)");
161   $finish;
162   end
163
164   if (RAM_ADDR_WIDTH != SEG_ADDR_WIDTH+$clog2(SEG_COUNT)+
$clog2(SEG_BE_WIDTH)) begin
165     $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
166   $finish;
167   end
168
169   if (AXIS_DATA_WIDTH > SEG_COUNT*SEG_DATA_WIDTH) begin
170     $error("Error: AXI stream interface width must not be
wider than RAM interface width (instance %m)");
171   $finish;
172   end
173
174   if (AXIS_DATA_WIDTH*2**$clog2(PART_COUNT) !=
SEG_COUNT*SEG_DATA_WIDTH) begin
175     $error("Error: AXI stream interface width must be a
power of two fraction of RAM interface width (instance %m)");
176   $finish;
177   end
178 end
179
180 localparam [0:0]
181   READ_STATE_IDLE = 1'd0,
182   READ_STATE_READ = 1'd1;
183
184 reg [0:0] read_state_reg = READ_STATE_IDLE, read_state_next;
185
186 localparam [0:0]
187   AXIS_STATE_IDLE = 1'd0,
188   AXIS_STATE_READ = 1'd1;
189
190 reg [0:0] axis_state_reg = AXIS_STATE_IDLE, axis_state_next;
191
192 // datapath control signals
193 reg axis_cmd_ready;
194
195 integer i;
196
197 reg [RAM_ADDR_WIDTH-1:0] read_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, read_addr_next;
198 reg [SEG_COUNT-1:0] read_ram_mask_reg = 0, read_ram_mask_next;
199 reg [CYCLE_COUNT_WIDTH-1:0] read_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, read_cycle_count_next;
200
201 reg [RAM_ADDR_WIDTH-1:0] axis_cmd_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, axis_cmd_addr_next;
202 reg [OFFSET_WIDTH-1:0] axis_cmd_last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, axis_cmd_last_cycle_offset_next;
203 reg [CYCLE_COUNT_WIDTH-1:0] axis_cmd_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, axis_cmd_cycle_count_next;
204 reg [TAG_WIDTH-1:0] axis_cmd_tag_reg = {TAG_WIDTH{1'b0}},
axis_cmd_tag_next;
205 reg [AXIS_ID_WIDTH-1:0] axis_cmd_axis_id_reg =
{AXIS_ID_WIDTH{1'b0}}, axis_cmd_axis_id_next;
206 reg [AXIS_DEST_WIDTH-1:0] axis_cmd_axis_dest_reg = {AXIS_D-
EST_WIDTH{1'b0}}, axis_cmd_axis_dest_next;
207 reg [AXIS_USER_WIDTH-1:0] axis_cmd_axis_user_reg =
{AXIS_USER_WIDTH{1'b0}}, axis_cmd_axis_user_next;
208 reg axis_cmd_valid_reg = 1'b0, axis_cmd_valid_next;
209
210 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
211 reg [SEG_COUNT-1:0] ram_mask_reg = 0, ram_mask_next;
212 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, last_cycle_offset_next;
213 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;

```



```

214 reg last_cycle_reg = 1'b0, last_cycle_next;
215
216 reg [AXIS_ID_WIDTH-1:0] axis_id_reg = {AXIS_ID_WIDTH{1'b0}},
axis_id_next;
217 reg [AXIS_DEST_WIDTH-1:0] axis_dest_reg = {AXIS_DEST_WIDTH{1'b0}}, axis_dest_next;
218 reg [AXIS_USER_WIDTH-1:0] axis_user_reg = {AXIS_USER_WIDTH{1'b0}}, axis_user_next;
219
220 reg s_axis_read_desc_ready_reg = 1'b0, s_axis_read_desc_ready_next;
221
222 reg [TAG_WIDTH-1:0] m_axis_read_desc_status_tag_reg = {TAG_WIDTH{1'b0}}, m_axis_read_desc_status_tag_next;
223 reg m_axis_read_desc_status_valid_reg = 1'b0, m_axis_read_desc_status_valid_next;
224
225 reg [SEG_COUNT*SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr_reg = 0, ram_rd_cmd_addr_next;
226 reg [SEG_COUNT-1:0] ram_rd_cmd_valid_reg = 0, ram_rd_cmd_valid_next;
227 reg [SEG_COUNT-1:0] ram_rd_resp_ready_cmb;
228
229 // internal datapath
230 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tdata_int;
231 reg [AXIS_KEEP_WIDTH-1:0] m_axis_read_data_tkeep_int;
232 reg m_axis_read_data_tvalid_int;
233 wire m_axis_read_data_tready_int;
234 reg m_axis_read_data_tlast_int;
235 reg [AXIS_ID_WIDTH-1:0] m_axis_read_data_tid_int;
236 reg [AXIS_DEST_WIDTH-1:0] m_axis_read_data_tdest_int;
237 reg [AXIS_USER_WIDTH-1:0] m_axis_read_data_tuser_int;
238
239 assign s_axis_read_desc_ready = s_axis_read_desc_ready_reg;
240
241 assign m_axis_read_desc_status_tag = m_axis_read_desc_status_tag_reg;
242 assign m_axis_read_desc_status_error = 4'd0;
243 assign m_axis_read_desc_status_valid = m_axis_read_desc_status_valid_reg;
244
245 assign ram_rd_cmd_addr = ram_rd_cmd_addr_reg;
246 assign ram_rd_cmd_valid = ram_rd_cmd_valid_reg;
247 assign ram_rd_resp_ready = ram_rd_resp_ready_cmb;
248
249 always @* begin
250     read_state_next = READ_STATE_IDLE;
251
252     s_axis_read_desc_ready_next = 1'b0;
253
254     ram_rd_cmd_addr_next = ram_rd_cmd_addr_reg;
255     ram_rd_cmd_valid_next = ram_rd_cmd_valid_reg & ~ram_rd_cmd_ready;
256
257     read_addr_next = read_addr_reg;
258     read_ram_mask_next = read_ram_mask_reg;
259     read_cycle_count_next = read_cycle_count_reg;
260
261     axis_cmd_addr_next = axis_cmd_addr_reg;
262     axis_cmd_last_cycle_offset_next = axis_cmd_last_cycle_offset_reg;
263     axis_cmd_cycle_count_next = axis_cmd_cycle_count_reg;
264     axis_cmd_tag_next = axis_cmd_tag_reg;
265     axis_cmd_axis_id_next = axis_cmd_axis_id_reg;
266     axis_cmd_axis_dest_next = axis_cmd_axis_dest_reg;
267     axis_cmd_axis_user_next = axis_cmd_axis_user_reg;
268     axis_cmd_valid_next = axis_cmd_valid_reg && !axis_cmd_ready;
269
270     case (read_state_reg)
271     READ_STATE_IDLE: begin
272         // idle state - load new descriptor to start operation
273         s_axis_read_desc_ready_next = !axis_cmd_valid_reg && enable;
274
275         if (s_axis_read_desc_ready && s_axis_read_desc_valid) begin
276             read_addr_next = s_axis_read_desc_ram_addr & ADDR_MASK;
277
278             if (PART_COUNT > 1) begin
279
280                 read_ram_mask_next = {SEGS_PER_PART{1'b1}} << (((read_addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}}) / PARTS_PER_SEG) * SEGS_PER_PART);
281                 end else begin
282                     read_ram_mask_next = {SEG_COUNT{1'b1}};
283                 end
284
285                 axis_cmd_addr_next = s_axis_read_desc_ram_addr & ADDR_MASK;
286                 axis_cmd_last_cycle_offset_next = s_axis_read_desc_len & OFFSET_MASK;
287
288                 axis_cmd_tag_next = s_axis_read_desc_tag;
289
290                 axis_cmd_axis_id_next = s_axis_read_desc_id;
291                 axis_cmd_axis_dest_next = s_axis_read_desc_dest;

```

```

is_read_desc_dest;
292     axis_cmd_axis_user_next = s_axis_read_desc_user;
293
294     axis_cmd_cycle_count_next = (s_axis_read_desc_len - 1) >> $clog2(AXIS_KEEP_WIDTH_INT);
295     read_cycle_count_next = (s_axis_read_desc_len - 1) >> $clog2(AXIS_KEEP_WIDTH_INT);
296
297     axis_cmd_valid_next = 1'b1;
298
299     s_axis_read_desc_ready_next = 1'b0;
300     read_state_next = READ_STATE_READ;
301 end else begin
302     read_state_next = READ_STATE_IDLE;
303 end
304 end
305 READ_STATE_READ: begin
306     // read state - start new read operations
307
308     if (!(ram_rd_cmd_valid & ~ram_rd_cmd_ready & read_ram_mask_reg)) begin
309
310         // update counters
311         read_addr_next = read_addr_reg + AXIS_KEEP_WIDTH_INT;
312
313         read_cycle_count_next = read_cycle_count_reg - 1;
314
315         if (PART_COUNT > 1) begin
316             read_ram_mask_next = {SEGS_PER_PART{1'b1}} << (((read_addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}}) / PARTS_PER_SEG) * SEGS_PER_PART);
317         end else begin
318             read_ram_mask_next = {SEG_COUNT{1'b1}};
319         end
320
321         for (i = 0; i < SEG_COUNT; i = i + 1) begin
322             if (read_ram_mask_reg[i]) begin
323                 ram_rd_cmd_addr_next[i*SEG_ADDR_WIDTH+: SEG_ADDR_WIDTH] = read_addr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-SEG_ADDR_WIDTH];
324                 ram_rd_cmd_valid_next[i] = 1'b1;
325             end
326         end
327
328         if (read_cycle_count_reg == 0) begin
329             s_axis_read_desc_ready_next = !axis_cmd_valid_reg && enable;
330             read_state_next = READ_STATE_IDLE;
331         end else begin
332             read_state_next = READ_STATE_READ;
333         end
334     end else begin
335         read_state_next = READ_STATE_READ;
336     end
337 endcase
338 end
339
340 always @* begin
341     axis_state_next = AXIS_STATE_IDLE;
342
343     m_axis_read_desc_status_tag_next = m_axis_read_desc_status_tag_reg;
344     m_axis_read_desc_status_valid_next = 1'b0;
345
346     if (PART_COUNT > 1) begin
347         m_axis_read_data_tdata_int = ram_rd_resp_data >> ((read_addr_reg >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}}) * AXIS_DATA_WIDTH);
348     end else begin
349         m_axis_read_data_tdata_int = ram_rd_resp_data;
350     end
351     m_axis_read_data_tkeep_int = {AXIS_KEEP_WIDTH{1'b1}};
352     m_axis_read_data_tlast_int = 1'b0;
353     m_axis_read_data_tvalid_int = 1'b0;
354     m_axis_read_data_tid_int = axis_id_reg;
355     m_axis_read_data_tdest_int = axis_dest_reg;
356     m_axis_read_data_tuser_int = axis_user_reg;
357
358     ram_rd_resp_ready_cmb = {SEG_COUNT{1'b0}};
359
360     axis_cmd_ready = 1'b0;
361
362     addr_next = addr_reg;
363     ram_mask_next = ram_mask_reg;
364     last_cycle_offset_next = last_cycle_offset_reg;
365     cycle_count_next = cycle_count_reg;
366     last_cycle_next = last_cycle_reg;
367
368     axis_id_next = axis_id_reg;
369     axis_dest_next = axis_dest_reg;
370     axis_user_next = axis_user_reg;
371
372     case (axis_state_reg)
373     AXIS_STATE_IDLE: begin
374         // idle state - load new descriptor to start operation

```

```

375
376 // store transfer parameters
377 addr_next = axis_cmd_addr_reg;
378 last_cycle_offset_next =
axis_cmd_last_cycle_offset_reg;
379 cycle_count_next = axis_cmd_cycle_count_reg;
380 last_cycle_next = axis_cmd_cycle_count_reg == 0;
381
382 if (PART_COUNT > 1) begin
383   ram_mask_next = {SEGS_PER_PART{1'b1}} <<
(((addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}})) /
PARTS_PER_SEG) * SEGS_PER_PART;
384   end else begin
385     ram_mask_next = {SEG_COUNT{1'b1}};
386   end
387
388 m_axis_read_desc_status_tag_next =
axis_cmd_tag_reg;
389 axis_id_next = axis_cmd_axis_id_reg;
390 axis_dest_next = axis_cmd_axis_dest_reg;
391 axis_user_next = axis_cmd_axis_user_reg;
392
393 if (axis_cmd_valid_reg) begin
394   axis_cmd_ready = 1'b1;
395   axis_state_next = AXIS_STATE_READ;
396 end
397 end
398 AXIS_STATE_READ: begin
399   // handle read data
400   ram_rd_resp_ready_cmb = {SEG_COUNT{1'b0}};
401
402   if (!(ram_mask_reg & ~ram_rd_resp_valid) && m_ax-
is_read_data_tready_int) begin
403     // transfer in read data
404     ram_rd_resp_ready_cmb = ram_mask_reg;
405
406     // update counters
407     addr_next = addr_reg + AXIS_KEEP_WIDTH_INT;
408     cycle_count_next = cycle_count_reg - 1;
409     last_cycle_next = cycle_count_next == 0;
410
411     if (PART_COUNT > 1) begin
412       ram_mask_next = {SEGS_PER_PART{1'b1}} <<
(((addr_next >> PART_OFFSET_WIDTH) & ({PART_COUNT_WIDTH{1'b1}})) /
PARTS_PER_SEG) * SEGS_PER_PART;
413     end else begin
414       ram_mask_next = {SEG_COUNT{1'b1}};
415     end
416
417     if (PART_COUNT > 1) begin
418       m_axis_read_data_tdata_int =
ram_rd_resp_data >> (((addr_reg >> PART_OFFSET_WIDTH) &
{PART_COUNT_WIDTH{1'b1}}) * AXIS_DATA_WIDTH);
419     end else begin
420       m_axis_read_data_tdata_int =
ram_rd_resp_data;
421     end
422     m_axis_read_data_tkeep_int =
{AXIS_KEEP_WIDTH_INT{1'b1}};
423     m_axis_read_data_tvalid_int = 1'b1;
424
425     if (last_cycle_reg) begin
426       // no more data to transfer, finish opera-
tion
427       if (last_cycle_offset_reg > 0) begin
428         m_axis_read_data_tkeep_int =
{AXIS_KEEP_WIDTH_INT{1'b1}} >> (AXIS_KEEP_WIDTH_INT -
last_cycle_offset_reg);
429       end
430       m_axis_read_data_tlast_int = 1'b1;
431
432       m_axis_read_desc_status_valid_next = 1'b1;
433
434       axis_state_next = AXIS_STATE_IDLE;
435     end else begin
436       // more cycles in AXI transfer
437       axis_state_next = AXIS_STATE_READ;
438     end
439   end else begin
440     axis_state_next = AXIS_STATE_READ;
441   end
442 end
443 endcase
444 end
445
446 always @(posedge clk) begin
447   read_state_reg <= read_state_next;
448   axis_state_reg <= axis_state_next;
449
450   s_axis_read_desc_ready_reg <= s_axis_read_desc_ready_next;
451
452   m_axis_read_desc_status_tag_reg <= m_ax-
is_read_desc_status_tag_next;
453   m_axis_read_desc_status_valid_reg <= m_ax-
is_read_desc_status_valid_next;
454
455   ram_rd_cmd_addr_reg <= ram_rd_cmd_addr_next;
456   ram_rd_cmd_valid_reg <= ram_rd_cmd_valid_next;
457
458   read_addr_reg <= read_addr_next;
459   read_ram_mask_reg <= read_ram_mask_next;

```

```

460   read_cycle_count_reg <= read_cycle_count_next;
461
462   axis_cmd_addr_reg <= axis_cmd_addr_next;
463   axis_cmd_last_cycle_offset_reg <=
axis_cmd_last_cycle_offset_next;
464   axis_cmd_cycle_count_reg <= axis_cmd_cycle_count_next;
465   axis_cmd_tag_reg <= axis_cmd_tag_next;
466   axis_cmd_axis_id_reg <= axis_cmd_axis_id_next;
467   axis_cmd_axis_dest_reg <= axis_cmd_axis_dest_next;
468   axis_cmd_axis_user_reg <= axis_cmd_axis_user_next;
469   axis_cmd_valid_reg <= axis_cmd_valid_next;
470
471   addr_reg <= addr_next;
472   ram_mask_reg <= ram_mask_next;
473   last_cycle_offset_reg <= last_cycle_offset_next;
474   cycle_count_reg <= cycle_count_next;
475   last_cycle_reg <= last_cycle_next;
476
477   axis_id_reg <= axis_id_next;
478   axis_dest_reg <= axis_dest_next;
479   axis_user_reg <= axis_user_next;
480
481   if (rst) begin
482     read_state_reg <= READ_STATE_IDLE;
483     axis_state_reg <= AXIS_STATE_IDLE;
484
485     axis_cmd_valid_reg <= 1'b0;
486
487     s_axis_read_desc_ready_reg <= 1'b0;
488     m_axis_read_desc_status_valid_reg <= 1'b0;
489
490     ram_rd_cmd_valid_reg <= {SEG_COUNT{1'b0}};
491   end
492 end
493
494 // output datapath logic
495 reg [AXIS_DATA_WIDTH-1:0] m_axis_read_data_tdata_reg =
{AXIS_DATA_WIDTH{1'b0}};
496 reg [AXIS_KEEP_WIDTH-1:0] m_axis_read_data_tkeep_reg =
{AXIS_KEEP_WIDTH{1'b0}};
497 reg m_axis_read_data_tvalid_reg = 1'b0;
498 reg m_axis_read_data_tlast_reg = 1'b0;
499 reg [AXIS_ID_WIDTH-1:0] m_axis_read_data_tid_reg =
{AXIS_ID_WIDTH{1'b0}};
500 reg [AXIS_DEST_WIDTH-1:0] m_axis_read_data_tdest_reg =
{AXIS_DEST_WIDTH{1'b0}};
501 reg [AXIS_USER_WIDTH-1:0] m_axis_read_data_tuser_reg =
{AXIS_USER_WIDTH{1'b0}};
502
503 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg = 0;
504 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg = 0;
505 reg out_fifo_half_full_reg = 1'b0;
506
507 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
508 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
509
510 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
511 reg [AXIS_DATA_WIDTH-1:0]
out_fifo_tdata[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
512 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
513 reg [AXIS_KEEP_WIDTH-1:0]
out_fifo_tkeep[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
514 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
515 reg
out_fifo_tlast[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
516 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
517 reg [AXIS_ID_WIDTH-1:0]
out_fifo_tid[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
518 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
519 reg [AXIS_DEST_WIDTH-1:0]
out_fifo_tdest[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
520 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
521 reg [AXIS_USER_WIDTH-1:0]
out_fifo_tuser[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
522
523 assign m_axis_read_data_tready_int = !out_fifo_half_full_reg;
524
525 assign m_axis_read_data_tdata = m_axis_read_data_tdata_reg;
526 assign m_axis_read_data_tkeep = AXIS_KEEP_ENABLE ? m_ax-
is_read_data_tkeep_reg : {AXIS_KEEP_WIDTH{1'b1}};
527 assign m_axis_read_data_tvalid = m_axis_read_data_tvalid_reg;
528 assign m_axis_read_data_tlast = AXIS_LAST_ENABLE ? m_ax-
is_read_data_tlast_reg : 1'b1;
529 assign m_axis_read_data_tid = AXIS_ID_ENABLE ? m_ax-
is_read_data_tid_reg : {AXIS_ID_WIDTH{1'b0}};
530 assign m_axis_read_data_tdest = AXIS_DEST_ENABLE ? m_ax-
is_read_data_tdest_reg : {AXIS_DEST_WIDTH{1'b0}};
531 assign m_axis_read_data_tuser = AXIS_USER_ENABLE ? m_ax-
is_read_data_tuser_reg : {AXIS_USER_WIDTH{1'b0}};
532
533 always @(posedge clk) begin
534   m_axis_read_data_tvalid_reg <= m_axis_read_data_tvalid_reg

```



```

535    && !m_axis_read_data_tready;
536    out_fifo_half_full_reg <= $unsigned(out_fifo_wr_ptr_reg -
out_fifo_rd_ptr_reg) >= 2**(OUTPUT_FIFO_ADDR_WIDTH-1);
537
538    if (!out_fifo_full && m_axis_read_data_tvalid_int) begin
539
out_fifo_tdata[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tdata_int;
540
out_fifo_tkeep[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tkeep_int;
541
out_fifo_tlast[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tlast_int;
542
out_fifo_tid[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tid_int;
543
out_fifo_tdest[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tdest_int;
544
out_fifo_tuser[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axis_read_data_tuser_int;
545    out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
546    end
547
548    if (!out_fifo_empty && (!m_axis_read_data_tvalid_reg ||
m_axis_read_data_tready)) begin
549        m_axis_read_data_tdata_reg <=
out_fifo_tdata[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
550        m_axis_read_data_tkeep_reg <=
out_fifo_tkeep[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
551        m_axis_read_data_tvalid_reg <= 1'b1;
552        m_axis_read_data_tlast_reg <=
out_fifo_tlast[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
553        m_axis_read_data_tid_reg <=
out_fifo_tid[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
554        m_axis_read_data_tdest_reg <=
out_fifo_tdest[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
555        m_axis_read_data_tuser_reg <=
out_fifo_tuser[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
556        out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
557    end
558
559    if (rst) begin
560        out_fifo_wr_ptr_reg <= 0;
561        out_fifo_rd_ptr_reg <= 0;
562        m_axis_read_data_tvalid_reg <= 1'b0;
563    end
564 end
565
566 endmodule
567
568 `resetall

```

5. alexforenrich_verilog-pcie/rtl/dma_if_axi.v (426 lines)

```

1  /*
2
3  Copyright (c) 2021 Alex Forenrich
4
5  Permission is hereby granted, free of charge, to any person
obtaining a copy
6  of this software and associated documentation files (the
"Software"), to deal
7  in the Software without restriction, including without limita-
tion the rights
8  to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9  copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none

```

```

30
31 /*
32  * AXI DMA interface
33  */
34 module dma_if_axi #
35 (
36     // Width of AXI data bus in bits
37     parameter AXI_DATA_WIDTH = 32,
38     // Width of AXI address bus in bits
39     parameter AXI_ADDR_WIDTH = 16,
40     // Width of AXI wstrb (width of data bus in words)
41     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
42     // Width of AXI ID signal
43     parameter AXI_ID_WIDTH = 8,
44     // Maximum AXI burst length to generate
45     parameter AXI_MAX_BURST_LEN = 256,
46     // RAM select width
47     parameter RAM_SEL_WIDTH = 2,
48     // RAM address width
49     parameter RAM_ADDR_WIDTH = 16,
50     // RAM segment count
51     parameter RAM_SEG_COUNT = 2,
52     // RAM segment data width
53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
g2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Immediate enable
59     parameter IMM_ENABLE = 0,
60     // Immediate width
61     parameter IMM_WIDTH = 32,
62     // Length field width
63     parameter LEN_WIDTH = 16,
64     // Tag field width
65     parameter TAG_WIDTH = 8,
66     // Operation table size (read)
67     parameter READ_OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
68     // Operation table size (write)
69     parameter WRITE_OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
70     // Use AXI ID signals (read)
71     parameter READ_USE_AXI_ID = 0,
72     // Use AXI ID signals (write)
73     parameter WRITE_USE_AXI_ID = 1
74 )
75 (
76     input  wire
77     input  wire
78
79     /*
80     * AXI master interface
81     */
82     output wire [AXI_ID_WIDTH-1:0]
m_axi_awid,
83     output wire [AXI_ADDR_WIDTH-1:0]
m_axi_awaddr,
84     output wire [7:0]
m_axi_awlen,
85     output wire [2:0]
m_axi_awsz,
86     output wire [1:0]
m_axi_awburst,
87     output wire
m_axi_awlock,
88     output wire [3:0]
m_axi_awcache,
89     output wire [2:0]
m_axi_awprot,
90     output wire
m_axi_awvalid,
91     input  wire
m_axi_awready,
92     output wire [AXI_DATA_WIDTH-1:0]
m_axi_wdata,
93     output wire [AXI_STRB_WIDTH-1:0]
m_axi_wstrb,
94     output wire
m_axi_wlast,
95     output wire
m_axi_wvalid,
96     input  wire
m_axi_wready,
97     input  wire [AXI_ID_WIDTH-1:0]
m_axi_bid,
98     input  wire [1:0]
m_axi_bresp,
99     input  wire
m_axi_bvalid,
100    output wire
m_axi_bready,
101    output wire [AXI_ID_WIDTH-1:0]
m_axi_arid,
102    output wire [AXI_ADDR_WIDTH-1:0]
m_axi_araddr,
103    output wire [7:0]
m_axi_arlen,
104    output wire [2:0]
m_axi_arsz,
105    output wire [1:0]

```

clk,
rst,

```

m_axi_arburst,
106 output wire
m_axi_arlock,
107 output wire [3:0]
m_axi_arscache,
108 output wire [2:0]
m_axi_arprot,
109 output wire
m_axi_arvalid,
110 input wire
m_axi_arready,
111 input wire [AXI_ID_WIDTH-1:0]
m_axi_rid,
112 input wire [AXI_DATA_WIDTH-1:0]
m_axi_rdata,
113 input wire [1:0]
m_axi_rresp,
114 input wire
m_axi_rlast,
115 input wire
m_axi_rvalid,
116 output wire
m_axi_rready,
117
118 /*
119  * AXI read descriptor input
120  */
121 input wire [AXI_ADDR_WIDTH-1:0]
is_read_desc_axi_addr,
122 input wire [RAM_SEL_WIDTH-1:0]
is_read_desc_ram_sel,
123 input wire [RAM_ADDR_WIDTH-1:0]
is_read_desc_ram_addr,
124 input wire [LEN_WIDTH-1:0]
is_read_desc_len,
125 input wire [TAG_WIDTH-1:0]
is_read_desc_tag,
126 input wire
is_read_desc_valid,
127 output wire
is_read_desc_ready,
128
129 /*
130  * AXI read descriptor status output
131  */
132 output wire [TAG_WIDTH-1:0]
is_read_desc_status_tag,
133 output wire [3:0]
is_read_desc_status_error,
134 output wire
is_read_desc_status_valid,
135
136 /*
137  * AXI write descriptor input
138  */
139 input wire [AXI_ADDR_WIDTH-1:0]
is_write_desc_axi_addr,
140 input wire [RAM_SEL_WIDTH-1:0]
is_write_desc_ram_sel,
141 input wire [RAM_ADDR_WIDTH-1:0]
is_write_desc_ram_addr,
142 input wire [IMM_WIDTH-1:0]
is_write_desc_imm,
143 input wire
is_write_desc_imm_en,
144 input wire [LEN_WIDTH-1:0]
is_write_desc_len,
145 input wire [TAG_WIDTH-1:0]
is_write_desc_tag,
146 input wire
is_write_desc_valid,
147 output wire
is_write_desc_ready,
148
149 /*
150  * AXI write descriptor status output
151  */
152 output wire [TAG_WIDTH-1:0]
is_write_desc_status_tag,
153 output wire [3:0]
is_write_desc_status_error,
154 output wire
is_write_desc_status_valid,
155
156 /*
157  * RAM interface
158  */
159 output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_wr_cmd_sel,
160 output wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0]
ram_wr_cmd_be,
161 output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_addr,
162 output wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data,
163 output wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_valid,
164 input wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready,
165 input wire [RAM_SEG_COUNT-1:0]
ram_wr_done,
166 output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_rd_cmd_sel,
167 output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_addr,
168 output wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_valid,
169 input wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_ready,
170 input wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_rd_resp_data,
171 input wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_valid,
172 output wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_ready,
173
174 /*
175  * Configuration
176  */
177 input wire
read_enable,
178 input wire
write_enable,
179
180 /*
181  * Status
182  */
183 output wire
status_rd_busy,
184 output wire
status_wr_busy,
185
186 /*
187  * Statistics
188  */
189 output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_op_start_tag,
190 output wire [LEN_WIDTH-1:0]
stat_rd_op_start_len,
191 output wire
stat_rd_op_start_valid,
192 output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_op_finish_tag,
193 output wire [3:0]
stat_rd_op_finish_status,
194 output wire
stat_rd_op_finish_valid,
195 output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_req_start_tag,
196 output wire [12:0]
stat_rd_req_start_len,
197 output wire
stat_rd_req_start_valid,
198 output wire [$clog2(READ_OP_TABLE_SIZE)-1:0]
stat_rd_req_finish_tag,
199 output wire [3:0]
stat_rd_req_finish_status,
200 output wire
stat_rd_req_finish_valid,
201 output wire
stat_rd_op_table_full,
202 output wire
stat_rd_tx_stall,
203 output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_op_start_tag,
204 output wire [LEN_WIDTH-1:0]
stat_wr_op_start_len,
205 output wire
stat_wr_op_start_valid,
206 output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_op_finish_tag,
207 output wire [3:0]
stat_wr_op_finish_status,
208 output wire
stat_wr_op_finish_valid,
209 output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_req_start_tag,
210 output wire [12:0]
stat_wr_req_start_len,
211 output wire
stat_wr_req_start_valid,
212 output wire [$clog2(WRITE_OP_TABLE_SIZE)-1:0]
stat_wr_req_finish_tag,
213 output wire [3:0]
stat_wr_req_finish_status,
214 output wire
stat_wr_req_finish_valid,
215 output wire
stat_wr_op_table_full,
216 output wire
stat_wr_tx_stall,
217 );
218
219 dma_if_axi_rd #(
220     .AXI_DATA_WIDTH(AXI_DATA_WIDTH),
221     .AXI_ADDR_WIDTH(AXI_ADDR_WIDTH),
222     .AXI_STRB_WIDTH(AXI_STRB_WIDTH),
223     .AXI_ID_WIDTH(AXI_ID_WIDTH),
224     .AXI_MAX_BURST_LEN(AXI_MAX_BURST_LEN),
225     .RAM_SEL_WIDTH(RAM_SEL_WIDTH),
226     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
227     .RAM_SEG_COUNT(RAM_SEG_COUNT),

```

```

228 .RAM_SEG_DATA_WIDTH(RAM_SEG_DATA_WIDTH),
229 .RAM_SEG_BE_WIDTH(RAM_SEG_BE_WIDTH),
230 .RAM_SEG_ADDR_WIDTH(RAM_SEG_ADDR_WIDTH),
231 .LEN_WIDTH(LEN_WIDTH),
232 .TAG_WIDTH(TAG_WIDTH),
233 .OP_TABLE_SIZE(READ_OP_TABLE_SIZE),
234 .USE_AXI_ID(READ_USE_AXI_ID)
235 )
236 dma_if_axi_rd_inst (
237     .clk(clk),
238     .rst(rst),
239
240     /*
241      * AXI master interface
242      */
243     .m_axi_arid(m_axi_arid),
244     .m_axi_araddr(m_axi_araddr),
245     .m_axi_arlen(m_axi_arlen),
246     .m_axi_arsize(m_axi_arsize),
247     .m_axi_arburst(m_axi_arburst),
248     .m_axi_arlock(m_axi_arlock),
249     .m_axi_arcache(m_axi_arcache),
250     .m_axi_arprot(m_axi_arprot),
251     .m_axi_arvalid(m_axi_arvalid),
252     .m_axi_arready(m_axi_arready),
253     .m_axi_rid(m_axi_rid),
254     .m_axi_rdata(m_axi_rdata),
255     .m_axi_rresp(m_axi_rresp),
256     .m_axi_rlast(m_axi_rlast),
257     .m_axi_rvalid(m_axi_rvalid),
258     .m_axi_rready(m_axi_rready),
259
260     /*
261      * AXI read descriptor input
262      */
263     .s_axis_read_desc_axi_addr(s_axis_read_desc_axi_addr),
264     .s_axis_read_desc_ram_sel(s_axis_read_desc_ram_sel),
265     .s_axis_read_desc_ram_addr(s_axis_read_desc_ram_addr),
266     .s_axis_read_desc_len(s_axis_read_desc_len),
267     .s_axis_read_desc_tag(s_axis_read_desc_tag),
268     .s_axis_read_desc_valid(s_axis_read_desc_valid),
269     .s_axis_read_desc_ready(s_axis_read_desc_ready),
270
271     /*
272      * AXI read descriptor status output
273      */
274     .m_axis_read_desc_status_tag(m_axis_read_desc_status_tag),
275     .m_ax-
276 is_read_desc_status_error(m_axis_read_desc_status_error),
277     .m_ax-
278 is_read_desc_status_valid(m_axis_read_desc_status_valid),
279
280     /*
281      * RAM interface
282      */
283     .ram_wr_cmd_sel(ram_wr_cmd_sel),
284     .ram_wr_cmd_be(ram_wr_cmd_be),
285     .ram_wr_cmd_addr(ram_wr_cmd_addr),
286     .ram_wr_cmd_data(ram_wr_cmd_data),
287     .ram_wr_cmd_valid(ram_wr_cmd_valid),
288     .ram_wr_cmd_ready(ram_wr_cmd_ready),
289     .ram_wr_done(ram_wr_done),
290
291     /*
292      * Configuration
293      */
294     .enable(read_enable),
295
296     /*
297      * Status
298      */
299     .status_busy(status_rd_busy),
300
301     /*
302      * Statistics
303      */
304     .stat_rd_op_start_tag(stat_rd_op_start_tag),
305     .stat_rd_op_start_len(stat_rd_op_start_len),
306     .stat_rd_op_start_valid(stat_rd_op_start_valid),
307     .stat_rd_op_finish_tag(stat_rd_op_finish_tag),
308     .stat_rd_op_finish_status(stat_rd_op_finish_status),
309     .stat_rd_op_finish_valid(stat_rd_op_finish_valid),
310     .stat_rd_req_start_tag(stat_rd_req_start_tag),
311     .stat_rd_req_start_len(stat_rd_req_start_len),
312     .stat_rd_req_start_valid(stat_rd_req_start_valid),
313     .stat_rd_req_finish_tag(stat_rd_req_finish_tag),
314     .stat_rd_req_finish_status(stat_rd_req_finish_status),
315     .stat_rd_req_finish_valid(stat_rd_req_finish_valid),
316     .stat_rd_op_table_full(stat_rd_op_table_full),
317     .stat_rd_tx_stall(stat_rd_tx_stall)
318 );
319 dma_if_axi_wr_inst #(
320     .AXI_DATA_WIDTH(AXI_DATA_WIDTH),
321     .AXI_ADDR_WIDTH(AXI_ADDR_WIDTH),
322     .AXI_STRB_WIDTH(AXI_STRB_WIDTH),
323     .AXI_ID_WIDTH(AXI_ID_WIDTH),
324     .AXI_MAX_BURST_LEN(AXI_MAX_BURST_LEN),
325     .RAM_SEL_WIDTH(RAM_SEL_WIDTH),
326     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
327     .RAM_SEG_COUNT(RAM_SEG_COUNT),

```

```

327 .RAM_SEG_DATA_WIDTH(RAM_SEG_DATA_WIDTH),
328 .RAM_SEG_BE_WIDTH(RAM_SEG_BE_WIDTH),
329 .RAM_SEG_ADDR_WIDTH(RAM_SEG_ADDR_WIDTH),
330 .IMM_ENABLE(IMM_ENABLE),
331 .IMM_WIDTH(IMM_WIDTH),
332 .LEN_WIDTH(LEN_WIDTH),
333 .TAG_WIDTH(TAG_WIDTH),
334 .OP_TABLE_SIZE(WRITE_OP_TABLE_SIZE),
335 .USE_AXI_ID(WRITE_USE_AXI_ID)
336 )
337 dma_if_axi_wr_inst (
338     .clk(clk),
339     .rst(rst),
340
341     /*
342      * AXI master interface
343      */
344     .m_axi_awid(m_axi_awid),
345     .m_axi_awaddr(m_axi_awaddr),
346     .m_axi_awlen(m_axi_awlen),
347     .m_axi_awsz(m_axi_awsz),
348     .m_axi_awburst(m_axi_awburst),
349     .m_axi_awlock(m_axi_awlock),
350     .m_axi_awcache(m_axi_awcache),
351     .m_axi_awprot(m_axi_awprot),
352     .m_axi_awvalid(m_axi_awvalid),
353     .m_axi_awready(m_axi_awready),
354     .m_axi_wdata(m_axi_wdata),
355     .m_axi_wstrb(m_axi_wstrb),
356     .m_axi_wlast(m_axi_wlast),
357     .m_axi_wvalid(m_axi_wvalid),
358     .m_axi_wready(m_axi_wready),
359     .m_axi_bid(m_axi_bid),
360     .m_axi_bresp(m_axi_bresp),
361     .m_axi_bvalid(m_axi_bvalid),
362     .m_axi_bready(m_axi_bready),
363
364     /*
365      * AXI write descriptor input
366      */
367     .s_axis_write_desc_axi_addr(s_axis_write_desc_axi_addr),
368     .s_axis_write_desc_ram_sel(s_axis_write_desc_ram_sel),
369     .s_axis_write_desc_ram_addr(s_axis_write_desc_ram_addr),
370     .s_axis_write_desc_imm(s_axis_write_desc_imm),
371     .s_axis_write_desc_imm_en(s_axis_write_desc_imm_en),
372     .s_axis_write_desc_len(s_axis_write_desc_len),
373     .s_axis_write_desc_tag(s_axis_write_desc_tag),
374     .s_axis_write_desc_valid(s_axis_write_desc_valid),
375     .s_axis_write_desc_ready(s_axis_write_desc_ready),
376
377     /*
378      * AXI write descriptor status output
379      */
380     .m_ax-
381 is_write_desc_status_tag(m_axis_write_desc_status_tag),
382     .m_ax-
383 is_write_desc_status_error(m_axis_write_desc_status_error),
384     .m_ax-
385 is_write_desc_status_valid(m_axis_write_desc_status_valid),
386
387     /*
388      * RAM interface
389      */
390     .ram_rd_cmd_sel(ram_rd_cmd_sel),
391     .ram_rd_cmd_addr(ram_rd_cmd_addr),
392     .ram_rd_cmd_valid(ram_rd_cmd_valid),
393     .ram_rd_cmd_ready(ram_rd_cmd_ready),
394     .ram_rd_resp_data(ram_rd_resp_data),
395     .ram_rd_resp_valid(ram_rd_resp_valid),
396     .ram_rd_resp_ready(ram_rd_resp_ready),
397
398     /*
399      * Configuration
400      */
401     .enable(write_enable),
402
403     /*
404      * Status
405      */
406     .status_busy(status_wr_busy),
407
408     /*
409      * Statistics
410      */
411     .stat_wr_op_start_tag(stat_wr_op_start_tag),
412     .stat_wr_op_start_len(stat_wr_op_start_len),
413     .stat_wr_op_start_valid(stat_wr_op_start_valid),
414     .stat_wr_op_finish_tag(stat_wr_op_finish_tag),
415     .stat_wr_op_finish_status(stat_wr_op_finish_status),
416     .stat_wr_op_finish_valid(stat_wr_op_finish_valid),
417     .stat_wr_req_start_tag(stat_wr_req_start_tag),
418     .stat_wr_req_start_len(stat_wr_req_start_len),
419     .stat_wr_req_start_valid(stat_wr_req_start_valid),
420     .stat_wr_req_finish_tag(stat_wr_req_finish_tag),
421     .stat_wr_req_finish_status(stat_wr_req_finish_status),
422     .stat_wr_req_finish_valid(stat_wr_req_finish_valid),
423     .stat_wr_op_table_full(stat_wr_op_table_full),
424     .stat_wr_tx_stall(stat_wr_tx_stall)
425 );
426 endmodule

```

```

425
426 `resetall

```

6. alexforenrich_verilog-pcie/rtl/dma_if_axi_rd.v (1,132 lines)

```

1 /*
2
3 Copyright (c) 2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI DMA read interface
33 */
34 module dma_if_axi_rd #
35 (
36     // Width of AXI data bus in bits
37     parameter AXI_DATA_WIDTH = 32,
38     // Width of AXI address bus in bits
39     parameter AXI_ADDR_WIDTH = 16,
40     // Width of AXI wstrb (width of data bus in words)
41     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
42     // Width of AXI ID signal
43     parameter AXI_ID_WIDTH = 8,
44     // Maximum AXI burst length to generate
45     parameter AXI_MAX_BURST_LEN = 256,
46     // RAM select width
47     parameter RAM_SEL_WIDTH = 2,
48     // RAM address width
49     parameter RAM_ADDR_WIDTH = 16,
50     // RAM segment count
51     parameter RAM_SEG_COUNT = 2,
52     // RAM segment data width
53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
g2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Length field width
59     parameter LEN_WIDTH = 16,
60     // Tag field width
61     parameter TAG_WIDTH = 8,
62     // Operation table size
63     parameter OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
64     // Use AXI ID signals
65     parameter USE_AXI_ID = 0
66 )
67 (
68     input wire clk,
69     input wire rst,
70
71     /*
72     * AXI master interface
73     */
74     output wire [AXI_ID_WIDTH-1:0] m_axi_arid,
75     output wire [AXI_ADDR_WIDTH-1:0] m_axi_araddr,
76     output wire [7:0] m_axi_arlen,
77     output wire [2:0]

```

```

m_axi_arsize,
78     output wire [1:0] m_axi_arburst,
79     output wire m_axi_arlock,
80     output wire [3:0] m_axi_arcache,
81     output wire [2:0] m_axi_arprot,
82     output wire m_axi_arvalid,
83     input wire m_axi_arready,
84     input wire [AXI_ID_WIDTH-1:0] m_axi_rid,
85     input wire [AXI_DATA_WIDTH-1:0] m_axi_rdata,
86     input wire [1:0] m_axi_rresp,
87     input wire m_axi_rlast,
88     input wire m_axi_rvalid,
89     output wire m_axi_rready,
90
91     /*
92     * AXI read descriptor input
93     */
94     input wire [AXI_ADDR_WIDTH-1:0] is_read_desc_axi_addr,
95     input wire [RAM_SEL_WIDTH-1:0] is_read_desc_ram_sel,
96     input wire [RAM_ADDR_WIDTH-1:0] is_read_desc_ram_addr,
97     input wire [LEN_WIDTH-1:0] is_read_desc_len,
98     input wire [TAG_WIDTH-1:0] is_read_desc_tag,
99     input wire is_read_desc_valid,
100     output wire is_read_desc_ready,
101
102     /*
103     * AXI read descriptor status output
104     */
105     output wire [TAG_WIDTH-1:0] is_read_desc_status_tag,
106     output wire [3:0] is_read_desc_status_error,
107     output wire is_read_desc_status_valid,
108
109     /*
110     * RAM interface
111     */
112     output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_wr_cmd_sel,
113     output wire [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be,
114     output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr,
115     output wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0] ram_wr_cmd_data,
116     output wire [RAM_SEG_COUNT-1:0] ram_wr_cmd_valid,
117     input wire [RAM_SEG_COUNT-1:0] ram_wr_cmd_ready,
118     input wire [RAM_SEG_COUNT-1:0] ram_wr_done,
119
120     /*
121     * Configuration
122     */
123     input wire enable,
124
125     /*
126     * Status
127     */
128     output wire status_busy,
129
130     /*
131     * Statistics
132     */
133     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_rd_op_start_tag,
134     output wire [LEN_WIDTH-1:0] stat_rd_op_start_len,
135     output wire stat_rd_op_start_valid,
136     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_rd_op_finish_tag,
137     output wire [3:0] stat_rd_op_finish_status,
138     output wire stat_rd_op_finish_valid,
139     output wire [$clog2(OP_TABLE_SIZE)-1:0] stat_rd_req_start_tag,

```

```

140 output wire [12:0]
stat_rd_req_start_len,
141 output wire
stat_rd_req_start_valid,
142 output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_rd_req_finish_tag,
143 output wire [3:0]
stat_rd_req_finish_status,
144 output wire
stat_rd_req_finish_valid,
145 output wire
stat_rd_op_table_full,
146 output wire
stat_rd_tx_stall
147 );
148
149 parameter RAM_DATA_WIDTH = RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH;
150 parameter RAM_WORD_WIDTH = RAM_SEG_BE_WIDTH;
151 parameter RAM_WORD_SIZE = RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH;
152
153 parameter AXI_WORD_WIDTH = AXI_STRB_WIDTH;
154 parameter AXI_WORD_SIZE = AXI_DATA_WIDTH/AXI_WORD_WIDTH;
155 parameter AXI_BURST_SIZE = $clog2(AXI_STRB_WIDTH);
156 parameter AXI_MAX_BURST_SIZE = AXI_MAX_BURST_LEN <<
AXI_BURST_SIZE;
157
158 parameter OFFSET_WIDTH = AXI_STRB_WIDTH > 1 ? $clog2(AXI_STRB_WIDTH) : 1;
159 parameter OFFSET_MASK = AXI_STRB_WIDTH > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
160 parameter RAM_OFFSET_WIDTH = $clog2(
RAM_SEG_COUNT*RAM_SEG_BE_WIDTH);
161 parameter ADDR_MASK = {AXI_ADDR_WIDTH{1'b1}} << $clog2(AXI_STRB_WIDTH);
162 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - AXI_BURST_SIZE + 1;
163
164 parameter OP_TAG_WIDTH = $clog2(OP_TABLE_SIZE);
165 parameter OP_TABLE_READ_COUNT_WIDTH = AXI_ID_WIDTH+1;
166 parameter OP_TABLE_WRITE_COUNT_WIDTH = LEN_WIDTH;
167
168 parameter STATUS_FIFO_ADDR_WIDTH = 5;
169 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
170
171 // bus width assertions
172 initial begin
173 if (AXI_WORD_SIZE * AXI_STRB_WIDTH != AXI_DATA_WIDTH) be-
gin
174 $error("Error: AXI data width not evenly divisble (in-
stance %m)");
175 $finish;
176 end
177
178 if (AXI_WORD_SIZE != RAM_WORD_SIZE) begin
179 $error("Error: word size mismatch (instance %m)");
180 $finish;
181 end
182
183 if (2**$clog2(AXI_WORD_WIDTH) != AXI_WORD_WIDTH) begin
184 $error("Error: AXI word width must be even power of
two (instance %m)");
185 $finish;
186 end
187
188 if (AXI_MAX_BURST_LEN < 1 || AXI_MAX_BURST_LEN > 256) be-
gin
189 $error("Error: AXI_MAX_BURST_LEN must be between 1 and
256 (instance %m)");
190 $finish;
191 end
192
193 if (RAM_SEG_COUNT < 2) begin
194 $error("Error: RAM interface requires at least 2 seg-
ments (instance %m)");
195 $finish;
196 end
197
198 if (RAM_DATA_WIDTH != AXI_DATA_WIDTH*2) begin
199 $error("Error: RAM interface width must be double the
AXI interface width (instance %m)");
200 $finish;
201 end
202
203 if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
204 $error("Error: RAM word width must be even power of
two (instance %m)");
205 $finish;
206 end
207
208 if (RAM_ADDR_WIDTH != RAM_SEG_ADDR_WIDTH+$clog2(
RAM_SEG_COUNT+$clog2(RAM_SEG_BE_WIDTH))) begin
209 $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
210 $finish;
211 end
212
213 if (OP_TABLE_SIZE > 2**AXI_ID_WIDTH) begin
214 $error("Error: AXI_ID_WIDTH insufficient for requested
OP_TABLE_SIZE (instance %m)");
215 $finish;

```

```

216 end
217 end
218
219 localparam [1:0]
220 AXI_RESP_OKAY = 2'b00,
221 AXI_RESP_EXOKAY = 2'b01,
222 AXI_RESP_SLVERR = 2'b10,
223 AXI_RESP_DECERR = 2'b11;
224
225 localparam [3:0]
226 DMA_ERROR_NONE = 4'd0,
227 DMA_ERROR_TIMEOUT = 4'd1,
228 DMA_ERROR_PARITY = 4'd2,
229 DMA_ERROR_AXI_RD_SLVERR = 4'd4,
230 DMA_ERROR_AXI_RD_DECERR = 4'd5,
231 DMA_ERROR_AXI_WR_SLVERR = 4'd6,
232 DMA_ERROR_AXI_WR_DECERR = 4'd7,
233 DMA_ERROR_PCIE_FLR = 4'd8,
234 DMA_ERROR_PCIE_CPL_POISONED = 4'd9,
235 DMA_ERROR_PCIE_CPL_STATUS_UR = 4'd10,
236 DMA_ERROR_PCIE_CPL_STATUS_CA = 4'd11;
237
238 localparam [0:0]
239 REQ_STATE_IDLE = 1'd0,
240 REQ_STATE_START = 1'd1;
241
242 reg [0:0] req_state_reg = REQ_STATE_IDLE, req_state_next;
243
244 localparam [0:0]
245 AXI_STATE_IDLE = 1'd0,
246 AXI_STATE_WRITE = 1'd1;
247
248 reg [0:0] axi_state_reg = AXI_STATE_IDLE, axi_state_next;
249
250 reg [AXI_ADDR_WIDTH-1:0] req_axi_addr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, req_axi_addr_next;
251 reg [RAM_SEL_WIDTH-1:0] req_ram_sel_reg =
{RAM_SEL_WIDTH{1'b0}}, req_ram_sel_next;
252 reg [RAM_ADDR_WIDTH-1:0] req_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, req_ram_addr_next;
253 reg [LEN_WIDTH-1:0] req_op_count_reg = {LEN_WIDTH{1'b0}},
req_op_count_next;
254 reg [LEN_WIDTH-1:0] req_tr_count_reg = {LEN_WIDTH{1'b0}},
req_tr_count_next;
255 reg req_zero_len_reg = 1'b0, req_zero_len_next;
256 reg [TAG_WIDTH-1:0] req_tag_reg = {TAG_WIDTH{1'b0}},
req_tag_next;
257
258 reg [RAM_SEL_WIDTH-1:0] ram_sel_reg = {RAM_SEL_WIDTH{1'b0}},
ram_sel_next;
259 reg [RAM_ADDR_WIDTH-1:0] addr_reg = {RAM_ADDR_WIDTH{1'b0}},
addr_next;
260 reg [RAM_ADDR_WIDTH-1:0] addr_delay_reg = {RAM_AD-
DR_WIDTH{1'b0}}, addr_delay_next;
261 reg [12:0] op_count_reg = 13'd0, op_count_next;
262 reg zero_len_reg = 1'b0, zero_len_next;
263 reg [RAM_SEG_COUNT-1:0] ram_mask_reg = {RAM_SEG_COUNT{1'b0}},
ram_mask_next;
264 reg [RAM_SEG_COUNT-1:0] ram_mask_0_reg =
{RAM_SEG_COUNT{1'b0}}, ram_mask_0_next;
265 reg [RAM_SEG_COUNT-1:0] ram_mask_1_reg =
{RAM_SEG_COUNT{1'b0}}, ram_mask_1_next;
266 reg ram_wrap_reg = 1'b0, ram_wrap_next;
267 reg [OFFSET_WIDTH+1-1:0] cycle_byte_count_reg = {OFF-
SET_WIDTH+1{1'b0}}, cycle_byte_count_next;
268 reg [RAM_OFFSET_WIDTH-1:0] start_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, start_offset_next;
269 reg [RAM_OFFSET_WIDTH-1:0] end_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, end_offset_next;
270 reg [OFFSET_WIDTH-1:0] offset_reg = {OFFSET_WIDTH{1'b0}}, off-
set_next;
271 reg [OP_TAG_WIDTH-1:0] op_tag_reg = {OP_TAG_WIDTH{1'b0}},
op_tag_next;
272
273 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_wr_ptr_reg = 0;
274 reg [STATUS_FIFO_ADDR_WIDTH+1-1:0] status_fifo_rd_ptr_reg = 0,
status_fifo_rd_ptr_next;
275 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
276 reg [OP_TAG_WIDTH-1:0]
status_fifo_op_tag[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
277 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
278 reg [RAM_SEG_COUNT-1:0]
status_fifo_mask[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
279 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
280 reg status_fifo_finish[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
281 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
282 reg [3:0] status_fifo_error[(2**STATUS_FIFO_ADDR_WIDTH)-1:0];
283 reg [OP_TAG_WIDTH-1:0] status_fifo_wr_op_tag;
284 reg [RAM_SEG_COUNT-1:0] status_fifo_wr_mask;
285 reg status_fifo_wr_finish;
286 reg [3:0] status_fifo_wr_error;
287 reg status_fifo_we;
288 reg status_fifo_mask_reg = 1'b0, status_fifo_mask_next;
289 reg status_fifo_finish_reg = 1'b0, status_fifo_finish_next;
290 reg [3:0] status_fifo_error_reg = 4'd0,
status_fifo_error_next;

```



```

291 reg status_fifo_we_reg = 1'b0, status_fifo_we_next;
292 reg status_fifo_half_full_reg = 1'b0;
293 reg [OP_TAG_WIDTH-1:0] status_fifo_rd_op_tag_reg = 0,
status_fifo_rd_op_tag_next;
294 reg [RAM_SEG_COUNT-1:0] status_fifo_rd_mask_reg = 0,
status_fifo_rd_mask_next;
295 reg status_fifo_rd_finish_reg = 1'b0,
status_fifo_rd_finish_next;
296 reg [3:0] status_fifo_rd_error_reg = 4'd0,
status_fifo_rd_error_next;
297 reg status_fifo_rd_valid_reg = 1'b0,
status_fifo_rd_valid_next;
298
299 reg [OP_TAG_WIDTH+1-1:0] active_op_count_reg = 0;
300 reg inc_active_op;
301 reg dec_active_op;
302
303 reg [AXI_DATA_WIDTH-1:0] m_axi_rdata_int_reg =
{AXI_DATA_WIDTH{1'b0}}, m_axi_rdata_int_next;
304 reg m_axi_rvalid_int_reg = 1'b0, m_axi_rvalid_int_next;
305
306 reg [AXI_ID_WIDTH-1:0] m_axi_arid_reg = {AXI_ID_WIDTH{1'b0}},
m_axi_arid_next;
307 reg [AXI_ADDR_WIDTH-1:0] m_axi_araddr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, m_axi_araddr_next;
308 reg [7:0] m_axi_arlen_reg = 8'd0, m_axi_arlen_next;
309 reg m_axi_arvalid_reg = 1'b0, m_axi_arvalid_next;
310 reg m_axi_rready_reg = 1'b0, m_axi_rready_next;
311
312 reg s_axis_read_desc_ready_reg = 1'b0, s_ax-
is_read_desc_ready_next;
313
314 reg [TAG_WIDTH-1:0] m_axis_read_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_read_desc_status_tag_next;
315 reg [3:0] m_axis_read_desc_status_error_reg = 4'd0, m_ax-
is_read_desc_status_error_next;
316 reg m_axis_read_desc_status_valid_reg = 1'b0, m_ax-
is_read_desc_status_valid_next;
317
318 reg status_busy_reg = 1'b0;
319
320 reg [OP_TAG_WIDTH-1:0] stat_rd_op_start_tag_reg = 0,
stat_rd_op_start_tag_next;
321 reg [LEN_WIDTH-1:0] stat_rd_op_start_len_reg = 0,
stat_rd_op_start_len_next;
322 reg stat_rd_op_start_valid_reg = 1'b0,
stat_rd_op_start_valid_next;
323 reg [OP_TAG_WIDTH-1:0] stat_rd_op_finish_tag_reg = 0,
stat_rd_op_finish_tag_next;
324 reg [3:0] stat_rd_op_finish_status_reg = 4'd0, stat_rd_op_fin-
ish_status_next;
325 reg stat_rd_op_finish_valid_reg = 1'b0, stat_rd_op_fin-
ish_valid_next;
326 reg [OP_TAG_WIDTH-1:0] stat_rd_req_start_tag_reg = 0,
stat_rd_req_start_tag_next;
327 reg [12:0] stat_rd_req_start_len_reg = 13'd0,
stat_rd_req_start_len_next;
328 reg stat_rd_req_start_valid_reg = 1'b0,
stat_rd_req_start_valid_next;
329 reg [OP_TAG_WIDTH-1:0] stat_rd_req_finish_tag_reg = 0,
stat_rd_req_finish_tag_next;
330 reg [3:0] stat_rd_req_finish_status_reg = 4'd0,
stat_rd_req_finish_status_next;
331 reg stat_rd_req_finish_valid_reg = 1'b0, stat_rd_req_fin-
ish_valid_next;
332 reg stat_rd_op_table_full_reg = 1'b0, stat_rd_op_t-
able_full_next;
333 reg stat_rd_tx_stall_reg = 1'b0, stat_rd_tx_stall_next;
334
335 // internal datapath
336 reg [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_wr_cmd_sel_int;
337 reg [RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be_int;
338 reg [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_ad-
dr_int;
339 reg [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data_int;
340 reg [RAM_SEG_COUNT-1:0] ram_wr_cmd_val-
id_int;
341 wire [RAM_SEG_COUNT-1:0]
ram_wr_cmd_ready_int;
342
343 wire [RAM_SEG_COUNT-1:0] out_done;
344 reg [RAM_SEG_COUNT-1:0] out_done_ack;
345
346 assign m_axi_arid = USE_AXI_ID ? m_axi_arid_reg :
{AXI_ID_WIDTH{1'b0}};
347 assign m_axi_araddr = m_axi_araddr_reg;
348 assign m_axi_arlen = m_axi_arlen_reg;
349 assign m_axi_arsize = AXI_BURST_SIZE;
350 assign m_axi_arburst = 2'b01;
351 assign m_axi_arlock = 1'b0;
352 assign m_axi_arcache = 4'b0011;
353 assign m_axi_arprot = 3'b010;
354 assign m_axi_arvalid = m_axi_arvalid_reg;
355 assign m_axi_rready = m_axi_rready_reg;
356
357 assign s_axis_read_desc_ready = s_axis_read_desc_ready_reg;
358
359 assign m_axis_read_desc_status_tag = m_ax-
is_read_desc_status_tag_reg;

```

```

360 assign m_axis_read_desc_status_error = m_ax-
is_read_desc_status_error_reg;
361 assign m_axis_read_desc_status_valid = m_ax-
is_read_desc_status_valid_reg;
362
363 assign status_busy = status_busy_reg;
364
365 assign stat_rd_op_start_tag = stat_rd_op_start_tag_reg;
366 assign stat_rd_op_start_len = stat_rd_op_start_len_reg;
367 assign stat_rd_op_start_valid = stat_rd_op_start_valid_reg;
368 assign stat_rd_op_finish_tag = stat_rd_op_finish_tag_reg;
369 assign stat_rd_op_finish_status = stat_rd_op_fin-
ish_status_reg;
370 assign stat_rd_op_finish_valid = stat_rd_op_finish_valid_reg;
371 assign stat_rd_req_start_tag = stat_rd_req_start_tag_reg;
372 assign stat_rd_req_start_len = stat_rd_req_start_len_reg;
373 assign stat_rd_req_start_valid = stat_rd_req_start_valid_reg;
374 assign stat_rd_req_finish_tag = stat_rd_req_finish_tag_reg;
375 assign stat_rd_req_finish_status = stat_rd_req_fin-
ish_status_reg;
376 assign stat_rd_req_finish_valid = stat_rd_req_fin-
ish_valid_reg;
377 assign stat_rd_op_table_full = stat_rd_op_table_full_reg;
378 assign stat_rd_tx_stall = stat_rd_tx_stall_reg;
379
380 // operation tag management
381 reg [OP_TAG_WIDTH+1-1:0] op_table_start_ptr_reg = 0;
382 reg [AXI_ADDR_WIDTH-1:0] op_table_start_axi_addr;
383 reg [RAM_SEL_WIDTH-1:0] op_table_start_ram_sel;
384 reg [RAM_ADDR_WIDTH-1:0] op_table_start_ram_addr;
385 reg [11:0] op_table_start_len;
386 reg op_table_start_zero_len;
387 reg [CYCLE_COUNT_WIDTH-1:0] op_table_start_cycle_count;
388 reg [TAG_WIDTH-1:0] op_table_start_tag;
389 reg op_table_start_last;
390 reg op_table_start_en;
391 reg [OP_TAG_WIDTH+1-1:0] op_table_read_complete_ptr_reg = 0;
392 reg op_table_read_complete_en;
393 reg [OP_TAG_WIDTH-1:0] op_table_update_status_ptr;
394 reg [3:0] op_table_update_status_error;
395 reg op_table_update_status_en;
396 reg [OP_TAG_WIDTH-1:0] op_table_write_complete_ptr;
397 reg op_table_write_complete_en;
398 reg [OP_TAG_WIDTH+1-1:0] op_table_finish_ptr_reg = 0;
399 reg op_table_finish_en;
400
401 reg [2**OP_TAG_WIDTH-1:0] op_table_active = 0;
402 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
403 reg [AXI_ADDR_WIDTH-1:0] op_table_axi_addr
[2**OP_TAG_WIDTH-1:0];
404 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
405 reg [RAM_SEL_WIDTH-1:0] op_table_ram_sel
[2**OP_TAG_WIDTH-1:0];
406 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
407 reg [RAM_ADDR_WIDTH-1:0] op_table_ram_addr
[2**OP_TAG_WIDTH-1:0];
408 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
409 reg [11:0] op_table_len[2**OP_TAG_WIDTH-1:0];
410 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
411 reg op_table_zero_len[2**OP_TAG_WIDTH-1:0];
412 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
413 reg [CYCLE_COUNT_WIDTH-1:0] op_t-
able_cycle_count[2**OP_TAG_WIDTH-1:0];
414 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
415 reg [TAG_WIDTH-1:0] op_table_tag[2**OP_TAG_WIDTH-1:0];
416 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
417 reg op_table_last[2**OP_TAG_WIDTH-1:0];
418 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
419 reg op_table_write_complete[2**OP_TAG_WIDTH-1:0];
420 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
421 reg op_table_error_a [2**OP_TAG_WIDTH-1:0];
422 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
423 reg op_table_error_b [2**OP_TAG_WIDTH-1:0];
424 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
425 reg [3:0] op_table_error_code [2**OP_TAG_WIDTH-1:0];
426
427 integer i;
428
429 initial begin
430   for (i = 0; i < 2**OP_TAG_WIDTH; i = i + 1) begin
431     op_table_axi_addr[i] = 0;
432     op_table_ram_sel[i] = 0;
433     op_table_ram_addr[i] = 0;
434     op_table_len[i] = 0;
435     op_table_zero_len[i] = 1'b0;
436     op_table_cycle_count[i] = 0;
437     op_table_tag[i] = 0;
438     op_table_last[i] = 0;
439     op_table_write_complete[i] = 0;

```

```

440     op_table_error_a[i] = 0;
441     op_table_error_b[i] = 0;
442     op_table_error_code[i] = 0;
443 end
444 end
445
446 always @* begin
447     req_state_next = REQ_STATE_IDLE;
448
449     s_axis_read_desc_ready_next = 1'b0;
450
451     stat_rd_op_start_tag_next = stat_rd_op_start_tag_reg;
452     stat_rd_op_start_len_next = stat_rd_op_start_len_reg;
453     stat_rd_op_start_valid_next = 1'b0;
454     stat_rd_req_start_tag_next = stat_rd_req_start_tag_reg;
455     stat_rd_req_start_len_next = stat_rd_req_start_len_reg;
456     stat_rd_req_start_valid_next = 1'b0;
457     stat_rd_op_table_full_next = !(op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH));
458     stat_rd_tx_stall_next = m_axi_arvalid_reg && !m_axi_ar-
ready;
459
460     req_axi_addr_next = req_axi_addr_reg;
461     req_ram_sel_next = req_ram_sel_reg;
462     req_ram_addr_next = req_ram_addr_reg;
463     req_op_count_next = req_op_count_reg;
464     req_tr_count_next = req_tr_count_reg;
465     req_zero_len_next = req_zero_len_reg;
466     req_tag_next = req_tag_reg;
467
468     m_axi_arid_next = m_axi_arid_reg;
469     m_axi_araddr_next = m_axi_araddr_reg;
470     m_axi_arlen_next = m_axi_arlen_reg;
471     m_axi_arvalid_next = m_axi_arvalid_reg && !m_axi_arready;
472
473     op_table_start_axi_addr = req_axi_addr_reg;
474     op_table_start_ram_sel = req_ram_sel_reg;
475     op_table_start_ram_addr = req_ram_addr_reg;
476     op_table_start_len = 0;
477     op_table_start_zero_len = req_zero_len_reg;
478     op_table_start_tag = req_tag_reg;
479     op_table_start_cycle_count = 0;
480     op_table_start_last = 0;
481     op_table_start_en = 1'b0;
482
483     inc_active_op = 1'b0;
484
485     // segmentation and request generation
486     case (req_state_reg)
487         REQ_STATE_IDLE: begin
488             s_axis_read_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && enable;
489
490             req_axi_addr_next = s_axis_read_desc_axi_addr;
491             req_ram_sel_next = s_axis_read_desc_ram_sel;
492             req_ram_addr_next = s_axis_read_desc_ram_addr;
493             if (s_axis_read_desc_len == 0) begin
494                 // zero-length operation
495                 req_op_count_next = 1;
496                 req_zero_len_next = 1'b1;
497             end else begin
498                 req_op_count_next = s_axis_read_desc_len;
499                 req_zero_len_next = 1'b0;
500             end
501             req_tag_next = s_axis_read_desc_tag;
502
503             if (req_op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
504                 // packet smaller than max burst size
505                 if (((req_axi_addr_next & 12'hfff) +
(req_op_count_next & 12'hfff)) >> 12 != 0 || req_op_count_next >>
12 != 0) begin
506                     // crosses 4k boundary
507                     req_tr_count_next = 13'h1000 - req_axi_ad-
dr_next[11:0];
508                 end else begin
509                     // does not cross 4k boundary
510                     req_tr_count_next = req_op_count_next;
511                 end
512             end else begin
513                 // packet larger than max burst size
514                 if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
515                     // crosses 4k boundary
516                     req_tr_count_next = 13'h1000 - req_axi_ad-
dr_next[11:0];
517                 end else begin
518                     // does not cross 4k boundary
519                     req_tr_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
520                 end
521             end
522
523             if (s_axis_read_desc_ready && s_ax-
is_read_desc_valid) begin
524                 s_axis_read_desc_ready_next = 1'b0;
525
526                 stat_rd_op_start_tag_next =
stat_rd_op_start_tag_reg+1;
527                 stat_rd_op_start_len_next = s_ax-
is_read_desc_len;
528                 stat_rd_op_start_valid_next = 1'b1;
529
530                 req_state_next = REQ_STATE_START;
531             end else begin
532                 req_state_next = REQ_STATE_IDLE;
533             end
534         end
535     end
536     REQ_STATE_START: begin
537         if (!op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && (!m_axi_arvalid || m_axi_arready)) begin
538             req_axi_addr_next = req_axi_addr_reg +
req_tr_count_reg;
539             req_ram_addr_next = req_ram_addr_reg +
req_tr_count_reg;
540             req_op_count_next = req_op_count_reg -
req_tr_count_reg;
541
542             op_table_start_axi_addr = req_axi_addr_reg;
543             op_table_start_ram_sel = req_ram_sel_reg;
544             op_table_start_ram_addr = req_ram_addr_reg;
545             op_table_start_len = req_tr_count_next;
546             op_table_start_zero_len = req_zero_len_reg;
547             op_table_start_tag = req_tag_reg;
548             op_table_start_cycle_count =
(req_tr_count_next + (req_axi_addr_reg & OFFSET_MASK) - 1) >>
AXI_BURST_SIZE;
549             op_table_start_last = req_op_count_reg ==
req_tr_count_next;
550             op_table_start_en = 1'b1;
551             inc_active_op = 1'b1;
552
553             stat_rd_req_start_tag_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
554             stat_rd_req_start_len_next =
req_zero_len_reg ? 0 : req_tr_count_reg;
555             stat_rd_req_start_valid_next = 1'b1;
556
557             m_axi_arid_next = op_t-
able_start_ptr_reg[OP_TAG_WIDTH-1:0];
558             m_axi_araddr_next = req_axi_addr_reg;
559             m_axi_arlen_next = op_table_start_cycle_count;
560             m_axi_arvalid_next = 1'b1;
561
562             if (req_op_count_next <= AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
gin
563                 // packet smaller than max burst size
564                 if (((req_axi_addr_next & 12'hfff) +
(req_op_count_next & 12'hfff)) >> 12 != 0 || req_op_count_next >>
12 != 0) begin
565                     // crosses 4k boundary
566                     req_tr_count_next = 13'h1000 -
req_axi_addr_next[11:0];
567                 end else begin
568                     // does not cross 4k boundary
569                     req_tr_count_next = req_op_count_next;
570                 end
571             end else begin
572                 // packet larger than max burst size
573                 if (((req_axi_addr_next & 12'hfff) +
AXI_MAX_BURST_SIZE) >> 12 != 0) begin
574                     // crosses 4k boundary
575                     req_tr_count_next = 13'h1000 -
req_axi_addr_next[11:0];
576                 end else begin
577                     // does not cross 4k boundary
578                     req_tr_count_next = AXI_MAX_BURST_SIZE -
(req_axi_addr_next & OFFSET_MASK);
579                 end
580             end
581
582             if (!op_table_start_last) begin
583                 req_state_next = REQ_STATE_START;
584             end else begin
585                 s_axis_read_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && enable;
586                 req_state_next = REQ_STATE_IDLE;
587             end
588         end
589     end
590 endcase
591 end
592
593 always @* begin
594     axi_state_next = AXI_STATE_IDLE;
595
596     m_axi_rready_next = 1'b0;
597
598     stat_rd_op_finish_tag_next = stat_rd_op_finish_tag_reg;

```



```

600 stat_rd_op_finish_status_next = stat_rd_op_fin-
ish_status_reg;
601 stat_rd_op_finish_valid_next = 1'b0;
602 stat_rd_req_finish_tag_next = stat_rd_req_finish_tag_reg;
603 stat_rd_req_finish_status_next = stat_rd_req_fin-
ish_status_reg;
604 stat_rd_req_finish_valid_next = 1'b0;
605
606 ram_sel_next = ram_sel_reg;
607 addr_next = addr_reg;
608 addr_delay_next = addr_delay_reg;
609 op_count_next = op_count_reg;
610 zero_len_next = zero_len_reg;
611 ram_mask_next = ram_mask_reg;
612 ram_mask_0_next = ram_mask_0_reg;
613 ram_mask_1_next = ram_mask_1_reg;
614 ram_wrap_next = ram_wrap_reg;
615 cycle_byte_count_next = cycle_byte_count_reg;
616 start_offset_next = start_offset_reg;
617 end_offset_next = end_offset_reg;
618 offset_next = offset_reg;
619 op_tag_next = op_tag_reg;
620
621 op_table_read_complete_en = 1'b0;
622
623 m_axi_rdata_int_next = m_axi_rdata_int_reg;
624 m_axi_rvalid_int_next = 1'b0;
625
626 status_fifo_mask_next = 1'b1;
627 status_fifo_finish_next = 1'b0;
628 status_fifo_error_next = DMA_ERROR_NONE;
629 status_fifo_we_next = 1'b0;
630
631 out_done_ack = {RAM_SEG_COUNT{1'b0}};
632
633 // Write generation
634 ram_wr_cmd_sel_int = {RAM_SEG_COUNT{ram_sel_reg}};
635 if (!ram_wrap_reg) begin
636     ram_wr_cmd_be_int =
637     ({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} << start_offset_reg) &
638     ({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} >>
639     (RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1-end_offset_reg));
640 end else begin
641     ram_wr_cmd_be_int =
642     ({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} << start_offset_reg) |
643     ({RAM_SEG_COUNT*RAM_SEG_BE_WIDTH{1'b1}} >>
644     (RAM_SEG_COUNT*RAM_SEG_BE_WIDTH-1-end_offset_reg));
645 end
646 for (i = 0; i < RAM_SEG_COUNT; i = i + 1) begin
647     ram_wr_cmd_addr_int[i*RAM_SEG_ADDR_WIDTH +:
648     RAM_SEG_ADDR_WIDTH] =
649     addr_delay_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH];
650     if (ram_mask_1_reg[i]) begin
651         ram_wr_cmd_addr_int[i*RAM_SEG_ADDR_WIDTH +:
652         RAM_SEG_ADDR_WIDTH] =
653         addr_delay_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH]
654         +1;
655     end
656 end
657 ram_wr_cmd_data_int = {3{m_axi_rdata_int_reg}} >>
658 (AXI_DATA_WIDTH - offset_reg*AXI_WORD_SIZE);
659 ram_wr_cmd_valid_int = {RAM_SEG_COUNT{1'b0}};
660
661 if (m_axi_rvalid_int_reg) begin
662     ram_wr_cmd_valid_int = ram_mask_reg;
663 end
664
665 // AXI read response handling
666 case (axi_state_reg)
667     AXI_STATE_IDLE: begin
668         // idle state, wait for read data
669         m_axi_rready_next = &ram_wr_cmd_ready_int && !
670         status_fifo_half_full_reg;
671     end
672     if (USE_AXI_ID) begin
673         op_tag_next = m_axi_rid[OP_TAG_WIDTH-1:0];
674     end else begin
675         op_tag_next = op_table_read_complete_ptr_reg;
676     end
677     ram_sel_next = op_table_ram_sel[op_tag_next];
678     addr_next = op_table_ram_addr[op_tag_next];
679     op_count_next = op_table_len[op_tag_next];
680     zero_len_next = op_table_zero_len[op_tag_next];
681     offset_next = op_table_ram_addr[op_tag_next]
682     [RAM_OFFSET_WIDTH-1:0]-(op_table_axi_addr[op_tag_next] & OFF-
683     SET_MASK);
684     if (m_axi_rready && m_axi_rvalid) begin
685         if (op_count_next > AXI_WORD_WIDTH-(op_t-
686         able_axi_addr[op_tag_next] & OFFSET_MASK)) begin
687             cycle_byte_count_next = AXI_WORD_WIDTH-
688             (op_table_axi_addr[op_tag_next] & OFFSET_MASK);
689         end else begin
690             cycle_byte_count_next = op_count_next;
691         end
692         start_offset_next = addr_next;
693         {ram_wrap_next, end_offset_next} =
694         start_offset_next+cycle_byte_count_next-1;
695         ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
696         (start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));

```

```

680     ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
681     (RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
682     if (!ram_wrap_next) begin
683         ram_mask_next = ram_mask_0_next &
684         ram_mask_1_next;
685         ram_mask_0_next = ram_mask_0_next &
686         ram_mask_1_next;
687         ram_mask_1_next = 0;
688     end else begin
689         ram_mask_next = ram_mask_0_next |
690         ram_mask_1_next;
691     end
692     addr_delay_next = addr_next;
693     addr_next = addr_next + cycle_byte_count_next;
694     op_count_next = op_count_next -
695     cycle_byte_count_next;
696
697     m_axi_rdata_int_next = m_axi_rdata;
698     m_axi_rvalid_int_next = 1'b1;
699
700     status_fifo_mask_next = 1'b1;
701     status_fifo_finish_next = 1'b0;
702     status_fifo_error_next = DMA_ERROR_NONE;
703     status_fifo_we_next = 1'b1;
704
705     if (zero_len_next) begin
706         m_axi_rvalid_int_next = 1'b0;
707         status_fifo_mask_next = 1'b0;
708     end
709
710     if (m_axi_rresp == AXI_RESP_SLVERR) begin
711         m_axi_rvalid_int_next = 1'b0;
712         status_fifo_mask_next = 1'b0;
713         status_fifo_error_next = DMA_ER-
714         R_AXI_RD_SLVERR;
715     end else if (m_axi_rresp == AXI_RESP_DECERR)
716     begin
717         m_axi_rvalid_int_next = 1'b0;
718         status_fifo_mask_next = 1'b0;
719         status_fifo_error_next = DMA_ER-
720         R_AXI_RD_DECERR;
721     end
722
723     stat_rd_req_finish_tag_next = op_tag_next;
724     stat_rd_req_finish_status_next =
725     status_fifo_error_next;
726     stat_rd_req_finish_valid_next = 1'b0;
727
728     if (!USE_AXI_ID) begin
729         op_table_read_complete_en = 1'b1;
730     end
731
732     if (m_axi_rlast) begin
733         status_fifo_finish_next = 1'b1;
734         stat_rd_req_finish_valid_next = 1'b1;
735         axi_state_next = AXI_STATE_IDLE;
736     end else begin
737         axi_state_next = AXI_STATE_WRITE;
738     end
739     end else begin
740         axi_state_next = AXI_STATE_IDLE;
741     end
742     end
743     AXI_STATE_WRITE: begin
744         // write state - generate write operations
745         m_axi_rready_next = &ram_wr_cmd_ready_int && !
746         status_fifo_half_full_reg;
747         if (m_axi_rready && m_axi_rvalid) begin
748             if (op_count_next > AXI_WORD_WIDTH) begin
749                 cycle_byte_count_next = AXI_WORD_WIDTH;
750             end else begin
751                 cycle_byte_count_next = op_count_next;
752             end
753             start_offset_next = addr_next;
754             {ram_wrap_next, end_offset_next} =
755             start_offset_next+cycle_byte_count_next-1;
756             ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
757             (start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
758             ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
759             (RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
760             if (!ram_wrap_next) begin
761                 ram_mask_next = ram_mask_0_next &
762                 ram_mask_1_next;
763                 ram_mask_0_next = ram_mask_0_next &
764                 ram_mask_1_next;
765                 ram_mask_1_next = 0;
766             end else begin
767                 ram_mask_next = ram_mask_0_next |
768                 ram_mask_1_next;
769             end
770             addr_delay_next = addr_next;
771             addr_next = addr_next + cycle_byte_count_next;
772             op_count_next = op_count_next -
773             cycle_byte_count_next;

```

```

764
765     m_axi_rdata_int_next = m_axi_rdata;
766     m_axi_rvalid_int_next = 1'b1;
767
768     status_fifo_mask_next = 1'b1;
769     status_fifo_finish_next = 1'b0;
770     status_fifo_error_next = DMA_ERROR_NONE;
771     status_fifo_we_next = 1'b1;
772
773     if (m_axi_rresp == AXI_RESP_SLVERR) begin
774         m_axi_rvalid_int_next = 1'b0;
775         status_fifo_mask_next = 1'b0;
776         status_fifo_error_next = DMA_ER-
ROR_AXI_RD_SLVERR;
777     end else if (m_axi_rresp == AXI_RESP_DECERR)
begin
778         m_axi_rvalid_int_next = 1'b0;
779         status_fifo_mask_next = 1'b0;
780         status_fifo_error_next = DMA_ER-
ROR_AXI_RD_DECERR;
781     end
782
783     stat_rd_req_finish_tag_next = op_tag_next;
784     stat_rd_req_finish_status_next =
status_fifo_error_next;
785     stat_rd_req_finish_valid_next = 1'b0;
786
787     if (m_axi_rlast) begin
788         status_fifo_finish_next = 1'b1;
789         stat_rd_req_finish_valid_next = 1'b1;
790         axi_state_next = AXI_STATE_IDLE;
791     end else begin
792         axi_state_next = AXI_STATE_WRITE;
793     end
794     end else begin
795         axi_state_next = AXI_STATE_WRITE;
796     end
797 end
798 endcase
799
800 status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg;
801
802 status_fifo_wr_op_tag = op_tag_reg;
803 status_fifo_wr_mask = status_fifo_mask_reg ?
ram_mask_reg : 0;
804 status_fifo_wr_finish = status_fifo_finish_reg;
805 status_fifo_wr_error = status_fifo_error_reg;
806 status_fifo_we = 1'b0;
807
808 if (status_fifo_we_reg) begin
809     status_fifo_wr_op_tag = op_tag_reg;
810     status_fifo_wr_mask = status_fifo_mask_reg ?
ram_mask_reg : 0;
811     status_fifo_wr_finish = status_fifo_finish_reg;
812     status_fifo_wr_error = status_fifo_error_reg;
813     status_fifo_we = 1'b1;
814 end
815
816 status_fifo_rd_op_tag_next = status_fifo_rd_op_tag_reg;
817 status_fifo_rd_mask_next = status_fifo_rd_mask_reg;
818 status_fifo_rd_finish_next = status_fifo_rd_finish_reg;
819 status_fifo_rd_valid_next = status_fifo_rd_valid_reg;
820 status_fifo_rd_error_next = status_fifo_rd_error_reg;
821
822 op_table_update_status_ptr = status_fifo_rd_op_tag_reg;
823 op_table_update_status_error = status_fifo_rd_error_reg;
824 op_table_update_status_en = 1'b0;
825
826 op_table_write_complete_ptr = status_fifo_rd_op_tag_reg;
827 op_table_write_complete_en = 1'b0;
828
829 if (status_fifo_rd_valid_reg && (status_fifo_rd_mask_reg &
-out_done) == 0) begin
830     // got write completion, pop and return status
831     status_fifo_rd_valid_next = 1'b0;
832     op_table_update_status_en = 1'b1;
833
834     out_done_ack = status_fifo_rd_mask_reg;
835
836     if (status_fifo_rd_finish_reg) begin
837         // mark done
838         op_table_write_complete_ptr =
status_fifo_rd_op_tag_reg;
839         op_table_write_complete_en = 1'b1;
840     end
841 end
842
843 if (!status_fifo_rd_valid_next && status_fifo_rd_ptr_reg !
= status_fifo_wr_ptr_reg) begin
844     // status FIFO not empty
845     status_fifo_rd_op_tag_next =
status_fifo_op_tag[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
846     status_fifo_rd_mask_next =
status_fifo_mask[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
;
847     status_fifo_rd_finish_next =
status_fifo_finish[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
848     status_fifo_rd_error_next =

```

```

status_fifo_error[status_fifo_rd_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]];
849     status_fifo_rd_valid_next = 1'b1;
850     status_fifo_rd_ptr_next = status_fifo_rd_ptr_reg + 1;
851 end
852
853 // commit operations in-order
854 op_table_finish_en = 1'b0;
855 dec_active_op = 1'b0;
856
857 if (m_axis_read_desc_status_valid_reg) begin
858     m_axis_read_desc_status_error_next = DMA_ERROR_NONE;
859 end else begin
860     m_axis_read_desc_status_error_next = m_ax-
is_read_desc_status_error_reg;
861 end
862
863 m_axis_read_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
864 m_axis_read_desc_status_valid_next = 1'b0;
865
866 stat_rd_op_finish_tag_next = stat_rd_op_finish_tag_reg;
867 stat_rd_op_finish_status_next = m_ax-
is_read_desc_status_error_next;
868 stat_rd_op_finish_valid_next = 1'b0;
869
870 if (op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_write_complete[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] &&
op_table_finish_ptr_reg != op_table_start_ptr_reg) begin
871     op_table_finish_en = 1'b1;
872     dec_active_op = 1'b1;
873
874 if (op_t-
able_error_a[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] != op_t-
able_error_b[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
875     m_axis_read_desc_status_error_next = op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
876 end
877
878 stat_rd_op_finish_status_next = m_ax-
is_read_desc_status_error_next;
879
880 if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
881     m_axis_read_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
882     m_axis_read_desc_status_valid_next = 1'b1;
883     stat_rd_op_finish_tag_next = stat_rd_op_fin-
ish_tag_reg + 1;
884     stat_rd_op_finish_valid_next = 1'b1;
885 end
886 end
887 end
888
889 always @(posedge clk) begin
890     req_state_reg <= req_state_next;
891     axi_state_reg <= axi_state_next;
892
893     req_axi_addr_reg <= req_axi_addr_next;
894     req_ram_sel_reg <= req_ram_sel_next;
895     req_ram_addr_reg <= req_ram_addr_next;
896     req_op_count_reg <= req_op_count_next;
897     req_tr_count_reg <= req_tr_count_next;
898     req_zero_len_reg <= req_zero_len_next;
899     req_tag_reg <= req_tag_next;
900
901     ram_sel_reg <= ram_sel_next;
902     addr_reg <= addr_next;
903     addr_delay_reg <= addr_delay_next;
904     op_count_reg <= op_count_next;
905     zero_len_reg <= zero_len_next;
906     ram_mask_reg <= ram_mask_next;
907     ram_mask_0_reg <= ram_mask_0_next;
908     ram_mask_1_reg <= ram_mask_1_next;
909     ram_wrap_reg <= ram_wrap_next;
910     cycle_byte_count_reg <= cycle_byte_count_next;
911     start_offset_reg <= start_offset_next;
912     end_offset_reg <= end_offset_next;
913     offset_reg <= offset_next;
914     op_tag_reg <= op_tag_next;
915
916     m_axi_rdata_int_reg <= m_axi_rdata_int_next;
917     m_axi_rvalid_int_reg <= m_axi_rvalid_int_next;
918
919     m_axi_arid_reg <= m_axi_arid_next;
920     m_axi_araddr_reg <= m_axi_araddr_next;
921     m_axi_arlen_reg <= m_axi_arlen_next;
922     m_axi_arvalid_reg <= m_axi_arvalid_next;
923     m_axi_rready_reg <= m_axi_rready_next;
924
925     s_axis_read_desc_ready_reg <= s_axis_read_desc_ready_next;
926
927     m_axis_read_desc_status_tag_reg <= m_ax-
is_read_desc_status_tag_next;
928     m_axis_read_desc_status_error_reg <= m_ax-
is_read_desc_status_error_next;
929     m_axis_read_desc_status_valid_reg <= m_ax-
is_read_desc_status_valid_next;
930
931     status_busy_reg <= active_op_count_reg != 0;

```

```

932
933 stat_rd_op_start_tag_reg <= stat_rd_op_start_tag_next;
934 stat_rd_op_start_len_reg <= stat_rd_op_start_len_next;
935 stat_rd_op_start_valid_reg <= stat_rd_op_start_valid_next;
936 stat_rd_op_finish_tag_reg <= stat_rd_op_finish_tag_next;
937 stat_rd_op_finish_status_reg <= stat_rd_op_fin-
ish_status_next;
938 stat_rd_op_finish_valid_reg <= stat_rd_op_fin-
ish_valid_next;
939 stat_rd_req_start_tag_reg <= stat_rd_req_start_tag_next;
940 stat_rd_req_start_len_reg <= stat_rd_req_start_len_next;
941 stat_rd_req_start_valid_reg <=
stat_rd_req_start_valid_next;
942 stat_rd_req_finish_tag_reg <= stat_rd_req_finish_tag_next;
943 stat_rd_req_finish_status_reg <= stat_rd_req_fin-
ish_status_next;
944 stat_rd_req_finish_valid_reg <= stat_rd_req_fin-
ish_valid_next;
945 stat_rd_op_table_full_reg <= stat_rd_op_table_full_next;
946 stat_rd_tx_stall_reg <= stat_rd_tx_stall_next;
947
948 if (status_fifo_we) begin
949
status_fifo_op_tag[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] <= status_fifo_wr_op_tag;
950
status_fifo_mask[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]]
<= status_fifo_wr_mask;
951
status_fifo_finish[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0
]] <= status_fifo_wr_finish;
952
status_fifo_error[status_fifo_wr_ptr_reg[STATUS_FIFO_ADDR_WIDTH-1:0]
] <= status_fifo_wr_error;
953
status_fifo_wr_ptr_reg <= status_fifo_wr_ptr_reg + 1;
954 end
955 status_fifo_rd_ptr_reg <= status_fifo_rd_ptr_next;
956
957 status_fifo_mask_reg <= status_fifo_mask_next;
958 status_fifo_finish_reg <= status_fifo_finish_next;
959 status_fifo_error_reg <= status_fifo_error_next;
960 status_fifo_we_reg <= status_fifo_we_next;
961
962 status_fifo_rd_op_tag_reg <= status_fifo_rd_op_tag_next;
963 status_fifo_rd_mask_reg <= status_fifo_rd_mask_next;
964 status_fifo_rd_finish_reg <= status_fifo_rd_finish_next;
965 status_fifo_rd_error_reg <= status_fifo_rd_error_next;
966 status_fifo_rd_valid_reg <= status_fifo_rd_valid_next;
967
968 status_fifo_half_full_reg <= $un-
signed(status_fifo_wr_ptr_reg - status_fifo_rd_ptr_reg) >=
2*(STATUS_FIFO_ADDR_WIDTH-1);
969
970 active_op_count_reg <= active_op_count_reg + inc_active_op -
dec_active_op;
971
972 if (op_table_start_en) begin
973 op_table_start_ptr_reg <= op_table_start_ptr_reg + 1;
974 op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b1;
975 op_t-
able_axi_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_axi_addr;
976 op_t-
able_ram_sel[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_ram_sel;
977 op_t-
able_ram_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_ram_addr;
978 op_table_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_len;
979 op_t-
able_zero_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_zero_len;
980 op_t-
able_cycle_count[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_cycle_count;
981 op_table_tag[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_tag;
982 op_t-
able_last[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_last;
983 op_t-
able_write_complete[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <=
1'b0;
984 op_t-
able_error_a[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_error_b[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]];
985 end
986
987 if (!USE_AXI_ID && op_table_read_complete_en) begin
988 op_table_read_complete_ptr_reg <= op_t-
able_read_complete_ptr_reg + 1;
989 end
990
991 if (op_table_update_status_en) begin
992 if (op_table_update_status_error != 0) begin

```

```

993 op_table_error_code[op_table_update_status_ptr] <=
op_table_update_status_error;
994 op_table_error_b[op_table_update_status_ptr] <= !
op_table_error_a[op_table_update_status_ptr];
995 end
996 end
997
998 if (op_table_write_complete_en) begin
999 op_table_write_complete[op_table_write_complete_ptr]
<= 1'b1;
1000 end
1001
1002 if (op_table_finish_en) begin
1003 op_table_finish_ptr_reg <= op_table_finish_ptr_reg +
1;
1004 op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b0;
1005 end
1006
1007 if (rst) begin
1008 req_state_reg <= REQ_STATE_IDLE;
1009 axi_state_reg <= AXI_STATE_IDLE;
1010
m_axi_rdata_int_reg <= 1'b0;
1011
m_axi_arvalid_reg <= 1'b0;
1012
m_axi_rready_reg <= 1'b0;
1013
s_axis_read_desc_ready_reg <= 1'b0;
1014
m_axis_read_desc_status_error_reg = 4'd0;
1015
m_axis_read_desc_status_valid_reg <= 1'b0;
1016
status_busy_reg <= 1'b0;
1017
stat_rd_op_start_tag_reg <= 0;
1018
stat_rd_op_start_valid_reg <= 1'b0;
1019
stat_rd_op_finish_tag_reg <= 0;
1020
stat_rd_op_finish_valid_reg <= 1'b0;
1021
stat_rd_req_start_valid_reg <= 1'b0;
1022
stat_rd_req_finish_valid_reg <= 1'b0;
1023
stat_rd_op_table_full_reg <= 1'b0;
1024
stat_rd_tx_stall_reg <= 1'b0;
1025
status_fifo_wr_ptr_reg <= 0;
1026
status_fifo_rd_ptr_reg <= 0;
1027
status_fifo_we_reg <= 1'b0;
1028
status_fifo_rd_valid_reg <= 1'b0;
1029
active_op_count_reg <= 0;
1030
op_table_start_ptr_reg <= 0;
1031
op_table_read_complete_ptr_reg <= 0;
1032
op_table_finish_ptr_reg <= 0;
1033
op_table_active <= 0;
1034 end
1035 end
1036
1037 // output datapath logic (write data)
1038 generate
1039 genvar n;
1040 for (n = 0; n < RAM_SEG_COUNT; n = n + 1) begin
1041
reg [RAM_SEL_WIDTH-1:0] ram_wr_cmd_sel_reg =
{RAM_SEL_WIDTH{1'b0}};
1042
reg [RAM_SEG_BE_WIDTH-1:0] ram_wr_cmd_be_reg =
{RAM_SEG_BE_WIDTH{1'b0}};
1043
reg [RAM_SEG_ADDR_WIDTH-1:0] ram_wr_cmd_addr_reg =
{RAM_SEG_ADDR_WIDTH{1'b0}};
1044
reg [RAM_SEG_DATA_WIDTH-1:0] ram_wr_cmd_data_reg =
{RAM_SEG_DATA_WIDTH{1'b0}};
1045
reg ram_wr_cmd_valid_reg = 1'b0;
1046
reg [OUTPUT_FIFO_ADDR_WIDTH-1:0] out_fifo_wr_ptr_reg = 0;
1047
reg [OUTPUT_FIFO_ADDR_WIDTH-1:0] out_fifo_rd_ptr_reg = 0;
1048
reg out_fifo_half_full_reg = 1'b0;
1049
wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
1050
wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
1051
(* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1052 reg [RAM_SEL_WIDTH-1:0]
out_fifo_wr_cmd_sel[2*OUTPUT_FIFO_ADDR_WIDTH-1:0];
1053 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1054 reg [RAM_SEG_BE_WIDTH-1:0]
out_fifo_wr_cmd_be[2*OUTPUT_FIFO_ADDR_WIDTH-1:0];
1055 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1056 reg [RAM_SEG_ADDR_WIDTH-1:0]
out_fifo_wr_cmd_addr[2*OUTPUT_FIFO_ADDR_WIDTH-1:0];
1057 (* ram_style = "distributed", ramstyle = "no_rw_check,
mlab" *)
1058 reg [RAM_SEG_DATA_WIDTH-1:0]
out_fifo_wr_cmd_data[2*OUTPUT_FIFO_ADDR_WIDTH-1:0];
1059
reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] done_count_reg = 0;
1060

```

```

1075     reg done_reg = 1'b0;
1076
1077     assign ram_wr_cmd_ready_int[n +: 1] = !
out_fifo_half_full_reg;
1078
1079     assign ram_wr_cmd_sel[n*RAM_SEL_WIDTH +: RAM_SEL_WIDTH] =
ram_wr_cmd_sel_reg;
1080     assign ram_wr_cmd_be[n*RAM_SEG_BE_WIDTH +:
RAM_SEG_BE_WIDTH] = ram_wr_cmd_be_reg;
1081     assign ram_wr_cmd_addr[n*RAM_SEG_ADDR_WIDTH +: RAM_SEG_AD-
DR_WIDTH] = ram_wr_cmd_addr_reg;
1082     assign ram_wr_cmd_data[n*RAM_SEG_DATA_WIDTH +:
RAM_SEG_DATA_WIDTH] = ram_wr_cmd_data_reg;
1083     assign ram_wr_cmd_valid[n +: 1] = ram_wr_cmd_valid_reg;
1084
1085     assign out_done[n] = done_reg;
1086
1087     always @(posedge clk) begin
1088         ram_wr_cmd_valid_reg <= ram_wr_cmd_valid_reg && !
ram_wr_cmd_ready[n +: 1];
1089
1090         out_fifo_half_full_reg <= $un-
signed(out_fifo_wr_ptr_reg - out_fifo_rd_ptr_reg) >= 2**((OUT-
PUT_FIFO_ADDR_WIDTH-1);
1091
1092         if (!out_fifo_full && ram_wr_cmd_valid_int[n +: 1])
begin
1093
1094             out_fifo_wr_cmd_sel[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_sel_int[n*RAM_SEL_WIDTH +: RAM_SEL_WIDTH];
1095
1096             out_fifo_wr_cmd_be[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_be_int[n*RAM_SEG_BE_WIDTH +: RAM_SEG_BE_WIDTH];
1097
1098             out_fifo_wr_cmd_addr[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_addr_int[n*RAM_SEG_ADDR_WIDTH +: RAM_SEG_AD-
DR_WIDTH];
1099
1100             out_fifo_wr_cmd_data[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
<= ram_wr_cmd_data_int[n*RAM_SEG_DATA_WIDTH +:
RAM_SEG_DATA_WIDTH];
1101
1102             out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
1103
1104             if (!out_fifo_empty && (!ram_wr_cmd_valid_reg ||
ram_wr_cmd_ready[n +: 1])) begin
1105                 ram_wr_cmd_sel_reg <=
out_fifo_wr_cmd_sel[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1106                 ram_wr_cmd_be_reg <=
out_fifo_wr_cmd_be[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1107                 ram_wr_cmd_addr_reg <=
out_fifo_wr_cmd_addr[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1108                 ram_wr_cmd_data_reg <=
out_fifo_wr_cmd_data[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]]
;
1109                 ram_wr_cmd_valid_reg <= 1'b1;
1110                 out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
1111             end
1112             if (done_count_reg < 2**OUTPUT_FIFO_ADDR_WIDTH &&
ram_wr_done[n] && !out_done_ack[n]) begin
1113                 done_count_reg <= done_count_reg + 1;
1114                 done_reg <= 1;
1115             end else if (done_count_reg > 0 && !ram_wr_done[n] &&
out_done_ack[n]) begin
1116                 done_count_reg <= done_count_reg - 1;
1117                 done_reg <= done_count_reg > 1;
1118             end
1119             if (rst) begin
1120                 out_fifo_wr_ptr_reg <= 0;
1121                 out_fifo_rd_ptr_reg <= 0;
1122                 ram_wr_cmd_valid_reg <= 1'b0;
1123                 done_count_reg <= 0;
1124                 done_reg <= 1'b0;
1125             end
1126         end
1127     endgenerate
1128 endmodule
1129
1130 `resetall
1131
1132 `resetall

```

7. alexforenrich_verilog-pcie/rtl/dma_if_axi_wr.v (1,249 lines)

```

1 /*
2
3 Copyright (c) 2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights

```

```

8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11

```

```

12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * AXI DMA write interface
33 */
34 module dma_if_axi_wr #
35 (

```

```

36     // Width of AXI data bus in bits
37     parameter AXI_DATA_WIDTH = 32,
38     // Width of AXI address bus in bits
39     parameter AXI_ADDR_WIDTH = 16,
40     // Width of AXI wstrb (width of data bus in words)
41     parameter AXI_STRB_WIDTH = (AXI_DATA_WIDTH/8),
42     // Width of AXI ID signal
43     parameter AXI_ID_WIDTH = 8,
44     // Maximum AXI burst length to generate
45     parameter AXI_MAX_BURST_LEN = 256,
46     // RAM select width
47     parameter RAM_SEL_WIDTH = 2,
48     // RAM address width
49     parameter RAM_ADDR_WIDTH = 16,
50     // RAM segment count
51     parameter RAM_SEG_COUNT = 2,
52     // RAM segment data width
53     parameter RAM_SEG_DATA_WIDTH = AXI_DATA_WIDTH*2/
RAM_SEG_COUNT,
54     // RAM segment byte enable width
55     parameter RAM_SEG_BE_WIDTH = RAM_SEG_DATA_WIDTH/8,
56     // RAM segment address width
57     parameter RAM_SEG_ADDR_WIDTH = RAM_ADDR_WIDTH-$clo-
g2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH),
58     // Immediate enable
59     parameter IMM_ENABLE = 0,
60     // Immediate width
61     parameter IMM_WIDTH = 32,
62     // Length field width
63     parameter LEN_WIDTH = 16,
64     // Tag field width
65     parameter TAG_WIDTH = 8,
66     // Operation table size
67     parameter OP_TABLE_SIZE = 2**(AXI_ID_WIDTH),
68     // Use AXI ID signals
69     parameter USE_AXI_ID = 1
70 )
71 (
72     input wire clk,
73     input wire rst,
74
75     /*
76     * AXI master interface
77     */
78     output wire [AXI_ID_WIDTH-1:0]
m_axi_awid,
79     output wire [AXI_ADDR_WIDTH-1:0]
m_axi_awaddr,
80     output wire [7:0]
m_axi_awlen,
81     output wire [2:0]
m_axi_awsiz,
82     output wire [1:0]
m_axi_awburst,
83     output wire
m_axi_awlock,
84     output wire [3:0]
m_axi_awcache,
85     output wire [2:0]
m_axi_awprot,
86     output wire
m_axi_awvalid,
87     input wire

```

```

m_axi_awready,
88     output wire [AXI_DATA_WIDTH-1:0]
m_axi_wdata,
89     output wire [AXI_STRB_WIDTH-1:0]
m_axi_wstrb,
90     output wire
m_axi_wlast,
91     output wire
m_axi_wvalid,
92     input wire
m_axi_wready,
93     input wire [AXI_ID_WIDTH-1:0]
m_axi_bid,
94     input wire [1:0]
m_axi_bresp,
95     input wire
m_axi_bvalid,
96     output wire
m_axi_bready,
97
98     /*
99     * AXI write descriptor input
100    */
101    input wire [AXI_ADDR_WIDTH-1:0]
is_write_desc_axi_addr,
102    input wire [RAM_SEL_WIDTH-1:0]
is_write_desc_ram_sel,
103    input wire [RAM_ADDR_WIDTH-1:0]
is_write_desc_ram_addr,
104    input wire [IMM_WIDTH-1:0]
is_write_desc_imm,
105    input wire
is_write_desc_imm_en,
106    input wire [LEN_WIDTH-1:0]
is_write_desc_len,
107    input wire [TAG_WIDTH-1:0]
is_write_desc_tag,
108    input wire
is_write_desc_valid,
109    output wire
is_write_desc_ready,
110
111    /*
112    * AXI write descriptor status output
113    */
114    output wire [TAG_WIDTH-1:0]
is_write_desc_status_tag,
115    output wire [3:0]
is_write_desc_status_error,
116    output wire
is_write_desc_status_valid,
117
118    /*
119    * RAM interface
120    */
121    output wire [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0]
ram_rd_cmd_sel,
122    output wire [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_addr,
123    output wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_valid,
124    input wire [RAM_SEG_COUNT-1:0]
ram_rd_cmd_ready,
125    input wire [RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH-1:0]
ram_rd_resp_data,
126    input wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_valid,
127    output wire [RAM_SEG_COUNT-1:0]
ram_rd_resp_ready,
128
129    /*
130    * Configuration
131    */
132    input wire
able,
133
134    /*
135    * Status
136    */
137    output wire
status_busy,
138
139    /*
140    * Statistics
141    */
142    output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_wr_op_start_tag,
143    output wire [LEN_WIDTH-1:0]
stat_wr_op_start_len,
144    output wire
stat_wr_op_start_valid,
145    output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_wr_op_finish_tag,
146    output wire [3:0]
stat_wr_op_finish_status,
147    output wire
stat_wr_op_finish_valid,
148    output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_wr_req_start_tag,
149    output wire [12:0]
stat_wr_req_start_len,

```

s_ax-

s_ax-

s_ax-

s_ax-

s_ax-

s_ax-

s_ax-

s_ax-

s_ax-

m_ax-

m_ax-

m_ax-

en-

```

150    output wire
stat_wr_req_start_valid,
151    output wire [$clog2(OP_TABLE_SIZE)-1:0]
stat_wr_req_finish_tag,
152    output wire [3:0]
stat_wr_req_finish_status,
153    output wire
stat_wr_req_finish_valid,
154    output wire
stat_wr_op_table_full,
155    output wire
stat_wr_tx_stall
156 );
157
158 parameter RAM_DATA_WIDTH = RAM_SEG_COUNT*RAM_SEG_DATA_WIDTH;
159 parameter RAM_WORD_WIDTH = RAM_SEG_BE_WIDTH;
160 parameter RAM_WORD_SIZE = RAM_SEG_DATA_WIDTH/RAM_WORD_WIDTH;
161
162 parameter AXI_WORD_WIDTH = AXI_STRB_WIDTH;
163 parameter AXI_WORD_SIZE = AXI_DATA_WIDTH/AXI_WORD_WIDTH;
164 parameter AXI_BURST_SIZE = $clog2(AXI_STRB_WIDTH);
165 parameter AXI_MAX_BURST_SIZE = AXI_MAX_BURST_LEN <<
AXI_BURST_SIZE;
166
167 parameter OFFSET_WIDTH = AXI_STRB_WIDTH > 1 ? $clog2(AXI_STRB_WIDTH) : 1;
168 parameter OFFSET_MASK = AXI_STRB_WIDTH > 1 ? {OFF-
SET_WIDTH{1'b1}} : 0;
169 parameter RAM_OFFSET_WIDTH = $clog2(RAM_SEG_COUNT*RAM_SEG_BE_WIDTH);
170 parameter ADDR_MASK = {AXI_ADDR_WIDTH{1'b1}} << $clog2(AXI_STRB_WIDTH);
171 parameter CYCLE_COUNT_WIDTH = LEN_WIDTH - AXI_BURST_SIZE + 1;
172
173 parameter MASK_FIFO_ADDR_WIDTH = $clog2(OP_TABLE_SIZE)+1;
174
175 parameter OP_TAG_WIDTH = $clog2(OP_TABLE_SIZE);
176
177 parameter OUTPUT_FIFO_ADDR_WIDTH = 5;
178
179 // bus width assertions
180 initial begin
181     if (AXI_WORD_SIZE * AXI_STRB_WIDTH != AXI_DATA_WIDTH) be-
gin
182         $error("Error: AXI data width not evenly divisble (in-
stance %m)");
183         $finish;
184     end
185
186     if (AXI_WORD_SIZE != RAM_WORD_SIZE) begin
187         $error("Error: word size mismatch (instance %m)");
188         $finish;
189     end
190
191     if (2**$clog2(AXI_WORD_WIDTH) != AXI_WORD_WIDTH) begin
192         $error("Error: AXI word width must be even power of
two (instance %m)");
193         $finish;
194     end
195
196     if (AXI_MAX_BURST_LEN < 1 || AXI_MAX_BURST_LEN > 256) be-
gin
197         $error("Error: AXI_MAX_BURST_LEN must be between 1 and
256 (instance %m)");
198         $finish;
199     end
200
201     if (RAM_SEG_COUNT < 2) begin
202         $error("Error: RAM interface requires at least 2 seg-
ments (instance %m)");
203         $finish;
204     end
205
206     if (RAM_DATA_WIDTH != AXI_DATA_WIDTH*2) begin
207         $error("Error: RAM interface width must be double the
AXI interface width (instance %m)");
208         $finish;
209     end
210
211     if (2**$clog2(RAM_WORD_WIDTH) != RAM_WORD_WIDTH) begin
212         $error("Error: RAM word width must be even power of
two (instance %m)");
213         $finish;
214     end
215
216     if (RAM_ADDR_WIDTH != RAM_SEG_ADDR_WIDTH+$clog2(RAM_SEG_COUNT)+$clog2(RAM_SEG_BE_WIDTH)) begin
217         $error("Error: RAM_ADDR_WIDTH does not match RAM con-
figuration (instance %m)");
218         $finish;
219     end
220
221     if (OP_TABLE_SIZE > 2**AXI_ID_WIDTH) begin
222         $error("Error: AXI_ID_WIDTH insufficient for requested
OP_TABLE_SIZE (instance %m)");
223         $finish;
224     end
225
226     if (IMM_ENABLE && IMM_WIDTH > AXI_DATA_WIDTH) begin

```



```

227 $error("Error: IMM_WIDTH must not be larger than the
AXI interface width (instance %m)");
228 $finish;
229 end
230 end
231
232 localparam [1:0]
233 AXI_RESP_OKAY = 2'b00,
234 AXI_RESP_EXOKAY = 2'b01,
235 AXI_RESP_SLVERR = 2'b10,
236 AXI_RESP_DECERR = 2'b11;
237
238 localparam [3:0]
239 DMA_ERROR_NONE = 4'd0,
240 DMA_ERROR_TIMEOUT = 4'd1,
241 DMA_ERROR_PARITY = 4'd2,
242 DMA_ERROR_AXI_RD_SLVERR = 4'd4,
243 DMA_ERROR_AXI_RD_DECERR = 4'd5,
244 DMA_ERROR_AXI_WR_SLVERR = 4'd6,
245 DMA_ERROR_AXI_WR_DECERR = 4'd7,
246 DMA_ERROR_PCIE_FLR = 4'd8,
247 DMA_ERROR_PCIE_CPL_POISONED = 4'd9,
248 DMA_ERROR_PCIE_CPL_STATUS_UR = 4'd10,
249 DMA_ERROR_PCIE_CPL_STATUS_CA = 4'd11;
250
251 localparam [0:0]
252 REQ_STATE_IDLE = 1'd0,
253 REQ_STATE_START = 1'd1;
254
255 reg [0:0] req_state_reg = REQ_STATE_IDLE, req_state_next;
256
257 localparam [0:0]
258 READ_STATE_IDLE = 1'd0,
259 READ_STATE_READ = 1'd1;
260
261 reg [0:0] read_state_reg = READ_STATE_IDLE, read_state_next;
262
263 localparam [0:0]
264 AXI_STATE_IDLE = 1'd0,
265 AXI_STATE_TRANSFER = 1'd1;
266
267 reg [0:0] axi_state_reg = AXI_STATE_IDLE, axi_state_next;
268
269 // datapath control signals
270 reg mask_fifo_we;
271
272 reg read_cmd_ready;
273
274 reg [AXI_ADDR_WIDTH-1:0] req_axi_addr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, req_axi_addr_next;
275 reg [RAM_SEL_WIDTH-1:0] ram_sel_reg = {RAM_SEL_WIDTH{1'b0}},
ram_sel_next;
276 reg [RAM_ADDR_WIDTH-1:0] ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, ram_addr_next;
277 reg [IMM_WIDTH-1:0] imm_reg = {IMM_WIDTH{1'b0}}, imm_next;
278 reg [imm_en-1:0] imm_en_reg = {1'b0, imm_en_next};
279 reg [LEN_WIDTH-1:0] op_count_reg = {LEN_WIDTH{1'b0}},
op_count_next;
280 reg [zero_len-1:0] zero_len_reg = {1'b0, zero_len_next};
281 reg [LEN_WIDTH-1:0] tr_count_reg = {LEN_WIDTH{1'b0}},
tr_count_next;
282 reg [12:0] tr_word_count_reg = 13'd0, tr_word_count_next;
283 reg [TAG_WIDTH-1:0] tag_reg = {TAG_WIDTH{1'b0}}, tag_next;
284
285 reg [AXI_ADDR_WIDTH-1:0] read_axi_addr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, read_axi_addr_next;
286 reg [RAM_SEL_WIDTH-1:0] read_ram_sel_reg =
{RAM_SEL_WIDTH{1'b0}}, read_ram_sel_next;
287 reg [RAM_ADDR_WIDTH-1:0] read_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, read_ram_addr_next;
288 reg [read_imm_en-1:0] read_imm_en_reg = {1'b0, read_imm_en_next};
289 reg [LEN_WIDTH-1:0] read_len_reg = {LEN_WIDTH{1'b0}},
read_len_next;
290 reg [RAM_SEG_COUNT-1:0] read_ram_mask_reg =
{RAM_SEG_COUNT{1'b0}}, read_ram_mask_next;
291 reg [RAM_SEG_COUNT-1:0] read_ram_mask_0_reg =
{RAM_SEG_COUNT{1'b0}}, read_ram_mask_0_next;
292 reg [RAM_SEG_COUNT-1:0] read_ram_mask_1_reg =
{RAM_SEG_COUNT{1'b0}}, read_ram_mask_1_next;
293 reg [ram_wrap-1:0] ram_wrap_reg = {1'b0, ram_wrap_next};
294 reg [CYCLE_COUNT_WIDTH-1:0] read_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, read_cycle_count_next;
295 reg [read_last_cycle-1:0] read_last_cycle_reg = {1'b0, read_last_cycle_next};
296 reg [OFFSET_WIDTH+1-1:0] cycle_byte_count_reg = {OFF-
SET_WIDTH+1{1'b0}}, cycle_byte_count_next;
297 reg [RAM_OFFSET_WIDTH-1:0] start_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, start_offset_next;
298 reg [RAM_OFFSET_WIDTH-1:0] end_offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, end_offset_next;
299
300 reg [AXI_ADDR_WIDTH-1:0] axi_addr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, axi_addr_next;
301 reg [IMM_WIDTH-1:0] axi_imm_reg = {IMM_WIDTH{1'b0}},
axi_imm_next;
302 reg [axi_imm_en-1:0] axi_imm_en_reg = {1'b0, axi_imm_en_next};
303 reg [12:0] axi_len_reg = 13'd0, axi_len_next;
304 reg [axi_zero_len-1:0] axi_zero_len_reg = {1'b0, axi_zero_len_next};
305 reg [RAM_OFFSET_WIDTH-1:0] offset_reg = {RAM_OFF-
SET_WIDTH{1'b0}}, offset_next;
306 reg [AXI_STRB_WIDTH-1:0] strb_offset_mask_reg =
{AXI_STRB_WIDTH{1'b1}}, strb_offset_mask_next;

```

```

307 reg [OFFSET_WIDTH-1:0] last_cycle_offset_reg = {OFF-
SET_WIDTH{1'b0}}, last_cycle_offset_next;
308 reg [RAM_SEG_COUNT-1:0] ram_mask_reg = {RAM_SEG_COUNT{1'b0}},
ram_mask_next;
309 reg [ram_mask_valid-1:0] ram_mask_valid_reg = {1'b0, ram_mask_valid_next};
310 reg [CYCLE_COUNT_WIDTH-1:0] cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, cycle_count_next;
311 reg [last_cycle-1:0] last_cycle_reg = {1'b0, last_cycle_next};
312
313 reg [AXI_ADDR_WIDTH-1:0] read_cmd_axi_addr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, read_cmd_axi_addr_next;
314 reg [RAM_SEL_WIDTH-1:0] read_cmd_ram_sel_reg =
{RAM_SEL_WIDTH{1'b0}}, read_cmd_ram_sel_next;
315 reg [RAM_ADDR_WIDTH-1:0] read_cmd_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}}, read_cmd_ram_addr_next;
316 reg [read_cmd_imm_en-1:0] read_cmd_imm_en_reg = {1'b0, read_cmd_imm_en_next};
317 reg [12:0] read_cmd_len_reg = 13'd0, read_cmd_len_next;
318 reg [CYCLE_COUNT_WIDTH-1:0] read_cmd_cycle_count_reg =
{CYCLE_COUNT_WIDTH{1'b0}}, read_cmd_cycle_count_next;
319 reg [read_cmd_last_cycle-1:0] read_cmd_last_cycle_reg = {1'b0, read_cmd_last_cycle_next};
320 reg [read_cmd_valid-1:0] read_cmd_valid_reg = {1'b0, read_cmd_valid_next};
321
322 reg [MASK_FIFO_ADDR_WIDTH+1-1:0] mask_fifo_wr_ptr_reg = 0;
323 reg [MASK_FIFO_ADDR_WIDTH+1-1:0] mask_fifo_rd_ptr_reg = 0,
mask_fifo_rd_ptr_next;
324 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
325 reg [RAM_SEG_COUNT-1:0]
mask_fifo_mask[(2*MASK_FIFO_ADDR_WIDTH)-1:0];
326 reg [RAM_SEG_COUNT-1:0] mask_fifo_wr_mask;
327
328 wire mask_fifo_empty = mask_fifo_wr_ptr_reg ==
mask_fifo_rd_ptr_reg;
329 wire mask_fifo_full = mask_fifo_wr_ptr_reg ==
(mask_fifo_rd_ptr_reg ^ (1 << MASK_FIFO_ADDR_WIDTH));
330
331 reg [OP_TAG_WIDTH+1-1:0] active_op_count_reg = 0;
332 reg [inc_active_op];
333 reg [dec_active_op];
334
335 reg [AXI_ID_WIDTH-1:0] m_axi_awid_reg = {AXI_ID_WIDTH{1'b0}},
m_axi_awid_next;
336 reg [AXI_ADDR_WIDTH-1:0] m_axi_awaddr_reg = {AXI_AD-
DR_WIDTH{1'b0}}, m_axi_awaddr_next;
337 reg [7:0] m_axi_awlen_reg = 8'd0, m_axi_awlen_next;
338 reg [m_axi_awvalid-1:0] m_axi_awvalid_reg = {1'b0, m_axi_awvalid_next};
339 reg [m_axi_bready-1:0] m_axi_bready_reg = {1'b0, m_axi_bready_next};
340
341 reg [s_axis_write_desc_ready-1:0] s_ax-
is_write_desc_ready_reg = {1'b0, s_ax-
is_write_desc_ready_next};
342
343 reg [TAG_WIDTH-1:0] m_axis_write_desc_status_tag_reg =
{TAG_WIDTH{1'b0}}, m_axis_write_desc_status_tag_next;
344 reg [3:0] m_axis_write_desc_status_error_reg = 4'd0, m_ax-
is_write_desc_status_error_next;
345 reg [m_axis_write_desc_status_valid-1:0] m_ax-
is_write_desc_status_valid_reg = {1'b0, m_ax-
is_write_desc_status_valid_next};
346
347 reg [RAM_SEG_COUNT*RAM_SEL_WIDTH-1:0] ram_rd_cmd_sel_reg = 0,
ram_rd_cmd_sel_next;
348 reg [RAM_SEG_COUNT*RAM_SEG_ADDR_WIDTH-1:0] ram_rd_cmd_addr_reg = 0,
ram_rd_cmd_addr_next;
349 reg [RAM_SEG_COUNT-1:0] ram_rd_cmd_valid_reg = 0,
ram_rd_cmd_valid_next;
350 reg [RAM_SEG_COUNT-1:0] ram_rd_resp_ready_cmb;
351
352 reg [status_busy-1:0] status_busy_reg = {1'b0};
353
354 reg [OP_TAG_WIDTH-1:0] stat_wr_op_start_tag_reg = 0,
stat_wr_op_start_tag_next;
355 reg [LEN_WIDTH-1:0] stat_wr_op_start_len_reg = 0,
stat_wr_op_start_len_next;
356 reg [stat_wr_op_start_valid-1:0] stat_wr_op_start_valid_reg = {1'b0},
stat_wr_op_start_valid_next;
357 reg [OP_TAG_WIDTH-1:0] stat_wr_op_finish_tag_reg = 0,
stat_wr_op_finish_tag_next;
358 reg [3:0] stat_wr_op_finish_status_reg = 0, stat_wr_op_fin-
ish_status_next;
359 reg [stat_wr_op_finish_valid-1:0] stat_wr_op_finish_valid_reg = {1'b0},
stat_wr_op_finish_valid_next;
360 reg [OP_TAG_WIDTH-1:0] stat_wr_req_start_tag_reg = 0,
stat_wr_req_start_tag_next;
361 reg [12:0] stat_wr_req_start_len_reg = 13'd0,
stat_wr_req_start_len_next;
362 reg [stat_wr_req_start_valid-1:0] stat_wr_req_start_valid_reg = {1'b0},
stat_wr_req_start_valid_next;
363 reg [OP_TAG_WIDTH-1:0] stat_wr_req_finish_tag_reg = 0,
stat_wr_req_finish_tag_next;
364 reg [3:0] stat_wr_req_finish_status_reg = 0, stat_wr_req_fin-
ish_status_next;
365 reg [stat_wr_req_finish_valid-1:0] stat_wr_req_finish_valid_reg = {1'b0},
stat_wr_req_finish_valid_next;
366 reg [stat_wr_op_table_full-1:0] stat_wr_op_t-
able_full_reg = {1'b0, stat_wr_op_t-
able_full_next};
367 reg [stat_wr_tx_stall-1:0] stat_wr_tx_stall_reg = {1'b0, stat_wr_tx_stall_next};
368
369 // internal datapath
370 reg [AXI_DATA_WIDTH-1:0] m_axi_wdata_int;
371 reg [AXI_STRB_WIDTH-1:0] m_axi_wstrb_int;
372 reg [m_axi_wlast-1:0] m_axi_wlast_int;

```

```

373 reg m_axi_wvalid_int;
374 wire m_axi_wready_int;
375
376 assign m_axi_awid = USE_AXI_ID ? m_axi_awid_reg :
{AXI_ID_WIDTH(1'b0)};
377 assign m_axi_awaddr = m_axi_awaddr_reg;
378 assign m_axi_awlen = m_axi_awlen_reg;
379 assign m_axi_awsz = AXI_BURST_SIZE;
380 assign m_axi_awburst = 2'b01;
381 assign m_axi_awlock = 1'b0;
382 assign m_axi_awcache = 4'b0011;
383 assign m_axi_awprot = 3'b010;
384 assign m_axi_awvalid = m_axi_awvalid_reg;
385 assign m_axi_bready = m_axi_bready_reg;
386
387 assign s_axis_write_desc_ready = s_axis_write_desc_ready_reg;
388
389 assign m_axis_write_desc_status_tag = m_ax-
is_write_desc_status_tag_reg;
390 assign m_axis_write_desc_status_error = m_ax-
is_write_desc_status_error_reg;
391 assign m_axis_write_desc_status_valid = m_ax-
is_write_desc_status_valid_reg;
392
393 assign ram_rd_cmd_sel = ram_rd_cmd_sel_reg;
394 assign ram_rd_cmd_addr = ram_rd_cmd_addr_reg;
395 assign ram_rd_cmd_valid = ram_rd_cmd_valid_reg;
396 assign ram_rd_resp_ready = ram_rd_resp_ready_cmb;
397
398 assign status_busy = status_busy_reg;
399
400 assign stat_wr_op_start_tag = stat_wr_op_start_tag_reg;
401 assign stat_wr_op_start_len = stat_wr_op_start_len_reg;
402 assign stat_wr_op_start_valid = stat_wr_op_start_valid_reg;
403 assign stat_wr_op_finish_tag = stat_wr_op_finish_tag_reg;
404 assign stat_wr_op_finish_status = stat_wr_op_fin-
ish_status_reg;
405 assign stat_wr_op_finish_valid = stat_wr_op_finish_valid_reg;
406 assign stat_wr_req_start_tag = stat_wr_req_start_tag_reg;
407 assign stat_wr_req_start_len = stat_wr_req_start_len_reg;
408 assign stat_wr_req_start_valid = stat_wr_req_start_valid_reg;
409 assign stat_wr_req_finish_tag = stat_wr_req_finish_tag_reg;
410 assign stat_wr_req_finish_status = stat_wr_req_fin-
ish_status_reg;
411 assign stat_wr_req_finish_valid = stat_wr_req_fin-
ish_valid_reg;
412 assign stat_wr_op_table_full = stat_wr_op_table_full_reg;
413 assign stat_wr_tx_stall = stat_wr_tx_stall_reg;
414
415 // operation tag management
416 reg [OP_TAG_WIDTH-1:0] op_table_start_ptr_reg = 0;
417 reg [AXI_ADDR_WIDTH-1:0] op_table_start_axi_addr;
418 reg [IMM_WIDTH-1:0] op_table_start_imm;
419 reg op_table_start_imm_en;
420 reg [11:0] op_table_start_len;
421 reg op_table_start_zero_len;
422 reg [CYCLE_COUNT_WIDTH-1:0] op_table_start_cycle_count;
423 reg [RAM_OFFSET_WIDTH-1:0] op_table_start_offset;
424 reg [TAG_WIDTH-1:0] op_table_start_tag;
425 reg op_table_start_last;
426 reg op_table_start_en;
427 reg [OP_TAG_WIDTH+1-1:0] op_table_tx_start_ptr_reg = 0;
428 reg op_table_tx_start_en;
429 reg [OP_TAG_WIDTH+1-1:0] op_table_tx_finish_ptr_reg = 0;
430 reg op_table_tx_finish_en;
431 reg [OP_TAG_WIDTH-1:0] op_table_write_complete_ptr;
432 reg [3:0] op_table_write_complete_error;
433 reg op_table_write_complete_en;
434 reg [OP_TAG_WIDTH+1-1:0] op_table_finish_ptr_reg = 0;
435 reg op_table_finish_en;
436
437 reg [2**OP_TAG_WIDTH-1:0] op_table_active = 0;
438 reg [2**OP_TAG_WIDTH-1:0] op_table_write_complete = 0;
439 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
440 reg [AXI_ADDR_WIDTH-1:0] op_t-
able_axi_addr[2**OP_TAG_WIDTH-1:0];
441 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
442 reg [IMM_WIDTH-1:0] op_table_imm[2**OP_TAG_WIDTH-1:0];
443 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
444 reg op_table_imm_en[2**OP_TAG_WIDTH-1:0];
445 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
446 reg [11:0] op_table_len[2**OP_TAG_WIDTH-1:0];
447 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
448 reg op_table_zero_len[2**OP_TAG_WIDTH-1:0];
449 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
450 reg [CYCLE_COUNT_WIDTH-1:0] op_t-
able_cycle_count[2**OP_TAG_WIDTH-1:0];
451 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
452 reg [RAM_OFFSET_WIDTH-1:0] op_t-
able_offset[2**OP_TAG_WIDTH-1:0];
453 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
454 reg [TAG_WIDTH-1:0] op_table_tag[2**OP_TAG_WIDTH-1:0];
455 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"

```

```

*)
456 reg op_table_last[2**OP_TAG_WIDTH-1:0];
457 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
458 reg [3:0] op_table_error_code [2**OP_TAG_WIDTH-1:0];
459
460 integer i;
461
462 initial begin
463   for (i = 0; i < 2**OP_TAG_WIDTH; i = i + 1) begin
464     op_table_axi_addr[i] = 0;
465     op_table_imm[i] = 0;
466     op_table_imm_en[i] = 0;
467     op_table_len[i] = 0;
468     op_table_zero_len[i] = 1'b0;
469     op_table_cycle_count[i] = 0;
470     op_table_offset[i] = 0;
471     op_table_tag[i] = 0;
472     op_table_last[i] = 0;
473     op_table_error_code[i] = 0;
474   end
475 end
476
477 always @* begin
478   req_state_next = REQ_STATE_IDLE;
479
480   s_axis_write_desc_ready_next = 1'b0;
481
482   stat_wr_op_start_tag_next = stat_wr_op_start_tag_reg;
483   stat_wr_op_start_len_next = stat_wr_op_start_len_reg;
484   stat_wr_op_start_valid_next = 1'b0;
485   stat_wr_req_start_tag_next = stat_wr_req_start_tag_reg;
486   stat_wr_req_start_len_next = stat_wr_req_start_len_reg;
487   stat_wr_req_start_valid_next = 1'b0;
488   stat_wr_op_table_full_next = !(op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH));
489   stat_wr_tx_stall_next = (m_axi_awvalid && !m_axi_wready)
|| (m_axi_wvalid && !m_axi_wready);
490
491   tag_next = tag_reg;
492   req_axi_addr_next = req_axi_addr_reg;
493   ram_sel_next = ram_sel_reg;
494   ram_addr_next = ram_addr_reg;
495   imm_next = imm_reg;
496   imm_en_next = imm_en_reg;
497   op_count_next = op_count_reg;
498   zero_len_next = zero_len_reg;
499   tr_count_next = tr_count_reg;
500   tr_word_count_next = tr_word_count_reg;
501
502   read_cmd_axi_addr_next = read_cmd_axi_addr_reg;
503   read_cmd_ram_sel_next = read_cmd_ram_sel_reg;
504   read_cmd_ram_addr_next = read_cmd_ram_addr_reg;
505   read_cmd_imm_en_next = read_cmd_imm_en_reg;
506   read_cmd_len_next = read_cmd_len_reg;
507   read_cmd_cycle_count_next = read_cmd_cycle_count_reg;
508   read_cmd_last_cycle_next = read_cmd_last_cycle_reg;
509   read_cmd_valid_next = read_cmd_valid_reg && !
read_cmd_ready;
510
511   op_table_start_axi_addr = req_axi_addr_reg;
512   op_table_start_imm = imm_reg;
513   op_table_start_imm_en = imm_en_reg;
514   op_table_start_len = 0;
515   op_table_start_zero_len = zero_len_reg;
516   op_table_start_cycle_count = 0;
517   op_table_start_offset = (req_axi_addr_reg & OFFSET_MASK)-
ram_addr_reg[RAM_OFFSET_WIDTH-1:0];
518   op_table_start_tag = tag_reg;
519   op_table_start_last = 0;
520   op_table_start_en = 1'b0;
521
522   inc_active_op = 1'b0;
523
524   // TLP segmentation
525   case (req_state_reg)
526     REQ_STATE_IDLE: begin
527       // idle state, wait for incoming descriptor
528       s_axis_write_desc_ready_next = !op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
2**OP_TAG_WIDTH) && enable;
529
530       req_axi_addr_next = s_axis_write_desc_axi_addr;
531       if (IMM_ENABLE && s_axis_write_desc_imm_en) begin
532         ram_sel_next = 0;
533         ram_addr_next = 0;
534       end else begin
535         ram_sel_next = s_axis_write_desc_ram_sel;
536         ram_addr_next = s_axis_write_desc_ram_addr;
537       end
538       imm_next = s_axis_write_desc_imm;
539       imm_en_next = IMM_ENABLE && s_ax-
is_write_desc_imm_en;
540       if (s_axis_write_desc_len == 0) begin
541         // zero-length operation
542         op_count_next = 1;
543         zero_len_next = 1'b1;
544       end else begin

```



```

545         op_count_next = s_axis_write_desc_len;
546         zero_len_next = 1'b0;
547     end
548     tag_next = s_axis_write_desc_tag;
549
550     if (op_count_next <= AXI_MAX_BURST_SIZE -
551 (req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
552 gin
553         // packet smaller than max burst size
554         if (((req_axi_addr_next & 12'hfff) +
555 (op_count_next & 12'hfff)) >> 12 != 0 || op_count_next >> 12 != 0)
556 begin
557             // crosses 4k boundary
558             tr_word_count_next = 13'h1000 -
559 req_axi_addr_next[11:0];
560         end else begin
561             // does not cross 4k boundary
562             tr_word_count_next = op_count_next;
563         end
564     end else begin
565         // packet larger than max burst size
566         if (((req_axi_addr_next & 12'hfff) +
567 AXI_MAX_BURST_SIZE) >> 12 != 0) begin
568             // crosses 4k boundary
569             tr_word_count_next = 13'h1000 -
570 req_axi_addr_next[11:0];
571         end else begin
572             // does not cross 4k boundary
573             tr_word_count_next = AXI_MAX_BURST_SIZE -
574 (req_axi_addr_next & OFFSET_MASK);
575         end
576     end
577
578     if (s_axis_write_desc_ready & s_ax-
579 is_write_desc_valid) begin
580         s_axis_write_desc_ready_next = 1'b0;
581
582         stat_wr_op_start_tag_next =
583 stat_wr_op_start_tag_reg+1;
584         stat_wr_op_start_len_next = s_ax-
585 is_write_desc_len;
586         stat_wr_op_start_valid_next = 1'b1;
587
588         req_state_next = REQ_STATE_START;
589     else begin
590         req_state_next = REQ_STATE_IDLE;
591     end
592 end
593
594 REQ_STATE_START: begin
595     // start state, compute length
596     if (!op_t-
597 able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
598 signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
599 2*OP_TAG_WIDTH) && (!ram_rd_cmd_valid_reg || ram_rd_cmd_ready) &&
600 (!read_cmd_valid_reg || read_cmd_ready)) begin
601         read_cmd_axi_addr_next = req_axi_addr_reg;
602         read_cmd_ram_sel_next = ram_sel_reg;
603         read_cmd_ram_addr_next = ram_addr_reg;
604         read_cmd_imm_en_next = imm_en_reg;
605         read_cmd_len_next = tr_word_count_next;
606         read_cmd_cycle_count_next =
607 (tr_word_count_next + (req_axi_addr_reg & OFFSET_MASK) - 1) >>
608 AXI_BURST_SIZE;
609         op_table_start_cycle_count =
610 read_cmd_cycle_count_next;
611         read_cmd_last_cycle_next =
612 read_cmd_cycle_count_next == 0;
613         read_cmd_valid_next = 1'b1;
614
615         req_axi_addr_next = req_axi_addr_reg +
616 tr_word_count_next;
617         ram_addr_next = ram_addr_reg +
618 tr_word_count_next;
619         op_count_next = op_count_reg -
620 tr_word_count_next;
621
622         op_table_start_axi_addr = req_axi_addr_reg;
623         op_table_start_imm = imm_en_reg;
624         op_table_start_imm_en = imm_en_reg;
625         op_table_start_len = tr_word_count_next;
626         op_table_start_zero_len = zero_len_reg;
627         op_table_start_offset = (req_axi_addr_reg &
628 OFFSET_MASK)-ram_addr_reg[RAM_OFFSET_WIDTH-1:0];
629         op_table_start_tag = tag_reg;
630         op_table_start_last = op_count_reg ==
631 tr_word_count_next;
632         op_table_start_en = 1'b1;
633         inc_active_op = 1'b1;
634
635         stat_wr_req_start_tag_next = op_t-
636 able_start_ptr_reg[OP_TAG_WIDTH-1:0];
637         stat_wr_req_start_len_next = zero_len_reg ?
638 0 : tr_word_count_next;
639         stat_wr_req_start_valid_next = 1'b1;
640
641         if (op_count_next <= AXI_MAX_BURST_SIZE -
642 (req_axi_addr_next & OFFSET_MASK) || AXI_MAX_BURST_SIZE >= 4096) be-
643 gin
644             // packet smaller than max burst size
645             if (((req_axi_addr_next & 12'hfff) +
646 (op_count_next & 12'hfff)) >> 12 != 0 || op_count_next >> 12 != 0)

```

```

begin
617         // crosses 4k boundary
618         tr_word_count_next = 13'h1000 -
619 req_axi_addr_next[11:0];
620     end else begin
621         // does not cross 4k boundary
622         tr_word_count_next = op_count_next;
623     end
624 end else begin
625     // packet larger than max burst size
626     if (((req_axi_addr_next & 12'hfff) +
627 AXI_MAX_BURST_SIZE) >> 12 != 0) begin
628         // crosses 4k boundary
629         tr_word_count_next = 13'h1000 -
630 req_axi_addr_next[11:0];
631     end else begin
632         // does not cross 4k boundary
633         tr_word_count_next =
634 AXI_MAX_BURST_SIZE - (req_axi_addr_next & OFFSET_MASK);
635     end
636 end
637
638 if (!op_table_start_last) begin
639     req_state_next = REQ_STATE_START;
640 end else begin
641     s_axis_write_desc_ready_next = !op_t-
642 able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] && ($un-
643 signed(op_table_start_ptr_reg - op_table_finish_ptr_reg) <
644 2*OP_TAG_WIDTH) && enable;
645     req_state_next = REQ_STATE_IDLE;
646 end
647 end else begin
648     req_state_next = REQ_STATE_START;
649 end
650 end
651 endcase
652
653 always @* begin
654     read_state_next = READ_STATE_IDLE;
655
656     read_cmd_ready = 1'b0;
657
658     ram_rd_cmd_sel_next = ram_rd_cmd_sel_reg;
659     ram_rd_cmd_addr_next = ram_rd_cmd_addr_reg;
660     ram_rd_cmd_valid_next = ram_rd_cmd_valid_reg &
661 ~ram_rd_cmd_ready;
662
663     read_axi_addr_next = read_axi_addr_reg;
664     read_ram_sel_next = read_ram_sel_reg;
665     read_ram_addr_next = read_ram_addr_reg;
666     read_imm_en_next = read_imm_en_reg;
667     read_len_next = read_len_reg;
668     read_ram_mask_next = read_ram_mask_reg;
669     read_ram_mask_0_next = read_ram_mask_0_reg;
670     read_ram_mask_1_next = read_ram_mask_1_reg;
671     ram_wrap_next = ram_wrap_reg;
672     read_cycle_count_next = read_cycle_count_reg;
673     read_last_cycle_next = read_last_cycle_reg;
674     cycle_byte_count_next = cycle_byte_count_reg;
675     start_offset_next = start_offset_reg;
676     end_offset_next = end_offset_reg;
677
678     mask_fifo_wr_mask = read_ram_mask_reg;
679     mask_fifo_we = 1'b0;
680
681     // Read request generation
682     case (read_state_reg)
683     READ_STATE_IDLE: begin
684         // idle state, wait for read command
685
686         read_axi_addr_next = read_cmd_axi_addr_reg;
687         read_ram_sel_next = read_cmd_ram_sel_reg;
688         read_ram_addr_next = read_cmd_ram_addr_reg;
689         read_imm_en_next = read_cmd_imm_en_reg;
690         read_len_next = read_cmd_len_reg;
691         read_cycle_count_next = read_cmd_cycle_count_reg;
692         read_last_cycle_next = read_cmd_last_cycle_reg;
693
694         if (read_len_next > AXI_STRB_WIDTH-(read_axi_ad-
695 dr_next & OFFSET_MASK)) begin
696             cycle_byte_count_next = AXI_STRB_WIDTH-
697 (read_axi_addr_next & OFFSET_MASK);
698         end else begin
699             cycle_byte_count_next = read_len_next;
700         end
701         start_offset_next = read_ram_addr_next;
702         {ram_wrap_next, end_offset_next} =
703 start_offset_next+cycle_byte_count_next-1;
704         read_ram_mask_0_next = {RAM_SEG_COUNT{1'b1}} <<
705 (start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
706         read_ram_mask_1_next = {RAM_SEG_COUNT{1'b1}} >>
707 (RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
708
709         if (!ram_wrap_next) begin
710             read_ram_mask_next = read_ram_mask_0_next &
711 read_ram_mask_1_next;
712             read_ram_mask_0_next = read_ram_mask_0_next &
713 read_ram_mask_1_next;
714             read_ram_mask_1_next = 0;

```

```

702         end else begin
703             read_ram_mask_next = read_ram_mask_0_next |
read_ram_mask_1_next;
704         end
705
706         if (read_cmd_valid_reg) begin
707             read_cmd_ready = 1'b1;
708             read_state_next = READ_STATE_READ;
709         end else begin
710             read_state_next = READ_STATE_IDLE;
711         end
712     end
713     READ_STATE_READ: begin
714         // read state - start new read operations
715
716         if (!(ram_rd_cmd_valid & ~ram_rd_cmd_ready &
read_ram_mask_reg) && !mask_fifo_full) begin
717             // update counters
718             read_ram_addr_next = read_ram_addr_reg +
cycle_byte_count_reg;
719             read_len_next = read_len_reg -
cycle_byte_count_reg;
720
721             read_cycle_count_next = read_cycle_count_reg -
1;
722             read_last_cycle_next = read_cycle_count_next
== 0;
723
724             for (i = 0; i < RAM_SEG_COUNT; i = i + 1) be-
gin
725                 if (read_ram_mask_reg[i]) begin
726                     ram_rd_cmd_sel_next[i*RAM_SEL_WIDTH +:
RAM_SEL_WIDTH] = read_ram_sel_reg;
727                     ram_rd_cmd_ad-
dr_next[i*RAM_SEG_ADDR_WIDTH +: RAM_SEG_ADDR_WIDTH] = read_ram_ad-
dr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH];
728                     ram_rd_cmd_valid_next[i] = !(IMM_EN-
ABLE && read_imm_en_reg);
729                 end
730                 if (read_ram_mask_1_reg[i]) begin
731                     ram_rd_cmd_ad-
dr_next[i*RAM_SEG_ADDR_WIDTH +: RAM_SEG_ADDR_WIDTH] = read_ram_ad-
dr_reg[RAM_ADDR_WIDTH-1:RAM_ADDR_WIDTH-RAM_SEG_ADDR_WIDTH]+1;
732                 end
733             end
734
735             mask_fifo_wr_mask = (IMM_ENABLE &&
read_imm_en_reg) ? 0 : read_ram_mask_reg;
736             mask_fifo_we = 1'b1;
737
738             if (read_len_next > AXI_STRB_WIDTH) begin
739                 cycle_byte_count_next = AXI_STRB_WIDTH;
740             end else begin
741                 cycle_byte_count_next = read_len_next;
742             end
743             start_offset_next = read_ram_addr_next;
744             {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
745
746             read_ram_mask_0_next = {RAM_SEG_COUNT{1'b1}}
<< (start_offset_next >> $clog2(RAM_SEG_BE_WIDTH));
747             read_ram_mask_1_next = {RAM_SEG_COUNT{1'b1}}
>> (RAM_SEG_COUNT-1-(end_offset_next >> $clog2(RAM_SEG_BE_WIDTH)));
748
749             if (!ram_wrap_next) begin
750                 read_ram_mask_next = read_ram_mask_0_next
& read_ram_mask_1_next;
751                 read_ram_mask_0_next =
read_ram_mask_0_next & read_ram_mask_1_next;
752                 read_ram_mask_1_next = 0;
753             end else begin
754                 read_ram_mask_next = read_ram_mask_0_next
| read_ram_mask_1_next;
755             end
756
757             if (!read_last_cycle_reg) begin
758                 read_state_next = READ_STATE_READ;
759             end else if (read_cmd_valid_reg) begin
760
761                 read_axi_addr_next = read_cmd_axi_ad-
dr_reg;
762                 read_ram_sel_next = read_cmd_ram_sel_reg;
763                 read_ram_addr_next = read_cmd_ram_ad-
dr_reg;
764                 read_imm_en_next = read_cmd_imm_en_reg;
765                 read_len_next = read_cmd_len_reg;
766                 read_cycle_count_next =
read_cmd_cycle_count_reg;
767                 read_last_cycle_next =
read_cmd_last_cycle_reg;
768
769                 if (read_len_next > AXI_STRB_WIDTH-
(read_axi_addr_next & OFFSET_MASK)) begin
770                     cycle_byte_count_next =
AXI_STRB_WIDTH-(read_axi_addr_next & OFFSET_MASK);
771                 end else begin
772                     cycle_byte_count_next = read_len_next;
773                 end
774                 start_offset_next = read_ram_addr_next;

```

```

775                 {ram_wrap_next, end_offset_next} =
start_offset_next+cycle_byte_count_next-1;
776
777                 read_ram_mask_0_next =
{RAM_SEG_COUNT{1'b1}} << (start_offset_next >> $clog-
2(RAM_SEG_BE_WIDTH));
778                 read_ram_mask_1_next =
{RAM_SEG_COUNT{1'b1}} >> (RAM_SEG_COUNT-1-(end_offset_next >> $clog-
2(RAM_SEG_BE_WIDTH)));
779
780                 if (!ram_wrap_next) begin
781                     read_ram_mask_next =
read_ram_mask_0_next & read_ram_mask_1_next;
782                     read_ram_mask_0_next =
read_ram_mask_0_next & read_ram_mask_1_next;
783                     read_ram_mask_1_next = 0;
784                 end else begin
785                     read_ram_mask_next =
read_ram_mask_0_next | read_ram_mask_1_next;
786                 end
787
788                 read_cmd_ready = 1'b1;
789
790                 read_state_next = READ_STATE_READ;
791             end else begin
792                 read_state_next = READ_STATE_IDLE;
793             end
794             end else begin
795                 read_state_next = READ_STATE_READ;
796             end
797         end
798     endcase
799 end
800
801 always @* begin
802     axi_state_next = AXI_STATE_IDLE;
803
804     ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
805
806     stat_wr_op_finish_tag_next = stat_wr_op_finish_tag_reg;
807     stat_wr_op_finish_status_next = stat_wr_op-fin-
ish_status_reg;
808     stat_wr_op_finish_valid_next = 1'b0;
809     stat_wr_req_finish_tag_next = stat_wr_req_finish_tag_reg;
810     stat_wr_req_finish_status_next = stat_wr_req-fin-
ish_status_reg;
811     stat_wr_req_finish_valid_next = 1'b0;
812
813     axi_addr_next = axi_addr_reg;
814     axi_imm_next = axi_imm_reg;
815     axi_imm_en_next = axi_imm_en_reg;
816     axi_len_next = axi_len_reg;
817     axi_zero_len_next = axi_zero_len_reg;
818     offset_next = offset_reg;
819     strb_offset_mask_next = strb_offset_mask_reg;
820     last_cycle_offset_next = last_cycle_offset_reg;
821     ram_mask_next = ram_mask_reg;
822     ram_mask_valid_next = ram_mask_valid_reg;
823     cycle_count_next = cycle_count_reg;
824     last_cycle_next = last_cycle_reg;
825
826     mask_fifo_rd_ptr_next = mask_fifo_rd_ptr_reg;
827
828     op_table_tx_start_en = 1'b0;
829     op_table_tx_finish_en = 1'b0;
830
831     m_axi_awid_next = m_axi_awid_reg;
832     m_axi_awaddr_next = m_axi_awaddr_reg;
833     m_axi_awlen_next = m_axi_awlen_reg;
834     m_axi_awvalid_next = m_axi_awvalid_reg && !m_axi_awready;
835     m_axi_bready_next = 1'b0;
836
837     m_axi_wdata_int = ((IMM_ENABLE && axi_imm_en_reg) ?
{2{RAM_DATA_WIDTH{1'b0}} | axi_imm_reg} : {2{ram_rd_resp_data}})
>> (RAM_DATA_WIDTH-offset_reg*AXI_WORD_SIZE);
838     m_axi_wstrb_int = strb_offset_mask_reg;
839     m_axi_wlast_int = 1'b0;
840     m_axi_wvalid_int = 1'b0;
841
842     // read response processing and AXI write generation
843     case (axi_state_reg)
844         AXI_STATE_IDLE: begin
845             // idle state, wait for command
846             ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
847
848             axi_addr_next = op_t-
able_axi_addr[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
849             axi_imm_next = op_t-
able_imm[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
850             axi_imm_en_next = op_t-
able_imm_en[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
851             axi_len_next = op_t-
able_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
852             axi_zero_len_next = op_t-
able_zero_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
853             offset_next = op_t-
able_offset[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
854             strb_offset_mask_next = axi_zero_len_next ?
{AXI_STRB_WIDTH{1'b0}} : ({AXI_STRB_WIDTH{1'b1}} << (axi_addr_next &
OFFSET_MASK));
855             last_cycle_offset_next = axi_addr_next +

```

```

(axi_len_next & OFFSET_MASK);
856     cycle_count_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
857     last_cycle_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] == 0;
858
859     if (op_t-
able_active[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_tx_start_ptr_reg != op_table_start_ptr_reg && (!m_axi_aval-
id_reg || m_axi_awready)) begin
860         m_axi_awid_next = op_t-
able_tx_start_ptr_reg[OP_TAG_WIDTH-1:0];
861         m_axi_awaddr_next = axi_addr_next;
862         m_axi_awlen_next = cycle_count_next;
863         m_axi_awvalid_next = 1'b1;
864         op_table_tx_start_en = 1'b1;
865         axi_state_next = AXI_STATE_TRANSFER;
866     end else begin
867         axi_state_next = AXI_STATE_IDLE;
868     end
869 end
870 AXI_STATE_TRANSFER: begin
871     // transfer state, transfer data
872     ram_rd_resp_ready_cmb = {RAM_SEG_COUNT{1'b0}};
873
874     if (!(ram_mask_reg & ~ram_rd_resp_valid) &&
ram_mask_valid_reg && m_axi_wready_int) begin
875         // transfer in read data
876         ram_rd_resp_ready_cmb = ram_mask_reg;
877         ram_mask_valid_next = 1'b0;
878
879         // update counters
880         cycle_count_next = cycle_count_reg - 1;
881         last_cycle_next = cycle_count_next == 0;
882         offset_next = offset_reg + AXI_STRB_WIDTH;
883         strb_offset_mask_next =
{AXI_STRB_WIDTH{1'b1}};
884
885         m_axi_wdata_int = ((IMM_ENABLE &&
axi_imm_en_reg) ? {2{RAM_DATA_WIDTH{1'b0}} | axi_imm_reg} :
{2{ram_rd_resp_data}}) >> (RAM_DATA_WIDTH-offset_reg*AXI_WORD_SIZE);
886         m_axi_wstrb_int = strb_offset_mask_reg;
887         m_axi_wlast_int = 1'b0;
888         m_axi_wvalid_int = 1'b1;
889
890         if (last_cycle_reg) begin
891             // no more data to transfer, finish opera-
tion
892             m_axi_wlast_int = 1'b1;
893             op_table_tx_finish_en = 1'b1;
894
895             if (last_cycle_offset_reg) begin
896                 m_axi_wstrb_int = strb_offset_mask_reg &
{AXI_STRB_WIDTH{1'b1}} >> (AXI_STRB_WIDTH - last_cycle_offset_reg);
897             end
898
899             // skip idle state if possible
900             axi_addr_next = op_t-
able_axi_addr[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
901             axi_imm_next = op_t-
able_imm[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
902             axi_imm_en_next = op_t-
able_imm_en[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
903             axi_len_next = op_t-
able_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
904             axi_zero_len_next = op_t-
able_zero_len[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
905             offset_next = op_t-
able_offset[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
906             strb_offset_mask_next =
axi_zero_len_next ? {AXI_STRB_WIDTH{1'b0}} : ({AXI_STRB_WIDTH{1'b1}}
<< (axi_addr_next & OFFSET_MASK));
907             last_cycle_offset_next = axi_addr_next +
(axi_len_next & OFFSET_MASK);
908             cycle_count_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]];
909             last_cycle_next = op_t-
able_cycle_count[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] == 0;
910
911             if (op_t-
able_active[op_table_tx_start_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_tx_start_ptr_reg != op_table_start_ptr_reg && (!m_axi_aval-
id_reg || m_axi_awready)) begin
912                 m_axi_awid_next = op_t-
able_tx_start_ptr_reg[OP_TAG_WIDTH-1:0];
913                 m_axi_awaddr_next = axi_addr_next;
914                 m_axi_awlen_next = cycle_count_next;
915                 m_axi_awvalid_next = 1'b1;
916                 op_table_tx_start_en = 1'b1;
917                 axi_state_next = AXI_STATE_TRANSFER;
918             end else begin
919                 axi_state_next = AXI_STATE_IDLE;
920             end
921         end else begin
922             axi_state_next = AXI_STATE_TRANSFER;
923         end
924     end else begin
925         axi_state_next = AXI_STATE_TRANSFER;
926     end
927 end

```

```

928     endcase
929
930     if (!ram_mask_valid_next && !mask_fifo_empty) begin
931         ram_mask_next =
mask_fifo_mask[mask_fifo_rd_ptr_reg[MASK_FIFO_ADDR_WIDTH-1:0]];
932         ram_mask_valid_next = 1'b1;
933         mask_fifo_rd_ptr_next = mask_fifo_rd_ptr_reg+1;
934     end
935
936     op_table_write_complete_ptr = m_axi_bid;
937     if (m_axi_bresp == AXI_RESP_SLVERR) begin
938         op_table_write_complete_error = DMA_ER-
ROR_AXI_WR_SLVERR;
939     end else if (m_axi_bresp == AXI_RESP_DECERR) begin
940         op_table_write_complete_error = DMA_ER-
ROR_AXI_WR_DECERR;
941     end else begin
942         op_table_write_complete_error = DMA_ERROR_NONE;
943     end
944     op_table_write_complete_en = 1'b0;
945
946     m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
947     if (m_axis_write_desc_status_valid_reg) begin
948         m_axis_write_desc_status_error_next = DMA_ERROR_NONE;
949     end else begin
950         m_axis_write_desc_status_error_next = m_ax-
is_write_desc_status_error_reg;
951     end
952     m_axis_write_desc_status_valid_next = 1'b0;
953
954     stat_wr_req_finish_status_next = op_t-
able_write_complete_error;
955     stat_wr_req_finish_valid_next = 1'b0;
956
957     stat_wr_op_finish_tag_next = stat_wr_op_finish_tag_reg;
958     stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;
959     stat_wr_op_finish_valid_next = 1'b0;
960
961     if (USE_AXI_ID) begin
962         // accept write completions
963         stat_wr_req_finish_tag_next = m_axi_bid;
964
965         m_axi_bready_next = 1'b1;
966         if (m_axi_bready && m_axi_bvalid) begin
967             op_table_write_complete_ptr = m_axi_bid;
968             op_table_write_complete_en = 1'b1;
969             stat_wr_req_finish_valid_next = 1'b1;
970         end
971
972         // commit operations in-order
973         op_table_finish_en = 1'b0;
974         dec_active_op = 1'b0;
975
976         if (op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] && op_t-
able_write_complete[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] &&
op_table_finish_ptr_reg != op_table_tx_finish_ptr_reg) begin
977             op_table_finish_en = 1'b1;
978             dec_active_op = 1'b1;
979
980             if (op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] !=
DMA_ERROR_NONE) begin
981                 m_axis_write_desc_status_error_next = op_t-
able_error_code[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
982             end
983
984             stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;
985
986             if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
987                 m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
988                 m_axis_write_desc_status_valid_next = 1'b1;
989                 stat_wr_op_finish_tag_next = stat_wr_op_fin-
ish_tag_reg + 1;
990                 stat_wr_op_finish_valid_next = 1'b1;
991             end
992         end
993     end else begin
994         // accept write completions
995         op_table_finish_en = 1'b0;
996         dec_active_op = 1'b0;
997
998         stat_wr_req_finish_tag_next = op_t-
able_finish_ptr_reg[OP_TAG_WIDTH-1:0];
999
1000         m_axi_bready_next = 1'b1;
1001         if (m_axi_bready && m_axi_bvalid) begin
1002             op_table_finish_en = 1'b1;
1003             dec_active_op = 1'b1;
1004             stat_wr_req_finish_valid_next = 1'b1;
1005
1006             if (m_axi_bresp == AXI_RESP_SLVERR) begin
1007                 m_axis_write_desc_status_error_next = DMA_ER-
ROR_AXI_WR_SLVERR;
1008             end else if (m_axi_bresp == AXI_RESP_DECERR) begin
1009                 m_axis_write_desc_status_error_next = DMA_ER-

```

```

ROR_AXI_WR_DECERR;
1010     end
1011
1012     stat_wr_op_finish_status_next = m_ax-
is_write_desc_status_error_next;
1013
1014     if (op_t-
able_last[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]]) begin
1015         m_axis_write_desc_status_tag_next = op_t-
able_tag[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]];
1016         m_axis_write_desc_status_valid_next = 1'b1;
1017         stat_wr_op_finish_tag_next = stat_wr_op_fin-
ish_tag_reg + 1;
1018         stat_wr_op_finish_valid_next = 1'b1;
1019     end
1020 end
1021 end
1022 end
1023
1024 always @(posedge clk) begin
1025     req_state_reg <= req_state_next;
1026     read_state_reg <= read_state_next;
1027     axi_state_reg <= axi_state_next;
1028
1029     req_axi_addr_reg <= req_axi_addr_next;
1030     ram_sel_reg <= ram_sel_next;
1031     ram_addr_reg <= ram_addr_next;
1032     imm_reg <= imm_next;
1033     imm_en_reg <= imm_en_next;
1034     op_count_reg <= op_count_next;
1035     zero_len_reg <= zero_len_next;
1036     tr_count_reg <= tr_count_next;
1037     tr_word_count_reg <= tr_word_count_next;
1038     tag_reg <= tag_next;
1039
1040     read_axi_addr_reg <= read_axi_addr_next;
1041     read_ram_sel_reg <= read_ram_sel_next;
1042     read_ram_addr_reg <= read_ram_addr_next;
1043     read_imm_en_reg <= read_imm_en_next;
1044     read_len_reg <= read_len_next;
1045     read_ram_mask_reg <= read_ram_mask_next;
1046     read_ram_mask_0_reg <= read_ram_mask_0_next;
1047     read_ram_mask_1_reg <= read_ram_mask_1_next;
1048     ram_wrap_reg <= ram_wrap_next;
1049     read_cycle_count_reg <= read_cycle_count_next;
1050     read_last_cycle_reg <= read_last_cycle_next;
1051     cycle_byte_count_reg <= cycle_byte_count_next;
1052     start_offset_reg <= start_offset_next;
1053     end_offset_reg <= end_offset_next;
1054
1055     axi_addr_reg <= axi_addr_next;
1056     axi_imm_reg <= axi_imm_next;
1057     axi_imm_en_reg <= axi_imm_en_next;
1058     axi_len_reg <= axi_len_next;
1059     axi_zero_len_reg <= axi_zero_len_next;
1060     offset_reg <= offset_next;
1061     strb_offset_mask_reg <= strb_offset_mask_next;
1062     last_cycle_offset_reg <= last_cycle_offset_next;
1063     ram_mask_reg <= ram_mask_next;
1064     ram_mask_valid_reg <= ram_mask_valid_next;
1065     cycle_count_reg <= cycle_count_next;
1066     last_cycle_reg <= last_cycle_next;
1067
1068     read_cmd_axi_addr_reg <= read_cmd_axi_addr_next;
1069     read_cmd_ram_sel_reg <= read_cmd_ram_sel_next;
1070     read_cmd_ram_addr_reg <= read_cmd_ram_addr_next;
1071     read_cmd_imm_en_reg <= read_cmd_imm_en_next;
1072     read_cmd_len_reg <= read_cmd_len_next;
1073     read_cmd_cycle_count_reg <= read_cmd_cycle_count_next;
1074     read_cmd_last_cycle_reg <= read_cmd_last_cycle_next;
1075     read_cmd_valid_reg <= read_cmd_valid_next;
1076
1077     m_axi_awid_reg <= m_axi_awid_next;
1078     m_axi_awaddr_reg <= m_axi_awaddr_next;
1079     m_axi_awlen_reg <= m_axi_awlen_next;
1080     m_axi_awvalid_reg <= m_axi_awvalid_next;
1081     m_axi_bready_reg <= m_axi_bready_next;
1082
1083     s_axis_write_desc_ready_reg <= s_ax-
is_write_desc_ready_next;
1084
1085     m_axis_write_desc_status_tag_reg <= m_ax-
is_write_desc_status_tag_next;
1086     m_axis_write_desc_status_error_reg <= m_ax-
is_write_desc_status_error_next;
1087     m_axis_write_desc_status_valid_reg <= m_ax-
is_write_desc_status_valid_next;
1088
1089     status_busy_reg <= active_op_count_reg != 0;
1090
1091     stat_wr_op_start_tag_reg <= stat_wr_op_start_tag_next;
1092     stat_wr_op_start_len_reg <= stat_wr_op_start_len_next;
1093     stat_wr_op_start_valid_reg <= stat_wr_op_start_valid_next;
1094     stat_wr_op_finish_tag_reg <= stat_wr_op_finish_tag_next;
1095     stat_wr_op_finish_status_reg <= stat_wr_op_fin-
ish_status_next;
1096     stat_wr_op_finish_valid_reg <= stat_wr_op_fin-
ish_valid_next;
1097     stat_wr_req_start_tag_reg <= stat_wr_req_start_tag_next;
1098     stat_wr_req_start_len_reg <= stat_wr_req_start_len_next;
1099     stat_wr_req_start_valid_reg <=
stat_wr_req_start_valid_next;
1100     stat_wr_req_finish_tag_reg <= stat_wr_req_finish_tag_next;
1101     stat_wr_req_finish_status_reg <= stat_wr_req_fin-
ish_status_next;
1102     stat_wr_req_finish_valid_reg <= stat_wr_req_fin-
ish_valid_next;
1103     stat_wr_op_table_full_reg <= stat_wr_op_table_full_next;
1104     stat_wr_tx_stall_reg <= stat_wr_tx_stall_next;
1105
1106     ram_rd_cmd_sel_reg <= ram_rd_cmd_sel_next;
1107     ram_rd_cmd_addr_reg <= ram_rd_cmd_addr_next;
1108     ram_rd_cmd_valid_reg <= ram_rd_cmd_valid_next;
1109
1110     active_op_count_reg <= active_op_count_reg + inc_active_op
- dec_active_op;
1111
1112     if (mask_fifo_we) begin
1113         mask_fifo_mask[mask_fifo_wr_ptr_reg[MASK_FIFO_ADDR_WIDTH-1:0]] <=
mask_fifo_wr_mask;
1114         mask_fifo_wr_ptr_reg <= mask_fifo_wr_ptr_reg + 1;
1115     end
1116     mask_fifo_rd_ptr_reg <= mask_fifo_rd_ptr_next;
1117
1118     if (op_table_start_en) begin
1119         op_table_start_ptr_reg <= op_table_start_ptr_reg + 1;
1120         op_t-
able_active[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b1;
1121         op_t-
able_write_complete[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <=
1'b0;
1122         op_t-
able_axi_addr[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_axi_addr;
1123         op_table_imm[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_imm;
1124         op_t-
able_imm_en[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_imm_en;
1125         op_table_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_len;
1126         op_t-
able_zero_len[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_zero_len;
1127         op_t-
able_cycle_count[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_cycle_count;
1128         op_t-
able_offset[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_offset;
1129         op_table_tag[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]]
<= op_table_start_tag;
1130         op_t-
able_last[op_table_start_ptr_reg[OP_TAG_WIDTH-1:0]] <= op_t-
able_start_last;
1131     end
1132
1133     if (op_table_tx_start_en) begin
1134         op_table_tx_start_ptr_reg <= op_table_tx_start_ptr_reg
+ 1;
1135     end
1136
1137     if (op_table_tx_finish_en) begin
1138         op_table_tx_finish_ptr_reg <= op_t-
able_tx_finish_ptr_reg + 1;
1139     end
1140
1141     if (USE_AXI_ID && op_table_write_complete_en) begin
1142         op_table_write_complete[op_table_write_complete_ptr]
<= 1'b1;
1143         op_table_error_code[op_table_write_complete_ptr] <=
op_table_write_complete_error;
1144     end
1145
1146     if (op_table_finish_en) begin
1147         op_table_finish_ptr_reg <= op_table_finish_ptr_reg +
1;
1148         op_t-
able_active[op_table_finish_ptr_reg[OP_TAG_WIDTH-1:0]] <= 1'b0;
1149     end
1150
1151     if (rst) begin
1152         req_state_reg <= REQ_STATE_IDLE;
1153         read_state_reg <= READ_STATE_IDLE;
1154         axi_state_reg <= AXI_STATE_IDLE;
1155
1156         read_cmd_valid_reg <= 1'b0;
1157
1158         ram_mask_valid_reg <= 1'b0;
1159
1160         m_axi_awvalid_reg <= 1'b0;
1161         m_axi_bready_reg <= 1'b0;
1162
1163         s_axis_write_desc_ready_reg <= 1'b0;
1164         m_axis_write_desc_status_error_reg <= 4'd0;
1165         m_axis_write_desc_status_valid_reg <= 1'b0;

```

```

1166
1167     status_busy_reg <= 1'b0;
1168
1169     stat_wr_op_start_tag_reg <= 0;
1170     stat_wr_op_start_valid_reg <= 1'b0;
1171     stat_wr_op_finish_tag_reg <= 0;
1172     stat_wr_op_finish_valid_reg <= 1'b0;
1173     stat_wr_req_start_valid_reg <= 1'b0;
1174     stat_wr_req_finish_valid_reg <= 1'b0;
1175     stat_wr_op_table_full_reg <= 1'b0;
1176     stat_wr_tx_stall_reg <= 1'b0;
1177
1178     ram_rd_cmd_valid_reg <= {RAM_SEG_COUNT{1'b0}};
1179
1180     active_op_count_reg <= 0;
1181
1182     mask_fifo_wr_ptr_reg <= 0;
1183     mask_fifo_rd_ptr_reg <= 0;
1184
1185     op_table_start_ptr_reg <= 0;
1186     op_table_tx_start_ptr_reg <= 0;
1187     op_table_tx_finish_ptr_reg <= 0;
1188     op_table_finish_ptr_reg <= 0;
1189     op_table_active <= 0;
1190 end
1191 end
1192
1193 // output datapath logic
1194 reg [AXI_DATA_WIDTH-1:0] m_axi_wdata_reg =
{AXI_DATA_WIDTH{1'b0}};
1195 reg [AXI_STRB_WIDTH-1:0] m_axi_wstrb_reg =
{AXI_STRB_WIDTH{1'b0}};
1196 reg m_axi_wlast_reg = 1'b0;
1197 reg m_axi_wvalid_reg = 1'b0;
1198
1199 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_wr_ptr_reg = 0;
1200 reg [OUTPUT_FIFO_ADDR_WIDTH+1-1:0] out_fifo_rd_ptr_reg = 0;
1201 reg out_fifo_half_full_reg = 1'b0;
1202
1203 wire out_fifo_full = out_fifo_wr_ptr_reg ==
(out_fifo_rd_ptr_reg ^ {1'b1, {OUTPUT_FIFO_ADDR_WIDTH{1'b0}}});
1204 wire out_fifo_empty = out_fifo_wr_ptr_reg ==
out_fifo_rd_ptr_reg;
1205
1206 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1207 reg [AXI_DATA_WIDTH-1:0]
out_fifo_wdata[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1208 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1209 reg [AXI_STRB_WIDTH-1:0]
out_fifo_wstrb[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1210 (* ram_style = "distributed", ramstyle = "no_rw_check, mlab"
*)
1211 reg
out_fifo_wlast[2**OUTPUT_FIFO_ADDR_WIDTH-1:0];
1212
1213 assign m_axi_wready_int = !out_fifo_half_full_reg;
1214
1215 assign m_axi_wdata = m_axi_wdata_reg;
1216 assign m_axi_wstrb = m_axi_wstrb_reg;
1217 assign m_axi_wvalid = m_axi_wvalid_reg;
1218 assign m_axi_wlast = m_axi_wlast_reg;
1219
1220 always @(posedge clk) begin
1221     m_axi_wvalid_reg <= m_axi_wvalid_reg && !m_axi_wready;
1222
1223     out_fifo_half_full_reg <= $unsigned(out_fifo_wr_ptr_reg -
out_fifo_rd_ptr_reg) >= 2** (OUTPUT_FIFO_ADDR_WIDTH-1);
1224
1225     if (!out_fifo_full && m_axi_wvalid_int) begin
1226         out_fifo_wdata[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wdata_int;
1227         out_fifo_wstrb[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wstrb_int;
1228         out_fifo_wlast[out_fifo_wr_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]] <=
m_axi_wlast_int;
1229         out_fifo_wr_ptr_reg <= out_fifo_wr_ptr_reg + 1;
1230     end
1231
1232     if (!out_fifo_empty && (!m_axi_wvalid_reg ||
m_axi_wready)) begin
1233         m_axi_wdata_reg <=
out_fifo_wdata[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1234         m_axi_wstrb_reg <=
out_fifo_wstrb[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1235         m_axi_wlast_reg <=
out_fifo_wlast[out_fifo_rd_ptr_reg[OUTPUT_FIFO_ADDR_WIDTH-1:0]];
1236         m_axi_wvalid_reg <= 1'b1;
1237         out_fifo_rd_ptr_reg <= out_fifo_rd_ptr_reg + 1;
1238     end
1239
1240     if (rst) begin
1241         out_fifo_wr_ptr_reg <= 0;
1242         out_fifo_rd_ptr_reg <= 0;
1243         m_axi_wvalid_reg <= 1'b0;
1244     end
1245 end

```

```

1246
1247 endmodule
1248
1249 `resetall

```

8. alexforench_verilog-pcie/rtl/dma_if_desc_mux.v (365 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forench
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32 * DMA interface descriptor mux
33 */
34 module dma_if_desc_mux #
35 (
36     // Number of ports
37     parameter PORTS = 2,
38     // Extend RAM select signal
39     parameter EXTEND_RAM_SEL = 0,
40     // Input RAM segment select width
41     parameter S_RAM_SEL_WIDTH = 2,
42     // Output RAM segment select width
43     // Additional bits required for response routing
44     parameter M_RAM_SEL_WIDTH = S_RAM_SEL_WIDTH+(EX-
TEND_RAM_SEL ? $clog2(PORTS) : 0),
45     // RAM address width
46     parameter RAM_ADDR_WIDTH = 16,
47     // DMA address width
48     parameter DMA_ADDR_WIDTH = 64,
49     // Immediate enable
50     parameter IMM_ENABLE = 0,
51     // Immediate width
52     parameter IMM_WIDTH = 32,
53     // Length field width
54     parameter LEN_WIDTH = 16,
55     // Input tag field width
56     parameter S_TAG_WIDTH = 8,
57     // Output tag field width (towards DMA interface module)
58     // Additional bits required for response routing
59     parameter M_TAG_WIDTH = S_TAG_WIDTH+$clog2(PORTS),
60     // select round robin arbitration
61     parameter ARB_TYPE_ROUND_ROBIN = 0,
62     // LSB priority selection
63     parameter ARB_LSB_HIGH_PRIORITY = 1
64 )
65 (
66     input wire clk,
67     input wire rst,
68
69     /*
70      * Read descriptor output (to DMA interface)
71      */
72     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_desc_dma_addr,
73     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_desc_ram_sel,
74     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_desc_ram_addr,
75     output wire [IMM_WIDTH-1:0] m_ax-
is_desc_imm,

```



```

76 output wire m_ax-
is_desc_imm_en, m_ax-
77 output wire [LEN_WIDTH-1:0] m_ax-
is_desc_len, m_ax-
78 output wire [M_TAG_WIDTH-1:0] m_ax-
is_desc_tag, m_ax-
79 output wire m_ax-
is_desc_valid, m_ax-
80 input wire m_ax-
is_desc_ready,
81
82 /*
83 * Read descriptor status input (from DMA interface)
84 */
85 input wire [M_TAG_WIDTH-1:0] s_ax-
is_desc_status_tag, s_ax-
86 input wire [3:0] s_ax-
is_desc_status_error, s_ax-
87 input wire s_ax-
is_desc_status_valid,
88
89 /*
90 * Read descriptor input
91 */
92 input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_desc_dma_addr, s_ax-
93 input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_desc_ram_sel, s_ax-
94 input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_desc_ram_addr, s_ax-
95 input wire [PORTS*IMM_WIDTH-1:0] s_ax-
is_desc_imm, s_ax-
96 input wire [PORTS:1:0] s_ax-
is_desc_imm_en, s_ax-
97 input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_desc_len, s_ax-
98 input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_desc_tag, s_ax-
99 input wire [PORTS:1:0] s_ax-
is_desc_valid, s_ax-
100 output wire [PORTS:1:0] s_ax-
is_desc_ready,
101
102 /*
103 * Read descriptor status output
104 */
105 output wire [PORTS*S_TAG_WIDTH-1:0] m_ax-
is_desc_status_tag, m_ax-
106 output wire [PORTS*4-1:0] m_ax-
is_desc_status_error, m_ax-
107 output wire [PORTS:1:0] m_ax-
is_desc_status_valid
108 );
109
110 parameter CL_PORTS = $clog2(PORTS);
111
112 parameter S_RAM_SEL_WIDTH_INT = S_RAM_SEL_WIDTH > 0 ?
S_RAM_SEL_WIDTH : 1;
113
114 parameter FIFO_ADDR_WIDTH = 5;
115
116 // check configuration
117 initial begin
118 if (M_TAG_WIDTH < S_TAG_WIDTH+$clog2(PORTS)) begin
119 $error("Error: M_TAG_WIDTH must be at least $clog-
g2(PORTS) larger than S_TAG_WIDTH (instance %m)");
120 $finish;
121 end
122
123 if (EXTEND_RAM_SEL) begin
124 if (M_RAM_SEL_WIDTH < S_RAM_SEL_WIDTH+$clog2(PORTS))
begin
125 $error("Error: M_RAM_SEL_WIDTH must be at least
$clog2(PORTS) larger than S_RAM_SEL_WIDTH (instance %m)");
126 $finish;
127 end
128 end else begin
129 if (M_RAM_SEL_WIDTH < S_RAM_SEL_WIDTH) begin
130 $error("Error: M_RAM_SEL_WIDTH must be no smaller
than S_RAM_SEL_WIDTH (instance %m)");
131 $finish;
132 end
133 end
134 end
135
136 // descriptor mux
137 wire [PORTS-1:0] request;
138 wire [PORTS-1:0] acknowledge;
139 wire [PORTS-1:0] grant;
140 wire grant_valid;
141 wire [CL_PORTS-1:0] grant_encoded;
142
143 // input registers to pipeline arbitration delay
144 reg [PORTS*DMA_ADDR_WIDTH-1:0] s_axis_desc_dma_addr_reg = 0;
145 reg [PORTS*S_RAM_SEL_WIDTH-1:0] s_axis_desc_ram_sel_reg = 0;
146 reg [PORTS*RAM_ADDR_WIDTH-1:0] s_axis_desc_ram_addr_reg = 0;
147 reg [PORTS*IMM_WIDTH-1:0] s_axis_desc_imm_reg = 0;
148 reg [PORTS:1:0] s_axis_desc_imm_en_reg = 0;
149 reg [PORTS*LEN_WIDTH-1:0] s_axis_desc_len_reg = 0;
150 reg [PORTS*S_TAG_WIDTH-1:0] s_axis_desc_tag_reg = 0;
151 reg [PORTS:1:0] s_axis_desc_valid_reg = 0;

```

```

152
153 // internal datapath
154 reg [DMA_ADDR_WIDTH-1:0] m_axis_desc_dma_addr_int;
155 reg [M_RAM_SEL_WIDTH-1:0] m_axis_desc_ram_sel_int;
156 reg [RAM_ADDR_WIDTH-1:0] m_axis_desc_ram_addr_int;
157 reg [IMM_WIDTH-1:0] m_axis_desc_imm_int;
158 reg m_axis_desc_imm_en_int;
159 reg [LEN_WIDTH-1:0] m_axis_desc_len_int;
160 reg [M_TAG_WIDTH-1:0] m_axis_desc_tag_int;
161 reg m_axis_desc_valid_int;
162 reg m_axis_desc_ready_int_reg = 1'b0;
163 wire m_axis_desc_ready_int_early;
164
165 assign s_axis_desc_ready = ~s_axis_desc_valid_reg |
({PORTS{m_axis_desc_ready_int_reg}} & grant);
166
167 // mux for incoming packet
168 wire [DMA_ADDR_WIDTH-1:0] current_s_desc_dma_addr = s_ax-
is_desc_dma_addr_reg[grant_encoded*DMA_ADDR_WIDTH +: DMA_AD-
DR_WIDTH];
169 wire [S_RAM_SEL_WIDTH-1:0] current_s_desc_ram_sel = s_ax-
is_desc_ram_sel_reg[grant_encoded*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT];
170 wire [RAM_ADDR_WIDTH-1:0] current_s_desc_ram_addr = s_ax-
is_desc_ram_addr_reg[grant_encoded*RAM_ADDR_WIDTH +: RAM_AD-
DR_WIDTH];
171 wire [IMM_WIDTH-1:0] current_s_desc_imm = s_ax-
is_desc_imm_reg[grant_encoded*IMM_WIDTH +: IMM_WIDTH];
172 wire current_s_desc_imm_en = s_ax-
is_desc_imm_en_reg[grant_encoded];
173 wire [LEN_WIDTH-1:0] current_s_desc_len = s_ax-
is_desc_len_reg[grant_encoded*LEN_WIDTH +: LEN_WIDTH];
174 wire [S_TAG_WIDTH-1:0] current_s_desc_tag = s_ax-
is_desc_tag_reg[grant_encoded*S_TAG_WIDTH +: S_TAG_WIDTH];
175 wire current_s_desc_valid = s_ax-
is_desc_valid_reg[grant_encoded];
176 wire current_s_desc_ready = s_ax-
is_desc_ready[grant_encoded];
177
178 // arbiter instance
179 arbiter #(
180 .PORTS(PORTS),
181 .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
182 .ARB_BLOCK(1),
183 .ARB_BLOCK_ACK(1),
184 .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
185 )
186 arb_inst (
187 .clk(clk),
188 .rst(rst),
189 .request(request),
190 .acknowledge(acknowledge),
191 .grant(grant),
192 .grant_valid(grant_valid),
193 .grant_encoded(grant_encoded)
194 );
195
196 assign request = (s_axis_desc_valid_reg & ~grant) | (s_ax-
is_desc_valid & grant);
197 assign acknowledge = grant & s_axis_desc_valid_reg &
{PORTS{m_axis_desc_ready_int_reg}};
198
199 always @* begin
200 // pass through selected packet data
201 m_axis_desc_dma_addr_int = current_s_desc_dma_addr;
202 m_axis_desc_ram_sel_int = current_s_desc_ram_sel;
203 if (EXTEND_RAM_SEL && PORTS > 1) begin
204 m_ax-
is_desc_ram_sel_int[M_RAM_SEL_WIDTH-1:M_RAM_SEL_WIDTH-CL_PORTS] =
grant_encoded;
205 end
206 m_axis_desc_ram_addr_int = current_s_desc_ram_addr;
207 m_axis_desc_imm_int = current_s_desc_imm;
208 m_axis_desc_imm_en_int = current_s_desc_imm_en;
209 m_axis_desc_len_int = current_s_desc_len;
210 m_axis_desc_tag_int = current_s_desc_tag;
211 if (PORTS > 1) begin
212 m_axis_desc_tag_int[M_TAG_WIDTH-1:M_TAG_WIDTH-
CL_PORTS] = grant_encoded;
213 end
214 m_axis_desc_valid_int = current_s_desc_valid && m_ax-
is_desc_ready_int_reg && grant_valid;
215 end
216
217 integer i;
218
219 always @(posedge clk) begin
220 // register inputs
221 for (i = 0; i < PORTS; i = i + 1) begin
222 if (s_axis_desc_ready[i]) begin
223 s_axis_desc_dma_addr_reg[i*DMA_ADDR_WIDTH +:
DMA_ADDR_WIDTH] <= s_axis_desc_dma_addr[i*DMA_ADDR_WIDTH +: DMA_AD-
DR_WIDTH];
224 s_axis_desc_ram_sel_reg[i*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT] <= s_axis_desc_ram_sel[i*S_RAM_SEL_WIDTH +:
S_RAM_SEL_WIDTH_INT];
225 s_axis_desc_ram_addr_reg[i*RAM_ADDR_WIDTH +:
RAM_ADDR_WIDTH] <= s_axis_desc_ram_addr[i*RAM_ADDR_WIDTH +: RAM_AD-
DR_WIDTH];
226 s_axis_desc_imm_reg[i*IMM_WIDTH +: IMM_WIDTH] <=
s_axis_desc_imm[i*IMM_WIDTH +: IMM_WIDTH];

```

```

227     s_axis_desc_imm_en_reg[i] <= s_ax-
is_desc_imm_en[i];
228     s_axis_desc_len_reg[i*LEN_WIDTH +: LEN_WIDTH] <=
s_axis_desc_len[i*LEN_WIDTH +: LEN_WIDTH];
229     s_axis_desc_tag_reg[i*S_TAG_WIDTH +: S_TAG_WIDTH]
<= s_axis_desc_tag[i*S_TAG_WIDTH +: S_TAG_WIDTH];
230     s_axis_desc_valid_reg[i] <= s_axis_desc_valid[i];
231 end
232 end
233
234 if (rst) begin
235     s_axis_desc_valid_reg <= 0;
236 end
237 end
238
239 // output datapath logic
240 reg [DMA_ADDR_WIDTH-1:0] m_axis_desc_dma_addr_reg = {DMA_AD-
DR_WIDTH{1'b0}};
241 reg [M_RAM_SEL_WIDTH-1:0] m_axis_desc_ram_sel_reg =
{M_RAM_SEL_WIDTH{1'b0}};
242 reg [RAM_ADDR_WIDTH-1:0] m_axis_desc_ram_addr_reg = {RAM_AD-
DR_WIDTH{1'b0}};
243 reg [IMM_WIDTH-1:0] m_axis_desc_imm_reg =
{IMM_WIDTH{1'b0}};
244 reg [LEN_WIDTH-1:0] m_axis_desc_imm_en_reg = 1'b0;
245 reg [LEN_WIDTH-1:0] m_axis_desc_len_reg =
{LEN_WIDTH{1'b0}};
246 reg [M_TAG_WIDTH-1:0] m_axis_desc_tag_reg =
{M_TAG_WIDTH{1'b0}};
247 reg [M_TAG_WIDTH-1:0] m_axis_desc_valid_reg = 1'b0,
m_axis_desc_valid_next;
248
249 reg [DMA_ADDR_WIDTH-1:0] temp_m_axis_desc_dma_addr_reg =
{DMA_ADDR_WIDTH{1'b0}};
250 reg [M_RAM_SEL_WIDTH-1:0] temp_m_axis_desc_ram_sel_reg =
{M_RAM_SEL_WIDTH{1'b0}};
251 reg [RAM_ADDR_WIDTH-1:0] temp_m_axis_desc_ram_addr_reg =
{RAM_ADDR_WIDTH{1'b0}};
252 reg [IMM_WIDTH-1:0] temp_m_axis_desc_imm_reg =
{IMM_WIDTH{1'b0}};
253 reg [IMM_WIDTH-1:0] temp_m_axis_desc_imm_en_reg =
1'b0;
254 reg [LEN_WIDTH-1:0] temp_m_axis_desc_len_reg =
{LEN_WIDTH{1'b0}};
255 reg [M_TAG_WIDTH-1:0] temp_m_axis_desc_tag_reg =
{M_TAG_WIDTH{1'b0}};
256 reg [M_TAG_WIDTH-1:0] temp_m_axis_desc_valid_reg =
1'b0, temp_m_axis_desc_valid_next;
257
258 // datapath control
259 reg store_axis_int_to_output;
260 reg store_axis_int_to_temp;
261 reg store_axis_temp_to_output;
262
263 assign m_axis_desc_dma_addr = m_axis_desc_dma_addr_reg;
264 assign m_axis_desc_ram_sel = m_axis_desc_ram_sel_reg;
265 assign m_axis_desc_ram_addr = m_axis_desc_ram_addr_reg;
266 assign m_axis_desc_imm = IMM_ENABLE ? m_ax-
is_desc_imm_reg : {IMM_WIDTH{1'b0}};
267 assign m_axis_desc_imm_en = IMM_ENABLE ? m_ax-
is_desc_imm_en_reg : 1'b0;
268 assign m_axis_desc_len = m_axis_desc_len_reg;
269 assign m_axis_desc_tag = m_axis_desc_tag_reg;
270 assign m_axis_desc_valid = m_axis_desc_valid_reg;
271
272 // enable ready input next cycle if output is ready or if both
output registers are empty
273 assign m_axis_desc_ready_int_early = m_axis_desc_ready || (!
temp_m_axis_desc_valid_reg && !m_axis_desc_valid_reg);
274
275 always @* begin
276     // transfer sink ready state to source
277     m_axis_desc_valid_next = m_axis_desc_valid_reg;
278     temp_m_axis_desc_valid_next = temp_m_axis_desc_valid_reg;
279
280     store_axis_int_to_output = 1'b0;
281     store_axis_int_to_temp = 1'b0;
282     store_axis_temp_to_output = 1'b0;
283
284     if (m_axis_desc_ready_int_reg) begin
285         // input is ready
286         if (m_axis_desc_ready || !m_axis_desc_valid_reg) begin
287             // output is ready or currently not valid, trans-
fer data to output
288             m_axis_desc_valid_next = m_axis_desc_valid_int;
289             store_axis_int_to_output = 1'b1;
290         end else begin
291             // output is not ready, store input in temp
292             temp_m_axis_desc_valid_next = m_ax-
is_desc_valid_int;
293             store_axis_int_to_temp = 1'b1;
294         end else if (m_axis_desc_ready) begin
295             // input is not ready, but output is ready
296             m_axis_desc_valid_next = temp_m_axis_desc_valid_reg;
297             temp_m_axis_desc_valid_next = 1'b0;
298             store_axis_temp_to_output = 1'b1;
299         end
300     end
301 end
302

```

```

303 always @(posedge clk) begin
304     m_axis_desc_valid_reg <= m_axis_desc_valid_next;
305     m_axis_desc_ready_int_reg <= m_axis_desc_ready_int_early;
306     temp_m_axis_desc_valid_reg <= temp_m_axis_desc_valid_next;
307
308     // datapath
309     if (store_axis_int_to_output) begin
310         m_axis_desc_dma_addr_reg <= m_axis_desc_dma_addr_int;
311         m_axis_desc_ram_sel_reg <= m_axis_desc_ram_sel_int;
312         m_axis_desc_ram_addr_reg <= m_axis_desc_ram_addr_int;
313         m_axis_desc_imm_reg <= m_axis_desc_imm_int;
314         m_axis_desc_imm_en_reg <= m_axis_desc_imm_en_int;
315         m_axis_desc_len_reg <= m_axis_desc_len_int;
316         m_axis_desc_tag_reg <= m_axis_desc_tag_int;
317     end else if (store_axis_temp_to_output) begin
318         m_axis_desc_dma_addr_reg <= temp_m_ax-
is_desc_dma_addr_reg;
319         m_axis_desc_ram_sel_reg <= temp_m_ax-
is_desc_ram_sel_reg;
320         m_axis_desc_ram_addr_reg <= temp_m_ax-
is_desc_ram_addr_reg;
321         m_axis_desc_imm_reg <= temp_m_axis_desc_imm_reg;
322         m_axis_desc_imm_en_reg <= temp_m_axis_desc_imm_en_reg;
323         m_axis_desc_len_reg <= temp_m_axis_desc_len_reg;
324         m_axis_desc_tag_reg <= temp_m_axis_desc_tag_reg;
325     end
326
327     if (store_axis_int_to_temp) begin
328         temp_m_axis_desc_dma_addr_reg <= m_ax-
is_desc_dma_addr_int;
329         temp_m_axis_desc_ram_sel_reg <= m_ax-
is_desc_ram_sel_int;
330         temp_m_axis_desc_ram_addr_reg <= m_ax-
is_desc_ram_addr_int;
331         temp_m_axis_desc_imm_reg <= m_axis_desc_imm_int;
332         temp_m_axis_desc_imm_en_reg <= m_axis_desc_imm_en_int;
333         temp_m_axis_desc_len_reg <= m_axis_desc_len_int;
334         temp_m_axis_desc_tag_reg <= m_axis_desc_tag_int;
335     end
336
337     if (rst) begin
338         m_axis_desc_valid_reg <= 1'b0;
339         m_axis_desc_ready_int_reg <= 1'b0;
340         temp_m_axis_desc_valid_reg <= 1'b0;
341     end
342 end
343
344 // descriptor status demux
345 reg [S_TAG_WIDTH-1:0] m_axis_desc_status_tag_reg =
{S_TAG_WIDTH{1'b0}};
346 reg [3:0] m_axis_desc_status_error_reg = 4'd0;
347 reg [PORTS-1:0] m_axis_desc_status_valid_reg = {PORTS{1'b0}};
348
349 assign m_axis_desc_status_tag =
{PORTS{m_axis_desc_status_tag_reg}};
350 assign m_axis_desc_status_error =
{PORTS{m_axis_desc_status_error_reg}};
351 assign m_axis_desc_status_valid = m_ax-
is_desc_status_valid_reg;
352
353 always @(posedge clk) begin
354     m_axis_desc_status_tag_reg <= s_axis_desc_status_tag;
355     m_axis_desc_status_error_reg <= s_axis_desc_status_error;
356     m_axis_desc_status_valid_reg <= s_axis_desc_status_valid
<< (PORTS > 1 ? s_axis_desc_status_tag[M_TAG_WIDTH-1:M_TAG_WIDTH-
CL_PORTS] : 0);
357
358     if (rst) begin
359         m_axis_desc_status_valid_reg <= {PORTS{1'b0}};
360     end
361 end
362
363 endmodule
364
365 `resetall

```

9. alexforenrich_verilog-pcie/rtl/dma_if_mux.v (354 lines)

```

1 /*
2
3 Copyright (c) 2019-2021 Alex Forenrich
4
5 Permission is hereby granted, free of charge, to any person
obtaining a copy
6 of this software and associated documentation files (the
"Software"), to deal
7 in the Software without restriction, including without limita-
tion the rights
8 to use, copy, modify, merge, publish, distribute, sublicense,
and/or sell
9 copies of the Software, and to permit persons to whom the
Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be
included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY

```



```

KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MER-
CHANTABILITY
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHER-
WISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
21 THE SOFTWARE.
22
23 */
24
25 // Language: Verilog 2001
26
27 `resetall
28 `timescale 1ns / 1ps
29 `default_nettype none
30
31 /*
32  * DMA interface mux
33  */
34 module dma_if_mux #
35 (
36     // Number of ports
37     parameter PORTS = 2,
38     // RAM segment count
39     parameter SEG_COUNT = 2,
40     // RAM segment data width
41     parameter SEG_DATA_WIDTH = 64,
42     // RAM segment address width
43     parameter SEG_ADDR_WIDTH = 8,
44     // RAM segment byte enable width
45     parameter SEG_BE_WIDTH = SEG_DATA_WIDTH/8,
46     // Input RAM segment select width
47     parameter S_RAM_SEL_WIDTH = 2,
48     // Output RAM segment select width
49     // Additional bits required for response routing
50     parameter M_RAM_SEL_WIDTH = S_RAM_SEL_WIDTH+$clog2(PORTS),
51     // RAM address width
52     parameter RAM_ADDR_WIDTH = SEG_ADDR_WIDTH+$clog2-
g2(SEG_COUNT)+$clog2(SEG_BE_WIDTH),
53     // DMA address width
54     parameter DMA_ADDR_WIDTH = 64,
55     // Immediate enable
56     parameter IMM_ENABLE = 0,
57     // Immediate width
58     parameter IMM_WIDTH = 32,
59     // Length field width
60     parameter LEN_WIDTH = 16,
61     // Input tag field width
62     parameter S_TAG_WIDTH = 8,
63     // Output tag field width (towards DMA module)
64     // Additional bits required for response routing
65     parameter M_TAG_WIDTH = S_TAG_WIDTH+$clog2(PORTS),
66     // select round robin arbitration
67     parameter ARB_TYPE_ROUND_ROBIN = 0,
68     // LSB priority selection
69     parameter ARB_LSB_HIGH_PRIORITY = 1
70 )
71 (
72     input wire clk,
73     input wire rst,
74
75     /*
76      * Read descriptor output (to DMA interface)
77      */
78     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_read_desc_dma_addr,
79     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_read_desc_ram_sel,
80     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_read_desc_ram_addr,
81     output wire [LEN_WIDTH-1:0] m_ax-
is_read_desc_len,
82     output wire [M_TAG_WIDTH-1:0] m_ax-
is_read_desc_tag,
83     output wire m_ax-
is_read_desc_valid,
84     input wire m_ax-
is_read_desc_ready,
85
86     /*
87      * Read descriptor status input (from DMA interface)
88      */
89     input wire [M_TAG_WIDTH-1:0] s_ax-
is_read_desc_status_tag,
90     input wire [3:0] s_ax-
is_read_desc_status_error,
91     input wire s_ax-
is_read_desc_status_valid,
92
93     /*
94      * Write descriptor output (to DMA interface)
95      */
96     output wire [DMA_ADDR_WIDTH-1:0] m_ax-
is_write_desc_dma_addr,
97     output wire [M_RAM_SEL_WIDTH-1:0] m_ax-
is_write_desc_ram_sel,

```

```

98     output wire [RAM_ADDR_WIDTH-1:0] m_ax-
is_write_desc_ram_addr,
99     output wire [IMM_WIDTH-1:0] m_ax-
is_write_desc_imm,
100    output wire m_ax-
is_write_desc_imm_en,
101    output wire [LEN_WIDTH-1:0] m_ax-
is_write_desc_len,
102    output wire [M_TAG_WIDTH-1:0] m_ax-
is_write_desc_tag,
103    output wire m_ax-
is_write_desc_valid,
104    input wire m_ax-
is_write_desc_ready,
105
106    /*
107     * Write descriptor status input (from DMA interface)
108     */
109    input wire [M_TAG_WIDTH-1:0] s_ax-
is_write_desc_status_tag,
110    input wire [3:0] s_ax-
is_write_desc_status_error,
111    input wire s_ax-
is_write_desc_status_valid,
112
113    /*
114     * Read descriptor input
115     */
116    input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_read_desc_dma_addr,
117    input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_read_desc_ram_sel,
118    input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_read_desc_ram_addr,
119    input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_read_desc_len,
120    input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_read_desc_tag,
121    input wire [PORTS-1:0] s_ax-
is_read_desc_valid,
122    output wire [PORTS-1:0] s_ax-
is_read_desc_ready,
123
124    /*
125     * Read descriptor status output
126     */
127    output wire [PORTS*S_TAG_WIDTH-1:0] m_ax-
is_read_desc_status_tag,
128    output wire [PORTS*4-1:0] m_ax-
is_read_desc_status_error,
129    output wire [PORTS-1:0] m_ax-
is_read_desc_status_valid,
130
131    /*
132     * Write descriptor input
133     */
134    input wire [PORTS*DMA_ADDR_WIDTH-1:0] s_ax-
is_write_desc_dma_addr,
135    input wire [PORTS*S_RAM_SEL_WIDTH-1:0] s_ax-
is_write_desc_ram_sel,
136    input wire [PORTS*RAM_ADDR_WIDTH-1:0] s_ax-
is_write_desc_ram_addr,
137    input wire [PORTS*IMM_WIDTH-1:0] s_ax-
is_write_desc_imm,
138    input wire [PORTS-1:0] s_ax-
is_write_desc_imm_en,
139    input wire [PORTS*LEN_WIDTH-1:0] s_ax-
is_write_desc_len,
140    input wire [PORTS*S_TAG_WIDTH-1:0] s_ax-
is_write_desc_tag,
141    input wire [PORTS-1:0] s_ax-
is_write_desc_valid,
142    output wire [PORTS-1:0] s_ax-
is_write_desc_ready,
143
144    /*
145     * Write descriptor status output
146     */
147    output wire [PORTS*S_TAG_WIDTH-1:0] m_ax-
is_write_desc_status_tag,
148    output wire [PORTS*4-1:0] m_ax-
is_write_desc_status_error,
149    output wire [PORTS-1:0] m_ax-
is_write_desc_status_valid,
150
151    /*
152     * RAM interface (from DMA interface)
153     */
154    input wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_wr_cmd_sel,
155    input wire [SEG_COUNT*SEG_BE_WIDTH-1:0]
if_ram_wr_cmd_be,
156    input wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_wr_cmd_addr,
157    input wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_wr_cmd_data,
158    input wire [SEG_COUNT-1:0]
if_ram_wr_cmd_valid,
159    output wire [SEG_COUNT-1:0]
if_ram_wr_cmd_ready,
160    output wire [SEG_COUNT-1:0]

```

```

if_ram_wr_done,
161 input wire [SEG_COUNT*M_RAM_SEL_WIDTH-1:0]
if_ram_rd_cmd_sel,
162 input wire [SEG_COUNT*SEG_ADDR_WIDTH-1:0]
if_ram_rd_cmd_addr,
163 input wire [SEG_COUNT-1:0]
if_ram_rd_cmd_valid,
164 output wire [SEG_COUNT-1:0]
if_ram_rd_cmd_ready,
165 output wire [SEG_COUNT*SEG_DATA_WIDTH-1:0]
if_ram_rd_resp_data,
166 output wire [SEG_COUNT-1:0]
if_ram_rd_resp_valid,
167 input wire [SEG_COUNT-1:0]
if_ram_rd_resp_ready,
168
169 /*
170 * RAM interface (towards RAM)
171 */
172 output wire [PORTS*SEG_COUNT*S_RAM_SEL_WIDTH-1:0]
ram_wr_cmd_sel,
173 output wire [PORTS*SEG_COUNT*SEG_BE_WIDTH-1:0]
ram_wr_cmd_be,
174 output wire [PORTS*SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_wr_cmd_addr,
175 output wire [PORTS*SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_wr_cmd_data,
176 output wire [PORTS*SEG_COUNT-1:0]
ram_wr_cmd_valid,
177 input wire [PORTS*SEG_COUNT-1:0]
ram_wr_cmd_ready,
178 input wire [PORTS*SEG_COUNT-1:0]
ram_wr_done,
179 output wire [PORTS*SEG_COUNT*S_RAM_SEL_WIDTH-1:0]
ram_rd_cmd_sel,
180 output wire [PORTS*SEG_COUNT*SEG_ADDR_WIDTH-1:0]
ram_rd_cmd_addr,
181 output wire [PORTS*SEG_COUNT-1:0]
ram_rd_cmd_valid,
182 input wire [PORTS*SEG_COUNT-1:0]
ram_rd_cmd_ready,
183 input wire [PORTS*SEG_COUNT*SEG_DATA_WIDTH-1:0]
ram_rd_resp_data,
184 input wire [PORTS*SEG_COUNT-1:0]
ram_rd_resp_valid,
185 output wire [PORTS*SEG_COUNT-1:0]
ram_rd_resp_ready
186 );
187
188 dma_if_mux_rd #(
189     .PORTS(PORTS),
190     .SEG_COUNT(SEG_COUNT),
191     .SEG_DATA_WIDTH(SEG_DATA_WIDTH),
192     .SEG_ADDR_WIDTH(SEG_ADDR_WIDTH),
193     .SEG_BE_WIDTH(SEG_BE_WIDTH),
194     .S_RAM_SEL_WIDTH(S_RAM_SEL_WIDTH),
195     .M_RAM_SEL_WIDTH(M_RAM_SEL_WIDTH),
196     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
197     .DMA_ADDR_WIDTH(DMA_ADDR_WIDTH),
198     .LEN_WIDTH(LEN_WIDTH),
199     .S_TAG_WIDTH(S_TAG_WIDTH),
200     .M_TAG_WIDTH(M_TAG_WIDTH),
201     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
202     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
203 )
204 dma_if_mux_rd_inst (
205     .clk(clk),
206     .rst(rst),
207
208     /*
209     * Descriptor output (to DMA interface)
210     */
211     .m_axis_read_desc_dma_addr(m_axis_read_desc_dma_addr),
212     .m_axis_read_desc_ram_sel(m_axis_read_desc_ram_sel),
213     .m_axis_read_desc_ram_addr(m_axis_read_desc_ram_addr),
214     .m_axis_read_desc_len(m_axis_read_desc_len),
215     .m_axis_read_desc_tag(m_axis_read_desc_tag),
216     .m_axis_read_desc_valid(m_axis_read_desc_valid),
217     .m_axis_read_desc_ready(m_axis_read_desc_ready),
218
219     /*
220     * Descriptor status input (from DMA interface)
221     */
222     .s_axis_read_desc_status_tag(s_axis_read_desc_status_tag),
223     .s_ax-
224     is_read_desc_status_error(s_axis_read_desc_status_error),
225     .s_ax-
226     is_read_desc_status_valid(s_axis_read_desc_status_valid),
227
228     /*
229     * Descriptor input
230     */
231     .s_axis_read_desc_dma_addr(s_axis_read_desc_dma_addr),
232     .s_axis_read_desc_ram_sel(s_axis_read_desc_ram_sel),
233     .s_axis_read_desc_ram_addr(s_axis_read_desc_ram_addr),
234     .s_axis_read_desc_len(s_axis_read_desc_len),
235     .s_axis_read_desc_tag(s_axis_read_desc_tag),
236     .s_axis_read_desc_valid(s_axis_read_desc_valid),
237     .s_axis_read_desc_ready(s_axis_read_desc_ready),
238
239     /*
240     * Descriptor status output
241     */
242     .m_ax-
243     is_write_desc_status_tag(m_axis_write_desc_status_tag),
244     .m_ax-
245     is_write_desc_status_error(m_axis_write_desc_status_error),
246     .m_ax-
247     is_write_desc_status_valid(m_axis_write_desc_status_valid),
248
249     /*
250     * Descriptor input
251     */
252     .s_axis_write_desc_dma_addr(s_axis_write_desc_dma_addr),
253     .s_axis_write_desc_ram_sel(s_axis_write_desc_ram_sel),
254     .s_axis_write_desc_ram_addr(s_axis_write_desc_ram_addr),
255     .s_axis_write_desc_imm(s_axis_write_desc_imm),
256     .s_axis_write_desc_imm_en(s_axis_write_desc_imm_en),
257     .s_axis_write_desc_len(s_axis_write_desc_len),
258     .s_axis_write_desc_tag(s_axis_write_desc_tag),
259     .s_axis_write_desc_valid(s_axis_write_desc_valid),
260     .s_axis_write_desc_ready(s_axis_write_desc_ready),
261
262     /*
263     * Descriptor status output
264     */
265     .m_ax-
266     is_write_desc_status_tag(m_axis_write_desc_status_tag),
267     .m_ax-
268     is_write_desc_status_error(m_axis_write_desc_status_error),
269     .m_ax-
270     is_write_desc_status_valid(m_axis_write_desc_status_valid),
271
272     /*
273     * RAM interface (from DMA interface)
274     */
275     .if_ram_wr_cmd_sel(if_ram_wr_cmd_sel),
276     .if_ram_wr_cmd_be(if_ram_wr_cmd_be),
277     .if_ram_wr_cmd_addr(if_ram_wr_cmd_addr),
278     .if_ram_wr_cmd_data(if_ram_wr_cmd_data),
279     .if_ram_wr_cmd_valid(if_ram_wr_cmd_valid),
280     .if_ram_wr_cmd_ready(if_ram_wr_cmd_ready),
281     .if_ram_wr_done(if_ram_wr_done),
282
283     /*
284     * RAM interface (towards RAM)
285     */
286     .ram_wr_cmd_sel(ram_wr_cmd_sel),
287     .ram_wr_cmd_be(ram_wr_cmd_be),
288     .ram_wr_cmd_addr(ram_wr_cmd_addr),
289     .ram_wr_cmd_data(ram_wr_cmd_data),
290     .ram_wr_cmd_valid(ram_wr_cmd_valid),
291     .ram_wr_cmd_ready(ram_wr_cmd_ready),
292     .ram_wr_done(ram_wr_done)
293 );
294
295 dma_if_mux_wr #(
296     .PORTS(PORTS),
297     .SEG_COUNT(SEG_COUNT),
298     .SEG_DATA_WIDTH(SEG_DATA_WIDTH),
299     .SEG_ADDR_WIDTH(SEG_ADDR_WIDTH),
300     .SEG_BE_WIDTH(SEG_BE_WIDTH),
301     .S_RAM_SEL_WIDTH(S_RAM_SEL_WIDTH),
302     .M_RAM_SEL_WIDTH(M_RAM_SEL_WIDTH),
303     .RAM_ADDR_WIDTH(RAM_ADDR_WIDTH),
304     .DMA_ADDR_WIDTH(DMA_ADDR_WIDTH),
305     .IMM_ENABLE(IMM_ENABLE),
306     .IMM_WIDTH(IMM_WIDTH),
307     .LEN_WIDTH(LEN_WIDTH),
308     .S_TAG_WIDTH(S_TAG_WIDTH),
309     .M_TAG_WIDTH(M_TAG_WIDTH),
310     .ARB_TYPE_ROUND_ROBIN(ARB_TYPE_ROUND_ROBIN),
311     .ARB_LSB_HIGH_PRIORITY(ARB_LSB_HIGH_PRIORITY)
312 )
313 dma_if_mux_wr_inst (
314     .clk(clk),
315     .rst(rst),
316
317     /*
318     * Descriptor output (to DMA interface)
319     */
320     .m_axis_write_desc_dma_addr(m_axis_write_desc_dma_addr),
321     .m_axis_write_desc_ram_sel(m_axis_write_desc_ram_sel),
322     .m_axis_write_desc_ram_addr(m_axis_write_desc_ram_addr),
323     .m_axis_write_desc_imm(m_axis_write_desc_imm),
324     .m_axis_write_desc_imm_en(m_axis_write_desc_imm_en),
325     .m_axis_write_desc_len(m_axis_write_desc_len),
326     .m_axis_write_desc_tag(m_axis_write_desc_tag),
327     .m_axis_write_desc_valid(m_axis_write_desc_valid),
328     .m_axis_write_desc_ready(m_axis_write_desc_ready),
329
330     /*
331     * Descriptor status input (from DMA interface)
332     */
333     .s_ax-
334     is_write_desc_status_tag(s_axis_write_desc_status_tag),
335     .s_ax-
336     is_write_desc_status_error(s_axis_write_desc_status_error),
337     .s_ax-
338     is_write_desc_status_valid(s_axis_write_desc_status_valid),
339
340     /*
341     * Descriptor input
342     */
343     .s_axis_write_desc_dma_addr(s_axis_write_desc_dma_addr),
344     .s_axis_write_desc_ram_sel(s_axis_write_desc_ram_sel),
345     .s_axis_write_desc_ram_addr(s_axis_write_desc_ram_addr),
346     .s_axis_write_desc_imm(s_axis_write_desc_imm),
347     .s_axis_write_desc_imm_en(s_axis_write_desc_imm_en),
348     .s_axis_write_desc_len(s_axis_write_desc_len),
349     .s_axis_write_desc_tag(s_axis_write_desc_tag),
350     .s_axis_write_desc_valid(s_axis_write_desc_valid),
351     .s_axis_write_desc_ready(s_axis_write_desc_ready),
352
353     /*
354     * Descriptor status output
355     */
356     .m_ax-
357     is_write_desc_status_tag(m_axis_write_desc_status_tag),
358     .m_ax-
359     is_write_desc_status_error(m_axis_write_desc_status_error),
360     .m_ax-
361     is_write_desc_status_valid(m_axis_write_desc_status_valid),
362
363     /*
364     * RAM interface (from DMA interface)
365     */
366     .if_ram_wr_cmd_sel(if_ram_wr_cmd_sel),
367     .if_ram_wr_cmd_be(if_ram_wr_cmd_be),
368     .if_ram_wr_cmd_addr(if_ram_wr_cmd_addr),
369     .if_ram_wr_cmd_data(if_ram_wr_cmd_data),
370     .if_ram_wr_cmd_valid(if_ram_wr_cmd_valid),
371     .if_ram_wr_cmd_ready(if_ram_wr_cmd_ready),
372     .if_ram_wr_done(if_ram_wr_done),
373
374     /*
375     * RAM interface (towards RAM)
376     */
377     .ram_wr_cmd_sel(ram_wr_cmd_sel),
378     .ram_wr_cmd_be(ram_wr_cmd_be),
379     .ram_wr_cmd_addr(ram_wr_cmd_addr),
380     .ram_wr_cmd_data(ram_wr_cmd_data),
381     .ram_wr_cmd_valid(ram_wr_cmd_valid),
382     .ram_wr_cmd_ready(ram_wr_cmd_ready),
383     .ram_wr_done(ram_wr_done)
384 );

```

```

331      */
332      .if_ram_rd_cmd_sel(if_ram_rd_cmd_sel),
333      .if_ram_rd_cmd_addr(if_ram_rd_cmd_addr),
334      .if_ram_rd_cmd_valid(if_ram_rd_cmd_valid),
335      .if_ram_rd_cmd_ready(if_ram_rd_cmd_ready),
336      .if_ram_rd_resp_data(if_ram_rd_resp_data),
337      .if_ram_rd_resp_valid(if_ram_rd_resp_valid),
338      .if_ram_rd_resp_ready(if_ram_rd_resp_ready),
339
340      /*
341      * RAM interface (towards RAM)
342      */
343      .ram_rd_cmd_sel(ram_rd_cmd_sel),
344      .ram_rd_cmd_addr(ram_rd_cmd_addr),
345      .ram_rd_cmd_valid(ram_rd_cmd_valid),
346      .ram_rd_cmd_ready(ram_rd_cmd_ready),
347      .ram_rd_resp_data(ram_rd_resp_data),
348      .ram_rd_resp_valid(ram_rd_resp_valid),
349      .ram_rd_resp_ready(ram_rd_resp_ready)
350  );
351
352  endmodule
353
354  `resetall

```

RVFI UVM 에이전트

15 files • 2,159 lines

API — uvma_rvfi_agent.sv

Table 1258. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_agent.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_AGENT_SV__		/**

API — uvma_rvfi_assert.sv

Table 1259. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_assert.sv — module / interface / package (1개)

이름	종류
uvma_rvfi_assert	module

Table 1260. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_assert.sv — parameter (2개)

파라미터	형/폭	기본값
ADDR_WIDTH	int	32
DATA_WIDTH	int	32

Table 1261. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_assert.sv — 포트 (11개)

포트	방향	형/폭
clk	input	
reset_n	input	
req	input	
gnt	input	
addr	input	[ADDR_WIDTH-1:0]
we	input	
be	input	[DATA_WIDTH/8-1:0]
wdata	input	[DATA_WIDTH-1:0]
rdata	input	[DATA_WIDTH-1:0]
rvalid	input	
rready	input	

API — uvma_rvfi_cfg.sv

Table 1262. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_cfg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_CFG_SV__		/**

API — uvma_rvfi_cntxt.sv

Table 1263. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_cntxt.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_CNTXT_SV__		/**

API — uvma_rvfi_constants.sv

Table 1264. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_constants.sv — localparam (24개)

상수	형/폭	값
ORDER_WL		64
MODE_WL		2
IXL_WL		2
TRAP_WL		12
GPR_ADDR_WL		5
RVFI_DBG_WL		3
RVFI_NMIP_WL		2
RVFI_INTR_WL		16
CYCLE_CNT_WL		32
TRAP_EXCP_LSB		0
TRAP_EXCP_WL		1
TRAP_NONDBG_ENTRY_LSB		1
TRAP_NONDBG_ENTRY_WL		1
TRAP_DBG_ENTRY_LSB		2
TRAP_DBG_ENTRY_WL		1
TRAP_CAUSE_LSB		3
TRAP_CAUSE_WL		6
TRAP_DBG_CAUSE_LSB		9
TRAP_DBG_CAUSE_WL		3
DEFAULT_ILEN		32
DEFAULT_XLEN		32
MAX_XLEN		64
DEFAULT_NRET		1
CSR_MAX_SIZE		4096

Table 1265. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_constants.sv — #define / `define 매크로 (2개)

매크로	인자	정의
__UVMA_RVFI_CONSTANTS_SV__		// RVFI field widths
FORMAT_INSTR_STR_MACRO		"%8s RVFI %8d %6d %8x %8s %s x%-2d %8x x%-2d %8x x%-2d %16x %s"

API — uvma_rvfi_csr_if.sv

Table 1266. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv — module / interface / package (2개)

이름	종류
uvma_rvfi_csr_if	interface
uvma_rvfi_unified_csr_if	interface

Table 1267. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv — 포트 (19개)

포트	방향	형/폭
clk	input	
reset_n	input	
rvfi_csr_rmask	input	[XLEN-1:0]
rvfi_csr_wmask	input	[XLEN-1:0]
rvfi_csr_rdata	input	[XLEN-1:0]
rvfi_csr_wdata	input	[XLEN-1:0]
1step	input	#
clk	input	
reset_n	input	
rvfi_named_csr_rmask	input	[MAX_CSR-1:0][XLEN-1:0]
rvfi_named_csr_wmask	input	[MAX_CSR-1:0][XLEN-1:0]
rvfi_named_csr_rdata	input	[MAX_CSR-1:0][XLEN-1:0]
rvfi_named_csr_wdata	input	[MAX_CSR-1:0][XLEN-1:0]
rvfi_csr_addr	input	[XLEN-1:0]
rvfi_csr_rmask	input	[XLEN-1:0]
rvfi_csr_wmask	input	[XLEN-1:0]
rvfi_csr_rdata	input	[XLEN-1:0]
rvfi_csr_wdata	input	[XLEN-1:0]
1step	input	#

Table 1268. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_CSR_IF_SV__		/**

API — uvma_rvfi_instr_if.sv

Table 1269. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv — module / interface / package (1개)

이름	종류
uvma_rvfi_instr_if	interface

Table 1270. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv — 포트 (31개)

포트	방향	형/폭
clk	input	
reset_n	input	
rvfi_valid	input	
rvfi_order	input	[ORDER_WL-1:0]
rvfi_insn	input	[ILEN-1:0]
rvfi_trap	input	[TRAP_WL-1:0]
rvfi_halt	input	
rvfi_dbg	input	[RVFI_DBG_WL-1:0]
rvfi_dbg_mode	input	
rvfi_nmip	input	[RVFI_NMIP_WL-1:0]
rvfi_intr	input	[RVFI_INTR_WL-1:0]
rvfi_mode	input	[MODE_WL-1:0]
rvfi_ixl	input	[IXL_WL-1:0]
rvfi_pc_rdata	input	[XLEN-1:0]
rvfi_pc_wdata	input	[XLEN-1:0]
rvfi_rs1_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rs1_rdata	input	[XLEN-1:0]
rvfi_rs2_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rs2_rdata	input	[XLEN-1:0]
rvfi_rs3_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rs3_rdata	input	[XLEN-1:0]
rvfi_rd1_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rd1_wdata	input	[XLEN-1:0]
rvfi_rd2_addr	input	[GPR_ADDR_WL-1:0]
rvfi_rd2_wdata	input	[XLEN-1:0]
rvfi_mem_addr	input	[XLEN-1:0]

포트	방향	형/폭
rvfi_mem_rdata	input	[XLEN-1:0]
rvfi_mem_rmask	input	[XLEN/8-1:0]
rvfi_mem_wdata	input	[XLEN-1:0]
rvfi_mem_wmask	input	[XLEN/8-1:0]
1step	input	#

Table 1271. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_INSTR_IF_SV__		/**

API — uvma_rvfi_instr_mon.sv

Table 1272. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_instr_mon.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_INSTR_MON_SV__		/**

API — uvma_rvfi_macros.sv

Table 1273. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_macros.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_MACROS_SV__		`endif // __UVMA_RVFI_MACROS_SV__

API — uvma_rvfi_mon_trn_logger.sv

Table 1274. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_mon_trn_logger.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_MON_TRN_LOGGER_SV__		/**

API — uvma_rvfi_pkg.sv

Table 1275. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_pkg.sv — module / interface / package (1개)

이름	종류
uvma_rvfi_pkg	package

Table 1276. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMA_RVFI_PKG_SV__		// Pre-processor macros

API — uvma_rvfi_tdefs.sv**Table 1277. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv — parameter (1개)**

파라미터	형/폭	기본값
ST_NUM_WORDS		(\$size(st_rvfi)/MAX_XLEN)

Table 1278. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv — typedef / using (1개)

새 이름	원래 형
vector_rvfi	bit [ST_NUM_WORDS-1:0] [MAX_XLEN-1:0]

Table 1279. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv — #define / `define 매크로 (2개)

매크로	인자	정의
__UVMA_RVFI_TDEFS_SV__		typedef enum bit[MODE_WL-1:0] {
ST_NUM_WORDS		(\$size(st_rvfi)/MAX_XLEN)

API — uvma_rvfi_csr_seq_item.sv**Table 1280. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/seq/uvma_rvfi_csr_seq_item.sv — #define / `define 매크로 (1개)**

매크로	인자	정의
__UVMA_RVFI_CSR_SEQ_ITEM_SV__		/**

API — uvma_rvfi_instr_seq_item.sv**Table 1281. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/seq/uvma_rvfi_instr_seq_item.sv — #define / `define 매크로 (1개)**

매크로	인자	정의
__UVMA_RVFI_INSTR_SEQ_ITEM_SV__		/**

API — uvma_rvfi_instr_table_seq_item.sv**Table 1282. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/seq/uvma_rvfi_instr_table_seq_item.sv — #define / `define 매크로 (1개)**

매크로	인자	정의
__UVMA_RVFI_INSTR_TABLE_SEQ_ITEM_SV__		/**

1. openhwgroup_core-v-verif/lib/uvm_agents/uvma_rvfi/uvma_rvfi_agent.sv (239 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
7 // (the "License");
8 // you may not use this file except in compliance with the Li-
9 // cense.
10 // You may obtain a copy of the License at
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,

```

```

software
13 // distributed under the License is distributed on an "AS IS"
BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
15 // See the License for the specific language governing permis-
sions and
16 // limitations under the License.
17 //
18
19
20 `ifndef __UVMA_RVFI_AGENT_SV__
21 `define __UVMA_RVFI_AGENT_SV__
22
23 /**
24 * Top-level component that encapsulates, builds and connects
all others.
25 * Capable of driving/monitoring Clock & Reset interface.
26 */

```

```

27 class uvma_rvfi_agent_c#(int ILEN=DEFAULT_ILEN,
28                          int XLEN=DEFAULT_XLEN) extends uvm_a-
gent;
29
30 // Objects
31 uvma_rvfi_cfg_c#(ILEN,XLEN)    cfg;
32 uvma_rvfi_cntxt_c#(ILEN,XLEN)  cntxt;
33
34 // Components
35 uvma_rvfi_instr_mon_c#(ILEN,XLEN)    in-
str_monitor;
36 uvma_rvfi_mon_trn_logger_c#(ILEN,XLEN)
mon_trn_logger;
37
38 uvm_analysis_port#(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN))
rvfi_core_ap;
39
40 `uvm_component_param_utils_begin(uvma_rvfi_agent_c)
41   `uvm_field_object(cfg, UVM_DEFAULT)
42   `uvm_field_object(cntxt, UVM_DEFAULT)
43   `uvm_component_utils_end
44
45 /**
46  * Default constructor.
47  */
48 extern function new(string name="uvma_rvfi_agent", uvm_com-
ponent parent=null);
49
50 /**
51  * 1. Ensures cfg & cntxt handles are not null
52  * 2. Builds all components
53  */
54 extern virtual function void build_phase(uvm_phase phase);
55
56 /**
57  * 1. Links agent's analysis ports to sub-components'
58  * 2. Connects coverage models and loggers
59  */
60 extern virtual function void connect_phase(uvm_phase
phase);
61
62 /**
63  * Uses uvm_config_db to retrieve cfg and hand out to sub-
components.
64  */
65 extern function void get_and_set_cfg();
66
67 /**
68  * Uses uvm_config_db to retrieve cntxt and hand out to
sub-components.
69  */
70 extern function void get_and_set_cntxt();
71
72 /**
73  * Uses uvm_config_db to retrieve the Virtual Interface
(vif) associated with this
74  * agent.
75  */
76 extern function void retrieve_vif();
77
78 /**
79  * Creates sub-components.
80  */
81 extern function void create_components();
82
83 /**
84  * Connects sequencer and driver's TLM port(s).
85  */
86 extern function void connect_sequencer_and_driver();
87
88 /**
89  * Connects agent's TLM ports to driver's and monitor's.
90  */
91 extern function void connect_analysis_ports();
92
93 /**
94  * Connects transaction loggers to monitor and driver's
analysis ports.
95  */
96 extern function void connect_trn_loggers();
97
98 endclass : uvma_rvfi_agent_c
99
100
101 function uvma_rvfi_agent_c::new(string name="uvma_rvfi_agent",
uvm_component parent=null);
102
103   super.new(name, parent);
104
105 endfunction : new
106
107
108 function void uvma_rvfi_agent_c::build_phase(uvm_phase phase);
109
110   super.build_phase(phase);
111
112   get_and_set_cfg ();
113   get_and_set_cntxt();
114   retrieve_vif ();
115   create_components();

```

```

116
117 endfunction : build_phase
118
119
120 function void uvma_rvfi_agent_c::connect_phase(uvm_phase
phase);
121
122   super.connect_phase(phase);
123
124   connect_sequencer_and_driver();
125   connect_analysis_ports();
126
127   if (cfg.trn_log_enabled) begin
128     connect_trn_loggers();
129   end
130
131 endfunction: connect_phase
132
133
134 function void uvma_rvfi_agent_c::get_and_set_cfg();
135
136   void'(uvm-con-
fig_db#(uvma_rvfi_cfg_c#(ILEN,XLEN))::get(this, "", "cfg", cfg));
137   if (!cfg) begin
138     `uvm_fatal("CFG", "Configuration handle is null")
139   end
140   else begin
141     `uvm_info("CFG", $sformatf("Found configuration handle:
\n%s", cfg.sprint()), UVM_DEBUG)
142     uvm_config_db#(uvma_rvfi_cfg_c#(ILEN,XLEN))::set(this,
"", "cfg", cfg);
143   end
144
145 endfunction : get_and_set_cfg
146
147
148 function void uvma_rvfi_agent_c::get_and_set_cntxt();
149
150   void'(uvm-con-
fig_db#(uvma_rvfi_cntxt_c#(ILEN,XLEN))::get(this, "", "cntxt",
cntxt));
151   if (!cntxt) begin
152     `uvm_info("CNTXT", "Context handle is null; creating.",
UVM_DEBUG)
153     cntxt = uvma_r-
vfi_cntxt_c#(ILEN,XLEN)::type_id::create("cntxt");
154   end
155   uvm_config_db#(uvma_rvfi_cntxt_c#(ILEN,XLEN))::set(this,
"", "cntxt", cntxt);
156
157 endfunction : get_and_set_cntxt
158
159
160 function void uvma_rvfi_agent_c::retrieve_vif();
161
162 // Retrieve instruction interface
163 cntxt.instr_vif = new[cfg.nret];
164 for (int i = 0; i < cfg.nret; i++) begin
165   if (!uvm_config_db#(virtual uvma_r-
vfi_instr_if#(ILEN,XLEN))::get(this, "", $sformatf("instr_vif%0d",
i), cntxt.instr_vif[i])) begin
166     `uvm_fatal("VIF", $sformatf("Could not find vif
handle of type %s in uvm_config_db",
167                               $type-
name(cntxt.instr_vif[i])))
168   end
169   else begin
170     `uvm_info("VIF", $sformatf("Found vif handle of type
%s in uvm_config_db",
171                               $type-
name(cntxt.instr_vif[i])), UVM_DEBUG)
172   end
173 end
174
175 // Create virtual interface and fetch virtual interface for
each supported CSR
176 if (!cfg.core_cfg.disable_all_csr_checks) begin
177   string csrs[$];
178
179   cfg.core_cfg.get_supported_csrs_names(csrs);
180   if (!cfg.unified_csr_vif) begin
181     foreach (csrs[c]) begin
182       string csr = csrs[c].tolower();
183       cntxt.csr_vif[csr] = new[cfg.nret];
184
185       for (int i = 0; i < cfg.nret; i++) begin
186         if (!uvm_config_db#(virtual uvma_r-
vfi_csr_if#(XLEN))::get(this, "", $sformatf("csr_%s_vif%0d", csr,
i), cntxt.csr_vif[csr][i])) begin
187           `uvm_fatal("VIF", $sformatf("Could not find
vif handle of type %s, csr [%s] in uvm_config_db",
188                                     $type-
name(cntxt.csr_vif[csr][i]), csr))
189         end else begin
190           `uvm_info("VIF", $sformatf("Found vif handle
of type %s, csr [%s] in uvm_config_db",
191                                     $type-
name(cntxt.csr_vif[csr][i]), csr), UVM_DEBUG)
192         end

```

```

193     end
194   end
195   end
196   else begin
197     cntxt.csr_unified_vif = new[cfg.nret];
198     for (int i = 0; i < cfg.nret; i++) begin
199       if (!uvm_config_db#(virtual uvma_r-
vfi_unified_csr_if#(4096, XLEN))::get(this, "", $sform-
atf("csr_vif%d", i), cntxt.csr_unified_vif[i])) begin
200         `uvm_fatal("VIF", $sformatf("Could not find vif
handle of type %s, csr_vif in uvm_config_db",
201                                     $type-
name(cntxt.csr_unified_vif[i])))
202       end else begin
203         `uvm_info("VIF", $sformatf("Found vif handle of
type %s, csr_vif in uvm_config_db",
204                                     $type-
name(cntxt.csr_unified_vif[i])), UVM_DEBUG)
205       end
206     end
207   end
208 end
209
210 endfunction : retrieve_vif
211
212
213 function void uvma_rvfi_agent_c::create_components();
214
215   instr_monitor = uvma_r-
vfi_instr_mon_c#(ILEN,XLEN)::type_id::create("instr_monitor", this);
216   mon_trn_logger = uvma_r-
vfi_mon_trn_logger_c#(ILEN,XLEN)::type_id::create("mon_trn_logger" ,
this);
217
218 endfunction : create_components
219
220
221 function void uvma_r-
vfi_agent_c::connect_sequencer_and_driver();
222
223 endfunction : connect_sequencer_and_driver
224
225
226 function void uvma_rvfi_agent_c::connect_analysis_ports();
227
228   rvfi_core_ap = instr_monitor.ap;
229
230 endfunction : connect_analysis_ports
231
232
233 function void uvma_rvfi_agent_c::connect_trn_loggers();
234
235   rvfi_core_ap.connect(mon_trn_logger.instr_export);
236
237 endfunction : connect_trn_loggers
238
239 `endif // __UVMA_RVFI_AGENT_SV__

```

2. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_assert.sv (133 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 //     https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17 module uvma_rvfi_assert
18   import uvm_pkg::*;
19   #(
20     parameter int ADDR_WIDTH=32,
21     parameter int DATA_WIDTH=32
22   )
23   (
24     input          clk,
25     input          reset_n,
26
27     input          req,
28     input          gnt,
29     input [ADDR_WIDTH-1:0] addr,
30     input          we,
31     input [DATA_WIDTH/8-1:0] be,
32     input [DATA_WIDTH-1:0] wdata,
33     input [DATA_WIDTH-1:0] rdata,

```

```

34     input          rvalid,
35     input          rready,
36   );
37
38   //
-----
39   // Local variables
40   //
-----
41   string info_tag = "CV32E40P-RVFI_ASSERT";
42
43   //
-----
44   // Clocking blocks
45   //
-----
46
47   // Single clock, single reset design, use default clocking
48   default clocking @(posedge clk); endclocking
49   default disable iff !(reset_n);
50
51   //
-----
52   // Begin module code
53   //
-----
54
55   // R-3.1.1 : Address phase signals stable during address
phase
56   property p_addr_signal_stable(sig);
57     req ##0 !gnt |>= $stable(sig);
58   endproperty : p_addr_signal_stable
59
60   a_addr_stable: assert property(p_addr_signal_stable(addr))
61   else
62     `uvm_error(info_tag, "addr signal not stable in address
phase")
63
64   a_we_stable: assert property(p_addr_signal_stable(we))
65   else
66     `uvm_error(info_tag, "we signal not stable in address
phase")
67
68   a_wdata_stable: assert property(p_addr_signal_stable(wdata))
69   else
70     `uvm_error(info_tag, "wdata signal not stable in address
phase")
71
72   a_be_stable: assert property(p_addr_signal_stable(be))
73   else
74     `uvm_error(info_tag, "be signal not stable in address
phase")
75
76   // R-3.1.2 : Req may not deassert until the gnt is asserted
77   property p_req_until_gnt;
78     req ##0 !gnt |>= req;
79   endproperty : p_req_until_gnt
80   a_req_until_gnt : assert property(p_req_until_gnt)
81   else
82     `uvm_error(info_tag, "req may not deassert until gnt as-
serted")
83
84   // R-7 At least one byte enable must be set
85   property p_be_not_zero;
86     req |>= be != 0;
87   endproperty : p_be_not_zero
88   a_be_not_zero : assert property(p_be_not_zero)
89   else
90     `uvm_error(info_tag, "be was zero during an address
cycle")
91
92   // R-7 All ones must be contiguous in writes
93   reg[3:0] contiguous_be[] = {
94     4'b0001,
95     4'b0011,
96     4'b0111,
97     4'b1111,
98     4'b0010,
99     4'b0110,
100    4'b1110,
101    4'b0100,
102    4'b1100,
103    4'b1000
104   };
105   bit be_inside_contiguous_be;
106   always_comb begin
107     be_inside_contiguous_be = be inside {contiguous_be};
108   end
109   property p_be_contiguous;
110     req |>= be_inside_contiguous_be;
111   endproperty : p_be_contiguous
112   a_be_contiguous : assert property(p_be_contiguous)
113   else
114     `uvm_error(info_tag, $sformatf("be of 0x%x was not con-
tiguous", $sampled(be)));

```

```

115
116 // R-8 Data address LSBs must be consistent with byte en-
ables on writes
117 function bit [1:0] get_addr_lsb(bit[3:0] be);
118     casex (be)
119         4'b???1: return 0;
120         4'b??10: return 1;
121         4'b?100: return 2;
122         4'b1000: return 3;
123     endcase
124 endfunction : get_addr_lsb
125
126 property p_addr_be_consistent;
127     req |-> addr[1:0] == get_addr_lsb(be);
128 endproperty : p_addr_be_consistent
129 a_addr_be_consistent: assert property(p_addr_be_consistent)
130 else
131     `uvm_error(info_tag, $sformatf("be of 0x%01x not consist-
ent with addr 0x%08x", $sampled(be), $sampled(addr)));
132
133 endmodule : uvma_rvfi_assert

```

3. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_cfg.sv (116 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
7 // you may not use this file except in compliance with the Li-
cense.
8 // You may obtain a copy of the License at
9 //
10 //     https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
software
13 // distributed under the License is distributed on an "AS IS"
BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
15 // See the License for the specific language governing permis-
sions and
16 // limitations under the License.
17 //
18
19 `ifndef __UVMA_RVFI_CFG_SV__
20 `define __UVMA_RVFI_CFG_SV__
21
22
23
24 /**
25  * Object encapsulating all parameters for creating, connect-
ing and running all
26  * Clock & Reset agent (uvma_rvfi_agent_c) components.
27  */
28 class uvma_rvfi_cfg_c#(int ILEN=DEFAULT_ILEN,
29                       int XLEN=DEFAULT_XLEN) extends uvm_ob-
ject;
30
31 // Core configuration (used to extract list of CSRs)
32 uvma_core_cntrl_cfg_c    core_cfg;
33
34 // Common options
35 rand bit                  enabled;
36 rand uvm_active_passive_enum is_active;
37
38 rand bit                  unified_csr_vif;
39 rand bit                  cov_model_enabled;
40 rand bit                  trn_log_enabled;
41
42
43 // Number of instructions that may be retired in a single
cycle
44 // This number cannot be zero
45 rand int unsigned         nret;
46
47 // Configuration of NMI faults
48 rand bit                  nmi_load_fault_enabled;
49 rand int unsigned         nmi_load_fault_cause;
50 rand bit                  nmi_store_fault_enabled;
51 rand int unsigned         nmi_store_fault_cause;
52
53 // Enable bus interrupt fault bit
54 rand bit                  insn_bus_fault_enabled;
55 rand int unsigned         insn_bus_fault_cause;
56
57 // Name for the instruction retirement ports (nret of these)
58 string                    instr_name[int];
59
60 // Merge exceptions with the first instruction after excep-
tion
61
62 `uvm_object_utils_begin(uvma_rvfi_cfg_c)
63     `uvm_field_int (          , UVM_DEFAULT)    en-
abled
64     `uvm_field_int

```

```

(
65     `uvm_field_int (          , UVM_DEFAULT)    uni-
fied_csr_vif
66     `uvm_field_enum(uvm_active_passive_enum, is_act-
ive          , UVM_DEFAULT)
67     `uvm_field_int (          , UVM_DEFAULT)    cov_mod-
el_enabled
68     `uvm_field_int (          , UVM_DEFAULT)    trn_log_en-
abled
69     `uvm_field_int
(
70         nmi_load_fault_enabled          , UVM_DEFAULT)
(
71         nmi_load_fault_cause            , UVM_DEFAULT)
(
72         nmi_store_fault_enabled         , UVM_DEFAULT)
(
73         nmi_store_fault_cause           , UVM_DEFAULT)
(
74         insn_bus_fault_enabled          , UVM_DEFAULT)
(
75         insn_bus_fault_cause            , UVM_DEFAULT)
76     `uvm_object_utils_end
77
78 constraint valid_nret {
79     nret != 0;
80 }
81
82 constraint valid_active_passive {
83     // Only designed to support passive mode
84     is_active == UVM_PASSIVE;
85 }
86
87 constraint defaults_cons {
88     soft enabled          == 1;
89     soft is_active        == UVM_PASSIVE;
90     soft cov_model_enabled == 0;
91     soft trn_log_enabled  == 1;
92     soft nret             == 1;
93     soft nmi_load_fault_enabled == 0;
94     soft nmi_load_fault_cause == 0;
95     soft nmi_store_fault_enabled == 0;
96     soft nmi_store_fault_cause == 0;
97     soft insn_bus_fault_enabled == 0;
98     soft insn_bus_fault_cause == 0;
99     soft unified_csr_vif == 0;
100 }
101
102 /**
103  * Default constructor.
104  */
105 extern function new(string name="uvma_rvfi_cfg");
106
107 endclass : uvma_rvfi_cfg_c
108
109 function uvma_rvfi_cfg_c::new(string name="uvma_rvfi_cfg");
110     super.new(name);
111
112 endfunction : new
113
114 `endif // __UVMA_RVFI_CFG_SV__
115
116

```

4. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_cntxt.sv (78 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 //     https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17
18 `ifndef __UVMA_RVFI_CNTXT_SV__
19 `define __UVMA_RVFI_CNTXT_SV__
20
21 /**
22  * Object encapsulating all state variables for all Rvfi agent
23  * (uvma_rvfi_agent_c) components.
24  */
25 class uvma_rvfi_cntxt_c#(int ILEN=DEFAULT_ILEN,
26                          int XLEN=DEFAULT_XLEN) extends
uvm_object;

```

```

27
28 // Handle to instruction retirement interface
29 // The number of interfaces will be equal to the _nret_
value in the configuration object
30 virtual uvma_rvfi_instr_if#(ILEN,XLEN) instr_vif[];
31
32 // Handle to CSR interfaces
33 virtual uvma_rvfi_csr_if#(XLEN) csr_vif[string][];
34 virtual uvma_rvfi_unified_csr_if#(4096,XLEN) csr_uni-
fied_vif[];
35
36 // Events
37 uvm_event sample_cfg_e;
38 uvm_event sample_cntxt_e;
39
40 `uvvm_object_utils_begin(uvma_rvfi_cntxt_c)
41 `uvvm_field_event(sample_cfg_e , UVM_DEFAULT)
42 `uvvm_field_event(sample_cntxt_e, UVM_DEFAULT)
43 `uvvm_object_utils_end
44
45 /**
46  * Builds events.
47  */
48 extern function new(string name="uvma_rvfi_cntxt");
49
50 /**
51  * TODO Describe uvma_rvfi_cntxt_c::reset()
52  */
53 extern function void reset();
54
55 endclass : uvma_rvfi_cntxt_c
56
57
58 `pragma protect begin
59
60
61 function uvma_rvfi_cntxt_c::new(string name="uvma_r-
vfi_cntxt");
62
63     super.new(name);
64
65     sample_cfg_e = new("sample_cfg_e" );
66     sample_cntxt_e = new("sample_cntxt_e");
67
68 endfunction : new
69
70 function void uvma_rvfi_cntxt_c::reset();
71
72 endfunction : reset
73
74
75 `pragma protect end
76
77
78 `endif // __UVMA_RVFI_CNTXT_SV__

```

5. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_constants.sv (56 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17
18 `ifndef __UVMA_RVFI_CONSTANTS_SV__
19 `define __UVMA_RVFI_CONSTANTS_SV__
20
21 // RVFI field widths
22 localparam ORDER_WL = 64;
23 localparam MODE_WL = 2;
24 localparam IXL_WL = 2;
25 localparam TRAP_WL = 12;
26 localparam GPR_ADDR_WL = 5;
27 localparam RVFI_DBG_WL = 3;
28 localparam RVFI_NMIP_WL = 2;
29 localparam RVFI_INTR_WL = 16;
30 localparam CYCLE_CNT_WL = 32;
31
32 // Fields within TRAP
33 localparam TRAP_EXCP_LSB = 0;
34 localparam TRAP_EXCP_WL = 1;
35 localparam TRAP_NONDBG_ENTRY_LSB = 1;

```

```

36 localparam TRAP_NONDBG_ENTRY_WL = 1;
37 localparam TRAP_DBG_ENTRY_LSB = 2;
38 localparam TRAP_DBG_ENTRY_WL = 1;
39 localparam TRAP_CAUSE_LSB = 3;
40 localparam TRAP_CAUSE_WL = 6;
41 localparam TRAP_DBG_CAUSE_LSB = 9;
42 localparam TRAP_DBG_CAUSE_WL = 3;
43
44 localparam DEFAULT_ILEN = 32;
45 localparam DEFAULT_XLEN = 32;
46 localparam MAX_XLEN = 64;
47 localparam DEFAULT_NRET = 1;
48
49 localparam CSR_MAX_SIZE = 4096;
50
51 const string format_header_str = "%8s | RVFI | %8s | %6s | %8s |
%8s | %s | %03s | %08s | %03s | %08s | %03s | %08s |
%08s | %s";
52 const string format_instr_str = "%8s | RVFI | %8d | %6d | %8x |
%8s | %s | x%-2d | %8x | x%-2d | %8x | x%-2d | %16x";
53 `define FORMAT_INSTR_STR_MACRO "%8s | RVFI | %8d | %6d | %8x |
%8s | %s | x%-2d | %8x | x%-2d | %8x | x%-2d | %16x | %s"
54 const string format_mem_str = "| %02s | %08x | %08s |";
55
56 `endif // __UVMA_RVFI_CONSTANTS_SV__

```

6. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_csr_if.sv (123 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16
17
18 `ifndef __UVMA_RVFI_CSR_IF_SV__
19 `define __UVMA_RVFI_CSR_IF_SV__
20
21 /**
22  * Encapsulates all signals and clocking of RVFI csruction in-
terface. Used by
23  * monitor,
24  */
25 interface uvma_rvfi_csr_if
26     import uvma_rvfi_pkg::*;
27     #(int XLEN=DEFAULT_XLEN)
28     (
29         input clk,
30         input reset_n,
31
32         input [XLEN-1:0] rvfi_csr_rmask,
33         input [XLEN-1:0] rvfi_csr_wmask,
34         input [XLEN-1:0] rvfi_csr_rdata,
35         input [XLEN-1:0] rvfi_csr_wdata
36     );
37
38 //
39 // Local variables
40 //
41
42 //
43 // Begin module code
44 //
45
46 /**
47  * Used by target DUT.
48  */
49 clocking dut_cb @(posedge clk or reset_n);
50 endclocking : dut_cb
51
52 /**
53  * Used by uvma_rvfi_csr_mon.c.
54  */
55 clocking mon_cb @(posedge clk or reset_n);
56     input #1step
57         rvfi_csr_rmask,

```

```

58     rvfi_csr_wmask,
59     rvfi_csr_rdata,
60     rvfi_csr_wdata;
61   endclocking : mon_cb;
62
63   modport passive_mp    (clocking mon_cb);
64
65   endinterface : uvma_rvfi_csr_if
66
67   interface uvma_rvfi_unified_csr_if
68     import uvma_rvfi_pkg::*;
69     #( int MAX_CSR=4096,
70       int XLEN=DEFAULT_XLEN)
71     (
72       input          clk,
73       input          reset_n,
74
75       input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_rmask,
76       input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_wmask,
77       input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_rdata,
78       input [MAX_CSR-1:0][XLEN-1:0] rvfi_named_csr_wdata,
79
80       // Generic READ/WRITE values
81       input [XLEN-1:0] rvfi_csr_addr,
82       input [XLEN-1:0] rvfi_csr_rmask,
83       input [XLEN-1:0] rvfi_csr_wmask,
84       input [XLEN-1:0] rvfi_csr_rdata,
85       input [XLEN-1:0] rvfi_csr_wdata
86     );
87
88     //
89     // Local variables
90     //
91
92     //
93     // Begin module code
94     //
95
96     /**
97      * Used by target DUT.
98      */
99     clocking dut_cb @(posedge clk or reset_n);
100    endclocking : dut_cb
101
102    /**
103     * Used by uvma_rvfi_csr_mon_c.
104     */
105    clocking mon_cb @(posedge clk or reset_n);
106    input #1step
107      rvfi_csr_addr,
108      rvfi_csr_rmask,
109      rvfi_csr_wmask,
110      rvfi_csr_rdata,
111      rvfi_csr_wdata,
112      rvfi_named_csr_rmask,
113      rvfi_named_csr_wmask,
114      rvfi_named_csr_rdata,
115      rvfi_named_csr_wdata;
116    endclocking : mon_cb
117
118    modport passive_mp    (clocking mon_cb);
119
120    endinterface : uvma_rvfi_unified_csr_if
121
122
123 `endif // __UVMA_RVFI_CSR_IF_SV__

```

7. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_instr_if.sv (132 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 //
11 // https://solderpad.org/licenses/
12 //
13 // Unless required by applicable law or agreed to in writing,
14 // software
15 // distributed under the License is distributed on an "AS IS"
16 // BASIS,
17 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
18 // press or implied.
19 // See the License for the specific language governing permis-
20 // sions and
21 // limitations under the License.
22
23 `ifndef __UVMA_RVFI_INSTR_IF_SV__
24 `define __UVMA_RVFI_INSTR_IF_SV__

```

```

21 /**
22  * Encapsulates all signals and clocking of RVFI Instruction
23  * interface. Used by
24  * monitor,
25  */
26 interface uvma_rvfi_instr_if
27   import uvma_rvfi_pkg::*;
28   #(int ILEN=DEFAULT_ILEN,
29     int XLEN=DEFAULT_XLEN)
30   //import uvma_rvfi_pkg::*;
31   (
32     input          clk,
33     input          reset_n,
34
35     input          rvfi_valid,
36     input [ORDER_WL-1:0] rvfi_order,
37     input [ILEN-1:0] rvfi_insn,
38     input [TRAP_WL-1:0] rvfi_trap,
39     input          rvfi_halt,
40     input [RVFI_DBG_WL-1:0] rvfi_dbg,
41     input          rvfi_dbg_mode,
42     input [RVFI_NMIP_WL-1:0] rvfi_nmip,
43     input [RVFI_INTR_WL-1:0] rvfi_intr,
44     input [MODE_WL-1:0] rvfi_mode,
45     input [IXL_WL-1:0] rvfi_ixl,
46     input [XLEN-1:0] rvfi_pc_rdata,
47     input [XLEN-1:0] rvfi_pc_wdata,
48
49     input [GPR_ADDR_WL-1:0] rvfi_rs1_addr,
50     input [XLEN-1:0] rvfi_rs1_rdata,
51
52     input [GPR_ADDR_WL-1:0] rvfi_rs2_addr,
53     input [XLEN-1:0] rvfi_rs2_rdata,
54
55     input [GPR_ADDR_WL-1:0] rvfi_rs3_addr,
56     input [XLEN-1:0] rvfi_rs3_rdata,
57
58     input [GPR_ADDR_WL-1:0] rvfi_rd1_addr,
59     input [XLEN-1:0] rvfi_rd1_wdata,
60
61     input [GPR_ADDR_WL-1:0] rvfi_rd2_addr,
62     input [XLEN-1:0] rvfi_rd2_wdata,
63
64     input [XLEN-1:0] rvfi_mem_addr,
65     input [XLEN-1:0] rvfi_mem_rdata,
66     input [XLEN/8-1:0] rvfi_mem_rmask,
67     input [XLEN-1:0] rvfi_mem_wdata,
68     input [XLEN/8-1:0] rvfi_mem_wmask
69   );
70
71   //
72   // Local variables
73   //
74
75   bit [CYCLE_CNT_WL-1:0] cycle_cnt = 0;
76
77   //
78   // Begin module code
79   //
80
81   always @(posedge clk) begin
82     cycle_cnt <= cycle_cnt + 1;
83   end
84
85   /**
86    * Used by target DUT.
87    */
88   clocking dut_cb @(posedge clk or reset_n);
89   endclocking : dut_cb
90
91   /**
92    * Used by uvma_rvfi_instr_mon_c.
93    */
94   clocking mon_cb @(posedge clk or reset_n);
95   input #1step
96     cycle_cnt,
97     rvfi_valid,
98     rvfi_order,
99     rvfi_insn,
100     rvfi_trap,
101     rvfi_halt,
102     rvfi_dbg,
103     rvfi_dbg_mode,
104     rvfi_nmip,
105     rvfi_intr,
106     rvfi_mode,
107     rvfi_ixl,
108     rvfi_pc_rdata,
109     rvfi_pc_wdata,
110     rvfi_rs1_addr,
111     rvfi_rs1_rdata,
112     rvfi_rs2_addr,
113     rvfi_rs2_rdata,
114     rvfi_rs3_addr,
115     rvfi_rs3_rdata,
116     rvfi_rd1_addr,
117     rvfi_rd1_wdata,

```



```

117     rvfi_rd2_addr,
118     rvfi_rd2_wdata,
119     rvfi_mem_addr,
120     rvfi_mem_rdata,
121     rvfi_mem_rmask,
122     rvfi_mem_wdata,
123     rvfi_mem_wmask;
124 endclocking : mon_cb
125
126 modport passive_mp    (clocking mon_cb);
127
128 endinterface : uvma_rvfi_instr_if
129
130 `endif // __UVMA_RVFI_INSTR_IF_SV__
131
132

```

8. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_instr_mon.sv (299 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
7 // (the "License");
8 // you may not use this file except in compliance with the Li-
9 // cense.
10 // You may obtain a copy of the License at
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // software
14 // distributed under the License is distributed on an "AS IS"
15 // BASIS,
16 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17 // press or implied.
18 // See the License for the specific language governing permis-
19 // sions and
20 // limitations under the License.
21 //
22
23 `ifndef __UVMA_RVFI_INSTR_MON_SV__
24 `define __UVMA_RVFI_INSTR_MON_SV__
25
26 /**
27  * Component sampling transactions from a Clock & Reset virtu-
28  * al interface
29  * (uvma_rvfi_instr_if).
30  */
31 class uvma_rvfi_instr_mon_c#(int ILEN=DEFAULT_ILEN,
32                             int XLEN=DEFAULT_XLEN) extends
33 uvmm_monitor;
34
35 // Objects
36 uvma_rvfi_cfg_c    cfg;
37 uvma_rvfi_cntxt_c  cntxt;
38
39 // TLM
40 uvmm_analysis_port#(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN))
41 ap;
42
43 // State of monitor
44 bit last_dcsr_nmip = 0;
45 bit [uvma_rvfi_csr_seq_item_c#(ILEN,XLEN)-1:0] dcsr_ret_data;
46
47 longint unsigned supported_csrs_addrs [$];
48 string supported_csrs_names [$];
49
50 string log_tag = "RVFIMONLOG";
51
52 `uvm_component_utils_begin(uvma_rvfi_instr_mon_c)
53   `uvm_field_int(last_dcsr_nmip, UVM_DEFAULT)
54   `uvm_field_int(dcsr_ret_data, UVM_DEFAULT)
55   `uvm_field_object(cfg, UVM_DEFAULT | UVM_REFER-
56 ENCE)
57   `uvm_field_object(cntxt, UVM_DEFAULT | UVM_REFER-
58 ENCE)
59   `uvm_component_utils_end
60
61 /**
62  * Default constructor.
63  */
64 extern function new(string name="uvma_rvfi_instr_mon",
65 uvmm_component parent=null);
66
67 /**
68  * 1. Ensures cfg & cntxt handles are not null.
69  * 2. Builds ap.
70  */
71 extern virtual function void build_phase(uvm_phase phase);
72
73 /**
74  * Oversees monitoring via monitor_clk() and monit-
75 or_reset() tasks in parallel
76  * forks.
77

```

```

68  */
69  extern virtual task run_phase(uvm_phase phase);
70
71  /**
72  * Monitor the instruction interface
73  */
74  extern virtual task monitor_rvfi_instr();
75
76  endclass : uvma_rvfi_instr_mon_c
77
78  function uvma_rvfi_instr_mon_c::new(string name="uvma_r-
79 vfi_instr_mon", uvmm_component parent=null);
80
81  super.new(name, parent);
82
83  endfunction : new
84
85  function void uvma_rvfi_instr_mon_c::build_phase(uvm_phase
86 phase);
87
88  super.build_phase(phase);
89
90  if ($test$plusargs("log_rvfi"))
91    set_report_id_verbosity(log_tag, UVM_HIGH);
92
93  void'(uvm_con-
94 fig_db#(uvma_rvfi_cfg_c#(ILEN,XLEN))::get(this, "", "cfg", cfg));
95  if (!cfg) begin
96    `uvm_fatal("CFG", "Configuration handle is null")
97  end
98
99  void'(uvm_con-
100 fig_db#(uvma_rvfi_cntxt_c#(ILEN,XLEN))::get(this, "", "cntxt",
101 cntxt));
102  if (!cntxt) begin
103    `uvm_fatal("CNTXT", "Context handle is null")
104  end
105
106  ap = new("ap", this);
107
108  endfunction : build_phase
109
110  task uvma_rvfi_instr_mon_c::run_phase(uvm_phase phase);
111
112  super.run_phase(phase);
113
114  cfg.core_cfg.get_support-
115 ed_csrs_names(supported_csrs_names);
116  cfg.core_cfg.get_support-
117 ed_csrs_addrs(supported_csrs_addrs);
118
119  // TODO Make this prettier
120  if (cfg.enabled) begin
121    while (1) begin
122      wait (cntxt.instr_vif[0].reset_n == 1'b0);
123      wait (cntxt.instr_vif[0].reset_n == 1'b1);
124
125      fork
126        begin
127          monitor_rvfi_instr();
128        end
129        begin
130          wait (cntxt.instr_vif[0].reset_n ==
131 1'b0);
132        end
133      join_any
134    end
135  end
136  endtask : run_phase
137
138  task uvma_rvfi_instr_mon_c::monitor_rvfi_instr();
139
140  while(1) begin
141    @(cntxt.instr_vif[0].mon_cb)
142
143    for(int i = 0; i < cfg.nret; i++)
144      begin
145        int nret_id;
146        bit monitor_condition;
147
148        nret_id = i;
149        monitor_condition = (!cfg.core_cfg.unified_traps) ?
150 (cntxt.instr_vif[nret_id].mon_cb.rvfi_valid ||
151 cntxt.instr_vif[nret_id].mon_cb.rvfi_trap[0]) :
152 (cntxt.instr_vif[nret_id].mon_cb.rvfi_valid);
153
154        if (monitor_condition) begin
155          uvma_rvfi_instr_seq_item_c#(ILEN,XLEN) mon_trn;
156
157          mon_trn = uvma_r-
158 vfi_instr_seq_item_c#(ILEN,XLEN)::type_id::create("mon_trn");
159
160          mon_trn.nret_id = nret_id;
161          mon_trn.cycle_cnt =
162 cntxt.instr_vif[nret_id].mon_cb.cycle_cnt;
163          mon_trn.order =
164 cntxt.instr_vif[nret_id].mon_cb.rvfi_order;
165

```

```

155         mon_trn.insn =
cntxt.instr_vif[nret_id].mon_cb.rvfi_insn;
156         mon_trn.trap =
cntxt.instr_vif[nret_id].mon_cb.rvfi_trap;
157         mon_trn.halt =
cntxt.instr_vif[nret_id].mon_cb.rvfi_halt;
158         mon_trn.dbg =
cntxt.instr_vif[nret_id].mon_cb.rvfi_dbg;
159         mon_trn.dbg_mode =
cntxt.instr_vif[nret_id].mon_cb.rvfi_dbg_mode;
160         mon_trn.nmip =
cntxt.instr_vif[nret_id].mon_cb.rvfi_nmip;
161         mon_trn.intr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_intr;
162         $cast(mon_trn.mode,
cntxt.instr_vif[nret_id].mon_cb.rvfi_mode);
163         mon_trn.ixl =
cntxt.instr_vif[nret_id].mon_cb.rvfi_ixl;
164         mon_trn.pc_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_pc_rdata;
165         mon_trn.pc_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_pc_wdata;
166         mon_trn.rs1_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs1_addr;
167         mon_trn.rs1_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs1_rdata;
168         mon_trn.rs2_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs2_addr;
169         mon_trn.rs2_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs2_rdata;
170         mon_trn.rs3_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs3_addr;
171         mon_trn.rs3_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rs3_rdata;
172         mon_trn.rd1_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd1_addr;
173         mon_trn.rd1_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd1_wdata;
174         mon_trn.rd2_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd2_addr;
175         mon_trn.rd2_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_rd2_wdata;
176         mon_trn.mem_addr =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_addr;
177         mon_trn.mem_rdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_rdata;
178         mon_trn.mem_rmask =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_rmask;
179         mon_trn.mem_wdata =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_wdata;
180         mon_trn.mem_wmask =
cntxt.instr_vif[nret_id].mon_cb.rvfi_mem_wmask;
181         // Get the CSRs
182         if (!cfg.core_cfg.disable_all_csr_checks) begin
183             if (!cfg.unified_csr_vif) begin
184                 foreach (cntxt.csr_vif[csr]) begin
185                     uvma_rvfi_csr_seq_item_c csr_trn =
186                         uvma_rvfi_csr_seq_item_c::create({csr, "_trn"});
187                     csr_trn.csr = csr;
188                     csr_trn.nret_id = nret_id;
189                     csr_trn.rmask = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_rmask;
190                     csr_trn.wmask = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_wmask;
191                     csr_trn.rdata = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_rdata;
192                     csr_trn.wdata = cntxt.csr_vif[csr]
[nret_id].mon_cb.rvfi_csr_wdata;
193                     mon_trn.name_csrs[csr] = csr_trn;
194                 end
195             end
196         else begin
197             uvma_rvfi_csr_seq_item_c csr_trn;
198             longint unsigned addr;
199             csr_trn = uvma_r-
vfi_csr_seq_item_c#(XLEN)::type_id::create({addr, "_trn"});
200             addr =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_addr;
201             csr_trn.addr = addr;
202             if
(cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wmask != 0) begin
203                 csr_trn.wmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wmask;
204                 csr_trn.wdata =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_wdata;
205                 csr_trn.rmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_rmask;
206                 csr_trn.rdata =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_csr_rdata;
207                 mon_trn.addr_csrs[addr] = csr_trn;
208             end
209         end
210     end
211 end

```

```

218         foreach(supported_csrs_addrs[i]) begin
219             addr = supported_csrs_addrs[i];
220             csr_trn = uvma_r-
vfi_csr_seq_item_c#(XLEN)::type_id::create({addr, "_trn"});
221             csr_trn.addr = addr;
222             csr_trn.wmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_wmask[addr];
223             csr_trn.wdata =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_wdata[addr];
224             csr_trn.rmask =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_rmask[addr];
225             csr_trn.rdata =
cntxt.csr_unified_vif[nret_id].mon_cb.rvfi_named_csr_rdata[addr];
226             mon_trn.addr_csrs[addr] = csr_trn;
227         end
228     end
229     // Decode interrupts that need to be commu-
nicated to ISS (external or NMI bus faults)
230     if (mon_trn.intr) begin
231         // The cause of the interrupt should be
in the "rdata" field of the mcause CSR RVFI port
232         bit [XLEN-1:0] csr_mcause;
233         if (!cfg.unified_csr_vif)
234             csr_mcause = mon_trn.name_c-
srs["mcause"].rdata;
235         else
236             csr_mcause = mon_trn.addr_c-
srs[MCAUSE].rdata;
237         // External interrupt
238         if (csr_mcause[31]) begin
239             // NMI - Load fault
240             if (cfg.nmi_load_fault_enabled &&
241                 csr_mcause[XLEN-2:0] == cfg.nmi_load_fault_cause) begin
242                 mon_trn.insn_nmi_load_fault = 1;
243             end
244             // NMI - Store fault
245             else if (cfg.nmi_store_fault_enabled
246                 && csr_mcause[XLEN-2:0] == cfg.nmi_store_fault_cause) begin
247                 mon_trn.insn_nmi_store_fault =
1;
248             end
249             // External interrupt
250             else begin
251                 mon_trn.insn_interrupt = 1;
252                 mon_trn.insn_interrupt_id =
{ 1'b0, csr_mcause[XLEN-2:0] };
253             end
254         end
255         if (!cfg.unified_csr_vif && mon_trn.name_c-
srs.exists("dcsr"))
256             dcsr_ret_data = mon_trn.name_c-
srs["dcsr"].get_csr_retirement_data();
257         else if (cfg.unified_csr_vif &&
258             mon_trn.addr_csrs.exists(DCSR))
259             dcsr_ret_data = mon_trn.addr_c-
srs[DCSR].get_csr_retirement_data();
260         // In debug mode, detect NMIP event for a
data bus error
261         if (mon_trn.dbg_mode &&
262             !last_dcsr_nmip &&
263             mon_trn.nmip[0] &&
264             dcsr_ret_data[3])
265             begin
266                 `uvm_info("RVFIMON", $sformatf("Debug
NMIP"), UVM_LOW);
267                 if (mon_trn.nmip[1] == 0) begin
268                     mon_trn.insn_nmi_load_fault = 1;
269                 end else begin
270                     mon_trn.insn_nmi_store_fault = 1;
271                 end
272             end
273         // Latch the last DCSR NMIP bit to detect
positive assertion of dcsr.nmip
274         last_dcsr_nmip = dcsr_ret_data[3];
275     end
276     // Detect instruction bus fault
277     if (cfg.insn_bus_fault_enabled &&
278         mon_trn.is_trap() &&
279         mon_trn.get_trap_cause() ==
cfg.insn_bus_fault_cause) begin
280         mon_trn.insn_bus_fault = 1;
281         `uvm_info("RVFIMON", $sformatf("Detected bus
fault"), UVM_LOW)
282     end
283     `uvm_info(log_tag, $sformatf("%s", mon_trn.con-
vert2string()), UVM_HIGH);
284     ap.write(mon_trn);
285 end
286 end

```

```

294 end
295
296 endtask : monitor_rvfi_instr
297
298 `endif // __UVMA_RVFI_INSTR_MON_SV__
299

```

9. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_macros.sv (23 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 //
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // software
14 // distributed under the License is distributed on an "AS IS"
15 // BASIS,
16 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17 // press or implied.
18 // See the License for the specific language governing permis-
19 // sions and
20 // limitations under the License.
21
22 `ifndef __UVMA_RVFI_MACROS_SV__
23 `define __UVMA_RVFI_MACROS_SV__
24
25 `endif // __UVMA_RVFI_MACROS_SV__

```

10. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_mon_trn_logger.sv (199 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 //
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // software
14 // distributed under the License is distributed on an "AS IS"
15 // BASIS,
16 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17 // press or implied.
18 // See the License for the specific language governing permis-
19 // sions and
20 // limitations under the License.
21
22 `ifndef __UVMA_RVFI_MON_TRN_LOGGER_SV__
23 `define __UVMA_RVFI_MON_TRN_LOGGER_SV__
24
25 /**
26 * Component writing Rvfi monitor transactions rvfi data to
27 * disk as plain text.
28 */
29 class uvma_rvfi_mon_trn_logger_c#(int ILEN=DEFAULT_ILEN,
30 int XLEN=DEFAULT_XLEN) ex-
31 tends uvml_logs_mon_trn_logger_c#(
32 .T_TRN (uvml_trn_seq_item_c),
33 .T_CFG (uvma_rvfi_cfg_c ),
34 .T_CNTXT(uvma_rvfi_cntxt_c )
35 );
36
37 uvml_analys-
38 is_imp_rvfi_instr#(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN), uvma_r-
39 vfi_mon_trn_logger_c) instr_export;
40
41 uvma_rvfi_instr_table_seq_item_c#(ILEN,XLEN) in-
42 str_table[bit[XLEN-1:0]];
43
44 `uvm_component_param_utils(uvma_rvfi_mon_trn_logger_c)
45
46 /**
47 * Default constructor.
48 */
49 function new(string name="uvma_rvfi_mon_trn_logger",
50 uvml_component parent=null);
51
52 super.new(name, parent);
53
54 instr_export = new("instr_export", this);
55

```

```

45
46 endfunction : new
47
48 /**
49 * Build phase - attempt to load a firmware itb file (in-
50 struction table file)
51 */
52 function void build_phase(uvm_phase phase);
53
54 read_itb_file();
55
56 endfunction : build_phase
57
58 /**
59 * Read the itb file and build an instruction library
60 */
61 function void read_itb_file();
62
63 string file;
64
65 if ($value$plusargs("itb_file=%s", file)) begin
66 int unsigned itb_lineno;
67 int fh;
68
69 fh = $fopen(file, "r");
70 if (fh == 0) begin
71
72 `uvm_fatal("RVFIMONLOG", $sformatf("Could not open
73 itb file: %s", file));
74 end
75
76 while (!$feof(fh)) begin
77 string str;
78
79 int unsigned num;
80 int unsigned c;
81 string src_code_q[$];
82 uvma_rvfi_instr_table_seq_item_c instr;
83
84 itb_lineno++;
85 c = $fscanf(fh, "%1s%d", str, num);
86 instr = uvma_r-
87 vfi_instr_table_seq_item_c#(ILEN,XLEN)::type_id::create("instr");
88
89 `uvm_info("RVFIMONLOG", $sformatf("Parsing
90 line_num: %0d with %0d lines of src", itb_lineno, num), UVM_HIGH)
91 if (c == -1) begin
92 break;
93 end
94 else if (num == 0) begin
95 src_code_q.push_back("N/A");
96 end
97 else begin
98 // Accumulate the source code
99 repeat (num) begin
100 itb_lineno++;
101 c = $fscanf(fh, "%1s", str);
102 c = $fgets(str, fh);
103
104 src_code_q.push_back(str.substr(0,str.len()-1));
105 end
106 end
107
108 instr.src_code = new[src_code_q.size()]
109 (src_code_q);
110
111 // Parse the context of the source code
112 c = $fscanf(fh, "%d %s %s %d %s %d", instr.addr,
113 instr.src_function, instr.filename, instr.lineno, instr.mcode, num);
114 for (int i = 0; i < num; i++) begin
115 string asm_str;
116
117 c = $fscanf(fh, "%s", asm_str);
118 if (i == 0)
119 instr.asm = asm_str;
120 else
121 instr.asm = { instr.asm, " ", asm_str };
122 end
123 itb_lineno++;
124
125 // Fatal error if instruction already exists at
126 the respective address
127 if (instr_table.exists(instr.addr)) begin
128 `uvm_info("RVFIMONLOG", $sformatf("%s", in-
129 str_table[instr.addr].sprint(), UVM_LOW));
130 `uvm_fatal("RVFIMONLOG", $sformatf("%s", in-
131 str_table[instr.addr].sprint(), UVM_LOW));
132 `uvm_fatal("RVFIMONLOG", $sformatf("%0d: In-
133 struction at address: 0x%08x already exists in instruction table",
134 itb_lineno, instr.addr));
135 end
136 instr_table[instr.addr] = instr;
137
138 `uvm_info("RVFIMONLOG", $sformatf("%s", in-
139 str_table[instr.addr].sprint(), UVM_DEBUG));
140 end
141 end
142
143 endfunction : read_itb_file
144

```

```

132 /**
133  * Writes contents of t to disk
134  */
135 virtual function void write(uvml_trn_seq_item_c t);
136 endfunction : write
137
138 virtual function void
139 write_rvfi_instr(uvma_rvfi_instr_seq_item_c#(ILEN,XLEN) t);
140 string instr;
141 instr = $sformatf(format_instr_str, $sformatf("%t",
142 $time),
143               t.cycle_cnt,
144               t.order,
145               t.pc_rdata,
146               t.get_insn_word_str(),
147               get_mode_str(t.mode),
148               t.rs1_addr, t.rs1_rdata,
149               t.rs2_addr, t.rs2_rdata,
150               t.rd1_addr, t.rd1_wdata);
151 if (t.mem_wmask)
152   instr = $sformatf({"%s ", format_mem_str}, instr, "
153   WR", t.mem_addr, t.get_mem_data_string());
154   else if (t.mem_rmask)
155     instr = $sformatf({"%s ", format_mem_str}, instr, "
156   RD", t.mem_addr, t.get_mem_data_string());
157   else
158     instr = $sformatf("%s | -- | ----- | ----- |",
159 instr);
160 if (t.insn_interrupt)
161   instr = $sformatf("%s INTR %0d", instr, t.insn_inter-
162 rupt_id);
163 if (t.insn_nmi_load_fault)
164   instr = $sformatf("%s NMI LOAD", instr);
165 if (t.insn_nmi_store_fault)
166   instr = $sformatf("%s NMI STORE", instr);
167 if (t.dbg)
168   instr = $sformatf("%s DEBUG", instr);
169 if (instr_table.exists(t.pc_rdata)) begin
170   instr = $sformatf("%s %s %s %0d - %s",
171 instr,
172   in-
173   str_table[t.pc_rdata].src_function,
174   instr_table[t.pc_rdata].filename,
175   instr_table[t.pc_rdata].lineno,
176   instr_table[t.pc_rdata].asm);
177 end
178 fwrite(instr);
179 endfunction : write_rvfi_instr
180 /**
181  * Writes log header to disk
182  */
183 virtual function void print_header();
184
185 string banner = {160{"-"};};
186
187 fwrite(banner);
188 fwrite($sformatf(format_header_str,
189 "TIME", "CYCLE", "ORDER", "PC", "IN-
190 STR", "M", "RS1", "RS1_DATA", "RS2", "RS2_DATA", "RD", "RD", "IN-
191 MEM_DATA", "INSTRUCTION"));
192 fwrite(banner);
193
194 endfunction : print_header
195 endclass : uvma_rvfi_mon_trn_logger_c
196
197 `endif // __UVMA_RVFI_MON_TRN_LOGGER_SV__
198
199

```

11. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_pkg.sv (71 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
7 // (the "License");
8 // you may not use this file except in compliance with the Li-
9 // cense.
10 // You may obtain a copy of the License at
11 // https://solderpad.org/licenses/
12 // Unless required by applicable law or agreed to in writing,
13 // software
14 // distributed under the License is distributed on an "AS IS"
15 // BASIS,
16 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-

```

```

17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20 //
21 //
22 //
23 // Pre-processor macros
24 `include "uvm_macros.svh"
25 `include "uvml_hrtbt_macros.svh"
26 `include "uvma_rvfi_macros.svh"
27
28 /**
29  * Encapsulates all the types needed for an UVM agent capable
30  * of driving and/or
31  * monitoring Clock & Reset.
32  */
33 package uvma_rvfi_pkg;
34
35 import uvm_pkg ::*;
36 import uvml_hrtbt_pkg ::*;
37 import uvml_trn_pkg ::*;
38 import uvml_logs_pkg ::*;
39 import uvma_core_cntrl_pkg ::*;
40
41 // Analysis implementation declarations
42 `uvm_analysis_imp_decl(_rvfi_instr)
43
44 // Constants / Structs / Enums
45 `include "uvma_rvfi_constants.sv"
46 `include "uvma_rvfi_tdefs.sv"
47
48 // Objects
49 `include "uvma_rvfi_cfg.sv"
50 `include "uvma_rvfi_cntxt.sv"
51
52 // High-level transactions
53 `include "seq/uvma_rvfi_csr_seq_item.sv"
54 `include "seq/uvma_rvfi_instr_seq_item.sv"
55 `include "seq/uvma_rvfi_instr_table_seq_item.sv"
56
57 // Agent components
58 `include "uvma_rvfi_mon_trn_logger.sv"
59 `include "uvma_rvfi_instr_mon.sv"
60 `include "uvma_rvfi_agent.sv"
61
62 endpackage : uvma_rvfi_pkg
63
64 // Interface(s) / Module(s) / Checker(s)
65 `include "uvma_rvfi_instr_if.sv"
66 `include "uvma_rvfi_csr_if.sv"
67 `include "uvma_rvfi_assert.sv"
68
69 `endif // __UVMA_RVFI_PKG_SV__
70
71

```

12. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/uvma_rvfi_tdefs.sv (109 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20 //
21 //
22 `ifndef __UVMA_RVFI_TDEFS_SV__
23 `define __UVMA_RVFI_TDEFS_SV__
24
25 typedef enum bit[MODE_WL-1:0] {
26   UVMA_RVFI_U_MODE = 0,
27   UVMA_RVFI_S_MODE = 1,
28   UVMA_RVFI_RESERVED_MODE = 2,
29   UVMA_RVFI_M_MODE = 3
30 } uvma_rvfi_mode;
31
32 typedef struct packed {
33   bit [MAX_XLEN-1:0] nret_id;
34   bit [MAX_XLEN-1:0] cycle_cnt;

```

```

31 bit [MAX_XLEN-1:0] order;
32 bit [MAX_XLEN-1:0] insn;
33 bit [MAX_XLEN-1:0] trap;
34 bit [MAX_XLEN-1:0] halt;
35 bit [MAX_XLEN-1:0] intr;
36 bit [MAX_XLEN-1:0] mode;
37 bit [MAX_XLEN-1:0] ixl;
38 bit [MAX_XLEN-1:0] dbg;
39 bit [MAX_XLEN-1:0] dbg_mode;
40 bit [MAX_XLEN-1:0] nmip;
41
42 bit [MAX_XLEN-1:0] insn_interrupt;
43 bit [MAX_XLEN-1:0] insn_interrupt_id;
44 bit [MAX_XLEN-1:0] insn_bus_fault;
45 bit [MAX_XLEN-1:0] insn_nmi_store_fault;
46 bit [MAX_XLEN-1:0] insn_nmi_load_fault;
47
48 bit [MAX_XLEN-1:0] pc_rdata;
49 bit [MAX_XLEN-1:0] pc_wdata;
50
51 bit [MAX_XLEN-1:0] rs1_addr;
52 bit [MAX_XLEN-1:0] rs1_rdata;
53
54 bit [MAX_XLEN-1:0] rs2_addr;
55 bit [MAX_XLEN-1:0] rs2_rdata;
56
57 bit [MAX_XLEN-1:0] rs3_addr;
58 bit [MAX_XLEN-1:0] rs3_rdata;
59
60 bit [MAX_XLEN-1:0] rd1_addr;
61 bit [MAX_XLEN-1:0] rd1_wdata;
62
63 bit [MAX_XLEN-1:0] rd2_addr;
64 bit [MAX_XLEN-1:0] rd2_wdata;
65
66 bit [MAX_XLEN-1:0] mem_addr;
67 bit [MAX_XLEN-1:0] mem_rdata;
68 bit [MAX_XLEN-1:0] mem_rmask;
69 bit [MAX_XLEN-1:0] mem_wdata;
70 bit [MAX_XLEN-1:0] mem_wmask;
71
72 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_valid;
73 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_addr;
74 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_rdata;
75 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_rmask;
76 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_wdata;
77 bit [CSR_MAX_SIZE-1:0] [MAX_XLEN-1:0] csr_wmask;
78
79 } st_rvfi;
80
81 `define ST_NUM_WORDS ($size(st_rvfi)/MAX_XLEN)
82 parameter ST_NUM_WORDS = ($size(st_rvfi)/MAX_XLEN);
83
84 typedef bit [ST_NUM_WORDS-1:0] [MAX_XLEN-1:0] vector_rvfi;
85 typedef union {
86     st_rvfi rvfi;
87     vector_rvfi array;
88 } union_rvfi;
89
90 function string get_mode_str(uvma_rvfi_mode mode);
91 case (mode)
92     UVMA_RVFI_U_MODE: return "U";
93     UVMA_RVFI_M_MODE: return "M";
94     UVMA_RVFI_S_MODE: return "S";
95 endcase
96
97 return "?";
98
99 endfunction : get_mode_str
100
101 function bit is_csr_insn (st_rvfi rvfi);
102     return (rvfi.insn[6:0] == 'b1110011);
103 endfunction : is_csr_insn
104
105 function bit [11:0] get_insn_rs1 (st_rvfi rvfi);
106     return rvfi.insn[31:20];
107 endfunction
108
109 `endif // __UVMA_RVFI_TDEFS_SV__

```

13.openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/seq/uvma_rvfi_csr_seq_item.sv (88 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 //
4 // Licensed under the Solderpad Hardware Licence, Version 2.0
5 // (the "License");
6 // You may not use this file except in compliance with the Li-
7 // cense.
8 // You may obtain a copy of the License at
9 //
10 // https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
13 // software
14 // distributed under the License is distributed on an "AS IS"
15 // BASIS,
16 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17 // press or implied.

```

```

13 // See the License for the specific language governing permis-
14 // sions and
15 // limitations under the License.
16
17 `ifndef __UVMA_RVFI_CSR_SEQ_ITEM_SV__
18 `define __UVMA_RVFI_CSR_SEQ_ITEM_SV__
19
20
21 /**
22  * Object created by RVFI agent sequences extending uvma_r-
23  * vfi_seq_base_c.
24  */
25
26 class uvma_rvfi_csr_seq_item_c#(int XLEN=DEFAULT_XLEN) extends
27     uvml_trn_seq_item_c;
28
29     rand int unsigned          nret_id;
30
31     string                    csr;
32     rand bit [XLEN-1:0]      addr;
33
34     rand bit [XLEN-1:0]      rmask;
35     rand bit [XLEN-1:0]      wmask;
36     rand bit [XLEN-1:0]      rdata;
37     rand bit [XLEN-1:0]      wdata;
38
39     static protected string _log_format_string = "0x%08x %s
40     0x%01x 0x%08x";
41
42     `uvvm_object_param_utils_begin(uvma_rvfi_csr_seq_item_c)
43     `uvvm_field_int(nret_id, UVM_DEFAULT)
44
45     `uvvm_field_int(rmask, UVM_DEFAULT)
46     `uvvm_field_int(wmask, UVM_DEFAULT)
47     `uvvm_field_int(rdata, UVM_DEFAULT)
48     `uvvm_field_int(wdata, UVM_DEFAULT)
49     `uvvm_object_utils_end
50
51 /**
52  * Default constructor.
53  */
54 extern function new(string name="uvma_rvfi_csr_seq_item");
55
56 /**
57  * Fetch the value of the CSR at instruction retirement
58  * taking into account write data and write mask
59  */
60 extern virtual function bit[uvma_r-
61     vfi_csr_seq_item_c::XLEN-1:0] get_csr_retirement_data();
62
63 /**
64  * One-liner log message
65  */
66 extern function string convert2string();
67
68 endclass : uvma_rvfi_csr_seq_item_c
69
70 `pragma protect begin
71 function uvma_rvfi_csr_seq_item_c::new(string name="uvma_r-
72     vfi_csr_seq_item");
73
74     super.new(name);
75
76 endfunction : new
77
78 function bit [uvma_rvfi_csr_seq_item_c::XLEN-1:0] uvma_r-
79     vfi_csr_seq_item_c::get_csr_retirement_data();
80
81 // Any bits with wmask set should use the wdata
82 // All other bits should use the rdata
83 // rmask is not used
84 return (wmask & wdata) | (~wmask & rdata);
85
86 endfunction : get_csr_retirement_data
87
88 function string uvma_rvfi_csr_seq_item_c::convert2string();
89
90 convert2string = $sformatf("csr: %s rdata(mask) 0x%08x:
91 0x%08x, wdata(mask) 0x%08x:0x%08x",
92     csr, rdata, rmask, wdata,
93     wmask);
94
95 endfunction : convert2string
96
97 `pragma protect end
98 `endif // __UVMA_RVFI_CSR_SEQ_ITEM_SV__

```

14.openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/seq/uvma_rvfi_instr_seq_item.sv (415 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 //
4 // Licensed under the Solderpad Hardware Licence, Version 2.0
5 // (the "License");
6 // You may not use this file except in compliance with the Li-
7 // cense.
8 // You may obtain a copy of the License at
9 //
10 // https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
13 // software
14 // distributed under the License is distributed on an "AS IS"
15 // BASIS,
16 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
17 // press or implied.

```



```

8 //      https://solderpad.org/licenses/
9 //
10 // Unless required by applicable law or agreed to in writing,
software
11 // distributed under the License is distributed on an "AS IS"
BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 // See the License for the specific language governing permis-
sions and
14 // limitations under the License.
15
16
17 `ifndef __UVMA_RVFI_INSTR_SEQ_ITEM_SV__
18 `define __UVMA_RVFI_INSTR_SEQ_ITEM_SV__
19
20
21 /**
22  * Object created by Rvfi agent sequences extending uvma_r-
vfi_seq_base_c.
23  */
24 class uvma_rvfi_instr_seq_item_c#(int ILEN=DEFAULT_ILEN,
25                                int XLEN=DEFAULT_XLEN) ex-
tends uvml_trn_seq_item_c;
26
27     rand int unsigned          nret_id;
28     rand bit [CYCLE_CNT_WL-1:0] cycle_cnt;
29     rand bit [ORDER_WL-1:0] order;
30     rand bit [ILEN-1:0] insn;
31     rand bit [TRAP_WL-1:0] trap;
32     rand bit halt;
33     rand bit [RVFI_INTR_WL-1:0] intr;
34     rand uvma_rvfi_mode mode;
35     rand bit [IXL_WL-1:0] ixl;
36     rand bit [RVFI_DBG_WL-1:0] dbg;
37     rand bit dbg_mode;
38     rand bit [RVFI_NMIP_WL-1:0] nmip;
39
40     rand bit insn_interrupt;
41     rand int unsigned insn_interrupt_id;
42     rand bit insn_bus_fault;
43     rand bit insn_nmi_store_fault;
44     rand bit insn_nmi_load_fault;
45
46     rand bit [XLEN-1:0] pc_rdata;
47     rand bit [XLEN-1:0] pc_wdata;
48
49     rand bit [GPR_ADDR_WL-1:0] rs1_addr;
50     rand bit [XLEN-1:0] rs1_rdata;
51
52     rand bit [GPR_ADDR_WL-1:0] rs2_addr;
53     rand bit [XLEN-1:0] rs2_rdata;
54
55     rand bit [GPR_ADDR_WL-1:0] rs3_addr;
56     rand bit [XLEN-1:0] rs3_rdata;
57
58     rand bit [GPR_ADDR_WL-1:0] rd1_addr;
59     rand bit [XLEN-1:0] rd1_wdata;
60
61     rand bit [GPR_ADDR_WL-1:0] rd2_addr;
62     rand bit [XLEN-1:0] rd2_wdata;
63
64     rand bit [XLEN-1:0] mem_addr;
65     rand bit [XLEN-1:0] mem_rdata;
66     rand bit [XLEN-1:0] mem_rmask;
67     rand bit [XLEN-1:0] mem_wdata;
68     rand bit [XLEN-1:0] mem_wmask;
69
70     uvma_rvfi_csr_seq_item_c name_csrs[string];
71     uvma_rvfi_csr_seq_item_c addr_csrs[longint unsigned];
72
73     static protected string _log_format_string = "0x%08x %s
0x%01x 0x%08x";
74
75     `uvm_object_param_utils_begin(uvma_rvfi_instr_seq_item_c)
76         `uvm_field_int(cycle_cnt, UVM_DEFAULT)
77         `uvm_field_int(order, UVM_DEFAULT)
78         `uvm_field_int(insn, UVM_DEFAULT)
79         `uvm_field_int(trap, UVM_DEFAULT)
80         `uvm_field_int(halt, UVM_DEFAULT)
81         `uvm_field_int(dbg, UVM_DEFAULT)
82         `uvm_field_int(dbg_mode, UVM_DEFAULT)
83         `uvm_field_int(nmip, UVM_DEFAULT)
84         `uvm_field_int(intr, UVM_DEFAULT)
85         `uvm_field_int(insn_interrupt, UVM_DEFAULT)
86         `uvm_field_int(insn_interrupt_id, UVM_DEFAULT)
87         `uvm_field_int(insn_bus_fault, UVM_DEFAULT)
88         `uvm_field_int(insn_nmi_load_fault, UVM_DEFAULT)
89         `uvm_field_int(insn_nmi_store_fault, UVM_DEFAULT)
90         `uvm_field_enum(uvma_rvfi_mode, mode, UVM_DEFAULT)
91         `uvm_field_int(ixl, UVM_DEFAULT)
92         `uvm_field_int(pc_rdata, UVM_DEFAULT)
93         `uvm_field_int(pc_wdata, UVM_DEFAULT)
94         `uvm_field_int(rs1_addr, UVM_DEFAULT)
95         `uvm_field_int(rs1_rdata, UVM_DEFAULT)
96         `uvm_field_int(rs2_addr, UVM_DEFAULT)
97         `uvm_field_int(rs2_rdata, UVM_DEFAULT)
98         `uvm_field_int(rs3_addr, UVM_DEFAULT)
99         `uvm_field_int(rs3_rdata, UVM_DEFAULT)
100        `uvm_field_int(rd1_addr, UVM_DEFAULT)
101        `uvm_field_int(rd1_wdata, UVM_DEFAULT)

```

```

102        `uvm_field_int(rd2_addr, UVM_DEFAULT)
103        `uvm_field_int(rd2_wdata, UVM_DEFAULT)
104        `uvm_field_int(mem_addr, UVM_DEFAULT)
105        `uvm_field_int(mem_rmask, UVM_DEFAULT)
106        `uvm_field_int(mem_rdata, UVM_DEFAULT)
107        `uvm_field_int(mem_wmask, UVM_DEFAULT)
108        `uvm_field_int(mem_wdata, UVM_DEFAULT)
109
110        `uvm_field_aa_object_string(name_csrs, UVM_DEFAULT)
111        `uvm_field_aa_object_int(addr_csrs, UVM_DEFAULT)
112        `uvm_object_utils_end
113
114    /**
115     * Default constructor.
116     */
117    extern function new(string name="uvma_rvfi_seq_item");
118
119    /**
120     * One-liner log message
121     */
122    extern function string convert2string();
123
124    /**
125     * Get instruction hex string with compressed instructions
displayed
126     */
127    extern function string get_insn_word_str();
128
129    /**
130     * Get string representing memory data
131     */
132    extern function string get_mem_data_string();
133
134    /**
135     * Decode compressed instruction
136     */
137    extern function bit is_compressed_insn();
138
139    /**
140     * Decode if instruction is a trap
141     */
142    extern function bit is_trap();
143
144    /*
145     * Decode if instruction is a synchronous trap with debug
entry
146     */
147    extern function bit is_debug_entry_trap();
148
149    /*
150     * Decode if instruction is a synchronous trap without de-
bug entry
151     */
152    extern function bit is_nondebug_entry_trap();
153
154    /*
155     * Retrieve trap cause field
156     */
157    extern function bit [TRAP_CAUSE_WL-1:0] get_trap_cause();
158
159    /*
160     * Retrieve trap debug cause
161     */
162    extern function bit [TRAP_DBG_CAUSE_WL-1:0]
get_trap_debug_cause();
163
164    extern function st_rvfi seq2rvfi();
165
166    extern function void rvfi2seq(st_rvfi rvfi);
167
168    endclass : uvma_rvfi_instr_seq_item_c
169
170    `pragma protect begin
171
172    173    function uvma_rvfi_instr_seq_item_c::new(string name="uvma_r-
vfi_seq_item");
174
175        super.new(name);
176
177    endfunction : new
178
179    function string uvma_rvfi_instr_seq_item_c::convert2string();
180
181        convert2string = $sprintf("Order: %0d, insn: 0x%08x, pc:
0x%08x, nret_id: %0d, mode: %, ixl: 0x%01x",
182                                order, insn, pc_rdata,
this.nret_id, mode.name(), ixl);
183        if (rs1_addr)
184            convert2string = $sprintf("%s rs1: x%0d = 0x%08x", con-
vert2string, rs1_addr, rs1_rdata);
185        if (rs2_addr)
186            convert2string = $sprintf("%s rs2: x%0d = 0x%08x", con-
vert2string, rs2_addr, rs2_rdata);
187        if (rs3_addr)
188            convert2string = $sprintf("%s rs3: x%0d = 0x%08x", con-
vert2string, rs3_addr, rs3_rdata);
189        if (rd1_addr)
190            convert2string = $sprintf("%s rd: x%0d = 0x%08x", con-
vert2string, rd1_addr, rd1_wdata);
191        if (rd2_addr)

```

```

192     convert2string = $sformatf("%s rd2: x%0d = 0x%08x", con-
vert2string, rd2_addr, rd2_wdata);
193     if (trap)
194     convert2string = $sformatf("%s TRAP %0d", conver-
t2string, trap);
195     if (halt)
196     convert2string = $sformatf("%s HALT", convert2string);
197     if (insn_interrupt)
198     convert2string = $sformatf("%s INTR %0d", conver-
t2string, this.insn_interrupt_id);
199     if (insn_nmi_load_fault)
200     convert2string = $sformatf("%s NMI LOAD", conver-
t2string);
201     if (insn_nmi_store_fault)
202     convert2string = $sformatf("%s NMI STORE", conver-
t2string);
203     if (insn_bus_fault)
204     convert2string = $sformatf("%s INSN_BUS_FAULT", conver-
t2string);
205     if (dbg)
206     convert2string = $sformatf("%s DEBUG", convert2string);
207
208     endfunction : convert2string
209
210     function string uvma_r-
vfi_instr_seq_item_c::get_insn_word_str();
211
212     if (is_compressed_insn)
213     return $sformatf("----%04x", insn[15:0]);
214
215     return $sformatf("%08x", insn);
216
217     endfunction : get_insn_word_str
218
219     function string uvma_r-
vfi_instr_seq_item_c::get_mem_data_string();
220     string mem_data_str;
221
222     if (mem_wmask) begin
223     for (int i = 0; i < XLEN/8; i++) begin
224     if (mem_wmask[i])
225     mem_data_str = $sformatf("%02x%s", mem_wdata[i*8+:
8], mem_data_str);
226     else
227     mem_data_str = $sformatf("--%s", mem_data_str);
228     end
229     end
230     else if (mem_rmask) begin
231     for (int i = 0; i < XLEN/8; i++) begin
232     if (mem_rmask[i])
233     mem_data_str = $sformatf("%02x%s", mem_rdata[i*8+:
8], mem_data_str);
234     else
235     mem_data_str = $sformatf("--%s", mem_data_str);
236     end
237     end
238     return mem_data_str;
239
240     endfunction : get_mem_data_string
241
242
243     function bit uvma_rvfi_instr_seq_item_c::is_compressed_insn();
244
245     if (insn[31:16] == 0 && insn[1:0] inside {0,1,2})
246     return 1;
247
248     return 0;
249
250     endfunction : is_compressed_insn
251
252     function bit uvma_rvfi_instr_seq_item_c::is_trap();
253
254     return trap[TRAP_EXCP_LSB];
255
256     endfunction : is_trap
257
258     function bit uvma_r-
vfi_instr_seq_item_c::is_debug_entry_trap();
259
260     return trap[TRAP_DBG_ENTRY_LSB];
261
262     endfunction : is_debug_entry_trap
263
264     function bit uvma_r-
vfi_instr_seq_item_c::is_nondebug_entry_trap();
265
266     return trap[TRAP_NONDBG_ENTRY_LSB];
267
268     endfunction : is_nondebug_entry_trap
269
270     function bit [TRAP_CAUSE_WL-1:0] uvma_r-
vfi_instr_seq_item_c::get_trap_cause();
271
272     return trap[TRAP_CAUSE_LSB +: TRAP_CAUSE_WL];
273
274     endfunction : get_trap_cause
275
276     function bit [TRAP_DBG_CAUSE_WL-1:0] uvma_r-
vfi_instr_seq_item_c::get_trap_debug_cause();
277
278     return trap[TRAP_DBG_CAUSE_LSB +: TRAP_DBG_CAUSE_WL];

```

```

279
280     endfunction : get_trap_debug_cause
281
282     function st_rvfi uvma_rvfi_instr_seq_item_c::seq2rvfi();
283
284     st_rvfi rvfi;
285     longint unsigned csr_index = 0;
286
287     rvfi.nret_id = nret_id;
288     rvfi.cycle_cnt = cycle_cnt;
289     rvfi.order = order;
290     rvfi.insn = insn;
291     rvfi.trap = trap;
292     rvfi.halt = halt;
293     rvfi.intr = intr;
294     rvfi.mode = mode;
295     rvfi.ixl = ixl;
296     rvfi.dbg = dbg;
297     rvfi.dbg_mode = dbg_mode;
298     rvfi.nmip = nmip;
299
300     rvfi.insn_interrupt = insn_interrupt;
301     rvfi.insn_interrupt_id = insn_interrupt_id;
302     rvfi.insn_bus_fault = insn_bus_fault;
303     rvfi.insn_nmi_store_fault = insn_nmi_store_fault;
304     rvfi.insn_nmi_load_fault = insn_nmi_load_fault;
305
306     rvfi.pc_rdata = pc_rdata;
307     rvfi.pc_wdata = pc_wdata;
308
309     rvfi.rs1_addr = rs1_addr;
310     rvfi.rs1_rdata = rs1_rdata;
311
312     rvfi.rs2_addr = rs2_addr;
313     rvfi.rs2_rdata = rs2_rdata;
314
315     rvfi.rs3_addr = rs3_addr;
316     rvfi.rs3_rdata = rs3_rdata;
317
318     rvfi.rd1_addr = rd1_addr;
319     rvfi.rd1_wdata = rd1_wdata;
320
321     rvfi.rd2_addr = rd2_addr;
322     rvfi.rd2_wdata = rd2_wdata;
323
324     rvfi.mem_addr = mem_addr;
325     rvfi.mem_rdata = mem_rdata;
326     rvfi.mem_rmask = mem_rmask;
327     rvfi.mem_wdata = mem_wdata;
328     rvfi.mem_wmask = mem_wmask;
329
330     foreach(addr_csrs[i]) begin
331     longint unsigned addr = addr_csrs[i].addr;
332
333     rvfi.csr_valid[addr] = 1;
334     rvfi.csr_addr [addr] = addr_csrs[i].addr;
335     rvfi.csr_rdata[addr] = addr_csrs[i].rdata;
336     rvfi.csr_rmask[addr] = addr_csrs[i].rmask;
337     rvfi.csr_wdata[addr] = addr_csrs[i].wdata;
338     rvfi.csr_wmask[addr] = addr_csrs[i].wmask;
339     end
340     foreach(name_csrs[i]) begin
341     longint unsigned addr = csr_name2addr[i];
342
343     rvfi.csr_valid[addr] = 1;
344     rvfi.csr_addr [addr] = csr_name2addr[i];
345     rvfi.csr_rdata[addr] = name_csrs[i].rdata;
346     rvfi.csr_rmask[addr] = name_csrs[i].rmask;
347     rvfi.csr_wdata[addr] = name_csrs[i].wdata;
348     rvfi.csr_wmask[addr] = name_csrs[i].wmask;
349     end
350     return rvfi;
351
352     endfunction : seq2rvfi
353
354     function void uvma_rvfi_instr_seq_item_c::rvfi2seq(st_rvfi
rvfi);
355
356     nret_id = rvfi.nret_id;
357     cycle_cnt = rvfi.cycle_cnt;
358     order = rvfi.order;
359     insn = rvfi.insn;
360     trap = rvfi.trap;
361     halt = rvfi.halt;
362     intr = rvfi.intr;
363     mode = rvfi.mode;
364     ixl = rvfi.ixl;
365     dbg = rvfi.dbg;
366     dbg_mode = rvfi.dbg_mode;
367     nmip = rvfi.nmip;
368
369     insn_interrupt = rvfi.insn_interrupt;
370     insn_interrupt_id = rvfi.insn_interrupt_id;
371     insn_bus_fault = rvfi.insn_bus_fault;
372     insn_nmi_store_fault = rvfi.insn_nmi_store_fault;
373     insn_nmi_load_fault = rvfi.insn_nmi_load_fault;
374
375     pc_rdata = rvfi.pc_rdata;
376     pc_wdata = rvfi.pc_wdata;
377
378     rs1_addr = rvfi.rs1_addr;

```



```

379 rs1_rdata = rvfi.rs1_rdata;
380
381 rs2_addr = rvfi.rs2_addr;
382 rs2_rdata = rvfi.rs2_rdata;
383
384 rs3_addr = rvfi.rs3_addr;
385 rs3_rdata = rvfi.rs3_rdata;
386
387 rd1_addr = rvfi.rd1_addr;
388 rd1_wdata = rvfi.rd1_wdata;
389
390 rd2_addr = rvfi.rd2_addr;
391 rd2_wdata = rvfi.rd2_wdata;
392
393 mem_addr = rvfi.mem_addr;
394 mem_rdata = rvfi.mem_rdata;
395 mem_rmask = rvfi.mem_rmask;
396 mem_wdata = rvfi.mem_wdata;
397 mem_wmask = rvfi.mem_wmask;
398
399 for (int i = 0; i < CSR_MAX_SIZE; i++) begin
400     if (rvfi.csr_valid[i]) begin
401         uvma_rvfi_csr_seq_item_c csr_trn = uvma_r-
vfi_csr_seq_item_c#(XLEN)::type_id::create({rvfi.csr_addr[i],
"-trn"});
402         csr_trn.addr = rvfi.csr_addr[i];
403         csr_trn.rdata = rvfi.csr_rdata[i];
404         csr_trn.rmask = rvfi.csr_rmask[i];
405         csr_trn.wdata = rvfi.csr_wdata[i];
406         csr_trn.wmask = rvfi.csr_wmask[i];
407         addr_csrs[rvfi.csr_addr[i]] = csr_trn;
408     end
409 end
410 endfunction : rvfi2seq
411
412 `pragma protect end
413
414 `endif // __UVMA_RVFI_SEQ_ITEM_SV__
415

```

15. openhwgroup_core-v-verif/lib/uvvm_agents/uvma_rvfi/seq/ uvma_rvfi_instr_table_seq_item.sv (78 lines)

```

1 // Copyright 2020 OpenHW Group
2 // Copyright 2020 Datum Technology Corporation
3 //
4 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
5 // you may not use this file except in compliance with the Li-
cense.
6 // You may obtain a copy of the License at
7 //
8 //     https://solderpad.org/licenses/
9 //
10 // Unless required by applicable law or agreed to in writing,
software
11 // distributed under the License is distributed on an "AS IS"
BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 // See the License for the specific language governing permis-
sions and
14 // limitations under the License.
15
16
17 `ifndef __UVMA_RVFI_INSTR_TABLE_SEQ_ITEM_SV__
18 `define __UVMA_RVFI_INSTR_TABLE_SEQ_ITEM_SV__
19

```

```

20
21 /**
22  * Object created by Rvfi agent sequences extending uvma_r-
vfi_seq_base_c.
23  */
24 class uvma_rvfi_instr_table_seq_item_c#(int ILEN=DEFAULT_ILEN,
25
int XLEN=DEFAULT_XLEN)
extends uvml_trn_seq_item_c;
26
27
28 bit[XLEN-1:0] addr;
29 string mcode;
30 string asm;
31 string src_function;
32 string filename;
33 int unsigned lineno;
34 string src_code[];
35
36 static protected string _log_format_string = "0x%08x %s
0x%01x 0x%08x";
37
38 `uvm_ob-
ject_param_utils_begin(uvma_rvfi_instr_table_seq_item_c)
39 `uvm_field_string(src_function , UVM_DEFAULT)
40 `uvm_field_int(addr , UVM_DEFAULT)
41 `uvm_field_string(mcode , UVM_DEFAULT)
42 `uvm_field_string(asm , UVM_DEFAULT)
43 `uvm_field_string(src_function , UVM_DEFAULT)
44 `uvm_field_string(filename , UVM_DEFAULT)
45 `uvm_field_int(lineno , UVM_DEFAULT |
UVM_DEC)
46 `uvm_field_array_string(src_code , UVM_DEFAULT)
47 `uvm_object_utils_end
48
49 /**
50  * Default constructor.
51  */
52 extern function new(string name="uvma_r-
vfi_instr_table_seq_item");
53
54 /**
55  * One-liner log message
56  */
57 extern function string convert2string();
58
59 endclass : uvma_rvfi_instr_table_seq_item_c
60
61 `pragma protect begin
62
63 function uvma_rvfi_instr_table_seq_item_c::new(string
name="uvma_rvfi_instr_table_seq_item");
64
65 super.new(name);
66
67 endfunction : new
68
69 function string uvma_r-
vfi_instr_table_seq_item_c::convert2string();
70
71 // Fixme: implement me!
72
73 endfunction : convert2string
74
75 `pragma protect end
76
77
78 `endif // __UVMA_RVFI_INSTR_TABLE_SEQ_ITEM_SV__

```

step-compare 테스트벤치

13 files • 3,086 lines

API — uvmt_cv32e40p_constants.sv

Table 1283. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_constants.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_CONSTANTS_SV__		`endif // __UVMT_CV32E40P_CONSTANTS_SV__

API — uvmt_cv32e40p_debug_assert.sv

Table 1284. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_debug_assert.sv — module / interface / package (1개)

이름	종류
uvmt_cv32e40p_debug_assert	module

Table 1285. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_debug_assert.sv — localparam (2개)

상수	형/폭	값
WFI_INSTR_MASK		32'hffffff
WFI_INSTR_DATA		32'h10500073

API — uvmt_cv32e40p_dut_chk.sv

Table 1286. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_dut_chk.sv — module / interface / package (1개)

이름	종류
uvmt_cv32e40p_dut_chk	module

Table 1287. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_dut_chk.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_DUT_CHK_SV__		/**

API — uvmt_cv32e40p_dut_wrap.sv

Table 1288. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_dut_wrap.sv — module / interface / package (1개)

이름	종류
uvmt_cv32e40p_dut_wrap	module

Table 1289. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_dut_wrap.sv — parameter (1개)

파라미터	형/폭	기본값
PULP_XPULP		0

Table 1290. `openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_dut_wrap.sv` — `#define / `define` 매크로 (1개)

매크로	인자	정의
<code>__UVMT_CV32E40P_DUT_WRAP_SV__</code>		<code>/**</code>

API — `uvmt_cv32e40p_interrupt_assert.sv`

Table 1291. `openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_interrupt_assert.sv` — `module / interface / package` (1개)

이름	종류
<code>uvmt_cv32e40p_interrupt_assert</code>	<code>module</code>

Table 1292. `openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_interrupt_assert.sv` — `local-param` (5개)

상수	형/폭	값
<code>NUM_IRQ</code>		32
<code>VALID_IRQ_MASK</code>		<code>32'hffff_0888</code>
<code>WFI_INSTR_MASK</code>		<code>32'hfffffff</code>
<code>WFI_INSTR_DATA</code>		<code>32'h10500073</code>
<code>WFI_WAKEUP_LATENCY</code>		40

Table 1293. `openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_interrupt_assert.sv` — 포트 (22개)

포트	방향	형/폭
<code>clk</code>	input	
<code>clk_i</code>	input	
<code>rst_ni</code>	input	
<code>fetch_enable_i</code>	input	
<code>irq_i</code>	input	[31:0]
<code>irq_ack_o</code>	input	
<code>irq_id_o</code>	input	[4:0]
<code>debug_req_i</code>	input	
<code>debug_mode_q</code>	input	
<code>mcause_n</code>	input	[5:0]
<code>mip</code>	input	[31:0]
<code>mie_q</code>	input	[31:0]
<code>mie_n</code>	input	[31:0]

포트	방향	형/폭
mstatus_mie	input	
mtvec_mode_q	input	[1:0]
if_stage_instr_rvalid_i	input	
if_stage_instr_rdata_i	input	[31:0]
id_stage_instr_valid_i	input	
id_stage_instr_rdata_i	input	[31:0]
ctrl_fsm_cs	input	[4:0]
branch_taken_ex	input	
core_sleep_o	input	

API — uvmt_cv32e40p_iss_wrap.sv

Table 1294. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_iss_wrap.sv — module / interface / package (1개)

이름	종류
uvmt_cv32e40p_iss_wrap	module

Table 1295. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_iss_wrap.sv — parameter (4개)

파라미터	형/폭	기본값
ROM_START_ADDR	int	'h00000000
ROM_BYTE_SIZE	int	'h0
RAM_BYTE_SIZE	int	'h1B000000
ID	int	0

Table 1296. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_iss_wrap.sv — 포트 (1개)

포트	방향	형/폭
clk_period	input	realtime

Table 1297. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_iss_wrap.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_ISS_WRAP_SV__		/**

API — uvmt_cv32e40p_macros.sv

Table 1298. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_macros.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_MACROS_SV__		`endif // __UVMT_CV32E40P_MACROS_SV__

API — uvmt_cv32e40p_pkg.sv**Table 1299. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_pkg.sv — module / interface / package (1개)**

이름	종류
uvmt_cv32e40p_pkg	package

Table 1300. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_pkg.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_PKG_SV__		// Pre-processor macros

API — uvmt_cv32e40p_step_compare.sv**Table 1301. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_step_compare.sv — module / interface / package (1개)**

이름	종류
uvmt_cv32e40p_step_compare	module

Table 1302. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_step_compare.sv — #define / `define 매크로 (7개)

매크로	인자	정의
__UVMT_CV32E40P_STEP_COMPARE_SV__		// Step-and-Compare between the CV32E40PE40P and Imperas OVPsim ISS
T0_TYPE		"RV32IMC"
CV32E40P_CORE		\$root.uvmt_cv32e40p_tb.dut_wrap.cv32e40p_wrapper_i.core_i
CV32E40P_TRACER		\$root.uvmt_cv32e40p_tb.dut_wrap.cv32e40p_wrapper_i.tracer_i
CV32E40P_RM		\$root.uvmt_cv32e40p_tb.iss_wrap.cpu
CV32E40P_RM_RVVI_STATE		\$root.uvmt_cv32e40p_tb.iss_wrap.cpu.state
CV32E40P_RM_RVVI_CONTROL		\$root.uvmt_cv32e40p_tb.iss_wrap.cpu.control

API — uvmt_cv32e40p_tb.sv**Table 1303. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb.sv — module / interface / package (1개)**

이름	종류
uvmt_cv32e40p_tb	module

Table 1304. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb.sv — parameter (14개)

파라미터	형/폭	기본값
CORE_PARAM_PULP_XPULP	int	0
CORE_PARAM_PULP_CLUSTER	int	0
CORE_PARAM_PULP_ZFINX	int	0

파라미터	형/폭	기본값
CORE_PARAM_PULP_XPULP	int	1
CORE_PARAM_PULP_CLUSTER	int	0
CORE_PARAM_PULP_ZFINX	int	0
CORE_PARAM_PULP_XPULP	int	0
CORE_PARAM_PULP_CLUSTER	int	0
CORE_PARAM_PULP_ZFINX	int	0
CORE_PARAM_NUM_MHPMCOUNTERS	int	`SET_NUM_MHPMCOUNTERS
CORE_PARAM_NUM_MHPMCOUNTERS	int	1
ENV_PARAM_INSTR_ADDR_WIDTH	int	32
ENV_PARAM_INSTR_DATA_WIDTH	int	32
ENV_PARAM_RAM_ADDR_WIDTH	int	22

Table 1305. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb.sv — typedef / using (1개)

새 이름	원래 형
dbg_state_e	enum logic [1:0] {INACTIVE, DBG_TAKEN, DRIVE_REQ}

Table 1306. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_TB_SV__		/**

API — uvmt_cv32e40p_tb_ifs.sv

Table 1307. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb_ifs.sv — module / interface / package (7개)

이름	종류
uvmt_cv32e40p_clk_gen_if	interface
uvmt_cv32e40p_vp_status_if	interface
uvmt_cv32e40p_core_cntrl_if	interface
uvmt_cv32e40p_core_status_if	interface
uvmt_cv32e40p_isa_covg_if	interface
uvmt_cv32e40p_step_compare_if	interface
uvmt_cv32e40p_debug_cov_assert_if	interface

Table 1308. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb_ifs.sv — localparam (14개)

상수	형/폭	값
DEFAULT_CLOCK_PERIOD_PS	real	1500.0
DEFAULT_RESET_DURATION_PS	real	7400.0
BOOT_ADDR	logic [31:0]	32'h0000_0080

상수	형/폭	값
MTVEC_ADDR	logic [31:0]	32'h0000_0000
DM_HALT_ADDR	logic [31:0]	32'h1A11_0800
DM_EXCEPTION_ADDR	logic [31:0]	32'h1A11_1000
HART_ID	logic [31:0]	32'h0000_0000
GO_FETCH_DELAY_CYCLES	int	10
ADDR_BIN_LOW_MIN	logic [31:0]	32'h0000_0000
ADDR_BIN_LOW_MAX	logic [31:0]	32'h0000_FFFF
ADDR_BIN_MED_MIN	logic [31:0]	32'h0001_0000
ADDR_BIN_MED_MAX	logic [31:0]	32'hEFFF_FFFF
ADDR_BIN_HIGH_MIN	logic [31:0]	32'hF000_0000
ADDR_BIN_HIGH_MAX	logic [31:0]	32'hFFFF_FFFF

Table 1309. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb_ifs.sv — 포트 (75개) — 앞 60개만 표시(전체 75개, 본문 소스 참조)

포트	방향	형/폭
tests_passed	output	reg
tests_failed	output	reg
exit_valid	output	reg
exit_value	output	reg [31:0]
clk	input	
fetch_en	output	logic
pulp_clock_en	output	logic
scan_cg_en	output	logic
boot_addr	output	logic [31:0]
mtvec_addr	output	logic [31:0]
dm_halt_addr	output	logic [31:0]
dm_exception_addr	output	logic [31:0]
hart_id	output	logic [31:0]
debug_req	output	logic
load_instr_mem	output	logic
fetch_en	output	
core_busy	input	wire
sec_lvl	input	logic
clk_i	input	
rst_ni	input	

포트	방향	형/폭
fetch_enable_i	input	
irq_i	input	[31:0]
irq_ack_o	input	
irq_id_o	input	[4:0]
mie_q	input	[31:0]
if_stage_instr_rvalid_i	input	
if_stage_instr_rdata_i	input	[31:0]
id_stage_instr_valid_i	input	
id_stage_instr_rdata_i	input	[31:0]
id_stage_is_compressed	input	
id_stage_pc	input	[31:0]
if_stage_pc	input	[31:0]
is_decoding	input	
id_valid	input	
ctrl_fsm_cs	input	wire ctrl_state_e
illegal_insn_i	input	
illegal_insn_q	input	
ecall_insn_i	input	
boot_addr_i	input	[31:0]
debug_req_i	input	
debug_mode_q	input	
dcsr_q	input	[31:0]
depc_q	input	[31:0]
depc_n	input	[31:0]
dm_halt_addr_i	input	[31:0]
dm_exception_addr_i	input	[31:0]
mcause_q	input	[5:0]
mtvec	input	[31:0]
mepc_q	input	[31:0]
tdata1	input	[31:0]
tdata2	input	[31:0]
trigger_match_i	input	
mcountinhibit_q	input	[31:0]
mcycle	input	[63:0]
minstret	input	[63:0]

포트	방향	형/폭
inst_ret	input	
core_sleep_o	input	
fence_i	input	
csr_access	input	
csr_op	input	[1:0]

Table 1310. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb_ifs.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_TB_IFS_SV__		/**

API — uvmt_cv32e40p_tdefs.sv

Table 1311. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tdefs.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_TDEFS_SV__		/**

API — uvmt_cv32e40p_uvm_macros_inc.sv

Table 1312. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_uvm_macros_inc.sv — #define / `define 매크로 (1개)

매크로	인자	정의
__UVMT_CV32E40P_UVM_MACROS_INC_SV__		// Simple inclusion of the uvm_macros.svh file into compilation scope.

1. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_constants.sv (25 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 //     https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16 //
17
18 `ifndef __UVMT_CV32E40P_CONSTANTS_SV__
19 `define __UVMT_CV32E40P_CONSTANTS_SV__
20
21
22
23
24
25 `endif // __UVMT_CV32E40P_CONSTANTS_SV__

```

2. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_debug_assert.sv (510 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 //     https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16 //
17
18 module uvmt_cv32e40p_debug_assert
19     import uvm_pkg::*;
20     import cv32e40p_pkg::*;
21 (
22     uvmt_cv32e40p_debug_cov_assert_if cov_assert_if
23 );
24
25 //
-----
26 // Local parameters
27 //
-----

```

```

28     localparam WFI_INSTR_MASK = 32'hffffffff;
29     localparam WFI_INSTR_DATA = 32'h10500073;
30     //
-----
31     // Local variables
32     //
-----
33     string info_tag = "CV32E40P_DEBUG_ASSERT";
34     logic [31:0] pc_at_dbg_req; // Capture PC when debug_req_i
or ebreak is active
35     logic [31:0] pc_at_ebreak; // Capture PC when ebreak
36     logic [31:0] halt_addr_at_entry;
37     logic halt_addr_at_entry_flag;
38     logic [31:0] exception_addr_at_entry;
39     logic exception_addr_at_entry_flag;
40     logic [31:0] tdata2_at_entry;
41     // Locally track which debug cause should be used
42     logic [2:0] debug_cause_pri;
43     logic [31:0] boot_addr_at_entry;
44
45     // Locally track pc in ID stage to detect first instruction
of debug code
46     logic [31:0] prev_id_pc;
47     logic first_debug_ins_flag;
48     logic first_debug_ins;
49     logic decode_valid;
50     //
-----
51     // Clocking blocks
52     //
-----
53
54     // Single clock, single reset design, use default clocking
55     default clocking @(posedge cov_assert_if.clk_i); endclocking
56     default disable iff !(cov_assert_if.rst_ni);
57
58     assign cov_assert_if.is_ebreak = cov_assert_if.is_decoding &
59
cov_assert_if.id_stage_instr_valid_i &
60         (cov_assert_if.id_stage_instr_rdata_i ==
32'h00100073) &
61         cov_assert_if.id_stage_is_compressed ==
1'b0;
62
63     assign cov_assert_if.is_ceilbreak = cov_assert_if.is_decoding
&
64
cov_assert_if.id_stage_instr_valid_i &
65         (cov_assert_if.id_stage_instr_rdata_i ==
32'h00100073) &
66         cov_assert_if.id_stage_is_compressed ==
1'b1;
67
68     assign cov_assert_if.is_mulhsu = cov_assert_if.is_decoding &
69
cov_assert_if.id_stage_instr_valid_i &
70
cov_assert_if.id_stage_instr_rdata_i[31:25] == 7'h1 &
71
cov_assert_if.id_stage_instr_rdata_i[14:12] == 3'b010 &
72
cov_assert_if.id_stage_instr_rdata_i[6:0] == 7'h33;
73
74
75     assign decode_valid = cov_assert_if.id_stage_instr_valid_i
& cov_assert_if.ctrl_fsm_cs == cv32e40p_pkg::DECODE;
76     // -----
77     // Assertions
78     // -----
79
80     // check that we enter debug mode when expected.
81     // CSR checks are done in other assertions
82     property p_enter_debug;
83         $changed(debug_cause_pri) && (debug_cause_pri !=
3'b000) && !cov_assert_if.debug_mode_q
84         |-> decode_valid [->1:2] ##0
cov_assert_if.debug_mode_q;
85     endproperty
86     a_enter_debug: assert property(p_enter_debug)
87     else
88         `uvm_error(info_tag, $sformatf("Debug mode not
entered after expected cause %d", debug_cause_pri));
89
90     // Check that depc gets the correct value when debug mode is
entered.
91     property p_debug_mode_pc;
92         $rose(first_debug_ins) |-> cov_assert_if.debug_mode_q
&& (prev_id_pc == halt_addr_at_entry) && (cov_assert_if.depc_q ==
pc_at_dbg_req);
93     endproperty
94
95     a_debug_mode_pc: assert property(p_debug_mode_pc)
96     else
97
`uvm_error(info_tag, $sformatf("Debug mode entered
with wrong pc. pc==%08x", prev_id_pc));

```

```

98
99     // Check that debug with cause haltreq is correct
100     property p_debug_mode_ext_req;
101         $rose(cov_assert_if.debug_mode_q) &&
(cov_assert_if.dcsr_q[8:6] == cv32e40p_pkg::DBG_CAUSE_HALTREQ)
102         |-> debug_cause_pri ==
cv32e40p_pkg::DBG_CAUSE_HALTREQ;
103     endproperty
104
105     a_debug_mode_ext_req: assert prop-
erty(p_debug_mode_ext_req)
106     else
107         `uvm_error(info_tag, $sformatf("Debug cause not
correct for haltreq, cause = %d", cov_assert_if.dcsr_q[8:6]));
108
109     // Check that debug with cause ebreak is correct
110     property p_ceilbreak_debug_mode;
111         $rose(cov_assert_if.debug_mode_q) &&
(cov_assert_if.dcsr_q[8:6] == cv32e40p_pkg::DBG_CAUSE_EBREAK)
112         |-> debug_cause_pri == cv32e40p_pkg::DBG_CAUSE_EBREAK;
113     endproperty
114
115     a_ceilbreak_debug_mode: assert prop-
erty(p_ceilbreak_debug_mode)
116     else
117         `uvm_error(info_tag, $sformatf("Debug mode with
wrong cause after ebreak, case = %d", cov_assert_if.dcsr_q[8:6]));
118
119     // c.ebreak without dcsr.ebreakm results in exception at
mtvec
120     // Exclude single stepping as the sequence gets very com-
plicated
121     property p_ceilbreak_exception;
122         disable iff (cov_assert_if.debug_req_i | !
cov_assert_if.rst_ni)
123         $rose(cov_assert_if.is_ceilbreak) &&
cov_assert_if.dcsr_q[15] == 1'b0 && !cov_assert_if.debug_mode_q &&
cov_assert_if.is_decoding && cov_assert_if.id_valid &&
124         !cov_assert_if.debug_req_i && !cov_assert_if.dcsr_q[2]
125         |-> (decode_valid) [->1:2] ##0 !
cov_assert_if.debug_mode_q && (cov_assert_if.mcause_q[5:0] ==
cv32e40p_pkg::EXC_CAUSE_BREAKPOINT)
126
&& (cov_assert_if.mepc_q == pc_at_ebreak) &&
127
(cov_assert_if.id_stage_pc == cov_assert_if.mtvec);
128     endproperty
129
130     a_ceilbreak_exception: assert property(p_ceilbreak_exception)
131     else
132         `uvm_error(info_tag, $sformatf("Exception not
entered correctly after c.ebreak with dcsr.ebreak=0"));
133
134     // ebreak without dcsr.ebreakm results in exception at
mtvec
135     // Exclude single stepping as the sequence gets very com-
plicated
136     property p_ebreak_exception;
137         disable iff (cov_assert_if.debug_req_i | !
cov_assert_if.rst_ni)
138         $rose(cov_assert_if.is_ebreak) &&
cov_assert_if.dcsr_q[15] == 1'b0 && !cov_assert_if.debug_mode_q &&
cov_assert_if.is_decoding && cov_assert_if.id_valid &&
139         !cov_assert_if.debug_req_i && !
cov_assert_if.dcsr_q[2]
140         |-> (decode_valid) [->1:2] ##0 !
cov_assert_if.debug_mode_q && (cov_assert_if.mcause_q[5:0] ==
cv32e40p_pkg::EXC_CAUSE_BREAKPOINT)
141
&& (cov_assert_if.mepc_q == pc_at_ebreak) &&
142
(cov_assert_if.id_stage_pc == cov_assert_if.mtvec);
143     endproperty
144
145     // TODO: Fails formal as above
146     a_ebreak_exception: assert property(p_ebreak_exception)
147     else
148         `uvm_error(info_tag, $sformatf("Exception not
entered correctly after ebreak with dcsr.ebreak=0"));
149
// c.ebreak during debug mode results in relaunch of debug
mode
150
151     property p_ceilbreak_during_debug_mode;
152         $rose(cov_assert_if.is_ceilbreak) ##0
cov_assert_if.debug_mode_q |-> decode_valid [->2] ##0
cov_assert_if.debug_mode_q &&
153
(cov_assert_if.id_stage_pc == halt_addr_at_entry); // TODO should
check no change in dpc and dcsr
154     endproperty
155
156     a_ceilbreak_during_debug_mode: assert prop-
erty(p_ceilbreak_during_debug_mode)
157     else
158         `uvm_error(info_tag, $sformatf("Debug mode not re-
started after c.ebreak"));

```

```

159
160 // ebreak during debug mode results in relaunch
161 property p_ebreak_during_debug_mode;
162 $rose(cov_assert_if.is_ebreak) ##0
cov_assert_if.debug_mode_q |-> decode_valid [->2] ##0
cov_assert_if.debug_mode_q &&
163
(cov_assert_if.id_stage_pc == halt_addr_at_entry); // TODO should
check no change in dpc and dcsr
164 endproperty
165
166 a_ebreak_during_debug_mode: assert prop-
erty(p_ebreak_during_debug_mode)
167 else
168 `uvm_error(info_tag, $sformatf("Debug mode not re-
started after ebreak"));
169
170 // Trigger match results in debug mode
171 property p_trigger_match;
172 cov_assert_if.trigger_match_i ##0
cov_assert_if.tdata1[2] ##0 !cov_assert_if.debug_mode_q ##0
cov_assert_if.id_stage_instr_valid_i ##0 cov_assert_if.is_decoding
173
|-> decode_valid [->2] ##0 (cov_assert_if.debug_mode_q
&& (cov_assert_if.dcsr_q[8:6] == cv32e40p_pkg::DBG_CAUSE_TRIGGER)
&&
174
(cov_assert_if.depc_q == tdata2_at_entry)) &&
175
(cov_assert_if.id_stage_pc == halt_addr_at_entry);
176 endproperty
177
178 a_trigger_match: assert property(p_trigger_match)
179 else
180 `uvm_error(info_tag, $sformatf("Debug mode not
correctly entered after trigger match depc=%08x, tdata2=%08x",
cov_assert_if.depc_q, tdata2_at_entry));
181
182 // Address match without trigger enabled should NOT result
in debug mode
183 property p_trigger_match_disabled;
184 $rose(cov_assert_if.addr_match) && !
cov_assert_if.debug_mode_q |-> ##[1:6] !cov_assert_if.debug_mode_q;
185 endproperty
186
187 a_trigger_match_disabled: assert prop-
erty(p_trigger_match_disabled)
188 else
189 `uvm_error(info_tag, "Trigger match with
tdata2[2]==0 resulted in debug mode");
190
191 // Exception in debug mode results in pc->dm_excep-
tion_addr_i
192 property p_debug_mode_exception;
193 $rose(cov_assert_if.illegal_insn_i) &&
cov_assert_if.debug_mode_q && cov_assert_if.is_decoding |-> (de-
code_valid & cov_assert_if.id_valid) [->2] ##0
cov_assert_if.debug_mode_q && (cov_assert_if.id_stage_pc == excep-
tion_addr_at_entry);
194 endproperty
195
196 a_debug_mode_exception : assert prop-
erty(p_debug_mode_exception)
197 else
198 `uvm_error(info_tag, $sformatf("Exception in debug
mode not handled incorrectly. dm=%d, pc=%08x",
cov_assert_if.debug_mode_q, cov_assert_if.id_stage_pc));
199
200 // ECALL in debug mode results in pc->dm_exception_addr_i
201 property p_debug_mode_ecall;
202 $rose(cov_assert_if.ecall_insn_i) &&
cov_assert_if.debug_mode_q && cov_assert_if.is_decoding &&
cov_assert_if.id_stage_instr_valid_i
203
|-> (decode_valid & cov_assert_if.id_valid) [->1:3]
##0 cov_assert_if.debug_mode_q && (cov_assert_if.id_stage_pc == ex-
ception_addr_at_entry);
204 endproperty
205
206 a_debug_mode_ecall : assert property(p_debug_mode_ecall)
207 else
208 `uvm_error(info_tag, $sformatf("ECALL in debug
mode not handled incorrectly. dm=%d, pc=%08x",
cov_assert_if.debug_mode_q, cov_assert_if.id_stage_pc));
209
210 // IRQ in debug mode are masked
211 property p_irq_in_debug;
212 cov_assert_if.debug_mode_q |-> !
cov_assert_if.irq_ack_o;
213 endproperty
214
215 a_irq_in_debug : assert property(p_irq_in_debug)
216 else
217 `uvm_error(info_tag, $sformatf("IRQ not ignored
while in debug mode"));
218
219 // WFI in debug mode does not sleep
220 property p_wfi_in_debug;
221 cov_assert_if.debug_mode_q &&
$rose(cov_assert_if.is_wfi) |-> ##6 !cov_assert_if.core_sleep_o;

```

```

222 endproperty
223
224 a_wfi_in_debug : assert property(p_wfi_in_debug)
225 else
226 `uvm_error(info_tag, $sformatf("WFI in debug mode
cause core_sleep_o=1"));
227
228 // Debug request while sleeping makes core wake up and
enter debug mode
229 // wit cause=haltreq
230 property p_sleep_debug_req;
231 cov_assert_if.in_wfi && cov_assert_if.debug_req_i |
=> !cov_assert_if.core_sleep_o |-> decode_valid [->1:2] ##0
cov_assert_if.debug_mode_q &&
232
cov_assert_if.dcsr_q[8:6] ==
cv32e40p_pkg::DBG_CAUSE_HALTREQ;
233 endproperty
234
235 a_sleep_debug_req : assert property(p_sleep_debug_req)
236 else
237 `uvm_error(info_tag, $sformatf("Did not exit
sleep(== %d) after debug_req_i. Debug_mode = %d cause = %d",
cov_assert_if.core_sleep_o, cov_assert_if.debug_mode_q,
cov_assert_if.dcsr_q[8:6]));
238
239 // Accessing debug regs in m-mode is illegal
240 property p_debug_regs_mmode;
241 cov_assert_if.debug_mode_q &&
cov_assert_if.id_stage_instr_rdata_i[31:20] inside {'h7B0, 'h7B1,
'h7B2, 'h7B3} |->
242
cov_assert_if.illegal_insn_i;
243 endproperty
244
245 a_debug_regs_mmode : assert property(p_debug_regs_mmode)
246 else
247 `uvm_error(info_tag, "Accessing debug regs in M-
mode did not result in illegal instruction");
248
249 // Exception while single step -> PC is set to exception
handler before
250 // debug
251 property p_single_step_exception;
252
!cov_assert_if.debug_mode_q && cov_assert_if.dcsr_q[2]
&& cov_assert_if.illegal_insn_q |-> ##[1:20]
cov_assert_if.debug_mode_q && (cov_assert_if.depc_q ==
cov_assert_if.mtvec);
253 endproperty
254
255 a_single_step_exception : assert prop-
erty(p_single_step_exception)
256 else
257 `uvm_error(info_tag, "PC not set to exception
handler after single step with exception");
258
259 // Trigger during single step
260 property p_single_step_trigger;
261
!cov_assert_if.debug_mode_q && cov_assert_if.dcsr_q[2]
&& cov_assert_if.addr_match && cov_assert_if.tdata1[2] &&
cov_assert_if.id_stage_instr_valid_i |->
262
##[1:20] cov_assert_if.debug_mode_q &&
(cov_assert_if.dcsr_q[8:6] == cv32e40p_pkg::DBG_CAUSE_TRIGGER) &&
(cov_assert_if.depc_q == pc_at_dbg_req);
263 endproperty
264
265 a_single_step_trigger : assert property
(p_single_step_trigger)
266 else
267 `uvm_error(info_tag, $sformatf("Single step and
trigger error: depc = %08x, cause = %d", cov_assert_if.depc_q,
cov_assert_if.dcsr_q[8:6]));
268
269 // Single step WFI must not result in sleeping
270 property p_single_step_wfi;
271
!cov_assert_if.debug_mode_q && cov_assert_if.dcsr_q[2]
&& cov_assert_if.is_wfi |->
272
decode_valid [->2] ##0
cov_assert_if.debug_mode_q && !cov_assert_if.core_sleep_o;
273 endproperty
274
275 a_single_step_wfi : assert property(p_single_step_wfi)
276 else
277 `uvm_error(info_tag, "Debug mode not entered after
single step WFI or core went sleeping");
278
279 // Executing with single step with no irq results in debug
mode
280 property p_single_step;
281
!cov_assert_if.debug_mode_q && cov_assert_if.dcsr_q[2]
&& !cov_assert_if.dcsr_q[11] && decode_valid |> decode_valid [->1]
##0 cov_assert_if.debug_mode_q;
282 endproperty
283
284 a_single_step: assert property(p_single_step)
285 else

```

```

286         `uvm_error(info_tag, "Debug mode not entered for
single step");
287
288         // dret in M-mode will cause illegal instruction
289         // If pending debug req, illegal insn will not assert
290         // until resume
291         property p_mmode_dret;
292         !cov_assert_if.debug_mode_q && cov_assert_if.is_dret
&& !cov_assert_if.debug_req_i |-> ##1
cov_assert_if.illegal_insn_q;
293     endproperty
294
295     a_mmode_dret : assert property(p_mmode_dret)
296     else
297
298         `uvm_error(info_tag, "Executing dret in M-mode did
not result in illegal instruction");
299
300         // dret in D-mode will restore pc and exit D-mode
301         property p_dmode_dret;
302         cov_assert_if.debug_mode_q && cov_assert_if.is_dret |
> decode_valid [->2] ##0 !cov_assert_if.debug_mode_q &&
(cov_assert_if.id_stage_pc == cov_assert_if.depc_q);
303     endproperty
304
305     a_dmode_dret : assert property(p_dmode_dret)
306     else
307         `uvm_error(info_tag, "Dret did not cause correct
return from debug mode");
308
309         // Check that trigger regs cannot be written from M-mode
310
311         // TSEL, and TDATA3 are tied to zero, hence no register to
check
312         property p_mmode_tdata1_write;
313         !cov_assert_if.debug_mode_q &&
cov_assert_if.csr_access && cov_assert_if.csr_op == 'h1 &&
cov_assert_if.id_stage_instr_rdata_i[31:20] == 'h7A1 |-> ##2
$stable(cov_assert_if.tdata1);
314     endproperty
315
316     a_mmode_tdata1_write : assert prop-
erty(p_mmode_tdata1_write)
317     else
318         `uvm_error(info_tag, "Writing tdata1 from M-mode
not allowed to change register value!");
319
320         property p_mmode_tdata2_write;
321         !cov_assert_if.debug_mode_q &&
cov_assert_if.csr_access && cov_assert_if.csr_op == 'h1 &&
cov_assert_if.id_stage_instr_rdata_i[31:20] == 'h7A2 |-> ##2
$stable(cov_assert_if.tdata2);
322     endproperty
323
324     a_mmode_tdata2_write : assert prop-
erty(p_mmode_tdata2_write)
325     else
326         `uvm_error(info_tag, "Writing tdata2 from M-mode
not allowed to change register value!");
327
328         // Check that mcycle works as expected when not sleeping
329         // Counter can be written an arbitrary value, check that
330         // it changed only when not being written to
331         property p_mcycle_count;
332         !cov_assert_if.mcountinhibit_q[0] && !
cov_assert_if.core_sleep_o && !(cov_assert_if.csr_we_int &&
(cov_assert_if.csr_addr == 12'hB00 || cov_assert_if.csr_addr ==
12'hB80)) |> $changed(cov_assert_if.mcycle);
333     endproperty
334
335     a_mcycle_count : assert property(p_mcycle_count)
336     else
337         `uvm_error(info_tag, "Mcycle not counting when
mcountinhibit[0] is cleared!");
338
339         // Check that minstret works as expected when not sleeping
340         // Check only when not written to
341         property p_minstret_count;
342         !cov_assert_if.mcountinhibit_q[2] &&
cov_assert_if.inst_ret && !cov_assert_if.core_sleep_o
&& !(cov_assert_if.csr_we_int &&
(cov_assert_if.csr_addr == 12'hB02 || cov_assert_if.csr_addr ==
12'hB82)) |> (cov_assert_if.minstret ==
($past(cov_assert_if.minstret)+1));
343     endproperty
344
345     a_minstret_count : assert property(p_minstret_count)
346     else
347         `uvm_error(info_tag, "Minstret not counting when
mcountinhibit[2] is cleared!");
348
349         // Check debug_req_i and irq on same cycle.
350         // Should result in debug mode with regular pc in depc,
351         // not pc from interrupt handler
352         // PC is checked in another assertion
353         property p_debug_req_and_irq;
354         cov_assert_if.debug_req_i &&
cov_assert_if.pending_enabled_irq && cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::DECODE
355         |-> (decode_valid & cov_assert_if.id_valid) [->1:2]

```

```

##0 cov_assert_if.debug_mode_q;
356     endproperty
357
358     a_debug_req_and_irq : assert property(p_debug_req_and_irq)
359     else
360
361         `uvm_error(info_tag, "Debug mode not entered after
debug_req_i and irq on same cycle");
362
363         // debug_req at reset should result in debug mode and no
instructions
364         // executed
365         property p_debug_at_reset;
366         cov_assert_if.ctrl_fsm_cs == cv32e40p_pkg::RESET &&
cov_assert_if.debug_req_i |->
367
368         decode_valid [->1:2] ##0 cov_assert_if.debug_mode_q &&
(cov_assert_if.depc_q == boot_addr_at_entry);
369     endproperty
370
371     a_debug_at_reset : assert property(p_debug_at_reset)
372     else
373         `uvm_error(info_tag, "Debug mode not entered cor-
rectly at reset!");
374
375         // Check that we cover the case where a debug_req_i
376         // comes while flushing due to an illegal insn, causing
377         // dpc to be set to the exception handler entry addr
378         property p_illegal_insn_debug_req;
379
380         (cov_assert_if.ctrl_fsm_cs == cv32e40p_pkg::FLUSH_EX |
cov_assert_if.ctrl_fsm_cs == cv32e40p_pkg::FLUSH_WB) &&
cov_assert_if.illegal_insn_q & cov_assert_if.debug_req_i & !
cov_assert_if.debug_mode_q |-> decode_valid [->1:2] ##0
cov_assert_if.debug_mode_q && cov_assert_if.depc_q ==
cov_assert_if.mtvec;
381     endproperty
382
383     a_illegal_insn_debug_req : assert prop-
erty(p_illegal_insn_debug_req)
384     else
385         `uvm_error(info_tag, "Debug mode not entered cor-
rectly while handling illegal instruction!");
386
387         // Capture internal states for use in checking
388         // -----
389         always @(posedge cov_assert_if.clk_i or negedge
cov_assert_if.rst_ni) begin
390             if(!cov_assert_if.rst_ni) begin
391                 pc_at_dbg_req <= 32'h0;
392                 pc_at_ebreak <= 32'h0;
393             end else begin
394                 // Capture debug pc
395                 if(cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::DBG_TAKEN_ID) begin
396                     pc_at_dbg_req <= cov_assert_if.id_stage_pc;
397                 end else if(cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::DBG_TAKEN_IF) begin
398                     pc_at_dbg_req <= cov_assert_if.if_stage_pc;
399                 end
400             end
401             // Capture pc at ebreak
402             if(cov_assert_if.is_ebreak ||
cov_assert_if.is_cebreak ) begin
403                 pc_at_ebreak <= cov_assert_if.id_stage_pc;
404             end
405         end
406
407         // Keep track of wfi state
408         always @(posedge cov_assert_if.clk_i or negedge
cov_assert_if.rst_ni) begin
409             if (!cov_assert_if.rst_ni) begin
410                 cov_assert_if.in_wfi <= 1'b0;
411             end
412             else begin
413                 // Enter wfi if we have a valid instruction, not in de-
bug mode and not
414                 // single stepping
415                 if (cov_assert_if.is_wfi && !cov_assert_if.debug_mode_q
&& cov_assert_if.is_decoding && cov_assert_if.id_stage_instr_valid_i
& !cov_assert_if.dcsr_q[2]) begin
416                     cov_assert_if.in_wfi <= 1'b1;
417                 end else if (cov_assert_if.pending_enabled_irq ||
cov_assert_if.debug_req_i)
418                     cov_assert_if.in_wfi <= 1'b0;
419             end
420         end
421
422         // Capture dm_halt_addr_i value
423         always@ (posedge cov_assert_if.clk_i or negedge
cov_assert_if.rst_ni) begin
424             if(!cov_assert_if.rst_ni) begin
425                 halt_addr_at_entry_flag <= 1'b0;
426             end else begin
427                 if(!halt_addr_at_entry_flag) begin
428                     if(cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::DBG_TAKEN_ID | cov_assert_if.ctrl_fsm_cs ==

```

```

cv32e40p_pkg::DBG_TAKEN_IF) begin
430     halt_addr_at_entry <=
{cov_assert_if.dm_halt_addr_i[31:2], 2'b00};
431     tdata2_at_entry <= cov_assert_if.tdata2;
432     halt_addr_at_entry_flag <= 1'b1;
433     end
434 end
435

436 // Clear flag while not in dmode or we see ebreak in
debug
437 if(!cov_assert_if.debug_mode_q & halt_ad-
dr_at_entry_flag) | (cov_assert_if.debug_mode_q &
(cov_assert_if.is_ebreak | cov_assert_if.is_cbreak)))
438     halt_addr_at_entry_flag <= 1'b0;
439
440 // Capture boot addr
441 if(cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::B00T_SET)
442     boot_addr_at_entry <=
{cov_assert_if.boot_addr_i[31:2], 2'b00};
443 end
444 end
445 always@ (posedge cov_assert_if.clk_i) begin
446     if((cov_assert_if.illegal_insn_i |
cov_assert_if.ecall_insn_i) & cov_assert_if.pc_set &
cov_assert_if.debug_mode_q)
447         exception_addr_at_entry =
{cov_assert_if.dm_exception_addr_i[31:2], 2'b00};
448     end
449
450     assign cov_assert_if.addr_match =
(cov_assert_if.id_stage_pc == cov_assert_if.tdata2);
451     assign cov_assert_if.dpc_will_hit = (cov_assert_if.depc_n
== cov_assert_if.tdata2);
452     assign cov_assert_if.is_wfi =
cov_assert_if.id_stage_instr_valid_i & cov_assert_if.id_valid &
453 ((cov_assert_if.id_stage_instr_rdata_i & WFI_INSTR_MASK) == WFI_IN-
STR_DATA);
454     assign cov_assert_if.pending_enabled_irq = |
(cov_assert_if.iqrq_i & cov_assert_if.mie_q);
455     assign cov_assert_if.is_dret =
cov_assert_if.id_valid & cov_assert_if.id_stage_instr_valid_i &
cov_assert_if.is_decoding & (cov_assert_if.id_stage_instr_rdata_i ==
32'h7B200073);
456
457 // Track which debug cause should be expected
458 always@ (posedge cov_assert_if.clk_i or negedge
cov_assert_if.rst_ni) begin
459     if(!cov_assert_if.rst_ni) begin
460         debug_cause_pri <= 3'b000;
461     end else begin
462         // Debug evaluated in decode state with valid in-
structions only
463         //if(cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::DECODE & !cov_assert_if.debug_mode_q) begin
464             if(cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::DECODE) begin
465                 if(cov_assert_if.is_decoding &
cov_assert_if.id_stage_instr_valid_i) begin
466                     if(cov_assert_if.trigger_match_i)
467                         debug_cause_pri <= 3'b010;
468                     else if((cov_assert_if.dcsr_q[15]) &
(cov_assert_if.is_ebreak | cov_assert_if.is_cbreak))
469                         debug_cause_pri <= 3'b001;
470                     else if(cov_assert_if.debug_req_i)
471                         debug_cause_pri <= 3'b011;
472                     else if(cov_assert_if.dcsr_q[2])
473                         debug_cause_pri <= 3'b100;
474                     else
475                         debug_cause_pri <= 3'b000;
476                 end
477             end
478         end else if(cov_assert_if.ctrl_fsm_cs ==
cv32e40p_pkg::DBG_TAKEN_IF) begin
479             if(cov_assert_if.debug_req_i) begin
480                 debug_cause_pri <= 3'b011;
481             end else if(cov_assert_if.dcsr_q[2]) begin
482                 debug_cause_pri <= 3'b100;
483             end
484         end
485     end
486 end
487 end
488
489 // Track PC in id stage to detect first instruction of de-
bug code
490 always@ (posedge cov_assert_if.clk_i or negedge
cov_assert_if.rst_ni) begin
491     if(!cov_assert_if.rst_ni) begin
492         prev_id_pc <= 32'h0;
493         first_debug_ins_flag <= 1'b0;
494         first_debug_ins <= 1'b0;
495     end else begin
496         prev_id_pc <= cov_assert_if.id_stage_pc;
497         first_debug_ins <= 1'b0;
498         if(cov_assert_if.debug_mode_q) begin
499             if(!first_debug_ins_flag) begin
500                 if(cov_assert_if.is_decoding &
cov_assert_if.id_stage_instr_valid_i) begin

```

```

501         first_debug_ins_flag <= 1'b1;
502         first_debug_ins <= 1'b1;
503     end
504 end
505 end else begin
506     first_debug_ins_flag <= 1'b0;
507 end
508 end
509 end
510 endmodule : uvmt_cv32e40p_debug_assert

```

3. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_dut_chk.sv (40 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 // https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16 //
17
18 `ifndef __UVMT_CV32E40P_DUT_CHK_SV__
19 `define __UVMT_CV32E40P_DUT_CHK_SV__
20
21
22 /**
23  * Module encapsulating assertions for CV32E40P RTL DUT wrap-
per.
24  * All ports are SV interfaces.
25  */
26 module uvmt_cv32e40p_dut_chk(
27     uvma_debug_if debug_if
28 );
29
30 `pragma protect begin
31
32 // TODO Add assertions to uvmt_cv32e40p_dut_chk
33
34 `pragma protect end
35
36 endmodule : uvmt_cv32e40p_dut_chk
37
38
39
40 `endif // __UVMT_CV32E40P_DUT_CHK_SV__

```

4. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_dut_wrap.sv (190 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
7 // you may not use this file except in compliance with the Li-
cense.
8 // You may obtain a copy of the License at
9 //
10 // https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
software
13 // distributed under the License is distributed on an "AS IS"
BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
15 // See the License for the specific language governing permis-
sions and
16 // limitations under the License.
17 //
18 ///////////////////////////////////////////////////////////////////
19 //
20 // Modified version of the wrapper for a RI5CY testbench, con-
taining RI5CY,
21 // plus Memory and stdout virtual peripherals.
22 // Contributor: Robert Balas <balasr@student.ethz.ch>
23 // Copyright 2018 Robert Balas <balasr@student.ethz.ch>
24 // Copyright and related rights are licensed under the Solder-
pad Hardware

```



```

25 // License, Version 0.51 (the "License"); you may not use this
file except in
26 // compliance with the License. You may obtain a copy of the
License at
27 // http://solderpad.org/licenses/SHL-0.51. Unless required by
applicable law
28 // or agreed to in writing, software, hardware and materials
distributed under
29 // this License is distributed on an "AS IS" BASIS, WITHOUT
WARRANTIES OR
30 // CONDITIONS OF ANY KIND, either express or implied. See the
License for the
31 // specific language governing permissions and limitations un-
der the License.
32 //
33
34
35 `ifndef __UVM CV32E40P_DUT_WRAP_SV__
36 `define __UVM CV32E40P_DUT_WRAP_SV__
37
38
39 /**
40 * Module wrapper for CV32E40P RTL DUT.
41 */
42 module uvm_cv32e40p_dut_wrap #(
43     // CV32E40P parameters. See User
Manual.
44     parameter PULP_XPULP      =
0,
45     PULP_CLUSTER              =
0,
46     FPU                        =
0,
47     PULP_ZFINX                =
0,
48     NUM_MHPMCOUNTERS          =
1,
49     // Remaining parameters are used
by TB components only
50     INSTR_ADDR_WIDTH          =
32,
51     INSTR_RDATA_WIDTH         =
32,
52     RAM_ADDR_WIDTH            =
22
53 )
54 (
55     uvma_clknrst_if
56     clknrst_if,
57     uvma_interrupt_if         in-
58     terrupt_if,
59     // vp_status_if is driven by ENV
60     uvma_interrupt_if
61     uvmt_cv32e40p_core_cntrl_if
62     core_cntrl_if,
63     uvmt_cv32e40p_core_status_if
64     core_status_if,
65     uvma_obi_memory_if
66     obi_memory_instr_if,
67     uvma_obi_memory_if
68     obi_memory_data_if
69 );
70
71 import uvm_pkg::*; // needed for the UVM messaging service
(' uvm_info(), etc.)
72
73 // signals connecting core to memory
74 logic instr_req;
75 logic instr_gnt;
76 logic instr_rvalid;
77 logic [INSTR_ADDR_WIDTH-1:0] instr_addr;
78 logic [INSTR_RDATA_WIDTH-1:0] instr_rdata;
79
80 logic data_req;
81 logic data_gnt;
82 logic data_rvalid;
83 logic [31:0] data_addr;
84 logic data_we;
85 logic [3:0] data_be;
86 logic [31:0] data_rdata;
87 logic [31:0] data_wdata;
88
89 logic [31:0] irq_vp;
90 logic [31:0] irq_uvma;
91 logic [31:0] irq;
92 logic irq_ack;
93 logic [4:0] irq_id;
94
95 logic debug_req_vp;
96 logic debug_req_uvma;
97 logic debug_req;
98 logic debug_havereset;
99 logic debug_running;
100 logic debug_halted;
101
102 assign debug_if.clk = clknrst_if.clk;
103 assign debug_if.reset_n = clknrst_if.reset_n;

```

```

99 assign debug_req_uvma = debug_if.debug_req;
100
101 assign debug_req = debug_req_vp | debug_req_uvma;
102
103
104 // -----
105 // Instruction bus is read-only, OBI v1.0
106 assign obi_memory_instr_if.we = 'b0;
107 assign obi_memory_instr_if.be = '1;
108 // Data bus is read/write, OBI v1.0
109
110 // -----
111 // Connect to uvma_interrupt_if
112 assign interrupt_if.clk =
113 clknrst_if.clk;
114 assign interrupt_if.reset_n =
115 clknrst_if.reset_n;
116 assign irq_uvma = inter-
117 rupt_if.irq;
118 assign vp_interrupt_if.clk =
119 clknrst_if.clk;
120 assign vp_interrupt_if.reset_n =
121 clknrst_if.reset_n;
122 assign irq_vp = vp_inter-
123 rupt_if.irq;
124 assign interrupt_if.irq_id = irq_id;
125 assign interrupt_if.irq_ack = irq_ack;
126
127 assign irq = irq_uvma | irq_vp;
128
129 // -----
130 // instantiate the core
131 cv32e40p_wrapper #(
132     .PULP_XPULP (PULP_XPULP),
133     .PULP_CLUSTER (PULP_CLUSTER),
134     .FPU (FPU),
135     .PULP_ZFINX (PULP_ZFINX),
136     .NUM_MHPMCOUNTERS (NUM_MHPMCOUNTERS)
137 )
138 cv32e40p_wrapper_i
139 (
140     .clk_i
141     ( clknrst_if.clk ),
142     .rst_ni
143     ( clknrst_if.reset_n ),
144     .pulp_clock_en_i
145     ( core_cntrl_if.pulp_clock_en ),
146     .scan_cg_en_i
147     ( core_cntrl_if.scan_cg_en ),
148     .boot_addr_i
149     ( core_cntrl_if.boot_addr ),
150     .mtvec_addr_i
151     ( core_cntrl_if.mtvec_addr ),
152     .dm_halt_addr_i
153     ( core_cntrl_if.dm_halt_addr ),
154     .hart_id_i
155     ( core_cntrl_if.hart_id ),
156     .dm_exception_addr_i
157     ( core_cntrl_if.dm_exception_addr ),
158     .instr_req_o
159     ( obi_memory_instr_if.req ), // core to agent
160     .instr_gnt_i
161     ( obi_memory_instr_if.gnt ), // agent to core
162     .instr_rvalid_i
163     ( obi_memory_instr_if.rvalid ),
164     .instr_addr_o
165     ( obi_memory_instr_if.addr ),
166     .instr_rdata_i
167     ( obi_memory_instr_if.rdata ),
168     .data_req_o
169     ( obi_memory_data_if.req ),
170     .data_gnt_i
171     ( obi_memory_data_if.gnt ),
172     .data_rvalid_i
173     ( obi_memory_data_if.rvalid ),
174     .data_we_o
175     ( obi_memory_data_if.we ),
176     .data_be_o
177     ( obi_memory_data_if.be ),
178     .data_addr_o
179     ( obi_memory_data_if.addr ),
180     .data_wdata_o
181     ( obi_memory_data_if.wdata ),
182     .data_rdata_i
183     ( obi_memory_data_if.rdata ),
184     // APU not verified in cv32e40p (future work)
185     .apu_req_o
186     (
187         .apu_gnt_i
188         (
189             1'b0
190             .apu_operands_o
191             (
192                 .apu_op_o
193                 (
194                     .apu_flags_o

```



```

(
  168      .apu_rvalid_i      ),
( 1'b0      .apu_result_i      ),
  169      .apu_result_i      ),
( {32{1'b0}}      .apu_flags_i      ),
  170      .apu_flags_i      ), // APU_NUSFLAGS_CPU
( {5{1'b0}}      .irq_i      ),
  171      .irq_i      ),
  172      .irq_uvma      ),
  173      .irq_ack_o      ),
( irq_ack      .irq_id_o      ),
  174      .irq_id_o      ),
( irq_id      .debug_req_i      ) ( de-
  175      .debug_req_uvma      ), ( de-
  176      .debug_havereset_o      ) ( de-
  177      .debug_havereset_o      ),
  178      .debug_running_o      ) ( de-
  179      .debug_running_o      ),
  180      .debug_halted_o      ) ( de-
  181      .debug_halted_o      ),
  182      .fetch_enable_i      ),
( core_cntrl_if.fetch_en      ),
  183      .core_sleep_o      ),
( core_status_if.core_busy      ),
  184      .core_sleep_o      ); // cv32e40p_wrapper_i
  185
  186 endmodule : uvmt_cv32e40p_dut_wrap
  187
  188 `endif // __UVMT_CV32E40P_DUT_WRAP_SV__
  189
  190

```

5. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_interrupt_assert.sv (394 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 //
11 // https://solderpad.org/licenses/
12 //
13 // Unless required by applicable law or agreed to in writing,
14 // software
15 // distributed under the License is distributed on an "AS IS"
16 // BASIS,
17 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
18 // press or implied.
19 // See the License for the specific language governing permis-
20 // sions and
21 // limitations under the License.
22 //
23 module uvmt_cv32e40p_interrupt_assert
24   import uvm_pkg::*;
25   import cv32e40p_pkg::*;
26   (
27     input clk, // Gated clock
28     input clk_i, // Free-running core clock
29     input rst_ni,
30
31     // Core inputs
32     input fetch_enable_i, // external core fetch enable
33
34     // External interrupt interface
35     input [31:0] irq_i,
36     input irq_ack_o, // machine interrupt enable
37     input [4:0] irq_id_o,
38
39     // External debug req (for WFI modeling)
40     input debug_req_i,
41     input debug_mode_q,
42
43     // CSR Interface
44     input [5:0] mcause_n, // mcause_n[5]: interrupt,
45     mcause_n[4]: vector
46
47     input [31:0] mip, // machine interrupt pending
48     input [31:0] mie_q, // machine interrupt enable
49     input [31:0] mie_n, // machine interrupt enable
50     input mstatus_mie, // machine mode interrupt enable
51     input [1:0] mtvec_mode_q, // machine mode interrupt vec-
52     tor mode
53
54     // Instruction fetch stage
55     input if_stage_instr_rvalid_i, // Instruction word
56     is valid
57     input [31:0] if_stage_instr_rdata_i, // Instruction word
58     data
59

```

```

51 // Instruction ID stage (determines executed instruc-
52 tions)
53 input id_stage_instr_valid_i, // instruction word
54 is valid
55 input [31:0] id_stage_instr_rdata_i, // Instruction word
56 data
57 input [4:0] ctrl_fsm_cs, // Controller FSM to
58 get hint for interrupt taken
59
60 // Determine whether to cancel instruction if branch taken
61 input branch_taken_ex,
62
63 // WFI Interface
64 input core_sleep_o
65 );
66
67 //
68
69 // Local parameters
70
71
72 localparam NUM_IRQ = 32;
73
74 localparam VALID_IRQ_MASK = 32'hffff_0888; // Valid external
75 interrupt signals
76
77 localparam WFI_INSTR_MASK = 32'hffffffff;
78 localparam WFI_INSTR_DATA = 32'h10500073;
79
80 localparam WFI_WAKEUP_LATENCY = 40;
81
82 //
83
84 // Local variables
85
86
87 string info_tag = "CV32E40P_IRQ_ASSERT";
88
89 wire [31:0] pending_enabled_irq;
90 wire [31:0] pending_enabled_irq_q;
91
92 wire id_instr_is_wfi; // ID instruction is a WFI
93 reg in_wfi; // Local model of WFI state of core
94
95 reg[31:0] irq_q;
96
97 reg[31:0] next_irq;
98 reg next_irq_valid;
99
100 reg[31:0] next_irq_q;
101 reg next_irq_valid_q;
102 reg[31:0] saved_mie_q;
103
104 reg[31:0] expected_irq;
105 logic expected_irq_ack;
106
107 reg[31:0] last_instr_rdata;
108
109 //
110
111 // Clocking blocks
112
113
114 // Single clock, single reset design, use default clocking
115 default clocking @(posedge clk_i); endclocking
116 default disable iff !(rst_ni);
117
118 //
119
120 // Begin module code
121
122
123 assign pending_enabled_irq = irq_i & mie_n;
124 assign pending_enabled_irq_q = irq_q & mie_n;
125
126 //
127
128 // Interrupt interface checks
129
130
131 // irq_ack_o is always a pulse
132 property p_irq_ack_o_pulse;
133 irq_ack_o |=> !irq_ack_o;
134 endproperty
135 a_irq_ack_o_pulse: assert property(p_irq_ack_o_pulse)
136 else
137   `uvm_error(info_tag,
138     "Interrupt ack was asserted for more than one

```

```

cycle");
125
126 // irq_id_o is never a reserved irq
127 property p_irq_id_o_not_reserved;
128   irq_ack_o |-> VALID_IRQ_MASK[irq_id_o];
129 endproperty
130 a_irq_id_o_not_reserved: assert prop-
erty(p_irq_id_o_not_reserved)
131   else
132     `uvm_error(info_tag,
133       $sformatf("int_id_o output is 0x%x which is
reserved", irq_id_o));
134
135 // irq_id_o is never a disabled irq
136 property p_irq_id_o_mie_enabled;
137   irq_ack_o |-> mie_n[irq_id_o];
138 endproperty
139 a_irq_id_o_mie_enabled: assert prop-
erty(p_irq_id_o_mie_enabled)
140   else
141     `uvm_error(info_tag,
142       $sformatf("irq_id_o output is 0x%x which is
disabled in MIE: 0x%x08x", irq_id_o, mie_n));
143
144 // irq_ack_o cannot be asserted if mstatus_mie is deasserted
145 property p_irq_id_o_mstatus_mie_enabled;
146   irq_ack_o |-> mstatus_mie;
147 endproperty
148 a_irq_id_o_mstatus_mie_enabled: assert prop-
erty(p_irq_id_o_mstatus_mie_enabled)
149   else
150     `uvm_error(info_tag,
151       $sformatf("int_id_o output is 0x%x but
MSTATUS.MIE is disabled", irq_id_o));
152
153 //
-----
154 // Interrupt CSR checks
155 //
-----
-----
156
157 // Coverage for individual interrupt assertions
158 sequence s_irq_taken(irq);
159   irq_i[irq] ##0 mie_q[irq] ##0 mstatus_mie ##0 irq_ack_o
##0 irq_id_o == irq;
160 endsequence : s_irq_taken
161
162 // Interrupt fired, global interrupts enabled, but not taken
due to global MSTATUS.MIE setting
163 property p_irq_masked(irq);
164   irq_i[irq] ##0 !mie_q[irq] ##0 mstatus_mie;
165 endproperty : p_irq_masked
166
167 // Interrupt fired and locally enabled in MIE, but masked
due to MSTATUS.MIE
168 property p_irq_masked_mstatus(irq);
169   irq_i[irq] ##0 mie_q[irq] ##0 !mstatus_mie;
170 endproperty : p_irq_masked_mstatus
171
172 // Interrupt taken
173 property p_irq_taken(irq);
174   s_irq_taken(irq);
175 endproperty : p_irq_taken
176
177 // Interrupt enabled via MIE locally masked
178 property p_irq_masked_then_enabled(irq);
179   irq_i[irq] ##0 !mie_q[irq] ##0 mstatus_mie ##1 irq_i[irq]
##0 mie_q[irq] ##0 mstatus_mie;
180 endproperty : p_irq_masked_then_enabled
181
182 // Interrupt enabled via MSTATUS.MIE locally masked
183 property p_irq_masked_mstatus_then_enabled(irq);
184   irq_i[irq] ##0 mie_q[irq] ##0 !mstatus_mie ##1 irq_i[irq]
##0 mie_q[irq] ##0 mstatus_mie;
185 endproperty : p_irq_masked_mstatus_then_enabled
186
187 // Interrupt request deasserted when enabled but not acked
188 property p_irq_deasserted_while_enabled_not_acked(irq);
189   irq_i[irq] ##0 mie_q[irq] ##0 mstatus_mie ##0 !irq_ack_o
##1
190   !irq_i[irq] ##0 !irq_ack_o;
191 endproperty : p_irq_deasserted_while_enabled_not_acked
192
193 // Interrupt taken in each supported mtvec mode
194 property p_irq_in_mtvec(irq, mtvec);
195   s_irq_taken(irq) ##0 mtvec_mode_q == mtvec;
196 endproperty
197
198 generate for(genvar gv_i = 0; gv_i < NUM_IRQ; gv_i++) be-
gin : gen_irq_cov
199   if (VALID_IRQ_MASK[gv_i]) begin : gen_valid
200     c_irq_masked: cover property(p_irq_masked(gv_i));
201     c_irq_masked_mstatus: cover prop-
erty(p_irq_masked_mstatus(gv_i));
202     c_irq_taken: cover property(p_irq_taken(gv_i));
203     c_irq_masked_then_enabled: cover prop-
erty(p_irq_masked_then_enabled(gv_i));
204     c_irq_masked_mstatus_then_enabled: cover prop-

```

```

erty(p_irq_masked_mstatus_then_enabled(gv_i));
205   c_irq_deasserted_while_enabled_not_acked: cover prop-
erty(p_irq_deasserted_while_enabled_not_acked(gv_i));
206   c_irq_in_mtvvec_fixed: cover prop-
erty(p_irq_in_mtvvec(gv_i, 0));
207   c_irq_in_mtvvec_vector: cover prop-
erty(p_irq_in_mtvvec(gv_i, 1));
208   end
209 end
210 endgenerate
211
212 // Detect arbitration of interrupt assertion
213 always @* begin
214   next_irq_valid = 1'b0;
215   next_irq = 0;
216   casex ({pending_enabled_irq_q[31:16],
pending_enabled_irq_q[11], pending_enabled_irq_q[3],
pending_enabled_irq_q[7]})
217     19'b1?????????????????: begin next_irq = 'd31;
next_irq_valid = '1; end
218     19'b01?????????????????: begin next_irq = 'd30;
next_irq_valid = '1; end
219     19'b001?????????????????: begin next_irq = 'd29;
next_irq_valid = '1; end
220     19'b0001?????????????????: begin next_irq = 'd28;
next_irq_valid = '1; end
221     19'b00001?????????????????: begin next_irq = 'd27;
next_irq_valid = '1; end
222     19'b000001?????????????????: begin next_irq = 'd26;
next_irq_valid = '1; end
223     19'b0000001?????????????????: begin next_irq = 'd25;
next_irq_valid = '1; end
224     19'b000000001?????????????????: begin next_irq = 'd24;
next_irq_valid = '1; end
225     19'b0000000001?????????????????: begin next_irq = 'd23;
next_irq_valid = '1; end
226     19'b00000000001?????????????????: begin next_irq = 'd22;
next_irq_valid = '1; end
227     19'b000000000001?????????????????: begin next_irq = 'd21;
next_irq_valid = '1; end
228     19'b000000000000001?????????????????: begin next_irq = 'd20;
next_irq_valid = '1; end
229     19'b0000000000000001?????????????????: begin next_irq = 'd19;
next_irq_valid = '1; end
230     19'b000000000000000001?????????????????: begin next_irq = 'd18;
next_irq_valid = '1; end
231     19'b00000000000000000001?????????????????: begin next_irq = 'd17;
next_irq_valid = '1; end
232     19'b000000000000000000001?????????????????: begin next_irq = 'd16;
next_irq_valid = '1; end
233     19'b0000000000000000000001?????????????????: begin next_irq = 'd11;
next_irq_valid = '1; end
234     19'b00000000000000000000001?????????????????: begin next_irq = 'd3;
next_irq_valid = '1; end
235     19'b000000000000000000000001?????????????????: begin next_irq = 'd7;
next_irq_valid = '1; end
236   endcase
237 end
238
239 always @(posedge clk_i or negedge rst_ni) begin
240   if (!rst_ni) begin
241     irq_q <= 0;
242     next_irq_q <= 0;
243     next_irq_valid_q <= 0;
244     saved_mie_q <= 0;
245   end
246   else begin
247     irq_q <= irq_i;
248     next_irq_q <= next_irq;
249     next_irq_valid_q <= next_irq_valid;
250     saved_mie_q <= mie_q;
251   end
252 end
253
254 always @(posedge clk_i or negedge rst_ni) begin
255   if (!rst_ni)
256     expected_irq <= 0;
257   else
258     expected_irq <= next_irq_q;
259 end
260
261 assign expected_irq_ack = next_irq_valid & mstatus_mie;
262
263 // Check expected interrupt wins
264 property p_irq_arb;
265   irq_ack_o |-> irq_id_o == next_irq;
266 endproperty
267 a_irq_arb: assert property(p_irq_arb)
268 else
269   `uvm_error(info_tag,
270     $sformatf("Expected winning interrupt: %0d,
actual interrupt: %0d", next_irq, irq_id_o))
271
272 // Check that an interrupt is expected
273 property p_irq_expected;
274   irq_ack_o |-> expected_irq_ack;
275 endproperty
276 a_irq_expected: assert property(p_irq_expected)
277 else
278   `uvm_error(info_tag,
279     $sformatf("Did not expect interrupt ack:

```

```

%0d", irq_id_o))
280
281 //
-----
282 // The infamous "first" flag (kludge for $past() handling of
t=0 values)
283 // Would like to use a leading ##1 in the property instead
but this currently
284 // does not work with dsim
285 //
-----
286 reg first;
287 always @(posedge clk or negedge rst_ni)
288   if (!rst_ni)
289     first <= 1'b1;
290   else
291     first <= 1'b0;
292
293 // mip reflects flopped interrupt inputs (irq_i) regardless
of other configuration
294 // Note that this runs on the gated clock
295 property p_mip_irq_i;
296   @(posedge clk)
297   !first |> mip == ($past(irq_i) & VALID_IRQ_MASK);
298 endproperty
299 a_mip_irq_i: assert property(p_mip_irq_i)
300 else
301   `uvm_error(info_tag,
302     $sformatf("MIP of 0x%08x does not follow
flopped irq_i input: 0x%08x", mip, $past(irq_i)));
303
304 // mip should not be reserved
305 property p_mip_not_reserved;
306   (mip & ~VALID_IRQ_MASK) == 0;
307 endproperty
308 a_mip_not_reserved: assert property(p_mip_not_reserved)
309 else
310   `uvm_error(info_tag,
311     $sformatf("MIP of reserved interrupt is as-
serted: mip = 0x%08x", mip));
312
313 //
-----
314 // Instruction coverage when taking an interrupt
315 //
-----
316 always @(posedge clk_i or negedge rst_ni) begin
317   if (!rst_ni) begin
318     last_instr_rdata <= '0;
319   end
320   else if (id_stage_instr_valid_i) begin
321     last_instr_rdata <= id_stage_instr_rdata_i;
322   end
323 end
324
325 //
-----
326 // WFI Checks
327 //
-----
328 assign is_wfi = id_stage_instr_valid_i &&
329   ((id_stage_instr_rdata_i & WFI_INSTR_MASK)
== WFI_INSTR_DATA) &&
330   !branch_taken_ex;
331 always @(posedge clk_i or negedge rst_ni) begin
332   if (!rst_ni) begin
333     in_wfi <= 1'b0;
334   end
335   else begin
336     if (is_wfi)
337       in_wfi <= 1'b1;
338     else if (!pending_enabled_irq || debug_req_i)
339       in_wfi <= 1'b0;
340   end
341 end
342
343 // WFI assertion will assert core_sleep_o in 6 clocks
344 property p_wfi_assert_core_sleep_o;
345   !pending_enabled_irq_q ##0 !in_wfi ##1 !
pending_enabled_irq_q ##0
346   ((!pending_enabled_irq && !debug_mode_q && !debug_req_i)
throughout in_wfi[*38])
347   |> core_sleep_o;
348 endproperty
349 a_wfi_assert_core_sleep_o: assert prop-
erty(p_wfi_assert_core_sleep_o)
350 else
351   `uvm_error(info_tag,
352     "Assertion of core_sleep_o did not occur
within 6 clocks")
353
354 // core_sleep_o deassertion in wfi should be followed by WFI

```

```

deassertion
355   property p_core_sleep_deassert;
356     $fell(core_sleep_o) ##0 in_wfi |> ##1 !in_wfi;
357   endproperty
358   a_core_sleep_deassert: assert prop-
erty(p_core_sleep_deassert)
359   else
360     `uvm_error(info_tag,
361       "Deassertion of core_sleep_o in WFI not fol-
lowed by WFI wakeup");
362
363 // When WFI deasserts the core should be awake
364 property p_wfi_deassert_core_sleep_o;
365   core_sleep_o ##1 pending_enabled_irq |> !core_sleep_o;
366 endproperty
367 a_wfi_deassert_core_sleep_o: assert prop-
erty(p_wfi_deassert_core_sleep_o)
368 else
369   `uvm_error(info_tag,
370     "Deassertion of WFI occurred and core is
still asleep");
371
372 // WFI wakeup to next instruction fetch
373 property p_wfi_wake_to_instr_fetch;
374   disable iff (!rst_ni || !fetch_enable_i || debug_mode_q)
375   core_sleep_o ##0 in_wfi ##1 !in_wfi[->1] |>
##[1:WFI_WAKEUP_LATENCY] if_stage_instr_rvalid_i;
376 endproperty
377 a_wfi_wake_to_instr_fetch: assert prop-
erty(p_wfi_wake_to_instr_fetch)
378 else
379   `uvm_error(info_tag,
380     $sformatf("Core did not start fetching %0d
cycles after WFI completed", WFI_WAKEUP_LATENCY));
381
382 // Cover property, detect sleep deassertion due to asserted
and non-asserted interrupts
383 property p_wfi_wake_mstatus_mie(irq, mie);
384   $fell(in_wfi) ##0 irq_i[irq] ##0 mie_q[irq] ##0
mstatus_mie == mie;
385 endproperty
386 generate for(genvar gv_i = 0; gv_i < 32; gv_i++) begin :
gen_wfi_cov
387   if (VALID_IRQ_MASK[gv_i]) begin
388     c_wfi_wake_mstatus_mie_0: cover prop-
erty(p_wfi_wake_mstatus_mie(gv_i, 0));
389     c_wfi_wake_mstatus_mie_1: cover prop-
erty(p_wfi_wake_mstatus_mie(gv_i, 1));
390   end
391 end
392 endgenerate
393
394 endmodule : uvmt_cv32e40p_interrupt_assert

```

6. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_iss_wrap.sv (140 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 //
4 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
5 // you may not use this file except in compliance with the Li-
cense.
6 // You may obtain a copy of the License at
7 //
8 // https://solderpad.org/licenses/
9 //
10 // Unless required by applicable law or agreed to in writing,
software
11 // distributed under the License is distributed on an "AS IS"
BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 // See the License for the specific language governing permis-
sions and
14 // limitations under the License.
15 //
16 //
17
18 `ifndef __UVMT_CV32E40P_ISS_WRAP_SV__
19 `define __UVMT_CV32E40P_ISS_WRAP_SV__
20
21 /**
22  * Module wrapper for Imperas OVP.
23  * Instantiates "CPU", the OVP wrapper, and "RAM" a spare
memory model.
24  */
25 module uvmt_cv32e40p_iss_wrap
26   #(
27     parameter int ROM_START_ADDR = 'h00000000,
28     parameter int ROM_BYTE_SIZE = 'h0,
29     parameter int RAM_BYTE_SIZE = 'h1B000000,
30     parameter int ID = 0
31   )
32
33   (
34     input realtime      clk_period,
35     uvma_clkrst_if      clkrst_if,
36     uvmt_cv32e40p_step_compare_if step_compare_if,

```

```

37   uvmt_cv32e40p_isa_covg_if    isa_covg_if
38   );
39
40   RVVI_bus    bus();
41   RVVI_io     io();
42
43   MONITOR     mon(bus, io);
44   RAM         #(
45       .ROM_START_ADDR(ROM_START_ADDR),
46       .ROM_BYTE_SIZE(ROM_BYTE_SIZE),
47       .RAM_BYTE_SIZE(RAM_BYTE_SIZE)) ram(bus);
48
49   CPU #(ID(ID), .VARIANT("CV32E40P")) cpu(bus, io);
50
51   assign bus.Clk = clknrst_if.clk;
52
53   // monitor rvvi updates
54   always @(cpu.state.notify) begin
55       int i;
56       step_compare_if.ovp_cpu_PCr = cpu.state.pcr;
57       for(i=0; i<32; i++)
58           step_compare_if.ovp_cpu_GPR[i] = cpu.state.x[i];
59
60       // generate events
61       if (cpu.state.valid) begin
62           // $display("Instruction Retired %08X %08X",
cpu.state.pcr, cpu.state.pcw);
63           -> step_compare_if.ovp_cpu_valid;
64       end else if (cpu.state.trap &&
65           cpu.state.decode == "ecall" &&
66           cpu.io.iirq_ack_o == 0 &&
67
68           (!cpu.io.DM || !cpu.state.csr["dcsr"][2]))
69   begin
68       // With introduction of OVPSIM model v20200821.377
69
70       // the valid behavior was changed, the above clause
71       was introduced to all signal valid
72       // instruction on valid ecalls, which must be
73       checked out by the step and compare interface
74       // Eventually this module should be replaced by a
75       RVFI/RVVI UVM scoreboard which will not rely on this
76       // $display("Instruction Retired %08X %08X",
cpu.state.pcr, cpu.state.pcw);
77       -> step_compare_if.ovp_cpu_valid;
78       end else if (cpu.state.trap) begin
79           // $display("Instruction Fault %08X %08X",
cpu.state.pcr, cpu.state.pcw);
80           -> step_compare_if.ovp_cpu_trap;
81       end
82   end
83
84   // monitor debug control updates
85   always @(cpu.control.notify) begin
86       step_compare_if.ovp_cpu_state_idle = cpu.con-
trol.state_idle;
87       step_compare_if.ovp_cpu_state_stepi = cpu.con-
trol.state_stepi;
88       step_compare_if.ovp_cpu_state_stop = cpu.con-
trol.state_stop;
89       step_compare_if.ovp_cpu_state_cont = cpu.con-
trol.state_cont;
90   end
91
92   function automatic void split(ref string in_s, ref string
s1, s2);
93
94       automatic int i;
95       for (i=0; i<in_s.len(); i++) begin
96           if (in_s.getc(i) == ":")
97               break;
98       end
99       if (i==0 ) begin
100           `uvm_fatal("ERROR not : found in split '%0s'",
in_s);
101       end
102       s1 = in_s.substr(0,i-1);
103       s2 = in_s.substr(i+1,in_s.len()-1);
104   endfunction
105
106   function automatic void sample();
107       string decode = cpu.state.decode;
108       string ins_str, op[4], key, val;
109       int i;
110
111       int num = $sscanf (decode, "%s %s %s %s %s", ins_str,
op[0], op[1], op[2], op[3]);
112       isa_covg_if.ins.ins_str = ins_str;
113       isa_covg_if.ins.pc = step_compare_if.ovp_cpu_PCr;
114       isa_covg_if.ins.compressed =
dut_wrap.cv32e40p_wrapper_i.tracer_i.insn_compressed;
115       for (i=0; i<num-1; i++) begin
116           split(op[i], key, val);
117           isa_covg_if.ins.ops[i].key=key;
118           isa_covg_if.ins.ops[i].val=val;
119       end
120
121       `uvm_info("OVPSIM", $sformatf("Decoded instr: %s%s pc:
0x%08x",
122
123       isa_covg_if.ins.compressed ? "c." : "",

```

```

118       decode,
119       isa_covg_if.ins.pc),
120       UVM_DEBUG)
121       ->isa_covg_if.ins_valid;
122   endfunction
123
124   always @(step_compare_if.ovp_cpu_valid) begin
125       sample();
126   end
127
128   initial begin
129       clknrst_if.clk = 1'b0;
130       #1; // time for clknrst_if_dut to set the clk_period
131       wait (clk_period != 0.0);
132       `uvm_info("OVPSIM", "Starting OVP clock", UVM_LOW)
133       clknrst_if.set_period(clk_period);
134       clknrst_if.start_clk();
135   end
136
137   endmodule : uvmt_cv32e40p_iss_wrap
138
139   `endif // __UVMT_CV32E40P_ISS_WRAP_SV__
140

```

7. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/ uvmt_cv32e40p_macros.sv (26 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20
21 `ifndef __UVMT_CV32E40P_MACROS_SV__
22 `define __UVMT_CV32E40P_MACROS_SV__
23
24
25
26 `endif // __UVMT_CV32E40P_MACROS_SV__

```

8. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/ uvmt_cv32e40p_pkg.sv (66 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20
21 // SPD-License-Identifier: Apache-2.0 WITH SHL-2.1
22
23 `ifndef __UVMT_CV32E40P_PKG_SV__
24 `define __UVMT_CV32E40P_PKG_SV__
25
26 // Pre-processor macros
27 `include "uvm_macros.svh"
28 `include "uvm_l_hrtbt_macros.sv"
29 `include "uvm_l_logs_macros.sv"
30 `include "uvmt_cv32e40p_macros.sv"
31

```

```

31
32 /**
33  * Encapsulates all the types and test cases for the verifica-
tion of an
34  * CV32E40P RTL design.
35  */
36 package uvmt_cv32e40p_pkg;
37
38 import uvm_pkg::*;
39 import uvme_cv32e40p_pkg::*;
40 import uvml_hrtbt_pkg::*;
41 import uvml_logs_pkg::*;
42
43 // Constants / Structs / Enums
44 `include "uvmt_cv32e40p_constants.sv"
45 `include "uvmt_cv32e40p_tdefs.sv"
46
47 // Virtual sequence library
48 // TODO Add virtual sequences
49 // Ex: `include "uvmt_cv32e40p_sanity_vseq.sv"
50 `include "uvmt_cv32e40p_vseq_lib.sv"
51
52 // Base test case
53 `include "uvmt_cv32e40p_test_cfg.sv"
54 `include "uvmt_cv32e40p_test_randvars.sv"
55 `include "uvmt_cv32e40p_base_test.sv" // all testcases
should extend from this testcase
56 // `include "uvmt_cv32e40p_smoke_test.sv" // smoke test has
multiple XMRs that are illegal according to the LRM
57
58 // Compliance tests
59 `include "uvmt_cv32e40p_firmware_test.sv"
60
61 endpackage : uvmt_cv32e40p_pkg
62
63 // All Interfaces used by the CV32E40P TB are here
64 `include "uvmt_cv32e40p_tb_ifs.sv"
65
66 `endif // __UVMT_CV32E40P_PKG_SV__

```

9. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/ uvmt_cv32e40p_step_compare.sv (567 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 //
4 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
5 // you may not use this file except in compliance with the Li-
cense.
6 // You may obtain a copy of the License at
7 //
8 // https://solderpad.org/licenses/
9 //
10 // Unless required by applicable law or agreed to in writing,
software
11 // distributed under the License is distributed on an "AS IS"
BASIS,
12 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
13 // See the License for the specific language governing permis-
sions and
14 // limitations under the License.
15 //
16 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.0
17 //
18
19
20 `ifndef __UVMT_CV32E40P_STEP_COMPARE_SV__
21 `define __UVMT_CV32E40P_STEP_COMPARE_SV__
22
23 // Step-and-Compare between the CV32E40PE40P and Imperas OVP-
sim ISS
24 // Cloned from the Imperas demo at $(IMPERAS_HOME)/RTL_OVPmod-
el_step_compare/verilog_testbench/testbench.sv
25
26 /**
27  * Copyright (c) 2005-2020 Imperas Software Ltd., www.imper-
as.com
28  * Copyright (C) Tumbush Enterprises, LLC
29  *
30  * Licensed under the Apache License, Version 2.0 (the "Li-
cense");
31  * you may not use this file except in compliance with the Li-
cense.
32  * You may obtain a copy of the License at
33  *
34  * http://www.apache.org/licenses/LICENSE-2.0
35  *
36  * Unless required by applicable law or agreed to in writing,
software
37  * distributed under the License is distributed on an "AS IS"
BASIS,
38  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND,
39  * either express or implied.
40  *
41  * See the License for the specific language governing permis-
sions and
42  * limitations under the License.
43  */

```

```

44 */
45
46 //
47 // Execute step and compare of dut OVP instance vs riscv RTL
instance
48 //
49 `ifndef T0_TYPE
50 `define T0_TYPE "RV32IMC"
51 `endif
52
53 import uvm_pkg::*; // needed for the UVM messaging ser-
vice (`uvm_info(), etc.)
54
55 `include "uvm_macros.svh"
56 `define CV32E40P_CORE
$root.uvmt_cv32e40p_tb.dut_wrap.cv32e40p_wrapper_i.core_i
57 `define CV32E40P_TRACER
$root.uvmt_cv32e40p_tb.dut_wrap.cv32e40p_wrapper_i.tracer_i
58
59 // TODO change names
60 `define CV32E40P_RM
$root.uvmt_cv32e40p_tb.iss_wrap.cpu
61 `define CV32E40P_RM_RVVI_STATE
$root.uvmt_cv32e40p_tb.iss_wrap.cpu.state
62 `define CV32E40P_RM_RVVI_CONTROL
$root.uvmt_cv32e40p_tb.iss_wrap.cpu.control
63
64
65 module uvmt_cv32e40p_step_compare
66 (
67     uvma_clknrst_if          clknrst_if,
68     uvmt_cv32e40p_step_compare_if step_compare_if
69 );
70
71 bit Clk;
72 bit miscompare;
73 bit is_stall_sim = 0;
74 bit ignore_dpc_check = 0;
75 bit use_iss = 0;
76
77 // FIXME: strichmo: when running random interrupts and random
debug requests it is possible to enter debug mode
78 // (while also acking an interrupt) when the debug program
counter may or may not yet be pointing to the interrupt
79 // vector (mtvec) upon debug entry. However the PC stream
should verify proper operation (the PC will mismatch when
80 // the debug ROM eventually executes a dret). For now we
will skip this comparison if the test requests it
81 initial begin
82     if ($test$plusargs("ignore_dpc_check")) begin
83         ignore_dpc_check = 1;
84         `uvm_info("Step-and-Compare", $sformatf("Requesting +ig-
nore_dpc_check"), UVM_NONE)
85     end
86     else begin
87         ignore_dpc_check = 0;
88     end
89 end
90
91 initial begin
92     if ($test$plusargs("USE_ISS"))
93         use_iss = 1;
94 end
95
96 // Set the is_stall_sim flag if random stalls are enabled
97 // This will turn off some unpredictable checks:
98 // - CSR wire checks
99 // - Non-written GPR registers (per instructions)
100 // Note that register writebacks to GPRs will still be
checked during each instruction retirement even with stalls
101 initial begin
102     if (!$test$plusargs("rand_stall_obi_disable")) begin
103         is_stall_sim = 1;
104     end
105 end
106
107
108 always begin
109     wait (clknrst_if.reset_n === 1'b0);
110     wait (clknrst_if.reset_n === 1'b1);
111     if (is_stall_sim) begin
112         `uvm_info("Step-and-Compare", $sformatf("is_stall_sim set
to 1, disabling CSR wire checks and stable GPR register checks"),
UVM_NONE)
113     end
114 end
115
116
117 // Waiving Verissimo SVTB.
32.2.0: Pass strings by reference unless otherwise needed
117 function void check_32bit(input string compared, input bit
[31:0] expected, input logic [31:0] actual); // @DVT_LINTER_WAIVER
"MT20211228_1" disable SVTB.32.2.0
118     static int now = 0;
119     if (now != $time) begin
120         miscompare = 0;
121         now = $time;
122     end
123     if (expected !== actual) begin

```



```

124     miscompare = 1;
125     `uvm_error("Step-and-Compare", $sformatf("%s expected=0x%8h and actual=0x%8h PC=0x%8h", compared, expected, actual,
step_compare_if.ovp_cpu_Pcr))
126     end else begin
127         `uvm_info("Step-and-Compare", $sformatf("%s expected=0x%8h==actual", compared, actual), UVM_DEBUG)
128     end
129 endfunction // check_32bit
130
131 function automatic void compare();
132     int idx;
133     logic [5:0] insn_regs_write_addr;
134     logic [31:0] insn_regs_write_value;
135     int insn_regs_write_size;
136     string compared_str;
137     bit ignore;
138
139     logic [31:0] csr_val;
140
141     // Compare PC
142     //
check_32bit(.compared("PC"), .expected(step_compare_if.ovp_cpu_Pcr),
.actual(step_compare_if.insn_pc));
143
check_32bit(.compared("PC"), .expected(`CV32E40P_RM_RVVI_STATE.pc),
.actual(step_compare_if.insn_pc));
144     step_compare_if.num_pc_checks++;
145
146     // Compare GPR's
147     // Assuming that `CV32E40P_TRACER.insn_regs_write_size
is never > 1. Check this.
148     // Note that dut_wrap is found 1 level up
149     insn_regs_write_size =
`CV32E40P_TRACER.insn_regs_write.size();
150     if (insn_regs_write_size > 1) begin
151         `uvm_error("Step-and-Compare", $sformatf("Assume
insn_regs_write_size is 0 or 1 but is %0d", insn_regs_write_size))
152     end
153     else if (insn_regs_write_size == 1) begin // Get
`CV32E40P_TRACER.insn_regs_write fields if size is 1
154         insn_regs_write_addr =
`CV32E40P_TRACER.insn_regs_write[0].addr;
155         insn_regs_write_value =
`CV32E40P_TRACER.insn_regs_write[0].value;
156         `uvm_info("Step-and-Compare", $sform-
atf("insn_regs_write queue[0] addr=0x%0x, value=0x%0x",
insn_regs_write_addr, insn_regs_write_value), UVM_DEBUG)
157     end
158
159     // Ignore insn_regs_write_addr=0 just like in
riscv_tracer.sv
160     for (idx=0; idx<32; idx++) begin
161         compared_str = $sformatf("GPR[%0d]", idx);
162         if ((idx == insn_regs_write_addr) && (idx != 0) &&
(insn_regs_write_size == 1)) // Use register in insn_regs_write
queue if it exists
163     check_32bit(.compared(compared_str), .expected(step_compare_if.ovp_
cpu_GPR[idx][31:0]), .actual(insn_regs_write_value));
164         // FIXME: strichmo: I am removing the static (non-writ-
ten) register checks, as they fail in presence of I and D bus RAM
stalls
165         // It would be highly desirable to find an alternat-
ive for this type of check to ensure unintended writes to do not
166         else if (!is_stall_sim && !
`CV32E40P_TRACER.insn_wb_bypass) // Use actual value from RTL to
compare registers which should have not changed
167     check_32bit(.compared(compared_str), .expected(step_compare_if.ovp_
cpu_GPR[idx][31:0]), .actual(step_compare_if.riscy_GPR[idx]));
168         step_compare_if.num_gpr_checks++;
169     end
170
171     // Compare CSR's
172     foreach( `CV32E40P_RM_RVVI_STATE.csr[index]) begin
173         step_compare_if.num_csr_checks++;
174         ignore = 0;
175         csr_val = 0;
176
177         // CSR timing at instruction retirement is not com-
pletely deterministic in this simple model in presence of OBI stalls
178         if (is_stall_sim)
179             ignore = 1;
180         case (index)
181             "marchid" : csr_val =
cv32e40p_pkg::MARCHID; // warning! defined in cv32e40p_pkg
182             "mcountinhibit" : csr_val =
`CV32E40P_CORE.cs_registers_i.mcountinhibit_q;
183             "mvendorid" : csr_val =
{cv32e40p_pkg::MVENDORID_BANK, cv32e40p_pkg::MVENDORID_OFFSET};
184             "mstatus" : if (step_com-
pare_if.deferint_prime == 0) ignore = 1;
185             else csr_val =
{`CV32E40P_CORE.cs_registers_i.mstatus_q.mprv, // Not documented in
Rev 4.5 of user_manual.doc but is in the design
186             4'b0,
187             190

```

```

`CV32E40P_CORE.cs_registers_i.mstatus_q.mpp,
191     3'b0,
192
`CV32E40P_CORE.cs_registers_i.mstatus_q.mpie,
193     2'b0,
194
`CV32E40P_CORE.cs_registers_i.mstatus_q.upie,
195
`CV32E40P_CORE.cs_registers_i.mstatus_q.mie,
196     2'b0,
197
`CV32E40P_CORE.cs_registers_i.mstatus_q.ueie;
198     "misa" : csr_val =
`CV32E40P_CORE.cs_registers_i.MISA_VALUE;
199     "mie" : csr_val =
`CV32E40P_CORE.cs_registers_i.mie_q;
200     "mtvec" : csr_val =
{`CV32E40P_CORE.cs_registers_i.mtvec_q, 6'h0,
`CV32E40P_CORE.cs_registers_i.mtvec_mode_q};
201     "mscratch" : csr_val =
`CV32E40P_CORE.cs_registers_i.mscratch_q;
202     "mepc" : if (step_com-
pare_if.deferint_prime == 0) ignore = 1;
203     else csr_val =
`CV32E40P_CORE.cs_registers_i.mepc_q;
204     "mcause" : if (step_com-
pare_if.deferint_prime == 0) ignore = 1;
205     else csr_val =
{`CV32E40P_CORE.cs_registers_i.mcause_q[5],
206     26'b0,
207
`CV32E40P_CORE.cs_registers_i.mcause_q[4:0]);
208     // "mip" : if (step_com-
pare_if.deferint_prime == 0 || iss_wrap.io.deferint == 0) ignore =
1;
209     // else csr_val =
`CV32E40P_CORE.cs_registers_i.mip;
210     "mip" : ignore = 1;
211     "mhartid" : csr_val =
`CV32E40P_CORE.cs_registers_i.hart_id_i;
212
213     // only valid in DEBUG Mode
214     "dcsr" : begin
215         csr_val =
`CV32E40P_CORE.cs_registers_i.dcsr_q;
216         if (iss_wrap.io.DM==0) ignore =
1;
217     end
218     "dpc" : begin
219         csr_val =
`CV32E40P_CORE.cs_registers_i.depc_q;
220         if (iss_wrap.io.DM==0) ignore =
1;
221         if (ignore_dpc_check) ignore =
1;
222     end
223
224     "dscratch0" : csr_val =
`CV32E40P_CORE.cs_registers_i.dscratch0_q;
225     "dscratch1" : csr_val =
`CV32E40P_CORE.cs_registers_i.dscratch1_q;
226     "pmpcfg0" : csr_val =
`CV32E40P_CORE.cs_registers_i.pmp_reg_q.pmpcfg_packed[0];
227     "pmpcfg1" : csr_val =
`CV32E40P_CORE.cs_registers_i.pmp_reg_q.pmpcfg_packed[1];
228     "pmpcfg2" : csr_val =
`CV32E40P_CORE.cs_registers_i.pmp_reg_q.pmpcfg_packed[2];
229     "pmpcfg3" : csr_val =
`CV32E40P_CORE.cs_registers_i.pmp_reg_q.pmpcfg_packed[3];
230     "tselect" : csr_val = 32'h0000_0000;
231     "tdata1" : csr_val =
`CV32E40P_CORE.cs_registers_i.tmatch_control_rdata;
232     "tdata2" : csr_val =
`CV32E40P_CORE.cs_registers_i.tmatch_value_rdata;
233     "tdata3" : csr_val = 32'h0000_0000;
234     "tinfo" : csr_val =
`CV32E40P_CORE.cs_registers_i.tinfo_types;
235
236     "time" : ignore = 1;
237
238     "mcontext" : ignore = 1;
239     "scontext" : ignore = 1;
240
241     "cycle" : ignore = 1;
242     "cycleh" : ignore = 1;
243     "instret" : ignore = 1;
244     "instreth" : ignore = 1;
245     "minstret" : ignore = 1;
246     "minstreth" : ignore = 1;
247
248     "mimpid" : ignore = 1;
249
250     "hpmcounter3" : ignore = 1;
251     "hpmcounter4" : ignore = 1;
252     "hpmcounter5" : ignore = 1;
253     "hpmcounter6" : ignore = 1;
254     "hpmcounter7" : ignore = 1;
255     "hpmcounter8" : ignore = 1;
256     "hpmcounter9" : ignore = 1;
257     "hpmcounter10" : ignore = 1;
258     "hpmcounter11" : ignore = 1;

```

```

259 "hpmcounter12" : ignore = 1;
260 "hpmcounter13" : ignore = 1;
261 "hpmcounter14" : ignore = 1;
262 "hpmcounter15" : ignore = 1;
263 "hpmcounter16" : ignore = 1;
264 "hpmcounter17" : ignore = 1;
265 "hpmcounter18" : ignore = 1;
266 "hpmcounter19" : ignore = 1;
267 "hpmcounter20" : ignore = 1;
268 "hpmcounter21" : ignore = 1;
269 "hpmcounter22" : ignore = 1;
270 "hpmcounter23" : ignore = 1;
271 "hpmcounter24" : ignore = 1;
272 "hpmcounter25" : ignore = 1;
273 "hpmcounter26" : ignore = 1;
274 "hpmcounter27" : ignore = 1;
275 "hpmcounter28" : ignore = 1;
276 "hpmcounter29" : ignore = 1;
277 "hpmcounter30" : ignore = 1;
278 "hpmcounter31" : ignore = 1;
279
280 "hpmcounterh3" : ignore = 1;
281 "hpmcounterh4" : ignore = 1;
282 "hpmcounterh5" : ignore = 1;
283 "hpmcounterh6" : ignore = 1;
284 "hpmcounterh7" : ignore = 1;
285 "hpmcounterh8" : ignore = 1;
286 "hpmcounterh9" : ignore = 1;
287 "hpmcounterh10" : ignore = 1;
288 "hpmcounterh11" : ignore = 1;
289 "hpmcounterh12" : ignore = 1;
290 "hpmcounterh13" : ignore = 1;
291 "hpmcounterh14" : ignore = 1;
292 "hpmcounterh15" : ignore = 1;
293 "hpmcounterh16" : ignore = 1;
294 "hpmcounterh17" : ignore = 1;
295 "hpmcounterh18" : ignore = 1;
296 "hpmcounterh19" : ignore = 1;
297 "hpmcounterh20" : ignore = 1;
298 "hpmcounterh21" : ignore = 1;
299 "hpmcounterh22" : ignore = 1;
300 "hpmcounterh23" : ignore = 1;
301 "hpmcounterh24" : ignore = 1;
302 "hpmcounterh25" : ignore = 1;
303 "hpmcounterh26" : ignore = 1;
304 "hpmcounterh27" : ignore = 1;
305 "hpmcounterh28" : ignore = 1;
306 "hpmcounterh29" : ignore = 1;
307 "hpmcounterh30" : ignore = 1;
308 "hpmcounterh31" : ignore = 1;
309
310 "mhpmcounter3" : ignore = 1;
311 "mhpmcounter4" : ignore = 1;
312 "mhpmcounter5" : ignore = 1;
313 "mhpmcounter6" : ignore = 1;
314 "mhpmcounter7" : ignore = 1;
315 "mhpmcounter8" : ignore = 1;
316 "mhpmcounter9" : ignore = 1;
317 "mhpmcounter10" : ignore = 1;
318 "mhpmcounter11" : ignore = 1;
319 "mhpmcounter12" : ignore = 1;
320 "mhpmcounter13" : ignore = 1;
321 "mhpmcounter14" : ignore = 1;
322 "mhpmcounter15" : ignore = 1;
323 "mhpmcounter16" : ignore = 1;
324 "mhpmcounter17" : ignore = 1;
325 "mhpmcounter18" : ignore = 1;
326 "mhpmcounter19" : ignore = 1;
327 "mhpmcounter20" : ignore = 1;
328 "mhpmcounter21" : ignore = 1;
329 "mhpmcounter22" : ignore = 1;
330 "mhpmcounter23" : ignore = 1;
331 "mhpmcounter24" : ignore = 1;
332 "mhpmcounter25" : ignore = 1;
333 "mhpmcounter26" : ignore = 1;
334 "mhpmcounter27" : ignore = 1;
335 "mhpmcounter28" : ignore = 1;
336 "mhpmcounter29" : ignore = 1;
337 "mhpmcounter30" : ignore = 1;
338 "mhpmcounter31" : ignore = 1;
339
340 "mhpmcounterh3" : ignore = 1;
341 "mhpmcounterh4" : ignore = 1;
342 "mhpmcounterh5" : ignore = 1;
343 "mhpmcounterh6" : ignore = 1;
344 "mhpmcounterh7" : ignore = 1;
345 "mhpmcounterh8" : ignore = 1;
346 "mhpmcounterh9" : ignore = 1;
347 "mhpmcounterh10" : ignore = 1;
348 "mhpmcounterh11" : ignore = 1;
349 "mhpmcounterh12" : ignore = 1;
350 "mhpmcounterh13" : ignore = 1;
351 "mhpmcounterh14" : ignore = 1;
352 "mhpmcounterh15" : ignore = 1;
353 "mhpmcounterh16" : ignore = 1;
354 "mhpmcounterh17" : ignore = 1;
355 "mhpmcounterh18" : ignore = 1;
356 "mhpmcounterh19" : ignore = 1;
357 "mhpmcounterh20" : ignore = 1;
358 "mhpmcounterh21" : ignore = 1;
359 "mhpmcounterh22" : ignore = 1;

360 "mhpmcounterh23" : ignore = 1;
361 "mhpmcounterh24" : ignore = 1;
362 "mhpmcounterh25" : ignore = 1;
363 "mhpmcounterh26" : ignore = 1;
364 "mhpmcounterh27" : ignore = 1;
365 "mhpmcounterh28" : ignore = 1;
366 "mhpmcounterh29" : ignore = 1;
367 "mhpmcounterh30" : ignore = 1;
368 "mhpmcounterh31" : ignore = 1;
369
370 default: begin
371     `uvm_error("STEP_COMPARE", $sformatf("index=%s
does not match a CSR name", index))
372     ignore = 1;
373 end
374 endcase // case (index)
375
376 if (!ignore)
377
378 check_32bit(.compared(index), .expected(`CV32E40P_RM_RVVI_STATE.csr[
index]), .actual(csr_val));
378
379 end // foreach (ovp.cpu.csr[index])
380 endfunction // compare
381
382 int cycles = 0;
383 always @(posedge `CV32E40P_CORE.clk) begin
384     cycles++;
385 end
386
387 // RTL->RM CSR : mcycle, minstret, mcycleh, minstreth
388 //
Waiving Verissimo SVTB.32.2.0: Pass strings by reference unless oth-
erwise needed
389 function automatic void pushRTL2RM(string message); //
@DVT_LINTER_WAIVER "MT20211228_2" disable SVTB.32.2.0
390     logic [ 5:0] gpr_addr;
391     logic [31:0] gpr_value;
392
393     if (`CV32E40P_TRACER.insn_regs_write.size()) begin
394         gpr_addr =
`CV32E40P_TRACER.insn_regs_write[0].addr;
395         gpr_value =
`CV32E40P_TRACER.insn_regs_write[0].value;
396         `CV32E40P_RM.state.GPR_rtl[gpr_addr] = gpr_value;
397     end
398     // Pass cycles and reset
399     `CV32E40P_RM.cycles = cycles;
400     cycles = 0;
401 endfunction // pushRTL2RM
402
403 /*
404     The schedule works like this
405     1. Run the RTL for 1 instruction retirement
406     2. if the RTL.RetiredPC == OVP.NextPC
407         then run OVP for 1 instruction retirement
408     3. Compare RTL <-> OVP
409 */
410 event ev_compare;
411 static int instruction_count = 0;
412
413 typedef enum {
414     INIT,
415     IDLE, // Needed to get an event on state so always
block is initially entered
416
417     RTL_STEP,
418     RTL_VALID,
419     RTL_TRAP,
420     RTL_HALT,
421
422     RM_STEP,
423     RM_VALID,
424     RM_TRAP,
425     RM_HALT,
426
427     CMP
428 } state_e;
429
430 state_e state = INIT;
431 initial state <= IDLE; // cause an event for always @*
432
433 always @(*) begin
434     if (use_iss) begin
435         case (state)
436             IDLE: begin
437                 state <= RTL_STEP;
438             end
439
440             RTL_STEP: begin
441                 clknrst_if.start_clk();
442                 fork
443                     begin
444                         @step_compare_if.riscv_retire;
445                         clknrst_if.stop_clk();
446                         state <= RTL_VALID;
447                     end
448                 begin
449                     @step_compare_if.riscv_trap;
450                     state <= RTL_TRAP;
451                 end

```



```

452         begin
453             @step_compare_if.riscv_halt;
454             state <= RTL_HALT;
455         end
456         join_any
457         disable fork;
458     end
459
460     RTL_VALID: begin
461         state <= RM_STEP;
462     end
463
464     RTL_TRAP: begin
465         //state <= RM_STEP; // TODO: RTL/RVVI needs ad-
ditional work
466         state <= RTL_STEP;
467     end
468
469     RTL_HALT: begin
470         state <= RTL_STEP;
471     end
472
473     RM_STEP: begin
474         pushRTL2RM("ret_rtl");
475         `CV32E40P_RM_RVVI_CONTROL.stepi();
476         fork
477         begin
478             @step_compare_if.ovp_cpu_valid;
479             -> `CV32E40P_TRACER.ovp_retire;
480             state <= RM_VALID;
481         end
482         begin
483             @step_compare_if.ovp_cpu_trap;
484             state <= RM_TRAP;
485         end
486         begin
487             @step_compare_if.ovp_cpu_halt;
488             state <= RM_HALT;
489         end
490         join_any
491         disable fork;
492     end
493
494     RM_VALID: begin
495         state <= CMP;
496     end
497
498     RM_TRAP: begin
499         //state <= CMP; // TODO: needs enabling after
RTL/RVVI fix
500         state <= RM_STEP;
501     end
502
503     RM_HALT: begin
504         state <= RM_STEP;
505     end
506
507     CMP: begin
508         compare();
509         -> ev_compare;
510         instruction_count += 1;
511         //state <= RTL_STEP;
512         state <= IDLE;
513     end
514     endcase // case (state)
515 end
516 end
517
518 always @(instruction_count) begin
519     if (!(instruction_count % 10000)) begin
520         `uvm_info("Step-and-Compare", $sformatf("Compared %0d
instructions", instruction_count), UVM_NONE)
521     end
522     if (instruction_count >= 10_000_000) begin
523         `uvm_fatal("Step-and-Compare", $sformatf("Compared
%0d instructions - that's too long!", instruction_count))
524     end
525 end
526
527 `ifndef COVERAGE
528     coverage cov1;
529     initial begin
530         cov1 = new();
531     end
532 end
533
534 function void split(input string in_s, output string s1,
s2);
535     automatic int i;
536     for (i=0; i<in_s.len(); i++) begin
537         if (in_s.getc(i) == ".")
538             break;
539     end
540     if (i==0 ) begin
541         `uvm_fatal("STEP COMPARE", $sformatf(": not found
in split '%0s'", in_s))
542     end
543     s1 = in_s.substr(0,i-1);
544     s2 = in_s.substr(i+1,in_s.len()-1);
545 endfunction

```

```

546
547     function automatic void sample();
548         string decode = `CV32E40P_RM.Decode;
549         string ins_str, op[4], key, val;
550         int i;
551         ins_t ins;
552         int num = $sscanf (decode, "%s %s %s %s %s", ins_str,
op[0], op[1], op[2], op[3]);
553         ins.ins_str = ins_str;
554         for (i=0; i<num-1; i++) begin
555             split(op[i], key, val);
556             ins.ops[i].key=key;
557             ins.ops[i].val=val;
558         end
559         cov1.sample (ins);
560     endfunction
561 `endif
562
563 endmodule: uvmt_cv32e40p_step_compare
564
565 `endif //__UVMT_CV32E40P_STEP_COMPARE_SV__
566
567

```

10. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb.sv (606 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 // Copyright 2020 Silicon Labs, Inc.
5 //
6 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
7 // you may not use this file except in compliance with the Li-
cense.
8 // You may obtain a copy of the License at
9 //
10 // https://solderpad.org/licenses/
11 //
12 // Unless required by applicable law or agreed to in writing,
software
13 // distributed under the License is distributed on an "AS IS"
BASIS,
14 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
15 // See the License for the specific language governing permis-
sions and
16 // limitations under the License.
17 //
18 // SPDX-License-Identifier: Apache-2.0 WITH SHL-2.0
19 //
20
21
22 `ifndef __UVMT_CV32E40P_TB_SV__
23 `define __UVMT_CV32E40P_TB_SV__
24
25 /**
26  * Module encapsulating the CV32E40P DUT wrapper, and associ-
ated SV interfaces.
27  * Also provide UVM environment entry and exit points.
28  */
29
30 `default_nettype none
31 module uvmt_cv32e40p_tb;
32
33     import uvm_pkg::*;
34     import uvmt_cv32e40p_pkg::*;
35     import uvme_cv32e40p_pkg::*;
36
37     // DUT (core) parameters: refer to the CV2E40P User Manual.
38     `ifndef NO_PULP
39         parameter int CORE_PARAM_PULP_XPULP = 0;
40         parameter int CORE_PARAM_PULP_CLUSTER = 0;
41         parameter int CORE_PARAM_PULP_ZFINX = 0;
42     `else
43         `ifdef PULP
44             parameter int CORE_PARAM_PULP_XPULP = 1;
45             parameter int CORE_PARAM_PULP_CLUSTER = 0;
46             parameter int CORE_PARAM_PULP_ZFINX = 0;
47         `else
48             // If you don't explicitly specify either NO_PULP or
PULP, you get NO_PULP
49             parameter int CORE_PARAM_PULP_XPULP = 0;
50             parameter int CORE_PARAM_PULP_CLUSTER = 0;
51             parameter int CORE_PARAM_PULP_ZFINX = 0;
52         `endif
53     `endif
54
55     `ifndef SET_NUM_MHPMCOUNTERS
56         parameter int CORE_PARAM_NUM_MHPMCOUNTERS = `SET_NUM_MHPM-
COUNTERS;
57     `else
58         parameter int CORE_PARAM_NUM_MHPMCOUNTERS = 1;
59     `endif
60
61     // ENV (testbench) parameters
62     parameter int ENV_PARAM_INSTR_ADDR_WIDTH = 32;
63     parameter int ENV_PARAM_INSTR_DATA_WIDTH = 32;
64     parameter int ENV_PARAM_RAM_ADDR_WIDTH = 22;

```

```

65
66 // Capture regs for test status from Virtual Peripheral in
dut_wrap.mem_i
67 bit tp;
68 bit tf;
69 bit evalid;
70 bit [31:0] evalue;
71
72 // Agent interfaces
73 uvmt_cv32e40p_clknrst_if clknrst_if (); // clock and
resets from the clknrst agent
74 uvmt_cv32e40p_clknrst_if clknrst_if_iss ();
75 uvmt_cv32e40p_debug_if debug_if ();
76 uvmt_cv32e40p_interrupt_if interrupt_if (); // Interrupts
77 uvmt_cv32e40p_interrupt_if vp_interrupt_if (); // Interrupts
78 uvmt_cv32e40p_obi_memory_if obi_memory_instr_if
(.clk(clknrst_if.clk),
79 ,.re-
set_n(clknrst_if.reset_n));
80 uvmt_cv32e40p_obi_memory_if obi_memory_data_if
(.clk(clknrst_if.clk),
81 ,.re-
set_n(clknrst_if.reset_n));
82
83 // DUT Wrapper Interfaces
84 uvmt_cv32e40p_vp_status_if
vp_status_if(.tests_passed(),
85 ,.tests_failed
(),
86 ,.exit_val-
id(),
87 ,.exit_value()
); // Status information generated by the Virtual Peripherals in the
DUT WRAPPER memory.
88 uvmt_cv32e40p_core_cntrl_if core_cntrl_if(.clk(),
89 ,.fetch_en(),
90 ,.pulp_c-
lock_en(),
91 ,.scan_cg_en(
),
92 ,.boot_ad-
dr(),
93 ,.mtvec_addr(
),
94 ,.dm_halt_ad
dr(),
95 ,.dm_excep-
tion_addr(),
96 ,.hart_id(),
97 ,.de-
bug_req(),
98 ,.load_in-
str_mem()); // Static and quasi-static core control inputs.
99 uvmt_cv32e40p_core_status_if
core_status_if(.core_busy(),
100 ,.sec_lvl())
; // Core status outputs
101
102 // Step and compare interface
103 uvmt_cv32e40p_step_compare_if step_compare_if();
104 uvmt_cv32e40p_isa_covg_if isa_covg_if();
105
106 /**
107 * DUT WRAPPER instance:
108 * This is an update of the riscv_wrapper.sv from PULP-Plat-
form RI5CY project with
109 * a few mods to bring unused ports from the CORE to this
level using SV interfaces.
110 */
111 uvmt_cv32e40p_dut_wrap #(
112 (CORE_PARAM_PULP_XPULP),
113 (CORE_PARAM_PULP_CLUSTER),
114 (CORE_PARAM_PULP_ZFINX),
115 (CORE_PARAM_PULP_ZFINX),
116 (CORE_PARAM_NUM_MHPMCOUNTERS),
117 (ENV_PARAM_INSTR_ADDR_WIDTH),
118 (ENV_PARAM_INSTR_DATA_WIDTH),
119 (ENV_PARAM_RAM_ADDR_WIDTH)
)
120 dut_wrap (.*)
121
122 bind uvmt_cv32e40p_dut_wrap
123 uvmt_cv32e40p_obi_memory_assert_if_wrp#(
124 .ADDR_WIDTH(32),
125 .DATA_WIDTH(32),
126 .AUSER_WIDTH(0),
127 .WUSER_WIDTH(0),
128 .RUSER_WIDTH(0),
129 .ID_WIDTH(0),
130 .ACHK_WIDTH(0),
131 .RCHK_WIDTH(0),
132 .IS_1P2(0)
) obi_instr_memory_assert_if(.obi(obi_memory_instr_if));
133
134
135

```

```

136 bind uvmt_cv32e40p_dut_wrap
137 uvmt_cv32e40p_obi_memory_assert_if_wrp#(
138 .ADDR_WIDTH(32),
139 .DATA_WIDTH(32),
140 .AUSER_WIDTH(0),
141 .WUSER_WIDTH(0),
142 .RUSER_WIDTH(0),
143 .ID_WIDTH(0),
144 .ACHK_WIDTH(0),
145 .RCHK_WIDTH(0),
146 .IS_1P2(0)
) obi_data_memory_assert_if(.obi(obi_memory_data_if));
147
148 // Bind in verification modules to the design
149 bind cv32e40p_core
150 uvmt_cv32e40p_interrupt_assert_inter-
rupt_assert_if(.mcause_n(cs_registers_i.mcause_n),
151 ,.mip(cs_
registers_i.mip),
152 ,.mie_q(
cs_registers_i.mie_q),
153 ,.mie_n(
cs_registers_i.mie_n),
154 ,.mstatu
s_mie(cs_registers_i.mstatus_q.mie),
155 ,.mtvec_
mode_q(cs_registers_i.mtvec_mode_q),
156 ,.if_sta
ge_instr_rvalid_i(if_stage_i.instr_rvalid_i),
157 ,.if_sta
ge_instr_rdata_i(if_stage_i.instr_rdata_i),
158 ,.id_sta
ge_instr_valid_i(id_stage_i.instr_valid_i),
159 ,.id_sta
ge_instr_rdata_i(id_stage_i.instr_rdata_i),
160 ,.branch_
taken_ex(id_stage_i.branch_taken_ex),
161 ,.ctrl_f
sm_cs(id_stage_i.controller_i.ctrl_fsm_cs),
162 ,.de-
bug_mode_q(id_stage_i.controller_i.debug_mode_q),
163 ,.*)
164
165 // Debug assertion and coverage interface
166 uvmt_cv32e40p_debug_cov_assert_if debug_cov_assert_if(
167 .clk_i(clknrst_if.clk),
168 .rst_ni(clknrst_if.reset_n),
169 .fetch_enable_i(dut_wrap.cv32e40p_wrapper_i.core_i.fetch_
enable_i),
170 ,.if_stage_instr_rvalid_i(dut_wrap.cv32e40p_wrapper_i.core_
i.if_stage_i.instr_rvalid_i),
171 ,.if_stage_instr_rdata_i(dut_wrap.cv32e40p_wrapper_i.core_i
.if_stage_i.instr_rdata_i),
172 ,.id_stage_instr_valid_i(dut_wrap.cv32e40p_wrapper_i.core_i
.id_stage_i.instr_valid_i),
173 ,.id_stage_instr_rdata_i(dut_wrap.cv32e40p_wrapper_i.core_i
.id_stage_i.instr_rdata_i),
174 ,.id_stage_is_compressed(dut_wrap.cv32e40p_wrapper_i.core_i
.id_stage_i.is_compressed_i),
175 ,.id_val-
id(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.id_va
lid_i),
176 ,.is_decod-
ing(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.is_d
ecoding_o),
177 ,.id_stage_pc(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i
.pc_id_i),
178 ,.if_stage_pc(dut_wrap.cv32e40p_wrapper_i.core_i.if_stage_i
.pc_if_o),
179 ,.mie_q(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
mie_q),
180 ,.ctrl_fsm_cs(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i
.controller_i.ctrl_fsm_cs),
181 ,.illeg-
al_in-
sn_i(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.ill
egal_insn_i),
182 ,.illeg-
al_in-
sn_q(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.ill
egal_insn_q),
183 ,.ecall_insn_i(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_
i.controller_i.ecall_insn_i),
184 ,.de-
bug_req_i(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i
.debug_req_pending),
185 ,.de-
bug_mode_q(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_
i.debug_mode_q),
186 ,.dcsr_q(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
dcsr_q),
187 ,.depc_q(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
depc_q),
188 ,.depc_n(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
depc_n),
189 ,.mcause_q(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_
i.mcause_q),
190 ,.mtvec({dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
mtvec_q, 8'h00}),
191 ,.mepc_q(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
mepc_q),
192

```

```

193 .tdata1(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
tmatch_control_rdata),
194 .tdata2(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
tmatch_value_rdata),
195 .trig-
ger_match_i(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controlle
r_i.trigger_match_i),
196 .mcountinhibi-
t_q(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.mcountinhibi
t_q),
197 .mcycle(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.
mhpmmcounter_q[0]),
198 .min-
stret(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.mhpmmcounter_
q[2]),
199 .fence_i(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.de
coder_i.fencei_insn_o),
200
201 // TODO: review this change from CV32E40P_HASH f6196bf to
a26b194. It should be logically equivalent.
202 //assign debug_cov_assert_if.inst_ret =
dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.inst_ret;
203 // First attempt: this causes unexpected failures of
a_minstret_count
204 //assign debug_cov_assert_if.inst_ret =
(dut_wrap.cv32e40p_wrapper_i.core_i.id_valid &
205 //
dut_wrap.cv32e40p_wrapper_i.core_i.is_decoding);
206 // Second attempt: (based on OK input). This passes, but
maybe only because p_minstret_count
207 // is the only prop-
erty sensitive to inst_ret. Will
208 // this work in the
general case?
209 .inst_ret(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_
i.mhpmevent_minstret_i),
210 .csr_ac-
cess(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.csr_access),
211 .csr_op(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.csr_
op),
212 .csr_op_dec(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.
decoder_i.csr_op),
213 .csr_addr(dut_wrap.cv32e40p_wrapper_i.core_i.csr_addr),
214 .csr_we_int(dut_wrap.cv32e40p_wrapper_i.core_i.cs_registe
rs_i.csr_we_int),
215 .irq_ack_o(dut_wrap.cv32e40p_wrapper_i.core_i.irq_ack_o),
216 .irq_id_o(dut_wrap.cv32e40p_wrapper_i.core_i.irq_id_o),
217 .dm_halt_addr_i(dut_wrap.cv32e40p_wrapper_i.core_i.dm_hal
t_addr_i),
218 .dm_excep-
tion_addr_i(dut_wrap.cv32e40p_wrapper_i.core_i.dm_exception_addr_i),
219 .core_sleep_o(dut_wrap.cv32e40p_wrapper_i.core_i.core_sle
ep_o),
220 .irq_i(dut_wrap.cv32e40p_wrapper_i.core_i.irq_i),
221 .pc_set(dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.pc_
set_o),
222 .boot_ad-
dr_i(dut_wrap.cv32e40p_wrapper_i.core_i.boot_addr_i),
223 .branch_in_decode(dut_wrap.cv32e40p_wrapper_i.core_i.id_s
tage_i.controller_i.branch_in_id),
224
225 .is_wfi(),
226 .in_wfi(),
227 .dpc_will_hit(),
228 .addr_match(),
229 .is_ebreak(),
230 .is_cbreak(),
231 .is_dret(),
232 .is_mulhsu(),
233 .pending_enabled_irq()
234 );
235
236 // Instantiate debug assertions
237 uvmt_cv32e40p_debug_assert u_de-
bug_assert(.cov_assert_if(debug_cov_assert_if));
238
239 /**
240 * ISS WRAPPER instance:
241 */
242 uvmt_cv32e40p_iss_wrap #(
243 .ID (0)
244 )
245 iss_wrap (.clk_peri-
od(clknrst_if.clk_period),
246 .clknrst_if(clknrst_i
f_iss),
247 .step_com-
pare_if(step_compare_if),
248 .isa_covg_if(isa_covg_
if)
249 );
250
251 /**
252 * Step-and-Compare logic
253 */
254 uvmt_cv32e40p_step_compare step_compare
(.clknrst_if(clknrst_if),
255 .step_com-
pare_if(step_compare_if) );
256
257 always @(dut_wrap.cv32e40p_wrapper_i.tracer_i.retire) ->

```

```

step_compare_if.riscv_retire;
258 assign step_compare_if.insn_pc =
dut_wrap.cv32e40p_wrapper_i.tracer_i.insn_pc;
259 assign step_compare_if.riscy_GPR =
dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.register_file_i.mem;
260 assign clknrst_if_iss.reset_n = clknrst_if.reset_n;
261
262 // Connect step-and-compare signals to interrupt_if for
functional coverage of instructions and interrupts
263 assign interrupt_if.deferint = iss_wrap.io.deferint;
264 assign interrupt_if.ovp_cpu_state_stepi = step_com-
pare_if.ovp_cpu_state_stepi;
265
266 // Interrupt modeling logic - used to time interrupt entry
from RTL to the ISS
267 wire [31:0] irq_enabled;
268 reg [31:0] irq_deferint_ack;
269 reg [31:0] irq_deferint_sleep;
270 reg deferint_ack;
271 reg [31:0] irq_mip;
272 reg core_sleep_o_d;
273
274 always @(posedge clknrst_if.clk or negedge
clknrst_if.reset_n) begin
275 if (!clknrst_if.reset_n)
276 core_sleep_o_d <= 1'b0;
277 else
278 core_sleep_o_d <=
dut_wrap.cv32e40p_wrapper_i.core_sleep_o;
279 end
280
281 // Advance acknowledged interrupt to ISS when next valid
instruction decode executes
282 // Ignoring any instruction decodes in debug mode
283 wire id_start =
dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.id_valid_o &
284 dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.is_decoding_o &
~dut_wrap.cv32e40p_wrapper_i.core_i.debug_mode;
285
286 assign irq_enabled = dut_wrap.cv32e40p_wrapper_i.irq_i &
dut_wrap.cv32e40p_wrapper_i.core_i.cs_registers_i.mie_n;
287
288 /**
289 * step_compare_if.deferint_prime is set to 0 (asserted)
when the controller in ID commits to an interrupt
290 derefint_prime is then reset to 1 when the ID stage
commits to the next instruction (which should be the MTVEC entry ad-
dress)
291 */
292
293 always @(posedge clknrst_if.clk or negedge
clknrst_if.reset_n) begin
294 if (!clknrst_if_iss.reset_n) begin
295 step_compare_if.deferint_prime <= 1'b1;
296 step_compare_if.deferint_prime_ack <= 1'b1;
297 end
298 else if (dut_wrap.irq_ack) begin
299 step_compare_if.deferint_prime <= 1'b0;
300 step_compare_if.deferint_prime_ack <= 1'b0;
301 end
302 else if (core_sleep_o_d && irq_enabled) begin
303 step_compare_if.deferint_prime <= 1'b0;
304 end
305 else if (id_start && !step_compare_if.deferint_prime)
begin
306 step_compare_if.deferint_prime <= 1'b1;
307 step_compare_if.deferint_prime_ack <= 1'b1;
308 end
309 end
310
311 /**
312 * When the ID stage commits, we set deferint to the ISS
to signal to look at the interrupts
313 */
314 always @(posedge clknrst_if.clk or negedge
clknrst_if.reset_n) begin
315 if (!clknrst_if_iss.reset_n) begin
316 iss_wrap.io.deferint <= 1'b1;
317 deferint_ack <= 1'b1;
318 end
319 else if (id_start && !step_compare_if.deferint_prime)
begin
320 iss_wrap.io.deferint <= 1'b0;
321 deferint_ack <= step_compare_if.deferint_prime_ack;
322 end
323 end
324
325 /**
326 * deferint deassertion logic, on negedge of
ovp_cpu_state_stepi from the ISS the deferint has been consumed
327 */
328 always @(negedge step_compare_if.ovp_cpu_state_stepi) be-
gin
329 if (iss_wrap.io.deferint == 0) begin
330 iss_wrap.io.deferint <= 1'b1;
331 deferint_ack <= 1'b1;
332 irq_deferint_ack <= '0;
333 end
334 irq_deferint_sleep <= '0;

```

```

335     end
336
337     /**
338      * irq_deferint_ack will capture the asserted interrupt
to present to the ISS later
339      * since the autoclear/ack interface can clear the IRQ
long before the ISS sees it
340      */
341     always @(posedge clknrst_if.clk or negedge
clknrst_if.reset_n) begin
342         if (!clknrst_if.reset_n)
343             irq_deferint_ack <= '0;
344         else if (dut_wrap.irq_ack)
345             irq_deferint_ack <= (1 << dut_wrap.irq_id);
346     end
347
348     always @(posedge clknrst_if.clk or negedge
clknrst_if.reset_n) begin
349         if (!clknrst_if.reset_n)
350             irq_deferint_sleep <= '0;
351         else if (core_sleep_o_d)
352             irq_deferint_sleep <= irq_enabled;
353     end
354
355     always @*
356         iss_wrap.io.irq_i = iss_wrap.io.deferint ?
dut_wrap.irq :
357             !deferint_ack ? irq_deferint_ack :
358             irq_deferint_sleep;
359
360     /**
361      * Interrupt assertion to iss_wrap, note this runs on the
ISS clock (skewed from core clock)
362      */
363     always @(posedge clknrst_if.clk or negedge
clknrst_if.reset_n) begin
364         if (!clknrst_if.reset_n) begin
365             irq_mip <= '0;
366         end
367         else begin
368             for (int irq_idx=0; irq_idx<32; irq_idx++) begin
369                 // Leave ISS side asserted as long as RTL interrupt
line is asserted
370                 if (dut_wrap.cv32e40p_wrapper_i.irq_i[irq_idx])
371                     irq_mip[irq_idx] <= 'b1;
372                 // If deferint is low and ovp.cpu_state_stepi is as-
serted, then interrupt was consumed by model
373                 // Clear it now to avoid mip miscompare
374                 else if (step_compare_if.ovp.cpu_state_stepi &&
iss_wrap.io.deferint == 0)
375                     irq_mip[irq_idx] <= 'b0;
376                 // If RTL interrupt deasserts, but the core has not
taken the interrupt, then clear ISS irq
377                 else if (iss_wrap.io.deferint == 1)
378                     irq_mip[irq_idx] <= 'b0;
379             end
380         end
381     end
382
383     // Count number of issued and retired instructions
384     // This makes synchronizing haltreq to RM easier
385     logic [31:0] count_issue;
386     logic [31:0] count_retire;
387
388     always @(posedge clknrst_if.clk or negedge
clknrst_if.reset_n) begin
389         if (!clknrst_if.reset_n) begin
390             count_issue <= 32'h0;
391         end else begin
392             if
393                 ((dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.id_valid_o &&
dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.is_decoding_o &&
394                 !
dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.illegal_
insn_i) ||
395                 (dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.is_deco
ding_o &&
dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.ebrk_ins
n_i &&
396                 !
dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.trigger_match_i &&
397                 (dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.ebrk_fo
rce_debug_mode ||
dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.debug_mo
de_q)) begin
398                 count_issue <= count_issue + 1;
399             end
400         end
401     end
402
403     always @(dut_wrap.cv32e40p_wrapper_i.tracer_i.retire or
negedge clknrst_if.reset_n) begin
404         if (!clknrst_if.reset_n) begin
405             count_retire <= 32'h0;
406         end else begin
407             count_retire <= count_retire + 1;
408         end

```

```

409     end
410
411     // A simple FSM for controlling haltreq into RM
412     typedef enum logic [1:0] {INACTIVE, DBG_TAKEN, DRIVE_REQ}
dbg_state_e;
413     dbg_state_e debug_req_state;
414
415     always @(posedge clknrst_if_iss.clk or negedge
clknrst_if_iss.reset_n) begin
416         if (!clknrst_if_iss.reset_n) begin
417             iss_wrap.io.haltreq <= 'b0;
418             debug_req_state <= INACTIVE;
419         end else begin
420             unique case(debug_req_state)
421                 INACTIVE: begin
422                     iss_wrap.io.haltreq <= 'b0;
423
424                     // Only drive haltreq if we have an external
request
425                     if
426                         (dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.ctrl_fs
m_cs inside {cv32e40p_pkg::DBG_TAKEN_ID, cv32e40p_pkg::DBG_TAKEN_IF}
&&
427                         dut_wrap.cv32e40p_wrapper_i.core_i.id_stage_i.controller_i.debug_re
q_pending) begin
428                             debug_req_state <= DBG_TAKEN;
429                             // Already in sync, assert halreq right
away
430                             if (count_retire == count_issue) begin
431                                 iss_wrap.io.haltreq <= 'b1;
432                             end
433                         end
434                     end
435                     DBG_TAKEN: begin
436                         // Assert haltreq when we are in sync
437                         if (count_retire == count_issue) begin
438                             iss_wrap.io.haltreq <= 'b1;
439                             debug_req_state <= DRIVE_REQ;
440                         end
441                     end
442                     DRIVE_REQ: begin
443                         // Deassert haltreq when DM is observed
444                         if(iss_wrap.io.DM == 'b1) begin
445                             debug_req_state <= INACTIVE;
446                         end
447                     end
448                 default: begin
449                     debug_req_state <= INACTIVE;
450                 end
451             endcase
452         end
453     end
454
455     /**
456      * Test bench entry point.
457      */
458     initial begin : test_bench_entry_point
459
460         `ifdef PULP
461             `ifdef NO_PULP
462                 `uvm_fatal("CV32E40P_TB", "PULP and NO_PULP macros
are mutually exclusive.")
463             `endif
464         `endif
465
466         // Specify time format for simulation (units_number, pre-
cision_number, suffix_string, minimum_field_width)
467         $timeformat(-9, 3, " ns", 8);
468
469         // Add interfaces handles to uvm_config_db
470         uvm_config_db#(virtual uvma_de-
bug_if
bug_a-
gent"),
            .field_name("vif"),
            .value(debug_if)
            );
471         uvm_config_db#(virtual
uvma_clknrst_if
env.clknrst_agent"),
            .field_name("vif"),
            .val
ue(clknrst_if)
            );
472         uvm_config_db#(virtual uvma_inter-
rupt_if
rupt_a-
gent"),
            .field_name("vif"),
            .value(interrupt_if)
            );
473         uvm_config_db#(virtual
uvma_obi_memory_if
env.obi_memory_instr_agent"),
            .field_name("vif"),
            .val
ue(obi_memory_instr_if)
            );
474         uvm_config_db#(virtual
uvma_obi_memory_if
env.obi_memory_data_agent"),
            .field_name("vif"),
            .val
ue(obi_memory_data_if)
            );
475         uvm_config_db#(virtual
uvmt_cv32e40p_vp_status_if
),
            .field_name("vp_status_vif"),
            .val
ue(vp_status_if)
            );
476         uvm_config_db#(virtual
uvmt_cv32e40p_core_cntrl_if
            )::set(.cntxt(null), .inst_name("**

```

```

),
    .field_name("core_cntrl_vif"), .val
ue(core_cntrl_if)
);
477 uvm_config_db#(virtual
uvm_cv32e40p_core_status_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("core_status_vif"), .val
ue(core_status_if)
);
478 uvm_config_db#(virtual
uvm_cv32e40p_step_compare_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("step_compare_vif"), .val
ue(step_compare_if)
);
479 uvm_config_db#(virtual
uvm_cv32e40p_isa_covg_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("isa_covg_vif"), .val
ue(isa_covg_if)
);
480 uvm_config_db#(virtual
uvm_cv32e40p_debug_cov_assert_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("debug_cov_vif"), .val
ue(debug_cov_assert_if)
);
481 uvm_config_db#(virtual
uvm_cv32e40p_vp_status_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("vp_status_vif"), .val
ue(vp_status_if)
);
482 uvm_config_db#(virtual
uvm_cv32e40p_isa_covg_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("isa_covg_vif"), .val
ue(isa_covg_if)
);
483 uvm_config_db#(virtual uvm_inter-
rupt_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("intr_vif"), .value(interrupt_
t_if)
);
484 uvm_config_db#(virtual uvm_de-
bug_if
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("debug_vif"), .value(debug_
if)
);
485 uvm_config_db#(virtual
RVV_memory
)::set(.cntxt(null), .inst_name(""));
),
    .field_name("rvvi_memory_vif"), .val
ue(iss_wrap.ram.memory)
);
486
487 // Make the DUT Wrapper Virtual Peripheral's status out-
puts available to the base_test
488 uvm_con-
fig_db#(bit)::set(.cntxt(null), .inst_name(""));
),
    .field_name("tp"), .value(1'b0)
);
489 uvm_con-
fig_db#(bit)::set(.cntxt(null), .inst_name(""));
),
    .field_name("tf"), .value(1'b0)
);
490 uvm_con-
fig_db#(bit)::set(.cntxt(null), .inst_name(""));
),
    .field_name("evalid"), .value(1'b0)
);
491 uvm_con-
fig_db#(bit[31:0])::set(.cntxt(null), .inst_name(""));
),
    .field_name("evalvalue"), .value(32'h00000000)
);
492
493 // DUT and ENV parameters
494 uvm_con-
fig_db#(int)::set(.cntxt(null), .inst_name(""));
),
    .field_name("CORE_
PARAM_PULP_XPULP"), .value(CORE_PARAM_PULP_XPULP)
);
495 uvm_con-
fig_db#(int)::set(.cntxt(null), .inst_name(""));
),
    .field_name("CORE_
PARAM_PULP_CLUSTER"), .value(CORE_PARAM_PULP_CLUSTER)
);
496 uvm_con-
fig_db#(int)::set(.cntxt(null), .inst_name(""));
),
    .field_name("CORE_
PARAM_PULP_ZFINX"), .value(CORE_PARAM_PULP_ZFINX)
);
497 uvm_con-
fig_db#(int)::set(.cntxt(null), .inst_name(""));
),
    .field_name("CORE_
PARAM_NUM_MHPMCOUNTERS"), .value(CORE_PARAM_NUM_MHPMCOUNTERS)
);
498 uvm_con-
fig_db#(int)::set(.cntxt(null), .inst_name(""));
),
    .field_name("ENV_P
ARAM_INSTR_ADDR_WIDTH"), .value(ENV_PARAM_INSTR_ADDR_WIDTH)
);
499 uvm_con-
fig_db#(int)::set(.cntxt(null), .inst_name(""));
),
    .field_name("ENV_P
ARAM_INSTR_DATA_WIDTH"), .value(ENV_PARAM_INSTR_DATA_WIDTH)
);
500 uvm_con-
fig_db#(int)::set(.cntxt(null), .inst_name(""));
),
    .field_name("ENV_P
ARAM_RAM_ADDR_WIDTH"), .value(ENV_PARAM_RAM_ADDR_WIDTH)
);
501
502 // Run test
503 uvm_top.enable_print_topology = 0; // ENV coders enable
this as a debug aid
504 uvm_top.finish_on_completion = 1;
505 uvm_top.run_test();
506 end : test_bench_entry_point
507
508 assign core_cntrl_if.clk = clkrst_if.clk;
509
510 // Capture the test status and exit pulse flags
511 // TODO: put this logic in the vp_status_if (makes it easi-
er to pass to ENV)
512 always @(posedge clkrst_if.clk) begin
513     if (!clkrst_if.reset_n) begin
514         tp <= 1'b0;
515         tf <= 1'b0;
516         evalid <= 1'b0;
517         evalvalue <= 32'h00000000;
518     end
519     else begin
520         if (vp_status_if.tests_passed) begin
521             tp <= 1'b1;
522             uvm_con-
fig_db#(bit)::set(.cntxt(null), .inst_name(""));
),
    .field_name("tp"),

```

```

    .value(1'b1));
523     end
524     if (vp_status_if.tests_failed) begin
525         tf <= 1'b1;
526         uvm_con-
fig_db#(bit)::set(.cntxt(null), .inst_name(""));
),
    .field_name("tf"),
    .value(1'b1));
527     end
528     if (vp_status_if.exit_valid) begin
529         evalid <= 1'b1;
530         uvm_con-
fig_db#(bit)::set(.cntxt(null), .inst_name(""));
),
    .field_name("evalid"),
    .value(1'b1));
531     end
532     if (vp_status_if.exit_valid) begin
533         evalvalue <= vp_status_if.exit_value;
534         uvm_con-
fig_db#(bit[31:0])::set(.cntxt(null), .inst_name(""));
),
    .field_name("evalvalue"),
    .value(vp_status_if.exit_value));
535     end
536 end
537 end
538
539 /**
540 * End-of-test summary printout.
541 */
542
543 final begin: end_of_test
544     uvm_report_server rs;
545     int err_count;
546     int warning_count;
547     int fatal_count;
548     static bit sim_finished = 0;
549
550     rs = uvm_top.get_report_server();
551     err_count = rs.get_severity_count(UVM_ERROR);
552     warning_count = rs.get_severity_count(UVM_WARNING);
553     fatal_count = rs.get_severity_count(UVM_FATAL);
554
555     void'(uvm_config_db#(bit)::get(null, "", "sim_finished",
sim_finished));
556
557     // In most other contexts, calls to $display() in a UVM
environment are
558 // illegal. Here they are OK because the UVM environment
has shut down
559 // and we are merely dumping a summary to stdout.
560 // @DVT_LINTER_WAIVER_START "MT20210811_3" disable SVTB.
29.1.7
561     $display("\n%0m: *** Test Summary ***\n");
562
563     if (sim_finished && (err_count == 0) && (fatal_count ==
0)) begin
564         $display("PPPPPPPP AAAAAA SSSSSS SSSSSS
EEEEEEEE DDDDDDD");
565         $display("PP PP AA AA SS SS SS
DD DD");
566         $display("PP PP AA AA SS SS
DD DD");
567         $display("PPPPPPPP AAAAAAAA SSSSSS SSSSSS
EEEEEE DD DD");
568         $display("PP AA AA SS SS SS
DD DD");
569         $display("PP AA AA SS SS SS SS
DD DD");
570         $display("PP AA AA SSSSSS SSSSSS
EEEEEEEE DDDDDDD");
571         $display("-----");
572         if (warning_count == 0) begin
573             $display("SIMULATION
PASSED");
574         end
575         else begin
576             $display("SIMULATION PASSED with
WARNINGS");
577         end
578         $display("-----");
579     end
580     else begin
581         $display("FFFFFFFF AAAAAA IIIIII LL
EEEEEEEE DDDDDDD");
582         $display("FF AA AA II LL
DD DD");
583         $display("FF AA AA II LL
DD DD");
584         $display("FFFFFF AAAAAAAA II LL
EEEEEE DD DD");
585         $display("FF AA AA II LL
DD DD");
586         $display("FF AA AA II LL
DD DD");

```



```

EE      DD      DD      ");
587      $display(" FF      AA      AA      IIIIII      LLLLLLLL
EEEEEEEE      DDDDDDD      ");
588
589      if (sim_finished == 0) begin
590          $display("
-----");
591          $display(" SIMULATION FAILED -
ABORTED      ");
592          $display("
-----");
593      end
594      else begin
595          $display("
-----");
596          $display(" SIMULATION
FAILED      ");
597          $display("
-----");
598      end
599      end
600      //@DVT_LINTER_WAIVER_END "MT20210811_3"
601  end
602
603 endmodule : uvmt_cv32e40p_tb
604 `default_nettype wire
605
606 `endif // __UVMT_CV32E40P_TB_SV__

```

11. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tb_ifs.sv (457 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 //      https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16 //
17 // This file specifies all interfaces used by the CV32E40P
test bench (uvmt_cv32e40p_tb).
18 // Most interfaces support tasks to allow control by the ENV
or test cases.
19
20
21 `ifndef __UVMT_CV32E40P_TB_IFS_SV__
22 `define __UVMT_CV32E40P_TB_IFS_SV__
23
24 /**
25  * Clock and reset generation. Drives core_clock and core_re-
set_n with
26  * configurable period and reset assertion/deassertion timing.
27  */
28 interface uvmt_cv32e40p_clk_gen_if (output logic core_clock,
output logic core_reset_n);
29
30 import uvm_pkg::*;
31
32 // Default timing (ps). TODO: drive from uvme_cv32e40p_* ENV
CFG.
33 localparam real DEFAULT_CLOCK_PERIOD_PS = 1500.0;
34 localparam real DEFAULT_RESET_DURATION_PS = 7400.0;
35
36 bit start_clk = 0;
37 realtime core_clock_period = DEFAULT_CLOCK_PERIOD_PS
* 1ps;
38 realtime reset_deassert_duration = DE-
FAULT_RESET_DURATION_PS * 1ps;
39 realtime reset_assert_duration = DEFAULT_RESET_DURATION_PS *
1ps;
40
41 /**
42  * Generates clock and reset signals.
43  * If reset_n comes up de-asserted (1'b1), wait a bit, then
assert, then de-assert.
44  * Otherwise, leave reset asserted, wait a bit, then de-as-
sert.
45  */
46 initial begin
47     core_clock = 0;
48     core_reset_n = 0;
49     wait (start_clk);
50     fork

```

```

51     forever begin
52         #(core_clock_period/2) core_clock = ~core_clock;
53     end
54     begin
55         if (core_reset_n == 1'b1) #(reset_deassert_duration);
56         core_reset_n = 1'b0;
57         #(reset_assert_duration);
58         core_reset_n = 1'b1;
59     end
60     join_none
61 end
62
63 /**
64  * Sets clock period in ps.
65  */
66 function static void set_clk_period(real clk_period);
67     core_clock_period = clk_period * 1ps;
68 endfunction : set_clk_period
69
70 /**
71  * Triggers the generation of clk.
72  */
73 function static void start();
74     start_clk = 1;
75     `uvm_info("CLK_GEN_IF", "uvmt_cv32e40p_clk_gen_if.start()
called", UVM_NONE)
76 endfunction : start
77
78 endinterface : uvmt_cv32e40p_clk_gen_if
79
80 /**
81  * Status information generated by the Virtual Peripherals in
the DUT WRAPPER memory.
82  */
83 interface uvmt_cv32e40p_vp_status_if (
84     output reg tests_passed,
85     output reg tests_failed,
86     output reg exit_valid,
87     output reg [31:0] exit_value
88 );
89
90 import uvm_pkg::*;
91
92 // TODO: X/Z checks
93 initial begin
94     end
95
96 endinterface : uvmt_cv32e40p_vp_status_if
97
98 /**
99  * Quasi-static core control signals.
100  */
101 interface uvmt_cv32e40p_core_cntrl_if (
102     input clk,
103     output logic fetch_en,
104     output logic pulp_clock_en,
105     output logic scan_cg_en,
106     output logic [31:0] boot_addr,
107     output logic [31:0] mtvec_addr,
108     output logic [31:0] dm_halt_addr,
109     output logic [31:0] dm_exception_addr,
110     output logic [31:0] hart_id,
111     output logic debug_req,
112     output logic load_instr_mem
113 );
114
115 import uvm_pkg::*;
116
117 // Quasi-static values (used once after reset; plusarg can
override mtvec_addr).
118 localparam logic [31:0] BOOT_ADDR = 32'h0000_0080;
119 localparam logic [31:0] MTVEC_ADDR = 32'h0000_0000;
120 localparam logic [31:0] DM_HALT_ADDR = 32'h1A11_0800;
121 localparam logic [31:0] DM_EXCEPTION_ADDR = 32'h1A11_1000;
122 localparam logic [31:0] HART_ID = 32'h0000_0000;
123 localparam int GO_FETCH_DELAY_CYCLES = 10;
124
125 // Address bin ranges for coverage (shared across cover-
points).
126 localparam logic [31:0] ADDR_BIN_LOW_MIN = 32'h0000_0000;
127 localparam logic [31:0] ADDR_BIN_LOW_MAX = 32'h0000_FFFF;
128 localparam logic [31:0] ADDR_BIN_MED_MIN = 32'h0001_0000;
129 localparam logic [31:0] ADDR_BIN_MED_MAX = 32'hFFFF_FFFF;
130 localparam logic [31:0] ADDR_BIN_HIGH_MIN = 32'hF000_0000;
131 localparam logic [31:0] ADDR_BIN_HIGH_MAX = 32'hFFFF_FFFF;
132
133 string qsc_stat_str; // Quasi-static control status
134
135 logic fb;
136 logic lfsr_reset;
137 logic [15:0] lfsr;
138
139 covergroup core_cntrl_cg;
140
141 // For CV32E40P, pulp_clock_en / scan_cg_en are tied to 0;
bins left for future use.
142 // pulp_clock_enable: coverpoint pulp_clock_en { bins en-
abled = {1'b1}; bins disabled = {1'b0}; }
143 // scan_enable: coverpoint scan_cg_en { bins en-
abled = {1'b1}; bins disabled = {1'b0}; }
144 boot_address: coverpoint boot_addr {

```

```

143     bins low = {[ADDR_BIN_LOW_MIN : ADDR_BIN_LOW_MAX]};
144     bins med = {[ADDR_BIN_MED_MIN : ADDR_BIN_MED_MAX]};
145     bins high = {[ADDR_BIN_HIGH_MIN : ADDR_BIN_HIGH_MAX]};
146 }
147 mtvec_address: coverpoint mtvec_addr {
148     bins low = {[ADDR_BIN_LOW_MIN : ADDR_BIN_LOW_MAX]};
149     bins med = {[ADDR_BIN_MED_MIN : ADDR_BIN_MED_MAX]};
150     bins high = {[ADDR_BIN_HIGH_MIN : ADDR_BIN_HIGH_MAX]};
151 }
152 debug_module_halt_address: coverpoint dm_halt_addr {
153     bins low = {[ADDR_BIN_LOW_MIN : ADDR_BIN_LOW_MAX]};
154     bins med = {[ADDR_BIN_MED_MIN : ADDR_BIN_MED_MAX]};
155     bins high = {[ADDR_BIN_HIGH_MIN : ADDR_BIN_HIGH_MAX]};
156 }
157 debug_module_exception_address: coverpoint dm_except-
tion_addr {
158     bins low = {[ADDR_BIN_LOW_MIN : ADDR_BIN_LOW_MAX]};
159     bins med = {[ADDR_BIN_MED_MIN : ADDR_BIN_MED_MAX]};
160     bins high = {[ADDR_BIN_HIGH_MIN : ADDR_BIN_HIGH_MAX]};
161 }
162 hart_id: coverpoint hart_id {
163     bins low = {[ADDR_BIN_LOW_MIN : ADDR_BIN_LOW_MAX]};
164     bins med = {[ADDR_BIN_MED_MIN : ADDR_BIN_MED_MAX]};
165     bins high = {[ADDR_BIN_HIGH_MIN : ADDR_BIN_HIGH_MAX]};
166 }
167 endgroup : core_cntrl_cg
168
169 core_cntrl_cg core_cntrl_cg_inst = new();
170
171 // TODO: randomize hart_id (should have no effect); random-
ize boot_addr/mtvec_addr
172 // in sync with test program start address.
173 initial begin : quasi_static_controls
174     lfsr_reset = 1'b1;
175     fetch_en = 1'b0;
176     debug_req = 1'b0;
177     pulp_clock_en = 1'b0;
178     scan_cg_en = 1'b0;
179     boot_addr = BOOT_ADDR;
180     mtvec_addr = MTVEC_ADDR;
181     dm_halt_addr = DM_HALT_ADDR;
182     dm_exception_addr = DM_EXCEPTION_ADDR;
183     hart_id = HART_ID;
184
185     if ($value$plusargs("mtvec_addr=0x%x", mtvec_addr)) begin
186         string override;
187         int fh;
188
189         override = $sformatf("--override root/cpu/mtvec=0x%08x",
{mtvec_addr[31:8], 8'h01});
190         fh = $fopen("ovpsim.ic", "a");
191         $fwrite(fh, " %s\n", override);
192         $fclose(fh);
193     end
194
195     qsc_stat_str = $sform-
atf("\tpulp_clock_en = %0d\n", pulp_clock_en);
196     qsc_stat_str = {qsc_stat_str, $sform-
atf("\tscan_cg_en = %0d\n", scan_cg_en)};
197     qsc_stat_str = {qsc_stat_str, $sform-
atf("\tboot_addr = %08h\n", boot_addr)};
198     qsc_stat_str = {qsc_stat_str, $sform-
atf("\tmtvec_addr = %08h\n", mtvec_addr)};
199     qsc_stat_str = {qsc_stat_str, $sform-
atf("\tdm_halt_addr = %08h\n", dm_halt_addr)};
200     qsc_stat_str = {qsc_stat_str, $sform-
atf("\tdm_exception_addr = %08h\n", dm_exception_addr)};
201     qsc_stat_str = {qsc_stat_str, $sform-
atf("\thart_id = %08h\n", hart_id)};
202
203     `uvm_info("CORE_CNTRL_IF", $sformatf("Quasi-static CORE
control inputs:\n%s", qsc_stat_str), UVM_NONE)
204 end
205
206 clocking drv_cb @(posedge clk);
207 output fetch_en;
208 endclocking : drv_cb
209
210 /** Drives fetch_en high; samples coverage and releases LFSR
after delay. */
211 task static go_fetch();
212     drv_cb.fetch_en <= 1'b1;
213     `uvm_info("CORE_CNTRL_IF",
"uvmt_cv32e40p_core_cntrl_if.go_fetch() called", UVM_DEBUG)
214     core_cntrl_cg_inst.sample();
215     repeat (GO_FETCH_DELAY_CYCLES) @(posedge clk);
216     lfsr_reset <= 1'b0;
217 endtask : go_fetch
218
219 /** Drives fetch_en low and resets LFSR. */
220 function void stop_fetch();
221     drv_cb.fetch_en <= 1'b0;
222     lfsr_reset <= 1'b1;
223     `uvm_info("CORE_CNTRL_IF",
"uvmt_cv32e40p_core_cntrl_if.stop_fetch() called", UVM_DEBUG)
224 endfunction : stop_fetch
225
226 // LFSR feedback and update: toggles fetch_en after initial
assertion to show
227 // core ignores fetch_en once running. TODO: make constrain-

```

```

able; remove DVT waiver.
227 assign fb = !(lfsr[15] ^ lfsr[13] ^ lfsr[12] ^ lfsr[10]);
228
229 always @(posedge clk) begin : lfsr_update
230     if (lfsr_reset) begin
231         lfsr <= $urandom(); //@DVT_LINTER_WAIVER "MT20210811_2"
232     end else begin
233         lfsr <= {lfsr[14:0], fb};
234         drv_cb.fetch_en <= lfsr[15];
235     end
236 end
237
238 endinterface : uvmt_cv32e40p_core_cntrl_if
239
240 /**
241  * Core status signals.
242  */
243 interface uvmt_cv32e40p_core_status_if (
244     input wire core_busy,
245     input logic sec_lvl
246 );
247
248 import uvm_pkg::*;
249
250 endinterface : uvmt_cv32e40p_core_status_if
251
252 /**
253  * ISA coverage interface
254  * ISS wrapper will fill in ins (instruction) and fire
ins_valid event
255  */
256 interface uvmt_cv32e40p_isa_covg_if;
257
258 import uvm_pkg::*;
259 import uvme_cv32e40p_pkg::*;
260
261 event ins_valid;
262 ins_t ins;
263
264 endinterface : uvmt_cv32e40p_isa_covg_if
265
266 /**
267  * Step and compare interface
268  * Xcelium does not support event types in the module port
list
269  */
270 interface uvmt_cv32e40p_step_compare_if;
271
272 import uvm_pkg::*;
273
274 // From RTL riscv_tracer.v
275 typedef struct {
276     logic [ 5:0] addr;
277     logic [31:0] value;
278 } reg_t;
279
280 event ovp_cpu_valid; // Instruction success-
fully retired (ISS)
281 event ovp_cpu_trap; // Exception occurred
(ISS)
282 event ovp_cpu_halt; // Halt occurred (ISS)
283 bit [31:0] ovp_cpu_PCR;
284 logic [31:0] ovp_cpu_GPR[32];
285 bit ovp_cpu_state_idle;
286 bit ovp_cpu_state_stepi;
287 bit ovp_cpu_state_stop;
288 bit ovp_cpu_state_cont;
289
290 event riscv_retire; // RTL retire
291 event riscv_trap; // RTL took a trap
292 event riscv_halt; // RTL took a halt
293
294 logic [31:0] insn_pc;
295 logic [31:0][31:0] riscy_GPR;
296 logic deferint_prime;
297 logic deferint_prime_ack;
298
299 int num_pc_checks;
300 int num_gpr_checks;
301 int num_csr_checks;
302
303 /** Report step-and-compare checker counts at end of simula-
tion. */
304 function void report_step_compare();
305     if (num_pc_checks > 0) begin
306         `uvm_info("step_compare", $sformatf("Checked PC %0d
times", num_pc_checks), UVM_LOW)
307     end else begin
308         `uvm_error("step_compare", "PC was checked 0 times!")
309     end
310     if (num_gpr_checks > 0) begin
311         `uvm_info("step_compare", $sformatf("Checked GPR %0d
times", num_gpr_checks), UVM_LOW)
312     end else begin
313         `uvm_error("step_compare", "GPR was checked 0 times!")
314     end
315     if (num_csr_checks > 0) begin
316         `uvm_info("step_compare", $sformatf("Checked CSR %0d
times", num_csr_checks), UVM_LOW)
317     end else begin

```



```

318     `uvm_error("step_compare", "CSR was checked 0 times!")
319   end
320   endfunction : report_step_compare
321
322   endinterface : uvmt_cv32e40p_step_compare_if
323
324   //
-----
325   // Debug and coverage assertions: internal core signals for
326   // SVA and covergroups.
-----
327   interface uvmt_cv32e40p_debug_cov_assert_if
328     import cv32e40p_pkg::*;
329     (
330       input  clk_i,
331       input  rst_ni,
332
333       input          fetch_enable_i,
334
335       input [31:0]  irq_i,
336       input          irq_ack_o,
337       input [4:0]   irq_id_o,
338       input [31:0]  mie_q,
339
340       input          if_stage_instr_rvalid_i,
341       input [31:0]   if_stage_instr_rdata_i,
342
343       input          id_stage_instr_valid_i,
344       input [31:0]   id_stage_instr_rdata_i,
345       input          id_stage_is_compressed,
346       input [31:0]   id_stage_pc,
347       input [31:0]   if_stage_pc,
348       input          is_decoding,
349       input          id_valid,
350       input wire     ctrl_state_e ctrl_fsm_cs,
351       input          illegal_insn_i,
352       input          illegal_insn_q,
353       input          ecall_insn_i,
354
355       input [31:0]   boot_addr_i,
356
357       input          debug_req_i,
358       input          debug_mode_q,
359       input [31:0]   dcsr_q,
360       input [31:0]   depc_q,
361       input [31:0]   depc_n,
362       input [31:0]   dm_halt_addr_i,
363       input [31:0]   dm_exception_addr_i,
364
365       input [5:0]    mcause_q,
366       input [31:0]   mtvec,
367       input [31:0]   mepc_q,
368       input [31:0]   tdata1,
369       input [31:0]   tdata2,
370       input          trigger_match_i,
371
372       input [31:0]   mcountinhibit_q,
373       input [63:0]   mcycle,
374       input [63:0]   minstret,
375       input          inst_ret,
376
377       input          core_sleep_o,
378
379       input          fence_i,
380
381       input          csr_access,
382       input [1:0]    csr_op,
383       input [1:0]    csr_op_dec,
384       input [11:0]   csr_addr,
385       input          csr_we_int,
386
387       output logic    is_wfi,
388       output logic    in_wfi,
389       output logic    dpc_will_hit,
390       output logic    addr_match,
391       output logic    is_ebreak,
392       output logic    is_ceilbreak,
393       output logic    is_dret,
394       output logic    is_mulhsu,
395       output logic [31:0] pending_enabled_irq,
396       input          pc_set,
397       input          branch_in_decode
398     );
399
400     clocking mon_cb @(posedge clk_i);
401     input #1step
402       fetch_enable_i,
403       irq_i,
404       irq_ack_o,
405       irq_id_o,
406       mie_q,
407       if_stage_instr_rvalid_i,
408       if_stage_instr_rdata_i,
409       id_stage_instr_valid_i,
410       id_stage_instr_rdata_i,
411       id_stage_is_compressed,
412       id_stage_pc,
413       if_stage_pc,

```

```

414     ctrl_fsm_cs,
415     illegal_insn_i,
416     illegal_insn_q,
417     ecall_insn_i,
418     boot_addr_i,
419     debug_req_i,
420     debug_mode_q,
421     dcsr_q,
422     depc_q,
423     depc_n,
424     dm_halt_addr_i,
425     dm_exception_addr_i,
426     mcause_q,
427     mtvec,
428     mepc_q,
429     tdata1,
430     tdata2,
431     trigger_match_i,
432     fence_i,
433     mcountinhibit_q,
434     mcycle,
435     minstret,
436     inst_ret,
437     core_sleep_o,
438     csr_access,
439     csr_op,
440     csr_op_dec,
441     csr_addr,
442     is_wfi,
443     in_wfi,
444     dpc_will_hit,
445     addr_match,
446     is_ebreak,
447     is_ceilbreak,
448     is_dret,
449     is_mulhsu,
450     pending_enabled_irq,
451     pc_set,
452     branch_in_decode;
453   endclocking : mon_cb
454
455   endinterface : uvmt_cv32e40p_debug_cov_assert_if
456
457   `endif // __UVMT_CV32E40P_TB_IFS_SV__

```

12. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_tdefs.sv (35 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Datum Technology Corporation
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0
6 // (the "License");
7 // you may not use this file except in compliance with the Li-
8 // cense.
9 // You may obtain a copy of the License at
10 // https://solderpad.org/licenses/
11 // Unless required by applicable law or agreed to in writing,
12 // software
13 // distributed under the License is distributed on an "AS IS"
14 // BASIS,
15 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
16 // press or implied.
17 // See the License for the specific language governing permis-
18 // sions and
19 // limitations under the License.
20 //
21
22 `ifndef __UVMT_CV32E40P_TDEFS_SV__
23 `define __UVMT_CV32E40P_TDEFS_SV__
24
25 /**
26  * Test Program Type. See the Verification Strategy for a
27  * discussion of this.
28  */
29 typedef enum {
30   PREEXISTING_SELFCHECKING,
31   PREEXISTING_NOTSELFCHECKING,
32   GENERATED_SELFCHECKING,
33   GENERATED_NOTSELFCHECKING,
34   NO_TEST_PROGRAM
35 } test_program_type;
36
37 `endif // __UVMT_CV32E40P_TDEFS_SV__

```

13. openhwgroup_core-v-verif/cv32e40p/tb/uvmt/uvmt_cv32e40p_uvm_macros_inc.sv (30 lines)

```

1 //
2 // Copyright 2020 OpenHW Group
3 // Copyright 2020 Silicon Labs, Inc.
4 //
5 // Licensed under the Solderpad Hardware Licence, Version 2.0

```

```

(the "License");
6 // you may not use this file except in compliance with the Li-
cense.
7 // You may obtain a copy of the License at
8 //
9 //      https://solderpad.org/licenses/
10 //
11 // Unless required by applicable law or agreed to in writing,
software
12 // distributed under the License is distributed on an "AS IS"
BASIS,
13 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either ex-
press or implied.
14 // See the License for the specific language governing permis-
sions and
15 // limitations under the License.
16 //
17

```

```

18
19 `ifndef __UVMT_CV32E40P_UVM_MACROS_INC_SV__
20 `define __UVMT_CV32E40P_UVM_MACROS_INC_SV__
21
22 // Simple inclusion of the uvm_macros.svh file into compila-
tion scope.
23 // This should only be used in Xcelium where automatic load of
UVM does not
24 // include the macros definition file.
25 // use of this include file "first" in the simulator compila-
tion filelist
26 // ensures all macros are properly defined for usage
27
28 `include "uvm_macros.svh"
29
30 `endif

```

Spike ISS -- 참조모델 본체

15 files • 5,116 lines

API — abstract_device.h

Table 1313. riscv-software-src_riscv-isa-sim/riscv/abstract_device.h — 클래스/구조체 (2개)

이름	템플릿 인자	종류	상속
abstract_device_t		class	
device_factory_t		class	

Table 1314. riscv-software-src_riscv-isa-sim/riscv/abstract_device.h — typedef / using (1개)

새 이름	원래 형
mmio_device_map_t	std::map<std::string, const device_factory_t*>

Table 1315. riscv-software-src_riscv-isa-sim/riscv/abstract_device.h — #define / `define 매크로 (3개)

매크로	인자	정의
_RISCV_ABSTRACT_DEVICE_H		#include "decode.h"
REGISTER_DEVICE	(name, parse, generate)	\
REGISTER_BUILTIN_DEVICE	(name, parse, generate)	\

API — abstract_interrupt_controller.h

Table 1316. riscv-software-src_riscv-isa-sim/riscv/abstract_interrupt_controller.h — 클래스/구조체 (1개)

이름	템플릿 인자	종류	상속
abstract_interrupt_controller_t		class	

Table 1317. riscv-software-src_riscv-isa-sim/riscv/abstract_interrupt_controller.h — #define / `define 매크로 (1개)

매크로	인자	정의
_RISCV_ABSTRACT_INTERRUPT_CONTROLLER_H		#include "decode.h"

API — aplic.cc

Table 1318. riscv-software-src_riscv-isa-sim/riscv/aplic.cc — 함수 (4개)

함수	반환형	받는 파라미터
aplic_m_generate_dts	std::string	const sim_t* sim, const std::vector<std::string>& sargs UNUSED
aplic_s_generate_dts	std::string	const sim_t* sim, const std::vector<std::string>& sargs UNUSED
ap-lic_m_parse_from_fdt	ap-lic_m_t*	const void* fdt, const sim_t* sim, reg_t* base, const std::vector<std::string>& sargs UNUSED
ap-lic_s_parse_from_fdt	aplic_s_t*	const void* fdt, const sim_t* sim, reg_t* base, const std::vector<std::string>& sargs UNUSED

Table 1319. riscv-software-src_riscv-isa-sim/riscv/aplic.cc — #define / `define 매크로 (30개)

매크로	인자	정의
DOMAINCFG		0x0
SOURCECFG_BASE		0x4
MMSIADDRCFG		0x1bc0
MMSIADDRCFGH		0x1bc4
SMSIADDRCFG		0x1bc8
SMSIADDRCFGH		0x1bcc
SETIP_BASE		0x1c00
SETIPNUM		0x1cdc
IN_CLRIP_BASE		0x1d00
CLRIPNUM		0x1ddc
SETIE_BASE		0x1e00
SETIENUM		0x1edc
CLRIE_BASE		0x1f00
CLRIENUM		0x1fdc
GENMSI		0x3000
TARGET_BASE		0x3004
IDC		0x4000
DOMAINCFG_IE_MASK		0x100
DOMAINCFG_DM_MASK		0x4
SOURCECFG_D_MASK		0x400
SOURCECFG_CHILD_MASK		0x3ff
SOURCECFG_SM_MASK		0x7
MMSIADDRCFGH_L_MASK		0x80000000
GENMSI_HART_MASK		0xffc0000
GENMSI_BUSY_MASK		0x1000
GENMSI_EIID_MASK		0x7ff
TARGET_HART_MASK		0xffc0000
TARGET_GUEST_MASK		0x3f000
TARGET_EIID_MASK		0x7ff
APLIC_MMIO_REG_SIZE		4

API — arith.h**Table 1320. riscv-software-src_riscv-isa-sim/riscv/arith.h — 함수 (18개)**

함수	반환형	받는 파라미터
mulhu	inline uint64_t	uint64_t a, uint64_t b
mulh	inline int64_t	int64_t a, int64_t b
mulhsu	inline int64_t	int64_t a, uint64_t b
sat_add	template<typename T, typename UT> static inline T	T x, T y, bool &sat
sat_add	template<typename T, typename UT> static inline T	T x, T y, T z, bool &sat
sat_sub	template<typename T, typename UT> static inline T	T x, T y, bool &sat
sat_addu	template<typename T> T	T x, T y, bool &sat
sat_subu	template<typename T> T	T x, T y, bool &sat
extract64	static inline uint64_t	uint64_t val, int pos, int len
make_mask64	static inline uint64_t	int pos, int len
popcount	static inline int	uint64_t val
ctz	static inline int	uint64_t val
clz	static inline int	uint64_t val
cto	static inline int	uint64_t val
log2	static inline int	uint64_t val
xperm	static inline uint64_t	uint64_t rs1, uint64_t rs2, size_t sz_log2, size_t len
rotate_right	template <typename T> static inline T	T x, std::size_t shiftamt
rotate_left	template <typename T> static inline T	T x, std::size_t shiftamt

Table 1321. riscv-software-src_riscv-isa-sim/riscv/arith.h — #define / `define 매크로 (1개)

매크로	인자	정의
_RISCV_ARITH_H		#include <cassert>

API — bloom_filter.h

Table 1322. riscv-software-src_riscv-isa-sim/riscv/bloom_filter.h — 클래스/구조체 (3개)

이름	템플릿 인자	종류	상속
simple_hash1		struct	
simple_hash2		struct	
bloom_filter_t		class	

Table 1323. riscv-software-src_riscv-isa-sim/riscv/bloom_filter.h — #define / `define 매크로 (1개)

매크로	인자	정의
_RISCV_BLOOM_FILTER_H		#include <bitset>

API — bulknormdot.h

Table 1324. riscv-software-src_riscv-isa-sim/riscv/bulknormdot.h — 클래스/구조체 (5개)

이름	템플릿 인자	종류	상속
bulk_norm_out_t		struct	
DotConfig		class	
FloatFormat	typename U, typename M, typename E	class	
IEEEFloatFormat	typename U, typename M, typename E, unsigned expWidth, unsigned mantWidth	class	FloatFormat<U, M, E>
fp16_norm_t		class	public IEEEFloatFormat<uint16_t, uint16_t, uint8_t, 5, 10>

Table 1325. riscv-software-src_riscv-isa-sim/riscv/bulknormdot.h — 함수 (5개)

함수	반환형	받는 파라미터
int_log2	template<typename T> static int	T n
shift_right_jam	template<typename T> static T	T n, int amt
bulk_norm_dot_bf16	static inline bulk_norm_out_t	const DotConfig cfg, const bf16_t* a, const bf16_t* b
bulk_norm_dot_fp16	static inline bulk_norm_out_t	const DotConfig cfg, const fp16_norm_t* a, const fp16_norm_t* b
bulk_norm_dot_ofp8	template <typename L, typename R> bulk_norm_out_t	const DotConfig cfg, const L* a, const R* b

Table 1326. riscv-software-src_riscv-isa-sim/riscv/bulknormdot.h — #define / `define 매크로 (1개)

매크로	인자	정의
_RISCV_BULKNORMDOT_H		#include <cstdint>

API — cachesim.cc

Table 1327. riscv-software-src_riscv-isa-sim/riscv/cachesim.cc — 함수 (1개)

함수	반환형	받는 파라미터
help	static void	

API — cachesim.h

Table 1328. riscv-software-src_riscv-isa-sim/riscv/cachesim.h — 클래스/구조체 (6개)

이름	템플릿 인자	종류	상속
lfsr_t		class	
cache_sim_t		class	
fa_cache_sim_t		class	public cache_sim_t
cache_memtracer_t		class	public memtracer_t
icache_sim_t		class	public cache_memtracer_t
dcache_sim_t		class	public cache_memtracer_t

Table 1329. riscv-software-src_riscv-isa-sim/riscv/cachesim.h — #define / `define 매크로 (1개)

매크로	인자	정의
<code>_RISCV_CACHE_SIM_H</code>		<code>#include "memtracer.h"</code>

API — `cfg.h`

Table 1330. `riscv-software-src_riscv-isa-sim/riscv/cfg.h` — 클래스/구조체 (4개)

이름	템플릿 인자	종류	상속
<code>cfg_arg_t</code>	<code>typename T</code>	class	
<code>mem_cfg_t</code>		class	
<code>start_pc_t</code>		class	
<code>cfg_t</code>		class	

Table 1331. `riscv-software-src_riscv-isa-sim/riscv/cfg.h` — `#define` / ``define` 매크로 (1개)

매크로	인자	정의
<code>_RISCV_CFG_H</code>		<code>#include <optional></code>

API — `check-opcode-overlap.t.cc`

Table 1332. `riscv-software-src_riscv-isa-sim/riscv/check-opcode-overlap.t.cc` — 클래스/구조체 (1개)

이름	템플릿 인자	종류	상속
<code>opcode</code>		struct	

Table 1333. `riscv-software-src_riscv-isa-sim/riscv/check-opcode-overlap.t.cc` — 함수 (2개)

함수	반환형	받는 파라미터
<code>overlaps</code>	<code>static bool</code>	<code>const opcode& a, const opcode& b</code>
<code>main</code>	<code>int</code>	

Table 1334. `riscv-software-src_riscv-isa-sim/riscv/check-opcode-overlap.t.cc` — `#define` / ``define` 매크로 (4개)

매크로	인자	정의
<code>DECLARE_INSN</code>	<code>(name, match, mask)</code>	<code>\</code>
<code>DEFINE_INSN</code>	<code>(name)</code>	<code>\</code>
<code>DECLARE_OVERLAP_INSN</code>	<code>(name, ext)</code>	<code>\</code>
<code>DECLARE_OVERLAP_INSN_RV64</code>	<code>(name, ext)</code>	<code>\</code>

API — `clint.cc`

Table 1335. `riscv-software-src_riscv-isa-sim/riscv/clint.cc` — 함수 (2개)

함수	반환형	받는 파라미터
<code>clint_parse_from_fdt</code>	<code>clint_t*</code>	<code>const void* fdt, const sim_t* sim, reg_t* base, const std::vector<std::string>& sargs UN-USED</code>
<code>clint_generate_dts</code>	<code>std::string</code>	<code>const sim_t* sim, const std::vector<std::string>& sargs UNUSED</code>

Table 1336. riscv-software-src_riscv-isa-sim/riscv/clint.cc — #define / `define 매크로 (3개)

매크로	인자	정의
MSIP_BASE		0x0
MTIMECMP_BASE		0x4000
MTIME_BASE		0xbff8

API — common.h**Table 1337. riscv-software-src_riscv-isa-sim/riscv/common.h — #define / `define 매크로 (15개)**

매크로	인자	정의
_RISCV_COMMON_H		#ifdef __GNUC__
likely	(x)	__builtin_expect(x, 1)
unlikely	(x)	__builtin_expect(x, 0)
NOINLINE		__attribute__((noinline))
NORETURN		__attribute__((noreturn))
ALWAYS_INLINE		__attribute__((always_inline))
UNUSED		__attribute__((unused))
likely	(x)	(x)
unlikely	(x)	(x)
NOINLINE		# define NORETURN
ALWAYS_INLINE		# define UNUSED
__has_builtin	(x)	0
assume	(x)	[[assume(x)]]
assume	(x)	__builtin_assume(x)
assume	(x)	((void) 0)

API — csr_init.cc**Table 1338. riscv-software-src_riscv-isa-sim/riscv/csr_init.cc — #define / `define 매크로 (5개)**

매크로	인자	정의
add_const_ext_csr	(ext, addr, csr)	do { auto csr__ = (csr); if (proc->extension_enabled_const(ext)) { add_csr(addr, csr__); }...
add_ext_csr	(ext, addr, csr)	do { auto csr__ = (csr); if (proc->extension_enabled(ext)) { add_csr(addr, csr__); } } whi...
add_user_csr	(addr, csr)	add_const_ext_csr('U', addr, csr)
add_super-visor_csr	(addr, csr)	add_const_ext_csr('S', addr, csr)
add_hyper-visor_csr	(addr, csr)	add_ext_csr('H', addr, csr)

API — csrs.cc

Table 1339. riscv-software-src_riscv-isa-sim/riscv/csrs.cc — #define / #define 매크로 (1개)

매크로	인자	정의
STATE		(*state)

1. riscv-software-src_riscv-isa-sim/riscv/abstract_device.h (56 lines)

```

1 #ifndef _RISCV_ABSTRACT_DEVICE_H
2 #define _RISCV_ABSTRACT_DEVICE_H
3
4 #include "decode.h"
5 #include "common.h"
6 #include <stdint>
7 #include <cstdint>
8 #include <string>
9 #include <map>
10 #include <stdexcept>
11 #include <vector>
12
13 class sim_t;
14
15 class abstract_device_t {
16 public:
17     virtual bool load(reg_t addr, size_t len, uint8_t* bytes) =
0;
18     virtual bool store(reg_t addr, size_t len, const uint8_t*
bytes) = 0;
19     virtual reg_t size() = 0;
20     virtual ~abstract_device_t() {}
21     virtual void tick(reg_t UNUSED rtc_ticks) {}
22 };
23
24 // factory for devices which should show up in the DTS, and
can be
25 // parameterized by parsing the DTS
26 class device_factory_t {
27 public:
28     virtual abstract_device_t* parse_from_fdt(const void* fdt,
const sim_t* sim, reg_t* base, const std::vector<std::string>&
sargs) const = 0;
29     virtual std::string generate_dts(const sim_t* sim, const
std::vector<std::string>& sargs) const = 0;
30     virtual ~device_factory_t() {}
31 };
32
33 // Type for holding all registered MMIO plugins by name.
34 using mmio_device_map_t = std::map<std::string, const
device_factory_t*>;
35
36 mmio_device_map_t& mmio_device_map();
37
38 #define REGISTER_DEVICE(name, parse, generate) \
39     class name##_factory_t : public device_factory_t { \
40     public: \
41         name##_factory_t() { \
42             std::string str(#name); \
43             if (!mmio_device_map().emplace(str, this).second) throw
std::runtime_error("Plugin \"" + str + "\" already registered"); \
44         }; \
45         name##_t* parse_from_fdt(const void* fdt, const sim_t* sim,
reg_t* base, const std::vector<std::string>& sargs) const override
{ return parse(fdt, sim, base, sargs); } \
46         std::string generate_dts(const sim_t* sim, const std::vec-
tor<std::string>& sargs) const override { return generate(sim,
sargs); } \
47     }; device_factory_t *name##_factory = new name##_fact-
ory_t();
48
49 #define REGISTER_BUILTIN_DEVICE(name, parse, generate) \
50     class name##_factory_t : public device_factory_t { \
51     public: \
52         name##_t* parse_from_fdt(const void* fdt, const sim_t* sim,
reg_t* base, const std::vector<std::string>& sargs) const override
{ return parse(fdt, sim, base, sargs); } \
53         std::string generate_dts(const sim_t* sim, const std::vec-
tor<std::string>& sargs) const override { return generate(sim,
sargs); } \
54     }; device_factory_t *name##_factory = new name##_fact-
ory_t();
55
56 #endif

```

2. riscv-software-src_riscv-isa-sim/riscv/abstract_interrupt_controller.h (14 lines)

```

1 #ifndef _RISCV_ABSTRACT_INTERRUPT_CONTROLLER_H
2 #define _RISCV_ABSTRACT_INTERRUPT_CONTROLLER_H
3
4 #include "decode.h"

```

```

5 #include <cstdint>
6 #include <cstdint>
7
8 class abstract_interrupt_controller_t {
9 public:
10     virtual void set_interrupt_level(uint32_t interrupt_id, int
level) = 0;
11     virtual ~abstract_interrupt_controller_t() {}
12 };
13
14 #endif

```

3. riscv-software-src_riscv-isa-sim/riscv/aplic.cc (327 lines)

```

1 // See LICENSE for license details.
2 #include "devices.h"
3 #include "processor.h"
4 #include "arith.h"
5 #include "sim.h"
6 #include "dts.h"
7 #include <cassert>
8
9 #define DOMAINCFG 0x0
10 #define SOURCECFG_BASE 0x4
11 #define MMSIADDRCFG 0x1bc0
12 #define MMSIADDRCFGH 0x1bc4
13 #define SMSIADDRCFG 0x1bc8
14 #define SMSIADDRCFGH 0x1bcc
15 #define SETIP_BASE 0x1c00
16 #define SETIPNUM 0x1cdc
17 #define IN_CLRIP_BASE 0x1d00
18 #define CLRIPIUM 0x1ddc
19 #define SETIE_BASE 0x1e00
20 #define SETIENUM 0x1edc
21 #define CLRIE_BASE 0x1f00
22 #define CLRIENUM 0x1fdc
23 #define GENMSI 0x3000
24 #define TARGET_BASE 0x3004
25 #define IDC 0x4000
26
27 #define DOMAINCFG_IE_MASK 0x100
28 #define DOMAINCFG_DM_MASK 0x4
29
30 #define SOURCECFG_D_MASK 0x400
31 #define SOURCECFG_CHILD_MASK 0x3ff
32 #define SOURCECFG_SM_MASK 0x7
33
34 #define MMSIADDRCFGH_L_MASK 0x80000000
35
36 #define GENMSI_HART_MASK 0xffff0000
37 #define GENMSI_BUSY_MASK 0x1000
38 #define GENMSI_EIID_MASK 0x7ff
39
40 #define TARGET_HART_MASK 0xffff0000
41 #define TARGET_GUEST_MASK 0x3f000
42 #define TARGET_EIID_MASK 0x7ff
43
44 #define APLIC_MMIO_REG_SIZE 4
45
46 aplic_t::aplic_t(const simif_t *simif, aplic_t *parent) :
simif(simif), parent(parent), child(nullptr), domaincfg((0x80 <= 24)
| DOMAINCFG_DM_MASK), sourcecfg(), mmsiad-
drcfgh(MMSIADDRCFGH_L_MASK), ie(), target(), level(), genmsi(0),
deleg_mask()
47 {
48 }
49
50 void aplic_t::delegate(uint32_t id, bool dm)
51 {
52     if (!id || id >= APLIC_MAX_DEVICES)
53         return;
54
55     if (!child) {
56         // child's sourcecfg = 0 when interrupt is changed to del-
egated
57         if (!parent->delegated(id) && dm)
58             sourcecfg[id] = 0;
59     } else {
60         deleg_mask[id / 32] = set_field((uint32_t)deleg_mask[id /
32], 1 << (id % 32), dm);
61         child->delegate(id, dm);
62     }
63 }
64
65 bool aplic_t::delegated(uint32_t id)
66 {
67     if (!id || id >= APLIC_MAX_DEVICES)
68         return false;

```

```

69 return (sourcecfg[id] & SOURCECFG_D_MASK);
70 }
71
72 uint32_t aplic_t::get_deleg_mask(uint32_t idx) {
73     if (idx >= APLIC_MAX_DEVICES / 32)
74         return 0;
75     if (parent)
76         return parent->get_deleg_mask(idx);
77     return deleg_mask[idx];
78 }
79
80 bool aplic_t::interrupt_enabled(uint32_t id) const
81 {
82     if (!id || id >= APLIC_MAX_DEVICES)
83         return false;
84     if (parent && !parent->delegated(id))
85         return false;
86     // Domain interrupt enabled, source IE and source mode !=
inactive (0)
87     return (domaincfg & DOMAINCFG_IE_MASK) && ((ie[id / 32] >>
(id % 32)) & 1) && (sourcecfg[id] & SOURCECFG_SM_MASK);
88 }
89
90 bool aplic_t::accessible(uint32_t id) const
91 {
92     if (!id || id >= APLIC_MAX_DEVICES)
93         return false;
94     if (!parent)
95         return true;
96     return parent->delegated(id);
97 }
98
99 void aplic_t::set_interrupt_level(uint32_t id, int lvl)
100 {
101     if (child && delegated(id)) {
102         child->set_interrupt_level(id, lvl);
103         return;
104     }
105
106     if (!accessible(id))
107         return;
108
109     level[id / 32] = set_field((uint32_t)level[id / 32], 1 <<
(id % 32), lvl);
110
111     update_interrupt(id);
112 }
113
114 void aplic_t::send_msi(uint32_t proc_id, uint32_t guest,
uint32_t eiid)
115 {
116     auto &procs = simif->get_harts();
117     auto it = procs.find(proc_id);
118
119     assert((eiid != 0) && (it != procs.end()));
120
121     processor_t *proc = it->second;
122     if (!parent) {
123         proc->imsic->m->pendei(eiid);
124     } else if (guest) {
125         if (proc->imsic->vs.count(guest))
126             proc->imsic->vs[guest]->pendei(eiid);
127     } else {
128         proc->imsic->s->pendei(eiid);
129     }
130 }
131
132 void aplic_t::send_msi(uint32_t id)
133 {
134     if (id >= APLIC_MAX_DEVICES)
135         return;
136     auto tgt = target[id];
137     send_msi(get_field(tgt, TARGET_HART_MASK), get_field(tgt,
TARGET_GUEST_MASK), get_field(tgt, TARGET_EIID_MASK));
138 }
139
140 void aplic_t::update_interrupt(uint32_t id)
141 {
142     // check level and if interrupt enabled, send MSI if neces-
sary
143     if (id >= APLIC_MAX_DEVICES)
144         return;
145     bool lvl = (level[id / 32] >> (id % 32)) & 1;
146     if (lvl && interrupt_enabled(id))
147         send_msi(id);
148 }
149
150 void aplic_t::update_interrupt_masked(uint32_t idx, uint32_t
mask)
151 {
152     if (idx >= APLIC_MAX_DEVICES / 32)
153         return;
154     while(mask) {
155         auto id = ctz(mask);
156         update_interrupt(idx * 32 + id);
157         mask &= ~(1 << id);
158     }
159 }
160
161 bool aplic_t::load(reg_t addr, size_t len, uint8_t* bytes)
162 {

```

```

163     uint32_t val = 0;
164
165     if (len != 4) {
166         return false;
167     }
168
169     addr &= ~reg_t(3);
170
171     if (addr == DOMAINCFG) {
172         val = domaincfg;
173     } else if (addr >= SOURCECFG_BASE && addr < MMSIADDRCFG) {
174
175         auto idx = (addr - SOURCECFG_BASE) / APLIC_MMIO_REG_SIZE +
1;
176         if (accessible(idx))
177             val = sourcecfg[idx];
178     } else if (addr == MMSIADDRCFGH) {
179         val = mmsiaddrcfgh;
180     } else if (addr >= IN_CLRIP_BASE && addr < CLRIPNUM) {
181         auto idx = (addr - IN_CLRIP_BASE) / APLIC_MMIO_REG_SIZE;
182         val = level[idx];
183     } else if (addr >= SETIE_BASE && addr < SETIENUM) {
184         auto idx = (addr - SETIE_BASE) / APLIC_MMIO_REG_SIZE;
185         val = ie[idx];
186     } else if (addr == GENMSI) {
187         val = genmsi;
188     } else if (addr >= TARGET_BASE && addr < IDC) {
189         auto idx = (addr - TARGET_BASE) / APLIC_MMIO_REG_SIZE + 1;
190         if (accessible(idx))
191             val = target[idx];
192     }
193
194     memcpy(bytes, &val, len);
195
196     return true;
197 }
198
199 bool aplic_t::store(reg_t addr, size_t len, const uint8_t*
bytes)
200 {
201     uint32_t val = 0;
202
203     if (len != 4) {
204         return false;
205     }
206
207     addr &= ~reg_t(3);
208     memcpy(&val, bytes, len);
209
210     if (addr == DOMAINCFG) {
211         domaincfg &= ~DOMAINCFG_IE_MASK;
212         domaincfg |= val & DOMAINCFG_IE_MASK;
213     } else if (addr >= SOURCECFG_BASE && addr < MMSIADDRCFG) {
214
215         auto idx = (addr - SOURCECFG_BASE) / APLIC_MMIO_REG_SIZE +
1;
216         // write to D bit sets child's sourcecfg to 0
217         // SM = inactive (0) or Level1 (6)
218         if (accessible(idx)) {
219             if (!child && (val & SOURCECFG_D_MASK))
220                 val = 0;
221             else if (child)
222                 delegate(idx, val & SOURCECFG_D_MASK);
223
224             // force SM to be 6 if non-zero
225             if (val & SOURCECFG_SM_MASK)
226                 val = set_field(val, SOURCECFG_SM_MASK, 6);
227
228             sourcecfg[idx] = val;
229         }
230     } else if (addr >= SETIP_BASE && addr < SETIPNUM) {
231         auto idx = (addr - SETIP_BASE) / APLIC_MMIO_REG_SIZE;
232         val &= get_deleg_mask(idx);
233         update_interrupt_masked(idx, val);
234     } else if (addr == SETIPNUM) {
235         update_interrupt(val);
236     } else if (addr >= SETIE_BASE && addr < SETIENUM) {
237         auto idx = (addr - SETIE_BASE) / APLIC_MMIO_REG_SIZE;
238         val &= get_deleg_mask(idx);
239         ie[idx] |= val;
240     } else if (addr == SETIENUM) {
241         if (accessible(val))
242             ie[val / 32] |= 1 << (val % 32);
243     } else if (addr >= CLRIE_BASE && addr < CLRIENUM) {
244         auto idx = (addr - CLRIE_BASE) / APLIC_MMIO_REG_SIZE;
245         val &= get_deleg_mask(idx);
246         ie[idx] &= ~val;
247     } else if (addr == CLRIENUM) {
248         if (accessible(val))
249             ie[val / 32] &= ~(1 << (val % 32));
250     } else if (addr == GENMSI) {
251         genmsi = val & (GENMSI_HART_MASK | GENMSI_EIID_MASK);
252         send_msi(get_field(genmsi, GENMSI_HART_MASK), 0,
get_field(genmsi, GENMSI_EIID_MASK));
253     } else if (addr >= TARGET_BASE && addr < IDC) {
254         auto idx = (addr - TARGET_BASE) / APLIC_MMIO_REG_SIZE + 1;
255         if (accessible(val))
256             target[idx] = val;
257     }
258
259     return true;

```

```

258 }
259
260 std::string aplic_m_generate_dts(const sim_t* sim, const
std::vector<std::string>& sargs UNUSED)
261 {
262     auto cfg = sim->get_cfg();
263     isa_parser_t isa(cfg.isa, cfg.priv);
264     std::stringstream s;
265     if (isa.extension_enabled(EXT_SMAIA)) {
266         s << std::hex
267         << "    APLIC_M: aplic@" << APLIC_M_BASE << " {\n"
268         << "        riscv,delegate = <&APLIC_S 0x01 0x35>;\n"
269         << "        riscv,children = <&APLIC_S>;\n"
270         << "        riscv,num-sources = <0x35>;\n"
271         << "        reg = <0x00 0x" << APLIC_M_BASE << " 0x00 0x"
<< APLIC_SIZE << ">;\n"
272         << "        msi-parent = <&IMISC_M>;\n"
273         << "        interrupt-controller;\n"
274         << "        #interrupt-cells = <0x02>;\n"
275         << "        compatible = \"riscv,aplic\";\n"
276         << "    };\n";
277     }
278     return s.str();
279 }
280
281 std::string aplic_s_generate_dts(const sim_t* sim, const
std::vector<std::string>& sargs UNUSED)
282 {
283     auto cfg = sim->get_cfg();
284     isa_parser_t isa(cfg.isa, cfg.priv);
285     std::stringstream s;
286     if (isa.extension_enabled(EXT_SSAIA)) {
287         s << std::hex
288         << "    APLIC_S: aplic@" << APLIC_S_BASE << " {\n"
289         << "        riscv,num-sources = <0x35>;\n"
290         << "        reg = <0x00 0x" << APLIC_S_BASE << " 0x00 0x"
<< APLIC_SIZE << ">;\n"
291         << "        msi-parent = <&IMISC_S>;\n"
292         << "        interrupt-controller;\n"
293         << "        #interrupt-cells = <0x02>;\n"
294         << "        compatible = \"riscv,aplic\";\n"
295         << "    };\n";
296     }
297     return s.str();
298 }
299
300 aplic_m_t* aplic_m_parse_from_fdt(const void* fdt, const
sim_t* sim, reg_t* base, const std::vector<std::string>& sargs UN-
USED)
301 {
302     reg_t aplic_m_base, aplic_s_base;
303     if (fdt_parse_aplic(fdt, &aplic_m_base, &aplic_s_base,
"riscv,aplic") == 0) {
304         if (aplic_m_base) {
305             *base = aplic_m_base;
306             return new aplic_m_t(sim, nullptr);
307         }
308     }
309     return nullptr;
310 }
311
312 aplic_s_t* aplic_s_parse_from_fdt(const void* fdt, const
sim_t* sim, reg_t* base, const std::vector<std::string>& sargs UN-
USED)
313 {
314     reg_t aplic_m_base, aplic_s_base;
315     if (fdt_parse_aplic(fdt, &aplic_m_base, &aplic_s_base,
"riscv,aplic") == 0) {
316         if (aplic_s_base) {
317             *base = aplic_s_base;
318             auto *ptr = new aplic_s_t(sim, sim->aplic_m.get());
319             if (sim->aplic_m)
320                 sim->aplic_m->set_child(ptr);
321             return ptr;
322         }
323     }
324     return nullptr;
325 }
326 REGISTER_DEVICE(aplic_m, aplic_m_parse_from_fdt, ap-
lic_m_generate_dts)
327 REGISTER_DEVICE(aplic_s, aplic_s_parse_from_fdt, ap-
lic_s_generate_dts)

```

4. riscv-software-src_riscv-isa-sim/riscv/arith.h (254 lines)

```

1 // See LICENSE for license details.
2
3 #ifndef _RISC_V_ARITH_H
4 #define _RISC_V_ARITH_H
5
6 #include <cassert>
7 #include <cstdint>
8 #include <climits>
9 #include <cstdint>
10 #include <type_traits>
11
12 inline uint64_t mulhu(uint64_t a, uint64_t b)
13 {
14     uint64_t t;
15     uint32_t y1, y2, y3;

```

```

16     uint64_t a0 = (uint32_t)a, a1 = a >> 32;
17     uint64_t b0 = (uint32_t)b, b1 = b >> 32;
18
19     t = a1*b0 + ((a0*b0) >> 32);
20     y1 = t;
21     y2 = t >> 32;
22
23     t = a0*b1 + y1;
24
25     t = a1*b1 + y2 + (t >> 32);
26     y2 = t;
27     y3 = t >> 32;
28
29     return ((uint64_t)y3 << 32) | y2;
30 }
31
32 inline int64_t mulh(int64_t a, int64_t b)
33 {
34     int negate = (a < 0) != (b < 0);
35     uint64_t res = mulhu(a < 0 ? -(uint64_t)a : a, b < 0 ? -
(uint64_t)b : b);
36     return negate ? ~res + ((uint64_t)a * (uint64_t)b == 0) :
res;
37 }
38
39 inline int64_t mulhsu(int64_t a, uint64_t b)
40 {
41     int negate = a < 0;
42     uint64_t res = mulhu(a < 0 ? -(uint64_t)a : a, b);
43     return negate ? ~res + ((uint64_t)a * b == 0) : res;
44 }
45
46 //ref: https://locklessinc.com/articles/sat_arithmetic/
47 template<typename T, typename UT>
48 static inline T sat_add(T x, T y, bool &sat)
49 {
50     UT ux = x;
51     UT uy = y;
52     UT res = ux + uy;
53     sat = false;
54     int sh = sizeof(T) * 8 - 1;
55
56     /* Calculate overflowed result. (Don't change the sign bit
of ux) */
57     ux = (ux >> sh) + (((UT)0x1 << sh) - 1);
58
59     /* Force compiler to use cmovns instruction */
60     if ((T)((ux ^ uy) | ~(uy ^ res)) >= 0) {
61         res = ux;
62         sat = true;
63     }
64
65     return res;
66 }
67
68 template<typename T, typename UT>
69 static inline T sat_add(T x, T y, T z, bool &sat)
70 {
71     bool sat1, sat2;
72     T a = y;
73     T b = z;
74     T res;
75
76     /* Force compiler to use cmovns instruction */
77     if (((y ^ z) & (x ^ z)) < 0) {
78         a = z;
79         b = y;
80     }
81
82     res = sat_add<T, UT>(x, a, sat1);
83     res = sat_add<T, UT>(res, b, sat2);
84     sat = sat1 || sat2;
85
86     return res;
87 }
88
89 template<typename T, typename UT>
90 static inline T sat_sub(T x, T y, bool &sat)
91 {
92     UT ux = x;
93     UT uy = y;
94     UT res = ux - uy;
95     sat = false;
96     int sh = sizeof(T) * 8 - 1;
97
98     /* Calculate overflowed result. (Don't change the sign bit
of ux) */
99     ux = (ux >> sh) + (((UT)0x1 << sh) - 1);
100
101     /* Force compiler to use cmovns instruction */
102     if ((T)((ux ^ uy) & (ux ^ res)) < 0) {
103         res = ux;
104         sat = true;
105     }
106
107     return res;
108 }
109
110 template<typename T>
111 T sat_addu(T x, T y, bool &sat)
112 {

```

```

113 T res = x + y;
114 sat = false;
115
116 sat = res < x;
117 res |= -(res < x);
118
119 return res;
120 }
121
122 template<typename T>
123 T sat_subu(T x, T y, bool &sat)
124 {
125     T res = x - y;
126     sat = false;
127
128     sat = !(res <= x);
129     res &= -(res <= x);
130
131     return res;
132 }
133
134 static inline uint64_t extract64(uint64_t val, int pos, int
len)
135 {
136     assert(pos >= 0 && len > 0 && len <= 64 - pos);
137     return (val >> pos) & (~UINT64_C(0) >> (64 - len));
138 }
139
140 static inline uint64_t make_mask64(int pos, int len)
141 {
142     assert(pos >= 0 && len > 0 && pos < 64 && len <= 64);
143     return (UINT64_MAX >> (64 - len)) << pos;
144 }
145
146 static inline int popcount(uint64_t val)
147 {
148     val = (val & 0x5555555555555555U) + ((val >> 1) &
0x5555555555555555U);
149     val = (val & 0x3333333333333333U) + ((val >> 2) &
0x3333333333333333U);
150     val = (val & 0x0f0f0f0f0f0f0f0fU) + ((val >> 4) &
0x0f0f0f0f0f0f0f0fU);
151     val = (val & 0x00ff00ff00ff00ffU) + ((val >> 8) &
0x00ff00ff00ff00ffU);
152     val = (val & 0x0000ffff0000ffffU) + ((val >> 16) &
0x0000ffff0000ffffU);
153     val = (val & 0x00000000ffffffffffU) + ((val >> 32) &
0x00000000ffffffffffU);
154     return val;
155 }
156
157 static inline int ctz(uint64_t val)
158 {
159     if (!val)
160         return 0;
161
162     int res = 0;
163
164     if ((val << 32) == 0) res += 32, val >>= 32;
165     if ((val << 48) == 0) res += 16, val >>= 16;
166     if ((val << 56) == 0) res += 8, val >>= 8;
167     if ((val << 60) == 0) res += 4, val >>= 4;
168     if ((val << 62) == 0) res += 2, val >>= 2;
169     if ((val << 63) == 0) res += 1, val >>= 1;
170
171     return res;
172 }
173
174 static inline int clz(uint64_t val)
175 {
176     if (!val)
177         return 0;
178
179     int res = 0;
180
181     if ((val >> 32) == 0) res += 32, val <<= 32;
182     if ((val >> 48) == 0) res += 16, val <<= 16;
183     if ((val >> 56) == 0) res += 8, val <<= 8;
184     if ((val >> 60) == 0) res += 4, val <<= 4;
185     if ((val >> 62) == 0) res += 2, val <<= 2;
186     if ((val >> 63) == 0) res += 1, val <<= 1;
187
188     return res;
189 }
190
191 // Count number of contiguous 1 bits starting from the LSB.
192 static inline int cto(uint64_t val)
193 {
194     int res = 0;
195     while ((val & 1) == 1)
196         val >>= 1, res++;
197     return res;
198 }
199
200 static inline int log2(uint64_t val)
201 {
202     if (!val)
203         return 0;
204
205     return 63 - clz(val);
206 }

```

```

207
208 static inline uint64_t xperm(uint64_t rs1, uint64_t rs2,
size_t sz_log2, size_t len)
209 {
210     uint64_t r = 0;
211     uint64_t sz = 1LL << sz_log2;
212     uint64_t mask = (1LL << sz) - 1;
213
214     assert(sz_log2 <= 6 && len <= 64);
215
216     for (size_t i = 0; i < len; i += sz) {
217         uint64_t pos = ((rs2 >> i) & mask) << sz_log2;
218         if (pos < len)
219             r |= ((rs1 >> pos) & mask) << i;
220     }
221
222     return r;
223 }
224
225 // Rotates right an unsigned integer by the given number of
bits.
226 template <typename T>
227 static inline T rotate_right(T x, std::size_t shiftamt) {
228     static_assert(std::is_unsigned<T>::value);
229     static constexpr T mask = (8 * sizeof(T)) - 1;
230     const std::size_t rshift = shiftamt & mask;
231     const std::size_t lshift = (-rshift) & mask;
232     return (x << lshift) | (x >> rshift);
233 }
234
235 // Rotates right an unsigned integer by the given number of
bits.
236 template <typename T>
237 static inline T rotate_left(T x, std::size_t shiftamt) {
238     static_assert(std::is_unsigned<T>::value);
239     static constexpr T mask = (8 * sizeof(T)) - 1;
240     const std::size_t lshift = shiftamt & mask;
241     const std::size_t rshift = (-lshift) & mask;
242     return (x << lshift) | (x >> rshift);
243 }
244
245 template<typename out_t, typename in1_t, typename in2_t>
246 static inline out_t dot_product(const in1_t* a, const in2_t*
b, size_t n)
247 {
248     out_t res = 0;
249     for (size_t i = 0; i < n; i++)
250         res += (out_t)a[i] * (out_t)b[i];
251     return res;
252 }
253
254 #endif

```

5. riscv-software-src_riscv-isa-sim/riscv/bloom_filter.h (64

lines)

```

1 // See LICENSE for license details.
2
3 #ifndef _RISC_V_BLOOM_FILTER_H
4 #define _RISC_V_BLOOM_FILTER_H
5
6 #include <bitset>
7 #include <cstdint>
8
9 struct simple_hash1 {
10     uint64_t operator()(uint64_t x) const
11     {
12         x = (x ^ (x >> 33)) * 0xff51afd7ed558ccd;
13         x = (x ^ (x >> 33)) * 0xc4ceb9fe1a85ec53;
14         return x ^ (x >> 33);
15     }
16 };
17
18 struct simple_hash2 {
19     uint64_t operator()(uint64_t x) const
20     {
21         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
22         x = (x ^ (x >> 27)) * 0x94d049b13c66a8ed;
23         return x ^ (x >> 31);
24     }
25 };
26
27 template <typename T, typename H1, typename H2, size_t M,
size_t K> // M: bit array size, K: number of hash functions
28 class bloom_filter_t {
29 public:
30     void clear()
31     {
32         bits.reset();
33     }
34
35     void insert(T value)
36     {
37         uint64_t h1 = H1()(value);
38         uint64_t h2 = H2()(value);
39
40         for (size_t i = 0; i < K; i++) {
41             size_t idx = (h1 + i * h2) % M;
42             bits[idx] = true;
43         }

```

```

44 }
45
46 bool contains(T value) const
47 {
48     uint64_t h1 = H1()(value);
49     uint64_t h2 = H2()(value);
50
51     for (size_t i = 0; i < K; i++) {
52         size_t idx = (h1 + i * h2) % M;
53         if (!bits[idx])
54             return false;
55     }
56     return true;
57 }
58
59 private:
60 std::bitset<M> bits;
61 };
62
63 #endif

```

6. riscv-software-src_riscv-isa-sim/riscv/bulknormdot.h (362 lines)

```

1 #ifndef _RISCV_BULKNORMDOT_H
2 #define _RISCV_BULKNORMDOT_H
3
4 #include <stdint>
5 #include <vector>
6 #include "softfloat.h"
7
8 struct bulk_norm_out_t {
9     uint32_t out;
10     uint8_t flags;
11 };
12
13 template<typename T>
14 static int int_log2(T n)
15 {
16     int res = 0;
17     while (n >= 1)
18         res++;
19     return res;
20 }
21
22 template<typename T>
23 static T shift_right_jam(T n, int amt)
24 {
25     int width = 8 * sizeof(T);
26     T shifted = amt >= width ? 0 : n >> amt;
27     T jam_mask = amt >= width ? T(-1) : (T(1) << amt) - 1;
28     bool jam = (n & jam_mask) != 0;
29     return shifted | jam;
30 }
31
32 /** Configuration description for dot product */
33 class DotConfig {
34 public:
35     int n; // number of products
36     int guardBits; // number of guard bits
37     bool flushSub; // flush subnormal (input/output) to zero
38     DotConfig(int numProd, int numGuardBits) : n(numProd),
guardBits(numGuardBits), flushSub(false) {}
39 };
40
41 const static int f32_exp_bits = 8;
42 const static int f32_exp_bias = (1 << (f32_exp_bits - 1)) - 1;
43 const static int f32_mant_bits = 23; // number of mantissa
bits (excluding implicit one)
44 const static int f32_exp_mask = (uint32_t(1) << f32_exp_bits)
- 1;
45 const static uint32_t f32_mant_mask = (uint32_t(1) <<
f32_mant_bits) - 1;
46
47 /** Template for a floating-point format class */
48 template <typename U, typename M, typename E> class Float-
Format {
49     virtual M mant() const = 0;
50     virtual M sig() const = 0;
51     virtual E exp() const = 0;
52
53     virtual bool subOrZero() const = 0;
54
55     virtual bool inf() const = 0;
56     virtual bool nan() const = 0;
57     virtual bool sigNan() const = 0;
58     virtual bool special() const = 0;
59
60 public:
61     virtual ~FloatFormat() = default;
62 };
63
64 /** Template for an IEEE-754 floating-point format class */
65 template <typename U, typename M, typename E, unsigned exp-
Width, unsigned mantWidth> class IEEEFloatFormat : FloatFormat<U, M,
E> {
66 public:
67     U n;
68     IEEEFloatFormat(U _n) : n(_n) {}

```

```

69 IEEEFloatFormat() {}
70
71 int bias = (1 << (expWidth - 1)) - 1;
72 int sigBits = mantWidth + 1;
73 int mant_bits = mantWidth;
74 public:
75     /** raw exponent field */
76     E exp() const override { return (n >> mantWidth) & ((1 <<
expWidth) - 1); }
77
78     /** raw exponent field with correction for subnormal */
79     E expSubFixed() const { return exp() + subOrZero(); }
80
81     /** number sign */
82     bool sign() const { return n >> (expWidth + mantWidth); }
83
84     /** bit mask for mantissa */
85     M mantMask() const { return (1 << mantWidth) - 1; }
86
87     /** Number mantissa */
88     M mant() const override { return n & mantMask(); }
89
90     /** Number significand */
91     M sig() const override { return mant() ^ (!subOrZero() <<
mantWidth); }
92
93     /** bit mask for exponent */
94     E expMask() const { return (1 << expWidth) - 1; }
95
96     /** predicate: is the value a subnormal number or a zero */
97     bool subOrZero() const override { return exp() == 0; }
98
99     /** predicate: is the value a special value (infinity or
NaN) */
100     bool special() const override { return exp() == expMask(); }
101
102     /** predicate: is the value an infinity */
103     bool inf() const override { return special() && mant() ==
0; }
104
105     /** predicate: is the value a NaN (Not A Number) */
106     bool nan() const override { return special() && mant() !=
0; }
107
108     bool sigNan() const override { return nan() && !inf() &&
( ( mant() >> (mantWidth - 1)) == 0); }
109
110     bool isZero() const { return exp() == 0 && mant() == 0; }
111 };
112
113 class bf16_t final : public IEEEFloatFormat<uint16_t, uint8_t,
uint8_t, 8, 7> {
114 public:
115     operator uint16_t() const { return n; }
116
117     bf16_t() : IEEEFloatFormat(0) {}
118     bf16_t(uint16_t _n) : IEEEFloatFormat(_n) {}
119
120     bf16_t flushed() const
121     {
122         if (exp() == 0)
123             return bf16_t(uint16_t(sign() << 15));
124         return *this;
125     }
126 };
127
128 class fp16_norm_t : public IEEEFloatFormat<uint16_t, uint16_t,
uint8_t, 5, 10> {
129 public:
130     operator uint16_t() const { return n; }
131
132     fp16_norm_t() : IEEEFloatFormat(0) {}
133     fp16_norm_t(uint16_t _n) : IEEEFloatFormat(_n) {}
134
135     fp16_norm_t flushed() const
136     {
137         if (exp() == 0)
138             return fp16_norm_t(uint16_t(sign() << 15));
139         return *this;
140     }
141 };
142
143 /** OpenCompute 8-bit Floating-point E5M2 (5-bit exponent, 2-
bit mantissa) */
144 class ofp8_e5m2 final : public IEEEFloatFormat<uint8_t,
uint8_t, uint8_t, 5, 2> {
145 public:
146     operator uint8_t() const { return n; }
147     ofp8_e5m2() : IEEEFloatFormat(0) {}
148     ofp8_e5m2(uint8_t _n) : IEEEFloatFormat(_n) {}
149
150     // OFP8 does not have signaling NaNs
151     bool sigNan() const override { return false; }
152
153     ofp8_e5m2 flushed() const
154     {
155         if (exp() == 0)
156             return ofp8_e5m2(uint8_t(sign() << 7));
157         return *this;

```



```

158 }
159 };
160
161 /** OpenCompute 8-bit Floating-point E4M3 (4-bit exponent, 3-
bit mantissa) */
162 class ofp8_e4m3 final : public IEEEFloatFormat<uint8_t,
uint8_t, uint8_t, 4, 3> {
163 public:
164 operator uint8_t() const { return n; }
165 ofp8_e4m3() : IEEEFloatFormat(0) {}
166 ofp8_e4m3(uint8_t _n) : IEEEFloatFormat(_n) {}
167
168 // E4M3 does not have infinities
169 bool inf() const override { return false; }
170
171 bool nan() const override { return exp() == expMask() &&
mant() == mantMask(); }
172
173 bool special() const override { return nan(); }
174
175 // OFP8 does not have signaling NaNs
176 bool sigNaN() const override { return false; }
177
178 ofp8_e4m3 flushed() const
179 {
180 if (exp() == 0)
181 return ofp8_e4m3(uint8_t(sign() << 7));
182 return *this;
183 }
184 };
185
186 /** bulk-normalization dot product (without accumulation) with
binary32 result
187 *
188 * The actual products of significands is provided as an argu-
ment such that the model can be used
189 * to match against RTL implementations with external product
implementation.
190 *
191 * @param cfg dot-product configuration
192 * @param a left-hand-side operand array
193 * @param b right-hand-side operand array
194 * @param prod_signs array of products of significands
195 *
196 */
197 template<typename ValueTypeLHS, typename ValueTypeRHS, type-
name SigProdType> bulk_norm_out_t bulk_norm_dot_no_mult(const Dot-
Config cfg, const ValueTypeLHS* a, const ValueTypeRHS* b, const Sig-
ProdType* prod_signs)
198 {
199 std::vector<int> approx_prod_exp(cfg.n);
200 std::vector<int> flushed_prods(cfg.n);
201
202 bool any_pos_inf = false;
203 bool any_neg_inf = false;
204 bool any_nan = false;
205 bool any_invalid_nan = false;
206 bool any_sigNaN = false;
207
208 // extracting format parameters from the first element in
each input arrays
209 int lhs_bias = a[0].bias;
210 int rhs_bias = b[0].bias;
211
212 int lhs_mant_bits = a[0].mant_bits;
213 int rhs_mant_bits = b[0].mant_bits;
214
215 for (int i = 0; i < cfg.n; i++) {
216 flushed_prods[i] = (cfg.flushSub && (a[i].subOrZero() ||
b[i].subOrZero()));
217
218 approx_prod_exp[i] = flushed_prods[i] ? 0 : // flush input
subnormals
219 a[i].isZero() || b[i].isZero() ?
(f32_exp_bias - (lhs_bias + rhs_bias)) : // minimize exp of zero
product
220 a[i].expSubFixed() + b[i].expSub-
Fixed() + (f32_exp_bias - (lhs_bias + rhs_bias));
221
222 bool a_is_zero = (a[i].subOrZero() && cfg.flushSub) ||
a[i].isZero();
223 bool b_is_zero = (b[i].subOrZero() && cfg.flushSub) ||
b[i].isZero();
224
225 bool either_inf = a[i].inf() || b[i].inf();
226 bool either_nan = a[i].nan() || b[i].nan();
227 bool either_zero = a_is_zero || b_is_zero;
228 any_pos_inf |= either_inf && !either_nan && !either_zero
&& a[i].sign() == b[i].sign();
229 any_neg_inf |= either_inf && !either_nan && !either_zero
&& a[i].sign() != b[i].sign();
230
231 any_invalid_nan |=
232 (a[i].inf() && b_is_zero) ||
233 (b[i].inf() && a_is_zero);
234
235 any_nan |= any_invalid_nan || a[i].nan() || b[i].nan();
236
237 any_sigNaN |= a[i].sigNaN() || b[i].sigNaN();
238 }
239
240 // find largest exponent
241 int max_approx_prod_exp = approx_prod_exp[0];
242 for (int i = 1; i < cfg.n; i++) {
243 max_approx_prod_exp = std::max(max_approx_prod_exp, ap-
prox_prod_exp[i]);
244 }
245
246 bool acc_sign = false; // assuming the accumulator is posit-
ive
247
248 int64_t acc = 0;
249
250 // compute products, normalize to largest exponent, accumu-
late
251 for (int i = 0; i < cfg.n; i++) {
252 int prod_sign = a[i].sign() ^ b[i].sign();
253 uint64_t prod_sig = uint64_t(prod_signs[i]); // 16 to 64-
bit zero extension
254 // align the product so the width of its fractional part
is: f32_mant_bits(23) + guardBits
255 prod_sig <<= f32_mant_bits - lhs_mant_bits - rhs_mant_bits +
cfg.guardBits;
256
257 int shiftAmt = max_approx_prod_exp - approx_prod_exp[i];
258 uint64_t shifted_sig = shift_right_jam(prod_sig,
shiftAmt);
259 acc += flushed_prods[i] ? 0 : // flush input subnormals
(prod_sign != acc_sign ? -shifted_sig : shifted_sig);
260 }
261
262 // normalize result to f32
263 bool sign = (acc < 0) != acc_sign;
264 uint64_t mag = acc < 0 ? -acc : acc; // absolute magnitude
265 int norm_dist = int_log2(mag);
266 int exp = max_approx_prod_exp - f32_mant_bits - cfg.guard-
Bits + norm_dist;
267
268 // fixing normalization distance for subnormal results
269
270 int sig_bits = (!cfg.flushSub && exp <= 0) ? f32_mant_bits -
(1-exp) : f32_mant_bits;
271 sig_bits = std::max(sig_bits, 0);
272 uint32_t rounded_sig = shift_right_jam(uint64_t(mag) <<
sig_bits, norm_dist);
273
274 bool any_inf = any_pos_inf || any_neg_inf;
275 bool overflow = (exp >= f32_exp_mask && mag != 0) ||
any_inf;
276
277 bool op_sign_inf = (any_pos_inf && any_neg_inf);
278 bool nan_out = any_nan || op_sign_inf;
279 bool overflowflag = (exp >= f32_exp_mask && mag != 0) && !
any_inf && !nan_out;
280
281 if (nan_out) {
282 sign = 0;
283 exp = f32_exp_mask;
284 rounded_sig = uint32_t(1) << (f32_mant_bits - 1);
285 } else if (overflow) {
286 exp = f32_exp_mask;
287 rounded_sig = 0;
288 if (any_inf)
289 sign = any_neg_inf;
290 } else if (mag == 0) {
291 // exact zero result
292 exp = 0;
293 } else if (exp <= 0) {
294 if (cfg.flushSub) {
295 // flush output subnormals
296 exp = 0;
297 rounded_sig = 0;
298 } else {
299 exp = 0;
300 // rounded_sig should have been properly denormalized
previously
301 }
302 }
303
304 bulk_norm_out_t su;
305 su.flags = 0;
306 su.out = (rounded_sig & f32_mant_mask)
307 | (exp < f32_mant_bits)
308 | (uint32_t(sign) << (f32_exp_bits + f32_mant_bits));
309
310 if (any_sigNaN) {
311 su.flags |= softfloat_flag_invalid;
312 }
313
314 if (any_invalid_nan || op_sign_inf) {
315 su.flags |= softfloat_flag_invalid;
316 }
317
318 if (overflowflag) {
319 su.flags |= softfloat_flag_overflow;
320 }
321
322 return su;
323 }
324
325 /** bf16_t dot product (without accumulation) */
326 static inline bulk_norm_out_t bulk_norm_dot_bf16(const DotCon-
fig cfg, const bf16_t* a, const bf16_t* b)

```



```

323 {
324 // product are extracted so that the no-mult version can be
more easily matched against the RTL implementation
325 std::vector<uint16_t> prod_sigs(cfg.n);
326
327 // compute products, normalize to largest exponent, accumu-
late
328 for (int i = 0; i < cfg.n; i++) {
329 prod_sigs[i] = a[i].sig() * (uint16_t) b[i].sig();
330 }
331
332 return bulk_norm_dot_no_mult<bf16_t, bf16_t, uint16_t>(cfg,
a, b, &prod_sigs[0]);
333 }
334
335 /** half-precision (fp16/binary16) dot product (without accu-
mulation) */
336 static inline bulk_norm_out_t bulk_norm_dot_fp16(const DotCon-
fig cfg, const fp16_norm_t* a, const fp16_norm_t* b)
337 {
338 // product are extracted so that the no-mult version can be
more easily matched against the RTL implementation
339 std::vector<uint32_t> prod_sigs(cfg.n);
340
341 // compute products, normalize to largest exponent, accumu-
late
342 for (int i = 0; i < cfg.n; i++) {
343 prod_sigs[i] = a[i].sig() * (uint16_t) b[i].sig();
344 }
345
346 return bulk_norm_dot_no_mult<fp16_norm_t, fp16_norm_t,
uint32_t>(cfg, a, b, &prod_sigs[0]);
347 }
348
349 template <typename L, typename R>
bulk_norm_out_t bulk_norm_dot_ofp8(const DotConfig cfg, const
L* a, const R* b)
351 {
352 // products are extracted so that the no-mult version can be
more easily matched against the RTL implementation
353 std::vector<uint16_t> prod_sigs(cfg.n);
354
355 // compute products, normalize to largest exponent, accumu-
late
356 for (int i = 0; i < cfg.n; i++) {
357 prod_sigs[i] = a[i].sig() * (uint16_t) b[i].sig();
358 }
359 return bulk_norm_dot_no_mult<L, R, uint16_t>(cfg, a, b,
&prod_sigs[0]);
360 }
361
362 #endif

```

7. riscv-software-src_riscv-isa-sim/riscv/cachesim.cc (207 lines)

```

1 // See LICENSE for license details.
2
3 #include "cachesim.h"
4 #include "common.h"
5 #include <cstdlib>
6 #include <iostream>
7 #include <iomanip>
8
9 cache_sim_t::cache_sim_t(size_t _sets, size_t _ways, size_t
_linesz, const char* _name)
10 : sets(_sets), ways(_ways), linesz(_linesz), name(_name),
log(false)
11 {
12 init();
13 }
14
15 static void help()
16 {
17 std::cerr << "Cache configurations must be of the form" <<
std::endl;
18 std::cerr << "  sets:ways:blocksize" << std::endl;
19 std::cerr << "where sets, ways, and blocksize are positive
integers, with" << std::endl;
20 std::cerr << "sets and blocksize both powers of two and
blocksize at least 8." << std::endl;
21 exit(1);
22 }
23
24 cache_sim_t* cache_sim_t::construct(const char* config, const
char* name)
25 {
26 const char* wp = strchr(config, ':');
27 if (!wp++) help();
28 const char* bp = strchr(wp, ':');
29 if (!bp++) help();
30
31 size_t sets = atoi(std::string(config, wp).c_str());
32 size_t ways = atoi(std::string(wp, bp).c_str());
33 size_t linesz = atoi(bp);
34
35 if (ways > 4 /* empirical */ && sets == 1)
36 return new fa_cache_sim_t(ways, linesz, name);
37 return new cache_sim_t(sets, ways, linesz, name);

```

```

38 }
39
40 void cache_sim_t::init()
41 {
42 if (sets == 0 || (sets & (sets-1)))
43 help();
44 if (linesz < 8 || (linesz & (linesz-1)))
45 help();
46
47 idx_shift = 0;
48 for (size_t x = linesz; x>1; x >>= 1)
49 idx_shift++;
50
51 tags = new uint64_t[sets*ways]();
52 read_accesses = 0;
53 read_misses = 0;
54 bytes_read = 0;
55 write_accesses = 0;
56 write_misses = 0;
57 bytes_written = 0;
58 writebacks = 0;
59
60 miss_handler = NULL;
61 }
62
63 cache_sim_t::cache_sim_t(const cache_sim_t& rhs)
64 : sets(rhs.sets), ways(rhs.ways), linesz(rhs.linesz),
idx_shift(rhs.idx_shift), name(rhs.name), log(false)
65 {
66 tags = new uint64_t[sets*ways];
67 memcpy(tags, rhs.tags, sets*ways*sizeof(uint64_t));
68 }
69
70
71 cache_sim_t::~cache_sim_t()
72 {
73 print_stats();
74 delete [] tags;
75 }
76
77 void cache_sim_t::print_stats()
78 {
79 float mr = 100.0f*(read_misses+write_misses)/(read_ac-
cesses+write_accesses);
80
81 std::cout << std::setprecision(3) << std::fixed;
82 std::cout << name << " ";
83 std::cout << "Bytes Read: " << bytes_read <<
std::endl;
84 std::cout << name << " ";
85 std::cout << "Bytes Written: " << bytes_written <<
std::endl;
86 std::cout << name << " ";
87 std::cout << "Read Accesses: " << read_accesses <<
std::endl;
88 std::cout << name << " ";
89 std::cout << "Write Accesses: " << write_accesses <<
std::endl;
90 std::cout << name << " ";
91 std::cout << "Read Misses: " << read_misses <<
std::endl;
92 std::cout << name << " ";
93 std::cout << "Write Misses: " << write_misses <<
std::endl;
94 std::cout << name << " ";
95 std::cout << "Writebacks: " << writebacks <<
std::endl;
96 std::cout << name << " ";
97 std::cout << "Miss Rate: " << mr << '%' <<
std::endl;
98 }
99
100 uint64_t* cache_sim_t::check_tag(uint64_t addr)
101 {
102 size_t idx = (addr >> idx_shift) & (sets-1);
103 size_t tag = (addr >> idx_shift) | VALID;
104
105 for (size_t i = 0; i < ways; i++)
106 if (tag == (tags[idx*ways + i] & ~DIRTY))
107 return &tags[idx*ways + i];
108
109 return NULL;
110 }
111
112 uint64_t cache_sim_t::victimize(uint64_t addr)
113 {
114 size_t idx = (addr >> idx_shift) & (sets-1);
115 size_t way = lfsr.next() % ways;
116 uint64_t victim = tags[idx*ways + way];
117 tags[idx*ways + way] = (addr >> idx_shift) | VALID;
118 return victim;
119 }
120
121 void cache_sim_t::access(uint64_t addr, size_t bytes, bool
store)
122 {
123 store ? write_accesses++ : read_accesses++;
124 (store ? bytes_written : bytes_read) += bytes;
125
126 uint64_t* hit_way = check_tag(addr);
127 if (likely(hit_way != NULL))
128 {

```

```

129     if (store)
130         *hit_way |= DIRTY;
131     return;
132 }
133
134 store ? write_misses++ : read_misses++;
135 if (log)
136 {
137     std::cerr << name << " "
138         << (store ? "write" : "read") << " miss 0x"
139         << std::hex << addr << std::endl;
140 }
141
142 uint64_t victim = victimize(addr);
143
144 if ((victim & (VALID | DIRTY)) == (VALID | DIRTY))
145 {
146     uint64_t dirty_addr = (victim & ~(VALID | DIRTY)) <<
idx_shift;
147     if (miss_handler)
148         miss_handler->access(dirty_addr, linesz, true);
149     writebacks++;
150 }
151
152 if (miss_handler)
153     miss_handler->access(addr & ~(linesz-1), linesz, false);
154
155 if (store)
156     *check_tag(addr) |= DIRTY;
157 }
158
159 void cache_sim_t::clean_invalidate(uint64_t addr, size_t
bytes, bool clean, bool inval)
160 {
161     uint64_t start_addr = addr & ~(linesz-1);
162     uint64_t end_addr = (addr + bytes + linesz-1) & ~(linesz-1);
163     uint64_t cur_addr = start_addr;
164     while (cur_addr < end_addr) {
165         uint64_t* hit_way = check_tag(cur_addr);
166         if (likely(hit_way != NULL))
167         {
168             if (clean) {
169                 if (*hit_way & DIRTY) {
170                     writebacks++;
171                     *hit_way &= ~DIRTY;
172                 }
173             }
174
175             if (inval)
176                 *hit_way &= ~VALID;
177         }
178         cur_addr += linesz;
179     }
180     if (miss_handler)
181         miss_handler->clean_invalidate(addr, bytes, clean, inval);
182 }
183
184 fa_cache_sim_t::fa_cache_sim_t(size_t ways, size_t linesz,
const char* name)
185 : cache_sim_t(1, ways, linesz, name)
186 {
187 }
188
189 uint64_t* fa_cache_sim_t::check_tag(uint64_t addr)
190 {
191     auto it = tags.find(addr >> idx_shift);
192     return it == tags.end() ? NULL : &it->second;
193 }
194
195 uint64_t fa_cache_sim_t::victimize(uint64_t addr)
196 {
197     uint64_t old_tag = 0;
198     if (tags.size() == ways)
199     {
200         auto it = tags.begin();
201         std::advance(it, lfsr.next() % ways);
202         old_tag = it->second;
203         tags.erase(it);
204     }
205     tags[addr >> idx_shift] = (addr >> idx_shift) | VALID;
206     return old_tag;
207 }

```

8. riscv-software-src_riscv-isa-sim/riscv/cachesim.h (142 lines)

```

1 // See LICENSE for license details.
2
3 #ifndef _RISCV_CACHE_SIM_H
4 #define _RISCV_CACHE_SIM_H
5
6 #include "memtracer.h"
7 #include "common.h"
8 #include <cstring>
9 #include <string>
10 #include <map>
11 #include <stdint>
12
13 class lfsr_t
14 {
15 public:

```

```

16     lfsr_t() : reg(1) {}
17     lfsr_t(const lfsr_t& lfsr) : reg(lfsr.reg) {}
18     uint32_t next() { return reg = (reg>>1)^(~(reg&1) &
0xd0000001); }
19 private:
20     uint32_t reg;
21 };
22
23 class cache_sim_t
24 {
25 public:
26     cache_sim_t(size_t sets, size_t ways, size_t linesz, const
char* name);
27     cache_sim_t(const cache_sim_t& rhs);
28     virtual ~cache_sim_t();
29
30     void access(uint64_t addr, size_t bytes, bool store);
31     void clean_invalidate(uint64_t addr, size_t bytes, bool
clean, bool inval);
32     void print_stats();
33     void set_miss_handler(cache_sim_t* mh) { miss_handler =
mh; }
34     void set_log(bool _log) { log = _log; }
35
36     static cache_sim_t* construct(const char* config, const
char* name);
37
38 protected:
39     static const uint64_t VALID = 1ULL << 63;
40     static const uint64_t DIRTY = 1ULL << 62;
41
42     virtual uint64_t* check_tag(uint64_t addr);
43     virtual uint64_t victimize(uint64_t addr);
44
45     lfsr_t lfsr;
46     cache_sim_t* miss_handler;
47
48     size_t sets;
49     size_t ways;
50     size_t linesz;
51     size_t idx_shift;
52
53     uint64_t* tags;
54
55     uint64_t read_accesses;
56     uint64_t read_misses;
57     uint64_t bytes_read;
58     uint64_t write_accesses;
59     uint64_t write_misses;
60     uint64_t bytes_written;
61     uint64_t writebacks;
62
63     std::string name;
64     bool log;
65
66     void init();
67 };
68
69 class fa_cache_sim_t : public cache_sim_t
70 {
71 public:
72     fa_cache_sim_t(size_t ways, size_t linesz, const char*
name);
73     uint64_t* check_tag(uint64_t addr) override;
74     uint64_t victimize(uint64_t addr) override;
75 private:
76     static bool cmp(uint64_t a, uint64_t b);
77     std::map<uint64_t, uint64_t> tags;
78 };
79
80 class cache_memtracer_t : public memtracer_t
81 {
82 public:
83     cache_memtracer_t(const char* config, const char* name)
84     {
85         cache = cache_sim_t::construct(config, name);
86     }
87     ~cache_memtracer_t() override
88     {
89         delete cache;
90     }
91     void set_miss_handler(cache_sim_t* mh)
92     {
93         cache->set_miss_handler(mh);
94     }
95     void clean_invalidate(uint64_t addr, size_t bytes, bool
clean, bool inval) override
96     {
97         cache->clean_invalidate(addr, bytes, clean, inval);
98     }
99     void set_log(bool log)
100     {
101         cache->set_log(log);
102     }
103     void print_stats()
104     {
105         cache->print_stats();
106     }
107
108 protected:
109     cache_sim_t* cache;

```

```

110 };
111
112 class icache_sim_t : public cache_memtracer_t
113 {
114 public:
115     icache_sim_t(const char* config, const char* name = "I$")
116         : cache_memtracer_t(config, name) {}
117     bool interested_in_range(uint64_t UNUSED begin, uint64_t UN-
USED end, access_type type) override
118     {
119         return type == FETCH;
120     }
121     void trace(uint64_t addr, size_t bytes, access_type type)
override
122     {
123         if (type == FETCH) cache->access(addr, bytes, false);
124     }
125 };
126
127 class dcache_sim_t : public cache_memtracer_t
128 {
129 public:
130     dcache_sim_t(const char* config, const char* name = "D$")
131         : cache_memtracer_t(config, name) {}
132     bool interested_in_range(uint64_t UNUSED begin, uint64_t UN-
USED end, access_type type) override
133     {
134         return type == LOAD || type == STORE;
135     }
136     void trace(uint64_t addr, size_t bytes, access_type type)
override
137     {
138         if (type == LOAD || type == STORE) cache->access(addr,
bytes, type == STORE);
139     }
140 };
141
142 #endif

```

9. riscv-software-src_riscv-isa-sim/riscv/cfg.cc (71 lines)

```

1 // See LICENSE for license details.
2
3 #include "config.h"
4 #include "cfg.h"
5 #include "mmu.h"
6 #include "decode.h"
7 #include "encoding.h"
8 #include "platform.h"
9
10 mem_cfg_t::mem_cfg_t(reg_t base, reg_t size) : base(base),
size(size)
11 {
12     assert(mem_cfg_t::check_if_supported(base, size));
13 }
14
15 bool mem_cfg_t::check_if_supported(reg_t base, reg_t size)
16 {
17     // The truth of these conditions should be ensured by
whatever is creating
18     // the regions in the first place, but we have them here to
make sure that
19     // we can't end up describing memory regions that don't make
sense. They
20     // ask that the page size is a multiple of the minimum page
size, that the
21     // page is aligned to the minimum page size, that the page
is non-empty,
22     // that the size doesn't overflow size_t, and that the top
address is still
23     // representable in a reg_t.
24     //
25     // Note: (base + size == 0) part of the assertion is to
handle cases like
26     // { base = 0xffff_ffff_ffff_f000, size: 0x1000 }
27     return (size % PGSIZE == 0) &&
(base % PGSIZE == 0) &&
(size_t(size) == size) &&
(size > 0) &&
((base + size > base) || (base + size == 0));
32 }
33
34 cfg_t::cfg_t()
35 {
36     // The default system configuration
37     initrd_bounds = std::make_pair((reg_t)0, (reg_t)0);
38     bootargs = nullptr;
39     isa = DEFAULT_ISA;
40     priv = DEFAULT_PRIV;
41     endianness = endianness_little;
42     pmpregions = 16;
43     pmpgranularity = (1 << PMP_SHIFT);
44     mem_layout = std::vec-
tor<mem_cfg_t>({mem_cfg_t(reg_t(DRAM_BASE), (size_t)2048 << 20)});
45     hartids = std::vector<size_t>({0});
46     explicit_hartids = false;
47     real_time_clint = false;
48     trigger_count = 4;
49     cache_blocksz = 64;

```

```

50 }
51
52 void start_pc_t::set_global(reg_t pc)
53 {
54     global_pc = pc;
55 }
56
57 void start_pc_t::set_override(size_t hart_id, reg_t pc)
58 {
59     hart_pcs[hart_id] = pc;
60 }
61
62 std::optional<reg_t> start_pc_t::get(size_t hart_id) const
63 {
64     auto it = hart_pcs.find(hart_id);
65
66     if (it != hart_pcs.end()) {
67         return it->second;
68     } else {
69         return global_pc;
70     }
71 }

```

10. riscv-software-src_riscv-isa-sim/riscv/cfg.h (102 lines)

```

1 // See LICENSE for license details.
2 #ifndef _RISCVCFG_H
3 #define _RISCVCFG_H
4
5 #include <optional>
6 #include <vector>
7 #include "decode.h"
8 #include <cassert>
9 #include <map>
10 class abstract_sim_if_t;
11
12 typedef enum {
13     endianness_little,
14     endianness_big
15 } endianness_t;
16
17 template <typename T>
18 class cfg_arg_t {
19 public:
20     cfg_arg_t(T default_val)
21         : value(default_val), was_set(false) {}
22
23     bool overridden() const { return was_set; }
24
25     T operator()() const { return value; }
26
27     T operator=(const T v) {
28         value = v;
29         was_set = true;
30         return value;
31     }
32 private:
33     T value;
34     bool was_set;
35 };
36
37 // Configuration that describes a memory region
38 class mem_cfg_t
39 {
40 public:
41     static bool check_if_supported(reg_t base, reg_t size);
42     mem_cfg_t(reg_t base, reg_t size);
43
44     reg_t get_base() const {
45         return base;
46     }
47
48     reg_t get_size() const {
49         return size;
50     }
51
52     reg_t get_inclusive_end() const {
53         return base + size - 1;
54     }
55 private:
56     reg_t base;
57     reg_t size;
58 };
59
60 class start_pc_t
61 {
62 public:
63     void set_global(reg_t pc);
64
65     void set_override(size_t hart_id, reg_t pc);
66
67     std::optional<reg_t> get(size_t hart_id) const;
68 private:
69     std::optional<reg_t> global_pc;
70     std::map<size_t, reg_t> hart_pcs;
71 };

```

```

76
77 class cfg_t
78 {
79 public:
80   cfg_t();
81
82   std::pair<reg_t, reg_t> initrd_bounds;
83   const char * bootargs;
84   const char * isa;
85   const char * priv;
86   endianness_t endianness;
87   reg_t pmpregions;
88   reg_t pmpgranularity;
89   std::vector<mem_cfg_t> mem_layout;
90   start_pc_t start_pc;
91   std::vector<size_t> hartids;
92   bool explicit_hartids;
93   bool real_time_clint;
94   reg_t trigger_count;
95   reg_t cache_blocksz;
96   std::optional<abstract_sim_if_t> external_simulator;
97
98   size_t nprocs() const { return hartids.size(); }
99   size_t max_hartid() const { return hartids.back(); }
100 };
101
102 #endif

```

11. riscv-software-src_riscv-isa-sim/riscv/check-opcode-overlap.t.cc (79 lines)

```

1 #include "decode.h"
2 #include "common.h"
3 #include <unordered_map>
4 #include <vector>
5 #include <string>
6 #include <cstdio>
7
8 struct opcode {
9   insn_bits_t match;
10  insn_bits_t mask;
11  std::string name;
12 };
13
14 static bool overlaps(const opcode& a, const opcode& b)
15 {
16   return (a.mask & b.mask & (a.match ^ b.match)) == 0;
17 }
18
19 int main()
20 {
21   #define DECLARE_INSN(name, match, mask) \
22     const insn_bits_t UNUSED name##_match = (match),
23   name##_mask = (mask);
24   #include "encoding.h"
25   #undef DECLARE_INSN
26
27   static const opcode static_list[] = {
28     #define DEFINE_INSN(name) \
29       {name##_match, name##_mask, #name},
30     #include "insn_list.h"
31     #undef DEFINE_INSN
32   };
33
34   std::unordered_map<std::string, bool> overlap_list;
35   #define DECLARE_OVERLAP_INSN(name, ext) \
36     overlap_list[std::string(#name)] = false;
37   #define DECLARE_OVERLAP_INSN_RV64(name, ext) \
38     overlap_list[std::string(#name)] = false;
39   #include "overlap_list.h"
40   #undef DECLARE_OVERLAP_INSN
41   #undef DECLARE_OVERLAP_INSN_RV64
42
43   std::vector<const opcode> list;
44   for (size_t i = 0; i < std::size(static_list); i++) {
45     if (!overlap_list.count(static_list[i].name))
46       list.push_back(&static_list[i]);
47   }
48
49   bool ok = true;
50
51   for (size_t i = 1; i < list.size(); i++) {
52     for (size_t j = 0; j < i; j++) {
53       if (overlaps(*list[i], *list[j])) {
54         fprintf(stderr, "Instruction %s (%" PRIx64 ") overlaps
55         instruction %s (%" PRIx64 ", mask %" PRIx64 ")\n",
56             list[i]->name.c_str(), list[i]->match,
57             list[j]->name.c_str(), list[j]->match, list[j]->mask);
58         ok = false;
59       }
60     }
61   }
62
63   // make sure nothing in the overlap list is unused
64   for (size_t i = 1; i < std::size(static_list); i++) {
65     for (size_t j = 0; j < i; j++) {
66       if (overlaps(static_list[i], static_list[j])) {
67         overlap_list[static_list[i].name] = true;
68         overlap_list[static_list[j].name] = true;
69       }
70     }
71   }

```

```

66   }
67 }
68 }
69
70 for (auto const& [name, used] : overlap_list) {
71   if (!used) {
72     fprintf(stderr, "Instruction %s overlaps nothing, so
73     overlap list entry has no effect\n",
74         name.c_str());
75     ok = false;
76   }
77 }
78
79 return ok ? 0 : -1;
80 }

```

12. riscv-software-src_riscv-isa-sim/riscv/clint.cc (148 lines)

```

1 #include <sys/time.h>
2 #include <sstream>
3 #include "devices.h"
4 #include "processor.h"
5 #include "simif.h"
6 #include "sim.h"
7 #include "dts.h"
8
9 clint_t::clint_t(const simif_t* sim, uint64_t freq_hz, bool
10 real_time)
11 : sim(sim), freq_hz(freq_hz), real_time(real_time), mtime(0)
12 {
13   struct timeval base;
14   gettimeofday(&base, NULL);
15
16   real_time_ref_secs = base.tv_sec;
17   real_time_ref_usec = base.tv_usec;
18   tick(0);
19 }
20
21 /* 0000 msip hart 0
22 * 0004 msip hart 1
23 * 4000 mtimecmp hart 0 lo
24 * 4004 mtimecmp hart 0 hi
25 * 4008 mtimecmp hart 1 lo
26 * 400c mtimecmp hart 1 hi
27 * bfff mtime lo
28 * bfff mtime hi
29 */
30
31 #define MSIP_BASE 0x0
32 #define MTIMECMP_BASE 0x4000
33 #define MTIME_BASE 0xbfff
34
35 bool clint_t::load(reg_t addr, size_t len, uint8_t* bytes)
36 {
37   if (len > 8)
38     return false;
39
40   tick(0);
41
42   static_assert(MSIP_BASE == 0);
43   if ((*addr >= MSIP_BASE && *addr < MTIMECMP_BASE) {
44     if (len == 8) {
45       // Implement double-word loads as a pair of word loads
46       return load(addr, 4, bytes) && load(addr + 4, 4, bytes +
47       4);
48     }
49
50     const auto hart_id = (addr - MSIP_BASE) / sizeof(msip_t);
51     const msip_t res = sim->get_harts().count(hart_id) &&
52     (sim->get_harts().at(hart_id)->state.mip->read() & MIP_MSIP);
53     read_little_endian_reg(res, addr, len, bytes);
54     return true;
55   } else if (addr >= MTIMECMP_BASE && addr < MTIME_BASE) {
56     const auto hart_id = (addr - MTIMECMP_BASE) /
57     sizeof(mtimecmp_t);
58     const mtime_t res = sim->get_harts().count(hart_id) ?
59     mtimecmp[hart_id] : 0;
60     read_little_endian_reg(res, addr, len, bytes);
61   } else if (addr >= MTIME_BASE && addr < MTIME_BASE +
62   sizeof(mtime_t)) {
63     read_little_endian_reg(mtime, addr, len, bytes);
64   } else if (addr + len <= CLINT_SIZE) {
65     memset(bytes, 0, len);
66   } else {
67     return false;
68   }
69   return true;
70 }
71
72 bool clint_t::store(reg_t addr, size_t len, const uint8_t*
73 bytes)
74 {
75   if (len > 8)
76     return false;
77
78   static_assert(MSIP_BASE == 0);
79   if ((*addr >= MSIP_BASE && *addr < MTIMECMP_BASE) {
80     if (len == 8) {

```

```

75 // Implement double-word stores as a pair of word stores
76 return store(addr, 4, bytes) && store(addr + 4, 4, bytes +
4);
77 }
78
79 if (addr % sizeof(msip_t) == 0) { // ignore in-between
bytes
80 msip_t msip = 0;
81 write_little_endian_reg(&msip, addr, len, bytes);
82
83 const auto hart_id = (addr - MSIP_BASE) /
sizeof(msip_t);
84 if (sim->get_harts().count(hart_id))
85 sim->get_harts().at(hart_id)->state.mip->back-
door_write_with_mask(MIP_MSIP, msip & 1 ? MIP_MSIP : 0);
86 }
87 } else if (addr >= MTIMECMP_BASE && addr < MTIME_BASE) {
88 const auto hart_id = (addr - MTIMECMP_BASE) /
sizeof(mtimecmp_t);
89 if (sim->get_harts().count(hart_id))
90 write_little_endian_reg(&mtimecmp[hart_id], addr, len,
bytes);
91 } else if (addr >= MTIME_BASE && addr < MTIME_BASE +
sizeof(mtime_t)) {
92 write_little_endian_reg(&mtime, addr, len, bytes);
93 } else if (addr + len <= CLINT_SIZE) {
94 // Do nothing
95 } else {
96 return false;
97 }
98 tick(0);
99 return true;
100 }
101
102 void clint_t::tick(reg_t rtc_ticks)
103 {
104 if (real_time) {
105 struct timeval now;
106 uint64_t diff_usec;
107
108 gettimeofday(&now, NULL);
109 diff_usec = ((now.tv_sec - real_time_ref_secs) * 1000000
+ (now.tv_usec - real_time_ref_usec));
110 mtime = diff_usec * freq_hz / 1000000;
111 } else {
112 mtime += rtc_ticks;
113 }
114
115 for (const auto& [hart_id, hart] : sim->get_harts()) {
116 hart->state.time->sync(mtime);
117 hart->state.mip->backdoor_write_with_mask(MIP_MTIP, mtime
>= mtimecmp[hart_id] ? MIP_MTIP : 0);
118 }
119 }
120
121 clint_t* clint_parse_from_fdt(const void* fdt, const sim_t*
sim, reg_t* base,
122 const std::vector<std::string& sargs UNUSED) {
123 if (fdt_parse_clint(fdt, base, "riscv,clint0") == 0 ||
fdt_parse_clint(fdt, base, "sifive,clint0") == 0)
124 return new clint_t(sim,
125 sim->CPU_HZ / sim->INSNS_PER_RTC_TICK,
126 sim->get_cfg().real_time_clint);
127
128 else
129 return nullptr;
130 }
131
132 std::string clint_generate_dts(const sim_t* sim, const
std::vector<std::string& sargs UNUSED) {
133 std::stringstream s;
134 s << std::hex
135 << " clint0@ " << CLINT_BASE << " {\n"
136 << " compatible = \"riscv,clint0\";\n"
137 << " interrupts-extended = " << std::dec;
138 for (size_t i = 0; i < sim->get_cfg().nprocs(); i++)
139 s << "&CPU" << i << "_intc 3 &CPU" << i << "_intc 7 ";
140 reg_t clintbs = CLINT_BASE;
141 reg_t clintsz = CLINT_SIZE;
142 s << std::hex << ">";
143 << " reg = <0x" << (clintbs >> 32) << " 0x" << (clintbs
& (uint32_t)-1) <<
144 << " 0x" << (clintsz >> 32) << " 0x" << (clintsz &
(uint32_t)-1) << ">;\n";
145 return s.str();
146 }
147
148 REGISTER_BUILTIN_DEVICE(clint, clint_parse_from_fdt,
clint_generate_dts)

```

13. riscv-software-src_riscv-isa-sim/riscv/common.h (34 lines)

```

1 // See LICENSE for license details.
2
3 #ifndef _RISCV_COMMON_H
4 #define _RISCV_COMMON_H
5
6 #ifdef __GNUC__
7 #define likely(x) __builtin_expect(x, 1)

```

```

8 #define unlikely(x) __builtin_expect(x, 0)
9 #define NOINLINE __attribute__((noinline))
10 #define NORETURN __attribute__((noreturn))
11 #define ALWAYS_INLINE __attribute__((always_inline))
12 #define UNUSED __attribute__((unused))
13 #else
14 #define likely(x) (x)
15 #define unlikely(x) (x)
16 #define NOINLINE
17 #define NORETURN
18 #define ALWAYS_INLINE
19 #define UNUSED
20 #endif
21
22 #ifndef __has_builtin
23 #define __has_builtin(x) 0
24 #endif
25
26 #if __has_cpp_attribute(assume)
27 #define assume(x) [[assume(x)]]
28 #elif __has_builtin(__builtin_assume)
29 #define assume(x) __builtin_assume(x)
30 #else
31 #define assume(x) ((void) 0)
32 #endif
33
34 #endif

```

14. riscv-software-src_riscv-isa-sim/riscv/csr_init.cc (641 lines)

```

1 #include "processor.h"
2 #include "debug_defines.h"
3
4 void state_t::add_csr(reg_t addr, const csr_t_p& csr)
5 {
6   csrmap[addr] = csr;
7 }
8
9 #define add_const_ext_csr(ext, addr, csr) do { auto csr__ =
(csr); if (proc->extension_enabled_const(ext)) { add_csr(addr,
csr__); } } while (0)
10 #define add_ext_csr(ext, addr, csr) do { auto csr__ = (csr);
if (proc->extension_enabled(ext)) { add_csr(addr, csr__); } } while
(0)
11 #define add_user_csr(addr, csr) add_const_ext_csr('U', addr,
csr)
12 #define add_supervisor_csr(addr, csr) add_const_ext_csr('S',
addr, csr)
13 #define add_hypervisor_csr(addr, csr) add_ext_csr('H', addr,
csr)
14
15 void state_t::add_ireg_proxy(sscsrind_reg_csr_t_p ireg,
aia_ireg_proxy_csr_t_p aia_proxy)
16 {
17   // reserved range RAZ/WI
18   ireg->add_ireg_proxy(0x71, aia_proxy);
19   for (int i = 0x73; i <= 0x7f; i++)
20     ireg->add_ireg_proxy(i, aia_proxy);
21 }
22
23 void state_t::csr_init(processor_t* const proc, reg_t max_isa)
24 {
25 // This assumes xlen is always max_xlen, which is true today
26 // mstatus_csr_t::unlogged_write():
27 auto xlen = proc->get_isa().get_max_xlen();
28
29 add_csr(CSR_MISA, misa = std::make_shared<misa_csr_t>(proc,
CSR_MISA, max_isa));
30 mstatus = std::make_shared<mstatus_csr_t>(proc, CSR_M-
STATUS);
31
32 if (xlen == 32) {
33   add_csr(CSR_MSTATUS,
std::make_shared<rv32_low_csr_t>(proc, CSR_MSTATUS, mstatus));
34   add_csr(CSR_MSTATUSH, mstatush =
std::make_shared<rv32_high_csr_t>(proc, CSR_MSTATUSH, mstatus));
35 } else {
36   add_csr(CSR_MSTATUS, mstatus);
37 }
38 add_csr(CSR_MEPC, mepc = std::make_shared<mepc_csr_t>(proc,
CSR_MEPC));
39 add_csr(CSR_MTVAL, mtval =
std::make_shared<basic_csr_t>(proc, CSR_MTVAL, 0));
40 add_csr(CSR_MSCRATCH, std::make_shared<basic_csr_t>(proc,
CSR_MSCRATCH, 0));
41 add_csr(CSR_MTVEC, mtvec =
std::make_shared<tvec_csr_t>(proc, CSR_MTVEC));
42 add_csr(CSR_MCAUSE, mcause =
std::make_shared<cause_csr_t>(proc, CSR_MCAUSE));
43
44 minstretcfg = std::make_shared<smcntrpmf_csr_t>(proc,
CSR_MINSTRETCFG);
45 mcyclecfg = std::make_shared<smcntrpmf_csr_t>(proc, CSR_M-
CYCLECFG);
46
47 minstret = std::make_shared<wide_counter_csr_t>(proc,
CSR_MINSTRET, minstretcfg);
48 mcycle = std::make_shared<wide_counter_csr_t>(proc, CSR_M-

```



```

CYCLE, mcyclecfg);
49   time = std::make_shared<time_counter_csr_t>(proc, CSR_TIME);
50   if (proc->extension_enabled_const(EXT_ZICNTR)) {
51       add_csr(CSR_INSTRET,
std::make_shared<counter_proxy_csr_t>(proc, CSR_INSTRET, minstret));
52       add_csr(CSR_CYCLE,
std::make_shared<counter_proxy_csr_t>(proc, CSR_CYCLE, mcycle));
53       add_csr(CSR_TIME, time_proxy =
std::make_shared<counter_proxy_csr_t>(proc, CSR_TIME, time));
54   }
55   if (xlen == 32) {
56       csr_t_p minstret, mcycle;
57       add_csr(CSR_MINSTRET,
std::make_shared<rv32_low_csr_t>(proc, CSR_MINSTRET, minstret));
58       add_csr(CSR_MINSTRETH, minstret =
std::make_shared<rv32_high_csr_t>(proc, CSR_MINSTRETH, minstret));

59       add_csr(CSR_MCYCLE, std::make_shared<rv32_low_csr_t>(proc,
CSR_MCYCLE, mcycle));
60       add_csr(CSR_MCYCLEH, mcycleh =
std::make_shared<rv32_high_csr_t>(proc, CSR_MCYCLEH, mcycle));
61       if (proc->extension_enabled_const(EXT_ZICNTR)) {
62           auto timeh = std::make_shared<rv32_high_csr_t>(proc,
CSR_TIMEH, time);
63           add_csr(CSR_INSTRETH,
std::make_shared<counter_proxy_csr_t>(proc, CSR_INSTRETH, min-
streth));
64           add_csr(CSR_CYCLEH,
std::make_shared<counter_proxy_csr_t>(proc, CSR_CYCLEH, mcycleh));
65           add_csr(CSR_TIMEH,
std::make_shared<counter_proxy_csr_t>(proc, CSR_TIMEH, timeh));
66       }
67       else {
68           add_csr(CSR_MINSTRET, minstret);
69           add_csr(CSR_MCYCLE, mcycle);
70       }
71       if (proc->extension_enabled_const(EXT_SMCNTRPMF)) {
72           if (xlen == 32) {
73               add_csr(CSR_MCYCLECFG,
std::make_shared<rv32_low_csr_t>(proc, CSR_MCYCLECFG, mcyclecfg));
74               add_csr(CSR_MCYCLECFGH,
std::make_shared<rv32_high_csr_t>(proc, CSR_MCYCLECFGH, mcyclecfg));
75               add_csr(CSR_MINSTRETCFG,
std::make_shared<rv32_low_csr_t>(proc, CSR_MINSTRETCFG, min-
stretcfg));
76               add_csr(CSR_MINSTRETCFHH,
std::make_shared<rv32_high_csr_t>(proc, CSR_MINSTRETCFHH, min-
stretcfg));
77           }
78           else {
79               add_csr(CSR_MCYCLECFG, mcyclecfg);
80               add_csr(CSR_MINSTRETCFG, minstretcfg);
81           }
82           for (reg_t i = 0; i < N_HPM_COUNTERS; ++i) {
83               const reg_t which_mevent = CSR_MHPMEVENT3 + i;
84               const reg_t which_meventh = CSR_MHPMEVENT3H + i;
85               const reg_t which_mcounter = CSR_MHPM_COUNTER3 + i;
86               const reg_t which_mcounterh = CSR_MHPM_COUNTER3H + i;
87               const reg_t which_counter = CSR_HPM_COUNTER3 + i;
88               const reg_t which_counterh = CSR_HPM_COUNTER3H + i;
89               mevent[i] = std::make_shared<mevent_csr_t>(proc,
which_mevent);
90               auto mcounter = std::make_shared<const_csr_t>(proc,
which_mcounter, 0);
91               add_csr(which_mcounter, mcounter);
92           }

93       auto counter = std::make_shared<counter_proxy_csr_t>(proc,
which_counter, mcounter);
94       add_const_ext_csr(EXT_ZIHPM, which_counter, counter);
95       if (xlen == 32) {
96           add_csr(which_mevent,
std::make_shared<rv32_low_csr_t>(proc, which_mevent, mevent[i]));
97           auto mcounterh = std::make_shared<const_csr_t>(proc,
which_mcounterh, 0);
98           add_csr(which_mcounterh, mcounterh);
99           add_const_ext_csr(EXT_ZIHPM, which_counterh, mcounterh);
100          add_const_ext_csr(EXT_ZIHPM, which_counterh, mcounterh);
101          add_const_ext_csr(EXT_SSCOPFPMF, which_meventh,
std::make_shared<rv32_high_csr_t>(proc, which_meventh, mevent[i]));
102      }
103      else {
104          add_csr(which_mevent, mevent[i]);
105      }
106      add_const_ext_csr(EXT_SSCOPFPMF, CSR_SCOUNTOVF,
std::make_shared<scountovf_csr_t>(proc, CSR_SCOUNTOVF));
107      mie = std::make_shared<mie_csr_t>(proc, CSR_MIE);
108      mip = std::make_shared<mip_csr_t>(proc, CSR_MIP);
109      if (xlen == 32 && proc->extension_enabled_const(EXT_SMAIA))
{
110          add_csr(CSR_MIE, std::make_shared<rv32_low_csr_t>(proc,
CSR_MIE, mie));
111          add_csr(CSR_MIEH, std::make_shared<rv32_high_csr_t>(proc,
CSR_MIEH, mie));
112          add_csr(CSR_MIP, std::make_shared<rv32_low_csr_t>(proc,
CSR_MIP, mip));
113          add_csr(CSR_MIPH, std::make_shared<rv32_high_csr_t>(proc,
CSR_MIPH, mip));
114      }
115      else {
116          add_csr(CSR_MIE, mie);
117          add_csr(CSR_MIP, mip);
118      }
119      auto sip_sie_accr =
std::make_shared<generic_int_accessor_t>(
120          this,
121          ~MIP_HS_MASK, // read_mask
122          MIP_SSIP | MIP_LCOFIP, // ip_write_mask
123          ~MIP_HS_MASK, // ie_write_mask
124          generic_int_accessor_t::mask_mode_t::MIDELEG,
125          0 // shiftamt
126      );
127      auto hip_hie_accr =
std::make_shared<generic_int_accessor_t>(
128          this,
129          MIP_HS_MASK, // read_mask
130          MIP_VSSIP, // ip_write_mask
131          MIP_HS_MASK, // ie_write_mask
132          generic_int_accessor_t::mask_mode_t::MIDELEG,
133          0 // shiftamt
134      );
135      auto vsip_vsie_accr =
std::make_shared<generic_int_accessor_t>(
136          this,
137          MIP_VS_MASK, // read_mask
138          MIP_VSSIP, // ip_write_mask
139          MIP_VS_MASK, // ie_write_mask
140          generic_int_accessor_t::mask_mode_t::HIDELEG,
141          1 // shiftamt
142      );
143      nonvirtual_sip = std::make_shared<sip_csr_t>(proc, CSR_SIP,
sip_sie_accr);
144      auto vsip = std::make_shared<mip_proxy_csr_t>(proc, CSR_V-
SIP, vsip_vsie_accr);
145      auto sip = std::make_shared<virtualized_csr_t>(proc, nonvir-
tual_sip, vsip);
146      if (xlen == 32 && proc->extension_enabled_const(EXT_SSAIA))
{
147          add_hypervisor_csr(CSR_VSIP,
std::make_shared<rv32_low_csr_t>(proc, CSR_VSIP, vsip));
148          add_hypervisor_csr(CSR_VSIPH,
std::make_shared<rv32_high_csr_t>(proc, CSR_VSIPH, vsip));
149          add_supervisor_csr(CSR_SIP,
std::make_shared<rv32_low_csr_t>(proc, CSR_SIP, sip));
150          add_supervisor_csr(CSR_SIPH,
std::make_shared<rv32_high_csr_t>(proc, CSR_SIPH, sip));
151      }
152      else {
153          add_hypervisor_csr(CSR_VSIP, vsip);
154          add_supervisor_csr(CSR_SIP, sip);
155      }
156      add_hypervisor_csr(CSR_HIP, hip =
std::make_shared<mip_proxy_csr_t>(proc, CSR_HIP, hip_hie_accr));
157      hvip = std::make_shared<hvip_csr_t>(proc, CSR_HVIP, 0);
158      if (xlen == 32 && proc->extension_enabled_const(EXT_SSAIA))
{
159          add_hypervisor_csr(CSR_HVIP,
std::make_shared<rv32_low_csr_t>(proc, CSR_HVIP, hvip));
160          add_hypervisor_csr(CSR_HVIPH,
std::make_shared<rv32_high_csr_t>(proc, CSR_HVIPH, hvip));
161      }
162      else {
163          add_hypervisor_csr(CSR_HVIP, hvip);
164      }
165      nonvirtual_sie = std::make_shared<sie_csr_t>(proc, CSR_SIE,
sip_sie_accr);
166      auto vsie = std::make_shared<mie_proxy_csr_t>(proc,
CSR_VSIE, vsip_vsie_accr);
167      auto sie = std::make_shared<virtualized_csr_t>(proc, nonvir-
tual_sie, vsie);
168      if (xlen == 32 && proc->extension_enabled_const(EXT_SSAIA))
{
169          add_hypervisor_csr(CSR_VSIE,
std::make_shared<rv32_low_csr_t>(proc, CSR_VSIE, vsie));
170          add_hypervisor_csr(CSR_VSIEH,
std::make_shared<rv32_high_csr_t>(proc, CSR_VSIEH, vsie));
171          add_supervisor_csr(CSR_SIE,
std::make_shared<rv32_low_csr_t>(proc, CSR_SIE, sie));
172          add_supervisor_csr(CSR_SIEH,
std::make_shared<rv32_high_csr_t>(proc, CSR_SIEH, sie));
173      }
174      else {
175          add_hypervisor_csr(CSR_VSIE, vsie);
176          add_supervisor_csr(CSR_SIE, sie);
177      }
178      add_hypervisor_csr(CSR_HIE, hie =
std::make_shared<mie_proxy_csr_t>(proc, CSR_HIE, hip_hie_accr));
179      medeleg = std::make_shared<mideleg_csr_t>(proc,
CSR_MEDELEG);
180      if (xlen == 32) {
181          add_supervisor_csr(CSR_MEDELEG,
std::make_shared<rv32_low_csr_t>(proc, CSR_MEDELEG, medeleg));
182          add_supervisor_csr(CSR_MEDELEGH,
std::make_shared<rv32_high_csr_t>(proc, CSR_MEDELEGH, medeleg));
183      }
184      else {
185          add_supervisor_csr(CSR_MEDELEG, medeleg);
186      }
187      mideleg = std::make_shared<mideleg_csr_t>(proc,

```

```

CSR_MIDELEG);
189 if (xlen == 32 && proc->extension_enabled_const(EXT_SMAIA))
{
190     add_supervisor_csr(CSR_MIDELEG,
std::make_shared<rv32_low_csr_t>(proc, CSR_MIDELEG, mideleg));
191     add_supervisor_csr(CSR_MIDELEGH,
std::make_shared<rv32_high_csr_t>(proc, CSR_MIDELEGH, mideleg));
192 } else {
193     add_supervisor_csr(CSR_MIDELEG, mideleg);
194 }
195 const reg_t counteren_mask = (proc->extension_enabled_const(EXT_ZICNTR) ? 0x7UL : 0x0) | (proc->extension_enabled_const(EXT_ZIHPM) ? 0xffffffffULL : 0x0);
196 add_user_csr(CSR_MCOUNTEN, mcounteren = std::make_shared<masked_csr_t>(proc, CSR_MCOUNTEN, counteren_mask, 0));
197 add_csr(CSR_MCOUNTINHIBIT, mcountinhibit = std::make_shared<masked_csr_t>(proc, CSR_MCOUNTINHIBIT, counteren_mask & (~MCOUNTEREN_TIME), 0));
198 add_supervisor_csr(CSR_SCOUNTEREN, scounteren = std::make_shared<masked_csr_t>(proc, CSR_SCOUNTEREN, counteren_mask, 0));
199 nonvirtual_sepc = std::make_shared<epc_csr_t>(proc, CSR_SEPC);
200 add_hypervisor_csr(CSR_VSEPC, vsepc = std::make_shared<epc_csr_t>(proc, CSR_VSEPC));
201 add_supervisor_csr(CSR_SEPC, sepc = std::make_shared<virtualized_csr_t>(proc, nonvirtual_sepc, vsepc));
202 nonvirtual_stval = std::make_shared<basic_csr_t>(proc, CSR_STVAL, 0);
203 add_hypervisor_csr(CSR_VSTVAL, vstval = std::make_shared<basic_csr_t>(proc, CSR_VSTVAL, 0));
204 add_supervisor_csr(CSR_STVAL, stval = std::make_shared<virtualized_csr_t>(proc, nonvirtual_stval, vstval));
205 auto sscratch = std::make_shared<basic_csr_t>(proc, CSR_SSCRATCH, 0);
206 auto vsscratch = std::make_shared<basic_csr_t>(proc, CSR_VSSCRATCH, 0);
207 // Note: if max_isa does not include H, we don't really need this virtualized_csr_t at all (though it doesn't hurt):
208 add_supervisor_csr(CSR_SSCRATCH, std::make_shared<virtualized_csr_t>(proc, sscratch, vsscratch));
209 add_hypervisor_csr(CSR_VSSCRATCH, vsscratch);
210 nonvirtual_stvec = std::make_shared<tvec_csr_t>(proc, CSR_STVEC);
211 add_hypervisor_csr(CSR_VSTVEC, vstvec = std::make_shared<tvec_csr_t>(proc, CSR_VSTVEC));
212 add_supervisor_csr(CSR_STVEC, stvec = std::make_shared<virtualized_csr_t>(proc, nonvirtual_stvec, vstvec));
213 auto nonvirtual_satp = std::make_shared<satp_csr_t>(proc, CSR_SATP);
214 add_hypervisor_csr(CSR_VSATP, vsatp = std::make_shared<base_atp_csr_t>(proc, CSR_VSATP));
215 add_supervisor_csr(CSR_SATP, satp = std::make_shared<virtualized_satp_csr_t>(proc, nonvirtual_satp, vsatp));
216 nonvirtual_scause = std::make_shared<cause_csr_t>(proc, CSR_SCAUSE);
217 add_hypervisor_csr(CSR_VSCAUSE, vscause = std::make_shared<cause_csr_t>(proc, CSR_VSCAUSE));
218 add_supervisor_csr(CSR_SCAUSE, scause = std::make_shared<virtualized_csr_t>(proc, nonvirtual_scause, vscause));
219 mtval2 = std::make_shared<mtval2_csr_t>(proc, CSR_MTVAL2);
220 if (proc->extension_enabled('H') || proc->extension_enabled(EXT_SDBLTRP))
221     add_csr(CSR_MTVAL2, mtval2);
222 add_hypervisor_csr(CSR_MTIMEINST, mtimeinst = std::make_shared<hypervisor_csr_t>(proc, CSR_MTIMEINST));
223 add_hypervisor_csr(CSR_HSTATUS, hstatus = std::make_shared<hstatus_csr_t>(proc, CSR_HSTATUS));
224 if (proc->extension_enabled_const(EXT_SSAIA)) {
225     add_hypervisor_csr(CSR_HGEIE, hgeie = std::make_shared<hgeie_csr_t>(proc, CSR_HGEIE, proc->geilen));
226     add_hypervisor_csr(CSR_HGEIP, hgeip = std::make_shared<hgeip_csr_t>(proc, CSR_HGEIP));
227 } else {
228     add_hypervisor_csr(CSR_HGEIE, hgeie = std::make_shared<const_csr_t>(proc, CSR_HGEIE, 0));
229     add_hypervisor_csr(CSR_HGEIP, hgeip = std::make_shared<const_csr_t>(proc, CSR_HGEIP, 0));
230 }
231 hideleg = std::make_shared<hideleg_csr_t>(proc, CSR_HIDELEG, mideleg);
232 if (xlen == 32 && proc->extension_enabled_const(EXT_SSAIA))
{
233     add_hypervisor_csr(CSR_HIDELEG, std::make_shared<rv32_low_csr_t>(proc, CSR_HIDELEG, hideleg));
234     add_hypervisor_csr(CSR_HIDELEGH, std::make_shared<rv32_high_csr_t>(proc, CSR_HIDELEGH, hideleg));
235 } else {
236     add_hypervisor_csr(CSR_HIDELEG, hideleg);
237 }
238
239 // The spec allows FIOM to read as zero only where there is no S-mode or no
240 // paging, so it is writable exactly when neither holds.
241 const bool fiom_writable = proc->extension_enabled('S') && proc->get_max_vaddr_bits() > 0;
242 const reg_t menvcfg_mask = (fiom_writable ? MENVCFG_FIOM : 0) |
243 (proc->extension_enabled(EXT_ZICBOM) ? MENVCFG_CBCFE | MENVCFG_CBIE : 0) |
244 (proc->extension_enabled(EXT_ZICBOZ) ? MENVCFG_CBZE : 0) |
245 (proc->extension_enabled(EXT_SMNPM) ? MENVCFG_PMM : 0) |
246 (proc->extension_enabled(EXT_SVADU) ? MENVCFG_ADUE : 0) |
247 (proc->extension_enabled(EXT_SVPBMT) ? MENVCFG_PBMTE : 0) |
248 (proc->extension_enabled(EXT_SSTC) ? MENVCFG_STCE : 0) |
249 (proc->extension_enabled(EXT_ZICFILP) ? MENVCFG_LPE : 0) |
250 (proc->extension_enabled(EXT_ZICFISS) ? MENVCFG_SSE : 0) |
251 (proc->extension_enabled(EXT_SSDBLTRP) ? MENVCFG_DTE : 0) |
252 (proc->extension_enabled(EXT_SMCDELEG) ? MENVCFG_CDE : 0);
253 menvcfg = std::make_shared<menvcfg_csr_t>(proc, CSR_MENVCFG, menvcfg_mask, 0);
254 if (xlen == 32) {
255     add_user_csr(CSR_MENVCFG, std::make_shared<rv32_low_csr_t>(proc, CSR_MENVCFG, menvcfg));
256     add_user_csr(CSR_MENVCFGH, std::make_shared<rv32_high_csr_t>(proc, CSR_MENVCFGH, menvcfg));
257 } else {
258     add_user_csr(CSR_MENVCFG, menvcfg);
259 }
260
261 const reg_t hedeleg_mask =
262 (1 << CAUSE_MISALIGNED_FETCH) |
263 (1 << CAUSE_FETCH_ACCESS) |
264 (1 << CAUSE_ILLEGAL_INSTRUCTION) |
265 (1 << CAUSE_BREAKPOINT) |
266 (1 << CAUSE_MISALIGNED_LOAD) |
267 (1 << CAUSE_LOAD_ACCESS) |
268 (1 << CAUSE_MISALIGNED_STORE) |
269 (1 << CAUSE_STORE_ACCESS) |
270 (1 << CAUSE_USER_ECALL) |
271 (1 << CAUSE_FETCH_PAGE_FAULT) |
272 (1 << CAUSE_LOAD_PAGE_FAULT) |
273 (1 << CAUSE_STORE_PAGE_FAULT) |
274 ((proc->extension_enabled(EXT_ZICFISS) || proc->extension_enabled(EXT_ZICFILP)) ?
275 (1 << CAUSE_SOFTWARE_CHECK_FAULT) : 0) |
276 (proc->extension_enabled(EXT_ZICNTR) ?
277 (1 << CAUSE_HARDWARE_ERROR_FAULT) : 0);
278 add_hypervisor_csr(CSR_HEDELEG, hedeleg = std::make_shared<masked_csr_t>(proc, CSR_HEDELEG, hedeleg_mask, 0));
279 add_hypervisor_csr(CSR_HCOUNTEN, hcounteren = std::make_shared<masked_csr_t>(proc, CSR_HCOUNTEN, counteren_mask, 0));
280 htimedelta = std::make_shared<basic_csr_t>(proc, CSR_HTIMEDELTA, 0);
281 if (xlen == 32) {
282     add_hypervisor_csr(CSR_HTIMEDELTA, std::make_shared<rv32_low_csr_t>(proc, CSR_HTIMEDELTA, htimedelta));
283     add_hypervisor_csr(CSR_HTIMEDELTAH, std::make_shared<rv32_high_csr_t>(proc, CSR_HTIMEDELTAH, htimedelta));
284 } else {
285     add_hypervisor_csr(CSR_HTIMEDELTA, htimedelta);
286 }
287 add_hypervisor_csr(CSR_HTVL, htval = std::make_shared<basic_csr_t>(proc, CSR_HTVL, 0));
288 add_hypervisor_csr(CSR_HTIMEINST, htinst = std::make_shared<basic_csr_t>(proc, CSR_HTIMEINST, 0));
289 add_hypervisor_csr(CSR_HGATP, hgatp = std::make_shared<hgatp_csr_t>(proc, CSR_HGATP));
290 nonvirtual_sstatus = std::make_shared<sstatus_proxy_csr_t>(proc, CSR_SSTATUS, mstatus);
291 add_hypervisor_csr(CSR_VSSTATUS, vsstatus = std::make_shared<vsstatus_csr_t>(proc, CSR_VSSTATUS));
292 add_supervisor_csr(CSR_SSTATUS, sstatus = std::make_shared<sstatus_csr_t>(proc, nonvirtual_sstatus, vsstatus));
293
294 add_csr(CSR_DPC, dpc = std::make_shared<dpc_csr_t>(proc, CSR_DPC));
295 add_csr(CSR_DSCRATCH0, std::make_shared<debug_mode_csr_t>(proc, CSR_DSCRATCH0));
296 add_csr(CSR_DSCRATCH1, std::make_shared<debug_mode_csr_t>(proc, CSR_DSCRATCH1));
297 add_csr(CSR_DCSR, dcsr = std::make_shared<dcsr_csr_t>(proc, CSR_DCSR));
298
299 add_csr(CSR_TSELECT, tselect = std::make_shared<tselect_csr_t>(proc, CSR_TSELECT));
300 if (proc->get_cfg().trigger_count > 0) {
301     add_csr(CSR_TDATA1, std::make_shared<tdata1_csr_t>(proc, CSR_TDATA1));
302     add_csr(CSR_TDATA2, tdata2 = std::make_shared<tdata2_csr_t>(proc, CSR_TDATA2));
303     add_csr(CSR_TDATA3, std::make_shared<tdata3_csr_t>(proc, CSR_TDATA3));

```



```

304   add_csr(CSR_TINFO, std::make_shared<tinfo_csr_t>(proc,
CSR_TINFO));
305   if (!proc->extension_enabled_const('S')) {
306     add_csr(CSR_TCONTROL, tcontrol =
std::make_shared<masked_csr_t>(proc, CSR_TCONTROL, CSR_TCONTROL_MPTC
| CSR_TCONTROL_MTE, 0));
307   }
308   } else {
309     add_csr(CSR_TDATA1, std::make_shared<const_csr_t>(proc,
CSR_TDATA1, 0));
310     add_csr(CSR_TDATA2, tdata2 =
std::make_shared<const_csr_t>(proc, CSR_TDATA2, 0));
311     add_csr(CSR_TDATA3, std::make_shared<const_csr_t>(proc,
CSR_TDATA3, 0));
312     add_csr(CSR_TINFO, std::make_shared<const_csr_t>(proc,
CSR_TINFO, 0));
313   }
314   unsigned scontext_length = (xlen == 32 ? 16 : 32); // debug
spec suggests 16-bit for RV32 and 32-bit for RV64
315   add_supervisor_csr(CSR_SCONTEXT, scontext =
std::make_shared<masked_csr_t>(proc, CSR_SCONTEXT, (reg_t(1) <<
scontext_length) - 1, 0));
316   unsigned hcontext_length = (xlen == 32 ? 6 : 13) + (proc-
>extension_enabled('H') ? 1 : 0); // debug spec suggest 7-bit (6-
bit) for RV32 and 14-bit (13-bit) for RV64 with (without) H exten-
sion
317   auto hcontext = std::make_shared<masked_csr_t>(proc, CSR_H-
CONTEXT, (reg_t(1) << hcontext_length) - 1, 0);
318   add_hypervisor_csr(CSR_HCONTEXT, hcontext);
319   add_csr(CSR_MCONTEXT, mcontext =
std::make_shared<proxy_csr_t>(proc, CSR_MCONTEXT, hcontext));
320
321   mseccfg = std::make_shared<mseccfg_csr_t>(proc, CSR_MSEC-
CFG);
322   if (xlen == 32) {
323     add_csr(CSR_MSECCFG,
std::make_shared<rv32_low_csr_t>(proc, CSR_MSECCFG, mseccfg));
324     add_csr(CSR_MSECCFGH, mseccfgh =
std::make_shared<rv32_high_csr_t>(proc, CSR_MSECCFGH, mseccfg));
325   } else {
326     add_csr(CSR_MSECCFG, mseccfg);
327   }
328
329   for (int i = 0; i < max_pmp; ++i) {
330     add_csr(CSR_PMPADDR0 + i, pmpaddr[i] =
std::make_shared<pmpaddr_csr_t>(proc, CSR_PMPADDR0 + i, i));
331   }
332   for (int i = 0; i < max_pmp; i += xlen / 8) {
333     reg_t addr = CSR_PMPCFG0 + i / 4;
334     add_csr(addr, std::make_shared<pmpcfg_csr_t>(proc, addr));
335   }
336
337   add_csr(CSR_FFLAGS, fflags =
std::make_shared<float_csr_t>(proc, CSR_FFLAGS, FSR_AEXC >>
FSR_AEXC_SHIFT, 0));
338   add_csr(CSR_FRM, frm = std::make_shared<float_csr_t>(proc,
CSR_FRM, FSR_RD >> FSR_RD_SHIFT, 0));
339   assert(FSR_AEXC_SHIFT == 0); // composite_csr_t assumes
fflags begins at bit 0
340   add_csr(CSR_FCSR, std::make_shared<composite_csr_t>(proc,
CSR_FCSR, frm, fflags, FSR_RD_SHIFT));
341
342   if (proc->extension_enabled('P') || proc-
>any_vector_extensions()) {
343     add_csr(CSR_VXSAT, vxsat =
std::make_shared<vxsat_csr_t>(proc, CSR_VXSAT));
344   }
345
346   add_ext_csr(EXT_ZKR, CSR_SEED,
std::make_shared<seed_csr_t>(proc, CSR_SEED));
347
348   add_csr(CSR_MARCHID, std::make_shared<const_csr_t>(proc,
CSR_MARCHID, 5));
349   add_csr(CSR_MIMPID, std::make_shared<const_csr_t>(proc,
CSR_MIMPID, 0));
350   add_csr(CSR_MVENDORID, std::make_shared<const_csr_t>(proc,
CSR_MVENDORID, 0));
351   add_csr(CSR_MHARTID, std::make_shared<const_csr_t>(proc,
CSR_MHARTID, proc->get_id()));
352   add_csr(CSR_MCONFIGPTR, std::make_shared<const_csr_t>(proc,
CSR_MCONFIGPTR, 0));
353   const reg_t senvcfg_mask = (fiom_writable ? SENVCFG_FIOM :
0) |
354     (proc->exten-
sion_enabled(EXT_ZICBOM) ? SENVCFG_CBCE | SENVCFG_CBIE : 0) |
355     (proc->exten-
sion_enabled(EXT_ZICBOZ) ? SENVCFG_CBZE : 0) |
356     (proc->exten-
sion_enabled(EXT_SVUKTE) ? SENVCFG_UKTE : 0) |
357     (proc->exten-
sion_enabled(EXT_SSNPM) ? SENVCFG_PMM : 0) |
358     (proc->exten-
sion_enabled(EXT_ZICFILP) ? SENVCFG_LPE : 0) |
359     (proc->exten-
sion_enabled(EXT_ZICFISS) ? SENVCFG_SSE : 0);
360   add_supervisor_csr(CSR_SENVCFG, senvcfg =
std::make_shared<senvcfg_csr_t>(proc, CSR_SENVCFG, senvcfg_mask,
0));
361   const reg_t henvcfg_mask = HENVCFG_FIOM |
362     (proc->exten-
sion_enabled(EXT_ZICBOM) ? HENVCFG_CBCE | HENVCFG_CBIE : 0) |

```

```

363     (proc->exten-
sion_enabled(EXT_ZICBOZ) ? HENVCFG_CBZE : 0) |
364     (proc->exten-
sion_enabled(EXT_SSNPM) ? HENVCFG_PMM : 0) |
365     (proc->exten-
sion_enabled(EXT_SVADU) ? HENVCFG_ADUE : 0) |
366     (proc->exten-
sion_enabled(EXT_SVPBMT) ? HENVCFG_PBMT : 0) |
367     (proc->exten-
sion_enabled(EXT_SSTC) ? HENVCFG_STCE : 0) |
368     (proc->exten-
sion_enabled(EXT_ZICFILP) ? HENVCFG_LPE : 0) |
369     (proc->exten-
sion_enabled(EXT_ZICFISS) ? HENVCFG_SSE : 0) |
370     (proc->exten-
sion_enabled(EXT_SDBLTRP) ? HENVCFG_DTE : 0);
371   henvcfg = std::make_shared<henvcfg_csr_t>(proc, CSR_HENVCFG,
henvcfg_mask, 0, henvcfg);
372   if (xlen == 32) {
373     add_hypervisor_csr(CSR_HENVCFG,
std::make_shared<rv32_low_csr_t>(proc, CSR_HENVCFG, henvcfg));
374     add_hypervisor_csr(CSR_HENVCFGH, henvcfg));
375   } else {
376     add_hypervisor_csr(CSR_HENVCFG, henvcfg);
377   }
378   if (proc->extension_enabled_const(EXT_SMSTATEEN)) {
379     const reg_t sstateen0_mask = (proc->exten-
sion_enabled(EXT_ZFINX) ? SSTATEEN0_FCSR : 0) |
380     (proc->exten-
sion_enabled(EXT_ZCMT) ? SSTATEEN0_JVT : 0) |
381     SSTATEEN0_CS;
382     const reg_t hstateen0_mask = sstateen0_mask |
HSTATEEN0_SENVCFG | HSTATEEN0_SSTATEEN |
383     (proc->exten-
sion_enabled_const(EXT_SSCSRIND) ? HSTATEEN0_CSRIND : 0) |
384     (proc->exten-
sion_enabled_const(EXT_SSAIA) ? HSTATEEN0_IMSIC | HSTATEEN0_AIA :
0);
385     const reg_t mstateen0_mask = hstateen0_mask | (proc->ex-
tension_enabled(EXT_SSQOSID) ? MSTATEEN0_PRIV114 : 0);
386     for (int i = 0; i < 4; i++) {
387       const reg_t mstateen_mask = i == 0 ? mstateen0_mask :
MSTATEEN0_HSTATEEN;
388       mstateen[i] = std::make_shared<masked_csr_t>(proc,
CSR_MSTATEEN0 + i, mstateen_mask, 0);
389       if (xlen == 32) {
390         add_csr(CSR_MSTATEEN0 + i,
std::make_shared<rv32_low_csr_t>(proc, CSR_MSTATEEN0 + i,
mstateen[i]));
391         add_csr(CSR_MSTATEEN0H + i,
std::make_shared<rv32_high_csr_t>(proc, CSR_MSTATEEN0H + i,
mstateen[i]));
392       } else {
393         add_csr(CSR_MSTATEEN0 + i, mstateen[i]);
394       }
395
396       const reg_t hstateen_mask = i == 0 ? hstateen0_mask :
HSTATEEN0_SSTATEEN;
397       hstateen[i] = std::make_shared<hstateen_csr_t>(proc,
CSR_HSTATEEN0 + i, hstateen_mask, 0, i);
398       if (xlen == 32) {
399         add_hypervisor_csr(CSR_HSTATEEN0 + i,
std::make_shared<rv32_low_csr_t>(proc, CSR_HSTATEEN0 + i,
hstateen[i]));
400         add_hypervisor_csr(CSR_HSTATEEN0H + i,
std::make_shared<rv32_high_csr_t>(proc, CSR_HSTATEEN0H + i,
hstateen[i]));
401       } else {
402         add_hypervisor_csr(CSR_HSTATEEN0 + i, hstateen[i]);
403       }
404
405       const reg_t sstateen_mask = i == 0 ? sstateen0_mask : 0;
406       add_supervisor_csr(CSR_SSTATEEN0 + i, sstateen[i] =
std::make_shared<sstateen_csr_t>(proc, CSR_SSTATEEN0 + i,
sstateen_mask, 0, i));
407     }
408   }
409
410   if (proc->extension_enabled_const(EXT_SMRNMI)) {
411     add_csr(CSR_MNSCRATCH, std::make_shared<basic_csr_t>(proc,
CSR_MNSCRATCH, 0));
412     add_csr(CSR_MNEPC, mnepc =
std::make_shared<epc_csr_t>(proc, CSR_MNEPC));
413     add_csr(CSR_MNCAUSE, std::make_shared<const_csr_t>(proc,
CSR_MNCAUSE, (reg_t)1 << (xlen - 1)));
414     add_csr(CSR_MNSTATUS, mnstatus =
std::make_shared<mnstatus_csr_t>(proc, CSR_MNSTATUS));
415   }
416
417   if (proc->extension_enabled_const(EXT_SSTC)) {
418     stimecmp = std::make_shared<stimecmp_csr_t>(proc, CSR_S-
TIMECMP, MIP_STIP);
419     vstimecmp = std::make_shared<stimecmp_csr_t>(proc, CSR_VS-
TIMECMP, MIP_VSTIP);
420     auto virtualized_stimecmp =
std::make_shared<virtualized_with_special_permission_csr_t>(proc,
stimecmp, vstimecmp);
421     if (xlen == 32) {

```

```

422     add_supervisor_csr(CSR_STIMECMP,
std::make_shared<rv32_low_csr_t>(proc, CSR_STIMECMP, virtual-
ized_stimecmp));
423     add_supervisor_csr(CSR_STIMECMPI,
std::make_shared<rv32_high_csr_t>(proc, CSR_STIMECMPI, virtual-
ized_stimecmp));
424     add_hypervisor_csr(CSR_VSTIMECMP,
std::make_shared<rv32_low_csr_t>(proc, CSR_VSTIMECMP, vstimecmp));
425     add_hypervisor_csr(CSR_VSTIMECMPI,
std::make_shared<rv32_high_csr_t>(proc, CSR_VSTIMECMPI, vstimecmp));
426 } else {
427     add_supervisor_csr(CSR_STIMECMP, virtualized_stimecmp);
428     add_hypervisor_csr(CSR_VSTIMECMP, vstimecmp);
429 }
430 }
431
432 add_ext_csr(EXT_ZCMT, CSR_JVT, jvt =
std::make_shared<jvt_csr_t>(proc, CSR_JVT, 0));
433
434 const reg_t ssp_mask = -reg_t(xlen / 8);
435 add_ext_csr(EXT_ZICFISS, CSR_SSP, ssp =
std::make_shared<ssp_csr_t>(proc, CSR_SSP, ssp_mask, 0));
436
437 // Smcdeleg
438 if (proc->extension_enabled_const(EXT_SMCDELEG) || proc->ex-
tension_enabled_const(EXT_SSOCFG)) {
439     add_supervisor_csr(CSR_SCOUNTINHIBIT, scountinhibit =
std::make_shared<scntinhibit_csr_t>(proc, CSR_SCOUNTINHIBIT,
mcountinhibit));
440 }
441
442 // Smcsrind / Sscsrind
443 if (proc->extension_enabled_const(EXT_SMCSRIND)) {
444     csr_t_p miselect = std::make_shared<basic_csr_t>(proc,
CSR_MISELECT, 0);
445     add_csr(CSR_MISELECT, miselect);
446
447     add_csr(CSR_MIREG, mireg[0] =
std::make_shared<sscsrind_reg_csr_t>(proc, CSR_MIREG, miselect));
448     const reg_t mireg_csrs[] = { CSR_MIREG2, CSR_MIREG3,
CSR_MIREG4, CSR_MIREG5, CSR_MIREG6 };
449     for (auto csr : mireg_csrs)
450         add_csr(CSR_MIREG, mireg[csr - CSR_MIREG] =
std::make_shared<sscsrind_reg_csr_t>(proc, csr, miselect));
451
452     if (proc->extension_enabled_const(EXT_SMAIA)) {
453         // *ireg needs to be a proxy of *iselect and a CSR map
keyed by the value in *iselect
454         // If address not in CSR map, throw illegal or virtual
instruction trap
455         auto aia_mireg =
std::make_shared<aia_ireg_proxy_csr_t>(proc, CSR_MIREG, miselect);
456         for (auto &csr : *aia_mireg->get_regmap())
457             mireg[0] += add_ireg_proxy(csr.first, aia_mireg);
458         add_ireg_proxy(mireg[0], aia_mireg);
459     }
460 }
461
462 if (proc->extension_enabled_const(EXT_SSCSRIND)) {
463     csr_t_p vsiselect = std::make_shared<vsiselect_csr_t>(proc,
CSR_VSISELECT, 0);
464     add_hypervisor_csr(CSR_VSISELECT, vsiselect);
465
466     csr_t_p siselect = std::make_shared<siselect_csr_t>(proc,
CSR_SISELECT, 0);
467     add_supervisor_csr(CSR_SISELECT,
std::make_shared<virtualized_with_special_permission_csr_t>(proc,
siselect, vsiselect));
468
469     const reg_t vsireg_csrs[] = { CSR_VSIREG, CSR_VSIREG2,
CSR_VSIREG3, CSR_VSIREG4, CSR_VSIREG5, CSR_VSIREG6 };
470     for (size_t i = 0; i < std::size(vsireg_csrs); i++) {
471         add_hypervisor_csr(vsireg_csrs[i], vsireg[i] =
std::make_shared<sscsrind_reg_csr_t>(proc, vsireg_csrs[i], vsire-
lect));
472     }
473
474     auto sireg = nonvirtual_sireg[0] =
std::make_shared<sscsrind_reg_csr_t>(proc, CSR_SIREG, siselect);
475     add_supervisor_csr(CSR_SIREG,
std::make_shared<virtualized_indirect_csr_t>(proc, nonvirtu-
al_sireg[0], vsireg[0]));
476     if (proc->extension_enabled(EXT_SSOCFG) || proc->exten-
sion_enabled(EXT_SMCDELEG)) {
477         // case CSR_SIREG
478         if (proc->extension_enabled_const(EXT_ZICNTR)) {
479             sireg->add_ireg_proxy(SISELECT_SMCDELEG_START,
csrmap[CSR_CYCLE]);
480             sireg->add_ireg_proxy(SISELECT_SMCDELEG_INSTRET,
csrmap[CSR_INSTRET]);
481         }
482         if (proc->extension_enabled_const(EXT_ZIHPM)) {
483             for (size_t j = 0; j < (SISELECT_SMCDELEG_END - SISE-
LECT_SMCDELEG_HPM_COUNTER_3 + 1); j++)
484                 sireg->add_ireg_proxy(SISELECT_SMCDELEG_HPM_COUNTER_3 +
j, csrmap[CSR_HPM_COUNTER_3 + j]);
485         }
486     }
487 }

```

```

488     const reg_t sireg_csrs[] = { CSR_SIREG2, CSR_SIREG3,
CSR_SIREG4, CSR_SIREG5, CSR_SIREG6 };
489     for (size_t i = 0; i < std::size(sireg_csrs); i++) {
490         auto ireg_idx = sireg_csrs[i] - CSR_SIREG;
491         auto sireg = nonvirtual_sireg[ireg_idx] =
std::make_shared<sscsrind_reg_csr_t>(proc, sireg_csrs[i], siselect);
492         add_supervisor_csr(sireg_csrs[i],
std::make_shared<virtualized_indirect_csr_t>(proc, nonvirtu-
al_sireg[ireg_idx], vsireg[ireg_idx]));
493
494         // SMCdeleg
495         if (proc->extension_enabled(EXT_SSOCFG) || proc->exten-
sion_enabled(EXT_SMCDELEG)) {
496             switch (sireg_csrs[i]) {
497                 case CSR_SIREG4:
498                     if (xlen == 32) {
499                         if (proc->extension_enabled_const(EXT_ZICNTR)) {
500                             sireg->add_ireg_proxy(SISELECT_SMCDELEG_START,
csrmap[CSR_CYCLEH]);
501                             sireg->add_ireg_proxy(SISELECT_SMCDELEG_INSTRET,
csrmap[CSR_INSTRET]);
502                         }
503                         if (proc->extension_enabled_const(EXT_ZIHPM)) {
504                             for (size_t j = 0; j < (SISELECT_SMCDELEG_END -
SISELECT_SMCDELEG_HPM_COUNTER_3 + 1); j++)
505                                 sireg-
>add_ireg_proxy(SISELECT_SMCDELEG_HPM_COUNTER_3 + j,
csrmap[CSR_HPM_COUNTER_3H + j]);
506                         }
507                     }
508                     break;
509                 case CSR_SIREG2:
510                     if (proc->extension_enabled_const(EXT_ZICNTR)) {
511                         // describe address as that of CYCLE rather than
MCYCLECFG,
512                         // so that perm checks and mcounteren checks do
the right thing
513                         auto cyclecfg_proxy =
std::make_shared<counter_proxy_csr_t>(proc, CSR_CYCLE,
csrmap[CSR_MCYCLECFG]);
514                         sireg->add_ireg_proxy(SISELECT_SMCDELEG_START,
cyclecfg_proxy);
515
516                         auto instretcfg_proxy =
std::make_shared<counter_proxy_csr_t>(proc, CSR_INSTRET,
csrmap[CSR_MINSTRETCFG]);
517                         sireg-
>add_ireg_proxy(SISELECT_SMCDELEG_INSTRETCFG, instretcfg_proxy);
518                     }
519                     if (proc->extension_enabled_const(EXT_ZIHPM)) {
520                         for (size_t j = 0; j < (SISELECT_SMCDELEG_END -
SISELECT_SMCDELEG_HPM_EVENT_3 + 1); j++) {
521                             auto proxy =
std::make_shared<counter_proxy_csr_t>(proc, CSR_HPM_COUNTER_3 + j,
csrmap[CSR_MHPM_EVENT_3 + j]);
522                             sireg-
>add_ireg_proxy(SISELECT_SMCDELEG_HPM_EVENT_3 + j, proxy);
523                         }
524                     }
525                     break;
526                 case CSR_SIREG5:
527                     if (xlen == 32) {
528                         // cyclecfg/instretcfg
529
530                         if (proc->extension_enabled_const(EXT_ZICNTR) &&
proc->extension_enabled_const(EXT_SMCNTRPMF)) {
531                             auto cyclecfgh_proxy =
std::make_shared<counter_proxy_csr_t>(proc, CSR_CYCLEH,
csrmap[CSR_MCYCLECFGH]);
532                             sireg->add_ireg_proxy(SISELECT_SMCDELEG_START,
cyclecfgh_proxy);
533
534                             auto instretcfgh_proxy =
std::make_shared<counter_proxy_csr_t>(proc, CSR_INSTRETH,
csrmap[CSR_MINSTRETCFHH]);
535                             sireg-
>add_ireg_proxy(SISELECT_SMCDELEG_INSTRETH, instretcfgh_proxy);
536                         }
537                         // hpmevent3h-hpmevent31h
538                         if (proc->extension_enabled_const(EXT_ZIHPM) &&
proc->extension_enabled_const(EXT_SSOCFPMF)) {
539                             for (size_t j = 0; j < (SISELECT_SMCDELEG_END -
SISELECT_SMCDELEG_HPM_EVENT_3); j++) {
540                                 auto proxy =
std::make_shared<counter_proxy_csr_t>(proc, CSR_HPM_COUNTER_3H + j,
csrmap[CSR_MHPM_EVENT_3H + j]);
541                                 sireg-
>add_ireg_proxy(SISELECT_SMCDELEG_HPM_EVENT_3 + j, proxy);
542                             }
543                         }
544                     }
545                     case CSR_SIREG3:
546                     case CSR_SIREG6:
547                     default:
548                         break;
549                 }
550             }
551         }
552     }

```

```

551
552 if (proc->extension_enabled_const(EXT_SSAIA)) {
553     auto aia_vsireg =
std::make_shared<aia_ireg_proxy_csr_t>(proc, CSR_VSIREG, vsiselect);
554     // csrmaps of vs files are the same as vsein = 1
555     auto *csrmap = aia_vsireg->get_regmap(1);
556     if (csrmap) {
557         for (auto &csr : *csrmap)
558             vsireg[0]->add_ireg_proxy(csr.first, aia_vsireg);
559         add_ireg_proxy(vsireg[0], aia_vsireg);
560     }
561
562     auto aia_sireg =
std::make_shared<aia_ireg_proxy_csr_t>(proc, CSR_SIREG, siselect);
563     for (auto &csr : *aia_sireg->get_regmap()) {
564         sireg->add_ireg_proxy(csr.first, aia_sireg);
565         add_ireg_proxy(sireg, aia_sireg);
566     }
567 }
568
569 const reg_t srmcfg_mask = SRMCFG_MCID | SRMCFG_RCID;
570 add_const_ext_csr(EXT_SSQOSID, CSR_SRMCFG,
std::make_shared<srmcfg_csr_t>(proc, CSR_SRMCFG, srmcfg_mask, 0));
571
572 mvien = std::make_shared<masked_csr_t>(proc, CSR_MVIEN,
MIP_SEIP | MIP_SSIP, 0);
573 mvip = std::make_shared<mvip_csr_t>(proc, CSR_MVIP, 0);
574 if (proc->extension_enabled_const(EXT_SMAIA)) {
575     add_csr(CSR_MTOPI, std::make_shared<mtopi_csr_t>(proc,
CSR_MTOPI));
576     add_csr(CSR_MTOPEI, std::make_shared<topei_csr_t>(proc,
CSR_MTOPEI, proc->imsic->m));
577     if (xlen == 32) {
578         add_supervisor_csr(CSR_MVIEN,
std::make_shared<rv32_low_csr_t>(proc, CSR_MVIEN, mvien));
579         add_supervisor_csr(CSR_MVIENH,
std::make_shared<rv32_high_csr_t>(proc, CSR_MVIENH, mvien));
580         add_supervisor_csr(CSR_MVIP,
std::make_shared<rv32_low_csr_t>(proc, CSR_MVIP, mvip));
581         add_supervisor_csr(CSR_MVIPH,
std::make_shared<rv32_high_csr_t>(proc, CSR_MVIPH, mvip));
582     } else {
583         add_supervisor_csr(CSR_MVIEN, mvien);
584         add_supervisor_csr(CSR_MVIP, mvip);
585     }
586 }
587
588 hvictl = std::make_shared<aia_csr_t>(proc, CSR_HVICTL,
HVICTL_VTI | HVICTL_IID | HVICTL_DPR | HVICTL_IPRIOM | HVICTL_IPRIO,
0);
589 vstopi = std::make_shared<vstopi_csr_t>(proc, CSR_VSTOPI);
590 if (proc->extension_enabled_const(EXT_SSAIA)) { // Included
by EXT_SMAIA
591     csr_t_p nonvirtual_stopi =
std::make_shared<nonvirtual_stopi_csr_t>(proc, CSR_STOPI);
592     add_supervisor_csr(CSR_STOPI,
std::make_shared<virtualized_with_special_permission_csr_t>(proc,
nonvirtual_stopi, vstopi));
593     auto vstopi = std::make_shared<vstopi_csr_t>(proc,
CSR_VSTOPEI);
594     auto nonvirtual_stopi =
std::make_shared<nonvirtual_stopi_csr_t>(proc, CSR_STOPEI, proc-
>imsic->s);
595     add_supervisor_csr(CSR_STOPEI,
std::make_shared<virtualized_with_special_permission_csr_t>(proc,
nonvirtual_stopi, vstopi));
596     auto hvien = std::make_shared<aia_csr_t>(proc, CSR_HVIEN,
0, 0);
597     auto hviprio1 = std::make_shared<aia_csr_t>(proc, CSR_H-
VIPRIO1, 0, 0);
598     auto hviprio2 = std::make_shared<aia_csr_t>(proc, CSR_H-
VIPRIO2, 0, 0);
599     if (xlen == 32) {
600         add_hypervisor_csr(CSR_HVIEN,
std::make_shared<rv32_low_csr_t>(proc, CSR_HVIEN, hvien));
601         add_hypervisor_csr(CSR_HVIENH,
std::make_shared<rv32_high_csr_t>(proc, CSR_HVIENH, hvien));
602         add_hypervisor_csr(CSR_HVIPRIO1,
std::make_shared<rv32_low_csr_t>(proc, CSR_HVIPRIO1, hviprio1));
603         add_hypervisor_csr(CSR_HVIPRIO1H,
std::make_shared<rv32_high_csr_t>(proc, CSR_HVIPRIO1H, hviprio1));
604         add_hypervisor_csr(CSR_HVIPRIO2,
std::make_shared<rv32_low_csr_t>(proc, CSR_HVIPRIO2, hviprio2));
605         add_hypervisor_csr(CSR_HVIPRIO2H,
std::make_shared<rv32_high_csr_t>(proc, CSR_HVIPRIO2H, hviprio2));
606     } else {
607         add_hypervisor_csr(CSR_HVIEN, hvien);
608         add_hypervisor_csr(CSR_HVIPRIO1, hviprio1);
609         add_hypervisor_csr(CSR_HVIPRIO2, hviprio2);
610     }
611     add_hypervisor_csr(CSR_HVICTL, hvictl);
612     add_hypervisor_csr(CSR_VSTOPI, vstopi);
613     add_hypervisor_csr(CSR_VSTOPEI, vstopi);
614 }
615
616 if (proc->extension_enabled_const(EXT_SMPMPDELEG)) {
617     add_csr(CSR_MPMPEDELEG, mpmpeleg =
std::make_shared<mpmpeleg_csr_t>(proc, CSR_MPMPEDELEG, MPMP-
DELEG_PMPNUM, proc->n_pmp));
618 }
619

```

```

620 if (proc->extension_enabled_const(EXT_SSPMP)) {
621     const reg_t spmp_cfg_mask = (SPMP_SHARED | SPMP_U | PMP_L
| PMP_A | PMP_X | PMP_W | PMP_R);
622     for (size_t i = 0; i < max_pmp; ++i) {
623         csr_t_p cfg = std::make_shared<spmpcfg_csr_t>(proc, i,
spmp_cfg_mask, 0);
624         mireg[1]->add_ireg_proxy(0x100+i, cfg);
625         nonvirtual_sireg[1]->add_ireg_proxy(0x100+i, cfg);
626     }
627
628     // add entry registers (index: 0x100..0x13F)
629     proc->set_spmp_addr_entry();
630
631     if (proc->extension_enabled_const(EXT_SSPMPEN)) {
632         spmpen = std::make_shared<spmpen_csr_t>(proc, CSR_SPM-
PEN, 0);
633         if (xlen == 32) {
634             add_supervisor_csr(CSR_SPMEN,
std::make_shared<rv32_low_csr_t>(proc, CSR_SPMEN, spmpen));
635             add_supervisor_csr(CSR_SPMENH,
std::make_shared<rv32_high_csr_t>(proc, CSR_SPMENH, spmpen));
636         } else {
637             add_supervisor_csr(CSR_SPMEN, spmpen);
638         }
639     }
640 }
641 }

```

15. riscv-software-src_riscv-isa-sim/riscv/csrs.cc (2,615 lines)

```

1 // See LICENSE for license details.
2
3 // For std::any_of
4 #include <algorithm>
5
6 #include "csrs.h"
7 // For processor_t:
8 #include "processor.h"
9 #include "mmu.h"
10 // For get_field():
11 #include "decode_macros.h"
12 // For trap_virtual_instruction and trap_illegal_instruction:
13 #include "trap.h"
14 // For require():
15 #include "insn_macros.h"
16 // For CSR_DCSR_V:
17 #include "debug_defines.h"
18 // For ctz:
19 #include "arith.h"
20
21 // STATE macro used by require_privilege() macro:
22 #undef STATE
23 #define STATE (*state)
24
25 // implement class csr_t
26 csr_t::csr_t(processor_t* const proc, const reg_t addr):
27     proc(proc),
28     state(proc->get_state()),
29     address(addr),
30     csr_priv(get_field(addr, 0x300)),
31     csr_read_only(get_field(addr, 0xC00) == 3) {
32 }
33
34 void csr_t::verify_permissions(insn_t insn, bool write) const
{
35 // Check permissions. Raise virtual-instruction exception if
V=1,
36 // privileges are insufficient, and the CSR belongs to su-
pervisor or
37 // hypervisor. Raise illegal-instruction exception other-
wise.
38 unsigned priv = state->prv == PRV_S && !state->v ? PRV_HS :
state->prv;
39
40 if ((csr_priv == PRV_S && !proc->extension_enabled('S')) ||
(csr_priv == PRV_HS && !proc->extension_enabled('H')))
41     throw trap_illegal_instruction(insn.bits());
42
43 if (write && csr_read_only)
44     throw trap_illegal_instruction(insn.bits());
45 if (priv < csr_priv) {
46     if (state->v && csr_priv <= PRV_HS)
47         throw trap_virtual_instruction(insn.bits());
48     throw trap_illegal_instruction(insn.bits());
49 }
50 }
51
52 csr_t::~csr_t() {
53 }
54
55 void csr_t::write(const reg_t val) noexcept {
56     const bool success = unlogged_write(val);
57     if (success) {
58         log_write();
59     }
60 }
61
62 void csr_t::log_write() const noexcept {
63     log_special_write(address, written_value());
64 }

```

```

65 }
66
67 void csr_t::log_special_write(const reg_t UNUSED address,
const reg_t UNUSED val) const noexcept {
68     if (proc->get_log_commits_enabled())
69         proc->get_state()->log_reg_write(((address) << 4) | 4] =
{val, 0};
70 }
71
72 reg_t csr_t::written_value() const noexcept {
73     return read();
74 }
75
76 // implement class basic_csr_t
77 basic_csr_t::basic_csr_t(processor_t* const proc, const reg_t
addr, const reg_t init):
78     csr_t(proc, addr),
79     val(init) {
80 }
81
82 bool basic_csr_t::unlogged_write(const reg_t val) noexcept {
83     this->val = val;
84     return true;
85 }
86
87 // implement class base_pmpaddr_csr_t
88 base_pmpaddr_csr_t::base_pmpaddr_csr_t(processor_t* const
proc, const reg_t addr, const size_t idx):
89     csr_t(proc, addr),
90     val(0),
91     cfg(0),
92     pmpidx(idx) {
93 }
94
95 bool base_pmpaddr_csr_t::next_locked_and_tor() const noexcept
{
96     if (pmpidx+1 >= state->max_pmp) return false; // this is
the last entry
97     const bool lock_bypass = state->mseccfg->get_rlb();
98     const bool next_locked = !lock_bypass && (state->pmpad-
dr[pmpidx+1]->cfg & PMP_L);
99     const bool next_tor = (state->pmpaddr[pmpidx+1]->cfg &
PMP_A) == PMP_TOR;
100     return next_locked && next_tor;
101 }
102
103 reg_t base_pmpaddr_csr_t::tor_paddr() const noexcept {
104     return (val & proc->pmp_tor_mask()) << PMP_SHIFT;
105 }
106
107 reg_t base_pmpaddr_csr_t::tor_base_paddr() const noexcept {
108     if (pmpidx == 0) return 0; // entry 0 always uses 0 as base
109     return state->pmpaddr[pmpidx-1]->tor_paddr();
110 }
111
112 reg_t base_pmpaddr_csr_t::napot_mask() const noexcept {
113     bool is_na4 = (cfg & PMP_A) == PMP_NA4;
114     reg_t mask = (val << 1) | (!is_na4) | ~proc->pmp_tor_mask();
115     return ~(mask & ~(mask + 1)) << PMP_SHIFT;
116 }
117
118 bool base_pmpaddr_csr_t::match4(reg_t addr) const noexcept {
119     if (proc->extension_enabled_const(EXT_SSMPEN) && (pmpidx >=
proc->n_pmp)) {
120         if (!((state->spmpen->read() >> (pmpidx - proc->n_pmp)) &
1))
121             return false;
122     }
123
124     if ((cfg & PMP_A) == 0) return false;
125     bool is_tor = (cfg & PMP_A) == PMP_TOR;
126     if (is_tor) return tor_base_paddr() <= addr && addr <
tor_paddr();
127     // NAPOT or NA4:
128     return ((addr ^ tor_paddr()) & napot_mask()) == 0;
129 }
130
131 bool base_pmpaddr_csr_t::subset_match(reg_t addr, reg_t len)
const noexcept {
132     if (proc->extension_enabled_const(EXT_SSMPEN) && (pmpidx >=
proc->n_pmp)) {
133         if (!((state->spmpen->read() >> (pmpidx - proc->n_pmp)) &
1))
134             return false;
135     }
136
137     if ((addr | len) & (len - 1))
138         abort();
139     reg_t base = tor_base_paddr();
140     reg_t tor = tor_paddr();
141
142     if ((cfg & PMP_A) == 0) return false;
143
144     bool is_tor = (cfg & PMP_A) == PMP_TOR;
145     bool begins_after_lower = addr >= base;
146     bool begins_after_upper = addr >= tor;
147     bool ends_before_lower = (addr & -len) < (base & -len);
148     bool ends_before_upper = (addr & -len) < (tor & -len);
149     bool tor_homogeneous = ends_before_lower || be-

```

```

gins_after_upper ||
150     (begins_after_lower && ends_before_upper);
151
152     bool mask_homogeneous = ~(napot_mask() << 1) & len;
153
154     bool napot_homogeneous = mask_homogeneous || ((addr ^ tor) /
len) != 0;
155     return !(is_tor ? tor_homogeneous : napot_homogeneous);
156 }
157
158 pmpaddr_csr_t::pmpaddr_csr_t(processor_t* const proc, const
reg_t addr, const size_t idx):
159     base_pmpaddr_csr_t(proc, addr, idx) {
160 }
161
162 void pmpaddr_csr_t::verify_permissions(insn_t insn, bool
write) const {
163     csr_t::verify_permissions(insn, write);
164
165     // If n_pmp is zero, that means pmp is not implemented hence
raise
166     // trap if it tries to access the csr. I would prefer to im-
plement
167     // this by not instantiating any base_pmpaddr_csr_t for
these regs, but
168     // n_pmp can change after reset() is run.
169     if (proc->n_pmp == 0)
170         throw trap_illegal_instruction(insn.bits());
171
172     reg_t pmpaddr_csr_t::read() const noexcept {
173         if (pmpidx >= proc->n_pmp)
174             return 0;
175
176         if ((cfg & PMP_A) >= PMP_NAPOT)
177             return val | (~proc->pmp_tor_mask() >> 1);
178         return val & proc->pmp_tor_mask();
179     }
180
181     bool pmpaddr_csr_t::unlogged_write(const reg_t val) noexcept {
182         // If no PMPs are configured, disallow access to all. Other-
wise,
183         // allow access to all, but unimplemented ones are hardwired
to
184         // zero. Note that n_pmp can change after reset(); otherwise I
would
185         // implement this in state_t::reset() by instantiating the
correct
186         // number of base_pmpaddr_csr_t.
187         if (proc->n_pmp == 0)
188             return false;
189
190         if (pmpidx >= proc->n_pmp)
191             return true;
192
193         const bool lock_bypass = state->mseccfg->get_rlb();
194         const bool locked = !lock_bypass && (cfg & PMP_L);
195
196         if (pmpidx >= proc->n_pmp || locked ||
next_locked_and_tor())
197             return false;
198
199         this->val = val & ((reg_t(1) << (proc->paddr_bits() -
PMP_SHIFT)) - 1);
200         proc->get_mmu()->flush_tlb();
201         return true;
202     }
203
204     bool pmpaddr_csr_t::access_ok(access_type type, reg_t mode,
bool hlvx) const noexcept {
205         const bool cfgx = cfg & PMP_X;
206         const bool cfgw = cfg & PMP_W;
207         const bool cfr = cfg & PMP_R;
208         const bool cfl = cfg & PMP_L;
209
210         const bool prvm = mode == PRV_M;
211
212         const bool typer = type == LOAD;
213         const bool typex = type == FETCH;
214         const bool typew = type == STORE;
215         const bool normal_rwx = (typer && cfr && (!hlvx || cfgx))
|| (typex && cfgw) || (typew && cfl);
216         const bool mseccfg_mml = state->mseccfg->get_mml();
217
218         if (mseccfg_mml) {
219             if (cfgx && cfgw && cfr && cfl) {
220                 // Locked Shared data region: Read only on both M and S/
U mode.
221                 return typer && !hlvx;
222             } else {
223                 const bool mml_shared_region = !cfr && cfgw;
224                 const bool mml_chk_normal = (prvm == cfl) && nor-
mal_rwx;
225                 const bool mml_chk_shared =
(!cfl && cfgx && ((typer && !hlvx) || typew))
||
226                 (!cfl && !cfgx && ((typer && !hlvx) || (typew
&& prvm))) ||

```



```

228         (cfgl && typex) ||
229         (cfgl && typer && cfgx && prvm);
230     return mml_shared_region ? mml_chk_shared : mml_chk_nor-
mal;
231 }
232 } else {
233     const bool m_bypass = (prvm && !cfgl);
234     return m_bypass || normal_rwx;
235 }
236 }
237
238 // implement class pmpcfg_csr_t
239 pmpcfg_csr_t::pmpcfg_csr_t(processor_t* const proc, const
reg_t addr):
240     csr_t(proc, addr) {
241 }
242
243 void pmpcfg_csr_t::verify_permissions(insn_t insn, bool write)
const {
244     csr_t::verify_permissions(insn, write);
245
246     // If n_pmp is zero, that means pmp is not implemented hence
raise
247     // trap if it tries to access the csr. I would prefer to im-
plement
248     // this by not instantiating any pmpcfg_csr_t for these
regs, but
249     // n_pmp can change after reset() is run.
250     if (proc->n_pmp == 0)
251         throw trap_illegal_instruction(insn.bits());
252 }
253 reg_t pmpcfg_csr_t::read() const noexcept {
254     reg_t cfg_res = 0;
255
256     for (size_t i0 = (address - CSR_PMPCFG0) * 4, i = i0; i < i0 +
proc->get_xlen() / 8 && i < state->max_pmp; i++)
257         cfg_res |= reg_t(state->pmpaddr[i]->cfg) << (8 * (i -
i0));
258     return cfg_res;
259 }
260 bool pmpcfg_csr_t::unlogged_write(const reg_t val) noexcept {
261     if (proc->n_pmp == 0)
262         return false;
263
264     bool write_success = false;
265     const bool rlb = state->mseccfg->get_rlb();
266     const bool mml = state->mseccfg->get_mml();
267
268     for (size_t i0 = (address - CSR_PMPCFG0) * 4, i = i0; i < i0 +
proc->get_xlen() / 8; i++) {
269         if (i < proc->n_pmp) {
270             const bool locked = (state->pmpaddr[i]->cfg & PMP_L);
271             if (rlb || !locked) {
272                 uint8_t all_cfg_fields = (PMP_R | PMP_W | PMP_X |
PMP_A |
273                 (proc->extension_enabled(EXT_SMPMPMT) ? PMP_MT :
0)) |
274                 PMP_L;
275                 uint8_t cfg = (val >> (8 * (i - i0))) &
all_cfg_fields;
276                 // Drop R=0 W=1 when MML = 0
277                 // Remove the restriction when MML = 1
278                 if (!mml) {
279                     cfg &= ~PMP_W | ((cfg & PMP_R) ? PMP_W : 0);
280                 }
281                 // Disallow A=NA4 when granularity > 4
282                 if (proc->lg_pmp_granularity != PMP_SHIFT && (cfg &
PMP_A) == PMP_NA4)
283                     cfg |= PMP_NAPOT;
284                 // MT value 0x3 is reserved
285                 if (get_field(cfg, PMP_MT) == 0x3)
286                     cfg = set_field(cfg, PMP_MT, 0);
287                 /*
288                  * Adding a rule with executable privileges that
289                  * either is M-mode-only or a locked Shared-Region
290                  * is not possible and such pmpcfg writes are ignored,
291                  * leaving pmpcfg unchanged.
292                  * This restriction can be temporarily lifted e.g.
293                  * during the boot process, by setting mseccfg.RLB.
294                  */
295                 const bool cfgx = cfg & PMP_X;
296                 const bool cfgw = cfg & PMP_W;
297                 const bool cfgr = cfg & PMP_R;
298
299                 if (rlb || !(mml && ((cfg & PMP_L) // M-mode-only
or a locked Shared-Region
300                 && !(cfgx && cfgw && cfgr) // RWX = 111
is allowed
301                 && (cfgx || (cfgw && !cfgr)) // X=1 or
RW=01 is not allowed
302                 ))) {
303                     state->pmpaddr[i]->cfg = cfg;
304                 }
305             }
306             write_success = true;
307         }
308     }
309 }

```

```

304     proc->get_mmu()->flush_tlb();
305     return write_success;
306 }
307
308 mmpmpdeleg_csr_t::mmpmpdeleg_csr_t(processor_t* const proc,
const reg_t addr, const reg_t mask, const reg_t init):
309     masked_csr_t(proc, addr, mask, init) {
310 }
311
312 bool mmpmpdeleg_csr_t::unlogged_write(const reg_t val) noexcept {
313     int max_locked_pmp_index = -1;
314     for (reg_t i = 0; i < proc->n_pmp; ++i) {
315         if (state->pmpaddr[i]->is_locked())
316             max_locked_pmp_index = i;
317     }
318
319     reg_t new_val = val & MPMPELEG_PMPNUM;
320     if (new_val > state->max_pmp)
321         new_val = state->max_pmp;
322
323     bool write_success = false;
324     if (max_locked_pmp_index < (int)new_val) {
325         for (size_t i = 0; i < proc->n_pmp; ++i)
326             state->csmap.erase(CSR_PMPADDR0 + i);
327
328         proc->n_pmp = new_val;
329         for (size_t i = 0; i < proc->n_pmp; ++i) {
330             state->add_csr(CSR_PMPADDR0 + i, state->pmpaddr[i] =
std::make_shared<pmpaddr_csr_t>(proc, CSR_PMPADDR0 + i, i));
331         }
332         proc->set_smpaddr_entry();
333         write_success = masked_csr_t::unlogged_write(new_val);
334     }
335
336     return write_success;
337 }
338
339 spmpaddr_csr_t::spmpaddr_csr_t(processor_t* const proc, const
reg_t addr):
340     base_pmpaddr_csr_t(proc, addr, addr) {
341 }
342
343 reg_t spmpaddr_csr_t::read() const noexcept {
344     if ((cfg & PMP_A) >= PMP_NAPOT)
345         return val | (~proc->pmp_tor_mask() >> 1);
346     return val & proc->pmp_tor_mask();
347 }
348
349 bool spmpaddr_csr_t::unlogged_write(const reg_t val) noexcept
{
350     if ((state->prv != PRV_M && (cfg & PMP_L)) ||
next_locked_and_tor())
351         return false;
352
353     this->val = val & ((reg_t(1) << (proc->paddr_bits() -
PMP_SHIFT)) - 1);
354     proc->get_mmu()->flush_tlb();
355     return true;
356 }
357
358 bool spmpaddr_csr_t::access_ok(access_type type, reg_t mode,
bool) const noexcept {
359     const bool cfgx = cfg & PMP_X;
360     const bool cfgw = cfg & PMP_W;
361     const bool cfgr = cfg & PMP_R;
362     const bool cfgu = cfg & SPMP_U;
363     const bool cfigs = cfg & SPMP_SHARED;
364
365     const bool prvs = mode == PRV_S;
366     const bool prvu = mode == PRV_U;
367
368     const bool typer = type == LOAD;
369     const bool typex = type == FETCH;
370     const bool typew = type == STORE;
371
372     const bool sum = STATE.sstatus->read() & SSTATUS_SUM;
373
374     const bool reserved = !cfgr && cfgr;
375
376     const bool deny = !cfigs && ((cfgu && prvs && !sum) || (!cfgu
&& prvu));
377
378     bool enforce_no_x(false), share_read_only(false),
share_exec_only(false);
379     if (cfgu) {
380         if (!cfigs) {
381             if (prvs && sum) enforce_no_x = true;
382         } else if (cfgr && cfgr) {
383             if (cfgr) share_exec_only = true;
384             else share_read_only = true;
385         }
386     }
387     const bool normal_rwx = (typer && cfgr && !share_exec_only)
||
388     (typew && cfgr && !share_read_only
&& !share_exec_only) ||
389     (typex && cfgr && !share_read_only
&& !enforce_no_x);
390 }

```

```

391 return !reserved && !deny && normal_rwx;
392 }
393
394 spmpcfg_csr_t::spmpcfg_csr_t(processor_t* const proc, const
reg_t addr, const reg_t mask, const reg_t init):
395 masked_csr_t(proc, addr, mask, init) {
396 }
397
398 reg_t spmpcfg_csr_t::read() const noexcept {
399 const size_t idx = proc->n_pmp + address;
400 if (idx >= state->max_pmp) return 0;
401 else return masked_csr_t::read();
402 }
403
404 bool spmpcfg_csr_t::unlogged_write(const reg_t val) noexcept {
405 const size_t idx = proc->n_pmp + address;
406 if (idx >= state->max_pmp) return false;
407
408 const bool locked = (state->pmpaddr[idx]->cfg & PMP_L);
409
410 bool write_success = false;
411 if (!locked) {
412 state->pmpaddr[idx]->cfg = val;
413 write_success = masked_csr_t::unlogged_write(val);
414 }
415 proc->get_mmu()->flush_tlb();
416
417 return write_success;
418 }
419
420 spmpen_csr_t::spmpen_csr_t(processor_t* const proc, const
reg_t addr, const reg_t init):
421 basic_csr_t(proc, addr, init) {
422 }
423
424 bool spmpen_csr_t::unlogged_write(const reg_t val) noexcept {
425 reg_t write_val = 0;
426 for (reg_t i = proc->n_pmp-1; i < state->max_pmp; ++i) {
427 if (!state->pmpaddr[i]->is_locked() && ((val >> i) & 1))
428 write_val |= reg_t(1) << i;
429 }
430
431 return basic_csr_t::unlogged_write(write_val);
432 }
433
434 // implement class mseccfg_csr_t
435 mseccfg_csr_t::mseccfg_csr_t(processor_t* const proc, const
reg_t addr):
436 basic_csr_t(proc, addr, 0) {
437 }
438
439 void mseccfg_csr_t::verify_permissions(insn_t insn, bool
write) const {
440 basic_csr_t::verify_permissions(insn, write);
441 if (!proc->extension_enabled(EXT_SMEPMP) &&
442 !proc->extension_enabled(EXT_SMPMP) &&
443 !proc->extension_enabled(EXT_ZICFILP) &&
444 !proc->extension_enabled(EXT_ZKR))
445 throw trap_illegal_instruction(insn.bits());
446 }
447
448 bool mseccfg_csr_t::get_mml() const noexcept {
449 return (read() & MSECCFG_MML);
450 }
451
452 bool mseccfg_csr_t::get_mmwp() const noexcept {
453 return (read() & MSECCFG_MMWP);
454 }
455
456 bool mseccfg_csr_t::get_rlb() const noexcept {
457 return (read() & MSECCFG_RLB);
458 }
459
460 bool mseccfg_csr_t::get_useed() const noexcept {
461 return (read() & MSECCFG_USEED);
462 }
463
464 bool mseccfg_csr_t::get_sseed() const noexcept {
465 return (read() & MSECCFG_SSEED);
466 }
467
468 bool mseccfg_csr_t::unlogged_write(const reg_t val) noexcept {
469 reg_t new_val = read();
470
471 if (proc->n_pmp != 0) {
472 // pmpcfg.L is 1 in any rule or entry (including disabled
entries)
473 const bool pmplock_recorded = std::any_of(state->pmpaddr,
state->pmpaddr + proc->n_pmp,
474 [](const base_pmpaddr_csr_t_p & c) { return c-
>is_locked(); } );
475
476 // When RLB is 0 and pmplock_recorded, RLB is locked to 0.
477 // Otherwise set the RLB bit according val
478 if (!(pmplock_recorded && (read() & MSECCFG_RLB) == 0)) {
479 new_val &= ~MSECCFG_RLB;
480 new_val |= (val & MSECCFG_RLB);
481 }
482
483 new_val |= (val & MSECCFG_MMWP); //MMWP is sticky
484 new_val |= (val & MSECCFG_MML); //MML is sticky

```

```

485
486 proc->get_mmu()->flush_tlb();
487 }
488
489 if (proc->extension_enabled(EXT_ZKR)) {
490 reg_t mask =
491 (proc->extension_enabled('U') ? MSECCFG_USEED : 0) |
492 (proc->extension_enabled('S') ? MSECCFG_SSEED : 0);
493 new_val = (new_val & ~mask) | (val & mask);
494 }
495
496 if (proc->extension_enabled(EXT_ZICFILP)) {
497 new_val &= ~MSECCFG_MLPE;
498 new_val |= (val & MSECCFG_MLPE);
499 }
500
501 if (proc->extension_enabled(EXT_SMPMP)) {
502 const reg_t pmm_reserved = 1; // Reserved value of msec-
cfg.PMM
503 reg_t pmm = get_field(val, MSECCFG_PMM);
504 new_val = set_field(new_val, MSECCFG_PMM, pmm != pmm_re-
served ? pmm : 0);
505 }
506
507 return basic_csr_t::unlogged_write(new_val);
508 }
509
510 // implement class virtualized_csr_t
511 virtualized_csr_t::virtualized_csr_t(processor_t* const proc,
csr_t_p orig, csr_t_p virt):
512 csr_t(proc, orig->address),
513 orig_csr(orig),
514 virt_csr(std::move(virt)) {
515 }
516
517 reg_t virtualized_csr_t::read() const noexcept {
518 return readvirt(state->v);
519 }
520
521 reg_t virtualized_csr_t::readvirt(bool virt) const noexcept {
522 return virt ? virt_csr->read() : orig_csr->read();
523 }
524
525 bool virtualized_csr_t::unlogged_write(const reg_t val) noex-
cept {
526 if (state->v)
527 virt_csr->write(val);
528 else
529 orig_csr->write(val);
530 return false; // virt_csr or orig_csr has already logged
531 }
532
533 // implement class epc_csr_t
534 epc_csr_t::epc_csr_t(processor_t* const proc, const reg_t
addr):
535 csr_t(proc, addr),
536 val(0) {
537 }
538
539 reg_t epc_csr_t::read() const noexcept {
540 return val & proc->pc_alignment_mask();
541 }
542
543 bool epc_csr_t::unlogged_write(const reg_t val) noexcept {
544 this->val = val & ~(reg_t)1;
545 return true;
546 }
547
548 // implement class tvec_csr_t
549 tvec_csr_t::tvec_csr_t(processor_t* const proc, const reg_t
addr):
550 csr_t(proc, addr),
551 val(0) {
552 }
553
554 reg_t tvec_csr_t::read() const noexcept {
555 return val;
556 }
557
558 bool tvec_csr_t::unlogged_write(const reg_t val) noexcept {
559 this->val = val & ~(reg_t)2;
560 return true;
561 }
562
563 // implement class cause_csr_t
564 cause_csr_t::cause_csr_t(processor_t* const proc, const reg_t
addr):
565 basic_csr_t(proc, addr, 0) {
566 }
567
568 reg_t cause_csr_t::read() const noexcept {
569 reg_t val = basic_csr_t::read();
570 // When reading, the interrupt bit needs to adjust to xlen.
Spike does
571 // not generally support dynamic xlen, but this code was
(partly)
572 // there since at least 2015 (ea58df8 and c4350ef).
573 if (proc->get_isa().get_max_xlen() > proc->get_xlen()) //
Move interrupt bit to top of xlen
574 return val | ((val >> (proc->get_isa().get_max_xlen()-1))
<< (proc->get_xlen()-1));

```

```

575     return val;
576 }
577
578 // implement class base_status_csr_t
579 base_status_csr_t::base_status_csr_t(processor_t* const proc,
const reg_t addr):
580     csr_t(proc, addr),
581     has_page(proc->extension_enabled_const('S') && proc-
>has_mmu()),
582     sstatus_write_mask(compute_sstatus_write_mask()),
583     sstatus_read_mask(sstatus_write_mask | SSTATUS_UBE |
SSTATUS_UXL
584                       | (proc->get_const_xlen() == 32 ?
SSTATUS32_SD : SSTATUS64_SD)) {
585 }
586
587 reg_t base_status_csr_t::compute_sstatus_write_mask() const
noexcept {
588     // If a configuration has FS bits, they will always be ac-
cessible no
589     // matter the state of misa.
590     const bool has_fs = (proc->extension_enabled('S') || proc-
>extension_enabled('F')) && !proc->extension_enabled(EXT_ZFINX);
591     // Implementations w/o V may still have mstatus.vs.
592     const bool has_vs = proc->any_vector_extensions();
593     return 0
594     | (proc->extension_enabled('S') ? (SSTATUS_SIE |
SSTATUS_SPIE | SSTATUS_SPP : 0)
595     | (has_page || proc->extension_enabled_const(EXT_SSPMP)?
(SSTATUS_SUM | SSTATUS_MXR) : 0)
596     | (has_fs ? SSTATUS_FS : 0)
597     | (proc->any_custom_extensions() ? SSTATUS_XS : 0)
598     | (has_vs ? SSTATUS_VS : 0)
599     | (proc->extension_enabled(EXT_ZVTBASE) ? SSTATUS_MS : 0)
600     | (proc->extension_enabled('S') && proc->exten-
sion_enabled(EXT_ZICFILP) ? SSTATUS_SPELP : 0)
601     | (proc->extension_enabled(EXT_SDBLTRP) ? SSTATUS_SDT :
0)
602     ;
603 }
604
605 reg_t base_status_csr_t::adjust_sd(const reg_t val) const no-
except {
606     // This uses get_const_xlen() instead of get_xlen()
607     // because the SD bit moves when XLEN changes, which means
we would
608     // need to call adjust_sd() on every read, instead of on
every
609     // write.
610     const reg_t sd_bit = proc->get_const_xlen() == 64 ?
SSTATUS64_SD : SSTATUS32_SD;
611     if (((val & SSTATUS_FS) == SSTATUS_FS) ||
612         ((val & SSTATUS_VS) == SSTATUS_VS) ||
613         ((val & SSTATUS_MS) == SSTATUS_MS) ||
614         ((val & SSTATUS_XS) == SSTATUS_XS)) {
615         return val | sd_bit;
616     }
617     return val & ~sd_bit;
618 }
619
620 void base_status_csr_t::maybe_flush_tlb(const reg_t newval)
noexcept {
621     if ((newval ^ read()) &
622         (MSTATUS_MPP | MSTATUS_MPRV
623          | (has_page ? (MSTATUS_MXR | MSTATUS_SUM) : 0)
624          ))
625         proc->get_mmu()->flush_tlb();
626 }
627
628 namespace {
629 int xlen_to_uhl(int xlen) {
630     if (xlen == 32)
631         return 1;
632     if (xlen == 64)
633         return 2;
634     abort();
635 }
636 }
637
638 // implement class vsstatus_csr_t
639 vsstatus_csr_t::vsstatus_csr_t(processor_t* const proc, const
reg_t addr):
640     base_status_csr_t(proc, addr),
641     val(proc->get_state()->mstatus->read() & sstatus_read_mask)
{
642 }
643
644 bool vsstatus_csr_t::unlogged_write(const reg_t val) noexcept
{
645     const reg_t hDTE = (state->henvcfg->read() & HENVCFG_DTE);
646     const reg_t adj_write_mask = sstatus_write_mask & ~(hDTE ?
0 : SSTATUS_SDT);
647     reg_t newval = (this->val & ~adj_write_mask) | (val &
adj_write_mask);
648
649     newval = hDTE && (newval & SSTATUS_SDT) ? (newval &
~SSTATUS_SIE) : newval;
650
651     if (state->v) maybe_flush_tlb(newval);
652     this->val = adjust_sd(newval);
653     return true;

```

```

654 }
655
656 reg_t vsstatus_csr_t::read() const noexcept {
657     const reg_t hDTE = state->henvcfg->read() & HENVCFG_DTE;
658     const reg_t adj_read_mask = sstatus_read_mask & ~(hDTE ? 0 :
SSTATUS_SDT);
659     return this->val & adj_read_mask;
660 }
661
662 // implement class sstatus_proxy_csr_t
663 sstatus_proxy_csr_t::sstatus_proxy_csr_t(processor_t* const
proc, const reg_t addr, mstatus_csr_t* p_mstatus):
664     base_status_csr_t(proc, addr),
665     mstatus(std::move(mstatus)) {
666 }
667
668 bool sstatus_proxy_csr_t::unlogged_write(const reg_t val) no-
except {
669     const reg_t mDTE = (state->menvcfg->read() & MENVCFG_DTE);
670     const reg_t adj_write_mask = sstatus_write_mask & ~(mDTE ?
0 : SSTATUS_SDT);
671     reg_t new_mstatus = (mstatus->read() & ~adj_write_mask) |
(val & adj_write_mask);
672
673     new_mstatus = mDTE && (new_mstatus & SSTATUS_SDT) ? (new_m-
status & ~SSTATUS_SIE) : new_mstatus;
674
675     // On RV32 this will only log the low 32 bits, so make sure
we're
676     // not modifying anything in the upper 32 bits.
677     assert((adj_write_mask & 0xfffffffU) == adj_write_mask);
678
679     mstatus->write(new_mstatus);
680     return false; // avoid double logging: already logged by
mstatus->write()
681 }
682
683 reg_t sstatus_proxy_csr_t::read() const noexcept {
684     const reg_t mDTE = state->menvcfg->read() & MENVCFG_DTE;
685
686     const reg_t adj_read_mask = sstatus_read_mask & ~(mDTE ? 0 :
SSTATUS_SDT);
687     return mstatus->read() & adj_read_mask;
688 }
689
690 // implement class mstatus_csr_t
691 mstatus_csr_t::mstatus_csr_t(processor_t* const proc, const
reg_t addr):
692     base_status_csr_t(proc, addr),
693     val(compute_mstatus_initial_value()) {
694 }
695
696 reg_t mstatus_csr_t::read() const noexcept {
697     return val & ~reg_t(state->menvcfg->read() & MENVCFG_DTE ?
0 : SSTATUS_SDT);
698 }
699
700 bool mstatus_csr_t::unlogged_write(const reg_t val) noexcept {
701     const bool has_mpv = proc->extension_enabled('H');
702     const bool has_gva = has_mpv;
703     const reg_t adj_write_mask = sstatus_write_mask &
~reg_t(state->menvcfg->read() & MENVCFG_DTE ? 0 : SSTATUS_SDT);
704     const reg_t mask = adj_write_mask
705     | MSTATUS_MIE | MSTATUS_MPIE
706     | (proc->extension_enabled('U') ?
707        MSTATUS_MPRV : 0)
708     | (proc->extension_enabled('U') ?
709        MSTATUS_TW : 0)
710     | (proc->extension_enabled('S') ?
711        MSTATUS_TSR : 0)
712     | (has_page ? MSTATUS_TVM : 0)
713     | (has_gva ? MSTATUS_GVA : 0)
714     | (has_mpv ? MSTATUS_MPV : 0)
715     | (proc->extension_enabled(EXT_SDBLTRP) ?
716        MSTATUS_MDT : 0)
717     | (proc->extension_enabled(EXT_ZICFILP) ?
(MSTATUS_MPELP | (proc->extension_enabled('S') ? MSTATUS_SPELP :
0)) : 0)
718     ;
719
720     const reg_t requested_mpp = proc->legal-
ize_privilege(get_field(val, MSTATUS_MPP));
721     const reg_t adjusted_val = set_field(val, MSTATUS_MPP, re-
quested_mpp);
722     reg_t new_mstatus = (this->val & ~mask) | (adjusted_val &
mask);
723     new_mstatus = (new_mstatus & MSTATUS_MDT) ? (new_mstatus &
~MSTATUS_MIE) : new_mstatus;
724     new_mstatus = (new_mstatus & MSTATUS_SDT) ? (new_mstatus &
~MSTATUS_SIE) : new_mstatus;
725     maybe_flush_tlb(new_mstatus);
726     this->val = adjust_sd(new_mstatus);
727     return true;
728 }
729
730 reg_t mstatus_csr_t::compute_mstatus_initial_value() const no-
except {
731     const reg_t big_endian_bits = (proc->exten-

```



```

sion_enabled_const('U') ? MSTATUS_UBE : 0)
729 | (proc->exten-
sion_enabled_const('S') ? MSTATUS_SBE : 0)
730 | MSTATUS_MBE;
731 return 0
732 | set_field((reg_t)0, MSTATUS_MPP, proc->exten-
sion_enabled_const('U') ? PRV_U : PRV_M)
733 | (proc->extension_enabled_const('U') && (proc-
>get_const_xlen() != 32) ? set_field((reg_t)0, MSTATUS_UXL,
xlen_to_uxl(proc->get_const_xlen())) : 0)
734 | (proc->extension_enabled_const('S') && (proc-
>get_const_xlen() != 32) ? set_field((reg_t)0, MSTATUS_SXL,
xlen_to_uxl(proc->get_const_xlen())) : 0)
735 | (proc->get_mmu()->is_target_big_endian() ? big_en-
dian_bits : 0)
736 | (proc->extension_enabled(EXT_SMDBLTRP) ?
MSTATUS_MDT : 0)
737 | 0; // initial value for mstatus
738 }
739
740 // implement class mnstatus_csr_t
741 mnstatus_csr_t::mnstatus_csr_t(processor_t* const proc, const
reg_t addr):
742 basic_csr_t(proc, addr, 0) {
743 }
744
745 bool mnstatus_csr_t::unlogged_write(const reg_t val) noexcept
{
746 const reg_t mask = MSTATUS_NMIE
747 | (proc->extension_enabled('H') ? MN-
STATUS_MNPV : 0)
748 | (proc->extension_enabled(EXT_ZICFILP) ?
MSTATUS_MNPELP : 0)
749 | MSTATUS_MNPP;
750
751 const reg_t requested_mnpp = proc->legal-
ize_privilege(get_field(val, MSTATUS_MNPP));
752 const reg_t adjusted_val = set_field(val, MSTATUS_MNPP, re-
quested_mnpp);
753 // NMIE can be set but not cleared
754 const reg_t new_mnstatus = (read() & MSTATUS_NMIE) | (ad-
justed_val & mask);
755
756 return basic_csr_t::unlogged_write(new_mnstatus);
757 }
758
759 // implement class rv32_low_csr_t
760 rv32_low_csr_t::rv32_low_csr_t(processor_t* const proc, const
reg_t addr, csr_t_p orig):
761 csr_t(proc, addr),
762 orig(std::move(orig)) {
763 }
764
765 reg_t rv32_low_csr_t::read() const noexcept {
766 return orig->read() & 0xffffffffFU;
767 }
768
769 void rv32_low_csr_t::verify_permissions(insn_t insn, bool
write) const {
770 orig->verify_permissions(insn, write);
771 }
772
773 bool rv32_low_csr_t::unlogged_write(const reg_t val) noexcept
{
774 return orig->unlogged_write((orig->written_value() >> 32 <<
32) | (val & 0xffffffffFU));
775 }
776
777 reg_t rv32_low_csr_t::written_value() const noexcept {
778 return orig->written_value() & 0xffffffffFU;
779 }
780
781 // implement class rv32_high_csr_t
782 rv32_high_csr_t::rv32_high_csr_t(processor_t* const proc,
const reg_t addr, csr_t_p orig):
783 csr_t(proc, addr),
784 orig(std::move(orig)) {
785 }
786
787 reg_t rv32_high_csr_t::read() const noexcept {
788 return (orig->read() >> 32) & 0xffffffffFU;
789 }
790
791 void rv32_high_csr_t::verify_permissions(insn_t insn, bool
write) const {
792 orig->verify_permissions(insn, write);
793 }
794
795 bool rv32_high_csr_t::unlogged_write(const reg_t val) noexcept {
796 return orig->unlogged_write((orig->written_value() << 32 >>
32) | ((val & 0xffffffffFU) << 32));
797 }
798
799 reg_t rv32_high_csr_t::written_value() const noexcept {
800 return (orig->written_value() >> 32) & 0xffffffffFU;
801 }
802
803 aia_rv32_high_csr_t::aia_rv32_high_csr_t(processor_t* const
proc, const reg_t addr, csr_t_p orig):
804 rv32_high_csr_t(proc, addr, orig) {

```

```

805 }
806
807 void aia_rv32_high_csr_t::verify_permissions(insn_t insn, bool
write) const {
808 if (proc->extension_enabled(EXT_SMSTATEEN)) {
809 if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_AIA))
810 throw trap_illegal_instruction(insn.bits());
811
812 if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_AIA))
813 throw trap_virtual_instruction(insn.bits());
814 }
815
816 rv32_high_csr_t::verify_permissions(insn, write);
817 }
818
819 // implement class sstatus_csr_t
820 sstatus_csr_t::sstatus_csr_t(processor_t* const proc,
sstatus_proxy_csr_t_p orig, vsstatus_csr_t_p virt):
821 virtualized_csr_t(proc, orig, virt),
822 orig_sstatus(orig),
823 virt_sstatus(virt) {
824 }
825
826 void sstatus_csr_t::dirty(const reg_t dirties) {
827 // As an optimization, return early if already dirty.
828 if ((orig_sstatus->read() & dirties) == dirties) {
829 if (likely(!state->v || (virt_sstatus->read() & dirties)
== dirties))
830 return;
831 }
832
833 // Catch problems like #823 where P-extension instructions
were not
834 // checking for mstatus.VS!=Off:
835 if (!enabled(dirties)) abort();
836
837 orig_sstatus->write(orig_sstatus->read() | dirties);
838 if (state->v) {
839 virt_sstatus->write(virt_sstatus->read() | dirties);
840 }
841 }
842
843 bool sstatus_csr_t::enabled(const reg_t which) {
844 if ((orig_sstatus->read() & which) != 0) {
845 if (!state->v || (virt_sstatus->read() & which) != 0)
846 return true;
847 }
848 return false;
849 }
850
851 // implement class misa_csr_t
852 misa_csr_t::misa_csr_t(processor_t* const proc, const reg_t
addr, const reg_t max_isa):
853 basic_csr_t(proc, addr, max_isa),
854 max_isa(max_isa),
855 write_mask(max_isa & (0 // allow MABFDQCHV bits in MISA to
be modified
856 | (1L << ('M' - 'A'))
857 | (1L << ('A' - 'A'))
858 | (1L << ('B' - 'A'))
859 | (1L << ('F' - 'A'))
860 | (1L << ('D' - 'A'))
861 | (1L << ('Q' - 'A'))
862 | (1L << ('C' - 'A'))
863 | (1L << ('H' - 'A'))
864 | (1L << ('V' - 'A'))
865 )
866 ) {
867 }
868
869 reg_t misa_csr_t::dependency(const reg_t val, const char fea-
ture, const char depends_on) const noexcept {
870 return (val & (1L << (depends_on - 'A'))) ? val : (val &
~(1L << (feature - 'A')));
871 }
872
873 bool misa_csr_t::unlogged_write(const reg_t val) noexcept {
874 const reg_t old_misa = read();
875
876 // the write is ignored if increasing IALIGN would misalign
the PC
877 if (!(val & (1L << ('C' - 'A')) && (old_misa & (1L << ('C'
- 'A')) && (state->pc & 2)))
878 return false;
879
880 reg_t adjusted_val = val;
881 adjusted_val = dependency(adjusted_val, 'D', 'F');
882 adjusted_val = dependency(adjusted_val, 'Q', 'D');
883 adjusted_val = dependency(adjusted_val, 'V', 'D');
884
885 const bool prev_h = old_misa & (1L << ('H' - 'A'));
886 const reg_t new_misa = (adjusted_val & write_mask) |
(old_misa & ~write_mask);
887 const bool new_h = new_misa & (1L << ('H' - 'A'));
888 const bool new_v = proc->get_isa().has_any_vector();
889
890 proc->set_extension_enable(EXT_ZCA, (new_misa & (1L << ('C'

```

```

- 'A')) || !proc->get_isa().extension_enabled('C'));
891 proc->set_extension_enabled(EXT_ZCF, (new_misa & (1L << ('F'
- 'A')))) && proc->extension_enabled(EXT_ZCA) && proc->get_xlen() ==
32);
892 proc->set_extension_enabled(EXT_ZCD, (new_misa & (1L << ('D'
- 'A')))) && proc->extension_enabled(EXT_ZCA));
893 proc->set_extension_enabled(EXT_ZCB, proc->extension_enabled(EXT_ZCA));
894 proc->set_extension_enabled(EXT_ZCMP, proc->extension_enabled(EXT_ZCA));
895 proc->set_extension_enabled(EXT_ZCMT, proc->extension_enabled(EXT_ZCA));

896 proc->set_extension_enabled(EXT_ZFH, new_misa & (1L << ('F' -
'A')));
897 proc->set_extension_enabled(EXT_ZFHM, new_misa & (1L <<
('F' - 'A')));
898 proc->set_extension_enabled(EXT_ZVFH, new_v && proc->get_isa().get_zvf() && proc->extension_enabled(EXT_ZFHM));
899 proc->set_extension_enabled(EXT_ZVFHM, new_v && proc->get_isa().get_zvf());
900 proc->set_extension_enabled(EXT_ZAAMO, (new_misa & (1L <<
('A' - 'A')))) || !proc->get_isa().extension_enabled('A'));
901 proc->set_extension_enabled(EXT_ZALRSC, (new_misa & (1L <<
('A' - 'A')))) || !proc->get_isa().extension_enabled('A'));
902 proc->set_extension_enabled(EXT_ZABHA, (new_misa & (1L <<
('A' - 'A')))) || !proc->get_isa().extension_enabled('A'));
903 proc->set_extension_enabled(EXT_ZACAS, (new_misa & (1L <<
('A' - 'A')))) || !proc->get_isa().extension_enabled('A'));
904 proc->set_extension_enabled(EXT_ZBA, (new_misa & (1L << ('B'
- 'A')))) || !proc->get_isa().extension_enabled('B'));
905 proc->set_extension_enabled(EXT_ZBB, (new_misa & (1L << ('B'
- 'A')))) || !proc->get_isa().extension_enabled('B'));
906 proc->set_extension_enabled(EXT_ZBS, (new_misa & (1L << ('B'
- 'A')))) || !proc->get_isa().extension_enabled('B'));
907
908 basic_csr_t::unlogged_write(new_misa);
909
910 // update the hypervisor-only bits in MEDELEG and other CSRs
911 if (!new_h && prev_h) {
912     state->medeleg->write(state->medeleg->read());
913     if (state->mnstatus) state->mnstatus->write(state->mnstatus->read());
914     const reg_t new_mstatus = state->mstatus->read() &
~(MSTATUS_GVA | MSTATUS_MPV);
915     state->mstatus->write(new_mstatus);

916 if (state->mstatus) state->mstatus->write(new_mstatus >>
32); // log mstatus change
917 state->mie->write_with_mask(MIP_HS_MASK, 0); // also
takes care of hie, sie
918 state->mip->write_with_mask(MIP_HS_MASK, 0); // also
takes care of hip, sip, hvip
919 state->hstatus->write(0);
920 for (reg_t i = 0; i < N_HPMCOUNTERS; ++i) {
921     state->mevent[i]->write(state->mevent[i]->read());
922 }
923 state->mcyclecfg->write(state->mcyclecfg->read());
924 state->minstretcfg->write(state->minstretcfg->read());
925 }
926
927 proc->get_mmu()->flush_tlb();
928 proc->build_opcode_map();
929
930 return true;
931 }
932
933 bool misa_csr_t::extension_enabled_const(unsigned char ext)
const noexcept {
934     assert(!(1 & (write_mask >> (ext - 'A'))));
935     return extension_enabled(ext);
936 }
937
938 // implement class mip_or_mie_csr_t
939 mip_or_mie_csr_t::mip_or_mie_csr_t(processor_t* const proc,
const reg_t addr):
940     csr_t(proc, addr),
941     val(0) {
942 }
943
944 reg_t mip_or_mie_csr_t::read() const noexcept {
945     return val;
946 }
947
948 void mip_or_mie_csr_t::write_with_mask(const reg_t mask, const
reg_t val) noexcept {
949     this->val = (this->val & ~mask) | (val & mask);
950     log_write();
951 }
952
953 bool mip_or_mie_csr_t::unlogged_write(const reg_t val) noexcept {
954     write_with_mask(write_mask(), val);
955     return false; // avoid double logging: already logged by
write_with_mask()
956 }
957
958 mip_csr_t::mip_csr_t(processor_t* const proc, const reg_t
addr):
959     mip_or_mie_csr_t(proc, addr) {
960 }
961
962 void mip_csr_t::write_with_mask(const reg_t mask, const reg_t
val) noexcept {
963     if (!(state->mvien->read() & MIP_SEIP) && (mask & MIP_SEIP))
964         state->mvip->write_with_mask(MIP_SEIP, val); // mvip.SEIP
is an alias of mip.SEIP when mvien.SEIP=0
965     mip_or_mie_csr_t::write_with_mask(mask & ~MIP_SEIP, val);
966 }
967
968 reg_t mip_csr_t::read() const noexcept {
969     return val | state->hvip->basic_csr_t::read() | ((state->mvien->read() & MIP_SEIP) ? 0 : (state->mvip->basic_csr_t::read() & MIP_SEIP));
970 }
971
972 void mip_csr_t::backdoor_write_with_mask(const reg_t mask,
const reg_t val) noexcept {
973     this->val = (this->val & ~mask) | (val & mask);
974 }
975
976 reg_t mip_csr_t::write_mask() const noexcept {
977     // MIP_STIP is writable unless SSTC exists and STCE is set
in MENVCFG
978
979 const reg_t supervisor_ints = proc->extension_enabled('S') ?
MIP_SSIP | ((state->menvcfg->read() & MENVCFG_STCE) ? 0 : MIP_STIP)
| MIP_SEIP : 0;
980 const reg_t lscof_int = proc->extension_enabled(EXT_SSCOFPMF) ?
MIP_LCOFIP : 0;
981 const reg_t vssip_int = proc->extension_enabled('H') ?
MIP_VSSIP : 0;

982 const reg_t hypervisor_ints = proc->extension_enabled('H') ?
MIP_HS_MASK : 0;
983 // We must mask off sgeip, vstip, and vseip. All three of
these
984 // bits are aliases for the same bits in hip. The hip spec
says:
985 // * sgeip is read-only -- write hgeip instead
986 // * vseip is read-only -- write hvip instead
987 // * vstip is read-only -- write hvip instead
988 return (supervisor_ints | hypervisor_ints | lscof_int) &
(MIP_SEIP | MIP_SSIP | MIP_STIP | MIP_LCOFIP | vs-sip_int);
989 }
990
991 mie_csr_t::mie_csr_t(processor_t* const proc, const reg_t
addr):
992     mip_or_mie_csr_t(proc, addr) {
993 }
994
995 reg_t mie_csr_t::write_mask() const noexcept {
996     const reg_t supervisor_ints = proc->extension_enabled('S') ?
MIP_SSIP | MIP_STIP | MIP_SEIP : 0;
997     const reg_t lscof_int = proc->extension_enabled(EXT_SSCOFPMF) ?
MIP_LCOFIP : 0;

998     const reg_t hypervisor_ints = proc->extension_enabled('H') ?
MIP_HS_MASK : 0;
999     const reg_t coprocessor_ints = (reg_t)proc->any_cus-
tom_extensions() << IRQ_COP;
1000     const reg_t delegable_ints = supervisor_ints | copro-
cessor_ints | lscof_int;
1001     const reg_t all_ints = delegable_ints | hypervisor_ints |
MIP_MSIP | MIP_MTIP | MIP_MEIP;
1002     return all_ints;
1003 }
1004
1005 // implement class generic_int_accessor_t
1006 generic_int_accessor_t::generic_int_accessor_t(state_t* const
state,
1007 const reg_t
read_mask,
1008 const reg_t
ip_write_mask,
1009 const reg_t
ie_write_mask,
1010 const
mask_mode_t mask_mode,
1011 const int
shiftamt):
1012     state(state),
1013     read_mask(read_mask),
1014     ip_write_mask(ip_write_mask),
1015     ie_write_mask(ie_write_mask),
1016     mask_mode(mask_mode == MIDELEG),
1017     mask_hideleg(mask_mode == HIDELEG),
1018     shiftamt(shiftamt) {
1019 }
1020
1021 reg_t generic_int_accessor_t::ip_read() const noexcept {
1022     return (state->mip->read() & deleg_mask() & read_mask) >>
shiftamt;
1023 }
1024
1025 void generic_int_accessor_t::ip_write(const reg_t val) noexcept {
1026     const reg_t mask = deleg_mask() & ip_write_mask;
1027     state->mip->write_with_mask(mask, val << shiftamt);

```

```

1028 }
1029
1030 reg_t generic_int_accessor_t::ie_read() const noexcept {
1031     return (state->mie->read() & deleg_mask() & read_mask) >>
shiftamt;
1032 }
1033
1034 void generic_int_accessor_t::ie_write(const reg_t val) noexcept {
1035     const reg_t mask = deleg_mask() & ie_write_mask;
1036     state->mie->write_with_mask(mask, val << shiftamt);
1037 }
1038
1039 reg_t generic_int_accessor_t::deleg_mask() const {
1040     const reg_t hideleg_mask = mask_hideleg ? state->hideleg-
>read() : (reg_t)0;
1041     const reg_t mideleg_mask = mask_mideleg ? state->mideleg-
>read() : (reg_t)0;
1042     return hideleg_mask & mideleg_mask;
1043 }
1044
1045 // implement class mip_proxy_csr_t
1046 mip_proxy_csr_t::mip_proxy_csr_t(processor_t* const proc,
const reg_t addr, generic_int_accessor_t_p accr):
1047     csr_t(proc, addr),
1048     accr(std::move(accr)) {
1049 }
1050
1051 void mip_proxy_csr_t::verify_permissions(insn_t insn, bool
write) const {
1052     csr_t::verify_permissions(insn, write);
1053     if (proc->extension_enabled_const(EXT_SSAIA) && proc->exten-
sion_enabled('H')) {
1054         if ((state->csrmap[CSR_HVICTL]->read() & HVICTL_VTI) &&
1055             proc->extension_enabled('S') && state->v)
1056             throw trap_virtual_instruction(insn.bits()); // VS-mode
attempts to access sip when hvictl.VTI=1
1057     }
1058 }
1059
1060 reg_t mip_proxy_csr_t::read() const noexcept {
1061     return accr->ip_read();
1062 }
1063
1064 bool mip_proxy_csr_t::unlogged_write(const reg_t val) noexcept {
1065     accr->ip_write(val);
1066     return false; // accr has already logged
1067 }
1068
1069 // implement class mie_proxy_csr_t
1070 mie_proxy_csr_t::mie_proxy_csr_t(processor_t* const proc,
const reg_t addr, generic_int_accessor_t_p accr):
1071     csr_t(proc, addr),
1072     accr(std::move(accr)) {
1073 }
1074
1075 void mie_proxy_csr_t::verify_permissions(insn_t insn, bool
write) const {
1076     csr_t::verify_permissions(insn, write);
1077     if (proc->extension_enabled_const(EXT_SSAIA) && proc->exten-
sion_enabled('H')) {
1078         if ((state->csrmap[CSR_HVICTL]->read() & HVICTL_VTI) &&
1079             proc->extension_enabled('S') && state->v)
1080             throw trap_virtual_instruction(insn.bits()); // VS-mode
attempts to access sie when hvictl.VTI=1
1081     }
1082 }
1083
1084 reg_t mie_proxy_csr_t::read() const noexcept {
1085     return accr->ie_read();
1086 }
1087
1088 bool mie_proxy_csr_t::unlogged_write(const reg_t val) noexcept {
1089     accr->ie_write(val);
1090     return false; // accr has already logged
1091 }
1092
1093 // implement class mideleg_csr_t
1094 mideleg_csr_t::mideleg_csr_t(processor_t* const proc, const
reg_t addr):
1095     basic_csr_t(proc, addr, 0) {
1096 }
1097
1098 reg_t mideleg_csr_t::read() const noexcept {
1099     reg_t val = basic_csr_t::read();
1100     if (proc->extension_enabled('H')) return val |
MIDELEG_FORCED_MASK;
1101     // No need to clear MIDELEG_FORCED_MASK because those bits
can never
1102     // get set in val.
1103     return val;
1104 }
1105
1106 void mideleg_csr_t::verify_permissions(insn_t insn, bool
write) const {
1107     basic_csr_t::verify_permissions(insn, write);
1108     if (!proc->extension_enabled('S'))

```

```

1109         throw trap_illegal_instruction(insn.bits());
1110 }
1111
1112 bool mideleg_csr_t::unlogged_write(const reg_t val) noexcept {
1113     const reg_t supervisor_ints = proc->extension_enabled('S') ?
MIP_SSIP | MIP_STIP | MIP_SEIP : 0;
1114     const reg_t lscf_int = proc->exten-
sion_enabled(EXT_SSCOFFPMF) ? MIP_LCOFIP : 0;
1115     const reg_t coprocessor_ints = (reg_t)proc->any_cus-
tom_extensions() << IRQ_COP;
1116     const reg_t delegable_ints = supervisor_ints | copro-
cessor_ints | lscf_int;
1117     return basic_csr_t::unlogged_write(val & delegable_ints);
1118 }
1119
1120 // implement class medeleg_csr_t
1121 medeleg_csr_t::medeleg_csr_t(processor_t* const proc, const
reg_t addr):
1122     basic_csr_t(proc, addr, 0),
1123     hypervisor_exceptions(0)
1124     {
1125         | (1 <<
CAUSE_VIRTUAL_SUPERVISOR_ECALL)
1126         | (1 << CAUSE_FETCH_GUEST_PAGE_FAULT)
1127         | (1 << CAUSE_LOAD_GUEST_PAGE_FAULT)
1128         | (1 << CAUSE_VIRTUAL_INSTRUCTION)
1129         | (1 <<
CAUSE_STORE_GUEST_PAGE_FAULT)),
1130     mmu_exceptions(0)
1131     {
1132         | (1 << CAUSE_FETCH_PAGE_FAULT)
1133         | (1 << CAUSE_LOAD_PAGE_FAULT)
1134         | (1 << CAUSE_STORE_PAGE_FAULT)) {
1135 }
1136
1137 void medeleg_csr_t::verify_permissions(insn_t insn, bool
write) const {
1138     basic_csr_t::verify_permissions(insn, write);
1139     if (!proc->extension_enabled('S'))
1140         throw trap_illegal_instruction(insn.bits());
1141 }
1142
1143 bool medeleg_csr_t::unlogged_write(const reg_t val) noexcept {
1144     const reg_t mask = 0
1145     | (proc->extension_enabled(EXT_ZCA) ? 0 : 1 <<
CAUSE_MISALIGNED_FETCH)
1146     | (1 << CAUSE_FETCH_ACCESS)
1147     | (1 << CAUSE_ILLEGAL_INSTRUCTION)
1148     | (1 << CAUSE_BREAKPOINT)
1149     | (1 << CAUSE_MISALIGNED_LOAD)
1150     | (1 << CAUSE_LOAD_ACCESS)
1151     | (1 << CAUSE_MISALIGNED_STORE)
1152     | (1 << CAUSE_STORE_ACCESS)
1153     | (1 << CAUSE_USER_ECALL)
1154     | (1 << CAUSE_SUPERVISOR_ECALL)
1155     | (proc->has_mmu() ? mmu_exceptions : 0)
1156     | (proc->extension_enabled('H') ? hypervisor_exceptions :
0)
1157     | ((proc->extension_enabled(EXT_ZICFISS) || proc->exten-
sion_enabled(EXT_ZICFILP)) ?
1158         (1 << CAUSE_SOFTWARE_CHECK_FAULT) : 0)
1159     | (proc->extension_enabled(EXT_ZICNTR) ?
1160         (1 << CAUSE_HARDWARE_ERROR_FAULT) : 0)
1161     ;
1162     return basic_csr_t::unlogged_write(val & mask);
1163 }
1164
1165 sip_csr_t::sip_csr_t(processor_t* const proc, const reg_t
addr, generic_int_accessor_t_p accr):
1166     mip_proxy_csr_t(proc, addr, accr) {
1167 }
1168
1169 reg_t sip_csr_t::read() const noexcept {
1170     const reg_t mask = ~state->mideleg->read() & state->mvien-
>read();
1171     return (mip_proxy_csr_t::read() & ~mask) | (state->mvip-
>read() & mask);
1172 }
1173
1174 bool sip_csr_t::unlogged_write(const reg_t val) noexcept {
1175     const reg_t mask = ~state->mideleg->read() & state->mvien-
>read();
1176     state->mvip->write_with_mask(mask & accr-
>get_ip_write_mask(), val);
1177     return mip_proxy_csr_t::unlogged_write(val & ~mask);
1178 }
1179
1180 sie_csr_t::sie_csr_t(processor_t* const proc, const reg_t
addr, generic_int_accessor_t_p accr):
1181     mie_proxy_csr_t(proc, addr, accr),
1182     val(0) {
1183 }
1184
1185 reg_t sie_csr_t::read() const noexcept {
1186     const reg_t mask = ~state->mideleg->read() & state->mvien-
>read();
1187     return (mie_proxy_csr_t::read() & ~mask) | (val & mask);
1188 }
1189
1190 bool sie_csr_t::unlogged_write(const reg_t val) noexcept {
1191     const reg_t mask = ~state->mideleg->read() & state->mvien-

```

```

>read();
1191 this->val = (this->val & ~mask) | (val & mask);
1192 mie_proxy_csr_t::unlogged_write(val & ~mask);
1193 return true;
1194 }
1195
1196 // implement class masked_csr_t
1197 masked_csr_t::masked_csr_t(processor_t* const proc, const
reg_t addr, const reg_t mask, const reg_t init):
1198     basic_csr_t(proc, addr, init),
1199     mask(mask) {
1200 }
1201
1202 bool masked_csr_t::unlogged_write(const reg_t val) noexcept {
1203     return basic_csr_t::unlogged_write((read() & ~mask) | (val &
mask));
1204 }
1205
1206 envcfg_csr_t::envcfg_csr_t(processor_t* const proc, const
reg_t addr, const reg_t mask,
1207                             const reg_t init):
1208     masked_csr_t(proc, addr, mask, init) {
1209     // In unlogged_write() we WARLize this field for all three
of [msh]envcfg
1210     assert(MENVCFG_CBIE == SENVCFG_CBIE && MENVCFG_CBIE == HEN-
VCFG_CBIE);
1211 }
1212
1213 bool envcfg_csr_t::unlogged_write(const reg_t val) noexcept {
1214     const reg_t cbie_reserved = 2; // Reserved value of xen-
vcfg.CBIE
1215     reg_t adjusted_val = get_field(val, MENVCFG_CBIE) !=
cbie_reserved ? val : set_field(val, MENVCFG_CBIE, 0);
1216
1217     const reg_t pmm_reserved = 1; // Reserved value of xsec-
cfg.PMM
1218     const reg_t pmm = get_field(adjusted_val, MENVCFG_PMM);
1219     adjusted_val = set_field(adjusted_val, MENVCFG_PMM, pmm !=
pmm_reserved ? pmm : 0);
1220
1221     if (get_field(adjusted_val, MENVCFG_PMM) !=
get_field(read(), MENVCFG_PMM))
1222         proc->get_mmu()->flush_tlb();
1223
1224     return masked_csr_t::unlogged_write(adjusted_val);
1225 }
1226
1227 // implement class envcfg_csr_t
1228 envcfg_csr_t::envcfg_csr_t(processor_t* const proc, const
reg_t addr, const reg_t mask, const reg_t init, csr_t_p envcfg):
1229     envcfg_csr_t(proc, addr, mask, init),
1230     envcfg(std::move(envcfg)) {
1231 }
1232
1233 // implement class base_atp_csr_t and family
1234 base_atp_csr_t::base_atp_csr_t(processor_t* const proc, const
reg_t addr):
1235     basic_csr_t(proc, addr, 0) {
1236 }
1237
1238 bool base_atp_csr_t::unlogged_write(const reg_t val) noexcept
{
1239     const reg_t newval = proc->has_mmu() ? com-
pute_new_satp(val) : 0;
1240     if (newval != read())
1241         proc->get_mmu()->flush_tlb();
1242     return basic_csr_t::unlogged_write(newval);
1243 }
1244
1245 bool base_atp_csr_t::satp_valid(reg_t val) const noexcept {
1246     if (proc->get_xlen() == 32) {
1247         switch (get_field(val, SATP32_MODE)) {
1248             case SATP_MODE_OFF: return true;
1249             case SATP_MODE_SV32: return proc->get_max_vaddr_bits()
>= 32;
1250             default: return false;
1251         }
1252     } else {
1253         switch (get_field(val, SATP64_MODE)) {
1254             case SATP_MODE_OFF: return true;
1255             case SATP_MODE_SV39: return proc->get_max_vaddr_bits()
>= 39;
1256             case SATP_MODE_SV48: return proc->get_max_vaddr_bits()
>= 48;
1257             case SATP_MODE_SV57: return proc->get_max_vaddr_bits()
>= 57;
1258             default: return false;
1259         }
1260     }
1261 }
1262
1263 reg_t base_atp_csr_t::compute_new_satp(reg_t val) const noex-
cept {
1264     reg_t rv64_ppn_mask = (reg_t(1) << (proc->paddr_bits() - PG-
SHIFT)) - 1;
1265
1266     reg_t mode_mask = proc->get_xlen() == 32 ? SATP32_MODE :
SATP64_MODE;
1267     reg_t asid_mask_if_enabled = proc->get_xlen() == 32 ? SAT-
P32_ASID : SATP64_ASID;
1268     reg_t asid_mask = proc->supports_impl(IMPL_MMU_ASID) ?
asid_mask_if_enabled : 0;
1269     reg_t ppn_mask = proc->get_xlen() == 32 ? SATP32_PPN : SAT-
P64_PPN & rv64_ppn_mask;
1270     reg_t new_mask = (satp_valid(val) ? mode_mask : 0) |
asid_mask | ppn_mask;
1271     reg_t old_mask = satp_valid(val) ? 0 : mode_mask;
1272
1273     return (new_mask & val) | (old_mask & read());
1274 }
1275
1276 satp_csr_t::satp_csr_t(processor_t* const proc, const reg_t
addr):
1277     base_atp_csr_t(proc, addr) {
1278 }
1279
1280 void satp_csr_t::verify_permissions(insn_t insn, bool write)
const {
1281     base_atp_csr_t::verify_permissions(insn, write);
1282     if (get_field(state->mstatus->read(), MSTATUS_TVM))
1283         require(state->prv == PRV_M);
1284 }
1285
1286 virtualized_satp_csr_t::virtualized_satp_csr_t(processor_t*
const proc, satp_csr_t_p orig, csr_t_p virt):
1287     virtualized_csr_t(proc, orig, virt),
1288     orig_satp(orig) {
1289 }
1290
1291 void virtualized_satp_csr_t::verify_permissions(insn_t insn,
bool write) const {
1292     virtualized_csr_t::verify_permissions(insn, write);
1293
1294     // If satp is accessed from VS mode, it's really accessing
vsatp,
1295     // and the hstatus.VTVM bit controls.
1296     if (state->v) {
1297         if (get_field(state->hstatus->read(), HSTATUS_VTVM))
1298             throw trap_virtual_instruction(insn.bits());
1299     }
1300     else {
1301         orig_csr->verify_permissions(insn, write);
1302     }
1303 }
1304
1305 bool virtualized_satp_csr_t::unlogged_write(const reg_t val)
noexcept {
1306     // If unsupported Mode field: no change to contents
1307     const reg_t newval = orig_satp->satp_valid(val) ? val :
read();
1308     return virtualized_csr_t::unlogged_write(newval);
1309 }
1310
1311 // implement class wide_counter_csr_t
1312 wide_counter_csr_t::wide_counter_csr_t(processor_t* const
proc, const reg_t addr, smcntrpmf_csr_t_p config_csr):
1313     csr_t(proc, addr),
1314     val(0),
1315     written(false),
1316     config_csr(std::move(config_csr)) {
1317 }
1318
1319 reg_t wide_counter_csr_t::read() const noexcept {
1320     return val;
1321 }
1322
1323 void wide_counter_csr_t::bump(const reg_t howmuch) noexcept {
1324     if (written) {
1325         // Because writing a CSR serializes the simulator, howmuch
should
1326         // reflect exactly one instruction: the explicit CSR
write.
1327         // If counting is disabled, though, howmuch will be zero.
1328         assert(howmuch <= 1);
1329         // The ISA mandates that explicit writes to instret take
precedence
1330         // over the instret, so simply skip the increment.
1331         written = false;
1332     } else if (is_counting_enabled()) {
1333         val += howmuch; // to keep log reasonable size, don't log
every bump
1334     }
1335     // Clear cached value
1336     config_csr->reset_prev();
1337 }
1338
1339 bool wide_counter_csr_t::unlogged_write(const reg_t val) noex-
cept {
1340     // Because writing a CSR serializes the simulator and is
followed by a
1341     // bump, back-to-back writes with no intervening bump should
never occur.
1342     assert(!written);
1343     written = true;
1344
1345     this->val = val;
1346     return true;
1347 }

```

```

1348
1349 // Returns true if counting is not inhibited by Smcntrpmf.
1350 // Note that minstretcfg / mcyclecfg / mhpmevent* share the
same inhibit bits.
1351 bool wide_counter_csr_t::is_counting_enabled() const noexcept
{
1352     auto prv = state->prv_changed ? state->prev_prv : state-
>prv;
1353     auto v = state->v_changed ? state->prev_v : state->v;
1354     auto mask = MHPMEVENT_MINH;
1355     if (prv == PRV_S) {
1356         mask = v ? MHPMEVENT_VSINH : MHPMEVENT_SINH;
1357     } else if (prv == PRV_U) {
1358         mask = v ? MHPMEVENT_VUINH : MHPMEVENT_UINH;
1359     }
1360     return (config_csr->read_prev() & mask) == 0;
1361 }
1362
1363 // implement class time_counter_csr_t
1364 time_counter_csr_t::time_counter_csr_t(processor_t* const
proc, const reg_t addr):
1365     csr_t(proc, addr),
1366     shadow_val(0) {
1367 }
1368
1369 reg_t time_counter_csr_t::read() const noexcept {
1370     // reading the time CSR in VS or VU mode returns the sum of
the contents of
1371     // htimedelta and the actual value of time.
1372     if (state->v)
1373         return shadow_val + state->htimedelta->read();
1374     else
1375         return shadow_val;
1376 }
1377
1378 void time_counter_csr_t::sync(const reg_t val) noexcept {
1379     shadow_val = val;
1380     if (proc->extension_enabled(EXT_SSTC)) {
1381         const reg_t mip_val = (shadow_val >= state->stimecmp-
>read() ? MIP_STIP : 0) |
1382             (shadow_val + state->htimedelta->read() >= state->vs-
timecmp->read() ? MIP_VSTIP : 0);
1383         const reg_t mask = ((state->menvcfg->read() & MEN-
VCFG_STCE) ? MIP_STIP : 0) | ((state->henvcfg->read() & HEN-
VCFG_STCE) ? MIP_VSTIP : 0);
1384         state->mip->backdoor_write_with_mask(mask, mip_val);
1385     }
1386 }
1387
1388 proxy_csr_t::proxy_csr_t(processor_t* const proc, const reg_t
addr, csr_t_p delegate):
1389     csr_t(proc, addr),
1390     delegate(std::move(delegate)) {
1391 }
1392
1393 reg_t proxy_csr_t::read() const noexcept {
1394     return delegate->read();
1395 }
1396
1397 bool proxy_csr_t::unlogged_write(const reg_t val) noexcept {
1398     delegate->write(val); // log only under the original (del-
egate's) name
1399     return false;
1400 }
1401
1402 const_csr_t::const_csr_t(processor_t* const proc, const reg_t
addr, reg_t val):
1403     csr_t(proc, addr),
1404     val(val) {
1405 }
1406
1407 reg_t const_csr_t::read() const noexcept {
1408     return val;
1409 }
1410
1411 bool const_csr_t::unlogged_write(const reg_t UNUSED val) noex-
cept {
1412     return false;
1413 }
1414
1415 counter_proxy_csr_t::counter_proxy_csr_t(processor_t* const
proc, const reg_t addr, csr_t_p delegate):
1416     proxy_csr_t(proc, addr, delegate) {
1417 }
1418
1419 bool counter_proxy_csr_t::myenable(csr_t_p counteren) const
noexcept {
1420     return 1 & (counteren->read() >> (address & 31));
1421 }
1422
1423 void counter_proxy_csr_t::verify_permissions(insn_t insn, bool
write) const {
1424     proxy_csr_t::verify_permissions(insn, write);
1425 }
1426     const bool mctr_ok = (state->prv < PRV_M) ? myenable(state-
>mcounteren) : true;
1427     const bool hctr_ok = state->v ? myenable(state->hcounter-
en) : true;
1428     const bool sctr_ok = (proc->extension_enabled('S') && state-
>prv < PRV_S) ? myenable(state->scounteren) : true;

```

```

1429
1430 if (!mctr_ok)
1431     throw trap_illegal_instruction(insn.bits());
1432 if (!hctr_ok)
1433     throw trap_virtual_instruction(insn.bits());
1434 if (!sctr_ok) {
1435     if (state->v)
1436         throw trap_virtual_instruction(insn.bits());
1437     else
1438         throw trap_illegal_instruction(insn.bits());
1439 }
1440 }
1441
1442 mevent_csr_t::mevent_csr_t(processor_t* const proc, const
reg_t addr):
1443     basic_csr_t(proc, addr, 0) {
1444 }
1445
1446 bool mevent_csr_t::unlogged_write(const reg_t val) noexcept {
1447     const reg_t mask = proc->extension_enabled(EXT_SSOFPMF) ?
MHPMEVENT_OF | MHPMEVENT_MINH
1448         | (proc->extension_enabled_const('U') ? MHPMEVENT_UINH :
0)
1449         | (proc->extension_enabled_const('S') ? MHPMEVENT_SINH :
0)
1450         | (proc->extension_enabled('H') ? MHPMEVENT_VUINH | MHP-
MEVENT_VSINH : 0) : 0;
1451     return basic_csr_t::unlogged_write(val & mask);
1452 }
1453
1454 hypervisor_csr_t::hypervisor_csr_t(processor_t* const proc,
const reg_t addr):
1455     basic_csr_t(proc, addr, 0) {
1456 }
1457
1458 void hypervisor_csr_t::verify_permissions(insn_t insn, bool
write) const {
1459     basic_csr_t::verify_permissions(insn, write);
1460     if (!proc->extension_enabled('H'))
1461         throw trap_illegal_instruction(insn.bits());
1462 }
1463
1464 hideleg_csr_t::hideleg_csr_t(processor_t* const proc, const
reg_t addr, csr_t_p mideleg):
1465     masked_csr_t(proc, addr, MIP_VS_MASK, 0),
1466     mideleg(std::move(mideleg)) {
1467 }
1468
1469 reg_t hideleg_csr_t::read() const noexcept {
1470     return masked_csr_t::read() & mideleg->read();
1471 }
1472
1473 hgatp_csr_t::hgatp_csr_t(processor_t* const proc, const reg_t
addr):
1474     basic_csr_t(proc, addr, 0) {
1475 }
1476
1477 void hgatp_csr_t::verify_permissions(insn_t insn, bool write)
const {
1478     basic_csr_t::verify_permissions(insn, write);
1479     if (!state->v && get_field(state->mstatus->read(),
MSTATUS_TVM))
1480         require_privilege(PRV_M);
1481 }
1482
1483 bool hgatp_csr_t::unlogged_write(const reg_t val) noexcept {
1484     proc->get_mmu()->flush_tlb();
1485
1486     reg_t mask;
1487     if (proc->get_const_xlen() == 32) {
1488         mask = HGATP32_PPN |
1489             HGATP32_MODE |
1490             (proc->supports_impl(IMPL_MMU_VMID) ? HGATP32_VMID :
0);
1491     } else {
1492         mask = (HGATP64_PPN & ((reg_t(1) << (proc->paddr_bits() -
PGSHIFT)) - 1)) |
1493             (proc->supports_impl(IMPL_MMU_VMID) ? HGATP64_VMID :
0);
1494
1495         if (get_field(val, HGATP64_MODE) == HGATP_MODE_OFF ||
1496             (proc->get_max_vaddr_bits() >= 39 && get_field(val,
HGATP64_MODE) == HGATP_MODE_SV39X4) ||
1497             (proc->get_max_vaddr_bits() >= 48 && get_field(val,
HGATP64_MODE) == HGATP_MODE_SV48X4) ||
1498             (proc->get_max_vaddr_bits() >= 57 && get_field(val,
HGATP64_MODE) == HGATP_MODE_SV57X4))
1499             mask |= HGATP64_MODE;
1500     }
1501     mask &= ~(reg_t)3;
1502     return basic_csr_t::unlogged_write((read() & ~mask) | (val &
mask));
1503 }
1504
1505 tselect_csr_t::tselect_csr_t(processor_t* const proc, const
reg_t addr):
1506     basic_csr_t(proc, addr, 0) {
1507 }
1508
1509 bool tselect_csr_t::unlogged_write(const reg_t val) noexcept {

```



```

1510 return basic_csr_t::unlogged_write((val < proc-
>TM.count()) ? val : read());
1511 }
1512
1513 tdata1_csr_t::tdata1_csr_t(processor_t* const proc, const
reg_t addr):
1514     csr_t(proc, addr) {
1515 }
1516
1517 reg_t tdata1_csr_t::read() const noexcept {
1518     return proc->TM.tdata1_read(state->tselect->read());
1519 }
1520
1521 bool tdata1_csr_t::unlogged_write(const reg_t val) noexcept {
1522     return proc->TM.tdata1_write(state->tselect->read(), val);
1523 }
1524
1525 tdata2_csr_t::tdata2_csr_t(processor_t* const proc, const
reg_t addr):
1526     csr_t(proc, addr) {
1527 }
1528
1529 reg_t tdata2_csr_t::read() const noexcept {
1530     return proc->TM.tdata2_read(state->tselect->read());
1531 }
1532
1533 bool tdata2_csr_t::unlogged_write(const reg_t val) noexcept {
1534     return proc->TM.tdata2_write(state->tselect->read(), val);
1535 }
1536
1537 tdata3_csr_t::tdata3_csr_t(processor_t* const proc, const
reg_t addr):
1538     csr_t(proc, addr) {
1539 }
1540
1541 reg_t tdata3_csr_t::read() const noexcept {
1542     return proc->TM.tdata3_read(state->tselect->read());
1543 }
1544
1545 bool tdata3_csr_t::unlogged_write(const reg_t val) noexcept {
1546     return proc->TM.tdata3_write(state->tselect->read(), val);
1547 }
1548
1549 tinfo_csr_t::tinfo_csr_t(processor_t* const proc, const reg_t
addr):
1550     csr_t(proc, addr) {
1551 }
1552
1553 reg_t tinfo_csr_t::read() const noexcept {
1554     return proc->TM.tinfo_read(state->tselect->read());
1555 }
1556
1557 debug_mode_csr_t::debug_mode_csr_t(processor_t* const proc,
const reg_t addr):
1558     basic_csr_t(proc, addr, 0) {
1559 }
1560
1561 void debug_mode_csr_t::verify_permissions(insn_t insn, bool
write) const {
1562     basic_csr_t::verify_permissions(insn, write);
1563     if (!state->debug_mode)
1564         throw trap_illegal_instruction(insn.bits());
1565 }
1566
1567 dpc_csr_t::dpc_csr_t(processor_t* const proc, const reg_t
addr):
1568     epc_csr_t(proc, addr) {
1569 }
1570
1571 void dpc_csr_t::verify_permissions(insn_t insn, bool write)
const {
1572     epc_csr_t::verify_permissions(insn, write);
1573     if (!state->debug_mode)
1574         throw trap_illegal_instruction(insn.bits());
1575 }
1576
1577 dcsr_csr_t::dcsr_csr_t(processor_t* const proc, const reg_t
addr):
1578     csr_t(proc, addr),
1579     prv(0),
1580     step(false),
1581     ebreakm(false),
1582     ebreaks(false),
1583     ebreaku(false),
1584     ebreakvs(false),
1585     ebreakvu(false),
1586     v(false),
1587     mprven(false),
1588     cause(0),
1589     ext_cause(0),
1590     cetrig(0),
1591     pelp(elp_t::NO_LP_EXPECTED) {
1592 }
1593
1594 void dcsr_csr_t::verify_permissions(insn_t insn, bool write)
const {
1595     csr_t::verify_permissions(insn, write);
1596     if (!state->debug_mode)
1597         throw trap_illegal_instruction(insn.bits());
1598 }
1599

```

```

1600 reg_t dcsr_csr_t::read() const noexcept {
1601     reg_t result = 0;
1602     result = set_field(result, DCSR_XDEBUGVER, 4);
1603     result = set_field(result, DCSR_EBREAKM, ebreakm);
1604     result = set_field(result, DCSR_EBREAKS, ebreaks);
1605     result = set_field(result, DCSR_EBREAKU, ebreaku);
1606     result = set_field(result, CSR_DCSR_EBREAKVS, ebreakvs);
1607     result = set_field(result, CSR_DCSR_EBREAKVU, ebreakvu);
1608     result = set_field(result, DCSR_STOPCOUNT, 0);
1609     result = set_field(result, DCSR_STOPTIME, 0);
1610     result = set_field(result, DCSR_CAUSE, cause);
1611     result = set_field(result, DCSR_EXTCAUSE, ext_cause);
1612     if (proc->extension_enabled(EXT_SMDBLTRP))
1613         result = set_field(result, DCSR_CETRIG, cetrig);
1614     result = set_field(result, DCSR_STEP, step);
1615     result = set_field(result, DCSR_PRV, prv);
1616     result = set_field(result, CSR_DCSR_V, v);
1617     result = set_field(result, DCSR_MPRVEN, mprven);
1618     result = set_field(result, DCSR_PELP, pelp);
1619     return result;
1620 }
1621
1622 bool dcsr_csr_t::unlogged_write(const reg_t val) noexcept {
1623     prv = get_field(val, DCSR_PRV);
1624     step = get_field(val, DCSR_STEP);
1625     // TODO: ndreset and fullreset
1626     ebreakm = get_field(val, DCSR_EBREAKM);
1627     ebreaks = proc->extension_enabled('S') ? get_field(val,
DCSR_EBREAKS) : false;
1628     ebreaku = proc->extension_enabled('U') ? get_field(val,
DCSR_EBREAKU) : false;
1629     ebreakvs = proc->extension_enabled('H') ? get_field(val,
CSR_DCSR_EBREAKVS) : false;
1630     ebreakvu = proc->extension_enabled('H') ? get_field(val,
CSR_DCSR_EBREAKVU) : false;
1631     v = proc->extension_enabled('H') ? get_field(val, CSR_DC-
SR_V) : false;
1632     mprven = get_field(val, CSR_DCSR_MPRVEN);
1633     pelp = proc->extension_enabled(EXT_ZICFILP) ?
static_cast<elp_t>(get_field(val, DCSR_PELP)) :
elp_t::NO_LP_EXPECTED;
1634     cetrig = proc->extension_enabled(EXT_SMDBLTRP) ?
get_field(val, DCSR_CETRIG) : false;
1635     return true;
1636 }
1637
1638 void dcsr_csr_t::update_fields(const uint8_t cause, uint8_t
ext_cause, const reg_t prv,
const bool v, const elp_t pelp)
noexcept {
1641     this->cause = cause;
1642     this->ext_cause = ext_cause;
1643     this->prv = prv;
1644     this->v = v;
1645     this->pelp = pelp;
1646     log_write();
1647 }
1648
1649 float_csr_t::float_csr_t(processor_t* const proc, const reg_t
addr, const reg_t mask, const reg_t init):
1650     masked_csr_t(proc, addr, mask, init) {
1651 }
1652
1653 void float_csr_t::verify_permissions(insn_t insn, bool write)
const {
1654     masked_csr_t::verify_permissions(insn, write);
1655
1656     if (!(proc->extension_enabled('F') && STATE.sstatus->en-
abled(SSSTATUS_FS))
        || proc->extension_enabled(EXT_ZFINX))
1657         throw trap_illegal_instruction(insn.bits());
1658
1659     if (proc->extension_enabled(EXT_SMSTATEEN) && proc->exten-
sion_enabled(EXT_ZFINX)) {
1661         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_FCSR))
1662             throw trap_illegal_instruction(insn.bits());
1663
1664         if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_FCSR))
1665             throw trap_virtual_instruction(insn.bits());
1666
1667         if ((proc->extension_enabled('S') && state->prv < PRV_S)
&& !(state->sstateen[0]->read() & SSTATEEN0_FCSR)) {
1668             if (state->v)
1669                 throw trap_virtual_instruction(insn.bits());
1670             else
1671                 throw trap_illegal_instruction(insn.bits());
1672         }
1673     }
1674 }
1675
1676 bool float_csr_t::unlogged_write(const reg_t val) noexcept {
1677     if (!proc->extension_enabled(EXT_ZFINX))
1678         dirty_fp_state;
1679     return masked_csr_t::unlogged_write(val);
1680 }
1681
1682 composite_csr_t::composite_csr_t(processor_t* const proc,

```

```

const reg_t addr, csr_t_p upper_csr, csr_t_p lower_csr, const unsigned upper_lsb):
1683     csr_t(proc, addr),
1684     upper_csr(std::move(upper_csr)),
1685     lower_csr(std::move(lower_csr)),
1686     upper_lsb(upper_lsb) {
1687 }
1688
1689 void composite_csr_t::verify_permissions(insn_t insn, bool write) const {
1690     // It is reasonable to assume that either underlying CSR will have
1691     // the same permissions as this composite.
1692     upper_csr->verify_permissions(insn, write);
1693 }
1694
1695 reg_t composite_csr_t::read() const noexcept {
1696     return (upper_csr->read() << upper_lsb) | lower_csr->read();
1697 }
1698
1699 bool composite_csr_t::unlogged_write(const reg_t val) noexcept {
1700     upper_csr->write(val >> upper_lsb);
1701     lower_csr->write(val);
1702     return false; // logging is done only by the underlying CSRs
1703 }
1704
1705 seed_csr_t::seed_csr_t(processor_t* const proc, const reg_t addr):
1706     csr_t(proc, addr) {
1707 }
1708
1709 void seed_csr_t::verify_permissions(insn_t insn, bool write) const {
1710     /* Read-only access disallowed due to wipe-on-read side effect */
1711     /* XXX mseccfg.sseed and mseccfg.useed should be verified. */
1712     if (!proc->extension_enabled(EXT_ZKR) || !write)
1713         throw trap_illegal_instruction(insn.bits());
1714     csr_t::verify_permissions(insn, write);
1715
1716     if (state->v) {
1717         if (state->mseccfg->get_sseed() && write)
1718             throw trap_virtual_instruction(insn.bits());
1719         else
1720             throw trap_illegal_instruction(insn.bits());
1721     } else if ((state->prv == PRV_U && !state->mseccfg->get_useed()) ||
1722                (state->prv == PRV_S && !state->mseccfg->get_sseed())) {
1723         throw trap_illegal_instruction(insn.bits());
1724     }
1725 }
1726
1727 reg_t seed_csr_t::read() const noexcept {
1728     return proc->es.get_seed();
1729 }
1730
1731 bool seed_csr_t::unlogged_write(const reg_t val) noexcept {
1732     proc->es.set_seed(val);
1733     return true;
1734 }
1735
1736 vector_csr_t::vector_csr_t(processor_t* const proc, const reg_t addr, const reg_t mask, const reg_t init):
1737     basic_csr_t(proc, addr, init),
1738     mask(mask) {
1739 }
1740
1741 void vector_csr_t::verify_permissions(insn_t insn, bool write) const {
1742     require(proc->any_vector_extensions() && STATE.sstatus->enabled(SSTATUS_VS));
1743     basic_csr_t::verify_permissions(insn, write);
1744 }
1745
1746 void vector_csr_t::write_raw(const reg_t val) noexcept {
1747     const bool success = basic_csr_t::unlogged_write(val);
1748     if (success)
1749         log_write();
1750 }
1751
1752 bool vector_csr_t::unlogged_write(const reg_t val) noexcept {
1753     if (mask == 0) return false;
1754     STATE.sstatus->dirty(SSTATUS_VS);
1755     return basic_csr_t::unlogged_write(val & mask);
1756 }
1757
1758 vxsat_csr_t::vxsat_csr_t(processor_t* const proc, const reg_t addr):
1759     masked_csr_t(proc, addr, /*mask*/ 1, /*init*/ 0) {
1760 }
1761
1762 void vxsat_csr_t::verify_permissions(insn_t insn, bool write) const {
1763     require(!proc->any_vector_extensions() || STATE.sstatus->enabled(SSTATUS_VS));
1764     masked_csr_t::verify_permissions(insn, write);
1765 }
1766
1767 bool vxsat_csr_t::unlogged_write(const reg_t val) noexcept {
1768     if (proc->any_vector_extensions())
1769         STATE.sstatus->dirty(SSTATUS_VS);
1770     return masked_csr_t::unlogged_write(val);
1771 }
1772
1773 // implement class hstateen_csr_t
1774 hstateen_csr_t::hstateen_csr_t(processor_t* const proc, const reg_t addr, const reg_t mask, const reg_t init, uint8_t index):
1775     basic_csr_t(proc, addr, init),
1776     index(index),
1777     mask(mask) {
1778 }
1779
1780 reg_t hstateen_csr_t::read() const noexcept {
1781     // For every bit in an mstateen CSR that is zero (whether read-only zero or set to zero),
1782     // the same bit appears as read-only zero in the matching hstateen and sstateen CSRs
1783     return basic_csr_t::read() & state->mstateen[index]->read();
1784 }
1785
1786 bool hstateen_csr_t::unlogged_write(const reg_t val) noexcept {
1787     // For every bit in an mstateen CSR that is zero (whether read-only zero or set to zero),
1788     // the same bit appears as read-only zero in the matching hstateen and sstateen CSRs
1789     const reg_t mask = this->mask & state->mstateen[index]->read();
1790     return basic_csr_t::unlogged_write((basic_csr_t::read() & ~mask) | (val & mask));
1791 }
1792
1793 void hstateen_csr_t::verify_permissions(insn_t insn, bool write) const {
1794     if ((state->prv < PRV_M) && !(state->mstateen[index]->read() & MSTATEEN_HSTATEEN))
1795         throw trap_illegal_instruction(insn.bits());
1796     basic_csr_t::verify_permissions(insn, write);
1797 }
1798
1799 // implement class sstateen_csr_t
1800 sstateen_csr_t::sstateen_csr_t(processor_t* const proc, const reg_t addr, const reg_t mask, const reg_t init, uint8_t index):
1801     hstateen_csr_t(proc, addr, mask, init, index) {
1802 }
1803
1804 reg_t sstateen_csr_t::read() const noexcept {
1805     // For every bit in an mstateen CSR that is zero (whether read-only zero or set to zero),
1806     // the same bit appears as read-only zero in the matching hstateen and sstateen CSRs
1807     // For every bit in an hstateen CSR that is zero (whether read-only zero or set to zero),
1808     // the same bit appears as read-only zero in sstateen when accessed in VS-mode
1809     if (state->v)
1810         return hstateen_csr_t::read() & state->hstateen[index]->read();
1811     else
1812         return hstateen_csr_t::read();
1813 }
1814
1815 bool sstateen_csr_t::unlogged_write(const reg_t val) noexcept {
1816     // For every bit in an mstateen CSR that is zero (whether read-only zero or set to zero),
1817     // the same bit appears as read-only zero in the matching hstateen and sstateen CSRs
1818     // For every bit in an hstateen CSR that is zero (whether read-only zero or set to zero),
1819     // the same bit appears as read-only zero in sstateen when accessed in VS-mode
1820     if (state->v)
1821         return hstateen_csr_t::unlogged_write(val & state->hstateen[index]->read());
1822     else
1823         return hstateen_csr_t::unlogged_write(val);
1824 }
1825
1826 void sstateen_csr_t::verify_permissions(insn_t insn, bool write) const {
1827     hstateen_csr_t::verify_permissions(insn, write);
1828     if (state->v && !(state->hstateen[index]->read() & HSTATEEN_SSTATEEN))
1829         throw trap_virtual_instruction(insn.bits());
1830 }
1831
1832 // implement class senvcfg_csr_t
1833 senvcfg_csr_t::senvcfg_csr_t(processor_t* const proc, const reg_t addr, const reg_t mask,

```



```

1837         const reg_t init):
1838     envcfg_csr_t(proc, addr, mask, init) {
1839 }
1840
1841 void senvcfg_csr_t::verify_permissions(insn_t insn, bool
write) const {
1842     if (proc->extension_enabled(EXT_SMSTATEEN)) {
1843         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_HENVCFG))
1844             throw trap_illegal_instruction(insn.bits());
1845
1846         if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_SENVCFG))
1847             throw trap_virtual_instruction(insn.bits());
1848     }
1849
1850     masked_csr_t::verify_permissions(insn, write);
1851 }
1852
1853 // senvcfg.sse is read_only 0 when menvcfg.sse = 0
1854 reg_t envcfg_csr_t::read() const noexcept {
1855     reg_t envcfg = state->v ? state->henvcfg->read() : state-
>menvcfg->read();
1856     return masked_csr_t::read() & ~(envcfg & MENVCFG_SSE ? 0 :
MENVCFG_SSE);
1857 }
1858
1859 bool senvcfg_csr_t::unlogged_write(const reg_t val) noexcept {
1860     reg_t envcfg = state->v ? state->henvcfg->read() : state-
>menvcfg->read();
1861     const reg_t mask = envcfg | ~MENVCFG_SSE;
1862     return envcfg_csr_t::unlogged_write((masked_csr_t::read() &
~mask) | (val & mask));
1863 }
1864
1865 void henvcfg_csr_t::verify_permissions(insn_t insn, bool
write) const {
1866     if (proc->extension_enabled(EXT_SMSTATEEN)) {
1867         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_HENVCFG))
1868             throw trap_illegal_instruction(insn.bits());
1869     }
1870
1871     masked_csr_t::verify_permissions(insn, write);
1872 }
1873
1874 bool henvcfg_csr_t::unlogged_write(const reg_t val) noexcept {
1875     const reg_t mask = menvcfg->read() | ~(MENVCFG_PBMTE | MEN-
VCFG_STCE | MENVCFG_ADUE | MENVCFG_DTE | MENVCFG_SSE);
1876     return envcfg_csr_t::unlogged_write((masked_csr_t::read() &
~mask) | (val & mask));
1877 }
1878
1879 stimecmp_csr_t::stimecmp_csr_t(processor_t* const proc, const
reg_t addr, const reg_t imask):
1880     basic_csr_t(proc, addr, 0), intr_mask(imask) {
1881 }
1882
1883 bool stimecmp_csr_t::unlogged_write(const reg_t val) noexcept
{
1884     const reg_t mask = ((state->menvcfg->read() & MEN-
VCFG_STCE) ? MIP_STIP : 0) | ((state->henvcfg->read() & HEN-
VCFG_STCE) ? MIP_VSTIP : 0);
1885     state->mip->backdoor_write_with_mask(mask, state->time-
>read() >= val ? intr_mask : 0);
1886     return basic_csr_t::unlogged_write(val);
1887 }
1888
1889 void stimecmp_csr_t::verify_permissions(insn_t insn, bool
write) const {
1890     if (!(state->menvcfg->read() & MENVCFG_STCE)) {
1891         // access to (v)stimecmp with MENVCFG.STCE = 0
1892         if (state->prv < PRV_M)
1893             throw trap_illegal_instruction(insn.bits());
1894     }
1895
1896     state->time_proxy->verify_permissions(insn, false);
1897
1898     if (state->v && !(state->henvcfg->read() & HENVCFG_STCE)) {
1899         // access to vstimecmp with MENVCFG.STCE = 1 and HEN-
VCFG.STCE = 0 when V = 1
1900         throw trap_virtual_instruction(insn.bits());
1901     }
1902
1903     basic_csr_t::verify_permissions(insn, write);
1904
1905     if (proc->extension_enabled_const(EXT_SSAIA) && proc->exten-
sion_enabled('H')) {
1906         if ((state->csrmap[CSR_HVICTL]->read() & HVICTL_VTI) &&
state->v && write)
1907             throw trap_virtual_instruction(insn.bits());
1908     }
1909 }
1910
1911 virtual-
ized_with_special_permission_csr_t::virtualized_with_special_permis-
sion_csr_t(processor_t* const proc, csr_t_p orig, csr_t_p virt):
1912     virtualized_csr_t(proc, orig, virt) {
1913 }

```

```

1914
1915 void virtual-
ized_with_special_permission_csr_t::verify_permissions(insn_t insn,
bool write) const {
1916     orig_csr->verify_permissions(insn, write);
1917 }
1918
1919 scountovf_csr_t::scountovf_csr_t(processor_t* const proc,
const reg_t addr):
1920     csr_t(proc, addr) {
1921 }
1922
1923 void scountovf_csr_t::verify_permissions(insn_t insn, bool
write) const {
1924     if (!proc->extension_enabled(EXT_SSCOPMF))
1925         throw trap_illegal_instruction(insn.bits());
1926
1927     if (proc->extension_enabled('H') &&
(proc->extension_enabled_const(EXT_SMCDELEG) || proc->ex-
tension_enabled(EXT_SSCCFG))
1928     ) {
1929         if (state->v && (state->menvcfg->read() & MENVCFG_CDE)) {
1930             throw trap_virtual_instruction(insn.bits());
1931         }
1932     }
1933     csr_t::verify_permissions(insn, write);
1934 }
1935
1936 reg_t scountovf_csr_t::read() const noexcept {
1937     reg_t val = 0;
1938     for (reg_t i = 0; i < N_HPM_COUNTERS; ++i) {
1939         bool of = state->mevent[i]->read() & MHPMEVENT_OF;
1940         val |= of << (i + FIRST_HPM_COUNTER);
1941     }
1942 }
1943
1944 /* In M-mode, scountovf bit X is always readable. */
1945
1946 /* In S/HS-mode, scountovf bit X is readable when mcounteren
bit X is set, */
1947 /* and otherwise reads as zero. Similarly, in VS mode,
scountovf bit X is readable */
1948 /* when mcounteren bit X and hcounteren bit X are both set,
and otherwise reads as zero. */
1949 if (state->prv < PRV_M)
1950     val &= state->mcounteren->read();
1951 if (state->v)
1952     val &= state->hcounteren->read();
1953 return val;
1954 }
1955
1956 bool scountovf_csr_t::unlogged_write(const reg_t UNUSED val)
noexcept {
1957     /* this function is unused */
1958     return false;
1959 }
1960
1961 // implement class jvt_csr_t
1962 jvt_csr_t::jvt_csr_t(processor_t* const proc, const reg_t
addr, const reg_t init):
1963     basic_csr_t(proc, addr, init) {
1964 }
1965
1966 void jvt_csr_t::verify_permissions(insn_t insn, bool write)
const {
1967     basic_csr_t::verify_permissions(insn, write);
1968
1969     if (!proc->extension_enabled(EXT_ZCMT))
1970         throw trap_illegal_instruction(insn.bits());
1971
1972     if (proc->extension_enabled(EXT_SMSTATEEN)) {
1973         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
SSTATEEN0_JVT))
1974             throw trap_illegal_instruction(insn.bits());
1975
1976         if (state->v && !(state->hstateen[0]->read() &
SSTATEEN0_JVT))
1977             throw trap_virtual_instruction(insn.bits());
1978
1979         if ((proc->extension_enabled('S') && state->prv < PRV_S)
&& !(state->sstateen[0]->read() & SSTATEEN0_JVT)) {
1980             if (state->v)
1981                 throw trap_virtual_instruction(insn.bits());
1982             else
1983                 throw trap_illegal_instruction(insn.bits());
1984         }
1985     }
1986
1987 virtual-
ized_indirect_csr_t::virtualized_indirect_csr_t(processor_t* const
proc, csr_t_p orig, csr_t_p virt):
1988     virtualized_csr_t(proc, orig, virt) {
1989 }
1990
1991 void virtualized_indirect_csr_t::verify_permissions(insn_t
insn, bool write) const {
1992     virtualized_csr_t::verify_permissions(insn, write);
1993     if (state->v)
1994         virt_csr->verify_permissions(insn, write);
1995     else

```

```

1996     orig_csr->verify_permissions(insn, write);
1997 }
1998
1999 sscsrind_reg_csr_t::sscsrind_reg_csr_t(processor_t* const
proc, const reg_t addr, csr_t_p iselect) :
2000     csr_t(proc, addr),
2001     iselect(std::move(iselect)) {
2002 }
2003
2004 void sscsrind_reg_csr_t::verify_permissions(insn_t insn, bool
write) const {
2005     if (proc->extension_enabled(EXT_SMSTATEEN)) {
2006         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_CSRIND))
2007             throw trap_illegal_instruction(insn.bits());
2008
2009         if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_CSRIND))
2010             throw trap_virtual_instruction(insn.bits());
2011     }
2012
2013     if (state->v && state->prv == PRV_U) {
2014         throw trap_virtual_instruction(insn.bits());
2015     }
2016
2017     // Don't call base verify_permission for VS registers
remapped to S-mode
2018     if (insn.csr() == address)
2019         csr_t::verify_permissions(insn, write);
2020
2021     if (proc->extension_enabled(EXT_SMCDELEG)) {
2022         auto iselect_val = iselect->read();
2023         if (iselect_val >= SISELECT_SMCDELEG_START && iselect_val
<= SISELECT_SMCDELEG_END) {
2024             if (address >= CSR_VSIREG && address <= CSR_VSIREG6) {
2025                 if (state->v) {
2026                     // An attempt to access any vsireg* from M or S mode
raises an illegal instruction exception.
2027                     throw trap_illegal_instruction(insn.bits());
2028                 } else {
2029                     if (state->prv == PRV_S) {
2030                         // An attempt from VS-mode to access any vsireg
raises an illegal instruction
2031                         // exception if menvcfg.CDE = 0, or a virtual in-
struction exception if menvcfg.CDE = 1
2032                         if ((state->menvcfg->read() & MENVCFG_CDE) != MEN-
VCFG_CDE) {
2033                             throw trap_illegal_instruction(insn.bits());
2034                         } else {
2035                             throw trap_virtual_instruction(insn.bits());
2036                         }
2037                     } else {
2038                         throw trap_virtual_instruction(insn.bits());
2039                     }
2040                 }
2041             }
2042             if (address >= CSR_SIREG && address <= CSR_SIREG6) {
2043                 // attempts to access any sireg* when menvcfg.CDE = 0;
2044                 if ((state->menvcfg->read() & MENVCFG_CDE) != MEN-
VCFG_CDE) {
2045                     if (!state->v) {
2046                         throw trap_illegal_instruction(insn.bits());
2047                     } else {
2048                         if (state->prv == PRV_S) {
2049                             // An attempt from VS-mode to access any sireg*
causes illegal instruction exception if menvcfg.CDE = 0
2050                             throw trap_illegal_instruction(insn.bits());
2051                         } else {
2052                             throw trap_virtual_instruction(insn.bits());
2053                         }
2054                     }
2055                 } else {
2056                     // menvcfg.CDE = 1;
2057                     if (state->v) {
2058                         // An attempt from VS-mode to access any sireg*
causes a virtual instruction exception if menvcfg.CDE = 1
2059                         throw trap_virtual_instruction(insn.bits());
2060                     }
2061                     // counter selected by siselect is not delegated to
S-mode (the corresponding bit in mcounteren = 0).
2062                     auto iselect_addr = iselect->read();
2063                     if (iselect_addr >= SISELECT_SMCDELEG_START && ise-
lect_addr <= SISELECT_SMCDELEG_END) {
2064                         reg_t counter_id_offset = iselect_addr - SISE-
LECT_SMCDELEG_START;
2065                         if (!state->mcounteren->read() & (1U <<
counter_id_offset))) {
2066                             if (!state->v) {
2067                                 throw trap_illegal_instruction(insn.bits());
2068                             } else {
2069                                 throw trap_virtual_instruction(insn.bits());
2070                             }
2071                         }
2072                     }
2073                 }
2074             }
2075         }
2076     }
2077 }

```

```

2078     csr_t_p proxy_csr = get_reg();
2079     if (proxy_csr == nullptr) {
2080         // An attempt from VS-mode to access any inaccessible
sireg* causes virtual instruction exception when AIA is enabled
2081         if (proc->extension_enabled_const(EXT_SMAIA)) {
2082             if (state->prv == PRV_S && state->v) {
2083                 throw trap_virtual_instruction(insn.bits());
2084             } else {
2085                 throw trap_illegal_instruction(insn.bits());
2086             }
2087         } else {
2088             throw trap_illegal_instruction(insn.bits());
2089         }
2090     }
2091     proxy_csr->verify_permissions(insn, write);
2092 }
2093
2094
2095 reg_t sscsrind_reg_csr_t::read() const noexcept {
2096     csr_t_p target_csr = get_reg();
2097     if (target_csr != nullptr) {
2098         return target_csr->read();
2099     }
2100     return 0;
2101 }
2102
2103 bool sscsrind_reg_csr_t::unlogged_write(const reg_t val) noex-
cept {
2104     csr_t_p proxy_csr = get_reg();
2105     if (proxy_csr != nullptr) {
2106         proxy_csr->write(val);
2107     }
2108     return false;
2109 }
2110
2111 // Returns the actual CSR that maps to value in *siselect or
nullptr if no mapping exists
2112 csr_t_p sscsrind_reg_csr_t::get_reg() const noexcept {
2113     auto proxy = ireg_proxy;
2114     auto isel = iselect->read();
2115     auto it = proxy.find(isel);
2116     return it != proxy.end() ? it->second : nullptr;
2117 }
2118
2119 void sscsrind_reg_csr_t::add_ireg_proxy(const reg_t ise-
lect_value, csr_t_p csr) {
2120     ireg_proxy[iselect_value] = csr;
2121 }
2122
2123 smcntrpmf_csr_t::smcntrpmf_csr_t(processor_t* const proc,
const reg_t addr) : basic_csr_t(proc, addr, 0) {
2124 }
2125
2126 reg_t smcntrpmf_csr_t::read_prev() const noexcept {
2127     reg_t val = prev_val.value_or(read());
2128     return val;
2129 }
2130
2131 void smcntrpmf_csr_t::reset_prev() noexcept {
2132     prev_val.reset();
2133 }
2134
2135 bool smcntrpmf_csr_t::unlogged_write(const reg_t val) noexcept
{
2136     prev_val = read();
2137
2138     const reg_t mask = !proc->exten-
sion_enabled_const(EXT_SMCNTRPMF) ? 0 :
2139         MHPMEVENT_MINH |
2140         (proc->extension_enabled_const('S') ? MHPMEVENT_SINH : 0)
|
2141         (proc->extension_enabled_const('U') ? MHPMEVENT_UINH : 0)
|
2142         (proc->extension_enabled('H') ? MHPMEVENT_VSINH | MHP-
MEVENT_VUINH : 0);
2143
2144     return basic_csr_t::unlogged_write(val & mask);
2145 }
2146
2147 srmcfg_csr_t::srmcfg_csr_t(processor_t* const proc, const
reg_t addr, const reg_t mask, const reg_t init):
2148     masked_csr_t(proc, addr, mask, init) {
2149 }
2150
2151 void srmcfg_csr_t::verify_permissions(insn_t insn, bool write
UNUSED) const {
2152     if (!proc->extension_enabled(EXT_SSQ0SID))
2153         throw trap_illegal_instruction(insn.bits());
2154
2155     if (proc->extension_enabled(EXT_SMSTATEEN)) {
2156         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_PRIV114))
2157             throw trap_illegal_instruction(insn.bits());
2158     }
2159
2160     if (state->v)
2161         throw trap_virtual_instruction(insn.bits());
2162
2163     if (state->prv < PRV_S) {

```

```

2164     throw trap_illegal_instruction(insn.bits());
2165 }
2166 }
2167
2168 hvip_csr_t::hvip_csr_t(processor_t* const proc, const reg_t
addr, const reg_t init):
2169     basic_csr_t(proc, addr, init) {
2170 }
2171
2172 reg_t hvip_csr_t::read() const noexcept {
2173     return basic_csr_t::read() | (state->mip->read() & MIP_VS-
SIP); // hvip.VSSIP is an alias of mip.VSSIP
2174 }
2175
2176 bool hvip_csr_t::unlogged_write(const reg_t val) noexcept {
2177     state->mip->write_with_mask(MIP_VSSIP, val); // hvip.VSSIP
is an alias of mip.VSSIP
2178     return basic_csr_t::unlogged_write(val & (MIP_VSEIP |
MIP_VSTIP));
2179 }
2180
2181 ssp_csr_t::ssp_csr_t(processor_t* const proc, const reg_t
addr, const reg_t mask, const reg_t init):
2182     masked_csr_t(proc, addr, mask, init) {
2183 }
2184
2185 void ssp_csr_t::verify_permissions(insn_t insn, bool write)
const {
2186     masked_csr_t::verify_permissions(insn, write);
2187     auto p = proc;
2188     require_envcfg(SSE);
2189 }
2190
2191 mtval2_csr_t::mtval2_csr_t(processor_t* const proc, const
reg_t addr):
2192     hypervisor_csr_t(proc, addr) {
2193 }
2194
2195 void mtval2_csr_t::verify_permissions(insn_t insn, bool write)
const {
2196     basic_csr_t::verify_permissions(insn, write);
2197     if (!proc->extension_enabled('H') && !proc->exten-
sion_enabled(EXT_SSDBLTRP))
2198         throw trap_illegal_instruction(insn.bits());
2199 }
2200
2201 hstatus_csr_t::hstatus_csr_t(processor_t* const proc, const
reg_t addr):
2202     basic_csr_t(proc, addr, set_field((reg_t)0, HSTATUS_VSXL,
xlen_to_uhl(proc->get_const_xlen())) {
2203 }
2204
2205 bool hstatus_csr_t::unlogged_write(const reg_t val) noexcept {
2206     const reg_t mask = (proc->extension_enabled(EXT_SVUKTE) ?
HSTATUS_HUKTE : 0)
2207     | HSTATUS_VTSR | HSTATUS_VTW
2208     | (proc->has_mmu() ? HSTATUS_VTVM : 0)
2209     | (proc->extension_enabled(EXT_SSNPM) ? HSTATUS_HUPMM : 0)
2210     | (proc->extension_enabled_const(EXT_SSAIA) ?
HSTATUS_VGEIN : 0)
2211     | HSTATUS_HU | HSTATUS_SPVP | HSTATUS_SPV | HSTATUS_GVA;
2212
2213     const reg_t pmm_reserved = 1; // Reserved value of msec-
cfg.PMM
2214     reg_t pmm = get_field(val, HSTATUS_HUPMM);
2215     const reg_t adjusted_val = set_field(val, HSTATUS_HUPMM,
pmm != pmm_reserved ? pmm : 0);
2216
2217     reg_t new_hstatus = (read() & ~mask) | (adjusted_val &
mask);
2218     if (get_field(new_hstatus, HSTATUS_HUPMM) !=
get_field(read(), HSTATUS_HUPMM))
2219         proc->get_mmu()->flush_tlb();
2220
2221     // VGEIN is WLR
2222     if (proc->extension_enabled_const(EXT_SSAIA)) {
2223         reg_t old_vgein = get_field(read(), HSTATUS_VGEIN);
2224         reg_t new_vgein = get_field((reg_t)val, HSTATUS_VGEIN);
2225         if (new_vgein && !proc->imsic->vgein_valid(new_vgein)) {
2226             new_vgein = old_vgein;
2227         }
2228         new_hstatus = set_field(new_hstatus, HSTATUS_VGEIN,
new_vgein);
2229         // update_mip() needs the new VGEIN, so hstatus must be
updated first
2230         bool ret = basic_csr_t::unlogged_write(new_hstatus);
2231         // if vgein = 0, HS controls VSEIP; otherwise IMSIC con-
trols it
2232         if (new_vgein && new_vgein != old_vgein)
2233             proc->imsic->vs[new_vgein]->update_mip();
2234         return ret;
2235     }
2236     return basic_csr_t::unlogged_write(new_hstatus);
2237 }
2238
2239 scntinhibit_csr_t::scntinhibit_csr_t(processor_t* const proc,
const reg_t addr, csr_t_p mcountinhibit):
2240     basic_csr_t(proc, addr, mcountinhibit->read()) {
2241 }
2242

```

```

2243 void scntinhibit_csr_t::verify_permissions(insn_t insn, bool
write) const {
2244     if (insn.csr() == address) {
2245         csr_t::verify_permissions(insn, write);
2246     }
2247
2248     if ((state->menvcfg->read() & MENVCFG_CDE) != MENVCFG_CDE) {
2249         throw trap_illegal_instruction(insn.bits());
2250     }
2251 }
2252
2253 bool scntinhibit_csr_t::unlogged_write(const reg_t val) noex-
cept {
2254     state->mcountinhibit->write(state->mcounteren->read() &
val);
2255     return true;
2256 }
2257
2258 reg_t scntinhibit_csr_t::read() const noexcept {
2259     return state->mcounteren->read() & state->mcountinhibit-
>read();
2260 }
2261
2262 mtopi_csr_t::mtopi_csr_t(processor_t* const proc, const reg_t
addr):
2263     csr_t(proc, addr) {
2264 }
2265
2266 reg_t mtopi_csr_t::read() const noexcept {
2267     reg_t enabled_interrupts = state->mip->read() & state->mie-
>read() & ~state->mideleg->read();
2268     if (!enabled_interrupts)
2269         return 0; // no enabled pending interrupt to M-mode
2270
2271     reg_t selected_interrupt = proc->se-
lect_an_interrupt_with_default_priority(enabled_interrupts);
2272     reg_t identity = ctz(selected_interrupt);
2273     return set_field((reg_t)1, MTOPI_IID, identity); // IPRIO
always 1 if iprio array is R00
2274 }
2275
2276 bool mtopi_csr_t::unlogged_write(const reg_t UNUSED val) noex-
cept {
2277     return false;
2278 }
2279
2280 mvip_csr_t::mvip_csr_t(processor_t* const proc, const reg_t
addr, const reg_t init):
2281     basic_csr_t(proc, addr, init) {
2282 }
2283
2284 reg_t mvip_csr_t::read() const noexcept {
2285     const reg_t val = basic_csr_t::read();
2286     const reg_t mvien = state->mvien->read();
2287     const reg_t mip = state->mip->read();
2288     const reg_t menvcfg = state->menvcfg->read();
2289     return 0
2290         | (val & MIP_SEIP)
2291         | ((menvcfg & MENVCFG_STCE) ? 0 : (mip & MIP_STIP))
2292         | ((mvien & MIP_SSSIP) ? val : mip) & MIP_SSSIP
2293         ;
2294 }
2295
2296 bool mvip_csr_t::unlogged_write(const reg_t val) noexcept {
2297     if (!(state->menvcfg->read() & MENVCFG_STCE))
2298         state->mip->write_with_mask(MIP_STIP, val); // mvip.STIP
is an alias of mip.STIP when mip.STIP is writable
2299     if (!(state->mvien->read() & MIP_SSSIP))
2300         state->mip->write_with_mask(MIP_SSSIP, val); // mvip.SSSIP
is an alias of mip.SSSIP when mvien.SSSIP=0
2301
2302     const reg_t new_val = (val & MIP_SEIP) | (((state->mvien-
>read() & MIP_SSSIP) ? val : basic_csr_t::read()) & MIP_SSSIP);
2303     return basic_csr_t::unlogged_write(new_val);
2304 }
2305
2306 void mvip_csr_t::write_with_mask(const reg_t mask, const reg_t
val) noexcept {
2307     basic_csr_t::unlogged_write((basic_csr_t::read() & ~mask) |
(val & mask));
2308     log_write();
2309 }
2310
2311 nonvirtual_stopi_csr_t::nonvirtual_stopi_csr_t(processor_t*
const proc, const reg_t addr):
2312     csr_t(proc, addr) {
2313 }
2314
2315 void nonvirtual_stopi_csr_t::verify_permissions(insn_t insn,
bool write) const {
2316     if (proc->extension_enabled(EXT_SMSTATEEN)) {
2317         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_AIA))
2318             throw trap_illegal_instruction(insn.bits());
2319
2320         if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_AIA))
2321             throw trap_virtual_instruction(insn.bits());
2322     }

```

```

2323
2324   csr_t::verify_permissions(insn, write);
2325 }
2326
2327 reg_t nonvirtual_stopi_csr_t::read() const noexcept {
2328   reg_t enabled_interruptions = state->nonvirtual_sip->read() &
state->nonvirtual_sie->read();
2329   // include hypervisor interrupts for HS stopi
2330   enabled_interruptions |= state->hip->read() & state->hie-
>read();
2331   // mask out delegated interrupts
2332   enabled_interruptions &= ~state->hideleg->read();
2333   if (!enabled_interruptions)
2334     return 0; // no enabled pending interrupt to S-mode
2335
2336   reg_t selected_interrupt = proc->se-
lect_an_interrupt_with_default_priority(enabled_interruptions);
2337   reg_t identity = ctz(selected_interrupt);
2338   return set_field((reg_t)1, MTOPI_IID, identity); // IPRIO
always 1 if iprio array is R00
2339 }
2340
2341 bool nonvirtual_stopi_csr_t::unlogged_write(const reg_t UNUSED
val) noexcept {
2342   return false;
2343 }
2344
2345 inaccessible_csr_t::inaccessible_csr_t(processor_t* const
proc, const reg_t addr):
2346   csr_t(proc, addr) {
2347 }
2348
2349 void inaccessible_csr_t::verify_permissions(insn_t insn, bool
UNUSED write) const {
2350   if (state->v)
2351     throw trap_virtual_instruction(insn.bits());
2352   else
2353     throw trap_illegal_instruction(insn.bits());
2354 }
2355
2356 vstopi_csr_t::vstopi_csr_t(processor_t* const proc, const
reg_t addr):
2357   csr_t(proc, addr) {
2358 }
2359
2360 void vstopi_csr_t::verify_permissions(insn_t insn, bool write)
const {
2361   if (proc->extension_enabled(EXT_SMSTATEEN)) {
2362     if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_AIA))
2363       throw trap_illegal_instruction(insn.bits());
2364     if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_AIA))
2365       throw trap_virtual_instruction(insn.bits());
2366   }
2367   csr_t::verify_permissions(insn, write);
2368 }
2369
2370 reg_t vstopi_csr_t::read() const noexcept {
2371   reg_t hvictl = state->hvictl->read();
2372   bool vti = hvictl & HVICTL_VTI;
2373   reg_t iid = get_field(hvictl, HVICTL_IID);
2374   bool dpr = hvictl & HVICTL_DPR;
2375   bool ipriom = hvictl & HVICTL_IPRIOM;
2376   reg_t iprio = get_field(hvictl, HVICTL_IPRIO);
2377
2378   reg_t enabled_interruptions = state->mip->read() & state->mie-
>read() & state->hideleg->read();
2379   enabled_interruptions >= 1; // VSSIP -> SSIP, etc
2380   reg_t vgein = get_field(state->hstatus->read(),
HSTATUS_VGEIN);
2381
2382   reg_t virtual_sei_priority = (vgein == 0 && iid == IRQ_S_EXT
&& iprio != 0) ? iprio : 255; // vstopi.IPRI0 is 255 for priority
number 256
2383
2384   reg_t identity, priority;
2385   if (vti) {
2386     if (!(enabled_interruptions & MIP_SEIP) && iid == IRQ_S_EXT)
return 0;
2387     identity = ((enabled_interruptions & MIP_SEIP) && (iid ==
IRQ_S_EXT || dpr)) ? IRQ_S_EXT : iid;
2388     priority = (identity == IRQ_S_EXT) ? virtu-
al_sei_priority : ((iprio != 0 || !dpr) ? iprio : 255);
2389   } else {
2390     if (!enabled_interruptions)
2391       return 0; // no enabled pending interrupt to VS-mode
2392     reg_t selected_interrupt = proc->se-
lect_an_interrupt_with_default_priority(enabled_interruptions);
2393     identity = ctz(selected_interrupt);
2394     priority = (identity == IRQ_S_EXT) ? virtu-
al_sei_priority : 255; // vstopi.IPRI0 is 255 for interrupt with de-
fault priority lower than VSEI
2395   }
2396 }

```

```

2400   return set_field((reg_t)(ipriom ? priority : 1), MTOPI_IID,
identity);
2401 }
2402
2403 bool vstopi_csr_t::unlogged_write(const reg_t UNUSED val) no-
except {
2404   return false;
2405 }
2406
2407 siselect_csr_t::siselect_csr_t(processor_t* const proc, const
reg_t addr, const reg_t init):
2408   basic_csr_t(proc, addr, init) {
2409 }
2410
2411 void siselect_csr_t::verify_permissions(insn_t insn, bool
write) const {
2412   if (proc->extension_enabled(EXT_SMSTATEEN)) {
2413     if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_CSRIND))
2414       throw trap_illegal_instruction(insn.bits());
2415     if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_CSRIND))
2416       throw trap_virtual_instruction(insn.bits());
2417   }
2418   basic_csr_t::verify_permissions(insn, write);
2419 }
2420
2421 aia_csr_t::aia_csr_t(processor_t* const proc, const reg_t
addr, const reg_t mask, const reg_t init):
2422   masked_csr_t(proc, addr, mask, init) {
2423 }
2424
2425 void aia_csr_t::verify_permissions(insn_t insn, bool write)
const {
2426   if (proc->extension_enabled(EXT_SMSTATEEN)) {
2427     if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_AIA))
2428       throw trap_illegal_instruction(insn.bits());
2429     if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_AIA))
2430       throw trap_virtual_instruction(insn.bits());
2431   }
2432   basic_csr_t::verify_permissions(insn, write);
2433 }
2434
2435 hgeip_csr_t::hgeip_csr_t(processor_t* const proc, const reg_t
addr) : csr_t(proc, addr) {
2436 }
2437
2438 reg_t hgeip_csr_t::read() const noexcept {
2439   // scan through all VGEINs
2440   reg_t v = 0;
2441   for (auto &i: proc->imsic->vs) {
2442     if (i.second->delivery() && i.second->topei())
2443       v |= reg_t(1) << i.first;
2444   }
2445   return v;
2446 }
2447
2448 bool hgeip_csr_t::unlogged_write(const reg_t UNUSED val) noex-
cept {
2449   // read-only register
2450   return false;
2451 }
2452
2453 hgeie_csr_t::hgeie_csr_t(processor_t* const proc, const reg_t
addr, const reg_t geilen) : masked_csr_t(proc, addr, ((reg_t(1) <<
geilen) - 1) << 1, 0) {
2454 }
2455
2456 bool hgeie_csr_t::unlogged_write(const reg_t val) noexcept {
2457   bool sgeip = val & proc->get_state()->hgeip->read();
2458   // update mip.SGEIP if the hypervisor traps guest SEIP to
itself
2459   state->mip->backdoor_write_with_mask(MIP_SGEIP, sgeip ?
MIP_SGEIP : 0);
2460   return masked_csr_t::unlogged_write(val);
2461 }
2462
2463 aia_ireg_proxy_csr_t::aia_ireg_proxy_csr_t(processor_t* const
proc, const reg_t addr, csr_t_p iselect) : csr_t(proc, addr), ise-
lect(std::move(iselect)), vs(false), ireg_map(nullptr) {
2464   switch (address) {
2465     case CSR_MIREG: case CSR_SIREG:
2466       ireg_map = proc->imsic->register_iprio(proc, address);
2467       break;
2468     case CSR_VSIREG:
2469       // Virtualized ireg (vsireg) does not have a ireg_map
because it changes based on hstatus.vgein
2470       vs = true;
2471       break;
2472     default:
2473       // Unexpected *ireg address
2474       assert(false);
2475   }
2476 }

```

```

2480 }
2481
2482 csr_t_p aia_ireg_proxy_csr_t::get_reg() const noexcept {
2483     reg_t reg = iselect->read();
2484     if (vs) {
2485         // vsireg - look up by vgein
2486         reg_t vgein = get_field(state->hstatus->read(),
HSTATUS_VGEIN);
2487         return vgein ? proc->imsic->get_vs_reg(vgein, reg) : null-
ptr;
2488     }
2489     // !vsireg
2490     return ireg_map->count(reg) ? (*ireg_map)[reg] : nullptr;
2491 }
2492
2493 reg_t aia_ireg_proxy_csr_t::read() const noexcept {
2494     csr_t_p reg = get_reg();
2495     return reg ? reg->read() : 0;
2496 }
2497
2498 void aia_ireg_proxy_csr_t::verify_permissions(insn_t insn,
bool write) const {
2499     reg_t isel = iselect->read();
2500     // skip verify_permissions chaining on a VS reg because the
address is remapped
2501     if (!vs) {
2502         // If the hart has an IMSIC, then when bit 9 of mvien is
one, attempts from S-mode to explicitly
2503         // access the supervisor-level interrupt file raise an il-
legal instruction exception.
2504         if (state->prv < PRV_M && (state->mvien->read() &
MIP_SEIP) && isel >= SISELECT_IMSIC && isel <= SISELECT_IMSIC_TOP)
2505             throw trap_illegal_instruction(insn.bits());
2506         csr_t::verify_permissions(insn, write);
2507     }
2508     if (get_reg() == nullptr) {
2509         if (state->v)
2510             throw trap_virtual_instruction(insn.bits());
2511         else
2512             throw trap_illegal_instruction(insn.bits());
2513     }
2514     if (proc->extension_enabled(EXT_SMSTATEEN)) {
2515         // if iselect >= IMSIC and xSTATEEN_IMSIC not set
2516         if (!state->v && state->prv < PRV_M && isel >= SISE-
LECT_IMSIC && isel <= SISELECT_IMSIC_TOP && !(state->mstateen[0]-
>read() & MSTATEEN0_IMSIC))
2517             throw trap_illegal_instruction(insn.bits());
2518         if (state->v && isel >= VSISELECT_IMSIC && isel <= VSISE-
LECT_IMSIC_TOP && !(state->hstateen[0]->read() & HSTATEEN0_IMSIC))
2519             throw trap_virtual_instruction(insn.bits());
2520     }
2521     // if VS & invalid VGEIN
2522     if (vs && !get_reg()) {
2523         if (state->v)
2524             throw trap_virtual_instruction(insn.bits());
2525         else
2526             throw trap_illegal_instruction(insn.bits());
2527     }
2528 }
2529
2530 bool aia_ireg_proxy_csr_t::unlogged_write(const reg_t val) no-
except {
2531     csr_t_p reg = get_reg();
2532     if (!reg)
2533         return false;
2534     reg->write(val);
2535     return true;
2536 }
2537
2538 csrmap_t_p aia_ireg_proxy_csr_t::get_regmap(reg_t vgein) {
2539     if (!vs)
2540         return ireg_map;
2541     return proc->imsic->get_vs_regmap(vgein ? vgein :
get_field(state->hstatus->read(), HSTATUS_VGEIN));
2542 }
2543
2544 topei_csr_t::topoi_csr_t(processor_t* const proc, const reg_t
addr, imsic_file_t_p const imsic) : csr_t(proc, addr, imsic(imsic)
{
2545 }
2546
2547 imsic_file_t_p topei_csr_t::get_imsic() const noexcept {

```

```

2548 // non-virtualized registers have pointers to IMSIC
2549 if (imsic)
2550     return imsic;
2551
2552 // Virtualized IMSIC depends on hstatus.vgein
2553 reg_t vgein = get_field(state->hstatus->read(),
HSTATUS_VGEIN);
2554 if (!vgein || !proc->imsic->vgein_valid(vgein))
2555     return nullptr;
2556 return proc->imsic->vs[vgein];
2557 }
2558
2559 reg_t topei_csr_t::read() const noexcept {
2560     imsic_file_t_p p = get_imsic();
2561     if (!p)
2562         return 0;
2563
2564     reg_t iid = p->topei();
2565     reg_t v = 0;
2566     v = set_field(v, IMSIC_TOPI_IPRIO, iid);
2567     v = set_field(v, IMSIC_TOPI_IID, iid);
2568     return v;
2569 }
2570
2571 bool topei_csr_t::unlogged_write(const reg_t UNUSED val) noex-
cept {
2572     imsic_file_t_p p = get_imsic();
2573     if (!p)
2574         return false;
2575     p->claimei(p->topei());
2576     return true;
2577 }
2578
2579 void nonvirtual_stopei_csr_t::verify_permissions(insn_t insn,
bool write) const {
2580     if (proc->extension_enabled(EXT_SMSTATEEN)) {
2581         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_IMSIC))
2582             throw trap_illegal_instruction(insn.bits());
2583         if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_IMSIC))
2584             throw trap_virtual_instruction(insn.bits());
2585     }
2586
2587     // If the hart has an IMSIC, then when bit 9 of mvien is
one, attempts from S-mode to explicitly
2588     // access the supervisor-level interrupt file raise an il-
legal instruction exception.
2589     if (state->prv < PRV_M && (state->mvien->read() & MIP_SEIP))
2590         throw trap_illegal_instruction(insn.bits());
2591
2592     csr_t::verify_permissions(insn, write);
2593 }
2594
2595 void vstopei_csr_t::verify_permissions(insn_t insn, bool
write) const {
2596     if (proc->extension_enabled(EXT_SMSTATEEN)) {
2597         if ((state->prv < PRV_M) && !(state->mstateen[0]->read() &
MSTATEEN0_IMSIC))
2598             throw trap_illegal_instruction(insn.bits());
2599         if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_IMSIC))
2600             throw trap_virtual_instruction(insn.bits());
2601         if (state->v && !(state->hstateen[0]->read() &
HSTATEEN0_IMSIC))
2602             throw trap_virtual_instruction(insn.bits());
2603     }
2604
2605     csr_t::verify_permissions(insn, write);
2606
2607     // VGEIN must be valid
2608     reg_t vgein = get_field(state->hstatus->read(),
HSTATUS_VGEIN);
2609     if (!vgein || !proc->imsic->vgein_valid(vgein)) {
2610         if (state->v)
2611             throw trap_virtual_instruction(insn.bits());
2612         else
2613             throw trap_illegal_instruction(insn.bits());
2614     }
2615 }

```

Berkeley SoftFloat

61 files • 5,175 lines

API — bf16_add.c

Table 1340. riscv-software-src_riscv-isa-sim/softfloat/bf16_add.c — 함수 (1개)

함수	반환형	받는 파라미터
bf16_add	bfloat16_t	bfloat16_t a, bfloat16_t b

API — bf16_classify.c

Table 1341. riscv-software-src_riscv-isa-sim/softfloat/bf16_classify.c — 함수 (1개)

함수	반환형	받는 파라미터
bf16_classify	uint_fast16_t	bfloat16_t a

API — bf16_cmp.c

Table 1342. riscv-software-src_riscv-isa-sim/softfloat/bf16_cmp.c — 함수 (3개)

함수	반환형	받는 파라미터
bf16_eq	bool	bfloat16_t a, bfloat16_t b
bf16_le	bool	bfloat16_t a, bfloat16_t b
bf16_lt	bool	bfloat16_t a, bfloat16_t b

API — bf16_div.c

Table 1343. riscv-software-src_riscv-isa-sim/softfloat/bf16_div.c — 함수 (1개)

함수	반환형	받는 파라미터
bf16_div	bfloat16_t	bfloat16_t a, bfloat16_t b

API — bf16_mul.c

Table 1344. riscv-software-src_riscv-isa-sim/softfloat/bf16_mul.c — 함수 (1개)

함수	반환형	받는 파라미터
bf16_mul	bfloat16_t	bfloat16_t a, bfloat16_t b

API — bf16_mulAdd.c

Table 1345. riscv-software-src_riscv-isa-sim/softfloat/bf16_mulAdd.c — 함수 (1개)

함수	반환형	받는 파라미터
bf16_mulAdd	bfloat16_t	bfloat16_t a, bfloat16_t b, bfloat16_t c

API — bf16_sqrt.c

Table 1346. riscv-software-src_riscv-isa-sim/softfloat/bf16_sqrt.c — 함수 (1개)

함수	반환형	받는 파라미터
bf16_sqrt	bfloat16_t	bfloat16_t a

API — bf16_sub.c**Table 1347. riscv-software-src_riscv-isa-sim/softfloat/bf16_sub.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_sub	bfloat16_t	bfloat16_t a, bfloat16_t b

API — bf16_to_e4m3.c**Table 1348. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_e4m3.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_e4m3	e4m3_t	bfloat16_t a, bool saturationMode

API — bf16_to_e5m2.c**Table 1349. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_e5m2.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_e5m2	e5m2_t	bfloat16_t a, bool saturationMode

API — bf16_to_f32.c**Table 1350. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_f32.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_f32	float32_t	bfloat16_t a

API — bf16_to_f64.c**Table 1351. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_f64.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_f64	float64_t	bfloat16_t a

API — bf16_to_i32.c**Table 1352. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_i32.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_i32	int_fast32_t	bfloat16_t a, uint_fast8_t roundingMode, bool exact

API — bf16_to_i8.c**Table 1353. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_i8.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_i8	int_fast8_t	bfloat16_t a, uint_fast8_t roundingMode, bool exact

API — bf16_to_ui32.c**Table 1354. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_ui32.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_ui32	uint_fast32_t	float16_t a, uint_fast8_t roundingMode, bool exact

API — bf16_to_ui8.c**Table 1355. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_ui8.c — 함수 (1개)**

함수	반환형	받는 파라미터
bf16_to_ui8	uint_fast8_t	bfloat16_t a, uint_fast8_t roundingMode, bool exact

API — e4m3_to_bf16.c**Table 1356. riscv-software-src_riscv-isa-sim/softfloat/e4m3_to_bf16.c — 함수 (1개)**

함수	반환형	받는 파라미터
e4m3_to_bf16	float16_t	float8_t a

API — e4m3_to_f16.c**Table 1357. riscv-software-src_riscv-isa-sim/softfloat/e4m3_to_f16.c — 함수 (1개)**

함수	반환형	받는 파라미터
e4m3_to_f16	float16_t	float8_t a

API — e5m2_to_bf16.c**Table 1358. riscv-software-src_riscv-isa-sim/softfloat/e5m2_to_bf16.c — 함수 (1개)**

함수	반환형	받는 파라미터
e5m2_to_bf16	float16_t	float8_t a

API — e5m2_to_f16.c**Table 1359. riscv-software-src_riscv-isa-sim/softfloat/e5m2_to_f16.c — 함수 (1개)**

함수	반환형	받는 파라미터
e5m2_to_f16	float16_t	float8_t a

API — f128_add.c**Table 1360. riscv-software-src_riscv-isa-sim/softfloat/f128_add.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_add	float128_t	float128_t a, float128_t b

API — f128_classify.c**Table 1361. riscv-software-src_riscv-isa-sim/softfloat/f128_classify.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_classify	uint_fast16_t	float128_t a

API — f128_div.c**Table 1362. riscv-software-src_riscv-isa-sim/softfloat/f128_div.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_div	float128_t	float128_t a, float128_t b

API — f128_eq.c**Table 1363. riscv-software-src_riscv-isa-sim/softfloat/f128_eq.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_eq	bool	float128_t a, float128_t b

API — f128_eq_signaling.c**Table 1364. riscv-software-src_riscv-isa-sim/softfloat/f128_eq_signaling.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_eq_signaling	bool	float128_t a, float128_t b

API — f128_isSignalingNaN.c**Table 1365. riscv-software-src_riscv-isa-sim/softfloat/f128_isSignalingNaN.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_isSignalingNaN	bool	float128_t a

API — f128_le.c**Table 1366. riscv-software-src_riscv-isa-sim/softfloat/f128_le.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_le	bool	float128_t a, float128_t b

API — f128_le_quiet.c**Table 1367. riscv-software-src_riscv-isa-sim/softfloat/f128_le_quiet.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_le_quiet	bool	float128_t a, float128_t b

API — f128_lt.c**Table 1368. riscv-software-src_riscv-isa-sim/softfloat/f128_lt.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_lt	bool	float128_t a, float128_t b

API — f128_lt_quiet.c**Table 1369. riscv-software-src_riscv-isa-sim/softfloat/f128_lt_quiet.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_lt_quiet	bool	float128_t a, float128_t b

API — f128_mul.c**Table 1370. riscv-software-src_riscv-isa-sim/softfloat/f128_mul.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_mul	float128_t	float128_t a, float128_t b

API — f128_mulAdd.c**Table 1371. riscv-software-src_riscv-isa-sim/softfloat/f128_mulAdd.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_mulAdd	float128_t	float128_t a, float128_t b, float128_t c

API — f128_rem.c**Table 1372. riscv-software-src_riscv-isa-sim/softfloat/f128_rem.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_rem	float128_t	float128_t a, float128_t b

API — f128_roundToInt.c**Table 1373. riscv-software-src_riscv-isa-sim/softfloat/f128_roundToInt.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_roundToInt	float128_t	float128_t a, uint_fast8_t roundingMode, bool exact

API — f128_sqrt.c**Table 1374. riscv-software-src_riscv-isa-sim/softfloat/f128_sqrt.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_sqrt	float128_t	float128_t a

API — f128_sub.c**Table 1375. riscv-software-src_riscv-isa-sim/softfloat/f128_sub.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_sub	float128_t	float128_t a, float128_t b

API — f128_to_f16.c**Table 1376. riscv-software-src_riscv-isa-sim/softfloat/f128_to_f16.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_f16	float16_t	float128_t a

API — f128_to_f32.c**Table 1377. riscv-software-src_riscv-isa-sim/softfloat/f128_to_f32.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_f32	float32_t	float128_t a

API — f128_to_f64.c**Table 1378. riscv-software-src_riscv-isa-sim/softfloat/f128_to_f64.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_f64	float64_t	float128_t a

API — f128_to_i32.c**Table 1379. riscv-software-src_riscv-isa-sim/softfloat/f128_to_i32.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_i32	int_fast32_t	float128_t a, uint_fast8_t roundingMode, bool exact

API — f128_to_i32_r_minMag.c**Table 1380. riscv-software-src_riscv-isa-sim/softfloat/f128_to_i32_r_minMag.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_i32_r_minMag	int_fast32_t	float128_t a, bool exact

API — f128_to_i64.c**Table 1381. riscv-software-src_riscv-isa-sim/softfloat/f128_to_i64.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_i64	int_fast64_t	float128_t a, uint_fast8_t roundingMode, bool exact

API — f128_to_i64_r_minMag.c**Table 1382. riscv-software-src_riscv-isa-sim/softfloat/f128_to_i64_r_minMag.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_i64_r_minMag	int_fast64_t	float128_t a, bool exact

API — f128_to_ui32.c**Table 1383. riscv-software-src_riscv-isa-sim/softfloat/f128_to_ui32.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_ui32	uint_fast32_t	float128_t a, uint_fast8_t roundingMode, bool exact

API — f128_to_ui32_r_minMag.c**Table 1384. riscv-software-src_riscv-isa-sim/softfloat/f128_to_ui32_r_minMag.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_ui32_r_minMag	uint_fast32_t	float128_t a, bool exact

API — f128_to_ui64.c**Table 1385. riscv-software-src_riscv-isa-sim/softfloat/f128_to_ui64.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_ui64	uint_fast64_t	float128_t a, uint_fast8_t roundingMode, bool exact

API — f128_to_ui64_r_minMag.c**Table 1386. riscv-software-src_riscv-isa-sim/softfloat/f128_to_ui64_r_minMag.c — 함수 (1개)**

함수	반환형	받는 파라미터
f128_to_ui64_r_minMag	uint_fast64_t	float128_t a, bool exact

API — f16_add.c**Table 1387. riscv-software-src_riscv-isa-sim/softfloat/f16_add.c — 함수 (1개)**

함수	반환형	받는 파라미터
f16_add	float16_t	float16_t a, float16_t b

API — f16_classify.c**Table 1388. riscv-software-src_riscv-isa-sim/softfloat/f16_classify.c — 함수 (1개)**

함수	반환형	받는 파라미터
f16_classify	uint_fast16_t	float16_t a

API — f16_div.c**Table 1389. riscv-software-src_riscv-isa-sim/softfloat/f16_div.c — 함수 (1개)**

함수	반환형	받는 파라미터
f16_div	float16_t	float16_t a, float16_t b

API — f16_eq.c**Table 1390. riscv-software-src_riscv-isa-sim/softfloat/f16_eq.c — 함수 (1개)**

함수	반환형	받는 파라미터
f16_eq	bool	float16_t a, float16_t b

API — f16_eq_signaling.c**Table 1391. riscv-software-src_riscv-isa-sim/softfloat/f16_eq_signaling.c — 함수 (1개)**

함수	반환형	받는 파라미터
f16_eq_signaling	bool	float16_t a, float16_t b

API — f16_isSignalingNaN.c

Table 1392. riscv-software-src_riscv-isa-sim/softfloat/f16_isSignalingNaN.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_isSignalingNaN	bool	float16_t a

API — f16_le.c

Table 1393. riscv-software-src_riscv-isa-sim/softfloat/f16_le.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_le	bool	float16_t a, float16_t b

API — f16_le_quiet.c

Table 1394. riscv-software-src_riscv-isa-sim/softfloat/f16_le_quiet.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_le_quiet	bool	float16_t a, float16_t b

API — f16_lt.c

Table 1395. riscv-software-src_riscv-isa-sim/softfloat/f16_lt.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_lt	bool	float16_t a, float16_t b

API — f16_lt_quiet.c

Table 1396. riscv-software-src_riscv-isa-sim/softfloat/f16_lt_quiet.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_lt_quiet	bool	float16_t a, float16_t b

API — f16_mul.c

Table 1397. riscv-software-src_riscv-isa-sim/softfloat/f16_mul.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_mul	float16_t	float16_t a, float16_t b

API — f16_mulAdd.c

Table 1398. riscv-software-src_riscv-isa-sim/softfloat/f16_mulAdd.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_mulAdd	float16_t	float16_t a, float16_t b, float16_t c

API — f16_rem.c

Table 1399. riscv-software-src_riscv-isa-sim/softfloat/f16_rem.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_rem	float16_t	float16_t a, float16_t b

API — f16_roundToInt.c

Table 1400. riscv-software-src_riscv-isa-sim/softfloat/f16_roundToInt.c — 함수 (1개)

함수	반환형	받는 파라미터
f16_roundToInt	float16_t	float16_t a, uint_fast8_t roundingMode, bool exact

1. riscv-software-src_riscv-isa-sim/softfloat/bf16_add.c (50 lines)

```

1  /
2  /
3  /
4  This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5  Package, Release 3d, by John R. Hauser.
6
7  Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8  California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bfloat16_t bf16_add( bfloat16_t a, bfloat16_t b )
44 {
45     float32_t f32A = { (uint_fast32_t)a.v << 16 };
46     float32_t f32B = { (uint_fast32_t)b.v << 16 };
47
48     return f32_to_bf16 ( f32_add ( f32A, f32B ) );

```

```

49 }
50

```

2. riscv-software-src_riscv-isa-sim/softfloat/bf16_classify.c (36 lines)

```

1
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include "platform.h"
5 #include "internals.h"
6 #include "specialize.h"
7 #include "softfloat.h"
8
9 uint_fast16_t bf16_classify( bfloat16_t a )
10 {
11     union ui16_f16 uA;
12     uint_fast16_t uiA;
13
14     uA.f = a;
15     uiA = uA.ui;
16
17     uint_fast16_t infOrNaN = expBF16UI( uiA ) == 0xFF;
18     uint_fast16_t subnormalOrZero = expBF16UI( uiA ) == 0;
19     bool sign = signBF16UI( uiA );
20     bool fracZero = fracBF16UI( uiA ) == 0;
21     bool isNaN = isNaNBF16UI( uiA );
22     bool isSNaN = softfloat_isNaNBF16UI( uiA );
23
24     return
25         ( sign && infOrNaN && fracZero )      << 0 |
26         ( sign && !infOrNaN && !subnormalOrZero ) << 1 |
27         ( sign && subnormalOrZero && !fracZero ) << 2 |
28         ( sign && subnormalOrZero && fracZero )  << 3 |
29         ( !sign && infOrNaN && fracZero )        << 7 |
30         ( !sign && !infOrNaN && !subnormalOrZero ) << 6 |
31         ( !sign && subnormalOrZero && !fracZero ) << 5 |
32         ( !sign && subnormalOrZero && fracZero )  << 4 |
33         ( isNaN && isSNaN )                    << 8 |
34         ( isNaN && !isSNaN )                    << 9;
35 }
36

```

3. riscv-software-src_riscv-isa-sim/softfloat/bf16_cmp.c (65 lines)

```

1  /
2  /
3  /
4  This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5  Package, Release 3d, by John R. Hauser.
6
7  Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8  California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.

```



```

19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bool bf16_eq( bfloat16_t a, bfloat16_t b )
44 {
45     float32_t f32A = { (uint_fast32_t)a.v << 16 };
46     float32_t f32B = { (uint_fast32_t)b.v << 16 };
47
48     return f32_eq ( f32A, f32B );
49 }
50
51 bool bf16_le( bfloat16_t a, bfloat16_t b )
52 {
53     float32_t f32A = { (uint_fast32_t)a.v << 16 };
54     float32_t f32B = { (uint_fast32_t)b.v << 16 };
55
56     return f32_le ( f32A, f32B );
57 }
58
59 bool bf16_lt( bfloat16_t a, bfloat16_t b )
60 {
61     float32_t f32A = { (uint_fast32_t)a.v << 16 };
62     float32_t f32B = { (uint_fast32_t)b.v << 16 };
63
64     return f32_lt ( f32A, f32B );
65 }

```

4. riscv-software-src_riscv-isa-sim/softfloat/bf16_div.c (50 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY

```

```

25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bfloat16_t bf16_div( bfloat16_t a, bfloat16_t b )
44 {
45     float32_t f32A = { (uint_fast32_t)a.v << 16 };
46     float32_t f32B = { (uint_fast32_t)b.v << 16 };
47
48     return f32_to_bf16 ( f32_div ( f32A, f32B ) );
49 }
50

```

5. riscv-software-src_riscv-isa-sim/softfloat/bf16_mul.c (50 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>

```

```

39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bfloat16_t bf16_mul( bfloat16_t a, bfloat16_t b )
44 {
45     float32_t f32A = { (uint_fast32_t)a.v << 16 };
46     float32_t f32B = { (uint_fast32_t)b.v << 16 };
47
48     return f32_to_bf16 ( f32_mul ( f32A, f32B ) );
49 }
50

```

6. riscv-software-src_riscv-isa-sim/softfloat/bf16_mulAdd.c

(67 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdint.h>
38 #include "platform.h"
39 #include "internals.h"
40 #include "softfloat.h"
41
42 bfloat16_t bf16_mulAdd( bfloat16_t a, bfloat16_t b, bfloat16_t
c )
43 {
44     uint_fast8_t roundingMode = softfloat_roundingMode;
45
46     float64_t f64A = bf16_to_f64( a );
47     float64_t f64B = bf16_to_f64( b );
48     float64_t f64C = bf16_to_f64( c );
49
50     float64_t prod = f64_mul( f64A, f64B );
51
52     softfloat_roundingMode = softfloat_round_odd;
53
54     float64_t sum = f64_add( prod, f64C );
55
56     softfloat_roundingMode = roundingMode;
57
58     /* When rounding down, return -0 instead of +0 when
product's sign

```

```

59     * differs from C's sign. */
60     if ( softfloat_roundingMode == softfloat_round_min
61         && sum.v == 0
62         && ( ( prod.v ^ f64C.v ) &
0x8000000000000000U ) != 0 )
63         return (bfloat16_t) { 0x8000 };
64
65     return f64_to_bf16( sum );
66 }
67

```

7. riscv-software-src_riscv-isa-sim/softfloat/bf16_sqrt.c (49 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bfloat16_t bf16_sqrt( bfloat16_t a )
44 {
45     float32_t f32A = { (uint_fast32_t)a.v << 16 };
46
47     return f32_to_bf16 ( f32_sqrt ( f32A ) );
48 }
49

```

8. riscv-software-src_riscv-isa-sim/softfloat/bf16_sub.c (50 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of

```

```

the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bfloat16_t bf16_sub( bfloat16_t a, bfloat16_t b )
44 {
45     float32_t f32A = { (uint_fast32_t)a.v << 16 };
46     float32_t f32B = { (uint_fast32_t)b.v << 16 };
47
48     return f32_to_bf16 ( f32_sub ( f32A, f32B ) );
49 }
50

```

9. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_e4m3.c (48 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2025 The Regents of the University of California.
All rights
8 reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23

```

```

24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 e4m3_t bf16_to_e4m3( bfloat16_t a, bool saturationMode )
45 {
46     return f32_to_e4m3( bf16_to_f32( a ), saturationMode );
47 }
48

```

10. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_e5m2.c (48 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2025 The Regents of the University of California.
All rights
8 reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>

```

```

39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 e5m2_t bf16_to_e5m2( bfloat16_t a, bool saturationMode )
45 {
46     return f32_to_e5m2( bf16_to_f32( a ), saturationMode );
47 }
48

```

11. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_f32.c (80 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float32_t bf16_to_f32( bfloat16_t a )
45 {
46     union ui16_f16 uA;
47     uint_fast16_t uiA;
48     bool sign;
49     int_fast16_t exp;
50     uint_fast16_t frac;
51     struct commonNaN commonNaN;
52     uint_fast32_t uiZ;
53     union ui32_f32 uZ;
54
55     /
=====
56
=====*/
57     uA.f = a;
58     uiA = uA.ui;
59     sign = signBF16UI( uiA );

```

```

60     exp = expBF16UI( uiA );
61     frac = fracBF16UI( uiA );
62     /
=====
63
=====*/
64     if ( exp == 0xFF ) {
65         if ( frac ) {
66             softfloat_bf16UIToCommonNaN( uiA, &commonNaN );
67             uiZ = softfloat_commonNaNToF32UI( &commonNaN );
68         } else {
69             uiZ = packToF32UI( sign, 0xFF, 0 );
70         }
71         goto uiZ;
72     }
73     /
=====
74
=====*/
75     uiZ = packToF32UI( sign, exp, (uint_fast32_t) frac<<16 );
76     uiZ:
77     uZ.ui = uiZ;
78     return uZ.f;
79
80 }

```

12. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_f64.c (49 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float64_t bf16_to_f64( bfloat16_t a )

```

```

45 {
46     float32_t f32A = (float32_t) { (uint_fast32_t)a.v << 16 };
47
48     return f32_to_f64( f32A );
49 }

```

13. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_i32.c (48 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 int_fast32_t bf16_to_i32( bfloat16_t a, uint_fast8_t rounding-
Mode, bool exact )
45 {
46     return f32_to_i32(bf16_to_f32(a), roundingMode, exact);
47 }
48

```

14. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_i8.c (57 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9

```

```

10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdint.h>
38 #include "specialize.h"
39 #include "softfloat.h"
40
41 int_fast8_t bf16_to_i8( bfloat16_t a, uint_fast8_t rounding-
Mode, bool exact )
42 {
43     uint_fast8_t old_flags = softfloat_exceptionFlags;
44
45     int_fast32_t sig32 = bf16_to_i32(a, roundingMode, exact);
46
47     if (sig32 > INT8_MAX) {
48         softfloat_exceptionFlags = old_flags | soft-
float_flag_invalid;
49         return i8_fromPosOverflow;
50     } else if (sig32 < INT8_MIN) {
51         softfloat_exceptionFlags = old_flags | soft-
float_flag_invalid;
52         return i8_fromNegOverflow;
53     } else {
54         return sig32;
55     }
56 }
57

```

15. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_ui32.c (48 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation

```

```

18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
21 contributors may
22 be used to endorse or promote products derived from this
23 software without
24 specific prior written permission.
25
26 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
27 IS", AND ANY
28 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
29 THE IMPLIED
30 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
31 PURPOSE, ARE
32 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
33 LIABLE FOR ANY
34 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
35 SEQUENTIAL DAMAGES
36 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
37 GOODS OR SERVICES;
38 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
39 EVER CAUSED AND
40 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
41 ILITY, OR TORT
42 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
43 THE USE OF THIS
44 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
45
46 =====*/
47
48 #include <stdbool.h>
49 #include <stdint.h>
50 #include "platform.h"
51 #include "internals.h"
52 #include "specialize.h"
53 #include "softfloat.h"
54
55 uint_fast32_t bf16_to_ui32( float16_t a, uint_fast8_t round-
56 ingMode, bool exact )
57 {
58     return f32_to_ui32(bf16_to_f32(a), roundingMode, exact);
59 }
60

```

16. riscv-software-src_riscv-isa-sim/softfloat/bf16_to_ui8.c (54 lines)

```

1
2 /
3
4 =====
5
6 4 This C source file is part of the SoftFloat IEEE Floating-
7 Point Arithmetic
8 Package, Release 3d, by John R. Hauser.
9
10 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
11 the
12 University of California. All rights reserved.
13
14 Redistribution and use in source and binary forms, with or
15 without
16 modification, are permitted provided that the following condi-
17 tions are met:
18
19 1. Redistributions of source code must retain the above copy-
20 right notice,
21 this list of conditions, and the following disclaimer.
22
23 2. Redistributions in binary form must reproduce the above
24 copyright notice,
25 this list of conditions, and the following disclaimer in
26 the documentation
27 and/or other materials provided with the distribution.
28
29 3. Neither the name of the University nor the names of its
30 contributors may
31 be used to endorse or promote products derived from this
32 software without
33 specific prior written permission.
34
35 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
36 IS", AND ANY
37 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
38 THE IMPLIED
39 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
40 PURPOSE, ARE
41 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
42 LIABLE FOR ANY
43 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
44 SEQUENTIAL DAMAGES
45 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
46 GOODS OR SERVICES;
47 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
48 EVER CAUSED AND
49 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
50 ILITY, OR TORT

```

```

32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
33 THE USE OF THIS
34 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
35
36 =====*/
37
38 #include <stdint.h>
39 #include "specialize.h"
40 #include "softfloat.h"
41
42 uint_fast8_t bf16_to_ui8( bfloat16_t a, uint_fast8_t rounding-
43 Mode, bool exact )
44 {
45     uint_fast8_t old_flags = softfloat_exceptionFlags;
46
47     uint_fast32_t sig32 = bf16_to_ui32(a, roundingMode, ex-
48 act);
49
50     if (sig32 > UINT8_MAX) {
51         softfloat_exceptionFlags = old_flags | soft-
52 float_flag_invalid;
53         return ui8_fromPosOverflow;
54     } else {
55         return sig32;
56     }
57 }
58

```

17. riscv-software-src_riscv-isa-sim/softfloat/e4m3_to_bf16.c (92 lines)

```

1
2 /
3
4 =====
5
6 4 This C source file is part of the SoftFloat IEEE Floating-
7 Point Arithmetic
8 Package, Release 3d, by John R. Hauser.
9
10 Copyright 2025 The Regents of the University of California.
11 All rights
12 reserved.
13
14 Redistribution and use in source and binary forms, with or
15 without
16 modification, are permitted provided that the following condi-
17 tions are met:
18
19 1. Redistributions of source code must retain the above copy-
20 right notice,
21 this list of conditions, and the following disclaimer.
22
23 2. Redistributions in binary form must reproduce the above
24 copyright notice,
25 this list of conditions, and the following disclaimer in
26 the documentation
27 and/or other materials provided with the distribution.
28
29 3. Neither the name of the University nor the names of its
30 contributors may
31 be used to endorse or promote products derived from this
32 software without
33 specific prior written permission.
34
35 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
36 IS", AND ANY
37 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
38 THE IMPLIED
39 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
40 PURPOSE, ARE
41 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
42 LIABLE FOR ANY
43 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
44 SEQUENTIAL DAMAGES
45 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
46 GOODS OR SERVICES;
47 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
48 EVER CAUSED AND
49 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
50 ILITY, OR TORT

```



```

46 union ui8_f8 uA;
47 uint_fast8_t uiA;
48 bool sign;
49 int_fast8_t exp;
50 uint_fast8_t frac;
51 struct commonNaN commonNaN;
52 uint_fast16_t uiZ;
53 struct exp8_sig8 normExpSig;
54 union ui16_f16 uZ;
55
56 /
-----
57
-----
*/
58 uA.f = a;
59 uiA = uA.ui;
60 sign = signE4M3UI( uiA );
61 exp = expE4M3UI( uiA );
62 frac = fracE4M3UI( uiA );
63 /
-----
*
-----
64
-----
*/
65 if ( (exp == 0xF) && (frac == 0x7) ) {
66     /* NaN */
67     softfloat_E4M3UIToCommonNaN( uiA, &commonNaN );
68     uiZ = softfloat_commonNaNToBF16UI( &commonNaN );
69     goto uiZ;
70 }
71 /
-----
*
-----
72
-----
*/
73 if ( ! exp ) {
74     if ( ! frac ) {
75         /* zero */
76         uiZ = packToBF16UI( sign, 0, 0 );
77         goto uiZ;
78     }
79     /* subnormal */
80     normExpSig = softfloat_normSubnormalE4M3Sig( frac );
81     exp = normExpSig.exp - 1;
82     frac = normExpSig.sig;
83 }
84 /
-----
*
-----
85
-----
*/
86 uiZ = packToBF16UI( sign, exp + 0x78, (uint_fast16_t)
frac<<4);
87 uiZ:
88 uZ.ui = uiZ;
89 return uZ.f;
90
91 }
92

```

18. riscv-software-src_riscv-isa-sim/softfloat/e4m3_to_f16.c (92 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2025 The Regents of the University of California.
All rights
8 reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright
17 notice,
18 this list of conditions, and the following disclaimer in
the documentation
19 and/or other materials provided with the distribution.
20
21 3. Neither the name of the University nor the names of its
contributors may
22 be used to endorse or promote products derived from this
software without

```

```

22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float16_t e4m3_to_f16( float8_t a )
45 {
46     union ui8_f8 uA;
47     uint_fast8_t uiA;
48     bool sign;
49     int_fast8_t exp;
50     uint_fast8_t frac;
51     struct commonNaN commonNaN;
52     uint_fast16_t uiZ;
53     struct exp8_sig8 normExpSig;
54     union ui16_f16 uZ;
55
56     /
-----
57
-----
*/
58 uA.f = a;
59 uiA = uA.ui;
60 sign = signE4M3UI( uiA );
61 exp = expE4M3UI( uiA );
62 frac = fracE4M3UI( uiA );
63 /
-----
*
-----
64
-----
*/
65 if ( (exp == 0xF) && (frac == 0x7) ) {
66     /* NaN */
67     softfloat_E4M3UIToCommonNaN( uiA, &commonNaN );
68     uiZ = softfloat_commonNaNToF16UI( &commonNaN );
69     goto uiZ;
70 }
71 /
-----
*
-----
72
-----
*/
73 if ( ! exp ) {
74     if ( ! frac ) {
75         /* zero */
76         uiZ = packToF16UI( sign, 0, 0 );
77         goto uiZ;
78     }
79     /* subnormal */
80     normExpSig = softfloat_normSubnormalE4M3Sig( frac );
81     exp = normExpSig.exp - 1;
82     frac = normExpSig.sig;
83 }
84 /
-----
*
-----
85
-----
*/
86 uiZ = packToF16UI( sign, exp + 0x8, (uint_fast16_t)
frac<<7);
87 uiZ:
88 uZ.ui = uiZ;
89 return uZ.f;
90
91 }
92

```


19. riscv-software-src_riscv-isa-sim/softfloat/e5m2_to_bf16.c (98 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float16_t e5m2_to_bf16( float8_t a )
45 {
46     union ui8_f8 uA;
47     uint_fast8_t uiA;
48     bool sign;
49     int_fast8_t exp;
50     uint_fast8_t frac;
51     struct commonNaN commonNaN;
52     uint_fast16_t uiZ;
53     struct exp8_sig8 normExpSig;
54     union ui16_f16 uZ;
55
56     /
=====
57
58     uA.f = a;
59     uiA = uA.ui;
60     sign = signE5M2UI( uiA );
61     exp = expE5M2UI( uiA );
62     frac = fracE5M2UI( uiA );
63     /
=====
64
65     if ( exp == 0x1F ) {
66         if ( frac ) {

```

```

67         /* NaN */
68         softfloat_E5M2UIToCommonNaN( uiA, &commonNaN );
69         uiZ = softfloat_commonNaNToBF16UI( &commonNaN );
70     } else {
71         /* Inf */
72         uiZ = packToBF16UI( sign, 0xFF, 0 );
73     }
74     goto uiZ;
75 }
76 /
=====
77
78     if ( ! exp ) {
79         if ( ! frac ) {
80             /* zero */
81             uiZ = packToBF16UI( sign, 0, 0 );
82             goto uiZ;
83         }
84         /* subnormal */
85         normExpSig = softfloat_normSubnormalE5M2Sig( frac );
86         //TODO: might be unnecessary to implement a dedicated
function
87         exp = normExpSig.exp - 1;
88         frac = normExpSig.sig;
89     }
90     /
=====
91
92     uiZ = packToBF16UI( sign, exp + 0x70, (uint_fast16_t)
frac<<5);
93     uiZ:
94     uZ.ui = uiZ;
95     return uZ.f;
96 }
97 }
98

```

20. riscv-software-src_riscv-isa-sim/softfloat/e5m2_to_f16.c (98 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34

```

```

35
=====
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float16_t e5m2_to_f16( float8_t a )
45 {
46     union ui8_f8 uA;
47     uint_fast8_t uiA;
48     bool sign;
49     int_fast8_t exp;
50     uint_fast8_t frac;
51     struct commonNaN commonNaN;
52     uint_fast16_t uiZ;
53     struct exp8_sig8 normExpSig;
54     union ui16_f16 uZ;
55
56     /
-----
57
*-----
*-----
58     uA.f = a;
59     uiA = uA.ui;
60     sign = signE5M2UI( uiA );
61     exp = expE5M2UI( uiA );
62     frac = fracE5M2UI( uiA );
63     /
-----
*-----
*-----
64
*-----
*-----
65     if ( exp == 0xF ) {
66         if ( frac ) {
67             /* NaN */
68             softfloat_E5M2UIToCommonNaN( uiA, &commonNaN );
69             uiZ = softfloat_commonNaNToF16UI( &commonNaN );
70         } else {
71             /* Inf */
72             uiZ = packToF16UI( sign, 0x1F, 0 );
73         }
74         goto uiZ;
75     }
76     /
-----
*-----
*-----
77
*-----
*-----
78     if ( ! exp ) {
79         if ( ! frac ) {
80             /* zero */
81             uiZ = packToF16UI( sign, 0, 0 );
82             goto uiZ;
83         }
84         /* subnormal */
85         normExpSig = softfloat_normSubnormalE5M2Sig( frac );
86         //TODO: might be unnecessary to implement a dedicated
function
87         exp = normExpSig.exp - 1;
88         frac = normExpSig.sig;
89     }
90     /
-----
*-----
*-----
91
*-----
*-----
92     uiZ = packToF16UI( sign, exp, (uint_fast16_t) frac<<8);
93     uiZ:
94     uZ.ui = uiZ;
95     return uZ.f;
96
97 }
98

```

21. riscv-software-src_riscv-isa-sim/softfloat/f128_add.c (78 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6

```

7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of California.
8 All rights reserved.

```

9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====

```

```

=====
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 float128_t f128_add( float128_t a, float128_t b )
44 {
45     union ui128_f128 uA;
46     uint_fast64_t uiA64, uiA0;
47     bool signA;
48     union ui128_f128 uB;
49     uint_fast64_t uiB64, uiB0;
50     bool signB;
51     #if ! defined INLINE_LEVEL || (INLINE_LEVEL < 2)
52     float128_t
53         (*magsFuncPtr)(
54             uint_fast64_t, uint_fast64_t, uint_fast64_t, uint_fast64_t,
55             uint_fast64_t, bool );
56     #endif
57
58     uA.f = a;
59     uiA64 = uA.ui.v64;
60     uiA0 = uA.ui.v0;
61     signA = signF128UI64( uiA64 );
62     uB.f = b;
63     uiB64 = uB.ui.v64;
64     uiB0 = uB.ui.v0;
65     signB = signF128UI64( uiB64 );
66     #if defined INLINE_LEVEL && (2 <= INLINE_LEVEL)
67     if ( signA == signB ) {
68         return softfloat_addMagsF128( uiA64, uiA0, uiB64,
69         uiB0, signA );
70     } else {
71         return softfloat_subMagsF128( uiA64, uiA0, uiB64,
72         uiB0, signA );
73     }
74     #else
75     magsFuncPtr =
76         (signA == signB) ? softfloat_addMagsF128 : soft-
float_subMagsF128;
77     return (*magsFuncPtr)( uiA64, uiA0, uiB64, uiB0, signA );
78     #endif
79 }
80

```

22. riscv-software-src_riscv-isa-sim/softfloat/f128_classify.c (37 lines)

```

1
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include "platform.h"

```

```

5 #include "internals.h"
6 #include "specialize.h"
7 #include "softfloat.h"
8
9 uint_fast16_t f128_classify( float128_t a )
10 {
11     union ui128_f128 uA;
12     uint_fast64_t uiA64, uiA0;
13
14     uA.f = a;
15     uiA64 = uA.ui.v64;
16     uiA0 = uA.ui.v0;
17
18     uint_fast16_t infOrNaN = expF128UI64( uiA64 ) == 0x7FFF;
19     uint_fast16_t subnormalOrZero = expF128UI64( uiA64 ) == 0;
20     bool sign = signF128UI64( uiA64 );
21     bool fracZero = fracF128UI64( uiA64 ) == 0 && uiA0 == 0;
22     bool isNaN = isNaNF128UI( uiA64, uiA0 );
23     bool isSNaN = softfloat_isNaNF128UI( uiA64, uiA0 );
24
25     return
26         ( sign && infOrNaN && fracZero )      << 0 |
27         ( sign && !infOrNaN && !subnormalOrZero ) << 1 |
28         ( sign && subnormalOrZero && !fracZero ) << 2 |
29         ( sign && subnormalOrZero && fracZero ) << 3 |
30         ( !sign && infOrNaN && fracZero )      << 7 |
31         ( !sign && !infOrNaN && !subnormalOrZero ) << 6 |
32         ( !sign && subnormalOrZero && !fracZero ) << 5 |
33         ( !sign && subnormalOrZero && fracZero ) << 4 |
34         ( isNaN && isSNaN )                  << 8 |
35         ( isNaN && !isSNaN )                  << 9;
36 }
37

```

23. riscv-software-src_riscv-isa-sim/softfloat/f128_div.c (199 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"

```

```

41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float128_t f128_div( float128_t a, float128_t b )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool signA;
49     int_fast32_t expA;
50     struct uint128 sigA;
51     union ui128_f128 uB;
52     uint_fast64_t uiB64, uiB0;
53     bool signB;
54     int_fast32_t expB;
55     struct uint128 sigB;
56     bool signZ;
57     struct exp32_sig128 normExpSig;
58     int_fast32_t expZ;
59     struct uint128 rem;
60     uint_fast32_t recip32;
61     int ix;
62     uint_fast64_t q64;
63     uint_fast32_t q;
64     struct uint128 term;
65     uint_fast32_t qs[3];
66     uint_fast64_t sigZExtra;
67     struct uint128 sigZ, uiZ;
68     union ui128_f128 uZ;
69
70     /
-----
71
-----*/
72     uA.f = a;
73     uiA64 = uA.ui.v64;
74     uiA0 = uA.ui.v0;
75     signA = signF128UI64( uiA64 );
76     expA = expF128UI64( uiA64 );
77     sigA.v64 = fracF128UI64( uiA64 );
78     sigA.v0 = uiA0;
79     uB.f = b;
80     uiB64 = uB.ui.v64;
81     uiB0 = uB.ui.v0;
82     signB = signF128UI64( uiB64 );
83     expB = expF128UI64( uiB64 );
84     sigB.v64 = fracF128UI64( uiB64 );
85     sigB.v0 = uiB0;
86     signZ = signA ^ signB;
87     /
-----
88
-----*/
89     if ( expA == 0x7FFF ) {
90         if ( sigA.v64 | sigA.v0 ) goto propagateNaN;
91         if ( expB == 0x7FFF ) {
92             if ( sigB.v64 | sigB.v0 ) goto propagateNaN;
93             goto invalid;
94         }
95         goto infinity;
96     }
97     if ( expB == 0x7FFF ) {
98         if ( sigB.v64 | sigB.v0 ) goto propagateNaN;
99         goto zero;
100     }
101     /
-----
102
-----*/
103     if ( ! expB ) {
104         if ( ! ( sigB.v64 | sigB.v0 ) ) {
105             if ( ! ( expA | sigA.v64 | sigA.v0 ) ) goto invalid;
106             softfloat_raiseFlags( softfloat_flag_infinite );
107             goto infinity;
108         }
109     }
110     normExpSig = softfloat_normSubnormalF128Sig( sigB.v64,
sigB.v0 );
111     expB = normExpSig.exp;
112     sigB = normExpSig.sig;
113     if ( ! expA ) {
114         if ( ! ( sigA.v64 | sigA.v0 ) ) goto zero;
115     }
116     normExpSig = softfloat_normSubnormalF128Sig( sigA.v64,
sigA.v0 );
117     expA = normExpSig.exp;
118     sigA = normExpSig.sig;
119     /
-----
120
-----*/
121     expZ = expA - expB + 0x3FFE;

```

```

122 sigA.v64 |= UINT64_C( 0x0001000000000000 );
123 sigB.v64 |= UINT64_C( 0x0001000000000000 );
124 rem = sigA;
125 if ( softfloat_lt128( sigA.v64, sigA.v0, sigB.v64,
sigB.v0 ) ) {
126     --expZ;
127     rem = softfloat_add128( sigA.v64, sigA.v0, sigA.v64,
sigA.v0 );
128 }
129 recip32 = softfloat_approxRecip32_1( sigB.v64>>17 );
130 ix = 3;
131 for (;;) {
132     q64 = (uint_fast64_t) (uint32_t) (rem.v64>>19) * re-
cip32;
133     q = (q64 + 0x80000000)>>32;
134     --ix;
135     if ( ix < 0 ) break;
136     rem = softfloat_shortShiftLeft128( rem.v64, rem.v0,
29 );
137     term = softfloat_mul128By32( sigB.v64, sigB.v0, q );
138     rem = softfloat_sub128( rem.v64, rem.v0, term.v64,
term.v0 );
139     if ( rem.v64 & UINT64_C( 0x8000000000000000 ) ) {
140         --q;
141         rem = softfloat_add128( rem.v64, rem.v0, sigB.v64,
sigB.v0 );
142     }
143     qs[ix] = q;
144 }
145 /
-----
146
-----*/
147 if ( ((q + 1) & 7) < 2 ) {
148     rem = softfloat_shortShiftLeft128( rem.v64, rem.v0,
29 );
149     term = softfloat_mul128By32( sigB.v64, sigB.v0, q );
150     rem = softfloat_sub128( rem.v64, rem.v0, term.v64,
term.v0 );
151     if ( rem.v64 & UINT64_C( 0x8000000000000000 ) ) {
152         --q;
153         rem = softfloat_add128( rem.v64, rem.v0, sigB.v64,
sigB.v0 );
154     } else if ( softfloat_le128( sigB.v64, sigB.v0,
rem.v64, rem.v0 ) ) {
155         ++q;
156         rem = softfloat_sub128( rem.v64, rem.v0, sigB.v64,
sigB.v0 );
157     }
158     if ( rem.v64 | rem.v0 ) q |= 1;
159 }
160 /
-----
161
-----*/
162 sigZExtra = (uint64_t) ((uint_fast64_t) q<<60);
163 term = softfloat_shortShiftLeft128( 0, qs[1], 54 );
164 sigZ =
165     softfloat_add128(
166         (uint_fast64_t) qs[2]<<19, ((uint_fast64_t)
qs[0]<<25) + (q>>4),
167         term.v64, term.v0
168     );
169 return
170     softfloat_roundPackToF128( signZ, expZ, sigZ.v64,
sigZ.v0, sigZExtra );
171 /
-----
172
-----*/
173 propagateNaN:
174 uiZ = softfloat_propagateNaNF128UI( uiA64, uiA0, uiB64,
uiB0 );
175 goto uiZ;
176 /
-----
177
-----*/
178 invalid:
179 softfloat_raiseFlags( softfloat_flag_invalid );
180 uiZ.v64 = defaultNaNF128UI64;
181 uiZ.v0 = defaultNaNF128UI0;
182 goto uiZ;
183 /
-----
184
-----*/
185 infinity:

```

```

186 uiZ.v64 = packToF128UI64( signZ, 0x7FFF, 0 );
187 goto uiZ0;
188 /
-----
189
-----*/
190 zero:
191 uiZ.v64 = packToF128UI64( signZ, 0, 0 );
192 uiZ0:
193 uiZ.v0 = 0;
194 uiZ:
195 uZ.ui = uiZ;
196 return uZ.f;
197 }
198 }
199

```

24. riscv-software-src_riscv-isa-sim/softfloat/f128_eq.c (73 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 bool f128_eq( float128_t a, float128_t b )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     union ui128_f128 uB;
49     uint_fast64_t uiB64, uiB0;
50
51     uA.f = a;
52     uiA64 = uA.ui.v64;
53     uiA0 = uA.ui.v0;
54     uB.f = b;
55     uiB64 = uB.ui.v64;

```

```

56     uiB0 = uB.ui.v0;
57     if ( isNaNF128UI( uiA64, uiA0 ) || isNaNF128UI( uiB64,
uiB0 ) ) {
58         if (
59             softfloat_isSigNaNF128UI( uiA64, uiA0 )
60             || softfloat_isSigNaNF128UI( uiB64, uiB0 )
61         ) {
62             softfloat_raiseFlags( softfloat_flag_invalid );
63         }
64         return false;
65     }
66     return
67         (uiA0 == uiB0)
68         && ( (uiA64 == uiB64)
69             || (! uiA0 && ! ((uiA64 | uiB64) &
UINT64_C( 0x7FFFFFFFFFFFFFFF )))
70             );
71 }
72 }
73

```

25. riscv-software-src_riscv-isa-sim/softfloat/ f128_eq_signaling.c (67 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bool f128_eq_signaling( float128_t a, float128_t b )
44 {
45     union ui128_f128 uA;
46     uint_fast64_t uiA64, uiA0;
47     union ui128_f128 uB;
48     uint_fast64_t uiB64, uiB0;
49
50     uA.f = a;
51     uiA64 = uA.ui.v64;
52     uiA0 = uA.ui.v0;
53     uB.f = b;

```

```

54     uiB64 = uB.ui.v64;
55     uiB0 = uB.ui.v0;
56     if ( isNaNF128UI( uiA64, uiA0 ) || isNaNF128UI( uiB64,
uiB0 ) ) {
57         softfloat_raiseFlags( softfloat_flag_invalid );
58         return false;
59     }
60     return
61         (uiA0 == uiB0)
62         && ( (uiA64 == uiB64)
63             || (! uiA0 && ! ((uiA64 | uiB64) &
UINT64_C( 0x7FFFFFFFFFFFFFFF )))
64             );
65 }
66 }
67

```

26. riscv-software-src_riscv-isa-sim/softfloat/ f128_isSignalingNaN.c (51 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include "platform.h"
39 #include "internals.h"
40 #include "specialize.h"
41 #include "softfloat.h"
42
43 bool f128_isSignalingNaN( float128_t a )
44 {
45     union ui128_f128 uA;
46
47     uA.f = a;
48     return softfloat_isSigNaNF128UI( uA.ui.v64, uA.ui.v0 );
49 }
50 }
51

```

27. riscv-software-src_riscv-isa-sim/softfloat/f128_le.c (72 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bool f128_le( float128_t a, float128_t b )
44 {
45     union ui128_f128 uA;
46     uint_fast64_t uiA64, uiA0;
47     union ui128_f128 uB;
48     uint_fast64_t uiB64, uiB0;
49     bool signA, signB;
50
51     uA.f = a;
52     uiA64 = uA.ui.v64;
53     uiA0 = uA.ui.v0;
54     uB.f = b;
55     uiB64 = uB.ui.v64;
56     uiB0 = uB.ui.v0;
57     if ( isNaNF128UI( uiA64, uiA0 ) || isNaNF128UI( uiB64,
uiB0 ) ) {
58         softfloat_raiseFlags( softfloat_flag_invalid );
59         return false;
60     }
61     signA = signF128UI64( uiA64 );
62     signB = signF128UI64( uiB64 );
63     return
64         (signA != signB)
65         ? signA
66         : ! (((uiA64 | uiB64) &
UINT64_C( 0x7FFFFFFFFFFFFFFF ))
67             | uiA0 | uiB0)
68         : ((uiA64 == uiB64) && (uiA0 == uiB0))
69         || (signA ^ softfloat_lt128( uiA64, uiA0,
uiB64, uiB0 ));
70

```

```

71 }
72

```

28. riscv-software-src_riscv-isa-sim/softfloat/f128_le_quiet.c (78 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 bool f128_le_quiet( float128_t a, float128_t b )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     union ui128_f128 uB;
49     uint_fast64_t uiB64, uiB0;
50     bool signA, signB;
51
52     uA.f = a;
53     uiA64 = uA.ui.v64;
54     uiA0 = uA.ui.v0;
55     uB.f = b;
56     uiB64 = uB.ui.v64;
57     uiB0 = uB.ui.v0;
58     if ( isNaNF128UI( uiA64, uiA0 ) || isNaNF128UI( uiB64,
uiB0 ) ) {
59         if (
60             softfloat_isNaNF128UI( uiA64, uiA0 )
61             || softfloat_isNaNF128UI( uiB64, uiB0 )
62         ) {
63             softfloat_raiseFlags( softfloat_flag_invalid );
64         }
65         return false;
66     }
67     signA = signF128UI64( uiA64 );
68     signB = signF128UI64( uiB64 );
69     return
70         (signA != signB)

```



```

71         ? signA
72         || ! (((uiA64 | uiB64) &
UINT64_C( 0x7FFFFFFFFFFFFFFF ))
73         | uiA0 | uiB0)
74         : ((uiA64 == uiB64) && (uiA0 == uiB0))
75         || (signA ^ softfloat_lt128( uiA64, uiA0,
uiB64, uiB0 ));
76
77 }
78

```

29. riscv-software-src_riscv-isa-sim/softfloat/f128_lt.c (72 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bool f128_lt( float128_t a, float128_t b )
44 {
45     union ui128_f128 uA;
46     uint_fast64_t uiA64, uiA0;
47     union ui128_f128 uB;
48     uint_fast64_t uiB64, uiB0;
49     bool signA, signB;
50
51     uA.f = a;
52     uiA64 = uA.ui.v64;
53     uiA0 = uA.ui.v0;
54     uB.f = b;
55     uiB64 = uB.ui.v64;
56     uiB0 = uB.ui.v0;
57     if ( isNaNF128UI( uiA64, uiA0 ) || isNaNF128UI( uiB64,
uiB0 ) ) {
58         softfloat_raiseFlags( softfloat_flag_invalid );
59         return false;
60     }
61     signA = signF128UI64( uiA64 );
62     signB = signF128UI64( uiB64 );

```

```

63     return
64     (signA != signB)
65     ? signA
66     && (((uiA64 | uiB64) &
UINT64_C( 0x7FFFFFFFFFFFFFFF ))
67     | uiA0 | uiB0)
68     : ((uiA64 != uiB64) || (uiA0 != uiB0))
69     && (signA ^ softfloat_lt128( uiA64, uiA0,
uiB64, uiB0 ));
70
71 }
72

```

30. riscv-software-src_riscv-isa-sim/softfloat/f128_lt_quiet.c (78 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 bool f128_lt_quiet( float128_t a, float128_t b )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     union ui128_f128 uB;
49     uint_fast64_t uiB64, uiB0;
50     bool signA, signB;
51
52     uA.f = a;
53     uiA64 = uA.ui.v64;
54     uiA0 = uA.ui.v0;
55     uB.f = b;
56     uiB64 = uB.ui.v64;
57     uiB0 = uB.ui.v0;
58     if ( isNaNF128UI( uiA64, uiA0 ) || isNaNF128UI( uiB64,
uiB0 ) ) {
59         if (
60             softfloat_isNaNF128UI( uiA64, uiA0 )

```



```

61         || softfloat_isSigNaNF128UI( uiB64, uiB0 )
62     ) {
63         softfloat_raiseFlags( softfloat_flag_invalid );
64     }
65     return false;
66 }
67 signA = signF128UI64( uiA64 );
68 signB = signF128UI64( uiB64 );
69 return
70     (signA != signB)
71     ? signA
72     && (((uiA64 | uiB64) &
UINT64_C( 0x7FFFFFFFFFFFFFFF ))
73         | uiA0 | uiB0)
74     : ((uiA64 != uiB64) || (uiA0 != uiB0))
75         && (signA ^ softfloat_lt128( uiA64, uiA0,
uiB64, uiB0 ));
76 }
77 }
78

```

31. riscv-software-src_riscv-isa-sim/softfloat/f128_mul.c (163 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36 #include <stdbool.h>
37 #include <stdint.h>
38 #include "platform.h"
39 #include "internals.h"
40 #include "specialize.h"
41 #include "softfloat.h"
42
43 float128_t f128_mul( float128_t a, float128_t b )
44 {
45     union ui128_f128 uA;
46     uint_fast64_t uiA64, uiA0;
47     bool signA;
48     int_fast32_t expA;
49     struct uint128 sigA;
50     union ui128_f128 uB;
51     uint_fast64_t uiB64, uiB0;
52     bool signB;

```

```

54     int_fast32_t expB;
55     struct uint128 sigB;
56     bool signZ;
57     uint_fast64_t magBits;
58     struct exp32_sig128 normExpSig;
59     int_fast32_t expZ;
60     uint64_t sig256Z[4];
61     uint_fast64_t sigZExtra;
62     struct uint128 sigZ;
63     struct uint128_extra sig128Extra;
64     struct uint128 uiZ;
65     union ui128_f128 uZ;
66
67     /
-----
68
-----*/
69     uA.f = a;
70     uiA64 = uA.ui.v64;
71     uiA0 = uA.ui.v0;
72     signA = signF128UI64( uiA64 );
73     expA = expF128UI64( uiA64 );
74     sigA.v64 = fracF128UI64( uiA64 );
75     sigA.v0 = uiA0;
76     uB.f = b;
77     uiB64 = uB.ui.v64;
78     uiB0 = uB.ui.v0;
79     signB = signF128UI64( uiB64 );
80     expB = expF128UI64( uiB64 );
81     sigB.v64 = fracF128UI64( uiB64 );
82     sigB.v0 = uiB0;
83     signZ = signA ^ signB;
84     /
-----
85
-----*/
86     if ( expA == 0x7FFF ) {
87         if (
88             (sigA.v64 | sigA.v0) || ((expB == 0x7FFF) &&
(sigB.v64 | sigB.v0))
89         ) {
90             goto propagateNaN;
91         }
92         magBits = expB | sigB.v64 | sigB.v0;
93         goto infArg;
94     }
95     if ( expB == 0x7FFF ) {
96         if ( sigB.v64 | sigB.v0 ) goto propagateNaN;
97         magBits = expA | sigA.v64 | sigA.v0;
98         goto infArg;
99     }
100     /
-----
101
-----*/
102     if ( ! expA ) {
103         if ( ! (sigA.v64 | sigA.v0) ) goto zero;
104
105         normExpSig = softfloat_normSubnormalF128Sig( sigA.v64,
sigA.v0 );
106         expA = normExpSig.exp;
107         sigA = normExpSig.sig;
108     }
109     if ( ! expB ) {
110         if ( ! (sigB.v64 | sigB.v0) ) goto zero;
111
112         normExpSig = softfloat_normSubnormalF128Sig( sigB.v64,
sigB.v0 );
113         expB = normExpSig.exp;
114         sigB = normExpSig.sig;
115     }
116     /
-----
117
-----*/
118     expZ = expA + expB - 0x4000;
119     sigA.v64 |= UINT64_C( 0x0001000000000000 );
120     sigB = softfloat_shortShiftLeft128( sigB.v64, sigB.v0,
16 );
121     softfloat_mul128To256M( sigA.v64, sigA.v0, sigB.v64,
sigB.v0, sig256Z );
122     sigZExtra = sig256Z[indexWord( 4, 1 )] |
(sig256Z[indexWord( 4, 0 )] != 0);
123     sigZ =
124         softfloat_add128(
125             sig256Z[indexWord( 4, 3 )], sig256Z[indexWord( 4,
2 )],
126             sigA.v64, sigA.v0
127         );
128     if ( UINT64_C( 0x0002000000000000 ) <= sigZ.v64 ) {
129         ++expZ;
130         sig128Extra =
131             softfloat_shortShiftRightJam128Extra(

```

```

130         sigZ.v64, sigZ.v0, sigZExtra, 1 );
131         sigZ = sig128Extra.v;
132         sigZExtra = sig128Extra.extra;
133     }
134     return
135         softfloat_roundPackToF128( signZ, expZ, sigZ.v64,
sigZ.v0, sigZExtra );
136     /
*-----
137
*-----
*-----*/
138     propagateNaN:
139         uiZ = softfloat_propagateNaNF128UI( uiA64, uiA0, uiB64,
uiB0 );
140         goto uiZ;
141     /
*-----
142
*-----
*-----*/
143     infArg:
144         if ( ! magBits ) {
145             softfloat_raiseFlags( softfloat_flag_invalid );
146             uiZ.v64 = defaultNaNF128UI64;
147             uiZ.v0 = defaultNaNF128UI0;
148             goto uiZ;
149         }
150         uiZ.v64 = packToF128UI64( signZ, 0x7FFF, 0 );
151         goto uiZ0;
152     /
*-----
153
*-----
*-----*/
154     zero:
155         uiZ.v64 = packToF128UI64( signZ, 0, 0 );
156     uiZ0:
157         uiZ.v0 = 0;
158     uiZ:
159         uZ.ui = uiZ;
160         return uZ.f;
161
162 }
163

```

32. riscv-software-src_riscv-isa-sim/softfloat/f128_mulAdd.c (63 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-

```

```

EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====*/
36
37 #include <stdint.h>
38 #include "platform.h"
39 #include "internals.h"
40 #include "softfloat.h"
41
42 float128_t f128_mulAdd( float128_t a, float128_t b, float128_t
c )
43 {
44     union ui128_f128 uA;
45     uint_fast64_t uiA64, uiA0;
46     union ui128_f128 uB;
47     uint_fast64_t uiB64, uiB0;
48     union ui128_f128 uC;
49     uint_fast64_t uiC64, uiC0;
50
51     uA.f = a;
52     uiA64 = uA.ui.v64;
53     uiA0 = uA.ui.v0;
54     uB.f = b;
55     uiB64 = uB.ui.v64;
56     uiB0 = uB.ui.v0;
57     uC.f = c;
58     uiC64 = uC.ui.v64;
59     uiC0 = uC.ui.v0;
60     return softfloat_mulAddF128( uiA64, uiA0, uiB64, uiB0,
uiC64, uiC0, 0 );
61
62 }
63

```

33. riscv-software-src_riscv-isa-sim/softfloat/f128_rem.c (190 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35

```

```

=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float128_t f128_rem( float128_t a, float128_t b )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool signA;
49     int_fast32_t expA;
50     struct uint128 sigA;
51     union ui128_f128 uB;
52     uint_fast64_t uiB64, uiB0;
53     int_fast32_t expB;
54     struct uint128 sigB;
55     struct exp32_sig128 normExpSig;
56     struct uint128 rem;
57     int_fast32_t expDiff;
58     uint_fast32_t q, recip32;
59     uint_fast64_t q64;
60     struct uint128 term, altRem, meanRem;
61     bool signRem;
62     struct uint128 uiZ;
63     union ui128_f128 uZ;
64
65     /
*
-----
66
*
-----*/
67     uA.f = a;
68     uiA64 = uA.ui.v64;
69     uiA0 = uA.ui.v0;
70     signA = signF128UI64( uiA64 );
71     expA = expF128UI64( uiA64 );
72     sigA.v64 = fracF128UI64( uiA64 );
73     sigA.v0 = uiA0;
74     uB.f = b;
75     uiB64 = uB.ui.v64;
76     uiB0 = uB.ui.v0;
77     expB = expF128UI64( uiB64 );
78     sigB.v64 = fracF128UI64( uiB64 );
79     sigB.v0 = uiB0;
80     /
*
-----
81
*
-----*/
82     if ( expA == 0x7FFF ) {
83         if (
84             (sigA.v64 | sigA.v0) || ((expB == 0x7FFF) &&
(sigB.v64 | sigB.v0))
85         ) {
86             goto propagateNaN;
87         }
88         goto invalid;
89     }
90     if ( expB == 0x7FFF ) {
91         if ( sigB.v64 | sigB.v0 ) goto propagateNaN;
92         return a;
93     }
94     /
*
-----
95
*
-----*/
96     if ( ! expB ) {
97         if ( ! (sigB.v64 | sigB.v0) ) goto invalid;
98         normExpSig = softfloat_normSubnormalF128Sig( sigB.v64,
sigB.v0 );
99         expB = normExpSig.exp;
100         sigB = normExpSig.sig;
101     }
102     if ( ! expA ) {
103         if ( ! (sigA.v64 | sigA.v0) ) return a;
104         normExpSig = softfloat_normSubnormalF128Sig( sigA.v64,
sigA.v0 );
105         expA = normExpSig.exp;
106         sigA = normExpSig.sig;
107     }
108     /
*
-----
109
*
-----*/
110     sigA.v64 |= UINT64_C( 0x0001000000000000 );
111     sigB.v64 |= UINT64_C( 0x0001000000000000 );
112     rem = sigA;
113     expDiff = expA - expB;

```

```

114     if ( expDiff < 1 ) {
115         if ( expDiff < -1 ) return a;
116         if ( expDiff ) {
117             --expB;
118             sigB = softfloat_add128( sigB.v64, sigB.v0,
sigB.v64, sigB.v0 );
119             q = 0;
120         } else {
121             q = softfloat_le128( sigB.v64, sigB.v0, rem.v64,
rem.v0 );
122             if ( q ) {
123                 rem = softfloat_sub128( rem.v64, rem.v0,
sigB.v64, sigB.v0 );
124             }
125         }
126     } else {
127         recip32 = softfloat_approxRecip32_1( sigB.v64>>17 );
128         expDiff -= 30;
129         for (;;) {
130             q64 = (uint_fast64_t) (uint32_t) (rem.v64>>19) *
recip32;
131             if ( expDiff < 0 ) break;
132             q = (q64 + 0x80000000)>>32;
133             rem = softfloat_shortShiftLeft128( rem.v64,
rem.v0, 29 );
134             term = softfloat_mul128By32( sigB.v64, sigB.v0,
q );
135             rem = softfloat_sub128( rem.v64, rem.v0, term.v64,
term.v0 );
136             if ( rem.v64 & UINT64_C( 0x8000000000000000 ) ) {
137                 rem = softfloat_add128( rem.v64, rem.v0,
sigB.v64, sigB.v0 );
138             }
139             expDiff -= 29;
140         }
141         /
*-----
-
142         | ('expDiff' cannot be less than -29 here.)
143
*-----
-*/
144         q = (uint32_t) (q64>>32)>>(~expDiff & 31);
145         rem = softfloat_shortShiftLeft128( rem.v64, rem.v0,
expDiff + 30 );
146         term = softfloat_mul128By32( sigB.v64, sigB.v0, q );
147         rem = softfloat_sub128( rem.v64, rem.v0, term.v64,
term.v0 );
148         if ( rem.v64 & UINT64_C( 0x8000000000000000 ) ) {
149             altRem = softfloat_add128( rem.v64, rem.v0,
sigB.v64, sigB.v0 );
150             goto selectRem;
151         }
152         /
153
*-----
-----
154
*-----
-----*/
155         do {
156             altRem = rem;
157             ++q;
158             rem = softfloat_sub128( rem.v64, rem.v0, sigB.v64,
sigB.v0 );
159         } while ( ! (rem.v64 & UINT64_C( 0x8000000000000000 )) );
160         selectRem:
161         meanRem = softfloat_add128( rem.v64, rem.v0, altRem.v64,
altRem.v0 );
162         if (
163             (meanRem.v64 & UINT64_C( 0x8000000000000000 ))
164             || (! (meanRem.v64 | meanRem.v0) && (q & 1))
165         ) {
166             rem = altRem;
167         }
168         signRem = signA;
169         if ( rem.v64 & UINT64_C( 0x8000000000000000 ) ) {
170             signRem = ! signRem;
171             rem = softfloat_sub128( 0, 0, rem.v64, rem.v0 );
172         }
173         return softfloat_normRoundPackToF128( signRem, expB - 1,
rem.v64, rem.v0 );
174         /
*-----
-----
175
*-----
-----*/
176         propagateNaN:
177         uiZ = softfloat_propagateNaNF128UI( uiA64, uiA0, uiB64,
uiB0 );
178         goto uiZ;
179         /
*-----
-----
180
*-----
-----*/
181         invalid:
182         softfloat_raiseFlags( softfloat_flag_invalid );

```

```

183     uiZ.v64 = defaultNaNF128UI64;
184     uiZ.v0 = defaultNaNF128UI0;
185     uiZ:
186     uZ.ui = uiZ;
187     return uZ.f;
188 }
189 }
190

```

34. riscv-software-src_riscv-isa-sim/softfloat/ f128_roundToInt.c (160 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2017 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float128_t
45 f128_roundToInt( float128_t a, uint_fast8_t roundingMode,
bool exact )
46 {
47     union ui128_f128 uA;
48     uint_fast64_t uiA64, uiA0;
49     int_fast32_t exp;
50     struct uint128 uiZ;
51     uint_fast64_t lastBitMask, roundBitsMask;
52     bool roundNearEven;
53     union ui128_f128 uZ;
54
55     /
=====
56
=====
57     uA.f = a;
58     uiA64 = uA.ui.v64;
59     uiA0 = uA.ui.v0;
60     exp = expF128UI64( uiA64 );
61     /
=====

```

```

-----
62
-----
63     if ( 0x402F <= exp ) {
64         /
=====
65
=====
66     if ( 0x406F <= exp ) {
67         if ( (exp == 0x7FFF) && (fracF128UI64( uiA64 ) |
uiA0) ) {
68             uiZ = softfloat_propagateNaNF128UI( uiA64,
uiA0, 0, 0 );
69             goto uiZ;
70         }
71         return a;
72     }
73     /
=====
74
=====
75     lastBitMask = (uint_fast64_t) 2<<(0x406E - exp);
76     roundBitsMask = lastBitMask - 1;
77     uiZ.v64 = uiA64;
78     uiZ.v0 = uiA0;
79     roundNearEven = (roundingMode == soft-
float_round_near_even);
80     if ( roundNearEven || (roundingMode == soft-
float_round_near_maxMag) ) {
81         if ( exp == 0x402F ) {
82             if ( UINT64_C( 0x8000000000000000 ) <=
uiZ.v0 ) {
83                 ++uiZ.v64;
84                 if (
85                     roundNearEven
86                     && (uiZ.v0 ==
UINT64_C( 0x8000000000000000 ))
87                 ) {
88                     uiZ.v64 &= ~1;
89                 }
90             } else {
91                 uiZ = softfloat_add128( uiZ.v64, uiZ.v0, 0,
lastBitMask>>1 );
92             }
93             if ( roundNearEven && ! (uiZ.v0 & roundBits-
Mask) ) {
94                 uiZ.v0 &= ~lastBitMask;
95             }
96         } else if (
97             roundingMode
98             == (signF128UI64( uiZ.v64 ) ? soft-
float_round_min
99                 : softfloat_round_max)
100         ) {
101             uiZ = softfloat_add128( uiZ.v64, uiZ.v0, 0, round-
BitsMask );
102         }
103         uiZ.v0 &= ~roundBitsMask;
104     } else {
105         /
=====
106
-----
107
=====
108     if ( exp < 0x3FFF ) {
109         if ( ! ( (uiA64 & UINT64_C( 0x7FFFFFFFFFFFFFFF )) |
uiA0) ) {
110             return a;
111         }
112         if ( exact ) softfloat_exceptionFlags |= soft-
float_flag_inexact;
113         uiZ.v64 = uiA64 & packToF128UI64( 1, 0, 0 );
114         uiZ.v0 = 0;
115         switch ( roundingMode ) {
116             case softfloat_round_near_even:
117                 if ( ! (fracF128UI64( uiA64 ) | uiA0) ) break;
118             case softfloat_round_near_maxMag:
119                 if ( exp == 0x3FFE ) uiZ.v64 |= pack-
ToF128UI64( 0, 0x3FFF, 0 );
120                 break;
121             case softfloat_round_min:
122                 if ( uiZ.v64 ) uiZ.v64 = packToF128UI64( 1,
0x3FFF, 0 );
123                 break;
124             case softfloat_round_max:
125                 if ( ! uiZ.v64 ) uiZ.v64 = packToF128UI64( 0,
0x3FFF, 0 );
126                 break;
127         }
128         goto uiZ;
129     }
130     /
=====

```

```

*-----
-
131
*-----
-*/
132     uiZ.v64 = uiA64;
133     uiZ.v0 = 0;
134     lastBitMask = (uint_fast64_t) 1<<(0x402F - exp);
135     roundBitsMask = lastBitMask - 1;
136     if ( roundingMode == softfloat_round_near_maxMag ) {
137         uiZ.v64 += lastBitMask>>1;
138     } else if ( roundingMode == soft-
float_round_near_even ) {
139         uiZ.v64 += lastBitMask>>1;
140         if ( ! ((uiZ.v64 & roundBitsMask) | uiA0) ) {
141             uiZ.v64 &= ~lastBitMask;
142         }
143     } else if (
144         roundingMode
145         == (signF128UI64( uiZ.v64 ) ? soft-
float_round_min
146             : softfloat_round_max)
147     ) {
148         uiZ.v64 = (uiZ.v64 | (uiA0 != 0)) + roundBitsMask;
149     }
150     uiZ.v64 &= ~roundBitsMask;
151 }
152 if ( exact && ((uiZ.v64 != uiA64) || (uiZ.v0 != uiA0)) ) {
153     softfloat_exceptionFlags |= softfloat_flag_inexact;
154 }
155 uiZ:
156     uZ.ui = uiZ;
157     return uZ.f;
158 }
159 }
160

```

35. riscv-software-src_riscv-isa-sim/softfloat/f128_sqrt.c (201 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>

```

```

38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float128_t f128_sqrt( float128_t a )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool signA;
49     int_fast32_t expA;
50     struct uint128 sigA, uiZ;
51     struct exp32_sig128 normExpSig;
52     int_fast32_t expZ;
53     uint_fast32_t sig32A, recipSqrt32, sig32Z;
54     struct uint128 rem;
55     uint32_t qs[3];
56     uint_fast32_t q;
57     uint_fast64_t x64, sig64Z;
58     struct uint128 y, term;
59     uint_fast64_t sigZExtra;
60     struct uint128 sigZ;
61     union ui128_f128 uZ;
62
63     /
*-----
-----
64
*-----
-----*/
65     uA.f = a;
66     uiA64 = uA.ui.v64;
67     uiA0 = uA.ui.v0;
68     signA = signF128UI64( uiA64 );
69     expA = expF128UI64( uiA64 );
70     sigA.v64 = fracF128UI64( uiA64 );
71     sigA.v0 = uiA0;
72     /
*-----
-----
73
*-----
-----*/
74     if ( expA == 0x7FFF ) {
75         if ( sigA.v64 | sigA.v0 ) {
76             uiZ = softfloat_propagateNaNF128UI( uiA64, uiA0,
0, 0 );
77             goto uiZ;
78         }
79         if ( ! signA ) return a;
80         goto invalid;
81     }
82     /
*-----
-----
83
*-----
-----*/
84     if ( signA ) {
85         if ( ! (expA | sigA.v64 | sigA.v0) ) return a;
86         goto invalid;
87     }
88     /
*-----
-----
89
*-----
-----*/
90     if ( ! expA ) {
91         if ( ! (sigA.v64 | sigA.v0) ) return a;
92     }
93     normExpSig = softfloat_normSubnormalF128Sig( sigA.v64,
sigA.v0 );
94     expA = normExpSig.exp;
95     sigA = normExpSig.sig;
96     /
*-----
-----
97     | ('sig32Z' is guaranteed to be a lower bound on the
square root of
98     | 'sig32A', which makes 'sig32Z' also a lower bound on the
square root of
99     | 'sigA'. )
100
*-----
-----*/
101     expZ = ((expA - 0x3FFF)>>1) + 0x3FFE;
102     expA &= 1;
103     sigA.v64 |= UINT64_C( 0x0001000000000000 );
104     sig32A = sigA.v64>>17;
105     recipSqrt32 = softfloat_approxRecipSqrt32_1( expA,
sig32A );
106     sig32Z = ((uint_fast64_t) sig32A * recipSqrt32)>>32;
107     if ( expA ) {
108         sig32Z >>= 1;
109         rem = softfloat_shortShiftLeft128( sigA.v64, sigA.v0,
12 );
110     } else {

```

```

111     rem = softfloat_shortShiftLeft128( sigA.v64, sigA.v0,
13 );
112 }
113 qs[2] = sig32Z;
114 rem.v64 -= (uint_fast64_t) sig32Z * sig32Z;
115 /
-----
116
-----*/
117 q = ((uint32_t) (rem.v64>>2) * (uint_fast64_t) re-
cipSqrt32)>>32;
118 x64 = (uint_fast64_t) sig32Z<<32;
119 sig64Z = x64 + ((uint_fast64_t) q<<3);
120 y = softfloat_shortShiftLeft128( rem.v64, rem.v0, 29 );
121 /
-----
122 | (Repeating this loop is a rare occurrence.)
123
-----
124 for (;;) {
125     term = softfloat_mul64ByShifted32To128( x64 + sig64Z,
q );
126     rem = softfloat_sub128( y.v64, y.v0, term.v64,
term.v0 );
127     if ( ! (rem.v64 & UINT64_C( 0x8000000000000000 )) )
break;
128     --q;
129     sig64Z -= 1<<3;
130 }
131 qs[1] = q;
132 /
-----
133
-----*/
134 q = ((rem.v64>>2) * recipSqrt32)>>32;
135 y = softfloat_shortShiftLeft128( rem.v64, rem.v0, 29 );
136 sig64Z <= 1;
137 /
-----
138 | (Repeating this loop is a rare occurrence.)
139
-----
140 for (;;) {
141     term = softfloat_shortShiftLeft128( 0, sig64Z, 32 );
142     term = softfloat_add128( term.v64, term.v0, 0,
(uint_fast64_t) q<<6 );
143     term = softfloat_mul128By32( term.v64, term.v0, q );
144     rem = softfloat_sub128( y.v64, y.v0, term.v64,
term.v0 );
145     if ( ! (rem.v64 & UINT64_C( 0x8000000000000000 )) )
break;
146     --q;
147 }
148 qs[0] = q;
149 /
-----
150
-----*/
151 q = (((rem.v64>>2) * recipSqrt32)>>32) + 2;
152 sigZExtra = (uint64_t) ((uint_fast64_t) q<<59);
153 term = softfloat_shortShiftLeft128( 0, qs[1], 53 );
154 sigZ =
155     softfloat_add128(
156         (uint_fast64_t) qs[2]<<18, ((uint_fast64_t)
qs[0]<<24) + (q>>5),
157         term.v64, term.v0
158     );
159 /
-----
160
-----
161 if ( (q & 0xF) <= 2 ) {
162     q &= ~3;
163     sigZExtra = (uint64_t) ((uint_fast64_t) q<<59);
164     y = softfloat_shortShiftLeft128( sigZ.v64, sigZ.v0,
6 );
165     y.v0 |= sigZExtra>>58;
166     term = softfloat_sub128( y.v64, y.v0, 0, q );
167     y = softfloat_mul64ByShifted32To128( term.v0, q );
168     term = softfloat_mul64ByShifted32To128( term.v64, q );
169     term = softfloat_add128( term.v64, term.v0, 0,
y.v64 );
170     rem = softfloat_shortShiftLeft128( rem.v64, rem.v0,
20 );
171     term = softfloat_sub128( term.v64, term.v0, rem.v64,
rem.v0 );
172     /
-----
173
-----

```

```

173 | The concatenation of `term' and `y.v0' is now the
negative remainder
174 | (3 words altogether).
175
-----
176 if ( term.v64 & UINT64_C( 0x8000000000000000 ) ) {
177     sigZExtra |= 1;
178 } else {
179     if ( term.v64 | term.v0 | y.v0 ) {
180         if ( sigZExtra ) {
181             --sigZExtra;
182         } else {
183             sigZ = softfloat_sub128( sigZ.v64,
sigZ.v0, 0, 1 );
184             sigZExtra = ~0;
185         }
186     }
187 }
188 }
189 return softfloat_roundPackToF128( 0, expZ, sigZ.v64,
sigZ.v0, sigZExtra );
190 /
-----
191
-----*/
192 invalid:
193     softfloat_raiseFlags( softfloat_flag_invalid );
194     uiZ.v64 = defaultNaNF128UI64;
195     uiZ.v0 = defaultNaNF128UI0;
196     uiZ:
197         uZ.ui = uiZ;
198         return uZ.f;
199
200 }
201

```

36. riscv-software-src_riscv-isa-sim/softfloat/f128_sub.c (78 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014 The Regents of the University of
California.
8 All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/

```



```

36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 float128_t f128_sub( float128_t a, float128_t b )
44 {
45     union ui128_f128 uA;
46     uint_fast64_t uiA64, uiA0;
47     bool signA;
48     union ui128_f128 uB;
49     uint_fast64_t uiB64, uiB0;
50     bool signB;
51     #if ! defined INLINE_LEVEL || (INLINE_LEVEL < 2)
52         float128_t
53         (*magsFuncPtr)(
54             uint_fast64_t, uint_fast64_t, uint_fast64_t,
55             uint_fast64_t, bool );
56     #endif
57     uA.f = a;
58     uiA64 = uA.ui.v64;
59     uiA0 = uA.ui.v0;
60     signA = signF128UI64( uiA64 );
61     uB.f = b;
62     uiB64 = uB.ui.v64;
63     uiB0 = uB.ui.v0;
64     signB = signF128UI64( uiB64 );
65     #if defined INLINE_LEVEL && (2 <= INLINE_LEVEL)
66         if ( signA == signB ) {
67             return softfloat_subMagsF128( uiA64, uiA0, uiB64,
68             uiB0, signA );
69         } else {
70             return softfloat_addMagsF128( uiA64, uiA0, uiB64,
71             uiB0, signA );
72         }
73     #else
74         magsFuncPtr =
75         (signA == signB) ? softfloat_subMagsF128 : soft-
76         float_addMagsF128;
77         return (*magsFuncPtr)( uiA64, uiA0, uiB64, uiB0, signA );
78     #endif
79 }

```

37. riscv-software-src_riscv-isa-sim/softfloat/f128_to_f16.c (95 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-

```

```

ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float16_t f128_to_f16( float128_t a )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool sign;
49     int_fast32_t exp;
50     uint_fast64_t frac64;
51     struct commonNaN commonNaN;
52     uint_fast16_t uiZ, frac16;
53     union ui16_f16 uZ;
54
55     /
-----
56
-----*/
57     uA.f = a;
58     uiA64 = uA.ui.v64;
59     uiA0 = uA.ui.v0;
60     sign = signF128UI64( uiA64 );
61     exp = expF128UI64( uiA64 );
62     frac64 = fracF128UI64( uiA64 ) | (uiA0 != 0);
63     /
-----
64
-----*/
65     if ( exp == 0x7FFF ) {
66         if ( frac64 ) {
67             softfloat_f128UIToCommonNaN( uiA64, uiA0, &common-
NaN );
68             uiZ = softfloat_commonNaNToF16UI( &commonNaN );
69         } else {
70             uiZ = packToF16UI( sign, 0x1F, 0 );
71         }
72         goto uiZ;
73     }
74     /
-----
75
-----*/
76     frac16 = softfloat_shortShiftRightJam64( frac64, 34 );
77     if ( ! (exp | frac16) ) {
78         uiZ = packToF16UI( sign, 0, 0 );
79         goto uiZ;
80     }
81     /
-----
82
-----*/
83     exp -= 0x3FF1;
84     if ( sizeof( int_fast16_t ) < sizeof( int_fast32_t ) ) {
85         if ( exp < -0x40 ) exp = -0x40;
86     }
87     return softfloat_roundPackToF16( sign, exp, frac16 |
0x4000 );
88     /
-----
89
-----*/
90     uiZ:
91     uZ.ui = uiZ;
92     return uZ.f;
93
94 }
95

```

38. riscv-software-src_riscv-isa-sim/softfloat/f128_to_f32.c (95 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-

```



```

Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float32_t f128_to_f32( float128_t a )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool sign;
49     int_fast32_t exp;
50     uint_fast64_t frac64;
51     struct commonNaN commonNaN;
52     uint_fast32_t uiZ, frac32;
53     union ui32_f32 uZ;
54
55     /
56
57     -----*/
58
59     -----*/
60     uA.f = a;
61     uiA64 = uA.ui.v64;
62     uiA0 = uA.ui.v0;
63     sign = signF128UI64( uiA64 );
64     exp = expF128UI64( uiA64 );
65     frac64 = fracF128UI64( uiA64 ) | (uiA0 != 0);
66     /
67
68     -----*/
69
70     -----*/
71     if ( exp == 0x7FFF ) {
72         if ( frac64 ) {
73             softfloat_f128UIToCommonNaN( uiA64, uiA0, &commonNaN );
74             uiZ = softfloat_commonNaNToF32UI( &commonNaN );
75         } else {
76             uiZ = packToF32UI( sign, 0xFF, 0 );
77         }
78         goto uiZ;
79     }
80     /
81
82     -----*/
83
84     -----*/
85
86     -----*/
87
88     -----*/
89
90     -----*/
91
92     -----*/
93
94     -----*/
95
96     -----*/
97
98     -----*/
99
100     -----*/
101
102     -----*/
103
104     -----*/
105
106     -----*/
107
108     -----*/
109
110     -----*/
111
112     -----*/
113
114     -----*/
115
116     -----*/
117
118     -----*/
119
120     -----*/
121
122     -----*/
123
124     -----*/
125
126     -----*/
127
128     -----*/
129
130     -----*/
131
132     -----*/
133
134     -----*/
135
136     -----*/
137
138     -----*/
139
140     -----*/
141
142     -----*/
143
144     -----*/
145
146     -----*/
147
148     -----*/
149
150     -----*/
151
152     -----*/
153
154     -----*/
155
156     -----*/
157
158     -----*/
159
160     -----*/
161
162     -----*/
163
164     -----*/
165
166     -----*/
167
168     -----*/
169
170     -----*/
171
172     -----*/
173
174     -----*/
175
176     -----*/
177
178     -----*/
179
180     -----*/
181
182     -----*/
183
184     -----*/
185
186     -----*/
187
188     -----*/
189
190     -----*/
191
192     -----*/
193
194     -----*/
195
196     -----*/
197
198     -----*/
199
200     -----*/
201
202     -----*/
203
204     -----*/
205
206     -----*/
207
208     -----*/
209
210     -----*/
211
212     -----*/
213
214     -----*/
215
216     -----*/
217
218     -----*/
219
220     -----*/
221
222     -----*/
223
224     -----*/
225
226     -----*/
227
228     -----*/
229
230     -----*/
231
232     -----*/
233
234     -----*/
235
236     -----*/
237
238     -----*/
239
240     -----*/
241
242     -----*/
243
244     -----*/
245
246     -----*/
247
248     -----*/
249
250     -----*/
251
252     -----*/
253
254     -----*/
255
256     -----*/
257
258     -----*/
259
260     -----*/
261
262     -----*/
263
264     -----*/
265
266     -----*/
267
268     -----*/
269
270     -----*/
271
272     -----*/
273
274     -----*/
275
276     -----*/
277
278     -----*/
279
280     -----*/
281
282     -----*/
283
284     -----*/
285
286     -----*/
287
288     -----*/
289
290     -----*/
291
292     -----*/
293
294     -----*/
295
296     -----*/
297
298     -----*/
299
300     -----*/
301
302     -----*/
303
304     -----*/
305
306     -----*/
307
308     -----*/
309
310     -----*/
311
312     -----*/
313
314     -----*/
315
316     -----*/
317
318     -----*/
319
320     -----*/
321
322     -----*/
323
324     -----*/
325
326     -----*/
327
328     -----*/
329
330     -----*/
331
332     -----*/
333
334     -----*/
335
336     -----*/
337
338     -----*/
339
340     -----*/
341
342     -----*/
343
344     -----*/
345
346     -----*/
347
348     -----*/
349
350     -----*/
351
352     -----*/
353
354     -----*/
355
356     -----*/
357
358     -----*/
359
360     -----*/
361
362     -----*/
363
364     -----*/
365
366     -----*/
367
368     -----*/
369
370     -----*/
371
372     -----*/
373
374     -----*/
375
376     -----*/
377
378     -----*/
379
380     -----*/
381
382     -----*/
383
384     -----*/
385
386     -----*/
387
388     -----*/
389
390     -----*/
391
392     -----*/
393
394     -----*/
395
396     -----*/
397
398     -----*/
399
400     -----*/
401
402     -----*/
403
404     -----*/
405
406     -----*/
407
408     -----*/
409
410     -----*/
411
412     -----*/
413
414     -----*/
415
416     -----*/
417
418     -----*/
419
420     -----*/
421
422     -----*/
423
424     -----*/
425
426     -----*/
427
428     -----*/
429
430     -----*/
431
432     -----*/
433
434     -----*/
435
436     -----*/
437
438     -----*/
439
440     -----*/
441
442     -----*/
443
444     -----*/
445
446     -----*/
447
448     -----*/
449
450     -----*/
451
452     -----*/
453
454     -----*/
455
456     -----*/
457
458     -----*/
459
460     -----*/
461
462     -----*/
463
464     -----*/
465
466     -----*/
467
468     -----*/
469
470     -----*/
471
472     -----*/
473
474     -----*/
475
476     -----*/
477
478     -----*/
479
480     -----*/
481
482     -----*/
483
484     -----*/
485
486     -----*/
487
488     -----*/
489
490     -----*/
491
492     -----*/
493
494     -----*/
495
496     -----*/
497
498     -----*/
499
500     -----*/
501
502     -----*/
503
504     -----*/
505
506     -----*/
507
508     -----*/
509
510     -----*/
511
512     -----*/
513
514     -----*/
515
516     -----*/
517
518     -----*/
519
520     -----*/
521
522     -----*/
523
524     -----*/
525
526     -----*/
527
528     -----*/
529
530     -----*/
531
532     -----*/
533
534     -----*/
535
536     -----*/
537
538     -----*/
539
540     -----*/
541
542     -----*/
543
544     -----*/
545
546     -----*/
547
548     -----*/
549
550     -----*/
551
552     -----*/
553
554     -----*/
555
556     -----*/
557
558     -----*/
559
560     -----*/
561
562     -----*/
563
564     -----*/
565
566     -----*/
567
568     -----*/
569
570     -----*/
571
572     -----*/
573
574     -----*/
575
576     -----*/
577
578     -----*/
579
580     -----*/
581
582     -----*/
583
584     -----*/
585
586     -----*/
587
588     -----*/
589
590     -----*/
591
592     -----*/
593
594     -----*/
595
596     -----*/
597
598     -----*/
599
600     -----*/
601
602     -----*/
603
604     -----*/
605
606     -----*/
607
608     -----*/
609
610     -----*/
611
612     -----*/
613
614     -----*/
615
616     -----*/
617
618     -----*/
619
620     -----*/
621
622     -----*/
623
624     -----*/
625
626     -----*/
627
628     -----*/
629
630     -----*/
631
632     -----*/
633
634     -----*/
635
636     -----*/
637
638     -----*/
639
640     -----*/
641
642     -----*/
643
644     -----*/
645
646     -----*/
647
648     -----*/
649
650     -----*/
651
652     -----*/
653
654     -----*/
655
656     -----*/
657
658     -----*/
659
660     -----*/
661
662     -----*/
663
664     -----*/
665
666     -----*/
667
668     -----*/
669
670     -----*/
671
672     -----*/
673
674     -----*/
675
676     -----*/
677
678     -----*/
679
680     -----*/
681
682     -----*/
683
684     -----*/
685
686     -----*/
687
688     -----*/
689
690     -----*/
691
692     -----*/
693
694     -----*/
695
696     -----*/
697
698     -----*/
699
700     -----*/
701
702     -----*/
703
704     -----*/
705
706     -----*/
707
708     -----*/
709
710     -----*/
711
712     -----*/
713
714     -----*/
715
716     -----*/
717
718     -----*/
719
720     -----*/
721
722     -----*/
723
724     -----*/
725
726     -----*/
727
728     -----*/
729
730     -----*/
731
732     -----*/
733
734     -----*/
735
736     -----*/
737
738     -----*/
739
740     -----*/
741
742     -----*/
743
744     -----*/
745
746     -----*/
747
748     -----*/
749
750     -----*/
751
752     -----*/
753
754     -----*/
755
756     -----*/
757
758     -----*/
759
760     -----*/
761
762     -----*/
763
764     -----*/
765
766     -----*/
767
768     -----*/
769
770     -----*/
771
772     -----*/
773
774     -----*/
775
776     -----*/
777
778     -----*/
779
780     -----*/
781
782     -----*/
783
784     -----*/
785
786     -----*/
787
788     -----*/
789
790     -----*/
791
792     -----*/
793
794     -----*/
795
796     -----*/
797
798     -----*/
799
800     -----*/
801
802     -----*/
803
804     -----*/
805
806     -----*/
807
808     -----*/
809
810     -----*/
811
812     -----*/
813
814     -----*/
815
816     -----*/
817
818     -----*/
819
820     -----*/
821
822     -----*/
823
824     -----*/
825
826     -----*/
827
828     -----*/
829
830     -----*/
831
832     -----*/
833
834     -----*/
835
836     -----*/
837
838     -----*/
839
840     -----*/
841
842     -----*/
843
844     -----*/
845
846     -----*/
847
848     -----*/
849
850     -----*/
851
852     -----*/
853
854     -----*/
855
856     -----*/
857
858     -----*/
859
860     -----*/
861
862     -----*/
863
864     -----*/
865
866     -----*/
867
868     -----*/
869
870     -----*/
871
872     -----*/
873
874     -----*/
875
876     -----*/
877
878     -----*/
879
880     -----*/
881
882     -----*/
883
884     -----*/
885
886     -----*/
887
888     -----*/
889
890     -----*/
891
892     -----*/
893
894     -----*/
895
896     -----*/
897
898     -----*/
899
900     -----*/
901
902     -----*/
903
904     -----*/
905
906     -----*/
907
908     -----*/
909
910     -----*/
911
912     -----*/
913
914     -----*/
915
916     -----*/
917
918     -----*/
919
920     -----*/
921
922     -----*/
923
924     -----*/
925
926     -----*/
927
928     -----*/
929
930     -----*/
931
932     -----*/
933
934     -----*/
935
936     -----*/
937
938     -----*/
939
940     -----*/
941
942     -----*/
943
944     -----*/
945
946     -----*/
947
948     -----*/
949
950     -----*/
951
952     -----*/
953
954     -----*/
955
956     -----*/
957
958     -----*/
959
960     -----*/
961
962     -----*/
963
964     -----*/
965
966     -----*/
967
968     -----*/
969
970     -----*/
971
972     -----*/
973
974     -----*/
975
976     -----*/
977
978     -----*/
979
980     -----*/
981
982     -----*/
983
984     -----*/
985
986     -----*/
987
988     -----*/
989
990     -----*/
991
992     -----*/
993
994     -----*/
995
996     -----*/
997
998     -----*/
999
1000     -----*/

```

```

75
*-----*/
76     frac32 = softfloat_shortShiftRightJam64( frac64, 18 );
77     if ( ! (exp | frac32) ) {
78         uiZ = packToF32UI( sign, 0, 0 );
79         goto uiZ;
80     }
81     /
82
83     -----*/
84     exp -= 0x3F81;
85     if ( sizeof( int_fast16_t ) < sizeof( int_fast32_t ) ) {
86         if ( exp < -0x1000 ) exp = -0x1000;
87     }
88     return softfloat_roundPackToF32( sign, exp, frac32 |
0x40000000 );
89
90     -----*/
91     uiZ:
92     uZ.ui = uiZ;
93     return uZ.f;
94 }
95

```

39. riscv-software-src_riscv-isa-sim/softfloat/f128_to_f64.c (100 lines)

```

1
2 /
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"

```

```

43
44 float64_t f128_to_f64( float128_t a )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool sign;
49     int_fast32_t exp;
50     uint_fast64_t frac64, frac0;
51     struct commonNaN commonNaN;
52     uint_fast64_t uiZ;
53     struct uint128 frac128;
54     union ui64_f64 uZ;
55
56     /
57
58     -----
59     */
60     uA.f = a;
61     uiA64 = uA.ui.v64;
62     uiA0 = uA.ui.v0;
63     sign = signF128UI64( uiA64 );
64     exp = expF128UI64( uiA64 );
65     frac64 = fracF128UI64( uiA64 );
66     frac0 = uiA0;
67
68     -----
69     */
70     if ( exp == 0x7FFF ) {
71         if ( frac64 | frac0 ) {
72             softfloat_f128UIToCommonNaN( uiA64, uiA0, &commonNaN );
73         }
74         uiZ = softfloat_commonNaNToF64UI( &commonNaN );
75     } else {
76         uiZ = packToF64UI( sign, 0x7FF, 0 );
77     }
78     goto uiZ;
79
80     -----
81     */
82     frac128 = softfloat_shortShiftLeft128( frac64, frac0,
83     14 );
84     frac64 = frac128.v64 | (frac128.v0 != 0);
85     if ( ! (exp | frac64) ) {
86         uiZ = packToF64UI( sign, 0, 0 );
87         goto uiZ;
88     }
89
90     -----
91     */
92     exp -= 0x3C01;
93     if ( sizeof( int_fast16_t ) < sizeof( int_fast32_t ) ) {
94         if ( exp < -0x1000 ) exp = -0x1000;
95     }
96     return
97         softfloat_roundPackToF64(
98             sign, exp, frac64 |
99             UINT64_C( 0x4000000000000000 ) );
100
101     -----
102     */
103     uiZ:
104     uZ.ui = uiZ;
105     return uZ.f;
106 }

```

40. riscv-software-src_riscv-isa-sim/softfloat/f128_to_i32.c (85 lines)

```

1
2 /
3
4 This C source file is part of the SoftFloat IEEE Floating-
5 Point Arithmetic
6 Package, Release 3d, by John R. Hauser.
7
8 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
9 the
10 University of California. All rights reserved.
11
12 Redistribution and use in source and binary forms, with or

```

```

without
11 modification, are permitted provided that the following condi-
12 tions are met:
13
14 1. Redistributions of source code must retain the above copy-
15 right notice,
16 this list of conditions, and the following disclaimer.
17
18 2. Redistributions in binary form must reproduce the above
19 copyright notice,
20 this list of conditions, and the following disclaimer in
21 the documentation
22 and/or other materials provided with the distribution.
23
24 3. Neither the name of the University nor the names of its
25 contributors may
26 be used to endorse or promote products derived from this
27 software without
28 specific prior written permission.
29
30 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
31 IS", AND ANY
32 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
33 THE IMPLIED
34 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
35 PURPOSE, ARE
36 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
37 LIABLE FOR ANY
38 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
39 SEQUENTIAL DAMAGES
40 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
41 GOODS OR SERVICES;
42 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
43 EVER CAUSED AND
44 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
45 ILITY, OR TORT
46 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
47 THE USE OF THIS
48 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
49
50 =====
51 */
52
53 #include <stdbool.h>
54 #include <stdint.h>
55 #include "platform.h"
56 #include "internals.h"
57 #include "specialize.h"
58 #include "softfloat.h"
59
60 int_fast32_t f128_to_i32( float128_t a, uint_fast8_t rounding-
61 Mode, bool exact )
62 {
63     union ui128_f128 uA;
64     uint_fast64_t uiA64, uiA0;
65     bool sign;
66     int_fast32_t exp;
67     uint_fast64_t sig64, sig0;
68     int_fast32_t shiftDist;
69
70     /
71
72     -----
73     */
74     uA.f = a;
75     uiA64 = uA.ui.v64;
76     uiA0 = uA.ui.v0;
77     sign = signF128UI64( uiA64 );
78     exp = expF128UI64( uiA64 );
79     sig64 = fracF128UI64( uiA64 );
80     sig0 = uiA0;
81
82     -----
83     */
84
85     -----
86     */
87     if ( i32_fromNaN != i32_fromPosOverflow ) || ( i32_fromNaN !=
88     i32_fromNegOverflow )
89     if ( ( exp == 0x7FFF ) && ( sig64 | sig0 ) ) {
90         if ( i32_fromNaN == i32_fromPosOverflow )
91             sign = 0;
92         else if ( i32_fromNaN == i32_fromNegOverflow )
93             sign = 1;
94         else
95             softfloat_raiseFlags( softfloat_flag_invalid );
96         return i32_fromNaN;
97     }
98     #endif
99     }
100     #endif
101
102     -----
103     */
104     if ( exp ) sig64 |= UINT64_C( 0x0001000000000000 );
105     sig64 |= ( sig0 != 0 );

```

```

80     shiftDist = 0x4023 - exp;
81     if ( 0 < shiftDist ) sig64 = soft-
float_shiftRightJam64( sig64, shiftDist );
82     return softfloat_roundToI32( sign, sig64, roundingMode,
exact );
83 }
84 }
85

```

41. riscv-software-src_riscv-isa-sim/softfloat/ f128_to_i32_r_minMag.c (100 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 int_fast32_t f128_to_i32_r_minMag( float128_t a, bool exact )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     int_fast32_t exp;
49     uint_fast64_t sig64;
50     int_fast32_t shiftDist;
51     bool sign;
52     int_fast32_t absZ;
53
54     /
-----
55
-----
-----*/
56     uA.f = a;
57     uiA64 = uA.ui.v64;
58     uiA0 = uA.ui.v0;
59     exp = expF128UI64( uiA64 );
60     sig64 = fracF128UI64( uiA64 ) | (uiA0 != 0);
61     /
-----

```

```

-----
62
-----*/
63     shiftDist = 0x402F - exp;
64     if ( 49 <= shiftDist ) {
65         if ( exact && (exp | sig64) ) {
66             softfloat_exceptionFlags |= soft-
float_flag_inexact;
67         }
68         return 0;
69     }
70     /
-----
71
-----
-----*/
72     sign = signF128UI64( uiA64 );
73     if ( shiftDist < 18 ) {
74         if (
75             sign && (shiftDist == 17)
76             && (sig64 < UINT64_C( 0x00000000000020000 ))
77         ) {
78             if ( exact && sig64 ) {
79                 softfloat_exceptionFlags |= soft-
float_flag_inexact;
80             }
81             return -0x7FFFFFFF - 1;
82         }
83         softfloat_raiseFlags( softfloat_flag_invalid );
84         return
85             (exp == 0x7FFF) && sig64 ? i32_fromNaN
86             : sign ? i32_fromNegOverflow : i32_fromPosOv-
erflow;
87     }
88     /
-----
89
-----
-----*/
90     sig64 |= UINT64_C( 0x0001000000000000 );
91     absZ = sig64>>shiftDist;
92     if (
93         exact && ((uint_fast64_t) (uint_fast32_t) absZ<<shift-
Dist != sig64)
94     ) {
95         softfloat_exceptionFlags |= softfloat_flag_inexact;
96     }
97     return sign ? -absZ : absZ;
98
99 }
100

```

42. riscv-software-src_riscv-isa-sim/softfloat/f128_to_i64.c (95 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE

```

```

TABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 int_fast64_t f128_to_i64( float128_t a, uint_fast8_t rounding-
Mode, bool exact )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool sign;
49     int_fast32_t exp;
50     uint_fast64_t sig64, sig0;
51     int_fast32_t shiftDist;
52     struct uint128 sig128;
53     struct uint64_extra sigExtra;
54
55     /
-----
56
-----*/
57     uA.f = a;
58     uiA64 = uA.ui.v64;
59     uiA0 = uA.ui.v0;
60     sign = signF128UI64( uiA64 );
61     exp = expF128UI64( uiA64 );
62     sig64 = fracF128UI64( uiA64 );
63     sig0 = uiA0;
64     /
-----
65
-----*/
66     shiftDist = 0x402F - exp;
67     if ( shiftDist <= 0 ) {
68         /
-----
69
-----*/
70         if ( shiftDist < -15 ) {
71             softfloat_raiseFlags( softfloat_flag_invalid );
72             return
73                 (exp == 0x7FFF) && (sig64 | sig0) ?
i64_fromNaN
74                 : sign ? i64_fromNegOverflow :
i64_fromPosOverflow;
75         }
76         /
-----
77
-----*/
78         sig64 |= UINT64_C( 0x0001000000000000 );
79         if ( shiftDist ) {
80             sig128 = softfloat_shortShiftLeft128( sig64, sig0, -
shiftDist );
81             sig64 = sig128.v64;
82             sig0 = sig128.v0;
83         } else {
84             /
85
-----
86
-----*/
87         if ( exp ) sig64 |= UINT64_C( 0x0001000000000000 );
88         sigExtra = softfloat_shiftRightJam64Extra( sig64,
sig0, shiftDist );
89         sig64 = sigExtra.v;
90         sig0 = sigExtra.extra;
91     }
92     return softfloat_roundToI64( sign, sig64, sig0, rounding-
Mode, exact );
93

```

94 }
95 }

43. [riscv-software-src_riscv-isa-sim/softfloat/f128_to_i64_r_minMag.c](#) (113 lines)

```

2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====
*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 int_fast64_t f128_to_i64_r_minMag( float128_t a, bool exact )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool sign;
49     int_fast32_t exp;
50     uint_fast64_t sig64, sig0;
51     int_fast32_t shiftDist;
52     int_fast8_t negShiftDist;
53     int_fast64_t absZ;
54
55     /
*-----
*-----
56
*-----
*-----
*/
57     uA.f = a;
58     uiA64 = uA.ui.v64;
59     uiA0 = uA.ui.v0;
60     sign = signF128UI64( uiA64 );
61     exp = expF128UI64( uiA64 );
62     sig64 = fracF128UI64( uiA64 );
63     sig0 = uiA0;
64     /
*-----
*-----
65
*
```

```

-----*/
66     shiftDist = 0x402F - exp;
67     if ( shiftDist < 0 ) {
68         /
*-----
69
*-----
-*/
70     if ( shiftDist < -14 ) {
71         if (
72             (uiA64 == UINT64_C( 0xC03E000000000000 ))
73             && (sig0 < UINT64_C( 0x0002000000000000 ))
74         ) {
75             if ( exact && sig0 ) {
76                 softfloat_exceptionFlags |= soft-
float_flag_inexact;
77             }
78             return -INT64_C( 0x7FFFFFFF ) - 1;
79         }
80         softfloat_raiseFlags( softfloat_flag_invalid );
81         return
82             (exp == 0x7FFF) && (sig64 | sig0) ?
i64_fromNaN
83             : sign ? i64_fromNegOverflow :
i64_fromPosOverflow;
84     }
85     /
*-----
-
86
*-----
-*/
87     sig64 |= UINT64_C( 0x0001000000000000 );
88     negShiftDist = -shiftDist;
89     absZ = sig64<<negShiftDist | sig0>>(shiftDist & 63);
90     if ( exact && (uint64_t) (sig0<<negShiftDist) ) {
91         softfloat_exceptionFlags |= soft-
float_flag_inexact;
92     }
93     } else {
94         /
*-----
-
95
*-----
-*/
96     if ( 49 <= shiftDist ) {
97         if ( exact && (exp | sig64 | sig0) ) {
98             softfloat_exceptionFlags |= soft-
float_flag_inexact;
99         }
100         return 0;
101     }
102     /
*-----
-
103
*-----
-*/
104     sig64 |= UINT64_C( 0x0001000000000000 );
105     absZ = sig64>>shiftDist;
106     if ( exact && (sig0 || (absZ<<shiftDist != sig64)) ) {
107         softfloat_exceptionFlags |= soft-
float_flag_inexact;
108     }
109     }
110     return sign ? -absZ : absZ;
111 }
112 }
113

```

44. riscv-software-src_riscv-isa-sim/softfloat/f128_to_ui32.c (86 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,

```

```

17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 uint_fast32_t
45 f128_to_ui32( float128_t a, uint_fast8_t roundingMode, bool
exact )
46 {
47     union ui128_f128 uA;
48     uint_fast64_t uiA64, uiA0;
49     bool sign;
50     int_fast32_t exp;
51     uint_fast64_t sig64;
52     int_fast32_t shiftDist;
53
54     /
*-----
----
55
*-----
-----*/
56     uA.f = a;
57     uiA64 = uA.ui.v64;
58     uiA0 = uA.ui.v0;
59     sign = signF128UI64( uiA64 );
60     exp = expF128UI64( uiA64 );
61     sig64 = fracF128UI64( uiA64 ) | (uiA0 != 0);
62     /
*-----
----
63
*-----
-----*/
64 #if (ui32_fromNaN != ui32_fromPosOverflow) || (ui32_fromNaN !=
ui32_fromNegOverflow)
65     if ( (exp == 0x7FFF) && sig64 ) {
66         #if (ui32_fromNaN == ui32_fromPosOverflow)
67             sign = 0;
68         #elif (ui32_fromNaN == ui32_fromNegOverflow)
69             sign = 1;
70         #else
71             softfloat_raiseFlags( softfloat_flag_invalid );
72             return ui32_fromNaN;
73         #endif
74     }
75     #endif
76     /
*-----
----
77
*-----
-----*/
78     if ( exp ) sig64 |= UINT64_C( 0x0001000000000000 );
79     shiftDist = 0x4023 - exp;
80     if ( 0 < shiftDist ) {
81         sig64 = softfloat_shiftRightJam64( sig64, shiftDist );
82     }
83     return softfloat_roundToUI32( sign, sig64, roundingMode,
exact );
84
85 }
86

```

45. riscv-software-src_riscv-isa-sim/softfloat/f128_to_ui32_r_minMag.c (89 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 uint_fast32_t f128_to_ui32_r_minMag( float128_t a, bool ex-
act )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     int_fast32_t exp;
49     uint_fast64_t sig64;
50     int_fast32_t shiftDist;
51     bool sign;
52     uint_fast32_t z;
53
54     /
55
56     uA.f = a;
57     uiA64 = uA.ui.v64;
58     uiA0 = uA.ui.v0;
59     exp = expF128UI64( uiA64 );
60     sig64 = fracF128UI64( uiA64 ) | (uiA0 != 0);
61     /
62
63     shiftDist = 0x402F - exp;
64     if ( 49 <= shiftDist ) {
65         if ( exact && (exp | sig64) ) {

```

```

66         softfloat_exceptionFlags |= soft-
float_flag_inexact;
67     }
68     return 0;
69     /
70
71
-----
72
73 sign = signF128UI64( uiA64 );
74 if ( sign || (shiftDist < 17) ) {
75     softfloat_raiseFlags( softfloat_flag_invalid );
76     return
77         (exp == 0x7FFF) && sig64 ? ui32_fromNaN
78         : sign ? ui32_fromNegOverflow :
79         ui32_fromPosOverflow;
80     /
81
82 sig64 |= UINT64_C( 0x0001000000000000 );
83 z = sig64>>shiftDist;
84 if ( exact && ((uint_fast64_t) z<<shiftDist != sig64) ) {
85     softfloat_exceptionFlags |= softfloat_flag_inexact;
86 }
87 return z;
88 }
89

```

46. riscv-software-src_riscv-isa-sim/softfloat/f128_to_ui64.c (96 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016, 2017 The Regents of
the
8 University of California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"

```



```

40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 uint_fast64_t
45 f128_to_ui64( float128_t a, uint_fast8_t roundingMode, bool
exact )
46 {
47     union ui128_f128 uA;
48     uint_fast64_t uiA64, uiA0;
49     bool sign;
50     int_fast32_t exp;
51     uint_fast64_t sig64, sig0;
52     int_fast32_t shiftDist;
53     struct uint128 sig128;
54     struct uint64_extra sigExtra;
55
56     /
57
58     *-----*/
59     uA.f = a;
60     uiA64 = uA.ui.v64;
61     uiA0 = uA.ui.v0;
62     sign = signF128UI64( uiA64 );
63     exp = expF128UI64( uiA64 );
64     sig64 = fracF128UI64( uiA64 );
65     sig0 = uiA0;
66
67     *-----*/
68     shiftDist = 0x402F - exp;
69     if ( shiftDist <= 0 ) {
70
71     *-----*/
72     if ( shiftDist < -15 ) {
73         softfloat_raiseFlags( softfloat_flag_invalid );
74         return
75         (exp == 0x7FFF) && (sig64 | sig0) ?
76         ui64_fromNaN : sign ? ui64_fromNegOverflow :
77         ui64_fromPosOverflow;
78
79     *-----*/
80     sig64 |= UINT64_C( 0x0001000000000000 );
81     if ( shiftDist ) {
82         sig128 = softfloat_shortShiftLeft128( sig64, sig0, -
83         shiftDist );
84         sig64 = sig128.v64;
85         sig0 = sig128.v0;
86     } else {
87
88     *-----*/
89     if ( exp ) sig64 |= UINT64_C( 0x0001000000000000 );
90     sigExtra = softfloat_shiftRightJam64Extra( sig64,
91     sig0, shiftDist );
92     sig64 = sigExtra.v;
93     sig0 = sigExtra.extra;
94     return softfloat_roundToUI64( sign, sig64, sig0, rounding-
95     Mode, exact );
96
97

```

47. riscv-software-src_riscv-isa-sim/softfloat/
f128_to_ui64_r_minMag.c (105 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.

```

```

9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 uint_fast64_t f128_to_ui64_r_minMag( float128_t a, bool ex-
act )
45 {
46     union ui128_f128 uA;
47     uint_fast64_t uiA64, uiA0;
48     bool sign;
49     int_fast32_t exp;
50     uint_fast64_t sig64, sig0;
51     int_fast32_t shiftDist;
52     int_fast8_t negShiftDist;
53     uint_fast64_t z;
54
55     /
56
57     *-----*/
58     uA.f = a;
59     uiA64 = uA.ui.v64;
60     uiA0 = uA.ui.v0;
61     sign = signF128UI64( uiA64 );
62     exp = expF128UI64( uiA64 );
63     sig64 = fracF128UI64( uiA64 );
64     sig0 = uiA0;
65
66     *-----*/
67     shiftDist = 0x402F - exp;
68     if ( shiftDist < 0 ) {
69
70     *-----*/
71     if ( sign || (shiftDist < -15) ) goto invalid;
72     sig64 |= UINT64_C( 0x0001000000000000 );
73     negShiftDist = -shiftDist;
74     z = sig64<<negShiftDist | sig0>>(shiftDist & 63);
75     if ( exact && (uint64_t) (sig0<<negShiftDist) ) {
76         softfloat_exceptionFlags |= soft-
77         float_flag_inexact;
78     } else {
79

```



```

78      /
*-----
-
79
*-----
-*/
80      if ( 49 <= shiftDist ) {
81          if ( exact && (exp | sig64 | sig0) ) {
82              softfloat_exceptionFlags |= soft-
float_flag_inexact;
83          }
84          return 0;
85      }
86      /
*-----
-
87
*-----
-*/
88      if ( sign ) goto invalid;
89      sig64 |= UINT64_C( 0x0001000000000000 );
90      z = sig64>>shiftDist;
91      if ( exact && (sig0 || (z<<shiftDist != sig64)) ) {
92          softfloat_exceptionFlags |= soft-
float_flag_inexact;
93      }
94      }
95      return z;
96      /
*-----
-----
97
*-----
-----*/
98      invalid:
99      softfloat_raiseFlags( softfloat_flag_invalid );
100      return
101      (exp == 0x7FFF) && (sig64 | sig0) ? ui64_fromNaN
102      : sign ? ui64_fromNegOverflow : ui64_fromPosOver-
flow;
103
104 }
105

```

48. riscv-software-src_riscv-isa-sim/softfloat/f16_add.c (70 lines)

```

1
2 /
*-----
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

```

```

34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 float16_t f16_add( float16_t a, float16_t b )
44 {
45     union ui16_f16 uA;
46     uint_fast16_t uiA;
47     union ui16_f16 uB;
48     uint_fast16_t uiB;
49     #if ! defined INLINE_LEVEL || (INLINE_LEVEL < 1)
50     float16_t (*magsFuncPtr)( uint_fast16_t, uint_fast16_t );
51     #endif
52
53     uA.f = a;
54     uiA = uA.ui;
55     uB.f = b;
56     uiB = uB.ui;
57     #if defined INLINE_LEVEL && (1 <= INLINE_LEVEL)
58     if ( signF16UI( uiA ^ uiB ) ) {
59         return softfloat_subMagsF16( uiA, uiB );
60     } else {
61         return softfloat_addMagsF16( uiA, uiB );
62     }
63     #else
64     magsFuncPtr =
65     signF16UI( uiA ^ uiB ) ? softfloat_subMagsF16 : soft-
float_addMagsF16;
66     return (*magsFuncPtr)( uiA, uiB );
67     #endif
68 }
69
70

```

49. riscv-software-src_riscv-isa-sim/softfloat/f16_classify.c (36 lines)

```

1
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include "platform.h"
5 #include "internals.h"
6 #include "specialize.h"
7 #include "softfloat.h"
8
9 uint_fast16_t f16_classify( float16_t a )
10 {
11     union ui16_f16 uA;
12     uint_fast16_t uiA;
13
14     uA.f = a;
15     uiA = uA.ui;
16
17     uint_fast16_t infOrNaN = expF16UI( uiA ) == 0x1F;
18     uint_fast16_t subnormalOrZero = expF16UI( uiA ) == 0;
19     bool sign = signF16UI( uiA );
20     bool fracZero = fracF16UI( uiA ) == 0;
21     bool isNaN = isNaNF16UI( uiA );
22     bool isSNaN = softfloat_isNaNF16UI( uiA );
23
24     return
25     ( sign && infOrNaN && fracZero ) << 0 |
26     ( sign && !infOrNaN && !subnormalOrZero ) << 1 |
27     ( sign && subnormalOrZero && !fracZero ) << 2 |
28     ( sign && subnormalOrZero && fracZero ) << 3 |
29     ( !sign && infOrNaN && fracZero ) << 7 |
30     ( !sign && !infOrNaN && !subnormalOrZero ) << 6 |
31     ( !sign && subnormalOrZero && !fracZero ) << 5 |
32     ( !sign && subnormalOrZero && fracZero ) << 4 |
33     ( isNaN && isSNaN ) << 8 |
34     ( isNaN && !isSNaN ) << 9;
35 }
36

```

50. riscv-software-src_riscv-isa-sim/softfloat/f16_div.c (186 lines)

```

1
2 /
*-----
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without

```

```

11 modification, are permitted provided that the following condi-
12 tions are met:
13
14 1. Redistributions of source code must retain the above copy-
15 right notice,
16 this list of conditions, and the following disclaimer.
17
18 2. Redistributions in binary form must reproduce the above
19 copyright notice,
20 this list of conditions, and the following disclaimer in
21 the documentation
22 and/or other materials provided with the distribution.
23
24 3. Neither the name of the University nor the names of its
25 contributors may
26 be used to endorse or promote products derived from this
27 software without
28 specific prior written permission.
29
30 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
31 IS", AND ANY
32 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
33 THE IMPLIED
34 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
35 PURPOSE, ARE
36 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
37 LIABLE FOR ANY
38 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
39 SEQUENTIAL DAMAGES
40 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
41 GOODS OR SERVICES;
42 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
43 EVER CAUSED AND
44 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
45 ILITY, OR TORT
46 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
47 THE USE OF THIS
48 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
49
50 =====*/
51
52 #include <stdbool.h>
53 #include <stdint.h>
54 #include <platform.h>
55 #include "internals.h"
56 #include "specialize.h"
57 #include "softfloat.h"
58
59 extern const uint16_t softfloat_approxRecip_1k0s[];
60 extern const uint16_t softfloat_approxRecip_1k1s[];
61
62 float16_t f16_div( float16_t a, float16_t b )
63 {
64     union ui16_f16 uA;
65     uint_fast16_t uiA;
66     bool signA;
67     int_fast8_t expA;
68     uint_fast16_t sigA;
69     union ui16_f16 uB;
70     uint_fast16_t uiB;
71     bool signB;
72     int_fast8_t expB;
73     uint_fast16_t sigB;
74     bool signZ;
75     struct exp8_sig16 normExpSig;
76     int_fast8_t expZ;
77     #ifdef SOFTFLOAT_FAST_DIV32TO16
78         uint_fast32_t sig32A;
79         uint_fast16_t sigZ;
80     #else
81         int index;
82         uint16_t r0;
83         uint_fast16_t sigZ, rem;
84     #endif
85     uint_fast16_t uiZ;
86     union ui16_f16 uZ;
87
88     /
89
90 -----
91
92 */
93
94 uA.f = a;
95 uiA = uA.ui;
96 signA = signF16UI( uiA );
97 expA = expF16UI( uiA );
98 sigA = fracF16UI( uiA );
99 uB.f = b;
100 uiB = uB.ui;
101 signB = signF16UI( uiB );
102 expB = expF16UI( uiB );
103 sigB = fracF16UI( uiB );
104 signZ = signA ^ signB;
105 /
106
107 -----
108
109 */

```

```

-----*/
88 if ( expA == 0x1F ) {
89     if ( sigA ) goto propagateNaN;
90     if ( expB == 0x1F ) {
91         if ( sigB ) goto propagateNaN;
92         goto invalid;
93     }
94     goto infinity;
95 }
96 if ( expB == 0x1F ) {
97     if ( sigB ) goto propagateNaN;
98     goto zero;
99 }
100 /
101
102 -----
103
104 */
105
106 if ( ! expB ) {
107     if ( ! sigB ) {
108         if ( ! (expA | sigA) ) goto invalid;
109         softfloat_raiseFlags( softfloat_flag_infinite );
110         goto infinity;
111     }
112     normExpSig = softfloat_normSubnormalF16Sig( sigB );
113     expB = normExpSig.exp;
114     sigB = normExpSig.sig;
115 }
116 if ( ! expA ) {
117     if ( ! sigA ) goto zero;
118     normExpSig = softfloat_normSubnormalF16Sig( sigA );
119     expA = normExpSig.exp;
120     sigA = normExpSig.sig;
121 }
122 /
123
124 -----
125
126 */
127
128 expZ = expA - expB + 0xE;
129 sigA |= 0x0400;
130 sigB |= 0x0400;
131 #ifdef SOFTFLOAT_FAST_DIV32TO16
132 if ( sigA < sigB ) {
133     --expZ;
134     sig32A = (uint_fast32_t) sigA<<15;
135 } else {
136     sig32A = (uint_fast32_t) sigA<<14;
137 }
138 sigZ = sig32A / sigB;
139 if ( ! (sigZ & 7) ) sigZ |= ((uint_fast32_t) sigB * sigZ !
140 = sig32A);
141 #else
142 if ( sigA < sigB ) {
143     --expZ;
144     sigA <= 5;
145 } else {
146     sigA <= 4;
147 }
148 index = sigB>>6 & 0xF;
149 r0 = softfloat_approxRecip_1k0s[index]
150 - (((uint_fast32_t) soft-
151 float_approxRecip_1k1s[index]
152 * (sigB & 0x3F))
153 >>10);
154 sigZ = ((uint_fast32_t) sigA * r0)>>16;
155 rem = (sigA<<10) - sigZ * sigB;
156 sigZ += (rem * (uint_fast32_t) r0)>>26;
157 /
158
159 -----
160
161 */
162
163 ++sigZ;
164 if ( ! (sigZ & 7) ) {
165     sigZ &= ~1;
166     rem = (sigA<<10) - sigZ * sigB;
167     if ( rem & 0x8000 ) {
168         sigZ -= 2;
169     } else {
170         if ( rem ) sigZ |= 1;
171     }
172 }
173 #endif
174 return softfloat_roundPackToF16( signZ, expZ, sigZ );
175 /
176
177 -----
178
179 */
180
181 propagateNaN:
182 uiZ = softfloat_propagateNaNF16UI( uiA, uiB );
183 goto uiZ;
184 /
185
186 -----
187
188 */

```

```

*-----
*-----*/
168  invalid:
169      softfloat_raiseFlags( softfloat_flag_invalid );
170      uiZ = defaultNaNF16UI;
171      goto uiZ;
172  /
*-----
*-----
173
*-----
*-----*/
174  infinity:
175      uiZ = packToF16UI( signZ, 0x1F, 0 );
176      goto uiZ;
177  /
*-----
*-----
178
*-----
*-----*/
179  zero:
180      uiZ = packToF16UI( signZ, 0, 0 );
181  uiZ:
182      uZ.ui = uiZ;
183      return uZ.f;
184
185 }
186

```

51. riscv-software-src_riscv-isa-sim/softfloat/f16_eq.c (66 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 bool f16_eq( float16_t a, float16_t b )
45 {

```

```

46  union ui16_f16 uA;
47  uint_fast16_t uiA;
48  union ui16_f16 uB;
49  uint_fast16_t uiB;
50
51  uA.f = a;
52  uiA = uA.ui;
53  uB.f = b;
54  uiB = uB.ui;
55  if ( isNaNF16UI( uiA ) || isNaNF16UI( uiB ) ) {
56      if (
57          softfloat_isSigNaNF16UI( uiA ) || soft-
float_isSigNaNF16UI( uiB )
58      ) {
59          softfloat_raiseFlags( softfloat_flag_invalid );
60      }
61      return false;
62  }
63  return (uiA == uiB) || ! (uint16_t) ((uiA | uiB)<<1);
64
65 }
66

```

52. riscv-software-src_riscv-isa-sim/softfloat/f16_eq_signaling.c (61 lines)

```

1
2 /
*=====
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bool f16_eq_signaling( float16_t a, float16_t b )
44 {
45  union ui16_f16 uA;
46  uint_fast16_t uiA;
47  union ui16_f16 uB;
48  uint_fast16_t uiB;
49
50  uA.f = a;
51  uiA = uA.ui;
52  uB.f = b;

```

```

53  uiB = uB.ui;
54  if ( isNaNF16UI( uiA ) || isNaNF16UI( uiB ) ) {
55      softfloat_raiseFlags( softfloat_flag_invalid );
56      return false;
57  }
58  return (uiA == uiB) || ! (uint16_t) ((uiA | uiB)<<1);
59 }
60 }
61

```

53. riscv-software-src_riscv-isa-sim/softfloat/f16_isSignalingNaN.c (51 lines)

```

1  /
2  /
*=====
=====
3
4  This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5  Package, Release 3d, by John R. Hauser.
6
7  Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8  California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include "platform.h"
39 #include "internals.h"
40 #include "specialize.h"
41 #include "softfloat.h"
42
43 bool f16_isSignalingNaN( float16_t a )
44 {
45     union ui16_f16 uA;
46
47     uA.f = a;
48     return softfloat_isNaNF16UI( uA.ui );
49 }
50 }
51

```

54. riscv-software-src_riscv-isa-sim/softfloat/f16_le.c (66 lines)

```

1  /
2  /
*=====
=====
3
4  This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5  Package, Release 3d, by John R. Hauser.
6
7  Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-

```

```

versity of
8  California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bool f16_le( float16_t a, float16_t b )
44 {
45     union ui16_f16 uA;
46     uint_fast16_t uiA;
47     union ui16_f16 uB;
48     uint_fast16_t uiB;
49     bool signA, signB;
50
51     uA.f = a;
52     uiA = uA.ui;
53     uB.f = b;
54     uiB = uB.ui;
55     if ( isNaNF16UI( uiA ) || isNaNF16UI( uiB ) ) {
56         softfloat_raiseFlags( softfloat_flag_invalid );
57         return false;
58     }
59     signA = signF16UI( uiA );
60     signB = signF16UI( uiB );
61     return
62         (signA != signB) ? signA || ! (uint16_t) ((uiA |
uiB)<<1)
63             : (uiA == uiB) || (signA ^ (uiA < uiB));
64
65 }
66

```

55. riscv-software-src_riscv-isa-sim/softfloat/f16_le_quiet.c (71 lines)

```

1  /
2  /
*=====
=====
3
4  This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5  Package, Release 3d, by John R. Hauser.
6
7  Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8  California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:

```

```

12
13 1. Redistributions of source code must retain the above copy-
right notice,
14     this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17     this list of conditions, and the following disclaimer in
the documentation
18     and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21     be used to endorse or promote products derived from this
software without
22     specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 bool f16_le_quiet( float16_t a, float16_t b )
45 {
46     union ui16_f16 uA;
47     uint_fast16_t uiA;
48     union ui16_f16 uB;
49     uint_fast16_t uiB;
50     bool signA, signB;
51
52     uA.f = a;
53     uiA = uA.ui;
54     uB.f = b;
55     uiB = uB.ui;
56     if ( isNaNF16UI( uiA ) || isNaNF16UI( uiB ) ) {
57         if (
58             softfloat_isSigNaNF16UI( uiA ) || soft-
float_isSigNaNF16UI( uiB )
59         ) {
60             softfloat_raiseFlags( softfloat_flag_invalid );
61         }
62         return false;
63     }
64     signA = signF16UI( uiA );
65     signB = signF16UI( uiB );
66     return
67         (signA != signB) ? signA || ! (uint16_t) ((uiA |
uiB)<1)
68             : (uiA == uiB) || (signA ^ (uiA < uiB));
69 }
70 }
71

```

56. riscv-software-src_riscv-isa-sim/softfloat/f16_lt.c (66 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-

```

```

right notice,
14     this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17     this list of conditions, and the following disclaimer in
the documentation
18     and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21     be used to endorse or promote products derived from this
software without
22     specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "softfloat.h"
42
43 bool f16_lt( float16_t a, float16_t b )
44 {
45     union ui16_f16 uA;
46     uint_fast16_t uiA;
47     union ui16_f16 uB;
48     uint_fast16_t uiB;
49     bool signA, signB;
50
51     uA.f = a;
52     uiA = uA.ui;
53     uB.f = b;
54     uiB = uB.ui;
55     if ( isNaNF16UI( uiA ) || isNaNF16UI( uiB ) ) {
56         softfloat_raiseFlags( softfloat_flag_invalid );
57         return false;
58     }
59     signA = signF16UI( uiA );
60     signB = signF16UI( uiB );
61     return
62         (signA != signB) ? signA && ((uint16_t) ((uiA |
uiB)<1) != 0)
63             : (uiA != uiB) && (signA ^ (uiA < uiB));
64 }
65 }
66

```

57. riscv-software-src_riscv-isa-sim/softfloat/f16_lt_quiet.c

(71 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14     this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17     this list of conditions, and the following disclaimer in
the documentation

```

```

18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
   contributors may
21 be used to endorse or promote products derived from this
   software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
   IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
   THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
   PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
   LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
   SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
   GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
   EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
   ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
   THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
******/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 bool f16_lt_quiet( float16_t a, float16_t b )
45 {
46     union ui16_f16 uA;
47     uint_fast16_t uiA;
48     union ui16_f16 uB;
49     uint_fast16_t uiB;
50     bool signA, signB;
51
52     uA.f = a;
53     uiA = uA.ui;
54     uB.f = b;
55     uiB = uB.ui;
56     if ( isNaNF16UI( uiA ) || isNaNF16UI( uiB ) ) {
57         if (
58             softfloat_isSigNaNF16UI( uiA ) || soft-
float_isSigNaNF16UI( uiB )
59         ) {
60             softfloat_raiseFlags( softfloat_flag_invalid );
61         }
62         return false;
63     }
64     signA = signF16UI( uiA );
65     signB = signF16UI( uiB );
66     return
67         (signA != signB) ? signA && ((uint16_t) ((uiA |
uiB)<<1) != 0)
68         : (uiA != uiB) && (signA ^ (uiA < uiB));
69
70 }
71

```

58. riscv-software-src_riscv-isa-sim/softfloat/f16_mul.c (140 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.

```

```

19
20 3. Neither the name of the University nor the names of its
   contributors may
21 be used to endorse or promote products derived from this
   software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
   IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
   THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
   PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
   LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
   SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
   GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
   EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
   ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
   THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
******/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float16_t f16_mul( float16_t a, float16_t b )
45 {
46     union ui16_f16 uA;
47     uint_fast16_t uiA;
48     bool signA;
49     int_fast8_t expA;
50     uint_fast16_t sigA;
51     union ui16_f16 uB;
52     uint_fast16_t uiB;
53     bool signB;
54     int_fast8_t expB;
55     uint_fast16_t sigB;
56     bool signZ;
57     uint_fast16_t magBits;
58     struct exp8_sig16 normExpSig;
59     int_fast8_t expZ;
60     uint_fast32_t sig32Z;
61     uint_fast16_t sigZ, uiZ;
62     union ui16_f16 uZ;
63
64     /
65
66     uA.f = a;
67     uiA = uA.ui;
68     signA = signF16UI( uiA );
69     expA = expF16UI( uiA );
70     sigA = fracF16UI( uiA );
71     uB.f = b;
72     uiB = uB.ui;
73     signB = signF16UI( uiB );
74     expB = expF16UI( uiB );
75     sigB = fracF16UI( uiB );
76     signZ = signA ^ signB;
77     /
78
79     if ( expA == 0x1F ) {
80         if ( sigA || ((expB == 0x1F) && sigB) ) goto propaga-
ateNaN;
81         magBits = expB | sigB;
82         goto infArg;
83     }
84     if ( expB == 0x1F ) {
85         if ( sigB ) goto propagateNaN;
86         magBits = expA | sigA;
87         goto infArg;
88     }
89     /
90
91     if ( ! expA ) {
92         if ( ! sigA ) goto zero;
93         normExpSig = softfloat_normSubnormalF16Sig( sigA );

```



```

94     expA = normExpSig.exp;
95     sigA = normExpSig.sig;
96 }
97 if ( ! expB ) {
98     if ( ! sigB ) goto zero;
99     normExpSig = softfloat_normSubnormalF16Sig( sigB );
100    expB = normExpSig.exp;
101    sigB = normExpSig.sig;
102 }
103 /
-----
104
-----
105    expZ = expA + expB - 0xF;
106    sigA = (sigA | 0x0400)<<4;
107    sigB = (sigB | 0x0400)<<5;
108    sig32Z = (uint_fast32_t) sigA * sigB;
109    sigZ = sig32Z>>16;
110    if ( sig32Z & 0xFFFF ) sigZ |= 1;
111    if ( sigZ < 0x4000 ) {
112        --expZ;
113        sigZ <= 1;
114    }
115    return softfloat_roundPackToF16( signZ, expZ, sigZ );
116 /
-----
117
-----
118    propagateNaN:
119    uiZ = softfloat_propagateNaNF16UI( uiA, uiB );
120    goto uiZ;
121 /
-----
122
-----
123    infArg:
124    if ( ! magBits ) {
125        softfloat_raiseFlags( softfloat_flag_invalid );
126        uiZ = defaultNaNF16UI;
127    } else {
128        uiZ = packToF16UI( signZ, 0x1F, 0 );
129    }
130    goto uiZ;
131 /
-----
132
-----
133    zero:
134    uiZ = packToF16UI( signZ, 0, 0 );
135    uiZ:
136    uZ.ui = uiZ;
137    return uZ.f;
138 }
139 }
140

```

59. riscv-software-src_riscv-isa-sim/softfloat/f16_mulAdd.c (60 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015 The Regents of the Uni-
versity of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.

```

```

23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
36
37 #include <stdint.h>
38 #include "platform.h"
39 #include "internals.h"
40 #include "softfloat.h"
41
42 float16_t f16_mulAdd( float16_t a, float16_t b, float16_t c )
43 {
44     union ui16_f16 uA;
45     uint_fast16_t uiA;
46     union ui16_f16 uB;
47     uint_fast16_t uiB;
48     union ui16_f16 uC;
49     uint_fast16_t uiC;
50
51     uA.f = a;
52     uiA = uA.ui;
53     uB.f = b;
54     uiB = uB.ui;
55     uC.f = c;
56     uiC = uC.ui;
57     return softfloat_mulAddF16( uiA, uiB, uiC, 0 );
58 }
59 }
60

```

60. riscv-software-src_riscv-isa-sim/softfloat/f16_rem.c (171 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-
Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6
7 Copyright 2011, 2012, 2013, 2014, 2015, 2016 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-

```



```

EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====*/
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float16_t f16_rem( float16_t a, float16_t b )
45 {
46     union ui16_f16 uA;
47     uint_fast16_t uiA;
48     bool signA;
49     int_fast8_t expA;
50     uint_fast16_t sigA;
51     union ui16_f16 uB;
52     uint_fast16_t uiB;
53     int_fast8_t expB;
54     uint_fast16_t sigB;
55     struct exp8_sig16 normExpSig;
56     uint16_t rem;
57     int_fast8_t expDiff;
58     uint_fast16_t q;
59     uint32_t recip32, q32;
60     uint16_t altRem, meanRem;
61     bool signRem;
62     uint_fast16_t uiZ;
63     union ui16_f16 uZ;
64
65     /
66
67     uA.f = a;
68     uiA = uA.ui;
69     signA = signF16UI( uiA );
70     expA = expF16UI( uiA );
71     sigA = fracF16UI( uiA );
72     uB.f = b;
73     uiB = uB.ui;
74     expB = expF16UI( uiB );
75     sigB = fracF16UI( uiB );
76     /
77
78     if ( expA == 0x1F ) {
79         if ( sigA || ((expB == 0x1F) && sigB) ) goto propagateNaN;
80         goto invalid;
81     }
82     if ( expB == 0x1F ) {
83         if ( sigB ) goto propagateNaN;
84         return a;
85     }
86     /
87
88     if ( ! expB ) {
89         if ( ! sigB ) goto invalid;
90         normExpSig = softfloat_normSubnormalF16Sig( sigB );
91         expB = normExpSig.exp;
92         sigB = normExpSig.sig;
93     }
94     if ( ! expA ) {
95         if ( ! sigA ) return a;
96         normExpSig = softfloat_normSubnormalF16Sig( sigA );
97         expA = normExpSig.exp;
98         sigA = normExpSig.sig;
99     }
100     /
101
102     rem = sigA | 0x0400;
103     sigB |= 0x0400;
104     expDiff = expA - expB;
105     if ( expDiff < 1 ) {
106         if ( expDiff < -1 ) return a;
107         sigB <= 3;
108         if ( expDiff ) {
109             rem <= 2;

```

```

110         q = 0;
111     } else {
112         rem <= 3;
113         q = (sigB <= rem);
114         if ( q ) rem -= sigB;
115     }
116 } else {
117     recip32 = softfloat_approxRecip32_1( (uint_fast32_t)
sigB<<21 );
118     /
119
120     | Changing the shift of `rem' here requires also changing the initial
121     | subtraction from `expDiff'.
122
123     rem <= 4;
124     expDiff -= 31;
125
126     | The scale of `sigB' affects how many bits are obtained during each
127     | cycle of the loop. Currently this is 29 bits per loop iteration,
128     | which is believed to be the maximum possible.
129
130     sigB <= 3;
131     for (;;) {
132         q32 = (rem * (uint_fast64_t) recip32)>>16;
133         if ( expDiff < 0 ) break;
134         rem = -((uint_fast16_t) q32 * sigB);
135         expDiff -= 29;
136     }
137
138     | (`expDiff' cannot be less than -30 here.)
139
140     q32 >= ~expDiff & 31;
141     q = q32;
142     rem = (rem<<(expDiff + 30)) - q * sigB;
143     /
144
145     do {
146         altRem = rem;
147         ++q;
148         rem -= sigB;
149     } while ( ! (rem & 0x8000) );
150     meanRem = rem + altRem;
151     if ( (meanRem & 0x8000) || ( ! meanRem && (q & 1)) ) rem =
altRem;
152     signRem = signA;
153     if ( 0x8000 <= rem ) {
154         signRem = ! signRem;
155         rem = -rem;
156     }
157     return softfloat_normRoundPackToF16( signRem, expB, rem );
158     /
159
160     propagateNaN:
161     uiZ = softfloat_propagateNaNF16UI( uiA, uiB );
162     goto uiZ;
163     invalid:
164     softfloat_raiseFlags( softfloat_flag_invalid );
165     uiZ = defaultNaNF16UI;
166     uiZ:
167     uZ.ui = uiZ;
168     return uZ.f;
169
170 }
171

```

61. riscv-software-src_riscv-isa-sim/softfloat/f16_roundToInt.c (112 lines)

```

1
2 /
=====
3
4 This C source file is part of the SoftFloat IEEE Floating-Point Arithmetic
5 Package, Release 3d, by John R. Hauser.
6

```

```

7 Copyright 2011, 2012, 2013, 2014, 2015, 2017 The Regents of
the University of
8 California. All rights reserved.
9
10 Redistribution and use in source and binary forms, with or
without
11 modification, are permitted provided that the following condi-
tions are met:
12
13 1. Redistributions of source code must retain the above copy-
right notice,
14 this list of conditions, and the following disclaimer.
15
16 2. Redistributions in binary form must reproduce the above
copyright notice,
17 this list of conditions, and the following disclaimer in
the documentation
18 and/or other materials provided with the distribution.
19
20 3. Neither the name of the University nor the names of its
contributors may
21 be used to endorse or promote products derived from this
software without
22 specific prior written permission.
23
24 THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS "AS
IS", AND ANY
25 EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
THE IMPLIED
26 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
PURPOSE, ARE
27 DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
LIABLE FOR ANY
28 DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON-
SEQUENTIAL DAMAGES
29 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE
GOODS OR SERVICES;
30 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOW-
EVER CAUSED AND
31 ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIAB-
ILITY, OR TORT
32 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF
THE USE OF THIS
33 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
34
35
=====
36
37 #include <stdbool.h>
38 #include <stdint.h>
39 #include "platform.h"
40 #include "internals.h"
41 #include "specialize.h"
42 #include "softfloat.h"
43
44 float16_t f16_roundToInt( float16_t a, uint_fast8_t rounding-
Mode, bool exact )
45 {
46     union ui16_f16 uA;
47     uint_fast16_t uiA;
48     int_fast8_t exp;
49     uint_fast16_t uiZ, lastBitMask, roundBitsMask;
50     union ui16_f16 uZ;
51
52     /
53
54     uA.f = a;

```

```

55     uiA = uA.ui;
56     exp = expF16UI( uiA );
57     /
58
59     if ( exp <= 0xE ) {
60         if ( ! (uint16_t) (uiA<<1) ) return a;
61         if ( exact ) softfloat_exceptionFlags |= soft-
float_flag_inexact;
62         uiZ = uiA & packToF16UI( 1, 0, 0 );
63         switch ( roundingMode ) {
64             case softfloat_round_near_even:
65                 if ( ! fracF16UI( uiA ) ) break;
66             case softfloat_round_near_maxMag:
67                 if ( exp == 0xE ) uiZ |= packToF16UI( 0, 0xF, 0 );
68                 break;
69             case softfloat_round_min:
70                 if ( uiZ ) uiZ = packToF16UI( 1, 0xF, 0 );
71                 break;
72             case softfloat_round_max:
73                 if ( ! uiZ ) uiZ = packToF16UI( 0, 0xF, 0 );
74                 break;
75         }
76         goto uiZ;
77     }
78     /
79
80     if ( 0x19 <= exp ) {
81         if ( (exp == 0x1F) && fracF16UI( uiA ) ) {
82             uiZ = softfloat_propagateNaNF16UI( uiA, 0 );
83             goto uiZ;
84         }
85         return a;
86     }
87     /
88
89     uiZ = uiA;
90     lastBitMask = (uint_fast16_t) 1<<(0x19 - exp);
91     roundBitsMask = lastBitMask - 1;
92     if ( roundingMode == softfloat_round_near_maxMag ) {
93         uiZ += lastBitMask>>1;
94     } else if ( roundingMode == softfloat_round_near_even ) {
95         uiZ += lastBitMask>>1;
96         if ( ! (uiZ & roundBitsMask) ) uiZ &= ~lastBitMask;
97     } else if (
98         roundingMode
99         == (signF16UI( uiZ ) ? softfloat_round_min : soft-
float_round_max)
100     ) {
101         uiZ += roundBitsMask;
102     }
103     uiZ &= ~roundBitsMask;
104     if ( exact && (uiZ != uiA) ) {
105         softfloat_exceptionFlags |= softfloat_flag_inexact;
106     }
107     uiZ:
108     uZ.ui = uiZ;
109     return uZ.f;
110
111 }
112

```

RISC-V 형식 명세 (Sail)

11 files • 1,939 lines

1. riscv_sail-riscv/model/extensions/Sstc/sstc.sail (61 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8 // Sstc - supervisor time compare
9 mapping clause csr_name_map = 0x14D <-> "stimecmp"
10 mapping clause csr_name_map = 0x15D <-> "stimecmph"
11
12 private function is_stimecmp_accessible(priv : Privilege) ->
bool =
13   currentlyEnabled(Ext_S) & currentlyEnabled(Ext_Sstc) &
14   (match priv {
15     Machine => true,
16     Supervisor => mcounteren[TM] == 0b1 & menvcfg[STCE] ==
0b1,
17     User => false,
18     VirtualSupervisor => mcounteren[TM] == 0b1 & menvcfg[STCE] ==
0b1 &
19     hcounteren[TM] == 0b1 & read_hen-
vcfg()[STCE] == 0b1,
20     VirtualUser => false,
21   })
22
23 // The `stimecmp` and `vstimecmp` CSRs are defined in `core/
interrupt_regs.sail`.
24 // Timer interrupts due to `stimecmp` and `vstimecmp` are up-
dated in
25 // `core/interrupt_regs:update_mip()`.
26
27 function clause is_CSR_accessible(0x14D, priv, _) = is_s-
timecmp_accessible(priv)
28 function clause is_CSR_accessible(0x15D, priv, _) = is_s-
timecmp_accessible(priv) & xlen == 32
29
30 function clause read_CSR(0x14D) = if inVirtualPrivilege()
then vstimecmp[xlen - 1 .. 0] else stimecmp[xlen - 1 .. 0]
31 function clause read_CSR(0x15D if xlen == 32) = if inVirtual-
Privilege() then vstimecmp[63 .. 32] else stimecmp[63 .. 32]
32
33 function clause write_CSR(0x14D, value) =
34   if inVirtualPrivilege()
35 then { vstimecmp[(xlen - 1) .. 0] = value; Ok(vstimecmp[xlen -
1 .. 0]) }
36 else { stimecmp[(xlen - 1) .. 0] = value; Ok(stimecmp[xlen -
1 .. 0]) }
37
38 function clause write_CSR((0x15D, value) if xlen == 32) =
39   if inVirtualPrivilege()
40 then { vstimecmp[63 .. 32] = value; Ok(vstimecmp[63 ..
32]) }
41 else { stimecmp[63 .. 32] = value; Ok(stimecmp[63 .. 32]) }
42
43 // virtual supervisor timer compare
44 mapping clause csr_name_map = 0x24D <-> "vstimecmp"
45 mapping clause csr_name_map = 0x25D <-> "vstimecmph"
46
47 // "when the TM bit in the mcounteren register is clear, at-
tempts to access the
48 // stimecmp or vstimecmp register while executing in a mode
less privileged than
49 // M will cause an illegal-instruction exception."
50 private function is_vstimecmp_accessible(priv : Privilege) ->
bool =
51   currentlyEnabled(Ext_H) & currentlyEnabled(Ext_Sstc) &
52   (priv == Machine | (mcounteren[TM] == 0b1 & menvcfg[STCE] ==
0b1))
53
54 function clause is_CSR_accessible(0x24D, priv, _) = is_vs-
timecmp_accessible(priv)
55 function clause is_CSR_accessible(0x25D, priv, _) = is_vs-
timecmp_accessible(priv) & xlen == 32
56
57 function clause read_CSR(0x24D) = vstimecmp[xlen - 1 .. 0]
58 function clause read_CSR(0x25D if xlen == 32) = vs-

```

```

timecmp[63 .. 32]
59
60 function clause write_CSR(0x24D, value) = { vstimecmp[(xlen -
1) .. 0] = value; Ok(vstimecmp[xlen - 1 .. 0]) }
61 function clause write_CSR((0x25D, value) if xlen == 32) =
{ vstimecmp[63 .. 32] = value; Ok(vstimecmp[63 .. 32]) }

```

2. riscv_sail-riscv/model/extensions/Zicntr/zicntr_control.sail (52 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8 function clause currentlyEnabled(Ext_Zicntr) = hartSup-
ports(Ext_Zicntr) & currentlyEnabled(Ext_Zicrs)
9
10 // unprivileged counters/timers
11 mapping clause csr_name_map = 0xC00 <-> "cycle"
12 mapping clause csr_name_map = 0xC01 <-> "time"
13 mapping clause csr_name_map = 0xC02 <-> "instret"
14 mapping clause csr_name_map = 0xC80 <-> "cycleh"
15 mapping clause csr_name_map = 0xC81 <-> "timeh"
16 mapping clause csr_name_map = 0xC82 <-> "instreth"
17
18 function clause is_CSR_accessible(0xC00, priv, _) = currently-
Enabled(Ext_Zicntr) & counter_enabled(0, priv) // cycle
19 function clause is_CSR_accessible(0xC01, priv, _) = currently-
Enabled(Ext_Zicntr) & counter_enabled(1, priv) // time
20 function clause is_CSR_accessible(0xC02, priv, _) = currently-
Enabled(Ext_Zicntr) & counter_enabled(2, priv) // instret
21 function clause is_CSR_accessible(0xC80, priv, _) = currently-
Enabled(Ext_Zicntr) & xlen == 32 & counter_enabled(0, priv) //
cycleh
22 function clause is_CSR_accessible(0xC81, priv, _) = currently-
Enabled(Ext_Zicntr) & xlen == 32 & counter_enabled(1, priv) //
timeh
23 function clause is_CSR_accessible(0xC82, priv, _) = currently-
Enabled(Ext_Zicntr) & xlen == 32 & counter_enabled(2, priv) //
instreth
24
25 function clause read_CSR(0xC00) = mcycle[(xlen - 1) .. 0]
26
26 function clause read_CSR(0xC01) = get_mtime()[(xlen - 1) .. 0] +
(if inVirtualPrivilege() then htimedelta[(xlen - 1) .. 0] else zer-
os())
27 function clause read_CSR(0xC02) = minstret[(xlen - 1) .. 0]
28 function clause read_CSR(0xC80 if xlen == 32) = mcycle[63 ..
32]
29
29 function clause read_CSR(0xC81 if xlen == 32) = (get_mtime() +
(if inVirtualPrivilege() then htimedelta else zeros()))[63 .. 32]
30
30 function clause read_CSR(0xC82 if xlen == 32) = minstret[63 ..
32]
31
32 // machine counters/timers
33 mapping clause csr_name_map = 0xB00 <-> "mcycle"
34 mapping clause csr_name_map = 0xB02 <-> "minstret"
35 mapping clause csr_name_map = 0xB80 <-> "mcycleh"
36 mapping clause csr_name_map = 0xB82 <-> "minstreth"
37
38
39 function clause is_CSR_accessible(0xB00, _, _) = currentlyEn-
abled(Ext_Zicntr) // mcycle
40 function clause is_CSR_accessible(0xB02, _, _) = currentlyEn-
abled(Ext_Zicntr) // minstret
41 function clause is_CSR_accessible(0xB80, _, _) = currentlyEn-
abled(Ext_Zicntr) & xlen == 32 // mcycleh
42 function clause is_CSR_accessible(0xB82, _, _) = currentlyEn-
abled(Ext_Zicntr) & xlen == 32 // minstreth
43
44 function clause read_CSR(0xB00) = mcycle[(xlen - 1) .. 0]
45 function clause read_CSR(0xB02) = minstret[(xlen - 1) .. 0]
46 function clause read_CSR(0xB80 if xlen == 32) = mcycle[63 ..
32]
47
47 function clause read_CSR(0xB82 if xlen == 32) = minstret[63 ..
32]
48
49 function clause write_CSR(0xB00, value) = { mcycle[(xlen -
1) .. 0] = value; Ok(value) }

```

```

50 function clause write_CSR(0xB02, value) = { minstret[(xlen -
1) .. 0] = value; minstret_increment = false; Ok(value) }
51 function clause write_CSR(0xB80, value) if xlen == 32) =
{ mcycle[63 .. 32] = value; Ok(value) }
52 function clause write_CSR(0xB82, value) if xlen == 32) =
{ minstret[63 .. 32] = value; minstret_increment = false;
Ok(value) }

```

3. riscv_sail-riscv/model/extensions/II/base_insts.sail (741 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8
9 //
*****
10 // This file specifies the instructions in the base integer
set.
11
12 function clause currentlyEnabled(Ext_C) = hartSupports(Ext_C)
& misa[C] == 0b1
13 function clause currentlyEnabled(Ext_Zca) = hartSup-
ports(Ext_Zca) & (currentlyEnabled(Ext_C) | not(hartSup-
ports(Ext_C)))
14
15 //
*****
16 union clause instruction = UTYPE : (bits(20), regidx, uop)
17
18 mapping encdec_uop : uop <-> bits(7) = {
19   LUI <-> 0b0110111,
20   AUIPC <-> 0b0010111
21 }
22
23 mapping clause encdec = UTYPE(imm, rd, op)
24 <-> imm @ encdec_reg(rd) @ encdec_uop(op)
25
26 $[split op]
27 function clause execute UTYPE(imm, rd, op) = {
28   let off : xlenbits = sign_extend(imm @ 0x000);
29   X(rd) = match op {
30     LUI => off,
31     AUIPC => get_arch_pc() + off
32   };
33   RETIRE_SUCCESS
34 }
35
36 mapping utype_mnemonic : uop <-> string = {
37   LUI <-> "lui",
38   AUIPC <-> "auipc"
39 }
40
41 mapping clause assembly = UTYPE(imm, rd, op)
42 <-> utype_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
hex_bits_signed_20(imm)
43
44 //
*****
45
46 // Jump execution to a specified target address. This can fail
47 // due to the target not being 4-byte aligned, or due to ex-
tension checks.
48 // Callers must ensure that the target address is 2-byte
aligned.
49 function jump_to(target : xlenbits) -> ExecutionResult = {
50 // Extensions get the first checks on the prospective target
address.
51 match ext_control_check_pc(target) {
52   Some(e) => return Ext_ControlAddr_Check_Failure(e),
53   None() => (),
54 };
55
56 // Perform standard alignment check.
57 // Check target is at least 2-byte aligned (callers must en-
sure
58 // this so it can be an assertion).
59 assert(target[0] == 0b0);
60 // If it is not 4-byte aligned and compressed instructions
are
61 // not enabled then raise an alignment exception.
62 if bit_to_bool(target[1]) & not(currentlyEnabled(Ext_Zca))
63 then return trap(exception_context(E_Fetch_Addr_Align(),
target));
64
65 set_next_pc(target);
66 RETIRE_SUCCESS
67 }
68
69 //

```

```

*****
70 union clause instruction = JAL : (bits(21), regidx)
71 $[wavedrom "_ offset[20:1] _ _ dest JAL"]
72 mapping clause encdec = JAL(imm @ 0b0, rd)
73 <-> imm[19] @ imm[9..0] @ imm[10] @ imm[18..11] @ enc-
dec_reg(rd) @ 0b1101111
74
75 function clause execute JAL(imm, rd) = {
76   let link_address = get_next_pc();
77   match jump_to(PC + sign_extend(imm)) {
78     Retire_Success() => { X(rd) = link_address; Re-
tire_Success() },
79     failure => failure,
80   }
81 }
82
83 // TODO: handle 2-byte-alignment in mappings
84
85 mapping clause assembly = JAL(imm, rd)
86 <-> "jal" ^ spc() ^ reg_name(rd) ^ sep() ^
hex_bits_signed_21(imm)
87
88 //
*****
89 union clause instruction = JALR : (bits(12), regidx, regidx)
90
91 $[wavedrom "offset[11:0] base 0 dest JALR"]
92 mapping clause encdec = JALR(imm, rs1, rd)
93 <-> imm @ encdec_reg(rs1) @ 0b000 @ encdec_reg(rd) @
0b1100111
94
95 mapping clause assembly = JALR(imm, rs1, rd)
96 <-> "jalr" ^ spc() ^ reg_name(rd) ^ sep() ^
hex_bits_signed_12(imm) ^ "(" ^ reg_name(rs1) ^ ")"
97
98 // see jalr_seq.sail or jalr_rmem.sail for the execute clause.
99
100 //
*****
101 union clause instruction = BTYPE : (bits(13), regidx, regidx,
bop)
102
103 mapping encdec_bop : bop <-> bits(3) = {
104   BEQ <-> 0b000,
105   BNE <-> 0b001,
106   BLT <-> 0b100,
107   BGE <-> 0b101,
108   BLTU <-> 0b110,
109   BGEU <-> 0b111
110 }
111
112 mapping clause encdec = BTYPE(imm @ 0b0, rs2, rs1, op)
113 <-> imm[11] @ imm[9..4] @ encdec_reg(rs2) @ encdec_reg(rs1)
@ encdec_bop(op) @ imm[3..0] @ imm[10] @ 0b1100011
114
115 $[split op]
116 function clause execute BTYPE(imm, rs2, rs1, op) = {
117   let taken : bool = match op {
118     BEQ => X(rs1) == X(rs2),
119     BNE => X(rs1) != X(rs2),
120     BLT => X(rs1) <_s X(rs2),
121     BGE => X(rs1) >=_s X(rs2),
122     BLTU => X(rs1) <_u X(rs2),
123     BGEU => X(rs1) >=_u X(rs2)
124   };
125   if taken
126   then jump_to(PC + sign_extend(imm))
127   else RETIRE_SUCCESS
128 }
129
130 mapping btype_mnemonic : bop <-> string = {
131   BEQ <-> "beq",
132   BNE <-> "bne",
133   BLT <-> "blt",
134   BGE <-> "bge",
135   BLTU <-> "bltu",
136   BGEU <-> "bgeu"
137 }
138
139 mapping clause assembly = BTYPE(imm, rs2, rs1, op)
140 <-> btype_mnemonic(op) ^ spc() ^ reg_name(rs1) ^ sep() ^
reg_name(rs2) ^ sep() ^ hex_bits_signed_13(imm)
141
142 //
*****
143 union clause instruction = ITYPE : (bits(12), regidx, regidx,
iop)
144
145 mapping encdec_iop : iop <-> bits(3) = {
146   ADDI <-> 0b000,
147   SLTI <-> 0b010,
148   SLTIU <-> 0b011,
149   ANDI <-> 0b111,
150   ORI <-> 0b110,
151   XORI <-> 0b100
152 }
153
154 mapping clause encdec = ITYPE(imm, rs1, rd, op)
155 <-> imm @ encdec_reg(rs1) @ encdec_iop(op) @ encdec_reg(rd)
@ 0b0010011
156

```

```

157 $[split op]
158 function clause execute ITYPE(imm, rs1, rd, op) = {
159   let immext : xlenbits = sign_extend(imm);
160   X(rd) = match op {
161     ADDI => X(rs1) + immext,
162     SLTI => zero_extend(bool_to_bit(X(rs1) <_s immext)),
163     SLTIU => zero_extend(bool_to_bit(X(rs1) <_u immext)),
164     ANDI => X(rs1) & immext,
165     ORI => X(rs1) | immext,
166     XORI => X(rs1) ^ immext
167   };
168   RETIRE_SUCCESS
169 }
170
171 mapping itype_mnemonic : iop <-> string = {
172   ADDI <-> "addi",
173   SLTI <-> "slti",
174   SLTIU <-> "sltiu",
175   XORI <-> "xori",
176   ORI <-> "ori",
177   ANDI <-> "andi"
178 }
179
180 mapping clause assembly = ITYPE(imm, rs1, rd, op)
181 <-> itype_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ hex_bits_signed_12(imm)
182
183 //
184 *****
184 union clause instruction = SHIFTIOP : (bits(6), regidx, re-
gidx, sop)
185
186 mapping encdec_sop : sop <-> bits(3) = {
187   SLLI <-> 0b001,
188   SRLI <-> 0b101,
189   SRAI <-> 0b101
190 }
191
192 mapping clause encdec = SHIFTIOP(shamt, rs1, rd, SLLI)
193 <-> 0b000000 @ shamt @ encdec_reg(rs1) @ 0b001 @ enc-
dec_reg(rd) @ 0b0010011
194 when xlen == 64 | shamt[5] == 0b0
195 mapping clause encdec = SHIFTIOP(shamt, rs1, rd, SRLI)
196 <-> 0b000000 @ shamt @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0010011
197 when xlen == 64 | shamt[5] == 0b0
198 mapping clause encdec = SHIFTIOP(shamt, rs1, rd, SRAI)
199 <-> 0b010000 @ shamt @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0010011
200 when xlen == 64 | shamt[5] == 0b0
201
202 $[split op]
203 function clause execute SHIFTIOP(shamt, rs1, rd, op) = {
204   let shamt = shamt[log2_xlen - 1 .. 0];
205   X(rd) = match op {
206     SLLI => X(rs1) << shamt,
207     SRLI => X(rs1) >> shamt,
208     SRAI => shift_bits_right_arith(X(rs1), shamt),
209   };
210   RETIRE_SUCCESS
211 }
212
213 mapping shiftpop_mnemonic : sop <-> string = {
214   SLLI <-> "slli",
215   SRLI <-> "srl",
216   SRAI <-> "srai"
217 }
218
219 mapping clause assembly = SHIFTIOP(shamt, rs1, rd, op)
220 <-> shiftpop_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ hex_bits_6(shamt)
221
222 //
223 *****
223 union clause instruction = RTYPE : (regidx, regidx, regidx,
rop)
224
225 mapping clause encdec = RTYPE(rs2, rs1, rd, ADD) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @ enc-
dec_reg(rd) @ 0b0110011
226 mapping clause encdec = RTYPE(rs2, rs1, rd, SLT) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b010 @ enc-
dec_reg(rd) @ 0b0110011
227 mapping clause encdec = RTYPE(rs2, rs1, rd, SLTU) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b011 @ enc-
dec_reg(rd) @ 0b0110011
228 mapping clause encdec = RTYPE(rs2, rs1, rd, AND) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b111 @ enc-
dec_reg(rd) @ 0b0110011
229 mapping clause encdec = RTYPE(rs2, rs1, rd, OR) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b110 @ enc-
dec_reg(rd) @ 0b0110011
230 mapping clause encdec = RTYPE(rs2, rs1, rd, XOR) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b100 @ enc-
dec_reg(rd) @ 0b0110011
231 mapping clause encdec = RTYPE(rs2, rs1, rd, SLL) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b001 @ enc-
dec_reg(rd) @ 0b0110011
232 mapping clause encdec = RTYPE(rs2, rs1, rd, SRL) <->
0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0110011
233 mapping clause encdec = RTYPE(rs2, rs1, rd, SUB) <->
0b01000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @ enc-
dec_reg(rd) @ 0b0110011
234 mapping clause encdec = RTYPE(rs2, rs1, rd, SRA) <->
0b01000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0110011
235
236 $[split op]
237 function clause execute RTYPE(rs2, rs1, rd, op) = {
238   X(rd) = match op {
239     ADD => X(rs1) + X(rs2),
240     SLT => zero_extend(bool_to_bit(X(rs1) <_s X(rs2))),
241     SLTU => zero_extend(bool_to_bit(X(rs1) <_u X(rs2))),
242     AND => X(rs1) & X(rs2),
243     OR => X(rs1) | X(rs2),
244     XOR => X(rs1) ^ X(rs2),
245     SLL => X(rs1) << X(rs2)[log2_xlen - 1 .. 0],
246     SRL => X(rs1) >> X(rs2)[log2_xlen - 1 .. 0],
247     SUB => X(rs1) - X(rs2),
248     SRA => shift_bits_right_arith(X(rs1), X(rs2)[log2_xlen -
1 .. 0]),
249   };
250   RETIRE_SUCCESS
251 }
252
253 mapping rtype_mnemonic : rop <-> string = {
254   ADD <-> "add",
255   SLT <-> "slt",
256   SLTU <-> "sltu",
257   AND <-> "and",
258   OR <-> "or",
259   XOR <-> "xor",
260   SLL <-> "sll",
261   SRL <-> "srl",
262   SUB <-> "sub",
263   SRA <-> "sra"
264 }
265
266 mapping clause assembly = RTYPE(rs2, rs1, rd, op)
267 <-> rtype_mnemonic(rop) ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
268
269 //
270 *****
270 union clause instruction = LOAD : (bits(12), regidx, regidx,
bool, word_width)
271
272 // unsigned loads are only present for widths strictly less
than xlen,
273 // signed loads also present for widths equal to xlen
274 function valid_load_encdec(width : word_width, is_unsigned :
bool) -> bool =
275   (width < xlen_bytes) | (not(is_unsigned) & width <=
xlen_bytes)
276
277 val extend_value : forall 'n, 0 < 'n <= xlen. (bool, bits('n)) -
> xlenbits
278 function extend_value(is_unsigned, value) = if is_unsigned
then zero_extend(value) else sign_extend(value)
279
280 $[wavedrom "offset[11:0] base _ width dest LOAD"]
281 mapping clause encdec = LOAD(imm, rs1, rd, is_unsigned, width)
282 <-> imm @ encdec_reg(rs1) @ bool_bit(is_unsigned) @
width_enc(width) @ encdec_reg(rd) @ 0b0000011
283 when valid_load_encdec(width, is_unsigned)
284
285 function clause execute LOAD(imm, rs1, rd, is_unsigned, width) =
{
286   let offset : xlenbits = sign_extend(imm);
287
288   // This is checked during decoding.
289   assert(width <= xlen_bytes);
290
291   match vmem_read(rs1, offset, width, Load(Data), false,
false, false) {
292     Ok(data) => {
293       X(rd) = extend_value(is_unsigned, data);
294       RETIRE_SUCCESS
295     },
296     Err(e) => e,
297   }
298 }
299
300 mapping maybe_u : bool <-> string = {
301   true <-> "u",
302   false <-> ""
303 }
304
305 mapping clause assembly = LOAD(imm, rs1, rd, is_unsigned,
width)
306 <-> "l" ^ width_mnemonic(width) ^ maybe_u(is_unsigned) ^
spc() ^ reg_name(rd) ^ sep() ^ hex_bits_signed_12(imm) ^ "(" ^
reg_name(rs1) ^ ")"
307
308 //
309 *****
309 union clause instruction = STORE : (bits(12), regidx, regidx,
word_width)
310

```

```

311 $[wavedrom "offset[11:5] src base _ width offset[4:0] STORE"]
312 mapping clause encdec = STORE(imm, rs2, rs1, width)
313 <-> imm[11..5] @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b0 @
width_enc(width) @ imm[4..0] @ 0b0100011
314 when width <= xlen_bytes
315
316 function clause execute STORE(imm, rs2, rs1, width) = {
317   let offset : xlenbits = sign_extend(imm);
318
319   // This is checked during decoding.
320   assert(width <= xlen_bytes);
321
322   // FIXME: the read of `rs2` should not be done before the
323   // `mem_write_ea` of the effective address.
324   // See https://github.com/riscv/sail-riscv/issues/1761.
325
326   let data = X(rs2)[width * 8 - 1 .. 0];
327   match vmem_write(rs1, offset, width, data, Store(Data),
false, false, false) {
328     Ok(_) => RETIRE_SUCCESS,
329     Err(e) => e,
330   }
331 }
332
333 mapping clause assembly = STORE(imm, rs2, rs1, width)
334 <-> "s" ^ width_mnemonic(width) ^ spc() ^ reg_name(rs2) ^
sep() ^ hex_bits_signed_12(imm) ^ opt_spc() ^ "(" ^ opt_spc() ^
reg_name(rs1) ^ opt_spc() ^ ")"
335
336 //
*****
337 union clause instruction = ADDIW : (bits(12), regidx, regidx)
338
339 $[wavedrom "I-immediate[11:0] src ADDIW dest OP-IMM-32"]
340 mapping clause encdec = ADDIW(imm, rs1, rd)
341 <-> imm @ encdec_reg(rs1) @ 0b000 @ encdec_reg(rd) @
0b0011011
342 when xlen == 64
343
344 function clause execute ADDIW(imm, rs1, rd) = {
345   let result = X(rs1) + sign_extend(imm);
346   X(rd) = sign_extend(result[31..0]);
347   RETIRE_SUCCESS
348 }
349
350 mapping clause assembly = ADDIW(imm, rs1, rd)
351
352 <-> "addiw" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1) ^
sep() ^ hex_bits_signed_12(imm)
353 when xlen == 64
354 //
*****
355 union clause instruction = RTYPEW : (regidx, regidx, regidx,
ropw)
356
357 mapping clause encdec = RTYPEW(rs2, rs1, rd, ADDW)
358 <-> 0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @
encdec_reg(rd) @ 0b0111011
359 when xlen == 64
360 mapping clause encdec = RTYPEW(rs2, rs1, rd, SUBW)
361 <-> 0b01000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @
encdec_reg(rd) @ 0b0111011
362 when xlen == 64
363 mapping clause encdec = RTYPEW(rs2, rs1, rd, SLLW)
364 <-> 0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b001 @
encdec_reg(rd) @ 0b0111011
365 when xlen == 64
366 mapping clause encdec = RTYPEW(rs2, rs1, rd, SRLW)
367 <-> 0b00000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b101 @
encdec_reg(rd) @ 0b0111011
368 when xlen == 64
369 mapping clause encdec = RTYPEW(rs2, rs1, rd, SRAW)
370 <-> 0b01000000 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b101 @
encdec_reg(rd) @ 0b0111011
371 when xlen == 64
372
373 $[split op]
374 function clause execute RTYPEW(rs2, rs1, rd, op) = {
375   let rs1_val = X(rs1)[31..0];
376   let rs2_val = X(rs2)[31..0];
377   let result : bits(32) = match op {
378     ADDW => rs1_val + rs2_val,
379     SUBW => rs1_val - rs2_val,
380     SLLW => rs1_val << rs2_val[4..0],
381     SRLW => rs1_val >> rs2_val[4..0],
382     SRAW => shift_bits_right_arith(rs1_val, rs2_val[4..0]),
383   };
384   X(rd) = sign_extend(result);
385   RETIRE_SUCCESS
386 }
387
388 mapping rtypew_mnemonic : ropw <-> string = {
389   ADDW <-> "addw",
390   SUBW <-> "subw",
391   SLLW <-> "sllw",
392   SRLW <-> "srlw",
393   SRAW <-> "sraw"
394 }
395
396 mapping clause assembly = RTYPEW(rs2, rs1, rd, op)

```

```

397 <-> rtypew_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
398 when xlen == 64
399
400 //
*****
401 union clause instruction = SHIFTIWOP : (bits(5), regidx, re-
gidx, sopw)
402
403 mapping clause encdec = SHIFTIWOP(shamt, rs1, rd, SLLIW)
404 <-> 0b00000000 @ shamt @ encdec_reg(rs1) @ 0b001 @ enc-
dec_reg(rd) @ 0b0011011
405 when xlen == 64
406 mapping clause encdec = SHIFTIWOP(shamt, rs1, rd, SRLIW)
407 <-> 0b00000000 @ shamt @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0011011
408 when xlen == 64
409 mapping clause encdec = SHIFTIWOP(shamt, rs1, rd, SRAIW)
410 <-> 0b01000000 @ shamt @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0011011
411 when xlen == 64
412
413 $[split op]
414 function clause execute SHIFTIWOP(shamt, rs1, rd, op) = {
415   let rs1_val = X(rs1)[31..0];
416   let result : bits(32) = match op {
417     SLLIW => rs1_val << shamt,
418     SRLIW => rs1_val >> shamt,
419     SRAIW => shift_bits_right_arith(rs1_val, shamt),
420   };
421   X(rd) = sign_extend(result);
422   RETIRE_SUCCESS
423 }
424
425 mapping shiftiwop_mnemonic : sopw <-> string = {
426   SLLIW <-> "slliw",
427   SRLIW <-> "srliw",
428   SRAIW <-> "sraiw"
429 }
430
431 mapping clause assembly = SHIFTIWOP(shamt, rs1, rd, op)
432 <-> shiftiwop_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ hex_bits_5(shamt)
433 when xlen == 64
434
435 //
*****
436 union clause instruction = FENCE_TSO : unit
437
438 mapping clause encdec = FENCE_TSO()
439 <-> 0b1000 @ 0b0011 @ 0b0011 @ 0b000000 @ 0b000 @ 0b000000 @
0b000111
440
441 function clause execute FENCE_TSO() = {
442   sail_barrier(Barrier_RISCV_tso);
443   RETIRE_SUCCESS
444 }
445
446 mapping clause assembly = FENCE_TSO()
447 <-> "fence.tso"
448
449 //
*****
450 union clause instruction = FENCE : (bits(4), bits(4), bits(4),
regidx, regidx)
451
452 // Note: FENCE_TSO and PAUSE are encoded as FENCE so they must
come first.
453 mapping clause encdec = FENCE(fm, pred, succ, rs, rd)
454 <-> fm : bits(4) @ pred : bits(4) @ succ : bits(4) @ enc-
dec_reg(rs) @ 0b000 @ encdec_reg(rd) @ 0b0001111
455
456 function effective_fence_set(set : bits(4), fiom : bool) ->
bits(4) = {
457   // The bits are IORW. If FIOM is set then I implies R and O
implies W.
458   if fiom then {
459     set[3 .. 2] @ (set[1 .. 0] | set[3 .. 2])
460   } else set
461 }
462
463 function clause execute FENCE(_fm, pred, succ, _rs, _rd) = {
464   // "The unused fields in the FENCE instructions - rs1 and rd -
are reserved for
465   // finer-grain fences in future extensions. For forward com-
patibility, base
466   // implementations shall ignore these fields. Likewise, many
fm and
467   // predecessor/successor set settings are also reserved for
future use.
468   // Base implementations shall treat all such reserved con-
figurations as FENCE
469   // instructions (with fm=0000)."
470
471   // If the FIOM bit in menvcfg/senvcfg is set then the I/O
bits can imply R/W.
472   let fiom = is_fiom_active();

```



```

473 let pred = effective_fence_set(pred, fiom);
474 let succ = effective_fence_set(succ, fiom);
475
476 // Ignore I/O bits.
477 match (pred[1 .. 0], succ[1 .. 0]) {
478   (0b11, 0b11) => sail_barrier(Barrier_RISCV_rw_rw),
479   (0b10, 0b11) => sail_barrier(Barrier_RISCV_r_rw),
480   (0b10, 0b10) => sail_barrier(Barrier_RISCV_r_r),
481   (0b11, 0b01) => sail_barrier(Barrier_RISCV_rw_w),
482   (0b01, 0b01) => sail_barrier(Barrier_RISCV_w_w),
483   (0b01, 0b11) => sail_barrier(Barrier_RISCV_w_rw),
484   (0b11, 0b10) => sail_barrier(Barrier_RISCV_rw_r),
485   (0b10, 0b01) => sail_barrier(Barrier_RISCV_r_w),
486   (0b01, 0b10) => sail_barrier(Barrier_RISCV_w_r),
487
488   (_, 0b00) => (),
489   (0b00, _) => (),
490 };
491 RETIRE_SUCCESS
492 }
493
494 mapping bit_maybe_r : bits(1) <-> string = {
495   0b1 <-> "r",
496   0b0 <-> ""
497 }
498
499 mapping bit_maybe_w : bits(1) <-> string = {
500   0b1 <-> "w",
501   0b0 <-> ""
502 }
503
504 mapping bit_maybe_i : bits(1) <-> string = {
505   0b1 <-> "i",
506   0b0 <-> ""
507 }
508
509 mapping bit_maybe_o : bits(1) <-> string = {
510   0b1 <-> "o",
511   0b0 <-> ""
512 }
513
514 mapping fence_bits : bits(4) <-> string = {
515   // Fences with empty predecessor or successor sets are nops.
516   // Clang/LLVM's assembly supports using `0` for these.
517
518   // Binutils/GAS does not so you would have to use `.insn` to
519   // encode these.
520   0b0000 <-> "0",
521   i : bits(1) @ o : bits(1) @ r : bits(1) @ w : bits(1) <->
522   bit_maybe_i(i) ^ bit_maybe_o(o) ^ bit_maybe_r(r) ^ bit_maybe_w(w),
523 }
524
525 // There's no assembly that encodes reserved fences as far as
526 // I know.
527 // Unfortunately we need to split forwards and backwards be-
528 // cause there's
529 // no way to write a zero regidx as a literal.
530 mapping clause assembly =
531   forwards FENCE(0b0000, pred, succ, rs, rd) => "fence" ^
532   spc() ^ fence_bits(pred) ^ sep() ^ fence_bits(succ)
533   when rs == zreg & rd == zreg
534
535 // TODO: This hits a Sail compiler bug, https://github.com/
536 // rems-project/sail/issues/1577
537 // however we don't currently use the assembler direction of
538 // this mapping so
539 // it doesn't really matter.
540 // mapping clause assembly =
541 //   backwards "fence" ^ spc() ^ fence_bits(pred) ^ sep() ^
542 //   fence_bits(succ) => FENCE(0b0000, pred, succ, zreg, zreg)
543
544 //
545 *****
546 union clause instruction = ECALL : unit
547
548 mapping clause encdec = ECALL()
549 <-> 0b000000000000 @ 0b000000 @ 0b0000 @ 0b000000 @ 0b1110011
550
551 function clause execute ECALL() = {
552   let exc_type : ExceptionType = match cur_privilege {
553     User => E_U_EnvCall(),
554     Supervisor => E_S_EnvCall(),
555     Machine => E_M_EnvCall(),
556     VirtualUser => E_U_EnvCall(),
557     VirtualSupervisor => E_VS_EnvCall(),
558   };
559   let c : ExceptionContext = struct {
560     trap = exc_type,
561     excinfo_is_gva = false,
562     excinfo = zeros(),
563     excinfo2 = None(),
564     excinst = None(),
565     ext = None(),
566   };
567   trap(c)
568 }
569
570 mapping clause assembly = ECALL() <-> "ecall"
571
572 //
573 *****

```

```

563 union clause instruction = MRET : unit
564
565 mapping clause encdec = MRET()
566 <-> 0b0011000 @ 0b00010 @ 0b00000 @ 0b000 @ 0b00000 @
567 0b1110011
568
569 function clause execute MRET() = {
570   if cur_privilege != Machine
571   then Illegal_Instruction()
572   else if not(ext_check_xret_priv(Machine))
573   then Ext_XRET_Priv_Failure()
574   else {
575     let prev_priv = cur_privilege;
576     mstatus[MIE] = mstatus[MPIE];
577     mstatus[MPIE] = 0b1;
578     // MPV is ignored when MPP=3, as there is no virtual ma-
579     // chine mode
580     cur_privilege = privLevel_bits((mstatus[MPP], if
581     mstatus[MPP] == 0b11 then 0b0 else mstatus[MPV]));
582     mstatus[MPP] = privLevel_to_bits(if currentlyEn-
583     abled(Ext_U) then User else Machine);
584     mstatus[MPV] = 0b0;
585     if cur_privilege != Machine
586     then mstatus[MPRV] = 0b0;
587     if hartSupports(Ext_Zicfilp)
588     then zicfilp_restore_elp_on_xret(mRET, prev_priv,
589     cur_privilege);
590     long_csr_write_callback("mstatus", "mstatush",
591     mstatus.bits);
592     if get_config_print_exception()
593     then print_log("ret-ing from " ^ to_str(prev_priv) ^ " to
594     " ^ to_str(cur_privilege));
595     set_next_pc(prepare_xret_target(Machine));
596     xret_callback(true);
597     RETIRE_SUCCESS
598   }
599 }
600
601 mapping clause assembly = MRET() <-> "mret"
602
603 //
604 *****
605 union clause instruction = SRET : unit
606
607 mapping clause encdec = SRET()
608 <-> 0b0001000 @ 0b00010 @ 0b00000 @ 0b000 @ 0b00000 @
609 0b1110011
610
611 function clause execute SRET() = {
612   let sret_illegal : bool = match cur_privilege {
613     User => true,
614     Supervisor => not(currentlyEnabled(Ext_S)) |
615     mstatus[TSR] == 0b1,
616     Machine => not(currentlyEnabled(Ext_S)),
617     VirtualUser => false,
618     VirtualSupervisor => false,
619   };
620   let sret_virtual : bool = match cur_privilege {
621     VirtualUser => true,
622     VirtualSupervisor => hstatus[VTSR] == 0b1,
623     _ => false,
624   };
625   if sret_illegal
626   then Illegal_Instruction()
627   else if sret_virtual
628   then Virtual_Instruction()
629   else if not(ext_check_xret_priv(Supervisor))
630   then Ext_XRET_Priv_Failure()
631   else {
632     let prev_priv = cur_privilege;
633     let ret =
634       // SRET in VS-Mode
635       if inVirtualPrivilege() then {
636         let new_priv = if vsstatus[SPP] == 0b1 then VirtualSu-
637         pervisor else VirtualUser;
638         vsstatus[SPP] = 0b0;
639         vsstatus[SIE] = vsstatus[SPIE];
640         vsstatus[SPIE] = 0b1;
641         cur_privilege = new_priv;
642         VirtualSupervisor
643       } else {
644         // SRET in HS-mode or M-mode
645
646         // SPV is only meaningful when H is enabled, otherwise
647         // treat as 0
648         let new_priv = privLevel_bits((0b0 @ mstatus[SPP],
649         hstatus[SPV] & bool_to_bit(currentlyEnabled(Ext_H))));
650         hstatus[SPV] = 0b0;
651         mstatus[SPP] = 0b0;
652         mstatus[SIE] = mstatus[SPIE];
653         mstatus[SPIE] = 0b1;
654         // TODO: Add Ssdbltrp part once supported
655         if new_priv != Machine
656         then mstatus[MPRV] = 0b0;

```



```

650     cur_privilege = new_priv;
651     Supervisor
652 };
653
654 if hartSupports(Ext_Zicfilp)
655 then zicfilp_restore_elp_on_xret(sRET, prev_priv,
cur_privilege);
656
657 long_csr_write_callback("mstatus", "mstatush",
mstatus.bits);
658 if privLevel_is_virtual(prev_priv)
659 then csr_write_callback("vsstatus", vsstatus.bits[xlen -
1 .. 0]);
660 else if currentlyEnabled(Ext_H)
661 then csr_write_callback("hstatus", hstatus.bits[xlen -
1 .. 0]);
662
663 if get_config_print_exception()
664 then print_log("ret-ing from " ^ to_str(prev_priv) ^ " to
" ^ to_str(cur_privilege));
665
666 set_next_pc(prepare_xret_target(ret));
667 xret_callback(false);
668 RETIRE_SUCCESS
669 }
670 }
671
672 mapping clause assembly = SRET() <-> "sret"
673
674 //
*****
675 union clause instruction = EBREAK : unit
676
677 mapping clause encdec = EBREAK()
678 <-> 0b000000000001 @ 0b000000 @ 0b0000 @ 0b000000 @ 0b1110011
679
680 function clause execute EBREAK() =
681 trap(exception_context(E_Breakpoint(Brk_Software), PC))
682
683 mapping clause assembly = EBREAK() <-> "ebreak"
684
685 //
*****
686 union clause instruction = WFI : unit
687
688 mapping clause encdec = WFI()
689 <-> 0b000100000101 @ 0b000000 @ 0b0000 @ 0b000000 @ 0b1110011
690
691 function clause execute WFI() =
692 match cur_privilege {
693     Machine => Enter_Wait(WAIT_WFI),
694     // mstatus[TW] is checked when the wait times out in
695     // `run_hart_waiting()`.
696     Supervisor => Enter_Wait(WAIT_WFI),
697     User => if plat_wfi_available_to_usermode
then Enter_Wait(WAIT_WFI) else Illegal_Instruction(),
698     // VU never waits. Raises virtual immediately when TW=0,
699     // illegal when TW=1
700     VirtualUser => if mstatus[TW] == 0b1
then Illegal_Instruction()
else Virtual_Instruction(),
701     // VS raises illegal immediately when TW=1
702     // when TW=0, enters wait, VTW checked on timeout in
703     run_hart_waiting()
704     VirtualSupervisor => if mstatus[TW] == 0b1
then Illegal_Instruction()
else Enter_Wait(WAIT_WFI),
705 }
706
707 mapping clause assembly = WFI() <-> "wfi"
708
709 //
*****
710 union clause instruction = SFENCE_VMA : (regidx, regidx)
711
712 mapping clause encdec = SFENCE_VMA(rs1, rs2)
713 <-> 0b0001001 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @
0b00000 @ 0b1110011
714
715 when virtual_memory_supported() | not(config exten-
sions.Svbare.sfence_vma_illegal_if_svbare_only : bool)
716
717 function clause execute SFENCE_VMA(rs1, rs2) = {
718 let addr = if rs1 != zreg then Some(X(rs1)) else None();
719 // Note, the Sail model does not currently support Sv32 &
SXLEN=32 on RV64.
720 // In that case this `asidlen` may be incorrect because
ASIDMAX for Sv32 is 9,
721 // but `asidlen` would have been validated against the ASID-
MAX for RV64.
722 let asid : option(asidbits) = if rs2 != zreg then Some(if
asidlen == 0 then zeros() else zero_extend(X(rs2)[asidlen - 1 ..
0])) else None();
723 let vmid = if cur_privilege == VirtualSupervisor then
Some(get_vmid(VS_Stage)) else None();
724 match cur_privilege {
725     User => Illegal_Instruction(),
726     Supervisor => match mstatus[TVM] {
727         0b1 => Illegal_Instruction(),
728         0b0 => { flush_TLB(asid, vmid,
addr, S_Stage); RETIRE_SUCCESS },
729     },
730 }

```

```

731 Machine => { flush_TLB(asid, vmid, addr,
S_Stage); RETIRE_SUCCESS },
732 VirtualUser => Virtual_Instruction(),
733 VirtualSupervisor => match hstatus[VTVM] {
734     0b1 => Virtual_Instruction(),
735     0b0 => { flush_TLB(asid, vmid,
addr, VS_Stage); RETIRE_SUCCESS },
736 },
737 }
738 }
739
740 mapping clause assembly = SFENCE_VMA(rs1, rs2)
741 <-> "sfence.vma" ^ spc() ^ reg_name(rs1) ^ sep() ^
reg_name(rs2)

```

4. riscv_sail-riscv/model/extensions/l/base_types.sail (17 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8
9 enum uop = {LUI, AUIPC} // upper im-
mediate ops
10 enum bop = {BEQ, BNE, BLT, BGE, BLTU, BGEU} // branch ops
11 enum iop = {ADDI, SLTI, SLTIU, XORI, ORI, ANDI} // immediate
ops
12 enum sop = {SLLI, SRLI, SRAI} // shift ops
13 enum rop = {ADD, SUB, SLL, SLT, SLTU, XOR,
14             SRL, SRA, OR, AND} // reg-reg
ops
15
16 enum ropw = {ADDW, SUBW, SLLW, SRLW, SRAW} // reg-reg
32-bit ops
17 enum sopw = {SLLIW, SRLIW, SRAIW} // RV64-only
shift ops

```

5. riscv_sail-riscv/model/extensions/l/jalr_rmem.sail (18 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8
9 // The definition for the memory model.
10
11 function clause execute JALR(imm, rs1, rd) = {
12     // FIXME: this does not check for a misaligned target ad-
dress. See jalr_seq.sail.
13     // write rd before anything else to prevent unintended
strength
14     X(rd) = nextPC; // compatible with JALR, C.JR and C.JALR
15     let newPC : xlenbits = X(rs1) + sign_extend(imm);
16     nextPC = [newPC with 0 = 0b0]; // Clear newPC[0]
17     RETIRE_SUCCESS
18 }

```

6. riscv_sail-riscv/model/extensions/l/jalr_seq.sail (26 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8
9 // The definition for the sequential model.
10
11 function clause execute JALR(imm, rs1, rd) = {
12     // For the sequential model, the memory-model definition
doesn't work directly
13     // if rs1 = rd. We would effectively have to keep a regfile
for reads and another for
14     // writes, and swap on instruction completion. This could
perhaps be optimized in
15     // some manner, but for now, we just keep a reordered defin-
ition to improve simulator
16     // performance.

```

```

17
18   update_elp_state(rs1);
19
20   let link_address = get_next_pc();
21   let target = X(rs1) + sign_extend(imm);
22   match jump_to([target with 0 = 0b0]) {
23     Retire_Success() => { X(rd) = link_address; Re-
tire_Success() },
24     failure => failure,
25   }
26 }

```

7. riscv_sail-riscv/model/extensions/K/types_kext.sail (323 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8
9 //
10 // This file contains types, mappings and functions used
across the
11 // cryptography extension instructions.
12 //
13 // This file must be included in the model build whatever the
value of XLEN.
14
15 //
16 // Cryptography extension shared / utility functions
17 //
-----
18
19 // Auxiliary function for performing GF multiplication.
20 function xt2(x : bits(8)) -> bits(8) = {
21   (x << 1) ^ (if bit_to_bool(x[7]) then 0x1b else 0x00)
22 }
23
24 function xt3(x : bits(8)) -> bits(8) = x ^ xt2(x)
25
26 // Multiply 8-bit field element by 4-bit value for AES Mix-
Cols step.
27 function gfmul(x : bits(8), y : bits(4)) -> bits(8) = {
28   (if bit_to_bool(y[0]) then x else 0x00) ^
29   (if bit_to_bool(y[1]) then xt2(x) else 0x00) ^
30   (if bit_to_bool(y[2]) then xt2(xt2(x)) else 0x00) ^
31   (if bit_to_bool(y[3]) then xt2(xt2(xt2(x))) else 0x00)
32 }
33
34 // 8-bit to 32-bit partial AES Mix Column - forwards.
35 function aes_mixcolumn_byte_fwd(so : bits(8)) -> bits(32) = {
36   gfmul(so, 0x3) @ so @ so @ gfmul(so, 0x2)
37 }
38
39 // 8-bit to 32-bit partial AES Mix Column - inverse.
40 function aes_mixcolumn_byte_inv(so : bits(8)) -> bits(32) = {
41   gfmul(so, 0xb) @ gfmul(so, 0xd) @ gfmul(so, 0x9) @ gfmul(so,
0xe)
42 }
43
44 // 32-bit to 32-bit AES forward MixColumn.
45 function aes_mixcolumn_fwd(x : bits(32)) -> bits(32) = {
46   let s0 : bits(8) = x[7..0];
47   let s1 : bits(8) = x[15..8];
48   let s2 : bits(8) = x[23..16];
49   let s3 : bits(8) = x[31..24];
50   let b0 : bits(8) = xt2(s0) ^ xt3(s1) ^ (s2) ^ (s3);
51   let b1 : bits(8) = (s0) ^ xt2(s1) ^ xt3(s2) ^ (s3);
52   let b2 : bits(8) = (s0) ^ (s1) ^ xt2(s2) ^ xt3(s3);
53   let b3 : bits(8) = xt3(s0) ^ (s1) ^ (s2) ^ xt2(s3);
54   b3 @ b2 @ b1 @ b0 // Return value
55 }
56
57 // 32-bit to 32-bit AES inverse MixColumn.
58 function aes_mixcolumn_inv(x : bits(32)) -> bits(32) = {
59   let s0 : bits(8) = x[7..0];
60   let s1 : bits(8) = x[15..8];
61   let s2 : bits(8) = x[23..16];
62   let s3 : bits(8) = x[31..24];
63   let b0 : bits(8) = gfmul(s0, 0xe) ^ gfmul(s1, 0xb) ^
gfmul(s2, 0xd) ^ gfmul(s3, 0x9);
64   let b1 : bits(8) = gfmul(s0, 0x9) ^ gfmul(s1, 0xe) ^
gfmul(s2, 0xb) ^ gfmul(s3, 0xd);
65   let b2 : bits(8) = gfmul(s0, 0xd) ^ gfmul(s1, 0x9) ^
gfmul(s2, 0xe) ^ gfmul(s3, 0xb);
66   let b3 : bits(8) = gfmul(s0, 0xb) ^ gfmul(s1, 0xd) ^
gfmul(s2, 0x9) ^ gfmul(s3, 0xe);
67   b3 @ b2 @ b1 @ b0 // Return value
68 }
69

```

```

70 // Turn a round number into a round constant for AES. Note
that the
71 // AES64KS1I instruction is defined such that the r argument
is always
72 // in the range 0x0..0xA. Values of rnum outside the range
0x0..0xA
73 // do not decode to the AES64KS1I instruction. The 0xA case
is used
74 // specifically for the AES-256 KeySchedule, and this func-
tion is never
75 // called in that case.
76 function aes_decode_rcon(r : bits(4)) -> bits(32) = {
77   assert(r <= 0xA);
78   match r {
79     0x0 => 0x00000001,
80     0x1 => 0x00000002,
81     0x2 => 0x00000004,
82     0x3 => 0x00000008,
83     0x4 => 0x00000010,
84     0x5 => 0x00000020,
85     0x6 => 0x00000040,
86     0x7 => 0x00000080,
87     0x8 => 0x0000001b,
88     0x9 => 0x00000036,
89     _ => internal_error(__FILE__, __LINE__, "Unexpected AES
r") // unreachable -- required to silence Sail warning
90   }
91 }
92
93 // SM4 SBox - only one sbox for forwards and inverse.
94 $anchor sm4_sbox_table_doc
95 let sm4_sbox_table : vector(256, bits(8)) = [
96   0xD6, 0x90, 0xE9, 0xFE, 0xCC, 0xE1, 0x3D, 0xB7, 0x16, 0xB6,
0x14, 0xC2, 0x28,
97   0xFB, 0x2C, 0x05, 0x2B, 0x67, 0x9A, 0x76, 0x2A, 0xBE, 0x04,
0xC3, 0xAA, 0x44,
98   0x13, 0x26, 0x49, 0x86, 0x06, 0x99, 0x9C, 0x42, 0x50, 0xF4,
0x91, 0xEF, 0x98,
99   0x7A, 0x33, 0x54, 0x0B, 0x43, 0xED, 0xCF, 0xAC, 0x62, 0xE4,
0xB3, 0x1C, 0xA9,
100  0xC9, 0x08, 0xE8, 0x95, 0x80, 0xDF, 0x94, 0xFA, 0x75, 0x8F,
0x3F, 0xA6, 0x47,
101  0x07, 0xA7, 0xFC, 0xF3, 0x73, 0x17, 0xBA, 0x83, 0x59, 0x3C,
0x19, 0xE6, 0x85,
102  0x4F, 0xA8, 0x68, 0x6B, 0x81, 0xB2, 0x71, 0x64, 0xDA, 0x8B,
0xF8, 0xEB, 0x0F,
103  0x4B, 0x70, 0x56, 0x9D, 0x35, 0x1E, 0x24, 0x0E, 0x5E, 0x63,
0x58, 0xD1, 0xA2,
104  0x25, 0x22, 0x7C, 0x3B, 0x01, 0x21, 0x78, 0x87, 0xD4, 0x00,
0x46, 0x57, 0x9F,
105  0xD3, 0x27, 0x52, 0x4C, 0x36, 0x02, 0xE7, 0xA0, 0xC4, 0xC8,
0x9E, 0xEA, 0xBF,
106  0x8A, 0xD2, 0x40, 0xC7, 0x38, 0xB5, 0xA3, 0xF7, 0xF2, 0xCE,
0xF9, 0x61, 0x15,
107  0xA1, 0xE0, 0xAE, 0x5D, 0xA4, 0x9B, 0x34, 0x1A, 0x55, 0xAD,
0x93, 0x32, 0x30,
108  0xF5, 0x8C, 0xB1, 0xE3, 0x1D, 0xF6, 0xE2, 0x2E, 0x82, 0x66,
0xCA, 0x60, 0xC0,
109  0x29, 0x23, 0xAB, 0x0D, 0x53, 0x4E, 0x6F, 0xD5, 0xDB, 0x37,
0x45, 0xDE, 0xFD,
110  0x8E, 0x2F, 0x03, 0xFF, 0x6A, 0x72, 0x6D, 0x6C, 0x5B, 0x51,
0x8D, 0x1B, 0xAF,
111  0x92, 0xBB, 0xDD, 0xBC, 0x7F, 0x11, 0xD9, 0x5C, 0x41, 0x1F,
0x10, 0x5A, 0xD8,
112  0x0A, 0xC1, 0x31, 0x88, 0xA5, 0xCD, 0x7B, 0xBD, 0x2D, 0x74,
0xD0, 0x12, 0xB8,
113  0xE5, 0xB4, 0xB0, 0x89, 0x69, 0x97, 0x4A, 0x0C, 0x96, 0x77,
0x7E, 0x65, 0xB9,
114  0xF1, 0x09, 0xC5, 0x6E, 0xC6, 0x84, 0x18, 0xF0, 0x7D, 0xEC,
0x3A, 0xDC, 0x4D,
115  0x20, 0x79, 0xEE, 0x5F, 0x3E, 0xD7, 0xCB, 0x39, 0x48,
116 ]
117
118 let aes_sbox_fwd_table : vector(256, bits(8)) = [
119   0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01,
0x67, 0x2b, 0xfe,
120   0xd7, 0xab, 0x76, 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47,
0xf0, 0xad, 0xd4,
121   0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0, 0xb7, 0xfd, 0x93, 0x26,
0x36, 0x3f, 0xf7,
122   0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15, 0x04,
0xc7, 0x23, 0xc3,
123   0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27,
0xb2, 0x75, 0x09,
124   0x83, 0x2c, 0x1a, 0x1b, 0xe6, 0x5a, 0xa0, 0x52, 0x3b, 0xd6,
0xb3, 0x29, 0xe3,
125   0x2f, 0x84, 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b,
0x6a, 0xcb, 0xbe,
126   0x39, 0x4a, 0x4c, 0x58, 0xcf, 0xd0, 0xef, 0xaa, 0xfb, 0x43,
0x4d, 0x33, 0x85,
127   0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8, 0x51, 0xa3,
0x40, 0x8f, 0x92,
128   0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3,
0xd2, 0xcd, 0xc0,
129   0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d,
0x64, 0x5d, 0x19,
130   0x73, 0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46,
0xee, 0xb8, 0x14,
131   0xde, 0x5e, 0x0b, 0xdb, 0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06,
0x24, 0x5c, 0xc2,
132   0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79, 0xe7, 0xc8, 0x37,

```

```

0x6d, 0x8d, 0xd5,
133 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
0xba, 0x78, 0x25,
134 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b,
0xbd, 0x8b, 0x8a,
135 0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35,
0x57, 0xb9, 0x86,
136 0xc1, 0x1d, 0x9e, 0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e,
0x94, 0x9b, 0x1e,
137 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf, 0x8c, 0xa1, 0x89, 0x0d,
0xbf, 0xe6, 0x42,
138 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16,
139 ]
140
141 let aes_sbox_inv_table : vector(256, bits(8)) = [
142 0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40,
0xa3, 0x9e, 0x81,
143 0xf3, 0xd7, 0xfb, 0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff,
0x87, 0x34, 0x8e,
144 0x43, 0x44, 0xc4, 0xde, 0xe9, 0xcb, 0x54, 0x7b, 0x94, 0x32,
0xa6, 0xc2, 0x23,
145 0x3d, 0xee, 0x4c, 0x95, 0x0b, 0x42, 0xfa, 0xc3, 0x4e, 0x08,
0x2e, 0xa1, 0x66,
146 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2, 0x49, 0x6d, 0x8b,
0xd1, 0x25, 0x72,
147 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c,
0xcc, 0x5d, 0x65,
148 0xb6, 0x92, 0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda,
0x5e, 0x15, 0x46,
149 0x57, 0xa7, 0x8d, 0x9d, 0x84, 0x90, 0xd8, 0xab, 0x00, 0x8c,
0xbc, 0xd3, 0x0a,
150 0xf7, 0xe4, 0x58, 0x05, 0xb8, 0xb3, 0x45, 0x06, 0xd0, 0x2c,
0x1e, 0x8f, 0xca,
151 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd, 0x03, 0x01, 0x13, 0x8a,
0x6b, 0x3a, 0x91,
152 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf, 0xce,
0xf0, 0xb4, 0xe6,
153 0x73, 0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2,
0xf9, 0x37, 0xe8,
154 0x1c, 0x75, 0xdf, 0x6e, 0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29,
0xc5, 0x89, 0x6f,
155 0xb7, 0x62, 0x0e, 0xaa, 0x18, 0xbe, 0x1b, 0xfc, 0x56, 0x3e,
0x4b, 0xc6, 0xd2,
156 0x79, 0x20, 0x9a, 0xdb, 0xc0, 0xfe, 0x78, 0xcd, 0x5a, 0xf4,
0x1f, 0xdd, 0xa8,
157 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10, 0x59, 0x27,
0x80, 0xec, 0x5f,
158 0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5,
0x7a, 0x9f, 0x93,
159 0xc9, 0x9c, 0xef, 0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5,
0xb0, 0xc8, 0xeb,
160 0xbb, 0x3c, 0x83, 0x53, 0x99, 0x61, 0x17, 0x2b, 0x04, 0x7e,
0xba, 0x77, 0xd6,
161 0x26, 0xe1, 0x69, 0x14, 0x63, 0x55, 0x21, 0x0c, 0x7d,
162 ]
163
164 /// Lookup function - takes an index and a table, and re-
trieves the
165 /// x'th element of that table. Note that the Sail vector lit-
erals
166 /// start at index 255, and go down to 0.
167 function sbox_lookup(x : bits(8), table : vector(256,
bits(8))) -> bits(8) = {
168   table[255 - unsigned(x)]
169 }
170
171 /// Easy function to perform a forward AES SBox operation on 1
byte.
172 function aes_sbox_fwd(x : bits(8)) -> bits(8) = sbox_lookup(x,
aes_sbox_fwd_table)
173
174 /// Easy function to perform an inverse AES SBox operation on
1 byte.
175 function aes_sbox_inv(x : bits(8)) -> bits(8) = sbox_lookup(x,
aes_sbox_inv_table)
176
177 /// AES SubWord function used in the key expansion
178 /// - Applies the forward sbox to each byte in the input word.
179 function aes_subword_fwd(x : bits(32)) -> bits(32) = {
180   aes_sbox_fwd(x[31..24]) @
181   aes_sbox_fwd(x[23..16]) @
182   aes_sbox_fwd(x[15.. 8]) @
183   aes_sbox_fwd(x[ 7.. 0])
184 }
185
186 /// AES Inverse SubWord function
187 /// - Applies the inverse sbox to each byte in the input word.
188 function aes_subword_inv(x : bits(32)) -> bits(32) = {
189   aes_sbox_inv(x[31..24]) @
190   aes_sbox_inv(x[23..16]) @
191   aes_sbox_inv(x[15.. 8]) @
192   aes_sbox_inv(x[ 7.. 0])
193 }
194
195 /// Easy function to perform an SM4 SBox operation on 1 byte.
196 function sm4_sbox(x : bits(8)) -> bits(8) = sbox_lookup(x,
sm4_sbox_table)
197

```

```

198 function aes_get_column(state : bits(128), c : range(0, 3)) ->
bits(32) =
199   state[32 * c + 31 .. 32 * c]
200
201 /// 64-bit to 64-bit function which applies the AES forward
sbox to each byte
202 /// in a 64-bit word.
203 function aes_apply_fwd_sbox_to_each_byte(x : bits(64)) ->
bits(64) = {
204   aes_sbox_fwd(x[63..56]) @
205   aes_sbox_fwd(x[55..48]) @
206   aes_sbox_fwd(x[47..40]) @
207   aes_sbox_fwd(x[39..32]) @
208   aes_sbox_fwd(x[31..24]) @
209   aes_sbox_fwd(x[23..16]) @
210   aes_sbox_fwd(x[15.. 8]) @
211   aes_sbox_fwd(x[ 7.. 0])
212 }
213
214 /// 64-bit to 64-bit function which applies the AES inverse
sbox to each byte
215 /// in a 64-bit word.
216 function aes_apply_inv_sbox_to_each_byte(x : bits(64)) ->
bits(64) = {
217   aes_sbox_inv(x[63..56]) @
218   aes_sbox_inv(x[55..48]) @
219   aes_sbox_inv(x[47..40]) @
220   aes_sbox_inv(x[39..32]) @
221   aes_sbox_inv(x[31..24]) @
222   aes_sbox_inv(x[23..16]) @
223   aes_sbox_inv(x[15.. 8]) @
224   aes_sbox_inv(x[ 7.. 0])
225 }
226
227 /// AES full-round transformation functions:
228 $anchor aes_full_round_transformations_doc
229
230 function getbyte(x : bits(64), i : range(0, 7)) -> bits(8) =
231   x[8 * i + 7 .. 8 * i]
232
233 function aes_rv64_shiftrows_fwd(rs2 : bits(64), rs1 :
bits(64)) -> bits(64) = {
234   getbyte(rs1, 3) @
235   getbyte(rs2, 6) @
236   getbyte(rs2, 1) @
237   getbyte(rs1, 4) @
238   getbyte(rs2, 7) @
239   getbyte(rs2, 2) @
240   getbyte(rs1, 5) @
241   getbyte(rs1, 0)
242 }
243
244 function aes_rv64_shiftrows_inv(rs2 : bits(64), rs1 :
bits(64)) -> bits(64) = {
245   getbyte(rs2, 3) @
246   getbyte(rs2, 6) @
247   getbyte(rs1, 1) @
248   getbyte(rs1, 4) @
249   getbyte(rs1, 7) @
250   getbyte(rs2, 2) @
251   getbyte(rs2, 5) @
252   getbyte(rs1, 0)
253 }
254
255 /// 128-bit to 128-bit implementation of the forward AES
ShiftRows transform.
256 /// Byte 0 of state is input column 0, bits 7..0.
257 /// Byte 5 of state is input column 1, bits 15..8.
258 function aes_shift_rows_fwd(x : bits(128)) -> bits(128) = {
259   let ic3 : bits(32) = aes_get_column(x, 3);
260   let ic2 : bits(32) = aes_get_column(x, 2);
261   let ic1 : bits(32) = aes_get_column(x, 1);
262   let ic0 : bits(32) = aes_get_column(x, 0);
263
264   let oc0 : bits(32) = ic3[31..24] @ ic2[23..16] @ ic1[15.. 8] @
ic0[ 7.. 0];
265
266   let oc1 : bits(32) = ic0[31..24] @ ic3[23..16] @ ic2[15.. 8] @
ic1[ 7.. 0];
267
268   let oc2 : bits(32) = ic1[31..24] @ ic0[23..16] @ ic3[15.. 8] @
ic2[ 7.. 0];
269
270   let oc3 : bits(32) = ic2[31..24] @ ic1[23..16] @ ic0[15.. 8] @
ic3[ 7.. 0];
271
272   (oc3 @ oc2 @ oc1 @ oc0) // Return value
273 }
274
275 /// 128-bit to 128-bit implementation of the inverse AES
ShiftRows transform.
276 /// Byte 0 of state is input column 0, bits 7..0.
277 /// Byte 5 of state is input column 1, bits 15..8.
278 function aes_shift_rows_inv(x : bits(128)) -> bits(128) = {
279   let ic3 : bits(32) = aes_get_column(x, 3); // In column 3
280   let ic2 : bits(32) = aes_get_column(x, 2);
281   let ic1 : bits(32) = aes_get_column(x, 1);
282   let ic0 : bits(32) = aes_get_column(x, 0);
283
284   let oc0 : bits(32) = ic1[31..24] @ ic2[23..16] @ ic3[15.. 8] @
ic0[ 7.. 0];
285

```

```

279 let oc1 : bits(32) = ic2[31..24] @ ic3[23..16] @ ic0[15.. 8] @
ic1[ 7.. 0];

280 let oc2 : bits(32) = ic3[31..24] @ ic0[23..16] @ ic1[15.. 8] @
ic2[ 7.. 0];

281 let oc3 : bits(32) = ic0[31..24] @ ic1[23..16] @ ic2[15.. 8] @
ic3[ 7.. 0];
282 (oc3 @ oc2 @ oc1 @ oc0) // Return value
283 }
284
285 /// Applies the forward sub-bytes step of AES to a 128-bit
vector
286 /// representation of its state.
287 function aes_subbytes_fwd(x : bits(128)) -> bits(128) = {
288 let oc0 : bits(32) = aes_subword_fwd(aes_get_column(x, 0));
289 let oc1 : bits(32) = aes_subword_fwd(aes_get_column(x, 1));
290 let oc2 : bits(32) = aes_subword_fwd(aes_get_column(x, 2));
291 let oc3 : bits(32) = aes_subword_fwd(aes_get_column(x, 3));
292 (oc3 @ oc2 @ oc1 @ oc0) // Return value
293 }
294
295 /// Applies the inverse sub-bytes step of AES to a 128-bit
vector
296 /// representation of its state.
297 function aes_subbytes_inv(x : bits(128)) -> bits(128) = {
298 let oc0 : bits(32) = aes_subword_inv(aes_get_column(x, 0));
299 let oc1 : bits(32) = aes_subword_inv(aes_get_column(x, 1));
300 let oc2 : bits(32) = aes_subword_inv(aes_get_column(x, 2));
301 let oc3 : bits(32) = aes_subword_inv(aes_get_column(x, 3));
302 (oc3 @ oc2 @ oc1 @ oc0) // Return value
303 }
304
305 /// Applies the forward MixColumns step of AES to a 128-bit
vector
306 /// representation of its state.
307 function aes_mixcolumns_fwd(x : bits(128)) -> bits(128) = {
308 let oc0 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x,
0));
309 let oc1 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x,
1));
310 let oc2 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x,
2));
311 let oc3 : bits(32) = aes_mixcolumn_fwd(aes_get_column(x,
3));
312 (oc3 @ oc2 @ oc1 @ oc0) // Return value
313 }
314
315 /// Applies the inverse MixColumns step of AES to a 128-bit
vector
316 /// representation of its state.
317 function aes_mixcolumns_inv(x : bits(128)) -> bits(128) = {
318 let oc0 : bits(32) = aes_mixcolumn_inv(aes_get_column(x,
0));
319 let oc1 : bits(32) = aes_mixcolumn_inv(aes_get_column(x,
1));
320 let oc2 : bits(32) = aes_mixcolumn_inv(aes_get_column(x,
2));
321 let oc3 : bits(32) = aes_mixcolumn_inv(aes_get_column(x,
3));
322 (oc3 @ oc2 @ oc1 @ oc0) // Return value
323 }

```

8. riscv_sail-riscv/model/extensions/K/zbkb_insts.sail (122 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8 //
9 //
=====
10 union clause instruction = ZBKB_RTYPE : (regidx, regidx, re-
gidx, brop_zbkb)
11
12 $[wavedrom "PACK _ _ PACK _ OP"]
13 mapping clause encdec = ZBKB_RTYPE(rs2, rs1, rd, PACK)
14 <-> 0b0000100 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b100 @
encdec_reg(rd) @ 0b0110011
15 when currentlyEnabled(Ext_Zbkb)
16
17 $[wavedrom "PACKH _ _ PACKH _ OP"]
18 mapping clause encdec = ZBKB_RTYPE(rs2, rs1, rd, PACKH)
19 <-> 0b0000100 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b111 @
encdec_reg(rd) @ 0b0110011
20 when currentlyEnabled(Ext_Zbkb)
21
22 mapping zbkb_rtype_mnemonic : brop_zbkb <-> string = {
23 PACK <-> "pack",
24 PACKH <-> "packh"

```

```

25 }
26
27 mapping clause assembly = ZBKB_RTYPE(rs2, rs1, rd, op)

28 <-> zbkb_rtype_mnemonic(op) ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
29
30 $[split op]
31 function clause execute ZBKB_RTYPE(rs2, rs1, rd, op) = {
32 let rs1_val = X(rs1);
33 let rs2_val = X(rs2);
34 let result : xlenbits = match op {
35 PACK => rs2_val[(xlen_bytes*4 - 1)..0] @
rs1_val[(xlen_bytes*4 - 1)..0],
36 PACKH => zero_extend(rs2_val[7..0] @ rs1_val[7..0])
37 };
38 X(rd) = result;
39 RETIRE_SUCCESS
40 }
41
42 //
=====
43 union clause instruction = ZBKB_PACKW : (regidx, regidx, re-
gidx)
44
45 $[wavedrom "_ _ _ _ OP-32"]
46 mapping clause encdec = ZBKB_PACKW(rs2, rs1, rd)
47 <-> 0b0000100 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b100 @
encdec_reg(rd) @ 0b0111011
48 when currentlyEnabled(Ext_Zbkb) & xlen == 64
49
50 mapping clause assembly = ZBKB_PACKW(rs2, rs1, rd)

51 <-> "packw" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1) ^
sep() ^ reg_name(rs2)
52
53 function clause execute ZBKB_PACKW(rs2, rs1, rd) = {
54 assert(xlen == 64);
55 let rs1_val = X(rs1);
56 let rs2_val = X(rs2);
57 let result : bits(32) = rs2_val[15..0] @ rs1_val[15..0];
58 X(rd) = sign_extend(result);
59 RETIRE_SUCCESS
60 }
61
62 //
=====
63 union clause instruction = ZIP : (regidx, regidx)
64
65 $[wavedrom "_ _ _ _ OP-IMM"]
66 mapping clause encdec = ZIP(rs1, rd)
67 <-> 0b000010001111 @ encdec_reg(rs1) @ 0b001 @ enc-
dec_reg(rd) @ 0b0010011
68 when currentlyEnabled(Ext_Zbkb) & xlen == 32
69
70 mapping clause assembly = ZIP(rs1, rd)
71 <-> "zip" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
72
73 function clause execute ZIP(rs1, rd) = {
74 assert(xlen == 32);
75 let rs1_val = X(rs1);
76 var result : xlenbits = zeros();
77 foreach (i from 0 to (xlen_bytes*4 - 1)) {
78 result[i*2] = rs1_val[i];
79 result[i*2 + 1] = rs1_val[i + xlen_bytes*4];
80 };
81 X(rd) = result;
82 RETIRE_SUCCESS
83 }
84
85 //
=====
86 union clause instruction = UNZIP : (regidx, regidx)
87
88 $[wavedrom "_ _ _ _ OP-IMM"]
89 mapping clause encdec = UNZIP(rs1, rd)
90 <-> 0b000010001111 @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0010011
91 when currentlyEnabled(Ext_Zbkb) & xlen == 32
92
93 mapping clause assembly = UNZIP(rs1, rd)
94 <-> "unzip" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
95
96 function clause execute UNZIP(rs1, rd) = {
97 assert(xlen == 32);
98 let rs1_val = X(rs1);
99 var result : xlenbits = zeros();
100 foreach (i from 0 to (xlen_bytes*4 - 1)) {
101 result[i] = rs1_val[i*2];
102 result[i + xlen_bytes*4] = rs1_val[i*2 + 1];
103 };
104 X(rd) = result;
105 RETIRE_SUCCESS
106 }
107
108 //
=====
109 union clause instruction = BREV8 : (regidx, regidx)
110
111 $[wavedrom "_ _ _ _ OP-IMM"]
112 mapping clause encdec = BREV8(rs1, rd)

```

```

113 <-> 0b011010000111 @ encdec_reg(rs1) @ 0b101 @ enc-
dec_reg(rd) @ 0b0010011
114 when currentlyEnabled(Ext_Zbkb)
115
116 mapping clause assembly = BREV8(rs1, rd)
117 <-> "brev8" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
118
119 function clause execute BREV8(rs1, rd) = {
120   X(rd) = brev8(X(rs1));
121   RETIRE_SUCCESS
122 }

```

9. riscv_sail-riscv/model/extensions/K/zbkx_insts.sail (59 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8
9 function clause currentlyEnabled(Ext_Zbkx) = hartSup-
ports(Ext_Zbkx)
10
11 //
*****
12 union clause instruction = XPERM8 : (regidx, regidx, regidx)
13
14 $[wavedrom " _ _ _ _ _ OP"]
15 mapping clause encdec = XPERM8(rs2, rs1, rd)
16 <-> 0b0010100 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b100 @
encdec_reg(rd) @ 0b0110011
17 when currentlyEnabled(Ext_Zbkx)
18
19 mapping clause assembly = XPERM8(rs2, rs1, rd)
20 <-> "xperm8" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
^ sep() ^ reg_name(rs2)
21
22 function clause execute XPERM8(rs2, rs1, rd) = {
23   let rs1_val = X(rs1);
24   let rs2_val = X(rs2);
25   var result : xlenbits = zeros();
26   foreach (i from 0 to (xlen - 8) by 8) {
27     let index = unsigned(rs2_val[i+7..i]);
28     result[i+7..i] = if 8*index < xlen
29                       then rs1_val[8*index+7..8*index]
30                       else zeros()
31   };
32   X(rd) = result;
33   RETIRE_SUCCESS
34 }
35
36 //
*****
37 union clause instruction = XPERM4 : (regidx, regidx, regidx)
38
39 $[wavedrom " _ _ _ _ _ OP"]
40 mapping clause encdec = XPERM4(rs2, rs1, rd)
41 <-> 0b0010100 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b010 @
encdec_reg(rd) @ 0b0110011
42 when currentlyEnabled(Ext_Zbkx)
43
44 mapping clause assembly = XPERM4(rs2, rs1, rd)
45 <-> "xperm4" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
^ sep() ^ reg_name(rs2)
46
47 function clause execute XPERM4(rs2, rs1, rd) = {
48   let rs1_val = X(rs1);
49   let rs2_val = X(rs2);
50   var result : xlenbits = zeros();
51   foreach (i from 0 to (xlen - 4) by 4) {
52     let index = unsigned(rs2_val[i+3..i]);
53     result[i+3..i] = if 4*index < xlen
54                       then rs1_val[4*index+3..4*index]
55                       else zeros()
56   };
57   X(rd) = result;
58   RETIRE_SUCCESS
59 }

```

10. riscv_sail-riscv/model/extensions/K/zkn_insts.sail (464 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause

```

```

7 //
=====
8
9 //
10 // Scalar Cryptography Extension - Scalar SHA256 instructions
(RV32/RV64)
11 //
-----
--
12
13 function clause currentlyEnabled(Ext_Zknh) = hartSup-
ports(Ext_Zknh)
14
15 union clause instruction = SHA256SIG0 : (regidx, regidx)
16 union clause instruction = SHA256SIG1 : (regidx, regidx)
17 union clause instruction = SHA256SUM0 : (regidx, regidx)
18 union clause instruction = SHA256SUM1 : (regidx, regidx)
19
20 $[wavedrom " _ _ _ _ _ "]
21 mapping clause encdec = SHA256SUM0 (rs1, rd)
22 <-> 0b00 @ 0b01000 @ 0b00000 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
23 when currentlyEnabled(Ext_Zknh)
24
25 $[wavedrom " _ _ _ _ _ "]
26 mapping clause encdec = SHA256SUM1 (rs1, rd)
27 <-> 0b00 @ 0b01000 @ 0b00001 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
28 when currentlyEnabled(Ext_Zknh)
29
30 $[wavedrom " _ _ _ _ _ "]
31 mapping clause encdec = SHA256SIG0 (rs1, rd)
32 <-> 0b00 @ 0b01000 @ 0b00010 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
33 when currentlyEnabled(Ext_Zknh)
34
35 $[wavedrom " _ _ _ _ _ "]
36 mapping clause encdec = SHA256SIG1 (rs1, rd)
37 <-> 0b00 @ 0b01000 @ 0b00011 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
38 when currentlyEnabled(Ext_Zknh)
39
40 mapping clause assembly = SHA256SIG0 (rs1, rd)
41 <-> "sha256sig0" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
42
43 mapping clause assembly = SHA256SIG1 (rs1, rd)
44 <-> "sha256sig1" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
45
46 mapping clause assembly = SHA256SUM0 (rs1, rd)
47 <-> "sha256sum0" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
48
49 mapping clause assembly = SHA256SUM1 (rs1, rd)
50 <-> "sha256sum1" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
51
52 function clause execute SHA256SIG0(rs1, rd) = {
53   let inb : bits(32) = X(rs1)[31..0];
54   let result : bits(32) = (inb >> 7) ^ (inb >> 18) ^ (inb
>> 3);
55   X(rd) = sign_extend(result);
56   RETIRE_SUCCESS
57 }
58
59 function clause execute SHA256SIG1(rs1, rd) = {
60   let inb : bits(32) = X(rs1)[31..0];
61   let result : bits(32) = (inb >> 17) ^ (inb >> 19) ^ (inb
>> 10);
62   X(rd) = sign_extend(result);
63   RETIRE_SUCCESS
64 }
65
66 function clause execute SHA256SUM0(rs1, rd) = {
67   let inb : bits(32) = X(rs1)[31..0];
68   let result : bits(32) = (inb >> 2) ^ (inb >> 13) ^ (inb
>>> 22);
69   X(rd) = sign_extend(result);
70   RETIRE_SUCCESS
71 }
72
73 function clause execute SHA256SUM1(rs1, rd) = {
74   let inb : bits(32) = X(rs1)[31..0];
75   let result : bits(32) = (inb >> 6) ^ (inb >> 11) ^ (inb
>>> 25);
76   X(rd) = sign_extend(result);
77   RETIRE_SUCCESS
78 }
79
80 // Scalar Cryptography Extension - Scalar 32-bit AES instruc-
tions (encrypt)
81 //
-----
--
82
83 function clause currentlyEnabled(Ext_Zkne) = hartSup-
ports(Ext_Zkne)
84
85 union clause instruction = AES32ESMI : (bits(2), regidx, re-

```



```

gidx, regidx)
86
87 $[wavedrom " _ _ _ _ _ "]
88 mapping clause encdec = AES32ESMI (bs, rs2, rs1, rd)

89 <-> bs @ 0b10011 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @
encdec_reg(rd) @ 0b0110011
90 when currentlyEnabled(Ext_Zkne) & xlen == 32
91
92 mapping clause assembly = AES32ESMI (bs, rs2, rs1, rd) <->

93 "aes32esmi" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1) ^
sep() ^ reg_name(rs2) ^ sep() ^ hex_bits_2(bs)
94
95 function clause execute AES32ESMI (bs, rs2, rs1, rd) = {
96 let shamt : bits( 5) = bs @ 0b000; // shamt = bs*8
97 let si     : bits( 8) = (X(rs2) >> shamt)[7..0]; // SBox
Input
98 let so     : bits( 8) = aes_sbox_fwd(si);
99 let mixed  : bits(32) = aes_mixcolumn_byte_fwd(so);
100 let result : bits(32) = X(rs1)[31..0] ^ (mixed <<< shamt);
101 X(rd) = sign_extend(result);
102 RETIRE_SUCCESS
103 }
104
105 union clause instruction = AES32ESI : (bits(2), regidx, re-
gidx, regidx)
106
107 $[wavedrom " _ _ _ _ _ "]
108 mapping clause encdec = AES32ESI (bs, rs2, rs1, rd)

109 <-> bs @ 0b10001 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @
encdec_reg(rd) @ 0b0110011
110 when currentlyEnabled(Ext_Zkne) & xlen == 32
111
112 mapping clause assembly = AES32ESI (bs, rs2, rs1, rd) <->
113 "aes32esi" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
^ sep() ^ reg_name(rs2) ^ sep() ^ hex_bits_2(bs)
114
115 function clause execute AES32ESI (bs, rs2, rs1, rd) = {
116 let shamt : bits( 5) = bs @ 0b000; // shamt = bs*8
117 let si     : bits( 8) = (X(rs2) >> shamt)[7..0]; // SBox
Input
118 let so     : bits(32) = 0x000000 @ aes_sbox_fwd(si);
119 let result : bits(32) = X(rs1)[31..0] ^ (so <<< shamt);
120 X(rd) = sign_extend(result);
121 RETIRE_SUCCESS
122 }
123
124 // Scalar Cryptography Extension - Scalar 32-bit AES instruc-
tions (decrypt)
125 //
-----
126
127 function clause currentlyEnabled(Ext_Zknd) = hartSup-
ports(Ext_Zknd)
128
129 union clause instruction = AES32DSMI : (bits(2), regidx, re-
gidx, regidx)
130
131 $[wavedrom " _ _ _ _ _ "]
132 mapping clause encdec = AES32DSMI (bs, rs2, rs1, rd)

133 <-> bs @ 0b10111 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @
encdec_reg(rd) @ 0b0110011
134 when currentlyEnabled(Ext_Zknd) & xlen == 32
135
136 mapping clause assembly = AES32DSMI (bs, rs2, rs1, rd) <->

137 "aes32dsmi" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1) ^
sep() ^ reg_name(rs2) ^ sep() ^ hex_bits_2(bs)
138
139 function clause execute AES32DSMI (bs, rs2, rs1, rd) = {
140 let shamt : bits( 5) = bs @ 0b000; // shamt = bs*8
141 let si     : bits( 8) = (X(rs2) >> shamt)[7..0]; // SBox
Input
142 let so     : bits( 8) = aes_sbox_inv(si);
143 let mixed  : bits(32) = aes_mixcolumn_byte_inv(so);
144 let result : bits(32) = X(rs1)[31..0] ^ (mixed <<< shamt);
145 X(rd) = sign_extend(result);
146 RETIRE_SUCCESS
147 }
148
149 union clause instruction = AES32DSI : (bits(2), regidx, re-
gidx, regidx)
150
151 $[wavedrom " _ _ _ _ _ "]
152 mapping clause encdec = AES32DSI (bs, rs2, rs1, rd)

153 <-> bs @ 0b10101 @ encdec_reg(rs2) @ encdec_reg(rs1) @ 0b000 @
encdec_reg(rd) @ 0b0110011
154 when currentlyEnabled(Ext_Zknd) & xlen == 32
155
156 mapping clause assembly = AES32DSI (bs, rs2, rs1, rd) <->
157 "aes32dsi" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
^ sep() ^ reg_name(rs2) ^ sep() ^ hex_bits_2(bs)
158
159 function clause execute AES32DSI (bs, rs2, rs1, rd) = {
160 let shamt : bits( 5) = bs @ 0b000; // shamt = bs*8
161 let si     : bits( 8) = (X(rs2) >> shamt)[7..0]; // SBox

```

```

Input
162 let so     : bits(32) = 0x000000 @ aes_sbox_inv(si);
163 let result : bits(32) = X(rs1)[31..0] ^ (so <<< shamt);
164 X(rd) = sign_extend(result);
165 RETIRE_SUCCESS
166 }
167
168 // Scalar Cryptography Extension - Scalar 32-bit SHA512 in-
structions
169 //
-----
170
171 union clause instruction = SHA512SIG0L : (regidx, regidx, re-
gidx)
172 union clause instruction = SHA512SIG0H : (regidx, regidx, re-
gidx)
173 union clause instruction = SHA512SIG1L : (regidx, regidx, re-
gidx)
174 union clause instruction = SHA512SIG1H : (regidx, regidx, re-
gidx)
175 union clause instruction = SHA512SUM0R : (regidx, regidx, re-
gidx)
176 union clause instruction = SHA512SUM1R : (regidx, regidx, re-
gidx)
177
178 $[wavedrom " _ _ _ _ _ "]
179 mapping clause encdec = SHA512SUM0R (rs2, rs1, rd)
180 <-> 0b01 @ 0b01000 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
181 when currentlyEnabled(Ext_Zknh) & xlen == 32
182
183 $[wavedrom " _ _ _ _ _ "]
184 mapping clause encdec = SHA512SUM1R (rs2, rs1, rd)
185 <-> 0b01 @ 0b01001 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
186 when currentlyEnabled(Ext_Zknh) & xlen == 32
187
188 $[wavedrom " _ _ _ _ _ "]
189 mapping clause encdec = SHA512SIG0L (rs2, rs1, rd)
190 <-> 0b01 @ 0b01010 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
191 when currentlyEnabled(Ext_Zknh) & xlen == 32
192
193 $[wavedrom " _ _ _ _ _ "]
194 mapping clause encdec = SHA512SIG0H (rs2, rs1, rd)
195 <-> 0b01 @ 0b01110 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
196 when currentlyEnabled(Ext_Zknh) & xlen == 32
197
198 $[wavedrom " _ _ _ _ _ "]
199 mapping clause encdec = SHA512SIG1L (rs2, rs1, rd)
200 <-> 0b01 @ 0b01011 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
201 when currentlyEnabled(Ext_Zknh) & xlen == 32
202
203 $[wavedrom " _ _ _ _ _ "]
204 mapping clause encdec = SHA512SIG1H (rs2, rs1, rd)
205 <-> 0b01 @ 0b01111 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
206 when currentlyEnabled(Ext_Zknh) & xlen == 32
207
208 mapping clause assembly = SHA512SIG0L (rs2, rs1, rd)
209 <-> "sha512sig0l" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
210
211 mapping clause assembly = SHA512SIG0H (rs2, rs1, rd)
212 <-> "sha512sig0h" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
213
214 mapping clause assembly = SHA512SIG1L (rs2, rs1, rd)
215 <-> "sha512sig1l" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
216
217 mapping clause assembly = SHA512SIG1H (rs2, rs1, rd)
218 <-> "sha512sig1h" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
219
220 mapping clause assembly = SHA512SUM0R (rs2, rs1, rd)
221 <-> "sha512sum0r" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
222
223 mapping clause assembly = SHA512SUM1R (rs2, rs1, rd)
224 <-> "sha512sum1r" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
225
226 function clause execute SHA512SIG0H(rs2, rs1, rd) = {
227 X(rd) = sign_extend((X(rs1) >> 1) ^ (X(rs1) >> 7) ^
(X(rs1) >> 8) ^
228 (X(rs2) << 31) ^ (X(rs2) <<
24));
229 RETIRE_SUCCESS
230 }
231
232 function clause execute SHA512SIG0L(rs2, rs1, rd) = {
233 X(rd) = sign_extend((X(rs1) >> 1) ^ (X(rs1) >> 7) ^
(X(rs1) >> 8) ^
234 (X(rs2) << 31) ^ (X(rs2) << 25) ^ (X(rs2) <<
24));
235 RETIRE_SUCCESS
236 }

```

```

237
238 function clause execute SHA512SIG1H(rs2, rs1, rd) = {
239   X(rd) = sign_extend((X(rs1) << 3) ^ (X(rs1) >> 6) ^
(X(rs1) >> 19) ^
240     (X(rs2) >> 29) ^ (X(rs2) <<
13));
241   RETIRE_SUCCESS
242 }
243
244 function clause execute SHA512SIG1L(rs2, rs1, rd) = {
245   X(rd) = sign_extend((X(rs1) << 3) ^ (X(rs1) >> 6) ^
(X(rs1) >> 19) ^
246     (X(rs2) >> 29) ^ (X(rs2) << 26) ^ (X(rs2) <<
13));
247   RETIRE_SUCCESS
248 }
249
250 function clause execute SHA512SUM0R(rs2, rs1, rd) = {
251   X(rd) = sign_extend((X(rs1) << 25) ^ (X(rs1) << 30) ^
(X(rs1) >> 28) ^
252     (X(rs2) >> 7) ^ (X(rs2) >> 2) ^ (X(rs2) <<
4));
253   RETIRE_SUCCESS
254 }
255
256 function clause execute SHA512SUM1R(rs2, rs1, rd) = {
257   X(rd) = sign_extend((X(rs1) << 23) ^ (X(rs1) >> 14) ^
(X(rs1) >> 18) ^
258     (X(rs2) >> 9) ^ (X(rs2) << 18) ^ (X(rs2) <<
14));
259   RETIRE_SUCCESS
260 }
261
262 // Scalar Cryptography Extension - Scalar 64-bit AES instruc-
tions
263 //
-----
264
265 union clause instruction = AES64KS1I : (bits(4), regidx, re-
gidx)
266 union clause instruction = AES64KS2 : (regidx, regidx, re-
gidx)
267 union clause instruction = AES64IM : (regidx, regidx)
268 union clause instruction = AES64ESM : (regidx, regidx, re-
gidx)
269 union clause instruction = AES64ES : (regidx, regidx, re-
gidx)
270 union clause instruction = AES64DSM : (regidx, regidx, re-
gidx)
271 union clause instruction = AES64DS : (regidx, regidx, re-
gidx)
272
273 $[wavedrom "-----"]
274 mapping clause encdec = AES64KS1I (rnum, rs1, rd)
275 <-> 0b00 @ 0b11000 @ 0b1 rnum @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
276 _when (currentlyEnabled(Ext_Zkne) | currentlyEn-
abled(Ext_Zknd)) & (xlen == 64) & (rnum <_u 0xB)
277
278 $[wavedrom "-----"]
279 mapping clause encdec = AES64IM (rs1, rd)
280 <-> 0b00 @ 0b11000 @ 0b00000 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
281 _when currentlyEnabled(Ext_Zknd) & xlen == 64
282
283 $[wavedrom "-----"]
284 mapping clause encdec = AES64KS2 (rs2, rs1, rd)
285 <-> 0b01 @ 0b11111 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
286 _when (currentlyEnabled(Ext_Zkne) | currentlyEn-
abled(Ext_Zknd)) & xlen == 64
287
288 $[wavedrom "-----"]
289 mapping clause encdec = AES64ESM (rs2, rs1, rd)
290 <-> 0b00 @ 0b11011 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
291 _when currentlyEnabled(Ext_Zkne) & xlen == 64
292
293 $[wavedrom "-----"]
294 mapping clause encdec = AES64ES (rs2, rs1, rd)
295 <-> 0b00 @ 0b11001 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
296 _when currentlyEnabled(Ext_Zkne) & xlen == 64
297
298 $[wavedrom "-----"]
299 mapping clause encdec = AES64DSM (rs2, rs1, rd)
300 <-> 0b00 @ 0b11111 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
301 _when currentlyEnabled(Ext_Zknd) & xlen == 64
302
303 $[wavedrom "-----"]
304 mapping clause encdec = AES64DS (rs2, rs1, rd)
305 <-> 0b00 @ 0b11011 @ encdec_reg(rs2) @ encdec_reg(rs1) @
0b000 @ encdec_reg(rd) @ 0b0110011
306 _when currentlyEnabled(Ext_Zknd) & xlen == 64
307
308 mapping clause assembly = AES64KS1I (rnum, rs1, rd)
309 <-> "aes64ks1i" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ hex_bits_4(rnum)
310

```

```

311 mapping clause assembly = AES64KS2 (rs2, rs1, rd)
312 <-> "aes64ks2" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
313
314 mapping clause assembly = AES64IM (rs1, rd)
315 <-> "aes64im" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1)
316
317 mapping clause assembly = AES64ESM (rs2, rs1, rd)
318 <-> "aes64esm" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
319
320 mapping clause assembly = AES64ES (rs2, rs1, rd)
321 <-> "aes64es" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1) ^
sep() ^ reg_name(rs2)
322
323 mapping clause assembly = AES64DSM (rs2, rs1, rd)
324 <-> "aes64dsm" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1) ^ sep() ^ reg_name(rs2)
325
326 mapping clause assembly = AES64DS (rs2, rs1, rd)
327 <-> "aes64ds" ^ spc() ^ reg_name(rd) ^ sep() ^ reg_name(rs1) ^
sep() ^ reg_name(rs2)
328
329 // Note: The decoding for this instruction ensures that `rnum`
is always in
330 // the range 0x0..0xA. See the encdec clause for AES64KS1I.
331 // The rnum == 0xA case is used specifically for the AES-256
KeySchedule
332 function clause execute AES64KS1I(rnum, rs1, rd) = {
333   assert(xlen == 64);
334   let prev : bits(32) = X(rs1)[63..32];
335   let subwords : bits(32) = aes_subword_fwd(prev);
336   let result : bits(32) = if (rnum == 0xA) then subwords
else (subwords >>> 8) ^ aes_de-
code_rcon(rnum);
337   X(rd) = result @ result;
338   RETIRE_SUCCESS
339 }
340
341
342 function clause execute AES64KS2(rs2, rs1, rd) = {
343   assert(xlen == 64);
344   let w0 : bits(32) = X(rs1)[63..32] ^ X(rs2)[31..0];
345   let w1 : bits(32) = X(rs1)[63..32] ^ X(rs2)[31..0] ^ X(rs2)
[63..32];
346   X(rd) = w1 @ w0;
347   RETIRE_SUCCESS
348 }
349
350 function clause execute AES64IM(rs1, rd) = {
351   assert(xlen == 64);
352   let w0 : bits(32) = aes_mixcolumn_inv(X(rs1)[31..0]);
353   let w1 : bits(32) = aes_mixcolumn_inv(X(rs1)[63..32]);
354   X(rd) = w1 @ w0;
355   RETIRE_SUCCESS
356 }
357
358 function clause execute AES64ESM(rs2, rs1, rd) = {
359   assert(xlen == 64);
360   let sr : bits(64) = aes_rv64_shiftrows_fwd(X(rs2), X(rs1));
361   let wd : bits(64) = sr[63..0];
362   let sb : bits(64) = aes_apply_fwd_sbox_to_each_byte(wd);
363   X(rd) = aes_mixcolumn_fwd(sb[63..32]) @ aes_mix-
column_fwd(sb[31..0]);
364   RETIRE_SUCCESS
365 }
366
367 function clause execute AES64ES(rs2, rs1, rd) = {
368   assert(xlen == 64);
369   let sr : bits(64) = aes_rv64_shiftrows_fwd(X(rs2), X(rs1));
370   let wd : bits(64) = sr[63..0];
371   X(rd) = aes_apply_fwd_sbox_to_each_byte(wd);
372   RETIRE_SUCCESS
373 }
374
375 function clause execute AES64DSM(rs2, rs1, rd) = {
376   assert(xlen == 64);
377   let sr : bits(64) = aes_rv64_shiftrows_inv(X(rs2), X(rs1));
378   let wd : bits(64) = sr[63..0];
379   let sb : bits(64) = aes_apply_inv_sbox_to_each_byte(wd);
380   X(rd) = aes_mixcolumn_inv(sb[63..32]) @ aes_mix-
column_inv(sb[31..0]);
381   RETIRE_SUCCESS
382 }
383
384 function clause execute AES64DS(rs2, rs1, rd) = {
385   assert(xlen == 64);
386   let sr : bits(64) = aes_rv64_shiftrows_inv(X(rs2), X(rs1));
387   let wd : bits(64) = sr[63..0];
388   X(rd) = aes_apply_inv_sbox_to_each_byte(wd);
389   RETIRE_SUCCESS
390 }
391
392 // Scalar Cryptography Extension - Scalar 64-bit SHA512 in-
structions
393 //
-----
394

```



```

395 union clause instruction = SHA512SIG0 : (regidx, regidx)
396 union clause instruction = SHA512SIG1 : (regidx, regidx)
397 union clause instruction = SHA512SUM0 : (regidx, regidx)
398 union clause instruction = SHA512SUM1 : (regidx, regidx)
399
400 $[wavedrom " _ _ _ _ _ "]
401 mapping clause encdec = SHA512SUM0 (rs1, rd)
402 <-> 0b00 @ 0b01000 @ 0b00100 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
403 when currentlyEnabled(Ext_Zknh) & xlen == 64
404
405 $[wavedrom " _ _ _ _ _ "]
406 mapping clause encdec = SHA512SUM1 (rs1, rd)
407 <-> 0b00 @ 0b01000 @ 0b00101 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
408 when currentlyEnabled(Ext_Zknh) & xlen == 64
409
410 $[wavedrom " _ _ _ _ _ "]
411 mapping clause encdec = SHA512SIG0 (rs1, rd)
412 <-> 0b00 @ 0b01000 @ 0b00110 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
413 when currentlyEnabled(Ext_Zknh) & xlen == 64
414
415 $[wavedrom " _ _ _ _ _ "]
416 mapping clause encdec = SHA512SIG1 (rs1, rd)
417 <-> 0b00 @ 0b01000 @ 0b00111 @ encdec_reg(rs1) @ 0b001 @ en-
cdec_reg(rd) @ 0b0010011
418 when currentlyEnabled(Ext_Zknh) & xlen == 64
419
420 mapping clause assembly = SHA512SIG0 (rs1, rd)
421 <-> "sha512sig0" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
422
423 mapping clause assembly = SHA512SIG1 (rs1, rd)
424 <-> "sha512sig1" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
425
426 mapping clause assembly = SHA512SUM0 (rs1, rd)
427 <-> "sha512sum0" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
428
429 mapping clause assembly = SHA512SUM1 (rs1, rd)
430 <-> "sha512sum1" ^ spc() ^ reg_name(rd) ^ sep() ^
reg_name(rs1)
431
432 // Execute clauses for the 64-bit SHA512 instructions.
433
434 function clause execute SHA512SIG0(rs1, rd) = {
435   assert(xlen == 64);
436   let input : bits(64) = X(rs1);
437   let result : bits(64) = (input >>> 1) ^ (input >>> 8) ^
(input >>> 7);
438   X(rd) = result;
439   RETIRE_SUCCESS
440 }
441
442 function clause execute SHA512SIG1(rs1, rd) = {
443   assert(xlen == 64);
444   let input : bits(64) = X(rs1);
445   let result : bits(64) = (input >>> 19) ^ (input >>> 61) ^
(input >>> 6);
446   X(rd) = result;
447   RETIRE_SUCCESS
448 }
449
450 function clause execute SHA512SUM0(rs1, rd) = {
451   assert(xlen == 64);
452   let input : bits(64) = X(rs1);
453   let result : bits(64) = (input >>> 28) ^ (input >>> 34) ^
(input >>> 39);
454   X(rd) = result;
455   RETIRE_SUCCESS
456 }
457
458 function clause execute SHA512SUM1(rs1, rd) = {
459   assert(xlen == 64);
460   let input : bits(64) = X(rs1);
461   let result : bits(64) = (input >>> 14) ^ (input >>> 18) ^
(input >>> 41);

```

```

462   X(rd) = result;
463   RETIRE_SUCCESS
464 }

```

11. riscv_sail-riscv/model/extensions/K/zkr_control.sail (56 lines)

```

1 //
=====
2 // This Sail RISC-V architecture model, comprising all files
and
3 // directories except where otherwise noted is subject the BSD
4 // two-clause license in the LICENSE file.
5 //
6 // SPDX-License-Identifier: BSD-2-Clause
7 //
=====
8
9 // Zkr entropy seed source
10 function clause currentlyEnabled(Ext_Zkr) = hartSup-
ports(Ext_Zkr)
11
12 // Valid return states for reading the seed CSR.
13 private enum seed_opst = {
14   BIST, // Built-in-self-test. No randomness sampled.
15   ES16, // Entropy-sample-16. Valid 16-bits of randomness
sampled.
16   WAIT, // Device still gathering entropy.
17   DEAD // Fatal device compromise. No randomness sampled.
18 }
19
20 // Mapping of status codes and their actual encodings.
21 private mapping opst_code : seed_opst <-> bits(2) = {
22   BIST <-> 0b00,
23   WAIT <-> 0b01,
24   ES16 <-> 0b10,
25   DEAD <-> 0b11
26 }
27
28 // Entropy source spec requires an Illegal opcode exception be
raised if the
29 // seed register is read without also being written. This
function is only
30 // called once we know the CSR is being written, and all other
access control
31 // checks have been done.
32 private function read_seed_csr() -> xlenbits = {
33   let reserved_bits : bits(6) = 0b000000;
34   let custom_bits : bits(8) = 0x00;
35   let seed : bits(16) = get_16_random_bits();
36   zero_extend(opst_code(ES16) @ reserved_bits @ custom_bits @
seed)
37 }
38
39 // Writes to the seed CSR are ignored
40 private function write_seed_csr () -> xlenbits = zeros()
41
42 // CSR mapping
43 mapping clause csr_name_map = 0x015 <-> "seed"
44 function clause is_CSR_accessible(0x015, priv, access_type) =
45   currentlyEnabled(Ext_Zkr) &
46   access_type != CSRRead &
47   (match priv {
48     Machine => true,
49     User => mseccfg[USEED] == 0b1,
50     Supervisor => mseccfg[SSEED] == 0b1,
51     VirtualSupervisor => false,
52     VirtualUser => false,
53   })
54
55 function clause read_CSR(0x015) = read_seed_csr()
56 function clause write_CSR(0x015, _value) =
Ok(write_seed_csr())

```